All You Need

Everything You Need to Become an Artificial Intelligence Researcher

Abdulla Ali Khamis Alshamsi

Contents

Introduction of the Volume of Books **1**
0.1 Why am I writing this volume of books . . . 1
0.2 We must be at the forefront . . . 1
0.3 What will this volume of books contain . . . 1
0.4 Artificial Intelligence Departments . . . 16

1 Tensor Mathematics and Neural Networks Guide **18**
1.1 Introduction . . . 18
1.1.1 What will the book teach you . . . 18
1.1.2 The Chapters (T00 - T08) included in this book . . . 18
1.2 T00 - Tensor Algebra Laws . . . 19
1.2.1 Exercise 00 - Tensor Addition Law . . . 19
1.2.2 Exercise 01 - Tensor subtraction Law . . . 20
1.2.3 Exercise 02 - Tensor Hadamard Multiplication Law . . . 22
1.2.4 Exercise 03 - Tensor Division Law . . . 23
1.2.5 Exercise 04 - Tensor Power Law . . . 24
1.2.6 Exercise 05 - Tensor Square Root Law . . . 25
1.2.7 Exercise 06 - Tensor Absolute Value Law . . . 27
1.2.8 Exercise 07 - Tensor Negation Law . . . 28
1.2.9 Exercise 08 - Tensor Sign Law . . . 31
1.2.10 Exercise 09 - Tensor Clamp Law . . . 33
1.2.11 Exercise 10 - Tensor Round Law . . . 35
1.2.12 Exercise 11 - Tensor Floor Law . . . 36
1.2.13 Exercise 12 - Tensor Ceil Law . . . 37
1.2.14 Exercise 13 - Tensor Reciprocal Law . . . 39
1.2.15 Exercise 14 - Tensor Modulo Law . . . 40
1.2.16 Exercise 15 - Tensor Logical AND Law . . . 41
1.2.17 Exercise 16 - Tensor Logical OR Law . . . 43
1.2.18 Exercise 17 - Tensor Logical NOT Law . . . 44
1.2.19 Exercise 18 - Tensor Equality Law (Comparison) . . . 46
1.2.20 Exercise 19 - Tensor Not Equal Law (Comparison) . . . 48
1.2.21 Exercise 20 - Tensor Greater Than Law (Comparison) . . . 49
1.2.22 Exercise 21 - Tensor Less Than Law (Comparison) . . . 50
1.2.23 Exercise 22 - Tensor Greater or Equal Law (Comparison) . . . 51
1.2.24 Exercise 23 - Tensor Less or Equal Law (Comparison) . . . 53
1.3 T01 - Vector Laws (1D) . . . 54
1.3.1 Exercise 00 - 1D Dot Product Law . . . 54
1.3.2 Exercise 01 - 3D Cross Product Law . . . 56
1.3.3 Exercise 02 - 1D Norm Law . . . 58

1.3.4 Exercise 03 - 1D Normalize Law . . . 59
1.3.5 Exercise 04 - 1D Projection Law . . . 60
1.3.6 Exercise 05 - 1D Angle Law . . . 62
1.3.7 Exercise 06 - 1D Distance Law . . . 63
1.3.8 Exercise 07 - 1D Sum Reduction Law . . . 64
1.3.9 Exercise 08 - 1D Max Reduction Law . . . 66
1.3.10 Exercise 09 - 1D Min Reduction Law . . . 67
1.3.11 Exercise 10 - 1D Mean Reduction Law . . . 68
1.3.12 Exercise 11 - 1D Variance Law . . . 70
1.3.13 Exercise 12 - 1D Standard Deviation Law . . . 71
1.3.14 Exercise 13 - 1D Cumulative Sum Law . . . 72
1.3.15 Exercise 14 - 1D Cumulative Product Law . . . 74
1.4 T02 - Matrix Laws (2D) . . . 75
1.4.1 Exercise 00 - 2D Matrix Multiplication Law . . . 75
1.4.2 Exercise 01 - 2D Hadamard Matrix Law . . . 76
1.4.3 Exercise 02 - 2D Frobenius Inner Product Law . . . 78
1.4.4 Exercise 03 - 2D Determinant Law . . . 79
1.4.5 Exercise 04 - 2D Inverse Matrix Law . . . 81
1.4.6 Exercise 05 - 2D Transpose Law . . . 82
1.4.7 Exercise 06 - 2D Trace Law . . . 84
1.4.8 Exercise 07 - 2D Cofactor Law . . . 85
1.4.9 Exercise 08 - 2D Adjugate Law . . . 87
1.4.10 Exercise 09 - 2D Rank Law . . . 89
1.4.11 Exercise 10 - 2D Row Sum Law . . . 91
1.4.12 Exercise 11 - 2D Column Sum Law . . . 92
1.4.13 Exercise 12 - 2D Row Mean Law . . . 93
1.4.14 Exercise 13 - 2D Column Mean Law . . . 94
1.4.15 Exercise 14 - 2D Broadcasting Law . . . 95
1.5 T03 - Higher-Dimension Tensor Laws . . . 96
1.5.1 Exercise 00 - 3D Tensor Multiplication Law . . . 97
1.5.2 Exercise 01 - 3D Outer Product Law . . . 97
1.5.3 Exercise 02 - 3D Einstein Summation Law . . . 98
1.5.4 Exercise 03 - Batch Matrix Multiplication Law . . . 99
1.5.5 Exercise 04 - Tensor Contraction Law . . . 100
1.5.6 Exercise 05 - Tensor Repetition Law . . . 101
1.5.7 Exercise 06 - Tensor Tiling Law . . . 102
1.5.8 Exercise 07 - Tensor Broadcasting Law . . . 103
1.5.9 Exercise 08 - Tensor Indexing Law . . . 103
1.5.10 Exercise 09 - Tensor Advanced Slicing Law . . . 104
1.5.11 Exercise 10 - Tensor Padding Law . . . 105
1.5.12 Exercise 11 - Tensor Cropping Law . . . 106
1.5.13 Exercise 12 - Tensor Stacking Law . . . 107
1.5.14 Exercise 13 - Tensor Splitting Law . . . 108
1.5.15 Exercise 14 - Tensor Chunking Law . . . 108
1.6 T04 - Tensor shape Tranformation Laws . . . 109
1.6.1 Exercise 00 - Tensor Reshape Law . . . 109
1.6.2 Exercise 01 - Tensor View Law . . . 110
1.6.3 Exercise 02 - Tensor Permute Law . . . 111

1.6.4 Exercise 03 - Tensor Transpose Law . . . 112
1.6.5 Exercise 04 - Tensor Expand Law . . . 112
1.6.6 Exercise 05 - Tensor Repeat Law . . . 113
1.6.7 Exercise 06 - Tensor Flip Law . . . 114
1.6.8 Exercise 07 - Tensor Rotate Law . . . 115
1.6.9 Exercise 08 - Tensor Squeeze Law . . . 116
1.6.10 Exercise 09 - Tensor Unsqueeze Law . . . 116
1.6.11 Exercise 10 - Tensor Flatten Law . . . 117
1.6.12 Exercise 11 - Tensor Unflatten Law . . . 118
1.7 T05 - Differential Calculus Laws . . . 119
1.7.1 Exercise 00 - Gradient Law . . . 119
1.7.2 Exercise 01 - Jacobian Matrix Law . . . 119
1.7.3 Exercise 02 - Hessian Matrix Law . . . 120
1.7.4 Exercise 03 - Divergence Law . . . 121
1.7.5 Exercise 04 - Curl Law . . . 122
1.7.6 Exercise 05 - Laplacian Law . . . 122
1.7.7 Exercise 06 - Partial Derivative Law . . . 123
1.7.8 Exercise 07 - Chain Rule Law . . . 124
1.7.9 Exercise 08 - Product Rule Law . . . 124
1.7.10 Exercise 09 - Quotient Rule Law . . . 125
1.7.11 Exercise 10 - Power Rule Law . . . 126
1.7.12 Exercise 11 - Exponential Derivative Law . . . 127
1.7.13 Exercise 12 - Logarithmic Derivative Law . . . 128
1.7.14 Exercise 13 - Sigmoid Derivative Law . . . 128
1.7.15 Exercise 14 - Tanh Derivative Law . . . 129
1.7.16 Exercise 15 - ReLU Derivative Law . . . 130
1.7.17 Exercise 16 - LeakyReLU Derivative Law . . . 131
1.7.18 Exercise 17 - Softmax Derivative Law . . . 131
1.7.19 Exercise 18 - Loss Gradient Law . . . 132
1.8 T06 - Optimization Laws . . . 133
1.8.1 Exercise 00 - Gradient Descent Law . . . 133
1.8.2 Exercise 01 - Stochastic Gradient Descent Law . . . 134
1.8.3 Exercise 02 - Momentum Law . . . 135
1.8.4 Exercise 03 - RMSProp Law . . . 136
1.8.5 Exercise 04 - Adam Optimizer Law . . . 136
1.8.6 Exercise 05 - AdamW Law . . . 137
1.8.7 Exercise 06 - Nesterov Momentum Law . . . 138
1.8.8 Exercise 07 - Learning Rate Decay Law . . . 139
1.8.9 Exercise 08 - Weight Decay Law . . . 140
1.9 T07 - Statistics Probability Laws . . . 141
1.9.1 Exercise 00 - Mean Law . . . 141
1.9.2 Exercise 01 - Median Law . . . 141
1.9.3 Exercise 02 - Mode Law . . . 142
1.9.4 Exercise 03 - Variance Law . . . 143
1.9.5 Exercise 04 - Standard Deviation Law . . . 144
1.9.6 Exercise 05 - Covariance Law . . . 144
1.9.7 Exercise 06 - Correlation Law . . . 145
1.9.8 Exercise 07 - Probability Density Function Law . . . 146

1.9.9 Exercise 08 - Normal Distribution Law . . . 147
1.9.10 Exercise 09 - Uniform Distribution Law . . . 147
1.9.11 Exercise 10 - Bernoulli Law . . . 148
1.9.12 Exercise 11 - Softmax Probability Law . . . 149
1.9.13 Exercise 12 - Entropy Law . . . 150
1.9.14 Exercise 13 - KL Divergence Law . . . 150
1.9.15 Exercise 14 - Jensen Inequality Law . . . 151
1.10 T08 - Neural Network Transformatin Laws . . . 152
1.10.1 Exercise 00 - Linear Transformation Law . . . 152
1.10.2 Exercise 01 - Convolution Law . . . 153
1.10.3 Exercise 02 - Transposed Convolution Law . . . 153
1.10.4 Exercise 03 - Depthwise Convolution Law . . . 154
1.10.5 Exercise 04 - Softmax Activation Law . . . 155
1.10.6 Exercise 05 - Sigmoid Activation Law . . . 156
1.10.7 Exercise 06 - ReLU Activation Law . . . 156
1.10.8 Exercise 07 - GELU Activation Law . . . 157
1.10.9 Exercise 08 - Swish Activation Law . . . 158
1.10.10 Exercise 09 - Cross Entropy Loss Law . . . 159
1.10.11 Exercise 10 - MSE Loss Law . . . 160
1.10.12 Exercise 11 - L1 Loss Law . . . 160
1.10.13 Exercise 12 - Binary Cross Entropy Loss Law . . . 161
1.10.14 Exercise 13 - LogSoftmax Law . . . 162
1.10.15 Exercise 14 - LogSoftmax Gradient Law . . . 163

2 Applied Linear Algebra in Code 164
2.1 Introduction . . . 164
2.1.1 What will the book teach you . . . 164
2.1.2 The Chapters (ALA00 - ALA06) included in this book . . . 164
2.2 ALA00 - Tensor Operations in Code . . . 165
2.2.1 Exercise 00 - Basic Tensor Creation . . . 165
2.2.2 Exercise 01 - Tensor Shape Inspection . . . 166
2.2.3 Exercise 02 - Tensor Type Conversion . . . 167
2.2.4 Exercise 03 - Tensor Device Assignment . . . 167
2.2.5 Exercise 04 - Element-wise Addition . . . 168
2.2.6 Exercise 05 - Element-wise Multiplication . . . 169
2.2.7 Exercise 06 - Scalar-Tensor Operations . . . 170
2.2.8 Exercise 07 - Tensor Comparison Operations . . . 171
2.2.9 Exercise 08 - Boolean Masking Challenge . . . 172
2.2.10 Exercise 09 - Tensor Operation Chain Challenge . . . 172
2.2.11 Exercise 10 - Debugging Tensor Type Errors . . . 173
2.2.12 Exercise 11 - Tensor Operation Optimization Challenge . . . 174
2.3 ALA01 - Vector Dot & Norm Computation . . . 175
2.3.1 Exercise 00 - Manual Dot Product Implementation . . . 175
2.3.2 Exercise 01 - Dot Product Using PyTorch . . . 176
2.3.3 Exercise 02 - L2 Norm Calculation . . . 177
2.3.4 Exercise 03 - L1 Norm Calculation . . . 178
2.3.5 Exercise 04 - Vector Normalization . . . 179
2.3.6 Exercise 05 - Cosine Similarity Implementation . . . 179

2.3.7 Exercise 06 - Angle Between Two Vectors . . . 180
2.3.8 Exercise 07 - Batched Dot Product . . . 181
2.3.9 Exercise 08 - Stability Issues in Norm Computation . . . 182
2.3.10 Exercise 09 - Gradient-safe Dot Product Challenge . . . 183
2.3.11 Exercise 10 - Custom Similarity Metric Design . . . 184
2.4 ALA02 - Matrix Multiplication (matmul / mm) . . . 184
2.4.1 Exercise 00 - Basic Matrix Multiplication . . . 185
2.4.2 Exercise 01 - matmul vs mm Comparison . . . 185
2.4.3 Exercise 02 - Manual Matrix Multiplication . . . 186
2.4.4 Exercise 03 - Shape Compatibility Check . . . 187
2.4.5 Exercise 04 - Batched Matrix Multiplication . . . 188
2.4.6 Exercise 05 - High-Dimensional matmul Challenge . . . 189
2.4.7 Exercise 06 - Performance Comparison Challenge . . . 190
2.4.8 Exercise 07 - Matrix Multiplication Debugging . . . 191
2.4.9 Exercise 08 - Numerical Precision Challenge . . . 191
2.4.10 Exercise 09 - Custom Linear Layer Implementation . . . 192
2.5 ALA03 - Outer vs Inner Product . . . 193
2.5.1 Exercise 00 - Inner Product Coding Challenge . . . 193
2.5.2 Exercise 01 - Outer Product Coding Challenge . . . 194
2.5.3 Exercise 02 - Shape Interpretation Challenge . . . 195
2.5.4 Exercise 03 - Rank Analysis Challenge . . . 196
2.5.5 Exercise 04 - Feature Interaction Matrix Design . . . 197
2.5.6 Exercise 05 - Memory Cost Comparison Challenge . . . 198
2.5.7 Exercise 06 - Use-Case Driven Product Selection . . . 198
2.6 ALA04 - Broadcasting Rules & Mechanics . . . 199
2.6.1 Exercise 00 - Simple Broadcasting Addition . . . 200
2.6.2 Exercise 01 - Broadcasting Multiplication . . . 201
2.6.3 Exercise 02 - Broadcasting with Scalars . . . 202
2.6.4 Exercise 03 - Broadcasting Across Dimensions . . . 203
2.6.5 Exercise 04 - Invalid Broadcasting Detection . . . 204
2.6.6 Exercise 05 - Broadcasting Debugging Challenge . . . 205
2.6.7 Exercise 06 - Memory-efficient Broadcasting . . . 206
2.6.8 Exercise 07 - Broadcasting in Neural Layers . . . 207
2.7 ALA05 - Shape Reasoning & Dimension Flow . . . 208
2.7.1 Exercise 00 - Shape Tracking Basics . . . 208
2.7.2 Exercise 01 - Multi-step Shape Prediction . . . 209
2.7.3 Exercise 02 - Shape Mismatch Debugging . . . 210
2.7.4 Exercise 03 - Dynamic Shape Handling . . . 211
2.7.5 Exercise 04 - Shape-safe Function Design . . . 212
2.7.6 Exercise 05 - Model Shape Pipeline Validation . . . 213
2.8 ALA06 - Basis, Span & Projection (Applied) . . . 214
2.8.1 Exercise 00 - Vector Projection Implementation . . . 214
2.8.2 Exercise 01 - Orthogonal Projection . . . 215
2.8.3 Exercise 02 - Projection Error Measurement . . . 216
2.8.4 Exercise 03 - Subspace Projection Challenge . . . 217
2.8.5 Exercise 04 - Feature Space Reduction . . . 218
2.8.6 Exercise 05 - Projection-based Optimization Insight . . . 219

3 Autograd & Backprop Mechanics 221
3.1 Introduction . . . 221
3.1.1 What will the book teach you . . . 221
3.1.2 The Chapters (ABM00 - ABM06) included in this book . . . 221
3.2 ABM00 - Computational Graph Construction . . . 222
3.2.1 Exercise 00 - Tensor with requires_grad Setup . . . 222
3.2.2 Exercise 01 - Simple Operation Graph Creation . . . 223
3.2.3 Exercise 02 - Multi-Operation Graph Building . . . 223
3.2.4 Exercise 03 - Leaf vs Non-Leaf Tensor Detection . . . 224
3.2.5 Exercise 04 - Graph Retention After Backward . . . 225
3.2.6 Exercise 05 - Detaching Tensor from Graph . . . 226
3.2.7 Exercise 06 - Stop-Gradient Simulation Challenge . . . 227
3.2.8 Exercise 07 - Graph Reuse Error Debugging . . . 228
3.2.9 Exercise 08 - In-place Operation Graph Failure . . . 229
3.2.10 Exercise 09 - Manual Graph Reasoning Challenge . . . 230
3.3 ABM01 - Forward vs Backward Pass . . . 231
3.3.1 Exercise 00 - Forward Pass Value Tracking . . . 231
3.3.2 Exercise 01 - Backward Call on Scalar Loss . . . 232
3.3.3 Exercise 02 - Gradient Accumulation Observation . . . 233
3.3.4 Exercise 03 - Zeroing Gradients Correctly . . . 234
3.3.5 Exercise 04 - Multiple Backward Calls Challenge . . . 235
3.3.6 Exercise 05 - retain_graph Usage Challenge . . . 236
3.3.7 Exercise 06 - Backward Through Branching Graph . . . 237
3.3.8 Exercise 07 - Gradient Flow Order Reasoning . . . 238
3.3.9 Exercise 08 - Backward Error Debugging . . . 239
3.3.10 Exercise 09 - Custom Forward–Backward Trace . . . 240
3.4 ABM02 - Manual Gradient Calculation . . . 241
3.4.1 Exercise 00 - Derivative of Linear Function . . . 241
3.4.2 Exercise 01 - Manual Gradient vs Autograd Check . . . 242
3.4.3 Exercise 02 - Gradient of Quadratic Function . . . 243
3.4.4 Exercise 03 - Chain Rule Manual Verification . . . 244
3.4.5 Exercise 04 - Vector Gradient Computation . . . 245
3.4.6 Exercise 05 - Matrix Gradient Reasoning . . . 245
3.4.7 Exercise 06 - Broadcasting Gradient Challenge . . . 246
3.4.8 Exercise 07 - Gradient Shape Validation . . . 247
3.4.9 Exercise 08 - Numerical Gradient Approximation . . . 248
3.4.10 Exercise 09 - Gradient Mismatch Debugging . . . 249
3.5 ABM03 - Chain Rule in Code . . . 250
3.5.1 Exercise 00 - Single-Chain Gradient Flow . . . 250
3.5.2 Exercise 01 - Multi-Operation Chain Rule . . . 251
3.5.3 Exercise 02 - Nested Function Gradient Flow . . . 252
3.5.4 Exercise 03 - Chain Rule with Activation . . . 253
3.5.5 Exercise 04 - Branching Chain Rule Challenge . . . 254
3.5.6 Exercise 05 - Gradient Path Contribution Analysis . . . 255
3.5.7 Exercise 06 - Partial Gradient Blocking . . . 256
3.5.8 Exercise 07 - Detach Effect on Chain Rule . . . 257
3.5.9 Exercise 08 - Chain Rule Failure Debugging . . . 257
3.5.10 Exercise 09 - Full Graph Gradient Trace . . . 258

3.6 ABM04 - Autograd Engine in PyTorch . . . 259
3.6.1 Exercise 00 - Autograd Basic Usage . . . 260
3.6.2 Exercise 01 - grad_fn Inspection . . . 260
3.6.3 Exercise 02 - Gradient Accumulation Behavior . . . 261
3.6.4 Exercise 03 - No-Grad Context Manager . . . 262
3.6.5 Exercise 04 - Inference vs Training Mode Gradient . . . 263
3.6.6 Exercise 05 - Custom Autograd Function (Forward) . . . 264
3.6.7 Exercise 06 - Custom Autograd Function (Backward) . . . 265
3.6.8 Exercise 07 - Autograd Graph Memory Analysis . . . 266
3.6.9 Exercise 08 - Gradient Hook Implementation . . . 267
3.6.10 Exercise 09 - Autograd Failure Debugging . . . 268
3.7 ABM05 - Backprop Through Linear Layers . . . 269
3.7.1 Exercise 00 - Single Linear Layer Backprop . . . 269
3.7.2 Exercise 01 - Weight Gradient Verification . . . 270
3.7.3 Exercise 02 - Bias Gradient Verification . . . 271
3.7.4 Exercise 03 - Batch Linear Layer Backprop . . . 272
3.7.5 Exercise 04 - Manual vs Autograd Linear Backprop . . . 272
3.7.6 Exercise 05 - Gradient Scaling with Batch Size . . . 273
3.7.7 Exercise 06 - Frozen Layer Gradient Check . . . 274
3.7.8 Exercise 07 - Linear Stack Gradient Flow . . . 275
3.7.9 Exercise 08 - Numerical Stability in Backprop . . . 276
3.7.10 Exercise 09 - End-to-End Linear Backprop Challenge . . . 277
3.8 ABM06 - Gradient Flow Debugging . . . 278
3.8.1 Exercise 00 - Detecting Vanishing Gradients . . . 278
3.8.2 Exercise 01 - Detecting Exploding Gradients . . . 279
3.8.3 Exercise 02 - Gradient Norm Monitoring . . . 280
3.8.4 Exercise 03 - Layer-wise Gradient Inspection . . . 281
3.8.5 Exercise 04 - Broken Graph Debugging . . . 282
3.8.6 Exercise 05 - requires_grad Misuse Debugging . . . 283
3.8.7 Exercise 06 - Gradient Blocking Detection . . . 284
3.8.8 Exercise 07 - Hook-based Gradient Debugging . . . 285
3.8.9 Exercise 08 - Training Collapse Diagnosis . . . 286
3.8.10 Exercise 09 - Full Backprop Debugging Scenario . . . 287

4 Optimization in Code 289
4.1 Introduction . . . 289
4.1.1 What will the book teach you . . . 289
4.1.2 How to approach the exercises . . . 290
4.1.3 The Chapters (OC00 - OC06) included in this book . . . 290
4.2 OC00 - Gradient Descent Implementation . . . 290
4.2.1 Exercise 00 - Simple Gradient Descent Step . . . 291
4.2.2 Exercise 01 - Gradient Descent on Scalar Loss . . . 291
4.2.3 Exercise 02 - Gradient Descent on Vector Parameters . . . 292
4.2.4 Exercise 03 - Learning Rate Effect Observation . . . 293
4.2.5 Exercise 04 - Convergence vs Divergence Test . . . 294
4.2.6 Exercise 05 - Manual Parameter Update Loop . . . 295
4.2.7 Exercise 06 - Gradient Descent without Autograd . . . 296
4.2.8 Exercise 07 - Loss Curve Tracking Challenge . . . 297

4.2.9 Exercise 08 - Gradient Clipping Integration . . . 298
4.2.10 Exercise 09 - End-to-End Gradient Descent Challenge . . . 299
4.3 OC01 - Learning Rate Effects & Tuning . . . 300
4.3.1 Exercise 00 - Fixed Learning Rate Experiment . . . 300
4.3.2 Exercise 01 - Too-Small Learning Rate Scenario . . . 301
4.3.3 Exercise 02 - Too-Large Learning Rate Scenario . . . 302
4.3.4 Exercise 03 - Learning Rate Sweep Experiment . . . 302
4.3.5 Exercise 04 - Loss Oscillation Detection . . . 303
4.3.6 Exercise 05 - Adaptive Learning Rate Logic . . . 304
4.3.7 Exercise 06 - Learning Rate Scheduling Basics . . . 305
4.3.8 Exercise 07 - Warm-up Learning Rate Strategy . . . 306
4.3.9 Exercise 08 - Learning Rate Debugging Challenge . . . 307
4.3.10 Exercise 09 - Optimal Learning Rate Search . . . 308
4.4 OC02 - Momentum Update Rule . . . 309
4.4.1 Exercise 00 - Momentum Concept Implementation . . . 309
4.4.2 Exercise 01 - Velocity Term Tracking . . . 310
4.4.3 Exercise 02 - Momentum vs Vanilla GD . . . 311
4.4.4 Exercise 03 - Momentum Acceleration Effect . . . 311
4.4.5 Exercise 04 - Damped Oscillation Observation . . . 312
4.4.6 Exercise 05 - Momentum Coefficient Tuning . . . 313
4.4.7 Exercise 06 - Momentum on Noisy Gradients . . . 314
4.4.8 Exercise 07 - Custom Momentum Optimizer . . . 315
4.4.9 Exercise 08 - Momentum Failure Debugging . . . 316
4.4.10 Exercise 09 - Momentum Convergence Challenge . . . 317
4.5 OC03 - RMSProp Logic & Behavior . . . 318
4.5.1 Exercise 00 - RMSProp Accumulator Initialization . . . 318
4.5.2 Exercise 01 - Running Average of Squared Gradients . . . 319
4.5.3 Exercise 02 - RMSProp Update Step Implementation . . . 320
4.5.4 Exercise 03 - Effect of Decay Rate . . . 320
4.5.5 Exercise 04 - RMSProp vs Momentum Comparison . . . 321
4.5.6 Exercise 05 - Stability under High Gradient Variance . . . 322
4.5.7 Exercise 06 - RMSProp without Autograd . . . 323
4.5.8 Exercise 07 - Custom RMSProp Optimizer . . . 324
4.5.9 Exercise 08 - RMSProp Debugging Challenge . . . 325
4.5.10 Exercise 09 - RMSProp Convergence Evaluation . . . 326
4.6 OC04 - Adam Optimizer Math + Code . . . 327
4.6.1 Exercise 00 - Adam First Moment Initialization . . . 327
4.6.2 Exercise 01 - Adam Second Moment Initialization . . . 328
4.6.3 Exercise 02 - Bias Correction Implementation . . . 328
4.6.4 Exercise 03 - Adam Update Rule from Scratch . . . 329
4.6.5 Exercise 04 - Adam vs RMSProp Comparison . . . 330
4.6.6 Exercise 05 - Adam Hyperparameter Sensitivity . . . 331
4.6.7 Exercise 06 - Adam on Sparse Gradients . . . 332
4.6.8 Exercise 07 - Custom Adam Optimizer Class . . . 332
4.6.9 Exercise 08 - Adam Failure Case Debugging . . . 334
4.6.10 Exercise 09 - Adam End-to-End Training Challenge . . . 334
4.7 OC05 - Convergence & Stability . . . 335
4.7.1 Exercise 00 - Convergence Criteria Detection . . . 335

4.7.2 Exercise 01 - Loss Plateau Detection . . . 336
4.7.3 Exercise 02 - Divergence Pattern Identification . . . 337
4.7.4 Exercise 03 - Early Stopping Logic . . . 338
4.7.5 Exercise 04 - Gradient Explosion Mitigation . . . 339
4.7.6 Exercise 05 - Gradient Vanishing Mitigation . . . 340
4.7.7 Exercise 06 - Stability under Noisy Data . . . 341
4.7.8 Exercise 07 - Optimizer Comparison Experiment . . . 341
4.7.9 Exercise 08 - Stability Debugging Scenario . . . 342
4.7.10 Exercise 09 - Long-Run Training Stability Test . . . 343
4.8 OC06 - Weight Initialization Impact . . . 344
4.8.1 Exercise 00 - Zero Initialization Experiment . . . 344
4.8.2 Exercise 01 - Random Uniform Initialization . . . 345
4.8.3 Exercise 02 - Random Normal Initialization . . . 346
4.8.4 Exercise 03 - Xavier Initialization Implementation . . . 347
4.8.5 Exercise 04 - He Initialization Implementation . . . 348
4.8.6 Exercise 05 - Initialization Effect on Gradient Flow . . . 349
4.8.7 Exercise 06 - Initialization vs Convergence Speed . . . 349
4.8.8 Exercise 07 - Bad Initialization Failure Case . . . 350
4.8.9 Exercise 08 - Initialization Debugging Challenge . . . 351
4.8.10 Exercise 09 - End-to-End Initialization Comparison . . . 352

5 Loss Functions & Training Loop 354
5.1 Introduction . . . 354
5.1.1 What will the book teach you . . . 354
5.1.2 How to approach the exercises . . . 355
5.1.3 The Chapters (LFTL00 - LFTL06) included in this book . . . 356
5.2 LFTL00 - What is Loss and Why . . . 356
5.2.1 Exercise 00 - Define Loss for Simple Regression . . . 356
5.2.2 Exercise 01 - Compare Prediction vs Target . . . 357
5.2.3 Exercise 02 - Scalar Loss Computation . . . 357
5.2.4 Exercise 03 - Vector Loss Aggregation . . . 358
5.2.5 Exercise 04 - Mean Loss vs Sum Loss . . . 359
5.2.6 Exercise 05 - Loss Sensitivity Observation . . . 360
5.2.7 Exercise 06 - Loss Landscape Visualization . . . 361
5.2.8 Exercise 07 - Loss Scaling Effect . . . 362
5.2.9 Exercise 08 - Poor Loss Choice Failure Case . . . 362
5.2.10 Exercise 09 - Loss Design Reasoning Challenge . . . 363
5.3 LFTL01 - MSE Loss Implementation . . . 364
5.3.1 Exercise 00 - Manual MSE Formula Implementation . . . 364
5.3.2 Exercise 01 - MSE Using PyTorch . . . 365
5.3.3 Exercise 02 - Batch-wise MSE Computation . . . 366
5.3.4 Exercise 03 - MSE Gradient Inspection . . . 367
5.3.5 Exercise 04 - Effect of Outliers on MSE . . . 368
5.3.6 Exercise 05 - MSE Stability with Large Values . . . 368
5.3.7 Exercise 06 - Weighted MSE Loss Challenge . . . 369
5.3.8 Exercise 07 - MSE vs MAE Comparison . . . 370
5.3.9 Exercise 08 - MSE Debugging Challenge . . . 371
5.3.10 Exercise 09 - Custom MSE Loss Class . . . 372

5.4 LFTL02 - Cross-Entropy Logic . . . 373
5.4.1 Exercise 00 - Probability Output Validation . . . 373
5.4.2 Exercise 01 - One-Hot Target Encoding . . . 373
5.4.3 Exercise 02 - Manual Cross-Entropy Computation . . . 374
5.4.4 Exercise 03 - Cross-Entropy Using PyTorch . . . 375
5.4.5 Exercise 04 - LogSoftmax + NLL Breakdown . . . 376
5.4.6 Exercise 05 - Numerical Stability (Log-Sum-Exp) . . . 377
5.4.7 Exercise 06 - Batch Cross-Entropy Loss . . . 378
5.4.8 Exercise 07 - Wrong Target Shape Debugging . . . 379
5.4.9 Exercise 08 - Class Imbalance Effect . . . 379
5.4.10 Exercise 09 - Custom Cross-Entropy Loss . . . 380
5.5 LFTL03 - Building Training Loop Manually . . . 381
5.5.1 Exercise 00 - Single-Step Training Loop . . . 381
5.5.2 Exercise 01 - Forward + Loss + Backward Loop . . . 382
5.5.3 Exercise 02 - Parameter Update Integration . . . 383
5.5.4 Exercise 03 - Gradient Zeroing Placement . . . 384
5.5.5 Exercise 04 - Training Loop without Optimizer . . . 385
5.5.6 Exercise 05 - Loss Logging per Iteration . . . 386
5.5.7 Exercise 06 - Training Loop Bug Debugging . . . 387
5.5.8 Exercise 07 - Manual vs Optimizer Loop Comparison . . . 388
5.5.9 Exercise 08 - Gradient Explosion inside Loop . . . 388
5.5.10 Exercise 09 - End-to-End Manual Training Loop . . . 389
5.6 LFTL04 - Batching + Epochs . . . 390
5.6.1 Exercise 00 - Single-Batch Training . . . 390
5.6.2 Exercise 01 - Mini-Batch Training Setup . . . 391
5.6.3 Exercise 02 - Epoch Loop Construction . . . 392
5.6.4 Exercise 03 - Shuffling Data per Epoch . . . 393
5.6.5 Exercise 04 - Batch Size Effect Observation . . . 394
5.6.6 Exercise 05 - Gradient Accumulation across Batches . . . 395
5.6.7 Exercise 06 - Last Incomplete Batch Handling . . . 396
5.6.8 Exercise 07 - Performance vs Batch Size . . . 397
5.6.9 Exercise 08 - Batch-related Bug Debugging . . . 397
5.6.10 Exercise 09 - Full Epoch Training Challenge . . . 398
5.7 LFTL05 - Evaluation & Validation Split . . . 399
5.7.1 Exercise 00 - Train vs Eval Mode Difference . . . 399
5.7.2 Exercise 01 - Validation Dataset Separation . . . 400
5.7.3 Exercise 02 - Validation Loss Computation . . . 401
5.7.4 Exercise 03 - Overfitting Detection . . . 402
5.7.5 Exercise 04 - Early Stopping Trigger . . . 403
5.7.6 Exercise 05 - Metric vs Loss Comparison . . . 404
5.7.7 Exercise 06 - Evaluation without Gradient Tracking . . . 405
5.7.8 Exercise 07 - Validation Loop Bug Debugging . . . 405
5.7.9 Exercise 08 - Model Selection based on Validation . . . 406
5.7.10 Exercise 09 - Full Train–Validation Pipeline . . . 407
5.8 LFTL06 - Overfitting Prevention Methods . . . 408
5.8.1 Exercise 00 - Detecting Overfitting Pattern . . . 408
5.8.2 Exercise 01 - Regularization Concept Experiment . . . 409
5.8.3 Exercise 02 - Weight Decay Integration . . . 410

5.8.4 Exercise 03 - Dropout Effect Observation . . . 411
5.8.5 Exercise 04 - Data Augmentation Impact . . . 412
5.8.6 Exercise 05 - Early Stopping Implementation . . . 413
5.8.7 Exercise 06 - Bias–Variance Tradeoff Inspection . . . 414
5.8.8 Exercise 07 - Underfitting vs Overfitting Diagnosis . . . 414
5.8.9 Exercise 08 - Regularization Debugging Challenge . . . 415
5.8.10 Exercise 09 - Robust Training Pipeline Design . . . 416

6 SQL Lessons (PostgreSQL) 418
6.1 Introduction . . . 418
6.1.1 What will this book teach you . . . 418
6.1.2 How PostgreSQL supports artificial intelligence systems . . . 419
6.1.3 How to approach the exercises . . . 419
6.1.4 The Chapters (SQL00–SQL03) included in this book . . . 420
6.1.5 SQL00 - Relational Data Fundamentals . . . 420
6.1.6 SQL01 - Tables, Keys & Schemas . . . 420
6.1.7 SQL02 - Users, Plans & Limits Storage . . . 420
6.1.8 SQL03 - Query Logic & Transactions . . . 421
6.2 SQL00 - Relational Data Fundamentals . . . 421
6.2.1 Exercise 00 - Create Database + Connect Challenge . . . 421
6.2.2 Exercise 01 - Create Table with Correct Data Types . . . 422
6.2.3 Exercise 02 - Insert Rows + Verify with SELECT . . . 423
6.2.4 Exercise 03 - Basic WHERE Filtering Challenge . . . 424
6.2.5 Exercise 04 - ORDER BY + LIMIT + OFFSET Challenge . . . 425
6.2.6 Exercise 05 - UPDATE with Condition + Verification . . . 426
6.2.7 Exercise 06 - DELETE with Safety Condition . . . 427
6.2.8 Exercise 07 - NULL Handling + COALESCE Challenge . . . 428
6.2.9 Exercise 08 - Aggregate Basics (COUNT, SUM, AVG) . . . 429
6.2.10 Exercise 09 - Group By + Having Challenge . . . 430
6.3 SQL01 - Tables, Keys & Schemas . . . 432
6.3.1 Exercise 00 - Design Schema: users + plans + sessions . . . 432
6.3.2 Exercise 01 - Primary Key + Unique Constraint Challenge . . . 433
6.3.3 Exercise 02 - Foreign Key Relationship Setup . . . 434
6.3.4 Exercise 03 - ON DELETE Behavior (RESTRICT/CASCADE) . . . 435
6.3.5 Exercise 04 - CHECK Constraints for Data Validation . . . 436
6.3.6 Exercise 05 - Default Values + Timestamps . . . 437
6.3.7 Exercise 06 - Normalize: Split Table into 2NF/3NF . . . 438
6.3.8 Exercise 07 - Create Index + Explain Its Purpose . . . 439
6.3.9 Exercise 08 - Join Basics (INNER / LEFT) Challenge . . . 440
6.3.10 Exercise 09 - Query with Multi-Table Join + Filters . . . 441
6.4 SQL02 - Users, Plans & Limits Storage . . . 443
6.4.1 Exercise 00 - Implement Plans Table + Limits Columns . . . 443
6.4.2 Exercise 01 - Assign User to Plan + Enforce FK . . . 444
6.4.3 Exercise 02 - Track Usage Table (tokens/requests) . . . 445
6.4.4 Exercise 03 - Daily Usage Aggregation Query . . . 446
6.4.5 Exercise 04 - Check Limit Breach Query (Over Limit) . . . 447
6.4.6 Exercise 05 - Create View: user_current_limits . . . 448
6.4.7 Exercise 06 - Create View: user_usage_summary . . . 449

6.4.8 Exercise 07 - Plan Upgrade Transaction Challenge . . . 450
6.4.9 Exercise 08 - Audit Log Table for Plan Changes . . . 452
6.4.10 Exercise 09 - Report Query: Top Users by Usage . . . 453
6.5 SQL03 - Query Logic & Transactions . . . 454
6.5.1 Exercise 00 - Transaction Basics (BEGIN/COMMIT/ROLLBACK) . . . 454
6.5.2 Exercise 01 - Atomic Insert into Multiple Tables . . . 455
6.5.3 Exercise 02 - Consistency Check + Rollback on Failure . . . 456
6.5.4 Exercise 03 - Isolation Concept Simulation (Two Sessions) . . . 457
6.5.5 Exercise 04 - Locking Behavior Observation Challenge . . . 459
6.5.6 Exercise 05 - Upsert Pattern (INSERT...ON CONFLICT) . . . 460
6.5.7 Exercise 06 - Common Table Expression (CTE) Challenge . . . 461
6.5.8 Exercise 07 - Query Performance: EXPLAIN ANALYZE . . . 462
6.5.9 Exercise 08 - Index Impact Benchmark Challenge . . . 463
6.5.10 Exercise 09 - Transaction-Safe Usage Deduction Scenario . . . 464

7 NoSQL Lessons (MongoDB) 467
7.1 Introduction . . . 467
7.1.1 What will this book teach you . . . 467
7.1.2 How MongoDB supports artificial intelligence systems . . . 468
7.1.3 How to approach the exercises . . . 468
7.1.4 The Chapters (NOSQL00–NOSQL03) included in this book . . . 469
7.1.5 NOSQL00 - Document-Based Data Structures . . . 469
7.1.6 NOSQL01 - Chat History & Session Memory . . . 469
7.1.7 NOSQL02 - Dynamic Storage Models . . . 470
7.1.8 NOSQL03 - Scaling with Replica & Sharding . . . 470
7.2 NOSQL00 - Document-Based Data Structures . . . 470
7.2.1 Exercise 00 - Create Database and Collection . . . 470
7.2.2 Exercise 01 - Insert Single Document (insertOne) . . . 471
7.2.3 Exercise 02 - Insert Multiple Documents (insertMany) . . . 472
7.2.4 Exercise 03 - Basic Find Query with Filters . . . 473
7.2.5 Exercise 04 - Projection: Include / Exclude Fields . . . 474
7.2.6 Exercise 05 - Update Document with $set . . . 475
7.2.7 Exercise 06 - Update with $inc and $push . . . 476
7.2.8 Exercise 07 - Delete Document with Condition . . . 477
7.2.9 Exercise 08 - Nested Document Design Challenge . . . 478
7.2.10 Exercise 09 - Schema-less Design Pitfall Analysis . . . 479
7.3 NOSQL01 - Chat History & Session Memory . . . 480
7.3.1 Exercise 00 - Design Chat Message Document Structure . . . 480
7.3.2 Exercise 01 - Insert Chat Messages with Session ID . . . 481
7.3.3 Exercise 02 - Query Messages by Session and Time . . . 482
7.3.4 Exercise 03 - Sort Messages Chronologically . . . 483
7.3.5 Exercise 04 - Limit Chat History per Session . . . 484
7.3.6 Exercise 05 - Session Expiration using TTL Index . . . 485
7.3.7 Exercise 06 - Update Last-Active Timestamp . . . 487
7.3.8 Exercise 07 - Archive Old Sessions Strategy . . . 488
7.3.9 Exercise 08 - Aggregation: Messages Count per Session . . . 489
7.3.10 Exercise 09 - Session Memory Retrieval Pipeline . . . 490
7.4 NOSQL02 - Dynamic Storage Models . . . 491

7.4.1 Exercise 00 - Flexible Document Schema Design . . . 491
7.4.2 Exercise 01 - Optional Fields Handling . . . 492
7.4.3 Exercise 02 - Polymorphic Documents in One Collection . . . 493
7.4.4 Exercise 03 - Versioned Document Schema Strategy . . . 494
7.4.5 Exercise 04 - Partial Updates without Overwrite . . . 495
7.4.6 Exercise 05 - Embedded vs Referenced Documents . . . 496
7.4.7 Exercise 06 - Aggregation Pipeline with $group . . . 497
7.4.8 Exercise 07 - Data Migration between Schema Versions . . . 498
7.4.9 Exercise 08 - Indexing Strategy for Dynamic Fields . . . 500
7.4.10 Exercise 09 - Performance Tradeoff Evaluation . . . 501
7.5 NOSQL03 - Scaling with Replica & Sharding . . . 502
7.5.1 Exercise 00 - Create Index for High-Read Query . . . 502
7.5.2 Exercise 01 - Explain Query Execution Plan . . . 503
7.5.3 Exercise 02 - Replica Set Concept Simulation . . . 504
7.5.4 Exercise 03 - Read Preference Configuration . . . 505
7.5.5 Exercise 04 - Write Concern Configuration . . . 506
7.5.6 Exercise 05 - Shard Key Selection Challenge . . . 507
7.5.7 Exercise 06 - Chunk Distribution Observation . . . 508
7.5.8 Exercise 07 - Balancer Behavior Analysis . . . 509
7.5.9 Exercise 08 - Sharding vs Indexing Comparison . . . 511
7.5.10 Exercise 09 - Scalable Chat System Data Design . . . 512

8 Object Storage Lessons (MinIO) 514
8.1 Introduction . . . 514
8.1.1 What will this book teach you . . . 514
8.1.2 How object storage supports artificial intelligence systems . . . 515
8.1.3 How to approach the exercises . . . 515
8.1.4 The Chapters (OB00–OB03) included in this book . . . 516
8.1.5 OB00 - Binary Data & Large File Storage . . . 516
8.1.6 OB01 - Datasets, Weights & Checkpoints . . . 516
8.1.7 OB02 - Buckets & Access Structure . . . 517
8.1.8 OB03 - Distributed Mode & Fault Tolerance . . . 517
8.2 OB00 - Binary Data & Large File Storage . . . 517
8.2.1 Exercise 00 - Install MinIO and Start Server . . . 518
8.2.2 Exercise 01 - Create Bucket using CLI . . . 518
8.2.3 Exercise 02 - Upload Single File (Object Put) . . . 519
8.2.4 Exercise 03 - Download Object and Verify Integrity . . . 520
8.2.5 Exercise 04 - List Objects with Prefix Filtering . . . 521
8.2.6 Exercise 05 - Delete Object Safely . . . 522
8.2.7 Exercise 06 - Upload Large File using Multipart . . . 523
8.2.8 Exercise 07 - Metadata Attachment to Objects . . . 524
8.2.9 Exercise 08 - Versioning Enablement Challenge . . . 525
8.2.10 Exercise 09 - Binary Storage Performance Test . . . 526
8.3 OB01 - Datasets, Weights & Checkpoints . . . 528
8.3.1 Exercise 00 - Design Dataset Bucket Structure . . . 528
8.3.2 Exercise 01 - Upload Dataset with Folder Hierarchy . . . 529
8.3.3 Exercise 02 - Store Model Weights after Training . . . 530
8.3.4 Exercise 03 - Retrieve Checkpoint for Resume Training . . . 531

8.3.5 Exercise 04 - Versioned Model Checkpoints Strategy . . . 532
8.3.6 Exercise 05 - Atomic Model Save Operation . . . 533
8.3.7 Exercise 06 - Compare Local vs Object Storage IO . . . 534
8.3.8 Exercise 07 - Garbage Collection of Old Checkpoints . . . 535
8.3.9 Exercise 08 - Integrity Validation with Hashes . . . 536
8.3.10 Exercise 09 - Training Pipeline Object Integration . . . 537
8.4 OB02 - Buckets & Access Structure . . . 539
8.4.1 Exercise 00 - Bucket Policy Configuration . . . 539
8.4.2 Exercise 01 - Read-Only vs Read-Write Policy . . . 540
8.4.3 Exercise 02 - User and Access Key Creation . . . 541
8.4.4 Exercise 03 - Temporary Access using Presigned URL . . . 542
8.4.5 Exercise 04 - Private vs Public Object Access . . . 543
8.4.6 Exercise 05 - Folder-like Access Control Design . . . 544
8.4.7 Exercise 06 - Access Denial Debugging Challenge . . . 545
8.4.8 Exercise 07 - Multi-Bucket Permission Strategy . . . 546
8.4.9 Exercise 08 - Secure Credential Rotation . . . 547
8.4.10 Exercise 09 - Access Control Audit Scenario . . . 548
8.5 OB03 - Distributed Mode & Fault Tolerance . . . 550
8.5.1 Exercise 00 - Start MinIO in Distributed Mode . . . 550
8.5.2 Exercise 01 - Node Failure Simulation . . . 551
8.5.3 Exercise 02 - Data Replication Verification . . . 552
8.5.4 Exercise 03 - Healing Process Observation . . . 553
8.5.5 Exercise 04 - Disk Failure Recovery Scenario . . . 554
8.5.6 Exercise 05 - Erasure Coding Concept Experiment . . . 555
8.5.7 Exercise 06 - Performance Impact of Replication . . . 556
8.5.8 Exercise 07 - High Availability Configuration . . . 557
8.5.9 Exercise 08 - Disaster Recovery Backup Strategy . . . 558
8.5.10 Exercise 09 - End-to-End Fault Tolerance Test . . . 559

9 Vector Database Lessons (Qdrant) 562
9.1 Introduction . . . 562
9.1.1 What will this book teach you . . . 562
9.1.2 How Qdrant supports artificial intelligence systems . . . 563
9.1.3 How to approach the exercises . . . 564
9.1.4 The Chapters (QD00–QD03) included in this book . . . 564
9.1.5 QD00 - Embeddings & Semantic Meaning . . . 565
9.1.6 QD01 - Similarity Search Mechanics . . . 565
9.1.7 QD02 - RAG Integration Workflow . . . 565
9.1.8 QD03 - Indexing & Scalable Retrieval . . . 565
9.2 QD00 - Embeddings & Semantic Meaning . . . 566
9.2.1 Exercise 00 - Install Qdrant and Start Service . . . 566
9.2.2 Exercise 01 - Create Collection with Vector Size . . . 567
9.2.3 Exercise 02 - Generate Embeddings using Model . . . 568
9.2.4 Exercise 03 - Insert Single Vector Point . . . 569
9.2.5 Exercise 04 - Insert Batch of Vectors . . . 570
9.2.6 Exercise 05 - Attach Payload Metadata to Vectors . . . 571
9.2.7 Exercise 06 - Retrieve Vector by ID . . . 572
9.2.8 Exercise 07 - Vector Normalization Effect Test . . . 573

9.2.9 Exercise 08 - Semantic Meaning Drift Observation . . . 574
9.2.10 Exercise 09 - Embedding Quality Evaluation Challenge . . . 575
9.3 QD01 - Similarity Search Mechanics . . . 577
9.3.1 Exercise 00 - Cosine Similarity Search Query . . . 577
9.3.2 Exercise 01 - Euclidean Distance Search Query . . . 578
9.3.3 Exercise 02 - Compare Distance Metrics . . . 579
9.3.4 Exercise 03 - Top-K Nearest Neighbors Retrieval . . . 580
9.3.5 Exercise 04 - Similarity Threshold Filtering . . . 581
9.3.6 Exercise 05 - Payload-based Filtering in Search . . . 582
9.3.7 Exercise 06 - Hybrid Filter + Vector Search . . . 583
9.3.8 Exercise 07 - Search Performance Benchmark . . . 584
9.3.9 Exercise 08 - Search Result Ranking Analysis . . . 585
9.3.10 Exercise 09 - Similarity Debugging Scenario . . . 587
9.4 QD02 - RAG Integration Workflow . . . 588
9.4.1 Exercise 00 - Chunk Text Data for Embedding . . . 588
9.4.2 Exercise 01 - Store Document Chunks as Vectors . . . 589
9.4.3 Exercise 02 - Query Embedding Generation . . . 590
9.4.4 Exercise 03 - Retrieve Relevant Context Chunks . . . 591
9.4.5 Exercise 04 - Context Ranking by Similarity . . . 592
9.4.6 Exercise 05 - Metadata Filtering in RAG . . . 593
9.4.7 Exercise 06 - Limit Context Window Size . . . 594
9.4.8 Exercise 07 - Handle Empty Retrieval Case . . . 595
9.4.9 Exercise 08 - End-to-End RAG Pipeline Test . . . 597
9.4.10 Exercise 09 - RAG Failure Mode Analysis . . . 598
9.5 QD03 - Indexing & Scalable Retrieval . . . 599
9.5.1 Exercise 00 - HNSW Index Configuration . . . 599
9.5.2 Exercise 01 - Index Build vs Query Speed Tradeoff . . . 600
9.5.3 Exercise 02 - Payload Index Creation . . . 601
9.5.4 Exercise 03 - Large-Scale Vector Ingestion . . . 602
9.5.5 Exercise 04 - Shard Configuration Experiment . . . 603
9.5.6 Exercise 05 - Replication Factor Impact Test . . . 604
9.5.7 Exercise 06 - Consistency vs Performance Tradeoff . . . 606
9.5.8 Exercise 07 - Search Latency Monitoring . . . 607
9.5.9 Exercise 08 - Index Corruption Recovery Scenario . . . 609
9.5.10 Exercise 09 - Scalable Semantic Search System Design . . . 610

10 Linear Regression 613
10.1 Introduction . . . 613
10.1.1 What will the book teach you . . . 613
10.1.2 How to study this book . . . 615
10.1.3 The Chapters (LR00 - LR08) included in this book . . . 615
10.2 LR00 - Introduction to Linear Regression . . . 616
10.2.1 Exercise 00 - Define Linear Model $y = wx + b$ . . . 616
10.2.2 Exercise 01 - Implement Linear Function in PyTorch . . . 617
10.2.3 Exercise 02 - Tensor Shape Verification for Inputs . . . 617
10.2.4 Exercise 03 - Single Sample Forward Pass . . . 618
10.2.5 Exercise 04 - Batch Input Forward Pass . . . 619
10.2.6 Exercise 05 - Random Weight Initialization Test . . . 620

10.2.7 Exercise 06 - Effect of Bias Term Experiment . . . 621
10.2.8 Exercise 07 - Output Sensitivity to Weight Change . . . 622
10.2.9 Exercise 08 - Linear Model as Matrix Multiplication . . . 623
10.2.10 Exercise 09 - Manual Prediction vs PyTorch Output . . . 624
10.2.11 Exercise 10 - Move Model to CUDA Device . . . 626
10.2.12 Exercise 11 - CPU vs GPU Forward Pass Comparison . . . 627
10.2.13 Exercise 12 - Precision Difference (float32 vs float64) . . . 628
10.2.14 Exercise 13 - Multiple Feature Linear Model . . . 629
10.2.15 Exercise 14 - Verify Output Dimensions . . . 630
10.2.16 Exercise 15 - Linear Layer Using nn.Linear . . . 631
10.2.17 Exercise 16 - Inspect Weight and Bias Parameters . . . 632
10.2.18 Exercise 17 - Freeze Parameters Test . . . 633
10.2.19 Exercise 18 - Manual Forward Without nn.Module . . . 634
10.2.20 Exercise 19 - Forward Pass Failure Debugging . . . 635
10.3 LR01 - Model Structure and Components . . . 637
10.3.1 Exercise 00 - Separate Weights and Bias Tensors . . . 637
10.3.2 Exercise 01 - Parameter Registration in nn.Module . . . 638
10.3.3 Exercise 02 - Inspect requires_grad Flags . . . 639
10.3.4 Exercise 03 - Count Trainable Parameters . . . 640
10.3.5 Exercise 04 - Initialize Model on CUDA . . . 641
10.3.6 Exercise 05 - Verify Parameter Device Placement . . . 642
10.3.7 Exercise 06 - Manual Parameter Update Test . . . 643
10.3.8 Exercise 07 - Parameter Sharing Experiment . . . 644
10.3.9 Exercise 08 - Reinitialize Model Weights . . . 645
10.3.10 Exercise 09 - Effect of Bad Initialization . . . 646
10.3.11 Exercise 10 - Bias-only Model Behavior . . . 647
10.3.12 Exercise 11 - Weight-only Model Behavior . . . 648
10.3.13 Exercise 12 - Model Copy vs Reference Test . . . 649
10.3.14 Exercise 13 - State Dict Inspection . . . 651
10.3.15 Exercise 14 - Save and Load Model Weights . . . 652
10.3.16 Exercise 15 - Device Mismatch Error Debugging . . . 653
10.3.17 Exercise 16 - Gradient Flow Verification . . . 654
10.3.18 Exercise 17 - Manual Backward Call Test . . . 655
10.3.19 Exercise 18 - Parameter Update Order Inspection . . . 656
10.3.20 Exercise 19 - Model Structure Failure Case . . . 658
10.4 LR02 - Data Preparation and Feature Inputs . . . 659
10.4.1 Exercise 00 - Generate Synthetic Linear Dataset . . . 659
10.4.2 Exercise 01 - Add Noise to Dataset . . . 660
10.4.3 Exercise 02 - Feature Scaling Experiment . . . 661
10.4.4 Exercise 03 - Train/Test Split Implementation . . . 662
10.4.5 Exercise 04 - Batch Dataset Creation . . . 663
10.4.6 Exercise 05 - Shuffle Dataset per Epoch . . . 665
10.4.7 Exercise 06 - Move Dataset to CUDA . . . 666
10.4.8 Exercise 07 - DataLoader Integration . . . 667
10.4.9 Exercise 08 - Incorrect Shape Debugging . . . 668
10.4.10 Exercise 09 - Missing Feature Handling . . . 669
10.4.11 Exercise 10 - Bias Column Injection . . . 670
10.4.12 Exercise 11 - High-Dimensional Feature Test . . . 671

10.4.13 Exercise 12 - Collinearity Effect Observation . . . 672
10.4.14 Exercise 13 - Data Precision Impact Test . . . 673
10.4.15 Exercise 14 - Large Dataset GPU Transfer Cost . . . 674
10.4.16 Exercise 15 - Dataset Caching Strategy . . . 676
10.4.17 Exercise 16 - Dataset Integrity Check . . . 677
10.4.18 Exercise 17 - Manual Batch Iteration . . . 678
10.4.19 Exercise 18 - Data Leakage Detection . . . 679
10.4.20 Exercise 19 - Data Pipeline Failure Debugging . . . 680
10.5 LR03 - Prediction Function Implementation . . . 682
10.5.1 Exercise 00 - Define Predict(x) Function . . . 682
10.5.2 Exercise 01 - Vectorized Prediction . . . 683
10.5.3 Exercise 02 - Batch Prediction on CPU . . . 684
10.5.4 Exercise 03 - Batch Prediction on CUDA . . . 685
10.5.5 Exercise 04 - Shape-Safe Prediction . . . 686
10.5.6 Exercise 05 - Disable Grad in Inference . . . 687
10.5.7 Exercise 06 - Precision Switch During Inference . . . 688
10.5.8 Exercise 07 - Prediction Consistency Test . . . 689
10.5.9 Exercise 08 - Streaming Prediction Simulation . . . 690
10.5.10 Exercise 09 - Prediction Latency Benchmark . . . 691
10.5.11 Exercise 10 - Predict with Frozen Model . . . 692
10.5.12 Exercise 11 - CPU-to-GPU Prediction Sync Check . . . 693
10.5.13 Exercise 12 - Large Batch Prediction Stress Test . . . 694
10.5.14 Exercise 13 - Numerical Drift Detection . . . 696
10.5.15 Exercise 14 - Prediction with Missing Features . . . 697
10.5.16 Exercise 15 - Prediction Error Propagation . . . 698
10.5.17 Exercise 16 - TorchScript Prediction Export . . . 699
10.5.18 Exercise 17 - Inference Memory Profiling . . . 700
10.5.19 Exercise 18 - Prediction API Wrapper . . . 701
10.5.20 Exercise 19 - Prediction Failure Debugging . . . 702
10.6 LR04 - Error Measurement and Loss Definition . . . 704
10.6.1 Exercise 00 - Define MSE Loss Manually . . . 704
10.6.2 Exercise 01 - MSE via PyTorch . . . 705
10.6.3 Exercise 02 - Loss Reduction Modes . . . 706
10.6.4 Exercise 03 - Batch Loss Computation . . . 707
10.6.5 Exercise 04 - Loss on CUDA . . . 708
10.6.6 Exercise 05 - Loss Gradient Inspection . . . 709
10.6.7 Exercise 06 - Outlier Impact on Loss . . . 710
10.6.8 Exercise 07 - Loss Scaling Effects . . . 711
10.6.9 Exercise 08 - Loss Precision Comparison . . . 712
10.6.10 Exercise 09 - Loss Stability Test . . . 713
10.6.11 Exercise 10 - Custom Loss Class . . . 714
10.6.12 Exercise 11 - Loss with Masked Samples . . . 715
10.6.13 Exercise 12 - Loss Backward Pass Check . . . 716
10.6.14 Exercise 13 - Loss Accumulation Debugging . . . 717
10.6.15 Exercise 14 - Loss Memory Footprint . . . 719
10.6.16 Exercise 15 - CPU vs GPU Loss Parity . . . 720
10.6.17 Exercise 16 - Loss under Mixed Precision . . . 721
10.6.18 Exercise 17 - Loss Curve Logging . . . 722

10.6.19 Exercise 18 - Loss Exploding Case . . . 723
10.6.20 Exercise 19 - Loss Failure Debugging . . . 724
10.7 LR05 - Parameter Adjustment and Gradient . . . 726
10.7.1 Exercise 00 - Backward on Scalar Loss . . . 726
10.7.2 Exercise 01 - Inspect Weight Gradients . . . 727
10.7.3 Exercise 02 - Inspect Bias Gradients . . . 728
10.7.4 Exercise 03 - Zero Grad Correctly . . . 729
10.7.5 Exercise 04 - Manual Gradient Update . . . 730
10.7.6 Exercise 05 - Learning Rate Sensitivity . . . 731
10.7.7 Exercise 06 - Gradient Clipping . . . 732
10.7.8 Exercise 07 - Gradient Accumulation . . . 733
10.7.9 Exercise 08 - Gradient Noise Injection . . . 734
10.7.10 Exercise 09 - Gradient Check via Finite Difference . . . 735
10.7.11 Exercise 10 - Optimizer Step vs Manual Step . . . 737
10.7.12 Exercise 11 - CUDA Gradient Sync Check . . . 738
10.7.13 Exercise 12 - Frozen Parameter Gradient Test . . . 739
10.7.14 Exercise 13 - Gradient Explosion Scenario . . . 740
10.7.15 Exercise 14 - Gradient Vanishing Scenario . . . 741
10.7.16 Exercise 15 - Mixed Precision Backward . . . 742
10.7.17 Exercise 16 - Backward Graph Retention . . . 743
10.7.18 Exercise 17 - Gradient Memory Profiling . . . 744
10.7.19 Exercise 18 - Multi-Batch Gradient Behavior . . . 745
10.7.20 Exercise 19 - Gradient Debugging Case . . . 746
10.8 LR06 - Training Iterations and Convergence . . . 748
10.8.1 Exercise 00 - Single-Epoch Training . . . 748
10.8.2 Exercise 01 - Multi-Epoch Training Loop . . . 749
10.8.3 Exercise 02 - Convergence Criterion Check . . . 750
10.8.4 Exercise 03 - Divergence Detection . . . 751
10.8.5 Exercise 04 - Early Stopping . . . 752
10.8.6 Exercise 05 - Learning Rate Schedule . . . 754
10.8.7 Exercise 06 - Training on CUDA End-to-End . . . 755
10.8.8 Exercise 07 - CPU vs GPU Convergence . . . 756
10.8.9 Exercise 08 - Batch Size Impact . . . 757
10.8.10 Exercise 09 - Noise Robustness Test . . . 758
10.8.11 Exercise 10 - Reproducibility with Seeds . . . 759
10.8.12 Exercise 11 - Checkpoint Save per Epoch . . . 760
10.8.13 Exercise 12 - Resume Training from Checkpoint . . . 761
10.8.14 Exercise 13 - Long-Run Stability Test . . . 762
10.8.15 Exercise 14 - Loss Plateau Analysis . . . 764
10.8.16 Exercise 15 - Throughput Benchmark . . . 765
10.8.17 Exercise 16 - Gradient Drift Over Epochs . . . 766
10.8.18 Exercise 17 - Mixed Precision Training . . . 767
10.8.19 Exercise 18 - Training Failure Recovery . . . 768
10.8.20 Exercise 19 - Full Training Debug Scenario . . . 769
10.9 LR07 - Model Evaluation and Performance . . . 771
10.9.1 Exercise 00 - Compute RMSE Metric . . . 771
10.9.2 Exercise 01 - Compute MAE Metric . . . 772
10.9.3 Exercise 02 - Evaluation without Grad . . . 773

10.9.4 Exercise 03 - Train vs Eval Mode . . . 774
10.9.5 Exercise 04 - Overfitting Detection . . . 775
10.9.6 Exercise 05 - Cross-Validation Split . . . 776
10.9.7 Exercise 06 - Performance on Noisy Data . . . 777
10.9.8 Exercise 07 - Performance Scaling with Data Size . . . 778
10.9.9 Exercise 08 - Inference Throughput Test . . . 779
10.9.10 Exercise 09 - Latency Profiling on CUDA . . . 780
10.9.11 Exercise 10 - Memory Usage Profiling . . . 781
10.9.12 Exercise 11 - Error Distribution Analysis . . . 783
10.9.13 Exercise 12 - Residual Analysis . . . 784
10.9.14 Exercise 13 - Model Robustness Test . . . 785
10.9.15 Exercise 14 - Performance Regression Detection . . . 786
10.9.16 Exercise 15 - GPU Utilization Monitoring . . . 787
10.9.17 Exercise 16 - Metric Precision Comparison . . . 789
10.9.18 Exercise 17 - Evaluation Pipeline Automation . . . 790
10.9.19 Exercise 18 - Evaluation Failure Debugging . . . 791
10.9.20 Exercise 19 - Final Evaluation Report . . . 792
10.10LR08 - Result Visualization and Interpretation . . . 793
10.10.1 Exercise 00 - Plot Predictions vs Targets . . . 793
10.10.2 Exercise 01 - Plot Loss Curve . . . 795
10.10.3 Exercise 02 - Plot Residuals . . . 796
10.10.4 Exercise 03 - Visualization with Large Batches . . . 797
10.10.5 Exercise 04 - CPU vs GPU Visualization Parity . . . 798
10.10.6 Exercise 05 - Visualize Convergence Speed . . . 799
10.10.7 Exercise 06 - Hyperparameter Impact Plot . . . 800
10.10.8 Exercise 07 - Data Noise Visualization . . . 801
10.10.9 Exercise 08 - Error Distribution Plot . . . 802
10.10.10 Exercise 09 - Visualization Performance Cost . . . 804
10.10.11 Exercise 10 - Interactive Visualization . . . 805
10.10.12 Exercise 11 - Visualization from Checkpoints . . . 806
10.10.13 Exercise 12 - Multi-Run Comparison Plot . . . 807
10.10.14 Exercise 13 - Visualization Bug Debugging . . . 808
10.10.15 Exercise 14 - Interpret Weight Values . . . 810
10.10.16 Exercise 15 - Interpret Bias Effect . . . 811
10.10.17 Exercise 16 - Feature Contribution Analysis . . . 812
10.10.18 Exercise 17 - Explain Failure Cases . . . 813
10.10.19 Exercise 18 - Visualization Reproducibility . . . 814
10.10.20 Exercise 19 - Final Model Interpretation Report . . . 816

11 Logistic Regression 818
11.1 Introduction . . . 818
11.1.1 What will the book teach you . . . 818
11.1.2 How to study this book . . . 820
11.1.3 The Chapters (LG00 - LG08) included in this book . . . 821
11.2 LG00 - Introduction to Logistic Regression . . . 821
11.2.1 Exercise 00 - Define Binary Classification Problem . . . 822
11.2.2 Exercise 01 - Linear Output vs Probability Output . . . 823
11.2.3 Exercise 02 - Implement Sigmoid Function Manually . . . 824

11.2.4 Exercise 03 - Sigmoid Using PyTorch . . . 825
11.2.5 Exercise 04 - Probability Range Validation . . . 826
11.2.6 Exercise 05 - Single Sample Classification . . . 827
11.2.7 Exercise 06 - Batch Classification Forward Pass . . . 828
11.2.8 Exercise 07 - Decision Threshold Experiment . . . 829
11.2.9 Exercise 08 - Logit Interpretation Test . . . 830
11.2.10 Exercise 09 - Binary Output Failure Case . . . 831
11.2.11 Exercise 10 - Move Logistic Model to CUDA . . . 832
11.2.12 Exercise 11 - CPU vs GPU Probability Comparison . . . 833
11.2.13 Exercise 12 - Precision Effects on Sigmoid . . . 834
11.2.14 Exercise 13 - Saturation Behavior Observation . . . 836
11.2.15 Exercise 14 - Numerical Stability Breakdown . . . 837
11.2.16 Exercise 15 - Vectorized Binary Prediction . . . 838
11.2.17 Exercise 16 - Probability Calibration Check . . . 839
11.2.18 Exercise 17 - Frozen Logistic Model Test . . . 840
11.2.19 Exercise 18 - Manual Forward without nn.Module . . . 841
11.2.20 Exercise 19 - Forward Pass Failure Debugging . . . 842
11.3 LG01 - Binary Output and Decision Logic . . . 844
11.3.1 Exercise 00 - Convert Probability to Class Label . . . 844
11.3.2 Exercise 01 - Threshold Sensitivity Experiment . . . 845
11.3.3 Exercise 02 - Confusion Matrix Construction . . . 846
11.3.4 Exercise 03 - False Positive / False Negative Analysis . . . 847
11.3.5 Exercise 04 - Imbalanced Dataset Effect . . . 848
11.3.6 Exercise 05 - Dynamic Threshold Selection . . . 849
11.3.7 Exercise 06 - ROC Curve Concept Implementation . . . 851
11.3.8 Exercise 07 - AUC Metric Calculation . . . 852
11.3.9 Exercise 08 - Precision vs Recall Tradeoff . . . 853
11.3.10 Exercise 09 - Classification Failure Case . . . 854
11.3.11 Exercise 10 - CUDA Evaluation without Grad . . . 855
11.3.12 Exercise 11 - Batch-wise Decision Pipeline . . . 856
11.3.13 Exercise 12 - Prediction Consistency Test . . . 857
11.3.14 Exercise 13 - Threshold Drift Detection . . . 859
11.3.15 Exercise 14 - Metric Stability under Noise . . . 860
11.3.16 Exercise 15 - Classification Latency Benchmark . . . 861
11.3.17 Exercise 16 - Memory Profiling for Inference . . . 862
11.3.18 Exercise 17 - Misclassification Debugging . . . 864
11.3.19 Exercise 18 - Evaluation Pipeline Automation . . . 865
11.3.20 Exercise 19 - Decision Logic Failure Scenario . . . 866
11.4 LG02 - Sigmoid Activation Behavior . . . 868
11.4.1 Exercise 00 - Plot Sigmoid Curve . . . 868
11.4.2 Exercise 01 - Gradient of Sigmoid Inspection . . . 869
11.4.3 Exercise 02 - Vanishing Gradient Demonstration . . . 870
11.4.4 Exercise 03 - Saturation Region Detection . . . 871
11.4.5 Exercise 04 - Input Scaling Effect . . . 872
11.4.6 Exercise 05 - Large Logit Stability Test . . . 873
11.4.7 Exercise 06 - CUDA Gradient Comparison . . . 875
11.4.8 Exercise 07 - Float32 vs Float64 Gradient . . . 876
11.4.9 Exercise 08 - Backward Pass Visualization . . . 877

11.4.10 Exercise 09 - Activation Failure Debugging . . . 878
11.4.11 Exercise 10 - Custom Sigmoid Module . . . 879
11.4.12 Exercise 11 - Sigmoid vs Tanh Comparison . . . 880
11.4.13 Exercise 12 - Sigmoid in Deep Stack Simulation . . . 881
11.4.14 Exercise 13 - Gradient Noise Injection . . . 883
11.4.15 Exercise 14 - Autograd Graph Inspection . . . 884
11.4.16 Exercise 15 - Mixed Precision Backward Test . . . 885
11.4.17 Exercise 16 - Activation Profiling on GPU . . . 887
11.4.18 Exercise 17 - Stability under Extreme Inputs . . . 888
11.4.19 Exercise 18 - Activation Optimization Attempt . . . 889
11.4.20 Exercise 19 - Sigmoid Failure Report . . . 890
11.5 LG03 - Probability Interpretation Framework . . . 892
11.5.1 Exercise 00 - Likelihood Interpretation . . . 892
11.5.2 Exercise 01 - Log-Likelihood Computation . . . 893
11.5.3 Exercise 02 - Negative Log-Likelihood Concept . . . 894
11.5.4 Exercise 03 - Likelihood vs Loss Mapping . . . 895
11.5.5 Exercise 04 - Batch Likelihood Aggregation . . . 896
11.5.6 Exercise 05 - CUDA Likelihood Computation . . . 897
11.5.7 Exercise 06 - Likelihood Sensitivity Test . . . 898
11.5.8 Exercise 07 - Confidence Calibration Check . . . 899
11.5.9 Exercise 08 - Overconfident Prediction Detection . . . 901
11.5.10 Exercise 09 - Probability Misinterpretation Case . . . 902
11.5.11 Exercise 10 - Bayesian View Comparison . . . 903
11.5.12 Exercise 11 - Prior Effect Simulation . . . 904
11.5.13 Exercise 12 - Data Imbalance Impact . . . 905
11.5.14 Exercise 13 - Likelihood under Noise . . . 907
11.5.15 Exercise 14 - Probability Drift over Epochs . . . 908
11.5.16 Exercise 15 - Numerical Precision Effects . . . 909
11.5.17 Exercise 16 - Likelihood Visualization . . . 911
11.5.18 Exercise 17 - Probability Pipeline Debugging . . . 912
11.5.19 Exercise 18 - Confidence Failure Detection . . . 913
11.5.20 Exercise 19 - Final Probability Analysis Report . . . 915
11.6 LG04 - Classification Boundary Definition . . . 916
11.6.1 Exercise 00 - Linear Decision Boundary Plot . . . 916
11.6.2 Exercise 01 - Boundary Shift with Bias . . . 918
11.6.3 Exercise 02 - Multi-Feature Boundary Visualization . . . 919
11.6.4 Exercise 03 - Boundary on CUDA Computation . . . 920
11.6.5 Exercise 04 - Margin Concept Experiment . . . 922
11.6.6 Exercise 05 - Overlapping Classes Scenario . . . 923
11.6.7 Exercise 06 - Noise Impact on Boundary . . . 924
11.6.8 Exercise 07 - Boundary Stability Test . . . 926
11.6.9 Exercise 08 - Misclassified Region Detection . . . 927
11.6.10 Exercise 09 - Boundary Failure Case . . . 928
11.6.11 Exercise 10 - High-Dimensional Boundary Projection . . . 930
11.6.12 Exercise 11 - Boundary under Feature Scaling . . . 931
11.6.13 Exercise 12 - Regularization Effect on Boundary . . . 932
11.6.14 Exercise 13 - Boundary Drift over Training . . . 934
11.6.15 Exercise 14 - Boundary Sharpness Measurement . . . 936

11.6.16 Exercise 15 - Visualization Performance Cost . . . 937
11.6.17 Exercise 16 - Boundary Interpretation Report . . . 939
11.6.18 Exercise 17 - Decision Surface Debugging . . . 940
11.6.19 Exercise 18 - Boundary Robustness Test . . . 942
11.6.20 Exercise 19 - Final Boundary Analysis . . . 943
11.7 LG05 - Loss Representation for Classification . . . 945
11.7.1 Exercise 00 - Binary Cross Entropy Formula . . . 945
11.7.2 Exercise 01 - BCE via PyTorch . . . 946
11.7.3 Exercise 02 - Logits vs Probabilities Loss . . . 947
11.7.4 Exercise 03 - Numerical Stability (Log-Sum-Exp) . . . 948
11.7.5 Exercise 04 - BCE on CUDA . . . 949
11.7.6 Exercise 05 - Loss Gradient Inspection . . . 950
11.7.7 Exercise 06 - Class Imbalance Weighting . . . 951
11.7.8 Exercise 07 - Loss Scaling Effect . . . 952
11.7.9 Exercise 08 - Loss Explosion Scenario . . . 953
11.7.10 Exercise 09 - Loss Debugging Case . . . 954
11.7.11 Exercise 10 - Custom BCE Loss Class . . . 955
11.7.12 Exercise 11 - Masked Loss Computation . . . 956
11.7.13 Exercise 12 - Reduction Mode Comparison . . . 957
11.7.14 Exercise 13 - Mixed Precision Loss . . . 958
11.7.15 Exercise 14 - Loss Memory Profiling . . . 959
11.7.16 Exercise 15 - CPU vs GPU Loss Parity . . . 960
11.7.17 Exercise 16 - Loss Curve Logging . . . 961
11.7.18 Exercise 17 - Loss Drift Detection . . . 962
11.7.19 Exercise 18 - Failure Recovery Test . . . 963
11.7.20 Exercise 19 - Final Loss Report . . . 964
11.8 LG06 - Model Adjustment and Gradient Direction . . . 965
11.8.1 Exercise 00 - Backward Pass on BCE Loss . . . 965
11.8.2 Exercise 01 - Inspect Weight Gradients . . . 967
11.8.3 Exercise 02 - Inspect Bias Gradients . . . 968
11.8.4 Exercise 03 - Manual Gradient Update . . . 969
11.8.5 Exercise 04 - Learning Rate Sensitivity . . . 970
11.8.6 Exercise 05 - Gradient Clipping in Classification . . . 971
11.8.7 Exercise 06 - Gradient Noise Injection . . . 972
11.8.8 Exercise 07 - Optimizer Step Comparison . . . 973
11.8.9 Exercise 08 - CUDA Gradient Sync Check . . . 975
11.8.10 Exercise 09 - Gradient Explosion Scenario . . . 976
11.8.11 Exercise 10 - Gradient Vanishing Scenario . . . 977
11.8.12 Exercise 11 - Mixed Precision Backward . . . 978
11.8.13 Exercise 12 - Frozen Layer Gradient Test . . . 979
11.8.14 Exercise 13 - Gradient Accumulation . . . 980
11.8.15 Exercise 14 - Backward Graph Retention . . . 981
11.8.16 Exercise 15 - Gradient Memory Profiling . . . 983
11.8.17 Exercise 16 - Multi-Batch Gradient Behavior . . . 984
11.8.18 Exercise 17 - Gradient Drift Detection . . . 985
11.8.19 Exercise 18 - Debugging Gradient Failure . . . 986
11.8.20 Exercise 19 - End-to-End Gradient Report . . . 987
11.9 LG07 - Accuracy Validation and Output Analysis . . . 988

11.9.1 Exercise 00 - Accuracy Metric Implementation 988
11.9.2 Exercise 01 - Precision Metric 989
11.9.3 Exercise 02 - Recall Metric 990
11.9.4 Exercise 03 - F1 Score Computation 991
11.9.5 Exercise 04 - Evaluation without Grad 992
11.9.6 Exercise 05 - GPU Evaluation Throughput 993
11.9.7 Exercise 06 - Confusion Matrix Visualization 994
11.9.8 Exercise 07 - Metric Stability under Noise 995
11.9.9 Exercise 08 - Performance Scaling with Data Size 996
11.9.10 Exercise 09 - Latency Profiling on CUDA 997
11.9.11 Exercise 10 - Memory Usage Profiling 999
11.9.12 Exercise 11 - Error Distribution Analysis 1000
11.9.13 Exercise 12 - Threshold Sensitivity Analysis 1001
11.9.14 Exercise 13 - Metric Regression Detection 1002
11.9.15 Exercise 14 - Robustness Test 1003
11.9.16 Exercise 15 - Evaluation Pipeline Automation 1004
11.9.17 Exercise 16 - Evaluation Failure Debugging 1005
11.9.18 Exercise 17 - Multi-Run Metric Comparison 1006
11.9.19 Exercise 18 - Final Evaluation Report 1007
11.9.20 Exercise 19 - Production Metric Sanity Check 1008
11.10LG08 - Multi-Class Extension Concepts 1009
11.10.1 Exercise 00 - One-vs-Rest Logistic Strategy 1009
11.10.2 Exercise 01 - Softmax vs Sigmoid Comparison 1010
11.10.3 Exercise 02 - Multi-Class Dataset Preparation 1011
11.10.4 Exercise 03 - Cross-Entropy for Multi-Class 1012
11.10.5 Exercise 04 - CUDA Multi-Class Training 1013
11.10.6 Exercise 05 - Class Imbalance in Multi-Class 1014
11.10.7 Exercise 06 - Confusion Matrix for Multi-Class 1015
11.10.8 Exercise 07 - Macro vs Micro Metrics 1016
11.10.9 Exercise 08 - Probability Calibration Multi-Class 1017
11.10.10 Exercise 09 - Boundary Visualization Multi-Class 1018
11.10.11 Exercise 10 - Gradient Stability Multi-Class 1019
11.10.12 Exercise 11 - Mixed Precision Multi-Class 1020
11.10.13 Exercise 12 - Performance Scaling Classes 1021
11.10.14 Exercise 13 - Error Analysis per Class 1023
11.10.15 Exercise 14 - Metric Drift Detection 1024
11.10.16 Exercise 15 - End-to-End Multi-Class Pipeline 1025
11.10.17 Exercise 16 - Production Readiness Check 1026
11.10.18 Exercise 17 - Failure Recovery Scenario 1027
11.10.19 Exercise 18 - Scalability Stress Test 1028
11.10.20 Exercise 19 - Final Multi-Class Report 1029

12 Perceptron **1031**
12.1 Introduction 1031
12.1.1 What will the book teach you 1031
12.1.2 How to study this book 1034
12.1.3 The Chapters (PC00 - PC08) included in this book 1034
12.2 PC00 - Perceptron Concept and Motivation 1035

12.2.1 Exercise 00 - Understand the Limitation of Linear Regression for Classification 1035
12.2.2 Exercise 01 - Define the Perceptron as a Binary Decision Unit with Step Function 1036
12.2.3 Exercise 02 - Implement a Basic Perceptron Forward Pass Using Pure Python 1037
12.2.4 Exercise 03 - Translate Mathematical Perceptron Formula into PyTorch Operations 1038
12.2.5 Exercise 04 - Compare Linear Regression Output vs Perceptron Binary Output 1039
12.2.6 Exercise 05 - Analyze Why Perceptron Uses a Hard Decision Boundary 1040
12.2.7 Exercise 06 - Implement the Step Activation Function and Test Its Behavior 1041
12.2.8 Exercise 07 - Visualize Binary Decisions Generated by a Single Perceptron 1042
12.2.9 Exercise 08 - Test Perceptron on Linearly Separable Dataset 1043
12.2.10 Exercise 09 - Observe Failure on Non-Linearly Separable Data 1044
12.2.11 Exercise 10 - Move Perceptron Computation to CUDA and Verify Output Consistency 1045
12.2.12 Exercise 11 - Compare CPU vs GPU Execution for Perceptron Forward Pass 1046
12.2.13 Exercise 12 - Inspect Weight and Bias Initialization Effects on Decisions 1047
12.2.14 Exercise 13 - Analyze Sensitivity of Decision Boundary to Weight Changes 1048
12.2.15 Exercise 14 - Implement Perceptron without Using nn.Module 1050
12.2.16 Exercise 15 - Validate Output Shape and Data Type in Classification Pipeline 1051
12.2.17 Exercise 16 - Freeze Perceptron Parameters and Test Prediction Stability 1052
12.2.18 Exercise 17 - Debug Incorrect Binary Outputs Caused by Shape Mismatch 1053
12.2.19 Exercise 18 - Stress Test Perceptron with Large Batch Inputs 1054
12.2.20 Exercise 19 - Document Conceptual and Practical Failures of the Perceptron Model 1055
12.3 PC01 - Mathematical Structure of the Perceptron 1056
12.3.1 Exercise 00 - Express the Perceptron as a Vector Dot Product Plus Bias 1057
12.3.2 Exercise 01 - Rewrite the Perceptron Equation Using Matrix Notation .1058
12.3.3 Exercise 02 - Analyze the Geometric Meaning of the Weight Vector . .1059
12.3.4 Exercise 03 - Interpret the Bias Term as a Decision Boundary Shift . .1060
12.3.5 Exercise 04 - Compute the Signed Distance from a Point to the Decision Boundary 1061
12.3.6 Exercise 05 - Verify Perceptron Output Using Manual Vector Calculations 1062
12.3.7 Exercise 06 - Compare Multiple Weight Configurations and Their Boundaries 1063
12.3.8 Exercise 07 - Visualize Decision Hyperplane in 2D Feature Space 1064
12.3.9 Exercise 08 - Extend Decision Boundary Visualization to Higher Dimensions 1065
12.3.10 Exercise 09 - Investigate Numerical Precision Effects on Boundary Placement 1066
12.3.11 Exercise 10 - Implement Mathematical Perceptron Using CUDA Tensors 1067

12.3.12 Exercise 11 - Compare Float32 vs Float64 in Boundary Computation .1068
12.3.13 Exercise 12 - Detect Boundary Instability Under Small Input Noise . .1069
12.3.14 Exercise 13 - Analyze Scaling Effects on Weights and Inputs 1070
12.3.15 Exercise 14 - Validate Linear Separability Assumption Experimentally .1072
12.3.16 Exercise 15 - Demonstrate Failure Case with XOR Dataset 1073
12.3.17 Exercise 16 - Explain Why Single Perceptron Cannot Solve XOR . . .1074
12.3.18 Exercise 17 - Inspect Internal Tensor Operations During Forward Pass 1075
12.3.19 Exercise 18 - Stress Test Dot Product Computation on GPU 1076
12.3.20 Exercise 19 - Produce a Formal Mathematical Interpretation Report . .1077
12.4 PC02 - Perceptron Learning Rule and Updates 1078
12.4.1 Exercise 00 - Derive the Perceptron Weight Update Rule from Classification Errors 1078
12.4.2 Exercise 01 - Implement Manual Weight Update Based on Misclassified Samples 1079
12.4.3 Exercise 02 - Analyze the Role of Learning Rate in Perceptron Convergence Behavior 1080
12.4.4 Exercise 03 - Implement Bias Update Rule and Observe Its Effect on Decision Boundary 1081
12.4.5 Exercise 04 - Track Weight Evolution Across Multiple Training Iterations 1082
12.4.6 Exercise 05 - Compare Single-Sample Updates vs Batch Updates in Perceptron Training 1083
12.4.7 Exercise 06 - Detect and Analyze Oscillation in Non-Separable Datasets 1084
12.4.8 Exercise 07 - Implement Training Loop Without Using PyTorch Optimizers 1085
12.4.9 Exercise 08 - Observe Convergence Speed Under Different Data Orderings 1086
12.4.10 Exercise 09 - Investigate Failure to Converge on XOR-like Distributions 1087
12.4.11 Exercise 10 - Move Training Computation to CUDA and Verify Deterministic Behavior 1088
12.4.12 Exercise 11 - Compare CPU vs GPU Weight Update Numerical Differences 1089
12.4.13 Exercise 12 - Measure the Effect of Feature Scaling on Update Magnitude 1090
12.4.14 Exercise 13 - Analyze Sensitivity of Updates to Noisy Labels 1091
12.4.15 Exercise 14 - Implement Early Stopping Criterion Based on Zero Classification Errors 1092
12.4.16 Exercise 15 - Log and Visualize Weight Trajectories During Training 1093
12.4.17 Exercise 16 - Demonstrate Catastrophic Update with Large Learning Rate 1094
12.4.18 Exercise 17 - Validate Correctness of Update Rule Using Manual Calculations 1095
12.4.19 Exercise 18 - Stress Test Update Rule with High-Dimensional Feature Space 1096

12.4.20 Exercise 19 - Produce a Detailed Failure Analysis of the Learning Rule 1097
12.5 PC03 - Training Dynamics and Convergence Theory 1098
12.5.1 Exercise 00 - Define Formal Convergence Criteria for the Perceptron Algorithm 1098
12.5.2 Exercise 01 - Experimentally Verify the Perceptron Convergence Theorem .1099
12.5.3 Exercise 02 - Analyze the Effect of Margin Size on Training Stability .1100
12.5.4 Exercise 03 - Measure Number of Updates Until Convergence 1101
12.5.5 Exercise 04 - Study the Impact of Data Ordering on Convergence Speed .1102
12.5.6 Exercise 05 - Investigate Training Behavior on Nearly Separable Data .1103
12.5.7 Exercise 06 - Demonstrate Divergence on Non-Separable Datasets 1103
12.5.8 Exercise 07 - Compare Deterministic vs Randomized Sample Ordering .1104
12.5.9 Exercise 08 - Observe Weight Norm Growth During Training 1105
12.5.10 Exercise 09 - Analyze the Relationship Between Margin and Generalization 1106
12.5.11 Exercise 10 - Run Long-Horizon Training on CUDA for Stability Observation 1107
12.5.12 Exercise 11 - Compare Float Precision Effects on Convergence 1108
12.5.13 Exercise 12 - Detect Numerical Drift in Extended Training Runs 1109
12.5.14 Exercise 13 - Visualize Convergence Trajectories in Parameter Space . .1110
12.5.15 Exercise 14 - Investigate Overfitting Behavior in Linearly Separable Data .1111
12.5.16 Exercise 15 - Analyze Training Failure Modes and Their Root Causes .1112
12.5.17 Exercise 16 - Compare Convergence Between CPU and GPU Implementations 1113
12.5.18 Exercise 17 - Stress Test Convergence with Large-Scale Synthetic Data 1114
12.5.19 Exercise 18 - Validate Theoretical Claims Using Empirical Results . . .1115
12.5.20 Exercise 19 - Write a Formal Convergence and Stability Report 1116
12.6 PC04 - Perceptron Loss Perspective and Errors 1117
12.6.1 Exercise 00 - Interpret Perceptron Learning as Error-Driven Optimization .1117
12.6.2 Exercise 01 - Define Implicit Loss Function Used by the Perceptron . .1118
12.6.3 Exercise 02 - Compare Perceptron Loss with Hinge Loss 1119
12.6.4 Exercise 03 - Analyze Zero-One Loss Limitations in Optimization . . .1120
12.6.5 Exercise 04 - Visualize Loss Landscape for Simple Perceptron Models .1120
12.6.6 Exercise 05 - Implement Perceptron Loss Manually in PyTorch 1121
12.6.7 Exercise 06 - Compare Gradient-Free Updates vs Gradient-Based Losses .1122
12.6.8 Exercise 07 - Observe Non-Smooth Loss Behavior at Decision Boundary .1123
12.6.9 Exercise 08 - Analyze Sensitivity of Loss to Margin Violations 1124
12.6.10 Exercise 09 - Study Error Accumulation Across Epochs 1125
12.6.11 Exercise 10 - Implement Loss Computation on CUDA 1126
12.6.12 Exercise 11 - Compare CPU vs GPU Loss Evaluation Consistency . . .1126
12.6.13 Exercise 12 - Examine Numerical Stability of Loss Under Extreme Values .1127
12.6.14 Exercise 13 - Relate Loss Behavior to Classification Performance 1128
12.6.15 Exercise 14 - Debug Unexpected Loss Values During Training 1129
12.6.16 Exercise 15 - Visualize Loss Evolution Over Training Iterations 1130
12.6.17 Exercise 16 - Contrast Loss-Based Training with Backprop-Based Models .1131
12.6.18 Exercise 17 - Evaluate Robustness of Loss to Label Noise 1132
12.6.19 Exercise 18 - Identify Scenarios Where Loss Minimization Fails 1133

12.6.20 Exercise 19 - Produce a Critical Analysis of Perceptron Loss Limitations 1134
12.7 PC05 - Multi-Class Perceptron Extensions 1135
12.7.1 Exercise 00 - Extend Binary Perceptron to One-vs-Rest Multi-Class Setting 1135
12.7.2 Exercise 01 - Implement Class-Specific Weight Vectors for Multi-Class Output 1136
12.7.3 Exercise 02 - Define Prediction Rule for Multi-Class Perceptron . . . 1137
12.7.4 Exercise 03 - Implement Multi-Class Training Loop with Error Updates 1137
12.7.5 Exercise 04 - Analyze Decision Regions in Multi-Class Feature Space 1138
12.7.6 Exercise 05 - Compare One-vs-Rest and One-vs-One Strategies 1139
12.7.7 Exercise 06 - Handle Class Imbalance in Multi-Class Perceptron Training 1140
12.7.8 Exercise 07 - Evaluate Multi-Class Accuracy and Confusion Matrix . . 1141
12.7.9 Exercise 08 - Observe Training Instability with Increasing Number of Classes 1142
12.7.10 Exercise 09 - Visualize Multi-Class Decision Boundaries 1143
12.7.11 Exercise 10 - Execute Multi-Class Perceptron Training on CUDA . . . 1144
12.7.12 Exercise 11 - Compare CPU vs GPU Scaling with Number of Classes 1144
12.7.13 Exercise 12 - Measure Memory Growth with Class Count 1145
12.7.14 Exercise 13 - Investigate Error Propagation Across Classes 1146
12.7.15 Exercise 14 - Analyze Misclassification Patterns in Multi-Class Setting 1147
12.7.16 Exercise 15 - Implement Weight Regularization for Multi-Class Stability 1148
12.7.17 Exercise 16 - Compare Multi-Class Perceptron with Softmax Regression 1149
12.7.18 Exercise 17 - Stress Test Multi-Class Model with High-Dimensional Inputs 1150
12.7.19 Exercise 18 - Debug Class-Specific Training Failures 1151
12.7.20 Exercise 19 - Produce a Comprehensive Multi-Class Analysis Report 1152
12.8 PC06 - Computational Performance and CUDA Effects 1153
12.8.1 Exercise 00 - Benchmark Perceptron Forward Pass on CPU vs GPU . . 1153
12.8.2 Exercise 01 - Benchmark Training Throughput on CUDA 1154
12.8.3 Exercise 02 - Measure Memory Usage During Training on GPU 1155
12.8.4 Exercise 03 - Analyze Kernel Launch Overhead for Small Batches 1156
12.8.5 Exercise 04 - Evaluate Scaling Behavior with Increasing Batch Size . . 1157
12.8.6 Exercise 05 - Compare Float32 vs Mixed Precision Performance 1158
12.8.7 Exercise 06 - Identify Performance Bottlenecks in Training Loop 1159
12.8.8 Exercise 07 - Profile CUDA Execution Using PyTorch Tools 1160
12.8.9 Exercise 08 - Optimize Data Transfer Between Host and Device 1161
12.8.10 Exercise 09 - Investigate Synchronization Overhead 1161
12.8.11 Exercise 10 - Stress Test GPU with Large-Scale Synthetic Data 1162
12.8.12 Exercise 11 - Compare Multi-GPU vs Single-GPU Perceptron Execution 1163
12.8.13 Exercise 12 - Measure Determinism Across GPU Runs 1164
12.8.14 Exercise 13 - Analyze Precision-Induced Performance Variations 1165
12.8.15 Exercise 14 - Benchmark Perceptron vs Logistic Regression Performance 1166
12.8.16 Exercise 15 - Identify When GPU Usage Is Counterproductive 1167
12.8.17 Exercise 16 - Evaluate Energy Efficiency of GPU Execution 1168
12.8.18 Exercise 17 - Optimize Training Loop for Throughput 1169
12.8.19 Exercise 18 - Detect Performance Regression After Code Changes . . . 1170
12.8.20 Exercise 19 - Produce a Performance Optimization Report 1171

12.9 PC07 - Generalization and Theoretical Limits 1172
12.9.1 Exercise 00 - Analyze Generalization Capability of the Perceptron Model 1173
12.9.2 Exercise 01 - Study the Impact of Margin on Generalization Performance 1173
12.9.3 Exercise 02 - Compare Training Accuracy vs Test Accuracy 1174
12.9.4 Exercise 03 - Investigate Overfitting in Linearly Separable Data 1175
12.9.5 Exercise 04 - Evaluate Robustness to Input Noise 1176
12.9.6 Exercise 05 - Analyze Sensitivity to Label Noise 1177
12.9.7 Exercise 06 - Explore the Effect of Regularization on Generalization . .1178
12.9.8 Exercise 07 - Compare Perceptron Generalization to Logistic Regression 1179
12.9.9 Exercise 08 - Study Bias-Variance Tradeoff in Simple Models 1180
12.9.10 Exercise 09 - Identify Scenarios Where Perceptron Generalization Fails 1181
12.9.11 Exercise 10 - Evaluate Generalization on High-Dimensional Sparse Data 1182
12.9.12 Exercise 11 - Measure Stability Across Different Random Seeds 1183
12.9.13 Exercise 12 - Compare Deterministic vs Stochastic Training Outcomes .1184
12.9.14 Exercise 13 - Analyze Generalization Under Distribution Shift 1185
12.9.15 Exercise 14 - Investigate Adversarial-Like Perturbations 1185
12.9.16 Exercise 15 - Produce Empirical Evidence Supporting Theoretical Claims 1186
12.9.17 Exercise 16 - Document Limitations of Perceptron in Real-World Data 1187
12.9.18 Exercise 17 - Compare Perceptron to Modern Linear Classifiers 1188
12.9.19 Exercise 18 - Evaluate Suitability of Perceptron for Production Use . .1189
12.9.20 Exercise 19 - Write a Critical Theoretical Review of the Perceptron . .1190
12.10PC08 - Transition to Multi-Layer Networks 1191
12.10.1 Exercise 00 - Demonstrate the Inability of Single Perceptron to Model XOR 1191
12.10.2 Exercise 01 - Manually Combine Two Perceptrons to Solve XOR 1192
12.10.3 Exercise 02 - Interpret Hidden Units as Feature Transformers 1193
12.10.4 Exercise 03 - Build a Two-Layer Perceptron Network Manually 1194
12.10.5 Exercise 04 - Compare Single-Layer vs Multi-Layer Expressiveness . . .1195
12.10.6 Exercise 05 - Analyze Decision Boundaries Formed by Two-Layer Models 1196
12.10.7 Exercise 06 - Observe Emergent Non-Linearity in Hidden Representations 1197
12.10.8 Exercise 07 - Implement Simple Two-Layer Network in PyTorch 1197
12.10.9 Exercise 08 - Compare Manual Network vs nn.Module Implementation 1198
12.10.10 Exercise 09 - Train Two-Layer Network Using Gradient-Based Optimization 1199
12.10.11 Exercise 10 - Execute Two-Layer Training on CUDA 1200
12.10.12 Exercise 11 - Visualize Hidden Layer Activations 1201
12.10.13 Exercise 12 - Study Gradient Flow Through Multiple Layers 1202
12.10.14 Exercise 13 - Analyze Training Stability in Multi-Layer Networks . . .1203
12.10.15 Exercise 14 - Compare Perceptron Training to Backpropagation 1204
12.10.16 Exercise 15 - Identify Failure Modes in Shallow Neural Networks 1205
12.10.17 Exercise 16 - Evaluate Expressive Power Gain with Additional Layers .1206
12.10.18 Exercise 17 - Bridge Conceptual Gap Between Perceptron and MLP . .1207
12.10.19 Exercise 18 - Stress Test Multi-Layer Perceptron with Complex Data .1208
12.10.20 Exercise 19 - Produce a Final Transition Report Toward Neural Networks 1209

13 MLP (Feed-Forward Network) 1211
13.1 Introduction 1211

13.1.1 What will the book teach you 1211
13.1.2 Why this book comes after the Perceptron 1213
13.1.3 How to study this book 1213
13.1.4 The Chapters (MLP00 - MLP08) included in this book 1213
13.2 MLP00 - Conceptual Foundation of MLP 1214
13.2.1 Exercise 00 - Understand Why Single Perceptron Is Insufficient for Complex Functions 1214
13.2.2 Exercise 01 - Define Feed-Forward Network as Layered Function Composition 1215
13.2.3 Exercise 02 - Interpret Hidden Layers as Learned Feature Transformers 1216
13.2.4 Exercise 03 - Compare Linear Model Output vs Multi-Layer Non-Linear Output 1216
13.2.5 Exercise 04 - Visualize Information Flow Through Multiple Layers . . .1217
13.2.6 Exercise 05 - Implement a Two-Layer Network Using Pure Tensor Operations 1218
13.2.7 Exercise 06 - Translate Mathematical MLP Formulation into PyTorch Code 1219
13.2.8 Exercise 07 - Analyze the Role of Non-Linearity Between Linear Layers 1220
13.2.9 Exercise 08 - Verify Dimensional Consistency Across All Layers 1221
13.2.10 Exercise 09 - Identify Common Conceptual Misunderstandings About Hidden Layers 1222
13.2.11 Exercise 10 - Execute Forward Pass of MLP on CUDA Device 1223
13.2.12 Exercise 11 - Compare CPU vs GPU Forward Execution Outputs . . .1223
13.2.13 Exercise 12 - Inspect Intermediate Activations at Each Hidden Layer .1224
13.2.14 Exercise 13 - Analyze the Effect of Depth on Representational Power .1225
13.2.15 Exercise 14 - Implement Forward Pass Without Using nn.Module 1226
13.2.16 Exercise 15 - Debug Shape Errors in Multi-Layer Tensor Computations 1227
13.2.17 Exercise 16 - Stress Test Forward Pass with Large Batch Sizes 1228
13.2.18 Exercise 17 - Validate Determinism of Forward Pass Under Fixed Seeds 1229
13.2.19 Exercise 18 - Examine Numerical Stability of Deep Forward Computation 1230
13.2.20 Exercise 19 - Write a Conceptual Summary Explaining Why MLP Enables Universal Approximation 1231
13.3 MLP01 - Architecture Design and Layer Composition 1233
13.3.1 Exercise 00 - Design a Configurable MLP Architecture with Variable Depth 1233
13.3.2 Exercise 01 - Implement MLP Using nn.Module and nn.Linear Layers .1234
13.3.3 Exercise 02 - Dynamically Build Network from Layer Configuration List 1235
13.3.4 Exercise 03 - Register Parameters Correctly for Arbitrary Network Depth 1235
13.3.5 Exercise 04 - Verify Parameter Count for Custom Architectures 1237
13.3.6 Exercise 05 - Inspect Weight and Bias Shapes for Each Layer 1238
13.3.7 Exercise 06 - Implement Forward Hook to Capture Intermediate Outputs 1238
13.3.8 Exercise 07 - Compare Shallow vs Deep Architectures on Same Task 1239
13.3.9 Exercise 08 - Analyze Trade-Off Between Width and Depth 1240

13.3.10 Exercise 09 - Detect Architecture-Induced Bottlenecks 1241
13.3.11 Exercise 10 - Move Entire Architecture to CUDA and Validate Placement 1242
13.3.12 Exercise 11 - Debug Device Mismatch Errors in Multi-Layer Networks 1243
13.3.13 Exercise 12 - Test Forward Compatibility with Different Input Shapes 1244
13.3.14 Exercise 13 - Clone Architecture and Compare Parameter Independence 1245
13.3.15 Exercise 14 - Save and Reload Architecture with State Dict 1246
13.3.16 Exercise 15 - Stress Test Architecture Creation with Large Layer Counts 1247
13.3.17 Exercise 16 - Evaluate Memory Footprint of Deep Architectures 1248
13.3.18 Exercise 17 - Compare Manual Tensor Implementation vs nn.Module 1250
13.3.19 Exercise 18 - Identify Architectural Failure Modes Early 1251
13.3.20 Exercise 19 - Produce an Architecture Design Rationale Document 1252
13.4 MLP02 - Activation Functions and Non-Linearity 1253
13.4.1 Exercise 00 - Implement ReLU Activation and Observe Its Piecewise Behavior 1253
13.4.2 Exercise 01 - Compare ReLU, Sigmoid, and Tanh in Forward Propagation 1254
13.4.3 Exercise 02 - Visualize Activation Output Distributions Across Layers 1255
13.4.4 Exercise 03 - Analyze Gradient Flow for Different Activation Choices 1256
13.4.5 Exercise 04 - Demonstrate Vanishing Gradient in Deep Sigmoid Networks 1257
13.4.6 Exercise 05 - Demonstrate Dying ReLU Phenomenon Experimentally 1258
13.4.7 Exercise 06 - Implement LeakyReLU and Compare Gradient Behavior 1259
13.4.8 Exercise 07 - Test GELU Activation in Deep Networks 1260
13.4.9 Exercise 08 - Evaluate Activation Choice Impact on Training Stability 1261
13.4.10 Exercise 09 - Debug Activation-Induced Training Failures 1262
13.4.11 Exercise 10 - Execute Activation Functions on CUDA and Compare Performance 1263
13.4.12 Exercise 11 - Compare Float Precision Effects on Activation Output 1264
13.4.13 Exercise 12 - Track Activation Saturation Across Layers 1265
13.4.14 Exercise 13 - Instrument Network to Log Activation Statistics 1266
13.4.15 Exercise 14 - Replace Activation Mid-Training and Observe Effects 1267
13.4.16 Exercise 15 - Analyze Activation Interaction with Weight Initialization 1268

13.4.17 Exercise 16 - Compare Custom Activation vs Built-In PyTorch Activation 1269
13.4.18 Exercise 17 - Stress Test Activations Under Extreme Inputs 1270
13.4.19 Exercise 18 - Identify Scenarios Where Activation Choice Dominates Performance 1271
13.4.20 Exercise 19 - Write an Activation Function Selection Guide Based on Experiments 1272
13.5 MLP03 - Backpropagation Mechanics in MLP 1273
13.5.1 Exercise 00 - Perform a Full Forward Pass and Confirm Autograd Graph Construction 1273
13.5.2 Exercise 01 - Compute Loss and Trigger Backward Pass Through Multiple Layers 1274
13.5.3 Exercise 02 - Inspect Gradients of Each Layer and Verify Their Tensor Shapes 1275
13.5.4 Exercise 03 - Validate Gradient Flow by Checking Non-Zero Gradients in Hidden Layers 1276
13.5.5 Exercise 04 - Demonstrate Vanishing Gradients in Deep Networks Using Saturating Activations 1277
13.5.6 Exercise 05 - Demonstrate Exploding Gradients and Apply Gradient Clipping Correctly 1278
13.5.7 Exercise 06 - Compare Manual Gradient Computation vs Autograd for a Small Network 1279
13.5.8 Exercise 07 - Verify Backward Correctness Using Finite Difference Gradient Checking 1280
13.5.9 Exercise 08 - Detect and Fix Common Backprop Bugs Caused by Detached Tensors 1281
13.5.10 Exercise 09 - Debug Gradient Mismatch Caused by Wrong Loss Reduction Mode 1282
13.5.11 Exercise 10 - Execute Backward Pass on CUDA and Validate Gradient Consistency 1283
13.5.12 Exercise 11 - Investigate CPU vs GPU Gradient Parity Under Different Random Seeds 1284
13.5.13 Exercise 12 - Compare float32 vs float64 Gradients for Stability and Precision Insights 1285
13.5.14 Exercise 13 - Use Autograd Hooks to Log Gradient Norms Layer by Layer During Training 1287
13.5.15 Exercise 14 - Prove the Need for zero_grad by Demonstrating Gradient Accumulation Errors 1288
13.5.16 Exercise 15 - Analyze the Effect of Batch Size on Gradient Variance and Update Stability 1289
13.5.17 Exercise 16 - Demonstrate Graph Retention Costs Using retain_graph and Memory Profiling 1290
13.5.18 Exercise 17 - Debug Device Mismatch Errors During Backward Computation on CUDA 1291
13.5.19 Exercise 18 - Stress Test Backpropagation with Deep Architecture and Large Batch Inputs 1292
13.5.20 Exercise 19 - Write a Full Backprop Debugging Playbook Based on Observed Failure Modes 1293

13.6 MLP04 - Weight Initialization and Training Stability 1295
13.6.1 Exercise 00 - Implement Zero Initialization and Observe Symmetry Failure in Hidden Units 1295
13.6.2 Exercise 01 - Compare Random Uniform vs Random Normal Initialization in Early Training 1296
13.6.3 Exercise 02 - Implement Xavier Initialization and Validate Activation Variance Preservation 1297
13.6.4 Exercise 03 - Implement He Initialization and Validate ReLU-Compatible Gradient Flow 1298
13.6.5 Exercise 04 - Track Activation Statistics to Confirm Initialization Quality Across Layers 1299
13.6.6 Exercise 05 - Track Gradient Statistics to Detect Early Instability from Bad Initialization 1300
13.6.7 Exercise 06 - Demonstrate Training Collapse Caused by Extreme Initialization Scale 1301
13.6.8 Exercise 07 - Compare Training Speed Under Different Initializations Using the Same Dataset 1302
13.6.9 Exercise 08 - Analyze How Initialization Interacts with Activation Choice (ReLU vs Tanh) 1303
13.6.10 Exercise 09 - Debug NaN Loss Caused by Unstable Initialization and Fix the Root Cause 1304
13.6.11 Exercise 10 - Run Initialization Experiments on CUDA and Profile Startup Training Stability 1305
13.6.12 Exercise 11 - Compare float32 vs mixed precision Training Stability for Each Initialization 1306
13.6.13 Exercise 12 - Implement Custom Parameter Initializer for Arbitrary Network Depth 1307
13.6.14 Exercise 13 - Measure Layer-Wise Signal Propagation Depth Before Saturation Occurs 1308
13.6.15 Exercise 14 - Diagnose Dying ReLU Caused by Initialization and Apply LeakyReLU Fix 1309
13.6.16 Exercise 15 - Show How Bias Initialization Influences Early Learning and Decision Boundaries 1309
13.6.17 Exercise 16 - Compare Built-In PyTorch init Methods with Your Manual Implementations 1310
13.6.18 Exercise 17 - Stress Test Initialization with Very Deep Networks and Monitor Memory Usage 1311
13.6.19 Exercise 18 - Create a Reproducible Initialization Benchmark Report with Controlled Seeds 1313
13.6.20 Exercise 19 - Produce a Practical Initialization Selection Guide for Real Training Projects 1314
13.7 MLP05 - Optimizers and Learning Rate Dynamics 1315
13.7.1 Exercise 00 - Implement Manual SGD Updates and Verify Parameter Changes Step by Step 1315
13.7.2 Exercise 01 - Compare SGD vs Momentum and Observe Convergence Differences 1316
13.7.3 Exercise 02 - Implement Adam Optimizer and Track Bias Correction Behavior 1317

13.7.4 Exercise 03 - Compare Adam vs AdamW and Identify Weight Decay Differences in Practice 1318
13.7.5 Exercise 04 - Perform Learning Rate Sweep and Identify Divergence Threshold 1319
13.7.6 Exercise 05 - Implement Learning Rate Scheduler and Validate Step Timing and Effects 1319
13.7.7 Exercise 06 - Diagnose Slow Convergence Caused by Too-Low Learning Rate and Fix It 1320
13.7.8 Exercise 07 - Diagnose Divergence Caused by Too-High Learning Rate and Stabilize Training 1321
13.7.9 Exercise 08 - Implement Gradient Clipping and Measure Its Effect on Optimization Stability 1322
13.7.10 Exercise 09 - Compare Optimization Under Different Batch Sizes to Observe Noise Benefits 1323
13.7.11 Exercise 10 - Execute Optimizer Steps on CUDA and Validate Parameter Device Placement 1323
13.7.12 Exercise 11 - Profile Optimizer Overhead on GPU and Identify Bottlenecks in Update Step 1324
13.7.13 Exercise 12 - Compare Mixed Precision vs Full Precision Optimization Dynamics on the Same Task 1325
13.7.14 Exercise 13 - Implement Gradient Accumulation to Simulate Larger Batch Size on Limited GPU Memory 1326
13.7.15 Exercise 14 - Verify Determinism and Reproducibility Under Fixed Seeds and Controlled Settings 1327
13.7.16 Exercise 15 - Detect and Fix Silent Training Bugs Caused by Missing zero_grad or Wrong Order 1328
13.7.17 Exercise 16 - Implement Weight Decay Correctly and Demonstrate Difference from L2 Regularization 1329
13.7.18 Exercise 17 - Benchmark Convergence Speed for Multiple Optimizers and Summarize Results 1330
13.7.19 Exercise 18 - Monitor Gradient Norms and Parameter Norms to Detect Impending Instability Early 1330
13.7.20 Exercise 19 - Write an Optimization Debugging Checklist Covering All Common Training Failures 1331
13.8 MLP06 - Loss Functions, Training Loop, and Logging 1332
13.8.1 Exercise 00 - Build a Minimal Training Loop with Proper Train/Eval Separation and Correct Modes 1332
13.8.2 Exercise 01 - Implement Classification Training Using CrossEntropyLoss and Validate Label Encoding 1333
13.8.3 Exercise 02 - Implement Regression Training Using MSELoss and Validate Output Shapes 1334
13.8.4 Exercise 03 - Add Accurate Metric Tracking and Ensure Metrics Match Ground Truth Calculations 1335
13.8.5 Exercise 04 - Implement Validation Loop and Detect Overfitting Using Train vs Val Curves 1336
13.8.6 Exercise 05 - Add Early Stopping Logic and Confirm It Prevents Wasteful Training Iterations 1337

13.8.7 Exercise 06 - Add Checkpoint Saving and Restore Training from Checkpoint Without Drift 1338
13.8.8 Exercise 07 - Implement Resume Training with Scheduler and Optimizer State Preservation 1339
13.8.9 Exercise 08 - Add Structured Logging and Verify Logs Are Correct for Every Epoch 1339
13.8.10 Exercise 09 - Debug Training Loop Bugs That Produce Good Loss but Bad Accuracy 1340
13.8.11 Exercise 10 - Train Entire Pipeline on CUDA and Ensure Data, Model, and Loss Are on Same Device 1341
13.8.12 Exercise 11 - Measure GPU Utilization During Training and Identify Under-Utilization Root Causes 1342
13.8.13 Exercise 12 - Implement Mixed Precision Training with GradScaler and Detect Silent Overflow Issues 1343
13.8.14 Exercise 13 - Create a Reproducible Experiment Template with Seeds, Configs, and Version Logs 1344
13.8.15 Exercise 14 - Implement Gradient Accumulation in Training Loop and Validate Equivalent Results 1345
13.8.16 Exercise 15 - Handle NaN Loss During Training by Adding Detection and Immediate Diagnostics 1346
13.8.17 Exercise 16 - Implement Batch-Level Debug Mode to Inspect Failing Samples During Training 1347
13.8.18 Exercise 17 - Benchmark Training Throughput Under Different DataLoader Settings and Pin Memory 1348
13.8.19 Exercise 18 - Validate End-to-End Training Correctness by Overfitting a Tiny Dataset on Purpose 1349
13.8.20 Exercise 19 - Produce a Full Training Loop Debugging and Verification Report 1350
13.9 MLP07 - Regularization, Dropout, and Generalization 1351
13.9.1 Exercise 00 - Demonstrate Overfitting in MLP by Training on Small Dataset Without Regularization 1351
13.9.2 Exercise 01 - Apply L2 Regularization and Measure Its Effect on Train vs Validation Gap 1351
13.9.3 Exercise 02 - Apply Weight Decay Correctly and Compare It Against L2 Regularization Implementation 1352
13.9.4 Exercise 03 - Implement Dropout Layer and Verify Behavior Differs Between Train and Eval Modes 1353
13.9.5 Exercise 04 - Tune Dropout Probability and Identify Underfitting vs Overfitting Patterns 1354
13.9.6 Exercise 05 - Combine Dropout with Weight Decay and Analyze Interaction Effects 1355
13.9.7 Exercise 06 - Implement Early Stopping and Compare Against Regularization Techniques 1356
13.9.8 Exercise 07 - Study the Effect of Model Capacity (Depth/Width) on Generalization Behavior 1357
13.9.9 Exercise 08 - Demonstrate Data Augmentation Concept for Non-Image Tabular Tasks 1357

13.9.10 Exercise 09 - Debug Regularization Misuse That Causes Training Collapse or No Learning 1358
13.9.11 Exercise 10 - Execute Regularized Training on CUDA and Measure Runtime Overhead of Dropout 1359
13.9.12 Exercise 11 - Compare Regularization Outcomes Under Mixed Precision Training 1360
13.9.13 Exercise 12 - Analyze Generalization Under Label Noise and Apply Robust Training Strategies 1361
13.9.14 Exercise 13 - Study the Effect of Input Feature Normalization on Generalization Stability 1362
13.9.15 Exercise 14 - Implement K-Fold Cross Validation and Compare Model Stability Across Folds 1363
13.9.16 Exercise 15 - Diagnose Underfitting and Identify Whether It Comes from Regularization or Capacity Limits 1363
13.9.17 Exercise 16 - Build a Generalization Report with Error Breakdown and Residual Analysis 1364
13.9.18 Exercise 17 - Evaluate Robustness Under Distribution Shift and Document Observed Failure Patterns 1365
13.9.19 Exercise 18 - Implement Model Ensembling of Multiple MLP Runs and Compare Generalization Gains 1366
13.9.20 Exercise 19 - Write a Practical Regularization Strategy Guide for Real Training Projects 1367
13.10MLP08 - Scaling, Profiling, and Production Readiness 1368
13.10.1 Exercise 00 - Profile MLP Training Performance and Identify Whether Compute or Data Is the Bottleneck 1368
13.10.2 Exercise 01 - Measure DataLoader Impact Using num_workers and Pin Memory Configuration 1369
13.10.3 Exercise 02 - Implement Gradient Accumulation for Memory-Constrained GPUs and Validate Equivalence 1370
13.10.4 Exercise 03 - Implement Mixed Precision Training and Quantify Speed vs Stability Tradeoffs 1371
13.10.5 Exercise 04 - Compare Different Batch Sizes and Identify the Throughput-Optimal Configuration 1372
13.10.6 Exercise 05 - Monitor GPU Utilization and Fix Under-Utilization Caused by Input Pipeline Issues 1372
13.10.7 Exercise 06 - Benchmark Inference Latency and Throughput for Production Deployment Settings 1373
13.10.8 Exercise 07 - Export MLP Model Using TorchScript or torch.compile and Validate Output Parity 1374
13.10.9 Exercise 08 - Implement Checkpoint Rotation and Storage Strategy for Long Training Runs 1375
13.10.10 Exercise 09 - Detect Memory Leaks and Unexpected GPU Growth Across Long Training Sessions 1376
13.10.11 Exercise 10 - Integrate Model Saving into Object Storage Workflow and Validate Retrieval Correctness 1377
13.10.12 Exercise 11 - Build a Minimal Inference API Wrapper and Validate Input Validation and Error Handling 1378

13.10.13 Exercise 12 - Implement Deterministic Training Controls and Explain Remaining Sources of Non-Determinism 1379
13.10.14 Exercise 13 - Add Full Experiment Tracking and Ensure Reproducibility Across Machines and GPUs 1380
13.10.15 Exercise 14 - Stress Test Training at Scale with Large Synthetic Dataset and Monitor System Limits 1381
13.10.16 Exercise 15 - Validate Correctness by Intentionally Overfitting Tiny Dataset and Confirming Perfect Fit 1382
13.10.17 Exercise 16 - Implement Regression Tests That Detect Training Quality Degradation After Refactors 1382
13.10.18 Exercise 17 - Build a Production Readiness Checklist Covering Performance, Stability, and Monitoring 1383
13.10.19 Exercise 18 - Simulate Failure Recovery by Resuming from Checkpoint After Forced Interruption 1384
13.10.20 Exercise 19 - Produce a Final End-to-End MLP Engineering Report with Lessons Learned and Best Practices 1385

14 CNN (Convolutional Neural Networks) 1387
14.1 Introduction 1387
14.1.1 What will the book teach you 1387
14.1.2 Why this book comes after the MLP 1389
14.1.3 How to study this book 1389
14.1.4 The Chapters (CNN00 - CNN08) included in this book 1390
14.2 CNN00 - Convolution Concept and Intuition 1390
14.2.1 Exercise 00 - Understand Convolution as Local Pattern Detection Instead of Global Fully-Connected Mixing 1390
14.2.2 Exercise 01 - Implement a Simple 2D Convolution Using Explicit Nested Loops for Full Transparency 1391
14.2.3 Exercise 02 - Implement Cross-Correlation and Explain Why Deep Learning Calls It Convolution 1392
14.2.4 Exercise 03 - Visualize How a Kernel Slides Over an Image and Produces a Feature Map 1393
14.2.5 Exercise 04 - Verify Convolution Output Shape by Manual Calculation Using Kernel Size and Stride 1394
14.2.6 Exercise 05 - Compare Different Kernel Sizes and Observe How Receptive Field Changes 1395
14.2.7 Exercise 06 - Demonstrate the Effect of Stride on Downsampling and Information Loss 1396
14.2.8 Exercise 07 - Demonstrate the Effect of Padding on Border Information Preservation and Output Size 1397

14.2.9 Exercise 08 - Implement Convolution with torch.nn.functional.conv2d and Validate It Against Manual Output 1398
14.2.10 Exercise 09 - Create Multiple Filters and Observe How Each Detects Different Visual Patterns 1399
14.2.11 Exercise 10 - Move Convolution to CUDA and Confirm Numerical Parity with CPU Results 1400
14.2.12 Exercise 11 - Benchmark CPU vs GPU Convolution Runtime for Small and Large Input Tensors 1401
14.2.13 Exercise 12 - Compare float32 and float16 Convolution Outputs and Detect Precision-Sensitive Cases 1402
14.2.14 Exercise 13 - Understand Channels-First Format (N,C,H,W) and Debug Common Shape Mistakes 1402
14.2.15 Exercise 14 - Implement 1x1 Convolution and Explain Channel Mixing Without Spatial Mixing 1403
14.2.16 Exercise 15 - Demonstrate Convolution as Matrix Multiplication with im2col on a Tiny Example 1404
14.2.17 Exercise 16 - Stress Test Convolution with Large Batches and Monitor GPU Memory Consumption 1405
14.2.18 Exercise 17 - Debug Device Mismatch and Non-Contiguous Tensor Issues in Convolution Pipelines 1406
14.2.19 Exercise 18 - Validate Determinism Settings and Observe When CUDA Convolution Becomes Non-Deterministic 1407
14.2.20 Exercise 19 - Write a Conceptual Report Explaining Why Convolution Is the Core Inductive Bias for Vision 1408
14.3 CNN01 - Feature Maps, Filters, and Receptive Fields 1409
14.3.1 Exercise 00 - Compute Feature Map Dimensions for Multiple Conv Layers Using Manual Step-by-Step Formula 1409
14.3.2 Exercise 01 - Visualize How Receptive Field Grows with Depth Using a Simple Layer Stack 1410
14.3.3 Exercise 02 - Show How Increasing Channels Raises Representation Capacity Without Increasing Resolution 1411
14.3.4 Exercise 03 - Implement Multi-Channel Convolution and Explain the Channel Summation Mechanism Clearly 1412

14.3.5 Exercise 04 - Verify Kernel Tensor Shape (out_channels,in_channels,kH,kW) and Debug Wrong Ordering Bugs 1413
14.3.6 Exercise 05 - Compare Edge-Detector Kernels on Synthetic Pattern Images and Observe Their Responses 1414
14.3.7 Exercise 06 - Create Custom Kernels to Detect Horizontal, Vertical, and Diagonal Structures 1415
14.3.8 Exercise 07 - Demonstrate Translation Equivariance by Shifting Input and Observing Shifted Feature Maps 1416
14.3.9 Exercise 08 - Demonstrate When Translation Equivariance Breaks Due to Padding or Stride Choices 1417
14.3.10 Exercise 09 - Analyze How Stride Impacts Aliasing and Feature Map Quality Using Visual Examples 1417
14.3.11 Exercise 10 - Run Feature Map Extraction on CUDA and Verify Output Shape and Device Placement 1418
14.3.12 Exercise 11 - Benchmark Different Kernel Sizes and Strides on GPU to Measure Real Performance Differences 1419
14.3.13 Exercise 12 - Investigate Numerical Differences Between CPU and GPU Feature Maps Under Various Seeds 1420
14.3.14 Exercise 13 - Implement Depthwise Convolution Manually and Explain How It Differs from Standard Convolution 1421
14.3.15 Exercise 14 - Implement Grouped Convolution and Prove It Reduces Parameters While Preserving Some Representation Power 1422
14.3.16 Exercise 15 - Compare Standard, Depthwise, and Grouped Convolutions in Parameters, Speed, and Feature Quality 1423
14.3.17 Exercise 16 - Detect and Fix Shape Errors When Switching Between RGB and Grayscale Inputs 1424
14.3.18 Exercise 17 - Stress Test a Very Deep Conv Stack and Monitor Activation Magnitudes for Stability Issues 1425
14.3.19 Exercise 18 - Build a Receptive Field Calculator and Validate It Against Empirical Behavior 1426
14.3.20 Exercise 19 - Produce a Receptive Field and Feature Map Interpretation Guide for Future Debugging 1427
14.4 CNN02 - Pooling, Downsampling, and Invariance 1428

14.4.1 Exercise 00 - Implement Max Pooling Manually and Verify It Matches torch.nn.functional.max_pool2d 1428
14.4.2 Exercise 01 - Implement Average Pooling and Compare Its Behavior to Max Pooling on the Same Input 1429
14.4.3 Exercise 02 - Explain and Demonstrate Why Pooling Introduces Local Translation Invariance 1430
14.4.4 Exercise 03 - Compare Pooling and Strided Convolution for Downsampling and Measure Quality Differences 1431
14.4.5 Exercise 04 - Analyze Pooling Window Size, Information Loss, and Classification Performance 1432
14.4.6 Exercise 05 - Visualize Aliasing from Aggressive Downsampling Without Anti-Aliasing 1433
14.4.7 Exercise 06 - Implement Global Average Pooling and Explain Its Role in Modern CNN Classifier Heads 1434
14.4.8 Exercise 07 - Demonstrate Adaptive Pooling with Variable Input Image Sizes 1435
14.4.9 Exercise 08 - Compare Feature Map Statistics Before and After Pooling to Understand Signal Compression 1436
14.4.10 Exercise 09 - Debug Output Shape Mistakes from Pooling and Stride Interactions in Complex Networks 1437
14.4.11 Exercise 10 - Run Pooling on CUDA and Confirm Correctness and Speed Benefits 1438
14.4.12 Exercise 11 - Benchmark Pooling and Strided Convolution Runtime and GPU Memory Usage 1439
14.4.13 Exercise 12 - Investigate Pooling Precision Under float16 Mixed Precision Settings 1440
14.4.14 Exercise 13 - Implement Anti-Aliased Pooling and Compare It with Standard Pooling 1441
14.4.15 Exercise 14 - Implement Blur Pooling and Measure Its Effect on Robustness 1441
14.4.16 Exercise 15 - Analyze When Pooling Helps or Hurts Based on Dataset Structure 1442
14.4.17 Exercise 16 - Stress Test Deep Downsampling Stacks and Detect Feature Collapse or Excessive Information Loss 1443
14.4.18 Exercise 17 - Build a Shape-Tracking Utility for Spatial Changes Through a CNN Pipeline 1444
14.4.19 Exercise 18 - Create a Downsampling Strategy Comparison Report Using Multiple Configurations 1445
14.4.20 Exercise 19 - Write a Practical Guide for Selecting Pooling, Stride, or Global Pooling in Real CNNs 1446
14.5 CNN03 - CNN Forward Architecture and Implementation 1447

14.5.1 Exercise 00 - Build a Minimal CNN Classifier and Verify Forward Pass Output Dimensions 1448
14.5.2 Exercise 01 - Implement CNN with nn.Sequential and Compare It to Explicit Forward Function Style 1449
14.5.3 Exercise 02 - Add ReLU Between Conv Layers and Confirm Non-Linearity Changes Representation 1450
14.5.4 Exercise 03 - Implement Flatten Correctly and Debug Batch Dimension Mistakes 1450
14.5.5 Exercise 04 - Replace Flatten and Fully Connected Head with Global Average Pooling and Compare Cost 1451
14.5.6 Exercise 05 - Add Dropout to the CNN Classifier Head and Verify Train and Eval Behavior 1452
14.5.7 Exercise 06 - Implement Batch Normalization and Verify Running Statistics and Mode Switching 1453
14.5.8 Exercise 07 - Add a Residual-Like Skip Connection and Verify Shape Compatibility 1454
14.5.9 Exercise 08 - Inspect Intermediate Feature Maps with Forward Hooks and Save Them for Debugging 1455
14.5.10 Exercise 09 - Debug CNN Forward Failures: Shape Mismatch, NaNs, and Wrong Channel Ordering 1456
14.5.11 Exercise 10 - Move a Full CNN Forward Pass to CUDA and Confirm Shared Device Placement 1457
14.5.12 Exercise 11 - Benchmark Forward Throughput and Identify Expensive Layers on GPU 1458
14.5.13 Exercise 12 - Compare float32 and Mixed Precision Forward Outputs and Detect Sensitive Layers 1459
14.5.14 Exercise 13 - Implement a Configurable CNN Builder from a Layer Specification List 1460
14.5.15 Exercise 14 - Validate Parameter Count and Memory Footprint for Multiple CNN Architecture Variants 1461
14.5.16 Exercise 15 - Test CNN Forward Robustness Across Input Sizes with Adaptive Pooling 1462
14.5.17 Exercise 16 - Create Unit Tests for Output Shapes and Dtypes at Every CNN Layer 1463
14.5.18 Exercise 17 - Stress Test CNN Forward with Large Batches and Monitor GPU Memory Peaks 1464

14.5.19 Exercise 18 - Implement a Minimal CNN Inference Wrapper and Validate Preprocessing Requirements 1465
14.5.20 Exercise 19 - Produce a CNN Forward Design Report Explaining Architecture Choices and Their Effects 1466
14.6 CNN04 - CNN Training Loop, Loss, and Backpropagation 1466
14.6.1 Exercise 00 - Build a Complete CNN Training Loop with Correct Train/Eval Modes and Loss Computation 1467
14.6.2 Exercise 01 - Overfit a Tiny Dataset to Validate End-to-End CNN Correctness 1468
14.6.3 Exercise 02 - Compare CrossEntropyLoss and BCEWithLogitsLoss for Binary Image Classification 1468
14.6.4 Exercise 03 - Inspect Layer-Wise Gradients and Detect Vanishing or Exploding Patterns 1469
14.6.5 Exercise 04 - Validate Backpropagation with Finite Differences on a Tiny CNN 1470
14.6.6 Exercise 05 - Debug NaN Loss by Checking Inputs, Initialization, and Learning Rate 1471
14.6.7 Exercise 06 - Implement Gradient Clipping and Measure Its Stabilizing Effect 1472
14.6.8 Exercise 07 - Implement Learning Rate Scheduling and Verify Step Timing and Outcome 1473
14.6.9 Exercise 08 - Compare SGD and Adam on the Same CNN and Explain Outcome Differences 1474
14.6.10 Exercise 09 - Debug Training Where Loss Improves but Accuracy Does Not Improve 1474
14.6.11 Exercise 10 - Train CNN on CUDA with Correct DataLoader Transfer and Pinned Memory 1475
14.6.12 Exercise 11 - Profile GPU Utilization and Fix Training Under-Utilization Bottlenecks 1476
14.6.13 Exercise 12 - Implement Mixed Precision CNN Training and Detect Silent Overflow or Underflow 1477
14.6.14 Exercise 13 - Validate Determinism Options and Explain CNN Training Non-Determinism 1478
14.6.15 Exercise 14 - Implement Checkpoint Resume with Optimizer and Scheduler State Preservation 1479
14.6.16 Exercise 15 - Benchmark Training Throughput Across Batch Sizes and Find the Best Trade-Off 1480

14.6.17 Exercise 16 - Implement Gradient Accumulation for Memory-Constrained Large-Batch Training 1481
14.6.18 Exercise 17 - Build Debug Mode That Saves Misclassified Images and Confidence Scores 1482
14.6.19 Exercise 18 - Validate Training by Reaching Near-Perfect Accuracy on a Simple Synthetic Task 1483
14.6.20 Exercise 19 - Produce a Full CNN Training Debugging Playbook Covering Common Failure Scenarios 1483
14.7 CNN05 - Data Pipeline and Augmentation for Vision 1484
14.7.1 Exercise 00 - Implement Image Dataset Loading with Correct Normalization and Tensor Conversion 1484
14.7.2 Exercise 01 - Validate Channel Ordering and Normalization by Visualizing a Preprocessed Batch 1485
14.7.3 Exercise 02 - Implement Train and Test Splits and Confirm No Data Leakage 1486
14.7.4 Exercise 03 - Implement Random Crop and Horizontal Flip and Measure Generalization Impact 1487
14.7.5 Exercise 04 - Implement Color Jitter and Analyze When It Helps or Hurts Specific Datasets 1488
14.7.6 Exercise 05 - Implement Random Rotation and Analyze Its Effect on Model Invariance 1488
14.7.7 Exercise 06 - Implement Random Erasing or Cutout and Evaluate Its Regularization Effect 1489
14.7.8 Exercise 07 - Implement MixUp or CutMix and Compare with Standard Augmentation 1490
14.7.9 Exercise 08 - Optimize DataLoader with num_workers and pin_memory and Benchmark It 1491
14.7.10 Exercise 09 - Debug Data Pipeline Bugs Causing Wrong Labels or Broken Image Tensors 1491
14.7.11 Exercise 10 - Move the Data Pipeline to CUDA Efficiently and Measure Transfer Overhead 1492
14.7.12 Exercise 11 - Implement Prefetching and Compare It with a Baseline DataLoader 1493
14.7.13 Exercise 12 - Analyze Image Resolution Trade-Offs in Accuracy, Speed, and GPU Memory 1494
14.7.14 Exercise 13 - Detect Augmentation Bugs That Destroy Class Semantics or Introduce Label Noise 1494
14.7.15 Exercise 14 - Implement Balanced Sampling for Imbalanced Image Datasets 1495

14.7.16 Exercise 15 - Build an Augmentation Ablation Study and Identify What Actually Helps 1496
14.7.17 Exercise 16 - Compare Per-Dataset and Per-Image Normalization Strategies 1497
14.7.18 Exercise 17 - Create a Data Quality Report for Corruption, Duplicates, and Outliers 1498
14.7.19 Exercise 18 - Stress Test DataLoader Throughput and Debug Crashes or Deadlocks 1498
14.7.20 Exercise 19 - Produce a Practical Vision Data Pipeline Checklist for Training-Ready Datasets 1499
14.8 CNN06 - Regularization, Normalization, and Robustness 1500
14.8.1 Exercise 00 - Demonstrate CNN Overfitting Without Regularization and Observe the Generalization Gap 1500
14.8.2 Exercise 01 - Apply Weight Decay and Quantify Training Stability and Validation Performance 1501
14.8.3 Exercise 02 - Compare Dropout in the Classifier Head and Convolutional Blocks 1502
14.8.4 Exercise 03 - Implement BatchNorm and Explain Its Effect on Optimization and Gradient Flow 1502
14.8.5 Exercise 04 - Compare BatchNorm and LayerNorm in CNN Settings and Explain When Each Fits 1503
14.8.6 Exercise 05 - Diagnose Instability from Wrong Normalization Placement and Fix the Root Cause 1504
14.8.7 Exercise 06 - Evaluate Robustness to Input Noise and Blurring and Compare Degradation Patterns 1504
14.8.8 Exercise 07 - Evaluate Random Occlusion and Test Use of Global Context 1505
14.8.9 Exercise 08 - Implement Label Smoothing and Measure Overconfidence and Calibration 1506
14.8.10 Exercise 09 - Debug Regularization Misconfiguration Causing Underfitting or Failure to Learn 1506
14.8.11 Exercise 10 - Train Regularized CNN on CUDA and Measure Technique Runtime and Memory Overhead 1507
14.8.12 Exercise 11 - Compare Mixed Precision Stability With and Without Normalization Layers 1508
14.8.13 Exercise 12 - Perform a Regularization Ablation Study and Identify What Actually Helps 1508

14.8.14 Exercise 13 - Detect Distribution Shift Under Changed Lighting or Color and Measure Generalization Failure 1509
14.8.15 Exercise 14 - Implement Test-Time Augmentation and Decide Whether It Adds Robustness or Noise 1510
14.8.16 Exercise 15 - Evaluate Adversarial-Like Perturbations and Document CNN Sensitivity Patterns 1510
14.8.17 Exercise 16 - Build a Robustness Report Grouping Misclassifications by Failure Type and Confidence 1511
14.8.18 Exercise 17 - Analyze Calibration Error and Build a Reliability Diagram for Overconfidence 1512
14.8.19 Exercise 18 - Implement CNN Ensembling and Quantify Accuracy and Robustness Improvements 1512
14.8.20 Exercise 19 - Write a Practical CNN Regularization Strategy Guide for Training and Deployment 1513
14.9 CNN07 - Modern CNN Blocks and Architectural Patterns 1514
14.9.1 Exercise 00 - Implement a VGG-Style Block and Analyze Parameters and Feature Behavior 1514
14.9.2 Exercise 01 - Implement a Residual Block and Explain Improved Gradient Flow 1514
14.9.3 Exercise 02 - Validate Residual Addition Shapes and Debug Channel Mismatch Errors 1515
14.9.4 Exercise 03 - Implement a Bottleneck Residual Block and Compare Its Efficiency with a Standard Block 1516
14.9.5 Exercise 04 - Implement Depthwise Separable Convolution and Quantify Parameter and FLOPs Reduction 1516
14.9.6 Exercise 05 - Implement a MobileNetV2-Style Inverted Residual and Explain the Expansion Ratio 1517
14.9.7 Exercise 06 - Implement Squeeze-and-Excitation Attention and Measure Feature Selectivity 1517
14.9.8 Exercise 07 - Compare Standard, Depthwise Separable, and Grouped Convolution Speed on GPU 1518
14.9.9 Exercise 08 - Build a Small CNN with Modern Blocks and Validate Its Forward Pass 1519
14.9.10 Exercise 09 - Debug Stride Placement and Residual Path Misalignment Bugs 1519
14.9.11 Exercise 10 - Train a Modern-Block CNN on CUDA and Benchmark It Against a Simple Baseline 1520
14.9.12 Exercise 11 - Profile GPU Utilization and Identify Block-Level Bottlenecks 1520

14.9.13 Exercise 12 - Compare Memory Usage of VGG-Style and Residual Architectures 1521
14.9.14 Exercise 13 - Implement Stochastic Depth and Observe Regularization in Deep Residual Networks 1522
14.9.15 Exercise 14 - Implement DropPath and Verify It Activates Only During Training 1522
14.9.16 Exercise 15 - Compare Accuracy and Robustness Across Architectural Pattern Choices 1523
14.9.17 Exercise 16 - Build an Ablation Study Removing One Block Type at a Time and Measure Impact 1523
14.9.18 Exercise 17 - Create an Architecture Summary with Parameters, FLOPs, Speed, and Accuracy 1524
14.9.19 Exercise 18 - Stress Test a Deep Modern CNN for NaNs, Instability, and Memory Blowups 1525
14.9.20 Exercise 19 - Produce a Modern CNN Architecture Guide for Efficient Vision Models 1525
14.10CNN08 - Deployment, Export, and Production Readiness 1526
14.10.1 Exercise 00 - Benchmark CNN Inference Latency and Throughput Under Production-Like Batch and Precision Settings . . .1526
14.10.2 Exercise 01 - Export CNN Using TorchScript and Validate Output Parity Across Exported and Native Models 1527
14.10.3 Exercise 02 - Export CNN Using ONNX and Validate Correctness Across Different Runtime Backends 1527
14.10.4 Exercise 03 - Use torch.compile for CNN and Measure Speed Gains vs Compilation Overhead 1528
14.10.5 Exercise 04 - Implement Quantization Conceptually and Measure Accuracy Drop vs Performance Gain 1529
14.10.6 Exercise 05 - Implement Batch Inference Pipeline with Efficient Transfer and Memory Reuse 1529
14.10.7 Exercise 06 - Build a Minimal Inference Service Wrapper with Input Validation and Error Handling 1530
14.10.8 Exercise 07 - Implement Model Versioning and Backward-Compatible Checkpoint Loading 1531
14.10.9 Exercise 08 - Add Monitoring Hooks for Confidence Drift and Data Distribution Shift 1531
14.10.10 Exercise 09 - Build Batch Failure Handling That Skips Corrupt Inputs Without Breaking the Pipeline 1532

14.10.11 Exercise 10 - Run Deployment-Oriented CUDA Inference and Measure Real GPU Utilization Under Load 1533
14.10.12 Exercise 11 - Compare float32, float16, and int8 Inference Accuracy and Performance Trade-Offs 1533
14.10.13 Exercise 12 - Build a Reproducible Deployment Benchmark Suite with Fixed Inputs and Seeds 1534
14.10.14 Exercise 13 - Implement Checkpoint Integrity Verification and Detect Corrupted Weight Files Early 1535
14.10.15 Exercise 14 - Simulate NaN Production Incident and Build an Automatic Rollback Strategy 1536
14.10.16 Exercise 15 - Validate Training–Serving Preprocessing Parity and Detect Silent Mismatch Bugs 1536
14.10.17 Exercise 16 - Implement A/B Testing for Two CNN Models on the Same Input Stream 1537
14.10.18 Exercise 17 - Create an End-to-End Production Readiness Checklist for Performance, Monitoring, and Stability 1538
14.10.19 Exercise 18 - Stress Test Inference Service for Memory Leaks and Tail Latency Spikes 1538
14.10.20 Exercise 19 - Produce a Final CNN Engineering Report Covering Architecture, Training, Deployment, and Failures 1539

15 RNN (Recurrent Neural Networks) 1541
15.1 Introduction 1541
15.1.1 What will the book teach you 1541
15.1.2 How to study this book 1545
15.1.3 The Chapters (RNN00 - RNN08) included in this book 1545
15.2 RNN00 - Sequence Modeling and Core RNN Intuition 1546
15.2.1 Exercise 00 - Understand Why Sequence Data Requires Memory Beyond Standard Feed-Forward Networks 1546
15.2.2 Exercise 01 - Represent a Sequence as Time Steps and Define the Input–Output Mapping Precisely 1546
15.2.3 Exercise 02 - Implement a Simple RNN Forward Pass Using Pure Python Recurrence 1547
15.2.4 Exercise 03 - Translate the RNN Recurrence Equation into PyTorch Tensor Operations 1548
15.2.5 Exercise 04 - Explain Hidden State as Compact Memory and Test Its Effect on Output Prediction 1549

15.2.6 Exercise 05 - Demonstrate Weight Sharing Across Time Steps and Explain Why It Matters 1549
15.2.7 Exercise 06 - Build a Tiny Synthetic Sequence Task Where History Is Required to Predict Correctly 1550
15.2.8 Exercise 07 - Compare One-Step Prediction vs Full-Sequence Prediction in a Controlled Experiment 1551
15.2.9 Exercise 08 - Implement Many-to-One RNN for Sequence Classification and Validate Output Shapes 1551
15.2.10 Exercise 09 - Implement Many-to-Many RNN for Sequence Labeling and Validate Temporal Alignment 1552
15.2.11 Exercise 10 - Move RNN Forward Computation to CUDA and Validate Output Parity with CPU Results 1553
15.2.12 Exercise 11 - Benchmark RNN Forward Speed on CPU vs GPU for Different Sequence Lengths 1553
15.2.13 Exercise 12 - Compare float32 vs float16 RNN Forward Outputs and Identify Precision-Sensitive Cases 1554
15.2.14 Exercise 13 - Understand Tensor Shapes for RNN Inputs (seq_len, batch, input_size) and Debug Mistakes 1555
15.2.15 Exercise 14 - Implement RNN with batch_first Option and Verify Shape Conversions Correctly 1556
15.2.16 Exercise 15 - Implement a Manual Unroll of RNN Over Time and Compare to torch.nn.RNN Behavior 1556
15.2.17 Exercise 16 - Stress Test RNN Forward with Long Sequences and Monitor GPU Memory and Runtime 1557
15.2.18 Exercise 17 - Debug Device Mismatch and Hidden-State Placement Errors in CUDA RNN Pipelines 1558
15.2.19 Exercise 18 - Validate Determinism Settings and Observe When CUDA RNN Becomes Non-Deterministic 1559
15.2.20 Exercise 19 - Write a Conceptual Report Explaining RNN Memory, Weight Sharing, and Temporal Computation 1559
15.3 RNN01 - Hidden State Dynamics and Temporal Behavior 1560
15.3.1 Exercise 00 - Visualize Hidden State Evolution Over Time for a Simple Repeating Pattern Sequence 1560
15.3.2 Exercise 01 - Analyze How Initial Hidden State Choice Affects Early Predictions and Convergence 1561

15.3.3 Exercise 02 - Implement Learnable Initial Hidden State and Compare Against Zero Initialization 1562
15.3.4 Exercise 03 - Demonstrate Information Decay Over Long Sequences Using Hidden State Probing 1562
15.3.5 Exercise 04 - Compare Single-Layer vs Multi-Layer RNN Hidden State Representations 1563
15.3.6 Exercise 05 - Implement Stacked RNN and Confirm Correct Hidden State Shape at Each Layer 1564
15.3.7 Exercise 06 - Investigate the Effect of Hidden Size on Memory Capacity and Sequence Accuracy 1565
15.3.8 Exercise 07 - Implement Bidirectional RNN and Verify Forward and Backward State Concatenation 1565
15.3.9 Exercise 08 - Compare Bidirectional vs Unidirectional RNN for a Sequence Labeling Task 1566
15.3.10 Exercise 09 - Debug Hidden State Shape Bugs in Multi-Layer and Bidirectional RNN Setups 1567
15.3.11 Exercise 10 - Move Hidden State Analysis to CUDA Without Excess Host–Device Transfers 1568
15.3.12 Exercise 11 - Benchmark Hidden State Computation Cost for Increasing Hidden Size on GPU 1568
15.3.13 Exercise 12 - Compare Floating-Point Precision Effects on Hidden State Stability and Temporal Drift 1569
15.3.14 Exercise 13 - Use Forward Hooks to Save Hidden States and Build a Visualization Pipeline 1570
15.3.15 Exercise 14 - Probe Hidden States to Detect Whether Past Information Is Encoded or Forgotten 1571
15.3.16 Exercise 15 - Implement Teacher Forcing Conceptually and Observe Its Impact on Sequence Generation 1571
15.3.17 Exercise 16 - Investigate Exposure Bias in Autoregressive Sequence Generation with RNN 1572
15.3.18 Exercise 17 - Stress Test Very Long Sequences and Identify Practical Limits for Vanilla RNN Memory 1573
15.3.19 Exercise 18 - Build a Hidden State Debugging Checklist for Shapes, Devices, and Temporal Alignment 1574
15.3.20 Exercise 19 - Produce a Detailed Report Explaining Hidden State Dynamics and Their Failure Modes 1575
15.4 RNN02 - Backpropagation Through Time (BPTT) 1575

15.4.1 Exercise 00 - Define Backpropagation Through Time and Explain How It Extends Standard Backprop 1575
15.4.2 Exercise 01 - Implement an RNN Training Loop and Confirm Gradients Flow Through Time Steps 1576
15.4.3 Exercise 02 - Inspect Gradient Norms Across Time and Detect Vanishing Gradient Behavior 1577
15.4.4 Exercise 03 - Inspect Gradient Norms Across Time and Detect Exploding Gradient Behavior 1578
15.4.5 Exercise 04 - Apply Gradient Clipping and Measure Its Stabilization Effect Quantitatively 1579
15.4.6 Exercise 05 - Implement Truncated BPTT and Compare Memory Usage and Accuracy Trade-Offs 1579
15.4.7 Exercise 06 - Compare Full BPTT vs Truncated BPTT on the Same Synthetic Long-Dependency Task 1580
15.4.8 Exercise 07 - Debug Silent Errors Caused by Hidden State Detaching at the Wrong Time 1581
15.4.9 Exercise 08 - Perform Finite Difference Gradient Checking for a Tiny RNN to Validate Backward Correctness 1582
15.4.10 Exercise 09 - Demonstrate Why Vanilla RNN Struggles with Long-Term Dependencies in Practice 1583
15.4.11 Exercise 10 - Execute BPTT on CUDA and Validate Gradient Consistency with CPU Under Controlled Settings 1583
15.4.12 Exercise 11 - Profile GPU Memory During BPTT and Identify Graph Retention Bottlenecks 1584
15.4.13 Exercise 12 - Compare float32 vs Mixed-Precision RNN Training and Detect Overflow or Underflow 1585
15.4.14 Exercise 13 - Use Autograd Hooks to Log Gradient Norms at Each Time Step and Summarize Results 1586
15.4.15 Exercise 14 - Demonstrate Gradient Accumulation When Forgetting zero_grad in an RNN Training Loop 1587
15.4.16 Exercise 15 - Analyze Batch Size Impact on Gradient Noise and Convergence in RNN Training 1587
15.4.17 Exercise 16 - Debug NaN Loss by Checking Learning Rate, Initialization, and Sequence Length 1588

15.4.18 Exercise 17 - Stress Test BPTT with Long Sequences and Large Hidden Size to Observe Practical Limits 1589
15.4.19 Exercise 18 - Build a BPTT Debugging Checklist for Detach Strategy, Truncation, and Gradient Clipping 1590
15.4.20 Exercise 19 - Write a Full BPTT Engineering Report Explaining Experimental Failures and Fixes 1591
15.5 RNN03 - Sequence Data Preparation and Batching 1592
15.5.1 Exercise 00 - Convert Raw Sequential Data into Sliding Windows and Validate Correct Input–Target Alignment 1592
15.5.2 Exercise 01 - Implement a Custom Dataset for Sequence Pairs and Validate getitem Correctness 1593
15.5.3 Exercise 02 - Implement Padding for Variable-Length Sequences and Explain Why Padding Is Needed 1594
15.5.4 Exercise 03 - Implement Packed Sequences with pack_padded_sequence and Validate Correctness 1595
15.5.5 Exercise 04 - Debug Packing Bugs from Wrong Length Sorting and Misaligned Outputs 1595
15.5.6 Exercise 05 - Compare Padded vs Packed Training and Measure Speed and Memory Differences 1596
15.5.7 Exercise 06 - Build a Collate Function That Produces Batch-First Tensors with Correct Padding Masks 1597
15.5.8 Exercise 07 - Validate Masking So Loss Ignores Padding Time Steps Correctly 1598
15.5.9 Exercise 08 - Create a Variable-Length Synthetic Dataset and Verify the Training Pipeline Handles It 1599
15.5.10 Exercise 09 - Debug Training Instability from Incorrect Padding Masks or Wrong Loss Reduction 1600
15.5.11 Exercise 10 - Move the Sequence Batching Pipeline to CUDA Efficiently and Minimize Host-to-Device Transfers 1601
15.5.12 Exercise 11 - Benchmark DataLoader Throughput with num_workers and pin_memory Settings 1602
15.5.13 Exercise 12 - Implement Gradient Accumulation for Long Sequences Under Limited GPU Memory 1603
15.5.14 Exercise 13 - Validate Reproducibility of Sequence Batching Under Random Shuffling and Fixed Seeds 1604
15.5.15 Exercise 14 - Implement a Train–Validation Split Without Leakage in Sequential Time-Series Data 1605

15.5.16 Exercise 15 - Implement Teacher-Forcing Dataset Format for Sequence-to-Sequence Tasks 1606
15.5.17 Exercise 16 - Implement Stateful RNN Batching and Decide When to Reset Hidden States Correctly 1607
15.5.18 Exercise 17 - Detect and Fix Alignment Bugs That Make an RNN Learn the Wrong Target Shift 1607
15.5.19 Exercise 18 - Build a Sequence Data Pipeline Checklist Covering Padding, Packing, and Masking 1608
15.5.20 Exercise 19 - Produce a Full Sequence Data Preparation Report with Verified Edge Cases 1609
15.6 RNN04 - Language Modeling with Vanilla RNN 1610
15.6.1 Exercise 00 - Implement Character-Level Tokenization and Build a Vocabulary with Correct Index Mapping 1610
15.6.2 Exercise 01 - Build a Simple Character-Level RNN Language Model and Verify Output Logit Shapes 1611
15.6.3 Exercise 02 - Implement Cross-Entropy Loss for Next-Character Prediction and Validate Target Alignment 1612
15.6.4 Exercise 03 - Train a Tiny RNN Language Model Until It Overfits a Small Text Sample on Purpose 1613
15.6.5 Exercise 04 - Generate Text Autoregressively and Observe Repetition and Instability Patterns 1614
15.6.6 Exercise 05 - Implement Temperature Sampling and Analyze Diversity vs Coherence 1615
15.6.7 Exercise 06 - Implement Top-k Sampling and Compare with Temperature-Only Sampling 1616
15.6.8 Exercise 07 - Debug Generation Bugs Caused by Incorrect Hidden-State Reset Behavior 1617
15.6.9 Exercise 08 - Compare Sequence Length and Its Effect on Learning Short vs Long Patterns 1618
15.6.10 Exercise 09 - Evaluate an RNN Language Model with Perplexity and Interpret What the Number Means 1619
15.6.11 Exercise 10 - Train a Character RNN on CUDA and Measure Speed Improvement over a CPU Baseline 1620
15.6.12 Exercise 11 - Profile GPU Utilization and Fix Language-Model Input Pipeline Bottlenecks 1620

15.6.13 Exercise 12 - Implement Mixed-Precision Training and Detect Instability in Long-Sequence Language Modeling 1621
15.6.14 Exercise 13 - Compare Vanilla RNN with GRU or LSTM and Document Why Vanilla RNN Fails 1622
15.6.15 Exercise 14 - Implement Truncated BPTT for Language Modeling and Quantify Memory Savings 1623
15.6.16 Exercise 15 - Debug Training Divergence through Learning Rate, Clipping, and Initialization 1624
15.6.17 Exercise 16 - Build a Generation Evaluation Script for Repetition and Novelty Metrics 1625
15.6.18 Exercise 17 - Create a Controlled Long-Term Dependency Failure Experiment for Vanilla RNN 1626
15.6.19 Exercise 18 - Stress Test Language Modeling with Longer Text and Identify GPU Memory and Speed Limits 1627
15.6.20 Exercise 19 - Produce a Full Character-Level RNN Language Modeling Report with Failure Analysis and Fixes 1628
15.7 RNN05 - Time Series Forecasting with Vanilla RNN 1629
15.7.1 Exercise 00 - Convert Time Series into Supervised Windows and Verify the Correct Target Horizon 1629
15.7.2 Exercise 01 - Implement Feature Scaling for Time Series and Explain How It Prevents Training Instability 1630
15.7.3 Exercise 02 - Build a Simple RNN Forecaster and Validate Output Shapes for Multi-Step Prediction 1630
15.7.4 Exercise 03 - Compare One-Step and Multi-Step Forecasting and Explain Error Accumulation 1631
15.7.5 Exercise 04 - Train an RNN on Synthetic Seasonal Data and Confirm It Learns Periodicity 1632
15.7.6 Exercise 05 - Implement Autoregressive Multi-Step Forecasting and Observe Drift over Time 1633
15.7.7 Exercise 06 - Implement Teacher Forcing for Forecasting and Compare with Pure Autoregressive Training 1634
15.7.8 Exercise 07 - Evaluate Forecast Accuracy with MAE and MSE and Interpret Their Different Penalties 1635

15.7.9 Exercise 08 - Debug Forecasting Bugs from Wrong Windowing or Misaligned Targets 1636
15.7.10 Exercise 09 - Diagnose Overfitting in Forecasting and Apply Regularization or Early Stopping 1637
15.7.11 Exercise 10 - Train a Time-Series RNN on CUDA and Benchmark End-to-End Throughput Including the Data Pipeline 1638
15.7.12 Exercise 11 - Profile GPU Memory and Runtime for Sequence Length and Hidden-Size Scaling 1639
15.7.13 Exercise 12 - Implement Mixed Precision for Forecasting and Identify Stability Failures 1640
15.7.14 Exercise 13 - Compare Vanilla RNN and LSTM Forecasting and Document Long-Term Dependency Gains 1641
15.7.15 Exercise 14 - Implement Truncated BPTT for Long Forecasting Windows and Quantify Memory Savings 1642
15.7.16 Exercise 15 - Build a Forecast Visualization Tool for Predictions and Ground Truth across Multiple Horizons 1643
15.7.17 Exercise 16 - Diagnose Instability from Seasonal Scale and Fix It with Normalization 1644
15.7.18 Exercise 17 - Create Walk-Forward Validation to Avoid Temporal Leakage in Forecast Evaluation 1645
15.7.19 Exercise 18 - Stress Test the Forecasting Pipeline with Large Data and Identify Practical Limits 1646
15.7.20 Exercise 19 - Produce a Full Time-Series RNN Forecasting Report with Failure Cases and Engineering Fixes 1647
15.8 RNN06 - Regularization and Stabilization for RNNs 1648
15.8.1 Exercise 00 - Demonstrate Overfitting in RNN and Measure Train vs Validation Gap in Sequence Tasks 1648
15.8.2 Exercise 01 - Apply Dropout in RNN and Compare Input Dropout with Dropout Between Recurrent Layers 1649
15.8.3 Exercise 02 - Use Weight Decay and Analyze Its Effect on RNN Stability and Generalization 1650
15.8.4 Exercise 03 - Implement Gradient Clipping and Compare Different Clip Norm Values Experimentally 1650
15.8.5 Exercise 04 - Implement Layer Normalization in RNN and Measure Hidden-State Stability 1651

15.8.6 Exercise 05 - Diagnose NaN Loss and Build Automatic Early-Failure Diagnostics for RNN Training 1652
15.8.7 Exercise 06 - Compare Recurrent Weight Initializations and Observe Training Differences 1653
15.8.8 Exercise 07 - Compare SGD and Adam for RNNs and Explain Optimizer Sensitivity 1654
15.8.9 Exercise 08 - Implement Learning-Rate Scheduling and Confirm Reduced Instability in RNN Training 1655
15.8.10 Exercise 09 - Debug Cases Where Loss Decreases but Sequence Quality Does Not 1656
15.8.11 Exercise 10 - Train a Stabilized RNN on CUDA and Measure Regularization Runtime Overhead 1657
15.8.12 Exercise 11 - Compare Mixed-Precision Stability With and Without Gradient Scaling in RNNs 1658
15.8.13 Exercise 12 - Analyze Generalization Under Label Noise and Sequence-Noise Propagation 1659
15.8.14 Exercise 13 - Perform an Ablation Study to Identify Which Stabilization Methods Actually Help 1660
15.8.15 Exercise 14 - Implement Early Stopping and Compare It with Regularization and Dropout 1661
15.8.16 Exercise 15 - Build a Comprehensive Debugging Checklist for RNN Training Stability 1662
15.8.17 Exercise 16 - Evaluate Robustness Under Time-Series Distribution Shift and Document Failure Modes 1663
15.8.18 Exercise 17 - Build Calibration and Confidence Analysis for RNN Predictions Across Time Steps 1664
15.8.19 Exercise 18 - Stress Test Very Long Sequences and Observe When Regularization Stops Helping 1665
15.8.20 Exercise 19 - Write a Practical RNN Stabilization Strategy Guide for Real Engineering Projects 1666
15.9 RNN07 - Efficiency, Profiling, and CUDA Performance 1666
15.9.1 Exercise 00 - Benchmark Vanilla RNN vs GRU vs LSTM Runtime and Memory to Understand Engineering Tradeoffs 1667
15.9.2 Exercise 01 - Profile RNN Training with PyTorch Profiler and Identify Kernel and Memory Bottlenecks 1668

15.9.3 Exercise 02 - Optimize Data Transfer by Minimizing Host-to-Device Copies in Sequence Training 1669
15.9.4 Exercise 03 - Measure Sequence-Length Impact on Throughput and Find the Practical GPU Limit 1670
15.9.5 Exercise 04 - Compare Packed vs Padded Sequence Performance and Measure Speedup and Memory Changes 1670
15.9.6 Exercise 05 - Evaluate batch_first and Its Effect on Memory Layout and Performance 1671
15.9.7 Exercise 06 - Benchmark Hidden Sizes and Layer Counts and Build a Scaling Curve 1672
15.9.8 Exercise 07 - Diagnose an Under-Utilized GPU and Fix DataLoader or Padding Bottlenecks 1673
15.9.9 Exercise 08 - Compare float32 and Mixed-Precision Speed and Document Stability Constraints 1674
15.9.10 Exercise 09 - Measure Gradient-Clipping Overhead and Decide Whether It Is Worth It 1675
15.9.11 Exercise 10 - Implement Gradient Accumulation and Compare It with True Large-Batch Training 1676
15.9.12 Exercise 11 - Run CUDA Efficiency Tests and Produce Reproducible Benchmark Tables 1677
15.9.13 Exercise 12 - Investigate Non-Determinism in cuDNN RNN Kernels and Document When It Appears 1678
15.9.14 Exercise 13 - Compare Stateful and Stateless RNN Training Efficiency and Explain Hidden-State Reset Costs 1679
15.9.15 Exercise 14 - Implement RNN Checkpointing and Resume and Measure Restart-Time Loss 1680
15.9.16 Exercise 15 - Build a Production RNN Inference Benchmark Across Batch Sizes 1681
15.9.17 Exercise 16 - Export an RNN with TorchScript and Validate Output Parity and Speed 1682
15.9.18 Exercise 17 - Test ONNX Export and Identify Unsupported Operators or Shape Edge Cases 1682
15.9.19 Exercise 18 - Stress Test Long-Sequence Inference and Detect Memory Leaks or Latency Spikes 1683
15.9.20 Exercise 19 - Produce a Complete RNN Engineering Performance Report with Scaling Curves and Best Practices 1684
15.10RNN08 - Transition to Gated RNNs (Why LSTM Exists) 1685

15.10.1 Exercise 00 - Demonstrate Long-Term Dependency Failure in Vanilla RNN with a Controlled Memory Task 1685
15.10.2 Exercise 01 - Explain Vanishing Gradients as Repeated Jacobian Multiplication 1686
15.10.3 Exercise 02 - Implement a Simple GRU Cell Manually and Compare It with Vanilla RNN Behavior 1687
15.10.4 Exercise 03 - Compare Vanilla RNN, GRU, and LSTM on the Same Long-Dependency Task 1688
15.10.5 Exercise 04 - Analyze Hidden-State Updates and Explain Why Gates Preserve Useful Gradients 1689
15.10.6 Exercise 05 - Implement a Minimal LSTM Cell and Verify Gate Shapes and Ranges Step by Step 1690
15.10.7 Exercise 06 - Train GRU and LSTM and Compare Stability, Convergence Speed, and Final Accuracy 1691
15.10.8 Exercise 07 - Debug Gate Misconfiguration Bugs Causing No Learning or Constant Outputs 1692
15.10.9 Exercise 08 - Run GRU and LSTM Experiments on CUDA and Measure Memory and Runtime Tradeoffs 1693
15.10.10 Exercise 09 - Explain Why LSTM and GRU Are Engineering Solutions to Vanilla RNN Limits 1694
15.10.11 Exercise 10 - Ablate the Forget Gate and Observe Memory Collapse . . .1695
15.10.12 Exercise 11 - Find the Sequence Length Where LSTM Clearly Outperforms Vanilla RNN 1696
15.10.13 Exercise 12 - Compare Gated and Vanilla Models Under Distribution Shift 1697
15.10.14 Exercise 13 - Implement Truncated BPTT for GRU and LSTM and Compare Memory Savings with Vanilla RNN 1697
15.10.15 Exercise 14 - Profile RNN and LSTM GPU Utilization and Explain Kernel-Level Differences 1698
15.10.16 Exercise 15 - Build a Checklist for Choosing RNN, GRU, or LSTM in Real Projects 1699
15.10.17 Exercise 16 - Export GRU and LSTM and Compare Deployment Complexity with Vanilla RNN 1700
15.10.18 Exercise 17 - Stress Test Gated Models with Long Sequences and Large Hidden Sizes 1701

15.10.19 Exercise 18 - Create an Ablation Report Quantifying What Gates Improve and Why 1702
15.10.20 Exercise 19 - Produce the Final RNN Track Report with Concepts, Experiments, Failures, and Next Steps 1703

16 LSTM (Long Short-Term Memory) 1705
16.1 Introduction 1705
16.1.1 What will the book teach you 1705
16.2 LSTM00 - LSTM Intuition and Why Gates Exist 1709
16.2.1 Exercise 00 - Explain Why Vanilla RNN Fails on Long Dependencies and Why LSTM Was Designed as a Fix 1709
16.2.2 Exercise 01 - Define Hidden State vs Cell State and Their Roles in Memory Preservation 1710
16.2.3 Exercise 02 - Write LSTM Update Equations and Map Every Term to Memory Control 1711
16.2.4 Exercise 03 - Implement One LSTM Time Step with Pure Torch Tensors Without nn.LSTM 1712
16.2.5 Exercise 04 - Validate Sigmoid and Tanh Gate Ranges Precisely 1713
16.2.6 Exercise 05 - Compare LSTM and Vanilla RNN Forward Behavior on the Same Sequence 1714
16.2.7 Exercise 06 - Force Forget-Gate Extremes and Observe Memory Retention 1714
16.2.8 Exercise 07 - Force Input-Gate Extremes and Observe Memory Writing 1715
16.2.9 Exercise 08 - Force Output-Gate Extremes and Observe newline Hidden-State Exposure 1716
16.2.10 Exercise 09 - Verify Gate and State Shapes for Batch-First and Time-First Layouts 1717
16.2.11 Exercise 10 - Run the Manual LSTM Step on CUDA and Confirm CPU Parity 1718
16.2.12 Exercise 11 - Benchmark Manual LSTM vs nn.LSTM and Explain Kernel Fusion Benefits 1719
16.2.13 Exercise 12 - Compare float32 and float16 LSTM Outputs and Find Precision-Sensitive Gate Cases 1720
16.2.14 Exercise 13 - Debug Gate Order, Concatenation, and newline Weight-Shape Mistakes 1721
16.2.15 Exercise 14 - Manually Unroll LSTM Through Time and Compare with nn.LSTM 1721
16.2.16 Exercise 15 - Stress Test Long-Sequence LSTM Forward Runtime and GPU Memory 1722
16.2.17 Exercise 16 - Investigate cuDNN LSTM Non-Determinism and Document It 1723
16.2.18 Exercise 17 - Build a Long-Memory Synthetic Task and Confirm LSTM Outperforms RNN 1724
16.2.19 Exercise 18 - Find the Smallest Hidden Size That Retains Long Dependencies 1725

16.2.20 Exercise 19 - Explain How Gates Enable More Stable Gradient Flow Through Time 1726
16.3 LSTM01 - LSTM Architecture Variants and Configurations 1727
16.3.1 Exercise 00 - Implement a Many-to-One LSTM Classifier and Select the Last Hidden State Correctly 1727
16.3.2 Exercise 01 - Implement Many-to-Many LSTM Sequence Labeling and Validate Temporal Alignment 1728
16.3.3 Exercise 02 - Implement a Stacked LSTM and Confirm Multi-Layer Hidden and Cell Shapes 1729
16.3.4 Exercise 03 - Implement Bidirectional LSTM and Verify Concatenation and Output Dimensions 1730
16.3.5 Exercise 04 - Compare Unidirectional and Bidirectional LSTM for Sequence Labeling 1731
16.3.6 Exercise 05 - Use batch_first and Keep Shapes Consistent Across the Pipeline 1732
16.3.7 Exercise 06 - Implement Inter-Layer Dropout and Verify the num_layers Condition 1732
16.3.8 Exercise 07 - Build a Capacity Curve Across Hidden Sizes and Layer Counts 1733
16.3.9 Exercise 08 - Implement LSTM Projection and Explain Why It Helps .1734
16.3.10 Exercise 09 - Debug Shape Errors in Multi-Layer and Bidirectional LSTM1735
16.3.11 Exercise 10 - Move the Complete LSTM Architecture to CUDA and Validate State Device Placement 1736
16.3.12 Exercise 11 - Benchmark LSTM Across Sequence Lengths and Find Practical GPU Limits 1737
16.3.13 Exercise 12 - Compare float32 and Mixed-Precision Stability and Find fp16 Gate Failures 1738
16.3.14 Exercise 13 - Wrap nn.LSTM in a Debuggable Module That Exposes States 1739
16.3.15 Exercise 14 - Create Unit Tests for LSTM Shapes Across Configurations1740
16.3.16 Exercise 15 - Validate LSTM Determinism and Reproducibility Across Runs and Machines 1741
16.3.17 Exercise 16 - Stress Test Deep Stacked LSTM for Gradient and Memory Limits 1742
16.3.18 Exercise 17 - Compare LSTM and GRU Parameter Cost with Accuracy1743
16.3.19 Exercise 18 - Build an Architecture Selection Checklist for Depth, Width, and Directionality 1744
16.3.20 Exercise 19 - Produce an LSTM Architecture Engineering Report . . .1744
16.4 LSTM02 - Backpropagation Through Time in LSTM 1745
16.4.1 Exercise 00 - Confirm Gradients Flow Through Hidden and Cell-State Paths 1745
16.4.2 Exercise 01 - Compare Temporal Gradient Norms with Vanilla RNN . .1746
16.4.3 Exercise 02 - Create Exploding Gradients and Apply Clipping Correctly1747
16.4.4 Exercise 03 - Implement Truncated BPTT and Measure Its Tradeoffs .1748
16.4.5 Exercise 04 - Debug Detach Strategies That Break Long-Term Learning1749
16.4.6 Exercise 05 - Finite-Difference Check a Tiny LSTM Cell 1750

16.4.7 Exercise 06 - Monitor Gate Gradients and Detect Saturation 1750
16.4.8 Exercise 07 - Fix Forget-Gate Saturation with Initialization and LR . .1751
16.4.9 Exercise 08 - Fix Input-Gate Saturation with Normalization or Initialization 1752
16.4.10 Exercise 09 - Diagnose NaN Loss from Precision, LR, and Sequence Length 1753
16.4.11 Exercise 10 - Validate CUDA and CPU Gradient Parity 1754
16.4.12 Exercise 11 - Profile BPTT Memory and Graph-Retention Bottlenecks 1754
16.4.13 Exercise 12 - Detect Mixed-Precision Overflow and Underflow 1755
16.4.14 Exercise 13 - Log Gate Gradient Norms with Autograd Hooks 1756
16.4.15 Exercise 14 - Demonstrate and Fix the zero_grad Accumulation Bug .1757
16.4.16 Exercise 15 - Analyze Batch Size, Gradient Noise, and Convergence . .1758
16.4.17 Exercise 16 - Stress Very Long Sequences and Locate LSTM Forgetting1759
16.4.18 Exercise 17 - Compare Full and Truncated BPTT on Long Dependencies1759
16.4.19 Exercise 18 - Build an LSTM-Specific BPTT newline Debugging Checklist 1760
16.4.20 Exercise 19 - Produce the Complete LSTM BPTT Engineering Report 1761
16.5 LSTM03 - Sequence Data Handling for LSTM 1762
16.5.1 Exercise 00 - Build Variable-Length Sequence Dataset and Validate Targets 1762
16.5.2 Exercise 01 - Implement Padding and Masks and Prove Loss Ignores Padding 1763
16.5.3 Exercise 02 - Implement Packed Sequences and Validate Benefits 1764
16.5.4 Exercise 03 - Debug Sorting, Length, and Packed-Output Bugs 1765
16.5.5 Exercise 04 - Compare Padded and Packed Training on CUDA 1765
16.5.6 Exercise 05 - Build a Clean Variable-Length Collate Function 1766
16.5.7 Exercise 06 - Implement Masked Sequence Loss and Match Manual Computation 1767
16.5.8 Exercise 07 - Implement Stateful Batching and Correct State Resets . .1768
16.5.9 Exercise 08 - Prevent Time-Series Leakage with Chronological Splits . .1769
16.5.10 Exercise 09 - Debug Instability from Masks and Loss Reduction 1770
16.5.11 Exercise 10 - Move the Sequence Pipeline to CUDA Efficiently 1771
16.5.12 Exercise 11 - Benchmark DataLoader Workers for Variable Sequences .1772
16.5.13 Exercise 12 - Accumulate Gradients for Long Sequences with Limited Memory 1772
16.5.14 Exercise 13 - Validate Reproducibility with Variable-Length Batches . .1773
16.5.15 Exercise 14 - Format Teacher-Forcing Data newline for LSTM Seq2Seq 1774
16.5.16 Exercise 15 - Build a Complete Sequence-Data Checklist 1775
16.5.17 Exercise 16 - Detect and Fix Off-by-One Target Alignment 1776
16.5.18 Exercise 17 - Create a Task That Fails Without Packing 1776
16.5.19 Exercise 18 - Stress Test a Large Variable-Length Pipeline 1777
16.5.20 Exercise 19 - Produce the LSTM Sequence-Data Engineering Report .1778
16.6 LSTM04 - Language Modeling and Text Generation with LSTM 1779
16.6.1 Exercise 00 - Build Character-Level Tokenization and Vocabulary . . .1779
16.6.2 Exercise 01 - Implement LSTM Language Model and Verify Logit Shapes1780
16.6.3 Exercise 02 - Implement Cross-Entropy Loss and Confirm Target Alignment 1781

16.6.4 Exercise 03 - Overfit a Tiny Text as a Correctness Test 1782
16.6.5 Exercise 04 - Generate Text Autoregressively and Analyze Repetition .1783
16.6.6 Exercise 05 - Implement Temperature Sampling and Measure the Tradeoff1784
16.6.7 Exercise 06 - Implement Top-k Sampling and Compare Stability 1785
16.6.8 Exercise 07 - Implement Nucleus Sampling and Compare p Values . . .1785
16.6.9 Exercise 08 - Debug Hidden and Cell State Carry-Over 1786
16.6.10 Exercise 09 - Evaluate the Language Model with Perplexity 1787
16.6.11 Exercise 10 - Train on CUDA and Benchmark Against CPU 1788
16.6.12 Exercise 11 - Profile GPU Utilization and Fix Pipeline Bottlenecks . .1789
16.6.13 Exercise 12 - Implement Mixed Precision and Identify Instability 1790
16.6.14 Exercise 13 - Compare LSTM and GRU for Language Modeling 1791
16.6.15 Exercise 14 - Implement Truncated BPTT for Longer Contexts 1792
16.6.16 Exercise 15 - Debug Divergence with Learning Rate, Clipping, and Gates1793
16.6.17 Exercise 16 - Build a Generation Evaluation Script 1794
16.6.18 Exercise 17 - Prove LSTM Captures a Controlled Long Dependency . .1795
16.6.19 Exercise 18 - Stress Test Long-Prompt Text Generation 1796
16.6.20 Exercise 19 - Produce the Full LSTM Language Modeling Report . . .1797
16.7 LSTM05 - Time Series Forecasting with LSTM 1798
16.7.1 Exercise 00 - Build Multi-Feature Time Series Windows 1798
16.7.2 Exercise 01 - Implement Feature Scaling Strategy 1798
16.7.3 Exercise 02 - Build a One-Step LSTM Forecaster 1799
16.7.4 Exercise 03 - Extend the Forecaster to Multi-Step Prediction 1800
16.7.5 Exercise 04 - Learn Long Seasonality Better Than a Vanilla RNN . . .1800
16.7.6 Exercise 05 - Compare Teacher Forcing and Autoregressive Generation 1801
16.7.7 Exercise 06 - Evaluate Forecast Accuracy with MAE, MSE, and MAPE1802
16.7.8 Exercise 07 - Debug Windowing and Alignment Bugs 1802
16.7.9 Exercise 08 - Diagnose and Reduce Forecasting Overfitting 1803
16.7.10 Exercise 09 - Build Walk-Forward Validation without Leakage 1804
16.7.11 Exercise 10 - Train the Full Pipeline on CUDA 1804
16.7.12 Exercise 11 - Profile Sequence Length and Hidden Size 1805
16.7.13 Exercise 12 - Implement Mixed Precision and Detect Gate Saturation .1806
16.7.14 Exercise 13 - Compare LSTM and GRU Forecasting 1806
16.7.15 Exercise 14 - Implement Truncated BPTT for Long Windows 1807
16.7.16 Exercise 15 - Build a Multi-Horizon Forecast Visualization 1808
16.7.17 Exercise 16 - Measure Robustness under Seasonal Shift 1808
16.7.18 Exercise 17 - Estimate Probabilistic Forecast Uncertainty 1809
16.7.19 Exercise 18 - Stress Test the Large Forecasting Pipeline 1810
16.7.20 Exercise 19 - Produce the Full Forecasting Engineering Report 1810
16.8 LSTM06 - Regularization, Stabilization,
and Robustness 1811
16.8.1 Exercise 00 - Demonstrate LSTM Overfitting and Measure the Gap . .1811
16.8.2 Exercise 01 - Compare Inter-Layer and Input Dropout 1812
16.8.3 Exercise 02 - Implement and Evaluate Weight Decay 1813
16.8.4 Exercise 03 - Compare Gradient Clipping Norms 1813
16.8.5 Exercise 04 - Apply Layer Normalization inside LSTM 1814
16.8.6 Exercise 05 - Diagnose NaN Loss and Numerical Failure 1815
16.8.7 Exercise 06 - Compare SGD, Adam, and AdamW 1816
16.8.8 Exercise 07 - Schedule Learning Rate and Prevent Late Divergence . .1816

16.8.9 Exercise 08 - Analyze Batch Size and Gradient Noise 1817
16.8.10 Exercise 09 - Debug Improving Loss but Poor Predictions 1818
16.8.11 Exercise 10 - Measure CUDA Overhead of Stabilization 1818
16.8.12 Exercise 11 - Compare Mixed Precision with and without GradScaler .1819
16.8.13 Exercise 12 - Perform a Stabilization Ablation Study 1820
16.8.14 Exercise 13 - Evaluate Robustness under Distribution Shift 1820
16.8.15 Exercise 14 - Calibrate Sequence Predictions over Time 1821
16.8.16 Exercise 15 - Compare Early Stopping and Regularization 1822
16.8.17 Exercise 16 - Create an LSTM Failure Debugging Checklist 1822
16.8.18 Exercise 17 - Analyze Long-Sequence Generalization Limits 1823
16.8.19 Exercise 18 - Stress Test Very Long and Wide LSTMs 1824
16.8.20 Exercise 19 - Write a Practical LSTM Stabilization Guide 1824
16.9 LSTM07 - Efficiency, Profiling, and Deployment 1825
16.9.1 Exercise 00 - Benchmark LSTM vs GRU Runtime and Memory 1825
16.9.2 Exercise 01 - Profile LSTM Training with PyTorch Profiler 1826
16.9.3 Exercise 02 - Reduce Padding Waste and Improve GPU Utilization . .1827
16.9.4 Exercise 03 - Compare Packed and Padded Runtime 1827
16.9.5 Exercise 04 - Measure Sequence-Length Scaling 1828
16.9.6 Exercise 05 - Compare batch_first and Memory Layout 1829
16.9.7 Exercise 06 - Scale Hidden Size and Layer Count 1829
16.9.8 Exercise 07 - Diagnose an Under-Utilized GPU 1830
16.9.9 Exercise 08 - Compare float32 and Mixed Precision 1831
16.9.10 Exercise 09 - Measure Gradient-Clipping Overhead 1831
16.9.11 Exercise 10 - Use Gradient Accumulation under Memory Limits 1832
16.9.12 Exercise 11 - Build Reproducible CUDA Benchmark Tables 1833
16.9.13 Exercise 12 - Investigate cuDNN LSTM Non-Determinism 1833
16.9.14 Exercise 13 - Export LSTM with TorchScript 1834
16.9.15 Exercise 14 - Export LSTM to ONNX 1835
16.9.16 Exercise 15 - Build a Production Inference Benchmark 1835
16.9.17 Exercise 16 - Checkpoint and Resume Training 1836
16.9.18 Exercise 17 - Stress Test Stateful Long-Sequence Inference 1837
16.9.19 Exercise 18 - Build a Deployment Readiness Checklist 1837
16.9.20 Exercise 19 - Produce the LSTM Performance Report 1838
16.10LSTM08 - Bridge to Attention and Why Transformers Win 1839
16.10.1 Exercise 00 - Demonstrate Very-Long-Context LSTM Failure 1839
16.10.2 Exercise 01 - Explain Sequential Computation Limits 1839
16.10.3 Exercise 02 - Add Simple Additive Attention 1840
16.10.4 Exercise 03 - Implement Dot-Product Attention 1841
16.10.5 Exercise 04 - Visualize Attention over Time 1841
16.10.6 Exercise 05 - Build an LSTM Encoder-Decoder with Attention 1842
16.10.7 Exercise 06 - Compare Copy Task with and without Attention 1843
16.10.8 Exercise 07 - Diagnose Attention Mask and Softmax Bugs 1843
16.10.9 Exercise 08 - Measure CUDA Cost and Quality of Attention 1844
16.10.10 Exercise 09 - Produce the Bridge-to-Transformers Report 1845
16.10.11 Exercise 10 - Quantify the Encoder-Decoder Bottleneck 1845
16.10.12 Exercise 11 - Masked Attention for Variable Lengths 1846
16.10.13 Exercise 12 - Compare Attention Distributions by Length 1847
16.10.14 Exercise 13 - Profile Parallelism Added by Attention 1847

16.10.15 Exercise 14 - Clip Gradients in LSTM plus Attention 1848
16.10.16 Exercise 15 - Choose LSTM, LSTM plus Attention, or Transformer . .1849
16.10.17 Exercise 16 - Export LSTM plus Attention 1849
16.10.18 Exercise 17 - Stress Test Long Attention Sequences 1850
16.10.19 Exercise 18 - Ablate Attention Components 1851
16.10.20 Exercise 19 - Produce the Final LSTM Track Report 1851

17 Transformers 1853
17.1 Introduction 1853
17.1.1 What will the book teach you 1853
17.1.2 Why this book comes after the LSTM 1856
17.1.3 How to study this book 1856
17.1.4 The Chapters (TRF00 - TRF08) included in this book 1857
17.2 TRF00 1858
17.2.1 Exercise 00 1858
17.2.2 Exercise 01 1861
17.2.3 Exercise 02 1864
17.2.4 Exercise 03 1867
17.2.5 Exercise 04 1870
17.2.6 Exercise 05 1873
17.2.7 Exercise 06 1876
17.2.8 Exercise 07 1879
17.2.9 Exercise 08 1882
17.2.10 Exercise 09 1885
17.2.11 Exercise 10 1888
17.2.12 Exercise 11 1890
17.2.13 Exercise 12 1893
17.2.14 Exercise 13 1895
17.2.15 Exercise 14 1898
17.2.16 Exercise 15 1900
17.2.17 Exercise 16 1903
17.2.18 Exercise 17 1905
17.2.19 Exercise 18 1908
17.2.20 Exercise 19 1910
17.3 TRF01 1913
17.3.1 Exercise 00 1913
17.3.2 Exercise 01 1916
17.3.3 Exercise 02 1919
17.3.4 Exercise 03 1921
17.3.5 Exercise 04 1923
17.3.6 Exercise 05 1925
17.3.7 Exercise 06 1927
17.3.8 Exercise 07 1930
17.3.9 Exercise 08 1932
17.3.10 Exercise 09 1934
17.3.11 Exercise 10 1937
17.3.12 Exercise 11 1939
17.3.13 Exercise 12 1941

17.3.14 Exercise 13 1943
17.3.15 Exercise 14 1945
17.3.16 Exercise 15 1948
17.3.17 Exercise 16 1950
17.3.18 Exercise 17 1952
17.3.19 Exercise 18 1954
17.3.20 Exercise 19 1957
17.4 TRF02 1959
17.4.1 Exercise 00 1959
17.4.2 Exercise 01 1962
17.4.3 Exercise 02 1964
17.4.4 Exercise 03 1966
17.4.5 Exercise 04 1968
17.4.6 Exercise 05 1970
17.4.7 Exercise 06 1973
17.4.8 Exercise 07 1975
17.4.9 Exercise 08 1977
17.4.10 Exercise 09 1979
17.4.11 Exercise 10 1981
17.4.12 Exercise 11 1983
17.4.13 Exercise 12 1985
17.4.14 Exercise 13 1987
17.4.15 Exercise 14 1990
17.4.16 Exercise 15 1992
17.4.17 Exercise 16 1994
17.4.18 Exercise 17 1996
17.4.19 Exercise 18 1998
17.4.20 Exercise 19 2000
17.5 TRF03 2002
17.5.1 Exercise 00 2003
17.5.2 Exercise 01 2005
17.5.3 Exercise 02 2007
17.5.4 Exercise 03 2009
17.5.5 Exercise 04 2011
17.5.6 Exercise 05 2013
17.5.7 Exercise 06 2016
17.5.8 Exercise 07 2018
17.5.9 Exercise 08 2020
17.5.10 Exercise 09 2022
17.5.11 Exercise 10 2024
17.5.12 Exercise 11 2026
17.5.13 Exercise 12 2028
17.5.14 Exercise 13 2030
17.5.15 Exercise 14 2032
17.5.16 Exercise 15 2034
17.5.17 Exercise 16 2036
17.5.18 Exercise 17 2038
17.5.19 Exercise 18 2040

17.5.20 Exercise 19 2042
17.6 TRF04 2044
17.6.1 Exercise 00 2044
17.6.2 Exercise 01 2046
17.6.3 Exercise 02 2048
17.6.4 Exercise 03 2050
17.6.5 Exercise 04 2052
17.6.6 Exercise 05 2054
17.6.7 Exercise 06 2056
17.6.8 Exercise 07 2057
17.6.9 Exercise 08 2059
17.6.10 Exercise 09 2061
17.6.11 Exercise 10 2063
17.6.12 Exercise 11 2065
17.6.13 Exercise 12 2067
17.6.14 Exercise 13 2069
17.6.15 Exercise 14 2070
17.6.16 Exercise 15 2072
17.6.17 Exercise 16 2074
17.6.18 Exercise 17 2076
17.6.19 Exercise 18 2078
17.6.20 Exercise 19 2080
17.7 TRF05 2082
17.7.1 Exercise 00 2082
17.7.2 Exercise 01 2084
17.7.3 Exercise 02 2086
17.7.4 Exercise 03 2088
17.7.5 Exercise 04 2090
17.7.6 Exercise 05 2092
17.7.7 Exercise 06 2093
17.7.8 Exercise 07 2095
17.7.9 Exercise 08 2097
17.7.10 Exercise 09 2099
17.7.11 Exercise 10 2101
17.7.12 Exercise 11 2103
17.7.13 Exercise 12 2105
17.7.14 Exercise 13 2107
17.7.15 Exercise 14 2109
17.7.16 Exercise 15 2111
17.7.17 Exercise 16 2113
17.7.18 Exercise 17 2115
17.7.19 Exercise 18 2116
17.7.20 Exercise 19 2118
17.8 TRF06 2120
17.8.1 Exercise 00 2121
17.8.2 Exercise 01 2123
17.8.3 Exercise 02 2125
17.8.4 Exercise 03 2127

17.8.5 Exercise 04 2129
17.8.6 Exercise 05 2130
17.8.7 Exercise 06 2132
17.8.8 Exercise 07 2134
17.8.9 Exercise 08 2136
17.8.10 Exercise 09 2138
17.8.11 Exercise 10 2140
17.8.12 Exercise 11 2142
17.8.13 Exercise 12 2144
17.8.14 Exercise 13 2146
17.8.15 Exercise 14 2148
17.8.16 Exercise 15 2150
17.8.17 Exercise 16 2151
17.8.18 Exercise 17 2154
17.8.19 Exercise 18 2156
17.8.20 Exercise 19 2159
17.9 TRF07 2162
17.9.1 Exercise 00 2162
17.9.2 Exercise 01 2164
17.9.3 Exercise 02 2167
17.9.4 Exercise 03 2169
17.9.5 Exercise 04 2171
17.9.6 Exercise 05 2174
17.9.7 Exercise 06 2176
17.9.8 Exercise 07 2178
17.9.9 Exercise 08 2180
17.9.10 Exercise 09 2182
17.9.11 Exercise 10 2184
17.9.12 Exercise 11 2186
17.9.13 Exercise 12 2188
17.9.14 Exercise 13 2191
17.9.15 Exercise 14 2193
17.9.16 Exercise 15 2194
17.9.17 Exercise 16 2197
17.9.18 Exercise 17 2199
17.9.19 Exercise 18 2202
17.9.20 Exercise 19 2204
17.10TRF08 2206
17.10.1 Exercise 00 2207
17.10.2 Exercise 01 2209
17.10.3 Exercise 02 2211
17.10.4 Exercise 03 2213
17.10.5 Exercise 04 2215
17.10.6 Exercise 05 2217
17.10.7 Exercise 06 2219
17.10.8 Exercise 07 2222
17.10.9 Exercise 08 2224
17.10.10 Exercise 09 2226

17.10.11 Exercise 10 2228
17.10.12 Exercise 11 2230
17.10.13 Exercise 12 2233
17.10.14 Exercise 13 2235
17.10.15 Exercise 14 2237
17.10.16 Exercise 15 2239
17.10.17 Exercise 16 2241
17.10.18 Exercise 17 2243
17.10.19 Exercise 18 2245
17.10.20 Exercise 19 2247

18 Large Language Models 2251
18.1 Introduction 2251
18.1.1 What will the book teach you 2251
18.1.2 Why this book comes after the Transformer 2255
18.1.3 How to study this book 2256
18.1.4 The Chapters (LLM00 - LLM08) included in this book 2257
18.2 LLM00 2258
18.2.1 Exercise 00 2258
18.2.2 Exercise 01 2261
18.2.3 Exercise 02 2263
18.2.4 Exercise 03 2266
18.2.5 Exercise 04 2268
18.2.6 Exercise 05 2272
18.2.7 Exercise 06 2274
18.2.8 Exercise 07 2277
18.2.9 Exercise 08 2279
18.2.10 Exercise 09 2281
18.2.11 Exercise 10 2284
18.2.12 Exercise 11 2286
18.2.13 Exercise 12 2288
18.2.14 Exercise 13 2291
18.2.15 Exercise 14 2293
18.2.16 Exercise 15 2295
18.2.17 Exercise 16 2298
18.2.18 Exercise 17 2300
18.2.19 Exercise 18 2302
18.2.20 Exercise 19 2305
18.3 LLM01 2307
18.3.1 Exercise 00 2307
18.3.2 Exercise 01 2310
18.3.3 Exercise 02 2312
18.3.4 Exercise 03 2315
18.3.5 Exercise 04 2317
18.3.6 Exercise 05 2319
18.3.7 Exercise 06 2321
18.3.8 Exercise 07 2323
18.3.9 Exercise 08 2325

18.3.10 Exercise 09 2327
18.3.11 Exercise 10 2329
18.3.12 Exercise 11 2332
18.3.13 Exercise 12 2334
18.3.14 Exercise 13 2336
18.3.15 Exercise 14 2338
18.3.16 Exercise 15 2340
18.3.17 Exercise 16 2343
18.3.18 Exercise 17 2345
18.3.19 Exercise 18 2347
18.3.20 Exercise 19 2349
18.4 LLM02 2351
18.4.1 Exercise 00 2351
18.4.2 Exercise 01 2354
18.4.3 Exercise 02 2356
18.4.4 Exercise 03 2358
18.4.5 Exercise 04 2360
18.4.6 Exercise 05 2362
18.4.7 Exercise 06 2364
18.4.8 Exercise 07 2366
18.4.9 Exercise 08 2368
18.4.10 Exercise 09 2370
18.4.11 Exercise 10 2372
18.4.12 Exercise 11 2374
18.4.13 Exercise 12 2376
18.4.14 Exercise 13 2378
18.4.15 Exercise 14 2380
18.4.16 Exercise 15 2382
18.4.17 Exercise 16 2384
18.4.18 Exercise 17 2386
18.4.19 Exercise 18 2388
18.4.20 Exercise 19 2390
18.5 LLM03 2392
18.5.1 Exercise 00 2392
18.5.2 Exercise 01 2394
18.5.3 Exercise 02 2397
18.5.4 Exercise 03 2399
18.5.5 Exercise 04 2402
18.5.6 Exercise 05 2404
18.5.7 Exercise 06 2407
18.5.8 Exercise 07 2409
18.5.9 Exercise 08 2412
18.5.10 Exercise 09 2414
18.5.11 Exercise 10 2416
18.5.12 Exercise 11 2419
18.5.13 Exercise 12 2421
18.5.14 Exercise 13 2423
18.5.15 Exercise 14 2425

18.5.16 Exercise 15 2428
18.5.17 Exercise 16 2430
18.5.18 Exercise 17 2432
18.5.19 Exercise 18 2434
18.5.20 Exercise 19 2437
18.6 LLM04 2439
18.6.1 Exercise 00 2439
18.6.2 Exercise 01 2441
18.6.3 Exercise 02 2443
18.6.4 Exercise 03 2445
18.6.5 Exercise 04 2447
18.6.6 Exercise 05 2449
18.6.7 Exercise 06 2451
18.6.8 Exercise 07 2453
18.6.9 Exercise 08 2454
18.6.10 Exercise 09 2456
18.6.11 Exercise 10 2458
18.6.12 Exercise 11 2459
18.6.13 Exercise 12 2461
18.6.14 Exercise 13 2463
18.6.15 Exercise 14 2465
18.6.16 Exercise 15 2466
18.6.17 Exercise 16 2468
18.6.18 Exercise 17 2469
18.6.19 Exercise 18 2471
18.6.20 Exercise 19 2473
18.7 LLM05 2474
18.7.1 Exercise 00 2475
18.7.2 Exercise 01 2476
18.7.3 Exercise 02 2478
18.7.4 Exercise 03 2479
18.7.5 Exercise 04 2481
18.7.6 Exercise 05 2482
18.7.7 Exercise 06 2484
18.7.8 Exercise 07 2485
18.7.9 Exercise 08 2487
18.7.10 Exercise 09 2488
18.7.11 Exercise 10 2489
18.7.12 Exercise 11 2491
18.7.13 Exercise 12 2492
18.7.14 Exercise 13 2494
18.7.15 Exercise 14 2495
18.7.16 Exercise 15 2497
18.7.17 Exercise 16 2498
18.7.18 Exercise 17 2499
18.7.19 Exercise 18 2501
18.7.20 Exercise 19 2502
18.8 LLM06 2503

18.8.1 Exercise 00 2504
18.8.2 Exercise 01 2505
18.8.3 Exercise 02 2507
18.8.4 Exercise 03 2509
18.8.5 Exercise 04 2511
18.8.6 Exercise 05 2513
18.8.7 Exercise 06 2515
18.8.8 Exercise 07 2516
18.8.9 Exercise 08 2518
18.8.10 Exercise 09 2520
18.8.11 Exercise 10 2522
18.8.12 Exercise 11 2524
18.8.13 Exercise 12 2526
18.8.14 Exercise 13 2527
18.8.15 Exercise 14 2530
18.8.16 Exercise 15 2531
18.8.17 Exercise 16 2533
18.8.18 Exercise 17 2537
18.8.19 Exercise 18 2541
18.8.20 Exercise 19 2545
18.9 LLM07 2549
18.9.1 Exercise 00 2549
18.9.2 Exercise 01 2552
18.9.3 Exercise 02 2554
18.9.4 Exercise 03 2556
18.9.5 Exercise 04 2559
18.9.6 Exercise 05 2561
18.9.7 Exercise 06 2563
18.9.8 Exercise 07 2565
18.9.9 Exercise 08 2568
18.9.10 Exercise 09 2570
18.9.11 Exercise 10 2572
18.9.12 Exercise 11 2574
18.9.13 Exercise 12 2577
18.9.14 Exercise 13 2578
18.9.15 Exercise 14 2581
18.9.16 Exercise 15 2583
18.9.17 Exercise 16 2585
18.9.18 Exercise 17 2587
18.9.19 Exercise 18 2590
18.9.20 Exercise 19 2592
18.10LLM08 2595
18.10.1 Exercise 00 2595
18.10.2 Exercise 01 2598
18.10.3 Exercise 02 2600
18.10.4 Exercise 03 2602
18.10.5 Exercise 04 2604
18.10.6 Exercise 05 2606

18.10.7 Exercise 06 2609
18.10.8 Exercise 07 2611
18.10.9 Exercise 08 2613
18.10.10 Exercise 09 2615
18.10.11 Exercise 10 2617
18.10.12 Exercise 11 2619
18.10.13 Exercise 12 2621
18.10.14 Exercise 13 2623
18.10.15 Exercise 14 2626
18.10.16 Exercise 15 2628
18.10.17 Exercise 16 2630
18.10.18 Exercise 17 2632
18.10.19 Exercise 18 2635
18.10.20 Exercise 19 2637

19 Dynamic Memory Management 2640
19.1 Introduction 2640
19.1.1 What will the book teach you 2640
19.1.2 Why this book comes after large language models 2643
19.1.3 The seven-part structure of every exercise 2644
19.1.4 How to study this book 2645
19.1.5 The seven stages of the book 2646
19.1.6 The Chapters (DMM00 - DMM49) included in this book 2647
19.2 DMM00 2648
19.2.1 Exercise 00 2649
19.2.2 Exercise 01 2654
19.2.3 Exercise 02 2657
19.2.4 Exercise 03 2659
19.2.5 Exercise 04 2662
19.2.6 Exercise 05 2664
19.2.7 Exercise 06 2666
19.2.8 Exercise 07 2668
19.2.9 Exercise 08 2670
19.2.10 Exercise 09 2673
19.2.11 Exercise 10 2675
19.2.12 Exercise 11 2677
19.2.13 Exercise 12 2679
19.2.14 Exercise 13 2681
19.2.15 Exercise 14 2684
19.2.16 Exercise 15 2686
19.2.17 Exercise 16 2689
19.2.18 Exercise 17 2691
19.2.19 Exercise 18 2694
19.2.20 Exercise 19 2696
19.3 DMM01 2699
19.3.1 Exercise 00 2700
19.3.2 Exercise 01 2704
19.3.3 Exercise 02 2706

19.3.4 Exercise 03 2708
19.3.5 Exercise 04 2710
19.3.6 Exercise 05 2712
19.3.7 Exercise 06 2714
19.3.8 Exercise 07 2716
19.3.9 Exercise 08 2718
19.3.10 Exercise 09 2720
19.3.11 Exercise 10 2721
19.3.12 Exercise 11 2723
19.3.13 Exercise 12 2725
19.3.14 Exercise 13 2727
19.3.15 Exercise 14 2729
19.3.16 Exercise 15 2731
19.3.17 Exercise 16 2734
19.3.18 Exercise 17 2736
19.3.19 Exercise 18 2738
19.3.20 Exercise 19 2740
19.4 DMM02 2743
19.4.1 Exercise 00 2743
19.4.2 Exercise 01 2745
19.4.3 Exercise 02 2747
19.4.4 Exercise 03 2749
19.4.5 Exercise 04 2750
19.4.6 Exercise 05 2752
19.4.7 Exercise 06 2754
19.4.8 Exercise 07 2755
19.4.9 Exercise 08 2757
19.4.10 Exercise 09 2759
19.4.11 Exercise 10 2760
19.4.12 Exercise 11 2762
19.4.13 Exercise 12 2764
19.4.14 Exercise 13 2765
19.4.15 Exercise 14 2767
19.4.16 Exercise 15 2769
19.4.17 Exercise 16 2771
19.4.18 Exercise 17 2773
19.4.19 Exercise 18 2775
19.4.20 Exercise 19 2776
19.5 DMM03 2778
19.5.1 Exercise 00 2778
19.5.2 Exercise 01 2781
19.5.3 Exercise 02 2782
19.5.4 Exercise 03 2784
19.5.5 Exercise 04 2785
19.5.6 Exercise 05 2787
19.5.7 Exercise 06 2789
19.5.8 Exercise 07 2790
19.5.9 Exercise 08 2792

19.5.10 Exercise 09 2793
19.5.11 Exercise 10 2795
19.5.12 Exercise 11 2797
19.5.13 Exercise 12 2799
19.5.14 Exercise 13 2800
19.5.15 Exercise 14 2802
19.5.16 Exercise 15 2803
19.5.17 Exercise 16 2805
19.5.18 Exercise 17 2807
19.5.19 Exercise 18 2808
19.5.20 Exercise 19 2810
19.6 DMM04 2812
19.6.1 Exercise 00 2812
19.6.2 Exercise 01 2814
19.6.3 Exercise 02 2816
19.6.4 Exercise 03 2817
19.6.5 Exercise 04 2818
19.6.6 Exercise 05 2820
19.6.7 Exercise 06 2822
19.6.8 Exercise 07 2823
19.6.9 Exercise 08 2824
19.6.10 Exercise 09 2826
19.6.11 Exercise 10 2827
19.6.12 Exercise 11 2829
19.6.13 Exercise 12 2830
19.6.14 Exercise 13 2831
19.6.15 Exercise 14 2833
19.6.16 Exercise 15 2834
19.6.17 Exercise 16 2836
19.6.18 Exercise 17 2837
19.6.19 Exercise 18 2839
19.6.20 Exercise 19 2841
19.7 DMM05 2842
19.7.1 Exercise 00 2842
19.7.2 Exercise 01 2845
19.7.3 Exercise 02 2846
19.7.4 Exercise 03 2848
19.7.5 Exercise 04 2849
19.7.6 Exercise 05 2851
19.7.7 Exercise 06 2852
19.7.8 Exercise 07 2854
19.7.9 Exercise 08 2856
19.7.10 Exercise 09 2857
19.7.11 Exercise 10 2859
19.7.12 Exercise 11 2860
19.7.13 Exercise 12 2862
19.7.14 Exercise 13 2864
19.7.15 Exercise 14 2865

19.7.16 Exercise 15 2867
19.7.17 Exercise 16 2869
19.7.18 Exercise 17 2871
19.7.19 Exercise 18 2872
19.7.20 Exercise 19 2874
19.8 DMM06 2876
19.8.1 Exercise 00 2876
19.8.2 Exercise 01 2878
19.8.3 Exercise 02 2880
19.8.4 Exercise 03 2881
19.8.5 Exercise 04 2883
19.8.6 Exercise 05 2884
19.8.7 Exercise 06 2886
19.8.8 Exercise 07 2888
19.8.9 Exercise 08 2889
19.8.10 Exercise 09 2891
19.8.11 Exercise 10 2893
19.8.12 Exercise 11 2894
19.8.13 Exercise 12 2896
19.8.14 Exercise 13 2897
19.8.15 Exercise 14 2899
19.8.16 Exercise 15 2900
19.8.17 Exercise 16 2902
19.8.18 Exercise 17 2904
19.8.19 Exercise 18 2906
19.8.20 Exercise 19 2907
19.9 DMM07 2910
19.9.1 Exercise 00 2911
19.9.2 Exercise 01 2912
19.9.3 Exercise 02 2914
19.9.4 Exercise 03 2915
19.9.5 Exercise 04 2916
19.9.6 Exercise 05 2918
19.9.7 Exercise 06 2919
19.9.8 Exercise 07 2921
19.9.9 Exercise 08 2922
19.9.10 Exercise 09 2924
19.9.11 Exercise 10 2925
19.9.12 Exercise 11 2926
19.9.13 Exercise 12 2927
19.9.14 Exercise 13 2929
19.9.15 Exercise 14 2930
19.9.16 Exercise 15 2932
19.9.17 Exercise 16 2933
19.9.18 Exercise 17 2935
19.9.19 Exercise 18 2936
19.9.20 Exercise 19 2938
19.10DMM08 2941

19.10.1 Exercise 00 2942
19.10.2 Exercise 01 2943
19.10.3 Exercise 02 2945
19.10.4 Exercise 03 2946
19.10.5 Exercise 04 2948
19.10.6 Exercise 05 2949
19.10.7 Exercise 06 2951
19.10.8 Exercise 07 2952
19.10.9 Exercise 08 2954
19.10.10 Exercise 09 2956
19.10.11 Exercise 10 2958
19.10.12 Exercise 11 2959
19.10.13 Exercise 12 2961
19.10.14 Exercise 13 2963
19.10.15 Exercise 14 2964
19.10.16 Exercise 15 2966
19.10.17 Exercise 16 2968
19.10.18 Exercise 17 2970
19.10.19 Exercise 18 2972
19.10.20 Exercise 19 2973
19.11DMM09 2976
19.11.1 Exercise 00 2976
19.11.2 Exercise 01 2978
19.11.3 Exercise 02 2979
19.11.4 Exercise 03 2981
19.11.5 Exercise 04 2982
19.11.6 Exercise 05 2984
19.11.7 Exercise 06 2985
19.11.8 Exercise 07 2987
19.11.9 Exercise 08 2989
19.11.10 Exercise 09 2990
19.11.11 Exercise 10 2992
19.11.12 Exercise 11 2993
19.11.13 Exercise 12 2995
19.11.14 Exercise 13 2996
19.11.15 Exercise 14 2998
19.11.16 Exercise 15 2999
19.11.17 Exercise 16 3001
19.11.18 Exercise 17 3002
19.11.19 Exercise 18 3004
19.11.20 Exercise 19 3005
19.12DMM10 3008
19.12.1 Exercise 00 3008
19.12.2 Exercise 01 3009
19.12.3 Exercise 02 3011
19.12.4 Exercise 03 3012
19.12.5 Exercise 04 3013
19.12.6 Exercise 05 3015

19.12.7 Exercise 06 3016
19.12.8 Exercise 07 3018
19.12.9 Exercise 08 3019
19.12.10 Exercise 09 3020
19.12.11 Exercise 10 3021
19.12.12 Exercise 11 3023
19.12.13 Exercise 12 3024
19.12.14 Exercise 13 3025
19.12.15 Exercise 14 3027
19.12.16 Exercise 15 3028
19.12.17 Exercise 16 3029
19.12.18 Exercise 17 3030
19.12.19 Exercise 18 3031
19.12.20 Exercise 19 3033
19.13DMM11 3035
19.13.1 Exercise 00 3035
19.13.2 Exercise 01 3037
19.13.3 Exercise 02 3039
19.13.4 Exercise 03 3040
19.13.5 Exercise 04 3042
19.13.6 Exercise 05 3043
19.13.7 Exercise 06 3045
19.13.8 Exercise 07 3047
19.13.9 Exercise 08 3049
19.13.10 Exercise 09 3050
19.13.11 Exercise 10 3052
19.13.12 Exercise 11 3054
19.13.13 Exercise 12 3056
19.13.14 Exercise 13 3057
19.13.15 Exercise 14 3059
19.13.16 Exercise 15 3061
19.13.17 Exercise 16 3063
19.13.18 Exercise 17 3065
19.13.19 Exercise 18 3067
19.13.20 Exercise 19 3068
19.14DMM12 3072
19.14.1 Exercise 00 3072
19.14.2 Exercise 01 3074
19.14.3 Exercise 02 3075
19.14.4 Exercise 03 3077
19.14.5 Exercise 04 3079
19.14.6 Exercise 05 3081
19.14.7 Exercise 06 3082
19.14.8 Exercise 07 3084
19.14.9 Exercise 08 3085
19.14.10 Exercise 09 3087
19.14.11 Exercise 10 3088
19.14.12 Exercise 11 3090

19.14.13 Exercise 12 3092
19.14.14 Exercise 13 3094
19.14.15 Exercise 14 3095
19.14.16 Exercise 15 3097
19.14.17 Exercise 16 3099
19.14.18 Exercise 17 3101
19.14.19 Exercise 18 3102
19.14.20 Exercise 19 3104
19.15DMM13 3107
19.15.1 Exercise 00 3108
19.15.2 Exercise 01 3109
19.15.3 Exercise 02 3111
19.15.4 Exercise 03 3112
19.15.5 Exercise 04 3114
19.15.6 Exercise 05 3116
19.15.7 Exercise 06 3117
19.15.8 Exercise 07 3119
19.15.9 Exercise 08 3120
19.15.10 Exercise 09 3122
19.15.11 Exercise 10 3123
19.15.12 Exercise 11 3125
19.15.13 Exercise 12 3127
19.15.14 Exercise 13 3128
19.15.15 Exercise 14 3130
19.15.16 Exercise 15 3132
19.15.17 Exercise 16 3133
19.15.18 Exercise 17 3135
19.15.19 Exercise 18 3137
19.15.20 Exercise 19 3138
19.16DMM14 3142
19.16.1 Exercise 00 3142
19.16.2 Exercise 01 3144
19.16.3 Exercise 02 3146
19.16.4 Exercise 03 3147
19.16.5 Exercise 04 3149
19.16.6 Exercise 05 3151
19.16.7 Exercise 06 3152
19.16.8 Exercise 07 3154
19.16.9 Exercise 08 3155
19.16.10 Exercise 09 3157
19.16.11 Exercise 10 3158
19.16.12 Exercise 11 3160
19.16.13 Exercise 12 3162
19.16.14 Exercise 13 3163
19.16.15 Exercise 14 3165
19.16.16 Exercise 15 3166
19.16.17 Exercise 16 3168
19.16.18 Exercise 17 3170

19.16.19 Exercise 18 3171
19.16.20 Exercise 19 3173
19.17DMM15 3175
19.17.1 Exercise 00 3176
19.17.2 Exercise 01 3177
19.17.3 Exercise 02 3179
19.17.4 Exercise 03 3180
19.17.5 Exercise 04 3182
19.17.6 Exercise 05 3183
19.17.7 Exercise 06 3185
19.17.8 Exercise 07 3186
19.17.9 Exercise 08 3188
19.17.10 Exercise 09 3189
19.17.11 Exercise 10 3190
19.17.12 Exercise 11 3192
19.17.13 Exercise 12 3194
19.17.14 Exercise 13 3195
19.17.15 Exercise 14 3197
19.17.16 Exercise 15 3199
19.17.17 Exercise 16 3200
19.17.18 Exercise 17 3202
19.17.19 Exercise 18 3203
19.17.20 Exercise 19 3205
19.18DMM16 3208
19.18.1 Exercise 00 3208
19.18.2 Exercise 01 3209
19.18.3 Exercise 02 3211
19.18.4 Exercise 03 3212
19.18.5 Exercise 04 3214
19.18.6 Exercise 05 3215
19.18.7 Exercise 06 3216
19.18.8 Exercise 07 3218
19.18.9 Exercise 08 3220
19.18.10 Exercise 09 3221
19.18.11 Exercise 10 3223
19.18.12 Exercise 11 3224
19.18.13 Exercise 12 3226
19.18.14 Exercise 13 3227
19.18.15 Exercise 14 3229
19.18.16 Exercise 15 3231
19.18.17 Exercise 16 3232
19.18.18 Exercise 17 3234
19.18.19 Exercise 18 3235
19.18.20 Exercise 19 3237
19.19DMM17 3239
19.19.1 Exercise 00 3239
19.19.2 Exercise 01 3241
19.19.3 Exercise 02 3242

19.19.4 Exercise 03 3244
19.19.5 Exercise 04 3245
19.19.6 Exercise 05 3247
19.19.7 Exercise 06 3248
19.19.8 Exercise 07 3249
19.19.9 Exercise 08 3251
19.19.10 Exercise 09 3252
19.19.11 Exercise 10 3253
19.19.12 Exercise 11 3255
19.19.13 Exercise 12 3256
19.19.14 Exercise 13 3257
19.19.15 Exercise 14 3259
19.19.16 Exercise 15 3261
19.19.17 Exercise 16 3262
19.19.18 Exercise 17 3263
19.19.19 Exercise 18 3265
19.19.20 Exercise 19 3267
19.20DMM18 3268
19.20.1 Exercise 00 3268
19.20.2 Exercise 01 3270
19.20.3 Exercise 02 3271
19.20.4 Exercise 03 3272
19.20.5 Exercise 04 3274
19.20.6 Exercise 05 3275
19.20.7 Exercise 06 3276
19.20.8 Exercise 07 3278
19.20.9 Exercise 08 3279
19.20.10 Exercise 09 3281
19.20.11 Exercise 10 3282
19.20.12 Exercise 11 3284
19.20.13 Exercise 12 3286
19.20.14 Exercise 13 3288
19.20.15 Exercise 14 3290
19.20.16 Exercise 15 3292
19.20.17 Exercise 16 3293
19.20.18 Exercise 17 3295
19.20.19 Exercise 18 3297
19.20.20 Exercise 19 3299
19.21DMM19 3301
19.21.1 Exercise 00 3302
19.21.2 Exercise 01 3303
19.21.3 Exercise 02 3304
19.21.4 Exercise 03 3306
19.21.5 Exercise 04 3307
19.21.6 Exercise 05 3309
19.21.7 Exercise 06 3310
19.21.8 Exercise 07 3311
19.21.9 Exercise 08 3313

19.21.10 Exercise 09 3314
19.21.11 Exercise 10 3315
19.21.12 Exercise 11 3316
19.21.13 Exercise 12 3318
19.21.14 Exercise 13 3319
19.21.15 Exercise 14 3320
19.21.16 Exercise 15 3322
19.21.17 Exercise 16 3323
19.21.18 Exercise 17 3324
19.21.19 Exercise 18 3326
19.21.20 Exercise 19 3328
19.22DMM20 3329
19.22.1 Exercise 00 3329
19.22.2 Exercise 01 3330
19.22.3 Exercise 02 3332
19.22.4 Exercise 03 3333
19.22.5 Exercise 04 3334
19.22.6 Exercise 05 3335
19.22.7 Exercise 06 3336
19.22.8 Exercise 07 3337
19.22.9 Exercise 08 3338
19.22.10 Exercise 09 3339
19.22.11 Exercise 10 3340
19.22.12 Exercise 11 3341
19.22.13 Exercise 12 3342
19.22.14 Exercise 13 3343
19.22.15 Exercise 14 3344
19.22.16 Exercise 15 3345
19.22.17 Exercise 16 3346
19.22.18 Exercise 17 3347
19.22.19 Exercise 18 3349
19.22.20 Exercise 19 3350
19.23DMM21 3351
19.23.1 Exercise 00 3352
19.23.2 Exercise 01 3354
19.23.3 Exercise 02 3357
19.23.4 Exercise 03 3359
19.23.5 Exercise 04 3362
19.23.6 Exercise 05 3364
19.23.7 Exercise 06 3367
19.23.8 Exercise 07 3369
19.23.9 Exercise 08 3371
19.23.10 Exercise 09 3373
19.23.11 Exercise 10 3376
19.23.12 Exercise 11 3378
19.23.13 Exercise 12 3381
19.23.14 Exercise 13 3383
19.23.15 Exercise 14 3385

19.23.16 Exercise 15 3387
19.23.17 Exercise 16 3390
19.23.18 Exercise 17 3392
19.23.19 Exercise 18 3395
19.23.20 Exercise 19 3397
19.24DMM22 3400
19.24.1 Exercise 00 3400
19.24.2 Exercise 01 3404
19.24.3 Exercise 02 3407
19.24.4 Exercise 03 3410
19.24.5 Exercise 04 3413
19.24.6 Exercise 05 3416
19.24.7 Exercise 06 3418
19.24.8 Exercise 07 3421
19.24.9 Exercise 08 3424
19.24.10 Exercise 09 3427
19.24.11 Exercise 10 3430
19.24.12 Exercise 11 3432
19.24.13 Exercise 12 3435
19.24.14 Exercise 13 3438
19.24.15 Exercise 14 3441
19.24.16 Exercise 15 3444
19.24.17 Exercise 16 3446
19.24.18 Exercise 17 3449
19.24.19 Exercise 18 3453
19.24.20 Exercise 19 3455
19.25DMM23 3460
19.25.1 Exercise 00 3460
19.25.2 Exercise 01 3463
19.25.3 Exercise 02 3466
19.25.4 Exercise 03 3469
19.25.5 Exercise 04 3472
19.25.6 Exercise 05 3474
19.25.7 Exercise 06 3477
19.25.8 Exercise 07 3480
19.25.9 Exercise 08 3483
19.25.10 Exercise 09 3486
19.25.11 Exercise 10 3489
19.25.12 Exercise 11 3491
19.25.13 Exercise 12 3494
19.25.14 Exercise 13 3497
19.25.15 Exercise 14 3500
19.25.16 Exercise 15 3503
19.25.17 Exercise 16 3506
19.25.18 Exercise 17 3509
19.25.19 Exercise 18 3512
19.25.20 Exercise 19 3515
19.26DMM24 3519

19.26.1 Exercise 00 3519
19.26.2 Exercise 01 3522
19.26.3 Exercise 02 3525
19.26.4 Exercise 03 3528
19.26.5 Exercise 04 3531
19.26.6 Exercise 05 3533
19.26.7 Exercise 06 3536
19.26.8 Exercise 07 3539
19.26.9 Exercise 08 3541
19.26.10 Exercise 09 3544
19.26.11 Exercise 10 3547
19.26.12 Exercise 11 3549
19.26.13 Exercise 12 3552
19.26.14 Exercise 13 3555
19.26.15 Exercise 14 3558
19.26.16 Exercise 15 3561
19.26.17 Exercise 16 3563
19.26.18 Exercise 17 3566
19.26.19 Exercise 18 3569
19.26.20 Exercise 19 3572
19.27DMM25 3576
19.27.1 Exercise 00 3576
19.27.2 Exercise 01 3579
19.27.3 Exercise 02 3583
19.27.4 Exercise 03 3586
19.27.5 Exercise 04 3589
19.27.6 Exercise 05 3592
19.27.7 Exercise 06 3595
19.27.8 Exercise 07 3598
19.27.9 Exercise 08 3600
19.27.10 Exercise 09 3604
19.27.11 Exercise 10 3607
19.27.12 Exercise 11 3610
19.27.13 Exercise 12 3613
19.27.14 Exercise 13 3616
19.27.15 Exercise 14 3619
19.27.16 Exercise 15 3621
19.27.17 Exercise 16 3624
19.27.18 Exercise 17 3628
19.27.19 Exercise 18 3630
19.27.20 Exercise 19 3634
19.28DMM26 3638
19.28.1 Exercise 00 3638
19.28.2 Exercise 01 3641
19.28.3 Exercise 02 3644
19.28.4 Exercise 03 3647
19.28.5 Exercise 04 3650
19.28.6 Exercise 05 3653

19.28.7 Exercise 06 3656
19.28.8 Exercise 07 3659
19.28.9 Exercise 08 3662
19.28.10 Exercise 09 3665
19.28.11 Exercise 10 3668
19.28.12 Exercise 11 3671
19.28.13 Exercise 12 3674
19.28.14 Exercise 13 3677
19.28.15 Exercise 14 3679
19.28.16 Exercise 15 3682
19.28.17 Exercise 16 3685
19.28.18 Exercise 17 3689
19.28.19 Exercise 18 3691
19.28.20 Exercise 19 3694
19.29DMM27 3699
19.29.1 Exercise 00 3699
19.29.2 Exercise 01 3700
19.29.3 Exercise 02 3702
19.29.4 Exercise 03 3703
19.29.5 Exercise 04 3705
19.29.6 Exercise 05 3706
19.29.7 Exercise 06 3708
19.29.8 Exercise 07 3709
19.29.9 Exercise 08 3711
19.29.10 Exercise 09 3713
19.29.11 Exercise 10 3714
19.29.12 Exercise 11 3716
19.29.13 Exercise 12 3717
19.29.14 Exercise 13 3719
19.29.15 Exercise 14 3720
19.29.16 Exercise 15 3722
19.29.17 Exercise 16 3723
19.29.18 Exercise 17 3725
19.29.19 Exercise 18 3727
19.29.20 Exercise 19 3728
19.30DMM28 3730
19.30.1 Exercise 00 3730
19.30.2 Exercise 01 3732
19.30.3 Exercise 02 3734
19.30.4 Exercise 03 3735
19.30.5 Exercise 04 3737
19.30.6 Exercise 05 3739
19.30.7 Exercise 06 3740
19.30.8 Exercise 07 3742
19.30.9 Exercise 08 3744
19.30.10 Exercise 09 3745
19.30.11 Exercise 10 3747
19.30.12 Exercise 11 3749

19.30.13 Exercise 12 3750
19.30.14 Exercise 13 3752
19.30.15 Exercise 14 3753
19.30.16 Exercise 15 3755
19.30.17 Exercise 16 3757
19.30.18 Exercise 17 3759
19.30.19 Exercise 18 3760
19.30.20 Exercise 19 3762
19.31DMM29 3764
19.31.1 Exercise 00 3765
19.31.2 Exercise 01 3766
19.31.3 Exercise 02 3768
19.31.4 Exercise 03 3770
19.31.5 Exercise 04 3771
19.31.6 Exercise 05 3773
19.31.7 Exercise 06 3775
19.31.8 Exercise 07 3776
19.31.9 Exercise 08 3778
19.31.10 Exercise 09 3780
19.31.11 Exercise 10 3781
19.31.12 Exercise 11 3783
19.31.13 Exercise 12 3785
19.31.14 Exercise 13 3786
19.31.15 Exercise 14 3788
19.31.16 Exercise 15 3790
19.31.17 Exercise 16 3791
19.31.18 Exercise 17 3793
19.31.19 Exercise 18 3795
19.31.20 Exercise 19 3797
19.32DMM30 3799
19.32.1 Exercise 00 3799
19.32.2 Exercise 01 3801
19.32.3 Exercise 02 3803
19.32.4 Exercise 03 3804
19.32.5 Exercise 04 3806
19.32.6 Exercise 05 3808
19.32.7 Exercise 06 3809
19.32.8 Exercise 07 3811
19.32.9 Exercise 08 3813
19.32.10 Exercise 09 3814
19.32.11 Exercise 10 3816
19.32.12 Exercise 11 3818
19.32.13 Exercise 12 3819
19.32.14 Exercise 13 3821
19.32.15 Exercise 14 3823
19.32.16 Exercise 15 3824
19.32.17 Exercise 16 3826
19.32.18 Exercise 17 3828

19.32.19 Exercise 18 3830
19.32.20 Exercise 19 3832
19.33DMM31 3834
19.33.1 Exercise 00 3834
19.33.2 Exercise 01 3836
19.33.3 Exercise 02 3838
19.33.4 Exercise 03 3840
19.33.5 Exercise 04 3841
19.33.6 Exercise 05 3843
19.33.7 Exercise 06 3845
19.33.8 Exercise 07 3847
19.33.9 Exercise 08 3849
19.33.10 Exercise 09 3851
19.33.11 Exercise 10 3853
19.33.12 Exercise 11 3854
19.33.13 Exercise 12 3856
19.33.14 Exercise 13 3858
19.33.15 Exercise 14 3860
19.33.16 Exercise 15 3862
19.33.17 Exercise 16 3864
19.33.18 Exercise 17 3866
19.33.19 Exercise 18 3868
19.33.20 Exercise 19 3870
19.34DMM32 3872
19.34.1 Exercise 00 3872
19.34.2 Exercise 01 3874
19.34.3 Exercise 02 3876
19.34.4 Exercise 03 3878
19.34.5 Exercise 04 3880
19.34.6 Exercise 05 3881
19.34.7 Exercise 06 3883
19.34.8 Exercise 07 3885
19.34.9 Exercise 08 3887
19.34.10 Exercise 09 3889
19.34.11 Exercise 10 3891
19.34.12 Exercise 11 3892
19.34.13 Exercise 12 3894
19.34.14 Exercise 13 3896
19.34.15 Exercise 14 3898
19.34.16 Exercise 15 3900
19.34.17 Exercise 16 3901
19.34.18 Exercise 17 3903
19.34.19 Exercise 18 3905
19.34.20 Exercise 19 3907
19.35DMM33 3910
19.35.1 Exercise 00 3910
19.35.2 Exercise 01 3912
19.35.3 Exercise 02 3914

19.35.4 Exercise 03 3916
19.35.5 Exercise 04 3917
19.35.6 Exercise 05 3919
19.35.7 Exercise 06 3921
19.35.8 Exercise 07 3922
19.35.9 Exercise 08 3924
19.35.10 Exercise 09 3926
19.35.11 Exercise 10 3928
19.35.12 Exercise 11 3929
19.35.13 Exercise 12 3931
19.35.14 Exercise 13 3933
19.35.15 Exercise 14 3934
19.35.16 Exercise 15 3936
19.35.17 Exercise 16 3938
19.35.18 Exercise 17 3940
19.35.19 Exercise 18 3941
19.35.20 Exercise 19 3943
19.36DMM34 3946
19.36.1 Exercise 00 3946
19.36.2 Exercise 01 3948
19.36.3 Exercise 02 3949
19.36.4 Exercise 03 3951
19.36.5 Exercise 04 3953
19.36.6 Exercise 05 3955
19.36.7 Exercise 06 3957
19.36.8 Exercise 07 3958
19.36.9 Exercise 08 3960
19.36.10 Exercise 09 3962
19.36.11 Exercise 10 3963
19.36.12 Exercise 11 3965
19.36.13 Exercise 12 3967
19.36.14 Exercise 13 3969
19.36.15 Exercise 14 3970
19.36.16 Exercise 15 3972
19.36.17 Exercise 16 3974
19.36.18 Exercise 17 3976
19.36.19 Exercise 18 3978
19.36.20 Exercise 19 3980
19.37DMM35 3982
19.37.1 Exercise 00 3982
19.37.2 Exercise 01 3984
19.37.3 Exercise 02 3986
19.37.4 Exercise 03 3988
19.37.5 Exercise 04 3989
19.37.6 Exercise 05 3991
19.37.7 Exercise 06 3993
19.37.8 Exercise 07 3995
19.37.9 Exercise 08 3996

19.37.10 Exercise 09 3998
19.37.11 Exercise 10 4000
19.37.12 Exercise 11 4002
19.37.13 Exercise 12 4004
19.37.14 Exercise 13 4005
19.37.15 Exercise 14 4007
19.37.16 Exercise 15 4009
19.37.17 Exercise 16 4011
19.37.18 Exercise 17 4013
19.37.19 Exercise 18 4014
19.37.20 Exercise 19 4016
19.38DMM36 4019
19.38.1 Exercise 00 4019
19.38.2 Exercise 01 4021
19.38.3 Exercise 02 4023
19.38.4 Exercise 03 4024
19.38.5 Exercise 04 4026
19.38.6 Exercise 05 4028
19.38.7 Exercise 06 4030
19.38.8 Exercise 07 4032
19.38.9 Exercise 08 4033
19.38.10 Exercise 09 4035
19.38.11 Exercise 10 4037
19.38.12 Exercise 11 4039
19.38.13 Exercise 12 4040
19.38.14 Exercise 13 4042
19.38.15 Exercise 14 4044
19.38.16 Exercise 15 4046
19.38.17 Exercise 16 4048
19.38.18 Exercise 17 4050
19.38.19 Exercise 18 4051
19.38.20 Exercise 19 4053
19.39DMM37 4056
19.39.1 Exercise 00 4056
19.39.2 Exercise 01 4058
19.39.3 Exercise 02 4059
19.39.4 Exercise 03 4061
19.39.5 Exercise 04 4063
19.39.6 Exercise 05 4065
19.39.7 Exercise 06 4066
19.39.8 Exercise 07 4068
19.39.9 Exercise 08 4070
19.39.10 Exercise 09 4072
19.39.11 Exercise 10 4074
19.39.12 Exercise 11 4075
19.39.13 Exercise 12 4077
19.39.14 Exercise 13 4079
19.39.15 Exercise 14 4081

19.39.16 Exercise 15 4083
19.39.17 Exercise 16 4084
19.39.18 Exercise 17 4086
19.39.19 Exercise 18 4088
19.39.20 Exercise 19 4090
19.40DMM38 4092
19.40.1 Exercise 00 4092
19.40.2 Exercise 01 4094
19.40.3 Exercise 02 4096
19.40.4 Exercise 03 4098
19.40.5 Exercise 04 4100
19.40.6 Exercise 05 4101
19.40.7 Exercise 06 4103
19.40.8 Exercise 07 4105
19.40.9 Exercise 08 4107
19.40.10 Exercise 09 4109
19.40.11 Exercise 10 4110
19.40.12 Exercise 11 4112
19.40.13 Exercise 12 4114
19.40.14 Exercise 13 4116
19.40.15 Exercise 14 4118
19.40.16 Exercise 15 4120
19.40.17 Exercise 16 4121
19.40.18 Exercise 17 4123
19.40.19 Exercise 18 4125
19.40.20 Exercise 19 4127
19.41DMM39 4129
19.41.1 Exercise 00 4130
19.41.2 Exercise 01 4131
19.41.3 Exercise 02 4133
19.41.4 Exercise 03 4135
19.41.5 Exercise 04 4137
19.41.6 Exercise 05 4139
19.41.7 Exercise 06 4140
19.41.8 Exercise 07 4142
19.41.9 Exercise 08 4143
19.41.10 Exercise 09 4145
19.41.11 Exercise 10 4147
19.41.12 Exercise 11 4149
19.41.13 Exercise 12 4150
19.41.14 Exercise 13 4152
19.41.15 Exercise 14 4154
19.41.16 Exercise 15 4155
19.41.17 Exercise 16 4157
19.41.18 Exercise 17 4159
19.41.19 Exercise 18 4160
19.41.20 Exercise 19 4162
19.42DMM40 4164

19.42.1 Exercise 00 4164
19.42.2 Exercise 01 4166
19.42.3 Exercise 02 4168
19.42.4 Exercise 03 4169
19.42.5 Exercise 04 4171
19.42.6 Exercise 05 4173
19.42.7 Exercise 06 4174
19.42.8 Exercise 07 4176
19.42.9 Exercise 08 4178
19.42.10 Exercise 09 4179
19.42.11 Exercise 10 4181
19.42.12 Exercise 11 4183
19.42.13 Exercise 12 4184
19.42.14 Exercise 13 4186
19.42.15 Exercise 14 4187
19.42.16 Exercise 15 4189
19.42.17 Exercise 16 4191
19.42.18 Exercise 17 4193
19.42.19 Exercise 18 4194
19.42.20 Exercise 19 4196
19.43DMM41 4198
19.43.1 Exercise 00 4198
19.43.2 Exercise 01 4200
19.43.3 Exercise 02 4201
19.43.4 Exercise 03 4203
19.43.5 Exercise 04 4205
19.43.6 Exercise 05 4206
19.43.7 Exercise 06 4208
19.43.8 Exercise 07 4210
19.43.9 Exercise 08 4211
19.43.10 Exercise 09 4213
19.43.11 Exercise 10 4215
19.43.12 Exercise 11 4216
19.43.13 Exercise 12 4218
19.43.14 Exercise 13 4220
19.43.15 Exercise 14 4221
19.43.16 Exercise 15 4223
19.43.17 Exercise 16 4225
19.43.18 Exercise 17 4226
19.43.19 Exercise 18 4228
19.43.20 Exercise 19 4229
19.44DMM42 4231
19.44.1 Exercise 00 4231
19.44.2 Exercise 01 4233
19.44.3 Exercise 02 4235
19.44.4 Exercise 03 4237
19.44.5 Exercise 04 4238
19.44.6 Exercise 05 4240

19.44.7 Exercise 06 4242
19.44.8 Exercise 07 4243
19.44.9 Exercise 08 4245
19.44.10 Exercise 09 4246
19.44.11 Exercise 10 4248
19.44.12 Exercise 11 4250
19.44.13 Exercise 12 4251
19.44.14 Exercise 13 4253
19.44.15 Exercise 14 4255
19.44.16 Exercise 15 4256
19.44.17 Exercise 16 4258
19.44.18 Exercise 17 4260
19.44.19 Exercise 18 4261
19.44.20 Exercise 19 4263
19.45DMM43 4265
19.45.1 Exercise 00 4265
19.45.2 Exercise 01 4267
19.45.3 Exercise 02 4268
19.45.4 Exercise 03 4270
19.45.5 Exercise 04 4271
19.45.6 Exercise 05 4273
19.45.7 Exercise 06 4274
19.45.8 Exercise 07 4276
19.45.9 Exercise 08 4278
19.45.10 Exercise 09 4279
19.45.11 Exercise 10 4281
19.45.12 Exercise 11 4282
19.45.13 Exercise 12 4284
19.45.14 Exercise 13 4286
19.45.15 Exercise 14 4287
19.45.16 Exercise 15 4289
19.45.17 Exercise 16 4290
19.45.18 Exercise 17 4292
19.45.19 Exercise 18 4293
19.45.20 Exercise 19 4295
19.46DMM44 4297
19.46.1 Exercise 00 4297
19.46.2 Exercise 01 4299
19.46.3 Exercise 02 4300
19.46.4 Exercise 03 4302
19.46.5 Exercise 04 4304
19.46.6 Exercise 05 4305
19.46.7 Exercise 06 4307
19.46.8 Exercise 07 4308
19.46.9 Exercise 08 4310
19.46.10 Exercise 09 4311
19.46.11 Exercise 10 4313
19.46.12 Exercise 11 4314

19.46.13 Exercise 12 4316
19.46.14 Exercise 13 4318
19.46.15 Exercise 14 4319
19.46.16 Exercise 15 4321
19.46.17 Exercise 16 4322
19.46.18 Exercise 17 4324
19.46.19 Exercise 18 4326
19.46.20 Exercise 19 4328
19.47DMM45 4329
19.47.1 Exercise 00 4330
19.47.2 Exercise 01 4331
19.47.3 Exercise 02 4333
19.47.4 Exercise 03 4334
19.47.5 Exercise 04 4336
19.47.6 Exercise 05 4337
19.47.7 Exercise 06 4339
19.47.8 Exercise 07 4341
19.47.9 Exercise 08 4342
19.47.10 Exercise 09 4344
19.47.11 Exercise 10 4345
19.47.12 Exercise 11 4347
19.47.13 Exercise 12 4348
19.47.14 Exercise 13 4350
19.47.15 Exercise 14 4351
19.47.16 Exercise 15 4353
19.47.17 Exercise 16 4355
19.47.18 Exercise 17 4356
19.47.19 Exercise 18 4358
19.47.20 Exercise 19 4359
19.48DMM46 4361
19.48.1 Exercise 00 4361
19.48.2 Exercise 01 4363
19.48.3 Exercise 02 4364
19.48.4 Exercise 03 4366
19.48.5 Exercise 04 4368
19.48.6 Exercise 05 4369
19.48.7 Exercise 06 4371
19.48.8 Exercise 07 4372
19.48.9 Exercise 08 4374
19.48.10 Exercise 09 4375
19.48.11 Exercise 10 4377
19.48.12 Exercise 11 4378
19.48.13 Exercise 12 4380
19.48.14 Exercise 13 4381
19.48.15 Exercise 14 4383
19.48.16 Exercise 15 4384
19.48.17 Exercise 16 4386
19.48.18 Exercise 17 4387

19.48.19 Exercise 18 4389
19.48.20 Exercise 19 4391
19.49DMM47 4392
19.49.1 Exercise 00 4392
19.49.2 Exercise 01 4394
19.49.3 Exercise 02 4396
19.49.4 Exercise 03 4397
19.49.5 Exercise 04 4399
19.49.6 Exercise 05 4400
19.49.7 Exercise 06 4402
19.49.8 Exercise 07 4403
19.49.9 Exercise 08 4405
19.49.10 Exercise 09 4406
19.49.11 Exercise 10 4408
19.49.12 Exercise 11 4409
19.49.13 Exercise 12 4411
19.49.14 Exercise 13 4412
19.49.15 Exercise 14 4414
19.49.16 Exercise 15 4415
19.49.17 Exercise 16 4417
19.49.18 Exercise 17 4418
19.49.19 Exercise 18 4420
19.49.20 Exercise 19 4421
19.50DMM48 4423
19.50.1 Exercise 00 4423
19.50.2 Exercise 01 4425
19.50.3 Exercise 02 4427
19.50.4 Exercise 03 4428
19.50.5 Exercise 04 4430
19.50.6 Exercise 05 4432
19.50.7 Exercise 06 4433
19.50.8 Exercise 07 4435
19.50.9 Exercise 08 4436
19.50.10 Exercise 09 4438
19.50.11 Exercise 10 4439
19.50.12 Exercise 11 4441
19.50.13 Exercise 12 4442
19.50.14 Exercise 13 4444
19.50.15 Exercise 14 4446
19.50.16 Exercise 15 4447
19.50.17 Exercise 16 4449
19.50.18 Exercise 17 4450
19.50.19 Exercise 18 4452
19.50.20 Exercise 19 4453
19.51DMM49 4455
19.51.1 Exercise 00 4455
19.51.2 Exercise 01 4457
19.51.3 Exercise 02 4459

19.51.4 Exercise 03 4460
19.51.5 Exercise 04 4462
19.51.6 Exercise 05 4463
19.51.7 Exercise 06 4465
19.51.8 Exercise 07 4466
19.51.9 Exercise 08 4468
19.51.10 Exercise 09 4469
19.51.11 Exercise 10 4471
19.51.12 Exercise 11 4472
19.51.13 Exercise 12 4474
19.51.14 Exercise 13 4476
19.51.15 Exercise 14 4477
19.51.16 Exercise 15 4479
19.51.17 Exercise 16 4480
19.51.18 Exercise 17 4482
19.51.19 Exercise 18 4483
19.51.20 Exercise 19 4485

20 Conclusion of the Volume of Books 4487

Conclusion of the Volume of Books 4487
20.1 What next 4487
20.2 Does this volume of books have an end 4487
20.3 There are those who are smarter than you in artificial intelligence. 4487

Introduction of the Volume of Books

0.1 Why am I writing this volume of books

One day I decided to learn artificial intelligence programming. What happened was strange; I discovered that the more I delved, the more I realized there was still so much to learn. The deeper I explored this world, the more I understood that I hadn't learned anything, or that what I had learned was just a drop in the ocean. That's why I decided to write down everything I learned on my journey to learning artificial intelligence programming.

What I saw in this science confirmed that we in the UAE need a lot of learning. Therefore, I will use this book to teach future generations in the UAE how to program artificial intelligence, based on the principle of self-learning, just as I implemented it at School 42 Abu Dhabi. What we need is to organize our ideas and begin.

0.2 We must be at the forefront

The competition between Saudi Arabia and the UAE to acquire artificial intelligence is a drop in the ocean compared to what companies like OpenAI, Google, and other major American firms are doing. We must accelerate development because the capabilities these companies possess rival those of most countries in the world. As Emiratis and residents of the UAE, we must expedite the development of this community in programming and advancing advanced artificial intelligence.

It's true we have educational and infrastructure projects, but we lack a huge number of educated personnel. That's why I decided to write this book for them—a solid foundation upon which to build this national workforce, and even for residents of the UAE.

0.3 What will this volume of books contain

Book 1: Tensor Mathematics and Neural Networks Guide

You will learn the fundamentals of mathematics and how to program it using Pytorch. While the first challenge will be easy, the second chapter will be quite difficult if you're new to this field. This isn't because it's impossible to grasp, but because you need four things to overcome the difficulties: first, understand the equation; then, write it using Pytorch; then, ask yourself what its application is in artificial intelligence; and finally, practice creating equations using it. Pay attention: you must complete all four of these points together in every chapter. Don't skip or skip any challenge without completing these four points, because the learning system is

built upon what you've learned previously.

Book 2: Applied Linear Algebra in Code

This book moves the reader from understanding mathematical laws to implementing and reasoning about them in code. Knowing an equation on paper is important, but artificial intelligence requires an additional ability: translating that equation into tensor operations that are mathematically correct, computationally efficient, and compatible with the dimensions of the data. The reader will therefore work directly with PyTorch tensors while learning how values, shapes, data types, and devices influence every operation.

The book begins with essential tensor operations, including tensor creation, shape inspection, type conversion, device assignment, element-wise operations, comparisons, and Boolean masking. These topics may appear simple, but they form the language through which every later model will be constructed. A small misunderstanding involving a tensor's shape, data type, or device can stop an entire training pipeline or, more dangerously, allow the program to run while producing an incorrect result. For this reason, the exercises require the reader to inspect and reason about tensors instead of treating them as ordinary lists of numbers.

The reader will then study vector dot products, norms, normalization, cosine similarity, angles, matrix multiplication, inner products, outer products, broadcasting, and dimension flow. These operations are not isolated mathematical tricks. They appear throughout artificial intelligence in similarity measurement, feature interaction, linear transformations, attention mechanisms, loss calculations, and neural-network layers. The exercises will require both manual implementations and PyTorch implementations so the reader understands what the framework performs internally.

Special attention is given to shape reasoning. Before executing an operation, the reader should be able to predict the shape of its output and explain why the operation is valid. This habit is essential because many problems in artificial intelligence programming are dimension problems disguised as coding problems. Broadcasting, batched matrix multiplication, and high-dimensional tensor operations become manageable only when the reader can follow how every dimension moves through the computation.

The final part of the book introduces basis, span, and projection through applied exercises. These concepts explain how information can be represented, compared, decomposed, and reduced inside a feature space. By completing this book, the reader should no longer see tensors as containers of numbers. Instead, they should see them as structured mathematical objects whose dimensions and operations describe how information flows through an artificial intelligence system.

Book 3: Autograd and Backprop Mechanics

This book explains how an artificial intelligence model learns from its mistakes. In the previous books, the reader learned how tensors are created and transformed. Here, those operations are connected into computational graphs that PyTorch can differentiate automatically. The objective is not merely to call `backward()`, but to understand what is constructed during the forward pass, what happens during the backward pass, and why a gradient may be correct, missing, accumulated, blocked, or numerically unstable.

The book begins with computational graph construction. The reader will learn the role of `requires_grad`, the difference between leaf and non-leaf tensors, how operations connect ten-

sors to their gradient functions, and what happens when a tensor is detached from the graph. These distinctions are fundamental. If the graph is broken unintentionally, the model cannot learn; if the graph is retained unnecessarily, memory consumption can increase significantly. The exercises will therefore require the reader to inspect and reason about graph behavior instead of treating automatic differentiation as an invisible process.

The forward pass computes predictions and intermediate values, while the backward pass propagates information about the final error toward the parameters that produced it. The reader will observe gradient accumulation, learn when gradients must be cleared, study repeated backward calls, and understand the purpose of `retain_graph`. Branching graphs will show that a parameter may influence the output through several paths and that its final gradient is the sum of all relevant path contributions.

Manual gradient calculation remains an essential part of this book. The reader will calculate derivatives of linear, quadratic, vector, and matrix operations before comparing those results with PyTorch Autograd. This is necessary because automatic differentiation can compute a result, but it cannot confirm that the mathematical function itself was designed correctly. Numerical gradient approximation will provide another method for checking suspicious gradients and identifying mismatches between theoretical and computed results.

The chain rule connects the entire learning process. The reader will trace gradients through sequences of operations, nested functions, activation functions, and branching paths. They will also investigate how detaching a tensor, using an in-place operation, or disabling gradient tracking changes the graph. By following each contribution from the loss back to a parameter, the reader will understand backpropagation as a structured application of the chain rule rather than as a mysterious framework command.

The later exercises examine PyTorch's Autograd engine, custom automatic-differentiation functions, gradient hooks, memory behavior, and backpropagation through linear layers. The book concludes with gradient-flow debugging, including vanishing gradients, exploding gradients, frozen layers, broken graphs, and training collapse. By the end, the reader should be able to explain not only how gradients are computed, but also why learning succeeds or fails inside a real computational graph.

Book 4: Optimization in Code

Computing gradients tells a model how its loss changes, but gradients alone do not train the model. Optimization determines how those gradients are converted into parameter updates. This book studies the mathematical rules, implementation details, and behavioral consequences of optimization algorithms. The reader will learn that choosing an optimizer is not simply selecting a PyTorch class; it is choosing a system that controls the direction, size, memory, and stability of every learning step.

The book begins with gradient descent implemented from first principles. The reader will update scalar and vector parameters manually, observe how the learning rate changes the optimization path, and compare convergence with divergence. These experiments are important because the same gradient can produce slow progress, stable learning, oscillation, or complete failure depending on the size of the update. Tracking the loss curve will teach the reader to interpret optimization as a process that develops over time rather than as a single calculation.

Learning-rate experiments will demonstrate why there is no universally correct step size. A very small learning rate may produce stable but impractically slow progress, while a very large one may repeatedly cross the minimum or cause the loss to explode. The reader will

explore fixed rates, learning-rate sweeps, schedules, warm-up strategies, and adaptive logic. The purpose is to develop the ability to recognize learning-rate problems from evidence such as loss curves, gradient behavior, and parameter movement.

Momentum introduces memory into parameter updates. Instead of responding only to the current gradient, the optimizer maintains a velocity that combines information from previous steps. The reader will investigate acceleration, damped oscillation, noisy gradients, momentum coefficients, and custom optimizer implementations. This prepares the reader to understand why optimization algorithms maintain internal states and how those states influence future updates.

RMSProp and Adam add adaptive scaling based on gradient history. RMSProp tracks a running average of squared gradients, while Adam combines first-moment and second-moment estimates with bias correction. The reader will implement these mechanisms step by step before using their PyTorch equivalents. Particular attention will be given to numerical stability, epsilon values, decay rates, sparse gradients, and hyperparameter sensitivity. This prevents the algorithms from becoming formulas that are memorized but not understood.

Optimization must also be evaluated through convergence and stability. The reader will detect plateaus, divergence, vanishing gradients, exploding gradients, and instability caused by noisy data. Early stopping, gradient clipping, and optimizer comparisons will be used as diagnostic tools rather than automatic solutions. The book also studies weight initialization because the starting point of optimization can determine whether gradients travel through a network effectively. Zero, uniform, normal, Xavier, and He initialization will be compared through controlled experiments.

By the end of this book, the reader should be able to implement major optimization rules, interpret their internal states, identify unstable training behavior, and justify the selection of an optimizer and its hyperparameters. The goal is to replace random trial and error with experiments supported by mathematical reasoning and observable evidence.

Book 5: Loss Functions and Training Loop

This book brings the previous mathematical and computational components together into a complete learning process. A model cannot learn merely because it produces an output. It requires a loss function that measures the difference between its prediction and the desired target, a backward pass that computes gradients, an optimizer that updates parameters, and a training loop that repeats this process correctly. A mistake in any one of these components can make the entire system unreliable.

The book begins by examining what a loss represents and why its design matters. The reader will compare predictions with targets, calculate scalar and vector losses, and study different aggregation methods. Mean and sum reductions may appear interchangeable, but they change the scale of the loss and its gradients. The reader will also investigate loss sensitivity, scaling effects, and loss landscapes to understand how the selected objective shapes the behavior that the model is encouraged to learn.

Mean Squared Error will be implemented manually and through PyTorch. The reader will examine its batch behavior, gradient, sensitivity to outliers, and numerical stability when values become large. Weighted MSE and comparisons with Mean Absolute Error will show that every loss function expresses assumptions about which errors matter and how strongly they should be penalized. Choosing a loss is therefore part of defining the problem, not merely a technical step after creating the model.

Cross-entropy introduces the logic of classification. The reader will work with probabilities,

class indices, one-hot targets, logarithms, LogSoftmax, and Negative Log-Likelihood. Numerical stability will receive special attention through the log-sum-exp principle. The reader must understand whether a function expects logits or probabilities and whether the target should contain class indices or encoded vectors. Incorrect assumptions about these inputs can produce errors or silently train the wrong objective.

After studying individual losses, the reader will construct a training loop manually. Every iteration must follow a deliberate order: obtain a batch, clear old gradients, perform the forward pass, calculate the loss, run backpropagation, and update the parameters. The reader will examine why changing this order causes gradient accumulation, stale updates, or misleading loss values. Manual parameter updates will then be compared with optimizer-based updates so the relationship between mathematical rules and framework tools remains visible.

The book then expands the loop to mini-batches and epochs. The reader will learn how to shuffle data, handle the final incomplete batch, calculate epoch-level loss correctly, and understand the effect of batch size on memory, speed, and gradient noise. Accumulating batch means without considering batch size can produce an incorrect epoch statistic, so sample counts and reduction rules must remain explicit throughout the pipeline.

Training performance alone cannot establish whether a model has learned a general pattern. The reader will therefore separate training, validation, and evaluation responsibilities. They will use the correct model modes, disable gradient tracking during evaluation, calculate validation loss, compare loss with task-specific metrics, detect overfitting, and select the best model based on validation evidence. The test set must remain untouched until model and hyperparameter decisions have been completed.

The final part focuses on preventing overfitting. The reader will study regularization, weight decay, dropout, data augmentation, early stopping, and the bias–variance tradeoff. They will distinguish underfitting from overfitting and debug cases where a regularization method is implemented incorrectly. These exercises emphasize that regularization is not a collection of switches to activate blindly; every method changes the learning process and must be evaluated through controlled experiments.

By completing this book, the reader should be able to build a robust training and validation pipeline from beginning to end. They should understand how predictions become losses, how losses produce gradients, how optimizers update parameters, how batches form epochs, and how validation protects the model from false confidence. At this stage, the reader will have assembled the fundamental mechanism through which neural networks learn.

Book 6: SQL Lessons (PostgreSQL)

This book moves the reader from files and in-memory objects into the disciplined world of relational data. PostgreSQL is used because it exposes the central ideas of database engineering without hiding them behind an application framework: tables, rows, columns, keys, constraints, joins, transactions, indexes, execution plans, and access control. The objective is not merely to memorize SQL syntax. It is to learn how to represent facts so that the database can preserve their meaning even when applications fail, requests arrive concurrently, or the dataset grows beyond what can be inspected manually.

The opening chapters establish relational thinking. The reader will design schemas, choose data types, distinguish primary keys from business identifiers, model one-to-one, one-to-many, and many-to-many relationships, and use constraints to reject invalid states. They will practice inserts, updates, deletes, filters, ordering, aggregation, subqueries, common table expressions, and joins while continuously checking whether the returned rows answer the intended ques-

tion. Special attention is given to `NULL`, duplicate rows, referential integrity, and the difference between a query that runs and a query that is logically correct.

The book then connects SQL to AI-oriented systems. Experiments model users, conversations, messages, datasets, model versions, training runs, metrics, and audit events. The reader will learn when normalized tables protect consistency, when denormalization may reduce read cost, and how to design reliable boundaries between relational metadata and large binary artifacts stored elsewhere. Transactions and isolation levels are explored through concurrent update scenarios so that atomicity, consistency, isolation, and durability become observable behaviors rather than definitions learned in isolation.

Performance work begins only after correctness is established. The reader will create indexes, compare sequential and indexed access, inspect query plans, identify unnecessary scans, and study the tradeoffs introduced by every additional index. Pagination, bulk insertion, connection management, prepared statements, backup, restoration, roles, permissions, and safe migration practices are included because a useful database must survive change as well as serve queries. Failure exercises expose missing constraints, accidental Cartesian products, ineffective indexes, lock contention, and transaction boundaries that are too wide or too weak.

By completing this book, the reader should be able to design and operate a PostgreSQL database that acts as a trustworthy system of record. They should know how to translate application requirements into schemas, prove data integrity through constraints and transactions, diagnose slow or incorrect queries with evidence, and decide which information belongs in relational storage. This foundation becomes essential when later books combine structured metadata with document stores, object storage, vector databases, and model-serving pipelines.

Book 7: NoSQL Lessons (MongoDB)

This book introduces document-oriented storage as a deliberate modeling choice rather than as an escape from schema design. MongoDB stores records as flexible BSON documents, allowing related fields and nested structures to remain close to the application objects that use them. That flexibility is powerful for rapidly evolving products, conversational histories, event records, and heterogeneous metadata, but it also transfers important responsibilities to the engineer. The reader will learn that “schema-less” never means “structure-less”: every production collection still has assumptions that must be documented, validated, indexed, and migrated.

The first part develops a precise understanding of documents, collections, identifiers, embedded objects, arrays, and references. The reader will insert, retrieve, update, and delete documents; query nested fields; use projections and aggregation pipelines; and examine how document shape affects both clarity and performance. Controlled comparisons between embedding and referencing reveal the central design tradeoff: embedding can make a complete object cheap to read, while references can reduce duplication and prevent oversized or frequently rewritten documents.

Chat history and session memory provide a practical setting for these decisions. The exercises organize conversations, messages, roles, timestamps, tool calls, user preferences, and evolving metadata. The reader will investigate bounded versus unbounded arrays, atomic document updates, time-based retention, indexes for recent-history retrieval, and safeguards against duplicated events. Dynamic storage experiments then introduce validation rules, optional fields, schema version tags, backward-compatible readers, and migration strategies for documents created by older application versions.

The final chapters move from a single process to operational reliability. Readers exam-

ine compound indexes, explain plans, replica sets, read and write concerns, failover, sharding keys, chunk distribution, and the consequences of uneven access patterns. They also study authentication, least-privilege roles, backup and restoration, idempotent writes, and consistency expectations across replicas. Failure scenarios are designed to expose silent problems such as unindexed queries, hot shards, documents that grow without limit, and application code that assumes every historical document has the newest shape.

By the end of the book, the reader should be able to decide when MongoDB is a better fit than a relational database and when it is not. They should be capable of designing document models that evolve safely, building efficient query and aggregation paths, preserving session and conversational data, and planning replication and sharding from actual workload behavior. The lasting lesson is that flexibility is valuable only when it is paired with explicit invariants, observability, and disciplined operational choices.

Book 8: Object Storage Lessons (MinIO)

This book addresses a class of data that does not fit naturally inside ordinary database rows: datasets, images, audio, archives, model weights, checkpoints, exported artifacts, and other large binary objects. MinIO provides an S3-compatible environment in which the reader can learn object storage locally while working with concepts that transfer directly to larger cloud systems. The key conceptual shift is from editing files inside a shared filesystem to storing immutable or versioned objects under bucket and key namespaces and accessing them through network APIs.

The opening exercises establish the object lifecycle. Readers install and start the service, configure a client, create buckets, upload and download individual objects, list keys by prefix, inspect metadata, and verify integrity with hashes. They will distinguish an object key from a real directory path, understand content types and object metadata, and observe why large transfers require streaming or multipart uploads. Experiments with retries and interrupted transfers make idempotency and integrity checking concrete.

The next stage connects object storage to machine-learning work. Datasets, model weights, and checkpoints are organized using predictable naming and versioning strategies. The reader will implement atomic-style save workflows in which an incomplete artifact is never mistaken for a valid checkpoint, retrieve the latest approved version, resume a training run, compare local and remote I/O behavior, and remove obsolete checkpoints without deleting required recovery points. Hash manifests and metadata records are used to detect corruption or accidental replacement.

Security and scale are treated as part of the storage design. The reader will configure bucket policies, create users and access keys, compare read-only and read-write permissions, generate temporary presigned URLs, rotate credentials, and audit denied requests. Distributed-mode experiments introduce erasure coding, replication, node and disk failure, healing, availability, backup, and disaster recovery. These exercises clarify the difference between redundancy and backup: a replicated mistake or deletion is still a mistake unless independent recovery material exists.

By completing this book, the reader should be able to build a reliable artifact layer for AI systems. They should understand how to structure buckets and keys, protect credentials, transfer large files safely, validate integrity, retain useful versions, and recover from infrastructure failures. They will also know how object storage complements PostgreSQL or MongoDB: databases hold searchable state and metadata, while MinIO holds the large immutable payloads referenced by that state.

Book 9: Vector Database Lessons (Qdrant)

This book introduces a different way of retrieving information. Traditional databases answer exact conditions such as identifiers, ranges, and field equality; a vector database can retrieve items whose numerical representations are close in meaning. Qdrant is used to make this process inspectable from end to end. The reader will learn that semantic search is not magic inside the database: it is a pipeline whose quality depends on the embedding model, vector dimensions, distance metric, metadata, indexing configuration, and evaluation procedure.

The first chapters build the representation layer. Readers create collections with explicit vector size and distance settings, generate embeddings, insert individual and batched points, attach payload metadata, and retrieve records by identifier. Experiments examine normalization, cosine similarity, Euclidean distance, and semantic drift. The reader will test near and distant examples, inspect unexpected neighbors, and separate an embedding-model failure from a database configuration failure.

Similarity search is then studied as a ranking mechanism. The exercises perform top-k retrieval, apply score thresholds, filter by payload, combine semantic and structured constraints, and measure ranking stability. Rather than accepting a plausible first result, the reader will construct small relevance sets and calculate whether the desired items appear at useful ranks. Performance benchmarks show how collection size, vector dimension, filtering selectivity, and search parameters change latency and recall.

The RAG chapters connect retrieval to document-grounded generation. Text is chunked with explicit overlap and provenance, embedded, stored, queried, ranked, filtered, and limited to a controlled context budget. Empty retrieval, conflicting chunks, duplicated passages, stale documents, and weak queries are handled as first-class failure modes. The final scaling work introduces HNSW configuration, payload indexes, batch ingestion, shards, replication, consistency, monitoring, snapshot recovery, and the difference between index speed and retrieval quality.

By the end of the book, the reader should be able to engineer and evaluate a semantic retrieval system instead of merely calling a search endpoint. They should understand how meaning becomes a vector, how approximate nearest-neighbor indexes trade recall for speed, how metadata narrows semantic results, and how retrieval quality constrains the final answer of a RAG system. This prepares the ground for models that consume, transform, and generate representations at a much deeper level.

Book 10: Linear Regression

This book begins the model-centered portion of the curriculum with the simplest trainable relationship that still contains the complete logic of machine learning. Linear regression expresses a prediction as a weighted combination of input features plus a bias. Because the model is transparent, every tensor, parameter, prediction, error, gradient, and update can be inspected directly. The simplicity is intentional: the reader can concentrate on the learning mechanism without losing the causal chain inside a large architecture.

The early chapters translate $y = wx + b$ from notation into PyTorch. Readers verify shapes for single samples and batches, extend the formula to multiple features through matrix multiplication, compare manual tensor operations with `nn.Linear`, inspect registered parameters, freeze and copy models, save state dictionaries, and move models safely between CPU and CUDA. Precision and device experiments reveal why two mathematically equivalent implementations may still behave differently in actual software.

Data preparation then becomes part of the model argument. The reader generates synthetic datasets with known parameters, adds controlled noise, scales features, builds train and test splits, uses data loaders, and detects leakage or target misalignment. Collinearity, missing features, high dimensionality, caching, transfer cost, and incorrect batch shapes are explored because a correct formula cannot compensate for a broken input pipeline.

The central chapters connect predictions to mean squared error and gradients. Readers implement MSE manually, compare reduction modes, inspect loss gradients, observe the disproportionate effect of outliers, perform manual parameter updates, verify gradients through finite differences, and compare hand-written steps with optimizers. Full training loops introduce epochs, convergence criteria, divergence detection, learning-rate schedules, early stopping, checkpoints, reproducible seeds, mixed precision, and recovery after interruption.

Evaluation completes the story. RMSE, MAE, residuals, error distributions, cross-validation, throughput, latency, and memory are measured on held-out data. Visualizations of predictions, residuals, loss curves, feature contributions, weights, and bias turn numeric output into evidence about model behavior. By completing the book, the reader should be able to explain exactly how a differentiable model learns, diagnose each stage of its pipeline, and distinguish a low training loss from a model that genuinely represents the underlying relationship.

Book 11: Logistic Regression

This book transforms the linear score into a probabilistic classifier. Logistic regression begins with the same weighted sum used in linear regression, but interprets that raw value as a logit and maps it through the sigmoid function. The result lies between zero and one and can be converted into a class decision through a threshold. This apparently small change introduces many of the central ideas of classification: probabilities, decision boundaries, likelihood, calibration, class imbalance, asymmetric errors, and threshold-dependent metrics.

The reader first implements sigmoid manually and with PyTorch, checks its numerical range, studies saturation under extreme logits, and compares CPU, CUDA, `float32`, and `float64` behavior. Single-sample and vectorized batch predictions make the relationship between tensor shape and output meaning explicit. Threshold experiments show that the model produces a probability-like score while the application chooses the decision rule; changing the threshold changes false positives and false negatives without retraining the weights.

Probability is then connected to likelihood and loss. Readers calculate likelihood, log-likelihood, negative log-likelihood, binary cross-entropy, and the stable logits-based form used by `BCEWithLogitsLoss`. They observe why taking a logarithm of a rounded probability can fail, how class weights affect imbalanced data, and how confidence can be poorly calibrated even when accuracy appears high. The geometry of the linear decision boundary is visualized as weights, bias, scaling, noise, overlap, and regularization change.

Training and evaluation chapters inspect gradients, optimizer updates, clipping, accumulation, mixed precision, and failure cases such as vanishing gradients caused by sigmoid saturation. Confusion matrices, precision, recall, F1, ROC, AUC, threshold sweeps, calibration checks, latency, and memory profiling are implemented from their definitions. A multi-class extension compares one-versus-rest sigmoid models with softmax and cross-entropy, making clear which assumptions change when classes are mutually exclusive.

By the end of the book, the reader should be able to build a numerically stable classification pipeline and defend its decision threshold using the costs of real errors. They should understand the distinction among logits, probabilities, labels, likelihood, and loss; recognize when accuracy is misleading; and analyze both a model's ranking ability and the reliability of its confidence.

This creates a direct bridge from regression into neural classification.

Book 12: Perceptron

This book studies the perceptron as both a historical foundation and a precise boundary between what one linear decision unit can and cannot represent. A perceptron computes a weighted sum plus bias and applies a hard step rule. Unlike logistic regression, it does not attempt to produce a smooth probability. Its prediction is a direct binary decision, and its learning rule responds to classification errors by moving the decision boundary toward a configuration that separates the observed classes.

Readers begin by implementing the forward calculation in pure Python and PyTorch, inspecting the step activation, and comparing continuous linear output, sigmoid probability, and hard binary output. The vector dot product, bias term, signed distance, weight direction, and hyperplane geometry are developed carefully. Two-dimensional plots make each parameter change visible, while higher-dimensional tensor experiments preserve the same mathematical interpretation.

The learning rule is derived from misclassified samples and then implemented without an optimizer. Readers track weights and bias after every update, compare online and batch-style variants, vary the learning rate and sample order, verify updates manually, and observe oscillation under noisy labels. The perceptron convergence theorem is tested on linearly separable data by measuring the number of mistakes and relating stability to the separating margin.

Limitations receive equal attention. XOR and overlapping classes demonstrate why no single hyperplane can solve a non-linearly separable problem. Perceptron loss, hinge loss, and zero-one loss are compared to reveal why smooth or margin-based objectives are easier to optimize. Multi-class strategies, class-specific weight vectors, generalization, regularization, robustness, high-dimensional behavior, CPU/GPU execution, and performance profiling broaden the model without hiding its assumptions.

The final chapter manually combines multiple perceptrons to form non-linear decision regions and solve XOR. This is the conceptual step from a single artificial neuron to a network. By completing the book, the reader should be able to derive and implement the perceptron, state precisely when it converges, diagnose why it fails, and explain why hidden layers and differentiable training are necessary for more expressive models.

Book 13: MLP (Feed-Forward Neural Networks)

This book turns individual linear units into a trainable multi-layer system. A multilayer perceptron composes affine transformations with non-linear activation functions so that each hidden layer can construct a new representation for the next. The central insight is that stacking linear layers without non-linearity remains only one linear transformation; expressive power arises from the interaction among depth, width, learned features, and non-linear activation.

The reader builds two-layer networks from raw tensor operations before using `nn.Module`, `nn.Linear`, and configurable layer lists. Parameter registration, shapes, hooks, copying, serialization, device placement, and architecture validation are treated explicitly. Shallow and deep networks are compared on the same task so that width, depth, bottlenecks, parameter count, memory, and representation capacity become measurable design choices rather than abstract preferences.

Activation and initialization experiments expose the internal health of a network. ReLU, LeakyReLU, sigmoid, tanh, and GELU are compared through activation distributions and gra-

dient flow. Dying ReLUs, saturated sigmoid units, vanishing gradients, and exploding gradients are produced intentionally. Zero, uniform, normal, Xavier, and He initialization are tested while layer-wise statistics show whether signals and gradients remain usable as depth increases.

Backpropagation is then followed through every layer. Readers inspect the autograd graph, calculate a small example manually, perform finite-difference gradient checks, use hooks to record gradient norms, and debug detached tensors, missing `zero_grad`, incorrect loss reduction, device mismatches, and retained graphs. Optimizer chapters compare SGD, momentum, Adam, and AdamW; explore learning-rate schedules, clipping, accumulation, weight decay, mixed precision, and silent update-order bugs; and measure convergence under controlled seeds.

The final portion assembles complete classification and regression pipelines with training, validation, checkpointing, logging, regularization, dropout, early stopping, cross-validation, profiling, export, inference wrappers, object-storage integration, and recovery tests. By completing this book, the reader should be able to design an MLP, observe its internal dynamics, stabilize its optimization, and prepare it for reproducible deployment. More importantly, they will possess a reusable debugging method that carries directly into deeper architectures.

Book 14: Convolutional Neural Networks (CNN)

This book introduces the architectural bias that made deep learning especially effective for images. A fully connected layer mixes every input location globally, while convolution applies shared local filters across space. This structure exploits locality, repeated visual patterns, and translation-related behavior while using far fewer parameters. Readers do not begin with a large pretrained network; they first implement the operation with nested loops so that every pixel, kernel value, stride, padding choice, and output position can be accounted for.

The foundations include cross-correlation, kernel sliding, channels-first tensors, feature maps, receptive fields, padding, stride, dilation, and output-shape formulas. Manual results are checked against PyTorch convolution. Multiple input and output channels reveal how kernels combine spatial and channel information, while custom edge detectors make feature responses visible. Depthwise, grouped, and 1×1 convolutions demonstrate different ways to trade computation and parameters for representational capacity.

Pooling and downsampling chapters examine max pooling, average pooling, global average pooling, adaptive pooling, strided convolution, aliasing, and information loss. The reader constructs full CNN forward architectures with activation functions, flattening or global pooling, classifier heads, batch normalization, dropout, hooks, and residual connections. Shape-tracking utilities and unit tests are used to catch the channel-order and spatial-dimension bugs that frequently appear before training begins.

Training is developed together with the vision data pipeline. Readers load and normalize images, prevent split leakage, apply crops, flips, rotations, color changes, erasing, MixUp or CutMix, balanced sampling, and augmentation ablations. They inspect gradients, overfit a tiny dataset as a correctness test, diagnose NaNs and misleading accuracy, tune optimizers and schedules, use CUDA and mixed precision, and profile data-transfer or DataLoader bottlenecks. Regularization, normalization, calibration, robustness to blur, noise, occlusion, and distribution shift are evaluated through categorized failure examples.

Modern blocks and deployment complete the track. VGG-style stacks, residual bottlenecks, depthwise-separable blocks, inverted residuals, squeeze-and-excitation, stochastic depth, and DropPath are implemented and compared by parameters, FLOPs, memory, speed, and accuracy. Export, compilation, quantization, versioning, monitoring, rollback, and inference-service stress tests connect research code to production constraints. By the end, the reader should

be able to explain what a CNN sees, build and train one reliably, select efficient blocks, and diagnose failures from raw inputs through deployment.

Book 15: Recurrent Neural Networks (RNN)

This book introduces models whose computation carries state through time. Sequence data cannot always be understood by treating each observation independently: the meaning of a word, signal, or event may depend on what appeared earlier. A recurrent neural network addresses this by combining the current input with a hidden state produced at the previous time step. The same weights are reused across the sequence, creating a compact memory and allowing one model to process inputs of different lengths.

The reader implements the recurrence first in pure Python and then with tensors, manually unrolls it through time, and compares the result with `nn.RNN`. Many-to-one and many-to-many mappings, batch-first layouts, stacked and bidirectional variants, hidden-state initialization, and shape rules are examined in detail. Synthetic tasks reveal whether the hidden state actually stores useful history, how information decays, and how hidden size affects memory capacity.

Backpropagation through time exposes both the power and the weakness of the architecture. Readers inspect gradients at successive time steps, reproduce vanishing and exploding behavior, apply clipping, perform finite-difference checks, and implement truncated BPTT. Detaching state at the wrong boundary, retaining graphs accidentally, forgetting to clear gradients, and carrying state across unrelated sequences are treated as concrete engineering failures with observable consequences.

Sequence preparation receives a complete pipeline of its own. Sliding windows, variable lengths, padding, masks, packed sequences, collate functions, teacher forcing, stateful batching, temporal train/validation splits, and target alignment are validated through small edge cases. Character-level language modeling and time-series forecasting then provide two complementary applications. Sampling temperature, top-k, perplexity, autoregressive drift, forecast horizons, walk-forward evaluation, and temporal leakage connect recurrent mechanics to real output quality.

The final chapters examine regularization, normalization, initialization, CUDA performance, mixed precision, profiling, export, and reproducibility. Long-dependency experiments show where vanilla recurrence fails even when code is correct. By completing the book, the reader should be able to build and debug an RNN pipeline, understand the computational and gradient paths through time, and explain from evidence why gated architectures such as GRU and LSTM were needed.

Book 16: Long Short-Term Memory (LSTM)

This book studies the LSTM as an engineering response to the fragile memory of the vanilla RNN. Instead of forcing all past information through one repeatedly transformed hidden state, an LSTM introduces a cell state and learned gates that regulate forgetting, writing, and exposure. The architecture does not grant unlimited memory, but it creates a more stable path through time and gives the model explicit mechanisms for preserving or discarding information.

Readers derive the forget, input, candidate, and output equations and implement a single LSTM cell with raw Torch tensors. Gate ranges, concatenation order, weight shapes, hidden state, and cell state are verified at every step. Forced gate values reveal causal behavior: a closed input gate prevents writing, an open forget gate preserves memory, and the output gate controls what becomes visible. A manual temporal unroll is compared with `nn.LSTM` on CPU

and CUDA.

Architecture chapters cover many-to-one and many-to-many mappings, stacked and bidirectional models, projection layers, dropout between layers, batch-first layouts, and custom wrappers that expose internal state. Backpropagation experiments measure gradients through both hidden and cell paths, identify gate saturation, apply clipping, compare full and truncated BPTT, and diagnose detach strategies that silently destroy long-term learning. Precision, initialization, sequence length, and learning rate are varied to show that an LSTM can still become unstable or forget.

The data and application sections revisit variable-length packing, masking, state reset, teacher forcing, character language modeling, text sampling, time-series forecasting, multi-step drift, and walk-forward validation at greater memory depth. Comparisons with RNN and GRU clarify where additional gates justify their parameter and runtime cost. Regularization, calibration, distribution shift, profiler traces, CUDA utilization, checkpoint recovery, TorchScript, ONNX, and deployment benchmarks turn the model into a complete engineering system.

The closing chapter adds attention on top of LSTM outputs. The reader observes how attention relaxes the encoder bottleneck by retrieving information from multiple time steps directly, while also seeing that the recurrent core still limits parallel execution. By completing the book, the reader should understand not only how LSTM gates work, but why they improve gradient flow, where they remain insufficient, and how their remaining limits lead naturally to attention and Transformers.

Book 17: Transformers

This book introduces the architecture that replaces sequential recurrence with content-based interaction among tokens. Attention allows each position to retrieve a weighted combination of information from other positions, making dependencies explicit and permitting the sequence to be processed in parallel during training. The reader will not treat the Transformer as an opaque library component. Every major tensor transformation is built, measured, tested, and related to the model's ability to represent context.

The opening chapters define queries, keys, and values and implement scaled dot-product attention from raw Torch operations. Shape reasoning follows the full `(batch, heads, sequence, dimension)` layout. Stable softmax, padding masks, causal masks, dropout, scaling by $\sqrt{d_k}$, and attention visualizations are validated through properties rather than example outputs alone. Runtime and memory experiments reveal the quadratic cost of dense attention as sequence length grows.

Multi-head attention, token embeddings, positional information, residual paths, layer normalization, and feed-forward sublayers are assembled into complete encoder and decoder blocks. Readers inspect head diversity, projection matrices, pre-norm and post-norm behavior, positional extrapolation, mask broadcasting, and gradient flow through residual connections. Small copy, classification, and next-token tasks provide controlled environments in which a wrong axis or mask produces a clear failure signal.

Training chapters cover tokenization boundaries, batching variable-length text, cross-entropy alignment, teacher forcing, autoregressive decoding, learning-rate warmup, weight decay, clipping, mixed precision, checkpointing, and reproducibility. Encoder-only, decoder-only, and encoder–decoder architectures are compared by information access and task objective. Profiling then connects theory to hardware through memory peaks, fused attention kernels, compilation, gradient checkpointing, sequence packing, caching, and throughput under realistic batch and context sizes.

By completing this book, the reader should be able to implement a small Transformer from first principles, prove that its masks prevent forbidden information flow, train it on controlled tasks, and identify failures in attention, positions, normalization, or optimization. They should also understand why Transformers displaced recurrent networks at scale and what their advantages cost in memory, data, and computation. This creates the architectural foundation needed to study large language models as systems rather than as mysterious text generators.

Book 18: Large Language Models (LLMs)

This book completes the volume by expanding the Transformer from a small inspectable model into the full lifecycle of a large language model. The scale changes more than the parameter count. Data construction, tokenization, distributed optimization, evaluation, alignment, inference memory, serving, monitoring, and safety become coupled parts of one system. The goal is to give the reader a rigorous map of these parts while preserving the experimental habit developed throughout the earlier books.

The first stage explains how text becomes training examples. Readers examine subword tokenization, vocabulary tradeoffs, context windows, document boundaries, deduplication, filtering, contamination, mixtures of data sources, and next-token prediction. Scaling experiments connect parameters, tokens, batch size, compute, and validation loss without implying that lower pretraining loss automatically produces a useful or trustworthy assistant. Checkpoints and optimizer states are treated as large distributed artifacts with explicit storage and recovery requirements.

Post-training then changes model behavior. Supervised instruction tuning, prompt formats, response masking, preference data, reward modeling, direct preference methods, and human evaluation are introduced through small reproducible analogues. Readers distinguish capability from instruction following, style from factuality, and refusal behavior from genuine safety. Dataset bias, reward hacking, catastrophic forgetting, and benchmark contamination are investigated as failure modes that can produce impressive metrics while weakening real-world reliability.

Inference chapters reveal why serving an LLM is a systems problem. Autoregressive decoding, temperature, top-k, top-p, repetition controls, KV caching, batching, streaming, quantization, memory estimation, latency, throughput, and context growth are measured together. Retrieval-augmented generation, tool calling, structured outputs, and agent loops connect the model to PostgreSQL, MongoDB, MinIO, Qdrant, and the engineering foundations established earlier. Every external action is surrounded by validation, permissions, provenance, timeouts, and failure handling.

Evaluation and production readiness form the final discipline. Readers build task suites, compare automatic and human judgments, test hallucination, calibration, robustness, prompt injection, data leakage, harmful output, distribution shift, and performance regression. Monitoring, versioning, red-team cases, rollback, cost accounting, and incident analysis prevent a successful demonstration from being mistaken for a dependable product. By completing the book, the reader should be able to reason about an LLM across data, architecture, training, alignment, inference, tools, evaluation, and deployment—and to identify which layer of the system must change when behavior fails.

Book 19: Dynamic Memory Management

This book treats memory as an active computational resource rather than a passive capacity limit. Modern PyTorch systems do not operate inside GPU memory alone: tensors, parameters, gradients, optimizer states, activations, input batches, and inference caches move through a hierarchy that includes CUDA VRAM, pinned and pageable system RAM, memory-mapped storage, and NVMe. Performance and correctness therefore depend not only on how many bytes exist, but on where those bytes reside, when they are created, how long they remain live, whether they can be reused, and what synchronization protects them while computation is in flight. Across fifty chapters and one thousand exercises, the reader constructs a measurable theory of this hierarchy and then turns it into a working dynamic-memory runtime.

The opening chapters establish the evidence layer. Readers learn to distinguish allocated, reserved, active, inactive, peak, free, and total memory; predict tensor storage from shape and dtype; and reconcile PyTorch counters with device-level observations. Operator profiling, caching-allocator behavior, detailed allocator statistics, and memory snapshots expose increasingly precise views of the same system. Instead of diagnosing every rise in reserved memory as a leak, the reader reconstructs segments, blocks, allocation histories, stack traces, retry events, and fragmentation. Pinned host buffers, non-blocking transfers, CUDA streams, events, prefetching, and compute–copy overlap then connect memory state to the physical data path between CPU and GPU. Every timing experiment separates submission latency, device execution, synchronization, page-locking cost, and end-to-end throughput so that asynchronous work cannot produce misleading measurements.

The middle of the book turns observation into control. Readers design eviction policies, dynamic tensor and parameter placement, layer offloading, CPU and NVMe offloading, and out-of-core execution across a three-tier hierarchy. Activation checkpointing and activation offloading trade storage for recomputation or transfer, while mixed precision, dynamic loss scaling, gradient accumulation, and adaptive batch sizing change the memory budget during training. Controlled CUDA out-of-memory experiments lead to recovery policies, memory-aware scheduling, buffer pools, fragmentation management, tensor reuse, in-place operations, and storage-alias analysis. Meta-device construction, lazy parameter materialization, memory-mapped datasets, DataLoader optimization, and virtualized or paged tensor storage show how large systems can be assembled without allocating every object at once.

The LLM and distributed chapters apply these principles where memory pressure becomes architectural. Readers implement paged KV caches, move cache blocks between VRAM and RAM, quantize cached keys and values, and design eviction policies that balance context retention against latency and capacity. Fully Sharded Data Parallel, ZeRO-style partitioning, tensor parallelism, pipeline scheduling, and multi-GPU balancing are studied as memory-placement algorithms rather than as unexplained framework switches. Parameters, gradients, optimizer states, activations, temporary communication buffers, and uneven device capacities are accounted for explicitly. Experiments are designed to run on one machine with the available GPUs while still revealing the ownership, communication, synchronization, and failure principles that govern larger distributed systems.

The closing chapters unite measurement and policy as one engineering discipline. CUDA events and bandwidth profiling establish trustworthy timing boundaries; leak investigations connect persistent references to Python source; autograd analysis reconstructs saved tensors and graph lifetimes; and detach, no-grad, and inference mode are treated as distinct controls with distinct correctness consequences. The reader then builds a custom PyTorch memory manager and a final offload engine that observes pressure, chooses placement, reuses bounded buffers, schedules transfers, applies backpressure, recovers from predictable failures, and vali-

dates numerical outputs after every optimization. By completing this book, the reader should be able to explain every major MiB used by a workload, locate the source of memory growth, distinguish capacity failure from fragmentation or lifetime failure, and design a runtime that adapts across VRAM, RAM, and storage without sacrificing correctness, stability, or reproducibility.

0.4 Artificial Intelligence Departments

When you start learning about artificial intelligence, you might encounter some problems, or in short, you'll fall into a black hole and won't know why this is happening, who I am, or what's around me.

At every step, you must know which AI specialization you are currently studying, because this is the simplest and most important question that helps you understand the AI world without confusion. When you clearly know where you are, learning becomes focused and stable, and you stop mixing theory with systems or tools that belong to a different stage. The harder questions, such as why you are studying this or what learning structure you want, can come later, but without first knowing your current specialization, progress feels chaotic even when you are moving forward. Knowing where you stand gives context, sets boundaries, and makes the entire learning journey intentional and clear.

For this reason, readers will notice that I explicitly show the AI specializations inside the book itself. These specializations are placed there intentionally so the reader always knows which domains they are focusing on, independent of the book title. By seeing the specializations clearly, the reader can immediately understand what they are learning at that moment and which part of the AI landscape they are operating in, making the learning process structured, contextual, and free from confusion.

1- Mathematics and Theory

This specialization focuses on the mathematical foundations that govern artificial intelligence, including linear algebra, probability theory, optimization, and information theory. It provides the formal language used to describe models, learning dynamics, and convergence behavior, making it essential for understanding why algorithms work, not just how to use them.

2- Models and Training

This specialization covers the design, training, and optimization of AI models such as neural networks, transformers, and other learning architectures. It includes loss functions, backpropagation, optimization strategies, and training stability, forming the core process through which models learn from data.

3- Data Engineering for AI

This specialization focuses on preparing data for machine learning systems, including data collection, cleaning, transformation, labeling, and feature engineering. Since model performance

is heavily dependent on data quality, this area plays a critical role in the success or failure of AI systems.

4- AI Evaluation and Experimentation

This specialization is concerned with measuring and validating model performance using metrics, benchmarks, validation strategies, and controlled experiments. It ensures that models generalize correctly, avoid overfitting, and behave as expected under different conditions.

5- Applied AI and Deployment

This specialization deals with integrating trained models into real-world applications. It includes inference pipelines, APIs, latency optimization, monitoring, and system reliability, transforming research models into usable and scalable AI products.

6- AI Systems and Distributed Training

This specialization focuses on scaling AI training and inference across multiple devices or machines. It includes distributed training techniques, parallelism strategies, memory optimization, and communication frameworks, which become necessary when working with large models or datasets.

7- AI Infrastructure and Hardware

This specialization addresses the underlying hardware and infrastructure that support AI workloads. It includes GPUs, accelerators, memory hierarchies, networking, and low-level software stacks, enabling efficient computation at scale.

8- AI Safety / Ethics / Alignment

This specialization examines the risks, limitations, and societal impact of AI systems. It covers topics such as bias, robustness, alignment with human intent, misuse prevention, and regulatory considerations, ensuring AI systems are safe and responsible.

9- AI + Domain Specialization

This specialization focuses on applying AI techniques to specific fields such as medicine, finance, cybersecurity, physics, or robotics. It combines AI knowledge with domain expertise, enabling the creation of effective and meaningful solutions within real-world contexts.

Book 1

Tensor Mathematics and Neural Networks Guide

1.1 Introduction

1.1.1 What will the book teach you

Before entering the world of artificial intelligence programming, you must understand the mathematical and programming foundations that make intelligent systems possible. This book begins with the most fundamental tensor operations and gradually progresses toward higher-dimensional tensor laws, differential calculus, optimization, probability, statistics, and neural-network transformations using PyTorch.

The purpose of this book is to build a strong connection between mathematical theory and practical implementation. You will learn how equations describe tensor operations, how tensor shapes control valid computations, and how these laws appear inside neural networks. Every chapter builds on the knowledge from the chapters before it, so you should understand each lesson and complete every exercise before moving forward.

The book consists of many exercises. Each exercise explains why a mathematical law matters, presents its mathematical form, provides hints and helpful PyTorch tools, and gives you an incomplete Python code skeleton. The complete solution will not be provided. You must analyse the problem, complete the code, test your implementation, and correct any mistakes yourself.

This approach may feel challenging, but it is designed to develop independent reasoning and genuine problem-solving ability. The objective is not simply to memorize equations or copy code. You must understand what each operation does, why its dimensions are valid, how it affects data, and where it is used in artificial intelligence. This book will establish the mathematical foundation required for the practical and advanced AI books that follow.

1.1.2 The Chapters (T00 - T08) included in this book

- T00 - Tensor Algebra Laws
- T01 - Vector Laws (1D)

- T02 - Matrix Laws (2D)
- T03 - Higher-Dimension Tensor Laws
- T04 - Tensor Shape Transformation Laws
- T05 - Differential Calculus Laws
- T06 - Optimization Laws
- T07 - Statistics Probability Laws
- T08 - Neural Network Transformation Laws

1.2 T00 - Tensor Algebra Laws

Mathematics and Theory

1.2.1 Exercise 00 - Tensor Addition Law

Part 1 - why this matters

Tensor addition is the first operation used to combine numerical information in artificial intelligence systems. It appears in almost every model, from adding bias terms to accumulating gradients during training. Understanding this operation is essential before moving to more complex tensor transformations.

Part 2 - Mathematical form

Mathematical Definition

Let $\mathbf{X}, \mathbf{Y} \in \mathbb{R}^{3\times3}$.

Tensor (matrix) addition is a binary operator defined as:

$$+ : \mathbb{R}^{3\times3} \times \mathbb{R}^{3\times3} \to \mathbb{R}^{3\times3}$$

Define:

$$\mathbf{Z} = \mathbf{X} + \mathbf{Y}$$

where $\mathbf{Z} \in \mathbb{R}^{3\times3}$ and the operation is defined component-wise by:

$$Z_{i,j} = X_{i,j} + Y_{i,j} \quad \text{for } i, j \in \{1, 2, 3\}.$$

Part 3 - Hints and Helper

Tensor addition in PyTorch is performed using the built-in function `torch.add(x, y)`. This operation applies element-wise addition and requires both tensors to have the same shape.

Before performing any tensor operation, ensure that the inputs are PyTorch tensors created using `torch.tensor()`. Understanding how tensors are constructed is essential before manipulating them.

PyTorch tensors can exist on different devices:

- CPU (default)
- GPU (CUDA-enabled devices)

When working with multiple tensors, both tensors must be located on the same device. Mixing CPU and GPU tensors will result in a runtime error.

To deepen your understanding, explore:

- How to create tensors using `torch.tensor()`
- How to check the device of a tensor
- How to move tensors between CPU and GPU

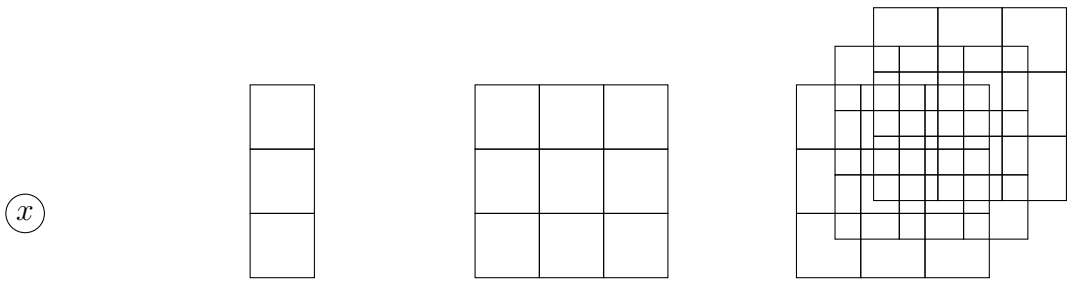
This exercise assumes CPU-based tensors, but the same logic applies to GPU tensors when explicitly moved to a CUDA device.

Part 4 - Python code skeleton

```
import torch

def E00(x,y):
    z = torch...
    return z

def main():
    x = torch.tensor...
    y = torch.tensor...
    print(E00(...))

if __name__ == "__main__":
    main()
```

1.2.2 Exercise 01 - Tensor subtraction Law

Part 0 - Before we begin



Scalar **Vector** **Matrix** **Tensor**

All numerical data in artificial intelligence is ultimately represented using the same underlying structure; the difference lies only in how many dimensions the data has.

- **Scalar**: A single numerical value with no dimensions.
- **Vector**: A one-dimensional collection of ordered values.
- **Matrix**: A two-dimensional grid of values arranged in rows and columns.
- **Tensor**: A generalization to three or more dimensions, used to represent complex data such as images, sequences, or batches.

Part 1 - why this matters

This operation has the same importance as the previous exercise. The only difference is that subtraction is used instead of addition. Understanding this change is essential before moving to more advanced tensor operations.

Part 2 - Mathematical form

Let $\mathbf{X}, \mathbf{Y} \in \mathbb{R}^{3\times3}$.

Tensor (matrix) subtraction is a binary operator defined as:

$$- : \mathbb{R}^{3\times3} \times \mathbb{R}^{3\times3} \to \mathbb{R}^{3\times3}$$

Define:

$$\mathbf{Z} = \mathbf{X} - \mathbf{Y}$$

where $\mathbf{Z} \in \mathbb{R}^{3\times3}$ and the operation is defined component-wise by:

$$Z_{i,j} = X_{i,j} - Y_{i,j} \quad \text{for } i, j \in \{1, 2, 3\}.$$

Part 3 - Hints and Helper

In PyTorch, tensor subtraction is performed using the built-in function `torch.sub(x, y)`. The reader is encouraged to search for this function and understand how it applies element-wise subtraction between tensors.

Part 4 - Python code skeleton

```
import torch

def E01(x,y):
    z = torch...
    return z

def main():
    x = torch.tensor...
    y = torch.tensor...
    print(E01(...))

if __name__ == "__main__":
    main()
```

1.2.3 Exercise 02 - Tensor Hadamard Multiplication Law

Part 1 - why this matters

This exercise continues the logical progression of tensor operations. After learning how tensors are combined through addition and how differences are expressed through subtraction, Hadamard multiplication introduces a new concept: controlling how information is scaled rather than simply accumulated or removed.

Unlike addition and subtraction, element-wise multiplication directly modifies the strength of each individual value. Each component can be amplified, reduced, or completely suppressed depending on the corresponding value in the second tensor. This changes the behavior of the data itself, not just its numerical total.

Understanding this operation is essential because it represents a foundational mechanism used later in more advanced tensor transformations. Mastering this concept at an early stage makes the transition to complex model structures more intuitive and coherent.

Part 2 - Mathematical form

Let $\mathbf{X}, \mathbf{Y} \in \mathbb{R}^{3\times3}$.

Hadamard multiplication (element-wise multiplication) is a binary operator defined as:

$$\odot : \mathbb{R}^{3\times3} \times \mathbb{R}^{3\times3} \to \mathbb{R}^{3\times3}$$

Define:

$$\mathbf{Z} = \mathbf{X} \odot \mathbf{Y}$$

where $\mathbf{Z} \in \mathbb{R}^{3\times3}$ and the operation is defined component-wise by:

$$Z_{i,j} = X_{i,j} \cdot Y_{i,j} \quad \text{for } i, j \in \{1, 2, 3\}.$$

Part 3 - Hints and Helper

In PyTorch, Hadamard (element-wise) multiplication can be performed using the built-in operator `x * y`.

The reader is encouraged to search for a function called `torch.mul()` and understand how it performs element-wise multiplication between tensors.

Part 4 - Python code skeleton

```
import torch

def E02(x,y):
    z = torch...
    return z

def main():
    x = torch.tensor...
    y = torch.tensor...
    print(E02(...))

if __name__ == "__main__":
    main()
```

1.2.4 Exercise 03 - Tensor Division Law

Part 0 - The weight

In artificial intelligence, a weight represents the strength of influence that an input has on the model's internal computation and final decision. Each input signal is multiplied by a weight, allowing the model to emphasize important information and suppress irrelevant or noisy signals. Learning in neural networks is therefore the process of continuously adjusting these weights to reflect how much the model should trust each input.

$$2000 \times 2000 = 4{,}000{,}000$$

$$0.2 \times 0.2 = 0.04$$

The large numerical gap produced by unscaled weights leads to unstable matrix computations, where values grow too fast and dominate subsequent operations. Keeping weights within a controlled and close numerical range ensures stable tensor operations, preserves relative relationships between features, and allows learning algorithms to update weights smoothly without numerical explosion or loss of precision.

Part 1 - why this matters

Tensor Division Law is required because repeated multiplication and accumulation in neural networks cause values to either explode or vanish, and division is used to rescale tensors into a numerical range where learning remains stable.

In normalization, division is not applied merely to modify data values, but to decouple magnitude from structure, allowing the model to learn patterns rather than be dominated by raw numerical scale.

When working with ratios, probabilities, and feature weights, division transforms absolute values into relative relationships, enabling meaningful comparison and scale-independent decision making.

Part 2 - Mathematical form

Let $\mathbf{X}, \mathbf{Y} \in \mathbb{R}^{3\times3}$.

Tensor Division Law (element-wise):

$$\oslash : \mathbb{R}^{3\times3} \times \mathbb{R}^{3\times3} \to \mathbb{R}^{3\times3}$$

Define:

$$\mathbf{Z} = \mathbf{X} \oslash \mathbf{Y}$$

where $\mathbf{Z} \in \mathbb{R}^{3\times3}$ and the law is defined component-wise by:

$$Z_{i,j} = \frac{X_{i,j}}{Y_{i,j}} \quad \text{for } i,j \in \{1,2,3\}, \quad Y_{i,j} \neq 0.$$

Part 3 - Hints and Helper

In PyTorch, tensor division (element-wise division) can be performed using the built-in operator `x / y`.

The reader is encouraged to search for a function called `torch.div()` and understand how it performs element-wise division between tensors.

Part 4 - Python code skeleton

```
import torch

def E03(x,y):
    z = torch...
    return z

def main():
    x = torch.tensor...
    y = torch.tensor...
    print(E03(...))

if __name__ == "__main__":
    main()
```

1.2.5 Exercise 04 - Tensor Power Law

Part 0 - The Magnitude

In artificial intelligence, knowing whether a value is positive or negative is often less important than understanding its magnitude relative to zero. Magnitude represents the size or intensity of an error or deviation without regard to direction, since a model must measure how large an error is rather than where it lies.

For this reason, power-based operations are used to represent magnitude in a numerically stable form, enabling reliable measurement and optimization of error and deviation.

Part 1 - why this matters

In artificial intelligence, it is essential to distinguish between the influence of a signal controlled by weights and the magnitude of a value measured independently of its direction.

The Tensor Power Law is used to convert values into measurable magnitudes, enabling stable numerical evaluation of error and deviation.

Part 2 - Mathematical form

Let $\mathbf{X} \in \mathbb{R}^{3\times3}$.

Tensor Power Law (element-wise) is defined as:

$$\circledast : \mathbb{R}^{3\times3} \to \mathbb{R}^{3\times3}$$

Define:

$$\mathbf{Z} = \mathbf{X} \circledast 2$$

where $\mathbf{Z} \in \mathbb{R}^{3\times3}$ and the law is defined component-wise by:

$$Z_{i,j} = (X_{i,j})^2 \quad \text{for } i, j \in \{1, 2, 3\}.$$

Part 3 - Hints and Helper

In PyTorch, tensor power (element-wise exponentiation) can be performed using the built-in operator `x ** 2`.

The reader is encouraged to search for a function called `torch.pow()` and understand how it applies element-wise power operations between tensors or between a tensor and a scalar.

Part 4 - Python code skeleton

```
import torch

def E04(x):
    z = torch...
    return z

def main():
    x = torch.tensor...
    print(E04(...))

if __name__ == "__main__":
    main()
```

1.2.6 Exercise 05 - Tensor Square Root Law

Part 0 - The element-wise

What "element-wise" means. An **element-wise** operation applies the same rule to each entry of a tensor *independently*, so the value at one position is computed only from the value(s) at the *same position*. As a result, the output typically has the **same shape** as the input (or the broadcasted shape if broadcasting is used).

Shape requirement (matrix × matrix). For two matrices, an element-wise binary law is defined only when the shapes match:

$$\odot : \mathbb{R}^{3\times3} \times \mathbb{R}^{3\times3} \to \mathbb{R}^{3\times3}, \qquad Z_{i,j} = X_{i,j} \odot Y_{i,j}.$$

Here, each output entry $Z_{i,j}$ depends only on $X_{i,j}$ and $Y_{i,j}$.

Why we use element-wise laws in AI. Element-wise laws are used to transform or scale data *without mixing information* across positions. They are essential for operations such as normalization, magnitude measurement (e.g., squaring), and pointwise non-linear transformations, where we want to adjust values while preserving the tensor structure.

Why $\mathbb{R}^{3\times3}$ and $\mathbb{R}^{3\times2}$ is not element-wise. If $\mathbf{X} \in \mathbb{R}^{3\times3}$ and $\mathbf{Y} \in \mathbb{R}^{3\times2}$, then an element-wise binary operation is **not defined** because there is no one-to-one alignment between entries:

$$\mathbb{R}^{3\times3} \not\odot \mathbb{R}^{3\times2} \quad \text{(shape mismatch)}.$$

So your intuition is correct: this is **not** an element-wise operation. However, this does **not** automatically mean it is a different "type of operation" by itself; it is simply a **shape mismatch** for element-wise binary laws. (Only if a broadcasting rule applies can some mismatched shapes still work, but 3×3 and 3×2 do not broadcast to a common shape.)

Part 1 - why this matters

In artificial intelligence, operations like squaring are used to measure intensity or error without being affected by sign, but they also change the measurement unit and push values into a much larger numerical range than the original scale. The Tensor Square Root Law restores values back to their natural scale after magnitude-based computations, making results interpretable and comparable by returning them to the same "unit" as the original signal rather than leaving them in squared units. Additionally, the square root compresses the numerical range and mitigates the growth caused by squaring and accumulation, improving tensor-level numerical stability and reducing the risk of explosion or precision loss during training.

Part 2 - Mathematical form

Let $\mathbf{X} \in \mathbb{R}^{3\times3}$ with $X_{i,j} \geq 0$.

Tensor Square Root Law (element-wise) is defined as:

$$\sqrt{\cdot} : \mathbb{R}^{3\times3}_{\geq 0} \to \mathbb{R}^{3\times3}$$

Define:

$$\mathbf{Z} = \sqrt{\mathbf{X}}$$

where $\mathbf{Z} \in \mathbb{R}^{3\times3}$ and the law is defined component-wise by:

$$Z_{i,j} = \sqrt{X_{i,j}} \quad \text{for } i, j \in \{1, 2, 3\}.$$

Part 3 - Hints and Helper

In PyTorch, tensor square root (element-wise square root) can be performed using the built-in function `torch.sqrt()`.

The reader is encouraged to explore how `torch.sqrt()` applies the square root operation element-wise on tensors and to understand its role in restoring scale after magnitude-based operations.

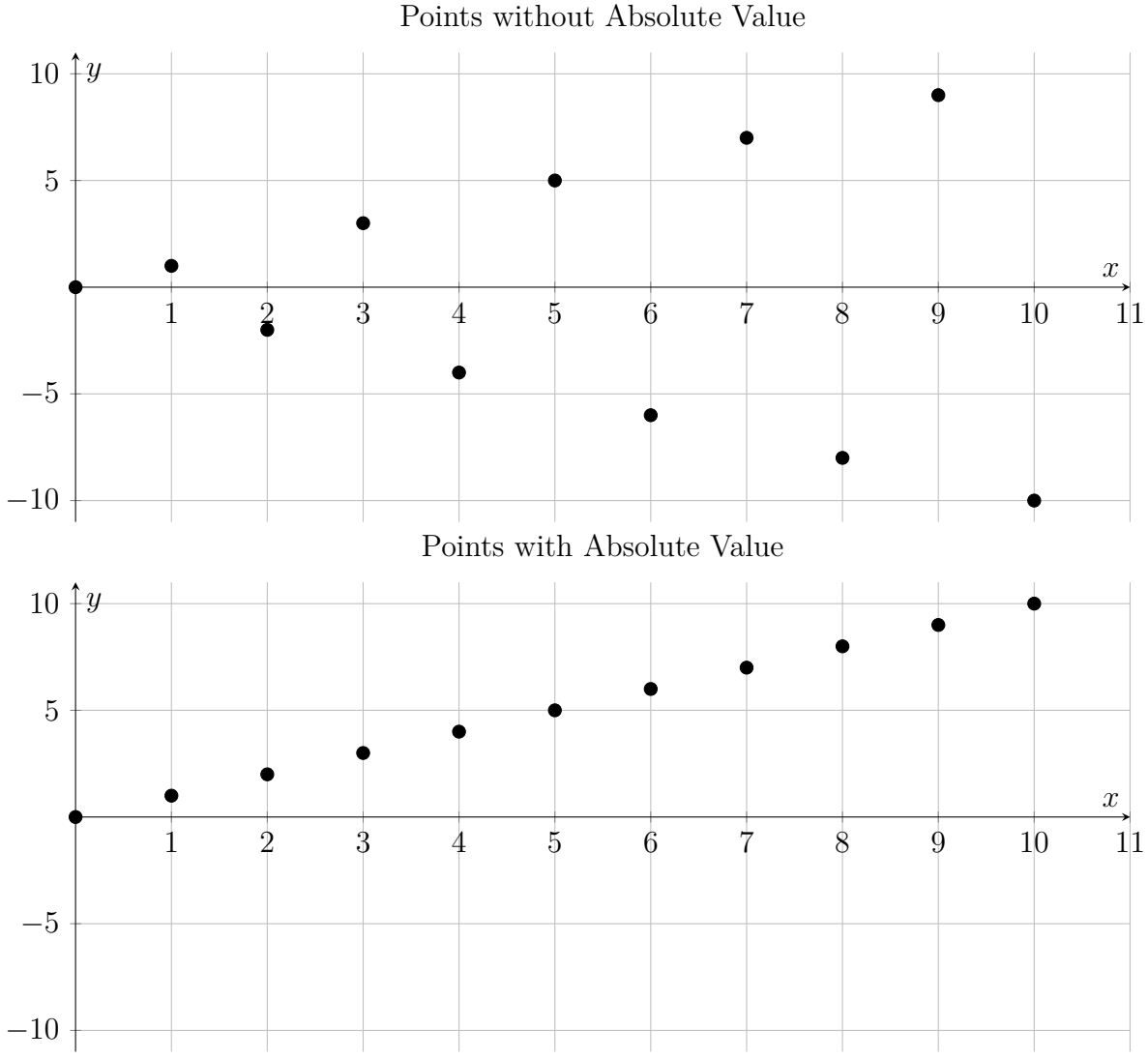
Part 4 - Python code skeleton

```
import torch

def E05(x):
    z = torch...
    return z

def main():
    x = torch.tensor...
    print(E05(...))

if __name__ == "__main__":
    main()
```

1.2.7 Exercise 06 - Tensor Absolute Value Law

Part 1 - why this matters

In artificial intelligence equations, the goal is not always to determine the direction of an error or the sign of a value, but rather to measure its true magnitude regardless of whether it is positive or negative. Positive and negative values represent different directions around a reference point, but when evaluating model performance or computing deviation, the sign can be misleading because it allows errors to cancel each other out during aggregation. For this reason, it becomes necessary to remove the sign and convert values to non-negative quantities, so that every deviation is counted as an independent contribution reflecting its actual intensity rather than its direction. This transformation ensures that no error disappears simply because an opposite error exists.



The absolute value provides a precise and straightforward way to achieve this goal without altering the numerical scale of values or artificially amplifying them. Unlike squaring, which changes the measurement unit and exaggerates large differences, the absolute value preserves linear relationships between values and enables direct, numerically stable measurement of deviation. This is why it is widely used in artificial intelligence equations whenever a fair and

balanced assessment of error or distance is required, especially in scenarios where excessive influence from large values must be avoided or clear interpretability of results is desired. In this sense, removing the sign is not a mathematical simplification, but a deliberate choice to separate measurement from direction within the model.

Part 2 - Mathematical form

Let $\mathbf{X} \in \mathbb{R}^{3\times3}$.

Tensor Absolute Value Law (element-wise) is defined as:

$$|\cdot| : \mathbb{R}^{3\times3} \to \mathbb{R}^{3\times3}$$

Define:

$$\mathbf{Z} = |\mathbf{X}|$$

where $\mathbf{Z} \in \mathbb{R}^{3\times3}$ and the law is defined component-wise by:

$$Z_{i,j} = |X_{i,j}| \quad \text{for } i,j \in \{1,2,3\}.$$

Part 3 - Hints and Helper

In PyTorch, the tensor absolute value (element-wise absolute value) can be computed using the built-in function `torch.abs()`.

The reader is encouraged to explore how `torch.abs()` removes the sign of each element independently and how it is commonly used in error measurement and magnitude-related computations.

Part 4 - Python code skeleton

```
import torch

def E06(x):
    z = torch...
    return z

def main():
    x = torch.tensor...
    print(E06(...))

if __name__ == "__main__":
    main()
```

1.2.8 Exercise 07 - Tensor Negation Law

Part 0.1 - Optimization and Gradients

In artificial intelligence, the term Optimization is not used in its general linguistic sense. Instead, it refers to the controlled adjustment of a model's numerical parameters within a high-dimensional mathematical space. Researchers treat the model as a purely numerical system

governed by a single objective function, and optimization defines how this system evolves over time toward configurations that reduce error according to a predefined criterion.

From a research perspective, Optimization transforms learning into a process that can be precisely managed and analyzed. Rather than being an abstract or emergent behavior, learning becomes a sequence of small, measurable parameter updates. This framing allows researchers to design learning algorithms whose behavior can be studied, compared, and refined, making model training both predictable and interpretable.

Within artificial intelligence research, the Gradient is understood as a mathematical sensing mechanism rather than merely a derivative. It provides local information about how sensitive the model is to small changes in its internal parameters, revealing the directions in which the objective function changes most rapidly.

In practice, the Gradient serves as raw directional information rather than a decision rule. Researchers do not follow it blindly; instead, they manipulate its influence through additional operations such as negation, scaling, or constraint enforcement. Thus, the Gradient supplies the signal, while Optimization defines how that signal is used to guide the learning process.

Part 0.2 - Difference and Error Function

In artificial intelligence, Difference is not understood as a simple subtraction operation, but as a foundational concept that signals the existence of a gap between two states: what the model produces and what it is expected to produce. Researchers treat Difference as the first mathematical acknowledgment that the current system deviates from its objective, without yet explaining the cause of that deviation.

From a research standpoint, Difference is the most primitive representation of error before any further processing. Once the difference is formulated, it can be treated as an independent mathematical object that can be scaled, transformed, sign-inverted, or passed through additional functions. For this reason, Difference is not the end of computation but its starting point, forming the basis for all subsequent correction steps.

The Error Function represents the next conceptual layer beyond Difference. In the researcher's view, an Error Function is a deliberate definition of what constitutes error and how severely it should be penalized. It is not merely a computational formula, but a formal policy that encodes the learning objective and determines which deviations matter and which can be ignored.

At a deeper level, the Error Function governs the long-term behavior of the model. It shapes the optimization landscape by deciding how raw differences are aggregated and weighted, thereby influencing the direction and stability of learning. As a result, changing the Error Function can fundamentally alter the learning dynamics of a system, even when the data and model architecture remain unchanged.

Part 0.3 - Signal directions and Opposition

In artificial intelligence, the direction of a signal is as important as its numerical magnitude. Researchers treat each signal as a force that pushes the model in a specific direction within the

solution space. The same numerical value can be beneficial or harmful depending on its sign, which means that model behavior cannot be understood by magnitude alone without considering direction.

From a training perspective, signal direction determines whether a proposed change moves the model closer to its objective or farther away from it. For this reason, many learning procedures prioritize controlling the direction of change before adjusting its strength. An incorrect direction, even if small, can accumulate over time and lead to significant divergence during training.

The concept of Opposition in artificial intelligence does not imply rejection, but rather a deliberate decision to act against a given signal direction. Researchers recognize that certain signals may carry valid information about the system while still leading to undesirable behavior if followed directly. Opposition allows this information to be used while preventing harmful movement.

At a deeper research level, opposition represents the point at which control is imposed on model behavior. Through operations such as negation, the learning process can exploit directional information without being dominated by it. This idea underlies practices like reversing the gradient during optimization and explains why tensor-level operations such as negation appear early and repeatedly in learning algorithms.

Part 1 - why this matters

The primary purpose of the Tensor Negation Law is to provide a simple and consistent way to reverse the effect of values without altering the structure or layout of the data. In many artificial intelligence systems, we do not need to introduce new information; instead, we need to express that an existing contribution should act in the opposite direction. Negation achieves this by preserving the same tensor shape and positions while inverting the sign of each value.

Tensor negation is used extensively because many core operations are based on comparison and correction. When computing differences between model outputs and target values, or when updating parameters during training, negation is required to move against an undesired direction. Without negation, it would be impossible to formally represent error, opposition, or corrective movement in a clean and structured mathematical way.

Most importantly, Tensor Negation preserves the tensor's internal structure. The dimensionality, alignment, and positional meaning of the data remain unchanged, while only the sign is inverted. This makes negation a safe and essential operation in learning systems, as it modifies numerical behavior without breaking the integrity of tensor-based representations or mixing information across locations.

Part 2 - Mathematical form

Let $\mathbf{X} \in \mathbb{R}^{3\times3}$.

Tensor Negation Law (element-wise) is defined as:

$$- : \mathbb{R}^{3\times3} \to \mathbb{R}^{3\times3}$$

Define:

$$\mathbf{Z} = -\mathbf{X}$$

where $\mathbf{Z} \in \mathbb{R}^{3\times3}$ and the law is defined component-wise by:

$$Z_{i,j} = -\,X_{i,j} \quad \text{for } i, j \in \{1, 2, 3\}.$$

Part 3 - Hints and Helper

In PyTorch, tensor negation (element-wise sign inversion) can be performed using the built-in function `torch.neg()`, or equivalently by applying the unary minus operator `-x`.

The reader is encouraged to explore how unary negation acts independently on each tensor entry without changing the tensor shape, and to compare the explicit use of `torch.neg()` with the operator form to understand their equivalence at the tensor level.

Part 4 - Python code skeleton

```
import torch

def E07(x):
    z = torch...
    return z

def main():
    x = torch.tensor...
    print(E07(...))

if __name__ == "__main__":
    main()
```

1.2.9 Exercise 08 - Tensor Sign Law

Part 1 - why this matters

In artificial intelligence systems, numerical values are constantly used to control decisions, updates, and internal signals. However, not every operation requires knowing the exact magnitude of a value. In many situations, what truly matters is the *direction* of influence rather than its size. Positive and negative values indicate opposite effects within the model, and identifying this direction alone is often sufficient to guide learning or adjustment processes.

Neural networks, in particular, repeatedly update their parameters based on signals that may vary widely in scale. Large values can cause unstable behavior, while very small values may become negligible during aggregation. By extracting only the sign of a tensor, the model can ignore harmful scale differences and focus solely on whether each component pushes the system forward, backward, or not at all. This leads to more stable and interpretable behavior in parts of the learning process where controlled directional movement is preferred over exact measurement.

Mathematically, the Tensor Sign Law allows the model to separate *decision* from *measurement*. Instead of treating numbers as precise quantities, the law converts them into directional indicators. This is especially useful in optimization and control-related operations, where consistent movement in the correct direction is more important than reacting strongly to large but possibly noisy values.

The Tensor Sign Law can be expressed in its simplest form as:

$$z = \text{sign}(x)$$

where:

- x represents a real-valued scalar or a single element of a tensor.
- $\text{sign}(x)$ extracts the direction of x.
- $z = 1$ if $x > 0$, indicating a positive direction.
- $z = -1$ if $x < 0$, indicating a negative direction.
- $z = 0$ if $x = 0$, indicating no directional influence.

When applied element-wise to tensors, this operation enables neural networks to reason about directional behavior independently of magnitude, forming a foundation for stable updates, directional analysis, and simplified control mechanisms within artificial intelligence models.

Part 2 - Mathematical form

Let $\mathbf{X} \in \mathbb{R}^{3\times3}$.

Tensor Sign Law (element-wise) is defined as:

$$\text{sign}(\cdot) : \mathbb{R}^{3\times3} \to \mathbb{R}^{3\times3}$$

Define:

$$\mathbf{Z} = \text{sign}(\mathbf{X})$$

where $\mathbf{Z} \in \mathbb{R}^{3\times3}$ and the law is defined component-wise by:

$$Z_{i,j} = \begin{cases} 1 & \text{if } X_{i,j} > 0 \\ 0 & \text{if } X_{i,j} = 0 \\ -1 & \text{if } X_{i,j} < 0 \end{cases} \quad \text{for } i, j \in \{1, 2, 3\}.$$

Part 3 - Hints and Helper

In PyTorch, the tensor sign operation (element-wise sign extraction) can be computed using the built-in function `torch.sign()`.

The reader is encouraged to explore how `torch.sign()` separates *direction* from *magnitude*, returning only the sign of each element, and how this operation is commonly paired with absolute value or norm based laws in optimization, gradient analysis, and control of update direction.

Part 4 - Python code skeleton

```
import torch

def E08(x):
    z = torch...
    return z

def main():
    x = torch.tensor...
    print(E08(...))

if __name__ == "__main__":
    main()
```

1.2.10 Exercise 09 - Tensor Clamp Law

Part 0 - conceptual overview

In a neural network, all numerical behavior revolves around a small set of interconnected quantities: weights, activations, scores, and gradients. These elements do not exist independently; instead, they form a continuous feedback loop that drives learning and decision-making within the model.

Weights represent the internal parameters of the network and determine how input information is transformed. Activations are the intermediate signals produced when inputs are combined with weights and passed through nonlinear functions. Scores are the final numerical outputs of the network before interpretation, often representing confidence, preference, or likelihood. Together, weights and activations shape the scores produced by the model.

Gradients complete the loop by measuring how changes in weights affect the final scores and error. They flow backward through the network and guide how weights should be adjusted. Because activations, scores, and gradients can grow unbounded or become unstable, controlling their numerical range becomes essential. Understanding this relationship explains why mechanisms such as clamping are needed to keep all components operating within safe and meaningful limits.

Part 1 - why this matters

Artificial intelligence systems continuously generate numerical values while processing data, updating parameters, and producing outputs. These values are not inherently constrained and can easily grow too large or become too small during training or inference. When such uncontrolled values propagate through a neural network, they may cause numerical instability, saturation, or unreliable behavior that degrades the overall performance of the model.

The Tensor Clamp Law introduces explicit boundaries that restrict values to a predefined and meaningful range. By enforcing lower and upper limits, the model is protected from extreme values that do not carry additional useful information. This controlled restriction allows learning processes to remain stable while still preserving the essential structure of the data. Instead of eliminating information, clamping preserves valid values and only corrects those that

exceed acceptable limits.

Within neural networks, clamping is commonly used to maintain safe operating ranges for activations, gradients, and outputs. It ensures that numerical values remain interpretable and physically or logically valid throughout computation. In this sense, the Tensor Clamp Law acts as a stabilizing mechanism that enables artificial intelligence systems to learn effectively without being disrupted by unbounded numerical behavior.

Part 2 - Mathematical form

Let $\mathbf{X} \in \mathbb{R}^{3\times3}$ and let $a, b \in \mathbb{R}$ with $a \leq b$.
Tensor Clamp Law (element-wise) is defined as:

$$\text{clamp}_{[a,b]}(\cdot) : \mathbb{R}^{3\times3} \to \mathbb{R}^{3\times3}$$

Define:

$$\mathbf{Z} = \text{clamp}_{[a,b]}(\mathbf{X})$$

where $\mathbf{Z} \in \mathbb{R}^{3\times3}$ and the law is defined component-wise by:

$$Z_{i,j} = \begin{cases} a & \text{if } X_{i,j} < a \\ X_{i,j} & \text{if } a \leq X_{i,j} \leq b \\ b & \text{if } X_{i,j} > b \end{cases} \quad \text{for } i, j \in \{1, 2, 3\}.$$

Part 3 - Hints and Helper

In PyTorch, the tensor clamp operation (element-wise value limiting) can be computed using the built-in function `torch.clamp()`.

The reader is encouraged to explore how `torch.clamp()` enforces lower and upper bounds on each tensor element independently, preventing values from exceeding predefined limits and ensuring numerical stability during computation.

Part 4 - Python code skeleton

```
import torch

def E09(x, m, M):
    z = torch...
    return z

def main():
    x = torch.tensor...
    m =...
    M =...
    print(E09(...))

if __name__ == "__main__":
    main()
```

1.2.11 Exercise 10 - Tensor Round Law

Part 1 - why this matters

Neural networks operate primarily in a continuous numerical space, where values such as weights, activations, and scores can take any real number. This continuous representation is essential for learning and optimization, but it does not always align with the requirements of decision-making or real-world execution. In many situations, a model must eventually produce discrete, integer-based outcomes rather than continuous values.

The Tensor Round Law provides a controlled way to convert continuous values into stable integer representations. By rounding values to the nearest integer, small numerical fluctuations are removed without introducing large distortions. This is particularly important when values are close to decision boundaries, where minor changes could otherwise cause inconsistent or unstable behavior.

Within artificial intelligence systems, rounding is commonly used when transitioning from internal numerical processing to final outputs, counters, or index-based operations. It enables models to move from soft, approximate representations to clear and interpretable decisions, ensuring that continuous learning processes can be safely integrated with discrete system requirements.

Part 2 - Mathematical form

Let $\mathbf{X} \in \mathbb{R}^{3\times3}$.

Tensor Round Law (element-wise) is defined as:

$$\text{round}(\cdot) : \mathbb{R}^{3\times3} \to \mathbb{R}^{3\times3}$$

Define:

$$\mathbf{Z} = \text{round}(\mathbf{X})$$

where $\mathbf{Z} \in \mathbb{R}^{3\times3}$ and the law is defined component-wise by:

$$Z_{i,j} = \text{round}(X_{i,j}) \quad \text{for } i, j \in \{1, 2, 3\}.$$

Part 3 - Hints and Helper

In PyTorch, the tensor rounding operation (element-wise rounding to the nearest integer) can be computed using the built-in function `torch.round()`.

The reader is encouraged to explore how `torch.round()` converts continuous values into discrete ones by rounding each element independently, and how this operation is commonly used when transitioning from continuous representations to discrete decisions or stabilized numerical outputs.

Part 4 - Python code skeleton

```
import torch

def E10(x):
```

```
    z = torch...
    return z

def main():
    x = torch.tensor...
    print(E10(...))

if __name__ == "__main__":
    main()
```

1.2.12 Exercise 11 - Tensor Floor Law

Part 1 - why this matters

Artificial intelligence systems often work with continuous numerical values, yet many decisions and operations within these systems require discrete and strictly bounded outcomes. In such cases, fractional values cannot be interpreted directly, and a clear rule is needed to convert continuous quantities into valid integers without violating system constraints.

The Tensor Floor Law provides a conservative transformation by mapping each value to the largest integer that is less than or equal to it. This guarantees that the result never exceeds the original value, making it suitable for situations where upper limits must be respected. By consistently rounding downward, the operation prevents unintended overestimation that could lead to invalid decisions or resource misuse.

Within artificial intelligence pipelines, floor-based transformations are used when values represent counts, indices, or execution steps that must remain within safe and allowable bounds. The Tensor Floor Law ensures that continuous numerical outputs can be safely translated into discrete, actionable values while preserving logical consistency throughout the system.

Part 2 - Mathematical form

Let $\mathbf{X} \in \mathbb{R}^{3\times3}$.

Tensor Floor Law (element-wise) is defined as:

$$\lfloor \cdot \rfloor : \mathbb{R}^{3\times3} \to \mathbb{R}^{3\times3}$$

Define:

$$\mathbf{Z} = \lfloor \mathbf{X} \rfloor$$

where $\mathbf{Z} \in \mathbb{R}^{3\times3}$ and the law is defined component-wise by:

$$Z_{i,j} = \lfloor X_{i,j} \rfloor \quad \text{for } i, j \in \{1, 2, 3\}.$$

Part 3 - Hints and Helper

In PyTorch, the tensor floor operation (element-wise rounding down to the nearest integer) can be computed using the built-in function `torch.floor()`.

The reader is encouraged to explore how `torch.floor()` consistently rounds values downward regardless of their fractional part, and how this operation is commonly used when enforcing conservative or lower-bound numerical decisions in artificial intelligence systems.

Part 4 - Python code skeleton

```
import torch

def E11(x):
    z = torch...
    return z

def main():
    x = torch.tensor...
    print(E11(...))

if __name__ == "__main__":
    main()
```

1.2.13 Exercise 12 - Tensor Ceil Law

Part 0 - Conservation and Safeguarding

Artificial intelligence systems are fundamentally numerical systems that rely on continuous values to represent uncertainty, estimates, and learned behavior. However, these systems do not operate in isolation; they must eventually interact with finite computational resources, discrete execution steps, and real-world constraints. As a result, numerical decisions in artificial intelligence are not only mathematical choices but also design decisions that reflect how the system handles risk, error, and failure.

Two fundamental design philosophies emerge when continuous values must be converted into discrete decisions: *conservation* and *safeguarding*. These philosophies define how an artificial intelligence system reacts when faced with uncertainty in numerical estimation. Rather than optimizing for mathematical elegance, they optimize for system reliability and practical correctness.

Conservation represents a cautious approach that prioritizes efficiency and resource control. Under this philosophy, the system avoids allocating or executing more than what is strictly justified by the current numerical estimate. The underlying assumption is that overestimation is more harmful than underestimation. Conservation is therefore used when excessive allocation, unnecessary computation, or inflated execution can degrade performance, increase cost, or destabilize the system.

Safeguarding represents a complementary but fundamentally different philosophy. Instead of minimizing usage, it prioritizes completeness and correctness. Under this approach, the system ensures that requirements are fully met, even if this means allocating slightly more resources or executing additional steps. The core assumption of safeguarding is that underestimation is more dangerous than overestimation, especially when missing data, incomplete processing, or early termination can lead to silent failures.

In artificial intelligence development, neither philosophy is universally superior. Each reflects a different tolerance for risk. Conservation accepts the possibility of doing less in exchange for efficiency, while safeguarding accepts controlled excess in exchange for reliability. The choice between them is not a mathematical preference but a system-level judgment about which type

of failure is unacceptable in a given context.

These philosophies directly influence how numerical transformations are applied within neural networks and AI pipelines. When values represent optional capacity, iterative refinement, or expandable processes, conservation-oriented decisions are often appropriate. When values represent mandatory coverage, irreversible decisions, or completeness requirements, safeguarding-oriented decisions become essential.

Understanding conservation and safeguarding as design principles allows AI developers to reason more clearly about numerical transformations such as floor and ceiling operations. Rather than viewing these operations as simple rounding rules, they can be understood as explicit expressions of system intent, encoding how the model chooses to balance efficiency against reliability within its computational logic.

Part 1 - why this matters

Artificial intelligence systems frequently work with continuous numerical values that represent estimated quantities such as required steps, resource usage, or data coverage. However, many execution-level decisions within these systems cannot operate on fractional values and must rely on integer-based quantities that fully satisfy the underlying requirement.

The Tensor Ceil Law provides a safe transformation by mapping each value to the smallest integer greater than or equal to it. This guarantees that the resulting value never underestimates the original quantity. By enforcing upward rounding, the model avoids incomplete processing, missed data, or insufficient allocation that could otherwise occur when fractional estimates are truncated.

Within artificial intelligence pipelines, ceiling-based transformations are used when full coverage and completeness are more important than minimizing numerical overhead. The Tensor Ceil Law ensures that continuous estimates are converted into discrete decisions that meet or exceed required thresholds, enabling reliable and robust system behavior.

Part 2 - Mathematical form

Let $\mathbf{X} \in \mathbb{R}^{3\times3}$.

Tensor Ceil Law (element-wise) is defined as:

$$\lceil \cdot \rceil : \mathbb{R}^{3\times3} \to \mathbb{R}^{3\times3}$$

Define:

$$\mathbf{Z} = \lceil \mathbf{X} \rceil$$

where $\mathbf{Z} \in \mathbb{R}^{3\times3}$ and the law is defined component-wise by:

$$Z_{i,j} = \lceil X_{i,j} \rceil \quad \text{for } i, j \in \{1, 2, 3\}.$$

Part 3 - Hints and Helper

In PyTorch, the tensor ceiling operation (element-wise rounding up to the nearest integer) can be computed using the built-in function `torch.ceil()`.

The reader is encouraged to explore how `torch.ceil()` consistently maps each value to the smallest integer greater than or equal to it, and how this operation is used when enforcing upward-safe or minimum-satisfying numerical decisions in artificial intelligence systems.

Part 4 - Python code skeleton

```
import torch

def E12(x):
    z = torch...
    return z

def main():
    x = torch.tensor...
    print(E12(...))

if __name__ == "__main__":
    main()
```

1.2.14 Exercise 13 - Tensor Reciprocal Law

Part 1 - why this matters

Artificial intelligence systems frequently rely on numerical values that represent relative importance, distance, confidence, or influence. In many such cases, a larger numerical value does not necessarily imply a stronger or more desirable effect. Instead, certain relationships require an inverse interpretation, where smaller values should exert greater influence and larger values should contribute less to the final outcome.

The Tensor Reciprocal Law enables this inverse relationship by transforming each value into its multiplicative inverse. Through this transformation, large values are attenuated while small values are amplified, allowing the model to rebalance the contribution of different elements. This is particularly useful when values encode penalties, distances, or uncertainty measures, where dominance by large magnitudes would otherwise distort the model's behavior.

Within artificial intelligence pipelines, reciprocal-based transformations are used to regulate influence, normalize contributions, and prevent disproportionate effects from dominating learning or decision processes. By explicitly reversing the numerical relationship, the Tensor Reciprocal Law provides a principled way to express inverse dependence, enabling more stable and meaningful interactions between numerical components in neural networks and related systems.

Part 2 - Mathematical form

Let $\mathbf{X} \in \mathbb{R}^{3\times3}$ such that $X_{i,j} \neq 0$ for all $i, j \in \{1, 2, 3\}$.
Tensor Reciprocal Law (element-wise) is defined as:

$$(\cdot)^{-1} : \mathbb{R}^{3\times3} \setminus \{0\} \to \mathbb{R}^{3\times3}$$

Define:

$$\mathbf{Z} = \mathbf{X}^{-1}$$

where $\mathbf{Z} \in \mathbb{R}^{3\times3}$ and the law is defined component-wise by:

$$Z_{i,j} = \frac{1}{X_{i,j}} \quad \text{for } i,j \in \{1,2,3\}.$$

Part 3 - Hints and Helper

In PyTorch, the tensor reciprocal operation (element-wise inversion) can be computed using the built-in function `torch.reciprocal()`.

The reader is encouraged to explore how `torch.reciprocal()` computes the inverse of each element independently, and why special care must be taken to avoid zero values, as division by zero leads to undefined or unstable numerical behavior in artificial intelligence systems.

Part 4 - Python code skeleton

```
import torch

def E13(x):
    z = torch...
    return z

def main():
    x = torch.tensor...
    print(E13(...))

if __name__ == "__main__":
    main()
```

1.2.15 Exercise 14 - Tensor Modulo Law

Part 1 - why this matters

Artificial intelligence systems often manipulate numerical values that grow unbounded through accumulation, iteration, or repeated updates. While such growth is mathematically valid, it is not always meaningful for the behavior the model is intended to represent. In many scenarios, the absolute size of a value is less important than its position within a fixed and repeating range.

The Tensor Modulo Law provides a principled way to map unbounded values back into a bounded, periodic domain. By computing the remainder of a division, the operation preserves the relative position of a value within a cycle while discarding the number of completed cycles. This allows models to reason about states, phases, or positions without being affected by uncontrolled numerical growth.

Within artificial intelligence pipelines, modulo-based transformations are used whenever periodic structure, bounded indexing, or cyclic behavior is required. They enable models to operate within finite domains, prevent numerical overflow, and encode repeating patterns directly into mathematical formulations. In this sense, the Tensor Modulo Law is not merely a

numerical convenience, but a formal mechanism for expressing periodicity and controlled value spaces in artificial intelligence systems.

Part 2 - Mathematical form

Let $\mathbf{X} \in \mathbb{R}^{3\times3}$ and let $m \in \mathbb{R}$ with $m \neq 0$.
Tensor Modulo Law (element-wise) is defined as:

$$\mathrm{mod}_m(\cdot) : \mathbb{R}^{3\times3} \to \mathbb{R}^{3\times3}$$

Define:

$$\mathbf{Z} = \mathbf{X} \bmod m$$

where $\mathbf{Z} \in \mathbb{R}^{3\times3}$ and the law is defined component-wise by:

$$Z_{i,j} = X_{i,j} \bmod m \quad \text{for } i, j \in \{1, 2, 3\}.$$

Part 3 - Hints and Helper

In PyTorch, the tensor modulo operation (element-wise remainder computation) can be performed using the built-in function `torch.remainder()`.

The reader is encouraged to explore how `torch.remainder()` computes the remainder of each tensor element with respect to a given modulus, and how this operation is commonly used in periodic behavior, index wrapping, and bounded numerical representations within artificial intelligence systems.

Part 4 - Python code skeleton

```
import torch

def E14(x, m):
    z = torch...
    return z

def main():
    x = torch.tensor...
    m =...
    print(E14(...))

if __name__ == "__main__":
    main()
```

1.2.16 Exercise 15 - Tensor Logical AND Law

Part 1 - why this matters

Artificial intelligence systems frequently make decisions based on multiple conditions rather than a single numerical value. In many scenarios, an action, update, or computation should only occur when several requirements are satisfied simultaneously. This necessity cannot be

expressed through standard arithmetic operations alone and requires an explicit logical mechanism.

The Tensor Logical AND Law provides a formal way to encode simultaneous validity within mathematical expressions. By combining multiple boolean conditions into a single logical outcome, the operation ensures that a result is considered valid only when all contributing conditions are true. This allows decision rules to be represented directly inside tensor-based computations, rather than being handled externally through procedural logic.

Within artificial intelligence pipelines, logical AND operations are widely used to construct masks, enforce constraints, and control conditional execution. They enable models to selectively activate or deactivate elements based on multiple criteria at once, ensuring consistency and correctness in complex decision processes. In this sense, the Tensor Logical AND Law serves as a foundational tool for integrating logical reasoning into numerical and tensor-based AI systems.

Part 2 - Mathematical form

Let $\mathbf{X}, \mathbf{Y} \in \{0,1\}^{3\times 3}$.

Tensor Logical AND Law (element-wise) is defined as:

$$\wedge : \{0,1\}^{3\times 3} \times \{0,1\}^{3\times 3} \to \{0,1\}^{3\times 3}$$

Define:

$$\mathbf{Z} = \mathbf{X} \wedge \mathbf{Y}$$

where $\mathbf{Z} \in \{0,1\}^{3\times 3}$ and the law is defined component-wise by:

$$Z_{i,j} = \begin{cases} 1 & \text{if } X_{i,j} = 1 \text{ and } Y_{i,j} = 1 \\ 0 & \text{otherwise} \end{cases} \quad \text{for } i,j \in \{1,2,3\}.$$

Part 3 - Hints and Helper

In PyTorch, the tensor logical AND operation (element-wise logical conjunction) can be computed using the built-in function `torch.logical_and()`.

The reader is encouraged to explore how `torch.logical_and()` evaluates each pair of tensor elements independently, and how logical conjunction is used to enforce simultaneous conditions, masks, and rule-based constraints in artificial intelligence systems.

Part 4 - Python code skeleton

```
import torch

def E15(X, Y):
    z = torch...
    return z

def main():
    X = torch.tensor...
    Y = torch.tenosr...
```

```
    print(E15(...))

if __name__ == "__main__":
    main()
```

1.2.17 Exercise 16 - Tensor Logical OR Law

Part 1 - why this matters

Artificial intelligence systems often operate in environments where information is incomplete, uncertain, or noisy. In such contexts, requiring all conditions to be satisfied simultaneously can make decision processes overly rigid and fragile. Instead, many AI systems must allow actions or conclusions to be validated through alternative acceptable conditions.

The Tensor Logical OR Law provides a formal mechanism for expressing such flexibility within mathematical formulations. By evaluating multiple conditions and accepting a result when at least one condition is true, the operation enables models to continue functioning even when some inputs or criteria fail. This supports robustness and adaptability in complex or imperfect environments.

Within artificial intelligence pipelines, logical OR operations are commonly used to construct fallback rules, permissive masks, and alternative decision paths. They allow models to represent "either–or" logic directly within tensor-based computations, ensuring that decision-making remains resilient without sacrificing logical clarity.

Part 2 - Mathematical form

Let $\mathbf{X}, \mathbf{Y} \in \{0,1\}^{3\times 3}$.

Tensor Logical OR Law (element-wise) is defined as:

$$\vee : \{0,1\}^{3\times 3} \times \{0,1\}^{3\times 3} \to \{0,1\}^{3\times 3}$$

Define:

$$\mathbf{Z} = \mathbf{X} \vee \mathbf{Y}$$

where $\mathbf{Z} \in \{0,1\}^{3\times 3}$ and the law is defined component-wise by:

$$Z_{i,j} = \begin{cases} 1 & \text{if } X_{i,j} = 1 \text{ or } Y_{i,j} = 1 \\ 0 & \text{otherwise} \end{cases} \quad \text{for } i,j \in \{1,2,3\}.$$

Part 3 - Hints and Helper

In PyTorch, the tensor logical OR operation (element-wise logical disjunction) can be computed using the built-in function `torch.logical_or()`.

The reader is encouraged to explore how `torch.logical_or()` evaluates each pair of tensor elements independently, and how logical disjunction is used to represent alternative conditions, fallback rules, and flexible masking in artificial intelligence systems.

Part 4 - Python code skeleton

```
import torch

def E16(X, Y):
    z = torch...
    return z

def main():
    X = torch.tensor...
    Y = torch.tenosr...
    print(E16(...))

if __name__ == "__main__":
    main()
```

1.2.18 Exercise 17 - Tensor Logical NOT Law

Part 0 - combined logical reasoning

Artificial intelligence systems rarely make decisions based on a single condition. Instead, real-world decision-making requires evaluating multiple constraints, allowing alternatives, and explicitly excluding invalid states. Logical operators such as AND, OR, and NOT each capture a distinct aspect of this process, but none of them alone is sufficient to express complete decision logic.

Combining logical AND, OR, and NOT into a single mathematical expression allows an AI system to represent full decision policies rather than isolated rules. Logical AND enforces simultaneous validity, ensuring that all required conditions are satisfied. Logical OR introduces flexibility by allowing multiple acceptable paths to a valid outcome. Logical NOT provides exclusion by explicitly rejecting states or conditions that must not occur. Together, these operators form a complete logical language for decision control.

The benefit of combining these operators lies in clarity and correctness. A single unified logical equation explicitly defines when an action is permitted, when it is rejected, and when alternative conditions are acceptable. This eliminates hidden assumptions, scattered conditional logic, and ambiguous decision pathways that often arise when logic is implemented procedurally rather than mathematically.

In artificial intelligence pipelines, such combined logical expressions are commonly used to construct complex masks, enforce hierarchical constraints, and control conditional execution within tensor-based computations. By embedding logical structure directly into mathematical formulations, models can apply decision rules efficiently and consistently across large-scale data structures.

Moreover, representing logic as a single equation enables analysis and verification. The combined expression can be simplified, tested for completeness, and examined for edge cases. This makes the decision process more transparent and robust, which is essential when AI systems operate in uncertain or safety-critical environments.

Ultimately, the combination of AND, OR, and NOT transforms logical reasoning from a set of disconnected checks into a coherent decision framework. Rather than asking isolated questions such as "is this condition true?", the system evaluates a complete policy that defines acceptance, rejection, and permissible alternatives in a unified and mathematically rigorous manner.

Part 1 - why this matters

Artificial intelligence systems frequently rely on boolean conditions to control execution, enforce constraints, and construct decision logic. In many situations, it is not sufficient to check whether a condition is true; the system must also reason about when a condition is explicitly false. This requires a formal way to invert logical states within mathematical expressions.

The Tensor Logical NOT Law provides a precise mechanism for logical negation, allowing models to reverse boolean values in a consistent and interpretable manner. By transforming true conditions into false ones and vice versa, the operation enables artificial intelligence systems to express exclusion, invalidation, and complementary conditions directly within tensor computations.

Within artificial intelligence pipelines, logical NOT operations are commonly used to deactivate masks, exclude invalid states, and construct complementary rules. They allow models to reason not only about what is permitted, but also about what must be explicitly rejected, ensuring clarity and completeness in logical decision-making processes.

Part 2 - Mathematical form

Let $\mathbf{X} \in \{0,1\}^{3\times 3}$.

Tensor Logical NOT Law (element-wise) is defined as:

$$\neg : \{0,1\}^{3\times 3} \to \{0,1\}^{3\times 3}$$

Define:

$$\mathbf{Z} = \neg \mathbf{X}$$

where $\mathbf{Z} \in \{0,1\}^{3\times 3}$ and the law is defined component-wise by:

$$Z_{i,j} = \begin{cases} 1 & \text{if } X_{i,j} = 0 \\ 0 & \text{if } X_{i,j} = 1 \end{cases} \quad \text{for } i,j \in \{1,2,3\}.$$

Part 3 - Hints and Helper

In PyTorch, the tensor logical NOT operation (element-wise logical negation) can be computed using the built-in function `torch.logical_not()`.

The reader is encouraged to explore how `torch.logical_not()` inverts each boolean tensor element independently, and how logical negation is used to exclude conditions, reverse masks, and express complementary constraints in artificial intelligence systems.

Part 4 - Python code skeleton

```
import torch

def E17(X):
    z = torch...
    return z

def main():
    X = torch.tensor...
    print(E17(...))

if __name__ == "__main__":
    main()
```

1.2.19 Exercise 18 - Tensor Equality Law (Comparison)

Part 0 - comparison vs logical operations

Artificial intelligence systems rely on both numerical evaluation and logical decision-making. Although comparison operations and logical operations may appear similar at a surface level, they serve fundamentally different roles within AI pipelines. Understanding the distinction between them is essential for designing clear, correct, and interpretable decision mechanisms.

Comparison operations are concerned with evaluating relationships between values. They answer questions such as whether two values are equal, or whether one value is greater or smaller than another. The result of a comparison is a boolean outcome derived directly from numerical properties. In this sense, comparison acts as a bridge between continuous numerical representations and discrete decision boundaries.

Logical operations, on the other hand, do not evaluate numerical relationships. Instead, they operate on boolean conditions that already represent decisions, validity, or state membership. Logical AND, OR, and NOT combine, relax, or invert these boolean states to construct higher-level decision logic. Their purpose is not to measure or compare, but to control how conditions interact.

In artificial intelligence workflows, comparison operations typically appear first. Numerical outputs from models are compared against thresholds, targets, or reference values to produce boolean indicators. These indicators alone are often insufficient to represent complex decision policies, as real-world decisions rarely depend on a single condition.

Logical operations are then applied to the results of comparisons. They combine multiple boolean signals, enforce constraints, allow alternatives, or exclude invalid states. This separation of roles ensures clarity: comparisons define elementary conditions, while logical operators define how those conditions govern behavior.

Treating comparison and logical operations as distinct stages prevents conceptual confusion and improves system design. Comparisons define "what is true" at the numerical level, while logical operations define "what should be done" at the decision level. Together, they form a structured pathway from raw numerical outputs to coherent and controllable artificial intelli-

gence behavior.

Part 1 - why this matters

Artificial intelligence systems frequently need to determine whether two values represent the same state, outcome, or decision rather than measuring how far apart they are. In such situations, numerical distance is less relevant than exact agreement, and a clear boundary must be drawn between matching and non-matching values.

The Tensor Equality Law provides a formal mechanism for performing exact comparisons within tensor-based computations. By converting element-wise comparisons into boolean outcomes, the operation allows models to identify matching conditions, validate predictions, and construct decision masks that separate valid states from invalid ones.

Within artificial intelligence pipelines, equality comparisons are essential for evaluation, rule enforcement, and conditional execution. They enable systems to verify correctness, trigger specific actions, and control subsequent processing steps based on precise matching criteria. In this sense, the Tensor Equality Law serves as a foundational tool for transforming numerical outputs into explicit logical decisions.

Part 2 - Mathematical form

Let $\mathbf{X}, \mathbf{Y} \in \mathbb{R}^{3\times3}$.

Tensor Equality Law (element-wise comparison) is defined as:

$$=: \mathbb{R}^{3\times3} \times \mathbb{R}^{3\times3} \to \{0,1\}^{3\times3}$$

Define:

$$\mathbf{Z} = (\mathbf{X} = \mathbf{Y})$$

where $\mathbf{Z} \in \{0,1\}^{3\times3}$ and the law is defined component-wise by:

$$Z_{i,j} = \begin{cases} 1 & \text{if } X_{i,j} = Y_{i,j} \\ 0 & \text{if } X_{i,j} \neq Y_{i,j} \end{cases} \quad \text{for } i,j \in \{1,2,3\}.$$

Part 3 - Hints and Helper

In PyTorch, the tensor equality comparison (element-wise equality check) can be performed using the built-in function `torch.eq()`.

The reader is encouraged to explore how `torch.eq()` compares each pair of tensor elements independently and produces a boolean tensor that represents exact equality, which is commonly used for masking, condition checking, and rule-based logic in artificial intelligence systems.

Part 4 - Python code skeleton

```
import torch

def E18(X, Y):
    z = torch...
```

```
    return z

def main():
    X = torch...
    Y = torch...
    print(E18(...))

if __name__ == "__main__":
    main()
```

1.2.20 Exercise 19 - Tensor Not Equal Law (Comparison)

Part 1 - why this matters

Artificial intelligence systems must not only confirm correct or expected states, but also detect when deviations occur. In many scenarios, meaningful actions are triggered not by agreement, but by difference. Identifying mismatches between values is therefore essential for monitoring, correction, and adaptive behavior.

The Tensor Not Equal Law provides a formal mechanism for detecting discrepancies within tensor-based computations. By producing a boolean outcome for each comparison, the operation highlights where values diverge from references, targets, or prior states. This allows models to isolate irregularities and focus attention on elements that require intervention.

Within artificial intelligence pipelines, not-equal comparisons are commonly used for error detection, change tracking, and conditional filtering. They enable systems to respond explicitly to unexpected outcomes rather than silently accepting them. In this sense, the Tensor Not Equal Law transforms numerical differences into actionable signals that guide correction and control within AI systems.

Part 2 - Mathematical form

Let $\mathbf{X}, \mathbf{Y} \in \mathbb{R}^{3\times3}$.

Tensor Not Equal Law (element-wise comparison) is defined as:

$$\neq: \mathbb{R}^{3\times3} \times \mathbb{R}^{3\times3} \to \{0,1\}^{3\times3}$$

Define:

$$\mathbf{Z} = (\mathbf{X} \neq \mathbf{Y})$$

where $\mathbf{Z} \in \{0,1\}^{3\times3}$ and the law is defined component-wise by:

$$Z_{i,j} = \begin{cases} 1 & \text{if } X_{i,j} \neq Y_{i,j} \\ 0 & \text{if } X_{i,j} = Y_{i,j} \end{cases} \quad \text{for } i,j \in \{1,2,3\}.$$

Part 3 - Hints and Helper

In PyTorch, the tensor not-equal comparison (element-wise inequality check) can be computed using the built-in function `torch.ne()`.

The reader is encouraged to explore how `torch.ne()` compares each pair of tensor elements independently and produces a boolean tensor that highlights mismatched values, which is commonly used for masking, filtering, and error detection in artificial intelligence systems.

Part 4 - Python code skeleton

```
import torch

def E19(X, Y):
    z = torch...
    return z

def main():
    X = torch...
    Y = torch...
    print(E19(...))

if __name__ == "__main__":
    main()
```

1.2.21 Exercise 20 - Tensor Greater Than Law (Comparison)

Part 1 - why this matters

Artificial intelligence systems often rely on threshold-based decisions rather than exact matches. In many cases, the precise value of a quantity is less important than whether it exceeds a certain level. Such comparisons are fundamental when determining activation, eligibility, or dominance within a model.

The Tensor Greater Than Law enables artificial intelligence systems to identify values that surpass reference points or constraints. By converting numerical comparisons into boolean outcomes, the operation allows models to distinguish significant signals from insignificant ones and to trigger decisions based on exceeding predefined criteria.

Within artificial intelligence pipelines, greater-than comparisons are commonly used to enforce thresholds, activate conditional logic, and control flow based on magnitude. They provide a clear and interpretable boundary between acceptable and insufficient values, making them essential for decision-oriented numerical processing.

Part 2 - Mathematical form

Let $\mathbf{X}, \mathbf{Y} \in \mathbb{R}^{3\times3}$.

Tensor Greater Than Law (element-wise comparison) is defined as:

$$>: \mathbb{R}^{3\times3} \times \mathbb{R}^{3\times3} \to \{0,1\}^{3\times3}$$

Define:

$$\mathbf{Z} = (\mathbf{X} > \mathbf{Y})$$

where $\mathbf{Z} \in \{0,1\}^{3\times3}$ and the law is defined component-wise by:

$$Z_{i,j} = \begin{cases} 1 & \text{if } X_{i,j} > Y_{i,j} \\ 0 & \text{if } X_{i,j} \leq Y_{i,j} \end{cases} \quad \text{for } i,j \in \{1,2,3\}.$$

Part 3 - Hints and Helper

In PyTorch, the tensor greater-than comparison (element-wise comparison) can be computed using the built-in function `torch.gt()`.

The reader is encouraged to explore how `torch.gt()` compares tensor elements independently and produces a boolean tensor that identifies values exceeding a given reference, which is commonly used for thresholding, masking, and conditional execution in artificial intelligence systems.

Part 4 - Python code skeleton

```
import torch

def E20(X, Y):
    z = torch...
    return z

def main():
    X = torch...
    Y = torch...
    print(E20(...))

if __name__ == "__main__":
    main()
```

1.2.22 Exercise 21 - Tensor Less Than Law (Comparison)

Part 1 - why this matters

Artificial intelligence systems frequently make decisions based on identifying insufficient, weak, or sub-threshold values rather than dominant ones. In many scenarios, recognizing when a value falls below a certain level is essential for filtering, suppression, or corrective action within a model.

The Tensor Less Than Law allows artificial intelligence systems to detect values that do not meet required criteria. By converting numerical comparisons into boolean outcomes, the operation enables models to isolate underperforming signals, enforce lower-bound constraints, and control behavior based on insufficiency rather than excess.

Within artificial intelligence pipelines, less-than comparisons are commonly used to deactivate elements, identify low-confidence regions, and regulate conditional execution. They provide a clear mechanism for detecting when values fail to reach necessary thresholds, supporting stable and controlled decision-making.

Part 2 - Mathematical form

Let $\mathbf{X}, \mathbf{Y} \in \mathbb{R}^{3\times3}$.

Tensor Less Than Law (element-wise comparison) is defined as:

$$<: \mathbb{R}^{3\times3} \times \mathbb{R}^{3\times3} \to \{0,1\}^{3\times3}$$

Define:

$$\mathbf{Z} = (\mathbf{X} < \mathbf{Y})$$

where $\mathbf{Z} \in \{0,1\}^{3\times3}$ and the law is defined component-wise by:

$$Z_{i,j} = \begin{cases} 1 & \text{if } X_{i,j} < Y_{i,j} \\ 0 & \text{if } X_{i,j} \geq Y_{i,j} \end{cases} \quad \text{for } i,j \in \{1,2,3\}.$$

Part 3 - Hints and Helper

In PyTorch, the tensor less-than comparison (element-wise comparison) can be computed using the built-in function `torch.lt()`.

The reader is encouraged to explore how `torch.lt()` evaluates each tensor element independently and produces a boolean tensor that highlights values below a given reference, which is commonly used for thresholding, masking, and conditional suppression in artificial intelligence systems.

Part 4 - Python code skeleton

```
import torch

def E21(X, Y):
    z = torch...
    return z

def main():
    X = torch...
    Y = torch...
    print(E21(...))

if __name__ == "__main__":
    main()
```

1.2.23 Exercise 22 - Tensor Greater or Equal Law (Comparison)

Part 1 - why this matters

Artificial intelligence systems often rely on inclusive threshold decisions, where meeting a requirement is sufficient even if the value does not exceed it. In many practical scenarios, equality with a boundary condition is just as valid as surpassing it. Ignoring this distinction can lead to unnecessary rejection of valid states or premature suppression of meaningful signals.

The Tensor Greater or Equal Law allows artificial intelligence systems to express inclusive dominance by treating equality and superiority as acceptable outcomes. By converting numerical comparisons into boolean results, the operation ensures that values meeting or exceeding

a reference are treated consistently within decision logic.

Within artificial intelligence pipelines, greater-or-equal comparisons are commonly used to enforce minimum requirements, validate eligibility, and activate conditional execution when values reach acceptable thresholds. This inclusive comparison supports stable decision boundaries and prevents unintended exclusion at critical limits.

Part 2 - Mathematical form

Let $\mathbf{X}, \mathbf{Y} \in \mathbb{R}^{3\times3}$.

Tensor Greater or Equal Law (element-wise comparison) is defined as:

$$\geq: \mathbb{R}^{3\times3} \times \mathbb{R}^{3\times3} \to \{0,1\}^{3\times3}$$

Define:

$$\mathbf{Z} = (\mathbf{X} \geq \mathbf{Y})$$

where $\mathbf{Z} \in \{0,1\}^{3\times3}$ and the law is defined component-wise by:

$$Z_{i,j} = \begin{cases} 1 & \text{if } X_{i,j} \geq Y_{i,j} \\ 0 & \text{if } X_{i,j} < Y_{i,j} \end{cases} \quad \text{for } i,j \in \{1,2,3\}.$$

Part 3 - Hints and Helper

In PyTorch, the tensor greater-or-equal comparison (element-wise comparison) can be computed using the built-in function `torch.ge()`.

The reader is encouraged to explore how `torch.ge()` evaluates tensor elements independently and produces a boolean tensor that identifies values meeting or exceeding a given reference, which is commonly used for thresholding, masking, and inclusive condition enforcement in artificial intelligence systems.

Part 4 - Python code skeleton

```
import torch

def E22(X, Y):
    z = torch...
    return z

def main():
    X = torch...
    Y = torch...
    print(E22(...))

if __name__ == "__main__":
    main()
```

1.2.24 Exercise 23 - Tensor Less or Equal Law (Comparison)

Part 1 - why this matters

Artificial intelligence systems often require inclusive lower-bound decisions, where values are considered acceptable not only when they are strictly smaller than a reference, but also when they exactly meet it. In many practical settings, reaching a boundary condition is sufficient and should not be treated as failure.

The Tensor Less or Equal Law allows artificial intelligence systems to express inclusive insufficiency by treating equality and inferiority as valid outcomes. By converting numerical comparisons into boolean results, the operation ensures that values below or equal to a reference are handled consistently within decision logic.

Within artificial intelligence pipelines, less-or-equal comparisons are commonly used to enforce upper limits, suppress excessive values, and validate constraints that allow boundary equality. This inclusive comparison supports stable control mechanisms and prevents unnecessary rejection at critical thresholds.

Part 2 - Mathematical form

Let $\mathbf{X}, \mathbf{Y} \in \mathbb{R}^{3\times3}$.

Tensor Less or Equal Law (element-wise comparison) is defined as:

$$\leq: \mathbb{R}^{3\times3} \times \mathbb{R}^{3\times3} \to \{0,1\}^{3\times3}$$

Define:

$$\mathbf{Z} = (\mathbf{X} \leq \mathbf{Y})$$

where $\mathbf{Z} \in \{0,1\}^{3\times3}$ and the law is defined component-wise by:

$$Z_{i,j} = \begin{cases} 1 & \text{if } X_{i,j} \leq Y_{i,j} \\ 0 & \text{if } X_{i,j} > Y_{i,j} \end{cases} \quad \text{for } i,j \in \{1,2,3\}.$$

Part 3 - Hints and Helper

In PyTorch, the tensor less-or-equal comparison (element-wise comparison) can be computed using the built-in function `torch.le()`.

The reader is encouraged to explore how `torch.le()` evaluates tensor elements independently and produces a boolean tensor that identifies values below or equal to a given reference, which is commonly used for constraint enforcement, masking, and boundary-aware decision logic in artificial intelligence systems.

Part 4 - Python code skeleton

```
import torch

def E23(X, Y):
    z = torch...
```

```
    return z

def main():
    X = torch...
    Y = torch...
    print(E23(...))

if __name__ == "__main__":
    main()
```

1.3 T01 - Vector Laws (1D)

Mathematics and Theory

1.3.1 Exercise 00 - 1D Dot Product Law

Part 0.1 - similarity

In artificial intelligence, similarity refers to how closely two pieces of information resemble each other in meaning or behavior. When data is represented as vectors, similarity is no longer judged by visual inspection or direct value comparison, but by how strongly two vectors align in the same direction. Vectors that point in similar directions are considered similar, even if their individual components differ in magnitude.

The dot product provides a natural mathematical foundation for measuring similarity between vectors. By combining corresponding elements and aggregating their interaction into a single scalar value, the dot product captures the degree of alignment between two representations. This makes it a fundamental tool for evaluating similarity in embedding spaces, feature representations, and learned vector models used throughout artificial intelligence systems.

Part 0.2 - relevance

Relevance in artificial intelligence describes how important or meaningful one vector is with respect to another. Unlike similarity, which focuses on mutual alignment, relevance is often asymmetric: one vector represents a query or context, while the other represents a candidate or response. The goal is to measure how strongly the candidate responds to the context.

The dot product is widely used to compute relevance scores by projecting one vector onto another. When the dot product is large, the candidate vector aligns strongly with the direction defined by the context, indicating high relevance. This principle underlies many scoring mechanisms in artificial intelligence, where decisions are made based on how strongly inputs activate learned patterns encoded in weight vectors.

Part 0.3 - matching

Matching refers to the process of determining whether two vectors correspond to each other under a specific criterion or task. In artificial intelligence, this often involves comparing an input representation against multiple candidates and selecting the best match. Unlike strict

equality, matching is based on degrees of compatibility rather than exact correspondence.

The dot product enables matching by producing comparable scalar scores across multiple vector pairs. By evaluating how well each candidate aligns with a given input, artificial intelligence systems can rank, select, or filter candidates based on their matching strength. This mechanism is central to tasks such as retrieval, classification, and association, where the goal is to identify the most compatible vector among many alternatives.

Part 1 - why this matters

Artificial intelligence systems represent information as vectors in order to capture structure, relationships, and meaning. However, representing data as vectors alone is not sufficient; models must also evaluate how these vectors interact with one another. The dot product provides a fundamental mechanism for measuring this interaction by converting two vectors into a single scalar value.

The 1D dot product quantifies the degree of alignment between two vectors. When the resulting value is large, the vectors exhibit strong agreement; when it is zero, they are orthogonal and unrelated; and when it is negative, they represent opposing directions. This ability to measure alignment makes the dot product a core operation for similarity evaluation, relevance scoring, and directional decision-making in artificial intelligence systems.

Within artificial intelligence pipelines, the dot product serves as the backbone of linear models, neural networks, and attention mechanisms. It enables models to combine inputs with learned weights, transform high-dimensional interactions into interpretable scalar scores, and make decisions based on the strength of vector relationships. In this sense, the dot product is not merely a mathematical tool, but a foundational operation that translates vector interactions into meaningful computational judgments.

Part 2 - Mathematical form

Let $\mathbf{X}, \mathbf{Y} \in \mathbb{R}^n$ be two one-dimensional vectors of the same length.

1D Dot Product Law is defined as:

$$\cdot : \mathbb{R}^n \times \mathbb{R}^n \to \mathbb{R}$$

Define:

$$z = \mathbf{X} \cdot \mathbf{Y}$$

where $z \in \mathbb{R}$ and the dot product is defined as:

$$z = \sum_{i=1}^{n} X_i \, Y_i$$

The operation computes a scalar value by multiplying corresponding elements of the two vectors and summing the results.

Part 3 - Hints and Helper

In PyTorch, the dot product between two one-dimensional tensors can be computed using the built-in function `torch.dot()`.

The reader is encouraged to explore how `torch.dot()` requires both input vectors to have the same length and produces a single scalar value, which is widely used in similarity measurement, projection, and interaction modeling in artificial intelligence systems.

Part 4 - Python code skeleton

```
import torch

def E00(X, Y):
    z = torch...
    return z

def main():
    X = torch...
    Y = torch...
    print(E00(...))

if __name__ == "__main__":
    main()
```

1.3.2 Exercise 01 - 3D Cross Product Law

Part 1 - why this matters

Artificial intelligence systems that operate in three-dimensional environments must reason not only about magnitude or similarity, but also about orientation and spatial structure. In such settings, understanding how vectors relate to one another in space requires more than measuring alignment; it requires identifying directions that are perpendicular to given surfaces or motions.

The 3D cross product provides a fundamental mechanism for generating a new vector that is orthogonal to two input vectors. This operation enables artificial intelligence systems to determine surface normals, orientation, and rotational direction, all of which are essential for geometric reasoning in three-dimensional space. Unlike the dot product, which reduces vector interaction to a scalar, the cross product creates a new directional entity that encodes spatial information absent from the original inputs.

Within artificial intelligence pipelines, the cross product is widely used in tasks involving 3D perception, robotics, physical simulation, and computer vision. It allows models to reason about facing direction, angular relationships, and spatial consistency, making it a critical operation whenever intelligent systems must interact with or interpret the physical structure of their environment.

Part 2 - Mathematical form

Let $\mathbf{X}, \mathbf{Y} \in \mathbb{R}^3$ be two three-dimensional vectors, where

$$\mathbf{X} = (X_1, X_2, X_3), \quad \mathbf{Y} = (Y_1, Y_2, Y_3).$$

3D Cross Product Law is defined as:

$$\times : \mathbb{R}^3 \times \mathbb{R}^3 \to \mathbb{R}^3$$

Define:

$$\mathbf{Z} = \mathbf{X} \times \mathbf{Y}$$

where $\mathbf{Z} \in \mathbb{R}^3$ is given by:

$$\mathbf{Z} = \begin{pmatrix} X_2Y_3 - X_3Y_2 \\ X_3Y_1 - X_1Y_3 \\ X_1Y_2 - X_2Y_1 \end{pmatrix}.$$

The resulting vector $\mathbf{Z}$ is orthogonal to both $\mathbf{X}$ and $\mathbf{Y}$, and its magnitude represents the area of the parallelogram spanned by the two input vectors.

The magnitude of the resulting vector is given by:

$$\|\mathbf{Z}\| = \|\mathbf{X} \times \mathbf{Y}\| = \|\mathbf{X}\| \, \|\mathbf{Y}\| \sin(\theta),$$

where θ is the angle between $\mathbf{X}$ and $\mathbf{Y}$.

This magnitude represents the area of the parallelogram spanned by the two vectors.

Part 3 - Hints and Helper

In PyTorch, the cross product between two three-dimensional tensors can be computed using the built-in function `torch.cross()`.

The reader is encouraged to explore how `torch.cross()` operates only on three-dimensional vectors, why the order of operands matters, and how the result represents a vector perpendicular to the input vectors. This operation is commonly used in geometric reasoning, orientation detection, and physics-inspired computations within artificial intelligence systems.

Part 4 - Python code skeleton

```
import torch

def E01(X, Y):
    z = torch...
    return z

def main():
    X = torch...
    Y = torch...
    print(E01(...))

if __name__ == "__main__":
    main()
```

1.3.3 Exercise 02 - 1D Norm Law

Part 0 - Magnitude

In vector-based representations, magnitude describes the strength or intensity of a vector independent of its direction. While a vector's direction encodes meaning or orientation, its magnitude reflects how strongly that information is expressed. Separating magnitude from direction allows artificial intelligence systems to analyze the scale of signals without conflating it with semantic alignment.

Magnitude plays a central role in controlling numerical behavior within models. Large magnitudes may dominate computations, while very small magnitudes may become negligible. By explicitly reasoning about magnitude, artificial intelligence systems can regulate signal strength, stabilize computations, and prepare vectors for fair comparison and transformation.

Part 1 - why this matters

Artificial intelligence systems frequently operate on vectors whose direction encodes meaning and whose magnitude encodes strength. Without explicitly measuring magnitude, models cannot distinguish between weak and dominant signals, even when they point in similar directions. This can lead to biased comparisons, unstable learning, or misleading similarity measurements.

The 1D norm provides a principled way to extract the magnitude of a vector as a single non-negative scalar. This enables artificial intelligence systems to monitor signal strength, normalize representations, and control numerical scale before applying further operations such as dot products or similarity measures. By isolating magnitude from direction, the norm supports stable, interpretable, and well-conditioned vector computations within artificial intelligence pipelines.

Part 2 - Mathematical form

Let $\mathbf{X} \in \mathbb{R}^n$ be a one-dimensional vector.

1D Norm Law is defined as:

$$\|\cdot\| : \mathbb{R}^n \to \mathbb{R}$$

Define:

$$z = \|\mathbf{X}\|$$

where $z \in \mathbb{R}$ and the norm is defined as:

$$z = \sqrt{\sum_{i=1}^{n} X_i^2}$$

The norm maps a vector to a non-negative scalar representing its magnitude independent of direction.

Part 3 - Hints and Helper

In PyTorch, the norm of a one-dimensional tensor can be computed using the built-in function `torch.norm()`.

The reader is encouraged to explore how `torch.norm()` collapses a vector into a single scalar value that represents its length, and how this operation is commonly used for normalization, scaling, and magnitude-based comparisons in artificial intelligence systems.

Part 4 - Python code skeleton

```
import torch

def E02(X):
    z = torch...
    return z

def main():
    X = torch...
    print(E02(...))

if __name__ == "__main__":
    main()
```

1.3.4 Exercise 03 - 1D Normalize Law

Part 1 - Why this matters

Artificial intelligence systems frequently compare and combine vectors to evaluate similarity, relevance, or alignment. However, raw vectors often differ significantly in magnitude due to scale, frequency, or accumulated effects. When magnitude is left uncontrolled, larger vectors can dominate computations and distort meaningful comparisons, even when their directional information is similar.

Normalization addresses this issue by enforcing a common scale across vectors. By rescaling each vector to unit length, artificial intelligence systems ensure that comparisons focus on direction rather than numerical dominance. This allows models to reason about relative orientation, semantic alignment, and structural relationships without bias introduced by vector magnitude.

Within artificial intelligence pipelines, normalization is a critical step before similarity measurements, attention scoring, and distance-based reasoning. It stabilizes computations, improves interpretability, and enables fair and consistent vector interactions. In this sense, normalization is not merely a numerical adjustment, but a foundational operation that shapes how intelligent systems interpret and compare vector representations.

Part 2 - Mathematical form

Let $\mathbf{X} \in \mathbb{R}^n$ be a one-dimensional vector with $\|\mathbf{X}\| \neq 0$.

1D Normalize Law is defined as:

$$\text{normalize}(\cdot) : \mathbb{R}^n \to \mathbb{R}^n$$

Define:

$$\mathbf{Z} = \frac{\mathbf{X}}{\|\mathbf{X}\|}$$

where $\mathbf{Z} \in \mathbb{R}^n$ and satisfies:

$$\|\mathbf{Z}\| = 1.$$

This operation rescales the vector so that its magnitude becomes one, while preserving its original direction.

Part 3 - Hints and Helper

In PyTorch, vector normalization can be implemented by dividing a tensor by its norm, which is commonly computed using `torch.norm()`.

The reader is encouraged to explore how normalization rescales vectors to unit length while preserving direction, and why checking that the norm is non-zero is essential to avoid undefined operations. Normalization is widely used to ensure stable similarity measurements, fair comparisons, and numerically balanced computations in artificial intelligence systems.

Part 4 - Python code skeleton

```
import torch

def E03(X):
    norm = torch...
    z...
    return z

def main():
    X = torch...
    print(E03(...))

if __name__ == "__main__":
    main()
```

1.3.5 Exercise 04 - 1D Projection Law

Part 1 - Why this matters

In artificial intelligence systems, information is rarely used in its raw form. High-dimensional vectors often contain a mixture of relevant structure and incidental variation, and treating all components equally can obscure meaningful relationships. Projection provides a principled way to isolate the portion of a vector that aligns with a specific direction, allowing models to focus on the information that truly contributes to a given representation or decision.

Unlike similarity measures that only quantify how related two vectors are, projection extracts the exact component of one vector that lies along another. This distinction is critical in mechanisms such as attention, feature weighting, and embedding analysis, where the goal is not merely to measure alignment, but to use it. By transforming relational scores into directional components, projection turns abstract similarity into actionable geometric structure.

From an interpretability perspective, projection offers a clear geometric explanation of how information flows through a model. It reveals how much of a signal contributes along a chosen axis, separating meaningful influence from orthogonal noise. In this sense, projection is not just a mathematical tool, but a lens through which artificial intelligence systems can reason about

structure, relevance, and decision-making in high-dimensional spaces.

Part 2 - Mathematical form

Let $\mathbf{X}, \mathbf{Y} \in \mathbb{R}^n$ be two one-dimensional vectors, with $\mathbf{Y} \neq \mathbf{0}$.

1D Projection Law describes how to project vector $\mathbf{X}$ onto vector $\mathbf{Y}$, extracting the component of $\mathbf{X}$ that lies in the direction of $\mathbf{Y}$.

The projection operator is defined as:

$$\text{proj}_{\mathbf{Y}}(\mathbf{X}) : \mathbb{R}^n \times \mathbb{R}^n \to \mathbb{R}^n$$

Define:

$$\mathbf{Z} = \frac{\mathbf{X} \cdot \mathbf{Y}}{\|\mathbf{Y}\|^2} \mathbf{Y}$$

where:

$$\mathbf{Z} \in \mathbb{R}^n$$

This operation computes the scalar contribution of $\mathbf{X}$ along $\mathbf{Y}$ using the dot product, then rescales $\mathbf{Y}$ accordingly. The resulting vector represents the directional component of $\mathbf{X}$ aligned with $\mathbf{Y}$.

Part 3 - Hints and Helper

PyTorch does not provide a single dedicated function for vector projection. Instead, projection is constructed explicitly using a combination of `torch.dot()` and `torch.norm()`, which together mirror the mathematical definition directly.

The reader is encouraged to express the projection formula explicitly using PyTorch tensor operations rather than searching for a built-in projection operator. This approach preserves clarity, avoids hidden abstractions, and reinforces understanding of how dot products and norms interact to form the projection. Care must be taken to ensure that the reference vector has a non-zero norm to prevent undefined divisions.

Part 4 - Python code skeleton

```
import torch

def E04(X, Y):
    scalar = torch...
    z = (...)
    return z

def main():
    X = torch...
    Y = torch...
    print(E04(...))

if __name__ == "__main__":
    main()
```

1.3.6 Exercise 05 - 1D Angle Law

Part 1 - Why this matters

In artificial intelligence systems, understanding how vectors relate to each other goes beyond measuring magnitude or extracting directional components. The angle between vectors provides a direct geometric interpretation of their relationship, revealing whether two representations point in similar, opposing, or orthogonal directions.

Angle-based reasoning is fundamental in similarity analysis, embedding spaces, and representation learning. When vectors are normalized, the angle between them becomes a pure measure of directional alignment, independent of scale. This allows artificial intelligence systems to compare patterns, concepts, or features based solely on orientation rather than numerical dominance.

From a modeling perspective, angle measurements enable robust comparisons in high-dimensional spaces where raw distances may be misleading. By reasoning in terms of angles, models gain a stable and interpretable way to assess alignment, separation, and opposition between learned representations, reinforcing both geometric intuition and analytical clarity.

Part 2 - Mathematical form

Let $\mathbf{X}, \mathbf{Y} \in \mathbb{R}^n$ be two one-dimensional vectors, with $\mathbf{X} \neq \mathbf{0}$ and $\mathbf{Y} \neq \mathbf{0}$.

1D Angle Law defines the angle between two vectors using their dot product and magnitudes.

The angle operator is defined as:

$$\text{angle}(\cdot, \cdot) : \mathbb{R}^n \times \mathbb{R}^n \to [0, \pi]$$

Define:

$$\theta = \arccos\left(\frac{\mathbf{X} \cdot \mathbf{Y}}{\|\mathbf{X}\| \, \|\mathbf{Y}\|}\right)$$

where:

$$\theta \in [0, \pi]$$

This formulation expresses the geometric angle between two vectors, capturing their directional alignment independently of their magnitudes.

Part 3 - Hints and Helper

In PyTorch, the angle between two vectors is constructed explicitly using `torch.dot()`, `torch.norm()`, and an inverse cosine operation applied to the resulting scalar. These tensor operations directly reflect the mathematical definition of the angle between vectors.

The reader is encouraged to rely on PyTorch's dot product and norm functions to compute the cosine of the angle, then apply an inverse trigonometric operation to recover the angle itself. Ensuring that both vectors have non-zero norms and that the cosine value remains within a valid numerical range is essential to avoid undefined or unstable results.

Part 4 - Python code skeleton

```
import torch

def E05(X, Y):
```

```
        cos_theta = torch...
        theta = (...)
        return theta

    def main():
        X = torch...
        Y = torch...
        print(E05(...))

    if __name__ == "__main__":
        main()
```

1.3.7 Exercise 06 - 1D Distance Law

Part 1 - Why this matters

In artificial intelligence systems, comparing representations often requires more than understanding direction or alignment. Distance provides a direct measure of separation between vectors, capturing how far apart two representations are within a given space. This notion is fundamental when models must reason about similarity, clustering, or deviation from a reference state.

Distance-based reasoning plays a central role in optimization, anomaly detection, and embedding analysis. During training, loss functions frequently rely on distance to quantify error, guiding models toward desired targets. In representation spaces, distance determines neighborhood structure, influencing how models group, separate, or prioritize information.

From an interpretability perspective, distance offers an intuitive geometric signal: small distances indicate closeness or similarity, while large distances signal divergence. By grounding comparisons in geometric separation, artificial intelligence systems gain a stable and scalable way to evaluate relationships between vectors across high-dimensional spaces.

Part 2 - Mathematical form

Let $\mathbf{X}, \mathbf{Y} \in \mathbb{R}^n$ be two one-dimensional vectors.
1D Distance Law defines the distance between two vectors as the norm of their difference. The distance operator is defined as:

$$\text{dist}(\cdot, \cdot) : \mathbb{R}^n \times \mathbb{R}^n \to \mathbb{R}$$

Define:

$$d = \|\mathbf{X} - \mathbf{Y}\|$$

where:

$$d \in \mathbb{R}, \quad d \geq 0$$

This formulation measures the geometric separation between two vectors, independent of their orientation, and reduces to the standard Euclidean distance in vector space.

Part 3 - Hints and Helper

In PyTorch, distance computation is expressed using tensor subtraction followed by a norm operation. There is no requirement for a dedicated distance function when the mathematical definition can be written directly using basic tensor operations.

The reader is encouraged to compute the vector difference explicitly and apply `torch.norm()` to obtain the distance. This approach maintains clarity, avoids unnecessary abstraction, and reinforces the geometric interpretation of distance as magnitude of separation.

Part 4 - Python code skeleton

```
import torch

def E06(X, Y):
    diff = X ...
    d = torch...
    return d

def main():
    X = torch...
    Y = torch...
    print(E06(...))

if __name__ == "__main__":
    main()
```

1.3.8 Exercise 07 - 1D Sum Reduction Law

Part 0 - Loss Functions

Loss functions quantify how far a model's predictions deviate from their target values during training. At early stages of learning, this deviation is often large, reflecting poor alignment between prediction and ground truth. As the model updates its parameters through optimization, the accumulated deviation is gradually reduced, driving the loss toward smaller values. For example, an initial error magnitude of 5.0 may decrease progressively through training iterations until it reaches a value such as 0.5, indicating improved model performance. This reduction process can be viewed as a sequence of deviation points:

$$5.0 \rightarrow 4.2 \rightarrow 3.6 \rightarrow 2.8 \rightarrow 2.1 \rightarrow 1.5 \rightarrow 1.0 \rightarrow 0.5$$

Each step represents a reduction in overall error achieved by aggregating individual deviations through a loss function and applying corrective updates. In this sense, loss reduction is not instantaneous, but a gradual convergence from high deviation toward minimal error.

Part 1 - Why this matters

In artificial intelligence systems, information is often represented in a distributed form across multiple dimensions. While this distributed structure is essential for capturing detailed patterns and local relationships, models ultimately require a mechanism to collapse these representations into a single signal that can guide decisions, comparisons, or optimization objectives.

Sum reduction provides this mechanism by aggregating all components of a vector into one scalar value. This operation transforms dispersed information into a global measure, enabling artificial intelligence systems to reason about total activation, accumulated error, or overall contribution without preserving positional detail. In doing so, sum reduction acts as a bridge between local computations and global model behavior.

From a learning perspective, sum reduction underpins many loss formulations and optimization processes. It allows models to evaluate collective outcomes rather than isolated components, ensuring that training signals reflect the full scope of model responses. In this sense, sum reduction is not merely a numerical convenience, but a structural operation that enables artificial intelligence systems to transition from representation to decision.

Part 2 - Mathematical form

Let $\mathbf{X} \in \mathbb{R}^n$ be a one-dimensional vector.

1D Sum Reduction Law defines the operation of reducing a vector to a single scalar by summing all of its components.

The sum reduction operator is defined as:

$$\text{sum}(\cdot) : \mathbb{R}^n \to \mathbb{R}$$

Define:

$$s = \sum_{i=1}^{n} X_i$$

where:

$$s \in \mathbb{R}$$

This operation collapses the vector dimension by aggregating all elements into a single scalar, preserving total magnitude while discarding positional structure.

Part 3 - Hints and Helper

In PyTorch, sum reduction is expressed using a dedicated reduction operation applied to a tensor. This operation directly reflects the mathematical summation of vector components and avoids manual iteration.

The reader is encouraged to use PyTorch's built-in summation functionality to reduce a one-dimensional tensor to a scalar, ensuring clarity and consistency with tensor-based computation. Care should be taken to understand how reduction operations alter tensor shape and dimensionality.

Part 4 - Python code skeleton

```
import torch

def E07(X):
    s = torch...
    return s

def main():
    X = torch...
```

```
    print(E07(...))

if __name__ == "__main__":
    main()
```

1.3.9 Exercise 08 - 1D Max Reduction Law

Part 1 - Why this matters

In artificial intelligence systems, not all information contributes equally to a decision. In many scenarios, the strongest signal within a set of values carries the most meaningful insight, while smaller contributions may represent noise or secondary effects. Max reduction enables models to focus on this dominant signal by selecting the largest component of a vector and discarding the rest.

This behavior is especially important in pattern detection and feature analysis, where the presence of a strong activation can be more informative than the overall distribution of values. By preserving only the maximum response, max reduction allows artificial intelligence systems to highlight salient features, trigger decisions, or identify critical evidence without dilution from weaker signals.

From a structural perspective, max reduction performs a decisive dimensional collapse. It transforms a distributed representation into a single value that reflects peak influence rather than aggregate behavior. In this sense, max reduction is not merely a numerical shortcut, but a selective operation that shapes how models prioritize information and respond to the most influential components within high-dimensional representations.

Part 2 - Mathematical form

Let $\mathbf{X} \in \mathbb{R}^n$ be a one-dimensional vector.

1D Max Reduction Law defines the operation of reducing a vector to a single scalar by selecting its maximum component.

The max reduction operator is defined as:

$$\max(\cdot) : \mathbb{R}^n \to \mathbb{R}$$

Define:

$$m = \max_{1 \le i \le n} X_i$$

where:

$$m \in \mathbb{R}$$

This operation collapses the vector dimension by preserving only the largest value, discarding all other components and their relative structure.

Part 3 - Hints and Helper

In PyTorch, max reduction is performed using `torch.max()`, which applies a reduction operation over the elements of a tensor. This function directly reflects the mathematical definition of selecting the maximum component of a vector.

The reader is encouraged to use `torch.max()` to reduce a one-dimensional tensor to its largest value, while being mindful that the function may also return auxiliary information such as the index of the maximum element. When only the reduced scalar is required, attention should be focused on extracting the value component of the operation.

Part 4 - Python code skeleton

```
import torch

def E08(X):
    m = torch...
    return m

def main():
    X = torch...
    print(E08(...))

if __name__ == "__main__":
    main()
```

1.3.10 Exercise 09 - 1D Min Reduction Law

Part 1 - Why this matters

In artificial intelligence systems, not all information signals are valuable in the same way. While some operations focus on dominant or peak values, others require identifying the weakest or most constrained component within a set of values. Min reduction enables models to isolate this limiting factor by selecting the smallest element of a vector and discarding all others.

This operation is particularly important in optimization, constraint handling, and safety-sensitive reasoning. By focusing on the minimum value, artificial intelligence systems can detect bottlenecks, identify failure conditions, or evaluate worst-case scenarios where the weakest signal determines overall behavior. In such contexts, the smallest component carries disproportionate importance.

From a structural perspective, min reduction performs a decisive dimensional collapse centered on limitation rather than dominance. It transforms a distributed representation into a single scalar that reflects the most restrictive element of the system. In this sense, min reduction is not merely a numerical counterpart to max reduction, but a complementary operation that enables artificial intelligence systems to reason about constraints, risk, and lower bounds within high-dimensional representations.

Part 2 - Mathematical form

Let $\mathbf{X} \in \mathbb{R}^n$ be a one-dimensional vector.

1D Min Reduction Law defines the operation of reducing a vector to a single scalar by selecting its minimum component.

The min reduction operator is defined as:

$$\min(\cdot) : \mathbb{R}^n \to \mathbb{R}$$

Define:

$$m = \min_{1 \le i \le n} X_i$$

where:

$$m \in \mathbb{R}$$

This operation collapses the vector dimension by preserving only the smallest value, discarding all other components and their relative structure.

Part 3 - Hints and Helper

In PyTorch, min reduction is performed using `torch.min()`, which applies a reduction operation over the elements of a tensor. This function directly reflects the mathematical definition of selecting the minimum component of a vector.

The reader is encouraged to use `torch.min()` to reduce a one-dimensional tensor to its smallest value, while being aware that the function may also return auxiliary information such as the index of the minimum element. When only the reduced scalar is required, attention should be focused on extracting the value component of the operation.

Part 4 - Python code skeleton

```
import torch

def E09(X):
    m = torch...
    return m

def main():
    X = torch...
    print(E09(...))

if __name__ == "__main__":
    main()
```

1.3.11 Exercise 10 - 1D Mean Reduction Law

Part 0.1 - Batch

In artificial intelligence training, data is often processed in batches rather than as individual samples. A batch represents a group of examples evaluated together in a single forward and backward pass, enabling efficient computation and stable learning. By operating on batches, models can estimate gradients more reliably while balancing memory constraints and training speed, making batch processing a fundamental concept in modern optimization workflows.

Part 0.2 - Workflows

In artificial intelligence systems, workflows describe the structured sequence of operations through which data moves from input to output during training or inference. A workflow typically includes data loading, batch processing, model evaluation, loss computation, and parameter updates, forming a coherent pipeline that governs how learning unfolds. By organizing computation into clear workflows, artificial intelligence systems ensure consistency,

reproducibility, and efficient coordination between individual operations within complex models.

Part 1 - Why this matters

In artificial intelligence systems, performance must be evaluated in a way that is independent of scale and data quantity. When representations vary in length or when training is performed over batches of different sizes, relying on total accumulated values can distort interpretation. Mean reduction addresses this challenge by providing a normalized measure that reflects average behavior rather than total magnitude.

By averaging components, mean reduction allows models to reason about quality instead of quantity. This is especially important in learning processes, where loss values, activations, and errors must remain comparable across iterations and datasets. Unlike sum reduction, which grows with vector size, mean reduction maintains a stable range, enabling consistent optimization and controlled parameter updates.

From a structural perspective, mean reduction performs a balanced dimensional collapse. It transforms a distributed representation into a single scalar while preserving proportional contribution from all components. In this sense, mean reduction is not merely a convenience, but a foundational operation that ensures fair evaluation, stable learning dynamics, and reliable comparison within artificial intelligence systems.

Part 2 - Mathematical form

Let $\mathbf{X} \in \mathbb{R}^n$ be a one-dimensional vector, with $n \geq 1$.

1D Mean Reduction Law defines the operation of reducing a vector to a single scalar by computing the average of its components.

The mean reduction operator is defined as:

$$\text{mean}(\cdot) : \mathbb{R}^n \to \mathbb{R}$$

Define:

$$\mu = \frac{1}{n} \sum_{i=1}^{n} X_i$$

where:

$$\mu \in \mathbb{R}$$

This operation collapses the vector dimension by aggregating all elements into a single scalar while normalizing by the number of components, preserving average magnitude rather than total or peak values.

Part 3 - Hints and Helper

In PyTorch, mean reduction is performed using `torch.mean()`, which computes the average value of all elements in a tensor. This function directly implements the mathematical definition of mean reduction.

The reader is encouraged to use `torch.mean()` to reduce a one-dimensional tensor to a scalar, while understanding that mean reduction differs conceptually from sum reduction by normalizing the result with respect to the vector length. Care should be taken to ensure the tensor is non-empty to avoid undefined behavior.

Part 4 - Python code skeleton

```
import torch

def E10(X):
    mu = torch...
    return mu

def main():
    X = torch...
    print(E10(...))

if __name__ == "__main__":
    main()
```

1.3.12 Exercise 11 - 1D Variance Law

Part 1 - Why this matters

In artificial intelligence systems, evaluating performance or behavior based solely on average values can be misleading. Two sets of representations may share the same mean while exhibiting fundamentally different levels of stability and reliability. Variance addresses this limitation by measuring how widely values are distributed around their average, revealing information that mean values alone cannot capture.

By quantifying dispersion, variance allows artificial intelligence systems to distinguish between stable signals and fluctuating ones. Low variance indicates consistency and predictable behavior, while high variance signals noise, instability, or sensitivity to input changes. This distinction is critical in training dynamics, data quality assessment, and model robustness analysis.

From a structural perspective, variance performs a dimensional collapse focused on deviation rather than magnitude. It transforms a distributed representation into a single scalar that reflects variability around a central tendency. In this sense, variance is not merely a statistical quantity, but a foundational measure that enables artificial intelligence systems to reason about stability, noise, and reliability within high-dimensional representations.

Part 2 - Mathematical form

Let $\mathbf{X} \in \mathbb{R}^n$ be a one-dimensional vector, with $n \geq 1$.

1D Variance Law defines the operation of measuring how widely the components of a vector are spread around their mean.

The variance operator is defined as:

$$\mathrm{var}(\cdot) : \mathbb{R}^n \to \mathbb{R}$$

Define:

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^{n} (X_i - \mu)^2$$

where:

$$\mu = \frac{1}{n} \sum_{i=1}^{n} X_i, \quad \sigma^2 \in \mathbb{R}, \quad \sigma^2 \geq 0$$

This formulation quantifies dispersion by averaging the squared deviations from the mean, collapsing the vector into a single scalar that reflects variability rather than magnitude.

Part 3 - Hints and Helper

In PyTorch, variance is computed using `torch.var()`, which internally combines mean subtraction and squared deviation reduction into a single operation. This function directly corresponds to the mathematical definition of variance.

The reader is encouraged to use `torch.var()` to compute variance over a one-dimensional tensor, while being mindful of normalization conventions. In particular, attention should be paid to whether the operation uses population normalization or an unbiased estimate, as this choice affects the scaling of the result.

Part 4 - Python code skeleton

```
import torch

def E11(X):
    var = torch...
    return var

def main():
    X = torch...
    print(E11(...))

if __name__ == "__main__":
    main()
```

1.3.13 Exercise 12 - 1D Standard Deviation Law

Part 1 - Why this matters

In artificial intelligence systems, understanding average behavior is not sufficient to characterize how a model or dataset behaves in practice. Standard deviation provides a direct and interpretable measure of how far individual values typically deviate from the mean, offering insight into the spread and stability of representations.

Unlike variance, which expresses dispersion in squared units, standard deviation operates in the same scale as the original data. This makes it particularly valuable for reasoning about noise, consistency, and sensitivity within models. A small standard deviation indicates tightly clustered values and stable behavior, while a large standard deviation signals fluctuation and uncertainty.

From a structural perspective, standard deviation performs a dimensional collapse that preserves interpretability. It transforms a distributed vector into a single scalar that reflects typical deviation rather than extreme or aggregate behavior. In this sense, standard deviation serves as a practical bridge between statistical measurement and intuitive understanding in artificial intelligence systems.

Part 2 - Mathematical form

Let $\mathbf{X} \in \mathbb{R}^n$ be a one-dimensional vector, with $n \geq 1$.

1D Standard Deviation Law defines the operation of measuring the typical magnitude of deviation of vector components from their mean.

The standard deviation operator is defined as:

$$\text{std}(\cdot) : \mathbb{R}^n \to \mathbb{R}$$

Define:

$$\sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(X_i - \mu)^2}$$

where:

$$\mu = \frac{1}{n}\sum_{i=1}^{n} X_i, \quad \sigma \in \mathbb{R}, \quad \sigma \geq 0$$

This formulation expresses dispersion in the same units as the original data, making standard deviation directly interpretable as an average distance from the mean.

Part 3 - Hints and Helper

In PyTorch, standard deviation is computed using `torch.std()`, which internally relies on variance computation followed by a square root operation. This function corresponds directly to the mathematical definition of standard deviation.

The reader is encouraged to use `torch.std()` to compute dispersion over a one-dimensional tensor, while paying attention to normalization conventions. In particular, understanding whether population or unbiased normalization is used is important, as it affects the scale of the resulting value.

Part 4 - Python code skeleton

```
import torch

def E12(X):
    std = torch...
    return std

def main():
    X = torch...
    print(E12(...))

if __name__ == "__main__":
    main()
```

1.3.14 Exercise 13 - 1D Cumulative Sum Law

Part 1 - Why this matters

In artificial intelligence systems, many forms of information cannot be fully understood when treated as isolated values. Data often arrives as a sequence, where each element gains meaning from what came before it. Cumulative sum addresses this need by transforming momentary values into a representation that encodes historical accumulation.

By allowing each element of the output vector to reflect all preceding inputs, cumulative sum introduces a simple yet powerful notion of memory. This makes it possible for artificial intelligence systems to reason about trends, growth, and long-term influence without relying on learned parameters or complex recurrent structures. In this sense, cumulative sum provides a foundational mechanism for capturing temporal context.

From a structural perspective, cumulative sum preserves dimensionality while changing interpretation. It converts a vector of independent values into a progressive representation where each position carries contextual information. This transformation enables artificial intelligence systems to analyze sequences, monitor convergence, and model accumulation processes in a transparent and deterministic manner.

Part 2 - Mathematical form

Let $\mathbf{X} \in \mathbb{R}^n$ be a one-dimensional vector.

1D Cumulative Sum Law defines the operation of transforming a vector into a new vector where each element represents the running total up to that position.

The cumulative sum operator is defined as:

$$\text{cumsum}(\cdot) : \mathbb{R}^n \to \mathbb{R}^n$$

Define:

$$Z_k = \sum_{i=1}^{k} X_i, \quad k = 1, 2, \ldots, n$$

where:

$$\mathbf{Z} = [Z_1, Z_2, \ldots, Z_n] \in \mathbb{R}^n$$

This operation preserves dimensionality while encoding progressive accumulation, allowing each position to reflect all preceding contributions.

Part 3 - Hints and Helper

In PyTorch, cumulative summation is performed using `torch.cumsum()`, which computes running totals along a specified dimension of a tensor. This function directly corresponds to the mathematical definition of cumulative summation.

The reader is encouraged to use `torch.cumsum()` to accumulate values along the one-dimensional axis, while understanding that unlike reduction operations, cumulative sum does not collapse dimensionality. Each output element depends on all previous elements in the sequence.

Part 4 - Python code skeleton

```
import torch

def E13(X):
    Z = torch...
    return Z

def main():
    X = torch...
```

```
    print(E13(...))

if __name__ == "__main__":
    main()
```

1.3.15 Exercise 14 - 1D Cumulative Product Law

Part 1 - Why this matters

In artificial intelligence systems, not all processes evolve through additive accumulation. Some dynamics grow, decay, or amplify multiplicatively, where the effect of each element compounds over time. Cumulative product captures this behavior by allowing each value to influence all subsequent values through multiplication rather than addition.

By encoding multiplicative history, cumulative product enables artificial intelligence systems to model growth rates, decay processes, scaling effects, and probabilistic chains. This is particularly important in scenarios where relative change matters more than absolute difference, such as likelihood propagation, signal attenuation, and sequential scaling.

From a structural perspective, cumulative product preserves dimensionality while introducing compounding dependency. Each output element reflects the entire multiplicative past, transforming a vector of independent factors into a progressive representation that encodes long-term influence and amplification effects.

Part 2 - Mathematical form

Let $\mathbf{X} \in \mathbb{R}^n$ be a one-dimensional vector.

1D Cumulative Product Law defines the operation of transforming a vector into a new vector where each element represents the running product up to that position.

The cumulative product operator is defined as:

$$\text{cumprod}(\cdot) : \mathbb{R}^n \to \mathbb{R}^n$$

Define:

$$Z_k = \prod_{i=1}^{k} X_i, \quad k = 1, 2, \ldots, n$$

where:

$$\mathbf{Z} = [Z_1, Z_2, \ldots, Z_n] \in \mathbb{R}^n$$

This operation preserves vector length while encoding multiplicative accumulation, allowing each position to reflect compounded influence from all preceding elements.

Part 3 - Hints and Helper

In PyTorch, cumulative product is performed using `torch.cumprod()`, which computes running products along a specified tensor dimension. This function directly corresponds to the mathematical definition of cumulative product.

The reader is encouraged to use `torch.cumprod()` along the one-dimensional axis, while being mindful that unlike reduction operations, this law does not collapse dimensionality. Each output element depends on all prior elements, and special care should be taken when vectors contain zeros or negative values, as these can significantly alter subsequent results.

Part 4 - Python code skeleton

```
import torch

def E14(X):
    Z = torch...
    return Z

def main():
    X = torch...
    print(E14(...))

if __name__ == "__main__":
    main()
```

1.4 T02 - Matrix Laws (2D)

Mathematics and Theory

1.4.1 Exercise 00 - 2D Matrix Multiplication Law

Part 1 - Why this matters

In artificial intelligence systems, raw data rarely holds value in its original form. Learning emerges from the ability to transform representations into new spaces where meaningful patterns become accessible. Two-dimensional matrix multiplication provides the fundamental mechanism for this transformation by combining multiple features through structured, weighted interactions.

By applying matrix multiplication, artificial intelligence models can process entire batches of data simultaneously, mapping input features to learned representations in a consistent and scalable manner. Each output element encodes a weighted aggregation of inputs, allowing models to capture relationships, dependencies, and correlations that cannot be expressed through one-dimensional operations alone.

From a structural perspective, matrix multiplication establishes the bridge between data and learning. It formalizes how parameters influence representations and defines how information flows through layers of a model. In this sense, two-dimensional matrix multiplication is not merely a computational operation, but the core transformation law that enables artificial intelligence systems to learn, generalize, and reason across high-dimensional spaces.

Part 2 - Mathematical form

Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{n \times p}$ be two two-dimensional matrices.

2D Matrix Multiplication Law defines the operation of combining two matrices by computing dot products between rows of the first matrix and columns of the second matrix.

The matrix multiplication operator is defined as:

$$(\cdot) : \mathbb{R}^{m \times n} \times \mathbb{R}^{n \times p} \rightarrow \mathbb{R}^{m \times p}$$

Define:

$$\mathbf{C} = \mathbf{AB}$$

with elements:

$$C_{ij} = \sum_{k=1}^{n} A_{ik}\, B_{kj}$$

where:

$$\mathbf{C} \in \mathbb{R}^{m \times p}$$

This operation combines rows of $\mathbf{A}$ with columns of $\mathbf{B}$, producing a new matrix whose entries encode weighted interactions between both inputs.

Part 3 - Hints and Helper

In PyTorch, matrix multiplication is performed using `torch.matmul()`, which automatically applies the correct multiplication rule based on tensor dimensionality. For two-dimensional tensors, this function implements standard matrix multiplication as defined mathematically.

The reader is encouraged to use `torch.matmul()` when multiplying matrices, rather than element-wise operations. Special attention should be paid to matrix shapes, as the inner dimensions must match for the multiplication to be valid.

Part 4 - Python code skeleton

```
import torch

def E00(A, B):
    C = torch...
    return C

def main():
    A = torch...
    B = torch...
    print(E00(...))

if __name__ == "__main__":
    main()
```

1.4.2 Exercise 01 - 2D Hadamard Matrix Law

Part 1 - Why this matters

In artificial intelligence systems, not all operations aim to transform data into new representations. In many scenarios, the objective is to regulate, filter, or modulate existing information without altering its structural form. Hadamard matrix multiplication provides this capability by enabling direct, element-wise control over matrix values.

By applying Hadamard multiplication, artificial intelligence models can selectively amplify, suppress, or mask individual elements within a matrix. This operation is essential in mechanisms such as masking, gating, and attention control, where decisions are made locally at the level of each element rather than through global aggregation or transformation.

From a structural perspective, Hadamard multiplication preserves dimensionality while modifying semantic influence. It allows models to intervene in information flow without reshaping or reprojecting data, making it a critical tool for fine-grained control within neural architectures. In this sense, the Hadamard matrix law complements matrix multiplication by enabling precision and control where transformation alone is insufficient.

Part 2 - Mathematical form

Let $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{m \times n}$ be two matrices of the same shape.

2D Hadamard Matrix Law defines element-wise multiplication between two matrices, where each element in the output matrix is obtained by multiplying corresponding elements from the input matrices.

The Hadamard multiplication operator is defined as:

$$\odot : \mathbb{R}^{m \times n} \times \mathbb{R}^{m \times n} \rightarrow \mathbb{R}^{m \times n}$$

Define:

$$\mathbf{C} = \mathbf{A} \odot \mathbf{B}$$

with elements:

$$C_{ij} = A_{ij} \cdot B_{ij}$$

where:

$$\mathbf{C} \in \mathbb{R}^{m \times n}$$

This operation requires both matrices to have identical dimensions and applies multiplication independently to each corresponding pair of elements.

Part 3 - Hints and Helper

In PyTorch, Hadamard (element-wise) matrix multiplication is performed using the `*` operator or the function `torch.mul()`. Both approaches apply element-wise multiplication and require the input matrices to share the same shape.

The reader is encouraged to distinguish clearly between Hadamard multiplication and standard matrix multiplication. Unlike `torch.matmul()`, Hadamard multiplication does not combine rows and columns and does not perform any form of dimensional reduction or aggregation.

Part 4 - Python code skeleton

```
import torch

def E01(A, B):
    C = torch...
    return C

def main():
    A = torch...
    B = torch...
    print(E01(...))

if __name__ == "__main__":
    main()
```

1.4.3 Exercise 02 - 2D Frobenius Inner Product Law

Part 1 - Why this matters

In artificial intelligence systems, matrices often represent batches of data, learned parameters, feature maps, attention scores, or intermediate activations. Comparing these matrices requires an operation that measures how strongly their entries align while preserving the structure of the original representations. The two-dimensional Frobenius inner product provides this mechanism by combining corresponding entries from two matrices into a single scalar value.

By applying the Frobenius inner product, artificial intelligence models can measure similarity, interaction, and agreement between matrix-valued representations. Each pair of corresponding entries contributes to the final result, allowing the operation to summarize the global relationship between two matrices without discarding the contribution of any individual position.

From a learning perspective, the Frobenius inner product appears naturally in loss functions, matrix similarity measures, optimization objectives, and gradient-based methods. It also provides the matrix-space extension of the standard vector dot product. In this sense, the two-dimensional Frobenius inner product is not merely a reduction operation, but a fundamental law for measuring alignment between structured representations in artificial intelligence systems.

Part 2 - Mathematical form

Let $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{m \times n}$ be two two-dimensional matrices with identical shapes.

2D Frobenius Inner Product Law defines the operation of multiplying corresponding entries from both matrices and summing all resulting products.

The Frobenius inner product operator is defined as:

$$\langle \cdot, \cdot \rangle_F : \mathbb{R}^{m \times n} \times \mathbb{R}^{m \times n} \to \mathbb{R}$$

Define:

$$c = \langle \mathbf{A}, \mathbf{B} \rangle_F$$

Then:

$$c = \sum_{i=1}^{m} \sum_{j=1}^{n} A_{ij} B_{ij}$$

Equivalently:

$$\langle \mathbf{A}, \mathbf{B} \rangle_F = \operatorname{tr}\left(\mathbf{A}^{\mathsf{T}} \mathbf{B}\right)$$

where:

$$c \in \mathbb{R}$$

This operation measures the global alignment between $\mathbf{A}$ and $\mathbf{B}$ by accumulating the products of all corresponding matrix entries into a single scalar.

Part 3 - Hints and Helper

In PyTorch, the Frobenius inner product can be computed by combining element-wise multiplication with a reduction over all matrix entries. The functions `torch.mul()` and `torch.sum()` may be used to express these two stages explicitly.

The reader should verify that both input tensors have the same shape before performing the operation. Special attention should also be paid to the difference between the Frobenius inner product and matrix multiplication: the Frobenius inner product multiplies corresponding entries and returns a scalar, whereas matrix multiplication combines rows and columns and generally returns a matrix.

Part 4 - Python code skeleton

```
import torch

def E02(A, B):
    C = torch...
    c = torch...
    return c

def main():
    A = torch...
    B = torch...
    print(E02(...))

if __name__ == "__main__":
    main()
```

1.4.4 Exercise 03 - 2D Determinant Law

Part 1 - Why this matters

In artificial intelligence systems, matrices frequently describe linear transformations applied to data, features, parameters, and learned representations. Understanding whether such a transformation preserves information or collapses part of the input space is essential when analyzing the behavior of a model. The two-dimensional determinant provides a single scalar value that summarizes this structural property for a square matrix.

The magnitude of the determinant describes how a matrix transformation scales area in two dimensions and, more generally, volume in higher-dimensional spaces. Its sign indicates whether the transformation preserves or reverses orientation. A determinant equal to zero reveals that the transformation compresses the space into a lower dimension, causing information to be lost and making the matrix non-invertible.

From a learning perspective, determinants appear in probabilistic models, normalizing flows, covariance analysis, change-of-variable formulas, and numerical methods involving matrix invertibility. In this sense, the two-dimensional determinant is not merely a matrix calculation, but a fundamental diagnostic law for understanding how learned transformations scale, preserve, or destroy information.

Part 2 - Mathematical form

Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be a two-dimensional square matrix.

2D Determinant Law defines an operation that maps a square matrix to a single scalar representing the signed scaling factor of the corresponding linear transformation.

The determinant operator is defined as:

$$\det : \mathbb{R}^{n \times n} \to \mathbb{R}$$

Define:

$$\delta = \det(\mathbf{A})$$

For a general $n \times n$ matrix, the determinant is defined by the Leibniz formula:

$$\delta = \sum_{\sigma \in S_n} \operatorname{sgn}(\sigma) \prod_{i=1}^{n} A_{i,\sigma(i)}$$

where S_n is the set of all permutations of $\{1, 2, \ldots, n\}$ and $\operatorname{sgn}(\sigma)$ is the sign of the permutation σ.

For the special case:

$$\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \in \mathbb{R}^{2 \times 2}$$

the determinant is:

$$\det(\mathbf{A}) = ad - bc$$

where:

$$\delta \in \mathbb{R}$$

If $\det(\mathbf{A}) \neq 0$, then $\mathbf{A}$ is invertible. If $\det(\mathbf{A}) = 0$, then $\mathbf{A}$ is singular and its transformation collapses at least one dimension of the input space.

Part 3 - Hints and Helper

In PyTorch, the determinant of a square matrix is computed using `torch.linalg.det()`. This function applies a numerically appropriate matrix-factorization method rather than directly expanding the full Leibniz formula.

The reader should verify that the input tensor is two-dimensional and square, meaning that its number of rows equals its number of columns. The tensor should also use a supported floating-point or complex data type. Special attention should be paid to matrices whose determinant is equal or very close to zero, as they may be singular or numerically ill-conditioned.

Part 4 - Python code skeleton

```
import torch

def E03(A):
    determinant = torch...
    return determinant

def main():
    A = torch...
    print(E03(...))

if __name__ == "__main__":
    main()
```

1.4.5 Exercise 04 - 2D Inverse Matrix Law

Part 1 - Why this matters

In artificial intelligence systems, matrices are used to transform data from one representation space into another. When a transformation preserves enough information to be reversed, its inverse matrix provides the mechanism for recovering the original representation. This reversibility is essential for understanding whether a learned transformation can be undone without losing information.

By applying an inverse matrix, artificial intelligence models and numerical algorithms can recover unknown variables, reverse coordinate transformations, and solve systems of linear equations. Inverse transformations also appear in probabilistic modeling, covariance-based methods, optimization, and invertible neural network architectures, where information must pass through a sequence of reversible mappings.

From a structural perspective, the existence of an inverse distinguishes information-preserving transformations from singular transformations that collapse one or more dimensions. In this sense, the two-dimensional inverse matrix law is not merely a method for reversing matrix multiplication, but a fundamental condition for determining whether a matrix transformation preserves the full structure of its input space.

Part 2 - Mathematical form

Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be a two-dimensional square matrix satisfying:

$$\det(\mathbf{A}) \neq 0$$

2D Inverse Matrix Law defines the unique matrix $\mathbf{A}^{-1}$ that reverses the transformation produced by $\mathbf{A}$.

The inverse operator is defined over the set of invertible matrices as:

$$(\cdot)^{-1} : \mathrm{GL}_n(\mathbb{R}) \to \mathrm{GL}_n(\mathbb{R})$$

Define:

$$\mathbf{B} = \mathbf{A}^{-1}$$

Then:

$$\mathbf{AB} = \mathbf{BA} = \mathbf{I}_n$$

where:

$$\mathbf{B} \in \mathbb{R}^{n \times n}$$

and $\mathbf{I}_n$ is the $n \times n$ identity matrix.

The inverse may also be expressed as:

$$\mathbf{A}^{-1} = \frac{1}{\det(\mathbf{A})} \operatorname{adj}(\mathbf{A})$$

For the special case:

$$\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

with $ad - bc \neq 0$, the inverse is:

$$\mathbf{A}^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

If $\det(\mathbf{A}) = 0$, then the inverse does not exist and $\mathbf{A}$ is called a singular matrix.

Part 3 - Hints and Helper

In PyTorch, the inverse of a square matrix is computed using `torch.linalg.inv()`. The function returns a matrix with the same shape as the input and requires the input matrix to be invertible.

The reader should verify that the input tensor is two-dimensional, square, and stored using a supported floating-point or complex data type. Special attention should be paid to matrices whose determinants are zero or very close to zero, because singular or ill-conditioned matrices may cause an error or produce numerically unstable results. When solving a system such as $\mathbf{Ax} = \mathbf{b}$, directly solving the system is generally more stable than explicitly forming $\mathbf{A}^{-1}$.

Part 4 - Python code skeleton

```
import torch

def E04(A):
    A_inverse = torch...
    return A_inverse

def main():
    A = torch...
    print(E04(...))

if __name__ == "__main__":
    main()
```

1.4.6 Exercise 05 - 2D Transpose Law

Part 1 - Why this matters

In artificial intelligence systems, matrices organize information along rows and columns, with each axis often representing a different semantic role. Rows may represent observations while columns represent features, or one axis may represent input units while the other represents output units. The two-dimensional transpose provides the fundamental operation for exchanging these roles without changing the values stored in the matrix.

By applying the transpose, artificial intelligence models can align matrix shapes for multiplication, compare features across observations, and reverse the direction in which relationships are represented. Transposition appears in linear layers, attention mechanisms, covariance calculations, gradient expressions, and inner-product operations where rows must interact with columns in a mathematically valid form.

From a structural perspective, transposition reorganizes a matrix by reflecting its entries across the main diagonal. It changes the orientation of a representation while preserving all of its information. In this sense, the two-dimensional transpose is not merely a shape manipulation, but a fundamental law for aligning, interpreting, and combining matrix-valued representations in artificial intelligence systems.

Part 2 - Mathematical form

Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ be a two-dimensional matrix.

2D Transpose Law defines the operation of exchanging the row and column indices of a matrix.

The transpose operator is defined as:

$$(\cdot)^{\mathsf{T}} : \mathbb{R}^{m \times n} \to \mathbb{R}^{n \times m}$$

Define:

$$\mathbf{B} = \mathbf{A}^{\mathsf{T}}$$

Then:

$$B_{ji} = A_{ij}, \qquad 1 \le i \le m, \quad 1 \le j \le n$$

where:

$$\mathbf{B} \in \mathbb{R}^{n \times m}$$

For example:

$$\begin{bmatrix} a & b & c \\ d & e & f \end{bmatrix}^{\mathsf{T}} = \begin{bmatrix} a & d \\ b & e \\ c & f \end{bmatrix}$$

Applying the transpose twice restores the original matrix:

$$\left(\mathbf{A}^{\mathsf{T}}\right)^{\mathsf{T}} = \mathbf{A}$$

For compatible matrices, transposition also reverses the order of matrix multiplication:

$$(\mathbf{AB})^{\mathsf{T}} = \mathbf{B}^{\mathsf{T}}\mathbf{A}^{\mathsf{T}}$$

Part 3 - Hints and Helper

In PyTorch, the transpose of a two-dimensional tensor can be obtained using `torch.transpose()` by exchanging dimensions 0 and 1. The `.T` property may also be used for a two-dimensional tensor.

The reader should verify that the input tensor has exactly two dimensions and should observe that the output shape reverses the row and column sizes. The transpose changes the arrangement of indices but does not change the stored values. Special attention should be paid to the difference between transposing a matrix and reshaping it, because reshaping does not generally exchange corresponding row and column indices.

Part 4 - Python code skeleton

```
import torch

def E05(A):
    A_transpose = torch...
    return A_transpose

def main():
    A = torch...
    print(E05(...))

if __name__ == "__main__":
    main()
```

1.4.7 Exercise 06 - 2D Trace Law

Part 1 - Why this matters

In artificial intelligence systems, square matrices often represent linear transformations, covariance structures, similarity relations, or interactions between features. Analyzing every entry of such a matrix individually can be unnecessary when the goal is to obtain a compact summary of its diagonal behavior. The two-dimensional trace provides this summary by reducing the main diagonal of a square matrix to a single scalar value.

By applying the trace, artificial intelligence models and numerical algorithms can measure quantities such as the total variance stored in a covariance matrix, summarize the diagonal contribution of a transformation, and express matrix objectives in a concise and differentiable form. Trace expressions also appear in optimization, regularization, matrix calculus, and statistical learning, where they simplify operations involving sums of quadratic forms.

From a structural perspective, the trace is invariant under a change of basis and equals the sum of the eigenvalues of a square matrix, counted with algebraic multiplicity. In this sense, the two-dimensional trace is not merely a diagonal sum, but a fundamental scalar summary of a matrix transformation that remains meaningful across equivalent coordinate representations.

Part 2 - Mathematical form

Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be a two-dimensional square matrix.

2D Trace Law defines the operation of summing all entries located on the main diagonal of a square matrix.

The trace operator is defined as:

$$\mathrm{tr} : \mathbb{R}^{n \times n} \to \mathbb{R}$$

Define:

$$t = \mathrm{tr}(\mathbf{A})$$

Then:

$$t = \sum_{i=1}^{n} A_{ii}$$

where:

$$t \in \mathbb{R}$$

For example:

$$\mathbf{A} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$$

produces:

$$\operatorname{tr}(\mathbf{A}) = a + e + i$$

The trace is linear. For matrices $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n \times n}$ and scalar $\alpha \in \mathbb{R}$:

$$\operatorname{tr}(\mathbf{A} + \mathbf{B}) = \operatorname{tr}(\mathbf{A}) + \operatorname{tr}(\mathbf{B})$$

$$\operatorname{tr}(\alpha \mathbf{A}) = \alpha \operatorname{tr}(\mathbf{A})$$

For compatible matrices, the trace also satisfies the cyclic property:

$$\operatorname{tr}(\mathbf{AB}) = \operatorname{tr}(\mathbf{BA})$$

Part 3 - Hints and Helper

In PyTorch, the trace of a two-dimensional matrix is computed using `torch.trace()`. The function extracts the main diagonal and returns the sum of its entries as a scalar tensor.

The reader should verify that the input tensor has exactly two dimensions. For the mathematical law presented here, the matrix should also be square. Special attention should be paid to the main diagonal, whose entries satisfy identical row and column indices. The trace should not be confused with summing every entry in the matrix, because only entries of the form A_{ii} contribute to the result.

Part 4 - Python code skeleton

```
import torch

def E06(A):
    trace_value = torch...
    return trace_value

def main():
    A = torch...
    print(E06(...))

if __name__ == "__main__":
    main()
```

1.4.8 Exercise 07 - 2D Cofactor Law

Part 1 - Why this matters

In artificial intelligence systems, matrix transformations are often analyzed through quantities such as determinants, inverses, and local sensitivities. The two-dimensional cofactor law

provides the structural components required to expand a determinant and construct the adjugate of a square matrix. Each cofactor measures how the determinant depends on the matrix entry located at a specific row and column.

By organizing these values into a cofactor matrix, mathematical and numerical methods can examine how individual entries contribute to the global behavior of a matrix transformation. Cofactors form a classical foundation for computing matrix inverses, differentiating determinants, and studying whether a transformation preserves or collapses information.

From a learning perspective, the cofactor law reveals the local structure hidden inside determinant-based operations. Although large artificial intelligence systems normally rely on efficient matrix factorizations instead of explicitly constructing cofactors, the law remains essential for understanding determinant gradients, inverse matrices, and the algebraic relationships behind differentiable linear transformations.

Part 2 - Mathematical form

Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be a two-dimensional square matrix.

For each entry A_{ij}, let $\mathbf{M}_{ij} \in \mathbb{R}^{(n-1)\times(n-1)}$ denote the minor matrix formed by deleting row i and column j from $\mathbf{A}$.

2D Cofactor Law defines the signed determinant associated with each matrix entry.

The cofactor-matrix operator is defined as:

$$\mathrm{cof} : \mathbb{R}^{n \times n} \to \mathbb{R}^{n \times n}$$

Define:

$$\mathbf{C} = \mathrm{cof}(\mathbf{A})$$

Then each cofactor is:

$$C_{ij} = (-1)^{i+j} \det(\mathbf{M}_{ij}), \qquad 1 \le i, j \le n$$

where:

$$\mathbf{C} \in \mathbb{R}^{n \times n}$$

The signs of the cofactors follow the alternating pattern:

$$\begin{bmatrix} + & - & + & \cdots \\ - & + & - & \cdots \\ + & - & + & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$

For example, if:

$$\mathbf{A} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$$

then the cofactor of A_{12} is:

$$C_{12} = (-1)^{1+2} \det \begin{bmatrix} d & f \\ g & i \end{bmatrix} = fg - di$$

The cofactor matrix is related to the adjugate by:

$$\text{adj}(\mathbf{A}) = \mathbf{C}^{\mathsf{T}}$$

and a determinant may be expanded along any fixed row i as:

$$\det(\mathbf{A}) = \sum_{j=1}^{n} A_{ij} C_{ij}$$

Part 3 - Hints and Helper

PyTorch does not provide a direct function that constructs the complete cofactor matrix. The reader can create each minor matrix by removing one row and one column, compute its determinant using `torch.linalg.det()`, and apply the alternating sign $(-1)^{i+j}$.

Python and PyTorch use zero-based indices, while the mathematical formula uses one-based indices. The alternating sign can therefore be generated carefully from the zero-based row and column positions. The input tensor must be two-dimensional and square, and it should use a supported floating-point or complex data type when its minors are passed to `torch.linalg.det()`.

Part 4 - Python code skeleton

```
import torch

def E07(A):
    C = torch...

    for i in range(...):
        for j in range(...):
            minor = A[...]
            sign = ...
            C[i, j] = sign * torch...

    return C

def main():
    A = torch...
    print(E07(...))

if __name__ == "__main__":
    main()
```

1.4.9 Exercise 08 - 2D Adjugate Law

Part 1 - Why this matters

In artificial intelligence systems, square matrices describe transformations of features, parameters, and structured representations. Understanding how such transformations relate to

determinants and inverses requires a matrix that collects the signed minor information of every entry in the correct orientation. The two-dimensional adjugate provides this matrix by transposing the cofactor matrix.

The adjugate connects the local determinant contribution of each matrix entry to the global behavior of the complete transformation. It provides a direct algebraic relationship between a matrix, its determinant, and the identity matrix. When the determinant is nonzero, this relationship leads to the classical formula for the inverse matrix.

From a learning perspective, the adjugate law clarifies the structure behind inverse and determinant operations used in differentiable linear algebra. Although modern artificial intelligence systems normally use efficient matrix factorizations rather than explicitly constructing an adjugate, the law remains fundamental for understanding matrix sensitivity, determinant derivatives, and the algebraic conditions that make transformations reversible.

Part 2 - Mathematical form

Let $\mathbf{A} \in \mathbb{R}^{n\times n}$ be a two-dimensional square matrix, and let $\mathbf{C} = \operatorname{cof}(\mathbf{A})$ be its cofactor matrix.

2D Adjugate Law defines the adjugate of a square matrix as the transpose of its cofactor matrix.

The adjugate operator is defined as:

$$\operatorname{adj} : \mathbb{R}^{n\times n} \to \mathbb{R}^{n\times n}$$

Define:

$$\mathbf{B} = \operatorname{adj}(\mathbf{A})$$

Then:

$$\mathbf{B} = \mathbf{C}^{\mathsf{T}} = \operatorname{cof}(\mathbf{A})^{\mathsf{T}}$$

Equivalently, each entry satisfies:

$$B_{ij} = C_{ji} = (-1)^{i+j} \det(\mathbf{M}_{ji})$$

where $\mathbf{M}_{ji}$ is formed by deleting row j and column i from $\mathbf{A}$, and:

$$\mathbf{B} \in \mathbb{R}^{n\times n}$$

For the special case:

$$\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

the adjugate is:

$$\operatorname{adj}(\mathbf{A}) = \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

The adjugate satisfies the fundamental identity:

$$\mathbf{A}\operatorname{adj}(\mathbf{A}) = \operatorname{adj}(\mathbf{A})\mathbf{A} = \det(\mathbf{A})\mathbf{I}_n$$

If $\det(\mathbf{A}) \neq 0$, then:

$$\mathbf{A}^{-1} = \frac{1}{\det(\mathbf{A})} \operatorname{adj}(\mathbf{A})$$

The adjugate remains defined even when $\mathbf{A}$ is singular, although the inverse formula cannot be used when $\det(\mathbf{A}) = 0$.

Part 3 - Hints and Helper

PyTorch does not provide a direct function for constructing the adjugate of a matrix. The reader can first construct the cofactor matrix using the method from the 2D Cofactor Law, then transpose it using `torch.transpose()` or the `.T` property.

The input tensor must be two-dimensional and square. Special attention should be paid to the order of operations: the adjugate is the transpose of the cofactor matrix, not the cofactor matrix itself. Computing $\det(\mathbf{A})\mathbf{A}^{-1}$ may reproduce the adjugate for a nonsingular matrix, but that approach is invalid for singular matrices even though their adjugates still exist.

Part 4 - Python code skeleton

```
import torch

def E08(A):
    cofactor_matrix = torch...
    adjugate_matrix = torch...
    return adjugate_matrix

def main():
    A = torch...
    print(E08(...))

if __name__ == "__main__":
    main()
```

1.4.10 Exercise 09 - 2D Rank Law

Part 1 - Why this matters

In artificial intelligence systems, matrices store features, transformations, and relationships between variables. Not every row or column necessarily adds new information: some may be linear combinations of others. The two-dimensional rank measures the number of independent directions contained in a matrix and therefore reveals its effective dimensionality.

Rank is important when diagnosing redundant features, compressed representations, singular transformations, and low-rank approximations. A full- rank weight matrix preserves the maximum number of directions allowed by its shape, while a rank-deficient matrix collapses part of the represented space.

From a learning perspective, rank connects matrix structure to information capacity. It appears in dimensionality reduction, matrix factorization, attention analysis, parameter-efficient learning, and numerical stability.

Part 2 - Mathematical form

Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ be a two-dimensional matrix.

2D Rank Law defines the dimension of the vector space spanned by the columns of $\mathbf{A}$, equivalently by its rows.

The rank operator is defined as:

$$\text{rank} : \mathbb{R}^{m \times n} \to \{0, 1, \ldots, \min(m, n)\}$$

Define:

$$r = \text{rank}(\mathbf{A})$$

Then:

$$r = \dim(\text{Col}(\mathbf{A})) = \dim(\text{Row}(\mathbf{A}))$$

and:

$$0 \leq r \leq \min(m, n)$$

The matrix has full rank when:

$$r = \min(m, n)$$

For a square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$:

$$\text{rank}(\mathbf{A}) = n \quad \iff \quad \det(\mathbf{A}) \neq 0$$

Part 3 - Hints and Helper

In PyTorch, matrix rank is computed using `torch.linalg.matrix_rank()`. The function estimates numerical rank by counting singular values that exceed a tolerance.

The reader should use a floating-point or complex tensor and remember that numerical rank can depend on tolerance. Values that are mathematically nonzero but extremely small may be treated as zero because of finite precision.

Part 4 - Python code skeleton

```
import torch

def E09(A):
    rank_value = torch...
    return rank_value

def main():
    A = torch...
    print(E09(...))

if __name__ == "__main__":
    main()
```

1.4.11 Exercise 10 - 2D Row Sum Law

Part 1 - Why this matters

In artificial intelligence systems, each row of a matrix often represents one observation, token, sample, or spatial position, while its columns represent features. The two-dimensional row sum compresses every row into one scalar by aggregating all of its feature values.

Row sums are useful for computing per-sample totals, checking probability normalization, constructing masks, and summarizing activations. They preserve the row axis while reducing the feature axis, allowing an entire batch to be processed simultaneously.

From a structural perspective, the row sum is an axis-specific reduction. Its meaning depends on which axis represents features, so selecting the correct dimension is as important as performing the addition itself.

Part 2 - Mathematical form

Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ be a two-dimensional matrix.
2D Row Sum Law defines the sum of all entries in each row.
The row-sum operator is defined as:

$$\text{rowsum} : \mathbb{R}^{m \times n} \to \mathbb{R}^{m}$$

Define:

$$\mathbf{s} = \text{rowsum}(\mathbf{A})$$

Then:

$$s_i = \sum_{j=1}^{n} A_{ij}, \qquad 1 \le i \le m$$

where:

$$\mathbf{s} \in \mathbb{R}^{m}$$

For example:

$$\begin{bmatrix} a & b & c \\ d & e & f \end{bmatrix} \mapsto \begin{bmatrix} a+b+c \\ d+e+f \end{bmatrix}$$

Part 3 - Hints and Helper

In PyTorch, use `torch.sum()` with the dimension corresponding to the columns. For a tensor shaped (m, n), rows are retained by reducing `dim=1`.

The reader should distinguish the mathematical axis from Python's zero-based dimension number. Using `dim=0` would calculate column sums instead.

Part 4 - Python code skeleton

```
import torch

def E10(A):
    row_sums = torch...
    return row_sums
```

```
def main():
    A = torch...
    print(E10(...))

if __name__ == "__main__":
    main()
```

1.4.12 Exercise 11 - 2D Column Sum Law

Part 1 - Why this matters

In artificial intelligence systems, matrix columns frequently represent features while rows represent samples, tokens, or observations. The two-dimensional column sum aggregates each feature across all rows, producing one total for every column.

Column sums can summarize feature activity across a batch, count activated conditions, accumulate statistics, and support normalization procedures. They reduce the observation axis while preserving the feature axis.

From a structural perspective, the column sum demonstrates that matrix reductions are axis-dependent. Reducing the wrong axis changes both the output shape and the meaning of the result.

Part 2 - Mathematical form

Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ be a two-dimensional matrix.
2D Column Sum Law defines the sum of all entries in each column.
The column-sum operator is defined as:

$$\text{colsum} : \mathbb{R}^{m \times n} \to \mathbb{R}^n$$

Define:

$$\mathbf{s} = \text{colsum}(\mathbf{A})$$

Then:

$$s_j = \sum_{i=1}^{m} A_{ij}, \qquad 1 \le j \le n$$

where:

$$\mathbf{s} \in \mathbb{R}^n$$

For example:

$$\begin{bmatrix} a & b & c \\ d & e & f \end{bmatrix} \mapsto \begin{bmatrix} a+d & b+e & c+f \end{bmatrix}$$

Part 3 - Hints and Helper

In PyTorch, use `torch.sum()` with the dimension corresponding to the rows. For a tensor shaped (m, n), columns are retained by reducing `dim=0`.

The reader should check the output shape carefully. Using `dim=1` would calculate row sums instead.

Part 4 - Python code skeleton

```
import torch

def E11(A):
    column_sums = torch...
    return column_sums

def main():
    A = torch...
    print(E11(...))

if __name__ == "__main__":
    main()
```

1.4.13 Exercise 12 - 2D Row Mean Law

Part 1 - Why this matters

In artificial intelligence systems, rows commonly represent individual samples, tokens, or positions and columns represent their features. The two-dimensional row mean summarizes every row by its average feature value, producing a scale-aware representation that does not grow with the number of columns.

Row means appear in pooling, per-sample statistics, activation monitoring, and data analysis. Unlike a row sum, the mean divides by the number of features, which makes comparisons across equally interpreted rows more direct.

From a structural perspective, the row mean is a reduction over the column axis. It retains one result per row and therefore preserves the matrix's sample organization while compressing its feature dimension.

Part 2 - Mathematical form

Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ be a two-dimensional matrix with $n > 0$.

2D Row Mean Law defines the arithmetic mean of each row.

The row-mean operator is defined as:

$$\text{rowmean} : \mathbb{R}^{m \times n} \to \mathbb{R}^m$$

Define:

$$\boldsymbol{\mu} = \text{rowmean}(\mathbf{A})$$

Then:

$$\mu_i = \frac{1}{n} \sum_{j=1}^{n} A_{ij}, \qquad 1 \le i \le m$$

where:

$$\boldsymbol{\mu} \in \mathbb{R}^m$$

Equivalently:

$$\text{rowmean}(\mathbf{A}) = \frac{1}{n} \text{rowsum}(\mathbf{A})$$

Part 3 - Hints and Helper

In PyTorch, use `torch.mean()` and reduce the column dimension. For a tensor shaped (m, n), this corresponds to `dim=1`.

The input should use a floating-point or complex data type. The reader should not use `dim=0`, because that would produce column means.

Part 4 - Python code skeleton

```
import torch

def E12(A):
    row_means = torch...
    return row_means

def main():
    A = torch...
    print(E12(...))

if __name__ == "__main__":
    main()
```

1.4.14 Exercise 13 - 2D Column Mean Law

Part 1 - Why this matters

In artificial intelligence systems, matrix columns often correspond to features and rows correspond to samples or observations. The two-dimensional column mean calculates the average value of each feature across the complete set of rows.

Column means are fundamental to feature centering, dataset statistics, normalization, bias analysis, and activation monitoring. They provide one representative value per feature while removing the batch or observation axis.

From a structural perspective, the column mean is a reduction over rows. It preserves the feature axis and produces the mean vector commonly subtracted from every row when centering a dataset.

Part 2 - Mathematical form

Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ be a two-dimensional matrix with $m > 0$.

2D Column Mean Law defines the arithmetic mean of each column.

The column-mean operator is defined as:

$$\text{colmean} : \mathbb{R}^{m \times n} \to \mathbb{R}^{n}$$

Define:

$$\boldsymbol{\mu} = \text{colmean}(\mathbf{A})$$

Then:

$$\mu_j = \frac{1}{m} \sum_{i=1}^{m} A_{ij}, \qquad 1 \leq j \leq n$$

where:

$$\boldsymbol{\mu} \in \mathbb{R}^n$$

Equivalently:

$$\text{colmean}(\mathbf{A}) = \frac{1}{m}\,\text{colsum}(\mathbf{A})$$

Part 3 - Hints and Helper

In PyTorch, use `torch.mean()` and reduce the row dimension. For a tensor shaped (m, n), this corresponds to `dim=0`.

The input should use a floating-point or complex data type. The reader should check that the output contains one mean for each original column.

Part 4 - Python code skeleton

```
import torch

def E13(A):
    column_means = torch...
    return column_means

def main():
    A = torch...
    print(E13(...))

if __name__ == "__main__":
    main()
```

1.4.15 Exercise 14 - 2D Broadcasting Law

Part 1 - Why this matters

In artificial intelligence systems, matrix operations frequently combine a large collection of values with a smaller tensor such as a bias vector, scale vector, or normalization statistic. Explicitly copying the smaller tensor to match the larger shape would waste memory and obscure the intended operation. Two-dimensional broadcasting provides a rule for aligning compatible shapes and applying the operation as if the smaller tensor had been repeated.

Broadcasting allows one bias value to be applied to every row, one scale value to be applied to every column, or one scalar to interact with an entire matrix. It is therefore central to neural-network layers, feature normalization, attention masks, and element-wise transformations.

From a structural perspective, broadcasting is a shape-compatibility law. It does not define a new arithmetic operator; instead, it determines how existing element-wise operators act on tensors with different but compatible shapes.

Part 2 - Mathematical form

Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{b} \in \mathbb{R}^n$.

2D Broadcasting Law allows **b** to behave as if it were repeated across all m rows of **A** during an element-wise operation.

For broadcasted addition, define:

$$\mathbf{C} = \mathbf{A} + \mathbf{b}$$

Then:

$$C_{ij} = A_{ij} + b_j, \qquad 1 \leq i \leq m, \quad 1 \leq j \leq n$$

where:

$$\mathbf{C} \in \mathbb{R}^{m \times n}$$

For general tensor shapes, dimensions are compared from right to left. Two dimensions are compatible when they are equal or when at least one of them is 1. Missing leading dimensions are treated as dimensions of size 1.

For example:

$$(m, n) \text{ and } (n) \quad \longrightarrow \quad (m, n)$$

and:

$$(m, n) \text{ and } (m, 1) \quad \longrightarrow \quad (m, n)$$

Broadcasting is invalid when corresponding dimensions are unequal and neither dimension is 1.

Part 3 - Hints and Helper

PyTorch applies broadcasting automatically to compatible element-wise operations such as addition, subtraction, multiplication, and division. The reader may use `torch.broadcast_tensors()` to inspect the aligned shapes explicitly.

The reader should compare shapes from the final dimension toward the first. Special attention should be paid to the difference between shapes $(m,)$, $(m, 1)$, and $(1, m)$. Broadcasting conceptually expands singleton dimensions, but it does not normally create repeated copies of the underlying data.

Part 4 - Python code skeleton

```
import torch

def E14(A, b):
    C = A ... b
    return C

def main():
    A = torch...
    b = torch...
    print(E14(...))

if __name__ == "__main__":
    main()
```

1.5 T03 - Higher-Dimension Tensor Laws

Mathematics and Theory

1.5.1 Exercise 00 - 3D Tensor Multiplication Law

Part 1 - Why this matters

Three-dimensional tensors commonly represent batches of feature maps, sequences, or volumetric data. Element-wise multiplication allows two aligned representations to interact without mixing their positions. It is used in gating, masking, attention weighting, and feature modulation, where every value is controlled by the value at the same index in another tensor.

Unlike matrix multiplication, this law preserves the complete tensor shape. It therefore expresses local interaction rather than contraction between axes.

Part 2 - Mathematical form

Let $\mathcal{A}, \mathcal{B} \in \mathbb{R}^{d_1 \times d_2 \times d_3}$.

3D Tensor Multiplication Law defines:

$$\odot : \mathbb{R}^{d_1 \times d_2 \times d_3} \times \mathbb{R}^{d_1 \times d_2 \times d_3} \to \mathbb{R}^{d_1 \times d_2 \times d_3}$$

$$\mathcal{C} = \mathcal{A} \odot \mathcal{B}, \qquad C_{ijk} = A_{ijk} B_{ijk}$$

for $1 \leq i \leq d_1$, $1 \leq j \leq d_2$, and $1 \leq k \leq d_3$.

Part 3 - Hints and Helper

In PyTorch, aligned element-wise multiplication may be written with `torch.mul()` or the `*` operator. Verify that the shapes are equal or broadcasting-compatible, and do not confuse this operation with `torch.matmul()`.

Part 4 - Python code skeleton

```
import torch

def E00(A, B):
    C = torch...
    return C

def main():
    A = torch...
    B = torch...
    print(E00(...))

if __name__ == "__main__":
    main()
```

1.5.2 Exercise 01 - 3D Outer Product Law

Part 1 - Why this matters

The outer product constructs higher-order interactions from lower-dimensional representations. Given two vectors, it records every pairwise product in a matrix; extending the idea to three vectors produces a three-dimensional tensor containing every triple interaction. This is

useful for multilinear models, feature interaction analysis, and tensor factorization.

The operation increases dimensionality without summing over any index, thereby preserving every combination of input coordinates.

Part 2 - Mathematical form

Let $\mathbf{a} \in \mathbb{R}^m$, $\mathbf{b} \in \mathbb{R}^n$, and $\mathbf{c} \in \mathbb{R}^p$.
3D Outer Product Law defines:

$$\otimes : \mathbb{R}^m \times \mathbb{R}^n \times \mathbb{R}^p \to \mathbb{R}^{m \times n \times p}$$

$$\mathcal{T} = \mathbf{a} \otimes \mathbf{b} \otimes \mathbf{c}, \qquad T_{ijk} = a_i b_j c_k$$

No axis is contracted, and the output contains mnp entries.

Part 3 - Hints and Helper

In PyTorch, `torch.einsum()` can express the complete operation directly. It may also be built by reshaping the vectors to singleton-aligned shapes and using broadcasting. Ensure that no index is repeated in the output equation.

Part 4 - Python code skeleton

```
import torch

def E01(a, b, c):
    T = torch...
    return T

def main():
    a = torch...
    b = torch...
    c = torch...
    print(E01(...))

if __name__ == "__main__":
    main()
```

1.5.3 Exercise 02 - 3D Einstein Summation Law

Part 1 - Why this matters

Einstein summation provides a compact language for tensor operations. Repeated indices identify axes to multiply and sum, while unrepeated output indices are preserved. A single notation can express dot products, matrix multiplication, batch operations, transposes, reductions, and general contractions.

This makes complex neural-network transformations easier to describe and often allows PyTorch to select an efficient computational path.

Part 2 - Mathematical form

Let $\mathcal{A} \in \mathbb{R}^{b\times m\times n}$ and $\mathcal{B} \in \mathbb{R}^{b\times n\times p}$.
Using Einstein notation, define:

$$C_{bij} = \sum_{k=1}^{n} A_{bik} B_{bkj}$$

or compactly:

$$C_{bij} = A_{bik} B_{bkj}$$

where the repeated index k is summed and b, i, j remain in the output. Thus $\mathcal{C} \in \mathbb{R}^{b\times m\times p}$.

Part 3 - Hints and Helper

In PyTorch, use `torch.einsum()` with an equation describing input and output indices. Every dimension sharing a label must have a compatible size. Carefully distinguish the batch index from the contracted index.

Part 4 - Python code skeleton

```
import torch

def E02(A, B):
    C = torch.einsum("...", A, B)
    return C

def main():
    A = torch...
    B = torch...
    print(E02(...))

if __name__ == "__main__":
    main()
```

1.5.4 Exercise 03 - Batch Matrix Multiplication Law

Part 1 - Why this matters

Neural networks process many samples simultaneously. Batch matrix multiplication applies an independent matrix product to every batch element, allowing sequences, attention heads, and feature maps to be transformed in parallel without explicit Python loops.

The batch axis is preserved while the final two axes follow ordinary matrix multiplication.

Part 2 - Mathematical form

Let $\mathcal{A} \in \mathbb{R}^{b\times m\times n}$ and $\mathcal{B} \in \mathbb{R}^{b\times n\times p}$.
Batch Matrix Multiplication Law defines:

$$\mathcal{C} = \mathcal{A} \mathbin{@} \mathcal{B} \in \mathbb{R}^{b\times m\times p}$$

$$C_{rij} = \sum_{k=1}^{n} A_{rik} B_{rkj}, \qquad 1 \le r \le b$$

Each batch slice satisfies $\mathbf{C}_r = \mathbf{A}_r\mathbf{B}_r$.

Part 3 - Hints and Helper

In PyTorch, `torch.bmm()` performs batch multiplication for two 3D tensors and does not broadcast the batch dimension. Check that batch sizes and inner matrix dimensions match. `torch.matmul()` supports broader broadcasting behavior.

Part 4 - Python code skeleton

```
import torch

def E03(A, B):
    C = torch...
    return C

def main():
    A = torch...
    B = torch...
    print(E03(...))

if __name__ == "__main__":
    main()
```

1.5.5 Exercise 04 - Tensor Contraction Law

Part 1 - Why this matters

Tensor contraction combines tensors by multiplying along selected axes and summing those axes away. It generalizes dot products and matrix multiplication, forming the basis of attention, multilinear layers, tensor networks, and many scientific machine-learning computations.

The choice of contracted axes determines both the meaning and shape of the result.

Part 2 - Mathematical form

Let $\mathcal{A} \in \mathbb{R}^{a\times b\times c}$ and $\mathcal{B} \in \mathbb{R}^{c\times d\times e}$.

Contracting the third axis of $\mathcal{A}$ with the first axis of $\mathcal{B}$ gives $\mathcal{C} \in \mathbb{R}^{a\times b\times d\times e}$:

$$C_{ij\ell q} = \sum_{k=1}^{c} A_{ijk}B_{k\ell q}$$

The contracted axes must have equal sizes and do not appear in the output.

Part 3 - Hints and Helper

In PyTorch, `torch.tensordot()` accepts the axes to contract. `torch.einsum()` can express the same operation with named index labels. Verify the axis order of the output after contraction.

Part 4 - Python code skeleton

```
import torch

def E04(A, B):
    C = torch.tensordot(A, B, dims=...)
    return C

def main():
    A = torch...
    B = torch...
    print(E04(...))

if __name__ == "__main__":
    main()
```

1.5.6 Exercise 05 - Tensor Repetition Law

Part 1 - Why this matters

Tensor repetition duplicates values along selected dimensions. It is useful when constructing repeated patterns, matching explicit batch shapes, or generating structured training data. Unlike broadcasting, repetition creates a larger tensor whose data pattern is physically repeated.

Because it can increase memory use substantially, the repetition counts should be chosen deliberately.

Part 2 - Mathematical form

Let $\mathcal{A} \in \mathbb{R}^{d_1 \times d_2 \times d_3}$ and let $r_1, r_2, r_3 \in \mathbb{N}$.

The repeated tensor has shape:

$$\mathcal{R} \in \mathbb{R}^{(r_1 d_1) \times (r_2 d_2) \times (r_3 d_3)}$$

and entries:

$$R_{ijk} = A_{((i-1) \bmod d_1)+1,((j-1) \bmod d_2)+1,((k-1) \bmod d_3)+1}$$

Part 3 - Hints and Helper

In PyTorch, `Tensor.repeat()` accepts one repetition count per dimension. The number of counts must be at least the number of tensor dimensions. Do not confuse `repeat()` with a broadcasted view.

Part 4 - Python code skeleton

```
import torch

def E05(A, repeats):
    R = A.repeat(...)
    return R

def main():
    A = torch...
```

```
    repeats = (...)
    print(E05(...))

if __name__ == "__main__":
    main()
```

1.5.7 Exercise 06 - Tensor Tiling Law

Part 1 - Why this matters

Tensor tiling repeats a complete tensor pattern across one or more axes. It can construct grids, repeated positional patterns, synthetic feature layouts, and explicitly expanded examples. Tiling resembles repetition conceptually but emphasizes placing copies of the full pattern in adjacent blocks.

Since the output stores all copies, its memory cost grows with the product of the repetition factors.

Part 2 - Mathematical form

Let $\mathcal{A} \in \mathbb{R}^{d_1 \times d_2 \times d_3}$ and $r_1, r_2, r_3 \in \mathbb{N}$.

Define:

$$\mathcal{T} = \text{tile}(\mathcal{A}; r_1, r_2, r_3) \in \mathbb{R}^{r_1 d_1 \times r_2 d_2 \times r_3 d_3}$$

with the periodic indexing rule:

$$T_{ijk} = A_{i \bmod d_1, j \bmod d_2, k \bmod d_3}$$

when zero-based indices are used.

Part 3 - Hints and Helper

In PyTorch, `torch.tile()` takes a tuple of repetition factors. If fewer factors than dimensions are supplied, leading factors of one are added. Check the resulting shape and distinguish tiling from singleton expansion.

Part 4 - Python code skeleton

```
import torch

def E06(A, dims):
    T = torch...
    return T

def main():
    A = torch...
    dims = (...)
    print(E06(...))

if __name__ == "__main__":
    main()
```

1.5.8 Exercise 07 - Tensor Broadcasting Law

Part 1 - Why this matters

Higher-dimensional broadcasting allows tensors such as biases, masks, and normalization statistics to interact with batches, channels, and spatial axes without explicitly copying data. It is fundamental to vectorized neural- network computation and reduces both code complexity and memory use.

Broadcasting aligns shapes from the final dimension toward the first and conceptually expands singleton axes.

Part 2 - Mathematical form

Two dimensions are broadcasting-compatible when they are equal or at least one is 1. Missing leading dimensions are treated as 1.

For $\mathcal{A} \in \mathbb{R}^{b\times m\times n}$ and $\mathbf{v} \in \mathbb{R}^{n}$:

$$\mathcal{C} = \mathcal{A} + \mathbf{v} \in \mathbb{R}^{b\times m\times n}$$

$$C_{ijk} = A_{ijk} + v_k$$

The output size of each aligned axis is the larger compatible size.

Part 3 - Hints and Helper

PyTorch broadcasts compatible tensors automatically in element-wise operations. Use `torch.broadcast_shapes()` or `torch.broadcast_tensors()` to inspect compatibility. Pay attention to the difference between shapes (n), $(1, n)$, and $(n, 1)$.

Part 4 - Python code skeleton

```
import torch

def E07(A, v):
    C = A ... v
    return C

def main():
    A = torch...
    v = torch...
    print(E07(...))

if __name__ == "__main__":
    main()
```

1.5.9 Exercise 08 - Tensor Indexing Law

Part 1 - Why this matters

Tensor indexing retrieves specific values or lower-dimensional sections from a higher-dimensional representation. Neural-network code uses indexing to select samples, tokens, channels, coordinates, and labels. Correct indexing preserves the semantic meaning of every axis and avoids

silently selecting the wrong data.

Selecting one integer index removes that axis from the output, while selecting a range can preserve it.

Part 2 - Mathematical form

Let $\mathcal{A} \in \mathbb{R}^{d_1 \times d_2 \times d_3}$. Selecting indices (i, j, k) produces the scalar:

$$x = A_{ijk} \in \mathbb{R}$$

Selecting only the first index produces a matrix slice:

$$\mathbf{M} = \mathcal{A}_{i,:,:} \in \mathbb{R}^{d_2 \times d_3}$$

Valid mathematical indices satisfy $1 \leq i \leq d_1$, $1 \leq j \leq d_2$, and $1 \leq k \leq d_3$.

Part 3 - Hints and Helper

PyTorch uses zero-based indexing, so the first position has index 0. Negative indices count backward from the end. Verify the output dimension after every integer index and avoid indices outside the valid range.

Part 4 - Python code skeleton

```
import torch

def E08(A, i, j, k):
    value = A[...]
    return value

def main():
    A = torch...
    print(E08(...))

if __name__ == "__main__":
    main()
```

1.5.10 Exercise 09 - Tensor Advanced Slicing Law

Part 1 - Why this matters

Advanced slicing extracts structured regions rather than single entries. It allows models and data pipelines to select time windows, spatial patches, channel intervals, reversed sequences, or regularly sampled positions without writing explicit loops.

Each axis may specify a start, stop, and step, making the resulting shape a direct consequence of the chosen index ranges.

Part 2 - Mathematical form

Let $\mathcal{A} \in \mathbb{R}^{d_1 \times d_2 \times d_3}$. A slice along each axis is described by index sets I, J, and K:

$$\mathcal{S} = \mathcal{A}_{I,J,K}$$

If $I = (s_1 : s_2 : q)$, it selects indices:

$$s_1,\ s_1 + q,\ s_1 + 2q, \ldots < s_2$$

for a positive step q. The output shape equals the number of selected indices on each preserved axis.

Part 3 - Hints and Helper

In PyTorch, use Python slice notation `start:stop:step`. The stop index is excluded. Use `:` to retain an entire axis and `...` to represent omitted intermediate axes. A negative step is not supported directly in ordinary PyTorch slicing; use `torch.flip()` when reversal is required.

Part 4 - Python code skeleton

```
import torch

def E09(A):
    S = A[..., ..., ...]
    return S

def main():
    A = torch...
    print(E09(A))

if __name__ == "__main__":
    main()
```

1.5.11 Exercise 10 - Tensor Padding Law

Part 1 - Why this matters

Padding adds values around tensor boundaries. Convolutional networks use it to control spatial output size, preserve edge information, align tensors, and make dimensions compatible with later operations. Padding may use constants, reflection, replication, or circular continuation.

The selected padding order and mode determine both the new shape and the values introduced at the boundary.

Part 2 - Mathematical form

Let $\mathcal{A} \in \mathbb{R}^{c \times h \times w}$ and add $p_l, p_r, p_t, p_b \geq 0$ values to the left, right, top, and bottom.

The padded tensor satisfies:

$$\mathcal{P} \in \mathbb{R}^{c \times (h + p_t + p_b) \times (w + p_l + p_r)}$$

For constant padding value v, original entries are shifted by (p_t, p_l), while all newly created boundary positions equal v.

Part 3 - Hints and Helper

In PyTorch, use `torch.nn.functional.pad()`. Padding values are listed from the final dimension backward; for 2D spatial padding the order is left, right, top, bottom. Check which modes support the input dimensionality.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E10(A, padding, value=0):
    P = F.pad(A, ..., mode="...", value=value)
    return P

def main():
    A = torch...
    print(E10(...))

if __name__ == "__main__":
    main()
```

1.5.12 Exercise 11 - Tensor Cropping Law

Part 1 - Why this matters

Cropping removes boundary regions or extracts a focused subregion from a tensor. It is used to align feature maps, remove padded areas, select image patches, and restrict computation to a region of interest. Cropping changes the shape without modifying values that remain inside the selected region.

Mathematically, cropping is a bounded slicing operation.

Part 2 - Mathematical form

Let $\mathcal{A} \in \mathbb{R}^{c \times h \times w}$. Removing $r_t, r_b, r_l, r_r \geq 0$ positions from its boundaries gives:

$$\mathcal{C} = \mathcal{A}_{:, r_t:h-r_b, r_l:w-r_r}$$

$$\mathcal{C} \in \mathbb{R}^{c \times (h-r_t-r_b) \times (w-r_l-r_r)}$$

The retained dimensions must remain positive.

Part 3 - Hints and Helper

PyTorch implements cropping naturally with slicing. Remember that stop indices are excluded. Validate that the crop sizes do not remove more elements than an axis contains, and preserve the channel axis with `:`.

Part 4 - Python code skeleton

```
import torch

def E11(A, top, bottom, left, right):
    C = A[..., ..., ...]
    return C

def main():
    A = torch...
    print(E11(...))

if __name__ == "__main__":
    main()
```

1.5.13 Exercise 12 - Tensor Stacking Law

Part 1 - Why this matters

Stacking combines equally shaped tensors by creating a new axis. It is used to assemble samples into batches, collect model outputs, organize time steps, and construct multi-view representations. Unlike concatenation, stacking does not extend an existing axis; it increases tensor dimensionality.

The position of the new axis determines the semantic organization of the result.

Part 2 - Mathematical form

Let $\mathcal{A}^{(1)}, \ldots, \mathcal{A}^{(q)} in \mathbb{R}^{d_1 \times d_2 \times d_3}$.

Stacking along a new leading axis defines:

$$\mathcal{S} = \text{stack}(\mathcal{A}^{(1)}, \ldots, \mathcal{A}^{(q)}) \in \mathbb{R}^{q \times d_1 \times d_2 \times d_3}$$

$$S_{rijk} = A^{(r)}_{ijk}$$

All input tensors must have identical shapes.

Part 3 - Hints and Helper

In PyTorch, use `torch.stack()` and specify `dim` for the new axis. Compare it with `torch.cat()`, which requires an existing axis and does not add a dimension.

Part 4 - Python code skeleton

```
import torch

def E12(tensors):
    S = torch.stack(tensors, dim=...)
    return S

def main():
    tensors = [torch..., torch...]
    print(E12(...))

if __name__ == "__main__":
    main()
```

1.5.14 Exercise 13 - Tensor Splitting Law

Part 1 - Why this matters

Tensor splitting divides one tensor into ordered sections along a chosen axis. It supports separating batches, partitioning features, distributing work, and recovering components that were previously concatenated. The original values and order are preserved across the output sections.

Split sizes may be equal or explicitly specified, subject to the size of the selected axis.

Part 2 - Mathematical form

Let $\mathcal{A} \in \mathbb{R}^{d_1 \times d_2 \times d_3}$ and split its second axis into sizes $s_1, \ldots, s_q$ satisfying:

$$\sum_{r=1}^{q} s_r = d_2$$

Then:

$$(\mathcal{A}^{(1)}, \ldots, \mathcal{A}^{(q)}) = \text{split}(\mathcal{A}; s_1, \ldots, s_q)$$

where $\mathcal{A}^{(r)} \in \mathbb{R}^{d_1 \times s_r \times d_3}$.

Part 3 - Hints and Helper

In PyTorch, `torch.split()` accepts either one section size or a list of sizes. When one size is supplied, the final section may be smaller. Select the axis with `dim` and verify that explicit sizes sum correctly.

Part 4 - Python code skeleton

```
import torch

def E13(A, sizes, dim):
    parts = torch.split(A, ..., dim=...)
    return parts

def main():
    A = torch...
    print(E13(...))

if __name__ == "__main__":
    main()
```

1.5.15 Exercise 14 - Tensor Chunking Law

Part 1 - Why this matters

Chunking partitions a tensor into a requested number of approximately equal pieces along one axis. It is useful for minibatch processing, parallel work, memory-limited computation, and dividing feature groups without manually calculating every boundary.

When the axis size is not divisible by the requested number, the final chunk may be smaller, and PyTorch may return fewer chunks than requested.

Part 2 - Mathematical form

Let $\mathcal{A} \in \mathbb{R}^{d_1 \times d_2 \times d_3}$ and request q chunks along an axis of size d. The tensor is partitioned into consecutive sections whose regular chunk size is determined from d and q, while the final returned section may contain fewer entries:

$$(\mathcal{A}^{(1)}, \ldots, \mathcal{A}^{(r)}) = \text{chunk}(\mathcal{A}; q)$$

where $r \leq q$ and concatenating the sections along the same axis reconstructs $\mathcal{A}$.

Part 3 - Hints and Helper

In PyTorch, use `torch.chunk()` with the requested number of chunks and the target `dim`. Do not assume that every chunk has the same size or that the number returned is always exactly the number requested; inspect the resulting tuple and shapes.

Part 4 - Python code skeleton

```
import torch

def E14(A, chunks, dim):
    parts = torch.chunk(A, ..., dim=...)
    return parts

def main():
    A = torch...
    print(E14(...))

if __name__ == "__main__":
    main()
```

1.6 T04 - Tensor shape Tranformation Laws

Mathematics and Theory

1.6.1 Exercise 00 - Tensor Reshape Law

Part 1 - Why this matters

Neural networks frequently reinterpret the same stored values under a new shape. Reshaping converts images into token sequences, separates attention heads, prepares activations for linear layers, and restores structured outputs. The values and their linear order remain unchanged; only their organization across dimensions changes.

Part 2 - Mathematical form

Let $\mathcal{A}$ have shape $(d_1, \ldots, d_k)$ and let the target shape be $(e_1, \ldots, e_r)$. Reshaping is valid when:

$$\prod_{i=1}^{k} d_i = \prod_{j=1}^{r} e_j$$

$$\mathcal{B} = \text{reshape}(\mathcal{A}; e_1, \ldots, e_r)$$

The flattened element order of $\mathcal{B}$ equals that of $\mathcal{A}$.

Part 3 - Hints and Helper

In PyTorch, use `torch.reshape()` or `Tensor.reshape()`. One target dimension may be `-1` so PyTorch infers its size. Reshape may return a view or a copy, so code should not rely on a particular storage result.

Part 4 - Python code skeleton

```
import torch

def E00(A, shape):
    B = torch...
    return B

def main():
    A = torch...
    print(E00(...))

if __name__ == "__main__":
    main()
```

1.6.2 Exercise 01 - Tensor View Law

Part 1 - Why this matters

A tensor view presents existing storage with a different shape without copying its data. Views make shape transformations memory-efficient and are common when preparing activations for attention, convolution, and linear algebra. Because the tensors share storage, a permitted in-place change can be visible through both representations.

Part 2 - Mathematical form

For $\mathcal{A}$ with N elements, a view of shape $(e_1, \ldots, e_r)$ requires:

$$N = \prod_{j=1}^{r} e_j$$

$$\mathcal{B} = \text{view}(\mathcal{A}; e_1, \ldots, e_r)$$

and $\mathcal{A}$ and $\mathcal{B}$ reference the same underlying data while using different index-to-storage mappings.

Part 3 - Hints and Helper

Use `Tensor.view()` in PyTorch. The requested shape must be compatible with the tensor's size and strides. A non-contiguous tensor may need `contiguous()` first; alternatively, `reshape()` can handle more cases by copying when necessary.

Part 4 - Python code skeleton

```
import torch

def E01(A, shape):
    B = A.view(...)
    return B

def main():
    A = torch...
    print(E01(...))

if __name__ == "__main__":
    main()
```

1.6.3 Exercise 02 - Tensor Permute Law

Part 1 - Why this matters

Permutation reorders every tensor axis. It converts between layouts such as channels-first and channels-last, rearranges batch and sequence axes, and aligns dimensions for later operations. No values are added or removed; only the meaning and position of axes change.

Part 2 - Mathematical form

Let $\mathcal{A}$ have shape $(d_0, \ldots, d_{k-1})$ and let π be a permutation of $(0, \ldots, k-1)$. Then:

$$\mathcal{B} = \text{permute}(\mathcal{A}; \pi)$$

has shape $(d_{\pi(0)}, \ldots, d_{\pi(k-1)})$, with:

$$B_{i_0 \ldots i_{k-1}} = A_{j_0 \ldots j_{k-1}}, \qquad i_r = j_{\pi(r)}$$

Part 3 - Hints and Helper

Use `Tensor.permute()` in PyTorch and provide every dimension exactly once. The result is usually a non-contiguous view; call `contiguous()` only when a later operation requires contiguous storage.

Part 4 - Python code skeleton

```
import torch

def E02(A, dims):
    B = A.permute(...)
    return B
```

```
def main():
    A = torch...
    print(E02(...))

if __name__ == "__main__":
    main()
```

1.6.4 Exercise 03 - Tensor Transpose Law

Part 1 - Why this matters

Tensor transpose swaps two selected dimensions while leaving all other axes in place. It aligns matrices inside batches, exchanges spatial axes, and prepares query, key, or feature dimensions for multiplication. Transpose is therefore a special case of axis permutation.

Part 2 - Mathematical form

Let $\mathcal{A}$ have shape $(d_0, \ldots, d_{k-1})$. Swapping dimensions p and q gives:

$$\mathcal{B} = \text{transpose}(\mathcal{A}; p, q)$$

where the output shape equals the input shape with d_p and d_q exchanged. Applying the same transpose twice restores $\mathcal{A}$.

Part 3 - Hints and Helper

Use `torch.transpose()` or `Tensor.transpose()` with two dimension indices. Unlike `permute()`, transpose swaps only two axes. The result generally shares storage and may be non-contiguous.

Part 4 - Python code skeleton

```
import torch

def E03(A, dim0, dim1):
    B = torch.transpose(A, ..., ...)
    return B

def main():
    A = torch...
    print(E03(...))

if __name__ == "__main__":
    main()
```

1.6.5 Exercise 04 - Tensor Expand Law

Part 1 - Why this matters

Expansion makes singleton dimensions behave as larger dimensions without copying their values. It efficiently prepares biases, masks, and shared features for operations across batches or spatial positions. Expanded entries can refer to the same storage, making the operation memory-efficient.

Part 2 - Mathematical form

If $\mathcal{A}$ has a dimension of size 1, that dimension may expand to size r. For $\mathcal{A} \in \mathbb{R}^{1\times n}$:

$$\mathcal{B} = \text{expand}(\mathcal{A}; m, n) \in \mathbb{R}^{m\times n}$$

$$B_{ij} = A_{1j}$$

Only singleton dimensions may grow; existing non-singleton sizes must remain unchanged.

Part 3 - Hints and Helper

Use `Tensor.expand()` in PyTorch. A size of `-1` preserves an existing dimension. Expansion creates a view with stride zero along expanded axes, so avoid unsafe in-place writes to the expanded result.

Part 4 - Python code skeleton

```
import torch

def E04(A, sizes):
    B = A.expand(...)
    return B

def main():
    A = torch...
    print(E04(...))

if __name__ == "__main__":
    main()
```

1.6.6 Exercise 05 - Tensor Repeat Law

Part 1 - Why this matters

Repeat creates explicit copies of a tensor pattern along chosen dimensions. It is useful when independent stored copies are required, when constructing grids, or when producing repeated training examples. Unlike expand, repeat allocates a larger tensor and can consume substantially more memory.

Part 2 - Mathematical form

For $\mathcal{A}$ with shape $(d_1, \ldots, d_k)$ and repetition factors $(r_1, \ldots, r_k)$:

$$\mathcal{B} = \text{repeat}(\mathcal{A}; r_1, \ldots, r_k)$$

$$\text{shape}(\mathcal{B}) = (r_1 d_1, \ldots, r_k d_k)$$

The values follow a periodic pattern along every repeated axis.

Part 3 - Hints and Helper

Use `Tensor.repeat()` in PyTorch with one factor per dimension. Repeat copies data, whereas `expand()` creates a broadcast-like view. Verify the memory and shape consequences before repeating large tensors.

Part 4 - Python code skeleton

```
import torch

def E05(A, repeats):
    B = A.repeat(...)
    return B

def main():
    A = torch...
    print(E05(...))

if __name__ == "__main__":
    main()
```

1.6.7 Exercise 06 - Tensor Flip Law

Part 1 - Why this matters

Flipping reverses the order of values along selected dimensions. It supports image augmentation, reversed sequences, symmetric processing, and orientation changes while preserving tensor shape. Multiple axes may be reversed in one operation.

Part 2 - Mathematical form

Let $\mathcal{A}$ have shape $(d_0, \ldots, d_{k-1})$. Flipping axis q gives:

$$\mathcal{B} = \text{flip}(\mathcal{A}; q)$$

$$B_{i_0 \ldots i_q \ldots i_{k-1}} = A_{i_0 \ldots (d_q - 1 - i_q) \ldots i_{k-1}}$$

using zero-based indices. The output shape is unchanged.

Part 3 - Hints and Helper

Use `torch.flip()` with a tuple or list of dimensions. Dimension order does not change; only the values along the selected axes are reversed. Confirm which axis represents width, height, sequence, or batch before flipping.

Part 4 - Python code skeleton

```
import torch

def E06(A, dims):
    B = torch.flip(A, dims=...)
    return B

def main():
    A = torch...
    print(E06(...))

if __name__ == "__main__":
    main()
```

1.6.8 Exercise 07 - Tensor Rotate Law

Part 1 - Why this matters

Tensor rotation turns data by multiples of 90° within a plane defined by two dimensions. It is widely used for image and feature-map augmentation, orientation normalization, and testing rotational behavior. Dimensions outside the selected plane remain unchanged.

Part 2 - Mathematical form

Let $\mathcal{A}$ contain a plane along dimensions p and q. Define:

$$\mathcal{B} = \mathrm{rot90}(\mathcal{A}; k, p, q)$$

where $k \in \mathbb{Z}$ counts counterclockwise quarter-turns. Rotations satisfy:

$$\mathrm{rot90}(\mathcal{A}; k+4, p, q) = \mathrm{rot90}(\mathcal{A}; k, p, q)$$

For odd k, the sizes of the selected plane dimensions are exchanged.

Part 3 - Hints and Helper

Use `torch.rot90()` with `k` and a pair of dimensions. Positive `k` rotates from the first selected axis toward the second. Verify both orientation and output shape, especially for non-square planes.

Part 4 - Python code skeleton

```
import torch

def E07(A, k, dims):
    B = torch.rot90(A, k=..., dims=...)
    return B

def main():
    A = torch...
    print(E07(...))

if __name__ == "__main__":
    main()
```

1.6.9 Exercise 08 - Tensor Squeeze Law

Part 1 - Why this matters

Squeeze removes dimensions whose size is one. Such singleton axes commonly appear after batching, indexing, reductions with preserved dimensions, or model outputs. Removing them can align shapes with downstream functions while leaving all stored values unchanged.

Part 2 - Mathematical form

If $\mathcal{A}$ has shape $(d_1, 1, d_3)$, squeezing the second axis gives:

$$\mathcal{B} = \text{squeeze}(\mathcal{A}; 2) \in \mathbb{R}^{d_1 \times d_3}$$

Only axes of size 1 may be removed. The total number and linear order of elements remain unchanged.

Part 3 - Hints and Helper

Use `torch.squeeze()`. Supplying `dim` removes only the intended singleton axis; omitting it removes every singleton dimension. Avoid accidentally removing a batch dimension when its current size happens to be one.

Part 4 - Python code skeleton

```
import torch

def E08(A, dim):
    B = torch.squeeze(A, dim=...)
    return B

def main():
    A = torch...
    print(E08(...))

if __name__ == "__main__":
    main()
```

1.6.10 Exercise 09 - Tensor Unsqueeze Law

Part 1 - Why this matters

Unsqueeze inserts a dimension of size one. It prepares vectors for broadcasting, adds batch or channel axes, and aligns tensors with functions that expect a specific number of dimensions. The operation changes shape but not the number or order of stored values.

Part 2 - Mathematical form

For $\mathbf{x} \in \mathbb{R}^n$, inserting a leading axis gives:

$$\mathbf{X} = \text{unsqueeze}(\mathbf{x}; 0) \in \mathbb{R}^{1 \times n}$$

Inserting a trailing axis gives a different shape:

$$\text{unsqueeze}(\mathbf{x}; 1) \in \mathbb{R}^{n \times 1}$$

Part 3 - Hints and Helper

Use `torch.unsqueeze()` or `Tensor.unsqueeze()`. The new axis may be inserted at any valid position from the beginning through the end. Choose its location according to the intended broadcasting semantics.

Part 4 - Python code skeleton

```
import torch

def E09(A, dim):
    B = torch.unsqueeze(A, dim=...)
    return B

def main():
    A = torch...
    print(E09(...))

if __name__ == "__main__":
    main()
```

1.6.11 Exercise 10 - Tensor Flatten Law

Part 1 - Why this matters

Flatten merges a consecutive range of dimensions into one. Convolutional features are often flattened before a linear layer, and groups of structural axes may be combined for storage or processing. Dimensions outside the selected range remain unchanged.

Part 2 - Mathematical form

Let $\mathcal{A}$ have shape $(d_0, \ldots, d_{k-1})$. Flattening dimensions s through t gives an output dimension of size:

$$D = \prod_{i=s}^{t} d_i$$

and shape:

$$(d_0, \ldots, d_{s-1}, D, d_{t+1}, \ldots, d_{k-1})$$

The total number and linear order of elements are preserved.

Part 3 - Hints and Helper

Use `torch.flatten()` with `start_dim` and `end_dim`. The default flattens every dimension. To preserve the batch axis, a common choice is `start_dim=1`.

Part 4 - Python code skeleton

```
import torch

def E10(A, start_dim, end_dim):
    B = torch.flatten(A, start_dim=..., end_dim=...)
    return B

def main():
    A = torch...
    print(E10(...))

if __name__ == "__main__":
    main()
```

1.6.12 Exercise 11 - Tensor Unflatten Law

Part 1 - Why this matters

Unflatten decomposes one dimension into several named or positional dimensions. It restores spatial structure, separates attention heads, and reconstructs multi-axis representations after a previous flattening operation. The product of the new sizes must equal the size of the original dimension.

Part 2 - Mathematical form

Let dimension q of $\mathcal{A}$ have size D, and choose $(e_1, \ldots, e_r)$ such that:

$$D = \prod_{j=1}^{r} e_j$$

Then:

$$\mathcal{B} = \text{unflatten}(\mathcal{A}; q, e_1, \ldots, e_r)$$

replaces dimension D by $(e_1, \ldots, e_r)$ while preserving all values and their linear order.

Part 3 - Hints and Helper

Use `torch.unflatten()` or `Tensor.unflatten()`. Specify the dimension to split and the new sizes. One size may be inferred with `-1` when the remaining sizes determine it uniquely.

Part 4 - Python code skeleton

```
import torch

def E11(A, dim, sizes):
    B = torch.unflatten(A, dim=..., sizes=...)
    return B

def main():
    A = torch...
    print(E11(...))

if __name__ == "__main__":
    main()
```

1.7 T05 - Differential Calculus Laws

Mathematics and Theory

1.7.1 Exercise 00 - Gradient Law

Part 1 - Why this matters

Learning adjusts parameters in the direction that changes a scalar objective. The gradient collects every first-order partial derivative, identifying the direction of steepest increase; its negative guides gradient descent.

Part 2 - Mathematical form

For differentiable $f : \mathbb{R}^n \to \mathbb{R}$:

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} & \cdots & \frac{\partial f}{\partial x_n} \end{bmatrix}^{\mathsf{T}} \in \mathbb{R}^n$$

For tensor input $\mathcal{X}$, $\nabla_{\mathcal{X}} f$ has the same shape as $\mathcal{X}$.

Part 3 - Hints and Helper

In PyTorch, set `requires_grad=True`, build a scalar output, and use `torch.autograd.grad()` or `backward()`. Gradients accumulate in `.grad`, so clear them between optimization steps.

Part 4 - Python code skeleton

```
import torch

def E00(x):
    y = ...
    gradient = torch.autograd.grad(y, x)[0]
    return gradient

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E00(x))

if __name__ == "__main__":
    main()
```

1.7.2 Exercise 01 - Jacobian Matrix Law

Part 1 - Why this matters

The Jacobian records how every component of a vector output responds to every input component. It describes local sensitivity, coordinate transformations, and the derivative propagated through vector-valued neural-network layers.

Part 2 - Mathematical form

For $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^m$:

$$\mathbf{J}_\mathbf{f}(\mathbf{x}) = \left[\frac{\partial f_i}{\partial x_j}\right] \in \mathbb{R}^{m \times n}$$

Its ith row is the transpose of ∇f_i.

Part 3 - Hints and Helper

Use `torch.autograd.functional.jacobian()` or `torch.func.jacrev()` in PyTorch. Check whether the function returns one tensor or a tuple, because that affects the Jacobian structure.

Part 4 - Python code skeleton

```
import torch

def E01(function, x):
    J = torch.autograd.functional.jacobian(...)
    return J

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E01(...))

if __name__ == "__main__":
    main()
```

1.7.3 Exercise 02 - Hessian Matrix Law

Part 1 - Why this matters

The Hessian measures how a gradient changes, revealing local curvature of a scalar objective. It supports second-order optimization, stability analysis, and identifying flat or sharply curved directions in loss landscapes.

Part 2 - Mathematical form

For twice-differentiable $f : \mathbb{R}^n \to \mathbb{R}$:

$$\mathbf{H}_f(\mathbf{x}) = \left[\frac{\partial^2 f}{\partial x_i \partial x_j}\right] \in \mathbb{R}^{n \times n}$$

If mixed partials are continuous, $\mathbf{H}_f = \mathbf{H}_f^\mathsf{T}$.

Part 3 - Hints and Helper

Use `torch.autograd.functional.hessian()` or `torch.func.hessian()`. The target function must return a scalar. Explicit Hessians can be expensive for large parameter vectors.

Part 4 - Python code skeleton

```
import torch

def E02(function, x):
    H = torch.autograd.functional.hessian(...)
    return H

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E02(...))

if __name__ == "__main__":
    main()
```

1.7.4 Exercise 03 - Divergence Law

Part 1 - Why this matters

Divergence measures the local net expansion of a vector field. It appears in continuous normalizing flows, physics-informed neural networks, conservation laws, and models that learn dynamical systems.

Part 2 - Mathematical form

For differentiable $\mathbf{F} : \mathbb{R}^n \to \mathbb{R}^n$:

$$\nabla \cdot \mathbf{F} = \sum_{i=1}^{n} \frac{\partial F_i}{\partial x_i} = \operatorname{tr}(\mathbf{J}_{\mathbf{F}}) \in \mathbb{R}$$

Positive values indicate local source-like expansion and negative values local sink-like contraction.

Part 3 - Hints and Helper

Compute the Jacobian with PyTorch autograd and sum its diagonal. Preserve the computational graph with `create_graph=True` if later derivatives are required.

Part 4 - Python code skeleton

```
import torch

def E03(function, x):
    J = torch.autograd.functional.jacobian(...)
    divergence = torch...
    return divergence

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E03(...))

if __name__ == "__main__":
    main()
```

1.7.5 Exercise 04 - Curl Law

Part 1 - Why this matters

Curl measures local rotational tendency in a three-dimensional vector field. It is important in learned fluid dynamics, electromagnetic models, and physics-informed systems where rotational structure must be represented.

Part 2 - Mathematical form

For $\mathbf{F} = (F_1, F_2, F_3) : \mathbb{R}^3 \to \mathbb{R}^3$:

$$\nabla \times \mathbf{F} = \begin{bmatrix} \frac{\partial F_3}{\partial x_2} - \frac{\partial F_2}{\partial x_3} \\ \frac{\partial F_1}{\partial x_3} - \frac{\partial F_3}{\partial x_1} \\ \frac{\partial F_2}{\partial x_1} - \frac{\partial F_1}{\partial x_2} \end{bmatrix}$$

Part 3 - Hints and Helper

PyTorch has no general high-level curl function. Construct the Jacobian with autograd, then combine its off-diagonal entries with the correct signs and order.

Part 4 - Python code skeleton

```
import torch

def E04(function, x):
    J = torch.autograd.functional.jacobian(...)
    curl = torch.stack((..., ..., ...))
    return curl

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E04(...))

if __name__ == "__main__":
    main()
```

1.7.6 Exercise 05 - Laplacian Law

Part 1 - Why this matters

The Laplacian summarizes second-order spatial variation. It appears in diffusion, heat equations, smoothing, regularization, and physics-informed neural networks that enforce partial differential equations.

Part 2 - Mathematical form

For twice-differentiable $f : \mathbb{R}^n \to \mathbb{R}$:

$$\Delta f = \nabla^2 f = \sum_{i=1}^{n} \frac{\partial^2 f}{\partial x_i^2} = \operatorname{tr}(\mathbf{H}_f)$$

Part 3 - Hints and Helper

Compute the Hessian using PyTorch autograd and take its trace. For large problems, forming the full Hessian may be expensive; this exercise focuses on the mathematical construction.

Part 4 - Python code skeleton

```
import torch

def E05(function, x):
    H = torch.autograd.functional.hessian(...)
    laplacian = torch...
    return laplacian

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E05(...))

if __name__ == "__main__":
    main()
```

1.7.7 Exercise 06 - Partial Derivative Law

Part 1 - Why this matters

A partial derivative isolates how a multivariable function responds to one input while the others are held fixed. Neural-network gradients are assembled from these local sensitivities.

Part 2 - Mathematical form

For $f(x_1, \ldots, x_n)$, the partial derivative with respect to x_i is:

$$\frac{\partial f}{\partial x_i}(\mathbf{x}) = \lim_{h \to 0} \frac{f(\mathbf{x} + h\mathbf{e}_i) - f(\mathbf{x})}{h}$$

when the limit exists.

Part 3 - Hints and Helper

Use PyTorch autograd to compute the full gradient and select the component for the desired variable. Ensure the output is scalar or provide an appropriate gradient output.

Part 4 - Python code skeleton

```
import torch

def E06(function, x, index):
    y = function(x)
    gradient = torch.autograd.grad(...)[0]
    return gradient[...]
```

```
def main():
    x = torch.tensor(..., requires_grad=True)
    print(E06(...))

if __name__ == "__main__":
    main()
```

1.7.8 Exercise 07 - Chain Rule Law

Part 1 - Why this matters

The chain rule is the foundation of backpropagation. It propagates derivatives through compositions of layers, allowing a final loss to influence every earlier parameter.

Part 2 - Mathematical form

For $y = f(u)$ and $u = g(x)$:

$$\frac{dy}{dx} = \frac{df}{du}\frac{dg}{dx}$$

For vector functions $\mathbf{f} \circ \mathbf{g}$:

$$\mathbf{J}_{\mathbf{f}\circ\mathbf{g}}(\mathbf{x}) = \mathbf{J}_{\mathbf{f}}(\mathbf{g}(\mathbf{x}))\mathbf{J}_{\mathbf{g}}(\mathbf{x})$$

Part 3 - Hints and Helper

PyTorch autograd applies the chain rule automatically. Build the composed expression from tracked tensors and differentiate the final scalar output with respect to the original input.

Part 4 - Python code skeleton

```
import torch

def E07(x):
    u = ...
    y = ...
    derivative = torch.autograd.grad(...)[0]
    return derivative

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E07(x))

if __name__ == "__main__":
    main()
```

1.7.9 Exercise 08 - Product Rule Law

Part 1 - Why this matters

The product rule differentiates multiplicative interactions, including gates, attention weights, regularizers, and products between learned functions. Both factors contribute to the final

derivative.

Part 2 - Mathematical form

For differentiable scalar functions $u(x)$ and $v(x)$:

$$\frac{d}{dx}[u(x)v(x)] = u'(x)v(x) + u(x)v'(x)$$

For scalar fields, $\nabla(uv) = v\nabla u + u\nabla v$.

Part 3 - Hints and Helper

Construct both differentiable factors in PyTorch, multiply them, and use autograd. To verify the law, compute the two right-hand contributions separately without detaching either factor.

Part 4 - Python code skeleton

```
import torch

def E08(x):
    u = ...
    v = ...
    y = u * v
    derivative = torch.autograd.grad(...)[0]
    return derivative

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E08(x))

if __name__ == "__main__":
    main()
```

1.7.10 Exercise 09 - Quotient Rule Law

Part 1 - Why this matters

The quotient rule differentiates ratios, which occur in normalization, probability formulas, similarity measures, and scale-invariant objectives. The denominator must remain nonzero.

Part 2 - Mathematical form

For differentiable $u(x)$ and $v(x) \neq 0$:

$$\frac{d}{dx}\left[\frac{u(x)}{v(x)}\right] = \frac{u'(x)v(x) - u(x)v'(x)}{v(x)^2}$$

Part 3 - Hints and Helper

Use ordinary differentiable division in PyTorch and autograd. Choose test values away from a zero denominator to avoid undefined derivatives and numerical instability.

Part 4 - Python code skeleton

```
import torch

def E09(x):
    u = ...
    v = ...
    y = u / v
    derivative = torch.autograd.grad(...)[0]
    return derivative

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E09(x))

if __name__ == "__main__":
    main()
```

1.7.11 Exercise 10 - Power Rule Law

Part 1 - Why this matters

The power rule differentiates polynomial terms used in losses, regularization, and nonlinear transformations. It is one of the basic laws from which more complex derivatives are assembled.

Part 2 - Mathematical form

For constant p and values where x^p is differentiable:

$$\frac{d}{dx}x^p = px^{p-1}$$

For a differentiable inner function $u(x)$:

$$\frac{d}{dx}u(x)^p = pu(x)^{p-1}u'(x)$$

Part 3 - Hints and Helper

Use `torch.pow()` or `**` and differentiate with autograd. Fractional or negative powers impose domain restrictions and may be undefined at some inputs.

Part 4 - Python code skeleton

```
import torch

def E10(x, p):
    y = torch...
```

```
    derivative = torch.autograd.grad(...)[0]
    return derivative

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E10(...))

if __name__ == "__main__":
    main()
```

1.7.12 Exercise 11 - Exponential Derivative Law

Part 1 - Why this matters

Exponential functions appear in probabilities, softmax, energy-based models, and continuous growth or decay. Their derivative preserves the exponential form, making them central to differentiable modeling.

Part 2 - Mathematical form

$$\frac{d}{dx}e^x = e^x$$

More generally, for differentiable $u(x)$:

$$\frac{d}{dx}e^{u(x)} = e^{u(x)}u'(x)$$

Part 3 - Hints and Helper

Use `torch.exp()` and autograd. Large positive inputs may overflow and large negative inputs may underflow, so stable formulations are important in probability computations.

Part 4 - Python code skeleton

```
import torch

def E11(x):
    y = torch...
    derivative = torch.autograd.grad(...)[0]
    return derivative

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E11(x))

if __name__ == "__main__":
    main()
```

1.7.13 Exercise 12 - Logarithmic Derivative Law

Part 1 - Why this matters

Logarithms turn products into sums and appear in likelihoods, cross-entropy, information theory, and numerically stable probability calculations. Their derivative grows near zero, revealing an important stability boundary.

Part 2 - Mathematical form

For $x > 0$:

$$\frac{d}{dx} \ln x = \frac{1}{x}$$

For differentiable $u(x) > 0$:

$$\frac{d}{dx} \ln u(x) = \frac{u'(x)}{u(x)}$$

Part 3 - Hints and Helper

Use `torch.log()` and autograd. Inputs must be positive for real-valued logarithms. Values close to zero can produce very large gradients.

Part 4 - Python code skeleton

```
import torch

def E12(x):
    y = torch...
    derivative = torch.autograd.grad(...)[0]
    return derivative

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E12(x))

if __name__ == "__main__":
    main()
```

1.7.14 Exercise 13 - Sigmoid Derivative Law

Part 1 - Why this matters

Sigmoid maps real values into $(0, 1)$ and is used for binary probabilities and gates. Its derivative can be written using its own output, which makes local gradient computation compact.

Part 2 - Mathematical form

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$\sigma'(x) = \sigma(x)[1 - \sigma(x)]$$

The derivative is positive and at most 1/4; it approaches zero for large positive or negative inputs.

Part 3 - Hints and Helper

Use `torch.sigmoid()` and autograd. Compare the automatic derivative with the analytical expression computed from the already obtained sigmoid output.

Part 4 - Python code skeleton

```
import torch

def E13(x):
    y = torch...
    derivative = torch.autograd.grad(...)[0]
    return derivative

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E13(x))

if __name__ == "__main__":
    main()
```

1.7.15 Exercise 14 - Tanh Derivative Law

Part 1 - Why this matters

Hyperbolic tangent maps values into $(-1, 1)$ and provides zero-centered activations in recurrent and feed-forward networks. Its derivative also depends directly on its output.

Part 2 - Mathematical form

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$\frac{d}{dx}\tanh(x) = 1 - \tanh^2(x)$$

The derivative is largest at zero and approaches zero as $|x|$ grows.

Part 3 - Hints and Helper

Use `torch.tanh()` and autograd. Compare the result with $1 - y^2$, where y is the tanh output.

Part 4 - Python code skeleton

```
import torch

def E14(x):
    y = torch...
    derivative = torch.autograd.grad(...)[0]
    return derivative

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E14(x))

if __name__ == "__main__":
    main()
```

1.7.16 Exercise 15 - ReLU Derivative Law

Part 1 - Why this matters

ReLU passes positive activations and suppresses negative ones. Its simple piecewise derivative supports efficient training, though negative inputs receive zero gradient.

Part 2 - Mathematical form

$$\mathrm{ReLU}(x) = \max(0, x)$$

$$\mathrm{ReLU}'(x) = \begin{cases} 0, & x < 0, \\ 1, & x > 0. \end{cases}$$

The classical derivative is undefined at $x = 0$; PyTorch uses the subgradient value 0 there.

Part 3 - Hints and Helper

Use `torch.relu()` or `torch.nn.functional.relu()` and autograd. Include negative, zero, and positive test values to observe the piecewise rule.

Part 4 - Python code skeleton

```
import torch

def E15(x):
    y = torch...
    derivative = torch.autograd.grad(y.sum(), x)[0]
    return derivative

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E15(x))

if __name__ == "__main__":
    main()
```

1.7.17 Exercise 16 - LeakyReLU Derivative Law

Part 1 - Why this matters

LeakyReLU retains a small nonzero slope for negative inputs, allowing gradients to continue flowing where ordinary ReLU would block them. The negative slope is a chosen constant.

Part 2 - Mathematical form

For $\alpha > 0$:

$$\mathrm{LReLU}_\alpha(x) = \begin{cases} x, & x \geq 0, \\ \alpha x, & x < 0, \end{cases}$$

$$\mathrm{LReLU}'_\alpha(x) = \begin{cases} 1, & x > 0, \\ \alpha, & x < 0. \end{cases}$$

At $x = 0$ the classical derivative is undefined when $\alpha \neq 1$; PyTorch uses the negative-slope branch value α.

Part 3 - Hints and Helper

Use `torch.nn.functional.leaky_relu()` with an explicit `negative_slope`. Sum vector outputs before calling autograd.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E16(x, alpha):
    y = F.leaky_relu(x, negative_slope=...)
    derivative = torch.autograd.grad(y.sum(), x)[0]
    return derivative

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E16(...))

if __name__ == "__main__":
    main()
```

1.7.18 Exercise 17 - Softmax Derivative Law

Part 1 - Why this matters

Softmax converts logits into a normalized probability vector. Because every output depends on every input, its derivative is a full Jacobian rather than an element-wise scalar derivative. This coupling is central to classification and attention.

Part 2 - Mathematical form

For $\mathbf{z} \in \mathbb{R}^n$:

$$s_i = \frac{e^{z_i}}{\sum_{k=1}^{n} e^{z_k}}$$

$$\frac{\partial s_i}{\partial z_j} = s_i(\delta_{ij} - s_j)$$

Equivalently:

$$\mathbf{J} = \operatorname{diag}(\mathbf{s}) - \mathbf{s}\mathbf{s}^{\mathsf{T}}$$

Part 3 - Hints and Helper

Use `torch.softmax()` with an explicit dimension, then compute the Jacobian with `torch.autograd.fu`
Do not treat the derivative as only $s_i(1 - s_i)$; off-diagonal terms are nonzero.

Part 4 - Python code skeleton

```
import torch

def E17(z):
    function = lambda value: torch.softmax(value, dim=...)
    J = torch.autograd.functional.jacobian(...)
    return J

def main():
    z = torch.tensor(..., requires_grad=True)
    print(E17(z))

if __name__ == "__main__":
    main()
```

1.7.19 Exercise 18 - Loss Gradient Law

Part 1 - Why this matters

The loss gradient connects prediction error to parameter updates. Backpropagation uses the chain rule to compute how a scalar loss changes with every model parameter, enabling optimization across the complete network.

Part 2 - Mathematical form

Let predictions be $\hat{\mathbf{y}} = f(\mathbf{x}; \boldsymbol{\theta})$ and let $L(\hat{\mathbf{y}}, \mathbf{y}) \in \mathbb{R}$. Then:

$$\nabla_{\boldsymbol{\theta}} L = \mathbf{J}_{f,\boldsymbol{\theta}}^{\mathsf{T}} \nabla_{\hat{\mathbf{y}}} L$$

A gradient-descent update is:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \nabla_{\boldsymbol{\theta}} L$$

Part 3 - Hints and Helper

Build predictions and a scalar loss from parameters requiring gradients. Call `loss.backward()` or `torch.autograd.grad()`. Do not detach the prediction, and clear accumulated parameter gradients before a new step.

Part 4 - Python code skeleton

```
import torch

def E18(parameters, x, target):
    prediction = ...
    loss = ...
    gradients = torch.autograd.grad(loss, parameters)
    return loss, gradients

def main():
    parameters = (...)
    x = torch...
    target = torch...
    print(E18(...))

if __name__ == "__main__":
    main()
```

1.8 T06 - Optimization Laws

Mathematics and Theory

1.8.1 Exercise 00 - Gradient Descent Law

Part 1 - Why this matters

Gradient descent is the basic mechanism that turns a loss gradient into a parameter update. It repeatedly moves parameters opposite the direction of steepest local increase, seeking values that reduce the objective.

Part 2 - Mathematical form

For loss $L(\boldsymbol{\theta})$ and learning rate $\eta > 0$:

$$\mathbf{g}_t = \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}_t)$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \mathbf{g}_t$$

The learning rate controls the update magnitude.

Part 3 - Hints and Helper

In PyTorch, compute a scalar loss, call `backward()`, and update parameters inside `torch.no_grad()`. Clear gradients before the next iteration. `torch.optim.SGD()` with zero momentum implements this rule.

Part 4 - Python code skeleton

```
import torch

def E00(parameter, loss, learning_rate):
```

```
    loss.backward()
    with torch.no_grad():
        parameter -= ...
    parameter.grad = None
    return parameter

def main():
    parameter = torch.tensor(..., requires_grad=True)
    loss = ...
    print(E00(...))

if __name__ == "__main__":
    main()
```

1.8.2 Exercise 01 - Stochastic Gradient Descent Law

Part 1 - Why this matters

Stochastic gradient descent estimates the full-data gradient from one example or a mini-batch. Its updates are cheaper and noisier than full-batch gradient descent, enabling training on large datasets and often helping exploration of the loss landscape.

Part 2 - Mathematical form

For mini-batch $\mathcal{B}_t$ of size B:

$$\widehat{\mathbf{g}}_t = \frac{1}{B} \sum_{i \in \mathcal{B}_t} \nabla_{\boldsymbol{\theta}} \ell_i(\boldsymbol{\theta}_t)$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \widehat{\mathbf{g}}_t$$

Under uniform sampling, this is an unbiased full-gradient estimate.

Part 3 - Hints and Helper

Use `torch.optim.SGD()` and compute loss on one mini-batch per step. Call `zero_grad()`, `backward()`, then `step()` in that order.

Part 4 - Python code skeleton

```
import torch

def E01(model, batch_x, batch_y, optimizer, loss_function):
    optimizer.zero_grad()
    prediction = model(...)
    loss = loss_function(...)
    loss.backward()
    optimizer.step()
    return loss

def main():
    model = ...
```

```
    optimizer = torch.optim.SGD(...)
    print(E01(...))

if __name__ == "__main__":
    main()
```

1.8.3 Exercise 02 - Momentum Law

Part 1 - Why this matters

Momentum accumulates a velocity from previous gradients. It accelerates motion along consistent directions and reduces oscillation across steep directions, often converging faster than plain SGD.

Part 2 - Mathematical form

With momentum coefficient $0 \leq \mu < 1$ and $\mathbf{v}_0 = \mathbf{0}$:

$$\mathbf{g}_t = \nabla L(\boldsymbol{\theta}_t)$$

$$\mathbf{v}_t = \mu \mathbf{v}_{t-1} + \mathbf{g}_t$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \mathbf{v}_t$$

Part 3 - Hints and Helper

Use `torch.optim.SGD()` with a positive `momentum`. The velocity is persistent optimizer state and must not be reinitialized at every step.

Part 4 - Python code skeleton

```
import torch

def E02(parameter, gradient, velocity, learning_rate, momentum):
    velocity = ...
    parameter = ...
    return parameter, velocity

def main():
    parameter = torch...
    gradient = torch...
    velocity = torch.zeros_like(parameter)
    print(E02(...))

if __name__ == "__main__":
    main()
```

1.8.4 Exercise 03 - RMSProp Law

Part 1 - Why this matters

RMSProp adapts each parameter's step size using an exponential moving average of squared gradients. Coordinates with consistently large gradients receive smaller normalized updates, improving behavior on objectives with uneven scales.

Part 2 - Mathematical form

With decay $0 \leq \alpha < 1$, $\mathbf{s}_0 = \mathbf{0}$, and $\varepsilon > 0$:

$$\mathbf{s}_t = \alpha \mathbf{s}_{t-1} + (1 - \alpha)(\mathbf{g}_t \odot \mathbf{g}_t)$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \frac{\mathbf{g}_t}{\sqrt{\mathbf{s}_t} + \varepsilon}$$

All vector operations are element-wise.

Part 3 - Hints and Helper

Use `torch.optim.RMSprop()`. Preserve the running squared-gradient state between updates, and keep ε to prevent division by zero.

Part 4 - Python code skeleton

```
import torch

def E03(parameter, gradient, square_avg, lr, alpha, eps):
    square_avg = ...
    parameter = ...
    return parameter, square_avg

def main():
    parameter = torch...
    gradient = torch...
    square_avg = torch.zeros_like(parameter)
    print(E03(...))

if __name__ == "__main__":
    main()
```

1.8.5 Exercise 04 - Adam Optimizer Law

Part 1 - Why this matters

Adam combines momentum-like first moments with RMSProp-like second moments. It adapts updates per parameter and uses bias correction to compensate for moment estimates initialized at zero.

Part 2 - Mathematical form

For step $t \geq 1$:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1)\mathbf{g}_t$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2)\mathbf{g}_t^2$$

$$\widehat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \qquad \widehat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \frac{\widehat{\mathbf{m}}_t}{\sqrt{\widehat{\mathbf{v}}_t} + \varepsilon}$$

Part 3 - Hints and Helper

Use `torch.optim.Adam()`. Preserve both moment tensors and the step counter. Squaring, division, and square roots are element-wise.

Part 4 - Python code skeleton

```
import torch

def E04(parameter, gradient, m, v, step, lr, betas, eps):
    m = ...
    v = ...
    m_hat = ...
    v_hat = ...
    parameter = ...
    return parameter, m, v

def main():
    parameter = torch...
    print(E04(...))

if __name__ == "__main__":
    main()
```

1.8.6 Exercise 05 - AdamW Law

Part 1 - Why this matters

AdamW separates weight decay from Adam's adaptive gradient normalization. This decoupling makes shrinkage act directly on parameters instead of being rescaled as part of the adaptive gradient, yielding clearer regularization behavior.

Part 2 - Mathematical form

After computing Adam's bias-corrected moments, AdamW applies:

$$\boldsymbol{\theta}_{t+1} = (1 - \eta\lambda)\boldsymbol{\theta}_t - \eta \frac{\widehat{\mathbf{m}}_t}{\sqrt{\widehat{\mathbf{v}}_t} + \varepsilon}$$

where $\lambda \geq 0$ is the decoupled weight-decay coefficient.

Part 3 - Hints and Helper

Use `torch.optim.AdamW()`. Do not also add the same L_2 penalty to the loss unless intentionally applying two regularization effects. Bias and normalization parameters are often placed in groups with no decay.

Part 4 - Python code skeleton

```
import torch

def E05(model, loss):
    optimizer = torch.optim.AdamW(
        model.parameters(), lr=..., weight_decay=...
    )
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    return model

def main():
    model = ...
    loss = ...
    print(E05(...))

if __name__ == "__main__":
    main()
```

1.8.7 Exercise 06 - Nesterov Momentum Law

Part 1 - Why this matters

Nesterov momentum evaluates the gradient at a look-ahead position suggested by the existing velocity. This anticipatory correction can reduce overshooting and respond sooner when the objective changes direction.

Part 2 - Mathematical form

With $\mathbf{v}_0 = \mathbf{0}$:

$$\widetilde{\boldsymbol{\theta}}_t = \boldsymbol{\theta}_t - \eta\mu\mathbf{v}_{t-1}$$

$$\mathbf{v}_t = \mu\mathbf{v}_{t-1} + \nabla L(\widetilde{\boldsymbol{\theta}}_t)$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta\mathbf{v}_t$$

Part 3 - Hints and Helper

Use `torch.optim.SGD()` with `momentum` and `nesterov=True`. PyTorch uses an algebraically related implementation; do not reset its momentum buffer between steps.

Part 4 - Python code skeleton

```
import torch

def E06(model, learning_rate, momentum):
    optimizer = torch.optim.SGD(
        model.parameters(), lr=...,
        momentum=..., nesterov=True
    )
    return optimizer

def main():
    model = ...
    print(E06(...))

if __name__ == "__main__":
    main()
```

1.8.8 Exercise 07 - Learning Rate Decay Law

Part 1 - Why this matters

Learning-rate decay uses larger exploratory steps early in training and smaller refinement steps later. A schedule can improve stability near a solution and reduce persistent oscillation.

Part 2 - Mathematical form

An exponential schedule is:

$$\eta_t = \eta_0 \gamma^t, \qquad 0 < \gamma < 1$$

A step schedule updated every s epochs is:

$$\eta_t = \eta_0 \gamma^{\lfloor t/s \rfloor}$$

The optimizer update then uses η_t in place of a constant learning rate.

Part 3 - Hints and Helper

PyTorch provides schedulers such as `ExponentialLR` and `StepLR`. Call scheduler steps at the intended frequency and monitor the current optimizer learning rate to avoid accidental extra decay.

Part 4 - Python code skeleton

```
import torch

def E07(optimizer, gamma):
    scheduler = torch.optim.lr_scheduler.ExponentialLR(
        optimizer, gamma=...
    )
    return scheduler
```

```
def main():
    optimizer = ...
    scheduler = E07(...)
    scheduler.step()
    print(optimizer.param_groups[0]["lr"])

if __name__ == "__main__":
    main()
```

1.8.9 Exercise 08 - Weight Decay Law

Part 1 - Why this matters

Weight decay discourages large parameter magnitudes and can improve generalization. In its classical coupled form, it is equivalent to adding an L_2 penalty to the objective for ordinary gradient descent.

Part 2 - Mathematical form

Define the regularized loss:

$$L_{\text{reg}}(\boldsymbol{\theta}) = L(\boldsymbol{\theta}) + \frac{\lambda}{2}\|\boldsymbol{\theta}\|_2^2$$

Then:

$$\nabla L_{\text{reg}} = \nabla L + \lambda\boldsymbol{\theta}$$

and gradient descent gives:

$$\boldsymbol{\theta}_{t+1} = (1 - \eta\lambda)\boldsymbol{\theta}_t - \eta\nabla L$$

Part 3 - Hints and Helper

Use the `weight_decay` option in an appropriate PyTorch optimizer. Classical coupled L_2 regularization is not generally equivalent to decoupled decay under adaptive optimizers; AdamW handles the latter explicitly.

Part 4 - Python code skeleton

```
import torch

def E08(model, learning_rate, decay):
    optimizer = torch.optim.SGD(
        model.parameters(), lr=..., weight_decay=...
    )
    return optimizer

def main():
    model = ...
    print(E08(...))

if __name__ == "__main__":
    main()
```

1.9 T07 - Statistics Probability Laws

Mathematics and Theory

1.9.1 Exercise 00 - Mean Law

Part 1 - Why this matters

The mean summarizes the central level of data and underlies centering, normalization, loss aggregation, and feature statistics in machine learning.

Part 2 - Mathematical form

For observations $x_1, \ldots, x_n$ with $n > 0$:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i$$

For a random variable X, the population mean is $\mu = \mathbb{E}[X]$ when the expectation exists.

Part 3 - Hints and Helper

Use `torch.mean()` with a floating-point or complex tensor. Specify `dim` when reducing selected axes and use `keepdim=True` when the result must remain broadcasting-compatible.

Part 4 - Python code skeleton

```
import torch

def E00(x):
    mean = torch...
    return mean

def main():
    x = torch...
    print(E00(x))

if __name__ == "__main__":
    main()
```

1.9.2 Exercise 01 - Median Law

Part 1 - Why this matters

The median identifies the middle of ordered data and is more resistant to extreme values than the mean. It supports robust statistics, preprocessing, and outlier-resistant summaries.

Part 2 - Mathematical form

Let $x_{(1)} \leq \cdots \leq x_{(n)}$ be sorted observations:

$$\text{median}(x) = \begin{cases} x_{((n+1)/2)}, & n \text{ odd}, \\ \frac{x_{(n/2)} + x_{(n/2+1)}}{2}, & n \text{ even}. \end{cases}$$

Part 3 - Hints and Helper

Use `torch.median()` carefully: for an even number of elements it returns the lower of the two middle values, not their average. Use `torch.quantile(x, 0.5)` for the interpolated statistical median.

Part 4 - Python code skeleton

```
import torch

def E01(x):
    median = torch...
    return median

def main():
    x = torch...
    print(E01(x))

if __name__ == "__main__":
    main()
```

1.9.3 Exercise 02 - Mode Law

Part 1 - Why this matters

The mode is the most frequent observed value. It is especially meaningful for categorical or discrete data, label analysis, majority decisions, and detecting dominant outcomes.

Part 2 - Mathematical form

For dataset $D = \{x_1, \ldots, x_n\}$, define frequency $c(a) = |\{i : x_i = a\}|$. Then:

$$\text{Mode}(D) = \arg\max_a c(a)$$

The mode may be non-unique when several values share the maximum frequency.

Part 3 - Hints and Helper

Use `torch.mode()` along a dimension. It returns both values and indices and selects one value when ties exist. Use `torch.unique(..., return_counts=True)` when every tied mode must be identified.

Part 4 - Python code skeleton

```
import torch

def E02(x):
    values, counts = torch.unique(...)
    modes = values[...]
    return modes

def main():
    x = torch...
    print(E02(x))

if __name__ == "__main__":
    main()
```

1.9.4 Exercise 03 - Variance Law

Part 1 - Why this matters

Variance measures dispersion around the mean. It is central to normalization, uncertainty estimation, initialization analysis, and statistical modeling.

Part 2 - Mathematical form

Population variance is:

$$\sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2$$

The unbiased sample variance for $n > 1$ is:

$$s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

Part 3 - Hints and Helper

Use `torch.var()`. Set `correction=0` for population variance and `correction=1` for the usual unbiased sample estimate. State which definition the exercise uses.

Part 4 - Python code skeleton

```
import torch

def E03(x, correction):
    variance = torch.var(x, correction=...)
    return variance

def main():
    x = torch...
    print(E03(...))

if __name__ == "__main__":
    main()
```

1.9.5 Exercise 04 - Standard Deviation Law

Part 1 - Why this matters

Standard deviation expresses spread in the same units as the original data. It supports standardization, uncertainty reporting, anomaly thresholds, and monitoring activation scale.

Part 2 - Mathematical form

Standard deviation is the nonnegative square root of variance:

$$\sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2}$$

For the unbiased sample-variance convention:

$$s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

Part 3 - Hints and Helper

Use `torch.std()` and select the desired `correction`. Standard deviation is not the unbiased estimator of population standard deviation merely because its underlying variance uses Bessel's correction.

Part 4 - Python code skeleton

```
import torch

def E04(x, correction):
    standard_deviation = torch.std(x, correction=...)
    return standard_deviation

def main():
    x = torch...
    print(E04(...))

if __name__ == "__main__":
    main()
```

1.9.6 Exercise 05 - Covariance Law

Part 1 - Why this matters

Covariance measures whether two variables vary together. Covariance matrices describe feature relationships and appear in PCA, Gaussian models, whitening, and uncertainty estimation.

Part 2 - Mathematical form

Population covariance is:

$$\mathrm{Cov}(X, Y) = \mathbb{E}[(X - \mu_X)(Y - \mu_Y)]$$

For paired samples, the unbiased estimate is:

$$s_{xy} = \frac{1}{n-1} \sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y})$$

Part 3 - Hints and Helper

Use `torch.cov()` with variables arranged in rows and observations in columns. Check tensor orientation and the correction convention before reading off diagonal covariances.

Part 4 - Python code skeleton

```
import torch

def E05(x, y):
    data = torch.stack((...))
    covariance = torch.cov(...)
    return covariance

def main():
    x = torch...
    y = torch...
    print(E05(...))

if __name__ == "__main__":
    main()
```

1.9.7 Exercise 06 - Correlation Law

Part 1 - Why this matters

Correlation normalizes covariance into a unitless measure of linear association. It supports feature analysis, redundancy detection, and interpreting relationships across variables with different scales.

Part 2 - Mathematical form

For nonzero standard deviations:

$$\rho_{XY} = \frac{\mathrm{Cov}(X, Y)}{\sigma_X \sigma_Y}$$

and $-1 \leq \rho_{XY} \leq 1$. Correlation is undefined if either variable has zero variance.

Part 3 - Hints and Helper

Use `torch.corrcoef()` with variables in rows. Correlation measures linear association and does not by itself establish causation or independence.

Part 4 - Python code skeleton

```
import torch

def E06(x, y):
    data = torch.stack((...))
    correlation = torch.corrcoef(...)
    return correlation

def main():
    x = torch...
    y = torch...
    print(E06(...))

if __name__ == "__main__":
    main()
```

1.9.8 Exercise 07 - Probability Density Function Law

Part 1 - Why this matters

A probability density function describes how probability is distributed over a continuous variable. Probabilistic models use densities for likelihoods, uncertainty, sampling, and expected values.

Part 2 - Mathematical form

A valid PDF f_X satisfies:

$$f_X(x) \geq 0, \qquad \int_{-\infty}^{\infty} f_X(x)\,dx = 1$$

and interval probability is:

$$\Pr(a \leq X \leq b) = \int_a^b f_X(x)\,dx$$

For a continuous variable, $\Pr(X = x) = 0$ even when $f_X(x) > 0$.

Part 3 - Hints and Helper

PyTorch distributions provide `log_prob()`, which returns log density for continuous distributions. Exponentiating gives density, not point probability.

Part 4 - Python code skeleton

```
import torch

def E07(distribution, x):
    log_density = distribution.log_prob(...)
    density = torch.exp(...)
    return density

def main():
```

```
    distribution = torch.distributions.Normal(...)
    x = torch...
    print(E07(...))

if __name__ == "__main__":
    main()
```

1.9.9 Exercise 08 - Normal Distribution Law

Part 1 - Why this matters

The normal distribution models continuous variation and noise and underlies Gaussian likelihoods, initialization schemes, latent-variable models, and many statistical approximations.

Part 2 - Mathematical form

For $X \sim \mathcal{N}(\mu, \sigma^2)$ with $\sigma > 0$:

$$f_X(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left[-\frac{(x-\mu)^2}{2\sigma^2}\right]$$

$$\mathbb{E}[X] = \mu, \qquad \mathrm{Var}(X) = \sigma^2$$

Part 3 - Hints and Helper

Use `torch.distributions.Normal(loc, scale)`. The second argument is standard deviation, not variance. Use `sample()` or `rsample()` when a reparameterized differentiable sample is required.

Part 4 - Python code skeleton

```
import torch

def E08(mean, standard_deviation, x):
    distribution = torch.distributions.Normal(...)
    density = torch.exp(distribution.log_prob(...))
    return density

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

1.9.10 Exercise 09 - Uniform Distribution Law

Part 1 - Why this matters

The uniform distribution assigns equal density across an interval. It is used for random initialization, bounded exploration, simulation, and constructing simple prior distributions.

Part 2 - Mathematical form

For $X \sim \mathcal{U}(a, b)$ with $a < b$:

$$f_X(x) = \begin{cases} \frac{1}{b-a}, & a \le x \le b, \\ 0, & \text{otherwise} \end{cases}$$

$$\mathbb{E}[X] = \frac{a+b}{2}, \qquad \mathrm{Var}(X) = \frac{(b-a)^2}{12}$$

Part 3 - Hints and Helper

Use `torch.distributions.Uniform(low, high)` or `torch.rand()` followed by affine scaling. Ensure every lower bound is strictly smaller than its corresponding upper bound.

Part 4 - Python code skeleton

```
import torch

def E09(low, high, x):
    distribution = torch.distributions.Uniform(...)
    density = torch.exp(distribution.log_prob(...))
    return density

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

1.9.11 Exercise 10 - Bernoulli Law

Part 1 - Why this matters

The Bernoulli distribution models one binary outcome. It supports binary classification, dropout masks, binary latent variables, and repeated-trial probability models.

Part 2 - Mathematical form

For $X \sim \mathrm{Bernoulli}(p)$ with $0 \le p \le 1$:

$$\Pr(X = x) = p^x (1-p)^{1-x}, \qquad x \in \{0, 1\}$$

$$\mathbb{E}[X] = p, \qquad \mathrm{Var}(X) = p(1-p)$$

Part 3 - Hints and Helper

Use `torch.distributions.Bernoulli()` with either probabilities or logits, but not both. Logits are often numerically convenient in neural-network models.

Part 4 - Python code skeleton

```
import torch

def E10(probability):
    distribution = torch.distributions.Bernoulli(probs=...)
    sample = distribution.sample()
    return sample

def main():
    print(E10(...))

if __name__ == "__main__":
    main()
```

1.9.12 Exercise 11 - Softmax Probability Law

Part 1 - Why this matters

Softmax converts arbitrary logits into a categorical probability vector. It is used for multi-class predictions and attention weights while preserving relative preference between alternatives.

Part 2 - Mathematical form

For logits $\mathbf{z} \in \mathbb{R}^K$:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

$$p_i > 0, \qquad \sum_{i=1}^{K} p_i = 1$$

Adding the same constant to all logits leaves the probabilities unchanged.

Part 3 - Hints and Helper

Use `torch.softmax(logits, dim=...)`. PyTorch applies a stable implementation; specify the class axis correctly. Use `log_softmax` when log probabilities are required.

Part 4 - Python code skeleton

```
import torch

def E11(logits, dim):
    probabilities = torch.softmax(...)
    return probabilities

def main():
    logits = torch...
    print(E11(...))

if __name__ == "__main__":
    main()
```

1.9.13 Exercise 12 - Entropy Law

Part 1 - Why this matters

Entropy measures uncertainty in a probability distribution. It appears in decision trees, reinforcement learning, information bottlenecks, uncertainty analysis, and regularization of predicted distributions.

Part 2 - Mathematical form

For a discrete distribution $\mathbf{p} = (p_1, \ldots, p_K)$:

$$H(\mathbf{p}) = -\sum_{i=1}^{K} p_i \log p_i$$

with $0 \log 0$ defined by continuity as 0. Natural logarithms measure entropy in nats; base-2 logarithms measure bits.

Part 3 - Hints and Helper

In PyTorch, compute entropy from probabilities and log probabilities, using `torch.special.xlogy()` or safe masking to handle zeros. Many `torch.distributions` objects also provide `entropy()`.

Part 4 - Python code skeleton

```
import torch

def E12(probabilities):
    entropy = -torch.sum(...)
    return entropy

def main():
    probabilities = torch...
    print(E12(probabilities))

if __name__ == "__main__":
    main()
```

1.9.14 Exercise 13 - KL Divergence Law

Part 1 - Why this matters

Kullback–Leibler divergence measures the information lost when distribution Q approximates distribution P. It is central to variational inference, knowledge distillation, distribution matching, and probabilistic regularization.

Part 2 - Mathematical form

For discrete distributions P and Q:

$$D_{\mathrm{KL}}(P \| Q) = \sum_i p_i \log \frac{p_i}{q_i}$$

assuming $q_i > 0$ whenever $p_i > 0$. It satisfies $D_{\mathrm{KL}}(P\|Q) \geq 0$ but is generally not symmetric and is not a distance metric.

Part 3 - Hints and Helper

Use `torch.nn.functional.kl_div()` carefully: by default its input is log probabilities and its target is probabilities. Choose an explicit reduction such as `batchmean` according to the intended definition.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E13(log_q, p):
    divergence = F.kl_div(..., reduction="...")
    return divergence

def main():
    log_q = torch...
    p = torch...
    print(E13(...))

if __name__ == "__main__":
    main()
```

1.9.15 Exercise 14 - Jensen Inequality Law

Part 1 - Why this matters

Jensen's inequality connects convex functions with expectations. It underlies many bounds in optimization, information theory, variational inference, and the analysis of losses and estimators.

Part 2 - Mathematical form

If φ is convex and X is integrable:

$$\varphi(\mathbb{E}[X]) \leq \mathbb{E}[\varphi(X)]$$

For weights $w_i \geq 0$ with $\sum_i w_i = 1$:

$$\varphi\left(\sum_i w_i x_i\right) \leq \sum_i w_i \varphi(x_i)$$

The inequality reverses for concave φ. Equality holds, among other cases, when all positively weighted inputs are equal.

Part 3 - Hints and Helper

Use PyTorch to compute both sides for a chosen convex function such as $\varphi(x) = x^2$. Verify that weights are nonnegative and normalized before comparing the two scalar results.

Part 4 - Python code skeleton

```
import torch

def E14(x, weights):
    left = (... ) ** 2
    right = torch.sum(...)
    return left, right

def main():
    x = torch...
    weights = torch...
    print(E14(...))

if __name__ == "__main__":
    main()
```

1.10 T08 - Neural Network Transformatin Laws

Mathematics and Theory

1.10.1 Exercise 00 - Linear Transformation Law

Part 1 - Why this matters

Linear layers mix input features with learned weights and add a bias. They form the core transformation in multilayer perceptrons, attention projections, classifiers, and many neural-network blocks.

Part 2 - Mathematical form

For $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{W} \in \mathbb{R}^{m \times n}$, and $\mathbf{b} \in \mathbb{R}^m$:

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b} \in \mathbb{R}^m$$

For a batch $\mathbf{X} \in \mathbb{R}^{B \times n}$, PyTorch uses $\mathbf{Y} = \mathbf{X}\mathbf{W}^\mathsf{T} + \mathbf{b}$.

Part 3 - Hints and Helper

Use `torch.nn.Linear(in_features, out_features)`. Its stored weight has shape (m, n), and it transforms the final input dimension while preserving leading batch dimensions.

Part 4 - Python code skeleton

```
import torch
import torch.nn as nn

def E00(x, in_features, out_features):
    layer = nn.Linear(...)
    y = layer(...)
    return y
```

```
def main():
    x = torch...
    print(E00(...))

if __name__ == "__main__":
    main()
```

1.10.2 Exercise 01 - Convolution Law

Part 1 - Why this matters

Convolution extracts local patterns using shared kernels. Weight sharing makes it efficient for images, signals, and spatial feature maps while preserving translation-aware structure.

Part 2 - Mathematical form

For 2D input X, PyTorch's convolution layer computes cross-correlation:

$$Y_{oij} = b_o + \sum_{c=1}^{C_{in}} \sum_{u=0}^{K_h-1} \sum_{v=0}^{K_w-1} W_{ocuv} X_{c,\, iS_h+uD_h-P_h,\, jS_w+vD_w-P_w}$$

using valid padded indices. Kernels are not spatially reversed in this practical definition.

Part 3 - Hints and Helper

Use `torch.nn.Conv2d()`. Track channels, kernel size, stride, padding, dilation, and groups. Inputs normally have shape (N, C, H, W).

Part 4 - Python code skeleton

```
import torch
import torch.nn as nn

def E01(x):
    convolution = nn.Conv2d(...)
    y = convolution(...)
    return y

def main():
    x = torch...
    print(E01(x))

if __name__ == "__main__":
    main()
```

1.10.3 Exercise 02 - Transposed Convolution Law

Part 1 - Why this matters

Transposed convolution learns spatial upsampling for decoders, segmentation, and generative networks. It applies the transpose of the linear map represented by convolution with respect to suitable vectorizations; it is not generally the inverse of convolution.

Part 2 - Mathematical form

For one spatial dimension, the output size is:

$$L_{out} = (L_{in} - 1)S - 2P + D(K - 1) + O + 1$$

where S is stride, P padding, D dilation, K kernel size, and O output padding. The same rule applies independently to height and width.

Part 3 - Hints and Helper

Use `torch.nn.ConvTranspose2d()`. Choose `output_padding` only to resolve output-shape ambiguity; it does not add ordinary zero padding to the output.

Part 4 - Python code skeleton

```
import torch
import torch.nn as nn

def E02(x):
    layer = nn.ConvTranspose2d(...)
    y = layer(...)
    return y

def main():
    x = torch...
    print(E02(x))

if __name__ == "__main__":
    main()
```

1.10.4 Exercise 03 - Depthwise Convolution Law

Part 1 - Why this matters

Depthwise convolution applies separate spatial filters to individual input channels. It greatly reduces parameters and computation and is commonly paired with a 1×1 pointwise convolution in efficient architectures.

Part 2 - Mathematical form

With depth multiplier M, each input channel c produces M outputs:

$$Y_{c,m,i,j} = b_{c,m} + \sum_{u,v} W_{c,m,u,v} X_{c,i+u,j+v}$$

There is no summation across input channels. The output channel count is $C_{out} = MC_{in}$.

Part 3 - Hints and Helper

Use `nn.Conv2d()` with `groups=in_channels` and `out_channels=M*in_channels`. Standard depthwise convolution requires the output channel count to be divisible by the input channel count.

Part 4 - Python code skeleton

```
import torch
import torch.nn as nn

def E03(x, channels, multiplier):
    layer = nn.Conv2d(
        channels, ..., kernel_size=..., groups=...
    )
    return layer(x)

def main():
    x = torch...
    print(E03(...))

if __name__ == "__main__":
    main()
```

1.10.5 Exercise 04 - Softmax Activation Law

Part 1 - Why this matters

Softmax converts logits into a normalized probability distribution and is used for multiclass prediction and attention weighting.

Part 2 - Mathematical form

For logits $\mathbf{z} \in \mathbb{R}^K$:

$$s_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}, \qquad s_i > 0, \quad \sum_i s_i = 1$$

Subtracting $\max_j z_j$ before exponentiation leaves the result unchanged and improves numerical stability.

Part 3 - Hints and Helper

Use `torch.softmax()` with the correct class dimension. Do not apply softmax before `CrossEntropyLoss`, because that loss expects raw logits.

Part 4 - Python code skeleton

```
import torch

def E04(logits, dim):
    probabilities = torch.softmax(...)
```

```
    return probabilities

def main():
    logits = torch...
    print(E04(...))

if __name__ == "__main__":
    main()
```

1.10.6 Exercise 05 - Sigmoid Activation Law

Part 1 - Why this matters

Sigmoid maps real inputs to $(0, 1)$, making it useful for binary probabilities, gates, and independent multi-label outputs. Large-magnitude inputs saturate and produce small gradients.

Part 2 - Mathematical form

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$0 < \sigma(x) < 1, \qquad \sigma'(x) = \sigma(x)[1 - \sigma(x)]$$

Part 3 - Hints and Helper

Use `torch.sigmoid()`. For binary loss from logits, prefer `BCEWithLogitsLoss` instead of applying sigmoid followed by ordinary BCE, because the combined implementation is more stable.

Part 4 - Python code skeleton

```
import torch

def E05(x):
    y = torch...
    return y

def main():
    x = torch...
    print(E05(x))

if __name__ == "__main__":
    main()
```

1.10.7 Exercise 06 - ReLU Activation Law

Part 1 - Why this matters

ReLU introduces nonlinearity while remaining computationally simple. It passes positive signals and sets negative signals to zero, but units can stop receiving gradients when they remain negative.

Part 2 - Mathematical form

$$\mathrm{ReLU}(x) = \max(0, x) = \begin{cases} 0, & x \leq 0, \\ x, & x > 0. \end{cases}$$

It is continuous but not classically differentiable at zero.

Part 3 - Hints and Helper

Use `torch.relu()`, `nn.ReLU()`, or the functional form. PyTorch uses derivative zero at $x = 0$. Avoid unsafe in-place activation when earlier values are needed by autograd.

Part 4 - Python code skeleton

```
import torch

def E06(x):
    y = torch...
    return y

def main():
    x = torch...
    print(E06(x))

if __name__ == "__main__":
    main()
```

1.10.8 Exercise 07 - GELU Activation Law

Part 1 - Why this matters

GELU smoothly weights inputs according to a Gaussian probability and is widely used in Transformer architectures. Unlike ReLU, it does not impose a hard zero threshold.

Part 2 - Mathematical form

$$\mathrm{GELU}(x) = x\Phi(x)$$

where Φ is the standard normal CDF. A common approximation is:

$$\frac{x}{2}\left[1 + \tanh\left(\sqrt{\frac{2}{\pi}}(x + 0.044715x^3)\right)\right]$$

Part 3 - Hints and Helper

Use `torch.nn.functional.gelu()` or `nn.GELU()`. The `approximate` option selects the exact or tanh approximation.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E07(x):
    y = F.gelu(x, approximate="...")
    return y

def main():
    x = torch...
    print(E07(x))

if __name__ == "__main__":
    main()
```

1.10.9 Exercise 08 - Swish Activation Law

Part 1 - Why this matters

Swish is a smooth, non-monotonic activation that gates an input by its sigmoid. It retains small negative outputs and can support smoother optimization than a hard threshold.

Part 2 - Mathematical form

For parameter $\beta > 0$:

$$\text{Swish}_\beta(x) = x\sigma(\beta x)$$

The common case $\beta = 1$ is also called SiLU:

$$\text{SiLU}(x) = x\sigma(x)$$

Part 3 - Hints and Helper

Use `torch.nn.functional.silu()` or `nn.SiLU()` for $\beta = 1$. For a different β, multiply the input explicitly by `torch.sigmoid(beta*x)`.

Part 4 - Python code skeleton

```
import torch

def E08(x, beta):
    y = x * torch.sigmoid(...)
    return y

def main():
    x = torch...
```

```
    print(E08(...))

if __name__ == "__main__":
    main()
```

1.10.10 Exercise 09 - Cross Entropy Loss Law

Part 1 - Why this matters

Cross entropy penalizes predicted class distributions that assign low probability to the correct class. It is the standard objective for multiclass classification and combines naturally with softmax logits.

Part 2 - Mathematical form

For logits $\mathbf{z}$ and correct class y:

$$L(\mathbf{z}, y) = -\log \frac{e^{z_y}}{\sum_{j=1}^{K} e^{z_j}} = -z_y + \log \sum_{j=1}^{K} e^{z_j}$$

For a target distribution $\mathbf{q}$:

$$L = -\sum_{i} q_i \log p_i$$

Part 3 - Hints and Helper

Use `nn.CrossEntropyLoss()` with raw logits. For class-index targets, use integer labels of dtype `torch.long`; do not apply softmax first.

Part 4 - Python code skeleton

```
import torch
import torch.nn as nn

def E09(logits, targets):
    loss_function = nn.CrossEntropyLoss()
    loss = loss_function(...)
    return loss

def main():
    logits = torch...
    targets = torch...
    print(E09(...))

if __name__ == "__main__":
    main()
```

1.10.11 Exercise 10 - MSE Loss Law

Part 1 - Why this matters

Mean squared error measures average squared prediction error. It strongly penalizes large deviations and is widely used in regression, reconstruction, and denoising.

Part 2 - Mathematical form

For predictions $\hat{y}_1, \ldots, \hat{y}_N$ and targets y_i:

$$L_{\text{MSE}} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$$

$$\frac{\partial L}{\partial \hat{y}_i} = \frac{2}{N}(\hat{y}_i - y_i)$$

Part 3 - Hints and Helper

Use `torch.nn.functional.mse_loss()` or `nn.MSELoss()`. Predictions and targets must have compatible shapes; choose reduction `mean`, `sum`, or `none` intentionally.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E10(prediction, target):
    loss = F.mse_loss(...)
    return loss

def main():
    prediction = torch...
    target = torch...
    print(E10(...))

if __name__ == "__main__":
    main()
```

1.10.12 Exercise 11 - L1 Loss Law

Part 1 - Why this matters

L1 loss measures average absolute error. It is less sensitive to extreme errors than MSE and is used in robust regression, reconstruction, and sparse-error settings.

Part 2 - Mathematical form

$$L_1 = \frac{1}{N}\sum_{i=1}^{N}|\hat{y}_i - y_i|$$

For nonzero error:

$$\frac{\partial L_1}{\partial \hat{y}_i} = \frac{1}{N}\,\mathrm{sign}(\hat{y}_i - y_i)$$

At zero error, PyTorch uses subgradient zero.

Part 3 - Hints and Helper

Use `torch.nn.functional.l1_loss()` or `nn.L1Loss()` and select the intended reduction.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E11(prediction, target):
    loss = F.l1_loss(...)
    return loss

def main():
    prediction = torch...
    target = torch...
    print(E11(...))

if __name__ == "__main__":
    main()
```

1.10.13 Exercise 12 - Binary Cross Entropy Loss Law

Part 1 - Why this matters

Binary cross entropy evaluates probabilities for binary or independent multi-label outcomes. It penalizes confident incorrect predictions strongly and corresponds to the negative Bernoulli log-likelihood.

Part 2 - Mathematical form

For target $y \in \{0, 1\}$ and probability $p \in (0, 1)$:

$$L_{\mathrm{BCE}}(p, y) = -[y \log p + (1 - y) \log(1 - p)]$$

The dataset loss is commonly the mean over all elements.

Part 3 - Hints and Helper

Use `binary_cross_entropy()` only with probabilities. For raw logits, prefer `binary_cross_entropy_w` or `BCEWithLogitsLoss` for numerical stability.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E12(logits, targets):
    loss = F.binary_cross_entropy_with_logits(...)
    return loss

def main():
    logits = torch...
    targets = torch...
    print(E12(...))

if __name__ == "__main__":
    main()
```

1.10.14 Exercise 13 - LogSoftmax Law

Part 1 - Why this matters

LogSoftmax produces normalized log probabilities directly. It avoids separately computing potentially tiny probabilities before taking logarithms and is used with negative log-likelihood objectives.

Part 2 - Mathematical form

For logits $\mathbf{z} \in \mathbb{R}^K$:

$$\ell_i = \log \frac{e^{z_i}}{\sum_j e^{z_j}} = z_i - \log \sum_{j=1}^{K} e^{z_j}$$

and e^{ℓ_i} forms a probability distribution.

Part 3 - Hints and Helper

Use `torch.log_softmax()` or `nn.LogSoftmax()` with the class dimension. Do not reproduce it as `log(softmax(x))`, which is less numerically stable.

Part 4 - Python code skeleton

```
import torch

def E13(logits, dim):
    log_probabilities = torch.log_softmax(...)
    return log_probabilities

def main():
    logits = torch...
    print(E13(...))

if __name__ == "__main__":
    main()
```

1.10.15 Exercise 14 - LogSoftmax Gradient Law

Part 1 - Why this matters

The LogSoftmax gradient shows how each log probability depends on every logit. This coupling is central to stable classification gradients and the behavior of negative log-likelihood losses.

Part 2 - Mathematical form

Let $\ell_i = \text{LogSoftmax}(\mathbf{z})_i$ and $s_i = \text{Softmax}(\mathbf{z})_i$. Then:

$$\frac{\partial \ell_i}{\partial z_j} = \delta_{ij} - s_j$$

For upstream gradient $g_i = \partial L / \partial \ell_i$:

$$\frac{\partial L}{\partial z_j} = g_j - s_j \sum_i g_i$$

Part 3 - Hints and Helper

Use `torch.autograd.functional.jacobian()` to inspect the full Jacobian, or supply an upstream gradient to `torch.autograd.grad()`. Differentiate `log_softmax` directly rather than detaching its output.

Part 4 - Python code skeleton

```
import torch

def E14(logits):
    function = lambda value: torch.log_softmax(value, dim=...)
    jacobian = torch.autograd.functional.jacobian(...)
    return jacobian

def main():
    logits = torch.tensor(..., requires_grad=True)
    print(E14(logits))

if __name__ == "__main__":
    main()
```

Book 2

Applied Linear Algebra in Code

2.1 Introduction

2.1.1 What will the book teach you

After building the mathematical foundations of tensors and neural networks, you must learn how to translate linear algebra into clear, correct, and efficient code. This book will teach you how to apply the most important linear algebra concepts using PyTorch, beginning with basic tensor construction and progressing through vector operations, matrix multiplication, broadcasting, shape reasoning, and projection onto vector spaces.

You will not study these concepts as isolated formulas. Instead, you will learn how tensor shapes, data types, devices, dimensions, and numerical precision affect real implementations. You will also practise identifying shape errors, debugging invalid operations, comparing different computational methods, and designing functions that remain safe when tensor dimensions change.

The book consists of practical exercises and coding challenges. Each exercise will introduce a problem and provide guidance for finding a solution, but the complete solution will not be provided. You must reason about the mathematics, predict the tensor shapes, write the PyTorch code, test the result, and correct your own mistakes. This process will develop the problem-solving skills needed to build reliable artificial intelligence systems rather than simply copy existing code.

Before moving to the next book, make sure that you can explain every operation, predict its output shape, and implement it without depending on a prepared solution. The objective is not only to make the code run, but to understand why it works, when it fails, and how it can be improved.

2.1.2 The Chapters (ALA00 - ALA06) included in this book

- ALA00 - Tensor Operations in Code
- ALA01 - Vector Dot & Norm Computation
- ALA02 - Matrix Multiplication (matmul / mm)
- ALA03 - Outer vs Inner Product

- ALA04 - Broadcasting Rules & Mechanics
- ALA05 - Shape Reasoning & Dimension Flow
- ALA06 - Basis, Span & Projection (Applied)

2.2 ALA00 - Tensor Operations in Code

Mathematics and Theory

2.2.1 Exercise 00 - Basic Tensor Creation

Part 1 - Why this matters

Every PyTorch program begins by representing data as tensors. Images, text embeddings, model parameters, labels, and numerical measurements all become tensors before a neural network can process them. Creating a tensor correctly determines its initial values, dimensions, and numerical behaviour.

Tensor creation is also the first point where implementation mistakes can enter a pipeline. An incorrect shape, data type, or value pattern may remain hidden until a later operation fails. Learning several creation methods helps you choose between fixed data, initialized storage, random samples, and structured sequences.

Part 2 - Mathematical form

A tensor of order k with shape $(d_1, \ldots, d_k)$ is an element of:

$$\mathcal{X} \in \mathbb{R}^{d_1 \times \cdots \times d_k}$$

Its number of elements is:

$$N = \prod_{i=1}^{k} d_i$$

Tensor creation assigns one value to every valid index of this shape.

Part 3 - Hints and Helper

Use `torch.tensor()` for explicit data, `torch.zeros()` and `torch.ones()` for constant initialization, `torch.arange()` for sequences, and `torch.rand()` for uniform random values. Specify `dtype` and `device` when defaults are unsuitable.

Part 4 - Python code skeleton

```
import torch

def E00():
    A = torch...
    B = torch...
    C = torch...
    return A, B, C
```

```
def main():
    tensors = E00()
    print(...)

if __name__ == "__main__":
    main()
```

2.2.2 Exercise 01 - Tensor Shape Inspection

Part 1 - Why this matters

Tensor shape describes how values are organized across dimensions. In a neural network, these dimensions may represent batches, channels, sequence positions, height, width, or features. Inspecting shape is therefore essential before combining tensors or passing them into a layer.

Shape inspection also supports debugging and documentation. A tensor can contain the expected values but still be unusable because its axes are ordered incorrectly. Understanding rank, dimension sizes, and element count allows you to predict whether an operation is mathematically valid.

Part 2 - Mathematical form

For $\mathcal{X} \in \mathbb{R}^{d_1 \times \cdots \times d_k}$:

$$\text{shape}(\mathcal{X}) = (d_1, \ldots, d_k)$$

$$\text{ndim}(\mathcal{X}) = k, qquad \text{numel}(\mathcal{X}) = \prod_{i=1}^{k} d_i$$

Part 3 - Hints and Helper

Inspect `tensor.shape`, `tensor.ndim`, `tensor.size()`, and `tensor.numel()`. Use `size(dim)` to read one axis. Do not confuse the number of dimensions with the total number of values.

Part 4 - Python code skeleton

```
import torch

def E01(A):
    shape = ...
    dimensions = ...
    elements = ...
    return shape, dimensions, elements

def main():
    A = torch...
    print(E01(A))

if __name__ == "__main__":
    main()
```

2.2.3 Exercise 02 - Tensor Type Conversion

Part 1 - Why this matters

Tensor data type controls how values are stored and which operations are available. Floating-point types support gradients and neural-network training, integer types represent indices and class labels, and Boolean tensors represent logical conditions and masks.

Type conversion affects precision, memory use, and compatibility. Converting a floating-point tensor to an integer removes its fractional information, while using a lower-precision floating type may introduce rounding error. A correct program converts types intentionally rather than relying on accidental defaults.

Part 2 - Mathematical form

A conversion maps stored values from one numerical set to another:

$$\text{cast}_{\tau_1 \to \tau_2} : \mathbb{T}_{\tau_1}^{d_1 \times \cdots \times d_k} \to \mathbb{T}_{\tau_2}^{d_1 \times \cdots \times d_k}$$

The shape is preserved, but values may be rounded, truncated, or represented with different precision.

Part 3 - Hints and Helper

Use `tensor.to(dtype=...)`, `tensor.float()`, `tensor.long()`, or `tensor.bool()`. Inspect `tensor.dtype` before and after conversion. Class indices for many PyTorch losses require `torch.long`.

Part 4 - Python code skeleton

```
import torch

def E02(A):
    float_tensor = A...
    integer_tensor = A...
    boolean_tensor = A...
    return float_tensor, integer_tensor, boolean_tensor

def main():
    A = torch...
    print(E02(A))

if __name__ == "__main__":
    main()
```

2.2.4 Exercise 03 - Tensor Device Assignment

Part 1 - Why this matters

PyTorch tensors perform computation on a device such as the CPU or a CUDA GPU. Moving tensors to a GPU can accelerate large parallel operations, while the CPU remains useful for

data loading, control logic, and workloads that are too small to benefit from transfer overhead.

All tensors participating in most operations must be on the same device. Device mismatch errors commonly occur when a model is moved to the GPU but its inputs or newly created helper tensors remain on the CPU. Device-aware code must work correctly even when CUDA is unavailable.

Part 2 - Mathematical form

Device assignment changes the storage location without changing mathematical content:

$$\mathcal{X}_{\text{CPU}} \longmapsto \mathcal{X}_{\text{device}}$$

$$\text{shape}(\mathcal{X}_{\text{CPU}}) = \text{shape}(\mathcal{X}_{\text{device}})$$

Part 3 - Hints and Helper

Choose a device with `torch.cuda.is_available()`, create `torch.device(...)`, and move tensors with `tensor.to(device)`. Inspect `tensor.device`. Moving data returns a tensor and does not always modify the original variable in place.

Part 4 - Python code skeleton

```
import torch

def E03(A):
    device = torch.device(...)
    A_device = A.to(...)
    return A_device

def main():
    A = torch...
    print(E03(A).device)

if __name__ == "__main__":
    main()
```

2.2.5 Exercise 04 - Element-wise Addition

Part 1 - Why this matters

Element-wise addition combines aligned tensor values without mixing positions. Neural networks use it to add biases, residual connections, learned updates, masks, and accumulated features. The operation is simple, but its validity depends on tensor shapes.

When shapes differ, PyTorch may apply broadcasting rather than reject the operation. This can be intentional, such as adding one bias per feature, or an unnoticed bug that produces a valid but incorrectly interpreted output.

Part 2 - Mathematical form

For aligned tensors $\mathcal{A}, \mathcal{B} \in \mathbb{R}^{d_1 \times \cdots \times d_k}$:

$$\mathcal{C} = \mathcal{A} + \mathcal{B}, \qquad C_{i_1 \ldots i_k} = A_{i_1 \ldots i_k} + B_{i_1 \ldots i_k}$$

The output has the common or broadcasted shape.

Part 3 - Hints and Helper

Use the `+` operator or `torch.add()`. Inspect both input shapes and the output shape. Use `torch.broadcast_shapes()` when you need to check compatibility before addition.

Part 4 - Python code skeleton

```
import torch

def E04(A, B):
    C = torch...
    return C

def main():
    A = torch...
    B = torch...
    print(E04(...))

if __name__ == "__main__":
    main()
```

2.2.6 Exercise 05 - Element-wise Multiplication

Part 1 - Why this matters

Element-wise multiplication lets one tensor scale, gate, or mask another at corresponding positions. It appears in attention weights, activation gates, feature modulation, dropout masks, and many loss calculations.

This operation is different from a dot product or matrix multiplication because it performs no summation and normally preserves the aligned shape. Confusing these operations changes both the mathematics and the output dimensions.

Part 2 - Mathematical form

For aligned tensors $\mathcal{A}$ and $\mathcal{B}$:

$$\mathcal{C} = \mathcal{A} \odot \mathcal{B}$$

$$C_{i_1 \ldots i_k} = A_{i_1 \ldots i_k} B_{i_1 \ldots i_k}$$

No index is contracted or reduced.

Part 3 - Hints and Helper

Use the `*` operator or `torch.mul()`. Verify shape compatibility and do not use `torch.matmul()` for this law. Broadcasting may apply when singleton dimensions are present.

Part 4 - Python code skeleton

```
import torch

def E05(A, B):
    C = torch...
    return C

def main():
    A = torch...
    B = torch...
    print(E05(...))

if __name__ == "__main__":
    main()
```

2.2.7 Exercise 06 - Scalar-Tensor Operations

Part 1 - Why this matters

Scalar-tensor operations apply one value to every element of a tensor. They are used to scale features, shift activations, normalize measurements, adjust thresholds, and implement learning formulas without constructing a second full tensor.

PyTorch handles the scalar through broadcasting, so the tensor shape is usually preserved. The selected operator still matters: multiplication changes scale, addition changes location, and division may create undefined or unstable values when the scalar is zero or extremely small.

Part 2 - Mathematical form

For scalar $\alpha \in \mathbb{R}$ and tensor $\mathcal{A} \in \mathbb{R}^{d_1 \times \cdots \times d_k}$:

$$(\alpha\mathcal{A})_{i_1 \ldots i_k} = \alpha A_{i_1 \ldots i_k}$$

$$(\mathcal{A} + \alpha)_{i_1 \ldots i_k} = A_{i_1 \ldots i_k} + \alpha$$

The output shape equals the input shape.

Part 3 - Hints and Helper

Use standard operators such as `+`, `-`, `*`, and `/`. Observe PyTorch type promotion when a scalar and tensor use different numerical types. Avoid division by zero.

Part 4 - Python code skeleton

```
import torch

def E06(A, scalar):
    scaled = ...
    shifted = ...
    divided = ...
    return scaled, shifted, divided

def main():
```

```
    A = torch...
    print(E06(...))

if __name__ == "__main__":
    main()
```

2.2.8 Exercise 07 - Tensor Comparison Operations

Part 1 - Why this matters

Tensor comparisons test relationships between aligned values and return Boolean results. They identify thresholds, classification decisions, valid ranges, matching labels, and conditions used to control later tensor operations.

Comparison output is not a single Boolean unless the tensor contains one element or the result is reduced. Each position produces its own truth value, and broadcasting rules may apply between differently shaped operands.

Part 2 - Mathematical form

For aligned tensors $\mathcal{A}$ and $\mathcal{B}$, equality defines:

$$C_{i_1 \ldots i_k} = \begin{cases} \text{True}, & A_{i_1 \ldots i_k} = B_{i_1 \ldots i_k}, \\ \text{False}, & \text{otherwise.} \end{cases}$$

Other relations replace equality with $\neq, <, \leq, >, \geq$.

Part 3 - Hints and Helper

Use operators such as `==`, `!=`, `>`, and `<=`, or PyTorch comparison functions. Use `torch.all()` or `torch.any()` only when a reduced Boolean conclusion is required.

Part 4 - Python code skeleton

```
import torch

def E07(A, B):
    equal = ...
    greater = ...
    less_or_equal = ...
    return equal, greater, less_or_equal

def main():
    A = torch...
    B = torch...
    print(E07(...))

if __name__ == "__main__":
    main()
```

2.2.9 Exercise 08 - Boolean Masking Challenge

Part 1 - Why this matters

Boolean masking selects or modifies tensor values according to logical conditions. It is used to remove invalid data, isolate classes, apply thresholds, ignore padded tokens, and restrict loss calculations to relevant positions.

Masking can either extract matching values into a one-dimensional result or preserve shape by replacing values conditionally. Understanding this difference is essential because direct Boolean indexing and `torch.where()` produce different output structures.

Part 2 - Mathematical form

For condition mask $\mathcal{M}$ with the same shape as $\mathcal{A}$:

$$\mathcal{S} = \{A_{i_1 \ldots i_k} : M_{i_1 \ldots i_k} = \text{True}\}$$

A shape-preserving conditional tensor is:

$$C_i = \begin{cases} A_i, & M_i = \text{True}, \\ B_i, & M_i = \text{False}. \end{cases}$$

Part 3 - Hints and Helper

Build masks with comparisons and combine conditions using `&`, `|`, and `~`, adding parentheses around each comparison. Use `A[mask]` for extraction and `torch.where()` for shape-preserving selection.

Part 4 - Python code skeleton

```
import torch

def E08(A, lower, upper):
    mask = (... ) & (... )
    selected = A[...]
    replaced = torch.where(...)
    return selected, replaced

def main():
    A = torch...
    print(E08(...))

if __name__ == "__main__":
    main()
```

2.2.10 Exercise 09 - Tensor Operation Chain Challenge

Part 1 - Why this matters

Real machine-learning programs rarely perform only one tensor operation. Data usually passes through a chain of scaling, shifting, masking, reduction, and shape transformations. Every stage must receive the shape and type produced by the stage before it.

Chained operations test whether you understand data flow rather than isolated syntax. A mathematically correct individual step can still break the full pipeline through unintended broadcasting, type conversion, device mismatch, or loss of dimensions.

Part 2 - Mathematical form

Let the transformation chain be:

$$\mathcal{B} = \alpha\mathcal{A} + \beta$$

$$\mathcal{M} = \mathcal{B} > \tau$$

$$\mathbf{s} = \sum_{j} \text{where}(\mathcal{M}, \mathcal{B}, 0)$$

Each output becomes the next operation's input.

Part 3 - Hints and Helper

Implement one step at a time and inspect shape, dtype, and device after every stage. Avoid unnecessary intermediate copies, but keep meaningful variable names while debugging. Confirm which axis the final reduction uses.

Part 4 - Python code skeleton

```
import torch

def E09(A, alpha, beta, threshold):
    B = ...
    mask = ...
    filtered = torch.where(...)
    result = torch.sum(...)
    return result

def main():
    A = torch...
    print(E09(...))

if __name__ == "__main__":
    main()
```

2.2.11 Exercise 10 - Debugging Tensor Type Errors

Part 1 - Why this matters

Tensor type errors occur when an operation receives a dtype that does not match its mathematical or API requirements. Examples include supplying floating-point class labels where integer indices are expected or requesting gradients from an integer tensor.

Debugging these errors requires understanding the data's role, not merely forcing every tensor to the same type. Features, parameters, labels, masks, and indices often require different dtypes, and an incorrect conversion can hide an error while changing the meaning of the computation.

Part 2 - Mathematical form

An operation F has a type domain:

$$F : \mathbb{T}_{\tau_1} \times \mathbb{T}_{\tau_2} o \mathbb{T}_{\tau_3}$$

A call is valid only when each input belongs to an accepted dtype domain or a defined promotion rule converts it safely.

Part 3 - Hints and Helper

Print `dtype`, `shape`, and `device` for every input near the failing operation. Read the API requirement, then convert only the tensor whose semantic role demands conversion. Remember that only floating-point and complex tensors can require gradients.

Part 4 - Python code skeleton

```
import torch

def E10(features, labels):
    print(features.dtype, labels.dtype)
    fixed_features = features...
    fixed_labels = labels...
    result = ...
    return result

def main():
    features = torch...
    labels = torch...
    print(E10(...))

if __name__ == "__main__":
    main()
```

2.2.12 Exercise 11 - Tensor Operation Optimization Challenge

Part 1 - Why this matters

Efficient tensor code expresses work as vectorized operations that PyTorch can execute in optimized native kernels and parallelize on accelerators. Replacing Python loops with tensor operations often improves speed, readability, and GPU utilization.

Optimization is not simply shortening code. A faster implementation must preserve numerical results, shape semantics, gradient flow, and reasonable memory use. Temporary tensors, repeated device transfers, explicit copies, and unnecessary synchronization can make otherwise vectorized code inefficient.

Part 2 - Mathematical form

An element-wise loop:

$$C_i = f(A_i), \qquad i = 1, \ldots, N$$

can be represented as the vectorized transformation:

$$\mathcal{C} = f(\mathcal{A})$$

when f acts independently on tensor elements. Correct optimization requires the vectorized and reference outputs to agree within an appropriate tolerance.

Part 3 - Hints and Helper

Identify Python loops, repeated allocations, and transfers inside the hot path. Use broadcasting, reductions, and fused PyTorch expressions where appropriate. Validate with `torch.allclose()` and benchmark after CUDA synchronization when measuring GPU execution.

Part 4 - Python code skeleton

```
import torch

def E11(A):
    optimized = ...
    return optimized

def main():
    A = torch...
    result = E11(A)
    reference = ...
    print(torch.allclose(result, reference))

if __name__ == "__main__":
    main()
```

2.3 ALA01 - Vector Dot & Norm Computation

Mathematics and Theory

2.3.1 Exercise 00 - Manual Dot Product Implementation

Part 1 - Why this matters

Implementing a dot product manually reveals how two vectors are combined: each pair of aligned components is multiplied, then every product is accumulated into one scalar. This pattern appears inside matrix multiplication, linear layers, attention scores, and similarity measures.

The manual version is not intended to outperform PyTorch. Its purpose is to connect loop logic with the mathematical summation and to practise validating length, dtype, and device compatibility before using optimized library operations.

Part 2 - Mathematical form

For $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$:

$$\mathbf{x} \cdot \mathbf{y} = \sum_{i=1}^{n} x_i y_i \in \mathbb{R}$$

Both vectors must have the same number of components.

Part 3 - Hints and Helper

Initialize an accumulator without converting tensor values into Python numbers, then iterate over valid indices and add one product at a time. Do not call `torch.dot()`, `torch.sum()`, or matrix multiplication in the manual implementation.

Part 4 - Python code skeleton

```
import torch

def E00(x, y):
    result = ...
    for i in range(...):
        result = result + ...
    return result

def main():
    x = torch...
    y = torch...
    print(E00(...))

if __name__ == "__main__":
    main()
```

2.3.2 Exercise 01 - Dot Product Using PyTorch

Part 1 - Why this matters

PyTorch provides an optimized dot-product operation that executes the same mathematics as the manual implementation while using efficient native kernels. This is the appropriate form for real models and large numerical workloads.

Using the library operation also makes the intended mathematics explicit. However, it still requires compatible one-dimensional tensors, so understanding the shape rule remains necessary even when the computation is automated.

Part 2 - Mathematical form

For $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$:

$$d = \mathbf{x}^\mathsf{T}\mathbf{y} = \sum_{i=1}^{n} x_i y_i$$

The output d is a scalar tensor.

Part 3 - Hints and Helper

Use `torch.dot()` for two 1D tensors. Inspect both tensors' shapes, dtypes, and devices first. Compare its result with the manual implementation using `torch.allclose()`.

Part 4 - Python code skeleton

```
import torch

def E01(x, y):
    result = torch...
    return result

def main():
    x = torch...
    y = torch...
    print(E01(...))

if __name__ == "__main__":
    main()
```

2.3.3 Exercise 02 - L2 Norm Calculation

Part 1 - Why this matters

The L2 norm measures Euclidean vector length. It is used to normalize embeddings, calculate distances, constrain parameters, and measure gradient or activation magnitude in neural networks.

Because the norm squares and sums all components, every coordinate contributes to the final magnitude. Large components receive greater influence, and the result is always nonnegative and equals zero only for the zero vector.

Part 2 - Mathematical form

For $\mathbf{x} \in \mathbb{R}^n$:

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^{n} x_i^2}$$

It satisfies homogeneity:

$$\|\alpha\mathbf{x}\|_2 = |\alpha|\|\mathbf{x}\|_2$$

Part 3 - Hints and Helper

Use `torch.linalg.vector_norm(x, ord=2)` or `torch.norm()`. Specify a dimension for batched vectors. Use a floating-point tensor when calculating norms.

Part 4 - Python code skeleton

```
import torch

def E02(x):
    norm = torch.linalg.vector_norm(...)
```

```
    return norm

def main():
    x = torch...
    print(E02(x))

if __name__ == "__main__":
    main()
```

2.3.4 Exercise 03 - L1 Norm Calculation

Part 1 - Why this matters

The L1 norm sums absolute component magnitudes. It appears in sparse regularization, robust error measures, feature selection, and comparisons where linear rather than squared magnitude is desired.

Unlike the L2 norm, the L1 norm does not increase the influence of large values through squaring. Its geometry can encourage solutions with exactly zero components when used as an optimization penalty.

Part 2 - Mathematical form

For $\mathbf{x} \in \mathbb{R}^n$:

$$\|\mathbf{x}\|_1 = \sum_{i=1}^{n} |x_i|$$

The result is nonnegative and equals zero exactly when $\mathbf{x} = \mathbf{0}$.

Part 3 - Hints and Helper

Use `torch.linalg.vector_norm(x, ord=1)` or explicitly combine `torch.abs()` and `torch.sum()`. Specify the vector dimension for batched input.

Part 4 - Python code skeleton

```
import torch

def E03(x):
    norm = torch.linalg.vector_norm(...)
    return norm

def main():
    x = torch...
    print(E03(x))

if __name__ == "__main__":
    main()
```

2.3.5 Exercise 04 - Vector Normalization

Part 1 - Why this matters

Vector normalization removes magnitude while preserving direction. It allows embeddings and features to be compared on a common scale and is a required step in cosine similarity and many geometric algorithms.

The zero vector has no direction and cannot be divided by its norm. Practical implementations must therefore define safe behaviour for zero or nearly zero norms rather than allowing division to generate invalid values.

Part 2 - Mathematical form

For nonzero $\mathbf{x} \in \mathbb{R}^n$:

$$\widehat{\mathbf{x}} = \frac{\mathbf{x}}{\|\mathbf{x}\|_2}$$

$$\|\widehat{\mathbf{x}}\|_2 = 1$$

Part 3 - Hints and Helper

Use `torch.nn.functional.normalize()` with `p=2` and the correct dimension, or divide by a safely clamped norm. Choose a small epsilon that protects zero norms without unnecessarily changing ordinary vectors.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E04(x):
    normalized = F.normalize(x, p=..., dim=..., eps=...)
    return normalized

def main():
    x = torch...
    print(E04(x))

if __name__ == "__main__":
    main()
```

2.3.6 Exercise 05 - Cosine Similarity Implementation

Part 1 - Why this matters

Cosine similarity compares vector direction independently of magnitude. It is widely used for embeddings, document retrieval, recommendation, clustering, and measuring similarity between learned representations.

The measure ranges from -1 to 1 for real nonzero vectors. Positive values indicate similar direction, zero indicates orthogonality, and negative values indicate opposing direction. Zero norms require numerical protection.

Part 2 - Mathematical form

For nonzero $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$:

$$\text{cos_sim}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\|_2 \|\mathbf{y}\|_2}$$

Part 3 - Hints and Helper

Implement the formula using dot products and norms, or use `torch.nn.functional.cosine_similari` Specify the feature dimension and an epsilon. Test with identical, orthogonal, and opposite vectors.

Part 4 - Python code skeleton

```
import torch

def E05(x, y, eps):
    numerator = ...
    denominator = ...
    similarity = numerator / ...
    return similarity

def main():
    x = torch...
    y = torch...
    print(E05(...))

if __name__ == "__main__":
    main()
```

2.3.7 Exercise 06 - Angle Between Two Vectors

Part 1 - Why this matters

The angle between vectors gives an interpretable geometric measure of directional difference. It is used in orientation analysis, embedding geometry, robotics, and algorithms that compare directions rather than raw distances.

The inverse cosine is sensitive when numerical rounding moves cosine similarity slightly outside $[-1, 1]$. A stable implementation clamps the computed cosine before applying the inverse trigonometric function.

Part 2 - Mathematical form

For nonzero vectors:

$$\theta = \arccos\left(\frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\|_2 \|\mathbf{y}\|_2}\right), \qquad 0 \le \theta \le \pi$$

Part 3 - Hints and Helper

Compute cosine similarity, clamp it to $[-1, 1]$, then use `torch.acos()`. The output is in radians. Convert to degrees only if the exercise explicitly requires it.

Part 4 - Python code skeleton

```
import torch

def E06(x, y):
    cosine = ...
    safe_cosine = torch.clamp(...)
    angle = torch.acos(...)
    return angle

def main():
    x = torch...
    y = torch...
    print(E06(...))

if __name__ == "__main__":
    main()
```

2.3.8 Exercise 07 - Batched Dot Product

Part 1 - Why this matters

Machine-learning systems process many vector pairs at once. A batched dot product computes one scalar interaction for every pair without a Python loop, supporting attention scores, batch similarity, and parallel feature comparison.

The batch dimension must be preserved while the feature dimension is reduced. Choosing the wrong reduction axis can collapse samples together or produce an output whose shape looks valid but has the wrong meaning.

Part 2 - Mathematical form

For $\mathbf{X}, \mathbf{Y} \in \mathbb{R}^{B \times n}$:

$$d_b = \sum_{i=1}^{n} X_{bi} Y_{bi}, \qquad b = 1, \ldots, B$$

$$\mathbf{d} \in \mathbb{R}^{B}$$

Part 3 - Hints and Helper

Multiply aligned batches element-wise and sum across the feature dimension, or use `torch.einsum()`. Confirm that the result has exactly one value per batch item.

Part 4 - Python code skeleton

```
import torch

def E07(X, Y):
    products = ...
    dots = torch.sum(..., dim=...)
    return dots

def main():
    X = torch...
    Y = torch...
    print(E07(...))

if __name__ == "__main__":
    main()
```

2.3.9 Exercise 08 - Stability Issues in Norm Computation

Part 1 - Why this matters

Directly squaring very large components can overflow, while squaring extremely small components can underflow. These problems can turn a mathematically finite norm into infinity or zero and corrupt later normalization or similarity calculations.

Numerical stability depends on dtype as well as formula. Higher precision offers a wider and more accurate numerical range, while stable norm algorithms can rescale intermediate values. Adding epsilon protects division but does not fix every overflow problem in the original norm computation.

Part 2 - Mathematical form

A scaled L2 norm may use $s = \max_i |x_i|$:

$$\|\mathbf{x}\|_2 = s\sqrt{\sum_i \left(\frac{x_i}{s}\right)^2}$$

for $s > 0$. This avoids directly squaring the largest original magnitude.

Part 3 - Hints and Helper

Compare a naive square-sum-square-root calculation with `torch.linalg.vector_norm()`. Test large and small magnitudes and compare `float32` with `float64`. Handle the zero vector separately in a manual scaled implementation.

Part 4 - Python code skeleton

```
import torch

def E08(x):
    naive = torch.sqrt(torch.sum(...))
    stable = torch.linalg.vector_norm(...)
    return naive, stable
```

```
def main():
    x = torch.tensor(..., dtype=torch.float32)
    print(E08(x))

if __name__ == "__main__":
    main()
```

2.3.10 Exercise 09 - Gradient-safe Dot Product Challenge

Part 1 - Why this matters

A differentiable dot product must remain connected to PyTorch's computation graph. This allows a loss built from the result to propagate gradients back to both input vectors and, ultimately, to the parameters that produced them.

Seemingly harmless actions can break this connection. Extracting values with `.item()`, converting through NumPy, wrapping an existing tensor with `torch.tensor()`, or detaching intermediate results prevents autograd from tracing the intended computation.

Part 2 - Mathematical form

For $d = \mathbf{x} \cdot \mathbf{y}$:

$$\nabla_{\mathbf{x}} d = \mathbf{y}, \qquad \nabla_{\mathbf{y}} d = \mathbf{x}$$

The implementation should produce these gradients through automatic differentiation.

Part 3 - Hints and Helper

Use only differentiable PyTorch operations on tensors requiring gradients. Call `torch.autograd.grad()` for both inputs and compare the returned gradients with the analytical expressions. Avoid in-place modification of tracked leaf tensors.

Part 4 - Python code skeleton

```
import torch

def E09(x, y):
    dot = ...
    gradients = torch.autograd.grad(dot, (..., ...))
    return dot, gradients

def main():
    x = torch.tensor(..., requires_grad=True)
    y = torch.tensor(..., requires_grad=True)
    print(E09(...))

if __name__ == "__main__":
    main()
```

2.3.11 Exercise 10 - Custom Similarity Metric Design

Part 1 - Why this matters

Different applications define similarity differently. Direction may matter more than magnitude for text embeddings, while scale or component importance may be essential in another dataset. Designing a custom metric requires translating the application's meaning into a precise mathematical rule.

A useful metric should also have understood range, symmetry, numerical behaviour, and differentiability. Combining several terms without controlling their scales can make one component dominate even when the formula appears balanced.

Part 2 - Mathematical form

One possible design combines cosine similarity with normalized distance:

$$S(\mathbf{x}, \mathbf{y}) = \alpha \cos_\text{sim}(\mathbf{x}, \mathbf{y}) + (1 - \alpha)\frac{1}{1 + \|\mathbf{x} - \mathbf{y}\|_2}$$

where $0 \leq \alpha \leq 1$. Other defensible formulas may be designed if their properties are explained.

Part 3 - Hints and Helper

Choose the desired invariances and output range before coding. Protect zero norms, keep operations differentiable, and test identical, opposite, orthogonal, scaled, and zero vectors. Explain what a high and low score mean.

Part 4 - Python code skeleton

```
import torch

def E10(x, y, alpha, eps):
    directional_score = ...
    distance_score = ...
    similarity = ...
    return similarity

def main():
    x = torch...
    y = torch...
    print(E10(...))

if __name__ == "__main__":
    main()
```

2.4 ALA02 - Matrix Multiplication (matmul / mm)

Mathematics and Theory

2.4.1 Exercise 00 - Basic Matrix Multiplication

Part 1 - Why this matters

Matrix multiplication combines input features through weighted sums. It is the central operation behind linear layers, coordinate transformations, attention, and many numerical algorithms used in artificial intelligence.

Unlike element-wise multiplication, each output entry depends on an entire row of the first matrix and an entire column of the second. Understanding this rule allows you to predict both shape compatibility and the meaning of every output dimension.

Part 2 - Mathematical form

For $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{n \times p}$:

$$\mathbf{C} = \mathbf{AB} \in \mathbb{R}^{m \times p}$$

$$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}$$

Part 3 - Hints and Helper

Use `torch.matmul()` or the `@` operator. Check that the columns of $\mathbf{A}$ equal the rows of $\mathbf{B}$. Do not replace matrix multiplication with the element-wise `*` operator.

Part 4 - Python code skeleton

```
import torch

def E00(A, B):
    C = torch...
    return C

def main():
    A = torch...
    B = torch...
    print(E00(...))

if __name__ == "__main__":
    main()
```

2.4.2 Exercise 01 - matmul vs mm Comparison

Part 1 - Why this matters

PyTorch offers both `matmul` and `mm`, but their accepted inputs are different. Choosing the correct function makes the intended dimensional behaviour clear and prevents code from silently relying on broadcasting.

The functions agree for ordinary 2D matrix multiplication. Outside that case, `matmul` supports vectors and higher-dimensional batches, whereas `mm` is restricted to two matrices and does not broadcast batch axes.

Part 2 - Mathematical form

For 2D matrices:

$$\text{mm}(\mathbf{A}, \mathbf{B}) = \text{matmul}(\mathbf{A}, \mathbf{B}) = \mathbf{AB}$$

For batch shapes, `matmul` treats the final two axes as matrices and broadcasts compatible leading axes.

Part 3 - Hints and Helper

Test 1D, 2D, and 3D inputs with both functions. Record output shapes and errors. Use `torch.allclose()` when both functions accept the same inputs; do not catch an error without explaining why it occurred.

Part 4 - Python code skeleton

```
import torch

def E01(A, B):
    matmul_result = torch.matmul(...)
    mm_result = torch.mm(...)
    return matmul_result, mm_result

def main():
    A = torch...
    B = torch...
    print(E01(...))

if __name__ == "__main__":
    main()
```

2.4.3 Exercise 02 - Manual Matrix Multiplication

Part 1 - Why this matters

Manual matrix multiplication exposes the three-index structure hidden inside an optimized library call. Two loops choose an output position, while a third loop accumulates products across the shared inner dimension.

This implementation is educational rather than performant. It helps you reason about row-column interaction, output initialization, and index boundaries before using the optimized operation in real models.

Part 2 - Mathematical form

For compatible $\mathbf{A}$ and $\mathbf{B}$:

$$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}, \quad i = 1, \ldots, m, \quad j = 1, \ldots, p$$

The output must be initialized with shape (m, p).

Part 3 - Hints and Helper

Validate the inner dimensions, allocate an output on the same dtype and device, then use three loops. Do not call `matmul`, `mm`, `einsum`, or `@`. Compare the result with PyTorch afterward.

Part 4 - Python code skeleton

```
import torch

def E02(A, B):
    C = torch.zeros(..., dtype=A.dtype, device=A.device)
    for i in range(...):
        for j in range(...):
            for k in range(...):
                C[i, j] = C[i, j] + ...
    return C

def main():
    A = torch...
    B = torch...
    print(E02(...))

if __name__ == "__main__":
    main()
```

2.4.4 Exercise 03 - Shape Compatibility Check

Part 1 - Why this matters

Matrix multiplication is valid only when its inner dimensions match. Checking this condition before computation produces clearer errors and prevents confusion between matrix, batch, and feature dimensions.

A robust compatibility function should also predict the result shape. This turns shape reasoning into an explicit contract that can be reused in data pipelines and custom neural-network layers.

Part 2 - Mathematical form

For shapes (m, n) and (q, p):

$$\mathbf{AB} \text{ is defined} \quad \Longleftrightarrow \quad n = q$$

When defined:

$$\text{shape}(\mathbf{AB}) = (m, p)$$

Part 3 - Hints and Helper

Inspect `shape[-1]` of the first matrix and `shape[-2]` of the second. Raise an informative `ValueError` that reports both shapes. Require two dimensions for this exercise before adding batch support later.

Part 4 - Python code skeleton

```
import torch

def E03(A, B):
    if A.ndim != ... or B.ndim != ...:
        raise ValueError(...)
    if ... != ...:
        raise ValueError(...)
    output_shape = (..., ...)
    return output_shape

def main():
    A = torch...
    B = torch...
    print(E03(...))

if __name__ == "__main__":
    main()
```

2.4.5 Exercise 04 - Batched Matrix Multiplication

Part 1 - Why this matters

Batched matrix multiplication performs many independent matrix products in one vectorized operation. It is used for sequences, attention heads, batches of transformations, and any workload where corresponding matrix pairs must be multiplied in parallel.

The batch axis is preserved rather than contracted. PyTorch's `bmm` requires both inputs to be 3D and requires equal batch sizes; it does not broadcast the batch dimension.

Part 2 - Mathematical form

For $\mathcal{A} \in \mathbb{R}^{B\times m\times n}$ and $\mathcal{B} \in \mathbb{R}^{B\times n\times p}$:

$$C_{bij} = \sum_{k=1}^{n} A_{bik} B_{bkj}$$

$$\mathcal{C} \in \mathbb{R}^{B\times m\times p}$$

Part 3 - Hints and Helper

Use `torch.bmm()` and verify rank, batch size, and inner dimensions. Compare one output slice with an ordinary matrix multiplication of the corresponding input slices.

Part 4 - Python code skeleton

```
import torch

def E04(A, B):
    C = torch.bmm(...)
    return C
```

```
def main():
    A = torch...
    B = torch...
    print(E04(...))

if __name__ == "__main__":
    main()
```

2.4.6 Exercise 05 - High-Dimensional matmul Challenge

Part 1 - Why this matters

High-dimensional matrix multiplication treats the final two dimensions as matrices and the earlier dimensions as batch structure. This behaviour supports multi-head attention, ensembles, time-dependent transformations, and nested batches without manual loops.

The leading dimensions follow broadcasting rules, while the final inner matrix dimensions must still match exactly. Predicting the result therefore requires combining broadcast reasoning with ordinary matrix shape reasoning.

Part 2 - Mathematical form

For shapes:

$$\mathcal{A} : (a_1, \ldots, a_r, m, n), \qquad \mathcal{B} : (b_1, \ldots, b_s, n, p)$$

the leading shapes are broadcast to $\boldsymbol{\beta}$, producing:

$$\text{shape}(\text{matmul}(\mathcal{A}, \mathcal{B})) = (\boldsymbol{\beta}, m, p)$$

Part 3 - Hints and Helper

Compare leading shapes from right to left, then check the final inner dimensions separately. Use `torch.broadcast_shapes()` to inspect batch compatibility and `torch.matmul()` for the final operation.

Part 4 - Python code skeleton

```
import torch

def E05(A, B):
    batch_shape = torch.broadcast_shapes(...)
    predicted_shape = (...)
    result = torch.matmul(...)
    return predicted_shape, result

def main():
    A = torch...
    B = torch...
    print(E05(...))

if __name__ == "__main__":
    main()
```

2.4.7 Exercise 06 - Performance Comparison Challenge

Part 1 - Why this matters

Optimized matrix multiplication uses highly tuned kernels and parallel hardware, while Python loops introduce interpreter overhead and prevent efficient use of the GPU. Measuring this difference demonstrates why vectorized linear algebra is essential for artificial intelligence workloads.

Reliable benchmarking requires more than reading the clock once. Startup costs, GPU asynchrony, caching, matrix size, dtype, warm-up, and repeated trials can all distort a result if the experiment is not designed carefully.

Part 2 - Mathematical form

If T_{manual} and $T_{optimized}$ are measured execution times, speedup is:

$$S = \frac{T_{manual}}{T_{optimized}}$$

Both implementations must compute equivalent outputs before performance is compared.

Part 3 - Hints and Helper

Warm up each method, repeat measurements, and compare a robust statistic such as the median. Synchronize CUDA before timing boundaries. Validate outputs with `torch.allclose()` and avoid including tensor creation in only one method.

Part 4 - Python code skeleton

```
import time
import torch

def E06(A, B):
    if A.is_cuda:
        torch.cuda.synchronize()
    start = time.perf_counter()
    optimized = ...
    if A.is_cuda:
        torch.cuda.synchronize()
    optimized_time = ...
    return optimized, optimized_time

def main():
    A = torch...
    B = torch...
    print(E06(...))

if __name__ == "__main__":
    main()
```

2.4.8 Exercise 07 - Matrix Multiplication Debugging

Part 1 - Why this matters

Matrix multiplication failures often originate from incorrect shape, axis order, dtype, or device rather than the multiplication function itself. A clear debugging process isolates these conditions before changing the mathematics.

Some bugs do not raise exceptions. Accidentally transposing the wrong matrix or broadcasting an unintended batch axis can produce a valid result with incorrect meaning, so debugging must include expected output shape and numerical tests.

Part 2 - Mathematical form

For $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{q \times p}$, the diagnostic condition is:

$$n = q$$

If a transpose is intended, its shape effect must be written explicitly before checking compatibility.

Part 3 - Hints and Helper

Print shape, ndim, dtype, and device immediately before multiplication. State the expected mathematical shapes, identify which axis is wrong, then make the smallest justified correction. Do not apply random transposes until the code runs.

Part 4 - Python code skeleton

```
import torch

def E07(A, B):
    print(A.shape, A.dtype, A.device)
    print(B.shape, B.dtype, B.device)
    fixed_A = ...
    fixed_B = ...
    result = torch.matmul(...)
    return result

def main():
    A = torch...
    B = torch...
    print(E07(...))

if __name__ == "__main__":
    main()
```

2.4.9 Exercise 08 - Numerical Precision Challenge

Part 1 - Why this matters

Matrix multiplication accumulates many products, so rounding error can grow with matrix size and value scale. Different dtypes balance precision, memory, speed, and hardware support, and may produce slightly different results.

Lower precision can accelerate neural-network training but is more vulnerable to overflow, underflow, and loss of small contributions. Comparing results requires tolerances rather than exact equality and a higher-precision reference when practical.

Part 2 - Mathematical form

For approximate result $\widehat{\mathbf{C}}$ and reference $\mathbf{C}$:

$$E_{abs} = \|\widehat{\mathbf{C}} - \mathbf{C}\|_\infty$$

$$E_{rel} = \frac{\|\widehat{\mathbf{C}} - \mathbf{C}\|_\infty}{\|\mathbf{C}\|_\infty + \varepsilon}$$

Part 3 - Hints and Helper

Compute the same product in multiple floating dtypes and compare against a `float64` reference where supported. Use `torch.allclose()` with explicit tolerances and report absolute and relative error, not only pass or fail.

Part 4 - Python code skeleton

```
import torch

def E08(A, B):
    reference = torch.matmul(A.to(...), B.to(...))
    lower_precision = torch.matmul(A.to(...), B.to(...))
    absolute_error = ...
    relative_error = ...
    return absolute_error, relative_error

def main():
    A = torch...
    B = torch...
    print(E08(...))

if __name__ == "__main__":
    main()
```

2.4.10 Exercise 09 - Custom Linear Layer Implementation

Part 1 - Why this matters

A linear neural-network layer is matrix multiplication combined with a broadcasted bias. Implementing one manually connects matrix-shape rules to model parameters, batching, initialization, and automatic differentiation.

A correct custom layer must register its weights and bias as parameters so optimizers can discover them. It must also transform the final feature axis and preserve any leading batch dimensions.

Part 2 - Mathematical form

For input $\mathbf{X} \in \mathbb{R}^{B \times n}$, weight $\mathbf{W} \in \mathbb{R}^{m \times n}$, and bias $\mathbf{b} \in \mathbb{R}^{m}$:

$$\mathbf{Y} = \mathbf{X}\mathbf{W}^{\mathsf{T}} + \mathbf{b} \in \mathbb{R}^{B \times m}$$

Part 3 - Hints and Helper

Subclass `torch.nn.Module`, store trainable tensors with `nn.Parameter`, and define `forward`. Initialize parameters with a scale appropriate to input width. Compare output and gradients with `nn.Linear` configured with identical parameters.

Part 4 - Python code skeleton

```
import torch
import torch.nn as nn

class CustomLinear(nn.Module):
    def __init__(self, in_features, out_features):
        super().__init__()
        self.weight = nn.Parameter(...)
        self.bias = nn.Parameter(...)

    def forward(self, x):
        return ...

def E09(x, in_features, out_features):
    layer = CustomLinear(...)
    return layer(...)

def main():
    x = torch...
    print(E09(...))

if __name__ == "__main__":
    main()
```

2.5 ALA03 - Outer vs Inner Product

Mathematics and Theory

2.5.1 Exercise 00 - Inner Product Coding Challenge

Part 1 - Why this matters

The inner product compresses two aligned vectors into one scalar that measures their weighted alignment. It appears in similarity scores, linear models, attention, projections, and the calculation of angles between vectors.

Coding the operation requires more than multiplying values. The vectors must share a compatible dimension, dtype, and device, and the implementation should preserve automatic differentiation when the inputs participate in a neural network.

Part 2 - Mathematical form

For $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$:

$$\langle \mathbf{x}, \mathbf{y} \rangle = \sum_{i=1}^{n} x_i y_i = \mathbf{x}^\mathsf{T} \mathbf{y} \in \mathbb{R}$$

Part 3 - Hints and Helper

Implement the calculation using aligned multiplication and reduction, then compare it with `torch.dot()`. Check that the result is scalar and that gradients can flow to both vectors.

Part 4 - Python code skeleton

```
import torch

def E00(x, y):
    products = ...
    inner_product = torch.sum(...)
    return inner_product

def main():
    x = torch...
    y = torch...
    print(E00(...))

if __name__ == "__main__":
    main()
```

2.5.2 Exercise 01 - Outer Product Coding Challenge

Part 1 - Why this matters

The outer product records every pairwise interaction between two vectors. It creates a matrix rather than a scalar and is useful for feature interactions, rank-one updates, covariance-like constructions, and tensor factorization.

Because its output contains one entry for every pair of input components, its memory cost grows as the product of vector lengths. The operation preserves all pairwise information instead of summing it away.

Part 2 - Mathematical form

For $\mathbf{x} \in \mathbb{R}^m$ and $\mathbf{y} \in \mathbb{R}^n$:

$$\mathbf{M} = \mathbf{x}\mathbf{y}^\mathsf{T} \in \mathbb{R}^{m \times n}$$

$$M_{ij} = x_i y_j$$

Part 3 - Hints and Helper

Use `torch.outer()` or reshape the vectors to $(m, 1)$ and $(1, n)$ before broadcasted multiplication. Verify that the output axes correspond to the first and second vectors in that order.

Part 4 - Python code skeleton

```
import torch

def E01(x, y):
    outer_product = torch...
    return outer_product

def main():
    x = torch...
    y = torch...
    print(E01(...))

if __name__ == "__main__":
    main()
```

2.5.3 Exercise 02 - Shape Interpretation Challenge

Part 1 - Why this matters

Inner and outer products use the same two vectors but produce fundamentally different shapes. Correct shape interpretation prevents a scalar similarity operation from being confused with a matrix of pairwise interactions.

Shape also reveals what information an operation preserves. The inner product contracts the shared coordinate axis, while the outer product introduces two independent axes and performs no reduction.

Part 2 - Mathematical form

For $\mathbf{x} \in \mathbb{R}^m$ and $\mathbf{y} \in \mathbb{R}^n$:

$$\text{shape}(\mathbf{x}\mathbf{y}^\mathsf{T}) = (m, n)$$

The inner product is defined only when $m = n$ and has scalar shape:

$$\text{shape}(\mathbf{x}^\mathsf{T}\mathbf{y}) = ()$$

Part 3 - Hints and Helper

Write a function that predicts validity and output shape before computing either product. Then compare the prediction with PyTorch. Remember that a scalar tensor has zero dimensions even though it contains one element.

Part 4 - Python code skeleton

```
import torch

def E02(x, y):
    inner_valid = ...
    inner_shape = ...
    outer_shape = (..., ...)
    return inner_valid, inner_shape, outer_shape

def main():
    x = torch...
    y = torch...
    print(E02(...))

if __name__ == "__main__":
    main()
```

2.5.4 Exercise 03 - Rank Analysis Challenge

Part 1 - Why this matters

An outer product creates a structured matrix whose columns are scaled versions of one vector. This means the matrix carries pairwise values but only one independent column direction when both vectors are nonzero.

Rank analysis connects a coding operation to linear dependence and low-rank models. It explains why sums of a small number of outer products can represent compressed matrices and why one outer product cannot describe an arbitrary full-rank matrix.

Part 2 - Mathematical form

For $\mathbf{M} = \mathbf{x}\mathbf{y}^\mathsf{T}$:

$$\operatorname{rank}(\mathbf{M}) \leq 1$$

More precisely:

$$\operatorname{rank}(\mathbf{M}) = \begin{cases} 0, & \mathbf{x} = \mathbf{0} \text{ or } \mathbf{y} = \mathbf{0}, \\ 1, & \text{otherwise.} \end{cases}$$

Part 3 - Hints and Helper

Construct outer products for nonzero and zero vectors, then use `torch.linalg.matrix_rank()`. Use floating-point tensors and remember that numerical rank depends on tolerance for nearly zero values.

Part 4 - Python code skeleton

```
import torch

def E03(x, y):
    matrix = torch.outer(...)
    rank = torch.linalg.matrix_rank(...)
```

```
    return matrix, rank

def main():
    x = torch...
    y = torch...
    print(E03(...))

if __name__ == "__main__":
    main()
```

2.5.5 Exercise 04 - Feature Interaction Matrix Design

Part 1 - Why this matters

A feature interaction matrix represents how every component of one feature vector combines with every component of another. Such matrices can expose cross-feature patterns that a single inner-product score would hide.

Designing the matrix requires deciding whether interactions should be computed per sample, averaged across a batch, normalized, or weighted. These choices change the interpretation even when the underlying operation remains an outer product.

Part 2 - Mathematical form

For one sample:

$$\mathbf{I} = \mathbf{x}\mathbf{y}^{\mathsf{T}}, \qquad I_{ij} = x_i y_j$$

For batches $\mathbf{X} \in \mathbb{R}^{B\times m}$ and $\mathbf{Y} \in \mathbb{R}^{B\times n}$, a mean interaction matrix is:

$$\bar{\mathbf{I}} = \frac{1}{B}\sum_{b=1}^{B} \mathbf{x}_b \mathbf{y}_b^{\mathsf{T}}$$

Part 3 - Hints and Helper

Use broadcasting, `torch.einsum()`, or batched matrix multiplication. Keep the batch axis until the final aggregation and verify whether the desired output is (B, m, n) or (m, n).

Part 4 - Python code skeleton

```
import torch

def E04(X, Y):
    batch_interactions = ...
    mean_interaction = torch.mean(..., dim=...)
    return mean_interaction

def main():
    X = torch...
    Y = torch...
    print(E04(...))

if __name__ == "__main__":
    main()
```

2.5.6 Exercise 05 - Memory Cost Comparison Challenge

Part 1 - Why this matters

An inner product stores one output value, while an outer product stores every pairwise interaction. For long vectors or large batches, this difference can turn a small computation into a major memory allocation.

Memory reasoning must include dtype size as well as element count. A matrix with the same shape consumes different bytes in `float16`, `float32`, and `float64`, and intermediate tensors may increase peak memory beyond the final output size.

Part 2 - Mathematical form

For lengths m and n and b bytes per element:

$$M_{inner} = b$$

$$M_{outer} = mnb$$

For batch size B, explicit batched outer products require $Bmnb$ bytes.

Part 3 - Hints and Helper

Use `tensor.numel() * tensor.element_size()` to calculate storage bytes. Compare predicted and actual output storage for several sizes and dtypes. Do not confuse storage size with the complete runtime's peak memory use.

Part 4 - Python code skeleton

```
import torch

def E05(x, y):
    inner = ...
    outer = ...
    inner_bytes = ...
    outer_bytes = ...
    return inner_bytes, outer_bytes

def main():
    x = torch...
    y = torch...
    print(E05(...))

if __name__ == "__main__":
    main()
```

2.5.7 Exercise 06 - Use-Case Driven Product Selection

Part 1 - Why this matters

Selecting an inner or outer product should follow the information required by the application. A similarity score, projection coefficient, or attention logit normally needs contraction

to a scalar, while a complete interaction map or rank-one matrix construction needs an outer product.

The wrong choice may still produce valid code but an unusable shape or excessive memory cost. A disciplined design process predicts the desired output, explains which indices should remain, and only then selects the PyTorch operation.

Part 2 - Mathematical form

Choose the inner product when the required output is:

$$s = \sum_i x_i y_i$$

Choose the outer product when the required output is:

$$M_{ij} = x_i y_j$$

The first contracts an aligned index; the second preserves both indices.

Part 3 - Hints and Helper

Write a function that accepts a use-case label and validates the input shapes before selecting `torch.dot()` or `torch.outer()`. Return both the result and a short reason for the selected operation.

Part 4 - Python code skeleton

```
import torch

def E06(x, y, use_case):
    if use_case == "similarity":
        result = ...
        reason = ...
    elif use_case == "interaction_matrix":
        result = ...
        reason = ...
    else:
        raise ValueError(...)
    return result, reason

def main():
    x = torch...
    y = torch...
    print(E06(...))

if __name__ == "__main__":
    main()
```

2.6 ALA04 - Broadcasting Rules & Mechanics

Mathematics and Theory

2.6.1 Exercise 00 - Simple Broadcasting Addition

Part 1 - Why this matters

Artificial intelligence programs frequently add information stored at different levels of detail. A matrix may contain a batch of feature vectors, while a single vector may contain a bias or offset shared by every example. Broadcasting makes this addition concise by matching the shared vector to every compatible row.

Understanding this mechanism is important because the short expression hides a precise shape transformation. Predicting the resulting dimensions before running the code helps prevent silent semantic mistakes and prepares the reader for bias addition in linear layers and other neural network operations.

Part 2 - Mathematical form

Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{b} \in \mathbb{R}^{n}$. Broadcasting treats $\mathbf{b}$ as if its row were available for every row of $\mathbf{A}$.

Define:

$$\mathbf{C} = \mathbf{A} + \mathbf{b}, \qquad \mathbf{C} \in \mathbb{R}^{m \times n}.$$

Element-wise:

$$C_{ij} = A_{ij} + b_j, \qquad 1 \leq i \leq m,\ 1 \leq j \leq n.$$

The dimension n matches directly, and the missing leading dimension of $\mathbf{b}$ is treated as a broadcastable dimension of size one.

Part 3 - Hints and Helper

In PyTorch, the addition operator and `torch.add()` both apply broadcasting automatically. Inspect `A.shape`, `b.shape`, and the expected output shape before performing the operation. Do not use `repeat()` merely to copy the vector into multiple rows; broadcasting can represent the same computation without explicitly constructing those copies.

Part 4 - Python code skeleton

```
import torch

def E00(A, b):
    C = torch...
    return C

def main():
    A = torch...
    b = torch...
    print(E00(A, b))

if __name__ == "__main__":
    main()
```

2.6.2 Exercise 01 - Broadcasting Multiplication

Part 1 - Why this matters

Feature maps often require a different scale for every channel while sharing that scale across all samples and spatial positions. Broadcasting multiplication expresses this operation directly, avoiding the need to manually construct a full-sized tensor of repeated channel values.

This pattern appears in normalization, attention, calibration, and learned gating mechanisms. Correctly positioning singleton dimensions is essential: the same numbers can produce a valid but conceptually incorrect result when their axes do not represent the intended batch, channel, height, and width roles.

Part 2 - Mathematical form

Let $\mathcal{X} \in \mathbb{R}^{B\times C\times H\times W}$ and let $\mathbf{s} \in \mathbb{R}^{C}$. Reshape the scale conceptually as $\tilde{\mathbf{s}} \in \mathbb{R}^{1\times C\times 1\times 1}$.

Define:

$$\mathcal{Y} = \mathcal{X} \odot \tilde{\mathbf{s}}, \qquad \mathcal{Y} \in \mathbb{R}^{B\times C\times H\times W}.$$

Element-wise:

$$Y_{bchw} = X_{bchw} s_c.$$

The singleton axes of $\tilde{\mathbf{s}}$ broadcast across the batch and spatial dimensions, while its channel axis matches C.

Part 3 - Hints and Helper

Use `unsqueeze()` or indexing with `None` to place the channel dimension correctly, then apply `torch.mul()` or `*`. Verify that the scale has conceptual shape `(1, C, 1, 1)` rather than relying only on the fact that an operation happens to run.

Part 4 - Python code skeleton

```
import torch

def E01(X, scale):
    scale = scale...
    Y = torch...
    return Y

def main():
    X = torch...
    scale = torch...
    print(E01(X, scale))

if __name__ == "__main__":
    main()
```

2.6.3 Exercise 02 - Broadcasting with Scalars

Part 1 - Why this matters

Scalar values are used throughout numerical computing to shift, scale, clip, or normalize every entry of a tensor consistently. Broadcasting allows a single number to interact with a tensor of any shape, making mathematical expressions both compact and close to their theoretical notation.

Although scalar broadcasting is the simplest broadcasting case, it establishes an important mental model: an operand does not need to be physically copied to match the other tensor. This becomes especially useful when interpreting learning rates, temperature parameters, regularization coefficients, and normalization constants in machine learning code.

Part 2 - Mathematical form

Let $\mathcal{X} \in \mathbb{R}^{d_1 \times \cdots \times d_k}$ and let $\alpha \in \mathbb{R}$ be a scalar.
Define a scaled and shifted tensor:

$$\mathcal{Y} = \alpha\mathcal{X} + \alpha.$$

For every valid multi-index $(i_1, \ldots, i_k)$:

$$Y_{i_1 \ldots i_k} = \alpha X_{i_1 \ldots i_k} + \alpha.$$

The output retains the shape of $\mathcal{X}$ because the scalar is compatible with every tensor dimension.

Part 3 - Hints and Helper

Create a tensor with two or more dimensions and a Python scalar or zero-dimensional PyTorch tensor. Apply ordinary arithmetic operators, then compare the input and output shapes. If the scalar is stored as a tensor, also consider whether its `dtype` and `device` are compatible with the input.

Part 4 - Python code skeleton

```
import torch

def E02(X, alpha):
    Y = ...
    return Y

def main():
    X = torch...
    alpha = ...
    print(E02(X, alpha))

if __name__ == "__main__":
    main()
```

2.6.4 Exercise 03 - Broadcasting Across Dimensions

Part 1 - Why this matters

Real tensor programs commonly combine operands with several independent roles, such as batches, channels, rows, and columns. Broadcasting across multiple axes allows each operand to vary only along the dimensions it represents while being shared across the remaining dimensions.

Mastering this operation reduces unnecessary data duplication and makes complex shape flows easier to express. It also requires disciplined dimension reasoning, because broadcasting compares sizes from the trailing dimension toward the leading dimension rather than matching axes according to their names.

Part 2 - Mathematical form

Let $\mathcal{A} \in \mathbb{R}^{B\times 1\times H\times W}$ and $\mathcal{B} \in \mathbb{R}^{1\times C\times 1\times 1}$.

Define:

$$\mathcal{C} = \mathcal{A} + \mathcal{B}, \qquad \mathcal{C} \in \mathbb{R}^{B\times C\times H\times W}.$$

Element-wise:

$$C_{bchw} = A_{b1hw} + B_{1c11}.$$

In general, two axes are compatible when their sizes are equal or when at least one size is 1. Missing leading axes are treated as axes of size 1.

Part 3 - Hints and Helper

Write both shapes with their trailing dimensions aligned. Compare each pair from right to left, selecting the non-singleton size for the output. Use `torch.broadcast_shapes()` to check your predicted shape, then perform the arithmetic operation and verify the result.

Part 4 - Python code skeleton

```
import torch

def E03(A, B):
    expected_shape = torch...
    C = ...
    return expected_shape, C

def main():
    A = torch...
    B = torch...
    print(E03(A, B))

if __name__ == "__main__":
    main()
```

2.6.5 Exercise 04 - Invalid Broadcasting Detection

Part 1 - Why this matters

Not every pair of tensor shapes can be combined through broadcasting. Detecting an incompatible axis before performing an operation produces clearer diagnostics than allowing a low-level runtime error to appear deep inside a data pipeline or neural network.

A compatibility checker also strengthens shape reasoning by separating the rule from the computation itself. The reader must align dimensions, account for missing leading axes, and identify the exact axis where two non-singleton sizes conflict.

Part 2 - Mathematical form

Let the aligned shapes of two tensors be $(a_1, \ldots, a_k)$ and $(b_1, \ldots, b_k)$, after padding the shorter shape with leading ones.

Broadcasting is valid if and only if:

$$\forall r \in \{1, \ldots, k\}, \qquad a_r = b_r \ \vee \ a_r = 1 \ \vee \ b_r = 1.$$

When valid, the output size along axis r is:

$$c_r = \max(a_r, b_r).$$

If any aligned axis fails the compatibility condition, the complete operation is invalid.

Part 3 - Hints and Helper

Read the two shape tuples and left-pad the shorter one with ones. Examine the paired sizes without carrying out tensor arithmetic. Return both a Boolean result and useful information about the conflicting axis. You may compare your custom logic with `torch.broadcast_shapes()`, which raises an error for incompatible shapes.

Part 4 - Python code skeleton

```
import torch

def E04(shape_a, shape_b):
    aligned_a = ...
    aligned_b = ...
    is_valid = ...
    return is_valid

def main():
    shape_a = (...)
    shape_b = (...)
    print(E04(shape_a, shape_b))

if __name__ == "__main__":
    main()
```

2.6.6 Exercise 05 - Broadcasting Debugging Challenge

Part 1 - Why this matters

Broadcasting errors often originate from a tensor whose values are correct but whose dimensions are arranged incorrectly. A feature vector intended for the channel axis may accidentally align with width, or an omitted singleton dimension may cause an otherwise meaningful operation to fail.

Effective debugging therefore requires more than changing shapes until the code runs. The programmer must identify the semantic role of every axis, predict the desired output, and introduce singleton dimensions only where the data is meant to be shared.

Part 2 - Mathematical form

Suppose $\mathcal{X} \in \mathbb{R}^{B\times C\times H\times W}$ and a bias $\mathbf{b} \in \mathbb{R}^{C}$ must be added along the channel axis.

The direct shapes

$$(B, C, H, W) \quad \text{and} \quad (C)$$

compare C with W at the trailing axis and may be invalid or semantically wrong. The intended bias shape is:

$$\widetilde{\mathbf{b}} \in \mathbb{R}^{1\times C\times 1\times 1}.$$

Then:

$$Y_{bchw} = X_{bchw} + b_c.$$

Part 3 - Hints and Helper

Print the input shapes and label their conceptual axes. Align them from the right, locate the first conflicting or misplaced dimension, and use `unsqueeze()`, `reshape()`, or indexed `None` dimensions to express the intended sharing. Avoid solving the error with arbitrary `repeat()` calls.

Part 4 - Python code skeleton

```
import torch

def E05(X, bias):
    print("X shape:", ...)
    print("bias shape:", ...)
    corrected_bias = bias...
    Y = ...
    return Y

def main():
    X = torch...
    bias = torch...
    print(E05(X, bias))

if __name__ == "__main__":
    main()
```

2.6.7 Exercise 06 - Memory-efficient Broadcasting

Part 1 - Why this matters

Large tensors can make explicit repetition expensive in both memory and execution time. Broadcasting and expanded views allow one set of values to participate in a larger computation without first allocating storage for every repeated copy.

This distinction becomes significant in high-resolution feature maps and large batches. Learning when to rely on implicit broadcasting or `expand()`, and when actual independent copies from `repeat()` are necessary, helps produce scalable and predictable tensor programs.

Part 2 - Mathematical form

Let $\mathbf{b} \in \mathbb{R}^C$ and suppose it is shared across a batch of size B. The logical expanded matrix $\mathbf{B} \in \mathbb{R}^{B \times C}$ satisfies:

$$B_{ij} = b_j.$$

Explicit repetition stores BC values, while a broadcasted or expanded view can reference the original C values through its stride information. For $\mathbf{X} \in \mathbb{R}^{B \times C}$:

$$Y_{ij} = X_{ij} + b_j$$

can therefore be computed without materializing B copies of $\mathbf{b}$.

Part 3 - Hints and Helper

Construct equivalent operands using `expand()` and `repeat()`. Compare their shapes, element counts, storage relationships, and strides. Remember that an expanded view can contain zero strides and should not be modified in place when multiple logical entries refer to the same stored value.

Part 4 - Python code skeleton

```
import torch

def E06(X, bias):
    expanded = bias...
    repeated = bias...
    Y = ...
    return Y, expanded, repeated

def main():
    X = torch...
    bias = torch...
    result = E06(X, bias)
    print(result)

if __name__ == "__main__":
    main()
```

2.6.8 Exercise 07 - Broadcasting in Neural Layers

Part 1 - Why this matters

Neural layers routinely apply one learned parameter across many observations. After a linear transformation, a bias vector is added to every item in a batch; in convolutional networks, one bias value may be shared across all spatial locations of its channel. Broadcasting expresses this parameter sharing directly.

Implementing the pattern manually reveals how a layer's algebra maps to tensor shapes. It also makes clear why the bias contains one value per output feature rather than one value per sample, helping the reader distinguish learned model parameters from temporary expanded views.

Part 2 - Mathematical form

Let $\mathbf{X} \in \mathbb{R}^{B \times d_{\text{in}}}$, $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, and $\mathbf{b} \in \mathbb{R}^{d_{\text{out}}}$.
A linear layer is defined as:

$$\mathbf{Y} = \mathbf{X}\mathbf{W}^{\mathsf{T}} + \mathbf{b}, \qquad \mathbf{Y} \in \mathbb{R}^{B \times d_{\text{out}}}.$$

Element-wise:

$$Y_{ij} = \sum_{k=1}^{d_{\text{in}}} X_{ik} W_{jk} + b_j.$$

Broadcasting shares b_j across the batch axis while preserving a distinct bias for every output feature.

Part 3 - Hints and Helper

First compute the matrix multiplication with `torch.matmul()` or the `@` operator. Confirm that its final dimension equals the length of the bias vector, then add the bias directly and let PyTorch broadcast it across the batch. Compare the result shape with the prediction from the mathematical form.

Part 4 - Python code skeleton

```
import torch

def E07(X, W, bias):
    linear_output = torch...
    Y = ...
    return Y

def main():
    X = torch...
    W = torch...
    bias = torch...
    print(E07(X, W, bias))

if __name__ == "__main__":
    main()
```

2.7 ALA05 - Shape Reasoning & Dimension Flow

Mathematics and Theory

2.7.1 Exercise 00 - Shape Tracking Basics

Part 1 - Why this matters

Tensor values cannot be interpreted correctly without knowing how their axes are organized. A shape describes the number of dimensions and the size of every dimension, allowing the programmer to distinguish batches, features, channels, rows, columns, and other structural roles.

Shape tracking is therefore a basic debugging and design skill in artificial intelligence programming. Predicting the shape before and after each operation helps expose incorrect assumptions early and makes tensor code easier to explain, test, and extend.

Part 2 - Mathematical form

Let $\mathcal{X}$ be a tensor with shape:

$$\text{shape}(\mathcal{X}) = (d_1, d_2, \ldots, d_k).$$

The number of dimensions is:

$$\dim(\mathcal{X}) = k,$$

and the total number of elements is:

$$\text{numel}(\mathcal{X}) = \prod_{i=1}^{k} d_i.$$

If $\mathcal{X} \in \mathbb{R}^{B\times C\times H\times W}$, its axes may represent batch, channel, height, and width respectively.

Part 3 - Hints and Helper

In PyTorch, inspect `tensor.shape`, `tensor.dim()`, and `tensor.numel()`. Record the meaning of each axis rather than reporting only its numeric size. Verify manually that multiplying all shape dimensions produces the same element count returned by PyTorch.

Part 4 - Python code skeleton

```
import torch

def E00(X):
    shape = X...
    dimensions = X...
    elements = X...
    return shape, dimensions, elements

def main():
    X = torch...
    print(E00(X))

if __name__ == "__main__":
    main()
```

2.7.2 Exercise 01 - Multi-step Shape Prediction

Part 1 - Why this matters

Neural network programs transform tensor shapes through several consecutive operations. A tensor may be transposed, reshaped, reduced, and expanded before it reaches the next layer, so understanding only one operation at a time is not enough to guarantee that the complete pipeline is valid.

Multi-step prediction trains the reader to follow dimension flow without relying on trial and error. It also reveals which transformations preserve the number of elements, which remove axes, and which change the logical interpretation of the same stored values.

Part 2 - Mathematical form

Let $\mathcal{X} \in \mathbb{R}^{B\times C\times H\times W}$. Consider the sequence:

$$\mathcal{X}_1 = \text{permute}(\mathcal{X}, 0, 2, 3, 1),$$

$$\mathcal{X}_2 = \text{reshape}(\mathcal{X}_1, B, HW, C),$$

$$\mathcal{X}_3 = \text{mean}(\mathcal{X}_2, \text{dim} = 1).$$

The corresponding shape flow is:

$$(B, C, H, W) \to (B, H, W, C) \to (B, HW, C) \to (B, C).$$

The reshape is valid because both intermediate tensors contain $BCHW$ elements.

Part 3 - Hints and Helper

Write the expected shape after every line before running the code. Track axis meaning as well as axis size, and check the element count before each reshape. Remember that a reduction removes its selected dimension unless `keepdim=True` is used.

Part 4 - Python code skeleton

```
import torch

def E01(X):
    X1 = X...
    X2 = X1...
    X3 = X2...
    return X1.shape, X2.shape, X3.shape

def main():
    X = torch...
    print(E01(X))

if __name__ == "__main__":
    main()
```

2.7.3 Exercise 02 - Shape Mismatch Debugging

Part 1 - Why this matters

Many tensor failures are caused by incompatible dimensions rather than incorrect numerical formulas. Matrix multiplication, concatenation, broadcasting, and neural layers each impose different shape requirements, and an error message may identify sizes without explaining their intended meanings.

Systematic shape debugging replaces random reshaping with evidence. By examining the operation's mathematical contract and tracing the origin of each axis, the programmer can correct the actual structural mistake without silently changing the data interpretation.

Part 2 - Mathematical form

Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{q \times p}$. The product $\mathbf{AB}$ is defined only when:

$$n = q.$$

When the condition holds:

$$\mathbf{C} = \mathbf{AB} \in \mathbb{R}^{m \times p}, \qquad C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}.$$

If $n \neq q$, the mismatch must be corrected at the source or through a mathematically justified transformation such as transposition.

Part 3 - Hints and Helper

Print both shapes and state which dimensions the operation expects to match. Create a compatibility check before calling `torch.matmul()`. Do not use `reshape()` merely to force matching sizes; first determine whether the second tensor has been constructed or oriented incorrectly.

Part 4 - Python code skeleton

```
import torch

def E02(A, B):
    compatible = ...
    if not compatible:
        return ...
    C = torch...
    return C

def main():
    A = torch...
    B = torch...
    print(E02(A, B))

if __name__ == "__main__":
    main()
```

2.7.4 Exercise 03 - Dynamic Shape Handling

Part 1 - Why this matters

Production models often receive batches whose size changes between calls, and image or sequence dimensions may also vary. Code that assumes fixed numeric sizes can work for one example yet fail on the final smaller batch or on data from a different source.

Dynamic shape handling makes tensor functions reusable across valid inputs. The key is to derive new dimensions from the current tensor while preserving required relationships, rather than embedding constants that describe only the original test case.

Part 2 - Mathematical form

Let $\mathcal{X} \in \mathbb{R}^{B\times C\times H\times W}$, where B, H, and W may vary between calls. Flatten the non-batch dimensions:

$$\mathbf{Y} = \text{reshape}(\mathcal{X}, B, CHW), \qquad \mathbf{Y} \in \mathbb{R}^{B\times CHW}.$$

Equivalently, the second dimension can be inferred from the invariant element count:

$$\text{numel}(\mathbf{Y}) = \text{numel}(\mathcal{X}) = BCHW.$$

The batch dimension remains independent for every valid positive value of B.

Part 3 - Hints and Helper

Read the batch size from `X.shape[0]` and let PyTorch infer one reshape dimension using `-1`. Test the function with several batch and spatial sizes. Confirm that the output always retains the original batch size and that its element count remains unchanged.

Part 4 - Python code skeleton

```
import torch

def E03(X):
    batch_size = X.shape[...]
    Y = X...
    return Y

def main():
    X1 = torch...
    X2 = torch...
    print(E03(X1).shape)
    print(E03(X2).shape)

if __name__ == "__main__":
    main()
```

2.7.5 Exercise 04 - Shape-safe Function Design

Part 1 - Why this matters

A tensor function should communicate and enforce the shapes it accepts. Without clear checks, an invalid input may fail much later or, more dangerously, produce a valid output whose axes carry the wrong meaning through accidental broadcasting or an unintended reduction.

Shape-safe design turns structural expectations into explicit contracts. Helpful validation messages make functions easier to reuse, while preservation checks ensure that an operation does not lose or mix information unexpectedly.

Part 2 - Mathematical form

Suppose a function normalizes a batch of feature vectors $\mathbf{X} \in \mathbb{R}^{B \times F}$. Its domain requires:

$$\dim(\mathbf{X}) = 2, \qquad B > 0, \qquad F > 0.$$

For each row, define:

$$Y_{ij} = \frac{X_{ij}}{\sqrt{\sum_{k=1}^{F} X_{ik}^2 + \varepsilon}}, \qquad \varepsilon > 0.$$

The function must preserve the shape:

$$\text{shape}(\mathbf{Y}) = (B, F).$$

Part 3 - Hints and Helper

Validate the tensor rank and feature size before computing the result. Use an informative `ValueError` or an assertion that describes the expected and received shapes. During norm calculation, preserve the reduced axis with `keepdim=True` so division broadcasts along the intended feature axis.

Part 4 - Python code skeleton

```
import torch

def E04(X, epsilon=1e-8):
    if X.dim() ...:
        raise ValueError(...)
    norms = torch...
    Y = ...
    return Y

def main():
    X = torch...
    print(E04(X))

if __name__ == "__main__":
    main()
```

2.7.6 Exercise 05 - Model Shape Pipeline Validation

Part 1 - Why this matters

A model is a sequence of shape contracts. The output of preprocessing becomes the input of a layer, the layer output feeds an activation or reduction, and a final representation must match the prediction head. One incorrect transition can invalidate every later stage.

Pipeline validation records these transitions and checks them at meaningful boundaries. This provides faster debugging than inspecting only the final error and creates a compact structural description that remains useful when batch sizes or model parameters change.

Part 2 - Mathematical form

Let a batch of images be $\mathcal{X} \in \mathbb{R}^{B\times C\times H\times W}$. Consider a pipeline that flattens each image and applies two linear transformations:

$$\mathbf{X}_1 = \text{flatten}(\mathcal{X}, 1) \in \mathbb{R}^{B\times CHW},$$
$$\mathbf{X}_2 = \mathbf{X}_1\mathbf{W}_1^\mathsf{T} + \mathbf{b}_1 \in \mathbb{R}^{B\times D},$$
$$\mathbf{Y} = \mathbf{X}_2\mathbf{W}_2^\mathsf{T} + \mathbf{b}_2 \in \mathbb{R}^{B\times K}.$$

Therefore $\mathbf{W}_1 \in \mathbb{R}^{D\times CHW}$ and $\mathbf{W}_2 \in \mathbb{R}^{K\times D}$ are required.

Part 3 - Hints and Helper

Store the shape after every pipeline stage and compare it with the expected contract. Validate the input feature dimension of each weight matrix before performing multiplication. Test with more than one batch size to confirm that only the batch axis changes while all learned feature dimensions remain fixed.

Part 4 - Python code skeleton

```
import torch

def E05(X, W1, b1, W2, b2):
    shapes = [X.shape]
    X1 = X...
    shapes.append(X1.shape)
    X2 = ...
    shapes.append(X2.shape)
    Y = ...
    shapes.append(Y.shape)
    return Y, shapes

def main():
    X = torch...
    W1, b1 = torch..., torch...
    W2, b2 = torch..., torch...
    print(E05(X, W1, b1, W2, b2))

if __name__ == "__main__":
    main()
```

2.8 ALA06 - Basis, Span & Projection (Applied)

Mathematics and Theory

2.8.1 Exercise 00 - Vector Projection Implementation

Part 1 - Why this matters

Vector projection extracts the part of one vector that points in the direction of another. In artificial intelligence and data analysis, this operation helps measure directional agreement, isolate a chosen component, and express data in terms of meaningful reference directions.

Implementing projection directly connects the geometric idea with tensor code. It also highlights an important numerical requirement: the direction vector must not have zero length, because the projection formula divides by its squared norm.

Part 2 - Mathematical form

Let $\mathbf{x}, \mathbf{v} \in \mathbb{R}^n$, where $\mathbf{v} \neq \mathbf{0}$. The projection of $\mathbf{x}$ onto $\mathbf{v}$ is:

$$\text{proj}_{\mathbf{v}}(\mathbf{x}) = \frac{\mathbf{x}^\mathsf{T}\mathbf{v}}{\mathbf{v}^\mathsf{T}\mathbf{v}}\mathbf{v}.$$

The scalar projection coefficient is:

$$\alpha = \frac{\mathbf{x}^\mathsf{T}\mathbf{v}}{\|\mathbf{v}\|_2^2},$$

so the projected vector is $\alpha\mathbf{v} \in \mathbb{R}^n$.

Part 3 - Hints and Helper

Use `torch.dot()` for the numerator and denominator. Check that both inputs are one-dimensional, have equal lengths, and that the denominator is not zero. Compute the scalar coefficient first, then multiply it by the direction vector while preserving its shape.

Part 4 - Python code skeleton

```
import torch

def E00(x, v):
    numerator = torch...
    denominator = torch...
    projection = ...
    return projection

def main():
    x = torch...
    v = torch...
    print(E00(x, v))

if __name__ == "__main__":
    main()
```

2.8.2 Exercise 01 - Orthogonal Projection

Part 1 - Why this matters

An orthogonal projection separates a vector into a component inside a target direction and a residual perpendicular to that direction. This decomposition is useful for removing unwanted components, analyzing errors, and understanding how data relates to a chosen axis.

The perpendicularity condition provides a strong correctness test that goes beyond comparing expected numbers. If the implementation is correct, the dot product between the residual and the projection direction should be zero, apart from small floating-point error.

Part 2 - Mathematical form

Let $\mathbf{x}, \mathbf{u} \in \mathbb{R}^n$ and assume that $\|\mathbf{u}\|_2 = 1$. Define:

$$\mathbf{p} = (\mathbf{x}^\mathsf{T}\mathbf{u})\mathbf{u},$$

where $\mathbf{p}$ is the orthogonal projection of $\mathbf{x}$ onto the line spanned by $\mathbf{u}$.

The residual is:

$$\mathbf{r} = \mathbf{x} - \mathbf{p}.$$

Orthogonality requires:

$$\mathbf{u}^\mathsf{T}\mathbf{r} = 0.$$

Part 3 - Hints and Helper

Normalize the direction vector with `torch.linalg.vector_norm()` before computing the projection. Return both the projected component and the residual. Use `torch.dot()` and `torch.isclose()` to test whether the residual is orthogonal within a reasonable tolerance.

Part 4 - Python code skeleton

```
import torch

def E01(x, direction):
    unit_direction = direction / torch...
    projection = ...
    residual = ...
    orthogonality = torch...
    return projection, residual, orthogonality

def main():
    x = torch...
    direction = torch...
    print(E01(x, direction))

if __name__ == "__main__":
    main()
```

2.8.3 Exercise 02 - Projection Error Measurement

Part 1 - Why this matters

A projection is an approximation whenever the original vector does not lie entirely in the selected direction or subspace. Measuring the remaining error quantifies how much information the projection fails to represent and allows different approximation choices to be compared objectively.

Projection error appears in dimensionality reduction, least-squares fitting, signal reconstruction, and model compression. Both the residual vector and its norm are useful: one shows the direction of the missing information, while the other summarizes its magnitude as a single value.

Part 2 - Mathematical form

Let $\mathbf{x} \in \mathbb{R}^n$ and let $\hat{\mathbf{x}}$ be its projection. Define the residual:

$$\mathbf{e} = \mathbf{x} - \hat{\mathbf{x}}.$$

The absolute projection error is:

$$E = \|\mathbf{e}\|_2.$$

When $\|\mathbf{x}\|_2 \neq 0$, the relative error is:

$$E_{\text{rel}} = \frac{\|\mathbf{x} - \hat{\mathbf{x}}\|_2}{\|\mathbf{x}\|_2}.$$

Smaller values indicate that the projection preserves more of the original vector.

Part 3 - Hints and Helper

Compute the residual before calling `torch.linalg.vector_norm()`. Validate that the original and projected vectors have the same shape. Handle the zero-vector case explicitly when computing relative error, because its norm cannot be used as a denominator.

Part 4 - Python code skeleton

```
import torch

def E02(x, projected_x):
    residual = ...
    absolute_error = torch...
    relative_error = ...
    return residual, absolute_error, relative_error

def main():
    x = torch...
    projected_x = torch...
    print(E02(x, projected_x))

if __name__ == "__main__":
    main()
```

2.8.4 Exercise 03 - Subspace Projection Challenge

Part 1 - Why this matters

Projecting onto a subspace extends vector projection from one direction to several directions. It allows a high-dimensional observation to be represented using a smaller collection of basis vectors, retaining only the components that the chosen subspace can explain.

This operation is central to least-squares approximation and many representation methods. The implementation depends on whether the basis is orthonormal; using the simpler orthonormal formula on a general basis produces an incorrect result even when all tensor shapes are valid.

Part 2 - Mathematical form

Let $\mathbf{x} \in \mathbb{R}^n$ and let the columns of $\mathbf{A} \in \mathbb{R}^{n \times k}$ be linearly independent basis vectors. The projection matrix onto $\text{span}(\mathbf{A})$ is:

$$\mathbf{P} = \mathbf{A}(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}.$$

The projected vector is:

$$\hat{\mathbf{x}} = \mathbf{P}\mathbf{x}.$$

If $\mathbf{Q}^\mathsf{T}\mathbf{Q} = \mathbf{I}$, then the orthonormal case simplifies to:

$$\hat{\mathbf{x}} = \mathbf{Q}\mathbf{Q}^\mathsf{T}\mathbf{x}.$$

Part 3 - Hints and Helper

Prefer solving the coefficient system with `torch.linalg.solve()` instead of explicitly computing an inverse. Check that the basis matrix has shape `(n, k)` and full column rank. Verify that the residual is orthogonal to every basis column by examining `A.T @ residual`.

Part 4 - Python code skeleton

```
import torch

def E03(x, A):
    gram_matrix = A.T @ ...
    right_side = A.T @ ...
    coefficients = torch.linalg...
    projected_x = ...
    return projected_x

def main():
    x = torch...
    A = torch...
    print(E03(x, A))

if __name__ == "__main__":
    main()
```

2.8.5 Exercise 04 - Feature Space Reduction

Part 1 - Why this matters

Feature vectors may contain many dimensions even when most useful variation lies inside a much smaller subspace. Projecting data onto a reduced basis replaces the original coordinates with fewer coefficients, decreasing storage and computation while preserving the information represented by that basis.

The reduced coordinates and reconstructed features serve different purposes. The coordinates are the compact representation used by later computations, whereas reconstruction maps those coordinates back to the original feature space so that information loss can be inspected.

Part 2 - Mathematical form

Let $\mathbf{X} \in \mathbb{R}^{B \times D}$ contain B observations and let $\mathbf{Q} \in \mathbb{R}^{D \times K}$ contain K orthonormal basis vectors, where $K < D$ and $\mathbf{Q}^\mathsf{T}\mathbf{Q} = \mathbf{I}_K$.

The reduced feature representation is:

$$\mathbf{Z} = \mathbf{X}\mathbf{Q} \in \mathbb{R}^{B \times K}.$$

The reconstruction is:

$$\widehat{\mathbf{X}} = \mathbf{Z}\mathbf{Q}^\mathsf{T} \in \mathbb{R}^{B \times D}.$$

Thus each observation changes from D features to K coordinates.

Part 3 - Hints and Helper

Use matrix multiplication for both reduction and reconstruction. Check the basis with `Q.T @ Q` and compare it with an identity matrix using `torch.allclose()`. Print all intermediate shapes and measure the reconstruction error without assuming that it must be zero.

Part 4 - Python code skeleton

```
import torch

def E04(X, Q):
    reduced = torch...
    reconstructed = torch...
    error = ...
    return reduced, reconstructed, error

def main():
    X = torch...
    Q = torch...
    print(E04(X, Q))

if __name__ == "__main__":
    main()
```

2.8.6 Exercise 05 - Projection-based Optimization Insight

Part 1 - Why this matters

Optimization steps sometimes need to satisfy a linear constraint. An ordinary gradient may point partly outside the set of allowed directions, so applying it directly can violate the constraint even when it reduces the objective function. Projection removes the forbidden component before the parameters are updated.

This idea provides a practical link between linear algebra and constrained learning. By decomposing a gradient into admissible and inadmissible parts, the reader can see how geometric structure changes an optimization direction without changing the original loss calculation.

Part 2 - Mathematical form

Let $\mathbf{g} \in \mathbb{R}^n$ be a gradient and let the feasible directions satisfy:

$$\mathbf{a}^\mathsf{T}\mathbf{d} = 0, \qquad \mathbf{a} \neq \mathbf{0}.$$

Remove the component of $\mathbf{g}$ parallel to $\mathbf{a}$:

$$\mathbf{g}_\perp = \mathbf{g} - \frac{\mathbf{g}^\mathsf{T}\mathbf{a}}{\mathbf{a}^\mathsf{T}\mathbf{a}}\mathbf{a}.$$

For learning rate $\eta > 0$, a projected update is:

$$\boldsymbol{\theta}_{\text{new}} = \boldsymbol{\theta} - \eta\mathbf{g}_\perp.$$

The projected direction satisfies $\mathbf{a}^\mathsf{T}\mathbf{g}_\perp = 0$.

Part 3 - Hints and Helper

Compute the forbidden gradient component using the vector projection law, then subtract it from the original gradient. Verify the constraint numerically before updating the parameters. Compare the ordinary and projected update directions to observe exactly what the projection removed.

Part 4 - Python code skeleton

```
import torch

def E05(theta, gradient, constraint, learning_rate):
    forbidden_component = ...
    projected_gradient = ...
    new_theta = ...
    constraint_check = torch...
    return new_theta, projected_gradient, constraint_check

def main():
    theta = torch...
    gradient = torch...
    constraint = torch...
    learning_rate = ...
```

```
    print(E05(theta, gradient, constraint, learning_rate))

if __name__ == "__main__":
    main()
```

Book 3

Autograd & Backprop Mechanics

3.1 Introduction

3.1.1 What will the book teach you

Before building and training advanced artificial intelligence models, you must understand how gradients move through tensor operations. This book teaches the mechanics of automatic differentiation and backpropagation using PyTorch. You will begin by constructing computational graphs and distinguishing leaf tensors from non-leaf tensors, then progress toward tracing gradient flow through complete neural network operations.

The book connects mathematical differentiation with practical code. You will calculate gradients manually, verify them with Autograd, apply the chain rule, and study how gradients are accumulated, retained, blocked, or detached. These skills will help you understand what PyTorch performs during training instead of treating the backward pass as an invisible process.

Each chapter contains exercises that combine mathematical reasoning, tensor shape analysis, implementation, observation, and debugging. Complete the challenges without copying a finished solution. The objective is to predict values and gradients, test your reasoning in code, diagnose failures, and gradually develop the ability to inspect an entire backpropagation pipeline independently.

3.1.2 The Chapters (ABM00 - ABM06) included in this book

- ABM00 - Computational Graph Construction
- ABM01 - Forward vs Backward Pass
- ABM02 - Manual Gradient Calculation
- ABM03 - Chain Rule in Code
- ABM04 - Autograd Engine in PyTorch
- ABM05 - Backprop Through Linear Layers
- ABM06 - Gradient Flow Debugging

3.2 ABM00 - Computational Graph Construction

Mathematics and Theory

3.2.1 Exercise 00 - Tensor with requires_grad Setup

Part 1 - Why this matters

Training begins by identifying which tensors need derivatives. In PyTorch, `requires_grad=True` tells Autograd to record operations involving a tensor so that a later backward pass can compute its gradient.

This setting should normally be enabled for trainable floating-point parameters, not every tensor in a program. Understanding it prevents unnecessary graph construction and clarifies why some tensors receive a value in `.grad` while others do not.

Part 2 - Mathematical form

Let $\mathbf{x} \in \mathbb{R}^n$ be a trainable tensor and define:

$$y = f(\mathbf{x}).$$

Enabling gradient tracking requests the derivative:

$$\nabla_{\mathbf{x}} y = \left[\frac{\partial y}{\partial x_1}, \ldots, \frac{\partial y}{\partial x_n} \right].$$

The tensor $\mathbf{x}$ is a leaf when it is created directly rather than as the result of a tracked operation.

Part 3 - Hints and Helper

Create a floating-point tensor using the `requires_grad` argument. Inspect `requires_grad`, `is_leaf`, and `grad_fn` before performing any operation. Integer tensors cannot require gradients.

Part 4 - Python code skeleton

```
import torch

def E00(values):
    x = torch.tensor(..., requires_grad=...)
    return x, x.requires_grad, x.is_leaf, x.grad_fn

def main():
    print(E00([...]))

if __name__ == "__main__":
    main()
```

3.2.2 Exercise 01 - Simple Operation Graph Creation

Part 1 - Why this matters

Every differentiable operation applied to a tracked tensor becomes a node in a computational graph. The graph records how an output was produced, allowing Autograd to follow the dependencies in reverse during backpropagation.

A small graph makes this mechanism visible without the complexity of a neural network. Inspecting its tensors builds the habit of connecting each Python expression with a mathematical operation and a local derivative.

Part 2 - Mathematical form

Let $x \in \mathbb{R}$ require gradients and define:

$$y = 3x + 2.$$

The graph contains multiplication followed by addition, and its derivative is:

$$\frac{dy}{dx} = 3.$$

Although the constant 2 affects the forward value, its derivative contribution with respect to x is zero.

Part 3 - Hints and Helper

Create the expression in two separate operations and inspect `grad_fn` after each one. Do not call `backward()` yet; the goal is to construct and observe the graph created during the forward computation.

Part 4 - Python code skeleton

```
import torch

def E01(x):
    multiplied = x * ...
    y = multiplied + ...
    return multiplied, y, multiplied.grad_fn, y.grad_fn

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E01(x))

if __name__ == "__main__":
    main()
```

3.2.3 Exercise 02 - Multi-Operation Graph Building

Part 1 - Why this matters

Real models combine many operations before producing a loss. Autograd links these operations into one graph, preserving every dependency needed to apply the chain rule across the complete forward computation.

Building a multi-operation graph develops the ability to trace intermediate values and predict gradient paths. This is essential when an output depends on the same input through nonlinear transformations.

Part 2 - Mathematical form

Let $x \in \mathbb{R}$ and define:

$$a = x^2, \qquad b = 3a, \qquad y = b + \sin(x).$$

Therefore:

$$y = 3x^2 + \sin(x), \qquad \frac{dy}{dx} = 6x + \cos(x).$$

The final gradient combines contributions from both paths leading from x to y.

Part 3 - Hints and Helper

Store every intermediate tensor instead of writing one long expression. Inspect its value, `requires_grad`, and `grad_fn`. Sketch the operations and arrows before using the graph for backpropagation.

Part 4 - Python code skeleton

```
import torch

def E02(x):
    a = ...
    b = ...
    c = torch...
    y = ...
    return a, b, c, y

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E02(x))

if __name__ == "__main__":
    main()
```

3.2.4 Exercise 03 - Leaf vs Non-Leaf Tensor Detection

Part 1 - Why this matters

Leaf tensors are the starting points of a computational graph and usually represent model parameters or direct inputs. Non-leaf tensors are intermediate results created by tracked operations and carry information about the operation that produced them.

The distinction determines where PyTorch stores gradients by default. Leaf tensors with gradient tracking populate `.grad` after backpropagation, whereas a non-leaf tensor requires `retain_grad()` if its intermediate gradient must be inspected.

Part 2 - Mathematical form

Let x be created directly with gradient tracking and define:

$$y = 2x, \qquad z = y^2.$$

Then x is a leaf, while y and z are non-leaf tensors. Their dependencies are represented as:

$$x \longrightarrow y \longrightarrow z.$$

For scalar x, $dz/dx = 8x$ and $dz/dy = 2y$.

Part 3 - Hints and Helper

Inspect `is_leaf` and `grad_fn` for all three tensors. Call `retain_grad()` on an intermediate only if you want its gradient stored. Compare the `.grad` fields after a scalar backward pass.

Part 4 - Python code skeleton

```
import torch

def E03(x):
    y = ...
    z = ...
    y.retain_grad()
    z.backward()
    return (...)

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E03(x))

if __name__ == "__main__":
    main()
```

3.2.5 Exercise 04 - Graph Retention After Backward

Part 1 - Why this matters

During a backward pass, PyTorch normally releases saved intermediate values once they are no longer needed. This memory-saving behavior means that the same graph cannot usually be traversed backward a second time.

Graph retention is useful when several gradient calculations genuinely depend on the same forward graph, but it increases memory use. Knowing when retention is necessary prevents both graph-reuse errors and unnecessary memory consumption.

Part 2 - Mathematical form

Let:

$$y = x^3.$$

A backward traversal computes:

$$\frac{dy}{dx} = 3x^2.$$

Repeated backward calls on the same y also accumulate gradients in $x.grad$. After two valid traversals without clearing the gradient, the stored value is:

$$x.grad = 2(3x^2).$$

Part 3 - Hints and Helper

Use `retain_graph=True` on the backward call that must preserve the graph for another traversal. Observe both graph lifetime and gradient accumulation. Retention does not automatically reset `x.grad`.

Part 4 - Python code skeleton

```
import torch

def E04(x):
    y = ...
    y.backward(retain_graph=...)
    first_gradient = x.grad...
    y.backward()
    second_gradient = x.grad...
    return first_gradient, second_gradient

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E04(x))

if __name__ == "__main__":
    main()
```

3.2.6 Exercise 05 - Detaching Tensor from Graph

Part 1 - Why this matters

Some computations need a tensor's current value without allowing later operations to send gradients back through its history. Detaching creates a tensor that shares data with the original tensor but is disconnected from the recorded computational graph.

This mechanism is useful for fixed targets, logging, selected model branches, and algorithms with intentionally blocked gradients. Because storage may be shared, detaching should not be confused with making an independent copy.

Part 2 - Mathematical form

Let $y = f(x)$ and define a detached value:

$$\widetilde{y} = \text{stopgrad}(y).$$

The forward values satisfy $\widetilde{y} = y$, but Autograd treats:

$$\frac{\partial \widetilde{y}}{\partial x} = 0.$$

If $z = g(\widetilde{y})$, the path from z through $\widetilde{y}$ to x does not contribute to the gradient.

Part 3 - Hints and Helper

Create an intermediate result, call `detach()`, and compare the original and detached tensors using `requires_grad`, `is_leaf`, and `grad_fn`. Use `detach().clone()` when independent storage is also required.

Part 4 - Python code skeleton

```
import torch

def E05(x):
    y = ...
    detached_y = y...
    return y, detached_y, (...)

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E05(x))

if __name__ == "__main__":
    main()
```

3.2.7 Exercise 06 - Stop-Gradient Simulation Challenge

Part 1 - Why this matters

A branching graph may require learning through one path while treating another path as a fixed signal. Stop-gradient behavior makes this distinction explicit, allowing both branches to affect the forward result but only selected branches to influence parameter updates.

This pattern appears in target networks, contrastive objectives, and staged training strategies. Reasoning about each path separately prevents the mistaken assumption that equal forward values imply equal gradient behavior.

Part 2 - Mathematical form

Let:

$$a = x^2, \qquad b = \text{stopgrad}(x^2), \qquad y = a + b.$$

The forward value is:

$$y = 2x^2,$$

but only a contributes to the derivative:

$$\frac{dy}{dx} = 2x.$$

Without stop-gradient, the derivative would instead be $4x$.

Part 3 - Hints and Helper

Build the two branches separately and detach only the branch intended to remain constant during differentiation. Predict the gradient before calling `backward()`, then compare it with an equivalent graph in which neither branch is detached.

Part 4 - Python code skeleton

```
import torch

def E06(x):
    trainable_branch = ...
    blocked_branch = (...).detach()
    y = ...
    y.backward()
    return y, x.grad

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E06(x))

if __name__ == "__main__":
    main()
```

3.2.8 Exercise 07 - Graph Reuse Error Debugging

Part 1 - Why this matters

Calling backward twice on a graph whose saved values were released produces a common Autograd error. The message can be confusing until the programmer distinguishes reusing output values from reusing the recorded graph itself.

Correct debugging depends on the intended algorithm. A graph may need explicit retention, or the forward computation may be safely performed again to create a fresh graph. Choosing between these options affects memory and program structure.

Part 2 - Mathematical form

For a forward computation:

$$y = f(x),$$

one graph represents the operations used to produce this particular y. After a normal backward traversal:

$$\nabla_x y = \frac{\partial f}{\partial x},$$

saved intermediates may be freed. Recomputing $y = f(x)$ constructs a new graph with the same mathematical derivative but a separate lifetime.

Part 3 - Hints and Helper

Reproduce the error with two backward calls, then repair it in two ways: retain the first graph when required, or recompute the forward output before the second call. Reset gradients when comparing results so accumulation does not hide the behavior being studied.

Part 4 - Python code skeleton

```
import torch

def E07(x):
    y = ...
    y.backward(...)
    first_gradient = x.grad...
    x.grad = ...
    second_output = ...
    second_output.backward()
    return first_gradient, x.grad

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E07(x))

if __name__ == "__main__":
    main()
```

3.2.9 Exercise 08 - In-place Operation Graph Failure

Part 1 - Why this matters

An in-place operation changes an existing tensor instead of producing a separate output. This can reduce allocations, but it may overwrite a value that Autograd saved for computing a derivative, making the backward calculation invalid.

PyTorch also protects leaf tensors requiring gradients from many in-place changes. Learning to recognize operations whose names end with an underscore helps prevent subtle graph corruption and version-counter errors.

Part 2 - Mathematical form

Suppose:

$$y = x^2.$$

Backward requires the original x because:

$$\frac{dy}{dx} = 2x.$$

If an in-place operation changes x after the forward computation, the stored forward value and current value no longer describe the same graph. An out-of-place operation instead creates x_{new} while preserving x.

Part 3 - Hints and Helper

Construct a small failing example and read the complete error message. Identify which tensor was modified and replace the in-place expression with an out-of-place equivalent. Do not hide the problem using `.data`, because that bypasses Autograd's safety checks.

Part 4 - Python code skeleton

```
import torch

def E08(x):
    y = ...
    modified_x = x ...
    loss = ...
    loss.backward()
    return modified_x, x.grad

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E08(x))

if __name__ == "__main__":
    main()
```

3.2.10 Exercise 09 - Manual Graph Reasoning Challenge

Part 1 - Why this matters

Autograd is most useful when its results can be predicted and explained. Manual graph reasoning breaks a complicated expression into nodes, identifies every path from an input to the output, and applies local derivatives in reverse order.

This final challenge combines branching, shared intermediates, and the chain rule. It prepares the reader to diagnose real models by reasoning about graph structure before relying on printed gradient values.

Part 2 - Mathematical form

Let:

$$a = x^2, \qquad b = 3a, \qquad c = \sin(a), \qquad y = b + c.$$

Both branches depend on the shared node a. The local derivatives are:

$$\frac{da}{dx} = 2x, \quad \frac{db}{da} = 3, \quad \frac{dc}{da} = \cos(a).$$

Therefore:

$$\frac{dy}{dx} = \left(3 + \cos(x^2)\right) 2x.$$

Part 3 - Hints and Helper

Draw the graph and label each edge with its local derivative. Begin at y and move backward, adding contributions where branches meet. Compute the manual gradient first, then use Autograd only to verify the prediction.

Part 4 - Python code skeleton

```
import torch

def E09(x):
    a = ...
    b = ...
    c = torch...
    y = ...
    manual_gradient = ...
    y.backward()
    return manual_gradient, x.grad

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E09(x))

if __name__ == "__main__":
    main()
```

3.3 ABM01 - Forward vs Backward Pass

Mathematics and Theory

3.3.1 Exercise 00 - Forward Pass Value Tracking

Part 1 - Why this matters

The forward pass transforms inputs through a sequence of operations and produces intermediate values and a final output. Recording these values makes it possible to verify the numerical behavior of a model before studying its gradients.

Forward tracking also reveals which quantities Autograd may need later during backpropagation. Separating forward values from backward gradients prevents the two stages from being treated as one unexplained process.

Part 2 - Mathematical form

Let $x \in \mathbb{R}$ and define:

$$a = 2x + 1, \qquad b = a^2, \qquad y = \tanh(b).$$

The forward pass evaluates the graph in dependency order:

$$x \longrightarrow a \longrightarrow b \longrightarrow y.$$

Each value is computed from values already available to its left.

Part 3 - Hints and Helper

Store every intermediate tensor and print its value, shape, and `grad_fn`. Do not call `backward()` in this exercise. Predict all values manually for a simple input before comparing them with PyTorch.

Part 4 - Python code skeleton

```
import torch

def E00(x):
    a = ...
    b = ...
    y = torch...
    return a, b, y

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E00(x))

if __name__ == "__main__":
    main()
```

3.3.2 Exercise 01 - Backward Call on Scalar Loss

Part 1 - Why this matters

Model training normally reduces many predictions to one scalar loss. Calling `backward()` on that scalar starts reverse-mode differentiation and computes how the loss changes with respect to every required leaf tensor.

A scalar output supplies an implicit starting derivative of one. Understanding this seed explains why backward can be called without an explicit argument on a scalar but requires additional gradient information for a non-scalar output.

Part 2 - Mathematical form

Let $\mathbf{x} \in \mathbb{R}^n$ and define:

$$L = \sum_{i=1}^{n} (x_i - t_i)^2.$$

Since $L \in \mathbb{R}$, backpropagation begins with:

$$\frac{\partial L}{\partial L} = 1,$$

and produces:

$$\frac{\partial L}{\partial x_i} = 2(x_i - t_i).$$

Part 3 - Hints and Helper

Compute element-wise errors, square them, and reduce them with `torch.sum()` before calling `backward()`. Confirm that the loss has zero dimensions and compare `x.grad` with the manual derivative.

Part 4 - Python code skeleton

```
import torch

def E01(x, target):
    error = ...
    loss = torch...
    loss.backward()
    return loss, x.grad

def main():
    x = torch.tensor([...], requires_grad=True)
    target = torch.tensor([...])
    print(E01(x, target))

if __name__ == "__main__":
    main()
```

3.3.3 Exercise 02 - Gradient Accumulation Observation

Part 1 - Why this matters

PyTorch adds newly computed gradients to the existing `.grad` field of a leaf tensor. This supports algorithms that combine contributions from several losses or smaller batches, but it can also cause incorrect updates when the previous gradient is forgotten.

Observing accumulation directly establishes an important training-loop rule: gradient buffers persist independently of the graph that produced them. Repeated backward calls therefore change stored gradients even when each graph is new.

Part 2 - Mathematical form

For $L = x^2$:

$$\frac{dL}{dx} = 2x.$$

If two backward passes contribute gradients g_1 and g_2 without clearing the buffer, PyTorch stores:

$$x.grad = g_1 + g_2.$$

For two identical passes, this becomes $x.grad = 4x$.

Part 3 - Hints and Helper

Construct a fresh scalar loss before each backward pass so graph reuse is not the variable being tested. Clone the gradient after the first pass, run the second pass, and compare both stored values without zeroing in between.

Part 4 - Python code skeleton

```
import torch

def E02(x):
    loss1 = ...
    loss1.backward()
    first = x.grad...
    loss2 = ...
    loss2.backward()
    second = x.grad...
    return first, second

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E02(x))

if __name__ == "__main__":
    main()
```

3.3.4 Exercise 03 - Zeroing Gradients Correctly

Part 1 - Why this matters

Independent optimization steps should not normally inherit gradients from the previous step. Clearing gradient buffers before a new backward pass ensures that the parameter update reflects the current loss rather than an unintended sum of old and new derivatives.

PyTorch allows a gradient to be set to zero or reset to `None`. Both can prevent unwanted accumulation, but setting it to `None` can avoid a fill operation and clearly indicates that no gradient has yet been computed.

Part 2 - Mathematical form

At training step t, let:

$$g_t = \nabla_\theta L_t(\theta).$$

The desired buffer before backward is empty, producing:

$$\theta.grad = g_t.$$

Without clearing a previous gradient g_{t-1}, the stored result becomes:

$$\theta.grad = g_{t-1} + g_t.$$

Part 3 - Hints and Helper

Run two separate losses and clear `x.grad` between them. Try both `x.grad.zero_()` and assigning `None`, taking care not to call `zero_()` before a gradient tensor exists.

Part 4 - Python code skeleton

```
import torch

def E03(x):
    first_loss = ...
    first_loss.backward()
    first_gradient = x.grad...
    x.grad = ...
    second_loss = ...
    second_loss.backward()
    return first_gradient, x.grad

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E03(x))

if __name__ == "__main__":
    main()
```

3.3.5 Exercise 04 - Multiple Backward Calls Challenge

Part 1 - Why this matters

Several objectives may depend on the same trainable tensor. Their gradient contributions can be computed through separate backward calls or through one combined loss, provided graph lifetimes and accumulation are handled correctly.

Comparing these approaches demonstrates the linearity of differentiation. It also separates the mathematical addition of loss gradients from PyTorch's mechanical accumulation inside the parameter gradient buffer.

Part 2 - Mathematical form

Let:

$$L_1 = x^2, \qquad L_2 = 3x, \qquad L = L_1 + L_2.$$

Then:

$$\frac{dL}{dx} = \frac{dL_1}{dx} + \frac{dL_2}{dx} = 2x + 3.$$

Separate backward calls accumulate the same total when both graphs are valid.

Part 3 - Hints and Helper

Compute two losses from the same leaf and call backward on each. Then reset the gradient, reconstruct the forward computation, sum the losses, and use one backward call. Compare the final gradients with `torch.allclose()`.

Part 4 - Python code skeleton

```
import torch

def E04(x):
```

```
    loss1 = ...
    loss2 = ...
    loss1.backward(...)
    loss2.backward()
    separate_gradient = x.grad...
    return separate_gradient

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E04(x))

if __name__ == "__main__":
    main()
```

3.3.6 Exercise 05 - retain_graph Usage Challenge

Part 1 - Why this matters

A backward pass normally frees saved intermediates to reduce memory use. `retain_graph=True` preserves them when another backward traversal must use the same forward graph, but careless retention can keep unnecessary memory alive.

The challenge is to use retention only when the algorithm requires graph reuse. Recomputing a forward pass is often clearer, while retention is appropriate when multiple derivatives must be obtained from exactly the same recorded graph.

Part 2 - Mathematical form

Let a shared intermediate be:

$$h = x^2, qquadL_1 = 2h, qquadL_2 = h^2.$$

The losses share the path $x \to h$. Their derivatives are:

$$\frac{dL_1}{dx} = 4x, \qquad \frac{dL_2}{dx} = 4x^3.$$

Traversing both from one forward graph requires preserving shared saved data until the final traversal.

Part 3 - Hints and Helper

Call backward on the first loss with graph retention, then call backward on the second normally. Observe accumulated gradients and compare them with the manual sum. Explain why retaining the graph on the final call is unnecessary.

Part 4 - Python code skeleton

```
import torch

def E05(x):
    shared = ...
```

```
    loss1 = ...
    loss2 = ...
    loss1.backward(retain_graph=...)
    loss2.backward()
    return x.grad

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E05(x))

if __name__ == "__main__":
    main()
```

3.3.7 Exercise 06 - Backward Through Branching Graph

Part 1 - Why this matters

Computational graphs branch when one tensor feeds several later operations. During backpropagation, every path contributes to the gradient of the shared ancestor, so ignoring one branch produces an incomplete derivative.

Branching is common in residual connections, multi-head models, and objectives with several terms. Manual path analysis makes the addition of gradient contributions visible before Autograd performs it automatically.

Part 2 - Mathematical form

Let:

$$a = x^2, \qquad b = 3x, \qquad y = a + b.$$

There are two paths from x to y, therefore:

$$\frac{dy}{dx} = \frac{\partial y}{\partial a}\frac{da}{dx} + \frac{\partial y}{\partial b}\frac{db}{dx} = 2x + 3.$$

Part 3 - Hints and Helper

Build each branch separately and retain the intermediate tensors for inspection. Draw both gradient paths, compute their contributions manually, and compare their sum with the leaf gradient after one backward call.

Part 4 - Python code skeleton

```
import torch

def E06(x):
    branch_a = ...
    branch_b = ...
    y = ...
    y.backward()
    manual_gradient = ...
```

```
    return manual_gradient, x.grad

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E06(x))

if __name__ == "__main__":
    main()
```

3.3.8 Exercise 07 - Gradient Flow Order Reasoning

Part 1 - Why this matters

The forward pass follows dependencies from inputs to outputs, whereas the backward pass propagates derivatives from the loss toward earlier tensors. This reverse order is required because each upstream gradient depends on gradients already obtained from downstream operations.

Understanding the order helps interpret hooks and debug deep graphs. The engine may schedule independent branches internally, but it must respect the dependency relationships defined by the chain rule.

Part 2 - Mathematical form

For:

$$a = f(x), \qquad b = g(a), \qquad L = h(b),$$

the forward order is $x \to a \to b \to L$. The backward calculation is:

$$\frac{dL}{db} \longrightarrow \frac{dL}{da} = \frac{dL}{db}\frac{db}{da} \longrightarrow \frac{dL}{dx} = \frac{dL}{da}\frac{da}{dx}.$$

Part 3 - Hints and Helper

Call `retain_grad()` on non-leaf intermediates and register simple hooks that report when their gradients arrive. Compare the observed trace with the reverse dependency order predicted on paper.

Part 4 - Python code skeleton

```
import torch

def E07(x):
    a = ...
    b = ...
    loss = ...
    a.retain_grad()
    b.retain_grad()
    loss.backward()
    return x.grad, a.grad, b.grad
```

```
def main():
    x = torch.tensor(..., requires_grad=True)
    print(E07(x))

if __name__ == "__main__":
    main()
```

3.3.9 Exercise 08 - Backward Error Debugging

Part 1 - Why this matters

Backward failures often result from a non-scalar output, a detached graph, a reused graph, or a tensor that never required gradients. Reading the tensor properties before changing code is more reliable than applying unrelated fixes.

A systematic diagnosis identifies whether the graph exists, whether the output has the expected shape, and whether the requested leaf participates in a valid path to the loss. This method turns runtime errors into structural evidence.

Part 2 - Mathematical form

For a vector output $\mathbf{y} = f(\mathbf{x}) \in \mathbb{R}^m$, a backward seed $\mathbf{v} \in \mathbb{R}^m$ computes a vector–Jacobian product:

$$\mathbf{v}^\top \frac{\partial \mathbf{y}}{\partial \mathbf{x}}.$$

Alternatively, reducing the output to a scalar loss:

$$L = \sum_{i=1}^{m} y_i$$

permits an implicit seed of one.

Part 3 - Hints and Helper

Inspect the output's shape, `requires_grad`, and `grad_fn`, then inspect the leaf tensor. For a vector output, either define a meaningful scalar reduction or provide a matching gradient argument to `backward()`.

Part 4 - Python code skeleton

```
import torch

def E08(x):
    output = ...
    print(output.shape, output.requires_grad, output.grad_fn)
    scalar_loss = torch...
    scalar_loss.backward()
    return x.grad

def main():
    x = torch.tensor([...], requires_grad=True)
```

```
    print(E08(x))

if __name__ == "__main__":
    main()
```

3.3.10 Exercise 09 - Custom Forward–Backward Trace

Part 1 - Why this matters

A complete trace records both the numerical forward path and the derivative path through the same graph. Pairing these stages produces an auditable explanation of how a final loss value leads to a particular input gradient.

This challenge combines intermediate tracking, retained gradients, local derivatives, and reverse-order reasoning. It prepares the reader to analyze a small model as a structured computation rather than a single formula.

Part 2 - Mathematical form

Let:

$$a = 2x, qquadb = a + 1, qquadc = b^2, qquadL = \frac{1}{2}c.$$

The backward trace is:

$$\frac{dL}{dc} = \frac{1}{2}, \qquad \frac{dL}{db} = b, qquad\frac{dL}{da} = b, \qquad \frac{dL}{dx} = 2b.$$

The trace should connect each stored forward value to its corresponding gradient.

Part 3 - Hints and Helper

Retain gradients for all non-leaf intermediates, store their forward values, and call backward once on the scalar loss. Return an ordered structure containing each node name, value, and gradient, then compare it with the manual trace.

Part 4 - Python code skeleton

```
import torch

def E09(x):
    a = ...
    b = ...
    c = ...
    loss = ...
    for tensor in (...):
        tensor.retain_grad()
    loss.backward()
    trace = ...
    return trace

def main():
    x = torch.tensor(..., requires_grad=True)
```

```
    print(E09(x))

if __name__ == "__main__":
    main()
```

3.4 ABM02 - Manual Gradient Calculation

Mathematics and Theory

3.4.1 Exercise 00 - Derivative of Linear Function

Part 1 - Why this matters

Linear functions provide the simplest setting for connecting a forward value to its derivative. Their constant slope shows how one small input change produces a proportional output change everywhere in the domain.

Manually deriving this gradient establishes the notation and verification process used throughout backpropagation. It also clarifies that the bias changes the forward output but does not change the derivative with respect to the input.

Part 2 - Mathematical form

Let:

$$y = ax + b, \qquad a, b, x \in \mathbb{R}.$$

The derivative with respect to x is:

$$\frac{dy}{dx} = a.$$

If a and b are also trainable, then:

$$\frac{\partial y}{\partial a} = x, \qquad \frac{\partial y}{\partial b} = 1.$$

Part 3 - Hints and Helper

Calculate the three derivatives on paper before writing code. Create floating-point tensors for x, a, and b, then compare the manual results with their `.grad` values after a scalar backward pass.

Part 4 - Python code skeleton

```
import torch

def E00(x, a, b):
    y = ...
    manual_dx = ...
    y.backward()
    return y, manual_dx, x.grad
```

```
def main():
    x = torch.tensor(..., requires_grad=True)
    a = torch.tensor(..., requires_grad=True)
    b = torch.tensor(..., requires_grad=True)
    print(E00(x, a, b))

if __name__ == "__main__":
    main()
```

3.4.2 Exercise 01 - Manual Gradient vs Autograd Check

Part 1 - Why this matters

Autograd is trustworthy only when the programmer understands what its result represents. Comparing an automatic gradient with a manually derived expression provides a direct correctness check for both the mathematics and implementation.

This comparison is especially useful while developing custom losses or unfamiliar tensor formulas. A mismatch becomes evidence that an operation, reduction, graph connection, or manual derivative has been interpreted incorrectly.

Part 2 - Mathematical form

Let:

$$f(x) = x^3 + 2x^2 - 5x + 1.$$

Its manual derivative is:

$$f'(x) = 3x^2 + 4x - 5.$$

For a selected value $x = x_0$, correctness requires:

$$g_{manual} = f'(x_0) \approx g_{autograd}.$$

Part 3 - Hints and Helper

Evaluate the polynomial using tracked PyTorch operations, derive the gradient independently, and call `backward()` once. Compare floating-point values with `torch.allclose()` rather than exact equality.

Part 4 - Python code skeleton

```
import torch

def E01(x):
    y = ...
    manual_gradient = ...
    y.backward()
    matches = torch...
    return manual_gradient, x.grad, matches

def main():
```

```
    x = torch.tensor(..., requires_grad=True)
    print(E01(x))

if __name__ == "__main__":
    main()
```

3.4.3 Exercise 02 - Gradient of Quadratic Function

Part 1 - Why this matters

Quadratic functions introduce gradients that depend on the current input rather than remaining constant. They are fundamental to squared-error losses, distance measures, regularization, and many optimization examples.

Their derivative also provides an intuitive view of optimization: the gradient is zero at the vertex and grows in magnitude farther away. Its sign identifies the direction in which the function increases.

Part 2 - Mathematical form

Let:

$$f(x) = ax^2 + bx + c.$$

The first derivative is:

$$\frac{df}{dx} = 2ax + b,$$

and the second derivative is:

$$\frac{d^2 f}{dx^2} = 2a.$$

When $a > 0$, the stationary point is a minimum at $x = -b/(2a)$.

Part 3 - Hints and Helper

Choose coefficients and evaluate both the function and manual derivative at several input values. Use a separate tracked tensor for each test or clear its gradient before constructing a new forward computation.

Part 4 - Python code skeleton

```
import torch

def E02(x, a, b, c):
    y = ...
    manual_gradient = ...
    y.backward()
    return y, manual_gradient, x.grad

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E02(x, ..., ..., ...))

if __name__ == "__main__":
    main()
```

3.4.4 Exercise 03 - Chain Rule Manual Verification

Part 1 - Why this matters

Most model outputs depend on parameters through several intermediate operations. The chain rule connects the local derivative of every operation so that a final loss gradient can reach the earliest input.

Manual verification makes the multiplication of local derivatives explicit. This prepares the reader to reason about deep graphs where Autograd applies the same principle repeatedly in reverse order.

Part 2 - Mathematical form

Let:

$$u = x^2 + 1, \qquad y = \sin(u).$$

The local derivatives are:

$$\frac{dy}{du} = \cos(u), \qquad \frac{du}{dx} = 2x.$$

Therefore:

$$\frac{dy}{dx} = \cos(x^2 + 1)\, 2x.$$

Part 3 - Hints and Helper

Store u and y separately. Calculate both local derivatives and multiply them before calling backward. Retain the intermediate gradient if you also want to compare Autograd's value for dy/du.

Part 4 - Python code skeleton

```
import torch

def E03(x):
    u = ...
    y = torch...
    local_dy_du = ...
    local_du_dx = ...
    manual_gradient = ...
    y.backward()
    return manual_gradient, x.grad

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E03(x))

if __name__ == "__main__":
    main()
```

3.4.5 Exercise 04 - Vector Gradient Computation

Part 1 - Why this matters

Machine learning losses usually depend on vectors of parameters or predictions. The gradient collects one partial derivative for every input component, producing a vector that has the same shape as the differentiated vector.

Computing this result manually demonstrates how a scalar reduction combines independent element-wise contributions. It also builds the shape reasoning needed to distinguish a gradient from a full Jacobian.

Part 2 - Mathematical form

Let $\mathbf{x} \in \mathbb{R}^n$ and define:

$$L(\mathbf{x}) = \sum_{i=1}^{n} x_i^2.$$

For each component:

$$\frac{\partial L}{\partial x_i} = 2x_i.$$

Therefore:

$$\nabla_{\mathbf{x}} L = 2\mathbf{x} \in \mathbb{R}^n.$$

Part 3 - Hints and Helper

Build the scalar loss with an element-wise square followed by a sum. Derive the complete gradient tensor without a Python loop, then verify its values and shape against `x.grad`.

Part 4 - Python code skeleton

```
import torch

def E04(x):
    loss = torch...
    manual_gradient = ...
    loss.backward()
    return manual_gradient, x.grad, x.grad.shape

def main():
    x = torch.tensor([...], requires_grad=True)
    print(E04(x))

if __name__ == "__main__":
    main()
```

3.4.6 Exercise 05 - Matrix Gradient Reasoning

Part 1 - Why this matters

Weights in linear layers are matrices, so their gradients must preserve both row and column roles. Matrix gradient reasoning connects the compact notation of a linear transformation to

the contribution made by every input-output pair.

Understanding this structure helps detect accidental transposes and incompatible shapes. It also reveals why a batch produces a sum of outer-product contributions to the weight gradient.

Part 2 - Mathematical form

Let $\mathbf{W} \in \mathbb{R}^{m \times n}$, $\mathbf{x} \in \mathbb{R}^n$, and $\mathbf{y} = \mathbf{W}\mathbf{x}$. Define:

$$L = \sum_{i=1}^{m} y_i.$$

Since $y_i = \sum_j W_{ij} x_j$:

$$\frac{\partial L}{\partial W_{ij}} = x_j.$$

Thus every row of $\nabla_{\mathbf{W}} L$ equals $\mathbf{x}^\mathsf{T}$.

Part 3 - Hints and Helper

Calculate the output with matrix-vector multiplication and reduce it to a scalar. Construct the expected matrix gradient using shape-aware tensor operations, then compare its values and orientation with `W.grad`.

Part 4 - Python code skeleton

```
import torch

def E05(W, x):
    y = torch...
    loss = torch...
    manual_gradient = ...
    loss.backward()
    return manual_gradient, W.grad

def main():
    W = torch.tensor(..., requires_grad=True)
    x = torch.tensor(...)
    print(E05(W, x))

if __name__ == "__main__":
    main()
```

3.4.7 Exercise 06 - Broadcasting Gradient Challenge

Part 1 - Why this matters

Broadcasting allows one parameter to affect many output elements without storing explicit copies. During backpropagation, all gradient contributions created by those logical copies must be reduced back to the parameter's original shape.

This reduction is easy to overlook because the forward expression appears as a simple addition. Understanding it is essential for bias gradients and any shared parameter applied across batches or spatial dimensions.

Part 2 - Mathematical form

Let $\mathbf{X} \in \mathbb{R}^{B \times F}$ and $\mathbf{b} \in \mathbb{R}^{F}$. Define:

$$Y_{ij} = X_{ij} + b_j, \qquad L = \sum_{i=1}^{B} \sum_{j=1}^{F} Y_{ij}.$$

The broadcasted bias appears once per batch item, so:

$$\frac{\partial L}{\partial b_j} = B.$$

Part 3 - Hints and Helper

Predict the bias gradient from the batch size before running the code. Confirm that Autograd reduces the expanded gradient along the broadcasted axis and returns a tensor with the original bias shape.

Part 4 - Python code skeleton

```
import torch

def E06(X, bias):
    Y = ...
    loss = torch...
    manual_bias_gradient = ...
    loss.backward()
    return manual_bias_gradient, bias.grad

def main():
    X = torch...
    bias = torch.tensor(..., requires_grad=True)
    print(E06(X, bias))

if __name__ == "__main__":
    main()
```

3.4.8 Exercise 07 - Gradient Shape Validation

Part 1 - Why this matters

A gradient with respect to a tensor must match that tensor's shape. Shape validation catches incomplete reductions, unintended broadcasting, and incorrect manual formulas even when some numerical values appear reasonable.

This rule is a practical invariant throughout neural network training. Parameters can be updated element by element only when the parameter tensor and its gradient have corresponding dimensions.

Part 2 - Mathematical form

For a scalar loss L and tensor $\mathcal{X} \in \mathbb{R}^{d_1 \times \cdots \times d_k}$:

$$\nabla_{\mathcal{X}} L = \left[\frac{\partial L}{\partial X_{i_1 \ldots i_k}} \right] \in \mathbb{R}^{d_1 \times \cdots \times d_k}.$$

Therefore:

$$\text{shape}(\nabla_{\mathcal{X}} L) = \text{shape}(\mathcal{X}).$$

Part 3 - Hints and Helper

Choose a multidimensional tensor and a scalar loss involving a reduction. Check the expected gradient shape before backward, then validate rank and every axis. Report a useful mismatch message rather than only returning `False`.

Part 4 - Python code skeleton

```
import torch

def E07(X):
    loss = ...
    manual_gradient = ...
    loss.backward()
    shape_valid = ...
    values_valid = torch...
    return shape_valid, values_valid

def main():
    X = torch.tensor(..., requires_grad=True)
    print(E07(X))

if __name__ == "__main__":
    main()
```

3.4.9 Exercise 08 - Numerical Gradient Approximation

Part 1 - Why this matters

A numerical gradient estimates a derivative using nearby function values rather than symbolic rules or Autograd. It provides an independent diagnostic when checking a manually implemented gradient or custom backward method.

The estimate is approximate because the step size introduces a tradeoff. A large step creates truncation error, while an extremely small step can magnify floating-point rounding error.

Part 2 - Mathematical form

For a scalar function f and small $h > 0$, the central-difference approximation is:

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}.$$

Its absolute comparison error is:

$$E = \left| g_{numerical} - g_{autograd} \right| .$$

Central differences generally provide better accuracy than a one-sided estimate for the same moderate step size.

Part 3 - Hints and Helper

Use double precision and evaluate the perturbed inputs without building an unnecessary gradient graph. Test several step sizes and compare the numerical estimate with both the analytical derivative and Autograd.

Part 4 - Python code skeleton

```
import torch

def function(x):
    return ...

def E08(x, h):
    forward_value = function(x + ...)
    backward_value = function(x - ...)
    numerical_gradient = ...
    return numerical_gradient

def main():
    x = torch.tensor(..., dtype=torch.float64)
    print(E08(x, ...))

if __name__ == "__main__":
    main()
```

3.4.10 Exercise 09 - Gradient Mismatch Debugging

Part 1 - Why this matters

Manual, numerical, and automatic gradients may disagree because of an incorrect derivative, a missing reduction, a detached path, accumulated gradients, poor numerical precision, or evaluation at different inputs. Random changes rarely identify which cause is responsible.

A structured comparison isolates the failure by checking the forward function, gradient shapes, graph connectivity, buffer state, and numerical tolerance in a fixed order. This method scales to debugging larger models.

Part 2 - Mathematical form

Let g_m, g_a, and g_n denote manual, Autograd, and numerical gradients. Define pairwise errors:

$$E_{ma} = \|g_m - g_a\|, \qquad E_{na} = \|g_n - g_a\|.$$

A relative error can be measured by:

$$E_{rel} = \frac{\|g_1 - g_2\|}{\max(\|g_1\|, \|g_2\|, \varepsilon)}.$$

Part 3 - Hints and Helper

Evaluate all methods at the same input and dtype. Clear old gradients, verify matching shapes, and print absolute and relative errors. Test simple input values to determine whether the mismatch is mathematical, structural, or numerical.

Part 4 - Python code skeleton

```
import torch

def E09(x, step_size, tolerance):
    manual_gradient = ...
    autograd_gradient = ...
    numerical_gradient = ...
    absolute_error = ...
    relative_error = ...
    matches = ...
    return (...)

def main():
    x = torch.tensor(..., dtype=torch.float64, requires_grad=True)
    print(E09(x, ..., ...))

if __name__ == "__main__":
    main()
```

3.5 ABM03 - Chain Rule in Code

Mathematics and Theory

3.5.1 Exercise 00 - Single-Chain Gradient Flow

Part 1 - Why this matters

A single chain shows the basic mechanism of backpropagation: every output depends on one previous intermediate until the original input is reached. The gradient is transported backward by multiplying local derivatives.

Studying a short chain makes forward and backward directions easy to separate. It provides the foundation for deeper graphs, where the same rule is repeated across many layers.

Part 2 - Mathematical form

Let $u = 2x + 1$ and $y = u^2$. The chain rule gives:

$$\frac{dy}{dx} = \frac{dy}{du}\frac{du}{dx} = (2u)(2) = 4(2x + 1).$$

The forward path is $x \to u \to y$, while gradient flow follows the reverse direction.

Part 3 - Hints and Helper

Store the intermediate tensor and retain its gradient. Calculate both local derivatives manually, multiply them, and compare the result with `x.grad` after one scalar backward call.

Part 4 - Python code skeleton

```
import torch

def E00(x):
    u = ...
    y = ...
    u.retain_grad()
    manual_gradient = ...
    y.backward()
    return manual_gradient, x.grad, u.grad

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E00(x))

if __name__ == "__main__":
    main()
```

3.5.2 Exercise 01 - Multi-Operation Chain Rule

Part 1 - Why this matters

Real tensor expressions combine several different operations, each contributing its own local derivative. The complete input gradient is the product of these derivatives along the dependency chain.

Writing every intermediate prevents skipped factors and makes errors easier to locate. This discipline is directly applicable to tracing gradients through multiple neural network layers.

Part 2 - Mathematical form

Let $a = x^2$, $b = 3a + 1$, and $y = \log(b)$. Then:

$$\frac{dy}{dx} = \frac{dy}{db}\frac{db}{da}\frac{da}{dx} = \frac{1}{b} \cdot 3 \cdot 2x = \frac{6x}{3x^2 + 1}.$$

Part 3 - Hints and Helper

Implement the three operations separately and ensure the logarithm receives a positive input. Compute each local derivative at its correct forward value before comparing the product with Autograd.

Part 4 - Python code skeleton

```
import torch

def E01(x):
    a = ...
    b = ...
    y = torch...
    manual_gradient = ...
    y.backward()
    return manual_gradient, x.grad

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E01(x))

if __name__ == "__main__":
    main()
```

3.5.3 Exercise 02 - Nested Function Gradient Flow

Part 1 - Why this matters

Nested functions express one transformation inside another, matching the layered structure of many artificial intelligence models. Differentiating the outer function alone is insufficient because its input also changes with the original variable.

Expanding a nested expression into named stages exposes the exact chain Autograd records. It also clarifies where each derivative must be evaluated during the backward pass.

Part 2 - Mathematical form

Let:

$$y = f(g(h(x))), \qquad h(x) = x^2, \quad g(h) = \sin(h), \quad f(g) = e^g.$$

Then:

$$\frac{dy}{dx} = e^{\sin(x^2)} \cos(x^2)\, 2x.$$

Part 3 - Hints and Helper

Create one tensor for each nested stage. Predict all forward values first, then calculate the derivative from the outside toward the inside and verify it using `backward()`.

Part 4 - Python code skeleton

```
import torch

def E02(x):
    h = ...
    g = torch...
    y = torch...
    manual_gradient = ...
    y.backward()
    return manual_gradient, x.grad

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E02(x))

if __name__ == "__main__":
    main()
```

3.5.4 Exercise 03 - Chain Rule with Activation

Part 1 - Why this matters

Activation functions insert nonlinear derivatives into a model's gradient path. Their local slope can preserve, reduce, or completely block an upstream gradient depending on the forward input.

Combining a linear transformation with an activation demonstrates how layer parameters and activation behavior interact. This is the smallest useful model of gradient flow through a neural unit.

Part 2 - Mathematical form

Let $z = wx + b$ and $y = \sigma(z)$, where:

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Since $\sigma'(z) = \sigma(z)(1 - \sigma(z))$:

$$\frac{dy}{dx} = y(1 - y)w.$$

Part 3 - Hints and Helper

Compute the pre-activation and activation separately. Use the already computed activation value in the manual sigmoid derivative, then compare gradients for several inputs including values where sigmoid saturates.

Part 4 - Python code skeleton

```
import torch

def E03(x, w, b):
```

```
    z = ...
    y = torch...
    manual_gradient = ...
    y.backward()
    return y, manual_gradient, x.grad

def main():
    x = torch.tensor(..., requires_grad=True)
    w = torch.tensor(...)
    b = torch.tensor(...)
    print(E03(x, w, b))

if __name__ == "__main__":
    main()
```

3.5.5 Exercise 04 - Branching Chain Rule Challenge

Part 1 - Why this matters

A graph branches when one intermediate feeds more than one later operation. During backpropagation, every route from that intermediate to the loss contributes to its total gradient.

Branching appears in residual connections and multi-objective models. Correct reasoning requires multiplying derivatives along each path and then adding the path contributions where they meet.

Part 2 - Mathematical form

Let $a = x^2$, $b = 3a$, $c = \sin(a)$, and $y = b + c$. Then:

$$\frac{dy}{dx} = \left(\frac{db}{da} + \frac{dc}{da}\right)\frac{da}{dx} = \left(3 + \cos(x^2)\right)2x.$$

Part 3 - Hints and Helper

Draw both paths from a to y. Calculate their contributions separately, add them at the shared node, and only then multiply by the derivative from a to x.

Part 4 - Python code skeleton

```
import torch

def E04(x):
    a = ...
    branch_b = ...
    branch_c = torch...
    y = ...
    path_b = ...
    path_c = ...
    manual_gradient = ...
```

```
    y.backward()
    return path_b, path_c, manual_gradient, x.grad

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E04(x))

if __name__ == "__main__":
    main()
```

3.5.6 Exercise 05 - Gradient Path Contribution Analysis

Part 1 - Why this matters

A total gradient can hide which branch dominates learning. Measuring each path contribution reveals whether several objectives reinforce one another, cancel one another, or differ greatly in scale.

This analysis is useful for debugging multi-loss systems and skip connections. It turns a single final gradient into an explanation of how individual computational routes affected the parameter.

Part 2 - Mathematical form

Let $L = L_1 + L_2$, where $L_1 = x^2$ and $L_2 = -3x$. The path contributions are:

$$g_1 = \frac{dL_1}{dx} = 2x, \qquad g_2 = \frac{dL_2}{dx} = -3.$$

Therefore:

$$\frac{dL}{dx} = g_1 + g_2 = 2x - 3.$$

Part 3 - Hints and Helper

Calculate each derivative manually and also obtain separate Autograd gradients with `torch.autograd.g` while retaining the shared graph when necessary. Verify that their sum equals the combined-loss gradient.

Part 4 - Python code skeleton

```
import torch

def E05(x):
    loss1 = ...
    loss2 = ...
    contribution1 = torch.autograd...
    contribution2 = torch.autograd...
    total = ...
    return contribution1, contribution2, total

def main():
```

```
    x = torch.tensor(..., requires_grad=True)
    print(E05(x))

if __name__ == "__main__":
    main()
```

3.5.7 Exercise 06 - Partial Gradient Blocking

Part 1 - Why this matters

Sometimes only one branch of a larger computation should update an earlier parameter. Partial blocking keeps every branch active in the forward calculation while preventing selected paths from contributing during backpropagation.

This mechanism is useful in multi-stage learning and fixed-target methods. The challenge is to block exactly the intended path without disconnecting other gradient routes that must remain trainable.

Part 2 - Mathematical form

Let:

$$a = x^2, \qquad b = 3x, \qquad y = a + \text{stopgrad}(b).$$

The forward value is $x^2 + 3x$, but:

$$\frac{dy}{dx} = 2x,$$

because the derivative contribution from b is blocked.

Part 3 - Hints and Helper

Build both branches and detach only the blocked branch before combining them. Compare forward values with an unblocked version and compare their gradients to identify the removed contribution.

Part 4 - Python code skeleton

```
import torch

def E06(x):
    active_branch = ...
    blocked_branch = (...).detach()
    y = ...
    y.backward()
    return y, x.grad

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E06(x))

if __name__ == "__main__":
    main()
```

3.5.8 Exercise 07 - Detach Effect on Chain Rule

Part 1 - Why this matters

Detaching an intermediate breaks the chain rule at a precise boundary. Operations after that boundary can still form a new graph, but no derivative crosses backward into the history that produced the detached value.

Understanding this effect prevents accidental loss of learning signals. Equal tensor values do not imply equal gradient behavior when one value remains connected and the other has been detached.

Part 2 - Mathematical form

Let $u = x^2$ and $\widetilde{u} = \text{stopgrad}(u)$. If $y = 3\widetilde{u}$, then:

$$\frac{\partial y}{\partial \widetilde{u}} = 3, \qquad \frac{\partial \widetilde{u}}{\partial x} = 0,$$

so:

$$\frac{dy}{dx} = 0.$$

Part 3 - Hints and Helper

Inspect `requires_grad` and `grad_fn` before and after detaching. If later operations need their own gradient tracking, create an appropriate new leaf without reconnecting the original history.

Part 4 - Python code skeleton

```
import torch

def E07(x):
    u = ...
    detached_u = u.detach()
    new_leaf = detached_u...
    y = ...
    y.backward()
    return x.grad, new_leaf.grad

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E07(x))

if __name__ == "__main__":
    main()
```

3.5.9 Exercise 08 - Chain Rule Failure Debugging

Part 1 - Why this matters

A missing gradient may result from detaching, converting to a Python value, using a non-differentiable operation, or inspecting a non-leaf gradient without retaining it. The mathematical formula can be correct while the implemented graph is broken.

Effective debugging follows the graph from the loss back toward the expected leaf. At each stage, tensor properties reveal where gradient tracking stopped or where the wrong object entered the computation.

Part 2 - Mathematical form

For a valid chain $x \to u \to L$:

$$\frac{dL}{dx} = \frac{dL}{du}\frac{du}{dx}.$$

If implementation replaces u with $\text{stopgrad}(u)$, then:

$$\frac{d\ \text{stopgrad}(u)}{dx} = 0,$$

and the chain contribution disappears.

Part 3 - Hints and Helper

Print `requires_grad`, `is_leaf`, and `grad_fn` for every node. Locate the first tensor that unexpectedly lacks history and repair the graph construction rather than assigning a gradient manually.

Part 4 - Python code skeleton

```
import torch

def E08(x):
    u = ...
    suspicious_value = ...
    loss = ...
    print(...)
    loss.backward()
    return x.grad

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E08(x))

if __name__ == "__main__":
    main()
```

3.5.10 Exercise 09 - Full Graph Gradient Trace

Part 1 - Why this matters

A full trace combines forward values, graph structure, local derivatives, path contributions, and final gradients. This produces a complete explanation of how a scalar loss responds to its

earliest inputs.

The exercise integrates chains and branches into one debugging method. It prepares the reader to inspect larger models where an unexpected gradient must be traced across several dependent operations.

Part 2 - Mathematical form

Let $a = 2x$, $b = a^2$, $c = \sin(a)$, and $L = b + c$. Then:

$$\frac{dL}{da} = 2a + \cos(a), \qquad \frac{da}{dx} = 2,$$

and:

$$\frac{dL}{dx} = 2\left(2a + \cos(a)\right).$$

Part 3 - Hints and Helper

Retain gradients for all non-leaf nodes and record forward values before backward. Construct an ordered trace containing each node, its local derivative, retained gradient, and relationship to the manual result.

Part 4 - Python code skeleton

```
import torch

def E09(x):
    a = ...
    b = ...
    c = torch...
    loss = ...
    for tensor in (...):
        tensor.retain_grad()
    manual_gradient = ...
    loss.backward()
    trace = ...
    return manual_gradient, trace

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E09(x))

if __name__ == "__main__":
    main()
```

3.6 ABM04 - Autograd Engine in PyTorch

Mathematics and Theory

3.6.1 Exercise 00 - Autograd Basic Usage

Part 1 - Why this matters

Autograd automatically records differentiable operations involving tensors that require gradients. A scalar backward call then applies reverse-mode differentiation and stores results in the appropriate leaf tensors.

This basic workflow connects a forward formula, its computational graph, and parameter gradients. Mastering it provides the operational foundation for every later training exercise.

Part 2 - Mathematical form

Let $x \in \mathbb{R}$ and $L = (x-3)^2$. Then:

$$\frac{dL}{dx} = 2(x-3).$$

The forward pass computes L, and the backward pass evaluates this derivative at the current x.

Part 3 - Hints and Helper

Create a floating-point leaf with gradient tracking, calculate a scalar loss, and inspect its graph information. Call `backward()` once and compare `x.grad` with the manual derivative.

Part 4 - Python code skeleton

```
import torch

def E00(x):
    loss = ...
    loss.backward()
    manual_gradient = ...
    return loss, manual_gradient, x.grad

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E00(x))

if __name__ == "__main__":
    main()
```

3.6.2 Exercise 01 - grad_fn Inspection

Part 1 - Why this matters

Every non-leaf output of a tracked operation contains a `grad_fn` describing the backward operation associated with its creation. Following these objects reveals the structure Autograd will traverse.

Graph inspection helps explain why a tensor is connected, detached, or unable to propagate a gradient. It is especially valuable when a long expression does not produce the expected leaf gradient.

Part 2 - Mathematical form

For:

$$a = x^2, \qquad b = 3a, \qquad L = b + 1,$$

the dependency chain is:

$$x \longrightarrow a \longrightarrow b \longrightarrow L.$$

The leaf x has no producing operation, while each non-leaf node has backward history.

Part 3 - Hints and Helper

Inspect `is_leaf`, `requires_grad`, and `grad_fn` at every stage. Explore `next_functions` carefully to identify predecessor nodes without modifying the graph.

Part 4 - Python code skeleton

```
import torch

def E01(x):
    a = ...
    b = ...
    loss = ...
    inspection = ...
    return inspection

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E01(x))

if __name__ == "__main__":
    main()
```

3.6.3 Exercise 02 - Gradient Accumulation Behavior

Part 1 - Why this matters

Leaf gradient buffers accumulate contributions instead of being overwritten automatically. This allows several backward computations to contribute to one update, but it also makes stale gradients a common training error.

Observing the buffer across multiple passes separates graph lifetime from gradient lifetime. A graph may be new for each forward pass while the same leaf's `.grad` continues to retain earlier values.

Part 2 - Mathematical form

For losses $L_1 = x^2$ and $L_2 = 3x$:

$$\nabla_x L_1 = 2x, \qquad \nabla_x L_2 = 3.$$

Without clearing the buffer, two backward passes store:

$$x.grad = 2x + 3.$$

Part 3 - Hints and Helper

Construct each loss through a valid forward graph and clone the gradient after each backward call. Then reset the buffer and show that a new backward result begins without the earlier contribution.

Part 4 - Python code skeleton

```
import torch

def E02(x):
    loss1 = ...
    loss1.backward()
    first = x.grad...
    loss2 = ...
    loss2.backward()
    accumulated = x.grad...
    return first, accumulated

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E02(x))

if __name__ == "__main__":
    main()
```

3.6.4 Exercise 03 - No-Grad Context Manager

Part 1 - Why this matters

Evaluation and parameter updates often need tensor calculations without constructing a backward graph. The `torch.no_grad()` context temporarily disables gradient recording even when its inputs normally require gradients.

Avoiding unnecessary graphs reduces memory use and execution overhead. The context is temporary, so operations performed after leaving it can resume gradient tracking according to their inputs.

Part 2 - Mathematical form

If x requires gradients and:

$$y = f(x)$$

is evaluated inside a no-gradient region, the forward value remains correct but no derivative path is recorded:

$$\frac{\partial y}{\partial x} \quad \text{is not tracked by Autograd.}$$

Part 3 - Hints and Helper

Compute the same expression inside and outside `torch.no_grad()`. Compare values, `requires_grad`, and `grad_fn`, then verify that tracking resumes after the context ends.

Part 4 - Python code skeleton

```
import torch

def E03(x):
    tracked = ...
    with torch.no_grad():
        untracked = ...
    tracked_again = ...
    return (...)

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E03(x))

if __name__ == "__main__":
    main()
```

3.6.5 Exercise 04 - Inference vs Training Mode Gradient

Part 1 - Why this matters

Module training mode and gradient recording control different behaviors. `model.train()` or `model.eval()` changes layers such as dropout and batch normalization, while gradient contexts determine whether Autograd records operations.

Inference normally combines evaluation mode with disabled gradient tracking, but neither setting automatically guarantees the other. Understanding the separation prevents unexpected stochastic outputs or unnecessary graph memory.

Part 2 - Mathematical form

For a model f_θ:

$$\widehat{\mathbf{y}} = f_\theta(\mathbf{x}).$$

Training requires derivatives such as $\nabla_\theta L$, whereas inference generally requires only the forward value $\widehat{\mathbf{y}}$. Evaluation mode may also change the definition of selected layer operations.

Part 3 - Hints and Helper

Run a small module once in training mode with gradients enabled and once in evaluation mode inside an inference or no-gradient context. Compare output properties and parameter gradient availability.

Part 4 - Python code skeleton

```
import torch

def E04(model, X):
    model...
    training_output = ...
    model...
    with torch.no_grad():
        inference_output = ...
    return (...)

def main():
    model = torch.nn...
    X = torch...
    print(E04(model, X))

if __name__ == "__main__":
    main()
```

3.6.6 Exercise 05 - Custom Autograd Function (Forward)

Part 1 - Why this matters

A custom Autograd function defines how a specialized tensor operation participates in PyTorch's graph. Its forward method computes the output and saves only the information that the matching backward calculation will require.

Separating forward logic from saved context encourages careful derivative design. The operation should use tensor expressions, preserve expected shapes, and avoid saving unnecessary data that increases graph memory.

Part 2 - Mathematical form

Define a custom operation:

$$y = x^3.$$

The forward method returns x^3 and saves x because the backward formula will require:

$$\frac{dy}{dx} = 3x^2.$$

Part 3 - Hints and Helper

Subclass `torch.autograd.Function` and implement a static `forward(ctx, input)` method. Save required tensors with `ctx.save_for_backward()` and invoke the operation using `apply()`.

Part 4 - Python code skeleton

```
import torch

class CubeFunction(torch.autograd.Function):
    @staticmethod
```

```
    def forward(ctx, input_tensor):
        ctx.save_for_backward(...)
        output = ...
        return output

def E05(x):
    return CubeFunction.apply(...)

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E05(x))

if __name__ == "__main__":
    main()
```

3.6.7 Exercise 06 - Custom Autograd Function (Backward)

Part 1 - Why this matters

The backward method receives the gradient arriving from later graph nodes and must return one gradient for every forward input. It combines that upstream value with the custom operation's local derivative.

An incorrect backward implementation can run without an obvious forward error yet train a model incorrectly. Manual analysis and numerical gradient checking are therefore essential parts of custom function development.

Part 2 - Mathematical form

For $y = x^3$ and scalar loss $L(y)$:

$$\frac{dL}{dx} = \frac{dL}{dy}\frac{dy}{dx} = \text{grad_output} \cdot 3x^2.$$

The returned gradient must have the same shape as x.

Part 3 - Hints and Helper

Retrieve saved tensors with `ctx.saved_tensors`. Multiply `grad_output` by the local derivative and return the result. Verify the custom operation against ordinary PyTorch operations and `gradcheck`.

Part 4 - Python code skeleton

```
import torch

class CubeFunction(torch.autograd.Function):
    @staticmethod
    def forward(ctx, input_tensor):
        ctx.save_for_backward(...)
        return ...
```

```
    @staticmethod
    def backward(ctx, grad_output):
        input_tensor, = ctx.saved_tensors
        grad_input = ...
        return grad_input

def E06(x):
    y = CubeFunction.apply(x)
    y.backward(...)
    return x.grad

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E06(x))

if __name__ == "__main__":
    main()
```

3.6.8 Exercise 07 - Autograd Graph Memory Analysis

Part 1 - Why this matters

Backward formulas may save forward tensors until gradient computation finishes. Large intermediates, retained graphs, and unnecessary tracking can therefore consume substantial memory even when the final output is small.

Memory analysis connects graph design with practical model capacity. Identifying which tensors remain reachable helps choose between graph retention, forward recomputation, checkpointing, and no-gradient execution.

Part 2 - Mathematical form

If a graph saves intermediates $\mathcal{S} = \{T_1, \ldots, T_k\}$, an approximate stored-data cost is:

$$M \approx \sum_{i=1}^{k} \text{numel}(T_i)\ \text{element_size}(T_i).$$

This excludes additional graph and allocator overhead.

Part 3 - Hints and Helper

Compare memory behavior for a tracked forward pass, a no-gradient pass, and a retained graph. On CUDA, synchronize appropriately and use PyTorch memory statistics; on CPU, focus on saved tensor sizes and graph references.

Part 4 - Python code skeleton

```
import torch

def E07(x):
```

```
    before = ...
    output = ...
    after_forward = ...
    loss = ...
    loss.backward(...)
    after_backward = ...
    return before, after_forward, after_backward

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E07(x))

if __name__ == "__main__":
    main()
```

3.6.9 Exercise 08 - Gradient Hook Implementation

Part 1 - Why this matters

A gradient hook executes when a tensor's gradient is produced during backpropagation. Hooks can observe, log, validate, or transform gradients without rewriting the forward expression.

Hooks are useful for debugging vanishing or exploding values and for specialized gradient transformations. Because a hook can change training behavior, observational hooks should return the original gradient or no replacement.

Part 2 - Mathematical form

Let an arriving gradient be $\mathbf{g}$. A monitoring hook computes a statistic such as:

$$s = \|\mathbf{g}\|_2.$$

A transforming hook instead returns:

$$\widetilde{\mathbf{g}} = H(\mathbf{g}),$$

which replaces the gradient passed farther backward.

Part 3 - Hints and Helper

Register a hook on a leaf or intermediate tensor, record its gradient norm, and run one backward pass. Store the returned handle and remove it afterward to prevent the hook from remaining active unexpectedly.

Part 4 - Python code skeleton

```
import torch

def E08(x):
    records = []
    def hook_function(gradient):
        records.append(...)
```

```
        return ...

    handle = x.register_hook(...)
    loss = ...
    loss.backward()
    handle.remove()
    return records, x.grad

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E08(x))

if __name__ == "__main__":
    main()
```

3.6.10 Exercise 09 - Autograd Failure Debugging

Part 1 - Why this matters

Autograd failures can arise from disabled tracking, detached values, non-scalar outputs, in-place changes, graph reuse, dtype errors, or inspection of non-leaf gradients. Similar symptoms may therefore have different structural causes.

A disciplined diagnosis starts at the loss and follows the graph toward the expected parameter. Tensor metadata, shapes, error messages, and minimal reproductions identify the first point where assumptions and actual graph behavior diverge.

Part 2 - Mathematical form

For a valid scalar loss L and parameter θ:

$$\nabla_\theta L$$

exists in Autograd only when a differentiable recorded path connects θ to L. If every path is cut, the requested gradient is absent rather than mathematically propagated as an ordinary connected value.

Part 3 - Hints and Helper

Inspect dtype, shape, `requires_grad`, `is_leaf`, and `grad_fn`. Check for no-gradient regions, `detach()`, conversions, in-place operations, and reused graphs before applying a fix.

Part 4 - Python code skeleton

```
import torch

def E09(parameter):
    intermediate = ...
    loss = ...
    diagnostics = {
        "loss_shape": ...,
```

```
        "requires_grad": ...,
        "grad_fn": ...,
        "parameter_is_leaf": ...
    }
    loss.backward(...)
    return diagnostics, parameter.grad

def main():
    x = torch.tensor(..., requires_grad=True)
    print(E09(x))

if __name__ == "__main__":
    main()
```

3.7 ABM05 - Backprop Through Linear Layers

Mathematics and Theory

3.7.1 Exercise 00 - Single Linear Layer Backprop

Part 1 - Why this matters

A linear layer combines input features with trainable weights and bias before any activation is applied. Backpropagation must distribute an upstream gradient to the input, weight matrix, and bias vector.

Deriving all three gradients reveals the structure hidden inside a standard PyTorch layer. It also establishes the matrix orientation used throughout deeper neural networks.

Part 2 - Mathematical form

Let $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$ and let $\mathbf{g} = \nabla_{\mathbf{y}} L$. Then:

$$\nabla_{\mathbf{x}} L = \mathbf{W}^{\mathsf{T}}\mathbf{g}, \quad \nabla_{\mathbf{W}} L = \mathbf{g}\mathbf{x}^{\mathsf{T}}, \quad \nabla_{\mathbf{b}} L = \mathbf{g}.$$

Part 3 - Hints and Helper

Create leaf tensors for input, weight, and bias. Produce a scalar loss from the layer output, calculate the three manual gradients, and compare their shapes and values with Autograd.

Part 4 - Python code skeleton

```
import torch

def E00(x, W, bias):
    y = ...
    loss = torch...
    manual_gradients = ...
    loss.backward()
```

```
    return manual_gradients, x.grad, W.grad, bias.grad

def main():
    X = torch...
    W = torch...
    bias = torch...
    print(E00(...))

if __name__ == "__main__":
    main()
```

3.7.2 Exercise 01 - Weight Gradient Verification

Part 1 - Why this matters

Each weight connects one input feature to one output feature. Its gradient is the product of the input activation and the upstream gradient reaching the corresponding output.

Weight-gradient verification exposes the outer-product structure used by linear-layer training. Incorrect transposes become visible because they change both axis meaning and gradient shape.

Part 2 - Mathematical form

For $\mathbf{y} = \mathbf{W}\mathbf{x}$ and upstream gradient $\mathbf{g}$:

$$\frac{\partial L}{\partial W_{ij}} = g_i x_j.$$

Therefore:

$$\nabla_{\mathbf{W}} L = \mathbf{g}\mathbf{x}^{\mathsf{T}}.$$

Part 3 - Hints and Helper

Supply an explicit upstream gradient to `y.backward(gradient)` or define an equivalent scalar loss. Construct the manual outer product and compare it with `W.grad`.

Part 4 - Python code skeleton

```
import torch

def E01(x, W, upstream):
    y = ...
    manual_weight_gradient = ...
    y.backward(...)
    return manual_weight_gradient, W.grad

def main():
    X = torch...
    W = torch...
    bias = torch...
```

```
    print(E01(...))

if __name__ == "__main__":
    main()
```

3.7.3 Exercise 02 - Bias Gradient Verification

Part 1 - Why this matters

A bias value is added directly to its output feature, so its local derivative is one. The bias gradient therefore equals the upstream gradient, summed across any axes where the bias was broadcast.

This simple rule becomes important with batches, where one bias parameter influences many examples. Verifying it prepares the reader to reason about broadcast reduction during backward.

Part 2 - Mathematical form

For one sample:

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b} \quad \Rightarrow \quad \nabla_{\mathbf{b}} L = \mathbf{g}.$$

For a batch with upstream matrix $\mathbf{G}$:

$$\nabla_{\mathbf{b}} L = \sum_{i=1}^{B} \mathbf{G}_{i:}.$$

Part 3 - Hints and Helper

Use a batch so the bias is broadcast across rows. Provide or derive an upstream gradient matrix, sum it along the batch axis manually, and compare the result with `bias.grad`.

Part 4 - Python code skeleton

```
import torch

def E02(X, W, bias, upstream):
    Y = ...
    manual_bias_gradient = ...
    Y.backward(...)
    return manual_bias_gradient, bias.grad

def main():
    X = torch...
    W = torch...
    bias = torch...
    print(E02(...))

if __name__ == "__main__":
    main()
```

3.7.4 Exercise 03 - Batch Linear Layer Backprop

Part 1 - Why this matters

Batch processing applies the same linear transformation to several samples simultaneously. Parameter gradients must aggregate contributions from every row while input gradients remain separate for each sample.

Matrix notation makes this aggregation efficient and explains how batch size enters gradient computation. It also clarifies the different shapes of input, weight, and bias gradients.

Part 2 - Mathematical form

Let $\mathbf{X} \in \mathbb{R}^{B \times D}$ and:

$$\mathbf{Y} = \mathbf{X}\mathbf{W}^{\mathsf{T}} + \mathbf{b}.$$

For $\mathbf{G} = \nabla_{\mathbf{Y}} L$:

$$\nabla_{\mathbf{X}} L = \mathbf{G}\mathbf{W}, \quad \nabla_{\mathbf{W}} L = \mathbf{G}^{\mathsf{T}}\mathbf{X}, \quad \nabla_{\mathbf{b}} L = \sum_{i=1}^{B} \mathbf{G}_{i:}.$$

Part 3 - Hints and Helper

Write all expected shapes before implementing the formulas. Use matrix multiplication and a batch-axis sum, then verify every manual gradient against a single backward pass.

Part 4 - Python code skeleton

```
import torch

def E03(X, W, bias, upstream):
    Y = ...
    manual_dX = ...
    manual_dW = ...
    manual_db = ...
    Y.backward(...)
    return (...)

def main():
    X = torch...
    W = torch...
    bias = torch...
    print(E03(...))

if __name__ == "__main__":
    main()
```

3.7.5 Exercise 04 - Manual vs Autograd Linear Backprop

Part 1 - Why this matters

Manual and automatic backpropagation should produce the same gradients when they use the same forward values, loss reduction, and tensor orientation. Comparing them validates

both the algebra and code.

A mismatch often identifies a forgotten batch sum, incorrect transpose, or confusion between mean and sum loss. The comparison is therefore a focused debugging method rather than only a mathematical exercise.

Part 2 - Mathematical form

For mean squared error:

$$L = \frac{1}{N}\|\mathbf{Y} - \mathbf{T}\|_F^2, \quad \mathbf{G} = \frac{2}{N}(\mathbf{Y} - \mathbf{T}).$$

The linear-layer gradients follow from $\mathbf{G}$ using the batch formulas.

Part 3 - Hints and Helper

Compute the forward output and scalar loss once. Derive the upstream loss gradient with the exact same reduction, calculate all manual layer gradients, and compare them using `torch.allclose()`.

Part 4 - Python code skeleton

```
import torch

def E04(X, W, bias, target):
    Y = ...
    loss = ...
    upstream = ...
    manual_gradients = ...
    loss.backward()
    checks = ...
    return manual_gradients, checks

def main():
    X = torch...
    W = torch...
    bias = torch...
    print(E04(...))

if __name__ == "__main__":
    main()
```

3.7.6 Exercise 05 - Gradient Scaling with Batch Size

Part 1 - Why this matters

The choice between summing and averaging per-sample losses changes how gradient magnitude depends on batch size. A summed loss generally grows with the number of examples, while a mean loss normalizes this effect.

Understanding this scaling is important when changing batch size or learning rate. Two mathematically related reductions can produce very different update magnitudes even on repeated data.

Part 2 - Mathematical form

For per-sample losses ℓ_i:

$$L_{\text{sum}} = \sum_{i=1}^{B} \ell_i, \qquad L_{\text{mean}} = \frac{1}{B} \sum_{i=1}^{B} \ell_i.$$

Therefore:

$$\nabla L_{\text{mean}} = \frac{1}{B} \nabla L_{\text{sum}}.$$

Part 3 - Hints and Helper

Use identical repeated samples and compute parameter gradients once with a sum and once with a mean. Clear gradient buffers and rebuild the forward graph between experiments, then compare their norm ratio.

Part 4 - Python code skeleton

```
import torch

def E05(X, W, bias, target):
    sum_loss = ...
    sum_loss.backward()
    sum_gradient = W.grad...
    W.grad = ...
    mean_loss = ...
    mean_loss.backward()
    mean_gradient = W.grad...
    return (...)

def main():
    X = torch...
    W = torch...
    bias = torch...
    print(E05(...))

if __name__ == "__main__":
    main()
```

3.7.7 Exercise 06 - Frozen Layer Gradient Check

Part 1 - Why this matters

Freezing a layer prevents its parameters from receiving gradients and being updated while still allowing its forward output to feed later trainable layers. This is central to transfer learning and staged training.

A frozen parameter is different from a detached activation. Correct freezing leaves gradient flow available for later components and, when needed, for earlier inputs connected through the fixed operation.

Part 2 - Mathematical form

For $\mathbf{h} = \mathbf{W}_f\mathbf{x}$ with frozen $\mathbf{W}_f$:

$$\nabla_{\mathbf{W}_f} L \text{ is not requested,}$$

while a downstream trainable layer may still receive:

$$\nabla_{\mathbf{W}_t} L \neq 0.$$

Part 3 - Hints and Helper

Set `requires_grad_(False)` on only the intended parameters. Run forward and backward, then inspect frozen and trainable parameter gradients without wrapping the whole forward pass in `no_grad()`.

Part 4 - Python code skeleton

```
import torch

def E06(X, frozen_layer, trainable_layer):
    frozen_layer...
    hidden = ...
    output = ...
    loss = ...
    loss.backward()
    return (...)

def main():
    X = torch...
    W = torch...
    bias = torch...
    print(E06(...))

if __name__ == "__main__":
    main()
```

3.7.8 Exercise 07 - Linear Stack Gradient Flow

Part 1 - Why this matters

Stacking linear layers causes gradients to be repeatedly multiplied by transposed weight matrices. The result explains how early-layer updates depend on every later layer in the chain.

Although a stack without activations is forward-equivalent to one linear transformation, its parameterization creates separate gradient tensors. Their magnitudes reveal how weight scales affect deep gradient flow.

Part 2 - Mathematical form

Let:

$$\mathbf{h} = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1, \qquad \mathbf{y} = \mathbf{W}_2\mathbf{h} + \mathbf{b}_2.$$

For upstream $\mathbf{g}$:

$$\nabla_{\mathbf{h}} L = \mathbf{W}_2^{\mathsf{T}}\mathbf{g}, \qquad \nabla_{\mathbf{x}} L = \mathbf{W}_1^{\mathsf{T}}\nabla_{\mathbf{h}} L.$$

Part 3 - Hints and Helper

Retain the hidden tensor gradient and perform one backward pass. Calculate the manual reverse flow from output to hidden to input, then compare values and gradient norms at each stage.

Part 4 - Python code skeleton

```
import torch

def E07(x, W1, b1, W2, b2):
    hidden = ...
    hidden.retain_grad()
    output = ...
    loss = ...
    manual_flow = ...
    loss.backward()
    return manual_flow, hidden.grad, x.grad

def main():
    X = torch...
    W = torch...
    bias = torch...
    print(E07(...))

if __name__ == "__main__":
    main()
```

3.7.9 Exercise 08 - Numerical Stability in Backprop

Part 1 - Why this matters

Large inputs or weights can create extremely large forward values and gradients, while small repeated factors can drive gradients toward zero. Limited floating-point range can then produce overflow, underflow, or loss of precision.

Monitoring finite values and gradient norms is essential before applying a parameter update. Higher precision, normalization, careful initialization, and gradient clipping address different sources of instability.

Part 2 - Mathematical form

For a depth-K scalar linear stack:

$$y = \left(\prod_{k=1}^{K} w_k \right) x.$$

The input derivative is:

$$\frac{dy}{dx} = \prod_{k=1}^{K} w_k.$$

Thus repeated $|w_k| > 1$ can amplify gradients, while repeated $|w_k| < 1$ can shrink them.

Part 3 - Hints and Helper

Build a controllable linear stack and test several depths and scales. Inspect `torch.isfinite()` and gradient norms, and compare float32 with float64 without using clipping to hide the original behavior.

Part 4 - Python code skeleton

```
import torch

def E08(x, weights):
    value = x
    for weight in weights:
        value = ...
    loss = ...
    loss.backward()
    diagnostics = ...
    return value, diagnostics

def main():
    X = torch...
    W = torch...
    bias = torch...
    print(E08(...))

if __name__ == "__main__":
    main()
```

3.7.10 Exercise 09 - End-to-End Linear Backprop Challenge

Part 1 - Why this matters

An end-to-end exercise combines batched input, multiple linear layers, a scalar loss, manual gradient propagation, and Autograd verification. It tests whether every earlier rule remains consistent in one pipeline.

The goal is not merely to obtain gradients, but to explain their shapes and trace them from the loss through every parameter and intermediate. This mirrors the analysis required in a small trainable model.

Part 2 - Mathematical form

Let:

$$\mathbf{H} = \mathbf{X}\mathbf{W}_1^\top + \mathbf{b}_1, \quad \mathbf{Y} = \mathbf{H}\mathbf{W}_2^\top + \mathbf{b}_2,$$

and $L = \frac{1}{N}\|\mathbf{Y} - \mathbf{T}\|_F^2$. Starting with $\nabla_{\mathbf{Y}} L$, apply the batch linear formulas backward through layer two and then layer one.

Part 3 - Hints and Helper

Record every forward shape, derive the loss gradient with the correct reduction, and compute all manual gradients in reverse order. Compare each result separately with Autograd and report the largest error.

Part 4 - Python code skeleton

```
import torch

def E09(X, W1, b1, W2, b2, target):
    hidden = ...
    output = ...
    loss = ...
    manual_gradients = ...
    loss.backward()
    comparisons = ...
    return loss, manual_gradients, comparisons

def main():
    X = torch...
    W = torch...
    bias = torch...
    print(E09(...))

if __name__ == "__main__":
    main()
```

3.8 ABM06 - Gradient Flow Debugging

Mathematics and Theory

3.8.1 Exercise 00 - Detecting Vanishing Gradients

Part 1 - Why this matters

Vanishing gradients occur when derivatives become extremely small as they pass through many layers. Early parameters then receive almost no learning signal, so their values change slowly even while later layers continue to train.

Detection requires comparing gradients across depth and over multiple steps. A small loss change alone does not identify vanishing flow; the parameter-gradient magnitudes provide direct evidence.

Part 2 - Mathematical form

For a chain with local derivatives $d_1, \ldots, d_K$:

$$\frac{\partial L}{\partial h_0} = \frac{\partial L}{\partial h_K} \prod_{k=1}^{K} d_k.$$

If $|d_k| < 1$ repeatedly, the product may approach zero as K grows.

Part 3 - Hints and Helper

Build a deep scalar or linear chain with controlled weight values. After backward, record every retained intermediate or parameter gradient norm and flag values below a chosen threshold without assuming one universal threshold.

Part 4 - Python code skeleton

```
import torch

def E00(model, X, target, threshold):
    output = ...
    loss = ...
    loss.backward()
    gradient_norms = ...
    vanishing_flags = ...
    return loss, gradient_norms, vanishing_flags

def main():
    model = torch.nn...
    X = torch...
    target = torch...
    print(E00(...))

if __name__ == "__main__":
    main()
```

3.8.2 Exercise 01 - Detecting Exploding Gradients

Part 1 - Why this matters

Exploding gradients arise when repeated derivative multiplication creates very large values. Parameter updates can become unstable, the loss may oscillate, and floating-point values may eventually become infinite or NaN.

Early detection should occur before the optimizer step corrupts parameters. Monitoring both magnitude and finiteness distinguishes rapid growth from a gradient that has already become numerically invalid.

Part 2 - Mathematical form

For a depth-K chain:

$$\left\| \frac{\partial L}{\partial h_0} \right\| \leq \left\| \frac{\partial L}{\partial h_K} \right\| \prod_{k=1}^{K} \|J_k\|.$$

Repeated Jacobian norms greater than one can amplify the gradient exponentially.

Part 3 - Hints and Helper

Create a controlled deep chain and inspect gradient norms after backward but before updating parameters. Check `torch.isfinite()` and compare each norm with a selected alert threshold.

Part 4 - Python code skeleton

```
import torch

def E01(model, X, target, threshold):
    loss = ...
    loss.backward()
    norms = ...
    finite_checks = ...
    exploding_flags = ...
    return norms, finite_checks, exploding_flags

def main():
    model = torch.nn...
    X = torch...
    target = torch...
    print(E01(...))

if __name__ == "__main__":
    main()
```

3.8.3 Exercise 02 - Gradient Norm Monitoring

Part 1 - Why this matters

A gradient tensor may contain many values, so its norm provides a compact measure of update magnitude. Recording norms across steps reveals trends that individual gradient prints cannot show clearly.

Norm monitoring supports diagnosis but does not replace inspecting shapes, signs, or distributions. A healthy range depends on the model, layer scale, loss reduction, and precision, so history and layer comparisons matter.

Part 2 - Mathematical form

For parameter gradient $\mathbf{g}$, the L_2 norm is:

$$\|\mathbf{g}\|_2 = \sqrt{\sum_i g_i^2}.$$

A global norm across parameters is:

$$G = \sqrt{\sum_p \|\nabla_p L\|_2^2}.$$

Part 3 - Hints and Helper

After backward, iterate over named parameters and skip entries whose gradient is `None`. Record per-parameter norms and compute a global norm without changing the gradients.

Part 4 - Python code skeleton

```
import torch

def E02(model):
    records = {}
    squared_total = ...
    for name, parameter in model.named_parameters():
        if parameter.grad is not None:
            norm = ...
            records[name] = norm
            squared_total = ...
    global_norm = ...
    return records, global_norm

def main():
    model = torch.nn...
    X = torch...
    target = torch...
    print(E02(...))

if __name__ == "__main__":
    main()
```

3.8.4 Exercise 03 - Layer-wise Gradient Inspection

Part 1 - Why this matters

A single global norm can hide which layer causes a problem. Layer-wise inspection compares gradient magnitude, finiteness, and parameter scale throughout the network to locate where useful flow disappears or instability begins.

The resulting profile should be interpreted in network order. A sudden change between neighboring layers often provides more useful evidence than declaring every small or large value independently abnormal.

Part 2 - Mathematical form

For layer ℓ with parameters θ_ℓ, define:

$$G_\ell = \|\nabla_{\theta_\ell} L\|_2.$$

A scale-aware ratio can be:

$$R_\ell = \frac{G_\ell}{\|\theta_\ell\|_2 + \varepsilon}.$$

Both quantities should be reported rather than relying on one alone.

Part 3 - Hints and Helper

Group named parameters by layer, then record parameter norms, gradient norms, ratios, and finite checks after one backward pass. Preserve network order so adjacent layers can be compared.

Part 4 - Python code skeleton

```
import torch

def E03(model, epsilon=1e-12):
    report = {}
    for name, parameter in model.named_parameters():
        gradient_norm = ...
        parameter_norm = ...
        ratio = ...
        report[name] = (...)
    return report

def main():
    model = torch.nn...
    X = torch...
    target = torch...
    print(E03(...))

if __name__ == "__main__":
    main()
```

3.8.5 Exercise 04 - Broken Graph Debugging

Part 1 - Why this matters

A broken graph prevents the loss from reaching an expected parameter. Common causes include `detach()`, no-gradient contexts, tensor reconstruction, conversion to Python or NumPy, and using an unrelated tensor in the loss.

The key symptom is often a missing gradient rather than a wrong numeric value. Debugging should follow the dependency path backward and locate the first tensor whose history differs from the intended graph.

Part 2 - Mathematical form

For a required path:

$$\theta \longrightarrow h_1 \longrightarrow \cdots \longrightarrow L,$$

Autograd can compute $\nabla_\theta L$ only if every edge is recorded and differentiable. If one intermediate becomes stopgrad(h_j), the earlier path is disconnected.

Part 3 - Hints and Helper

Print `requires_grad`, `is_leaf`, and `grad_fn` for each intermediate. Find the earliest unexpected loss of graph history and correct that operation rather than assigning a gradient manually.

Part 4 - Python code skeleton

```
import torch

def E04(parameter, X):
    first = ...
    suspicious = ...
    output = ...
    loss = ...
    diagnostics = ...
    loss.backward(...)
    return diagnostics, parameter.grad

def main():
    model = torch.nn...
    X = torch...
    target = torch...
    print(E04(...))

if __name__ == "__main__":
    main()
```

3.8.6 Exercise 05 - requires_grad Misuse Debugging

Part 1 - Why this matters

Incorrect `requires_grad` settings can track constant data unnecessarily or prevent trainable parameters from receiving gradients. Enabling the flag after a disconnected computation does not recreate the history that was already lost.

Correct configuration begins by classifying tensors as trainable parameters, differentiable inputs, constants, or temporary intermediates. Only the intended leaves normally need explicit setup.

Part 2 - Mathematical form

If $y = f(x)$ is computed while x does not require gradients, later changing a disconnected tensor's flag cannot reconstruct:

$$x \longrightarrow y.$$

Gradient tracking must be enabled on the appropriate leaf before the forward operations whose derivatives are needed.

Part 3 - Hints and Helper

List every relevant tensor with its role and metadata before the forward pass. Check dtype and leaf status, then repair the original tensor creation or parameter configuration and rebuild the graph.

Part 4 - Python code skeleton

```
import torch

def E05(tensors):
    diagnostics = {}
    for name, tensor in tensors.items():
        diagnostics[name] = (...)
    corrected_tensors = ...
    return diagnostics, corrected_tensors

def main():
    model = torch.nn...
    X = torch...
    target = torch...
    print(E05(...))

if __name__ == "__main__":
    main()
```

3.8.7 Exercise 06 - Gradient Blocking Detection

Part 1 - Why this matters

Gradient blocking may be intentional, as with frozen targets, or accidental, as with detachments and saturated activations. A zero or missing gradient must therefore be traced to determine whether the behavior matches the design.

Comparing forward activity with backward flow helps locate the boundary. An intermediate can have a meaningful nonzero forward value while allowing little or no derivative to pass to its ancestors.

Part 2 - Mathematical form

For $h = f(x)$ and $y = g(h)$:

$$\frac{dy}{dx} = \frac{dy}{dh}\frac{dh}{dx}.$$

Flow is blocked if a factor is exactly zero, if h is detached, or if the implemented operation is not differentiable along the selected path.

Part 3 - Hints and Helper

Retain gradients on intermediate tensors and compare their norms from output toward input. Inspect activation ranges and graph metadata, and distinguish a zero tensor from `None`, since they indicate different situations.

Part 4 - Python code skeleton

```
import torch

def E06(model, X, target):
    intermediates = ...
    for tensor in intermediates:
```

```
        tensor.retain_grad()
    loss = ...
    loss.backward()
    flow_report = ...
    return flow_report

def main():
    model = torch.nn...
    X = torch...
    target = torch...
    print(E06(...))

if __name__ == "__main__":
    main()
```

3.8.8 Exercise 07 - Hook-based Gradient Debugging

Part 1 - Why this matters

Hooks observe gradients at the moment they pass through selected tensors or parameters. They allow a model to collect a backward trace without adding gradient-printing logic to every training operation.

A diagnostic hook should record compact statistics and avoid unintended gradient modification. Hook handles must be removed after the experiment so repeated registrations do not duplicate logs or affect later runs.

Part 2 - Mathematical form

For a gradient $\mathbf{g}$, a hook may record:

$$\min(\mathbf{g}), \quad \max(\mathbf{g}), \quad \text{mean}(\mathbf{g}), \quad \|\mathbf{g}\|_2.$$

Returning $\mathbf{g}$ unchanged preserves the original backward computation.

Part 3 - Hints and Helper

Register hooks on selected named parameters, store detached statistics, run one backward pass, and remove every handle. Include shape and finite-value checks in the report.

Part 4 - Python code skeleton

```
import torch

def E07(model, loss):
    records = {}
    handles = []
    for name, parameter in model.named_parameters():
        handle = parameter.register_hook(...)
        handles.append(handle)
    loss.backward()
```

```
    for handle in handles:
        handle.remove()
    return records

def main():
    model = torch.nn...
    X = torch...
    target = torch...
    print(E07(...))

if __name__ == "__main__":
    main()
```

3.8.9 Exercise 08 - Training Collapse Diagnosis

Part 1 - Why this matters

Training collapse may appear as constant predictions, NaN loss, zero gradients, extreme updates, or failure to improve. These symptoms can originate from data, forward activations, loss configuration, gradients, optimizer settings, or parameter corruption.

Diagnosis should inspect the pipeline in order rather than immediately changing the model. A reproducible batch and measurements taken before and after each stage make it possible to identify when valid values first become abnormal.

Part 2 - Mathematical form

For update:

$$\theta_{t+1} = \theta_t - \eta g_t,$$

collapse may occur when $g_t \approx 0$, when $\|\eta g_t\|$ is excessive, or when either θ_t, L_t, or g_t is non-finite. Relative update size can be monitored as:

$$U_t = \frac{\|\eta g_t\|}{\|\theta_t\| + \varepsilon}.$$

Part 3 - Hints and Helper

Check input and target statistics, forward activation ranges, loss finiteness, gradient norms, learning rate, and parameter values in that order. Do not perform an optimizer step until all pre-update diagnostics are recorded.

Part 4 - Python code skeleton

```
import torch

def E08(model, X, target, learning_rate):
    diagnostics = {}
    output = ...
    loss = ...
    diagnostics["forward"] = ...
```

```
    loss.backward()
    diagnostics["gradients"] = ...
    diagnostics["relative_updates"] = ...
    return diagnostics

def main():
    model = torch.nn...
    X = torch...
    target = torch...
    print(E08(...))

if __name__ == "__main__":
    main()
```

3.8.10 Exercise 09 - Full Backprop Debugging Scenario

Part 1 - Why this matters

A complete debugging scenario combines shape validation, graph connectivity, activation inspection, gradient monitoring, and update analysis. The objective is to locate a fault using evidence rather than replacing several components at once.

The final report should identify the first abnormal stage, explain its downstream effects, and propose one targeted correction. Repeating the same controlled batch after that correction verifies whether the diagnosis was accurate.

Part 2 - Mathematical form

For a pipeline:

$$\mathbf{X} \to h_1 \to \cdots \to \widehat{\mathbf{Y}} \to L \to \nabla_\theta L \to \theta',$$

validate each transition. Useful invariants include compatible shapes, finite values, a recorded path to trainable parameters, and an update magnitude appropriate to parameter scale.

Part 3 - Hints and Helper

Create one deliberately faulty small model or training step. Capture input, forward, loss, graph, gradient, and proposed-update diagnostics; name the earliest failing check and change only its direct cause before rerunning.

Part 4 - Python code skeleton

```
import torch

def E09(model, X, target, optimizer):
    report = {}
    report["input"] = ...
    output = ...
    report["forward"] = ...
    loss = ...
    report["loss"] = ...
```

```
    optimizer.zero_grad(...)
    loss.backward()
    report["gradients"] = ...
    report["diagnosis"] = ...
    return report

def main():
    model = torch.nn...
    X = torch...
    target = torch...
    print(E09(...))

if __name__ == "__main__":
    main()
```

Book 4

Optimization in Code

4.1 Introduction

4.1.1 What will the book teach you

Training an artificial intelligence model is not completed by calculating a gradient. A gradient only describes how a loss changes locally with respect to the model parameters. Optimization is the process that converts this information into a sequence of controlled parameter updates, with the objective of moving the model toward better solutions while preserving numerical and training stability. This book teaches how that process works mathematically and how it is implemented directly in PyTorch code.

You will begin with the fundamental gradient descent update and construct a training loop without hiding its individual steps. For a parameter vector $\boldsymbol{\theta}$, a loss function $L(\boldsymbol{\theta})$, and a learning rate η, the basic update is:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}_t).$$

Although this equation is short, its behavior depends on the scale and direction of the gradient, the geometry of the loss surface, the learning rate, the parameter initialization, the noise in the data, and the history of previous updates. The exercises will separate these factors so that you can observe how each one changes the optimization path.

The learning rate will be studied as more than a number selected by trial and error. You will examine how a small rate can make useful progress impractically slow, while a large rate can overshoot a minimum, oscillate, or cause divergence. You will implement learning-rate sweeps, schedules, warm-up strategies, and adaptive logic, then use recorded loss values and update behavior to distinguish slow convergence from instability. The objective is to develop evidence-based tuning skills instead of changing hyperparameters without a diagnosis.

After mastering vanilla gradient descent, you will study optimizers that maintain an internal state. Momentum stores a velocity that combines current and previous gradients, helping an update preserve useful direction and reduce oscillation. RMSProp stores a running average of squared gradients to adapt the update scale for each parameter. Adam combines estimates of the first and second moments and corrects their initialization bias. You will implement these equations manually before using or reproducing optimizer classes, so variables such as velocity, decay rate, moment estimates, bias correction, and numerical epsilon have clear mathematical

meanings.

This book also treats convergence as a measurable process rather than the simple completion of a fixed number of epochs. You will track loss curves, gradient norms, parameter changes, plateaus, oscillations, non-finite values, and long-run behavior. You will implement early stopping and stability checks, compare optimizers under controlled conditions, and investigate noisy gradients, vanishing gradients, exploding gradients, and training collapse. These exercises will teach you to determine whether training is improving, stalled, unstable, or only appearing successful because the chosen metric is incomplete.

Weight initialization is included because optimization begins before the first parameter update. Initial values determine the scale of forward activations and backward gradients, influence symmetry between units, and can make a model easy or difficult to optimize. You will compare zero, uniform, normal, Xavier, and He initialization, then connect their assumptions to activation behavior and gradient flow. The goal is not to memorize initialization function names, but to understand why different network structures require different starting scales.

4.1.2 How to approach the exercises

Every exercise should be treated as a controlled experiment. Begin by writing the mathematical update, predicting the expected behavior, and identifying which values must be recorded. Then implement the incomplete code, run it with a simple reproducible example, and compare the observation with the prediction. When two optimizers or hyperparameters are compared, keep the data, model, initialization, loss function, and random seed consistent so that the comparison measures the intended variable.

Do not judge an optimizer from one final loss value. Record the complete loss history, gradient statistics, parameter updates, convergence speed, and signs of instability. When training fails, locate the earliest abnormal stage before applying a correction. This method will prepare you to design and debug real training systems in which several optimization choices interact simultaneously.

4.1.3 The Chapters (OC00 - OC06) included in this book

- OC00 - Gradient Descent Implementation
- OC01 - Learning Rate Effects & Tuning
- OC02 - Momentum Update Rule
- OC03 - RMSProp Logic & Behavior
- OC04 - Adam Optimizer Math + Code
- OC05 - Convergence & Stability
- OC06 - Weight Initialization Impact

4.2 OC00 - Gradient Descent Implementation

Mathematics and Theory

4.2.1 Exercise 00 - Simple Gradient Descent Step

Part 1 - Why this matters

Gradient descent changes a parameter in the direction opposite to the local loss gradient. A single update is the smallest unit of optimization and makes the roles of the gradient and learning rate completely visible.

Studying one step before building a loop prevents parameter updates from becoming hidden optimizer behavior. It also establishes the rule that gradients are computed from the old parameter value before the new value is created.

Part 2 - Mathematical form

For parameter θ_t, gradient $g_t = \nabla_\theta L(\theta_t)$, and learning rate $\eta > 0$:

$$\theta_{t+1} = \theta_t - \eta g_t.$$

If $g_t > 0$, the update decreases θ; if $g_t < 0$, it increases θ.

Part 3 - Hints and Helper

Accept a parameter, its gradient, and a learning rate. Compute a new parameter without modifying the original tensor in place, then report the update displacement and verify its direction.

Part 4 - Python code skeleton

```
import torch

def E00(parameter, gradient, learning_rate):
    update = ...
    new_parameter = ...
    return new_parameter, update

def main():
    parameter = torch.tensor(..., requires_grad=True)
    print(E00(...))

if __name__ == "__main__":
    main()
```

4.2.2 Exercise 01 - Gradient Descent on Scalar Loss

Part 1 - Why this matters

A scalar objective allows the complete optimization path to be observed one value at a time. The derivative identifies the local downhill direction, while repeated updates reveal whether the chosen step size approaches the minimum.

This controlled example connects calculus, Autograd, and optimization without model complexity. It also shows that a decreasing loss is an observed result of appropriate updates, not a guarantee of every gradient descent configuration.

Part 2 - Mathematical form

Let:

$$L(\theta) = (\theta - a)^2, \qquad \frac{dL}{d\theta} = 2(\theta - a).$$

Gradient descent applies:

$$\theta_{t+1} = \theta_t - 2\eta(\theta_t - a),$$

whose minimum is at $\theta = a$.

Part 3 - Hints and Helper

Create a scalar leaf parameter and compute the loss and gradient. Apply the update inside `torch.no_grad()`, clear the gradient afterward, and record both old and new loss values.

Part 4 - Python code skeleton

```
import torch

def E01(parameter, target, learning_rate):
    loss = ...
    loss.backward()
    gradient = parameter.grad...
    with torch.no_grad():
        parameter ...
    parameter.grad = ...
    return loss, gradient, parameter

def main():
    parameter = torch.tensor(..., requires_grad=True)
    print(E01(...))

if __name__ == "__main__":
    main()
```

4.2.3 Exercise 02 - Gradient Descent on Vector Parameters

Part 1 - Why this matters

Models contain vectors and tensors of parameters rather than one scalar. Gradient descent applies the same update element by element, with a gradient tensor whose shape matches the parameter tensor.

Vector optimization introduces directional geometry. The negative gradient is the direction of steepest local decrease under the Euclidean norm, while its magnitude controls how far the learning-rate-scaled update moves.

Part 2 - Mathematical form

For $\boldsymbol{\theta} \in \mathbb{R}^n$ and:

$$L(\boldsymbol{\theta}) = \|\boldsymbol{\theta} - \mathbf{a}\|_2^2,$$

the gradient is:

$$\nabla_{\boldsymbol{\theta}} L = 2(\boldsymbol{\theta} - \mathbf{a}),$$

and $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \nabla L$.

Part 3 - Hints and Helper

Check that the parameter, target, and gradient shapes match. Compute the vector loss with a reduction, call backward, update every component without tracking the update, and clear the gradient buffer.

Part 4 - Python code skeleton

```
import torch

def E02(parameters, target, learning_rate):
    loss = torch...
    loss.backward()
    gradient = parameters.grad...
    with torch.no_grad():
        parameters ...
    parameters.grad = ...
    return loss, gradient, parameters

def main():
    parameter = torch.tensor(..., requires_grad=True)
    print(E02(...))

if __name__ == "__main__":
    main()
```

4.2.4 Exercise 03 - Learning Rate Effect Observation

Part 1 - Why this matters

The learning rate scales every gradient step and strongly affects the optimization trajectory. A very small value can make progress slow, while a large value can overshoot the minimum or move the loss into an unstable region.

A fair learning-rate experiment must begin each run from the same parameter value and use the same number of steps. Tracking the complete history reveals speed, oscillation, and divergence that a final value may hide.

Part 2 - Mathematical form

For $L(\theta) = \frac{1}{2} c \theta^2$:

$$\theta_{t+1} = (1 - \eta c)\theta_t.$$

Convergence to zero requires:

$$|1 - \eta c| < 1,$$

showing that an acceptable learning rate depends on the curvature c.

Part 3 - Hints and Helper

Run several learning rates from identical initial conditions. Store parameter and loss histories separately for every run, and compare early progress, oscillation, and final distance from the minimum.

Part 4 - Python code skeleton

```
import torch

def E03(initial_value, learning_rates, steps):
    histories = {}
    for learning_rate in learning_rates:
        parameter = ...
        history = []
        for step in range(steps):
            loss = ...
            ...
        histories[learning_rate] = history
    return histories

def main():
    parameter = torch.tensor(..., requires_grad=True)
    print(E03(...))

if __name__ == "__main__":
    main()
```

4.2.5 Exercise 04 - Convergence vs Divergence Test

Part 1 - Why this matters

An optimization loop needs measurable criteria for deciding whether it is approaching a solution or becoming unstable. Loss decrease alone may be noisy, so parameter changes, gradient norms, and finite-value checks provide additional evidence.

Automatic detection helps stop harmful runs early and makes hyperparameter experiments comparable. The classification should be based on a defined observation window rather than one isolated fluctuation.

Part 2 - Mathematical form

Possible convergence indicators include:

$$|L_t - L_{t-1}| < \varepsilon_L, \qquad \|\nabla L_t\| < \varepsilon_g.$$

Divergence indicators may include non-finite values or sustained growth such as:

$$L_t > \rho L_{t-k}, \qquad \rho > 1.$$

Part 3 - Hints and Helper

Track loss, gradient norm, and parameter displacement at every step. Define tolerances and a patience window, then classify the run as converging, diverging, or inconclusive without relying on exact equality.

Part 4 - Python code skeleton

```
import torch

def E04(loss_history, gradient_history, tolerance, patience):
    finite = ...
    plateau = ...
    sustained_growth = ...
    status = ...
    return status

def main():
    parameter = torch.tensor(..., requires_grad=True)
    print(E04(...))

if __name__ == "__main__":
    main()
```

4.2.6 Exercise 05 - Manual Parameter Update Loop

Part 1 - Why this matters

A complete gradient descent loop repeatedly performs forward computation, loss calculation, gradient clearing, backpropagation, and parameter update. The order matters because stale gradients or tracked update operations change later behavior.

Implementing the loop manually exposes every state transition that high-level optimizers normally manage. This makes it easier to debug a model before introducing momentum or adaptive optimizer state.

Part 2 - Mathematical form

At each iteration t:

$$L_t = L(\theta_t), \quad g_t = \nabla_\theta L_t, \quad \theta_{t+1} = \theta_t - \eta g_t.$$

The gradient buffer must represent g_t alone unless deliberate accumulation is intended.

Part 3 - Hints and Helper

Write the loop without `torch.optim`. Clear gradients before or after each iteration consistently, update parameters under `torch.no_grad()`, and append detached scalar values to history.

Part 4 - Python code skeleton

```
import torch

def E05(parameter, target, learning_rate, steps):
    history = []
    for step in range(steps):
        loss = ...
        ...
        history.append(...)
    return parameter, history

def main():
    parameter = torch.tensor(..., requires_grad=True)
    print(E05(...))

if __name__ == "__main__":
    main()
```

4.2.7 Exercise 06 - Gradient Descent without Autograd

Part 1 - Why this matters

Manual differentiation separates the optimizer from PyTorch's automatic gradient engine. It tests whether the analytical gradient and update rule are understood independently of graph construction.

This method is practical for simple objectives and useful for verification. It becomes difficult for large models, which explains why Autograd automates derivatives while the mathematical optimizer rule remains unchanged.

Part 2 - Mathematical form

For:

$$L(\theta) = (\theta - a)^4,$$

the analytical gradient is:

$$\frac{dL}{d\theta} = 4(\theta - a)^3.$$

Thus:

$$\theta_{t+1} = \theta_t - 4\eta(\theta_t - a)^3.$$

Part 3 - Hints and Helper

Use ordinary tensors or Python numeric values without `requires_grad`. Implement the analytical gradient directly, record the loss before each update, and compare the path with an Autograd version.

Part 4 - Python code skeleton

```
import torch

def E06(parameter, target, learning_rate, steps):
```

```
    history = []
    for step in range(steps):
        loss = ...
        gradient = ...
        parameter = ...
        history.append(...)
    return parameter, history

def main():
    parameter = torch.tensor(..., requires_grad=True)
    print(E06(...))

if __name__ == "__main__":
    main()
```

4.2.8 Exercise 07 - Loss Curve Tracking Challenge

Part 1 - Why this matters

A loss curve records optimization behavior across time. Its shape can reveal smooth convergence, plateaus, oscillation, sudden instability, or noisy progress that one initial and final value cannot communicate.

Reliable tracking requires storing detached numeric values without preserving every computational graph. The history should also include step numbers and relevant experimental settings so different runs can be interpreted correctly.

Part 2 - Mathematical form

For iterations $t = 0, \ldots, T-1$, define the history:

$$\mathcal{H} = \{(t, L_t)\}_{t=0}^{T-1}.$$

A simple improvement measure is:

$$\Delta_t = L_{t-1} - L_t,$$

where positive Δ_t indicates a one-step decrease.

Part 3 - Hints and Helper

Record `loss.item()` after each forward pass and avoid storing live loss tensors. Return the history and summary statistics such as best loss, best step, final loss, and number of increases.

Part 4 - Python code skeleton

```
import torch

def E07(parameter, loss_function, learning_rate, steps):
    history = []
    for step in range(steps):
        loss = loss_function(...)
        history.append(...)
```

```
        ...
    summary = ...
    return history, summary

def main():
    parameter = torch.tensor(..., requires_grad=True)
    print(E07(...))

if __name__ == "__main__":
    main()
```

4.2.9 Exercise 08 - Gradient Clipping Integration

Part 1 - Why this matters

Gradient clipping limits an update-driving gradient before it reaches the parameter update. It can reduce damage from occasional extreme gradients, but it does not correct the underlying source of instability.

Integrating clipping in the correct order is essential: gradients must first be computed, then measured and clipped, and only afterward used for the update. Monitoring pre- and post-clipping norms shows whether clipping is frequently active.

Part 2 - Mathematical form

For gradient $\mathbf{g}$ and maximum norm $c > 0$:

$$\widetilde{\mathbf{g}} = \mathbf{g} \min\left(1, \frac{c}{\|\mathbf{g}\|_2 + \varepsilon}\right).$$

The update becomes:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \widetilde{\mathbf{g}}.$$

Part 3 - Hints and Helper

After backward, record the original norm, apply `torch.nn.utils.clip_grad_norm_()`, record the clipped norm, and update parameters. Do not clip before gradients exist.

Part 4 - Python code skeleton

```
import torch

def E08(parameters, loss, learning_rate, max_norm):
    loss.backward()
    before_norm = ...
    torch.nn.utils...
    after_norm = ...
    with torch.no_grad():
        for parameter in parameters:
            parameter ...
    return before_norm, after_norm
```

```
def main():
    parameter = torch.tensor(..., requires_grad=True)
    print(E08(...))

if __name__ == "__main__":
    main()
```

4.2.10 Exercise 09 - End-to-End Gradient Descent Challenge

Part 1 - Why this matters

An end-to-end optimization task combines model prediction, scalar loss, Autograd, gradient diagnostics, parameter updates, and stopping logic. Every stage must cooperate for learning to occur correctly.

The final challenge should produce more than trained parameters. It should return evidence describing the optimization path, including loss history, gradient norms, stopping reason, and checks for non-finite or divergent behavior.

Part 2 - Mathematical form

For linear regression:

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b, \qquad L = \frac{1}{N}\|\hat{\mathbf{y}} - \mathbf{y}\|_2^2.$$

Gradient descent updates both parameters:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla_{\mathbf{w}} L, \quad b_{t+1} = b_t - \eta \frac{\partial L}{\partial b}.$$

Part 3 - Hints and Helper

Initialize trainable weight and bias tensors, implement the full loop without `torch.optim`, monitor loss and gradient norms, and stop safely on convergence or non-finite values. Preserve no live graphs in the returned history.

Part 4 - Python code skeleton

```
import torch

def E09(X, target, weight, bias, learning_rate, max_steps):
    history = []
    status = ...
    for step in range(max_steps):
        prediction = ...
        loss = ...
        ...
        history.append(...)
        if ...:
            break
    return weight, bias, history, status
```

```
def main():
    parameter = torch.tensor(..., requires_grad=True)
    print(E09(...))

if __name__ == "__main__":
    main()
```

4.3 OC01 - Learning Rate Effects & Tuning

Mathematics and Theory

4.3.1 Exercise 00 - Fixed Learning Rate Experiment

Part 1 - Why this matters

A fixed learning rate keeps the update scale constant throughout training. It provides a stable baseline for observing how one selected value interacts with the objective's curvature and gradient magnitude.

Because no schedule or adaptive logic changes the rate, every difference across steps comes from the evolving loss surface. This makes the experiment useful for learning how a simple optimizer behaves.

Part 2 - Mathematical form

For fixed $\eta > 0$:

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t), \qquad t = 0, \ldots, T-1.$$

The same η is used at every iteration even though the gradient changes.

Part 3 - Hints and Helper

Choose a simple objective and record the step, learning rate, parameter, loss, gradient, and update size. Verify that the learning rate remains constant and avoid storing live graphs in history.

Part 4 - Python code skeleton

```
import torch

def E00(parameter, learning_rate, steps):
    history = []
    for step in range(steps):
        loss = ...
        ...
        history.append(...)
    return parameter, history

def main():
    print(E00(...))
```

```
if __name__ == "__main__":
    main()
```

4.3.2 Exercise 01 - Too-Small Learning Rate Scenario

Part 1 - Why this matters

A learning rate can be stable yet too small to make useful progress. The loss may decrease at every step while the parameter remains far from the minimum after the available training budget.

This scenario must be distinguished from a plateau caused by a vanishing gradient. Measuring the raw gradient and the learning-rate-scaled update shows whether the optimizer is receiving a signal but moving too cautiously.

Part 2 - Mathematical form

For update:

$$\Delta\theta_t = -\eta g_t.$$

Even when $\|g_t\|$ is meaningful, a very small η gives:

$$\|\Delta\theta_t\| = \eta\|g_t\| \approx 0.$$

Part 3 - Hints and Helper

Run a small and a reference learning rate from identical initial values. Compare loss improvement, gradient norm, update norm, and distance from the optimum after the same number of steps.

Part 4 - Python code skeleton

```
import torch

def E01(initial_value, small_rate, reference_rate, steps):
    small_history = ...
    reference_history = ...
    comparison = ...
    return small_history, reference_history, comparison

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

4.3.3 Exercise 02 - Too-Large Learning Rate Scenario

Part 1 - Why this matters

An excessive learning rate can move parameters past a minimum and repeatedly reverse their position. With stronger instability, both parameters and loss can grow until they become non-finite.

Large-rate failure is not identical to an exploding gradient: an ordinary gradient multiplied by an excessive learning rate can already create a destructive update. Both quantities should therefore be monitored separately.

Part 2 - Mathematical form

For $L(\theta) = \frac{1}{2}c\theta^2$:

$$\theta_{t+1} = (1 - \eta c)\theta_t.$$

If:

$$|1 - \eta c| > 1,$$

the distance from the minimum grows and the iteration diverges.

Part 3 - Hints and Helper

Use a quadratic with known curvature and choose a rate outside its stable interval. Record signs, magnitudes, loss ratios, and finite checks, then stop safely if a limit or non-finite value is reached.

Part 4 - Python code skeleton

```
import torch

def E02(parameter, learning_rate, steps, safety_limit):
    history = []
    status = ...
    for step in range(steps):
        ...
        if ...:
            status = ...
            break
    return history, status

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

4.3.4 Exercise 03 - Learning Rate Sweep Experiment

Part 1 - Why this matters

A learning-rate sweep evaluates several candidate scales under the same controlled conditions. It reveals ranges that are too slow, productive, oscillatory, or immediately unstable before committing to a long training run.

Fair comparison requires resetting the model and optimizer state for every candidate. Otherwise later rates inherit parameters from earlier runs and the experiment no longer isolates the learning rate.

Part 2 - Mathematical form

For candidates:

$$\mathcal{E} = \{\eta_1, \eta_2, \ldots, \eta_K\},$$

run:

$$\theta_{t+1}^{(k)} = \theta_t^{(k)} - \eta_k \nabla L(\theta_t^{(k)}).$$

Each run must start from the same θ_0.

Part 3 - Hints and Helper

Use logarithmically spaced candidate rates, clone the same initialization, and collect identical metrics for a fixed budget. Rank rates by a defined score while rejecting non-finite runs.

Part 4 - Python code skeleton

```
import torch

def E03(initial_parameter, learning_rates, steps):
    results = {}
    for rate in learning_rates:
        parameter = initial_parameter...
        history = ...
        results[rate] = ...
    return results

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

4.3.5 Exercise 04 - Loss Oscillation Detection

Part 1 - Why this matters

Oscillation occurs when optimization repeatedly crosses a valley or minimum instead of settling smoothly. The loss may alternate, and parameter updates or gradients often reverse direction across successive steps.

Some oscillation can coexist with overall progress, so detection should use a window and quantify frequency or amplitude. One isolated loss increase is not enough to classify the complete run.

Part 2 - Mathematical form

Let:

$$\Delta_t = L_t - L_{t-1}.$$

A simple oscillation indicator counts sign changes:

$$S = \sum_{t=2}^{T-1} \mathbf{1}[\Delta_t \Delta_{t-1} < 0] \,.$$

Amplitude can be measured from the loss range inside a recent window.

Part 3 - Hints and Helper

Calculate loss differences over a selected window and count direction reversals. Combine this with parameter or gradient sign changes and return evidence rather than only a Boolean label.

Part 4 - Python code skeleton

```
import torch

def E04(loss_history, parameter_history, window):
    recent_losses = ...
    differences = ...
    sign_changes = ...
    amplitude = ...
    oscillating = ...
    return (...)

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

4.3.6 Exercise 05 - Adaptive Learning Rate Logic

Part 1 - Why this matters

Simple adaptive logic changes the learning rate in response to observed progress. It can reduce the rate after repeated non-improvement or cautiously increase it when training remains stable.

Any rule must define its metric, patience, bounds, and update timing. Reacting to every noisy step can make the learning rate itself oscillate and obscure the behavior it is intended to improve.

Part 2 - Mathematical form

One plateau-based rule is:

$$\eta_{t+1} = \begin{cases} \gamma \eta_t, & \text{if no improvement for } p \text{ steps,} \\ \eta_t, & \text{otherwise,} \end{cases}$$

where $0 < \gamma < 1$, with $\eta_{\min} \leq \eta_t \leq \eta_{\max}$.

Part 3 - Hints and Helper

Track the best loss and a non-improvement counter. Change the rate only after the patience condition, clamp it to defined bounds, and record every change with its triggering evidence.

Part 4 - Python code skeleton

```
import torch

def E05(loss_history, current_rate, factor, patience, min_rate):
    best_loss = ...
    wait = ...
    if ...:
        new_rate = ...
    else:
        new_rate = ...
    return new_rate

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

4.3.7 Exercise 06 - Learning Rate Scheduling Basics

Part 1 - Why this matters

A schedule changes the learning rate according to training progress rather than immediate gradient values. Larger early steps can accelerate exploration, while smaller later steps can refine parameters near a solution.

Schedules make the effective update rule time-dependent. Their units must be clear: a schedule may advance per optimizer step or per epoch, and calling it at the wrong frequency changes the intended curve.

Part 2 - Mathematical form

A step-decay schedule can be written:

$$\eta_t = \eta_0 \gamma^{\lfloor t/s \rfloor},$$

where s is the decay interval and $0 < \gamma < 1$. An exponential schedule uses:

$$\eta_t = \eta_0 \gamma^t.$$

Part 3 - Hints and Helper

Implement at least one schedule manually and print the rate for every step before using it in updates. Compare the generated sequence with a PyTorch scheduler and verify when the scheduler advances.

Part 4 - Python code skeleton

```
import torch

def E06(initial_rate, decay_factor, interval, total_steps):
    rates = []
    for step in range(total_steps):
        rate = ...
        rates.append(rate)
    return rates

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

4.3.8 Exercise 07 - Warm-up Learning Rate Strategy

Part 1 - Why this matters

Warm-up begins training with a reduced learning rate and increases it gradually to a target value. It can protect unstable early steps when parameters, activations, or optimizer statistics have not yet reached useful scales.

Warm-up is a temporary phase rather than a complete long-run schedule. The transition at its boundary should be continuous and followed by a defined constant or decay strategy.

Part 2 - Mathematical form

For linear warm-up over T_w steps:

$$\eta_t = \eta_{\max} \frac{t+1}{T_w}, \qquad 0 \le t < T_w.$$

Afterward:

$$\eta_t = \eta_{\max}$$

or another schedule begins from that value.

Part 3 - Hints and Helper

Generate and inspect the complete rate sequence before training. Check the first value, target value, warm-up length, and boundary transition, then compare early update norms with and without warm-up.

Part 4 - Python code skeleton

```
import torch

def E07(max_rate, warmup_steps, total_steps):
    rates = []
    for step in range(total_steps):
        if step < warmup_steps:
            rate = ...
        else:
            rate = ...
        rates.append(rate)
    return rates

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

4.3.9 Exercise 08 - Learning Rate Debugging Challenge

Part 1 - Why this matters

Poor learning-rate behavior can resemble other failures: slow progress may come from tiny gradients, while instability may come from non-finite data or incorrect loss scaling. Diagnosing the rate requires comparing gradients with actual update magnitudes.

A useful report separates the base rate, scheduled rate, gradient norm, update norm, loss trend, and parameter scale. This identifies whether the selected rate is wrong or merely receiving misleading inputs.

Part 2 - Mathematical form

The effective relative update is:

$$R_t = \frac{\eta_t \|g_t\|}{\|\theta_t\| + \varepsilon}.$$

A problematic R_t may result from η_t, g_t, or an unusually small parameter norm, so all components must be inspected.

Part 3 - Hints and Helper

Construct a run with one intentional rate-related fault. Record rate history and gradient/update statistics, check scheduler call order and loss reduction, then name the earliest inconsistent quantity.

Part 4 - Python code skeleton

```
import torch

def E08(parameter, loss_history, rate_history, gradient_history):
    diagnostics = {
```

```
        "loss_trend": ...,
        "rate_changes": ...,
        "relative_updates": ...,
        "finite": ...
    }
    diagnosis = ...
    return diagnostics, diagnosis

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

4.3.10 Exercise 09 - Optimal Learning Rate Search

Part 1 - Why this matters

There is no universally optimal learning rate; the useful value depends on the model, initialization, data, optimizer, batch size, and training budget. A search must therefore define what optimal means for the current experiment.

Selecting only the lowest training loss can favor unstable or overfitting runs. A robust score can include validation performance, convergence speed, stability, and repeatability across random seeds.

Part 2 - Mathematical form

For candidate η_k, define a score such as:

$$S_k = L_{\text{val},k} + \lambda_1 P_{\text{instability},k} + \lambda_2 T_k,$$

where penalties and time terms are normalized consistently. The selected rate is:

$$\eta^* = \arg\min_{\eta_k} S_k.$$

Part 3 - Hints and Helper

Use a coarse logarithmic sweep followed by a finer search around promising rates. Reinitialize every run, evaluate more than one seed, reject divergent trials, and return the full ranked results with the chosen criterion.

Part 4 - Python code skeleton

```
import torch

def E09(train_function, learning_rates, seeds):
    results = []
    for rate in learning_rates:
        trials = []
```

```
        for seed in seeds:
            trial = train_function(...)
            trials.append(trial)
        score = ...
        results.append(...)
    best_rate = ...
    return best_rate, results

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

4.4 OC02 - Momentum Update Rule

Mathematics and Theory

4.4.1 Exercise 00 - Momentum Concept Implementation

Part 1 - Why this matters

Momentum extends gradient descent with a state variable that remembers recent update direction. Consistent gradients build velocity, while conflicting gradients partially cancel, creating smoother and often faster motion.

Implementing the rule manually shows that the optimizer state is separate from both parameters and gradients. The velocity must persist across steps and match each parameter's shape.

Part 2 - Mathematical form

Using the convention in this chapter:

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \mathbf{g}_t, \qquad \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \mathbf{v}_t,$$

where $0 \leq \beta < 1$ and $\mathbf{v}_0 = \mathbf{0}$.

Part 3 - Hints and Helper

Initialize velocity with `torch.zeros_like(parameter)` and update it before updating the parameter. Keep both updates outside Autograd's graph and return the new state explicitly.

Part 4 - Python code skeleton

```
import torch

def E00(parameter, gradient, velocity, learning_rate, momentum):
    new_velocity = ...
    new_parameter = ...
```

```
    return new_parameter, new_velocity

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

4.4.2 Exercise 01 - Velocity Term Tracking

Part 1 - Why this matters

The velocity contains historical information that cannot be reconstructed from the current gradient alone. Tracking it across steps reveals how earlier gradients continue to influence later parameter movement.

A velocity history also helps debug accidental reinitialization and excessive momentum. Recording gradient, velocity, and update norms makes the optimizer's memory observable.

Part 2 - Mathematical form

Expanding the recurrence gives:

$$\mathbf{v}_t = \mathbf{g}_t + \beta \mathbf{g}_{t-1} + \beta^2 \mathbf{g}_{t-2} + \cdots .$$

Older gradients receive exponentially decreasing weights.

Part 3 - Hints and Helper

Use a predefined gradient sequence and record velocity at each step without a loss function. Compare the computed state with a manual recurrence and verify that it is not reset inside the loop.

Part 4 - Python code skeleton

```
import torch

def E01(gradients, momentum):
    velocity = torch...
    history = []
    for gradient in gradients:
        velocity = ...
        history.append(velocity...)
    return history

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

4.4.3 Exercise 02 - Momentum vs Vanilla GD

Part 1 - Why this matters

Vanilla gradient descent uses only the current gradient, whereas momentum combines it with accumulated direction. Comparing both methods on the same objective reveals when history accelerates movement or changes oscillation.

A valid comparison must share initialization, learning rate, loss, and step budget. Separate parameter and optimizer states are required so one method does not influence the other.

Part 2 - Mathematical form

Vanilla gradient descent:

$$\theta_{t+1}^{GD} = \theta_t^{GD} - \eta g_t^{GD}.$$

Momentum:

$$v_t = \beta v_{t-1} + g_t^M, \qquad \theta_{t+1}^M = \theta_t^M - \eta v_t.$$

Part 3 - Hints and Helper

Run both optimizers from cloned initial parameters and record equal metrics. Compare loss, distance to the optimum, update size, and oscillation rather than only the final loss.

Part 4 - Python code skeleton

```
import torch

def E02(initial_parameter, learning_rate, momentum, steps):
    gd_parameter = ...
    momentum_parameter = ...
    velocity = ...
    gd_history, momentum_history = [], []
    for step in range(steps):
        ...
    return gd_history, momentum_history

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

4.4.4 Exercise 03 - Momentum Acceleration Effect

Part 1 - Why this matters

When gradients point consistently in one direction, momentum accumulates them and increases the effective step. This can move parameters rapidly through long, gently sloped regions where vanilla gradient descent progresses slowly.

Acceleration is not free: the stored velocity can carry an update past a minimum. Observing both benefits and overshoot builds a balanced understanding of momentum behavior.

Part 2 - Mathematical form

For an approximately constant gradient g:

$$v_t = g \sum_{k=0}^{t} \beta^k = g \frac{1 - \beta^{t+1}}{1 - \beta}.$$

As t grows:

$$v_t \to \frac{g}{1 - \beta}.$$

Part 3 - Hints and Helper

Use a region with nearly consistent gradient direction and compare velocity growth with vanilla update magnitude. Track when acceleration begins, when the gradient changes sign, and how long velocity persists.

Part 4 - Python code skeleton

```
import torch

def E03(parameter, learning_rate, momentum, steps):
    velocity = torch...
    history = []
    for step in range(steps):
        gradient = ...
        velocity = ...
        parameter = ...
        history.append(...)
    return history

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

4.4.5 Exercise 04 - Damped Oscillation Observation

Part 1 - Why this matters

On narrow valleys, vanilla updates can alternate sharply across the steep direction. Momentum can smooth changing gradients, but an aggressive coefficient or learning rate can also sustain oscillation.

Damping should be measured through amplitude over time. A trajectory is damped when successive excursions decrease, not merely because the parameter crosses the minimum several times.

Part 2 - Mathematical form

For a quadratic objective and momentum:

$$v_t = \beta v_{t-1} + c\theta_t, \qquad \theta_{t+1} = \theta_t - \eta v_t.$$

The pair (η, β) determines whether oscillations decay, persist, or grow.

Part 3 - Hints and Helper

Record parameter sign changes and peak absolute distances from the minimum. Compare early and late amplitudes over equal windows and classify the motion using defined tolerances.

Part 4 - Python code skeleton

```
import torch

def E04(parameter, learning_rate, momentum, steps):
    velocity = torch...
    trajectory = []
    for step in range(steps):
        ...
    peaks = ...
    damping_ratio = ...
    return trajectory, peaks, damping_ratio

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

4.4.6 Exercise 05 - Momentum Coefficient Tuning

Part 1 - Why this matters

The momentum coefficient controls how strongly past gradients influence the current velocity. Low values behave closer to vanilla gradient descent, while values near one preserve history for longer.

The best coefficient depends on learning rate, curvature, noise, and training budget. Tuning momentum independently while allowing other conditions to change produces misleading conclusions.

Part 2 - Mathematical form

The contribution of a gradient k steps old is:

$$\beta^k \mathbf{g}_{t-k}.$$

A rough memory timescale increases as β approaches one, while $\beta = 0$ reduces the rule to vanilla gradient descent.

Part 3 - Hints and Helper

Sweep several coefficients from the same initialization with a fixed learning rate. Record convergence speed, maximum overshoot, oscillation, and final loss, and reject non-finite runs.

Part 4 - Python code skeleton

```
import torch

def E05(initial_parameter, coefficients, learning_rate, steps):
    results = {}
    for coefficient in coefficients:
        velocity = ...
        parameter = ...
        history = ...
        results[coefficient] = ...
    return results

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

4.4.7 Exercise 06 - Momentum on Noisy Gradients

Part 1 - Why this matters

Mini-batch gradients contain sampling noise that changes from step to step. Momentum averages recent directions through its state, often reducing high-frequency variation while preserving a repeated underlying trend.

Excessive momentum can also retain outdated direction after the true gradient changes. The experiment should compare noise reduction with responsiveness rather than assuming more smoothing is always better.

Part 2 - Mathematical form

Let:

$$g_t = \mu_t + \varepsilon_t, \qquad v_t = \beta v_{t-1} + g_t,$$

where ε_t is zero-mean noise. The recurrence combines several noisy observations, but it also delays response when μ_t changes.

Part 3 - Hints and Helper

Generate a reproducible noisy gradient sequence and compare raw gradients with normalized or scaled velocity direction. Measure variance and response after an intentional change in the underlying trend.

Part 4 - Python code skeleton

```
import torch

def E06(true_gradients, noise_scale, momentum, seed):
    generator = torch...
    velocity = torch...
    records = []
    for true_gradient in true_gradients:
        noisy_gradient = ...
        velocity = ...
        records.append(...)
    return records

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

4.4.8 Exercise 07 - Custom Momentum Optimizer

Part 1 - Why this matters

A custom optimizer packages parameter updates and persistent velocity states into a reusable object. It must create one state tensor per parameter, skip missing gradients, update without graph tracking, and support clearing gradients.

Building the class exposes responsibilities normally handled by `torch.optim`. Correct state ownership is essential when parameters have different shapes, devices, or dtypes.

Part 2 - Mathematical form

For every parameter $\theta^{(i)}$:

$$v_t^{(i)} = \beta v_{t-1}^{(i)} + g_t^{(i)}, \qquad \theta_{t+1}^{(i)} = \theta_t^{(i)} - \eta v_t^{(i)}.$$

Each parameter requires an independent velocity state.

Part 3 - Hints and Helper

Store a parameter list and shape-matched velocities. Implement `step()` under `torch.no_grad()`, ignore parameters with `grad is None`, and provide an explicit gradient-reset method.

Part 4 - Python code skeleton

```
import torch

class CustomMomentum:
    def __init__(self, parameters, learning_rate, momentum):
        self.parameters = ...
        self.velocities = ...
```

```
    @torch.no_grad()
    def step(self):
        for parameter, velocity in zip(...):
            if parameter.grad is None:
                continue
            ...

    def zero_grad(self):
        ...

def E07(...):
    optimizer = CustomMomentum(...)
    return optimizer

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

4.4.9 Exercise 08 - Momentum Failure Debugging

Part 1 - Why this matters

Momentum can fail because velocity is reset each step, signs are inconsistent, gradients accumulate unintentionally, states use wrong shapes or devices, or hyperparameters create unstable dynamics.

Debugging must inspect both gradient and optimizer state. A correct gradient does not guarantee a correct update if velocity is stale, missing, or applied twice.

Part 2 - Mathematical form

For the chosen convention, the required invariants are:

$$\text{shape}(v_t) = \text{shape}(\theta_t) = \text{shape}(g_t),$$

and:

$$\Delta\theta_t = -\eta v_t.$$

Violating either relationship identifies an implementation error.

Part 3 - Hints and Helper

Log parameter, gradient, old velocity, new velocity, and actual displacement for a few deterministic steps. Verify update order, sign, state persistence, finite values, and gradient clearing.

Part 4 - Python code skeleton

```
import torch

def E08(parameter, gradients, learning_rate, momentum):
```

```
    velocity = torch...
    diagnostics = []
    for gradient in gradients:
        old_parameter = parameter...
        old_velocity = velocity...
        ...
        diagnostics.append(...)
    diagnosis = ...
    return diagnostics, diagnosis

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

4.4.10 Exercise 09 - Momentum Convergence Challenge

Part 1 - Why this matters

The final challenge combines a scalar or vector objective, Autograd, persistent velocity, convergence tests, loss tracking, and comparison with vanilla gradient descent. It measures whether momentum improves the complete optimization process.

A successful run should report its trajectory and stopping reason, not only the final parameter. The comparison must use identical starting conditions and an explicit definition of convergence.

Part 2 - Mathematical form

At every step:

$$g_t = \nabla_\theta L(\theta_t), \quad v_t = \beta v_{t-1} + g_t, \quad \theta_{t+1} = \theta_t - \eta v_t.$$

Possible stopping criteria include:

$$\|g_t\| < \varepsilon_g \quad \text{or} \quad |L_t - L_{t-1}| < \varepsilon_L$$

for a defined patience window.

Part 3 - Hints and Helper

Implement momentum without `torch.optim`, record all relevant states, and stop on convergence, divergence, or the step limit. Run a vanilla baseline and compare steps, best loss, overshoot, and stability.

Part 4 - Python code skeleton

```
import torch

def E09(parameter, loss_function, learning_rate, momentum,
        max_steps, tolerance):
    velocity = torch.zeros_like(parameter)
```

```
    history = []
    status = ...
    for step in range(max_steps):
        loss = loss_function(parameter)
        ...
        history.append(...)
        if ...:
            break
    return parameter, history, status

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

4.5 OC03 - RMSProp Logic & Behavior

Mathematics and Theory

4.5.1 Exercise 00 - RMSProp Accumulator Initialization

Part 1 - Why this matters

RMSProp stores a separate running estimate of squared gradients for every parameter element. This state begins at zero and gradually learns the recent scale of the gradients.

Correct initialization preserves parameter shape, dtype, and device. A shared scalar or incorrectly shaped accumulator would destroy RMSProp's element-wise adaptation.

Part 2 - Mathematical form

For parameter $\boldsymbol{\theta} \in \mathbb{R}^n$, initialize:

$$\mathbf{s}_0 = \mathbf{0} \in \mathbb{R}^n.$$

The state invariant is:

$$\text{shape}(\mathbf{s}_t) = \text{shape}(\boldsymbol{\theta}_t).$$

Part 3 - Hints and Helper

Use `torch.zeros_like(parameter)` and inspect shape, dtype, device, and gradient requirements. The accumulator is optimizer state and should not require gradients.

Part 4 - Python code skeleton

```
import torch

def E00(parameter):
    accumulator = torch...
    checks = ...
```

```
    return accumulator, checks

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

4.5.2 Exercise 01 - Running Average of Squared Gradients

Part 1 - Why this matters

Squaring gradients removes their sign and measures magnitude. RMSProp combines the new squared gradient with an exponentially decayed history, producing a local scale estimate for each parameter.

The running average smooths short-term variation without storing every past gradient. Its decay rate controls how quickly old information is forgotten.

Part 2 - Mathematical form

For gradient $\mathbf{g}_t$ and decay rate ρ:

$$\mathbf{s}_t = \rho \mathbf{s}_{t-1} + (1-\rho)\mathbf{g}_t \odot \mathbf{g}_t, \qquad 0 \leq \rho < 1.$$

Part 3 - Hints and Helper

Process a known gradient sequence and record the accumulator after every step. Verify element-wise squaring and ensure the state persists instead of being reinitialized inside the loop.

Part 4 - Python code skeleton

```
import torch

def E01(gradients, decay_rate):
    accumulator = torch...
    history = []
    for gradient in gradients:
        accumulator = ...
        history.append(accumulator...)
    return history

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

4.5.3 Exercise 02 - RMSProp Update Step Implementation

Part 1 - Why this matters

RMSProp divides each gradient by the square root of its recent squared-gradient average. Coordinates with persistently large gradients receive smaller effective steps, while smaller-scale coordinates are reduced less.

A positive epsilon protects division when the accumulator is zero or extremely small. Its placement must remain consistent with the selected RMSProp definition.

Part 2 - Mathematical form

Using the convention in this chapter:

$$\mathbf{s}_t = \rho \mathbf{s}_{t-1} + (1-\rho)\mathbf{g}_t^2,$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \frac{\mathbf{g}_t}{\sqrt{\mathbf{s}_t} + \varepsilon}.$$

Part 3 - Hints and Helper

Update the accumulator before computing the normalized gradient. Perform state and parameter updates without graph tracking, and record the effective element-wise learning rates.

Part 4 - Python code skeleton

```
import torch

def E02(parameter, gradient, accumulator,
        learning_rate, decay_rate, epsilon):
    new_accumulator = ...
    normalized_gradient = ...
    new_parameter = ...
    return new_parameter, new_accumulator, normalized_gradient

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

4.5.4 Exercise 03 - Effect of Decay Rate

Part 1 - Why this matters

The decay rate determines the memory length of RMSProp's squared-gradient estimate. A high value changes slowly and emphasizes longer history, while a lower value reacts quickly to recent scale changes.

Neither behavior is always superior. Slow adaptation may lag after a regime change, while rapid adaptation may follow noisy spikes too closely.

Part 2 - Mathematical form

A gradient from k steps earlier contributes a weight proportional to:

$$(1-\rho)\rho^k.$$

Thus larger ρ preserves old squared gradients for more steps.

Part 3 - Hints and Helper

Run identical gradient sequences with several decay rates. Include a sudden magnitude change and compare response time, smoothness, and normalized update sizes.

Part 4 - Python code skeleton

```
import torch

def E03(gradients, decay_rates, epsilon):
    results = {}
    for decay_rate in decay_rates:
        accumulator = ...
        history = ...
        results[decay_rate] = history
    return results

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

4.5.5 Exercise 04 - RMSProp vs Momentum Comparison

Part 1 - Why this matters

Momentum adapts direction using a running first-order state, while RMSProp adapts coordinate scale using squared gradients. Their states answer different optimization problems and should not be treated as interchangeable.

A fair comparison uses identical initialization, data, learning budget, and evaluation metrics. State trajectories help explain differences that a final loss alone cannot reveal.

Part 2 - Mathematical form

Momentum:

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \mathbf{g}_t.$$

RMSProp:

$$\mathbf{s}_t = \rho \mathbf{s}_{t-1} + (1-\rho)\mathbf{g}_t^2, \quad \Delta\theta_t = -\eta \frac{\mathbf{g}_t}{\sqrt{\mathbf{s}_t} + \varepsilon}.$$

Part 3 - Hints and Helper

Run both methods from cloned parameters on an objective with different curvature across dimensions. Compare loss, path, update norms, oscillation, and optimizer states.

Part 4 - Python code skeleton

```
import torch

def E04(initial_parameter, loss_function, steps, settings):
    momentum_history = ...
    rmsprop_history = ...
    comparison = ...
    return momentum_history, rmsprop_history, comparison

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

4.5.6 Exercise 05 - Stability under High Gradient Variance

Part 1 - Why this matters

When gradient magnitudes vary greatly between steps or coordinates, one global learning rate can create uneven updates. RMSProp's scale estimate can reduce the effect of repeatedly large-gradient coordinates.

The accumulator responds to squared magnitude, so extreme outliers can also suppress later updates until their influence decays. Stability must be evaluated together with recovery speed.

Part 2 - Mathematical form

For noisy gradient:

$$\mathbf{g}_t = \boldsymbol{\mu}_t + \boldsymbol{\varepsilon}_t,$$

RMSProp uses:

$$\mathbf{s}_t = \rho \mathbf{s}_{t-1} + (1-\rho)\mathbf{g}_t^2.$$

The normalized magnitude is $\mathbf{g}_t/(\sqrt{\mathbf{s}_t} + \varepsilon)$.

Part 3 - Hints and Helper

Generate reproducible gradients with different variances and occasional spikes. Compare raw and normalized update distributions, finite values, and recovery after a spike.

Part 4 - Python code skeleton

```
import torch

def E05(gradients, decay_rate, epsilon):
    accumulator = torch...
    records = []
```

```
    for gradient in gradients:
        accumulator = ...
        normalized = ...
        records.append(...)
    return records

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

4.5.7 Exercise 06 - RMSProp without Autograd

Part 1 - Why this matters

Manual gradients allow RMSProp to be studied independently of computational graphs. This separates analytical differentiation from the optimizer state and parameter update rule.

The exercise confirms that RMSProp does not require Autograd conceptually; Autograd only provides gradients for complex objectives. State persistence and update ordering remain unchanged.

Part 2 - Mathematical form

For $L(\theta) = (\theta - a)^2$:

$$g_t = 2(\theta_t - a),$$

then:

$$s_t = \rho s_{t-1} + (1-\rho) g_t^2, \qquad \theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{s_t} + \varepsilon}.$$

Part 3 - Hints and Helper

Use ordinary tensors without `requires_grad`. Implement the analytical gradient and RMSProp recurrence, recording loss, gradient, accumulator, and update at every step.

Part 4 - Python code skeleton

```
import torch

def E06(parameter, target, learning_rate,
        decay_rate, epsilon, steps):
    accumulator = torch.zeros_like(parameter)
    history = []
    for step in range(steps):
        gradient = ...
        accumulator = ...
        parameter = ...
        history.append(...)
    return parameter, history
```

```
def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

4.5.8 Exercise 07 - Custom RMSProp Optimizer

Part 1 - Why this matters

A custom optimizer stores one squared-gradient accumulator for each parameter and applies the RMSProp rule whenever gradients are available. It must also manage gradient clearing and preserve device and dtype compatibility.

Packaging the algorithm exposes the distinction between model parameters, temporary gradients, and persistent optimizer state. Missing gradients must be skipped without shifting state-to-parameter alignment.

Part 2 - Mathematical form

For each parameter $\theta^{(i)}$:

$$s_t^{(i)} = \rho s_{t-1}^{(i)} + (1-\rho)(g_t^{(i)})^2,$$

$$\theta_{t+1}^{(i)} = \theta_t^{(i)} - \eta \frac{g_t^{(i)}}{\sqrt{s_t^{(i)}} + \varepsilon}.$$

Part 3 - Hints and Helper

Store parameters and `zeros_like` accumulators in matching order. Implement `step()` under `torch.no_grad()`, skip `None` gradients, and add `zero_grad()`.

Part 4 - Python code skeleton

```
import torch

class CustomRMSProp:
    def __init__(self, parameters, learning_rate,
                 decay_rate, epsilon):
        self.parameters = ...
        self.accumulators = ...

    @torch.no_grad()
    def step(self):
        for parameter, accumulator in zip(...):
            if parameter.grad is None:
                continue
            ...

    def zero_grad(self):
```

```
        ...

def E07(...):
    return CustomRMSProp(...)

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

4.5.9 Exercise 08 - RMSProp Debugging Challenge

Part 1 - Why this matters

RMSProp failures can result from resetting the accumulator, using the gradient instead of its square, reversing decay weights, omitting epsilon, or applying the update before refreshing state.

Debugging requires inspecting optimizer state alongside gradients and parameters. A correct loss gradient can still produce an incorrect training path when the accumulator recurrence is wrong.

Part 2 - Mathematical form

The required checks include:

$$s_t \geq 0, \qquad \text{shape}(s_t) = \text{shape}(g_t),$$

and:

$$s_t = \rho s_{t-1} + (1-\rho) g_t^2.$$

The actual displacement must equal the chosen normalized update rule.

Part 3 - Hints and Helper

Use a deterministic short gradient sequence and log old state, squared gradient, new state, denominator, and displacement. Compare each line with a manual calculation.

Part 4 - Python code skeleton

```
import torch

def E08(parameter, gradients, learning_rate,
        decay_rate, epsilon):
    accumulator = torch.zeros_like(parameter)
    diagnostics = []
    for gradient in gradients:
        old_accumulator = accumulator...
        ...
        diagnostics.append(...)
    diagnosis = ...
```

```
    return diagnostics, diagnosis

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

4.5.10 Exercise 09 - RMSProp Convergence Evaluation

Part 1 - Why this matters

Convergence evaluation measures whether RMSProp reaches a useful solution efficiently and stably. It should include trajectory evidence, stopping reason, gradient behavior, and comparison with a baseline.

One final loss cannot reveal oscillation, stalled normalized updates, or sensitivity to decay and epsilon. Controlled repeated trials provide stronger conclusions.

Part 2 - Mathematical form

At each step:

$$g_t = \nabla L(\theta_t), \quad s_t = \rho s_{t-1} + (1-\rho) g_t^2,$$
$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{s_t} + \varepsilon}.$$

Stop using defined gradient, loss-change, divergence, or step-limit criteria.

Part 3 - Hints and Helper

Implement RMSProp without `torch.optim`, record all state summaries, and compare it with vanilla gradient descent from the same initialization. Report steps, best loss, final loss, stability, and stopping reason.

Part 4 - Python code skeleton

```
import torch

def E09(parameter, loss_function, learning_rate,
        decay_rate, epsilon, max_steps, tolerance):
    accumulator = torch.zeros_like(parameter)
    history = []
    status = ...
    for step in range(max_steps):
        ...
        if ...:
            break
    return parameter, history, status

def main():
    print(E09(...))
```

```
if __name__ == "__main__":
    main()
```

4.6 OC04 - Adam Optimizer Math + Code

Mathematics and Theory

4.6.1 Exercise 00 - Adam First Moment Initialization

Part 1 - Why this matters

Adam's first moment is an exponential moving average of gradients. It represents a smoothed direction and must be initialized separately for every parameter tensor.

Initializing the state with matching zeros preserves shape, dtype, and device while introducing no artificial direction before the first gradient arrives.

Part 2 - Mathematical form

For parameter $\boldsymbol{\theta}$, initialize:

$$\mathbf{m}_0 = \mathbf{0}.$$

The recurrence used later is:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1)\mathbf{g}_t.$$

Part 3 - Hints and Helper

Use `torch.zeros_like(parameter)` and verify that the state does not require gradients. Record the invariant that its shape and device match the parameter.

Part 4 - Python code skeleton

```
import torch

def E00(parameter):
    first_moment = torch...
    checks = ...
    return first_moment, checks

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

4.6.2 Exercise 01 - Adam Second Moment Initialization

Part 1 - Why this matters

Adam's second moment tracks an exponential average of squared gradients. It provides an element-wise estimate of gradient scale used to normalize the first moment.

This state is nonnegative and independent from the first moment. Keeping two correctly aligned states is essential because they represent direction and scale separately.

Part 2 - Mathematical form

Initialize:

$$\mathbf{v}_0 = \mathbf{0}.$$

Then:

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2)\mathbf{g}_t \odot \mathbf{g}_t.$$

Part 3 - Hints and Helper

Create a zero state matching the parameter and verify shape, dtype, device, and nonnegativity. Do not reuse the same tensor object for first and second moments.

Part 4 - Python code skeleton

```
import torch

def E01(parameter):
    second_moment = torch...
    checks = ...
    return second_moment, checks

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

4.6.3 Exercise 02 - Bias Correction Implementation

Part 1 - Why this matters

Because Adam's moment estimates start at zero, their early values are biased toward zero. Bias correction divides by the remaining initialization weight, producing estimates with the intended scale.

The correction depends on a one-based update counter. Starting at zero causes division by zero, while incrementing the counter at the wrong point shifts every correction factor.

Part 2 - Mathematical form

At update $t \geq 1$:

$$\widehat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1-\beta_1^t}, \qquad \widehat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1-\beta_2^t}.$$

Part 3 - Hints and Helper

Update raw moments first, increment or define the step consistently, and then apply correction. Test the first step with a constant gradient, where corrected moments have an easy expected value.

Part 4 - Python code skeleton

```
import torch

def E02(first_moment, second_moment,
        beta1, beta2, step):
    corrected_first = ...
    corrected_second = ...
    return corrected_first, corrected_second

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

4.6.4 Exercise 03 - Adam Update Rule from Scratch

Part 1 - Why this matters

Adam combines a smoothed gradient direction with an adaptive scale estimate. The complete update requires both moment recurrences, both bias corrections, epsilon, and a persistent step counter.

Implementing the rule from scratch reveals the exact ordering hidden inside a library optimizer. A small error in state or counter handling changes all later parameter updates.

Part 2 - Mathematical form

For $t \geq 1$:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2,$$

$$\theta_{t+1} = \theta_t - \eta \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \varepsilon}.$$

Part 3 - Hints and Helper

Increment the step once per optimizer update, refresh raw moments, correct them, and update parameters outside the graph. Return all new states so the caller can preserve them.

Part 4 - Python code skeleton

```
import torch

def E03(parameter, gradient, first_moment, second_moment,
        step, learning_rate, beta1, beta2, epsilon):
    step = ...
    new_first = ...
    new_second = ...
    corrected_first = ...
    corrected_second = ...
    new_parameter = ...
    return (...)

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

4.6.5 Exercise 04 - Adam vs RMSProp Comparison

Part 1 - Why this matters

RMSProp normalizes the current gradient using a squared-gradient average, whereas Adam also smooths gradient direction and corrects both moments during early steps.

Their similar adaptive denominators can hide important behavioral differences. A controlled comparison should inspect states and trajectories, not only final loss.

Part 2 - Mathematical form

RMSProp uses:

$$s_t = \rho s_{t-1} + (1-\rho) g_t^2.$$

Adam uses:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2,$$

with bias-corrected m_t and v_t.

Part 3 - Hints and Helper

Clone identical initial parameters and run equal training budgets. Compare loss, updates, stability, and state response to direction changes while documenting each method's hyperparameters.

Part 4 - Python code skeleton

```
import torch

def E04(initial_parameter, loss_function, settings, steps):
    rmsprop_history = ...
    adam_history = ...
    comparison = ...
```

```
    return rmsprop_history, adam_history, comparison

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

4.6.6 Exercise 05 - Adam Hyperparameter Sensitivity

Part 1 - Why this matters

Adam behavior depends on learning rate, first-moment decay, second-moment decay, and epsilon. Each controls a different part of the update, so tuning several at once makes the cause of a change difficult to identify.

Sensitivity experiments reveal whether performance is robust or depends on a narrow configuration. Every candidate must start with fresh parameters, moments, and step counter.

Part 2 - Mathematical form

Adam is controlled by:

$$\eta, \quad \beta_1, \quad \beta_2, \quad \varepsilon.$$

The effective update is:

$$\Delta\theta_t = -\eta \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \varepsilon}.$$

Part 3 - Hints and Helper

Vary one hyperparameter at a time around a baseline and reinitialize all state. Record convergence speed, loss, update norms, and finite checks before attempting a combined search.

Part 4 - Python code skeleton

```
import torch

def E05(initial_parameter, configurations, loss_function, steps):
    results = {}
    for configuration in configurations:
        parameter = ...
        first_moment = ...
        second_moment = ...
        history = ...
        results[...] = ...
    return results

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

4.6.7 Exercise 06 - Adam on Sparse Gradients

Part 1 - Why this matters

Sparse gradients update only a small subset of parameter coordinates at each step. Adam stores moment states per coordinate, so active and inactive entries can carry different histories.

Zero gradients still interact with exponential decay in a dense implementation. The reader must distinguish truly absent sparse entries from explicit zeros and understand the optimizer support of the chosen representation.

Part 2 - Mathematical form

For coordinate i:

$$m_{t,i} = \beta_1 m_{t-1,i} + (1 - \beta_1) g_{t,i},$$
$$v_{t,i} = \beta_2 v_{t-1,i} + (1 - \beta_2) g_{t,i}^2.$$

When $g_{t,i} = 0$, stored moments decay but may remain nonzero.

Part 3 - Hints and Helper

Use a small gradient sequence with changing active indices. Track every coordinate's raw and corrected moments and compare active updates with the residual influence on inactive entries.

Part 4 - Python code skeleton

```
import torch

def E06(parameter, sparse_gradients, settings):
    first_moment = torch.zeros_like(parameter)
    second_moment = torch.zeros_like(parameter)
    history = []
    for step, gradient in enumerate(sparse_gradients, start=1):
        ...
        history.append(...)
    return history

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

4.6.8 Exercise 07 - Custom Adam Optimizer Class

Part 1 - Why this matters

A custom Adam class must own parameter references, two moment tensors per parameter, a consistent update counter, hyperparameters, and gradient-reset behavior.

Correct state alignment is critical across parameters with different shapes and missing gradients. The class should make update ordering explicit and avoid recording optimizer operations in Autograd.

Part 2 - Mathematical form

For each parameter $\theta^{(i)}$, store:

$$m^{(i)}, \quad v^{(i)}.$$

At global step t, update both states, compute bias-corrected estimates, and apply:

$$\theta^{(i)} \leftarrow \theta^{(i)} - \eta \frac{\widehat{m}^{(i)}}{\sqrt{\widehat{v}^{(i)}} + \varepsilon}.$$

Part 3 - Hints and Helper

Initialize states with `zeros_like`, implement `step()` under `torch.no_grad()`, handle `None` gradients, and test against `torch.optim.Adam` on a deterministic sequence.

Part 4 - Python code skeleton

```
import torch

class CustomAdam:
    def __init__(self, parameters, learning_rate,
                 beta1, beta2, epsilon):
        self.parameters = ...
        self.first_moments = ...
        self.second_moments = ...
        self.step_count = ...

    @torch.no_grad()
    def step(self):
        self.step_count += ...
        for parameter, first, second in zip(...):
            if parameter.grad is None:
                continue
            ...

    def zero_grad(self):
        ...

def E07(...):
    return CustomAdam(...)

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

4.6.9 Exercise 08 - Adam Failure Case Debugging

Part 1 - Why this matters

Adam can fail through an off-by-one step counter, missing bias correction, moment reinitialization, unsquared second moment, misplaced epsilon, stale gradients, or mismatched state tensors.

Debugging requires a deterministic short sequence and full state visibility. Checking only the parameter output cannot identify which internal recurrence first became incorrect.

Part 2 - Mathematical form

Required invariants include:

$$v_t \geq 0, \quad t \geq 1,$$

$$\widehat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \widehat{v}_t = \frac{v_t}{1-\beta_2^t}.$$

The actual displacement must match the normalized corrected update.

Part 3 - Hints and Helper

Log old states, gradient, new raw states, correction denominators, corrected states, denominator, and displacement. Compare the first incorrect quantity with a manual reference.

Part 4 - Python code skeleton

```
import torch

def E08(parameter, gradients, settings):
    first = torch.zeros_like(parameter)
    second = torch.zeros_like(parameter)
    diagnostics = []
    for step, gradient in enumerate(gradients, start=1):
        ...
        diagnostics.append(...)
    diagnosis = ...
    return diagnostics, diagnosis

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

4.6.10 Exercise 09 - Adam End-to-End Training Challenge

Part 1 - Why this matters

The final Adam challenge combines a model, scalar loss, Autograd gradients, two moment states, bias correction, parameter updates, convergence monitoring, and stability checks.

A complete result includes training history and diagnostic evidence. The implementation should be compared with PyTorch Adam under identical initialization to validate both mathematics and state handling.

Part 2 - Mathematical form

For each step t:

$$g_t = \nabla_\theta L, \quad m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t,$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \quad \theta_{t+1} = \theta_t - \eta \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \varepsilon}.$$

Part 3 - Hints and Helper

Implement training without `torch.optim`, record loss and state norms, and stop on convergence, divergence, or step limit. Compare parameters and histories with a library optimizer using the same settings.

Part 4 - Python code skeleton

```
import torch

def E09(model, X, target, settings, max_steps):
    parameters = list(model.parameters())
    first_moments = ...
    second_moments = ...
    history = []
    status = ...
    for step in range(1, max_steps + 1):
        ...
        history.append(...)
        if ...:
            break
    return model, history, status

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

4.7 OC05 - Convergence & Stability

Mathematics and Theory

4.7.1 Exercise 00 - Convergence Criteria Detection

Part 1 - Why this matters

Optimization needs explicit evidence for deciding when useful progress has become sufficiently small. A fixed epoch count alone cannot distinguish an unfinished run from one that

has already stabilized.

Combining loss change, gradient norm, and parameter displacement produces a stronger decision than any single signal. Criteria should hold for a patience window to resist noise.

Part 2 - Mathematical form

Possible conditions are:

$$|L_t - L_{t-1}| < \varepsilon_L, \quad \|g_t\| < \varepsilon_g, \quad \|\theta_t - \theta_{t-1}\| < \varepsilon_\theta.$$

Convergence may be declared when selected conditions persist for p steps.

Part 3 - Hints and Helper

Implement configurable tolerances and patience. Reject non-finite values before checking convergence and return the evidence and exact stopping reason.

Part 4 - Python code skeleton

```
import torch

def E00(losses, gradient_norms, update_norms,
        tolerances, patience):
    checks = ...
    converged = ...
    reason = ...
    return converged, reason, checks

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

4.7.2 Exercise 01 - Loss Plateau Detection

Part 1 - Why this matters

A plateau is a period in which the monitored loss shows little meaningful improvement. It may indicate convergence, an inadequate learning rate, vanishing gradients, or a region that the current optimizer cannot escape.

Plateau detection should compare improvement over a window rather than exact equality between adjacent floating-point values. Validation loss is often more useful than training loss for stopping decisions.

Part 2 - Mathematical form

For window length w, define:

$$I_t = \min(L_{t-w:t}) - L_t.$$

A plateau can be reported when:

$$I_t < \delta$$

for a specified patience interval, using a consistent minimum-improvement rule.

Part 3 - Hints and Helper

Track the best loss, minimum required improvement, and waiting counter. Handle noisy temporary increases without resetting the best value incorrectly.

Part 4 - Python code skeleton

```
import torch

def E01(loss_history, min_improvement, patience):
    best = ...
    wait = ...
    plateau_start = ...
    for step, loss in enumerate(loss_history):
        ...
    return plateau_start, wait

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

4.7.3 Exercise 02 - Divergence Pattern Identification

Part 1 - Why this matters

Divergence may appear as sustained loss growth, growing parameter or gradient norms, repeated oscillations with increasing amplitude, or non-finite values. Detecting it early prevents corrupted updates and wasted computation.

One loss increase is not sufficient because stochastic training is noisy. Pattern identification combines trend, ratios, windows, and safety limits.

Part 2 - Mathematical form

Possible evidence includes:

$$L_t > \rho L_{t-k}, \qquad \|g_t\| > \tau_g,$$

or:

$$L_t \notin \mathbb{R}_{finite}.$$

A sustained-growth rule requires the condition across several observations.

Part 3 - Hints and Helper

Inspect loss, gradient norm, update norm, and finiteness. Classify non-finite failure immediately, but use a patience window for ordinary growth or oscillation.

Part 4 - Python code skeleton

```
import torch

def E02(losses, gradient_norms, update_norms,
        growth_factor, patience):
    finite = ...
    growth_pattern = ...
    oscillation_growth = ...
    status = ...
    return status

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

4.7.4 Exercise 03 - Early Stopping Logic

Part 1 - Why this matters

Early stopping ends training when validation performance no longer improves, limiting unnecessary computation and reducing the chance of continued overfitting.

A correct implementation preserves the best model state rather than the final state. Minimum improvement, patience, evaluation frequency, and metric direction must all be explicit.

Part 2 - Mathematical form

For validation metric M_t to be minimized, improvement occurs when:

$$M_t < M_{best} - \delta.$$

Otherwise a counter increases, and training stops when:

$$\text{wait} \geq p.$$

Part 3 - Hints and Helper

Track best metric, best step, waiting count, and a cloned model state. Restore the best state after stopping and test the logic on a predefined metric sequence.

Part 4 - Python code skeleton

```
import torch

def E03(model, validation_metric, state):
    improved = ...
    if improved:
        state["best"] = ...
        state["weights"] = ...
        state["wait"] = ...
```

```
    else:
        state["wait"] = ...
    stop = ...
    return stop, state

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

4.7.5 Exercise 04 - Gradient Explosion Mitigation

Part 1 - Why this matters

Exploding gradients create excessive update magnitudes and can make parameters or losses non-finite. Mitigation must occur after diagnosis and before the optimizer applies a harmful step.

Gradient clipping limits immediate damage, while initialization, normalization, architecture, loss scaling, and learning-rate changes address different root causes. Clipping should therefore be monitored rather than applied invisibly.

Part 2 - Mathematical form

Global norm clipping with threshold c uses:

$$\tilde{\mathbf{g}} = \mathbf{g} \min\left(1, \frac{c}{\|\mathbf{g}\| + \varepsilon}\right).$$

The clipped gradient satisfies $\|\tilde{\mathbf{g}}\| \leq c$.

Part 3 - Hints and Helper

Record pre-clipping norms and finite checks, apply clipping after backward, record post-clipping norms, and update only if values remain valid. Count how often clipping activates.

Part 4 - Python code skeleton

```
import torch

def E04(model, loss, max_norm):
    loss.backward()
    before = ...
    clipping_result = torch.nn.utils...
    after = ...
    safe = ...
    return before, after, safe

def main():
    print(E04(...))
```

```
if __name__ == "__main__":
    main()
```

4.7.6 Exercise 05 - Gradient Vanishing Mitigation

Part 1 - Why this matters

Vanishing gradients prevent early layers from receiving enough signal to learn. Mitigation may involve activation choice, initialization, normalization, residual paths, or architectural depth.

There is no meaningful way to clip a vanished gradient upward without changing the optimization problem. The appropriate response is to identify where attenuation begins and address its mechanism.

Part 2 - Mathematical form

For chained Jacobians:

$$\nabla_{h_0} L = \nabla_{h_K} L \prod_{k=1}^{K} J_k.$$

Repeated operator norms below one can make:

$$\|\nabla_{h_0} L\| \to 0.$$

Part 3 - Hints and Helper

Measure gradients layer by layer before changing the model. Compare a saturating activation or poor initialization with a suitable activation, initialization, or residual connection under identical data.

Part 4 - Python code skeleton

```
import torch

def E05(model_variants, X, target):
    reports = {}
    for name, model in model_variants.items():
        output = ...
        loss = ...
        loss.backward()
        reports[name] = ...
    return reports

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

4.7.7 Exercise 06 - Stability under Noisy Data

Part 1 - Why this matters

Noisy samples and mini-batches create fluctuating losses and gradients even when the expected optimization direction is useful. Stability means maintaining bounded, finite progress without overreacting to every observation.

Experiments should use repeated seeds and summarize distributions rather than one trajectory. Batch size, optimizer state, clipping, and learning-rate schedules influence the noise response.

Part 2 - Mathematical form

Let stochastic gradient be:

$$g_t = \nabla L(\theta_t) + \varepsilon_t, \qquad \mathbb{E}[\varepsilon_t] = 0.$$

Stability can be described using the mean and variance of loss, gradient norm, and parameter displacement over a window.

Part 3 - Hints and Helper

Add controlled noise with a seeded generator and run several trials. Report mean, variance, worst finite loss, and divergence frequency for each configuration.

Part 4 - Python code skeleton

```
import torch

def E06(train_function, noise_levels, seeds):
    results = {}
    for noise in noise_levels:
        trials = []
        for seed in seeds:
            trials.append(train_function(...))
        results[noise] = ...
    return results

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

4.7.8 Exercise 07 - Optimizer Comparison Experiment

Part 1 - Why this matters

Optimizer comparison determines how different update rules behave under the same task and computational budget. A fair experiment controls initialization, data order, model, loss,

batch size, and stopping rules.

The lowest final training loss is not enough. Speed, validation performance, stability, memory state, and sensitivity across seeds should also be considered.

Part 2 - Mathematical form

For optimizer o, define measurements:

$$\mathcal{M}_o = (L_{best}, L_{final}, T_{target}, G_{max}, D_o),$$

where T_{target} is time or steps to a target and D_o indicates divergence frequency.

Part 3 - Hints and Helper

Clone the same initial state for every optimizer and repeat across seeds. Store full histories and aggregate metrics with identical evaluation frequency.

Part 4 - Python code skeleton

```
import torch

def E07(model_factory, optimizer_factories, data, seeds):
    results = {}
    for name, optimizer_factory in optimizer_factories.items():
        trials = []
        for seed in seeds:
            model = model_factory(...)
            ...
            trials.append(...)
        results[name] = ...
    return results

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

4.7.9 Exercise 08 - Stability Debugging Scenario

Part 1 - Why this matters

Stability failures often involve interacting causes such as an excessive rate, poor initialization, incorrect loss reduction, stale gradients, non-finite data, or optimizer state errors.

A disciplined scenario captures evidence before changing anything. The first abnormal stage should determine one targeted correction, followed by a repeat of the same controlled run.

Part 2 - Mathematical form

For update:

$$\theta_{t+1} = \theta_t + \Delta\theta_t,$$

monitor relative size:

$$R_t = \frac{\|\Delta\theta_t\|}{\|\theta_t\| + \varepsilon}.$$

Also require finite inputs, activations, loss, gradients, optimizer states, and parameters.

Part 3 - Hints and Helper

Create one faulty training step and record diagnostics in pipeline order. Identify the earliest failed invariant, apply only its direct fix, and compare before-and-after histories.

Part 4 - Python code skeleton

```
import torch

def E08(model, optimizer, X, target):
    report = {}
    report["data"] = ...
    output = ...
    report["forward"] = ...
    loss = ...
    report["loss"] = ...
    loss.backward()
    report["gradients"] = ...
    report["diagnosis"] = ...
    return report

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

4.7.10 Exercise 09 - Long-Run Training Stability Test

Part 1 - Why this matters

A configuration that survives a few steps may fail after optimizer states grow, schedules change, rare batches occur, or numerical errors accumulate. Long-run testing exposes delayed instability.

The test must use periodic summaries and checkpoints rather than retaining every graph or printing every tensor. Reproducibility and recovery information are part of stability engineering.

Part 2 - Mathematical form

Across T steps, track:

$$L_t, \quad \|g_t\|, \quad \|\Delta\theta_t\|, \quad \|\theta_t\|.$$

A stable run requires finite bounded behavior and defined progress criteria throughout the horizon.

Part 3 - Hints and Helper

Run many steps with scheduled diagnostic intervals, finite checks, validation evaluation, and recoverable checkpoints. Report worst values, trend summaries, stopping reason, and the last valid step.

Part 4 - Python code skeleton

```
import torch

def E09(model, optimizer, train_loader,
        validation_function, max_steps, check_interval):
    summaries = []
    status = ...
    last_valid_step = ...
    for step in range(max_steps):
        ...
        if step % check_interval == 0:
            summary = ...
            summaries.append(summary)
        if ...:
            break
    return summaries, status, last_valid_step

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

4.8 OC06 - Weight Initialization Impact

Mathematics and Theory

4.8.1 Exercise 00 - Zero Initialization Experiment

Part 1 - Why this matters

Zero initialization provides a controlled demonstration of symmetry. When hidden units begin with identical weights, they receive identical signals and remain unable to learn distinct features.

Zero weights are not universally invalid: some biases and final parameters may safely begin at zero. The experiment should identify which tensors suffer from symmetry rather than applying one rule to every parameter.

Part 2 - Mathematical form

If two hidden units have:

$$\mathbf{w}_1 = \mathbf{w}_2 = \mathbf{0},$$

then they produce equal activations and, under symmetric conditions:

$$\nabla_{\mathbf{w}_1} L = \nabla_{\mathbf{w}_2} L.$$

Their updates remain identical.

Part 3 - Hints and Helper

Initialize all weights to zero, run one forward and backward pass, and compare unit activations and gradient rows. Repeat with zero biases but nonzero random weights to isolate the cause.

Part 4 - Python code skeleton

```
import torch

def E00(model, X, target):
    with torch.no_grad():
        for parameter in model.parameters():
            parameter...
    output = ...
    loss = ...
    loss.backward()
    diagnostics = ...
    return loss, diagnostics

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

4.8.2 Exercise 01 - Random Uniform Initialization

Part 1 - Why this matters

Uniform initialization samples weights from a bounded interval, breaking symmetry between units. The interval width determines the initial variance and therefore affects activation and gradient scale.

A reproducible experiment should use a seeded generator and report observed statistics. Choosing bounds without considering layer fan size can make deep signals grow or shrink.

Part 2 - Mathematical form

For:

$$W_{ij} \sim \mathcal{U}(-a, a),$$

the expectation and variance are:

$$\mathbb{E}[W_{ij}] = 0, \qquad \mathrm{Var}(W_{ij}) = \frac{a^2}{3}.$$

Part 3 - Hints and Helper

Initialize a weight tensor uniformly under `torch.no_grad()`. Record min, max, mean, variance, and reproducibility under a fixed seed, then inspect one forward pass.

Part 4 - Python code skeleton

```
import torch

def E01(shape, lower, upper, seed):
    generator = torch...
    weights = torch.empty(...)
    weights.uniform_(..., generator=generator)
    statistics = ...
    return weights, statistics

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

4.8.3 Exercise 02 - Random Normal Initialization

Part 1 - Why this matters

Normal initialization samples most weights near the mean while allowing occasional larger magnitudes. Its standard deviation directly controls the initial variance of linear combinations.

Using the same standard deviation for layers with different fan-in can produce inconsistent activation scales. Measuring empirical and predicted variance connects distribution choice with network structure.

Part 2 - Mathematical form

For:

$$W_{ij} \sim \mathcal{N}(\mu, \sigma^2),$$

usually $\mu = 0$. If independent inputs have variance q, a linear pre-activation approximately satisfies:

$$\mathrm{Var}(z) \approx fan_{in}\sigma^2 q.$$

Part 3 - Hints and Helper

Generate weights with a seeded normal distribution and compare empirical mean and variance with targets. Pass random inputs through the layer and record output variance.

Part 4 - Python code skeleton

```
import torch

def E02(shape, mean, standard_deviation, seed):
    generator = torch...
    weights = torch.empty(...)
    weights.normal_(..., generator=generator)
    statistics = ...
    return weights, statistics

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

4.8.4 Exercise 03 - Xavier Initialization Implementation

Part 1 - Why this matters

Xavier initialization chooses weight scale using both fan-in and fan-out. It is designed to preserve signal variance approximately through layers with symmetric activations such as tanh.

Implementing the formula requires correct interpretation of weight orientation. For a linear weight shaped `(out_features, in_features)`, fan-in and fan-out correspond to its second and first dimensions.

Part 2 - Mathematical form

Xavier normal uses:

$$\mathrm{Var}(W) = \frac{2}{fan_{in} + fan_{out}}.$$

Xavier uniform uses:

$$W_{ij} \sim \mathcal{U}(-a, a), \qquad a = \sqrt{\frac{6}{fan_{in} + fan_{out}}}.$$

Part 3 - Hints and Helper

Calculate fans from the layer shape and implement one Xavier variant manually. Compare observed statistics with `torch.nn.init.xavier_uniform_()` or its normal counterpart.

Part 4 - Python code skeleton

```
import torch

def E03(weight, seed):
    fan_in = ...
    fan_out = ...
    bound = ...
    with torch.no_grad():
        weight.uniform_(...)
```

```
    return weight, (...)

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

4.8.5 Exercise 04 - He Initialization Implementation

Part 1 - Why this matters

He initialization accounts for the variance reduction introduced by ReLU-like activations. It uses fan-in to maintain useful forward signal scale through deep rectified networks.

The formula is connected to the intended nonlinearity and propagation direction. Applying it mechanically to every activation can create a mismatch between its assumptions and actual network behavior.

Part 2 - Mathematical form

He normal for ReLU uses:

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{fan_{in}}\right),$$

so:

$$\sigma = \sqrt{\frac{2}{fan_{in}}}.$$

A corresponding uniform bound is $a = \sqrt{6/fan_{in}}$.

Part 3 - Hints and Helper

Compute fan-in correctly and implement He normal under no-gradient mode. Compare empirical variance and activation variance with PyTorch Kaiming initialization.

Part 4 - Python code skeleton

```
import torch

def E04(weight, seed):
    fan_in = ...
    standard_deviation = ...
    with torch.no_grad():
        weight.normal_(...)
    return weight, (...)

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

4.8.6 Exercise 05 - Initialization Effect on Gradient Flow

Part 1 - Why this matters

Initialization determines the scales of forward activations and backward gradients before learning has a chance to correct them. Poor scales can saturate activations, create dead units, or amplify and attenuate gradients across depth.

A useful analysis measures every layer rather than only the final loss. Comparing activation and gradient norms shows where signal quality first changes.

Part 2 - Mathematical form

For a deep chain:

$$h_\ell = \phi(W_\ell h_{\ell-1}),$$

backward flow contains:

$$\nabla_{h_{\ell-1}} L = W_\ell^\top \operatorname{diag}(\phi'(z_\ell))\nabla_{h_\ell} L.$$

The initial W_ℓ scales both directions.

Part 3 - Hints and Helper

Create identical architectures with several initialization schemes and the same input. Retain activation gradients, then record per-layer activation variance, saturation or zero rate, and gradient norm.

Part 4 - Python code skeleton

```
import torch

def E05(model_factories, X, target, seed):
    reports = {}
    for name, factory in model_factories.items():
        model = factory(...)
        activations = ...
        loss = ...
        loss.backward()
        reports[name] = ...
    return reports

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

4.8.7 Exercise 06 - Initialization vs Convergence Speed

Part 1 - Why this matters

A suitable initialization can reduce the number of optimization steps required to reach a target loss. It starts the model in a region where activations and gradients support effective

learning.

Convergence comparison must control optimizer, learning rate, data order, model architecture, and random seeds. One lucky run is not sufficient to judge an initialization method.

Part 2 - Mathematical form

For initialization method i, define:

$$T_i(\tau) = \min\{t : L_t^{(i)} \leq \tau\}.$$

If the target is never reached, report $T_i(\tau)$ as censored rather than assigning an artificial successful time.

Part 3 - Hints and Helper

Train each initialization under identical settings across multiple seeds. Report time or steps to target, best and final loss, divergence frequency, and variability.

Part 4 - Python code skeleton

```
import torch

def E06(model_factory, initialization_methods,
        train_function, seeds, target_loss):
    results = {}
    for method in initialization_methods:
        trials = []
        for seed in seeds:
            model = model_factory(...)
            method(...)
            trials.append(train_function(...))
        results[...] = ...
    return results

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

4.8.8 Exercise 07 - Bad Initialization Failure Case

Part 1 - Why this matters

Bad initialization can make weights too small, too large, identical, or incorrectly scaled for the activation. Symptoms include constant activations, saturation, dead ReLUs, vanished gradients, exploding gradients, and non-finite loss.

Constructing a controlled failure teaches how to connect observable symptoms to the starting parameter distribution. The correction should target the initialization rather than hide its effect with unrelated changes.

Part 2 - Mathematical form

If linear inputs are independent with variance q:

$$\mathrm{Var}(z) \approx fan_{in}\,\mathrm{Var}(W)q.$$

An excessively large $\mathrm{Var}(W)$ amplifies signals, while an excessively small value suppresses them.

Part 3 - Hints and Helper

Choose one intentionally poor scale and record weight statistics, layer activations, gradient norms, and finite checks before any optimizer step. Then apply one appropriate initialization and repeat.

Part 4 - Python code skeleton

```
import torch

def E07(model, bad_scale, X, target):
    initialize_badly = ...
    output = ...
    loss = ...
    loss.backward()
    failure_report = ...
    return failure_report

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

4.8.9 Exercise 08 - Initialization Debugging Challenge

Part 1 - Why this matters

Initialization bugs include wrong fan calculation, transposed shape assumptions, accidental reinitialization, mixed schemes, ignored bias policy, missing no-gradient context, and lost reproducibility.

Debugging should inspect the actual initialized tensors rather than trust the function name. Empirical statistics and first-pass signal measurements reveal whether the implementation matches the intended formula.

Part 2 - Mathematical form

For intended variance σ^2_{target}, compare:

$$r = \frac{\mathrm{Var}(W)_{observed}}{\sigma^2_{target}}.$$

Large deviation, incorrect mean, or identical rows can identify formula, sampling, or symmetry errors.

Part 3 - Hints and Helper

Record shape, fans, expected and observed statistics, duplicate-row checks, seed, activation variance, and gradient norm. Name the earliest inconsistency and change only its cause.

Part 4 - Python code skeleton

```
import torch

def E08(model, initialization_name, seed, X, target):
    report = {}
    report["parameter_statistics"] = ...
    output = ...
    report["activation_statistics"] = ...
    loss = ...
    loss.backward()
    report["gradient_statistics"] = ...
    report["diagnosis"] = ...
    return report

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

4.8.10 Exercise 09 - End-to-End Initialization Comparison

Part 1 - Why this matters

The final comparison evaluates initialization as part of a complete training system. It combines statistical validation, first-pass activation and gradient analysis, convergence speed, stability, and final performance.

A fair conclusion must aggregate multiple random seeds and keep all non-initialization settings fixed. The best method is the one that matches the architecture and activation under the chosen criteria, not a universal winner.

Part 2 - Mathematical form

For method i, collect:

$$\mathcal{M}_i = (V_W, V_h, G_1, T_\tau, L_{best}, D),$$

representing weight variance, activation variance, early gradient scale, time to target, best loss, and divergence. Aggregate these measurements across seeds.

Part 3 - Hints and Helper

Compare zero, uniform, normal, Xavier, and He methods on the same model family. Reset model and optimizer for every trial, store full histories, and rank methods with a stated criterion.

Part 4 - Python code skeleton

```
import torch

def E09(model_factory, initialization_methods,
        train_function, seeds):
    results = {}
    for method in initialization_methods:
        trials = []
        for seed in seeds:
            model = model_factory(...)
            method(model, seed)
            diagnostics = ...
            training_result = train_function(...)
            trials.append((diagnostics, training_result))
        results[method.__name__] = ...
    return results

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

Book 5

Loss Functions & Training Loop

5.1 Introduction

5.1.1 What will the book teach you

An artificial intelligence model does not learn merely because it produces an output. Learning begins when the output is compared with a desired target and that comparison is translated into a mathematical signal that the optimization process can use. A loss function performs this translation. It converts the difference between prediction and target into a scalar objective whose gradient describes how the model parameters should change. This book teaches how to design, implement, inspect, and use that signal inside a complete training system.

The choice of loss defines what the model is rewarded for improving. Two losses can evaluate the same predictions yet assign very different importance to large errors, outliers, class imbalance, uncertainty, and incorrect confidence. A loss is therefore not only a technical requirement for calling `backward()`; it is a mathematical statement about the behavior the model should learn. You will study how per-example errors become vector losses, how reduction converts them into a scalar, and how `sum` and `mean` change gradient scale as batch size changes.

For regression, you will implement mean squared error manually and through PyTorch. Given predictions $\widehat{y}_i$ and targets y_i, the standard mean squared error is:

$$L_{MSE} = \frac{1}{N}\sum_{i=1}^{N}(\widehat{y}_i - y_i)^2.$$

You will inspect its gradient, sensitivity to outliers, numerical behavior with large values, weighted variants, and differences from mean absolute error. The objective is to understand not only how to compute MSE, but why squaring errors changes the optimization signal and when that behavior is appropriate.

For classification, you will study cross-entropy from probabilities, one-hot targets, class indices, logits, LogSoftmax, and negative log-likelihood. You will derive the relationship:

$$\text{CrossEntropy}(\mathbf{z}, y) = -\log\left(\frac{e^{z_y}}{\sum_{c=1}^{C} e^{z_c}}\right),$$

while also learning why direct exponentiation can be numerically unstable. The exercises will connect the mathematical formula with PyTorch's expectation that `CrossEntropyLoss` normally receives raw logits and class-index targets, not already normalized probabilities. You

will debug incorrect shapes, class imbalance, unstable logarithms, and common cases where mathematically meaningful data is passed to the wrong interface.

After the loss functions are understood, the book builds the training loop from its individual state transitions. A complete training step follows a deliberate order: read a batch, calculate predictions, calculate a scalar loss, clear old gradients, perform backpropagation, update parameters, and record detached diagnostics. You will implement this process manually and with a PyTorch optimizer, then investigate bugs caused by misplaced gradient clearing, accumulated graphs, missing parameter updates, non-finite gradients, and incorrect logging.

Training on a dataset introduces batching and epochs. A mini-batch provides a stochastic estimate of the full-dataset gradient, while an epoch represents one complete traversal of the training data. You will construct batch and epoch loops, shuffle data correctly, handle the final incomplete batch, compare batch sizes, and deliberately accumulate gradients across several batches. Particular attention will be given to loss reduction and accumulation scaling, because the meaning of an update can change when the number of samples contributing to it is not accounted for.

A falling training loss does not prove that a model has learned a general rule. The model may instead memorize patterns specific to its training examples. You will therefore separate training and validation data, use `train()` and `eval()` modes correctly, disable gradient tracking during evaluation, and compute validation metrics without updating model parameters. You will learn to identify overfitting from the relationship between training and validation histories and to select a model from validation evidence rather than its final training loss.

The final part of the book treats overfitting prevention as a collection of controlled interventions. You will study weight decay, dropout, data augmentation, and early stopping, then connect them to the bias–variance tradeoff. Each method changes a different part of the learning system: the objective, the network behavior, the data distribution, or the training duration. The exercises will teach you to diagnose underfitting and overfitting before choosing a method, instead of adding every regularization technique at once.

5.1.2 How to approach the exercises

Treat every exercise as a small scientific experiment. Begin by identifying the prediction shape, target representation, unreduced loss shape, scalar reduction, and expected gradient scale. Predict the behavior before running the program, then compare the prediction with recorded values. When two losses, batch sizes, or regularization methods are compared, preserve the model initialization, data, optimizer settings, random seed, and training budget so that the intended factor is the only meaningful difference.

Do not evaluate a training pipeline using one number. Record training loss, validation loss, task metrics, gradient norms, learning rate, batch size, and the reason training stopped. Keep logging tensors detached from their computational graphs, and never allow validation data to influence parameter updates or hyperparameter decisions indirectly through data leakage. The goal is to produce training code that is not only functional, but explainable, reproducible, and safe to debug.

5.1.3 The Chapters (LFTL00 - LFTL06) included in this book

- LFTL00 - What is Loss and Why
- LFTL01 - MSE Loss Implementation
- LFTL02 - Cross-Entropy Logic
- LFTL03 - Building Training Loop Manually
- LFTL04 - Batching + Epochs
- LFTL05 - Evaluation & Validation Split
- LFTL06 - Overfitting Prevention Methods

5.2 LFTL00 - What is Loss and Why

Mathematics and Theory

5.2.1 Exercise 00 - Define Loss for Simple Regression

Part 1 - Why this matters

A regression model produces continuous predictions, but optimization requires a scalar objective describing their disagreement with targets. A loss function turns prediction error into a quantity that can be minimized.

Defining the loss makes the task objective explicit. Different transformations of the same error create different gradients and therefore different learning behavior.

Part 2 - Mathematical form

For prediction $\widehat{y}$ and target y, a simple squared loss is:

$$L(\widehat{y}, y) = (\widehat{y} - y)^2.$$

It is nonnegative and equals zero exactly when $\widehat{y} = y$.

Part 3 - Hints and Helper

Accept scalar floating-point tensors with compatible shapes. Compute the residual first and then the loss, returning both so their relationship can be inspected.

Part 4 - Python code skeleton

```
import torch

def E00(prediction, target):
    error = ...
    loss = ...
    return error, loss
```

```
def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

5.2.2 Exercise 01 - Compare Prediction vs Target

Part 1 - Why this matters

Loss calculation begins with a valid comparison between prediction and target. Their shapes, units, ordering, and semantic meaning must agree before a numerical difference can represent model error.

Broadcasting can make an incorrectly shaped comparison run without an exception. Shape validation protects the loss from silently comparing the wrong values.

Part 2 - Mathematical form

For vectors $\widehat{\mathbf{y}}, \mathbf{y} \in \mathbb{R}^N$, define residuals:

$$\mathbf{e} = \widehat{\mathbf{y}} - \mathbf{y}, \qquad e_i = \widehat{y}_i - y_i.$$

Useful raw comparisons include signed error e_i and absolute error $|e_i|$.

Part 3 - Hints and Helper

Check exact shape agreement before subtraction unless broadcasting is intentionally part of the task. Return signed errors, absolute errors, and a validity check.

Part 4 - Python code skeleton

```
import torch

def E01(predictions, targets):
    if ...:
        raise ValueError(...)
    signed_errors = ...
    absolute_errors = torch...
    return signed_errors, absolute_errors

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

5.2.3 Exercise 02 - Scalar Loss Computation

Part 1 - Why this matters

Backpropagation commonly begins from one scalar loss. When several errors exist, a reduction defines how they contribute to this single optimization objective.

A scalar loss is not merely required by software convenience. Its reduction determines the relative scale of gradients and the meaning of reported training values.

Part 2 - Mathematical form

Given per-element losses ℓ_i:

$$L = \frac{1}{N}\sum_{i=1}^{N} \ell_i$$

is a scalar mean reduction. For squared error:

$$\ell_i = (\widehat{y}_i - y_i)^2.$$

Part 3 - Hints and Helper

Compute an unreduced loss tensor first, inspect its shape, and then reduce it explicitly. Verify that the final tensor has zero dimensions before calling backward.

Part 4 - Python code skeleton

```
import torch

def E02(predictions, targets):
    element_losses = ...
    scalar_loss = torch...
    return element_losses, scalar_loss, scalar_loss.dim()

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

5.2.4 Exercise 03 - Vector Loss Aggregation

Part 1 - Why this matters

Multi-output predictions produce a vector or matrix of local losses. Aggregation controls how samples and features combine into one training signal.

The aggregation may weight dimensions differently when errors have different importance or units. Treating every component equally is a modeling choice rather than a universal rule.

Part 2 - Mathematical form

For element losses $\boldsymbol{\ell} \in \mathbb{R}^N$ and nonnegative weights $\mathbf{w}$:

$$L = \frac{\sum_{i=1}^{N} w_i \ell_i}{\sum_{i=1}^{N} w_i},$$

provided $\sum_i w_i > 0$. Equal weights recover the ordinary mean.

Part 3 - Hints and Helper

Create element-wise losses without reduction, validate optional weight shape and positivity, and implement an explicit aggregation. Return local contributions as well as the scalar result.

Part 4 - Python code skeleton

```
import torch

def E03(predictions, targets, weights=None):
    element_losses = ...
    if weights is None:
        aggregate = ...
    else:
        aggregate = ...
    return element_losses, aggregate

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

5.2.5 Exercise 04 - Mean Loss vs Sum Loss

Part 1 - Why this matters

Mean and sum reductions use the same local errors but produce different scalar and gradient scales. Sum grows with the number of elements, while mean normalizes by that count.

This distinction matters when batch size changes or gradients are accumulated. Comparing final loss values without naming the reduction can be misleading.

Part 2 - Mathematical form

For N local losses:

$$L_{sum} = \sum_{i=1}^{N} \ell_i, \qquad L_{mean} = \frac{1}{N} \sum_{i=1}^{N} \ell_i.$$

Therefore:

$$\nabla L_{mean} = \frac{1}{N} \nabla L_{sum}.$$

Part 3 - Hints and Helper

Compute both reductions from the same unreduced tensor. Use separate forward graphs or `torch.autograd.grad()` to compare prediction gradients and verify the expected scaling ratio.

Part 4 - Python code skeleton

```
import torch

def E04(predictions, targets):
    element_losses = ...
    sum_loss = ...
    mean_loss = ...
    sum_gradient = torch.autograd...
    mean_gradient = torch.autograd...
    return (...)

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

5.2.6 Exercise 05 - Loss Sensitivity Observation

Part 1 - Why this matters

Loss sensitivity describes how strongly the objective changes when a prediction moves away from its target. Squared error becomes increasingly sensitive to large residuals, while absolute error changes at a constant rate away from zero.

Observing values and gradients together explains why a loss reacts strongly to outliers or remains robust. Sensitivity is part of loss design, not an accidental side effect.

Part 2 - Mathematical form

For residual $e = \widehat{y} - y$:

$$L_{square} = e^2, \qquad \frac{\partial L_{square}}{\partial \widehat{y}} = 2e.$$

For absolute loss:

$$L_{absolute} = |e|,$$

whose slope magnitude is one away from $e = 0$.

Part 3 - Hints and Helper

Evaluate several residual sizes on both sides of zero. Record each loss and gradient, then compare how magnitude and sign change without mixing different target values.

Part 4 - Python code skeleton

```
import torch

def E05(prediction_values, target):
    records = []
    for value in prediction_values:
        prediction = torch.tensor(..., requires_grad=True)
        squared_loss = ...
```

```
        ...
        records.append(...)
    return records

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

5.2.7 Exercise 06 - Loss Landscape Visualization

Part 1 - Why this matters

A loss landscape shows how objective value changes as model parameters vary. For one or two parameters it can reveal minima, slopes, curvature, flat regions, and paths followed by optimization.

Visualization should be generated from a fixed dataset and model definition. Changing data or other parameters while scanning the surface changes the landscape being measured.

Part 2 - Mathematical form

For linear regression $\widehat{y}_i = wx_i + b$:

$$L(w, b) = \frac{1}{N} \sum_{i=1}^{N} (wx_i + b - y_i)^2.$$

Evaluating L over a grid of (w, b) values creates a two-dimensional loss surface.

Part 3 - Hints and Helper

Create parameter grids with `torch.linspace()` and evaluate the loss at every pair without training. Return grid coordinates, loss values, the minimum grid location, and data suitable for a surface or contour plot.

Part 4 - Python code skeleton

```
import torch

def E06(X, targets, weight_values, bias_values):
    landscape = torch...
    for i, weight in enumerate(weight_values):
        for j, bias in enumerate(bias_values):
            predictions = ...
            landscape[i, j] = ...
    minimum = ...
    return landscape, minimum

def main():
    print(E06(...))
```

```
if __name__ == "__main__":
    main()
```

5.2.8 Exercise 07 - Loss Scaling Effect

Part 1 - Why this matters

Multiplying a loss by a constant leaves its minimizer unchanged when the constant is positive, but it scales every gradient and therefore changes optimizer step magnitudes.

Loss scaling can interact with learning rate, clipping, numerical precision, batch accumulation, and mixed-precision training. Equivalent minima do not imply identical training trajectories.

Part 2 - Mathematical form

For scaled loss:

$$\widetilde{L} = \alpha L, \qquad \alpha > 0,$$

the gradient is:

$$\nabla \widetilde{L} = \alpha \nabla L.$$

The gradient-descent update becomes $\Delta\theta = -\eta\alpha\nabla L$.

Part 3 - Hints and Helper

Compute gradients for the original and scaled loss from identical predictions. Verify the scaling ratio, then compare updates while keeping the learning rate fixed.

Part 4 - Python code skeleton

```
import torch

def E07(predictions, targets, scale):
    base_loss = ...
    scaled_loss = ...
    base_gradient = torch.autograd...
    scaled_gradient = torch.autograd...
    return (...)

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

5.2.9 Exercise 08 - Poor Loss Choice Failure Case

Part 1 - Why this matters

A mathematically valid loss can be poorly aligned with the task. It may overemphasize outliers, ignore class imbalance, use incompatible units, or optimize a surrogate unrelated to the desired evaluation behavior.

The resulting model can minimize its training objective while performing poorly on the real goal. Diagnosing this requires comparing loss behavior with task metrics and error cases.

Part 2 - Mathematical form

Suppose one regression outlier has residual M. Under squared error its contribution is:

$$M^2,$$

while a typical residual m contributes m^2. Their ratio is:

$$\frac{M^2}{m^2},$$

which can allow one outlier to dominate the aggregate objective.

Part 3 - Hints and Helper

Construct a dataset with one controlled outlier and compare per-sample MSE and MAE contributions. Explain which samples dominate each loss and relate this to a chosen task requirement.

Part 4 - Python code skeleton

```
import torch

def E08(predictions, targets):
    squared_contributions = ...
    absolute_contributions = ...
    mse = ...
    mae = ...
    dominance_report = ...
    return (...)

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

5.2.10 Exercise 09 - Loss Design Reasoning Challenge

Part 1 - Why this matters

Designing a loss requires translating task priorities into mathematical penalties while preserving useful gradients and numerical stability. No single loss is optimal for every data distribution or evaluation goal.

A strong design states its assumptions, reduction, weighting, edge-case behavior, and relationship to the task metric. It should also identify failure modes before implementation.

Part 2 - Mathematical form

A composite objective may be:

$$L = \lambda_1 L_1 + \lambda_2 L_2, \qquad \lambda_1, \lambda_2 \geq 0.$$

Its gradient is:

$$\nabla L = \lambda_1 \nabla L_1 + \lambda_2 \nabla L_2.$$

The weights therefore control both objective importance and gradient scale.

Part 3 - Hints and Helper

Given a task description, define prediction and target representations, local penalty, reduction, optional weights, and stability safeguards. Implement only a skeleton and return diagnostic components separately from the final scalar.

Part 4 - Python code skeleton

```
import torch

def E09(predictions, targets, settings):
    primary_loss = ...
    secondary_loss = ...
    weights = ...
    total_loss = ...
    diagnostics = ...
    return total_loss, diagnostics

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

5.3 LFTL01 - MSE Loss Implementation

Mathematics and Theory

5.3.1 Exercise 00 - Manual MSE Formula Implementation

Part 1 - Why this matters

Implementing mean squared error manually reveals its three stages: compute residuals, square them, and average all selected elements. Each stage affects the final value and gradient.

The exercise provides a reference for validating library functions and custom reductions. Shape agreement must be checked before arithmetic to prevent accidental broadcasting.

Part 2 - Mathematical form

For N prediction-target pairs:

$$L_{MSE} = \frac{1}{N}\sum_{i=1}^{N}(\widehat{y}_i - y_i)^2.$$

The result is nonnegative and equals zero when every prediction matches its target.

Part 3 - Hints and Helper

Validate exact shape equality, calculate residuals and squared errors, then use `torch.mean()` for the scalar reduction. Return the intermediate tensors for inspection.

Part 4 - Python code skeleton

```
import torch

def E00(predictions, targets):
    if ...:
        raise ValueError(...)
    errors = ...
    squared_errors = ...
    mse = torch...
    return errors, squared_errors, mse

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

5.3.2 Exercise 01 - MSE Using PyTorch

Part 1 - Why this matters

PyTorch provides tested MSE implementations with explicit reduction options. Using them correctly requires understanding that `mean`, `sum`, and `none` produce different shapes and gradient scales.

Comparing library output with the manual formula verifies both the API and the user's interpretation of reduction. Matching numeric values alone is insufficient if shapes or reductions differ.

Part 2 - Mathematical form

With mean reduction:

$$\text{MSE}(\widehat{\mathbf{y}}, \mathbf{y}) = \frac{1}{N}\sum_{i}(\widehat{y}_i - y_i)^2.$$

With no reduction, the output is the element-wise tensor:

$$\ell_i = (\widehat{y}_i - y_i)^2.$$

Part 3 - Hints and Helper

Use `torch.nn.functional.mse_loss()` with multiple reductions. Compare the mean result with a manual implementation and report each output shape.

Part 4 - Python code skeleton

```
import torch

def E01(predictions, targets):
    unreduced = torch.nn.functional...
    mean_loss = torch.nn.functional...
    sum_loss = torch.nn.functional...
    manual = ...
    check = torch...
    return (...)

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

5.3.3 Exercise 02 - Batch-wise MSE Computation

Part 1 - Why this matters

A batch may require one loss per sample before those values are combined. Batch-wise MSE preserves sample identity by reducing only feature dimensions.

This distinction is useful for sample weighting, outlier detection, and analysis. A global mean instead mixes batch and feature dimensions immediately.

Part 2 - Mathematical form

For predictions $\widehat{\mathbf{Y}} \in \mathbb{R}^{B\times F}$:

$$L_i = \frac{1}{F}\sum_{j=1}^{F}(\widehat{Y}_{ij} - Y_{ij})^2,$$

giving $\mathbf{L} \in \mathbb{R}^B$. The batch mean is:

$$L = \frac{1}{B}\sum_i L_i.$$

Part 3 - Hints and Helper

Compute unreduced squared errors, identify all non-batch axes, and reduce them while retaining one value per sample. Then aggregate the sample losses explicitly.

Part 4 - Python code skeleton

```
import torch

def E02(predictions, targets):
    squared_errors = ...
    sample_losses = torch.mean(..., dim=...)
    batch_loss = torch...
    return sample_losses, batch_loss

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

5.3.4 Exercise 03 - MSE Gradient Inspection

Part 1 - Why this matters

The MSE gradient is proportional to the residual, so larger errors create larger corrective signals. Mean reduction also divides the gradient by the number of reduced elements.

Inspecting the gradient connects loss values with optimizer behavior. It makes reduction-related scaling visible before a training loop is introduced.

Part 2 - Mathematical form

For mean MSE:

$$L = \frac{1}{N}\sum_i (\widehat{y}_i - y_i)^2,$$

$$\frac{\partial L}{\partial \widehat{y}_i} = \frac{2}{N}(\widehat{y}_i - y_i).$$

Part 3 - Hints and Helper

Use tracked predictions, derive the manual gradient with the correct element count, call backward on the scalar loss, and compare shape, sign, and value.

Part 4 - Python code skeleton

```
import torch

def E03(predictions, targets):
    loss = ...
    manual_gradient = ...
    loss.backward()
    return loss, manual_gradient, predictions.grad

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

5.3.5 Exercise 04 - Effect of Outliers on MSE

Part 1 - Why this matters

Squaring residuals causes large errors to contribute disproportionately to MSE. One outlier can dominate the loss and gradient even when most predictions are accurate.

This sensitivity may be appropriate when large errors are especially costly, but harmful when targets contain corruption. Per-sample contributions reveal the effect clearly.

Part 2 - Mathematical form

If typical residual magnitude is a and an outlier has magnitude ka, their squared-loss ratio is:

$$\frac{(ka)^2}{a^2} = k^2.$$

Thus a residual ten times larger contributes one hundred times more squared error.

Part 3 - Hints and Helper

Create clean and outlier versions of the same data. Compare unreduced contributions, aggregate MSE, and prediction gradients while changing only one target.

Part 4 - Python code skeleton

```
import torch

def E04(predictions, clean_targets, outlier_targets):
    clean_losses = ...
    outlier_losses = ...
    contribution_ratio = ...
    return clean_losses, outlier_losses, contribution_ratio

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

5.3.6 Exercise 05 - MSE Stability with Large Values

Part 1 - Why this matters

Squaring very large residuals can overflow limited floating-point formats, producing infinite loss and unusable gradients. Large but finite values may also lose precision.

Numerical diagnosis should inspect residuals before squaring and compare dtypes. Scaling data or choosing a suitable precision addresses different causes from gradient clipping.

Part 2 - Mathematical form

For residual e:

$$L = e^2, \qquad \frac{dL}{d\widehat{y}} = 2e.$$

If $|e| > \sqrt{x_{max}}$ for the dtype, e^2 exceeds the largest representable finite value.

Part 3 - Hints and Helper

Evaluate increasing magnitudes in float32 and float64. Check residual, squared error, loss, and gradient with `torch.isfinite()`, then report the first unstable stage.

Part 4 - Python code skeleton

```
import torch

def E05(values, target, dtypes):
    report = {}
    for dtype in dtypes:
        predictions = torch.tensor(..., dtype=dtype,
                                   requires_grad=True)
        loss = ...
        ...
        report[str(dtype)] = ...
    return report

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

5.3.7 Exercise 06 - Weighted MSE Loss Challenge

Part 1 - Why this matters

Weighted MSE gives different samples or output dimensions different importance. Weights can represent reliability, priority, or unequal contribution to the task.

The normalization convention must be stated. Dividing by the number of elements and dividing by the total weight define different objectives when weights do not average to one.

Part 2 - Mathematical form

A normalized weighted MSE is:

$$L = \frac{\sum_i w_i(\widehat{y}_i - y_i)^2}{\sum_i w_i}, \qquad w_i \geq 0, \quad \sum_i w_i > 0.$$

Part 3 - Hints and Helper

Validate weight shape, nonnegativity, and positive total weight. Support only intentional broadcasting and return weighted local contributions before reduction.

Part 4 - Python code skeleton

```
import torch

def E06(predictions, targets, weights):
    if ...:
        raise ValueError(...)
    squared_errors = ...
    weighted_errors = ...
    loss = ...
    return weighted_errors, loss

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

5.3.8 Exercise 07 - MSE vs MAE Comparison

Part 1 - Why this matters

MSE and MAE measure regression error differently. MSE emphasizes large residuals with increasing gradient magnitude, while MAE grows linearly and has constant slope magnitude away from zero.

Their comparison should include loss contributions and gradients, not only aggregate values. The preferred behavior depends on noise, outliers, and task costs.

Part 2 - Mathematical form

For residual e:

$$L_{MSE} = e^2, \qquad L_{MAE} = |e|.$$

Their derivatives away from $e = 0$ are:

$$\frac{dL_{MSE}}{d\widehat{y}} = 2e, \qquad \frac{dL_{MAE}}{d\widehat{y}} = \text{sign}(e).$$

Part 3 - Hints and Helper

Use the same prediction-target pairs, including a controlled outlier. Compare unreduced losses, aggregate values, and gradients from separate graphs.

Part 4 - Python code skeleton

```
import torch

def E07(predictions, targets):
    mse = ...
    mae = ...
    mse_gradient = torch.autograd...
    mae_gradient = torch.autograd...
```

```
    return mse, mae, mse_gradient, mae_gradient

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

5.3.9 Exercise 08 - MSE Debugging Challenge

Part 1 - Why this matters

MSE bugs often arise from shape broadcasting, wrong target dtype, incorrect reduction axes, stale gradients, detached predictions, or confusion between squared error and root mean squared error.

A systematic diagnosis inspects inputs, unreduced loss, scalar reduction, graph connectivity, and gradients in order. Randomly reshaping tensors can hide the original semantic error.

Part 2 - Mathematical form

The expected invariants are:

$$\text{shape}(\hat{\mathbf{y}}) = \text{shape}(\mathbf{y}),$$

and for scalar mean loss:

$$\dim(L) = 0, \qquad \nabla_{\hat{\mathbf{y}}} L = \frac{2}{N}(\hat{\mathbf{y}} - \mathbf{y}).$$

Part 3 - Hints and Helper

Create one faulty example and print shape, dtype, device, reduction, `requires_grad`, and `grad_fn`. Identify the first violated invariant and fix only its cause.

Part 4 - Python code skeleton

```
import torch

def E08(predictions, targets):
    diagnostics = {
        "shapes": ...,
        "dtypes": ...,
        "devices": ...,
        "graph": ...
    }
    loss = ...
    diagnosis = ...
    return diagnostics, loss, diagnosis

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

5.3.10 Exercise 09 - Custom MSE Loss Class

Part 1 - Why this matters

A custom loss class packages validation, reduction, and optional weighting into a reusable module. It should preserve Autograd connectivity and follow clear PyTorch shape and reduction conventions.

Building the class demonstrates that custom loss code is ordinary differentiable tensor computation. State should be registered appropriately if persistent weights are used.

Part 2 - Mathematical form

For reduction r:

$$\ell_i = (\widehat{y}_i - y_i)^2,$$

$$L_r = \begin{cases} \boldsymbol{\ell}, & r = \text{none}, \\ \sum_i \ell_i, & r = \text{sum}, \\ \frac{1}{N}\sum_i \ell_i, & r = \text{mean}. \end{cases}$$

Part 3 - Hints and Helper

Subclass `torch.nn.Module`, validate the reduction in `__init__()`, and implement `forward()` with tensor operations. Compare all modes with PyTorch MSE.

Part 4 - Python code skeleton

```
import torch

class CustomMSELoss(torch.nn.Module):
    def __init__(self, reduction="mean"):
        super().__init__()
        if reduction not in (...):
            raise ValueError(...)
        self.reduction = reduction

    def forward(self, predictions, targets):
        squared_errors = ...
        if self.reduction == "none":
            return ...
        if self.reduction == "sum":
            return ...
        return ...

def E09(predictions, targets, reduction):
    criterion = CustomMSELoss(...)
    return criterion(...)

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

5.4 LFTL02 - Cross-Entropy Logic

Mathematics and Theory

5.4.1 Exercise 00 - Probability Output Validation

Part 1 - Why this matters

Cross-entropy defined on probabilities assumes each sample contains a valid categorical distribution. Values must be finite, nonnegative, no greater than one, and sum to one across the class axis.

Validation separates probabilities from logits. Logits are unrestricted scores and should not be rejected merely because they fall outside the probability interval.

Part 2 - Mathematical form

For probabilities $\mathbf{p} \in \mathbb{R}^C$:

$$0 \leq p_c \leq 1, \qquad \sum_{c=1}^{C} p_c = 1.$$

For a batch $\mathbf{P} \in \mathbb{R}^{B \times C}$, this condition is checked independently for every row.

Part 3 - Hints and Helper

Check dtype, finiteness, bounds, and row sums with a tolerance. Return per-sample validity and avoid applying this validator to raw logits.

Part 4 - Python code skeleton

```
import torch

def E00(probabilities, tolerance):
    finite = ...
    bounds = ...
    row_sums = ...
    normalized = torch...
    valid_rows = ...
    return valid_rows

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

5.4.2 Exercise 01 - One-Hot Target Encoding

Part 1 - Why this matters

One-hot encoding represents a class index as a vector with one active class. It connects the general cross-entropy sum with the simpler negative log-probability of the correct class.

The number of classes and valid index range must be known. PyTorch cross-entropy commonly expects integer indices directly, so one-hot encoding should be used only when the selected formula requires it.

Part 2 - Mathematical form

For class index $y \in \{0, \ldots, C-1\}$:

$$q_c = \begin{cases} 1, & c = y, \\ 0, & c \neq y. \end{cases}$$

Thus $\sum_c q_c = 1$.

Part 3 - Hints and Helper

Validate integer targets and bounds, then use an appropriate PyTorch encoding operation. Verify output shape, dtype, row sums, and recovered indices.

Part 4 - Python code skeleton

```
import torch

def E01(class_indices, number_of_classes):
    if ...:
        raise ValueError(...)
    one_hot = torch.nn.functional...
    checks = ...
    return one_hot, checks

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

5.4.3 Exercise 02 - Manual Cross-Entropy Computation

Part 1 - Why this matters

Manual cross-entropy reveals that the loss penalizes low probability assigned to the target distribution. For a one-hot target, only the correct class contributes directly to the sum.

Taking a logarithm requires strictly positive probabilities. Clamping can prevent undefined values for an educational probability-based formula, but stable logits-based computation is preferred in training.

Part 2 - Mathematical form

For target distribution $\mathbf{q}$ and predicted probabilities $\mathbf{p}$:

$$H(\mathbf{q}, \mathbf{p}) = -\sum_{c=1}^{C} q_c \log p_c.$$

For one-hot target y:

$$H = -\log p_y.$$

Part 3 - Hints and Helper

Validate probability and target shapes, clamp only with a documented epsilon, compute per-sample losses, and apply an explicit reduction. Compare one-hot and gathered-index forms.

Part 4 - Python code skeleton

```
import torch

def E02(probabilities, one_hot_targets, epsilon):
    safe_probabilities = torch...
    log_probabilities = torch...
    sample_losses = ...
    loss = ...
    return sample_losses, loss

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

5.4.4 Exercise 03 - Cross-Entropy Using PyTorch

Part 1 - Why this matters

PyTorch cross-entropy combines stable LogSoftmax and negative log-likelihood. Its usual classification interface expects raw logits shaped by batch and class, plus integer class indices.

Applying Softmax before this function changes the mathematical input and can weaken or distort the intended computation. Knowing the contract prevents a common double-normalization error.

Part 2 - Mathematical form

For logits $\mathbf{z}$ and target class y:

$$L = -z_y + \log \sum_{c=1}^{C} e^{z_c}.$$

This equals:

$$-\log\left(\frac{e^{z_y}}{\sum_c e^{z_c}}\right).$$

Part 3 - Hints and Helper

Use `torch.nn.functional.cross_entropy()` with logits and long class-index targets. Explore `none`, `mean`, and `sum` reductions and verify output shapes.

Part 4 - Python code skeleton

```
import torch

def E03(logits, class_targets):
    unreduced = torch.nn.functional...
    mean_loss = torch.nn.functional...
    sum_loss = torch.nn.functional...
    return unreduced, mean_loss, sum_loss

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

5.4.5 Exercise 04 - LogSoftmax + NLL Breakdown

Part 1 - Why this matters

Cross-entropy can be decomposed into LogSoftmax followed by negative log-likelihood. The first operation converts logits into stable log-probabilities; the second selects and negates the target-class value.

This decomposition makes intermediate values inspectable without changing the objective. It also explains why NLLLoss expects log-probabilities rather than logits or ordinary probabilities.

Part 2 - Mathematical form

LogSoftmax is:

$$\log p_c = z_c - \log \sum_j e^{z_j}.$$

For target y, NLL is:

$$L = -\log p_y.$$

Therefore $\text{CrossEntropy}(\mathbf{z}, y) = \text{NLL}(\log \text{Softmax}(\mathbf{z}), y)$.

Part 3 - Hints and Helper

Compute `log_softmax(logits, dim=1)` and pass it to `nll_loss`. Compare values and gradients with direct cross-entropy using separate graphs.

Part 4 - Python code skeleton

```
import torch

def E04(logits, targets):
    log_probabilities = torch.nn.functional...
    nll = torch.nn.functional...
    direct = torch.nn.functional...
    comparison = torch...
    return log_probabilities, nll, direct, comparison

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

5.4.6 Exercise 05 - Numerical Stability (Log-Sum-Exp)

Part 1 - Why this matters

Direct exponentiation of large logits can overflow, while very negative values can underflow. The log-sum-exp transformation computes the same quantity after shifting logits by their maximum.

The shift does not change Softmax probabilities because it affects numerator and denominator equally. Stable computation preserves finite losses and useful gradients over a wider range.

Part 2 - Mathematical form

Let $m = \max_c z_c$. Then:

$$\log \sum_c e^{z_c} = m + \log \sum_c e^{z_c - m}.$$

Since $z_c - m \leq 0$, the exponentials cannot overflow from large positive arguments.

Part 3 - Hints and Helper

Implement naive and stable log-sum-exp on moderate and extreme logits. Compare with `torch.logsumexp()` and report finiteness and numerical error.

Part 4 - Python code skeleton

```
import torch

def E05(logits, dimension):
    naive = torch.log(torch.sum(torch.exp(...), dim=...))
    maximum = torch.max(...).values
    stable = ...
    reference = torch.logsumexp(...)
    return naive, stable, reference

def main():
```

```
    print(E05(...))

if __name__ == "__main__":
    main()
```

5.4.7 Exercise 06 - Batch Cross-Entropy Loss

Part 1 - Why this matters

Batch cross-entropy produces one loss per sample before reduction. Keeping these values visible supports weighting, debugging mislabeled examples, and interpreting how mean and sum scale with batch size.

The class axis and batch axis have different roles. Only the target class is selected per sample, while reduction combines sample losses.

Part 2 - Mathematical form

For logits $\mathbf{Z} \in \mathbb{R}^{B \times C}$ and targets y_i:

$$\ell_i = -Z_{i,y_i} + \log \sum_{c=1}^{C} e^{Z_{ic}}.$$

Mean batch loss is:

$$L = \frac{1}{B} \sum_{i=1}^{B} \ell_i.$$

Part 3 - Hints and Helper

Compute unreduced cross-entropy, verify the result has shape `(B,)`, and aggregate explicitly. Compare mean and sum gradients using separate forward graphs.

Part 4 - Python code skeleton

```
import torch

def E06(logits, targets):
    sample_losses = torch.nn.functional...
    mean_loss = ...
    sum_loss = ...
    return sample_losses, mean_loss, sum_loss

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

5.4.8 Exercise 07 - Wrong Target Shape Debugging

Part 1 - Why this matters

Classification targets are a frequent source of cross-entropy errors. The usual PyTorch case expects one integer class index per sample, not an extra singleton dimension or a one-hot matrix.

A tensor can contain correct labels but still violate the function's representation contract. Debugging should inspect shape, dtype, range, and correspondence with the logits batch.

Part 2 - Mathematical form

For logits:

$$\mathbf{Z} \in \mathbb{R}^{B \times C},$$

class-index targets should satisfy:

$$\mathbf{y} \in \{0, \ldots, C-1\}^{B}.$$

Thus target shape is (B) and its dtype is integer.

Part 3 - Hints and Helper

Create faulty target examples such as `(B,1)`, floating indices, and one-hot `(B,C)`. Diagnose each before applying a representation-aware correction.

Part 4 - Python code skeleton

```
import torch

def E07(logits, targets):
    diagnostics = {
        "logits_shape": ...,
        "target_shape": ...,
        "target_dtype": ...,
        "target_range": ...
    }
    corrected_targets = ...
    return diagnostics, corrected_targets

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

5.4.9 Exercise 08 - Class Imbalance Effect

Part 1 - Why this matters

With class imbalance, ordinary mean cross-entropy can be dominated by frequent classes. A model may reduce average loss while learning weak behavior for rare but important classes.

Class weighting changes each target class's contribution to the objective and gradient. Weights must be chosen from task priorities or data statistics and evaluated with class-sensitive metrics.

Part 2 - Mathematical form

With class weight w_c:

$$\ell_i = -w_{y_i} \log p_{i,y_i}.$$

A common inverse-frequency idea is:

$$w_c \propto \frac{1}{n_c},$$

followed by a documented normalization convention.

Part 3 - Hints and Helper

Count class frequencies, construct positive class weights, and compare per-class loss contributions with and without weighting. Keep logits and targets identical.

Part 4 - Python code skeleton

```
import torch

def E08(logits, targets, number_of_classes):
    counts = torch...
    class_weights = ...
    unweighted = torch.nn.functional...
    weighted = torch.nn.functional...
    report = ...
    return class_weights, report

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

5.4.10 Exercise 09 - Custom Cross-Entropy Loss

Part 1 - Why this matters

A custom cross-entropy module combines stable log-probabilities, target selection, optional class weighting, and reduction. Its output and gradients should match PyTorch for supported cases.

The implementation must avoid explicit Softmax and logarithm of probabilities when logits are available. Shape validation and stable LogSoftmax belong inside the module contract.

Part 2 - Mathematical form

For log-probabilities:

$$\log p_{ic} = Z_{ic} - \log \sum_j e^{Z_{ij}},$$

the sample loss is:

$$\ell_i = -w_{y_i} \log p_{i,y_i},$$

with $w_{y_i} = 1$ when no class weights are supplied.

Part 3 - Hints and Helper

Subclass `torch.nn.Module`, use stable `log_softmax`, gather target entries, apply optional registered weights, and support explicit reduction modes. Compare values and gradients with PyTorch.

Part 4 - Python code skeleton

```
import torch

class CustomCrossEntropy(torch.nn.Module):
    def __init__(self, reduction="mean", class_weights=None):
        super().__init__()
        self.reduction = ...
        self.register_buffer("class_weights", ...)

    def forward(self, logits, targets):
        log_probabilities = ...
        selected = ...
        sample_losses = ...
        if ...:
            return ...
        ...

def E09(logits, targets):
    criterion = CustomCrossEntropy(...)
    return criterion(...)

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

5.5 LFTL03 - Building Training Loop Manually

Mathematics and Theory

5.5.1 Exercise 00 - Single-Step Training Loop

Part 1 - Why this matters

A single training step contains the complete learning cycle for one batch. It transforms inputs into predictions, measures error, computes gradients, and changes parameters once.

Studying one step makes the order and state transitions visible before repetition is introduced. Every multi-epoch training system is built from this same unit.

Part 2 - Mathematical form

For model f_θ, batch $(\mathbf{X}, \mathbf{y})$, and learning rate η:

$$\widehat{\mathbf{y}} = f_\theta(\mathbf{X}), \quad L = \ell(\widehat{\mathbf{y}}, \mathbf{y}),$$

$$g = \nabla_\theta L, \quad \theta' = \theta - \eta g.$$

Part 3 - Hints and Helper

Use one batch and return predictions, loss, pre-update gradients, and updated parameters. Ensure the update occurs outside gradient tracking and do not reuse stale gradients.

Part 4 - Python code skeleton

```
import torch

def E00(model, X, targets, loss_function, learning_rate):
    predictions = ...
    loss = ...
    ...
    loss.backward()
    gradients = ...
    with torch.no_grad():
        ...
    return predictions, loss, gradients

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

5.5.2 Exercise 01 - Forward + Loss + Backward Loop

Part 1 - Why this matters

The forward, loss, and backward stages create the information required for learning before any parameter update occurs. Separating them allows each stage to be verified independently.

A valid backward pass needs a scalar loss connected to trainable parameters. Shape, dtype, and graph mistakes should be found before optimization is added.

Part 2 - Mathematical form

At iteration t:

$$\widehat{\mathbf{y}}_t = f_{\theta_t}(\mathbf{X}), \qquad L_t = \ell(\widehat{\mathbf{y}}_t, \mathbf{y}),$$

$$\nabla_{\theta_t} L_t$$

is computed while θ_t remains unchanged.

Part 3 - Hints and Helper

Loop for several iterations but omit parameter updates intentionally. Clear gradients, run forward, compute scalar loss, call backward, and record detached loss and gradient norms.

Part 4 - Python code skeleton

```
import torch

def E01(model, X, targets, loss_function, steps):
    history = []
    for step in range(steps):
        model.zero_grad(...)
        predictions = ...
        loss = ...
        loss.backward()
        history.append(...)
    return history

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

5.5.3 Exercise 02 - Parameter Update Integration

Part 1 - Why this matters

A training loop learns only when computed gradients are applied to parameters. The update must use current gradients and must not become part of the next computational graph.

Integrating updates reveals the boundary between differentiation and optimization. Parameter identity should be preserved while values change under no-gradient mode.

Part 2 - Mathematical form

For each parameter $\theta^{(i)}$:

$$\theta_{t+1}^{(i)} = \theta_t^{(i)} - \eta \nabla_{\theta^{(i)}} L_t.$$

The displacement must satisfy:

$$\Delta\theta^{(i)} = -\eta\, \theta^{(i)}.grad.$$

Part 3 - Hints and Helper

Clone parameter values before updating, apply updates under `torch.no_grad()`, and compare actual displacement with the expected rule. Skip or diagnose `None` gradients.

Part 4 - Python code skeleton

```
import torch

def E02(model, learning_rate):
    before = ...
    with torch.no_grad():
        for parameter in model.parameters():
            if parameter.grad is not None:
                parameter ...
    after = ...
    return before, after

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

5.5.4 Exercise 03 - Gradient Zeroing Placement

Part 1 - Why this matters

PyTorch accumulates parameter gradients, so a training loop must clear them once per intended update. Zeroing too late uses stale gradients; zeroing after backward but before step erases the gradients needed for the update.

Clearing before the forward pass or immediately after the completed update can both work when used consistently. The exercise focuses on preserving one batch's gradient for one update.

Part 2 - Mathematical form

Without clearing:

$$g_{stored,t} = g_{stored,t-1} + g_t.$$

For ordinary independent steps, the desired state is:

$$g_{stored,t} = g_t.$$

Part 3 - Hints and Helper

Implement two short loops with correct and incorrect placement. Record gradients before every update and identify accumulation or accidental erasure.

Part 4 - Python code skeleton

```
import torch

def E03(model, batches, loss_function):
    records = []
    for X, targets in batches:
        ...
        predictions = ...
```

```
        loss = ...
        loss.backward()
        records.append(...)
        ...
    return records

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

5.5.5 Exercise 04 - Training Loop without Optimizer

Part 1 - Why this matters

A manual training loop applies gradient descent directly instead of using `torch.optim`. It exposes gradient clearing, update magnitude, and parameter state in every iteration.

This implementation provides a reference for understanding optimizer classes. It also ensures the reader can separate Autograd's derivative calculation from the chosen update rule.

Part 2 - Mathematical form

At each step:

$$L_t = \ell(f_{\theta_t}(X), y), \quad g_t = \nabla_\theta L_t,$$
$$\theta_{t+1} = \theta_t - \eta g_t.$$

Part 3 - Hints and Helper

Clear gradients, compute the current loss, call backward, update every parameter under no-gradient mode, and log detached values. Do not replace parameters with new tensor objects.

Part 4 - Python code skeleton

```
import torch

def E04(model, X, targets, loss_function,
        learning_rate, steps):
    history = []
    for step in range(steps):
        ...
        with torch.no_grad():
            for parameter in model.parameters():
                parameter ...
        history.append(...)
    return model, history

def main():
    print(E04(...))
```

```
if __name__ == "__main__":
    main()
```

5.5.6 Exercise 05 - Loss Logging per Iteration

Part 1 - Why this matters

Loss history is the primary trace of training behavior. Correct logging records numerical snapshots without retaining the graph from every iteration.

A useful entry also includes iteration, learning rate, and selected diagnostics. Logging live tensors can cause unnecessary memory retention and misleading later values.

Part 2 - Mathematical form

For T iterations, define:

$$\mathcal{H} = \{(t, L_t)\}_{t=0}^{T-1}.$$

An improvement value may be:

$$\Delta_t = L_{t-1} - L_t.$$

Part 3 - Hints and Helper

Store `loss.detach().item()` rather than the loss tensor. Add step and optional gradient norm, then summarize best loss, best step, and number of increases.

Part 4 - Python code skeleton

```
import torch

def E05(training_step, steps):
    history = []
    for step in range(steps):
        loss, gradient_norm = training_step(...)
        history.append({
            "step": step,
            "loss": ...,
            "gradient_norm": ...
        })
    summary = ...
    return history, summary

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

5.5.7 Exercise 06 - Training Loop Bug Debugging

Part 1 - Why this matters

Training loops can fail through missing updates, missing backward calls, stale gradients, detached losses, wrong modes, or incorrect operation order. Several bugs can produce a flat loss, so symptoms alone are insufficient.

Debugging should inspect each stage and verify that parameters actually change. A minimal deterministic batch makes the first broken invariant easier to locate.

Part 2 - Mathematical form

A valid step must satisfy:

$$L \to \nabla_\theta L \to \Delta\theta.$$

For a trainable parameter, check:

$$\theta.grad \neq \text{None}, \qquad \theta_{after} - \theta_{before} = -\eta\theta.grad$$

for manual gradient descent.

Part 3 - Hints and Helper

Instrument forward shape, loss scalarity, graph metadata, gradients, and parameter displacement. Identify the first failed check and repair only that operation.

Part 4 - Python code skeleton

```
import torch

def E06(model, X, targets, loss_function, learning_rate):
    report = {}
    before = ...
    predictions = ...
    report["forward"] = ...
    loss = ...
    report["loss"] = ...
    ...
    report["gradients"] = ...
    report["updates"] = ...
    report["diagnosis"] = ...
    return report

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

5.5.8 Exercise 07 - Manual vs Optimizer Loop Comparison

Part 1 - Why this matters

A manual update loop and a library optimizer should match when they implement the same rule with identical initial parameters, data, reduction, learning rate, and gradient clearing.

The comparison validates manual logic and clarifies which responsibilities the optimizer automates. Stateful optimizers require additional state matching, so basic SGD is the cleanest reference.

Part 2 - Mathematical form

For SGD without momentum:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L_t.$$

Both loops should generate equal parameter sequences up to floating-point tolerance when every condition is identical.

Part 3 - Hints and Helper

Clone one initial model into two copies. Use a manual update for one and `torch.optim.SGD` for the other, then compare loss and parameters after every step.

Part 4 - Python code skeleton

```
import torch

def E07(model_manual, model_optimizer, batches,
        loss_function, learning_rate):
    optimizer = torch.optim...
    comparisons = []
    for X, targets in batches:
        ...
        comparisons.append(...)
    return comparisons

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

5.5.9 Exercise 08 - Gradient Explosion inside Loop

Part 1 - Why this matters

A loop can become unstable when gradients or update magnitudes grow rapidly. Detecting this before the optimizer step can prevent non-finite parameters from contaminating later iterations.

Clipping can limit immediate damage but should be accompanied by diagnostics for learning rate, loss scaling, initialization, and data. The loop must record pre-clipping evidence.

Part 2 - Mathematical form

For global gradient norm:

$$G_t = \sqrt{\sum_p \|p.grad\|_2^2}.$$

Norm clipping at c uses:

$$\widetilde{g} = g \min\left(1, \frac{c}{G_t + \varepsilon}\right).$$

Part 3 - Hints and Helper

After backward, compute finiteness and global norm before any update. Stop or clip according to defined rules, record pre- and post-clipping values, and never update with non-finite gradients.

Part 4 - Python code skeleton

```
import torch

def E08(model, X, targets, loss_function,
        learning_rate, max_norm, steps):
    history = []
    for step in range(steps):
        ...
        loss.backward()
        before_norm = ...
        finite = ...
        if not finite:
            break
        torch.nn.utils...
        ...
        history.append(...)
    return history

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

5.5.10 Exercise 09 - End-to-End Manual Training Loop

Part 1 - Why this matters

The final challenge combines data input, forward computation, loss, gradient management, manual parameter updates, logging, stability checks, and stopping logic in one reproducible loop.

A completed loop should return evidence, not only a trained model. Its history and stopping reason must explain whether training converged, diverged, or exhausted the budget.

Part 2 - Mathematical form

For batches indexed by t:

$$\widehat{y}_t = f_{\theta_t}(X_t), \quad L_t = \ell(\widehat{y}_t, y_t),$$

$$g_t = \nabla_\theta L_t, \quad \theta_{t+1} = \theta_t - \eta g_t.$$

All diagnostics must be detached before storage.

Part 3 - Hints and Helper

Implement without `torch.optim`. Include gradient clearing, scalar loss validation, finite checks, optional clipping, parameter updates, loss logging, and a defined stopping criterion.

Part 4 - Python code skeleton

```
import torch

def E09(model, batches, loss_function, learning_rate,
        max_steps, tolerance, max_grad_norm=None):
    history = []
    status = ...
    for step, (X, targets) in enumerate(batches):
        if step >= max_steps:
            break
        ...
        history.append(...)
        if ...:
            status = ...
            break
    return model, history, status

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

5.6 LFTL04 - Batching + Epochs

Mathematics and Theory

5.6.1 Exercise 00 - Single-Batch Training

Part 1 - Why this matters

Single-batch training applies one complete update using a selected subset of the dataset. It provides the simplest bridge between an abstract training step and data organized into batches.

The batch dimension must align between inputs and targets. Loss reduction determines whether the update scale depends on the number of samples.

Part 2 - Mathematical form

For batch $\mathcal{B}$ of size B:

$$L_{\mathcal{B}}(\theta) = \frac{1}{B} \sum_{i \in \mathcal{B}} \ell(f_\theta(x_i), y_i).$$

One update uses $\theta' = \theta - \eta \nabla_\theta L_{\mathcal{B}}$.

Part 3 - Hints and Helper

Validate matching batch dimensions, run one forward-loss-backward-update cycle, and return batch size, detached loss, gradient norms, and parameter displacement.

Part 4 - Python code skeleton

```
import torch

def E00(model, X_batch, target_batch,
        loss_function, optimizer):
    optimizer.zero_grad(...)
    predictions = ...
    loss = ...
    loss.backward()
    diagnostics = ...
    optimizer.step()
    return loss, diagnostics

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

5.6.2 Exercise 01 - Mini-Batch Training Setup

Part 1 - Why this matters

Mini-batching divides a dataset into manageable subsets that produce stochastic gradient estimates. It balances memory use, computational efficiency, and gradient variability.

A correct setup preserves input-target correspondence and defines whether data are shuffled, whether the final batch is kept, and how batch size affects reduction.

Part 2 - Mathematical form

For N samples and batch size B, the number of batches when keeping the remainder is:

$$K = \left\lceil \frac{N}{B} \right\rceil .$$

Batch k contains aligned pairs (X_i, y_i) from the same selected indices.

Part 3 - Hints and Helper

Use `TensorDataset` and `DataLoader` or implement index slicing manually. Inspect every batch shape and verify that all sample indices appear exactly once when shuffling is disabled.

Part 4 - Python code skeleton

```
import torch

def E01(features, targets, batch_size):
    dataset = torch.utils.data...
    loader = torch.utils.data...
    batch_shapes = []
    for X_batch, target_batch in loader:
        batch_shapes.append(...)
    return loader, batch_shapes

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

5.6.3 Exercise 02 - Epoch Loop Construction

Part 1 - Why this matters

An epoch is one complete pass through the training dataset. The epoch loop nests a batch-training loop and aggregates batch statistics into an epoch-level summary.

A simple average of batch means is biased when the final batch is smaller. Correct epoch loss weights each batch mean by its sample count.

Part 2 - Mathematical form

If batch k has size B_k and mean loss L_k, the epoch mean is:

$$L_{epoch} = \frac{\sum_{k=1}^{K} B_k L_k}{\sum_{k=1}^{K} B_k}.$$

Part 3 - Hints and Helper

Nest the DataLoader loop inside an epoch loop. Accumulate `loss.item() * batch_size` and the total sample count, then divide after the epoch.

Part 4 - Python code skeleton

```
import torch

def E02(model, loader, loss_function, optimizer, epochs):
    epoch_history = []
    for epoch in range(epochs):
        total_loss = ...
        sample_count = ...
        for X_batch, targets in loader:
            ...
        epoch_loss = ...
        epoch_history.append(...)
    return epoch_history

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

5.6.4 Exercise 03 - Shuffling Data per Epoch

Part 1 - Why this matters

Shuffling changes sample order so batches do not repeatedly contain the same neighboring examples. This reduces order-related bias and changes the stochastic gradient sequence across epochs.

Features and targets must be shuffled with the same permutation. Reproducible experiments should control the random generator while still producing a new order each epoch.

Part 2 - Mathematical form

For epoch e, let π_e be a permutation of:

$$\{0, 1, \ldots, N-1\}.$$

The shuffled dataset uses aligned pairs:

$$(X_{\pi_e(i)}, y_{\pi_e(i)}).$$

Part 3 - Hints and Helper

Use a DataLoader with shuffling and a seeded generator, or apply one shared `randperm`. Record sample identifiers to verify complete coverage, alignment, and changed order across epochs.

Part 4 - Python code skeleton

```
import torch

def E03(features, targets, epochs, seed):
```

```
    generator = torch.Generator().manual_seed(...)
    orders = []
    for epoch in range(epochs):
        permutation = torch...
        shuffled_features = ...
        shuffled_targets = ...
        orders.append(permutation...)
    return orders

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

5.6.5 Exercise 04 - Batch Size Effect Observation

Part 1 - Why this matters

Batch size changes gradient noise, number of updates per epoch, memory consumption, and throughput. Smaller batches provide more frequent noisy updates, while larger batches provide smoother estimates.

A fair experiment keeps dataset, model initialization, optimizer, epoch count, and loss reduction fixed. Equal epochs do not imply equal numbers of updates across batch sizes.

Part 2 - Mathematical form

For batch gradient:

$$g_{\mathcal{B}} = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla_\theta \ell_i.$$

As batch size grows, its variance often decreases, while updates per epoch approximately decrease as N/B.

Part 3 - Hints and Helper

Train cloned models with several batch sizes and multiple seeds. Record updates, gradient variance, loss, memory, and elapsed time rather than only final loss.

Part 4 - Python code skeleton

```
import torch

def E04(model_factory, dataset, batch_sizes,
        train_function, seeds):
    results = {}
    for batch_size in batch_sizes:
        trials = []
        for seed in seeds:
            trials.append(train_function(...))
```

```
        results[batch_size] = ...
    return results

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

5.6.6 Exercise 05 - Gradient Accumulation across Batches

Part 1 - Why this matters

Gradient accumulation combines several micro-batch gradients before one optimizer update, approximating a larger effective batch when memory is limited.

Loss scaling must preserve the intended mean gradient. Optimizer stepping and gradient clearing occur once per accumulation window, including a possibly incomplete final window.

Part 2 - Mathematical form

For K equal micro-batches and mean losses L_k:

$$g = \frac{1}{K} \sum_{k=1}^{K} \nabla_\theta L_k.$$

This can be obtained by backpropagating L_k/K before one update.

Part 3 - Hints and Helper

Divide each micro-batch loss by the accumulation count, call backward repeatedly, and step only at the boundary. Handle the final shorter window with a documented scaling strategy.

Part 4 - Python code skeleton

```
import torch

def E05(model, loader, loss_function,
        optimizer, accumulation_steps):
    optimizer.zero_grad(...)
    records = []
    for batch_index, (X_batch, targets) in enumerate(loader):
        loss = ...
        scaled_loss = ...
        scaled_loss.backward()
        if ...:
            optimizer.step()
            optimizer.zero_grad(...)
        records.append(...)
    return records
```

```
def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

5.6.7 Exercise 06 - Last Incomplete Batch Handling

Part 1 - Why this matters

When dataset size is not divisible by batch size, the final batch contains fewer samples. Keeping it uses all data, while dropping it creates equal shapes but omits samples each epoch.

Loss aggregation and gradient accumulation must account for the smaller count. Code that assumes every batch has the configured size can fail or misreport epoch metrics.

Part 2 - Mathematical form

For $N = qB + r$ with $0 < r < B$:

$$K = q + 1$$

when the final batch is kept, and its size is r. With `drop_last=True`, only qB samples are processed.

Part 3 - Hints and Helper

Choose a dataset with a nonzero remainder and compare both policies. Record batch sizes, processed sample count, epoch loss weighting, and any model constraints that require fixed size.

Part 4 - Python code skeleton

```
import torch

def E06(features, targets, batch_size, drop_last):
    loader = torch.utils.data...
    batch_sizes = []
    processed = ...
    for X_batch, target_batch in loader:
        batch_sizes.append(...)
        processed += ...
    return batch_sizes, processed

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

5.6.8 Exercise 07 - Performance vs Batch Size

Part 1 - Why this matters

Batch size affects hardware utilization, memory allocation, data-loading overhead, and the number of optimizer steps. Throughput may improve up to a point and then plateau or exceed memory.

Performance measurement must separate warm-up, data loading, forward/backward time, and synchronization. Training speed should be considered together with optimization quality.

Part 2 - Mathematical form

Sample throughput is:

$$R = \frac{\text{processed samples}}{\text{elapsed time}}.$$

Memory and time per optimizer step can also be recorded, while convergence efficiency considers samples or time required to reach a target metric.

Part 3 - Hints and Helper

Benchmark several batch sizes after warm-up, synchronize CUDA when applicable, and report throughput, peak memory, step time, and loss progress under equal conditions.

Part 4 - Python code skeleton

```
import torch

def E07(model_factory, dataset, batch_sizes, device):
    results = {}
    for batch_size in batch_sizes:
        model = model_factory(...).to(device)
        warmup = ...
        elapsed = ...
        peak_memory = ...
        results[batch_size] = ...
    return results

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

5.6.9 Exercise 08 - Batch-related Bug Debugging

Part 1 - Why this matters

Batch bugs include mismatched feature-target indices, incorrect class axes, fixed-size assumptions, wrong epoch averaging, lost samples, unintended accumulation, and device mismatches.

Many errors occur only on the final batch or after the first update. Diagnostics should include batch index, actual size, shapes, devices, and processed sample identifiers.

Part 2 - Mathematical form

Useful invariants are:

$$X.shape[0] = y.shape[0],$$

$$\sum_k B_k = N$$

when no samples are dropped, and:

$$L_{epoch} = \frac{\sum_k B_k L_k}{\sum_k B_k}.$$

Part 3 - Hints and Helper

Create one faulty loader or loop and inspect every batch, including the remainder. Find the first violated invariant and correct its direct cause rather than forcing a reshape.

Part 4 - Python code skeleton

```
import torch

def E08(loader, expected_sample_count):
    report = []
    processed = ...
    for batch_index, (X_batch, targets) in enumerate(loader):
        checks = ...
        report.append(checks)
        processed += ...
    diagnosis = ...
    return report, processed, diagnosis

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

5.6.10 Exercise 09 - Full Epoch Training Challenge

Part 1 - Why this matters

The final challenge combines shuffled mini-batches, repeated epochs, correct gradient handling, sample-weighted loss aggregation, final-batch support, and reproducible logging.

A complete pipeline must explain how many samples and updates occurred in each epoch. It should stop safely on non-finite values and return model and training evidence.

Part 2 - Mathematical form

For epochs $e = 1, \ldots, E$:

$$L_e = \frac{\sum_k B_{e,k} L_{e,k}}{\sum_k B_{e,k}}.$$

Each ordinary mini-batch performs:

$$\theta \leftarrow \theta - \eta \nabla_\theta L_{e,k}.$$

Part 3 - Hints and Helper

Use a shuffled DataLoader, call zero-grad, forward, loss, backward, and step for each batch, aggregate epoch loss by sample count, and log update counts and stopping reason.

Part 4 - Python code skeleton

```
import torch

def E09(model, loader, loss_function, optimizer, epochs):
    history = []
    status = ...
    for epoch in range(epochs):
        total_loss = ...
        sample_count = ...
        update_count = ...
        for X_batch, targets in loader:
            ...
        history.append(...)
        if ...:
            break
    return model, history, status

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

5.7 LFTL05 - Evaluation & Validation Split

Mathematics and Theory

5.7.1 Exercise 00 - Train vs Eval Mode Difference

Part 1 - Why this matters

PyTorch modules can behave differently during training and evaluation. Dropout changes from random masking to deterministic scaling behavior, while batch normalization switches from batch statistics to stored running statistics.

Mode selection is separate from gradient tracking. Calling `eval()` does not disable Autograd, and entering a no-gradient context does not switch the module out of training mode.

Part 2 - Mathematical form

For a model f_θ:

$$\widehat{y}_{train} = f_\theta^{train}(x), \qquad \widehat{y}_{eval} = f_\theta^{eval}(x).$$

The functions may differ for stateful or stochastic layers even though parameters are unchanged.

Part 3 - Hints and Helper

Use a small model containing dropout or batch normalization. Compare repeated outputs in both modes, inspect the `training` flag, and separately test gradient recording.

Part 4 - Python code skeleton

```
import torch

def E00(model, X):
    model.train()
    training_outputs = ...
    model.eval()
    evaluation_outputs = ...
    return training_outputs, evaluation_outputs

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

5.7.2 Exercise 01 - Validation Dataset Separation

Part 1 - Why this matters

A validation set estimates behavior on data not used for parameter updates. It supports hyperparameter tuning, early stopping, and model selection without directly optimizing on evaluation examples.

Separation must occur before data-dependent preprocessing is fitted. Duplicate entities, future information, or related records crossing the boundary can create leakage and overly optimistic results.

Part 2 - Mathematical form

For dataset $\mathcal{D}$:

$$\mathcal{D}_{train} \cap \mathcal{D}_{val} = \varnothing,$$

and:

$$|\mathcal{D}_{train}| + |\mathcal{D}_{val}| = |\mathcal{D}|.$$

Additional grouping or temporal constraints may be required by the task.

Part 3 - Hints and Helper

Create reproducible split indices with a seeded generator. Verify disjointness, complete coverage, class or group requirements, and fit preprocessing only on the training subset.

Part 4 - Python code skeleton

```
import torch

def E01(features, targets, validation_fraction, seed):
    permutation = torch...
    split_point = ...
    train_indices = ...
    validation_indices = ...
    checks = ...
    return train_indices, validation_indices, checks

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

5.7.3 Exercise 02 - Validation Loss Computation

Part 1 - Why this matters

Validation loss measures the training objective on held-out data without changing parameters. It must use evaluation mode, disabled gradient recording, and the same loss definition used for comparison.

Epoch validation loss should be weighted by actual batch sizes. Averaging batch means equally biases the result when the final batch is smaller.

Part 2 - Mathematical form

For validation batches with size B_k and mean loss L_k:

$$L_{val} = \frac{\sum_k B_k L_k}{\sum_k B_k}.$$

No optimizer update is performed during this computation.

Part 3 - Hints and Helper

Switch to `eval()`, enter `torch.inference_mode()` or `no_grad()`, accumulate loss times batch size, and restore training mode when appropriate.

Part 4 - Python code skeleton

```
import torch

def E02(model, validation_loader, loss_function):
    model.eval()
    total_loss = ...
    sample_count = ...
    with torch.inference_mode():
        for X_batch, targets in validation_loader:
            predictions = ...
            loss = ...
            total_loss += ...
            sample_count += ...
    return total_loss / sample_count

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

5.7.4 Exercise 03 - Overfitting Detection

Part 1 - Why this matters

Overfitting occurs when training performance continues improving while validation performance stops improving or becomes worse. The model is fitting training-specific patterns that do not generalize.

Detection requires aligned histories over several epochs. A temporary noisy validation increase is not enough; a persistent generalization gap provides stronger evidence.

Part 2 - Mathematical form

Define the generalization gap:

$$G_e = L_{val,e} - L_{train,e}.$$

A common pattern is decreasing $L_{train,e}$ with increasing or stagnant $L_{val,e}$ and a widening positive G_e.

Part 3 - Hints and Helper

Compare smoothed or patience-based trends over a window. Return the suspected start epoch, gap history, and supporting trend statistics rather than only a Boolean.

Part 4 - Python code skeleton

```
import torch

def E03(training_losses, validation_losses, window, tolerance):
    gaps = ...
```

```
    training_trend = ...
    validation_trend = ...
    overfitting_start = ...
    return gaps, overfitting_start

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

5.7.5 Exercise 04 - Early Stopping Trigger

Part 1 - Why this matters

Early stopping ends training after validation performance fails to improve for a defined period. It limits wasted computation and can prevent continued movement away from the best generalizing state.

The trigger must preserve the best model, not merely stop at the current model. Minimum improvement and patience prevent small fluctuations from resetting the counter indefinitely.

Part 2 - Mathematical form

For validation loss V_e, improvement occurs when:

$$V_e < V_{best} - \delta.$$

Otherwise wait increases, and stopping is triggered when:

$$wait \geq patience.$$

Part 3 - Hints and Helper

Maintain best loss, best epoch, wait counter, and a deep copy of the best state dictionary. Test the trigger using a predefined validation sequence before integrating it into training.

Part 4 - Python code skeleton

```
import torch

def E04(model, validation_loss, state,
        min_improvement, patience):
    improved = ...
    if improved:
        state["best_loss"] = ...
        state["best_weights"] = ...
        state["wait"] = ...
    else:
        state["wait"] += ...
    stop = ...
```

```
    return stop, state

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

5.7.6 Exercise 05 - Metric vs Loss Comparison

Part 1 - Why this matters

Loss is a differentiable optimization objective, while a metric communicates task performance. They may respond differently because a metric can be discrete, thresholded, or aligned with a different practical priority.

A lower loss can improve confidence without changing accuracy, while accuracy can hide class imbalance. Training should optimize a suitable loss and report metrics chosen for the task.

Part 2 - Mathematical form

For classification accuracy:

$$A = \frac{1}{N} \sum_{i} \mathbf{1}[\arg\max_{c} z_{ic} = y_i].$$

Cross-entropy instead measures target log-probability and can change even when every predicted class remains the same.

Part 3 - Hints and Helper

Construct logits whose correct classes do not change while confidence changes. Compare cross-entropy and accuracy, then add a class-sensitive metric for imbalanced data.

Part 4 - Python code skeleton

```
import torch

def E05(logits_versions, targets):
    records = []
    for logits in logits_versions:
        loss = ...
        predictions = ...
        accuracy = ...
        records.append(...)
    return records

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

5.7.7 Exercise 06 - Evaluation without Gradient Tracking

Part 1 - Why this matters

Evaluation requires forward values and metrics but not backward graphs. Disabling gradient tracking reduces memory use and overhead while ensuring validation code cannot accidentally perform backpropagation.

No-gradient execution remains distinct from evaluation mode. Both should normally be used together, with gradient tracking restored automatically after leaving the context.

Part 2 - Mathematical form

During evaluation:

$$\widehat{y} = f_\theta(x)$$

is computed, but no derivative graph for $\nabla_\theta L$ is retained. Model parameters remain unchanged:

$$\theta_{after} = \theta_{before}.$$

Part 3 - Hints and Helper

Clone parameters, run evaluation under `torch.inference_mode()`, inspect output metadata, and verify parameter equality afterward. Confirm normal gradient tracking resumes outside the context.

Part 4 - Python code skeleton

```
import torch

def E06(model, loader):
    model.eval()
    before = ...
    outputs = []
    with torch.inference_mode():
        for X_batch, _ in loader:
            outputs.append(model(...))
    after = ...
    checks = ...
    return outputs, checks

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

5.7.8 Exercise 07 - Validation Loop Bug Debugging

Part 1 - Why this matters

Validation loops fail through training mode, enabled gradients, parameter updates, shuffled target misalignment, wrong loss aggregation, data leakage, or failure to evaluate all samples.

These errors may not raise exceptions and can produce believable metrics. A diagnostic loop should verify state, sample counts, parameter immutability, and reduction logic.

Part 2 - Mathematical form

Required invariants include:

$$\theta_{after} = \theta_{before}, \quad N_{processed} = N_{validation},$$

and:

$$L_{val} = \frac{\sum_k B_k L_k}{\sum_k B_k}.$$

Part 3 - Hints and Helper

Instrument mode, gradient-enabled state, input-target shapes, processed sample count, batch losses, and parameter checksums. Identify the first violated invariant.

Part 4 - Python code skeleton

```
import torch

def E07(model, validation_loader, loss_function):
    report = {}
    before = ...
    report["mode"] = ...
    report["gradient_enabled"] = ...
    ...
    after = ...
    report["parameters_unchanged"] = ...
    report["diagnosis"] = ...
    return report

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

5.7.9 Exercise 08 - Model Selection based on Validation

Part 1 - Why this matters

Model selection chooses the checkpoint that performs best on validation evidence rather than the final training state. This separates optimization progress from generalization quality.

Repeatedly choosing from validation results can itself overfit the validation set. A final test set should remain untouched until the selection and tuning process is complete.

Part 2 - Mathematical form

For checkpoints $\theta_1, \ldots, \theta_E$ and minimized validation metric V:

$$e^* = \arg\min_e V(\theta_e), \qquad \theta^* = \theta_{e^*}.$$

Ties require a documented rule such as earliest epoch or secondary metric.

Part 3 - Hints and Helper

Store detached validation records and cloned checkpoint states. Select according to a declared metric direction and restore the corresponding state without inspecting test data.

Part 4 - Python code skeleton

```
import torch

def E08(checkpoints, validation_records,
        metric_name, minimize=True):
    scores = ...
    best_index = ...
    best_checkpoint = ...
    return best_index, best_checkpoint

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

5.7.10 Exercise 09 - Full Train–Validation Pipeline

Part 1 - Why this matters

A full pipeline alternates parameter-updating training epochs with parameter-preserving validation epochs. It records both histories, manages modes and gradient contexts, selects checkpoints, and supports early stopping.

The final result must restore the best validation model and preserve reproducibility. Training and validation sample accounting should remain separate and explicit.

Part 2 - Mathematical form

For epoch e:

$$\theta_e = \text{TrainEpoch}(\theta_{e-1}, \mathcal{D}_{train}),$$
$$V_e = \text{Evaluate}(\theta_e, \mathcal{D}_{val}).$$

The selected model is θ_{e^*} where e^* optimizes the validation criterion.

Part 3 - Hints and Helper

Implement train and validation phases with correct modes, contexts, sample-weighted losses, and no validation updates. Save the best state and return histories, stopping reason, and selected epoch.

Part 4 - Python code skeleton

```
import torch

def E09(model, train_loader, validation_loader,
        loss_function, optimizer, epochs, early_stopping):
    history = []
    best_state = ...
    status = ...
    for epoch in range(epochs):
        model.train()
        train_result = ...
        model.eval()
        with torch.inference_mode():
            validation_result = ...
        ...
        history.append(...)
        if ...:
            break
    model.load_state_dict(...)
    return model, history, status

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

5.8 LFTL06 - Overfitting Prevention Methods

Mathematics and Theory

5.8.1 Exercise 00 - Detecting Overfitting Pattern

Part 1 - Why this matters

Overfitting occurs when a model reduces error on its training samples without improving its behavior on unseen data. A falling training loss is therefore not sufficient evidence of progress; it must be interpreted together with validation loss across multiple epochs.

A reliable detector looks for a persistent generalization gap rather than reacting to one noisy measurement. Recognizing this pattern early prevents wasted computation and provides evidence for choosing regularization, more data, a smaller model, or early stopping.

Part 2 - Mathematical form

Let the losses at epoch e be $L_{\text{train}}^{(e)}$ and $L_{\text{val}}^{(e)}$. Define

$$G^{(e)} = L_{\text{val}}^{(e)} - L_{\text{train}}^{(e)}.$$

An overfitting signal appears when training loss continues to decrease while validation loss or the persistent gap increases:

$$L_{\text{train}}^{(e)} < L_{\text{train}}^{(e-1)}, \qquad G^{(e)} > G^{(e-1)}.$$

Part 3 - Hints and Helper

Track both histories using the same reduction rule. Examine a window of epochs and require the pattern to persist; one validation fluctuation may only be noise.

Part 4 - Python code skeleton

```
import torch

def E00(train_losses, validation_losses, window):
    train_losses = torch...
    validation_losses = torch...
    gaps = ...
    recent_train_trend = ...
    recent_validation_trend = ...
    is_overfitting = ...
    return is_overfitting, gaps

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

5.8.2 Exercise 01 - Regularization Concept Experiment

Part 1 - Why this matters

Regularization changes the learning objective so that fitting observed samples is no longer the model's only goal. By penalizing unnecessary parameter complexity, it can discourage fragile solutions that memorize training-specific patterns.

A controlled experiment makes the tradeoff visible: weak regularization may leave the generalization gap unchanged, whereas excessive regularization can prevent the model from fitting useful structure. Comparing identical models and data isolates the effect of regularization strength.

Part 2 - Mathematical form

For parameters θ, define

$$J(\theta) = L_{\text{data}}(\theta) + \lambda\Omega(\theta),$$

where $\lambda \geq 0$ controls regularization. For L2 regularization,

$$\Omega(\theta) = \frac{1}{2}\|\theta\|_2^2.$$

Compare $\lambda = 0$ with positive values using both training and validation loss.

Part 3 - Hints and Helper

Keep initialization, split, optimizer, and epochs fixed across runs. Add the penalty to the training objective only, and record data loss and penalty separately.

Part 4 - Python code skeleton

```
import torch

def E01(model, prediction, target, strength):
    data_loss = torch...
    penalty = sum(... for parameter in model.parameters())
    objective = ...
    return objective, data_loss, penalty

def main():
    model = torch.nn...
    print(E01(...))

if __name__ == "__main__":
    main()
```

5.8.3 Exercise 02 - Weight Decay Integration

Part 1 - Why this matters

Weight decay shrinks parameters during optimization and can reduce reliance on excessively large weights. This often produces smoother, less sensitive models and is one of the most common regularization mechanisms in neural-network training.

Its placement matters because optimizers may treat weight decay as either an L2 term coupled to the gradient or a separate shrinkage step. Integrating it deliberately avoids accidental double regularization and makes comparisons meaningful.

Part 2 - Mathematical form

For basic gradient descent with learning rate η and decay λ,

$$\theta_{t+1} = (1 - \eta\lambda)\theta_t - \eta\nabla_\theta L(\theta_t).$$

For plain gradient descent this matches differentiating

$$J(\theta) = L(\theta) + \frac{\lambda}{2}\|\theta\|_2^2,$$

but the equivalence does not automatically extend to every adaptive optimizer.

Part 3 - Hints and Helper

Configure `weight_decay` once and inspect which parameter groups receive it. Biases and normalization parameters are often handled separately.

Part 4 - Python code skeleton

```
import torch

def E02(model, learning_rate, decay):
    decay_parameters = [...]
    no_decay_parameters = [...]
    optimizer = torch.optim...([
        {"params": decay_parameters, "weight_decay": ...},
        {"params": no_decay_parameters, "weight_decay": ...},
    ], lr=...)
    return optimizer

def main():
    model = torch.nn...
    print(E02(...))

if __name__ == "__main__":
    main()
```

5.8.4 Exercise 03 - Dropout Effect Observation

Part 1 - Why this matters

Dropout randomly removes a fraction of activations during training, preventing units from depending too strongly on particular neighboring features. The resulting stochastic subnetworks encourage representations that remain useful under different activation patterns.

The operation behaves differently during evaluation, so this exercise also tests correct model-mode control. Observing several dropout probabilities reveals when the method improves robustness and when it destroys too much information.

Part 2 - Mathematical form

For activation h_i and dropout probability p,

$$m_i \sim \text{Bernoulli}(1-p), \qquad \widetilde{h}_i = \frac{m_i}{1-p} h_i$$

during training. During evaluation, $\widetilde{h}_i = h_i$. Inverted scaling preserves $\mathbb{E}[\widetilde{h}_i] = h_i$.

Part 3 - Hints and Helper

Use `torch.nn.Dropout`. Compare repeated outputs in `model.train()` and `model.eval()` modes, and restore the previous mode if a helper changes it.

Part 4 - Python code skeleton

```
import torch

def E03(features, probability):
    layer = torch.nn.Dropout(p=...)
    layer.train()
```

```
    training_outputs = [layer(features) for _ in range(...)]
    layer.eval()
    evaluation_output = ...
    return training_outputs, evaluation_output

def main():
    features = torch...
    print(E03(...))

if __name__ == "__main__":
    main()
```

5.8.5 Exercise 04 - Data Augmentation Impact

Part 1 - Why this matters

Data augmentation expands the effective training distribution through transformations that preserve the intended target. It teaches the model that irrelevant changes should not alter the prediction.

Useful augmentation must reflect valid invariances of the problem. An unrealistic transformation can corrupt labels, while augmenting validation data makes evaluation unstable and hides true generalization behavior.

Part 2 - Mathematical form

Let $T \sim \mathcal{T}$ be a random label-preserving transformation. The augmented objective is

$$J(\theta) = \mathbb{E}_{(x,y)\sim\mathcal{D}_{\text{train}}} \mathbb{E}_{T\sim\mathcal{T}} \left[\ell(f_\theta(T(x)), y) \right].$$

Its effect is measured by comparing validation metrics for models trained with and without $\mathcal{T}$.

Part 3 - Hints and Helper

Apply transformations only in the training pipeline. Hold the split, initialization, optimizer, and budget constant; compare multiple runs when augmentation is stochastic.

Part 4 - Python code skeleton

```
import torch

def E04(batch, labels, augmentation):
    augmented_batch = augmentation(...)
    original_shape = batch.shape
    valid_shape = ...
    valid_labels = ...
    return augmented_batch, valid_shape, valid_labels

def main():
    batch = torch...
    labels = torch...
```

```
    augmentation = ...
    print(E04(...))

if __name__ == "__main__":
    main()
```

5.8.6 Exercise 05 - Early Stopping Implementation

Part 1 - Why this matters

Early stopping treats validation performance as evidence about when continued fitting stops improving generalization. It can reduce overfitting and training cost by ending optimization after the monitored metric has failed to improve for a defined period.

A correct implementation must preserve the best model, not merely the final model before termination. Patience and a minimum improvement threshold prevent small fluctuations from repeatedly resetting the stopping counter.

Part 2 - Mathematical form

For validation loss V_e, an improvement occurs when

$$V_e < V_{\text{best}} - \delta,$$

where $\delta \geq 0$ is the minimum change. Otherwise $c \leftarrow c + 1$. Training stops when $c \geq P$, where P is patience, and the best parameters are restored.

Part 3 - Hints and Helper

Store a deep copy of `state_dict()` whenever validation loss improves. Ensure the comparison direction matches the monitored metric.

Part 4 - Python code skeleton

```
import torch

def E05(validation_loss, model, state, patience, min_delta):
    improved = ...
    if improved:
        state["best_loss"] = ...
        state["best_model"] = ...
        state["counter"] = ...
    else:
        state["counter"] = ...
    should_stop = ...
    return should_stop, state

def main():
    state = {...}
    print(E05(...))

if __name__ == "__main__":
    main()
```

5.8.7 Exercise 06 - Bias–Variance Tradeoff Inspection

Part 1 - Why this matters

Prediction error can arise because a model is too restricted to represent the underlying relationship or because it changes too much when trained on different samples. These behaviors correspond to high bias and high variance, and they require different remedies.

Inspecting both components prevents blind changes to model capacity or regularization. Repeated training across resampled datasets reveals whether errors are systematic or caused by sensitivity to a particular training set.

Part 2 - Mathematical form

For squared error at input x,

$$\mathbb{E}_{\mathcal{D},\varepsilon}[(y - \widehat{f}_{\mathcal{D}}(x))^2] = \mathrm{Bias}(x)^2 + \mathrm{Var}(x) + \sigma_{\varepsilon}^2,$$

where

$$\mathrm{Bias}(x) = \mathbb{E}_{\mathcal{D}}[\widehat{f}_{\mathcal{D}}(x)] - f(x).$$

The terms can be estimated from models trained on multiple resampled datasets.

Part 3 - Hints and Helper

Stack predictions from independent runs along a run dimension. Estimate mean prediction, squared bias, and variance using a clearly stated variance convention.

Part 4 - Python code skeleton

```
import torch

def E06(run_predictions, reference):
    mean_prediction = torch...
    squared_bias = ...
    variance = torch...
    return squared_bias, variance

def main():
    run_predictions = torch...
    reference = torch...
    print(E06(...))

if __name__ == "__main__":
    main()
```

5.8.8 Exercise 07 - Underfitting vs Overfitting Diagnosis

Part 1 - Why this matters

Underfitting and overfitting can both produce poor validation results, but they represent opposite problems. Underfitting leaves substantial error on training data, whereas overfitting

fits training samples well but fails to transfer that performance.

Correct diagnosis determines the next intervention. Increasing capacity may help an underfit model but worsen an overfit one; stronger regularization may help overfitting but further restrict a model that already has high bias.

Part 2 - Mathematical form

Using final training loss L_T, validation loss L_V, and $G = L_V - L_T$,

$$\begin{cases} L_T \text{ high and } L_V \text{ high} & \text{underfitting,} \\ L_T \text{ low and } G \text{ large} & \text{overfitting,} \\ L_T \text{ low and } G \text{ small} & \text{balanced fit.} \end{cases}$$

The thresholds must be defined relative to the task or a baseline.

Part 3 - Hints and Helper

Accept explicit thresholds instead of embedding unexplained constants. Average several recent epochs and return the measured losses and gap with the diagnosis.

Part 4 - Python code skeleton

```
import torch

def E07(train_losses, validation_losses,
        loss_threshold, gap_threshold, window):
    train_level = torch...
    validation_level = torch...
    gap = ...
    if ...:
        diagnosis = ...
    elif ...:
        diagnosis = ...
    else:
        diagnosis = ...
    return diagnosis, train_level, validation_level, gap

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

5.8.9 Exercise 08 - Regularization Debugging Challenge

Part 1 - Why this matters

Regularization can appear ineffective because of implementation errors rather than a poor method. Common failures include dropout during validation, augmented validation samples, decay on the wrong parameters, or an L2 penalty combined accidentally with optimizer weight

decay.

Debugging should verify the entire data and optimization path, not only the coefficient. Mode changes, parameter groups, loss decomposition, and validation isolation provide evidence about where the intended behavior was lost.

Part 2 - Mathematical form

The intended training objective may be

$$J(\theta) = L_{\text{train}}(\theta) + \lambda\Omega(\theta),$$

while validation measures $V(\theta) = L_{\text{val}}(\theta)$ without training-only penalties or augmentation. A duplicated penalty produces

$$J_{\text{bug}}(\theta) = L_{\text{train}}(\theta) + 2\lambda\Omega(\theta).$$

Part 3 - Hints and Helper

Inspect optimizer groups, model mode, data transforms, and loss components. Detect duplicate parameters and confirm validation uses `model.eval()` with gradients disabled.

Part 4 - Python code skeleton

```
import torch

def E08(model, optimizer, train_transform, validation_transform):
    issues = []
    if model.training ...:
        issues.append(...)
    parameter_ids = [...]
    if ...:
        issues.append(...)
    if validation_transform ...:
        issues.append(...)
    return issues

def main():
    model = torch.nn...
    optimizer = torch.optim...
    print(E08(...))

if __name__ == "__main__":
    main()
```

5.8.10 Exercise 09 - Robust Training Pipeline Design

Part 1 - Why this matters

A robust training pipeline coordinates data separation, regularization, optimization, validation, checkpointing, and reproducibility as one system. Correct individual components can

still interact in ways that cause leakage, unstable metrics, or selection of the wrong model.

The final pipeline should produce evidence that reported performance belongs to unseen data. Training-only augmentation, correct model modes, best-state restoration, and an untouched test set protect that claim while making failures traceable.

Part 2 - Mathematical form

At epoch e, update only with training data:

$$\theta_e = \text{TrainEpoch}(\theta_{e-1}, \mathcal{D}_{\text{train}}).$$

Select the best epoch using validation performance:

$$e^* = \arg\min_e L_{\text{val}}(\theta_e; \mathcal{D}_{\text{val}}).$$

After design decisions are fixed, report

$$R_{\text{test}} = L_{\text{test}}(\theta_{e^*}; \mathcal{D}_{\text{test}}).$$

Part 3 - Hints and Helper

Separate train, validation, and test loaders. Use training augmentation only for training, log metrics, checkpoint the best state, support early stopping, and record seeds and configuration.

Part 4 - Python code skeleton

```
import torch

def E09(model, train_loader, validation_loader, loss_function,
        optimizer, epochs, early_stopper):
    history = []
    best_state = ...
    for epoch in range(epochs):
        model.train()
        train_result = ...
        model.eval()
        with torch.inference_mode():
            validation_result = ...
        history.append(...)
        improved, should_stop = ...
        if improved:
            best_state = ...
        if should_stop:
            break
    model.load_state_dict(...)
    return model, history

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

Book 6

SQL Lessons (PostgreSQL)

6.1 Introduction

6.1.1 What will this book teach you

Artificial intelligence systems do not operate on models alone. They also need a reliable place to store users, permissions, plans, limits, sessions, usage records, configuration values, and the results produced by applications. When this information has clear relationships and must remain consistent, a relational database becomes one of the most important components of the system. This book introduces PostgreSQL as the relational database that will support these responsibilities.

The purpose of this book is not to teach SQL as a collection of commands that must be memorized. You will learn how to think about structured data, how to decide which tables should exist, how those tables should be connected, and how PostgreSQL protects the correctness of the information stored inside them. Every query will represent a decision about the data. Before writing code, you should be able to explain what information you want, where it is stored, which rows are allowed to change, and how you will verify the result.

Python will be the primary programming language throughout the book. SQL statements will be executed from Python so that the database is always treated as part of an application rather than as an isolated tool. You will open connections, execute parameterized queries, retrieve results, handle errors, control transactions, and close resources correctly. Some exercises may show SQL directly to make the database operation clear, but the final implementation and verification will be performed through Python.

You must pay careful attention to safety. A missing condition in an `UPDATE` or `DELETE` statement can modify every row in a table. A poorly chosen data type can allow invalid values or waste storage. A missing constraint can create relationships that should never exist, while an incorrectly managed transaction can leave an application in a partially updated state. For this reason, the exercises require verification after important operations and encourage you to predict the result before executing the query.

This book follows the principle of learning through experimentation. You will create databases and tables, insert valid and invalid records, observe constraint failures, compare query results, test transaction rollback, examine locking behavior, and measure the effect of

indexes. Errors are an intentional part of the learning process. The objective is to understand what PostgreSQL is protecting, why an operation failed, and how the database behaves under different conditions.

The exercises will provide guidance and incomplete Python code skeletons, but they will not provide finished solutions. You are responsible for completing each connection, query, transaction, and validation step. Do not continue merely because a program runs without raising an exception. Inspect the returned rows, confirm the number of affected records, and verify the final database state. A successful command is not necessarily a correct result.

6.1.2 How PostgreSQL supports artificial intelligence systems

PostgreSQL does not train a neural network and does not replace PyTorch. Its role is to provide dependable structured storage around the model. An AI application may use PostgreSQL to store account information, subscription plans, request limits, authentication sessions, experiment metadata, model versions, job states, or usage history. These records require consistency, traceability, and relationships that should be enforced even when several parts of the application access the database simultaneously.

This distinction is important. Large binary files such as datasets and model weights may later be stored in object storage, flexible documents may be stored in a document database, and embeddings may be stored in a vector database. PostgreSQL is used when the information has a relational structure and the application needs strong rules governing that structure. Learning when to use it is as important as learning how to query it.

By the end of the book, you should be able to design a small relational system, connect to it securely from Python, perform safe create, read, update, and delete operations, enforce relationships and limits, use transactions to protect multi-step changes, and investigate query performance. These skills will become part of the data foundation upon which later artificial intelligence applications can be built.

6.1.3 How to approach the exercises

For every exercise, begin by identifying the expected database state before execution and the expected state afterward. Write down which table is affected, which rows should be returned or modified, and which condition prevents unintended changes. After running the Python program, verify these expectations with a separate query. This creates a disciplined cycle of prediction, execution, observation, and correction.

Use parameterized queries whenever values come from Python. Do not construct SQL statements by joining untrusted values into strings. Keep credentials outside the source code, preferably in environment variables, and never upload real passwords to a repository. Use context managers or explicit cleanup so connections and cursors are closed even when an operation fails. When an exercise uses a transaction, deliberately test both the successful path and the rollback path.

Do not rush through the early exercises. Later chapters assume that you understand data types, filtering, aggregation, keys, joins, and constraints. If a later transaction produces an

unexpected result, the true problem may be an incorrect relationship or query condition introduced earlier. The exercises are ordered so that each database behavior becomes a foundation for the next one.

6.1.4 The Chapters (SQL00–SQL03) included in this book

- SQL00 - Relational Data Fundamentals
- SQL01 - Tables, Keys & Schemas
- SQL02 - Users, Plans & Limits Storage
- SQL03 - Query Logic & Transactions

6.1.5 SQL00 - Relational Data Fundamentals

This chapter establishes the operational foundation of PostgreSQL. You will create a database, connect to it from Python, create tables with appropriate data types, insert rows, and verify stored information using `SELECT`. You will then filter, order, paginate, update, and delete records while learning how every condition changes the scope of an operation.

The chapter also introduces missing values, `NULL`, `COALESCE`, aggregate functions, grouping, and group-level filtering. These concepts transform individual rows into useful summaries. By the end of the chapter, you should be able to perform basic data operations safely and explain the difference between filtering rows before aggregation and filtering groups after aggregation.

6.1.6 SQL01 - Tables, Keys & Schemas

This chapter moves from operating on one table to designing a connected relational structure. You will model users, plans, and sessions; create primary keys and unique constraints; establish foreign-key relationships; and observe how different deletion behaviors protect or remove dependent records. Check constraints, default values, and timestamps will move validation closer to the data itself.

You will also normalize tables, create indexes, and combine information using inner and left joins. The goal is to understand that schema design determines which invalid states the database can prevent. By the end of the chapter, you should be able to justify table boundaries, relationships, constraints, and basic indexing decisions rather than creating them by habit.

6.1.7 SQL02 - Users, Plans & Limits Storage

This chapter applies relational design to a realistic application scenario. You will store subscription plans and their limits, assign users to plans, track token and request usage, calculate daily totals, and detect when a user exceeds an allowed limit. Views will be used to expose current limits and usage summaries through reusable queries.

The later exercises introduce transaction-based plan upgrades, audit records, and reports showing the users with the highest usage. This chapter connects database structure with application rules. It prepares you to represent operational policies in a way that can be queried, verified, and traced from Python.

6.1.8 SQL03 - Query Logic & Transactions

This chapter focuses on operations that must remain correct when several changes or users are involved. You will learn how `BEGIN`, `COMMIT`, and `ROLLBACK` define an atomic unit of work. Multi-table inserts, consistency checks, upserts, and transaction-safe usage deduction will demonstrate why related changes must either succeed together or fail together.

You will also simulate concurrent sessions, observe isolation and locking behavior, construct common table expressions, and inspect query execution using `EXPLAIN ANALYZE`. Finally, you will benchmark the effect of an index rather than assuming that every index improves performance. By the end of the chapter, you should be able to reason about correctness, concurrency, and performance as connected properties of a database operation.

6.2 SQL00 - Relational Data Fundamentals

Data Engineering for AI

6.2.1 Exercise 00 - Create Database + Connect Challenge

Part 1 - Why this matters

Every PostgreSQL application begins with a valid database and a dependable connection. The connection is the boundary through which Python sends SQL statements, receives results, and learns whether an operation succeeded or failed.

This first challenge establishes habits that will be used throughout the book: keep credentials outside the source code, connect to an existing maintenance database when creating a new database, verify the connection explicitly, and close resources even when an error occurs.

Part 2 - Technical concept and behavior

A PostgreSQL server can contain several databases. A client connects using parameters such as host, port, database name, user, and password. Because `CREATE DATABASE` cannot run inside a normal transaction block, the creation connection must use autocommit or an equivalent approach. After creation, open a separate connection to the new database and execute a small verification query.

Part 3 - Hints and Helper

Use `psycopg.connect()` and read configuration from environment variables. Connect first to a maintenance database such as `postgres`. Handle the case where the requested database already exists without hiding unrelated errors.

Part 4 - Python code skeleton

```
import os
import psycopg

def E00(database_name):
    admin_connection = psycopg.connect(
```

```
        host=os.getenv(...),
        port=os.getenv(...),
        dbname=...,
        user=os.getenv(...),
        password=os.getenv(...),
        autocommit=...,
    )
    with admin_connection.cursor() as cursor:
        cursor.execute(...)
    admin_connection.close()

    connection = psycopg.connect(...)
    with connection.cursor() as cursor:
        cursor.execute(...)
        verification = cursor.fetchone()
    connection.close()
    return verification

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

6.2.2 Exercise 01 - Create Table with Correct Data Types

Part 1 - Why this matters

A table definition determines which values PostgreSQL can store and how those values behave in later queries. Choosing a type only because it accepts the current example can create invalid states, inaccurate calculations, or unnecessary conversions as the application grows.

In this challenge, the reader translates a small data requirement into a schema. The goal is to distinguish identifiers, text, numeric values, Boolean states, and timestamps, then verify that PostgreSQL created the intended columns rather than assuming the statement was correct.

Part 2 - Technical concept and behavior

PostgreSQL validates inserted values according to each column's data type. Common choices include `INTEGER` or `BIGINT` for whole numbers, `TEXT` for variable-length text, `BOOLEAN` for logical states, `NUMERIC` for exact decimal quantities, and `TIMESTAMPTZ` for timestamps representing real instants. Table definitions may also specify nullability and default behavior.

Part 3 - Hints and Helper

Create a table representing a simple application entity. Use `CREATE TABLE IF NOT EXISTS`, then inspect `information_schema.columns` from Python to verify column names, types, and nullability.

Part 4 - Python code skeleton

```
import psycopg

def E01(connection):
    create_query = """
        CREATE TABLE IF NOT EXISTS ... (
            id ...,
            name ...,
            usage_amount ...,
            is_active ...,
            created_at ...
        )
    """
    with connection.cursor() as cursor:
        cursor.execute(...)
        cursor.execute("""
            SELECT column_name, data_type, is_nullable
            FROM information_schema.columns
            WHERE table_name = %s
            ORDER BY ordinal_position
        """, (...,))
        columns = cursor.fetchall()
    connection.commit()
    return columns

def main():
    connection = psycopg.connect(...)
    print(E01(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.2.3 Exercise 02 - Insert Rows + Verify with SELECT

Part 1 - Why this matters

Inserting data is useful only when the application can confirm what PostgreSQL stored. Input values may be transformed by defaults, generated identifiers, type conversion, or database rules, so verification should inspect the database result rather than repeat the original Python values.

This challenge also introduces parameterized queries. Separating SQL structure from data values prevents quoting mistakes and protects the application from SQL injection when values originate outside the program.

Part 2 - Technical concept and behavior

An `INSERT` statement creates one or more rows. PostgreSQL can return generated or normalized values through a `RETURNING` clause. A later `SELECT` verifies persistence after the transaction

is committed. Psycopg uses `%s` placeholders for values regardless of the underlying PostgreSQL type.

Part 3 - Hints and Helper

Never add values to an SQL string with concatenation or f-strings. Insert at least two rows with parameters, commit the transaction, and select them using a stable column in the `ORDER BY` clause.

Part 4 - Python code skeleton

```
import psycopg

def E02(connection, rows):
    inserted = []
    with connection.cursor() as cursor:
        for row in rows:
            cursor.execute(
                "INSERT INTO ... (...) VALUES (%s, %s, %s) RETURNING ...",
                (...),
            )
            inserted.append(cursor.fetchone())
    connection.commit()

    with connection.cursor() as cursor:
        cursor.execute("SELECT ... FROM ... ORDER BY ...")
        verified = cursor.fetchall()
    return inserted, verified

def main():
    connection = psycopg.connect(...)
    print(E02(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.2.4 Exercise 03 - Basic WHERE Filtering Challenge

Part 1 - Why this matters

Applications rarely need every row from a table. Filtering allows PostgreSQL to return only the records that satisfy a precise requirement, reducing transferred data and preventing Python from reproducing work that the database can perform directly.

A correct filter must reflect the intended logic. Confusing `AND` with `OR`, mishandling parentheses, or using unsafe string construction can return the wrong population while the program still appears to work.

Part 2 - Technical concept and behavior

The `WHERE` clause evaluates a Boolean condition for each candidate row. Comparisons, ranges, membership tests, and logical operators may be combined. Operator precedence affects compound expressions, so parentheses should make the intended grouping explicit whenever conditions mix `AND` and `OR`.

Part 3 - Hints and Helper

Accept filter values as Python arguments and pass them as query parameters. Return selected columns rather than using `SELECT *`, and test a condition that returns rows as well as one that returns an empty result.

Part 4 - Python code skeleton

```
import psycopg

def E03(connection, minimum_usage, active_state):
    query = """
        SELECT ...
        FROM ...
        WHERE ... >= %s
          AND ... = %s
        ORDER BY ...
    """
    with connection.cursor() as cursor:
        cursor.execute(query, (..., ...))
        rows = cursor.fetchall()
    return rows

def main():
    connection = psycopg.connect(...)
    print(E03(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.2.5 Exercise 04 - ORDER BY + LIMIT + OFFSET Challenge

Part 1 - Why this matters

Ordering and pagination allow an application to present large result sets in manageable portions. Without an explicit order, PostgreSQL does not guarantee which rows appear first, so repeated requests can produce unstable pages even when the underlying data has not changed.

This challenge teaches the reader to define deterministic ordering and calculate page boundaries in Python. It also reveals why an offset is a position in an ordered result rather than an identifier of the last row returned.

Part 2 - Technical concept and behavior

`ORDER BY` sorts the result using one or more expressions and directions. `LIMIT` restricts the number of returned rows, while `OFFSET` skips a number of rows after ordering. For one-based page number p and page size s,

$$\text{offset} = (p - 1)s.$$

A unique tie-breaking column makes the order deterministic.

Part 3 - Hints and Helper

Validate that page number and page size are positive. Order by a meaningful value and then by a unique identifier. Pass limit and offset as parameters rather than inserting them into the query string.

Part 4 - Python code skeleton

```
import psycopg

def E04(connection, page_number, page_size):
    if ...:
        raise ValueError(...)
    offset = ...
    query = """
        SELECT ...
        FROM ...
        ORDER BY ... DESC, ... ASC
        LIMIT %s OFFSET %s
    """
    with connection.cursor() as cursor:
        cursor.execute(query, (..., ...))
        page = cursor.fetchall()
    return page

def main():
    connection = psycopg.connect(...)
    print(E04(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.2.6 Exercise 05 - UPDATE with Condition + Verification

Part 1 - Why this matters

An update changes existing information and can therefore damage many records if its scope is wrong. The most important part of an `UPDATE` statement is often not the new value but the condition that identifies exactly which rows are allowed to change.

Verification makes the modification observable. The program should know how many rows were affected and inspect their final values, allowing it to reject cases where no row or an unexpected number of rows matched the condition.

Part 2 - Technical concept and behavior

An `UPDATE` statement assigns new values to rows satisfying its `WHERE` clause. PostgreSQL's `RETURNING` clause exposes the changed rows without requiring a separate lookup. Psycopg also provides `cursor.rowcount`, but returned identifiers and values offer stronger evidence about what changed.

Part 3 - Hints and Helper

Require a unique identifier or another deliberate condition. Run the update inside a transaction, inspect the returned row before committing, and roll back when the number of matches violates the exercise's expectation.

Part 4 - Python code skeleton

```
import psycopg

def E05(connection, record_id, new_value):
    try:
        with connection.cursor() as cursor:
            cursor.execute("""
                UPDATE ...
                SET ... = %s
                WHERE ... = %s
                RETURNING ...
            """, (..., ...))
            changed = cursor.fetchall()
            if len(changed) != ...:
                raise ValueError(...)
        connection.commit()
    except Exception:
        connection.rollback()
        raise
    return changed

def main():
    connection = psycopg.connect(...)
    print(E05(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.2.7 Exercise 06 - DELETE with Safety Condition

Part 1 - Why this matters

Deletion is irreversible after a transaction is committed, making an imprecise `DELETE` one of the most dangerous database operations. A safety condition must be specific enough to protect unrelated rows and should be checked before the final change is accepted.

This challenge treats deletion as a controlled procedure rather than a single command. The reader will preview the target, delete it inside a transaction, verify the affected row, and decide whether the transaction is safe to commit.

Part 2 - Technical concept and behavior

`DELETE FROM table WHERE condition` removes every row for which the condition is true. Without `WHERE`, all rows are eligible. `RETURNING` can report deleted records, while a transaction allows Python to roll back when the affected count differs from the intended count.

Part 3 - Hints and Helper

Do not accept an empty identifier. Select the target first, then delete using the same specific condition. Require exactly one returned row for this exercise and roll back otherwise.

Part 4 - Python code skeleton

```
import psycopg

def E06(connection, record_id):
    if ...:
        raise ValueError(...)
    try:
        with connection.cursor() as cursor:
            cursor.execute("SELECT ... FROM ... WHERE ... = %s", (...,))
            preview = cursor.fetchone()
            cursor.execute(
                "DELETE FROM ... WHERE ... = %s RETURNING ...",
                (...,),
            )
            deleted = cursor.fetchall()
            if ...:
                raise ValueError(...)
        connection.commit()
    except Exception:
        connection.rollback()
        raise
    return preview, deleted

def main():
    connection = psycopg.connect(...)
    print(E06(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.2.8 Exercise 07 - NULL Handling + COALESCE Challenge

Part 1 - Why this matters

A missing database value is not automatically zero, an empty string, or `False`. PostgreSQL represents absence with `NULL`, and treating it as an ordinary value can produce filters and calculations that behave differently from Python expectations.

The reader must learn when to preserve missingness and when to replace it for a particular output. `COALESCE` is useful for presentation and calculations, but replacing every missing value can hide an important distinction in the original data.

Part 2 - Technical concept and behavior

Comparisons with `NULL` produce an unknown logical result, so SQL uses `IS NULL` and `IS NOT NULL`. The expression `COALESCE(a, b, ...)` returns the first non-null argument. Psycopg maps SQL `NULL` to Python `None` when rows are retrieved.

Part 3 - Hints and Helper

Insert or identify rows containing a nullable value. Return both the original column and a derived `COALESCE` value so their meanings remain visible. Compare `IS NULL` with an intentionally incorrect equality test.

Part 4 - Python code skeleton

```
import psycopg

def E07(connection, fallback_value):
    query = """
        SELECT
            ...,
            ... AS original_value,
            COALESCE(..., %s) AS resolved_value,
            ... IS NULL AS was_missing
        FROM ...
        ORDER BY ...
    """
    with connection.cursor() as cursor:
        cursor.execute(query, (...,))
        rows = cursor.fetchall()
    return rows

def main():
    connection = psycopg.connect(...)
    print(E07(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.2.9 Exercise 08 - Aggregate Basics (COUNT, SUM, AVG)

Part 1 - Why this matters

Applications frequently need summaries rather than individual records. Aggregate functions allow PostgreSQL to calculate counts, totals, and averages close to the stored data, avoiding unnecessary transfer of every row into Python.

Correct interpretation is essential because aggregates do not all treat `NULL` identically. The reader will compare record count with non-null value count and verify that a summary describes the intended population.

Part 2 - Technical concept and behavior

`COUNT(*)` counts input rows, whereas `COUNT(column)` counts only non-null values in that column. `SUM(column)` and `AVG(column)` ignore null inputs and return `NULL` when no non-null values are available. A `WHERE` clause restricts rows before aggregation.

Part 3 - Hints and Helper

Calculate all three aggregates in one query. Include `COUNT(*)` and `COUNT(nullable_column)` to expose their difference, and test both a populated filter and a filter matching no rows.

Part 4 - Python code skeleton

```
import psycopg

def E08(connection, active_state):
    query = """
        SELECT
            COUNT(*) AS row_count,
            COUNT(...) AS value_count,
            SUM(...) AS total_value,
            AVG(...) AS average_value
        FROM ...
        WHERE ... = %s
    """
    with connection.cursor() as cursor:
        cursor.execute(query, (...,))
        summary = cursor.fetchone()
    return summary

def main():
    connection = psycopg.connect(...)
    print(E08(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.2.10 Exercise 09 - Group By + Having Challenge

Part 1 - Why this matters

Grouping transforms rows into category-level summaries, allowing an application to compare plans, statuses, dates, or other dimensions. It is the foundation of usage reports, monitoring dashboards, and many data checks surrounding an artificial intelligence service.

The challenge is to separate row-level logic from group-level logic. Filtering individual rows changes which records contribute to a summary, while filtering after aggregation decides which completed summaries are shown.

Part 2 - Technical concept and behavior

`GROUP BY` creates one group for each distinct combination of grouping expressions. Selected columns must normally be grouped or aggregated. `WHERE` filters rows before groups are formed, whereas `HAVING` filters groups after aggregate values have been calculated.

Part 3 - Hints and Helper

Group records by a categorical column and calculate count, sum, and average. Use a parameterized `WHERE` condition for eligible rows and a separate parameterized `HAVING` condition for the minimum group total. Order the final groups explicitly.

Part 4 - Python code skeleton

```
import psycopg

def E09(connection, active_state, minimum_total):
    query = """
        SELECT
            ... AS category,
            COUNT(*) AS record_count,
            SUM(...) AS total_value,
            AVG(...) AS average_value
        FROM ...
        WHERE ... = %s
        GROUP BY ...
        HAVING SUM(...) >= %s
        ORDER BY total_value DESC, category ASC
    """
    with connection.cursor() as cursor:
        cursor.execute(query, (..., ...))
        groups = cursor.fetchall()
    return groups

def main():
    connection = psycopg.connect(...)
    print(E09(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.3 SQL01 - Tables, Keys & Schemas

Data Engineering for AI

6.3.1 Exercise 00 - Design Schema: users + plans + sessions

Part 1 - Why this matters

A relational schema is a model of the application's rules. Separating users, plans, and sessions prevents repeated information and gives each table a clear responsibility, while relationships describe which records are allowed to depend on others.

Good design begins before `CREATE TABLE`. The reader must identify entities, attributes, identifiers, optional fields, and relationship cardinalities, then translate those decisions into PostgreSQL and inspect the resulting schema from Python.

Part 2 - Technical concept and behavior

In this scenario, one plan may belong to many users and one user may own many sessions. Each table needs a primary key, while dependent tables store foreign keys referencing their parent records. Creation order matters because a referenced table must exist before a foreign-key constraint can target it.

Part 3 - Hints and Helper

Write the design in plain language first. Create tables in dependency order: plans, users, then sessions. Use a transaction so an incomplete schema is rolled back, and query PostgreSQL metadata to verify all three tables.

Part 4 - Python code skeleton

```
import psycopg

def E00(connection):
    statements = [
        "CREATE TABLE ...",
        "CREATE TABLE ...",
        "CREATE TABLE ...",
    ]
    try:
        with connection.cursor() as cursor:
            for statement in statements:
                cursor.execute(...)
            cursor.execute("""
                SELECT table_name
                FROM information_schema.tables
                WHERE table_schema = %s
                  AND table_name = ANY(%s)
                ORDER BY table_name
            """, (..., ...))
            tables = cursor.fetchall()
```

```
        connection.commit()
    except Exception:
        connection.rollback()
        raise
    return tables

def main():
    connection = psycopg.connect(...)
    print(E00(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.3.2 Exercise 01 - Primary Key + Unique Constraint Challenge

Part 1 - Why this matters

A primary key gives every row a stable identity, allowing other records and application operations to refer to it without ambiguity. A unique constraint protects business identifiers such as usernames or email addresses from duplication.

Although both constraints enforce uniqueness, they express different meanings. This challenge requires the reader to create both, attempt deliberate duplicates, observe PostgreSQL's response, and confirm that failed inserts do not corrupt the transaction state.

Part 2 - Technical concept and behavior

A table has one primary-key constraint, which implies uniqueness and non-nullability and may contain one or several columns. A table may have multiple unique constraints. PostgreSQL reports a constraint violation when an insert or update would duplicate a protected value.

Part 3 - Hints and Helper

Insert one valid row, then separately test a duplicate primary key and a duplicate unique value. Roll back after each expected failure before using the connection again. Record the constraint name or SQLSTATE for verification.

Part 4 - Python code skeleton

```
import psycopg

def E01(connection, valid_row, duplicate_rows):
    results = []
    with connection.cursor() as cursor:
        cursor.execute("INSERT INTO ... VALUES (...) RETURNING ...", (...))
    connection.commit()

    for row in duplicate_rows:
        try:
            with connection.cursor() as cursor:
```

```
                cursor.execute("INSERT INTO ... VALUES (...)", (...))
            connection.commit()
            results.append(...)
        except psycopg.errors.UniqueViolation as error:
            connection.rollback()
            results.append(...)
    return results

def main():
    connection = psycopg.connect(...)
    print(E01(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.3.3 Exercise 02 - Foreign Key Relationship Setup

Part 1 - Why this matters

Foreign keys prevent dependent records from referring to parents that do not exist. Without this protection, a user could reference an imaginary plan or a session could remain connected to no valid user, leaving application data internally inconsistent.

This exercise makes referential integrity observable. The reader will create a relationship, insert valid parent and child rows, deliberately attempt an invalid reference, and verify the database state after rollback.

Part 2 - Technical concept and behavior

A foreign key constrains values in child columns to matching values in a referenced primary key or unique key. PostgreSQL checks the relationship on inserts and updates. The child value may still be `NULL` unless its column is also declared `NOT NULL`.

Part 3 - Hints and Helper

Create the parent row before the child row. Test both a valid and nonexistent parent identifier, catch `ForeignKeyViolation`, roll back the failed transaction, and count remaining child records.

Part 4 - Python code skeleton

```
import psycopg

def E02(connection, parent_id, invalid_parent_id):
    with connection.cursor() as cursor:
        cursor.execute("INSERT INTO ... RETURNING ...", (...,))
        cursor.execute("INSERT INTO ... VALUES (..., %s) RETURNING ...",
                       (parent_id,))
        valid_child = cursor.fetchone()
    connection.commit()
```

```
    try:
        with connection.cursor() as cursor:
            cursor.execute("INSERT INTO ... VALUES (..., %s)",
                           (invalid_parent_id,))
        connection.commit()
    except psycopg.errors.ForeignKeyViolation as error:
        connection.rollback()
        failure = ...
    return valid_child, failure

def main():
    connection = psycopg.connect(...)
    print(E02(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.3.4 Exercise 03 - ON DELETE Behavior (RESTRICT/CASCADE)

Part 1 - Why this matters

Deleting a parent record raises an important design question: should PostgreSQL reject the deletion or automatically remove dependent records? The answer represents an application policy, not merely a convenient SQL option.

This challenge compares restrictive and cascading relationships experimentally. The reader will create isolated test records, attempt both deletion paths, and inspect which rows survive so the consequences are understood before these rules are used with real data.

Part 2 - Technical concept and behavior

`ON DELETE RESTRICT` prevents parent deletion while referenced child rows exist. `ON DELETE CASCADE` deletes matching child rows automatically with the parent. The default behavior is commonly `NO ACTION`, whose timing can differ from `RESTRICT` for deferrable checks.

Part 3 - Hints and Helper

Use separate parent records for the two experiments. Catch and roll back the restricted deletion, then execute the cascading deletion and verify parent and child counts before committing conclusions.

Part 4 - Python code skeleton

```
import psycopg

def E03(connection, restricted_parent, cascading_parent):
    observations = {}
    try:
        with connection.cursor() as cursor:
```

```
            cursor.execute("DELETE FROM ... WHERE id = %s",
                           (restricted_parent,))
        connection.commit()
    except psycopg.errors.ForeignKeyViolation:
        connection.rollback()
        observations["restrict"] = ...

    with connection.cursor() as cursor:
        cursor.execute("DELETE FROM ... WHERE id = %s RETURNING id",
                       (cascading_parent,))
        observations["cascade_parent"] = cursor.fetchone()
        cursor.execute("SELECT COUNT(*) FROM ... WHERE ... = %s",
                       (cascading_parent,))
        observations["cascade_children"] = cursor.fetchone()
    connection.commit()
    return observations

def main():
    connection = psycopg.connect(...)
    print(E03(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.3.5 Exercise 04 - CHECK Constraints for Data Validation

Part 1 - Why this matters

Application validation can be bypassed by another script, service, or manual database operation. A check constraint places essential rules beside the stored data so every writer must obey the same condition.

The challenge is to encode a rule that is both strict and semantically correct. The reader will test boundary values and missing values, because a condition that seems obvious may still allow `NULL` unless nullability is constrained separately.

Part 2 - Technical concept and behavior

A `CHECK` constraint accepts rows when its Boolean expression is true or unknown and rejects rows when it is false. Therefore, `CHECK (limit >= 0)` does not replace `NOT NULL`. Named constraints make database errors easier to interpret and maintain.

Part 3 - Hints and Helper

Define valid ranges for plan limits or session values. Test the lower boundary, upper boundary, a value outside each boundary, and `None`. Roll back each expected failure and report its constraint name.

Part 4 - Python code skeleton

```
import psycopg

def E04(connection, test_values):
    outcomes = []
    for value in test_values:
        try:
            with connection.cursor() as cursor:
                cursor.execute(
                    "INSERT INTO ... (...) VALUES (%s) RETURNING ...",
                    (value,),
                )
                result = cursor.fetchone()
            connection.commit()
            outcomes.append((value, ...))
        except psycopg.errors.CheckViolation as error:
            connection.rollback()
            outcomes.append((value, ...))
    return outcomes

def main():
    connection = psycopg.connect(...)
    print(E04(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.3.6 Exercise 05 - Default Values + Timestamps

Part 1 - Why this matters

Defaults let PostgreSQL supply predictable values when an insert intentionally omits a column. They reduce repeated application code and ensure records created by different clients begin with the same state.

Timestamps require particular care because database time, application time, and local display time may differ. This exercise uses PostgreSQL to generate creation times and verifies the difference between omitting a value, explicitly providing one, and explicitly inserting `NULL`.

Part 2 - Technical concept and behavior

A `DEFAULT` expression is evaluated when an insert omits the column or uses the `DEFAULT` keyword. It is not applied when the client explicitly supplies `NULL`. `TIMESTAMPTZ` represents an instant and displays it according to the session time zone.

Part 3 - Hints and Helper

Add a Boolean default and a timestamp default such as `CURRENT_TIMESTAMP`. Insert rows using different column lists, return generated fields, and compare their Python types and values.

Part 4 - Python code skeleton

```
import psycopg

def E05(connection, name):
    with connection.cursor() as cursor:
        cursor.execute("""
            INSERT INTO ... (...)
            VALUES (%s)
            RETURNING ..., ..., ...
        """, (name,))
        generated = cursor.fetchone()
        cursor.execute("""
            INSERT INTO ... (..., ..., ...)
            VALUES (%s, %s, %s)
            RETURNING ..., ..., ...
        """, (..., ..., ...))
        explicit = cursor.fetchone()
    connection.commit()
    return generated, explicit

def main():
    connection = psycopg.connect(...)
    print(E05(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.3.7 Exercise 06 - Normalize: Split Table into 2NF/3NF

Part 1 - Why this matters

Repeated plan details inside every user row create update anomalies: changing a plan may require many rows to be modified, and inconsistent copies can describe the same plan differently. Normalization separates facts according to what they describe.

This exercise begins with a deliberately mixed table and transforms it into related tables. The reader must preserve data during migration and prove that a join can reconstruct the useful combined view without storing duplicate facts.

Part 2 - Technical concept and behavior

Second Normal Form removes partial dependencies on part of a composite key, while Third Normal Form removes non-key attributes that depend transitively on another non-key attribute. In this scenario, plan limits belong to a plan entity rather than directly to each user entity.

Part 3 - Hints and Helper

Identify functional dependencies before writing SQL. Create normalized tables, insert distinct plan records, migrate users using a lookup or join, compare counts, and keep the migration inside one transaction.

Part 4 - Python code skeleton

```
import psycopg

def E06(connection):
    try:
        with connection.cursor() as cursor:
            cursor.execute("CREATE TABLE ...")
            cursor.execute("CREATE TABLE ...")
            cursor.execute("""
                INSERT INTO ... (...)
                SELECT DISTINCT ...
                FROM ...
            """)
            cursor.execute("""
                INSERT INTO ... (...)
                SELECT ..., ...
                FROM ...
                JOIN ... ON ...
            """)
            cursor.execute("SELECT ...")
            verification = cursor.fetchall()
        connection.commit()
    except Exception:
        connection.rollback()
        raise
    return verification

def main():
    connection = psycopg.connect(...)
    print(E06(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.3.8 Exercise 07 - Create Index + Explain Its Purpose

Part 1 - Why this matters

An index can help PostgreSQL locate rows without scanning an entire table, but it also consumes storage and adds work to inserts, updates, and deletes. Creating an index is therefore a measured design choice, not a universal performance switch.

This challenge connects an index to a real query pattern. The reader will identify columns used for filtering or ordering, create an appropriate index from Python, and inspect PostgreSQL's plan rather than assuming the index must be used.

Part 2 - Technical concept and behavior

A B-tree index supports many equality, range, and ordering operations. Column order matters in a multi-column index. PostgreSQL's planner chooses between index and sequential scans based on table statistics and estimated cost; a small table may correctly use a sequential scan.

Part 3 - Hints and Helper

Write the target query before creating the index. Use `CREATE INDEX IF NOT EXISTS`, run `ANALYZE`, then execute `EXPLAIN` for the parameterized query and return the plan lines.

Part 4 - Python code skeleton

```
import psycopg

def E07(connection, filter_value):
    with connection.cursor() as cursor:
        cursor.execute("""
            CREATE INDEX IF NOT EXISTS ...
            ON ... (..., ...)
        """)
        cursor.execute("ANALYZE ...")
        cursor.execute("""
            EXPLAIN
            SELECT ...
            FROM ...
            WHERE ... = %s
            ORDER BY ...
        """, (filter_value,))
        plan = cursor.fetchall()
    connection.commit()
    return plan

def main():
    connection = psycopg.connect(...)
    print(E07(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.3.9 Exercise 08 - Join Basics (INNER / LEFT) Challenge

Part 1 - Why this matters

Normalized tables store related facts separately, so joins are required to reconstruct information for an application. The chosen join type controls whether records without a matching relationship disappear or remain visible.

This exercise compares the same data using inner and left joins. The difference becomes clear when a parent record has no matching child, teaching the reader to choose a join according to the question rather than by habit.

Part 2 - Technical concept and behavior

An `INNER JOIN` returns only combinations satisfying its join condition. A `LEFT JOIN` preserves every row from the left input and fills right-side columns with `NULL` when no match exists. A one-to-many relationship can produce several result rows for one parent.

Part 3 - Hints and Helper

Create at least one parent with children and one without children. Run inner and left versions of the query, select explicit columns with aliases, and compare both result counts and identities.

Part 4 - Python code skeleton

```
import psycopg

def E08(connection):
    inner_query = """
        SELECT ...
        FROM ... AS parent
        INNER JOIN ... AS child ON ...
        ORDER BY ...
    """
    left_query = """
        SELECT ...
        FROM ... AS parent
        LEFT JOIN ... AS child ON ...
        ORDER BY ...
    """
    with connection.cursor() as cursor:
        cursor.execute(inner_query)
        inner_rows = cursor.fetchall()
        cursor.execute(left_query)
        left_rows = cursor.fetchall()
    return inner_rows, left_rows

def main():
    connection = psycopg.connect(...)
    print(E08(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.3.10 Exercise 09 - Query with Multi-Table Join + Filters

Part 1 - Why this matters

Real application queries often combine several relationships while applying filters from different tables. A user-session report, for example, may require user identity, plan information, session state, and a time boundary in one result.

Complexity grows when filters interact with outer joins. Placing a child-table condition in `WHERE` can remove unmatched parents and silently change a left join into inner-join behavior, so query structure must reflect the intended result population.

Part 2 - Technical concept and behavior

A multi-table join forms a relational path through foreign keys. Conditions in an `ON` clause control which rows match during that join, whereas `WHERE` filters the combined rows afterward. Table aliases and qualified column names prevent ambiguity and make the relationship path visible.

Part 3 - Hints and Helper

Join users to plans and sessions. Parameterize plan state and time filters, decide whether users without matching sessions should remain, use explicit aliases, and order by a stable combination of columns.

Part 4 - Python code skeleton

```
import psycopg

def E09(connection, plan_name, session_start, include_without_sessions):
    join_type = ...
    query = f"""
        SELECT
            u.... AS user_id,
            u.... AS user_name,
            p.... AS plan_name,
            s.... AS session_time
        FROM ... AS u
        JOIN ... AS p ON ...
        {join_type} JOIN ... AS s
            ON ... AND s.... >= %s
        WHERE p.... = %s
        ORDER BY ...
    """
    with connection.cursor() as cursor:
        cursor.execute(query, (..., ...))
        rows = cursor.fetchall()
    return rows

def main():
    connection = psycopg.connect(...)
    print(E09(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.4 SQL02 - Users, Plans & Limits Storage

Data Engineering for AI

6.4.1 Exercise 00 - Implement Plans Table + Limits Columns

Part 1 - Why this matters

An AI service often offers plans that control requests, tokens, storage, or other resources. Storing these limits in PostgreSQL gives the application one consistent source of truth instead of scattering policy values throughout Python code.

The schema must define what every value means. Zero, a positive number, and an unlimited allowance are different states, so the reader must choose an explicit representation and protect invalid negative values with database constraints.

Part 2 - Technical concept and behavior

The plans table requires a stable identifier, unique plan name, limit columns with sufficient numeric range, an active state, and timestamps. If `NULL` represents unlimited usage, queries must handle it deliberately because comparisons with `NULL` do not return true or false in the ordinary way.

Part 3 - Hints and Helper

Use `BIGINT` for potentially large counters and named `CHECK` constraints permitting non-negative values or `NULL`. Create the table from Python, insert several boundary cases, and inspect the stored rows.

Part 4 - Python code skeleton

```
import psycopg

def E00(connection):
    create_query = """
        CREATE TABLE IF NOT EXISTS plans (
            id ...,
            name ...,
            request_limit ...,
            token_limit ...,
            is_active ...,
            created_at ...,
            CONSTRAINT ... CHECK (...),
            CONSTRAINT ... CHECK (...)
        )
    """
    with connection.cursor() as cursor:
        cursor.execute(create_query)
        cursor.execute("INSERT INTO plans (...) VALUES (...) RETURNING ...",
                       (...))
        result = cursor.fetchone()
```

```
    connection.commit()
    return result

def main():
    connection = psycopg.connect(...)
    print(E00(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.4.2 Exercise 01 - Assign User to Plan + Enforce FK

Part 1 - Why this matters

Assigning a plan to a user connects account identity with service policy. A foreign key ensures that an application cannot assign a plan identifier that does not exist, even if validation is missing from one Python code path.

The relationship also requires a policy for deleted plans and unassigned users. This exercise makes those choices explicit and verifies both valid assignment and deliberate referential failure.

Part 2 - Technical concept and behavior

A `plan_id` column in `users` references `plans(id)`. `NOT NULL` requires every user to have a plan, while a nullable column permits an unassigned state. An `ON DELETE` action determines whether a referenced plan can be removed.

Part 3 - Hints and Helper

Choose the nullability and deletion behavior before altering the table. Insert or update one valid assignment, attempt an invalid plan identifier, roll back the expected failure, and join the tables to verify the surviving relationship.

Part 4 - Python code skeleton

```
import psycopg

def E01(connection, user_id, plan_id, invalid_plan_id):
    with connection.cursor() as cursor:
        cursor.execute("""
            UPDATE users
            SET plan_id = %s
            WHERE id = %s
            RETURNING id, plan_id
        """, (..., ...))
        assigned = cursor.fetchone()
    connection.commit()

    try:
        with connection.cursor() as cursor:
```

```
            cursor.execute("UPDATE users SET plan_id = %s WHERE id = %s",
                           (..., ...))
        connection.commit()
    except psycopg.errors.ForeignKeyViolation as error:
        connection.rollback()
        failure = ...
    return assigned, failure

def main():
    connection = psycopg.connect(...)
    print(E01(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.4.3 Exercise 02 - Track Usage Table (tokens/requests)

Part 1 - Why this matters

Limits cannot be enforced without trustworthy usage records. Recording each usage event creates evidence for billing, monitoring, debugging, and later aggregation while preserving when and by whom a resource was consumed.

The table must reject impossible values and maintain user relationships. The reader will distinguish an immutable event record from a mutable total, then verify inserted events from Python.

Part 2 - Technical concept and behavior

A usage event can store a user foreign key, request count, token count, event timestamp, and optional request identifier for idempotency. Non-negative checks protect counters, and a unique external request identifier can prevent the same event from being recorded twice.

Part 3 - Hints and Helper

Prefer append-only events for this exercise. Add constraints for counts, choose a timestamp type, and use parameterized inserts with `RETURNING`. Test valid, negative, and duplicate event cases with rollback.

Part 4 - Python code skeleton

```
import psycopg

def E02(connection, user_id, request_id, requests, tokens):
    try:
        with connection.cursor() as cursor:
            cursor.execute("""
                INSERT INTO usage_events
                    (user_id, request_id, request_count, token_count, used_at)
                VALUES (%s, %s, %s, %s, ...)
```

```
                RETURNING ...
            """, (..., ..., ..., ...))
            event = cursor.fetchone()
        connection.commit()
    except Exception:
        connection.rollback()
        raise
    return event

def main():
    connection = psycopg.connect(...)
    print(E02(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.4.4 Exercise 03 - Daily Usage Aggregation Query

Part 1 - Why this matters

Raw events explain individual activity, but limit enforcement and reporting often require totals for each user and day. PostgreSQL can group these events efficiently without transferring the entire event history into Python.

The meaning of a day depends on a time zone. A timestamp near midnight may belong to different calendar dates in UTC and the UAE, so the query must state which business time zone defines the aggregation boundary.

Part 2 - Technical concept and behavior

Daily aggregation converts each event timestamp into a business-local date, groups by user and date, and applies `SUM` to request and token counts. A half-open interval $[t_{start}, t_{end})$ avoids overlapping boundary events between consecutive reporting periods.

Part 3 - Hints and Helper

Pass the reporting interval and time-zone name as parameters. Group by both user and derived date, select explicit totals, and order results deterministically. Verify the query with events around midnight.

Part 4 - Python code skeleton

```
import psycopg

def E03(connection, start_time, end_time, timezone_name):
    query = """
        SELECT
            user_id,
            (... AT TIME ZONE %s)::date AS usage_date,
            SUM(request_count) AS total_requests,
```

```
            SUM(token_count) AS total_tokens
        FROM usage_events
        WHERE used_at >= %s AND used_at < %s
        GROUP BY user_id, usage_date
        ORDER BY usage_date, user_id
    """
    with connection.cursor() as cursor:
        cursor.execute(query, (..., ..., ...))
        rows = cursor.fetchall()
    return rows

def main():
    connection = psycopg.connect(...)
    print(E03(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.4.5 Exercise 04 - Check Limit Breach Query (Over Limit)

Part 1 - Why this matters

A service must compare accumulated usage with the limits belonging to each user's current plan. Performing this comparison in one relational query reduces inconsistent decisions between separate Python lookups.

Boundary semantics matter: reaching a limit and exceeding a limit are different policies. Unlimited values also require explicit handling, so the reader must define the rule before writing the Boolean expression.

Part 2 - Technical concept and behavior

The query joins users with plans and a usage summary. `COALESCE` can convert missing usage to zero, but a null plan limit may represent unlimited usage and should not be converted blindly. A breach condition may use $>$ for over-limit detection or $\geq$ when the next request must be blocked at the limit.

Part 3 - Hints and Helper

Return usage, limits, and separate Boolean flags for requests and tokens. Test below-limit, exactly-at-limit, above-limit, no-usage, and unlimited cases so the policy is observable.

Part 4 - Python code skeleton

```
import psycopg

def E04(connection, user_id, usage_date):
    query = """
        SELECT
            u.id,
```

```
            p.request_limit,
            p.token_limit,
            COALESCE(SUM(e.request_count), 0) AS used_requests,
            COALESCE(SUM(e.token_count), 0) AS used_tokens,
            (...) AS request_limit_breached,
            (...) AS token_limit_breached
        FROM users AS u
        JOIN plans AS p ON ...
        LEFT JOIN usage_events AS e ON ...
        WHERE u.id = %s
        GROUP BY ...
    """
    with connection.cursor() as cursor:
        cursor.execute(query, (..., ...))
        status = cursor.fetchone()
    return status

def main():
    connection = psycopg.connect(...)
    print(E04(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.4.6 Exercise 05 - Create View: user_current_limits

Part 1 - Why this matters

Applications repeatedly need the current limits attached to each user. A view can give this relationship a stable name and expose a consistent set of columns without duplicating plan data into the users table.

A view is still a query, not an independent stored copy in this exercise. Changes to users or plans become visible through it, allowing the reader to centralize read logic while preserving normalized storage.

Part 2 - Technical concept and behavior

`CREATE VIEW` stores a query definition that PostgreSQL evaluates when the view is queried. The view can join users and plans, rename columns, and expose only fields required by consumers. `CREATE OR REPLACE VIEW` updates a compatible definition without dropping it first.

Part 3 - Hints and Helper

Expose user identity, plan identity, active state, and both limits with unambiguous aliases. Create the view from Python, query it for one user, then modify a plan and verify that the view reflects the change.

Part 4 - Python code skeleton

```
import psycopg

def E05(connection, user_id):
    view_query = """
        CREATE OR REPLACE VIEW user_current_limits AS
        SELECT
            u.... AS user_id,
            u.... AS user_name,
            p.... AS plan_id,
            p.... AS plan_name,
            p.... AS request_limit,
            p.... AS token_limit
        FROM users AS u
        JOIN plans AS p ON ...
    """
    with connection.cursor() as cursor:
        cursor.execute(view_query)
        cursor.execute(
            "SELECT ... FROM user_current_limits WHERE user_id = %s",
            (user_id,),
        )
        row = cursor.fetchone()
    connection.commit()
    return row

def main():
    connection = psycopg.connect(...)
    print(E05(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.4.7 Exercise 06 - Create View: user_usage_summary

Part 1 - Why this matters

Usage summaries are needed by dashboards, limit checks, and operational reports. Repeating the same aggregation in several Python functions increases the chance that date boundaries or null handling will differ between consumers.

A view can provide a shared relational definition of the summary. The reader must still understand its cost, because an ordinary view does not automatically precompute or cache aggregate results.

Part 2 - Technical concept and behavior

The view groups usage events by user and a defined date representation, then returns request totals, token totals, and event count. An ordinary view executes its underlying query when read. A materialized view behaves differently and is not introduced as a replacement here.

Part 3 - Hints and Helper

Choose and document the view's time-zone convention. Use clear aliases and preserve one row per user per date. Query a date range from Python and compare one row with a direct aggregation.

Part 4 - Python code skeleton

```
import psycopg

def E06(connection, start_date, end_date):
    create_view = """
        CREATE OR REPLACE VIEW user_usage_summary AS
        SELECT
            user_id,
            (...)::date AS usage_date,
            COUNT(*) AS event_count,
            SUM(request_count) AS total_requests,
            SUM(token_count) AS total_tokens
        FROM usage_events
        GROUP BY user_id, usage_date
    """
    with connection.cursor() as cursor:
        cursor.execute(create_view)
        cursor.execute("""
            SELECT ...
            FROM user_usage_summary
            WHERE usage_date >= %s AND usage_date < %s
            ORDER BY usage_date, user_id
        """, (..., ...))
        rows = cursor.fetchall()
    connection.commit()
    return rows

def main():
    connection = psycopg.connect(...)
    print(E06(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.4.8 Exercise 07 - Plan Upgrade Transaction Challenge

Part 1 - Why this matters

A plan upgrade may require several related changes, such as validating the target plan, changing the user's assignment, and recording when the change occurred. If only part of this work succeeds, account state and history can disagree.

Transactions make the upgrade atomic, while row locking protects the decision from concurrent modifications. The reader will test successful commit and deliberate rollback paths instead of assuming the multi-step operation is safe.

Part 2 - Technical concept and behavior

The transaction selects the target user and plan, validates their states, updates the assignment, and commits only when every step succeeds. `SELECT ... FOR UPDATE` can lock the selected user row until transaction completion, preventing another transaction from changing it concurrently.

Part 3 - Hints and Helper

Do not commit inside intermediate steps. Lock the user, reject a missing or inactive target plan, update with `RETURNING`, and roll back on every exception. Verify the old plan remains after a forced failure.

Part 4 - Python code skeleton

```
import psycopg

def E07(connection, user_id, target_plan_id):
    try:
        with connection.cursor() as cursor:
            cursor.execute("""
                SELECT id, plan_id
                FROM users
                WHERE id = %s
                FOR UPDATE
            """, (user_id,))
            user = cursor.fetchone()
            cursor.execute("SELECT id, is_active FROM plans WHERE id = %s",
                           (target_plan_id,))
            plan = cursor.fetchone()
            if ...:
                raise ValueError(...)
            cursor.execute("""
                UPDATE users SET plan_id = %s
                WHERE id = %s
                RETURNING id, plan_id
            """, (..., ...))
            upgraded = cursor.fetchone()
        connection.commit()
    except Exception:
        connection.rollback()
        raise
    return user, upgraded

def main():
    connection = psycopg.connect(...)
    print(E07(...))
```

```
    connection.close()

if __name__ == "__main__":
    main()
```

6.4.9 Exercise 08 - Audit Log Table for Plan Changes

Part 1 - Why this matters

Current state answers which plan a user has now, but it does not explain how that state was reached. An audit log preserves plan-change history for support, investigation, policy review, and operational accountability.

The log must be written in the same transaction as the assignment change. Otherwise, a user could be upgraded without history or an audit record could describe a change that was rolled back.

Part 2 - Technical concept and behavior

An audit row can store user identity, previous and new plan identifiers, change timestamp, actor, and reason. It is generally append-only. The previous plan may be nullable for an initial assignment, while foreign-key deletion policies should preserve the desired historical meaning.

Part 3 - Hints and Helper

Create the audit table with named constraints and timestamps. Extend the upgrade transaction to capture the old plan, update the user, and insert the log before one final commit. Force an audit insert failure and verify that the user update rolls back too.

Part 4 - Python code skeleton

```
import psycopg

def E08(connection, user_id, new_plan_id, actor, reason):
    try:
        with connection.cursor() as cursor:
            cursor.execute("""
                SELECT plan_id FROM users
                WHERE id = %s FOR UPDATE
            """, (user_id,))
            old_plan_id = ...
            cursor.execute("UPDATE users SET plan_id = %s WHERE id = %s",
                           (..., ...))
            cursor.execute("""
                INSERT INTO plan_change_audit
                    (user_id, old_plan_id, new_plan_id, actor, reason)
                VALUES (%s, %s, %s, %s, %s)
                RETURNING ...
            """, (..., ..., ..., ..., ...))
            audit_row = cursor.fetchone()
        connection.commit()
```

```
    except Exception:
        connection.rollback()
        raise
    return audit_row

def main():
    connection = psycopg.connect(...)
    print(E08(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.4.10 Exercise 09 - Report Query: Top Users by Usage

Part 1 - Why this matters

Usage rankings help an AI service identify heavy workloads, plan adoption, unusual activity, and capacity requirements. A useful report must define its time range and ranking measure so that readers do not mistake an ambiguous total for a meaningful comparison.

This final challenge combines users, plans, and usage events. It requires safe filters, correct aggregation, deterministic ordering, and a limited result set returned to Python with enough context to interpret each ranking.

Part 2 - Technical concept and behavior

The query filters events using a half-open timestamp interval, joins account context, groups by user and plan, and orders by an aggregate such as total tokens. Secondary ordering resolves ties. A left join can retain users with zero usage, while an inner join naturally reports only users with matching events.

Part 3 - Hints and Helper

Parameterize start time, end time, and result limit. Return request total, token total, event count, and plan name. Decide whether inactive or zero-usage users belong in the report and encode that decision explicitly.

Part 4 - Python code skeleton

```
import psycopg

def E09(connection, start_time, end_time, top_n):
    if ...:
        raise ValueError(...)
    query = """
        SELECT
            u.id AS user_id,
            u.... AS user_name,
            p.... AS plan_name,
            COUNT(e.id) AS event_count,
```

```
            COALESCE(SUM(e.request_count), 0) AS total_requests,
            COALESCE(SUM(e.token_count), 0) AS total_tokens
        FROM users AS u
        JOIN plans AS p ON ...
        JOIN usage_events AS e ON ...
        WHERE e.used_at >= %s AND e.used_at < %s
        GROUP BY ...
        ORDER BY total_tokens DESC, user_id ASC
        LIMIT %s
    """
    with connection.cursor() as cursor:
        cursor.execute(query, (..., ..., ...))
        report = cursor.fetchall()
    return report

def main():
    connection = psycopg.connect(...)
    print(E09(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.5 SQL03 - Query Logic & Transactions

Data Engineering for AI

6.5.1 Exercise 00 - Transaction Basics (BEGIN/COMMIT/ROLLBACK)

Part 1 - Why this matters

A transaction defines a boundary around database work. Changes inside that boundary remain provisional until commit and can be discarded together with rollback, allowing an application to recover from errors without preserving a partial operation.

This exercise makes transaction state visible rather than treating commit as a routine final line. The reader will perform equivalent changes on commit and rollback paths, then verify their different outcomes using a separate observation query.

Part 2 - Technical concept and behavior

`BEGIN` starts a transaction, `COMMIT` makes its changes durable, and `ROLLBACK` abandons them. Psycopg normally begins a transaction automatically when the first command requires one. After a database error, the transaction remains failed until rollback.

Part 3 - Hints and Helper

Use explicit commit and rollback from Python. Insert a uniquely identifiable row in each path, verify visibility after completion, and demonstrate that the connection can be reused only after a failed transaction is rolled back.

Part 4 - Python code skeleton

```
import psycopg

def E00(connection, committed_value, rolled_back_value):
    with connection.cursor() as cursor:
        cursor.execute("INSERT INTO ... (...) VALUES (%s) RETURNING ...",
                       (committed_value,))
        committed_candidate = cursor.fetchone()
    connection.commit()

    with connection.cursor() as cursor:
        cursor.execute("INSERT INTO ... (...) VALUES (%s) RETURNING ...",
                       (rolled_back_value,))
        rolled_back_candidate = cursor.fetchone()
    connection.rollback()

    with connection.cursor() as cursor:
        cursor.execute("SELECT ... FROM ... WHERE ... = ANY(%s)", (...,))
        verification = cursor.fetchall()
    return committed_candidate, rolled_back_candidate, verification

def main():
    connection = psycopg.connect(...)
    print(E00(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.5.2 Exercise 01 - Atomic Insert into Multiple Tables

Part 1 - Why this matters

One application action may create records in several tables. Creating a user and its initial session separately can leave an incomplete account when the second insert fails after the first has already committed.

Atomicity requires all related inserts to succeed or none to survive. The reader will pass a generated identifier from the first insert to the second, force a failure, and verify that PostgreSQL removes the earlier provisional work.

Part 2 - Technical concept and behavior

Statements executed through one connection before commit belong to the same transaction. `INSERT ... RETURNING` provides a generated key for dependent inserts. Any exception should lead to one rollback covering every statement in the unit of work.

Part 3 - Hints and Helper

Insert the parent first, obtain its identifier, and insert the child without committing between them. Test a successful path and a path where the child violates a constraint, then query both tables after rollback.

Part 4 - Python code skeleton

```
import psycopg

def E01(connection, user_data, session_data):
    try:
        with connection.cursor() as cursor:
            cursor.execute("""
                INSERT INTO users (...) VALUES (...)
                RETURNING id
            """, (...))
            user_id = cursor.fetchone()[0]
            cursor.execute("""
                INSERT INTO sessions (user_id, ...)
                VALUES (%s, ...)
                RETURNING id
            """, (user_id, ...))
            session_id = cursor.fetchone()[0]
        connection.commit()
    except Exception:
        connection.rollback()
        raise
    return user_id, session_id

def main():
    connection = psycopg.connect(...)
    print(E01(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.5.3 Exercise 02 - Consistency Check + Rollback on Failure

Part 1 - Why this matters

Some business rules require reading several values before deciding whether a change is valid. Database constraints should enforce rules they can express, while transaction-level checks protect conditions that depend on current related state.

This challenge adds a deliberate consistency gate before commit. The reader will update provisional data, calculate an invariant, raise an error when it is violated, and prove that rollback restores the original state.

Part 2 - Technical concept and behavior

An invariant is a condition that must hold when the transaction commits. The application may query the provisional state because its own uncommitted changes are visible within the same transaction. If the check fails, rollback discards every preceding modification.

Part 3 - Hints and Helper

Choose an invariant involving limits and usage or related row counts. Lock records when concurrent changes could invalidate the check, perform the update, calculate the invariant, and commit only when it remains true.

Part 4 - Python code skeleton

```
import psycopg

def E02(connection, user_id, proposed_change):
    try:
        with connection.cursor() as cursor:
            cursor.execute("SELECT ... FROM ... WHERE ... = %s FOR UPDATE",
                           (user_id,))
            before = cursor.fetchone()
            cursor.execute("UPDATE ... SET ... WHERE ... = %s RETURNING ...",
                           (..., user_id))
            after = cursor.fetchone()
            cursor.execute("SELECT ...")
            is_consistent = ...
            if not is_consistent:
                raise ValueError(...)
        connection.commit()
    except Exception:
        connection.rollback()
        raise
    return before, after

def main():
    connection = psycopg.connect(...)
    print(E02(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.5.4 Exercise 03 - Isolation Concept Simulation (Two Sessions)

Part 1 - Why this matters

Concurrent requests can read and change the same data through different database sessions. Isolation determines which effects of one transaction another is allowed to observe before and during its own transaction.

The concept becomes clearer through two real connections. The reader will coordinate a writer and reader, observe visibility before and after commit, and repeat the experiment at selected isolation levels without confusing Python connection state with PostgreSQL transaction state.

Part 2 - Technical concept and behavior

At `READ COMMITTED`, each statement sees data committed before that statement begins. At `REPEATABLE READ`, a transaction generally keeps one consistent snapshot. Uncommitted changes from another transaction are not visible at these levels.

Part 3 - Hints and Helper

Open two independent connections and label every observation. Set the isolation level before the transaction begins, use deterministic checkpoints rather than arbitrary sleeps, and always roll back or commit both sessions in a `finally` block.

Part 4 - Python code skeleton

```
import psycopg

def E03(connection_a, connection_b, record_id):
    observations = []
    try:
        with connection_a.cursor() as cursor_a, \
             connection_b.cursor() as cursor_b:
            cursor_b.execute("SET TRANSACTION ISOLATION LEVEL ...")
            cursor_b.execute("SELECT ... WHERE id = %s", (record_id,))
            observations.append(("before", cursor_b.fetchone()))

            cursor_a.execute("UPDATE ... SET ... WHERE id = %s",
                             (record_id,))
            cursor_b.execute("SELECT ... WHERE id = %s", (record_id,))
            observations.append(("uncommitted", cursor_b.fetchone()))
            connection_a.commit()

            cursor_b.execute("SELECT ... WHERE id = %s", (record_id,))
            observations.append(("after_commit", cursor_b.fetchone()))
        connection_b.commit()
    finally:
        connection_a.rollback()
        connection_b.rollback()
    return observations

def main():
    first = psycopg.connect(...)
    second = psycopg.connect(...)
    print(E03(first, second, ...))
    first.close()
    second.close()
```

```
if __name__ == "__main__":
    main()
```

6.5.5 Exercise 04 - Locking Behavior Observation Challenge

Part 1 - Why this matters

Locks protect records while a transaction makes a decision and applies a change. They also cause competing operations to wait, which can become a performance problem or deadlock when lock order is poorly designed.

This exercise observes blocking safely with two connections. A short lock timeout converts indefinite waiting into a controlled error, allowing the reader to inspect behavior and release every lock predictably.

Part 2 - Technical concept and behavior

`SELECT ... FOR UPDATE` acquires row-level locks that conflict with other updates and locking reads on the same rows. Locks normally remain until commit or rollback. `lock_timeout` limits how long a statement waits to acquire a lock; it does not limit total transaction duration.

Part 3 - Hints and Helper

Lock one known row in connection A. Set a local lock timeout in connection B and attempt a conflicting update. Catch the timeout, roll back B, release A, and retry to demonstrate that the same update can then succeed.

Part 4 - Python code skeleton

```
import psycopg

def E04(connection_a, connection_b, record_id):
    observations = []
    try:
        with connection_a.cursor() as cursor_a:
            cursor_a.execute("SELECT ... WHERE id = %s FOR UPDATE",
                             (record_id,))
        try:
            with connection_b.cursor() as cursor_b:
                cursor_b.execute("SET LOCAL lock_timeout = %s", (...,))
                cursor_b.execute("UPDATE ... WHERE id = %s", (record_id,))
            connection_b.commit()
        except psycopg.errors.LockNotAvailable as error:
            connection_b.rollback()
            observations.append(...)
        connection_a.commit()
        with connection_b.cursor() as cursor_b:
            cursor_b.execute("UPDATE ... WHERE id = %s RETURNING ...",
                             (record_id,))
            observations.append(cursor_b.fetchone())
        connection_b.commit()
```

```
    finally:
        connection_a.rollback()
        connection_b.rollback()
    return observations

def main():
    first = psycopg.connect(...)
    second = psycopg.connect(...)
    print(E04(first, second, ...))
    first.close()
    second.close()

if __name__ == "__main__":
    main()
```

6.5.6 Exercise 05 - Upsert Pattern (INSERT...ON CONFLICT)

Part 1 - Why this matters

Applications often receive data that should create a row the first time and update it on later arrivals. Performing a separate check followed by insert or update creates a race window in which another transaction can change the answer.

PostgreSQL's upsert pattern lets one statement resolve a uniqueness conflict atomically. The reader will identify the correct conflict key and ensure that only intended columns change during the update path.

Part 2 - Technical concept and behavior

`INSERT ... ON CONFLICT` uses a unique constraint or index to identify a conflict. `DO NOTHING` skips the conflicting insert, while `DO UPDATE` can reference proposed values through `EXCLUDED`. `RETURNING` reports the resulting row.

Part 3 - Hints and Helper

Use a unique request identifier or other natural idempotency key. Execute the same key twice with changed values, verify that only one row exists, and return the stored values after each execution.

Part 4 - Python code skeleton

```
import psycopg

def E05(connection, external_key, value):
    query = """
        INSERT INTO ... (..., ...)
        VALUES (%s, %s)
        ON CONFLICT (...)
        DO UPDATE SET
            ... = EXCLUDED....,
            ... = ...
```

```
        RETURNING ...
    """
    try:
        with connection.cursor() as cursor:
            cursor.execute(query, (..., ...))
            row = cursor.fetchone()
        connection.commit()
    except Exception:
        connection.rollback()
        raise
    return row

def main():
    connection = psycopg.connect(...)
    print(E05(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.5.7 Exercise 06 - Common Table Expression (CTE) Challenge

Part 1 - Why this matters

Complex reports become difficult to reason about when filtering, aggregation, ranking, and final selection are compressed into one deeply nested query. A Common Table Expression can name an intermediate result and expose the logical stages of the computation.

This challenge uses a CTE to make query intent clearer, not merely to replace a subquery with different syntax. The reader will verify each stage independently before combining it into the final Python-executed statement.

Part 2 - Technical concept and behavior

A non-recursive CTE begins with `WITH name AS (query)` and can be referenced by the following statement. PostgreSQL may inline eligible CTEs, so a CTE is not automatically a materialized temporary result or a performance improvement.

Part 3 - Hints and Helper

Build a CTE that aggregates usage by user, then join or filter it in the outer query. Parameterize the date range and threshold, select explicit columns, and compare its results with a simpler reference query.

Part 4 - Python code skeleton

```
import psycopg

def E06(connection, start_time, end_time, minimum_usage):
    query = """
        WITH usage_totals AS (
```

```
            SELECT
                user_id,
                SUM(...) AS total_usage
            FROM ...
            WHERE ... >= %s AND ... < %s
            GROUP BY user_id
        )
        SELECT
            u....,
            totals.total_usage
        FROM usage_totals AS totals
        JOIN users AS u ON ...
        WHERE totals.total_usage >= %s
        ORDER BY totals.total_usage DESC, u.id
    """
    with connection.cursor() as cursor:
        cursor.execute(query, (..., ..., ...))
        rows = cursor.fetchall()
    return rows

def main():
    connection = psycopg.connect(...)
    print(E06(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.5.8 Exercise 07 - Query Performance: EXPLAIN ANALYZE

Part 1 - Why this matters

Query speed cannot be understood reliably from SQL appearance alone. PostgreSQL's planner chooses scans, joins, and aggregation strategies using statistics and estimated costs, while actual execution reveals where time and rows were spent.

The reader will compare estimates with actual behavior and look for large discrepancies. Performance analysis must remain controlled because `EXPLAIN ANALYZE` actually executes the statement being examined.

Part 2 - Technical concept and behavior

`EXPLAIN` returns the planned execution tree. Adding `ANALYZE` runs the query and adds actual timing and row counts. Options such as `BUFFERS` report cache and I/O activity. Costs are planner units, not milliseconds.

Part 3 - Hints and Helper

Use `EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)` on a safe `SELECT`. Run `ANALYZE` on the table first, extract the plan from Python, and compare estimated rows with actual rows at selected nodes.

Part 4 - Python code skeleton

```
import psycopg

def E07(connection, filter_value):
    with connection.cursor() as cursor:
        cursor.execute("ANALYZE ...")
        cursor.execute("""
            EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)
            SELECT ...
            FROM ...
            WHERE ... = %s
            ORDER BY ...
        """, (filter_value,))
        raw_plan = cursor.fetchone()[0]
    plan = ...
    execution_time = ...
    estimated_rows = ...
    actual_rows = ...
    return plan, execution_time, estimated_rows, actual_rows

def main():
    connection = psycopg.connect(...)
    print(E07(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.5.9 Exercise 08 - Index Impact Benchmark Challenge

Part 1 - Why this matters

An index should be justified by a workload, not by the belief that indexes always make databases faster. It may improve a selective read while increasing storage and write cost, and PostgreSQL may correctly ignore it for queries returning much of a table.

This challenge creates a controlled before-and-after experiment. The reader will keep data and query constant, collect multiple measurements, inspect plan changes, and avoid treating one warm-cache timing as a reliable conclusion.

Part 2 - Technical concept and behavior

The benchmark compares execution plans and repeated timings before and after creating a candidate index. Selectivity, table size, statistics, cache state, and index column order influence the result. Dropping the index afterward restores the experimental environment.

Part 3 - Hints and Helper

Use a dedicated test table with enough rows to make planning meaningful. Run warm-up and measured trials, call `ANALYZE`, summarize several execution times, and compare read benefit with an observed or discussed write tradeoff.

Part 4 - Python code skeleton

```
import psycopg

def measure(connection, filter_value, repetitions):
    measurements = []
    for _ in range(repetitions):
        with connection.cursor() as cursor:
            cursor.execute("""
                EXPLAIN (ANALYZE, FORMAT JSON)
                SELECT ... FROM ... WHERE ... = %s
            """, (filter_value,))
            plan = cursor.fetchone()[0]
            measurements.append(...)
    return measurements

def E08(connection, filter_value, repetitions):
    before = measure(...)
    with connection.cursor() as cursor:
        cursor.execute("CREATE INDEX ... ON ... (...)")
        cursor.execute("ANALYZE ...")
    connection.commit()
    after = measure(...)
    return before, after

def main():
    connection = psycopg.connect(...)
    print(E08(...))
    connection.close()

if __name__ == "__main__":
    main()
```

6.5.10 Exercise 09 - Transaction-Safe Usage Deduction Scenario

Part 1 - Why this matters

When concurrent requests consume a limited allowance, a read followed later by an update can allow both requests to approve themselves from the same old balance. The final counter may then exceed the plan even though each Python process believed its check was valid.

The final challenge combines conditional SQL, transaction control, row locking or atomic update logic, and auditability. A rejected deduction must change nothing, while an accepted deduction must update all related records exactly once.

Part 2 - Technical concept and behavior

A safe pattern performs the limit condition and counter update atomically, for example with an `UPDATE` whose `WHERE` clause verifies remaining capacity and whose `RETURNING` clause reports success. Another valid pattern locks the relevant row before checking and updating. Related usage events belong in the same transaction.

Part 3 - Hints and Helper

Reject non-positive deductions before SQL. Make one statement decide whether capacity remains, treat no returned row as denial, insert the usage event only after approval, and test two competing connections against the last available capacity.

Part 4 - Python code skeleton

```
import psycopg

def E09(connection, user_id, request_id, requested_tokens):
    if ...:
        raise ValueError(...)
    try:
        with connection.cursor() as cursor:
            cursor.execute("""
                UPDATE ...
                SET used_tokens = used_tokens + %s
                WHERE user_id = %s
                  AND used_tokens + %s <= token_limit
                RETURNING used_tokens, token_limit
            """, (..., ..., ...))
            balance = cursor.fetchone()
            if balance is None:
                connection.rollback()
                return ...
            cursor.execute("""
                INSERT INTO usage_events (...)
                VALUES (...)
                RETURNING ...
            """, (...))
            event = cursor.fetchone()
        connection.commit()
    except Exception:
        connection.rollback()
        raise
    return balance, event

def main():
    connection = psycopg.connect(...)
    print(E09(...))
    connection.close()
```

```
if __name__ == "__main__":
    main()
```

Book 7

NoSQL Lessons (MongoDB)

7.1 Introduction

7.1.1 What will this book teach you

The previous book introduced relational databases, where information is organized into tables and protected through explicitly defined relationships. This book introduces a different data model. MongoDB stores records as documents inside collections, allowing related values, arrays, and nested structures to exist together in one document. This flexibility is useful when application data naturally behaves like an object or when its structure must evolve over time.

The purpose of this book is not to present MongoDB as a replacement for PostgreSQL. Each system solves different problems and imposes different tradeoffs. PostgreSQL is often the stronger choice when relationships, transactions, and strict relational consistency define the core of the data. MongoDB becomes valuable when document-shaped data, nested structures, changing attributes, session memory, event payloads, or rapid schema evolution are central to the application. A good engineer must understand both tools and choose according to the data and access patterns rather than according to popularity.

Python will remain the main programming language throughout the book. You will connect to MongoDB using PyMongo, select databases and collections, insert documents, construct filters, control projections, update selected fields, build aggregation pipelines, create indexes, and inspect operation results. MongoDB query documents and update operators will be written inside Python dictionaries, allowing every exercise to remain part of a real application workflow.

MongoDB is commonly described as schema-less, but this description can be misleading. Documents in one collection may have different fields or shapes, yet the application still depends on assumptions about names, types, nested structures, required values, and versions. If these assumptions are not designed and validated, flexibility becomes inconsistency. This book therefore treats document design as an engineering responsibility rather than permission to store arbitrary data without rules.

The reader will learn through experimentation. You will insert documents with matching and conflicting structures, query missing and optional fields, compare embedded and referenced designs, migrate documents between schema versions, inspect execution plans, and observe how indexes affect performance. Later exercises introduce replica-set and sharding concepts so that

the reader can understand how data behavior changes when one database server becomes a distributed system.

The exercises provide explanations, hints, and incomplete Python skeletons, but not finished solutions. Before executing an operation, predict which documents should match, which fields should change, and what result object PyMongo should return. After execution, inspect both the operation result and the stored documents. A command that completes without an exception may still have matched zero documents or modified the wrong structure.

7.1.2 How MongoDB supports artificial intelligence systems

MongoDB does not train artificial intelligence models and does not calculate neural-network gradients. It supports the application and data layers surrounding those models. An AI system may use MongoDB to store chat messages, session metadata, user preferences, tool-call results, generated content metadata, experiment configurations, model responses, or documents whose fields differ according to their source or type.

Chat applications provide a useful example. Each message may contain a role, text, timestamp, token count, attachments, tool information, or model metadata. These values fit naturally into a document, and additional fields may appear as the system evolves. MongoDB can retrieve recent messages for one session, maintain session activity, expire temporary records, and aggregate message counts. However, the design must still consider ordering, document growth, indexes, retention, and the boundary between one session document and many message documents.

MongoDB will later complement the other data systems in this volume. PostgreSQL can preserve structured users, plans, permissions, and billing relationships. MinIO can store large binary objects such as datasets, images, and model files. Qdrant can store embeddings for semantic retrieval. MongoDB can hold flexible documents and operational context. The objective is not to force all data into one technology, but to understand the responsibility of each component inside a larger artificial intelligence system.

7.1.3 How to approach the exercises

Begin every exercise by describing the expected document shape and access pattern. Decide whether a field is required, optional, nested, repeated as an array, or represented by a reference. Then predict the filter and update behavior before running Python. Afterward, use a separate find operation to verify the final state. This cycle of design, prediction, execution, and verification follows the same experimental learning strategy used in the previous books.

Treat PyMongo result objects as evidence. For insert operations, inspect inserted identifiers. For updates and deletions, distinguish the number of matched documents from the number actually modified or removed. For bulk operations, consider whether ordered execution should stop at the first failure. For queries, define sorting explicitly whenever order matters and request only the fields the application needs.

Keep database addresses and credentials outside the source code, preferably in environment variables. Close clients or use controlled application lifetimes, use timeouts so failed connec-

tions do not wait indefinitely, and avoid storing real secrets in examples or repositories. When an exercise requires replica sets or sharding, use an isolated learning environment because those features require server-side configuration beyond ordinary Python commands.

Do not add an index merely because a field appears in a query. First identify the important read pattern, observe its execution plan, create a candidate index, and measure the result. Every index consumes storage and adds work to writes. Likewise, do not embed or reference documents by habit. Base the decision on ownership, access frequency, update patterns, cardinality, growth, and consistency requirements.

7.1.4 The Chapters (NOSQL00–NOSQL03) included in this book

- NOSQL00 - Document-Based Data Structures
- NOSQL01 - Chat History & Session Memory
- NOSQL02 - Dynamic Storage Models
- NOSQL03 - Scaling with Replica & Sharding

7.1.5 NOSQL00 - Document-Based Data Structures

This chapter introduces databases, collections, and BSON documents through PyMongo. You will create the first collection, insert one or many documents, query with filters, control returned fields with projections, and modify existing data with operators such as `$set`, `$inc`, and `$push`. Safe conditional deletion will reinforce the same verification discipline used with PostgreSQL.

The chapter then moves beyond basic operations into nested-document design and an analysis of schema-less pitfalls. You will observe how inconsistent field names, incompatible types, uncontrolled arrays, and missing assumptions make future queries difficult. By the end, you should be able to perform basic document operations and explain the intended structure of a collection even when MongoDB does not enforce every rule automatically.

7.1.6 NOSQL01 - Chat History & Session Memory

This chapter applies MongoDB to a realistic AI chat scenario. You will design message documents, connect each message to a session, query by session and time, sort messages chronologically, and retrieve a limited history window. These operations form the data foundation for constructing conversation context before a model receives a request.

You will also create a TTL index for expiration, update last-active timestamps, plan an archive strategy, and aggregate message counts per session. The final retrieval pipeline will combine filtering, ordering, limiting, projection, and output preparation. The goal is to understand that session memory is not an unlimited list: it requires retention rules, deterministic ordering, efficient access, and clear separation between active and archived data.

7.1.7 NOSQL02 - Dynamic Storage Models

This chapter studies flexibility as a deliberate design problem. You will handle optional fields, store polymorphic document types, add schema-version fields, and apply partial updates without overwriting unrelated data. Embedded and referenced models will be compared according to how the application reads, writes, owns, and grows its information.

Aggregation with `$group`, migrations between document versions, indexing of dynamic fields, and performance tradeoff evaluation will show the cost of flexibility. By the end of the chapter, you should be able to evolve a collection safely and recognize when variation is meaningful domain structure versus accidental inconsistency.

7.1.8 NOSQL03 - Scaling with Replica & Sharding

This chapter moves from document design on one logical database to the behavior of a distributed MongoDB system. It begins with indexes and query execution plans, providing the performance foundation needed before additional servers are introduced. You will then simulate or inspect replica-set behavior and configure read preference and write concern from Python.

The later exercises examine shard-key selection, chunk distribution, balancer behavior, and the difference between sharding and indexing. Sharding distributes data and workload; it does not repair inefficient queries or replace useful indexes. The chapter concludes with a scalable chat-system design that brings together document structure, access patterns, indexes, replication requirements, shard-key tradeoffs, retention, and failure behavior.

Some exercises in this chapter require multiple MongoDB processes or a configured cluster. Python will still be used to connect, configure supported client behavior, insert test data, and inspect results, but Python cannot replace the server topology itself. The learning environment must therefore be prepared carefully, and every experiment must distinguish client settings from server-side configuration.

7.2 NOSQL00 - Document-Based Data Structures

Data Engineering for AI

7.2.1 Exercise 00 - Create Database and Collection

Part 1 - Why this matters

Every MongoDB application begins by selecting a server, database, and collection. These objects define where documents belong and give Python a stable path through which later inserts, queries, and updates will be executed.

MongoDB creates many resources lazily. Merely obtaining a database or collection object in PyMongo does not necessarily create persistent server metadata, so the reader must verify existence from the server rather than from Python object creation alone.

Part 2 - Technical concept and behavior

`MongoClient` represents access to a MongoDB deployment. Bracket notation selects a database and collection by name. A database normally appears after it contains a collection with stored data, while `create_collection()` can create a collection explicitly. A `ping` command verifies communication with the server.

Part 3 - Hints and Helper

Read the MongoDB URI and names from environment variables. Configure a finite server-selection timeout, ping the server, create the collection only if it is absent, and verify it through `list_collection_names()`.

Part 4 - Python code skeleton

```
import os
from pymongo import MongoClient

def E00(database_name, collection_name):
    client = MongoClient(
        os.getenv(...),
        serverSelectionTimeoutMS=...,
    )
    client.admin.command(...)
    database = client[...]
    if collection_name not in database.list_collection_names():
        database.create_collection(...)
    collections = database.list_collection_names()
    client.close()
    return collections

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

7.2.2 Exercise 01 - Insert Single Document (insertOne)

Part 1 - Why this matters

A single document can preserve related fields, nested values, and arrays as one application record. Inserting one document is the simplest way to observe how a Python dictionary becomes BSON stored by MongoDB.

Verification must use the identifier returned by the server. The stored document may contain an automatically generated `_id`, and checking that record prevents the program from confusing its input dictionary with confirmed database state.

Part 2 - Technical concept and behavior

PyMongo's `insert_one()` sends one mapping to a collection and returns an `InsertOneResult`. If no `_id` is supplied, an identifier is generated. The result's `inserted_id` can be used in `find_one()` to retrieve the stored document.

Part 3 - Hints and Helper

Create a document containing several BSON-compatible types. Insert it, inspect acknowledgement and identifier, retrieve by `_id`, and avoid assuming field order carries application meaning.

Part 4 - Python code skeleton

```
from datetime import datetime, timezone
from pymongo import MongoClient

def E01(collection, document):
    result = collection.insert_one(...)
    inserted_id = result.inserted_id
    stored = collection.find_one({"_id": ...})
    return result.acknowledged, inserted_id, stored

def main():
    client = MongoClient(...)
    collection = client[...][...]
    document = {
        "name": ...,
        "active": ...,
        "created_at": datetime.now(timezone.utc),
    }
    print(E01(...))
    client.close()

if __name__ == "__main__":
    main()
```

7.2.3 Exercise 02 - Insert Multiple Documents (insertMany)

Part 1 - Why this matters

Applications often create batches of events, messages, or imported records. A bulk insert reduces client-server round trips and returns identifiers corresponding to the submitted document order.

Batch failure behavior must be understood. Ordered insertion normally stops at the first failing document, while unordered insertion may continue with independent operations, so verification must inspect what was actually stored rather than treating the batch as automatically atomic.

Part 2 - Technical concept and behavior

`insert_many()` accepts a non-empty iterable of documents and returns `inserted_ids`. The `ordered` option controls whether later inserts are attempted after an error. A bulk operation is not automatically equivalent to a multi-document transaction.

Part 3 - Hints and Helper

Insert a clean batch first and retrieve documents by the returned identifiers. Then use an isolated test collection with a uniqueness rule to compare ordered and unordered failure behavior without contaminating other exercises.

Part 4 - Python code skeleton

```
from pymongo import MongoClient

def E02(collection, documents, ordered=True):
    if not documents:
        raise ValueError(...)
    result = collection.insert_many(..., ordered=...)
    inserted_ids = result.inserted_ids
    stored = list(collection.find(
        {"_id": {"$in": ...}},
        sort=[("_id", ...)],
    ))
    return inserted_ids, stored

def main():
    client = MongoClient(...)
    collection = client[...][...]
    documents = [{...}, {...}, {...}]
    print(E02(...))
    client.close()

if __name__ == "__main__":
    main()
```

7.2.4 Exercise 03 - Basic Find Query with Filters

Part 1 - Why this matters

MongoDB filters let an application select documents according to field values, ranges, membership, and logical combinations. Sending the intended filter to the database avoids loading an entire collection and filtering it later in Python.

Document flexibility makes type awareness important. A numeric comparison does not mean the same thing for documents where the field is missing or stored as text, so the reader must inspect the actual matched population.

Part 2 - Technical concept and behavior

`find()` accepts a filter document whose field conditions may use operators such as `$gt`, `$gte`, `$in`, `$and`, and `$or`. Multiple top-level field conditions are combined as a logical AND. The method returns a cursor evaluated as results are consumed.

Part 3 - Hints and Helper

Build filters from controlled Python values, not raw user-supplied operator documents. Combine an equality condition with a numeric range, sort explicitly, materialize the cursor once, and test filters producing many, one, and zero matches.

Part 4 - Python code skeleton

```
from pymongo import MongoClient

def E03(collection, category, minimum_score):
    query_filter = {
        "category": ...,
        "score": {"$gte": ...},
        "active": ...,
    }
    cursor = collection.find(...).sort([
        ("score", ...),
        ("_id", ...),
    ])
    documents = list(cursor)
    return documents

def main():
    client = MongoClient(...)
    collection = client[...][...]
    print(E03(...))
    client.close()

if __name__ == "__main__":
    main()
```

7.2.5 Exercise 04 - Projection: Include / Exclude Fields

Part 1 - Why this matters

An application often needs only part of each document. Projection reduces transferred data and prevents unrelated or sensitive fields from being returned to code that does not require them.

MongoDB distinguishes inclusion and exclusion projection styles. Mixing them incorrectly causes an error or unclear intent, so the reader must design the returned shape as deliberately as the filter.

Part 2 - Technical concept and behavior

An inclusion projection assigns `1` to selected fields, while an exclusion projection assigns `0` to omitted fields. The two styles generally cannot be mixed in one projection, except that `_id` may be excluded from an inclusion projection. Nested fields can be addressed with dot notation.

Part 3 - Hints and Helper

Run one inclusion and one exclusion query against the same filter. Verify keys in every returned document, decide explicitly whether `_id` is required, and avoid using projection as a substitute for access control.

Part 4 - Python code skeleton

```
from pymongo import MongoClient

def E04(collection, query_filter):
    include_projection = {
        "_id": ...,
        "name": ...,
        "profile.region": ...,
    }
    exclude_projection = {
        "secret": ...,
        "internal_notes": ...,
    }
    included = list(collection.find(..., ...))
    excluded = list(collection.find(..., ...))
    return included, excluded

def main():
    client = MongoClient(...)
    collection = client[...][...]
    print(E04(...))
    client.close()

if __name__ == "__main__":
    main()
```

7.2.6 Exercise 05 - Update Document with $set

Part 1 - Why this matters

Replacing an entire document to change one field can accidentally remove fields added by another part of the application. Targeted update operators express exactly which values should change while leaving the rest of the document untouched.

The operation result distinguishes matching from modification. A filter may find a document whose field already contains the requested value, producing one match but zero modifications, which is different from matching no document at all.

Part 2 - Technical concept and behavior

`update_one(filter, update)` updates at most one matching document. The `$set` operator assigns values to selected fields and can target nested paths with dot notation. `matched_count` reports matches, while `modified_count` reports actual changes.

Part 3 - Hints and Helper

Require a specific identifier or another unique condition. Use `$set` for one top-level and one nested field, inspect both counts, then retrieve the document to verify that unrelated fields remain.

Part 4 - Python code skeleton

```
from pymongo import MongoClient

def E05(collection, document_id, new_status, new_region):
    query_filter = {"_id": ...}
    update = {
        "$set": {
            "status": ...,
            "profile.region": ...,
        }
    }
    result = collection.update_one(..., ...)
    stored = collection.find_one(query_filter)
    return result.matched_count, result.modified_count, stored

def main():
    client = MongoClient(...)
    collection = client[...][...]
    print(E05(...))
    client.close()

if __name__ == "__main__":
    main()
```

7.2.7 Exercise 06 - Update with $inc and $push

Part 1 - Why this matters

Counters and arrays are common inside document-based applications. Reading a counter into Python, changing it, and writing it back creates unnecessary race opportunities, while replacing an array can discard elements written by another request.

MongoDB update operators perform these changes on the server. The reader will increment a numeric field and append an array element in one update, then examine type and growth risks rather than assuming every document can accept the operation.

Part 2 - Technical concept and behavior

`$inc` adds a numeric amount to a numeric field and can create a missing field with that amount. `$push` appends an element to an array and can create a missing array field. Both can fail when an existing field has an incompatible type. Unbounded arrays can also cause documents to grow excessively.

Part 3 - Hints and Helper

Use one `update_one()` call containing both operators. Verify counts and final values, test an incompatible field type in an isolated document, and consider `$each` with `$slice` when the application requires a bounded array.

Part 4 - Python code skeleton

```
from datetime import datetime, timezone
from pymongo import MongoClient

def E06(collection, document_id, increment, event):
    update = {
        "$inc": {"usage_count": ...},
        "$push": {
            "events": {
                "$each": [...],
                "$slice": ...,
            }
        },
        "$set": {"updated_at": datetime.now(timezone.utc)},
    }
    result = collection.update_one({"_id": ...}, update)
    stored = collection.find_one({"_id": ...})
    return result.matched_count, result.modified_count, stored

def main():
    client = MongoClient(...)
    collection = client[...][...]
    print(E06(...))
    client.close()

if __name__ == "__main__":
    main()
```

7.2.8 Exercise 07 - Delete Document with Condition

Part 1 - Why this matters

Deletion removes persistent information and must be scoped more carefully than an ordinary read. A broad or empty filter can remove an unintended document, while a weak condition can delete a valid record that merely shares one field.

This challenge treats deletion as a verified operation. The reader will preview one specific target, delete with the same condition, inspect `deleted_count`, and confirm that neighboring documents remain.

Part 2 - Technical concept and behavior

`delete_one(filter)` removes at most one matching document and returns a `DeleteResult`. If several documents match, which one is removed should not be relied upon without a unique filter. A count of zero means no document matched at deletion time.

Part 3 - Hints and Helper

Reject an empty filter in Python. Require `_id` plus an expected state, preview with `find_one()`, perform the deletion, and verify both absence of the target and preservation of collection count expectations.

Part 4 - Python code skeleton

```
from pymongo import MongoClient

def E07(collection, document_id, expected_status):
    query_filter = {
        "_id": ...,
        "status": ...,
    }
    if not query_filter:
        raise ValueError(...)
    preview = collection.find_one(query_filter)
    if preview is None:
        return ...
    result = collection.delete_one(query_filter)
    verification = collection.find_one({"_id": ...})
    return preview, result.deleted_count, verification

def main():
    client = MongoClient(...)
    collection = client[...][...]
    print(E07(...))
    client.close()

if __name__ == "__main__":
    main()
```

7.2.9 Exercise 08 - Nested Document Design Challenge

Part 1 - Why this matters

Nested documents keep values together when they belong to one conceptual object, such as a user's profile, model configuration, or request metadata. This can make one-document reads natural and preserve the structure already used by Python.

Nesting is not automatically correct for every relationship. Deep structures, frequently changing shared values, and uncontrolled embedded arrays can make updates and indexes difficult, so the reader must justify the boundary of each embedded object.

Part 2 - Technical concept and behavior

BSON documents can contain other documents and arrays. Dot notation addresses nested paths in filters, projections, updates, and indexes. Embedding is most suitable when nested data is owned by the parent, commonly read with it, and bounded in size.

Part 3 - Hints and Helper

Design a document containing profile, preferences, and bounded model settings. Insert it, query by a nested field, project selected nested values, and update one nested path without replacing the parent object.

Part 4 - Python code skeleton

```
from pymongo import MongoClient

def E08(collection, user_id):
    document = {
        "user_id": ...,
        "profile": {
            "name": ...,
            "region": ...,
        },
        "preferences": {
            "language": ...,
            "notifications": {...},
        },
        "model_settings": {...},
    }
    inserted = collection.insert_one(...)
    matched = collection.find_one(
        {"profile.region": ...},
        {"profile.name": 1, "preferences.language": 1},
    )
    updated = collection.update_one(
        {"_id": inserted.inserted_id},
        {"$set": {"preferences.language": ...}},
    )
    return inserted.inserted_id, matched, updated.modified_count

def main():
    client = MongoClient(...)
    collection = client[...][...]
    print(E08(...))
    client.close()

if __name__ == "__main__":
    main()
```

7.2.10 Exercise 09 - Schema-less Design Pitfall Analysis

Part 1 - Why this matters

Allowing every document to take an unrelated shape moves complexity into every query and update. Misspelled fields, mixed numeric and string types, inconsistent date formats, and missing version information can silently divide one logical collection into incompatible populations.

The final challenge asks the reader to discover and classify inconsistency before repairing it. The goal is to understand that flexible schema supports controlled evolution, while uncontrolled variation creates hidden data debt.

Part 2 - Technical concept and behavior

MongoDB filters such as `$exists` and `$type` can identify missing or incompatible fields. Aggregation can count shapes or types across a collection. Collection validators can enforce selected rules for future writes, but existing data may require migration before stronger validation is introduced.

Part 3 - Hints and Helper

Create isolated documents containing deliberate field-name, type, and nesting inconsistencies. Produce a report before changing data, define one canonical shape, migrate carefully, and verify that the number of repaired documents matches expectations.

Part 4 - Python code skeleton

```
from pymongo import MongoClient

def E09(collection):
    missing_field = list(collection.find({"required_field": {"$exists": ...}}))
    wrong_type = list(collection.find({"score": {"$not": {"$type": ...}}}))
    type_pipeline = [
        {"$project": {"observed_type": {"$type": "$score"}}},
        {"$group": {"_id": "$observed_type", "count": {"$sum": 1}}},
        {"$sort": {"count": -1}},
    ]
    type_report = list(collection.aggregate(...))
    migration_result = collection.update_many(..., ...)
    return {
        "missing": missing_field,
        "wrong_type": wrong_type,
        "types": type_report,
        "migrated": migration_result.modified_count,
    }

def main():
    client = MongoClient(...)
    collection = client[...][...]
    print(E09(...))
    client.close()

if __name__ == "__main__":
    main()
```

7.3 NOSQL01 - Chat History & Session Memory

Data Engineering for AI

7.3.1 Exercise 00 - Design Chat Message Document Structure

Part 1 - Why this matters

Chat memory begins with a document structure that can preserve message identity, session membership, speaker role, content, and time. These fields determine whether an application can reconstruct context accurately and distinguish user messages from model or tool outputs.

The design must remain useful as message metadata evolves. Optional token counts, model names, attachments, and tool-call details should not make basic retrieval ambiguous, while uncontrolled embedded histories must not cause one session document to grow without bound.

Part 2 - Technical concept and behavior

This chapter models each message as a separate document linked by `session_id`. Core fields include a stable identifier, role, content, creation timestamp, and optional sequence number or metadata subdocument. Separate message documents support independent insertion, indexing, retention, and pagination.

Part 3 - Hints and Helper

Write the field contract before inserting data. Validate allowed roles and timezone-aware timestamps in Python, keep optional metadata nested, and include a deterministic tie-breaker for messages sharing the same timestamp.

Part 4 - Python code skeleton

```
from datetime import datetime, timezone

def E00(session_id, role, content, sequence, metadata=None):
    allowed_roles = {...}
    if role not in allowed_roles:
        raise ValueError(...)
    if ...:
        raise ValueError(...)
    document = {
        "session_id": ...,
        "role": ...,
        "content": ...,
        "sequence": ...,
        "created_at": datetime.now(timezone.utc),
        "metadata": metadata or {},
        "schema_version": ...,
    }
    return document

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

7.3.2 Exercise 01 - Insert Chat Messages with Session ID

Part 1 - Why this matters

Every message must be assigned to the correct conversation. A missing or incorrect session identifier can mix unrelated histories and send another user's context to the model, making session association both a correctness and privacy concern.

This exercise inserts messages through one controlled Python function and verifies them by returned identifiers. It also considers duplicate retries, because a network retry should not silently create the same logical message twice.

Part 2 - Technical concept and behavior

PyMongo stores the message document with `insert_one()`. A unique client-generated message identifier can provide idempotency across retries. An index on session and ordering fields supports later retrieval, while uniqueness on an idempotency key prevents duplicate logical events.

Part 3 - Hints and Helper

Confirm that required fields exist before insertion. Include `session_id` and a unique message key, inspect `inserted_id`, then retrieve by both identifiers to prove the document belongs to the intended session.

Part 4 - Python code skeleton

```
from pymongo import MongoClient

def E01(collection, message_document):
    required_fields = {...}
    if ...:
        raise ValueError(...)
    result = collection.insert_one(...)
    stored = collection.find_one({
        "_id": result.inserted_id,
        "session_id": message_document["session_id"],
    })
    return result.inserted_id, stored

def main():
    client = MongoClient(...)
    collection = client[...][...]
    print(E01(...))
    client.close()

if __name__ == "__main__":
    main()
```

7.3.3 Exercise 02 - Query Messages by Session and Time

Part 1 - Why this matters

An AI application may need only the messages from one session and one time window. Filtering at the database prevents unrelated conversations from entering model context and

avoids transferring a complete message collection into Python.

Time boundaries must be explicit. Half-open intervals prevent one message at a boundary from appearing in two adjacent windows, and timezone-aware values prevent local-time assumptions from changing the meaning of the query.

Part 2 - Technical concept and behavior

The filter combines exact `session_id` equality with a timestamp condition using `$gte` for the start and `$lt` for the end. BSON dates represent instants at millisecond precision. A compound index can support both session selection and chronological range access.

Part 3 - Hints and Helper

Require timezone-aware Python datetimes and validate that start precedes end. Use one filter document, return only message fields needed by the caller, and verify boundary messages deliberately.

Part 4 - Python code skeleton

```
from pymongo import MongoClient

def E02(collection, session_id, start_time, end_time):
    if ...:
        raise ValueError(...)
    query_filter = {
        "session_id": ...,
        "created_at": {
            "$gte": ...,
            "$lt": ...,
        },
    }
    projection = {...}
    messages = list(collection.find(query_filter, projection))
    return messages

def main():
    client = MongoClient(...)
    collection = client[...][...]
    print(E02(...))
    client.close()

if __name__ == "__main__":
    main()
```

7.3.4 Exercise 03 - Sort Messages Chronologically

Part 1 - Why this matters

Conversation meaning depends on order. Returning the correct messages in an unstable sequence can reverse questions and answers, place tool output before its call, or construct invalid

model context.

A timestamp alone may not provide a total order when several messages share its stored precision. The reader must add a deterministic tie-breaker and verify that repeated retrieval returns the same sequence.

Part 2 - Technical concept and behavior

PyMongo cursors use `sort()` with one or more field-direction pairs. Ascending `created_at` provides chronological order, while a sequence number or unique identifier resolves ties. The index order should follow the important equality and sorting pattern.

Part 3 - Hints and Helper

Filter by one session, sort by timestamp and sequence, and insert test messages sharing a timestamp. Compare the returned sequence across repeated executions and inspect the query pattern needed by a compound index.

Part 4 - Python code skeleton

```
from pymongo import ASCENDING, MongoClient

def E03(collection, session_id):
    cursor = collection.find(
        {"session_id": ...},
        {...},
    ).sort([
        ("created_at", ASCENDING),
        ("sequence", ASCENDING),
        ("_id", ASCENDING),
    ])
    messages = list(cursor)
    sequence_values = [...]
    is_ordered = ...
    return messages, is_ordered

def main():
    client = MongoClient(...)
    collection = client[...][...]
    print(E03(...))
    client.close()

if __name__ == "__main__":
    main()
```

7.3.5 Exercise 04 - Limit Chat History per Session

Part 1 - Why this matters

Sending an entire long conversation to a model increases latency, token use, and cost, and may exceed the model's context capacity. A retrieval limit gives the application a controlled

recent-memory window.

The latest messages are commonly fetched in reverse order for efficiency, but model context usually requires chronological order. The reader must distinguish database retrieval order from the final order presented to the application.

Part 2 - Technical concept and behavior

The query filters one session, sorts newest first, and applies `limit(n)`. Python then reverses the small result list to restore chronological order. The limit controls document count, not token count, so later systems may require an additional token-budget step.

Part 3 - Hints and Helper

Reject non-positive limits. Sort by timestamp and deterministic tie-breakers in descending order, limit on the server, reverse once in Python, and verify the returned set contains the newest n messages in oldest-to-newest order.

Part 4 - Python code skeleton

```
from pymongo import DESCENDING, MongoClient

def E04(collection, session_id, history_limit):
    if ...:
        raise ValueError(...)
    newest_first = list(
        collection.find({"session_id": ...}, {...})
        .sort([
            ("created_at", DESCENDING),
            ("sequence", DESCENDING),
            ("_id", DESCENDING),
        ])
        .limit(...)
    )
    chronological = ...
    return chronological

def main():
    client = MongoClient(...)
    collection = client[...][...]
    print(E04(...))
    client.close()

if __name__ == "__main__":
    main()
```

7.3.6 Exercise 05 - Session Expiration using TTL Index

Part 1 - Why this matters

Temporary sessions should not remain forever when retention rules say they are no longer needed. Automatic expiration reduces stale data and cleanup code, but it must be designed around the correct date field and must not be mistaken for exact scheduling.

TTL deletion is performed asynchronously by MongoDB. An expired document may remain visible briefly, so application correctness or immediate access revocation must not depend only on the background deletion time.

Part 2 - Technical concept and behavior

A TTL index is a special single-field index on a BSON date field. With `expireAfterSeconds`, MongoDB considers documents eligible after the indexed time plus the configured duration. TTL indexes do not provide immediate deletion and unsuitable or missing field values may not expire as expected.

Part 3 - Hints and Helper

Use a dedicated sessions collection and timezone-aware expiration dates. Create the TTL index through PyMongo, inspect its metadata, insert an already expired test session, and poll with a reasonable deadline rather than assuming instant removal.

Part 4 - Python code skeleton

```
import time
from pymongo import ASCENDING, MongoClient

def E05(sessions, expire_after_seconds, deadline_seconds):
    index_name = sessions.create_index(
        [("expires_at", ASCENDING)],
        expireAfterSeconds=...,
        name=...,
    )
    index_info = sessions.index_information()[index_name]
    test_id = sessions.insert_one({...}).inserted_id
    deadline = time.monotonic() + deadline_seconds
    while time.monotonic() < deadline:
        if sessions.find_one({"_id": test_id}) is None:
            return index_info, ...
        ...
    return index_info, ...

def main():
    client = MongoClient(...)
    sessions = client[...][...]
    print(E05(...))
    client.close()

if __name__ == "__main__":
    main()
```

7.3.7 Exercise 06 - Update Last-Active Timestamp

Part 1 - Why this matters

The last-active time supports expiration, monitoring, and the identification of currently used sessions. Concurrent requests may finish out of order, so blindly setting the field can allow an older event to overwrite a newer timestamp.

This exercise updates activity without moving time backward. It also verifies whether the session existed and whether the stored value changed, preserving the distinction between a missing session and an already newer timestamp.

Part 2 - Technical concept and behavior

The `$max` update operator changes a field only when the proposed value compares greater than the current value. With consistent BSON date types, this supports monotonic last-active timestamps. `matched_count` may be one while `modified_count` is zero for an older proposal.

Part 3 - Hints and Helper

Require a timezone-aware timestamp. Apply `$max` to one uniquely identified session, return the result counts, and test updates in newer-then-older order to prove the stored timestamp does not regress.

Part 4 - Python code skeleton

```
from pymongo import MongoClient, ReturnDocument

def E06(sessions, session_id, activity_time):
    if ...:
        raise ValueError(...)
    document = sessions.find_one_and_update(
        {"session_id": ...},
        {"$max": {"last_active_at": ...}},
        return_document=ReturnDocument.AFTER,
    )
    if document is None:
        return ...
    return document["last_active_at"]

def main():
    client = MongoClient(...)
    sessions = client[...][...]
    print(E06(...))
    client.close()

if __name__ == "__main__":
    main()
```

7.3.8 Exercise 07 - Archive Old Sessions Strategy

Part 1 - Why this matters

Inactive sessions may need to leave the operational collection without being destroyed. Archiving reduces the active working set while preserving records required for analysis, support, or policy-defined retention.

Copying and deleting are separate risks. The strategy must be restartable, avoid duplicate archive records, verify successful copies before deletion, and define what happens when new messages arrive during archival.

Part 2 - Technical concept and behavior

An archive workflow selects sessions older than a cutoff, writes them to an archive collection using a stable unique key, verifies the archived set, and then deletes only confirmed source records. Multi-document transactions require a supported deployment such as a replica set; otherwise the workflow must use idempotency and recoverable phases.

Part 3 - Hints and Helper

Process a bounded batch. Use upserts keyed by session identity, record archive metadata, compare expected and confirmed identifiers, and construct the source deletion filter from confirmed IDs rather than repeating only the original age condition.

Part 4 - Python code skeleton

```
from datetime import datetime, timezone
from pymongo import MongoClient, UpdateOne

def E07(active_sessions, archive_sessions, cutoff, batch_size):
    candidates = list(
        active_sessions.find({"last_active_at": {"$lt": ...}})
        .sort("last_active_at", 1)
        .limit(...)
    )
    operations = [
        UpdateOne(
            {"session_id": document["session_id"]},
            {"$setOnInsert": {...}, "$set": {"archived_at": ...}},
            upsert=True,
        )
        for document in candidates
    ]
    archive_result = archive_sessions.bulk_write(..., ordered=...)
    confirmed_ids = [...]
    delete_result = active_sessions.delete_many({"session_id": {"$in": ...}})
    return archive_result, delete_result.deleted_count

def main():
    client = MongoClient(...)
```

```
    print(E07(...))
    client.close()

if __name__ == "__main__":
    main()
```

7.3.9 Exercise 08 - Aggregation: Messages Count per Session

Part 1 - Why this matters

Counting messages per session supports monitoring, retention decisions, debugging, and identification of unusually large conversations. Performing the grouping in MongoDB avoids retrieving every message merely to count it in Python.

A useful summary may need a time filter and additional measures such as first and last message time. The reader must recognize that sessions with zero messages do not appear when aggregation starts from the messages collection.

Part 2 - Technical concept and behavior

An aggregation pipeline passes documents through ordered stages. `$match` restricts the input population, `$group` collects documents by `session_id`, and `$sort` orders summary documents. Accumulators such as `$sum`, `$min`, and `$max` calculate group values.

Part 3 - Hints and Helper

Place a selective time filter before grouping. Return session ID, count, first time, and last time, then sort by count and session ID. Compare the total of group counts with a direct count over the same filter.

Part 4 - Python code skeleton

```
from pymongo import MongoClient

def E08(messages, start_time, end_time):
    pipeline = [
        {"$match": {
            "created_at": {"$gte": ..., "$lt": ...},
        }},
        {"$group": {
            "_id": "$session_id",
            "message_count": {"$sum": 1},
            "first_message_at": {"$min": "$created_at"},
            "last_message_at": {"$max": "$created_at"},
        }},
        {"$sort": {"message_count": -1, "_id": 1}},
    ]
    summary = list(messages.aggregate(...))
    verification_count = messages.count_documents(...)
    return summary, verification_count
```

```
def main():
    client = MongoClient(...)
    messages = client[...][...]
    print(E08(...))
    client.close()

if __name__ == "__main__":
    main()
```

7.3.10 Exercise 09 - Session Memory Retrieval Pipeline

Part 1 - Why this matters

A model should receive a controlled and correctly ordered context, not a raw dump of every stored message. The retrieval pipeline must isolate one session, exclude unusable records, select required fields, take the newest permitted window, and return it chronologically.

This final challenge combines document design, filtering, sorting, projection, and limiting. It also preserves a boundary between database retrieval and later token-budget or prompt-formatting logic.

Part 2 - Technical concept and behavior

The aggregation pipeline uses `$match`, descending `$sort`, `$limit`, and `$project`. A final ascending sort can restore chronological order within MongoDB after the newest window is selected. Stable tie-breakers are required in both ordering stages.

Part 3 - Hints and Helper

Validate the history limit and session identity. Match allowed roles and usable content, select the newest messages first, then reorder only that selected window. Return plain Python records suitable for a later model-input formatter.

Part 4 - Python code skeleton

```
from pymongo import MongoClient

def E09(messages, session_id, history_limit, allowed_roles):
    if ...:
        raise ValueError(...)
    pipeline = [
        {"$match": {
            "session_id": ...,
            "role": {"$in": ...},
            "content": {"$type": "string", "$ne": ""},
        }},
        {"$sort": {"created_at": -1, "sequence": -1, "_id": -1}},
        {"$limit": ...},
        {"$sort": {"created_at": 1, "sequence": 1, "_id": 1}},
        {"$project": {
            "_id": 0,
```

```
            "role": 1,
            "content": 1,
            "created_at": 1,
            "metadata": 1,
        }},
    ]
    context = list(messages.aggregate(...))
    return context

def main():
    client = MongoClient(...)
    messages = client[...][...]
    print(E09(...))
    client.close()

if __name__ == "__main__":
    main()
```

7.4 NOSQL02 - Dynamic Storage Models

Data Engineering for AI

7.4.1 Exercise 00 - Flexible Document Schema Design

Part 1 - Why this matters

Flexible documents allow an application to store records whose optional details differ without creating a column for every possible variation. This is useful for model configurations, tool results, events, and metadata produced by several sources.

Flexibility still requires a stable contract. Identifiers, document type, timestamps, and schema version should remain predictable, while variation should be placed in deliberate nested areas rather than spread across misspelled or incompatible top-level fields.

Part 2 - Technical concept and behavior

A flexible design separates common fields from a type-specific payload. MongoDB permits different document shapes in one collection, and optional collection validation can enforce selected common rules. The design should support known queries and updates while allowing controlled extension.

Part 3 - Hints and Helper

Define required common fields and a bounded payload structure. Create several valid variations, inspect their shapes, and reject documents missing the core contract in Python before considering server-side validation.

Part 4 - Python code skeleton

```
from datetime import datetime, timezone

def E00(document_type, owner_id, payload):
    allowed_types = {...}
    if document_type not in allowed_types:
        raise ValueError(...)
    if not isinstance(payload, dict):
        raise TypeError(...)
    document = {
        "document_type": ...,
        "owner_id": ...,
        "schema_version": ...,
        "created_at": datetime.now(timezone.utc),
        "payload": ...,
    }
    required_fields = {...}
    is_valid = ...
    return document, is_valid

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

7.4.2 Exercise 01 - Optional Fields Handling

Part 1 - Why this matters

An optional field may be absent because it does not apply, unknown because its value has not arrived, or explicitly set to null. Collapsing these states without thought can change query results and cause Python to confuse missing information with a meaningful empty value.

This exercise makes each state observable. The reader will query missing, null, and present values separately, then normalize application output without overwriting the original database meaning.

Part 2 - Technical concept and behavior

`$exists` tests whether a field is present. A query for `None` can match explicit null values and, in common equality behavior, documents where the field is absent. Combining existence and type conditions provides more precise populations. Python dictionaries distinguish a missing key from a key whose value is `None`.

Part 3 - Hints and Helper

Insert three isolated documents representing absent, null, and populated states. Query each category explicitly, use `dict.get()` only when its collapsing behavior is intended, and report the original state beside any fallback value.

Part 4 - Python code skeleton

```
from pymongo import MongoClient

def E01(collection, field_name, fallback):
    missing_filter = {field_name: {"$exists": ...}}
    explicit_null_filter = {
        "$and": [
            {field_name: None},
            {field_name: {"$exists": ...}},
        ]
    }
    present_filter = {field_name: {"$exists": ...}}
    missing = list(collection.find(missing_filter))
    explicit_null = list(collection.find(explicit_null_filter))
    present = list(collection.find(present_filter))
    normalized = [...]
    return missing, explicit_null, present, normalized

def main():
    client = MongoClient(...)
    collection = client[...][...]
    print(E01(...))
    client.close()

if __name__ == "__main__":
    main()
```

7.4.3 Exercise 02 - Polymorphic Documents in One Collection

Part 1 - Why this matters

One collection may hold related records with different type-specific fields, such as text, image, and tool-result events. Keeping them together can simplify chronological retrieval when they share ownership, lifecycle, and access patterns.

Polymorphism becomes dangerous when the application must guess a document's type from whichever fields happen to exist. An explicit discriminator allows validation, querying, and Python processing to branch intentionally.

Part 2 - Technical concept and behavior

A discriminator such as `document_type` identifies the variant. Common fields remain stable, while each type owns a defined payload contract. Queries can filter one variant, and compound or partial indexes can support important type-specific access patterns.

Part 3 - Hints and Helper

Define at least three variants with shared metadata. Validate required payload keys according to the discriminator, insert examples, and query each type without relying on field-existence guessing.

Part 4 - Python code skeleton

```
from pymongo import MongoClient

def validate_document(document):
    document_type = document.get(...)
    rules = {
        "text": {...},
        "image": {...},
        "tool_result": {...},
    }
    if document_type not in rules:
        return ...
    required_payload_fields = rules[document_type]
    return ...

def E02(collection, documents, requested_type):
    valid_documents = [document for document in documents if ...]
    result = collection.insert_many(...)
    selected = list(collection.find({"document_type": ...}))
    return result.inserted_ids, selected

def main():
    client = MongoClient(...)
    collection = client[...][...]
    print(E02(...))
    client.close()

if __name__ == "__main__":
    main()
```

7.4.4 Exercise 03 - Versioned Document Schema Strategy

Part 1 - Why this matters

Application requirements evolve, and a collection may temporarily contain documents written by several software versions. Without an explicit schema version, readers must infer structure from fields and cannot distinguish intentional old shapes from corrupted data.

Versioning makes change visible. Python can route each document through the correct reader or migration path, while new writes use the latest contract without requiring every old document to change immediately.

Part 2 - Technical concept and behavior

A numeric `schema_version` records the document contract version. Read-time adapters can normalize older documents in memory, while write-time migrations persist the new shape. Migrations should advance in ordered steps and remain safe when retried.

Part 3 - Hints and Helper

Define versions 1 and 2 with one meaningful structural difference. Write a dispatcher that rejects unknown future versions, adapts version 1 without mutating the retrieved input, and leaves version 2 unchanged.

Part 4 - Python code skeleton

```
from copy import deepcopy

CURRENT_VERSION = ...

def adapt_v1_to_v2(document):
    adapted = deepcopy(document)
    adapted[...] = ...
    adapted.pop(..., None)
    adapted["schema_version"] = ...
    return adapted

def E03(document):
    version = document.get("schema_version", ...)
    if version == ...:
        normalized = adapt_v1_to_v2(...)
    elif version == CURRENT_VERSION:
        normalized = deepcopy(...)
    else:
        raise ValueError(...)
    return normalized

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

7.4.5 Exercise 04 - Partial Updates without Overwrite

Part 1 - Why this matters

Replacing a document with a partial Python payload can erase fields that the caller did not load or does not own. This becomes especially dangerous in flexible schemas where different components manage different nested values.

Targeted operators allow the application to state which paths should change, be removed, or be initialized. The reader will compare a safe path update with accidental replacement of an entire nested object.

Part 2 - Technical concept and behavior

`$set` changes named fields, `$unset` removes named fields, and dot notation targets nested paths without replacing their sibling fields. Setting a whole embedded document replaces that embedded value, so update granularity must match ownership.

Part 3 - Hints and Helper

Start with a nested document containing several sibling fields. Apply a patch represented by approved paths, reject protected fields such as `_id`, update through `update_one()`, and compare the complete document before and after.

Part 4 - Python code skeleton

```
from pymongo import MongoClient

def E04(collection, document_id, set_fields, unset_fields):
    protected_paths = {...}
    if ...:
        raise ValueError(...)
    update = {}
    if set_fields:
        update["$set"] = ...
    if unset_fields:
        update["$unset"] = {field: "" for field in ...}
    before = collection.find_one({"_id": ...})
    result = collection.update_one({"_id": ...}, update)
    after = collection.find_one({"_id": ...})
    return before, result.modified_count, after

def main():
    client = MongoClient(...)
    collection = client[...][...]
    print(E04(...))
    client.close()

if __name__ == "__main__":
    main()
```

7.4.6 Exercise 05 - Embedded vs Referenced Documents

Part 1 - Why this matters

Embedding keeps related data in one document, while referencing stores it separately and connects it with identifiers. The choice affects read count, update duplication, document growth, consistency, and the ability to share one record among many parents.

There is no universal winner. The reader must base the decision on ownership, cardinality, access patterns, update frequency, and size rather than translating a relational design mechanically into MongoDB.

Part 2 - Technical concept and behavior

Embedded data can be read atomically with its parent and is suitable for owned, bounded substructures. References are useful when data is shared, independently updated, large, or unbounded. MongoDB aggregation can resolve references with `$lookup`, but this adds query work and does not create a foreign-key constraint.

Part 3 - Hints and Helper

Model the same small scenario both ways. Implement one read and one update for each design, count database operations, inspect duplicated data, and write a decision supported by the scenario's expected growth and access pattern.

Part 4 - Python code skeleton

```
from pymongo import MongoClient

def E05(embedded_collection, parent_collection, child_collection, parent_id):
    embedded_result = embedded_collection.find_one({"_id": ...})
    referenced_pipeline = [
        {"$match": {"_id": ...}},
        {"$lookup": {
            "from": ...,
            "localField": ...,
            "foreignField": ...,
            "as": ...,
        }},
    ]
    referenced_result = list(parent_collection.aggregate(...))
    comparison = {
        "embedded_reads": ...,
        "referenced_reads": ...,
        "duplication": ...,
        "growth_risk": ...,
    }
    return embedded_result, referenced_result, comparison

def main():
    client = MongoClient(...)
    print(E05(...))
    client.close()

if __name__ == "__main__":
    main()
```

7.4.7 Exercise 06 - Aggregation Pipeline with $group

Part 1 - Why this matters

Dynamic documents often need summaries by type, owner, status, or another shared field. Aggregation allows MongoDB to transform varied documents into consistent analytical results without transferring every source record to Python.

Grouping exposes design inconsistencies as well as useful totals. Missing fields, null values, and incompatible numeric types can create unexpected groups or affect accumulators, so the reader must inspect input quality before trusting the summary.

Part 2 - Technical concept and behavior

An aggregation pipeline processes ordered stages. `$match` selects relevant documents, `$group` creates groups using an `_id` expression, and accumulators calculate count, sum, average, minimum, or maximum. Later stages can project and sort the summaries.

Part 3 - Hints and Helper

Filter to one schema version and numeric type before grouping. Group by a discriminator and status, calculate count and total, rename the group key with `$project`, and compare summed group counts with a direct document count.

Part 4 - Python code skeleton

```
from pymongo import MongoClient

def E06(collection, schema_version):
    pipeline = [
        {"$match": {
            "schema_version": ...,
            "payload.value": {"$type": ...},
        }},
        {"$group": {
            "_id": {
                "type": "$document_type",
                "status": "$status",
            },
            "document_count": {"$sum": 1},
            "total_value": {"$sum": "$payload.value"},
            "average_value": {"$avg": "$payload.value"},
        }},
        {"$project": {...}},
        {"$sort": {...}},
    ]
    summary = list(collection.aggregate(...))
    return summary

def main():
    client = MongoClient(...)
    collection = client[...][...]
    print(E06(...))
    client.close()

if __name__ == "__main__":
    main()
```

7.4.8 Exercise 07 - Data Migration between Schema Versions

Part 1 - Why this matters

When a document contract changes, old records do not update themselves. A migration must transform existing documents without skipping records, corrupting already migrated data, or becoming unsafe when interrupted and restarted.

This exercise treats migration as a controlled batch process. The reader will filter by the exact source version, update only expected shapes, record progress, and verify source and target version counts after each batch.

Part 2 - Technical concept and behavior

An idempotent migration changes documents only when `schema_version` equals the source version. Update pipelines or bulk operations can rename, derive, set, and unset fields. Bounded batches reduce operational impact and allow progress to be resumed.

Part 3 - Hints and Helper

Write the transformation as one version step. Select a stable batch by identifiers, include the old version in every update filter, advance the version only with the successful transformation, and run the migration twice to test idempotency.

Part 4 - Python code skeleton

```
from pymongo import MongoClient, UpdateOne

def E07(collection, source_version, target_version, batch_size):
    candidates = list(
        collection.find(
            {"schema_version": ...},
            {...},
        ).sort("_id", 1).limit(...)
    )
    operations = []
    for document in candidates:
        transformed_fields = ...
        operations.append(UpdateOne(
            {"_id": document["_id"], "schema_version": source_version},
            {"$set": {...}, "$unset": {...}},
        ))
    result = collection.bulk_write(..., ordered=...)
    remaining = collection.count_documents({"schema_version": source_version})
    migrated = collection.count_documents({"schema_version": target_version})
    return result.modified_count, remaining, migrated

def main():
    client = MongoClient(...)
    collection = client[...][...]
    print(E07(...))
    client.close()

if __name__ == "__main__":
    main()
```

7.4.9 Exercise 08 - Indexing Strategy for Dynamic Fields

Part 1 - Why this matters

Dynamic fields make it tempting to index every possible path, but excessive indexes consume storage, slow writes, and may still fail to support the actual query shape. Index strategy must begin with observed filters and sorts.

Different index types handle variation differently. The reader will compare a focused compound index, a partial index for one document subtype, and a wildcard-index candidate, then inspect which execution plan MongoDB actually selects.

Part 2 - Technical concept and behavior

A compound index supports a known field sequence. A partial index stores entries only for documents matching a filter, while a sparse index omits documents missing the indexed field. A wildcard index can cover many dynamic paths but may cost more and be less focused than workload-specific indexes.

Part 3 - Hints and Helper

Start from three concrete query patterns. Create one candidate at a time in an isolated collection, call `explain()` on matching queries, inspect key and document counts, and record write and storage tradeoffs.

Part 4 - Python code skeleton

```
from pymongo import ASCENDING, MongoClient

def E08(collection, document_type, dynamic_value):
    focused_name = collection.create_index([
        ("document_type", ASCENDING),
        ("payload.dynamic_field", ASCENDING),
    ], name=...)
    partial_name = collection.create_index(
        [("payload.special_field", ASCENDING)],
        partialFilterExpression={"document_type": ...},
        name=...,
    )
    plan = collection.find({
        "document_type": ...,
        "payload.dynamic_field": ...,
    }).explain()
    indexes = collection.index_information()
    return focused_name, partial_name, plan, indexes

def main():
    client = MongoClient(...)
    collection = client[...][...]
    print(E08(...))
    client.close()
```

```
if __name__ == "__main__":
    main()
```

7.4.10 Exercise 09 - Performance Tradeoff Evaluation

Part 1 - Why this matters

Document design changes read latency, write cost, storage size, network transfer, and migration complexity. A design that makes one query fast may duplicate data or make updates expensive, so performance conclusions require several measures and a stated workload.

The final exercise compares two storage models under controlled conditions. The reader must keep data volume and requested output equivalent, repeat measurements, inspect execution statistics, and avoid declaring a universal winner from one timing.

Part 2 - Technical concept and behavior

Evaluation can compare execution time, documents examined, keys examined, returned-document size, write operations, and index storage. Cache state and dataset size influence observed latency. Functional correctness and operational complexity remain part of the tradeoff even when one path is faster.

Part 3 - Hints and Helper

Define one read-heavy and one update scenario for embedded and referenced designs. Run warm-up and measured trials, collect plans, use equivalent filters and projections, summarize several runs, and document limitations of the experiment.

Part 4 - Python code skeleton

```
from statistics import median
from time import perf_counter
from pymongo import MongoClient

def measure(operation, repetitions):
    timings = []
    for _ in range(repetitions):
        started = perf_counter()
        result = operation()
        timings.append(perf_counter() - started)
    return {"median": median(timings), "runs": timings, "result": result}

def E09(embedded_operation, referenced_operation, repetitions):
    if ...:
        raise ValueError(...)
    embedded = measure(...)
    referenced = measure(...)
    comparison = {
        "embedded": embedded,
        "referenced": referenced,
```

```
        "execution_plans": ...,
        "documents_examined": ...,
        "write_tradeoff": ...,
    }
    return comparison

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

7.5 NOSQL03 - Scaling with Replica & Sharding

Data Engineering for AI

7.5.1 Exercise 00 - Create Index for High-Read Query

Part 1 - Why this matters

A frequently executed chat query can become expensive as message volume grows. An index can reduce examined documents and avoid an in-memory sort, but only when its fields and order match the query's equality, range, and sorting pattern.

Scaling begins by improving access on the current data model. Adding replicas or shards before understanding the query may distribute inefficient work instead of removing it.

Part 2 - Technical concept and behavior

For a query that filters by `session_id` and sorts recent messages by time, a compound index can place the equality field first and ordering fields afterward. The index also adds storage and write maintenance cost, so it should be justified by a high-read workload.

Part 3 - Hints and Helper

Write the target query first. Capture its plan, create one named compound index through PyMongo, run the same query again, and inspect keys examined, documents examined, and whether a blocking sort remains.

Part 4 - Python code skeleton

```
from pymongo import ASCENDING, DESCENDING, MongoClient

def E00(collection, session_id):
    target_filter = {"session_id": ...}
    sort_specification = [
        ("created_at", DESCENDING),
        ("sequence", DESCENDING),
    ]
    before = collection.database.command(...)
```

```
    index_name = collection.create_index([
        ("session_id", ASCENDING),
        ("created_at", DESCENDING),
        ("sequence", DESCENDING),
    ], name=...)
    after = collection.database.command(...)
    return index_name, before, after

def main():
    client = MongoClient(...)
    collection = client[...][...]
    print(E00(...))
    client.close()

if __name__ == "__main__":
    main()
```

7.5.2 Exercise 01 - Explain Query Execution Plan

Part 1 - Why this matters

An execution plan reveals how MongoDB answered a query rather than how the developer hoped it would. It can show collection scans, index scans, document fetches, sorting, examined records, and differences between estimated access and actual execution.

The reader must interpret evidence instead of searching only for the word "index." An index scan can still examine too many keys, and a collection scan may be reasonable for a tiny or non-selective dataset.

Part 2 - Technical concept and behavior

The `explain` command can return planner information or execution statistics. Important values include winning plan stages, returned documents, total keys examined, and total documents examined. Sharded explain output may include plans from multiple shards.

Part 3 - Hints and Helper

Use the database `explain` command with a safe find operation and execution-statistics verbosity. Write a recursive helper that searches nested plan structures rather than assuming all fields occur at one fixed path.

Part 4 - Python code skeleton

```
from pymongo import MongoClient

def collect_stages(node, stages=None):
    stages = stages or []
    if isinstance(node, dict):
        if "stage" in node:
            stages.append(node["stage"])
        for value in node.values():
```

```
            collect_stages(value, stages)
    elif isinstance(node, list):
        for value in node:
            collect_stages(value, stages)
    return stages

def E01(collection, query_filter):
    command = {
        "find": collection.name,
        "filter": ...,
        "sort": {...},
        "limit": ...,
    }
    explanation = collection.database.command(
        "explain", command, verbosity=...,
    )
    stages = collect_stages(...)
    statistics = ...
    return stages, statistics, explanation

def main():
    client = MongoClient(...)
    print(E01(client[...][...], ...))
    client.close()

if __name__ == "__main__":
    main()
```

7.5.3 Exercise 02 - Replica Set Concept Simulation

Part 1 - Why this matters

A replica set maintains copies of data across members and can elect a new primary when the current primary becomes unavailable. This improves availability and supports configurable durability, but it does not eliminate replication delay or application retry requirements.

This exercise observes topology rather than pretending one standalone server is a replica set. The reader will connect to an isolated replica-set deployment, identify members and roles, perform a write, and inspect how the driver views the topology.

Part 2 - Technical concept and behavior

Writes normally go to the primary and secondaries replicate the primary's operation log asynchronously. PyMongo discovers replica-set members and performs server selection. Election and failover are server behaviors; Python observes and handles them but does not replace the configured deployment.

Part 3 - Hints and Helper

Use a test replica set with at least three data-bearing members when possible. Connect using a seed list and replica-set name, run `hello`, inspect discovered nodes, insert a uniquely identified record, and avoid forcing failover outside an isolated lab.

Part 4 - Python code skeleton

```
from pymongo import MongoClient

def E02(uri, replica_set_name):
    client = MongoClient(
        uri,
        replicaSet=...,
        serverSelectionTimeoutMS=...,
    )
    topology = {
        "primary": client.primary,
        "secondaries": list(client.secondaries),
        "nodes": list(client.nodes),
    }
    hello = client.admin.command("hello")
    collection = client[...][...]
    result = collection.insert_one({...})
    verified = collection.find_one({"_id": result.inserted_id})
    client.close()
    return topology, hello, verified

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

7.5.4 Exercise 03 - Read Preference Configuration

Part 1 - Why this matters

Read preference controls which replica-set members are eligible for a read. Reading from secondaries may distribute read work or improve locality, but asynchronous replication means those reads can return older data.

The correct mode depends on whether the application prioritizes freshness, availability, or workload distribution. Chat context and account state may have different tolerance for stale reads, so one global choice should not be made blindly.

Part 2 - Technical concept and behavior

The default mode is primary. Other modes include primary preferred, secondary, secondary preferred, and nearest. Read preference chooses eligible servers; read concern controls visibility and consistency guarantees. Transactions containing reads must use primary read preference.

Part 3 - Hints and Helper

Create collection handles with different read preferences using `with_options()`. Run the same identifiable read, inspect the preference attached to each handle, and use command monitoring or server information if available to observe selection without claiming that secondary reads are always stale.

Part 4 - Python code skeleton

```
from pymongo import MongoClient, ReadPreference

def E03(collection, query_filter):
    primary_collection = collection.with_options(
        read_preference=ReadPreference....
    )
    secondary_preferred_collection = collection.with_options(
        read_preference=ReadPreference....
    )
    primary_result = primary_collection.find_one(...)
    alternate_result = secondary_preferred_collection.find_one(...)
    return {
        "primary_mode": primary_collection.read_preference,
        "alternate_mode": secondary_preferred_collection.read_preference,
        "primary_result": primary_result,
        "alternate_result": alternate_result,
    }

def main():
    client = MongoClient(...)
    print(E03(client[...][...], ...))
    client.close()

if __name__ == "__main__":
    main()
```

7.5.5 Exercise 04 - Write Concern Configuration

Part 1 - Why this matters

Write concern determines the acknowledgment requested before MongoDB reports a write as successful. Stronger acknowledgment can improve durability confidence but may increase latency or reduce availability when required members cannot respond.

Acknowledgment is not the same as application correctness. A majority-acknowledged write can still contain invalid business data, while an unacknowledged write gives Python little evidence about whether the server accepted it.

Part 2 - Technical concept and behavior

Write concern options include `w`, journaling, and a write-concern timeout. `w="majority"` requests acknowledgment from a calculated majority of data-bearing voting members. In multi-document transactions, write concern belongs at transaction level rather than individual operations.

Part 3 - Hints and Helper

Use `with_options()` to create collection handles with explicit write concerns. Compare acknowledged results and latency in an isolated replica set, test an impossible requirement with a timeout, and do not interpret timeout as proof that no member applied the write.

Part 4 - Python code skeleton

```
from time import perf_counter
from pymongo import MongoClient
from pymongo.write_concern import WriteConcern

def E04(collection, document):
    majority_collection = collection.with_options(
        write_concern=WriteConcern(w=..., j=..., wtimeout=...),
    )
    started = perf_counter()
    result = majority_collection.insert_one(...)
    elapsed = perf_counter() - started
    verification = collection.find_one({"_id": result.inserted_id})
    return {
        "acknowledged": result.acknowledged,
        "elapsed": elapsed,
        "stored": verification,
    }

def main():
    client = MongoClient(...)
    print(E04(client[...][...], ...))
    client.close()

if __name__ == "__main__":
    main()
```

7.5.6 Exercise 05 - Shard Key Selection Challenge

Part 1 - Why this matters

The shard key determines how documents are partitioned and which queries can be routed to a limited set of shards. A poor key can create uneven distribution, write hotspots, scatter-gather reads, or ranges that are difficult to move.

Shard-key choice is difficult to reverse operationally and cannot be judged from cardinality alone. The reader must evaluate distribution, frequency, monotonic growth, query targeting, and the needs of a real chat workload.

Part 2 - Technical concept and behavior

A shard key is an indexed field or compound field set used to divide key space into ranges. Range-based keys preserve locality, while hashed sharding can spread values more evenly at the cost of some range-query targeting. Queries containing the shard key or a compound-key prefix can often be targeted more efficiently.

Part 3 - Hints and Helper

Compare candidates such as `session_id`, user plus time, and a monotonically increasing timestamp. Calculate cardinality and frequency skew from sample data, map each candidate to the main queries, and return a reasoned score rather than executing sharding automatically.

Part 4 - Python code skeleton

```
from collections import Counter

def evaluate_candidate(documents, field_path, query_patterns):
    values = [...]
    frequencies = Counter(values)
    metrics = {
        "cardinality": ...,
        "maximum_frequency": ...,
        "missing_count": ...,
        "monotonic_risk": ...,
        "targeted_query_count": ...,
    }
    score = ...
    return metrics, score

def E05(documents, candidates, query_patterns):
    evaluations = {
        candidate: evaluate_candidate(...)
        for candidate in candidates
    }
    recommendation = ...
    return evaluations, recommendation

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

7.5.7 Exercise 06 - Chunk Distribution Observation

Part 1 - Why this matters

A sharded collection is divided into non-overlapping shard-key ranges associated with chunks. Observing where those ranges reside helps reveal uneven distribution, insufficient key diversity,

zones, or migrations that have not completed.

Chunk count alone does not equal balanced workload. Chunks can contain different data volumes or receive very different request rates, so metadata observations must be combined with document and workload evidence.

Part 2 - Technical concept and behavior

Chunk ranges have inclusive lower and exclusive upper shard-key bounds. Cluster metadata identifies the sharded collection, its key, and range ownership. Metadata representation can vary by server version, and inspecting it requires connection through `mongos` with suitable privileges.

Part 3 - Hints and Helper

Run only against an isolated sharded cluster. Verify that the client is connected through `mongos`, identify the collection metadata and UUID, inspect chunk or range ownership using supported cluster metadata, and summarize counts by shard without modifying placement.

Part 4 - Python code skeleton

```
from collections import Counter
from pymongo import MongoClient

def E06(client, database_name, collection_name):
    hello = client.admin.command("hello")
    if hello.get("msg") != ...:
        raise RuntimeError(...)
    namespace = f"{database_name}.{collection_name}"
    collection_metadata = client["config"]["collections"].find_one({
        "_id": namespace,
    })
    collection_uuid = collection_metadata.get(...)
    ranges = list(client["config"][...].find({
        ...: collection_uuid,
    }))
    distribution = Counter(range_document[...] for range_document in ranges)
    return collection_metadata, dict(distribution), ranges

def main():
    client = MongoClient(...)
    print(E06(client, ..., ...))
    client.close()

if __name__ == "__main__":
    main()
```

7.5.8 Exercise 07 - Balancer Behavior Analysis

Part 1 - Why this matters

The balancer moves shard-key ranges to improve data distribution and respect configured zones. These migrations consume network, storage, and server resources, so balancing behavior must be observed rather than assumed to be free or immediate.

An uneven cluster is not automatically evidence that the balancer is broken. Thresholds, active migrations, zones, jumbo ranges, low data volume, or a poor shard key can explain the current state.

Part 2 - Technical concept and behavior

MongoDB's balancer runs in the background and migrates ranges among shards. Cluster commands expose balancer state, while metadata and logs provide evidence about distribution and migration. Starting or stopping balancing changes cluster behavior and should not be performed casually in a shared environment.

Part 3 - Hints and Helper

Use read-only observations in this challenge. Query balancer status through `mongos`, capture distribution snapshots at controlled intervals, and classify whether the evidence shows movement, stability, or an inconclusive state. Do not block with long sleeps.

Part 4 - Python code skeleton

```
from datetime import datetime, timezone
from pymongo import MongoClient

def E07(client, distribution_reader):
    balancer_status = client.admin.command(...)
    before = distribution_reader(...)
    after = distribution_reader(...)
    comparison = {
        "observed_at": datetime.now(timezone.utc),
        "mode": balancer_status.get(...),
        "in_balancing_round": balancer_status.get(...),
        "before": before,
        "after": after,
        "changed": ...,
        "conclusion": ...,
    }
    return comparison

def main():
    client = MongoClient(...)
    print(E07(...))
    client.close()

if __name__ == "__main__":
    main()
```

7.5.9 Exercise 08 - Sharding vs Indexing Comparison

Part 1 - Why this matters

Indexes and sharding solve different scaling problems. An index improves access paths within a collection's storage, while sharding distributes data and workload across multiple shards. One does not replace the other.

A sharded query without the right shard-key filter may broadcast work, and each targeted shard may still need suitable indexes. The reader must identify whether a problem comes from local query access, machine capacity, distribution, or all three.

Part 2 - Technical concept and behavior

Indexing affects keys and documents examined on each participating shard. Sharding affects routing and the number of shards participating. A targeted query includes sufficient shard-key information, while a scatter-gather query is sent to multiple shards by `mongos`. Explain output can reveal both routing and local plan behavior.

Part 3 - Hints and Helper

Compare four cases where available: targeted with index, targeted without useful index, scatter-gather with index, and scatter-gather without useful index. Use the same logical result and record participating shards, keys examined, documents examined, and latency.

Part 4 - Python code skeleton

```
from pymongo import MongoClient

def explain_case(collection, query_filter):
    command = {
        "find": collection.name,
        "filter": ...,
        "sort": {...},
        "limit": ...,
    }
    explanation = collection.database.command(
        "explain", command, verbosity=...,
    )
    return {
        "participating_shards": ...,
        "stages": ...,
        "keys_examined": ...,
        "documents_examined": ...,
        "execution_time": ...,
    }

def E08(collection, targeted_filter, scatter_filter):
    targeted = explain_case(...)
    scatter = explain_case(...)
    comparison = ...
```

```
    return targeted, scatter, comparison

def main():
    client = MongoClient(...)
    print(E08(client[...][...], ..., ...))
    client.close()

if __name__ == "__main__":
    main()
```

7.5.10 Exercise 09 - Scalable Chat System Data Design

Part 1 - Why this matters

A scalable chat system must preserve message order and session isolation while supporting heavy writes, recent-history reads, retention, availability, and growth across servers. Optimizing only one requirement can create hotspots, stale context, oversized documents, or expensive broadcast queries.

The final exercise combines the entire MongoDB book into one evidence-based design. The reader must define document boundaries, indexes, replica behavior, shard-key strategy, archive policy, and failure expectations before implementing a small Python validation harness.

Part 2 - Technical concept and behavior

Messages remain separate, bounded documents connected by session identity and deterministic order. Compound indexes support recent-session reads. Replica settings express freshness and durability needs, while a shard key balances write distribution against targeted session retrieval. TTL or archival rules control lifecycle without serving as immediate authorization logic.

Part 3 - Hints and Helper

Write explicit workload assumptions: message rate, session size, read pattern, retention, regions, and consistency tolerance. Return the proposed schemas and index definitions, score shard-key candidates, and run checks for deterministic retrieval, duplicate-message protection, and query targeting.

Part 4 - Python code skeleton

```
from pymongo import ASCENDING, DESCENDING, MongoClient
from pymongo.write_concern import WriteConcern

def E09(client, configuration, sample_messages):
    messages = client[...][...].with_options(
        write_concern=WriteConcern(...),
    )
    index_definitions = [
        [
            ("session_id", ASCENDING),
            ("created_at", DESCENDING),
```

```
            ("sequence", DESCENDING),
        ],
        [...],
    ]
    created_indexes = [messages.create_index(...) for ... in index_definitions]
    inserted = messages.insert_many(...)
    recent_context = list(
        messages.find({"session_id": ...}, {...})
        .sort([...])
        .limit(...)
    )
    explanation = messages.database.command(...)
    validation = {
        "deterministic_order": ...,
        "duplicate_protection": ...,
        "targeted_query": ...,
        "retention_defined": ...,
        "failure_behavior_defined": ...,
    }
    return created_indexes, inserted.inserted_ids, recent_context, explanation, validati

def main():
    client = MongoClient(...)
    print(E09(...))
    client.close()

if __name__ == "__main__":
    main()
```

Book 8

Object Storage Lessons (MinIO)

8.1 Introduction

8.1.1 What will this book teach you

Artificial intelligence systems work with more than rows and documents. They also produce and consume datasets, images, audio, archives, model weights, checkpoints, logs, and other large binary objects. Storing these files only on the local disk of one application server makes them difficult to share, scale, protect, and recover. This book introduces MinIO as an object-storage system for managing that class of data through an S3-compatible interface.

Object storage organizes data differently from PostgreSQL, MongoDB, and an ordinary local filesystem. Every object belongs to a bucket and is identified by an object name. The stored object can contain arbitrary binary data and can also carry metadata, tags, version information, and access rules. Names may contain slash characters that look like folders, but these paths are prefixes inside a flat object namespace rather than ordinary operating-system directories.

Python will remain the primary programming language throughout this book. You will connect to MinIO using its Python SDK, create and inspect buckets, upload and download objects, stream large files, attach metadata, list by prefix, work with versions, generate temporary access links, and integrate object operations into a training pipeline. The MinIO command-line client and terminal commands will be used only when they are necessary for installation, server administration, policy configuration, or direct comparison with Python behavior.

This book does not treat successful upload as sufficient proof. A stored object may have the correct name but incorrect bytes, incomplete metadata, an unintended access policy, or the wrong version. The exercises therefore use verification techniques such as object statistics, content length, hashes, version identifiers, metadata inspection, and controlled download comparisons. The reader should learn to distinguish confirmation from the client library from independent validation of the stored content.

Large objects introduce concerns that do not appear in small database records. Transfers may fail partway, memory cannot hold an entire file, repeated uploads can consume bandwidth, and old checkpoint versions can fill storage. The reader will learn to stream data, reason about multipart uploads, create stable object naming conventions, compare local and network storage

performance, and design lifecycle or garbage-collection strategies that preserve required recovery points.

The final chapter introduces distributed storage and fault tolerance. These exercises must be performed only in an isolated laboratory deployment using disposable test data. Stopping nodes, removing test disks, or changing cluster configuration can cause data loss when the environment is not designed with the required redundancy and quorum. The purpose is to observe failure and recovery safely, not to experiment on production storage.

The exercises provide technical explanations, hints, and incomplete Python skeletons rather than finished solutions. Before every operation, predict the target bucket, object name, version, metadata, and expected result. Afterward, verify the server state and content. This preserves the volume's learning strategy: design, predict, execute, observe, explain, and correct.

8.1.2 How object storage supports artificial intelligence systems

Object storage does not train a model or replace a database. It provides durable, network-accessible storage for large objects surrounding the model. A training system may read dataset partitions from object storage, write checkpoints after selected epochs, publish final weights, store evaluation artifacts, and allow another machine to resume work without depending on the first machine's local disk.

The four data technologies in this volume have different responsibilities. PostgreSQL stores strongly structured relationships such as users, plans, limits, and audits. MongoDB stores flexible documents such as messages, sessions, and evolving metadata. MinIO stores large binary objects such as dataset files, images, archives, weights, and checkpoints. Qdrant, introduced in the next book, stores vectors used for similarity and semantic retrieval. A real AI system may store an object's binary bytes in MinIO while preserving its business record in PostgreSQL, flexible processing metadata in MongoDB, and its embedding in Qdrant.

This separation prevents large files from being forced into systems designed primarily for records or documents. It also prevents MinIO from being misused as a query database. Object metadata and prefixes can support organization and limited filtering, but they do not replace relational joins, document queries, or vector search. The application must decide which system owns each type of information and store stable identifiers that connect those systems safely.

8.1.3 How to approach the exercises

Use a dedicated learning deployment and test buckets. Keep endpoints, access keys, and secret keys outside source files, preferably in environment variables. Never upload real credentials to a repository. Configure TLS and certificate verification for environments beyond local development, and do not disable secure verification merely to silence a connection error.

Object names require deliberate design. Include stable identifiers and meaningful prefixes, but do not depend on folder behavior that object storage does not provide. Avoid placing secrets in object names because names may appear in logs and operational tools. When overwriting an existing name, determine whether versioning is enabled and record the returned version identifier when recovery depends on that exact version.

Treat deletion and public access as high-risk operations. Preview exact targets before deleting, avoid broad recursive operations unless the exercise explicitly requires an isolated test prefix, and understand that deleting the latest object in a versioned bucket may create a delete marker rather than erase every stored version. Generate presigned URLs with short expirations and minimum required permissions, and never mistake possession of a temporary URL for permanent identity-based authorization.

Performance exercises must compare equivalent bytes and environments. Record file size, elapsed time, throughput, number of runs, network conditions, and whether data may be cached. Do not conclude that object storage is universally slower or faster than local storage from one transfer. Local disks and network object storage provide different sharing, durability, scaling, and failure properties.

Distributed exercises require a prepared topology with multiple drives or nodes and an understood parity configuration. A single-node, single-drive learning server does not demonstrate high availability. Before simulating failure, verify redundancy and make an external backup of anything valuable. After failure, validate both availability and byte-level integrity, then observe recovery and healing before declaring the system healthy.

8.1.4 The Chapters (OB00–OB03) included in this book

- OB00 - Binary Data & Large File Storage
- OB01 - Datasets, Weights & Checkpoints
- OB02 - Buckets & Access Structure
- OB03 - Distributed Mode & Fault Tolerance

8.1.5 OB00 - Binary Data & Large File Storage

This chapter establishes the object-storage foundation. You will install and start a learning MinIO server, configure the command-line client, create a bucket, and then perform object operations using Python. Upload, download, listing, prefix filtering, metadata, deletion, and integrity verification will build a complete understanding of the object lifecycle.

The chapter also examines large-file transfer, multipart behavior, bucket versioning, and performance measurement. The Python SDK may manage multipart upload details when a file exceeds relevant thresholds, but the reader must still understand why parts exist, how failed transfers differ from completed objects, and how memory-safe streaming changes the application. By the end, you should be able to store and retrieve binary content reliably and prove that the retrieved bytes match the source.

8.1.6 OB01 - Datasets, Weights & Checkpoints

This chapter applies object storage directly to machine-learning artifacts. You will design bucket and prefix conventions for datasets, upload a local hierarchy without assuming real folders, save model weights after training, and retrieve a checkpoint for resuming execution.

Dataset and checkpoint metadata will connect each object to its source, configuration, and training state.

The later exercises introduce versioned checkpoint strategies, atomic publication patterns, local-versus-object-storage I/O comparison, checkpoint garbage collection, and hash validation. Atomicity requires careful interpretation in object storage: the application should upload a complete candidate under a temporary or immutable name, verify it, and publish a small manifest or final reference only after success. The chapter concludes by integrating these operations into a training pipeline without embedding storage concerns unpredictably throughout the model code.

8.1.7 OB02 - Buckets & Access Structure

This chapter focuses on authorization and controlled sharing. You will configure bucket policies, compare read-only with read-write permissions, create application identities or access keys in a learning environment, and generate presigned URLs for temporary object access. Private and public behavior will be tested explicitly rather than inferred from successful administrator access.

You will also design prefix-based permissions, diagnose denied operations, coordinate access across multiple buckets, rotate credentials, and conduct an access-control audit. The guiding principle is least privilege: each identity should receive only the actions and resources required for its task. Credentials must never be written into exercise output, logs, source repositories, or object metadata.

8.1.8 OB03 - Distributed Mode & Fault Tolerance

This chapter moves from one learning server to a multi-drive or multi-node deployment. You will prepare distributed storage, simulate controlled node and disk failures, verify object availability and integrity, and observe healing after resources return or are replaced. These exercises demonstrate that fault tolerance depends on topology, erasure-set layout, parity, and quorum rather than on the MinIO name alone.

Erasure coding divides object data into data and parity shards distributed across drives. Increasing parity can improve failure tolerance but reduces usable capacity and may affect performance. The reader will measure these tradeoffs and distinguish erasure coding within a deployment from replication or disaster recovery across separate locations.

The final exercises cover high availability, external backup or replication strategy, and an end-to-end fault-tolerance test. A complete test records the initial topology, uploads verified objects, introduces one permitted failure, checks read and write behavior, restores the failed component, observes healing, and validates the same object hashes again. Passing the test requires both service availability and correct data, not merely a running process.

8.2 OB00 - Binary Data & Large File Storage

Data Engineering for AI

8.2.1 Exercise 00 - Install MinIO and Start Server

Part 1 - Why this matters

Before Python can store objects, a reachable MinIO service must be running with a known endpoint and protected credentials. Installation is therefore part of understanding the boundary between the client application and the storage server.

This exercise uses an isolated learning deployment and verifies it programmatically. A running process alone is insufficient; the reader must confirm network access, authentication, and a valid S3-compatible response.

Part 2 - Technical concept and behavior

MinIO exposes an object-storage API endpoint and normally a separate administration console endpoint. The server needs a data path and initial credentials supplied through environment configuration. Deployment commands depend on the selected package, container, and release, so they must be checked against the official documentation for that environment.

Part 3 - Hints and Helper

Use disposable local storage and never place real secrets in the script. Start the server through the chosen supported method, then use the Python SDK to list buckets as an authenticated readiness check. Configure a finite connection timeout.

Part 4 - Python code skeleton

```
import os
from minio import Minio

def E00():
    client = Minio(
        os.getenv(...),
        access_key=os.getenv(...),
        secret_key=os.getenv(...),
        secure=...,
    )
    buckets = client.list_buckets()
    return [bucket.name for bucket in buckets]

def main():
    print(E00())

if __name__ == "__main__":
    main()
```

8.2.2 Exercise 01 - Create Bucket using CLI

Part 1 - Why this matters

A bucket is the top-level namespace and policy boundary for stored objects. Creating it with the command-line client teaches the reader how operators configure storage outside application code while Python verifies the resulting server state.

Bucket creation should be repeatable and safe. The program must distinguish a successful creation from an already existing bucket and must not hide authentication, naming, or connectivity failures.

Part 2 - Technical concept and behavior

The MinIO client uses an alias that combines endpoint and credentials. The `mc mb` command creates a bucket, and an option can make an existing bucket a non-fatal condition. Bucket names follow S3-compatible naming constraints and are shared across all objects in that bucket.

Part 3 - Hints and Helper

Configure the alias outside captured output so credentials are not printed. Use Python's `subprocess.run()` with an argument list, check the return code, and verify creation with the Python SDK rather than trusting command text alone.

Part 4 - Python code skeleton

```
import subprocess
from minio import Minio

def E01(client, alias, bucket_name):
    command = ["mc", "mb", ..., f"{alias}/{bucket_name}"]
    completed = subprocess.run(
        command,
        capture_output=True,
        text=True,
        check=...,
    )
    exists = client.bucket_exists(...)
    return completed.returncode, exists

def main():
    client = Minio(...)
    print(E01(...))

if __name__ == "__main__":
    main()
```

8.2.3 Exercise 02 - Upload Single File (Object Put)

Part 1 - Why this matters

Uploading converts a local file into an object addressed by bucket and object name. The operation must preserve the file bytes while recording enough information for later retrieval and interpretation.

The local path is not automatically the object name. Separating them allows the application to create stable remote naming conventions independent of where a file happened to exist on one machine.

Part 2 - Technical concept and behavior

The Python SDK's `fput_object()` uploads a file from disk and returns information about the stored object. Reusing the same object name may replace the current object or create a new version when versioning is enabled. Content type describes representation but does not validate bytes.

Part 3 - Hints and Helper

Verify that the source is a regular file, define the object name explicitly, supply a suitable content type, inspect the returned ETag and version ID, and call `stat_object()` to verify the stored size.

Part 4 - Python code skeleton

```
from pathlib import Path
from minio import Minio

def E02(client, bucket_name, object_name, source_path, content_type):
    source = Path(source_path)
    if ...:
        raise FileNotFoundError(...)
    result = client.fput_object(
        bucket_name,
        object_name,
        str(source),
        content_type=...,
    )
    stored = client.stat_object(bucket_name, object_name)
    valid_size = stored.size == source.stat().st_size
    return result.etag, result.version_id, valid_size

def main():
    client = Minio(...)
    print(E02(...))

if __name__ == "__main__":
    main()
```

8.2.4 Exercise 03 - Download Object and Verify Integrity

Part 1 - Why this matters

A successful download confirms transfer, but not necessarily identity with the intended source. Integrity verification compares content-derived hashes so corruption or selection of the wrong object version becomes visible.

Hashing must stream data in bounded chunks. Large AI artifacts should not be loaded entirely into memory merely to verify them, and partial output should not replace a trusted local file before validation succeeds.

Part 2 - Technical concept and behavior

`fget_object()` downloads an object to a file. A cryptographic digest such as SHA-256 can be calculated independently for source and destination. ETags are useful object metadata but are not a universal substitute for a client-computed content hash, especially for multipart objects.

Part 3 - Hints and Helper

Download to a temporary path, calculate both hashes with fixed-size reads, compare sizes and digests, then publish or retain the file only when validation succeeds. Delete the temporary file after a failed comparison.

Part 4 - Python code skeleton

```
import hashlib
from pathlib import Path
from minio import Minio

def file_hash(path, chunk_size=1024 * 1024):
    digest = hashlib.sha256()
    with open(path, "rb") as source:
        while chunk := source.read(chunk_size):
            digest.update(...)
    return digest.hexdigest()

def E03(client, bucket_name, object_name, source_path, download_path):
    client.fget_object(...)
    source_digest = file_hash(...)
    downloaded_digest = file_hash(...)
    matches = ...
    return source_digest, downloaded_digest, matches

def main():
    client = Minio(...)
    print(E03(...))

if __name__ == "__main__":
    main()
```

8.2.5 Exercise 04 - List Objects with Prefix Filtering

Part 1 - Why this matters

Object stores can contain millions of names, so applications must request only the relevant namespace. Prefix filtering supports dataset partitions, model runs, dates, and tenants without

pretending that prefixes are true filesystem directories.

Listing is often paginated and may be lazy in the SDK. The reader must consume results deliberately, distinguish recursive from delimiter-based behavior, and avoid broad production listings when a precise prefix is available.

Part 2 - Technical concept and behavior

`list_objects()` returns an iterable of object information. The prefix is a leading string match on object names. Recursive listing returns objects beneath all matching name levels, while non-recursive behavior can expose common prefix groupings.

Part 3 - Hints and Helper

Upload test objects under several prefixes. List one prefix recursively, return names, sizes, and modification times, and verify that every name begins with the requested prefix.

Part 4 - Python code skeleton

```
from minio import Minio

def E04(client, bucket_name, prefix, recursive=True):
    objects = client.list_objects(
        bucket_name,
        prefix=...,
        recursive=...,
    )
    results = [
        {
            "name": item.object_name,
            "size": item.size,
            "modified": item.last_modified,
        }
        for item in objects
    ]
    valid_prefixes = all(... for item in results)
    return results, valid_prefixes

def main():
    client = Minio(...)
    print(E04(...))

if __name__ == "__main__":
    main()
```

8.2.6 Exercise 05 - Delete Object Safely

Part 1 - Why this matters

Deleting the wrong object can remove a dataset, model, or checkpoint required for recovery. Safe deletion therefore begins with an exact bucket, object name, and optional version rather

than a broad prefix or unverified user input.

Versioning changes deletion semantics. Removing the current name may create a delete marker while older versions remain, whereas deleting a specific version targets that stored version directly.

Part 2 - Technical concept and behavior

`remove_object()` requests deletion of one object and can accept a version identifier. Some removal responses are confirmed through later observation rather than a returned object body. A pre-delete stat and post-delete read or listing provide stronger evidence.

Part 3 - Hints and Helper

Reject empty names and require an expected size or ETag before deletion. Preview with `stat_object()`, remove exactly one target, then verify expected behavior while accounting for bucket versioning.

Part 4 - Python code skeleton

```
from minio import Minio
from minio.error import S3Error

def E05(client, bucket_name, object_name, expected_size, version_id=None):
    if not object_name:
        raise ValueError(...)
    preview = client.stat_object(
        bucket_name, object_name, version_id=...,
    )
    if preview.size != expected_size:
        raise ValueError(...)
    client.remove_object(
        bucket_name, object_name, version_id=...,
    )
    try:
        verification = client.stat_object(...)
    except S3Error as error:
        verification = ...
    return preview, verification

def main():
    client = Minio(...)
    print(E05(...))

if __name__ == "__main__":
    main()
```

8.2.7 Exercise 06 - Upload Large File using Multipart

Part 1 - Why this matters

Large files must be transferred without loading their complete contents into memory. Multipart upload divides the transfer into parts, allowing the client and server to handle large objects more efficiently and recover from some transfer interruptions.

The MinIO Python SDK can select multipart behavior automatically for suitable upload methods and sizes. The reader should observe this abstraction and verify the completed object rather than reimplementing the multipart protocol unnecessarily.

Part 2 - Technical concept and behavior

`fput_object()` uploads from a file path and manages multipart transfer when required. `put_object()` can stream from a readable object when length is known or configured appropriately. The final object becomes visible after successful completion; abandoned parts require server or administrative cleanup policies.

Part 3 - Hints and Helper

Generate a disposable file larger than the selected part size without holding it all in RAM. Upload with a documented part size accepted by the SDK, verify stored size and hash, and record memory and elapsed time.

Part 4 - Python code skeleton

```
from pathlib import Path
from time import perf_counter
from minio import Minio

def E06(client, bucket_name, object_name, source_path, part_size):
    source = Path(source_path)
    started = perf_counter()
    result = client.fput_object(
        bucket_name,
        object_name,
        str(source),
        part_size=...,
    )
    elapsed = perf_counter() - started
    stored = client.stat_object(bucket_name, object_name)
    throughput = ...
    return result.etag, stored.size, elapsed, throughput

def main():
    client = Minio(...)
    print(E06(...))

if __name__ == "__main__":
    main()
```

8.2.8 Exercise 07 - Metadata Attachment to Objects

Part 1 - Why this matters

Metadata helps an application interpret binary bytes without opening the complete object. A dataset object may record source, schema version, hash, or run identifier, while a checkpoint may record epoch and framework version.

Metadata should remain small and non-sensitive. It is not a full query database, and changing user metadata may require copying or rewriting the object depending on the storage API.

Part 2 - Technical concept and behavior

Upload methods can attach user metadata as string key-value pairs along with system metadata such as content type. `stat_object()` returns stored headers and metadata. Key normalization may differ from the original Python spelling, so verification should account for the SDK's returned form.

Part 3 - Hints and Helper

Attach a SHA-256 digest, artifact type, and schema version during upload. Do not store credentials or personal information. Retrieve object statistics and compare required metadata values with normalized expectations.

Part 4 - Python code skeleton

```
from minio import Minio

def E07(client, bucket_name, object_name, source_path, metadata):
    safe_keys = {...}
    if not set(metadata).issubset(safe_keys):
        raise ValueError(...)
    result = client.fput_object(
        bucket_name,
        object_name,
        source_path,
        metadata=...,
        content_type=...,
    )
    stored = client.stat_object(bucket_name, object_name)
    verified = ...
    return result.etag, stored.metadata, verified

def main():
    client = Minio(...)
    print(E07(...))

if __name__ == "__main__":
    main()
```

8.2.9 Exercise 08 - Versioning Enablement Challenge

Part 1 - Why this matters

Versioning preserves multiple iterations stored under the same object name. It protects against accidental overwrite and supports recovery of earlier datasets or checkpoints, but every version consumes storage and requires lifecycle management.

Deletion also changes under versioning because a delete marker can hide the latest object without erasing older versions. The reader must record version IDs whenever a workflow depends on an exact artifact.

Part 2 - Technical concept and behavior

Bucket versioning is enabled at bucket level. Each later write to the same object namespace creates a version with its own identifier. The latest version is returned by default, while listing and retrieval can target older versions explicitly.

Part 3 - Hints and Helper

Enable versioning on an isolated bucket using the SDK, verify status, upload two different contents under one name, record both version IDs, and retrieve each version to prove they remain distinct.

Part 4 - Python code skeleton

```
from minio import Minio
from minio.commonconfig import ENABLED
from minio.versioningconfig import VersioningConfig

def E08(client, bucket_name, object_name, first_path, second_path):
    client.set_bucket_versioning(
        bucket_name,
        VersioningConfig(...),
    )
    status = client.get_bucket_versioning(bucket_name)
    first = client.fput_object(bucket_name, object_name, first_path)
    second = client.fput_object(bucket_name, object_name, second_path)
    versions = list(client.list_objects(
        bucket_name,
        prefix=object_name,
        include_version=True,
    ))
    return status.status, first.version_id, second.version_id, versions

def main():
    client = Minio(...)
    print(E08(...))

if __name__ == "__main__":
    main()
```

8.2.10 Exercise 09 - Binary Storage Performance Test

Part 1 - Why this matters

Transfer performance influences dataset loading, checkpoint frequency, and recovery time. One measurement cannot represent the storage system because file size, network state, cache, concurrency, hashing, and server load all affect the result.

The final challenge builds a repeatable benchmark rather than chasing a flattering number. It records both throughput and integrity so a fast but incorrect transfer can never pass.

Part 2 - Technical concept and behavior

Upload throughput is transferred bytes divided by elapsed time, and download throughput is calculated similarly. Warm-up and repeated trials reveal variation. End-to-end timing may include hashing and local I/O, so the benchmark must state exactly which work occurs inside the timed region.

Part 3 - Hints and Helper

Test several file sizes, use unique object names, repeat each operation, report median and individual timings, and clean up only the isolated benchmark prefix after verifying exact targets.

Part 4 - Python code skeleton

```
from statistics import median
from time import perf_counter
from minio import Minio

def measure(operation, byte_count, repetitions):
    trials = []
    for _ in range(repetitions):
        started = perf_counter()
        operation()
        elapsed = perf_counter() - started
        trials.append({"seconds": elapsed, "bytes_per_second": ...})
    return trials

def E09(client, bucket_name, test_cases, repetitions):
    results = {}
    for test_case in test_cases:
        upload_trials = measure(...)
        download_trials = measure(...)
        integrity = ...
        results[test_case["name"]] = {
            "upload": upload_trials,
            "download": download_trials,
            "integrity": integrity,
        }
    return results

def main():
    client = Minio(...)
    print(E09(...))
```

```
if __name__ == "__main__":
    main()
```

8.3 OB01 - Datasets, Weights & Checkpoints

Data Engineering for AI

8.3.1 Exercise 00 - Design Dataset Bucket Structure

Part 1 - Why this matters

A stable object-name structure lets training jobs locate dataset versions and splits without depending on one machine's folders. Poor naming can mix raw and processed data or allow a mutable upload to change the meaning of an earlier experiment.

The design should express ownership, dataset identity, immutable version, split, and artifact name. Buckets should represent meaningful policy or lifecycle boundaries rather than being created for every small folder.

Part 2 - Technical concept and behavior

Object storage uses bucket names and object-name prefixes, not true nested directories. A structure such as dataset, version, split, and relative file path can group objects while a manifest records expected names, sizes, hashes, and schema information.

Part 3 - Hints and Helper

Define naming rules in Python and reject traversal-like components, empty versions, and ambiguous separators. Produce sample names for train, validation, and test splits, plus one manifest name.

Part 4 - Python code skeleton

```
from pathlib import PurePosixPath

def object_name(dataset_name, version, split, relative_path):
    allowed_splits = {...}
    if split not in allowed_splits:
        raise ValueError(...)
    parts = [dataset_name, version, split, relative_path]
    if ...:
        raise ValueError(...)
    return str(PurePosixPath(...))

def E00(dataset_name, version, files_by_split):
    names = [
        object_name(dataset_name, version, split, path)
        for split, paths in files_by_split.items()
        for path in paths
    ]
```

```
    manifest_name = ...
    return names, manifest_name

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

8.3.2 Exercise 01 - Upload Dataset with Folder Hierarchy

Part 1 - Why this matters

Datasets often arrive as local directories containing splits and nested class or partition folders. Uploading them correctly requires preserving relative names while preventing machine-specific absolute paths from entering the object namespace.

A robust uploader must skip directories, handle failures per file, and produce a manifest so the remote dataset can be verified independently from the source tree.

Part 2 - Technical concept and behavior

Python can walk the local tree, compute each file's path relative to the dataset root, convert separators to POSIX-style object names, and upload with `fput_object()`. Object storage does not require separate folder creation.

Part 3 - Hints and Helper

Resolve and validate the root, sort files for reproducibility, refuse symlinks that escape the root, compute size and hash, upload each object, and return a manifest containing success or failure for every file.

Part 4 - Python code skeleton

```
from pathlib import Path, PurePosixPath
from minio import Minio

def E01(client, bucket_name, local_root, remote_prefix):
    root = Path(local_root).resolve()
    manifest = []
    for path in sorted(root.rglob("*")):
        if not path.is_file():
            continue
        relative = path.relative_to(root)
        object_name = str(PurePosixPath(remote_prefix, *relative.parts))
        try:
            result = client.fput_object(...)
            manifest.append({...})
        except Exception as error:
            manifest.append({...})
    return manifest
```

```
def main():
    client = Minio(...)
    print(E01(...))

if __name__ == "__main__":
    main()
```

8.3.3 Exercise 02 - Store Model Weights after Training

Part 1 - Why this matters

Model weights are the learned output of training and must survive beyond one process or GPU machine. Uploading them to object storage makes a named artifact available to evaluation, deployment, and recovery workflows.

Weights should be accompanied by metadata that identifies architecture, training run, epoch, framework version, and integrity hash. A filename alone is not enough to reproduce how the artifact was produced.

Part 2 - Technical concept and behavior

PyTorch can serialize a model's `state_dict()` to a local temporary file or binary stream. The completed bytes are then uploaded under an immutable run-specific object name. Saving only weights differs from saving a complete training checkpoint.

Part 3 - Hints and Helper

Move tensors to a suitable serializable state, save to a temporary file, calculate SHA-256, upload with metadata, verify remote size, and remove only the local temporary file in a `finally` block.

Part 4 - Python code skeleton

```
import tempfile
import torch
from minio import Minio

def E02(client, bucket_name, object_name, model, metadata):
    temporary_path = ...
    try:
        torch.save(model.state_dict(), temporary_path)
        digest = ...
        result = client.fput_object(
            bucket_name,
            object_name,
            temporary_path,
            metadata={**metadata, "sha256": digest},
        )
        verification = client.stat_object(...)
    finally:
```

```
        ...
    return result.version_id, digest, verification

def main():
    model = torch.nn...
    client = Minio(...)
    print(E02(...))

if __name__ == "__main__":
    main()
```

8.3.4 Exercise 03 - Retrieve Checkpoint for Resume Training

Part 1 - Why this matters

Resuming training requires more than model weights. The checkpoint may need optimizer state, epoch, scheduler state, scaler state, and reproducibility information so the new process continues from the intended training state.

Serialized checkpoints can execute unsafe deserialization behavior when loaded through permissive mechanisms. Only trusted artifacts should be loaded, and integrity and provenance must be checked before PyTorch reads the file.

Part 2 - Technical concept and behavior

The application downloads one exact checkpoint name or version, verifies size and cryptographic hash, and then loads it using the safest mode compatible with its controlled checkpoint format. State dictionaries are applied to already constructed model and optimizer objects.

Part 3 - Hints and Helper

Download to a temporary file, compare its hash with trusted metadata or a manifest, reject mismatch, load only trusted content, validate required keys, and restore each state before returning the next epoch.

Part 4 - Python code skeleton

```
import tempfile
import torch
from minio import Minio

def E03(client, bucket_name, object_name, model, optimizer,
        expected_hash, version_id=None):
    temporary_path = ...
    client.fget_object(
        bucket_name, object_name, temporary_path, version_id=...,
    )
    actual_hash = ...
    if actual_hash != expected_hash:
        raise ValueError(...)
    checkpoint = torch.load(temporary_path, ...)
```

```
    required_keys = {...}
    if ...:
        raise ValueError(...)
    model.load_state_dict(...)
    optimizer.load_state_dict(...)
    next_epoch = checkpoint["epoch"] + 1
    return next_epoch, checkpoint.get("metrics")

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

8.3.5 Exercise 04 - Versioned Model Checkpoints Strategy

Part 1 - Why this matters

Checkpoint history supports rollback and comparison, but repeatedly overwriting one name hides which training step produced each artifact and can consume uncontrolled storage when bucket versioning is enabled.

A clear strategy separates immutable checkpoint names from small pointers such as latest or best. The manifest records identity and hash, allowing readers to select an exact artifact rather than trusting whichever bytes happen to be current.

Part 2 - Technical concept and behavior

Immutable names can include run ID, epoch, and content or configuration identity. Bucket versioning may preserve overwrites, while explicit unique names make lineage visible in listings. A latest-manifest object can reference the selected immutable checkpoint and its version ID.

Part 3 - Hints and Helper

Generate deterministic checkpoint names, reject duplicate epoch publication, upload the immutable checkpoint first, verify it, then update a JSON manifest containing name, version ID, hash, epoch, and timestamp.

Part 4 - Python code skeleton

```
import json
from io import BytesIO
from minio import Minio

def E04(client, bucket_name, run_id, epoch, checkpoint_path):
    checkpoint_name = ...
    checkpoint_result = client.fput_object(...)
    checkpoint_hash = ...
    manifest = {
        "run_id": run_id,
        "epoch": epoch,
```

```
        "object_name": checkpoint_name,
        "version_id": checkpoint_result.version_id,
        "sha256": checkpoint_hash,
    }
    payload = json.dumps(manifest).encode("utf-8")
    manifest_result = client.put_object(
        bucket_name,
        ...,
        BytesIO(payload),
        len(payload),
        content_type="application/json",
    )
    return checkpoint_result, manifest_result, manifest

def main():
    client = Minio(...)
    print(E04(...))

if __name__ == "__main__":
    main()
```

8.3.6 Exercise 05 - Atomic Model Save Operation

Part 1 - Why this matters

Consumers must never discover a checkpoint before its bytes and metadata are complete. Object storage does not provide an ordinary atomic filesystem rename, so a local write-then-rename pattern cannot be copied directly.

A safe publication workflow uploads an immutable candidate, verifies it, and only then updates a small manifest or pointer. Readers trust the pointer and therefore see either the previous valid artifact or the newly completed one.

Part 2 - Technical concept and behavior

An object put becomes visible as a complete object after success, but a multi-object workflow is not automatically atomic. Immutable object names avoid destructive overwrite. Publishing a manifest after verification establishes an application-level commit point, especially when versioning preserves manifest history.

Part 3 - Hints and Helper

Upload to a unique final artifact name, not a shared temporary name. Verify size and hash, write the manifest last, and simulate failure before manifest publication to prove readers continue using the previous checkpoint.

Part 4 - Python code skeleton

```
import json
from io import BytesIO
from minio import Minio
```

```
def E05(client, bucket_name, artifact_name, artifact_path, pointer_name):
    artifact_result = client.fput_object(...)
    stored = client.stat_object(...)
    digest = ...
    if ...:
        raise ValueError(...)
    pointer = {
        "object_name": artifact_name,
        "version_id": artifact_result.version_id,
        "sha256": digest,
    }
    payload = json.dumps(pointer).encode()
    pointer_result = client.put_object(
        bucket_name, pointer_name, BytesIO(payload), len(payload),
        content_type="application/json",
    )
    return artifact_result, pointer_result, pointer

def main():
    client = Minio(...)
    print(E05(...))

if __name__ == "__main__":
    main()
```

8.3.7 Exercise 06 - Compare Local vs Object Storage IO

Part 1 - Why this matters

Local storage and object storage serve different goals. Local disks can provide low-latency access on one machine, while object storage provides network sharing, centralized access, and storage-system durability features.

A fair experiment separates first-time transfer from repeated local reads and compares equivalent bytes. The result should explain tradeoffs instead of declaring one technology universally superior.

Part 2 - Technical concept and behavior

The benchmark measures local read time, object download time, and optionally download-then-read time. Throughput equals bytes divided by elapsed seconds. Cache state, network distance, concurrency, and integrity hashing must be recorded as experimental conditions.

Part 3 - Hints and Helper

Use the same checkpoint bytes, perform warm-up and repeated trials, calculate hashes outside or inside timing consistently, report median and raw results, and state what object-storage benefits are not represented by speed alone.

Part 4 - Python code skeleton

```
from statistics import median
from time import perf_counter
from minio import Minio

def timed(operation, repetitions):
    values = []
    for _ in range(repetitions):
        started = perf_counter()
        operation()
        values.append(perf_counter() - started)
    return values

def E06(client, bucket_name, object_name, local_path, repetitions):
    local_trials = timed(...)
    object_trials = timed(...)
    integrity = ...
    return {
        "local": local_trials,
        "object": object_trials,
        "local_median": median(local_trials),
        "object_median": median(object_trials),
        "integrity": integrity,
    }

def main():
    client = Minio(...)
    print(E06(...))

if __name__ == "__main__":
    main()
```

8.3.8 Exercise 07 - Garbage Collection of Old Checkpoints

Part 1 - Why this matters

Frequent checkpointing can exhaust storage, especially when every overwrite creates another version. Garbage collection must reclaim unnecessary artifacts while preserving the latest checkpoint, best checkpoint, protected milestones, and anything referenced by active manifests.

Deletion should be planned from metadata before it is executed. A dry-run report lets the reader verify exact object names and versions and prevents a broad prefix from becoming an uncontrolled destructive operation.

Part 2 - Technical concept and behavior

A retention policy can keep the newest k checkpoints, selected epochs, and manifest-referenced artifacts. Versioned buckets require decisions about specific versions and delete markers. Listing, policy evaluation, and removal should remain separate phases.

Part 3 - Hints and Helper

List one isolated run prefix including versions, build a protected set, sort candidates by trusted metadata, return a dry-run deletion plan, and require explicit execution input before deleting exact name-version pairs.

Part 4 - Python code skeleton

```
from minio import Minio

def build_plan(objects, keep_latest, protected_versions):
    ordered = sorted(objects, key=..., reverse=True)
    protected = set(...)
    candidates = [item for item in ordered if ...]
    return protected, candidates

def E07(client, bucket_name, prefix, keep_latest, execute=False):
    objects = list(client.list_objects(
        bucket_name,
        prefix=prefix,
        recursive=True,
        include_version=True,
    ))
    protected, candidates = build_plan(...)
    if execute:
        for item in candidates:
            client.remove_object(
                bucket_name,
                item.object_name,
                version_id=item.version_id,
            )
    return protected, candidates, execute

def main():
    client = Minio(...)
    print(E07(...))

if __name__ == "__main__":
    main()
```

8.3.9 Exercise 08 - Integrity Validation with Hashes

Part 1 - Why this matters

Hashes provide content identity across local files, uploaded objects, downloads, and manifests. They help detect corruption, wrong versions, and accidental replacement even when names and sizes appear correct.

Trust requires an independent expected digest. Recalculating a hash only after corruption and then storing that new value would verify the corrupted bytes against themselves rather than against the intended artifact.

Part 2 - Technical concept and behavior

SHA-256 processes bytes incrementally and returns a fixed-length digest. The expected digest can be stored in a signed or otherwise trusted manifest and optionally repeated as object metadata. Verification streams the object response and must close and release the HTTP connection.

Part 3 - Hints and Helper

Hash the source before upload, store the expected value, stream the object with `get_object()`, update the digest by chunks, and use `finally` to close and release the response.

Part 4 - Python code skeleton

```
import hashlib
from minio import Minio

def E08(client, bucket_name, object_name, expected_hash, version_id=None):
    response = client.get_object(
        bucket_name,
        object_name,
        version_id=...,
    )
    digest = hashlib.sha256()
    total_bytes = 0
    try:
        for chunk in response.stream(...):
            digest.update(...)
            total_bytes += len(chunk)
    finally:
        response.close()
        response.release_conn()
    actual_hash = digest.hexdigest()
    return actual_hash, total_bytes, actual_hash == expected_hash

def main():
    client = Minio(...)
    print(E08(...))

if __name__ == "__main__":
    main()
```

8.3.10 Exercise 09 - Training Pipeline Object Integration

Part 1 - Why this matters

A training pipeline must coordinate local computation with durable artifact publication without allowing storage failures to corrupt model state or falsely mark an incomplete run as successful. Storage operations need clear boundaries, retries, and verification.

The final challenge connects dataset manifests, periodic checkpoints, best-model selection, resume logic, and final publication. Model code should depend on a small artifact interface instead of calling object-storage operations from unrelated layers.

Part 2 - Technical concept and behavior

The pipeline resolves an immutable dataset version, trains locally, publishes verified checkpoint artifacts, and updates manifests only after successful upload. Resume reads one trusted pointer and exact version. Storage errors are recorded and handled according to policy rather than silently ignored.

Part 3 - Hints and Helper

Create an artifact-store helper with upload, download, verify, and publish methods. Keep credentials outside checkpoints, use run-specific prefixes, save at controlled intervals, restore the best or latest pointer explicitly, and return a complete artifact history.

Part 4 - Python code skeleton

```
import torch
from minio import Minio

class ArtifactStore:
    def __init__(self, client, bucket_name, run_prefix):
        self.client = client
        self.bucket_name = bucket_name
        self.run_prefix = run_prefix

    def upload_verified(self, local_path, object_name, metadata):
        ...

    def publish_pointer(self, pointer_name, artifact):
        ...

    def download_verified(self, artifact, destination):
        ...

def E09(model, optimizer, data_loader, artifact_store, epochs, save_every):
    history = []
    best_artifact = ...
    for epoch in range(epochs):
        training_result = ...
        if ...:
            checkpoint_path = ...
            artifact = artifact_store.upload_verified(...)
            artifact_store.publish_pointer(...)
            history.append(artifact)
        if ...:
            best_artifact = ...
    return model, history, best_artifact
```

```
def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

8.4 OB02 - Buckets & Access Structure

AI Safety / Ethics / Alignment

8.4.1 Exercise 00 - Bucket Policy Configuration

Part 1 - Why this matters

A bucket policy defines which actions are allowed or denied for selected resources and identities. A small mistake can expose private artifacts or block a training pipeline, so policy configuration must be treated as security-sensitive code.

This exercise applies a narrowly scoped test policy and verifies it through an identity that does not possess administrator privileges. Administrator success cannot prove that the application policy works.

Part 2 - Technical concept and behavior

S3-compatible policies are JSON documents containing version, statements, effects, actions, resources, principals, and optional conditions. Explicit deny takes precedence over allow. Bucket and object actions may require different resource forms.

Part 3 - Hints and Helper

Generate JSON with Python's `json` module, restrict resources to one disposable bucket and prefix, validate the serialized document, apply it with `set_bucket_policy()`, and retrieve it for comparison.

Part 4 - Python code skeleton

```
import json
from minio import Minio

def E00(admin_client, bucket_name, principal):
    policy = {
        "Version": ...,
        "Statement": [
            {
                "Effect": ...,
                "Principal": ...,
                "Action": [...],
                "Resource": [...],
            }
        ],
```

```
    }
    policy_json = json.dumps(policy)
    admin_client.set_bucket_policy(bucket_name, policy_json)
    stored_policy = json.loads(admin_client.get_bucket_policy(bucket_name))
    return policy, stored_policy, policy == stored_policy

def main():
    admin_client = Minio(...)
    print(E00(...))

if __name__ == "__main__":
    main()
```

8.4.2 Exercise 01 - Read-Only vs Read-Write Policy

Part 1 - Why this matters

Inference workers may need to download model weights without permission to replace or delete them, while training publishers may need controlled write access. Giving both identities full access violates least privilege and increases the impact of credential compromise.

The reader will test permissions as behavior, not only inspect policy JSON. Each identity must pass allowed operations and fail expected denied operations on the exact resources in scope.

Part 2 - Technical concept and behavior

A read-only policy permits selected list, stat, and get actions. A read-write policy adds controlled put and possibly delete actions. Bucket listing and object retrieval are different permissions, and write permission should not automatically imply administrative policy control.

Part 3 - Hints and Helper

Create two non-admin test clients. Build an operation matrix for list, get, put, and delete under one prefix. Catch access-denied responses separately from missing-object or connectivity failures.

Part 4 - Python code skeleton

```
from minio import Minio
from minio.error import S3Error

def attempt(label, operation):
    try:
        result = operation()
        return {"operation": label, "allowed": True, "result": result}
    except S3Error as error:
        return {"operation": label, "allowed": False, "code": error.code}

def E01(read_client, write_client, bucket_name, object_name, source_path):
    tests = {
```

```
        "read_get": lambda: read_client.stat_object(...),
        "read_put": lambda: read_client.fput_object(...),
        "write_get": lambda: write_client.stat_object(...),
        "write_put": lambda: write_client.fput_object(...),
    }
    outcomes = {name: attempt(name, operation) for name, operation in tests.items()}
    expected = {...}
    return outcomes, ...

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

8.4.3 Exercise 02 - User and Access Key Creation

Part 1 - Why this matters

Applications should use dedicated identities instead of administrator credentials. Separate access keys allow permissions, rotation, revocation, and audit to follow the application or service that actually performs each operation.

Identity creation is administrative and is not provided by ordinary object data operations. Python may orchestrate a supported administrative CLI in the lab, but secrets must not appear in logs, exceptions, repositories, or returned exercise values.

Part 2 - Technical concept and behavior

An access key identifies the principal and a secret key authenticates signed requests. Policies are attached through administrative configuration. The new credentials must be tested with a separate MinIO client whose permissions are limited to the intended resources.

Part 3 - Hints and Helper

Generate a strong temporary secret, pass administrative configuration only through the supported mechanism for the installed release, redact captured output, attach a least-privilege policy, test the client, and remove the identity after the lab.

Part 4 - Python code skeleton

```
import os
import secrets
import subprocess
from minio import Minio

def E02(alias, access_key, policy_name):
    secret_key = secrets.token_urlsafe(...)
    environment = os.environ.copy()
    environment[...] = secret_key
    create_command = ["mc", "admin", "user", "add", alias, access_key, ...]
```

```
    attach_command = ["mc", "admin", "policy", "attach", ...]
    subprocess.run(create_command, env=environment, check=True,
                   stdout=subprocess.DEVNULL, stderr=subprocess.PIPE)
    subprocess.run(attach_command, check=True, capture_output=True)
    application_client = Minio(..., access_key=access_key,
                               secret_key=secret_key, secure=...)
    verification = ...
    return {"access_key": access_key, "verified": verification}

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

8.4.4 Exercise 03 - Temporary Access using Presigned URL

Part 1 - Why this matters

A presigned URL lets a client perform one time-limited object operation without receiving the storage account's secret key. It is useful for controlled downloads or uploads from browsers and external workers.

The URL behaves like a bearer credential until it expires. Anyone who obtains it may use its granted operation, so expiration, object scope, transport security, and log handling matter.

Part 2 - Technical concept and behavior

The SDK signs a request for a specific bucket, object name, HTTP operation, and expiration. The server validates the signature when the URL is used. Revoking the underlying credential or changing object access can affect validity, but the design should assume the URL remains usable for its intended lifetime.

Part 3 - Hints and Helper

Generate a short-lived GET URL for one disposable object. Use Python's HTTP client to download it without MinIO credentials, verify content hash, and confirm that an altered object name or expired URL fails.

Part 4 - Python code skeleton

```
from datetime import timedelta
import hashlib
import urllib.request
from minio import Minio

def E03(client, bucket_name, object_name, expiration):
    if expiration > timedelta(...):
        raise ValueError(...)
    url = client.presigned_get_object(
        bucket_name,
```

```
        object_name,
        expires=...,
    )
    with urllib.request.urlopen(url, timeout=...) as response:
        data = response.read()
    digest = hashlib.sha256(data).hexdigest()
    return url, len(data), digest

def main():
    client = Minio(...)
    print(E03(...))

if __name__ == "__main__":
    main()
```

8.4.5 Exercise 04 - Private vs Public Object Access

Part 1 - Why this matters

Model weights, private datasets, and checkpoints should normally require authenticated access. Public access is suitable only for artifacts intentionally released to everyone and can expose sensitive bytes instantly when a policy is too broad.

Testing with administrator credentials cannot reveal anonymous behavior. This exercise compares authenticated and unsigned HTTP requests and restores private policy after the isolated experiment.

Part 2 - Technical concept and behavior

Private objects require a signed request from an allowed identity. A bucket policy can permit anonymous GET for a selected resource or prefix. Public read does not need to imply public listing, write, delete, or policy modification.

Part 3 - Hints and Helper

Use one harmless test object. Apply a policy granting only anonymous object retrieval for its prefix, test unsigned GET and unsigned PUT separately, then remove the policy and verify GET is denied again.

Part 4 - Python code skeleton

```
import json
import urllib.error
import urllib.request
from minio import Minio

def anonymous_status(url):
    try:
        with urllib.request.urlopen(url, timeout=...) as response:
            return response.status
    except urllib.error.HTTPError as error:
```

```
        return error.code

def E04(admin_client, bucket_name, object_name, public_url):
    private_status = anonymous_status(...)
    public_policy = {...}
    admin_client.set_bucket_policy(bucket_name, json.dumps(public_policy))
    public_status = anonymous_status(...)
    admin_client.delete_bucket_policy(bucket_name)
    restored_status = anonymous_status(...)
    return private_status, public_status, restored_status

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

8.4.6 Exercise 05 - Folder-like Access Control Design

Part 1 - Why this matters

Object names can use prefixes such as team, project, or tenant to organize access without creating real folders. Correct prefix policies can let one identity operate only inside its namespace while sharing the same bucket infrastructure.

Prefix matching must be exact. Ambiguous names, missing separators, or permission to list the entire bucket can reveal objects outside the intended area even when object retrieval is restricted.

Part 2 - Technical concept and behavior

Policies can restrict object resources to an ARN ending with a controlled prefix and wildcard. Bucket-list actions can use conditions on requested prefixes. Naming validation in Python should prevent callers from escaping or impersonating another namespace.

Part 3 - Hints and Helper

Design prefixes as `tenants/<id>/...`, validate identifier characters, create separate list and object statements, and test own-prefix read/write against neighboring and similarly named prefixes.

Part 4 - Python code skeleton

```
import json
import re

def E05(bucket_name, tenant_id, actions):
    if re.fullmatch(..., tenant_id) is None:
        raise ValueError(...)
    prefix = f"tenants/{tenant_id}/"
    policy = {
```

```
        "Version": ...,
        "Statement": [
            {
                "Effect": "Allow",
                "Action": [...],
                "Resource": [f"arn:aws:s3:::{bucket_name}/{prefix}*"],
            },
            {
                "Effect": "Allow",
                "Action": ["s3:ListBucket"],
                "Resource": [f"arn:aws:s3:::{bucket_name}"],
                "Condition": {"StringLike": {"s3:prefix": [...]}},
            },
        ],
    }
    return prefix, json.dumps(policy)

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

8.4.7 Exercise 06 - Access Denial Debugging Challenge

Part 1 - Why this matters

An access-denied response can result from identity policy, bucket policy, explicit deny, incorrect resource name, missing list permission, expired credentials, clock problems, or a request signed for the wrong endpoint. Guessing by granting broader access creates new security risk.

The reader will debug from evidence while preserving least privilege. The objective is to identify the failing action and resource, make the smallest correction, and prove unrelated operations remain denied.

Part 2 - Technical concept and behavior

S3 errors include a code, message, resource, request ID, and host ID where available. Authorization evaluates all relevant policy statements and conditions. A successful administrator retry proves object existence but does not prove the application identity is configured correctly.

Part 3 - Hints and Helper

Capture sanitized error fields, confirm endpoint and identity, reduce the operation to stat, list, get, or put, compare exact resource ARNs and prefixes, and execute a positive and negative regression test after correction.

Part 4 - Python code skeleton

```
from minio.error import S3Error
```

```
def diagnose(operation, expected_action, expected_resource):
    try:
        result = operation()
        return {"status": "allowed", "result": result}
    except S3Error as error:
        evidence = {
            "code": error.code,
            "message": error.message,
            "resource": error.resource,
            "request_id": error.request_id,
        }
        hypotheses = [...]
        return {"status": "denied", "evidence": evidence,
                "hypotheses": hypotheses}

def E06(allowed_operation, denied_operation):
    before = diagnose(...)
    correction = ...
    after_allowed = diagnose(...)
    after_denied = diagnose(...)
    return before, correction, after_allowed, after_denied

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

8.4.8 Exercise 07 - Multi-Bucket Permission Strategy

Part 1 - Why this matters

One pipeline may read datasets, write checkpoints, and publish final models in different buckets. Using identical broad permissions across all three weakens isolation and allows a compromised worker to modify data it should only consume.

The policy should reflect directional data flow. Training identities may read immutable datasets, write only their run prefix, and have no authority to change bucket policies or delete published models.

Part 2 - Technical concept and behavior

A policy can contain several statements, each scoped to actions and bucket or object resources. Bucket-level list permission is distinct from object actions. Explicit deny statements may protect particularly sensitive namespaces in addition to omitted allows.

Part 3 - Hints and Helper

Create an access matrix before JSON. Map each role to dataset, checkpoint, and model operations, generate statements from validated configuration, and test every cell with separate non-admin credentials.

Part 4 - Python code skeleton

```
import json

def statement(effect, actions, resources, conditions=None):
    item = {
        "Effect": effect,
        "Action": actions,
        "Resource": resources,
    }
    if conditions:
        item["Condition"] = conditions
    return item

def E07(dataset_bucket, checkpoint_bucket, model_bucket, run_id):
    access_matrix = {
        "dataset": {"read": True, "write": False, "delete": False},
        "checkpoint": {"read": True, "write": True, "delete": ...},
        "model": {"read": ..., "write": ..., "delete": False},
    }
    statements = [
        statement(...),
        statement(...),
        statement(...),
    ]
    policy = {"Version": ..., "Statement": statements}
    return access_matrix, json.dumps(policy)

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

8.4.9 Exercise 08 - Secure Credential Rotation

Part 1 - Why this matters

Credentials must be rotated when they age, leak, or change ownership. Replacing the only active key before clients adopt the new one can cause an outage, while leaving the old key enabled indefinitely defeats the purpose of rotation.

Safe rotation overlaps credentials briefly: create the new credential, attach equivalent least privilege, test it, update the application's secret source, confirm use, revoke the old credential, and verify the old key fails.

Part 2 - Technical concept and behavior

Access-key lifecycle is an administrative operation. Applications should obtain active secrets from a protected secret source rather than hard-coded files. Rotation evidence should record key identifiers and timestamps, never secret values.

Part 3 - Hints and Helper

Use an isolated identity, never print the new secret, and implement the workflow as explicit phases with rollback guidance. Test one allowed and one denied action before revoking the old key.

Part 4 - Python code skeleton

```
from dataclasses import dataclass
from datetime import datetime, timezone

@dataclass
class RotationState:
    old_access_key: str
    new_access_key: str
    phase: str
    started_at: object

def E08(admin_create, attach_policy, update_secret_store,
        verify_client, revoke_old, old_access_key):
    new_access_key, new_secret = admin_create(...)
    state = RotationState(old_access_key, new_access_key,
                          "created", datetime.now(timezone.utc))
    attach_policy(...)
    verification = verify_client(...)
    if not verification:
        raise RuntimeError(...)
    update_secret_store(...)
    confirmation = ...
    if confirmation:
        revoke_old(...)
        state.phase = "completed"
    return state, verification

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

8.4.10 Exercise 09 - Access Control Audit Scenario

Part 1 - Why this matters

Access control drifts as identities, buckets, prefixes, and pipeline responsibilities change. An audit compares intended permissions with observed behavior and identifies excessive access, missing controls, stale credentials, and public exposure.

The final challenge produces evidence without revealing secrets. It treats denied tests as expected outcomes when the policy requires denial and separates policy findings from connectivity or missing-object errors.

Part 2 - Technical concept and behavior

An audit inventory includes principals, policies, bucket policies, credential age, public access, and an operation matrix. Findings should include severity, affected resource, evidence, expected state, and a minimal remediation. Audit code should be read-only except inside disposable canary prefixes.

Part 3 - Hints and Helper

Define the expected matrix first, use harmless canary objects for write and delete tests, redact secrets and presigned URLs, detect unexpected allows and denies, and return a deterministic report suitable for later comparison.

Part 4 - Python code skeleton

```
from datetime import datetime, timezone

def test_permission(identity, operation_name, operation):
    try:
        operation()
        observed = "allow"
        evidence = ...
    except Exception as error:
        observed = ...
        evidence = ...
    return {"identity": identity, "operation": operation_name,
            "observed": observed, "evidence": evidence}

def E09(expected_matrix, test_operations, credential_metadata):
    observations = [
        test_permission(identity, operation_name, operation)
        for identity, operations in test_operations.items()
        for operation_name, operation in operations.items()
    ]
    findings = [...]
    report = {
        "generated_at": datetime.now(timezone.utc),
        "observations": observations,
        "findings": findings,
        "credential_findings": ...,
    }
    return report
```

```
def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

8.5 OB03 - Distributed Mode & Fault Tolerance

AI Infrastructure and Hardware

8.5.1 Exercise 00 - Start MinIO in Distributed Mode

Part 1 - Why this matters

A distributed object-storage deployment uses several drives or nodes so data availability does not depend on one local disk or process. The topology must be planned before failure testing because resilience comes from failure domains, parity, and quorum.

This exercise starts only an isolated laboratory deployment. Python verifies the resulting service and topology-facing health, while server or container configuration remains an infrastructure responsibility.

Part 2 - Technical concept and behavior

MinIO distributes erasure-coded data and parity shards across an erasure set. A multi-node endpoint list and persistent drives define the deployment. Starting several processes that all use the same unsafe local path does not create meaningful hardware fault tolerance.

Part 3 - Hints and Helper

Use disposable volumes, equal lab node configuration, unique hostnames, synchronized clocks, and protected credentials. Start through a supported orchestration method, wait with a deadline, then verify authenticated bucket and object operations from Python.

Part 4 - Python code skeleton

```
import subprocess
import time
from minio import Minio

def E00(start_command, client_factory, deadline_seconds):
    completed = subprocess.run(start_command, check=...,
                               capture_output=True, text=True)
    deadline = time.monotonic() + deadline_seconds
    observations = []
    while time.monotonic() < deadline:
        try:
            client = client_factory()
            buckets = client.list_buckets()
            observations.append(...)
```

```
            return completed.returncode, observations
        except Exception as error:
            observations.append(...)
            ...
    raise TimeoutError(...)

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

8.5.2 Exercise 01 - Node Failure Simulation

Part 1 - Why this matters

Fault tolerance should be demonstrated through controlled loss within the topology's permitted failure budget. Stopping too many nodes can remove quorum and turn a learning exercise into data loss.

The reader will establish a healthy baseline, stop exactly one designated lab node, test read and write behavior, restore it, and verify the same object bytes again.

Part 2 - Technical concept and behavior

Availability during node loss depends on the erasure-set layout, parity, and read or write quorum. Losing one process does not produce one universal result for every topology. Client retries and load-balanced endpoints also influence observed behavior.

Part 3 - Hints and Helper

Use disposable test objects with known hashes. Resolve the exact container or service target before stopping it, enforce command timeouts, never stop additional nodes after an unexpected result, and restore the node in a `finally` block.

Part 4 - Python code skeleton

```
import subprocess

def E01(client, object_reference, stop_command, start_command):
    baseline = ...
    stopped = False
    try:
        subprocess.run(stop_command, check=True, timeout=...)
        stopped = True
        read_during_failure = ...
        write_during_failure = ...
        integrity_during_failure = ...
    finally:
        if stopped:
            subprocess.run(start_command, check=True, timeout=...)
```

```
    recovered = ...
    return {
        "baseline": baseline,
        "read_during_failure": read_during_failure,
        "write_during_failure": write_during_failure,
        "integrity": integrity_during_failure,
        "recovered": recovered,
    }

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

8.5.3 Exercise 02 - Data Replication Verification

Part 1 - Why this matters

In a distributed erasure-coded deployment, an object is not necessarily stored as one complete copy on every node. Assuming full per-node replicas leads to incorrect tests and dangerous recovery decisions.

This exercise verifies protection through client-visible availability and integrity under an allowed failure. If separate bucket or site replication is configured, that is a different mechanism and requires an additional destination-level check.

Part 2 - Technical concept and behavior

Erasure coding splits bytes into data and parity shards across drives. The service reconstructs the object when enough shards remain. Replication copies object versions to another bucket or site and has independent configuration, versioning, and lag considerations.

Part 3 - Hints and Helper

Record which protection mechanism is under test. For erasure coding, verify one object's hash before, during, and after a permitted node loss. For configured replication, compare exact object name, version, size, metadata, and digest at the destination.

Part 4 - Python code skeleton

```
def verify_object(client, bucket_name, object_name, expected_hash,
                  version_id=None):
    stored = client.stat_object(
        bucket_name, object_name, version_id=version_id,
    )
    actual_hash = ...
    return {
        "size": stored.size,
        "version_id": stored.version_id,
        "hash": actual_hash,
```

```
        "valid": actual_hash == expected_hash,
    }

def E02(source_client, destination_client, reference, expected_hash):
    source = verify_object(...)
    destination = verify_object(...) if destination_client else None
    protected_during_failure = ...
    return source, destination, protected_during_failure

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

8.5.4 Exercise 03 - Healing Process Observation

Part 1 - Why this matters

After a failed drive or node returns or is replaced, missing or damaged shards may need healing. Service availability before healing completes does not mean redundancy has already returned to its original state.

The reader will observe healing without repeatedly starting destructive repair operations. Progress, remaining work, object integrity, and final redundancy evidence belong in the result.

Part 2 - Technical concept and behavior

Healing reconstructs damaged or missing object shards when enough intact data and parity remain. Current MinIO behavior and administrative commands depend on the installed release. Observation should use supported health, admin, and object-inspection interfaces for that deployment.

Part 3 - Hints and Helper

Restore the failed component first. Invoke only documented read or status commands through Python subprocess, parse structured output when available, poll with a bounded deadline, and verify known object hashes throughout the process.

Part 4 - Python code skeleton

```
import json
import subprocess
import time

def E03(status_command, integrity_check, deadline_seconds):
    deadline = time.monotonic() + deadline_seconds
    history = []
    while time.monotonic() < deadline:
        completed = subprocess.run(
            status_command,
```

```
            check=True,
            capture_output=True,
            text=True,
            timeout=...,
        )
        status = ...
        integrity = integrity_check()
        history.append({"status": status, "integrity": integrity})
        if ...:
            return history
        ...
    raise TimeoutError(...)

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

8.5.5 Exercise 04 - Disk Failure Recovery Scenario

Part 1 - Why this matters

Drive failure tests whether data protection extends below the process level. A careless simulation can remove the wrong volume or exceed parity, so the exact disposable drive and recovery procedure must be resolved before any mutation.

Recovery includes more than making the server process run. The replacement storage must join correctly, healing must complete, and previously stored objects must retain their expected bytes and metadata.

Part 2 - Technical concept and behavior

A failed drive removes some shards from its erasure set. Availability depends on remaining shards and quorum. Replacement should use supported procedures and appropriate hardware or lab-volume characteristics; copying internal MinIO storage files manually is not a valid object-level recovery method.

Part 3 - Hints and Helper

Use a disposable mounted volume and external backups. Stop the exact lab node before detaching its test drive, replace only that explicit target, restore the service, observe healing, and compare a fixed integrity manifest before and after.

Part 4 - Python code skeleton

```
from pathlib import Path

def E04(topology, failed_drive_id, manifest, recovery_actions):
    target = topology.resolve_drive(...)
    if not target.is_disposable or target.path in {...}:
```

```
        raise RuntimeError(...)
    baseline = {item.name: ... for item in manifest}
    confirmation = ...
    if not confirmation:
        return ...
    recovery_actions.stop_node(target.node)
    recovery_actions.detach_test_drive(target)
    degraded = ...
    recovery_actions.attach_replacement(target)
    recovery_actions.start_node(target.node)
    healing = ...
    final_validation = {item.name: ... for item in manifest}
    return baseline, degraded, healing, final_validation

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

8.5.6 Exercise 05 - Erasure Coding Concept Experiment

Part 1 - Why this matters

Erasure coding protects data by combining data shards with parity shards. It can tolerate selected shard losses without storing a complete copy on every drive, but higher protection reduces usable capacity and affects quorum.

The reader will model these tradeoffs in Python before touching a cluster. A conceptual simulation makes failure limits visible and prevents parity settings from becoming unexplained configuration values.

Part 2 - Technical concept and behavior

For an erasure set of N shards with P parity shards, the data-shard count is

$$D = N - P.$$

The approximate storage efficiency is D/N. Actual read and write availability follows the deployment's documented quorum rules and must not be inferred from this ratio alone.

Part 3 - Hints and Helper

Validate that shard and parity counts describe a supported conceptual case. Enumerate combinations of missing shards, classify whether loss remains within parity, calculate usable ratio, and compare the model with observed lab behavior.

Part 4 - Python code skeleton

```
from itertools import combinations
```

```
def E05(total_shards, parity_shards):
    if ...:
        raise ValueError(...)
    data_shards = ...
    storage_efficiency = ...
    shard_ids = list(range(total_shards))
    scenarios = []
    for failed_count in range(total_shards + 1):
        for failed in combinations(shard_ids, failed_count):
            theoretically_recoverable = ...
            scenarios.append({
                "failed": failed,
                "recoverable": theoretically_recoverable,
            })
    return data_shards, storage_efficiency, scenarios

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

8.5.7 Exercise 06 - Performance Impact of Replication

Part 1 - Why this matters

Additional protection can increase the work required before or after an upload is considered complete. Replication to another destination consumes bandwidth and storage, while stronger synchronous acknowledgment or parity settings may affect client-visible latency differently.

The experiment must name the mechanism being measured. Erasure coding inside one deployment and asynchronous bucket or site replication are not interchangeable, and their performance results answer different questions.

Part 2 - Technical concept and behavior

Client latency measures how long the originating operation takes. Replication completion lag measures when the same protected version becomes available at the destination. Throughput, destination lag, CPU, network, and integrity can all be relevant; one elapsed time cannot describe the entire cost.

Part 3 - Hints and Helper

Compare equivalent objects with protection disabled and enabled only in a disposable environment. Repeat trials, record source acknowledgment separately from destination verification, and validate exact version hash at both locations.

Part 4 - Python code skeleton

```
from statistics import median
from time import perf_counter, monotonic
```

```
def run_trial(source_client, destination_client, artifact, deadline):
    started = perf_counter()
    result = source_client.fput_object(...)
    source_latency = perf_counter() - started
    replication_started = monotonic()
    destination_version = ...
    while monotonic() < deadline:
        destination_version = ...
        if ...:
            break
        ...
    replication_lag = ...
    integrity = ...
    return source_latency, replication_lag, integrity

def E06(configurations, repetitions):
    results = {}
    for configuration in configurations:
        trials = [run_trial(...) for _ in range(repetitions)]
        results[configuration.name] = ...
    return results

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

8.5.8 Exercise 07 - High Availability Configuration

Part 1 - Why this matters

Storage can remain healthy while applications experience an outage if every client uses one failed endpoint. High availability therefore requires both resilient storage nodes and a client-facing endpoint strategy with health checks and failover.

Configuration must avoid hidden single points of failure in load balancing, DNS, credentials, networking, and monitoring. Availability claims should be tested against defined failures rather than assumed from node count.

Part 2 - Technical concept and behavior

A highly available design distributes failure domains, preserves read and write quorum, and exposes healthy endpoints through supported routing. Clients use retry and timeout policies but must not retry non-idempotent workflows blindly. Monitoring distinguishes degraded service from complete failure.

Part 3 - Hints and Helper

Describe nodes, drives, network paths, parity, endpoint routing, and monitoring. Validate configuration in Python, test one permitted failure at a time, and measure both request success and recovery time objectives.

Part 4 - Python code skeleton

```
from dataclasses import dataclass

@dataclass
class HAConfiguration:
    nodes: list
    drives_by_node: dict
    endpoints: list
    parity: int
    health_checks: list

def E07(configuration, probe):
    findings = []
    if len(configuration.nodes) < ...:
        findings.append(...)
    if len(configuration.endpoints) < ...:
        findings.append(...)
    if ...:
        findings.append(...)
    baseline = probe(...)
    failure_scenarios = [...]
    observations = [probe(scenario) for scenario in failure_scenarios]
    return findings, baseline, observations

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

8.5.9 Exercise 08 - Disaster Recovery Backup Strategy

Part 1 - Why this matters

High availability protects service during selected component failures, but it may not protect against site loss, credential compromise, destructive deletion, or a policy error replicated everywhere. Disaster recovery requires an independent recovery copy and a tested restoration process.

A backup that has never been restored is only an assumption. The strategy must define recovery point and time objectives, version retention, encryption, credential separation, and verification of restored bytes.

Part 2 - Technical concept and behavior

Replication can maintain a secondary object destination, while backups may preserve isolated historical copies. Versioning and object locking can reduce some deletion risks but do not replace independent recovery planning. Metadata, policies, manifests, and encryption keys may require separate protection.

Part 3 - Hints and Helper

Build an inventory and integrity manifest, copy to a separately controlled destination, simulate loss only by hiding or deleting disposable source objects, restore to a new bucket, and compare every expected name, version, size, and hash.

Part 4 - Python code skeleton

```
from datetime import datetime, timezone

def build_manifest(client, bucket_name, prefix):
    objects = list(client.list_objects(
        bucket_name, prefix=prefix, recursive=True, include_version=True,
    ))
    return [{
        "name": item.object_name,
        "version_id": item.version_id,
        "size": item.size,
        "sha256": ...,
    } for item in objects if ...]

def E08(source_client, backup_client, restore_client,
        source_bucket, backup_bucket, restore_bucket, prefix):
    manifest = build_manifest(...)
    backup_results = [...]
    restore_results = [...]
    restored_manifest = build_manifest(...)
    comparison = ...
    return {
        "created_at": datetime.now(timezone.utc),
        "manifest": manifest,
        "backup": backup_results,
        "restore": restore_results,
        "valid": comparison,
    }

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

8.5.10 Exercise 09 - End-to-End Fault Tolerance Test

Part 1 - Why this matters

An end-to-end test connects topology assumptions with actual application behavior. It checks whether known objects remain correct, whether permitted reads and writes continue, whether recovery occurs, and whether redundancy returns after healing.

Passing requires more than a green process status. The same artifact hashes, metadata, versions, access controls, and recovery objectives must remain valid throughout the allowed failure scenario.

Part 2 - Technical concept and behavior

The test moves through baseline, fault injection, degraded operation, restoration, healing, and final validation. Every phase records timestamps and observations. A stop condition prevents additional faults when an unexpected quorum or integrity failure occurs.

Part 3 - Hints and Helper

Use disposable infrastructure and externally backed-up test data. Resolve one exact failure target, validate the permitted failure budget, guarantee restoration in `finally`, and produce a report that distinguishes availability, durability, integrity, and recovery time.

Part 4 - Python code skeleton

```
from datetime import datetime, timezone

def record_phase(name, operation):
    started = datetime.now(timezone.utc)
    try:
        result = operation()
        status = "passed" if ... else "failed"
        error = None
    except Exception as exception:
        result = None
        status = "error"
        error = ...
    return {"phase": name, "started": started,
            "status": status, "result": result, "error": error}

def E09(lab, manifest, failure_target, recovery_objectives):
    report = []
    report.append(record_phase("baseline", lambda: ...))
    if report[-1]["status"] != "passed":
        return report
    fault_active = False
    try:
        report.append(record_phase("fault", lambda: lab.fail(failure_target)))
        fault_active = ...
        report.append(record_phase("degraded", lambda: ...))
    finally:
        if fault_active:
            report.append(record_phase("restore", lambda: lab.restore(failure_target)))
    report.append(record_phase("healing", lambda: ...))
```

```
    report.append(record_phase("final_validation", lambda: ...))
    summary = ...
    return report, summary

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

Book 9

Vector Database Lessons (Qdrant)

9.1 Introduction

9.1.1 What will this book teach you

The previous books established three important data layers: PostgreSQL for structured relationships, MongoDB for flexible documents, and MinIO for large binary objects. This book introduces a fourth layer: vector storage and similarity retrieval. Qdrant stores vectors together with identifiers and payload metadata, then searches for points whose vectors are close to a query according to a configured distance measure.

A vector database does not create meaning by itself. An embedding model converts text, images, or other inputs into numerical vectors. The quality, dimension, normalization behavior, training objective, language coverage, and version of that model determine what relationships may appear in the vector space. Qdrant stores and searches the resulting representations, but it cannot repair a poor embedding model or guarantee that vector proximity matches the application's definition of relevance.

Python will remain the main programming language throughout this book. You will connect through `qdrant-client`, create collections, generate embeddings with a selected model, insert points, attach payloads, retrieve by identifier, execute similarity queries, apply metadata filters, configure indexes, and integrate retrieval into a Retrieval-Augmented Generation pipeline. Docker or terminal commands will be used only when the Qdrant service or a distributed learning environment must be started.

Every collection has a vector configuration. The vector size must match the embedding dimension, and the selected distance metric must match the intended model behavior. Changing from one embedding model to another can change vector size and geometry even when both models accept the same text. For this reason, embedding model identity and version must be recorded as part of the collection or ingestion design. Mixing incompatible vectors inside one unnamed vector space produces results that may look valid but have no consistent meaning.

The reader will learn through controlled experiments rather than relying on impressive demonstrations. You will compare semantically related and unrelated examples, inspect nearest-neighbor rankings, test normalization, measure drift after model or data changes, build small relevance judgments, and evaluate whether retrieved results are actually useful. A search re-

quest that returns results is not evidence that those results are correct.

The book also introduces Retrieval-Augmented Generation, commonly called RAG. In a RAG workflow, source documents are divided into chunks, converted to vectors, stored with provenance metadata, retrieved for a user query, and supplied as context to a language model. Retrieval can improve access to external knowledge, but it does not guarantee truthful answers. Poor chunking, missing documents, weak embeddings, incorrect filters, stale indexes, irrelevant context, and model hallucination can still cause failure.

The final chapter studies scalable retrieval. Approximate vector indexes such as HNSW trade exactness, memory, construction cost, and query latency. Payload indexes accelerate filtering and support query planning, while shards and replicas distribute data and availability responsibilities. These features should be configured only after the reader can measure retrieval quality and understand the workload they are attempting to scale.

The exercises provide explanations, hints, and incomplete Python code skeletons rather than finished solutions. Before inserting a point, predict its identifier, dimension, payload, and collection. Before searching, state which results should be relevant and why. Afterward, inspect scores, payloads, ordering, filters, latency, and relevance judgments. This maintains the volume's experimental learning cycle: design, predict, execute, observe, evaluate, and improve.

9.1.2 How Qdrant supports artificial intelligence systems

Qdrant supports retrieval around artificial intelligence models. It can store embeddings for document chunks, products, images, users, memories, or other items and return nearby candidates for semantic search, recommendations, RAG, deduplication, and similar workflows. It does not replace the embedding model, the source-of-truth database, or the object store containing original files.

A practical system may divide responsibility across all four data technologies. PostgreSQL can store users, permissions, plans, and authoritative document records. MongoDB can store chat sessions and flexible processing metadata. MinIO can store original PDFs, datasets, images, weights, and checkpoints. Qdrant can store embeddings and retrieval payloads that refer back to those authoritative records. Stable source identifiers allow retrieved points to be traced to their original document and verified before they are used.

This separation is important because vectors are derived data. When a source document changes, the corresponding chunks and embeddings may become stale. When an embedding model changes, vectors may need to be regenerated into a new collection or named vector configuration. The system must preserve provenance such as source ID, source version, chunk position, embedding model, embedding version, and ingestion timestamp so stale or incompatible points can be found and replaced.

Payload metadata supports filtering and returning context, but it should not become an uncontrolled duplicate of every source record. Store the fields required for retrieval, authorization filtering, provenance, and display, while keeping authoritative business state in the system designed to own it. Security filters must be applied as part of retrieval; filtering sensitive results only after vector search can leak existence or ranking information and waste candidate capacity.

9.1.3 How to approach the exercises

Use a small, understandable dataset before scaling. Include clearly related pairs, difficult negatives, repeated concepts, short and long texts, and examples from the languages or domain the application must support. Write expected relevance judgments before changing search settings. Without a ground truth or at least human judgments, it is impossible to tell whether one embedding model, distance metric, or index configuration improved retrieval quality.

Record the embedding contract for every experiment: model name, exact version or revision, vector dimension, preprocessing, normalization, and whether the model uses different instructions for documents and queries. Generate document and query embeddings according to the model's intended interface. Two vectors with equal length are not automatically compatible merely because their dimensions match.

Use stable point IDs and payload provenance. Batch ingestion should verify that IDs, vectors, and payloads remain aligned, because a shifted list can attach the correct vector to the wrong source text without producing a server error. Count submitted and stored points, retrieve a sample by ID, and compare its payload with the original source before trusting the collection.

Treat similarity scores as metric- and model-specific values. A threshold chosen for cosine similarity cannot be transferred blindly to Euclidean distance, another embedding model, or another domain. Tune thresholds on labeled examples, inspect false positives and false negatives, and report retrieval metrics such as precision at k, recall at k, Mean Reciprocal Rank, or normalized Discounted Cumulative Gain when the experiment supports them.

RAG exercises must preserve source attribution and context budgets. Chunk boundaries should retain enough meaning, overlap should not create excessive duplication, and retrieval should return source identifiers with text. Empty retrieval must produce a controlled response rather than inviting the language model to invent an answer. Retrieved content should also be treated as untrusted input because source documents can contain misleading or malicious instructions.

Scaling experiments require isolated collections and reproducible datasets. Measure ingestion throughput, indexing time, query latency distribution, memory or disk use, and retrieval quality together. Approximate search that is faster but loses necessary relevant results may not be an improvement. Never simulate index corruption by modifying Qdrant's internal storage files; recovery should be tested with supported snapshots, replicas, backups, and restore procedures.

9.1.4 The Chapters (QD00–QD03) included in this book

- QD00 - Embeddings & Semantic Meaning
- QD01 - Similarity Search Mechanics
- QD02 - RAG Integration Workflow
- QD03 - Indexing & Scalable Retrieval

9.1.5 QD00 - Embeddings & Semantic Meaning

This chapter establishes the vector-storage foundation. You will install and start Qdrant, create a collection with the correct vector size and distance metric, generate embeddings with a chosen model, and insert individual and batched points. Payload metadata and stable identifiers will connect each vector to the object it represents.

The later exercises retrieve points, test vector normalization, observe semantic drift, and evaluate embedding quality. Drift may come from a model revision, preprocessing change, language distribution, domain shift, or changes in the source data. By the end, you should understand that an embedding is a model-dependent representation requiring evaluation and provenance, not a permanent universal encoding of meaning.

9.1.6 QD01 - Similarity Search Mechanics

This chapter studies retrieval behavior through cosine similarity, Euclidean distance, top-k selection, thresholds, and payload filters. You will compare metrics using controlled collections rather than changing the interpretation of one collection unpredictably. The current Qdrant Query API will serve as the main search interface where appropriate.

Performance and quality are examined together. You will benchmark latency, inspect rankings, identify ties and surprising neighbors, and debug mismatches involving vector dimension, model choice, normalization, filters, metric direction, or incompatible query preprocessing. The goal is to explain why a result was returned and whether it should be considered relevant.

9.1.7 QD02 - RAG Integration Workflow

This chapter builds a complete retrieval workflow from source text to model context. You will chunk documents, generate and store chunk embeddings, embed a query using the compatible model path, retrieve candidates, rank context, filter by metadata, and respect a context-size budget. Every chunk will retain provenance so it can be traced to its source.

The final exercises handle empty retrieval, test an end-to-end RAG pipeline, and analyze failure modes. Evaluation separates retrieval failure from generation failure: the system may retrieve the wrong context, retrieve the right context that the model ignores, or retrieve nothing and still produce an unsupported answer. A robust design records evidence for each stage.

9.1.8 QD03 - Indexing & Scalable Retrieval

This chapter studies HNSW configuration, index-build cost, query speed, payload indexes, large-scale ingestion, shards, replicas, and consistency. HNSW performs approximate nearest-neighbor search and exposes tradeoffs through construction and search parameters. Payload indexes help filtering and cardinality estimation; they should be created for fields used meaningfully in filters rather than every payload key.

The distributed exercises compare shard count, replication factor, write consistency, and latency monitoring using isolated deployments. More shards or replicas do not automatically improve every workload, and configuration changes must be evaluated against data volume,

failure behavior, query routing, write throughput, and recovery goals.

The chapter concludes with supported recovery testing and a scalable semantic-search design. The final system must define embedding contracts, collection layout, payload schema, indexes, ingestion idempotency, authorization filters, quality metrics, latency objectives, backup or snapshot recovery, and source-of-truth integration. Scaling is considered successful only when retrieval remains relevant, secure, observable, and recoverable.

9.2 QD00 - Embeddings & Semantic Meaning

Data Engineering for AI

9.2.1 Exercise 00 - Install Qdrant and Start Service

Part 1 - Why this matters

Vector operations require a reachable Qdrant service with persistent storage and known network settings. Starting the service is the first boundary between Python code and the vector database that will hold later collections and points.

The exercise verifies more than a running process. Python must connect, receive service information, and list collections within a deadline so configuration or network failures are visible before data ingestion begins.

Part 2 - Technical concept and behavior

Qdrant exposes HTTP and gRPC interfaces and can run locally through a supported package or container deployment. Persistent storage must be mounted deliberately when data should survive restarts. Production authentication and TLS requirements differ from an isolated local lab.

Part 3 - Hints and Helper

Use the current official installation instructions for the chosen environment. Keep URLs and API keys outside source code, configure a finite timeout, and use `get_collections()` as an authenticated readiness check.

Part 4 - Python code skeleton

```
import os
from qdrant_client import QdrantClient

def E00():
    client = QdrantClient(
        url=os.getenv(...),
        api_key=os.getenv(...),
        timeout=...,
    )
    response = client.get_collections()
    collection_names = [item.name for item in response.collections]
```

```
    return collection_names

def main():
    print(E00())

if __name__ == "__main__":
    main()
```

9.2.2 Exercise 01 - Create Collection with Vector Size

Part 1 - Why this matters

A Qdrant collection defines the contract for stored vectors. Its dimension must match the embedding model, and its distance metric determines how query vectors are compared with stored points.

Creating a collection with a guessed size can block ingestion or, worse, encourage padding or truncation that destroys embedding meaning. The reader must obtain the dimension from the actual model output and verify the stored configuration.

Part 2 - Technical concept and behavior

A collection can define one vector space or several named vector spaces. Each dense vector configuration specifies size and distance such as cosine, Euclidean, or dot product. Additional index and storage settings can be added later when the workload is understood.

Part 3 - Hints and Helper

Generate one sample embedding, validate that it is one-dimensional and finite, and use its length. Refuse to overwrite an existing collection silently; inspect its current configuration or use a disposable name.

Part 4 - Python code skeleton

```
import math
from qdrant_client import QdrantClient, models

def E01(client, collection_name, sample_vector):
    if not sample_vector or not all(math.isfinite(value) for value in sample_vector):
        raise ValueError(...)
    vector_size = len(sample_vector)
    existing = {item.name for item in client.get_collections().collections}
    if collection_name in existing:
        raise ValueError(...)
    client.create_collection(
        collection_name=collection_name,
        vectors_config=models.VectorParams(
            size=...,
            distance=models.Distance....,
        ),
    )
```

```
    information = client.get_collection(collection_name)
    return vector_size, information

def main():
    client = QdrantClient(...)
    print(E01(...))

if __name__ == "__main__":
    main()
```

9.2.3 Exercise 02 - Generate Embeddings using Model

Part 1 - Why this matters

The embedding model determines which relationships are represented by vector proximity. Qdrant receives numbers, not raw understanding, so preprocessing and model configuration must be reproducible and compatible between documents and queries.

This exercise generates embeddings and records their contract. The reader will check shape, finite values, and deterministic behavior before any vector is stored.

Part 2 - Technical concept and behavior

An encoder maps a sequence of texts to a matrix with one vector per input. Some models require different instructions or methods for document and query embeddings. Normalization may be performed by the model or application and must be recorded rather than assumed.

Part 3 - Hints and Helper

Use a local or configured embedding model through one wrapper. Preserve input order, reject empty text according to policy, convert outputs to ordinary finite floats, and return model name, revision, dimension, and normalization setting.

Part 4 - Python code skeleton

```
import math

def E02(encoder, texts, purpose):
    if purpose not in {...}:
        raise ValueError(...)
    if ...:
        raise ValueError(...)
    if purpose == "document":
        raw_vectors = encoder.encode_documents(...)
    else:
        raw_vectors = encoder.encode_queries(...)
    vectors = [[float(value) for value in vector] for vector in raw_vectors]
    dimensions = {len(vector) for vector in vectors}
    valid = len(vectors) == len(texts) and len(dimensions) == 1 and all(
        math.isfinite(value) for vector in vectors for value in vector
    )
```

```
    contract = {...}
    return vectors, contract, valid

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

9.2.4 Exercise 03 - Insert Single Vector Point

Part 1 - Why this matters

A Qdrant point combines an identifier, vector, and optional payload. Stable identity allows the application to update or regenerate derived vectors without losing the connection to the source record.

Insertion must validate the embedding contract before sending data. A point accepted under the wrong source identity or payload can produce semantically plausible but factually incorrect retrieval results.

Part 2 - Technical concept and behavior

`upsert()` inserts a new point or replaces data for an existing point ID. A point can use an integer or UUID-compatible identifier. Setting `wait=True` requests completion before the client continues, after which retrieval can verify vector and payload alignment.

Part 3 - Hints and Helper

Validate vector dimension against collection configuration, use a deterministic source-derived ID policy, insert one `PointStruct`, retrieve it with vectors and payloads, and compare every stored field.

Part 4 - Python code skeleton

```
from qdrant_client import QdrantClient, models

def E03(client, collection_name, point_id, vector, payload, expected_size):
    if len(vector) != expected_size:
        raise ValueError(...)
    point = models.PointStruct(
        id=...,
        vector=...,
        payload=...,
    )
    operation = client.upsert(
        collection_name=collection_name,
        points=[point],
        wait=...,
    )
    stored = client.retrieve(
```

```
        collection_name=collection_name,
        ids=[point_id],
        with_vectors=True,
        with_payload=True,
    )
    return operation.status, stored

def main():
    client = QdrantClient(...)
    print(E03(...))

if __name__ == "__main__":
    main()
```

9.2.5 Exercise 04 - Insert Batch of Vectors

Part 1 - Why this matters

Embedding ingestion normally involves many source records. Batch operations reduce request overhead, but they also create a dangerous alignment problem: IDs, vectors, texts, and payloads must refer to the same item at every position.

The reader will validate the complete batch before sending it and verify stored samples afterward. Partial assumptions based only on submitted count are not enough.

Part 2 - Technical concept and behavior

A batch can be represented as a list of point structures or through parallel batch fields. Upsert semantics mean duplicate IDs replace earlier point data. Bounded batches control request size and allow retry or progress tracking.

Part 3 - Hints and Helper

Check equal lengths, unique IDs, fixed vector dimension, finite values, and source ID inside each payload. Split into bounded batches, use `wait=True` for the exercise, count collection points, and retrieve random samples.

Part 4 - Python code skeleton

```
import math
from qdrant_client import QdrantClient, models

def E04(client, collection_name, ids, vectors, payloads, batch_size):
    if not (len(ids) == len(vectors) == len(payloads)):
        raise ValueError(...)
    if len(set(ids)) != len(ids):
        raise ValueError(...)
    operations = []
    for start in range(0, len(ids), batch_size):
        points = [
            models.PointStruct(id=..., vector=..., payload=...)
```

```
            for ... in zip(...)
        ]
        operations.append(client.upsert(
            collection_name=collection_name,
            points=points,
            wait=True,
        ))
    samples = client.retrieve(...)
    return operations, samples

def main():
    client = QdrantClient(...)
    print(E04(...))

if __name__ == "__main__":
    main()
```

9.2.6 Exercise 05 - Attach Payload Metadata to Vectors

Part 1 - Why this matters

Payload metadata gives a retrieved vector provenance and filterable context. Without source ID, version, chunk location, access scope, and embedding information, a nearby point may be impossible to trace or authorize safely.

Payload should remain purposeful. Duplicating complete source systems into Qdrant increases inconsistency, while missing authorization fields can force unsafe post-retrieval filtering.

Part 2 - Technical concept and behavior

Payloads are JSON-like values attached to points and can be inserted with vectors or changed later. Filters can use indexed payload fields. The vector represents similarity; the payload describes and constrains the item represented.

Part 3 - Hints and Helper

Define a required payload contract containing source, source version, tenant, embedding model, and ingestion time. Reject secrets and oversized raw content, update one point's payload, and retrieve it for verification.

Part 4 - Python code skeleton

```
from datetime import datetime, timezone
from qdrant_client import QdrantClient, models

def E05(client, collection_name, point_id, source_metadata):
    payload = {
        "source_id": ...,
        "source_version": ...,
        "tenant_id": ...,
        "embedding_model": ...,
```

```
        "embedded_at": datetime.now(timezone.utc).isoformat(),
    }
    required = {...}
    if ...:
        raise ValueError(...)
    client.set_payload(
        collection_name=collection_name,
        payload=payload,
        points=[point_id],
        wait=True,
    )
    stored = client.retrieve(..., with_payload=True)
    return payload, stored

def main():
    client = QdrantClient(...)
    print(E05(...))

if __name__ == "__main__":
    main()
```

9.2.7 Exercise 06 - Retrieve Vector by ID

Part 1 - Why this matters

Direct retrieval confirms that one known point exists and still matches its source record. It is essential for ingestion verification, debugging, updates, deletion checks, and embedding migration.

Vectors can be large, so applications should request them only when needed. Payload-only retrieval is often sufficient for normal workflows, while vector retrieval belongs in validation or specialized processing.

Part 2 - Technical concept and behavior

`retrieve()` accepts one or more point IDs and independent flags for vectors and payloads. Missing IDs simply do not produce corresponding records, so callers must compare requested and returned identities explicitly.

Part 3 - Hints and Helper

Retrieve an existing and missing ID together, build a map by returned ID, validate dimension and finite values for the existing point, and report missing IDs without confusing them with transport failure.

Part 4 - Python code skeleton

```
import math
from qdrant_client import QdrantClient

def E06(client, collection_name, requested_ids, expected_size):
```

```
    records = client.retrieve(
        collection_name=collection_name,
        ids=...,
        with_vectors=True,
        with_payload=True,
    )
    by_id = {record.id: record for record in records}
    missing = [point_id for point_id in requested_ids if point_id not in by_id]
    validations = {
        point_id: len(record.vector) == expected_size and all(
            math.isfinite(value) for value in record.vector
        )
        for point_id, record in by_id.items()
    }
    return by_id, missing, validations

def main():
    client = QdrantClient(...)
    print(E06(...))

if __name__ == "__main__":
    main()
```

9.2.8 Exercise 07 - Vector Normalization Effect Test

Part 1 - Why this matters

Normalization removes vector magnitude and preserves direction. Its effect depends on the metric and embedding model, so applying it automatically can be redundant, useful, or harmful to the model's intended geometry.

The reader will compare rankings produced by original and normalized representations on controlled examples. The experiment records norms and scores rather than assuming cosine and dot-product behavior are interchangeable in every pipeline.

Part 2 - Technical concept and behavior

For a nonzero vector $\mathbf{x}$, L2 normalization is

$$\widehat{\mathbf{x}} = \frac{\mathbf{x}}{\|\mathbf{x}\|_2}.$$

Cosine similarity depends on direction, while dot product also depends on magnitude unless vectors are normalized. Zero vectors cannot be normalized by ordinary division.

Part 3 - Hints and Helper

Implement normalization with PyTorch, reject zero or near-zero norms, create separate test collections for metric configurations, and compare top-k identities and scores before and after normalization.

Part 4 - Python code skeleton

```
import torch
from qdrant_client import QdrantClient

def normalize(vector, epsilon=1e-12):
    tensor = torch.tensor(vector, dtype=torch.float32)
    norm = torch.linalg.vector_norm(tensor)
    if norm <= epsilon:
        raise ValueError(...)
    return (tensor / norm).tolist(), norm.item()

def E07(client, original_collection, normalized_collection,
        query_vector, limit):
    normalized_query, query_norm = normalize(...)
    original_result = client.query_points(
        collection_name=original_collection,
        query=query_vector,
        limit=limit,
    ).points
    normalized_result = client.query_points(
        collection_name=normalized_collection,
        query=normalized_query,
        limit=limit,
    ).points
    comparison = ...
    return query_norm, original_result, normalized_result, comparison

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

9.2.9 Exercise 08 - Semantic Meaning Drift Observation

Part 1 - Why this matters

Embedding behavior can change when the model revision, preprocessing, language distribution, or domain changes. A retrieval system may therefore drift even when its Qdrant query code remains unchanged.

Drift must be measured on stable probe examples. Comparing vector coordinates alone is insufficient when models use different spaces; ranking and relevance changes reveal the application impact more directly.

Part 2 - Technical concept and behavior

Within a compatible vector space, cosine similarity between old and new vectors can measure representation movement. Across incompatible model spaces, use separate collections and compare neighbor overlap, rank movement, and labeled retrieval metrics. Model changes should not mix vectors in one unnamed space.

Part 3 - Hints and Helper

Keep a fixed probe corpus and query set. Generate embeddings under two controlled configurations, store them separately, calculate same-item similarity only when valid, and report top-k overlap and changed relevance.

Part 4 - Python code skeleton

```
import torch

def cosine(left, right):
    left_tensor = torch.tensor(left, dtype=torch.float32)
    right_tensor = torch.tensor(right, dtype=torch.float32)
    return torch.nn.functional.cosine_similarity(
        left_tensor.unsqueeze(0),
        right_tensor.unsqueeze(0),
    ).item()

def E08(old_encoder, new_encoder, probe_texts, probe_queries, searcher):
    old_vectors = old_encoder.encode_documents(...)
    new_vectors = new_encoder.encode_documents(...)
    coordinate_comparison_allowed = ...
    same_item_scores = [...] if coordinate_comparison_allowed else None
    old_rankings = [searcher(..., configuration="old") for query in probe_queries]
    new_rankings = [searcher(..., configuration="new") for query in probe_queries]
    overlap = ...
    rank_movements = ...
    return same_item_scores, overlap, rank_movements

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

9.2.10 Exercise 09 - Embedding Quality Evaluation Challenge

Part 1 - Why this matters

Embedding quality is task-specific. A visually convincing nearest-neighbor example cannot establish whether the model retrieves the correct documents across languages, topics, difficult negatives, and ambiguous queries.

Evaluation requires query relevance judgments and repeatable metrics. The reader will separate model quality from Qdrant latency and compare configurations using the same corpus and ground truth.

Part 2 - Technical concept and behavior

For a query with relevant set R and retrieved top-k set S_k,

$$\text{Precision@}\,k = \frac{|R \cap S_k|}{k}, \qquad \text{Recall@}\,k = \frac{|R \cap S_k|}{|R|}.$$

Ranking-sensitive measures such as MRR or NDCG capture where relevant items appear.

Part 3 - Hints and Helper

Create judgments before tuning. Include easy, difficult, multilingual, and no-answer queries; run each embedding configuration against its own compatible collection; calculate per-query and aggregate metrics; and inspect false positives and false negatives.

Part 4 - Python code skeleton

```
from statistics import mean

def precision_at_k(retrieved_ids, relevant_ids, k):
    selected = retrieved_ids[:k]
    return ...

def recall_at_k(retrieved_ids, relevant_ids, k):
    if not relevant_ids:
        return ...
    return ...

def reciprocal_rank(retrieved_ids, relevant_ids):
    for rank, point_id in enumerate(retrieved_ids, start=1):
        if point_id in relevant_ids:
            return ...
    return 0.0

def E09(searcher, judgments, k):
    per_query = []
    for query_id, item in judgments.items():
        retrieved_ids = searcher(item["query"], k)
        per_query.append({...})
    aggregate = {
        "precision": mean(...),
        "recall": mean(...),
        "mrr": mean(...),
    }
    return per_query, aggregate

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

9.3 QD01 - Similarity Search Mechanics

AI Evaluation and Experimentation

9.3.1 Exercise 00 - Cosine Similarity Search Query

Part 1 - Why this matters

Cosine-based retrieval compares vector direction and is widely used with text embeddings. It can retrieve semantically related items even when they do not share exact words, provided the embedding model represents the relationship well.

The query must use the same embedding contract as the stored points. A valid vector dimension and successful response do not prove compatibility when preprocessing or model revisions differ.

Part 2 - Technical concept and behavior

A collection configured with cosine distance ranks vectors according to cosine-based similarity behavior. Qdrant's query interface returns scored points ordered by its configured metric. Payload and vectors are optional in the response and should be requested only when needed.

Part 3 - Hints and Helper

Create or use a cosine collection, embed one query with the compatible query encoder, call `query_points()`, and return IDs, scores, and source payloads. Compare results with written expectations.

Part 4 - Python code skeleton

```
from qdrant_client import QdrantClient

def E00(client, collection_name, query_encoder, query_text, limit):
    query_vector = query_encoder.encode_queries([query_text])[0]
    response = client.query_points(
        collection_name=collection_name,
        query=...,
        limit=...,
        with_payload=True,
        with_vectors=False,
    )
    results = [
        {"id": point.id, "score": point.score, "payload": point.payload}
        for point in response.points
    ]
    return query_vector, results

def main():
    client = QdrantClient(...)
    print(E00(...))
```

```
if __name__ == "__main__":
    main()
```

9.3.2 Exercise 01 - Euclidean Distance Search Query

Part 1 - Why this matters

Euclidean distance measures straight-line separation in vector space and is useful when both direction and magnitude have meaning under the selected representation. It should be chosen because it matches the model and task, not simply to produce a different ranking.

The reader will use a collection configured explicitly for Euclidean distance and compare returned neighbors with manually calculated distances on a small dataset.

Part 2 - Technical concept and behavior

For vectors $\mathbf{x}$ and $\mathbf{y}$,

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_i (x_i - y_i)^2}.$$

Smaller mathematical distance means closer vectors. Qdrant's returned score and threshold behavior must be interpreted according to the configured metric and client documentation.

Part 3 - Hints and Helper

Use a separate Euclidean collection containing the same controlled points. Query it, retrieve vectors for the small test, calculate distances with PyTorch, and verify neighbor order rather than assuming score direction.

Part 4 - Python code skeleton

```
import torch
from qdrant_client import QdrantClient

def euclidean(left, right):
    return torch.linalg.vector_norm(
        torch.tensor(left) - torch.tensor(right)
    ).item()

def E01(client, collection_name, query_vector, limit):
    response = client.query_points(
        collection_name=collection_name,
        query=query_vector,
        limit=limit,
        with_vectors=True,
        with_payload=True,
    )
    manual = [
        (point.id, euclidean(query_vector, point.vector))
        for point in response.points
    ]
```

```
    verified_order = ...
    return response.points, manual, verified_order

def main():
    client = QdrantClient(...)
    print(E01(...))

if __name__ == "__main__":
    main()
```

9.3.3 Exercise 02 - Compare Distance Metrics

Part 1 - Why this matters

Distance metrics impose different geometric assumptions and can change which items are considered neighbors. Comparing them reveals whether vector magnitude contributes useful signal or distorts the application's relevance.

Scores from different metrics are not directly comparable by numeric value. The experiment should compare rankings and labeled quality while each collection preserves its own metric contract.

Part 2 - Technical concept and behavior

Cosine emphasizes angular similarity, Euclidean measures geometric separation, and dot product combines direction with magnitude. Normalized vectors can make some ranking relationships more similar, but storage and scoring behavior must still be evaluated explicitly.

Part 3 - Hints and Helper

Insert identical test vectors and IDs into separate metric collections. Run the same query vector, compare top-k IDs with rank overlap and movement, and evaluate each result against the same relevance judgments.

Part 4 - Python code skeleton

```
from qdrant_client import QdrantClient

def rank_map(points):
    return {point.id: rank for rank, point in enumerate(points, start=1)}

def E02(client, collections_by_metric, query_vector, limit, relevant_ids):
    results = {
        metric: client.query_points(
            collection_name=collection_name,
            query=query_vector,
            limit=limit,
        ).points
        for metric, collection_name in collections_by_metric.items()
    }
    rankings = {metric: rank_map(points) for metric, points in results.items()}
```

```
    quality = {...}
    overlap = {...}
    return results, rankings, quality, overlap

def main():
    client = QdrantClient(...)
    print(E02(...))

if __name__ == "__main__":
    main()
```

9.3.4 Exercise 03 - Top-K Nearest Neighbors Retrieval

Part 1 - Why this matters

Top-k retrieval limits the candidate set returned to an application. A small k may miss useful context, while a large k increases latency, noise, and downstream processing.

The correct value depends on evaluation and the next pipeline stage. This exercise compares several limits and checks that smaller result sets are consistent prefixes of a stable larger query under unchanged conditions.

Part 2 - Technical concept and behavior

The query `limit` controls the maximum number of scored points returned. Approximate search may introduce variation under some configurations, while deterministic tie-breaking among equal scores should not be assumed without additional analysis.

Part 3 - Hints and Helper

Validate positive k values and ensure they do not exceed the experiment's intended range. Run several limits with identical query parameters, compare IDs and score order, and calculate precision and recall where judgments exist.

Part 4 - Python code skeleton

```
from qdrant_client import QdrantClient

def E03(client, collection_name, query_vector, k_values):
    if any(k <= 0 for k in k_values):
        raise ValueError(...)
    results = {}
    for k in sorted(set(k_values)):
        response = client.query_points(
            collection_name=collection_name,
            query=query_vector,
            limit=k,
            with_payload=True,
        )
        results[k] = response.points
    largest = results[max(results)]
```

```
    prefix_checks = {
        k: [point.id for point in results[k]] == [...]
        for k in results
    }
    return results, prefix_checks

def main():
    client = QdrantClient(...)
    print(E03(...))

if __name__ == "__main__":
    main()
```

9.3.5 Exercise 04 - Similarity Threshold Filtering

Part 1 - Why this matters

A fixed top-k query returns candidates even when every candidate is weak. A score threshold can reject low-quality neighbors and support an empty-retrieval path instead of forcing irrelevant context into the application.

Thresholds are specific to model, metric, preprocessing, and domain. One convenient number cannot be reused safely across collections without labeled calibration.

Part 2 - Technical concept and behavior

The query interface accepts a `score_threshold`. Whether acceptable values lie above or below a numeric boundary depends on metric semantics, and Qdrant applies the configured metric's threshold behavior. Thresholding trades recall for precision.

Part 3 - Hints and Helper

Collect positive and negative query examples, inspect score distributions, test several thresholds, and calculate accepted true positives, false positives, and false negatives. Preserve an explicit no-result case.

Part 4 - Python code skeleton

```
from qdrant_client import QdrantClient

def E04(client, collection_name, query_vector, thresholds,
        limit, relevant_ids):
    experiments = []
    for threshold in thresholds:
        points = client.query_points(
            collection_name=collection_name,
            query=query_vector,
            score_threshold=threshold,
            limit=limit,
            with_payload=True,
        ).points
```

```
        retrieved_ids = [point.id for point in points]
        experiments.append({
            "threshold": threshold,
            "results": points,
            "precision": ...,
            "recall": ...,
            "empty": len(points) == 0,
        })
    return experiments

def main():
    client = QdrantClient(...)
    print(E04(...))

if __name__ == "__main__":
    main()
```

9.3.6 Exercise 05 - Payload-based Filtering in Search

Part 1 - Why this matters

Semantic similarity alone may retrieve the wrong tenant, language, source version, or document state. Payload filters constrain vector search to records that are eligible for the current request.

Authorization filters must be applied inside retrieval rather than after top-k selection. Post-filtering can leak information and may leave no useful candidates because disallowed points consumed the result limit.

Part 2 - Technical concept and behavior

Qdrant filters combine conditions such as must, should, and must-not. Field conditions can match keyword, numeric range, Boolean, or other supported payload types. Payload indexes improve filtering and cardinality estimation for important fields.

Part 3 - Hints and Helper

Filter by tenant, language, and active source version. Test a valid tenant and a neighboring tenant using the same vector, verify every returned payload, and distinguish an empty eligible set from poor vector similarity.

Part 4 - Python code skeleton

```
from qdrant_client import QdrantClient, models

def E05(client, collection_name, query_vector,
        tenant_id, language, source_version, limit):
    query_filter = models.Filter(
        must=[
            models.FieldCondition(
                key="tenant_id",
```

```
                match=models.MatchValue(value=...),
            ),
            models.FieldCondition(
                key="language",
                match=models.MatchValue(value=...),
            ),
            models.FieldCondition(
                key="source_version",
                match=models.MatchValue(value=...),
            ),
        ]
    )
    points = client.query_points(
        collection_name=collection_name,
        query=query_vector,
        query_filter=query_filter,
        limit=limit,
        with_payload=True,
    ).points
    valid = all(... for point in points)
    return points, valid

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

9.3.7 Exercise 06 - Hybrid Filter + Vector Search

Part 1 - Why this matters

Practical retrieval combines semantic proximity with structured eligibility rules. The vector expresses what is similar, while filters express what is allowed, current, or applicable.

Strict filters can change search planning and result availability. The reader will compare unfiltered and filtered rankings and determine whether quality changed because relevant points were excluded or because the vector representation was weak.

Part 2 - Technical concept and behavior

This exercise uses one dense vector query together with compound payload conditions. It is hybrid in the sense of structured filtering plus vector search, not dense–sparse fusion. Qdrant plans filtered search using vector and payload indexes according to collection state and estimated cardinality.

Part 3 - Hints and Helper

Combine required tenant and status conditions with an exclusion condition for stale sources and an optional category condition. Create payload indexes for real filtering fields before ingestion in a fresh experiment, then compare latency and relevance.

Part 4 - Python code skeleton

```
from qdrant_client import QdrantClient, models

def E06(client, collection_name, query_vector, requirements, limit):
    query_filter = models.Filter(
        must=[...],
        should=[...],
        must_not=[...],
    )
    filtered = client.query_points(
        collection_name=collection_name,
        query=query_vector,
        query_filter=query_filter,
        limit=limit,
        with_payload=True,
    ).points
    unfiltered = client.query_points(
        collection_name=collection_name,
        query=query_vector,
        limit=limit,
        with_payload=True,
    ).points
    comparison = ...
    return filtered, unfiltered, comparison

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

9.3.8 Exercise 07 - Search Performance Benchmark

Part 1 - Why this matters

Search performance affects interactive latency and system capacity, but average time hides slow requests. A useful benchmark records latency distribution, throughput, data scale, query mix, filters, index settings, and retrieval quality.

The client benchmark must avoid measuring embedding generation accidentally unless end-to-end latency is the stated target. Warm-up, concurrency, and network location must also be documented.

Part 2 - Technical concept and behavior

Latency percentiles describe how long a chosen fraction of requests completed within. Queries per second describe throughput over an interval. Approximate index parameters can trade recall for speed, so benchmark results must include relevance or exact-search comparison when possible.

Part 3 - Hints and Helper

Precompute query vectors, run warm-up separately, randomize a reproducible query set, measure with a monotonic clock, report p50, p95, p99, and errors, and compare filtered with unfiltered workloads.

Part 4 - Python code skeleton

```
from statistics import median
from time import perf_counter

def percentile(values, fraction):
    ordered = sorted(values)
    index = ...
    return ordered[index]

def E07(search_operation, query_vectors, warmup_count, repetitions):
    for vector in query_vectors[:warmup_count]:
        search_operation(vector)
    latencies = []
    errors = []
    started_all = perf_counter()
    for _ in range(repetitions):
        for vector in query_vectors:
            started = perf_counter()
            try:
                search_operation(vector)
                latencies.append(perf_counter() - started)
            except Exception as error:
                errors.append(...)
    total_time = perf_counter() - started_all
    return {
        "p50": percentile(latencies, 0.50),
        "p95": percentile(latencies, 0.95),
        "p99": percentile(latencies, 0.99),
        "queries_per_second": ...,
        "errors": errors,
    }

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

9.3.9 Exercise 08 - Search Result Ranking Analysis

Part 1 - Why this matters

A ranking can contain the correct item but place it too low to be useful. Analysis examines where relevant and irrelevant items appear, how stable ranks are across configurations, and which query categories fail.

Scores alone cannot explain relevance. The reader must connect every point to a human judgment and inspect difficult negatives that appear close for superficial or domain-specific reasons.

Part 2 - Technical concept and behavior

Rank-based evaluation includes reciprocal rank, precision and recall at k, and NDCG when judgments have graded relevance. Rank correlation or overlap can compare configurations, while per-query inspection reveals failure patterns hidden by aggregate means.

Part 3 - Hints and Helper

Return point ID, score, rank, payload source, and relevance grade. Group errors by language, length, topic, and query type; compare at least two configurations; and preserve examples for future regression tests.

Part 4 - Python code skeleton

```
def analyze_ranking(points, relevance_grades, k):
    rows = []
    for rank, point in enumerate(points, start=1):
        grade = relevance_grades.get(point.id, 0)
        rows.append({
            "rank": rank,
            "id": point.id,
            "score": point.score,
            "grade": grade,
            "source": ...,
        })
    metrics = {
        "precision_at_k": ...,
        "recall_at_k": ...,
        "reciprocal_rank": ...,
        "ndcg_at_k": ...,
    }
    return rows, metrics

def E08(results_by_query, judgments, k):
    analyses = {
        query_id: analyze_ranking(...)
        for query_id, points in results_by_query.items()
    }
    failure_groups = ...
    return analyses, failure_groups

def main():
    print(E08(...))
```

```
if __name__ == "__main__":
    main()
```

9.3.10 Exercise 09 - Similarity Debugging Scenario

Part 1 - Why this matters

Unexpected similarity results can originate from the embedding model, query preprocessing, vector dimension, normalization, distance metric, stale points, payload filters, or index approximation. Changing search parameters before locating the layer can hide the real defect.

The final challenge builds a diagnostic sequence from input text to stored point and returned ranking. Each check produces evidence and stops unsafe conclusions when an earlier contract fails.

Part 2 - Technical concept and behavior

The debugging path validates collection configuration, embedding contract, finite values, vector norms, stored payload provenance, filter cardinality, exact versus approximate results, and labeled relevance. A transport success only proves that Qdrant processed the request.

Part 3 - Hints and Helper

Use one known query and controlled corpus. Compare the query embedding with a freshly generated document embedding, retrieve stored vectors by ID, temporarily run an exact search in the lab, and remove filters one at a time without weakening production authorization.

Part 4 - Python code skeleton

```
import math

def E09(client, collection_name, encoder, query_text,
        expected_contract, known_point_ids, query_filter):
    report = []
    collection_info = client.get_collection(collection_name)
    report.append({"check": "collection_contract", "result": ...})

    query_vector = encoder.encode_queries([query_text])[0]
    report.append({"check": "dimension", "result": ...})
    report.append({"check": "finite", "result": all(
        math.isfinite(value) for value in query_vector
    )})

    stored = client.retrieve(
        collection_name=collection_name,
        ids=known_point_ids,
        with_vectors=True,
        with_payload=True,
    )
    report.append({"check": "stored_provenance", "result": ...})
```

```
    approximate = client.query_points(...).points
    exact = client.query_points(..., search_params=...).points
    report.append({"check": "approximate_vs_exact", "result": ...})
    report.append({"check": "filter_effect", "result": ...})
    diagnosis = ...
    return report, approximate, exact, diagnosis

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

9.4 QD02 - RAG Integration Workflow

Models and Training

9.4.1 Exercise 00 - Chunk Text Data for Embedding

Part 1 - Why this matters

Long documents must be divided into retrievable units before embedding. Chunks that are too large may combine unrelated topics, while chunks that are too small may lose the context required to answer a question.

Chunking is therefore part of retrieval quality, not simple string slicing. Every chunk must preserve source identity, order, offsets, and enough overlap or structural context for later attribution.

Part 2 - Technical concept and behavior

A tokenizer-aware chunker limits each chunk by token count and may retain an overlap with the previous chunk. Structural boundaries such as paragraphs or headings should be preferred when possible. Overlap increases continuity but also storage and duplicate retrieval.

Part 3 - Hints and Helper

Accept a tokenizer function, positive chunk size, and smaller non-negative overlap. Preserve source ID, chunk index, character or token range, and source version. Reconstruct or inspect adjacent chunks to verify no unintended gaps.

Part 4 - Python code skeleton

```
def E00(text, source_id, source_version, tokenizer,
        chunk_size, overlap):
    if chunk_size <= 0 or overlap < 0 or overlap >= chunk_size:
        raise ValueError(...)
    tokens = tokenizer.encode(...)
    chunks = []
```

```
    step = chunk_size - overlap
    for start in range(0, len(tokens), step):
        end = min(start + chunk_size, len(tokens))
        chunk_tokens = tokens[start:end]
        chunk_text = tokenizer.decode(...)
        chunks.append({
            "source_id": source_id,
            "source_version": source_version,
            "chunk_index": len(chunks),
            "token_start": start,
            "token_end": end,
            "text": chunk_text,
        })
        if end == len(tokens):
            break
    return chunks

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

9.4.2 Exercise 01 - Store Document Chunks as Vectors

Part 1 - Why this matters

Each chunk vector must remain aligned with its source text and provenance. A shifted list or reused identifier can store a valid embedding under the wrong chunk, producing convincing but falsely attributed retrieval.

The ingestion function should be idempotent for one source version. Stable point IDs and payload fields allow regeneration, deletion of stale versions, and verification after upload.

Part 2 - Technical concept and behavior

The document encoder produces one vector per chunk. Qdrant points store stable IDs, vectors, and payloads containing source ID, version, chunk index, text, and embedding contract. Bounded upsert batches control request size.

Part 3 - Hints and Helper

Generate deterministic UUIDs from source, version, and chunk index. Verify equal counts and dimensions, include authorization and language fields, upsert with waiting enabled for the exercise, and retrieve samples by ID.

Part 4 - Python code skeleton

```
import uuid
from qdrant_client import QdrantClient, models
```

```
def chunk_id(source_id, source_version, chunk_index):
    identity = f"{source_id}:{source_version}:{chunk_index}"
    return str(uuid.uuid5(uuid.NAMESPACE_URL, identity))

def E01(client, collection_name, encoder, chunks, batch_size):
    vectors = encoder.encode_documents([chunk["text"] for chunk in chunks])
    if len(vectors) != len(chunks):
        raise ValueError(...)
    points = [
        models.PointStruct(
            id=chunk_id(...),
            vector=vector,
            payload={...},
        )
        for chunk, vector in zip(chunks, vectors)
    ]
    operations = []
    for start in range(0, len(points), batch_size):
        operations.append(client.upsert(..., wait=True))
    return points, operations

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

9.4.3 Exercise 02 - Query Embedding Generation

Part 1 - Why this matters

The query vector must live in the same compatible space as document vectors. Some embedding models use separate query instructions, prefixes, or encoder methods, so sending the raw question through the document path can reduce retrieval quality.

This exercise validates query preprocessing and embedding provenance before contacting Qdrant. Empty, non-finite, or dimensionally incompatible vectors must fail early.

Part 2 - Technical concept and behavior

A query encoder maps one question to a fixed-length dense vector. The embedding contract includes model identity, revision, dimension, normalization, language handling, and query instruction. It must match the collection's document space.

Part 3 - Hints and Helper

Normalize whitespace according to a documented policy, reject an empty query, use the model's query method, convert values to finite floats, and compare dimension and model revision with the expected contract.

Part 4 - Python code skeleton

```
import math

def E02(encoder, query_text, expected_contract):
    normalized_text = ...
    if not normalized_text:
        raise ValueError(...)
    raw_vector = encoder.encode_queries([normalized_text])[0]
    vector = [float(value) for value in raw_vector]
    observed_contract = {
        "model": ...,
        "revision": ...,
        "dimension": len(vector),
        "normalized": ...,
    }
    valid = observed_contract == expected_contract and all(
        math.isfinite(value) for value in vector
    )
    if not valid:
        raise ValueError(...)
    return vector, observed_contract

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

9.4.4 Exercise 03 - Retrieve Relevant Context Chunks

Part 1 - Why this matters

Retrieval selects the evidence that a later language model may use. Irrelevant or unauthorized chunks can mislead the answer, while missing provenance prevents the reader from checking where a claim originated.

The retrieval function must return scored chunks and source metadata, not concatenate text immediately. Keeping the intermediate result visible supports thresholding, debugging, evaluation, and attribution.

Part 2 - Technical concept and behavior

Qdrant's query interface receives the compatible query vector, limit, optional threshold, and payload filter. Each scored point can return payload text and provenance. The result remains a candidate set until application relevance checks are applied.

Part 3 - Hints and Helper

Require tenant and active source-version filters, request payload but not stored vectors, preserve rank and score, validate required payload fields, and return an empty list without inventing fallback chunks.

Part 4 - Python code skeleton

```
from qdrant_client import QdrantClient, models

def E03(client, collection_name, query_vector,
        tenant_id, limit, score_threshold):
    query_filter = models.Filter(must=[
        models.FieldCondition(
            key="tenant_id",
            match=models.MatchValue(value=tenant_id),
        ),
        ...,
    ])
    response = client.query_points(
        collection_name=collection_name,
        query=query_vector,
        query_filter=query_filter,
        score_threshold=score_threshold,
        limit=limit,
        with_payload=True,
        with_vectors=False,
    )
    chunks = [
        {"rank": rank, "id": point.id, "score": point.score, **point.payload}
        for rank, point in enumerate(response.points, start=1)
    ]
    return chunks

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

9.4.5 Exercise 04 - Context Ranking by Similarity

Part 1 - Why this matters

Initial vector search produces a ranking, but overlapping chunks or semantically broad passages can occupy several top positions. Context selection may need deduplication, source diversity, or a separate reranker.

Reordering must be evaluated because a reranker can improve or damage relevance. The original vector score and rank should remain available for comparison and debugging.

Part 2 - Technical concept and behavior

Candidate generation retrieves a wider set, after which a ranking function can combine vector score, reranker score, source diversity, or duplicate penalty. The final score should not mix incompatible scales without normalization or a documented rule.

Part 3 - Hints and Helper

Retrieve more candidates than the final limit, identify near-duplicate adjacent chunks, apply one transparent ranking rule, preserve original rank and scores, and compare NDCG or another ranking metric before and after.

Part 4 - Python code skeleton

```
def E04(query_text, candidates, reranker, final_limit):
    if final_limit <= 0:
        raise ValueError(...)
    reranker_scores = reranker.score(
        query_text,
        [candidate["text"] for candidate in candidates],
    )
    enriched = []
    for candidate, reranker_score in zip(candidates, reranker_scores):
        enriched.append({
            **candidate,
            "vector_rank": candidate["rank"],
            "reranker_score": float(reranker_score),
            "duplicate_penalty": ...,
            "final_score": ...,
        })
    ranked = sorted(enriched, key=..., reverse=True)
    selected = ...
    comparison = ...
    return selected, comparison

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

9.4.6 Exercise 05 - Metadata Filtering in RAG

Part 1 - Why this matters

RAG retrieval must respect tenant, permission, language, source status, and document version. Semantic similarity cannot decide whether the current user is allowed to receive a chunk.

Security filtering belongs inside the vector query. Retrieving globally and removing unauthorized results afterward can expose ranking information and allow forbidden points to consume the candidate limit.

Part 2 - Technical concept and behavior

Payload filters can combine mandatory identity and version conditions, optional categories, and exclusions. Indexed payload fields improve filtered retrieval and cardinality estimation. The

authoritative permission decision may still originate in PostgreSQL or another access system and be translated into retrieval conditions.

Part 3 - Hints and Helper

Build filters only from validated authorization context, not arbitrary client-supplied Qdrant expressions. Test cross-tenant and stale-version negatives, verify every returned payload, and create payload indexes for important fields.

Part 4 - Python code skeleton

```
from qdrant_client import models

def E05(authorization_context, language, allowed_sources):
    if not authorization_context.is_authenticated:
        raise PermissionError(...)
    query_filter = models.Filter(
        must=[
            models.FieldCondition(
                key="tenant_id",
                match=models.MatchValue(value=...),
            ),
            models.FieldCondition(
                key="language",
                match=models.MatchValue(value=...),
            ),
            models.FieldCondition(
                key="source_id",
                match=models.MatchAny(any=...),
            ),
            ...,
        ],
        must_not=[...],
    )
    return query_filter

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

9.4.7 Exercise 06 - Limit Context Window Size

Part 1 - Why this matters

Retrieved chunks compete with the question, instructions, conversation, and answer for limited model context. Selecting by document count alone can exceed the token budget because chunks have different lengths.

Budgeting should preserve the best evidence while avoiding partial or duplicate context. The reader will count formatted tokens and record why each candidate was included or excluded.

Part 2 - Technical concept and behavior

The usable retrieval budget is the total context allowance minus reserved tokens for other prompt components and the generated answer. Greedy rank-order selection is simple, while diversity-aware selection may use the budget more effectively. Token counts must use the target model's tokenizer and final formatting.

Part 3 - Hints and Helper

Validate a positive budget, format each chunk with source labels before counting, reject a single chunk that exceeds policy, avoid overlapping duplicates, and return selected chunks, token total, and excluded reasons.

Part 4 - Python code skeleton

```
def E06(ranked_chunks, tokenizer, total_limit,
        reserved_prompt_tokens, reserved_answer_tokens):
    available = total_limit - reserved_prompt_tokens - reserved_answer_tokens
    if available <= 0:
        raise ValueError(...)
    selected = []
    excluded = []
    used = 0
    for chunk in ranked_chunks:
        formatted = ...
        token_count = len(tokenizer.encode(formatted))
        if used + token_count <= available and ...:
            selected.append({...})
            used += token_count
        else:
            excluded.append({...})
    return selected, used, available, excluded

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

9.4.8 Exercise 07 - Handle Empty Retrieval Case

Part 1 - Why this matters

Some questions have no eligible or sufficiently similar source. Calling the language model with an empty evidence block encourages unsupported answers and hides whether the knowledge base lacks coverage.

A robust RAG system treats empty retrieval as a normal state with explicit policy. It may abstain, request clarification, suggest another search, or route to an approved alternative source.

Part 2 - Technical concept and behavior

Empty retrieval can result from no indexed source, strict authorization, metadata mismatch, score threshold, stale version removal, or embedding failure. The response should preserve a machine-readable reason and must not weaken security filters automatically.

Part 3 - Hints and Helper

Distinguish zero candidates before filtering from zero eligible candidates after filtering where observability permits. Do not expose forbidden-source details. Ensure the generator callback is never invoked on the abstention path.

Part 4 - Python code skeleton

```
from dataclasses import dataclass

@dataclass
class RetrievalOutcome:
    status: str
    chunks: list
    message: str
    diagnostic_code: str

def E07(retrieved_chunks, generator, policy):
    if not retrieved_chunks:
        return RetrievalOutcome(
            status="no_context",
            chunks=[],
            message=...,
            diagnostic_code=...,
        )
    answer = generator(...)
    return RetrievalOutcome(
        status="answered",
        chunks=retrieved_chunks,
        message=answer,
        diagnostic_code="ok",
    )

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

9.4.9 Exercise 08 - End-to-End RAG Pipeline Test

Part 1 - Why this matters

Individual components can pass while their integration fails. A document may be chunked correctly but stored under the wrong tenant, or relevant chunks may be retrieved but omitted from the final prompt.

An end-to-end test preserves evidence from ingestion through answer generation. It evaluates retrieval and answer support separately so a fluent response cannot hide a broken retriever.

Part 2 - Technical concept and behavior

The pipeline performs source validation, chunking, document embedding, point upsert, query embedding, authorized retrieval, ranking, token budgeting, prompt construction, generation, and citation validation. Each stage returns structured traces and timing.

Part 3 - Hints and Helper

Use a tiny source with known questions and answers. Assert deterministic IDs, expected relevant chunk presence, no forbidden payloads, context within budget, generator abstention on empty retrieval, and citations referencing selected sources.

Part 4 - Python code skeleton

```
def E08(source, question, components, authorization_context):
    trace = []
    chunks = components.chunker(...)
    trace.append({"stage": "chunk", "count": len(chunks)})
    ingestion = components.ingester(...)
    trace.append({"stage": "ingest", "result": ingestion})
    query_vector = components.query_encoder(...)
    candidates = components.retriever(
        query_vector,
        authorization_context=authorization_context,
    )
    trace.append({"stage": "retrieve", "candidates": candidates})
    ranked = components.ranker(question, candidates)
    context, budget = components.budgeter(ranked)
    if not context:
        return components.empty_handler(...), trace
    prompt = components.prompt_builder(question, context)
    answer = components.generator(prompt)
    citations_valid = ...
    evaluation = {
        "expected_chunk_retrieved": ...,
        "authorization_valid": ...,
        "budget_valid": ...,
        "citations_valid": citations_valid,
        "answer_supported": ...,
    }
```

```
    return answer, context, trace, evaluation

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

9.4.10 Exercise 09 - RAG Failure Mode Analysis

Part 1 - Why this matters

RAG can fail at source ingestion, chunking, embedding, filtering, retrieval, ranking, context construction, generation, or citation. Treating every wrong answer as a model problem prevents effective diagnosis.

The final challenge injects one controlled fault at a time and records where evidence first diverges from expectations. The purpose is to build observability and targeted remediation, not to tune every component simultaneously.

Part 2 - Technical concept and behavior

Failure modes include missing sources, stale vectors, incompatible embeddings, wrong tenant filters, weak recall, overly strict thresholds, duplicated chunks, context truncation, prompt injection, unsupported generation, and invalid citations. Each has a different detection signal.

Part 3 - Hints and Helper

Create a fault matrix with expected stage, observable signal, safety impact, and remediation. Run a clean baseline, inject exactly one fault, compare traces, and verify that security faults fail closed and empty evidence leads to abstention.

Part 4 - Python code skeleton

```
from copy import deepcopy

def run_scenario(pipeline, test_case, fault):
    configuration = deepcopy(test_case["configuration"])
    fault.apply(configuration)
    result = pipeline.run(test_case["question"], configuration)
    first_divergence = ...
    detected = ...
    safe_behavior = ...
    return {
        "fault": fault.name,
        "expected_stage": fault.expected_stage,
        "first_divergence": first_divergence,
        "detected": detected,
        "safe_behavior": safe_behavior,
        "trace": result.trace,
    }
```

```
def E09(pipeline, baseline_case, faults):
    baseline = pipeline.run(...)
    scenarios = [run_scenario(...) for fault in faults]
    summary = {...}
    return baseline, scenarios, summary

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

9.5 QD03 - Indexing & Scalable Retrieval

AI Infrastructure and Hardware

9.5.1 Exercise 00 - HNSW Index Configuration

Part 1 - Why this matters

HNSW provides approximate nearest-neighbor retrieval by navigating a layered graph instead of comparing every stored vector. Its configuration affects memory, construction effort, latency, and recall.

Larger or more expensive settings are not automatically better. The reader must connect index parameters to a fixed dataset and labeled query workload before changing production configuration.

Part 2 - Technical concept and behavior

The parameter `m` influences graph connectivity, `ef_construct` affects index construction quality and cost, and query-time `hnsw_ef` affects search effort. `full_scan_threshold` helps determine when scanning may be preferable for small segments or candidate sets.

Part 3 - Hints and Helper

Create disposable collections with controlled parameter sets. Keep vectors and queries identical, wait for indexing and optimization state, compare approximate results with exact search, and record build time, memory, latency, and recall.

Part 4 - Python code skeleton

```
from qdrant_client import QdrantClient, models

def E00(client, collection_name, vector_size, configuration):
    client.create_collection(
        collection_name=collection_name,
        vectors_config=models.VectorParams(
            size=vector_size,
```

```
            distance=models.Distance.COSINE,
        ),
        hnsw_config=models.HnswConfigDiff(
            m=configuration["m"],
            ef_construct=configuration["ef_construct"],
            full_scan_threshold=configuration["full_scan_threshold"],
        ),
    )
    information = client.get_collection(collection_name)
    verified = ...
    return information, verified

def main():
    client = QdrantClient(...)
    print(E00(...))

if __name__ == "__main__":
    main()
```

9.5.2 Exercise 01 - Index Build vs Query Speed Tradeoff

Part 1 - Why this matters

An index can reduce query latency while requiring construction time, memory, and background optimization. A configuration that builds quickly may provide lower recall, while an expensive index may exceed the needs of the application.

The experiment must measure quality and speed together. Exact search provides a reference neighbor set for recall comparison, not necessarily the desired production mode.

Part 2 - Technical concept and behavior

Index build time spans ingestion and the transition to an indexed or stable collection state. Query-time `hnsw_ef` can vary search effort without rebuilding the index. Recall at k compares approximate IDs with exact top-k IDs.

Part 3 - Hints and Helper

Use identical points in separate collections, poll status with a deadline, precompute query vectors, warm up, and compare exact and approximate results across several `hnsw_ef` values.

Part 4 - Python code skeleton

```
from time import perf_counter, monotonic
from qdrant_client import models

def recall_at_k(approximate_ids, exact_ids):
    if not exact_ids:
        return ...
    return len(set(approximate_ids) & set(exact_ids)) / len(exact_ids)
```

```
def E01(client, collection_name, query_vectors, ef_values, k):
    experiments = []
    for query_vector in query_vectors:
        exact = client.query_points(
            collection_name=collection_name,
            query=query_vector,
            search_params=models.SearchParams(exact=True),
            limit=k,
        ).points
        for ef in ef_values:
            started = perf_counter()
            approximate = client.query_points(
                collection_name=collection_name,
                query=query_vector,
                search_params=models.SearchParams(hnsw_ef=ef, exact=False),
                limit=k,
            ).points
            experiments.append({...})
    return experiments

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

9.5.3 Exercise 02 - Payload Index Creation

Part 1 - Why this matters

Vector indexes accelerate similarity navigation, while payload indexes accelerate and inform structured filtering. A tenant or source filter without an appropriate payload index can become expensive as the collection grows.

Every payload index consumes resources. The reader should create indexes only for fields used in meaningful filters and use the correct schema type for stored values.

Part 2 - Technical concept and behavior

Qdrant payload indexes are defined by field name and schema such as keyword, integer, float, Boolean, datetime, text, or geographic types. They also support cardinality estimation and filtered-search planning. Important payload indexes are best planned before large ingestion.

Part 3 - Hints and Helper

Inspect representative payloads, map tenant and source fields to keyword types and time to datetime where appropriate, create indexes, verify collection payload schema, and benchmark the same filtered queries before and after in isolated collections.

Part 4 - Python code skeleton

```
from qdrant_client import QdrantClient, models

def E02(client, collection_name, index_definitions):
    operations = []
    for field_name, schema_type in index_definitions:
        operations.append(client.create_payload_index(
            collection_name=collection_name,
            field_name=field_name,
            field_schema=schema_type,
            wait=True,
        ))
    information = client.get_collection(collection_name)
    payload_schema = information.payload_schema
    verified = all(field in payload_schema for field, _ in index_definitions)
    return operations, payload_schema, verified

def main():
    client = QdrantClient(...)
    definitions = [
        ("tenant_id", models.PayloadSchemaType.KEYWORD),
        ("source_version", models.PayloadSchemaType.KEYWORD),
        (...),
    ]
    print(E02(client, ..., definitions))

if __name__ == "__main__":
    main()
```

9.5.4 Exercise 03 - Large-Scale Vector Ingestion

Part 1 - Why this matters

Large ingestion must balance throughput, memory, request size, retries, index construction, and verification. Sending everything in one request risks failure, while tiny requests waste network overhead.

Idempotent point IDs make retries safe. The reader will checkpoint progress, distinguish failed batches, and verify stored points without trusting only the number submitted.

Part 2 - Technical concept and behavior

Bounded upsert batches write points through Qdrant's write path and WAL. Index and optimizer work may continue after client acknowledgment. Parallel ingestion can improve throughput but may overload the service or reorder application progress reporting.

Part 3 - Hints and Helper

Stream source records, generate deterministic IDs, validate dimension and payload contract, use configurable batch size and bounded concurrency, retry only understood transient failures, and save progress after acknowledged batches.

Part 4 - Python code skeleton

```
from time import perf_counter
from qdrant_client import models

def batched(iterable, batch_size):
    batch = []
    for item in iterable:
        batch.append(item)
        if len(batch) == batch_size:
            yield batch
            batch = []
    if batch:
        yield batch

def E03(client, collection_name, record_stream, encoder,
        batch_size, progress_store):
    statistics = {"submitted": 0, "batches": 0, "errors": []}
    started = perf_counter()
    for records in batched(record_stream, batch_size):
        vectors = encoder.encode_documents([...])
        points = [models.PointStruct(...) for ... in zip(records, vectors)]
        try:
            result = client.upsert(..., wait=True)
            statistics["submitted"] += len(points)
            statistics["batches"] += 1
            progress_store.save(...)
        except Exception as error:
            statistics["errors"].append(...)
            ...
    statistics["elapsed"] = perf_counter() - started
    statistics["verification"] = ...
    return statistics

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

9.5.5 Exercise 04 - Shard Configuration Experiment

Part 1 - Why this matters

Shards partition a collection's points and can distribute storage and processing. Too few shards may limit parallelism or placement, while too many can add overhead and create small segments for a modest dataset.

Shard count must be evaluated on a distributed topology and realistic workload. A single-node experiment can reveal overhead but cannot prove multi-node scalability.

Part 2 - Technical concept and behavior

Collection creation can specify `shard_number`. In a cluster, shard replicas are assigned among peers according to configuration. Sharding affects ingestion and query distribution, while payload-based multitenancy may introduce additional partitioning strategies.

Part 3 - Hints and Helper

Create separate disposable collections with equal data and different shard counts. Record topology, ingestion throughput, collection status, p95 search latency, and retrieval quality. Do not change multiple scaling settings in the same comparison.

Part 4 - Python code skeleton

```
from qdrant_client import QdrantClient, models

def E04(client, base_name, vector_size, shard_counts,
        points, benchmark):
    experiments = []
    for shard_count in shard_counts:
        collection_name = f"{base_name}-shards-{shard_count}"
        client.create_collection(
            collection_name=collection_name,
            vectors_config=models.VectorParams(
                size=vector_size,
                distance=models.Distance.COSINE,
            ),
            shard_number=shard_count,
        )
        ingestion = ...
        status = client.get_collection(collection_name)
        search = benchmark(...)
        experiments.append({...})
    return experiments

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

9.5.6 Exercise 05 - Replication Factor Impact Test

Part 1 - Why this matters

Replication protects a vector search service from losing availability when a node becomes unavailable. A collection with several replicas can continue serving data from another peer, but every additional replica also increases storage consumption, network traffic, and the work

required to confirm writes. Testing this tradeoff helps engineers select a replication factor from measured requirements instead of treating a larger value as automatically better.

This experiment must run only on an isolated distributed Qdrant environment whose failure budget is known in advance. The reader will compare ingestion time, query latency, error rate, and result integrity across collections with different replication factors. A controlled single-peer interruption may be included only when the remaining replicas can safely preserve quorum and the test environment can be restored.

Part 2 - Technical concept and behavior

Let r denote the replication factor, meaning the number of copies maintained for each shard. For each tested value of r, collect comparable measurements under the same dataset, query set, shard count, and hardware conditions:

$$M_r = (T_{\text{ingest}}, L_{50}, L_{95}, E, A, I)$$

where T_{ingest} is ingestion time, L_{50} and L_{95} are query-latency percentiles, E is the error rate, A is observed availability, and I represents integrity checks on returned identifiers and payloads. Replication should be evaluated together with the cluster topology because a requested factor cannot provide resilience if the replicas are not distributed across independent failure domains.

Part 3 - Hints and Helper

Create separate disposable collections rather than repeatedly changing one collection during measurement. In Python, pass `replication_factor` when creating each collection, wait until the collection reports a healthy state, and run the same ingestion and query workload. Record the cluster state before any failure simulation. Never stop more peers than the declared test permits, and always restore the peer before starting the next configuration.

Part 4 - Python code skeleton

```
from qdrant_client import QdrantClient, models

def create_test_collection(client, name, vector_size, replicas):
    client.create_collection(
        collection_name=name,
        vectors_config=models.VectorParams(
            size=vector_size,
            distance=models.Distance.COSINE,
        ),
        replication_factor=replicas,
        shard_number=...,
    )

def E05(client, dataset, queries, replica_values):
    results = {}
    for replicas in replica_values:
        name = f"replication_test_{replicas}"
        create_test_collection(client, name, ..., replicas)
        ...  # wait for a healthy collection
```

```
            ...  # ingest the same dataset
            ...  # measure normal search behavior
            ...  # optionally perform one approved peer-failure drill
            results[replicas] = ...
    return results

def main():
    client = QdrantClient(url=...)
    print(E05(client, ..., ..., [1, 2, 3]))

if __name__ == "__main__":
    main()
```

9.5.7 Exercise 06 - Consistency vs Performance Tradeoff

Part 1 - Why this matters

Distributed vector databases must decide how much coordination is required before a write is acknowledged and which replicas may answer a read. Stronger coordination can reduce the chance of observing stale state, but it can also add network round trips and make operations more sensitive to slow or unavailable peers. A scalable application therefore needs an explicit consistency policy rather than an undocumented reliance on defaults.

The correct setting depends on the operation. A newly uploaded private document may require strict visibility and access-control guarantees, while an approximate recommendation query may tolerate a short delay in exchange for lower latency. This exercise teaches the reader to measure visibility, correctness, and latency together so that performance improvements do not silently weaken application guarantees.

Part 2 - Technical concept and behavior

For a configuration c, measure the time between a successful write and the first read that observes it:

$$V_c = t_{\text{visible}} - t_{\text{acknowledged}}$$

The experiment also records query latency and operation failures. Write ordering, write consistency, and read consistency describe related but distinct controls; they must not be collapsed into a single label. A useful report associates every observation with the exact collection topology and client options used during that run.

Part 3 - Hints and Helper

Use a distributed test cluster and generate a unique point identifier for every trial. Upsert the point, immediately query or retrieve it using the selected consistency option, and retry with a bounded deadline. Store raw observations rather than only an average. Because client APIs can evolve, consult the installed Qdrant client signature for the supported read-consistency and write-ordering arguments instead of inventing unsupported values.

Part 4 - Python code skeleton

```
import time
from qdrant_client import QdrantClient, models

def observe_visibility(client, collection, point, read_options,
                       write_options, timeout_s):
    acknowledged_at = time.perf_counter()
    client.upsert(
        collection_name=collection,
        points=[point],
        **write_options,
    )
    while time.perf_counter() - acknowledged_at < timeout_s:
        records = client.retrieve(
            collection_name=collection,
            ids=[point.id],
            **read_options,
        )
        if records:
            return ...
    return ...

def E06(client, collection, configurations):
    observations = []
    for config in configurations:
        point = models.PointStruct(id=..., vector=..., payload=...)
        observations.append(
            observe_visibility(client, collection, point, ..., ..., ...)
        )
    return ...  # p50/p95 latency, visibility delay, and errors

def main():
    client = QdrantClient(url=...)
    print(E06(client, ..., ...))

if __name__ == "__main__":
    main()
```

9.5.8 Exercise 07 - Search Latency Monitoring

Part 1 - Why this matters

Average latency can make an unhealthy search service appear acceptable because a small number of very slow requests are hidden by many fast ones. Users experience individual requests, so tail percentiles such as p95 and p99 are essential for detecting pauses caused by filters, index changes, resource pressure, or network instability. Monitoring must also include errors because a fast failed request is not a successful search.

Semantic-search latency should be divided into meaningful stages. Embedding generation, network transfer, Qdrant retrieval, reranking, and response construction have different causes

and remedies. This exercise focuses on retrieval timing while allowing an end-to-end measurement to be recorded separately, preventing an embedding bottleneck from being incorrectly blamed on the vector index.

Part 2 - Technical concept and behavior

For n successful requests with sorted latencies $l_{(1)}, \ldots, l_{(n)}$, percentile p can be estimated from the corresponding ranked observation. Monitoring should report at least:

$$(L_{50}, L_{95}, L_{99}, R_{\text{error}}, QPS)$$

Measurements must be labeled by collection, workload, filter type, result limit, and index parameters. Warm-up traffic should be distinguished from steady-state traffic, and both successful and failed requests must contribute to the request count.

Part 3 - Hints and Helper

Use `time.perf_counter()` for elapsed time and preserve each sample until percentiles are computed. Run representative filtered and unfiltered query groups instead of repeating one vector. Avoid printing inside the timed region. For a production system, export the same concepts to a metrics platform; this exercise keeps them in Python so the calculation remains visible.

Part 4 - Python code skeleton

```
import time

def percentile(sorted_values, fraction):
    index = ...
    return sorted_values[index]

def timed_query(client, collection, query_vector, query_filter=None):
    started = time.perf_counter()
    try:
        response = client.query_points(
            collection_name=collection,
            query=query_vector,
            query_filter=query_filter,
            limit=...,
        )
        return time.perf_counter() - started, True, response
    except Exception as error:
        return time.perf_counter() - started, False, error

def E07(client, collection, workload):
    samples = []
    failures = 0
    for item in workload:
        elapsed, ok, _ = timed_query(client, collection, ..., ...)
        ...
    ordered = sorted(samples)
```

```
    return {
        "p50": ...,
        "p95": ...,
        "p99": ...,
        "error_rate": ...,
    }

def main():
    client = ...
    print(E07(client, ..., ...))

if __name__ == "__main__":
    main()
```

9.5.9 Exercise 08 - Index Corruption Recovery Scenario

Part 1 - Why this matters

Search infrastructure can become unavailable because of disk faults, interrupted operations, operator mistakes, or damaged state. A recovery plan is useful only when it has been rehearsed and when the restored collection is checked for semantic correctness, not merely reported as running. Known queries, point counts, payloads, and provenance must survive recovery before downstream applications can trust the service again.

This exercise treats corruption as a safe recovery drill. The reader must not edit, delete, or deliberately damage Qdrant's internal storage files. Instead, the test creates a supported snapshot, simulates the original collection being unavailable in an isolated environment, restores through supported Qdrant mechanisms, and validates the recovered collection. This preserves the lesson without normalizing unsafe manipulation of database internals.

Part 2 - Technical concept and behavior

Let D be the reference set and D' the restored set. Basic recovery validation requires matching identifiers and critical metadata:

$$\{\mathrm{id}(x) : x \in D\} = \{\mathrm{id}(x) : x \in D'\}$$

It should also compare a sample of vectors or stable source hashes and run a fixed query suite against both versions. Recovery time objective measures how long restoration takes, while recovery point objective describes how much data may be absent relative to the snapshot time. Both must be recorded with the snapshot identifier.

Part 3 - Hints and Helper

Use `create_snapshot()` and preserve the returned snapshot metadata. Perform the restore into a clean test collection or test service using the supported snapshot recovery API for your deployment. Keep the actual restore call behind a clearly named helper because local and distributed deployments may use different supported workflows. Abort validation if the snapshot identity or expected collection schema does not match.

Part 4 - Python code skeleton

```
from qdrant_client import QdrantClient

def create_recovery_snapshot(client, collection):
    snapshot = client.create_snapshot(collection_name=collection)
    return snapshot

def restore_with_supported_api(client, snapshot, target_collection):
    ...  # use the documented recovery API for this deployment

def validate_recovery(client, source_manifest, target_collection,
                      known_queries):
    ...  # compare collection configuration and point count
    ...  # compare IDs, payload provenance, and source hashes
    ...  # execute known queries and compare expected results
    return ...

def E08(client, source_collection, target_collection, manifest, queries):
    snapshot = create_recovery_snapshot(client, source_collection)
    ...  # verify and preserve snapshot metadata
    restore_with_supported_api(client, snapshot, target_collection)
    return validate_recovery(client, manifest, target_collection, queries)

def main():
    client = QdrantClient(url=...)
    print(E08(client, ..., ..., ..., ...))

if __name__ == "__main__":
    main()
```

9.5.10 Exercise 09 - Scalable Semantic Search System Design

Part 1 - Why this matters

A scalable semantic-search system is more than a vector collection. It must preserve the embedding contract, ingest data idempotently, enforce tenant and authorization filters, monitor retrieval quality, and recover from infrastructure failures. A weakness in any one of these boundaries can produce irrelevant answers, data leakage, duplicate points, or an outage even when nearest-neighbor search itself works correctly.

The final design challenge joins the chapter's experiments into one defensible architecture. The reader must translate expected dataset size, query volume, latency objectives, availability targets, and update frequency into index, shard, replication, payload, and recovery decisions. Every decision should have a measurable reason and a test that can detect when its assumption stops being true.

Part 2 - Technical concept and behavior

The design can be viewed as a constrained optimization problem:

$$\min C_{\text{system}} \quad \text{subject to} \quad L_{95} \leq L_{\text{SLO}},\ R@k \geq R_{\text{target}},\ A \geq A_{\text{target}}$$

where cost includes compute, storage, and operational complexity. The system must define vector dimensions and distance, payload indexes, HNSW parameters, shard and replica counts, batch ingestion, consistency policy, quality evaluation, latency monitoring, and snapshot recovery. The relational or document source of truth must remain connected to every vector through stable provenance.

Part 3 - Hints and Helper

Begin with a configuration object and validate incompatible values before provisioning. Separate offline evaluation from online monitoring: recall and relevance require labeled or reference queries, while latency and errors require continuous observation. Use deterministic point identifiers so retries are safe, apply authorization filters inside retrieval, and include a recovery drill in the acceptance criteria. Do not claim scalability from one successful query; test representative volume and concurrency.

Part 4 - Python code skeleton

```
from dataclasses import dataclass
from qdrant_client import QdrantClient

@dataclass
class SearchSystemConfig:
    collection: str
    vector_size: int
    shard_number: int
    replication_factor: int
    target_p95_ms: float
    target_recall: float

def validate_config(config):
    ...

def provision_collection(client, config):
    ...

def ingest_idempotently(client, config, records):
    ...

def evaluate_quality_and_latency(client, config, evaluation_set):
    ...

def run_recovery_drill(client, config):
    ...

def E09(client, config, records, evaluation_set):
    validate_config(config)
    provision_collection(client, config)
    ingest_idempotently(client, config, records)
    report = evaluate_quality_and_latency(client, config, evaluation_set)
    recovery = run_recovery_drill(client, config)
```

```
    return ...

def main():
    client = QdrantClient(url=...)
    config = SearchSystemConfig(...)
    print(E09(client, config, ..., ...))

if __name__ == "__main__":
    main()
```

Book 10

Linear Regression

10.1 Introduction

10.1.1 What will the book teach you

Linear regression is the first complete machine learning model in this volume. In the previous books, you learned how tensors are represented, how gradients are calculated, how optimization changes parameters, how losses measure error, and how data can be stored and retrieved. This book brings those ideas together inside one model that learns a measurable relationship from data. The objective is no longer to execute an isolated mathematical operation. The objective is to construct a system whose parameters improve through observation, error measurement, and repeated correction.

The basic model may appear simple:

$$\hat{y} = wx + b$$

However, this equation contains the essential structure of supervised learning. The input x contains the information available to the model, the weight w controls how strongly the input affects the prediction, the bias b shifts the prediction independently of the input, and $\hat{y}$ is the value predicted by the model. During training, the prediction is compared with the true target y. The resulting error is converted into a loss, gradients describe how the parameters contributed to that loss, and an optimizer adjusts the parameters in a direction expected to reduce future error.

The reader must understand that linear regression is not limited to drawing a line through a two-dimensional graph. With several input features, the model becomes:

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b$$

where $\mathbf{X}$ may contain many samples and many features. The same law can estimate energy consumption from weather conditions, predict demand from historical measurements, approximate a physical relationship, or provide a transparent baseline before a more complex neural network is introduced. In higher-dimensional spaces, the learned relationship forms a hyperplane even when it can no longer be visualized as an ordinary line.

This book will teach the complete lifecycle of a linear regression model rather than presenting only the final PyTorch command. You will begin by constructing the prediction equation

manually, checking tensor shapes, observing the effects of weights and bias, and comparing manual calculations with `torch.nn.Linear`. You will then inspect model parameters, gradient requirements, device placement, state dictionaries, parameter freezing, saving, loading, and the failures caused by incorrect model structure. These experiments will make the internal behavior visible before automation is introduced.

Data preparation is treated as part of the model rather than as an unrelated preliminary step. You will generate controlled synthetic datasets, add noise, scale features, construct training and testing partitions, create batches, and use a `DataLoader`. You will also investigate missing values, incorrect dimensions, data leakage, feature collinearity, numerical precision, caching, and transfer costs between the CPU and GPU. A mathematically correct model can still fail when its data pipeline is incorrect, so every transformation must preserve meaning as well as shape.

After preparing the data, you will build prediction functions for individual samples and batches. You will compare CPU and CUDA execution, disable gradient tracking during inference, test numerical consistency, measure latency and memory use, and examine the behavior of large or streaming workloads. Prediction is not considered correct merely because the program runs. Its output shape, device, data type, numerical stability, and agreement with an independently calculated reference must all be verified.

The book then connects prediction error to learning through the mean squared error loss:

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$$

You will implement this equation manually and compare it with PyTorch, inspect reduction modes, observe the influence of outliers, and study how scale and precision affect the loss. Because squaring gives larger errors greater influence, the loss function does more than report model quality: it determines which mistakes produce the strongest training signal. The reader must therefore learn to interpret a loss value instead of treating it as an unexplained number that should simply decrease.

Gradient-based parameter adjustment follows the loss. You will inspect weight and bias gradients, perform manual updates, compare them with optimizer steps, verify gradients using finite differences, and study accumulation, clipping, noise, frozen parameters, mixed precision, and graph retention. These exercises connect the derivative to a physical change in the model. A gradient is useful only when the reader can explain which parameter it belongs to, why it has its shape and sign, and how the learning rate converts it into an update.

Training will then be extended from one update into a controlled multi-epoch process. You will monitor convergence and divergence, apply early stopping and learning-rate schedules, preserve reproducibility with random seeds, save checkpoints, resume interrupted training, and compare CPU and GPU behavior. The goal is not to obtain a decreasing loss once, but to build a training process that can be repeated, inspected, interrupted, restored, and trusted.

Finally, the trained model will be evaluated and interpreted. You will calculate RMSE and MAE, separate training behavior from evaluation behavior, detect overfitting, use validation strategies, analyze residuals, profile latency and memory, and test robustness under noise. Vi-

sualizations will compare predictions with targets, expose residual patterns, show convergence behavior, and reveal the effects of weights, bias, features, and hyperparameters. A final report must explain not only how well the model performed, but also where it failed and whether the evidence supports using it on unseen data.

Linear regression is intentionally studied in depth because it establishes a reference point for every model that follows. Logistic regression will reuse a linear score but convert it into a probability for classification. The perceptron will reuse weighted inputs and a bias but apply a decision rule. Neural networks will later combine many such transformations into layers. If the reader understands the entire linear regression pipeline, the next models will appear as structured extensions rather than unrelated algorithms.

10.1.2 How to study this book

Every exercise should be treated as an experiment. Before running the code, predict the expected tensor shape, device, data type, numerical direction, and model behavior. After running it, compare the observation with the prediction and explain any difference. When an exercise changes one variable, such as the learning rate, batch size, precision, noise level, or initialization, keep the other conditions fixed so that the result has a clear cause.

Do not use CUDA merely because it is available. Begin with small CPU examples whose values can be checked manually, then repeat selected experiments on the GPU and measure whether the workload is large enough to benefit from parallel execution. Similarly, do not accept a low training loss without testing unseen data. Preserve a clean evaluation split, prevent data leakage, and compare the learned model with a simple baseline.

For every completed challenge, the reader should be able to answer four questions: What mathematical relationship was implemented? What did PyTorch do with the tensors and parameters? What evidence shows that the result is correct? Under what condition could the same code fail? If these questions cannot be answered, the exercise should be repeated before continuing. The purpose of this book is not to memorize a training loop, but to develop the ability to reason about a learning system from its data to its final evaluation.

10.1.3 The Chapters (LR00 - LR08) included in this book

- LR00 - Introduction to Linear Regression
- LR01 - Model Structure and Components
- LR02 - Data Preparation and Feature Inputs
- LR03 - Prediction Function Implementation
- LR04 - Error Measurement and Loss Definition
- LR05 - Parameter Adjustment and Gradient
- LR06 - Training Iterations and Convergence
- LR07 - Model Evaluation and Performance
- LR08 - Result Visualization and Interpretation

10.2 LR00 - Introduction to Linear Regression

Models and Training

10.2.1 Exercise 00 - Define Linear Model $y = wx + b$

Part 1 - Why this matters

Linear regression is the smallest complete model that can learn a relationship between an input and a continuous target. Its simplicity makes every component visible: the input carries evidence, the weight determines how strongly that evidence changes the output, and the bias provides a baseline prediction. Understanding this model prepares the reader to recognize the same weighted transformation inside larger neural networks.

The equation is also a controlled environment for learning model reasoning. A positive weight, a negative weight, a zero weight, and different bias values produce predictable changes. By calculating several outputs manually before using PyTorch, the reader can distinguish mathematical behavior from framework behavior and detect implementation errors early.

Part 2 - Mathematical form

For a scalar input $x \in \mathbb{R}$, define the predicted value:

$$\hat{y} = wx + b$$

where $w \in \mathbb{R}$ is the weight, $b \in \mathbb{R}$ is the bias, and $\hat{y} \in \mathbb{R}$ is the prediction. The weight controls the slope of the line and the bias equals the predicted value when $x = 0$. This exercise defines a forward calculation only; the parameters are not trained yet.

Part 3 - Hints and Helper

Choose small values that can be verified by hand. Calculate the multiplication before adding the bias, and compare the result with the function output. Test at least one positive input, one negative input, and zero. Do not introduce a loss function or optimizer in this exercise.

Part 4 - Python code skeleton

```
def E00(x, w, b):
    y_hat = ...
    return y_hat

def main():
    x = ...
    w = ...
    b = ...
    print(E00(x, w, b))

if __name__ == "__main__":
    main()
```

10.2.2 Exercise 01 - Implement Linear Function in PyTorch

Part 1 - Why this matters

A mathematical model becomes useful in machine learning only when its operations are represented by tensors. PyTorch allows the same linear equation to participate later in automatic differentiation, batching, device transfer, and optimization. Implementing the equation directly first prevents `nn.Linear` from hiding the relationship between the formula and the computed tensor.

This exercise also establishes a verification habit. The reader should inspect the input values, output value, shape, and data type instead of accepting a printed tensor without interpretation. A framework call is correct only when its behavior matches the mathematical model that was intended.

Part 2 - Mathematical form

Let x, w, and b be scalar floating-point tensors. The PyTorch computation represents:

$$\hat{y} = x \odot w + b$$

For scalar tensors, element-wise multiplication and ordinary scalar multiplication are equivalent. The returned value remains a tensor, preserving information required for later tensor operations and gradient tracking.

Part 3 - Hints and Helper

Create all three values with `torch.tensor()` and use the same floating-point data type. Print both the result and its type. Compare it with a manual Python calculation using the same numerical values, but keep the function itself tensor-based.

Part 4 - Python code skeleton

```
import torch

def E01(x, w, b):
    y_hat = torch...
    return y_hat

def main():
    x = torch.tensor(..., dtype=torch.float32)
    w = torch.tensor(..., dtype=torch.float32)
    b = torch.tensor(..., dtype=torch.float32)
    print(E01(x, w, b))

if __name__ == "__main__":
    main()
```

10.2.3 Exercise 02 - Tensor Shape Verification for Inputs

Part 1 - Why this matters

Many model failures are shape failures rather than mathematical failures. Two tensors may contain correct values but organize samples and features differently, causing broadcasting, matrix multiplication, or a linear layer to behave incorrectly. Shape verification creates an explicit contract at the boundary of the model.

A useful validation step should fail early and explain the violated expectation. If an invalid tensor reaches a later operation, the resulting error may mention an internal matrix dimension and hide the real source of the problem. This exercise teaches the reader to identify dimensional meaning before performing the forward pass.

Part 2 - Technical concept and behavior

For a batch containing N samples and F features per sample, the expected input shape is:

$$\mathbf{X} \in \mathbb{R}^{N \times F}$$

The final dimension represents features. For a single-feature model, $F = 1$, so a batch normally has shape $(N, 1)$ rather than $(N,)$. The validation should distinguish the number of dimensions from the size of each dimension.

Part 3 - Hints and Helper

Inspect `x.ndim` and `x.shape`. Accept the expected feature count as an argument instead of hard-coding it. Raise a clear `ValueError` when the tensor is not two-dimensional or its last dimension is incorrect. Test one valid and at least two invalid tensors.

Part 4 - Python code skeleton

```
import torch

def E02(x, expected_features):
    if x.ndim != ...:
        raise ValueError(...)
    if x.shape[-1] != ...:
        raise ValueError(...)
    return x.shape

def main():
    valid_x = torch...
    invalid_x = torch...
    print(E02(valid_x, ...))
    print(E02(invalid_x, ...))

if __name__ == "__main__":
    main()
```

10.2.4 Exercise 03 - Single Sample Forward Pass

Part 1 - Why this matters

A single-sample forward pass is the clearest way to observe how one input becomes one prediction. It removes batching from the problem and allows every multiplication and addition

to be checked manually. This is especially valuable when debugging the first implementation of a model.

Even with one sample, shape conventions remain important. A scalar, a one-dimensional feature vector, and a two-dimensional batch containing one row may represent related data but interact differently with model code. The reader must choose and document one representation rather than relying on accidental broadcasting.

Part 2 - Mathematical form

For one sample with F features, let $\mathbf{x} \in \mathbb{R}^F$ and $\mathbf{w} \in \mathbb{R}^F$. The prediction is:

$$\hat{y} = \sum_{j=1}^{F} x_j w_j + b$$

The dot product produces one scalar prediction. If the implementation retains a batch dimension, the input may instead have shape $(1, F)$ and the output shape should reflect one sample.

Part 3 - Hints and Helper

Use a small feature vector and calculate the expected result by hand. Verify feature-count compatibility before computing the dot product. Print the input shape and output shape, and decide whether the returned prediction should be a scalar tensor or a one-element tensor.

Part 4 - Python code skeleton

```
import torch

def E03(x, weight, bias):
    if x.shape[-1] != ...:
        raise ValueError(...)
    prediction = torch... + ...
    return prediction

def main():
    x = torch.tensor([...], dtype=torch.float32)
    weight = torch.tensor([...], dtype=torch.float32)
    bias = torch.tensor(..., dtype=torch.float32)
    print(E03(x, weight, bias))

if __name__ == "__main__":
    main()
```

10.2.5 Exercise 04 - Batch Input Forward Pass

Part 1 - Why this matters

Training models one sample at a time is often inefficient because tensor hardware is designed to process many values together. A batch forward pass applies the same parameters to several samples in one vectorized operation. This improves throughput and matches the structure used

by most training loops.

Batching must not change the meaning of the model. Each row must receive the same weight vector and bias, and its result should agree with a separate single-sample calculation. Comparing vectorized and row-by-row results is therefore an important correctness test.

Part 2 - Mathematical form

Let $\mathbf{X} \in \mathbb{R}^{N \times F}$ contain N samples, $\mathbf{w} \in \mathbb{R}^{F \times 1}$ contain the weights, and $b \in \mathbb{R}$. The batch prediction is:

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b$$

The matrix product has shape $(N, 1)$. The scalar bias broadcasts across the N predictions without creating a separate parameter for each sample.

Part 3 - Hints and Helper

Use `torch.matmul()` or the `@` operator after validating the inner dimensions. Keep the weight as a column tensor if the expected output is $(N, 1)$. Compare the vectorized output with predictions produced by iterating over the rows only for verification, not as the main solution.

Part 4 - Python code skeleton

```
import torch

def E04(X, weight, bias):
    if X.ndim != ... or weight.ndim != ...:
        raise ValueError(...)
    predictions = torch... + ...
    return predictions

def main():
    X = torch.tensor([...], dtype=torch.float32)
    weight = torch.tensor([...], dtype=torch.float32)
    bias = torch.tensor(..., dtype=torch.float32)
    output = E04(X, weight, bias)
    print(output)
    print(output.shape)

if __name__ == "__main__":
    main()
```

10.2.6 Exercise 05 - Random Weight Initialization Test

Part 1 - Why this matters

Before training begins, a model must start with parameter values. Random initialization prevents every new model from beginning with the same prediction behavior, but it also means the initial output and loss can differ between runs. Observing this variation teaches the reader

that initialization is part of an experiment, not an invisible detail.

Linear regression has a simple loss surface, so poor random values can usually be corrected during training. Nevertheless, initialization affects the starting loss, early gradients, and reproducibility. A controlled test should separate the effect of random weights from changes in data or code.

Part 2 - Technical concept and behavior

A weight vector may be sampled from a distribution such as:

$$\mathbf{w}_0 \sim \mathcal{N}(0, \sigma^2)$$

Different seeds produce different samples, while resetting the same seed before initialization reproduces the same tensor. The experiment compares initial predictions from several weight tensors while holding $\mathbf{X}$ and b constant.

Part 3 - Hints and Helper

Use `torch.manual_seed()` and `torch.randn()`. Run the experiment with repeated and different seeds. Store the weights and predictions, then use `torch.equal()` or `torch.allclose()` to test reproducibility. Do not train the parameters in this exercise.

Part 4 - Python code skeleton

```
import torch

def E05(X, feature_count, seeds):
    observations = []
    for seed in seeds:
        torch.manual_seed(...)
        weight = torch.randn(...)
        prediction = ...
        observations.append(...)
    return observations

def main():
    X = torch...
    print(E05(X, ..., [..., ..., ...]))

if __name__ == "__main__":
    main()
```

10.2.7 Exercise 06 - Effect of Bias Term Experiment

Part 1 - Why this matters

The bias allows a linear model to produce a nonzero prediction when every input feature is zero. Without it, the fitted relationship is forced through the origin, which may be an unjustified restriction on the data. Comparing models with different biases makes this structural role visible.

Bias does not change how rapidly a prediction responds to an input; it shifts all predictions by the same amount. This distinction helps the reader separate slope from intercept and later understand why a linear layer stores weights and bias as different parameters.

Part 2 - Mathematical form

For fixed w and two biases b_1 and b_2:

$$\hat{y}_1 = wx + b_1, \qquad \hat{y}_2 = wx + b_2$$

Their difference is independent of the input:

$$\hat{y}_2 - \hat{y}_1 = b_2 - b_1$$

Therefore, changing only the bias translates the prediction line vertically without changing its slope.

Part 3 - Hints and Helper

Keep the input values and weight fixed. Evaluate at least three biases, including zero, and calculate the pairwise output differences. Include $x = 0$ so the connection between bias and intercept is directly observable.

Part 4 - Python code skeleton

```
import torch

def E06(x, weight, bias_values):
    outputs = {}
    for bias in bias_values:
        outputs[...] = ...
    return outputs

def main():
    x = torch.tensor([...], dtype=torch.float32)
    weight = torch.tensor(..., dtype=torch.float32)
    print(E06(x, weight, [..., ..., ...]))

if __name__ == "__main__":
    main()
```

10.2.8 Exercise 07 - Output Sensitivity to Weight Change

Part 1 - Why this matters

Model sensitivity describes how strongly a prediction reacts when a parameter changes. In a linear model, this relationship can be derived exactly, making it an ideal introduction to the reasoning later used by gradients. A small weight change may have little effect for a small input but a large effect for a large input.

Testing sensitivity also reveals why feature scale matters. If one feature has values thousands of times larger than another, the same change in their weights produces very different output changes. This can influence optimization even when both features are conceptually important.

Part 2 - Mathematical form

If the weight changes from w to $w + \Delta w$, then:

$$\Delta\hat{y} = ((w + \Delta w)x + b) - (wx + b) = x\Delta w$$

Thus the sensitivity of the output to the weight is:

$$\frac{\partial\hat{y}}{\partial w} = x$$

The experiment verifies the finite change against this exact prediction.

Part 3 - Hints and Helper

Evaluate several input magnitudes with one fixed `delta_w`. Compute the original and modified predictions, then compare their difference with `x * delta_w`. Include negative inputs and explain how the sign changes the direction of the response.

Part 4 - Python code skeleton

```
import torch

def E07(x, weight, bias, delta_w):
    original = ...
    changed = ...
    observed_delta = ...
    expected_delta = ...
    return original, changed, observed_delta, expected_delta

def main():
    x = torch.tensor([...], dtype=torch.float32)
    result = E07(x, ..., ..., ...)
    print(result)

if __name__ == "__main__":
    main()
```

10.2.9 Exercise 08 - Linear Model as Matrix Multiplication

Part 1 - Why this matters

A multiple-feature linear model is naturally expressed as matrix multiplication. This representation processes an entire batch without explicit Python loops and connects linear regression to the same operation used inside neural-network layers. Understanding the matrix form makes later models easier to inspect.

The matrix form also forces precise shape reasoning. Samples, input features, output features, and bias values occupy different axes. If these roles are confused, the computation may fail or broadcast into a plausible but incorrect result.

Part 2 - Mathematical form

For N samples, F input features, and O outputs:

$$\mathbf{X} \in \mathbb{R}^{N \times F}, \quad \mathbf{W} \in \mathbb{R}^{F \times O}, \quad \mathbf{b} \in \mathbb{R}^{O}$$

The model is:

$$\hat{\mathbf{Y}} = \mathbf{X}\mathbf{W} + \mathbf{b}$$

The output has shape (N, O), and the bias broadcasts across the sample dimension.

Part 3 - Hints and Helper

Write the shapes beside each tensor before coding. Use `torch.matmul()` and verify that the final dimension of $\mathbf{X}$ matches the first dimension of $\mathbf{W}$. Compare one selected output element with a manually calculated dot product plus its bias.

Part 4 - Python code skeleton

```
import torch

def E08(X, W, b):
    if X.shape[-1] != W.shape[...]:
        raise ValueError(...)
    Y_hat = torch.matmul(..., ...) + ...
    return Y_hat

def main():
    X = torch.tensor([...], dtype=torch.float32)
    W = torch.tensor([...], dtype=torch.float32)
    b = torch.tensor([...], dtype=torch.float32)
    print(E08(X, W, b))

if __name__ == "__main__":
    main()
```

10.2.10 Exercise 09 - Manual Prediction vs PyTorch Output

Part 1 - Why this matters

Independent verification protects the reader from trusting an implementation merely because it executes without an exception. A manual prediction exposes the individual products and sums, while the vectorized PyTorch calculation provides the implementation intended for real workloads. Agreement between them is evidence that the tensor formula is correct.

This comparison also introduces numerical tolerance. Floating-point operations may be performed in a different order, so mathematically equivalent implementations can differ by a

very small amount. Exact equality is therefore not always the correct test for floating-point results.

Part 2 - Technical concept and behavior

For sample i, the manual reference is:

$$\hat{y}_i^{\text{manual}} = \sum_{j=1}^{F} X_{ij} w_j + b$$

The vectorized reference is $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b$. Correctness can be tested with:

$$\left| \hat{y}_i^{\text{manual}} - \hat{y}_i^{\text{torch}} \right| \leq \varepsilon$$

for a justified tolerance ε.

Part 3 - Hints and Helper

Build the manual result with explicit indexing or a loop so it is independent of the matrix multiplication being tested. Use `torch.allclose()` for comparison and report the maximum absolute difference. Deliberately alter one weight afterward to confirm that the test can fail.

Part 4 - Python code skeleton

```
import torch

def manual_prediction(X, weight, bias):
    values = []
    for row in X:
        total = ...
        for feature_index in range(...):
            total = ...
        values.append(...)
    return torch...

def E09(X, weight, bias):
    manual = manual_prediction(X, weight, bias)
    vectorized = ...
    matches = torch.allclose(manual, vectorized, atol=..., rtol=...)
    return manual, vectorized, matches

def main():
    X = torch...
    weight = torch...
    bias = torch...
    print(E09(X, weight, bias))

if __name__ == "__main__":
    main()
```

10.2.11 Exercise 10 - Move Model to CUDA Device

Part 1 - Why this matters

CUDA allows compatible PyTorch operations to execute on an NVIDIA GPU, but a model does not move there automatically. Parameters and inputs involved in the same operation must be placed on a compatible device. Making device placement explicit prevents one of the most common runtime errors in model code.

A GPU is not proof of correctness or speed. This exercise is about controlled placement and verification, not benchmarking. The reader will first detect whether CUDA is available, choose a safe fallback, move the model and input together, and confirm where every relevant tensor resides.

Part 2 - Technical concept and behavior

Let d be the selected device:

$$d = \begin{cases} \texttt{cuda}, & \text{if CUDA is available,} \\ \texttt{cpu}, & \text{otherwise.} \end{cases}$$

Calling `model.to(d)` moves registered parameters and buffers. The input must also be moved with `x.to(d)`. The output is created on the device used by the operation.

Part 3 - Hints and Helper

Use `torch.cuda.is_available()` before selecting CUDA. Inspect `next(model.parameters()).devic` `x.device`, and `output.device`. Do not force `cuda:0` on a machine that does not provide CUDA, and do not silently catch device mismatch errors.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def E10(model, x):
    device = torch.device("cuda" if ... else "cpu")
    model = model.to(...)
    x = x.to(...)
    output = model(...)
    return output, device

def main():
    model = nn.Linear(..., ...)
    x = torch...
    output, device = E10(model, x)
    print(device, output.device)

if __name__ == "__main__":
    main()
```

10.2.12 Exercise 11 - CPU vs GPU Forward Pass Comparison

Part 1 - Why this matters

CPU and GPU execution have different performance characteristics. GPUs can process large parallel workloads efficiently, but transferring data and launching kernels adds overhead. A tiny linear model may therefore run faster on a CPU even though a larger batch benefits from the GPU.

A fair comparison must also account for asynchronous CUDA execution. Measuring only the time taken to submit GPU work produces an artificially small value. The reader must use identical parameters and inputs, warm up the devices, synchronize at the correct boundaries, and compare numerical outputs as well as elapsed time.

Part 2 - Technical concept and behavior

For device d, approximate average forward latency over R repetitions as:

$$L_d = \frac{t_{\text{end}} - t_{\text{start}}}{R}$$

CUDA synchronization is required before the start and after the end of the timed region. Data-transfer time must either be excluded from both runs or reported as a separate measurement. Speedup is $S = L_{\text{CPU}}/L_{\text{GPU}}$.

Part 3 - Hints and Helper

Copy one model state to both devices so their parameters are identical. Prepare inputs before timing, call `torch.cuda.synchronize()` around CUDA measurement, and use `torch.no_grad()`. Test several batch sizes rather than drawing a conclusion from one tiny tensor.

Part 4 - Python code skeleton

```
import time
import torch
from torch import nn

def measure_forward(model, x, repetitions, use_cuda):
    ...  # warm-up
    if use_cuda:
        torch.cuda.synchronize()
    started = time.perf_counter()
    with torch.no_grad():
        for _ in range(repetitions):
            output = model(x)
    if use_cuda:
        torch.cuda.synchronize()
    return ..., output

def E11(in_features, out_features, batch_sizes):
    if not torch.cuda.is_available():
```

```
        return ...
    base_model = nn.Linear(in_features, out_features)
    ...  # construct identical CPU and GPU models
    return ...

def main():
    print(E11(..., ..., [...]))

if __name__ == "__main__":
    main()
```

10.2.13 Exercise 12 - Precision Difference (float32 vs float64)

Part 1 - Why this matters

Floating-point data types trade numerical precision against memory use and hardware throughput. Double precision can preserve smaller differences and reduce accumulated rounding error, while single precision is commonly faster and consumes half as many bytes per value. The best choice depends on the scale and sensitivity of the problem.

A precision comparison must begin from equivalent values and parameters. If the two models are initialized separately, random differences can be mistaken for numerical differences. This exercise isolates the effect of `float32` and `float64` by converting the same reference data and model state.

Part 2 - Technical concept and behavior

Let $\hat{\mathbf{y}}_{32}$ and $\hat{\mathbf{y}}_{64}$ be predictions computed from equivalent inputs. Compare them after converting to a common precision:

$$D_{\max} = \max_i |\text{float64}(\hat{y}_{32,i}) - \hat{y}_{64,i}|$$

Memory required by a dense tensor is approximately its number of elements multiplied by the bytes per element.

Part 3 - Hints and Helper

Create a reference model, copy its state, and then call `.float()` and `.double()` on separate copies. Match each input dtype to its model dtype. Test ordinary values and values with a wide numerical range, then report maximum absolute and relative differences.

Part 4 - Python code skeleton

```
import copy
import torch
from torch import nn

def E12(model, x):
    model32 = copy.deepcopy(model).float()
    model64 = copy.deepcopy(model).double()
    x32 = x.to(dtype=torch.float32)
```

```
    x64 = x.to(dtype=torch.float64)
    with torch.no_grad():
        y32 = ...
        y64 = ...
    absolute_difference = ...
    relative_difference = ...
    return y32, y64, absolute_difference, relative_difference

def main():
    model = nn.Linear(..., ...)
    x = torch...
    print(E12(model, x))

if __name__ == "__main__":
    main()
```

10.2.14 Exercise 13 - Multiple Feature Linear Model

Part 1 - Why this matters

Real predictions usually depend on several measurements rather than one. Energy demand may depend on temperature, time, occupancy, and previous usage; each feature receives its own weight. A multiple-feature model combines these contributions while remaining interpretable.

Adding features creates new responsibilities. Their order must remain consistent, units can differ greatly, and correlated features can make individual weights difficult to interpret. This exercise focuses on the forward model and records the feature contract explicitly before later chapters address data preparation and training.

Part 2 - Mathematical form

For sample $\mathbf{x} \in \mathbb{R}^F$:

$$\hat{y} = \mathbf{x}^\mathsf{T}\mathbf{w} + b = \sum_{j=1}^{F} x_j w_j + b$$

For a batch $\mathbf{X} \in \mathbb{R}^{N \times F}$, the prediction vector is $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b$. Each weight corresponds to exactly one ordered feature.

Part 3 - Hints and Helper

Define a list of feature names beside the tensor construction. Assert that the number of names, input columns, and weights agree. Calculate the contribution `X * weight` before summing so that the influence of each feature can be inspected.

Part 4 - Python code skeleton

```
import torch

def E13(X, weight, bias, feature_names):
    if X.shape[-1] != len(feature_names):
```

```
        raise ValueError(...)
    if weight.numel() != ...:
        raise ValueError(...)
    contributions = ...
    predictions = contributions.sum(dim=..., keepdim=...) + ...
    return predictions, contributions

def main():
    feature_names = [...]
    X = torch.tensor([...], dtype=torch.float32)
    weight = torch.tensor([...], dtype=torch.float32)
    bias = torch.tensor(..., dtype=torch.float32)
    print(E13(X, weight, bias, feature_names))

if __name__ == "__main__":
    main()
```

10.2.15 Exercise 14 - Verify Output Dimensions

Part 1 - Why this matters

An output can contain reasonable numbers and still have the wrong structure. A missing output dimension may break concatenation, loss computation, or downstream batching, while an unintended extra dimension may trigger broadcasting and produce a larger error tensor than expected. Output validation protects the interface between model stages.

The expected dimension is determined by the model contract, not by appearance when printed. This exercise requires the reader to calculate the correct shape from batch size and output-feature count before executing the forward pass, then enforce that expectation programmatically.

Part 2 - Technical concept and behavior

A linear transformation maps:

$$(*, F_{\text{in}}) \longrightarrow (*, F_{\text{out}})$$

where $*$ represents any preserved leading dimensions. For a two-dimensional batch $\mathbf{X} \in \mathbb{R}^{N \times F_{\text{in}}}$, the expected output is $\hat{\mathbf{Y}} \in \mathbb{R}^{N \times F_{\text{out}}}$.

Part 3 - Hints and Helper

Construct the expected shape from `x.shape[:-1]` and the declared output-feature count. Compare tuples exactly and raise an informative error that includes both expected and observed shapes. Test several batch sizes and at least two output sizes.

Part 4 - Python code skeleton

```
import torch
from torch import nn
```

```
def E14(model, x, expected_out_features):
    output = model(x)
    expected_shape = ... + (...)
    if tuple(output.shape) != expected_shape:
        raise ValueError(
            f"expected {expected_shape}, observed {tuple(output.shape)}"
        )
    return output

def main():
    model = nn.Linear(..., ...)
    x = torch...
    print(E14(model, x, ...).shape)

if __name__ == "__main__":
    main()
```

10.2.16 Exercise 15 - Linear Layer Using nn.Linear

Part 1 - Why this matters

The `torch.nn.Linear` module packages a learnable affine transformation as a reusable model component. It registers its weight and optional bias as parameters, participates in device and dtype conversion, and connects automatically to PyTorch's gradient system. This is the standard bridge from the manually written equation to a trainable layer.

Using the module should not hide its mathematics. The reader must understand how PyTorch stores the weight, how the input's final dimension is interpreted, and how the module's output relates to the matrix equation already studied. A correct comparison requires accounting for the orientation of the stored weight matrix.

Part 2 - Mathematical form

For `nn.Linear(in_features, out_features)`, PyTorch represents:

$$\mathbf{Y} = \mathbf{X}\mathbf{W}^{\mathsf{T}} + \mathbf{b}$$

where $\mathbf{W} \in \mathbb{R}^{F_{\text{out}} \times F_{\text{in}}}$ and $\mathbf{b} \in \mathbb{R}^{F_{\text{out}}}$. The module preserves leading input dimensions and replaces the final feature dimension.

Part 3 - Hints and Helper

Instantiate `nn.Linear()` with explicit feature counts. Print the parameter shapes before the forward call and compare the module output with `X @ layer.weight.T + layer.bias`. Use `torch.allclose()` to verify agreement.

Part 4 - Python code skeleton

```
import torch
from torch import nn
```

```
def E15(X, in_features, out_features):
    layer = nn.Linear(..., ...)
    module_output = layer(...)
    manual_output = ... @ ... + ...
    matches = torch.allclose(module_output, manual_output)
    return layer, module_output, matches

def main():
    X = torch...
    layer, output, matches = E15(X, ..., ...)
    print(layer.weight.shape, layer.bias.shape)
    print(output, matches)

if __name__ == "__main__":
    main()
```

10.2.17 Exercise 16 - Inspect Weight and Bias Parameters

Part 1 - Why this matters

Training changes parameters, so the reader must be able to identify them before examining gradients or optimizer behavior. A linear layer exposes weight and bias tensors with distinct shapes and roles. Inspection confirms that they are registered, trainable, and located on the expected device.

Parameter inspection is also a debugging tool. A missing bias, unexpected feature count, frozen parameter, or incorrect dtype can be detected immediately rather than after a failed training run. The goal is to produce a structured report, not merely print the entire tensor without context.

Part 2 - Technical concept and behavior

For F_{in} input features and F_{out} outputs:

$$\text{shape}(\mathbf{W}) = (F_{\text{out}}, F_{\text{in}}), \qquad \text{shape}(\mathbf{b}) = (F_{\text{out}})$$

Each registered `Parameter` also has a device, dtype, and `requires_grad` property. The total parameter count is $F_{\text{out}}F_{\text{in}} + F_{\text{out}}$ when bias is enabled.

Part 3 - Hints and Helper

Iterate over `model.named_parameters()` and record the name, shape, number of elements, dtype, device, and gradient flag. Calculate the expected total independently, then compare it with the sum of `numel()`.

Part 4 - Python code skeleton

```
from torch import nn

def E16(model):
    report = []
```

```
    total = 0
    for name, parameter in model.named_parameters():
        report.append({
            "name": ...,
            "shape": ...,
            "count": ...,
            "dtype": ...,
            "device": ...,
            "requires_grad": ...,
        })
        total += ...
    return report, total

def main():
    model = nn.Linear(..., ...)
    report, total = E16(model)
    print(report)
    print(total)

if __name__ == "__main__":
    main()
```

10.2.18 Exercise 17 - Freeze Parameters Test

Part 1 - Why this matters

Freezing a parameter prevents automatic differentiation from storing a gradient for it. This is useful when part of a model should remain fixed during fine-tuning or when an experiment needs to isolate the effect of another parameter. In a linear model, freezing the weight while leaving the bias trainable makes the behavior easy to observe.

Freezing does not remove a parameter from the forward equation. The weight still affects predictions; it simply should not receive a gradient-based update. The reader must distinguish computational participation from trainability and verify both after the backward pass.

Part 2 - Technical concept and behavior

If `weight.requires_grad=False`, the forward model remains:

$$\hat{\mathbf{Y}} = \mathbf{X}\mathbf{W}^{\mathsf{T}} + \mathbf{b}$$

but gradient computation targets only trainable parameters. After scalar loss $\mathcal{L}$ is back-propagated, the frozen weight should have no stored gradient, while an unfrozen bias may have $\partial\mathcal{L}/\partial\mathbf{b}$.

Part 3 - Hints and Helper

Set the gradient flag before the forward pass, construct a scalar loss, and call `backward()`. Inspect both `weight.grad` and `bias.grad`. If an optimizer is used for confirmation, build it only from parameters whose gradient flag is true and verify the frozen value remains unchanged.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def E17(model, X, target):
    model.weight.requires_grad_(...)
    before_weight = model.weight.detach().clone()
    prediction = model(X)
    loss = ...
    loss.backward()
    return {
        "weight_unchanged": ...,
        "weight_grad": ...,
        "bias_grad": ...,
    }

def main():
    model = nn.Linear(..., ...)
    X = torch...
    target = torch...
    print(E17(model, X, target))

if __name__ == "__main__":
    main()
```

10.2.19 Exercise 18 - Manual Forward Without nn.Module

Part 1 - Why this matters

A model can be computed with ordinary tensors without inheriting from `nn.Module`. Rebuilding the forward pass manually confirms that the framework module is a structured container around the same affine equation, not a different mathematical law. This strengthens the connection between tensor algebra and model code.

The manual version also exposes what `nn.Module` normally provides: parameter registration, convenient device conversion, state dictionaries, mode management, and integration with model utilities. The exercise should reveal both mathematical equivalence and the engineering benefits that are absent.

Part 2 - Mathematical form

Given an input $\mathbf{X}$ and tensors stored in the same orientation as `nn.Linear`:

$$\hat{\mathbf{Y}} = \mathbf{X}\mathbf{W}^{\mathsf{T}} + \mathbf{b}$$

If `requires_grad=True` is set on the manual tensors, autograd can still compute gradients. However, these tensors are not automatically returned by `model.parameters()` because no module owns them.

Part 3 - Hints and Helper

Create weight and bias tensors explicitly and implement a plain function. For verification, copy them into an `nn.Linear` layer inside `torch.no_grad()`, then compare outputs. Inspect the objects available from the module and explain what the plain function does not manage.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def manual_forward(X, weight, bias):
    return ... @ ... + ...

def E18(X, weight, bias):
    manual_output = manual_forward(X, weight, bias)
    layer = nn.Linear(..., ...)
    with torch.no_grad():
        layer.weight.copy_(...)
        layer.bias.copy_(...)
    module_output = layer(X)
    return manual_output, module_output, torch.allclose(..., ...)

def main():
    X = torch...
    weight = torch...
    bias = torch...
    print(E18(X, weight, bias))

if __name__ == "__main__":
    main()
```

10.2.20 Exercise 19 - Forward Pass Failure Debugging

Part 1 - Why this matters

A forward pass can fail because of incompatible shapes, devices, data types, missing bias assumptions, or invalid numerical values. Effective debugging begins by turning the model's input contract into observable checks rather than repeatedly changing code until an exception disappears. Fixing only the first visible error may leave the underlying data problem unresolved.

The final exercise combines the chapter's reasoning skills into a diagnostic workflow. The reader will reproduce several controlled failures, classify each cause, and return a concise report. The goal is not to suppress exceptions, but to provide enough evidence to correct the source safely.

Part 2 - Technical concept and behavior

For `nn.Linear(F_in, F_out)`, a valid input must have final dimension F_{in}, a compatible floating-point dtype, and the same device as the parameters. Before the operation, verify:

$$X_{\text{last}} = F_{\text{in}}, \qquad d_X = d_W, \qquad \tau_X = \tau_W$$

The output should also be checked for non-finite values using a deliberate policy.

Part 3 - Hints and Helper

Write a validator that reports shape, dtype, device, expected input features, and whether all values are finite. Test it against wrong feature count, integer input, and device mismatch when CUDA is available. Catch an exception only at the outer experiment boundary so the original error type and message remain visible.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def inspect_forward_contract(model, X):
    parameter = next(model.parameters())
    checks = {
        "shape_valid": ...,
        "dtype_valid": ...,
        "device_valid": ...,
        "finite_input": ...,
    }
    return checks

def E19(model, test_inputs):
    report = []
    for label, X in test_inputs.items():
        checks = inspect_forward_contract(model, X)
        try:
            output = model(X)
            status = ...
        except (RuntimeError, ValueError, TypeError) as error:
            status = {
                "error_type": type(error).__name__,
                "message": str(error),
            }
        report.append((label, checks, status))
    return report

def main():
    model = nn.Linear(..., ...)
    test_inputs = {
        "valid": torch...,
        "wrong_shape": torch...,
        "wrong_dtype": torch...,
    }
    print(E19(model, test_inputs))

if __name__ == "__main__":
    main()
```

10.3 LR01 - Model Structure and Components

Models and Training

10.3.1 Exercise 00 - Separate Weights and Bias Tensors

Part 1 - Why this matters

A linear model contains two parameter roles that should remain conceptually separate. Weights connect input features to outputs, while bias provides an additive baseline. Storing them in different tensors makes their shapes, contributions, and future gradients visible before they are placed inside a model class.

This separation also supports precise debugging. If a prediction is incorrect, the reader can inspect the weighted contribution and the bias contribution independently. It becomes easier to distinguish a wrong matrix shape from an unintended intercept or broadcasting rule.

Part 2 - Mathematical form

For F input features and O outputs:

$$\mathbf{W} \in \mathbb{R}^{O \times F}, \qquad \mathbf{b} \in \mathbb{R}^{O}$$

The batch forward pass is:

$$\hat{\mathbf{Y}} = \mathbf{X}\mathbf{W}^{\mathsf{T}} + \mathbf{b}$$

The bias broadcasts across samples but not across unrelated output definitions.

Part 3 - Hints and Helper

Create weight and bias as independent floating-point tensors. Print their shapes and calculate each contribution separately. Keep the storage orientation consistent with `nn.Linear`: output features first and input features second.

Part 4 - Python code skeleton

```
import torch

def E00(X, in_features, out_features):
    weight = torch...
    bias = torch...
    weighted = X @ ...
    output = weighted + ...
    return weight, bias, output

def main():
    X = torch...
    weight, bias, output = E00(X, ..., ...)
    print(weight.shape, bias.shape, output.shape)

if __name__ == "__main__":
    main()
```

10.3.2 Exercise 01 - Parameter Registration in nn.Module

Part 1 - Why this matters

An ordinary tensor assigned to a model attribute is not automatically treated as a trainable parameter. PyTorch uses `nn.Parameter` to mark tensors that belong to a module's learnable state. Registered parameters appear in parameter iterators, move with the module, enter state dictionaries, and can be supplied to an optimizer.

Understanding registration prevents a quiet failure in which the forward pass works but training never changes the intended tensor. The model can produce predictions from an unregistered tensor, so successful execution alone does not prove that the structure is trainable.

Part 2 - Technical concept and behavior

A custom module represents:

$$f(\mathbf{X}; \mathbf{W}, \mathbf{b}) = \mathbf{X}\mathbf{W}^\mathsf{T} + \mathbf{b}$$

When $\mathbf{W}$ and $\mathbf{b}$ are assigned as `nn.Parameter` attributes after calling the parent constructor, they are returned by `named_parameters()`. An ordinary tensor attribute is not included unless registered through an appropriate module mechanism.

Part 3 - Hints and Helper

Subclass `nn.Module`, call `super().__init__()` first, and wrap the learnable tensors with `nn.Parameter`. Add one ordinary tensor attribute for comparison. Inspect `dict(model.named_parameters` and explain why only selected attributes appear.

Part 4 - Python code skeleton

```
import torch
from torch import nn

class LinearModel(nn.Module):
    def __init__(self, in_features, out_features):
        super().__init__()
        self.weight = nn.Parameter(torch...)
        self.bias = nn.Parameter(torch...)
        self.temporary = torch...

    def forward(self, X):
        return ...

def E01(model):
    registered = dict(model.named_parameters())
    return ...

def main():
    model = LinearModel(..., ...)
    print(E01(model))
```

```
if __name__ == "__main__":
    main()
```

10.3.3 Exercise 02 - Inspect requires_grad Flags

Part 1 - Why this matters

The `requires_grad` flag determines whether autograd should track operations needed to calculate a tensor's gradient. A model may contain registered parameters that are intentionally frozen and others that remain trainable. Inspecting the flag makes this training boundary explicit.

Registration and gradient tracking are related but different ideas. A parameter can remain part of the model state while its gradient flag is disabled. Confusing the two can cause incorrect parameter counts, unexpected optimizer behavior, or mistaken conclusions about whether a layer participates in the forward pass.

Part 2 - Technical concept and behavior

For model parameters $\theta_1, \ldots, \theta_P$, define the trainable subset:

$$\Theta_{\text{train}} = \{\theta_i \mid \theta_i.\texttt{requires_grad} = \text{True}\}$$

Only operations involving this subset need to create the corresponding parameter gradients. A frozen parameter still contributes its numerical value to the output.

Part 3 - Hints and Helper

Iterate through `named_parameters()` and return each name with its flag. Freeze only the weight with `requires_grad_()` and repeat the report. Do not use `torch.no_grad()` as a substitute for changing a parameter's trainability.

Part 4 - Python code skeleton

```
from torch import nn

def inspect_flags(model):
    return {
        name: parameter.requires_grad
        for name, parameter in ...
    }

def E02(model):
    before = inspect_flags(model)
    model.weight.requires_grad_(...)
    after = inspect_flags(model)
    return before, after

def main():
    model = nn.Linear(..., ...)
    print(E02(model))
```

```
if __name__ == "__main__":
    main()
```

10.3.4 Exercise 03 - Count Trainable Parameters

Part 1 - Why this matters

Parameter count describes the number of scalar values a model can adjust. It helps estimate model capacity, memory requirements, and optimizer-state cost. Counting only modules or tensor objects is insufficient because one tensor may contain many scalar parameters.

The trainable count can differ from the total registered count when some parameters are frozen. Reporting both values prevents an architectural description from being confused with the subset currently selected for learning.

Part 2 - Mathematical form

For a linear layer with F inputs and O outputs, including bias:

$$P_{\text{total}} = OF + O$$

For an arbitrary module:

$$P_{\text{train}} = \sum_{\theta \in \Theta} \text{numel}(\theta) \mathbb{1}[\theta.\texttt{requires_grad}]$$

Shared parameters should not be counted twice when the objective is unique storage.

Part 3 - Hints and Helper

Use `parameter.numel()` and compute total and trainable counts separately. Compare the result with the analytical formula for one linear layer. Then freeze the bias and confirm that only the trainable count changes.

Part 4 - Python code skeleton

```
from torch import nn

def E03(model):
    total = sum(... for parameter in model.parameters())
    trainable = sum(
        ...
        for parameter in model.parameters()
        if ...
    )
    return total, trainable

def main():
    model = nn.Linear(..., ...)
    print(E03(model))
    model.bias.requires_grad_(...)
```

```
    print(E03(model))

if __name__ == "__main__":
    main()
```

10.3.5 Exercise 04 - Initialize Model on CUDA

Part 1 - Why this matters

A model intended for GPU training should be created or moved to CUDA before the optimizer is constructed. Device conversion affects registered parameters and buffers, and PyTorch documents that CUDA conversion can make associated parameters different objects. Establishing device placement first avoids stale optimizer references.

CUDA availability must never be assumed. A reusable experiment detects support, selects a device explicitly, and provides a CPU fallback. This makes the model runnable across development machines while keeping the chosen execution path visible.

Part 2 - Technical concept and behavior

Select device d as:

$$d = \begin{cases} \texttt{cuda}, & \text{if available,} \\ \texttt{cpu}, & \text{otherwise.} \end{cases}$$

A module initialized with compatible device arguments or moved using `to(d)` should have every registered parameter on d. The optimizer, if introduced later, must be created after this placement step.

Part 3 - Hints and Helper

Use `torch.cuda.is_available()` and `torch.device`. Construct the model, move it to the selected device, and inspect every parameter. Return the device rather than hiding the fallback inside the function.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def E04(in_features, out_features):
    device = torch.device("cuda" if ... else "cpu")
    model = nn.Linear(..., ...)
    model = model.to(...)
    parameter_devices = {
        name: parameter.device
        for name, parameter in model.named_parameters()
    }
    return model, device, parameter_devices

def main():
```

```
    model, device, report = E04(..., ...)
    print(device)
    print(report)

if __name__ == "__main__":
    main()
```

10.3.6 Exercise 05 - Verify Parameter Device Placement

Part 1 - Why this matters

Moving a model is useful only when all of its computational state reaches the intended device. Checking one parameter can miss a misplaced submodule or buffer in a larger architecture. A complete device report gives evidence that the model is ready to receive inputs on the same device.

Device verification should include the data used by the forward pass. A perfectly placed model still fails if the input or target remains on another device. This exercise builds a reusable contract check before any expensive training begins.

Part 2 - Technical concept and behavior

For a valid forward operation on device d, require:

$$d_{\theta_i} = d \quad \text{for every parameter } \theta_i, \qquad d_X = d$$

Registered buffers involved in the operation should satisfy the same condition. The verification result is false if even one required tensor differs.

Part 3 - Hints and Helper

Inspect `named_parameters()` and `named_buffers()`. Report the exact name and observed device for failures. Accept the expected device as an argument, and validate an input tensor separately from the model state.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def E05(model, X, expected_device):
    mismatches = []
    for name, parameter in model.named_parameters():
        if parameter.device != expected_device:
            mismatches.append(...)
    for name, buffer in model.named_buffers():
        if buffer.device != expected_device:
            mismatches.append(...)
    if X.device != expected_device:
        mismatches.append(...)
    return len(mismatches) == 0, mismatches
```

```
def main():
    device = torch.device(...)
    model = nn.Linear(..., ...).to(device)
    X = torch...
    print(E05(model, X, device))

if __name__ == "__main__":
    main()
```

10.3.7 Exercise 06 - Manual Parameter Update Test

Part 1 - Why this matters

Learning occurs when computed gradients are converted into parameter changes. Performing one manual update exposes the exact arithmetic normally handled by an optimizer. The reader can observe the parameter before the step, its gradient, and the value produced afterward.

A manual update must not become part of the recorded autograd graph. PyTorch parameters requiring gradients are leaf tensors, so modifying them carelessly in place can raise an error or create incorrect graph behavior. The update should occur in a no-gradient context and only after gradients have been calculated.

Part 2 - Mathematical form

For learning rate $\eta > 0$ and parameter θ:

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla_\theta \mathcal{L}$$

The expected change is:

$$\Delta\theta = -\eta \nabla_\theta \mathcal{L}$$

A test should compare the observed tensor difference with this expression.

Part 3 - Hints and Helper

Clone the parameter before updating it. Compute a scalar loss and call `backward()`, then update inside `torch.no_grad()`. Do not use `.data`. Check that the observed change matches the gradient rule and clear gradients before another experiment.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def E06(model, X, target, learning_rate):
    prediction = model(X)
    loss = ...
    loss.backward()
    before = {
```

```
        name: parameter.detach().clone()
        for name, parameter in model.named_parameters()
    }
    with torch.no_grad():
        for parameter in model.parameters():
            parameter -= ...
    observed_changes = ...
    return loss.item(), before, observed_changes

def main():
    model = nn.Linear(..., ...)
    print(E06(model, torch..., torch..., ...))

if __name__ == "__main__":
    main()
```

10.3.8 Exercise 07 - Parameter Sharing Experiment

Part 1 - Why this matters

Parameter sharing means that several computation paths use the same underlying learnable tensor. It reduces unique storage and forces those paths to learn a common transformation. Shared weights appear in recurrent structures, tied embeddings, and models that apply one operation repeatedly.

Sharing is different from copying equal values. Two independent parameters may begin identically and later diverge, while shared references remain the same object and accumulate gradient contributions from every path that uses them. This exercise makes that distinction observable.

Part 2 - Technical concept and behavior

If one parameter θ contributes through paths f_1 and f_2, then:

$$\frac{\partial \mathcal{L}}{\partial \theta} = \left.\frac{\partial \mathcal{L}}{\partial \theta}\right|_{f_1} + \left.\frac{\partial \mathcal{L}}{\partial \theta}\right|_{f_2}$$

Object identity and storage identity can be inspected separately from numerical equality. Unique-parameter iteration commonly removes duplicates by default.

Part 3 - Hints and Helper

Construct a model that applies the same `nn.Linear` instance twice. Compare it with two independently created layers whose initial state is copied. Inspect object identity, parameter data pointers, unique parameter count, and gradients after a scalar loss.

Part 4 - Python code skeleton

```
import copy
import torch
from torch import nn
```

```
class SharedModel(nn.Module):
    def __init__(self, layer):
        super().__init__()
        self.first = ...
        self.second = ...

    def forward(self, X):
        return self.first(X) + self.second(X)

def E07(X):
    base = nn.Linear(..., ...)
    shared_model = SharedModel(base)
    independent_first = copy.deepcopy(base)
    independent_second = copy.deepcopy(base)
    ...  # compare identity, storage, outputs, and gradients
    return ...

def main():
    print(E07(torch...))

if __name__ == "__main__":
    main()
```

10.3.9 Exercise 08 - Reinitialize Model Weights

Part 1 - Why this matters

Reinitialization returns model parameters to a new starting state without rebuilding the surrounding experiment. It is useful when comparing repeated training runs, recovering from an intentionally bad setup, or applying a defined initialization policy across a model.

Reinitialization must occur before training statistics from the new run are collected. Optimizer state from a previous run may still contain momentum or adaptive estimates, so resetting model weights alone is not equivalent to starting an entirely fresh experiment.

Part 2 - Technical concept and behavior

A chosen initializer samples or assigns new values according to a defined rule. For example:

$$W_{ij} \sim \mathcal{U}(-a, a), \qquad b_j = 0$$

The operation changes parameter values but should preserve parameter shapes, registration, dtype, device, and gradient flags.

Part 3 - Hints and Helper

Write an initialization function for `nn.Linear` and apply it with `model.apply()`. Use `torch.no_grad()` for direct filling. Clone the old values and verify that values changed while structural properties remained constant. Recreate the optimizer after reinitialization if an optimizer had already accumulated state.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def initialize_linear(module):
    if isinstance(module, nn.Linear):
        with torch.no_grad():
            module.weight.uniform_(...)
            if module.bias is not None:
                module.bias.fill_(...)

def E08(model):
    before = {
        name: parameter.detach().clone()
        for name, parameter in model.named_parameters()
    }
    model.apply(...)
    after = ...
    checks = ...
    return before, after, checks

def main():
    model = nn.Sequential(nn.Linear(..., ...), nn.Linear(..., ...))
    print(E08(model))

if __name__ == "__main__":
    main()
```

10.3.10 Exercise 09 - Effect of Bad Initialization

Part 1 - Why this matters

Initialization values that are excessively large can create huge predictions, losses, and gradients before meaningful learning begins. Values that remove useful distinctions can also produce restricted behavior. The reader should recognize these symptoms from measurements rather than label an initialization bad only after training fails.

Linear regression offers a controlled experiment because the direct effect of parameter scale is easy to calculate. Comparing several starting scales with identical data reveals how initialization changes the initial loss and gradient norm while every other condition remains fixed.

Part 2 - Mathematical form

If weights are scaled by factor c:

$$\hat{\mathbf{Y}}_c = \mathbf{X}(c\mathbf{W})^\top + \mathbf{b}$$

For mean squared error, large prediction magnitudes may produce errors that grow approximately quadratically with scale. The experiment records finite status, loss, and gradient norm for each initialization policy.

Part 3 - Hints and Helper

Use cloned models and identical inputs and targets. Test zero, moderate, and very large weight scales. After each backward pass, calculate a combined gradient norm and check `torch.isfinite()` for predictions, loss, and gradients. Do not perform a long training run.

Part 4 - Python code skeleton

```
import copy
import torch
from torch import nn

def evaluate_initialization(model, X, target, scale):
    candidate = copy.deepcopy(model)
    with torch.no_grad():
        candidate.weight.fill_(...)
        candidate.bias.fill_(...)
    prediction = candidate(X)
    loss = ...
    loss.backward()
    gradient_norm = ...
    return {
        "scale": scale,
        "loss": ...,
        "gradient_norm": ...,
        "finite": ...,
    }

def E09(model, X, target, scales):
    return [evaluate_initialization(model, X, target, s) for s in scales]

def main():
    model = nn.Linear(..., ...)
    print(E09(model, torch..., torch..., [..., ..., ...]))

if __name__ == "__main__":
    main()
```

10.3.11 Exercise 10 - Bias-only Model Behavior

Part 1 - Why this matters

A bias-only model ignores all input variation and predicts the same learned value for every sample. Although extremely limited, it provides a useful baseline: a more complex regression model should demonstrate value beyond a constant prediction.

Freezing or removing the weight also isolates the bias gradient. The reader can observe how one scalar or output vector adjusts toward the overall target level while remaining unable to explain differences between samples.

Part 2 - Mathematical form

With zero or absent weights:

$$\hat{y}_i = b \quad \text{for every sample } i$$

Under mean squared error, the best constant prediction for a dataset is its target mean:

$$b^\star = \frac{1}{N}\sum_{i=1}^{N} y_i$$

This relationship provides a reference for checking the experiment.

Part 3 - Hints and Helper

Construct a module containing only an `nn.Parameter` bias. Use the batch size to expand the prediction without creating separate learnable biases per sample. Compare outputs from very different inputs and verify that input changes do not alter predictions.

Part 4 - Python code skeleton

```
import torch
from torch import nn

class BiasOnlyModel(nn.Module):
    def __init__(self, out_features):
        super().__init__()
        self.bias = nn.Parameter(torch...)

    def forward(self, X):
        batch_shape = X.shape[:-1]
        return self.bias.expand(...)

def E10(X, out_features):
    model = BiasOnlyModel(...)
    output = model(X)
    return model, output

def main():
    X = torch...
    model, output = E10(X, ...)
    print(output)

if __name__ == "__main__":
    main()
```

10.3.12 Exercise 11 - Weight-only Model Behavior

Part 1 - Why this matters

A weight-only linear model has no additive intercept, so every prediction is determined entirely by the input. This forces the learned relationship through the origin. The restriction

can be appropriate in some physical systems but harmful when the target has a nonzero baseline.

Comparing weight-only and full affine models reveals that bias is a structural assumption rather than a decorative parameter. The reader should identify cases where the missing intercept produces systematic residual error even when the slope is reasonable.

Part 2 - Mathematical form

Without bias:

$$\hat{\mathbf{Y}} = \mathbf{X}\mathbf{W}^{\mathsf{T}}$$

Consequently:

$$f(\mathbf{0}) = \mathbf{0}$$

A standard linear layer can enforce this structure by setting `bias=False`. Its parameter count is then OF instead of $OF + O$.

Part 3 - Hints and Helper

Create two layers with identical weights, one with bias and one without. Set a known nonzero bias in the full layer, then test zero input and nonzero input. Verify parameter names and counts as well as predictions.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def E11(X, in_features, out_features):
    weight_only = nn.Linear(..., ..., bias=False)
    with_bias = nn.Linear(..., ..., bias=True)
    with torch.no_grad():
        with_bias.weight.copy_(weight_only.weight)
        with_bias.bias.fill_(...)
    output_weight_only = ...
    output_with_bias = ...
    return ..., ..., list(weight_only.named_parameters())

def main():
    X = torch...
    print(E11(X, ..., ...))

if __name__ == "__main__":
    main()
```

10.3.13 Exercise 12 - Model Copy vs Reference Test

Part 1 - Why this matters

Assigning a model to another variable creates another reference to the same object; it does not create an independent model. Updating parameters through either name changes the single shared instance. This can invalidate experiments that appear to compare two models but actually modify one.

A deep copy creates independent parameter storage with initially equal values. The reader must verify both identity and behavior because numerical equality at the start does not reveal whether later updates will remain isolated.

Part 2 - Technical concept and behavior

Let $M_2 = M_1$ denote reference assignment. Then both names identify the same object and parameters. For a deep copy $M_3 = \text{deepcopy}(M_1)$:

$$\theta(M_3) = \theta(M_1) \quad \text{initially, but} \quad \theta(M_3) \not\equiv \theta(M_1)$$

where $\not\equiv$ indicates independent object or storage identity.

Part 3 - Hints and Helper

Use `is` to test object identity and compare parameter data pointers for storage. Modify one model inside `torch.no_grad()` and inspect all three afterward. Use `copy.deepcopy()`, not a second variable assignment, for the independent case.

Part 4 - Python code skeleton

```
import copy
import torch
from torch import nn

def E12(model):
    reference = ...
    independent = copy.deepcopy(...)
    before = ...
    with torch.no_grad():
        model.weight.add_(...)
    checks = {
        "same_model_object": ...,
        "reference_changed": ...,
        "independent_changed": ...,
        "shared_storage": ...,
    }
    return checks

def main():
    print(E12(nn.Linear(..., ...)))

if __name__ == "__main__":
    main()
```

10.3.14 Exercise 13 - State Dict Inspection

Part 1 - Why this matters

A state dictionary provides named access to a module's persistent state. It contains registered parameters and persistent buffers, making it the standard boundary for checkpoints, model comparison, and weight transfer. Inspecting its keys reveals the hierarchy encoded by nested module names.

The returned state dictionary is a shallow mapping whose tensor values refer to model state. If the reader needs a frozen in-memory snapshot for later comparison, those tensors must be cloned or deep-copied. Otherwise, later model updates may appear inside the supposed snapshot.

Part 2 - Technical concept and behavior

For module state S:

$$S = \{\text{name} \mapsto \text{tensor}\}$$

A simple linear layer typically exposes `weight` and `bias`. Nested models use qualified keys such as `layer1.weight`. Each entry should be inspected by key, shape, dtype, device, and number of elements.

Part 3 - Hints and Helper

Call `model.state_dict()` and build a metadata report without changing the tensors. Create a cloned snapshot with a dictionary comprehension, update the model, and compare the live state with the snapshot to demonstrate reference behavior safely.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def E13(model):
    state = model.state_dict()
    report = {
        key: {
            "shape": ...,
            "dtype": ...,
            "device": ...,
            "count": ...,
        }
        for key, value in state.items()
    }
    snapshot = {
        key: value.detach().clone()
        for key, value in state.items()
    }
    return report, snapshot
```

```
def main():
    model = nn.Sequential(nn.Linear(..., ...), nn.Linear(..., ...))
    report, snapshot = E13(model)
    print(report)

if __name__ == "__main__":
    main()
```

10.3.15 Exercise 14 - Save and Load Model Weights

Part 1 - Why this matters

Saving weights preserves learned state independently of the running Python process. Loading them into a correctly constructed model allows evaluation, resumed training, or deployment without repeating the original optimization. A reliable workflow verifies the restored output instead of assuming that file creation was sufficient.

The architecture and state play different roles. A state dictionary stores parameter and persistent-buffer tensors, but the receiving model must be constructed with a compatible structure. Missing or unexpected keys and incompatible shapes should be treated as meaningful errors.

Part 2 - Technical concept and behavior

If S_A is saved from model A and loaded into compatible model B, then:

$$S_B \leftarrow S_A$$

For the same input $\mathbf{X}$ and deterministic forward behavior, the predictions should satisfy:

$$f_A(\mathbf{X}) \approx f_B(\mathbf{X})$$

within an appropriate floating-point tolerance.

Part 3 - Hints and Helper

Save `model.state_dict()` with `torch.save()`. Construct a fresh compatible model and load using `torch.load(..., map_location=..., weights_only=True)` followed by `load_state_dict()`. Use a caller-provided path, verify the returned key information, and compare predictions.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def E14(model, fresh_model, X, checkpoint_path, device):
    with torch.no_grad():
        expected = model(X)
    torch.save(..., checkpoint_path)
    state = torch.load(
        checkpoint_path,
        map_location=device,
```

```
        weights_only=True,
    )
    load_result = fresh_model.load_state_dict(...)
    fresh_model.to(device)
    with torch.no_grad():
        restored = fresh_model(X.to(device))
    return load_result, expected, restored, torch.allclose(..., ...)

def main():
    model = nn.Linear(..., ...)
    fresh_model = nn.Linear(..., ...)
    print(E14(model, fresh_model, torch..., ..., torch.device("cpu")))

if __name__ == "__main__":
    main()
```

10.3.16 Exercise 15 - Device Mismatch Error Debugging

Part 1 - Why this matters

A model and its input may each be valid but incompatible when stored on different devices. The resulting failure often appears during matrix multiplication, yet the real problem is placement rather than shape or mathematics. A preflight check produces a clearer explanation before the forward call.

Debugging should not automatically move every tensor without permission. Silent movement can create unexpected transfer costs or place data on the wrong GPU. The function should report the mismatch and let the caller choose the intended device policy.

Part 2 - Technical concept and behavior

For a forward pass using input $\mathbf{X}$ and parameter $\mathbf{W}$:

$$d_X = d_W$$

must hold. In a model with several parameters and buffers, the check extends to every tensor involved. A mixed-device module is itself a structural warning even before an input is supplied.

Part 3 - Hints and Helper

Collect devices from `named_parameters()` and `named_buffers()`, then compare the input. If CUDA is available, construct a controlled mismatch and capture the original exception at the experiment boundary. Return a diagnostic report before applying the explicit fix.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def inspect_devices(model, X):
```

```
    model_devices = {
        str(tensor.device)
        for tensor in list(model.parameters()) + list(model.buffers())
    }
    return {
        "model_devices": ...,
        "input_device": ...,
        "compatible": ...,
    }

def E15(model, X):
    diagnostic = inspect_devices(model, X)
    try:
        output = model(X)
        result = ...
    except RuntimeError as error:
        result = {"type": type(error).__name__, "message": str(error)}
    return diagnostic, result

def main():
    model = nn.Linear(..., ...)
    X = torch...
    print(E15(model, X))

if __name__ == "__main__":
    main()
```

10.3.17 Exercise 16 - Gradient Flow Verification

Part 1 - Why this matters

A model cannot learn if the loss is disconnected from its trainable parameters. The forward output may look correct even when a detach operation, disabled gradient flag, or non-differentiable conversion has broken the path. Gradient-flow verification checks the learning connection directly.

The presence of a gradient tensor is only the first test. Its shape must match the parameter, its values should be finite, and its magnitude should be interpreted in the context of the input and loss. Zero gradients can be mathematically correct, so the experiment must use data designed to produce a nonzero signal.

Part 2 - Mathematical form

For loss $\mathcal{L}$ and parameter θ, a connected differentiable graph allows:

$$\nabla_\theta \mathcal{L} = \frac{\partial \mathcal{L}}{\partial \theta}$$

After `backward()`, verify gradient existence, shape compatibility, finite values, and norm. A missing gradient is represented by `None`, which differs from a tensor filled with zeros.

Part 3 - Hints and Helper

Clear old gradients, perform a forward pass, create a scalar loss, and call `backward()`. Build a report for every named parameter. Repeat once with an intentionally detached prediction and capture the failure or missing connection without hiding it.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def gradient_report(model):
    report = {}
    for name, parameter in model.named_parameters():
        gradient = parameter.grad
        report[name] = {
            "exists": ...,
            "shape_matches": ...,
            "finite": ...,
            "norm": ...,
        }
    return report

def E16(model, X, target):
    model.zero_grad(set_to_none=True)
    prediction = model(X)
    loss = ...
    loss.backward()
    return loss.item(), gradient_report(model)

def main():
    model = nn.Linear(..., ...)
    print(E16(model, torch..., torch...))

if __name__ == "__main__":
    main()
```

10.3.18 Exercise 17 - Manual Backward Call Test

Part 1 - Why this matters

Calling `backward()` begins reverse-mode automatic differentiation from an output. For an ordinary training loss, that output is expected to be scalar, providing the starting value needed to propagate gradients through the graph. Observing one manual backward call links the loss equation to the stored parameter gradients.

Repeated backward calls also reveal that gradients accumulate by default and that a completed graph is normally released. This exercise uses one controlled call first, then tests accumulation only after the baseline behavior is understood.

Part 2 - Mathematical form

For scalar loss $\mathcal{L}$:

$$\mathcal{L}.\texttt{backward()} \quad \Longrightarrow \quad \theta.\texttt{grad} \leftarrow \theta.\texttt{grad} + \nabla_\theta \mathcal{L}$$

If the output is non-scalar, an explicit upstream gradient of compatible shape is required. The exercise uses scalar reduction for the main case.

Part 3 - Hints and Helper

Print `loss.grad_fn` before calling backward and confirm parameter gradients are initially `None`. Save the first gradients, clear them, and recompute the forward pass before a second independent call. Do not reuse a freed graph accidentally.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def E17(model, X, target):
    model.zero_grad(set_to_none=True)
    prediction = model(X)
    loss = ...
    before = {
        name: parameter.grad
        for name, parameter in model.named_parameters()
    }
    graph_function = loss.grad_fn
    loss.backward()
    after = {
        name: parameter.grad.detach().clone()
        for name, parameter in model.named_parameters()
    }
    return loss.item(), graph_function, before, after

def main():
    model = nn.Linear(..., ...)
    print(E17(model, torch..., torch...))

if __name__ == "__main__":
    main()
```

10.3.19 Exercise 18 - Parameter Update Order Inspection

Part 1 - Why this matters

The operations in a training step must occur in a meaningful order. Gradients belong to the parameters and forward values that created the loss; updating parameters before backward or forgetting to clear old gradients changes that relationship. A correct sequence makes every

state transition traceable.

Inspection is more informative than memorization. By recording parameters and gradients at each stage, the reader can explain why zeroing occurs before the next backward pass, why the optimizer step follows gradient calculation, and why a new prediction is required to measure the updated model.

Part 2 - Technical concept and behavior

A standard step follows:

$$\text{clear gradients} \rightarrow \text{forward} \rightarrow \text{loss} \rightarrow \text{backward} \rightarrow \text{update}$$

The prediction used to create $\mathcal{L}_t$ belongs to parameters θ_t. After updating to θ_{t+1}, a fresh forward pass is required to calculate $\mathcal{L}_{t+1}$.

Part 3 - Hints and Helper

Record cloned parameters and gradient states before zeroing, after backward, and after the optimizer step. Use one small optimizer only to study order. Do not interpret the old loss tensor as the post-update loss; recompute it explicitly.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def snapshot(model):
    return {
        name: {
            "value": parameter.detach().clone(),
            "grad": ...,
        }
        for name, parameter in model.named_parameters()
    }

def E18(model, optimizer, X, target):
    stages = {}
    stages["start"] = snapshot(model)
    optimizer.zero_grad(set_to_none=True)
    stages["after_zero"] = snapshot(model)
    prediction = model(X)
    loss_before = ...
    loss_before.backward()
    stages["after_backward"] = snapshot(model)
    optimizer.step()
    stages["after_step"] = snapshot(model)
    with torch.no_grad():
        loss_after = ...
    return stages, loss_before.item(), loss_after.item()
```

```
def main():
    model = nn.Linear(..., ...)
    optimizer = torch.optim.SGD(model.parameters(), lr=...)
    print(E18(model, optimizer, torch..., torch...))

if __name__ == "__main__":
    main()
```

10.3.20 Exercise 19 - Model Structure Failure Case

Part 1 - Why this matters

Structural errors can leave a model partially functional: it may execute a forward pass while hiding unregistered tensors, omitting expected parameters, or returning an incorrect output shape. These failures are dangerous because they can survive until training or checkpoint loading.

The final challenge turns the chapter into a validation checklist. The reader must inspect inheritance, registration, parameter counts, shapes, devices, gradient flags, state keys, and forward behavior. The report should identify evidence and likely cause rather than return only a generic pass or fail.

Part 2 - Technical concept and behavior

For an expected affine model, structural invariants include:

$$\text{shape}(W) = (O, F), \qquad \text{shape}(b) = (O), \qquad \text{shape}(f(X)) = (*, O)$$

The expected parameters must be registered, compatible with the input device and dtype, and connected to a scalar loss when trainable.

Part 3 - Hints and Helper

Create one deliberately broken custom module, such as one storing weight as an ordinary tensor or transposing it incorrectly. Write independent checks before attempting the forward pass. Capture exceptions at the outer boundary and preserve their types and messages.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def validate_model_structure(model, X, expected_in, expected_out):
    report = {
        "is_module": isinstance(model, nn.Module),
        "registered_names": ...,
        "parameter_count": ...,
        "device_compatible": ...,
        "dtype_compatible": ...,
    }
    try:
```

```
        output = model(X)
        report["output_shape_valid"] = ...
        loss = output.square().mean()
        loss.backward()
        report["gradient_flow"] = ...
    except (RuntimeError, ValueError, TypeError) as error:
        report["failure"] = {
            "type": type(error).__name__,
            "message": str(error),
        }
    return report

def E19(model, X):
    return validate_model_structure(model, X, ..., ...)

def main():
    broken_model = ...
    X = torch...
    print(E19(broken_model, X))

if __name__ == "__main__":
    main()
```

10.4 LR02 - Data Preparation and Feature Inputs

Data Engineering for AI

10.4.1 Exercise 00 - Generate Synthetic Linear Dataset

Part 1 - Why this matters

A synthetic dataset gives the reader complete knowledge of the relationship that generated each target. Because the true weights and bias are known, later training results can be compared with a controlled reference instead of relying only on a decreasing loss. This makes synthetic data ideal for testing a first regression pipeline.

Controlled generation also allows one factor to change at a time. The reader can vary sample count, feature count, parameter scale, or random seed while preserving every other condition. Unexpected behavior can then be traced to the experiment rather than hidden properties of an external dataset.

Part 2 - Mathematical form

Let $\mathbf{X} \in \mathbb{R}^{N \times F}$ be sampled feature data, $\mathbf{w}_{\text{true}} \in \mathbb{R}^{F \times 1}$ the true weights, and $b_{\text{true}} \in \mathbb{R}$. Generate targets by:

$$\mathbf{y} = \mathbf{X}\mathbf{w}_{\text{true}} + b_{\text{true}}$$

The expected target shape is $(N, 1)$. This first dataset is noiseless so every target can be reproduced exactly from its source parameters.

Part 3 - Hints and Helper

Use a local `torch.Generator` with an explicit seed so dataset creation does not unexpectedly alter unrelated global randomness. Validate positive sample and feature counts. Return the true parameters with the features and targets so later exercises can verify provenance.

Part 4 - Python code skeleton

```
import torch

def E00(sample_count, feature_count, seed):
    generator = torch.Generator().manual_seed(...)
    X = torch.randn(..., generator=generator)
    true_weight = torch.randn(..., generator=generator)
    true_bias = torch.randn(..., generator=generator)
    y = ...
    return X, y, true_weight, true_bias

def main():
    X, y, weight, bias = E00(..., ..., ...)
    print(X.shape, y.shape)
    print(weight, bias)

if __name__ == "__main__":
    main()
```

10.4.2 Exercise 01 - Add Noise to Dataset

Part 1 - Why this matters

Real measurements rarely follow a perfect equation. Sensors, unobserved variables, rounding, and natural variation introduce noise into targets. Adding controlled noise makes a synthetic regression problem more realistic while preserving knowledge of the underlying relationship.

Noise should be separated from the signal and recorded as part of the experiment. If the random process is not reproducible, changes in model performance may be caused by a different dataset rather than a different model or training configuration.

Part 2 - Mathematical form

For noiseless target $\mathbf{y}_{\text{clean}}$, define:

$$\mathbf{y}_{\text{noisy}} = \mathbf{y}_{\text{clean}} + \boldsymbol{\epsilon}, \qquad \epsilon_i \sim \mathcal{N}(0, \sigma^2)$$

The standard deviation σ controls noise magnitude. The features and clean targets remain unchanged, allowing signal and noise to be compared directly.

Part 3 - Hints and Helper

Require a nonnegative noise standard deviation and use a seeded generator. Return the sampled noise as well as the noisy target. Verify that zero noise reproduces the clean target

and that the empirical noise mean and standard deviation are reasonable for a sufficiently large sample.

Part 4 - Python code skeleton

```
import torch

def E01(clean_target, noise_std, seed):
    if noise_std < 0:
        raise ValueError(...)
    generator = torch.Generator().manual_seed(...)
    noise = torch.randn(
        clean_target.shape,
        generator=generator,
        dtype=clean_target.dtype,
        device=clean_target.device,
    ) * ...
    noisy_target = ...
    return noisy_target, noise

def main():
    clean_target = torch...
    noisy_target, noise = E01(clean_target, ..., ...)
    print(noisy_target)
    print(noise.mean(), noise.std())

if __name__ == "__main__":
    main()
```

10.4.3 Exercise 02 - Feature Scaling Experiment

Part 1 - Why this matters

Features measured in different units can have dramatically different numerical ranges. A large-scale feature may dominate gradient magnitudes even when it is not more informative. Scaling can make optimization better conditioned and makes the effect of learning-rate choices easier to interpret.

Scaling statistics must be learned from training data only. Using test values to calculate means or standard deviations leaks information from evaluation into preparation. This exercise therefore separates fitting a scaler from applying it to another tensor.

Part 2 - Mathematical form

For training feature j:

$$\mu_j = \frac{1}{N}\sum_{i=1}^{N} X_{ij}, \qquad s_j = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(X_{ij} - \mu_j)^2}$$

Standardized values are:

$$Z_{ij} = \frac{X_{ij} - \mu_j}{s_j}$$

A zero or nearly zero scale requires an explicit policy to avoid division by zero.

Part 3 - Hints and Helper

Create separate `fit_scaler()` and `transform()` functions. Preserve the feature dimension with `keepdim=True`. Detect constant features and replace an unsafe denominator only according to a documented rule. Compare model behavior before and after scaling without changing the split.

Part 4 - Python code skeleton

```
import torch

def fit_scaler(X_train):
    mean = X_train.mean(dim=..., keepdim=True)
    scale = X_train.std(dim=..., keepdim=True, correction=...)
    safe_scale = torch.where(..., ..., ...)
    return mean, safe_scale

def transform(X, mean, scale):
    return ...

def E02(X_train, X_test):
    mean, scale = fit_scaler(X_train)
    train_scaled = transform(...)
    test_scaled = transform(...)
    return train_scaled, test_scaled, mean, scale

def main():
    X_train = torch...
    X_test = torch...
    print(E02(X_train, X_test))

if __name__ == "__main__":
    main()
```

10.4.4 Exercise 03 - Train/Test Split Implementation

Part 1 - Why this matters

A model must be evaluated on samples that were not used to adjust its parameters. A train/test split creates this boundary and provides evidence about generalization. Evaluating on training data alone can make memorization or leakage look like successful learning.

The split must preserve alignment between every feature row and its target. Reproducible random indices allow different model experiments to use the same partitions, while time-dependent data may require chronological splitting instead of random shuffling.

Part 2 - Technical concept and behavior

For N samples and test fraction $r \in (0, 1)$, define a test count such as:

$$N_{\text{test}} = \lfloor rN \rfloor, \qquad N_{\text{train}} = N - N_{\text{test}}$$

A permutation of sample indices is divided into disjoint train and test sets whose union contains every original index exactly once.

Part 3 - Hints and Helper

Validate equal sample counts in $\mathbf{X}$ and $\mathbf{y}$. Generate one permutation with a seeded `torch.Generator`, then index both tensors with the same subsets. Return the indices so disjointness, completeness, and reproducibility can be checked.

Part 4 - Python code skeleton

```
import torch

def E03(X, y, test_fraction, seed):
    if X.shape[0] != y.shape[0]:
        raise ValueError(...)
    if not 0 < test_fraction < 1:
        raise ValueError(...)
    generator = torch.Generator().manual_seed(...)
    indices = torch.randperm(..., generator=generator)
    test_count = ...
    test_indices = ...
    train_indices = ...
    return (
        X[train_indices],
        y[train_indices],
        X[test_indices],
        y[test_indices],
        train_indices,
        test_indices,
    )

def main():
    X = torch...
    y = torch...
    print(E03(X, y, ..., ...))

if __name__ == "__main__":
    main()
```

10.4.5 Exercise 04 - Batch Dataset Creation

Part 1 - Why this matters

Mini-batches divide a dataset into manageable groups that can be processed by a model. They reduce peak memory requirements and allow parameter updates to occur more frequently than full-dataset training. Before introducing a loader, the reader should understand what a valid batch contains.

Features and targets in a batch must refer to the same sample indices. A batching method that slices them differently can produce valid shapes and meaningless supervision. The final incomplete batch also requires a deliberate keep-or-drop policy.

Part 2 - Technical concept and behavior

For N samples and batch size B, the number of batches when the remainder is retained is:

$$K = \left\lceil \frac{N}{B} \right\rceil$$

Batch k uses the aligned slice $[kB : \min((k+1)B, N)]$ from both features and targets. Every sample should occur exactly once when shuffling is disabled.

Part 3 - Hints and Helper

Validate a positive batch size and equal sample counts. Use slicing to construct a list of feature-target pairs. Record original sample identifiers during testing so coverage and order can be verified, especially for the final batch.

Part 4 - Python code skeleton

```
import torch

def E04(X, y, batch_size, drop_last=False):
    if batch_size <= 0 or X.shape[0] != y.shape[0]:
        raise ValueError(...)
    batches = []
    for start in range(0, X.shape[0], batch_size):
        end = min(..., ...)
        if drop_last and end - start < batch_size:
            ...
        batches.append((X[start:end], y[start:end]))
    return batches

def main():
    X = torch...
    y = torch...
    for X_batch, y_batch in E04(X, y, ...):
        print(X_batch.shape, y_batch.shape)

if __name__ == "__main__":
    main()
```

10.4.6 Exercise 05 - Shuffle Dataset per Epoch

Part 1 - Why this matters

When training repeatedly over a dataset, using the same order can preserve unwanted sequence patterns and create highly correlated neighboring batches. Reshuffling changes batch composition between epochs while keeping the dataset itself unchanged.

A correct shuffle must apply one shared permutation to features, targets, and sample identifiers. It should change order without dropping, duplicating, or relabeling samples. Reproducibility also requires control over how random state advances across epochs.

Part 2 - Technical concept and behavior

For epoch e, generate a permutation π_e of $\{0, \ldots, N-1\}$:

$$\mathbf{X}^{(e)} = \mathbf{X}[\pi_e], \qquad \mathbf{y}^{(e)} = \mathbf{y}[\pi_e]$$

Each π_e must be bijective. Different epochs may use different permutations while a repeated experiment with the same seed reproduces the full sequence.

Part 3 - Hints and Helper

Use one `torch.Generator` initialized before the epoch loop rather than resetting the same seed inside every epoch. Preserve sample IDs for validation. Compare sorted IDs before and after each shuffle and confirm that at least two epoch orders differ.

Part 4 - Python code skeleton

```
import torch

def E05(X, y, epochs, seed):
    generator = torch.Generator().manual_seed(...)
    epoch_orders = []
    for epoch in range(epochs):
        permutation = torch.randperm(..., generator=generator)
        X_epoch = ...
        y_epoch = ...
        epoch_orders.append((permutation, X_epoch, y_epoch))
    return epoch_orders

def main():
    X = torch...
    y = torch...
    for order, X_epoch, y_epoch in E05(X, y, ..., ...):
        print(order)

if __name__ == "__main__":
    main()
```

10.4.7 Exercise 06 - Move Dataset to CUDA

Part 1 - Why this matters

A CUDA model requires its input batches and targets on the same GPU. Moving an entire dataset at once can be convenient for small data, but it may exhaust device memory and prevent other model state from fitting. Larger pipelines normally keep data on the CPU and transfer one batch at a time.

This exercise compares those strategies rather than declaring one universally correct. The reader will check CUDA availability, estimate tensor storage, verify device placement, and preserve a CPU fallback. Device transfer is treated as a resource decision.

Part 2 - Technical concept and behavior

Approximate dataset storage is:

$$M = \text{numel}(\mathbf{X})s_X + \text{numel}(\mathbf{y})s_y$$

where s_X and s_y are bytes per element. After transfer to device d, both tensors must report d before they participate in the same loss computation.

Part 3 - Hints and Helper

Use `element_size()` to estimate bytes and `torch.cuda.is_available()` before selecting CUDA. Support `move_all=False` to leave the full dataset on CPU and transfer a chosen batch later. Do not return CUDA tensors from multi-process dataset workers.

Part 4 - Python code skeleton

```
import torch

def tensor_bytes(tensor):
    return tensor.numel() * tensor.element_size()

def E06(X, y, move_all):
    device = torch.device("cuda" if ... else "cpu")
    estimated_bytes = tensor_bytes(X) + tensor_bytes(y)
    if move_all:
        X = X.to(...)
        y = y.to(...)
    return X, y, device, estimated_bytes

def main():
    X = torch...
    y = torch...
    print(E06(X, y, move_all=...))

if __name__ == "__main__":
    main()
```

10.4.8 Exercise 07 - DataLoader Integration

Part 1 - Why this matters

A `Dataset` defines how samples and targets are stored or retrieved, while a `DataLoader` supplies iteration, automatic batching, optional shuffling, and worker configuration. Separating these roles keeps data code independent from the training loop.

Integration must be verified before training. The reader should inspect the number of batches, feature and target shapes, final-batch behavior, and sample alignment. Loader settings change data delivery, not the mathematical meaning of the examples.

Part 2 - Technical concept and behavior

For a map-style dataset of N samples, batch size B, and `drop_last=False`:

$$K = \left\lceil \frac{N}{B} \right\rceil$$

With `drop_last=True`, $K = \lfloor N/B \rfloor$. The loader yields paired tensors formed from the same indexed samples.

Part 3 - Hints and Helper

Wrap features and targets with `TensorDataset`, then construct `DataLoader`. Begin with `num_workers=0` for transparent debugging. Make `shuffle`, `drop_last`, and `pin_memory` explicit arguments and inspect every batch once.

Part 4 - Python code skeleton

```
import torch
from torch.utils.data import DataLoader, TensorDataset

def E07(X, y, batch_size, shuffle, drop_last):
    dataset = TensorDataset(..., ...)
    loader = DataLoader(
        dataset,
        batch_size=...,
        shuffle=...,
        drop_last=...,
        num_workers=...,
        pin_memory=...,
    )
    summary = []
    for X_batch, y_batch in loader:
        summary.append(...)
    return loader, summary

def main():
    X = torch...
    y = torch...
    _, summary = E07(X, y, ..., ..., ...)
```

```
    print(summary)

if __name__ == "__main__":
    main()
```

10.4.9 Exercise 08 - Incorrect Shape Debugging

Part 1 - Why this matters

Data pipelines often produce a one-dimensional target when the model emits a two-dimensional column. PyTorch broadcasting may then compare shapes $(N, 1)$ and (N) as if they were (N, N), creating a loss tensor that is valid to compute but mathematically wrong. Shape errors therefore require validation before the loss.

Debugging should attach meaning to every axis: samples, features, and outputs. Reshaping until an exception disappears is unsafe because it may mix sample values or conceal a wrong feature layout.

Part 2 - Technical concept and behavior

For scalar regression with a batch:

$$\mathbf{X} \in \mathbb{R}^{N \times F}, \qquad \mathbf{y} \in \mathbb{R}^{N \times 1}, \qquad \hat{\mathbf{y}} \in \mathbb{R}^{N \times 1}$$

The sample counts must match, and prediction and target should have identical shapes before element-wise loss evaluation.

Part 3 - Hints and Helper

Build a contract validator that checks dimensions and reports observed shapes. Test targets shaped (N), $(N, 1)$, and $(1, N)$. Apply `unsqueeze()` only when the semantic correction is known, then verify the result exactly.

Part 4 - Python code skeleton

```
import torch

def validate_regression_shapes(X, y, prediction=None):
    errors = []
    if X.ndim != ...:
        errors.append(...)
    if y.ndim != ... or y.shape[-1] != ...:
        errors.append(...)
    if X.shape[0] != y.shape[0]:
        errors.append(...)
    if prediction is not None and prediction.shape != y.shape:
        errors.append(...)
    return errors

def E08(X, y):
    errors_before = validate_regression_shapes(X, y)
```

```
    corrected_y = ...
    errors_after = validate_regression_shapes(X, corrected_y)
    return corrected_y, errors_before, errors_after

def main():
    print(E08(torch..., torch...))

if __name__ == "__main__":
    main()
```

10.4.10 Exercise 09 - Missing Feature Handling

Part 1 - Why this matters

Missing feature values cannot be passed blindly into a linear model because they can create non-finite predictions and losses. Handling them requires a policy based on why data is missing, not only a numerical replacement. Dropping samples, imputing values, and adding missingness indicators carry different assumptions.

Imputation statistics must be fitted on training data only. Filling test values with a mean calculated from the complete dataset leaks information across the evaluation boundary. The fitted values and missing masks should be preserved for auditing.

Part 2 - Technical concept and behavior

For feature j, training-mean imputation may use:

$$\mu_j = \frac{\sum_i X_{ij} \mathbb{1}[X_{ij} \text{ observed}]}{\sum_i \mathbb{1}[X_{ij} \text{ observed}]}$$

Every missing entry in that feature is replaced by μ_j. A feature with no observed training values requires an explicit failure or fallback policy.

Part 3 - Hints and Helper

Detect missing values with `torch.isnan()`. Write separate fit and transform functions, store a boolean mask, and check for all-missing columns. Compare the number of missing entries before and after transformation without altering genuinely observed values.

Part 4 - Python code skeleton

```
import torch

def fit_imputer(X_train):
    missing = torch.isnan(X_train)
    observed_count = (~missing).sum(dim=...)
    if torch.any(observed_count == 0):
        raise ValueError(...)
    fill_values = torch.nanmean(X_train, dim=...)
    return fill_values
```

```
def transform_missing(X, fill_values):
    mask = torch.isnan(X)
    filled = torch.where(mask, ..., X)
    return filled, mask

def E09(X_train, X_test):
    fill_values = fit_imputer(X_train)
    train_filled, train_mask = transform_missing(...)
    test_filled, test_mask = transform_missing(...)
    return ...

def main():
    print(E09(torch..., torch...))

if __name__ == "__main__":
    main()
```

10.4.11 Exercise 10 - Bias Column Injection

Part 1 - Why this matters

An affine model can be rewritten as one matrix multiplication by adding a column of ones to the feature matrix. This connects the explicit bias parameter to ordinary linear algebra and helps explain closed-form formulations of regression.

The transformation must be used deliberately. If a ones column is added while `nn.Linear` also keeps its own bias, the intercept is represented twice. Feature names and shapes must be updated so the injected column is not mistaken for measured data.

Part 2 - Mathematical form

Define the augmented tensors:

$$\tilde{\mathbf{X}} = \begin{bmatrix} \mathbf{X} & \mathbf{1} \end{bmatrix}, \qquad \tilde{\mathbf{w}} = \begin{bmatrix} \mathbf{w} \\ b \end{bmatrix}$$

Then:

$$\tilde{\mathbf{X}}\tilde{\mathbf{w}} = \mathbf{X}\mathbf{w} + b$$

The augmented feature count is $F + 1$.

Part 3 - Hints and Helper

Create the ones column with the same dtype and device as $\mathbf{X}$. Concatenate along the feature dimension and compare augmented prediction with the explicit-bias equation. If later using `nn.Linear`, set `bias=False` for the augmented representation.

Part 4 - Python code skeleton

```
import torch

def E10(X, weight, bias):
    ones = torch.ones(
        (..., ...),
        dtype=X.dtype,
        device=X.device,
    )
    X_augmented = torch.cat((..., ...), dim=...)
    weight_augmented = torch.cat((..., ...), dim=...)
    direct = ...
    augmented = ...
    return X_augmented, weight_augmented, direct, augmented

def main():
    print(E10(torch..., torch..., torch...))

if __name__ == "__main__":
    main()
```

10.4.12 Exercise 11 - High-Dimensional Feature Test

Part 1 - Why this matters

Datasets may contain far more features than can be visualized, and sometimes more features than samples. A linear forward pass still follows the same matrix law, but memory use, rank, optimization behavior, and overfitting risk change substantially.

A high-dimensional test should verify resource requirements before allocation. Generating an unnecessarily enormous tensor can exhaust memory without teaching anything. The experiment scales dimensions gradually and records shape, storage, rank limits, and forward behavior.

Part 2 - Technical concept and behavior

For $\mathbf{X} \in \mathbb{R}^{N \times F}$:

$$\operatorname{rank}(\mathbf{X}) \leq \min(N, F)$$

Tensor storage grows approximately as NFs, where s is bytes per element. When $F > N$, many parameter vectors may explain training data similarly unless additional constraints or information are introduced.

Part 3 - Hints and Helper

Estimate bytes before creating the tensor and enforce a caller-provided memory budget. Test several feature counts with fixed sample count. Record the model parameter count, output shape, and an estimated or computed matrix rank for manageable cases.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def estimated_bytes(shape, dtype):
    element_size = torch.empty((), dtype=dtype).element_size()
    return ... * element_size

def E11(sample_count, feature_counts, memory_budget_bytes):
    results = []
    for feature_count in feature_counts:
        required = estimated_bytes((sample_count, feature_count), ...)
        if required > memory_budget_bytes:
            results.append(...)
            continue
        X = torch.randn(...)
        model = nn.Linear(..., ...)
        output = model(X)
        results.append(...)
    return results

def main():
    print(E11(..., [..., ..., ...], ...))

if __name__ == "__main__":
    main()
```

10.4.13 Exercise 12 - Collinearity Effect Observation

Part 1 - Why this matters

Collinearity occurs when one feature can be closely approximated by other features. A model may still predict accurately, but individual weights can become unstable because several parameter combinations explain nearly the same input variation.

This matters for interpretation. A large coefficient is not automatically evidence that one collinear feature is uniquely important. Small perturbations to data may shift weight between correlated columns while leaving predictions almost unchanged.

Part 2 - Mathematical form

For two nearly dependent features:

$$\mathbf{x}_2 = c\mathbf{x}_1 + \boldsymbol{\delta}$$

with small $\boldsymbol{\delta}$, the design matrix is close to rank deficient. Its singular values may have a large ratio:

$$\kappa(\mathbf{X}) = \frac{\sigma_{\max}}{\sigma_{\min}}$$

A large condition number indicates sensitivity in parameter estimation.

Part 3 - Hints and Helper

Generate one feature from another plus small noise. Compare singular values and condition estimates with an independent-feature dataset. Fit or solve repeated perturbed datasets, then compare changes in weights and predictions separately.

Part 4 - Python code skeleton

```
import torch

def condition_summary(X):
    singular_values = torch.linalg.svdvals(X)
    condition = ...
    return singular_values, condition

def E12(sample_count, perturbation_std, seed):
    generator = torch.Generator().manual_seed(...)
    x1 = torch.randn(..., generator=generator)
    x2 = ... * x1 + torch.randn(..., generator=generator) * ...
    X_collinear = torch.cat((x1, x2), dim=...)
    X_independent = torch.randn(..., generator=generator)
    return {
        "collinear": condition_summary(X_collinear),
        "independent": condition_summary(X_independent),
        "matrices": (..., ...),
    }

def main():
    print(E12(..., ..., ...))

if __name__ == "__main__":
    main()
```

10.4.14 Exercise 13 - Data Precision Impact Test

Part 1 - Why this matters

The dtype chosen for a dataset affects memory, transfer cost, numerical resolution, and compatibility with model parameters. Single precision is common for training, while double precision can expose smaller numerical differences at additional cost.

A fair precision test uses the same underlying values. Creating two random datasets independently would compare sampling variation instead of dtype. The reader should convert one high-precision reference and measure both storage and numerical changes.

Part 2 - Technical concept and behavior

For reference data $\mathbf{X}_{64}$ and its single-precision conversion:

$$\mathbf{X}_{32} = \mathrm{cast}_{32}(\mathbf{X}_{64})$$

Measure reconstruction difference:

$$D_{\max} = \max_{i,j} |X_{64,ij} - \mathrm{cast}_{64}(X_{32,ij})|$$

The `float64` tensor uses twice the element storage of `float32`.

Part 3 - Hints and Helper

Create data in `float64`, convert it to `float32`, then convert back only for difference calculation. Report bytes, maximum absolute difference, and forward-output difference using model states derived from one common reference.

Part 4 - Python code skeleton

```
import copy
import torch
from torch import nn

def E13(X64, model):
    X32 = X64.to(dtype=torch.float32)
    reconstructed = X32.to(dtype=torch.float64)
    data_difference = ...
    model64 = copy.deepcopy(model).double()
    model32 = copy.deepcopy(model).float()
    with torch.no_grad():
        output64 = ...
        output32 = ...
    return {
        "bytes64": ...,
        "bytes32": ...,
        "data_difference": ...,
        "output_difference": ...,
    }

def main():
    X64 = torch.randn(..., dtype=torch.float64)
    model = nn.Linear(..., ...)
    print(E13(X64, model))

if __name__ == "__main__":
    main()
```

10.4.15 Exercise 14 - Large Dataset GPU Transfer Cost

Part 1 - Why this matters

Moving data from host memory to a GPU consumes time and memory bandwidth. If batches are too small, transfer overhead may dominate; if they are too large, device memory may be exhausted. Transfer strategy must therefore be measured on the actual system.

CUDA operations can be asynchronous, so a timer may stop before the copy finishes. A valid benchmark synchronizes at measurement boundaries and distinguishes dataset preparation, optional pinning, transfer, and model computation.

Part 2 - Technical concept and behavior

For M transferred bytes and elapsed copy time T:

$$\text{effective bandwidth} = \frac{M}{T}$$

The experiment compares several batch sizes and transfer options. It should report latency and throughput distributions rather than one unverified timing value.

Part 3 - Hints and Helper

Skip the CUDA benchmark clearly when CUDA is unavailable. Warm up transfers, use `torch.cuda.synchronize()` around timed regions, and reuse allocated source data. When testing `non_blocking=True`, use a justified pinned-memory source and synchronize before reading the result.

Part 4 - Python code skeleton

```
import time
import torch

def measure_transfer(X_cpu, device, non_blocking, repetitions):
    ...  # warm-up
    torch.cuda.synchronize()
    started = time.perf_counter()
    for _ in range(repetitions):
        X_gpu = X_cpu.to(device, non_blocking=non_blocking)
    torch.cuda.synchronize()
    elapsed = time.perf_counter() - started
    bytes_moved = ... * repetitions
    return ..., ...

def E14(shapes, repetitions):
    if not torch.cuda.is_available():
        return {"skipped": ...}
    device = torch.device("cuda")
    results = []
    for shape in shapes:
        X = torch.randn(...)
        ...  # compare supported transfer strategies
        results.append(...)
    return results

def main():
    print(E14([...], ...))

if __name__ == "__main__":
    main()
```

10.4.16 Exercise 15 - Dataset Caching Strategy

Part 1 - Why this matters

Repeated deterministic preprocessing can waste time when the same sample is requested across many epochs. Caching stores a prepared result so later access avoids duplicate work. The benefit depends on preparation cost, access frequency, and available memory.

A cache can also return stale or unsafe data. If transformation settings change, the cache key must change or stored entries must be invalidated. Mutable cached tensors should not be modified by callers in a way that corrupts future samples.

Part 2 - Technical concept and behavior

For uncached preparation cost C_p, cache lookup cost C_l, and hit rate h, a simplified expected access cost is:

$$E[C] = (1 - h)C_p + hC_l$$

Memory use grows with cached sample size and count. A bounded policy should track hits, misses, capacity, and a transformation-version identifier.

Part 3 - Hints and Helper

Create a small custom `Dataset` with an in-memory dictionary. Include sample index and transform version in each key. Return a clone when mutation could be a risk, and clear or replace the cache when preprocessing configuration changes.

Part 4 - Python code skeleton

```
import torch
from torch.utils.data import Dataset

class CachedRegressionDataset(Dataset):
    def __init__(self, X, y, transform, version, capacity):
        self.X = X
        self.y = y
        self.transform = transform
        self.version = version
        self.capacity = capacity
        self.cache = {}
        self.hits = 0
        self.misses = 0

    def __len__(self):
        return ...

    def __getitem__(self, index):
        key = (..., ...)
        if key in self.cache:
            self.hits += 1
            return ...
```

```
        self.misses += 1
        item = ...
        ...  # enforce the capacity policy
        return item

def E15(X, y):
    dataset = CachedRegressionDataset(X, y, ..., ..., ...)
    ...  # request repeated indices
    return dataset.hits, dataset.misses, len(dataset.cache)

def main():
    print(E15(torch..., torch...))

if __name__ == "__main__":
    main()
```

10.4.17 Exercise 16 - Dataset Integrity Check

Part 1 - Why this matters

A dataset can have valid tensor shapes while containing corrupted values, misaligned targets, duplicate sample identifiers, or an unexpected dtype. Training on such data produces misleading results and may make a model problem appear to be an optimization problem.

Integrity checks should run at clearly defined pipeline boundaries and produce actionable evidence. They are not a guarantee that data is scientifically correct, but they can detect violations of the structural and numerical contract before training.

Part 2 - Technical concept and behavior

For regression tensors, basic invariants include:

$$N_X = N_y = N_{\mathrm{id}}, \qquad F > 0, \qquad \mathbf{X}, \mathbf{y} \text{ finite}$$

Sample identifiers should satisfy the chosen uniqueness policy, and tensor dtypes should be compatible with the intended model. Target alignment requires stable provenance beyond shape alone.

Part 3 - Hints and Helper

Return a list of named failures instead of stopping at the first one. Check sample counts, dimensions, floating dtype, finite values, duplicate IDs, and target shape. Include summary statistics but do not silently replace invalid values during validation.

Part 4 - Python code skeleton

```
import torch

def E16(X, y, sample_ids):
    failures = []
    if X.ndim != ...:
```

```
        failures.append(...)
    if y.ndim != ...:
        failures.append(...)
    if not (X.shape[0] == y.shape[0] == len(sample_ids)):
        failures.append(...)
    if not torch.isfinite(X).all():
        failures.append(...)
    if not torch.isfinite(y).all():
        failures.append(...)
    if len(set(sample_ids)) != len(sample_ids):
        failures.append(...)
    return {
        "valid": len(failures) == 0,
        "failures": failures,
        "feature_summary": ...,
        "target_summary": ...,
    }

def main():
    print(E16(torch..., torch..., ...))

if __name__ == "__main__":
    main()
```

10.4.18 Exercise 17 - Manual Batch Iteration

Part 1 - Why this matters

Manual batching exposes the indexing behavior that a loader usually handles. The reader can observe batch boundaries, incomplete final batches, aligned slicing, and total sample coverage without additional abstractions.

This implementation is useful as a correctness reference, not necessarily as the final production pipeline. Comparing it with a `DataLoader` helps separate the essential batching law from multiprocessing, sampling, pinning, and prefetch options.

Part 2 - Technical concept and behavior

For a permutation π and batch index k:

$$I_k = \pi[kB : \min((k+1)B, N)]$$

The batch is $(\mathbf{X}[I_k], \mathbf{y}[I_k])$. Across a full retained epoch, the concatenated indices must contain each sample once.

Part 3 - Hints and Helper

Generate indices first and index both tensors with the same batch indices. Make shuffle and `drop_last` explicit. Yield the indices with each batch so coverage, duplicates, and alignment can be tested after iteration.

Part 4 - Python code skeleton

```
import torch

def manual_batches(X, y, batch_size, shuffle, drop_last, generator):
    indices = (
        torch.randperm(X.shape[0], generator=generator)
        if shuffle else torch.arange(X.shape[0])
    )
    for start in range(0, X.shape[0], batch_size):
        batch_indices = indices[start:start + batch_size]
        if drop_last and batch_indices.numel() < batch_size:
            ...
        yield X[batch_indices], y[batch_indices], batch_indices

def E17(X, y, batch_size):
    generator = torch.Generator().manual_seed(...)
    observed = []
    for X_batch, y_batch, indices in manual_batches(...):
        observed.append(...)
    return observed

def main():
    print(E17(torch..., torch..., ...))

if __name__ == "__main__":
    main()
```

10.4.19 Exercise 18 - Data Leakage Detection

Part 1 - Why this matters

Data leakage allows information unavailable at prediction time to influence training or preprocessing. It can produce excellent test metrics that disappear in real use. Common sources include duplicate samples across splits, scaling fitted on all data, target-derived features, and future observations placed in a training set.

Leakage detection requires provenance and pipeline reasoning, not a single tensor operation. This exercise implements several observable checks while acknowledging that domain knowledge is still required to identify whether a feature would truly be available at inference time.

Part 2 - Technical concept and behavior

For train and test sample identifiers:

$$I_{\text{train}} \cap I_{\text{test}} = \varnothing$$

Preprocessing parameters must satisfy:

$$\phi = \text{fit}(\mathbf{X}_{\text{train}})$$

and test data may only be transformed by ϕ. For time-ordered forecasting, training timestamps should precede the evaluation window according to the chosen protocol.

Part 3 - Hints and Helper

Check identifier overlap, duplicate feature rows, preprocessing-fit provenance, and forbidden feature names. For temporal data, compare boundary timestamps. Return warnings with evidence rather than automatically deleting suspicious data.

Part 4 - Python code skeleton

```
import torch

def E18(train, test, preprocessing_metadata, forbidden_features):
    warnings = []
    train_ids = set(train["ids"])
    test_ids = set(test["ids"])
    overlap = train_ids.intersection(test_ids)
    if overlap:
        warnings.append(...)
    if preprocessing_metadata["fit_split"] != "train":
        warnings.append(...)
    leaked_names = set(train["feature_names"]).intersection(forbidden_features)
    if leaked_names:
        warnings.append(...)
    ...  # inspect duplicate rows and temporal boundaries
    return {
        "leakage_detected": len(warnings) > 0,
        "warnings": warnings,
    }

def main():
    train = ...
    test = ...
    metadata = ...
    print(E18(train, test, metadata, {...}))

if __name__ == "__main__":
    main()
```

10.4.20 Exercise 19 - Data Pipeline Failure Debugging

Part 1 - Why this matters

A data pipeline can fail during generation, splitting, transformation, batching, transfer, or model input. If all stages are executed inside one opaque function, the final exception provides little evidence about where the data contract was first violated.

The final exercise builds a staged diagnostic pipeline. Each stage records input and output metadata, validates its own invariants, and stops safely when a failure would contaminate later results. The report should identify the earliest failing stage and preserve its cause.

Part 2 - Technical concept and behavior

Represent the pipeline as composed transformations:

$$D_0 \xrightarrow{g_1} D_1 \xrightarrow{g_2} \cdots \xrightarrow{g_K} D_K$$

Every stage g_k has preconditions and postconditions for shape, dtype, device, finite values, alignment, and provenance. Failure at k means later output cannot be trusted even if another stage can still execute.

Part 3 - Hints and Helper

Give every stage a name and validator. Catch expected exceptions only around the individual stage, record the original type and message, and stop or mark downstream stages as skipped. Test wrong shapes, missing values, split overlap, and device mismatch as separate injected failures.

Part 4 - Python code skeleton

```
import torch

def summarize(data):
    return {
        "feature_shape": ...,
        "target_shape": ...,
        "feature_dtype": ...,
        "target_dtype": ...,
        "feature_device": ...,
        "finite": ...,
    }

def run_stage(name, function, data):
    before = summarize(data)
    try:
        output = function(data)
        after = summarize(output)
        return output, {"stage": name, "status": "passed",
                        "before": before, "after": after}
    except (RuntimeError, ValueError, TypeError) as error:
        return None, {
            "stage": name,
            "status": "failed",
            "error_type": type(error).__name__,
            "message": str(error),
            "before": before,
        }

def E19(initial_data, stages):
    reports = []
    current = initial_data
    for name, function in stages:
        current, report = run_stage(name, function, current)
```

```
        reports.append(report)
        if current is None:
            break
    return current, reports

def main():
    initial_data = ...
    stages = [...]
    print(E19(initial_data, stages))

if __name__ == "__main__":
    main()
```

10.5 LR03 - Prediction Function Implementation

Applied AI and Deployment

10.5.1 Exercise 00 - Define Predict(x) Function

Part 1 - Why this matters

A prediction function is the boundary between a trained model and the system that uses it. It accepts prepared features, applies the model, and returns outputs with a documented shape, dtype, and device policy. Defining this boundary early prevents inference logic from being scattered throughout an application.

Prediction is not the same as training. It should not update parameters or accumulate gradients, and it must preserve the preprocessing assumptions used during training. A useful function makes these expectations explicit instead of silently accepting any tensor.

Part 2 - Mathematical form

For a linear model with input $\mathbf{x} \in \mathbb{R}^F$:

$$\text{predict}(\mathbf{x}) = \hat{\mathbf{y}} = \mathbf{x}^\mathsf{T}\mathbf{w} + b$$

For a model object, the function delegates this transformation to its forward computation after validating the input contract. The initial exercise keeps the output as a tensor.

Part 3 - Hints and Helper

Accept the model and tensor as arguments rather than relying on global state. Put the model in evaluation mode and validate the final feature dimension. Use a gradient-disabled context and return metadata with the prediction so the behavior can be inspected.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def E00(model, x, expected_features):
```

```
    if x.shape[-1] != expected_features:
        raise ValueError(...)
    model.eval()
    with torch.no_grad():
        prediction = model(...)
    return {
        "prediction": prediction,
        "shape": ...,
        "dtype": ...,
        "device": ...,
    }

def main():
    model = nn.Linear(..., ...)
    x = torch...
    print(E00(model, x, ...))

if __name__ == "__main__":
    main()
```

10.5.2 Exercise 01 - Vectorized Prediction

Part 1 - Why this matters

Vectorized prediction applies one tensor operation to many samples instead of using a Python loop. This aligns the computation with optimized matrix kernels and usually improves throughput while keeping the mathematical structure clear.

Vectorization must preserve sample independence and order. The prediction in row i should equal the result obtained by evaluating only input row i. Comparing vectorized and row-wise references is a useful correctness test.

Part 2 - Mathematical form

For $\mathbf{X} \in \mathbb{R}^{N \times F}$:

$$\hat{\mathbf{Y}} = \mathbf{X}\mathbf{W}^{\mathsf{T}} + \mathbf{b}$$

The same weight and bias are applied to every row. For O outputs, the resulting shape is (N, O).

Part 3 - Hints and Helper

Require a two-dimensional input for this exercise. Perform one model call for the full batch, then build a row-wise reference only for validation. Use `torch.allclose()` and report the maximum absolute difference.

Part 4 - Python code skeleton

```
import torch
```

```
def E01(model, X):
    model.eval()
    with torch.no_grad():
        vectorized = model(...)
        rowwise = torch.cat(
            [model(row.unsqueeze(...)) for row in X],
            dim=...,
        )
    difference = ...
    return vectorized, rowwise, torch.allclose(..., ...), difference

def main():
    model = torch.nn.Linear(..., ...)
    X = torch...
    print(E01(model, X))

if __name__ == "__main__":
    main()
```

10.5.3 Exercise 02 - Batch Prediction on CPU

Part 1 - Why this matters

CPU inference is widely available and can be efficient for small models, low request volume, or environments where GPU transfer overhead would dominate. A controlled CPU implementation also provides a portable reference for later device comparisons.

Batch size changes both throughput and memory use. The prediction function should process all samples exactly once, preserve their order, and handle a final batch that is smaller than the requested size.

Part 2 - Technical concept and behavior

For N samples and batch size B, prediction is divided into:

$$K = \left\lceil \frac{N}{B} \right\rceil$$

batches. Concatenating the ordered batch outputs along the sample dimension should reproduce the full vectorized CPU prediction.

Part 3 - Hints and Helper

Move the model and input explicitly to `cpu`. Iterate by slices, use an inference context, and concatenate outputs. Validate positive batch size and compare with a one-call reference for a manageable dataset.

Part 4 - Python code skeleton

```
import torch

def E02(model, X, batch_size):
```

```
    if batch_size <= 0:
        raise ValueError(...)
    model = model.to("cpu").eval()
    X = X.to("cpu")
    outputs = []
    with torch.inference_mode():
        for start in range(0, X.shape[0], batch_size):
            batch = X[start:start + batch_size]
            outputs.append(model(...))
    predictions = torch.cat(outputs, dim=...)
    return predictions

def main():
    model = torch.nn.Linear(..., ...)
    X = torch...
    print(E02(model, X, ...))

if __name__ == "__main__":
    main()
```

10.5.4 Exercise 03 - Batch Prediction on CUDA

Part 1 - Why this matters

CUDA can accelerate large parallel prediction workloads, but the model and every input batch must be on the same GPU. A reliable function checks availability, controls transfer, and reports when it falls back instead of assuming that CUDA exists.

Keeping the full dataset on the GPU is not always necessary or safe. Batch-wise transfer limits peak device memory and better represents a deployment pipeline in which requests arrive from CPU memory.

Part 2 - Technical concept and behavior

For device $d =$ `cuda`, every batch follows:

$$\mathbf{X}_{k,d} \rightarrow f_d(\mathbf{X}_{k,d}) \rightarrow \hat{\mathbf{Y}}_{k,d}$$

Outputs may remain on the GPU or be copied to CPU according to the function contract. Concatenation requires all returned batch tensors to share a device.

Part 3 - Hints and Helper

Check `torch.cuda.is_available()`. Move the model before prediction and transfer each batch explicitly. Decide whether outputs should be returned on CPU, and do not call `torch.cuda.empty_ca` inside every batch.

Part 4 - Python code skeleton

```
import torch
```

```
def E03(model, X_cpu, batch_size, return_to_cpu=True):
    if not torch.cuda.is_available():
        return {"skipped": ...}
    device = torch.device("cuda")
    model = model.to(device).eval()
    outputs = []
    with torch.inference_mode():
        for start in range(0, X_cpu.shape[0], batch_size):
            batch = X_cpu[start:start + batch_size].to(device)
            prediction = model(batch)
            outputs.append(prediction.cpu() if return_to_cpu else prediction)
    return torch.cat(outputs, dim=...)

def main():
    print(E03(torch.nn.Linear(..., ...), torch..., ...))

if __name__ == "__main__":
    main()
```

10.5.5 Exercise 04 - Shape-Safe Prediction

Part 1 - Why this matters

An inference function should reject ambiguous or incompatible shapes before they reach matrix multiplication. Silent broadcasting or accidental squeezing can return plausible outputs attached to the wrong sample structure. Shape safety turns assumptions into executable checks.

The function must distinguish a single feature vector from a batch. It may normalize supported forms internally, but it should record whether a batch dimension was added and restore only the documented output form.

Part 2 - Technical concept and behavior

For expected feature count F, supported inputs may be:

$$\mathbf{x} \in \mathbb{R}^F \quad \text{or} \quad \mathbf{X} \in \mathbb{R}^{N \times F}$$

A single sample can be normalized to shape $(1, F)$. Any other final dimension is invalid, and rank greater than the declared contract should not be accepted accidentally.

Part 3 - Hints and Helper

Inspect `ndim` and the last dimension. Add a batch dimension only for a one-dimensional sample. Preserve a boolean indicating this normalization and decide whether to remove the output batch dimension afterward.

Part 4 - Python code skeleton

```
import torch

def E04(model, x, expected_features):
```

```
    single_sample = False
    if x.ndim == 1:
        x = x.unsqueeze(...)
        single_sample = True
    if x.ndim != 2 or x.shape[-1] != expected_features:
        raise ValueError(...)
    model.eval()
    with torch.inference_mode():
        output = model(x)
    if single_sample:
        output = output.squeeze(...)
    return output

def main():
    model = torch.nn.Linear(..., ...)
    print(E04(model, torch..., ...))

if __name__ == "__main__":
    main()
```

10.5.6 Exercise 05 - Disable Grad in Inference

Part 1 - Why this matters

Inference does not normally need parameter gradients. Disabling gradient recording reduces autograd overhead and prevents prediction code from accidentally accumulating graph history. This is especially important for long-running services that process repeated requests.

Calling `model.eval()` and disabling gradients solve different problems. Evaluation mode changes the behavior of specific modules such as dropout or batch normalization, while `no_grad` or `inference_mode` controls gradient-related execution. A correct inference function usually considers both.

Part 2 - Technical concept and behavior

During ordinary gradient-enabled execution, an output may carry a graph connection:

$$\hat{\mathbf{Y}}.\texttt{grad_fn} \neq \texttt{None}$$

Inside a gradient-disabled inference context, the returned output should not require gradients. `inference_mode` is more restrictive than `no_grad` and is appropriate when created tensors will remain in inference usage.

Part 3 - Hints and Helper

Run the same model once with gradients enabled and once inside `torch.inference_mode()`. Inspect `requires_grad` and `grad_fn`. Do not change parameter flags merely to perform temporary inference.

Part 4 - Python code skeleton

```
import torch

def E05(model, X):
    model.eval()
    enabled_output = model(X)
    with torch.inference_mode():
        inference_output = model(X)
        mode_enabled = torch.is_inference_mode_enabled()
    report = {
        "enabled_requires_grad": ...,
        "enabled_grad_fn": ...,
        "inference_requires_grad": ...,
        "inference_grad_fn": ...,
        "mode_enabled_inside": mode_enabled,
        "values_match": ...,
    }
    return report

def main():
    print(E05(torch.nn.Linear(..., ...), torch...))

if __name__ == "__main__":
    main()
```

10.5.7 Exercise 06 - Precision Switch During Inference

Part 1 - Why this matters

Changing inference precision affects numerical resolution, memory use, and device performance. A prediction service may choose a dtype different from an analytical experiment, but the model and input must be converted together and the resulting drift must be measured.

Precision conversion cannot be evaluated with independently initialized models. The comparison must begin from one state so observed differences come from representation and arithmetic rather than different weights.

Part 2 - Technical concept and behavior

For equivalent models:

$$D_{\max} = \max_i |\hat{y}_{64,i} - \text{float64}(\hat{y}_{32,i})|$$

The experiment reports absolute and relative differences as well as output dtype. Lower precision should not be selected solely from storage size without an application tolerance.

Part 3 - Hints and Helper

Use `copy.deepcopy()` before converting models with `.float()` and `.double()`. Match input dtype to parameter dtype, run both in inference mode, and compare on a common CPU `float64` representation.

Part 4 - Python code skeleton

```
import copy
import torch

def E06(model, X):
    model32 = copy.deepcopy(model).float().eval()
    model64 = copy.deepcopy(model).double().eval()
    with torch.inference_mode():
        output32 = model32(X.float())
        output64 = model64(X.double())
    output32_as_64 = ...
    absolute = ...
    relative = ...
    return output32, output64, absolute, relative

def main():
    print(E06(torch.nn.Linear(..., ...), torch...))

if __name__ == "__main__":
    main()
```

10.5.8 Exercise 07 - Prediction Consistency Test

Part 1 - Why this matters

A fixed deterministic model should return consistent predictions for the same prepared input. Unexpected differences can reveal accidental parameter mutation, stochastic training-mode behavior, dtype conversion, device changes, or nondeterministic operations.

Consistency is defined by a tolerance and context rather than exact equality in every environment. The test should record model state, input metadata, and maximum difference so a failure can be investigated instead of reduced to one boolean.

Part 2 - Technical concept and behavior

For repeated predictions $\hat{\mathbf{Y}}^{(1)}, \ldots, \hat{\mathbf{Y}}^{(R)}$, compare each with the first:

$$D_r = \max_i \left| \hat{Y}_i^{(r)} - \hat{Y}_i^{(1)} \right|$$

Consistency holds when every D_r lies within selected absolute and relative tolerances and the model state remains unchanged.

Part 3 - Hints and Helper

Set evaluation mode, use inference mode, and clone the state dictionary before testing. Run several repetitions without modifying the input. Compare outputs with `torch.allclose()` and verify parameters still match the snapshot.

Part 4 - Python code skeleton

```
import torch
```

```
def E07(model, X, repetitions, atol, rtol):
    snapshot = {
        key: value.detach().clone()
        for key, value in model.state_dict().items()
    }
    outputs = []
    model.eval()
    with torch.inference_mode():
        for _ in range(repetitions):
            outputs.append(model(X).clone())
    differences = [...]
    outputs_match = all(
        torch.allclose(outputs[0], output, atol=atol, rtol=rtol)
        for output in outputs[1:]
    )
    state_unchanged = ...
    return outputs_match, state_unchanged, differences

def main():
    print(E07(torch.nn.Linear(..., ...), torch..., ..., ..., ...))

if __name__ == "__main__":
    main()
```

10.5.9 Exercise 08 - Streaming Prediction Simulation

Part 1 - Why this matters

Some applications receive samples continuously rather than as one complete dataset. Streaming inference must preserve order, associate each prediction with its request identifier, and prevent an unbounded queue from exhausting memory.

Micro-batching can improve throughput by grouping nearby requests, but it introduces waiting time. This exercise simulates the data path and records batch formation without requiring a network service or hiding the tradeoff behind an API framework.

Part 2 - Technical concept and behavior

For stream items $(i_t, \mathbf{x}_t)$, a micro-batch B_k contains a bounded consecutive group. The model produces:

$$\hat{\mathbf{Y}}_k = f(\mathbf{X}_k)$$

and each row must be paired back with its original identifier. The buffer is flushed when it reaches the batch limit or the stream ends.

Part 3 - Hints and Helper

Represent the stream with a Python generator yielding `(id, tensor)`. Accumulate only up to `max_batch_size`, stack features, predict once, and yield paired outputs. Validate feature shapes before adding an item to the buffer.

Part 4 - Python code skeleton

```
import torch

def stream_source(sample_count, feature_count):
    for sample_id in range(sample_count):
        yield sample_id, torch...

def predict_buffer(model, identifiers, samples):
    batch = torch.stack(samples, dim=...)
    with torch.inference_mode():
        outputs = model(batch)
    return list(zip(identifiers, outputs))

def E08(model, stream, max_batch_size):
    buffer_ids, buffer_samples = [], []
    for sample_id, sample in stream:
        ...
        if len(buffer_samples) == max_batch_size:
            yield from predict_buffer(...)
            buffer_ids, buffer_samples = [], []
    if buffer_samples:
        yield from predict_buffer(...)

def main():
    model = torch.nn.Linear(..., ...)
    print(list(E08(model, stream_source(..., ...), ...)))

if __name__ == "__main__":
    main()
```

10.5.10 Exercise 09 - Prediction Latency Benchmark

Part 1 - Why this matters

Average inference latency can hide slow requests that determine user experience. A meaningful benchmark reports percentile distributions, separates warm-up from steady-state execution, and labels batch size, dtype, and device.

CUDA timing requires synchronization because kernel launches are asynchronous. CPU and GPU comparisons are also affected by data transfer, so the benchmark must state whether transfer is included rather than combining different measurement boundaries.

Part 2 - Technical concept and behavior

For ordered latency samples $l_{(1)}, \ldots, l_{(R)}$, report:

$$(L_{50}, L_{95}, L_{99})$$

alongside throughput and error count. Warm-up iterations are excluded from measured samples but their number remains part of benchmark metadata.

Part 3 - Hints and Helper

Use `time.perf_counter()`, evaluation mode, and inference mode. Synchronize immediately before and after a timed CUDA forward pass. Avoid printing inside the measured region and test multiple batch sizes with identical model state.

Part 4 - Python code skeleton

```
import time
import torch

def percentile(values, fraction):
    ordered = sorted(values)
    index = ...
    return ordered[index]

def benchmark(model, X, warmup, repetitions):
    device = next(model.parameters()).device
    with torch.inference_mode():
        for _ in range(warmup):
            model(X)
        samples = []
        for _ in range(repetitions):
            if device.type == "cuda":
                torch.cuda.synchronize()
            started = time.perf_counter()
            model(X)
            if device.type == "cuda":
                torch.cuda.synchronize()
            samples.append(time.perf_counter() - started)
    return {"p50": ..., "p95": ..., "p99": ...}

def E09(model, inputs):
    return ...

def main():
    print(E09(torch.nn.Linear(..., ...), ...))

if __name__ == "__main__":
    main()
```

10.5.11 Exercise 10 - Predict with Frozen Model

Part 1 - Why this matters

A frozen model has parameters whose gradient flags are disabled, preventing them from being selected for gradient-based training. This is useful when a model should serve predictions without accidental fine-tuning or when only another component may learn.

Freezing and inference mode are not interchangeable. Freezing changes parameter trainability, while inference mode temporarily changes autograd execution for the prediction. A robust test verifies both parameter flags and unchanged values.

Part 2 - Technical concept and behavior

For every model parameter θ_i, freezing enforces:

$$\theta_i.\texttt{requires_grad} = \text{False}$$

The forward equation remains unchanged. Predictions should match the pre-freeze model for identical state and input because freezing does not alter parameter values.

Part 3 - Hints and Helper

Clone the state before freezing, call `model.requires_grad_(False)`, and inspect all flags. Predict in inference mode, then compare state values with the snapshot. Do not rebuild the model or change its dtype during this test.

Part 4 - Python code skeleton

```
import torch

def E10(model, X):
    before = {
        key: value.detach().clone()
        for key, value in model.state_dict().items()
    }
    model.requires_grad_(...)
    flags = {
        name: parameter.requires_grad
        for name, parameter in model.named_parameters()
    }
    model.eval()
    with torch.inference_mode():
        output = model(X)
    unchanged = ...
    return output, flags, unchanged

def main():
    print(E10(torch.nn.Linear(..., ...), torch...))

if __name__ == "__main__":
    main()
```

10.5.12 Exercise 11 - CPU-to-GPU Prediction Sync Check

Part 1 - Why this matters

Equivalent CPU and GPU models should produce numerically close predictions when they use the same parameters, input values, and dtype. A mismatch may reveal incomplete state

transfer, device-specific precision effects, or an incorrect comparison boundary.

Synchronization has two meanings in this experiment. CUDA work must complete before timing or reading results, and model states must be synchronized by copying one reference state rather than initializing each device independently.

Part 2 - Technical concept and behavior

For CPU and GPU predictions:

$$D_{\max} = \max_i |\hat{y}_{\mathrm{CPU},i} - \mathrm{CPU}(\hat{y}_{\mathrm{GPU},i})|$$

The outputs are compared using justified tolerances. Exact equality is not required for floating-point kernels executed by different backends.

Part 3 - Hints and Helper

Deep-copy one model to the GPU, move the same input values, and use inference mode. Call `torch.cuda.synchronize()` before final GPU timing boundaries and copy the result to CPU before comparison. Report maximum absolute and relative differences.

Part 4 - Python code skeleton

```
import copy
import torch

def E11(model, X, atol, rtol):
    if not torch.cuda.is_available():
        return {"skipped": ...}
    cpu_model = copy.deepcopy(model).cpu().eval()
    gpu_model = copy.deepcopy(model).cuda().eval()
    with torch.inference_mode():
        cpu_output = cpu_model(X.cpu())
        gpu_output = gpu_model(X.cuda())
        torch.cuda.synchronize()
        gpu_on_cpu = gpu_output.cpu()
    return {
        "matches": torch.allclose(..., ..., atol=atol, rtol=rtol),
        "max_absolute_difference": ...,
        "max_relative_difference": ...,
    }

def main():
    print(E11(torch.nn.Linear(..., ...), torch..., ..., ...))

if __name__ == "__main__":
    main()
```

10.5.13 Exercise 12 - Large Batch Prediction Stress Test

Part 1 - Why this matters

Increasing batch size can improve throughput until memory pressure or kernel behavior becomes limiting. A stress test discovers practical boundaries, but it must not allocate recklessly or treat an out-of-memory crash as the only measurement.

A safe experiment estimates required tensor and output storage, imposes a memory budget, increases batch size gradually, and stops before exceeding the approved limit. Results should include latency, throughput, and peak memory rather than only the largest completed batch.

Part 2 - Technical concept and behavior

Approximate input and output storage for batch size B is:

$$M_B \approx B(F_{\text{in}} + F_{\text{out}})s$$

where s is bytes per element. Actual inference memory includes parameters, temporary workspace, allocator state, and framework overhead, so the estimate is a lower bound.

Part 3 - Hints and Helper

Accept an explicit maximum byte budget and pre-check each candidate shape. On CUDA, reset and read peak allocated memory for each approved run and synchronize timing. Stop when the budget is crossed; do not automatically keep doubling after a resource failure.

Part 4 - Python code skeleton

```
import time
import torch

def estimate_io_bytes(batch_size, in_features, out_features, dtype):
    element_size = torch.empty((), dtype=dtype).element_size()
    return ... * element_size

def E12(model, batch_sizes, in_features, max_bytes):
    device = next(model.parameters()).device
    results = []
    for batch_size in batch_sizes:
        estimate = estimate_io_bytes(...)
        if estimate > max_bytes:
            results.append({"batch_size": batch_size, "status": "skipped"})
            break
        X = torch.randn(batch_size, in_features,
                        device=device, dtype=next(model.parameters()).dtype)
        ...  # measure synchronized latency, throughput, and peak memory
        results.append(...)
    return results

def main():
    print(E12(torch.nn.Linear(..., ...), [...], ..., ...))

if __name__ == "__main__":
    main()
```

10.5.14 Exercise 13 - Numerical Drift Detection

Part 1 - Why this matters

Predictions can drift when model state, preprocessing, dtype, library behavior, or input distribution changes. A service may continue returning finite values while no longer matching its validated reference. Numerical drift detection turns a known prediction set into a regression test.

Not every tiny difference is a failure. Floating-point comparisons require tolerances chosen from application risk and baseline measurements. The report should distinguish output drift from input-data drift and model-state changes.

Part 2 - Technical concept and behavior

For baseline $\hat{\mathbf{Y}}_0$ and current output $\hat{\mathbf{Y}}_t$:

$$D_{\max} = \max_i |\hat{Y}_{t,i} - \hat{Y}_{0,i}|, \qquad D_{\text{mean}} = \frac{1}{N}\sum_i |\hat{Y}_{t,i} - \hat{Y}_{0,i}|$$

Drift is flagged when selected metrics exceed documented tolerances or when state and preprocessing identifiers differ.

Part 3 - Hints and Helper

Store baseline predictions, input checksum or identifier, model version, preprocessing version, dtype, and device metadata. Run the same reference set later and compare with `torch.allclose()`. Deliberately perturb one parameter to confirm detection.

Part 4 - Python code skeleton

```
import torch

def E13(model, reference_X, baseline, atol, rtol, metadata):
    model.eval()
    with torch.inference_mode():
        current = model(reference_X)
    absolute = (current - baseline).abs()
    report = {
        "matches": torch.allclose(current, baseline,
                                  atol=atol, rtol=rtol),
        "max_absolute": ...,
        "mean_absolute": ...,
        "model_version": ...,
        "preprocessing_version": ...,
    }
    return report

def main():
    model = torch.nn.Linear(..., ...)
    X = torch...
    baseline = torch...
```

```
    print(E13(model, X, baseline, ..., ..., ...))

if __name__ == "__main__":
    main()
```

10.5.15 Exercise 14 - Prediction with Missing Features

Part 1 - Why this matters

A prediction request may omit features because of sensor failure, incomplete user input, or an upstream schema change. Passing missing values directly can create non-finite output, while silently substituting arbitrary zeros can change the model's meaning.

Inference must reuse the missing-value policy fitted during training. The same feature order, fill values, and optional missingness indicators are part of the deployed model contract and should be versioned with it.

Part 2 - Technical concept and behavior

For stored training fill values $\boldsymbol{\mu}$ and missing mask $\mathbf{M}$:

$$X'_{ij} = \begin{cases} \mu_j, & M_{ij} = 1, \\ X_{ij}, & M_{ij} = 0. \end{cases}$$

The model receives $\mathbf{X}'$ only after checking that every required feature position exists. A completely absent required field is different from a present field containing `NaN`.

Part 3 - Hints and Helper

Validate feature count and names first. Use the stored training imputer values rather than recalculating them from prediction requests. Return the missing mask with the output so the caller can audit how much of each prediction was imputed.

Part 4 - Python code skeleton

```
import torch

def prepare_missing(X, fill_values):
    if X.shape[-1] != fill_values.numel():
        raise ValueError(...)
    mask = torch.isnan(X)
    prepared = torch.where(mask, fill_values.expand_as(X), X)
    if not torch.isfinite(prepared).all():
        raise ValueError(...)
    return prepared, mask

def E14(model, X, fill_values):
    prepared, mask = prepare_missing(X, fill_values)
    model.eval()
    with torch.inference_mode():
        prediction = model(prepared)
```

```
    return prediction, mask

def main():
    print(E14(torch.nn.Linear(..., ...), torch..., torch...))

if __name__ == "__main__":
    main()
```

10.5.16 Exercise 15 - Prediction Error Propagation

Part 1 - Why this matters

Input measurements contain error, and a linear model transfers that error into its prediction according to the learned weights. Understanding this propagation helps determine whether a prediction is sensitive to uncertain sensors or rounded features.

This is different from model residual error. Input uncertainty describes variation before the model is evaluated, while residuals describe disagreement between predictions and observed targets. A useful report keeps these sources separate.

Part 2 - Mathematical form

For perturbed input $\mathbf{x} + \Delta\mathbf{x}$:

$$\Delta\hat{y} = (\mathbf{x} + \Delta\mathbf{x})^{\mathsf{T}}\mathbf{w} + b - (\mathbf{x}^{\mathsf{T}}\mathbf{w} + b) = \Delta\mathbf{x}^{\mathsf{T}}\mathbf{w}$$

If feature errors are independent with variances σ_j^2, a simple propagated prediction variance is:

$$\sigma_{\hat{y}}^2 = \sum_j w_j^2 \sigma_j^2$$

Part 3 - Hints and Helper

Use a known linear layer and input perturbation. Compare the observed prediction change with the analytical change from the weights. Then accept per-feature standard deviations and calculate the expected output standard deviation under the stated independence assumption.

Part 4 - Python code skeleton

```
import torch

def E15(model, X, input_delta, feature_std):
    model.eval()
    with torch.inference_mode():
        original = model(X)
        perturbed = model(X + input_delta)
    observed_change = ...
    analytical_change = input_delta @ model.weight.T
    propagated_variance = ...  # use squared weights and variances
    propagated_std = torch.sqrt(...)
```

```
    return {
        "observed_change": observed_change,
        "analytical_change": analytical_change,
        "propagated_std": propagated_std,
        "matches": torch.allclose(..., ...),
    }

def main():
    print(E15(torch.nn.Linear(..., ...), torch..., torch..., torch...))

if __name__ == "__main__":
    main()
```

10.5.17 Exercise 16 - TorchScript Prediction Export

Part 1 - Why this matters

Exporting a prediction graph can separate inference from the original Python model definition and support deployment workflows. TorchScript historically provided scripting and tracing for this purpose, so understanding a simple trace remains useful when maintaining existing systems.

TorchScript is now deprecated in current PyTorch documentation in favor of `torch.export`. This exercise therefore treats TorchScript as a legacy compatibility task, verifies its limitations, and requires the reader to record the migration alternative rather than presenting it as the default for new systems.

Part 2 - Technical concept and behavior

Tracing observes tensor operations executed for example input $\mathbf{X}_e$ and produces a traced module T. Validation requires:

$$T(\mathbf{X}_{\text{test}}) \approx f(\mathbf{X}_{\text{test}})$$

for representative supported inputs. A trace may not correctly preserve unobserved data-dependent Python control flow.

Part 3 - Hints and Helper

Use a simple evaluated linear module and representative example input. Create a trace with `torch.jit.trace()`, save and load it using TorchScript APIs, then compare eager and loaded outputs on more than the tracing input. Add a note that new export work should evaluate `torch.export`.

Part 4 - Python code skeleton

```
import torch

def E16(model, example_input, test_inputs, export_path):
    model.eval()
    traced = torch.jit.trace(..., ...)
```

```
    torch.jit.save(..., export_path)
    loaded = torch.jit.load(...)
    comparisons = []
    with torch.inference_mode():
        for X in test_inputs:
            eager_output = model(X)
            exported_output = loaded(X)
            comparisons.append(torch.allclose(..., ...))
    return traced, comparisons

def main():
    model = torch.nn.Linear(..., ...)
    print(E16(model, torch..., [...], ...))

if __name__ == "__main__":
    main()
```

10.5.18 Exercise 17 - Inference Memory Profiling

Part 1 - Why this matters

Inference memory includes model parameters, input batches, outputs, and temporary workspace. A function that succeeds with one request may fail under a larger batch or concurrent workload. Profiling establishes a measured capacity boundary.

Memory numbers must be labeled by device and measurement method. Python object tracking does not describe all native tensor storage, while CUDA allocator statistics do not represent CPU memory. This exercise uses tensor estimates everywhere and device-specific peak statistics when CUDA is used.

Part 2 - Technical concept and behavior

Known tensor storage is:

$$M_{\text{known}} = \sum_{T \in \{\theta, X, \hat{Y}\}} \text{numel}(T)\,\text{element_size}(T)$$

Peak device allocation can exceed this value because operations may require temporary buffers. The experiment compares known storage with measured CUDA peak allocation where available.

Part 3 - Hints and Helper

Calculate parameter, input, and output bytes separately. On CUDA, reset peak memory statistics before the forward pass, synchronize after it, and read the peak allocation. Repeat after warm-up and do not confuse reserved memory with allocated memory.

Part 4 - Python code skeleton

```
import torch
```

```
def tensor_bytes(tensor):
    return tensor.numel() * tensor.element_size()

def E17(model, X):
    device = next(model.parameters()).device
    parameter_bytes = sum(tensor_bytes(p) for p in model.parameters())
    if device.type == "cuda":
        torch.cuda.reset_peak_memory_stats(device)
    with torch.inference_mode():
        output = model(X)
    if device.type == "cuda":
        torch.cuda.synchronize(device)
        peak_allocated = torch.cuda.max_memory_allocated(device)
    else:
        peak_allocated = None
    known_bytes = parameter_bytes + tensor_bytes(X) + tensor_bytes(output)
    return {
        "known_bytes": known_bytes,
        "peak_allocated": peak_allocated,
        "output": output,
    }

def main():
    print(E17(torch.nn.Linear(..., ...), torch...))

if __name__ == "__main__":
    main()
```

10.5.19 Exercise 18 - Prediction API Wrapper

Part 1 - Why this matters

An application should not call a model directly from unvalidated external data. A prediction wrapper enforces feature names and order, preprocessing version, batch limits, dtype, device placement, and a stable response structure.

The wrapper is also the correct place to attach model-version metadata and controlled error messages. It should not expose internal tensors or silently repair an incompatible request. This exercise builds the Python boundary without requiring a web framework.

Part 2 - Technical concept and behavior

Represent a request as records with a declared schema and transform them into:

$$\mathbf{X}_{\text{validated}} \rightarrow P_{\text{versioned}} \rightarrow f_{\text{versioned}} \rightarrow \hat{\mathbf{Y}}$$

The response pairs predictions with request identifiers and model metadata. Validation occurs before device transfer or model execution.

Part 3 - Hints and Helper

Create a class that owns the model, expected feature list, preprocessing object, version, device, and maximum batch size. Validate missing and extra fields, preserve request IDs, build a tensor in fixed order, predict in inference mode, and convert outputs to ordinary Python values only at the response boundary.

Part 4 - Python code skeleton

```
import torch

class PredictionWrapper:
    def __init__(self, model, feature_names, preprocessor,
                 model_version, device, max_batch_size):
        self.model = model.to(device).eval()
        self.feature_names = feature_names
        self.preprocessor = preprocessor
        self.model_version = model_version
        self.device = device
        self.max_batch_size = max_batch_size

    def validate(self, records):
        ...

    def build_tensor(self, records):
        ...

    def predict(self, records):
        self.validate(records)
        X = self.build_tensor(records)
        X = self.preprocessor.transform(X).to(self.device)
        with torch.inference_mode():
            output = self.model(X)
        return {
            "model_version": self.model_version,
            "predictions": ...,
        }

def E18(wrapper, records):
    return wrapper.predict(records)

def main():
    wrapper = PredictionWrapper(...)
    print(E18(wrapper, ...))

if __name__ == "__main__":
    main()
```

10.5.20 Exercise 19 - Prediction Failure Debugging

Part 1 - Why this matters

Prediction failures can originate in request schema, preprocessing, shape, dtype, device, model state, non-finite values, or resource limits. Handling all failures as one generic model error hides the stage that must be corrected.

The final exercise creates a diagnostic path that validates each boundary in order and records structured evidence. It must preserve unexpected programming errors rather than catching every exception indiscriminately, while expected input and runtime failures receive controlled reports.

Part 2 - Technical concept and behavior

Inference can be represented as:

$$R \xrightarrow{\text{validate}} X \xrightarrow{\text{prepare}} X' \xrightarrow{\text{place}} X'_d \xrightarrow{f} \hat{Y} \xrightarrow{\text{serialize}} S$$

Each transition has preconditions and postconditions. The earliest failed transition determines which later outputs are untrustworthy.

Part 3 - Hints and Helper

Give each stage a name and record shapes, dtype, device, finite status, and elapsed time where relevant. Catch expected `ValueError`, `TypeError`, and selected `RuntimeError` cases at the outer request boundary. Test wrong feature count, NaN input, device mismatch, and oversized batch separately.

Part 4 - Python code skeleton

```
import time
import torch

def inspect_tensor(X):
    return {
        "shape": tuple(X.shape),
        "dtype": str(X.dtype),
        "device": str(X.device),
        "finite": bool(torch.isfinite(X).all()),
    }

def E19(model, request, prepare, device, max_batch_size):
    report = {"stages": []}
    try:
        ...  # validate request schema and batch size
        X = prepare(request)
        report["stages"].append(("prepared", inspect_tensor(X)))
        ...  # verify features, dtype, and finite values
        X = X.to(device)
        report["stages"].append(("placed", inspect_tensor(X)))
        model = model.to(device).eval()
        with torch.inference_mode():
            output = model(X)
        ...  # validate and serialize output
```

```
        report["status"] = "passed"
    except (ValueError, TypeError, RuntimeError) as error:
        report["status"] = "failed"
        report["error"] = {
            "type": type(error).__name__,
            "message": str(error),
        }
    return report

def main():
    print(E19(torch.nn.Linear(..., ...), ..., ..., ..., ...))

if __name__ == "__main__":
    main()
```

10.6 LR04 - Error Measurement and Loss Definition

Models and Training

10.6.1 Exercise 00 - Define MSE Loss Manually

Part 1 - Why this matters

Mean squared error converts the difference between regression predictions and targets into one scalar training objective. Implementing it manually exposes the subtraction, squaring, and reduction that a library function normally combines.

The formula also reveals the behavior of the loss. Squaring removes the error sign and gives large errors disproportionate influence. The reader should understand this consequence before using MSE as an automatic default.

Part 2 - Mathematical form

For N scalar predictions:

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$$

For tensors with several output elements per sample, the ordinary element-wise mean divides by the total number of compared elements. Prediction and target shapes must match exactly.

Part 3 - Hints and Helper

Validate equal shapes before subtracting to prevent accidental broadcasting. Calculate and return the residuals, squared errors, and scalar mean so every stage can be inspected. Use floating-point tensors.

Part 4 - Python code skeleton

```
import torch

def E00(prediction, target):
    if prediction.shape != target.shape:
        raise ValueError(...)
    residual = ...
    squared_error = ...
    mse = ...
    return residual, squared_error, mse

def main():
    prediction = torch...
    target = torch...
    print(E00(prediction, target))

if __name__ == "__main__":
    main()
```

10.6.2 Exercise 01 - MSE via PyTorch

Part 1 - Why this matters

PyTorch provides tested MSE implementations that integrate with autograd and support standard reduction modes. Comparing the framework result with the manual equation verifies that the reader understands what the loss object calculates.

A loss function does not correct wrong tensor semantics. PyTorch may warn or broadcast incompatible shapes, so the calling code remains responsible for validating prediction and target structure before evaluation.

Part 2 - Technical concept and behavior

With default mean reduction, `nn.MSELoss` computes:

$$\mathcal{L} = \text{mean}\left((\hat{\mathbf{Y}} - \mathbf{Y})^2\right)$$

The result is a scalar tensor connected to the prediction graph when the prediction requires gradients.

Part 3 - Hints and Helper

Construct `nn.MSELoss(reduction="mean")` and compare its result with the manual calculation. Inspect the output shape, dtype, device, `requires_grad`, and `grad_fn`.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def E01(prediction, target):
    if prediction.shape != target.shape:
```

```
        raise ValueError(...)
    criterion = nn.MSELoss(reduction=...)
    pytorch_loss = criterion(..., ...)
    manual_loss = ...
    return pytorch_loss, manual_loss, torch.allclose(..., ...)

def main():
    prediction = torch.tensor(..., requires_grad=True)
    target = torch...
    print(E01(prediction, target))

if __name__ == "__main__":
    main()
```

10.6.3 Exercise 02 - Loss Reduction Modes

Part 1 - Why this matters

Reduction controls how individual squared errors become a returned loss. The choice changes loss scale, gradient scale, and how batch results should be aggregated. It is therefore part of the mathematical definition, not a display option.

The unreduced form preserves every element and supports masks or custom weighting. Sum reduction grows with the number of elements, while mean reduction normalizes by that count. Comparing all three modes makes these relationships visible.

Part 2 - Mathematical form

Let $l_i = (\hat{y}_i - y_i)^2$. Then:

$$\mathcal{L}_{\text{none}} = (l_1, \ldots, l_N), \qquad \mathcal{L}_{\text{sum}} = \sum_i l_i, \qquad \mathcal{L}_{\text{mean}} = \frac{1}{N} \sum_i l_i$$

Thus $\mathcal{L}_{\text{sum}} = N\mathcal{L}_{\text{mean}}$ when N counts every output element.

Part 3 - Hints and Helper

Compute all modes from the same tensors using `nn.MSELoss`. Record output shapes and verify the sum-mean identity with `prediction.numel()`. Do not assume N means only batch size when there are multiple outputs.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def E02(prediction, target):
    losses = {}
    for reduction in ("none", "sum", "mean"):
        criterion = nn.MSELoss(reduction=reduction)
        losses[reduction] = criterion(...)
```

```
    relation_holds = torch.allclose(
        losses["sum"],
        losses["mean"] * ...,
    )
    return losses, relation_holds

def main():
    print(E02(torch..., torch...))

if __name__ == "__main__":
    main()
```

10.6.4 Exercise 03 - Batch Loss Computation

Part 1 - Why this matters

Computing a mean loss for each batch and then averaging those means can bias an epoch result when the final batch is smaller. The epoch loss must represent all compared elements with the intended weighting.

Correct aggregation also affects comparisons between batch sizes. The same predictions and targets should produce the same dataset-level MSE regardless of how they were divided into batches.

Part 2 - Mathematical form

For batches with squared-error sums S_k and element counts n_k:

$$\mathcal{L}_{\text{epoch}} = \frac{\sum_k S_k}{\sum_k n_k}$$

If every sample has one output, accumulating `batch_mean * batch_size` is equivalent. For general outputs, element count is the safer denominator.

Part 3 - Hints and Helper

Use sum reduction inside the loop and accumulate `prediction.numel()`. Divide only after all batches. Compare the result with MSE calculated in one call over the concatenated dataset and test an incomplete final batch.

Part 4 - Python code skeleton

```
import torch
from torch.utils.data import DataLoader, TensorDataset

def E03(model, loader):
    total_squared_error = 0.0
    total_elements = 0
    for X_batch, y_batch in loader:
        prediction = model(X_batch)
        batch_sum = torch.nn.functional.mse_loss(
            prediction, y_batch, reduction="sum"
```

```
        )
        total_squared_error += ...
        total_elements += ...
    return total_squared_error / ...

def main():
    dataset = TensorDataset(torch..., torch...)
    loader = DataLoader(dataset, batch_size=..., drop_last=False)
    model = torch.nn.Linear(..., ...)
    print(E03(model, loader))

if __name__ == "__main__":
    main()
```

10.6.5 Exercise 04 - Loss on CUDA

Part 1 - Why this matters

CUDA can compute predictions and loss on the GPU, but model parameters, input, target, and resulting operations must share a compatible device. Explicit placement prevents hidden transfers and device mismatch failures.

The scalar loss often remains on the GPU until its value is needed by Python. Calling `loss.item()` transfers a scalar value and should not be inserted unnecessarily inside a tightly timed region.

Part 2 - Technical concept and behavior

For device d:

$$d_\theta = d_X = d_y = d_{\mathcal{L}}$$

must hold during the CUDA loss calculation. Numerical comparison with CPU should use tolerances because floating-point results across devices are not guaranteed to be bitwise identical.

Part 3 - Hints and Helper

Check CUDA availability and return a clear skipped status otherwise. Move model, features, and target explicitly. Compute the loss, inspect every device, and synchronize before timing or comparing completed results.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def E04(model, X, target):
    if not torch.cuda.is_available():
        return {"skipped": ...}
    device = torch.device("cuda")
```

```
    model = model.to(device)
    X = X.to(device)
    target = target.to(device)
    prediction = model(X)
    loss = nn.MSELoss()(..., ...)
    return {
        "loss": loss,
        "model_device": next(model.parameters()).device,
        "prediction_device": ...,
        "target_device": ...,
        "loss_device": ...,
    }

def main():
    print(E04(nn.Linear(..., ...), torch..., torch...))

if __name__ == "__main__":
    main()
```

10.6.6 Exercise 05 - Loss Gradient Inspection

Part 1 - Why this matters

The loss gradient tells the model how each prediction contributed to error. For MSE, the gradient grows with residual magnitude and points away from the target, explaining why gradient descent moves predictions toward observed values.

Inspecting this derivative directly connects the loss equation to autograd. It also helps detect unintended reduction, broadcasting, or scaling because each of those choices changes the expected gradient.

Part 2 - Mathematical form

For N compared elements:

$$\mathcal{L} = \frac{1}{N}\sum_i (\hat{y}_i - y_i)^2$$

and:

$$\frac{\partial \mathcal{L}}{\partial \hat{y}_i} = \frac{2}{N}(\hat{y}_i - y_i)$$

The parameter gradient then follows through the model by the chain rule.

Part 3 - Hints and Helper

Create a leaf prediction tensor requiring gradients. Use `torch.autograd.grad()` or call backward and inspect its gradient. Calculate the analytical value independently and compare shapes and values.

Part 4 - Python code skeleton

```
import torch

def E05(prediction, target):
    loss = torch.nn.functional.mse_loss(
        prediction, target, reduction="mean"
    )
    automatic = torch.autograd.grad(loss, prediction)[0]
    analytical = ...
    return loss, automatic, analytical, torch.allclose(..., ...)

def main():
    prediction = torch.tensor(..., requires_grad=True)
    target = torch...
    print(E05(prediction, target))

if __name__ == "__main__":
    main()
```

10.6.7 Exercise 06 - Outlier Impact on Loss

Part 1 - Why this matters

Because MSE squares residuals, one extreme error can dominate many ordinary errors. This makes the loss responsive to serious mistakes but also sensitive to corrupted targets or rare events.

The correct response is not automatically to remove the outlier. The reader should quantify its contribution, investigate its provenance, and compare a robust reference such as absolute error before making a data or loss decision.

Part 2 - Mathematical form

For residuals e_i, each MSE contribution is e_i^2. The fraction contributed by selected outlier o is:

$$q_o = \frac{e_o^2}{\sum_i e_i^2}$$

As $|e_o|$ grows, its squared contribution can approach the entire loss sum.

Part 3 - Hints and Helper

Compute unreduced squared and absolute errors for a clean target and a copy containing one controlled outlier. Report total losses and the outlier contribution fraction. Keep predictions identical in both cases.

Part 4 - Python code skeleton

```
import torch

def summarize_errors(prediction, target, outlier_index):
```

```
    squared = torch.nn.functional.mse_loss(
        prediction, target, reduction="none"
    )
    absolute = (prediction - target).abs()
    fraction = squared[outlier_index] / squared.sum()
    return squared.mean(), absolute.mean(), fraction

def E06(prediction, clean_target, outlier_index, outlier_change):
    changed_target = clean_target.clone()
    changed_target[outlier_index] += ...
    return {
        "clean": summarize_errors(...),
        "with_outlier": summarize_errors(...),
    }

def main():
    print(E06(torch..., torch..., ..., ...))

if __name__ == "__main__":
    main()
```

10.6.8 Exercise 07 - Loss Scaling Effects

Part 1 - Why this matters

Changing the numerical scale of regression targets changes the numerical scale of MSE and its gradients. Two datasets describing the same relationship in different units can therefore produce losses that are not directly comparable.

This effect influences learning-rate selection and monitoring thresholds. A large loss may reflect units rather than a worse relative model, so the target scale must accompany every interpretation.

Part 2 - Mathematical form

If every residual is multiplied by constant c:

$$e'_i = ce_i$$

then:

$$\mathrm{MSE}' = c^2\,\mathrm{MSE}$$

while the gradient with respect to the scaled prediction is proportional to c. The experiment verifies these expected ratios.

Part 3 - Hints and Helper

Scale both prediction and target by the same nonzero factor so residuals scale consistently. Compare loss ratio with c^2 and gradient norms with the analytical expectation. Test both a larger and smaller unit scale.

Part 4 - Python code skeleton

```
import torch

def loss_and_gradient(prediction, target):
    prediction = prediction.detach().clone().requires_grad_(True)
    loss = torch.nn.functional.mse_loss(prediction, target)
    gradient = torch.autograd.grad(loss, prediction)[0]
    return loss, gradient

def E07(prediction, target, scale):
    base_loss, base_gradient = loss_and_gradient(...)
    scaled_loss, scaled_gradient = loss_and_gradient(
        prediction * scale,
        target * scale,
    )
    return {
        "loss_ratio": ...,
        "expected_loss_ratio": ...,
        "gradient_norm_ratio": ...,
    }

def main():
    print(E07(torch..., torch..., ...))

if __name__ == "__main__":
    main()
```

10.6.9 Exercise 08 - Loss Precision Comparison

Part 1 - Why this matters

MSE performs subtraction, squaring, and reduction, so rounding effects can accumulate across many values. Comparing single and double precision reveals whether the chosen dtype materially changes the loss for a specific numerical range.

PyTorch does not guarantee bitwise-identical floating-point results across devices or mathematically equivalent computation orders. The comparison should use meaningful tolerances and one common source of values.

Part 2 - Technical concept and behavior

Let:

$$\mathcal{L}_{32} = \mathrm{MSE}(\hat{\mathbf{y}}_{32}, \mathbf{y}_{32}), \qquad \mathcal{L}_{64} = \mathrm{MSE}(\hat{\mathbf{y}}_{64}, \mathbf{y}_{64})$$

Compare absolute and relative difference after converting $\mathcal{L}_{32}$ to double precision.

Part 3 - Hints and Helper

Create prediction and target in `float64`, then derive `float32` copies. Test ordinary values and a difficult range with large baselines and small residuals. Report dtypes and both difference measures.

Part 4 - Python code skeleton

```
import torch

def E08(prediction64, target64):
    prediction32 = prediction64.float()
    target32 = target64.float()
    loss64 = torch.nn.functional.mse_loss(prediction64, target64)
    loss32 = torch.nn.functional.mse_loss(prediction32, target32)
    absolute = ...
    relative = ...
    return loss32, loss64, absolute, relative

def main():
    prediction64 = torch.tensor(..., dtype=torch.float64)
    target64 = torch.tensor(..., dtype=torch.float64)
    print(E08(prediction64, target64))

if __name__ == "__main__":
    main()
```

10.6.10 Exercise 09 - Loss Stability Test

Part 1 - Why this matters

Finite predictions do not guarantee a finite squared loss. Large residuals can overflow when squared, while NaN or infinite inputs propagate through the reduction. A stable pipeline checks numerical validity before backward contaminates gradients.

The objective is to identify a safe operating range and failure symptoms, not to hide invalid values with arbitrary replacement. Higher precision, scaling, clipping upstream data, or revising the model may be appropriate depending on the cause.

Part 2 - Technical concept and behavior

For maximum representable finite value M in a dtype, residual magnitude larger than approximately $\sqrt{M}$ may overflow when squared. The stability report tests:

$$\text{isfinite}(\hat{\mathbf{y}}), \quad \text{isfinite}(\mathbf{y}), \quad \text{isfinite}(\mathcal{L})$$

and, when backward is performed, finite gradients.

Part 3 - Hints and Helper

Generate controlled residual magnitudes across several scales. Use `torch.finfo(dtype)` to inspect the dtype range and `torch.isfinite()` at each stage. Stop before backward when the loss is non-finite.

Part 4 - Python code skeleton

```
import torch

def test_scale(dtype, residual_scale):
    prediction = torch.full((...), residual_scale, dtype=dtype,
                            requires_grad=True)
    target = torch.zeros_like(prediction)
    loss = torch.nn.functional.mse_loss(prediction, target)
    report = {
        "scale": residual_scale,
        "finite_prediction": ...,
        "finite_loss": ...,
    }
    if torch.isfinite(loss):
        loss.backward()
        report["finite_gradient"] = ...
    return report

def E09(dtypes, scales):
    return [test_scale(dtype, scale)
            for dtype in dtypes for scale in scales]

def main():
    print(E09([...], [..., ..., ...]))

if __name__ == "__main__":
    main()
```

10.6.11 Exercise 10 - Custom Loss Class

Part 1 - Why this matters

A custom loss module can enforce shape checks, expose a controlled reduction policy, and remain reusable inside training code. Reimplementing MSE as a class shows how a mathematical objective becomes an `nn.Module` without introducing hidden state.

Custom code also creates responsibility. Its forward values and gradients must match a trusted reference, and invalid reduction names or mismatched shapes should fail clearly before producing misleading results.

Part 2 - Mathematical form

The class calculates element-wise loss:

$$L = (\hat{\mathbf{Y}} - \mathbf{Y})^2$$

and returns L, $\sum L$, or $\text{mean}(L)$ according to the configured reduction. It has no learnable parameters.

Part 3 - Hints and Helper

Validate the reduction in `__init__()` and shapes in `forward()`. Compare every mode with `nn.MSELoss` using both values and gradients. Do not call `.item()` inside the forward method.

Part 4 - Python code skeleton

```
import torch
from torch import nn

class CustomMSELoss(nn.Module):
    def __init__(self, reduction="mean"):
        super().__init__()
        if reduction not in {...}:
            raise ValueError(...)
        self.reduction = reduction

    def forward(self, prediction, target):
        if prediction.shape != target.shape:
            raise ValueError(...)
        elementwise = ...
        if self.reduction == "none":
            return elementwise
        if self.reduction == "sum":
            return ...
        return ...

def E10(prediction, target):
    custom = CustomMSELoss(...)
    reference = nn.MSELoss(reduction=...)
    return custom(...), reference(...)

def main():
    print(E10(torch..., torch...))

if __name__ == "__main__":
    main()
```

10.6.12 Exercise 11 - Loss with Masked Samples

Part 1 - Why this matters

Some targets may be unavailable or intentionally excluded from a loss. A mask allows valid elements to contribute while ignored elements produce no training signal. This is common in partially labeled or quality-controlled datasets.

Multiplying errors by a mask and then taking an ordinary mean is usually wrong because masked zeros remain in the denominator. The correct mean divides by the number or weight of valid elements and must define what happens when none are valid.

Part 2 - Mathematical form

For binary mask $m_i \in \{0, 1\}$:

$$\mathcal{L}_{\text{masked}} = \frac{\sum_i m_i(\hat{y}_i - y_i)^2}{\sum_i m_i}$$

The denominator must be positive. Mask shape must match the loss elements or follow an explicitly documented expansion rule.

Part 3 - Hints and Helper

Convert a boolean mask to the loss dtype only for arithmetic. Validate exact shape and at least one valid element. Return the valid count and compare with manually selecting the valid entries before calling MSE.

Part 4 - Python code skeleton

```
import torch

def E11(prediction, target, mask):
    if prediction.shape != target.shape or mask.shape != prediction.shape:
        raise ValueError(...)
    valid_count = mask.sum()
    if valid_count == 0:
        raise ValueError(...)
    squared = ...
    masked_sum = (squared * mask.to(squared.dtype)).sum()
    masked_mean = masked_sum / ...
    selected_reference = torch.nn.functional.mse_loss(
        prediction[mask], target[mask]
    )
    return masked_mean, selected_reference, valid_count

def main():
    print(E11(torch..., torch..., torch...))

if __name__ == "__main__":
    main()
```

10.6.13 Exercise 12 - Loss Backward Pass Check

Part 1 - Why this matters

A scalar loss is useful for training only if backward propagation reaches the intended model parameters. Checking gradients after MSE verifies that the prediction graph is connected and that every trainable component receives a finite, correctly shaped signal.

A missing gradient differs from a zero gradient. `None` indicates no stored gradient, while zeros may be the mathematically correct derivative for a perfectly fitted sample or zero-valued input.

Part 2 - Mathematical form

For $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b$:

$$\nabla_{\mathbf{w}}\mathcal{L} = \mathbf{X}^{\intercal}\frac{2}{N}(\hat{\mathbf{y}} - \mathbf{y})$$

and the bias gradient sums the prediction-gradient contributions across samples. Shapes must match their respective parameters.

Part 3 - Hints and Helper

Clear gradients to `None`, compute prediction and scalar MSE, then call backward once. Report gradient existence, shape, finiteness, and norm for every parameter. Use a small case to compare the weight gradient manually.

Part 4 - Python code skeleton

```
import torch

def E12(model, X, target):
    model.zero_grad(set_to_none=True)
    prediction = model(X)
    loss = torch.nn.functional.mse_loss(prediction, target)
    loss.backward()
    report = {}
    for name, parameter in model.named_parameters():
        gradient = parameter.grad
        report[name] = {
            "exists": gradient is not None,
            "shape_matches": ...,
            "finite": ...,
            "norm": ...,
        }
    analytical_weight_gradient = ...
    return loss.item(), report, analytical_weight_gradient

def main():
    print(E12(torch.nn.Linear(..., ...), torch..., torch...))

if __name__ == "__main__":
    main()
```

10.6.14 Exercise 13 - Loss Accumulation Debugging

Part 1 - Why this matters

Loss accumulation and gradient accumulation are separate processes. Summing tensor losses across many batches can retain their computation graphs and consume memory, while repeated backward calls without clearing gradients intentionally or accidentally add parameter gradients.

For logging, detached scalar values should be accumulated. For training, the gradient-clear schedule must match the intended effective batch size. This exercise contrasts these states so a memory or optimization bug can be identified precisely.

Part 2 - Technical concept and behavior

Logging epoch loss may accumulate:

$$S_{\text{loss}} \leftarrow S_{\text{loss}} + \text{detach}(S_k)$$

while gradients follow:

$$g \leftarrow g + \nabla_\theta \mathcal{L}_k$$

until explicitly cleared. These operations should not be confused merely because both use addition.

Part 3 - Hints and Helper

Accumulate `loss.item()` or a detached sum for metrics. Record gradient norms before backward, after backward, and after zeroing. Demonstrate two batches with and without zeroing, but do not retain every graph in an epoch-level tensor.

Part 4 - Python code skeleton

```
import torch

def gradient_norm(model):
    values = [
        parameter.grad.norm()
        for parameter in model.parameters()
        if parameter.grad is not None
    ]
    return ... if values else ...

def E13(model, batches, accumulate_gradients):
    logged_loss_sum = 0.0
    stages = []
    model.zero_grad(set_to_none=True)
    for X, target in batches:
        if not accumulate_gradients:
            model.zero_grad(set_to_none=True)
        loss = torch.nn.functional.mse_loss(model(X), target)
        logged_loss_sum += ...
        before = gradient_norm(model)
        loss.backward()
        after = gradient_norm(model)
        stages.append((before, after))
    return logged_loss_sum, stages

def main():
    print(E13(torch.nn.Linear(..., ...), ..., ...))

if __name__ == "__main__":
    main()
```

10.6.15 Exercise 14 - Loss Memory Footprint

Part 1 - Why this matters

Unreduced loss preserves one value per prediction element, while mean and sum reductions return a scalar. On large outputs, retaining element-wise losses or their computation graphs can substantially increase memory use.

The returned loss tensor is only part of the footprint. Autograd may retain intermediate values required for backward, so the experiment distinguishes visible output storage from measured peak allocation on CUDA.

Part 2 - Technical concept and behavior

For N compared elements and element size s:

$$M_{\text{none}} \approx Ns, \qquad M_{\text{mean}} \approx s$$

for returned tensor storage. Actual training memory includes prediction, target, model parameters, gradients, and graph intermediates.

Part 3 - Hints and Helper

Compute losses with `none`, `sum`, and `mean` from the same tensors. Report `numel() * element_size()` for each output. On CUDA, optionally reset peak statistics and synchronize around each isolated case.

Part 4 - Python code skeleton

```
import torch

def tensor_bytes(tensor):
    return tensor.numel() * tensor.element_size()

def E14(prediction, target):
    results = {}
    for reduction in ("none", "sum", "mean"):
        loss = torch.nn.functional.mse_loss(
            prediction, target, reduction=reduction
        )
        results[reduction] = {
            "shape": tuple(loss.shape),
            "visible_bytes": tensor_bytes(loss),
            "requires_grad": loss.requires_grad,
        }
        ...  # optionally measure isolated CUDA peak allocation
    return results

def main():
    prediction = torch.randn(..., requires_grad=True)
    target = torch...
    print(E14(prediction, target))
```

```
if __name__ == "__main__":
    main()
```

10.6.16 Exercise 15 - CPU vs GPU Loss Parity

Part 1 - Why this matters

The same model state and data should produce numerically close losses on CPU and GPU. Differences beyond a justified tolerance may indicate unequal parameters, dtype changes, incomplete transfers, or an incorrect comparison procedure.

Floating-point computations are not guaranteed to be bitwise identical across devices. Parity therefore means acceptable numerical agreement under a documented tolerance, not exact equality in every environment.

Part 2 - Technical concept and behavior

For equivalent inputs and states:

$$D = |\mathcal{L}_{\text{CPU}} - \mathcal{L}_{\text{GPU}}|$$

The comparison also records prediction difference so the reader can determine whether loss drift originated before or during reduction.

Part 3 - Hints and Helper

Deep-copy one reference model, preserve dtype, and move copies to each device. Use the same input values, synchronize CUDA, and return GPU results to CPU. Report absolute and relative differences for predictions and loss.

Part 4 - Python code skeleton

```
import copy
import torch

def E15(model, X, target, atol, rtol):
    if not torch.cuda.is_available():
        return {"skipped": ...}
    cpu_model = copy.deepcopy(model).cpu()
    gpu_model = copy.deepcopy(model).cuda()
    cpu_prediction = cpu_model(X.cpu())
    cpu_loss = torch.nn.functional.mse_loss(
        cpu_prediction, target.cpu()
    )
    gpu_prediction = gpu_model(X.cuda())
    gpu_loss = torch.nn.functional.mse_loss(
        gpu_prediction, target.cuda()
    )
    torch.cuda.synchronize()
    return {
```

```
        "loss_matches": torch.allclose(
            cpu_loss, gpu_loss.cpu(), atol=atol, rtol=rtol
        ),
        "prediction_difference": ...,
        "loss_difference": ...,
    }

def main():
    print(E15(torch.nn.Linear(..., ...), torch..., torch..., ..., ...))

if __name__ == "__main__":
    main()
```

10.6.17 Exercise 16 - Loss under Mixed Precision

Part 1 - Why this matters

Automatic mixed precision allows selected operations to use lower precision while numerically sensitive operations use a safer dtype. This can improve CUDA performance and reduce memory use, but the resulting loss and gradients must remain finite and close to a full-precision reference.

PyTorch's autocast policy may run a linear operation in lower precision and MSE loss in float32. The reader should inspect actual output and loss dtypes instead of assuming that every operation uses one global dtype.

Part 2 - Technical concept and behavior

Compare full-precision loss $\mathcal{L}_{32}$ with autocast loss $\mathcal{L}_{\text{AMP}}$:

$$D = |\mathcal{L}_{32} - \mathcal{L}_{\text{AMP}}|$$

For float16 training, gradient scaling can multiply the loss before backward to reduce underflow risk, then unscale gradients before the optimizer update.

Part 3 - Hints and Helper

Use current `torch.autocast(device_type="cuda")` and `torch.amp.GradScaler("cuda")` APIs. Do not manually call `half()` on model or inputs when using autocast. Skip clearly without CUDA and inspect finite loss and gradients.

Part 4 - Python code skeleton

```
import copy
import torch

def E16(model, X, target):
    if not torch.cuda.is_available():
        return {"skipped": ...}
    model = copy.deepcopy(model).cuda()
    X, target = X.cuda(), target.cuda()
```

```
    optimizer = torch.optim.SGD(model.parameters(), lr=...)
    scaler = torch.amp.GradScaler("cuda")
    optimizer.zero_grad(set_to_none=True)
    with torch.autocast(device_type="cuda", dtype=torch.float16):
        prediction = model(X)
        loss = torch.nn.functional.mse_loss(prediction, target)
    scaler.scale(loss).backward()
    ...  # inspect, unscale, verify, and optionally step
    return {
        "prediction_dtype": prediction.dtype,
        "loss_dtype": loss.dtype,
        "finite_loss": ...,
        "finite_gradients": ...,
    }

def main():
    print(E16(torch.nn.Linear(..., ...), torch..., torch...))

if __name__ == "__main__":
    main()
```

10.6.18 Exercise 17 - Loss Curve Logging

Part 1 - Why this matters

A loss curve reveals whether training is improving, oscillating, diverging, or reaching a plateau. Reliable logging records comparable epoch-level values without retaining autograd graphs or allowing unequal batch sizes to bias the result.

The curve is diagnostic evidence, not proof of generalization. Training and validation losses should be stored separately with configuration and epoch identifiers so the source of each point remains clear.

Part 2 - Technical concept and behavior

For epoch e:

$$\mathcal{L}_e = \frac{\sum_k S_{e,k}}{\sum_k n_{e,k}}$$

where $S_{e,k}$ is squared-error sum and $n_{e,k}$ is compared-element count. Logging a detached Python number prevents the history from retaining computation graphs.

Part 3 - Hints and Helper

Accumulate sum-reduced losses using `.item()` and element counts. Append the final mean once per epoch. Record phase, epoch, learning rate, and timestamp or run ID in a structured dictionary.

Part 4 - Python code skeleton

```
import torch

def evaluate_epoch_loss(model, loader, device):
    total_sum = 0.0
    total_elements = 0
    model.eval()
    with torch.inference_mode():
        for X, target in loader:
            X, target = X.to(device), target.to(device)
            prediction = model(X)
            total_sum += torch.nn.functional.mse_loss(
                prediction, target, reduction="sum"
            ).item()
            total_elements += prediction.numel()
    return total_sum / ...

def E17(model, loaders, epochs, device):
    history = []
    for epoch in range(epochs):
        ...  # perform or receive the training epoch
        train_loss = evaluate_epoch_loss(...)
        validation_loss = evaluate_epoch_loss(...)
        history.append({...})
    return history

def main():
    print(E17(...))

if __name__ == "__main__":
    main()
```

10.6.19 Exercise 18 - Loss Exploding Case

Part 1 - Why this matters

An exploding loss may result from excessive learning rate, large unscaled features, extreme initialization, corrupted targets, or repeated unstable updates. Detecting the first non-finite or rapidly increasing value is more useful than observing that training eventually crashed.

The exercise uses bounded synthetic conditions and stops before applying an unsafe update. It records predictions, residuals, loss, and gradient norms so the cause can be separated from the final symptom.

Part 2 - Technical concept and behavior

A simple growth indicator is:

$$r_t = \frac{\mathcal{L}_t}{\max(\mathcal{L}_{t-1}, \varepsilon)}$$

A policy may flag loss when it becomes non-finite or exceeds a chosen growth ratio for a defined number of steps. Thresholds must be tied to the experiment scale.

Part 3 - Hints and Helper

Create separate scenarios for large input scale, large target scale, and large parameter scale. Check `torch.isfinite()` before backward and again on gradients. Skip the optimizer step when the safety condition fails.

Part 4 - Python code skeleton

```
import torch

def E18(model, X, target, optimizer, previous_loss,
        growth_threshold):
    optimizer.zero_grad(set_to_none=True)
    prediction = model(X)
    loss = torch.nn.functional.mse_loss(prediction, target)
    report = {
        "finite_prediction": ...,
        "finite_loss": ...,
        "loss": ...,
    }
    if not torch.isfinite(loss):
        report["action"] = "stop"
        return report
    loss.backward()
    gradient_norm = ...
    growth_ratio = ...
    report.update({
        "gradient_norm": gradient_norm,
        "growth_ratio": growth_ratio,
        "safe_to_step": ...,
    })
    if report["safe_to_step"]:
        optimizer.step()
    return report

def main():
    print(E18(...))

if __name__ == "__main__":
    main()
```

10.6.20 Exercise 19 - Loss Failure Debugging

Part 1 - Why this matters

Loss failures often begin before the loss function: wrong target shape, integer dtype, device mismatch, NaN values, detached predictions, or an incorrect reduction can all produce errors

or misleading training behavior. A structured diagnostic should examine the contract before calling backward.

The final exercise combines value checks with graph checks. A finite scalar loss is not sufficient if it has no path to trainable parameters, and a connected graph is not useful if gradients become non-finite.

Part 2 - Technical concept and behavior

Required invariants include:

$$\text{shape}(\hat{\mathbf{Y}}) = \text{shape}(\mathbf{Y}), \quad d_{\hat{Y}} = d_Y, \quad \hat{\mathbf{Y}}, \mathbf{Y}, \mathcal{L} \text{ finite}$$

For training, the reduced loss should be scalar and connected to parameters expected to learn.

Part 3 - Hints and Helper

Validate shape, floating dtype, device, finite values, reduction, and gradient connection in sequence. Catch expected exceptions at the outer test boundary and preserve their type and message. Inject one failure at a time.

Part 4 - Python code skeleton

```
import torch

def diagnose_loss(model, prediction, target, reduction):
    checks = {
        "shape_match": ...,
        "dtype_valid": ...,
        "device_match": ...,
        "finite_prediction": ...,
        "finite_target": ...,
        "reduction_valid": reduction in {...},
    }
    if not all(checks.values()):
        return {"status": "invalid_input", "checks": checks}
    try:
        loss = torch.nn.functional.mse_loss(
            prediction, target, reduction=reduction
        )
        checks["scalar_for_backward"] = loss.ndim == 0
        checks["finite_loss"] = bool(torch.isfinite(loss).all())
        if checks["scalar_for_backward"] and checks["finite_loss"]:
            loss.backward()
            checks["finite_gradients"] = ...
        return {"status": ..., "checks": checks}
    except (RuntimeError, ValueError, TypeError) as error:
        return {
            "status": "failed",
            "checks": checks,
```

```
            "error_type": type(error).__name__,
            "message": str(error),
        }

def E19(model, X, target, reduction):
    prediction = model(X)
    return diagnose_loss(model, prediction, target, reduction)

def main():
    print(E19(torch.nn.Linear(..., ...), torch..., torch..., ...))

if __name__ == "__main__":
    main()
```

10.7 LR05 - Parameter Adjustment and Gradient

Models and Training

10.7.1 Exercise 00 - Backward on Scalar Loss

Part 1 - Why this matters

Backward propagation begins from a loss and calculates how every connected trainable parameter contributed to it. A scalar loss provides the ordinary starting point for reverse-mode differentiation in a training step.

Calling backward is a state-changing operation because gradients are stored on leaf parameters and accumulate with existing values. The reader must inspect gradient state before and after the call rather than treating backward as a function that merely returns a result.

Part 2 - Mathematical form

For scalar loss $\mathcal{L}$ and parameters θ:

$$\mathcal{L}.\texttt{backward()} \Longrightarrow \theta.\texttt{grad} \leftarrow \theta.\texttt{grad} + \nabla_\theta \mathcal{L}$$

The computational graph links the model output to the loss and then to weight and bias parameters.

Part 3 - Hints and Helper

Clear gradients to `None`, calculate prediction and mean loss, and verify the loss has zero dimensions. Record `grad_fn` before backward and parameter gradient metadata afterward.

Part 4 - Python code skeleton

```
import torch

def E00(model, X, target):
    model.zero_grad(set_to_none=True)
    prediction = model(X)
```

```
    loss = torch.nn.functional.mse_loss(prediction, target)
    if loss.ndim != 0:
        raise ValueError(...)
    before = {name: p.grad for name, p in model.named_parameters()}
    graph_function = loss.grad_fn
    loss.backward()
    after = {
        name: p.grad.detach().clone()
        for name, p in model.named_parameters()
    }
    return loss.item(), graph_function, before, after

def main():
    print(E00(torch.nn.Linear(..., ...), torch..., torch...))

if __name__ == "__main__":
    main()
```

10.7.2 Exercise 01 - Inspect Weight Gradients

Part 1 - Why this matters

The weight gradient connects feature values and prediction residuals to parameter adjustment. Its sign describes the local direction of change, while its magnitude reflects input scale, error scale, and loss reduction.

Inspecting only one gradient norm can hide shape or element-level problems. A complete report compares the autograd tensor with an analytical reference for a small batch and checks finiteness before any update.

Part 2 - Mathematical form

For $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b$ and mean squared error over N scalar outputs:

$$\nabla_{\mathbf{w}}\mathcal{L} = \frac{2}{N}\mathbf{X}^{\mathsf{T}}(\hat{\mathbf{y}} - \mathbf{y})$$

The gradient shape must equal the weight shape, accounting for the orientation used by `nn.Linear`.

Part 3 - Hints and Helper

Use a one-output linear layer and a small matrix. Call backward, calculate the analytical gradient independently, transpose only according to stored weight orientation, and compare using `torch.allclose()`.

Part 4 - Python code skeleton

```
import torch

def E01(model, X, target):
    model.zero_grad(set_to_none=True)
```

```
    prediction = model(X)
    residual = prediction - target
    loss = residual.square().mean()
    loss.backward()
    automatic = model.weight.grad.detach().clone()
    analytical = ...
    return {
        "automatic": automatic,
        "analytical": analytical,
        "shape_valid": ...,
        "finite": ...,
        "matches": torch.allclose(..., ...),
    }

def main():
    print(E01(torch.nn.Linear(..., 1), torch..., torch...))

if __name__ == "__main__":
    main()
```

10.7.3 Exercise 02 - Inspect Bias Gradients

Part 1 - Why this matters

The bias gradient aggregates error signals across the batch because one bias value affects every sample for its output. This makes it structurally different from a sample-specific adjustment.

A wrong target shape or unintended broadcasting can multiply the number of residual contributions and produce an unexpected bias gradient. Comparing with the analytical sum is therefore a sensitive shape check.

Part 2 - Mathematical form

For scalar-output MSE:

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{2}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)$$

For multiple outputs, this sum is calculated independently for each output column, producing a gradient with the bias shape.

Part 3 - Hints and Helper

Use exact prediction-target shape equality. Retain the residual before backward and calculate its reduction over the batch dimension. Compare values, shape, dtype, and device with `bias.grad`.

Part 4 - Python code skeleton

```
import torch
```

```
def E02(model, X, target):
    model.zero_grad(set_to_none=True)
    prediction = model(X)
    if prediction.shape != target.shape:
        raise ValueError(...)
    residual = prediction - target
    loss = residual.square().mean()
    loss.backward()
    automatic = model.bias.grad.detach().clone()
    analytical = ...
    return automatic, analytical, torch.allclose(..., ...)

def main():
    print(E02(torch.nn.Linear(..., ...), torch..., torch...))

if __name__ == "__main__":
    main()
```

10.7.4 Exercise 03 - Zero Grad Correctly

Part 1 - Why this matters

PyTorch accumulates parameter gradients, so each independent training step normally clears old gradients before backward. Forgetting this changes the update into a sum of current and previous signals.

Setting gradients to `None` and filling existing tensors with zero are not identical states. The default `set_to_none=True` can reduce memory use and reveals which parameters received no gradient in the next backward pass.

Part 2 - Technical concept and behavior

Before an independent step, apply:

$$g_\theta \leftarrow \texttt{None} \quad \text{or} \quad g_\theta \leftarrow 0$$

according to policy. With `None`, an optimizer may skip a parameter that receives no new gradient; a zero tensor represents an explicitly present zero gradient.

Part 3 - Hints and Helper

Create gradients with one backward call, inspect them, call `optimizer.zero_grad(set_to_none=True)` and inspect again. Repeat with `False`. Do not assume both reports should be identical.

Part 4 - Python code skeleton

```
import torch

def gradient_states(model):
    return {
        name: ("none" if p.grad is None else p.grad.detach().clone())
```

```
        for name, p in model.named_parameters()
    }

def E03(model, optimizer, X, target):
    loss = torch.nn.functional.mse_loss(model(X), target)
    loss.backward()
    after_backward = gradient_states(model)
    optimizer.zero_grad(set_to_none=True)
    after_none = gradient_states(model)
    ...  # recreate gradients
    optimizer.zero_grad(set_to_none=False)
    after_zero = gradient_states(model)
    return after_backward, after_none, after_zero

def main():
    model = torch.nn.Linear(..., ...)
    optimizer = torch.optim.SGD(model.parameters(), lr=...)
    print(E03(model, optimizer, torch..., torch...))

if __name__ == "__main__":
    main()
```

10.7.5 Exercise 04 - Manual Gradient Update

Part 1 - Why this matters

A manual gradient update exposes the exact operation used by basic gradient descent. The reader can verify that parameters move opposite the local gradient and that the observed change equals learning rate times gradient.

The update should not be recorded in the autograd graph and should not use the unsafe `.data` shortcut. It belongs after backward and inside a no-gradient context.

Part 2 - Mathematical form

For parameter θ_t:

$$\theta_{t+1} = \theta_t - \eta g_t, \qquad g_t = \nabla_\theta \mathcal{L}_t$$

The observed difference should satisfy $\theta_{t+1} - \theta_t = -\eta g_t$.

Part 3 - Hints and Helper

Clone parameter values and gradients before updating. Apply the update under `torch.no_grad()`, then compare observed and expected changes for every named parameter. Clear gradients before starting another step.

Part 4 - Python code skeleton

```
import torch
```

```
def E04(model, X, target, learning_rate):
    model.zero_grad(set_to_none=True)
    loss = torch.nn.functional.mse_loss(model(X), target)
    loss.backward()
    before = {name: p.detach().clone()
              for name, p in model.named_parameters()}
    gradients = {name: p.grad.detach().clone()
                 for name, p in model.named_parameters()}
    with torch.no_grad():
        for parameter in model.parameters():
            parameter -= ...
    checks = {
        name: torch.allclose(
            parameter.detach() - before[name],
            -learning_rate * gradients[name],
        )
        for name, parameter in model.named_parameters()
    }
    return loss.item(), checks

def main():
    print(E04(torch.nn.Linear(..., ...), torch..., torch..., ...))

if __name__ == "__main__":
    main()
```

10.7.6 Exercise 05 - Learning Rate Sensitivity

Part 1 - Why this matters

The learning rate converts a gradient into a parameter step. A very small value may produce negligible progress, while a very large value can overshoot the minimum and increase loss.

A fair sensitivity test begins from identical model state and uses identical data. Comparing separately initialized models would mix learning-rate effects with initialization variation.

Part 2 - Mathematical form

For candidate rates η_j:

$$\theta'_j = \theta - \eta_j \nabla_\theta \mathcal{L}$$

A one-step experiment records parameter-change norm and freshly recomputed post-update loss for each candidate. The old loss tensor does not represent updated parameters.

Part 3 - Hints and Helper

Deep-copy one model for every rate. Compute one gradient and update per copy, then perform a new forward pass for post-step loss. Include zero, small, moderate, and deliberately large but bounded rates.

Part 4 - Python code skeleton

```
import copy
import torch

def one_rate(model, X, target, learning_rate):
    candidate = copy.deepcopy(model)
    before = [p.detach().clone() for p in candidate.parameters()]
    loss_before = torch.nn.functional.mse_loss(candidate(X), target)
    loss_before.backward()
    with torch.no_grad():
        for parameter in candidate.parameters():
            parameter -= ...
        loss_after = torch.nn.functional.mse_loss(candidate(X), target)
    change_norm = ...
    return {...}

def E05(model, X, target, learning_rates):
    return [one_rate(model, X, target, rate)
            for rate in learning_rates]

def main():
    print(E05(torch.nn.Linear(..., ...), torch..., torch..., [...]))

if __name__ == "__main__":
    main()
```

10.7.7 Exercise 06 - Gradient Clipping

Part 1 - Why this matters

Gradient clipping limits update magnitude when gradient norms become excessive. It can protect a training step from a transient spike, but it does not fix the data scale, learning rate, or numerical error that caused repeated explosions.

Norm clipping rescales all participating gradients together when their combined norm exceeds a threshold. The reader should measure norms before and after clipping and confirm that direction is preserved when scaling occurs.

Part 2 - Mathematical form

For combined gradient vector g and limit c:

$$g' = g \min\left(1, \frac{c}{\|g\|_2}\right)$$

PyTorch's `clip_grad_norm_()` modifies gradients in place and returns the total norm measured before clipping.

Part 3 - Hints and Helper

Calculate loss and backward first. Call `torch.nn.utils.clip_grad_norm_()` before optimizer step, use `error_if_nonfinite=True`, and compute the post-clip norm independently.

Part 4 - Python code skeleton

```
import torch

def total_gradient_norm(model):
    gradients = [p.grad for p in model.parameters() if p.grad is not None]
    return ...

def E06(model, X, target, max_norm):
    model.zero_grad(set_to_none=True)
    loss = torch.nn.functional.mse_loss(model(X), target)
    loss.backward()
    measured_before = torch.nn.utils.clip_grad_norm_(
        model.parameters(),
        max_norm=...,
        error_if_nonfinite=True,
    )
    measured_after = total_gradient_norm(model)
    return measured_before, measured_after

def main():
    print(E06(torch.nn.Linear(..., ...), torch..., torch..., ...))

if __name__ == "__main__":
    main()
```

10.7.8 Exercise 07 - Gradient Accumulation

Part 1 - Why this matters

Gradient accumulation combines several micro-batches before one optimizer step, approximating or reproducing a larger effective batch when memory is limited. It differs from accidentally forgetting to clear gradients because the accumulation window and scaling are deliberate.

Simply averaging every micro-batch loss equally is incorrect when micro-batches contain different element counts. Exact equivalence to one mean-reduced large batch requires weighting each squared-error sum by the total element count across the window.

Part 2 - Mathematical form

For micro-batches with sums S_k and counts n_k:

$$\mathcal{L}_{\text{window}} = \frac{\sum_k S_k}{\sum_k n_k}$$

Backward may be applied to each $S_k / \sum_j n_j$, allowing gradients to accumulate to the same result as one concatenated batch.

Part 3 - Hints and Helper

Know the total valid element count for the accumulation window. Clear once before it, backpropagate correctly scaled batch sums, and step once afterward. Compare gradients with a single concatenated reference batch.

Part 4 - Python code skeleton

```
import copy
import torch

def E07(model, micro_batches):
    accumulated_model = copy.deepcopy(model)
    reference_model = copy.deepcopy(model)
    total_elements = sum(target.numel() for _, target in micro_batches)
    accumulated_model.zero_grad(set_to_none=True)
    for X, target in micro_batches:
        batch_sum = torch.nn.functional.mse_loss(
            accumulated_model(X), target, reduction="sum"
        )
        (batch_sum / ...).backward()
    reference_X = torch.cat([...], dim=0)
    reference_target = torch.cat([...], dim=0)
    reference_loss = torch.nn.functional.mse_loss(
        reference_model(reference_X), reference_target
    )
    reference_loss.backward()
    return ...  # compare every gradient

def main():
    print(E07(torch.nn.Linear(..., ...), ...))

if __name__ == "__main__":
    main()
```

10.7.9 Exercise 08 - Gradient Noise Injection

Part 1 - Why this matters

Adding controlled random noise to gradients is an experimental way to study update robustness and stochastic behavior. It can perturb optimization away from an exact deterministic path, but it should never be introduced without a reproducible scale and a baseline comparison.

Noise injection changes the optimizer input rather than the loss itself. The original gradients should be preserved for measurement, and noise must be added only to trainable parameters with existing finite gradients.

Part 2 - Mathematical form

For gradient g:

$$g' = g + \epsilon, \qquad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

The experiment records signal norm, noise norm, and their ratio. Zero σ should reproduce the original gradient exactly.

Part 3 - Hints and Helper

Use a seeded generator compatible with the gradient device. Clone original gradients, sample matching noise tensors, and add them inside a no-gradient context. Do not inject noise before AMP gradients have been unscaled.

Part 4 - Python code skeleton

```
import torch

def E08(model, X, target, noise_std, seed):
    model.zero_grad(set_to_none=True)
    loss = torch.nn.functional.mse_loss(model(X), target)
    loss.backward()
    generator = torch.Generator(device=...).manual_seed(...)
    report = {}
    with torch.no_grad():
        for name, parameter in model.named_parameters():
            if parameter.grad is None:
                continue
            original = parameter.grad.clone()
            noise = torch.randn(
                original.shape,
                device=original.device,
                dtype=original.dtype,
                generator=generator,
            ) * ...
            parameter.grad.add_(noise)
            report[name] = ...
    return report

def main():
    print(E08(torch.nn.Linear(..., ...), torch..., torch..., ..., ...))

if __name__ == "__main__":
    main()
```

10.7.10 Exercise 09 - Gradient Check via Finite Difference

Part 1 - Why this matters

Finite differences estimate a derivative from nearby loss values without using autograd. Comparing this numerical estimate with the automatic gradient is a powerful way to detect wrong equations, detached paths, or custom differentiation errors.

The perturbation size must balance truncation error and floating-point cancellation. Double precision and central differences usually provide a clearer reference for this small diagnostic experiment.

Part 2 - Mathematical form

For parameter element θ_j and step h:

$$\frac{\partial \mathcal{L}}{\partial \theta_j} \approx \frac{\mathcal{L}(\theta_j + h) - \mathcal{L}(\theta_j - h)}{2h}$$

Every perturbed value must be restored before moving to the next element.

Part 3 - Hints and Helper

Use a tiny double-precision model. Compute autograd gradients once, then perturb selected parameter elements under `torch.no_grad()`. Restore the original value exactly and report absolute and relative errors.

Part 4 - Python code skeleton

```
import torch

def loss_value(model, X, target):
    return torch.nn.functional.mse_loss(model(X), target)

def E09(model, X, target, step):
    model = model.double()
    X, target = X.double(), target.double()
    model.zero_grad(set_to_none=True)
    loss_value(model, X, target).backward()
    automatic = model.weight.grad.detach().clone()
    numerical = torch.zeros_like(automatic)
    for index in ...:
        original = model.weight[index].item()
        with torch.no_grad():
            model.weight[index] = original + step
        plus = loss_value(model, X, target).item()
        with torch.no_grad():
            model.weight[index] = original - step
        minus = loss_value(model, X, target).item()
        with torch.no_grad():
            model.weight[index] = original
        numerical[index] = ...
    return automatic, numerical, ...

def main():
    print(E09(torch.nn.Linear(..., ...), torch..., torch..., ...))

if __name__ == "__main__":
    main()
```

10.7.11 Exercise 10 - Optimizer Step vs Manual Step

Part 1 - Why this matters

A basic SGD optimizer step should match the manual rule when momentum, weight decay, dampening, and other modifiers are disabled. This comparison verifies understanding of what the optimizer automates.

Both paths must begin from identical parameters and use gradients from identical data. Any extra optimizer option changes the expected equation and invalidates the simple parity test.

Part 2 - Mathematical form

For plain SGD:

$$\theta' = \theta - \eta \nabla_\theta \mathcal{L}$$

The manual model and optimizer model should have numerically equal parameters after one step within floating-point tolerance.

Part 3 - Hints and Helper

Deep-copy one base model. Calculate gradients independently for each copy. Update one inside `torch.no_grad()` and update the other with `torch.optim.SGD` configured with zero momentum and zero weight decay.

Part 4 - Python code skeleton

```
import copy
import torch

def E10(model, X, target, learning_rate):
    manual_model = copy.deepcopy(model)
    optimizer_model = copy.deepcopy(model)
    ...  # calculate matching gradients
    with torch.no_grad():
        for parameter in manual_model.parameters():
            parameter -= ...
    optimizer = torch.optim.SGD(
        optimizer_model.parameters(),
        lr=learning_rate,
        momentum=0.0,
        weight_decay=0.0,
    )
    optimizer.step()
    comparisons = ...
    return comparisons

def main():
    print(E10(torch.nn.Linear(..., ...), torch..., torch..., ...))
```

```
if __name__ == "__main__":
    main()
```

10.7.12 Exercise 11 - CUDA Gradient Sync Check

Part 1 - Why this matters

Equivalent CPU and CUDA computations should produce numerically close gradients when state, data, dtype, and loss definition match. This test can expose incomplete state copying or unintended device-specific settings.

CUDA execution is asynchronous, so synchronization is required before timing and before assuming a benchmarked backward pass has completed. Numerical parity still uses tolerances rather than bitwise equality.

Part 2 - Technical concept and behavior

For corresponding gradients:

$$D_{\max} = \max_j |g_{\mathrm{CPU},j} - g_{\mathrm{CUDA},j}|$$

Every named parameter should be compared after moving CUDA gradients to CPU. Missing gradients are reported separately.

Part 3 - Hints and Helper

Deep-copy one model to each device, use identical input values, calculate the same scalar loss, and call backward. Synchronize CUDA, then compare gradients by parameter name with absolute and relative tolerances.

Part 4 - Python code skeleton

```
import copy
import torch

def E11(model, X, target, atol, rtol):
    if not torch.cuda.is_available():
        return {"skipped": ...}
    cpu_model = copy.deepcopy(model).cpu()
    gpu_model = copy.deepcopy(model).cuda()
    cpu_loss = torch.nn.functional.mse_loss(
        cpu_model(X.cpu()), target.cpu()
    )
    gpu_loss = torch.nn.functional.mse_loss(
        gpu_model(X.cuda()), target.cuda()
    )
    cpu_loss.backward()
    gpu_loss.backward()
    torch.cuda.synchronize()
    report = {}
    for (cpu_name, cpu_parameter), (gpu_name, gpu_parameter) in zip(
```

```
        cpu_model.named_parameters(), gpu_model.named_parameters()
    ):
        ...  # compare names, existence, values, and maximum difference
    return report

def main():
    print(E11(torch.nn.Linear(..., ...), torch..., torch..., ..., ...))

if __name__ == "__main__":
    main()
```

10.7.13 Exercise 12 - Frozen Parameter Gradient Test

Part 1 - Why this matters

A frozen parameter remains part of the forward computation but should not receive a stored gradient or update. This distinction supports fine-tuning and controlled experiments.

Testing only `requires_grad` is insufficient. The reader should verify the parameter value stays unchanged across backward and optimizer step while an unfrozen parameter receives and applies a gradient.

Part 2 - Technical concept and behavior

For frozen weight W_f:

$$W_f.\texttt{requires_grad} = \text{False}, \qquad W_f.\texttt{grad} = \texttt{None}, \qquad W_f' = W_f$$

The bias can remain trainable and change normally.

Part 3 - Hints and Helper

Freeze the weight before constructing the optimizer, pass only trainable parameters, and clone weight and bias values. Perform one step, then inspect gradient states and value changes separately.

Part 4 - Python code skeleton

```
import torch

def E12(model, X, target, learning_rate):
    model.weight.requires_grad_(False)
    optimizer = torch.optim.SGD(
        (p for p in model.parameters() if p.requires_grad),
        lr=learning_rate,
    )
    weight_before = model.weight.detach().clone()
    bias_before = model.bias.detach().clone()
    optimizer.zero_grad(set_to_none=True)
    loss = torch.nn.functional.mse_loss(model(X), target)
    loss.backward()
```

```
    optimizer.step()
    return {
        "weight_grad": model.weight.grad,
        "bias_grad_exists": ...,
        "weight_unchanged": ...,
        "bias_changed": ...,
    }

def main():
    print(E12(torch.nn.Linear(..., ...), torch..., torch..., ...))

if __name__ == "__main__":
    main()
```

10.7.14 Exercise 13 - Gradient Explosion Scenario

Part 1 - Why this matters

Exploding gradients can produce enormous parameter updates, non-finite state, and rapid divergence. In linear regression they may arise from large feature scale, target scale, parameter magnitude, or learning rate.

The exercise should detect unsafe gradients before stepping. Clipping may protect one update, but the report must retain evidence about the underlying scale problem rather than hiding it.

Part 2 - Technical concept and behavior

Track total norm:

$$G_t = \left(\sum_{\theta} \|\nabla_\theta \mathcal{L}_t\|_2^2 \right)^{1/2}$$

A scenario is flagged when G_t is non-finite or exceeds a documented threshold relative to the experiment.

Part 3 - Hints and Helper

Generate separate bounded cases for large inputs and large residuals. Check loss and gradients with `torch.isfinite()`, record the norm, and skip optimizer step when unsafe. Optionally compare the clipped norm without claiming the cause is solved.

Part 4 - Python code skeleton

```
import torch

def E13(model, X, target, threshold):
    model.zero_grad(set_to_none=True)
    loss = torch.nn.functional.mse_loss(model(X), target)
    report = {"finite_loss": bool(torch.isfinite(loss))}
    if not torch.isfinite(loss):
```

```
        return report
    loss.backward()
    gradients = [p.grad for p in model.parameters() if p.grad is not None]
    total_norm = ...
    report.update({
        "total_norm": total_norm,
        "finite_gradients": ...,
        "exploding": ...,
        "safe_to_step": ...,
    })
    return report

def main():
    print(E13(torch.nn.Linear(..., ...), torch..., torch..., ...))

if __name__ == "__main__":
    main()
```

10.7.15 Exercise 14 - Gradient Vanishing Scenario

Part 1 - Why this matters

Very small gradients can make parameter changes indistinguishable from numerical noise or too slow to improve within a practical number of steps. In a linear model, small feature values, small residuals, loss scaling, or a perfect fit can all produce small gradients.

A small gradient is not automatically a defect. Near an optimum it is expected. Diagnosis must compare gradient magnitude with loss, parameter scale, and the observed effect of one update.

Part 2 - Mathematical form

For linear MSE:

$$\nabla_w \mathcal{L} = \frac{2}{N} X^\top (\hat{y} - y)$$

Scaling X or residuals toward zero reduces the gradient. An update may satisfy:

$$\|\eta g\| \ll \|\theta\|$$

and create negligible relative change.

Part 3 - Hints and Helper

Construct a perfect-fit case and a non-perfect case with tiny input scale. Report loss, gradient norm, proposed update norm, and relative update size. Distinguish expected convergence from a scale-induced optimization problem.

Part 4 - Python code skeleton

```
import torch

def E14(model, X, target, learning_rate, threshold):
    model.zero_grad(set_to_none=True)
    loss = torch.nn.functional.mse_loss(model(X), target)
    loss.backward()
    report = {}
    for name, parameter in model.named_parameters():
        gradient = parameter.grad
        update_norm = learning_rate * gradient.norm()
        parameter_norm = parameter.detach().norm()
        report[name] = {
            "gradient_norm": ...,
            "update_norm": ...,
            "relative_update": ...,
            "below_threshold": ...,
        }
    return loss.item(), report

def main():
    print(E14(torch.nn.Linear(..., ...), torch..., torch..., ..., ...))

if __name__ == "__main__":
    main()
```

10.7.16 Exercise 15 - Mixed Precision Backward

Part 1 - Why this matters

Float16 gradients can underflow or overflow during training. Automatic mixed precision chooses operation dtypes, while gradient scaling multiplies the loss before backward so small gradients are more likely to remain representable.

Clipping and inspection must occur after gradients are unscaled. Otherwise, measured norms and clipping thresholds refer to scaled values rather than the gradients that would update parameters.

Part 2 - Technical concept and behavior

With scale s:

$$\nabla_\theta(s\mathcal{L}) = s\nabla_\theta\mathcal{L}$$

The scaler later unscales gradients before the optimizer step. It may skip an unsafe step and adjust its scale when non-finite gradients are detected.

Part 3 - Hints and Helper

Use `torch.autocast(device_type="cuda")` and `torch.amp.GradScaler("cuda")`. Scale loss, call backward, unscale the optimizer, inspect or clip gradients, then call scaler step and update. Skip clearly when CUDA is unavailable.

Part 4 - Python code skeleton

```
import torch

def E15(model, X, target, optimizer, max_norm):
    if not torch.cuda.is_available():
        return {"skipped": ...}
    model, X, target = model.cuda(), X.cuda(), target.cuda()
    scaler = torch.amp.GradScaler("cuda")
    optimizer.zero_grad(set_to_none=True)
    with torch.autocast(device_type="cuda", dtype=torch.float16):
        prediction = model(X)
        loss = torch.nn.functional.mse_loss(prediction, target)
    scaler.scale(loss).backward()
    scaler.unscale_(optimizer)
    norm = torch.nn.utils.clip_grad_norm_(
        model.parameters(), max_norm, error_if_nonfinite=True
    )
    scaler.step(optimizer)
    scaler.update()
    return {"loss": loss.item(), "unscaled_norm": norm, "scale": ...}

def main():
    model = torch.nn.Linear(..., ...)
    optimizer = torch.optim.SGD(model.parameters(), lr=...)
    print(E15(model, torch..., torch..., optimizer, ...))

if __name__ == "__main__":
    main()
```

10.7.17 Exercise 16 - Backward Graph Retention

Part 1 - Why this matters

Autograd normally frees saved graph data after backward because a new graph is built on the next forward pass. Retaining a graph allows another backward traversal, but it increases memory lifetime and is often unnecessary in ordinary training.

Using `retain_graph=True` to silence an error can hide incorrect control flow. The reader should first determine whether repeated derivatives genuinely require the same graph or whether a fresh forward pass is the correct solution.

Part 2 - Technical concept and behavior

Two backward calls on the same graph require retention during the first call:

$$\mathcal{L}\texttt{.backward(retain_graph=True)}$$

Gradients accumulate across both traversals unless cleared. Retention preserves graph data, not a copy of parameter gradients.

Part 3 - Hints and Helper

Run one controlled repeated-backward case with retention and record the doubled or accumulated gradient. Compare with two fresh forward passes. Also capture the expected error when attempting an unsupported second backward without retention.

Part 4 - Python code skeleton

```
import torch

def E16(model, X, target):
    model.zero_grad(set_to_none=True)
    loss = torch.nn.functional.mse_loss(model(X), target)
    loss.backward(retain_graph=True)
    first = {name: p.grad.detach().clone()
             for name, p in model.named_parameters()}
    loss.backward()
    second = {name: p.grad.detach().clone()
              for name, p in model.named_parameters()}
    accumulation_checks = ...
    ...  # compare with fresh-forward behavior
    return first, second, accumulation_checks

def main():
    print(E16(torch.nn.Linear(..., ...), torch..., torch...))

if __name__ == "__main__":
    main()
```

10.7.18 Exercise 17 - Gradient Memory Profiling

Part 1 - Why this matters

Gradients require storage comparable to trainable parameter storage, and backward may need additional saved tensors and temporary workspace. Profiling helps determine whether batch size, precision, or model structure fits the available device memory.

Visible gradient tensor bytes are only a lower bound on backward memory. CUDA peak allocation captures more of the runtime footprint, while CPU profiling requires a separately chosen measurement method.

Part 2 - Technical concept and behavior

Known gradient storage is:

$$M_g = \sum_{\theta:\theta.grad\neq \texttt{None}} \text{numel}(\theta.grad)\,\text{element_size}(\theta.grad)$$

Peak allocated memory is measured around an isolated forward-backward operation after warm-up.

Part 3 - Hints and Helper

Calculate parameter and gradient bytes separately. On CUDA, reset peak memory statistics before the measured pass and synchronize afterward. Compare `set_to_none=True` with zero tensors only after controlling all other variables.

Part 4 - Python code skeleton

```
import torch

def tensor_bytes(tensor):
    return tensor.numel() * tensor.element_size()

def E17(model, X, target):
    device = next(model.parameters()).device
    model.zero_grad(set_to_none=True)
    if device.type == "cuda":
        torch.cuda.reset_peak_memory_stats(device)
    loss = torch.nn.functional.mse_loss(model(X), target)
    loss.backward()
    if device.type == "cuda":
        torch.cuda.synchronize(device)
        peak = torch.cuda.max_memory_allocated(device)
    else:
        peak = None
    gradient_bytes = sum(
        tensor_bytes(p.grad)
        for p in model.parameters()
        if p.grad is not None
    )
    return {"gradient_bytes": gradient_bytes, "peak_allocated": peak}

def main():
    print(E17(torch.nn.Linear(..., ...), torch..., torch...))

if __name__ == "__main__":
    main()
```

10.7.19 Exercise 18 - Multi-Batch Gradient Behavior

Part 1 - Why this matters

Gradients across batches can represent independent updates, an accumulated effective batch, or an accidental mixture of past data. Observing these alternatives clarifies how the location of `zero_grad()` changes training mathematics.

Batch means also create weighting issues when batch sizes differ. The reader should compare independent batch gradients, their correctly weighted combination, and the gradient of the concatenated data.

Part 2 - Mathematical form

For full mean loss across batches:

$$g_{\text{full}} = \sum_k \frac{n_k}{\sum_j n_j} g_k$$

where g_k is the gradient of the mean loss for batch k and n_k is its number of loss elements. An unweighted average is correct only when counts are equal.

Part 3 - Hints and Helper

Use model copies with identical state. Calculate each batch gradient independently, combine it with count weights, and compare against one concatenated batch. Then demonstrate accumulation without clearing as a separate state.

Part 4 - Python code skeleton

```
import copy
import torch

def gradient_for_batch(model, X, target):
    model.zero_grad(set_to_none=True)
    loss = torch.nn.functional.mse_loss(model(X), target)
    loss.backward()
    return {name: p.grad.detach().clone()
            for name, p in model.named_parameters()}

def E18(model, batches):
    individual = []
    counts = []
    for X, target in batches:
        individual.append(gradient_for_batch(copy.deepcopy(model), X, target))
        counts.append(target.numel())
    weighted = ...
    full_model = copy.deepcopy(model)
    full_gradient = gradient_for_batch(
        full_model,
        torch.cat([...], dim=0),
        torch.cat([...], dim=0),
    )
    return individual, weighted, full_gradient, ...

def main():
    print(E18(torch.nn.Linear(..., ...), ...))

if __name__ == "__main__":
    main()
```

10.7.20 Exercise 19 - Gradient Debugging Case

Part 1 - Why this matters

Gradient failures can appear as `None`, zero, non-finite, wrong-shaped, stale, excessively large, or unexpectedly accumulated values. Each symptom has different causes, so debugging requires structured evidence from the graph and training-step order.

The final exercise diagnoses the first broken boundary rather than applying random fixes. It checks parameter registration and flags before forward, graph connection after loss, gradient properties after backward, and update safety before optimizer step.

Part 2 - Technical concept and behavior

For every trainable parameter θ:

$$\theta.grad \neq \texttt{None}, \quad \text{shape}(\theta.grad) = \text{shape}(\theta), \quad \text{isfinite}(\theta.grad)$$

should normally hold after a connected valid backward pass. Magnitude thresholds remain experiment-specific.

Part 3 - Hints and Helper

Record named parameters, `requires_grad`, loss shape, `grad_fn`, gradient state, and norm. Inject detach, frozen parameters, wrong zeroing order, and non-finite data separately. Skip optimizer step when validation fails.

Part 4 - Python code skeleton

```
import torch

def inspect_gradients(model):
    report = {}
    for name, parameter in model.named_parameters():
        gradient = parameter.grad
        report[name] = {
            "requires_grad": parameter.requires_grad,
            "exists": gradient is not None,
            "shape_matches": ...,
            "finite": ...,
            "norm": ...,
        }
    return report

def E19(model, X, target, optimizer):
    report = {"before": inspect_gradients(model)}
    optimizer.zero_grad(set_to_none=True)
    try:
        prediction = model(X)
        loss = torch.nn.functional.mse_loss(prediction, target)
        report["loss_scalar"] = loss.ndim == 0
        report["loss_grad_fn"] = str(loss.grad_fn)
        loss.backward()
        report["after_backward"] = inspect_gradients(model)
        report["safe_to_step"] = ...
```

```
        if report["safe_to_step"]:
            optimizer.step()
    except (RuntimeError, ValueError, TypeError) as error:
        report["error"] = {
            "type": type(error).__name__,
            "message": str(error),
        }
    return report

def main():
    model = torch.nn.Linear(..., ...)
    optimizer = torch.optim.SGD(model.parameters(), lr=...)
    print(E19(model, torch..., torch..., optimizer))

if __name__ == "__main__":
    main()
```

10.8 LR06 - Training Iterations and Convergence

Models and Training

10.8.1 Exercise 00 - Single-Epoch Training

Part 1 - Why this matters

A single epoch processes the training dataset once and applies a sequence of parameter updates. It is the smallest complete training unit containing mode selection, gradient clearing, forward prediction, loss, backward propagation, and optimizer step.

The returned epoch loss must represent all samples rather than an unweighted average of batch means. This is especially important when the final batch is smaller.

Part 2 - Technical concept and behavior

For batch k with squared-error sum S_k and element count n_k:

$$\mathcal{L}_{\text{epoch}} = \frac{\sum_k S_k}{\sum_k n_k}$$

Every training batch follows clear, forward, loss, backward, and update in that order.

Part 3 - Hints and Helper

Call `model.train()` once before iteration. Use mean loss for backward if desired, but separately accumulate sum-reduced detached loss and element count for reporting. Verify gradient finiteness before stepping.

Part 4 - Python code skeleton

```
import torch

def E00(model, loader, optimizer, device):
    model.train()
    total_sum = 0.0
    total_elements = 0
    for X, target in loader:
        X, target = X.to(device), target.to(device)
        optimizer.zero_grad(set_to_none=True)
        prediction = model(X)
        loss = torch.nn.functional.mse_loss(prediction, target)
        loss.backward()
        ...  # verify gradients
        optimizer.step()
        total_sum += torch.nn.functional.mse_loss(
            prediction.detach(), target, reduction="sum"
        ).item()
        total_elements += prediction.numel()
    return total_sum / ...

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

10.8.2 Exercise 01 - Multi-Epoch Training Loop

Part 1 - Why this matters

Multiple epochs allow repeated parameter refinement. The loop must preserve a history of comparable measurements so improvement, instability, and overfitting can be observed rather than inferred from the final state alone.

Training and validation have different execution rules. Validation uses evaluation mode and no gradient tracking, while training creates gradients and updates parameters. Mixing these phases can corrupt results.

Part 2 - Technical concept and behavior

For epochs $e = 0, \ldots, E-1$, record:

$$H_e = (\mathcal{L}_{\text{train},e}, \mathcal{L}_{\text{validation},e}, \eta_e)$$

The history should contain detached numerical values and run metadata, not loss tensors retaining graphs.

Part 3 - Hints and Helper

Reuse a single-epoch training function and a separate evaluation function. Record losses, current learning rate, and elapsed time. Keep the test set outside all training decisions.

Part 4 - Python code skeleton

```
import time
import torch

def evaluate(model, loader, device):
    model.eval()
    ...  # calculate element-weighted mean under inference_mode

def E01(model, train_loader, validation_loader,
        optimizer, device, epochs):
    history = []
    for epoch in range(epochs):
        started = time.perf_counter()
        train_loss = ...
        validation_loss = evaluate(...)
        history.append({
            "epoch": epoch,
            "train_loss": train_loss,
            "validation_loss": validation_loss,
            "learning_rate": ...,
            "seconds": time.perf_counter() - started,
        })
    return history

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

10.8.3 Exercise 02 - Convergence Criterion Check

Part 1 - Why this matters

Training needs a defined stopping condition. Reaching the maximum epoch count is a limit, not evidence of convergence. A useful criterion observes whether meaningful improvement has stopped or gradients have become sufficiently small.

A single tiny loss change may be noise, so convergence should usually require a condition to hold for several checks. The criterion must be chosen before inspecting test performance.

Part 2 - Mathematical form

For monitored loss L_e, define improvement:

$$\Delta_e = L_{e-1} - L_e$$

A patience-based criterion may declare convergence after p consecutive epochs with $\Delta_e < \delta$, optionally combined with gradient norm below $g_{\min}$.

Part 3 - Hints and Helper

Track best or previous validation loss, minimum meaningful improvement, and a counter. Treat non-finite values as failure rather than convergence. Return a reason and epoch, not only a boolean.

Part 4 - Python code skeleton

```
import math

class ConvergenceMonitor:
    def __init__(self, min_improvement, patience, gradient_threshold=None):
        self.min_improvement = min_improvement
        self.patience = patience
        self.gradient_threshold = gradient_threshold
        self.previous = None
        self.count = 0

    def update(self, loss_value, gradient_norm):
        if not math.isfinite(loss_value):
            return {"converged": False, "reason": "non_finite"}
        improvement = ...
        ...  # update counter using loss and optional gradient condition
        return {"converged": ..., "reason": ..., "improvement": ...}

def E02(loss_history, gradient_history, monitor):
    return [monitor.update(loss, grad)
            for loss, grad in zip(loss_history, gradient_history)]

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

10.8.4 Exercise 03 - Divergence Detection

Part 1 - Why this matters

Divergence can turn a recoverable training problem into corrupted parameters if updates continue after loss or gradients become non-finite. Early detection protects checkpoints and provides evidence about learning rate, scaling, or data failures.

One increasing epoch does not prove divergence. The policy should combine non-finite checks with sustained or extreme growth relative to a stable reference.

Part 2 - Mathematical form

Define growth ratio:

$$r_e = \frac{L_e}{\max(L_{\text{best}}, \varepsilon)}$$

Training can be stopped when loss or gradients are non-finite, or when r_e exceeds a threshold for a declared number of checks.

Part 3 - Hints and Helper

Monitor training and validation losses, gradient norm, and parameter finiteness. Preserve the last known-good checkpoint before risky steps. Return severity, evidence, and recommended action without automatically altering hyperparameters.

Part 4 - Python code skeleton

```
import math
import torch

def E03(loss_history, model, gradient_norm,
        growth_threshold, consecutive_limit):
    report = {
        "finite_loss": all(math.isfinite(v) for v in loss_history),
        "finite_parameters": all(
            torch.isfinite(p).all() for p in model.parameters()
        ),
        "finite_gradient_norm": math.isfinite(gradient_norm),
    }
    best = min(loss_history)
    growth_ratios = [...]
    consecutive_growth = ...
    report["diverging"] = ...
    report["growth_ratios"] = growth_ratios
    report["action"] = ...  # continue, warn, or stop
    return report

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

10.8.5 Exercise 04 - Early Stopping

Part 1 - Why this matters

Early stopping ends training when validation performance no longer improves meaningfully. It can reduce wasted computation and limit continued fitting after the best observed generalization point.

The final epoch is not necessarily the best epoch. A correct implementation preserves a deep copy of the best model state and restores it after stopping instead of returning the most recent state.

Part 2 - Technical concept and behavior

Let best validation loss be $L^\star$. Improvement occurs when:

$$L_e < L^\star - \delta$$

Otherwise a patience counter increases. Stopping triggers when the counter reaches p, and the stored state associated with $L^\star$ is restored.

Part 3 - Hints and Helper

Clone every tensor in the best state dictionary or use a deep copy. Store best epoch and loss, reset patience on real improvement, and never use test loss for the decision.

Part 4 - Python code skeleton

```
import copy
import torch

class EarlyStopping:
    def __init__(self, patience, min_delta):
        self.patience = patience
        self.min_delta = min_delta
        self.best_loss = float("inf")
        self.best_state = None
        self.best_epoch = None
        self.counter = 0

    def update(self, model, validation_loss, epoch):
        if validation_loss < self.best_loss - self.min_delta:
            self.best_loss = ...
            self.best_state = copy.deepcopy(model.state_dict())
            self.best_epoch = ...
            self.counter = 0
        else:
            self.counter += 1
        return self.counter >= self.patience

def E04(model, validation_losses, stopper):
    for epoch, loss in enumerate(validation_losses):
        if stopper.update(model, loss, epoch):
            break
        ...  # simulation may update model state
    model.load_state_dict(...)
    return model, stopper.best_epoch, stopper.best_loss

def main():
```

```
    print(E04(...))

if __name__ == "__main__":
    main()
```

10.8.6 Exercise 05 - Learning Rate Schedule

Part 1 - Why this matters

A learning-rate schedule changes update size as training progresses. Large early steps may accelerate movement, while smaller later steps can refine parameters near a minimum.

Scheduler timing is part of its definition. Most epoch-based PyTorch schedulers step after optimizer updates; calling them before the optimizer can skip the first scheduled value.

Part 2 - Technical concept and behavior

For step decay:

$$\eta_e = \eta_0 \gamma^{\lfloor e/s \rfloor}$$

where s is the step interval and γ the decay factor. The experiment records the rate actually used for each epoch.

Part 3 - Hints and Helper

Use a scheduler such as `StepLR`, call `optimizer.step()` during training, then `scheduler.step()` at the documented boundary. Record optimizer group rates and `get_last_lr()`.

Part 4 - Python code skeleton

```
import torch

def E05(model, loader, optimizer, scheduler, epochs, device):
    history = []
    for epoch in range(epochs):
        learning_rate_used = optimizer.param_groups[0]["lr"]
        ...  # train the epoch, including optimizer steps
        scheduler.step()
        history.append({
            "epoch": epoch,
            "learning_rate_used": learning_rate_used,
            "next_learning_rates": scheduler.get_last_lr(),
            "loss": ...,
        })
    return history

def main():
    model = torch.nn.Linear(..., ...)
    optimizer = torch.optim.SGD(model.parameters(), lr=...)
    scheduler = torch.optim.lr_scheduler.StepLR(
```

```
        optimizer, step_size=..., gamma=...
    )
    print(E05(model, ..., optimizer, scheduler, ..., ...))

if __name__ == "__main__":
    main()
```

10.8.7 Exercise 06 - Training on CUDA End-to-End

Part 1 - Why this matters

End-to-end CUDA training requires model, batches, targets, loss, and gradients to remain device compatible throughout every step. Moving only the model is insufficient.

The data pipeline should normally keep multi-process loader outputs on CPU and transfer batches in the training process. Pinned memory and non-blocking copies may help, but they must be benchmarked rather than assumed.

Part 2 - Technical concept and behavior

For every batch on device d:

$$d_\theta = d_X = d_y = d_{\mathcal{L}} = d_g$$

The epoch metric may be transferred as detached scalar values, while training computations remain on CUDA.

Part 3 - Hints and Helper

Check CUDA availability, move the model before constructing the optimizer, and transfer each batch. Validate devices and finite gradients. Synchronize only for accurate timing, not after every operation unnecessarily.

Part 4 - Python code skeleton

```
import time
import torch

def E06(model, loader, learning_rate):
    if not torch.cuda.is_available():
        return {"skipped": ...}
    device = torch.device("cuda")
    model = model.to(device)
    optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
    model.train()
    torch.cuda.synchronize()
    started = time.perf_counter()
    for X, target in loader:
        X = X.to(device, non_blocking=...)
        target = target.to(device, non_blocking=...)
        optimizer.zero_grad(set_to_none=True)
```

```
        loss = torch.nn.functional.mse_loss(model(X), target)
        loss.backward()
        ...  # verify and update
    torch.cuda.synchronize()
    return {"seconds": time.perf_counter() - started, "loss": ...}

def main():
    print(E06(torch.nn.Linear(..., ...), ..., ...))

if __name__ == "__main__":
    main()
```

10.8.8 Exercise 07 - CPU vs GPU Convergence

Part 1 - Why this matters

CPU and GPU training should follow similar convergence behavior when they begin from identical state and consume the same batches. Exact trajectories may differ slightly because floating-point kernels are not guaranteed to be bitwise identical.

A fair comparison controls initialization, sample order, dtype, optimizer settings, and number of updates. Timing and convergence quality are separate outputs.

Part 2 - Technical concept and behavior

For loss histories L_e^C and L_e^G, compare:

$$D_e = |L_e^C - L_e^G|$$

along with final parameter and prediction differences. Tolerances should be declared before observing the result.

Part 3 - Hints and Helper

Deep-copy one initial model, precompute batch index order, and feed equivalent tensors to each device. Synchronize CUDA timing. Report both loss curves and final numerical differences.

Part 4 - Python code skeleton

```
import copy
import torch

def train_with_fixed_batches(model, batches, device, epochs, lr):
    model = model.to(device)
    optimizer = torch.optim.SGD(model.parameters(), lr=lr)
    history = []
    for epoch in range(epochs):
        ...  # use identical batch order
        history.append(...)
    return model, history
```

```
def E07(base_model, batches, epochs, lr):
    if not torch.cuda.is_available():
        return {"skipped": ...}
    cpu_model, cpu_history = train_with_fixed_batches(
        copy.deepcopy(base_model), batches, torch.device("cpu"), epochs, lr
    )
    gpu_model, gpu_history = train_with_fixed_batches(
        copy.deepcopy(base_model), batches, torch.device("cuda"), epochs, lr
    )
    return ...  # compare histories, parameters, and predictions

def main():
    print(E07(torch.nn.Linear(..., ...), ..., ..., ...))

if __name__ == "__main__":
    main()
```

10.8.9 Exercise 08 - Batch Size Impact

Part 1 - Why this matters

Batch size changes gradient variance, number of optimizer updates per epoch, memory use, and hardware utilization. Comparing batch sizes without controlling the training budget can confuse these effects.

The reader should decide whether experiments use equal epochs, equal optimizer steps, or equal processed samples. Each answers a different question and must be recorded.

Part 2 - Technical concept and behavior

For dataset size N and batch size B:

$$U_{\text{epoch}} = \left\lceil \frac{N}{B} \right\rceil$$

optimizer updates occur per epoch when every batch steps once. Larger B usually reduces update count and may reduce gradient noise.

Part 3 - Hints and Helper

Start every candidate from the same state, use the same split, and log update count, samples processed, loss, time, and peak memory. Do not change learning rate silently with batch size.

Part 4 - Python code skeleton

```
import copy
import torch
from torch.utils.data import DataLoader

def run_batch_size(base_model, dataset, batch_size, epochs, lr, device):
    model = copy.deepcopy(base_model).to(device)
    loader = DataLoader(dataset, batch_size=batch_size,
```

```
                        shuffle=..., generator=...)
    optimizer = torch.optim.SGD(model.parameters(), lr=lr)
    updates = 0
    samples = 0
    ...  # train and record time, losses, updates, and samples
    return {...}

def E08(base_model, dataset, batch_sizes, epochs, lr, device):
    return [run_batch_size(base_model, dataset, size, epochs, lr, device)
            for size in batch_sizes]

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

10.8.10 Exercise 09 - Noise Robustness Test

Part 1 - Why this matters

A regression model should be evaluated under realistic feature or target noise rather than only on perfectly generated data. Robustness testing measures how performance changes as controlled noise increases.

One random draw can be misleading. Repeated seeds allow the reader to report variability and distinguish a general trend from one favorable or unfavorable sample.

Part 2 - Mathematical form

For noise level σ:

$$X' = X + \epsilon_X \quad \text{or} \quad y' = y + \epsilon_y, \qquad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

Report metric distribution across seeds for each σ, while keeping clean evaluation data as a reference.

Part 3 - Hints and Helper

Separate training noise from evaluation noise and label which is applied. Clone the base dataset and model state per run. Record mean and standard deviation of final clean and noisy losses.

Part 4 - Python code skeleton

```
import copy
import torch

def run_noise_case(base_model, clean_data, noise_std, seed, config):
    generator = torch.Generator().manual_seed(seed)
    ...  # create an independent noisy dataset
```

```
    model = copy.deepcopy(base_model)
    ...  # train under the declared noise policy
    return {
        "noise_std": noise_std,
        "seed": seed,
        "clean_loss": ...,
        "noisy_loss": ...,
    }

def E09(base_model, clean_data, noise_levels, seeds, config):
    results = []
    for level in noise_levels:
        for seed in seeds:
            results.append(run_noise_case(...))
    return ...  # aggregate without discarding raw runs

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

10.8.11 Exercise 10 - Reproducibility with Seeds

Part 1 - Why this matters

Reproducible experiments help determine whether a result changed because of code or configuration rather than a different random initialization or shuffle order. Seeds are part of the experiment record.

A seed does not guarantee identical results across PyTorch versions, platforms, or CPU and GPU. Deterministic algorithms may also reduce performance or reject unsupported operations, so the reproducibility scope must be documented.

Part 2 - Technical concept and behavior

For fixed environment and deterministic execution, repeated runs aim for:

$$H^{(1)} = H^{(2)}$$

or agreement within declared tolerance. Sources include model initialization, synthetic data, loaders, Python randomness, and external libraries.

Part 3 - Hints and Helper

Seed Python, PyTorch, and any used NumPy generators. Give `DataLoader` an explicit generator and worker-seeding policy. Optionally enable deterministic algorithms and record environment versions and device.

Part 4 - Python code skeleton

```
import random
import torch

def configure_reproducibility(seed, deterministic):
    random.seed(seed)
    torch.manual_seed(seed)
    if deterministic:
        torch.use_deterministic_algorithms(...)
        torch.backends.cudnn.benchmark = ...
    generator = torch.Generator().manual_seed(seed)
    return generator

def E10(run_training, seed, deterministic):
    generator = configure_reproducibility(seed, deterministic)
    first = run_training(generator)
    generator = configure_reproducibility(seed, deterministic)
    second = run_training(generator)
    return {
        "history_matches": ...,
        "state_matches": ...,
        "scope": ...,
    }

def main():
    print(E10(...))

if __name__ == "__main__":
    main()
```

10.8.12 Exercise 11 - Checkpoint Save per Epoch

Part 1 - Why this matters

An epoch checkpoint preserves a recoverable training boundary. Saving only model weights is enough for inference, but resuming training also requires optimizer and scheduler state, completed epoch, configuration, and random state.

Checkpoint frequency trades storage and write time against potential lost work. A partially written file should not replace the last good checkpoint, so production workflows use an atomic replacement strategy.

Part 2 - Technical concept and behavior

After completed epoch e, store:

$$C_e = (S_{\text{model}}, S_{\text{optimizer}}, S_{\text{scheduler}}, e, H, R, \Gamma)$$

where H is history, R random state, and Γ configuration metadata.

Part 3 - Hints and Helper

Save state dictionaries rather than whole model objects. Include CPU RNG state and CUDA RNG states when available. Write to a temporary path and atomically replace the intended checkpoint only after success.

Part 4 - Python code skeleton

```
import os
import torch

def build_checkpoint(model, optimizer, scheduler,
                     completed_epoch, history, config):
    return {
        "model_state": model.state_dict(),
        "optimizer_state": optimizer.state_dict(),
        "scheduler_state": None if scheduler is None else scheduler.state_dict(),
        "completed_epoch": completed_epoch,
        "history": history,
        "config": config,
        "torch_rng_state": torch.get_rng_state(),
        "cuda_rng_states": (
            torch.cuda.get_rng_state_all() if torch.cuda.is_available() else None
        ),
    }

def E11(checkpoint, final_path):
    temporary_path = ...
    torch.save(checkpoint, temporary_path)
    ...  # verify the temporary checkpoint
    os.replace(temporary_path, final_path)
    return final_path

def main():
    print(E11(...))

if __name__ == "__main__":
    main()
```

10.8.13 Exercise 12 - Resume Training from Checkpoint

Part 1 - Why this matters

Resuming should continue the same optimization process rather than merely begin from similar weights. Momentum, scheduler position, random state, history, and the next epoch index all influence the continuation.

The receiving model and optimizer must be constructed compatibly before loading state. The checkpoint should be validated before it replaces live training state.

Part 2 - Technical concept and behavior

If checkpoint records completed epoch e, continuation begins at:

$$e_{\text{start}} = e + 1$$

Restored optimizer and scheduler states ensure the next update and learning rate follow the original sequence.

Part 3 - Hints and Helper

Load with an explicit `map_location` and `weights_only=True` when the checkpoint contains supported tensor and primitive state. Validate keys and configuration, load model before optimizer state, restore scheduler and RNG states, and return the next epoch.

Part 4 - Python code skeleton

```
import torch

def E12(checkpoint_path, model, optimizer, scheduler, device):
    checkpoint = torch.load(
        checkpoint_path,
        map_location=device,
        weights_only=True,
    )
    required_keys = {...}
    if not required_keys.issubset(checkpoint):
        raise ValueError(...)
    model.load_state_dict(checkpoint["model_state"])
    optimizer.load_state_dict(checkpoint["optimizer_state"])
    if scheduler is not None and checkpoint["scheduler_state"] is not None:
        scheduler.load_state_dict(...)
    torch.set_rng_state(...)
    if device.type == "cuda" and checkpoint["cuda_rng_states"] is not None:
        torch.cuda.set_rng_state_all(...)
    next_epoch = checkpoint["completed_epoch"] + 1
    return next_epoch, checkpoint["history"], checkpoint["config"]

def main():
    print(E12(...))

if __name__ == "__main__":
    main()
```

10.8.14 Exercise 13 - Long-Run Stability Test

Part 1 - Why this matters

A training loop that succeeds for a few epochs may still leak memory, drift numerically, corrupt state, or become unstable after many updates. A long-run test observes trends under a bounded resource and time budget.

The test should stream summaries rather than retain every tensor. Periodic known-good checkpoints allow recovery and comparison if a later failure occurs.

Part 2 - Technical concept and behavior

At interval q, record:

$$M_e = (L_{\text{train}}, L_{\text{validation}}, \|g\|, \|\theta\|, \eta, M_{\text{memory}}, T)$$

Non-finite values, sustained memory growth, and severe loss growth trigger a controlled stop.

Part 3 - Hints and Helper

Define maximum epochs, wall time, memory budget, and checkpoint interval before running. Store detached scalars only, sample memory consistently, and validate parameters after each checkpoint boundary.

Part 4 - Python code skeleton

```
import time
import torch

def E13(model, train_epoch, evaluate, save_checkpoint,
        max_epochs, max_seconds, checkpoint_interval):
    started = time.perf_counter()
    history = []
    last_good = None
    for epoch in range(max_epochs):
        if time.perf_counter() - started > max_seconds:
            break
        metrics = train_epoch(model)
        validation = evaluate(model)
        record = {...}
        history.append(record)
        safe = ...
        if not safe:
            return {"status": "stopped", "history": history,
                    "last_good": last_good}
        if (epoch + 1) % checkpoint_interval == 0:
            last_good = save_checkpoint(...)
    return {"status": "completed", "history": history,
            "last_good": last_good}

def main():
    print(E13(...))

if __name__ == "__main__":
    main()
```

10.8.15 Exercise 14 - Loss Plateau Analysis

Part 1 - Why this matters

A plateau is a period of little measured improvement. It may indicate convergence, a learning rate that is too small, noisy validation estimates, insufficient model capacity, or poor feature scaling.

Declaring a plateau from two points is unreliable. The analysis needs a window, meaningful improvement threshold, and supporting signals such as gradient norm and learning rate.

Part 2 - Mathematical form

For window length w, total window improvement is:

$$\Delta_w = L_{e-w} - L_e$$

A simple plateau policy flags when $|\Delta_w| < \delta$ and variability inside the window remains within a declared range for enough checks.

Part 3 - Hints and Helper

Use validation loss rather than test loss. Report window slope or improvement, variability, current learning rate, and gradient norm. Return possible interpretations rather than automatically changing the optimizer.

Part 4 - Python code skeleton

```
import torch

def E14(loss_history, learning_rates, gradient_norms,
        window_size, min_improvement, max_variation):
    if len(loss_history) < window_size:
        return {"plateau": False, "reason": "insufficient_history"}
    window = torch.tensor(loss_history[-window_size:], dtype=torch.float64)
    improvement = ...
    variation = ...
    plateau = ...
    return {
        "plateau": plateau,
        "improvement": improvement,
        "variation": variation,
        "learning_rate": learning_rates[-1],
        "gradient_norm": gradient_norms[-1],
        "possible_causes": ...,
    }

def main():
    print(E14(...))

if __name__ == "__main__":
    main()
```

10.8.16 Exercise 15 - Throughput Benchmark

Part 1 - Why this matters

Training throughput measures how much useful data is processed per unit time. It helps compare batch sizes, devices, data-loader settings, and precision modes, but it does not measure convergence quality by itself.

A valid benchmark excludes warm-up, synchronizes asynchronous CUDA work, and reports both samples and optimizer updates. Data loading may be included or isolated depending on the stated boundary.

Part 2 - Mathematical form

For N processed samples over elapsed time T:

$$Q_{\text{samples}} = \frac{N}{T}$$

Updates per second are U/T. The report labels device, batch size, dtype, worker count, and whether transfer is included.

Part 3 - Hints and Helper

Warm up several steps, use `time.perf_counter()`, and synchronize CUDA before starting and stopping. Avoid logging inside the timed region and repeat trials to report median throughput.

Part 4 - Python code skeleton

```
import time
import torch

def benchmark_trial(train_step, loader, device, warmup_steps, measured_steps):
    iterator = iter(loader)
    for _ in range(warmup_steps):
        batch = next(iterator)
        train_step(batch)
    if device.type == "cuda":
        torch.cuda.synchronize()
    started = time.perf_counter()
    samples = 0
    updates = 0
    for _ in range(measured_steps):
        batch = next(iterator)
        train_step(batch)
        samples += ...
        updates += 1
    if device.type == "cuda":
        torch.cuda.synchronize()
    elapsed = time.perf_counter() - started
    return {"samples_per_second": ..., "updates_per_second": ...}
```

```
def E15(...):
    ...

def main():
    print(E15(...))

if __name__ == "__main__":
    main()
```

10.8.17 Exercise 16 - Gradient Drift Over Epochs

Part 1 - Why this matters

Gradient magnitude and direction change as model parameters move through the loss surface. Tracking these changes can reveal convergence, oscillation, sudden data shifts, or an unstable learning rate.

A norm alone cannot describe direction. Comparing consecutive flattened gradient vectors with cosine similarity provides a second view, but zero gradients require an explicit undefined or special policy.

Part 2 - Mathematical form

For consecutive epoch reference gradients g_{e-1} and g_e:

$$\cos(g_{e-1}, g_e) = \frac{g_{e-1}^{\mathsf{T}} g_e}{\|g_{e-1}\|_2 \|g_e\|_2}$$

Also record norm ratio and absolute difference. Reference gradients should be computed on the same fixed probe batch.

Part 3 - Hints and Helper

After each epoch, clear gradients and calculate one gradient on an unchanged probe batch without stepping. Flatten named gradients in a stable order, detach them, and compare with the previous epoch.

Part 4 - Python code skeleton

```
import torch

def probe_gradient(model, X_probe, y_probe):
    model.zero_grad(set_to_none=True)
    loss = torch.nn.functional.mse_loss(model(X_probe), y_probe)
    loss.backward()
    vectors = [
        parameter.grad.detach().reshape(-1)
        for _, parameter in model.named_parameters()
        if parameter.grad is not None
    ]
```

```
    return torch.cat(vectors)

def E16(model, train_epoch, probe_batch, epochs):
    history = []
    previous = None
    for epoch in range(epochs):
        train_epoch(model)
        current = probe_gradient(model, *probe_batch)
        similarity = ... if previous is not None else None
        history.append({...})
        previous = current.clone()
    return history

def main():
    print(E16(...))

if __name__ == "__main__":
    main()
```

10.8.18 Exercise 17 - Mixed Precision Training

Part 1 - Why this matters

Mixed precision can improve CUDA throughput and reduce memory by selecting lower precision for suitable operations while retaining safer precision for sensitive reductions. Gradient scaling helps protect float16 backward values.

The training loop must unscale gradients before clipping or inspecting them. It must also monitor skipped updates and numerical agreement rather than assuming mixed precision is stable for every scale.

Part 2 - Technical concept and behavior

For loss scale s:

$$\nabla_\theta(s\mathcal{L}) = s\nabla_\theta\mathcal{L}$$

The scaler unscales before the effective update and adjusts s according to gradient finiteness.

Part 3 - Hints and Helper

Use `torch.autocast(device_type="cuda")` and `torch.amp.GradScaler("cuda")`. Keep master parameters in their normal dtype, unscale before clipping, and record loss, scale, throughput, and finite status.

Part 4 - Python code skeleton

```
import torch

def E17(model, loader, optimizer, epochs, max_grad_norm):
    if not torch.cuda.is_available():
```

```
        return {"skipped": ...}
    model = model.cuda()
    scaler = torch.amp.GradScaler("cuda")
    history = []
    for epoch in range(epochs):
        for X, target in loader:
            X, target = X.cuda(), target.cuda()
            optimizer.zero_grad(set_to_none=True)
            with torch.autocast(device_type="cuda", dtype=torch.float16):
                prediction = model(X)
                loss = torch.nn.functional.mse_loss(prediction, target)
            scaler.scale(loss).backward()
            scaler.unscale_(optimizer)
            norm = torch.nn.utils.clip_grad_norm_(...)
            scaler.step(optimizer)
            scaler.update()
        history.append({"loss": ..., "scale": ..., "gradient_norm": ...})
    return history

def main():
    print(E17(...))

if __name__ == "__main__":
    main()
```

10.8.19 Exercise 18 - Training Failure Recovery

Part 1 - Why this matters

A recoverable training system stops when it detects unsafe state, preserves diagnostics, and restores a validated checkpoint. Recovery should not continue from non-finite parameters or retry forever without changing the failure condition.

The recovery policy must distinguish transient infrastructure errors from deterministic data or configuration failures. Restoring the same checkpoint and replaying the same bad batch indefinitely is not recovery.

Part 2 - Technical concept and behavior

Let $C^\star$ be the last validated checkpoint. On failure at step t:

$$(\theta_t, S_t) \leftarrow C^\star$$

then validate restored model, optimizer, scheduler, and next data position. A bounded retry policy decides whether to resume or terminate.

Part 3 - Hints and Helper

Save checkpoint identity and batch identifier in the report. Restore all training state, run a fixed validation probe, and enforce a maximum retry count. Require caller approval or declared policy for hyperparameter changes.

Part 4 - Python code skeleton

```
import torch

def validate_restored_state(model, probe_batch):
    ...  # finite parameters, prediction, and loss

def E18(train_step, load_checkpoint, checkpoint_path,
        model, optimizer, scheduler, batches, max_retries):
    report = []
    retries = 0
    for batch_id, batch in enumerate(batches):
        try:
            metrics = train_step(batch)
            if not metrics["safe"]:
                raise RuntimeError("unsafe training state")
        except RuntimeError as error:
            report.append({...})
            if retries >= max_retries:
                return {"status": "failed", "events": report}
            checkpoint = load_checkpoint(checkpoint_path)
            ...  # restore model, optimizer, scheduler, epoch, and RNG
            if not validate_restored_state(model, ...):
                return {"status": "invalid_checkpoint", "events": report}
            retries += 1
            ...  # follow the declared replay or skip policy
    return {"status": "completed", "events": report}

def main():
    print(E18(...))

if __name__ == "__main__":
    main()
```

10.8.20 Exercise 19 - Full Training Debug Scenario

Part 1 - Why this matters

A full training failure may originate in data, model mode, device placement, loss shape, gradient state, optimizer order, scheduler timing, checkpoint state, or validation leakage. Debugging requires evidence from the whole pipeline.

The final challenge creates a structured trace instead of printing disconnected values. Each stage records its contract, and the first unsafe stage prevents later mutation such as an optimizer step or checkpoint replacement.

Part 2 - Technical concept and behavior

The training state transition is:

$$D \to f_\theta(D) \to \mathcal{L} \to \nabla_\theta \mathcal{L} \to \theta' \to V(\theta') \to C$$

Every arrow requires shape, dtype, device, finite, and ordering invariants appropriate to that stage.

Part 3 - Hints and Helper

Record sample IDs, tensor metadata, loss, gradient norms, learning rate, parameter changes, validation metric, and checkpoint identity. Inject one failure at a time, including leakage, wrong target shape, device mismatch, stale gradients, scheduler order, and non-finite values.

Part 4 - Python code skeleton

```
import torch

def E19(model, batch, optimizer, scheduler,
        validate, checkpoint, device):
    trace = []
    try:
        X, target, sample_ids = batch
        trace.append({"stage": "data", ...})
        ...  # validate split provenance, shape, dtype, and finite values
        X, target = X.to(device), target.to(device)
        optimizer.zero_grad(set_to_none=True)
        prediction = model(X)
        trace.append({"stage": "forward", ...})
        loss = torch.nn.functional.mse_loss(prediction, target)
        trace.append({"stage": "loss", ...})
        loss.backward()
        trace.append({"stage": "backward", ...})
        safe = ...
        if not safe:
            return {"status": "stopped", "trace": trace}
        before = ...
        optimizer.step()
        scheduler.step()
        trace.append({"stage": "update", ...})
        validation = validate(model)
        ...  # save only if the complete state is valid
        return {"status": "passed", "trace": trace}
    except (RuntimeError, ValueError, TypeError) as error:
        trace.append({
            "stage": "failure",
            "type": type(error).__name__,
            "message": str(error),
        })
        return {"status": "failed", "trace": trace}

def main():
```

```
    print(E19(...))

if __name__ == "__main__":
    main()
```

10.9 LR07 - Model Evaluation and Performance

Models and Training

10.9.1 Exercise 00 - Compute RMSE Metric

Part 1 - Why this matters

Root Mean Squared Error measures the typical prediction error while penalizing large residuals more strongly than small ones. Because the square root returns the result to the target's original unit, RMSE can be interpreted alongside the values the model predicts.

A correct implementation must aggregate squared errors across every evaluated element before taking the square root. Averaging separate batch RMSE values can produce a biased result when batch sizes differ.

Part 2 - Mathematical definition

For targets y_i, predictions $\hat{y}_i$, and N evaluated elements:

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2}$$

The square root is applied once after the global mean squared error is computed.

Part 3 - Hints and Helper

Run evaluation without gradient tracking. Accumulate `reduction="sum"` squared error and the number of prediction elements, then divide and apply `torch.sqrt`. Check that prediction and target shapes match exactly.

Part 4 - Python code skeleton

```
import torch

def E00(model, loader, device):
    model.eval()
    squared_error_sum = 0.0
    element_count = 0
    with torch.inference_mode():
        for X, target in loader:
            X, target = X.to(device), target.to(device)
            prediction = model(X)
            ...
            squared_error_sum += torch.nn.functional.mse_loss(
```

```
                prediction, target, reduction="sum"
            ).item()
            element_count += prediction.numel()
    return (squared_error_sum / ...) ** 0.5

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

10.9.2 Exercise 01 - Compute MAE Metric

Part 1 - Why this matters

Mean Absolute Error reports the average magnitude of prediction errors without squaring them. It is easier to interpret than a squared loss and is less dominated by a small number of unusually large errors.

MAE and RMSE reveal different properties of the same model. Comparing both helps determine whether performance is broadly weak or whether a few severe failures are responsible for most of the error.

Part 2 - Mathematical definition

For residual $r_i = \hat{y}_i - y_i$:

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N}|r_i|$$

A dataset-level MAE requires the absolute-error sum and total element count across all batches.

Part 3 - Hints and Helper

Use `torch.nn.functional.l1_loss(..., reduction="sum")`. Do not average batch means unless all batches contain exactly the same number of evaluated elements. Return both MAE and the count used to make the result auditable.

Part 4 - Python code skeleton

```
import torch

def E01(model, loader, device):
    model.eval()
    absolute_error_sum = 0.0
    element_count = 0
    with torch.inference_mode():
        for X, target in loader:
            X, target = X.to(device), target.to(device)
            prediction = model(X)
```

```
            if prediction.shape != target.shape:
                raise ValueError(...)
            absolute_error_sum += torch.nn.functional.l1_loss(
                prediction, target, reduction="sum"
            ).item()
            element_count += ...
    return {"mae": absolute_error_sum / element_count,
            "elements": element_count}

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

10.9.3 Exercise 02 - Evaluation without Grad

Part 1 - Why this matters

Evaluation needs forward predictions but does not need gradients or a backward graph. Disabling gradient tracking reduces unnecessary graph construction and can lower memory use during validation and inference.

This optimization does not replace evaluation mode. Gradient state controls automatic differentiation, whereas `model.eval()` controls the behavior of modules whose training and evaluation rules differ.

Part 2 - Technical concept and behavior

An evaluation function should satisfy:

$$\text{gradient tracking} = \text{disabled}, \qquad \text{model.training} = \text{False}$$

Its predictions should not require gradients, and model parameter gradients should remain unchanged.

Part 3 - Hints and Helper

Save copies of existing parameter gradients before evaluation. Use `model.eval()` with `torch.inference_mode()` or `torch.no_grad()`, verify `prediction.requires_grad` is false, and confirm the stored gradients were not modified.

Part 4 - Python code skeleton

```
import torch

def E02(model, X):
    previous = [
        None if p.grad is None else p.grad.detach().clone()
        for p in model.parameters()
    ]
```

```
    model.eval()
    with torch.no_grad():
        prediction = model(X)
        tracked = prediction.requires_grad
    unchanged = []
    for parameter, old_grad in zip(model.parameters(), previous):
        ...
    return {"prediction": prediction,
            "requires_grad": tracked,
            "gradients_unchanged": all(unchanged)}

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

10.9.4 Exercise 03 - Train vs Eval Mode

Part 1 - Why this matters

PyTorch modules can behave differently during training and evaluation. Dropout changes activations stochastically during training, while batch normalization changes how its statistics are selected. A forgotten mode switch can therefore invalidate evaluation results.

A simple linear layer may produce identical outputs in both modes, so the reader must not infer that mode selection is universally irrelevant. The experiment should use a model containing a mode-sensitive component.

Part 2 - Technical concept and behavior

The state flag follows:

`model.train()` ⇒ `model.training=True`

`model.eval()` ⇒ `model.training=False`

Evaluation mode is equivalent to calling `model.train(False)`, but it does not disable gradients by itself.

Part 3 - Hints and Helper

Use a small model with dropout, keep the input fixed, and collect repeated outputs in both modes. Measure variation across repetitions. Wrap only the evaluation experiment in a no-gradient context.

Part 4 - Python code skeleton

```
import torch

def E03(model, X, repeats=20):
```

```
    model.train()
    train_outputs = [model(X).detach() for _ in range(repeats)]
    model.eval()
    with torch.inference_mode():
        eval_outputs = [model(X) for _ in range(repeats)]
    train_stack = torch.stack(train_outputs)
    eval_stack = torch.stack(eval_outputs)
    return {
        "train_variation": train_stack.std(dim=0).mean().item(),
        "eval_variation": ...,
        "mode_after_test": model.training,
    }

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

10.9.5 Exercise 04 - Overfitting Detection

Part 1 - Why this matters

Overfitting occurs when a model continues improving on training data while its performance on unseen data stops improving or becomes worse. A low training loss alone is therefore not evidence that the model has learned a useful relationship.

Detection requires comparable measurements collected over time. Training and validation metrics must use consistent reductions and fixed validation data, otherwise an apparent generalization gap may be caused by measurement changes.

Part 2 - Technical concept and behavior

At epoch t, define the generalization gap:

$$G_t = \mathcal{L}_{\text{validation},t} - \mathcal{L}_{\text{training},t}$$

A persistent increase in G_t combined with improving training loss is evidence of possible overfitting, not an automatic proof.

Part 3 - Hints and Helper

Accept training and validation histories of equal length. Compare recent windows rather than one noisy epoch. Report the best validation epoch, final gap, training trend, validation trend, and the rule that triggered the warning.

Part 4 - Python code skeleton

```
import torch

def E04(train_losses, validation_losses, window=5, tolerance=0.0):
```

```
    train = torch.as_tensor(train_losses, dtype=torch.float64)
    validation = torch.as_tensor(validation_losses, dtype=torch.float64)
    if train.shape != validation.shape or train.numel() < window + 1:
        raise ValueError(...)
    recent_train = train[-window:].mean()
    earlier_train = train[-2 * window:-window].mean()
    train_change = recent_train - earlier_train
    validation_change = ...
    gap = validation - train
    suspected = bool(
        train_change < -tolerance and validation_change > tolerance
    )
    return {"overfitting_suspected": suspected,
            "best_validation_epoch": int(validation.argmin()),
            "final_gap": gap[-1].item()}

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

10.9.6 Exercise 05 - Cross-Validation Split

Part 1 - Why this matters

A single train–validation split may give an unusually optimistic or pessimistic estimate, especially when a dataset is small. Cross-validation rotates the validation subset so every sample can contribute to evaluation.

The method is only trustworthy when information does not leak between folds. Model parameters, optimizer state, and transformations learned from data must be rebuilt using only the training portion of each fold.

Part 2 - Mathematical definition

For K disjoint folds $F_1, \ldots, F_K$, fold k uses:

$$D^{(k)}_{\text{validation}} = F_k, \qquad D^{(k)}_{\text{train}} = \bigcup_{j \neq k} F_j$$

The final estimate can be reported as the mean and standard deviation of the fold metrics.

Part 3 - Hints and Helper

Shuffle indices once with a seeded generator, then use `torch.tensor_split` so uneven sizes are handled. Create a new model and optimizer inside every fold. Fit scalers only on that fold's training indices.

Part 4 - Python code skeleton

```
import torch

def E05(dataset, folds, seed, model_factory, train_fold, evaluate_fold):
    generator = torch.Generator().manual_seed(seed)
    indices = torch.randperm(len(dataset), generator=generator)
    fold_indices = torch.tensor_split(indices, folds)
    scores = []
    for k in range(folds):
        validation_indices = fold_indices[k]
        training_indices = torch.cat([
            fold_indices[j] for j in range(folds) if j != k
        ])
        model = model_factory()
        ...  # train only on training_indices
        score = evaluate_fold(model, dataset, validation_indices)
        scores.append(score)
    values = torch.tensor(scores, dtype=torch.float64)
    return {"scores": scores, "mean": ..., "std": ...}

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

10.9.7 Exercise 06 - Performance on Noisy Data

Part 1 - Why this matters

Real measurements contain sensor error, rounding, transmission artifacts, and other disturbances. A model that performs well only on perfectly clean inputs may fail as soon as it leaves the controlled training environment.

Noise testing should change one condition at a time and repeat random experiments. Otherwise, one lucky noise sample can be mistaken for genuine robustness or one unlucky sample can exaggerate weakness.

Part 2 - Technical concept and behavior

For clean input $\mathbf{x}$ and Gaussian perturbation:

$$\tilde{\mathbf{x}} = \mathbf{x} + \sigma\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

Performance degradation at level σ can be measured relative to the clean metric.

Part 3 - Hints and Helper

Evaluate a fixed model at several noise levels and several seeded trials per level. Do not add noise to protected identifier or categorical columns. Report mean metric, standard deviation, and degradation from the clean baseline.

Part 4 - Python code skeleton

```
import torch

def E06(model, X, target, noise_levels, trials, seed):
    model.eval()
    results = {}
    with torch.inference_mode():
        clean_prediction = model(X)
        clean_rmse = ...
        for level in noise_levels:
            trial_scores = []
            for trial in range(trials):
                generator = torch.Generator(device=X.device)
                generator.manual_seed(seed + trial)
                noise = torch.randn(
                    X.shape, generator=generator,
                    device=X.device, dtype=X.dtype
                )
                prediction = model(X + level * noise)
                trial_scores.append(...)
            results[level] = {"mean_rmse": ..., "std_rmse": ...}
    return {"clean_rmse": clean_rmse, "noise_results": results}

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

10.9.8 Exercise 07 - Performance Scaling with Data Size

Part 1 - Why this matters

A learning curve reveals whether additional training data is likely to improve a model. If validation error continues falling as the training set grows, more data may be valuable; if both errors remain high and close, model capacity or features may be the stronger limitation.

A fair experiment must keep the validation set and training procedure fixed while changing training size. Randomly changing every condition at once makes the source of improvement impossible to identify.

Part 2 - Technical concept and behavior

For nested training subsets:

$$D_1 \subset D_2 \subset \cdots \subset D_m$$

record training and validation metrics as functions of $|D_j|$. Repeated seeds provide uncertainty around each point.

Part 3 - Hints and Helper

Create one seeded ordering and use prefixes so subsets are nested. Reinitialize the model for every size and seed, keep the validation set fixed, and use the same epoch or update budget. Record runtime as well as accuracy.

Part 4 - Python code skeleton

```
import torch

def E07(train_dataset, validation_loader, sizes, seeds,
        model_factory, train_model, evaluate):
    records = []
    for seed in seeds:
        generator = torch.Generator().manual_seed(seed)
        order = torch.randperm(len(train_dataset), generator=generator)
        for size in sizes:
            subset_indices = order[:size]
            model = model_factory()
            ...  # train with a controlled budget
            metric = evaluate(model, validation_loader)
            records.append({
                "seed": seed,
                "training_size": int(size),
                "validation_metric": metric,
            })
    return records

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

10.9.9 Exercise 08 - Inference Throughput Test

Part 1 - Why this matters

Inference throughput measures how many samples a system processes per unit of time. It matters when predictions arrive in large batches or when a service must handle sustained demand.

A valid benchmark separates warm-up from measurement and accounts for asynchronous accelerator execution. It also records batch size, dtype, device, and total work because throughput numbers without conditions cannot be compared responsibly.

Part 2 - Mathematical definition

If N samples complete in elapsed time T:

$$\text{throughput} = \frac{N}{T} \quad \text{samples per second}$$

Several timed repetitions should be used after warm-up.

Part 3 - Hints and Helper

Set evaluation mode and disable gradients. Warm up the model, synchronize CUDA before starting and after finishing, use `time.perf_counter()`, and report both total throughput and benchmark conditions.

Part 4 - Python code skeleton

```
import time
import torch

def E08(model, X, warmup=10, repetitions=100):
    model.eval()
    device = X.device
    with torch.inference_mode():
        for _ in range(warmup):
            model(X)
        if device.type == "cuda":
            torch.cuda.synchronize(device)
        start = time.perf_counter()
        for _ in range(repetitions):
            model(X)
        if device.type == "cuda":
            torch.cuda.synchronize(device)
        elapsed = time.perf_counter() - start
    samples = X.shape[0] * repetitions
    return {"samples_per_second": ...,
            "elapsed_seconds": elapsed,
            "batch_size": X.shape[0],
            "device": str(device)}

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

10.9.10 Exercise 09 - Latency Profiling on CUDA

Part 1 - Why this matters

Latency measures the time required for an individual request or batch. A system may have high throughput but still respond too slowly for an interactive application, so throughput and latency must be evaluated separately.

CUDA operations are asynchronous with respect to the CPU. Measuring ordinary wall-clock time without synchronization can stop the timer before GPU work finishes and produce misleadingly small latency values.

Part 2 - Technical concept and behavior

For measured latencies $t_1, \ldots, t_R$, report more than the mean:

$$\bar{t} = \frac{1}{R} \sum_{r=1}^{R} t_r$$

Percentiles such as the median, p_{95}, and p_{99} expose tail behavior hidden by the mean.

Part 3 - Hints and Helper

Require a CUDA tensor and perform warm-up iterations. Create two CUDA events with timing enabled. Record both events, synchronize before reading elapsed time, and calculate percentiles from all repetitions.

Part 4 - Python code skeleton

```
import torch

def E09(model, X, warmup=20, repetitions=100):
    if X.device.type != "cuda":
        raise ValueError(...)
    model.eval()
    latencies_ms = []
    with torch.inference_mode():
        for _ in range(warmup):
            model(X)
        torch.cuda.synchronize(X.device)
        for _ in range(repetitions):
            start = torch.cuda.Event(enable_timing=True)
            end = torch.cuda.Event(enable_timing=True)
            start.record()
            model(X)
            end.record()
            torch.cuda.synchronize(X.device)
            latencies_ms.append(start.elapsed_time(end))
    values = torch.tensor(latencies_ms)
    return {"mean_ms": values.mean().item(),
            "p50_ms": ..., "p95_ms": ..., "p99_ms": ...}

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

10.9.11 Exercise 10 - Memory Usage Profiling

Part 1 - Why this matters

Memory limits determine which models, dtypes, and batch sizes can run on an accelerator. An evaluation may be numerically correct yet unusable because it exhausts memory or leaves too little capacity for concurrent requests.

PyTorch distinguishes memory occupied by live tensors from memory reserved by its caching allocator. Both current and peak values matter, and external tools may report reserved memory even after individual tensors are released.

Part 2 - Technical concept and behavior

For a controlled evaluation region, record:

$$M_{\text{allocated}}^{\text{peak}} \quad \text{and} \quad M_{\text{reserved}}^{\text{peak}}$$

The first describes tensor allocations tracked by PyTorch; the second describes memory managed by the caching allocator.

Part 3 - Hints and Helper

Require CUDA, complete warm-up first, synchronize, and reset the peak CUDA memory statistics. Measure baseline, current, and peak values. Do not treat cache clearing as a way to free memory still owned by live tensors.

Part 4 - Python code skeleton

```
import torch

def E10(model, X):
    if X.device.type != "cuda":
        raise ValueError(...)
    model.eval()
    with torch.inference_mode():
        model(X)
        torch.cuda.synchronize(X.device)
        baseline = torch.cuda.memory_allocated(X.device)
        torch.cuda.reset_peak_memory_stats(X.device)
        output = model(X)
        torch.cuda.synchronize(X.device)
        report = {
            "baseline_allocated_bytes": baseline,
            "current_allocated_bytes":
                torch.cuda.memory_allocated(X.device),
            "peak_allocated_bytes": ...,
            "peak_reserved_bytes": ...,
        }
    return report

def main():
    print(E10(...))

if __name__ == "__main__":
    main()
```

10.9.12 Exercise 11 - Error Distribution Analysis

Part 1 - Why this matters

A single average metric hides how errors are distributed. Two models can share the same MAE while one makes consistent moderate errors and the other is usually accurate but occasionally fails catastrophically.

Examining signed and absolute errors reveals bias, spread, asymmetry, and extreme cases. This helps connect a numerical score to the actual behavior users may experience.

Part 2 - Mathematical definition

For residuals $r_i = \hat{y}_i - y_i$, useful summaries include:

$$\bar{r} = \frac{1}{N}\sum_{i=1}^{N} r_i, \qquad q_p(|r|) = p\text{th quantile of absolute error}$$

A positive mean residual indicates systematic overprediction under this sign convention.

Part 3 - Hints and Helper

Flatten and concatenate detached residuals from all batches. Report count, mean, standard deviation, minimum, maximum, and absolute-error quantiles. Also construct histogram counts with explicit bin boundaries.

Part 4 - Python code skeleton

```
import torch

def E11(model, loader, device, bins=20):
    model.eval()
    residual_parts = []
    with torch.inference_mode():
        for X, target in loader:
            X, target = X.to(device), target.to(device)
            residual = model(X) - target
            residual_parts.append(residual.detach().cpu().reshape(-1))
    residuals = torch.cat(residual_parts)
    absolute = residuals.abs()
    histogram = torch.histogram(residuals, bins=bins)
    return {
        "count": residuals.numel(),
        "mean_residual": residuals.mean().item(),
        "std_residual": ...,
        "absolute_q50": torch.quantile(absolute, 0.50).item(),
        "absolute_q95": ...,
        "histogram_counts": histogram.hist.tolist(),
        "histogram_edges": histogram.bin_edges.tolist(),
    }

def main():
```

```
    print(E11(...))

if __name__ == "__main__":
    main()
```

10.9.13 Exercise 12 - Residual Analysis

Part 1 - Why this matters

Residual analysis checks whether the remaining errors look random or still contain structure the model failed to learn. Trends, curves, changing variance, or clusters in residuals can reveal missing features, an unsuitable model form, or data quality problems.

Linear regression assumptions cannot be judged from training loss alone. Residuals should be examined against predictions and important inputs, while remembering that visual or correlation evidence is diagnostic rather than a complete statistical proof.

Part 2 - Technical concept and behavior

The residual is defined as:

$$r_i = y_i - \hat{y}_i$$

A desirable residual pattern is centered near zero without a systematic relationship to fitted values or relevant features.

Part 3 - Hints and Helper

Collect targets, predictions, and inputs in the same order. Compute residual mean and correlations with predictions and each feature. Divide predictions into bins and compare the residual mean and variance inside each bin.

Part 4 - Python code skeleton

```
import torch

def correlation(a, b):
    a = a - a.mean()
    b = b - b.mean()
    return (a * b).sum() / ...

def E12(model, X, target, bin_count=10):
    model.eval()
    with torch.inference_mode():
        prediction = model(X)
    residual = target - prediction
    flat_prediction = prediction.reshape(-1).double()
    flat_residual = residual.reshape(-1).double()
    edges = torch.linspace(
        flat_prediction.min(), flat_prediction.max(),
        bin_count + 1, device=flat_prediction.device
```

```
    )
    bin_summary = []
    for index in range(bin_count):
        mask = ...
        bin_summary.append({
            "count": int(mask.sum()),
            "mean_residual": ...,
            "residual_std": ...,
        })
    return {"mean_residual": flat_residual.mean().item(),
            "prediction_correlation":
                correlation(flat_prediction, flat_residual).item(),
            "bins": bin_summary}

def main():
    print(E12(...))

if __name__ == "__main__":
    main()
```

10.9.14 Exercise 13 - Model Robustness Test

Part 1 - Why this matters

Robustness describes whether model behavior remains acceptable when inputs differ slightly from ideal evaluation data. Small measurement changes, feature dropout, or bounded shifts should not cause uncontrolled prediction changes.

A robustness test must define realistic perturbations and acceptance limits. Arbitrary extreme corruption measures failure outside the intended operating range and may say little about the model's practical reliability.

Part 2 - Technical concept and behavior

For perturbation $\boldsymbol{\delta}$ bounded by ϵ:

$$\|\boldsymbol{\delta}\|_\infty \leq \epsilon$$

measure both metric degradation and prediction sensitivity:

$$S = \frac{\|f(\mathbf{x} + \boldsymbol{\delta}) - f(\mathbf{x})\|}{\|\boldsymbol{\delta}\| + \eta}$$

where η prevents division by zero.

Part 3 - Hints and Helper

Define several domain-relevant perturbation functions, preserve immutable features, and evaluate them on the same examples. Report clean score, perturbed score, degradation, maximum prediction shift, and pass or fail against explicit thresholds.

Part 4 - Python code skeleton

```
import torch

def E13(model, X, target, perturbations, max_degradation):
    model.eval()
    results = {}
    with torch.inference_mode():
        clean_prediction = model(X)
        clean_rmse = torch.sqrt(
            torch.mean((clean_prediction - target) ** 2)
        ).item()
        for name, transform in perturbations.items():
            changed_X = transform(X.clone())
            changed_prediction = model(changed_X)
            changed_rmse = ...
            shift = (changed_prediction - clean_prediction).abs()
            degradation = changed_rmse - clean_rmse
            results[name] = {
                "rmse": changed_rmse,
                "degradation": degradation,
                "max_prediction_shift": shift.max().item(),
                "passed": degradation <= ...,
            }
    return {"clean_rmse": clean_rmse, "tests": results}

def main():
    print(E13(...))

if __name__ == "__main__":
    main()
```

10.9.15 Exercise 14 - Performance Regression Detection

Part 1 - Why this matters

A code or model update can silently reduce accuracy, throughput, or memory efficiency. Performance regression testing converts known acceptable behavior into automated checks before a new version is accepted.

Thresholds must reflect measurement noise and metric direction. Lower RMSE is better, higher throughput is better, and benchmark variability means exact equality is usually an inappropriate requirement.

Part 2 - Technical concept and behavior

For a lower-is-better metric with baseline b, candidate c, and allowed relative increase τ:

$$\frac{c - b}{|b| + \epsilon} \leq \tau$$

Higher-is-better metrics require the corresponding allowed relative decrease rule.

Part 3 - Hints and Helper

Store baseline values with hardware and configuration metadata. Repeat timing tests and compare robust summaries such as medians. Define metric direction and tolerance in a specification rather than hard-coding one rule for every value.

Part 4 - Python code skeleton

```
def E14(baseline, candidate, specification, epsilon=1e-12):
    checks = {}
    for metric, rule in specification.items():
        old = baseline[metric]
        new = candidate[metric]
        relative_change = (new - old) / (abs(old) + epsilon)
        if rule["direction"] == "lower":
            passed = relative_change <= rule["tolerance"]
        elif rule["direction"] == "higher":
            passed = relative_change >= -rule["tolerance"]
        else:
            raise ValueError(...)
        checks[metric] = {
            "baseline": old,
            "candidate": new,
            "relative_change": relative_change,
            "passed": ...,
        }
    return {"passed": all(x["passed"] for x in checks.values()),
            "checks": checks}

def main():
    print(E14(...))

if __name__ == "__main__":
    main()
```

10.9.16 Exercise 15 - GPU Utilization Monitoring

Part 1 - Why this matters

A CUDA workload can be slower than expected even when the model is correct. Low utilization may indicate tiny batches, input pipeline delays, excessive synchronization, or CPU work that leaves the GPU waiting.

Utilization is a sampled system measurement rather than an exact per-operation property. The sampling interval and other processes on the same device must be recorded before conclusions are drawn.

Part 2 - Technical concept and behavior

For utilization samples $u_1, \ldots, u_m$ expressed as percentages, summarize:

$$\bar{u} = \frac{1}{m} \sum_{j=1}^{m} u_j$$

Also report minimum, maximum, sample count, throughput, and device identity.

Part 3 - Hints and Helper

Check CUDA availability and warm up the workload. Sample CUDA utilization while repeated inference is active, not only after it finishes. Handle unsupported monitoring environments clearly instead of inventing a zero value.

Part 4 - Python code skeleton

```
import threading
import time
import torch

def E15(model, X, repetitions=500, interval=0.1):
    if X.device.type != "cuda":
        raise ValueError(...)
    samples = []
    stop = threading.Event()

    def monitor():
        while not stop.is_set():
            samples.append(torch.cuda.utilization(X.device))
            time.sleep(interval)

    worker = threading.Thread(target=monitor)
    model.eval()
    worker.start()
    try:
        with torch.inference_mode():
            for _ in range(repetitions):
                model(X)
        torch.cuda.synchronize(X.device)
    finally:
        stop.set()
        worker.join()
    values = torch.tensor(samples, dtype=torch.float32)
    return {"mean_utilization_percent": ...,
            "minimum_percent": ..., "maximum_percent": ...,
            "samples": len(samples)}

def main():
    print(E15(...))

if __name__ == "__main__":
    main()
```

10.9.17 Exercise 16 - Metric Precision Comparison

Part 1 - Why this matters

Floating-point dtype affects metric values because reductions combine many finite-precision operations. Differences are often small, but they can matter when errors are extremely large, extremely small, or close to a release threshold.

A precision comparison should use identical prediction data cast to different dtypes. Re-running the full model in different dtypes mixes metric-reduction effects with possible changes in model computation.

Part 2 - Technical concept and behavior

For a metric computed in two dtypes:

$$\Delta_{\text{abs}} = |M_{64} - M_{32}|, \qquad \Delta_{\text{rel}} = \frac{|M_{64} - M_{32}|}{|M_{64}| + \epsilon}$$

Float64 is useful as a higher-precision comparison, not an assumption of exact arithmetic.

Part 3 - Hints and Helper

Create one set of predictions and targets, detach them, then compute MAE and RMSE in float32 and float64. Test ordinary, large-magnitude, and small-error cases. Report absolute and relative differences.

Part 4 - Python code skeleton

```
import torch

def metrics(prediction, target, dtype):
    error = prediction.to(dtype) - target.to(dtype)
    return {
        "mae": error.abs().mean().item(),
        "rmse": torch.sqrt((error ** 2).mean()).item(),
    }

def E16(prediction, target, epsilon=1e-15):
    fp32 = metrics(prediction.detach(), target.detach(), torch.float32)
    fp64 = metrics(prediction.detach(), target.detach(), torch.float64)
    differences = {}
    for name in fp32:
        absolute = abs(fp64[name] - fp32[name])
        differences[name] = {
            "absolute": absolute,
            "relative": absolute / (... + epsilon),
        }
    return {"float32": fp32, "float64": fp64,
            "differences": differences}

def main():
```

```
    print(E16(...))

if __name__ == "__main__":
    main()
```

10.9.18 Exercise 17 - Evaluation Pipeline Automation

Part 1 - Why this matters

An automated evaluation pipeline applies the same data handling, model state, metrics, and benchmark rules to every model candidate. This reduces manual mistakes and makes results reproducible enough to compare over time.

Automation also creates evidence. A useful output contains configuration, environment, dataset identity, metric definitions, sample counts, and failures rather than only one final score.

Part 2 - Technical concept and behavior

A pipeline can be viewed as a deterministic mapping:

$$(\text{model}, \text{dataset}, \text{configuration}) \longrightarrow (\text{metrics}, \text{diagnostics}, \text{metadata})$$

Each stage should validate its inputs and return a structured result.

Part 3 - Hints and Helper

Separate data validation, metric evaluation, robustness checks, and speed benchmarks into functions. Use one configuration dictionary, fixed seeds, and a stable result schema. Capture exceptions per optional check while failing immediately on invalid core metrics.

Part 4 - Python code skeleton

```
import platform
import time
import torch

def E17(model, loader, device, config):
    started = time.time()
    report = {
        "metadata": {
            "torch_version": torch.__version__,
            "python": platform.python_version(),
            "device": str(device),
            "seed": config["seed"],
        },
        "metrics": {},
        "diagnostics": {},
        "status": "running",
    }
    try:
        ...  # validate data and model output
```

```
        report["metrics"]["rmse"] = ...
        report["metrics"]["mae"] = ...
        report["diagnostics"]["residuals"] = ...
        report["status"] = "passed"
    except Exception as error:
        report["status"] = "failed"
        report["error"] = {"type": type(error).__name__,
                           "message": str(error)}
    report["duration_seconds"] = time.time() - started
    return report

def main():
    print(E17(...))

if __name__ == "__main__":
    main()
```

10.9.19 Exercise 18 - Evaluation Failure Debugging

Part 1 - Why this matters

Evaluation can fail through shape broadcasting, device mismatch, non-finite values, empty loaders, incorrect model mode, or biased batch aggregation. Some of these bugs produce plausible numbers instead of exceptions, making structured debugging essential.

The debugging goal is to locate the first broken invariant. Checking inputs, outputs, and accumulators batch by batch is more informative than modifying several parts of the pipeline at once.

Part 2 - Technical concept and behavior

Before metric accumulation, require:

$$\text{shape}(\hat{\mathbf{y}}) = \text{shape}(\mathbf{y}), \qquad \text{isfinite}(\hat{\mathbf{y}}) = \text{True}$$

Device, dtype, batch count, and accumulated element count should also be validated.

Part 3 - Hints and Helper

Return a diagnostic record containing the failing stage and batch index. Check loader emptiness, evaluation mode, devices, shapes, finiteness, and accumulator counts in a fixed order. Never silently reshape predictions merely to make an error disappear.

Part 4 - Python code skeleton

```
import torch

def E18(model, loader, device):
    model.eval()
    diagnostics = {"batches": 0, "elements": 0, "errors": []}
    with torch.inference_mode():
```

```
        for batch_index, (X, target) in enumerate(loader):
            diagnostics["batches"] += 1
            try:
                X, target = X.to(device), target.to(device)
                prediction = model(X)
                if prediction.shape != target.shape:
                    raise ValueError(
                        f"shape mismatch: {prediction.shape} vs "
                        f"{target.shape}"
                    )
                if not torch.isfinite(prediction).all():
                    raise FloatingPointError(...)
                diagnostics["elements"] += prediction.numel()
                ...
            except Exception as error:
                diagnostics["errors"].append({
                    "batch": batch_index,
                    "type": type(error).__name__,
                    "message": str(error),
                })
                break
    if diagnostics["batches"] == 0:
        diagnostics["errors"].append({"type": "EmptyLoader", ...})
    return diagnostics

def main():
    print(E18(...))

if __name__ == "__main__":
    main()
```

10.9.20 Exercise 19 - Final Evaluation Report

Part 1 - Why this matters

A final report turns disconnected measurements into a decision-ready description of model quality and operational behavior. It should state what was evaluated, under which conditions, and where the model fails, not simply announce that performance is good.

The report must preserve both strengths and limitations. Accuracy, residual behavior, robustness, latency, throughput, and memory answer different questions and should remain separately visible before an overall conclusion is made.

Part 2 - Technical concept and behavior

A complete report combines:

$$R = \{\text{data}, \text{metrics}, \text{residuals}, \text{robustness}, \text{performance}, \text{decision}\}$$

The final decision should follow explicit acceptance criteria and identify every failed check.

Part 3 - Hints and Helper

Build the report from structured outputs of earlier exercises. Include sample counts, RMSE, MAE, error quantiles, robustness results, latency percentiles, throughput, peak memory, configuration, and warnings. Save machine-readable data separately from the human summary.

Part 4 - Python code skeleton

```
import json
from datetime import datetime, timezone

def E19(model_name, dataset_name, results, acceptance):
    checks = {
        "rmse": results["metrics"]["rmse"] <= acceptance["max_rmse"],
        "mae": results["metrics"]["mae"] <= acceptance["max_mae"],
        "p95_latency": (
            results["performance"]["p95_latency_ms"]
            <= acceptance["max_p95_latency_ms"]
        ),
        "robustness": ...,
    }
    report = {
        "generated_at": datetime.now(timezone.utc).isoformat(),
        "model": model_name,
        "dataset": dataset_name,
        "results": results,
        "acceptance_checks": checks,
        "decision": "accepted" if all(checks.values()) else "rejected",
        "failed_checks": [name for name, passed in checks.items()
                          if not passed],
    }
    return report

def main():
    report = E19(...)
    print(json.dumps(report, indent=2))

if __name__ == "__main__":
    main()
```

10.10 LR08 - Result Visualization and Interpretation

Models and Training

10.10.1 Exercise 00 - Plot Predictions vs Targets

Part 1 - Why this matters

A predictions-versus-targets plot shows whether a regression model follows the correct scale and direction across the evaluation set. Points near the identity line indicate close agreement, while systematic displacement can reveal bias or calibration problems.

The graph must preserve equal meaning on both axes. Unequal limits or an incorrect reference line can visually exaggerate good performance, so the plotted range should be computed from both targets and predictions.

Part 2 - Visual and mathematical interpretation

For a perfect prediction:

$$\hat{y}_i = y_i$$

Therefore, the reference line is $y = x$. Vertical distance from this line represents the signed prediction error under the plotted coordinate convention.

Part 3 - Hints and Helper

Evaluate the model without gradients, flatten predictions and targets, move only detached values to CPU, and create a scatter plot. Use common minimum and maximum limits, equal axis scaling, labels, a legend, and a descriptive title.

Part 4 - Python code skeleton

```
import matplotlib.pyplot as plt
import torch

def E00(model, loader, device, output_path):
    model.eval()
    predictions, targets = [], []
    with torch.inference_mode():
        for X, target in loader:
            prediction = model(X.to(device))
            predictions.append(prediction.detach().cpu().reshape(-1))
            targets.append(target.detach().cpu().reshape(-1))
    prediction = torch.cat(predictions)
    target = torch.cat(targets)
    lower = torch.minimum(prediction.min(), target.min()).item()
    upper = ...
    fig, ax = plt.subplots()
    ax.scatter(target.numpy(), prediction.numpy(), alpha=0.5)
    ax.plot([lower, upper], [lower, upper], "--", label="Ideal")
    ax.set(...)
    fig.savefig(output_path, dpi=150, bbox_inches="tight")
    plt.close(fig)

def main():
    E00(...)

if __name__ == "__main__":
    main()
```

10.10.2 Exercise 01 - Plot Loss Curve

Part 1 - Why this matters

A loss curve shows how optimization develops across epochs. It can reveal rapid improvement, slow learning, plateaus, instability, or divergence that would be hidden by reporting only the final loss.

Training and validation loss should be shown together when available. Their separation helps distinguish optimization progress from generalization, but only if both series use the same metric definition and epoch alignment.

Part 2 - Visual and mathematical interpretation

For epoch index t, plot:

$$(t, \mathcal{L}_{\text{train},t}) \quad \text{and} \quad (t, \mathcal{L}_{\text{validation},t})$$

A logarithmic vertical scale may clarify changes across several orders of magnitude, but it requires strictly positive values.

Part 3 - Hints and Helper

Validate that histories have equal, nonzero length. Start epoch labels at one, add labels and a legend, and optionally support a logarithmic scale. Mark the epoch with the lowest validation loss.

Part 4 - Python code skeleton

```
import matplotlib.pyplot as plt
import torch

def E01(train_loss, validation_loss, output_path, log_scale=False):
    train = torch.as_tensor(train_loss, dtype=torch.float64)
    validation = torch.as_tensor(validation_loss, dtype=torch.float64)
    if train.shape != validation.shape or train.numel() == 0:
        raise ValueError(...)
    epochs = torch.arange(1, train.numel() + 1)
    best = int(validation.argmin())
    fig, ax = plt.subplots()
    ax.plot(epochs.numpy(), train.numpy(), label="Train")
    ax.plot(epochs.numpy(), validation.numpy(), label="Validation")
    ax.scatter(..., label="Best validation")
    if log_scale:
        ax.set_yscale("log")
    ax.set(...)
    fig.savefig(output_path, dpi=150, bbox_inches="tight")
    plt.close(fig)

def main():
    E01(...)
```

```
if __name__ == "__main__":
    main()
```

10.10.3 Exercise 02 - Plot Residuals

Part 1 - Why this matters

A residual plot tests whether prediction errors contain patterns that the model did not learn. Curvature, clusters, or changing spread can indicate nonlinearity, missing variables, unequal variance, or problematic observations.

Residual plots support diagnosis rather than automatic proof. A visible pattern suggests a question to investigate, while a random-looking plot does not guarantee that every modeling assumption is satisfied.

Part 2 - Visual and mathematical interpretation

Using residual definition:

$$r_i = y_i - \hat{y}_i$$

plot $(\hat{y}_i, r_i)$ and add the horizontal reference $r = 0$. Balanced points around zero are preferable to a systematic positive or negative band.

Part 3 - Hints and Helper

Detach and move prediction and target tensors to CPU before plotting. Use transparency for overlapping points, include the zero line, and report residual mean in the title or returned metadata.

Part 4 - Python code skeleton

```
import matplotlib.pyplot as plt
import torch

def E02(prediction, target, output_path):
    prediction = prediction.detach().cpu().reshape(-1)
    target = target.detach().cpu().reshape(-1)
    if prediction.shape != target.shape:
        raise ValueError(...)
    residual = target - prediction
    fig, ax = plt.subplots()
    ax.scatter(prediction.numpy(), residual.numpy(),
               alpha=0.5, s=16)
    ax.axhline(0.0, color="black", linestyle="--")
    ax.set_xlabel(...)
    ax.set_ylabel(...)
    ax.set_title(f"Residual mean: {residual.mean().item():.4f}")
    fig.savefig(output_path, dpi=150, bbox_inches="tight")
    plt.close(fig)
    return {"mean_residual": ..., "count": residual.numel()}
```

```
def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

10.10.4 Exercise 03 - Visualization with Large Batches

Part 1 - Why this matters

Plotting millions of points can consume excessive memory, produce an unreadable solid region, and make figure generation slower than model evaluation. A visualization should preserve the important distribution without displaying every observation.

Sampling must be explicit and reproducible. Selecting only the first batch can introduce ordering bias, while random sampling without a recorded seed makes figures difficult to compare.

Part 2 - Technical concept and behavior

For N available points and budget $M < N$, select indices:

$$I \subset \{1, \ldots, N\}, \qquad |I| = M$$

The sample should represent the full evaluation stream or use aggregation such as bins when density itself is important.

Part 3 - Hints and Helper

Use reservoir sampling or collect a seeded random subset after concatenation when memory allows. State the total and displayed counts in the title. Consider a two-dimensional histogram instead of scatter for very dense data.

Part 4 - Python code skeleton

```
import matplotlib.pyplot as plt
import torch

def E03(prediction, target, max_points, seed, output_path):
    prediction = prediction.detach().cpu().reshape(-1)
    target = target.detach().cpu().reshape(-1)
    total = prediction.numel()
    generator = torch.Generator().manual_seed(seed)
    if total > max_points:
        indices = torch.randperm(total, generator=generator)[:max_points]
    else:
        indices = torch.arange(total)
    shown_prediction = prediction[indices]
    shown_target = ...
    fig, ax = plt.subplots()
    ax.scatter(shown_target.numpy(), shown_prediction.numpy(),
```

```
                s=8, alpha=0.25)
    ax.set_title(f"Displayed {indices.numel()} of {total} points")
    ax.set(...)
    fig.savefig(output_path, dpi=150, bbox_inches="tight")
    plt.close(fig)

def main():
    E03(...)

if __name__ == "__main__":
    main()
```

10.10.5 Exercise 04 - CPU vs GPU Visualization Parity

Part 1 - Why this matters

Model predictions may be computed on CPU or CUDA, but the final visual interpretation should remain consistent within a justified numerical tolerance. A device-specific mismatch can reveal dtype, model-state, checkpoint, or data-transfer mistakes.

Exact equality is usually the wrong requirement for floating-point computation across devices. The comparison should measure numerical differences first, then verify that both plots use identical samples, limits, and styling.

Part 2 - Technical concept and behavior

For CPU and GPU predictions:

$$\Delta_{\max} = \max_i |\hat{y}_i^{\mathrm{CPU}} - \hat{y}_i^{\mathrm{GPU}}|$$

Parity passes when the selected absolute and relative tolerances are satisfied and both plotted datasets are aligned.

Part 3 - Hints and Helper

Create equivalent model copies with the same state dictionary, use evaluation mode and no gradients, and bring both outputs to CPU. Use `torch.allclose` before drawing their difference plot.

Part 4 - Python code skeleton

```
import copy
import matplotlib.pyplot as plt
import torch

def E04(model, X, output_path, atol=1e-5, rtol=1e-4):
    if not torch.cuda.is_available():
        return {"skipped": ...}
    cpu_model = copy.deepcopy(model).cpu().eval()
    gpu_model = copy.deepcopy(model).cuda().eval()
    with torch.inference_mode():
```

```
        cpu_prediction = cpu_model(X.cpu()).cpu()
        gpu_prediction = gpu_model(X.cuda()).cpu()
    difference = (cpu_prediction - gpu_prediction).reshape(-1)
    parity = torch.allclose(
        cpu_prediction, gpu_prediction, atol=atol, rtol=rtol
    )
    fig, ax = plt.subplots()
    ax.plot(difference.numpy())
    ax.axhline(0.0, color="black", linestyle="--")
    ax.set(...)
    fig.savefig(output_path, dpi=150, bbox_inches="tight")
    plt.close(fig)
    return {"parity": parity, "maximum_difference": ...}

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

10.10.6 Exercise 05 - Visualize Convergence Speed

Part 1 - Why this matters

Convergence speed describes how quickly training reaches a useful performance level, not merely which run has the lowest final loss. Visualizing it helps compare optimizers, learning rates, and initialization choices under controlled budgets.

Epoch count alone can be misleading when configurations process data at different speeds. A complete view can compare loss against epochs, optimizer steps, and elapsed training time.

Part 2 - Visual and mathematical interpretation

For threshold τ, define time to target:

$$t_\tau = \min\{t : \mathcal{L}_t \leq \tau\}$$

If the threshold is never reached, the report should mark it as unreached rather than substitute the final epoch.

Part 3 - Hints and Helper

Accept aligned loss and elapsed-time histories for several runs. Plot each run with a distinct label, draw the threshold, and mark the first crossing. Do not compare runs trained on different data splits without stating it.

Part 4 - Python code skeleton

```
import matplotlib.pyplot as plt
import torch
```

```
def E05(run_histories, threshold, output_path):
    fig, ax = plt.subplots()
    summary = {}
    for name, history in run_histories.items():
        times = torch.as_tensor(history["seconds"])
        losses = torch.as_tensor(history["loss"])
        if times.shape != losses.shape:
            raise ValueError(...)
        ax.plot(times.numpy(), losses.numpy(), label=name)
        reached = torch.nonzero(losses <= threshold).reshape(-1)
        first = int(reached[0]) if reached.numel() else None
        summary[name] = {
            "time_to_threshold": None if first is None
            else times[first].item()
        }
        if first is not None:
            ax.scatter(... )
    ax.axhline(threshold, color="black", linestyle="--")
    ax.set(...)
    ax.legend()
    fig.savefig(output_path, dpi=150, bbox_inches="tight")
    plt.close(fig)
    return summary

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

10.10.7 Exercise 06 - Hyperparameter Impact Plot

Part 1 - Why this matters

Hyperparameters influence accuracy, stability, speed, and resource use. A structured plot helps reveal useful ranges and tradeoffs that are difficult to understand from a table of isolated experiments.

The horizontal axis must represent the actual parameter scale. Learning rates spanning several powers of ten are usually clearer on a logarithmic axis, while categorical settings require discrete positions.

Part 2 - Visual and mathematical interpretation

For hyperparameter value λ_j and repeated score $M_{j,r}$, plot:

$$\bar{M}_j = \frac{1}{R}\sum_{r=1}^{R} M_{j,r}$$

with uncertainty such as standard deviation across runs rather than showing only the best trial.

Part 3 - Hints and Helper

Group records by hyperparameter value, compute mean and standard deviation, and use error bars. Preserve all failed trials in the metadata. Label whether a lower or higher metric is better.

Part 4 - Python code skeleton

```
import matplotlib.pyplot as plt
import torch

def E06(records, parameter_name, metric_name, output_path,
        logarithmic_x=False):
    grouped = {}
    for record in records:
        grouped.setdefault(record[parameter_name], []).append(
            record[metric_name]
        )
    values = sorted(grouped)
    means, deviations = [], []
    for value in values:
        scores = torch.tensor(grouped[value], dtype=torch.float64)
        means.append(scores.mean().item())
        deviations.append(scores.std(unbiased=False).item())
    fig, ax = plt.subplots()
    ax.errorbar(values, means, yerr=deviations, marker="o")
    if logarithmic_x:
        ax.set_xscale("log")
    ax.set_xlabel(...)
    ax.set_ylabel(...)
    fig.savefig(output_path, dpi=150, bbox_inches="tight")
    plt.close(fig)

def main():
    E06(...)

if __name__ == "__main__":
    main()
```

10.10.8 Exercise 07 - Data Noise Visualization

Part 1 - Why this matters

Noise visualization shows how observed data deviates from an underlying relationship and how that disturbance affects fitted predictions. It can separate model limitations from irreducible variation in the measurements.

The graph should distinguish clean signal, noisy observations, and model output. Mixing them with identical styling or plotting unrelated samples can create a false impression of agreement.

Part 2 - Visual and mathematical interpretation

A synthetic noisy target may follow:

$$y_i = wx_i + b + \epsilon_i, \qquad \epsilon_i \sim \mathcal{N}(0, \sigma^2)$$

Increasing σ widens the observations around the clean line without changing the intended slope or intercept.

Part 3 - Hints and Helper

Sort one-dimensional inputs before drawing lines, but keep scatter points paired with their targets. Use a fixed random seed and shared inputs across noise levels. Include σ in labels or titles.

Part 4 - Python code skeleton

```
import matplotlib.pyplot as plt
import torch

def E07(X, clean_target, noisy_targets, prediction, output_path):
    x = X.detach().cpu().reshape(-1)
    clean = clean_target.detach().cpu().reshape(-1)
    predicted = prediction.detach().cpu().reshape(-1)
    order = torch.argsort(x)
    fig, ax = plt.subplots()
    ax.plot(x[order].numpy(), clean[order].numpy(),
            label="Clean relationship")
    ax.plot(x[order].numpy(), predicted[order].numpy(),
            label="Model prediction")
    for label, noisy_target in noisy_targets.items():
        noisy = noisy_target.detach().cpu().reshape(-1)
        ax.scatter(x.numpy(), noisy.numpy(), s=10, alpha=0.25,
                   label=...)
    ax.set(...)
    ax.legend()
    fig.savefig(output_path, dpi=150, bbox_inches="tight")
    plt.close(fig)

def main():
    E07(...)

if __name__ == "__main__":
    main()
```

10.10.9 Exercise 08 - Error Distribution Plot

Part 1 - Why this matters

An error-distribution plot reveals center, spread, asymmetry, and tails that a single average metric cannot show. It helps identify whether most predictions are acceptable while a small

group produces severe failures.

Histogram appearance depends on bin boundaries and count normalization. The chosen bins must therefore be stated, and comparisons between datasets should use consistent boundaries or density definitions.

Part 2 - Visual and mathematical interpretation

For residuals $r_i = y_i - \hat{y}_i$, the empirical histogram estimates how frequently residuals fall inside intervals:

$$c_j = \sum_{i=1}^{N} \mathbf{1}[a_j \leq r_i < a_{j+1}]$$

The zero reference identifies unbiased error under this residual convention.

Part 3 - Hints and Helper

Validate finite residuals, use explicit shared bin edges, add a zero line, and mark the mean and median. Report extreme quantiles beside the figure instead of trimming them silently.

Part 4 - Python code skeleton

```
import matplotlib.pyplot as plt
import torch

def E08(prediction, target, bin_count, output_path):
    residual = (
        target.detach().cpu().reshape(-1)
        - prediction.detach().cpu().reshape(-1)
    )
    if not torch.isfinite(residual).all():
        raise ValueError(...)
    mean = residual.mean().item()
    median = residual.median().item()
    fig, ax = plt.subplots()
    ax.hist(residual.numpy(), bins=bin_count, alpha=0.75)
    ax.axvline(0.0, color="black", linestyle="--", label="Zero")
    ax.axvline(mean, label="Mean")
    ax.axvline(median, label="Median")
    ax.set(...)
    ax.legend()
    fig.savefig(output_path, dpi=150, bbox_inches="tight")
    plt.close(fig)
    return {"mean": mean, "median": median, "q95_absolute": ...}

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

10.10.10 Exercise 09 - Visualization Performance Cost

Part 1 - Why this matters

Visualization has its own computational cost: device-to-host transfer, tensor conversion, figure construction, rendering, and file writing. If these steps occur inside a training loop, they can reduce throughput or force unwanted synchronization.

Measuring the entire operation as one block cannot identify the bottleneck. Separating collection, transfer, rendering, and saving makes optimization decisions evidence-based.

Part 2 - Technical concept and behavior

Total visualization time can be decomposed as:

$$T_{\text{total}} = T_{\text{transfer}} + T_{\text{prepare}} + T_{\text{render}} + T_{\text{save}}$$

Peak host memory and output file size are additional practical costs.

Part 3 - Hints and Helper

Synchronize CUDA before and after timing a transfer. Use `time.perf_counter()` for host stages, compare several point budgets, close every figure, and keep image settings constant.

Part 4 - Python code skeleton

```
import time
import matplotlib.pyplot as plt
import torch

def E09(prediction, target, output_path):
    if prediction.device.type == "cuda":
        torch.cuda.synchronize(prediction.device)
    start = time.perf_counter()
    cpu_prediction = prediction.detach().cpu()
    cpu_target = target.detach().cpu()
    if prediction.device.type == "cuda":
        torch.cuda.synchronize(prediction.device)
    transfer_seconds = time.perf_counter() - start

    render_start = time.perf_counter()
    fig, ax = plt.subplots()
    ax.scatter(cpu_target.numpy(), cpu_prediction.numpy(), s=4)
    fig.canvas.draw()
    render_seconds = ...
    save_start = time.perf_counter()
    fig.savefig(output_path, dpi=150)
    save_seconds = ...
    plt.close(fig)
    return {"transfer_seconds": transfer_seconds,
            "render_seconds": render_seconds,
            "save_seconds": save_seconds}
```

```
def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

10.10.11 Exercise 10 - Interactive Visualization

Part 1 - Why this matters

Interactive controls let the reader explore how slope, bias, noise, or another parameter changes predictions without regenerating a separate static figure for every value. This makes sensitivity and model behavior easier to understand through direct experimentation.

Interaction must preserve a defined mathematical relationship. A callback should update the existing plotted objects and labels rather than create new lines repeatedly, which would leak memory and clutter the axes.

Part 2 - Visual and mathematical interpretation

For a linear model controlled by sliders:

$$\hat{y}(x; w, b) = wx + b$$

Changing w rotates the line, while changing b shifts it vertically. Fixed axis limits make these changes visually comparable.

Part 3 - Hints and Helper

Use Matplotlib's `Slider`, reserve space for controls, and keep a reference to the prediction line. In each callback, call `set_ydata` and request a canvas redraw. Provide a reset control if desired.

Part 4 - Python code skeleton

```
import matplotlib.pyplot as plt
from matplotlib.widgets import Slider
import torch

def E10(X, target, initial_weight, initial_bias):
    x = X.detach().cpu().reshape(-1)
    y = target.detach().cpu().reshape(-1)
    order = torch.argsort(x)
    fig, ax = plt.subplots()
    fig.subplots_adjust(bottom=0.25)
    ax.scatter(x.numpy(), y.numpy(), alpha=0.4)
    initial = initial_weight * x[order] + initial_bias
    line, = ax.plot(x[order].numpy(), initial.numpy())
    weight_axis = fig.add_axes([0.2, 0.10, 0.65, 0.03])
    bias_axis = fig.add_axes([0.2, 0.05, 0.65, 0.03])
```

```
    weight_slider = Slider(weight_axis, "Weight", -5.0, 5.0,
                           valinit=initial_weight)
    bias_slider = Slider(bias_axis, "Bias", -5.0, 5.0,
                         valinit=initial_bias)

    def update(_):
        new_y = ...
        line.set_ydata(new_y.numpy())
        fig.canvas.draw_idle()

    weight_slider.on_changed(update)
    bias_slider.on_changed(update)
    plt.show()

def main():
    E10(...)

if __name__ == "__main__":
    main()
```

10.10.12 Exercise 11 - Visualization from Checkpoints

Part 1 - Why this matters

Checkpoint visualization shows how a model's predictions and parameters changed during training. It can reveal whether progress was smooth, whether later epochs became unstable, or whether several checkpoints produce nearly identical behavior.

Checkpoint files must be aligned with their epoch metadata and loaded safely. A graph using mislabeled or incompatible states creates a false training story even if every individual file loads successfully.

Part 2 - Technical concept and behavior

For checkpoints at epochs $t_1, \ldots, t_k$, compare:

$$f(\mathbf{x}; \boldsymbol{\theta}_{t_1}), \ldots, f(\mathbf{x}; \boldsymbol{\theta}_{t_k})$$

using the same input values, preprocessing, model architecture, and plotting limits.

Part 3 - Hints and Helper

Load only trusted checkpoints, map storage to CPU, validate required keys, and reuse one model factory. Sort checkpoints by stored epoch rather than filename. Plot a limited number of representative epochs if the directory is large.

Part 4 - Python code skeleton

```
from pathlib import Path
import matplotlib.pyplot as plt
import torch
```

```
def E11(checkpoint_paths, model_factory, X, output_path):
    x = X.detach().cpu()
    order = torch.argsort(x.reshape(-1))
    curves = []
    for path in checkpoint_paths:
        checkpoint = torch.load(
            Path(path), map_location="cpu", weights_only=True
        )
        if "model_state" not in checkpoint or "epoch" not in checkpoint:
            raise KeyError(...)
        model = model_factory().cpu()
        model.load_state_dict(checkpoint["model_state"])
        model.eval()
        with torch.inference_mode():
            prediction = model(x).reshape(-1)
        curves.append((int(checkpoint["epoch"]), prediction))
    curves.sort(key=lambda item: item[0])
    fig, ax = plt.subplots()
    for epoch, prediction in curves:
        ax.plot(x.reshape(-1)[order].numpy(),
                prediction[order].numpy(), label=f"Epoch {epoch}")
    ax.set(...)
    ax.legend()
    fig.savefig(output_path, dpi=150, bbox_inches="tight")
    plt.close(fig)

def main():
    E11(...)

if __name__ == "__main__":
    main()
```

10.10.13 Exercise 12 - Multi-Run Comparison Plot

Part 1 - Why this matters

Training results vary with initialization, data order, and stochastic operations. Comparing only one run can make an unstable method appear reliable or hide a configuration that succeeds only occasionally.

A multi-run plot should preserve individual variability while also showing a central trend. Mean lines without uncertainty bands can imply confidence that the experiment did not establish.

Part 2 - Visual and mathematical interpretation

For loss from run r at epoch t:

$$\bar{\mathcal{L}}_t = \frac{1}{R}\sum_{r=1}^{R}\mathcal{L}_{r,t}, \qquad s_t = \sqrt{\frac{1}{R}\sum_{r=1}^{R}(\mathcal{L}_{r,t} - \bar{\mathcal{L}}_t)^2}$$

The band $\bar{\mathcal{L}}_t \pm s_t$ summarizes variation, not a confidence interval unless defined accordingly.

Part 3 - Hints and Helper

Require aligned epoch histories or explicitly truncate to a shared range. Draw individual runs with low opacity, then the mean and standard-deviation band. Keep seeds and experimental settings in returned metadata.

Part 4 - Python code skeleton

```
import matplotlib.pyplot as plt
import torch

def E12(run_losses, output_path):
    lengths = {len(losses) for losses in run_losses.values()}
    if len(lengths) != 1:
        raise ValueError(...)
    names = list(run_losses)
    matrix = torch.tensor(
        [run_losses[name] for name in names], dtype=torch.float64
    )
    epochs = torch.arange(1, matrix.shape[1] + 1)
    mean = matrix.mean(dim=0)
    deviation = matrix.std(dim=0, unbiased=False)
    fig, ax = plt.subplots()
    for name, losses in zip(names, matrix):
        ax.plot(epochs.numpy(), losses.numpy(),
                alpha=0.2, label=f"Run {name}")
    ax.plot(epochs.numpy(), mean.numpy(), linewidth=2, label="Mean")
    ax.fill_between(epochs.numpy(), (mean - deviation).numpy(),
                    (mean + deviation).numpy(), alpha=0.2)
    ax.set(...)
    fig.savefig(output_path, dpi=150, bbox_inches="tight")
    plt.close(fig)
    return {"final_mean": ..., "final_std": ...}

def main():
    print(E12(...))

if __name__ == "__main__":
    main()
```

10.10.14 Exercise 13 - Visualization Bug Debugging

Part 1 - Why this matters

A visualization bug can reverse axes, break sample alignment, hide non-finite values, apply an accidental sort, or reuse stale figure state. The resulting image may still look convincing, making it more dangerous than a clear exception.

Debugging should verify the data lineage before adjusting colors or labels. Every displayed coordinate must be traceable to the same sample and to the tensor values intended for that axis.

Part 2 - Technical concept and behavior

For paired data, preserve the invariant:

$$(\text{target}_i, \text{prediction}_i) \longrightarrow \text{one plotted point}$$

Any filtering, sorting, or sampling operation must apply the same indices to both arrays.

Part 3 - Hints and Helper

Check shapes, element counts, devices, finiteness, and paired indices before plotting. Create a new figure explicitly, label axes from variables rather than memory, and close the figure after saving. Return diagnostics with the image.

Part 4 - Python code skeleton

```
import matplotlib.pyplot as plt
import torch

def E13(prediction, target, output_path):
    diagnostics = {}
    prediction = prediction.detach().cpu().reshape(-1)
    target = target.detach().cpu().reshape(-1)
    diagnostics["prediction_count"] = prediction.numel()
    diagnostics["target_count"] = target.numel()
    if prediction.shape != target.shape:
        raise ValueError(...)
    finite = torch.isfinite(prediction) & torch.isfinite(target)
    diagnostics["nonfinite_count"] = int((~finite).sum())
    if not finite.all():
        ...
    sample_indices = torch.arange(prediction.numel())
    paired_target = target[sample_indices]
    paired_prediction = ...
    fig, ax = plt.subplots()
    ax.scatter(paired_target.numpy(), paired_prediction.numpy())
    ax.set_xlabel("Target")
    ax.set_ylabel("Prediction")
    fig.savefig(output_path, dpi=150, bbox_inches="tight")
    plt.close(fig)
    return diagnostics

def main():
```

```
    print(E13(...))

if __name__ == "__main__":
    main()
```

10.10.15 Exercise 14 - Interpret Weight Values

Part 1 - Why this matters

In linear regression, each weight describes how the prediction changes when one feature increases while the other features remain fixed. Inspecting weights can reveal direction and model sensitivity in a directly interpretable architecture.

Magnitude comparisons require caution because feature units matter. A weight for meters cannot be compared fairly with a weight for millimeters unless inputs are transformed to a common, meaningful scale.

Part 2 - Mathematical interpretation

For multiple features:

$$\hat{y} = \sum_{j=1}^{d} w_j x_j + b$$

Holding other features fixed:

$$\frac{\partial \hat{y}}{\partial x_j} = w_j$$

Thus, a one-unit increase in x_j changes the prediction by w_j in target units.

Part 3 - Hints and Helper

Extract a linear layer's weight tensor, match every column to a feature name, and plot a signed horizontal bar chart. Include feature units and whether the inputs were standardized. Avoid causal language.

Part 4 - Python code skeleton

```
import matplotlib.pyplot as plt
import torch

def E14(linear_layer, feature_names, output_path):
    weights = linear_layer.weight.detach().cpu().reshape(-1)
    if weights.numel() != len(feature_names):
        raise ValueError(...)
    order = torch.argsort(weights.abs())
    ordered_weights = weights[order]
    ordered_names = [feature_names[index] for index in order.tolist()]
    colors = [
        "tab:blue" if value >= 0 else "tab:red"
        for value in ordered_weights.tolist()
```

```
    ]
    fig, ax = plt.subplots()
    ax.barh(ordered_names, ordered_weights.numpy(), color=colors)
    ax.axvline(0.0, color="black", linewidth=1)
    ax.set_xlabel(...)
    ax.set_title(...)
    fig.savefig(output_path, dpi=150, bbox_inches="tight")
    plt.close(fig)
    return dict(zip(feature_names, weights.tolist()))

def main():
    print(E14(...))

if __name__ == "__main__":
    main()
```

10.10.16 Exercise 15 - Interpret Bias Effect

Part 1 - Why this matters

The bias shifts a linear prediction independently of feature multiplication. It represents the model output when every input feature equals zero, which can be meaningful, artificial, or outside the observed data range.

Feature centering changes the interpretation of zero and therefore changes the meaning of the bias. A large bias is not automatically important or problematic without understanding preprocessing and feature units.

Part 2 - Mathematical interpretation

For one feature:

$$\hat{y} = wx + b$$

At $x = 0$:

$$\hat{y}(0) = b$$

Changing the bias from b to $b + \Delta b$ shifts every prediction vertically by Δb.

Part 3 - Hints and Helper

Plot several prediction lines with the same weight and different bias values. Keep axis limits fixed, include observed data, and mark $x = 0$ when it lies in a relevant range. Return the vertical shift for verification.

Part 4 - Python code skeleton

```
import matplotlib.pyplot as plt
import torch

def E15(X, target, weight, biases, output_path):
```

```
    x = X.detach().cpu().reshape(-1)
    y = target.detach().cpu().reshape(-1)
    order = torch.argsort(x)
    fig, ax = plt.subplots()
    ax.scatter(x.numpy(), y.numpy(), alpha=0.35, label="Observed")
    for bias in biases:
        prediction = weight * x[order] + bias
        ax.plot(x[order].numpy(), prediction.numpy(),
                label=f"Bias = {bias:.3f}")
    if x.min() <= 0 <= x.max():
        ax.axvline(0.0, color="black", linestyle="--")
    ax.set(...)
    ax.legend()
    fig.savefig(output_path, dpi=150, bbox_inches="tight")
    plt.close(fig)
    return {"biases": list(biases), "shared_weight": weight}

def main():
    print(E15(...))

if __name__ == "__main__":
    main()
```

10.10.17 Exercise 16 - Feature Contribution Analysis

Part 1 - Why this matters

A linear prediction can be decomposed into additive feature contributions. This allows the reader to see which features push one prediction upward or downward before the bias is added.

Contribution magnitude combines the learned weight with the current feature value. It is local to a sample and should not be confused with global importance or evidence that a feature causes the predicted outcome.

Part 2 - Mathematical interpretation

For sample i, define:

$$c_{ij} = w_j x_{ij}$$

Then:

$$\hat{y}_i = b + \sum_{j=1}^{d} c_{ij}$$

The contributions must reconstruct the original prediction within numerical tolerance.

Part 3 - Hints and Helper

Select one sample, compute element-wise products between features and weights, and verify the sum plus bias. Plot signed bars sorted by absolute contribution and include the reconstructed prediction in the title.

Part 4 - Python code skeleton

```
import matplotlib.pyplot as plt
import torch

def E16(linear_layer, sample, feature_names, output_path):
    x = sample.detach().cpu().reshape(-1)
    weights = linear_layer.weight.detach().cpu().reshape(-1)
    bias = linear_layer.bias.detach().cpu().reshape(-1)[0]
    if x.numel() != weights.numel():
        raise ValueError(...)
    contributions = x * weights
    reconstruction = contributions.sum() + bias
    with torch.inference_mode():
        direct = linear_layer(sample).detach().cpu().reshape(-1)[0]
    if not torch.allclose(reconstruction, direct):
        raise RuntimeError(...)
    order = torch.argsort(contributions.abs())
    names = [feature_names[i] for i in order.tolist()]
    fig, ax = plt.subplots()
    ax.barh(names, contributions[order].numpy())
    ax.axvline(0.0, color="black")
    ax.set_title(f"Prediction = {reconstruction.item():.4f}")
    fig.savefig(output_path, dpi=150, bbox_inches="tight")
    plt.close(fig)
    return {"contributions": ..., "bias": bias.item()}

def main():
    print(E16(...))

if __name__ == "__main__":
    main()
```

10.10.18 Exercise 17 - Explain Failure Cases

Part 1 - Why this matters

Average performance does not explain the examples where a model fails most severely. Inspecting high-error cases can uncover out-of-range inputs, missing values, rare subgroups, incorrect labels, or relationships that a linear model cannot represent.

Failure analysis must avoid choosing only examples that support a preferred story. Selection should follow a declared error rule, preserve original sample identifiers, and compare failures with typical cases.

Part 2 - Mathematical interpretation

Rank samples by absolute residual:

$$e_i = |y_i - \hat{y}_i|$$

The top-k set is:

$$F_k = \mathrm{TopK}_i(e_i)$$

These are the largest observed errors in the evaluated dataset, not necessarily every possible failure mode.

Part 3 - Hints and Helper

Use `torch.topk` on absolute residuals, retain identifiers and feature rows, and create a plot contrasting target and prediction for each selected case. Return a structured table for deeper inspection.

Part 4 - Python code skeleton

```
import matplotlib.pyplot as plt
import torch

def E17(X, target, prediction, sample_ids, k, output_path):
    target = target.detach().cpu().reshape(-1)
    prediction = prediction.detach().cpu().reshape(-1)
    absolute_error = (target - prediction).abs()
    k = min(k, absolute_error.numel())
    values, indices = torch.topk(absolute_error, k=k)
    positions = torch.arange(k)
    fig, ax = plt.subplots()
    ax.scatter(positions.numpy(), target[indices].numpy(),
               label="Target")
    ax.scatter(positions.numpy(), prediction[indices].numpy(),
               label="Prediction")
    ax.set_xticks(positions.numpy())
    ax.set_xticklabels([str(sample_ids[i]) for i in indices.tolist()],
                       rotation=45, ha="right")
    ax.set(...)
    ax.legend()
    fig.savefig(output_path, dpi=150, bbox_inches="tight")
    plt.close(fig)
    return [{"sample_id": sample_ids[i], "absolute_error": ...}
            for i in indices.tolist()]

def main():
    print(E17(...))

if __name__ == "__main__":
    main()
```

10.10.19 Exercise 18 - Visualization Reproducibility

Part 1 - Why this matters

A reproducible visualization uses the same data selection, ordering, scales, style, and output settings each time. This is essential when figures are compared across model versions or included in a technical report.

Identical code does not guarantee byte-identical images across every library version, backend, font, or platform. Reproducibility therefore includes recording the environment and preserving the data used to construct the figure.

Part 2 - Technical concept and behavior

A figure can be treated as a function:

$$\mathcal{V} = g(D, I, S, C)$$

where D is plotted data, I is selected indices, S is style configuration, and C is the software and rendering context.

Part 3 - Hints and Helper

Use a fixed seed for sampling, explicit axis limits, an explicit style context, and stable sorting. Save figure metadata and a separate tensor or JSON record containing selected indices and plotted values.

Part 4 - Python code skeleton

```
import json
import matplotlib
import matplotlib.pyplot as plt
import torch

def E18(prediction, target, seed, max_points,
        image_path, metadata_path):
    generator = torch.Generator().manual_seed(seed)
    prediction = prediction.detach().cpu().reshape(-1)
    target = target.detach().cpu().reshape(-1)
    total = prediction.numel()
    indices = torch.randperm(total, generator=generator)
    indices = indices[:min(max_points, total)]
    selected_prediction = prediction[indices]
    selected_target = target[indices]
    lower = torch.minimum(
        selected_prediction.min(), selected_target.min()
    ).item()
    upper = ...
    with plt.rc_context({"figure.figsize": (7, 5),
                         "font.size": 10}):
        fig, ax = plt.subplots()
        ax.scatter(selected_target.numpy(),
                   selected_prediction.numpy(), s=10)
        ax.set_xlim(lower, upper)
        ax.set_ylim(lower, upper)
```

```
        fig.savefig(image_path, dpi=150, metadata={"Creator": "LR08"})
        plt.close(fig)
    metadata = {"seed": seed, "indices": indices.tolist(),
                "torch_version": torch.__version__,
                "matplotlib_version": matplotlib.__version__}
    with open(metadata_path, "w", encoding="utf-8") as file:
        json.dump(..., file, indent=2)

def main():
    E18(...)

if __name__ == "__main__":
    main()
```

10.10.20 Exercise 19 - Final Model Interpretation Report

Part 1 - Why this matters

A final interpretation report combines model performance with evidence explaining how predictions are produced and where they fail. It gives the reader a coherent view instead of a folder of disconnected images.

Interpretation must remain within what linear regression supports. Weights, bias, feature contributions, residuals, and failure examples can describe associations and model behavior, but they do not establish causal relationships by themselves.

Part 2 - Report structure and interpretation

A complete interpretation package can contain:

$$\mathcal{R} = \{\text{fit}, \text{learning}, \text{residuals}, \text{parameters}, \text{contributions}, \text{failures}, \text{limitations}\}$$

Every conclusion should refer to a metric, figure, or validated model property.

Part 3 - Hints and Helper

Build the report from saved results of earlier exercises. Generate figures with consistent style, assign unique filenames, include captions and limitations, and write a machine-readable manifest linking every claim to its artifact.

Part 4 - Python code skeleton

```
import json
from pathlib import Path

def E19(model, evaluation, feature_names, output_directory):
    output = Path(output_directory)
    output.mkdir(parents=True, exist_ok=True)
    artifacts = {}
    artifacts["prediction_plot"] = ...
    artifacts["loss_plot"] = ...
```

```
    artifacts["residual_plot"] = ...
    artifacts["weight_plot"] = ...
    artifacts["failure_plot"] = ...
    conclusions = {
        "fit_quality": ...,
        "largest_weight_interpretation": ...,
        "residual_pattern": ...,
        "known_failure_cases": ...,
        "limitations": [
            "Model associations are not causal conclusions",
            ...,
        ],
    }
    report = {
        "artifacts": artifacts,
        "metrics": evaluation["metrics"],
        "conclusions": conclusions,
    }
    with open(output / "report.json", "w", encoding="utf-8") as file:
        json.dump(report, file, indent=2)
    return report

def main():
    print(E19(...))

if __name__ == "__main__":
    main()
```

Book 11

Logistic Regression

11.1 Introduction

11.1.1 What will the book teach you

Linear regression taught you how a model learns a continuous relationship between inputs and a numerical target. Logistic regression begins with the same weighted combination of features, but it changes the meaning of the output. The objective is no longer to predict an unrestricted value such as temperature, demand, or energy consumption. The objective is to estimate the probability of a class, such as whether an event will occur, whether a transaction is suspicious, whether a machine requires maintenance, or whether a measured condition belongs to the positive class.

The model first calculates a linear score called a logit:

$$z = \mathbf{w}^\top \mathbf{x} + b$$

The logit may take any real value, so it cannot be interpreted directly as a probability. Logistic regression passes this score through the sigmoid function:

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}$$

This transformation maps the unrestricted logit to a value between zero and one. The result p is interpreted as the model's estimated probability that the target belongs to the positive class, under the assumptions and data used to train the model. The first part of this book will teach you to construct this process manually, compare it with PyTorch operations, verify tensor shapes, examine single and batched predictions, and understand the difference between a logit, a probability, and a class label.

A probability does not become a class label until a decision rule is applied. For a threshold τ:

$$\widehat{y} = \begin{cases} 1, & p \geq \tau, \\ 0, & p < \tau. \end{cases}$$

The common threshold 0.5 is not a universal law. Changing the threshold changes the balance between false positives and false negatives. In a medical screening system, missing a positive case may be more costly than requesting an additional examination. In another system,

a false alarm may itself carry a serious cost. You will therefore experiment with thresholds, construct confusion matrices, inspect class imbalance, and study precision, recall, ROC curves, and AUC. The goal is to learn that the probability model and the operational decision policy are related but distinct components.

The sigmoid function will be studied as a mathematical object rather than treated as a hidden command. Its derivative is:

$$\sigma'(z) = \sigma(z)\Big(1 - \sigma(z)\Big)$$

This derivative becomes small when the logit is far from zero because the sigmoid approaches saturation near zero or one. You will visualize this behavior, inspect gradients with autograd, test extreme inputs, compare float32 and float64, and investigate stability on CPU and CUDA. These exercises will explain why a formula that is mathematically valid can still fail numerically when implemented through direct exponentiation with very large magnitudes.

Logistic regression is also a probability model. For a binary target $y_i \in \{0, 1\}$ and predicted probability p_i, the Bernoulli likelihood for one observation is:

$$P(y_i \mid \mathbf{x}_i) = p_i^{y_i}(1 - p_i)^{1-y_i}$$

Across independent observations, the likelihood is the product of these terms. Products of many probabilities can become extremely small, so practical computation uses the log-likelihood. Maximizing likelihood is equivalent to minimizing the negative log-likelihood, which leads directly to binary cross-entropy:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(p_i) + (1 - y_i)\log(1 - p_i)]$$

You will implement likelihood, log-likelihood, negative log-likelihood, and BCE manually before relying on PyTorch. This sequence will connect probability theory to the loss used during training instead of presenting them as unrelated formulas.

Numerical stability is especially important in classification. Calculating sigmoid probabilities first and then applying logarithms can produce unstable values for extreme logits. PyTorch's `BCEWithLogitsLoss` combines the sigmoid and binary cross-entropy operations in a more stable form and expects raw logits rather than probabilities. The exercises will require you to compare logits with probabilities, inspect reduction modes, apply class weighting, work with masks, test mixed precision, and debug non-finite losses. You must learn to identify which representation an operation expects before passing a tensor into it.

The learned parameters also define a decision boundary. For the threshold 0.5, sigmoid output reaches the threshold when the logit is zero:

$$\mathbf{w}^\top \mathbf{x} + b = 0$$

In two dimensions, this equation describes a line; in higher dimensions, it describes a hyperplane. The weights control its orientation, while the bias changes its position. You will plot decision boundaries, observe how scaling, noise, overlap, regularization, and training alter them, and identify regions that the model classifies incorrectly. These experiments will also

demonstrate an important limitation: logistic regression creates a linear boundary in the original feature space unless the inputs are transformed.

Training connects the BCE loss to parameter adjustment. You will inspect weight and bias gradients, perform manual updates, compare optimizer behavior, test accumulation and clipping, and study vanishing, exploding, noisy, or missing gradients. For a single binary example, the gradient signal with respect to the logit has the compact form:

$$\frac{\partial \mathcal{L}}{\partial z} = p - y$$

This relationship explains the direction of correction. When the predicted probability is greater than the target, the logit is pushed downward; when it is smaller than the target, it is pushed upward. The later gradient exercises will connect this local signal to the shapes and updates of every trainable parameter.

Evaluation in classification requires more than accuracy. Accuracy can appear high when one class dominates the dataset even if the model fails on the minority class. You will calculate precision, recall, and F1 score, analyze confusion matrices, test threshold sensitivity, measure robustness under noise, and compare repeated runs. You will also profile inference throughput, CUDA latency, and memory use so that model quality and system performance remain separately visible. A final evaluation must state the target distribution, threshold, metric definitions, and costs of different errors.

Probability values also require careful interpretation. A prediction of 0.9 is not automatically trustworthy merely because it is close to one. Calibration asks whether events assigned a probability near 0.9 occur approximately ninety percent of the time within an appropriate evaluation setting. The book will examine overconfidence, confidence drift, likelihood under noise, class imbalance, and probability calibration. This prevents the reader from confusing confidence expressed by the model with verified reliability in the real world.

The final chapter extends the ideas from binary classification to multiple classes. You will compare one-vs-rest logistic models with softmax-based classification, prepare multi-class targets, use cross-entropy, and examine macro and micro metrics. You will also test class imbalance, confusion matrices, probability calibration, gradient stability, mixed precision, scalability, and failure recovery. This extension will show which ideas belong specifically to binary sigmoid classification and which ideas generalize to larger classification systems.

Logistic regression is deliberately studied in depth because it forms a bridge between a linear model and neural classification. It introduces logits, probabilities, decision thresholds, classification losses, confusion-based metrics, calibration, and decision boundaries while remaining mathematically transparent. The perceptron will reuse weighted inputs and a bias but apply a harder decision mechanism, while later neural networks will learn many stacked transformations before producing their final logits. If you understand logistic regression as a complete system, those later models will be easier to reason about and debug.

11.1.2 How to study this book

Treat every exercise as a controlled classification experiment. Before executing the code, predict the tensor shape, whether the value is a logit or probability, the expected probability range,

the target representation, and the effect of the chosen threshold. After execution, verify these expectations numerically. Never use the words logit, probability, confidence, and class label as if they describe the same quantity.

Begin with small CPU tensors whose sigmoid values, BCE terms, confusion counts, and gradients can be checked manually. After the result is understood, repeat selected experiments on CUDA and compare them using justified tolerances. When testing precision, mixed precision, or extreme logits, record finite status and gradient behavior instead of accepting a result merely because no exception was raised.

When comparing thresholds, class weights, learning rates, or regularization, preserve the dataset split, initialization, seed, training budget, and evaluation examples. Change one intended variable at a time. Do not select a threshold using the final test set, and do not allow preprocessing statistics from validation or test data to enter training. These mistakes create data leakage and make the reported performance unreliable.

For every completed challenge, answer four questions: What quantity was computed? What mathematical rule produced it? What evidence shows that it is correct? Under what condition could it become misleading or fail? If you cannot explain why a model produced a class decision, how the threshold affected the error types, or why the loss changed the parameters, repeat the experiment before continuing.

11.1.3 The Chapters (LG00 - LG08) included in this book

- LG00 - Introduction to Logistic Regression
- LG01 - Binary Output and Decision Logic
- LG02 - Sigmoid Activation Behavior
- LG03 - Probability Interpretation Framework
- LG04 - Classification Boundary Definition
- LG05 - Loss Representation for Classification
- LG06 - Model Adjustment and Gradient Direction
- LG07 - Accuracy Validation and Output Analysis
- LG08 - Multi-Class Extension Concepts

11.2 LG00 - Introduction to Logistic Regression

Models and Training

11.2.1 Exercise 00 - Define Binary Classification Problem

Part 1 - Why this matters

Binary classification begins with a precise definition of the two outcomes a model must distinguish. Inputs may contain several numerical features, but every target must represent either the negative class or the positive class according to a documented meaning.

A model cannot be evaluated correctly when labels are ambiguous, shapes are inconsistent, or class values silently use an unexpected encoding. This exercise establishes the data contract before any sigmoid function or training operation is introduced.

Part 2 - Mathematical form

Let the feature matrix and binary targets be:

$$\mathbf{X} \in \mathbb{R}^{N \times F}, \qquad \mathbf{y} \in \{0, 1\}^{N \times 1}$$

where N is the number of samples and F is the number of input features. The value 0 represents the negative class and 1 represents the positive class.

Part 3 - Hints and Helper

Create a small floating-point feature tensor and a target tensor containing both classes. Validate dimensions, sample count, finite inputs, target values, and class counts. Do not build the model yet.

Part 4 - Python code skeleton

```
import torch

def E00(X, target):
    if X.ndim != 2:
        raise ValueError(...)
    if target.shape != (X.shape[0], 1):
        raise ValueError(...)
    valid_labels = torch.all((target == 0) | (target == 1))
    if not valid_labels:
        raise ValueError(...)
    return {
        "samples": X.shape[0],
        "features": X.shape[1],
        "negative_count": int((target == 0).sum()),
        "positive_count": ...,
        "finite_features": bool(torch.isfinite(X).all()),
    }

def main():
    X = torch.tensor(..., dtype=torch.float32)
    target = torch.tensor(..., dtype=torch.float32)
    print(E00(X, target))
```

```
if __name__ == "__main__":
    main()
```

11.2.2 Exercise 01 - Linear Output vs Probability Output

Part 1 - Why this matters

Logistic regression first produces an unrestricted linear score and then converts it into a bounded probability. Keeping these two quantities separate is essential because losses, thresholds, and interpretation rules may expect one representation but not the other.

A raw score greater than one is not an invalid logit, but it is an invalid probability. Applying a decision threshold directly to logits as though they were probabilities changes the intended decision rule.

Part 2 - Mathematical form

The linear output is the logit:

$$z = \mathbf{w}^\top \mathbf{x} + b, \qquad z \in \mathbb{R}$$

The probability is:

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}, \qquad p \in (0, 1)$$

The sigmoid is monotonic, so larger logits produce larger probabilities.

Part 3 - Hints and Helper

Use known weights and bias to calculate logits, then apply sigmoid. Return both tensors with clear names. Test negative, zero, and positive logits and verify that a zero logit maps to probability 0.5.

Part 4 - Python code skeleton

```
import torch

def E01(X, weight, bias):
    logits = X @ weight + bias
    probabilities = ...
    return {"logits": logits, "probabilities": probabilities}

def main():
    X = torch.tensor(..., dtype=torch.float32)
    weight = torch.tensor(..., dtype=torch.float32)
    bias = torch.tensor(..., dtype=torch.float32)
    result = E01(X, weight, bias)
    print(result)

if __name__ == "__main__":
    main()
```

11.2.3 Exercise 02 - Implement Sigmoid Function Manually

Part 1 - Why this matters

Implementing sigmoid manually connects the probability output to exponentiation and reciprocal operations. It also exposes the numerical weakness of evaluating the same algebraic expression for very large positive and negative logits.

A useful manual implementation should agree with the mathematical definition for ordinary values and remain finite for extreme finite inputs. Comparing a direct formula with a branch-aware version makes numerical stability observable.

Part 2 - Mathematical form

The direct sigmoid formula is:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

An equivalent form for negative z is:

$$\sigma(z) = \frac{e^{z}}{1 + e^{z}}$$

Using separate forms avoids exponentiating a large positive magnitude unnecessarily.

Part 3 - Hints and Helper

Create an output tensor with the same shape and dtype as the input. Apply the direct expression where $z \geq 0$ and the equivalent negative-input expression elsewhere. Compare the result with hand-calculated ordinary cases.

Part 4 - Python code skeleton

```
import torch

def E02(logits):
    logits = torch.as_tensor(logits)
    positive = logits >= 0
    output = torch.empty_like(logits)
    output[positive] = 1 / (1 + torch.exp(-logits[positive]))
    exp_value = torch.exp(logits[~positive])
    output[~positive] = ...
    return output

def main():
    logits = torch.tensor(
        [...], dtype=torch.float64
    )
    print(E02(logits))

if __name__ == "__main__":
    main()
```

11.2.4 Exercise 03 - Sigmoid Using PyTorch

Part 1 - Why this matters

PyTorch provides a vectorized sigmoid implementation that works with autograd, batches, devices, and supported floating-point dtypes. Using the library operation removes manual indexing while preserving the mathematical transformation.

Library use should still be verified rather than accepted blindly. The output must match a manual reference within tolerance, preserve shape, and participate in the computational graph when its input requires gradients.

Part 2 - Technical concept and behavior

For every element z_i of a tensor:

$$p_i = \text{sigmoid}(z_i)$$

The operation is element-wise, so:

$$\text{shape}(\mathbf{p}) = \text{shape}(\mathbf{z})$$

Part 3 - Hints and Helper

Use `torch.sigmoid`. Test a multidimensional tensor and compare it with the manual function from the previous exercise. Set `requires_grad=True` and inspect the output's gradient metadata.

Part 4 - Python code skeleton

```
import torch

def E03(logits):
    probabilities = torch.sigmoid(...)
    return {
        "probabilities": probabilities,
        "shape_preserved": probabilities.shape == logits.shape,
        "requires_grad": probabilities.requires_grad,
        "gradient_function": type(probabilities.grad_fn).__name__
        if probabilities.grad_fn is not None else None,
    }

def main():
    logits = torch.tensor(
        [...], dtype=torch.float32, requires_grad=True
    )
    print(E03(logits))

if __name__ == "__main__":
    main()
```

11.2.5 Exercise 04 - Probability Range Validation

Part 1 - Why this matters

A logistic probability must remain inside the closed numerical range from zero to one. Range validation detects accidental use of raw logits, corrupted values, and non-finite results before they enter thresholding or evaluation code.

Range validity alone does not prove that values are correct probabilities. A constant tensor of 0.5 passes the range check but may still indicate a broken model, so shape and finiteness must be validated as well.

Part 2 - Technical concept and behavior

For each predicted probability:

$$0 \leq p_i \leq 1$$

The validation contract also requires:

$$\text{isfinite}(p_i) = \text{True}$$

and an output shape consistent with the number of classification decisions.

Part 3 - Hints and Helper

Return a structured report containing minimum, maximum, finite status, range status, and expected-shape status. Test valid sigmoid output and deliberately invalid tensors containing logits, NaN, or infinity.

Part 4 - Python code skeleton

```
import torch

def E04(probabilities, expected_shape):
    finite = torch.isfinite(probabilities)
    in_range = (probabilities >= 0) & (probabilities <= 1)
    report = {
        "shape_valid": probabilities.shape == expected_shape,
        "all_finite": bool(finite.all()),
        "all_in_range": bool(in_range.all()),
        "minimum": probabilities.min().item()
        if finite.all() else None,
        "maximum": ...,
    }
    report["valid"] = ...
    return report

def main():
    probabilities = torch.tensor(...)
    print(E04(probabilities, expected_shape=(..., ...)))

if __name__ == "__main__":
    main()
```

11.2.6 Exercise 05 - Single Sample Classification

Part 1 - Why this matters

A single-sample forward pass makes every stage of binary classification visible: feature validation, logit calculation, sigmoid conversion, and threshold decision. It is the smallest complete inference example.

Preserving a batch dimension even for one sample prevents shape rules from changing between single and batched inference. This reduces hidden broadcasting and makes the same model interface reusable.

Part 2 - Mathematical form

For one sample $\mathbf{x} \in \mathbb{R}^{1\times F}$:

$$z = \mathbf{x}\mathbf{w} + b, \qquad p = \sigma(z)$$

For threshold τ:

$$\widehat{y} = \mathbf{1}[p \geq \tau]$$

Part 3 - Hints and Helper

Require shape $(1, F)$ rather than flattening the input silently. Validate the threshold lies between zero and one. Return the logit, probability, class label, and threshold separately.

Part 4 - Python code skeleton

```
import torch

def E05(X, weight, bias, threshold=0.5):
    if X.ndim != 2 or X.shape[0] != 1:
        raise ValueError(...)
    if not 0 <= threshold <= 1:
        raise ValueError(...)
    logit = X @ weight + bias
    probability = torch.sigmoid(logit)
    label = (probability >= threshold).to(torch.int64)
    return {
        "logit": logit.item(),
        "probability": ...,
        "label": ...,
        "threshold": threshold,
    }

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

11.2.7 Exercise 06 - Batch Classification Forward Pass

Part 1 - Why this matters

Batch inference applies one parameter set to many samples simultaneously. Vectorization uses tensor operations efficiently and ensures that the same feature-to-output rule is applied consistently across the dataset.

The main risk is silent shape broadcasting. A target of shape (N) and prediction of shape $(N, 1)$ may combine into an unintended (N, N) result, so output contracts must be explicit.

Part 2 - Mathematical form

For $\mathbf{X} \in \mathbb{R}^{N \times F}$ and $\mathbf{w} \in \mathbb{R}^{F \times 1}$:

$$\mathbf{z} = \mathbf{X}\mathbf{w} + b \in \mathbb{R}^{N \times 1}$$

$$\mathbf{p} = \sigma(\mathbf{z}) \in (0, 1)^{N \times 1}$$

Part 3 - Hints and Helper

Check the feature dimension against the weight's first dimension. Preserve the $(N, 1)$ result, generate integer labels through one threshold, and verify every output shares the expected sample count.

Part 4 - Python code skeleton

```
import torch

def E06(X, weight, bias, threshold=0.5):
    if X.ndim != 2 or weight.shape != (X.shape[1], 1):
        raise ValueError(...)
    logits = X @ weight + bias
    probabilities = torch.sigmoid(logits)
    labels = ...
    expected = (X.shape[0], 1)
    if logits.shape != expected or labels.shape != expected:
        raise RuntimeError(...)
    return logits, probabilities, labels

def main():
    logits, probabilities, labels = E06(...)
    print(logits, probabilities, labels)

if __name__ == "__main__":
    main()
```

11.2.8 Exercise 07 - Decision Threshold Experiment

Part 1 - Why this matters

The decision threshold converts a continuous probability into a discrete action. Lowering it usually predicts more positive cases, while raising it usually predicts fewer positives. The chosen value therefore changes the types of errors a system makes.

Threshold selection must be studied on validation evidence and connected to the cost of false positives and false negatives. Choosing it on the final test set leaks evaluation information into model design.

Part 2 - Mathematical form

For threshold τ:

$$\widehat{y}_i(\tau) = \mathbf{1}[p_i \geq \tau]$$

Compute:

$$TP,\ TN,\ FP,\ FN$$

for each candidate threshold. These counts show how the decision policy changes even though model probabilities remain fixed.

Part 3 - Hints and Helper

Test several ordered thresholds between zero and one. Keep probabilities and targets fixed, construct Boolean masks, and return positive prediction count and confusion counts for every threshold.

Part 4 - Python code skeleton

```
import torch

def E07(probabilities, target, thresholds):
    probabilities = probabilities.reshape(-1)
    target = target.reshape(-1).to(torch.int64)
    if probabilities.shape != target.shape:
        raise ValueError(...)
    results = []
    for threshold in thresholds:
        predicted = (probabilities >= threshold).to(torch.int64)
        tp = int(((predicted == 1) & (target == 1)).sum())
        tn = int(((predicted == 0) & (target == 0)).sum())
        fp = ...
        fn = ...
        results.append({
            "threshold": threshold,
            "predicted_positive": int(predicted.sum()),
            "tp": tp, "tn": tn, "fp": fp, "fn": fn,
        })
```

```
    return results

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

11.2.9 Exercise 08 - Logit Interpretation Test

Part 1 - Why this matters

A logit is not only a raw model score; it is the logarithm of the predicted odds. This interpretation connects a linear transformation to probability without treating sigmoid as an unexplained conversion.

The sign of the logit has direct meaning. A positive logit corresponds to probability above 0.5, zero corresponds to 0.5, and a negative logit corresponds to probability below 0.5.

Part 2 - Mathematical form

For probability p and odds $p/(1-p)$:

$$z = \log\left(\frac{p}{1-p}\right)$$

The inverse transformation is:

$$p = \sigma(z)$$

Therefore, adding Δz multiplies the odds by $e^{\Delta z}$.

Part 3 - Hints and Helper

Choose moderate logits, convert them to probabilities, calculate odds, and reconstruct logits with the natural logarithm. Avoid probabilities numerically equal to zero or one when applying the inverse formula.

Part 4 - Python code skeleton

```
import torch

def E08(logits):
    probabilities = torch.sigmoid(logits)
    odds = probabilities / (1 - probabilities)
    reconstructed = torch.log(odds)
    return {
        "probabilities": probabilities,
        "odds": odds,
        "reconstructed_logits": reconstructed,
        "agreement": torch.allclose(
            logits, reconstructed, atol=..., rtol=...
```

```
        ),
    }

def main():
    logits = torch.tensor([...], dtype=torch.float64)
    print(E08(logits))

if __name__ == "__main__":
    main()
```

11.2.10 Exercise 09 - Binary Output Failure Case

Part 1 - Why this matters

Binary output pipelines can fail without raising an exception. Common examples include thresholding logits as probabilities, applying sigmoid twice, broadcasting mismatched targets, or interpreting class labels as calibrated confidence.

Controlled failure cases teach the reader to diagnose representation errors from evidence. The objective is to identify which contract was violated rather than to force every tensor into a convenient shape.

Part 2 - Technical concept and behavior

A correct binary pipeline follows:

$$\text{features} \longrightarrow \text{logits} \longrightarrow \text{probabilities} \longrightarrow \text{labels}$$

Each transition has a separate valid range, shape, dtype, and interpretation. Reordering or repeating transitions changes the result.

Part 3 - Hints and Helper

Create one correct output and several deliberately incorrect variants. Detect raw logits outside $[0, 1]$, double sigmoid transformation, shape broadcasting risk, invalid thresholds, and non-binary labels. Return a named diagnostic for each case.

Part 4 - Python code skeleton

```
import torch

def inspect_binary_output(logits, probabilities, target, threshold):
    issues = []
    if not torch.isfinite(logits).all():
        issues.append(...)
    if not ((probabilities >= 0) & (probabilities <= 1)).all():
        issues.append("probability_out_of_range")
    if probabilities.shape != target.shape:
        issues.append(...)
    if not 0 <= threshold <= 1:
        issues.append(...)
```

```
    if not torch.all((target == 0) | (target == 1)):
        issues.append(...)
    return issues

def E09(X, weight, bias, target):
    logits = X @ weight + bias
    correct_probability = torch.sigmoid(logits)
    wrong_probability = torch.sigmoid(correct_probability)
    return {
        "correct": inspect_binary_output(
            logits, correct_probability, target, 0.5
        ),
        "double_sigmoid": ...,
        "thresholded_logits": (logits >= 0.5).to(torch.int64),
    }

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

11.2.11 Exercise 10 - Move Logistic Model to CUDA

Part 1 - Why this matters

CUDA execution requires the model parameters and input tensors to reside on the same device. Moving only the data or only the model causes a device mismatch instead of accelerating inference.

Device placement should be explicit and verifiable. A model should be moved before an optimizer is created for training because device conversion can replace parameter objects managed by the optimizer.

Part 2 - Technical concept and behavior

For a CUDA forward pass:

$$d_{\mathbf{X}} = d_{\mathbf{w}} = d_b = \text{CUDA}$$

The linear layer returns CUDA logits, and sigmoid preserves the output device. The numerical meaning remains the same as the CPU model.

Part 3 - Hints and Helper

Check `torch.cuda.is_available()` and return a clear skipped result when necessary. Move the model and input to one selected device, inspect every parameter device, and run inference without gradients.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def E10(model, X, device_index=0):
    if not torch.cuda.is_available():
        return {"skipped": True, "reason": ...}
    device = torch.device(f"cuda:{device_index}")
    model = model.to(device)
    X = X.to(device)
    model.eval()
    with torch.inference_mode():
        logits = model(X)
        probabilities = torch.sigmoid(logits)
    parameter_devices = {
        str(parameter.device) for parameter in model.parameters()
    }
    return {
        "skipped": False,
        "logit_device": str(logits.device),
        "probability_device": ...,
        "parameter_devices": sorted(parameter_devices),
        "probability_shape": tuple(probabilities.shape),
    }

def main():
    model = nn.Linear(..., 1)
    X = torch.randn(..., ...)
    print(E10(model, X))

if __name__ == "__main__":
    main()
```

11.2.12 Exercise 11 - CPU vs GPU Probability Comparison

Part 1 - Why this matters

CPU and GPU implementations should produce numerically close probabilities when they use identical parameters, inputs, dtype, and evaluation state. This comparison detects inconsistent model copies and incorrect device transfers.

Floating-point computations across devices are not guaranteed to be bitwise identical. A valid parity test therefore uses justified absolute and relative tolerances and reports the observed difference.

Part 2 - Technical concept and behavior

For paired predictions:

$$\Delta_{\max} = \max_i |p_i^{\mathrm{CPU}} - p_i^{\mathrm{GPU}}|$$

Parity passes when:

$$|p_i^{\text{CPU}} - p_i^{\text{GPU}}| \leq \text{atol} + \text{rtol}|p_i^{\text{GPU}}|$$

for every element.

Part 3 - Hints and Helper

Deep-copy one initialized model before moving either copy. Use the same CPU input, transfer a copy to CUDA, disable gradients, and bring GPU output back to CPU before comparison.

Part 4 - Python code skeleton

```
import copy
import torch

def E11(model, X, atol=1e-6, rtol=1e-5):
    if not torch.cuda.is_available():
        return {"skipped": True}
    cpu_model = copy.deepcopy(model).cpu().eval()
    gpu_model = copy.deepcopy(model).cuda().eval()
    with torch.inference_mode():
        cpu_probability = torch.sigmoid(cpu_model(X.cpu()))
        gpu_probability = torch.sigmoid(
            gpu_model(X.cuda())
        ).cpu()
    difference = (cpu_probability - gpu_probability).abs()
    return {
        "skipped": False,
        "allclose": torch.allclose(
            cpu_probability, gpu_probability,
            atol=atol, rtol=rtol
        ),
        "maximum_absolute_difference": ...,
        "mean_absolute_difference": ...,
    }

def main():
    print(E11(...))

if __name__ == "__main__":
    main()
```

11.2.13 Exercise 12 - Precision Effects on Sigmoid

Part 1 - Why this matters

Floating-point precision affects how closely sigmoid probabilities can represent values near zero, one, and the center of the curve. Extreme logits may round to exact boundary values earlier in lower precision.

A precision experiment should distinguish expected rounding from a broken implementation. Float64 offers a higher-precision reference for comparison, but it is still finite-precision arithmetic rather than an exact result.

Part 2 - Technical concept and behavior

For dtype d, compute:

$$p_d = \sigma(z_d)$$

and compare with float64:

$$\Delta_d = |p_d - p_{64}|$$

The largest differences are not necessarily at the largest logits because saturated outputs may round to the same boundary.

Part 3 - Hints and Helper

Use one float64 source tensor and cast it separately to supported dtypes. Convert results back to float64 for comparison. Record finite status, exact zeros, exact ones, and maximum deviation.

Part 4 - Python code skeleton

```
import torch

def E12(values, dtypes):
    source = torch.as_tensor(values, dtype=torch.float64)
    reference = torch.sigmoid(source)
    report = {}
    for dtype in dtypes:
        current = torch.sigmoid(source.to(dtype)).to(torch.float64)
        difference = (current - reference).abs()
        report[str(dtype)] = {
            "probabilities": current.tolist(),
            "all_finite": bool(torch.isfinite(current).all()),
            "exact_zero_count": int((current == 0).sum()),
            "exact_one_count": int((current == 1).sum()),
            "maximum_difference": ...,
        }
    return report

def main():
    values = [...]
    print(E12(values, [torch.float32, torch.float64]))

if __name__ == "__main__":
    main()
```

11.2.14 Exercise 13 - Saturation Behavior Observation

Part 1 - Why this matters

Sigmoid saturation occurs when extreme logits produce probabilities very close to zero or one. In these regions, changes in the logit produce only small changes in probability.

Saturation helps create confident outputs but also reduces the local sigmoid derivative. Observing both probability and derivative prevents confidence from being confused with strong gradient flow.

Part 2 - Mathematical form

The derivative is:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Its maximum is 0.25 at $z = 0$, and it approaches zero as z moves toward positive or negative infinity.

Part 3 - Hints and Helper

Evaluate a symmetric logit range. Calculate probabilities and analytical derivatives, then identify entries below a chosen derivative tolerance. Verify the derivative maximum occurs near zero.

Part 4 - Python code skeleton

```
import torch

def E13(logits, derivative_tolerance=1e-4):
    probabilities = torch.sigmoid(logits)
    derivatives = probabilities * (1 - probabilities)
    saturated = derivatives < derivative_tolerance
    maximum_index = int(derivatives.argmax())
    return {
        "probabilities": probabilities,
        "derivatives": derivatives,
        "saturated_mask": saturated,
        "saturated_count": int(saturated.sum()),
        "maximum_derivative": ...,
        "logit_at_maximum": logits[maximum_index].item(),
    }

def main():
    logits = torch.linspace(..., ..., steps=...)
    print(E13(logits))

if __name__ == "__main__":
    main()
```

11.2.15 Exercise 14 - Numerical Stability Breakdown

Part 1 - Why this matters

A direct sigmoid followed by logarithmic binary cross-entropy may overflow, round probabilities to zero or one, or evaluate an undefined logarithm for extreme logits. The mathematical model remains valid while the computational route becomes unstable.

Stable loss functions operate on logits and combine algebraic steps to avoid dangerous intermediate values. This exercise demonstrates why selecting the correct PyTorch interface is part of mathematical correctness.

Part 2 - Mathematical form

The probability-based binary loss is:

$$\ell = -\left[y \log(\sigma(z)) + (1 - y) \log(1 - \sigma(z))\right]$$

A stable logit form can be written as:

$$\ell = \max(z, 0) - zy + \log(1 + e^{-|z|})$$

which avoids directly taking the logarithm of a rounded boundary probability.

Part 3 - Hints and Helper

Use extreme positive and negative logits with both target classes. Compare a naive manual calculation, the stable formula, and `binary_cross_entropy_with_logits`. Record finite status and numerical agreement.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E14(logits, target):
    probability = 1 / (1 + torch.exp(-logits))
    naive = -(
        target * torch.log(probability)
        + (1 - target) * torch.log(1 - probability)
    )
    stable_manual = (
        torch.clamp(logits, min=0)
        - logits * target
        + torch.log1p(torch.exp(-logits.abs()))
    )
    pytorch_stable = F.binary_cross_entropy_with_logits(
        logits, target, reduction="none"
    )
    return {
        "naive": naive,
        "stable_manual": stable_manual,
```

```
        "pytorch_stable": pytorch_stable,
        "naive_finite": bool(torch.isfinite(naive).all()),
        "stable_agreement": torch.allclose(
            stable_manual, pytorch_stable, atol=..., rtol=...
        ),
    }

def main():
    print(E14(...))

if __name__ == "__main__":
    main()
```

11.2.16 Exercise 15 - Vectorized Binary Prediction

Part 1 - Why this matters

Vectorized prediction processes an entire tensor through matrix multiplication, sigmoid, and thresholding without a Python loop over samples. This produces simpler shape reasoning and can use optimized tensor kernels.

Vectorization must preserve the same result as an independently checked sample-wise reference. Faster code is not useful when a broadcasting error changes which features or samples are combined.

Part 2 - Mathematical form

For a batch:

$$\hat{\mathbf{y}} = \mathbf{1}\left[\sigma(\mathbf{X}\mathbf{w} + b) \geq \tau\right]$$

All operations act on the batch while preserving output shape $(N, 1)$.

Part 3 - Hints and Helper

Validate $\mathbf{X}$ and $\mathbf{w}$ dimensions, use one matrix multiplication, and avoid iteration over samples. Compare several output rows against manual scalar calculations.

Part 4 - Python code skeleton

```
import torch

def E15(X, weight, bias, threshold=0.5):
    if X.ndim != 2 or weight.shape != (X.shape[1], 1):
        raise ValueError(...)
    logits = ...
    probabilities = ...
    labels = ...
    return {
        "logits": logits,
        "probabilities": probabilities,
```

```
        "labels": labels,
        "batch_size": X.shape[0],
    }

def main():
    result = E15(...)
    print(result)

if __name__ == "__main__":
    main()
```

11.2.17 Exercise 16 - Probability Calibration Check

Part 1 - Why this matters

Calibration checks whether predicted probabilities match observed outcome frequencies. If predictions near 0.8 are well calibrated, roughly eighty percent of corresponding targets should be positive over an appropriate sample.

Calibration is different from classification accuracy. A model can rank examples well or classify many correctly while producing probabilities that are systematically too confident or too cautious.

Part 2 - Mathematical form

For probability bin B_m, define confidence and empirical accuracy:

$$\operatorname{conf}(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} p_i$$

$$\operatorname{acc}(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} y_i$$

A simple expected calibration error weights their absolute gaps by bin size.

Part 3 - Hints and Helper

Flatten aligned tensors, create fixed bin edges from zero to one, include probability one in the final bin, and skip empty bins. Return count, confidence, accuracy, gap, and total weighted error.

Part 4 - Python code skeleton

```
import torch

def E16(probabilities, target, bin_count=10):
    probabilities = probabilities.detach().reshape(-1)
    target = target.detach().reshape(-1).to(probabilities.dtype)
    if probabilities.shape != target.shape:
        raise ValueError(...)
    edges = torch.linspace(
        0, 1, bin_count + 1, device=probabilities.device
```

```
    )
    bins = []
    expected_error = 0.0
    total = probabilities.numel()
    for index in range(bin_count):
        lower, upper = edges[index], edges[index + 1]
        mask = (probabilities >= lower) & (probabilities < upper)
        if index == bin_count - 1:
            mask = mask | (probabilities == upper)
        count = int(mask.sum())
        if count == 0:
            continue
        confidence = probabilities[mask].mean().item()
        accuracy = ...
        gap = abs(confidence - accuracy)
        expected_error += (count / total) * gap
        bins.append({"count": count, "confidence": confidence,
                     "accuracy": accuracy, "gap": gap})
    return {"bins": bins, "expected_calibration_error": ...}

def main():
    print(E16(...))

if __name__ == "__main__":
    main()
```

11.2.18 Exercise 17 - Frozen Logistic Model Test

Part 1 - Why this matters

Freezing a logistic model prevents its parameters from receiving gradients during a backward pass. This is useful when a trained classifier is reused as a fixed component or when only selected parts of a larger system should learn.

Freezing parameters is different from evaluation mode and from disabling every gradient. Input gradients may still be required for analysis even while weight and bias gradients remain absent.

Part 2 - Technical concept and behavior

For frozen parameters $\boldsymbol{\theta}$:

$$\texttt{requires_grad}(\boldsymbol{\theta}) = \text{False}$$

A backward pass may still calculate:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{X}}$$

when the input explicitly requires gradients.

Part 3 - Hints and Helper

Clone the state dictionary, call `requires_grad_(False)`, and use an input with gradient tracking enabled. Backpropagate a stable logit loss, then verify parameter gradients are absent, input gradients exist, and parameter values are unchanged.

Part 4 - Python code skeleton

```
import copy
import torch
import torch.nn.functional as F

def E17(model, X, target):
    before = copy.deepcopy(model.state_dict())
    model.requires_grad_(False)
    X = X.detach().clone().requires_grad_(True)
    logits = model(X)
    loss = F.binary_cross_entropy_with_logits(logits, target)
    loss.backward()
    parameter_gradients_absent = all(
        parameter.grad is None for parameter in model.parameters()
    )
    unchanged = all(
        torch.equal(before[name], model.state_dict()[name])
        for name in before
    )
    return {
        "loss": loss.item(),
        "parameter_gradients_absent": ...,
        "input_gradient_present": X.grad is not None,
        "parameters_unchanged": unchanged,
    }

def main():
    print(E17(...))

if __name__ == "__main__":
    main()
```

11.2.19 Exercise 18 - Manual Forward without nn.Module

Part 1 - Why this matters

A manual logistic forward pass proves that the model consists of ordinary tensor operations rather than hidden framework behavior. It also makes parameter shapes and broadcasting rules directly observable.

Using tensors with gradient tracking preserves the ability to train manually later. The exercise separates parameter registration provided by `nn.Module` from the mathematics required to produce logits and probabilities.

Part 2 - Mathematical form

For batch input:

$$\mathbf{z} = \mathbf{X}\mathbf{w} + b$$

$$\mathbf{p} = \sigma(\mathbf{z})$$

where $\mathbf{X} \in \mathbb{R}^{N \times F}$, $\mathbf{w} \in \mathbb{R}^{F \times 1}$, and $b \in \mathbb{R}$.

Part 3 - Hints and Helper

Create weight and bias tensors with `requires_grad=True`. Validate all shapes before multiplication, return logits and probabilities separately, and compare with an `nn.Linear` layer holding copied values.

Part 4 - Python code skeleton

```
import torch

def E18(X, weight, bias):
    if X.ndim != 2:
        raise ValueError(...)
    if weight.shape != (X.shape[1], 1):
        raise ValueError(...)
    if bias.numel() != 1:
        raise ValueError(...)
    logits = ...
    probabilities = ...
    return logits, probabilities

def main():
    X = torch.randn(..., ...)
    weight = torch.randn(..., 1, requires_grad=True)
    bias = torch.zeros(1, requires_grad=True)
    logits, probabilities = E18(X, weight, bias)
    print(logits, probabilities)

if __name__ == "__main__":
    main()
```

11.2.20 Exercise 19 - Forward Pass Failure Debugging

Part 1 - Why this matters

A logistic forward pass can fail through feature mismatch, device mismatch, integer input, invalid values, incorrect output shape, double sigmoid, or confusion between logits and probabilities. Several of these errors can produce plausible outputs instead of exceptions.

A reliable debugger checks the input and representation contracts in a fixed order. The final exercise combines chapter concepts into one report that identifies the first broken invariant without hiding the original evidence.

Part 2 - Technical concept and behavior

For a model with F input features, require:

$$X_{\text{last}} = F, \qquad d_X = d_\theta, \qquad \tau_X = \tau_\theta$$

After forward calculation, logits must be finite, probability shape must match logit shape, and probabilities must lie in $[0, 1]$.

Part 3 - Hints and Helper

Inspect shape, dtype, device, finiteness, and expected feature count before calling the model. Catch errors only around each controlled test case. Include valid, wrong-shape, wrong-dtype, non-finite, and device-mismatch inputs when CUDA is available.

Part 4 - Python code skeleton

```
import torch

def inspect_contract(model, X):
    parameter = next(model.parameters())
    expected_features = model.in_features
    return {
        "shape_valid": X.ndim >= 2
        and X.shape[-1] == expected_features,
        "floating_input": X.is_floating_point(),
        "device_valid": X.device == parameter.device,
        "dtype_valid": X.dtype == parameter.dtype,
        "finite_input": bool(torch.isfinite(X).all())
        if X.is_floating_point() else False,
    }

def E19(model, test_inputs):
    report = []
    for name, X in test_inputs.items():
        checks = inspect_contract(model, X)
        try:
            logits = model(X)
            probabilities = torch.sigmoid(logits)
            status = {
                "success": True,
                "logit_shape": tuple(logits.shape),
                "probability_valid": ...,
            }
        except (RuntimeError, ValueError, TypeError) as error:
            status = {"success": False,
                      "error_type": type(error).__name__,
```

```
                        "message": str(error)}
        report.append({"case": name, "checks": checks,
                       "status": status})
    return report

def main():
    print(E19(...))

if __name__ == "__main__":
    main()
```

11.3 LG01 - Binary Output and Decision Logic

Models and Training

11.3.1 Exercise 00 - Convert Probability to Class Label

Part 1 - Why this matters

A logistic model produces probabilities, but many systems must eventually make a discrete decision. Thresholding defines the exact rule that converts the model's continuous output into a negative or positive class label.

The conversion must preserve shape, use a documented comparison rule, and return an appropriate label dtype. Confusing a probability with its final class removes information and makes later threshold analysis impossible.

Part 2 - Mathematical form

For probability p_i and threshold τ:

$$\widehat{y}_i = \begin{cases} 1, & p_i \geq \tau, \\ 0, & p_i < \tau \end{cases}$$

where $0 \leq \tau \leq 1$. The equality rule at the threshold must be stated explicitly.

Part 3 - Hints and Helper

Validate finite probabilities inside $[0, 1]$ and a valid threshold. Use one vectorized comparison, convert Boolean results to `torch.int64`, and confirm the output shape matches the input.

Part 4 - Python code skeleton

```
import torch

def E00(probabilities, threshold=0.5):
    if not 0 <= threshold <= 1:
        raise ValueError(...)
    if not torch.isfinite(probabilities).all():
```

```
        raise ValueError(...)
    if not ((probabilities >= 0) & (probabilities <= 1)).all():
        raise ValueError(...)
    labels = ...
    return {
        "labels": labels,
        "shape_preserved": labels.shape == probabilities.shape,
        "positive_count": int(labels.sum()),
        "threshold": threshold,
    }

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

11.3.2 Exercise 01 - Threshold Sensitivity Experiment

Part 1 - Why this matters

A threshold changes decisions without retraining the probability model. Lower thresholds generally increase positive predictions, while higher thresholds generally decrease them and alter the balance of classification errors.

Sensitivity analysis shows whether a model's reported performance depends strongly on one arbitrary cutoff. It also reveals ranges where small threshold changes cause large operational changes.

Part 2 - Mathematical form

For thresholds $\tau_1, \ldots, \tau_K$, calculate:

$$\hat{\mathbf{y}}(\tau_k) = \mathbf{1}[\mathbf{p} \geq \tau_k]$$

and measure the positive prediction rate:

$$R_+(\tau_k) = \frac{1}{N} \sum_{i=1}^{N} \hat{y}_i(\tau_k)$$

Part 3 - Hints and Helper

Sort and validate thresholds, keep probabilities fixed, and record positive count, accuracy, false positives, and false negatives at each value. Verify positive count never increases as the threshold rises.

Part 4 - Python code skeleton

```
import torch

def E01(probabilities, target, thresholds):
```

```
    probabilities = probabilities.reshape(-1)
    target = target.reshape(-1).to(torch.int64)
    results = []
    previous_positive = probabilities.numel() + 1
    for threshold in sorted(thresholds):
        predicted = (probabilities >= threshold).to(torch.int64)
        positive_count = int(predicted.sum())
        if positive_count > previous_positive:
            raise RuntimeError(...)
        previous_positive = positive_count
        fp = int(((predicted == 1) & (target == 0)).sum())
        fn = ...
        accuracy = (predicted == target).float().mean().item()
        results.append({
            "threshold": threshold,
            "positive_count": positive_count,
            "accuracy": accuracy, "fp": fp, "fn": fn,
        })
    return results

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

11.3.3 Exercise 02 - Confusion Matrix Construction

Part 1 - Why this matters

A confusion matrix separates correct and incorrect predictions by class. Unlike one accuracy value, it shows whether the classifier succeeds through true positive and true negative decisions or hides a serious one-sided failure.

Its row and column convention must be documented. Swapping actual and predicted axes preserves the four counts but can reverse interpretation when the matrix is read later.

Part 2 - Mathematical form

Using rows for actual class and columns for predicted class:

$$\mathbf{C} = \begin{bmatrix} TN & FP \\ FN & TP \end{bmatrix}$$

The invariant is:

$$TN + FP + FN + TP = N$$

Part 3 - Hints and Helper

Flatten aligned binary tensors, compute the four counts with Boolean masks, construct an integer matrix, and verify its sum equals the number of evaluated samples.

Part 4 - Python code skeleton

```
import torch

def E02(predicted, target):
    predicted = predicted.reshape(-1).to(torch.int64)
    target = target.reshape(-1).to(torch.int64)
    if predicted.shape != target.shape:
        raise ValueError(...)
    tn = ((target == 0) & (predicted == 0)).sum()
    fp = ((target == 0) & (predicted == 1)).sum()
    fn = ...
    tp = ...
    matrix = torch.tensor([[tn, fp], [fn, tp]], dtype=torch.int64)
    if int(matrix.sum()) != target.numel():
        raise RuntimeError(...)
    return {"matrix": matrix, "tn": int(tn), "fp": int(fp),
            "fn": int(fn), "tp": int(tp)}

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

11.3.4 Exercise 03 - False Positive / False Negative Analysis

Part 1 - Why this matters

False positives and false negatives represent different real-world failures. Their consequences may differ sharply, so a decision rule should not be selected from total error count alone.

Analyzing the two errors separately connects model evaluation to system requirements. The preferred threshold depends on their costs, not on a universal claim that one error type is always worse.

Part 2 - Mathematical form

Define rates:

$$\text{FPR} = \frac{FP}{FP + TN}$$

$$\text{FNR} = \frac{FN}{FN + TP}$$

For specified costs c_{FP} and c_{FN}:

$$C = c_{FP}FP + c_{FN}FN$$

Part 3 - Hints and Helper

Accept confusion counts and explicit costs. Guard denominators when a class is absent. Return rates, weighted cost, and the indices of false-positive and false-negative samples.

Part 4 - Python code skeleton

```
import torch

def safe_divide(numerator, denominator):
    return numerator / denominator if denominator > 0 else None

def E03(probabilities, target, threshold, fp_cost, fn_cost):
    predicted = (probabilities.reshape(-1) >= threshold).long()
    target = target.reshape(-1).long()
    fp_mask = (predicted == 1) & (target == 0)
    fn_mask = (predicted == 0) & (target == 1)
    tn = int(((predicted == 0) & (target == 0)).sum())
    tp = int(((predicted == 1) & (target == 1)).sum())
    fp, fn = int(fp_mask.sum()), int(fn_mask.sum())
    return {
        "false_positive_rate": safe_divide(fp, fp + tn),
        "false_negative_rate": ...,
        "weighted_cost": ...,
        "false_positive_indices": torch.where(fp_mask)[0],
        "false_negative_indices": ...,
    }

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

11.3.5 Exercise 04 - Imbalanced Dataset Effect

Part 1 - Why this matters

When one class dominates a dataset, a classifier can achieve high accuracy by predicting only the majority class. This produces a reassuring number while completely failing the minority outcome the system may care about most.

The class distribution must therefore be reported beside every classification metric. A majority-class baseline provides a minimum reference that a learned decision system should meaningfully exceed.

Part 2 - Mathematical form

If class counts are N_0 and N_1, majority baseline accuracy is:

$$A_{\text{majority}} = \frac{\max(N_0, N_1)}{N_0 + N_1}$$

Balanced accuracy gives each class equal influence:

$$A_{\text{balanced}} = \frac{1}{2}\left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP}\right)$$

Part 3 - Hints and Helper

Construct imbalanced targets and compare an all-majority predictor with a real prediction tensor. Report class prevalence, ordinary accuracy, balanced accuracy, and minority recall. Handle absent classes explicitly.

Part 4 - Python code skeleton

```
import torch

def E04(predicted, target):
    predicted = predicted.reshape(-1).long()
    target = target.reshape(-1).long()
    negative = int((target == 0).sum())
    positive = int((target == 1).sum())
    accuracy = (predicted == target).float().mean().item()
    tp = int(((predicted == 1) & (target == 1)).sum())
    tn = int(((predicted == 0) & (target == 0)).sum())
    recall_positive = tp / positive if positive else None
    recall_negative = tn / negative if negative else None
    balanced = ... if None not in (
        recall_positive, recall_negative
    ) else None
    majority_accuracy = max(negative, positive) / target.numel()
    return {"class_counts": [negative, positive],
            "accuracy": accuracy,
            "balanced_accuracy": balanced,
            "majority_baseline_accuracy": majority_accuracy}

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

11.3.6 Exercise 05 - Dynamic Threshold Selection

Part 1 - Why this matters

A dynamic threshold is selected from validation evidence to satisfy a defined objective, such as maximizing F1 score or minimizing a weighted error cost. It adapts decision logic to the

system's priorities without changing model parameters.

Threshold search must not use the final test set. Reusing test labels to choose the best cutoff makes the reported test result optimistic because the decision policy has already been tuned to those examples.

Part 2 - Mathematical form

For validation thresholds $\mathcal{T}$ and objective J:

$$\tau^\star = \operatorname*{argmax}_{\tau \in \mathcal{T}} J(\tau)$$

A deterministic tie rule, such as choosing the largest threshold, must be documented.

Part 3 - Hints and Helper

Evaluate each candidate on validation probabilities only. Calculate the requested objective, preserve every result, and resolve ties explicitly. Return both selected threshold and selection evidence.

Part 4 - Python code skeleton

```
import torch

def f1_score(predicted, target):
    tp = ((predicted == 1) & (target == 1)).sum()
    fp = ((predicted == 1) & (target == 0)).sum()
    fn = ((predicted == 0) & (target == 1)).sum()
    denominator = 2 * tp + fp + fn
    return (2 * tp / denominator).item() if denominator else 0.0

def E05(validation_probabilities, validation_target, candidates):
    probability = validation_probabilities.reshape(-1)
    target = validation_target.reshape(-1).long()
    records = []
    for threshold in sorted(candidates):
        predicted = (probability >= threshold).long()
        score = ...
        records.append({"threshold": threshold, "score": score})
    best_score = max(record["score"] for record in records)
    tied = [r for r in records if r["score"] == best_score]
    selected = max(tied, key=lambda record: record["threshold"])
    return {"selected_threshold": ..., "records": records}

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

11.3.7 Exercise 06 - ROC Curve Concept Implementation

Part 1 - Why this matters

The receiver operating characteristic curve describes classification behavior across many thresholds. It separates the model's ranking ability from one chosen operating point by comparing true-positive and false-positive rates.

ROC construction requires both classes. If every target is identical, one rate has a zero denominator and the curve is not defined for ordinary binary evaluation.

Part 2 - Mathematical form

For each threshold τ:

$$\mathrm{TPR}(\tau) = \frac{TP(\tau)}{TP(\tau) + FN(\tau)}$$

$$\mathrm{FPR}(\tau) = \frac{FP(\tau)}{FP(\tau) + TN(\tau)}$$

Thresholds descend from above the maximum score to below the minimum score, tracing the curve from $(0, 0)$ to $(1, 1)$.

Part 3 - Hints and Helper

Use unique probability values in descending order and include boundary thresholds. At each threshold, calculate labels and both rates. Verify FPR and TPR are nondecreasing along the returned curve.

Part 4 - Python code skeleton

```
import torch

def E06(probabilities, target):
    probability = probabilities.reshape(-1)
    target = target.reshape(-1).long()
    positive = int((target == 1).sum())
    negative = int((target == 0).sum())
    if positive == 0 or negative == 0:
        raise ValueError(...)
    unique = torch.unique(probability).sort(descending=True).values
    thresholds = torch.cat([
        torch.tensor([torch.inf], device=probability.device),
        unique,
        torch.tensor([-torch.inf], device=probability.device),
    ])
    points = []
    for threshold in thresholds:
        predicted = probability >= threshold
        tp = int((predicted & (target == 1)).sum())
        fp = int((predicted & (target == 0)).sum())
```

```
        points.append({
            "threshold": float(threshold),
            "tpr": tp / positive,
            "fpr": ...,
        })
    return points

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

11.3.8 Exercise 07 - AUC Metric Calculation

Part 1 - Why this matters

Area under the ROC curve summarizes the full ROC path into one ranking metric. It measures how well positive examples tend to receive higher scores than negative examples across thresholds.

AUC does not choose a threshold, measure calibration, or reveal the cost of a specific false decision. It should be reported with the curve and class context rather than used as a complete evaluation by itself.

Part 2 - Mathematical form

For ROC points sorted by increasing false-positive rate, trapezoidal area is:

$$\text{AUC} = \sum_{k=1}^{K-1} \frac{TPR_k + TPR_{k+1}}{2}(FPR_{k+1} - FPR_k)$$

A valid binary ROC AUC lies between zero and one.

Part 3 - Hints and Helper

Accept ROC points from the previous exercise and sort them by FPR. Calculate trapezoidal area with PyTorch, then check monotonic coordinates, endpoints, finiteness, and final range.

Part 4 - Python code skeleton

```
import torch

def E07(roc_points):
    ordered = sorted(roc_points, key=lambda point: point["fpr"])
    fpr = torch.tensor(
        [point["fpr"] for point in ordered], dtype=torch.float64
    )
    tpr = torch.tensor(
        [point["tpr"] for point in ordered], dtype=torch.float64
    )
```

```
    if not torch.all(fpr[1:] >= fpr[:-1]):
        raise ValueError(...)
    auc = torch.trapezoid(tpr, fpr).item()
    if not 0 <= auc <= 1:
        raise RuntimeError(...)
    return {"auc": auc, "point_count": len(ordered),
            "starts_at_origin": ..., "ends_at_one": ...}

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

11.3.9 Exercise 08 - Precision vs Recall Tradeoff

Part 1 - Why this matters

Precision asks how many predicted positives are correct, while recall asks how many actual positives were found. Changing the threshold often improves one while weakening the other.

The preferred balance depends on application costs and class prevalence. A threshold should not be declared best without stating whether missed positives or false alarms are more damaging.

Part 2 - Mathematical form

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

Their harmonic mean is:

$$F_1 = \frac{2PR}{P + R}$$

when the required denominators are defined.

Part 3 - Hints and Helper

Calculate precision, recall, and F1 at several thresholds. Return `None` or a documented fallback when no positives are predicted. Show the threshold that maximizes F1 without claiming it is universally optimal.

Part 4 - Python code skeleton

```
import torch

def E08(probabilities, target, thresholds):
    probability = probabilities.reshape(-1)
```

```
    target = target.reshape(-1).long()
    records = []
    for threshold in thresholds:
        predicted = (probability >= threshold).long()
        tp = int(((predicted == 1) & (target == 1)).sum())
        fp = int(((predicted == 1) & (target == 0)).sum())
        fn = int(((predicted == 0) & (target == 1)).sum())
        precision = tp / (tp + fp) if tp + fp else None
        recall = tp / (tp + fn) if tp + fn else None
        metrics_defined = precision is not None and recall is not None
        f1 = ... if metrics_defined else None
        records.append({"threshold": threshold,
                        "precision": precision,
                        "recall": recall, "f1": f1})
    return records

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

11.3.10 Exercise 09 - Classification Failure Case

Part 1 - Why this matters

Classification logic can produce misleading results through invalid probabilities, mismatched sample order, incorrect target encoding, or an unsuitable threshold. Some failures preserve shape and avoid exceptions, making validation essential.

A controlled failure experiment teaches the reader to distinguish a model error from a decision-logic error. The same probabilities can generate different labels, while corrupted alignment makes every derived metric unreliable.

Part 2 - Technical concept and behavior

Before evaluation require:

$$\text{shape}(\mathbf{p}) = \text{shape}(\mathbf{y})$$

$$\mathbf{p} \in [0,1]^N, \qquad \mathbf{y} \in \{0,1\}^N$$

and a threshold in $[0, 1]$. Sample identifiers should also remain aligned after filtering or sorting.

Part 3 - Hints and Helper

Build a validator that reports every failed contract. Test logits passed as probabilities, shuffled targets, invalid labels, NaN values, and thresholds outside the allowed range.

Part 4 - Python code skeleton

```
import torch

def E09(probabilities, target, threshold):
    issues = []
    if probabilities.shape != target.shape:
        issues.append("shape_mismatch")
    if not probabilities.is_floating_point():
        issues.append("probability_dtype")
    elif not torch.isfinite(probabilities).all():
        issues.append("nonfinite_probability")
    elif not ((probabilities >= 0) & (probabilities <= 1)).all():
        issues.append(...)
    if not torch.all((target == 0) | (target == 1)):
        issues.append(...)
    if not 0 <= threshold <= 1:
        issues.append(...)
    return {"valid": len(issues) == 0, "issues": issues}

def main():
    cases = {...}
    for name, values in cases.items():
        print(name, E09(...))

if __name__ == "__main__":
    main()
```

11.3.11 Exercise 10 - CUDA Evaluation without Grad

Part 1 - Why this matters

CUDA evaluation should calculate logits, probabilities, decisions, and global confusion counts without constructing backward graphs. Disabling gradient tracking reduces unnecessary state while preserving the trained parameters.

Evaluation mode and inference mode solve different problems. The model should use both deliberately, even when a simple linear layer happens to produce the same output in training and evaluation modes.

Part 2 - Technical concept and behavior

For every batch, require:

$$d_{\mathbf{X}} = d_{\mathbf{y}} = d_{\boldsymbol{\theta}} = \text{CUDA}$$

Confusion counts can be accumulated as integers and used to calculate dataset-level metrics after every batch has completed.

Part 3 - Hints and Helper

Check CUDA availability, move model and batches to one device, call `eval()`, and use `torch.inference_mode()`. Accumulate counts rather than retaining every computational output on GPU.

Part 4 - Python code skeleton

```
import torch

def E10(model, loader, threshold=0.5):
    if not torch.cuda.is_available():
        return {"skipped": True, "reason": ...}
    device = torch.device("cuda")
    model = model.to(device).eval()
    counts = {"tp": 0, "tn": 0, "fp": 0, "fn": 0}
    mode_was_enabled = False
    with torch.inference_mode():
        mode_was_enabled = torch.is_inference_mode_enabled()
        for X, target in loader:
            X = X.to(device)
            target = target.to(device).reshape(-1).long()
            probability = torch.sigmoid(model(X)).reshape(-1)
            predicted = (probability >= threshold).long()
            counts["tp"] += int(
                ((predicted == 1) & (target == 1)).sum()
            )
            counts["tn"] += ...
            counts["fp"] += ...
            counts["fn"] += ...
    return {"skipped": False, "counts": counts,
            "inference_mode": mode_was_enabled}

def main():
    print(E10(...))

if __name__ == "__main__":
    main()
```

11.3.12 Exercise 11 - Batch-wise Decision Pipeline

Part 1 - Why this matters

A batch-wise decision pipeline evaluates datasets too large to process at once. It must preserve global metric meaning even when the final batch is smaller than the others.

Averaging metric values computed separately per batch can bias the result. Confusion counts should be summed across batches first, then converted into accuracy, precision, recall, and related metrics.

Part 2 - Mathematical form

For batches $b = 1, \ldots, B$:

$$TP_{\text{global}} = \sum_{b=1}^{B} TP_b$$

and similarly for TN, FP, and FN. Global accuracy is:

$$\frac{TP + TN}{TP + TN + FP + FN}$$

Part 3 - Hints and Helper

Validate output and target shape inside every batch. Accumulate integer counts and total sample count, calculate metrics after the loop, and raise a clear error for an empty loader.

Part 4 - Python code skeleton

```
import torch

def E11(model, loader, device, threshold=0.5):
    model.eval()
    tp = tn = fp = fn = 0
    batches = 0
    with torch.inference_mode():
        for X, target in loader:
            batches += 1
            X, target = X.to(device), target.to(device)
            probability = torch.sigmoid(model(X))
            if probability.shape != target.shape:
                raise ValueError(...)
            predicted = (probability >= threshold).long()
            target = target.long()
            tp += int(((predicted == 1) & (target == 1)).sum())
            tn += ...
            fp += ...
            fn += ...
    if batches == 0:
        raise ValueError(...)
    total = tp + tn + fp + fn
    return {"accuracy": (tp + tn) / total,
            "precision": tp / (tp + fp) if tp + fp else None,
            "recall": ..., "counts": [tn, fp, fn, tp]}

def main():
    print(E11(...))

if __name__ == "__main__":
    main()
```

11.3.13 Exercise 12 - Prediction Consistency Test

Part 1 - Why this matters

Repeated evaluation of a fixed model and fixed input should produce consistent probabilities and labels under a controlled environment. Unexpected variation can reveal active stochastic

layers, changed model state, mutation, or nondeterministic execution.

Consistency does not require universal bitwise identity across devices or software versions. The test must define whether it checks one environment exactly or compares environments with numerical tolerances.

Part 2 - Technical concept and behavior

For repeated outputs $\mathbf{p}^{(1)}, \ldots, \mathbf{p}^{(R)}$, define:

$$\Delta_{\max} = \max_{r,i} |p_i^{(r)} - p_i^{(1)}|$$

Labels should also remain equal for a fixed threshold, especially for probabilities near the cutoff.

Part 3 - Hints and Helper

Call evaluation mode, disable gradients, repeat the same forward pass, and preserve outputs for comparison. Report maximum probability difference, label agreement, and minimum distance from the threshold.

Part 4 - Python code skeleton

```
import torch

def E12(model, X, threshold=0.5, repetitions=10,
        atol=1e-7, rtol=1e-6):
    model.eval()
    outputs = []
    with torch.inference_mode():
        for _ in range(repetitions):
            outputs.append(torch.sigmoid(model(X)).detach().clone())
    reference = outputs[0]
    differences = torch.stack(
        [(current - reference).abs() for current in outputs]
    )
    probability_consistent = all(
        torch.allclose(current, reference, atol=atol, rtol=rtol)
        for current in outputs
    )
    labels = [(current >= threshold).long() for current in outputs]
    return {
        "probability_consistent": probability_consistent,
        "labels_consistent": all(torch.equal(x, labels[0])
                                 for x in labels),
        "maximum_difference": ...,
        "minimum_threshold_distance":
            (reference - threshold).abs().min().item(),
    }

def main():
```

```
    print(E12(...))

if __name__ == "__main__":
    main()
```

11.3.14 Exercise 13 - Threshold Drift Detection

Part 1 - Why this matters

A threshold selected for one model or data period may no longer provide the same error balance after probabilities drift. Monitoring threshold behavior can reveal that the decision policy needs investigation even when the code has not changed.

A new threshold should not be adopted automatically from every small fluctuation. Drift detection should use a fixed evaluation protocol, sufficient sample size, and explicit alert tolerances.

Part 2 - Mathematical form

For baseline threshold τ_0 and newly selected threshold τ_t:

$$D_\tau = |\tau_t - \tau_0|$$

Also compare predicted-positive rates:

$$D_R = |R_+(\tau_t) - R_{+,0}|$$

A monitoring alert can require either or both quantities to exceed predefined limits.

Part 3 - Hints and Helper

Use the same threshold-selection objective on baseline and current validation sets. Record class prevalence, selected threshold, score, and positive rate. Do not interpret threshold movement without checking data distribution.

Part 4 - Python code skeleton

```
def E13(baseline, current, threshold_tolerance,
        positive_rate_tolerance):
    threshold_drift = abs(
        current["selected_threshold"]
        - baseline["selected_threshold"]
    )
    rate_drift = abs(
        current["predicted_positive_rate"]
        - baseline["predicted_positive_rate"]
    )
    alerts = {
        "threshold": threshold_drift > threshold_tolerance,
        "positive_rate": rate_drift > positive_rate_tolerance,
    }
    return {
```

```
        "threshold_drift": threshold_drift,
        "positive_rate_drift": rate_drift,
        "alert": ...,
        "alert_reasons": [name for name, active in alerts.items()
                          if active],
        "baseline_prevalence": baseline["prevalence"],
        "current_prevalence": ...,
    }

def main():
    print(E13(...))

if __name__ == "__main__":
    main()
```

11.3.15 Exercise 14 - Metric Stability under Noise

Part 1 - Why this matters

Input noise can shift probabilities across the decision threshold and change classification metrics. A model whose score is stable on clean data may become unreliable when measurements contain realistic perturbations.

One random noise sample is not enough to establish stability. Repeated seeded trials at several noise levels reveal both average degradation and variability.

Part 2 - Technical concept and behavior

For perturbed input:

$$\widetilde{\mathbf{X}} = \mathbf{X} + \sigma\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$$

record metric $M_{\sigma,r}$ across trial r, then summarize its mean and standard deviation.

Part 3 - Hints and Helper

Keep model, targets, threshold, and test examples fixed. Generate noise with controlled seeds, avoid perturbing protected categorical fields, and compare each noise level with the clean baseline.

Part 4 - Python code skeleton

```
import torch

def accuracy(model, X, target, threshold):
    probability = torch.sigmoid(model(X)).reshape(-1)
    predicted = (probability >= threshold).long()
    return (predicted == target.reshape(-1).long()).float().mean()

def E14(model, X, target, levels, trials, threshold, seed):
    model.eval()
```

```
    results = {}
    with torch.inference_mode():
        clean = accuracy(model, X, target, threshold).item()
        for level in levels:
            scores = []
            for trial in range(trials):
                generator = torch.Generator(device=X.device)
                generator.manual_seed(seed + trial)
                noise = torch.randn(
                    X.shape, generator=generator,
                    device=X.device, dtype=X.dtype
                )
                score = accuracy(
                    model, X + level * noise, target, threshold
                )
                scores.append(score)
            values = torch.stack(scores)
            results[level] = {
                "mean_accuracy": ...,
                "std_accuracy": ...,
                "degradation": clean - values.mean().item(),
            }
    return {"clean_accuracy": clean, "noise": results}

def main():
    print(E14(...))

if __name__ == "__main__":
    main()
```

11.3.16 Exercise 15 - Classification Latency Benchmark

Part 1 - Why this matters

Classification latency includes the forward pass, sigmoid transformation, and threshold decision required to produce labels. It matters when the system must respond to individual requests or small batches quickly.

CUDA execution is asynchronous, so ordinary host timing without synchronization can finish before device work completes. A useful benchmark also separates warm-up from measured repetitions.

Part 2 - Mathematical form

For measured times $t_1, \ldots, t_R$, report:

$$\bar{t} = \frac{1}{R} \sum_{r=1}^{R} t_r$$

along with median and tail percentiles such as p_{95}. Batch size and device must accompany the result.

Part 3 - Hints and Helper

Use CUDA timing events for a CUDA tensor and a synchronized high-resolution host timer for CPU. Warm up first, disable gradients, and time the complete decision operation consistently.

Part 4 - Python code skeleton

```
import time
import torch

def decision(model, X, threshold):
    return (torch.sigmoid(model(X)) >= threshold).long()

def E15(model, X, threshold=0.5, warmup=20, repetitions=100):
    model.eval()
    times_ms = []
    with torch.inference_mode():
        for _ in range(warmup):
            decision(model, X, threshold)
        if X.device.type == "cuda":
            torch.cuda.synchronize(X.device)
        for _ in range(repetitions):
            if X.device.type == "cuda":
                start = torch.cuda.Event(enable_timing=True)
                end = torch.cuda.Event(enable_timing=True)
                start.record()
                decision(model, X, threshold)
                end.record()
                torch.cuda.synchronize(X.device)
                times_ms.append(start.elapsed_time(end))
            else:
                start = time.perf_counter()
                decision(model, X, threshold)
                times_ms.append((time.perf_counter() - start) * 1000)
    values = torch.tensor(times_ms)
    return {"mean_ms": ..., "median_ms": ...,
            "p95_ms": torch.quantile(values, 0.95).item()}

def main():
    print(E15(...))

if __name__ == "__main__":
    main()
```

11.3.17 Exercise 16 - Memory Profiling for Inference

Part 1 - Why this matters

Inference memory determines feasible batch size and whether a classifier can share a device with other workloads. The final labels are small, but intermediate logits, probabilities, and model parameters still consume memory.

PyTorch distinguishes memory occupied by live tensors from memory reserved by its caching allocator. Current and peak values should be reported separately instead of inferred from an external display alone.

Part 2 - Technical concept and behavior

For a controlled CUDA region, measure:

$$M_{\text{allocated}}^{\text{peak}} \quad \text{and} \quad M_{\text{reserved}}^{\text{peak}}$$

after resetting peak counters and completing warm-up.

Part 3 - Hints and Helper

Require CUDA, warm up and synchronize, then reset peak memory statistics. Run the complete forward, sigmoid, and threshold operation. Keep the result alive until measurements are recorded.

Part 4 - Python code skeleton

```
import torch

def E16(model, X, threshold=0.5):
    if X.device.type != "cuda":
        raise ValueError(...)
    model.eval()
    with torch.inference_mode():
        model(X)
        torch.cuda.synchronize(X.device)
        baseline = torch.cuda.memory_allocated(X.device)
        torch.cuda.reset_peak_memory_stats(X.device)
        logits = model(X)
        probabilities = torch.sigmoid(logits)
        labels = (probabilities >= threshold).long()
        torch.cuda.synchronize(X.device)
        report = {
            "baseline_allocated_bytes": baseline,
            "current_allocated_bytes":
                torch.cuda.memory_allocated(X.device),
            "peak_allocated_bytes":
                torch.cuda.max_memory_allocated(X.device),
            "peak_reserved_bytes": ...,
            "output_elements": labels.numel(),
        }
    return report

def main():
```

```
    print(E16(...))

if __name__ == "__main__":
    main()
```

11.3.18 Exercise 17 - Misclassification Debugging

Part 1 - Why this matters

Misclassified examples show where the final decision disagrees with the target. Inspecting their probabilities, features, and identifiers can reveal threshold effects, data errors, rare subgroups, or limitations of a linear boundary.

Failure analysis should use a declared selection rule rather than hand-picked examples. High-confidence errors deserve special attention, but a probability far from the threshold is not proof that the target is wrong.

Part 2 - Technical concept and behavior

For misclassification mask:

$$m_i = \mathbf{1}[\widehat{y}_i \neq y_i]$$

Define confidence in the chosen class:

$$c_i = \begin{cases} p_i, & \widehat{y}_i = 1, \\ 1 - p_i, & \widehat{y}_i = 0 \end{cases}$$

Sort misclassified samples by c_i to inspect the strongest incorrect decisions.

Part 3 - Hints and Helper

Preserve sample identifiers and feature rows. Return every misclassified index, error type, probability, threshold distance, and predicted-class confidence. Limit displayed examples without discarding the full count.

Part 4 - Python code skeleton

```
import torch

def E17(X, probabilities, target, sample_ids,
        threshold=0.5, top_k=10):
    probability = probabilities.reshape(-1)
    target = target.reshape(-1).long()
    predicted = (probability >= threshold).long()
    wrong = predicted != target
    confidence = torch.where(
        predicted == 1, probability, 1 - probability
    )
    wrong_indices = torch.where(wrong)[0]
    if wrong_indices.numel() == 0:
```

```
        return {"count": 0, "examples": []}
    scores = confidence[wrong_indices]
    order = torch.argsort(scores, descending=True)
    selected = wrong_indices[order[:top_k]]
    examples = []
    for index in selected.tolist():
        examples.append({
            "sample_id": sample_ids[index],
            "probability": probability[index].item(),
            "target": target[index].item(),
            "predicted": ...,
            "error_type": ...,
            "threshold_distance": ...,
        })
    return {"count": int(wrong.sum()), "examples": examples}

def main():
    print(E17(...))

if __name__ == "__main__":
    main()
```

11.3.19 Exercise 18 - Evaluation Pipeline Automation

Part 1 - Why this matters

An automated pipeline applies the same input validation, model state, threshold, confusion logic, metrics, and performance checks to each classifier version. This reduces manual inconsistency and creates comparable evidence.

The pipeline must preserve intermediate counts and configuration rather than return only a final score. A result cannot be reproduced when its threshold, class distribution, sample count, or model version is missing.

Part 2 - Technical concept and behavior

The evaluation mapping is:

$$(\text{model}, \text{data}, \tau, \text{configuration}) \longrightarrow (\text{probabilities}, \text{decisions}, \text{counts}, \text{metrics}, \text{diagnostics})$$

Core validation failures should stop the run; optional diagnostics may report their own failure status.

Part 3 - Hints and Helper

Create separate functions for validation, inference, confusion counts, metrics, latency, and failure analysis. Return a stable dictionary schema with device, dtype, threshold, sample count, and timestamps.

Part 4 - Python code skeleton

```
from datetime import datetime, timezone
import torch

def E18(model, loader, device, threshold, configuration):
    report = {
        "generated_at": datetime.now(timezone.utc).isoformat(),
        "configuration": configuration,
        "device": str(device),
        "threshold": threshold,
        "status": "running",
    }
    try:
        ...  # validate threshold and dataset
        outputs = ...  # run inference without gradients
        counts = ...  # aggregate globally
        metrics = ...  # derive from global counts
        report.update({
            "counts": counts,
            "metrics": metrics,
            "sample_count": sum(counts.values()),
            "status": "passed",
        })
    except Exception as error:
        report["status"] = "failed"
        report["error"] = {
            "type": type(error).__name__,
            "message": str(error),
        }
    return report

def main():
    print(E18(...))

if __name__ == "__main__":
    main()
```

11.3.20 Exercise 19 - Decision Logic Failure Scenario

Part 1 - Why this matters

Decision logic can fail through invalid thresholds, empty datasets, absent classes, probability corruption, target misalignment, or undefined metric denominators. A robust system must detect these conditions before publishing performance.

The final exercise treats failure handling as part of evaluation design. It should distinguish an invalid run from a valid run whose metric is undefined for a documented reason.

Part 2 - Technical concept and behavior

A valid evaluation requires:

$$N > 0, \qquad 0 \leq \tau \leq 1, \qquad \mathbf{p} \in [0,1]^N, \qquad \mathbf{y} \in \{0,1\}^N$$

Metrics must use explicit zero-denominator policies, and ROC/AUC additionally requires both target classes.

Part 3 - Hints and Helper

Build named scenarios for empty input, one-class target, invalid probability, mismatched length, and out-of-range threshold. Return stage, severity, metric availability, and original error details.

Part 4 - Python code skeleton

```
import torch

def validate_decision_inputs(probability, target, threshold):
    if probability.numel() == 0:
        raise ValueError("empty evaluation")
    if probability.shape != target.shape:
        raise ValueError("shape mismatch")
    if not 0 <= threshold <= 1:
        raise ValueError("invalid threshold")
    if not torch.isfinite(probability).all():
        raise ValueError("nonfinite probability")
    if not ((probability >= 0) & (probability <= 1)).all():
        raise ValueError("probability outside range")
    if not torch.all((target == 0) | (target == 1)):
        raise ValueError("target is not binary")

def E19(scenarios):
    report = []
    for name, scenario in scenarios.items():
        try:
            validate_decision_inputs(...)
            classes = torch.unique(scenario["target"])
            roc_available = classes.numel() == 2
            metrics = ...
            result = {"status": "valid",
                      "roc_available": roc_available,
                      "metrics": metrics}
        except (ValueError, RuntimeError) as error:
            result = {"status": "invalid",
                      "error_type": type(error).__name__,
                      "message": str(error)}
        report.append({"scenario": name, "result": result})
    return report

def main():
    print(E19(...))
```

```
if __name__ == "__main__":
    main()
```

11.4 LG02 - Sigmoid Activation Behavior

Models and Training

11.4.1 Exercise 00 - Plot Sigmoid Curve

Part 1 - Why this matters

The sigmoid curve makes the connection between unrestricted logits and bounded probabilities visible. Its central region responds strongly to input changes, while its tails approach zero and one without crossing those theoretical limits.

A correct plot should communicate both transformation and interpretation. Reference lines at logit zero and probability 0.5 identify the decision center, while a sufficiently wide input range exposes saturation.

Part 2 - Mathematical form

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The function is monotonic and satisfies:

$$\sigma(0) = 0.5, \qquad \lim_{z \to -\infty} \sigma(z) = 0, \qquad \lim_{z \to \infty} \sigma(z) = 1$$

Part 3 - Hints and Helper

Generate an ordered one-dimensional tensor with `torch.linspace`, calculate sigmoid with PyTorch, detach values, and move them to CPU before plotting. Label both axes and include horizontal and vertical reference lines.

Part 4 - Python code skeleton

```
import matplotlib.pyplot as plt
import torch

def E00(start, end, steps, output_path):
    logits = torch.linspace(start, end, steps=steps)
    probabilities = torch.sigmoid(logits)
    fig, ax = plt.subplots()
    ax.plot(logits.numpy(), probabilities.numpy(),
            label="Sigmoid")
    ax.axvline(0.0, color="black", linestyle="--")
    ax.axhline(0.5, color="black", linestyle=":")
    ax.set_xlabel(...)
```

```
    ax.set_ylabel(...)
    ax.set_ylim(...)
    ax.legend()
    fig.savefig(output_path, dpi=150, bbox_inches="tight")
    plt.close(fig)

def main():
    E00(...)

if __name__ == "__main__":
    main()
```

11.4.2 Exercise 01 - Gradient of Sigmoid Inspection

Part 1 - Why this matters

The sigmoid derivative determines how a change in a logit affects the probability and how gradients pass backward through the activation. Inspecting it connects the analytical rule to PyTorch autograd.

Agreement between analytical and automatic gradients provides stronger evidence than printing one result. It also reveals that the largest local sensitivity occurs near the center rather than in the confident tails.

Part 2 - Mathematical form

$$\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z))$$

The derivative is nonnegative and reaches its maximum value 0.25 at $z = 0$.

Part 3 - Hints and Helper

Use a float64 leaf tensor with gradient tracking. Sum sigmoid outputs to create a scalar whose backward gradient contains the element-wise derivatives, then compare with the analytical formula.

Part 4 - Python code skeleton

```
import torch

def E01(logits):
    if not logits.is_leaf or not logits.requires_grad:
        raise ValueError(...)
    probabilities = torch.sigmoid(logits)
    probabilities.sum().backward()
    automatic = logits.grad.detach().clone()
    analytical = (
        probabilities.detach() * (1 - probabilities.detach())
    )
    return {
```

```
        "automatic": automatic,
        "analytical": analytical,
        "agreement": torch.allclose(
            automatic, analytical, atol=..., rtol=...
        ),
        "maximum_gradient": automatic.max().item(),
    }

def main():
    logits = torch.tensor(
        [...], dtype=torch.float64, requires_grad=True
    )
    print(E01(logits))

if __name__ == "__main__":
    main()
```

11.4.3 Exercise 02 - Vanishing Gradient Demonstration

Part 1 - Why this matters

Sigmoid gradients become very small for large-magnitude logits. When many such factors appear in a chain, the gradient reaching early values can shrink until parameter updates become ineffective.

A demonstration should measure gradient magnitude across input regions instead of declaring that every small value is a failure. The scale of the upstream gradient and the chosen dtype also affect what is observed.

Part 2 - Mathematical form

For one sigmoid:

$$0 < \sigma'(z) \leq 0.25$$

For a chain of L derivative factors:

$$\left|\frac{\partial a_L}{\partial a_0}\right| = \prod_{\ell=1}^{L} |\sigma'(z_\ell)|$$

which can decrease rapidly as depth increases.

Part 3 - Hints and Helper

Evaluate negative extreme, central, and positive extreme logits as separate leaf tensors. Backpropagate the sum of outputs and compare absolute gradient magnitudes and their ratios.

Part 4 - Python code skeleton

```
import torch

def measure(values):
```

```
    logits = torch.tensor(
        values, dtype=torch.float64, requires_grad=True
    )
    torch.sigmoid(logits).sum().backward()
    return logits.grad.detach()

def E02(regions, vanishing_tolerance=1e-6):
    report = {}
    for name, values in regions.items():
        gradient = measure(values)
        report[name] = {
            "gradient": gradient,
            "mean_absolute_gradient": gradient.abs().mean().item(),
            "vanishing_count": int(
                (gradient.abs() < vanishing_tolerance).sum()
            ),
        }
    report["tail_to_center_ratio"] = ...
    return report

def main():
    regions = {"negative_tail": [...],
               "center": [...], "positive_tail": [...]}
    print(E02(regions))

if __name__ == "__main__":
    main()
```

11.4.4 Exercise 03 - Saturation Region Detection

Part 1 - Why this matters

Saturation detection identifies logits whose probabilities lie close to zero or one and whose local derivatives are small. This helps distinguish confident output from a responsive activation region.

The boundary depends on an explicit tolerance and dtype. A probability may round to an exact boundary in one precision before it does in another, so the detection rule must be reported with its settings.

Part 2 - Mathematical form

For probability tolerance ϵ and derivative tolerance δ, define saturation when:

$$p \leq \epsilon \quad \text{or} \quad p \geq 1 - \epsilon \quad \text{or} \quad p(1-p) \leq \delta$$

The exercise should report which condition triggered each detection.

Part 3 - Hints and Helper

Calculate probabilities and derivatives vectorially. Create separate masks for low probability, high probability, and small derivative, then combine them without losing the individual reasons.

Part 4 - Python code skeleton

```
import torch

def E03(logits, probability_tolerance=1e-4,
        derivative_tolerance=1e-4):
    probability = torch.sigmoid(logits)
    derivative = probability * (1 - probability)
    low = probability <= probability_tolerance
    high = probability >= 1 - probability_tolerance
    flat = derivative <= derivative_tolerance
    saturated = ...
    return {
        "probability": probability,
        "derivative": derivative,
        "low_probability_mask": low,
        "high_probability_mask": high,
        "small_derivative_mask": flat,
        "saturated_mask": saturated,
        "saturated_count": int(saturated.sum()),
    }

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

11.4.5 Exercise 04 - Input Scaling Effect

Part 1 - Why this matters

Feature scaling changes logit magnitude and therefore determines which part of sigmoid receives most samples. Large scales can push outputs into saturation, while very small scales can collapse probabilities near 0.5.

Scaling can affect optimization even when the final decision boundary could theoretically be represented. This experiment isolates activation behavior before training complicates the observation.

Part 2 - Mathematical form

For base logit z and scale s:

$$p_s = \sigma(sz)$$

$$g_s = p_s(1 - p_s)$$

As $|s|$ grows for nonzero z, probabilities move toward the tails and derivatives usually shrink.

Part 3 - Hints and Helper

Use the same base logits for every positive scale. Record probability range, mean derivative, and saturated fraction. Keep dtype and tolerance fixed across comparisons.

Part 4 - Python code skeleton

```
import torch

def E04(base_logits, scales, saturation_tolerance=1e-4):
    report = []
    for scale in scales:
        scaled = base_logits * scale
        probability = torch.sigmoid(scaled)
        derivative = probability * (1 - probability)
        saturated = derivative <= saturation_tolerance
        report.append({
            "scale": scale,
            "probability_min": probability.min().item(),
            "probability_max": probability.max().item(),
            "mean_derivative": ...,
            "saturated_fraction": saturated.float().mean().item(),
        })
    return report

def main():
    base_logits = torch.linspace(..., ..., steps=...)
    print(E04(base_logits, scales=[...]))

if __name__ == "__main__":
    main()
```

11.4.6 Exercise 05 - Large Logit Stability Test

Part 1 - Why this matters

Large logits expose overflow and rounding behavior hidden by ordinary examples. A direct exponential formula may create an infinite intermediate even when the final sigmoid value is representable.

A stability test should distinguish finite output, mathematical accuracy, monotonicity, and boundary rounding. Returning zero or one for an extreme finite logit may be expected rounding, while returning NaN indicates a stronger failure.

Part 2 - Mathematical form

The direct formula is:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

For negative z, the equivalent expression:

$$\sigma(z) = \frac{e^{z}}{1 + e^{z}}$$

avoids evaluating e^{-z} at a large positive magnitude.

Part 3 - Hints and Helper

Compare a naive formula, a branch-aware manual formula, and `torch.sigmoid` over symmetric extreme inputs. Report finite status, maximum difference, exact boundary counts, and monotonicity.

Part 4 - Python code skeleton

```
import torch

def stable_manual(logits):
    positive = logits >= 0
    result = torch.empty_like(logits)
    result[positive] = 1 / (1 + torch.exp(-logits[positive]))
    value = torch.exp(logits[~positive])
    result[~positive] = value / (1 + value)
    return result

def E05(logits):
    naive = 1 / (1 + torch.exp(-logits))
    manual = stable_manual(logits)
    reference = torch.sigmoid(logits)
    return {
        "naive_finite": bool(torch.isfinite(naive).all()),
        "manual_finite": bool(torch.isfinite(manual).all()),
        "reference_finite": bool(torch.isfinite(reference).all()),
        "manual_agreement": torch.allclose(
            manual, reference, atol=..., rtol=...
        ),
        "monotonic": bool(torch.all(reference[1:] >= reference[:-1])),
        "exact_boundaries": ...,
    }

def main():
    print(E05(torch.tensor([...], dtype=torch.float64)))

if __name__ == "__main__":
    main()
```

11.4.7 Exercise 06 - CUDA Gradient Comparison

Part 1 - Why this matters

CPU and CUDA should produce numerically close sigmoid gradients when inputs, dtype, and upstream objective are equivalent. This test detects device-copy mistakes and establishes realistic expectations for cross-device comparison.

Floating-point results across devices are not guaranteed to be bitwise identical. The comparison must use stated tolerances and report actual differences rather than forcing exact equality.

Part 2 - Technical concept and behavior

For CPU and CUDA gradients:

$$\Delta_{\max} = \max_i |g_i^{\text{CPU}} - g_i^{\text{CUDA}}|$$

Parity uses:

$$|g_i^{\text{CPU}} - g_i^{\text{CUDA}}| \leq \text{atol} + \text{rtol}|g_i^{\text{CUDA}}|$$

Part 3 - Hints and Helper

Create independent leaf copies on CPU and CUDA, calculate the same scalar sum, call backward separately, and move the CUDA gradient to CPU before comparison.

Part 4 - Python code skeleton

```
import torch

def sigmoid_gradient(values, device):
    logits = values.detach().clone().to(device).requires_grad_(True)
    torch.sigmoid(logits).sum().backward()
    return logits.grad.detach().cpu()

def E06(values, atol=1e-6, rtol=1e-5):
    if not torch.cuda.is_available():
        return {"skipped": True, "reason": ...}
    cpu_gradient = sigmoid_gradient(values, "cpu")
    cuda_gradient = sigmoid_gradient(values, "cuda")
    difference = (cpu_gradient - cuda_gradient).abs()
    return {
        "skipped": False,
        "allclose": torch.allclose(
            cpu_gradient, cuda_gradient, atol=atol, rtol=rtol
        ),
        "maximum_difference": difference.max().item(),
        "mean_difference": ...,
    }

def main():
```

```
    print(E06(...))

if __name__ == "__main__":
    main()
```

11.4.8 Exercise 07 - Float32 vs Float64 Gradient

Part 1 - Why this matters

Float32 and float64 represent sigmoid probabilities and gradients with different precision. Tail gradients may round differently or reach zero earlier, while the central region usually agrees more closely.

A dtype comparison should begin from the same high-precision source values. Creating separate approximate inputs can mix input differences with gradient-computation differences.

Part 2 - Technical concept and behavior

For gradient in dtype d:

$$g_d = \sigma(z_d)(1 - \sigma(z_d))$$

Compare with float64 reference:

$$\Delta = |g_{32} - g_{64}|$$

and record relative difference only where the reference gradient is nonzero.

Part 3 - Hints and Helper

Create a float64 source, cast independent leaf tensors, backpropagate the same sum, and convert gradients to float64 before comparing. Include central and extreme logits.

Part 4 - Python code skeleton

```
import torch

def gradient_for_dtype(source, dtype):
    logits = source.to(dtype).detach().requires_grad_(True)
    torch.sigmoid(logits).sum().backward()
    return logits.grad.detach().to(torch.float64)

def E07(values, epsilon=1e-30):
    source = torch.as_tensor(values, dtype=torch.float64)
    gradient32 = gradient_for_dtype(source, torch.float32)
    gradient64 = gradient_for_dtype(source, torch.float64)
    absolute = (gradient32 - gradient64).abs()
    relative = absolute / (gradient64.abs() + epsilon)
    return {
        "float32": gradient32,
        "float64": gradient64,
        "maximum_absolute_difference": ...,
```

```
        "maximum_relative_difference": ...,
        "float32_zero_count": int((gradient32 == 0).sum()),
    }

def main():
    print(E07([... ]))

if __name__ == "__main__":
    main()
```

11.4.9 Exercise 08 - Backward Pass Visualization

Part 1 - Why this matters

Plotting sigmoid output and its backward gradient on aligned axes reveals where the activation is responsive and where it becomes flat. This joins forward interpretation with backward learning behavior.

A linear vertical scale may hide very small tail gradients. Supporting a logarithmic gradient view can expose their decay, but zeros require careful handling because the logarithm is undefined.

Part 2 - Mathematical form

Plot:

$$p(z) = \sigma(z)$$

and:

$$g(z) = \left|\frac{dp}{dz}\right| = \sigma(z)(1 - \sigma(z))$$

using the same ordered logit coordinates.

Part 3 - Hints and Helper

Use a leaf logit tensor, retain the analytical ordering, backpropagate the probability sum, and detach before plotting. Use separate axes or subplots so probability and gradient scales are not confused.

Part 4 - Python code skeleton

```
import matplotlib.pyplot as plt
import torch

def E08(start, end, steps, output_path, log_gradient=False):
    logits = torch.linspace(
        start, end, steps=steps, requires_grad=True
    )
    probability = torch.sigmoid(logits)
    probability.sum().backward()
```

```
    gradient = logits.grad.detach().abs()
    x = logits.detach()
    fig, axes = plt.subplots(2, 1, sharex=True)
    axes[0].plot(x.numpy(), probability.detach().numpy())
    axes[0].set_ylabel("Probability")
    axes[1].plot(x.numpy(), gradient.numpy())
    if log_gradient:
        axes[1].set_yscale("log")
    axes[1].set_xlabel(...)
    axes[1].set_ylabel(...)
    fig.savefig(output_path, dpi=150, bbox_inches="tight")
    plt.close(fig)

def main():
    E08(...)

if __name__ == "__main__":
    main()
```

11.4.10 Exercise 09 - Activation Failure Debugging

Part 1 - Why this matters

Sigmoid experiments can fail through integer inputs, detached graphs, in-place mutation, non-finite logits, unstable manual exponentiation, or accidental double activation. Some cases return plausible probabilities while breaking gradient flow.

Debugging should inspect representation and graph metadata before changing formulas. A value inside $[0, 1]$ does not prove that the activation was applied exactly once or remains connected to its source.

Part 2 - Technical concept and behavior

A gradient-capable sigmoid output requires:

$$z \in \mathbb{R}, \qquad \text{isfinite}(z) = \text{True}$$

and, when gradients are expected:

$$z.\texttt{requires_grad} = \text{True}, \qquad \sigma(z).\texttt{grad_fn} \neq \texttt{None}$$

Part 3 - Hints and Helper

Build controlled cases for integer dtype, NaN input, detached output, and double sigmoid. Report dtype, device, finite status, gradient flags, output range, and detected issue without silently casting invalid data.

Part 4 - Python code skeleton

```
import torch

def inspect_activation(logits, output, expect_gradient=True):
    issues = []
    if not logits.is_floating_point():
        issues.append("non_floating_input")
    if logits.is_floating_point() and not torch.isfinite(logits).all():
        issues.append("nonfinite_input")
    if output.is_floating_point() and not torch.isfinite(output).all():
        issues.append("nonfinite_output")
    if expect_gradient and output.grad_fn is None:
        issues.append(...)
    if not ((output >= 0) & (output <= 1)).all():
        issues.append(...)
    return {"issues": issues, "valid": len(issues) == 0}

def E09(cases):
    report = {}
    for name, logits in cases.items():
        try:
            output = torch.sigmoid(logits)
            report[name] = inspect_activation(logits, output)
        except (RuntimeError, TypeError) as error:
            report[name] = {"error": type(error).__name__,
                            "message": str(error)}
    return report

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

11.4.11 Exercise 10 - Custom Sigmoid Module

Part 1 - Why this matters

A custom module packages sigmoid behind the standard PyTorch module interface. This makes it easy to place inside larger models while preserving device movement, mode handling, and autograd behavior.

The module should not reimplement a fragile exponential formula merely to appear custom. Its purpose is to understand module structure while delegating the stable element-wise operation to PyTorch.

Part 2 - Technical concept and behavior

The forward mapping is:

$$\text{forward}(\mathbf{z}) = \sigma(\mathbf{z})$$

The module contains no trainable parameters and should preserve input shape and device.

Part 3 - Hints and Helper

Subclass `torch.nn.Module`, call the parent constructor, and implement `forward` with `torch.sigmoid`. Compare values and gradients with the functional operation.

Part 4 - Python code skeleton

```
import torch
from torch import nn

class CustomSigmoid(nn.Module):
    def __init__(self):
        super().__init__()

    def forward(self, logits):
        return ...

def E10(logits):
    module = CustomSigmoid()
    custom = module(logits)
    reference = torch.sigmoid(logits)
    return {
        "value_agreement": torch.allclose(
            custom, reference, atol=..., rtol=...
        ),
        "shape_preserved": custom.shape == logits.shape,
        "parameter_count": sum(
            parameter.numel() for parameter in module.parameters()
        ),
        "gradient_function": type(custom.grad_fn).__name__
        if custom.grad_fn is not None else None,
    }

def main():
    print(E10(...))

if __name__ == "__main__":
    main()
```

11.4.12 Exercise 11 - Sigmoid vs Tanh Comparison

Part 1 - Why this matters

Sigmoid and hyperbolic tangent have similar S-shaped curves but different output ranges and centers. Sigmoid maps to $(0, 1)$, while tanh maps to $(-1, 1)$ and is centered at zero.

Both activations saturate in their tails, but their derivatives have different scales near zero. Comparing them prevents activation choice from being reduced to visual similarity.

Part 2 - Mathematical form

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \qquad \sigma'(z) = \sigma(z)(1 - \sigma(z))$$

$$\tanh'(z) = 1 - \tanh^2(z)$$

At zero, sigmoid derivative is 0.25 while tanh derivative is 1.

Part 3 - Hints and Helper

Evaluate both functions over the same symmetric input range. Calculate analytical derivatives, compare output means, central gradients, and saturated fractions using declared tolerances.

Part 4 - Python code skeleton

```
import torch

def E11(logits, derivative_tolerance=1e-4):
    sigmoid_output = torch.sigmoid(logits)
    tanh_output = torch.tanh(logits)
    sigmoid_gradient = sigmoid_output * (1 - sigmoid_output)
    tanh_gradient = 1 - tanh_output.square()
    center = torch.argmin(logits.abs())
    return {
        "sigmoid_range": (
            sigmoid_output.min().item(),
            sigmoid_output.max().item(),
        ),
        "tanh_range": (..., ...),
        "sigmoid_center_gradient":
            sigmoid_gradient[center].item(),
        "tanh_center_gradient": ...,
        "sigmoid_saturated_fraction": ...,
        "tanh_saturated_fraction": ...,
    }

def main():
    print(E11(torch.linspace(..., ..., steps=...)))

if __name__ == "__main__":
    main()
```

11.4.13 Exercise 12 - Sigmoid in Deep Stack Simulation

Part 1 - Why this matters

Repeated sigmoid transformations simulate how gradients can weaken through depth. Even when every operation is valid, multiplying many local derivatives can greatly reduce the gradient reaching the original input.

The experiment should preserve intermediate activations and gradients so the reader can see where attenuation begins. One final gradient value does not explain the layer-by-layer process.

Part 2 - Mathematical form

For:

$$a_\ell = \sigma(w_\ell a_{\ell-1} + b_\ell)$$

the chain rule gives:

$$\frac{\partial a_L}{\partial a_0} = \prod_{\ell=1}^{L} [w_\ell \sigma'(z_\ell)]$$

Both weights and activation derivatives control the final gradient.

Part 3 - Hints and Helper

Start with a scalar leaf input, create each non-leaf pre-activation and activation, and call `retain_grad()` on values you want to inspect. Backpropagate the final activation.

Part 4 - Python code skeleton

```
import torch

def E12(initial_value, weights, biases):
    activation = torch.tensor(
        initial_value, dtype=torch.float64, requires_grad=True
    )
    original = activation
    layers = []
    for weight, bias in zip(weights, biases):
        preactivation = weight * activation + bias
        activation = torch.sigmoid(preactivation)
        preactivation.retain_grad()
        activation.retain_grad()
        layers.append((preactivation, activation))
    activation.backward()
    report = []
    for index, (preactivation, current) in enumerate(layers):
        report.append({
            "layer": index,
            "preactivation": preactivation.item(),
            "activation": current.item(),
            "preactivation_gradient": ...,
            "activation_gradient": ...,
        })
    return {"input_gradient": original.grad.item(),
            "layers": report}

def main():
    print(E12(...))
```

```
if __name__ == "__main__":
    main()
```

11.4.14 Exercise 13 - Gradient Noise Injection

Part 1 - Why this matters

Gradient noise changes the direction and magnitude of an update signal. Controlled injection helps test whether small perturbations dominate already weak sigmoid gradients, especially in saturated regions.

Noise must be applied to a detached gradient copy or through an explicit training rule. Accidentally modifying stored gradients without recording the change makes the experiment impossible to interpret.

Part 2 - Mathematical form

For clean gradient $\mathbf{g}$:

$$\widetilde{\mathbf{g}} = \mathbf{g} + \sigma_n \boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$$

Signal-to-noise ratio can compare:

$$\frac{\|\mathbf{g}\|_2}{\|\widetilde{\mathbf{g}} - \mathbf{g}\|_2}$$

Part 3 - Hints and Helper

Calculate the clean sigmoid gradient first. Use a seeded generator for noise, preserve the original, and report norms, cosine similarity, sign changes, and signal-to-noise ratio.

Part 4 - Python code skeleton

```
import torch

def E13(logits, noise_scale, seed, epsilon=1e-12):
    logits = logits.detach().clone().requires_grad_(True)
    torch.sigmoid(logits).sum().backward()
    clean = logits.grad.detach().clone()
    generator = torch.Generator(device=clean.device)
    generator.manual_seed(seed)
    noise = torch.randn(
        clean.shape, generator=generator,
        device=clean.device, dtype=clean.dtype
    ) * noise_scale
    noisy = clean + noise
    cosine = torch.nn.functional.cosine_similarity(
        clean.reshape(1, -1), noisy.reshape(1, -1)
    ).item()
    return {
```

```
        "clean_norm": clean.norm().item(),
        "noise_norm": noise.norm().item(),
        "noisy_norm": ...,
        "signal_to_noise": clean.norm().item()
        / (noise.norm().item() + epsilon),
        "cosine_similarity": cosine,
        "sign_change_count": ...,
    }

def main():
    print(E13(...))

if __name__ == "__main__":
    main()
```

11.4.15 Exercise 14 - Autograd Graph Inspection

Part 1 - Why this matters

Autograd graph inspection shows that sigmoid output retains a recorded relationship to its input when gradient tracking is enabled. The `grad_fn` node is the entry point to the operations used during backward.

Graph inspection is a debugging aid, not an interface for rewriting autograd internals. The exercise should report node names and connections without mutating saved tensors or private state.

Part 2 - Technical concept and behavior

For a leaf input z requiring gradients:

$$p = \sigma(z)$$

has:

$$p.\texttt{grad_fn} \neq \texttt{None}$$

while $z.\texttt{grad_fn} = \texttt{None}$ because z is a leaf.

Part 3 - Hints and Helper

Inspect `is_leaf`, `requires_grad`, `grad_fn`, and immediate `next_functions`. Compare the normal output with a detached copy and with output created under no-gradient mode.

Part 4 - Python code skeleton

```
import torch

def describe(tensor):
    node = tensor.grad_fn
    return {
        "is_leaf": tensor.is_leaf,
```

```
        "requires_grad": tensor.requires_grad,
        "grad_fn": type(node).__name__ if node else None,
        "next_functions": [
            type(next_node).__name__ if next_node else None
            for next_node, _ in node.next_functions
        ] if node else [],
    }

def E14(values):
    logits = torch.tensor(
        values, dtype=torch.float32, requires_grad=True
    )
    probability = torch.sigmoid(logits)
    detached = probability.detach()
    with torch.no_grad():
        no_grad_probability = torch.sigmoid(logits)
    return {
        "input": describe(logits),
        "probability": describe(probability),
        "detached": describe(detached),
        "no_grad": ...,
    }

def main():
    print(E14(...))

if __name__ == "__main__":
    main()
```

11.4.16 Exercise 15 - Mixed Precision Backward Test

Part 1 - Why this matters

Mixed precision can accelerate suitable CUDA operations, but lower-precision intermediates may alter small gradients or overflow in incompatible workloads. Gradient scaling helps preserve backward signals through float16 paths.

The test must compare AMP gradients with a float32 reference and monitor finite status. Successful execution alone does not establish numerical agreement.

Part 2 - Technical concept and behavior

With loss scale s:

$$\nabla_\theta(s\mathcal{L}) = s\nabla_\theta\mathcal{L}$$

The scaler unscales gradients before comparison or clipping, while model parameters remain in their ordinary precision.

Part 3 - Hints and Helper

Use two model copies with identical states. Run one float32 backward pass and one autocast forward with a scaled backward pass. Exit autocast before backward and unscale through the AMP optimizer before comparing gradients.

Part 4 - Python code skeleton

```
import copy
import torch
import torch.nn.functional as F

def E15(model, X, target):
    if not torch.cuda.is_available():
        return {"skipped": True}
    reference = copy.deepcopy(model).cuda()
    mixed = copy.deepcopy(model).cuda()
    X, target = X.cuda(), target.cuda()

    reference.zero_grad(set_to_none=True)
    reference_loss = F.binary_cross_entropy_with_logits(
        reference(X), target
    )
    reference_loss.backward()
    reference_gradients = [
        parameter.grad.detach().clone()
        for parameter in reference.parameters()
    ]

    optimizer = torch.optim.SGD(mixed.parameters(), lr=0.0)
    scaler = torch.amp.GradScaler("cuda")
    optimizer.zero_grad(set_to_none=True)
    with torch.autocast(device_type="cuda", dtype=torch.float16):
        logits = mixed(X)
        loss = F.binary_cross_entropy_with_logits(logits, target)
    scaler.scale(loss).backward()
    scaler.unscale_(optimizer)
    mixed_gradients = [...]
    differences = [...]
    return {"skipped": False, "loss": loss.item(),
            "all_finite": ...,
            "maximum_gradient_difference": ...,
            "scale": scaler.get_scale()}

def main():
    print(E15(...))

if __name__ == "__main__":
    main()
```

11.4.17 Exercise 16 - Activation Profiling on GPU

Part 1 - Why this matters

GPU profiling reveals the operations, device time, shapes, and memory activity associated with sigmoid evaluation. It helps determine whether activation work is substantial or merely one small part of a larger pipeline.

Profiling adds overhead and should not be confused with a clean latency benchmark. Warm-up and a limited active region make the trace more representative and manageable.

Part 2 - Technical concept and behavior

For repeated activation workload:

$$\mathbf{p}_r = \sigma(\mathbf{z}), \qquad r = 1, \ldots, R$$

the profiler attributes CPU dispatch and CUDA kernel activity to recorded operations. Workload shape, dtype, repetitions, and device must accompany the results.

Part 3 - Hints and Helper

Require CUDA, warm up first, synchronize, and profile CPU plus CUDA activities. Enable shape and memory recording deliberately, then summarize key averages by CUDA time.

Part 4 - Python code skeleton

```
import torch
from torch.profiler import profile, ProfilerActivity

def E16(logits, repetitions=100):
    if logits.device.type != "cuda":
        raise ValueError(...)
    for _ in range(10):
        torch.sigmoid(logits)
    torch.cuda.synchronize(logits.device)
    with profile(
        activities=[
            ProfilerActivity.CPU,
            ProfilerActivity.CUDA,
        ],
        record_shapes=True,
        profile_memory=True,
    ) as profiler:
        for _ in range(repetitions):
            output = torch.sigmoid(logits)
    torch.cuda.synchronize(logits.device)
    table = profiler.key_averages().table(
        sort_by="self_cuda_time_total", row_limit=...
    )
    return {"table": table,
            "output_shape": tuple(output.shape),
```

```
            "repetitions": repetitions}

def main():
    print(E16(...))

if __name__ == "__main__":
    main()
```

11.4.18 Exercise 17 - Stability under Extreme Inputs

Part 1 - Why this matters

Extreme finite inputs test sigmoid's range, monotonicity, symmetry, and gradient behavior near numerical limits. They also reveal the point where a dtype rounds outputs or derivatives to exact zero or one.

Stability does not mean every extreme input retains a useful gradient. A correct implementation may remain finite while the derivative vanishes numerically, and both facts should be reported.

Part 2 - Mathematical form

Sigmoid symmetry is:

$$\sigma(-z) = 1 - \sigma(z)$$

The derivative symmetry is:

$$\sigma'(-z) = \sigma'(z)$$

within the limits of floating-point representation.

Part 3 - Hints and Helper

Use sorted symmetric values including zero and dtype extremes that remain finite. Check output finiteness, range, monotonicity, symmetry error, derivative symmetry, and boundary counts.

Part 4 - Python code skeleton

```
import torch

def E17(positive_values, dtype=torch.float64):
    positive = torch.as_tensor(positive_values, dtype=dtype)
    positive = torch.sort(positive.abs()).values
    negative = -positive.flip(0)
    logits = torch.cat([negative, torch.zeros(1, dtype=dtype),
                        positive])
    probability = torch.sigmoid(logits)
    derivative = probability * (1 - probability)
    symmetry = (
        torch.sigmoid(-positive)
```

```
        - (1 - torch.sigmoid(positive))
    ).abs()
    return {
        "all_finite": bool(torch.isfinite(probability).all()),
        "in_range": bool(
            ((probability >= 0) & (probability <= 1)).all()
        ),
        "monotonic": bool(
            torch.all(probability[1:] >= probability[:-1])
        ),
        "maximum_symmetry_error": ...,
        "zero_derivative_count": int((derivative == 0).sum()),
        "boundary_probability_count": ...,
    }

def main():
    print(E17([... ]))

if __name__ == "__main__":
    main()
```

11.4.19 Exercise 18 - Activation Optimization Attempt

Part 1 - Why this matters

An optimization attempt should compare correctness, runtime, and memory rather than assuming a shorter formula is faster. PyTorch's built-in sigmoid uses optimized kernels that a Python-level manual implementation is unlikely to outperform.

Benchmarking must separate warm-up and use synchronization on CUDA. A candidate is acceptable only if its accuracy across ordinary and extreme inputs satisfies a declared tolerance.

Part 2 - Technical concept and behavior

For candidate f and reference σ:

$$E_{\max} = \max_i |f(z_i) - \sigma(z_i)|$$

Speedup is:

$$S = \frac{T_{\text{reference}}}{T_{\text{candidate}}}$$

and should be interpreted with repeated measurements.

Part 3 - Hints and Helper

Compare built-in, direct manual, and branch-aware implementations on identical tensors. Warm up, synchronize the device, measure repetitions, and reject non-finite or inaccurate candidates.

Part 4 - Python code skeleton

```
import time
import torch

def benchmark(function, logits, repetitions):
    for _ in range(10):
        function(logits)
    if logits.device.type == "cuda":
        torch.cuda.synchronize(logits.device)
    start = time.perf_counter()
    for _ in range(repetitions):
        output = function(logits)
    if logits.device.type == "cuda":
        torch.cuda.synchronize(logits.device)
    return time.perf_counter() - start, output

def E18(logits, candidates, repetitions, tolerance):
    reference_time, reference = benchmark(
        torch.sigmoid, logits, repetitions
    )
    report = {}
    for name, function in candidates.items():
        elapsed, output = benchmark(function, logits, repetitions)
        error = (output - reference).abs().max().item()
        report[name] = {
            "seconds": elapsed,
            "speedup": reference_time / elapsed,
            "maximum_error": error,
            "accepted": ...,
            "all_finite": ...,
        }
    return {"reference_seconds": reference_time,
            "candidates": report}

def main():
    print(E18(...))

if __name__ == "__main__":
    main()
```

11.4.20 Exercise 19 - Sigmoid Failure Report

Part 1 - Why this matters

A final failure report combines evidence from values, gradients, devices, dtypes, graph state, and performance. It distinguishes expected saturation from numerical corruption and accidental graph breaks.

The report should preserve reproducible inputs and explicit rules. Labels such as unstable or vanishing are meaningless unless their tolerances and observed measurements are included.

Part 2 - Report structure

For each scenario s, record:

$$R_s = \{\text{input}, \text{dtype}, \text{device}, \text{output}, \text{gradient}, \text{checks}, \text{classification}\}$$

Classifications may include valid, saturated, graph-broken, non-finite, inaccurate, or unsupported.

Part 3 - Hints and Helper

Create scenarios for central, saturated, extreme, detached, integer, and CUDA inputs when available. Run independent checks so one failure does not erase the remaining evidence, and produce a machine-readable summary.

Part 4 - Python code skeleton

```
import torch

def analyze_case(name, logits, expect_gradient):
    result = {
        "name": name,
        "dtype": str(logits.dtype),
        "device": str(logits.device),
        "shape": tuple(logits.shape),
    }
    try:
        probability = torch.sigmoid(logits)
        result["output_finite"] = bool(
            torch.isfinite(probability).all()
        )
        result["output_in_range"] = bool(
            ((probability >= 0) & (probability <= 1)).all()
        )
        result["graph_connected"] = probability.grad_fn is not None
        if expect_gradient and probability.requires_grad:
            probability.sum().backward()
            result["gradient_finite"] = ...
            result["gradient_norm"] = ...
        result["classification"] = ...
    except (RuntimeError, TypeError) as error:
        result["classification"] = "failed"
        result["error_type"] = type(error).__name__
        result["message"] = str(error)
    return result

def E19(scenarios):
    return [
        analyze_case(name, values["logits"],
                     values["expect_gradient"])
        for name, values in scenarios.items()
```

```
    ]

def main():
    print(E19(...))

if __name__ == "__main__":
    main()
```

11.5 LG03 - Probability Interpretation Framework

Models and Training

11.5.1 Exercise 00 - Likelihood Interpretation

Part 1 - Why this matters

A logistic-regression output is not only a number used to choose a class. Once the observed label is fixed, that output measures how compatible the model parameters are with the observation. This change of viewpoint turns a predicted probability into a likelihood contribution.

Probability and likelihood must not be confused. In frequentist logistic regression, parameters are unknown fixed values and the observed data define a function that scores candidate parameters; likelihood is not the probability that a parameter is correct.

Part 2 - Mathematical form

For $z_i = \mathbf{w}^\mathsf{T}\mathbf{x}_i + b$ and $p_i = \sigma(z_i)$,

$$L_i(\mathbf{w}, b) = p_i^{y_i}(1 - p_i)^{1-y_i}, \qquad y_i \in \{0, 1\}.$$

Thus $L_i = p_i$ when $y_i = 1$, and $L_i = 1 - p_i$ when $y_i = 0$. Under conditional independence,

$$L(\mathbf{w}, b) = \prod_{i=1}^{N} L_i(\mathbf{w}, b).$$

Part 3 - Hints and Helper

Accept probabilities and binary targets with identical shapes. Select the probability assigned to the observed label without a Python loop. Return per-sample terms first; a product is safe only for a small demonstration because it can underflow.

Part 4 - Python code skeleton

```
import torch

def E00(probabilities, targets):
    if probabilities.shape != targets.shape:
        raise ValueError(...)
    valid = (targets == 0) | (targets == 1)
    if not torch.all(valid):
```

```
        raise ValueError(...)
    terms = torch.where(targets == 1, ..., ...)
    return terms

def main():
    probabilities = torch.tensor(...)
    targets = torch.tensor(...)
    print(E00(probabilities, targets))

if __name__ == "__main__":
    main()
```

11.5.2 Exercise 01 - Log-Likelihood Computation

Part 1 - Why this matters

Multiplying many likelihood contributions quickly creates values too small for floating-point arithmetic. Taking the logarithm converts products into sums, preserves the ordering of positive likelihoods, and exposes the evidence supplied by each observation.

Log-likelihood is easier to diagnose than a tiny product. A confident correct prediction contributes a value near zero, whereas a confident incorrect prediction contributes a large negative value. Individual samples can therefore be inspected directly.

Part 2 - Mathematical form

The binary log-likelihood is

$$\ell(\mathbf{w}, b) = \sum_{i=1}^{N} \left[y_i \log(p_i) + (1 - y_i) \log(1 - p_i) \right].$$

Every term is non-positive for $0 < p_i < 1$. Maximizing ℓ is equivalent to maximizing L because logarithm is strictly increasing.

Part 3 - Hints and Helper

For this probability-based experiment, clamp values away from exactly zero and one before taking logarithms. Keep the per-sample terms and their sum. Later exercises should use logits directly rather than treating clamping as the universal numerical solution.

Part 4 - Python code skeleton

```
import torch

def E01(probabilities, targets, epsilon):
    probabilities = probabilities.clamp(
        min=epsilon, max=1.0 - epsilon
    )
    terms = (
        targets * torch.log(...)
        + (1.0 - targets) * torch.log1p(...)
```

```
    )
    return terms, ...

def main():
    probabilities = torch.tensor(..., dtype=torch.float64)
    targets = torch.tensor(..., dtype=torch.float64)
    print(E01(probabilities, targets, epsilon=...))

if __name__ == "__main__":
    main()
```

11.5.3 Exercise 02 - Negative Log-Likelihood Concept

Part 1 - Why this matters

Optimization tools usually minimize an objective, while likelihood estimation asks us to maximize agreement with observed data. Negating the log-likelihood reverses the direction without changing the parameter value considered optimal.

Negative log-likelihood has a useful penalty interpretation. A well-explained label adds little cost, while assigning tiny probability to an observed label produces a large cost. This connects statistical estimation to model training.

Part 2 - Mathematical form

Define

$$\mathrm{NLL} = -\ell = -\sum_{i=1}^{N} \left[y_i \log(p_i) + (1 - y_i) \log(1 - p_i) \right].$$

Therefore,

$$\arg\max_{\theta} L(\theta) = \arg\max_{\theta} \ell(\theta) = \arg\min_{\theta} \mathrm{NLL}(\theta).$$

Part 3 - Hints and Helper

Compute summed and mean NLL from per-sample log-likelihood terms. Verify that increasing the probability assigned to every observed label decreases both reductions, while their scales differ by the number of evaluated elements.

Part 4 - Python code skeleton

```
import torch

def E02(log_likelihood_terms):
    nll_terms = ...
    return {
        "terms": nll_terms,
        "sum": ...,
        "mean": ...,
    }
```

```
def main():
    terms = torch.tensor(...)
    print(E02(terms))

if __name__ == "__main__":
    main()
```

11.5.4 Exercise 03 - Likelihood vs Loss Mapping

Part 1 - Why this matters

Likelihood and binary cross-entropy are often presented as separate tools. For a Bernoulli logistic model they are the same sample-wise quantity viewed from opposite optimization directions: maximize agreement or minimize its negative.

The reduction mode must remain visible. A summed loss equals negative total log-likelihood, while a mean loss divides by the element count. Ignoring this detail makes comparisons across dataset and batch sizes misleading.

Part 2 - Mathematical form

For one target,

$$\mathrm{BCE}(p_i, y_i) = -\left[y_i \log(p_i) + (1 - y_i) \log(1 - p_i)\right] = -\ell_i.$$

For N elements,

$$\mathrm{BCE}_{\mathrm{sum}} = -\ell, \qquad \mathrm{BCE}_{\mathrm{mean}} = -\frac{\ell}{N}.$$

Part 3 - Hints and Helper

Use a stable logits-based PyTorch function with reduction `none`. Sum and average its terms manually, then verify their equality to negative log-likelihood terms. Do not apply sigmoid before a function that expects logits.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E03(logits, targets):
    bce_terms = F.binary_cross_entropy_with_logits(
        logits, targets, reduction="none"
    )
    log_likelihood_terms = ...
    return {
        "mapping_equal": torch.allclose(
            bce_terms, -log_likelihood_terms
        ),
        "nll_sum": ...,
        "nll_mean": ...,
```

```
    }

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

11.5.5 Exercise 04 - Batch Likelihood Aggregation

Part 1 - Why this matters

Large datasets are processed as batches, but the statistical objective belongs to the entire dataset. A correct epoch result must combine every contribution with the same meaning it would have in a single full-tensor computation.

Averaging batch means is wrong when the last batch is smaller because it gives every batch equal weight instead of every element. Accumulating summed NLL and the element count produces the correct dataset-wide mean.

Part 2 - Mathematical form

For batches $\mathcal{B}_1, \ldots, \mathcal{B}_K$,

$$\ell_{\text{epoch}} = \sum_{k=1}^{K} \sum_{i \in \mathcal{B}_k} \ell_i, \qquad \overline{\text{NLL}} = -\frac{\ell_{\text{epoch}}}{\sum_{k=1}^{K} |\mathcal{B}_k|}.$$

Part 3 - Hints and Helper

Compute a summed logits-based BCE for each batch, add its detached scalar, and add `targets.numel()` to the count. Divide only after the loop. Also calculate the intentionally incorrect unweighted average of batch means.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E04(model, data_loader, device):
    total_nll = 0.0
    total_elements = 0
    batch_means = []
    with torch.no_grad():
        for features, targets in data_loader:
            features = features.to(device)
            targets = targets.to(device)
            logits = model(features).reshape_as(targets)
            batch_nll = F.binary_cross_entropy_with_logits(
                logits, targets, reduction="sum"
            )
```

```
            total_nll += ...
            total_elements += ...
            batch_means.append(...)
    return {
        "correct_mean_nll": ...,
        "wrong_mean_of_means": ...,
    }

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

11.5.6 Exercise 05 - CUDA Likelihood Computation

Part 1 - Why this matters

GPU evaluation can accelerate large batches, but it introduces device coordination, asynchronous execution, and transfer costs. Model parameters, inputs, targets, and intermediate values must remain on compatible devices.

A CUDA implementation should preserve the statistical definition instead of silently changing reduction or precision. Comparing CPU and GPU summaries within a justified tolerance separates expected floating-point variation from implementation errors.

Part 2 - Mathematical form

Using logits directly, a stable NLL term is

$$-\ell_i = \max(z_i, 0) - z_i y_i + \log\left(1 + e^{-|z_i|}\right).$$

Compare datasets through sums or means of these terms, not products of rounded probabilities.

Part 3 - Hints and Helper

Select CUDA only when available, move a copied model and both tensors to the chosen device, and use summed stable BCE. Use the same model state, inputs, targets, and reduction for CPU and CUDA.

Part 4 - Python code skeleton

```
import copy

import torch
import torch.nn.functional as F

def compute_nll(model, features, targets, device):
    model = copy.deepcopy(model).to(device)
    features = features.to(device)
    targets = targets.to(device)
```

```
    with torch.no_grad():
        logits = model(features).reshape_as(targets)
        total = F.binary_cross_entropy_with_logits(
            logits, targets, reduction="sum"
        )
    return total.item(), targets.numel()

def E05(model, features, targets):
    cpu_total, count = compute_nll(
        model, features, targets, torch.device("cpu")
    )
    if not torch.cuda.is_available():
        return {"cuda_available": False, "cpu_mean": ...}
    cuda_total, cuda_count = compute_nll(
        model, features, targets, torch.device("cuda")
    )
    return {
        "cuda_available": True,
        "cpu_mean": ...,
        "cuda_mean": ...,
        "close": ...,
    }

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

11.5.7 Exercise 06 - Likelihood Sensitivity Test

Part 1 - Why this matters

Likelihood sensitivity shows how strongly an observation reacts when its prediction changes. Near a confident error, movement toward the observed label can improve log-likelihood far more than the same movement near an already reasonable prediction.

Perturbing logits makes the behavior observable without hiding it inside an optimizer. Comparing autograd with a finite difference connects local statistical evidence to the direction later used for parameter adjustment.

Part 2 - Mathematical form

For one observation,

$$\ell_i(z_i) = y_i \log \sigma(z_i) + (1 - y_i) \log(1 - \sigma(z_i)), \qquad \frac{\partial \ell_i}{\partial z_i} = y_i - \sigma(z_i).$$

A centered approximation is

$$\frac{\ell_i(z_i + h) - \ell_i(z_i - h)}{2h}.$$

Part 3 - Hints and Helper

Compute log-likelihood as negative stable BCE. Compare autograd with centered finite differences for confident correct, uncertain, and confident incorrect cases. Recreate the input tensor for every independent test.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def log_likelihood(logit, target):
    return -F.binary_cross_entropy_with_logits(
        logit, target, reduction="sum"
    )

def E06(logit_value, target_value, step):
    logit = torch.tensor(
        [logit_value],
        dtype=torch.float64,
        requires_grad=True,
    )
    target = torch.tensor([target_value], dtype=torch.float64)
    value = log_likelihood(logit, target)
    value.backward()
    automatic = ...
    with torch.no_grad():
        upper = log_likelihood(logit + step, target)
        lower = log_likelihood(logit - step, target)
        finite_difference = ...
    return automatic, finite_difference

def main():
    for case in ...:
        print(E06(*case, step=...))

if __name__ == "__main__":
    main()
```

11.5.8 Exercise 07 - Confidence Calibration Check

Part 1 - Why this matters

A probability is calibrated when predictions made with similar probability occur at approximately that frequency. Among cases assigned values near 0.8, roughly eighty percent should have positive labels. Accuracy alone does not guarantee this.

Calibration matters when probabilities influence risk, ranking, or human decisions. A bin report must include counts because a large gap supported by two samples is not equivalent to the same gap supported by thousands.

Part 2 - Mathematical form

For bin B_m,

$$\mathrm{conf}(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} p_i, \quad \mathrm{freq}(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} y_i.$$

An empirical summary is

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \left| \mathrm{freq}(B_m) - \mathrm{conf}(B_m) \right|.$$

Part 3 - Hints and Helper

Create equal-width boundaries from zero to one and assign each probability to one bin. Handle the right endpoint carefully. Preserve empty bins in the report with count zero but omit their contribution from ECE.

Part 4 - Python code skeleton

```
import torch

def E07(probabilities, targets, number_of_bins):
    boundaries = torch.linspace(
        0.0, 1.0, steps=number_of_bins + 1,
        device=probabilities.device,
    )
    bin_ids = torch.bucketize(
        probabilities, boundaries[1:-1], right=...
    )
    rows = []
    ece = torch.zeros((), device=probabilities.device)
    for bin_id in range(number_of_bins):
        mask = bin_ids == bin_id
        count = int(mask.sum().item())
        if count == 0:
            rows.append(...)
            continue
        confidence = ...
        frequency = ...
        gap = ...
        ece += ...
        rows.append(...)
    return rows, ece.item()

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

11.5.9 Exercise 08 - Overconfident Prediction Detection

Part 1 - Why this matters

An incorrect prediction becomes especially dangerous when the model presents it with extreme confidence. These cases can dominate log-loss, mislead downstream decisions, and reveal missing features or distribution changes that accuracy conceals.

For binary classification, predicted-class confidence is not always the positive-class probability. When the predicted class is zero, its confidence is $1 - p$. A detector must make this distinction before labeling an error overconfident.

Part 2 - Mathematical form

With threshold τ,

$$\widehat{y}_i = \mathbb{I}[p_i \geq \tau], \qquad c_i = \max(p_i, 1 - p_i).$$

For confidence cutoff γ, an overconfident error satisfies

$$\widehat{y}_i \neq y_i \quad \text{and} \quad c_i \geq \gamma.$$

Part 3 - Hints and Helper

Return indices, probabilities, targets, predicted labels, predicted-class confidence, and NLL for every detected case. Sort by decreasing confidence or NLL so the most severe failures appear first.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E08(logits, targets, threshold, confidence_cutoff):
    probabilities = torch.sigmoid(logits)
    predictions = (probabilities >= threshold).to(targets.dtype)
    confidence = torch.maximum(
        probabilities, 1.0 - probabilities
    )
    selected = (
        (predictions != targets)
        & (confidence >= confidence_cutoff)
    )
    nll = F.binary_cross_entropy_with_logits(
        logits, targets, reduction="none"
    )
    indices = torch.nonzero(
        selected, as_tuple=False
    ).flatten()
    return {
        "indices": ...,
        "probabilities": ...,
```

```
        "targets": ...,
        "predictions": ...,
        "confidence": ...,
        "nll": ...,
    }

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

11.5.10 Exercise 09 - Probability Misinterpretation Case

Part 1 - Why this matters

A probability pipeline can produce valid numbers and still communicate the wrong conclusion. Probability is not a class label, confidence does not guarantee correctness, likelihood is not a posterior over parameters, and a threshold is a decision policy.

A useful debugging exercise must detect semantic errors as well as range and shape errors. The reader should explain why each suspicious claim fails and state which assumptions or evidence would support a corrected statement.

Part 2 - Mathematical form

For $p_i = P(Y_i = 1 \mid \mathbf{x}_i, \widehat{\theta})$,

$$\widehat{y}_i = \mathbb{I}[p_i \geq \tau]$$

is a decision based on p_i. With fixed data $\mathcal{D}$,

$$L(\theta; \mathcal{D}) = P(\mathcal{D} \mid \theta)$$

must not be replaced by $P(\theta \mid \mathcal{D})$ without a prior and Bayesian inference.

Part 3 - Hints and Helper

Represent claims as records containing a value, representation, and verbal statement. Check ranges, logit-versus-probability confusion, threshold assumptions, likelihood language, and unsupported certainty. Return explanations, not only flags.

Part 4 - Python code skeleton

```
import torch

def validate_claim(claim):
    issues = []
    value = claim["value"]
    representation = claim["representation"]
    statement = claim["statement"].lower()
    if representation == "probability":
        if not torch.isfinite(value).all():
```

```
            issues.append(...)
        if not ((value >= 0) & (value <= 1)).all():
            issues.append(...)
    if representation == "logit" and "probability" in statement:
        issues.append(...)
    if "likelihood" in statement and "parameter is true" in statement:
        issues.append(...)
    if "guaranteed" in statement and "confidence" in statement:
        issues.append(...)
    return issues

def E09(claims):
    return [
        {"claim_id": item["id"], "issues": validate_claim(item)}
        for item in claims
    ]

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

11.5.11 Exercise 10 - Bayesian View Comparison

Part 1 - Why this matters

Classical logistic regression commonly produces one fitted parameter vector and one probability for each input. A Bayesian treatment instead represents uncertainty about the parameters and propagates it into a distribution of plausible predictions.

The two viewpoints answer different questions and should not be mixed casually. Variation across posterior parameter samples represents epistemic uncertainty under the chosen prior and model, while the Bernoulli output still represents outcome uncertainty conditional on each sampled parameter.

Part 2 - Mathematical form

A point estimate gives

$$p_{\text{point}}(y = 1 \mid \mathbf{x}) = \sigma(\widehat{\mathbf{w}}^{\mathsf{T}}\mathbf{x} + \widehat{b}).$$

With posterior samples $\theta^{(s)} = (\mathbf{w}^{(s)}, b^{(s)})$,

$$p_{\text{Bayes}}(y = 1 \mid \mathbf{x}, \mathcal{D}) \approx \frac{1}{S}\sum_{s=1}^{S} \sigma\left(\mathbf{w}^{(s)\mathsf{T}}\mathbf{x} + b^{(s)}\right).$$

Part 3 - Hints and Helper

Receive already generated parameter samples rather than pretending that random perturbations are a valid posterior. Compute every sampled probability, the posterior-predictive mean,

and a spread measure. Compare them with the point-estimate probability and state what the spread does and does not mean.

Part 4 - Python code skeleton

```
import torch

def E10(features, point_weight, point_bias,
        weight_samples, bias_samples):
    point_logit = features @ point_weight + point_bias
    point_probability = torch.sigmoid(point_logit)

    sampled_logits = (
        features @ weight_samples.T
        + bias_samples.reshape(1, -1)
    )
    sampled_probabilities = torch.sigmoid(...)
    return {
        "point_probability": ...,
        "predictive_mean": ...,
        "predictive_std": ...,
        "number_of_samples": ...,
    }

def main():
    print(E10(...))

if __name__ == "__main__":
    main()
```

11.5.12 Exercise 11 - Prior Effect Simulation

Part 1 - Why this matters

A model may be trained where positive examples have one prevalence and deployed where that prevalence differs. If the class-conditional evidence remains stable, changing prior odds should shift posterior odds even when the input features are unchanged.

This correction is conditional, not a universal calibration trick. It assumes prior-probability shift: the class prior changes while the feature distributions within each class remain sufficiently stable. The exercise must report that assumption beside the adjusted values.

Part 2 - Mathematical form

Let π_{train} and π_{new} be old and new positive-class priors. Under prior shift,

$$\operatorname{logit}(p_{\text{new}}) = \operatorname{logit}(p_{\text{train}}) - \operatorname{logit}(\pi_{\text{train}}) + \operatorname{logit}(\pi_{\text{new}}),$$

where

$$\operatorname{logit}(q) = \log \frac{q}{1-q}.$$

Part 3 - Hints and Helper

Validate that both priors lie strictly between zero and one. Work in log-odds space, then apply sigmoid once. Test no-change, lower-prevalence, and higher-prevalence scenarios while preserving the original probabilities in the report.

Part 4 - Python code skeleton

```
import torch

def safe_logit(probability, epsilon):
    probability = probability.clamp(
        min=epsilon, max=1.0 - epsilon
    )
    return torch.log(probability) - torch.log1p(-probability)

def E11(probabilities, train_prior, new_prior, epsilon):
    if not (0.0 < train_prior < 1.0):
        raise ValueError(...)
    if not (0.0 < new_prior < 1.0):
        raise ValueError(...)
    train_prior_tensor = probabilities.new_tensor(train_prior)
    new_prior_tensor = probabilities.new_tensor(new_prior)
    adjusted_logits = (
        safe_logit(probabilities, epsilon)
        - safe_logit(train_prior_tensor, epsilon)
        + safe_logit(new_prior_tensor, epsilon)
    )
    return {
        "original": probabilities,
        "adjusted": torch.sigmoid(...),
        "assumption": ...,
    }

def main():
    print(E11(...))

if __name__ == "__main__":
    main()
```

11.5.13 Exercise 12 - Data Imbalance Impact

Part 1 - Why this matters

On an imbalanced dataset, total likelihood and overall mean loss are influenced mainly by the majority class because that class supplies most observations. A model can therefore improve the global objective while providing weak probabilities for the minority class.

Class-specific summaries reveal this hidden behavior. They do not replace the original likelihood automatically; instead, they make the chosen evaluation or weighting policy explicit and show which population contributes each part of the objective.

Part 2 - Mathematical form

For class $c \in \{0, 1\}$,

$$\overline{\mathrm{NLL}}_c = \frac{1}{N_c} \sum_{i:y_i=c} -\ell_i.$$

A balanced diagnostic summary is

$$\mathrm{NLL}_{\mathrm{balanced}} = \frac{1}{2} \left(\overline{\mathrm{NLL}}_0 + \overline{\mathrm{NLL}}_1 \right).$$

Part 3 - Hints and Helper

Compute unreduced stable NLL terms, class counts, class-wise means, the ordinary mean, and the balanced diagnostic mean. Handle an absent class explicitly instead of returning a silent division by zero.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def class_mean(values, mask):
    count = int(mask.sum().item())
    if count == 0:
        return ..., count
    return ..., count

def E12(logits, targets):
    terms = F.binary_cross_entropy_with_logits(
        logits, targets, reduction="none"
    )
    negative_mean, negative_count = class_mean(
        terms, targets == 0
    )
    positive_mean, positive_count = class_mean(
        terms, targets == 1
    )
    balanced = ...
    return {
        "ordinary_mean": ...,
        "balanced_mean": balanced,
        "class_0": ...,
        "class_1": ...,
    }

def main():
```

```
    print(E12(...))

if __name__ == "__main__":
    main()
```

11.5.14 Exercise 13 - Likelihood under Noise

Part 1 - Why this matters

Real inputs contain measurement errors, sensor variation, missing precision, and preprocessing noise. A probability interpretation is trustworthy only when the reader understands how these perturbations alter the evidence assigned to observed labels.

One noisy draw is not enough to characterize sensitivity. Repeated perturbations with controlled random generators produce a distribution of NLL changes, allowing the experiment to separate systematic degradation from random variation.

Part 2 - Mathematical form

For perturbation $\boldsymbol{\varepsilon}^{(r)}$,

$$\widetilde{\mathbf{x}}_i^{(r)} = \mathbf{x}_i + \boldsymbol{\varepsilon}_i^{(r)}, \qquad \boldsymbol{\varepsilon}_i^{(r)} \sim \mathcal{N}(\mathbf{0}, s^2\mathbf{I}).$$

Record

$$\Delta_r = \mathrm{NLL}(\widetilde{\mathbf{X}}^{(r)}) - \mathrm{NLL}(\mathbf{X}).$$

Part 3 - Hints and Helper

Evaluate the clean baseline once, then use a seeded generator for repeated perturbations. Keep the model in evaluation mode and disable gradients. Report the mean, standard deviation, maximum, and raw NLL changes for each noise scale.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def mean_nll(model, features, targets):
    logits = model(features).reshape_as(targets)
    return F.binary_cross_entropy_with_logits(
        logits, targets, reduction="mean"
    )

def E13(model, features, targets, noise_scale,
        repetitions, seed):
    model.eval()
    generator = torch.Generator(device=features.device)
    generator.manual_seed(seed)
    changes = []
    with torch.no_grad():
        baseline = mean_nll(model, features, targets)
```

```
        for _ in range(repetitions):
            noise = torch.randn(
                features.shape,
                dtype=features.dtype,
                device=features.device,
                generator=generator,
            )
            noisy_nll = mean_nll(
                model, features + noise_scale * noise, targets
            )
            changes.append(...)
    changes = torch.stack(changes)
    return {
        "baseline": ...,
        "mean_change": ...,
        "std_change": ...,
        "maximum_change": ...,
        "changes": ...,
    }

def main():
    print(E13(...))

if __name__ == "__main__":
    main()
```

11.5.15 Exercise 14 - Probability Drift over Epochs

Part 1 - Why this matters

Predicted probabilities for a fixed probe set can move substantially during training even when class labels remain unchanged. Monitoring this movement reveals whether the model is settling, oscillating, or becoming increasingly extreme.

This is model-output drift across epochs, not automatically evidence of external data drift. The probe inputs must stay fixed, the sample order must be preserved, and the report should distinguish changes in probabilities from changes in thresholded decisions.

Part 2 - Mathematical form

For fixed probes and epoch t,

$$d_t = \frac{1}{N} \sum_{i=1}^{N} \left| p_i^{(t)} - p_i^{(t-1)} \right|.$$

A complementary decision summary is

$$r_t = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}[p_i^{(t)} \geq \tau].$$

Part 3 - Hints and Helper

After each epoch, evaluate the same probe tensor without gradients and store a detached CPU copy. Compare adjacent epochs by sample index. Record mean absolute drift, maximum drift, mean probability, and predicted-positive rate.

Part 4 - Python code skeleton

```
import torch

def snapshot(model, probe_features):
    model.eval()
    with torch.no_grad():
        return torch.sigmoid(
            model(probe_features)
        ).flatten().cpu()

def E14(probability_history, threshold):
    rows = []
    previous = None
    for epoch, probabilities in enumerate(probability_history):
        if previous is None:
            mean_drift = ...
            maximum_drift = ...
        else:
            difference = (probabilities - previous).abs()
            mean_drift = ...
            maximum_drift = ...
        rows.append({
            "epoch": epoch,
            "mean_probability": ...,
            "positive_rate": ...,
            "mean_drift": mean_drift,
            "maximum_drift": maximum_drift,
        })
        previous = probabilities
    return rows

def main():
    print(E14(...))

if __name__ == "__main__":
    main()
```

11.5.16 Exercise 15 - Numerical Precision Effects

Part 1 - Why this matters

Likelihood products may underflow even when every individual probability is representable. Lower precision reaches this failure sooner, while computing in log-space preserves a useful

finite summary over a much wider range.

Higher precision is not a replacement for a stable formulation. This experiment should compare both dtype and representation so the reader can see that float64 delays underflow, whereas summed stable log-likelihood addresses the structural cause.

Part 2 - Mathematical form

For repeated observed-label probability q,

$$L = q^N, \qquad \log L = N \log q.$$

The direct product may become zero numerically even while $\log L$ remains finite. From logits, stable NLL avoids explicitly computing $\log(1 - \sigma(z))$ in saturated regions.

Part 3 - Hints and Helper

Evaluate the same cases in float32 and float64. Compare direct probability products, sums of logarithms, and logits-based stable NLL. Record finiteness, zeros, and absolute differences after converting comparable results to float64.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def evaluate_dtype(logit_values, targets, dtype):
    logits = logit_values.to(dtype)
    targets = targets.to(dtype)
    probabilities = torch.sigmoid(logits)
    observed_probabilities = torch.where(
        targets == 1, probabilities, 1.0 - probabilities
    )
    direct_product = ...
    probability_log_sum = ...
    stable_nll = F.binary_cross_entropy_with_logits(
        logits, targets, reduction="sum"
    )
    return {
        "dtype": str(dtype),
        "direct_product": ...,
        "log_sum": ...,
        "stable_nll": ...,
        "all_finite": ...,
    }

def E15(logits, targets):
    report32 = evaluate_dtype(logits, targets, torch.float32)
    report64 = evaluate_dtype(logits, targets, torch.float64)
    return report32, report64, ...
```

```
def main():
    print(E15(...))

if __name__ == "__main__":
    main()
```

11.5.17 Exercise 16 - Likelihood Visualization

Part 1 - Why this matters

A likelihood visualization makes parameter fitting visible: candidate parameter values receive different support from the same fixed observations. Plotting log-likelihood rather than a tiny raw product preserves structure that would otherwise collapse to zero.

The graph must state what varies and what remains fixed. In a one-parameter scan, the data and bias are fixed while the weight changes; the highest point is the best candidate only within the scanned range and model assumptions.

Part 2 - Mathematical form

For candidate weight w with fixed bias b,

$$\ell(w) = \sum_{i=1}^{N} \left[y_i \log \sigma(wx_i + b) + (1 - y_i) \log(1 - \sigma(wx_i + b)) \right].$$

The scanned estimate is

$$\widehat{w}_{\text{grid}} = \arg\max_{w \in \mathcal{G}} \ell(w).$$

Part 3 - Hints and Helper

Create an ordered grid with `torch.linspace`. For each candidate, form logits and calculate log-likelihood as negative summed stable BCE. Move detached values to CPU for plotting, label axes, and mark the maximum grid point.

Part 4 - Python code skeleton

```
import matplotlib.pyplot as plt
import torch
import torch.nn.functional as F

def E16(features, targets, bias, start, end,
        steps, output_path):
    weight_grid = torch.linspace(
        start, end, steps=steps,
        dtype=features.dtype,
        device=features.device,
    )
    values = []
    with torch.no_grad():
        for weight in weight_grid:
```

```
            logits = weight * features + bias
            log_likelihood = -F.binary_cross_entropy_with_logits(
                logits, targets, reduction="sum"
            )
            values.append(log_likelihood)
    values = torch.stack(values)
    best_index = ...
    fig, ax = plt.subplots()
    ax.plot(..., ...)
    ax.scatter(..., ...)
    ax.set_xlabel(...)
    ax.set_ylabel(...)
    fig.savefig(output_path, dpi=150, bbox_inches="tight")
    plt.close(fig)
    return ...

def main():
    print(E16(...))

if __name__ == "__main__":
    main()
```

11.5.18 Exercise 17 - Probability Pipeline Debugging

Part 1 - Why this matters

Probability failures often occur at interfaces rather than inside sigmoid itself. A pipeline may apply sigmoid twice, pair misaligned samples, mix devices, accept invalid labels, average batch means incorrectly, or pass probabilities to a logits-based loss.

A robust debugger should stop near the first violated contract and preserve evidence. Shape, dtype, device, representation, range, finiteness, and reduction semantics must be inspected before interpreting any metric.

Part 2 - Mathematical form

A valid binary pipeline should maintain

$$\mathbf{z}, \mathbf{y} \in \mathbb{R}^N, \qquad y_i \in \{0, 1\}, \qquad p_i = \sigma(z_i) \in [0, 1].$$

For unequal batches,

$$\overline{\mathrm{NLL}} = \frac{\sum_k \mathrm{NLL}_{k,\mathrm{sum}}}{\sum_k N_k},$$

not an unweighted mean of batch means.

Part 3 - Hints and Helper

Build validation functions around named representations: logits, probabilities, targets, and losses. Never infer representation only from a convenient value range because valid logits can also lie between zero and one. Include actionable error messages.

Part 4 - Python code skeleton

```
import torch

def inspect_tensor(name, tensor, expected_shape=None):
    issues = []
    if not tensor.is_floating_point():
        issues.append(...)
    if not torch.isfinite(tensor).all():
        issues.append(...)
    if expected_shape is not None:
        if tensor.shape != expected_shape:
            issues.append(...)
    return {"name": name, "issues": issues}

def E17(logits, probabilities, targets, metadata):
    report = []
    report.append(inspect_tensor("logits", logits))
    report.append(
        inspect_tensor(
            "probabilities",
            probabilities,
            expected_shape=targets.shape,
        )
    )
    if logits.device != targets.device:
        report.append(...)
    if not ((probabilities >= 0) & (probabilities <= 1)).all():
        report.append(...)
    if not torch.all((targets == 0) | (targets == 1)):
        report.append(...)
    if metadata["loss_input"] != "logits":
        report.append(...)
    if metadata["aggregation"] == "mean_of_batch_means":
        report.append(...)
    return report

def main():
    print(E17(...))

if __name__ == "__main__":
    main()
```

11.5.19 Exercise 18 - Confidence Failure Detection

Part 1 - Why this matters

Confidence failures include more than high-confidence mistakes. A model can be systematically overconfident, underconfident, unstable under small perturbations, or extremely certain on inputs unlike those used for evaluation.

Detection rules should separate observed evidence from explanations. A high-confidence error is directly measurable, while its cause may involve data shift, label noise, leakage, or model misspecification and requires additional investigation.

Part 2 - Mathematical form

For predicted-class confidence c_i and correctness a_i,

$$c_i = \max(p_i, 1 - p_i), \qquad a_i = \mathbb{I}[\widehat{y}_i = y_i].$$

For a selected region S,

$$g_S = \frac{1}{|S|} \sum_{i \in S} c_i - \frac{1}{|S|} \sum_{i \in S} a_i.$$

A positive gap indicates empirical overconfidence in that region.

Part 3 - Hints and Helper

Combine high-confidence error detection with confidence-bin gaps and optional paired probabilities from perturbed inputs. Return sample-level evidence and aggregate counts. Use a minimum bin count so tiny groups are not promoted as strong conclusions.

Part 4 - Python code skeleton

```
import torch

def E18(probabilities, targets, threshold,
        confidence_cutoff, minimum_count):
    predictions = (probabilities >= threshold).to(targets.dtype)
    correctness = (predictions == targets).to(
        probabilities.dtype
    )
    confidence = torch.maximum(
        probabilities, 1.0 - probabilities
    )
    high_confidence = confidence >= confidence_cutoff
    high_confidence_errors = high_confidence & (
        correctness == 0
    )
    selected_count = int(high_confidence.sum().item())
    if selected_count >= minimum_count:
        regional_gap = (
            confidence[high_confidence].mean()
            - correctness[high_confidence].mean()
        )
    else:
        regional_gap = ...
    return {
        "error_indices": ...,
        "selected_count": selected_count,
        "regional_gap": ...,
        "evidence_strength": ...,
```

```
    }

def main():
    print(E18(...))

if __name__ == "__main__":
    main()
```

11.5.20 Exercise 19 - Final Probability Analysis Report

Part 1 - Why this matters

A final probability report should connect statistical fit, numerical stability, calibration, confidence failures, class imbalance, and output drift. No single number can establish that a probabilistic classifier is trustworthy.

The report must preserve assumptions and reproducibility details beside the measurements. Thresholds, binning rules, class counts, reduction modes, dtypes, devices, and any prior-shift assumption are part of the result rather than optional notes.

Part 2 - Mathematical form

A structured report may be represented as

$$R = \{\text{data}, \text{likelihood}, \text{calibration}, \text{confidence}, \text{drift}, \text{numerics}, \text{assumptions}, \text{status}\}.$$

A failure status should follow explicit checks, while unsupported sections should be marked unavailable rather than silently omitted.

Part 3 - Hints and Helper

Accept precomputed section results so this exercise focuses on integration rather than duplicating every earlier calculation. Validate required keys, convert tensors to serializable values, preserve warnings, and determine final status from declared rules.

Part 4 - Python code skeleton

```
import json

import torch

def serializable(value):
    if isinstance(value, torch.Tensor):
        if value.numel() == 1:
            return value.item()
        return value.detach().cpu().tolist()
    if isinstance(value, dict):
        return {key: serializable(item)
                for key, item in value.items()}
    if isinstance(value, list):
        return [serializable(item) for item in value]
```

```
    return value

def E19(sections, rules, output_path):
    required = {
        "data",
        "likelihood",
        "calibration",
        "confidence",
        "drift",
        "numerics",
        "assumptions",
    }
    missing = required - sections.keys()
    warnings = []
    if missing:
        warnings.append(...)
    if sections.get("calibration", {}).get("ece", 0.0) > \
            rules["maximum_ece"]:
        warnings.append(...)
    if sections.get("confidence", {}).get(
            "high_confidence_errors", 0
    ) > rules["maximum_high_confidence_errors"]:
        warnings.append(...)
    report = {
        "sections": serializable(sections),
        "warnings": warnings,
        "status": ...,
    }
    with open(output_path, "w", encoding="utf-8") as file:
        json.dump(report, file, indent=2)
    return report

def main():
    print(E19(...))

if __name__ == "__main__":
    main()
```

11.6 LG04 - Classification Boundary Definition

Models and Training

11.6.1 Exercise 00 - Linear Decision Boundary Plot

Part 1 - Why this matters

A decision-boundary plot converts the algebra of logistic regression into a visible division of feature space. It shows where the model changes its predicted class and allows the reader to

compare that division with the actual distribution of labels.

The plotted line is determined by the learned parameters and the chosen probability threshold. It is not a line fitted independently for visualization, and it does not prove that the classes are truly separable.

Part 2 - Mathematical form

For two features,

$$z = w_1 x_1 + w_2 x_2 + b, \qquad p = \sigma(z).$$

At threshold $\tau = 0.5$, the boundary satisfies $z = 0$, so when $w_2 \neq 0$,

$$x_2 = -\frac{w_1 x_1 + b}{w_2}.$$

Points on opposite sides have logits with opposite signs.

Part 3 - Hints and Helper

Scatter the two classes with different colors. Generate ordered x_1 values across the observed range and calculate the matching x_2 values. Detect the special case $w_2 \approx 0$, where the boundary is vertical instead of dividing by a nearly zero value.

Part 4 - Python code skeleton

```
import matplotlib.pyplot as plt
import torch

def E00(features, targets, weight, bias, output_path):
    class_zero = targets == 0
    class_one = targets == 1
    fig, ax = plt.subplots()
    ax.scatter(..., ..., label="Class 0")
    ax.scatter(..., ..., label="Class 1")

    tolerance = ...
    if weight[1].abs() > tolerance:
        x_values = torch.linspace(
            features[:, 0].min(),
            features[:, 0].max(),
            steps=...,
        )
        y_values = ...
        ax.plot(x_values.cpu(), y_values.cpu(),
                label="Decision boundary")
    else:
        vertical_x = ...
        ax.axvline(vertical_x.item(),
                   label="Decision boundary")

    ax.set_xlabel(...)
```

```
    ax.set_ylabel(...)
    ax.legend()
    fig.savefig(output_path, dpi=150, bbox_inches="tight")
    plt.close(fig)

def main():
    E00(...)

if __name__ == "__main__":
    main()
```

11.6.2 Exercise 01 - Boundary Shift with Bias

Part 1 - Why this matters

The bias term controls where a linear boundary is positioned even when its orientation remains fixed. Changing only the bias translates the boundary parallel to itself and changes the baseline tendency to predict the positive class.

Observing several bias values prevents the intercept from being treated as an unimportant extra parameter. A large shift may change many classifications even though every feature weight remains exactly the same.

Part 2 - Mathematical form

For threshold 0.5, boundaries with fixed weight $\mathbf{w}$ and biases b_a, b_b satisfy

$$\mathbf{w}^\mathsf{T}\mathbf{x} + b_a = 0, \qquad \mathbf{w}^\mathsf{T}\mathbf{x} + b_b = 0.$$

Their perpendicular displacement is

$$d = \frac{|b_b - b_a|}{\|\mathbf{w}\|_2}.$$

Part 3 - Hints and Helper

Plot several parallel boundaries on the same axes and label each bias. Calculate the theoretical perpendicular displacement relative to a reference bias. Reject an all-zero weight vector because it does not define a geometric hyperplane.

Part 4 - Python code skeleton

```
import matplotlib.pyplot as plt
import torch

def boundary_values(x_values, weight, bias):
    if weight[1].abs() <= ...:
        raise ValueError(...)
    return ...

def E01(x_values, weight, biases, output_path):
```

```
    if torch.linalg.vector_norm(weight) <= ...:
        raise ValueError(...)
    reference_bias = biases[0]
    rows = []
    fig, ax = plt.subplots()
    for bias in biases:
        y_values = boundary_values(
            x_values, weight, bias
        )
        displacement = (
            torch.abs(bias - reference_bias)
            / torch.linalg.vector_norm(weight)
        )
        rows.append(...)
        ax.plot(..., ..., label=...)
    ax.legend()
    fig.savefig(output_path, dpi=150, bbox_inches="tight")
    plt.close(fig)
    return rows

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

11.6.3 Exercise 02 - Multi-Feature Boundary Visualization

Part 1 - Why this matters

A logistic model with more than two features still has a linear decision boundary, but that boundary is a hyperplane that cannot be drawn directly on a two-dimensional page. A projection or slice is therefore required for visualization.

Every projection discards information. The reader must state whether the graph fixes unshown features, projects data and weights into a lower-dimensional basis, or displays only selected coordinates; otherwise the picture can misrepresent the original classifier.

Part 2 - Mathematical form

In d dimensions, the threshold-0.5 boundary is

$$\mathcal{H} = \left\{ \mathbf{x} \in \mathbb{R}^d : \mathbf{w}^\mathsf{T}\mathbf{x} + b = 0 \right\}.$$

For a projection basis $\mathbf{P} \in \mathbb{R}^{d\times 2}$ and reference point $\mathbf{x}_0$,

$$\mathbf{x} = \mathbf{x}_0 + \mathbf{P}\mathbf{u}, \qquad (\mathbf{P}^\mathsf{T}\mathbf{w})^\mathsf{T}\mathbf{u} + \mathbf{w}^\mathsf{T}\mathbf{x}_0 + b = 0.$$

Part 3 - Hints and Helper

Require two orthonormal projection directions or verify them numerically. Project samples for the scatter plot, transform the weight into the same basis, and include the fixed reference point in the effective bias. Report how much input variance the chosen view preserves if available.

Part 4 - Python code skeleton

```
import matplotlib.pyplot as plt
import torch

def E02(features, targets, weight, bias,
        projection, reference, output_path):
    if projection.shape != (features.shape[1], 2):
        raise ValueError(...)
    gram = projection.T @ projection
    if not torch.allclose(
        gram,
        torch.eye(2, dtype=gram.dtype,
                  device=gram.device),
        atol=...,
    ):
        raise ValueError(...)

    coordinates = (
        features - reference
    ) @ projection
    projected_weight = projection.T @ weight
    effective_bias = weight @ reference + bias
    fig, ax = plt.subplots()
    ax.scatter(..., ..., c=targets.cpu())
    if projected_weight[1].abs() > ...:
        x_values = torch.linspace(..., ..., steps=...)
        y_values = ...
        ax.plot(..., ...)
    else:
        ax.axvline(...)
    fig.savefig(output_path, dpi=150, bbox_inches="tight")
    plt.close(fig)
    return ...

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

11.6.4 Exercise 03 - Boundary on CUDA Computation

Part 1 - Why this matters

A large decision surface may require millions of grid points, making vectorized GPU evaluation useful. The mathematical boundary is unchanged by the device, but tensor placement, synchronization, and memory allocation affect whether the experiment is correct and efficient.

CUDA should calculate the grid in batches when a full mesh would exceed available memory. The final plotting data can be transferred to CPU after computation; repeatedly moving small tensors during the inner loop can erase the performance benefit.

Part 2 - Mathematical form

For grid points $\mathbf{X}_{\text{grid}}$,

$$\mathbf{z} = \mathbf{X}_{\text{grid}}\mathbf{w} + b, \qquad \mathbf{p} = \sigma(\mathbf{z}).$$

The boundary contour for threshold τ is the level set

$$\mathbf{p} = \tau \quad \Longleftrightarrow \quad \mathbf{z} = \log \frac{\tau}{1-\tau}.$$

Part 3 - Hints and Helper

Choose CUDA only when available and create the grid directly on the selected device. Evaluate flattened grid points in chunks, concatenate results on that device, and transfer one final probability surface to CPU. Keep a CPU result for a numerical parity check.

Part 4 - Python code skeleton

```
import torch

def evaluate_grid(weight, bias, grid_points,
                  chunk_size):
    outputs = []
    with torch.no_grad():
        for start in range(0, grid_points.shape[0],
                           chunk_size):
            chunk = grid_points[
                start:start + chunk_size
            ]
            logits = chunk @ weight + bias
            outputs.append(torch.sigmoid(...))
    return torch.cat(outputs)

def E03(weight, bias, x_axis, y_axis, chunk_size):
    device = torch.device(
        "cuda" if torch.cuda.is_available() else "cpu"
    )
    grid_x, grid_y = torch.meshgrid(
        x_axis.to(device),
        y_axis.to(device),
        indexing="xy",
    )
    grid_points = torch.stack(
```

```
        [grid_x.flatten(), grid_y.flatten()],
        dim=1,
    )
    probabilities = evaluate_grid(
        weight.to(device),
        bias.to(device),
        grid_points,
        chunk_size,
    )
    return {
        "device": str(device),
        "shape": ...,
        "surface": ...,
    }

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

11.6.5 Exercise 04 - Margin Concept Experiment

Part 1 - Why this matters

The signed logit tells which side of the boundary contains a sample, but its magnitude depends on the scale of the weight vector. Dividing by the weight norm produces a geometric signed distance to the hyperplane.

Logistic regression does not maximize a hard margin in the same way as a support-vector machine. This experiment uses margin as an interpretation and robustness diagnostic, not as a claim that logistic regression learned a maximum-margin solution.

Part 2 - Mathematical form

For the threshold-0.5 hyperplane,

$$m_i = \frac{\mathbf{w}^\mathsf{T}\mathbf{x}_i + b}{\|\mathbf{w}\|_2}$$

is the signed geometric margin. A label-oriented margin is

$$\widetilde{m}_i = (2y_i - 1)m_i.$$

It is positive when the sample is correctly placed relative to the boundary.

Part 3 - Hints and Helper

Calculate logits, signed distance, and label-oriented margin. Compare samples with equal logits under rescaled weights and biases to see why normalization matters. Reject a zero-norm weight vector.

Part 4 - Python code skeleton

```
import torch

def E04(features, targets, weight, bias):
    weight_norm = torch.linalg.vector_norm(weight)
    if weight_norm <= ...:
        raise ValueError(...)
    logits = features @ weight + bias
    signed_distance = ...
    signed_labels = 2.0 * targets - 1.0
    label_margin = ...
    return {
        "logits": logits,
        "signed_distance": signed_distance,
        "label_margin": label_margin,
        "minimum_margin": ...,
        "median_margin": ...,
    }

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

11.6.6 Exercise 05 - Overlapping Classes Scenario

Part 1 - Why this matters

Real classes often overlap, so no linear boundary can classify every sample correctly. In that situation the model represents a probability transition through the shared region rather than discovering a perfect separator.

Forcing the reader to inspect overlap prevents accuracy failures from being blamed automatically on code. The limitation may come from the available features, label noise, or an inappropriate linear assumption rather than from optimization.

Part 2 - Mathematical form

For class-conditional data distributions with shared support,

$$P(\mathbf{X} \in A \mid Y = 0) > 0, \qquad P(\mathbf{X} \in A \mid Y = 1) > 0$$

for some region A. Any deterministic classifier must then accept nonzero irreducible error within that representation. Logistic regression estimates

$$P(Y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^{\mathsf{T}}\mathbf{x} + b).$$

Part 3 - Hints and Helper

Generate two clouds with adjustable separation and noise. Train or receive a fitted linear model, then report accuracy, NLL, and errors inside a chosen overlap region. Plot probabilities or a contour instead of pretending the boundary explains all uncertainty.

Part 4 - Python code skeleton

```
import torch

def generate_overlap(samples_per_class, separation,
                     noise_scale, generator):
    class_zero = ...
    class_one = ...
    features = torch.cat(
        [class_zero, class_one], dim=0
    )
    targets = torch.cat([
        torch.zeros(samples_per_class),
        torch.ones(samples_per_class),
    ])
    return features, targets

def E05(model, features, targets, threshold):
    with torch.no_grad():
        logits = model(features).flatten()
        probabilities = torch.sigmoid(logits)
        predictions = (
            probabilities >= threshold
        ).to(targets.dtype)
    error_mask = predictions != targets
    return {
        "accuracy": ...,
        "error_count": ...,
        "mean_probability_class_0": ...,
        "mean_probability_class_1": ...,
        "error_indices": ...,
    }

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

11.6.7 Exercise 06 - Noise Impact on Boundary

Part 1 - Why this matters

Noise in training features or labels can rotate and translate a learned boundary. The resulting change may be small in parameter space yet large near densely populated regions, where many predictions sit close to the decision surface.

A controlled experiment should separate input noise, label noise, and random training variation. Changing all three at once makes it impossible to attribute the observed boundary movement to a specific source.

Part 2 - Mathematical form

For clean parameters $(\mathbf{w}, b)$ and noisy-data parameters $(\widetilde{\mathbf{w}}, \widetilde{b})$, orientation change can be measured by

$$\alpha = \arccos\left(\frac{\mathbf{w}^{\mathsf{T}}\widetilde{\mathbf{w}}}{\|\mathbf{w}\|_2\|\widetilde{\mathbf{w}}\|_2}\right).$$

Prediction disagreement on probe points is

$$r = \frac{1}{N}\sum_i \mathbb{I}[\widehat{y}_i \neq \widetilde{y}_i].$$

Part 3 - Hints and Helper

Use the same initialization and training settings for clean and noisy datasets. Clamp the cosine value before `arccos` to protect against rounding. Compare angle, bias change, probability change, and decision disagreement on one fixed probe set.

Part 4 - Python code skeleton

```
import torch

def boundary_comparison(clean_weight, clean_bias,
                        noisy_weight, noisy_bias,
                        probe_features, threshold):
    denominator = (
        torch.linalg.vector_norm(clean_weight)
        * torch.linalg.vector_norm(noisy_weight)
    )
    if denominator <= ...:
        raise ValueError(...)
    cosine = (
        clean_weight @ noisy_weight / denominator
    ).clamp(-1.0, 1.0)
    angle = torch.arccos(cosine)

    clean_probabilities = torch.sigmoid(
        probe_features @ clean_weight + clean_bias
    )
    noisy_probabilities = torch.sigmoid(
        probe_features @ noisy_weight + noisy_bias
    )
    disagreement = (
        (clean_probabilities >= threshold)
        != (noisy_probabilities >= threshold)
    ).float().mean()
    return angle, disagreement, ...
```

```
def E06(...):
    return boundary_comparison(...)

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

11.6.8 Exercise 07 - Boundary Stability Test

Part 1 - Why this matters

A reliable boundary should not change unpredictably when training is repeated with small changes in sampling or initialization. Stability analysis measures whether the learned decision rule is supported consistently by the available data.

Parameter differences alone can be deceptive because scaling or correlated features may alter coefficients without changing many decisions. A useful test combines parameter statistics with probability and decision agreement on fixed reference inputs.

Part 2 - Mathematical form

For R trained models and probe $\mathbf{x}_i$, let

$$p_i^{(r)} = \sigma\left(\mathbf{w}^{(r)\mathsf{T}}\mathbf{x}_i + b^{(r)}\right).$$

Pointwise probability variability is

$$s_i = \mathrm{Std}_r\left(p_i^{(r)}\right),$$

and boundary instability can be summarized by prediction disagreement across runs.

Part 3 - Hints and Helper

Evaluate every model on the same ordered probe tensor. Stack probabilities with shape runs by samples, then compute per-sample standard deviation and the fraction of runs disagreeing with the majority label. Preserve seeds and model states in the report.

Part 4 - Python code skeleton

```
import torch

def E07(models, probe_features, threshold):
    probability_runs = []
    with torch.no_grad():
        for model in models:
            model.eval()
            logits = model(probe_features).flatten()
            probability_runs.append(torch.sigmoid(logits))
    probabilities = torch.stack(
```

```
        probability_runs, dim=0
    )
    predictions = probabilities >= threshold
    positive_votes = predictions.float().mean(dim=0)
    majority = positive_votes >= 0.5
    disagreement = (
        predictions != majority.unsqueeze(0)
    ).float().mean(dim=0)
    return {
        "mean_probability": ...,
        "probability_std": ...,
        "mean_disagreement": ...,
        "maximum_disagreement": ...,
    }

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

11.6.9 Exercise 08 - Misclassified Region Detection

Part 1 - Why this matters

Listing individual mistakes is useful, but identifying regions where mistakes concentrate reveals structure. A cluster of errors may indicate overlap, missing nonlinear interactions, poor scaling, or a subgroup that the model does not represent well.

The region definition must be explicit and reproducible. Grid cells, geometric neighborhoods, or feature ranges can all be used, but the report must include support counts so a region with one error is not treated like a region with many observations.

Part 2 - Mathematical form

For region R_j, define

$$N_j = \sum_i \mathbb{I}[\mathbf{x}_i \in R_j], \qquad E_j = \sum_i \mathbb{I}[\mathbf{x}_i \in R_j \wedge \widehat{y}_i \neq y_i].$$

The regional error rate is

$$e_j = \frac{E_j}{N_j}$$

when $N_j > 0$.

Part 3 - Hints and Helper

Partition two selected features with fixed bin edges. Convert each sample to a cell index, accumulate support and error counts, and filter the final ranking by a minimum support. Return boundaries and counts with every region.

Part 4 - Python code skeleton

```
import torch

def E08(features, targets, probabilities,
        x_edges, y_edges, threshold,
        minimum_support):
    predictions = (
        probabilities >= threshold
    ).to(targets.dtype)
    errors = predictions != targets
    x_bins = torch.bucketize(
        features[:, 0], x_edges[1:-1]
    )
    y_bins = torch.bucketize(
        features[:, 1], y_edges[1:-1]
    )
    rows = []
    for x_bin in range(x_edges.numel() - 1):
        for y_bin in range(y_edges.numel() - 1):
            mask = (x_bins == x_bin) & (y_bins == y_bin)
            support = int(mask.sum().item())
            if support < minimum_support:
                continue
            error_count = int((errors & mask).sum().item())
            rows.append({
                "x_bin": x_bin,
                "y_bin": y_bin,
                "support": support,
                "error_count": error_count,
                "error_rate": ...,
            })
    return sorted(rows, key=..., reverse=True)

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

11.6.10 Exercise 09 - Boundary Failure Case

Part 1 - Why this matters

A boundary can fail because of invalid shapes, non-finite parameters, zero weights, device mismatches, extreme scaling, or a dataset that requires nonlinear structure. These failures require different responses and should not be collapsed into one generic error.

A diagnostic case should preserve the evidence that distinguishes computational failure from modeling failure. Valid code can still produce a poor boundary, while invalid tensors can prevent any meaningful boundary from being defined.

Part 2 - Mathematical form

A linear boundary exists geometrically when

$$\|\mathbf{w}\|_2 > 0 \quad \text{and} \quad \mathbf{w}, b \text{ are finite.}$$

Given valid parameters, model failure on evaluation data may be summarized by

$$\widehat{R} = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}[\mathbb{I}[\mathbf{w}^{\mathsf{T}} \mathbf{x}_i + b \geq 0] \neq y_i].$$

Part 3 - Hints and Helper

Validate rank, feature count, dtype, device, and finiteness before calculating predictions. Classify results into invalid input, degenerate boundary, numerically risky, or valid but inaccurate. Do not use low accuracy as proof of a software bug.

Part 4 - Python code skeleton

```
import torch

def E09(features, targets, weight, bias,
        maximum_error_rate):
    issues = []
    if features.ndim != 2:
        issues.append(...)
    if weight.ndim != 1:
        issues.append(...)
    if features.shape[-1] != weight.numel():
        issues.append(...)
    if features.device != weight.device:
        issues.append(...)
    if not torch.isfinite(features).all():
        issues.append(...)
    if not torch.isfinite(weight).all():
        issues.append(...)
    if torch.linalg.vector_norm(weight) <= ...:
        issues.append(...)
    if issues:
        return {"status": ..., "issues": issues}

    logits = features @ weight + bias
    predictions = (logits >= 0).to(targets.dtype)
    error_rate = (predictions != targets).float().mean()
    return {
        "status": ...,
        "error_rate": error_rate.item(),
        "issues": ...,
    }
```

```
def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

11.6.11 Exercise 10 - High-Dimensional Boundary Projection

Part 1 - Why this matters

In high-dimensional data, the decision boundary is a hyperplane that cannot be inspected directly. Projection provides a controlled view of how samples and the learned normal vector relate inside a chosen two-dimensional subspace.

A projected boundary is not necessarily the complete classifier. Components of the weight vector outside the displayed subspace still affect logits unless the visualization uses a stated reference point or projects the full geometry carefully.

Part 2 - Mathematical form

Let $\mathbf{P} \in \mathbb{R}^{d \times 2}$ contain orthonormal directions and let

$$\mathbf{x} = \mathbf{x}_0 + \mathbf{P}\mathbf{u}.$$

The boundary restricted to this affine plane is

$$(\mathbf{P}^\mathsf{T}\mathbf{w})^\mathsf{T}\mathbf{u} + \mathbf{w}^\mathsf{T}\mathbf{x}_0 + b = 0.$$

The fraction of weight energy visible in the plane is

$$q = \frac{\|\mathbf{P}^\mathsf{T}\mathbf{w}\|_2^2}{\|\mathbf{w}\|_2^2}.$$

Part 3 - Hints and Helper

Use an orthonormal projection such as two selected principal directions or a declared custom basis. Calculate the restricted boundary and visible weight-energy ratio. Warn when the projected weight is nearly zero because the chosen view cannot display the classifier orientation meaningfully.

Part 4 - Python code skeleton

```
import torch

def E10(features, weight, bias,
        projection, reference):
    if projection.shape != (features.shape[1], 2):
        raise ValueError(...)
    gram = projection.T @ projection
    identity = torch.eye(
        2, dtype=gram.dtype, device=gram.device
```

```
    )
    if not torch.allclose(gram, identity, atol=...):
        raise ValueError(...)

    coordinates = (
        features - reference
    ) @ projection
    projected_weight = projection.T @ weight
    effective_bias = weight @ reference + bias
    total_energy = weight.square().sum()
    visible_energy = projected_weight.square().sum()
    ratio = ...
    return {
        "coordinates": coordinates,
        "projected_weight": projected_weight,
        "effective_bias": effective_bias,
        "visible_weight_energy": ratio,
        "projection_is_informative": ...,
    }

def main():
    print(E10(...))

if __name__ == "__main__":
    main()
```

11.6.12 Exercise 11 - Boundary under Feature Scaling

Part 1 - Why this matters

Feature scaling changes the numerical coefficients used by a model. A boundary trained in standardized coordinates must be transformed back correctly before it can be interpreted in the original measurement units.

Comparing raw and standardized weights directly is misleading because each coefficient refers to a different unit. The transformed boundary should produce the same logits for corresponding inputs, aside from ordinary floating-point error.

Part 2 - Mathematical form

For element-wise standardization

$$\mathbf{x}' = \frac{\mathbf{x} - \boldsymbol{\mu}}{\boldsymbol{s}}, \qquad z = \mathbf{w}'^{\mathsf{T}}\mathbf{x}' + b',$$

the equivalent original-space parameters are

$$\mathbf{w} = \frac{\mathbf{w}'}{\boldsymbol{s}}, \qquad b = b' - \mathbf{w}'^{\mathsf{T}}\frac{\boldsymbol{\mu}}{\boldsymbol{s}}.$$

Part 3 - Hints and Helper

Require every scale value to be positive and nonzero. Convert standardized parameters to original units, then calculate logits by both routes on the same samples. Report maximum absolute discrepancy and prediction parity.

Part 4 - Python code skeleton

```
import torch

def original_space_parameters(
        scaled_weight, scaled_bias, mean, scale):
    if torch.any(scale <= 0):
        raise ValueError(...)
    original_weight = ...
    original_bias = ...
    return original_weight, original_bias

def E11(features, scaled_weight, scaled_bias,
        mean, scale, threshold):
    standardized = (features - mean) / scale
    scaled_logits = (
        standardized @ scaled_weight + scaled_bias
    )
    weight, bias = original_space_parameters(
        scaled_weight, scaled_bias, mean, scale
    )
    original_logits = features @ weight + bias
    discrepancy = (
        scaled_logits - original_logits
    ).abs()
    return {
        "original_weight": weight,
        "original_bias": bias,
        "maximum_logit_difference": ...,
        "prediction_parity": ...,
    }

def main():
    print(E11(...))

if __name__ == "__main__":
    main()
```

11.6.13 Exercise 12 - Regularization Effect on Boundary

Part 1 - Why this matters

Regularization changes the fitted coefficients and can alter both the orientation and confidence profile of a logistic model. Stronger L2 regularization usually discourages large weight

norms, but its exact geometric effect depends on feature scaling and data structure.

A smaller weight norm does not automatically mean a better boundary. The experiment must compare validation loss, decision agreement, coefficient norm, and probability behavior rather than selecting a model from geometry alone.

Part 2 - Mathematical form

For L2-regularized empirical loss,

$$J(\mathbf{w}, b) = \frac{1}{N}\sum_{i=1}^{N} \text{BCEWithLogits}(z_i, y_i) + \frac{\lambda}{2}\|\mathbf{w}\|_2^2.$$

For two fitted models, orientation difference is

$$\alpha = \arccos\left(\frac{\mathbf{w}_a^{\mathsf{T}}\mathbf{w}_b}{\|\mathbf{w}_a\|_2\|\mathbf{w}_b\|_2}\right).$$

Part 3 - Hints and Helper

Train independent models across declared regularization strengths using the same split and initialization policy. Do not regularize bias unless that is an explicit design choice. Compare norms, angle, validation NLL, and decisions on fixed probes.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def summarize_model(model, features, targets):
    with torch.no_grad():
        logits = model(features).flatten()
        nll = F.binary_cross_entropy_with_logits(
            logits, targets, reduction="mean"
        )
    weight = ...
    bias = ...
    return {
        "weight": weight,
        "bias": bias,
        "weight_norm": ...,
        "validation_nll": ...,
        "probabilities": torch.sigmoid(logits),
    }

def E12(models_by_strength, features, targets,
        threshold):
    summaries = {}
    for strength, model in models_by_strength.items():
        summaries[strength] = summarize_model(
```

```
            model, features, targets
        )
    reference = summaries[...]
    for strength, summary in summaries.items():
        denominator = (
            reference["weight_norm"]
            * summary["weight_norm"]
        )
        summary["angle_from_reference"] = ...
        summary["decision_disagreement"] = ...
    return summaries

def main():
    print(E12(...))

if __name__ == "__main__":
    main()
```

11.6.14 Exercise 13 - Boundary Drift over Training

Part 1 - Why this matters

The boundary moves as gradient updates change weights and bias. Tracking this motion reveals whether training converges smoothly, oscillates, or continues changing geometry after the average loss appears nearly flat.

Parameter drift and prediction drift answer different questions. Coefficients may change in directions that have little effect on observed data, while a small parameter change can flip many samples concentrated near the boundary.

Part 2 - Mathematical form

Between epochs $t-1$ and t, define

$$\alpha_t = \arccos\left(\frac{\mathbf{w}^{(t-1)\mathsf{T}}\mathbf{w}^{(t)}}{\|\mathbf{w}^{(t-1)}\|_2\|\mathbf{w}^{(t)}\|_2}\right),$$

and probe disagreement

$$r_t = \frac{1}{N}\sum_i \mathbb{I}[\hat{y}_i^{(t)} \neq \hat{y}_i^{(t-1)}].$$

Part 3 - Hints and Helper

Save detached parameter snapshots and probabilities on one fixed probe set after every epoch. Compare adjacent snapshots, clamp cosine values before taking the angle, and handle zero-norm states explicitly. Plot drift beside loss instead of replacing loss.

Part 4 - Python code skeleton

```
import torch

def E13(snapshots, probe_features, threshold):
    rows = []
    previous = None
    for epoch, state in enumerate(snapshots):
        weight = state["weight"]
        bias = state["bias"]
        probabilities = torch.sigmoid(
            probe_features @ weight + bias
        )
        if previous is None:
            angle = ...
            disagreement = ...
            bias_change = ...
        else:
            denominator = (
                torch.linalg.vector_norm(previous["weight"])
                * torch.linalg.vector_norm(weight)
            )
            cosine = (
                previous["weight"] @ weight / denominator
            ).clamp(-1.0, 1.0)
            angle = torch.arccos(cosine)
            old_predictions = (
                previous["probabilities"] >= threshold
            )
            new_predictions = probabilities >= threshold
            disagreement = ...
            bias_change = ...
        rows.append({
            "epoch": epoch,
            "angle": ...,
            "bias_change": ...,
            "decision_disagreement": ...,
        })
        previous = {
            "weight": weight,
            "bias": bias,
            "probabilities": probabilities,
        }
    return rows

def main():
    print(E13(...))

if __name__ == "__main__":
    main()
```

11.6.15 Exercise 14 - Boundary Sharpness Measurement

Part 1 - Why this matters

A geometric hyperplane has no thickness, but the probability transition around it can be gradual or steep. In logistic regression this transition sharpness is controlled by the weight norm when distance is measured in the same feature coordinate system.

Sharpness must be interpreted with feature scaling. Rescaling an input axis changes the numerical weight norm without necessarily changing the physical decision rule, so comparisons require a common representation or original-space transformation.

Part 2 - Mathematical form

For $p(\mathbf{x}) = \sigma(z)$ with $z = \mathbf{w}^\top \mathbf{x} + b$,

$$\nabla_{\mathbf{x}} p = p(1-p)\mathbf{w}.$$

At the threshold-0.5 boundary, $p = 0.5$, hence

$$\|\nabla_{\mathbf{x}} p\|_2 = \frac{1}{4}\|\mathbf{w}\|_2.$$

A transition from probabilities p_a to p_b along the unit normal has width

$$\Delta d = \frac{\operatorname{logit}(p_b) - \operatorname{logit}(p_a)}{\|\mathbf{w}\|_2}.$$

Part 3 - Hints and Helper

Calculate boundary gradient magnitude and a declared probability-transition width, such as 0.1 to 0.9. Verify the analytical gradient against autograd at a constructed boundary point. Reject invalid probability limits and zero-norm weights.

Part 4 - Python code skeleton

```
import torch

def logit(probability):
    return torch.log(probability) - torch.log1p(
        -probability
    )

def E14(weight, lower_probability,
        upper_probability):
    weight_norm = torch.linalg.vector_norm(weight)
    if weight_norm <= ...:
        raise ValueError(...)
    if not (
        0.0 < lower_probability
        < upper_probability < 1.0
    ):
```

```
        raise ValueError(...)
    lower = weight.new_tensor(lower_probability)
    upper = weight.new_tensor(upper_probability)
    boundary_gradient_norm = ...
    transition_width = (
        logit(upper) - logit(lower)
    ) / weight_norm
    return {
        "weight_norm": ...,
        "boundary_gradient_norm": ...,
        "transition_width": ...,
    }

def main():
    print(E14(...))

if __name__ == "__main__":
    main()
```

11.6.16 Exercise 15 - Visualization Performance Cost

Part 1 - Why this matters

Decision-surface plots can become expensive because a small grid-spacing reduction increases the number of evaluated points in two dimensions quadratically. High visual resolution may consume far more memory and time without improving interpretation.

A fair benchmark separates grid creation, model evaluation, device transfer, and rendering. Timing only the final plotting call hides the main computational cost, while CUDA timing without synchronization can report incomplete work.

Part 2 - Mathematical form

For a square grid with r points per axis,

$$N_{\text{grid}} = r^2.$$

If each point has d features using s bytes per value, the feature-grid storage alone is approximately

$$M = r^2 ds$$

bytes, excluding logits, probabilities, plotting copies, and framework overhead.

Part 3 - Hints and Helper

Benchmark several resolutions with warm-up runs. Synchronize CUDA before and after timed device work, record peak allocated memory when available, and use chunked evaluation. Return measurements without automatically generating every high-resolution figure.

Part 4 - Python code skeleton

```
import time

import torch

def synchronize(device):
    if device.type == "cuda":
        torch.cuda.synchronize(device)

def E15(model, bounds, resolutions,
        device, chunk_size):
    rows = []
    model = model.to(device).eval()
    for resolution in resolutions:
        synchronize(device)
        start = time.perf_counter()
        x_axis = torch.linspace(
            bounds[0], bounds[1],
            steps=resolution,
            device=device,
        )
        y_axis = torch.linspace(
            bounds[2], bounds[3],
            steps=resolution,
            device=device,
        )
        grid_x, grid_y = torch.meshgrid(
            x_axis, y_axis, indexing="xy"
        )
        points = torch.stack(
            [grid_x.flatten(), grid_y.flatten()],
            dim=1,
        )
        probabilities = []
        with torch.no_grad():
            for offset in range(
                0, points.shape[0], chunk_size
            ):
                probabilities.append(
                    torch.sigmoid(
                        model(
                            points[
                                offset:offset + chunk_size
                            ]
                        )
                    )
                )
        synchronize(device)
        elapsed = time.perf_counter() - start
```

```
        rows.append({
            "resolution": resolution,
            "grid_points": ...,
            "seconds": elapsed,
            "output_elements": ...,
        })
    return rows

def main():
    print(E15(...))

if __name__ == "__main__":
    main()
```

11.6.17 Exercise 16 - Boundary Interpretation Report

Part 1 - Why this matters

A boundary interpretation should translate coefficients into geometry and decisions without claiming causation. It should state the feature units, scaling process, threshold, orientation, bias effect, and observed behavior on evaluation data.

Interpretation is incomplete when it lists coefficients alone. Correlated features, projection loss, class overlap, and sampling uncertainty can all limit what a coefficient or plotted line supports.

Part 2 - Mathematical form

At probability threshold τ, define

$$c_\tau = \operatorname{logit}(\tau).$$

The decision boundary is

$$\mathbf{w}^\mathsf{T}\mathbf{x} + b = c_\tau.$$

Holding all other features fixed, the coordinate crossing for feature j is

$$x_j = \frac{c_\tau - b - \sum_{k \neq j} w_k x_k}{w_j}, \qquad w_j \neq 0.$$

Part 3 - Hints and Helper

Create a structured report rather than free text only. Include model-space and original-space parameters when scaling was used, the threshold, coefficient units, probe-set error, margin statistics, projection limitations, and explicit cautions against causal interpretation.

Part 4 - Python code skeleton

```
import torch

def E16(feature_names, weight, bias, threshold,
```

```
        scaling, evaluation, limitations):
    threshold_tensor = weight.new_tensor(threshold)
    threshold_logit = torch.log(threshold_tensor) \
        - torch.log1p(-threshold_tensor)
    coefficients = []
    for name, value in zip(feature_names, weight):
        coefficients.append({
            "feature": name,
            "coefficient": value.item(),
            "direction": ...,
            "unit_interpretation": ...,
        })
    return {
        "threshold": threshold,
        "threshold_logit": threshold_logit.item(),
        "bias": bias.item(),
        "weight_norm": ...,
        "coefficients": coefficients,
        "scaling": scaling,
        "evaluation": evaluation,
        "limitations": limitations,
        "causal_claim": False,
    }

def main():
    print(E16(...))

if __name__ == "__main__":
    main()
```

11.6.18 Exercise 17 - Decision Surface Debugging

Part 1 - Why this matters

A decision-surface plot can look plausible even when axes are reversed, features are scaled inconsistently, sigmoid is applied twice, grid points use the wrong order, or the contour threshold is interpreted in logit space incorrectly.

Debugging should compare a small number of manually chosen points with the plotted surface. Corner values, one known boundary point, and direct model calls provide stronger evidence than visual appearance alone.

Part 2 - Mathematical form

For a probability threshold τ,

$$p(\mathbf{x}) = \tau \quad \Longleftrightarrow \quad z(\mathbf{x}) = \operatorname{logit}(\tau).$$

A valid reshaped surface must preserve

$$S_{j,i} = p([x_i, y_j])$$

when the mesh uses `indexing="xy"`.

Part 3 - Hints and Helper

Validate axis ordering, grid shape, preprocessing, model device, output shape, and finite values. Select several flattened indices, reconstruct their coordinate pairs, and compare stored surface probabilities with independent model evaluations.

Part 4 - Python code skeleton

```
import torch

def E17(model, x_axis, y_axis, surface,
        preprocessing, device, tolerance):
    issues = []
    expected_shape = (
        y_axis.numel(), x_axis.numel()
    )
    if surface.shape != expected_shape:
        issues.append(...)
    if not torch.isfinite(surface).all():
        issues.append(...)

    grid_x, grid_y = torch.meshgrid(
        x_axis, y_axis, indexing="xy"
    )
    points = torch.stack(
        [grid_x.flatten(), grid_y.flatten()],
        dim=1,
    )
    selected_indices = torch.tensor(
        [...], dtype=torch.long
    )
    selected_points = points[selected_indices]
    transformed = preprocessing(selected_points)
    with torch.no_grad():
        direct = torch.sigmoid(
            model(transformed.to(device))
        ).flatten().cpu()
    stored = surface.flatten()[selected_indices].cpu()
    if not torch.allclose(
        direct, stored, atol=tolerance, rtol=...
    ):
        issues.append(...)
    return {
        "status": ...,
        "issues": issues,
        "checked_points": ...,
    }
```

```
def main():
    print(E17(...))

if __name__ == "__main__":
    main()
```

11.6.19 Exercise 18 - Boundary Robustness Test

Part 1 - Why this matters

Boundary robustness measures whether small, permitted input changes can alter decisions. Samples close to the hyperplane are naturally more vulnerable, while samples with large geometric margins require larger perturbations along the boundary normal.

Robustness is meaningful only under a declared perturbation model and feature scale. A Euclidean change of one unit can represent very different physical changes across features, so standardized and domain-constrained tests should be distinguished.

Part 2 - Mathematical form

For threshold logit c_τ, the geometric distance to the boundary is

$$d_i = \frac{\left|\mathbf{w}^\mathsf{T}\mathbf{x}_i + b - c_\tau\right|}{\|\mathbf{w}\|_2}.$$

The minimum unconstrained L2 perturbation that reaches the hyperplane is

$$\boldsymbol{\delta}_i = -\frac{\mathbf{w}^\mathsf{T}\mathbf{x}_i + b - c_\tau}{\|\mathbf{w}\|_2^2}\mathbf{w}.$$

Part 3 - Hints and Helper

Calculate analytical perturbations and verify that perturbed logits reach the threshold within tolerance. Compare distances with an allowed perturbation budget, but do not apply changes to immutable or categorical features unless the domain permits them.

Part 4 - Python code skeleton

```
import torch

def E18(features, weight, bias, threshold,
        perturbation_budget, tolerance):
    weight_norm = torch.linalg.vector_norm(weight)
    if weight_norm <= ...:
        raise ValueError(...)
    threshold_tensor = weight.new_tensor(threshold)
    threshold_logit = (
        torch.log(threshold_tensor)
        - torch.log1p(-threshold_tensor)
    )
```

```
    logits = features @ weight + bias
    offsets = logits - threshold_logit
    perturbations = (
        -offsets.unsqueeze(1)
        / weight_norm.square()
        * weight.unsqueeze(0)
    )
    boundary_logits = (
        (features + perturbations) @ weight + bias
    )
    verification_error = (
        boundary_logits - threshold_logit
    ).abs()
    return {
        "distance": ...,
        "vulnerable": ...,
        "perturbations": perturbations,
        "verified": ...,
        "maximum_verification_error": ...,
    }

def main():
    print(E18(...))

if __name__ == "__main__":
    main()
```

11.6.20 Exercise 19 - Final Boundary Analysis

Part 1 - Why this matters

A final boundary analysis combines geometry, predictive behavior, stability, scaling, robustness, and visualization limitations. A visually attractive separator is not enough to establish that the model is accurate, stable, or suitable for deployment.

The report should preserve the evidence needed to reproduce every conclusion. Feature definitions, preprocessing state, threshold, model parameters, data split, seeds, projection basis, tolerances, and failure rules all belong beside the results.

Part 2 - Mathematical form

A structured analysis may be represented as

$$R = \{\text{definition}, \text{geometry}, \text{performance}, \text{stability}, \text{robustness}, \text{visualization}, \text{limitations}, \text{status}\}.$$

Status must follow declared checks rather than an informal impression of the boundary plot.

Part 3 - Hints and Helper

Accept results from earlier exercises, validate required sections, and convert tensors into serializable values. Add warnings for degenerate weights, excessive regional error, instability,

weak projection coverage, or robustness below the declared budget. Mark missing evidence explicitly.

Part 4 - Python code skeleton

```
import json

import torch

def serializable(value):
    if isinstance(value, torch.Tensor):
        if value.numel() == 1:
            return value.item()
        return value.detach().cpu().tolist()
    if isinstance(value, dict):
        return {
            key: serializable(item)
            for key, item in value.items()
        }
    if isinstance(value, list):
        return [serializable(item) for item in value]
    return value

def E19(sections, rules, output_path):
    required = {
        "definition",
        "geometry",
        "performance",
        "stability",
        "robustness",
        "visualization",
        "limitations",
    }
    missing = sorted(required - sections.keys())
    warnings = []
    if missing:
        warnings.append(...)
    if sections.get("geometry", {}).get(
        "weight_norm", 0.0
    ) <= rules["minimum_weight_norm"]:
        warnings.append(...)
    if sections.get("stability", {}).get(
        "mean_disagreement", 0.0
    ) > rules["maximum_disagreement"]:
        warnings.append(...)
    if sections.get("robustness", {}).get(
        "vulnerable_fraction", 0.0
    ) > rules["maximum_vulnerable_fraction"]:
        warnings.append(...)
```

```
    report = {
        "sections": serializable(sections),
        "missing_sections": missing,
        "warnings": warnings,
        "status": ...,
    }
    with open(output_path, "w", encoding="utf-8") as file:
        json.dump(report, file, indent=2)
    return report

def main():
    print(E19(...))

if __name__ == "__main__":
    main()
```

11.7 LG05 - Loss Representation for Classification

Models and Training

11.7.1 Exercise 00 - Binary Cross Entropy Formula

Part 1 - Why this matters

Binary cross entropy measures how much probability a binary model assigns to the observed label. It penalizes confident mistakes strongly while giving small loss to confident correct predictions.

The formula is also the negative Bernoulli log-likelihood, so it connects classification training with maximum-likelihood estimation. The target must describe the same event as the predicted positive-class probability.

Part 2 - Mathematical form

$$\ell_i = -\left[y_i \log(p_i) + (1 - y_i) \log(1 - p_i)\right], \qquad y_i \in \{0, 1\}.$$

For N elements,

$$L_{\text{mean}} = \frac{1}{N} \sum_{i=1}^{N} \ell_i.$$

Part 3 - Hints and Helper

Implement the two target-dependent terms manually. Clamp only for this educational probability-space version, preserve unreduced terms, and compare their mean with PyTorch.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E00(probabilities, targets, epsilon):
    probabilities = probabilities.clamp(
        epsilon, 1.0 - epsilon
    )
    terms = -(
        targets * torch.log(...)
        + (1.0 - targets) * torch.log1p(...)
    )
    return terms, ...

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

11.7.2 Exercise 01 - BCE via PyTorch

Part 1 - Why this matters

PyTorch provides BCE operations with explicit input contracts. Selecting the correct function prevents duplicated sigmoid operations and improves numerical reliability.

The exercise should compare manual probability-space BCE with the library result while preserving shape and reduction choices. Agreement is meaningful only when both routes consume equivalent values.

Part 2 - Mathematical form

$$\text{BCELoss}(p, y) = -\left[y \log p + (1 - y) \log(1 - p)\right].$$

The input p must be a probability in $[0, 1]$.

Part 3 - Hints and Helper

Use `torch.nn.functional.binary_cross_entropy` with reduction `none`. Validate identical shapes and floating targets, then compare per-element values.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E01(probabilities, targets):
    if probabilities.shape != targets.shape:
        raise ValueError(...)
    terms = F.binary_cross_entropy(
```

```
        probabilities,
        targets,
        reduction="none",
    )
    return {
        "terms": terms,
        "mean": ...,
        "sum": ...,
    }

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

11.7.3 Exercise 02 - Logits vs Probabilities Loss

Part 1 - Why this matters

Logits and probabilities represent different stages of the model. Passing one representation to a function expecting the other may produce an error or, more dangerously, a plausible but incorrect loss.

A stable logits-based loss combines sigmoid and BCE internally. Probability-based BCE remains useful when probabilities are genuinely the input, but it should not be placed after an unnecessary second sigmoid.

Part 2 - Mathematical form

With $p = \sigma(z)$,

$$\text{BCEWithLogits}(z, y) = \text{BCE}(p, y).$$

The equality is mathematical; the logits formulation is numerically more stable.

Part 3 - Hints and Helper

Calculate both correct routes and one intentionally wrong double-sigmoid route. Report equality and the size of the semantic error.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E02(logits, targets):
    probabilities = torch.sigmoid(logits)
    from_logits = F.binary_cross_entropy_with_logits(
        logits, targets, reduction="none"
    )
    from_probabilities = F.binary_cross_entropy(
```

```
        probabilities, targets, reduction="none"
    )
    wrong_double_sigmoid = F.binary_cross_entropy(
        torch.sigmoid(probabilities),
        targets,
        reduction="none",
    )
    return {
        "correct_routes_equal": ...,
        "wrong_difference": ...,
    }

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

11.7.4 Exercise 03 - Numerical Stability (Log-Sum-Exp)

Part 1 - Why this matters

Extreme logits can round sigmoid outputs to exactly zero or one, making direct logarithms infinite. Stable loss computation avoids materializing those dangerous probability expressions.

This is a structural numerical issue, not merely a reason to increase precision. A log-sum-exp form remains useful over a much wider range than sigmoid followed by logarithm.

Part 2 - Mathematical form

A stable binary loss is

$$\ell(z, y) = \max(z, 0) - zy + \log(1 + e^{-|z|}).$$

This is algebraically equal to BCE but avoids unstable cancellation and overflow.

Part 3 - Hints and Helper

Implement the stable expression with torch.clamp and torch.log1p, then compare it with BCEWithLogitsLoss on extreme positive and negative logits.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E03(logits, targets):
    stable_manual = (
        torch.clamp(logits, min=0)
        - logits * targets
        + torch.log1p(torch.exp(-torch.abs(logits)))
```

```
    )
    reference = F.binary_cross_entropy_with_logits(
        logits, targets, reduction="none"
    )
    return stable_manual, reference, ...

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

11.7.5 Exercise 04 - BCE on CUDA

Part 1 - Why this matters

CUDA can accelerate BCE over large tensors, but inputs, targets, weights, and model outputs must share a compatible device. Transfer cost can dominate small evaluations.

The GPU result should preserve dtype and reduction semantics. CPU and CUDA values may differ slightly due to floating-point execution, so parity needs declared tolerances rather than exact equality.

Part 2 - Mathematical form

$$L_{\text{sum}} = \sum_{i=1}^{N} \text{BCEWithLogits}(z_i, y_i).$$

The device changes computation placement, not the objective definition.

Part 3 - Hints and Helper

Choose CUDA only when available, move both tensors, calculate a summed loss, and compare with the same CPU calculation. Return ordinary scalars after computation.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E04(logits, targets):
    cpu_loss = F.binary_cross_entropy_with_logits(
        logits.cpu(), targets.cpu(), reduction="sum"
    )
    if not torch.cuda.is_available():
        return {"cuda_available": False, "cpu": ...}
    cuda_loss = F.binary_cross_entropy_with_logits(
        logits.cuda(), targets.cuda(), reduction="sum"
    )
    return {
```

```
        "cuda_available": True,
        "cpu": ...,
        "cuda": ...,
        "close": ...,
    }

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

11.7.6 Exercise 05 - Loss Gradient Inspection

Part 1 - Why this matters

The loss value describes model error, while its gradient describes how each logit should move to reduce that error. Inspecting gradients exposes confident mistakes and verifies graph connectivity.

For BCE with logits, the derivative has an especially clear interpretation: predicted probability minus target. Reduction changes its scale, so gradient comparisons must use the same reduction.

Part 2 - Mathematical form

$$\frac{\partial \ell_i}{\partial z_i} = \sigma(z_i) - y_i.$$

For mean reduction, the batch gradient is divided by the number of elements.

Part 3 - Hints and Helper

Create leaf logits with `requires_grad`, retain unreduced terms for inspection, backpropagate their mean, and compare `logits.grad` with the analytical expression.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E05(logit_values, targets):
    logits = logit_values.detach().clone()
    logits.requires_grad_(True)
    loss = F.binary_cross_entropy_with_logits(
        logits, targets, reduction="mean"
    )
    loss.backward()
    analytical = (
        torch.sigmoid(logits.detach()) - targets
    ) / targets.numel()
```

```
    return logits.grad, analytical, ...

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

11.7.7 Exercise 06 - Class Imbalance Weighting

Part 1 - Why this matters

When positives are rare, an unweighted mean can be dominated by negative examples. Weighting can change the training objective so positive errors contribute more strongly.

Weights encode a learning policy, not a correction that automatically creates calibrated probabilities. The report must record class counts, the chosen weight, and whether it applies to positive terms or all elements.

Part 2 - Mathematical form

With positive weight q,

$$\ell_i = -\left[q\, y_i \log \sigma(z_i) + (1 - y_i) \log(1 - \sigma(z_i))\right].$$

A common data-derived choice is $q = N_0/N_1$ when $N_1 > 0$.

Part 3 - Hints and Helper

Calculate class counts, reject an absent positive class, create `pos_weight` on the logits device, and compare class-wise contributions before and after weighting.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E06(logits, targets):
    negative_count = (targets == 0).sum()
    positive_count = (targets == 1).sum()
    if positive_count == 0:
        raise ValueError(...)
    positive_weight = (
        negative_count / positive_count
    ).to(logits.dtype)
    weighted = F.binary_cross_entropy_with_logits(
        logits,
        targets,
        pos_weight=positive_weight,
        reduction="none",
    )
```

```
    return positive_weight, weighted, ...

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

11.7.8 Exercise 07 - Loss Scaling Effect

Part 1 - Why this matters

Multiplying a loss scales its gradients even though it does not change which parameter minimizes the exact objective when the scale is positive and constant.

Loss scaling can be intentional for mixed precision or multi-objective balancing, but an unexplained factor changes optimization dynamics. Learning rate, clipping, and reported metrics must be interpreted accordingly.

Part 2 - Mathematical form

For constant $c > 0$,

$$L' = cL, \qquad \nabla_\theta L' = c\nabla_\theta L, \qquad \arg\min L' = \arg\min L.$$

Part 3 - Hints and Helper

Use identical cloned logits for several scales, run independent backward passes, and compare gradient norms and element-wise gradient ratios.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def gradient_for_scale(values, targets, scale):
    logits = values.detach().clone().requires_grad_(True)
    base = F.binary_cross_entropy_with_logits(
        logits, targets
    )
    scaled = scale * base
    scaled.backward()
    return scaled.detach(), logits.grad.detach()

def E07(values, targets, scales):
    return {
        scale: gradient_for_scale(
            values, targets, scale
        )
        for scale in scales
```

```
    }

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

11.7.9 Exercise 08 - Loss Explosion Scenario

Part 1 - Why this matters

Very large loss can arise from extreme wrong logits, invalid learning rates, non-finite inputs, or corrupted targets. The magnitude is a symptom that needs classification, not proof of one specific bug.

Stable BCE prevents avoidable numerical failure, but it cannot make genuinely terrible predictions inexpensive. Monitoring logits, gradients, and finiteness distinguishes valid large penalties from NaN propagation.

Part 2 - Mathematical form

For $y = 1$ and $z \to -\infty$,

$$\ell(z, 1) \approx -z.$$

For $y = 0$ and $z \to +\infty$,

$$\ell(z, 0) \approx z.$$

Part 3 - Hints and Helper

Construct controlled extreme correct and incorrect cases. Record unreduced losses, maximum absolute logit, finite checks, and which samples exceed a declared warning threshold.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E08(logits, targets, warning_threshold):
    terms = F.binary_cross_entropy_with_logits(
        logits, targets, reduction="none"
    )
    return {
        "all_finite": ...,
        "maximum_loss": ...,
        "maximum_absolute_logit": ...,
        "warning_indices": ...,
    }

def main():
    print(E08(...))
```

```
if __name__ == "__main__":
    main()
```

11.7.10 Exercise 09 - Loss Debugging Case

Part 1 - Why this matters

Loss bugs commonly come from wrong shapes, integer targets, device mismatch, double sigmoid, invalid target ranges, or incorrect aggregation. Each failure requires a different explanation.

A debugger should validate contracts before calling the loss function and preserve actionable evidence. Broadcasting should never silently pair unrelated predictions and labels.

Part 2 - Mathematical form

A valid binary logits loss requires

$$\text{shape}(\mathbf{z}) = \text{shape}(\mathbf{y}), \qquad 0 \le y_i \le 1,$$

with compatible floating dtypes and devices.

Part 3 - Hints and Helper

Return a list of contract violations covering rank, shape, dtype, device, range, and finiteness. Accept an explicit representation label instead of guessing from value ranges.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E09(inputs, targets, representation):
    issues = []
    if inputs.shape != targets.shape:
        issues.append(...)
    if not inputs.is_floating_point():
        issues.append(...)
    if not targets.is_floating_point():
        issues.append(...)
    if inputs.device != targets.device:
        issues.append(...)
    if not torch.isfinite(inputs).all():
        issues.append(...)
    if not ((targets >= 0) & (targets <= 1)).all():
        issues.append(...)
    if representation != "logits":
        issues.append(...)
    return issues
```

```
def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

11.7.11 Exercise 10 - Custom BCE Loss Class

Part 1 - Why this matters

A custom loss module makes validation, weighting, and reduction policy reusable. It also teaches how PyTorch modules register configuration without hiding the mathematical objective.

Reimplementing a stable primitive manually is unnecessary for production use. The custom class should delegate numerical computation to BCEWithLogitsLoss while adding explicit project-level contracts.

Part 2 - Mathematical form

$$L = \text{reduce}\left(\text{BCEWithLogits}(\mathbf{z}, \mathbf{y}; q)\right).$$

The reduction is one of none, sum, or mean.

Part 3 - Hints and Helper

Subclass `nn.Module`, validate reduction in the constructor, register `pos_weight` as a buffer when supplied, and check shapes in `forward`.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

class CustomBCE(torch.nn.Module):
    def __init__(self, reduction="mean",
                 positive_weight=None):
        super().__init__()
        if reduction not in {"none", "mean", "sum"}:
            raise ValueError(...)
        self.reduction = reduction
        self.register_buffer(
            "positive_weight",
            positive_weight,
        )

    def forward(self, logits, targets):
        if logits.shape != targets.shape:
            raise ValueError(...)
        return F.binary_cross_entropy_with_logits(
            logits,
```

```
            targets,
            pos_weight=...,
            reduction=self.reduction,
        )

def E10(...):
    return CustomBCE(...)(...)

def main():
    print(E10(...))

if __name__ == "__main__":
    main()
```

11.7.12 Exercise 11 - Masked Loss Computation

Part 1 - Why this matters

Some labels may be padded, missing, or intentionally excluded. Masking ensures these elements do not influence either the reported loss or the gradient.

Multiplying a mean loss by a mask after reduction is incorrect because excluded elements have already contributed. Mask unreduced terms first, then divide by the number or total weight of valid elements.

Part 2 - Mathematical form

For mask $m_i \in \{0, 1\}$,

$$L_{\text{masked}} = \frac{\sum_i m_i \ell_i}{\sum_i m_i}, \qquad \sum_i m_i > 0.$$

Part 3 - Hints and Helper

Calculate unreduced BCE, convert the mask to a compatible dtype, validate identical shapes, and reject an empty mask. Return valid count and masked sum as well as mean.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E11(logits, targets, mask):
    if not (
        logits.shape == targets.shape == mask.shape
    ):
        raise ValueError(...)
    terms = F.binary_cross_entropy_with_logits(
        logits, targets, reduction="none"
    )
    weights = mask.to(terms.dtype)
```

```
    valid_count = weights.sum()
    if valid_count <= 0:
        raise ValueError(...)
    masked_sum = ...
    return masked_sum / valid_count, ...

def main():
    print(E11(...))

if __name__ == "__main__":
    main()
```

11.7.13 Exercise 12 - Reduction Mode Comparison

Part 1 - Why this matters

Reduction changes the numerical scale of both loss and gradients. The none, sum, and mean modes serve different purposes and must not be compared as if their values were interchangeable.

Dataset-wide means require element-weighted aggregation across batches. An average of batch means becomes biased when batch sizes differ.

Part 2 - Mathematical form

$$L_{\text{none}} = (\ell_1, \ldots, \ell_N), \quad L_{\text{sum}} = \sum_i \ell_i, \quad L_{\text{mean}} = \frac{1}{N} \sum_i \ell_i.$$

Part 3 - Hints and Helper

Compute unreduced terms once, derive sum and mean manually, and compare with all three PyTorch reductions. Verify the corresponding gradient scaling in a second experiment.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E12(logits, targets):
    none = F.binary_cross_entropy_with_logits(
        logits, targets, reduction="none"
    )
    summed = F.binary_cross_entropy_with_logits(
        logits, targets, reduction="sum"
    )
    mean = F.binary_cross_entropy_with_logits(
        logits, targets, reduction="mean"
    )
    return {
        "none": none,
```

```
        "sum_matches": ...,
        "mean_matches": ...,
        "element_count": ...,
    }

def main():
    print(E12(...))

if __name__ == "__main__":
    main()
```

11.7.14 Exercise 13 - Mixed Precision Loss

Part 1 - Why this matters

Mixed precision can accelerate training and reduce memory, but extreme values and small gradients require careful dtype handling. Loss computation is often kept in a numerically suitable autocast path.

Gradient scaling protects small gradients from underflow; it does not change the intended optimizer update after unscaling. The experiment should run only on supported devices and include a full-precision reference.

Part 2 - Mathematical form

With scale s,

$$L' = sL, \qquad \nabla L' = s\nabla L.$$

After unscaling, the optimizer should receive the intended gradient ∇L.

Part 3 - Hints and Helper

Use torch.autocast with a device-appropriate dtype and GradScaler for CUDA training. Compare finite loss and gradients with float32; do not force CUDA when unavailable.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E13(model, features, targets, optimizer):
    device = features.device
    scaler = torch.amp.GradScaler(
        "cuda",
        enabled=device.type == "cuda",
    )
    optimizer.zero_grad(set_to_none=True)
    with torch.autocast(
        device_type=device.type,
        dtype=...,
        enabled=...,
```

```
    ):
        logits = model(features).reshape_as(targets)
        loss = F.binary_cross_entropy_with_logits(
            logits, targets
        )
    scaler.scale(loss).backward()
    scaler.unscale_(optimizer)
    gradient_finite = ...
    scaler.step(optimizer)
    scaler.update()
    return loss.detach(), gradient_finite

def main():
    print(E13(...))

if __name__ == "__main__":
    main()
```

11.7.15 Exercise 14 - Loss Memory Profiling

Part 1 - Why this matters

Unreduced loss tensors, retained graphs, and large batches can consume substantial memory. Profiling reveals whether memory belongs to inputs, activations, loss terms, gradients, or accidental graph retention.

A meaningful measurement resets peak statistics, synchronizes device work, and compares the same workload across reduction modes. Allocated and reserved CUDA memory are different quantities.

Part 2 - Mathematical form

For N elements and dtype size s, an unreduced loss alone stores approximately

$$M_{\text{terms}} = Ns$$

bytes, excluding graph metadata and intermediate activations.

Part 3 - Hints and Helper

Profile none, sum, and mean with identical inputs. On CUDA reset peak statistics before each isolated run and synchronize before reading; on CPU provide tensor-byte estimates.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def tensor_bytes(tensor):
    return tensor.numel() * tensor.element_size()
```

```
def E14(logits, targets, reduction):
    device = logits.device
    if device.type == "cuda":
        torch.cuda.reset_peak_memory_stats(device)
    loss = F.binary_cross_entropy_with_logits(
        logits, targets, reduction=reduction
    )
    if device.type == "cuda":
        torch.cuda.synchronize(device)
        peak = torch.cuda.max_memory_allocated(device)
    else:
        peak = ...
    return {
        "loss_shape": tuple(loss.shape),
        "loss_bytes": tensor_bytes(loss),
        "peak_bytes": peak,
    }

def main():
    print(E14(...))

if __name__ == "__main__":
    main()
```

11.7.16 Exercise 15 - CPU vs GPU Loss Parity

Part 1 - Why this matters

CPU and GPU should implement the same objective, but operation ordering and hardware arithmetic can create small numerical differences. Exact bitwise equality is generally the wrong criterion.

Parity testing should preserve model state, dtype, shapes, weights, and reduction. Absolute and relative error should be reported together with declared tolerances.

Part 2 - Mathematical form

For results L_{cpu} and L_{gpu},

$$\Delta_{\text{abs}} = |L_{\text{cpu}} - L_{\text{gpu}}|, \qquad \Delta_{\text{rel}} = \frac{\Delta_{\text{abs}}}{|L_{\text{cpu}}| + \varepsilon}.$$

Part 3 - Hints and Helper

Use float64 comparison when supported for a strict reference and float32 for the practical path. Transfer the final GPU result to CPU once and report both errors.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E15(logits, targets, epsilon, atol, rtol):
    cpu = F.binary_cross_entropy_with_logits(
        logits.cpu(), targets.cpu(), reduction="sum"
    )
    if not torch.cuda.is_available():
        return {"cuda_available": False}
    gpu = F.binary_cross_entropy_with_logits(
        logits.cuda(), targets.cuda(), reduction="sum"
    ).cpu()
    difference = (cpu - gpu).abs()
    return {
        "cpu": cpu.item(),
        "gpu": gpu.item(),
        "absolute_error": ...,
        "relative_error": ...,
        "close": torch.allclose(
            cpu, gpu, atol=atol, rtol=rtol
        ),
    }

def main():
    print(E15(...))

if __name__ == "__main__":
    main()
```

11.7.17 Exercise 16 - Loss Curve Logging

Part 1 - Why this matters

A loss curve exposes convergence, oscillation, divergence, and plateaus across training. Reliable logging uses one well-defined aggregation for every epoch.

Logging the final batch alone or averaging unequal batch means produces misleading curves. Training and validation losses should be labeled separately and calculated with consistent reductions.

Part 2 - Mathematical form

For epoch t,

$$\overline{L}_t = \frac{\sum_k L_{t,k,\mathrm{sum}}}{\sum_k N_{t,k}}.$$

Part 3 - Hints and Helper

Accumulate loss.item and element counts from summed batch losses. Store plain numbers after each epoch, include learning rate, and serialize without retaining computation graphs.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E16(model, data_loader, device):
    total_loss = 0.0
    total_elements = 0
    model.eval()
    with torch.no_grad():
        for features, targets in data_loader:
            features = features.to(device)
            targets = targets.to(device)
            logits = model(features).reshape_as(targets)
            loss = F.binary_cross_entropy_with_logits(
                logits, targets, reduction="sum"
            )
            total_loss += ...
            total_elements += ...
    return total_loss / total_elements

def main():
    print(E16(...))

if __name__ == "__main__":
    main()
```

11.7.18 Exercise 17 - Loss Drift Detection

Part 1 - Why this matters

Loss may drift after data, preprocessing, model, or deployment conditions change. Comparing only a global mean can hide class-specific or subgroup-specific degradation.

A drift alert requires a reference distribution, a current window, minimum sample counts, and a declared threshold. Statistical change does not identify its cause automatically.

Part 2 - Mathematical form

For reference mean μ_0 and current mean μ_t,

$$d_t = \mu_t - \mu_0, \qquad r_t = \frac{\mu_t - \mu_0}{|\mu_0| + \varepsilon}.$$

Both absolute and relative changes provide context.

Part 3 - Hints and Helper

Store unreduced losses for reference and current windows. Compare mean, quantiles, class-wise means, and valid counts; require sustained violations before declaring drift.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E17(reference_losses, current_losses,
        absolute_limit, relative_limit, epsilon):
    reference_mean = reference_losses.mean()
    current_mean = current_losses.mean()
    absolute_change = current_mean - reference_mean
    relative_change = absolute_change / (
        reference_mean.abs() + epsilon
    )
    return {
        "absolute_change": ...,
        "relative_change": ...,
        "alert": ...,
        "reference_count": ...,
        "current_count": ...,
    }

def main():
    print(E17(...))

if __name__ == "__main__":
    main()
```

11.7.19 Exercise 18 - Failure Recovery Test

Part 1 - Why this matters

Training should detect non-finite loss before corrupting parameters. A recovery mechanism can skip an unsafe update, preserve diagnostics, and restore a known checkpoint when policy permits.

Recovery must not hide repeated failure. The system should count consecutive incidents, record inputs and optimizer state safely, and stop after a declared limit rather than continuing indefinitely.

Part 2 - Mathematical form

A safe update requires

$$L \in \mathbb{R}, \qquad \|\nabla_\theta L\|_2 < \infty.$$

If either condition fails, parameters must not be updated.

Part 3 - Hints and Helper

Snapshot parameters before the test step, check loss and every gradient for finiteness, and call optimizer.step only when all checks pass. Verify unchanged parameters after a rejected update.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E18(model, optimizer, logits, targets):
    before = {
        name: parameter.detach().clone()
        for name, parameter in model.named_parameters()
    }
    optimizer.zero_grad(set_to_none=True)
    loss = F.binary_cross_entropy_with_logits(
        logits, targets
    )
    if not torch.isfinite(loss):
        return {"updated": False, "reason": ...}
    loss.backward()
    gradients_finite = all(
        parameter.grad is None
        or torch.isfinite(parameter.grad).all()
        for parameter in model.parameters()
    )
    if not gradients_finite:
        optimizer.zero_grad(set_to_none=True)
        return {"updated": False, "reason": ...}
    optimizer.step()
    return {
        "updated": True,
        "parameters_changed": ...,
    }

def main():
    print(E18(...))

if __name__ == "__main__":
    main()
```

11.7.20 Exercise 19 - Final Loss Report

Part 1 - Why this matters

A final loss report should connect the loss definition, representation, reduction, weighting, numerical stability, gradients, device parity, memory, drift, and recovery behavior.

No isolated scalar can establish that a loss pipeline is correct. Reproducibility requires shapes, dtypes, devices, thresholds, class counts, masks, and tolerances beside the measurements.

Part 2 - Mathematical form

$$R = \{\text{contract}, \text{values}, \text{gradients}, \text{numerics}, \text{aggregation}, \text{resources}, \text{monitoring}, \text{status}\}.$$

Part 3 - Hints and Helper

Accept validated results from earlier exercises, mark missing sections, convert tensors to serializable values, and derive status from explicit rules instead of visual judgment.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E19(sections, rules):
    required = {
        "contract", "values", "gradients",
        "numerics", "aggregation",
        "resources", "monitoring",
    }
    missing = sorted(required - sections.keys())
    warnings = []
    if missing:
        warnings.append(...)
    if not sections.get(
        "numerics", {}
    ).get("all_finite", False):
        warnings.append(...)
    if sections.get(
        "monitoring", {}
    ).get("drift_alert", False):
        warnings.append(...)
    return {
        "sections": sections,
        "missing": missing,
        "warnings": warnings,
        "status": ...,
    }

def main():
    print(E19(...))

if __name__ == "__main__":
    main()
```

11.8 LG06 - Model Adjustment and Gradient Direction

Models and Training

11.8.1 Exercise 00 - Backward Pass on BCE Loss

Part 1 - Why this matters

This experiment studies backward pass on bce loss inside logistic-regression training. Gradients convert classification error into parameter movement, so their values, shapes, devices, and graph connections must be inspected rather than assumed.

A plausible loss does not guarantee a valid update. Accidental accumulation, detached tensors, non-finite values, incorrect scaling, and stale gradients can silently change learning, so the exercise records evidence before and after backward.

Part 2 - Mathematical form

For parameters θ and binary cross-entropy loss L,

$$\mathbf{g} = \nabla_\theta L, \qquad \theta_{t+1} = \theta_t - \eta \mathbf{g}.$$

The experiment must state its reduction, batch size, learning rate, and any transformation applied to $\mathbf{g}$.

Part 3 - Hints and Helper

Use `binary_cross_entropy_with_logits`, clear gradients at the declared point, and preserve detached parameter copies. Check shape, dtype, device, finiteness, gradient norm, and whether parameters changed. Complete the missing lines without hiding the behavior in a training utility.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E00(model, features, targets, configuration):
    model.train()
    model.zero_grad(set_to_none=True)
    before = {
        name: parameter.detach().clone()
        for name, parameter in model.named_parameters()
    }
    logits = model(features).reshape_as(targets)
    loss = F.binary_cross_entropy_with_logits(
        logits, targets, reduction=...
    )
    ...
    return {
        "loss": ...,
        "gradient_norms": ...,
        "parameters_changed": ...,
        "all_finite": ...,
    }

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

11.8.2 Exercise 01 - Inspect Weight Gradients

Part 1 - Why this matters

This experiment studies inspect weight gradients inside logistic-regression training. Gradients convert classification error into parameter movement, so their values, shapes, devices, and graph connections must be inspected rather than assumed.

A plausible loss does not guarantee a valid update. Accidental accumulation, detached tensors, non-finite values, incorrect scaling, and stale gradients can silently change learning, so the exercise records evidence before and after backward.

Part 2 - Mathematical form

For parameters θ and binary cross-entropy loss L,

$$\mathbf{g} = \nabla_\theta L, \qquad \theta_{t+1} = \theta_t - \eta \mathbf{g}.$$

The experiment must state its reduction, batch size, learning rate, and any transformation applied to $\mathbf{g}$.

Part 3 - Hints and Helper

Use `binary_cross_entropy_with_logits`, clear gradients at the declared point, and preserve detached parameter copies. Check shape, dtype, device, finiteness, gradient norm, and whether parameters changed. Complete the missing lines without hiding the behavior in a training utility.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E01(model, features, targets, configuration):
    model.train()
    model.zero_grad(set_to_none=True)
    before = {
        name: parameter.detach().clone()
        for name, parameter in model.named_parameters()
    }
    logits = model(features).reshape_as(targets)
    loss = F.binary_cross_entropy_with_logits(
        logits, targets, reduction=...
    )
    ...
    return {
        "loss": ...,
        "gradient_norms": ...,
        "parameters_changed": ...,
        "all_finite": ...,
    }
```

```
def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

11.8.3 Exercise 02 - Inspect Bias Gradients

Part 1 - Why this matters

This experiment studies inspect bias gradients inside logistic-regression training. Gradients convert classification error into parameter movement, so their values, shapes, devices, and graph connections must be inspected rather than assumed.

A plausible loss does not guarantee a valid update. Accidental accumulation, detached tensors, non-finite values, incorrect scaling, and stale gradients can silently change learning, so the exercise records evidence before and after backward.

Part 2 - Mathematical form

For parameters θ and binary cross-entropy loss L,

$$\mathbf{g} = \nabla_\theta L, \qquad \theta_{t+1} = \theta_t - \eta \mathbf{g}.$$

The experiment must state its reduction, batch size, learning rate, and any transformation applied to $\mathbf{g}$.

Part 3 - Hints and Helper

Use `binary_cross_entropy_with_logits`, clear gradients at the declared point, and preserve detached parameter copies. Check shape, dtype, device, finiteness, gradient norm, and whether parameters changed. Complete the missing lines without hiding the behavior in a training utility.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E02(model, features, targets, configuration):
    model.train()
    model.zero_grad(set_to_none=True)
    before = {
        name: parameter.detach().clone()
        for name, parameter in model.named_parameters()
    }
    logits = model(features).reshape_as(targets)
    loss = F.binary_cross_entropy_with_logits(
        logits, targets, reduction=...
    )
```

```
    ...
    return {
        "loss": ...,
        "gradient_norms": ...,
        "parameters_changed": ...,
        "all_finite": ...,
    }

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

11.8.4 Exercise 03 - Manual Gradient Update

Part 1 - Why this matters

This experiment studies manual gradient update inside logistic-regression training. Gradients convert classification error into parameter movement, so their values, shapes, devices, and graph connections must be inspected rather than assumed.

A plausible loss does not guarantee a valid update. Accidental accumulation, detached tensors, non-finite values, incorrect scaling, and stale gradients can silently change learning, so the exercise records evidence before and after backward.

Part 2 - Mathematical form

For parameters θ and binary cross-entropy loss L,

$$\mathbf{g} = \nabla_\theta L, \qquad \theta_{t+1} = \theta_t - \eta \mathbf{g}.$$

The experiment must state its reduction, batch size, learning rate, and any transformation applied to $\mathbf{g}$.

Part 3 - Hints and Helper

Use `binary_cross_entropy_with_logits`, clear gradients at the declared point, and preserve detached parameter copies. Check shape, dtype, device, finiteness, gradient norm, and whether parameters changed. Complete the missing lines without hiding the behavior in a training utility.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E03(model, features, targets, configuration):
    model.train()
    model.zero_grad(set_to_none=True)
    before = {
```

```
        name: parameter.detach().clone()
        for name, parameter in model.named_parameters()
    }
    logits = model(features).reshape_as(targets)
    loss = F.binary_cross_entropy_with_logits(
        logits, targets, reduction=...
    )
    ...
    return {
        "loss": ...,
        "gradient_norms": ...,
        "parameters_changed": ...,
        "all_finite": ...,
    }

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

11.8.5 Exercise 04 - Learning Rate Sensitivity

Part 1 - Why this matters

This experiment studies learning rate sensitivity inside logistic-regression training. Gradients convert classification error into parameter movement, so their values, shapes, devices, and graph connections must be inspected rather than assumed.

A plausible loss does not guarantee a valid update. Accidental accumulation, detached tensors, non-finite values, incorrect scaling, and stale gradients can silently change learning, so the exercise records evidence before and after backward.

Part 2 - Mathematical form

For parameters θ and binary cross-entropy loss L,

$$\mathbf{g} = \nabla_\theta L, \qquad \theta_{t+1} = \theta_t - \eta \mathbf{g}.$$

The experiment must state its reduction, batch size, learning rate, and any transformation applied to $\mathbf{g}$.

Part 3 - Hints and Helper

Use `binary_cross_entropy_with_logits`, clear gradients at the declared point, and preserve detached parameter copies. Check shape, dtype, device, finiteness, gradient norm, and whether parameters changed. Complete the missing lines without hiding the behavior in a training utility.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E04(model, features, targets, configuration):
    model.train()
    model.zero_grad(set_to_none=True)
    before = {
        name: parameter.detach().clone()
        for name, parameter in model.named_parameters()
    }
    logits = model(features).reshape_as(targets)
    loss = F.binary_cross_entropy_with_logits(
        logits, targets, reduction=...
    )
    ...
    return {
        "loss": ...,
        "gradient_norms": ...,
        "parameters_changed": ...,
        "all_finite": ...,
    }

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

11.8.6 Exercise 05 - Gradient Clipping in Classification

Part 1 - Why this matters

This experiment studies gradient clipping in classification inside logistic-regression training. Gradients convert classification error into parameter movement, so their values, shapes, devices, and graph connections must be inspected rather than assumed.

A plausible loss does not guarantee a valid update. Accidental accumulation, detached tensors, non-finite values, incorrect scaling, and stale gradients can silently change learning, so the exercise records evidence before and after backward.

Part 2 - Mathematical form

For parameters θ and binary cross-entropy loss L,

$$\mathbf{g} = \nabla_\theta L, \qquad \theta_{t+1} = \theta_t - \eta \mathbf{g}.$$

The experiment must state its reduction, batch size, learning rate, and any transformation applied to $\mathbf{g}$.

Part 3 - Hints and Helper

Use `binary_cross_entropy_with_logits`, clear gradients at the declared point, and preserve detached parameter copies. Check shape, dtype, device, finiteness, gradient norm, and whether parameters changed. Complete the missing lines without hiding the behavior in a training utility.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E05(model, features, targets, configuration):
    model.train()
    model.zero_grad(set_to_none=True)
    before = {
        name: parameter.detach().clone()
        for name, parameter in model.named_parameters()
    }
    logits = model(features).reshape_as(targets)
    loss = F.binary_cross_entropy_with_logits(
        logits, targets, reduction=...
    )
    ...
    return {
        "loss": ...,
        "gradient_norms": ...,
        "parameters_changed": ...,
        "all_finite": ...,
    }

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

11.8.7 Exercise 06 - Gradient Noise Injection

Part 1 - Why this matters

This experiment studies gradient noise injection inside logistic-regression training. Gradients convert classification error into parameter movement, so their values, shapes, devices, and graph connections must be inspected rather than assumed.

A plausible loss does not guarantee a valid update. Accidental accumulation, detached tensors, non-finite values, incorrect scaling, and stale gradients can silently change learning, so the exercise records evidence before and after backward.

Part 2 - Mathematical form

For parameters θ and binary cross-entropy loss L,

$$\mathbf{g} = \nabla_\theta L, \qquad \theta_{t+1} = \theta_t - \eta \mathbf{g}.$$

The experiment must state its reduction, batch size, learning rate, and any transformation applied to $\mathbf{g}$.

Part 3 - Hints and Helper

Use `binary_cross_entropy_with_logits`, clear gradients at the declared point, and preserve detached parameter copies. Check shape, dtype, device, finiteness, gradient norm, and whether parameters changed. Complete the missing lines without hiding the behavior in a training utility.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E06(model, features, targets, configuration):
    model.train()
    model.zero_grad(set_to_none=True)
    before = {
        name: parameter.detach().clone()
        for name, parameter in model.named_parameters()
    }
    logits = model(features).reshape_as(targets)
    loss = F.binary_cross_entropy_with_logits(
        logits, targets, reduction=...
    )
    ...
    return {
        "loss": ...,
        "gradient_norms": ...,
        "parameters_changed": ...,
        "all_finite": ...,
    }

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

11.8.8 Exercise 07 - Optimizer Step Comparison

Part 1 - Why this matters

This experiment studies optimizer step comparison inside logistic-regression training. Gradients convert classification error into parameter movement, so their values, shapes, devices,

and graph connections must be inspected rather than assumed.

A plausible loss does not guarantee a valid update. Accidental accumulation, detached tensors, non-finite values, incorrect scaling, and stale gradients can silently change learning, so the exercise records evidence before and after backward.

Part 2 - Mathematical form

For parameters θ and binary cross-entropy loss L,

$$\mathbf{g} = \nabla_\theta L, \qquad \theta_{t+1} = \theta_t - \eta \mathbf{g}.$$

The experiment must state its reduction, batch size, learning rate, and any transformation applied to $\mathbf{g}$.

Part 3 - Hints and Helper

Use `binary_cross_entropy_with_logits`, clear gradients at the declared point, and preserve detached parameter copies. Check shape, dtype, device, finiteness, gradient norm, and whether parameters changed. Complete the missing lines without hiding the behavior in a training utility.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E07(model, features, targets, configuration):
    model.train()
    model.zero_grad(set_to_none=True)
    before = {
        name: parameter.detach().clone()
        for name, parameter in model.named_parameters()
    }
    logits = model(features).reshape_as(targets)
    loss = F.binary_cross_entropy_with_logits(
        logits, targets, reduction=...
    )
    ...
    return {
        "loss": ...,
        "gradient_norms": ...,
        "parameters_changed": ...,
        "all_finite": ...,
    }

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

11.8.9 Exercise 08 - CUDA Gradient Sync Check

Part 1 - Why this matters

This experiment studies cuda gradient sync check inside logistic-regression training. Gradients convert classification error into parameter movement, so their values, shapes, devices, and graph connections must be inspected rather than assumed.

A plausible loss does not guarantee a valid update. Accidental accumulation, detached tensors, non-finite values, incorrect scaling, and stale gradients can silently change learning, so the exercise records evidence before and after backward.

Part 2 - Mathematical form

For parameters θ and binary cross-entropy loss L,

$$\mathbf{g} = \nabla_\theta L, \qquad \theta_{t+1} = \theta_t - \eta \mathbf{g}.$$

The experiment must state its reduction, batch size, learning rate, and any transformation applied to $\mathbf{g}$.

Part 3 - Hints and Helper

Use `binary_cross_entropy_with_logits`, clear gradients at the declared point, and preserve detached parameter copies. Check shape, dtype, device, finiteness, gradient norm, and whether parameters changed. Complete the missing lines without hiding the behavior in a training utility.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E08(model, features, targets, configuration):
    model.train()
    model.zero_grad(set_to_none=True)
    before = {
        name: parameter.detach().clone()
        for name, parameter in model.named_parameters()
    }
    logits = model(features).reshape_as(targets)
    loss = F.binary_cross_entropy_with_logits(
        logits, targets, reduction=...
    )
    ...
    return {
        "loss": ...,
        "gradient_norms": ...,
        "parameters_changed": ...,
        "all_finite": ...,
    }
```

```
def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

11.8.10 Exercise 09 - Gradient Explosion Scenario

Part 1 - Why this matters

This experiment studies gradient explosion scenario inside logistic-regression training. Gradients convert classification error into parameter movement, so their values, shapes, devices, and graph connections must be inspected rather than assumed.

A plausible loss does not guarantee a valid update. Accidental accumulation, detached tensors, non-finite values, incorrect scaling, and stale gradients can silently change learning, so the exercise records evidence before and after backward.

Part 2 - Mathematical form

For parameters θ and binary cross-entropy loss L,

$$\mathbf{g} = \nabla_\theta L, \qquad \theta_{t+1} = \theta_t - \eta \mathbf{g}.$$

The experiment must state its reduction, batch size, learning rate, and any transformation applied to $\mathbf{g}$.

Part 3 - Hints and Helper

Use `binary_cross_entropy_with_logits`, clear gradients at the declared point, and preserve detached parameter copies. Check shape, dtype, device, finiteness, gradient norm, and whether parameters changed. Complete the missing lines without hiding the behavior in a training utility.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E09(model, features, targets, configuration):
    model.train()
    model.zero_grad(set_to_none=True)
    before = {
        name: parameter.detach().clone()
        for name, parameter in model.named_parameters()
    }
    logits = model(features).reshape_as(targets)
    loss = F.binary_cross_entropy_with_logits(
        logits, targets, reduction=...
    )
```

```
    ...
    return {
        "loss": ...,
        "gradient_norms": ...,
        "parameters_changed": ...,
        "all_finite": ...,
    }

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

11.8.11 Exercise 10 - Gradient Vanishing Scenario

Part 1 - Why this matters

This experiment studies gradient vanishing scenario inside logistic-regression training. Gradients convert classification error into parameter movement, so their values, shapes, devices, and graph connections must be inspected rather than assumed.

A plausible loss does not guarantee a valid update. Accidental accumulation, detached tensors, non-finite values, incorrect scaling, and stale gradients can silently change learning, so the exercise records evidence before and after backward.

Part 2 - Mathematical form

For parameters θ and binary cross-entropy loss L,

$$\mathbf{g} = \nabla_\theta L, \qquad \theta_{t+1} = \theta_t - \eta \mathbf{g}.$$

The experiment must state its reduction, batch size, learning rate, and any transformation applied to $\mathbf{g}$.

Part 3 - Hints and Helper

Use `binary_cross_entropy_with_logits`, clear gradients at the declared point, and preserve detached parameter copies. Check shape, dtype, device, finiteness, gradient norm, and whether parameters changed. Complete the missing lines without hiding the behavior in a training utility.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E10(model, features, targets, configuration):
    model.train()
    model.zero_grad(set_to_none=True)
    before = {
```

```
        name: parameter.detach().clone()
        for name, parameter in model.named_parameters()
    }
    logits = model(features).reshape_as(targets)
    loss = F.binary_cross_entropy_with_logits(
        logits, targets, reduction=...
    )
    ...
    return {
        "loss": ...,
        "gradient_norms": ...,
        "parameters_changed": ...,
        "all_finite": ...,
    }

def main():
    print(E10(...))

if __name__ == "__main__":
    main()
```

11.8.12 Exercise 11 - Mixed Precision Backward

Part 1 - Why this matters

This experiment studies mixed precision backward inside logistic-regression training. Gradients convert classification error into parameter movement, so their values, shapes, devices, and graph connections must be inspected rather than assumed.

A plausible loss does not guarantee a valid update. Accidental accumulation, detached tensors, non-finite values, incorrect scaling, and stale gradients can silently change learning, so the exercise records evidence before and after backward.

Part 2 - Mathematical form

For parameters θ and binary cross-entropy loss L,

$$\mathbf{g} = \nabla_\theta L, \qquad \theta_{t+1} = \theta_t - \eta \mathbf{g}.$$

The experiment must state its reduction, batch size, learning rate, and any transformation applied to $\mathbf{g}$.

Part 3 - Hints and Helper

Use `binary_cross_entropy_with_logits`, clear gradients at the declared point, and preserve detached parameter copies. Check shape, dtype, device, finiteness, gradient norm, and whether parameters changed. Complete the missing lines without hiding the behavior in a training utility.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E11(model, features, targets, configuration):
    model.train()
    model.zero_grad(set_to_none=True)
    before = {
        name: parameter.detach().clone()
        for name, parameter in model.named_parameters()
    }
    logits = model(features).reshape_as(targets)
    loss = F.binary_cross_entropy_with_logits(
        logits, targets, reduction=...
    )
    ...
    return {
        "loss": ...,
        "gradient_norms": ...,
        "parameters_changed": ...,
        "all_finite": ...,
    }

def main():
    print(E11(...))

if __name__ == "__main__":
    main()
```

11.8.13 Exercise 12 - Frozen Layer Gradient Test

Part 1 - Why this matters

This experiment studies frozen layer gradient test inside logistic-regression training. Gradients convert classification error into parameter movement, so their values, shapes, devices, and graph connections must be inspected rather than assumed.

A plausible loss does not guarantee a valid update. Accidental accumulation, detached tensors, non-finite values, incorrect scaling, and stale gradients can silently change learning, so the exercise records evidence before and after backward.

Part 2 - Mathematical form

For parameters θ and binary cross-entropy loss L,

$$\mathbf{g} = \nabla_\theta L, \qquad \theta_{t+1} = \theta_t - \eta \mathbf{g}.$$

The experiment must state its reduction, batch size, learning rate, and any transformation applied to $\mathbf{g}$.

Part 3 - Hints and Helper

Use `binary_cross_entropy_with_logits`, clear gradients at the declared point, and preserve detached parameter copies. Check shape, dtype, device, finiteness, gradient norm, and whether parameters changed. Complete the missing lines without hiding the behavior in a training utility.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E12(model, features, targets, configuration):
    model.train()
    model.zero_grad(set_to_none=True)
    before = {
        name: parameter.detach().clone()
        for name, parameter in model.named_parameters()
    }
    logits = model(features).reshape_as(targets)
    loss = F.binary_cross_entropy_with_logits(
        logits, targets, reduction=...
    )
    ...
    return {
        "loss": ...,
        "gradient_norms": ...,
        "parameters_changed": ...,
        "all_finite": ...,
    }

def main():
    print(E12(...))

if __name__ == "__main__":
    main()
```

11.8.14 Exercise 13 - Gradient Accumulation

Part 1 - Why this matters

This experiment studies gradient accumulation inside logistic-regression training. Gradients convert classification error into parameter movement, so their values, shapes, devices, and graph connections must be inspected rather than assumed.

A plausible loss does not guarantee a valid update. Accidental accumulation, detached tensors, non-finite values, incorrect scaling, and stale gradients can silently change learning, so the exercise records evidence before and after backward.

Part 2 - Mathematical form

For parameters θ and binary cross-entropy loss L,

$$\mathbf{g} = \nabla_\theta L, \qquad \theta_{t+1} = \theta_t - \eta \mathbf{g}.$$

The experiment must state its reduction, batch size, learning rate, and any transformation applied to $\mathbf{g}$.

Part 3 - Hints and Helper

Use `binary_cross_entropy_with_logits`, clear gradients at the declared point, and preserve detached parameter copies. Check shape, dtype, device, finiteness, gradient norm, and whether parameters changed. Complete the missing lines without hiding the behavior in a training utility.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E13(model, features, targets, configuration):
    model.train()
    model.zero_grad(set_to_none=True)
    before = {
        name: parameter.detach().clone()
        for name, parameter in model.named_parameters()
    }
    logits = model(features).reshape_as(targets)
    loss = F.binary_cross_entropy_with_logits(
        logits, targets, reduction=...
    )
    ...
    return {
        "loss": ...,
        "gradient_norms": ...,
        "parameters_changed": ...,
        "all_finite": ...,
    }

def main():
    print(E13(...))

if __name__ == "__main__":
    main()
```

11.8.15 Exercise 14 - Backward Graph Retention

Part 1 - Why this matters

This experiment studies backward graph retention inside logistic-regression training. Gradients convert classification error into parameter movement, so their values, shapes, devices,

and graph connections must be inspected rather than assumed.

A plausible loss does not guarantee a valid update. Accidental accumulation, detached tensors, non-finite values, incorrect scaling, and stale gradients can silently change learning, so the exercise records evidence before and after backward.

Part 2 - Mathematical form

For parameters θ and binary cross-entropy loss L,

$$\mathbf{g} = \nabla_\theta L, \qquad \theta_{t+1} = \theta_t - \eta \mathbf{g}.$$

The experiment must state its reduction, batch size, learning rate, and any transformation applied to $\mathbf{g}$.

Part 3 - Hints and Helper

Use `binary_cross_entropy_with_logits`, clear gradients at the declared point, and preserve detached parameter copies. Check shape, dtype, device, finiteness, gradient norm, and whether parameters changed. Complete the missing lines without hiding the behavior in a training utility.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E14(model, features, targets, configuration):
    model.train()
    model.zero_grad(set_to_none=True)
    before = {
        name: parameter.detach().clone()
        for name, parameter in model.named_parameters()
    }
    logits = model(features).reshape_as(targets)
    loss = F.binary_cross_entropy_with_logits(
        logits, targets, reduction=...
    )
    ...
    return {
        "loss": ...,
        "gradient_norms": ...,
        "parameters_changed": ...,
        "all_finite": ...,
    }

def main():
    print(E14(...))

if __name__ == "__main__":
    main()
```

11.8.16 Exercise 15 - Gradient Memory Profiling

Part 1 - Why this matters

This experiment studies gradient memory profiling inside logistic-regression training. Gradients convert classification error into parameter movement, so their values, shapes, devices, and graph connections must be inspected rather than assumed.

A plausible loss does not guarantee a valid update. Accidental accumulation, detached tensors, non-finite values, incorrect scaling, and stale gradients can silently change learning, so the exercise records evidence before and after backward.

Part 2 - Mathematical form

For parameters θ and binary cross-entropy loss L,

$$\mathbf{g} = \nabla_\theta L, \qquad \theta_{t+1} = \theta_t - \eta \mathbf{g}.$$

The experiment must state its reduction, batch size, learning rate, and any transformation applied to $\mathbf{g}$.

Part 3 - Hints and Helper

Use `binary_cross_entropy_with_logits`, clear gradients at the declared point, and preserve detached parameter copies. Check shape, dtype, device, finiteness, gradient norm, and whether parameters changed. Complete the missing lines without hiding the behavior in a training utility.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E15(model, features, targets, configuration):
    model.train()
    model.zero_grad(set_to_none=True)
    before = {
        name: parameter.detach().clone()
        for name, parameter in model.named_parameters()
    }
    logits = model(features).reshape_as(targets)
    loss = F.binary_cross_entropy_with_logits(
        logits, targets, reduction=...
    )
    ...
    return {
        "loss": ...,
        "gradient_norms": ...,
        "parameters_changed": ...,
        "all_finite": ...,
    }
```

```
def main():
    print(E15(...))

if __name__ == "__main__":
    main()
```

11.8.17 Exercise 16 - Multi-Batch Gradient Behavior

Part 1 - Why this matters

This experiment studies multi-batch gradient behavior inside logistic-regression training. Gradients convert classification error into parameter movement, so their values, shapes, devices, and graph connections must be inspected rather than assumed.

A plausible loss does not guarantee a valid update. Accidental accumulation, detached tensors, non-finite values, incorrect scaling, and stale gradients can silently change learning, so the exercise records evidence before and after backward.

Part 2 - Mathematical form

For parameters θ and binary cross-entropy loss L,

$$\mathbf{g} = \nabla_\theta L, \qquad \theta_{t+1} = \theta_t - \eta \mathbf{g}.$$

The experiment must state its reduction, batch size, learning rate, and any transformation applied to $\mathbf{g}$.

Part 3 - Hints and Helper

Use `binary_cross_entropy_with_logits`, clear gradients at the declared point, and preserve detached parameter copies. Check shape, dtype, device, finiteness, gradient norm, and whether parameters changed. Complete the missing lines without hiding the behavior in a training utility.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E16(model, features, targets, configuration):
    model.train()
    model.zero_grad(set_to_none=True)
    before = {
        name: parameter.detach().clone()
        for name, parameter in model.named_parameters()
    }
    logits = model(features).reshape_as(targets)
    loss = F.binary_cross_entropy_with_logits(
        logits, targets, reduction=...
    )
```

```
    ...
    return {
        "loss": ...,
        "gradient_norms": ...,
        "parameters_changed": ...,
        "all_finite": ...,
    }

def main():
    print(E16(...))

if __name__ == "__main__":
    main()
```

11.8.18 Exercise 17 - Gradient Drift Detection

Part 1 - Why this matters

This experiment studies gradient drift detection inside logistic-regression training. Gradients convert classification error into parameter movement, so their values, shapes, devices, and graph connections must be inspected rather than assumed.

A plausible loss does not guarantee a valid update. Accidental accumulation, detached tensors, non-finite values, incorrect scaling, and stale gradients can silently change learning, so the exercise records evidence before and after backward.

Part 2 - Mathematical form

For parameters θ and binary cross-entropy loss L,

$$\mathbf{g} = \nabla_\theta L, \qquad \theta_{t+1} = \theta_t - \eta \mathbf{g}.$$

The experiment must state its reduction, batch size, learning rate, and any transformation applied to $\mathbf{g}$.

Part 3 - Hints and Helper

Use `binary_cross_entropy_with_logits`, clear gradients at the declared point, and preserve detached parameter copies. Check shape, dtype, device, finiteness, gradient norm, and whether parameters changed. Complete the missing lines without hiding the behavior in a training utility.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E17(model, features, targets, configuration):
    model.train()
    model.zero_grad(set_to_none=True)
    before = {
```

```
        name: parameter.detach().clone()
        for name, parameter in model.named_parameters()
    }
    logits = model(features).reshape_as(targets)
    loss = F.binary_cross_entropy_with_logits(
        logits, targets, reduction=...
    )
    ...
    return {
        "loss": ...,
        "gradient_norms": ...,
        "parameters_changed": ...,
        "all_finite": ...,
    }

def main():
    print(E17(...))

if __name__ == "__main__":
    main()
```

11.8.19 Exercise 18 - Debugging Gradient Failure

Part 1 - Why this matters

This experiment studies debugging gradient failure inside logistic-regression training. Gradients convert classification error into parameter movement, so their values, shapes, devices, and graph connections must be inspected rather than assumed.

A plausible loss does not guarantee a valid update. Accidental accumulation, detached tensors, non-finite values, incorrect scaling, and stale gradients can silently change learning, so the exercise records evidence before and after backward.

Part 2 - Mathematical form

For parameters θ and binary cross-entropy loss L,

$$\mathbf{g} = \nabla_\theta L, \qquad \theta_{t+1} = \theta_t - \eta \mathbf{g}.$$

The experiment must state its reduction, batch size, learning rate, and any transformation applied to $\mathbf{g}$.

Part 3 - Hints and Helper

Use `binary_cross_entropy_with_logits`, clear gradients at the declared point, and preserve detached parameter copies. Check shape, dtype, device, finiteness, gradient norm, and whether parameters changed. Complete the missing lines without hiding the behavior in a training utility.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E18(model, features, targets, configuration):
    model.train()
    model.zero_grad(set_to_none=True)
    before = {
        name: parameter.detach().clone()
        for name, parameter in model.named_parameters()
    }
    logits = model(features).reshape_as(targets)
    loss = F.binary_cross_entropy_with_logits(
        logits, targets, reduction=...
    )
    ...
    return {
        "loss": ...,
        "gradient_norms": ...,
        "parameters_changed": ...,
        "all_finite": ...,
    }

def main():
    print(E18(...))

if __name__ == "__main__":
    main()
```

11.8.20 Exercise 19 - End-to-End Gradient Report

Part 1 - Why this matters

This experiment studies end-to-end gradient report inside logistic-regression training. Gradients convert classification error into parameter movement, so their values, shapes, devices, and graph connections must be inspected rather than assumed.

A plausible loss does not guarantee a valid update. Accidental accumulation, detached tensors, non-finite values, incorrect scaling, and stale gradients can silently change learning, so the exercise records evidence before and after backward.

Part 2 - Mathematical form

For parameters θ and binary cross-entropy loss L,

$$\mathbf{g} = \nabla_\theta L, \qquad \theta_{t+1} = \theta_t - \eta \mathbf{g}.$$

The experiment must state its reduction, batch size, learning rate, and any transformation applied to $\mathbf{g}$.

Part 3 - Hints and Helper

Use `binary_cross_entropy_with_logits`, clear gradients at the declared point, and preserve detached parameter copies. Check shape, dtype, device, finiteness, gradient norm, and whether parameters changed. Complete the missing lines without hiding the behavior in a training utility.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E19(model, features, targets, configuration):
    model.train()
    model.zero_grad(set_to_none=True)
    before = {
        name: parameter.detach().clone()
        for name, parameter in model.named_parameters()
    }
    logits = model(features).reshape_as(targets)
    loss = F.binary_cross_entropy_with_logits(
        logits, targets, reduction=...
    )
    ...
    return {
        "loss": ...,
        "gradient_norms": ...,
        "parameters_changed": ...,
        "all_finite": ...,
    }

def main():
    print(E19(...))

if __name__ == "__main__":
    main()
```

11.9 LG07 - Accuracy Validation and Output Analysis

Models and Training

11.9.1 Exercise 00 - Accuracy Metric Implementation

Part 1 - Why this matters

Accuracy Metric Implementation turns raw model outputs into evidence about behavior. A useful evaluation must preserve the positive-class definition, decision threshold, sample count, and dataset context so the resulting number can be interpreted correctly.

No metric is meaningful in isolation. Class imbalance, undefined denominators, data noise, device timing, and subgroup errors can make an apparently strong summary conceal important failures.

Part 2 - Mathematical form

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}.$$

Part 3 - Hints and Helper

Convert probabilities to labels with a declared threshold, validate equal shapes, and return both count and ratio.

Part 4 - Python code skeleton

```
import math
import time

import matplotlib.pyplot as plt
import torch

def E00(probabilities, targets, threshold):
    predictions = (
        probabilities >= threshold
    ).to(targets.dtype)
    correct = predictions == targets
    return {
        "correct": ...,
        "total": ...,
        "accuracy": ...,
    }

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

11.9.2 Exercise 01 - Precision Metric

Part 1 - Why this matters

Precision Metric turns raw model outputs into evidence about behavior. A useful evaluation must preserve the positive-class definition, decision threshold, sample count, and dataset context so the resulting number can be interpreted correctly.

No metric is meaningful in isolation. Class imbalance, undefined denominators, data noise, device timing, and subgroup errors can make an apparently strong summary conceal important failures.

Part 2 - Mathematical form

$$\text{Precision} = \frac{TP}{TP + FP}.$$

Part 3 - Hints and Helper

Define what happens when there are no predicted positives. Return the denominator and an undefined flag instead of silently hiding the case.

Part 4 - Python code skeleton

```
import math
import time

import matplotlib.pyplot as plt
import torch

def E01(predictions, targets, zero_division):
    true_positive = ((predictions == 1) & (targets == 1)).sum()
    false_positive = ((predictions == 1) & (targets == 0)).sum()
    denominator = true_positive + false_positive
    precision = ... if denominator > 0 else ...
    return true_positive, false_positive, precision

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

11.9.3 Exercise 02 - Recall Metric

Part 1 - Why this matters

Recall Metric turns raw model outputs into evidence about behavior. A useful evaluation must preserve the positive-class definition, decision threshold, sample count, and dataset context so the resulting number can be interpreted correctly.

No metric is meaningful in isolation. Class imbalance, undefined denominators, data noise, device timing, and subgroup errors can make an apparently strong summary conceal important failures.

Part 2 - Mathematical form

$$\text{Recall} = \frac{TP}{TP + FN}.$$

Part 3 - Hints and Helper

Treat the positive class explicitly and handle a dataset containing no actual positives. Compare recall with precision on the same predictions.

Part 4 - Python code skeleton

```
import math
import time

import matplotlib.pyplot as plt
import torch

def E02(predictions, targets, zero_division):
    true_positive = ...
    false_negative = ...
    denominator = true_positive + false_negative
    recall = ... if denominator > 0 else ...
    return {
        "true_positive": ...,
        "false_negative": ...,
        "recall": recall,
    }

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

11.9.4 Exercise 03 - F1 Score Computation

Part 1 - Why this matters

F1 Score Computation turns raw model outputs into evidence about behavior. A useful evaluation must preserve the positive-class definition, decision threshold, sample count, and dataset context so the resulting number can be interpreted correctly.

No metric is meaningful in isolation. Class imbalance, undefined denominators, data noise, device timing, and subgroup errors can make an apparently strong summary conceal important failures.

Part 2 - Mathematical form

$$F_1 = \frac{2PR}{P+R} = \frac{2TP}{2TP+FP+FN}.$$

Part 3 - Hints and Helper

Compute F1 from confusion counts and independently from precision and recall. Verify equality and define the all-zero denominator policy.

Part 4 - Python code skeleton

```
import math
import time

import matplotlib.pyplot as plt
import torch

def E03(true_positive, false_positive,
        false_negative, zero_division):
    denominator = (
        2 * true_positive
        + false_positive
        + false_negative
    )
    f1_from_counts = ... if denominator > 0 else ...
    precision = ...
    recall = ...
    f1_from_metrics = ...
    return f1_from_counts, f1_from_metrics, ...

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

11.9.5 Exercise 04 - Evaluation without Grad

Part 1 - Why this matters

Evaluation without Grad turns raw model outputs into evidence about behavior. A useful evaluation must preserve the positive-class definition, decision threshold, sample count, and dataset context so the resulting number can be interpreted correctly.

No metric is meaningful in isolation. Class imbalance, undefined denominators, data noise, device timing, and subgroup errors can make an apparently strong summary conceal important failures.

Part 2 - Mathematical form

$$\widehat{\mathbf{p}} = \sigma(f_\theta(\mathbf{X})), \qquad \nabla_\theta \widehat{\mathbf{p}} \text{ is not required during evaluation.}$$

Part 3 - Hints and Helper

Call `model.eval()` and use `torch.inference_mode()`. Confirm that outputs do not require gradients while leaving parameter `requires_grad` flags unchanged.

Part 4 - Python code skeleton

```
import math
import time

import matplotlib.pyplot as plt
import torch

def E04(model, data_loader, device):
    model.eval()
    total_correct = 0
    total_samples = 0
    with torch.inference_mode():
        for features, targets in data_loader:
            features = features.to(device)
            targets = targets.to(device)
            probabilities = torch.sigmoid(
                model(features).reshape_as(targets)
            )
            total_correct += ...
            total_samples += ...
    return total_correct / total_samples

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

11.9.6 Exercise 05 - GPU Evaluation Throughput

Part 1 - Why this matters

GPU Evaluation Throughput turns raw model outputs into evidence about behavior. A useful evaluation must preserve the positive-class definition, decision threshold, sample count, and dataset context so the resulting number can be interpreted correctly.

No metric is meaningful in isolation. Class imbalance, undefined denominators, data noise, device timing, and subgroup errors can make an apparently strong summary conceal important failures.

Part 2 - Mathematical form

$$\text{Throughput} = \frac{N_{\text{samples}}}{T_{\text{seconds}}}.$$

Part 3 - Hints and Helper

Warm up the GPU, synchronize before and after the timed region, exclude one-time model transfer, and report batch size with throughput.

Part 4 - Python code skeleton

```
import math
import time

import matplotlib.pyplot as plt
import torch

def E05(model, batches, device):
    model = model.to(device).eval()
    ...
    if device.type == "cuda":
        torch.cuda.synchronize(device)
    start = time.perf_counter()
    sample_count = 0
    with torch.inference_mode():
        for features in batches:
            output = model(features.to(device))
            sample_count += features.shape[0]
    if device.type == "cuda":
        torch.cuda.synchronize(device)
    elapsed = time.perf_counter() - start
    return sample_count / elapsed, elapsed

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

11.9.7 Exercise 06 - Confusion Matrix Visualization

Part 1 - Why this matters

Confusion Matrix Visualization turns raw model outputs into evidence about behavior. A useful evaluation must preserve the positive-class definition, decision threshold, sample count, and dataset context so the resulting number can be interpreted correctly.

No metric is meaningful in isolation. Class imbalance, undefined denominators, data noise, device timing, and subgroup errors can make an apparently strong summary conceal important failures.

Part 2 - Mathematical form

$$\mathbf{C} = \begin{bmatrix} TN & FP \\ FN & TP \end{bmatrix}.$$

Part 3 - Hints and Helper

Calculate the four cells manually, verify their sum equals the sample count, then plot a labeled two-by-two heatmap without swapping true and predicted axes.

Part 4 - Python code skeleton

```
import math
import time

import matplotlib.pyplot as plt
import torch

def E06(predictions, targets, output_path):
    matrix = torch.tensor([
        [..., ...],
        [..., ...],
    ])
    if matrix.sum() != targets.numel():
        raise RuntimeError(...)
    fig, ax = plt.subplots()
    image = ax.imshow(matrix.cpu(), cmap="Blues")
    ax.set_xlabel(...)
    ax.set_ylabel(...)
    ...
    fig.savefig(output_path, dpi=150,
                bbox_inches="tight")
    plt.close(fig)
    return matrix

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

11.9.8 Exercise 07 - Metric Stability under Noise

Part 1 - Why this matters

Metric Stability under Noise turns raw model outputs into evidence about behavior. A useful evaluation must preserve the positive-class definition, decision threshold, sample count, and dataset context so the resulting number can be interpreted correctly.

No metric is meaningful in isolation. Class imbalance, undefined denominators, data noise, device timing, and subgroup errors can make an apparently strong summary conceal important failures.

Part 2 - Mathematical form

$$\Delta M_r = M(\mathbf{X} + \varepsilon_r) - M(\mathbf{X}), \qquad \varepsilon_r \sim \mathcal{N}(0, \sigma^2 I).$$

Part 3 - Hints and Helper

Use repeated seeded perturbations and fixed targets. Report mean, standard deviation, worst change, and class-specific metric changes.

Part 4 - Python code skeleton

```
import math
import time

import matplotlib.pyplot as plt
import torch

def E07(model, features, targets,
        noise_scale, repetitions, seed):
    generator = torch.Generator(
        device=features.device
    ).manual_seed(seed)
    baseline = ...
    results = []
    with torch.inference_mode():
        for _ in range(repetitions):
            noise = torch.randn(
                features.shape,
                generator=generator,
                device=features.device,
            )
            results.append(
                evaluate(model,
                         features + noise_scale * noise,
                         targets)
            )
    return baseline, summarize(...)

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

11.9.9 Exercise 08 - Performance Scaling with Data Size

Part 1 - Why this matters

Performance Scaling with Data Size turns raw model outputs into evidence about behavior. A useful evaluation must preserve the positive-class definition, decision threshold, sample count, and dataset context so the resulting number can be interpreted correctly.

No metric is meaningful in isolation. Class imbalance, undefined denominators, data noise, device timing, and subgroup errors can make an apparently strong summary conceal important failures.

Part 2 - Mathematical form

$$T(N), \quad \frac{T(N)}{N}, \quad \text{Throughput}(N) = \frac{N}{T(N)}.$$

Part 3 - Hints and Helper

Benchmark increasing sample counts with identical dtype, device, and batch policy. Separate model execution from data generation and include warm-up.

Part 4 - Python code skeleton

```
import math
import time

import matplotlib.pyplot as plt
import torch

def E08(model, datasets, device, batch_size):
    rows = []
    model = model.to(device).eval()
    for sample_count, features in datasets.items():
        ...
        start = time.perf_counter()
        with torch.inference_mode():
            for batch in features.split(batch_size):
                model(batch.to(device))
        ...
        rows.append({
            "samples": sample_count,
            "seconds": ...,
            "throughput": ...,
        })
    return rows

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

11.9.10 Exercise 09 - Latency Profiling on CUDA

Part 1 - Why this matters

Latency Profiling on CUDA turns raw model outputs into evidence about behavior. A useful evaluation must preserve the positive-class definition, decision threshold, sample count, and dataset context so the resulting number can be interpreted correctly.

No metric is meaningful in isolation. Class imbalance, undefined denominators, data noise, device timing, and subgroup errors can make an apparently strong summary conceal important failures.

Part 2 - Mathematical form

$$\text{Latency}_q = Q_q(t_1, \ldots, t_R),$$

where Q_q is a latency quantile.

Part 3 - Hints and Helper

Use CUDA events or explicit synchronization, perform warm-up, and report median and tail percentiles rather than only the arithmetic mean.

Part 4 - Python code skeleton

```
import math
import time

import matplotlib.pyplot as plt
import torch

def E09(model, sample, repetitions):
    if not torch.cuda.is_available():
        return {"cuda_available": False}
    model = model.cuda().eval()
    sample = sample.cuda()
    timings = []
    with torch.inference_mode():
        for _ in range(...):
            model(sample)
        for _ in range(repetitions):
            start = torch.cuda.Event(enable_timing=True)
            end = torch.cuda.Event(enable_timing=True)
            start.record()
            model(sample)
            end.record()
            torch.cuda.synchronize()
            timings.append(start.elapsed_time(end))
    values = torch.tensor(timings)
    return {"median_ms": ..., "p95_ms": ...}

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

11.9.11 Exercise 10 - Memory Usage Profiling

Part 1 - Why this matters

Memory Usage Profiling turns raw model outputs into evidence about behavior. A useful evaluation must preserve the positive-class definition, decision threshold, sample count, and dataset context so the resulting number can be interpreted correctly.

No metric is meaningful in isolation. Class imbalance, undefined denominators, data noise, device timing, and subgroup errors can make an apparently strong summary conceal important failures.

Part 2 - Mathematical form

$$M_{\text{peak}} = \max_t M_{\text{allocated}}(t).$$

Part 3 - Hints and Helper

Reset CUDA peak statistics immediately before the measured workload. Report allocated and reserved memory separately and state that external allocations may be invisible.

Part 4 - Python code skeleton

```
import math
import time

import matplotlib.pyplot as plt
import torch

def E10(model, features, device):
    if device.type != "cuda":
        return {"tensor_bytes": ...}
    model = model.to(device).eval()
    features = features.to(device)
    torch.cuda.reset_peak_memory_stats(device)
    with torch.inference_mode():
        output = model(features)
    torch.cuda.synchronize(device)
    return {
        "peak_allocated": ...,
        "peak_reserved": ...,
        "output_bytes": ...,
    }

def main():
    print(E10(...))

if __name__ == "__main__":
    main()
```

11.9.12 Exercise 11 - Error Distribution Analysis

Part 1 - Why this matters

Error Distribution Analysis turns raw model outputs into evidence about behavior. A useful evaluation must preserve the positive-class definition, decision threshold, sample count, and dataset context so the resulting number can be interpreted correctly.

No metric is meaningful in isolation. Class imbalance, undefined denominators, data noise, device timing, and subgroup errors can make an apparently strong summary conceal important failures.

Part 2 - Mathematical form

$$e_i = y_i - p_i, \qquad \mathcal{E} = \{e_1, \ldots, e_N\}.$$

Part 3 - Hints and Helper

Analyze errors by target class, predicted class, confidence, and feature subgroup. Return quantiles and severe-error indices, not only a histogram.

Part 4 - Python code skeleton

```
import math
import time

import matplotlib.pyplot as plt
import torch

def E11(probabilities, targets):
    residuals = targets - probabilities
    absolute_errors = residuals.abs()
    quantiles = torch.quantile(
        absolute_errors,
        torch.tensor(
            [...],
            device=absolute_errors.device,
        ),
    )
    return {
        "quantiles": ...,
        "class_0_mean": ...,
        "class_1_mean": ...,
        "largest_error_indices": ...,
    }

def main():
    print(E11(...))

if __name__ == "__main__":
    main()
```

11.9.13 Exercise 12 - Threshold Sensitivity Analysis

Part 1 - Why this matters

Threshold Sensitivity Analysis turns raw model outputs into evidence about behavior. A useful evaluation must preserve the positive-class definition, decision threshold, sample count, and dataset context so the resulting number can be interpreted correctly.

No metric is meaningful in isolation. Class imbalance, undefined denominators, data noise, device timing, and subgroup errors can make an apparently strong summary conceal important failures.

Part 2 - Mathematical form

$$\widehat{y}_i(\tau) = \mathbb{I}[p_i \geq \tau], \qquad M = M(\tau).$$

Part 3 - Hints and Helper

Sweep ordered thresholds over zero to one and recompute confusion counts and metrics. Select a threshold only from a declared objective and validation data.

Part 4 - Python code skeleton

```
import math
import time

import matplotlib.pyplot as plt
import torch

def E12(probabilities, targets, thresholds):
    rows = []
    for threshold in thresholds:
        predictions = (
            probabilities >= threshold
        ).to(targets.dtype)
        counts = confusion_counts(
            predictions, targets
        )
        rows.append({
            "threshold": ...,
            "accuracy": ...,
            "precision": ...,
            "recall": ...,
            "f1": ...,
        })
    return rows

def main():
    print(E12(...))
```

```
if __name__ == "__main__":
    main()
```

11.9.14 Exercise 13 - Metric Regression Detection

Part 1 - Why this matters

Metric Regression Detection turns raw model outputs into evidence about behavior. A useful evaluation must preserve the positive-class definition, decision threshold, sample count, and dataset context so the resulting number can be interpreted correctly.

No metric is meaningful in isolation. Class imbalance, undefined denominators, data noise, device timing, and subgroup errors can make an apparently strong summary conceal important failures.

Part 2 - Mathematical form

$$\Delta M = M_{\text{candidate}} - M_{\text{reference}}.$$

Part 3 - Hints and Helper

Compare the same frozen evaluation dataset, threshold, preprocessing, and metric definitions. Support one-sided tolerances and minimum sample size.

Part 4 - Python code skeleton

```
import math
import time

import matplotlib.pyplot as plt
import torch

def E13(reference, candidate, tolerances):
    regressions = []
    for metric, reference_value in reference.items():
        candidate_value = candidate[metric]
        change = candidate_value - reference_value
        if change < -tolerances[metric]:
            regressions.append({
                "metric": metric,
                "change": change,
            })
    return {
        "passed": len(regressions) == 0,
        "regressions": regressions,
    }

def main():
    print(E13(...))
```

```
if __name__ == "__main__":
    main()
```

11.9.15 Exercise 14 - Robustness Test

Part 1 - Why this matters

Robustness Test turns raw model outputs into evidence about behavior. A useful evaluation must preserve the positive-class definition, decision threshold, sample count, and dataset context so the resulting number can be interpreted correctly.

No metric is meaningful in isolation. Class imbalance, undefined denominators, data noise, device timing, and subgroup errors can make an apparently strong summary conceal important failures.

Part 2 - Mathematical form

$$R(\delta) = \frac{1}{N} \sum_i \mathbb{I}[\widehat{y}(\mathbf{x}_i) = \widehat{y}(\mathbf{x}_i + \delta_i)].$$

Part 3 - Hints and Helper

Declare permitted perturbations and feature units. Compare clean and perturbed metrics across multiple seeds, including decision flip rate and confidence change.

Part 4 - Python code skeleton

```
import math
import time

import matplotlib.pyplot as plt
import torch

def E14(model, features, targets,
        perturbations, threshold):
    clean = evaluate(model, features,
                     targets, threshold)
    rows = []
    for name, modified_features in perturbations.items():
        result = evaluate(
            model, modified_features,
            targets, threshold
        )
        rows.append({
            "scenario": name,
            "metric_change": ...,
            "flip_rate": ...,
            "confidence_change": ...,
```

```
        })
    return clean, rows

def main():
    print(E14(...))

if __name__ == "__main__":
    main()
```

11.9.16 Exercise 15 - Evaluation Pipeline Automation

Part 1 - Why this matters

Evaluation Pipeline Automation turns raw model outputs into evidence about behavior. A useful evaluation must preserve the positive-class definition, decision threshold, sample count, and dataset context so the resulting number can be interpreted correctly.

No metric is meaningful in isolation. Class imbalance, undefined denominators, data noise, device timing, and subgroup errors can make an apparently strong summary conceal important failures.

Part 2 - Mathematical form

$$\text{data} \rightarrow \text{inference} \rightarrow \text{aggregation} \rightarrow \text{metrics} \rightarrow \text{artifact}.$$

Part 3 - Hints and Helper

Build a deterministic function with explicit configuration. Aggregate predictions in sample order, validate counts, and write a machine-readable report without storing computation graphs.

Part 4 - Python code skeleton

```
import math
import time

import matplotlib.pyplot as plt
import torch

def E15(model, data_loader, configuration):
    probabilities = []
    targets = []
    model.eval()
    with torch.inference_mode():
        for features, batch_targets in data_loader:
            logits = model(
                features.to(configuration["device"])
            )
            probabilities.append(
```

```
                torch.sigmoid(logits).cpu()
            )
            targets.append(batch_targets.cpu())
    probabilities = torch.cat(probabilities)
    targets = torch.cat(targets)
    return build_report(
        probabilities, targets, configuration
    )

def main():
    print(E15(...))

if __name__ == "__main__":
    main()
```

11.9.17 Exercise 16 - Evaluation Failure Debugging

Part 1 - Why this matters

Evaluation Failure Debugging turns raw model outputs into evidence about behavior. A useful evaluation must preserve the positive-class definition, decision threshold, sample count, and dataset context so the resulting number can be interpreted correctly.

No metric is meaningful in isolation. Class imbalance, undefined denominators, data noise, device timing, and subgroup errors can make an apparently strong summary conceal important failures.

Part 2 - Mathematical form

$$\mathcal{C} \in \{\text{shape, dtype, device, range, order, mode, metric}\}.$$

Part 3 - Hints and Helper

Validate contracts before calculating metrics: shapes, binary targets, finite outputs, sample order, model mode, dataset identity, and threshold range.

Part 4 - Python code skeleton

```
import math
import time

import matplotlib.pyplot as plt
import torch

def E16(probabilities, targets, metadata):
    issues = []
    if probabilities.shape != targets.shape:
        issues.append(...)
    if not torch.isfinite(probabilities).all():
```

```
        issues.append(...)
    if not (
        (probabilities >= 0)
        & (probabilities <= 1)
    ).all():
        issues.append(...)
    if not torch.all((targets == 0) | (targets == 1)):
        issues.append(...)
    if metadata["sample_count"] != targets.numel():
        issues.append(...)
    if not 0.0 <= metadata["threshold"] <= 1.0:
        issues.append(...)
    return issues

def main():
    print(E16(...))

if __name__ == "__main__":
    main()
```

11.9.18 Exercise 17 - Multi-Run Metric Comparison

Part 1 - Why this matters

Multi-Run Metric Comparison turns raw model outputs into evidence about behavior. A useful evaluation must preserve the positive-class definition, decision threshold, sample count, and dataset context so the resulting number can be interpreted correctly.

No metric is meaningful in isolation. Class imbalance, undefined denominators, data noise, device timing, and subgroup errors can make an apparently strong summary conceal important failures.

Part 2 - Mathematical form

$$\overline{M} = \frac{1}{R}\sum_{r=1}^{R} M_r, \qquad s_M = \mathrm{Std}(M_1, \ldots, M_R).$$

Part 3 - Hints and Helper

Use identical splits and metric definitions across seeds. Report every run, mean, standard deviation, minimum, maximum, and paired differences when appropriate.

Part 4 - Python code skeleton

```
import math
import time

import matplotlib.pyplot as plt
import torch
```

```
def E17(run_reports):
    metric_names = run_reports[0]["metrics"].keys()
    summary = {}
    for metric in metric_names:
        values = torch.tensor([
            report["metrics"][metric]
            for report in run_reports
        ])
        summary[metric] = {
            "values": ...,
            "mean": ...,
            "std": ...,
            "minimum": ...,
            "maximum": ...,
        }
    return summary

def main():
    print(E17(...))

if __name__ == "__main__":
    main()
```

11.9.19 Exercise 18 - Final Evaluation Report

Part 1 - Why this matters

Final Evaluation Report turns raw model outputs into evidence about behavior. A useful evaluation must preserve the positive-class definition, decision threshold, sample count, and dataset context so the resulting number can be interpreted correctly.

No metric is meaningful in isolation. Class imbalance, undefined denominators, data noise, device timing, and subgroup errors can make an apparently strong summary conceal important failures.

Part 2 - Mathematical form

$$R = \{\text{data}, \text{threshold}, \text{confusion}, \text{metrics}, \text{performance}, \text{robustness}, \text{limitations}\}.$$

Part 3 - Hints and Helper

Combine validated outputs from earlier exercises. Preserve sample counts, undefined metrics, confidence intervals or run variability, hardware context, and limitations.

Part 4 - Python code skeleton

```
import math
import time
```

```
import matplotlib.pyplot as plt
import torch

def E18(sections, required_sections):
    missing = sorted(
        set(required_sections) - sections.keys()
    )
    warnings = []
    if missing:
        warnings.append(...)
    if sections.get(
        "metrics", {}
    ).get("undefined"):
        warnings.append(...)
    return {
        "sections": sections,
        "missing": missing,
        "warnings": warnings,
        "status": ...,
    }

def main():
    print(E18(...))

if __name__ == "__main__":
    main()
```

11.9.20 Exercise 19 - Production Metric Sanity Check

Part 1 - Why this matters

Production Metric Sanity Check turns raw model outputs into evidence about behavior. A useful evaluation must preserve the positive-class definition, decision threshold, sample count, and dataset context so the resulting number can be interpreted correctly.

No metric is meaningful in isolation. Class imbalance, undefined denominators, data noise, device timing, and subgroup errors can make an apparently strong summary conceal important failures.

Part 2 - Mathematical form

$$\text{valid} = \text{finite} \wedge \text{range-valid} \wedge \text{count-consistent} \wedge \text{fresh}.$$

Part 3 - Hints and Helper

Check metric ranges, confusion-count totals, timestamp freshness, class support, threshold identity, and comparison-dataset version before publishing results.

Part 4 - Python code skeleton

```
import math
import time

import matplotlib.pyplot as plt
import torch

def E19(report, expected):
    issues = []
    metrics = report["metrics"]
    for name, value in metrics.items():
        if not math.isfinite(value):
            issues.append(...)
        if name in expected["unit_interval_metrics"]:
            if not 0.0 <= value <= 1.0:
                issues.append(...)
    if sum(report["confusion"].values()) != \
            report["sample_count"]:
        issues.append(...)
    if report["dataset_version"] != \
            expected["dataset_version"]:
        issues.append(...)
    return {
        "publish": len(issues) == 0,
        "issues": issues,
    }

def main():
    print(E19(...))

if __name__ == "__main__":
    main()
```

11.10 LG08 - Multi-Class Extension Concepts

Models and Training

11.10.1 Exercise 00 - One-vs-Rest Logistic Strategy

Part 1 - Why this matters

One-vs-Rest Logistic Strategy extends binary classification reasoning to problems containing several possible classes. Correct implementation depends on preserving class identity, tensor shape, and the distinction between mutually exclusive and independently valid labels.

A multi-class result can look correct while hiding imbalance, calibration error, class-specific failure, or excessive resource cost. The experiment therefore records both aggregate behavior and evidence for individual classes.

Part 2 - Mathematical form

$$p_c(\mathbf{x}) = \sigma(\mathbf{w}_c^{\mathsf{T}}\mathbf{x} + b_c), \qquad \widehat{y} = \arg\max_c p_c.$$

Part 3 - Hints and Helper

Train or evaluate one binary classifier per class. Preserve class order and note that independent probabilities need not sum to one.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch
import torch.nn.functional as F

def E00(models, features):
    probabilities = []
    with torch.inference_mode():
        for model in models:
            logits = model(features).flatten()
            probabilities.append(torch.sigmoid(logits))
    scores = torch.stack(probabilities, dim=1)
    predictions = ...
    return scores, predictions, ...

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

11.10.2 Exercise 01 - Softmax vs Sigmoid Comparison

Part 1 - Why this matters

Softmax vs Sigmoid Comparison extends binary classification reasoning to problems containing several possible classes. Correct implementation depends on preserving class identity, tensor shape, and the distinction between mutually exclusive and independently valid labels.

A multi-class result can look correct while hiding imbalance, calibration error, class-specific failure, or excessive resource cost. The experiment therefore records both aggregate behavior and evidence for individual classes.

Part 2 - Mathematical form

$$\sigma(z_c) = \frac{1}{1 + e^{-z_c}}, \qquad \operatorname{softmax}(\mathbf{z})_c = \frac{e^{z_c}}{\sum_j e^{z_j}}.$$

Part 3 - Hints and Helper

Apply both functions to identical logits. Verify softmax row sums and show that sigmoid treats classes independently, which is useful for multi-label rather than mutually exclusive targets.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch
import torch.nn.functional as F

def E01(logits):
    sigmoid_values = torch.sigmoid(logits)
    softmax_values = torch.softmax(logits, dim=1)
    return {
        "sigmoid": sigmoid_values,
        "softmax": softmax_values,
        "sigmoid_sums": ...,
        "softmax_sums": ...,
    }

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

11.10.3 Exercise 02 - Multi-Class Dataset Preparation

Part 1 - Why this matters

Multi-Class Dataset Preparation extends binary classification reasoning to problems containing several possible classes. Correct implementation depends on preserving class identity, tensor shape, and the distinction between mutually exclusive and independently valid labels.

A multi-class result can look correct while hiding imbalance, calibration error, class-specific failure, or excessive resource cost. The experiment therefore records both aggregate behavior and evidence for individual classes.

Part 2 - Mathematical form

$$\mathbf{X} \in \mathbb{R}^{N \times d}, \qquad \mathbf{y} \in \{0, \ldots, C-1\}^{N}.$$

Part 3 - Hints and Helper

Map original labels deterministically to contiguous indices, split before fitting preprocessing, stratify when appropriate, and store the inverse mapping.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch
import torch.nn.functional as F

def E02(features, raw_labels, class_names):
    mapping = {
        name: index
        for index, name in enumerate(class_names)
    }
    targets = torch.tensor(
        [mapping[label] for label in raw_labels],
        dtype=torch.long,
    )
    if targets.min() < 0 or targets.max() >= len(class_names):
        raise ValueError(...)
    return features, targets, mapping, ...

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

11.10.4 Exercise 03 - Cross-Entropy for Multi-Class

Part 1 - Why this matters

Cross-Entropy for Multi-Class extends binary classification reasoning to problems containing several possible classes. Correct implementation depends on preserving class identity, tensor shape, and the distinction between mutually exclusive and independently valid labels.

A multi-class result can look correct while hiding imbalance, calibration error, class-specific failure, or excessive resource cost. The experiment therefore records both aggregate behavior and evidence for individual classes.

Part 2 - Mathematical form

$$\ell_i = -\log \frac{\exp(z_{i,y_i})}{\sum_{c=1}^{C} \exp(z_{i,c})}.$$

Part 3 - Hints and Helper

Pass raw logits and long class indices to `cross_entropy`. Compare with negative gathered `log_softmax` values and keep reduction explicit.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch
import torch.nn.functional as F

def E03(logits, targets):
    if logits.ndim != 2 or targets.ndim != 1:
        raise ValueError(...)
    if targets.dtype != torch.long:
        raise TypeError(...)
    reference = F.cross_entropy(
        logits, targets, reduction="none"
    )
    log_probabilities = F.log_softmax(logits, dim=1)
    manual = -log_probabilities.gather(
        1, targets.unsqueeze(1)
    ).squeeze(1)
    return reference, manual, ...

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

11.10.5 Exercise 04 - CUDA Multi-Class Training

Part 1 - Why this matters

CUDA Multi-Class Training extends binary classification reasoning to problems containing several possible classes. Correct implementation depends on preserving class identity, tensor shape, and the distinction between mutually exclusive and independently valid labels.

A multi-class result can look correct while hiding imbalance, calibration error, class-specific failure, or excessive resource cost. The experiment therefore records both aggregate behavior and evidence for individual classes.

Part 2 - Mathematical form

$$L = \frac{1}{N} \sum_{i} \mathrm{CE}(\mathbf{z}_i, y_i), \qquad \theta \leftarrow \theta - \eta \nabla_\theta L.$$

Part 3 - Hints and Helper

Move model, inputs, and class-index targets to the same selected device. Use a full zero-grad, forward, loss, backward, step cycle and return detached evidence.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch
import torch.nn.functional as F

def E04(model, optimizer, features, targets):
    device = torch.device(
        "cuda" if torch.cuda.is_available() else "cpu"
    )
    model = model.to(device).train()
    features = features.to(device)
    targets = targets.to(device)
    optimizer.zero_grad(set_to_none=True)
    logits = model(features)
    loss = F.cross_entropy(logits, targets)
    ...
    return {
        "device": str(device),
        "loss": ...,
        "gradient_finite": ...,
    }

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

11.10.6 Exercise 05 - Class Imbalance in Multi-Class

Part 1 - Why this matters

Class Imbalance in Multi-Class extends binary classification reasoning to problems containing several possible classes. Correct implementation depends on preserving class identity, tensor shape, and the distinction between mutually exclusive and independently valid labels.

A multi-class result can look correct while hiding imbalance, calibration error, class-specific failure, or excessive resource cost. The experiment therefore records both aggregate behavior and evidence for individual classes.

Part 2 - Mathematical form

$$w_c = \frac{N}{C\,N_c}, \qquad L = \frac{1}{N}\sum_i w_{y_i}\ell_i.$$

Part 3 - Hints and Helper

Calculate class counts with minlength equal to C, reject absent classes or define a policy, and place weights on the logits device.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch
import torch.nn.functional as F

def E05(logits, targets, number_of_classes):
    counts = torch.bincount(
        targets, minlength=number_of_classes
    )
    if torch.any(counts == 0):
        raise ValueError(...)
    weights = (
        targets.numel()
        / (number_of_classes * counts)
    ).to(logits.dtype).to(logits.device)
    weighted_loss = F.cross_entropy(
        logits, targets, weight=weights
    )
    return counts, weights, weighted_loss

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

11.10.7 Exercise 06 - Confusion Matrix for Multi-Class

Part 1 - Why this matters

Confusion Matrix for Multi-Class extends binary classification reasoning to problems containing several possible classes. Correct implementation depends on preserving class identity, tensor shape, and the distinction between mutually exclusive and independently valid labels.

A multi-class result can look correct while hiding imbalance, calibration error, class-specific failure, or excessive resource cost. The experiment therefore records both aggregate behavior and evidence for individual classes.

Part 2 - Mathematical form

$$C_{r,c} = \#\{i : y_i = r, \widehat{y}_i = c\}.$$

Part 3 - Hints and Helper

Create a C by C tensor using true labels as rows and predictions as columns. Verify non-negative valid indices and total count before plotting.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch
import torch.nn.functional as F

def E06(predictions, targets, number_of_classes,
        output_path):
    matrix = torch.zeros(
        number_of_classes,
        number_of_classes,
        dtype=torch.long,
    )
    for actual, predicted in zip(targets, predictions):
        matrix[actual, predicted] += ...
    if matrix.sum() != targets.numel():
        raise RuntimeError(...)
    ...
    return matrix

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

11.10.8 Exercise 07 - Macro vs Micro Metrics

Part 1 - Why this matters

Macro vs Micro Metrics extends binary classification reasoning to problems containing several possible classes. Correct implementation depends on preserving class identity, tensor shape, and the distinction between mutually exclusive and independently valid labels.

A multi-class result can look correct while hiding imbalance, calibration error, class-specific failure, or excessive resource cost. The experiment therefore records both aggregate behavior and evidence for individual classes.

Part 2 - Mathematical form

$$F_{\text{macro}} = \frac{1}{C}\sum_{c} F_c, \qquad F_{\text{micro}} = \frac{2\sum_c TP_c}{2\sum_c TP_c + \sum_c FP_c + \sum_c FN_c}.$$

Part 3 - Hints and Helper

Derive per-class one-vs-rest counts from the confusion matrix. Report support and undefined cases before averaging.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch
import torch.nn.functional as F

def E07(confusion_matrix):
    class_rows = []
    for class_id in range(confusion_matrix.shape[0]):
        true_positive = confusion_matrix[
            class_id, class_id
        ]
        false_positive = ...
        false_negative = ...
        class_rows.append(
            compute_metrics(
                true_positive,
                false_positive,
                false_negative,
            )
        )
    macro = ...
    micro = ...
    return class_rows, macro, micro

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

11.10.9 Exercise 08 - Probability Calibration Multi-Class

Part 1 - Why this matters

Probability Calibration Multi-Class extends binary classification reasoning to problems containing several possible classes. Correct implementation depends on preserving class identity, tensor shape, and the distinction between mutually exclusive and independently valid labels.

A multi-class result can look correct while hiding imbalance, calibration error, class-specific failure, or excessive resource cost. The experiment therefore records both aggregate behavior and evidence for individual classes.

Part 2 - Mathematical form

$$\mathrm{ECE} = \sum_{m} \frac{|B_m|}{N} |\operatorname{acc}(B_m) - \operatorname{conf}(B_m)|.$$

Part 3 - Hints and Helper

Use maximum softmax probability as top-label confidence and correctness of argmax. Preserve bin counts and also consider class-wise calibration.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch
import torch.nn.functional as F

def E08(logits, targets, number_of_bins):
    probabilities = torch.softmax(logits, dim=1)
    confidence, predictions = probabilities.max(dim=1)
    correctness = (predictions == targets).float()
    boundaries = torch.linspace(
        0.0, 1.0, number_of_bins + 1,
        device=logits.device,
    )
    rows = []
    ece = logits.new_zeros(())
    for bin_id in range(number_of_bins):
        mask = ...
        ...
    return rows, ece.item()

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

11.10.10 Exercise 09 - Boundary Visualization Multi-Class

Part 1 - Why this matters

Boundary Visualization Multi-Class extends binary classification reasoning to problems containing several possible classes. Correct implementation depends on preserving class identity, tensor shape, and the distinction between mutually exclusive and independently valid labels.

A multi-class result can look correct while hiding imbalance, calibration error, class-specific failure, or excessive resource cost. The experiment therefore records both aggregate behavior and evidence for individual classes.

Part 2 - Mathematical form

$$\widehat{y}(\mathbf{x}) = \arg\max_{c}(\mathbf{w}_c^{\mathsf{T}}\mathbf{x} + b_c).$$

Part 3 - Hints and Helper

Build a two-feature grid, evaluate it in chunks, reshape argmax predictions, and overlay training samples. State if the original model uses more than two features.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch
import torch.nn.functional as F

def E09(model, x_axis, y_axis, output_path):
    grid_x, grid_y = torch.meshgrid(
        x_axis, y_axis, indexing="xy"
    )
    points = torch.stack(
        [grid_x.flatten(), grid_y.flatten()],
        dim=1,
    )
    with torch.inference_mode():
        logits = model(points)
        regions = logits.argmax(dim=1).reshape(
            grid_x.shape
        )
    fig, ax = plt.subplots()
    ax.contourf(..., ..., ..., alpha=...)
    ...
    fig.savefig(output_path, dpi=150,
                bbox_inches="tight")
    plt.close(fig)
    return regions

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

11.10.11 Exercise 10 - Gradient Stability Multi-Class

Part 1 - Why this matters

Gradient Stability Multi-Class extends binary classification reasoning to problems containing several possible classes. Correct implementation depends on preserving class identity, tensor shape, and the distinction between mutually exclusive and independently valid labels.

A multi-class result can look correct while hiding imbalance, calibration error, class-specific failure, or excessive resource cost. The experiment therefore records both aggregate behavior and evidence for individual classes.

Part 2 - Mathematical form

$$g_t = \|\nabla_\theta L_t\|_2, \qquad r_t = \frac{g_t}{g_{t-1} + \varepsilon}.$$

Part 3 - Hints and Helper

Collect global and layer-wise gradient norms after backward. Flag non-finite, exploding, vanishing, and abruptly changing gradients.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch
import torch.nn.functional as F

def E10(model, logits, targets, thresholds):
    loss = F.cross_entropy(logits, targets)
    loss.backward()
    squared_norm = 0.0
    rows = []
    for name, parameter in model.named_parameters():
        if parameter.grad is None:
            rows.append(...)
            continue
        norm = parameter.grad.norm()
        squared_norm += norm.square()
        rows.append(...)
    global_norm = torch.sqrt(squared_norm)
    return global_norm, rows, ...

def main():
    print(E10(...))

if __name__ == "__main__":
    main()
```

11.10.12 Exercise 11 - Mixed Precision Multi-Class

Part 1 - Why this matters

Mixed Precision Multi-Class extends binary classification reasoning to problems containing several possible classes. Correct implementation depends on preserving class identity, tensor shape, and the distinction between mutually exclusive and independently valid labels.

A multi-class result can look correct while hiding imbalance, calibration error, class-specific failure, or excessive resource cost. The experiment therefore records both aggregate behavior and evidence for individual classes.

Part 2 - Mathematical form

$$L_s = sL, \qquad \nabla L_s = s\nabla L,$$

followed by gradient unscaling.

Part 3 - Hints and Helper

Use `autocast` and `GradScaler` on CUDA. Pass raw logits to cross-entropy and inspect gradients only after unscaling.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch
import torch.nn.functional as F

def E11(model, optimizer, features, targets):
    enabled = features.device.type == "cuda"
    scaler = torch.amp.GradScaler(
        "cuda", enabled=enabled
    )
    optimizer.zero_grad(set_to_none=True)
    with torch.autocast(
        device_type=features.device.type,
        dtype=...,
        enabled=enabled,
    ):
        logits = model(features)
        loss = F.cross_entropy(logits, targets)
    scaler.scale(loss).backward()
    scaler.unscale_(optimizer)
    gradient_finite = ...
    scaler.step(optimizer)
    scaler.update()
    return loss.detach(), gradient_finite

def main():
    print(E11(...))

if __name__ == "__main__":
    main()
```

11.10.13 Exercise 12 - Performance Scaling Classes

Part 1 - Why this matters

Performance Scaling Classes extends binary classification reasoning to problems containing several possible classes. Correct implementation depends on preserving class identity, tensor shape, and the distinction between mutually exclusive and independently valid labels.

A multi-class result can look correct while hiding imbalance, calibration error, class-specific failure, or excessive resource cost. The experiment therefore records both aggregate behavior and evidence for individual classes.

Part 2 - Mathematical form

$$T(C), \qquad M(C), \qquad \text{output elements} = NC.$$

Part 3 - Hints and Helper

Benchmark increasing class counts with fixed N, feature width, device, dtype, and warm-up. Separate model construction from timed inference.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch
import torch.nn.functional as F

def E12(model_factory, features,
        class_counts, device):
    rows = []
    for class_count in class_counts:
        model = model_factory(class_count).to(device)
        ...
        start = time.perf_counter()
        with torch.inference_mode():
            logits = model(features.to(device))
        ...
        rows.append({
            "classes": class_count,
            "seconds": ...,
            "output_elements": logits.numel(),
            "output_bytes": ...,
        })
    return rows

def main():
    print(E12(...))

if __name__ == "__main__":
    main()
```

11.10.14 Exercise 13 - Error Analysis per Class

Part 1 - Why this matters

Error Analysis per Class extends binary classification reasoning to problems containing several possible classes. Correct implementation depends on preserving class identity, tensor shape, and the distinction between mutually exclusive and independently valid labels.

A multi-class result can look correct while hiding imbalance, calibration error, class-specific failure, or excessive resource cost. The experiment therefore records both aggregate behavior and evidence for individual classes.

Part 2 - Mathematical form

$$e_c = 1 - \frac{C_{c,c}}{\sum_j C_{c,j}}, \qquad s_c = \sum_j C_{c,j}.$$

Part 3 - Hints and Helper

Rank classes by error rate only after applying minimum support. Include most frequent confusion destinations and mean confidence for errors.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch
import torch.nn.functional as F

def E13(confusion_matrix, probabilities,
        targets, predictions, minimum_support):
    rows = []
    for class_id in range(confusion_matrix.shape[0]):
        support = confusion_matrix[class_id].sum()
        if support < minimum_support:
            continue
        correct = confusion_matrix[
            class_id, class_id
        ]
        row = confusion_matrix[class_id].clone()
        row[class_id] = 0
        rows.append({
            "class": class_id,
            "support": ...,
            "error_rate": ...,
            "most_confused_with": ...,
            "error_confidence": ...,
        })
    return sorted(rows, key=..., reverse=True)
```

```
def main():
    print(E13(...))

if __name__ == "__main__":
    main()
```

11.10.15 Exercise 14 - Metric Drift Detection

Part 1 - Why this matters

Metric Drift Detection extends binary classification reasoning to problems containing several possible classes. Correct implementation depends on preserving class identity, tensor shape, and the distinction between mutually exclusive and independently valid labels.

A multi-class result can look correct while hiding imbalance, calibration error, class-specific failure, or excessive resource cost. The experiment therefore records both aggregate behavior and evidence for individual classes.

Part 2 - Mathematical form

$$\Delta M_c = M_{c,t} - M_{c,0}.$$

Part 3 - Hints and Helper

Compare reference and current per-class metrics using the same label mapping and threshold policy. Require minimum support and report population changes.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch
import torch.nn.functional as F

def E14(reference, current, tolerances):
    alerts = []
    for class_id in reference["per_class"]:
        if class_id not in current["per_class"]:
            alerts.append(...)
            continue
        for metric, old_value in \
                reference["per_class"][class_id].items():
            new_value = current[
                "per_class"
            ][class_id][metric]
            change = new_value - old_value
            if change < -tolerances[metric]:
```

```
                alerts.append(...)
    return alerts

def main():
    print(E14(...))

if __name__ == "__main__":
    main()
```

11.10.16 Exercise 15 - End-to-End Multi-Class Pipeline

Part 1 - Why this matters

End-to-End Multi-Class Pipeline extends binary classification reasoning to problems containing several possible classes. Correct implementation depends on preserving class identity, tensor shape, and the distinction between mutually exclusive and independently valid labels.

A multi-class result can look correct while hiding imbalance, calibration error, class-specific failure, or excessive resource cost. The experiment therefore records both aggregate behavior and evidence for individual classes.

Part 2 - Mathematical form

$$\mathbf{X} \to \mathbf{z} \to \text{softmax}(\mathbf{z}) \to \hat{\mathbf{y}} \to \text{report}.$$

Part 3 - Hints and Helper

Preserve preprocessing and label mapping, train with logits, evaluate without gradients, aggregate sample-weighted loss, and save a versioned report.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch
import torch.nn.functional as F

def E15(model, train_loader, test_loader,
        optimizer, configuration):
    for epoch in range(configuration["epochs"]):
        model.train()
        for features, targets in train_loader:
            optimizer.zero_grad(set_to_none=True)
            logits = model(features)
            loss = F.cross_entropy(logits, targets)
            ...
    model.eval()
    with torch.inference_mode():
```

```
        evaluation = evaluate(
            model, test_loader, configuration
        )
    return evaluation

def main():
    print(E15(...))

if __name__ == "__main__":
    main()
```

11.10.17 Exercise 16 - Production Readiness Check

Part 1 - Why this matters

Production Readiness Check extends binary classification reasoning to problems containing several possible classes. Correct implementation depends on preserving class identity, tensor shape, and the distinction between mutually exclusive and independently valid labels.

A multi-class result can look correct while hiding imbalance, calibration error, class-specific failure, or excessive resource cost. The experiment therefore records both aggregate behavior and evidence for individual classes.

Part 2 - Mathematical form

$$\text{ready} = \bigwedge_{j} \text{check}_j.$$

Part 3 - Hints and Helper

Check label-map version, input schema, finite logits, probability sums, class support, latency, memory, calibration, robustness, and artifact loading.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch
import torch.nn.functional as F

def E16(model, sample_batch, requirements):
    checks = {}
    model.eval()
    with torch.inference_mode():
        logits = model(sample_batch)
        probabilities = torch.softmax(logits, dim=1)
    checks["shape"] = ...
    checks["finite"] = ...
```

```
    checks["probability_sum"] = ...
    checks["latency"] = ...
    checks["class_mapping"] = ...
    return {
        "checks": checks,
        "ready": all(checks.values()),
    }

def main():
    print(E16(...))

if __name__ == "__main__":
    main()
```

11.10.18 Exercise 17 - Failure Recovery Scenario

Part 1 - Why this matters

Failure Recovery Scenario extends binary classification reasoning to problems containing several possible classes. Correct implementation depends on preserving class identity, tensor shape, and the distinction between mutually exclusive and independently valid labels.

A multi-class result can look correct while hiding imbalance, calibration error, class-specific failure, or excessive resource cost. The experiment therefore records both aggregate behavior and evidence for individual classes.

Part 2 - Mathematical form

$$\text{recover} = \text{detect} \rightarrow \text{contain} \rightarrow \text{restore} \rightarrow \text{verify}.$$

Part 3 - Hints and Helper

Simulate invalid labels, non-finite loss, checkpoint corruption, and missing classes independently. Never overwrite the last known-good state before verification.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch
import torch.nn.functional as F

def E17(model, checkpoint, batch, policy):
    diagnostics = []
    try:
        features, targets = batch
        if targets.min() < 0 or \
                targets.max() >= policy["classes"]:
```

```
            raise ValueError(...)
        logits = model(features)
        loss = F.cross_entropy(logits, targets)
        if not torch.isfinite(loss):
            raise FloatingPointError(...)
        return {"recovered": False, "status": "valid"}
    except (ValueError, RuntimeError,
            FloatingPointError) as error:
        diagnostics.append(...)
        model.load_state_dict(checkpoint)
        verification = ...
        return {
            "recovered": ...,
            "diagnostics": diagnostics,
            "verification": verification,
        }

def main():
    print(E17(...))

if __name__ == "__main__":
    main()
```

11.10.19 Exercise 18 - Scalability Stress Test

Part 1 - Why this matters

Scalability Stress Test extends binary classification reasoning to problems containing several possible classes. Correct implementation depends on preserving class identity, tensor shape, and the distinction between mutually exclusive and independently valid labels.

A multi-class result can look correct while hiding imbalance, calibration error, class-specific failure, or excessive resource cost. The experiment therefore records both aggregate behavior and evidence for individual classes.

Part 2 - Mathematical form

$$(N, C, d) \mapsto \{\text{latency}, \text{throughput}, \text{memory}, \text{status}\}.$$

Part 3 - Hints and Helper

Sweep one dimension at a time, warm up accelerators, synchronize timing, catch out-of-memory safely, and preserve the largest successful configuration.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
```

```
import torch
import torch.nn.functional as F

def E18(model_factory, configurations, device):
    rows = []
    for configuration in configurations:
        try:
            model = model_factory(
                configuration
            ).to(device)
            features = torch.randn(
                configuration["samples"],
                configuration["features"],
                device=device,
            )
            ...
            rows.append({
                "configuration": configuration,
                "status": "passed",
                "latency": ...,
                "peak_memory": ...,
            })
        except torch.OutOfMemoryError:
            rows.append({
                "configuration": configuration,
                "status": "out_of_memory",
            })
            ...
    return rows

def main():
    print(E18(...))

if __name__ == "__main__":
    main()
```

11.10.20 Exercise 19 - Final Multi-Class Report

Part 1 - Why this matters

Final Multi-Class Report extends binary classification reasoning to problems containing several possible classes. Correct implementation depends on preserving class identity, tensor shape, and the distinction between mutually exclusive and independently valid labels.

A multi-class result can look correct while hiding imbalance, calibration error, class-specific failure, or excessive resource cost. The experiment therefore records both aggregate behavior and evidence for individual classes.

Part 2 - Mathematical form

$$R = \{\text{data}, \text{model}, \text{loss}, \text{metrics}, \text{calibration}, \text{errors}, \text{resources}, \text{robustness}, \text{status}\}.$$

Part 3 - Hints and Helper

Combine validated sections, mark missing evidence, convert tensors to serializable values, and derive status from explicit deployment rules.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch
import torch.nn.functional as F

def E19(sections, rules):
    required = {
        "data", "model", "loss", "metrics",
        "calibration", "errors", "resources",
        "robustness",
    }
    missing = sorted(required - sections.keys())
    warnings = []
    if missing:
        warnings.append(...)
    if sections.get(
        "calibration", {}
    ).get("ece", 0.0) > rules["maximum_ece"]:
        warnings.append(...)
    if sections.get(
        "metrics", {}
    ).get("macro_f1", 0.0) < rules["minimum_macro_f1"]:
        warnings.append(...)
    return {
        "sections": sections,
        "missing": missing,
        "warnings": warnings,
        "status": ...,
    }

def main():
    print(E19(...))

if __name__ == "__main__":
    main()
```

Book 12

Perceptron

12.1 Introduction

12.1.1 What will the book teach you

The previous books established the mathematical and computational foundations of model training. Linear regression introduced weighted inputs and continuous prediction, while logistic regression transformed a linear score into a probability and connected classification to likelihood, cross-entropy, gradients, decision thresholds, and evaluation metrics. This book now studies the perceptron: one of the simplest models that can learn a binary decision boundary directly from classification errors. Its small size makes every operation visible, but its apparent simplicity should not be mistaken for a lack of depth. The perceptron exposes several ideas that later become central to neural networks: weighted inputs, bias, activation, prediction, error-driven parameter updates, repeated training, convergence, generalization, and the need for multiple computational units.

A perceptron receives a feature vector $\mathbf{x} \in \mathbb{R}^d$, combines it with a weight vector $\mathbf{w} \in \mathbb{R}^d$ and a bias b, and calculates the linear score

$$z = \mathbf{w}^\top \mathbf{x} + b.$$

The score is then passed through a step function. Using binary labels $\{0, 1\}$, one common definition is

$$\widehat{y} = H(z) = \begin{cases} 1, & z \geq 0, \\ 0, & z < 0. \end{cases}$$

This produces a hard binary decision rather than a probability. A score of 0.01 and a score of 100 both produce the same class under this rule, even though their distances from the boundary are very different. The early exercises will require you to implement this operation using pure Python and PyTorch, test individual and batched inputs, verify shapes and dtypes, and compare CPU and CUDA results. You will also compare the perceptron with linear and logistic regression so that an unrestricted numerical output, a probability, and a hard class decision never become interchangeable concepts.

The parameters define a geometric boundary:

$$\mathbf{w}^\top \mathbf{x} + b = 0.$$

The weight vector is normal to this boundary, while the bias controls its position. In two dimensions the boundary is a line; in three dimensions it is a plane; and in higher dimensions it is a hyperplane. The book will examine this geometry through vector calculations, boundary plots, signed distances, feature scaling, perturbations, and numerical precision. You will investigate how small changes to weights, bias, inputs, or dtype can move samples from one side of the boundary to the other. These experiments are essential because a binary output alone hides how close a decision was to changing.

Unlike logistic regression, the classical perceptron does not learn by minimizing binary cross-entropy through ordinary backpropagation. Its hard step function is constant on either side of the boundary and is not differentiable at the boundary itself. Consequently, the useful derivative structure needed for gradient-based learning is absent. The classical perceptron instead uses an error-driven update rule. For a training example $(\mathbf{x}_i, y_i)$ with prediction $\widehat{y}_i$, one binary-label form is

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \left(y_i - \widehat{y}_i\right) \mathbf{x}_i,$$

$$b \leftarrow b + \eta \left(y_i - \widehat{y}_i\right),$$

where $\eta > 0$ is the learning rate. A correctly classified example produces no update in this formulation, while a mistake moves the boundary in a direction intended to correct that example. You will derive this rule, implement it manually, inspect the direction of every update, compare online and batch-like behavior, test sample ordering, inject controlled noise, and verify calculations by hand before trusting automated code.

The update rule leads to one of the most important theoretical results in early machine learning: the perceptron convergence theorem. If the training data are linearly separable, the classical algorithm will find a separating hyperplane after a finite number of mistakes under its assumptions. The theorem does not say that every dataset is separable, that the first separator is unique, or that the resulting classifier is automatically the best model for unseen data. It also does not promise convergence when classes overlap or labels contradict one another. This book will test the theorem experimentally by measuring updates, epochs, margins, weight norms, and the effects of data order. You will contrast cleanly separable data with nearly separable and non-separable cases instead of treating non-convergence as an unexplained programming failure.

The margin provides a more informative view of a classified point than its hard label alone. With labels $t_i \in \{-1, +1\}$, the signed functional margin is

$$\gamma_i = t_i \left(\mathbf{w}^\top \mathbf{x}_i + b\right).$$

A positive value indicates correct placement relative to the boundary, while a negative value indicates a mistake. After normalization by $\|\mathbf{w}\|_2$, the quantity gains a geometric distance interpretation. You will observe how margin size influences stability, convergence speed, sensitivity to noise, and generalization. You must nevertheless avoid claiming that the classical perceptron automatically finds the maximum-margin separator; that is a different optimization objective.

The book will also reinterpret perceptron learning through the perspective of loss and error. Zero–one loss expresses whether a decision is correct, but its discontinuous landscape is unsuitable for ordinary gradient optimization. Perceptron loss provides a margin-based view of mis-

classified examples and can be compared with hinge loss and differentiable classification losses. You will implement these quantities, visualize their behavior, inspect their non-smoothness, compare error-driven updates with gradient-based methods, and identify situations in which reducing a training objective does not guarantee useful real-world behavior. This section connects the historical perceptron algorithm to the broader optimization language developed in earlier books.

A single perceptron is limited to one linear boundary. The XOR problem provides the clearest demonstration:

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

No single line can separate the positive XOR points from the negative points. The exercises will require you to visualize this failure, try multiple initializations, and explain the geometric reason rather than repeatedly changing the learning rate in search of an impossible solution. This limitation is not a minor edge case; it establishes why a single linear decision unit cannot represent every classification rule.

The perceptron can be extended to multiple classes by maintaining class-specific weight vectors and selecting the class with the largest score. You will implement one-vs-rest and direct multi-class update strategies, construct decision regions, analyze confusion matrices and class imbalance, and measure how computation and memory grow with the number of classes. These experiments will be compared with softmax regression so that a collection of hard class scores is not confused with a normalized probability model.

Computational performance will remain part of the learning process. Because a single perceptron performs little work, moving a small batch to a GPU may be slower than executing it on the CPU due to transfer and kernel-launch overhead. You will benchmark forward passes and training, measure throughput, latency, allocated memory, synchronization cost, batch-size scaling, precision effects, and multi-GPU overhead. CUDA will be used when it provides a meaningful experiment, not treated as an automatic guarantee of speed. Every comparison must include warm-up, synchronization where required, identical workloads, and the hardware context needed to interpret the result.

The later chapters examine generalization and theoretical limits. You will separate training accuracy from test accuracy, repeat experiments across random seeds, introduce label and input noise, study distribution shift, and inspect high-dimensional sparse data. A classifier can fit linearly separable training examples and still perform poorly on new data. Conversely, failure to achieve perfect training accuracy may reflect irreducible overlap rather than defective code. The goal is to connect theoretical assumptions to measured evidence and to state clearly when those assumptions do not represent the dataset.

The final chapter provides the bridge from one perceptron to multi-layer neural networks. By combining several decision units, intermediate units can transform the original feature space and make patterns such as XOR separable in a learned representation. You will manu-

ally combine perceptrons, interpret hidden units, construct a two-layer model, inspect hidden activations, compare manual code with `nn.Module`, and study gradient flow through multiple layers. At this point the classical perceptron learning rule is no longer sufficient for coordinating all parameters; backpropagation and differentiable activations become necessary. The chapter therefore ends where the next book on multi-layer perceptrons begins.

This book is not intended to present the perceptron as a modern solution for every classification task. Its purpose is to make the transition from a single linear decision rule to a trainable network intellectually complete. By the end, you should be able to explain exactly what the perceptron computes, why its update rule moves the boundary, when convergence is guaranteed, why it can fail, how its implementation behaves on CPU and CUDA, and why additional layers create capabilities that one unit cannot possess.

12.1.2 How to study this book

Treat each exercise as a controlled experiment. Before running code, write down the feature shape, weight shape, score shape, label convention, expected output dtype, and the precise condition that triggers an update. Predict whether a sample is correctly classified and calculate at least one update manually. After execution, compare the stored tensors with the manual result rather than accepting the output because the program completed.

Keep the label convention explicit. Some derivations use $\{0, 1\}$ while others use $\{-1, +1\}$. Both are valid when the prediction and update formulas are transformed consistently, but mixing them silently reverses or removes updates. Record the convention at the beginning of every experiment, and test boundary cases such as a score equal to zero.

When studying convergence, change one factor at a time. Preserve the dataset, initial parameters, learning rate, random seed, precision, and sample order unless one of those variables is the subject of the experiment. Never claim that the convergence theorem failed until you have verified linear separability, label consistency, finite values, and the exact implementation of the update rule.

Begin with small CPU examples that can be drawn and calculated by hand. Move to CUDA only after the mathematical behavior is verified. For performance experiments, separate data-transfer time from computation, warm up the device, synchronize timed CUDA work, and repeat measurements. For numerical experiments, compare tolerances and decision changes rather than demanding bitwise equality between devices.

For every challenge, answer four questions: What did the perceptron compute? Why did it update or remain unchanged? Which assumption makes the result valid? Under what condition would the same method fail? If you cannot answer all four using the recorded tensors and equations, repeat the exercise before moving to the next chapter.

12.1.3 The Chapters (PC00 - PC08) included in this book

- PC00 - Perceptron Concept and Motivation
- PC01 - Mathematical Structure of the Perceptron

- PC02 - Perceptron Learning Rule and Updates
- PC03 - Training Dynamics and Convergence Theory
- PC04 - Perceptron Loss Perspective and Errors
- PC05 - Multi-Class Perceptron Extensions
- PC06 - Computational Performance and CUDA Effects
- PC07 - Generalization and Theoretical Limits
- PC08 - Transition to Multi-Layer Networks

12.2 PC00 - Perceptron Concept and Motivation

Models and Training

12.2.1 Exercise 00 - Understand the Limitation of Linear Regression for Classification

Part 1 - Why this matters

Understand the Limitation of Linear Regression for Classification establishes one part of the perceptron's role as a hard binary decision model. Because the model is small, every score, shape, threshold, and parameter choice can be inspected directly instead of being hidden inside a large architecture.

The same simplicity can create misleading confidence. A binary output does not express probability, a fast computation does not prove suitability, and failure to converge may reflect non-separable data rather than incorrect code. This exercise therefore records both the result and the assumption supporting it.

Part 2 - Mathematical form

$$\widehat{y}_{\text{linear}} = \mathbf{w}^{\mathsf{T}}\mathbf{x} + b \in \mathbb{R}.$$

Part 3 - Hints and Helper

Create inputs whose linear outputs fall below zero and above one. Show why an unrestricted regression value is not itself a valid binary decision or probability.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E00(features, weight, bias, threshold):
```

```
    outputs = features @ weight + bias
    decisions = (outputs >= threshold).to(torch.int64)
    return {
        "raw_outputs": outputs,
        "decisions": decisions,
        "outside_unit_interval": ...,
    }

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

12.2.2 Exercise 01 - Define the Perceptron as a Binary Decision Unit with Step Function

Part 1 - Why this matters

Define the Perceptron as a Binary Decision Unit with Step Function establishes one part of the perceptron's role as a hard binary decision model. Because the model is small, every score, shape, threshold, and parameter choice can be inspected directly instead of being hidden inside a large architecture.

The same simplicity can create misleading confidence. A binary output does not express probability, a fast computation does not prove suitability, and failure to converge may reflect non-separable data rather than incorrect code. This exercise therefore records both the result and the assumption supporting it.

Part 2 - Mathematical form

$$z = \mathbf{w}^{\mathsf{T}}\mathbf{x} + b, \qquad \widehat{y} = H(z) = \mathbb{I}[z \geq 0].$$

Part 3 - Hints and Helper

Keep the score and decision as separate outputs. State the convention for an exact zero score and use it consistently.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E01(features, weight, bias):
    scores = features @ weight + bias
    predictions = ...
    return scores, predictions
```

```
def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

12.2.3 Exercise 02 - Implement a Basic Perceptron Forward Pass Using Pure Python

Part 1 - Why this matters

Implement a Basic Perceptron Forward Pass Using Pure Python establishes one part of the perceptron's role as a hard binary decision model. Because the model is small, every score, shape, threshold, and parameter choice can be inspected directly instead of being hidden inside a large architecture.

The same simplicity can create misleading confidence. A binary output does not express probability, a fast computation does not prove suitability, and failure to converge may reflect non-separable data rather than incorrect code. This exercise therefore records both the result and the assumption supporting it.

Part 2 - Mathematical form

$$z = \sum_{j=1}^{d} w_j x_j + b.$$

Part 3 - Hints and Helper

Use Python lists and a loop without NumPy or PyTorch. Validate equal lengths, calculate the score, then apply the declared step rule.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E02(features, weights, bias):
    if len(features) != len(weights):
        raise ValueError(...)
    score = bias
    for feature, weight in zip(features, weights):
        score += ...
    prediction = ...
    return score, prediction

def main():
```

```
    print(E02(...))

if __name__ == "__main__":
    main()
```

12.2.4 Exercise 03 - Translate Mathematical Perceptron Formula into PyTorch Operations

Part 1 - Why this matters

Translate Mathematical Perceptron Formula into PyTorch Operations establishes one part of the perceptron's role as a hard binary decision model. Because the model is small, every score, shape, threshold, and parameter choice can be inspected directly instead of being hidden inside a large architecture.

The same simplicity can create misleading confidence. A binary output does not express probability, a fast computation does not prove suitability, and failure to converge may reflect non-separable data rather than incorrect code. This exercise therefore records both the result and the assumption supporting it.

Part 2 - Mathematical form

$$\mathbf{z} = \mathbf{X}\mathbf{w} + b, \qquad \widehat{\mathbf{y}} = \mathbb{I}[\mathbf{z} \geq 0].$$

Part 3 - Hints and Helper

Use a two-dimensional batch and one-dimensional weight tensor. Predict output shapes before calculation and reject mismatched feature width.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E03(features, weight, bias):
    if features.ndim != 2 or weight.ndim != 1:
        raise ValueError(...)
    if features.shape[1] != weight.shape[0]:
        raise ValueError(...)
    scores = torch.matmul(features, weight) + bias
    predictions = ...
    return scores, predictions

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

12.2.5 Exercise 04 - Compare Linear Regression Output vs Perceptron Binary Output

Part 1 - Why this matters

Compare Linear Regression Output vs Perceptron Binary Output establishes one part of the perceptron's role as a hard binary decision model. Because the model is small, every score, shape, threshold, and parameter choice can be inspected directly instead of being hidden inside a large architecture.

The same simplicity can create misleading confidence. A binary output does not express probability, a fast computation does not prove suitability, and failure to converge may reflect non-separable data rather than incorrect code. This exercise therefore records both the result and the assumption supporting it.

Part 2 - Mathematical form

$$r(\mathbf{x}) = \mathbf{w}^{\mathsf{T}}\mathbf{x} + b, \qquad H(r) \in \{0, 1\}.$$

Part 3 - Hints and Helper

Use identical parameters and inputs for both views. Report raw magnitude, hard decision, and samples whose regression output cannot be interpreted as probability.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E04(features, weight, bias):
    linear_outputs = features @ weight + bias
    perceptron_outputs = (
        linear_outputs >= 0
    ).to(torch.int64)
    return {
        "linear": linear_outputs,
        "perceptron": perceptron_outputs,
        "same_shape": ...,
        "information_removed": ...,
    }

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

12.2.6 Exercise 05 - Analyze Why Perceptron Uses a Hard Decision Boundary

Part 1 - Why this matters

Analyze Why Perceptron Uses a Hard Decision Boundary establishes one part of the perceptron's role as a hard binary decision model. Because the model is small, every score, shape, threshold, and parameter choice can be inspected directly instead of being hidden inside a large architecture.

The same simplicity can create misleading confidence. A binary output does not express probability, a fast computation does not prove suitability, and failure to converge may reflect non-separable data rather than incorrect code. This exercise therefore records both the result and the assumption supporting it.

Part 2 - Mathematical form

$$\mathbf{w}^{\mathsf{T}}\mathbf{x} + b = 0$$

divides the feature space into two half-spaces.

Part 3 - Hints and Helper

Evaluate points immediately on both sides of the boundary. Record scores and show that arbitrarily small sign changes can flip the class.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E05(boundary_point, normal, bias, epsilons):
    unit_normal = normal / torch.linalg.vector_norm(normal)
    rows = []
    for epsilon in epsilons:
        positive_point = boundary_point + epsilon * unit_normal
        negative_point = boundary_point - epsilon * unit_normal
        rows.append({
            "epsilon": epsilon,
            "positive_score": ...,
            "negative_score": ...,
            "predictions": ...,
        })
    return rows

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

12.2.7 Exercise 06 - Implement the Step Activation Function and Test Its Behavior

Part 1 - Why this matters

Implement the Step Activation Function and Test Its Behavior establishes one part of the perceptron's role as a hard binary decision model. Because the model is small, every score, shape, threshold, and parameter choice can be inspected directly instead of being hidden inside a large architecture.

The same simplicity can create misleading confidence. A binary output does not express probability, a fast computation does not prove suitability, and failure to converge may reflect non-separable data rather than incorrect code. This exercise therefore records both the result and the assumption supporting it.

Part 2 - Mathematical form

$$H(z) = \begin{cases} 1, & z \geq 0, \\ 0, & z < 0. \end{cases}$$

Part 3 - Hints and Helper

Test negative, zero, positive, extreme, and non-finite inputs. Decide whether non-finite scores should raise an error instead of becoming classes.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def step(scores):
    if not torch.isfinite(scores).all():
        raise ValueError(...)
    return (scores >= 0).to(torch.int64)

def E06(test_values):
    outputs = step(test_values)
    return {
        "inputs": test_values,
        "outputs": outputs,
        "binary_only": ...,
    }

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

12.2.8 Exercise 07 - Visualize Binary Decisions Generated by a Single Perceptron

Part 1 - Why this matters

Visualize Binary Decisions Generated by a Single Perceptron establishes one part of the perceptron's role as a hard binary decision model. Because the model is small, every score, shape, threshold, and parameter choice can be inspected directly instead of being hidden inside a large architecture.

The same simplicity can create misleading confidence. A binary output does not express probability, a fast computation does not prove suitability, and failure to converge may reflect non-separable data rather than incorrect code. This exercise therefore records both the result and the assumption supporting it.

Part 2 - Mathematical form

$$\widehat{y}(\mathbf{x}) = \mathbb{I}[w_1 x_1 + w_2 x_2 + b \geq 0].$$

Part 3 - Hints and Helper

Build a two-feature grid, evaluate all points vectorially, reshape decisions, and overlay the boundary and labeled samples.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E07(weight, bias, x_axis, y_axis,
        output_path):
    grid_x, grid_y = torch.meshgrid(
        x_axis, y_axis, indexing="xy"
    )
    points = torch.stack(
        [grid_x.flatten(), grid_y.flatten()],
        dim=1,
    )
    decisions = (
        points @ weight + bias >= 0
    ).reshape(grid_x.shape)
    fig, ax = plt.subplots()
    ax.contourf(..., ..., ..., alpha=...)
    ...
    fig.savefig(output_path, dpi=150,
                bbox_inches="tight")
    plt.close(fig)
```

```
    return decisions

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

12.2.9 Exercise 08 - Test Perceptron on Linearly Separable Dataset

Part 1 - Why this matters

Test Perceptron on Linearly Separable Dataset establishes one part of the perceptron's role as a hard binary decision model. Because the model is small, every score, shape, threshold, and parameter choice can be inspected directly instead of being hidden inside a large architecture.

The same simplicity can create misleading confidence. A binary output does not express probability, a fast computation does not prove suitability, and failure to converge may reflect non-separable data rather than incorrect code. This exercise therefore records both the result and the assumption supporting it.

Part 2 - Mathematical form

$$\exists(\mathbf{w}, b): \quad t_i(\mathbf{w}^\mathsf{T}\mathbf{x}_i + b) > 0 \quad \forall i.$$

Part 3 - Hints and Helper

Generate or receive clearly separable data, train with the classical update rule, and record mistakes per epoch. Stop only after a full mistake-free pass.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E08(features, targets, learning_rate,
        maximum_epochs):
    weight = torch.zeros(features.shape[1])
    bias = torch.tensor(0.0)
    history = []
    for epoch in range(maximum_epochs):
        mistakes = 0
        for sample, target in zip(features, targets):
            prediction = ...
            error = target - prediction
            if error != 0:
                weight += ...
```

```
                bias += ...
                mistakes += 1
        history.append(mistakes)
        if mistakes == 0:
            break
    return weight, bias, history

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

12.2.10 Exercise 09 - Observe Failure on Non-Linearly Separable Data

Part 1 - Why this matters

Observe Failure on Non-Linearly Separable Data establishes one part of the perceptron's role as a hard binary decision model. Because the model is small, every score, shape, threshold, and parameter choice can be inspected directly instead of being hidden inside a large architecture.

The same simplicity can create misleading confidence. A binary output does not express probability, a fast computation does not prove suitability, and failure to converge may reflect non-separable data rather than incorrect code. This exercise therefore records both the result and the assumption supporting it.

Part 2 - Mathematical form

$$\nexists(\mathbf{w}, b): \quad t_i(\mathbf{w}^\mathsf{T}\mathbf{x}_i + b) > 0 \quad \forall i.$$

Part 3 - Hints and Helper

Use XOR or contradictory labels. Set a finite epoch budget, log repeated mistakes and weight states, and avoid calling non-convergence a software error without checking separability.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E09(features, targets, learning_rate,
        maximum_epochs):
    weight = torch.zeros(features.shape[1])
    bias = torch.tensor(0.0)
    history = []
```

```
    for epoch in range(maximum_epochs):
        mistakes = train_one_epoch(
            features, targets, weight,
            bias, learning_rate
        )
        history.append({
            "epoch": epoch,
            "mistakes": mistakes,
            "weight": weight.clone(),
            "bias": bias.clone(),
        })
    return {
        "converged": ...,
        "history": history,
    }

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

12.2.11 Exercise 10 - Move Perceptron Computation to CUDA and Verify Output Consistency

Part 1 - Why this matters

Move Perceptron Computation to CUDA and Verify Output Consistency establishes one part of the perceptron's role as a hard binary decision model. Because the model is small, every score, shape, threshold, and parameter choice can be inspected directly instead of being hidden inside a large architecture.

The same simplicity can create misleading confidence. A binary output does not express probability, a fast computation does not prove suitability, and failure to converge may reflect non-separable data rather than incorrect code. This exercise therefore records both the result and the assumption supporting it.

Part 2 - Mathematical form

$$\hat{\mathbf{y}}_{\text{cpu}} \approx \hat{\mathbf{y}}_{\text{cuda}}.$$

Part 3 - Hints and Helper

Use CUDA only when available and move features, weights, and bias together. Compare scores with tolerance and decisions exactly, while flagging near-zero scores.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E10(features, weight, bias):
    cpu_scores = features @ weight + bias
    cpu_predictions = (cpu_scores >= 0)
    if not torch.cuda.is_available():
        return {"cuda_available": False}
    cuda_scores = (
        features.cuda() @ weight.cuda()
        + bias.cuda()
    )
    cuda_predictions = cuda_scores >= 0
    return {
        "score_close": ...,
        "prediction_equal": ...,
        "minimum_absolute_score": ...,
    }

def main():
    print(E10(...))

if __name__ == "__main__":
    main()
```

12.2.12 Exercise 11 - Compare CPU vs GPU Execution for Perceptron Forward Pass

Part 1 - Why this matters

Compare CPU vs GPU Execution for Perceptron Forward Pass establishes one part of the perceptron's role as a hard binary decision model. Because the model is small, every score, shape, threshold, and parameter choice can be inspected directly instead of being hidden inside a large architecture.

The same simplicity can create misleading confidence. A binary output does not express probability, a fast computation does not prove suitability, and failure to converge may reflect non-separable data rather than incorrect code. This exercise therefore records both the result and the assumption supporting it.

Part 2 - Mathematical form

$$\text{Throughput} = \frac{N}{T}, \qquad \text{Speedup} = \frac{T_{\text{cpu}}}{T_{\text{gpu}}}.$$

Part 3 - Hints and Helper

Warm up CUDA, synchronize timed GPU work, and report transfer-inclusive and compute-only timing separately. Repeat enough iterations for a stable comparison.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E11(features, weight, bias, repetitions):
    cpu_seconds = benchmark(
        features, weight, bias,
        repetitions, synchronize=False
    )
    if not torch.cuda.is_available():
        return {"cuda_available": False}
    cuda_features = features.cuda()
    cuda_weight = weight.cuda()
    cuda_bias = bias.cuda()
    ...
    cuda_seconds = benchmark(
        cuda_features, cuda_weight,
        cuda_bias, repetitions,
        synchronize=True,
    )
    return {
        "cpu_seconds": cpu_seconds,
        "cuda_seconds": cuda_seconds,
        "speedup": ...,
    }

def main():
    print(E11(...))

if __name__ == "__main__":
    main()
```

12.2.13 Exercise 12 - Inspect Weight and Bias Initialization Effects on Decisions

Part 1 - Why this matters

Inspect Weight and Bias Initialization Effects on Decisions establishes one part of the perceptron's role as a hard binary decision model. Because the model is small, every score, shape, threshold, and parameter choice can be inspected directly instead of being hidden inside a large architecture.

The same simplicity can create misleading confidence. A binary output does not express probability, a fast computation does not prove suitability, and failure to converge may reflect non-separable data rather than incorrect code. This exercise therefore records both the result and the assumption supporting it.

Part 2 - Mathematical form

$$z^{(k)} = \mathbf{w}^{(k)^\mathsf{T}}\mathbf{x} + b^{(k)}.$$

Part 3 - Hints and Helper

Evaluate zero, small random, large random, and biased initializations on the same fixed inputs. Compare positive rate, score spread, and boundary orientation.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E12(features, initializations):
    rows = []
    for name, state in initializations.items():
        scores = (
            features @ state["weight"]
            + state["bias"]
        )
        predictions = scores >= 0
        rows.append({
            "name": name,
            "positive_rate": ...,
            "score_mean": ...,
            "score_std": ...,
            "weight_norm": ...,
        })
    return rows

def main():
    print(E12(...))

if __name__ == "__main__":
    main()
```

12.2.14 Exercise 13 - Analyze Sensitivity of Decision Boundary to Weight Changes

Part 1 - Why this matters

Analyze Sensitivity of Decision Boundary to Weight Changes establishes one part of the perceptron's role as a hard binary decision model. Because the model is small, every score, shape, threshold, and parameter choice can be inspected directly instead of being hidden inside a large architecture.

The same simplicity can create misleading confidence. A binary output does not express probability, a fast computation does not prove suitability, and failure to converge may reflect non-separable data rather than incorrect code. This exercise therefore records both the result and the assumption supporting it.

Part 2 - Mathematical form

$$\Delta z_i = \Delta \mathbf{w}^\mathsf{T} \mathbf{x}_i, \qquad \widehat{y}_i \text{ flips if the sign changes.}$$

Part 3 - Hints and Helper

Perturb one weight or the full vector while keeping inputs and bias fixed. Report score changes, flip indices, and distance of affected samples from the original boundary.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E13(features, weight, bias, perturbations):
    original_scores = features @ weight + bias
    original_predictions = original_scores >= 0
    rows = []
    for perturbation in perturbations:
        new_scores = (
            features @ (weight + perturbation) + bias
        )
        flips = (
            (new_scores >= 0)
            != original_predictions
        )
        rows.append({
            "perturbation_norm": ...,
            "maximum_score_change": ...,
            "flip_indices": ...,
        })
    return rows

def main():
    print(E13(...))

if __name__ == "__main__":
    main()
```

12.2.15 Exercise 14 - Implement Perceptron without Using nn.Module

Part 1 - Why this matters

Implement Perceptron without Using nn.Module establishes one part of the perceptron's role as a hard binary decision model. Because the model is small, every score, shape, threshold, and parameter choice can be inspected directly instead of being hidden inside a large architecture.

The same simplicity can create misleading confidence. A binary output does not express probability, a fast computation does not prove suitability, and failure to converge may reflect non-separable data rather than incorrect code. This exercise therefore records both the result and the assumption supporting it.

Part 2 - Mathematical form

$$f(\mathbf{X}; \mathbf{w}, b) = H(\mathbf{X}\mathbf{w} + b).$$

Part 3 - Hints and Helper

Store tensors directly and expose explicit forward and update functions. Do not use nn.Module, nn.Linear, or an optimizer in this exercise.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

class PlainPerceptron:
    def __init__(self, feature_count):
        self.weight = torch.zeros(feature_count)
        self.bias = torch.tensor(0.0)

    def forward(self, features):
        scores = ...
        return scores, ...

    def update(self, sample, target, learning_rate):
        _, prediction = self.forward(
            sample.unsqueeze(0)
        )
        error = ...
        self.weight += ...
        self.bias += ...

def E14(...):
```

```
    model = PlainPerceptron(...)
    ...
    return model

def main():
    print(E14(...))

if __name__ == "__main__":
    main()
```

12.2.16 Exercise 15 - Validate Output Shape and Data Type in Classification Pipeline

Part 1 - Why this matters

Validate Output Shape and Data Type in Classification Pipeline establishes one part of the perceptron's role as a hard binary decision model. Because the model is small, every score, shape, threshold, and parameter choice can be inspected directly instead of being hidden inside a large architecture.

The same simplicity can create misleading confidence. A binary output does not express probability, a fast computation does not prove suitability, and failure to converge may reflect non-separable data rather than incorrect code. This exercise therefore records both the result and the assumption supporting it.

Part 2 - Mathematical form

$$\mathbf{X} \in \mathbb{R}^{N \times d} \Rightarrow \mathbf{z} \in \mathbb{R}^{N}, \quad \hat{\mathbf{y}} \in \{0,1\}^{N}.$$

Part 3 - Hints and Helper

Require floating features and scores, integer or Boolean decisions, and one output per sample. Reject unintended broadcasting.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E15(features, weight, bias):
    issues = []
    if features.ndim != 2:
        issues.append(...)
    if weight.ndim != 1:
        issues.append(...)
    if features.shape[-1] != weight.numel():
        issues.append(...)
```

```
    scores = features @ weight + bias
    predictions = (scores >= 0).to(torch.int64)
    if predictions.shape != (features.shape[0],):
        issues.append(...)
    return scores, predictions, issues

def main():
    print(E15(...))

if __name__ == "__main__":
    main()
```

12.2.17 Exercise 16 - Freeze Perceptron Parameters and Test Prediction Stability

Part 1 - Why this matters

Freeze Perceptron Parameters and Test Prediction Stability establishes one part of the perceptron's role as a hard binary decision model. Because the model is small, every score, shape, threshold, and parameter choice can be inspected directly instead of being hidden inside a large architecture.

The same simplicity can create misleading confidence. A binary output does not express probability, a fast computation does not prove suitability, and failure to converge may reflect non-separable data rather than incorrect code. This exercise therefore records both the result and the assumption supporting it.

Part 2 - Mathematical form

$$\theta_{\text{before}} = \theta_{\text{after}} \Rightarrow \widehat{\mathbf{y}}_{\text{before}} = \widehat{\mathbf{y}}_{\text{after}}$$

for identical deterministic inputs.

Part 3 - Hints and Helper

Disable parameter gradients or skip manual updates, preserve cloned parameters, run repeated predictions, and verify both state and output equality.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E16(model, features, repetitions):
    for parameter in model.parameters():
        parameter.requires_grad_(False)
    before = {
        name: parameter.detach().clone()
```

```
        for name, parameter in model.named_parameters()
    }
    outputs = []
    with torch.inference_mode():
        for _ in range(repetitions):
            outputs.append(model(features))
    after = dict(model.named_parameters())
    return {
        "parameters_unchanged": ...,
        "predictions_stable": ...,
    }

def main():
    print(E16(...))

if __name__ == "__main__":
    main()
```

12.2.18 Exercise 17 - Debug Incorrect Binary Outputs Caused by Shape Mismatch

Part 1 - Why this matters

Debug Incorrect Binary Outputs Caused by Shape Mismatch establishes one part of the perceptron's role as a hard binary decision model. Because the model is small, every score, shape, threshold, and parameter choice can be inspected directly instead of being hidden inside a large architecture.

The same simplicity can create misleading confidence. A binary output does not express probability, a fast computation does not prove suitability, and failure to converge may reflect non-separable data rather than incorrect code. This exercise therefore records both the result and the assumption supporting it.

Part 2 - Mathematical form

$$(N, d)(d, 1) \to (N, 1) \quad \neq \quad (N, d)(d) \to (N).$$

Part 3 - Hints and Helper

Inspect every intermediate shape, forbid silent broadcasting between scores and targets, and normalize only after deciding the intended interface.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch
```

```
def E17(features, weight, bias, targets):
    report = {
        "feature_shape": tuple(features.shape),
        "weight_shape": tuple(weight.shape),
        "target_shape": tuple(targets.shape),
    }
    issues = []
    if features.ndim != 2:
        issues.append(...)
    if weight.ndim not in {1, 2}:
        issues.append(...)
    scores = ...
    if scores.shape != targets.shape:
        issues.append(...)
    return report, issues

def main():
    print(E17(...))

if __name__ == "__main__":
    main()
```

12.2.19 Exercise 18 - Stress Test Perceptron with Large Batch Inputs

Part 1 - Why this matters

Stress Test Perceptron with Large Batch Inputs establishes one part of the perceptron's role as a hard binary decision model. Because the model is small, every score, shape, threshold, and parameter choice can be inspected directly instead of being hidden inside a large architecture.

The same simplicity can create misleading confidence. A binary output does not express probability, a fast computation does not prove suitability, and failure to converge may reflect non-separable data rather than incorrect code. This exercise therefore records both the result and the assumption supporting it.

Part 2 - Mathematical form

$$\mathbf{z} = \mathbf{X}\mathbf{w} + b, \qquad \text{cost} = O(Nd).$$

Part 3 - Hints and Helper

Generate batches incrementally, evaluate in chunks to control memory, time synchronized device work, and verify finite scores and correct output count.

Part 4 - Python code skeleton

```
import time
```

```
import matplotlib.pyplot as plt
import torch

def E18(weight, bias, batch_sizes,
        feature_count, device, chunk_size):
    rows = []
    for batch_size in batch_sizes:
        features = torch.randn(
            batch_size, feature_count,
            device=device,
        )
        outputs = []
        start = time.perf_counter()
        for chunk in features.split(chunk_size):
            outputs.append(
                (chunk @ weight + bias) >= 0
            )
        if device.type == "cuda":
            torch.cuda.synchronize(device)
        predictions = torch.cat(outputs)
        rows.append({
            "batch_size": batch_size,
            "seconds": ...,
            "output_count": ...,
            "all_valid": ...,
        })
    return rows

def main():
    print(E18(...))

if __name__ == "__main__":
    main()
```

12.2.20 Exercise 19 - Document Conceptual and Practical Failures of the Perceptron Model

Part 1 - Why this matters

Document Conceptual and Practical Failures of the Perceptron Model establishes one part of the perceptron's role as a hard binary decision model. Because the model is small, every score, shape, threshold, and parameter choice can be inspected directly instead of being hidden inside a large architecture.

The same simplicity can create misleading confidence. A binary output does not express probability, a fast computation does not prove suitability, and failure to converge may reflect non-separable data rather than incorrect code. This exercise therefore records both the result and the assumption supporting it.

Part 2 - Mathematical form

$$R = \{\text{assumptions}, \text{geometry}, \text{data}, \text{numerics}, \text{performance}, \text{failures}\}.$$

Part 3 - Hints and Helper

Combine results from separable, XOR, shape, precision, initialization, and performance experiments. Distinguish theoretical limitation from implementation failure.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E19(sections, required_sections):
    missing = sorted(
        set(required_sections) - sections.keys()
    )
    failures = []
    if sections.get(
        "nonlinear_data", {}
    ).get("converged", False):
        failures.append(...)
    if sections.get(
        "validation", {}
    ).get("issues"):
        failures.append(...)
    return {
        "sections": sections,
        "missing": missing,
        "failures": failures,
        "status": ...,
    }

def main():
    print(E19(...))

if __name__ == "__main__":
    main()
```

12.3 PC01 - Mathematical Structure of the Perceptron

Models and Training

12.3.1 Exercise 00 - Express the Perceptron as a Vector Dot Product Plus Bias

Part 1 - Why this matters

Express the Perceptron as a Vector Dot Product Plus Bias makes one mathematical component of the perceptron observable. The model's hard output hides score magnitude and geometry, so the underlying vectors, dimensions, boundary, and numerical representation must be studied directly.

A correct formula can still be interpreted incorrectly when weights are unnormalized, features use different scales, or a lower-dimensional plot discards information. This exercise therefore records the assumptions and transformations used with every result.

Part 2 - Mathematical form

$$z = \mathbf{w}^{\mathsf{T}}\mathbf{x} + b = \sum_{j=1}^{d} w_j x_j + b.$$

Part 3 - Hints and Helper

Calculate the dot product manually and with torch.dot. Verify identical feature counts and keep the scalar score separate from the step output.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E00(features, weight, bias):
    if features.ndim != 1 or weight.ndim != 1:
        raise ValueError(...)
    if features.numel() != weight.numel():
        raise ValueError(...)
    score = torch.dot(weight, features) + bias
    prediction = ...
    return score, prediction

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

12.3.2 Exercise 01 - Rewrite the Perceptron Equation Using Matrix Notation

Part 1 - Why this matters

Rewrite the Perceptron Equation Using Matrix Notation makes one mathematical component of the perceptron observable. The model's hard output hides score magnitude and geometry, so the underlying vectors, dimensions, boundary, and numerical representation must be studied directly.

A correct formula can still be interpreted incorrectly when weights are unnormalized, features use different scales, or a lower-dimensional plot discards information. This exercise therefore records the assumptions and transformations used with every result.

Part 2 - Mathematical form

$$\mathbf{z} = \mathbf{X}\mathbf{w} + b\mathbf{1}, \qquad \hat{\mathbf{y}} = \mathbb{I}[\mathbf{z} \geq 0].$$

Part 3 - Hints and Helper

Use X with shape N by d and weight with shape d. Compare the matrix result with stacking individual dot products.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E01(features, weight, bias):
    matrix_scores = torch.matmul(features, weight) + bias
    manual_scores = torch.stack([
        torch.dot(row, weight) + bias
        for row in features
    ])
    return {
        "scores": matrix_scores,
        "predictions": ...,
        "manual_equal": ...,
    }

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

12.3.3 Exercise 02 - Analyze the Geometric Meaning of the Weight Vector

Part 1 - Why this matters

Analyze the Geometric Meaning of the Weight Vector makes one mathematical component of the perceptron observable. The model's hard output hides score magnitude and geometry, so the underlying vectors, dimensions, boundary, and numerical representation must be studied directly.

A correct formula can still be interpreted incorrectly when weights are unnormalized, features use different scales, or a lower-dimensional plot discards information. This exercise therefore records the assumptions and transformations used with every result.

Part 2 - Mathematical form

$$\mathcal{H} = \{\mathbf{x} : \mathbf{w}^\mathsf{T}\mathbf{x} + b = 0\}, \qquad \mathbf{w} \perp \mathcal{H}.$$

Part 3 - Hints and Helper

Construct direction vectors lying inside a 2D or 3D boundary and verify their dot products with the weight are approximately zero. Normalize only for interpretation.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E02(weight, boundary_directions, tolerance):
    weight_norm = torch.linalg.vector_norm(weight)
    if weight_norm <= tolerance:
        raise ValueError(...)
    dot_products = boundary_directions @ weight
    return {
        "unit_normal": ...,
        "dot_products": dot_products,
        "orthogonal": ...,
    }

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

12.3.4 Exercise 03 - Interpret the Bias Term as a Decision Boundary Shift

Part 1 - Why this matters

Interpret the Bias Term as a Decision Boundary Shift makes one mathematical component of the perceptron observable. The model's hard output hides score magnitude and geometry, so the underlying vectors, dimensions, boundary, and numerical representation must be studied directly.

A correct formula can still be interpreted incorrectly when weights are unnormalized, features use different scales, or a lower-dimensional plot discards information. This exercise therefore records the assumptions and transformations used with every result.

Part 2 - Mathematical form

$$\mathbf{w}^{\mathsf{T}}\mathbf{x} = -b, \qquad d = \frac{|b_2 - b_1|}{\|\mathbf{w}\|_2}.$$

Part 3 - Hints and Helper

Hold the weight fixed and compare multiple biases. Measure perpendicular displacement and verify that the resulting boundaries remain parallel.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E03(weight, biases):
    norm = torch.linalg.vector_norm(weight)
    if norm <= ...:
        raise ValueError(...)
    reference = biases[0]
    return [{
        "bias": bias,
        "signed_shift": ...,
        "distance_from_reference": ...,
    } for bias in biases]

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

12.3.5 Exercise 04 - Compute the Signed Distance from a Point to the Decision Boundary

Part 1 - Why this matters

Compute the Signed Distance from a Point to the Decision Boundary makes one mathematical component of the perceptron observable. The model's hard output hides score magnitude and geometry, so the underlying vectors, dimensions, boundary, and numerical representation must be studied directly.

A correct formula can still be interpreted incorrectly when weights are unnormalized, features use different scales, or a lower-dimensional plot discards information. This exercise therefore records the assumptions and transformations used with every result.

Part 2 - Mathematical form

$$d(\mathbf{x}, \mathcal{H}) = \frac{\mathbf{w}^\top \mathbf{x} + b}{\|\mathbf{w}\|_2}.$$

Part 3 - Hints and Helper

Reject a zero weight vector. Test a point on the boundary and points displaced along the unit normal, then compare distance sign with the predicted class.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E04(points, weight, bias):
    norm = torch.linalg.vector_norm(weight)
    if norm <= ...:
        raise ValueError(...)
    scores = points @ weight + bias
    signed_distance = scores / norm
    return {
        "scores": scores,
        "signed_distance": signed_distance,
        "predictions": ...,
    }

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

12.3.6 Exercise 05 - Verify Perceptron Output Using Manual Vector Calculations

Part 1 - Why this matters

Verify Perceptron Output Using Manual Vector Calculations makes one mathematical component of the perceptron observable. The model's hard output hides score magnitude and geometry, so the underlying vectors, dimensions, boundary, and numerical representation must be studied directly.

A correct formula can still be interpreted incorrectly when weights are unnormalized, features use different scales, or a lower-dimensional plot discards information. This exercise therefore records the assumptions and transformations used with every result.

Part 2 - Mathematical form

$$z_i = \sum_j w_j x_{ij} + b, \qquad \widehat{y}_i = \mathbb{I}[z_i \geq 0].$$

Part 3 - Hints and Helper

Write nested loops for a small batch, calculate every contribution, and compare scores and decisions with vectorized PyTorch.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def manual_score(sample, weight, bias):
    score = float(bias)
    contributions = []
    for feature, coefficient in zip(sample, weight):
        contribution = ...
        contributions.append(contribution)
        score += contribution
    return score, contributions

def E05(features, weight, bias):
    manual = [
        manual_score(row.tolist(),
                     weight.tolist(),
                     bias.item())
        for row in features
    ]
    vectorized = features @ weight + bias
    return manual, vectorized, ...
```

```
def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

12.3.7 Exercise 06 - Compare Multiple Weight Configurations and Their Boundaries

Part 1 - Why this matters

Compare Multiple Weight Configurations and Their Boundaries makes one mathematical component of the perceptron observable. The model's hard output hides score magnitude and geometry, so the underlying vectors, dimensions, boundary, and numerical representation must be studied directly.

A correct formula can still be interpreted incorrectly when weights are unnormalized, features use different scales, or a lower-dimensional plot discards information. This exercise therefore records the assumptions and transformations used with every result.

Part 2 - Mathematical form

$$\mathcal{H}_k = \{\mathbf{x} : {\mathbf{w}^{(k)}}^{\mathsf{T}}\mathbf{x} + b^{(k)} = 0\}.$$

Part 3 - Hints and Helper

Evaluate all configurations on one fixed probe grid. Compare orientation angle, bias shift, and decision disagreement instead of coefficients alone.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E06(configurations, probe_points):
    rows = []
    reference_predictions = None
    for name, state in configurations.items():
        scores = (
            probe_points @ state["weight"]
            + state["bias"]
        )
        predictions = scores >= 0
        if reference_predictions is None:
            reference_predictions = predictions
        rows.append({
            "name": name,
```

```
            "weight_norm": ...,
            "positive_rate": ...,
            "disagreement": ...,
        })
    return rows

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

12.3.8 Exercise 07 - Visualize Decision Hyperplane in 2D Feature Space

Part 1 - Why this matters

Visualize Decision Hyperplane in 2D Feature Space makes one mathematical component of the perceptron observable. The model's hard output hides score magnitude and geometry, so the underlying vectors, dimensions, boundary, and numerical representation must be studied directly.

A correct formula can still be interpreted incorrectly when weights are unnormalized, features use different scales, or a lower-dimensional plot discards information. This exercise therefore records the assumptions and transformations used with every result.

Part 2 - Mathematical form

$$x_2 = -\frac{w_1 x_1 + b}{w_2}, \qquad w_2 \neq 0.$$

Part 3 - Hints and Helper

Plot labeled samples and the boundary. Handle the vertical case when the second weight is near zero instead of dividing by it.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E07(features, targets, weight, bias,
        output_path):
    fig, ax = plt.subplots()
    ax.scatter(
        features[:, 0].cpu(),
        features[:, 1].cpu(),
```

```
        c=targets.cpu(),
    )
    if weight[1].abs() > ...:
        x_values = torch.linspace(..., ..., steps=...)
        y_values = ...
        ax.plot(..., ...)
    else:
        ax.axvline(...)
    fig.savefig(output_path, dpi=150,
                bbox_inches="tight")
    plt.close(fig)

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

12.3.9 Exercise 08 - Extend Decision Boundary Visualization to Higher Dimensions

Part 1 - Why this matters

Extend Decision Boundary Visualization to Higher Dimensions makes one mathematical component of the perceptron observable. The model's hard output hides score magnitude and geometry, so the underlying vectors, dimensions, boundary, and numerical representation must be studied directly.

A correct formula can still be interpreted incorrectly when weights are unnormalized, features use different scales, or a lower-dimensional plot discards information. This exercise therefore records the assumptions and transformations used with every result.

Part 2 - Mathematical form

$$\mathbf{x} = \mathbf{x}_0 + \mathbf{P}\mathbf{u}, \quad (\mathbf{P}^\mathsf{T}\mathbf{w})^\mathsf{T}\mathbf{u} + \mathbf{w}^\mathsf{T}\mathbf{x}_0 + b = 0.$$

Part 3 - Hints and Helper

Visualize an explicit two-dimensional slice or projection. Report the basis, reference point, and visible fraction of weight energy so the plot is not mistaken for the full boundary.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E08(features, weight, bias,
```

```
        projection, reference):
    coordinates = (
        features - reference
    ) @ projection
    projected_weight = projection.T @ weight
    effective_bias = weight @ reference + bias
    visible_ratio = (
        projected_weight.square().sum()
        / weight.square().sum()
    )
    return {
        "coordinates": coordinates,
        "weight": projected_weight,
        "bias": effective_bias,
        "visible_ratio": ...,
    }

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

12.3.10 Exercise 09 - Investigate Numerical Precision Effects on Boundary Placement

Part 1 - Why this matters

Investigate Numerical Precision Effects on Boundary Placement makes one mathematical component of the perceptron observable. The model's hard output hides score magnitude and geometry, so the underlying vectors, dimensions, boundary, and numerical representation must be studied directly.

A correct formula can still be interpreted incorrectly when weights are unnormalized, features use different scales, or a lower-dimensional plot discards information. This exercise therefore records the assumptions and transformations used with every result.

Part 2 - Mathematical form

$$\Delta z_i = z_i^{(32)} - z_i^{(64)}.$$

Part 3 - Hints and Helper

Use points close to and far from the boundary. Compare score error separately from decision flips because tiny rounding differences matter most near zero.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E09(features, weight, bias):
    score32 = (
        features.float() @ weight.float()
        + bias.float()
    )
    score64 = (
        features.double() @ weight.double()
        + bias.double()
    )
    difference = (
        score32.double() - score64
    ).abs()
    return {
        "maximum_score_error": ...,
        "prediction_flips": ...,
        "near_boundary": ...,
    }

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

12.3.11 Exercise 10 - Implement Mathematical Perceptron Using CUDA Tensors

Part 1 - Why this matters

Implement Mathematical Perceptron Using CUDA Tensors makes one mathematical component of the perceptron observable. The model's hard output hides score magnitude and geometry, so the underlying vectors, dimensions, boundary, and numerical representation must be studied directly.

A correct formula can still be interpreted incorrectly when weights are unnormalized, features use different scales, or a lower-dimensional plot discards information. This exercise therefore records the assumptions and transformations used with every result.

Part 2 - Mathematical form

$$\mathbf{z}_{\text{cuda}} = \mathbf{X}_{\text{cuda}}\mathbf{w}_{\text{cuda}} + b_{\text{cuda}}.$$

Part 3 - Hints and Helper

Select CUDA only when available and construct or transfer every operand to one device. Return CPU copies only after the complete calculation.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E10(features, weight, bias):
    device = torch.device(
        "cuda" if torch.cuda.is_available()
        else "cpu"
    )
    features = features.to(device)
    weight = weight.to(device)
    bias = bias.to(device)
    scores = features @ weight + bias
    predictions = (scores >= 0).to(torch.int64)
    return {
        "device": str(device),
        "scores": ...,
        "predictions": ...,
    }

def main():
    print(E10(...))

if __name__ == "__main__":
    main()
```

12.3.12 Exercise 11 - Compare Float32 vs Float64 in Boundary Computation

Part 1 - Why this matters

Compare Float32 vs Float64 in Boundary Computation makes one mathematical component of the perceptron observable. The model's hard output hides score magnitude and geometry, so the underlying vectors, dimensions, boundary, and numerical representation must be studied directly.

A correct formula can still be interpreted incorrectly when weights are unnormalized, features use different scales, or a lower-dimensional plot discards information. This exercise therefore records the assumptions and transformations used with every result.

Part 2 - Mathematical form

$$e_{\text{abs}} = |z_{32} - z_{64}|, \qquad e_{\text{rel}} = \frac{e_{\text{abs}}}{|z_{64}| + \varepsilon}.$$

Part 3 - Hints and Helper

Use identical decimal inputs converted from one high-precision source. Report absolute and relative errors, finite status, and class disagreement.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E11(features, weight, bias, epsilon):
    result32 = compute_score(
        features.float(),
        weight.float(),
        bias.float(),
    )
    result64 = compute_score(
        features.double(),
        weight.double(),
        bias.double(),
    )
    absolute = (result32.double() - result64).abs()
    relative = absolute / (
        result64.abs() + epsilon
    )
    return absolute, relative, ...

def main():
    print(E11(...))

if __name__ == "__main__":
    main()
```

12.3.13 Exercise 12 - Detect Boundary Instability Under Small Input Noise

Part 1 - Why this matters

Detect Boundary Instability Under Small Input Noise makes one mathematical component of the perceptron observable. The model's hard output hides score magnitude and geometry, so the underlying vectors, dimensions, boundary, and numerical representation must be studied directly.

A correct formula can still be interpreted incorrectly when weights are unnormalized, features use different scales, or a lower-dimensional plot discards information. This exercise therefore records the assumptions and transformations used with every result.

Part 2 - Mathematical form

$$\widetilde{\mathbf{x}}_i = \mathbf{x}_i + \boldsymbol{\varepsilon}_i, \qquad r = \frac{1}{N}\sum_i \mathbb{I}[\widehat{y}_i \neq \widetilde{y}_i].$$

Part 3 - Hints and Helper

Repeat seeded perturbations at several noise scales. Relate flip rate to original signed distance and distinguish random variation from systematic sensitivity.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E12(features, weight, bias, scales,
        repetitions, seed):
    generator = torch.Generator(
        device=features.device
    ).manual_seed(seed)
    original = features @ weight + bias
    rows = []
    for scale in scales:
        flip_rates = []
        for _ in range(repetitions):
            noise = torch.randn(
                features.shape,
                generator=generator,
                device=features.device,
            )
            noisy_scores = (
                (features + scale * noise)
                @ weight + bias
            )
            flip_rates.append(...)
        rows.append(...)
    return rows

def main():
    print(E12(...))

if __name__ == "__main__":
    main()
```

12.3.14 Exercise 13 - Analyze Scaling Effects on Weights and Inputs

Part 1 - Why this matters

Analyze Scaling Effects on Weights and Inputs makes one mathematical component of the perceptron observable. The model's hard output hides score magnitude and geometry, so the underlying vectors, dimensions, boundary, and numerical representation must be studied directly.

A correct formula can still be interpreted incorrectly when weights are unnormalized, features use different scales, or a lower-dimensional plot discards information. This exercise therefore records the assumptions and transformations used with every result.

Part 2 - Mathematical form

$$\mathbf{x}' = \mathbf{S}\mathbf{x}, \qquad \mathbf{w}' = \mathbf{S}^{-1}\mathbf{w} \Rightarrow \mathbf{w}'^{\mathsf{T}}\mathbf{x}' = \mathbf{w}^{\mathsf{T}}\mathbf{x}.$$

Part 3 - Hints and Helper

Use positive nonzero feature scales. Compare correct inverse weight transformation with scaling inputs alone, and verify score and prediction preservation.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E13(features, weight, bias, scales):
    if torch.any(scales <= 0):
        raise ValueError(...)
    scaled_features = features * scales
    equivalent_weight = weight / scales
    original_scores = features @ weight + bias
    equivalent_scores = (
        scaled_features @ equivalent_weight + bias
    )
    wrong_scores = scaled_features @ weight + bias
    return {
        "equivalent": ...,
        "wrong_difference": ...,
        "wrong_prediction_flips": ...,
    }

def main():
    print(E13(...))

if __name__ == "__main__":
    main()
```

12.3.15 Exercise 14 - Validate Linear Separability Assumption Experimentally

Part 1 - Why this matters

Validate Linear Separability Assumption Experimentally makes one mathematical component of the perceptron observable. The model's hard output hides score magnitude and geometry, so the underlying vectors, dimensions, boundary, and numerical representation must be studied directly.

A correct formula can still be interpreted incorrectly when weights are unnormalized, features use different scales, or a lower-dimensional plot discards information. This exercise therefore records the assumptions and transformations used with every result.

Part 2 - Mathematical form

$$\gamma_{\min} = \min_i t_i \frac{\mathbf{w}^\top \mathbf{x}_i + b}{\|\mathbf{w}\|_2}.$$

Part 3 - Hints and Helper

Train across multiple orders and seeds, then verify every label-oriented margin. Treat success as evidence for the tested dataset, not a universal proof.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E14(features, signed_targets,
        candidate_weight, candidate_bias):
    norm = torch.linalg.vector_norm(
        candidate_weight
    )
    if norm <= ...:
        raise ValueError(...)
    margins = (
        signed_targets
        * (features @ candidate_weight
           + candidate_bias)
        / norm
    )
    return {
        "all_separated": bool((margins > 0).all()),
        "minimum_margin": ...,
        "violating_indices": ...,
    }
```

```
def main():
    print(E14(...))

if __name__ == "__main__":
    main()
```

12.3.16 Exercise 15 - Demonstrate Failure Case with XOR Dataset

Part 1 - Why this matters

Demonstrate Failure Case with XOR Dataset makes one mathematical component of the perceptron observable. The model's hard output hides score magnitude and geometry, so the underlying vectors, dimensions, boundary, and numerical representation must be studied directly.

A correct formula can still be interpreted incorrectly when weights are unnormalized, features use different scales, or a lower-dimensional plot discards information. This exercise therefore records the assumptions and transformations used with every result.

Part 2 - Mathematical form

$$(0,0) \mapsto 0, \quad (0,1) \mapsto 1, \quad (1,0) \mapsto 1, \quad (1,1) \mapsto 0.$$

Part 3 - Hints and Helper

Train for a finite budget under several sample orders. Plot the four points and final boundary, then record recurring mistakes rather than claiming convergence.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E15(maximum_epochs, learning_rate):
    features = torch.tensor([
        [0.0, 0.0],
        [0.0, 1.0],
        [1.0, 0.0],
        [1.0, 1.0],
    ])
    targets = torch.tensor([0, 1, 1, 0])
    model, history = train_perceptron(
        features, targets,
        maximum_epochs, learning_rate
    )
    return {
        "model": model,
```

```
        "history": history,
        "converged": ...,
    }

def main():
    print(E15(...))

if __name__ == "__main__":
    main()
```

12.3.17 Exercise 16 - Explain Why Single Perceptron Cannot Solve XOR

Part 1 - Why this matters

Explain Why Single Perceptron Cannot Solve XOR makes one mathematical component of the perceptron observable. The model's hard output hides score magnitude and geometry, so the underlying vectors, dimensions, boundary, and numerical representation must be studied directly.

A correct formula can still be interpreted incorrectly when weights are unnormalized, features use different scales, or a lower-dimensional plot discards information. This exercise therefore records the assumptions and transformations used with every result.

Part 2 - Mathematical form

$$b < 0, \quad w_1 + b \geq 0, \quad w_2 + b \geq 0, \quad w_1 + w_2 + b < 0$$

form an inconsistent system for XOR.

Part 3 - Hints and Helper

Derive the four inequalities from the desired labels. Combine the middle inequalities with b less than zero and show the contradiction with the final inequality.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E16(weight_candidates, bias_candidates):
    rows = []
    for weight in weight_candidates:
        for bias in bias_candidates:
            scores = xor_features() @ weight + bias
            predictions = scores >= 0
            rows.append({
                "weight": weight,
```

```
                "bias": bias,
                "correct_count": ...,
                "solves_xor": ...,
            })
    return {
        "search_results": rows,
        "any_solution": ...,
        "inequality_explanation": ...,
    }

def main():
    print(E16(...))

if __name__ == "__main__":
    main()
```

12.3.18 Exercise 17 - Inspect Internal Tensor Operations During Forward Pass

Part 1 - Why this matters

Inspect Internal Tensor Operations During Forward Pass makes one mathematical component of the perceptron observable. The model's hard output hides score magnitude and geometry, so the underlying vectors, dimensions, boundary, and numerical representation must be studied directly.

A correct formula can still be interpreted incorrectly when weights are unnormalized, features use different scales, or a lower-dimensional plot discards information. This exercise therefore records the assumptions and transformations used with every result.

Part 2 - Mathematical form

$$(N, d) \odot (d) \to (N, d) \xrightarrow{\sum_j} (N) \xrightarrow{+b} (N).$$

Part 3 - Hints and Helper

Store shapes, strides, dtypes, devices, products, reductions, and final decisions. Compare explicit element-wise multiplication and sum with matmul.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E17(features, weight, bias):
    products = features * weight
    reduced = products.sum(dim=1)
```

```
    scores = reduced + bias
    matmul_scores = features @ weight + bias
    return {
        "feature_shape": tuple(features.shape),
        "product_shape": tuple(products.shape),
        "score_shape": tuple(scores.shape),
        "products": products,
        "scores": scores,
        "matmul_equal": ...,
        "predictions": ...,
    }

def main():
    print(E17(...))

if __name__ == "__main__":
    main()
```

12.3.19 Exercise 18 - Stress Test Dot Product Computation on GPU

Part 1 - Why this matters

Stress Test Dot Product Computation on GPU makes one mathematical component of the perceptron observable. The model's hard output hides score magnitude and geometry, so the underlying vectors, dimensions, boundary, and numerical representation must be studied directly.

A correct formula can still be interpreted incorrectly when weights are unnormalized, features use different scales, or a lower-dimensional plot discards information. This exercise therefore records the assumptions and transformations used with every result.

Part 2 - Mathematical form

$$\mathbf{z} = \mathbf{X}\mathbf{w}, \qquad \text{cost} = O(Nd).$$

Part 3 - Hints and Helper

Sweep batch and feature sizes, warm up CUDA, synchronize timing, catch out-of-memory safely, and verify a sampled CPU reference.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E18(configurations):
    if not torch.cuda.is_available():
```

```
        return [{"status": "cuda_unavailable"}]
    rows = []
    for batch_size, feature_count in configurations:
        try:
            features = torch.randn(
                batch_size, feature_count,
                device="cuda",
            )
            weight = torch.randn(
                feature_count, device="cuda"
            )
            ...
            torch.cuda.synchronize()
            start = time.perf_counter()
            scores = features @ weight
            torch.cuda.synchronize()
            rows.append({
                "shape": tuple(features.shape),
                "seconds": ...,
                "finite": ...,
            })
        except torch.OutOfMemoryError:
            rows.append({"status": "out_of_memory"})
    return rows

def main():
    print(E18(...))

if __name__ == "__main__":
    main()
```

12.3.20 Exercise 19 - Produce a Formal Mathematical Interpretation Report

Part 1 - Why this matters

Produce a Formal Mathematical Interpretation Report makes one mathematical component of the perceptron observable. The model's hard output hides score magnitude and geometry, so the underlying vectors, dimensions, boundary, and numerical representation must be studied directly.

A correct formula can still be interpreted incorrectly when weights are unnormalized, features use different scales, or a lower-dimensional plot discards information. This exercise therefore records the assumptions and transformations used with every result.

Part 2 - Mathematical form

$$R = \{\text{notation}, \text{dimensions}, \text{geometry}, \text{distance}, \text{precision}, \text{separability}, \text{limitations}\}.$$

Part 3 - Hints and Helper

Combine validated outputs from previous exercises. Preserve label convention, scaling, dtype, projection basis, tolerances, and empirical versus proven claims.

Part 4 - Python code skeleton

```
import time

import matplotlib.pyplot as plt
import torch

def E19(sections):
    required = {
        "notation", "dimensions", "geometry",
        "distance", "precision",
        "separability", "limitations",
    }
    missing = sorted(required - sections.keys())
    warnings = []
    if missing:
        warnings.append(...)
    if sections.get(
        "geometry", {}
    ).get("weight_norm", 0.0) <= 0:
        warnings.append(...)
    return {
        "sections": sections,
        "missing": missing,
        "warnings": warnings,
        "status": ...,
    }

def main():
    print(E19(...))

if __name__ == "__main__":
    main()
```

12.4 PC02 - Perceptron Learning Rule and Updates

Models and Training

12.4.1 Exercise 00 - Derive the Perceptron Weight Update Rule from Classification Errors

Part 1 - Why this matters

The perceptron does not learn by minimizing a smooth loss with ordinary gradient descent. It changes its parameters only when the current decision disagrees with the true class, moving the decision boundary toward a configuration that classifies the mistaken sample correctly. Understanding this mechanism reveals how feedback becomes a concrete parameter update.

Deriving the rule also prevents sign and label errors. The formulas differ depending on whether labels use $\{-1, +1\}$ or $\{0, 1\}$, so the label convention must be fixed before the update is implemented or interpreted.

Part 2 - Mathematical form

For $y_i \in \{-1, +1\}$ and $\hat{y}_i = \text{sign}(\mathbf{w}^\mathsf{T}\mathbf{x}_i + b)$,

$$\mathbf{w} \leftarrow \mathbf{w} + \eta y_i \mathbf{x}_i, \qquad b \leftarrow b + \eta y_i \quad \text{if } y_i(\mathbf{w}^\mathsf{T}\mathbf{x}_i + b) \leq 0.$$

Part 3 - Hints and Helper

Write the correctness condition as a signed margin. Derive the direction needed to increase that margin, then verify that correctly classified samples cause no update.

Part 4 - Python code skeleton

```
import torch

def E00(features, label, weight, bias, learning_rate):
    score = torch.dot(weight, features) + bias
    margin = label * score
    if margin <= 0:
        new_weight = weight + ...
        new_bias = bias + ...
    else:
        new_weight, new_bias = weight, bias
    return new_weight, new_bias, margin

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

12.4.2 Exercise 01 - Implement Manual Weight Update Based on Misclassified Samples

Part 1 - Why this matters

A manual update makes the learning rule visible at the level of individual samples. Instead of delegating parameter changes to an optimizer, the reader can observe which mistake triggered the update and how every feature contributed to the new weight vector.

This visibility is important for debugging. An incorrect label encoding, accidental in-place mutation, or update applied to a correct prediction can produce a training loop that runs normally while learning the wrong boundary.

Part 2 - Mathematical form

$$\Delta \mathbf{w}_i = \begin{cases} \eta y_i \mathbf{x}_i, & y_i z_i \le 0, \\ \mathbf{0}, & y_i z_i > 0, \end{cases} \qquad z_i = \mathbf{w}^\top \mathbf{x}_i + b.$$

Part 3 - Hints and Helper

Clone the original parameters before updating them. Return the prediction, error flag, and update vector so the operation can be verified independently.

Part 4 - Python code skeleton

```
import torch

def E01(features, labels, weight, bias, learning_rate):
    history = []
    for features_i, label_i in zip(features, labels):
        score = torch.dot(weight, features_i) + bias
        mistake = ...
        update = ... if mistake else torch.zeros_like(weight)
        weight = weight + update
        bias = bias + (... if mistake else 0.0)
        history.append((score.item(), mistake, update.clone()))
    return weight, bias, history

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

12.4.3 Exercise 02 - Analyze the Role of Learning Rate in Perceptron Convergence Behavior

Part 1 - Why this matters

The learning rate controls the magnitude of every corrective step. Although positive constant learning rates preserve the classical convergence guarantee for linearly separable data, they change the numerical path, parameter scale, and sensitivity to finite precision.

Comparing several values prevents the simplistic belief that a larger step always learns faster. The number of mistakes, weight norm, boundary position, and stopping epoch should all be measured because a single final accuracy value hides the training dynamics.

Part 2 - Mathematical form

$$\Delta \mathbf{w}_i = \eta y_i \mathbf{x}_i, \qquad \|\Delta \mathbf{w}_i\|_2 = \eta \|\mathbf{x}_i\|_2.$$

Part 3 - Hints and Helper

Use identical initialization and data order for every learning rate. Record mistakes per epoch and parameter norms instead of comparing only the final predictions.

Part 4 - Python code skeleton

```
import torch

def E02(features, labels, learning_rates, epochs):
    results = {}
    for learning_rate in learning_rates:
        weight = torch.zeros(features.shape[1])
        bias = torch.tensor(0.0)
        mistakes = []
        for _ in range(epochs):
            epoch_mistakes = ...
            mistakes.append(epoch_mistakes)
        results[learning_rate] = {
            "mistakes": mistakes,
            "weight_norm": ...,
        }
    return results

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

12.4.4 Exercise 03 - Implement Bias Update Rule and Observe Its Effect on Decision Boundary

Part 1 - Why this matters

The bias allows the separating hyperplane to move away from the origin. Updating weights without updating the bias restricts the family of boundaries the perceptron can learn and may make an otherwise separable dataset appear impossible.

Observing the bias separately clarifies its geometric role. Weight changes can rotate and scale the boundary, while the normalized displacement from the origin is controlled by $-b/\|\mathbf{w}\|_2$.

Part 2 - Mathematical form

$$b \leftarrow b + \eta y_i, \qquad d_{\text{origin}} = -\frac{b}{\|\mathbf{w}\|_2}.$$

Part 3 - Hints and Helper

Train once with the bias frozen and once with it updated. Compare error counts and boundary displacement using the same initial weights and sample order.

Part 4 - Python code skeleton

```
import torch

def E03(features, labels, weight, bias, learning_rate):
    before = bias.clone()
    for features_i, label_i in zip(features, labels):
        score = weight @ features_i + bias
        if label_i * score <= 0:
            weight = weight + learning_rate * label_i * features_i
            bias = ...
    displacement = -bias / torch.linalg.vector_norm(weight)
    return before, bias, displacement

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

12.4.5 Exercise 04 - Track Weight Evolution Across Multiple Training Iterations

Part 1 - Why this matters

Final parameters do not explain how the model arrived at its decision boundary. A trajectory of weight vectors reveals the sequence of mistakes, repeated corrections, quiet periods, and sudden changes that define perceptron training.

Tracking the trajectory also exposes implementation defects. If weights change on correctly classified samples, remain constant after mistakes, or share storage with every history entry, the recorded path will reveal the problem immediately.

Part 2 - Mathematical form

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + \eta y_{i_t} \mathbf{x}_{i_t} \mathbb{I}\left[y_{i_t} z_{i_t}^{(t)} \leq 0\right].$$

Part 3 - Hints and Helper

Clone the weight and bias after each processed sample. Store the epoch, sample index, margin, mistake flag, and parameter state together.

Part 4 - Python code skeleton

```
import torch

def E04(features, labels, epochs, learning_rate):
    weight = torch.zeros(features.shape[1])
    bias = torch.tensor(0.0)
    trajectory = []
    for epoch in range(epochs):
        for sample_id, (features_i, label_i) in enumerate(
            zip(features, labels)
        ):
            margin = label_i * (weight @ features_i + bias)
            mistake = ...
            if mistake:
                weight = ...
                bias = ...
            trajectory.append({"epoch": epoch, "weight": weight.clone(), ...})
    return trajectory

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

12.4.6 Exercise 05 - Compare Single-Sample Updates vs Batch Updates in Perceptron Training

Part 1 - Why this matters

The classical perceptron updates immediately after each mistake, so later samples in the same epoch see the newly corrected boundary. A batch variant accumulates corrections under one frozen parameter state and applies their sum afterward, producing different learning dynamics.

Comparing the two methods teaches that mathematically similar update terms do not imply identical algorithms. Update timing changes which samples are considered mistakes, the trajectory through parameter space, and sometimes the number of epochs needed to separate the data.

Part 2 - Mathematical form

$$\text{online: } \mathbf{w} \leftarrow \mathbf{w} + \eta y_i \mathbf{x}_i, \qquad \text{batch: } \mathbf{w} \leftarrow \mathbf{w} + \eta \sum_{i \in \mathcal{M}} y_i \mathbf{x}_i.$$

Part 3 - Hints and Helper

Start both variants from identical parameters. In the batch version, compute the full mistake mask before modifying either weight or bias.

Part 4 - Python code skeleton

```
import torch

def E05(features, labels, weight, bias, learning_rate, mode):
    weight = weight.clone()
    bias = bias.clone()
    if mode == "online":
        for features_i, label_i in zip(features, labels):
            if label_i * (weight @ features_i + bias) <= 0:
                weight = ...
                bias = ...
    elif mode == "batch":
        margins = labels * (features @ weight + bias)
        mask = margins <= 0
        weight = weight + ...
        bias = bias + ...
    return weight, bias

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

12.4.7 Exercise 06 - Detect and Analyze Oscillation in Non-Separable Datasets

Part 1 - Why this matters

The perceptron convergence theorem applies only when a separating hyperplane exists. On non-separable data, the algorithm may revisit similar parameter states, alternate between conflicting corrections, and continue making mistakes indefinitely.

Detecting this behavior is more useful than merely reaching a maximum epoch limit. A learner should distinguish slow progress from a repeating trajectory and report evidence that the dataset or model assumptions prevent convergence.

Part 2 - Mathematical form

$$\mathbf{s}^{(t)} = (\mathbf{w}^{(t)}, b^{(t)}), \qquad \|\mathbf{s}^{(t)} - \mathbf{s}^{(t-k)}\|_2 < \varepsilon$$

indicates an approximate cycle of period k when errors remain positive.

Part 3 - Hints and Helper

Store detached parameter states and compare recent states within a tolerance. Confirm that a detected repetition occurs while the epoch still contains classification errors.

Part 4 - Python code skeleton

```
import torch

def E06(states, mistake_counts, tolerance):
    cycles = []
    for current in range(1, len(states)):
        for previous in range(current):
            distance = torch.linalg.vector_norm(
                states[current] - states[previous]
            )
            if distance < tolerance and mistake_counts[current] > 0:
                cycles.append({"start": previous, "end": current, ...})
    return cycles

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

12.4.8 Exercise 07 - Implement Training Loop Without Using PyTorch Optimizers

Part 1 - Why this matters

Implementing the full loop manually connects prediction, mistake detection, parameter correction, and stopping behavior in one transparent process. It shows that the perceptron rule is a complete learning algorithm rather than an ordinary differentiable model waiting for an optimizer.

Avoiding automatic gradients also clarifies state management. Parameters should be updated intentionally, histories should contain independent copies, and the loop should terminate only according to an explicit convergence or safety condition.

Part 2 - Mathematical form

$$M_e = \sum_{i=1}^{n} \mathbb{I}\Big[y_i(\mathbf{w}^\mathsf{T}\mathbf{x}_i + b) \le 0\Big], \qquad \text{stop if } M_e = 0.$$

Part 3 - Hints and Helper

Do not call backward or an optimizer. Reset the epoch mistake counter, update only on mistakes, and return both parameters and a training history.

Part 4 - Python code skeleton

```
import torch

def E07(features, labels, learning_rate, max_epochs):
    weight = torch.zeros(features.shape[1], device=features.device)
```

```
    bias = torch.zeros((), device=features.device)
    history = []
    for epoch in range(max_epochs):
        mistakes = 0
        for features_i, label_i in zip(features, labels):
            if label_i * (weight @ features_i + bias) <= 0:
                weight = ...
                bias = ...
                mistakes += 1
        history.append(mistakes)
        if ...:
            break
    return weight, bias, history

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

12.4.9 Exercise 08 - Observe Convergence Speed Under Different Data Orderings

Part 1 - Why this matters

Online perceptron updates depend on the order in which samples are presented. Two orderings of the same separable dataset can generate different intermediate boundaries and different numbers of mistakes before reaching a valid separator.

This experiment distinguishes model capability from optimization path. Repeated trials with controlled random seeds reveal whether a reported convergence speed is stable or merely the consequence of a fortunate ordering.

Part 2 - Mathematical form

$$\pi_r : \{1, \ldots, n\} \to \{1, \ldots, n\}, \qquad T_r = \min\{e : M_e^{(\pi_r)} = 0\}.$$

Part 3 - Hints and Helper

Use torch.randperm with a separate seeded generator for each trial. Preserve the same initialization and report epochs, total mistakes, and whether convergence occurred.

Part 4 - Python code skeleton

```
import torch

def E08(features, labels, seeds, max_epochs):
    results = []
    for seed in seeds:
```

```
        generator = torch.Generator().manual_seed(seed)
        weight = torch.zeros(features.shape[1])
        bias = torch.tensor(0.0)
        total_mistakes = 0
        for epoch in range(max_epochs):
            order = torch.randperm(len(features), generator=generator)
            mistakes = ...
            total_mistakes += mistakes
            if mistakes == 0:
                break
        results.append({"seed": seed, "epochs": ..., "mistakes": ...})
    return results

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

12.4.10 Exercise 09 - Investigate Failure to Converge on XOR-like Distributions

Part 1 - Why this matters

XOR-like data exposes the representational limit of a single linear decision boundary. Training longer or changing the learning rate cannot repair a model whose hypothesis class contains no separator for the target pattern.

The exercise trains the reader to diagnose the correct layer of failure. Persistent mistakes accompanied by rotating or repeating boundaries indicate a structural mismatch, not necessarily broken code or insufficient compute.

Part 2 - Mathematical form

For XOR labels, there is no $(\mathbf{w}, b)$ satisfying

$$y_i(\mathbf{w}^\mathsf{T}\mathbf{x}_i + b) > 0 \qquad \text{for every } i \in \{1, 2, 3, 4\}.$$

Part 3 - Hints and Helper

Train for a fixed maximum number of epochs and retain the best error count. Compare the recurring mistakes with a linearly separable control dataset.

Part 4 - Python code skeleton

```
import torch

def E09(features, labels, learning_rate, max_epochs):
    weight = torch.zeros(features.shape[1])
    bias = torch.tensor(0.0)
```

```
    history = []
    for epoch in range(max_epochs):
        mistakes = 0
        for features_i, label_i in zip(features, labels):
            if label_i * (weight @ features_i + bias) <= 0:
                weight = weight + ...
                bias = bias + ...
                mistakes += 1
        history.append(mistakes)
    return {"converged": ..., "best_errors": ..., "history": history}

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

12.4.11 Exercise 10 - Move Training Computation to CUDA and Verify Deterministic Behavior

Part 1 - Why this matters

Moving training to a GPU is not only a device-placement exercise. Inputs, labels, parameters, random permutations, and temporary tensors must remain on compatible devices while the learning rule preserves the same logical behavior.

Determinism must also be defined carefully. Repeating one CUDA run under controlled settings should reproduce its parameters and mistake history, while exact agreement with CPU may still depend on operation order, dtype, and floating-point rounding near the boundary.

Part 2 - Mathematical form

$$(\mathbf{w}_r, b_r, H_r) = \mathcal{A}_{\mathrm{CUDA}}(X, y, s), \qquad (\mathbf{w}_1, b_1, H_1) = (\mathbf{w}_2, b_2, H_2)$$

for repeated runs using the same seed s and deterministic configuration.

Part 3 - Hints and Helper

Check `torch.cuda.is_available()` before selecting the device. Seed CPU and CUDA, enable deterministic algorithms, and compare complete histories rather than final accuracy alone.

Part 4 - Python code skeleton

```
import torch

def E10(features, labels, seed, epochs):
    if not torch.cuda.is_available():
        raise RuntimeError(...)
    torch.manual_seed(seed)
```

```
    torch.cuda.manual_seed_all(seed)
    torch.use_deterministic_algorithms(True)
    device = torch.device("cuda")
    features = features.to(device)
    labels = labels.to(device)
    weight = torch.zeros(features.shape[1], device=device)
    bias = torch.zeros((), device=device)
    history = ...
    return weight.cpu(), bias.cpu(), history

def main():
    first = E10(...)
    second = E10(...)
    print(...)

if __name__ == "__main__":
    main()
```

12.4.12 Exercise 11 - Compare CPU vs GPU Weight Update Numerical Differences

Part 1 - Why this matters

CPU and GPU implementations can be logically equivalent without producing bitwise-identical floating-point states. Small rounding differences can accumulate across many updates and may alter a hard prediction when a sample lies extremely close to the boundary.

A useful comparison therefore measures parameter deviation, score deviation, and decision disagreement. This separates harmless numerical variation from a device bug that changes the learning rule or data order.

Part 2 - Mathematical form

$$\delta_w = \|\mathbf{w}_{\text{CPU}} - \mathbf{w}_{\text{GPU}}\|_2, \quad \delta_b = |b_{\text{CPU}} - b_{\text{GPU}}|, \quad D = \sum_i \mathbb{I}[\hat{y}_i^C \neq \hat{y}_i^G].$$

Part 3 - Hints and Helper

Use the same dtype, initialization, sample order, and epoch count. Move GPU results to CPU before calling torch.allclose and report both absolute and relative tolerances.

Part 4 - Python code skeleton

```
import torch

def E11(cpu_state, gpu_state, probe_features, atol, rtol):
    cpu_weight, cpu_bias = cpu_state
    gpu_weight, gpu_bias = [value.cpu() for value in gpu_state]
    weight_difference = torch.linalg.vector_norm(cpu_weight - gpu_weight)
```

```
    bias_difference = torch.abs(cpu_bias - gpu_bias)
    cpu_predictions = probe_features @ cpu_weight + cpu_bias >= 0
    gpu_predictions = ...
    return {
        "parameters_close": torch.allclose(...),
        "weight_difference": weight_difference.item(),
        "bias_difference": bias_difference.item(),
        "decision_disagreement": ...,
    }

def main():
    print(E11(...))

if __name__ == "__main__":
    main()
```

12.4.13 Exercise 12 - Measure the Effect of Feature Scaling on Update Magnitude

Part 1 - Why this matters

Each perceptron correction is proportional to the input vector. Features with large numeric scales can dominate the update even when their semantic importance is not greater, bending the training path toward units rather than information.

Measuring update magnitudes before and after standardization demonstrates why preprocessing affects convergence. The experiment should preserve labels and learning rate so the observed difference can be attributed to feature scale.

Part 2 - Mathematical form

$$\|\Delta \mathbf{w}_i\|_2 = \eta \|\mathbf{x}_i\|_2, \qquad x'_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}.$$

Part 3 - Hints and Helper

Compute per-feature standard deviations with a small-value guard. Compare the distribution of correction norms and the number of mistakes under raw and scaled inputs.

Part 4 - Python code skeleton

```
import torch

def E12(features, labels, learning_rate, epsilon=1e-8):
    mean = features.mean(dim=0)
    scale = features.std(dim=0, unbiased=False).clamp_min(epsilon)
    scaled = (features - mean) / scale
    raw_update_norms = []
    scaled_update_norms = []
```

```
    for raw_i, scaled_i, label_i in zip(features, scaled, labels):
        raw_update_norms.append(...)
        scaled_update_norms.append(...)
    return {
        "raw": torch.stack(raw_update_norms),
        "scaled": torch.stack(scaled_update_norms),
    }

def main():
    print(E12(...))

if __name__ == "__main__":
    main()
```

12.4.14 Exercise 13 - Analyze Sensitivity of Updates to Noisy Labels

Part 1 - Why this matters

A mislabeled sample pushes the perceptron in the direction demanded by the incorrect target. Because the update is discrete and mistake-driven, a few contradictory labels can create repeated corrections and destroy the possibility of zero training error.

Sensitivity analysis should measure more than accuracy. Comparing clean and corrupted runs reveals changes in update count, parameter direction, convergence status, and predictions on a clean evaluation set.

Part 2 - Mathematical form

$$\tilde{y}_i = \begin{cases} -y_i, & i \in \mathcal{N}, \\ y_i, & i \notin \mathcal{N}, \end{cases} \qquad \Delta \mathbf{w}_i = \eta \tilde{y}_i \mathbf{x}_i.$$

Part 3 - Hints and Helper

Flip a controlled fraction of labels using saved indices. Train from identical initialization and compare the cosine similarity between the resulting weight vectors.

Part 4 - Python code skeleton

```
import torch

def E13(features, labels, noise_rate, seed):
    generator = torch.Generator().manual_seed(seed)
    noisy_labels = labels.clone()
    count = int(len(labels) * noise_rate)
    noisy_indices = torch.randperm(len(labels), generator=generator)[:count]
    noisy_labels[noisy_indices] *= ...
    clean_state = ...
    noisy_state = ...
```

```
    cosine = torch.nn.functional.cosine_similarity(
        clean_state[0], noisy_state[0], dim=0
    )
    return noisy_indices, cosine, clean_state, noisy_state

def main():
    print(E13(...))

if __name__ == "__main__":
    main()
```

12.4.15 Exercise 14 - Implement Early Stopping Criterion Based on Zero Classification Errors

Part 1 - Why this matters

For a separable training set, an epoch with zero classification errors means every sample is currently on the correct side of the boundary. Continuing to process identical data would waste computation because the perceptron performs no update on correct samples.

The stopping condition must be evaluated over a complete epoch. Stopping after one correct sample or after a short error-free interval can terminate training before the model has been tested against the remaining observations.

Part 2 - Mathematical form

$$M_e = \sum_{i=1}^{n} \mathbb{I}[y_i z_i \leq 0], \qquad e_{\text{stop}} = \min\{e : M_e = 0\}.$$

Part 3 - Hints and Helper

Reset mistakes at the start of each epoch and check it only after all samples have been processed. Retain `max_epochs` as protection for non-separable data.

Part 4 - Python code skeleton

```
import torch

def E14(features, labels, learning_rate, max_epochs):
    weight = torch.zeros(features.shape[1])
    bias = torch.tensor(0.0)
    history = []
    converged = False
    for epoch in range(max_epochs):
        mistakes = 0
        for features_i, label_i in zip(features, labels):
            if label_i * (weight @ features_i + bias) <= 0:
                weight = ...
                bias = ...
```

```
                mistakes += 1
        history.append(mistakes)
        if mistakes == ...:
            converged = True
            break
    return weight, bias, converged, epoch + 1, history

def main():
    print(E14(...))

if __name__ == "__main__":
    main()
```

12.4.16 Exercise 15 - Log and Visualize Weight Trajectories During Training

Part 1 - Why this matters

A visual trajectory turns a sequence of parameter updates into an interpretable learning path. In two-weight models, points and arrows can reveal direct movement, repeated loops, large jumps, and stabilization near a separating solution.

The log remains essential even when a plot is produced. Epoch numbers, triggering samples, mistake flags, and bias values provide the context needed to explain why each segment of the trajectory exists.

Part 2 - Mathematical form

$$\mathcal{T} = \{(w_1^{(t)}, w_2^{(t)}, b^{(t)}, i_t, e_t)\}_{t=0}^{T}, \qquad \Delta\mathbf{w}^{(t)} = \mathbf{w}^{(t+1)} - \mathbf{w}^{(t)}.$$

Part 3 - Hints and Helper

Detach and move logged tensors to CPU before plotting. Use a scatter plot for states and arrows or connected lines for update order.

Part 4 - Python code skeleton

```
import matplotlib.pyplot as plt
import torch

def E15(weight_history):
    trajectory = torch.stack(weight_history).detach().cpu()
    if trajectory.shape[1] != 2:
        raise ValueError(...)
    figure, axis = plt.subplots()
    axis.plot(trajectory[:, 0], trajectory[:, 1], marker="o")
    for index in range(len(trajectory) - 1):
        delta = trajectory[index + 1] - trajectory[index]
        axis.arrow(
```

```
            trajectory[index, 0], trajectory[index, 1],
            delta[0], delta[1], ...
        )
    return figure

def main():
    E15(...)
    plt.show()

if __name__ == "__main__":
    main()
```

12.4.17 Exercise 16 - Demonstrate Catastrophic Update with Large Learning Rate

Part 1 - Why this matters

A very large learning rate can produce parameter jumps that are enormous relative to the data scale. The perceptron may still classify separable training data eventually, but the intermediate scores, parameter norms, and numerical stability can become difficult to control.

This experiment defines catastrophe through measurable symptoms rather than a large number alone. Non-finite values, explosive norm growth, severe boundary jumps, or unstable decisions near the margin are stronger evidence than the learning rate by itself.

Part 2 - Mathematical form

$$\|\mathbf{w}^{(t+1)} - \mathbf{w}^{(t)}\|_2 = \eta\|\mathbf{x}_{i_t}\|_2, \qquad \rho_t = \frac{\|\Delta\mathbf{w}^{(t)}\|_2}{\|\mathbf{w}^{(t)}\|_2 + \varepsilon}.$$

Part 3 - Hints and Helper

Compare a baseline and an extreme learning rate under identical data order. Monitor finiteness, update ratio, maximum score magnitude, and mistake history.

Part 4 - Python code skeleton

```
import torch

def E16(features, labels, weight, bias, learning_rate, threshold):
    events = []
    for features_i, label_i in zip(features, labels):
        score = weight @ features_i + bias
        if label_i * score <= 0:
            update = learning_rate * label_i * features_i
            ratio = torch.linalg.vector_norm(update) / (
                torch.linalg.vector_norm(weight) + 1e-12
            )
            weight = weight + update
```

```
            bias = bias + learning_rate * label_i
            events.append({"ratio": ratio.item(), "catastrophic": ...})
    return weight, bias, events, torch.isfinite(weight).all()

def main():
    print(E16(...))

if __name__ == "__main__":
    main()
```

12.4.18 Exercise 17 - Validate Correctness of Update Rule Using Manual Calculations

Part 1 - Why this matters

A small hand-calculated example is one of the strongest tests of a learning-rule implementation. It verifies the score, margin, mistake decision, weight correction, and bias correction without relying on another training library as the reference.

The validation should include both a mistaken sample and a correct sample. This proves not only that the code updates in the right direction, but also that it leaves parameters unchanged when no correction is required.

Part 2 - Mathematical form

For $\mathbf{x} = (2, -1)$, $y = -1$, $\mathbf{w} = (1, 1)$, $b = 0$, and $\eta = 0.5$,

$$z = 1, \qquad \mathbf{w}' = (1, 1) + 0.5(-1)(2, -1) = (0, 1.5), \qquad b' = -0.5.$$

Part 3 - Hints and Helper

Write the expected values explicitly before running the function. Compare tensors with `torch.testing.assert_close()` and verify the mistake flag.

Part 4 - Python code skeleton

```
import torch

def E17(features, label, weight, bias, learning_rate):
    score = torch.dot(weight, features) + bias
    mistake = label * score <= 0
    if mistake:
        weight = weight + ...
        bias = bias + ...
    return score, mistake, weight, bias

def main():
    actual = E17(...)
    expected_weight = torch.tensor([0.0, 1.5])
    expected_bias = torch.tensor(-0.5)
```

```
    torch.testing.assert_close(actual[2], expected_weight)
    torch.testing.assert_close(actual[3], expected_bias)
    print(actual)

if __name__ == "__main__":
    main()
```

12.4.19 Exercise 18 - Stress Test Update Rule with High-Dimensional Feature Space

Part 1 - Why this matters

High-dimensional inputs test whether the implementation is genuinely vectorized and shape-safe. They also reveal the memory and transfer costs hidden by tiny demonstrations, especially when histories accidentally store a full weight copy after every sample.

A meaningful stress test separates correctness from performance. It verifies finite parameters and valid predictions while reporting elapsed time, update count, memory use, and throughput for a controlled workload.

Part 2 - Mathematical form

$$X \in \mathbb{R}^{n \times d}, \quad \mathbf{w} \in \mathbb{R}^{d}, \qquad \text{work per epoch} = \mathcal{O}(nd), \qquad \text{parameter memory} = \mathcal{O}(d).$$

Part 3 - Hints and Helper

Generate data from a known separating weight vector. Synchronize CUDA around timing and store aggregate statistics instead of every high-dimensional state.

Part 4 - Python code skeleton

```
import time

import torch

def E18(samples, dimensions, epochs, device):
    features = torch.randn(samples, dimensions, device=device)
    teacher = torch.randn(dimensions, device=device)
    labels = torch.where(features @ teacher >= 0, 1.0, -1.0)
    weight = torch.zeros(dimensions, device=device)
    bias = torch.zeros((), device=device)
    if device.type == "cuda":
        torch.cuda.synchronize()
    start = time.perf_counter()
    update_count = ...
    for _ in range(epochs):
        ...
    if device.type == "cuda":
```

```
        torch.cuda.synchronize()
    elapsed = time.perf_counter() - start
    return {"elapsed": elapsed, "updates": update_count, "finite": ...}

def main():
    print(E18(...))

if __name__ == "__main__":
    main()
```

12.4.20 Exercise 19 - Produce a Detailed Failure Analysis of the Learning Rule

Part 1 - Why this matters

When perceptron training fails, the cause may lie in the data, representation, numerical environment, update implementation, or stopping logic. A structured failure analysis prevents repeated parameter tuning when the real problem is non-separability, label noise, shape corruption, or device inconsistency.

The report should connect evidence to conclusions. Mistake histories, cycle detections, parameter norms, scaling statistics, manual checks, and CPU–GPU comparisons together provide a reproducible explanation rather than a vague statement that the model did not learn.

Part 2 - Mathematical form

$$R = \{M_e, \|\mathbf{w}^{(e)}\|_2, \|\Delta\mathbf{w}\|_2, \text{cycles}, \text{label noise}, \text{scaling}, \text{numerics}, \text{separability}\}.$$

Part 3 - Hints and Helper

Require evidence for each diagnosis and distinguish observations from proven causes. Include the label convention, device, dtype, seed, sample order, and stopping criterion.

Part 4 - Python code skeleton

```
import torch

def E19(evidence):
    required = {
        "mistake_history", "weight_norms", "update_norms",
        "label_check", "scaling_check", "separability_check",
        "device", "dtype", "seed", "stopping_rule",
    }
    missing = sorted(required - evidence.keys())
    diagnoses = []
    if evidence.get("cycle_detected"):
        diagnoses.append(...)
    if evidence.get("non_finite_parameters"):
        diagnoses.append(...)
```

```
    if evidence.get("separability_check") is False:
        diagnoses.append(...)
    return {
        "complete": not missing,
        "missing": missing,
        "diagnoses": diagnoses,
        "evidence": evidence,
    }

def main():
    print(E19(...))

if __name__ == "__main__":
    main()
```

12.5 PC03 - Training Dynamics and Convergence Theory

Models and Training

12.5.1 Exercise 00 - Define Formal Convergence Criteria for the Perceptron Algorithm

Part 1 - Why this matters

A training loop needs a precise definition of convergence. For the perceptron, convergence means that a complete pass over the training set produces no classification errors and therefore no parameter updates.

This criterion must be separated from a maximum epoch limit, a small parameter change, or high accuracy. Those conditions may stop computation, but they do not prove that the current hyperplane separates every training sample.

Part 2 - Mathematical form

$$M_e = \sum_{i=1}^{n} \mathbb{I}[y_i(\mathbf{w}^\mathsf{T}\mathbf{x}_i + b) \le 0], \qquad \text{converged} \iff M_e = 0.$$

Part 3 - Hints and Helper

Evaluate the criterion only after a complete epoch. Return separate flags for proven training convergence and termination caused by the safety limit.

Part 4 - Python code skeleton

```
import torch

def E00(features, labels, weight, bias):
```

```
    margins = labels * (features @ weight + bias)
    mistakes = ...
    minimum_margin = ...
    converged = mistakes == 0
    return converged, mistakes, minimum_margin

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

12.5.2 Exercise 01 - Experimentally Verify the Perceptron Convergence Theorem

Part 1 - Why this matters

The convergence theorem predicts that the perceptron makes finitely many mistakes when the data are linearly separable and have bounded norm. An experiment can connect this abstract statement to an observable training history.

Empirical convergence does not prove the theorem by itself. The experiment must verify its assumptions, record the number of updates, and compare the observation with the theoretical mistake bound.

Part 2 - Mathematical form

$$K \leq \left(\frac{R}{\gamma}\right)^2, \qquad R = \max_i \|\mathbf{x}_i\|_2, \quad \gamma = \min_i y_i \mathbf{u}^{\mathsf{T}} \mathbf{x}_i.$$

Part 3 - Hints and Helper

Generate data with a known unit separator. Compute its margin and radius before training, then check whether the observed update count respects the bound.

Part 4 - Python code skeleton

```
import torch

def E01(features, labels, separator, max_epochs):
    radius = torch.linalg.vector_norm(features, dim=1).max()
    margin = (labels * (features @ separator)).min()
    theoretical_bound = ...
    state, updates, converged = ...
    return {
        "converged": converged,
        "updates": updates,
        "bound": theoretical_bound.item(),
        "within_bound": ...,
```

```
    }

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

12.5.3 Exercise 02 - Analyze the Effect of Margin Size on Training Stability

Part 1 - Why this matters

The geometric margin measures how far the closest labeled sample lies from the separating boundary. A larger margin leaves more room for perturbations before a prediction changes and usually produces a smaller theoretical mistake bound.

Margin experiments must normalize the separating weight vector. Otherwise multiplying the parameters changes the numeric score while leaving the same boundary, creating a false impression that the geometric margin improved.

Part 2 - Mathematical form

$$\gamma = \min_i \frac{y_i(\mathbf{w}^\mathsf{T}\mathbf{x}_i + b)}{\|\mathbf{w}\|_2}, \qquad |\delta_i| < \gamma \Rightarrow \text{label stability along the normal direction.}$$

Part 3 - Hints and Helper

Create datasets with controlled separation while keeping radius and sample count comparable. Measure update counts and prediction changes after equal perturbations.

Part 4 - Python code skeleton

```
import torch

def E02(datasets, perturbation):
    results = {}
    for name, (features, labels) in datasets.items():
        weight, bias, history = ...
        norm = torch.linalg.vector_norm(weight)
        margin = (labels * (features @ weight + bias) / norm).min()
        shifted = features + perturbation
        disagreement = ...
        results[name] = {"margin": margin.item(), "disagreement": ...}
    return results

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

12.5.4 Exercise 03 - Measure Number of Updates Until Convergence

Part 1 - Why this matters

The number of updates is the perceptron's natural measure of learning effort because correct samples do not modify the parameters. Counting epochs alone can be misleading when dataset sizes or sample orderings differ.

A correct measurement distinguishes processed samples, mistakes, updates, and epochs. These counts answer different questions and together describe how efficiently the algorithm found a separator.

Part 2 - Mathematical form

$$U_T = \sum_{t=1}^{T} \mathbb{I}[y_{i_t} z_{i_t} \leq 0], \qquad e_* = \min\{e : M_e = 0\}.$$

Part 3 - Hints and Helper

Increment the update counter exactly where the parameters change. Keep a per-epoch history and verify that the final epoch contributes zero updates.

Part 4 - Python code skeleton

```
import torch

def E03(features, labels, learning_rate, max_epochs):
    weight = torch.zeros(features.shape[1])
    bias = torch.tensor(0.0)
    total_updates = 0
    history = []
    for epoch in range(max_epochs):
        updates = ...
        for features_i, label_i in zip(features, labels):
            if label_i * (weight @ features_i + bias) <= 0:
                weight, bias = ...
                updates += 1
        total_updates += updates
        history.append(updates)
        if updates == 0:
            break
    return total_updates, epoch + 1, history

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

12.5.5 Exercise 04 - Study the Impact of Data Ordering on Convergence Speed

Part 1 - Why this matters

Online updates make the optimization path dependent on sample order. Different permutations of the same separable dataset can produce different intermediate boundaries and different numbers of mistakes before convergence.

One ordering is not sufficient evidence. Repeated seeded permutations allow convergence speed to be summarized as a distribution and reveal unusually easy or difficult presentations of the data.

Part 2 - Mathematical form

$$U(\pi) = \sum_{t} \mathbb{I}[y_{\pi(t)} z_{\pi(t)} \leq 0], \qquad \bar{U} = \frac{1}{q} \sum_{r=1}^{q} U(\pi_r).$$

Part 3 - Hints and Helper

Reset the parameters for every trial and use a separate seeded generator. Report minimum, mean, maximum, and standard deviation of update counts.

Part 4 - Python code skeleton

```
import torch

def E04(features, labels, seeds, max_epochs):
    update_counts = []
    for seed in seeds:
        generator = torch.Generator().manual_seed(seed)
        weight = torch.zeros(features.shape[1])
        bias = torch.tensor(0.0)
        updates = 0
        for _ in range(max_epochs):
            order = torch.randperm(len(features), generator=generator)
            epoch_updates = ...
            updates += epoch_updates
            if epoch_updates == 0:
                break
        update_counts.append(updates)
    values = torch.tensor(update_counts, dtype=torch.float64)
    return {"minimum": ..., "mean": ..., "maximum": ..., "std": ...}

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

12.5.6 Exercise 05 - Investigate Training Behavior on Nearly Separable Data

Part 1 - Why this matters

Nearly separable data contain samples with extremely small margins or a few points that conflict with an otherwise clear boundary. Such datasets can cause many corrections and make convergence fragile under tiny perturbations.

The analysis should not assume that a low error rate implies convergence. It must inspect the minimum margin, persistent mistake identities, trajectory length, and sensitivity to noise.

Part 2 - Mathematical form

$$\gamma_{\min} = \min_i \frac{y_i(\mathbf{w}^\mathsf{T}\mathbf{x}_i + b)}{\|\mathbf{w}\|_2}, \qquad \gamma_{\min} \approx 0.$$

Part 3 - Hints and Helper

Move selected points progressively toward the boundary. For each distance, record convergence, update count, and the indices that are repeatedly misclassified.

Part 4 - Python code skeleton

```
import torch

def E05(dataset_versions, max_epochs):
    report = {}
    for distance, (features, labels) in dataset_versions.items():
        weight, bias, history, mistake_ids = ...
        norm = torch.linalg.vector_norm(weight).clamp_min(1e-12)
        minimum_margin = (labels * (features @ weight + bias) / norm).min()
        report[distance] = {
            "converged": history[-1] == 0,
            "updates": sum(history),
            "minimum_margin": minimum_margin.item(),
            "persistent_samples": ...,
        }
    return report

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

12.5.7 Exercise 06 - Demonstrate Divergence on Non-Separable Datasets

Part 1 - Why this matters

When no separating hyperplane exists, the convergence theorem does not apply. The perceptron continues correcting contradictory samples, and the mistake count never reaches zero under ordinary training.

Divergence here does not necessarily mean that parameter magnitudes approach infinity. It can appear as endless updates, cycles, or irregular movement, so the report must define and measure the observed behavior precisely.

Part 2 - Mathematical form

$$\nexists(\mathbf{w}, b) \text{ such that } y_i(\mathbf{w}^\mathsf{T}\mathbf{x}_i + b) > 0 \ \forall i \quad \Longrightarrow \quad M_e \neq 0 \text{ may persist.}$$

Part 3 - Hints and Helper

Use a known non-separable dataset and a separable control. Track positive mistake counts, repeated states, and failure to meet the formal convergence criterion.

Part 4 - Python code skeleton

```
import torch

def E06(features, labels, epochs):
    states = []
    mistake_history = []
    weight = torch.zeros(features.shape[1])
    bias = torch.tensor(0.0)
    for _ in range(epochs):
        mistakes = ...
        state = torch.cat((weight, bias.reshape(1))).clone()
        states.append(state)
        mistake_history.append(mistakes)
    cycle = ...
    return {
        "converged": 0 in mistake_history,
        "cycle": cycle,
        "mistakes": mistake_history,
    }

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

12.5.8 Exercise 07 - Compare Deterministic vs Randomized Sample Ordering

Part 1 - Why this matters

A fixed order produces a reproducible trajectory but can repeatedly expose the model to an unfavorable pattern. Randomized ordering changes the correction sequence and may reduce or increase the number of updates in a particular run.

The comparison should evaluate both efficiency and variability. Deterministic training provides one path, while randomized training requires several seeded trials and a distribution of outcomes.

Part 2 - Mathematical form

$$U_{\text{fixed}} = U(\pi_0), \qquad \{U_r\}_{r=1}^{q} = \{U(\pi_r)\}_{r=1}^{q}.$$

Part 3 - Hints and Helper

Use identical initialization and stopping rules. Report convergence rate, update statistics, and the reproducibility of each seeded randomized trial.

Part 4 - Python code skeleton

```
import torch

def E07(features, labels, seeds, max_epochs):
    fixed_result = ...
    randomized_results = []
    for seed in seeds:
        randomized_results.append(...)
    updates = torch.tensor([item["updates"] for item in randomized_results])
    return {
        "fixed": fixed_result,
        "random_mean": updates.float().mean().item(),
        "random_range": (..., ...),
        "convergence_rate": ...,
    }

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

12.5.9 Exercise 08 - Observe Weight Norm Growth During Training

Part 1 - Why this matters

The perceptron adds a scaled training vector after every mistake, so the weight norm evolves with the correction history. Growth can reflect progress toward a separator, repeated conflict, large feature scales, or an excessive learning rate.

Norm growth alone is not convergence. It should be interpreted beside the signed margins, mistake counts, and normalized boundary because multiplying all parameters can increase scores without changing classifications.

Part 2 - Mathematical form

$$\|\mathbf{w}^{(t+1)}\|_2^2 = \|\mathbf{w}^{(t)}\|_2^2 + 2\eta y_{i_t} {\mathbf{w}^{(t)}}^{\mathsf{T}} \mathbf{x}_{i_t} + \eta^2 \|\mathbf{x}_{i_t}\|_2^2.$$

Part 3 - Hints and Helper

Record the norm before and after each update. Plot it with mistake counts and inspect whether growth stabilizes, repeats, or becomes non-finite.

Part 4 - Python code skeleton

```
import torch

def E08(features, labels, learning_rate, epochs):
    weight = torch.zeros(features.shape[1])
    bias = torch.tensor(0.0)
    norms = [torch.linalg.vector_norm(weight).item()]
    margins = []
    for _ in range(epochs):
        for features_i, label_i in zip(features, labels):
            margin = label_i * (weight @ features_i + bias)
            if margin <= 0:
                weight = weight + ...
                bias = bias + ...
                norms.append(...)
                margins.append(margin.item())
    return norms, margins

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

12.5.10 Exercise 09 - Analyze the Relationship Between Margin and Generalization

Part 1 - Why this matters

A separator that classifies training samples correctly can still behave poorly on unseen data. Among valid separators, a larger normalized margin often indicates greater robustness to small perturbations and can support better generalization.

This is a relationship to investigate, not a guarantee that margin alone determines test performance. Data distribution, sample size, noise, and model mismatch must remain controlled when comparing runs.

Part 2 - Mathematical form

$$\hat{\gamma} = \min_{i \in \text{train}} \frac{y_i(\mathbf{w}^\mathsf{T}\mathbf{x}_i + b)}{\|\mathbf{w}\|_2}, \qquad E_{\text{test}} = \frac{1}{m}\sum_{j=1}^{m} \mathbb{I}[\hat{y}_j \neq y_j].$$

Part 3 - Hints and Helper

Train many seeded models, then pair each training margin with test error. Plot the pairs and compute correlation without claiming causation from one experiment.

Part 4 - Python code skeleton

```
import torch

def E09(train_runs, test_features, test_labels):
    rows = []
    for weight, bias in train_runs:
        norm = torch.linalg.vector_norm(weight).clamp_min(1e-12)
        train_margin = ...
        test_scores = test_features @ weight + bias
        test_predictions = torch.where(test_scores >= 0, 1, -1)
        test_error = ...
        rows.append((train_margin.item(), test_error.item()))
    values = torch.tensor(rows)
    correlation = torch.corrcoef(values.T)[0, 1]
    return rows, correlation.item()

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

12.5.11 Exercise 10 - Run Long-Horizon Training on CUDA for Stability Observation

Part 1 - Why this matters

Long GPU runs reveal problems that short demonstrations may hide, including gradual norm growth, non-finite values, memory accumulation, and unnecessary updates after the best state has already appeared.

A stability run should use periodic summaries rather than storing every tensor state. It must synchronize around timing, monitor device memory, and preserve enough checkpoints to identify when instability began.

Part 2 - Mathematical form

$$S_t = \{M_t, \|\mathbf{w}^{(t)}\|_2, \max_i |z_i^{(t)}|, \text{finite}(\mathbf{w}^{(t)}, b^{(t)})\}.$$

Part 3 - Hints and Helper

Check CUDA availability and log at fixed intervals. Stop safely on non-finite parameters while keeping the last valid checkpoint and diagnostic record.

Part 4 - Python code skeleton

```
import torch

def E10(features, labels, epochs, log_interval):
    if not torch.cuda.is_available():
        raise RuntimeError(...)
    device = torch.device("cuda")
    features, labels = features.to(device), labels.to(device)
    weight = torch.zeros(features.shape[1], device=device)
    bias = torch.zeros((), device=device)
    summaries = []
    for epoch in range(epochs):
        mistakes = ...
        if epoch % log_interval == 0:
            summaries.append({
                "epoch": epoch,
                "norm": ...,
                "mistakes": mistakes,
            })
        if not torch.isfinite(weight).all():
            break
    return weight.cpu(), bias.cpu(), summaries

def main():
    print(E10(...))

if __name__ == "__main__":
    main()
```

12.5.12 Exercise 11 - Compare Float Precision Effects on Convergence

Part 1 - Why this matters

Floating-point precision affects dot products, margins, and the exact moment a hard decision changes sign. Differences are most visible when samples lie close to the boundary or training continues for many updates.

The comparison must use the same original data values, initialization, and order. Converting separately generated datasets would mix precision effects with different random samples.

Part 2 - Mathematical form

$$\delta_t = \|\mathbf{w}_{32}^{(t)} - \mathbf{w}_{64}^{(t)}\|_2, \qquad D_t = \sum_i \mathbb{I}[\text{sign}(z_{i,32}^{(t)}) \neq \text{sign}(z_{i,64}^{(t)})].$$

Part 3 - Hints and Helper

Generate the reference data once in float64 and cast a copy to float32. Compare update histories, convergence epoch, parameters, and prediction disagreement.

Part 4 - Python code skeleton

```
import torch

def E11(features64, labels, epochs):
    results = {}
    for dtype in (torch.float32, torch.float64):
        features = features64.to(dtype)
        weight, bias, history = ...
        results[str(dtype)] = (
            weight.to(torch.float64),
            bias.to(torch.float64),
            history,
        )
    difference = (
        results["torch.float32"][0]
        - results["torch.float64"][0]
    )
    weight_difference = torch.linalg.vector_norm(difference)
    return results, weight_difference.item()

def main():
    print(E11(...))

if __name__ == "__main__":
    main()
```

12.5.13 Exercise 12 - Detect Numerical Drift in Extended Training Runs

Part 1 - Why this matters

Numerical drift is the gradual separation of states that began from equivalent conditions but used different precision, devices, or operation order. Small early differences can alter later mistake decisions near a hard threshold.

Detection requires aligned checkpoints. Comparing only final weights cannot show when the trajectories separated or whether the drift remained harmless until a decision disagreement occurred.

Part 2 - Mathematical form

$$d_t = \frac{\|\mathbf{w}_A^{(t)} - \mathbf{w}_B^{(t)}\|_2}{\|\mathbf{w}_B^{(t)}\|_2 + \varepsilon}, \qquad t_* = \min\{t : d_t > \tau\}.$$

Part 3 - Hints and Helper

Save states at identical update counts. Compute absolute and relative deviations, then locate the first tolerance breach and first prediction disagreement.

Part 4 - Python code skeleton

```
import torch

def E12(checkpoints_a, checkpoints_b, tolerance):
    drift = []
    first_breach = None
    paired_states = zip(checkpoints_a, checkpoints_b)
    for step, (state_a, state_b) in enumerate(paired_states):
        absolute = torch.linalg.vector_norm(state_a - state_b)
        relative = absolute / (torch.linalg.vector_norm(state_b) + 1e-12)
        drift.append(relative.item())
        if first_breach is None and relative > tolerance:
            first_breach = step
    return drift, first_breach

def main():
    print(E12(...))

if __name__ == "__main__":
    main()
```

12.5.14 Exercise 13 - Visualize Convergence Trajectories in Parameter Space

Part 1 - Why this matters

A parameter-space trajectory shows how mistake-driven updates move the model from initialization toward a separating solution. Direct paths, zigzags, repeated cycles, and large jumps become visible in a way that final parameters cannot provide.

Visualization is exact only for low-dimensional parameter vectors. Higher-dimensional states require a declared projection, and the plot must report how much variation the chosen axes preserve.

Part 2 - Mathematical form

$$\mathbf{s}^{(t)} = (\mathbf{w}^{(t)}, b^{(t)}), \qquad \mathbf{q}^{(t)} = P^{\mathsf{T}}(\mathbf{s}^{(t)} - \bar{\mathbf{s}}) \in \mathbb{R}^2.$$

Part 3 - Hints and Helper

Stack detached states, use direct coordinates for two dimensions or PCA for larger states, and color points by epoch or mistake count.

Part 4 - Python code skeleton

```
import matplotlib.pyplot as plt
import torch

def E13(states):
    matrix = torch.stack(states).detach().cpu()
    centered = matrix - matrix.mean(dim=0)
    _, singular_values, vectors = torch.linalg.svd(
        centered, full_matrices=False
    )
    coordinates = centered @ vectors[:2].T
    visible_variance = singular_values[:2].square().sum()
    total_variance = singular_values.square().sum()
    explained = visible_variance / total_variance
    figure, axis = plt.subplots()
    axis.plot(coordinates[:, 0], coordinates[:, 1], marker="o")
    return figure, explained.item()

def main():
    print(E13(...))

if __name__ == "__main__":
    main()
```

12.5.15 Exercise 14 - Investigate Overfitting Behavior in Linearly Separable Data

Part 1 - Why this matters

A perceptron can reach zero training error while relying on a boundary that is sensitive to sampling noise or unrepresentative examples. Perfect separation of the observed data therefore does not establish reliable performance on new samples.

The experiment should track train and validation behavior across epochs. It must also recognize that the standard perceptron stops updating after zero training mistakes, so overfitting is studied through data selection, noisy labels, ordering, or alternative retained states.

Part 2 - Mathematical form

$$E_{\text{train}}^{(e)} = 0, \qquad E_{\text{val}}^{(e)} > 0, \qquad G^{(e)} = E_{\text{val}}^{(e)} - E_{\text{train}}^{(e)}.$$

Part 3 - Hints and Helper

Evaluate both sets after every epoch and retain the state with minimum validation error. Compare it with the first zero-training-error state.

Part 4 - Python code skeleton

```
import torch

def E14(train_data, validation_data, epochs):
    train_features, train_labels = train_data
    validation_features, validation_labels = validation_data
    history = []
    best_state = None
    best_validation_error = float("inf")
    for epoch in range(epochs):
        weight, bias = ...
        train_error = ...
        validation_error = ...
        history.append((train_error, validation_error))
        if validation_error < best_validation_error:
            best_validation_error = validation_error
            best_state = (weight.clone(), bias.clone())
    return history, best_state

def main():
    print(E14(...))

if __name__ == "__main__":
    main()
```

12.5.16 Exercise 15 - Analyze Training Failure Modes and Their Root Causes

Part 1 - Why this matters

Perceptron failure can originate from non-separable data, label noise, incorrect label encoding, feature-scale imbalance, numerical overflow, device mismatch, or faulty stopping logic. These causes require different remedies.

A root-cause analysis must connect symptoms to evidence. Persistent mistakes alone are insufficient because the same symptom may arise from a mathematical limitation or an implementation defect.

Part 2 - Mathematical form

$$F = \{\text{assumptions}, \text{data}, \text{updates}, \text{numerics}, \text{devices}, \text{termination}\}.$$

Part 3 - Hints and Helper

Build diagnostic checks in a fixed order: shapes and labels, finiteness, separability evidence, update correctness, trajectory behavior, and stopping conditions.

Part 4 - Python code skeleton

```
import torch

def E15(features, labels, training_log):
    diagnostics = {}
    diagnostics["shape_valid"] = ...
    allowed_labels = torch.tensor([-1, 1], device=labels.device)
    diagnostics["labels_valid"] = torch.isin(
        labels, allowed_labels
    ).all().item()
    diagnostics["finite"] = torch.isfinite(features).all().item()
    diagnostics["persistent_errors"] = ...
    diagnostics["cycle_detected"] = ...
    causes = []
    if not diagnostics["labels_valid"]:
        causes.append(...)
    if diagnostics["cycle_detected"]:
        causes.append(...)
    return diagnostics, causes

def main():
    print(E15(...))

if __name__ == "__main__":
    main()
```

12.5.17 Exercise 16 - Compare Convergence Between CPU and GPU Implementations

Part 1 - Why this matters

CPU and GPU versions should implement the same mistake condition and update rule. Their trajectories may still differ slightly because floating-point reductions and threshold decisions can respond to rounding near zero.

A strong comparison aligns sample order and checkpoints, then measures both numerical distance and behavioral agreement. Final accuracy alone can hide diverging parameter paths.

Part 2 - Mathematical form

$$\delta_w = \|\mathbf{w}_C - \mathbf{w}_G\|_2, \qquad \delta_U = |U_C - U_G|, \qquad D = \sum_i \mathbb{I}[\hat{y}_i^C \neq \hat{y}_i^G].$$

Part 3 - Hints and Helper

Use identical dtype, initialization, and permutations. Move GPU checkpoints to CPU before comparison and report tolerances explicitly.

Part 4 - Python code skeleton

```
import torch

def E16(cpu_result, gpu_result, probe_features, atol, rtol):
    cpu_weight, cpu_bias, cpu_history = cpu_result
    gpu_weight, gpu_bias, gpu_history = gpu_result
    gpu_weight, gpu_bias = gpu_weight.cpu(), gpu_bias.cpu()
    parameter_close = torch.allclose(
        cpu_weight, gpu_weight, atol=atol, rtol=rtol
    )
    cpu_predictions = probe_features @ cpu_weight + cpu_bias >= 0
    gpu_predictions = probe_features @ gpu_weight + gpu_bias >= 0
    return {
        "parameter_close": parameter_close,
        "history_equal": cpu_history == gpu_history,
        "disagreement": ...,
    }

def main():
    print(E16(...))

if __name__ == "__main__":
    main()
```

12.5.18 Exercise 17 - Stress Test Convergence with Large-Scale Synthetic Data

Part 1 - Why this matters

Large synthetic datasets test whether convergence measurements remain correct under realistic memory, throughput, and dimensionality pressure. They also expose accidental history growth and inefficient device transfers.

The generator should provide a known separator and controllable margin so the convergence assumptions are not ambiguous. Performance statistics must be reported beside correctness and stability checks.

Part 2 - Mathematical form

$$X \in \mathbb{R}^{n \times d}, \quad y_i = \operatorname{sign}(\mathbf{u}^{\mathsf{T}} \mathbf{x}_i), \quad \text{work per epoch} = \mathcal{O}(nd).$$

Part 3 - Hints and Helper

Generate data in chunks if memory is limited. Synchronize CUDA around timing and store aggregate histories rather than every full parameter vector.

Part 4 - Python code skeleton

```
import time
```

```
import torch

def E17(samples, dimensions, epochs, device):
    features = torch.randn(samples, dimensions, device=device)
    teacher = torch.randn(dimensions, device=device)
    labels = torch.where(features @ teacher >= 0, 1.0, -1.0)
    weight = torch.zeros(dimensions, device=device)
    bias = torch.zeros((), device=device)
    if device.type == "cuda":
        torch.cuda.synchronize()
    start = time.perf_counter()
    updates = ...
    if device.type == "cuda":
        torch.cuda.synchronize()
    elapsed = time.perf_counter() - start
    return {"updates": updates, "elapsed": elapsed, "finite": ...}

def main():
    print(E17(...))

if __name__ == "__main__":
    main()
```

12.5.19 Exercise 18 - Validate Theoretical Claims Using Empirical Results

Part 1 - Why this matters

Theory states conclusions under explicit assumptions, while experiments observe finite datasets and finite-precision implementations. Validation connects the two by translating each claim into measurable quantities and checking its assumptions first.

An empirical match supports the implementation and illustrates the theorem, but it does not constitute a universal proof. A mismatch should trigger investigation of assumptions, measurements, and code before the theory is rejected.

Part 2 - Mathematical form

$$C_j = (A_j, P_j, O_j), \qquad V_j = \mathbb{I}[A_j \text{ verified and } O_j \text{ is consistent with } P_j].$$

Part 3 - Hints and Helper

Create a table containing the assumption, prediction, observation, tolerance, and verdict for each claim. Include convergence, mistake bound, margin stability, and non-separable failure.

Part 4 - Python code skeleton

```
import torch
```

```
def E18(claims):
    results = []
    for claim in claims:
        assumptions_hold = all(check() for check in claim["assumptions"])
        observed = claim["measure"]()
        consistent = claim["compare"](observed) if assumptions_hold else None
        results.append({
            "name": claim["name"],
            "assumptions_hold": assumptions_hold,
            "observed": observed,
            "consistent": consistent,
        })
    return results

def main():
    print(E18(...))

if __name__ == "__main__":
    main()
```

12.5.20 Exercise 19 - Write a Formal Convergence and Stability Report

Part 1 - Why this matters

A formal report combines mathematical assumptions, experimental design, convergence evidence, numerical diagnostics, and limitations into one reproducible account. It prevents isolated plots or final accuracies from being mistaken for complete analysis.

The conclusion must distinguish theorem-supported statements, empirical observations, and unresolved failure causes. Seeds, data order, dtype, device, tolerance, and stopping rules are part of the result and must be recorded.

Part 2 - Mathematical form

$$R = \{\text{assumptions}, R, \gamma, U, K, \text{ordering}, \text{precision}, \text{drift}, \text{generalization}, \text{limitations}\}.$$

Part 3 - Hints and Helper

Require every section before marking the report complete. Cross-reference each conclusion to a formula, measurement, or diagnostic artifact.

Part 4 - Python code skeleton

```
import torch

def E19(sections):
    required = {
```

```
        "assumptions", "radius", "margin", "update_count",
        "mistake_bound", "ordering", "precision", "drift",
        "generalization", "failure_modes", "limitations",
    }
    missing = sorted(required - sections.keys())
    warnings = []
    if sections.get("margin", 0.0) <= 0:
        warnings.append(...)
    if not sections.get("assumptions", {}).get("linearly_separable", False):
        warnings.append(...)
    return {
        "complete": not missing,
        "missing": missing,
        "warnings": warnings,
        "sections": sections,
    }

def main():
    print(E19(...))

if __name__ == "__main__":
    main()
```

12.6 PC04 - Perceptron Loss Perspective and Errors

Models and Training

12.6.1 Exercise 00 - Interpret Perceptron Learning as Error-Driven Optimization

Part 1 - Why this matters

Perceptron learning reacts only to samples whose signed scores are non-positive. Each mistake creates a corrective movement that increases the sample's signed score, while correctly classified samples leave the parameters unchanged.

This view connects the historical update rule to optimization language without pretending that the hard step prediction is differentiable. The update can be interpreted as a subgradient step on a piecewise-linear perceptron criterion.

Part 2 - Mathematical form

$$m_i = y_i(\mathbf{w}^\mathsf{T}\mathbf{x}_i + b), \qquad (\mathbf{w}, b) \leftarrow (\mathbf{w}, b) + \eta y_i(\mathbf{x}_i, 1) \text{ when } m_i \leq 0.$$

Part 3 - Hints and Helper

Compute the signed margin before the prediction. Return the mistake mask and parameter correction so that the optimization signal remains observable.

Part 4 - Python code skeleton

```
import torch

def E00(features, labels, weight, bias, learning_rate):
    margins = labels * (features @ weight + bias)
    mistakes = margins <= 0
    correction_w = learning_rate * ...
    correction_b = learning_rate * ...
    return mistakes, correction_w, correction_b

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

12.6.2 Exercise 01 - Define Implicit Loss Function Used by the Perceptron

Part 1 - Why this matters

The perceptron criterion assigns zero loss to positive signed margins and a linear penalty to non-positive margins. It therefore measures both whether a sample is mistaken and how far its score lies on the wrong side of the boundary.

The boundary convention must be stated because a zero margin is normally treated as a violation during training. Reduction by sum or mean also changes the scale of a batch loss and its resulting update.

Part 2 - Mathematical form

$$\ell_P(m) = \max(0, -m), \qquad m = y(\mathbf{w}^\mathsf{T}\mathbf{x} + b), \qquad L_P = \sum_{i=1}^{n} \ell_P(m_i).$$

Part 3 - Hints and Helper

Use torch.clamp with a minimum of zero. Return unreduced losses first, then apply the requested reduction explicitly.

Part 4 - Python code skeleton

```
import torch

def E01(scores, labels, reduction="mean"):
    margins = labels * scores
    losses = torch.clamp(-margins, min=0)
    if reduction == "none":
        return losses
```

```
    if reduction == "sum":
        return ...
    if reduction == "mean":
        return ...
    raise ValueError(...)

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

12.6.3 Exercise 02 - Compare Perceptron Loss with Hinge Loss

Part 1 - Why this matters

Perceptron and hinge losses are both piecewise linear, but they define different violation regions. Perceptron loss stops penalizing as soon as the signed margin is positive, whereas hinge loss continues until the margin reaches one.

This distinction makes hinge loss encourage a safety margin rather than mere correct classification. Comparing both curves and subgradients explains why two models can have identical accuracy but different loss values.

Part 2 - Mathematical form

$$\ell_P(m) = \max(0, -m), \qquad \ell_H(m) = \max(0, 1 - m).$$

Part 3 - Hints and Helper

Evaluate both losses on the same margin grid, especially below zero, between zero and one, and above one. Compare active masks and slopes.

Part 4 - Python code skeleton

```
import torch

def E02(margins):
    perceptron_loss = torch.clamp(-margins, min=0)
    hinge_loss = torch.clamp(1 - margins, min=0)
    return {
        "perceptron": perceptron_loss,
        "hinge": hinge_loss,
        "different": ...,
    }

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

12.6.4 Exercise 03 - Analyze Zero-One Loss Limitations in Optimization

Part 1 - Why this matters

Zero-one loss directly counts incorrect predictions, making it natural for evaluation. As an optimization objective, however, it is constant across broad parameter regions and discontinuous when a score crosses the decision boundary.

Its derivative is zero almost everywhere and undefined at the boundary, so ordinary gradient methods receive no useful direction. Surrogate losses replace this flat signal with magnitude-sensitive penalties.

Part 2 - Mathematical form

$$\ell_{0\text{–}1}(m) = \mathbb{I}[m \leq 0], \qquad \frac{d\ell_{0\text{–}1}}{dm} = 0 \text{ a.e.}$$

Part 3 - Hints and Helper

Perturb one score across zero and compare zero-one loss with perceptron and hinge losses. Observe information lost when every mistake has the same cost.

Part 4 - Python code skeleton

```
import torch

def E03(margins):
    zero_one = (margins <= 0).to(margins.dtype)
    perceptron = torch.clamp(-margins, min=0)
    hinge = torch.clamp(1 - margins, min=0)
    return zero_one, perceptron, hinge

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

12.6.5 Exercise 04 - Visualize Loss Landscape for Simple Perceptron Models

Part 1 - Why this matters

A loss landscape shows how candidate weights and biases change the total perceptron criterion. Even a two-parameter slice reveals flat zero-loss regions, linear faces, ridges, and non-smooth boundaries created by different active samples.

The visualization is a slice of parameter space rather than the entire high-dimensional objective. Fixed parameters, axis ranges, reduction choice, and dataset must be recorded so the picture can be interpreted correctly.

Part 2 - Mathematical form

$$L(w, b) = \sum_{i=1}^{n} \max\{0, -y_i(wx_i + b)\}.$$

Part 3 - Hints and Helper

Create a mesh with torch.meshgrid, evaluate every parameter pair without Python loops when possible, and label the plotted reduction.

Part 4 - Python code skeleton

```
import matplotlib.pyplot as plt
import torch

def E04(features, labels, weight_values, bias_values):
    weight_grid, bias_grid = torch.meshgrid(
        weight_values, bias_values, indexing="ij"
    )
    scores = ...
    margins = labels * scores
    landscape = torch.clamp(-margins, min=0).sum(dim=-1)
    figure, axis = plt.subplots()
    contour = axis.contourf(...)
    figure.colorbar(contour, ax=axis)
    return figure, landscape

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

12.6.6 Exercise 05 - Implement Perceptron Loss Manually in PyTorch

Part 1 - Why this matters

A manual PyTorch implementation exposes every assumption hidden inside a packaged objective: label encoding, score shape, margin sign, boundary treatment, dtype, and reduction. These details determine whether the returned value matches the learning rule.

The function should remain vectorized and preserve gradient tracking when its inputs require gradients. Shape validation is necessary because accidental broadcasting can produce a plausible but incorrect loss tensor.

Part 2 - Mathematical form

$$L_P(X, y; \mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^{n} \max\{0, -y_i(\mathbf{x}_i^\top \mathbf{w} + b)\}.$$

Part 3 - Hints and Helper

Require a two-dimensional feature matrix and one-dimensional labels. Check batch sizes before computing scores and make reduction an explicit argument.

Part 4 - Python code skeleton

```
import torch

def E05(features, labels, weight, bias, reduction="mean"):
    if features.ndim != 2 or labels.ndim != 1:
        raise ValueError(...)
    if features.shape[0] != labels.shape[0]:
        raise ValueError(...)
    scores = features @ weight + bias
    losses = torch.clamp(-(labels * scores), min=0)
    return ...

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

12.6.7 Exercise 06 - Compare Gradient-Free Updates vs Gradient-Based Losses

Part 1 - Why this matters

The classical perceptron applies an explicit mistake-driven update without calling automatic differentiation. The same correction direction can also emerge as a subgradient of the perceptron criterion away from its non-smooth boundary.

This comparison separates implementation mechanism from mathematical objective. Autograd, manual subgradients, and optimizer state can change how updates are produced even when their first-step directions agree.

Part 2 - Mathematical form

$$\partial_{\mathbf{w}} \ell_P = \begin{cases} -y\mathbf{x}, & m < 0, \\ \mathbf{0}, & m > 0, \end{cases} \qquad \mathbf{w} \leftarrow \mathbf{w} - \eta \mathbf{g}.$$

Part 3 - Hints and Helper

Use one violated sample away from zero. Compare the explicit perceptron correction with an autograd step and state how the boundary subgradient is chosen.

Part 4 - Python code skeleton

```
import torch

def E06(features, label, weight, bias, learning_rate):
    manual_weight = weight.detach().clone()
    if label * (manual_weight @ features + bias.detach()) <= 0:
        manual_weight = manual_weight + learning_rate * label * features
    differentiable_weight = weight.detach().clone().requires_grad_(True)
    loss = torch.clamp(
        -label * (differentiable_weight @ features + bias), min=0
    )
    loss.backward()
    gradient_weight = differentiable_weight - learning_rate * ...
    return manual_weight, gradient_weight.detach()

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

12.6.8 Exercise 07 - Observe Non-Smooth Loss Behavior at Decision Boundary

Part 1 - Why this matters

At zero signed margin, the perceptron loss changes from a linear penalty to a flat region. The function is continuous there, but its slope changes abruptly, so the ordinary derivative is not uniquely defined.

Finite differences from the left and right reveal the two slopes. An automatic differentiation system selects one valid convention for the boundary, which should not be mistaken for a unique mathematical derivative.

Part 2 - Mathematical form

$$\ell_P(m) = \max(0, -m), \qquad \ell'_P(0^-) = -1, \qquad \ell'_P(0^+) = 0.$$

Part 3 - Hints and Helper

Evaluate symmetric points around zero and compute one-sided finite differences. Inspect the autograd gradient at exactly zero separately.

Part 4 - Python code skeleton

```
import torch

def E07(epsilon):
    margins = torch.tensor(
        [-epsilon, 0.0, epsilon], requires_grad=True
    )
    losses = torch.clamp(-margins, min=0)
    left_slope = ...
    right_slope = ...
    losses.sum().backward()
    return losses.detach(), left_slope, right_slope, margins.grad

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

12.6.9 Exercise 08 - Analyze Sensitivity of Loss to Margin Violations

Part 1 - Why this matters

Perceptron loss distinguishes shallow and deep violations because the penalty grows linearly with negative signed margin. Two mistaken samples therefore contribute differently even though zero-one loss counts both as one error.

Sensitivity should be studied under controlled scaling. Changing feature units or parameter magnitude changes score margins and loss values without necessarily changing predicted classes.

Part 2 - Mathematical form

$$m < 0 \Rightarrow \ell_P(m) = -m, \qquad \frac{\partial \ell_P}{\partial m} = -1.$$

Part 3 - Hints and Helper

Create margins of increasing negative magnitude. Compare per-sample loss, zero-one error, and the effect of multiplying all scores by a positive constant.

Part 4 - Python code skeleton

```
import torch

def E08(margins, scales):
    results = {}
    for scale in scales:
        scaled = scale * margins
        results[scale] = {
            "loss": torch.clamp(-scaled, min=0),
            "errors": (scaled <= 0).to(torch.int64),
```

```
        }
    return results

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

12.6.10 Exercise 09 - Study Error Accumulation Across Epochs

Part 1 - Why this matters

Summing losses and mistakes across epochs describes the total correction pressure experienced during training. It also reveals whether errors disappear, oscillate, or remain concentrated in a small set of difficult samples.

Cumulative quantities must not replace per-epoch values because they never decrease. Both views are needed to distinguish continuing failure from historical errors that were already corrected.

Part 2 - Mathematical form

$$C_E = \sum_{e=1}^{E} M_e, \qquad Q_E = \sum_{e=1}^{E} L_e, \qquad M_e = \sum_{i} \mathbb{I}[m_i^{(e)} \leq 0].$$

Part 3 - Hints and Helper

Log epoch loss and mistake count before or after updates consistently. Then compute cumulative sums with torch.cumsum and retain sample-level counts if possible.

Part 4 - Python code skeleton

```
import torch

def E09(epoch_losses, epoch_errors):
    losses = torch.as_tensor(epoch_losses, dtype=torch.float64)
    errors = torch.as_tensor(epoch_errors, dtype=torch.int64)
    return {
        "cumulative_loss": torch.cumsum(losses, dim=0),
        "cumulative_errors": torch.cumsum(errors, dim=0),
        "final_epoch_loss": ...,
        "final_epoch_errors": ...,
    }

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

12.6.11 Exercise 10 - Implement Loss Computation on CUDA

Part 1 - Why this matters

CUDA loss computation requires every participating tensor to share a compatible device and dtype. Hidden CPU scalars, labels, or parameters can interrupt an otherwise vectorized operation.

The exercise also distinguishes device correctness from performance. Small workloads may be slower on a GPU, so timing should synchronize CUDA and should not be used as proof that the formula is correct.

Part 2 - Mathematical form

$$L_P^{\mathrm{GPU}} = \mathrm{reduce}(\max\{0, -y \odot (X\mathbf{w} + b)\}) .$$

Part 3 - Hints and Helper

Check CUDA availability, transfer tensors once, and compute scores, margins, and reduction on the device. Move only the final reporting values back to CPU.

Part 4 - Python code skeleton

```
import torch

def E10(features, labels, weight, bias):
    if not torch.cuda.is_available():
        raise RuntimeError(...)
    device = torch.device("cuda")
    features = features.to(device)
    labels = labels.to(device)
    weight = weight.to(device)
    bias = bias.to(device)
    scores = features @ weight + bias
    losses = torch.clamp(-(labels * scores), min=0)
    return losses.mean(), losses

def main():
    print(E10(...))

if __name__ == "__main__":
    main()
```

12.6.12 Exercise 11 - Compare CPU vs GPU Loss Evaluation Consistency

Part 1 - Why this matters

Equivalent CPU and GPU formulas should produce close loss values when they receive the same data and dtype. Exact equality is not always expected because floating-point operation

order can differ across hardware.

A useful parity test compares unreduced losses, reduced loss, active violation masks, and tolerances. Agreement of only the mean can hide compensating per-sample differences.

Part 2 - Mathematical form

$$\delta = |L_C - L_G|, \qquad D = \sum_i \mathbb{I}[(m_i^C \le 0) \neq (m_i^G \le 0)].$$

Part 3 - Hints and Helper

Move GPU results to CPU, call `torch.testing.assert_close()` with documented tolerances, and separately count violation-mask disagreement.

Part 4 - Python code skeleton

```
import torch

def E11(cpu_losses, gpu_losses, atol, rtol):
    gpu_losses = gpu_losses.detach().cpu()
    close = torch.allclose(cpu_losses, gpu_losses, atol=atol, rtol=rtol)
    absolute_difference = torch.abs(cpu_losses - gpu_losses)
    return {
        "close": close,
        "maximum_difference": absolute_difference.max().item(),
        "mean_difference": absolute_difference.mean().item(),
    }

def main():
    print(E11(...))

if __name__ == "__main__":
    main()
```

12.6.13 Exercise 12 - Examine Numerical Stability of Loss Under Extreme Values

Part 1 - Why this matters

Very large features or weights can create scores that overflow finite floating-point ranges. The perceptron loss has no logarithm or exponential, but its dot products and reductions can still become infinite or lose precision.

Stability analysis should inspect intermediate scores and unreduced losses before trusting a finite-looking reduction. Float32 and float64 comparisons help locate whether the failure is caused by representation limits.

Part 2 - Mathematical form

$$z = \mathbf{w}^\top \mathbf{x} + b, \qquad |z| > F_{\max} \Rightarrow z = \pm\infty \text{ in finite precision.}$$

Part 3 - Hints and Helper

Sweep input magnitudes across dtypes. Report finiteness, maximum absolute score, maximum loss, and whether the reduction overflowed.

Part 4 - Python code skeleton

```
import torch

def E12(features, labels, weight, bias, dtypes):
    report = {}
    for dtype in dtypes:
        scores = features.to(dtype) @ weight.to(dtype) + bias.to(dtype)
        losses = torch.clamp(-(labels.to(dtype) * scores), min=0)
        report[str(dtype)] = {
            "scores_finite": torch.isfinite(scores).all().item(),
            "losses_finite": torch.isfinite(losses).all().item(),
            "maximum_loss": losses.max().item(),
        }
    return report

def main():
    print(E12(...))

if __name__ == "__main__":
    main()
```

12.6.14 Exercise 13 - Relate Loss Behavior to Classification Performance

Part 1 - Why this matters

Loss and classification error answer different questions. Error counts wrong labels, while perceptron loss also measures how far violated scores lie beyond the boundary; neither alone fully describes confidence or generalization.

A model can reduce loss without changing accuracy, or maintain zero perceptron loss while positive margins remain extremely small. Training and validation curves should therefore be interpreted together with normalized margins.

Part 2 - Mathematical form

$$E = \frac{1}{n}\sum_i \mathbb{I}[m_i \le 0], \qquad L_P = \frac{1}{n}\sum_i \max(0, -m_i), \qquad \gamma_{\min} = \min_i \frac{m_i}{\|\mathbf{w}\|_2}.$$

Part 3 - Hints and Helper

For every checkpoint, compute loss, error rate, and normalized minimum margin. Identify epochs where loss changes but error remains constant.

Part 4 - Python code skeleton

```
import torch

def E13(checkpoints, features, labels):
    rows = []
    for weight, bias in checkpoints:
        margins = labels * (features @ weight + bias)
        loss = torch.clamp(-margins, min=0).mean()
        error = (margins <= 0).float().mean()
        normalized_margin = margins.min() / (
            torch.linalg.vector_norm(weight) + 1e-12
        )
        rows.append((loss.item(), error.item(), normalized_margin.item()))
    return rows

def main():
    print(E13(...))

if __name__ == "__main__":
    main()
```

12.6.15 Exercise 14 - Debug Unexpected Loss Values During Training

Part 1 - Why this matters

Unexpected losses often originate from incorrect label encoding, reversed margin signs, accidental broadcasting, wrong reduction, stale parameters, or non-finite input values. The scalar loss alone cannot identify which defect occurred.

A disciplined debugger inspects shapes, devices, dtypes, label values, scores, signed margins, active masks, and per-sample losses in that order. Small hand-computed cases provide an independent reference.

Part 2 - Mathematical form

$$\ell_i = \max\{0, -y_i z_i\}, \qquad y_i \in \{-1, +1\}, \qquad z \in \mathbb{R}^n.$$

Part 3 - Hints and Helper

Reject invalid shapes and labels before reduction. Return a diagnostic dictionary containing intermediate tensors rather than printing only the final mean.

Part 4 - Python code skeleton

```
import torch

def E14(features, labels, weight, bias):
    diagnostics = {}
    diagnostics["shape_valid"] = ...
    allowed = torch.tensor([-1, 1], device=labels.device)
    diagnostics["labels_valid"] = torch.isin(labels, allowed).all().item()
    scores = features @ weight + bias
    margins = labels * scores
    losses = torch.clamp(-margins, min=0)
    diagnostics.update({
        "scores": scores,
        "margins": margins,
        "losses": losses,
        "finite": torch.isfinite(losses).all().item(),
    })
    return diagnostics

def main():
    print(E14(...))

if __name__ == "__main__":
    main()
```

12.6.16 Exercise 15 - Visualize Loss Evolution Over Training Iterations

Part 1 - Why this matters

A loss curve exposes rapid corrections, plateaus, oscillations, and unstable jumps during training. Placing mistake counts beside loss shows whether magnitude improvements translate into changed classifications.

The curve must state whether values were recorded before or after each epoch's updates and whether the reduction was a sum or mean. Otherwise comparisons across batch sizes or datasets become misleading.

Part 2 - Mathematical form

$$\mathcal{C} = \{(t, L_t, M_t)\}_{t=0}^{T}.$$

Part 3 - Hints and Helper

Store plain numeric values rather than graph-connected tensors. Plot loss and errors on labeled axes, using separate scales when their ranges differ.

Part 4 - Python code skeleton

```
import matplotlib.pyplot as plt
import torch

def E15(loss_history, error_history):
    if len(loss_history) != len(error_history):
        raise ValueError(...)
    steps = range(len(loss_history))
    figure, loss_axis = plt.subplots()
    error_axis = loss_axis.twinx()
    loss_axis.plot(steps, loss_history, label="loss")
    error_axis.plot(steps, error_history, color="tab:red", label="errors")
    return figure

def main():
    print(E15(...))

if __name__ == "__main__":
    main()
```

12.6.17 Exercise 16 - Contrast Loss-Based Training with Backprop-Based Models

Part 1 - Why this matters

Classical perceptron training uses an explicit local correction after a mistake. Backpropagation instead differentiates a scalar objective through a computational graph, allowing credit assignment across many layers and parameters.

The contrast is not simply loss versus no loss: perceptron updates can be viewed as subgradient steps on a loss. The central differences are differentiability, graph depth, update aggregation, and the machinery used to obtain parameter derivatives.

Part 2 - Mathematical form

$$\text{perceptron: } \Delta \mathbf{w} = \eta y \mathbf{x} \mathbb{I}[m \leq 0], \qquad \text{backprop: } \theta \leftarrow \theta - \eta \nabla_\theta L.$$

Part 3 - Hints and Helper

Apply both mechanisms to a one-layer example. Compare update direction, batch reduction, gradient storage, and behavior at the non-smooth boundary.

Part 4 - Python code skeleton

```
import torch

def E16(features, labels, manual_state, model, learning_rate):
    manual_weight, manual_bias = manual_state
    manual_weight, manual_bias = ...
    optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
```

```
    optimizer.zero_grad()
    scores = model(features).squeeze(-1)
    loss = torch.clamp(-(labels * scores), min=0).mean()
    loss.backward()
    optimizer.step()
    return (manual_weight, manual_bias), model.state_dict(), loss.item()

def main():
    print(E16(...))

if __name__ == "__main__":
    main()
```

12.6.18 Exercise 17 - Evaluate Robustness of Loss to Label Noise

Part 1 - Why this matters

A flipped label converts a previously correct margin into a violation and can create a large loss when the model was confidently correct under the clean target. Repeated noisy corrections may dominate training and prevent convergence.

Robustness should be measured across controlled noise rates and repeated seeds. Clean evaluation performance, total loss, update count, and parameter drift provide a fuller picture than noisy training accuracy.

Part 2 - Mathematical form

$$\tilde{y}_i = -y_i \text{ for } i \in \mathcal{N}, \qquad \ell_P(\tilde{y}_i z_i) = \max(0, y_i z_i).$$

Part 3 - Hints and Helper

Save the flipped indices and train clean and noisy variants from the same initialization. Compare weight direction and performance on untouched labels.

Part 4 - Python code skeleton

```
import torch

def E17(features, labels, noise_rates, seed):
    generator = torch.Generator().manual_seed(seed)
    report = {}
    for rate in noise_rates:
        noisy = labels.clone()
        count = int(rate * len(labels))
        indices = torch.randperm(len(labels), generator=generator)[:count]
        noisy[indices] *= -1
        state, loss_history = ...
        clean_error = ...
        report[rate] = {
```

```
            "flipped": indices,
            "final_loss": loss_history[-1],
            "clean_error": clean_error,
        }
    return report

def main():
    print(E17(...))

if __name__ == "__main__":
    main()
```

12.6.19 Exercise 18 - Identify Scenarios Where Loss Minimization Fails

Part 1 - Why this matters

A decreasing training loss does not guarantee a useful model. Failure can arise from non-separable data, corrupted labels, poor feature representation, distribution shift, numerical overflow, or optimizing a surrogate that is misaligned with the real evaluation cost.

The diagnosis must distinguish optimization failure from objective failure and generalization failure. Each category produces different evidence and requires a different remedy.

Part 2 - Mathematical form

$$\begin{aligned} \text{success requires} \quad & \text{an optimizable objective} \\ & + \text{ a suitable representation} \\ & + \text{ representative data} \\ & + \text{ an aligned metric.} \end{aligned}$$

Part 3 - Hints and Helper

Construct at least one controlled example for each failure category. Record loss, train error, validation error, finiteness, and convergence status.

Part 4 - Python code skeleton

```
import torch

def E18(experiments):
    results = []
    for experiment in experiments:
        training = experiment["run"]()
        category = ...
        results.append({
            "name": experiment["name"],
            "category": category,
            "train_loss": training["train_loss"],
```

```
            "train_error": training["train_error"],
            "validation_error": training["validation_error"],
            "finite": training["finite"],
        })
    return results

def main():
    print(E18(...))

if __name__ == "__main__":
    main()
```

12.6.20 Exercise 19 - Produce a Critical Analysis of Perceptron Loss Limitations

Part 1 - Why this matters

A critical analysis should explain what the perceptron criterion captures and what it ignores. It provides a simple correction signal for violated margins but does not reward positive margin size, handle label noise robustly, or supply probabilistic outputs.

The report must support each limitation with equations or experiments and distinguish the loss from the broader limitations of a single linear classifier. Alternatives should be proposed according to the identified failure rather than presented as universal replacements.

Part 2 - Mathematical form

$$R = \{\text{definition}, \text{subgradient}, \text{boundary}, \text{margin}, \text{noise}, \text{numerics}, \text{generalization}, \text{alternatives}\}.$$

Part 3 - Hints and Helper

Require the label convention, reduction, device, dtype, and evidence for every conclusion. Compare zero-one, perceptron, and hinge loss explicitly.

Part 4 - Python code skeleton

```
import torch

def E19(sections):
    required = {
        "definition", "zero_one_comparison", "hinge_comparison",
        "subgradient", "boundary", "margin", "label_noise",
        "numerical_stability", "generalization", "alternatives",
    }
    missing = sorted(required - sections.keys())
    unsupported = [
        name for name, value in sections.items()
        if isinstance(value, dict) and not value.get("evidence")
```

```
    ]
    return {
        "complete": not missing and not unsupported,
        "missing": missing,
        "unsupported": unsupported,
        "sections": sections,
    }

def main():
    print(E19(...))

if __name__ == "__main__":
    main()
```

12.7 PC05 - Multi-Class Perceptron Extensions

Models and Training

12.7.1 Exercise 00 - Extend Binary Perceptron to One-vs-Rest Multi-Class Setting

Part 1 - Why this matters

One-vs-rest converts a problem with many classes into one binary perceptron per class. Each classifier learns whether a sample belongs to its assigned class or to the combined set of all remaining classes.

The strategy is simple and reuses binary training, but the negative class can be much larger and classifier scores may not be calibrated. A prediction rule is still required to resolve simultaneous positive outputs or complete rejection.

Part 2 - Mathematical form

$$y_i^{(k)} = \begin{cases} +1, & y_i = k, \\ -1, & y_i \neq k, \end{cases} \qquad f_k(\mathbf{x}) = \mathbf{w}_k^{\mathsf{T}}\mathbf{x} + b_k.$$

Part 3 - Hints and Helper

Construct a binary target vector for every class and train classifiers independently. Preserve the mapping between row index and original class label.

Part 4 - Python code skeleton

```
import torch

def E00(features, labels, classes, train_binary):
    models = {}
    for class_id in classes:
```

```
        binary_labels = torch.where(labels == class_id, 1, -1)
        models[int(class_id)] = train_binary(features, binary_labels, ...)
    return models

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

12.7.2 Exercise 01 - Implement Class-Specific Weight Vectors for Multi-Class Output

Part 1 - Why this matters

A direct multi-class perceptron assigns one weight vector and one bias to every class. Stacking them creates a parameter matrix that scores all candidate classes in one matrix multiplication.

Shape reasoning is essential because rows represent classes and columns represent features. A transposed matrix may still multiply after other accidental transpositions while attaching scores to the wrong classes.

Part 2 - Mathematical form

$$W = [\mathbf{w}_1^\mathsf{T}; \ldots; \mathbf{w}_K^\mathsf{T}] \in \mathbb{R}^{K \times d}, \quad \mathbf{b} \in \mathbb{R}^K, \quad S = XW^\mathsf{T} + \mathbf{b}.$$

Part 3 - Hints and Helper

Validate the feature count and class count. Return a score matrix of shape batch size by number of classes.

Part 4 - Python code skeleton

```
import torch

def E01(features, class_count):
    feature_count = features.shape[1]
    weight = torch.zeros(class_count, feature_count)
    bias = torch.zeros(class_count)
    scores = features @ weight.T + bias
    if scores.shape != (len(features), class_count):
        raise RuntimeError(...)
    return weight, bias, scores

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

12.7.3 Exercise 02 - Define Prediction Rule for Multi-Class Perceptron

Part 1 - Why this matters

The direct multi-class perceptron predicts the class with the largest linear score. Unlike softmax, these scores are not probabilities and their absolute scale has no probabilistic interpretation.

Ties require an explicit deterministic or randomized policy. The default argmax rule selects the first maximum, which can create systematic preference for lower class indices.

Part 2 - Mathematical form

$$\hat{y} = \underset{k \in \{1,\ldots,K\}}{\arg\max} (\mathbf{w}_k^\top \mathbf{x} + b_k).$$

Part 3 - Hints and Helper

Compute all scores together and retain both the selected index and winning score. Detect ties before applying the chosen policy.

Part 4 - Python code skeleton

```
import torch

def E02(features, weight, bias):
    scores = features @ weight.T + bias
    predictions = torch.argmax(scores, dim=1)
    maximum = scores.max(dim=1, keepdim=True).values
    tie_counts = (scores == maximum).sum(dim=1)
    return predictions, scores, tie_counts

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

12.7.4 Exercise 03 - Implement Multi-Class Training Loop with Error Updates

Part 1 - Why this matters

On a mistake, the multi-class perceptron increases the score of the true class and decreases the score of the incorrectly predicted class. This paired update changes only two class-specific parameter rows.

The symmetry matters: updating only the correct row may leave the competing wrong score high. Training logs should identify both affected classes and verify that unrelated rows remain unchanged.

Part 2 - Mathematical form

$$W_{y_i} \leftarrow W_{y_i} + \eta \mathbf{x}_i, \quad W_{\hat{y}_i} \leftarrow W_{\hat{y}_i} - \eta \mathbf{x}_i, \quad \text{if } \hat{y}_i \neq y_i.$$

Part 3 - Hints and Helper

Clone rows when logging them and update biases with the same positive and negative pattern. Stop only after a complete error-free epoch.

Part 4 - Python code skeleton

```
import torch

def E03(features, labels, weight, bias, learning_rate, epochs):
    history = []
    for _ in range(epochs):
        mistakes = 0
        for features_i, label_i in zip(features, labels):
            prediction = torch.argmax(weight @ features_i + bias)
            if prediction != label_i:
                weight[label_i] += ...
                weight[prediction] -= ...
                bias[label_i] += learning_rate
                bias[prediction] -= learning_rate
                mistakes += 1
        history.append(mistakes)
        if mistakes == 0:
            break
    return weight, bias, history

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

12.7.5 Exercise 04 - Analyze Decision Regions in Multi-Class Feature Space

Part 1 - Why this matters

Every class owns the points where its score exceeds all competing scores. These regions are intersections of linear half-spaces and can form polyhedral areas rather than one isolated binary split.

Pairwise equality lines explain the borders, but only portions that dominate every other class appear in the final decision map. Plotting all pairwise lines without checking dominance can therefore be misleading.

Part 2 - Mathematical form

$$\mathcal{R}_k = \bigcap_{j \neq k} \{\mathbf{x} : (\mathbf{w}_k - \mathbf{w}_j)^\mathsf{T}\mathbf{x} + (b_k - b_j) \geq 0\}.$$

Part 3 - Hints and Helper

Evaluate a dense two-dimensional grid, assign each point by argmax, and compare visible regions with pairwise score-equality boundaries.

Part 4 - Python code skeleton

```
import torch

def E04(weight, bias, x_values, y_values):
    grid_x, grid_y = torch.meshgrid(x_values, y_values, indexing="xy")
    points = torch.stack((grid_x.reshape(-1), grid_y.reshape(-1)), dim=1)
    scores = points @ weight.T + bias
    regions = torch.argmax(scores, dim=1).reshape(grid_x.shape)
    return grid_x, grid_y, regions

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

12.7.6 Exercise 05 - Compare One-vs-Rest and One-vs-One Strategies

Part 1 - Why this matters

One-vs-rest trains one classifier per class, whereas one-vs-one trains a classifier for every pair of classes. The second approach uses smaller pairwise datasets but increases the number of models quadratically.

Their outputs are combined differently: one-vs-rest compares class scores, while one-vs-one usually uses votes. Accuracy, training cost, memory, tie frequency, and imbalance should all be compared.

Part 2 - Mathematical form

$$N_{\mathrm{OvR}} = K, \qquad N_{\mathrm{OvO}} = \binom{K}{2} = \frac{K(K-1)}{2}.$$

Part 3 - Hints and Helper

Generate every unordered class pair without duplication. Report model count and define deterministic tie-breaking for both score and vote aggregation.

Part 4 - Python code skeleton

```
import torch

def E05(classes):
    one_vs_rest = [(class_id, "rest") for class_id in classes]
    one_vs_one = []
    for left in range(len(classes)):
        for right in range(left + 1, len(classes)):
            one_vs_one.append((classes[left], classes[right]))
    return {
        "ovr_models": one_vs_rest,
        "ovo_models": one_vs_one,
        "counts": (..., ...),
    }

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

12.7.7 Exercise 06 - Handle Class Imbalance in Multi-Class Perceptron Training

Part 1 - Why this matters

Rare classes may receive too few corrective opportunities, while majority classes dominate the update history and global accuracy. Multi-class training must therefore inspect error rates separately for every class.

Class weighting or balanced sampling can amplify minority corrections, but they also change update magnitudes and training dynamics. The chosen method and its effect on each class must be measured explicitly.

Part 2 - Mathematical form

$$\alpha_k = \frac{n}{Kn_k}, \qquad W_{y_i} \leftarrow W_{y_i} + \eta\alpha_{y_i}\mathbf{x}_i, \quad W_{\hat{y}_i} \leftarrow W_{\hat{y}_i} - \eta\alpha_{y_i}\mathbf{x}_i.$$

Part 3 - Hints and Helper

Compute weights only from the training split and guard against absent classes. Compare macro recall and per-class updates before and after balancing.

Part 4 - Python code skeleton

```
import torch

def E06(labels, class_count):
    counts = torch.bincount(labels, minlength=class_count)
    if (counts == 0).any():
        raise ValueError(...)
    class_weights = len(labels) / (class_count * counts.float())
    sample_weights = class_weights[labels]
    return counts, class_weights, sample_weights

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

12.7.8 Exercise 07 - Evaluate Multi-Class Accuracy and Confusion Matrix

Part 1 - Why this matters

Overall accuracy summarizes correct decisions but hides which classes are confused. A confusion matrix records true classes by predicted classes and exposes asymmetric errors and minority-class failure.

Metric conventions must be fixed before interpretation. Class order, row normalization, ignored labels, and macro versus weighted averaging can materially change the reported conclusion.

Part 2 - Mathematical form

$$C_{ab} = \sum_{i=1}^{n} \mathbb{I}[y_i = a \wedge \hat{y}_i = b], \qquad \text{Accuracy} = \frac{\operatorname{tr}(C)}{\sum_{a,b} C_{ab}}.$$

Part 3 - Hints and Helper

Build the matrix vectorially with flattened class-pair indices. Verify that its total equals the number of evaluated samples.

Part 4 - Python code skeleton

```
import torch

def E07(labels, predictions, class_count):
    indices = labels * class_count + predictions
    matrix = torch.bincount(
        indices, minlength=class_count * class_count
    ).reshape(class_count, class_count)
    accuracy = matrix.diag().sum() / matrix.sum()
```

```
    per_class_recall = matrix.diag() / matrix.sum(dim=1).clamp_min(1)
    return accuracy, matrix, per_class_recall

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

12.7.9 Exercise 08 - Observe Training Instability with Increasing Number of Classes

Part 1 - Why this matters

Adding classes creates more competing scores and more opportunities for a sample to be overtaken by an incorrect class. The update path can become longer, more variable, and more sensitive to initialization or sample order.

Instability should be measured rather than assumed. Update counts, convergence rate, score margins, seed variance, and class-specific errors reveal whether complexity actually damaged a given experiment.

Part 2 - Mathematical form

$$\Delta_i = s_{y_i}(\mathbf{x}_i) - \max_{k \neq y_i} s_k(\mathbf{x}_i), \qquad \hat{y}_i = y_i \iff \Delta_i \geq 0.$$

Part 3 - Hints and Helper

Train nested class subsets under repeated seeds. Keep samples per class and feature scale comparable so class count is the main changing factor.

Part 4 - Python code skeleton

```
import torch

def E08(experiments, seeds):
    report = {}
    for class_count, dataset in experiments.items():
        trials = [train_trial(dataset, seed, ...) for seed in seeds]
        report[class_count] = {
            "mean_updates": ...,
            "convergence_rate": ...,
            "margin_variability": ...,
        }
    return report

def main():
    print(E08(...))
```

```
if __name__ == "__main__":
    main()
```

12.7.10 Exercise 09 - Visualize Multi-Class Decision Boundaries

Part 1 - Why this matters

A colored decision map reveals how class-specific linear scores partition a two-dimensional feature space. It can expose missing regions, narrow unstable areas, and boundaries shaped by unexpected competitors.

The picture is exact only for two input features. Higher-dimensional data require a declared slice or projection, and predictions must be recomputed in that same representation.

Part 2 - Mathematical form

$$\hat{y}(x_1, x_2) = \arg\max_k (w_{k1}x_1 + w_{k2}x_2 + b_k).$$

Part 3 - Hints and Helper

Evaluate a grid in batches if necessary. Overlay labeled samples and use a discrete color map with a documented class-to-color mapping.

Part 4 - Python code skeleton

```
import matplotlib.pyplot as plt
import torch

def E09(features, labels, weight, bias, resolution=200):
    x_values = torch.linspace(
        features[:, 0].min(), features[:, 0].max(), resolution
    )
    y_values = torch.linspace(
        features[:, 1].min(), features[:, 1].max(), resolution
    )
    grid_x, grid_y = torch.meshgrid(x_values, y_values, indexing="xy")
    points = torch.stack((grid_x.flatten(), grid_y.flatten()), dim=1)
    predictions = torch.argmax(points @ weight.T + bias, dim=1)
    figure, axis = plt.subplots()
    axis.contourf(grid_x, grid_y, predictions.reshape(grid_x.shape), ...)
    axis.scatter(features[:, 0], features[:, 1], c=labels)
    return figure

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

12.7.11 Exercise 10 - Execute Multi-Class Perceptron Training on CUDA

Part 1 - Why this matters

CUDA can accelerate score computation when batches, feature counts, or class counts are large. Correct device placement must include features, labels, the full weight matrix, biases, and temporary indices.

A sample-by-sample Python loop may still dominate runtime even when tensors live on the GPU. The exercise separates device correctness from true parallel efficiency and reports both.

Part 2 - Mathematical form

$$S = XW^{\mathsf{T}} + \mathbf{b}, \qquad \hat{\mathbf{y}} = \operatorname{argmax}_k S_{:k}.$$

Part 3 - Hints and Helper

Check CUDA availability and avoid repeated transfers inside epochs. Synchronize before and after timing and return CPU copies only for reporting.

Part 4 - Python code skeleton

```
import torch

def E10(features, labels, class_count, epochs):
    if not torch.cuda.is_available():
        raise RuntimeError(...)
    device = torch.device("cuda")
    features, labels = features.to(device), labels.to(device)
    weight = torch.zeros(class_count, features.shape[1], device=device)
    bias = torch.zeros(class_count, device=device)
    history = ...
    return weight.cpu(), bias.cpu(), history

def main():
    print(E10(...))

if __name__ == "__main__":
    main()
```

12.7.12 Exercise 11 - Compare CPU vs GPU Scaling with Number of Classes

Part 1 - Why this matters

The score matrix grows with class count, increasing arithmetic on both CPU and GPU. GPU parallelism may become advantageous only after the workload is large enough to outweigh launch and synchronization overhead.

Benchmarking must include warm-up, repeated trials, identical dtypes, and synchronization. A single unsynchronized GPU time measures command submission rather than completed computation.

Part 2 - Mathematical form

$$\text{score work} = \mathcal{O}(nKd), \qquad \text{score memory} = \mathcal{O}(nK).$$

Part 3 - Hints and Helper

Sweep class count while holding batch size and feature count fixed. Report median latency, throughput, and CPU–GPU output parity.

Part 4 - Python code skeleton

```
import torch

def E11(features, class_counts, repeats):
    results = []
    for class_count in class_counts:
        cpu_weight = torch.randn(class_count, features.shape[1])
        cpu_time = benchmark(features, cpu_weight, repeats, ...)
        gpu_time = benchmark(
            features.cuda(), cpu_weight.cuda(), repeats, ...
        )
        results.append({
            "classes": class_count,
            "cpu": cpu_time,
            "gpu": gpu_time,
            "speedup": cpu_time / gpu_time,
        })
    return results

def main():
    print(E11(...))

if __name__ == "__main__":
    main()
```

12.7.13 Exercise 12 - Measure Memory Growth with Class Count

Part 1 - Why this matters

Every additional class adds one feature-length weight vector and one bias. Batched score tensors and optimizer or history state can consume even more memory than the parameters themselves.

Theoretical byte counts should be compared with measured allocations. This reveals hidden copies, dtype effects, and temporary tensors that a parameter-only estimate ignores.

Part 2 - Mathematical form

$$M_{\text{parameters}} = K(d+1)q \text{ bytes}, \qquad M_{\text{scores}} = nKq \text{ bytes},$$

Part 3 - Hints and Helper

Use tensor element size rather than assuming four bytes. On CUDA, reset and read peak memory statistics around a controlled forward or training step.

Part 4 - Python code skeleton

```
import torch

def E12(batch_size, feature_count, class_counts, dtype):
    element_size = torch.empty((), dtype=dtype).element_size()
    rows = []
    for class_count in class_counts:
        parameter_bytes = class_count * (feature_count + 1) * element_size
        score_bytes = batch_size * class_count * element_size
        rows.append({
            "classes": class_count,
            "parameter_bytes": parameter_bytes,
            "score_bytes": score_bytes,
            "measured_peak": ...,
        })
    return rows

def main():
    print(E12(...))

if __name__ == "__main__":
    main()
```

12.7.14 Exercise 13 - Investigate Error Propagation Across Classes

Part 1 - Why this matters

A multi-class mistake modifies both the true class and the winning incorrect class. Future scores for these two classes change for every sample, so one correction can repair some decisions while disrupting others.

This interaction is absent from independent binary intuition. Measuring the confusion matrix before and after one update reveals which classes and samples inherit the consequences.

Part 2 - Mathematical form

$$\Delta s_k(\mathbf{x}_j) = \Delta \mathbf{w}_k^{\mathsf{T}} \mathbf{x}_j + \Delta b_k, \qquad k \in \{y_i, \hat{y}_i\}.$$

Part 3 - Hints and Helper

Save the entire score matrix before a selected update. Recompute it afterward and verify that parameter rows outside the two involved classes remain unchanged.

Part 4 - Python code skeleton

```
import torch

def E13(probe_features, weight, bias, sample, true_class, learning_rate):
    scores_before = probe_features @ weight.T + bias
    predicted_class = torch.argmax(weight @ sample + bias)
    weight_after = weight.clone()
    bias_after = bias.clone()
    if predicted_class != true_class:
        weight_after[true_class] += learning_rate * sample
        weight_after[predicted_class] -= learning_rate * sample
        bias_after[true_class] += learning_rate
        bias_after[predicted_class] -= learning_rate
    scores_after = probe_features @ weight_after.T + bias_after
    return scores_after - scores_before, predicted_class

def main():
    print(E13(...))

if __name__ == "__main__":
    main()
```

12.7.15 Exercise 14 - Analyze Misclassification Patterns in Multi-Class Setting

Part 1 - Why this matters

Multi-class errors often form patterns: two visually similar classes may be confused in both directions, while a dominant class may absorb examples from several minorities. Aggregate accuracy cannot expose these structures.

Analysis should combine confusion counts, normalized rates, score gaps, and representative failed samples. Small winning-score gaps indicate ambiguity, whereas large gaps may reveal systematic feature or label problems.

Part 2 - Mathematical form

$$g_i = s_{\hat{y}_i}(\mathbf{x}_i) - s_{y_i}(\mathbf{x}_i), \qquad \mathcal{E}_{ab} = \{i : y_i = a, \hat{y}_i = b\}.$$

Part 3 - Hints and Helper

Rank off-diagonal confusion pairs and then inspect samples with the largest and smallest error gaps for each pair.

Part 4 - Python code skeleton

```
import torch

def E14(scores, labels):
    predictions = torch.argmax(scores, dim=1)
    true_scores = scores.gather(1, labels[:, None]).squeeze(1)
    winning_scores = scores.max(dim=1).values
    gaps = winning_scores - true_scores
    errors = predictions != labels
    pairs = torch.stack((labels[errors], predictions[errors]), dim=1)
    return pairs, gaps[errors], ...

def main():
    print(E14(...))

if __name__ == "__main__":
    main()
```

12.7.16 Exercise 15 - Implement Weight Regularization for Multi-Class Stability

Part 1 - Why this matters

Regularization discourages large class-specific weights and can reduce sensitivity to noisy features. In a manual perceptron loop, weight decay must be inserted explicitly because no optimizer applies it automatically.

Decay changes the classical algorithm and its convergence properties, so it should be treated as an experimental extension. Bias regularization and decay timing must also be stated.

Part 2 - Mathematical form

$$W \leftarrow (1 - \eta\lambda)W + \eta(\mathbf{e}_y - \mathbf{e}_{\hat{y}})\mathbf{x}^\mathsf{T}.$$

Part 3 - Hints and Helper

Apply decay consistently before or after each sample or epoch and document the choice. Compare norms, accuracy, and margins across regularization strengths.

Part 4 - Python code skeleton

```
import torch

def E15(weight, bias, features, label, learning_rate, decay):
    weight = (1 - learning_rate * decay) * weight
    prediction = torch.argmax(weight @ features + bias)
    if prediction != label:
        weight[label] += learning_rate * features
        weight[prediction] -= learning_rate * features
        bias[label] += learning_rate
```

```
        bias[prediction] -= learning_rate
    return weight, bias, prediction

def main():
    print(E15(...))

if __name__ == "__main__":
    main()
```

12.7.17 Exercise 16 - Compare Multi-Class Perceptron with Softmax Regression

Part 1 - Why this matters

Both models use linear class scores and argmax prediction. The perceptron applies mistake-driven paired updates, while softmax regression converts logits to probabilities and minimizes cross-entropy through gradients.

Their decision boundaries can be similar, but their training signals differ. Softmax uses every sample and score, whereas the classical perceptron ignores correctly classified samples regardless of margin size.

Part 2 - Mathematical form

$$p_k = \frac{e^{s_k}}{\sum_j e^{s_j}}, \quad L_{CE} = -\log p_y, \qquad \Delta W_P = \eta(\mathbf{e}_y - \mathbf{e}_{\hat{y}})\mathbf{x}^\mathsf{T}.$$

Part 3 - Hints and Helper

Train both from matched initial scores. Compare accuracy, negative log-likelihood, calibration, margins, update count, and runtime.

Part 4 - Python code skeleton

```
import torch

def E16(features, labels, perceptron_state, softmax_model):
    perceptron_scores = ...
    perceptron_predictions = torch.argmax(perceptron_scores, dim=1)
    softmax_logits = softmax_model(features)
    softmax_probabilities = torch.softmax(softmax_logits, dim=1)
    softmax_loss = torch.nn.functional.cross_entropy(
        softmax_logits, labels
    )
    return {
        "perceptron_accuracy": ...,
        "softmax_accuracy": ...,
        "softmax_loss": softmax_loss.item(),
        "probabilities": softmax_probabilities,
```

```
    }

def main():
    print(E16(...))

if __name__ == "__main__":
    main()
```

12.7.18 Exercise 17 - Stress Test Multi-Class Model with High-Dimensional Inputs

Part 1 - Why this matters

High-dimensional multi-class data expands both the weight matrix and score computation. The stress test reveals memory pressure, throughput limits, non-finite states, and whether shape assumptions survive realistic scale.

Synthetic labels should come from a known teacher matrix so the task remains controlled. Aggregate statistics are safer than saving every large parameter state.

Part 2 - Mathematical form

$$X \in \mathbb{R}^{n \times d}, \quad W \in \mathbb{R}^{K \times d}, \quad S = XW^{\mathsf{T}} \in \mathbb{R}^{n \times K}.$$

Part 3 - Hints and Helper

Generate tensors directly on the target device and synchronize CUDA around timing. Report peak memory, examples per second, updates, and finiteness.

Part 4 - Python code skeleton

```
import torch

def E17(samples, dimensions, classes, device):
    features = torch.randn(samples, dimensions, device=device)
    teacher = torch.randn(classes, dimensions, device=device)
    labels = torch.argmax(features @ teacher.T, dim=1)
    weight = torch.zeros_like(teacher)
    bias = torch.zeros(classes, device=device)
    if device.type == "cuda":
        torch.cuda.reset_peak_memory_stats()
    history = ...
    return {
        "history": history,
        "finite": torch.isfinite(weight).all().item(),
        "peak_memory": ...,
    }

def main():
```

```
    print(E17(...))

if __name__ == "__main__":
    main()
```

12.7.19 Exercise 18 - Debug Class-Specific Training Failures

Part 1 - Why this matters

A model may perform well overall while one class never learns. Common causes include a missing training class, incorrect class-index mapping, imbalance, frozen parameter rows, invalid labels, or an update applied to the wrong rows.

Debugging must inspect state per class. Counts, update frequency, row norms, recall, and score distributions show whether the class lacks data, lacks corrections, or loses every competition.

Part 2 - Mathematical form

$$D_k = \{n_k, U_k, \|\mathbf{w}_k\|_2, \mathrm{Recall}_k, \bar{s}_k\}.$$

Part 3 - Hints and Helper

Validate contiguous class indices or maintain an explicit mapping. Flag classes with zero samples, zero updates, non-finite rows, or persistently zero recall.

Part 4 - Python code skeleton

```
import torch

def E18(labels, updates_per_class, weight, recalls):
    class_count = weight.shape[0]
    counts = torch.bincount(labels, minlength=class_count)
    norms = torch.linalg.vector_norm(weight, dim=1)
    failures = []
    for class_id in range(class_count):
        reasons = []
        if counts[class_id] == 0:
            reasons.append(...)
        if updates_per_class[class_id] == 0:
            reasons.append(...)
        if recalls[class_id] == 0:
            reasons.append(...)
        if reasons:
            failures.append({"class": class_id, "reasons": reasons})
    return counts, norms, failures

def main():
    print(E18(...))
```

```
if __name__ == "__main__":
    main()
```

12.7.20 Exercise 19 - Produce a Comprehensive Multi-Class Analysis Report

Part 1 - Why this matters

A complete report connects formulation, training dynamics, class-specific metrics, resource scaling, and failure analysis. It prevents a single accuracy value from hiding unstable classes, memory costs, or unresolved tie behavior.

Conclusions should distinguish one-vs-rest, one-vs-one, and direct multi-class results. Class mapping, data balance, device, dtype, seeds, update convention, and evaluation averaging are required for reproducibility.

Part 2 - Mathematical form

$$R = \{\text{strategy}, W, \text{updates}, C, \text{per-class metrics}, \text{regions}, \text{scaling}, \text{failures}, \text{limitations}\}.$$

Part 3 - Hints and Helper

Require evidence for every class and every compared strategy. Include confusion matrices, resource measurements, and unresolved limitations.

Part 4 - Python code skeleton

```
import torch

def E19(sections):
    required = {
        "strategy", "class_mapping", "training", "confusion_matrix",
        "per_class_metrics", "decision_regions", "imbalance",
        "cpu_gpu_scaling", "memory", "failure_analysis", "limitations",
    }
    missing = sorted(required - sections.keys())
    failed_classes = sections.get("failure_analysis", {}).get(
        "failed_classes", []
    )
    return {
        "complete": not missing and not failed_classes,
        "missing": missing,
        "failed_classes": failed_classes,
        "sections": sections,
    }

def main():
```

```
    print(E19(...))

if __name__ == "__main__":
    main()
```

12.8 PC06 - Computational Performance and CUDA Effects

Models and Training

12.8.1 Exercise 00 - Benchmark Perceptron Forward Pass on CPU vs GPU

Part 1 - Why this matters

A fair forward-pass benchmark determines whether matrix and vector operations are large enough to benefit from GPU parallelism. Tiny perceptron workloads can run faster on a CPU because GPU launch and transfer costs dominate the arithmetic.

CPU wall-clock timing and asynchronous CUDA execution cannot be compared naively. Both paths need warm-up and repeated measurements, while CUDA timing must use events or explicit synchronization.

Part 2 - Mathematical form

$$T_C = \text{median}(t_C), \qquad T_G = \text{median}(t_G), \qquad S = \frac{T_C}{T_G}.$$

Part 3 - Hints and Helper

Preallocate inputs and parameters on each device. Exclude host-to-device transfer unless end-to-end latency is the metric being studied.

Part 4 - Python code skeleton

```
import time

import torch

def E00(features, weight, bias, repeats):
    cpu_times = benchmark_cpu(features, weight, bias, repeats, ...)
    if not torch.cuda.is_available():
        raise RuntimeError(...)
    gpu_features = features.cuda()
    gpu_weight = weight.cuda()
    gpu_bias = bias.cuda()
    gpu_times = benchmark_cuda(
        gpu_features, gpu_weight, gpu_bias, repeats, ...
```

```
    )
    return summarize(cpu_times, gpu_times, ...)

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

12.8.2 Exercise 01 - Benchmark Training Throughput on CUDA

Part 1 - Why this matters

Training throughput measures how many samples or updates are processed per second under a specified workload. It combines score computation, mistake detection, corrections, and loop overhead rather than timing only one matrix multiplication.

The result depends on update strategy because sequential online updates expose little parallelism, while a batch perceptron can aggregate corrections. The algorithmic variant must be reported beside throughput.

Part 2 - Mathematical form

$$\text{throughput} = \frac{En}{T_{\text{train}}} \text{ samples/s.}$$

Part 3 - Hints and Helper

Warm up the complete training step, synchronize around the measured region, and report batch size, feature count, class count, dtype, and update policy.

Part 4 - Python code skeleton

```
import time

import torch

def E01(features, labels, weight, bias, epochs):
    for _ in range(...):
        weight, bias = training_epoch(features, labels, weight, bias, ...)
    torch.cuda.synchronize()
    start = time.perf_counter()
    for _ in range(epochs):
        weight, bias = training_epoch(features, labels, weight, bias, ...)
    torch.cuda.synchronize()
    elapsed = time.perf_counter() - start
    return {"elapsed": elapsed, "samples_per_second": ...}

def main():
    print(E01(...))
```

```
if __name__ == "__main__":
    main()
```

12.8.3 Exercise 02 - Measure Memory Usage During Training on GPU

Part 1 - Why this matters

GPU memory usage includes persistent parameters, input batches, intermediate scores, temporary masks, allocator reservations, and optional histories. Peak memory can therefore be much larger than the mathematical parameter size.

Allocated and reserved memory describe different quantities. Resetting peak statistics around a controlled step makes before-and-after experiments comparable and helps identify hidden retained tensors.

Part 2 - Mathematical form

$$M_{\text{peak}} = \max_t M_{\text{allocated}}(t), \qquad M_{\text{reserved}} \geq M_{\text{allocated}}.$$

Part 3 - Hints and Helper

Synchronize before reading final statistics and avoid storing graph-connected tensors. Report bytes and convert units only when presenting results.

Part 4 - Python code skeleton

```
import time

import torch

def E02(training_step):
    if not torch.cuda.is_available():
        raise RuntimeError(...)
    torch.cuda.empty_cache()
    torch.cuda.reset_peak_memory_stats()
    before = torch.cuda.memory_allocated()
    output = training_step()
    torch.cuda.synchronize()
    return {
        "before": before,
        "after": torch.cuda.memory_allocated(),
        "peak_allocated": torch.cuda.max_memory_allocated(),
        "peak_reserved": torch.cuda.max_memory_reserved(),
        "output": output,
    }

def main():
```

```
    print(E02(...))

if __name__ == "__main__":
    main()
```

12.8.4 Exercise 03 - Analyze Kernel Launch Overhead for Small Batches

Part 1 - Why this matters

Every CUDA operation launches work through the GPU runtime. For a small batch, launch latency and Python dispatch can exceed the time spent performing the perceptron's dot products.

The experiment should compare many batch sizes and separate a fused or vectorized operation from a sequence of tiny operations. This reveals the workload size at which useful parallel work begins to dominate overhead.

Part 2 - Mathematical form

$$T_G(n) = T_{\text{launch}} + T_{\text{compute}}(n), \qquad n \to 0 \Rightarrow T_{\text{launch}}/T_G \to 1.$$

Part 3 - Hints and Helper

Use CUDA events over many repetitions and divide by the repetition count. Do not call synchronize or item inside the measured inner loop.

Part 4 - Python code skeleton

```
import time

import torch

def E03(batch_sizes, dimensions, repeats):
    rows = []
    for batch_size in batch_sizes:
        features = torch.randn(batch_size, dimensions, device="cuda")
        weight = torch.randn(dimensions, device="cuda")
        elapsed = cuda_event_benchmark(
            lambda: features @ weight, repeats, ...
        )
        rows.append({"batch_size": batch_size, "microseconds": ...})
    return rows

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

12.8.5 Exercise 04 - Evaluate Scaling Behavior with Increasing Batch Size

Part 1 - Why this matters

Batch-size scaling shows how fixed overhead is amortized and how device utilization improves before memory bandwidth or capacity becomes limiting. Latency usually rises while throughput may improve and later plateau.

A benchmark must report both latency and samples per second. Failed allocations and the last complete batch are part of the scaling result rather than values to silently discard.

Part 2 - Mathematical form

$$Q(B) = \frac{B}{T(B)}, \qquad E(B) = \frac{Q(B)}{Q(B_0)}.$$

Part 3 - Hints and Helper

Sweep powers of two, reuse the same feature count, and record peak memory. Catch out-of-memory conditions without continuing from a corrupted measurement state.

Part 4 - Python code skeleton

```
import time

import torch

def E04(batch_sizes, feature_count, repeats):
    results = []
    for batch_size in batch_sizes:
        try:
            features = torch.randn(
                batch_size, feature_count, device="cuda"
            )
            latency = benchmark(...)
            results.append({
                "batch_size": batch_size,
                "latency": latency,
                "throughput": batch_size / latency,
                "peak_memory": ...,
            })
        except torch.OutOfMemoryError:
            results.append({"batch_size": batch_size, "status": "oom"})
            break
    return results

def main():
    print(E04(...))
```

```
if __name__ == "__main__":
    main()
```

12.8.6 Exercise 05 - Compare Float32 vs Mixed Precision Performance

Part 1 - Why this matters

Mixed precision can increase throughput and reduce memory when hardware supports fast lower-precision operations. A perceptron's hard boundary makes it especially important to verify that reduced precision does not change too many decisions near zero.

Performance and correctness must be evaluated together. Speedup without score, prediction, and update comparisons may hide a different training trajectory rather than an equivalent faster computation.

Part 2 - Mathematical form

$$S = \frac{T_{32}}{T_{\mathrm{MP}}}, \qquad D = \sum_i \mathbb{I}[\hat{y}_i^{32} \neq \hat{y}_i^{\mathrm{MP}}].$$

Part 3 - Hints and Helper

Use autocast for the measured operations and keep a float32 reference. Report latency, peak memory, maximum score error, and decision disagreement.

Part 4 - Python code skeleton

```
import time

import torch

def E05(features, weight, bias, repeats):
    float32_result = benchmark_mode(
        features, weight, bias, repeats, autocast_enabled=False
    )
    mixed_result = benchmark_mode(
        features, weight, bias, repeats, autocast_enabled=True
    )
    disagreement = (
        float32_result["predictions"]
        != mixed_result["predictions"]
    ).sum()
    return float32_result, mixed_result, disagreement.item()

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

12.8.7 Exercise 06 - Identify Performance Bottlenecks in Training Loop

Part 1 - Why this matters

A slow training loop may be limited by Python control flow, host-to-device copies, synchronization, memory allocation, score computation, or update scattering. Optimizing the wrong component produces complexity without meaningful speedup.

Bottleneck analysis decomposes time and validates the finding with controlled changes. Percentages should be based on synchronized measurements and add up to the measured iteration time within expected overhead.

Part 2 - Mathematical form

$$T_{\text{iter}} = T_{\text{input}} + T_{\text{forward}} + T_{\text{decision}} + T_{\text{update}} + T_{\text{sync}} + T_{\text{other}}.$$

Part 3 - Hints and Helper

Instrument named regions and compare online and vectorized variants. Check GPU utilization and kernel counts before rewriting the algorithm.

Part 4 - Python code skeleton

```
import time

import torch

def E06(batch):
    timings = {}
    with timed_region("transfer", timings, ...):
        device_batch = move_to_device(batch, ...)
    with timed_region("forward", timings, ...):
        scores = ...
    with timed_region("decision", timings, ...):
        mistakes = ...
    with timed_region("update", timings, ...):
        state = ...
    total = sum(timings.values())
    shares = {name: value / total for name, value in timings.items()}
    return timings, shares

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

12.8.8 Exercise 07 - Profile CUDA Execution Using PyTorch Tools

Part 1 - Why this matters

A profiler exposes CPU operators, CUDA kernels, memory activity, shapes, and call stacks. It turns a broad statement such as GPU training is slow into evidence about the exact operations consuming time.

Profiling adds overhead and should cover a short representative schedule rather than the entire training run. The profiled workload, warm-up steps, and exported trace must be reproducible.

Part 2 - Mathematical form

$$P = \{\text{operator}, \text{CPU time}, \text{CUDA time}, \text{calls}, \text{shapes}, \text{memory}\}.$$

Part 3 - Hints and Helper

Use torch.profiler with CPU and CUDA activities, record shapes, and a wait–warmup–active schedule. Avoid reading only aggregate wall time.

Part 4 - Python code skeleton

```
import time

import torch

def E07(training_step, steps):
    schedule = torch.profiler.schedule(wait=1, warmup=1, active=3)
    with torch.profiler.profile(
        activities=[
            torch.profiler.ProfilerActivity.CPU,
            torch.profiler.ProfilerActivity.CUDA,
        ],
        schedule=schedule,
        record_shapes=True,
        profile_memory=True,
    ) as profiler:
        for _ in range(steps):
            training_step()
            profiler.step()
    return profiler.key_averages().table(...)

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

12.8.9 Exercise 08 - Optimize Data Transfer Between Host and Device

Part 1 - Why this matters

Repeated blocking transfers can starve the GPU and serialize the pipeline. Pinned host memory and non-blocking copies allow transfer to overlap with computation when the hardware and execution stream support it.

Optimization must compare identical end-to-end workloads. Moving data once before training may outperform elaborate overlap when the dataset fits in device memory.

Part 2 - Mathematical form

$$T_{\text{pipeline}} \approx \max(T_{\text{copy}}, T_{\text{compute}}) \text{ with effective overlap.}$$

Part 3 - Hints and Helper

Use a DataLoader with pinned memory and transfer batches with non-blocking enabled. Synchronize only at measurement boundaries and verify output parity.

Part 4 - Python code skeleton

```
import time

import torch

def E08(data_loader, model_step, device):
    processed = 0
    torch.cuda.synchronize()
    start = time.perf_counter()
    for features, labels in data_loader:
        features = features.to(device, non_blocking=True)
        labels = labels.to(device, non_blocking=True)
        model_step(features, labels)
        processed += len(features)
    torch.cuda.synchronize()
    elapsed = time.perf_counter() - start
    return processed / elapsed

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

12.8.10 Exercise 09 - Investigate Synchronization Overhead

Part 1 - Why this matters

CUDA launches are asynchronous with respect to the CPU. Calling synchronize, item, or transferring results to the host forces the CPU to wait and can destroy overlap when done inside a tight loop.

Removing synchronization blindly is also wrong because timing and dependencies may require it. The goal is to place necessary synchronization at clear boundaries and measure its cost.

Part 2 - Mathematical form

$$T_{\text{loop}}^{\text{sync}} - T_{\text{loop}}^{\text{async}} \approx NT_{\text{wait}}.$$

Part 3 - Hints and Helper

Compare a loop with per-step scalar extraction against one that accumulates device tensors and synchronizes once. Ensure both perform equivalent work.

Part 4 - Python code skeleton

```
import time

import torch

def E09(step, iterations):
    synchronized_values = []
    start_sync = time.perf_counter()
    for _ in range(iterations):
        synchronized_values.append(step().item())
    sync_elapsed = time.perf_counter() - start_sync
    start_async = time.perf_counter()
    device_values = [step() for _ in range(iterations)]
    torch.cuda.synchronize()
    async_elapsed = time.perf_counter() - start_async
    return sync_elapsed, async_elapsed, device_values

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

12.8.11 Exercise 10 - Stress Test GPU with Large-Scale Synthetic Data

Part 1 - Why this matters

A stress test explores throughput, peak memory, numerical finiteness, and failure behavior near device limits. Synthetic generation allows sample count, dimensions, classes, and margin

to be controlled.

The test should increase load gradually and stop safely on out-of-memory or non-finite states. A crash is not a useful benchmark result unless the last successful configuration is preserved.

Part 2 - Mathematical form

$$X \in \mathbb{R}^{n \times d}, \qquad \text{forward work} = \mathcal{O}(nd), \qquad \text{input memory} = ndq \text{ bytes}.$$

Part 3 - Hints and Helper

Generate data directly on the GPU or in chunks to avoid measuring transfer unintentionally. Record temperature or throttling context for long runs.

Part 4 - Python code skeleton

```
import time

import torch

def E10(configurations):
    results = []
    for samples, dimensions in configurations:
        try:
            features = torch.randn(samples, dimensions, device="cuda")
            weight = torch.randn(dimensions, device="cuda")
            result = benchmark_and_validate(features, weight, ...)
            results.append(result)
        except torch.OutOfMemoryError:
            results.append({
                "samples": samples,
                "dimensions": dimensions,
                "status": "oom",
            })
            break
    return results

def main():
    print(E10(...))

if __name__ == "__main__":
    main()
```

12.8.12 Exercise 11 - Compare Multi-GPU vs Single-GPU Perceptron Execution

Part 1 - Why this matters

Multiple GPUs can split large batches or datasets, but the perceptron has very few parameters and limited computation per sample. Communication and process coordination can easily cost more than the saved compute time.

The comparison must use the same global batch and algorithmic update. Data-parallel local corrections require an explicit reduction rule so that multi-GPU training remains mathematically comparable.

Part 2 - Mathematical form

$$T_p \approx \frac{T_{\text{compute}}}{p} + T_{\text{communication}} + T_{\text{imbalance}}, \qquad S_p = \frac{T_1}{T_p}.$$

Part 3 - Hints and Helper

Use one process per GPU and aggregate corrections with a collective reduction. Report world size, interconnect, scaling efficiency, and output agreement.

Part 4 - Python code skeleton

```
import time

import torch

def E11(local_features, local_labels, weight, bias):
    if not torch.distributed.is_initialized():
        raise RuntimeError(...)
    local_weight_update, local_bias_update = batch_update(
        local_features, local_labels, weight, bias, ...
    )
    torch.distributed.all_reduce(local_weight_update)
    torch.distributed.all_reduce(local_bias_update)
    world_size = torch.distributed.get_world_size()
    weight = weight + local_weight_update / world_size
    bias = bias + local_bias_update / world_size
    return weight, bias

def main():
    print(E11(...))

if __name__ == "__main__":
    main()
```

12.8.13 Exercise 12 - Measure Determinism Across GPU Runs

Part 1 - Why this matters

Repeated GPU runs may differ when random order, nondeterministic kernels, or uncontrolled initialization changes the training path. Hard threshold decisions can amplify a tiny

numerical difference into a different update sequence.

Determinism must be tested at multiple levels: final parameters, mistake histories, predictions, and bitwise or tolerance-based equality. Performance cost should also be measured when deterministic algorithms are enabled.

Part 2 - Mathematical form

$$D_r = \|\mathbf{w}^{(r)} - \mathbf{w}^{(1)}\|_2, \qquad H_r = \mathbb{I}[\text{history}_r = \text{history}_1].$$

Part 3 - Hints and Helper

Seed Python and PyTorch, enable deterministic algorithms, and run from fresh state. Record environment and library versions with the result.

Part 4 - Python code skeleton

```
import time

import torch

def E12(run_training, seed, repeats):
    outputs = []
    torch.use_deterministic_algorithms(True)
    for _ in range(repeats):
        torch.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
        outputs.append(run_training())
    reference = outputs[0]
    comparisons = [compare_runs(reference, output, ...) for output in outputs]
    return outputs, comparisons

def main():
    print(E12(...))

if __name__ == "__main__":
    main()
```

12.8.14 Exercise 13 - Analyze Precision-Induced Performance Variations

Part 1 - Why this matters

Dtype changes arithmetic throughput, tensor-core eligibility, memory traffic, and capacity. The fastest dtype depends on hardware, shapes, operator support, and whether casts introduce extra kernels.

A precision benchmark is incomplete without numerical analysis. Score error, boundary disagreement, convergence changes, and non-finite values determine whether a speedup is usable.

Part 2 - Mathematical form

$$R_q = (T_q, M_q, \epsilon_q, D_q), \qquad q \in \{16, \text{bf16}, 32, 64\}.$$

Part 3 - Hints and Helper

Run supported dtypes over the same base data and warm-up each mode independently. Report unsupported modes instead of silently substituting another dtype.

Part 4 - Python code skeleton

```
import time

import torch

def E13(features, weight, dtypes, repeats):
    reference = features.float() @ weight.float()
    results = {}
    for dtype in dtypes:
        try:
            output, latency, memory = benchmark_dtype(
                features, weight, dtype, repeats, ...
            )
            results[str(dtype)] = {
                "latency": latency,
                "memory": memory,
                "maximum_error": (
                    output.float() - reference
                ).abs().max().item(),
                "finite": torch.isfinite(output).all().item(),
            }
        except RuntimeError as error:
            results[str(dtype)] = {
                "status": "unsupported",
                "error": str(error),
            }
    return results

def main():
    print(E13(...))

if __name__ == "__main__":
    main()
```

12.8.15 Exercise 14 - Benchmark Perceptron vs Logistic Regression Performance

Part 1 - Why this matters

Both models compute a linear score, but logistic regression adds a differentiable loss, backward pass, gradient storage, and optimizer step. The perceptron's explicit correction may be cheaper while providing different statistical behavior.

The comparison must define the same data, batch size, device, dtype, and stopping budget. Accuracy and convergence quality belong beside runtime because unequal work can produce a misleading speed claim.

Part 2 - Mathematical form

$$C_P \approx C_{\text{forward}} + C_{\text{update}}, \qquad C_L \approx C_{\text{forward}} + C_{\text{loss}} + C_{\text{backward}} + C_{\text{optimizer}}.$$

Part 3 - Hints and Helper

Benchmark forward-only and full-training paths separately. Report throughput, peak memory, accuracy, and total time to the chosen target metric.

Part 4 - Python code skeleton

```
import time

import torch

def E14(perceptron_step, logistic_step, evaluation, epochs):
    perceptron_result = benchmark_training(
        perceptron_step, evaluation, epochs, ...
    )
    logistic_result = benchmark_training(
        logistic_step, evaluation, epochs, ...
    )
    return {
        "perceptron": perceptron_result,
        "logistic_regression": logistic_result,
    }

def main():
    print(E14(...))

if __name__ == "__main__":
    main()
```

12.8.16 Exercise 15 - Identify When GPU Usage Is Counterproductive

Part 1 - Why this matters

A GPU is counterproductive when launch, transfer, synchronization, or setup costs exceed the parallel compute benefit. This often occurs for tiny batches, few features, sequential online

updates, or infrequent inference.

The break-even point is workload- and machine-specific. It should be measured across shapes and include both compute-only and end-to-end latency rather than declared from one example.

Part 2 - Mathematical form

$$\text{GPU counterproductive} \iff T_G^{\text{end-to-end}} > T_C^{\text{end-to-end}}.$$

Part 3 - Hints and Helper

Sweep batch and feature sizes, include transfer time, and find the smallest workload where the GPU wins consistently across repeated trials.

Part 4 - Python code skeleton

```
import time

import torch

def E15(workloads, repeats):
    rows = []
    for workload in workloads:
        cpu_latency = benchmark_cpu(workload, repeats, ...)
        gpu_compute = benchmark_gpu(workload, repeats, include_transfer=False)
        gpu_end_to_end = benchmark_gpu(
            workload, repeats, include_transfer=True
        )
        rows.append({
            "workload": workload,
            "cpu": cpu_latency,
            "gpu_compute": gpu_compute,
            "gpu_end_to_end": gpu_end_to_end,
            "gpu_counterproductive": gpu_end_to_end > cpu_latency,
        })
    return rows

def main():
    print(E15(...))

if __name__ == "__main__":
    main()
```

12.8.17 Exercise 16 - Evaluate Energy Efficiency of GPU Execution

Part 1 - Why this matters

Energy efficiency considers work completed per joule rather than speed alone. A faster GPU run can consume less total energy if it finishes quickly, or more energy if utilization is poor and

power remains high.

Reliable energy measurement requires sampled device power or an external energy counter over the same synchronized workload. Idle baseline, sampling interval, and measurement uncertainty must be recorded.

Part 2 - Mathematical form

$$E \approx \sum_{j=1}^{m} P_j \Delta t_j, \qquad \eta_E = \frac{N_{\text{samples}}}{E} \text{ samples/J}.$$

Part 3 - Hints and Helper

Sample power from Python while the workload runs, subtract idle power only if the chosen methodology requires it, and align timestamps with synchronized start and end boundaries.

Part 4 - Python code skeleton

```
import time

import torch

def E16(run_workload, power_sampler, interval):
    samples = []
    start = time.perf_counter()
    while workload_is_running(...):
        samples.append((time.perf_counter(), power_sampler()))
        time.sleep(interval)
    processed = run_workload.result(...)
    energy_joules = integrate_power(samples, start, ...)
    return {
        "energy_joules": energy_joules,
        "samples_per_joule": processed / energy_joules,
        "power_samples": samples,
    }

def main():
    print(E16(...))

if __name__ == "__main__":
    main()
```

12.8.18 Exercise 17 - Optimize Training Loop for Throughput

Part 1 - Why this matters

Throughput optimization removes Python work, repeated allocation, unnecessary transfers, and frequent synchronization from the critical path. Vectorized batch corrections can expose more parallelism than online sample-by-sample updates.

Changing the update schedule can change the algorithm, so every optimization must preserve a declared reference or report the behavioral difference. Correctness gates should precede speed comparisons.

Part 2 - Mathematical form

$$\Delta \mathbf{w} = \eta \sum_{i \in \mathcal{M}} y_i \mathbf{x}_i, \qquad \Delta b = \eta \sum_{i \in \mathcal{M}} y_i.$$

Part 3 - Hints and Helper

Profile first, vectorize active-sample aggregation, reuse buffers, and consider compilation only after shape stability is established. Compare predictions and update counts with the reference.

Part 4 - Python code skeleton

```
import time

import torch

def E17(features, labels, weight, bias, learning_rate):
    scores = features @ weight + bias
    margins = labels * scores
    active = margins <= 0
    active_features = features[active]
    active_labels = labels[active]
    weight_update = learning_rate * (
        active_labels[:, None] * active_features
    ).sum(dim=0)
    bias_update = learning_rate * active_labels.sum()
    return weight + weight_update, bias + bias_update, active.sum()

def main():
    print(E17(...))

if __name__ == "__main__":
    main()
```

12.8.19 Exercise 18 - Detect Performance Regression After Code Changes

Part 1 - Why this matters

A performance regression occurs when a code change makes a stable workload slower, more memory-intensive, or less efficient beyond expected measurement noise. Automated benchmarks can catch these changes before they accumulate.

A single timing comparison is unreliable. Baselines need repeated samples, controlled environments, robust statistics, and thresholds that distinguish meaningful degradation from ordinary variance.

Part 2 - Mathematical form

$$r = \frac{\tilde{T}_{\text{new}} - \tilde{T}_{\text{base}}}{\tilde{T}_{\text{base}}}, \qquad \text{regression} \iff r > \tau.$$

Part 3 - Hints and Helper

Store workload metadata with the baseline and compare medians or confidence intervals. Check correctness before accepting a faster result.

Part 4 - Python code skeleton

```
import time

import torch

def E18(baseline_times, candidate_times, threshold):
    baseline = torch.tensor(baseline_times, dtype=torch.float64)
    candidate = torch.tensor(candidate_times, dtype=torch.float64)
    baseline_median = baseline.median()
    candidate_median = candidate.median()
    relative_change = (
        candidate_median - baseline_median
    ) / baseline_median
    return {
        "baseline_median": baseline_median.item(),
        "candidate_median": candidate_median.item(),
        "relative_change": relative_change.item(),
        "regression": relative_change > threshold,
    }

def main():
    print(E18(...))

if __name__ == "__main__":
    main()
```

12.8.20 Exercise 19 - Produce a Performance Optimization Report

Part 1 - Why this matters

A performance report converts measurements into an optimization decision. It connects workload shapes, methodology, profiler evidence, latency, throughput, memory, precision, scaling, energy, and correctness.

The report must distinguish observed facts from recommendations and preserve raw measurements. An optimization is accepted only when it improves the target metric without violating numerical or model-quality tolerances.

Part 2 - Mathematical form

$$R = \{\text{method}, \text{hardware}, T, Q, M, P, E, \text{precision}, \text{parity}, \text{bottlenecks}, \text{actions}\}.$$

Part 3 - Hints and Helper

Require hardware and software context, synchronization policy, warm-up, repetitions, and statistical summary. Rank recommendations by measured impact and implementation risk.

Part 4 - Python code skeleton

```
import time

import torch

def E19(sections):
    required = {
        "methodology", "hardware", "software", "latency",
        "throughput", "memory", "precision", "cpu_gpu_parity",
        "profiling", "scaling", "energy", "regressions",
        "recommendations",
    }
    missing = sorted(required - sections.keys())
    failed_checks = sections.get("cpu_gpu_parity", {}).get(
        "failed_checks", []
    )
    return {
        "complete": not missing and not failed_checks,
        "missing": missing,
        "failed_checks": failed_checks,
        "sections": sections,
    }

def main():
    print(E19(...))

if __name__ == "__main__":
    main()
```

12.9 PC07 - Generalization and Theoretical Limits

Models and Training

12.9.1 Exercise 00 - Analyze Generalization Capability of the Perceptron Model

Part 1 - Why this matters

Generalization is the ability to classify unseen samples drawn from the target population, not merely to separate the training set. A perceptron can achieve zero training error while selecting a boundary that performs poorly outside the observed examples.

Capacity and sample size provide theoretical context. Affine linear separators in a fixed feature space have limited capacity, but high dimension, weak margin, biased sampling, and distribution mismatch can still produce a large generalization gap.

Part 2 - Mathematical form

$$d_{\mathrm{VC}} = d + 1, \qquad G = E_{\mathrm{test}} - E_{\mathrm{train}}.$$

Part 3 - Hints and Helper

Use an untouched test split and report feature dimension, sample count, normalized margin, train error, test error, and their gap.

Part 4 - Python code skeleton

```
import torch

def E00(model, train_data, test_data):
    train_features, train_labels = train_data
    test_features, test_labels = test_data
    train_error = classification_error(model, train_features, train_labels)
    test_error = classification_error(model, test_features, test_labels)
    return {
        "train_error": train_error,
        "test_error": test_error,
        "generalization_gap": test_error - train_error,
        "feature_dimension": train_features.shape[1],
    }

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

12.9.2 Exercise 01 - Study the Impact of Margin on Generalization Performance

Part 1 - Why this matters

The normalized margin measures the distance of the closest labeled training sample from the decision boundary. A larger margin often supports robustness and tighter capacity bounds than a separator that barely classifies every sample correctly.

Raw scores are not geometric margins because multiplying weights and bias changes their magnitude without moving the boundary. Every comparison must normalize by the weight norm and control data radius.

Part 2 - Mathematical form

$$\gamma = \min_i \frac{y_i(\mathbf{w}^\mathsf{T}\mathbf{x}_i + b)}{\|\mathbf{w}\|_2}, \qquad \text{complexity depends on } R^2/\gamma^2.$$

Part 3 - Hints and Helper

Train multiple separators or orderings, pair each normalized margin with test error, and report correlation without claiming that the experiment proves causation.

Part 4 - Python code skeleton

```
import torch

def E01(models, train_data, test_data):
    rows = []
    train_features, train_labels = train_data
    for weight, bias in models:
        norm = torch.linalg.vector_norm(weight).clamp_min(1e-12)
        margins = train_labels * (train_features @ weight + bias) / norm
        test_error = classification_error(
            (weight, bias), test_data[0], test_data[1]
        )
        rows.append((margins.min().item(), test_error))
    return rows, ...

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

12.9.3 Exercise 02 - Compare Training Accuracy vs Test Accuracy

Part 1 - Why this matters

Training accuracy measures fit to observed examples, while test accuracy estimates performance on unseen data. Their difference reveals possible overfitting, underfitting, leakage, or distribution mismatch.

A valid comparison requires identical preprocessing learned only from training data and a test set that was not used for model selection. Confidence intervals or repeated splits make the estimate more informative.

Part 2 - Mathematical form

$$A_{\text{train}} = \frac{1}{n}\sum_{i} \mathbb{I}[\hat{y}_i = y_i], \qquad A_{\text{test}} = \frac{1}{m}\sum_{j} \mathbb{I}[\hat{y}_j = y_j].$$

Part 3 - Hints and Helper

Evaluate in one function with no parameter updates. Check class balance and report both accuracy and class-specific metrics when the distribution is imbalanced.

Part 4 - Python code skeleton

```
import torch

def E02(weight, bias, train_data, test_data):
    metrics = {}
    for name, (features, labels) in {
        "train": train_data, "test": test_data
    }.items():
        predictions = torch.where(features @ weight + bias >= 0, 1, -1)
        metrics[name] = {
            "accuracy": (predictions == labels).float().mean().item(),
            "sample_count": len(labels),
        }
    metrics["gap"] = (
        metrics["train"]["accuracy"]
        - metrics["test"]["accuracy"]
    )
    return metrics

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

12.9.4 Exercise 03 - Investigate Overfitting in Linearly Separable Data

Part 1 - Why this matters

Linear separability guarantees that some boundary fits the training labels, but it does not guarantee that the selected boundary reflects the population. Sparse samples, outliers, and mislabeled points can steer the perceptron toward a fragile separator.

Because the standard perceptron stops changing after zero training errors, overfitting is investigated across datasets, orderings, or retained states rather than by assuming that endless zero-loss epochs continue to fit the data.

Part 2 - Mathematical form

$$E_{\text{train}} = 0 \not\Rightarrow E_{\text{population}} = 0.$$

Part 3 - Hints and Helper

Vary training size and outlier placement while using a fixed population test set. Compare margins, test gaps, and boundary variability.

Part 4 - Python code skeleton

```
import torch

def E03(training_sets, fixed_test):
    report = []
    for name, train_data in training_sets.items():
        weight, bias, history = train_perceptron(train_data, ...)
        report.append({
            "name": name,
            "train_error": ...,
            "test_error": ...,
            "margin": ...,
            "updates": sum(history),
        })
    return report

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

12.9.5 Exercise 04 - Evaluate Robustness to Input Noise

Part 1 - Why this matters

Input noise perturbs a sample's score by the projection of the disturbance onto the weight vector. Samples with small normalized margin can change class under much weaker noise than samples far from the boundary.

Robustness curves should use multiple noise magnitudes and random trials. Average accuracy alone can hide a vulnerable subgroup, so flip rates should also be reported by original margin range.

Part 2 - Mathematical form

$$f(\mathbf{x}+\boldsymbol{\delta}) = f(\mathbf{x}) + \mathbf{w}^{\mathsf{T}}\boldsymbol{\delta}, \qquad |\mathbf{w}^{\mathsf{T}}\boldsymbol{\delta}| < |f(\mathbf{x})| \Rightarrow \text{stable}.$$

Part 3 - Hints and Helper

Generate noise with controlled norm and keep one clean prediction reference. Report prediction flips, label accuracy, and results grouped by clean margin.

Part 4 - Python code skeleton

```
import torch

def E04(features, labels, weight, bias, noise_levels, trials):
    clean_predictions = torch.where(features @ weight + bias >= 0, 1, -1)
    rows = []
    for level in noise_levels:
        flip_rates = []
        accuracies = []
        for _ in range(trials):
            noisy = features + level * torch.randn_like(features)
            predictions = torch.where(noisy @ weight + bias >= 0, 1, -1)
            flip_rates.append(
                (predictions != clean_predictions).float().mean()
            )
            accuracies.append((predictions == labels).float().mean())
        rows.append({"level": level, "flip_rate": ..., "accuracy": ...})
    return rows

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

12.9.6 Exercise 05 - Analyze Sensitivity to Label Noise

Part 1 - Why this matters

Incorrect labels create contradictory corrections and can destroy linear separability. A single confidently mislabeled point may repeatedly pull the boundary away from a solution that generalizes well on clean data.

Sensitivity should be measured across controlled noise rates and seeds. Clean test performance, update count, convergence status, and parameter drift reveal damage hidden by noisy training accuracy.

Part 2 - Mathematical form

$$\tilde{y}_i = \begin{cases} -y_i, & i \in \mathcal{N}, \\ y_i, & i \notin \mathcal{N}, \end{cases} \qquad |\mathcal{N}|/n = \rho.$$

Part 3 - Hints and Helper

Save flipped indices and begin every run from identical parameters. Evaluate on untouched labels and compare weight direction with the clean reference.

Part 4 - Python code skeleton

```
import torch

def E05(features, labels, noise_rates, seed):
    generator = torch.Generator().manual_seed(seed)
    report = {}
    for rate in noise_rates:
        noisy_labels = labels.clone()
        count = int(rate * len(labels))
        indices = torch.randperm(len(labels), generator=generator)[:count]
        noisy_labels[indices] *= -1
        state, history = train_perceptron(
            (features, noisy_labels), ...
        )
        report[rate] = {
            "state": state,
            "updates": sum(history),
            "clean_error": ...,
        }
    return report

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

12.9.7 Exercise 06 - Explore the Effect of Regularization on Generalization

Part 1 - Why this matters

Weight regularization discourages large parameter norms and may reduce sensitivity to noisy or weak features. In a perceptron, decay is an extension to the classical update rather than an automatic property of the algorithm.

Smaller norms alone do not guarantee a larger geometric margin because scores and fit also change. Regularization strength must be selected on validation data and evaluated on an untouched test set.

Part 2 - Mathematical form

$$W \leftarrow (1 - \eta\lambda)W + \Delta W_P, \qquad J = L_P + \frac{\lambda}{2}\|W\|_2^2.$$

Part 3 - Hints and Helper

Sweep regularization values with fixed seeds. Report train error, validation error, test error, norm, and normalized margin.

Part 4 - Python code skeleton

```
import torch

def E06(train_data, validation_data, strengths):
    rows = []
    for strength in strengths:
        weight, bias = train_regularized_perceptron(
            train_data, strength, ...
        )
        rows.append({
            "strength": strength,
            "weight_norm": torch.linalg.vector_norm(weight).item(),
            "validation_error": classification_error(
                (weight, bias), *validation_data
            ),
            "test_error": ...,
        })
    return rows

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

12.9.8 Exercise 07 - Compare Perceptron Generalization to Logistic Regression

Part 1 - Why this matters

Both classifiers use a linear score, but logistic regression optimizes probabilistic log loss while the perceptron updates only on mistakes. Their boundaries, calibration, noise response, and regularization behavior can therefore differ.

A fair comparison uses identical splits, preprocessing, feature representation, and tuning budget. Accuracy should be accompanied by log loss or calibration only for the probabilistic model and by margin or update statistics for the perceptron.

Part 2 - Mathematical form

$$\hat{y}_P = \operatorname{sign}(\mathbf{w}_P^\top \mathbf{x} + b_P), \qquad p_L(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}_L^\top \mathbf{x} + b_L).$$

Part 3 - Hints and Helper

Train both across repeated seeds and report mean and variance of test metrics. Do not interpret perceptron scores as probabilities.

Part 4 - Python code skeleton

```
import torch

def E07(perceptron, logistic, test_data):
    features, labels = test_data
    perceptron_predictions = perceptron.predict(features)
    logistic_logits = logistic(features).squeeze(-1)
    logistic_predictions = torch.where(logistic_logits >= 0, 1, -1)
    return {
        "perceptron_accuracy": ...,
        "logistic_accuracy": ...,
        "logistic_log_loss": ...,
        "perceptron_margin": ...,
    }

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

12.9.9 Exercise 08 - Study Bias-Variance Tradeoff in Simple Models

Part 1 - Why this matters

Bias describes systematic error from an overly restricted hypothesis class, while variance describes sensitivity to the sampled training data. A linear perceptron has high bias on non-linear structure but can still show substantial variance in sparse high-dimensional settings.

Classification bias and variance do not have one universal decomposition like squared regression error. This exercise uses repeated datasets and prediction disagreement as empirical proxies and states that limitation.

Part 2 - Mathematical form

$$\mathrm{Var}_{\mathcal{D}}[\hat{y}(\mathbf{x})] \approx 1 - \max_{c} \Pr_{\mathcal{D}}(\hat{y}(\mathbf{x}) = c).$$

Part 3 - Hints and Helper

Train on many resampled datasets and predict one fixed evaluation grid. Measure consensus, disagreement, and systematic error against known target labels.

Part 4 - Python code skeleton

```
import torch

def E08(training_sets, evaluation_features, evaluation_labels):
    predictions = []
    for training_set in training_sets:
        model = train_perceptron(training_set, ...)
        predictions.append(model.predict(evaluation_features))
    matrix = torch.stack(predictions)
    positive_rate = (matrix == 1).float().mean(dim=0)
    consensus = torch.where(positive_rate >= 0.5, 1, -1)
    disagreement = 1 - torch.maximum(positive_rate, 1 - positive_rate)
    systematic_error = (consensus != evaluation_labels).float().mean()
    return systematic_error, disagreement

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

12.9.10 Exercise 09 - Identify Scenarios Where Perceptron Generalization Fails

Part 1 - Why this matters

Generalization can fail when classes are nonlinear, labels are noisy, features omit causal information, dimension is large relative to sample count, classes are imbalanced, or deployment data differ from training data.

These failures belong to different layers: representation, data, objective, optimization, or deployment. A useful diagnosis maps evidence to the correct layer instead of treating every low test score as the same problem.

Part 2 - Mathematical form

$$F = \{\text{nonlinearity}, \text{noise}, \text{sampling}, \text{dimension}, \text{imbalance}, \text{shift}\}.$$

Part 3 - Hints and Helper

Construct controlled datasets where only one assumption changes. Record train fit, test gap, margin, and the evidence supporting each failure label.

Part 4 - Python code skeleton

```
import torch

def E09(scenarios):
    results = []
    for scenario in scenarios:
```

```
        model = train_perceptron(scenario["train"], ...)
        metrics = evaluate(model, scenario["train"], scenario["test"])
        results.append({
            "name": scenario["name"],
            "expected_failure": scenario["failure_type"],
            "metrics": metrics,
            "evidence": ...,
        })
    return results

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

12.9.11 Exercise 10 - Evaluate Generalization on High-Dimensional Sparse Data

Part 1 - Why this matters

Sparse data can contain far more features than samples, allowing a linear classifier to fit training labels with weak evidence for unseen cases. Efficient storage helps computation but does not remove statistical overfitting risk.

Evaluation should vary dimension, density, sample count, and regularization. Dense conversion may erase the computational advantage and exceed memory, so representation costs belong in the result.

Part 2 - Mathematical form

$$X \in \mathbb{R}^{n \times d}, \qquad \text{density} = \frac{\text{nnz}(X)}{nd}, \qquad d \gg n.$$

Part 3 - Hints and Helper

Generate sparse tensors with controlled informative features. Report train and test error, nonzero count, memory, update time, and weight sparsity.

Part 4 - Python code skeleton

```
import torch

def E10(sparse_features, labels, weight, bias):
    if sparse_features.layout != torch.sparse_coo:
        raise ValueError(...)
    scores = torch.sparse.mm(
        sparse_features, weight[:, None]
    ).squeeze(1) + bias
```

```
    predictions = torch.where(scores >= 0, 1, -1)
    return {
        "accuracy": (predictions == labels).float().mean().item(),
        "nonzero_values": sparse_features._nnz(),
        "density": sparse_features._nnz() / sparse_features.numel(),
    }

def main():
    print(E10(...))

if __name__ == "__main__":
    main()
```

12.9.12 Exercise 11 - Measure Stability Across Different Random Seeds

Part 1 - Why this matters

Random initialization, sample order, split generation, and noise can produce different perceptron boundaries. Stable generalization requires conclusions that survive these sources of randomness rather than one favorable seed.

Seed analysis should preserve both aggregate metrics and individual runs. Mean performance can hide a catastrophic tail, while parameter variation can be large even when predictions remain similar.

Part 2 - Mathematical form

$$\bar{A} = \frac{1}{q}\sum_{r=1}^{q} A_r, \qquad s_A^2 = \frac{1}{q-1}\sum_{r}(A_r - \bar{A})^2.$$

Part 3 - Hints and Helper

Run the full pipeline for each seed, including splitting if that randomness is part of the study. Report mean, standard deviation, range, and worst run.

Part 4 - Python code skeleton

```
import torch

def E11(run_experiment, seeds):
    rows = [run_experiment(seed) for seed in seeds]
    accuracies = torch.tensor(
        [row["test_accuracy"] for row in rows], dtype=torch.float64
    )
    return {
        "runs": rows,
        "mean": accuracies.mean().item(),
        "std": accuracies.std(unbiased=True).item(),
```

```
        "minimum": accuracies.min().item(),
        "maximum": accuracies.max().item(),
    }

def main():
    print(E11(...))

if __name__ == "__main__":
    main()
```

12.9.13 Exercise 12 - Compare Deterministic vs Stochastic Training Outcomes

Part 1 - Why this matters

A deterministic order produces one reproducible trajectory, while stochastic shuffling explores different mistake sequences. Both can converge on separable data but may select different valid separators with different margins and test behavior.

The comparison concerns outcomes as well as speed. Update count, parameter similarity, prediction disagreement, margin, and test error reveal whether randomness changes only the path or also the deployed behavior.

Part 2 - Mathematical form

$$\pi_e = \pi_0 \text{ deterministic}, \qquad \pi_e \sim \text{Uniform}(S_n) \text{ stochastic}.$$

Part 3 - Hints and Helper

Keep initialization and stopping rules fixed. Repeat stochastic runs and compare their distribution with the single deterministic reference.

Part 4 - Python code skeleton

```
import torch

def E12(train_data, deterministic_order, seeds):
    deterministic = train_with_order(
        train_data, deterministic_order, ...
    )
    stochastic = []
    for seed in seeds:
        generator = torch.Generator().manual_seed(seed)
        stochastic.append(train_with_shuffle(train_data, generator, ...))
    return compare_outcomes(deterministic, stochastic, ...)

def main():
    print(E12(...))

if __name__ == "__main__":
    main()
```

12.9.14 Exercise 13 - Analyze Generalization Under Distribution Shift

Part 1 - Why this matters

Distribution shift occurs when deployment inputs, label frequencies, or relationships differ from training conditions. A boundary that generalized well in-distribution may fail even though its code and training metrics remain unchanged.

Shift analysis must define what changed. Covariate shift, label shift, and concept shift require different evidence and cannot be summarized by one accuracy drop alone.

Part 2 - Mathematical form

$$P_{\text{train}}(X, Y) \neq P_{\text{test}}(X, Y), \qquad \Delta A = A_{\text{ID}} - A_{\text{shift}}.$$

Part 3 - Hints and Helper

Create controlled shifted sets and evaluate them separately. Compare feature statistics, class priors, error rates, margins, and calibration only where applicable.

Part 4 - Python code skeleton

```
import torch

def E13(model, in_distribution, shifted_sets):
    baseline = evaluate(model, in_distribution)
    report = {"in_distribution": baseline}
    for shift_name, dataset in shifted_sets.items():
        metrics = evaluate(model, dataset)
        report[shift_name] = {
            "metrics": metrics,
            "accuracy_drop": baseline["accuracy"] - metrics["accuracy"],
            "feature_shift": ...,
            "class_prior_shift": ...,
        }
    return report

def main():
    print(E13(...))

if __name__ == "__main__":
    main()
```

12.9.15 Exercise 14 - Investigate Adversarial-Like Perturbations

Part 1 - Why this matters

For a linear classifier, the shortest direction toward the boundary is aligned with the weight vector. This makes a minimal boundary-crossing perturbation analytically understandable

rather than dependent on a complex attack algorithm.

The experiment should distinguish a perturbation that reaches the boundary from one that crosses it by a chosen safety amount. Feature constraints and semantic validity still matter in real data.

Part 2 - Mathematical form

$$\boldsymbol{\delta}^* = -\frac{\mathbf{w}^\mathsf{T}\mathbf{x} + b}{\|\mathbf{w}\|_2^2}\mathbf{w}, \qquad \|\boldsymbol{\delta}^*\|_2 = \frac{|f(\mathbf{x})|}{\|\mathbf{w}\|_2}.$$

Part 3 - Hints and Helper

Compute the analytic perturbation, verify the new score is near zero, then add a small step across the boundary and check the prediction flip.

Part 4 - Python code skeleton

```
import torch

def E14(features, weight, bias, crossing_epsilon):
    scores = features @ weight + bias
    norm_squared = torch.dot(weight, weight).clamp_min(1e-12)
    boundary_delta = -(scores / norm_squared)[:, None] * weight
    direction = -torch.sign(scores)[:, None] * weight
    direction = direction / torch.linalg.vector_norm(weight)
    adversarial = features + boundary_delta + crossing_epsilon * direction
    new_scores = adversarial @ weight + bias
    return boundary_delta, adversarial, new_scores

def main():
    print(E14(...))

if __name__ == "__main__":
    main()
```

12.9.16 Exercise 15 - Produce Empirical Evidence Supporting Theoretical Claims

Part 1 - Why this matters

Empirical evidence translates a theoretical statement into verified assumptions, measurable predictions, and observed results. It can illustrate a theorem and test an implementation, but finite experiments do not replace a proof.

Every claim should include a negative control where an assumption is violated. This shows whether the experiment can distinguish theorem-supported behavior from cases outside its scope.

Part 2 - Mathematical form

$$V_j = (A_j, P_j, O_j, \tau_j), \qquad \text{supported}_j = \mathbb{I}[A_j \wedge d(P_j, O_j) \leq \tau_j].$$

Part 3 - Hints and Helper

Build a structured table for margin robustness, convergence, capacity, noise, and shift. Preserve raw measurements and tolerances.

Part 4 - Python code skeleton

```
import torch

def E15(claims):
    results = []
    for claim in claims:
        assumptions_hold = all(check() for check in claim["assumptions"])
        observation = claim["experiment"]()
        supported = (
            claim["compare"](observation, claim["tolerance"])
            if assumptions_hold else None
        )
        results.append({
            "claim": claim["name"],
            "assumptions_hold": assumptions_hold,
            "observation": observation,
            "supported": supported,
        })
    return results

def main():
    print(E15(...))

if __name__ == "__main__":
    main()
```

12.9.17 Exercise 16 - Document Limitations of Perceptron in Real-World Data

Part 1 - Why this matters

Real-world data can contain nonlinear structure, missing values, changing distributions, noisy labels, unequal costs, correlated groups, and ambiguous classes. A single hard linear boundary cannot represent or communicate all of these conditions.

Limitations must be tied to consequences and evidence. The report should separate the perceptron's representational limits from failures caused by poor preprocessing, evaluation, monitoring, or data governance.

Part 2 - Mathematical form

$$L = \{\text{linearity}, \text{hard output}, \text{noise}, \text{imbalance}, \text{missingness}, \text{shift}, \text{uncertainty}\}.$$

Part 3 - Hints and Helper

Create a limitation registry with a detection test, observed impact, mitigation, residual risk, and responsible monitoring metric.

Part 4 - Python code skeleton

```
import torch

def E16(observations):
    limitations = []
    for name, evidence in observations.items():
        limitations.append({
            "name": name,
            "evidence": evidence,
            "impact": ...,
            "mitigation": ...,
            "residual_risk": ...,
        })
    return limitations

def main():
    print(E16(...))

if __name__ == "__main__":
    main()
```

12.9.18 Exercise 17 - Compare Perceptron to Modern Linear Classifiers

Part 1 - Why this matters

Modern linear classifiers retain efficient linear prediction while using objectives and regularization designed for margins, probabilities, noisy data, or large-scale stochastic optimization. The perceptron remains a valuable baseline and conceptual model.

A useful comparison includes logistic regression, linear support vector machines, and averaged or regularized perceptron variants. Each should receive a fair tuning budget and identical data processing.

Part 2 - Mathematical form

$$\text{objectives: } L_P = \max(0, -m), \quad L_H = \max(0, 1 - m), \quad L_L = \log(1 + e^{-m}).$$

Part 3 - Hints and Helper

Compare predictive quality, calibration where defined, margin, runtime, memory, robustness, and interpretability. Do not rank models by accuracy alone.

Part 4 - Python code skeleton

```
import torch

def E17(models, datasets):
    report = {}
    for name, model in models.items():
        trained = model.fit(datasets["train"], ...)
        report[name] = {
            "validation": evaluate(trained, datasets["validation"]),
            "test": evaluate(trained, datasets["test"]),
            "runtime": ...,
            "memory": ...,
            "supports_probability": ...,
        }
    return report

def main():
    print(E17(...))

if __name__ == "__main__":
    main()
```

12.9.19 Exercise 18 - Evaluate Suitability of Perceptron for Production Use

Part 1 - Why this matters

Production suitability depends on more than test accuracy. Latency, throughput, memory, stability, explainability, calibration needs, class costs, drift monitoring, update safety, and rollback behavior all influence the decision.

A perceptron may be appropriate for a simple low-latency linearly separable task or as a baseline. It is unsuitable when probabilistic confidence, nonlinear structure, robust noise handling, or complex class interactions are required.

Part 2 - Mathematical form

$$S = \sum_j \omega_j q_j, \qquad \text{deploy only if every critical gate } g_j = 1.$$

Part 3 - Hints and Helper

Define critical gates separately from weighted preferences. Include shadow evaluation, drift thresholds, failure fallback, and model-version reproducibility.

Part 4 - Python code skeleton

```
import torch

def E18(evidence, requirements):
    gates = {
        name: check(evidence, rule)
        for name, rule in requirements["critical_gates"].items()
    }
    scores = {
        name: score(evidence, rule)
        for name, rule in requirements["weighted_criteria"].items()
    }
    weighted_score = sum(
        requirements["weights"][name] * value
        for name, value in scores.items()
    )
    return {
        "deploy": all(gates.values()),
        "gates": gates,
        "score": weighted_score,
    }

def main():
    print(E18(...))

if __name__ == "__main__":
    main()
```

12.9.20 Exercise 19 - Write a Critical Theoretical Review of the Perceptron

Part 1 - Why this matters

A critical review should explain why the perceptron is historically and mathematically important while evaluating its assumptions, guarantees, capacity, generalization behavior, and failure modes without exaggeration.

The review must distinguish proven statements from empirical findings and practical judgments. It should state when later linear methods solve a limitation and when only a richer representation or nonlinear model can do so.

Part 2 - Mathematical form

$$R = \{\text{model}, \text{algorithm}, \text{convergence}, \text{capacity}, \text{margin}, \text{robustness}, \text{shift}, \text{comparisons}, \text{limits}\}.$$

Part 3 - Hints and Helper

Require sources or internal experimental evidence for every major claim. Include assumptions, counterexamples, unresolved questions, and a balanced conclusion.

Part 4 - Python code skeleton

```
import torch

def E19(sections):
    required = {
        "model", "learning_rule", "convergence", "capacity",
        "margin", "generalization", "noise", "distribution_shift",
        "adversarial_robustness", "modern_comparisons",
        "production_suitability", "limitations", "conclusion",
    }
    missing = sorted(required - sections.keys())
    unsupported = [
        name for name, value in sections.items()
        if isinstance(value, dict) and not value.get("evidence")
    ]
    return {
        "complete": not missing and not unsupported,
        "missing": missing,
        "unsupported": unsupported,
        "sections": sections,
    }

def main():
    print(E19(...))

if __name__ == "__main__":
    main()
```

12.10 PC08 - Transition to Multi-Layer Networks

Models and Training

12.10.1 Exercise 00 - Demonstrate the Inability of Single Perceptron to Model XOR

Part 1 - Why this matters

XOR assigns the same class to opposite corners of a square, so no single line can place both positive corners on one side and both negative corners on the other. This is a representational failure rather than a training failure.

Changing the learning rate, initialization, epoch count, or hardware cannot create a missing linear separator. The contradiction should be demonstrated mathematically and confirmed through persistent training errors.

Part 2 - Mathematical form

$$w_1 + b < 0, \quad w_2 + b > 0, \quad w_1 + b > 0, \quad w_1 + w_2 + b < 0 \quad \text{cannot all hold for XOR.}$$

Part 3 - Hints and Helper

Use all four binary inputs and try several data orders. Record the best achievable error and explain why non-convergence is expected.

Part 4 - Python code skeleton

```
import torch

def E00(features, labels, learning_rate, epochs):
    best_errors = len(labels)
    history = []
    weight = torch.zeros(features.shape[1])
    bias = torch.tensor(0.0)
    for _ in range(epochs):
        mistakes = ...
        best_errors = min(best_errors, mistakes)
        history.append(mistakes)
    return {
        "converged": best_errors == 0,
        "best_errors": best_errors,
        "history": history,
    }

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

12.10.2 Exercise 01 - Manually Combine Two Perceptrons to Solve XOR

Part 1 - Why this matters

A hidden layer can transform the four XOR points into features that become linearly separable. One construction computes OR and NAND with two perceptrons, then applies AND to their outputs.

This network solves XOR by composition, not because any individual unit becomes nonlinear in its boundary geometry. The intermediate binary representation is the crucial new capability.

Part 2 - Mathematical form

$$h_1 = H(x_1 + x_2 - 0.5), \quad h_2 = H(-x_1 - x_2 + 1.5), \quad y = H(h_1 + h_2 - 1.5).$$

Part 3 - Hints and Helper

Implement the step function once and print every hidden pair. Verify the truth table manually rather than training these known parameters.

Part 4 - Python code skeleton

```
import torch

def step(value):
    return (value >= 0).to(value.dtype)

def E01(features):
    hidden_or = step(features[:, 0] + features[:, 1] - 0.5)
    hidden_nand = step(-features[:, 0] - features[:, 1] + 1.5)
    hidden = torch.stack((hidden_or, hidden_nand), dim=1)
    output = step(hidden[:, 0] + hidden[:, 1] - 1.5)
    return hidden, output

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

12.10.3 Exercise 02 - Interpret Hidden Units as Feature Transformers

Part 1 - Why this matters

Hidden units do more than produce intermediate predictions. They create a new coordinate system in which combinations such as OR, NAND, thresholds, or learned directions become features for the next layer.

The output layer sees hidden activations rather than the original inputs. A problem that is inseparable in input space can therefore become separable after a suitable nonlinear transformation.

Part 2 - Mathematical form

$$\mathbf{h} = \phi(W_1\mathbf{x} + \mathbf{b}_1), \qquad \hat{y} = H(\mathbf{w}_2^{\mathsf{T}}\mathbf{h} + b_2).$$

Part 3 - Hints and Helper

Compare pairwise distances and class separability before and after the hidden transformation. Inspect each unit's activation pattern.

Part 4 - Python code skeleton

```
import torch

def E02(features, first_weight, first_bias):
    pre_activation = features @ first_weight.T + first_bias
    hidden_features = (pre_activation >= 0).to(features.dtype)
    input_distances = torch.cdist(features, features)
    hidden_distances = torch.cdist(hidden_features, hidden_features)
    return pre_activation, hidden_features, input_distances, hidden_distances

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

12.10.4 Exercise 03 - Build a Two-Layer Perceptron Network Manually

Part 1 - Why this matters

A manual two-layer network exposes the sequence of affine transformation, hidden activation, second affine transformation, and output decision. Shape discipline becomes more important because matrices connect groups of units.

Hard step activations are suitable for a logical construction but do not provide useful ordinary gradients. This manual model therefore demonstrates expressiveness before gradient-based learning is introduced.

Part 2 - Mathematical form

$$\mathbf{h} = H(XW_1^{\mathsf{T}} + \mathbf{b}_1), \qquad \hat{\mathbf{y}} = H(\mathbf{h}W_2^{\mathsf{T}} + \mathbf{b}_2).$$

Part 3 - Hints and Helper

Validate every matrix dimension and return pre-activations as well as hard outputs. Keep the batch dimension first.

Part 4 - Python code skeleton

```
import torch

def E03(features, weight_1, bias_1, weight_2, bias_2):
    hidden_scores = features @ weight_1.T + bias_1
    hidden = (hidden_scores >= 0).to(features.dtype)
    output_scores = hidden @ weight_2.T + bias_2
    output = (output_scores >= 0).to(features.dtype)
    return {
        "hidden_scores": hidden_scores,
        "hidden": hidden,
```

```
        "output_scores": output_scores,
        "output": output,
    }

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

12.10.5 Exercise 04 - Compare Single-Layer vs Multi-Layer Expressiveness

Part 1 - Why this matters

A single perceptron represents one linear half-space. Multiple hidden units can create several half-spaces whose activation patterns are combined into unions, intersections, and piecewise-defined regions.

Greater expressiveness does not guarantee successful training or generalization. Width, activation, parameter values, and optimization determine which of the available functions the model actually learns.

Part 2 - Mathematical form

$$\mathcal{H}_1 = \{H(\mathbf{w}^\mathsf{T}\mathbf{x} + b)\}, \qquad \mathcal{H}_2 = \{H(W_2 H(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2)\}.$$

Part 3 - Hints and Helper

Evaluate both models on linearly separable and XOR datasets. Compare best error, parameter count, decision-region count, and training method.

Part 4 - Python code skeleton

```
import torch

def E04(datasets, single_layer, multi_layer):
    report = {}
    for name, (features, labels) in datasets.items():
        single_predictions = single_layer(features)
        multi_predictions = multi_layer(features)
        report[name] = {
            "single_error": ...,
            "multi_error": ...,
            "single_parameters": ...,
            "multi_parameters": ...,
        }
    return report
```

```
def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

12.10.6 Exercise 05 - Analyze Decision Boundaries Formed by Two-Layer Models

Part 1 - Why this matters

Each hidden unit introduces a linear boundary in input space, dividing it into activation regions. The output unit assigns classes to combinations of these regions, producing a piecewise-linear or piecewise-constant decision map.

Not every theoretical activation pattern occurs for a given set of weights. Region analysis should sample the space and identify observed patterns instead of assuming all combinations are reachable.

Part 2 - Mathematical form

$$\mathcal{R}_{\mathbf{a}} = \{\mathbf{x} : H(W_1\mathbf{x} + \mathbf{b}_1) = \mathbf{a}\}, \qquad \mathbf{a} \in \{0,1\}^h.$$

Part 3 - Hints and Helper

Evaluate a two-dimensional grid, encode each hidden pattern as an integer, and compare pattern borders with final class borders.

Part 4 - Python code skeleton

```
import torch

def E05(points, weight_1, bias_1, weight_2, bias_2):
    hidden = (points @ weight_1.T + bias_1 >= 0).to(torch.int64)
    powers = 2 ** torch.arange(hidden.shape[1])
    region_ids = (hidden * powers).sum(dim=1)
    outputs = (
        hidden.to(weight_2.dtype) @ weight_2.T + bias_2 >= 0
    ).to(torch.int64)
    return region_ids, outputs

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

12.10.7 Exercise 06 - Observe Emergent Non-Linearity in Hidden Representations

Part 1 - Why this matters

The affine transformations in a network remain linear, but applying a nonlinear activation between them prevents the layers from collapsing into one matrix. The hidden representation can bend, fold, or gate the original geometry.

Without a nonlinear activation, two affine layers are equivalent to one affine layer and still cannot solve XOR. This comparison isolates the source of the expressive gain.

Part 2 - Mathematical form

$$W_2(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (W_2W_1)\mathbf{x} + (W_2\mathbf{b}_1 + \mathbf{b}_2).$$

Part 3 - Hints and Helper

Run the same weights with identity and nonlinear activations. Compare XOR error and verify the collapsed affine parameters reproduce the identity case.

Part 4 - Python code skeleton

```
import torch

def E06(features, weight_1, bias_1, weight_2, bias_2):
    linear_hidden = features @ weight_1.T + bias_1
    linear_output = linear_hidden @ weight_2.T + bias_2
    collapsed_weight = weight_2 @ weight_1
    collapsed_bias = weight_2 @ bias_1 + bias_2
    collapsed_output = features @ collapsed_weight.T + collapsed_bias
    nonlinear_hidden = torch.relu(linear_hidden)
    nonlinear_output = nonlinear_hidden @ weight_2.T + bias_2
    return linear_output, collapsed_output, nonlinear_output

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

12.10.8 Exercise 07 - Implement Simple Two-Layer Network in PyTorch

Part 1 - Why this matters

PyTorch modules register parameters, manage devices, and build an automatic differentiation graph. A two-layer network combines two linear transformations with a nonlinear hidden activation.

The final layer should usually return logits during training. Applying sigmoid inside the model is unnecessary when a numerically stable logits-based binary loss is used.

Part 2 - Mathematical form

$$\mathbf{h} = \mathrm{ReLU}(W_1\mathbf{x} + \mathbf{b}_1), \qquad z = W_2\mathbf{h} + b_2.$$

Part 3 - Hints and Helper

Subclass torch.nn.Module, define layers in the constructor, and keep forward free of device transfers or parameter creation.

Part 4 - Python code skeleton

```
import torch

class TwoLayerNetwork(torch.nn.Module):
    def __init__(self, input_size, hidden_size):
        super().__init__()
        self.hidden = torch.nn.Linear(input_size, hidden_size)
        self.output = torch.nn.Linear(hidden_size, 1)

    def forward(self, features):
        hidden = torch.relu(self.hidden(features))
        return self.output(hidden).squeeze(-1)

def E07(input_size, hidden_size):
    model = TwoLayerNetwork(input_size, hidden_size)
    return model, sum(parameter.numel() for parameter in model.parameters())

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

12.10.9 Exercise 08 - Compare Manual Network vs nn.Module Implementation

Part 1 - Why this matters

A manual tensor implementation and an nn.Module implementation should agree when they use identical parameters, activation, shapes, and dtype. This parity test verifies that abstraction has not changed the mathematics.

Parameters must be copied with the correct orientation and without creating a tracked assignment operation. Comparing intermediate hidden activations locates discrepancies more precisely than final outputs alone.

Part 2 - Mathematical form

$$Z_{\text{manual}} = \phi(XW_1^\top + \mathbf{b}_1)W_2^\top + \mathbf{b}_2 = Z_{\text{module}}.$$

Part 3 - Hints and Helper

Copy weights inside `torch.no_grad()`, switch the module to evaluation mode, and compare hidden pre-activations, activations, and logits.

Part 4 - Python code skeleton

```
import torch

def E08(features, parameters, model):
    weight_1, bias_1, weight_2, bias_2 = parameters
    with torch.no_grad():
        model.hidden.weight.copy_(weight_1)
        model.hidden.bias.copy_(bias_1)
        model.output.weight.copy_(weight_2)
        model.output.bias.copy_(bias_2)
    manual_hidden = torch.relu(features @ weight_1.T + bias_1)
    manual_logits = (manual_hidden @ weight_2.T + bias_2).squeeze(-1)
    module_logits = model(features)
    outputs_close = torch.allclose(manual_logits, module_logits)
    return manual_logits, module_logits, outputs_close

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

12.10.10 Exercise 09 - Train Two-Layer Network Using Gradient-Based Optimization

Part 1 - Why this matters

Learning hidden features requires credit assignment from the final error back through the output and hidden layers. A differentiable activation and loss allow backpropagation to compute these parameter gradients.

The hard step used by the classical perceptron has zero derivative almost everywhere, so it is replaced by a trainable activation such as ReLU or tanh while logits feed a stable loss function.

Part 2 - Mathematical form

$$L = \text{BCEWithLogits}(z, y), \qquad \theta \leftarrow \theta - \eta \nabla_\theta L.$$

Part 3 - Hints and Helper

Use the standard order: clear gradients, forward, loss, backward, optimizer step. Record both loss and XOR accuracy.

Part 4 - Python code skeleton

```
import torch

def E09(model, features, targets, optimizer, epochs):
    criterion = torch.nn.BCEWithLogitsLoss()
    history = []
    for _ in range(epochs):
        optimizer.zero_grad()
        logits = model(features)
        loss = criterion(logits, targets.float())
        loss.backward()
        optimizer.step()
        predictions = (logits.detach() >= 0).to(targets.dtype)
        history.append({"loss": loss.item(), "accuracy": ...})
    return model, history

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

12.10.11 Exercise 10 - Execute Two-Layer Training on CUDA

Part 1 - Why this matters

CUDA training requires model parameters, inputs, targets, loss computation, and optimizer state to use the same device. Moving only the input leaves hidden device mismatches that interrupt the forward or update step.

A tiny XOR dataset will not demonstrate GPU speedup because launch overhead dominates. The goal is device-correct end-to-end training and CPU–GPU result comparison under controlled seeds.

Part 2 - Mathematical form

$$\theta_G^{(t+1)} = \theta_G^{(t)} - \eta \nabla_{\theta_G} L_G.$$

Part 3 - Hints and Helper

Check CUDA availability, move the model before constructing or restoring optimizer state, and synchronize around timing only.

Part 4 - Python code skeleton

```
import torch

def E10(model, features, targets, epochs):
    if not torch.cuda.is_available():
        raise RuntimeError(...)
    device = torch.device("cuda")
    model = model.to(device)
    features = features.to(device)
    targets = targets.to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=...)
    trained_model, history = E09(
        model, features, targets, optimizer, epochs
    )
    return trained_model.cpu(), history

def main():
    print(E10(...))

if __name__ == "__main__":
    main()
```

12.10.12 Exercise 11 - Visualize Hidden Layer Activations

Part 1 - Why this matters

Hidden activation plots reveal which units respond to each input region and whether different classes receive distinct internal representations. Dead, saturated, or redundant units become visible.

For more than two hidden units, a heatmap preserves the full activation matrix better than forcing it into two axes. Any dimensionality reduction must report that it is a projection.

Part 2 - Mathematical form

$$H_{ij} = \phi(\mathbf{w}_j^{\mathsf{T}}\mathbf{x}_i + b_j).$$

Part 3 - Hints and Helper

Capture activations directly in a modified forward pass or with a temporary hook. Detach them before plotting and label samples and units.

Part 4 - Python code skeleton

```
import matplotlib.pyplot as plt
import torch

def E11(model, features):
    captured = {}
```

```
    def save_hidden(module, inputs, output):
        captured["hidden_pre_activation"] = output.detach().cpu()

    handle = model.hidden.register_forward_hook(save_hidden)
    try:
        logits = model(features)
    finally:
        handle.remove()
    hidden = torch.relu(captured["hidden_pre_activation"])
    figure, axis = plt.subplots()
    axis.imshow(hidden, aspect="auto", ...)
    return logits, hidden, figure

def main():
    print(E11(...))

if __name__ == "__main__":
    main()
```

12.10.13 Exercise 12 - Study Gradient Flow Through Multiple Layers

Part 1 - Why this matters

Backpropagation multiplies local derivatives along paths from the loss to earlier parameters. Small derivatives can weaken gradients, while large products can amplify them and destabilize training.

Layer-wise gradient norms provide a practical view of this chain rule. Missing gradients can also indicate a detached tensor, frozen parameter, in-place graph damage, or inactive units.

Part 2 - Mathematical form

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial z_2}\frac{\partial z_2}{\partial \mathbf{h}}\frac{\partial \mathbf{h}}{\partial z_1}\frac{\partial z_1}{\partial W_1}.$$

Part 3 - Hints and Helper

Run one backward pass and record parameter and gradient norms by layer. Treat None gradients separately from zero gradients.

Part 4 - Python code skeleton

```
import torch

def E12(model, features, targets):
    model.zero_grad(set_to_none=True)
    logits = model(features)
    loss = torch.nn.functional.binary_cross_entropy_with_logits(
```

```
        logits, targets.float()
    )
    loss.backward()
    report = {}
    for name, parameter in model.named_parameters():
        report[name] = {
            "parameter_norm": parameter.detach().norm().item(),
            "gradient_norm": (
                None
                if parameter.grad is None
                else parameter.grad.norm().item()
            ),
        }
    return loss.item(), report

def main():
    print(E12(...))

if __name__ == "__main__":
    main()
```

12.10.14 Exercise 13 - Analyze Training Stability in Multi-Layer Networks

Part 1 - Why this matters

Multi-layer optimization can oscillate, diverge, plateau, or become highly seed-dependent. Learning rate, initialization, activation choice, feature scale, batch order, and optimizer interact more strongly than in a single perceptron.

Stability analysis combines loss, accuracy, gradient norms, parameter norms, activation statistics, and finiteness. One decreasing loss curve is insufficient evidence of a reliable configuration.

Part 2 - Mathematical form

$$S_t = \{L_t, A_t, \|\nabla\theta_t\|_2, \|\theta_t\|_2, \text{finite}(\theta_t)\}.$$

Part 3 - Hints and Helper

Repeat training over seeds and learning rates. Stop safely on non-finite values and retain the last valid checkpoint for diagnosis.

Part 4 - Python code skeleton

```
import torch

def E13(run_training, seeds, learning_rates):
    results = []
```

```
    for seed in seeds:
        for learning_rate in learning_rates:
            torch.manual_seed(seed)
            history = run_training(learning_rate, ...)
            results.append({
                "seed": seed,
                "learning_rate": learning_rate,
                "history": history,
                "finite": ...,
                "converged": ...,
            })
    return results

def main():
    print(E13(...))

if __name__ == "__main__":
    main()
```

12.10.15 Exercise 14 - Compare Perceptron Training to Backpropagation

Part 1 - Why this matters

Classical perceptron training applies a local correction to one linear unit after a mistake. Backpropagation propagates a differentiable loss through a graph and assigns gradients to every parameter that influenced the output.

The comparison should preserve the distinction between architecture and learning algorithm. A network may contain perceptron-like affine units while being trained by backpropagation rather than the classical rule.

Part 2 - Mathematical form

$$\Delta \mathbf{w}_P = \eta y \mathbf{x} \mathbb{I}[yz \leq 0], \qquad \Delta \theta_{BP} = -\eta \nabla_\theta L.$$

Part 3 - Hints and Helper

Compare required derivatives, affected parameters, batch behavior, hidden credit assignment, memory, and ability to train an XOR solution.

Part 4 - Python code skeleton

```
import torch

def E14(perceptron_step, network_step, batch):
    perceptron_before, perceptron_after = perceptron_step(batch, ...)
    network_before, network_after, gradients = network_step(batch, ...)
    return {
```

```
        "perceptron_changed": changed_parameters(
            perceptron_before, perceptron_after
        ),
        "network_changed": changed_parameters(network_before, network_after),
        "network_gradient_paths": ...,
        "xor_result": ...,
    }

def main():
    print(E14(...))

if __name__ == "__main__":
    main()
```

12.10.16 Exercise 15 - Identify Failure Modes in Shallow Neural Networks

Part 1 - Why this matters

A shallow network can fail through insufficient hidden width, unsuitable activation, dead ReLU units, saturated sigmoids, poor initialization, unstable gradients, bad scaling, overfitting, or an inappropriate loss.

These failures can share the same symptom of low accuracy. Diagnosis must inspect activations, gradients, train–validation behavior, parameter changes, and data assumptions before selecting a remedy.

Part 2 - Mathematical form

$$F = \{\text{capacity}, \text{activation}, \text{initialization}, \text{gradient}, \text{optimization}, \text{data}, \text{generalization}\}.$$

Part 3 - Hints and Helper

Run diagnostic checks in a fixed order and connect every proposed cause to measured evidence. Avoid declaring dead units from one zero activation alone.

Part 4 - Python code skeleton

```
import torch

def E15(model, activations, gradients, history):
    diagnostics = {
        "dead_unit_rates": (activations == 0).float().mean(dim=0),
        "gradient_norms": gradients,
        "loss_plateau": ...,
        "train_validation_gap": ...,
        "finite_parameters": all(
            torch.isfinite(parameter).all()
```

```
            for parameter in model.parameters()
        ),
    }
    causes = infer_failure_causes(diagnostics, ...)
    return diagnostics, causes

def main():
    print(E15(...))

if __name__ == "__main__":
    main()
```

12.10.17 Exercise 16 - Evaluate Expressive Power Gain with Additional Layers

Part 1 - Why this matters

Additional layers compose learned transformations and can represent complex decision regions with reusable intermediate features. Depth may express some functions more compactly than a much wider shallow network.

More layers do not automatically improve results. Optimization difficulty, data volume, regularization, and architecture choice determine whether theoretical capacity becomes useful performance.

Part 2 - Mathematical form

$$f_L(\mathbf{x}) = W_L\phi(W_{L-1}\phi(\cdots\phi(W_1\mathbf{x}+\mathbf{b}_1)\cdots)+\mathbf{b}_{L-1})+\mathbf{b}_L.$$

Part 3 - Hints and Helper

Compare architectures under matched parameter budgets where possible. Report train fit, test behavior, region complexity, runtime, and gradient stability.

Part 4 - Python code skeleton

```
import torch

def E16(architectures, train_data, test_data, seeds):
    report = []
    for name, build_model in architectures.items():
        for seed in seeds:
            torch.manual_seed(seed)
            model = build_model()
            history = train_model(model, train_data, ...)
            report.append({
                "architecture": name,
                "parameters": sum(p.numel() for p in model.parameters()),
                "train_metrics": ...,
```

```
                "test_metrics": ...,
                "history": history,
            })
    return report

def main():
    print(E16(...))

if __name__ == "__main__":
    main()
```

12.10.18 Exercise 17 - Bridge Conceptual Gap Between Perceptron and MLP

Part 1 - Why this matters

An MLP preserves the perceptron's affine score as a basic building block, then repeats it across many units and layers with nonlinear activations. The key transition is from one fixed representation to learned hidden representations.

The learning rule also changes. Hard local corrections give way to a scalar loss, differentiable computation graph, chain rule, and optimizer that coordinate all parameters.

Part 2 - Mathematical form

$$\begin{aligned} \text{perceptron} \subset\ & \text{affine unit}, \\ \text{MLP} = & \text{ stacked affine units} \\ & + \text{nonlinear activations} + \text{backpropagation}. \end{aligned}$$

Part 3 - Hints and Helper

Create an explicit mapping between perceptron concepts and MLP concepts: score, activation, parameters, error signal, update, representation, and evaluation.

Part 4 - Python code skeleton

```
import torch

def E17():
    return {
        "linear_score": ("w^T x + b", "W x + b per layer"),
        "activation": ("hard step", "differentiable non-linearity"),
        "representation": ("input features", "learned hidden features"),
        "error_signal": ("misclassification", "scalar differentiable loss"),
        "update": ("local perceptron rule", "backpropagation + optimizer"),
        "capacity": ("one half-space", "composed decision regions"),
    }

def main():
```

```
    print(E17(...))

if __name__ == "__main__":
    main()
```

12.10.19 Exercise 18 - Stress Test Multi-Layer Perceptron with Complex Data

Part 1 - Why this matters

Complex nonlinear data tests whether hidden layers learn useful representations beyond XOR. Increasing samples, dimensions, class overlap, noise, and hidden width reveals capacity and computational limits.

A stress test must preserve evaluation quality while measuring throughput and memory. Success on training data alone may indicate memorization rather than a meaningful representation.

Part 2 - Mathematical form

$$X \in \mathbb{R}^{n \times d}, \qquad h \text{ hidden units}, \qquad \text{parameter scale} = \mathcal{O}(dh + h).$$

Part 3 - Hints and Helper

Generate controlled train and test sets, sweep hidden width, and monitor loss, accuracy, gradient norms, peak memory, and runtime on the target device.

Part 4 - Python code skeleton

```
import torch

def E18(configurations, dataset_factory, device):
    results = []
    for configuration in configurations:
        train_data, test_data = dataset_factory(configuration, ...)
        model = build_network(configuration).to(device)
        history, performance = train_and_profile(
            model, train_data, device, ...
        )
        results.append({
            "configuration": configuration,
            "history": history,
            "test_metrics": evaluate(model, test_data, device),
            "performance": performance,
        })
    return results

def main():
    print(E18(...))
```

```
if __name__ == "__main__":
    main()
```

12.10.20 Exercise 19 - Produce a Final Transition Report Toward Neural Networks

Part 1 - Why this matters

The final report should explain how the perceptron's limitation motivates hidden units, how nonlinear representations solve XOR, and why differentiable activations and backpropagation are needed to learn those representations.

It must distinguish manually constructed logical networks from trainable MLPs and connect expressiveness to optimization, stability, generalization, and computational cost. The report becomes the bridge to the next neural-network book.

Part 2 - Mathematical form

$$R = \{\text{XOR proof}, \text{hidden features}, \text{depth}, \text{activation},\\ \text{loss}, \text{backpropagation}, \text{stability},\\ \text{limits}, \text{next steps}\}.$$

Part 3 - Hints and Helper

Require equations, code evidence, activation views, gradient diagnostics, and train–test results. Mark unsupported conclusions and unresolved failure modes.

Part 4 - Python code skeleton

```
import torch

def E19(sections):
    required = {
        "xor_limit", "manual_xor", "hidden_representation",
        "two_layer_architecture", "nonlinearity", "pytorch_module",
        "gradient_training", "gradient_flow", "stability",
        "expressiveness", "failure_modes", "generalization",
        "computational_cost", "next_steps",
    }
    missing = sorted(required - sections.keys())
    unresolved = sections.get("failure_modes", {}).get("unresolved", [])
    return {
        "complete": not missing and not unresolved,
        "missing": missing,
        "unresolved": unresolved,
        "sections": sections,
    }
```

```
def main():
    print(E19(...))

if __name__ == "__main__":
    main()
```

Book 13

MLP (Feed-Forward Network)

13.1 Introduction

13.1.1 What will the book teach you

This book marks the transition from studying individual linear models to engineering complete neural networks. In the previous books, the reader learned how tensors, gradients, loss functions, optimizers, training loops, linear regression, logistic regression, and the perceptron work as separate ideas. The Multi-Layer Perceptron brings these ideas together inside one trainable system. Instead of using a single decision boundary, an MLP composes multiple affine transformations with non-linear activation functions, allowing the model to learn intermediate representations and approximate relationships that cannot be captured by one linear transformation.

The reader will begin by understanding why stacking linear layers without an activation function does not create a more expressive model. Two consecutive linear transformations can always be reduced to one linear transformation. Non-linearity is therefore not an optional decoration between layers; it is the mechanism that prevents the network from collapsing into a single linear model. The exercises will require the reader to inspect hidden activations, follow the dimensions through every layer, and explain what information each transformation creates, preserves, suppresses, or loses.

The book then moves from mathematical composition to architecture design. The reader will build configurable networks with different depths and widths, register parameters correctly with `nn.Module`, count trainable parameters, capture intermediate outputs with hooks, and compare shallow and deep architectures under controlled conditions. The purpose is not to build the largest possible network, but to learn how architectural decisions affect capacity, computation, memory, optimization difficulty, and generalization. A network should never be described only by the number of layers; its input and output dimensions, hidden widths, activation functions, parameter count, and data requirements must all be understood together.

Activation functions receive dedicated attention because they determine how signals and gradients move through the network. The reader will implement and compare ReLU, LeakyReLU, sigmoid, tanh, GELU, and other useful transformations, then observe saturation, dying units, activation distributions, and numerical behavior under different precisions. These experiments will show that choosing an activation is not merely selecting a familiar function. The choice

interacts with initialization, depth, input scale, gradient flow, hardware performance, and the type of representation the model can learn.

Backpropagation will be studied as the central learning mechanism of the MLP. The reader will trace gradients through several layers, verify the chain rule manually, inspect parameter gradients, use hooks to locate broken paths, and diagnose vanishing or exploding gradients. Automatic differentiation will be used as a tool, but it will not be treated as an invisible process. The reader must understand which tensors belong to the computational graph, how the final loss influences early layers, why gradients accumulate, and how an apparently small coding decision can prevent an entire section of the network from learning.

Weight initialization connects architecture to training stability. The book will demonstrate why setting all hidden weights to zero creates a symmetry failure, why arbitrary scales can destroy activation and gradient statistics, and how Xavier and He initialization are derived from signal-preservation requirements. The reader will measure activation variance and gradient variance across depth instead of accepting an initializer only because it is commonly used. Initialization will also be tested with different activation functions, floating-point formats, random seeds, and very deep architectures.

Optimization is approached as a dynamic process rather than a function call. The reader will implement and compare SGD, momentum, Adam, and AdamW, investigate learning-rate schedules, gradient clipping, weight decay, batch-size effects, and gradient accumulation, and identify silent failures such as forgetting to clear gradients or applying operations in the wrong order. Every optimizer experiment will connect parameter changes to loss behavior, gradient norms, convergence speed, memory cost, and stability. Faster initial loss reduction will not automatically be interpreted as a better final model.

The complete training pipeline will then be assembled. The reader will build correct classification and regression loops, separate training and evaluation modes, calculate metrics over complete datasets, create validation loops, add early stopping, save checkpoints, resume optimizer and scheduler state, and record structured experiment logs. The exercises will also include deliberate failure cases: good loss with bad accuracy, NaN loss, incorrect label encoding, wrong output shapes, data-device mismatch, silent overflow, and checkpoints that resume with unexpected drift. Debugging these failures is a core learning goal, not an optional appendix.

Generalization will be studied through the gap between fitting the training data and performing reliably on unseen data. The reader will experiment with weight decay, explicit L2 penalties, dropout, early stopping, data augmentation, feature normalization, model capacity, label noise, cross-validation, and model ensembling. Train and validation behavior will be interpreted together so that underfitting, overfitting, optimization failure, and distribution shift are not confused with one another. Regularization will be evaluated as a trade-off with measurable effects rather than as a collection of settings that should always be enabled.

Finally, the book moves from a correct experiment to an engineering-ready system. The reader will profile training, study the DataLoader pipeline, use pinned memory, gradient accumulation, mixed precision, and CUDA carefully, measure latency and throughput, detect memory growth, rotate checkpoints, test failure recovery, and validate exported or compiled

models against a trusted reference. Production readiness will include input validation, deterministic controls, experiment tracking, regression tests, monitoring expectations, and a clear record of remaining sources of non-determinism. The goal is not merely to train an MLP once, but to build one that can be explained, reproduced, debugged, scaled, evaluated, saved, restored, and trusted within defined limits.

13.1.2 Why this book comes after the Perceptron

The perceptron introduced the essential idea of a trainable decision unit: an input is multiplied by weights, combined with a bias, and converted into a decision. It also exposed a fundamental limitation. A single perceptron can represent only one linear separation and therefore cannot solve problems such as XOR. Combining units into layers creates new feature spaces in which these problems may become separable, but architecture alone is not enough. The hard step function used by the classical perceptron does not provide the useful derivatives required to coordinate the learning of many hidden units.

An MLP resolves this limitation by combining differentiable non-linear activations with backpropagation. The output error is transformed into gradients for the final layer and then propagated backward through every earlier layer. This turns hidden units from manually configured logical components into learned feature transformers. The transition from perceptron to MLP is therefore not simply an increase in layer count. It is a change in representation, optimization, credit assignment, computational cost, and the types of failure that must be diagnosed.

13.1.3 How to study this book

Each exercise should be treated as an experiment with a claim that must be verified. Before writing code, identify the tensor shapes, mathematical operation, expected output, and failure condition. After writing the code, inspect intermediate activations, gradients, parameter changes, devices, dtypes, and memory behavior rather than checking only the final prediction. When an exercise compares two configurations, keep the dataset, random seed, evaluation procedure, and computational budget controlled so that the comparison has a clear meaning.

The reader should resist the temptation to hide the training process behind large abstractions too early. PyTorch modules and optimizers will be used, but manual calculations and explicit tensor operations remain necessary for verification. A successful result should be reproducible and explainable: the reader must know why the shapes are valid, why gradients reach every intended parameter, why the chosen loss matches the task, why the model generalizes, and which limitations remain after training.

13.1.4 The Chapters (MLP00 - MLP08) included in this book

- MLP00 - Conceptual Foundation of MLP
- MLP01 - Architecture Design and Layer Composition
- MLP02 - Activation Functions and Non-Linearity
- MLP03 - Backpropagation Mechanics in MLP

- MLP04 - Weight Initialization and Training Stability
- MLP05 - Optimizers and Learning Rate Dynamics
- MLP06 - Loss Functions, Training Loop, and Logging
- MLP07 - Regularization, Dropout, and Generalization
- MLP08 - Scaling, Profiling, and Production Readiness

13.2 MLP00 - Conceptual Foundation of MLP

Models and Training

13.2.1 Exercise 00 - Understand Why Single Perceptron Is Insufficient for Complex Functions

Part 1 - Why this matters

A single perceptron creates one affine score and one linear decision boundary. It can separate two classes only when one half-space is sufficient, so patterns such as XOR and curved class regions remain outside its representational capacity.

This is a limitation of the hypothesis class rather than insufficient training. More epochs, a different optimizer, or a GPU cannot make one linear boundary represent a fundamentally non-linear relationship.

Part 2 - Mathematical form

$$\hat{y} = H(\mathbf{w}^\mathsf{T}\mathbf{x} + b), \qquad \mathcal{B} = \{\mathbf{x} : \mathbf{w}^\mathsf{T}\mathbf{x} + b = 0\}.$$

Part 3 - Hints and Helper

Compare a linearly separable dataset with XOR. Record the best error and identify whether failure comes from representation or optimization.

Part 4 - Python code skeleton

```
import torch

def E00(datasets, train_perceptron):
    report = {}
    for name, (features, labels) in datasets.items():
        model, history = train_perceptron(features, labels, ...)
        predictions = model(features)
        report[name] = {
            "best_errors": ...,
            "final_accuracy": ...,
            "converged": ...,
            "history": history,
```

```
        }
    return report

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

13.2.2 Exercise 01 - Define Feed-Forward Network as Layered Function Composition

Part 1 - Why this matters

A feed-forward network applies an ordered sequence of functions in which information moves from the input toward the output without recurrent feedback. Each layer receives the representation produced by the previous layer.

Writing the model as a composition makes depth explicit and prepares the reader for backpropagation, where the chain rule follows the same sequence in reverse.

Part 2 - Mathematical form

$$f(\mathbf{x}) = f_L \circ f_{L-1} \circ \cdots \circ f_1(\mathbf{x}), \qquad f_l(\mathbf{h}_{l-1}) = \phi_l(W_l \mathbf{h}_{l-1} + \mathbf{b}_l).$$

Part 3 - Hints and Helper

Store activations in order and verify that every layer consumes the previous output. Keep the batch dimension unchanged.

Part 4 - Python code skeleton

```
import torch

def E01(features, layers, activations):
    hidden = features
    trace = [hidden]
    for index, (weight, bias) in enumerate(layers):
        hidden = hidden @ weight.T + bias
        hidden = activations[index](hidden)
        trace.append(hidden)
    return hidden, trace

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

13.2.3 Exercise 02 - Interpret Hidden Layers as Learned Feature Transformers

Part 1 - Why this matters

Hidden layers convert raw inputs into intermediate representations that can make the final task easier. Their units are not fixed semantic concepts; their meaning arises from parameters learned jointly for the objective.

A useful representation can separate classes, suppress irrelevant variation, or combine simple patterns into higher-level features. Interpretation requires inspecting activation behavior rather than assigning names from weight values alone.

Part 2 - Mathematical form

$$\mathbf{h}_l = \phi_l(W_l \mathbf{h}_{l-1} + \mathbf{b}_l), \qquad \mathbf{h}_0 = \mathbf{x}.$$

Part 3 - Hints and Helper

Compare distances and class separation in input and hidden spaces. Inspect unit activation distributions over meaningful sample groups.

Part 4 - Python code skeleton

```
import torch

def E02(model, features, labels):
    representations = model.forward_with_activations(features)
    analysis = []
    for layer_index, hidden in enumerate(representations):
        analysis.append({
            "layer": layer_index,
            "shape": tuple(hidden.shape),
            "mean": hidden.mean(dim=0),
            "standard_deviation": hidden.std(dim=0),
            "class_separation": ...,
        })
    return analysis

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

13.2.4 Exercise 03 - Compare Linear Model Output vs Multi-Layer Non-Linear Output

Part 1 - Why this matters

A linear model maps inputs through one affine transformation, while an MLP inserts non-linear transformations between affine layers. This prevents the complete mapping from collapsing into one matrix.

The comparison must hold the dataset and output convention fixed. A larger parameter count alone is not the source of non-linearity; removing every activation makes the deep model affine again.

Part 2 - Mathematical form

$$f_{\text{linear}}(X) = XW^{\mathsf{T}} + \mathbf{b}, \qquad f_{\text{MLP}}(X) = W_2\phi(W_1X + \mathbf{b}_1) + \mathbf{b}_2.$$

Part 3 - Hints and Helper

Evaluate both models on linear and non-linear targets. Also replace the hidden activation with identity and verify the network collapses to a linear mapping.

Part 4 - Python code skeleton

```
import torch

def E03(features, linear_model, mlp_model, identity_mlp):
    linear_output = linear_model(features)
    nonlinear_output = mlp_model(features)
    identity_output = identity_mlp(features)
    return {
        "linear": linear_output,
        "nonlinear": nonlinear_output,
        "identity_mlp": identity_output,
        "nonlinear_difference": ...,
    }

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

13.2.5 Exercise 04 - Visualize Information Flow Through Multiple Layers

Part 1 - Why this matters

Information flow can be represented as a sequence of tensor shapes, affine scores, activations, and output logits. This trace reveals where dimensionality changes and where information is compressed or expanded.

A diagram should reflect the actual forward pass rather than a generic architecture sketch. Batch size, feature width, activation, and device should be captured from real tensors.

Part 2 - Mathematical form

$$X^{(B\times d_0)} \to H_1^{(B\times d_1)} \to \cdots \to H_L^{(B\times d_L)}.$$

Part 3 - Hints and Helper

Collect one summary per layer and draw nodes connected in execution order. Avoid storing gradient-connected tensors solely for visualization.

Part 4 - Python code skeleton

```
import torch

def E04(model, features):
    trace = []
    hidden = features
    trace.append({"name": "input", "shape": tuple(hidden.shape)})
    for name, layer in model.named_children():
        hidden = layer(hidden)
        trace.append({
            "name": name,
            "shape": tuple(hidden.shape),
            "minimum": hidden.detach().min().item(),
            "maximum": hidden.detach().max().item(),
        })
    return trace, build_flow_visualization(trace, ...)

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

13.2.6 Exercise 05 - Implement a Two-Layer Network Using Pure Tensor Operations

Part 1 - Why this matters

A pure tensor implementation exposes the exact operations behind a two-layer MLP: first affine transformation, hidden activation, and output affine transformation. No module abstraction hides parameter orientation or broadcasting.

This implementation becomes a reference for later nn.Module code. It should validate dimensions and return intermediate tensors so each operation can be tested independently.

Part 2 - Mathematical form

$$H = \phi(XW_1^\mathsf{T} + \mathbf{b}_1), \qquad Z = HW_2^\mathsf{T} + \mathbf{b}_2.$$

Part 3 - Hints and Helper

Use matrix multiplication rather than loops. Keep output as logits and apply task-specific probabilities outside this function.

Part 4 - Python code skeleton

```
import torch

def E05(features, weight_1, bias_1, weight_2, bias_2):
    if features.shape[1] != weight_1.shape[1]:
        raise ValueError(...)
    hidden_scores = features @ weight_1.T + bias_1
    hidden = torch.relu(hidden_scores)
    logits = hidden @ weight_2.T + bias_2
    return hidden_scores, hidden, logits

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

13.2.7 Exercise 06 - Translate Mathematical MLP Formulation into PyTorch Code

Part 1 - Why this matters

Translating equations into PyTorch requires a consistent convention for batches, layer widths, and matrix orientation. Mathematical column-vector notation often becomes batch-first matrix multiplication in code.

A correct translation preserves both values and dimensions. The reader should be able to connect every code statement to one equation term and explain each broadcasted bias.

Part 2 - Mathematical form

$$Z_l = H_{l-1} W_l^{\mathsf{T}} + \mathbf{b}_l, \qquad H_l = \phi_l(Z_l).$$

Part 3 - Hints and Helper

Write the tensor shape beside every variable before coding. Compare a manual result with nn.Linear initialized from the same parameters.

Part 4 - Python code skeleton

```
import torch

def E06(features, weights, biases, activations):
    hidden = features
```

```
    states = []
    for weight, bias, activation in zip(weights, biases, activations):
        pre_activation = torch.matmul(hidden, weight.T) + bias
        hidden = activation(pre_activation)
        states.append((pre_activation, hidden))
    return hidden, states

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

13.2.8 Exercise 07 - Analyze the Role of Non-Linearity Between Linear Layers

Part 1 - Why this matters

Without a non-linear activation, several affine layers are algebraically equivalent to one affine transformation. Depth alone therefore cannot produce curved or disconnected decision regions.

Non-linearity changes which input-dependent patterns survive into the next layer. Different activation functions also alter gradient flow, sparsity, smoothness, and numerical behavior.

Part 2 - Mathematical form

$$W_2(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (W_2W_1)\mathbf{x} + (W_2\mathbf{b}_1 + \mathbf{b}_2).$$

Part 3 - Hints and Helper

Verify equality between stacked affine layers and collapsed parameters. Insert ReLU and show that one global affine map no longer reproduces the result.

Part 4 - Python code skeleton

```
import torch

def E07(features, weight_1, bias_1, weight_2, bias_2):
    stacked_linear = (features @ weight_1.T + bias_1) @ weight_2.T + bias_2
    collapsed_weight = weight_2 @ weight_1
    collapsed_bias = bias_1 @ weight_2.T + bias_2
    collapsed = features @ collapsed_weight.T + collapsed_bias
    hidden = torch.relu(features @ weight_1.T + bias_1)
    nonlinear = hidden @ weight_2.T + bias_2
    outputs_close = torch.allclose(stacked_linear, collapsed)
    return stacked_linear, collapsed, nonlinear, outputs_close

def main():
```

```
    print(E07(...))

if __name__ == "__main__":
    main()
```

13.2.9 Exercise 08 - Verify Dimensional Consistency Across All Layers

Part 1 - Why this matters

Every layer requires its input width to match the previous layer's output width. Dimensional consistency is a structural condition that can be checked before parameters are initialized or data are processed.

Batch size is not part of the layer configuration and should remain flexible. Confusing batch width with feature width creates designs that work for one test tensor and fail on real data.

Part 2 - Mathematical form

$$W_l \in \mathbb{R}^{d_l \times d_{l-1}}, \quad \mathbf{b}_l \in \mathbb{R}^{d_l}, \quad H_l \in \mathbb{R}^{B \times d_l}.$$

Part 3 - Hints and Helper

Validate every adjacent pair and report the exact layer index on failure. Test several batch sizes after the architecture passes static checks.

Part 4 - Python code skeleton

```
import torch

def E08(layer_sizes, weights, biases):
    if len(weights) != len(layer_sizes) - 1:
        raise ValueError(...)
    checks = []
    paired_parameters = zip(weights, biases)
    for layer_index, (weight, bias) in enumerate(
        paired_parameters, start=1
    ):
        expected = (layer_sizes[layer_index], layer_sizes[layer_index - 1])
        checks.append({
            "layer": layer_index,
            "expected_weight": expected,
            "actual_weight": tuple(weight.shape),
            "bias_valid": tuple(bias.shape) == (layer_sizes[layer_index],),
            "valid": ...,
        })
    return checks

def main():
```

```
    print(E08(...))

if __name__ == "__main__":
    main()
```

13.2.10 Exercise 09 - Identify Common Conceptual Misunderstandings About Hidden Layers

Part 1 - Why this matters

Hidden units are neither automatically interpretable concepts nor independent miniature models. They are jointly optimized coordinates whose usefulness depends on downstream layers, activation functions, data, and loss.

More hidden units or layers do not guarantee better learning. Capacity, optimization, regularization, signal propagation, and dataset size interact, so architecture claims require controlled evidence.

Part 2 - Mathematical form

$$\text{hidden representation} \neq \text{guaranteed semantic feature},$$
$$\text{greater capacity} \neq \text{better generalization}.$$

Part 3 - Hints and Helper

Turn each misconception into a falsifiable experiment. Record the claim, counterexample, measurement, and corrected interpretation.

Part 4 - Python code skeleton

```
import torch

def E09(experiments):
    corrections = []
    for experiment in experiments:
        evidence = experiment["run"]()
        corrections.append({
            "misconception": experiment["claim"],
            "evidence": evidence,
            "supported": experiment["test"](evidence),
            "corrected_interpretation": ...,
        })
    return corrections

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

13.2.11 Exercise 10 - Execute Forward Pass of MLP on CUDA Device

Part 1 - Why this matters

A CUDA forward pass requires inputs, parameters, buffers, and temporary operations to use the same device. A model copied to CUDA does not automatically move a CPU input batch.

Device correctness should be verified independently from speed. Small MLPs may run slower on a GPU because transfer and launch overhead exceed the computation.

Part 2 - Mathematical form

$$H_l^G = \phi_l(H_{l-1}^G {W_l^G}^{\mathsf{T}} + \mathbf{b}_l^G).$$

Part 3 - Hints and Helper

Check availability, move the model and input once, disable gradient tracking for inference, and return only required reporting tensors to CPU.

Part 4 - Python code skeleton

```
import torch

def E10(model, features):
    if not torch.cuda.is_available():
        raise RuntimeError(...)
    device = torch.device("cuda")
    model = model.to(device)
    features = features.to(device)
    model.eval()
    with torch.inference_mode():
        output = model(features)
    return output.cpu(), next(model.parameters()).device

def main():
    print(E10(...))

if __name__ == "__main__":
    main()
```

13.2.12 Exercise 11 - Compare CPU vs GPU Forward Execution Outputs

Part 1 - Why this matters

Equivalent CPU and GPU forward passes should produce numerically close outputs when model state, input values, dtype, and evaluation mode match. Exact bitwise equality is not always expected.

Comparing only predicted classes can hide score drift. Layer-wise activations and final logits reveal where numerical differences first appear.

Part 2 - Mathematical form

$$\delta_l = \|H_l^C - H_l^G\|_\infty, \qquad \text{close}_l = \mathbb{I}[\delta_l \le \text{atol} + \text{rtol}|H_l^C|].$$

Part 3 - Hints and Helper

Deep-copy one model state to both devices and use identical inputs. Compare outputs with documented tolerances and count prediction disagreement separately.

Part 4 - Python code skeleton

```
import torch

def E11(cpu_model, gpu_model, features, atol, rtol):
    cpu_model.eval()
    gpu_model.load_state_dict(cpu_model.state_dict())
    gpu_model = gpu_model.cuda().eval()
    with torch.inference_mode():
        cpu_output = cpu_model(features)
        gpu_output = gpu_model(features.cuda()).cpu()
    return {
        "close": torch.allclose(cpu_output, gpu_output, atol=atol, rtol=rtol),
        "maximum_difference": (cpu_output - gpu_output).abs().max().item(),
        "cpu_output": cpu_output,
        "gpu_output": gpu_output,
    }

def main():
    print(E11(...))

if __name__ == "__main__":
    main()
```

13.2.13 Exercise 12 - Inspect Intermediate Activations at Each Hidden Layer

Part 1 - Why this matters

Intermediate activations show how the representation evolves through depth. Their ranges, means, variances, sparsity, and finiteness can expose saturation, dead units, scale explosion, or information collapse.

Inspection should not retain computational graphs across many batches. Detached summaries are usually safer than saving every complete activation tensor.

Part 2 - Mathematical form

$$A_l = \{\mu(H_l), \sigma(H_l), \min(H_l), \max(H_l), \Pr(H_l = 0), \text{finite}(H_l)\}.$$

Part 3 - Hints and Helper

Use temporary forward hooks and remove them after the pass. Label summaries by module name and execution order.

Part 4 - Python code skeleton

```
import torch

def E12(model, features):
    summaries = {}
    handles = []

    def capture(name):
        def hook(module, inputs, output):
            value = output.detach()
            summaries[name] = {
                "shape": tuple(value.shape),
                "mean": value.mean().item(),
                "std": value.std().item(),
                "zero_rate": (value == 0).float().mean().item(),
                "finite": torch.isfinite(value).all().item(),
            }
        return hook

    for name, module in model.named_modules():
        if should_capture(module):
            handles.append(module.register_forward_hook(capture(name)))
    try:
        output = model(features)
    finally:
        for handle in handles:
            handle.remove()
    return output, summaries

def main():
    print(E12(...))

if __name__ == "__main__":
    main()
```

13.2.14 Exercise 13 - Analyze the Effect of Depth on Representational Power

Part 1 - Why this matters

Depth composes transformations so later layers can reuse features created earlier. Some complex functions can be represented more efficiently by deep compositions than by one extremely wide layer.

Representational possibility is not the same as trainability. Deeper networks can suffer unstable signals, difficult optimization, higher latency, and overfitting if architecture and data are not matched.

Part 2 - Mathematical form

$$f_L = f_L \circ f_{L-1} \circ \cdots \circ f_1, \qquad \text{capacity generally grows with depth and width.}$$

Part 3 - Hints and Helper

Compare architectures under similar parameter budgets. Measure train fit, validation behavior, activation health, gradient flow, runtime, and memory.

Part 4 - Python code skeleton

```
import torch

def E13(architectures, dataset, seeds):
    results = []
    for name, build_model in architectures.items():
        for seed in seeds:
            torch.manual_seed(seed)
            model = build_model()
            history = train_model(model, dataset, ...)
            results.append({
                "architecture": name,
                "depth": count_linear_layers(model),
                "parameters": sum(p.numel() for p in model.parameters()),
                "history": history,
                "validation_metric": ...,
            })
    return results

def main():
    print(E13(...))

if __name__ == "__main__":
    main()
```

13.2.15 Exercise 14 - Implement Forward Pass Without Using nn.Module

Part 1 - Why this matters

A function-based forward pass separates the mathematics of inference from PyTorch's module system. It demonstrates that an MLP is fundamentally a sequence of tensor operations over parameter collections.

Without nn.Module, parameters are not registered automatically, device movement is manual, and state serialization requires an explicit structure. These limitations explain the value of the abstraction.

Part 2 - Mathematical form

$$H_0 = X, \qquad H_l = \phi_l(H_{l-1}W_l^{\mathsf{T}} + \mathbf{b}_l), \quad l = 1, \ldots, L.$$

Part 3 - Hints and Helper

Store weights, biases, and activations in aligned lists. Validate their lengths and avoid creating parameters inside the forward function.

Part 4 - Python code skeleton

```
import torch

def E14(features, parameters, activations):
    weights = parameters["weights"]
    biases = parameters["biases"]
    if not (len(weights) == len(biases) == len(activations)):
        raise ValueError(...)
    hidden = features
    for weight, bias, activation in zip(weights, biases, activations):
        hidden = activation(hidden @ weight.T + bias)
    return hidden

def main():
    print(E14(...))

if __name__ == "__main__":
    main()
```

13.2.16 Exercise 15 - Debug Shape Errors in Multi-Layer Tensor Computations

Part 1 - Why this matters

Shape errors can originate several layers before the operation that finally fails. A wrong width, transposed matrix, lost batch dimension, or unintended broadcast may propagate until a later matrix multiplication.

A useful debugger reports expected and actual shapes at every boundary. It should detect semantically wrong broadcasting even when PyTorch accepts the operation.

Part 2 - Mathematical form

$$H_{l-1}^{(B\times d_{l-1})} W_l^{\mathsf{T}^{(d_{l-1}\times d_l)}} = Z_l^{(B\times d_l)}.$$

Part 3 - Hints and Helper

Trace shapes before each operation and raise an error containing the layer index. Test one sample with an explicit batch dimension rather than squeezing it away.

Part 4 - Python code skeleton

```
import torch

def E15(features, weights, biases):
    hidden = features
    trace = []
    for layer_index, (weight, bias) in enumerate(zip(weights, biases)):
        expected_width = weight.shape[1]
        trace.append({
            "layer": layer_index,
            "input": tuple(hidden.shape),
            "weight": tuple(weight.shape),
            "bias": tuple(bias.shape),
        })
        if hidden.ndim != 2 or hidden.shape[1] != expected_width:
            raise ValueError(...)
        if bias.shape != (weight.shape[0],):
            raise ValueError(...)
        hidden = torch.relu(hidden @ weight.T + bias)
    return hidden, trace

def main():
    print(E15(...))

if __name__ == "__main__":
    main()
```

13.2.17 Exercise 16 - Stress Test Forward Pass with Large Batch Sizes

Part 1 - Why this matters

Large batches test throughput, peak memory, numerical finiteness, and the ability of the architecture to process realistic workloads. Activation memory can exceed parameter memory by a large factor.

A stress test should increase load gradually and retain the last successful configuration. Out-of-memory behavior is part of the result and must be handled safely.

Part 2 - Mathematical form

$$M_{\text{activations}} \approx qB \sum_{l=1}^{L} d_l \text{ bytes.}$$

Part 3 - Hints and Helper

Use inference mode when testing forward-only memory, synchronize CUDA around timing, and report batch size, latency, throughput, and peak memory.

Part 4 - Python code skeleton

```
import torch

def E16(model, batch_sizes, input_size, device):
    results = []
    model = model.to(device).eval()
    for batch_size in batch_sizes:
        try:
            features = torch.randn(batch_size, input_size, device=device)
            output, latency, peak_memory = benchmark_forward(
                model, features, ...
            )
            results.append({
                "batch_size": batch_size,
                "latency": latency,
                "throughput": batch_size / latency,
                "peak_memory": peak_memory,
                "finite": torch.isfinite(output).all().item(),
            })
        except torch.OutOfMemoryError:
            results.append({"batch_size": batch_size, "status": "oom"})
            break
    return results

def main():
    print(E16(...))

if __name__ == "__main__":
    main()
```

13.2.18 Exercise 17 - Validate Determinism of Forward Pass Under Fixed Seeds

Part 1 - Why this matters

A fixed model state and identical input should normally produce repeatable deterministic forward outputs when the selected operations are deterministic. Random initialization and

stochastic layers must be controlled separately.

Dropout behaves randomly in training mode even under a fixed initial seed if repeated calls advance the random state. Evaluation mode disables it and changes the meaning of the determinism test.

Part 2 - Mathematical form

$$f_{\theta,s}(X) = f_{\theta,s}(X) \text{ for repeated deterministic evaluation under fixed state.}$$

Part 3 - Hints and Helper

Seed all relevant generators, enable deterministic algorithms, fix model mode, and compare repeated outputs both bitwise and within tolerance.

Part 4 - Python code skeleton

```
import torch

def E17(build_model, features, seed, repeats):
    torch.manual_seed(seed)
    model = build_model().eval()
    torch.use_deterministic_algorithms(True)
    outputs = []
    with torch.inference_mode():
        for _ in range(repeats):
            outputs.append(model(features).clone())
    reference = outputs[0]
    return {
        "exact": [torch.equal(reference, output) for output in outputs],
        "close": [torch.allclose(reference, output) for output in outputs],
        "outputs": outputs,
    }

def main():
    print(E17(...))

if __name__ == "__main__":
    main()
```

13.2.19 Exercise 18 - Examine Numerical Stability of Deep Forward Computation

Part 1 - Why this matters

Repeated affine transformations can cause activation magnitudes to shrink, grow, saturate, or overflow with depth. Initialization, activation choice, normalization, dtype, and input scale determine how signals propagate.

The final output may be finite even after useful variation has collapsed. Layer-wise statistics are necessary to detect loss of signal before visible failure.

Part 2 - Mathematical form

$$v_l = \mathrm{Var}(H_l), \qquad r_l = \frac{v_l}{v_{l-1} + \varepsilon}, \qquad \mathrm{finite}(H_l)\ \forall l.$$

Part 3 - Hints and Helper

Sweep depth, dtype, and initialization scale. Record mean, variance, extrema, saturation rate, and finiteness at every layer.

Part 4 - Python code skeleton

```
import torch

def E18(layer_outputs):
    report = []
    previous_variance = None
    for layer_index, hidden in enumerate(layer_outputs):
        variance = hidden.var(unbiased=False)
        report.append({
            "layer": layer_index,
            "mean": hidden.mean().item(),
            "variance": variance.item(),
            "variance_ratio": None if previous_variance is None else ...,
            "maximum_absolute": hidden.abs().max().item(),
            "finite": torch.isfinite(hidden).all().item(),
        })
        previous_variance = variance
    return report

def main():
    print(E18(...))

if __name__ == "__main__":
    main()
```

13.2.20 Exercise 19 - Write a Conceptual Summary Explaining Why MLP Enables Universal Approximation

Part 1 - Why this matters

Universal approximation results show that, under suitable conditions, a feed-forward network with a non-linear activation and enough hidden units can approximate continuous functions on compact domains as closely as desired. This explains the broad representational potential of MLPs.

The theorem does not guarantee that a finite network is small, that training will find the required parameters, that data are sufficient, or that the learned function will generalize. Approximation capacity, optimization, computation, and statistical learning remain separate problems.

Part 2 - Mathematical form

$$\forall f \in C(K),\ \forall \varepsilon > 0,\ \exists N :\ \sup_{\mathbf{x} \in K} |f(\mathbf{x}) - N(\mathbf{x})| < \varepsilon.$$

Part 3 - Hints and Helper

State the domain, target function class, activation assumptions, and approximation norm. Separate theorem claims from practical training claims.

Part 4 - Python code skeleton

```
import torch

def E19(sections):
    required = {
        "single_perceptron_limit", "layered_composition",
        "hidden_representations", "nonlinearity", "dimensions",
        "depth", "universal_approximation_statement",
        "activation_assumptions", "compact_domain",
        "optimization_limitations", "generalization_limitations",
        "computational_limitations",
    }
    missing = sorted(required - sections.keys())
    warnings = []
    if sections.get("claims_training_always_succeeds"):
        warnings.append(...)
    if sections.get("claims_finite_data_is_sufficient"):
        warnings.append(...)
    return {
        "complete": not missing and not warnings,
        "missing": missing,
        "warnings": warnings,
        "sections": sections,
    }

def main():
    print(E19(...))

if __name__ == "__main__":
    main()
```

13.3 MLP01 - Architecture Design and Layer Composition

Models and Training

13.3.1 Exercise 00 - Design a Configurable MLP Architecture with Variable Depth

Part 1 - Why this matters

A configurable architecture separates the definition of a network from one fixed implementation. Instead of rewriting the model whenever the number or width of hidden layers changes, the reader represents the architecture as data and lets the program construct the corresponding transformations.

This design makes controlled experiments possible because depth, width, and output size can change while the surrounding training pipeline remains stable. It also introduces an important engineering principle: every valid configuration must preserve a continuous dimension flow from the input features to the final output.

Part 2 - Mathematical form

$$\mathbf{d} = (d_0, d_1, \ldots, d_L), \qquad f(\mathbf{x}) = f_L \circ f_{L-1} \circ \cdots \circ f_1(\mathbf{x}), \quad f_l : \mathbb{R}^{d_{l-1}} \to \mathbb{R}^{d_l}.$$

Part 3 - Hints and Helper

Accept the layer widths as a sequence. Validate that it contains at least an input and an output width, and do not hard-code the number of hidden layers.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def E00(layer_sizes, activation_factory):
    if len(layer_sizes) < 2:
        raise ValueError(...)
    layers = []
    for index in range(len(layer_sizes) - 1):
        layers.append(nn.Linear(..., ...))
        if index < len(layer_sizes) - 2:
            layers.append(activation_factory())
    return nn.Sequential(...)

def main():
    model = E00([8, 16, 12, 4], nn.ReLU)
    print(model)

if __name__ == "__main__":
    main()
```

13.3.2 Exercise 01 - Implement MLP Using nn.Module and nn.Linear Layers

Part 1 - Why this matters

Subclassing nn.Module gives an MLP a clear computational interface while allowing PyTorch to discover its trainable state. Each nn.Linear layer owns a weight matrix and bias vector, and the forward method specifies how data pass through those registered transformations.

The exercise connects mathematical composition to the module system used in real training projects. A correct implementation must define layers during initialization, call them in the intended order, and apply non-linearity between linear transformations rather than accidentally omitting it.

Part 2 - Mathematical form

$$\mathbf{h}_1 = \phi(\mathbf{x}W_1^\mathsf{T} + \mathbf{b}_1), \qquad \hat{\mathbf{y}} = \mathbf{h}_1 W_2^\mathsf{T} + \mathbf{b}_2.$$

Part 3 - Hints and Helper

Create the submodules in the initializer and use them inside the forward method. Inspect the model parameters to confirm that both linear layers are registered.

Part 4 - Python code skeleton

```
import torch
from torch import nn

class MLP(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super().__init__()
        self.hidden = nn.Linear(..., ...)
        self.activation = nn.ReLU()
        self.output = nn.Linear(..., ...)

    def forward(self, features):
        hidden = self.activation(...)
        return ...

def E01(features):
    model = MLP(features.shape[-1], ..., ...)
    return model(...)

def main():
    print(E01(torch.randn(5, 8)))

if __name__ == "__main__":
    main()
```

13.3.3 Exercise 02 - Dynamically Build Network from Layer Configuration List

Part 1 - Why this matters

A layer configuration list turns architecture construction into a systematic transformation. This is useful for experiments, configuration files, and automated searches where the network layout is selected at runtime rather than written directly into the source code.

Dynamic construction must still preserve architectural intent. The reader must distinguish hidden transformations from the output transformation, place activation functions only where intended, and reject invalid widths before a malformed network reaches training.

Part 2 - Mathematical form

$$(d_0, d_1, \ldots, d_L) \longmapsto \{(W_l, \mathbf{b}_l)\}_{l=1}^{L}, \qquad W_l \in \mathbb{R}^{d_l \times d_{l-1}}.$$

Part 3 - Hints and Helper

Iterate over adjacent pairs from the configuration list. Decide explicitly whether the output layer receives an activation.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def E02(layer_sizes, activation_factory):
    modules = []
    pairs = zip(layer_sizes[:-1], layer_sizes[1:])
    for index, (in_size, out_size) in enumerate(pairs):
        modules.append(nn.Linear(..., ...))
        is_hidden = index < ...
        if is_hidden:
            modules.append(...)
    return nn.Sequential(*modules)

def main():
    model = E02([10, 32, 16, 2], nn.GELU)
    print(model(torch.randn(4, 10)))

if __name__ == "__main__":
    main()
```

13.3.4 Exercise 03 - Register Parameters Correctly for Arbitrary Network Depth

Part 1 - Why this matters

PyTorch optimizers can update only parameters that are registered under a module. Storing layers in an ordinary Python list may produce a forward pass that appears correct while silently hiding the weights from model.parameters(), state dictionaries, and device transfers.

Networks with arbitrary depth therefore require containers that register their modules. This exercise exposes a subtle failure mode in which computation succeeds but learning, saving, or CUDA placement is incomplete because the framework cannot see part of the model.

Part 2 - Mathematical form

$$\Theta = \bigcup_{l=1}^{L} \{W_l, \mathbf{b}_l\}, \qquad \Theta_{\text{registered}} = \Theta.$$

Part 3 - Hints and Helper

Build one correct model with a registered module container and one broken model with a plain list. Compare the named parameters and explain the missing entries.

Part 4 - Python code skeleton

```
import torch
from torch import nn

class DeepMLP(nn.Module):
    def __init__(self, layer_sizes):
        super().__init__()
        self.layers = nn.ModuleList([
            nn.Linear(in_size, out_size)
            for in_size, out_size in zip(..., ...)
        ])

    def forward(self, features):
        hidden = features
        for index, layer in enumerate(self.layers):
            hidden = layer(hidden)
            if index < len(self.layers) - 1:
                hidden = torch.relu(...)
        return hidden

def E03(layer_sizes):
    model = DeepMLP(layer_sizes)
    return model, dict(model.named_parameters())

def main():
    print(E03([6, 12, 8, 1]))

if __name__ == "__main__":
    main()
```

13.3.5 Exercise 04 - Verify Parameter Count for Custom Architectures

Part 1 - Why this matters

Parameter counting connects architectural choices to storage cost, optimization difficulty, and model capacity. A seemingly small increase in adjacent layer widths can create a large weight matrix and dominate the total number of trainable values.

Manual counting also provides a direct correctness test for dynamically constructed networks. If the theoretical count differs from the registered count, the architecture may contain an unintended layer, a missing bias, or parameters that were not registered correctly.

Part 2 - Mathematical form

$$P = \sum_{l=1}^{L} \left(d_l d_{l-1} + d_l \right), \qquad P_{\text{trainable}} = \sum_{\theta \in \Theta,\, \theta \text{ trainable}} \text{numel}(\theta).$$

Part 3 - Hints and Helper

Compute the formula from the width list, then compare it with the sum of `numel()` over parameters whose `requires_grad` flag is true.

Part 4 - Python code skeleton

```
import torch

def E04(model, layer_sizes):
    expected = 0
    for in_size, out_size in zip(..., ...):
        weights = ...
        biases = ...
        expected += weights + biases
    registered = sum(
        parameter.numel()
        for parameter in model.parameters()
        if parameter.requires_grad
    )
    return expected, registered, ...

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

13.3.6 Exercise 05 - Inspect Weight and Bias Shapes for Each Layer

Part 1 - Why this matters

Every linear layer defines a contract between its incoming and outgoing feature dimensions. Inspecting parameter shapes makes this contract visible and helps the reader relate PyTorch storage conventions to batch-first matrix multiplication.

Shape inspection is especially important in deep configurable models because an early width mistake propagates into later layers. A structured report can reveal the first incompatible transition before the error becomes buried inside a long forward trace.

Part 2 - Mathematical form

$$W_l \in \mathbb{R}^{d_l \times d_{l-1}}, \qquad \mathbf{b}_l \in \mathbb{R}^{d_l}, \qquad H_l \in \mathbb{R}^{B \times d_l}.$$

Part 3 - Hints and Helper

Iterate through `named_modules()`, select `nn.Linear` instances, and record the input width, output width, weight shape, and bias shape.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def E05(model):
    report = []
    for name, module in model.named_modules():
        if isinstance(module, nn.Linear):
            report.append({
                "name": name,
                "weight_shape": tuple(...),
                "bias_shape": tuple(...) if module.bias is not None else None,
                "transition": (..., ...),
            })
    return report

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

13.3.7 Exercise 06 - Implement Forward Hook to Capture Intermediate Outputs

Part 1 - Why this matters

Forward hooks allow intermediate activations to be observed without rewriting a model's forward method. They are valuable for debugging, representation analysis, and verifying that signals retain sensible shapes and values across deep architectures.

Hooks must be managed carefully because they remain attached until removed and may retain tensors longer than expected. The exercise therefore treats observation as a temporary resource: attach hooks, execute one pass, save only the required information, and remove every handle.

Part 2 - Mathematical form

$$H_l = \phi_l(H_{l-1}W_l^{\mathsf{T}} + \mathbf{b}_l), \qquad \mathcal{R}_l = (\text{shape}(H_l), \min H_l, \max H_l).$$

Part 3 - Hints and Helper

Create one closure per layer name. Detach captured tensors when gradients are unnecessary, and store the removable handles returned by `register_forward_hook()`.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def E06(model, features):
    captured = {}
    handles = []

    def make_hook(name):
        def hook(module, inputs, output):
            captured[name] = ...
        return hook

    for name, module in model.named_modules():
        if isinstance(module, nn.Linear):
            handles.append(module.register_forward_hook(...))
    output = model(features)
    for handle in handles:
        handle.remove()
    return output, captured

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

13.3.8 Exercise 07 - Compare Shallow vs Deep Architectures on Same Task

Part 1 - Why this matters

A fair architecture comparison changes depth while controlling the dataset, evaluation protocol, optimizer budget, and preferably parameter count. Without these controls, an apparent benefit may come from greater capacity or additional training rather than depth itself.

Shallow and deep networks can express different functions efficiently, but depth also changes optimization behavior and computational cost. The reader should compare training loss, validation performance, runtime, and parameter count instead of reducing the experiment to one final accuracy value.

Part 2 - Mathematical form

$$\Delta_m = m(f_{\text{deep}}) - m(f_{\text{shallow}}), \qquad \Delta_P = P_{\text{deep}} - P_{\text{shallow}}, \qquad \Delta_t = t_{\text{deep}} - t_{\text{shallow}}.$$

Part 3 - Hints and Helper

Use identical data splits and random-seed policy. Record capacity and compute alongside predictive metrics, and repeat the comparison across several seeds.

Part 4 - Python code skeleton

```
import torch

def E07(build_model, train_and_evaluate, configurations, seeds):
    results = []
    for seed in seeds:
        torch.manual_seed(...)
        for name, layer_sizes in configurations.items():
            model = build_model(...)
            metrics = train_and_evaluate(model, ...)
            results.append({
                "seed": seed,
                "architecture": name,
                "parameters": sum(p.numel() for p in model.parameters()),
                "metrics": ...,
            })
    return results

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

13.3.9 Exercise 08 - Analyze Trade-Off Between Width and Depth

Part 1 - Why this matters

Width determines how many features a layer can represent simultaneously, whereas depth determines how many stages of transformation can be composed. Both can increase expressiveness, but they create different parameter, memory, latency, and optimization profiles.

A meaningful trade-off analysis compares architectures under a declared constraint such as equal parameter count, equal latency, or equal validation performance. This prevents the conclusion from being predetermined by giving one model substantially more resources than another.

Part 2 - Mathematical form

$$P = \sum_{l=1}^{L}(d_{l-1}+1)d_l, \qquad A = \sum_{l=1}^{L} Bd_l, \qquad C \propto \sum_{l=1}^{L} Bd_{l-1}d_l.$$

Part 3 - Hints and Helper

Generate several width-depth configurations, calculate theoretical parameters and activation elements, and compare them with measured validation and timing results.

Part 4 - Python code skeleton

```
import torch

def E08(configurations, build_model, benchmark):
    analysis = []
    for layer_sizes in configurations:
        model = build_model(layer_sizes)
        parameter_count = sum(...)
        activation_elements = sum(...)
        measurements = benchmark(model, ...)
        analysis.append({
            "layers": layer_sizes,
            "parameters": parameter_count,
            "activation_elements": activation_elements,
            "measurements": measurements,
        })
    return analysis

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

13.3.10 Exercise 09 - Detect Architecture-Induced Bottlenecks

Part 1 - Why this matters

A narrow hidden layer forces information through a lower-dimensional representation. This may be useful when compression is intentional, but an accidental bottleneck can discard task-relevant variation and restrict every later layer regardless of its width.

Bottlenecks can be detected structurally and empirically. The reader should inspect abrupt width reductions, activation rank, saturation, and downstream performance, while remembering that a narrow layer is not automatically defective if the data truly lie on a low-dimensional manifold.

Part 2 - Mathematical form

$$\operatorname{rank}(H_l) \le \min(B, d_l), \qquad d_l \ll \min(d_{l-1}, d_{l+1}) \implies \text{candidate bottleneck.}$$

Part 3 - Hints and Helper

Flag sharp contractions in the width list, then use captured activations to estimate matrix rank and inactive-unit ratios for each hidden layer.

Part 4 - Python code skeleton

```
import torch

def E09(layer_sizes, activations, contraction_ratio=0.25):
    report = []
    for index, hidden in enumerate(activations):
        previous_width = layer_sizes[index]
        width = layer_sizes[index + 1]
        next_width = layer_sizes[index + 2]
        structural = width < contraction_ratio * min(..., ...)
        rank = torch.linalg.matrix_rank(hidden.float()).item()
        inactive = (...).float().mean().item()
        report.append({
            "layer": index,
            "structural": structural,
            "rank": rank,
            "inactive_ratio": inactive,
        })
    return report

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

13.3.11 Exercise 10 - Move Entire Architecture to CUDA and Validate Placement

Part 1 - Why this matters

Moving a model to CUDA must relocate every registered parameter and buffer, not merely the input tensor or one visible layer. A partially moved architecture fails during computation because PyTorch operations require participating tensors to share a compatible device.

Device validation is also a registration test. If model.to(device) leaves a layer on the CPU, that layer may have been stored outside the module hierarchy, revealing an architectural defect that would also affect optimization and serialization.

Part 2 - Mathematical form

$$\forall \theta \in \Theta,\ \text{device}(\theta) = D, \qquad \text{device}(X) = D \implies \text{device}(f_\Theta(X)) = D.$$

Part 3 - Hints and Helper

Select CUDA only when it is available. Check every named parameter and named buffer after moving the model, then run a sample forward pass on the same device.

Part 4 - Python code skeleton

```
import torch

def E10(model, features):
    device = torch.device(
        "cuda" if torch.cuda.is_available() else "cpu"
    )
    model = model.to(...)
    features = features.to(...)
    parameter_devices = {
        name: parameter.device
        for name, parameter in model.named_parameters()
    }
    all_placed = all(value == device for value in ...)
    output = model(...)
    return output, parameter_devices, all_placed

def main():
    print(E10(...))

if __name__ == "__main__":
    main()
```

13.3.12 Exercise 11 - Debug Device Mismatch Errors in Multi-Layer Networks

Part 1 - Why this matters

Device mismatch errors often appear at the first operation that combines tensors, even when the real cause was an earlier unregistered layer, a newly created CPU tensor, or an input batch that was never transferred. Deep networks make the source harder to locate because many

modules may execute correctly before the failure.

A systematic debugger records the device of inputs, parameters, buffers, and intermediate outputs layer by layer. This turns a vague runtime exception into an exact report of the first transition whose tensors are not colocated.

Part 2 - Mathematical form

$$D(H_{l-1}) = D(W_l) = D(\mathbf{b}_l) \quad \text{for every } l.$$

Part 3 - Hints and Helper

Inspect devices before changing them. Avoid silently moving individual tensors inside forward because that can hide the architectural cause and add repeated transfer costs.

Part 4 - Python code skeleton

```
import torch

def E11(model, features):
    report = {"input": str(features.device), "parameters": {}}
    for name, parameter in model.named_parameters():
        report["parameters"][name] = str(...)
    unique_devices = set(report["parameters"].values())
    expected = str(features.device)
    mismatches = [
        name for name, device in report["parameters"].items()
        if device != ...
    ]
    return {
        "unique_devices": unique_devices,
        "mismatches": mismatches,
        "ready": ...,
    }

def main():
    print(E11(...))

if __name__ == "__main__":
    main()
```

13.3.13 Exercise 12 - Test Forward Compatibility with Different Input Shapes

Part 1 - Why this matters

An MLP generally treats the final tensor dimension as the feature dimension and preserves all leading dimensions. This allows the same linear layers to process a single sample, a batch,

or additional grouped dimensions when the final width matches the architecture input size.

Compatibility should be defined deliberately rather than inferred from one successful example. The reader must separate valid leading-dimension variation from invalid feature widths and verify the exact output-shape contract for every accepted input form.

Part 2 - Mathematical form

$$X \in \mathbb{R}^{s_1 \times \cdots \times s_k \times d_0} \implies f(X) \in \mathbb{R}^{s_1 \times \cdots \times s_k \times d_L}.$$

Part 3 - Hints and Helper

Test scalar-free single samples, ordinary batches, and multi-leading-dimension inputs. Include one intentionally wrong final dimension and capture the expected failure.

Part 4 - Python code skeleton

```
import torch

def E12(model, input_size, output_size):
    shapes = [(input_size,), (4, input_size), (2, 3, input_size)]
    report = []
    for shape in shapes:
        features = torch.randn(*shape)
        output = model(...)
        expected = shape[:-1] + (...,)
        report.append((shape, tuple(output.shape), ...))
    try:
        model(torch.randn(4, input_size + 1))
        invalid_rejected = False
    except RuntimeError:
        invalid_rejected = ...
    return report, invalid_rejected

def main():
    print(E12(...))

if __name__ == "__main__":
    main()
```

13.3.14 Exercise 13 - Clone Architecture and Compare Parameter Independence

Part 1 - Why this matters

Two models may have the same architecture while requiring completely independent parameter storage. Simple assignment creates another reference to the same object, so modifying one name modifies the same underlying model rather than an independent clone.

Parameter independence matters for ensemble members, experiment baselines, target networks, and checkpoint comparisons. The reader must distinguish equality of initial values from identity of storage and confirm that a later update to one clone does not alter the other.

Part 2 - Mathematical form

$$\theta_l^{(A)} = \theta_l^{(B)} \text{ initially}, \qquad \text{ptr}(\theta_l^{(A)}) \neq \text{ptr}(\theta_l^{(B)}).$$

Part 3 - Hints and Helper

Use `deepcopy` for a full independent clone. Compare `data_ptr()` values, then mutate one model under `torch.no_grad()` and test whether the other remains unchanged.

Part 4 - Python code skeleton

```
import copy
import torch

def E13(model):
    clone = copy.deepcopy(...)
    pairs = zip(model.parameters(), clone.parameters())
    value_equal = []
    storage_independent = []
    for original, copied in pairs:
        value_equal.append(torch.equal(..., ...))
        storage_independent.append(original.data_ptr() != ...)
    before = [parameter.detach().clone() for parameter in clone.parameters()]
    with torch.no_grad():
        next(model.parameters()).add_(...)
    unchanged = all(
        torch.equal(old, new)
        for old, new in zip(before, clone.parameters())
    )
    return clone, value_equal, storage_independent, unchanged

def main():
    print(E13(...))

if __name__ == "__main__":
    main()
```

13.3.15 Exercise 14 - Save and Reload Architecture with State Dict

Part 1 - Why this matters

A state dictionary stores learned parameters and persistent buffers while leaving the architecture definition in code. Reloading therefore succeeds only when the destination model is constructed with a compatible arrangement of layer names and tensor shapes.

Serialization should be verified by behavior, not merely by the absence of an exception. After loading, the reader should compare keys, parameters, and predictions under evaluation mode using the same input tensor.

Part 2 - Mathematical form

$$S = \{k \mapsto T_k\}, \qquad S_A \cong S_B \implies f_A(X) = f_B(X) \text{ after loading.}$$

Part 3 - Hints and Helper

Save only the `state_dict` for this exercise. Rebuild the same architecture, load with `strict=True`, and compare outputs with `torch.allclose()`.

Part 4 - Python code skeleton

```
import io
import torch

def E14(model, model_factory, features):
    buffer = io.BytesIO()
    torch.save(..., buffer)
    buffer.seek(0)
    restored = model_factory()
    state = torch.load(buffer, map_location=features.device)
    load_result = restored.load_state_dict(..., strict=True)
    model.eval()
    restored.eval()
    with torch.no_grad():
        original_output = model(features)
        restored_output = restored(features)
    return restored, load_result, torch.allclose(..., ...)

def main():
    print(E14(...))

if __name__ == "__main__":
    main()
```

13.3.16 Exercise 15 - Stress Test Architecture Creation with Large Layer Counts

Part 1 - Why this matters

Very deep architecture construction tests the scalability of configuration logic before expensive training begins. It can expose recursion assumptions, registration mistakes, initialization cost, and naming behavior that remain invisible in a small demonstration network.

Creation success alone is not enough: the resulting model must contain the expected number of layers and parameters, and a forward pass must preserve valid shapes. The stress test

should use explicit resource limits so it investigates construction without accidentally exhausting memory.

Part 2 - Mathematical form

$$L \gg 1, \qquad P = \sum_{l=1}^{L} (d_{l-1} + 1) d_l, \qquad \#\{\text{registered linear layers}\} = L.$$

Part 3 - Hints and Helper

Begin with narrow layers and increase depth gradually. Measure construction time, confirm module count, and run a small batch before attempting wider configurations.

Part 4 - Python code skeleton

```
import time
import torch
from torch import nn

def E15(build_model, depth, width, input_size, output_size):
    layer_sizes = [input_size] + [width] * depth + [output_size]
    start = time.perf_counter()
    model = build_model(...)
    creation_time = time.perf_counter() - start
    linear_count = sum(
        isinstance(module, nn.Linear) for module in model.modules()
    )
    with torch.no_grad():
        output = model(torch.randn(2, input_size))
    return {
        "model": model,
        "creation_time": creation_time,
        "linear_count": linear_count,
        "output_shape": tuple(...),
    }

def main():
    print(E15(...))

if __name__ == "__main__":
    main()
```

13.3.17 Exercise 16 - Evaluate Memory Footprint of Deep Architectures

Part 1 - Why this matters

Model memory includes parameter values, gradients, optimizer state, and activations retained for backward computation. Counting parameter bytes alone can substantially underestimate training requirements, especially when batch size and hidden widths make activations

dominant.

A useful memory report distinguishes persistent architecture storage from the runtime memory required by a workload. This helps the reader predict whether a design fits the selected device and identify whether reducing width, depth, precision, or batch size will address the actual source of pressure.

Part 2 - Mathematical form

$$M_{\text{train}} \approx M_{\text{param}} + M_{\text{grad}} + M_{\text{optimizer}} + M_{\text{activation}},$$
$$M_{\text{param}} = \sum_{\theta \in \Theta} \text{numel}(\theta)\, \text{size}(\theta).$$

Part 3 - Hints and Helper

Compute exact parameter and buffer bytes first. On CUDA, reset peak memory statistics before the measured forward-backward step and synchronize before reading the result.

Part 4 - Python code skeleton

```
import torch

def E16(model, features, loss_function):
    parameter_bytes = sum(
        parameter.numel() * parameter.element_size()
        for parameter in model.parameters()
    )
    buffer_bytes = sum(
        buffer.numel() * buffer.element_size()
        for buffer in model.buffers()
    )
    peak_bytes = None
    if features.is_cuda:
        torch.cuda.reset_peak_memory_stats(features.device)
    output = model(features)
    loss = loss_function(output, ...)
    loss.backward()
    if features.is_cuda:
        torch.cuda.synchronize(features.device)
        peak_bytes = torch.cuda.max_memory_allocated(...)
    return parameter_bytes, buffer_bytes, peak_bytes

def main():
    print(E16(...))

if __name__ == "__main__":
    main()
```

13.3.18 Exercise 17 - Compare Manual Tensor Implementation vs nn.Module

Part 1 - Why this matters

A manual tensor implementation makes every affine transformation and activation explicit, while nn.Module organizes the same computation into registered, reusable components. Comparing them reveals which responsibilities belong to the mathematics and which are supplied by the framework.

The comparison is valid only when both paths use identical parameter values, matrix orientation, activation placement, and bias broadcasting. Matching outputs then demonstrate computational equivalence, while differences in registration, serialization, and device management reveal the practical value of the module abstraction.

Part 2 - Mathematical form

$$H_l^{\text{manual}} = \phi_l(H_{l-1}W_l^{\mathsf{T}} + \mathbf{b}_l), \qquad H_l^{\text{module}} = \phi_l(\text{Linear}_l(H_{l-1})).$$

Part 3 - Hints and Helper

Copy each module weight and bias into the manual parameter lists. Run both paths in evaluation mode and report maximum absolute and relative differences.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def E17(model, features, activation):
    manual = features
    linear_layers = [
        module for module in model.modules()
        if isinstance(module, nn.Linear)
    ]
    for index, layer in enumerate(linear_layers):
        manual = manual @ layer.weight.T + layer.bias
        if index < len(linear_layers) - 1:
            manual = activation(...)
    with torch.no_grad():
        modular = model(...)
    difference = (manual - modular).abs()
    return manual, modular, difference.max().item(), torch.allclose(..., ...)

def main():
    print(E17(...))

if __name__ == "__main__":
    main()
```

13.3.19 Exercise 18 - Identify Architectural Failure Modes Early

Part 1 - Why this matters

Architectural defects are cheaper to detect before training consumes data and compute. Invalid widths, missing non-linearities, unregistered layers, incompatible output dimensions, and mixed devices can all make a model impossible to train or silently different from the intended design.

An architecture preflight check converts assumptions into executable conditions. It should inspect structure, registration, dimension flow, devices, data types, forward output, and gradient reachability using a small synthetic batch before the real training loop begins.

Part 2 - Mathematical form

$$\mathcal{V} = V_{\text{shape}} \wedge V_{\text{register}} \wedge V_{\text{device}} \wedge V_{\text{forward}} \wedge V_{\text{gradient}}.$$

Part 3 - Hints and Helper

Return a structured list of failures instead of stopping after the first one. Include both static checks and a safe synthetic forward-backward test.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def E18(model, input_size, output_size, device):
    failures = []
    if not list(model.parameters()):
        failures.append(...)
    parameter_devices = {parameter.device for parameter in model.parameters()}
    if parameter_devices != {device}:
        failures.append(...)
    try:
        sample = torch.randn(3, input_size, device=device)
        output = model(sample)
        if output.shape != (3, output_size):
            failures.append(...)
        output.sum().backward()
        missing_gradients = [
            name for name, parameter in model.named_parameters()
            if parameter.requires_grad and parameter.grad is None
        ]
        if missing_gradients:
            failures.append(...)
    except (RuntimeError, ValueError) as error:
        failures.append(str(error))
    return {"ready": not failures, "failures": failures}
```

```
def main():
    print(E18(...))

if __name__ == "__main__":
    main()
```

13.3.20 Exercise 19 - Produce an Architecture Design Rationale Document

Part 1 - Why this matters

An architecture should be justified by the input structure, output objective, available data, compute budget, and risks discovered during validation. A rationale document makes these decisions reviewable instead of presenting layer widths and activation choices as unexplained guesses.

The document must separate evidence from assumptions and state what alternatives were considered. It should also define validation criteria and failure signals so future experiments can confirm, revise, or reject the proposed architecture rather than preserving it by habit.

Part 2 - Mathematical form

$$A^* = \arg\min_{A \in \mathcal{A}} \Big(\widehat{R}_{\text{val}}(A) + \lambda_C C(A) + \lambda_M M(A) + \lambda_L L(A) \Big),$$

Part 3 - Hints and Helper

Require sections for requirements, dimension flow, depth-width choice, activation choice, parameter budget, memory, alternatives, risks, tests, and revision conditions.

Part 4 - Python code skeleton

```
def E19(sections):
    required = {
        "problem_requirements", "input_output_contract",
        "layer_configuration", "depth_width_rationale",
        "activation_rationale", "parameter_budget",
        "memory_and_latency", "alternatives_considered",
        "failure_modes", "validation_plan",
        "revision_conditions",
    }
    missing = sorted(required - sections.keys())
    unsupported = [
        key for key, value in sections.items()
        if isinstance(value, dict)
        and value.get("claim")
        and not value.get("evidence")
    ]
    return {
        "complete": not missing and not unsupported,
```

```
        "missing": missing,
        "unsupported_claims": unsupported,
        "document": sections,
    }

def main():
    print(E19(...))

if __name__ == "__main__":
    main()
```

13.4 MLP02 - Activation Functions and Non-Linearity

Models and Training

13.4.1 Exercise 00 - Implement ReLU Activation and Observe Its Piecewise Behavior

Part 1 - Why this matters

The rectified linear unit keeps positive values unchanged and maps negative values to zero. Its piecewise-linear form is simple, inexpensive, and capable of introducing the non-linearity that prevents stacked affine layers from collapsing into one affine transformation.

The point at zero separates two behaviors and makes the derivative undefined in classical calculus, so software adopts a conventional subgradient there. Observing values and gradients on both sides of zero connects the mathematical definition to PyTorch's actual backward behavior.

Part 2 - Mathematical form

$$\mathrm{ReLU}(x) = \max(0, x) = \begin{cases} 0, & x \le 0, \\ x, & x > 0, \end{cases} \qquad \mathrm{ReLU}'(x) = \begin{cases} 0, & x \le 0, \\ 1, & x > 0. \end{cases}$$

Part 3 - Hints and Helper

Create inputs below, at, and above zero. Compare a manual maximum operation with the built-in function, then inspect the gradients produced by backward.

Part 4 - Python code skeleton

```
import torch

def E00(values):
    values = values.clone().requires_grad_(True)
    manual = torch.maximum(values, torch.zeros_like(values))
    built_in = torch.relu(...)
    built_in.sum().backward()
```

```
    return manual, built_in, values.grad, torch.equal(..., ...)

def main():
    values = torch.tensor([-2.0, -0.1, 0.0, 0.1, 2.0])
    print(E00(values))

if __name__ == "__main__":
    main()
```

13.4.2 Exercise 01 - Compare ReLU, Sigmoid, and Tanh in Forward Propagation

Part 1 - Why this matters

ReLU, sigmoid, and tanh transform the same pre-activation values into very different ranges. ReLU is unbounded above and sparse on negative inputs, sigmoid compresses values into probabilities between zero and one, and tanh produces zero-centered outputs between negative one and one.

These differences change the distributions delivered to later layers and therefore affect optimization. A controlled forward comparison should reuse the same inputs and parameters so the observed changes arise from the activation rather than from different network weights.

Part 2 - Mathematical form

$$r(z) = \max(0, z), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}, \qquad \tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}.$$

Part 3 - Hints and Helper

Calculate all three activations from one tensor. Report range, mean, standard deviation, and the proportion of exact zeros for each output.

Part 4 - Python code skeleton

```
import torch

def E01(pre_activation):
    functions = {
        "relu": torch.relu,
        "sigmoid": torch.sigmoid,
        "tanh": torch.tanh,
    }
    report = {}
    for name, function in functions.items():
        output = function(...)
        report[name] = {
            "output": output,
            "mean": output.mean().item(),
```

```
            "std": output.std().item(),
            "zero_ratio": (...).float().mean().item(),
        }
    return report

def main():
    print(E01(torch.linspace(-6, 6, 25)))

if __name__ == "__main__":
    main()
```

13.4.3 Exercise 02 - Visualize Activation Output Distributions Across Layers

Part 1 - Why this matters

Activation distributions reveal how signals evolve as they pass through a network. Means that drift, variances that collapse or explode, and large concentrations at activation limits can indicate that useful information is being lost before the output layer.

A layer-by-layer visualization is more informative than inspecting only the final prediction. Histograms should be collected from the same batch and paired with numerical summaries so that visual impressions can be verified quantitatively.

Part 2 - Mathematical form

$$H_l = \phi(Z_l), \qquad \mu_l = \mathbb{E}[H_l], \qquad s_l^2 = \mathrm{Var}(H_l).$$

Part 3 - Hints and Helper

Capture hidden outputs with hooks or an explicit forward trace. Detach the values, move them to the CPU, and plot comparable histogram ranges for all layers.

Part 4 - Python code skeleton

```
import torch
import matplotlib.pyplot as plt

def E02(activations, bins=40):
    figure, axes = plt.subplots(len(activations), 1, ...)
    summaries = []
    for index, (axis, values) in enumerate(zip(axes, activations)):
        flat = values.detach().float().cpu().flatten()
        axis.hist(..., bins=bins)
        axis.set_title(f"Layer {index}")
        summaries.append({
            "mean": flat.mean().item(),
            "std": flat.std().item(),
        })
```

```
    return figure, summaries

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

13.4.4 Exercise 03 - Analyze Gradient Flow for Different Activation Choices

Part 1 - Why this matters

Backpropagation multiplies local derivatives along computational paths. The activation derivative therefore controls how strongly an error signal reaches earlier layers, and repeated derivatives smaller than one can progressively weaken the gradient.

A fair experiment gives networks identical initial weights, inputs, targets, and loss functions while changing only the activation. Layer-wise gradient norms then show whether the signal remains balanced, vanishes, or becomes unstable across depth.

Part 2 - Mathematical form

$$\frac{\partial \mathcal{L}}{\partial H_{l-1}} = \frac{\partial \mathcal{L}}{\partial H_l} W_l \operatorname{diag}(\phi'(Z_{l-1})) .$$

Part 3 - Hints and Helper

Clone one initialized model for each activation. After one backward pass, record weight-gradient norms in forward layer order and compare their ratios.

Part 4 - Python code skeleton

```
import copy
import torch

def E03(base_model, activation_factories, features, targets, loss_fn):
    reports = {}
    for name, factory in activation_factories.items():
        model = copy.deepcopy(base_model)
        model.set_activation(...)
        model.zero_grad(set_to_none=True)
        loss = loss_fn(model(features), targets)
        loss.backward()
        reports[name] = [
            parameter.grad.norm().item()
            for parameter in model.weight_parameters()
        ]
    return reports
```

```
def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

13.4.5 Exercise 04 - Demonstrate Vanishing Gradient in Deep Sigmoid Networks

Part 1 - Why this matters

The sigmoid derivative never exceeds one quarter and becomes close to zero when its input is strongly positive or negative. Repeating this behavior through many layers can make gradients at early layers extremely small even though later layers still receive a measurable signal.

The experiment should demonstrate the mechanism rather than merely produce a failed training run. Recording pre-activation saturation and gradient norms across depth links the observed vanishing gradient to the local derivatives that caused it.

Part 2 - Mathematical form

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq \frac{1}{4}, \qquad \left\| \prod_{l=1}^{L} \sigma'(z_l) \right\| \longrightarrow 0 \text{ as } L \text{ grows.}$$

Part 3 - Hints and Helper

Build a deep sigmoid MLP, perform one backward pass, and compare the first and last layer gradient norms. Repeat across several depths using the same seed.

Part 4 - Python code skeleton

```
import torch

def E04(build_sigmoid_model, depths, features, targets, loss_fn):
    report = []
    for depth in depths:
        torch.manual_seed(0)
        model = build_sigmoid_model(...)
        loss = loss_fn(model(features), targets)
        loss.backward()
        norms = [
            layer.weight.grad.norm().item()
            for layer in model.linear_layers()
        ]
        report.append({
            "depth": depth,
            "first_norm": norms[0],
            "last_norm": norms[-1],
```

```
            "ratio": norms[0] / (... + 1e-12),
        })
    return report

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

13.4.6 Exercise 05 - Demonstrate Dying ReLU Phenomenon Experimentally

Part 1 - Why this matters

A ReLU unit becomes inactive when its pre-activation remains non-positive for all relevant samples. Its output and local derivative are then zero, so ordinary gradient descent may be unable to move the unit back into an active region.

Dying ReLU is influenced by initialization, data distribution, bias, and update size. The experiment should track individual units over time and distinguish a temporarily inactive unit from one that remains inactive across many batches.

Part 2 - Mathematical form

$$z_j^{(i)} \leq 0 \ \forall i \quad \Longrightarrow \quad h_j^{(i)} = 0 \text{ and } \frac{\partial \mathcal{L}}{\partial z_j^{(i)}} = 0 \ \forall i.$$

Part 3 - Hints and Helper

Record the fraction of hidden units whose output is zero for every sample in a batch. Use an aggressive learning rate as one experimental condition, not as an unexplained default.

Part 4 - Python code skeleton

```
import torch

def E05(model, data_loader, optimizer, loss_fn, epochs):
    history = []
    for epoch in range(epochs):
        for features, targets in data_loader:
            optimizer.zero_grad(set_to_none=True)
            output, hidden = model.forward_with_hidden(features)
            loss = loss_fn(output, targets)
            loss.backward()
            optimizer.step()
        dead_units = (hidden == 0).all(dim=0)
        history.append({
            "epoch": epoch,
```

```
            "dead_ratio": dead_units.float().mean().item(),
            "loss": ...,
        })
    return history

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

13.4.7 Exercise 06 - Implement LeakyReLU and Compare Gradient Behavior

Part 1 - Why this matters

LeakyReLU replaces ReLU's zero negative branch with a small linear slope. Negative inputs therefore retain a gradient path, reducing the chance that a unit becomes permanently disconnected from learning.

The slope is a design choice rather than a universal solution. Comparing several negative slopes reveals the trade-off between preserving gradient flow and changing the activation's asymmetry and output distribution.

Part 2 - Mathematical form

$$\mathrm{LeakyReLU}_\alpha(x) = \begin{cases} x, & x \geq 0, \\ \alpha x, & x < 0, \end{cases} \qquad \phi'(x) = \begin{cases} 1, & x \geq 0, \\ \alpha, & x < 0. \end{cases}$$

Part 3 - Hints and Helper

Implement the piecewise rule with a tensor operation. Compare values and gradients against the built-in function for negative, zero, and positive inputs.

Part 4 - Python code skeleton

```
import torch
from torch.nn import functional as F

def E06(values, negative_slope=0.01):
    values = values.clone().requires_grad_(True)
    manual = torch.where(values >= 0, values, negative_slope * values)
    built_in = F.leaky_relu(values, negative_slope=...)
    built_in.sum().backward()
    return {
        "manual": manual,
        "built_in": built_in,
        "gradients": values.grad,
        "matches": torch.allclose(..., ...),
```

```
    }

def main():
    print(E06(torch.tensor([-2.0, 0.0, 2.0])))

if __name__ == "__main__":
    main()
```

13.4.8 Exercise 07 - Test GELU Activation in Deep Networks

Part 1 - Why this matters

GELU scales an input smoothly according to its magnitude instead of applying a hard gate at zero. Small negative values can pass with reduced influence, while sufficiently positive values remain close to the identity transformation.

Its smooth behavior may improve some deep-network optimization problems, but the benefit depends on the task, initialization, and compute environment. The reader should compare learning curves and gradient statistics rather than assuming that a more complex activation is always superior.

Part 2 - Mathematical form

$$\mathrm{GELU}(x) = x\Phi(x) \approx \frac{x}{2}\left(1 + \tanh\left[\sqrt{\frac{2}{\pi}}\left(x + 0.044715x^3\right)\right]\right).$$

Part 3 - Hints and Helper

Train matched ReLU and GELU networks across the same seeds. Record convergence, gradient norms, validation performance, and execution time.

Part 4 - Python code skeleton

```
import copy
import time
import torch
from torch import nn

def E07(base_model, train_model, seeds):
    results = []
    for seed in seeds:
        for name, activation in {
            "relu": nn.ReLU(), "gelu": nn.GELU()
        }.items():
            torch.manual_seed(...)
            model = copy.deepcopy(base_model)
            model.set_activation(activation)
            start = time.perf_counter()
            metrics = train_model(model, ...)
```

```
            elapsed = time.perf_counter() - start
            results.append((seed, name, metrics, elapsed))
    return results

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

13.4.9 Exercise 08 - Evaluate Activation Choice Impact on Training Stability

Part 1 - Why this matters

Training stability includes more than the final loss. A stable run avoids non-finite values, uncontrolled gradient growth, severe oscillation, prolonged plateaus, and large sensitivity to the random seed.

Activation functions influence all of these behaviors through their output ranges and derivatives. Evaluation should therefore aggregate multiple runs and report failure frequency and variability alongside average predictive performance.

Part 2 - Mathematical form

$$S(\phi) = \Big(p_{\text{finite}}, \operatorname{Var}_s(L_T), \max_t \|g_t\|, \operatorname{osc}(L_{1:T})\Big).$$

Part 3 - Hints and Helper

Use the same architecture and hyperparameters for each activation. Define instability criteria before running the experiment so conclusions are not chosen after seeing results.

Part 4 - Python code skeleton

```
import math
import torch

def E08(run_training, activations, seeds):
    report = {}
    for name, activation in activations.items():
        runs = [run_training(activation, seed, ...) for seed in seeds]
        finite = [math.isfinite(run["final_loss"]) for run in runs]
        report[name] = {
            "success_rate": sum(finite) / len(finite),
            "mean_final_loss": ...,
            "loss_variance": ...,
            "max_gradient_norm": max(...),
            "runs": runs,
        }
```

```
    return report

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

13.4.10 Exercise 09 - Debug Activation-Induced Training Failures

Part 1 - Why this matters

An activation-related failure may appear as dead units, saturated outputs, vanishing gradients, exploding activations, or a mismatch between the output activation and loss function. These symptoms can resemble data, initialization, or optimizer problems.

Debugging should connect each symptom to evidence at the layer where it begins. Logging pre-activation values, activation statistics, and gradients makes it possible to identify whether the activation is a cause, an amplifier, or merely the location where another defect becomes visible.

Part 2 - Mathematical form

$$D_l = (p_{\text{dead},l}, p_{\text{sat},l}, \|H_l\|, \|\nabla_{W_l}\mathcal{L}\|, p_{\text{finite},l}).$$

Part 3 - Hints and Helper

Collect diagnostics before changing the activation. Compare the failing model with one controlled alternative while keeping the weights, batch, and loss fixed.

Part 4 - Python code skeleton

```
import torch

def E09(layer_records, gradient_records, threshold=1e-6):
    failures = []
    for name, values in layer_records.items():
        finite = torch.isfinite(values).all().item()
        near_zero = (values.abs() < threshold).float().mean().item()
        gradient_norm = gradient_records[name].norm().item()
        if not finite:
            failures.append((name, "non_finite_activation"))
        if near_zero > ...:
            failures.append((name, "mostly_inactive"))
        if gradient_norm < ...:
            failures.append((name, "vanishing_gradient"))
    return failures

def main():
```

```
    print(E09(...))

if __name__ == "__main__":
    main()
```

13.4.11 Exercise 10 - Execute Activation Functions on CUDA and Compare Performance

Part 1 - Why this matters

CUDA can accelerate activation processing when tensors are large enough to amortize transfer and kernel-launch costs. Small isolated operations may be faster on the CPU, so device choice should be based on synchronized measurements rather than assumptions.

A correct benchmark separates one-time setup from repeated execution and keeps data on the measured device. CUDA work is asynchronous, which means timing without synchronization can measure dispatch rather than completed computation.

Part 2 - Mathematical form

$$T_D(\phi; N) = \frac{1}{R} \sum_{r=1}^{R} \left(t_{r,\text{end}} - t_{r,\text{start}} \right), \qquad D \in \{\text{CPU}, \text{CUDA}\}.$$

Part 3 - Hints and Helper

Warm up both paths, synchronize CUDA around timed regions, and test several tensor sizes. Do not include host-to-device transfer unless transfer is explicitly part of the scenario.

Part 4 - Python code skeleton

```
import time
import torch

def benchmark(function, values, repeats):
    for _ in range(10):
        function(values)
    if values.is_cuda:
        torch.cuda.synchronize(values.device)
    start = time.perf_counter()
    for _ in range(repeats):
        output = function(...)
    if values.is_cuda:
        torch.cuda.synchronize(values.device)
    return (time.perf_counter() - start) / repeats, output

def E10(function, shape, repeats=100):
    cpu_values = torch.randn(*shape)
    cpu_time, cpu_output = benchmark(function, cpu_values, repeats)
    if not torch.cuda.is_available():
```

```
        return {"cpu": cpu_time, "cuda": None}
    cuda_values = cpu_values.to("cuda")
    cuda_time, cuda_output = benchmark(function, cuda_values, repeats)
    return {"cpu": cpu_time, "cuda": cuda_time, "matches": ...}

def main():
    print(E10(torch.relu, (4096, 4096)))

if __name__ == "__main__":
    main()
```

13.4.12 Exercise 11 - Compare Float Precision Effects on Activation Output

Part 1 - Why this matters

Floating-point precision affects the range and resolution with which activation outputs are represented. Saturating functions may reach exact limits sooner in lower precision, while exponential terms can underflow or overflow for less extreme inputs.

Precision differences should be measured against a higher-precision reference and separated into ordinary rounding error, saturation changes, and non-finite results. The impact on gradients can be more important than the forward difference alone.

Part 2 - Mathematical form

$$e_p(x) = |\phi_p(x) - \phi_{64}(x)|, \qquad r_p(x) = \frac{e_p(x)}{|\phi_{64}(x)| + \varepsilon}.$$

Part 3 - Hints and Helper

Evaluate a wide input range in several supported data types. Convert outputs to double precision before calculating errors against the reference.

Part 4 - Python code skeleton

```
import torch

def E11(function, values, dtypes):
    reference = function(values.to(torch.float64))
    report = {}
    for dtype in dtypes:
        output = function(values.to(dtype)).to(torch.float64)
        absolute = (output - reference).abs()
        relative = absolute / (reference.abs() + 1e-12)
        report[str(dtype)] = {
            "max_absolute": absolute.max().item(),
            "max_relative": relative.max().item(),
            "finite": torch.isfinite(output).all().item(),
```

```
        }
    return report

def main():
    values = torch.linspace(-50, 50, 10001)
    print(E11(torch.sigmoid, values, [...]))

if __name__ == "__main__":
    main()
```

13.4.13 Exercise 12 - Track Activation Saturation Across Layers

Part 1 - Why this matters

Saturation occurs when an activation enters a region where its derivative is very small. In sigmoid and tanh networks, a high saturation ratio indicates that many units transmit little gradient even if their outputs appear numerically valid.

The appropriate saturation rule depends on the activation. A derivative-based definition is more general than checking output values alone and allows statistics from different functions to be compared on a shared gradient-flow interpretation.

Part 2 - Mathematical form

$$s_l(\tau) = \frac{1}{|Z_l|} \sum_i \mathbf{1}(|\phi'(z_{l,i})| < \tau) \, .$$

Part 3 - Hints and Helper

Capture pre-activations, calculate the analytical derivative for the selected function, and report the fraction below a declared threshold at every layer.

Part 4 - Python code skeleton

```
import torch

def E12(pre_activations, derivative_function, threshold=1e-3):
    report = []
    for index, values in enumerate(pre_activations):
        derivatives = derivative_function(...)
        saturated = derivatives.abs() < threshold
        report.append({
            "layer": index,
            "saturation_ratio": saturated.float().mean().item(),
            "minimum_derivative": derivatives.abs().min().item(),
        })
    return report

def main():
```

```
    print(E12(...))

if __name__ == "__main__":
    main()
```

13.4.14 Exercise 13 - Instrument Network to Log Activation Statistics

Part 1 - Why this matters

Instrumentation turns hidden activation behavior into time-series evidence during training. Layer means, standard deviations, extrema, sparsity, saturation, and non-finite counts can reveal degradation well before the final loss becomes unusable.

The logger must remain lightweight and avoid retaining computation graphs. Logging every tensor on every step can distort performance and memory, so the reader should detach summaries, choose a sampling interval, and remove hooks when monitoring ends.

Part 2 - Mathematical form

$$R_{l,t} = (\mu_{l,t}, s_{l,t}, \min H_{l,t}, \max H_{l,t}, p_{0,l,t}, p_{\neg\text{finite},l,t}).$$

Part 3 - Hints and Helper

Attach hooks only to activation modules. Store scalar summaries rather than full tensors, include the training step, and return a function that removes all hook handles.

Part 4 - Python code skeleton

```
import torch

def E13(model, current_step):
    records = []
    handles = []

    def make_hook(name):
        def hook(module, inputs, output):
            values = output.detach()
            records.append({
                "step": current_step(),
                "layer": name,
                "mean": values.float().mean().item(),
                "std": values.float().std().item(),
                "zero_ratio": (...).float().mean().item(),
                "finite": torch.isfinite(values).all().item(),
            })
        return hook

    for name, module in model.named_modules():
```

```
        if model.is_activation(module):
            handles.append(module.register_forward_hook(...))
    return records, handles

def main():
    print(E13(...))

if __name__ == "__main__":
    main()
```

13.4.15 Exercise 14 - Replace Activation Mid-Training and Observe Effects

Part 1 - Why this matters

Changing an activation during training changes the function represented by the current parameters and the derivative landscape used for all future updates. Even without modifying weights, predictions and loss may jump immediately after replacement.

This intervention can reveal how strongly learned parameters depend on the original activation, but it is not a standard tuning shortcut. The experiment must checkpoint the pre-change state and compare a switched run with an unchanged control from the same point.

Part 2 - Mathematical form

$$f_{\theta,\phi_a}(X) \longrightarrow f_{\theta,\phi_b}(X), \qquad \Delta\mathcal{L} = \mathcal{L}(f_{\theta,\phi_b}(X), Y) - \mathcal{L}(f_{\theta,\phi_a}(X), Y).$$

Part 3 - Hints and Helper

Record predictions and loss immediately before and after replacement, then continue both a control and intervention run with identical data order and optimizer settings.

Part 4 - Python code skeleton

```
import copy

def E14(model, replace_activation, continue_training, batch, loss_fn):
    control = copy.deepcopy(model)
    switched = copy.deepcopy(model)
    features, targets = batch
    before = loss_fn(switched(features), targets).item()
    replace_activation(switched, ...)
    after = loss_fn(switched(features), targets).item()
    control_history = continue_training(control, ...)
    switched_history = continue_training(switched, ...)
    return {
        "immediate_loss_change": after - before,
        "control": control_history,
        "switched": switched_history,
```

```
    }

def main():
    print(E14(...))

if __name__ == "__main__":
    main()
```

13.4.16 Exercise 15 - Analyze Activation Interaction with Weight Initialization

Part 1 - Why this matters

Weight initialization controls the scale of pre-activations presented to an activation. An unsuitable scale can push sigmoid or tanh units into saturation, or cause ReLU networks to lose or amplify variance as depth increases.

Initialization and activation must therefore be studied as a pair. Comparing Xavier-style and He-style scaling across activations demonstrates that a method's assumptions about derivative and variance behavior determine where it is appropriate.

Part 2 - Mathematical form

$$\mathrm{Var}(W) \approx \frac{1}{n_{\text{in}}} \text{ for balanced activations,} \qquad \mathrm{Var}(W) \approx \frac{2}{n_{\text{in}}} \text{ for ReLU.}$$

Part 3 - Hints and Helper

Construct activation-initialization pairs under fixed seeds. Before training, record layer-wise pre-activation and output variances; then compare gradient norms after one backward pass.

Part 4 - Python code skeleton

```
import torch

def E15(build_model, combinations, features, targets, loss_fn):
    report = []
    for activation_name, initializer_name in combinations:
        torch.manual_seed(0)
        model = build_model(activation_name)
        model.apply_initialization(initializer_name)
        output, states = model.forward_with_states(features)
        loss = loss_fn(output, targets)
        loss.backward()
        report.append({
            "activation": activation_name,
            "initializer": initializer_name,
            "variances": [state.float().var().item() for state in states],
            "gradient_norms": model.gradient_norms(),
```

```
        })
    return report

def main():
    print(E15(...))

if __name__ == "__main__":
    main()
```

13.4.17 Exercise 16 - Compare Custom Activation vs Built-In PyTorch Activation

Part 1 - Why this matters

A custom activation should match the intended forward values and gradients before it is used in a model. Forward agreement alone is insufficient because an incorrect derivative can silently train a different system.

Built-in PyTorch operations provide optimized kernels and tested autograd behavior. A manual implementation is valuable for understanding and specialized research, but equivalence testing should cover shapes, data types, devices, extreme inputs, and gradient calculations.

Part 2 - Mathematical form

$$\max_x |\phi_{\text{custom}}(x) - \phi_{\text{built-in}}(x)| < \varepsilon_f, \qquad \max_x |\phi'_{\text{custom}}(x) - \phi'_{\text{built-in}}(x)| < \varepsilon_g.$$

Part 3 - Hints and Helper

Use cloned inputs that require gradients, apply identical upstream gradients, and compare both outputs and input gradients with tolerances appropriate to the data type.

Part 4 - Python code skeleton

```
import torch

def E16(custom_function, built_in_function, values, tolerance):
    custom_input = values.clone().requires_grad_(True)
    built_in_input = values.clone().requires_grad_(True)
    custom_output = custom_function(custom_input)
    built_in_output = built_in_function(built_in_input)
    upstream = torch.randn_like(custom_output)
    custom_output.backward(upstream)
    built_in_output.backward(upstream)
    return {
        "forward_match": torch.allclose(
            custom_output, built_in_output, atol=tolerance, rtol=tolerance
        ),
        "gradient_match": torch.allclose(...),
        "maximum_output_error": (...).abs().max().item(),
```

```
        "maximum_gradient_error": (...).abs().max().item(),
    }

def main():
    print(E16(...))

if __name__ == "__main__":
    main()
```

13.4.18 Exercise 17 - Stress Test Activations Under Extreme Inputs

Part 1 - Why this matters

Extreme inputs expose overflow, underflow, saturation, and invalid-value behavior that ordinary samples may never reveal. This is particularly important for formulas containing exponentials or divisions, where a mathematically equivalent naive implementation may be numerically unsafe.

A stress test should include large finite values, signed zeros, infinities, and not-a-number inputs when supported. The objective is to document behavior and gradient safety, not to demand that every special input produce a finite output.

Part 2 - Mathematical form

$$X_{\text{stress}} = \{-\infty, -M, \ldots, -0, +0, \ldots, M, +\infty, \text{NaN}\}.$$

Part 3 - Hints and Helper

Compare custom and built-in functions, classify finite and non-finite results, and run backward only on a safe finite subset when a full reduction would contaminate every gradient.

Part 4 - Python code skeleton

```
import torch

def E17(function, dtype=torch.float32):
    limit = torch.finfo(dtype).max
    values = torch.tensor(
        [-float("inf"), -limit, -1e4, -0.0,
         0.0, 1e4, limit, float("inf"), float("nan")],
        dtype=dtype,
    )
    output = function(...)
    return {
        "input": values,
        "output": output,
        "finite_mask": torch.isfinite(output),
        "nan_mask": torch.isnan(output),
        "positive_infinity": torch.isposinf(output),
```

```
        "negative_infinity": torch.isneginf(output),
    }

def main():
    print(E17(torch.sigmoid))

if __name__ == "__main__":
    main()
```

13.4.19 Exercise 18 - Identify Scenarios Where Activation Choice Dominates Performance

Part 1 - Why this matters

Activation choice is most influential when depth, input scale, initialization, precision, or task geometry repeatedly places units in regions with very different outputs or derivatives. In other settings, data quality or model capacity may dominate and activation changes may have little effect.

Determining dominance requires a factorial experiment rather than one pairwise run. The reader should measure the variation explained by activation across several controlled contexts and compare it with seed variation and interactions with other design factors.

Part 2 - Mathematical form

$$Y = \mu + A_\phi + B_c + (AB)_{\phi,c} + \varepsilon, \qquad \eta_\phi^2 = \frac{SS_\phi}{SS_{\text{total}}}.$$

Part 3 - Hints and Helper

Vary activation, depth, initialization, and input scaling systematically. Repeat seeds, store every configuration, and identify contexts where activation effect exceeds run-to-run noise.

Part 4 - Python code skeleton

```
import itertools

def E18(run_experiment, factors, seeds):
    results = []
    keys = list(factors)
    combinations = itertools.product(*(factors[key] for key in keys))
    for values in combinations:
        configuration = dict(zip(keys, values))
        for seed in seeds:
            metrics = run_experiment(configuration, seed, ...)
            results.append({
                "configuration": configuration.copy(),
                "seed": seed,
                "metrics": metrics,
```

```
            })
    activation_effect = ...
    interaction_effects = ...
    return results, activation_effect, interaction_effects

def main():
    print(E18(...))

if __name__ == "__main__":
    main()
```

13.4.20 Exercise 19 - Write an Activation Function Selection Guide Based on Experiments

Part 1 - Why this matters

An activation selection guide should translate experimental evidence into conditional recommendations. It must explain which properties matter for deep gradient flow, bounded outputs, sparse representations, numerical precision, hardware cost, and compatibility with initialization and loss design.

The guide should not declare one universal winner. It must record the tested conditions, uncertainty, failure cases, and situations where differences were too small to support a recommendation, allowing future readers to revise conclusions when architectures or workloads change.

Part 2 - Mathematical form

$$\phi^*(c) = \arg\min_{\phi \in \Phi} \left(\widehat{R}(\phi, c) + \lambda_s I_{\text{unstable}} + \lambda_t T(\phi, c) + \lambda_m M(\phi, c) \right).$$

Part 3 - Hints and Helper

Require evidence for every recommendation. Include a decision table, experimental scope, stability findings, precision and CUDA results, limitations, and retesting conditions.

Part 4 - Python code skeleton

```
def E19(sections):
    required = {
        "activation_properties", "decision_table",
        "forward_experiments", "gradient_experiments",
        "saturation_and_dead_units", "initialization_interactions",
        "precision_results", "cuda_results",
        "stability_results", "failure_diagnostics",
        "recommendations", "limitations", "retesting_conditions",
    }
    missing = sorted(required - sections.keys())
    unsupported = [
        recommendation
```

```
        for recommendation in sections.get("recommendations", [])
        if not recommendation.get("evidence")
    ]
    return {
        "complete": not missing and not unsupported,
        "missing": missing,
        "unsupported_recommendations": unsupported,
        "guide": sections,
    }

def main():
    print(E19(...))

if __name__ == "__main__":
    main()
```

13.5 MLP03 - Backpropagation Mechanics in MLP

Models and Training

13.5.1 Exercise 00 - Perform a Full Forward Pass and Confirm Autograd Graph Construction

Part 1 - Why this matters

A forward pass does more than produce predictions when trainable tensors require gradients. PyTorch records differentiable operations and connects their outputs to gradient functions, creating the graph that will later carry error information back to every contributing parameter.

Confirming graph construction prevents a model from appearing correct while being disconnected from learning. The reader should distinguish leaf parameters from intermediate tensors and verify that the loss remains connected to the expected chain of operations.

Part 2 - Mathematical form

$$H_1 = \phi(XW_1^\mathsf{T} + \mathbf{b}_1), \quad \widehat{Y} = H_1 W_2^\mathsf{T} + \mathbf{b}_2, \quad \widehat{Y}.\,\text{grad_fn} \neq \varnothing.$$

Part 3 - Hints and Helper

Inspect the gradient requirement and gradient function of the input, parameters, hidden state, prediction, and scalar loss. Do not call backward yet.

Part 4 - Python code skeleton

```
import torch

def E00(model, features, targets, loss_fn):
    prediction, hidden = model.forward_with_hidden(features)
```

```
    loss = loss_fn(prediction, targets)
    report = {
        "prediction_requires_grad": prediction.requires_grad,
        "prediction_grad_fn": type(prediction.grad_fn).__name__,
        "hidden_grad_fn": type(hidden.grad_fn).__name__,
        "loss_grad_fn": type(loss.grad_fn).__name__,
        "parameters": {
            name: (parameter.is_leaf, parameter.requires_grad)
            for name, parameter in model.named_parameters()
        },
    }
    return prediction, loss, report

def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

13.5.2 Exercise 01 - Compute Loss and Trigger Backward Pass Through Multiple Layers

Part 1 - Why this matters

A scalar loss summarizes prediction error and becomes the starting point for reverse-mode automatic differentiation. Calling backward sends derivatives through the output layer, hidden activations, and earlier affine transformations in the reverse order of the forward computation.

The operation computes gradients but does not update parameters. Keeping loss computation, gradient creation, and optimization as separate steps helps the reader reason about which stage failed when a model does not learn.

Part 2 - Mathematical form

$$\mathcal{L} = \ell(f_\Theta(X), Y), \qquad \mathcal{L}.\,\text{backward}() \Longrightarrow \nabla_\theta \mathcal{L} \text{ for every } \theta \in \Theta.$$

Part 3 - Hints and Helper

Clear existing gradients, run one forward pass, confirm that the reduced loss is scalar, and call backward exactly once before inspecting parameters.

Part 4 - Python code skeleton

```
import torch

def E01(model, features, targets, loss_fn):
    model.zero_grad(set_to_none=True)
    predictions = model(...)
    loss = loss_fn(..., ...)
```

```
    if loss.ndim != 0:
        raise ValueError(...)
    loss.backward()
    gradients = {
        name: parameter.grad
        for name, parameter in model.named_parameters()
    }
    return loss.item(), gradients

def main():
    print(E01(...))

if __name__ == "__main__":
    main()
```

13.5.3 Exercise 02 - Inspect Gradients of Each Layer and Verify Their Tensor Shapes

Part 1 - Why this matters

Every parameter gradient must have the same shape as the parameter it differentiates. A mismatch would make an element-wise optimizer update undefined, while a missing gradient indicates that the parameter did not contribute to the loss or was disconnected from the graph.

A layer-wise shape report provides an immediate structural check after backward. It also reinforces that a weight matrix receives one sensitivity value per weight and a bias vector receives one aggregated sensitivity per output feature.

Part 2 - Mathematical form

$$\nabla_{W_l}\mathcal{L} \in \mathbb{R}^{d_l \times d_{l-1}}, \qquad \nabla_{\mathbf{b}_l}\mathcal{L} \in \mathbb{R}^{d_l}.$$

Part 3 - Hints and Helper

Iterate through named parameters after backward. Record parameter shape, gradient shape, missing status, and whether the two shapes match exactly.

Part 4 - Python code skeleton

```
def E02(model):
    report = []
    for name, parameter in model.named_parameters():
        gradient = parameter.grad
        report.append({
            "name": name,
            "parameter_shape": tuple(parameter.shape),
            "gradient_shape": None if gradient is None else tuple(...),
            "missing": gradient is None,
            "shape_match": gradient is not None and gradient.shape == ...,
```

```
        })
    return report

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

13.5.4 Exercise 03 - Validate Gradient Flow by Checking Non-Zero Gradients in Hidden Layers

Part 1 - Why this matters

Hidden layers learn only when the loss has a differentiable path to their parameters and that path carries a meaningful signal. A tensor can possess a gradient object while every element is zero, so checking only for a non-missing gradient is not sufficient.

Non-zero gradients are evidence of flow for a specific batch, but they are not automatically healthy. The reader should record norms and non-zero ratios, then interpret exact zeros in relation to activations, frozen parameters, loss construction, and the data.

Part 2 - Mathematical form

$$g_l = \nabla_{W_l} \mathcal{L}, \qquad q_l = \frac{1}{|g_l|} \sum_i \mathbf{1}(g_{l,i} \neq 0), \qquad \|g_l\|_2 > 0.$$

Part 3 - Hints and Helper

Check only trainable hidden-layer parameters. Use a tolerance for near-zero norms and report each failing layer instead of returning one uninformative Boolean.

Part 4 - Python code skeleton

```
import torch

def E03(model, tolerance=1e-12):
    report = {}
    for name, parameter in model.hidden_named_parameters():
        gradient = parameter.grad
        if gradient is None:
            report[name] = {"flow": False, "reason": "missing"}
            continue
        norm = gradient.norm().item()
        nonzero_ratio = (gradient != 0).float().mean().item()
        report[name] = {
            "flow": norm > tolerance,
            "norm": norm,
            "nonzero_ratio": nonzero_ratio,
```

```
        }
    return report

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

13.5.5 Exercise 04 - Demonstrate Vanishing Gradients in Deep Networks Using Saturating Activations

Part 1 - Why this matters

Saturating activations such as sigmoid have derivatives close to zero for large-magnitude inputs. In a deep network, repeated multiplication by small local derivatives can make early-layer gradients orders of magnitude smaller than gradients near the output.

The experiment should connect gradient decay to activation saturation instead of treating small gradients as an unexplained observation. Recording both derivative statistics and layer gradient norms across several depths reveals how the problem develops.

Part 2 - Mathematical form

$$\frac{\partial \mathcal{L}}{\partial H_l} = \frac{\partial \mathcal{L}}{\partial H_L} \prod_{k=l+1}^{L} W_k \operatorname{diag}(\phi'(Z_{k-1})), \qquad |\phi'| \ll 1.$$

Part 3 - Hints and Helper

Use identical seeds for each tested depth. Compare first-to-last gradient ratios and the saturation ratio of every hidden layer after one backward pass.

Part 4 - Python code skeleton

```
import torch

def E04(build_model, depths, batch, loss_fn):
    features, targets = batch
    results = []
    for depth in depths:
        torch.manual_seed(0)
        model = build_model(depth=depth, activation="sigmoid")
        output, states = model.forward_with_states(features)
        loss = loss_fn(output, targets)
        loss.backward()
        norms = model.weight_gradient_norms()
        results.append({
            "depth": depth,
```

```
            "gradient_norms": norms,
            "first_to_last_ratio": norms[0] / (norms[-1] + 1e-12),
            "saturation": model.saturation_statistics(states),
        })
    return results

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

13.5.6 Exercise 05 - Demonstrate Exploding Gradients and Apply Gradient Clipping Correctly

Part 1 - Why this matters

Exploding gradients produce very large update directions that can destabilize parameters, overflow numerical ranges, and turn the loss into a non-finite value. Deep products, large weights, and recurrent amplification are common mechanisms behind the growth.

Gradient clipping limits an update after backward has computed gradients and before the optimizer changes parameters. It treats the immediate symptom rather than proving the underlying architecture or learning rate is correct, so norms must be recorded before and after clipping.

Part 2 - Mathematical form

$$g = \nabla_\Theta \mathcal{L}, \qquad \widetilde{g} = g \min\left(1, \frac{c}{\|g\|_2}\right), \qquad \|\widetilde{g}\|_2 \le c.$$

Part 3 - Hints and Helper

Use global norm clipping after backward and before the optimizer step. Save the norm returned by the clipping function and calculate the post-clipping norm separately.

Part 4 - Python code skeleton

```
import torch

def E05(model, loss, optimizer, max_norm):
    optimizer.zero_grad(set_to_none=True)
    loss.backward()
    before = torch.nn.utils.clip_grad_norm_(
        model.parameters(), max_norm=...
    )
    after = torch.sqrt(sum(
        parameter.grad.detach().pow(2).sum()
        for parameter in model.parameters()
```

```
        if parameter.grad is not None
    ))
    optimizer.step()
    return before.item(), after.item(), after.item() <= max_norm + 1e-6

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

13.5.7 Exercise 06 - Compare Manual Gradient Computation vs Autograd for a Small Network

Part 1 - Why this matters

Manual differentiation exposes the matrix operations hidden inside automatic differentiation. For a small two-layer network, the reader can propagate the loss derivative through the output weights, activation derivative, hidden weights, and bias reductions explicitly.

Agreement with autograd validates both mathematical reasoning and tensor orientation. The comparison must use the same forward values, reduction convention, and batch treatment because a missing transpose or averaging factor can create a plausible but incorrect result.

Part 2 - Mathematical form

$$\nabla_{W_2}\mathcal{L} = \Delta_2^{\mathsf{T}} H, \qquad \Delta_1 = (\Delta_2 W_2) \odot \phi'(Z_1), \qquad \nabla_{W_1}\mathcal{L} = \Delta_1^{\mathsf{T}} X.$$

Part 3 - Hints and Helper

Choose a simple activation with a known derivative and a scalar mean-squared loss. Retain every forward intermediate needed by the manual backward calculation.

Part 4 - Python code skeleton

```
import torch

def E06(features, targets, w1, b1, w2, b2):
    z1 = features @ w1.T + b1
    hidden = torch.tanh(z1)
    predictions = hidden @ w2.T + b2
    loss = ((predictions - targets) ** 2).mean()
    loss.backward()

    delta2 = 2 * (predictions.detach() - targets) / predictions.numel()
    manual_w2 = delta2.T @ hidden.detach()
    manual_b2 = delta2.sum(dim=0)
    delta1 = (delta2 @ w2.detach()) * (1 - hidden.detach() ** 2)
```

```
    manual_w1 = delta1.T @ features
    manual_b1 = delta1.sum(dim=0)
    return {
        "w1_match": torch.allclose(manual_w1, w1.grad, atol=...),
        "b1_match": torch.allclose(manual_b1, b1.grad, atol=...),
        "w2_match": torch.allclose(manual_w2, w2.grad, atol=...),
        "b2_match": torch.allclose(manual_b2, b2.grad, atol=...),
    }

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

13.5.8 Exercise 07 - Verify Backward Correctness Using Finite Difference Gradient Checking

Part 1 - Why this matters

Finite differences approximate a derivative by measuring how the loss changes after a small positive and negative perturbation. This provides an implementation-independent check of autograd for selected parameters in a small deterministic network.

The perturbation must balance truncation error against floating-point cancellation. Gradient checking is therefore usually performed in double precision with smooth functions, a small model, and no stochastic layers or state-changing operations.

Part 2 - Mathematical form

$$\frac{\partial \mathcal{L}}{\partial \theta_i} \approx \frac{\mathcal{L}(\theta_i + \varepsilon) - \mathcal{L}(\theta_i - \varepsilon)}{2\varepsilon}, \qquad e_i = \frac{|g_i - \widetilde{g}_i|}{|g_i| + |\widetilde{g}_i| + \delta}.$$

Part 3 - Hints and Helper

Use double precision, restore the original parameter after each perturbation, and compare a sampled set of entries rather than modifying all parameters at once.

Part 4 - Python code skeleton

```
import torch

def E07(model, loss_closure, parameter, indices, epsilon=1e-6):
    model.zero_grad(set_to_none=True)
    loss = loss_closure()
    loss.backward()
    analytical = parameter.grad.detach().clone()
    results = []
    with torch.no_grad():
```

```
        for index in indices:
            original = parameter[index].item()
            parameter[index] = original + epsilon
            plus = loss_closure().item()
            parameter[index] = original - epsilon
            minus = loss_closure().item()
            parameter[index] = original
            numerical = (plus - minus) / (2 * epsilon)
            relative_error = abs(analytical[index].item() - numerical) / (...)
            results.append({
                "index": index,
                "numerical": numerical,
                "analytical": analytical[index].item(),
                "relative_error": relative_error,
            })
    return results

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

13.5.9 Exercise 08 - Detect and Fix Common Backprop Bugs Caused by Detached Tensors

Part 1 - Why this matters

Detaching a tensor creates a value that shares no gradient history with the operations that produced it. If this happens inside a training path, later layers may receive gradients while every parameter before the detach point remains unchanged.

The bug is difficult to notice because the forward values and loss can remain valid. Diagnosis requires locating the first intermediate tensor whose gradient function disappears, then removing unintended conversion, detachment, or reconstruction without retaining unnecessary graphs.

Part 2 - Mathematical form

$$H_k^{\text{stop}} = \text{detach}(H_k), \qquad \frac{\partial H_k^{\text{stop}}}{\partial H_k} = 0, \qquad \nabla_{W_l}\mathcal{L} = 0 \text{ for } l \leq k.$$

Part 3 - Hints and Helper

Inspect gradient functions across intermediate states and parameter gradients after backward. Fix the path at its source rather than manually setting gradient requirements on the detached result.

Part 4 - Python code skeleton

```
def E08(model, features, targets, loss_fn):
    model.zero_grad(set_to_none=True)
    output, states = model.forward_with_states(features)
    loss = loss_fn(output, targets)
    loss.backward()
    report = []
    for index, state in enumerate(states):
        report.append({
            "layer": index,
            "requires_grad": state.requires_grad,
            "has_grad_fn": state.grad_fn is not None,
        })
    missing = [
        name for name, parameter in model.named_parameters()
        if parameter.requires_grad and parameter.grad is None
    ]
    first_break = next(
        (item["layer"] for item in report if not item["has_grad_fn"]),
        None,
    )
    return report, missing, first_break

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

13.5.10 Exercise 09 - Debug Gradient Mismatch Caused by Wrong Loss Reduction Mode

Part 1 - Why this matters

Loss reduction determines how per-sample errors become the scalar used by backward. Summing and averaging produce gradients with identical directions for a fixed batch but different magnitudes, usually by a factor related to the number of reduced elements.

A mismatch between a manual derivation and PyTorch may therefore come from reduction rather than calculus. The reader must state whether the loss is summed, averaged, or left unreduced and apply the same convention to every gradient comparison.

Part 2 - Mathematical form

$$\mathcal{L}_{\text{sum}} = \sum_{i=1}^{N} \ell_i, \qquad \mathcal{L}_{\text{mean}} = \frac{1}{N} \sum_{i=1}^{N} \ell_i, \qquad \nabla \mathcal{L}_{\text{sum}} = N \nabla \mathcal{L}_{\text{mean}}.$$

Part 3 - Hints and Helper

Compute gradients from identical predictions with sum and mean reductions. Compare their direction, norm ratio, and the exact number of elements included in the reduction.

Part 4 - Python code skeleton

```
import copy
import torch

def E09(model, features, targets, elementwise_loss):
    gradients = {}
    for reduction in ("sum", "mean"):
        current = copy.deepcopy(model)
        losses = elementwise_loss(current(features), targets)
        loss = losses.sum() if reduction == "sum" else losses.mean()
        loss.backward()
        gradients[reduction] = [
            parameter.grad.detach().clone()
            for parameter in current.parameters()
        ]
    ratios = [
        summed.norm() / (meaned.norm() + 1e-12)
        for summed, meaned in zip(gradients["sum"], gradients["mean"])
    ]
    return gradients, ratios, targets.numel()

def main():
    print(E09(...))

if __name__ == "__main__":
    main()
```

13.5.11 Exercise 10 - Execute Backward Pass on CUDA and Validate Gradient Consistency

Part 1 - Why this matters

CUDA executes the same computational graph on GPU kernels and should produce gradients consistent with a CPU reference within numerical tolerances. Exact bitwise equality is not generally required because operation order and hardware arithmetic may differ.

A valid comparison starts both models from identical state, uses identical data and precision, and transfers the resulting gradients to one device before comparison. It must also verify that every CUDA parameter receives a gradient on the CUDA device.

Part 2 - Mathematical form

$$g_{\text{CPU}} \approx g_{\text{CUDA}}, \qquad \frac{\|g_{\text{CPU}} - g_{\text{CUDA}}\|}{\|g_{\text{CPU}}\| + \varepsilon} < \tau.$$

Part 3 - Hints and Helper

Skip the CUDA branch cleanly when unavailable. Load one state dictionary into both models, disable stochastic layers, and compare named gradients with declared tolerances.

Part 4 - Python code skeleton

```
import copy
import torch

def run_backward(model, features, targets, loss_fn):
    model.zero_grad(set_to_none=True)
    loss = loss_fn(model(features), targets)
    loss.backward()
    return {
        name: parameter.grad.detach().cpu()
        for name, parameter in model.named_parameters()
    }

def E10(model, features, targets, loss_fn):
    cpu_model = copy.deepcopy(model).cpu().eval()
    cpu_gradients = run_backward(
        cpu_model, features.cpu(), targets.cpu(), loss_fn
    )
    if not torch.cuda.is_available():
        return {"available": False, "cpu": cpu_gradients}
    cuda_model = copy.deepcopy(model).cuda().eval()
    cuda_gradients = run_backward(
        cuda_model, features.cuda(), targets.cuda(), loss_fn
    )
    matches = {
        name: torch.allclose(cpu_gradients[name], cuda_gradients[name], ...)
        for name in cpu_gradients
    }
    return {"available": True, "matches": matches}

def main():
    print(E10(...))

if __name__ == "__main__":
    main()
```

13.5.12 Exercise 11 - Investigate CPU vs GPU Gradient Parity Under Different Random Seeds

Part 1 - Why this matters

One successful CPU-GPU comparison does not measure sensitivity to initialization and input sampling. Repeating the experiment across seeds reveals whether small numerical differences remain consistently bounded or become amplified by particular parameter states.

Seed control must cover model initialization and generated data, while nondeterministic operations should be documented. The result should report a distribution of absolute and relative errors instead of only a pass or fail conclusion.

Part 2 - Mathematical form

$$e_s = \max_{\theta} \|g_{\theta,s}^{\mathrm{CPU}} - g_{\theta,s}^{\mathrm{GPU}}\|_{\infty}, \qquad s \in \mathcal{S}.$$

Part 3 - Hints and Helper

Recreate the model and batch after setting each seed, then copy identical state to both devices. Aggregate maximum, mean, and worst-seed errors.

Part 4 - Python code skeleton

```
import torch

def E11(build_case, compare_devices, seeds):
    results = []
    for seed in seeds:
        torch.manual_seed(seed)
        model, features, targets = build_case(...)
        comparison = compare_devices(model, features, targets)
        results.append({"seed": seed, **comparison})
    worst = max(results, key=lambda item: item["max_relative_error"])
    mean_error = sum(
        item["max_relative_error"] for item in results
    ) / len(results)
    return {
        "runs": results,
        "mean_error": mean_error,
        "worst_seed": worst["seed"],
        "worst_error": worst["max_relative_error"],
    }

def main():
    print(E11(...))

if __name__ == "__main__":
    main()
```

13.5.13 Exercise 12 - Compare float32 vs float64 Gradients for Stability and Precision Insights

Part 1 - Why this matters

Double precision represents smaller changes and a wider range of magnitudes than single precision, making it a useful reference for sensitive gradient calculations. Differences become more visible in deep chains, ill-conditioned problems, and finite-difference verification.

Higher precision does not correct a broken graph or an unsuitable optimization problem, and it increases memory and compute cost. The comparison should measure gradient error and non-finite behavior while keeping model values and data logically identical.

Part 2 - Mathematical form

$$e_{32} = \frac{\|g_{32}^{(64)} - g_{64}\|_2}{\|g_{64}\|_2 + \varepsilon}, \qquad g_{32}^{(64)} = \mathrm{cast}_{64}(g_{32}).$$

Part 3 - Hints and Helper

Create both models from the same double-precision state, cast one copy to single precision, and compare gradients after moving both results to double precision on the CPU.

Part 4 - Python code skeleton

```
import copy
import torch

def E12(model, features, targets, loss_fn):
    reports = {}
    for dtype in (torch.float32, torch.float64):
        current = copy.deepcopy(model).to(dtype=dtype)
        current_features = features.to(dtype=dtype)
        current_targets = targets.to(dtype=dtype)
        loss = loss_fn(current(current_features), current_targets)
        loss.backward()
        reports[dtype] = {
            name: parameter.grad.detach().cpu().double()
            for name, parameter in current.named_parameters()
        }
    errors = {}
    for name in reports[torch.float64]:
        reference = reports[torch.float64][name]
        candidate = reports[torch.float32][name]
        errors[name] = (candidate - reference).norm() / (...)
    return reports, errors

def main():
    print(E12(...))

if __name__ == "__main__":
    main()
```

13.5.14 Exercise 13 - Use Autograd Hooks to Log Gradient Norms Layer by Layer During Training

Part 1 - Why this matters

Gradient hooks observe tensors as backward computes their gradients, allowing layer-wise monitoring without rewriting autograd. A time series of norms can reveal the first layer where gradients vanish, explode, or become non-finite during training.

Hooks are persistent callbacks and must be removed when monitoring ends. The logger should store detached scalar summaries rather than full gradient tensors so the diagnostic system does not create unnecessary memory pressure.

Part 2 - Mathematical form

$$R_{l,t} = (\|g_{l,t}\|_2, \|g_{l,t}\|_\infty, p_{\neg\text{finite}}(g_{l,t})) \,.$$

Part 3 - Hints and Helper

Register one hook per trainable parameter and capture the current step through a closure. Keep every removable handle and test cleanup after logging.

Part 4 - Python code skeleton

```
import torch

def E13(model, current_step):
    records = []
    handles = []

    def make_hook(name):
        def hook(gradient):
            records.append({
                "step": current_step(),
                "parameter": name,
                "l2_norm": gradient.norm().item(),
                "max_abs": gradient.abs().max().item(),
                "finite": torch.isfinite(gradient).all().item(),
            })
            return gradient
        return hook

    for name, parameter in model.named_parameters():
        if parameter.requires_grad:
            handles.append(parameter.register_hook(...))
    return records, handles

def main():
    print(E13(...))
```

```
if __name__ == "__main__":
    main()
```

13.5.15 Exercise 14 - Prove the Need for zero_grad by Demonstrating Gradient Accumulation Errors

Part 1 - Why this matters

PyTorch accumulates new derivatives into an existing parameter gradient. This supports deliberate accumulation across micro-batches, but forgetting to clear gradients causes an optimizer step to use the sum of current and previous backward passes.

The effect should be demonstrated with repeated backward calls on the same controlled batch. Comparing uncleared, cleared, and intentionally accumulated cases proves that gradient clearing is part of training-loop semantics rather than optional housekeeping.

Part 2 - Mathematical form

$$g^{(t)} \leftarrow g^{(t-1)} + \nabla_\theta \mathcal{L}_t, \qquad \text{zero_grad} \Longrightarrow g^{(t-1)} = 0 \text{ or } \varnothing.$$

Part 3 - Hints and Helper

Run two identical backward passes without an optimizer update. Verify that the second uncleared gradient is twice the first, then repeat with gradients cleared between passes.

Part 4 - Python code skeleton

```
import torch

def E14(model, loss_closure):
    model.zero_grad(set_to_none=True)
    loss_closure(model).backward()
    first = {
        name: parameter.grad.detach().clone()
        for name, parameter in model.named_parameters()
    }
    loss_closure(model).backward()
    accumulated = {
        name: parameter.grad.detach().clone()
        for name, parameter in model.named_parameters()
    }
    model.zero_grad(set_to_none=True)
    loss_closure(model).backward()
    cleared = {
        name: parameter.grad.detach().clone()
        for name, parameter in model.named_parameters()
    }
```

```
    return {
        name: {
            "doubled": torch.allclose(accumulated[name], 2 * first[name]),
            "cleared_matches": torch.allclose(cleared[name], first[name]),
        }
        for name in first
    }

def main():
    print(E14(...))

if __name__ == "__main__":
    main()
```

13.5.16 Exercise 15 - Analyze the Effect of Batch Size on Gradient Variance and Update Stability

Part 1 - Why this matters

A mini-batch gradient estimates the full-dataset gradient from a sample. Smaller batches usually contain more sampling noise, while larger batches reduce variance but require more memory and may change the optimization behavior for a fixed learning rate.

The analysis must distinguish variance across independently sampled batches from the scale introduced by loss reduction. Comparing gradients with mean reduction and repeated batches allows update direction variability to be measured fairly.

Part 2 - Mathematical form

$$g_B = \frac{1}{B}\sum_{i=1}^{B} \nabla_\theta \ell_i, \qquad \mathrm{Var}(g_B) \approx \frac{\Sigma}{B}.$$

Part 3 - Hints and Helper

For each batch size, sample many batches without changing parameters. Flatten all parameter gradients, then report their mean norm, component variance, and cosine similarity to a large-batch reference.

Part 4 - Python code skeleton

```
import torch

def E15(model, sample_batch, batch_sizes, repeats, loss_fn):
    report = {}
    for batch_size in batch_sizes:
        vectors = []
        for _ in range(repeats):
            features, targets = sample_batch(batch_size)
            model.zero_grad(set_to_none=True)
```

```
            loss_fn(model(features), targets).backward()
            vector = torch.cat([
                parameter.grad.detach().flatten()
                for parameter in model.parameters()
            ])
            vectors.append(vector)
        stacked = torch.stack(vectors)
        report[batch_size] = {
            "mean_norm": stacked.norm(dim=1).mean().item(),
            "component_variance": stacked.var(dim=0).mean().item(),
            "vectors": stacked,
        }
    return report

def main():
    print(E15(...))

if __name__ == "__main__":
    main()
```

13.5.17 Exercise 16 - Demonstrate Graph Retention Costs Using retain_graph and Memory Profiling

Part 1 - Why this matters

Autograd normally frees saved intermediate values after backward because they are no longer needed. Retaining the graph permits another backward traversal through the same forward computation but keeps those intermediates alive and increases memory lifetime.

Graph retention is justified only when the same graph must be differentiated again. Repeatedly retaining graphs in an ordinary training loop can create severe memory growth, so the reader should measure peak allocation and compare it with recomputing a fresh forward pass.

Part 2 - Mathematical form

$$M_{\text{retain}} \approx M_{\Theta} + M_g + M_{\text{saved states}}, \qquad M_{\text{fresh}} \text{ releases saved states after backward.}$$

Part 3 - Hints and Helper

On CUDA, reset peak memory statistics and synchronize measurements. Compare repeated backward with retention against repeated fresh forward-backward passes, then release references.

Part 4 - Python code skeleton

```
import torch

def E16(model, features, targets, loss_fn, repeats):
```

```
    if not features.is_cuda:
        raise ValueError(...)
    torch.cuda.reset_peak_memory_stats(features.device)
    output = model(features)
    loss = loss_fn(output, targets)
    for index in range(repeats):
        model.zero_grad(set_to_none=True)
        loss.backward(retain_graph=index < repeats - 1)
    torch.cuda.synchronize(features.device)
    retained_peak = torch.cuda.max_memory_allocated(features.device)
    fresh_peak = ...
    return {"retained_peak": retained_peak, "fresh_peak": fresh_peak}

def main():
    print(E16(...))

if __name__ == "__main__":
    main()
```

13.5.18 Exercise 17 - Debug Device Mismatch Errors During Backward Computation on CUDA

Part 1 - Why this matters

A device mismatch may appear only during backward when a custom loss, target, mask, hook, or saved tensor resides on a different device from the graph. The forward prediction can therefore succeed while gradient computation fails later.

Debugging requires checking every tensor that participates in loss creation and every registered parameter or buffer. Moving tensors silently inside a backward hook can hide the source and introduce transfers, so placement should be corrected before graph construction.

Part 2 - Mathematical form

$$D(X) = D(Y) = D(\Theta) = D(\mathcal{L}) = \text{CUDA}, \qquad D(\nabla_\theta \mathcal{L}) = D(\theta).$$

Part 3 - Hints and Helper

Build a placement report before backward for inputs, targets, parameters, buffers, predictions, and loss. Identify mismatches relative to the model's first parameter device.

Part 4 - Python code skeleton

```
def E17(model, features, targets, loss_fn):
    expected = next(model.parameters()).device
    prediction = model(features)
    loss = loss_fn(prediction, targets)
    report = {
```

```
        "expected": str(expected),
        "features": str(features.device),
        "targets": str(targets.device),
        "prediction": str(prediction.device),
        "loss": str(loss.device),
        "parameters": {
            name: str(parameter.device)
            for name, parameter in model.named_parameters()
        },
        "buffers": {
            name: str(buffer.device)
            for name, buffer in model.named_buffers()
        },
    }
    mismatches = ...
    if not mismatches:
        loss.backward()
    return report, mismatches

def main():
    print(E17(...))

if __name__ == "__main__":
    main()
```

13.5.19 Exercise 18 - Stress Test Backpropagation with Deep Architecture and Large Batch Inputs

Part 1 - Why this matters

A stress test explores whether backward remains numerically finite and computationally feasible as depth, width, and batch size increase. It can expose memory exhaustion, weak gradient flow, excessive runtime, and precision limits that small examples conceal.

The test must increase load gradually and record the last successful configuration. Resource failure is a result to document, but the experiment should avoid destabilizing the environment by defining maximum dimensions and cleaning references between trials.

Part 2 - Mathematical form

$$C_{\text{backward}} \propto \sum_{l=1}^{L} B d_{l-1} d_l, \qquad M_{\text{activation}} \propto \sum_{l=1}^{L} B d_l.$$

Part 3 - Hints and Helper

Sweep bounded configurations from small to large. Synchronize device timings, monitor peak memory, test gradient finiteness, and catch out-of-memory failures separately from mathematical failures.

Part 4 - Python code skeleton

```
import time
import torch

def E18(build_case, configurations):
    results = []
    for configuration in configurations:
        try:
            model, features, targets, loss_fn = build_case(configuration)
            if features.is_cuda:
                torch.cuda.reset_peak_memory_stats(features.device)
            start = time.perf_counter()
            loss = loss_fn(model(features), targets)
            loss.backward()
            if features.is_cuda:
                torch.cuda.synchronize(features.device)
            elapsed = time.perf_counter() - start
            finite = all(
                parameter.grad is not None
                and torch.isfinite(parameter.grad).all()
                for parameter in model.parameters()
            )
            results.append({
                "configuration": configuration,
                "success": True,
                "finite": finite,
                "elapsed": elapsed,
                "peak_memory": ...,
            })
        except RuntimeError as error:
            results.append({
                "configuration": configuration,
                "success": False,
                "error": str(error),
            })
    return results

def main():
    print(E18(...))

if __name__ == "__main__":
    main()
```

13.5.20 Exercise 19 - Write a Full Backprop Debugging Playbook Based on Observed Failure Modes

Part 1 - Why this matters

A backpropagation playbook converts isolated debugging experiments into a repeatable di-

agnostic sequence. It should begin with graph connectivity and loss construction, then inspect gradient presence, shape, magnitude, finiteness, accumulation, device placement, and numerical parity.

The playbook must map each symptom to evidence, likely causes, controlled tests, and safe corrections. It should also state when clipping or graph retention only masks a deeper problem and preserve experiment records so a future failure can be compared with known patterns.

Part 2 - Mathematical form

$$\text{symptom} \longrightarrow \text{measurement} \longrightarrow \text{hypotheses} \longrightarrow \text{controlled test} \longrightarrow \text{verified correction}.$$

Part 3 - Hints and Helper

Require observed evidence for every failure entry. Include graph, loss, gradients, accumulation, precision, CUDA, memory, stress testing, and escalation criteria.

Part 4 - Python code skeleton

```
def E19(sections):
    required = {
        "graph_connectivity", "loss_and_reduction",
        "missing_gradients", "gradient_shapes",
        "vanishing_gradients", "exploding_gradients",
        "gradient_accumulation", "detached_tensors",
        "finite_difference_check", "precision_parity",
        "cuda_device_checks", "hooks_and_logging",
        "graph_retention_memory", "stress_test_protocol",
        "symptom_decision_tree", "verified_fixes",
    }
    missing = sorted(required - sections.keys())
    unsupported = [
        name for name, entry in sections.get("verified_fixes", {}).items()
        if not entry.get("evidence") or not entry.get("verification")
    ]
    return {
        "complete": not missing and not unsupported,
        "missing": missing,
        "unsupported_fixes": unsupported,
        "playbook": sections,
    }

def main():
    print(E19(...))

if __name__ == "__main__":
    main()
```

13.6 MLP04 - Weight Initialization and Training Stability

Models and Training

13.6.1 Exercise 00 - Implement Zero Initialization and Observe Symmetry Failure in Hidden Units

Part 1 - Why this matters

When every hidden unit begins with identical zero weights and biases, the units produce identical outputs and receive identical gradients. Training cannot assign them different roles, so a wide hidden layer behaves like repeated copies of one unit.

Zero initialization is therefore not merely slow for hidden weights; it creates a symmetry that gradient-based learning cannot break. The experiment should compare unit outputs and weight gradients after one update to show that equality persists.

Part 2 - Mathematical form

$$W_l^{(0)} = 0,\ \mathbf{b}_l^{(0)} = 0 \quad \Longrightarrow \quad h_{l,1} = \cdots = h_{l,d_l},\ \nabla W_{l,1} = \cdots = \nabla W_{l,d_l}.$$

Part 3 - Hints and Helper

Initialize every linear parameter to zero, run one backward step, and compare hidden-unit activations and gradient rows before applying the update.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def E00(model, features, targets, loss_fn):
    for module in model.modules():
        if isinstance(module, nn.Linear):
            nn.init.zeros_(...)
            nn.init.zeros_(...)
    output, hidden = model.forward_with_hidden(features)
    loss_fn(output, targets).backward()
    first = model.first_linear_layer()
    return {
        "identical_hidden_units": torch.allclose(hidden[:, 0], hidden[:, 1]),
        "identical_gradient_rows": torch.allclose(
            first.weight.grad[0], first.weight.grad[1]
        ),
    }
```

```
def main():
    print(E00(...))

if __name__ == "__main__":
    main()
```

13.6.2 Exercise 01 - Compare Random Uniform vs Random Normal Initialization in Early Training

Part 1 - Why this matters

Uniform and normal distributions can share the same variance while assigning different probabilities to extreme values. If their scales are not matched, an early-training comparison confounds distribution shape with parameter magnitude.

A controlled experiment should initialize identical architectures from both distributions with equal target variance and then compare initial loss, activation statistics, gradient norms, and the first training steps across several seeds.

Part 2 - Mathematical form

$$W_U \sim \mathcal{U}(-a, a),\ \mathrm{Var}(W_U) = a^2/3, \qquad W_N \sim \mathcal{N}(0, \sigma^2),\ \sigma^2 = a^2/3.$$

Part 3 - Hints and Helper

Match the theoretical variances and reset the seed policy for every run. Do not judge the methods from one initialization.

Part 4 - Python code skeleton

```
import torch

def E01(build_model, initialize, train_steps, seeds, variance):
    results = []
    for seed in seeds:
        for distribution in ("uniform", "normal"):
            torch.manual_seed(seed)
            model = build_model()
            initialize(model, distribution, variance)
            metrics = train_steps(model, ...)
            results.append({
                "seed": seed,
                "distribution": distribution,
                "metrics": metrics,
            })
    return results

def main():
    print(E01(...))
```

```
if __name__ == "__main__":
    main()
```

13.6.3 Exercise 02 - Implement Xavier Initialization and Validate Activation Variance Preservation

Part 1 - Why this matters

Xavier initialization selects a weight scale from both the incoming and outgoing widths. It is designed to keep forward and backward signal variance reasonably balanced for approximately symmetric activations such as tanh.

Validation requires more than calling an initializer. The reader should measure input, pre-activation, and activation variance layer by layer in a deep untrained network and compare observed behavior with a poorly scaled baseline.

Part 2 - Mathematical form

$$\mathrm{Var}(W) = \frac{2}{n_{\text{in}} + n_{\text{out}}}, \qquad W \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right).$$

Part 3 - Hints and Helper

Implement the uniform formula manually, keep biases at zero, and estimate layer variances from a large random batch.

Part 4 - Python code skeleton

```
import math
import torch
from torch import nn

def E02(model, features):
    with torch.no_grad():
        for module in model.modules():
            if isinstance(module, nn.Linear):
                fan_in, fan_out = module.in_features, module.out_features
                bound = math.sqrt(6.0 / (... + ...))
                module.weight.uniform_(-bound, bound)
                module.bias.zero_()
        output, states = model.forward_with_states(features)
    variances = [state.float().var().item() for state in states]
    return output, variances

def main():
    print(E02(...))

if __name__ == "__main__":
    main()
```

13.6.4 Exercise 03 - Implement He Initialization and Validate ReLU-Compatible Gradient Flow

Part 1 - Why this matters

ReLU removes the negative portion of a roughly symmetric pre-activation distribution. He initialization compensates for this loss by using a variance near two divided by the incoming width, helping signals survive through deep ReLU networks.

The assumption is tied to the activation and network structure. The reader should verify activation variance and weight-gradient norms across depth rather than treating He initialization as universally optimal.

Part 2 - Mathematical form

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right), \qquad \text{Std}(W) = \sqrt{\frac{2}{n_{\text{in}}}}.$$

Part 3 - Hints and Helper

Initialize ReLU layers manually, run a forward-backward pass, and report variance and gradient norms for every layer.

Part 4 - Python code skeleton

```
import math
import torch
from torch import nn

def E03(model, features, targets, loss_fn):
    with torch.no_grad():
        for module in model.modules():
            if isinstance(module, nn.Linear):
                std = math.sqrt(2.0 / module.in_features)
                module.weight.normal_(0.0, ...)
                module.bias.zero_()
    output, states = model.forward_with_states(features)
    loss_fn(output, targets).backward()
    gradients = model.weight_gradient_norms()
    variances = [state.float().var().item() for state in states]
    return variances, gradients

def main():
    print(E03(...))

if __name__ == "__main__":
    main()
```

13.6.5 Exercise 04 - Track Activation Statistics to Confirm Initialization Quality Across Layers

Part 1 - Why this matters

A useful initialization keeps activation scale informative across many layers. Collapsing variance, rapidly increasing magnitude, excessive sparsity, saturation, or non-finite values can reveal instability before the first optimizer step.

Layer-wise statistics turn initialization quality into measurable evidence. The same fixed batch should be used for all methods so differences reflect the initial parameters rather than sampling noise.

Part 2 - Mathematical form

$$R_l = (\mu_l, s_l^2, \min H_l, \max H_l, p_{0,l}, p_{\neg\text{finite},l}).$$

Part 3 - Hints and Helper

Capture every hidden activation without gradients and report scalar summaries. Flag large variance ratios between adjacent layers.

Part 4 - Python code skeleton

```
import torch

def E04(model, features, variance_ratio_limit=10.0):
    with torch.no_grad():
        _, states = model.forward_with_states(features)
    report = []
    previous_variance = features.float().var().item()
    for index, state in enumerate(states):
        values = state.float()
        variance = values.var().item()
        report.append({
            "layer": index,
            "mean": values.mean().item(),
            "variance": variance,
            "zero_ratio": (values == 0).float().mean().item(),
            "finite": torch.isfinite(values).all().item(),
            "unstable_ratio": variance / (previous_variance + 1e-12) > ...,
        })
        previous_variance = variance
    return report

def main():
    print(E04(...))

if __name__ == "__main__":
    main()
```

13.6.6 Exercise 05 - Track Gradient Statistics to Detect Early Instability from Bad Initialization

Part 1 - Why this matters

Poor initialization can produce vanishing, exploding, highly sparse, or non-finite gradients on the first batch. Detecting these patterns before a long training run saves compute and identifies the problem before optimizer dynamics obscure its origin.

Gradient statistics should be interpreted by layer position and parameter type. A global norm alone can hide an early layer with no signal or one layer that dominates the entire update.

Part 2 - Mathematical form

$$G_l = (\|g_l\|_2, \|g_l\|_\infty, \mathbb{E}|g_l|, p_0(g_l), p_{\neg\text{finite}}(g_l)).$$

Part 3 - Hints and Helper

Run one backward pass before any update. Log every weight and bias separately and flag norms outside declared bounds.

Part 4 - Python code skeleton

```
import torch

def E05(model, lower=1e-10, upper=1e3):
    report = []
    for name, parameter in model.named_parameters():
        gradient = parameter.grad
        if gradient is None:
            report.append({"name": name, "status": "missing"})
            continue
        norm = gradient.norm().item()
        report.append({
            "name": name,
            "norm": norm,
            "max_abs": gradient.abs().max().item(),
            "zero_ratio": (gradient == 0).float().mean().item(),
            "finite": torch.isfinite(gradient).all().item(),
            "unstable": norm < lower or norm > upper,
        })
    return report

def main():
    print(E05(...))

if __name__ == "__main__":
    main()
```

13.6.7 Exercise 06 - Demonstrate Training Collapse Caused by Extreme Initialization Scale

Part 1 - Why this matters

Weights initialized at an extreme scale can make activations and gradients explode, saturate bounded functions, or create non-finite loss values. Very small scales can instead collapse representations and weaken learning signals.

A controlled scale sweep should locate the transition from stable to unstable behavior. The objective is to explain collapse through measured signals, not simply to produce a NaN result.

Part 2 - Mathematical form

$$W_l = s\widetilde{W}_l, \qquad s \in \{10^{-k}, \ldots, 1, \ldots, 10^k\}, \qquad \text{stability} = F(s, L, \phi).$$

Part 3 - Hints and Helper

Reuse the same base random tensors and multiply them by each scale. Stop a run when loss, activations, gradients, or parameters become non-finite.

Part 4 - Python code skeleton

```
import copy
import torch

def E06(base_model, scales, train_steps):
    results = []
    for scale in scales:
        model = copy.deepcopy(base_model)
        with torch.no_grad():
            for parameter in model.weight_parameters():
                parameter.mul_(scale)
        history = train_steps(model, stop_on_nonfinite=True, ...)
        results.append({
            "scale": scale,
            "collapsed": not history["all_finite"],
            "first_failure_step": history.get("first_failure_step"),
            "history": history,
        })
    return results

def main():
    print(E06(...))

if __name__ == "__main__":
    main()
```

13.6.8 Exercise 07 - Compare Training Speed Under Different Initializations Using the Same Dataset

Part 1 - Why this matters

Initialization changes the starting region of the loss landscape and the quality of early signal propagation. It can therefore affect how quickly otherwise identical models reach a target validation loss or accuracy.

Fair timing requires the same data order, optimizer, architecture, stopping rule, hardware, and seed schedule. Results should combine wall-clock time with optimization steps because one method may have identical computation per step but need fewer steps.

Part 2 - Mathematical form

$$T_m = \min\{t : \widehat{R}_{\text{val}}(\theta_t^{(m)}) \le \tau\}, \qquad m \in \mathcal{M}.$$

Part 3 - Hints and Helper

Repeat every initializer across seeds and record time and steps to a predetermined target. Report failures to reach the target within the budget.

Part 4 - Python code skeleton

```
import time
import torch

def E07(build_model, initializers, train_to_target, seeds):
    results = []
    for seed in seeds:
        for name, initializer in initializers.items():
            torch.manual_seed(seed)
            model = build_model()
            initializer(model)
            start = time.perf_counter()
            outcome = train_to_target(model, ...)
            elapsed = time.perf_counter() - start
            results.append({
                "seed": seed, "initializer": name,
                "elapsed": elapsed, **outcome,
            })
    return results

def main():
    print(E07(...))

if __name__ == "__main__":
    main()
```

13.6.9 Exercise 08 - Analyze How Initialization Interacts with Activation Choice (ReLU vs Tanh)

Part 1 - Why this matters

ReLU and tanh have different output ranges and derivative behavior, so they do not respond identically to one weight scale. He initialization is designed around rectification, while Xavier often provides a more balanced starting scale for tanh.

The interaction should be tested factorially rather than by comparing one favored pair with one poor pair. Every activation-initializer combination must be evaluated under matched seeds and training conditions.

Part 2 - Mathematical form

$$Y = \mu + A_\phi + B_I + (AB)_{\phi,I} + \varepsilon.$$

Part 3 - Hints and Helper

Test ReLU and tanh with both Xavier and He initialization. Compare saturation, sparsity, activation variance, gradient flow, and training metrics.

Part 4 - Python code skeleton

```
import itertools
import torch

def E08(run_case, activations, initializers, seeds):
    results = []
    for activation, initializer, seed in itertools.product(
        activations, initializers, seeds
    ):
        torch.manual_seed(seed)
        metrics = run_case(activation, initializer, ...)
        results.append({
            "activation": activation,
            "initializer": initializer,
            "seed": seed,
            "metrics": metrics,
        })
    return results

def main():
    print(E08(...))

if __name__ == "__main__":
    main()
```

13.6.10 Exercise 09 - Debug NaN Loss Caused by Unstable Initialization and Fix the Root Cause

Part 1 - Why this matters

A NaN loss is often detected far after the first invalid value appears. Excessive initial weights may create infinite pre-activations, invalid loss operations, or non-finite gradients that contaminate parameters during the first update.

Root-cause debugging locates the earliest non-finite tensor and corrects the initialization scale or formula. Lowering the learning rate or clipping gradients may delay failure without fixing an already invalid forward pass.

Part 2 - Mathematical form

$$k^* = \min\{k : T_k \notin \mathbb{R}^{\text{finite}}\}, \qquad \text{repair} \Rightarrow T_j \in \mathbb{R}^{\text{finite}} \ \forall j.$$

Part 3 - Hints and Helper

Check parameters, pre-activations, activations, loss, gradients, and updated parameters in order. Re-run the same batch after applying a justified initializer.

Part 4 - Python code skeleton

```
import torch

def E09(model, batch, loss_fn, stable_initializer):
    features, targets = batch
    output, states = model.forward_with_states(features)
    loss = loss_fn(output, targets)
    parameters_finite = all(
        torch.isfinite(parameter).all()
        for parameter in model.parameters()
    )
    diagnostics = {
        "parameters": parameters_finite,
        "states": [torch.isfinite(state).all().item() for state in states],
        "loss": torch.isfinite(loss).item(),
    }
    stable_initializer(model)
    fixed_output, fixed_states = model.forward_with_states(features)
    fixed_loss = loss_fn(fixed_output, targets)
    verified = torch.isfinite(fixed_loss).item() and all(
        torch.isfinite(state).all() for state in fixed_states
    )
    return diagnostics, verified

def main():
```

```
    print(E09(...))

if __name__ == "__main__":
    main()
```

13.6.11 Exercise 10 - Run Initialization Experiments on CUDA and Profile Startup Training Stability

Part 1 - Why this matters

CUDA enables larger initialization experiments but introduces asynchronous execution and device-specific memory constraints. Startup stability should be profiled with synchronized timing and all model, data, and generated parameters placed on the same device.

The first iterations may include one-time setup costs, so timing and mathematical stability should be reported separately. Peak memory, finite ratios, initial gradient norms, and early loss curves form a more useful profile than runtime alone.

Part 2 - Mathematical form

$$P_{m,D} = (t_{\text{warm}}, t_{\text{step}}, M_{\text{peak}}, p_{\text{finite}}, \|g_0\|).$$

Part 3 - Hints and Helper

Warm up CUDA before timing, synchronize around measurements, reset peak-memory statistics, and keep the initialized model on CUDA throughout the run.

Part 4 - Python code skeleton

```
import time
import torch

def E10(run_startup, initializer, device):
    if device.type == "cuda":
        torch.cuda.reset_peak_memory_stats(device)
        torch.cuda.synchronize(device)
    start = time.perf_counter()
    result = run_startup(initializer, device, ...)
    if device.type == "cuda":
        torch.cuda.synchronize(device)
        peak = torch.cuda.max_memory_allocated(device)
    else:
        peak = None
    return {"elapsed": time.perf_counter() - start,
            "peak_memory": peak, **result}

def main():
    print(E10(...))

if __name__ == "__main__":
    main()
```

13.6.12 Exercise 11 - Compare float32 vs mixed precision Training Stability for Each Initialization

Part 1 - Why this matters

Mixed precision reduces memory and can accelerate training, but its narrower numerical range may expose instability caused by poor initialization. Loss scaling protects small gradients, yet it cannot make non-finite forward activations valid.

Each initializer should be tested in full and mixed precision under the same seeds. The comparison must record skipped optimizer steps, scale changes, non-finite events, convergence, memory, and speed.

Part 2 - Mathematical form

$$\widetilde{\mathcal{L}} = s\mathcal{L}, \qquad \widetilde{g} = s\nabla\mathcal{L}, \qquad g = \widetilde{g}/s.$$

Part 3 - Hints and Helper

Use automatic mixed precision only on supported devices. Keep a full-precision control and report scaler behavior alongside loss values.

Part 4 - Python code skeleton

```
import torch

def E11(run_training, initializers, seeds):
    report = []
    for name, initializer in initializers.items():
        for seed in seeds:
            for precision in ("float32", "mixed"):
                outcome = run_training(
                    initializer=initializer,
                    seed=seed,
                    precision=precision,
                    ...
                )
                report.append({
                    "initializer": name,
                    "seed": seed,
                    "precision": precision,
                    "outcome": outcome,
                })
    return report

def main():
    print(E11(...))

if __name__ == "__main__":
    main()
```

13.6.13 Exercise 12 - Implement Custom Parameter Initializer for Arbitrary Network Depth

Part 1 - Why this matters

A reusable initializer must discover all applicable modules regardless of network depth. It should compute fan values from each layer's actual shape, initialize weights according to a declared rule, and handle biases separately.

A custom initializer also needs validation and reproducibility. Unsupported modules, invalid gains, and accidental reinitialization of frozen or loaded parameters should be handled explicitly rather than silently.

Part 2 - Mathematical form

$$I(W_l; \phi_l, n_{\text{in}}, n_{\text{out}}, g), \qquad l = 1, \ldots, L.$$

Part 3 - Hints and Helper

Write a function suitable for model application, select the rule from activation metadata, and return a report of every modified module.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def E12(model, rule, bias_value=0.0):
    report = []
    with torch.no_grad():
        for name, module in model.named_modules():
            if not isinstance(module, nn.Linear):
                continue
            if rule == "xavier":
                ...
            elif rule == "he":
                ...
            else:
                raise ValueError(...)
            if module.bias is not None:
                module.bias.fill_(bias_value)
            report.append((name, tuple(module.weight.shape), rule))
    return report

def main():
    print(E12(...))

if __name__ == "__main__":
    main()
```

13.6.14 Exercise 13 - Measure Layer-Wise Signal Propagation Depth Before Saturation Occurs

Part 1 - Why this matters

Signal propagation depth is the number of layers through which an input remains informative before activations saturate, vanish, explode, or become non-finite. Initialization directly influences where this first failure appears.

A declared criterion is required because saturation differs by activation. Tracking variance and derivative magnitude gives a principled stopping point and supports comparisons across depth and initialization scale.

Part 2 - Mathematical form

$$L^* = \min\{l : s_l < \tau_s \ \vee \ s_l > \tau_e \ \vee \ p_{\text{sat},l} > \tau_p\}.$$

Part 3 - Hints and Helper

Use an untrained deep network, capture sequential states, and return both the first failing layer and all preceding statistics.

Part 4 - Python code skeleton

```
import torch

def E13(states, saturation_function, min_var, max_var, max_sat):
    report = []
    failure_depth = None
    for index, state in enumerate(states, start=1):
        variance = state.float().var().item()
        saturation = saturation_function(state)
        failed = (
            variance < min_var or variance > max_var
            or saturation > max_sat
            or not torch.isfinite(state).all()
        )
        report.append((index, variance, saturation, failed))
        if failed and failure_depth is None:
            failure_depth = index
    return failure_depth, report

def main():
    print(E13(...))

if __name__ == "__main__":
    main()
```

13.6.15 Exercise 14 - Diagnose Dying ReLU Caused by Initialization and Apply LeakyReLU Fix

Part 1 - Why this matters

A biased or badly scaled initialization can place many ReLU units entirely in the negative region. Their outputs and local gradients become zero, preventing those units from recovering under ordinary updates.

LeakyReLU restores a negative-side gradient path, but the experiment should also correct the initialization root cause. Comparing ReLU, LeakyReLU, and repaired initialization separates symptom mitigation from a structurally healthier start.

Part 2 - Mathematical form

$$z_j^{(i)} < 0 \ \forall i \Rightarrow \mathrm{ReLU}'(z_j^{(i)}) = 0, \qquad \mathrm{LeakyReLU}'(z) = \alpha > 0 \text{ for } z < 0.$$

Part 3 - Hints and Helper

Measure dead-unit ratios and gradient norms before and after each intervention using the same batch and copied parameters.

Part 4 - Python code skeleton

```
import copy

def E14(model, batch, evaluate, repair_initialization):
    relu_case = copy.deepcopy(model)
    leaky_case = copy.deepcopy(model)
    repaired_case = copy.deepcopy(model)
    leaky_case.replace_relu_with_leaky_relu(negative_slope=...)
    repair_initialization(repaired_case)
    return {
        "relu": evaluate(relu_case, batch),
        "leaky_relu": evaluate(leaky_case, batch),
        "repaired_initialization": evaluate(repaired_case, batch),
    }

def main():
    print(E14(...))

if __name__ == "__main__":
    main()
```

13.6.16 Exercise 15 - Show How Bias Initialization Influences Early Learning and Decision Boundaries

Part 1 - Why this matters

Biases shift pre-activations independently of the input direction. Their initialization can control early activation rates and move decision boundaries before weights have learned useful orientations.

Zero bias is a common neutral choice, but small positive or class-informed biases can be useful in specific contexts. The reader should measure output prevalence, activation sparsity, initial loss, and boundary displacement rather than applying one value blindly.

Part 2 - Mathematical form

$$z = \mathbf{w}^\top \mathbf{x} + b, \qquad \mathcal{B} = \{\mathbf{x} : \mathbf{w}^\top \mathbf{x} + b = 0\}.$$

Part 3 - Hints and Helper

Hold weights fixed and sweep several bias values. Compare activation rates and predictions before any training, then observe the first updates.

Part 4 - Python code skeleton

```
import copy
import torch

def E15(base_model, bias_values, features, targets, loss_fn):
    report = []
    for value in bias_values:
        model = copy.deepcopy(base_model)
        with torch.no_grad():
            for parameter in model.bias_parameters():
                parameter.fill_(value)
        output, states = model.forward_with_states(features)
        report.append({
            "bias": value,
            "loss": loss_fn(output, targets).item(),
            "positive_output_ratio": (output > 0).float().mean().item(),
            "activation_zero_ratios": [
                (state == 0).float().mean().item() for state in states
            ],
        })
    return report

def main():
    print(E15(...))

if __name__ == "__main__":
    main()
```

13.6.17 Exercise 16 - Compare Built-In PyTorch init Methods with Your Manual Implementations

Part 1 - Why this matters

Manual formulas explain initialization mathematics, while PyTorch methods provide tested implementations with explicit gain and fan conventions. Equivalence should be evaluated statistically because separate random draws are not expected to match element by element.

The comparison should validate bounds, means, variances, and reproducibility under controlled generators. A large tensor gives more reliable empirical moments than one small model layer.

Part 2 - Mathematical form

$$|\widehat{\mu} - \mu| < \varepsilon_{\mu}, \qquad |\widehat{\sigma}^2 - \sigma^2| < \varepsilon_{v}, \qquad \max|W| \leq a \text{ when bounded.}$$

Part 3 - Hints and Helper

Generate large manual and built-in samples with independent controlled seeds. Compare both against the theoretical distribution, not directly against each other.

Part 4 - Python code skeleton

```
import torch

def E16(manual_initializer, builtin_initializer, shape, seeds):
    results = []
    for seed in seeds:
        manual = torch.empty(shape)
        builtin = torch.empty(shape)
        torch.manual_seed(seed)
        manual_initializer(manual)
        torch.manual_seed(seed + 1)
        builtin_initializer(builtin)
        results.append({
            "seed": seed,
            "manual_mean": manual.mean().item(),
            "manual_variance": manual.var().item(),
            "builtin_mean": builtin.mean().item(),
            "builtin_variance": builtin.var().item(),
        })
    return results

def main():
    print(E16(...))

if __name__ == "__main__":
    main()
```

13.6.18 Exercise 17 - Stress Test Initialization with Very Deep Networks and Monitor Memory Usage

Part 1 - Why this matters

Very deep networks magnify small variance errors and make initialization assumptions easier to test. They also retain many activations during backward, so mathematical instability and memory exhaustion must be measured separately.

A safe stress test increases depth gradually, uses bounded widths, records the last finite layer and peak memory, and stops before uncontrolled resource use. Failure depth is valuable evidence rather than an invitation to continue until the environment crashes.

Part 2 - Mathematical form

$$M_{\text{activation}} \propto B \sum_{l=1}^{L} d_l, \qquad s_l^2 \approx s_0^2 \prod_{k=1}^{l} r_k.$$

Part 3 - Hints and Helper

Sweep depth under a fixed memory budget. On CUDA, reset peak statistics and synchronize before reading memory values.

Part 4 - Python code skeleton

```
import torch

def E17(build_model, depths, width, batch, memory_limit):
    results = []
    for depth in depths:
        model, features = build_model(depth, width, batch)
        if features.is_cuda:
            torch.cuda.reset_peak_memory_stats(features.device)
        try:
            _, states = model.forward_with_states(features)
            finite_depth = next(
                (index for index, state in enumerate(states)
                 if not torch.isfinite(state).all()),
                len(states),
            )
            peak = ...
            results.append((depth, finite_depth, peak))
            if peak is not None and peak > memory_limit:
                break
        except RuntimeError as error:
            results.append((depth, None, str(error)))
            break
    return results

def main():
    print(E17(...))

if __name__ == "__main__":
    main()
```

13.6.19 Exercise 18 - Create a Reproducible Initialization Benchmark Report with Controlled Seeds

Part 1 - Why this matters

A reproducible benchmark records enough information for another reader to reconstruct every initialization and training run. Seeds alone are insufficient without architecture, data split, software, device, precision, and deterministic settings.

The report should contain raw per-run results and aggregated conclusions. Preserving failures and uncertainty prevents a favorable average from hiding unstable seeds or unsupported claims.

Part 2 - Mathematical form

$$R = \Big(C, \mathcal{S}, \mathcal{M}, D, P, H, \{r_{m,s}\}, \text{aggregate}(r)\Big).$$

Part 3 - Hints and Helper

Store configuration before running experiments, use a fixed seed list, and verify that repeating one case reproduces metrics within declared tolerances.

Part 4 - Python code skeleton

```
import platform
import torch

def E18(configuration, run_case, initializers, seeds):
    runs = []
    for name, initializer in initializers.items():
        for seed in seeds:
            torch.manual_seed(seed)
            result = run_case(initializer, seed, configuration)
            runs.append({
                "initializer": name,
                "seed": seed,
                "result": result,
            })
    return {
        "configuration": configuration,
        "torch_version": torch.__version__,
        "python_version": platform.python_version(),
        "runs": runs,
        "aggregates": ...,
        "reproduction_check": ...,
    }

def main():
```

```
    print(E18(...))

if __name__ == "__main__":
    main()
```

13.6.20 Exercise 19 - Produce a Practical Initialization Selection Guide for Real Training Projects

Part 1 - Why this matters

A practical guide maps architecture, activation, output objective, precision, and depth to an initialization strategy supported by experiments. It should include defaults, diagnostics to run before training, and warning signs that require revision.

The guide must avoid universal claims. It should state tested conditions, explain interactions with bias and mixed precision, distinguish symptom fixes from root-cause repairs, and require new evidence when the project departs from benchmark conditions.

Part 2 - Mathematical form

$$I^*(c) = \arg\min_{I \in \mathcal{I}} \left(\widehat{R}(I, c) + \lambda_s U(I, c) + \lambda_t T(I, c) + \lambda_m M(I, c) \right).$$

Part 3 - Hints and Helper

Require a decision table, validation checklist, failure diagnostics, reproducibility rules, and evidence for every recommendation.

Part 4 - Python code skeleton

```
def E19(sections):
    required = {
        "project_context", "activation_initializer_map",
        "zero_initialization_warning", "xavier_guidance",
        "he_guidance", "bias_guidance",
        "pretraining_signal_checks", "gradient_checks",
        "mixed_precision_guidance", "cuda_profile",
        "failure_diagnostics", "reproducibility_protocol",
        "decision_table", "limitations", "retesting_conditions",
    }
    missing = sorted(required - sections.keys())
    unsupported = [
        item for item in sections.get("decision_table", [])
        if not item.get("evidence")
    ]
    return {
        "complete": not missing and not unsupported,
        "missing": missing,
        "unsupported_recommendations": unsupported,
        "guide": sections,
```

```
    }

def main():
    print(E19(...))

if __name__ == "__main__":
    main()
```

13.7 MLP05 - Optimizers and Learning Rate Dynamics

Models and Training

13.7.1 Exercise 00 - Implement Manual SGD Updates and Verify Parameter Changes Step by Step

Part 1 - Why this matters

Stochastic gradient descent changes each parameter in the direction opposite its current gradient. Implementing the update manually separates gradient computation from parameter mutation and reveals exactly what an optimizer step performs.

The update must occur without creating a new autograd graph, and gradients must be cleared before the next batch. Comparing the measured parameter difference with the predicted change verifies the sign, scale, and ordering of the step.

Part 2 - Mathematical form

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}_t, \qquad \Delta\theta = -\eta g_t.$$

Part 3 - Hints and Helper

Clone parameters before the update, apply changes under a no-gradient context, and compare actual and expected differences.

Part 4 - Python code skeleton

```
import torch

def E00(model, loss, learning_rate):
    model.zero_grad(set_to_none=True)
    loss.backward()
    before = {n: p.detach().clone() for n, p in model.named_parameters()}
    gradients = {
        name: parameter.grad.detach().clone()
        for name, parameter in model.named_parameters()
    }
    with torch.no_grad():
        for parameter in model.parameters():
```

```
            parameter.add_(parameter.grad, alpha=-learning_rate)
    checks = {
        name: torch.allclose(parameter - before[name],
                             -learning_rate * gradients[name])
        for name, parameter in model.named_parameters()
    }
    return checks

def main():
    print(E00(...))

if __name__ == "__main__": main()
```

13.7.2 Exercise 01 - Compare SGD vs Momentum and Observe Convergence Differences

Part 1 - Why this matters

Momentum maintains a velocity that combines current and past gradients. It can accelerate movement along persistent descent directions and reduce oscillation across directions whose gradients repeatedly change sign.

A fair comparison must use identical initial parameters, data order, learning-rate budget, and stopping criteria. Loss curves, velocity norms, and steps to target reveal more than one final metric.

Part 2 - Mathematical form

$$v_t = \mu v_{t-1} + g_t, \qquad \theta_{t+1} = \theta_t - \eta v_t.$$

Part 3 - Hints and Helper

Clone one initial model for both optimizers and repeat across seeds. Record convergence speed and oscillation rather than selecting one favorable run.

Part 4 - Python code skeleton

```
import copy
import torch

def E01(model, train, learning_rate, momentum):
    sgd_model = copy.deepcopy(model)
    momentum_model = copy.deepcopy(model)
    sgd = torch.optim.SGD(sgd_model.parameters(), lr=learning_rate)
    mom = torch.optim.SGD(momentum_model.parameters(),
                          lr=learning_rate, momentum=momentum)
    return {
        "sgd": train(sgd_model, sgd, ...),
        "momentum": train(momentum_model, mom, ...),
```

```
    }

def main(): print(E01(...))
if __name__ == "__main__": main()
```

13.7.3 Exercise 02 - Implement Adam Optimizer and Track Bias Correction Behavior

Part 1 - Why this matters

Adam combines exponential averages of gradients and squared gradients to produce adaptive parameter-wise steps. Because both averages start at zero, their early values are biased toward zero and require time-dependent correction.

Implementing the equations exposes how the first and second moments, epsilon, and correction factors interact. The manual result should be compared with PyTorch using identical gradients and hyperparameters.

Part 2 - Mathematical form

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t,\ v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2,\ \hat{m}_t = \frac{m_t}{1-\beta_1^t},\ \hat{v}_t = \frac{v_t}{1-\beta_2^t}.$$

Part 3 - Hints and Helper

Initialize moments with zeros, increment the step before correction, and log raw and corrected moment norms at every update.

Part 4 - Python code skeleton

```
import torch

def E02(parameter, gradients, learning_rate, beta1, beta2, epsilon):
    first = torch.zeros_like(parameter)
    second = torch.zeros_like(parameter)
    history = []
    with torch.no_grad():
        for step, gradient in enumerate(gradients, start=1):
            first.mul_(beta1).add_(gradient, alpha=1 - beta1)
            second.mul_(beta2).addcmul_(gradient, gradient,
                                        value=1 - beta2)
            corrected_first = first / (...)
            corrected_second = second / (...)
            parameter.addcdiv_(corrected_first,
                               corrected_second.sqrt().add(epsilon),
                               value=-learning_rate)
            history.append((step, first.norm().item(),
                            corrected_first.norm().item()))
```

```
    return parameter, history

def main(): print(E02(...))
if __name__ == "__main__": main()
```

13.7.4 Exercise 03 - Compare Adam vs AdamW and Identify Weight Decay Differences in Practice

Part 1 - Why this matters

Adam with an L2 penalty adds a parameter-dependent term to the gradient before adaptive normalization. AdamW instead decouples weight decay from the loss gradient and applies direct shrinkage to parameters.

These procedures are not generally equivalent under adaptive scaling. A controlled one-step comparison should use the same parameters and gradient and isolate decay from all other optimizer differences.

Part 2 - Mathematical form

$$g_t^{L2} = g_t + \lambda\theta_t, \qquad \theta_{t+1}^{AdamW} = (1-\eta\lambda)\theta_t - \eta\frac{\hat{m}_t}{\sqrt{\hat{v}_t}+\varepsilon}.$$

Part 3 - Hints and Helper

Use copied models and identical gradients. Report parameter norms before and after one step and exclude biases from decay in a separate parameter group.

Part 4 - Python code skeleton

```
import copy
import torch

def E03(model, assign_gradients, learning_rate, decay):
    adam_model = copy.deepcopy(model)
    adamw_model = copy.deepcopy(model)
    assign_gradients(adam_model, ...)
    assign_gradients(adamw_model, ...)
    adam = torch.optim.Adam(adam_model.parameters(), lr=learning_rate,
                            weight_decay=decay)
    adamw = torch.optim.AdamW(adamw_model.parameters(), lr=learning_rate,
                              weight_decay=decay)
    adam.step(); adamw.step()
    return adam_model.state_dict(), adamw_model.state_dict()

def main(): print(E03(...))
if __name__ == "__main__": main()
```

13.7.5 Exercise 04 - Perform Learning Rate Sweep and Identify Divergence Threshold

Part 1 - Why this matters

The learning rate controls update magnitude and often separates useful progress from oscillation or divergence. A sweep across logarithmically spaced values reveals stable, slow, and unstable regions for one fixed training setup.

The divergence threshold depends on optimizer, batch size, initialization, and task. It must be defined using observable criteria such as non-finite loss, rapid growth, or failure to recover within a fixed budget.

Part 2 - Mathematical form

$$\eta \in \{10^a, \ldots, 10^b\}, \qquad \eta_{\text{div}} = \inf\{\eta : \text{unstable}(\eta)\}.$$

Part 3 - Hints and Helper

Restart from identical initial parameters for every rate. Stop unstable runs early and record the first divergence step.

Part 4 - Python code skeleton

```
import copy

def E04(base_model, learning_rates, run_training):
    results = []
    for rate in learning_rates:
        model = copy.deepcopy(base_model)
        outcome = run_training(model, learning_rate=rate,
                               stop_on_divergence=True, ...)
        results.append({"learning_rate": rate, **outcome})
    divergent = [r for r in results if r["diverged"]]
    threshold = min((r["learning_rate"] for r in divergent), default=None)
    return results, threshold

def main(): print(E04(...))
if __name__ == "__main__": main()
```

13.7.6 Exercise 05 - Implement Learning Rate Scheduler and Validate Step Timing and Effects

Part 1 - Why this matters

A scheduler changes the optimizer learning rate according to training progress. Calling it at the wrong time can shift the entire schedule, skip the intended initial rate, or make metric-driven schedules consume the wrong measurement.

Validation requires logging the rate used for each optimizer update, not merely the rate after scheduling. The exercise should distinguish per-step, per-epoch, and metric-based timing.

Part 2 - Mathematical form

$$\eta_t = S(t, \eta_0), \qquad \theta_{t+1} = \theta_t - \eta_t g_t.$$

Part 3 - Hints and Helper

Log the rate immediately before the optimizer step and immediately after the scheduler step. Compare the trace with the analytical schedule.

Part 4 - Python code skeleton

```
def E05(model, optimizer, scheduler, batches, loss_fn):
    trace = []
    for step, (features, targets) in enumerate(batches):
        optimizer.zero_grad(set_to_none=True)
        loss = loss_fn(model(features), targets)
        loss.backward()
        used_rate = optimizer.param_groups[0]["lr"]
        optimizer.step()
        scheduler.step(...)
        next_rate = optimizer.param_groups[0]["lr"]
        trace.append((step, used_rate, next_rate, loss.item()))
    return trace

def main(): print(E05(...))
if __name__ == "__main__": main()
```

13.7.7 Exercise 06 - Diagnose Slow Convergence Caused by Too-Low Learning Rate and Fix It

Part 1 - Why this matters

A learning rate that is too small produces updates that are valid but negligible relative to parameter scale. Loss may decrease monotonically while making so little progress that training appears stuck.

Diagnosis should compare gradient norms, update-to-parameter ratios, and loss slope. Increasing the rate must then be verified through faster progress without introducing instability.

Part 2 - Mathematical form

$$r_t = \frac{\|\Delta\theta_t\|}{\|\theta_t\| + \varepsilon} \approx \eta \frac{\|g_t\|}{\|\theta_t\| + \varepsilon}.$$

Part 3 - Hints and Helper

Measure update ratios before changing the rate, then resume copied runs with several larger values and the same batches.

Part 4 - Python code skeleton

```
def E06(run_training, low_rate, candidate_rates):
    baseline = run_training(low_rate, log_update_ratios=True, ...)
    candidates = {
        rate: run_training(rate, log_update_ratios=True, ...)
        for rate in candidate_rates
    }
    stable = {rate: result for rate, result in candidates.items()
              if not result["diverged"]}
    selected = min(stable, key=lambda rate: stable[rate]["final_loss"])
    return baseline, candidates, selected

def main(): print(E06(...))
if __name__ == "__main__": main()
```

13.7.8 Exercise 07 - Diagnose Divergence Caused by Too-High Learning Rate and Stabilize Training

Part 1 - Why this matters

An excessive learning rate can overshoot useful regions, create oscillating loss, expand parameter norms, and eventually produce non-finite values. Clipping may reduce one symptom while leaving the step size fundamentally inappropriate.

The root-cause test is to restore the same checkpoint and lower only the rate. Stability is confirmed when losses and parameter norms remain finite and progress resumes under the same data order.

Part 2 - Mathematical form

$$\eta > \eta_{\text{stable}} \Rightarrow \|\theta_{t+1} - \theta^*\| > \|\theta_t - \theta^*\| \text{ for repeated steps.}$$

Part 3 - Hints and Helper

Detect the first abnormal step, restore the last finite checkpoint, reduce the rate, and compare both trajectories.

Part 4 - Python code skeleton

```
def E07(checkpoint, run_training, high_rate, reduction_factor):
    unstable = run_training(checkpoint, high_rate,
                            stop_on_divergence=True, ...)
    stable_rate = high_rate * reduction_factor
    stabilized = run_training(checkpoint, stable_rate,
                              stop_on_divergence=True, ...)
```

```
    return {"unstable": unstable, "stable_rate": stable_rate,
            "stabilized": stabilized}

def main(): print(E07(...))
if __name__ == "__main__": main()
```

13.7.9 Exercise 08 - Implement Gradient Clipping and Measure Its Effect on Optimization Stability

Part 1 - Why this matters

Gradient clipping limits unusually large update directions before the optimizer step. Global norm clipping preserves direction while rescaling all gradients together when their combined norm exceeds a threshold.

The threshold should be evaluated experimentally. Excessive clipping can suppress learning, while rare clipping may protect against spikes without materially changing ordinary updates.

Part 2 - Mathematical form

$$\widetilde{g} = g \min(1, c/\|g\|_2), \qquad \|\widetilde{g}\|_2 \leq c.$$

Part 3 - Hints and Helper

Record pre-clipping norm, post-clipping norm, clipping frequency, loss, and update size across the run.

Part 4 - Python code skeleton

```
import torch

def E08(model, optimizer, loss, max_norm):
    optimizer.zero_grad(set_to_none=True)
    loss.backward()
    before = torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm)
    after = torch.sqrt(sum(
        p.grad.pow(2).sum() for p in model.parameters() if p.grad is not None
    ))
    optimizer.step()
    return before.item(), after.item(), before.item() > max_norm

def main(): print(E08(...))
if __name__ == "__main__": main()
```

13.7.10 Exercise 09 - Compare Optimization Under Different Batch Sizes to Observe Noise Benefits

Part 1 - Why this matters

Small batches create noisier gradient estimates, which can help exploration but also increase update variability. Large batches reduce sampling noise and improve throughput on some hardware while requiring more memory.

A fair comparison uses mean loss reduction and considers both steps and processed samples. Learning-rate scaling must be declared because changing batch size without deciding how to change the rate alters two optimization factors.

Part 2 - Mathematical form

$$g_B = \frac{1}{B}\sum_{i=1}^{B} g_i, \qquad \mathrm{Var}(g_B) \approx \Sigma/B.$$

Part 3 - Hints and Helper

Track gradient variance, training loss, validation performance, throughput, and samples to target for every batch size.

Part 4 - Python code skeleton

```
def E09(run_training, batch_sizes, rate_policy, seeds):
    results = []
    for batch_size in batch_sizes:
        rate = rate_policy(batch_size)
        for seed in seeds:
            metrics = run_training(batch_size=batch_size,
                                   learning_rate=rate, seed=seed, ...)
            results.append({"batch_size": batch_size,
                            "learning_rate": rate,
                            "seed": seed, "metrics": metrics})
    return results

def main(): print(E09(...))
if __name__ == "__main__": main()
```

13.7.11 Exercise 10 - Execute Optimizer Steps on CUDA and Validate Parameter Device Placement

Part 1 - Why this matters

CUDA optimization requires model parameters, gradients, and optimizer state to remain on compatible devices. Creating an optimizer before or after moving the model can be valid,

but stale or replaced parameter objects can break the intended connection.

A placement report should verify parameters before backward, gradients after backward, and state tensors after the first optimizer step. The actual parameter change must also occur on CUDA.

Part 2 - Mathematical form

$$D(\theta) = D(g) = D(s_{\text{optimizer}}) = \text{CUDA}.$$

Part 3 - Hints and Helper

Run one full step, inspect every state tensor recursively, and compare parameter snapshots before and after the step.

Part 4 - Python code skeleton

```
import torch

def E10(model, optimizer, loss):
    before = [p.detach().clone() for p in model.parameters()]
    optimizer.zero_grad(set_to_none=True)
    loss.backward(); optimizer.step()
    parameter_devices = {str(p.device) for p in model.parameters()}
    gradient_devices = {str(p.grad.device) for p in model.parameters()
                        if p.grad is not None}
    state_devices = {str(value.device)
                     for state in optimizer.state.values()
                     for value in state.values()
                     if torch.is_tensor(value)}
    changed = [
        not torch.equal(old, new)
        for old, new in zip(before, model.parameters())
    ]
    return parameter_devices, gradient_devices, state_devices, changed

def main(): print(E10(...))
if __name__ == "__main__": main()
```

13.7.12 Exercise 11 - Profile Optimizer Overhead on GPU and Identify Bottlenecks in Update Step

Part 1 - Why this matters

An optimizer reads gradients, updates its state, and changes parameters. For small tensors or many parameter groups, repeated kernel launches and fragmented updates may dominate the step.

CUDA timing must be synchronized, warmed up, and separated from forward and backward time. Comparing optimizers also requires accounting for their different state memory and arithmetic.

Part 2 - Mathematical form

$$T_{\text{train}} = T_f + T_b + T_o + T_z, \qquad \rho_o = T_o / T_{\text{train}}.$$

Part 3 - Hints and Helper

Time the optimizer step in isolation after gradients exist. Report median latency, optimizer-state bytes, parameter count, and group count.

Part 4 - Python code skeleton

```
import time
import torch

def E11(optimizer, device, repeats):
    times = []
    for _ in range(repeats):
        if device.type == "cuda": torch.cuda.synchronize(device)
        start = time.perf_counter()
        optimizer.step()
        if device.type == "cuda": torch.cuda.synchronize(device)
        times.append(time.perf_counter() - start)
    state_bytes = sum(v.numel() * v.element_size()
                      for state in optimizer.state.values()
                      for v in state.values() if torch.is_tensor(v))
    return times, state_bytes

def main(): print(E11(...))
if __name__ == "__main__": main()
```

13.7.13 Exercise 12 - Compare Mixed Precision vs Full Precision Optimization Dynamics on the Same Task

Part 1 - Why this matters

Mixed precision changes numerical range, memory use, and throughput while usually retaining full-precision master parameters in standard training workflows. Dynamic loss scaling protects small gradients but may skip steps when non-finite values appear.

The comparison should begin from the same state and data order. Loss trajectory, gradient norms, skipped steps, scaler values, memory, speed, and validation performance together describe the optimization dynamics.

Part 2 - Mathematical form

$$\widetilde{\mathcal{L}} = s\mathcal{L}, \quad \widetilde{g} = sg, \quad g = \widetilde{g}/s.$$

Part 3 - Hints and Helper

Use matched checkpoints and seeds. Record scale changes and skipped updates rather than reporting only final accuracy.

Part 4 - Python code skeleton

```
def E12(run_training, checkpoint, seeds):
    results = []
    for seed in seeds:
        for precision in ("full", "mixed"):
            outcome = run_training(checkpoint=checkpoint, seed=seed,
                                   precision=precision, ...)
            results.append({"seed": seed, "precision": precision,
                            "outcome": outcome})
    return results

def main(): print(E12(...))
if __name__ == "__main__": main()
```

13.7.14 Exercise 13 - Implement Gradient Accumulation to Simulate Larger Batch Size on Limited GPU Memory

Part 1 - Why this matters

Gradient accumulation processes several micro-batches before one optimizer step, approximating a larger batch without storing all activations simultaneously. Each micro-loss must be scaled so the accumulated gradient matches the desired mean reduction.

The optimizer and scheduler should advance once per effective batch, not once per micro-batch. Incomplete accumulation at the end of an epoch also requires deliberate handling.

Part 2 - Mathematical form

$$g_{\text{eff}} = \frac{1}{K}\sum_{k=1}^{K} g_k, \qquad B_{\text{eff}} = KB_{\text{micro}}.$$

Part 3 - Hints and Helper

Divide each loss by the accumulation count, clear gradients before the cycle, and step only at the cycle boundary or final partial cycle.

Part 4 - Python code skeleton

```
def E13(model, optimizer, loader, loss_fn, accumulation_steps):
    optimizer.zero_grad(set_to_none=True)
    updates = 0
    for index, (features, targets) in enumerate(loader, start=1):
        loss = loss_fn(model(features), targets) / accumulation_steps
        loss.backward()
        boundary = index % accumulation_steps == 0
        final_batch = index == len(loader)
        if boundary or final_batch:
            optimizer.step()
            optimizer.zero_grad(set_to_none=True)
            updates += 1
    return updates

def main(): print(E13(...))
if __name__ == "__main__": main()
```

13.7.15 Exercise 14 - Verify Determinism and Reproducibility Under Fixed Seeds and Controlled Settings

Part 1 - Why this matters

A fixed seed controls pseudorandom sequences but does not automatically guarantee identical training. Data-loader workers, nondeterministic kernels, library versions, and asynchronous execution can still change results.

Reproducibility should be tested by repeating the full run and comparing losses, parameters, and metrics. The report must distinguish exact determinism from statistical reproducibility within a tolerance.

Part 2 - Mathematical form

$$R(C, s)_1 \stackrel{?}{=} R(C, s)_2, \qquad C = \text{complete controlled configuration.}$$

Part 3 - Hints and Helper

Centralize seed setup, enable deterministic algorithms where required, and record environment and configuration metadata.

Part 4 - Python code skeleton

```
import random
import torch

def E14(run_training, seed):
    def set_seed():
        random.seed(seed)
        torch.manual_seed(seed)
```

```
        if torch.cuda.is_available(): torch.cuda.manual_seed_all(seed)
        torch.use_deterministic_algorithms(...)
    set_seed(); first = run_training(...)
    set_seed(); second = run_training(...)
    return {
        "loss_match": first["losses"] == second["losses"],
        "parameter_match": ...,
        "first": first, "second": second,
    }

def main(): print(E14(...))
if __name__ == "__main__": main()
```

13.7.16 Exercise 15 - Detect and Fix Silent Training Bugs Caused by Missing zero_grad or Wrong Order

Part 1 - Why this matters

A training loop can run without exceptions while accumulating old gradients, stepping before backward, clearing gradients too late, or logging a loss detached from the update path. These ordering defects silently change the algorithm.

A correct iteration follows a testable state transition: clear gradients, forward, scalar loss, backward, optional clipping, optimizer step, and scheduler step at its defined frequency.

Part 2 - Mathematical form

$$\text{clear} \to \text{forward} \to \text{loss} \to \text{backward} \to \text{step}\,.$$

Part 3 - Hints and Helper

Instrument one iteration and assert gradient state and parameter changes after each stage. Compare the observed event order with the required sequence.

Part 4 - Python code skeleton

```
def E15(training_step):
    events, snapshots = training_step(instrument=True, ...)
    required = ["zero_grad", "forward", "loss", "backward",
                "optimizer_step"]
    problems = []
    if events[:len(required)] != required:
        problems.append("wrong_order")
    if snapshots["before_backward_has_gradients"]:
        problems.append("uncleared_gradients")
    if not snapshots["parameters_changed_after_step"]:
        problems.append("missing_update")
    return {"correct": not problems, "problems": problems,
```

```
            "events": events}

def main(): print(E15(...))
if __name__ == "__main__": main()
```

13.7.17 Exercise 16 - Implement Weight Decay Correctly and Demonstrate Difference from L2 Regularization

Part 1 - Why this matters

L2 regularization adds a squared-parameter penalty to the objective, while decoupled weight decay directly shrinks parameters during the optimizer step. They coincide for basic SGD under common conventions but differ for adaptive optimizers.

Implementation must also decide which parameters are decayed. Biases and normalization parameters are often excluded through explicit parameter groups rather than name-blind global settings.

Part 2 - Mathematical form

$$\mathcal{L}_{L2} = \mathcal{L} + \frac{\lambda}{2}\|\theta\|_2^2, \qquad \theta^+ = (1-\eta\lambda)\theta - \eta u(g).$$

Part 3 - Hints and Helper

Compare one step of manual L2, decoupled decay, Adam, and AdamW from identical parameters and gradients.

Part 4 - Python code skeleton

```
import torch

def E16(parameter, gradient, learning_rate, decay):
    l2_parameter = parameter.clone()
    decoupled_parameter = parameter.clone()
    with torch.no_grad():
        l2_parameter.add_(gradient + decay * l2_parameter,
                          alpha=-learning_rate)
        decoupled_parameter.mul_(1 - learning_rate * decay)
        decoupled_parameter.add_(gradient, alpha=-learning_rate)
    return (
        l2_parameter,
        decoupled_parameter,
        torch.allclose(l2_parameter, decoupled_parameter),
    )

def main(): print(E16(...))
if __name__ == "__main__": main()
```

13.7.18 Exercise 17 - Benchmark Convergence Speed for Multiple Optimizers and Summarize Results

Part 1 - Why this matters

Optimizer benchmarks should compare convergence in steps, processed samples, and wall-clock time because a method may require fewer updates while costing more per update. Final validation quality and failure rate must also be included.

Reliable conclusions require matched initialization, data order, tuning budgets, and multiple seeds. Each optimizer should receive a reasonable learning-rate search rather than one shared rate that favors a particular method.

Part 2 - Mathematical form

$$B_m = (T_{\tau,m}, S_{\tau,m}, R_{\text{val},m}, p_{\text{fail},m}, M_m).$$

Part 3 - Hints and Helper

Store raw runs, define the target before benchmarking, and summarize median values with variability and failures.

Part 4 - Python code skeleton

```
def E17(optimizer_factories, learning_rates, seeds, run_case):
    results = []
    for name, factory in optimizer_factories.items():
        for rate in learning_rates[name]:
            for seed in seeds:
                result = run_case(factory, rate, seed, ...)
                results.append({"optimizer": name, "rate": rate,
                                "seed": seed, "result": result})
    summary = ...
    return results, summary

def main(): print(E17(...))
if __name__ == "__main__": main()
```

13.7.19 Exercise 18 - Monitor Gradient Norms and Parameter Norms to Detect Impending Instability Early

Part 1 - Why this matters

Gradient and parameter norms often reveal instability before the loss becomes non-finite. Rapid growth, persistent collapse, or extreme update-to-parameter ratios provide actionable warning signals.

Monitoring should operate per layer and globally because one layer can fail while aggregate values appear normal. Thresholds should be based on baseline behavior and sustained changes rather than one harmless spike.

Part 2 - Mathematical form

$$r_t = \|g_t\|_2/(\|\theta_t\|_2 + \varepsilon), \qquad q_t = \|\Delta\theta_t\|_2/(\|\theta_t\|_2 + \varepsilon).$$

Part 3 - Hints and Helper

Log norms before each step, track rolling growth, and emit warnings for non-finite values or repeated threshold violations.

Part 4 - Python code skeleton

```
import torch

def E18(model, history, ratio_limit):
    records = []
    for name, parameter in model.named_parameters():
        gradient = parameter.grad
        if gradient is None: continue
        parameter_norm = parameter.detach().norm().item()
        gradient_norm = gradient.detach().norm().item()
        ratio = gradient_norm / (parameter_norm + 1e-12)
        records.append({"name": name, "parameter_norm": parameter_norm,
                        "gradient_norm": gradient_norm, "ratio": ratio,
                        "warning": ratio > ratio_limit or
                                   not torch.isfinite(gradient).all()})
    history.append(records)
    return records

def main(): print(E18(...))
if __name__ == "__main__": main()
```

13.7.20 Exercise 19 - Write an Optimization Debugging Checklist Covering All Common Training Failures

Part 1 - Why this matters

An optimization checklist converts experiments into a repeatable response to slow learning, divergence, noisy updates, stale gradients, incorrect decay, device errors, precision failures, and scheduler mistakes.

Each item should connect a symptom to a measurement, controlled test, correction, and verification step. The checklist must preserve evidence and avoid recommending clipping or optimizer replacement before the underlying cause is measured.

Part 2 - Mathematical form

$$\text{symptom} \to \text{evidence} \to \text{test} \to \text{fix} \to \text{verification}.$$

Part 3 - Hints and Helper

Include learning rate, gradient order, accumulation, batch size, clipping, decay, scheduler, CUDA, precision, determinism, and monitoring.

Part 4 - Python code skeleton

```
def E19(sections):
    required = {
        "learning_rate", "sgd_and_momentum", "adam_bias_correction",
        "adamw_and_decay", "scheduler_timing", "gradient_clipping",
        "batch_size_noise", "gradient_accumulation", "step_order",
        "cuda_placement", "mixed_precision", "determinism",
        "norm_monitoring", "benchmark_protocol", "decision_tree",
    }
    missing = sorted(required - sections.keys())
    unverified = [name for name, item in sections.items()
                  if isinstance(item, dict) and item.get("fix")
                  and not item.get("verification")]
    return {"complete": not missing and not unverified,
            "missing": missing, "unverified": unverified,
            "checklist": sections}

def main(): print(E19(...))
if __name__ == "__main__": main()
```

13.8 MLP06 - Loss Functions, Training Loop, and Logging

Models and Training

13.8.1 Exercise 00 - Build a Minimal Training Loop with Proper Train/Eval Separation and Correct Modes

Part 1 - Why this matters

A correct training loop changes model parameters only during training and evaluates learned behavior without constructing unnecessary gradient graphs. Training and evaluation modes also control modules whose behavior depends on phase, such as dropout and batch normalization.

Separating both phases prevents validation data from influencing optimization and makes reported metrics meaningful. The loop must restore the intended mode explicitly rather than assuming that one mode change affects only a single batch.

Part 2 - Mathematical form

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}_{\text{train}}, \qquad \mathcal{L}_{\text{val}} = \ell(f_\theta(X_v), Y_v) \text{ without gradient tracking.}$$

Part 3 - Hints and Helper

Use model training mode inside the update loop and evaluation mode with inference context inside validation. Verify that validation leaves parameters unchanged.

Part 4 - Python code skeleton

```
import torch

def E00(model, train_loader, val_loader, optimizer, loss_fn):
    model.train()
    for features, targets in train_loader:
        optimizer.zero_grad(set_to_none=True)
        loss = loss_fn(model(features), targets)
        loss.backward(); optimizer.step()
    before = [p.detach().clone() for p in model.parameters()]
    model.eval()
    validation_losses = []
    with torch.inference_mode():
        for features, targets in val_loader:
            validation_losses.append(loss_fn(model(features), targets).item())
    unchanged = all(
        torch.equal(old, new)
        for old, new in zip(before, model.parameters())
    )
    return sum(validation_losses) / len(validation_losses), unchanged

def main(): print(E00(...))
if __name__ == "__main__": main()
```

13.8.2 Exercise 01 - Implement Classification Training Using CrossEntropyLoss and Validate Label Encoding

Part 1 - Why this matters

Multiclass cross-entropy in PyTorch expects unnormalized class logits and integer class indices. Applying softmax before the loss or passing incorrectly shaped one-hot labels changes or breaks the intended calculation.

Label validation should occur before training so range, data type, and shape errors cannot silently corrupt metrics. The output width must equal the number of classes, and every target must identify a valid class.

Part 2 - Mathematical form

$$\mathcal{L} = -\frac{1}{B}\sum_{i=1}^{B} \log \frac{e^{z_{i,y_i}}}{\sum_{c=1}^{C} e^{z_{i,c}}}, \qquad y_i \in \{0, \ldots, C-1\}.$$

Part 3 - Hints and Helper

Pass raw logits directly to the loss. Assert that labels are one-dimensional integer tensors and remain inside the class range.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def E01(model, features, labels, optimizer, class_count):
    if labels.dtype != torch.long or labels.ndim != 1:
        raise ValueError(...)
    if labels.min() < 0 or labels.max() >= class_count:
        raise ValueError(...)
    logits = model(features)
    if logits.shape != (features.shape[0], class_count):
        raise ValueError(...)
    loss = nn.CrossEntropyLoss()(logits, labels)
    optimizer.zero_grad(set_to_none=True)
    loss.backward(); optimizer.step()
    return loss.item(), logits.argmax(dim=1)

def main(): print(E01(...))
if __name__ == "__main__": main()
```

13.8.3 Exercise 02 - Implement Regression Training Using MSELoss and Validate Output Shapes

Part 1 - Why this matters

Mean-squared error compares every predicted value with a corresponding continuous target. Accidental broadcasting can produce a valid scalar loss even when prediction and target shapes represent different structures.

Shape equality should therefore be verified before loss computation. The exercise should also preserve floating-point targets and distinguish a single-output batch from a one-dimensional tensor whose meaning is ambiguous.

Part 2 - Mathematical form

$$\mathcal{L}_{\mathrm{MSE}} = \frac{1}{N}\sum_{i=1}^{N}(\widehat{y}_i - y_i)^2, \qquad \mathrm{shape}(\widehat{Y}) = \mathrm{shape}(Y).$$

Part 3 - Hints and Helper

Require exact shape equality and floating-point targets. Use an explicit final dimension for single-output regression.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def E02(model, features, targets, optimizer):
    predictions = model(features)
    if predictions.shape != targets.shape:
        raise ValueError(...)
    if not targets.is_floating_point():
        raise TypeError(...)
    loss = nn.MSELoss(reduction="mean")(predictions, targets)
    optimizer.zero_grad(set_to_none=True)
    loss.backward(); optimizer.step()
    return loss.item(), predictions.detach()

def main(): print(E02(...))
if __name__ == "__main__": main()
```

13.8.4 Exercise 03 - Add Accurate Metric Tracking and Ensure Metrics Match Ground Truth Calculations

Part 1 - Why this matters

Epoch metrics must aggregate samples correctly across batches of unequal size. Averaging batch means gives each batch equal weight and produces an error when the final batch is smaller.

Metric implementations should be compared with direct ground-truth calculations on stored predictions and targets. Classification counts and regression sums must use the correct denominator and avoid carrying gradient graphs into logging.

Part 2 - Mathematical form

$$\overline{L} = \frac{\sum_b B_b L_b}{\sum_b B_b}, \qquad \text{Accuracy} = \frac{\sum_b \text{correct}_b}{\sum_b B_b}.$$

Part 3 - Hints and Helper

Accumulate loss multiplied by batch size and total sample count. Verify the result against a direct calculation after concatenating detached outputs.

Part 4 - Python code skeleton

```
import torch

def E03(batch_records):
    loss_sum = 0.0; correct = 0; samples = 0
    predictions = []; targets = []
    for record in batch_records:
        size = record["targets"].shape[0]
```

```
        loss_sum += record["loss"] * size
        correct += (record["predictions"] == record["targets"]).sum().item()
        samples += size
        predictions.append(record["predictions"].detach().cpu())
        targets.append(record["targets"].detach().cpu())
    all_predictions = torch.cat(predictions)
    all_targets = torch.cat(targets)
    direct = (all_predictions == all_targets).float().mean().item()
    return {"loss": loss_sum / samples, "accuracy": correct / samples,
            "matches_direct": abs(correct / samples - direct) < 1e-12}

def main(): print(E03(...))
if __name__ == "__main__": main()
```

13.8.5 Exercise 04 - Implement Validation Loop and Detect Overfitting Using Train vs Val Curves

Part 1 - Why this matters

Training loss measures fit to examples used for updates, while validation loss estimates behavior on held-out data. A widening gap in which training improves and validation degrades is evidence of overfitting.

Both curves must be calculated with comparable reduction and recorded once per epoch. Validation must not shuffle labels into training, update parameters, or include data augmentation intended only for training unless the experiment explicitly requires it.

Part 2 - Mathematical form

$$G_t = \mathcal{L}_{\text{val},t} - \mathcal{L}_{\text{train},t}, \qquad \mathcal{L}_{\text{train}} \downarrow,\ \mathcal{L}_{\text{val}} \uparrow \Rightarrow \text{overfitting signal.}$$

Part 3 - Hints and Helper

Log train and validation curves with exact sample weighting. Detect a sustained validation increase rather than reacting to one noisy epoch.

Part 4 - Python code skeleton

```
def E04(train_epoch, validate_epoch, epochs, patience):
    history = []
    worsening = 0
    best_validation = float("inf")
    for epoch in range(epochs):
        train_metrics = train_epoch(...)
        validation_metrics = validate_epoch(...)
        value = validation_metrics["loss"]
        worsening = worsening + 1 if value > best_validation else 0
        best_validation = min(best_validation, value)
```

```
        history.append({"epoch": epoch, "train": train_metrics,
                        "validation": validation_metrics,
                        "overfitting_signal": worsening >= patience})
    return history

def main(): print(E04(...))
if __name__ == "__main__": main()
```

13.8.6 Exercise 05 - Add Early Stopping Logic and Confirm It Prevents Wasteful Training Iterations

Part 1 - Why this matters

Early stopping ends training when a monitored validation quantity has not improved for a declared patience period. It limits wasted computation and can reduce overfitting, but only if the best model state is preserved.

Minimum improvement, metric direction, patience semantics, and restoration behavior must be explicit. Testing with a synthetic metric sequence makes off-by-one errors visible before using the logic in real training.

Part 2 - Mathematical form

$$t_{\text{stop}} = \min\{t : t - t_{\text{best}} \geq p\}, \qquad m_t < m_{\text{best}} - \delta \text{ for loss improvement.}$$

Part 3 - Hints and Helper

Save the best state when improvement occurs, reset the counter, and restore that state when stopping triggers.

Part 4 - Python code skeleton

```
import copy

def E05(model, validation_losses, patience, min_delta):
    best = float("inf"); best_state = None; stale = 0
    stop_epoch = None
    for epoch, value in enumerate(validation_losses):
        if value < best - min_delta:
            best = value
            best_state = copy.deepcopy(model.state_dict())
            stale = 0
        else:
            stale += 1
        if stale >= patience:
            stop_epoch = epoch; break
    if best_state is not None: model.load_state_dict(best_state)
    return stop_epoch, best, model
```

```
def main(): print(E05(...))
if __name__ == "__main__": main()
```

13.8.7 Exercise 06 - Add Checkpoint Saving and Restore Training from Checkpoint Without Drift

Part 1 - Why this matters

A training checkpoint must preserve more than model weights when exact continuation matters. Optimizer state, epoch, random states, metrics, and relevant configuration all influence the next update.

Restoration should be verified by branching one controlled run: continue directly from a checkpoint and compare it with a save-load continuation using identical next batches. Any divergence identifies missing state.

Part 2 - Mathematical form

$$C_t = (\theta_t, S_t^{\text{opt}}, t, R_t^{\text{CPU}}, R_t^{\text{CUDA}}, H_t, C_{\text{config}}).$$

Part 3 - Hints and Helper

Save to a temporary path first and replace the final checkpoint atomically. Compare the first post-restore loss and parameter update with an uninterrupted control.

Part 4 - Python code skeleton

```
import random
import torch

def E06(path, model, optimizer, epoch, history):
    checkpoint = {
        "model": model.state_dict(), "optimizer": optimizer.state_dict(),
        "epoch": epoch, "history": history,
        "torch_rng": torch.get_rng_state(),
        "python_rng": random.getstate(),
        "cuda_rng": torch.cuda.get_rng_state_all()
                    if torch.cuda.is_available() else None,
    }
    torch.save(checkpoint, path)
    restored = torch.load(path, map_location=next(model.parameters()).device)
    model.load_state_dict(restored["model"])
    optimizer.load_state_dict(restored["optimizer"])
    return restored

def main(): print(E06(...))
if __name__ == "__main__": main()
```

13.8.8 Exercise 07 - Implement Resume Training with Scheduler and Optimizer State Preservation

Part 1 - Why this matters

Resuming only model weights restarts optimizer moments and learning-rate schedule, producing a different training trajectory. The scheduler state and the optimizer state must correspond to the same completed step or epoch.

The resume point should record the next epoch and global update count. A verification run must compare learning rate, optimizer state tensors, and the next parameter update with an uninterrupted control.

Part 2 - Mathematical form

$$C_t = (\theta_t, S_t^{\text{opt}}, S_t^{\text{sched}}, e_t, u_t).$$

Part 3 - Hints and Helper

Restore model, optimizer, and scheduler in a compatible order, then inspect the current learning rate before performing the next update.

Part 4 - Python code skeleton

```
import torch

def E07(checkpoint_path, model, optimizer, scheduler):
    state = torch.load(checkpoint_path,
                       map_location=next(model.parameters()).device)
    model.load_state_dict(state["model"])
    optimizer.load_state_dict(state["optimizer"])
    scheduler.load_state_dict(state["scheduler"])
    return {
        "next_epoch": state["epoch"] + 1,
        "global_step": state["global_step"],
        "learning_rates": [group["lr"] for group in optimizer.param_groups],
    }

def main(): print(E07(...))
if __name__ == "__main__": main()
```

13.8.9 Exercise 08 - Add Structured Logging and Verify Logs Are Correct for Every Epoch

Part 1 - Why this matters

Structured logs make experiments machine-readable and comparable. Every record should identify the epoch, phase, sample count, loss, metrics, learning rate, timing, and relevant diagnostic values with stable field names.

Logging correctness requires schema validation and consistency checks. Missing epochs, duplicate phases, impossible sample counts, or a learning rate recorded at the wrong time can mislead later analysis even when training itself is correct.

Part 2 - Mathematical form

$$R_{e,p} = \{e, p, N, L, M, \eta, T, D\}, \qquad p \in \{\text{train}, \text{validation}\}.$$

Part 3 - Hints and Helper

Append one record per phase and epoch, validate required fields and types, and compare logged metrics with the in-memory epoch result.

Part 4 - Python code skeleton

```
import json

def E08(path, records):
    required = {"epoch", "phase", "samples", "loss",
                "metrics", "learning_rate", "elapsed"}
    errors = []
    with open(path, "w", encoding="utf-8") as stream:
        for record in records:
            missing = required - record.keys()
            if missing: errors.append((record.get("epoch"), sorted(missing)))
            stream.write(json.dumps(record) + "
")
    expected = {(r["epoch"], r["phase"]) for r in records}
    complete = len(expected) == len(records) and not errors
    return complete, errors

def main(): print(E08(...))
if __name__ == "__main__": main()
```

13.8.10 Exercise 09 - Debug Training Loop Bugs That Produce Good Loss but Bad Accuracy

Part 1 - Why this matters

A decreasing loss with poor accuracy can be legitimate early in training, but persistent disagreement may indicate label mapping errors, wrong prediction axes, inappropriate thresholds, leakage, or a metric computed from probabilities incorrectly.

Debugging must recompute accuracy directly from logits and labels on a small known batch. Confusion matrices and sample-level predictions show whether the problem belongs to the model, label encoding, or metric pipeline.

Part 2 - Mathematical form

$$\widehat{y}_i = \arg\max_c z_{i,c}, \qquad \text{Accuracy} = B^{-1} \sum_i \mathbf{1}(\widehat{y}_i = y_i).$$

Part 3 - Hints and Helper

Inspect raw logits, target indices, predicted indices, class mapping, and confusion counts. Compare the logged accuracy with a direct tensor calculation.

Part 4 - Python code skeleton

```
import torch

def E09(logits, labels, logged_accuracy, class_count):
    predictions = logits.argmax(dim=1)
    direct = (predictions == labels).float().mean().item()
    confusion = torch.zeros(class_count, class_count, dtype=torch.long)
    for target, prediction in zip(labels.cpu(), predictions.cpu()):
        confusion[target, prediction] += 1
    problems = []
    if logits.ndim != 2: problems.append("wrong_logit_shape")
    if not 0 <= labels.min() or labels.max() >= class_count:
        problems.append("invalid_labels")
    if abs(direct - logged_accuracy) > 1e-12:
        problems.append("metric_mismatch")
    return direct, confusion, problems

def main(): print(E09(...))
if __name__ == "__main__": main()
```

13.8.11 Exercise 10 - Train Entire Pipeline on CUDA and Ensure Data, Model, and Loss Are on Same Device

Part 1 - Why this matters

A CUDA training pipeline requires every tensor participating in forward and loss computation to share the model device. Moving only the model leaves inputs or targets on the CPU and causes failure at the first mixed-device operation.

Device placement should be centralized rather than scattered across model code. The loop must validate model parameters, buffers, batches, outputs, loss, and gradients while avoiding repeated host-device transfers.

Part 2 - Mathematical form

$$D(X) = D(Y) = D(\theta) = D(\widehat{Y}) = D(\mathcal{L}) = D(\nabla\theta).$$

Part 3 - Hints and Helper

Choose one device, move each batch with appropriate non-blocking settings, and inspect devices after the first complete update.

Part 4 - Python code skeleton

```
import torch

def E10(model, loader, optimizer, loss_fn, device):
    model.to(device).train()
    report = []
    for features, targets in loader:
        features = features.to(device, non_blocking=True)
        targets = targets.to(device, non_blocking=True)
        optimizer.zero_grad(set_to_none=True)
        output = model(features); loss = loss_fn(output, targets)
        loss.backward(); optimizer.step()
        report.append({"features": str(features.device),
                       "targets": str(targets.device),
                       "output": str(output.device),
                       "loss": str(loss.device)})
        break
    return report

def main(): print(E10(...))
if __name__ == "__main__": main()
```

13.8.12 Exercise 11 - Measure GPU Utilization During Training and Identify Under-Utilization Root Causes

Part 1 - Why this matters

Low GPU utilization often indicates that the accelerator is waiting for data loading, CPU transformations, synchronization, or many tiny kernels. A faster GPU cannot help when the pipeline delivers work too slowly.

Diagnosis should separate data time, host-to-device transfer, forward, backward, and optimizer time. Utilization, throughput, and queue behavior must be observed over sustained training rather than one startup batch.

Part 2 - Mathematical form

$$T_{\text{step}} = T_{\text{data}} + T_{\text{transfer}} + T_f + T_b + T_o + T_{\text{sync}}.$$

Part 3 - Hints and Helper

Synchronize only around measured CUDA regions, warm up first, and compare the time fraction of each pipeline stage.

Part 4 - Python code skeleton

```
import time
import torch

def E11(run_profiled_step, steps, device):
    records = []
    for step in range(steps):
        if device.type == "cuda": torch.cuda.synchronize(device)
        start = time.perf_counter()
        timings = run_profiled_step(...)
        if device.type == "cuda": torch.cuda.synchronize(device)
        timings["total"] = time.perf_counter() - start
        records.append(timings)
    averages = {key: sum(r[key] for r in records) / len(records)
                for key in records[0]}
    bottleneck = max(averages, key=averages.get)
    return records, averages, bottleneck

def main(): print(E11(...))
if __name__ == "__main__": main()
```

13.8.13 Exercise 12 - Implement Mixed Precision Training with GradScaler and Detect Silent Overflow Issues

Part 1 - Why this matters

Automatic mixed precision accelerates supported CUDA operations and reduces activation memory. Gradient scaling protects small gradients from underflow, while the scaler skips optimizer updates when non-finite gradients are detected.

A training loop can appear to run while many steps are silently skipped. Monitoring scale changes, parameter updates, loss finiteness, and gradient norms is necessary to distinguish healthy scaling from persistent overflow.

Part 2 - Mathematical form

$$\widetilde{\mathcal{L}} = s\mathcal{L}, \qquad \widetilde{g} = sg, \qquad g = \widetilde{g}/s.$$

Part 3 - Hints and Helper

Use autocast for forward and loss, scale backward, unscale before clipping, then step and update the scaler. Log whether parameters actually changed.

Part 4 - Python code skeleton

```
import torch

def E12(model, batch, optimizer, loss_fn, scaler, device_type):
    features, targets = batch
    optimizer.zero_grad(set_to_none=True)
    before = [p.detach().clone() for p in model.parameters()]
    old_scale = scaler.get_scale()
    with torch.autocast(device_type=device_type, enabled=...):
        loss = loss_fn(model(features), targets)
    scaler.scale(loss).backward()
    scaler.unscale_(optimizer)
    scaler.step(optimizer); scaler.update()
    changed = any(not torch.equal(a, b)
                  for a, b in zip(before, model.parameters()))
    return {"loss_finite": torch.isfinite(loss).item(),
            "old_scale": old_scale, "new_scale": scaler.get_scale(),
            "parameter_update": changed}

def main(): print(E12(...))
if __name__ == "__main__": main()
```

13.8.14 Exercise 13 - Create a Reproducible Experiment Template with Seeds, Configs, and Version Logs

Part 1 - Why this matters

A reproducible experiment records the complete conditions that generated a result. Seeds must be paired with immutable configuration, data identifiers, code version, library versions, device information, and deterministic settings.

The template should create a unique run directory and save configuration before training begins. Repeating a run should either reproduce outputs exactly or document the tolerated variation and its source.

Part 2 - Mathematical form

$$E = (C, s, V_{\text{code}}, V_{\text{libs}}, D, I_{\text{data}}, R).$$

Part 3 - Hints and Helper

Serialize the resolved configuration rather than defaults alone, log environment metadata, and perform a repeated-run verification.

Part 4 - Python code skeleton

```
import json
import platform
import torch
```

```
def E13(config, seed, code_version, data_id, output_path):
    torch.manual_seed(seed)
    metadata = {
        "config": config, "seed": seed,
        "code_version": code_version, "data_id": data_id,
        "torch_version": torch.__version__,
        "python_version": platform.python_version(),
        "cuda_version": torch.version.cuda,
        "device": torch.cuda.get_device_name(0)
                  if torch.cuda.is_available() else "cpu",
    }
    with open(output_path, "w", encoding="utf-8") as stream:
        json.dump(metadata, stream, indent=2, sort_keys=True)
    return metadata

def main(): print(E13(...))
if __name__ == "__main__": main()
```

13.8.15 Exercise 14 - Implement Gradient Accumulation in Training Loop and Validate Equivalent Results

Part 1 - Why this matters

Gradient accumulation combines several micro-batches before one optimizer step, reducing peak activation memory while approximating a larger effective batch. Loss scaling must match the intended mean reduction.

Equivalence should be tested without stochastic layers by comparing accumulated gradients and the resulting parameter update with one full-batch calculation. Unequal final micro-batches require weighting by sample count rather than dividing blindly.

Part 2 - Mathematical form

$$g_{\text{full}} = \frac{1}{N}\sum_{i=1}^{N} g_i, \qquad g_{\text{acc}} = \sum_{k} \frac{B_k}{N} g_k.$$

Part 3 - Hints and Helper

Use copied models, identical sample order, and no dropout. Compare gradients before stepping and parameters after stepping.

Part 4 - Python code skeleton

```
import copy
import torch

def E14(model, micro_batches, optimizer_factory, loss_fn):
```

```
    accumulated = copy.deepcopy(model)
    optimizer = optimizer_factory(accumulated.parameters())
    total = sum(targets.shape[0] for _, targets in micro_batches)
    optimizer.zero_grad(set_to_none=True)
    for features, targets in micro_batches:
        weight = targets.shape[0] / total
        loss = loss_fn(accumulated(features), targets) * weight
        loss.backward()
    gradients = [p.grad.detach().clone() for p in accumulated.parameters()]
    optimizer.step()
    full_batch_result = ...
    return gradients, full_batch_result, ...

def main(): print(E14(...))
if __name__ == "__main__": main()
```

13.8.16 Exercise 15 - Handle NaN Loss During Training by Adding Detection and Immediate Diagnostics

Part 1 - Why this matters

A non-finite loss should stop parameter updates immediately because one invalid step can contaminate optimizer state and every parameter. The visible NaN may be downstream from the first invalid activation or input.

Immediate diagnostics should preserve the failing batch and record input ranges, targets, outputs, activations, gradients, parameter norms, learning rate, precision, and recent losses. The system should fail loudly instead of continuing with corrupted state.

Part 2 - Mathematical form

$$\neg\,\mathrm{finite}(\mathcal{L}_t) \Rightarrow \text{abort step} \wedge \mathrm{capture}(D_t).$$

Part 3 - Hints and Helper

Check the loss before backward and gradients before stepping. Save detached diagnostic tensors without exposing sensitive raw data unnecessarily.

Part 4 - Python code skeleton

```
import torch

def E15(model, features, targets, optimizer, loss_fn):
    optimizer.zero_grad(set_to_none=True)
    output, states = model.forward_with_states(features)
    loss = loss_fn(output, targets)
    diagnostics = {"loss": loss.detach().cpu(),
                   "input_min": features.min().item(),
```

```
                    "input_max": features.max().item(),
                    "output_finite": torch.isfinite(output).all().item(),
                    "state_finite": [torch.isfinite(s).all().item()
                                     for s in states]}
    if not torch.isfinite(loss):
        return {"aborted": True, "diagnostics": diagnostics}
    loss.backward()
    gradients_finite = all(p.grad is None or torch.isfinite(p.grad).all()
                           for p in model.parameters())
    if gradients_finite: optimizer.step()
    return {"aborted": not gradients_finite,
            "diagnostics": diagnostics}

def main(): print(E15(...))
if __name__ == "__main__": main()
```

13.8.17 Exercise 16 - Implement Batch-Level Debug Mode to Inspect Failing Samples During Training

Part 1 - Why this matters

Batch-level debugging identifies which samples produce extreme loss, invalid outputs, or unexpected labels. Aggregate epoch metrics can hide a small group that repeatedly destabilizes training.

Debug mode should be opt-in because retaining per-sample information adds cost and may expose sensitive data. It should store sample identifiers and compact statistics rather than duplicating full datasets.

Part 2 - Mathematical form

$$i^* = \arg\max_i \ell_i, \qquad F = \{i : \neg\,\mathrm{finite}(\ell_i) \ \vee \ \ell_i > \tau\}.$$

Part 3 - Hints and Helper

Use an unreduced loss, rank samples by loss, and capture identifiers, labels, predictions, ranges, and finiteness for only the failing subset.

Part 4 - Python code skeleton

```
import torch

def E16(model, features, targets, sample_ids, elementwise_loss, threshold):
    outputs = model(features)
    losses = elementwise_loss(outputs, targets)
    per_sample = losses.reshape(losses.shape[0], -1).mean(dim=1)
    failing = (~torch.isfinite(per_sample)) | (per_sample > threshold)
```

```
    indices = failing.nonzero(as_tuple=False).flatten()
    report = []
    for index in indices.tolist():
        report.append({"sample_id": sample_ids[index],
                       "loss": per_sample[index].item(),
                       "target": targets[index].detach().cpu(),
                       "output": outputs[index].detach().cpu()})
    return report

def main(): print(E16(...))
if __name__ == "__main__": main()
```

13.8.18 Exercise 17 - Benchmark Training Throughput Under Different DataLoader Settings and Pin Memory

Part 1 - Why this matters

DataLoader configuration determines whether the accelerator receives batches fast enough. Worker count, pinned memory, prefetching, persistent workers, batch size, and transformation cost can each change throughput.

The benchmark must include steady-state training rather than data loading alone and should measure samples per second, data wait time, transfer time, and memory. Pinned memory is useful only when paired with compatible asynchronous transfers.

Part 2 - Mathematical form

$$Q = \frac{N_{\text{samples}}}{T_{\text{steady}}}, \qquad T_{\text{step}} = T_{\text{wait}} + T_{\text{transfer}} + T_{\text{compute}}.$$

Part 3 - Hints and Helper

Warm up workers and CUDA, hold the model computation fixed, and repeat each loader configuration several times.

Part 4 - Python code skeleton

```
import itertools

def E17(build_loader, benchmark, configurations, repeats):
    results = []
    for configuration in configurations:
        runs = []
        for _ in range(repeats):
            loader = build_loader(**configuration)
            use_non_blocking = configuration["pin_memory"]
            result = benchmark(
                loader, non_blocking=use_non_blocking, ...
```

```
            )
            runs.append(result)
        results.append({"configuration": configuration,
                        "runs": runs,
                        "mean_throughput": sum(r["throughput"] for r in runs)
                                           / len(runs)})
    return results

def main(): print(E17(...))
if __name__ == "__main__": main()
```

13.8.19 Exercise 18 - Validate End-to-End Training Correctness by Overfitting a Tiny Dataset on Purpose

Part 1 - Why this matters

A sufficiently expressive model should often fit a tiny clean dataset almost perfectly. Failure to do so is a powerful signal of bugs in labels, shapes, loss, gradient flow, optimizer order, learning rate, or model capacity.

This is a pipeline correctness test, not a generalization strategy. Regularization and augmentation should be disabled, the same examples repeated, and a strict target loss or accuracy declared before the run.

Part 2 - Mathematical form

$$\exists T : \mathcal{L}_{\text{tiny},T} < \varepsilon \ \wedge \ \text{Accuracy}_{\text{tiny},T} \approx 1.$$

Part 3 - Hints and Helper

Use a very small deterministic subset, verify labels manually, disable dropout and weight decay, and stop when the predefined target is reached.

Part 4 - Python code skeleton

```
def E18(model, tiny_loader, optimizer, loss_fn, max_steps, target):
    history = []
    iterator = iter(tiny_loader)
    for step in range(max_steps):
        try: features, labels = next(iterator)
        except StopIteration:
            iterator = iter(tiny_loader); features, labels = next(iterator)
        optimizer.zero_grad(set_to_none=True)
        logits = model(features); loss = loss_fn(logits, labels)
        loss.backward(); optimizer.step()
        accuracy = (logits.argmax(1) == labels).float().mean().item()
        history.append((step, loss.item(), accuracy))
        if loss.item() < target["loss"] and accuracy >= target["accuracy"]:
            return {"passed": True, "history": history}
```

```
    return {"passed": False, "history": history}

def main(): print(E18(...))
if __name__ == "__main__": main()
```

13.8.20 Exercise 19 - Produce a Full Training Loop Debugging and Verification Report

Part 1 - Why this matters

A complete report converts training-loop experiments into an auditable verification record. It should cover data contracts, modes, losses, metrics, updates, validation, checkpoint restoration, device placement, precision, throughput, and failure handling.

Every conclusion must cite an observed test and its result. The report should distinguish passed checks, unresolved risks, failed checks, corrective actions, and re-verification so that future changes can be evaluated against a stable baseline.

Part 2 - Mathematical form

$$\text{requirement} \rightarrow \text{test} \rightarrow \text{evidence} \rightarrow \text{status} \rightarrow \text{correction} \rightarrow \text{retest}.$$

Part 3 - Hints and Helper

Require evidence for all statuses and include the tiny-dataset overfit test as the final end-to-end verification.

Part 4 - Python code skeleton

```
def E19(sections):
    required = {
        "data_contracts", "train_eval_modes", "classification_loss",
        "regression_loss", "metric_validation", "validation_loop",
        "early_stopping", "checkpoint_restore", "resume_state",
        "structured_logs", "cuda_placement", "gpu_utilization",
        "mixed_precision", "reproducibility", "accumulation",
        "nan_diagnostics", "batch_debugging", "loader_benchmark",
        "tiny_dataset_overfit", "corrective_actions",
    }
    missing = sorted(required - sections.keys())
    unsupported = [name for name, result in sections.items()
                   if isinstance(result, dict) and result.get("status")
                   and not result.get("evidence")]
    return {"complete": not missing and not unsupported,
            "missing": missing, "unsupported": unsupported,
            "report": sections}

def main(): print(E19(...))
if __name__ == "__main__": main()
```

13.9 MLP07 - Regularization, Dropout, and Generalization

Models and Training

13.9.1 Exercise 00 - Demonstrate Overfitting in MLP by Training on Small Dataset Without Regularization

Part 1 - Why this matters

A high-capacity MLP can memorize a small training set while failing to learn patterns that transfer to unseen samples. Training performance continues to improve even as validation loss worsens, creating a growing generalization gap.

The experiment establishes an unregularized baseline before any remedy is introduced. Data splits, model capacity, and training duration must remain fixed so later regularization experiments can be compared with the same failure pattern.

Part 2 - Mathematical form

$$G_t = \mathcal{L}_{\text{val},t} - \mathcal{L}_{\text{train},t}, \qquad \mathcal{L}_{\text{train}} \downarrow,\ \mathcal{L}_{\text{val}} \uparrow .$$

Part 3 - Hints and Helper

Use a deliberately small training subset and a larger held-out set. Log both curves and identify the epoch with minimum validation loss.

Part 4 - Python code skeleton

```
def E00(train_epoch, validate_epoch, epochs):
    history = []
    for epoch in range(epochs):
        train = train_epoch(...)
        validation = validate_epoch(...)
        history.append({"epoch": epoch, "train": train,
                        "validation": validation,
                        "gap": validation["loss"] - train["loss"]})
    best_epoch = min(history, key=lambda item: item["validation"]["loss"])
    return history, best_epoch

def main(): print(E00(...))
if __name__ == "__main__": main()
```

13.9.2 Exercise 01 - Apply L2 Regularization and Measure Its Effect on Train vs Validation Gap

Part 1 - Why this matters

L2 regularization adds a penalty proportional to squared parameter magnitude. It discourages large weights and can reduce sensitivity, but excessive strength can prevent the model from fitting even the training data.

Its effect must be measured on both sides of the bias-variance trade-off. The reader should compare weight norms, training loss, validation loss, and the generalization gap across several penalty strengths.

Part 2 - Mathematical form

$$\mathcal{J}(\theta) = \mathcal{L}(\theta) + \frac{\lambda}{2} \sum_{\theta_i \in \Theta_w} \theta_i^2.$$

Part 3 - Hints and Helper

Apply the penalty to weights only unless a different policy is justified. Report the task loss separately from the regularized objective.

Part 4 - Python code skeleton

```
def E01(model, task_loss, strength):
    penalty = sum(parameter.pow(2).sum()
                  for name, parameter in model.named_parameters()
                  if "weight" in name)
    objective = task_loss + 0.5 * strength * penalty
    objective.backward()
    return {"task_loss": task_loss.item(),
            "penalty": penalty.item(),
            "objective": objective.item()}

def main(): print(E01(...))
if __name__ == "__main__": main()
```

13.9.3 Exercise 02 - Apply Weight Decay Correctly and Compare It Against L2 Regularization Implementation

Part 1 - Why this matters

Weight decay directly shrinks parameters during an optimizer update, whereas L2 regularization modifies the objective gradient. They can coincide for ordinary SGD but differ for adaptive optimizers because the added L2 gradient is adaptively scaled.

A controlled comparison should start from identical parameters and gradients. Parameter groups must make the decay policy explicit, particularly when biases should remain unregularized.

Part 2 - Mathematical form

$$g^{L2} = g + \lambda\theta, \qquad \theta^{+}_{WD} = (1 - \eta\lambda)\theta - \eta u(g).$$

Part 3 - Hints and Helper

Compare one update with SGD and one with Adam-style adaptation. Record parameter norms and exact element-wise differences.

Part 4 - Python code skeleton

```
import copy
import torch

def E02(model, loss_closure, learning_rate, decay):
    l2_model = copy.deepcopy(model)
    wd_model = copy.deepcopy(model)
    l2_loss = loss_closure(l2_model) + 0.5 * decay * sum(
        p.pow(2).sum() for p in l2_model.parameters())
    l2_loss.backward()
    l2_optimizer = torch.optim.Adam(l2_model.parameters(), lr=learning_rate)
    l2_optimizer.step()
    wd_loss = loss_closure(wd_model); wd_loss.backward()
    wd_optimizer = torch.optim.AdamW(wd_model.parameters(),
                                     lr=learning_rate, weight_decay=decay)
    wd_optimizer.step()
    return l2_model.state_dict(), wd_model.state_dict()

def main(): print(E02(...))
if __name__ == "__main__": main()
```

13.9.4 Exercise 03 - Implement Dropout Layer and Verify Behavior Differs Between Train and Eval Modes

Part 1 - Why this matters

Dropout randomly masks activations during training and rescales surviving values so their expectation is preserved. During evaluation it becomes the identity transformation, producing deterministic outputs.

Mode-dependent behavior must be verified explicitly. Repeated training passes should differ, repeated evaluation passes should match, and the average training output should approach the evaluation output over many samples.

Part 2 - Mathematical form

$$\widetilde{h} = \frac{m \odot h}{1 - p}, \quad m_i \sim \text{Bernoulli}(1 - p), \qquad \mathbb{E}[\widetilde{h}] = h.$$

Part 3 - Hints and Helper

Use the same input repeatedly in both modes. Measure zero ratio, determinism, and empirical mean preservation.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def E03(values, probability, repeats=1000):
    dropout = nn.Dropout(probability)
    dropout.train()
    training_outputs = torch.stack([dropout(values) for _ in range(repeats)])
    dropout.eval()
    evaluation_a = dropout(values); evaluation_b = dropout(values)
    training_zero_ratio = (
        (training_outputs == 0).float().mean().item()
    )
    evaluation_deterministic = torch.equal(
        evaluation_a, evaluation_b
    )
    return {
        "training_zero_ratio": training_zero_ratio,
        "evaluation_deterministic": evaluation_deterministic,
        "mean_preserved": torch.allclose(
            training_outputs.mean(0), evaluation_a, atol=...
        ),
    }

def main(): print(E03(...))
if __name__ == "__main__": main()
```

13.9.5 Exercise 04 - Tune Dropout Probability and Identify Underfitting vs Overfitting Patterns

Part 1 - Why this matters

Dropout probability controls the strength of stochastic regularization. A low value may leave overfitting unchanged, while a high value can remove too much capacity and cause both training and validation performance to deteriorate.

Tuning should compare full learning curves across probabilities and seeds. The selected value should minimize held-out error, not maximize the gap reduction by making training uniformly poor.

Part 2 - Mathematical form

$$p^* = \arg\min_{p \in \mathcal{P}} \widehat{R}_{\text{val}}(p), \qquad p \in [0, 1).$$

Part 3 - Hints and Helper

Restart from matched initial conditions for every probability. Classify patterns using train loss, validation loss, and their gap together.

Part 4 - Python code skeleton

```
def E04(run_training, probabilities, seeds):
    results = []
    for probability in probabilities:
        for seed in seeds:
            metrics = run_training(dropout_probability=probability,
                                   seed=seed, ...)
            results.append({"probability": probability,
                            "seed": seed, "metrics": metrics})
    summary = ...
    selected = min(summary, key=lambda p: summary[p]["validation_loss"])
    return results, summary, selected

def main(): print(E04(...))
if __name__ == "__main__": main()
```

13.9.6 Exercise 05 - Combine Dropout with Weight Decay and Analyze Interaction Effects

Part 1 - Why this matters

Dropout regularizes activations stochastically, while weight decay constrains parameter magnitude. Combining them may improve generalization, have little added benefit, or over-regularize the same model.

Interaction cannot be inferred from separate one-factor tests. A factorial experiment must compare neither method, each method alone, and both together under identical data and seed conditions.

Part 2 - Mathematical form

$$Y = \mu + A_{\text{dropout}} + B_{\text{decay}} + (AB) + \varepsilon.$$

Part 3 - Hints and Helper

Use a grid of dropout probabilities and decay strengths. Report main effects, interaction, training fit, validation performance, and weight norms.

Part 4 - Python code skeleton

```
import itertools

def E05(run_case, dropout_values, decay_values, seeds):
    results = []
    for dropout, decay, seed in itertools.product(
        dropout_values, decay_values, seeds
    ):
        metrics = run_case(dropout=dropout, decay=decay,
                           seed=seed, ...)
        results.append({"dropout": dropout, "decay": decay,
                        "seed": seed, "metrics": metrics})
    interaction = ...
    return results, interaction

def main(): print(E05(...))
if __name__ == "__main__": main()
```

13.9.7 Exercise 06 - Implement Early Stopping and Compare Against Regularization Techniques

Part 1 - Why this matters

Early stopping limits effective training duration by restoring the model from the best validation epoch. Unlike explicit penalties, it does not alter the per-step objective or architecture.

Comparing early stopping with dropout and weight decay reveals whether controlling training time or constraining the model better addresses the observed overfitting. Compute cost and selected epoch should be reported alongside accuracy.

Part 2 - Mathematical form

$$t^* = \arg\min_t \mathcal{L}_{\text{val},t}, \qquad \theta_{\text{final}} = \theta_{t^*}.$$

Part 3 - Hints and Helper

Use the same maximum epoch budget and validation sequence. Restore the best state, then evaluate all methods on an untouched test set.

Part 4 - Python code skeleton

```
def E06(run_method, methods, seeds):
    results = []
    for method in methods:
        for seed in seeds:
            outcome = run_method(method, seed, restore_best=True, ...)
            results.append({"method": method, "seed": seed,
                            "best_epoch": outcome["best_epoch"],
                            "test_metrics": outcome["test_metrics"],
                            "training_time": outcome["training_time"]})
```

```
    return results

def main(): print(E06(...))
if __name__ == "__main__": main()
```

13.9.8 Exercise 07 - Study the Effect of Model Capacity (Depth/Width) on Generalization Behavior

Part 1 - Why this matters

Depth and width change the number and composition of functions an MLP can represent. Insufficient capacity causes underfitting, while excessive capacity can memorize small datasets unless data and regularization support it.

Capacity comparisons should include parameter count, training fit, validation performance, runtime, and memory. A larger model is not proven worse merely because it can overfit; its observed behavior under controlled training is what matters.

Part 2 - Mathematical form

$$P = \sum_{l=1}^{L} (d_{l-1} + 1)d_l, \qquad G(P) = R_{\text{val}}(P) - R_{\text{train}}(P).$$

Part 3 - Hints and Helper

Sweep controlled architectures and repeat seeds. Compare equal training budgets and separately consider roughly parameter-matched depth-width pairs.

Part 4 - Python code skeleton

```
def E07(build_model, train_and_evaluate, architectures, seeds):
    results = []
    for architecture in architectures:
        for seed in seeds:
            model = build_model(architecture, seed)
            metrics = train_and_evaluate(model, ...)
            parameters = sum(p.numel() for p in model.parameters())
            results.append({"architecture": architecture,
                            "seed": seed, "parameters": parameters,
                            "metrics": metrics})
    return results

def main(): print(E07(...))
if __name__ == "__main__": main()
```

13.9.9 Exercise 08 - Demonstrate Data Augmentation Concept for Non-Image Tabular Tasks

Part 1 - Why this matters

Tabular augmentation creates plausible training examples by perturbing continuous features, masking selected inputs, mixing compatible samples, or resampling within domain constraints. It must preserve the semantic meaning of the target.

Blind noise addition can create impossible records or leak information across splits. The augmentation must be fitted from training data only, respect feature types and bounds, and be validated against domain rules.

Part 2 - Mathematical form

$$\widetilde{x}_j = x_j + \epsilon_j, \quad \epsilon_j \sim \mathcal{N}(0, \sigma_j^2), \qquad \widetilde{y} = y \text{ only if label semantics are preserved.}$$

Part 3 - Hints and Helper

Select perturbable columns explicitly, scale noise by training statistics, and test constraint validity before adding samples to a batch.

Part 4 - Python code skeleton

```
import torch

def E08(features, labels, continuous_columns, scales, validator):
    augmented = features.clone()
    noise = torch.randn_like(augmented[:, continuous_columns])
    augmented[:, continuous_columns] += noise * scales
    valid = validator(augmented)
    if not valid.all():
        augmented = augmented[valid]
        augmented_labels = labels[valid]
    else:
        augmented_labels = labels.clone()
    return augmented, augmented_labels, valid.float().mean().item()

def main(): print(E08(...))
if __name__ == "__main__": main()
```

13.9.10 Exercise 09 - Debug Regularization Misuse That Causes Training Collapse or No Learning

Part 1 - Why this matters

Excessive decay, extreme dropout, a penalty applied twice, regularization of inappropriate parameters, or early stopping configured too aggressively can make a valid model appear incapable of learning.

Diagnosis begins by confirming that the unregularized model can fit a tiny dataset. Regularizers should then be reintroduced one at a time while logging task loss, penalty, active-unit ratio, gradient norms, and update magnitude.

Part 2 - Mathematical form

$$\mathcal{J} = \mathcal{L} + \lambda\Omega(\theta), \qquad \lambda\Omega \gg \mathcal{L} \text{ or } p \to 1 \Rightarrow \text{collapse risk.}$$

Part 3 - Hints and Helper

Disable every regularizer, establish a learning baseline, then add one mechanism per controlled run and identify the first failure.

Part 4 - Python code skeleton

```
def E09(run_case, regularization_config):
    baseline = run_case(dropout=0.0, decay=0.0,
                        early_stopping=False, ...)
    diagnostics = []
    for name in ("dropout", "decay", "early_stopping"):
        isolated = {"dropout": 0.0, "decay": 0.0,
                    "early_stopping": False}
        isolated[name] = regularization_config[name]
        result = run_case(**isolated, ...)
        diagnostics.append({"mechanism": name, "result": result})
    return baseline, diagnostics

def main(): print(E09(...))
if __name__ == "__main__": main()
```

13.9.11 Exercise 10 - Execute Regularized Training on CUDA and Measure Runtime Overhead of Dropout

Part 1 - Why this matters

Dropout adds random-mask generation and element-wise operations during CUDA training. Its relative overhead depends on tensor size, model compute, kernel behavior, and whether timing includes data transfer.

A valid benchmark compares identical networks with dropout disabled and enabled, keeps data on the GPU, warms up kernels, and synchronizes timed regions. Evaluation mode should show that dropout overhead disappears.

Part 2 - Mathematical form

$$O_{\text{drop}} = (T_p - T_0)/T_0, \qquad p > 0.$$

Part 3 - Hints and Helper

Measure forward-backward time in training mode and forward time in evaluation mode across several batch sizes.

Part 4 - Python code skeleton

```
import time
import torch

def E10(run_step, dropout_values, device, repeats):
    results = {}
    for probability in dropout_values:
        for _ in range(10): run_step(probability, ...)
        torch.cuda.synchronize(device)
        start = time.perf_counter()
        for _ in range(repeats): run_step(probability, ...)
        torch.cuda.synchronize(device)
        results[probability] = (time.perf_counter() - start) / repeats
    baseline = results[0.0]
    overhead = {p: (value - baseline) / baseline
                for p, value in results.items()}
    return results, overhead

def main(): print(E10(...))
if __name__ == "__main__": main()
```

13.9.12 Exercise 11 - Compare Regularization Outcomes Under Mixed Precision Training

Part 1 - Why this matters

Mixed precision changes numerical behavior while dropout and weight decay alter optimization and capacity. Their interaction may affect gradient scaling, update skipping, and the effective strength of small penalties.

The comparison should use identical initial states, data order, and regularization settings in full and mixed precision. Validation performance, non-finite events, scaler history, norms, speed, and memory all belong in the result.

Part 2 - Mathematical form

$$R = F(p, \lambda, \text{precision}, \text{seed}).$$

Part 3 - Hints and Helper

Test each regularization configuration in both precision modes and repeat seeds. Track whether mixed-precision steps are skipped.

Part 4 - Python code skeleton

```
def E11(run_training, configurations, precisions, seeds):
    results = []
    for configuration in configurations:
        for precision in precisions:
            for seed in seeds:
```

```
                outcome = run_training(configuration=configuration,
                                       precision=precision,
                                       seed=seed, ...)
                results.append({"configuration": configuration,
                                "precision": precision,
                                "seed": seed, "outcome": outcome})
    return results

def main(): print(E11(...))
if __name__ == "__main__": main()
```

13.9.13 Exercise 12 - Analyze Generalization Under Label Noise and Apply Robust Training Strategies

Part 1 - Why this matters

Incorrect labels encourage a sufficiently flexible MLP to memorize contradictions. Training loss may eventually decrease while clean validation performance degrades, especially during long training.

Robust strategies can include early stopping, reduced capacity, label smoothing, sample reweighting, or verified data cleaning. Evaluation requires known clean validation labels and controlled noise injection only into the training split.

Part 2 - Mathematical form

$$\widetilde{y}_i = \begin{cases} y_i, & \text{with probability } 1-\rho, \\ c \neq y_i, & \text{with probability } \rho. \end{cases}$$

Part 3 - Hints and Helper

Sweep known noise rates and compare strategies across seeds. Preserve the clean labels separately so corrupted examples can be analyzed.

Part 4 - Python code skeleton

```
import torch

def E12(labels, noise_rate, class_count, generator):
    noisy = labels.clone()
    mask = torch.rand(labels.shape, generator=generator) < noise_rate
    offsets = torch.randint(1, class_count, labels.shape,
                            generator=generator)
    noisy[mask] = (labels[mask] + offsets[mask]) % class_count
    return noisy, mask

def run_experiment(noise_rates, strategies, seeds):
```

```
    return [...]

def main(): print(E12(...))
if __name__ == "__main__": main()
```

13.9.14 Exercise 13 - Study the Effect of Input Feature Normalization on Generalization Stability

Part 1 - Why this matters

Features with very different scales create poorly conditioned optimization and make some inputs dominate early gradients. Normalization can improve convergence stability and make regularization strength more comparable across features.

Statistics must be fitted on training data only and applied unchanged to validation and test sets. Recomputing them on held-out data leaks information and produces an unrealistic evaluation pipeline.

Part 2 - Mathematical form

$$\widetilde{x}_j = (x_j - \mu_j^{\text{train}})/(s_j^{\text{train}} + \varepsilon).$$

Part 3 - Hints and Helper

Fit means and standard deviations on the training split, handle constant columns, and compare seed-to-seed variation with and without normalization.

Part 4 - Python code skeleton

```
import torch

def E13(train_features, validation_features, epsilon=1e-8):
    mean = train_features.mean(dim=0)
    std = train_features.std(dim=0)
    safe_std = torch.where(std < epsilon, torch.ones_like(std), std)
    normalized_train = (train_features - mean) / safe_std
    normalized_validation = (validation_features - mean) / safe_std
    return {"train": normalized_train,
            "validation": normalized_validation,
            "mean": mean, "std": safe_std,
            "constant_columns": std < epsilon}

def main(): print(E13(...))
if __name__ == "__main__": main()
```

13.9.15 Exercise 14 - Implement K-Fold Cross Validation and Compare Model Stability Across Folds

Part 1 - Why this matters

K-fold cross validation rotates each partition through validation while training on the remaining partitions. It estimates how sensitive results are to one particular split and uses limited data more completely.

Every fold requires a newly initialized model and preprocessing fitted only on that fold's training portion. The final report should include per-fold metrics, mean, variation, and any fold whose data composition explains unusual behavior.

Part 2 - Mathematical form

$$\widehat{R}_{CV} = K^{-1} \sum_{k=1}^{K} R_k, \qquad s_{CV}^2 = (K-1)^{-1} \sum_{k} (R_k - \widehat{R}_{CV})^2.$$

Part 3 - Hints and Helper

Use disjoint validation indices whose union covers the dataset. Reinitialize model, optimizer, scheduler, and preprocessing for every fold.

Part 4 - Python code skeleton

```
import torch

def E14(folds, build_pipeline, train_and_evaluate):
    results = []
    validation_indices = []
    for fold, (train_indices, val_indices) in enumerate(folds):
        pipeline = build_pipeline(fold)
        metrics = train_and_evaluate(pipeline, train_indices,
                                     val_indices, ...)
        results.append({"fold": fold, "metrics": metrics})
        validation_indices.extend(val_indices.tolist())
    coverage = len(set(validation_indices)) == len(validation_indices)
    scores = torch.tensor([r["metrics"]["score"] for r in results])
    return results, scores.mean().item(), scores.std().item(), coverage

def main(): print(E14(...))
if __name__ == "__main__": main()
```

13.9.16 Exercise 15 - Diagnose Underfitting and Identify Whether It Comes from Regularization or Capacity Limits

Part 1 - Why this matters

Underfitting appears when training performance remains poor, but the cause may be insufficient model capacity, excessive regularization, inadequate optimization, or uninformative features. Validation performance alone cannot distinguish them.

A diagnostic experiment changes one constraint at a time. Removing regularization tests whether penalties are suppressing fit, while increasing capacity tests whether the hypothesis class is too limited.

Part 2 - Mathematical form

$$R_{\text{train}} \text{ high} \Rightarrow H_{\text{capacity}} \vee H_{\text{regularization}} \vee H_{\text{optimization}} \vee H_{\text{data}}.$$

Part 3 - Hints and Helper

First verify that optimization progresses, then run copied cases with regularization removed and capacity increased. Compare tiny-set fitting ability.

Part 4 - Python code skeleton

```
def E15(run_case, baseline_config):
    baseline = run_case(baseline_config, ...)
    no_regularization = dict(baseline_config)
    no_regularization.update({"dropout": 0.0, "decay": 0.0})
    larger = dict(baseline_config)
    larger["hidden_sizes"] = [size * 2
                              for size in baseline_config["hidden_sizes"]]
    tests = {"baseline": baseline,
             "no_regularization": run_case(no_regularization, ...),
             "larger_capacity": run_case(larger, ...),
             "tiny_set": run_case(baseline_config, tiny_set=True, ...)}
    cause = ...
    return tests, cause

def main(): print(E15(...))
if __name__ == "__main__": main()
```

13.9.17 Exercise 16 - Build a Generalization Report with Error Breakdown and Residual Analysis

Part 1 - Why this matters

Aggregate metrics can hide systematic errors affecting particular classes, target ranges, or subgroups. A generalization report decomposes failures so a model with an acceptable average cannot conceal a harmful pattern.

Classification requires class-wise confusion and error rates, while regression requires residual distributions, bias, scale, and relationships with features or predictions. Subgroup analysis must use meaningful groups and adequate sample counts.

Part 2 - Mathematical form

$$r_i = y_i - \widehat{y}_i, \qquad \mathbb{E}[r],\ \mathrm{Var}(r),\ \mathbb{E}[|r| \mid G = g].$$

Part 3 - Hints and Helper

Store detached predictions and targets, calculate global and subgroup metrics, and report uncertainty and low-count warnings.

Part 4 - Python code skeleton

```
import torch

def E16(predictions, targets, group_ids):
    residuals = targets - predictions
    report = {"mean_residual": residuals.mean().item(),
              "residual_std": residuals.std().item(),
              "mae": residuals.abs().mean().item(),
              "groups": {}}
    for group in torch.unique(group_ids):
        mask = group_ids == group
        group_residuals = residuals[mask]
        report["groups"][group.item()] = {
            "count": mask.sum().item(),
            "mae": group_residuals.abs().mean().item(),
            "bias": group_residuals.mean().item()}
    return report

def main(): print(E16(...))
if __name__ == "__main__": main()
```

13.9.18 Exercise 17 - Evaluate Robustness Under Distribution Shift and Document Observed Failure Patterns

Part 1 - Why this matters

Distribution shift occurs when deployment inputs or label relationships differ from training conditions. A model that generalizes within one random split may fail under changed scale, noise, missingness, subgroup prevalence, or time period.

Robustness evaluation should define realistic shifts and compare performance with the in-distribution baseline. Failure patterns must be documented by shift severity and subgroup rather than summarized as one worst-case number.

Part 2 - Mathematical form

$$\Delta R_s = R_{Q_s}(f) - R_P(f), \qquad Q_s \neq P.$$

Part 3 - Hints and Helper

Create controlled shift functions without changing labels incorrectly. Sweep severity and preserve both aggregate and sample-level errors.

Part 4 - Python code skeleton

```
def E17(model, dataset, shift_functions, evaluate, severities):
    baseline = evaluate(model, dataset)
    results = []
    for name, shift in shift_functions.items():
        for severity in severities:
            shifted = shift(dataset, severity)
            metrics = evaluate(model, shifted)
            results.append({"shift": name, "severity": severity,
                            "metrics": metrics,
                            "degradation": metrics["error"]
                                           - baseline["error"]})
    return baseline, results

def main(): print(E17(...))
if __name__ == "__main__": main()
```

13.9.19 Exercise 18 - Implement Model Ensembling of Multiple MLP Runs and Compare Generalization Gains

Part 1 - Why this matters

Models trained from different initializations may make partially independent errors. Averaging regression predictions or class probabilities can reduce variance and improve generalization when members are individually competent and sufficiently diverse.

Ensembling adds training, storage, and inference cost. The experiment should compare each member, the ensemble, prediction diversity, calibration, latency, and the marginal benefit of additional members.

Part 2 - Mathematical form

$$\widehat{y}_{\text{ens}} = M^{-1} \sum_{m=1}^{M} \widehat{y}_m, \qquad \operatorname{Var}(\widehat{y}_{\text{ens}}) \leq \operatorname{Var}(\widehat{y}_m) \text{ when errors are not perfectly correlated.}$$

Part 3 - Hints and Helper

Train members with controlled different seeds and the same data protocol. Average probabilities rather than class labels for classification.

Part 4 - Python code skeleton

```
import torch

def E18(models, features, task):
    for model in models: model.eval()
    with torch.inference_mode():
        outputs = torch.stack([model(features) for model in models])
        if task == "classification":
            member_values = outputs.softmax(dim=-1)
            ensemble = member_values.mean(dim=0)
            predictions = ensemble.argmax(dim=-1)
        else:
            member_values = outputs
            ensemble = outputs.mean(dim=0)
            predictions = ensemble
    diversity = member_values.var(dim=0).mean().item()
    return predictions, ensemble, diversity

def main(): print(E18(...))
if __name__ == "__main__": main()
```

13.9.20 Exercise 19 - Write a Practical Regularization Strategy Guide for Real Training Projects

Part 1 - Why this matters

A practical strategy guide selects regularization from evidence about dataset size, noise, model capacity, validation gaps, shift risk, compute budget, and deployment constraints. It should define a baseline before combining multiple mechanisms.

The guide must distinguish overfitting from underfitting and optimization failure. Each recommendation should state its expected signal, tuning range, controlled test, cost, failure mode, and verification condition.

Part 2 - Mathematical form

$$r^*(c) = \arg\min_{r \in \mathcal{R}} \left(\widehat{R}_{\text{val}}(r, c) + \lambda_c C(r) + \lambda_s S(r, c) \right).$$

Part 3 - Hints and Helper

Require a decision table, experiment order, misuse checks, shift tests, and evidence for every final recommendation.

Part 4 - Python code skeleton

```
def E19(sections):
    required = {
        "baseline_overfitting_test", "l2_and_weight_decay",
        "dropout", "early_stopping", "capacity_analysis",
        "tabular_augmentation", "misuse_diagnostics",
```

```
        "mixed_precision", "label_noise", "normalization",
        "cross_validation", "underfitting_diagnosis",
        "error_breakdown", "distribution_shift", "ensembling",
        "decision_table", "limitations", "retesting_conditions",
    }
    missing = sorted(required - sections.keys())
    unsupported = [item for item in sections.get("decision_table", [])
                   if not item.get("evidence")]
    return {"complete": not missing and not unsupported,
            "missing": missing, "unsupported": unsupported,
            "guide": sections}

def main(): print(E19(...))
if __name__ == "__main__": main()
```

13.10 MLP08 - Scaling, Profiling, and Production Readiness

Models and Training

13.10.1 Exercise 00 - Profile MLP Training Performance and Identify Whether Compute or Data Is the Bottleneck

Part 1 - Why this matters

Training throughput is limited by the slowest part of the pipeline. The accelerator may be busy computing forward and backward operations, or it may wait for data loading, preprocessing, transfer, synchronization, and Python control flow.

A profile must separate stages before optimization begins. Improving matrix computation cannot help a data-bound system, while adding workers cannot fix a model whose kernels already saturate the device.

Part 2 - Mathematical form

$$T_{\text{step}} = T_{\text{data}} + T_{\text{transfer}} + T_f + T_b + T_o + T_{\text{sync}}, \qquad Q = B/T_{\text{step}}.$$

Part 3 - Hints and Helper

Warm up the pipeline, profile representative steady-state steps, and classify the bottleneck from time fractions and device activity.

Part 4 - Python code skeleton

```
import time
import torch
```

```
def E00(profile_step, steps, device):
    records = []
    for _ in range(steps):
        if device.type == "cuda": torch.cuda.synchronize(device)
        start = time.perf_counter()
        record = profile_step(...)
        if device.type == "cuda": torch.cuda.synchronize(device)
        record["total"] = time.perf_counter() - start
        records.append(record)
    means = {key: sum(item[key] for item in records) / len(records)
             for key in records[0]}
    bottleneck = max(("data", "transfer", "forward", "backward", "optimizer"),
                     key=lambda key: means[key])
    return records, means, bottleneck

def main(): print(E00(...))
if __name__ == "__main__": main()
```

13.10.2 Exercise 01 - Measure DataLoader Impact Using num_workers and Pin Memory Configuration

Part 1 - Why this matters

DataLoader worker count determines how much data preparation can overlap with model computation. Pinned host memory can accelerate transfers to CUDA when batches use non-blocking copies, but it consumes system resources and is not automatically beneficial.

The best configuration depends on storage, CPU cores, transforms, batch size, and hardware. A benchmark should include startup and steady-state separately and guard against excessive workers or pinned-memory pressure.

Part 2 - Mathematical form

$$Q(w,p) = N/T_{\text{steady}}(w,p), \qquad w = \text{worker count},\ p \in \{0,1\}.$$

Part 3 - Hints and Helper

Sweep bounded worker counts and pin-memory settings. Repeat runs, preserve identical sample work, and report throughput and data-wait time.

Part 4 - Python code skeleton

```
import itertools

def E01(build_loader, benchmark, worker_counts, pin_options):
    results = []
    for workers, pin_memory in itertools.product(worker_counts, pin_options):
        loader = build_loader(num_workers=workers,
                              pin_memory=pin_memory,
```

```
                                persistent_workers=workers > 0)
            metrics = benchmark(loader, non_blocking=pin_memory, ...)
            results.append({"workers": workers, "pin_memory": pin_memory,
                            "metrics": metrics})
    best = max(results, key=lambda item: item["metrics"]["throughput"])
    return results, best

def main(): print(E01(...))
if __name__ == "__main__": main()
```

13.10.3 Exercise 02 - Implement Gradient Accumulation for Memory-Constrained GPUs and Validate Equivalence

Part 1 - Why this matters

Gradient accumulation divides an effective batch into micro-batches and delays the optimizer update. It reduces activation memory while preserving the desired mean gradient when each micro-batch contribution is weighted correctly.

Equivalence is exact only under controlled conditions without stochastic layers or batch-dependent behavior. The validation should compare gradients and updated parameters with a single full-batch step.

Part 2 - Mathematical form

$$g_{\text{eff}} = \sum_k \frac{B_k}{\sum_j B_j} g_k, \qquad B_{\text{eff}} = \sum_k B_k.$$

Part 3 - Hints and Helper

Use cloned models and identical samples. Keep stochastic behavior compatible with evaluation, and weight the loss by sample count when the final micro-batch is incomplete.

Part 4 - Python code skeleton

```
import copy
import torch

def E02(model, micro_batches, loss_fn, optimizer_factory):
    accumulated = copy.deepcopy(model)
    optimizer = optimizer_factory(accumulated.parameters())
    total = sum(targets.shape[0] for _, targets in micro_batches)
    optimizer.zero_grad(set_to_none=True)
    for features, targets in micro_batches:
        weight = targets.shape[0] / total
        (loss_fn(accumulated(features), targets) * weight).backward()
    gradients = [p.grad.detach().clone() for p in accumulated.parameters()]
    optimizer.step()
```

```
    full_batch_reference = ...
    return gradients, full_batch_reference, ...

def main(): print(E02(...))
if __name__ == "__main__": main()
```

13.10.4 Exercise 03 - Implement Mixed Precision Training and Quantify Speed vs Stability Tradeoffs

Part 1 - Why this matters

Mixed precision can improve tensor-core throughput and reduce memory, allowing larger batches. Its smaller numerical range can also introduce overflow, underflow, or skipped optimizer steps that alter training dynamics.

A useful comparison records throughput, peak memory, scaler history, non-finite events, skipped updates, convergence, and validation quality. Speed is valuable only when numerical stability and model quality remain acceptable.

Part 2 - Mathematical form

$$\text{benefit} = Q_{MP}/Q_{FP}, \qquad \text{cost} = R_{MP} - R_{FP} + p_{\text{skipped}}.$$

Part 3 - Hints and Helper

Start both modes from the same checkpoint and data order. Synchronize timing and compare several seeds.

Part 4 - Python code skeleton

```
def E03(run_training, checkpoint, precisions, seeds):
    results = []
    for precision in precisions:
        for seed in seeds:
            outcome = run_training(checkpoint=checkpoint,
                                   precision=precision,
                                   seed=seed, ...)
            results.append({"precision": precision,
                            "seed": seed, "outcome": outcome})
    tradeoff = ...
    return results, tradeoff

def main(): print(E03(...))
if __name__ == "__main__": main()
```

13.10.5 Exercise 04 - Compare Different Batch Sizes and Identify the Throughput-Optimal Configuration

Part 1 - Why this matters

Increasing batch size can improve device occupancy and amortize overhead until memory pressure, bandwidth, or computation reaches saturation. The largest batch that fits is not necessarily the one with the best throughput or latency.

Batch size also changes optimization noise, so systems throughput and model convergence should be reported separately. Production selection may optimize samples per second under a memory and quality constraint.

Part 2 - Mathematical form

$$B^* = \arg\max_{B \in \mathcal{B}} Q(B) \quad \text{subject to} \quad M(B) \le M_{\max},\ R(B) \le R_{\max}.$$

Part 3 - Hints and Helper

Warm up every size, synchronize measurements, track peak memory, and stop cleanly at out-of-memory configurations.

Part 4 - Python code skeleton

```
def E04(run_benchmark, batch_sizes, memory_limit):
    results = []
    for batch_size in batch_sizes:
        try:
            metrics = run_benchmark(batch_size, ...)
            results.append({"batch_size": batch_size,
                            "success": True, **metrics})
            if metrics["peak_memory"] > memory_limit: break
        except RuntimeError as error:
            results.append({"batch_size": batch_size,
                            "success": False, "error": str(error)})
            break
    valid = [item for item in results if item["success"]]
    best = max(valid, key=lambda item: item["throughput"])
    return results, best

def main(): print(E04(...))
if __name__ == "__main__": main()
```

13.10.6 Exercise 05 - Monitor GPU Utilization and Fix Under-Utilization Caused by Input Pipeline Issues

Part 1 - Why this matters

Low utilization with long data-wait intervals indicates that the GPU is starved by the input pipeline. Slow storage, expensive CPU transforms, insufficient workers, synchronous copies, or small batches can create this pattern.

The fix must target the measured cause. Increasing workers blindly can create contention, while caching or moving transforms may help only when the workload and memory budget support them.

Part 2 - Mathematical form

$$U \downarrow \wedge T_{\text{data}}/T_{\text{step}} \uparrow \Rightarrow \text{input starvation candidate.}$$

Part 3 - Hints and Helper

Establish a baseline, change one pipeline factor at a time, and verify higher utilization together with higher end-to-end throughput.

Part 4 - Python code skeleton

```
def E05(profile_pipeline, interventions):
    baseline = profile_pipeline({}, ...)
    results = []
    for name, configuration in interventions.items():
        profile = profile_pipeline(configuration, ...)
        results.append({"intervention": name,
                        "profile": profile,
                        "throughput_gain": profile["throughput"]
                                           / baseline["throughput"] - 1})
    successful = [r for r in results if r["throughput_gain"] > 0]
    return baseline, results, successful

def main(): print(E05(...))
if __name__ == "__main__": main()
```

13.10.7 Exercise 06 - Benchmark Inference Latency and Throughput for Production Deployment Settings

Part 1 - Why this matters

Inference latency measures the response time of individual requests, while throughput measures completed samples per unit time. They depend on batch size, concurrency, precision, device, compilation, and service overhead.

Averages hide tail behavior important to users. Production benchmarking should report warm and cold latency, median and high percentiles, throughput, memory, and output correctness under realistic input shapes.

Part 2 - Mathematical form

$$Q = N/T, \qquad L_{50} = P_{50}(L), \quad L_{95} = P_{95}(L), \quad L_{99} = P_{99}(L).$$

Part 3 - Hints and Helper

Use evaluation mode and inference context, warm up the model, synchronize device work, and exclude or include preprocessing explicitly.

Part 4 - Python code skeleton

```
import time
import torch

def E06(model, batches, device):
    model.eval(); latencies = []; samples = 0
    with torch.inference_mode():
        for batch in batches:
            if device.type == "cuda": torch.cuda.synchronize(device)
            start = time.perf_counter()
            output = model(batch)
            if device.type == "cuda": torch.cuda.synchronize(device)
            latencies.append(time.perf_counter() - start)
            samples += batch.shape[0]
    values = torch.tensor(latencies)
    return {"throughput": samples / values.sum().item(),
            "p50": values.quantile(0.50).item(),
            "p95": values.quantile(0.95).item(),
            "p99": values.quantile(0.99).item(), "last_output": output}

def main(): print(E06(...))
if __name__ == "__main__": main()
```

13.10.8 Exercise 07 - Export MLP Model Using TorchScript or torch.compile and Validate Output Parity

Part 1 - Why this matters

Model export or compilation can reduce Python overhead and enable optimized execution, but graph capture may impose shape, control-flow, or environment constraints. A successful export is not proof that behavior remains correct.

Parity tests should cover representative shapes, data types, devices, boundary values, and repeated calls. Differences must be evaluated with tolerances appropriate to precision, followed by latency and portability measurements.

Part 2 - Mathematical form

$$\max_X \|f_{\text{reference}}(X) - f_{\text{export}}(X)\|_\infty < \varepsilon.$$

Part 3 - Hints and Helper

Export in evaluation mode, test multiple inputs against the original model, save and reload the artifact when applicable, then benchmark.

Part 4 - Python code skeleton

```
import torch

def E07(model, example, test_inputs, method):
    model.eval()
    if method == "torchscript":
        exported = torch.jit.trace(model, example)
    elif method == "compile":
        exported = torch.compile(model)
    else:
        raise ValueError(...)
    parity = []
    with torch.inference_mode():
        for values in test_inputs:
            reference = model(values)
            candidate = exported(values)
            parity.append(torch.allclose(reference, candidate,
                                         atol=..., rtol=...))
    return exported, parity

def main(): print(E07(...))
if __name__ == "__main__": main()
```

13.10.9 Exercise 08 - Implement Checkpoint Rotation and Storage Strategy for Long Training Runs

Part 1 - Why this matters

Long runs can generate many large checkpoints and exhaust storage. Rotation preserves recent recovery points and selected milestones while removing redundant files according to an explicit retention policy.

Deletion must never remove the last known-good checkpoint or a file still being written. Atomic saves, integrity hashes, manifests, and separate best-model retention protect recovery from partial or corrupted state.

Part 2 - Mathematical form

$$K = K_{\text{recent}} \cup K_{\text{milestone}} \cup \{K_{\text{best}}\}, \qquad D = K_{\text{all}} \setminus K.$$

Part 3 - Hints and Helper

Write checkpoints atomically, verify them, update a manifest, then delete only files outside the retention set.

Part 4 - Python code skeleton

```
def E08(checkpoints, keep_recent, milestone_interval, best_id):
    ordered = sorted(checkpoints, key=lambda item: item["step"])
    keep = {item["id"] for item in ordered[-keep_recent:]}
    keep.update(item["id"] for item in ordered
                if item["step"] % milestone_interval == 0)
    keep.add(best_id)
    delete = [item for item in ordered
              if item["id"] not in keep and item["verified"]]
    manifest = {"keep": sorted(keep),
                "delete": [item["id"] for item in delete]}
    return manifest

def main(): print(E08(...))
if __name__ == "__main__": main()
```

13.10.10 Exercise 09 - Detect Memory Leaks and Unexpected GPU Growth Across Long Training Sessions

Part 1 - Why this matters

A memory leak occurs when references to computation graphs or tensors survive beyond their intended step. Appending losses without detaching, retaining graphs repeatedly, persistent hooks, or cached outputs can cause allocated memory to grow over time.

Caching by the allocator is not automatically a leak, so allocated, reserved, and peak memory must be distinguished. A leak test repeats identical work and examines sustained allocated growth after warm-up.

Part 2 - Mathematical form

$$\Delta M_t = M_{\text{allocated},t} - M_{\text{allocated},t-k}, \qquad \Delta M_t > 0 \text{ persistently} \Rightarrow \text{leak candidate.}$$

Part 3 - Hints and Helper

Log memory at fixed intervals, keep workload constant, and inspect collections for tensors that still have gradient functions.

Part 4 - Python code skeleton

```
import torch

def E09(run_step, steps, interval, device):
    records = []
    for step in range(steps):
        run_step(...)
```

```
        if step % interval == 0:
            torch.cuda.synchronize(device)
            records.append({"step": step,
                            "allocated": torch.cuda.memory_allocated(device),
                            "reserved": torch.cuda.memory_reserved(device),
                            "peak": torch.cuda.max_memory_allocated(device)})
    growth = records[-1]["allocated"] - records[0]["allocated"]
    return records, growth, growth > ...

def main(): print(E09(...))
if __name__ == "__main__": main()
```

13.10.11 Exercise 10 - Integrate Model Saving into Object Storage Workflow and Validate Retrieval Correctness

Part 1 - Why this matters

Object storage separates model artifacts from one machine and supports durable training and deployment workflows. A checkpoint should include metadata, a stable object key, integrity information, and a local atomic serialization step before upload.

Upload success alone is insufficient. The object must be retrieved, verified by hash, loaded into a compatible model, and tested for prediction parity with the pre-upload state.

Part 2 - Mathematical form

$$h_{\text{local}} = H(B_{\text{local}}), \qquad h_{\text{download}} = H(B_{\text{download}}), \qquad f_\theta(X) = f_{\theta'}(X).$$

Part 3 - Hints and Helper

Upload a versioned object without embedding credentials in code. Download to a new path, verify its hash, load it, and compare outputs.

Part 4 - Python code skeleton

```
import hashlib
import torch

def file_hash(path):
    digest = hashlib.sha256()
    with open(path, "rb") as stream:
        for block in iter(lambda: stream.read(1024 * 1024), b""):
            digest.update(block)
    return digest.hexdigest()

def E10(storage, local_path, object_key, download_path,
        model, model_factory, example):
    storage.upload(local_path, object_key)
    storage.download(object_key, download_path)
```

```
    integrity = file_hash(local_path) == file_hash(download_path)
    restored = model_factory()
    restored.load_state_dict(torch.load(download_path, map_location="cpu"))
    parity = torch.allclose(model.cpu()(example), restored(example), ...)
    return integrity, parity

def main(): print(E10(...))
if __name__ == "__main__": main()
```

13.10.12 Exercise 11 - Build a Minimal Inference API Wrapper and Validate Input Validation and Error Handling

Part 1 - Why this matters

An inference wrapper defines a stable contract between raw requests and the model. It must validate feature count, batch limits, data type, finite values, schema version, and device placement before execution.

Errors should be explicit and safe rather than exposing internal traces or producing silent coercions. Responses require version metadata and consistent output formatting so clients can interpret predictions correctly.

Part 2 - Mathematical form

$$\text{request} \xrightarrow{\text{validate}} X \xrightarrow{\text{preprocess}} \widetilde{X} \xrightarrow{f_\theta} \widehat{Y} \xrightarrow{\text{format}} \text{response}\,.$$

Part 3 - Hints and Helper

Separate validation, preprocessing, inference, and formatting functions. Test valid, malformed, oversized, and non-finite requests.

Part 4 - Python code skeleton

```
import torch

class InferenceWrapper:
    def __init__(self, model, feature_count, max_batch, version):
        self.model = model.eval()
        self.feature_count = feature_count
        self.max_batch = max_batch
        self.version = version

    def predict(self, payload):
        values = torch.as_tensor(payload, dtype=torch.float32)
        if values.ndim != 2 or values.shape[1] != self.feature_count:
            raise ValueError(...)
        oversized = values.shape[0] > self.max_batch
        invalid = not torch.isfinite(values).all()
```

```
        if oversized or invalid:
            raise ValueError(...)
        with torch.inference_mode(): output = self.model(values)
        return {"model_version": self.version,
                "predictions": output.cpu().tolist()}

def E11(wrapper, test_cases): return [...]
def main(): print(E11(...))
if __name__ == "__main__": main()
```

13.10.13 Exercise 12 - Implement Deterministic Training Controls and Explain Remaining Sources of Non-Determinism

Part 1 - Why this matters

Deterministic controls fix pseudorandom seeds and request deterministic algorithms, helping repeated runs reproduce results. They can reduce performance and do not control every external source such as hardware, driver, library, distributed timing, or data-order changes.

The experiment should repeat an entire short training run and compare losses and parameters. Any remaining difference must be documented with environment and operation evidence rather than attributed vaguely to the GPU.

Part 2 - Mathematical form

$$R(C, s)_1 \stackrel{?}{=} R(C, s)_2, \qquad C = \text{software, hardware, data, algorithm controls.}$$

Part 3 - Hints and Helper

Centralize seed setup, control loader workers, enable deterministic algorithms, and record operations that lack deterministic implementations.

Part 4 - Python code skeleton

```
import random
import torch

def E12(seed, run_training):
    def configure():
        random.seed(seed); torch.manual_seed(seed)
        if torch.cuda.is_available(): torch.cuda.manual_seed_all(seed)
        torch.use_deterministic_algorithms(True)
        torch.backends.cudnn.benchmark = False
    configure(); first = run_training(...)
    configure(); second = run_training(...)
    return {"losses_equal": first["losses"] == second["losses"],
            "parameters_equal": ...,
            "remaining_sources": [...]}
```

```
def main(): print(E12(...))
if __name__ == "__main__": main()
```

13.10.14 Exercise 13 - Add Full Experiment Tracking and Ensure Reproducibility Across Machines and GPUs

Part 1 - Why this matters

Cross-machine reproducibility requires more than a seed. The run must preserve resolved configuration, code revision, environment, data identity, model state, optimizer state, metrics, hardware, precision, and artifact hashes.

Exact numerical equality may not be possible across GPU architectures, so the experiment must define whether it requires bitwise determinism or metric parity within tolerance. A replay procedure should reconstruct the run from recorded artifacts.

Part 2 - Mathematical form

$$E = (C, V, D, H, P, s, A, M), \qquad \text{replay}(E) \approx R.$$

Part 3 - Hints and Helper

Create a run manifest before training, append immutable metrics, hash artifacts, and test replay on a second environment when available.

Part 4 - Python code skeleton

```
import platform
import torch

def E13(config, artifacts, metrics, code_revision, data_hash):
    manifest = {"config": config, "code_revision": code_revision,
                "data_hash": data_hash, "torch": torch.__version__,
                "cuda": torch.version.cuda,
                "python": platform.python_version(),
                "gpu": torch.cuda.get_device_name(0)
                       if torch.cuda.is_available() else "cpu",
                "artifacts": artifacts, "metrics": metrics,
                "artifact_hashes": ...}
    replay_result = ...
    return manifest, replay_result

def main(): print(E13(...))
if __name__ == "__main__": main()
```

13.10.15 Exercise 14 - Stress Test Training at Scale with Large Synthetic Dataset and Monitor System Limits

Part 1 - Why this matters

Synthetic data isolates system capacity from dataset storage and labeling concerns. It can reveal throughput ceilings, out-of-memory boundaries, host-memory pressure, thermal throttling, and long-run numerical instability.

A stress test must be bounded and recoverable. Load should increase gradually, monitoring should cover GPU and host resources, and the test should stop at defined safety thresholds rather than treating a crash as the target.

Part 2 - Mathematical form

$$S = (B, d, L, N, Q, M_{GPU}, M_{host}, T, \text{finite}).$$

Part 3 - Hints and Helper

Sweep one dimension at a time, record the last successful configuration, and separate system-limit failures from model numerical failures.

Part 4 - Python code skeleton

```
import torch

def E14(run_configuration, configurations, limits):
    results = []
    for configuration in configurations:
        try:
            metrics = run_configuration(configuration, ...)
            safe = (metrics["gpu_memory"] <= limits["gpu_memory"]
                    and metrics["host_memory"] <= limits["host_memory"]
                    and metrics["temperature"] <= limits["temperature"])
            results.append({"configuration": configuration,
                            "success": True, "safe": safe,
                            "metrics": metrics})
            if not safe: break
        except RuntimeError as error:
            results.append({"configuration": configuration,
                            "success": False, "error": str(error)})
            break
    return results

def main(): print(E14(...))
if __name__ == "__main__": main()
```

13.10.16 Exercise 15 - Validate Correctness by Intentionally Overfitting Tiny Dataset and Confirming Perfect Fit

Part 1 - Why this matters

A production pipeline should first prove that model, loss, labels, gradients, optimizer, and data flow can fit a tiny deterministic dataset. Failure indicates a correctness or capacity problem before scaling concerns matter.

The test is intentionally opposite to generalization. Augmentation and regularization should be disabled, a strict target set in advance, and the failing artifacts preserved if perfect fit is not reached.

Part 2 - Mathematical form

$$\exists T : \mathcal{L}_T < \varepsilon \ \wedge \ \text{Accuracy}_T = 1 \quad \text{on the tiny training set.}$$

Part 3 - Hints and Helper

Use a handful of verified examples and enough model capacity. Stop at the target or emit a full diagnostic report.

Part 4 - Python code skeleton

```
def E15(model, tiny_loader, optimizer, loss_fn, max_steps, target_loss):
    history = []
    for step in range(max_steps):
        for features, labels in tiny_loader:
            optimizer.zero_grad(set_to_none=True)
            logits = model(features); loss = loss_fn(logits, labels)
            loss.backward(); optimizer.step()
        accuracy = (logits.argmax(1) == labels).float().mean().item()
        history.append((step, loss.item(), accuracy))
        if loss.item() < target_loss and accuracy == 1.0:
            return {"passed": True, "history": history}
    return {"passed": False, "history": history,
            "diagnostics": ...}

def main(): print(E15(...))
if __name__ == "__main__": main()
```

13.10.17 Exercise 16 - Implement Regression Tests That Detect Training Quality Degradation After Refactors

Part 1 - Why this matters

Code refactors can preserve syntax and forward execution while changing initialization, loss scaling, data order, optimizer timing, or checkpoint behavior. Regression tests protect known training properties from silent degradation.

Tests should use small deterministic workloads and compare against toleranced baselines. They must cover functional correctness, short-run learning, tiny-set fit, numerical finiteness, throughput budgets, and serialization parity.

Part 2 - Mathematical form

$$|m_{\text{new}} - m_{\text{base}}| \le \tau_m, \qquad Q_{\text{new}} \ge (1 - \tau_q) Q_{\text{base}}.$$

Part 3 - Hints and Helper

Store baseline ranges rather than one fragile floating-point value and fail with a diagnostic diff that identifies the changed property.

Part 4 - Python code skeleton

```
def E16(run_suite, baseline, tolerances):
    current = run_suite(...)
    checks = {}
    for metric, expected in baseline.items():
        difference = current[metric] - expected
        checks[metric] = {"passed": abs(difference) <= tolerances[metric],
                          "expected": expected,
                          "current": current[metric],
                          "difference": difference}
    checks["all_finite"] = {"passed": current["all_finite"]}
    return {"passed": all(item["passed"] for item in checks.values()),
            "checks": checks}

def main(): print(E16(...))
if __name__ == "__main__": main()
```

13.10.18 Exercise 17 - Build a Production Readiness Checklist Covering Performance, Stability, and Monitoring

Part 1 - Why this matters

Production readiness requires verified behavior under realistic load and failure, not merely a good validation score. The system must meet latency, throughput, memory, stability, observability, security, rollback, and data-contract requirements.

A checklist should contain measurable acceptance criteria and evidence links. Unknown or failed items must block release when they concern correctness, recovery, or safety rather than being converted into optimistic assumptions.

Part 2 - Mathematical form

$$\text{release} = \bigwedge_{r \in R_{\text{critical}}} \text{pass}(r).$$

Part 3 - Hints and Helper

Classify requirements by severity, require evidence, identify owners, and keep rollback and monitoring tests in the release gate.

Part 4 - Python code skeleton

```
def E17(items):
    required = {"correctness", "latency", "throughput", "memory",
                "stability", "monitoring", "data_validation",
                "artifact_integrity", "rollback", "recovery",
                "security", "capacity_limits"}
    missing = sorted(required - items.keys())
    blockers = [name for name, item in items.items()
                if item.get("critical")
                and (item.get("status") != "passed"
                     or not item.get("evidence"))]
    return {"ready": not missing and not blockers,
            "missing": missing, "blockers": blockers, "items": items}

def main(): print(E17(...))
if __name__ == "__main__": main()
```

13.10.19 Exercise 18 - Simulate Failure Recovery by Resuming from Checkpoint After Forced Interruption

Part 1 - Why this matters

Recovery testing proves that a checkpoint can restore a run after process interruption. The recovered trajectory requires model, optimizer, scheduler, scaler, counters, random states, and data-sampler position to be preserved consistently.

A forced interruption should occur at a known step, followed by restoration in a fresh process-like state. The recovered next update must be compared with an uninterrupted control to detect drift.

Part 2 - Mathematical form

$$C_t \xrightarrow{\text{restore}} S_t, \qquad U(S_t, B_{t+1}) \stackrel{?}{=} U(S_t^{\text{control}}, B_{t+1}).$$

Part 3 - Hints and Helper

Save before interruption, discard in-memory objects, rebuild them, restore all states, and compare the next loss, learning rate, and parameters.

Part 4 - Python code skeleton

```
def E18(run_until, save_checkpoint, rebuild_and_restore,
        continue_training, interruption_step):
    control_state = run_until(interruption_step, ...)
    checkpoint_path = save_checkpoint(control_state, ...)
    control_next = continue_training(control_state, steps=1, ...)
    recovered_state = rebuild_and_restore(checkpoint_path, ...)
    recovered_next = continue_training(recovered_state, steps=1, ...)
    return {"loss_match": ...,
            "learning_rate_match": ...,
            "parameter_match": ...,
            "control": control_next,
            "recovered": recovered_next}

def main(): print(E18(...))
if __name__ == "__main__": main()
```

13.10.20 Exercise 19 - Produce a Final End-to-End MLP Engineering Report with Lessons Learned and Best Practices

Part 1 - Why this matters

The final engineering report connects mathematical correctness, training behavior, system performance, artifact management, inference reliability, monitoring, and recovery into one auditable account. It explains not only what worked but why the evidence supports deployment.

The report must preserve failures and lessons alongside successful results. Remaining risks, capacity boundaries, environment assumptions, rollback procedures, and future tests should be explicit so the MLP can evolve without losing verified behavior.

Part 2 - Mathematical form

$$\text{design} \to \text{train} \to \text{validate} \to \text{profile} \to \text{package} \to \text{deploy} \to \text{monitor} \to \text{recover}.$$

Part 3 - Hints and Helper

Require evidence for every major conclusion and include links to benchmarks, parity tests, regression tests, failure recovery, and the production gate.

Part 4 - Python code skeleton

```
def E19(sections):
    required = {
        "architecture", "training_correctness", "generalization",
        "profiling", "data_pipeline", "memory_and_scaling",
        "mixed_precision", "inference_benchmarks", "export_parity",
```

```
        "checkpoint_strategy", "leak_detection", "object_storage",
        "api_contract", "determinism", "experiment_tracking",
        "stress_testing", "tiny_dataset_test", "regression_tests",
        "production_checklist", "failure_recovery",
        "lessons_learned", "remaining_risks", "best_practices",
    }
    missing = sorted(required - sections.keys())
    unsupported = [name for name, item in sections.items()
                   if isinstance(item, dict) and item.get("conclusion")
                   and not item.get("evidence")]
    return {"complete": not missing and not unsupported,
            "missing": missing, "unsupported": unsupported,
            "report": sections}

def main(): print(E19(...))
if __name__ == "__main__": main()
```

Book 14

CNN (Convolutional Neural Networks)

14.1 Introduction

14.1.1 What will the book teach you

This book marks the transition from general-purpose feed-forward networks to models that are designed to reason about spatial structure. In an MLP, every input feature can be connected to every unit in the following layer. That approach is useful, but it ignores the fact that nearby pixels usually form local patterns and that the same pattern may appear in different positions. Convolutional Neural Networks introduce a stronger assumption: useful visual information can be detected locally with shared filters and combined gradually into more complex representations. The reader will learn why this inductive bias makes CNNs more parameter-efficient and better suited to images than a large fully connected network.

The book begins by making convolution completely visible. The reader will implement the operation with explicit loops, compare convolution with cross-correlation, follow a kernel as it moves across an image, and calculate every output dimension manually. Kernel size, stride, padding, dilation, channels, and batch dimensions will not be treated as isolated settings. Each one changes what information can be observed, how much spatial detail is preserved, and how expensive the computation becomes. Manual results will then be compared with PyTorch implementations so that library functions are used with understanding rather than memorization.

Particular attention will be given to tensor layout. Images commonly enter a PyTorch CNN in the form `(N,C,H,W)`, while convolutional kernels use the compact form `(Cout,Cin,kH,kW)` for output channels, input channels, and kernel height and width. The reader will learn how multiple input channels contribute to one output feature map and how multiple filters create several output channels. This foundation is essential because a CNN may run without an obvious error while still using incorrect channel order, normalization, or spatial dimensions. Shape reasoning will therefore remain a continuous responsibility throughout the book.

Feature maps and receptive fields will connect tensor operations to visual meaning. Early layers may respond to edges, orientations, colors, or textures, while deeper layers combine these responses over progressively larger regions. The reader will calculate receptive-field growth, create and inspect custom filters, capture intermediate activations, and test translation equivariance. The exercises will also show where this useful property can weaken because of padding, stride, downsampling, aliasing, or boundary effects. A feature map must be interpreted as a

learned response, not simply as another tensor produced by the network.

Downsampling will be studied as a deliberate information trade-off. Max pooling, average pooling, adaptive pooling, global average pooling, strided convolution, blur pooling, and anti-aliased alternatives will be compared in terms of output size, computational cost, invariance, and lost detail. Reducing resolution can enlarge the effective receptive field and lower memory usage, but aggressive reduction can destroy small or rare features before later layers can use them. The reader will learn to measure this loss and design a downsampling strategy according to the data rather than following a fixed architectural habit.

After the individual operations are understood, the reader will assemble a complete CNN forward architecture. Convolutional blocks, activation functions, normalization, pooling, classifier heads, global average pooling, dropout, and skip connections will be combined while every intermediate shape is verified. The book will compare explicit forward functions with `nn.Sequential` and configurable builders, inspect parameter counts and memory footprints, and use forward hooks to expose hidden states. The objective is to make architecture design explainable: every block must have a purpose, a valid tensor contract, and a measurable effect on representation or efficiency.

Training a CNN introduces interactions that are easy to overlook. The reader will construct complete training and validation loops, select loss functions that match the output representation, inspect layer-wise gradients, verify backpropagation numerically on small examples, and diagnose vanishing, exploding, or non-finite values. Learning-rate schedules, gradient clipping, gradient accumulation, mixed precision, checkpoint restoration, and CUDA execution will be tested as parts of one system. A decreasing loss will not be accepted as sufficient evidence of correctness; output shapes, labels, gradients, metrics, and deliberately overfitted tiny datasets will all be used to validate the pipeline.

The data pipeline receives its own chapter because a vision model can learn only from the images that actually reach it. The reader will load and normalize images, verify transformations visually, prevent leakage between dataset splits, and study random crops, flips, rotations, color changes, erasing, MixUp, and CutMix. Augmentation will be treated as an assumption about which changes should preserve a class label. An invalid transformation may silently teach the wrong task, while insufficient variation may encourage memorization. The exercises will therefore include ablation studies, corrupted-image checks, duplicate detection, balanced sampling, and DataLoader performance analysis.

Generalization and robustness will be examined beyond ordinary validation accuracy. The reader will compare weight decay, dropout, normalization, label smoothing, test-time augmentation, and ensembling, then observe performance under noise, blur, occlusion, lighting changes, color shifts, and adversarial-like perturbations. Reliability diagrams and calibration error will show that a confident prediction is not necessarily a correct or trustworthy prediction. Failure cases will be grouped and interpreted so that optimization failure, overfitting, data shift, and model sensitivity are not confused with one another.

Modern architectural patterns will explain how contemporary CNNs become deeper and more efficient. The reader will implement VGG-style blocks, residual connections, bottlenecks, grouped convolution, depthwise separable convolution, inverted residuals, squeeze-and-

excitation attention, stochastic depth, and DropPath. These designs will be compared using parameter count, computational operations, activation memory, GPU speed, accuracy, and stability. A block that uses fewer mathematical operations does not automatically run faster on every device, so theoretical efficiency and measured hardware behavior will be kept separate.

Finally, the book moves from a trained CNN to a model that can survive outside the notebook. The reader will benchmark latency and throughput, export models with TorchScript and ONNX, investigate `torch.compile`, compare numerical precision and quantization choices, and verify output parity against a trusted reference. Production exercises will cover preprocessing contracts, input validation, corrupt-batch handling, checkpoint integrity, model versioning, monitoring, distribution drift, rollback, A/B testing, memory leaks, and load testing. The final goal is not merely a CNN that achieves good accuracy once, but a system whose architecture, data, training behavior, performance, limitations, and recovery procedures can all be explained and tested.

14.1.2 Why this book comes after the MLP

The MLP established the central language of neural networks: layers transform representations, non-linear activations create expressive functions, backpropagation assigns credit, and optimizers update parameters. A CNN uses the same learning machinery, but changes the structure of the transformation. If an image with height H, width W, and C channels is flattened and passed to a fully connected layer with M outputs, the weight count grows as $MCHW$. A convolutional layer instead reuses a local kernel across spatial positions, allowing the same detector to respond wherever a pattern appears. This weight sharing preserves spatial relationships and usually requires far fewer parameters.

This progression matters because convolution should not feel like an unrelated collection of image functions. It is a specialized linear transformation placed inside the same forward, loss, backward, and optimization process already learned in the MLP book. The reader can therefore reuse knowledge about activations, gradients, initialization, optimizers, generalization, CUDA, and deployment while focusing on the new questions introduced by space: locality, channels, receptive fields, equivariance, downsampling, image preprocessing, and visual failure analysis.

14.1.3 How to study this book

Before implementing an operation, write down the input shape, kernel shape, stride, padding, expected output shape, parameter count, and receptive-field effect. For a two-dimensional convolution, the spatial output should be checked manually using

$$H_{\text{out}} = \left\lfloor \frac{H + 2P_h - D_h(K_h - 1) - 1}{S_h} + 1 \right\rfloor, \qquad W_{\text{out}} = \left\lfloor \frac{W + 2P_w - D_w(K_w - 1) - 1}{S_w} + 1 \right\rfloor.$$

After the code runs, inspect more than the final class label. Visualize inputs after preprocessing, check intermediate feature maps, record activation and gradient statistics, and verify devices, data types, memory use, and output parity. When comparing architectures or training settings, hold the dataset, split, seed, evaluation procedure, and computational budget constant. A speed comparison should include warm-up and synchronization, while an accuracy comparison should include repeated runs or clearly state its uncertainty.

The exercises intentionally include manual implementations, controlled experiments, CUDA measurements, and failure scenarios. Do not skip the manual work simply because PyTorch offers a ready-made layer. The manual version reveals the operation; the library version provides the efficient tool; and the comparison proves that both represent the same intended computation. Every failure should end with a root-cause explanation, a correction, and a test that would detect the same problem if it returned later.

14.1.4 The Chapters (CNN00 - CNN08) included in this book

- CNN00 - Convolution Concept and Intuition
- CNN01 - Feature Maps, Filters, and Receptive Fields
- CNN02 - Pooling, Downsampling, and Invariance
- CNN03 - CNN Forward Architecture and Implementation
- CNN04 - CNN Training Loop, Loss, and Backpropagation
- CNN05 - Data Pipeline and Augmentation for Vision
- CNN06 - Regularization, Normalization, and Robustness
- CNN07 - Modern CNN Blocks and Architectural Patterns
- CNN08 - Deployment, Export, and Production Readiness

14.2 CNN00 - Convolution Concept and Intuition

Models and Training

14.2.1 Exercise 00 - Understand Convolution as Local Pattern Detection Instead of Global Fully-Connected Mixing

Part 1 - Why this matters

A fully connected layer mixes every input location into every output unit, so it does not preserve locality by design. Convolution instead examines small neighborhoods and reuses the same detector across positions, making local structure an explicit assumption of the model.

This exercise asks you to compare both transformations on a tiny image. The goal is to explain parameter sharing, spatial output, and why the same edge or texture can be recognized at several locations without learning a separate weight for each location.

Part 2 - Mathematical form

$$Y_{n,o,i,j} = b_o + \sum_c \sum_u \sum_v X_{n,c,i+u,j+v} K_{o,c,u,v}.$$

Part 3 - Hints and Helper

Count how many weights are needed by each method, then move one local pattern to another valid position and compare the responses.

Part 4 - Python code skeleton

```
import torch

def E00(image, kernel):
    local_response = ...
    flattened = ...
    dense_weights = ...
    dense_response = ...
    return {
        "local": local_response,
        "dense": dense_response,
        "conv_parameters": ...,
        "dense_parameters": ...,
    }

def main():
    image = torch.tensor(..., dtype=torch.float32)
    kernel = torch.tensor(..., dtype=torch.float32)
    print(E00(image, kernel))

if __name__ == "__main__":
    main()
```

14.2.2 Exercise 01 - Implement a Simple 2D Convolution Using Explicit Nested Loops for Full Transparency

Part 1 - Why this matters

A manual implementation reveals exactly which input patch contributes to each output value. It makes indexing, kernel orientation, accumulation, and boundary assumptions visible before optimized library operations hide these details.

Implement one-channel mathematical convolution with valid boundaries and unit stride. Flip the kernel in both spatial dimensions, calculate every output element with nested loops, and retain enough intermediate information to inspect one selected patch.

Part 2 - Mathematical form

$$Y_{i,j} = \sum_{u=0}^{K_h-1} \sum_{v=0}^{K_w-1} X_{i+u,j+v} K_{K_h-1-u,K_w-1-v}.$$

Part 3 - Hints and Helper

Derive the output height and width first. For every output coordinate, extract a patch, multiply it by the flipped kernel, and sum all products.

Part 4 - Python code skeleton

```
import torch

def manual_convolution_2d(image, kernel):
    if image.ndim != 2 or kernel.ndim != 2:
        raise ValueError(...)
    out_h = ...
    out_w = ...
    flipped = ...
    output = torch.empty(...)
    for i in range(out_h):
        for j in range(out_w):
            patch = ...
            output[i, j] = ...
    return output

def E01():
    image = torch.tensor(..., dtype=torch.float32)
    kernel = torch.tensor(..., dtype=torch.float32)
    return manual_convolution_2d(image, kernel)

if __name__ == "__main__":
    print(E01())
```

14.2.3 Exercise 02 - Implement Cross-Correlation and Explain Why Deep Learning Calls It Convolution

Part 1 - Why this matters

Most deep-learning libraries implement cross-correlation: the kernel is applied without spatial flipping. It is commonly called convolution because the kernel is learned, so its orientation can adapt during training and the distinction rarely reduces model expressiveness.

Build convolution and cross-correlation from the same loop structure, compare their outputs with an asymmetric kernel, and verify which operation matches PyTorch. A symmetric kernel is a poor test because flipping it may leave it unchanged.

Part 2 - Mathematical form

$$Y_{i,j}^{\text{corr}} = \sum_{u,v} X_{i+u,j+v} K_{u,v}, \qquad Y_{i,j}^{\text{conv}} = \sum_{u,v} X_{i+u,j+v} K_{K_h-1-u,K_w-1-v}.$$

Part 3 - Hints and Helper

Use a kernel whose four corners have different values. Compare the manual results with `torch.nn.functional.conv2d` after adding batch and channel dimensions.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def manual_operation(image, kernel, flip_kernel):
    selected_kernel = ... if flip_kernel else ...
    output = torch.empty(...)
    for i in range(...):
        for j in range(...):
            output[i, j] = ...
    return output

def E02(image, kernel):
    correlation = manual_operation(image, kernel, ...)
    convolution = manual_operation(image, kernel, ...)
    pytorch_output = F.conv2d(...).squeeze()
    return correlation, convolution, pytorch_output

if __name__ == "__main__":
    print(E02(torch.tensor(...), torch.tensor(...)))
```

14.2.4 Exercise 03 - Visualize How a Kernel Slides Over an Image and Produces a Feature Map

Part 1 - Why this matters

A feature-map value is meaningful only when it can be traced back to the receptive patch that produced it. Visualizing the sliding process connects output coordinates with input locations and exposes indexing errors immediately.

Record every valid patch, its element-wise products with the kernel, and the resulting sum. Create a figure that highlights the current patch in the image and places the computed response in the corresponding feature-map cell.

Part 2 - Mathematical form

$$P_{i,j} = X[i : i + K_h,\ j : j + K_w], \qquad Y_{i,j} = \langle P_{i,j}, K \rangle_F.$$

Part 3 - Hints and Helper

Separate computation from plotting. First return a trace containing coordinates, patches, products, and sums; then use that trace to construct reproducible frames.

Part 4 - Python code skeleton

```
import matplotlib.pyplot as plt
import torch

def build_slide_trace(image, kernel):
    trace = []
    feature_map = torch.empty(...)
    for i in range(...):
        for j in range(...):
            patch = ...
            products = ...
            value = ...
            trace.append({...})
            feature_map[i, j] = value
    return feature_map, trace

def E03(image, kernel):
    feature_map, trace = build_slide_trace(image, kernel)
    fig, axes = plt.subplots(...)
    ...
    return fig, feature_map, trace
```

14.2.5 Exercise 04 - Verify Convolution Output Shape by Manual Calculation Using Kernel Size and Stride

Part 1 - Why this matters

Output-shape errors propagate into flattening, residual connections, classifier heads, and memory estimates. The shape must therefore be predicted before the convolution is executed rather than discovered from an exception afterward.

Implement a shape calculator for two spatial dimensions and validate it against several PyTorch convolutions. Include non-square inputs, kernels, strides, and padding so the calculation is not accidentally correct only for symmetric cases.

Part 2 - Mathematical form

$$H_{\text{out}} = \left\lfloor \frac{H + 2P_h - D_h(K_h - 1) - 1}{S_h} + 1 \right\rfloor, \quad W_{\text{out}} = \left\lfloor \frac{W + 2P_w - D_w(K_w - 1) - 1}{S_w} + 1 \right\rfloor.$$

Part 3 - Hints and Helper

Normalize every scalar or pair into a two-value tuple. Reject configurations that produce a non-positive output dimension.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def pair(value):
    return ...

def conv2d_output_shape(hw, kernel, stride=1, padding=0, dilation=1):
    h, w = ...
    kh, kw = ...
    sh, sw = ...
    ph, pw = ...
    dh, dw = ...
    out_h = ...
    out_w = ...
    return out_h, out_w

def E04(configs):
    checks = []
    for config in configs:
        predicted = conv2d_output_shape(...)
        actual = F.conv2d(...).shape[-2:]
        checks.append((predicted, actual, ...))
    return checks
```

14.2.6 Exercise 05 - Compare Different Kernel Sizes and Observe How Receptive Field Changes

Part 1 - Why this matters

Kernel size determines how much local context one layer can combine in a single response. Small kernels preserve locality and can be stacked, while larger kernels see wider regions immediately but require more parameters and computation.

Apply several normalized kernels to the same controlled image and compare their feature maps. Keep output dimensions comparable through appropriate padding, then distinguish the theoretical receptive field from the actual pattern response.

Part 2 - Mathematical form

$$R_1 = K, \qquad \#\theta = C_{\text{out}} C_{\text{in}} K_h K_w + C_{\text{out}}.$$

Part 3 - Hints and Helper

Use identical input and output channels for every trial. Report kernel size, parameter count, spatial output size, and response statistics.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def make_normalized_kernel(size, device):
    kernel = torch.ones(..., device=device)
    return ...

def E05(image, kernel_sizes):
    results = {}
    for size in kernel_sizes:
        kernel = make_normalized_kernel(size, image.device)
        padding = ...
        feature_map = F.conv2d(image, kernel, padding=padding)
        results[size] = {
            "output": feature_map,
            "parameters": ...,
            "mean": ...,
            "std": ...,
        }
    return results
```

14.2.7 Exercise 06 - Demonstrate the Effect of Stride on Downsampling and Information Loss

Part 1 - Why this matters

Stride controls how far the kernel moves between neighboring outputs. A larger stride reduces spatial resolution and computation, but it also skips overlapping observations and can make high-frequency structures disappear or alias into misleading patterns.

Compare stride values on smooth and rapidly alternating synthetic images. Measure output size, retained energy, and the error after a simple reconstruction so that information loss is demonstrated quantitatively as well as visually.

Part 2 - Mathematical form

$$Y_{i,j} = \sum_{u,v} X_{iS_h+u,jS_w+v} K_{u,v}, \qquad S > 1 \Rightarrow H_{\text{out}}, W_{\text{out}} \downarrow .$$

Part 3 - Hints and Helper

Use the same kernel for all trials. Upsample each reduced feature map back to a common size only for comparison; do not present interpolation as recovered information.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F
```

```
def evaluate_stride(image, kernel, stride):
    output = F.conv2d(image, kernel, stride=stride, padding=...)
    reconstructed = F.interpolate(
        output, size=image.shape[-2:], mode=...
    )
    return {
        "shape": tuple(output.shape),
        "output": output,
        "energy_ratio": ...,
        "reconstruction_error": ...,
    }

def E06(images, kernel, strides):
    return {...}
```

14.2.8 Exercise 07 - Demonstrate the Effect of Padding on Border Information Preservation and Output Size

Part 1 - Why this matters

Without padding, border pixels participate in fewer valid patches and spatial dimensions shrink after every layer. Padding can preserve size and give edge locations additional context, but the chosen padding values introduce their own assumptions outside the observed image.

Compare valid, zero, reflect, and replicate padding on an image containing strong border patterns. Count how often each original pixel contributes and inspect whether artificial edge responses are created by the padding mode.

Part 2 - Mathematical form

$$H_{\text{out}} = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1, \qquad P = \frac{K-1}{2} \text{ preserves size when } S = 1 \text{ and } K \text{ is odd.}$$

Part 3 - Hints and Helper

Apply non-zero padding modes explicitly with `F.pad`, then run convolution with zero internal padding. Compare center and border response statistics separately.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def convolve_with_padding(image, kernel, amount, mode):
    if mode == "constant":
        padded = F.pad(image, (...), mode=mode, value=...)
```

```
    else:
        padded = F.pad(image, (...), mode=mode)
    return F.conv2d(padded, kernel)

def E07(image, kernel, modes):
    results = {}
    for mode in modes:
        output = convolve_with_padding(image, kernel, ..., mode)
        results[mode] = {
            "output": output,
            "shape": tuple(output.shape),
            "border_mean": ...,
            "center_mean": ...,
        }
    return results
```

14.2.9 Exercise 08 - Implement Convolution with torch.nn.functional.conv2d and Validate It Against Manual Output

Part 1 - Why this matters

An optimized operation should be trusted only after its input convention and numerical result are matched to a transparent reference. PyTorch expects batched, channel-first tensors and implements cross-correlation rather than a flipped mathematical convolution.

Extend the manual implementation to batch, input-channel, and output-channel dimensions. Compare it with `conv2d` across several configurations using both absolute and relative tolerances, and report the largest disagreement.

Part 2 - Mathematical form

$$\text{close}(A, B) \iff |A - B| \leq \text{atol} + \text{rtol}|B|.$$

Part 3 - Hints and Helper

Start with stride one and no padding, then add options one at a time. Use float64 during the strictest reference test to reduce rounding ambiguity.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def manual_conv2d(x, weight, bias=None, stride=1, padding=0):
    output = torch.empty(..., dtype=x.dtype, device=x.device)
    for n in range(...):
        for oc in range(...):
            for i in range(...):
```

```
                for j in range(...):
                    output[n, oc, i, j] = ...
    return output

def E08(x, weight, bias):
    manual = manual_conv2d(x, weight, bias, ...)
    library = F.conv2d(x, weight, bias, ...)
    return {
        "allclose": torch.allclose(manual, library, ...),
        "maximum_error": ...,
    }
```

14.2.10 Exercise 09 - Create Multiple Filters and Observe How Each Detects Different Visual Patterns

Part 1 - Why this matters

A convolutional layer produces multiple output channels because one filter cannot represent every useful local structure. Different filters may respond to horizontal, vertical, diagonal, bright-dark, or texture patterns in the same input.

Construct a bank of named filters and apply them in one convolution call. Use synthetic images with known structures, identify where each filter should respond strongly, and compare signed responses rather than keeping only their magnitudes.

Part 2 - Mathematical form

$$Y_o = \sum_{c=1}^{C_{\text{in}}} X_c \star K_{o,c} + b_o, \qquad o = 1, \ldots, C_{\text{out}}.$$

Part 3 - Hints and Helper

Stack filters along the output-channel dimension. Normalize their scales when response magnitudes must be compared fairly.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def build_filter_bank(dtype, device):
    horizontal = torch.tensor(..., dtype=dtype, device=device)
    vertical = torch.tensor(..., dtype=dtype, device=device)
    diagonal = torch.tensor(..., dtype=dtype, device=device)
    kernels = torch.stack([...]).unsqueeze(1)
    names = [...]
    return names, kernels
```

```
def E09(images):
    names, kernels = build_filter_bank(images.dtype, images.device)
    maps = F.conv2d(images, kernels, padding=...)
    summaries = {...}
    return names, maps, summaries
```

14.2.11 Exercise 10 - Move Convolution to CUDA and Confirm Numerical Parity with CPU Results

Part 1 - Why this matters

Moving a convolution to CUDA requires the input, weights, bias, and any related state to share a compatible device and data type. Device placement errors are explicit, while small floating-point differences may appear even when both computations are correct.

Run the same fixed convolution on CPU and CUDA, move the result back for comparison, and use a justified tolerance. Record device metadata and maximum error so parity is demonstrated rather than assumed.

Part 2 - Mathematical form

$$\Delta_{\max} = \max_i \left| Y_i^{\mathrm{CPU}} - Y_i^{\mathrm{GPU}} \right|.$$

Part 3 - Hints and Helper

Create the CPU tensors first, clone them to CUDA, and avoid generating separate random inputs on each device. Skip cleanly when CUDA is unavailable.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def E10(x_cpu, weight_cpu, bias_cpu=None):
    if not torch.cuda.is_available():
        return {"available": False}
    y_cpu = F.conv2d(x_cpu, weight_cpu, bias_cpu, padding=...)
    x_gpu = ...
    weight_gpu = ...
    bias_gpu = ...
    y_gpu = F.conv2d(x_gpu, weight_gpu, bias_gpu, padding=...)
    difference = ...
    return {
        "available": True,
        "allclose": ...,
        "maximum_error": ...,
        "gpu_device": str(y_gpu.device),
    }
```

14.2.12 Exercise 11 - Benchmark CPU vs GPU Convolution Runtime for Small and Large Input Tensors

Part 1 - Why this matters

A GPU can execute large convolutions efficiently, but launch overhead, data transfer, synchronization, and insufficient parallel work can make it slower for small tensors. Performance claims must therefore be measured across realistic shapes.

Build a benchmark with warm-up, repeated measurements, and explicit CUDA synchronization. Time compute separately from host-to-device transfer and report latency, throughput, and memory for small and large cases.

Part 2 - Mathematical form

$$\text{throughput} = \frac{N_{\text{samples}}}{t}, \qquad \text{speedup} = \frac{t_{\text{CPU}}}{t_{\text{GPU}}}.$$

Part 3 - Hints and Helper

Reuse allocated tensors during the compute benchmark. Summarize a distribution of timings instead of reporting one potentially noisy observation.

Part 4 - Python code skeleton

```
import time
import torch
import torch.nn.functional as F

def synchronize(device):
    if device.type == "cuda":
        torch.cuda.synchronize(device)

def benchmark(x, weight, repeats, warmup):
    for _ in range(warmup):
        F.conv2d(x, weight, padding=...)
    times = []
    for _ in range(repeats):
        synchronize(x.device)
        start = time.perf_counter()
        ...
        synchronize(x.device)
        times.append(...)
    return {...}

def E11(cases):
    return [...]
```

14.2.13 Exercise 12 - Compare float32 and float16 Convolution Outputs and Detect Precision-Sensitive Cases

Part 1 - Why this matters

Lower precision can reduce memory use and increase GPU throughput, but float16 represents a smaller numeric range and fewer significant digits. Convolution sums many products, so scale, cancellation, and accumulation length influence the final error.

Generate controlled inputs at several magnitudes, compare float16 results with a float32 reference, and locate overflow, underflow, and relative-error growth. The exercise should identify when mixed precision is useful and when a sensitive operation needs higher precision.

Part 2 - Mathematical form

$$\epsilon_{\text{rel}} = \frac{|Y_{16} - Y_{32}|}{|Y_{32}| + \varepsilon}.$$

Part 3 - Hints and Helper

Perform the comparison on CUDA when supported. Include ordinary, very small, very large, and cancellation-heavy cases, and inspect finite values before computing summary errors.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def compare_precision(x32, w32):
    reference = F.conv2d(x32, w32, padding=...)
    low = F.conv2d(x32.half(), w32.half(), padding=...).float()
    absolute = ...
    relative = ...
    return {
        "finite": bool(torch.isfinite(low).all()),
        "max_absolute_error": ...,
        "max_relative_error": ...,
        "zero_fraction": ...,
    }

def E12(device):
    cases = {...}
    return {name: compare_precision(x, w) for name, (x, w) in cases.items()}
```

14.2.14 Exercise 13 - Understand Channels-First Format (N,C,H,W) and Debug Common Shape Mistakes

Part 1 - Why this matters

PyTorch convolution expects batch, channel, height, and width dimensions in that order. Image libraries may supply height, width, and channels instead, while a single image may omit its batch dimension entirely.

Create a validation and conversion function that accepts documented layouts and produces one canonical tensor. It must reject ambiguous shapes, verify the expected channel count, and preserve the association between images and batches.

Part 2 - Mathematical form

$$X_{\mathrm{NHWC}} \xrightarrow{\mathrm{permute}(0,3,1,2)} X_{\mathrm{NCHW}}.$$

Part 3 - Hints and Helper

Do not infer layout from shape alone when dimensions can share the same size. Require layout metadata and call `contiguous` when later operations need contiguous storage.

Part 4 - Python code skeleton

```
import torch

def to_nchw(x, layout, expected_channels):
    if layout == "HW":
        x = ...
    elif layout == "HWC":
        x = ...
    elif layout == "NHWC":
        x = ...
    elif layout == "NCHW":
        x = ...
    else:
        raise ValueError(...)
    if x.ndim != 4 or x.shape[1] != expected_channels:
        raise ValueError(...)
    return ...

def E13(test_cases):
    return [...]
```

14.2.15 Exercise 14 - Implement 1x1 Convolution and Explain Channel Mixing Without Spatial Mixing

Part 1 - Why this matters

A one-by-one convolution observes one spatial coordinate at a time but combines all input channels through a learned linear transformation. It can expand, compress, or reorganize channel representations without directly mixing neighboring pixels.

Implement the operation manually and verify that it matches matrix multiplication applied independently at every location. Compare parameter count with a larger spatial kernel and explain why one-by-one layers appear in bottlenecks and efficient architectures.

Part 2 - Mathematical form

$$Y_{n,:,i,j} = W X_{n,:,i,j} + b, \qquad W \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}}}.$$

Part 3 - Hints and Helper

Move the channel dimension to the end, apply one matrix multiplication to all positions, and permute the result back to channel-first form.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def manual_pointwise(x, weight, bias):
    matrix = weight[..., 0, 0]
    positions = x.permute(...)
    mixed = ...
    if bias is not None:
        mixed = ...
    return mixed.permute(...).contiguous()

def E14(x, out_channels):
    weight = torch.randn(..., device=x.device)
    bias = torch.randn(..., device=x.device)
    manual = manual_pointwise(x, weight, bias)
    library = F.conv2d(x, weight, bias)
    return manual, library, ...
```

14.2.16 Exercise 15 - Demonstrate Convolution as Matrix Multiplication with im2col on a Tiny Example

Part 1 - Why this matters

The im2col view rearranges every receptive patch into a column so convolution can be expressed as a matrix multiplication. This reveals the algebra used by many implementations and explains the connection between convolutional and linear transformations.

Use a tiny tensor to build the patch matrix manually, flatten the filters into rows, multiply them, and reshape the result into a feature map. Compare the result with both `unfold` and `conv2d` while observing the temporary memory expansion.

Part 2 - Mathematical form

$$Y_{\text{matrix}} = K_{\text{rows}} X_{\text{cols}}, \qquad Y = \text{reshape}(Y_{\text{matrix}}).$$

Part 3 - Hints and Helper

Choose and document one column ordering. Verify each column against its original spatial patch before multiplying the matrices.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def manual_im2col(x, kernel_size):
    columns = []
    coordinates = []
    for i in range(...):
        for j in range(...):
            patch = ...
            columns.append(...)
            coordinates.append((i, j))
    return torch.stack(columns, dim=...), coordinates

def E15(x, weight):
    columns, coordinates = manual_im2col(x, weight.shape[-2:])
    kernel_rows = ...
    matrix_output = ...
    feature_map = ...
    unfolded = F.unfold(x, kernel_size=...)
    direct = F.conv2d(x, weight)
    return {...}
```

14.2.17 Exercise 16 - Stress Test Convolution with Large Batches and Monitor GPU Memory Consumption

Part 1 - Why this matters

Convolution memory includes inputs, outputs, parameters, gradients, saved activations, and temporary workspaces. Increasing batch size may improve utilization until a sudden out-of-memory failure occurs, so usable capacity must be measured with safety margins.

Increase batch size systematically, record allocated and peak reserved GPU memory, and stop safely before destabilizing the process. Compare inference with training because backward propagation retains additional state and produces parameter gradients.

Part 2 - Mathematical form

$$M_{\text{peak}} = \max_t M_{\text{allocated}}(t), \qquad \text{bytes}(X) = \#X \cdot \text{bytes per element}.$$

Part 3 - Hints and Helper

Reset peak statistics before each trial and release references after it. Catch only CUDA out-of-memory errors, record the failed configuration, and avoid assuming reserved memory is a leak.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def run_trial(batch_size, shape, weight, training):
    torch.cuda.reset_peak_memory_stats()
    x = torch.randn((batch_size, *shape), device="cuda")
    x.requires_grad_(training)
    output = F.conv2d(x, weight, padding=...)
    if training:
        ...
    return {
        "batch_size": batch_size,
        "peak_allocated": torch.cuda.max_memory_allocated(),
        "peak_reserved": torch.cuda.max_memory_reserved(),
    }

def E16(batch_sizes, shape, weight, training):
    results = []
    for batch_size in batch_sizes:
        try:
            results.append(run_trial(...))
        except torch.cuda.OutOfMemoryError:
            ...
            break
    return results
```

14.2.18 Exercise 17 - Debug Device Mismatch and Non-Contiguous Tensor Issues in Convolution Pipelines

Part 1 - Why this matters

A convolution pipeline can fail when tensors are divided across CPU and GPU or when layout-changing operations produce unexpected strides. Non-contiguous tensors are valid for many operations, but some views or downstream kernels require a compatible memory layout.

Construct failing cases deliberately, inspect device, shape, stride, and contiguity, then repair the root cause without unnecessary copies. Distinguish a semantic layout error from a storage-contiguity problem because calling `contiguous` cannot correct swapped dimensions.

Part 2 - Mathematical form

stride(X) describes storage steps, shape(X) describes logical dimensions.

Part 3 - Hints and Helper

Create one diagnostic function that reports all tensor metadata before execution. Move related tensors through one chosen device and use `reshape` or `contiguous` only where justified.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def describe(name, tensor):
    return {
        "name": name,
        "shape": tuple(tensor.shape),
        "device": str(tensor.device),
        "stride": tensor.stride(),
        "contiguous": tensor.is_contiguous(),
    }

def safe_conv2d(x, weight, bias=None):
    target_device = ...
    x = ...
    weight = ...
    bias = ...
    if ...:
        x = ...
    return F.conv2d(x, weight, bias, padding=...)

def E17(failing_cases):
    return [...]
```

14.2.19 Exercise 18 - Validate Determinism Settings and Observe When CUDA Convolution Becomes Non-Deterministic

Part 1 - Why this matters

Repeated CNN runs can differ because of random initialization, sampling, stochastic layers, asynchronous execution, or nondeterministic CUDA algorithms. Setting one seed controls only part of this system.

Run identical convolution experiments under controlled and uncontrolled settings, compare outputs and gradients bitwise and numerically, and record the software and hardware environment. Explain the performance cost and the limits of deterministic guarantees.

Part 2 - Mathematical form

$$\text{bitwise deterministic: } Y^{(1)} = Y^{(2)}, \qquad \text{numerically stable: } \|Y^{(1)} - Y^{(2)}\| \le \tau.$$

Part 3 - Hints and Helper

Use `torch.use_deterministic_algorithms`, fixed seeds, and documented backend settings. Test repeated execution in one process separately from independent process runs.

Part 4 - Python code skeleton

```
import os
import random
import torch

def configure(seed, deterministic):
    random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.use_deterministic_algorithms(deterministic)
    ...

def one_run(seed, deterministic, device):
    configure(seed, deterministic)
    x = torch.randn(..., device=device, requires_grad=True)
    layer = torch.nn.Conv2d(...).to(device)
    output = layer(x)
    output.sum().backward()
    return output.detach().cpu(), x.grad.detach().cpu()

def E18(repeats, seed, device):
    runs = [one_run(seed, ..., device) for _ in range(repeats)]
    return {...}
```

14.2.20 Exercise 19 - Write a Conceptual Report Explaining Why Convolution Is the Core Inductive Bias for Vision

Part 1 - Why this matters

An inductive bias is a structural preference that guides learning toward some solutions more easily than others. Convolution favors local connectivity and shared detectors, reflecting the observation that nearby pixels interact strongly and visual patterns can recur across positions.

The final report must connect locality, weight sharing, translation equivariance, hierarchical composition, parameter efficiency, and computational behavior. It must also identify limitations, including boundary effects, loss of exact invariance, restricted global context, and cases where another architecture may be more appropriate.

Part 2 - Mathematical form

$$f(T_\delta X) = T_\delta f(X) \text{ for ideal translation equivariance}, \qquad \#\theta_{\text{conv}} \ll \#\theta_{\text{dense}}.$$

Part 3 - Hints and Helper

Support every major claim with one earlier experiment. Separate equivariance from invariance, and describe when padding, stride, pooling, or finite image boundaries weaken the ideal property.

Part 4 - Python code skeleton

```
from pathlib import Path

REQUIRED_SECTIONS = [
    "Locality",
    "Weight Sharing",
    "Translation Equivariance",
    "Hierarchical Features",
    "Efficiency",
    "Limitations",
    "Experimental Evidence",
]

def validate_report(report):
    missing = ...
    evidence_count = ...
    return {
        "missing_sections": missing,
        "evidence_count": evidence_count,
        "ready": ...,
    }

def E19(experiment_summaries, output_path):
    report = ...
    checks = validate_report(report)
    Path(output_path).write_text(..., encoding="utf-8")
    return checks
```

14.3 CNN01 - Feature Maps, Filters, and Receptive Fields

Models and Training

14.3.1 Exercise 00 - Compute Feature Map Dimensions for Multiple Conv Layers Using Manual Step-by-Step Formula

Part 1 - Why this matters

A CNN changes spatial size layer by layer, and one incorrect prediction can break a classifier head, skip connection, or memory estimate. Shape reasoning must therefore follow the complete stack rather than examining each layer only after execution.

Calculate height, width, and channel count after every convolution. Record the values substituted into the formula, then verify the entire trace against PyTorch using both square and non-square configurations.

Part 2 - Mathematical form

$$H_l = \left\lfloor \frac{H_{l-1} + 2P_l - D_l(K_l - 1) - 1}{S_l} + 1 \right\rfloor, \qquad C_l = C_{\text{out},l}.$$

Part 3 - Hints and Helper

Normalize scalar settings into pairs and reject a layer when either predicted spatial dimension is non-positive.

Part 4 - Python code skeleton

```
import torch

def output_size(size, kernel, stride, padding, dilation):
    return ...

def E00(input_shape, layer_specs):
    n, channels, height, width = input_shape
    trace = []
    for index, spec in enumerate(layer_specs):
        height = output_size(height, ...)
        width = output_size(width, ...)
        channels = ...
        trace.append({"layer": index, "shape": (...)})
    x = torch.randn(input_shape)
    model = torch.nn.Sequential(...)
    actual = ...
    return trace, actual
```

14.3.2 Exercise 01 - Visualize How Receptive Field Grows with Depth Using a Simple Layer Stack

Part 1 - Why this matters

A deep unit can depend on a wide input region even when every kernel is small. Receptive-field growth explains how local detectors become capable of representing larger structures as information passes through several layers.

Track receptive-field size and the spacing between neighboring receptive centers through a controlled stack. Visualize the input locations that influence one selected output and compare the calculation with an empirical gradient mask.

Part 2 - Mathematical form

$$R_l = R_{l-1} + (K_l - 1)J_{l-1}, \qquad J_l = J_{l-1}S_l, \qquad R_0 = J_0 = 1.$$

Part 3 - Hints and Helper

Calculate the jump before updating it. Backpropagate from one output scalar and mark input pixels with non-zero gradients.

Part 4 - Python code skeleton

```
import torch

def receptive_field_trace(layer_specs):
    receptive_field, jump = 1, 1
    trace = []
    for spec in layer_specs:
        receptive_field = ...
        jump = ...
        trace.append((receptive_field, jump))
    return trace

def empirical_mask(model, input_shape, output_index):
    x = torch.randn(input_shape, requires_grad=True)
    y = model(x)
    y[output_index].backward()
    return ...

def E01(layer_specs, model):
    return receptive_field_trace(layer_specs), empirical_mask(...)
```

14.3.3 Exercise 02 - Show How Increasing Channels Raises Representation Capacity Without Increasing Resolution

Part 1 - Why this matters

Spatial resolution describes where responses occur, while channel count describes how many learned response types are stored at each location. Increasing channels can represent more patterns without changing feature-map height or width.

Compare layers with identical spatial settings and different output-channel counts. Measure output shape, parameter count, activation memory, and diversity between channel responses instead of assuming that more channels automatically provide useful capacity.

Part 2 - Mathematical form

$$Y \in \mathbb{R}^{N \times C_{\text{out}} \times H' \times W'}, \qquad \#\theta = C_{\text{out}}(C_{\text{in}} K_h K_w + 1).$$

Part 3 - Hints and Helper

Keep kernels, stride, padding, input, and seed controlled. Use channel correlation or rank as one imperfect indicator of redundant responses.

Part 4 - Python code skeleton

```
import torch

def channel_summary(output, layer):
    flattened = output.flatten(start_dim=2)
    correlation = ...
    return {
        "shape": tuple(output.shape),
        "parameters": ...,
        "activation_bytes": ...,
        "mean_channel_correlation": ...,
    }

def E02(x, channel_options):
    results = {}
    for out_channels in channel_options:
        torch.manual_seed(...)
        layer = torch.nn.Conv2d(...)
        results[out_channels] = channel_summary(layer(x), layer)
    return results
```

14.3.4 Exercise 03 - Implement Multi-Channel Convolution and Explain the Channel Summation Mechanism Clearly

Part 1 - Why this matters

Each output channel combines every input channel with a separate spatial kernel and then sums those contributions. Omitting that summation or confusing channel order changes the mathematical operation even when spatial indexing is correct.

Implement batch and multi-channel cross-correlation with explicit loops. Preserve each input-channel contribution temporarily, verify that their sum plus bias creates one output channel, and compare the result with PyTorch.

Part 2 - Mathematical form

$$Y_{n,o,i,j} = b_o + \sum_{c=1}^{C_{\text{in}}} \sum_{u,v} X_{n,c,i+u,j+v} K_{o,c,u,v}.$$

Part 3 - Hints and Helper

Test with channels containing visibly different patterns and weights. A one-channel test cannot verify the summation mechanism.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def manual_multi_channel(x, weight, bias=None):
    output = torch.empty(...)
    contributions = torch.empty(...)
    for n in range(...):
        for oc in range(...):
            for i in range(...):
                for j in range(...):
                    for ic in range(...):
                        contributions[...] = ...
                    output[n, oc, i, j] = ...
    return output, contributions

def E03(x, weight, bias):
    manual, parts = manual_multi_channel(x, weight, bias)
    reference = F.conv2d(x, weight, bias)
    return manual, parts, torch.allclose(manual, reference)
```

14.3.5 Exercise 04 - Verify Kernel Tensor Shape (out_channels,in_channels,kH,kW) and Debug Wrong Ordering Bugs

Part 1 - Why this matters

PyTorch stores convolution weights by output channel, input channel, kernel height, and kernel width. A weight tensor built in another ordering may fail immediately or silently attach the wrong filter to each channel after an incorrect reshape.

Create a validator that checks the expected dimensions and interprets every axis. Demonstrate why permutation is required when converting from a documented external layout and why reshaping alone cannot exchange semantic axes.

Part 2 - Mathematical form

$$K \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times K_h \times K_w}.$$

Part 3 - Hints and Helper

Fill each logical channel pair with a distinct constant so an ordering mistake remains visible in the output.

Part 4 - Python code skeleton

```
import torch

def validate_kernel(weight, in_channels, out_channels, kernel_hw):
    expected = (...)
    if tuple(weight.shape) != expected:
        raise ValueError(...)
    return True

def convert_kernel(weight, source_layout):
    if source_layout == "HWIO":
        return weight.permute(...).contiguous()
    if source_layout == "OIHW":
        return ...
    raise ValueError(...)

def E04(weight_cases):
    return [...]
```

14.3.6 Exercise 05 - Compare Edge-Detector Kernels on Synthetic Pattern Images and Observe Their Responses

Part 1 - Why this matters

Hand-designed edge filters provide an interpretable example of feature-map behavior. Their positive and negative lobes respond to intensity changes with particular orientations, while flat regions ideally produce responses near zero.

Apply several edge detectors to vertical, horizontal, diagonal, constant, and noisy images. Compare signed maps, magnitude maps, and peak locations so orientation selectivity and noise sensitivity are both visible.

Part 2 - Mathematical form

$$G_x = X \star K_x, \qquad G_y = X \star K_y, \qquad M = \sqrt{G_x^2 + G_y^2}.$$

Part 3 - Hints and Helper

Normalize image scale, keep padding consistent, and do not discard response sign before explaining what it represents.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def edge_kernels(dtype, device):
    sobel_x = torch.tensor(..., dtype=dtype, device=device)
    sobel_y = torch.tensor(..., dtype=dtype, device=device)
```

```
    prewitt_x = torch.tensor(..., dtype=dtype, device=device)
    return torch.stack([...]).unsqueeze(1)

def E05(images):
    kernels = edge_kernels(images.dtype, images.device)
    responses = F.conv2d(images, kernels, padding=...)
    magnitude = ...
    peaks = ...
    return responses, magnitude, peaks
```

14.3.7 Exercise 06 - Create Custom Kernels to Detect Horizontal, Vertical, and Diagonal Structures

Part 1 - Why this matters

Designing a kernel turns a visual hypothesis into weights. The location and signs of the weights determine which local arrangement is reinforced, which is suppressed, and how the response changes when contrast is reversed.

Construct custom kernels for three orientations, test them on controlled patterns, and revise them using measured responses. Require each detector to respond more strongly to its target orientation than to non-target patterns.

Part 2 - Mathematical form

$$s(K, X) = \max_{i,j} |(X \star K)_{i,j}|, \qquad s(K, X_{\text{target}}) > s(K, X_{\text{other}}).$$

Part 3 - Hints and Helper

Use zero-sum kernels for contrast transitions and normalize their norms before comparing selectivity.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def design_kernels():
    horizontal = torch.tensor(..., dtype=torch.float32)
    vertical = torch.tensor(..., dtype=torch.float32)
    diagonal = torch.tensor(..., dtype=torch.float32)
    kernels = torch.stack([...]).unsqueeze(1)
    return kernels / ...

def selectivity_matrix(images, kernels):
    responses = F.conv2d(images, kernels, padding=...)
    return responses.abs().amax(dim=(-2, -1))
```

```
def E06(images):
    scores = selectivity_matrix(images, design_kernels())
    return scores, ...
```

14.3.8 Exercise 07 - Demonstrate Translation Equivariance by Shifting Input and Observing Shifted Feature Maps

Part 1 - Why this matters

An ideal convolution is translation equivariant: shifting the input shifts the feature map by the corresponding amount. This property allows one shared detector to recognize a pattern at several locations.

Shift a synthetic image without wrapping content across boundaries, apply the same convolution before and after the shift, and align the feature maps for comparison. Evaluate only locations whose receptive fields remain valid in both cases.

Part 2 - Mathematical form

$$f(T_\delta X) = T_\delta f(X).$$

Part 3 - Hints and Helper

Use an interior comparison mask so pixels introduced or removed at the boundary do not invalidate the intended test.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def translate_without_wrap(x, dy, dx, fill=0.0):
    shifted = torch.full_like(x, fill)
    source_slices = ...
    target_slices = ...
    shifted[target_slices] = x[source_slices]
    return shifted

def E07(x, weight, shift):
    base = F.conv2d(x, weight, padding=...)
    moved_input = translate_without_wrap(x, *shift)
    moved_output = F.conv2d(moved_input, weight, padding=...)
    expected = translate_without_wrap(base, *shift)
    mask = ...
    return (moved_output - expected)[mask], ...
```

14.3.9 Exercise 08 - Demonstrate When Translation Equivariance Breaks Due to Padding or Stride Choices

Part 1 - Why this matters

Finite images, padding rules, and subsampling weaken the ideal translation behavior of convolution. A one-pixel input shift may interact with a different sampling phase when stride is larger than one, while borders receive artificial context from padding.

Repeat the equivariance test across valid, zero, reflect, and strided configurations. Separate boundary error from interior sampling error and explain which transformations remain approximately equivariant.

Part 2 - Mathematical form

$$f_S(T_\delta X) = T_{\delta/S} f_S(X) \text{ only when the shift is compatible with stride } S.$$

Part 3 - Hints and Helper

Test shifts that are and are not multiples of the stride. Report interior and border errors separately.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def equivariance_error(x, weight, shift, stride, pad_mode):
    padded = ...
    base = F.conv2d(padded, weight, stride=stride)
    shifted_x = ...
    shifted_output = F.conv2d(...)
    expected = ...
    interior_mask = ...
    return {
        "interior_error": ...,
        "border_error": ...,
        "shape": tuple(base.shape),
    }

def E08(x, weight, configurations):
    return [equivariance_error(x, weight, **c) for c in configurations]
```

14.3.10 Exercise 09 - Analyze How Stride Impacts Aliasing and Feature Map Quality Using Visual Examples

Part 1 - Why this matters

Strided convolution samples a filtered signal on a coarser grid. If the input contains frequencies above what that grid can represent, distinct patterns may produce similar or distorted feature maps through aliasing.

Create checkerboards and sinusoidal patterns at several frequencies, then compare direct strided convolution with a low-pass-filtered alternative. Visualize the maps and quantify how strongly results change under a one-pixel phase shift.

Part 2 - Mathematical form

$$f_{\text{input}} > \frac{f_{\text{sample}}}{2} \;\Rightarrow\; \text{aliasing risk.}$$

Part 3 - Hints and Helper

Keep downsampling factor fixed and vary pattern frequency. Blur before subsampling rather than after information has already been discarded.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def low_pass_then_stride(x, blur_kernel, stride):
    smoothed = F.conv2d(x, blur_kernel, padding=...)
    return smoothed[..., ::stride, ::stride]

def phase_sensitivity(a, b):
    return ...

def E09(patterns, conv_kernel, blur_kernel, stride):
    results = []
    for pattern in patterns:
        direct = F.conv2d(pattern, conv_kernel, stride=stride)
        antialiased = low_pass_then_stride(pattern, blur_kernel, stride)
        shifted = ...
        results.append({...})
    return results
```

14.3.11 Exercise 10 - Run Feature Map Extraction on CUDA and Verify Output Shape and Device Placement

Part 1 - Why this matters

Feature extraction on CUDA succeeds only when the model, input, and relevant state share the intended device. Correct device placement must be checked together with feature-map shapes because moving computation does not repair architectural mistakes.

Capture outputs from selected convolutional layers on CUDA, compare their shapes with manual predictions, and verify that hooks do not retain computational graphs unnecessarily during analysis.

Part 2 - Mathematical form

$$X \in \text{CUDA},\ \theta \in \text{CUDA} \implies f_\theta(X) \in \text{CUDA}.$$

Part 3 - Hints and Helper

Register hooks before the forward pass, detach captured outputs, and remove every hook afterward.

Part 4 - Python code skeleton

```
import torch

def extract_features(model, x, layer_names):
    captured, handles = {}, []
    for name, module in model.named_modules():
        if name in layer_names:
            handles.append(module.register_forward_hook(...))
    try:
        with torch.inference_mode():
            output = model(x)
    finally:
        for handle in handles:
            handle.remove()
    return output, captured

def E10(model, x, expected_shapes):
    if not torch.cuda.is_available():
        return {"available": False}
    model, x = ...
    output, maps = extract_features(model, x, ...)
    return {"available": True, "checks": ...}
```

14.3.12 Exercise 11 - Benchmark Different Kernel Sizes and Strides on GPU to Measure Real Performance Differences

Part 1 - Why this matters

Kernel size and stride change mathematical work, output size, memory traffic, and the algorithms selected by the CUDA backend. Parameter count or FLOPs alone cannot predict the fastest configuration on a specific GPU.

Benchmark a controlled grid of kernel and stride choices with warm-up and synchronization. Report latency, throughput, output shape, estimated operations, and peak memory while keeping batch and channel dimensions fixed.

Part 2 - Mathematical form

$$\text{MACs} = NH'W'C_{\text{out}}C_{\text{in}}K_hK_w.$$

Part 3 - Hints and Helper

Allocate each layer before timing, reuse tensors, and summarize repeated measurements rather than one call.

Part 4 - Python code skeleton

```
import time
import torch

def benchmark_layer(layer, x, warmup, repeats):
    for _ in range(warmup):
        layer(x)
    times = []
    for _ in range(repeats):
        torch.cuda.synchronize()
        start = time.perf_counter()
        output = layer(x)
        torch.cuda.synchronize()
        times.append(time.perf_counter() - start)
    return output, ...

def E11(x, configurations):
    results = []
    for config in configurations:
        layer = torch.nn.Conv2d(...).cuda()
        output, timing = benchmark_layer(layer, x, ..., ...)
        results.append({...})
    return results
```

14.3.13 Exercise 12 - Investigate Numerical Differences Between CPU and GPU Feature Maps Under Various Seeds

Part 1 - Why this matters

CPU and GPU convolution may use different floating-point instruction orders or algorithms, producing small numerical differences without indicating a logical error. Seeds control generated inputs and parameters but do not make all hardware arithmetic identical.

Repeat matched CPU and GPU feature extraction under several seeds. Record absolute and relative errors per layer, determine whether discrepancies grow with depth, and distinguish reproducibility within one device from parity across devices.

Part 2 - Mathematical form

$$\Delta_l = \max_i |F_{l,i}^{\mathrm{CPU}} - F_{l,i}^{\mathrm{GPU}}|.$$

Part 3 - Hints and Helper

Clone one initialized CPU state to GPU for each seed; do not initialize the two models independently.

Part 4 - Python code skeleton

```
import copy
import torch

def layerwise_features(model, x):
    features = {}
    handles = [...]
    with torch.inference_mode():
        model(x)
    for handle in handles:
        handle.remove()
    return features

def compare_seed(model_factory, x_factory, seed):
    torch.manual_seed(seed)
    cpu_model = model_factory().eval()
    gpu_model = copy.deepcopy(cpu_model).cuda().eval()
    x_cpu = x_factory()
    cpu_maps = layerwise_features(cpu_model, x_cpu)
    gpu_maps = layerwise_features(gpu_model, x_cpu.cuda())
    return {...}

def E12(seeds, model_factory, x_factory):
    return [compare_seed(model_factory, x_factory, s) for s in seeds]
```

14.3.14 Exercise 13 - Implement Depthwise Convolution Manually and Explain How It Differs from Standard Convolution

Part 1 - Why this matters

Depthwise convolution applies separate spatial filters to individual input channels instead of summing across all channels for every output. It extracts channel-specific spatial patterns but does not perform general channel mixing by itself.

Implement the operation manually for a depth multiplier, compare it with grouped PyTorch convolution, and contrast its parameter count and connectivity with a standard convolution having the same output channels.

Part 2 - Mathematical form

$$C_{\text{out}} = mC_{\text{in}}, \qquad \#\theta_{\text{depthwise}} = mC_{\text{in}}K_hK_w.$$

Part 3 - Hints and Helper

Map each output channel to its source input channel using integer division by the depth multiplier.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def manual_depthwise(x, weight, multiplier):
    output = torch.empty(...)
    for n in range(...):
        for oc in range(...):
            source_channel = ...
            for i in range(...):
                for j in range(...):
                    output[n, oc, i, j] = ...
    return output

def E13(x, weight, multiplier):
    manual = manual_depthwise(x, weight, multiplier)
    reference = F.conv2d(x, weight, groups=x.shape[1])
    return manual, reference, {
        "depthwise_parameters": ...,
        "standard_parameters": ...,
    }
```

14.3.15 Exercise 14 - Implement Grouped Convolution and Prove It Reduces Parameters While Preserving Some Representation Power

Part 1 - Why this matters

Grouped convolution partitions input and output channels into independent channel groups. It retains full spatial filtering within each group while removing connections between groups, reducing parameters and computation at the cost of restricted channel interaction.

Implement grouped convolution by slicing channels and concatenating independent group outputs. Validate it against PyTorch, calculate connectivity, and construct an input perturbation that proves which outputs cannot be affected across group boundaries.

Part 2 - Mathematical form

$$\#\theta_{\text{grouped}} = \frac{C_{\text{out}} C_{\text{in}} K_h K_w}{G}.$$

Part 3 - Hints and Helper

Require both channel counts to be divisible by the group count and test forbidden cross-group influence explicitly.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def manual_grouped_conv(x, weight, groups):
    in_per_group = ...
    out_per_group = ...
    outputs = []
    for group in range(groups):
        x_group = ...
        w_group = ...
        outputs.append(F.conv2d(x_group, w_group))
    return torch.cat(outputs, dim=1)

def E14(x, weight, groups):
    manual = manual_grouped_conv(x, weight, groups)
    reference = F.conv2d(x, weight, groups=groups)
    influence_test = ...
    return manual, reference, influence_test
```

14.3.16 Exercise 15 - Compare Standard, Depthwise, and Grouped Convolutions in Parameters, Speed, and Feature Quality

Part 1 - Why this matters

Standard, depthwise, and grouped convolutions make different connectivity assumptions. Lower parameter count can improve efficiency, but runtime depends on hardware kernels and restricted connectivity may reduce feature quality for a particular task.

Build comparable blocks with matched input and output shapes, benchmark them, and evaluate their feature maps on a controlled prediction task. Separate depthwise-only filtering from depthwise plus pointwise channel mixing.

Part 2 - Mathematical form

$$\text{compression ratio} = \frac{\#\theta_{\text{variant}}}{\#\theta_{\text{standard}}}.$$

Part 3 - Hints and Helper

Control initialization, training budget, and evaluation data. Compare parameters, latency, peak memory, and task metric in one table.

Part 4 - Python code skeleton

```
import time
import torch

def build_variants(in_channels, out_channels, kernel_size, groups):
    standard = torch.nn.Conv2d(...)
    depthwise_separable = torch.nn.Sequential(...)
    grouped = torch.nn.Conv2d(...)
    return {...}

def profile_block(block, x):
    parameters = sum(p.numel() for p in block.parameters())
    timing = ...
    output = ...
    quality = ...
    return {
        "parameters": parameters,
        "latency": timing,
        "quality": quality,
        "shape": tuple(output.shape),
    }

def E15(x, configuration):
    return {name: profile_block(block, x) for name, block in ...}
```

14.3.17 Exercise 16 - Detect and Fix Shape Errors When Switching Between RGB and Grayscale Inputs

Part 1 - Why this matters

RGB images normally provide three channels while grayscale images provide one. A convolution configured for one format cannot consume the other without an explicit conversion or architectural change.

Design a preprocessing contract that validates layout and channel count, then compare grayscale-to-RGB replication, RGB-to-luminance conversion, and first-layer weight adaptation. Explain which transformations preserve compatibility and which change learned behavior.

Part 2 - Mathematical form

$$X_{\text{gray}} = \alpha R + \beta G + \gamma B, \qquad C_{\text{gray}} = 1,\ C_{\text{RGB}} = 3.$$

Part 3 - Hints and Helper

Never repair the error with an arbitrary reshape. Channel conversion must describe how values are transformed.

Part 4 - Python code skeleton

```
import torch

def convert_channels(x, target_channels):
    if x.ndim != 4:
        raise ValueError(...)
    if x.shape[1] == target_channels:
        return x
    if x.shape[1] == 1 and target_channels == 3:
        return ...
    if x.shape[1] == 3 and target_channels == 1:
        coefficients = x.new_tensor(...).view(1, 3, 1, 1)
        return ...
    raise ValueError(...)

def adapt_first_layer(weight, target_channels):
    return ...

def E16(model, test_inputs):
    return [...]
```

14.3.18 Exercise 17 - Stress Test a Very Deep Conv Stack and Monitor Activation Magnitudes for Stability Issues

Part 1 - Why this matters

Repeated convolution and activation can cause signals to shrink, grow, saturate, or become non-finite with depth. The final output alone cannot reveal where the instability began.

Build a configurable deep stack and capture layer-wise mean, standard deviation, norm, zero fraction, and finite-value status. Compare initialization and activation choices under controlled inputs and stop safely when failure is detected.

Part 2 - Mathematical form

$$\mu_l = \mathbb{E}[A_l], \qquad \sigma_l^2 = \mathrm{Var}(A_l), \qquad r_l = \frac{\|A_l\|_2}{\|A_{l-1}\|_2}.$$

Part 3 - Hints and Helper

Detach statistics immediately instead of storing full graphs. Use fixed seeds and identical depth when comparing initialization strategies.

Part 4 - Python code skeleton

```
import torch

def activation_statistics(tensor):
    return {
        "mean": ...,
        "std": ...,
        "norm": ...,
        "zero_fraction": ...,
        "finite": ...,
    }

def run_with_monitoring(model, x):
    records, handles = [], []
    for name, module in model.named_modules():
        if isinstance(module, ...):
            handles.append(module.register_forward_hook(...))
    try:
        output = model(x)
    finally:
        for handle in handles:
            handle.remove()
    return output, records

def E17(depths, initializers, input_tensor):
    return {...}
```

14.3.19 Exercise 18 - Build a Receptive Field Calculator and Validate It Against Empirical Behavior

Part 1 - Why this matters

A reusable receptive-field calculator prevents vague claims about how much context a unit can see. It must track receptive size, effective jump, and center offset through convolutions and pooling layers.

Implement the calculator from layer specifications, then validate selected output units by differentiating them with respect to an input. Compare predicted receptive bounds with the empirical non-zero gradient support.

Part 2 - Mathematical form

$$R_l = R_{l-1} + (K_l - 1)D_l J_{l-1}, \quad J_l = J_{l-1} S_l, \quad O_l = O_{l-1} + \left(\frac{K_l - 1}{2} D_l - P_l\right) J_{l-1}.$$

Part 3 - Hints and Helper

Track height and width independently. Use simple non-zero weights and avoid activations that can accidentally block the gradient during validation.

Part 4 - Python code skeleton

```
import torch

def receptive_field_calculator(layer_specs):
    state = {
        "rf": (1, 1),
        "jump": (1, 1),
        "offset": (0.5, 0.5),
    }
    trace = []
    for spec in layer_specs:
        state = ...
        trace.append(state.copy())
    return trace

def empirical_support(model, input_shape, output_coordinate):
    x = torch.ones(input_shape, requires_grad=True)
    y = model(x)
    y[output_coordinate].backward()
    coordinates = torch.nonzero(...)
    return ...

def E18(layer_specs, model, cases):
    predicted = receptive_field_calculator(layer_specs)
    empirical = [...]
    return predicted, empirical, ...
```

14.3.20 Exercise 19 - Produce a Receptive Field and Feature Map Interpretation Guide for Future Debugging

Part 1 - Why this matters

Feature maps should not be interpreted only by their shapes, and receptive fields should not be confused with proof that every location inside them contributes equally. A debugging guide must connect theoretical coverage with observed activations and gradients.

Create a structured guide covering shape traces, receptive-field calculations, channel semantics, visualization, numerical statistics, equivariance tests, grouped connectivity, device parity, and common failure patterns. Every recommendation should point to evidence produced in earlier exercises.

Part 2 - Mathematical form

theoretical receptive field $\supseteq$ effective influential region measured for a particular input.

Part 3 - Hints and Helper

Include a symptom, diagnostic measurement, likely causes, and verification test for every failure pattern.

Part 4 - Python code skeleton

```
from pathlib import Path

REQUIRED_TOPICS = [
    "Shape Trace",
    "Receptive Field",
    "Channel Interpretation",
    "Equivariance",
    "Numerical Stability",
    "Connectivity",
    "Debugging Checklist",
]

def validate_guide(text, evidence):
    missing = [topic for topic in REQUIRED_TOPICS if topic not in text]
    linked_evidence = ...
    return {
        "missing": missing,
        "linked_evidence": linked_evidence,
        "ready": ...,
    }

def E19(experiment_records, output_path):
    guide = ...
    checks = validate_guide(guide, experiment_records)
    Path(output_path).write_text(guide, encoding="utf-8")
    return checks
```

14.4 CNN02 - Pooling, Downsampling, and Invariance

Models and Training

14.4.1 Exercise 00 - Implement Max Pooling Manually and Verify It Matches torch.nn.functional.max_pool2d

Part 1 - Why this matters

Max pooling divides each feature map into local windows and retains the largest value from each window. A manual version makes window placement, stride, padding, overlap, and boundary behavior explicit before an optimized implementation is trusted.

Implement batched channel-first max pooling with explicit loops, calculate its output shape, and compare values with PyTorch. Include negative inputs so incorrect zero padding cannot silently replace the true maximum near a boundary.

Part 2 - Mathematical form

$$Y_{n,c,i,j} = \max_{0 \le u < K_h, 0 \le v < K_w} X_{n,c,iS_h+u,jS_w+v}.$$

Part 3 - Hints and Helper

Begin with no padding and equal kernel and stride, then add overlapping windows. Use negative infinity when mathematical max padding is required.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def manual_max_pool2d(x, kernel_size, stride):
    out_h, out_w = ...
    output = torch.empty(..., dtype=x.dtype, device=x.device)
    for n in range(...):
        for c in range(...):
            for i in range(out_h):
                for j in range(out_w):
                    window = ...
                    output[n, c, i, j] = ...
    return output

def E00(x):
    manual = manual_max_pool2d(x, ..., ...)
    reference = F.max_pool2d(x, ..., ...)
    return manual, reference, torch.allclose(manual, reference)
```

14.4.2 Exercise 01 - Implement Average Pooling and Compare Its Behavior to Max Pooling on the Same Input

Part 1 - Why this matters

Average pooling summarizes the complete window, whereas max pooling preserves only its strongest activation. Their outputs therefore respond differently to noise, sparse peaks, uniform regions, and distributed evidence.

Implement average pooling manually and compare both methods on synthetic feature maps with known structures. Measure changes in mean, variance, peak preservation, and sensitivity to one extreme value.

Part 2 - Mathematical form

$$Y_{i,j}^{\text{avg}} = \frac{1}{K_h K_w} \sum_{u,v} X_{iS_h+u,jS_w+v}.$$

Part 3 - Hints and Helper

Use identical windows and stride for both methods. Include constant, sparse, noisy, and smoothly varying inputs.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def manual_average_pool2d(x, kernel_size, stride):
    output = torch.empty(...)
    for i in range(...):
        for j in range(...):
            output[..., i, j] = ...
    return output

def summarize(tensor):
    return {"mean": ..., "std": ..., "maximum": ...}

def E01(x):
    average = manual_average_pool2d(x, ..., ...)
    maximum = F.max_pool2d(x, ..., ...)
    return average, maximum, summarize(average), summarize(maximum)
```

14.4.3 Exercise 02 - Explain and Demonstrate Why Pooling Introduces Local Translation Invariance

Part 1 - Why this matters

A small translation within one pooling window may leave the pooled value unchanged, especially when the same maximum remains inside that window. This creates limited local invariance, not complete invariance to arbitrary image shifts.

Move a distinctive activation through several positions inside and across pooling windows. Compare pooled outputs and identify exactly which shifts are ignored and which change the sampled region or output coordinate.

Part 2 - Mathematical form

$$p(T_\delta X) \approx p(X) \text{ for some small local shifts, but not for every } \delta.$$

Part 3 - Hints and Helper

Test within-window and cross-window shifts separately. Avoid describing convolutional equivariance and pooling invariance as the same property.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def place_peak(shape, row, column, value):
    x = torch.zeros(shape)
    x[..., row, column] = value
    return x

def E02(positions, pool_size):
    outputs = []
    for row, column in positions:
        x = place_peak(...)
        pooled = F.max_pool2d(x, pool_size, pool_size)
        outputs.append({"position": (row, column), "output": pooled})
    invariance_matrix = ...
    return outputs, invariance_matrix
```

14.4.4 Exercise 03 - Compare Pooling and Strided Convolution for Downsampling and Measure Quality Differences

Part 1 - Why this matters

Pooling uses a fixed summary rule, while strided convolution learns a spatial filter during downsampling. The learned method may preserve task-relevant information, but it introduces parameters and can alias if sampling occurs without suitable filtering.

Construct max-pooling, average-pooling, and strided-convolution branches with matched output shapes. Compare reconstruction error, downstream task quality, parameters, feature statistics, and sensitivity to shifts.

Part 2 - Mathematical form

$$Y_{\text{pool}} = p(X), \qquad Y_{\text{stride}} = X \star_S K.$$

Part 3 - Hints and Helper

Match output channels and spatial dimensions before comparing. Train the convolution branch only when the experiment explicitly evaluates learned downsampling.

Part 4 - Python code skeleton

```
import torch

def build_downsamplers(channels):
    return {
        "max": torch.nn.MaxPool2d(...),
        "average": torch.nn.AvgPool2d(...),
        "strided_conv": torch.nn.Conv2d(...),
    }

def evaluate(name, module, x, target):
    reduced = module(x)
    reconstructed = ...
    return {
        "name": name,
        "shape": tuple(reduced.shape),
        "parameters": ...,
        "quality": ...,
    }

def E03(x, target):
    modules = build_downsamplers(x.shape[1])
    results = []
    for name, module in modules.items():
        results.append(evaluate(name, module, x, target))
    return results
```

14.4.5 Exercise 04 - Analyze Pooling Window Size, Information Loss, and Classification Performance

Part 1 - Why this matters

A larger pooling window compresses a wider region into one value, reducing memory and increasing local invariance while discarding more spatial detail. The effect depends on whether the task relies on large structures or small localized evidence.

Sweep window sizes under a controlled classifier and dataset. Record spatial compression, retained signal, accuracy, latency, and failure cases, keeping the remaining architecture and training budget fixed.

Part 2 - Mathematical form

$$\rho = \frac{H'W'}{HW}, \qquad \text{information loss generally increases as } \rho \downarrow .$$

Part 3 - Hints and Helper

Include a dataset where small objects determine the label and another dominated by broad patterns. Do not infer information quality from output size alone.

Part 4 - Python code skeleton

```
import torch

def run_experiment(window_size, train_loader, val_loader, seed):
    torch.manual_seed(seed)
    model = torch.nn.Sequential(...)
    history = ...
    metrics = ...
    return {
        "window": window_size,
        "compression": ...,
        "accuracy": ...,
        "latency": ...,
        "history": history,
    }

def E04(window_sizes, train_loader, val_loader):
    results = []
    for size in window_sizes:
        result = run_experiment(size, train_loader, val_loader, ...)
        results.append(result)
    return results
```

14.4.6 Exercise 05 - Visualize Aliasing from Aggressive Downsampling Without Anti-Aliasing

Part 1 - Why this matters

Downsampling removes sample locations. When a feature map contains frequencies too high for the reduced grid, different structures can collapse into distorted or misleading patterns known as aliases.

Generate checkerboards and sinusoidal inputs at several frequencies, downsample them aggressively, and compare direct sampling with low-pass filtering before reduction. Visualize phase-dependent artifacts and quantify reconstruction or shift error.

Part 2 - Mathematical form

$$f_{\max} > \frac{f_s}{2} \implies \text{aliasing can occur after sampling.}$$

Part 3 - Hints and Helper

Change the phase by one pixel while preserving frequency. Large output changes reveal sampling sensitivity.

Part 4 - Python code skeleton

```
import matplotlib.pyplot as plt
import torch
import torch.nn.functional as F

def downsample_direct(x, factor):
    return x[..., ::factor, ::factor]

def downsample_filtered(x, blur, factor):
    smooth = F.conv2d(x, blur, padding=...)
    return smooth[..., ::factor, ::factor]

def E05(patterns, blur, factor):
    records = []
    for pattern in patterns:
        direct = downsample_direct(pattern, factor)
        filtered = downsample_filtered(pattern, blur, factor)
        records.append({...})
    figure = ...
    return figure, records
```

14.4.7 Exercise 06 - Implement Global Average Pooling and Explain Its Role in Modern CNN Classifier Heads

Part 1 - Why this matters

Global average pooling replaces every spatial feature map with its mean, producing one value per channel. This creates an input-size-tolerant representation and can remove a large fully connected layer from the classifier head.

Implement it manually, verify it against PyTorch, and compare parameter count with flattening followed by a dense layer. Explain what spatial information is intentionally discarded and how channels become global evidence summaries.

Part 2 - Mathematical form

$$z_{n,c} = \frac{1}{HW} \sum_{i=1}^{H} \sum_{j=1}^{W} X_{n,c,i,j}.$$

Part 3 - Hints and Helper

Average only spatial axes and preserve batch and channel axes. Test several input resolutions with the same classifier.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F
```

```
def manual_global_average_pool(x):
    if x.ndim != 4:
        raise ValueError(...)
    return ...

def E06(x, class_count):
    manual = manual_global_average_pool(x)
    reference = F.adaptive_avg_pool2d(x, 1).flatten(1)
    global_head = torch.nn.Linear(x.shape[1], class_count)
    dense_head = torch.nn.Linear(..., class_count)
    return {
        "allclose": ...,
        "global_parameters": ...,
        "dense_parameters": ...,
    }
```

14.4.8 Exercise 07 - Demonstrate Adaptive Pooling with Variable Input Image Sizes

Part 1 - Why this matters

Adaptive pooling chooses window boundaries so different input sizes produce a requested output size. It allows a later network stage to receive a stable tensor shape without requiring every image to share one spatial resolution.

Apply adaptive max and average pooling to several non-square inputs, verify output dimensions, and inspect how uneven partitions affect which input locations contribute. Compare adaptive pooling with resizing before feature extraction.

Part 2 - Mathematical form

$$X \in \mathbb{R}^{N \times C \times H \times W} \xrightarrow{\text{adaptive pool}} Y \in \mathbb{R}^{N \times C \times H_t \times W_t}.$$

Part 3 - Hints and Helper

Use one fixed target shape and record source-to-output region boundaries for at least one irregular input size.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def region_bounds(index, input_size, output_size):
    start = ...
    end = ...
    return start, end
```

```
def E07(inputs, target_hw):
    results = []
    for x in inputs:
        average = F.adaptive_avg_pool2d(x, target_hw)
        maximum = F.adaptive_max_pool2d(x, target_hw)
        regions = ...
        results.append({"input": tuple(x.shape), "average": average,
                        "maximum": maximum, "regions": regions})
    return results
```

14.4.9 Exercise 08 - Compare Feature Map Statistics Before and After Pooling to Understand Signal Compression

Part 1 - Why this matters

Pooling changes more than shape. Max pooling often raises local response magnitude and sparsity, while averaging reduces variance and smooths peaks. These changes influence later normalization, activation, and gradient behavior.

Measure mean, variance, norm, sparsity, extrema, entropy estimate, and channel correlation before and after several pooling methods. Interpret which statistics reflect compression and which depend strongly on the input distribution.

Part 2 - Mathematical form

$$\Delta\mu = \mu(Y) - \mu(X), \qquad \Delta\sigma^2 = \mathrm{Var}(Y) - \mathrm{Var}(X).$$

Part 3 - Hints and Helper

Compute statistics per channel before aggregating. A global summary can hide collapse in only a subset of channels.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def statistics(x):
    return {
        "mean_per_channel": ...,
        "variance_per_channel": ...,
        "norm_per_channel": ...,
        "zero_fraction": ...,
        "minimum": ...,
        "maximum": ...,
    }

def E08(x):
```

```
    variants = {
        "before": x,
        "max": F.max_pool2d(x, ...),
        "average": F.avg_pool2d(x, ...),
    }
    return {name: statistics(value) for name, value in variants.items()}
```

14.4.10 Exercise 09 - Debug Output Shape Mistakes from Pooling and Stride Interactions in Complex Networks

Part 1 - Why this matters

Several reductions in one architecture can shrink feature maps faster than expected, create fractional predictions that are floored, or make a later kernel larger than its input. The visible failure may occur many layers after the original mistake.

Build a shape tracer for convolution and pooling modules and compare predicted shapes with forward-hook observations. Report the first mismatching layer with its complete configuration instead of only showing the final exception.

Part 2 - Mathematical form

$$H_l = \left\lfloor \frac{H_{l-1} + 2P_l - D_l(K_l - 1) - 1}{S_l} + 1 \right\rfloor .$$

Part 3 - Hints and Helper

Trace height and width independently and include ceil mode when it is enabled. Validate every transition before running expensive training.

Part 4 - Python code skeleton

```
import torch

def predict_transition(shape, module):
    if isinstance(module, torch.nn.Conv2d):
        return ...
    if isinstance(module, (torch.nn.MaxPool2d, torch.nn.AvgPool2d)):
        return ...
    return shape

def E09(model, input_shape):
    predicted, observed = [], []
    shape = input_shape
    handles = []
    for name, module in model.named_modules():
        shape = predict_transition(shape, module)
        predicted.append((name, shape))
```

```
        handles.append(...)
    model(torch.randn(input_shape))
    ...
    return predicted, observed, ...
```

14.4.11 Exercise 10 - Run Pooling on CUDA and Confirm Correctness and Speed
Benefits

Part 1 - Why this matters

Pooling is usually inexpensive compared with convolution, so GPU acceleration depends on tensor size and transfer overhead. Correctness and speed must be measured separately using identical inputs and synchronized timings.

Compare CPU and CUDA outputs for several pooling modes, then benchmark small and large tensors with warm-up. Keep transfer timing separate from compute timing and justify numerical tolerances.

Part 2 - Mathematical form

$$\text{speedup} = t_{\text{CPU}}/t_{\text{GPU}}, \qquad \Delta_{\max} = \max |Y_{\text{CPU}} - Y_{\text{GPU}}|.$$

Part 3 - Hints and Helper

Synchronize CUDA immediately before starting and after finishing each timed region. Reuse already transferred tensors.

Part 4 - Python code skeleton

```
import time
import torch
import torch.nn.functional as F

def benchmark(operation, x, repeats):
    times = []
    for _ in range(...):
        operation(x)
    for _ in range(repeats):
        if x.is_cuda: torch.cuda.synchronize()
        start = time.perf_counter()
        y = operation(x)
        if x.is_cuda: torch.cuda.synchronize()
        times.append(time.perf_counter() - start)
    return y, ...

def E10(x_cpu, operation):
    if not torch.cuda.is_available(): return {"available": False}
    cpu_output, cpu_time = benchmark(operation, x_cpu, ...)
```

```
    gpu_output, gpu_time = benchmark(operation, x_cpu.cuda(), ...)
    return {...}
```

14.4.12 Exercise 11 - Benchmark Pooling and Strided Convolution Runtime and GPU Memory Usage

Part 1 - Why this matters

Pooling has no trainable weights, while strided convolution performs learned multiply-accumulate operations and stores parameter gradients during training. Their real memory and runtime differences depend on shape, backend, precision, and whether backward is included.

Benchmark matched-output variants in inference and training. Report forward and backward time, allocated and peak memory, parameter count, and output size so the comparison separates computation from representational flexibility.

Part 2 - Mathematical form

$$M_{\text{peak}} = \max_t M_{\text{allocated}}(t), \qquad \#\theta_{\text{pool}} = 0.$$

Part 3 - Hints and Helper

Reset peak memory statistics for each isolated trial and clear references between variants.

Part 4 - Python code skeleton

```
import torch

def profile(module, x, training):
    torch.cuda.reset_peak_memory_stats()
    x = x.detach().requires_grad_(training)
    output = module(x)
    if training:
        output.mean().backward()
    return {
        "parameters": sum(p.numel() for p in module.parameters()),
        "peak_memory": torch.cuda.max_memory_allocated(),
        "output_shape": tuple(output.shape),
        "time": ...,
    }

def E11(x, channels):
    variants = {"max": ..., "average": ..., "strided_conv": ...}
    results = {}
    for name, module in variants.items():
        results[name] = profile(module, x, ...)
    return results
```

14.4.13 Exercise 12 - Investigate Pooling Precision Under float16 Mixed Precision Settings

Part 1 - Why this matters

Max pooling selects an existing value and is usually insensitive to accumulation precision, while average pooling sums values and divides, making it more vulnerable to rounding, underflow, or overflow in low precision.

Compare float16 outputs and gradients with float32 references across normal, tiny, large, and cancellation-heavy inputs. Test both max and average pooling and report non-finite values separately from finite numerical error.

Part 2 - Mathematical form

$$\epsilon_{\text{rel}} = \frac{|Y_{16} - Y_{32}|}{|Y_{32}| + \varepsilon}.$$

Part 3 - Hints and Helper

Run on hardware that supports float16 pooling and convert low-precision results to float32 before error calculation.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def compare_precision(x32, operation):
    x32 = x32.detach().requires_grad_(True)
    reference = operation(x32)
    reference.sum().backward()
    grad32 = x32.grad.detach().clone()
    x16 = x32.detach().half().requires_grad_(True)
    low = operation(x16)
    low.sum().backward()
    return {
        "output_error": ...,
        "gradient_error": ...,
        "finite": ...,
    }

def E12(cases):
    operations = {"max": ..., "average": ...}
    return {...}
```

14.4.14 Exercise 13 - Implement Anti-Aliased Pooling and Compare It with Standard Pooling

Part 1 - Why this matters

Anti-aliased pooling applies a low-pass filter before reducing the sampling rate. It suppresses frequencies that the coarser output grid cannot represent, trading some sharpness and computation for improved shift stability.

Construct a normalized blur kernel, filter each channel independently, and then downsample. Compare it with standard pooling on high-frequency patterns, shifted inputs, and a controlled classification task.

Part 2 - Mathematical form

$$Y = \downarrow_S (h \star X), \qquad \sum_{u,v} h_{u,v} = 1.$$

Part 3 - Hints and Helper

Apply the same blur independently to every channel by using grouped convolution. Compare phase sensitivity and accuracy, not only visual smoothness.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def blur_kernel_2d(coefficients, channels, device, dtype):
    vector = torch.tensor(coefficients, device=device, dtype=dtype)
    kernel = torch.outer(vector, vector)
    kernel = kernel / kernel.sum()
    return kernel.view(1, 1, *kernel.shape).repeat(channels, 1, 1, 1)

def anti_aliased_pool(x, kernel, stride):
    smooth = F.conv2d(x, kernel, padding=..., groups=x.shape[1])
    return smooth[..., ::stride, ::stride]

def E13(x, shifted_x):
    kernel = blur_kernel_2d(...)
    standard = F.avg_pool2d(x, ...)
    antialiased = anti_aliased_pool(x, kernel, ...)
    return standard, antialiased, ...
```

14.4.15 Exercise 14 - Implement Blur Pooling and Measure Its Effect on Robustness

Part 1 - Why this matters

Blur pooling is one practical anti-aliasing design that separates smoothing from subsampling. It aims to reduce prediction changes caused by small translations or corruptions that interact badly with a coarse sampling grid.

Insert blur pooling into a small CNN and compare it with the original downsampling path. Evaluate clean accuracy, shifted accuracy, noise and blur robustness, latency, and memory so robustness gains are weighed against their cost.

Part 2 - Mathematical form

$$\mathcal{R}_{\text{shift}} = \mathbb{E}_{\delta} \| f(X) - f(T_{\delta} X) \|.$$

Part 3 - Hints and Helper

Copy shared model weights before changing the downsampling block and evaluate both models on identical perturbations.

Part 4 - Python code skeleton

```
import torch

class BlurPool(torch.nn.Module):
    def __init__(self, channels, stride=2):
        super().__init__()
        self.register_buffer("kernel", ...)
        self.stride = stride

    def forward(self, x):
        return ...

def robustness_suite(model, batches):
    return {
        "clean": ...,
        "shifted": ...,
        "noisy": ...,
        "blurred": ...,
    }

def E14(baseline, blur_model, batches):
    baseline_results = robustness_suite(baseline, batches)
    blur_results = robustness_suite(blur_model, batches)
    return baseline_results, blur_results
```

14.4.16 Exercise 15 - Analyze When Pooling Helps or Hurts Based on Dataset Structure

Part 1 - Why this matters

Pooling is beneficial when exact location is less important than the presence of a pattern and when moderate compression removes redundancy. It can be harmful when labels depend on tiny objects, precise coordinates, counts, or fine boundaries.

Create controlled datasets representing location-invariant detection, small-object recognition, and spatial-relation tasks. Compare no pooling, mild pooling, and aggressive pooling under the same architecture budget.

Part 2 - Mathematical form

$$\text{utility of pooling} = \text{task invariance benefit} - \text{lost spatial information}.$$

Part 3 - Hints and Helper

Inspect per-task errors and examples. One average accuracy cannot reveal which structural assumption succeeded or failed.

Part 4 - Python code skeleton

```
import torch

def build_model(pooling_strategy, task):
    return torch.nn.Sequential(...)

def controlled_run(task_data, strategy, seed):
    torch.manual_seed(seed)
    model = build_model(strategy, task_data.name)
    train_metrics = ...
    test_metrics = ...
    failure_examples = ...
    return {
        "task": task_data.name,
        "strategy": strategy,
        "train": train_metrics,
        "test": test_metrics,
        "failures": failure_examples,
    }

def E15(tasks, strategies):
    results = []
    for task in tasks:
        for strategy in strategies:
            results.append(controlled_run(task, strategy, ...))
    return results
```

14.4.17 Exercise 16 - Stress Test Deep Downsampling Stacks and Detect Feature Collapse or Excessive Information Loss

Part 1 - Why this matters

Repeated downsampling can reduce a feature map to a tiny grid before the network has extracted enough useful structure. Activation variance, rank, spatial diversity, or class separability may collapse even when tensor shapes remain valid.

Build stacks with increasing numbers of reduction stages and monitor shape, variance, norm, unique-value ratio, and representation quality after every stage. Detect the earliest point where compression becomes destructive.

Part 2 - Mathematical form

$$H_L \approx H_0 \prod_{l=1}^{L} \frac{1}{S_l}, \qquad W_L \approx W_0 \prod_{l=1}^{L} \frac{1}{S_l}.$$

Part 3 - Hints and Helper

Use hooks and detached summaries instead of retaining all computational graphs. Define collapse thresholds before viewing results.

Part 4 - Python code skeleton

```
import torch

def stage_statistics(x):
    return {
        "shape": tuple(x.shape),
        "variance": ...,
        "norm": ...,
        "spatial_diversity": ...,
        "finite": ...,
    }

def E16(x, stage_factory, maximum_depth):
    records = []
    current = x
    for depth in range(maximum_depth):
        stage = stage_factory(depth)
        current = stage(current)
        record = stage_statistics(current)
        records.append(record)
        if ...:
            break
    return records
```

14.4.18 Exercise 17 - Build a Shape-Tracking Utility for Spatial Changes Through a CNN Pipeline

Part 1 - Why this matters

A reusable shape tracker converts architecture inspection from guesswork into a recorded contract. It should show which module changed channels or spatial dimensions and whether the observed output matches a manual prediction.

Attach temporary hooks to relevant modules, run a dry input, and log input shape, output shape, kernel, stride, padding, dilation, and ceil mode. Export the trace in a form that can be compared across architecture revisions.

Part 2 - Mathematical form

$$S_l = (N_l, C_l, H_l, W_l), \qquad \text{trace} = \{S_0 \to S_1 \to \cdots \to S_L\}.$$

Part 3 - Hints and Helper

Handle modules with tuple outputs and remove hooks in a finally block. Do not store full tensors in the log.

Part 4 - Python code skeleton

```
import torch

def module_config(module):
    return {"kernel": ..., "stride": ..., "padding": ...}

def track_shapes(model, example):
    records, handles = [], []
    def make_hook(name):
        def hook(module, inputs, output):
            records.append({"name": name,
                            "input": tuple(inputs[0].shape),
                            "output": tuple(output.shape),
                            "config": module_config(module)})
        return hook
    try:
        for name, module in model.named_modules():
            if isinstance(module, ...):
                handles.append(module.register_forward_hook(make_hook(name)))
        model(example)
    finally:
        for handle in handles: handle.remove()
    return records

def E17(models, example):
    return {...}
```

14.4.19 Exercise 18 - Create a Downsampling Strategy Comparison Report Using Multiple Configurations

Part 1 - Why this matters

Choosing a downsampling strategy requires evidence across information quality, task accuracy, robustness, speed, memory, and implementation complexity. A report prevents one attractive metric from hiding an unacceptable trade-off elsewhere.

Evaluate pooling, strided convolution, adaptive pooling, and anti-aliased alternatives using a shared protocol. Record configurations and seeds, aggregate repeated runs, and state which conclusions are task-specific.

Part 2 - Mathematical form

$$\text{decision score} = \sum_m w_m \, \widetilde{q}_m, \qquad \sum_m w_m = 1.$$

Part 3 - Hints and Helper

Keep raw metrics in the report even if a weighted summary is used. Document weights before ranking methods.

Part 4 - Python code skeleton

```
from pathlib import Path

def aggregate_runs(records):
    return {
        "accuracy_mean": ...,
        "accuracy_std": ...,
        "latency_median": ...,
        "peak_memory": ...,
        "shift_robustness": ...,
    }

def rank_strategies(summaries, weights):
    normalized = ...
    return ...

def E18(experiment_records, weights, output_path):
    summaries = aggregate_runs(experiment_records)
    ranking = rank_strategies(summaries, weights)
    report = ...
    Path(output_path).write_text(report, encoding="utf-8")
    return summaries, ranking
```

14.4.20 Exercise 19 - Write a Practical Guide for Selecting Pooling, Stride, or Global Pooling in Real CNNs

Part 1 - Why this matters

No downsampling method is universally correct. The selection should follow the task's spatial requirements, expected invariances, input resolution, compute budget, robustness needs,

and deployment constraints.

Write a decision guide that distinguishes local pooling, learned strided reduction, anti-aliased reduction, adaptive pooling, and global pooling. Include warning signs, validation experiments, and recovery steps for information loss, aliasing, shape errors, and feature collapse.

Part 2 - Mathematical form

$$\text{method} = g(\text{task geometry, resolution, invariance, accuracy, latency, memory}).$$

Part 3 - Hints and Helper

For every recommendation, provide a condition, trade-off, measurement, and failure test rather than a rule without evidence.

Part 4 - Python code skeleton

```
from pathlib import Path

REQUIRED_SECTIONS = [
    "Task Requirements",
    "Pooling Options",
    "Aliasing",
    "Shape Validation",
    "Performance",
    "Failure Tests",
    "Decision Checklist",
]

def validate_guide(text, evidence):
    missing = []
    for section in REQUIRED_SECTIONS:
        if section not in text:
            missing.append(section)
    return {"missing": missing, "evidence_links": ..., "ready": ...}

def E19(experiments, output_path):
    guide = ...
    validation = validate_guide(guide, experiments)
    Path(output_path).write_text(guide, encoding="utf-8")
    return validation
```

14.5 CNN03 - CNN Forward Architecture and Implementation

Models and Training

14.5.1 Exercise 00 - Build a Minimal CNN Classifier and Verify Forward Pass Output Dimensions

Part 1 - Why this matters

A minimal classifier connects convolutional feature extraction to class logits and exposes the complete tensor path. Every transformation must preserve the batch dimension and produce exactly one score vector per sample.

Build a small network with convolution, activation, downsampling, and a classifier head. Predict each intermediate shape manually, then verify the predictions with a dry forward pass before any training begins.

Part 2 - Mathematical form

$$X \in \mathbb{R}^{N \times C \times H \times W} \xrightarrow{f_\theta} Z \in \mathbb{R}^{N \times K}.$$

Part 3 - Hints and Helper

Start with one documented input resolution and assert the final shape. Record intermediate shapes rather than checking only the logits.

Part 4 - Python code skeleton

```
import torch

class MinimalCNN(torch.nn.Module):
    def __init__(self, in_channels, class_count):
        super().__init__()
        self.conv1 = torch.nn.Conv2d(...)
        self.conv2 = torch.nn.Conv2d(...)
        self.pool = torch.nn.MaxPool2d(...)
        self.classifier = torch.nn.Linear(...)

    def forward(self, x):
        x = ...
        x = ...
        x = ...
        return self.classifier(x)

def E00(input_shape, class_count):
    model = MinimalCNN(input_shape[1], class_count)
    output = model(torch.randn(input_shape))
    return tuple(output.shape), ...
```

14.5.2 Exercise 01 - Implement CNN with nn.Sequential and Compare It to Explicit Forward Function Style

Part 1 - Why this matters

Sequential composition is concise for a single linear path, while an explicit forward method makes branching, reused outputs, debugging, and custom control flow easier to express. Both styles should implement identical mathematics when their modules and order match.

Create equivalent models in both styles, copy one state dictionary into the other, and compare outputs and intermediate observability. Explain which architecture changes would no longer fit a simple sequential container.

Part 2 - Mathematical form

$$f(x) = f_L \circ f_{L-1} \circ \cdots \circ f_1(x).$$

Part 3 - Hints and Helper

Use named ordered blocks or matching parameter keys, then switch both models to evaluation mode before parity testing.

Part 4 - Python code skeleton

```
import torch

class ExplicitCNN(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = torch.nn.Conv2d(...)
        self.activation = torch.nn.ReLU()
        self.pool = torch.nn.MaxPool2d(...)

    def forward(self, x):
        x = self.conv(x)
        x = ...
        return ...

def build_sequential():
    return torch.nn.Sequential(...)

def E01(x):
    explicit, sequential = ExplicitCNN(), build_sequential()
    sequential.load_state_dict(...)
    return explicit(x), sequential(x), ...
```

14.5.3 Exercise 02 - Add ReLU Between Conv Layers and Confirm Non-Linearity Changes Representation

Part 1 - Why this matters

Several convolutional layers without non-linear activation collapse into one linear transformation apart from boundary details. ReLU prevents this collapse by suppressing negative responses and creating input-dependent activation patterns.

Compare a two-layer linear convolutional stack with the same stack containing ReLU. Test scaling and additivity, inspect zero fractions, and demonstrate a pair of inputs for which the activated model violates linear superposition.

Part 2 - Mathematical form

$$\mathrm{ReLU}(z) = \max(0, z), \qquad f(aX + bY) \neq af(X) + bf(Y) \text{ in general.}$$

Part 3 - Hints and Helper

Copy identical convolution weights between variants. Test additivity with controlled inputs containing positive and negative pre-activations.

Part 4 - Python code skeleton

```
import copy
import torch

def build_variants():
    linear = torch.nn.Sequential(...)
    nonlinear = torch.nn.Sequential(...)
    ...
    return linear, nonlinear

def superposition_error(model, x, y, a, b):
    combined = model(a * x + b * y)
    separate = a * model(x) + b * model(y)
    return ...

def E02(x, y):
    linear, nonlinear = build_variants()
    return {"linear": superposition_error(...),
            "nonlinear": superposition_error(...),
            "zero_fraction": ...}
```

14.5.4 Exercise 03 - Implement Flatten Correctly and Debug Batch Dimension Mistakes

Part 1 - Why this matters

Flattening converts spatial features into vectors for a dense head, but the batch axis must remain separate. Flattening the entire tensor merges samples, while using a fixed view can fail on different batch sizes or non-contiguous storage.

Implement safe flattening, compare it with incorrect alternatives, and validate feature count for several batch sizes. Detect the first point where sample identity or expected classifier input is lost.

Part 2 - Mathematical form

$$X \in \mathbb{R}^{N \times C \times H \times W} \mapsto X' \in \mathbb{R}^{N \times CHW}.$$

Part 3 - Hints and Helper

Use flatten starting at dimension one or reshape with the current batch size. Do not hard-code the batch dimension.

Part 4 - Python code skeleton

```
import torch

def safe_flatten(x):
    if x.ndim < 2:
        raise ValueError(...)
    return ...

def validate_flatten(x, expected_features):
    flattened = safe_flatten(x)
    if flattened.shape != (x.shape[0], expected_features):
        raise ValueError(...)
    return flattened

def E03(shapes):
    results = []
    for shape in shapes:
        x = torch.randn(shape)
        results.append(validate_flatten(x, ...).shape)
    return results
```

14.5.5 Exercise 04 - Replace Flatten and Fully Connected Head with Global Average Pooling and Compare Cost

Part 1 - Why this matters

A flatten-plus-dense head assigns separate weights to every spatial position and can dominate parameter count. Global average pooling summarizes each channel before classification, reducing parameters and allowing variable spatial sizes.

Build matched CNN bodies with both heads and compare logits shape, parameter count, activation memory, latency, and validation performance. Explain what spatial information the global head removes.

Part 2 - Mathematical form

$$z_c = \frac{1}{HW}\sum_{i,j} X_{c,i,j}, \qquad \#\theta_{\text{GAP head}} = CK + K.$$

Part 3 - Hints and Helper

Keep the feature extractor identical and use adaptive average pooling to one spatial cell.

Part 4 - Python code skeleton

```
import copy
import torch

class GlobalHead(torch.nn.Module):
    def __init__(self, channels, classes):
        super().__init__()
        self.pool = torch.nn.AdaptiveAvgPool2d(1)
        self.fc = torch.nn.Linear(channels, classes)

    def forward(self, x):
        return self.fc(...)

def E04(backbone, example, classes):
    dense_model = ...
    global_model = ...
    return {"dense_parameters": ...,
            "global_parameters": ...,
            "output_shapes": ...,
            "latency": ...}
```

14.5.6 Exercise 05 - Add Dropout to the CNN Classifier Head and Verify Train and Eval Behavior

Part 1 - Why this matters

Dropout randomly suppresses activations during training and rescales surviving values, creating stochastic subnetworks. During evaluation it must become inactive so repeated inference is stable.

Insert dropout in the classifier head, perform repeated forward passes in both modes, and compare output variance and zero fractions. Also verify that changing mode does not disable gradient tracking by itself.

Part 2 - Mathematical form

$$\widetilde{h}_i = \frac{m_i}{1-p} h_i, \qquad m_i \sim \text{Bernoulli}(1-p).$$

Part 3 - Hints and Helper

Use the same input repeatedly. Call train and eval explicitly and separate mode behavior from inference-mode behavior.

Part 4 - Python code skeleton

```
import torch

class DropoutHead(torch.nn.Module):
    def __init__(self, features, classes, probability):
        super().__init__()
        self.dropout = torch.nn.Dropout(probability)
        self.linear = torch.nn.Linear(features, classes)

    def forward(self, x):
        return self.linear(self.dropout(x))

def repeated_outputs(model, x, repeats):
    return torch.stack([...])

def E05(model, x):
    model.train()
    train_outputs = repeated_outputs(model, x, ...)
    model.eval()
    eval_outputs = repeated_outputs(model, x, ...)
    return {"train_variance": ..., "eval_variance": ...}
```

14.5.7 Exercise 06 - Implement Batch Normalization and Verify Running Statistics and Mode Switching

Part 1 - Why this matters

Batch normalization uses current batch statistics during training while updating running estimates for evaluation. Incorrect mode switching can make validation depend on batch composition and produce unstable predictions.

Insert BatchNorm after convolution, inspect learned parameters and buffers, and observe running mean and variance across controlled batches. Verify that evaluation uses stored statistics without updating them.

Part 2 - Mathematical form

$$\widehat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \qquad y = \gamma \widehat{x} + \beta.$$

Part 3 - Hints and Helper

Record buffers before and after each pass. Use deliberately shifted batches so statistic updates remain visible.

Part 4 - Python code skeleton

```
import torch

class NormalizedBlock(torch.nn.Module):
    def __init__(self, in_channels, out_channels):
        super().__init__()
        self.conv = torch.nn.Conv2d(...)
        self.norm = torch.nn.BatchNorm2d(out_channels)
        self.activation = torch.nn.ReLU()

    def forward(self, x):
        return ...

def snapshot(norm):
    return {"mean": norm.running_mean.clone(),
            "variance": norm.running_var.clone()}

def E06(block, batches):
    before = snapshot(block.norm)
    block.train(); ...
    after_train = snapshot(block.norm)
    block.eval(); ...
    after_eval = snapshot(block.norm)
    return before, after_train, after_eval
```

14.5.8 Exercise 07 - Add a Residual-Like Skip Connection and Verify Shape Compatibility

Part 1 - Why this matters

A skip connection adds an earlier representation to a transformed path, providing a direct route for information and gradients. Addition is valid only when batch, channel, and spatial dimensions are compatible.

Implement an identity skip and a projection skip for changed channels or stride. Check shapes before addition and compare parameter count and output behavior for both cases.

Part 2 - Mathematical form

$$Y = F(X) + P(X), \qquad \text{shape}(F(X)) = \text{shape}(P(X)).$$

Part 3 - Hints and Helper

Use an identity projection only when all dimensions already match; otherwise use a one-by-one convolution with appropriate stride.

Part 4 - Python code skeleton

```
import torch

class ResidualLikeBlock(torch.nn.Module):
    def __init__(self, in_channels, out_channels, stride=1):
        super().__init__()
        self.main = torch.nn.Sequential(...)
        self.skip = ...

    def forward(self, x):
        main = self.main(x)
        skip = self.skip(x)
        if main.shape != skip.shape:
            raise ValueError(...)
        return ...

def E07(configurations):
    return [...]
```

14.5.9 Exercise 08 - Inspect Intermediate Feature Maps with Forward Hooks and Save Them for Debugging

Part 1 - Why this matters

Forward hooks expose module outputs without rewriting the model's forward method. They are useful for locating shape changes, dead activations, non-finite values, and unexpected feature distributions.

Register hooks on selected layers, store detached CPU copies or compact statistics, and remove every handle after one diagnostic pass. Save metadata that links each feature map to its module and input.

Part 2 - Mathematical form

$$\mathcal{H}_l : (M_l, X_l, Y_l) \mapsto \text{diagnostic}(Y_l).$$

Part 3 - Hints and Helper

Do not retain graph-connected GPU tensors across batches. Use a context-managed pattern so hooks cannot accumulate accidentally.

Part 4 - Python code skeleton

```
import torch

def capture_features(model, x, selected_names, save_full=False):
    records, handles = {}, []
    def make_hook(name):
        def hook(module, inputs, output):
            records[name] = {
                "shape": tuple(output.shape),
                "mean": ...,
                "std": ...,
                "values": output.detach().cpu() if save_full else None,
            }
        return hook
    try:
        for name, module in model.named_modules():
            if name in selected_names:
                handles.append(module.register_forward_hook(make_hook(name)))
        model(x)
    finally:
        for handle in handles: handle.remove()
    return records

def E08(model, x):
    return capture_features(model, x, ..., ...)
```

14.5.10 Exercise 09 - Debug CNN Forward Failures: Shape Mismatch, NaNs, and Wrong Channel Ordering

Part 1 - Why this matters

Forward failures often appear far from their cause. A wrong channel layout may reach the first convolution, a bad scale may create non-finite values later, and an incorrect spatial prediction may break only at the classifier.

Build a diagnostic forward routine that validates input contract and records shape, device, dtype, finite status, and value range layer by layer. Return the first violated invariant with actionable context.

Part 2 - Mathematical form

$$\text{valid}(A_l) = \text{shape ok} \wedge \text{finite}(A_l) \wedge \text{device ok}\,.$$

Part 3 - Hints and Helper

Fail at the earliest invalid layer. Do not repair wrong channel order with an arbitrary reshape.

Part 4 - Python code skeleton

```
import torch

def validate_tensor(name, tensor, expected_channels=None):
    checks = {
        "shape": tuple(tensor.shape),
        "dtype": str(tensor.dtype),
        "device": str(tensor.device),
        "finite": bool(torch.isfinite(tensor).all()),
    }
    if expected_channels is not None and tensor.shape[1] != expected_channels:
        raise ValueError(...)
    if not checks["finite"]:
        raise FloatingPointError(...)
    return checks

def E09(model, failing_inputs):
    reports = []
    for case in failing_inputs:
        try: reports.append(...)
        except Exception as error: reports.append({...})
    return reports
```

14.5.11 Exercise 10 - Move a Full CNN Forward Pass to CUDA and Confirm Shared Device Placement

Part 1 - Why this matters

Moving only the input or only the model creates a device mismatch. A complete CUDA forward path includes parameters, buffers, inputs, and newly created tensors inside the model.

Move a model and batch to one selected CUDA device, audit all parameter and buffer devices, and compare output shape and numerical parity with a copied CPU reference.

Part 2 - Mathematical form

$$\text{device}(X) = \text{device}(\theta) = \text{device}(B) = \text{device}(Y).$$

Part 3 - Hints and Helper

Create constants from existing tensors or register them as buffers. Do not call CUDA blindly inside forward.

Part 4 - Python code skeleton

```
import copy
import torch

def audit_devices(model):
    devices = {str(p.device) for p in model.parameters()}
    devices.update(str(b.device) for b in model.buffers())
    return devices

def E10(model, x_cpu):
    if not torch.cuda.is_available():
        return {"available": False}
    cpu_model = copy.deepcopy(model).eval()
    gpu_model = copy.deepcopy(model).cuda().eval()
    with torch.inference_mode():
        cpu_output = cpu_model(x_cpu)
        gpu_output = gpu_model(x_cpu.cuda())
    return {"available": True, "devices": audit_devices(gpu_model),
            "shape": tuple(gpu_output.shape), "maximum_error": ...}
```

14.5.12 Exercise 11 - Benchmark Forward Throughput and Identify Expensive Layers on GPU

Part 1 - Why this matters

End-to-end latency shows whether a model meets a target, but it does not identify which layers consume time. Layer profiling reveals compute-heavy operations, synchronization costs, and unexpected bottlenecks.

Measure warmed-up end-to-end latency and per-layer CUDA time across batch sizes. Report throughput and each layer's share while avoiding hook overhead in the final production benchmark.

Part 2 - Mathematical form

$$\text{throughput} = N/t, \qquad q_l = t_l / \sum_j t_j.$$

Part 3 - Hints and Helper

Use CUDA events or a profiler and synchronize measurement boundaries. Repeat enough times to summarize stable timing distributions.

Part 4 - Python code skeleton

```
import torch

def profile_forward(model, x, warmup, repeats):
    for _ in range(warmup): model(x)
```

```
    layer_times = {}
    handles = [...]
    for _ in range(repeats):
        ...
    for handle in handles: handle.remove()
    return {"latency": ..., "throughput": ...,
            "layer_times": layer_times}

def E11(model, batch_shapes):
    model = model.cuda().eval()
    results = []
    for shape in batch_shapes:
        x = torch.randn(shape, device="cuda")
        results.append(profile_forward(model, x, ..., ...))
    return results
```

14.5.13 Exercise 12 - Compare float32 and Mixed Precision Forward Outputs and Detect Sensitive Layers

Part 1 - Why this matters

Mixed precision can accelerate convolution and reduce activation memory, but some layers or input scales may lose accuracy or become non-finite. End-to-end error alone cannot reveal where divergence begins.

Capture matched layer outputs in float32 and autocast execution, then calculate absolute and relative errors per layer. Include extreme input scales and identify the first precision-sensitive transformation.

Part 2 - Mathematical form

$$\epsilon_l = \frac{\|A_l^{\text{mixed}} - A_l^{32}\|}{\|A_l^{32}\| + \varepsilon}.$$

Part 3 - Hints and Helper

Use one model state and identical input. Convert captured mixed outputs to float32 before comparison.

Part 4 - Python code skeleton

```
import torch

def forward_features(model, x, use_autocast):
    records, handles = {}, [...]
    with torch.autocast("cuda", enabled=use_autocast):
        output = model(x)
    for handle in handles: handle.remove()
```

```
    return output, records

def E12(model, x):
    model = model.cuda().eval()
    x = x.cuda()
    reference, maps32 = forward_features(model, x, False)
    mixed, maps_mixed = forward_features(model, x, True)
    errors = {...}
    return reference, mixed, errors
```

14.5.14 Exercise 13 - Implement a Configurable CNN Builder from a Layer Specification List

Part 1 - Why this matters

A configuration-driven builder makes architecture experiments reproducible and prevents repeated hand editing. It must validate channel flow, supported layer types, arguments, and output contracts before constructing modules.

Define a small explicit schema for convolution, activation, pooling, normalization, and adaptive pooling. Build named modules in order, track predicted shape, and reject invalid or ambiguous specifications.

Part 2 - Mathematical form

$$\mathcal{A} = \left[(t_1, \phi_1), \ldots, (t_L, \phi_L)\right] \xrightarrow{\text{build}} f_\theta.$$

Part 3 - Hints and Helper

Avoid evaluating arbitrary strings. Map approved type names to constructors and validate every field.

Part 4 - Python code skeleton

```
import torch

LAYER_TYPES = {
    "conv": torch.nn.Conv2d,
    "relu": torch.nn.ReLU,
    "max_pool": torch.nn.MaxPool2d,
    "batch_norm": torch.nn.BatchNorm2d,
    "adaptive_average": torch.nn.AdaptiveAvgPool2d,
}

def build_cnn(specifications, input_shape):
    modules = []
    shape = input_shape
    trace = []
```

```
    for index, spec in enumerate(specifications):
        layer_type = spec.get("type")
        if layer_type not in LAYER_TYPES:
            raise ValueError(...)
        module = LAYER_TYPES[layer_type](...)
        shape = ...
        modules.append(module)
        trace.append((index, shape))
    return torch.nn.Sequential(*modules), trace

def E13(specifications, input_shape):
    return build_cnn(specifications, input_shape)
```

14.5.15 Exercise 14 - Validate Parameter Count and Memory Footprint for Multiple CNN Architecture Variants

Part 1 - Why this matters

Two CNNs with similar output accuracy can differ greatly in parameter storage, activation memory, and temporary workspaces. Parameter count alone does not predict training memory because saved activations often dominate.

Compare architecture variants with manual and programmatic parameter counts. Measure parameter bytes, buffer bytes, output activation bytes, and peak CUDA memory in inference and training.

Part 2 - Mathematical form

$$M_{\text{parameters}} = \sum_{p} \#p \cdot \text{bytes}(p), \qquad M_{\text{peak}} \geq M_{\text{parameters}} + M_{\text{activations}}.$$

Part 3 - Hints and Helper

Reset CUDA peak statistics for each isolated model and include gradients and optimizer state only when the scenario requires them.

Part 4 - Python code skeleton

```
import torch

def model_storage(model):
    parameter_bytes = sum(p.numel() * p.element_size()
                          for p in model.parameters())
    buffer_bytes = sum(b.numel() * b.element_size()
                       for b in model.buffers())
    return parameter_bytes, buffer_bytes

def profile_memory(model, x, training):
```

```
    ...
    return {"parameters": ..., "buffers": ...,
            "peak_memory": ..., "output_bytes": ...}

def E14(variants, x):
    results = {}
    for name, model in variants.items():
        results[name] = profile_memory(model, x, ...)
    return results
```

14.5.16 Exercise 15 - Test CNN Forward Robustness Across Input Sizes with Adaptive Pooling

Part 1 - Why this matters

A CNN with adaptive pooling can accept several spatial sizes while preserving a fixed classifier input. Earlier convolutions still impose minimum-size and aspect-ratio constraints that must be tested.

Run one model on square, rectangular, small, and large inputs. Validate output shape, finite values, memory, and consistency of preprocessing, then document the smallest supported dimensions.

Part 2 - Mathematical form

$$H, W \text{ variable} \xrightarrow{\text{adaptive pool}} H_t, W_t \text{ fixed.}$$

Part 3 - Hints and Helper

Keep channel count and normalization contract fixed. Catch invalid small inputs with a clear pre-forward validation error.

Part 4 - Python code skeleton

```
import torch

def validate_input(x, channels, minimum_hw):
    if x.ndim != 4 or x.shape[1] != channels:
        raise ValueError(...)
    if x.shape[-2] < minimum_hw[0] or x.shape[-1] < minimum_hw[1]:
        raise ValueError(...)

def E15(model, shapes, class_count):
    results = []
    model.eval()
    for shape in shapes:
        try:
            x = torch.randn(shape)
```

```
            validate_input(x, ..., ...)
            output = model(x)
            results.append({"shape": shape, "output": tuple(output.shape),
                            "finite": ...})
        except Exception as error:
            results.append({"shape": shape, "error": str(error)})
    return results
```

14.5.17 Exercise 16 - Create Unit Tests for Output Shapes and Dtypes at Every CNN Layer

Part 1 - Why this matters

Layer-level tests detect architectural regressions before training hides them behind slow failures. Shapes, data types, devices, and finite-value expectations form a practical forward contract.

Create parameterized tests for supported input sizes and precisions, capture intermediate outputs, and compare them with an expected specification. Include deliberately invalid inputs and verify informative failures.

Part 2 - Mathematical form

$$\forall l : \ \text{shape}(A_l) = S_l \wedge \text{dtype}(A_l) = D_l.$$

Part 3 - Hints and Helper

Test the architecture in evaluation mode and keep expected values independent of the implementation being tested.

Part 4 - Python code skeleton

```
import torch

def collect_layer_metadata(model, x):
    records, handles = {}, [...]
    model(x)
    for handle in handles: handle.remove()
    return records

def assert_contract(actual, expected):
    for name, contract in expected.items():
        assert actual[name]["shape"] == contract["shape"]
        assert actual[name]["dtype"] == contract["dtype"]
        assert actual[name]["finite"]

def E16(model, test_cases):
    for case in test_cases:
```

```
        actual = collect_layer_metadata(model, case["input"])
        assert_contract(actual, case["expected"])
    return True
```

14.5.18 Exercise 17 - Stress Test CNN Forward with Large Batches and Monitor GPU Memory Peaks

Part 1 - Why this matters

Increasing batch size improves parallel work until activation and workspace memory approach device capacity. A safe stress test must find practical limits without repeatedly destabilizing the CUDA process.

Increase batch size systematically, record latency, throughput, allocated memory, reserved memory, and failure point, and compare full precision with mixed precision under inference mode.

Part 2 - Mathematical form

$$M_{\text{peak}}(B) \approx M_{\text{fixed}} + B\, M_{\text{sample}}.$$

Part 3 - Hints and Helper

Reset peak statistics per trial and stop after the first out-of-memory result. Keep a safety margin below the largest successful batch.

Part 4 - Python code skeleton

```
import torch

def one_trial(model, batch_size, sample_shape, mixed):
    torch.cuda.reset_peak_memory_stats()
    x = torch.randn((batch_size, *sample_shape), device="cuda")
    with torch.inference_mode(), torch.autocast("cuda", enabled=mixed):
        output = model(x)
    return {"batch": batch_size,
            "peak": torch.cuda.max_memory_allocated(),
            "throughput": ..., "shape": tuple(output.shape)}

def E17(model, batch_sizes, sample_shape):
    results = []
    for batch_size in batch_sizes:
        try: results.append(one_trial(model, batch_size, sample_shape, ...))
        except torch.cuda.OutOfMemoryError:
            results.append({"batch": batch_size, "failed": True})
            break
    return results
```

14.5.19 Exercise 18 - Implement a Minimal CNN Inference Wrapper and Validate Preprocessing Requirements

Part 1 - Why this matters

An inference wrapper protects the model from malformed images and ensures that resizing, channel order, value range, normalization, device, and batch rules match training. Silent preprocessing drift can invalidate an otherwise correct model.

Create separate validation, preprocessing, inference, and formatting stages. Test valid images, wrong channels, non-finite values, unsupported sizes, and oversized batches with explicit error messages.

Part 2 - Mathematical form

$$\text{request} \to \text{validate} \to \text{preprocess} \to f_\theta \to \text{response}\,.$$

Part 3 - Hints and Helper

Store preprocessing configuration with the model version and run inference without gradient tracking.

Part 4 - Python code skeleton

```
import torch

class CNNInferenceWrapper:
    def __init__(self, model, config, device):
        self.model = model.to(device).eval()
        self.config = config
        self.device = device

    def validate(self, images):
        ...

    def preprocess(self, images):
        ...

    def predict(self, images):
        self.validate(images)
        batch = self.preprocess(images).to(self.device)
        with torch.inference_mode():
            logits = self.model(batch)
        return {"version": ..., "predictions": ...}

def E18(wrapper, test_cases):
    return [...]
```

14.5.20 Exercise 19 - Produce a CNN Forward Design Report Explaining Architecture Choices and Their Effects

Part 1 - Why this matters

A forward architecture is a chain of assumptions about locality, resolution, channels, normalization, regularization, and classifier structure. A design report makes these assumptions reviewable and connects them to measurable consequences.

Document the tensor trace, receptive-field progression, parameter and memory budget, non-linearities, normalization, downsampling, skip paths, supported inputs, CUDA behavior, precision limits, and known failure cases. Support choices with evidence from earlier exercises.

Part 2 - Mathematical form

$$\mathcal{D} = \{\text{shape}, \text{capacity}, \text{receptive field}, \text{latency}, \text{memory}, \text{robustness}\}.$$

Part 3 - Hints and Helper

For each architectural choice, state its purpose, alternative, measured effect, and verification test.

Part 4 - Python code skeleton

```
from pathlib import Path

REQUIRED_SECTIONS = [
    "Input Contract", "Shape Trace", "Feature Extractor",
    "Classifier Head", "Parameters and Memory", "CUDA and Precision",
    "Failure Modes", "Verification Checklist",
]

def validate_report(report, evidence):
    missing = []
    for section in REQUIRED_SECTIONS:
        if section not in report:
            missing.append(section)
    return {"missing": missing, "evidence_links": ..., "ready": ...}

def E19(experiment_records, output_path):
    report = ...
    checks = validate_report(report, experiment_records)
    Path(output_path).write_text(report, encoding="utf-8")
    return checks
```

14.6 CNN04 - CNN Training Loop, Loss, and Backpropagation

Models and Training

14.6.1 Exercise 00 - Build a Complete CNN Training Loop with Correct Train/Eval Modes and Loss Computation

Part 1 - Why this matters

A correct training loop coordinates model mode, device transfer, forward computation, scalar loss, gradient clearing, backward propagation, and parameter updates. Validation must disable gradient tracking and aggregate metrics over the complete dataset.

Implement separate training and evaluation functions. Weight accumulated loss by batch size, count correct predictions from logits, and verify that dropout and normalization receive the intended mode.

Part 2 - Mathematical form

$$\bar{L} = \frac{\sum_b n_b L_b}{\sum_b n_b}, \qquad \text{accuracy} = \frac{\sum_b \text{correct}_b}{\sum_b n_b}.$$

Part 3 - Hints and Helper

Call `zero_grad` before backward, never update during validation, and divide accumulated totals only after the loader finishes.

Part 4 - Python code skeleton

```
import torch

def run_epoch(model, loader, loss_fn, device, optimizer=None):
    training = optimizer is not None
    model.train(training)
    total_loss = total_correct = total_samples = 0
    context = torch.enable_grad() if training else torch.inference_mode()
    with context:
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            if training: optimizer.zero_grad(set_to_none=True)
            logits = model(images)
            loss = loss_fn(logits, labels)
            if training:
                loss.backward(); optimizer.step()
            batch_size = labels.shape[0]
            total_loss += ...
            total_correct += ...
            total_samples += batch_size
    return {"loss": ..., "accuracy": ...}

def E00(model, train_loader, val_loader, optimizer, loss_fn, device):
    return run_epoch(...), run_epoch(...)
```

14.6.2 Exercise 01 - Overfit a Tiny Dataset to Validate End-to-End CNN Correctness

Part 1 - Why this matters

A sufficiently expressive CNN should memorize a very small clean dataset. Failure to do so is a strong sign of a bug in labels, output shape, loss, gradients, optimizer order, preprocessing, or model capacity.

Select a fixed tiny subset, disable unnecessary regularization and augmentation, and train until loss approaches zero and accuracy approaches perfection. Record diagnostics if the target is not reached.

Part 2 - Mathematical form

$$\text{tiny-set accuracy} \to 100\%, \qquad L \to 0.$$

Part 3 - Hints and Helper

Use deterministic samples and inspect every label. This is a pipeline test, not evidence of generalization.

Part 4 - Python code skeleton

```
import torch

def E01(model, tiny_loader, optimizer, loss_fn, epochs, target_accuracy):
    history = []
    for epoch in range(epochs):
        metrics = ...
        history.append(metrics)
        if metrics["accuracy"] >= target_accuracy:
            break
    return {"passed": ..., "history": history,
            "final_parameters_finite": ...}
```

14.6.3 Exercise 02 - Compare CrossEntropyLoss and BCEWithLogitsLoss for Binary Image Classification

Part 1 - Why this matters

Binary classification can use two logits with integer class labels and cross-entropy, or one logit with floating binary targets and binary cross-entropy. Mixing output shapes, target encodings, or an extra sigmoid produces incorrect training.

Implement both valid formulations on the same batch, inspect loss and gradients, and map logits to class predictions correctly. Compare parameter count in the final layer without claiming the objectives are numerically identical.

Part 2 - Mathematical form

$$L_{\mathrm{BCE}} = -y \log \sigma(z) - (1-y)\log(1-\sigma(z)), \qquad L_{\mathrm{CE}} = -\log \mathrm{softmax}(z)_y.$$

Part 3 - Hints and Helper

Pass raw logits to both PyTorch losses. BCE targets must match the single-logit shape and use floating type.

Part 4 - Python code skeleton

```
import torch

def binary_formulation(features, labels, head):
    logits = head(features).squeeze(1)
    loss = torch.nn.BCEWithLogitsLoss()(logits, labels.float())
    predictions = (logits >= 0).long()
    return loss, predictions

def two_class_formulation(features, labels, head):
    logits = head(features)
    loss = torch.nn.CrossEntropyLoss()(logits, labels.long())
    return loss, logits.argmax(dim=1)

def E02(features, labels):
    return binary_formulation(...), two_class_formulation(...)
```

14.6.4 Exercise 03 - Inspect Layer-Wise Gradients and Detect Vanishing or Exploding Patterns

Part 1 - Why this matters

Layer-wise gradient statistics show whether the loss can influence early and late parameters. Extremely small norms may indicate blocked or vanishing paths, while rapidly growing or non-finite norms can destabilize updates.

After one backward pass, record norm, maximum magnitude, mean, zero fraction, and finite status for every trainable parameter. Compare layers and repeat across several steps to distinguish one noisy batch from a persistent pattern.

Part 2 - Mathematical form

$$g_l = \|\nabla_{\theta_l} L\|_2, \qquad r_l = g_l / (\|\theta_l\|_2 + \varepsilon).$$

Part 3 - Hints and Helper

Check for a missing gradient separately from a zero gradient. Log before optimizer.step and after backward.

Part 4 - Python code skeleton

```
import torch

def gradient_report(model):
    report = {}
    for name, parameter in model.named_parameters():
        gradient = parameter.grad
        report[name] = {"missing": gradient is None,
                        "norm": ... if gradient is not None else None,
                        "maximum": ...,
                        "finite": ...}
    return report

def E03(model, batch, loss_fn):
    model.zero_grad(set_to_none=True)
    loss = loss_fn(model(batch[0]), batch[1])
    loss.backward()
    return loss.item(), gradient_report(model)
```

14.6.5 Exercise 04 - Validate Backpropagation with Finite Differences on a Tiny CNN

Part 1 - Why this matters

Finite differences provide an independent numerical approximation of a selected derivative. Comparing them with autograd can expose detached tensors, wrong custom operations, and incorrect gradient assumptions.

Use a tiny float64 CNN and deterministic input, perturb selected parameter elements in both directions, and compare the central-difference estimate with autograd. Keep the network smooth around tested points.

Part 2 - Mathematical form

$$\frac{\partial L}{\partial \theta_i} \approx \frac{L(\theta_i + \epsilon) - L(\theta_i - \epsilon)}{2\epsilon}.$$

Part 3 - Hints and Helper

Disable dropout, avoid ReLU points near zero, restore each parameter after perturbation, and choose epsilon appropriate for float64.

Part 4 - Python code skeleton

```
import torch

def finite_difference(model, loss_fn, inputs, targets, parameter, index, eps):
    original = parameter[index].item()
    with torch.no_grad(): parameter[index] = original + eps
    plus = loss_fn(model(inputs), targets).item()
    with torch.no_grad(): parameter[index] = original - eps
    minus = loss_fn(model(inputs), targets).item()
    with torch.no_grad(): parameter[index] = original
    return ...

def E04(model, inputs, targets, checks):
    model.zero_grad(); loss = ...; loss.backward()
    return [...]
```

14.6.6 Exercise 05 - Debug NaN Loss by Checking Inputs, Initialization, and Learning Rate

Part 1 - Why this matters

A non-finite loss is a symptom rather than a diagnosis. It may originate in corrupt inputs, unstable normalization, extreme initialization, invalid labels, excessive learning rate, or an earlier non-finite activation or gradient.

Build a diagnostic step that checks finite values at input, logits, loss, gradients, and updated parameters. Stop at the first failure and attach layer statistics so the root cause is localized.

Part 2 - Mathematical form

$$\text{finite}(X) \wedge \text{finite}(Z) \wedge \text{finite}(L) \wedge \text{finite}(\nabla L).$$

Part 3 - Hints and Helper

Never continue optimizer updates after a non-finite loss. Reproduce the failing batch before changing several settings at once.

Part 4 - Python code skeleton

```
import torch

def require_finite(name, tensor):
    if not torch.isfinite(tensor).all():
        raise FloatingPointError(name)

def diagnostic_step(model, batch, loss_fn, optimizer):
    images, labels = batch
    require_finite("images", images)
    optimizer.zero_grad(set_to_none=True)
```

```
    logits = model(images); require_finite("logits", logits)
    loss = loss_fn(logits, labels); require_finite("loss", loss)
    loss.backward()
    for name, parameter in model.named_parameters():
        if parameter.grad is not None: require_finite(name, parameter.grad)
    ...
    return ...

def E05(failing_cases):
    return [...]
```

14.6.7 Exercise 06 - Implement Gradient Clipping and Measure Its Stabilizing Effect

Part 1 - Why this matters

Gradient clipping limits an update when its norm or individual values become excessive. It can prevent a transient spike from destroying training, but constant clipping may hide a bad learning rate or unstable architecture.

Compare identical runs with and without global norm clipping. Record pre-clip norm, returned norm, post-clip norm, loss, and parameter change, then calculate how often clipping activates.

Part 2 - Mathematical form

$$g \leftarrow g \min\left(1, \frac{\tau}{\|g\|_2}\right).$$

Part 3 - Hints and Helper

Clip after backward and before optimizer.step. Diagnose the cause if nearly every step is clipped.

Part 4 - Python code skeleton

```
import torch

def train_step(model, batch, optimizer, loss_fn, threshold):
    optimizer.zero_grad(set_to_none=True)
    loss = loss_fn(model(batch[0]), batch[1])
    loss.backward()
    before = ...
    returned = torch.nn.utils.clip_grad_norm_(model.parameters(), threshold)
    after = ...
    optimizer.step()
    return {"loss": loss.item(), "before": before,
            "returned": returned.item(), "after": after,
```

```
            "clipped": ...}

def E06(controlled_runs):
    return {...}
```

14.6.8 Exercise 07 - Implement Learning Rate Scheduling and Verify Step Timing and Outcome

Part 1 - Why this matters

A scheduler changes the optimizer learning rate according to epoch, update, or validation behavior. Calling it at the wrong frequency or position produces a different schedule from the intended one.

Implement one epoch-based and one metric-based schedule. Log the learning rate used for every update and verify the trace against a manually calculated expectation.

Part 2 - Mathematical form

$$\eta_t = s(t, \eta_0, \text{metrics}), \qquad \theta_{t+1} = \theta_t - \eta_t g_t.$$

Part 3 - Hints and Helper

Decide whether scheduling occurs before or after an optimizer step based on the scheduler contract, and resume its state from checkpoints.

Part 4 - Python code skeleton

```
import torch

def E07(model, loader, optimizer, scheduler, loss_fn, epochs):
    history = []
    for epoch in range(epochs):
        learning_rate_used = optimizer.param_groups[0]["lr"]
        train_metrics = ...
        validation_metrics = ...
        if isinstance(scheduler, torch.optim.lr_scheduler.ReduceLROnPlateau):
            scheduler.step(...)
        else:
            scheduler.step()
        history.append({
            "epoch": epoch,
            "lr": learning_rate_used,
            "train": train_metrics,
            "validation": validation_metrics,
        })
    return history, ...
```

14.6.9 Exercise 08 - Compare SGD and Adam on the Same CNN and Explain Outcome Differences

Part 1 - Why this matters

SGD applies one shared learning-rate scale, while Adam normalizes updates using running first and second moments. Their early speed, final generalization, memory cost, and sensitivity to hyperparameters can differ.

Run controlled experiments from identical initial weights and data order. Compare loss, accuracy, gradient and update norms, runtime, optimizer-state memory, and final validation behavior.

Part 2 - Mathematical form

$$\text{SGD} : \Delta\theta = -\eta g, \qquad \text{Adam} : \Delta\theta = -\eta \frac{\widehat{m}}{\sqrt{\widehat{v}} + \epsilon}.$$

Part 3 - Hints and Helper

Clone the model state before constructing optimizers and tune learning rates fairly instead of forcing identical values.

Part 4 - Python code skeleton

```
import copy
import torch

def controlled_run(model, loader, optimizer_factory, seed):
    torch.manual_seed(seed)
    optimizer = optimizer_factory(model.parameters())
    history = ...
    state_bytes = ...
    return {"history": history, "state_bytes": state_bytes,
            "final_state": copy.deepcopy(model.state_dict())}

def E08(base_model, loader):
    sgd_model = copy.deepcopy(base_model)
    adam_model = copy.deepcopy(base_model)
    sgd_results = controlled_run(sgd_model, loader, ...)
    adam_results = controlled_run(adam_model, loader, ...)
    return sgd_results, adam_results
```

14.6.10 Exercise 09 - Debug Training Where Loss Improves but Accuracy Does Not Improve

Part 1 - Why this matters

Loss can decrease while accuracy remains unchanged because confidence improves without crossing class boundaries. It can also signal wrong target encoding, metric logic, thresholding, class imbalance, or mismatched sample aggregation.

Inspect logits, predictions, labels, confusion matrix, confidence, per-class loss, and metric code on a fixed batch. Recalculate accuracy manually and test whether decision thresholds or label mappings are correct.

Part 2 - Mathematical form

$$\widehat{y} = \arg\max_k z_k, \qquad \text{accuracy} = N^{-1} \sum_i \mathbf{1}[\widehat{y}_i = y_i].$$

Part 3 - Hints and Helper

Confirm that loss and metric consume the same semantic task. Aggregate correct counts and sample counts, not unweighted batch accuracies.

Part 4 - Python code skeleton

```
import torch

def diagnostic_metrics(logits, labels):
    predictions = logits.argmax(dim=1)
    confusion = ...
    return {"accuracy": ...,
            "confidence": logits.softmax(dim=1).amax(dim=1),
            "confusion": confusion,
            "predictions": predictions}

def E09(model, fixed_batch, loss_fn):
    logits = model(fixed_batch[0])
    loss = loss_fn(logits, fixed_batch[1])
    report = diagnostic_metrics(logits, fixed_batch[1])
    return loss.item(), report, ...
```

14.6.11 Exercise 10 - Train CNN on CUDA with Correct DataLoader Transfer and Pinned Memory

Part 1 - Why this matters

Efficient CUDA training requires model placement, pinned host buffers, non-blocking copies, and enough input-pipeline throughput. Pinning does not move data to the GPU and may not help when the source or workload is unsuitable.

Configure a DataLoader, transfer each batch to the selected device, and verify parameter, buffer, input, target, and output devices. Benchmark blocking and non-blocking paths under controlled settings.

Part 2 - Mathematical form

$$X_{\text{CPU,pinned}} \xrightarrow{\text{non-blocking}} X_{\text{CUDA}}.$$

Part 3 - Hints and Helper

Enable `pin_memory` only for CUDA use, and measure a full pipeline rather than timing an asynchronous copy without synchronization.

Part 4 - Python code skeleton

```
import torch

def cuda_epoch(model, loader, optimizer, loss_fn, device):
    model.train()
    for images, labels in loader:
        images = images.to(device, non_blocking=True)
        labels = labels.to(device, non_blocking=True)
        optimizer.zero_grad(set_to_none=True)
        logits = model(images)
        loss = loss_fn(logits, labels)
        loss.backward(); optimizer.step()
        assert logits.device == device
    return ...

def E10(dataset, model):
    loader = torch.utils.data.DataLoader(dataset, batch_size=...,
        shuffle=True, num_workers=..., pin_memory=True)
    return cuda_epoch(model.cuda(), loader, ..., ..., torch.device("cuda"))
```

14.6.12 Exercise 11 - Profile GPU Utilization and Fix Training Under-Utilization Bottlenecks

Part 1 - Why this matters

Low utilization may come from slow loading, CPU augmentation, small batches, synchronization, or insufficient computation. The correct fix depends on whether time is spent waiting for data, forward, backward, or optimizer work.

Instrument the training step into data-wait and compute intervals, profile representative batches, and change one pipeline factor at a time. Report throughput and utilization before and after the measured fix.

Part 2 - Mathematical form

$$t_{\text{step}} = t_{\text{data}} + t_{\text{transfer}} + t_{\text{forward}} + t_{\text{backward}} + t_{\text{update}}.$$

Part 3 - Hints and Helper

Warm up first and avoid unnecessary item calls or synchronizations inside the hot loop.

Part 4 - Python code skeleton

```
import time
import torch

def profiled_epoch(model, loader, optimizer, loss_fn):
    records = []
    previous_end = time.perf_counter()
    for images, labels in loader:
        data_wait = time.perf_counter() - previous_end
        ...
        records.append({"data_wait": data_wait,
                        "transfer": ..., "forward": ...,
                        "backward": ..., "update": ...})
        previous_end = time.perf_counter()
    return records

def E11(configurations):
    return [...]
```

14.6.13 Exercise 12 - Implement Mixed Precision CNN Training and Detect Silent Overflow or Underflow

Part 1 - Why this matters

Autocast selects lower precision for suitable operations, while gradient scaling protects small gradients from underflow. A skipped optimizer update or falling scale may reveal overflow even when no visible exception occurs.

Implement mixed precision with the current PyTorch scaling API, log scale changes, skipped steps, finite gradients, memory, and throughput, and compare training quality with float32.

Part 2 - Mathematical form

$$L_s = sL, \qquad \nabla L = s^{-1}\nabla L_s.$$

Part 3 - Hints and Helper

Unscale before clipping or gradient inspection. Check whether the optimizer actually advanced when overflow is detected.

Part 4 - Python code skeleton

```
import torch

def mixed_step(model, batch, optimizer, loss_fn, scaler):
```

```
    optimizer.zero_grad(set_to_none=True)
    old_scale = scaler.get_scale()
    with torch.autocast("cuda", dtype=torch.float16):
        loss = loss_fn(model(batch[0]), batch[1])
    scaler.scale(loss).backward()
    scaler.unscale_(optimizer)
    gradient_finite = ...
    scaler.step(optimizer); scaler.update()
    new_scale = scaler.get_scale()
    return {"loss": loss.item(), "old_scale": old_scale,
            "new_scale": new_scale, "possible_skip": ...,
            "gradient_finite": gradient_finite}

def E12(training_inputs):
    return [...]
```

14.6.14 Exercise 13 - Validate Determinism Options and Explain CNN Training Non-Determinism

Part 1 - Why this matters

Reproducible CNN training depends on seeds, data order, augmentation workers, dropout, backend algorithms, floating-point reduction order, and hardware. One seed does not control all sources.

Run repeated experiments under strict and relaxed settings, compare batches, initial states, losses, final parameters, and metrics, and record the environment and performance cost of deterministic algorithms.

Part 2 - Mathematical form

$$\Delta_\theta = \max_i |\theta_i^{(1)} - \theta_i^{(2)}|.$$

Part 3 - Hints and Helper

Control DataLoader generators and worker seeds, and distinguish repeatability on one system from reproducibility across systems.

Part 4 - Python code skeleton

```
import random
import torch

def configure(seed, deterministic):
    random.seed(seed); torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.use_deterministic_algorithms(deterministic)
```

```
    ...

def one_run(seed, deterministic, factory, loader_factory):
    configure(seed, deterministic)
    model = factory().cuda()
    history = ...
    return history, {k: v.cpu() for k, v in model.state_dict().items()}

def E13(repeats, seed):
    runs = [one_run(seed, True, ..., ...) for _ in range(repeats)]
    return {...}
```

14.6.15 Exercise 14 - Implement Checkpoint Resume with Optimizer and Scheduler State Preservation

Part 1 - Why this matters

A training checkpoint must preserve more than model weights. Optimizer moments, scheduler position, scaler state, epoch, best metric, and random states influence whether resumed training continues consistently.

Save atomically, load into freshly constructed objects, restore all states, and compare a continuous run with an interrupted-and-resumed run. Validate keys and devices before continuing.

Part 2 - Mathematical form

$$C_t = (\theta_t, O_t, S_t, G_t, R_t, t).$$

Part 3 - Hints and Helper

Write to a temporary path and rename after success. Call optimizer state loading after constructing it for the restored model.

Part 4 - Python code skeleton

```
import torch

def save_checkpoint(path, model, optimizer, scheduler, scaler, epoch):
    state = {"model": model.state_dict(),
             "optimizer": optimizer.state_dict(),
             "scheduler": scheduler.state_dict(),
             "scaler": scaler.state_dict() if scaler else None,
             "epoch": epoch, "rng": torch.get_rng_state(),
             "cuda_rng": torch.cuda.get_rng_state_all()}
    temporary = str(path) + ".tmp"
    torch.save(state, temporary)
    ...
```

```
def load_checkpoint(path, model, optimizer, scheduler, scaler):
    state = torch.load(path, map_location=...)
    ...
    return state["epoch"]

def E14(continuous_run, resumed_run):
    return {...}
```

14.6.16 Exercise 15 - Benchmark Training Throughput Across Batch Sizes and Find the Best Trade-Off

Part 1 - Why this matters

Larger batches can improve GPU utilization and samples per second but consume more memory and change gradient noise. The throughput-optimal batch may not provide the best convergence or validation result.

Benchmark forward, backward, and update time after warm-up across feasible batch sizes. Record peak memory, throughput, loss progress per sample, and failure point, then select according to an explicit objective.

Part 2 - Mathematical form

$$\text{throughput}(B) = B/t_{\text{step}}(B), \qquad \text{efficiency} = \text{throughput}/M_{\text{peak}}.$$

Part 3 - Hints and Helper

Keep model, precision, workers, and input shape fixed. Synchronize CUDA timing and leave memory headroom.

Part 4 - Python code skeleton

```
import torch

def benchmark_batch(model, batch_size, sample_shape, repeats):
    torch.cuda.reset_peak_memory_stats()
    times = []
    for _ in range(repeats):
        batch = ...
        torch.cuda.synchronize(); start = ...
        ...
        torch.cuda.synchronize(); times.append(...)
    return {"batch": batch_size, "throughput": ...,
            "peak_memory": torch.cuda.max_memory_allocated(),
            "median_step": ...}

def E15(batch_sizes):
    results = []
```

```
    for size in batch_sizes:
        try: results.append(benchmark_batch(...))
        except torch.cuda.OutOfMemoryError: break
    return results, ...
```

14.6.17 Exercise 16 - Implement Gradient Accumulation for Memory-Constrained Large-Batch Training

Part 1 - Why this matters

Gradient accumulation divides one effective batch into micro-batches and delays the optimizer update. Correct scaling must preserve a sample-weighted mean, especially when the final micro-batch has a different size.

Compare one full-batch step with accumulated micro-batches from cloned models. Disable stochastic and batch-dependent differences during strict equivalence testing, then measure memory savings.

Part 2 - Mathematical form

$$g_{\text{eff}} = \sum_k \frac{n_k}{\sum_j n_j} g_k.$$

Part 3 - Hints and Helper

Clear gradients once per effective batch and update once after all contributions. Weight by samples rather than blindly dividing by micro-batch count.

Part 4 - Python code skeleton

```
import torch

def accumulated_step(model, micro_batches, optimizer, loss_fn):
    optimizer.zero_grad(set_to_none=True)
    total_samples = sum(labels.shape[0] for _, labels in micro_batches)
    for images, labels in micro_batches:
        loss = loss_fn(model(images), labels)
        scaled_loss = loss * labels.shape[0] / total_samples
        scaled_loss.backward()
    optimizer.step()
    return ...

def E16(base_model, full_batch, micro_batches):
    full_model = ...
    accumulated_model = ...
    ...
    return {"parameter_error": ..., "gradient_error": ...,
            "memory_reduction": ...}
```

14.6.18 Exercise 17 - Build Debug Mode That Saves Misclassified Images and Confidence Scores

Part 1 - Why this matters

Aggregate metrics reveal that errors exist but not what the model is confusing. Saving misclassified samples with labels, predictions, confidence, and identifiers supports visual and subgroup analysis.

Create an evaluation-only debug mode that reverses display normalization, limits storage, avoids duplicates, and writes metadata beside image artifacts. Group errors by true and predicted class.

Part 2 - Mathematical form

$$c_i = \max_k \operatorname{softmax}(z_i)_k, \qquad \mathcal{E} = \{i : \widehat{y}_i \neq y_i\}.$$

Part 3 - Hints and Helper

Preserve sample identifiers and never use augmented training views as if they were original evidence unless explicitly marked.

Part 4 - Python code skeleton

```
from pathlib import Path
import torch

def collect_errors(model, loader, device, maximum):
    model.eval(); errors = []
    with torch.inference_mode():
        for images, labels, identifiers in loader:
            logits = model(images.to(device))
            probabilities = logits.softmax(dim=1)
            predictions = probabilities.argmax(dim=1)
            for index in torch.nonzero(predictions.cpu() != labels).flatten():
                errors.append({...})
                if len(errors) >= maximum: return errors
    return errors

def E17(model, loader, output_directory):
    errors = collect_errors(model, loader, ..., ...)
    Path(output_directory).mkdir(...)
    ...
    return errors
```

14.6.19 Exercise 18 - Validate Training by Reaching Near-Perfect Accuracy on a Simple Synthetic Task

Part 1 - Why this matters

A synthetic task with a known visual rule isolates the learning pipeline from uncertain real data. A small CNN should solve a clean separable pattern task; failure indicates an implementation or optimization problem.

Generate labeled images from a deterministic rule, split without leakage, train, and require near-perfect train and validation accuracy. Add controlled noise only after the clean task succeeds.

Part 2 - Mathematical form

$$y = \mathbf{1}[\text{known pattern is present in } X].$$

Part 3 - Hints and Helper

Visualize samples and verify labels before training. Keep train and validation generators independent.

Part 4 - Python code skeleton

```
import torch

def make_synthetic_dataset(count, image_size, seed):
    generator = torch.Generator().manual_seed(seed)
    images = torch.zeros(...)
    labels = torch.randint(..., generator=generator)
    for index, label in enumerate(labels):
        if label == 1: ...
        else: ...
    return torch.utils.data.TensorDataset(images, labels)

def E18(model_factory):
    train_data = make_synthetic_dataset(..., seed=...)
    validation_data = make_synthetic_dataset(..., seed=...)
    model = model_factory()
    history = ...
    return {"history": history, "passed": ...,
            "train_accuracy": ..., "validation_accuracy": ...}
```

14.6.20 Exercise 19 - Produce a Full CNN Training Debugging Playbook Covering Common Failure Scenarios

Part 1 - Why this matters

CNN training failures span data, shapes, losses, gradients, optimization, precision, performance, reproducibility, checkpointing, and evaluation. A playbook converts these recurring failures into an ordered diagnostic process.

Write symptom-driven procedures for no learning, NaN loss, low accuracy, overfitting, vanishing or exploding gradients, CUDA under-utilization, out-of-memory, mixed-precision skips, nondeterminism, and resume drift. Link each procedure to evidence from earlier exercises.

Part 2 - Mathematical form

$$\text{symptom} \rightarrow \text{measurement} \rightarrow \text{hypothesis} \rightarrow \text{controlled test} \rightarrow \text{fix} \rightarrow \text{regression test}.$$

Part 3 - Hints and Helper

Begin with one batch and one tiny dataset before tuning the full run. Every fix needs a test that would detect recurrence.

Part 4 - Python code skeleton

```
from pathlib import Path

REQUIRED = ["Data", "Forward Pass", "Loss", "Gradients", "Optimizer",
            "CUDA", "Mixed Precision", "Checkpoints", "Evaluation",
            "Reproducibility", "Regression Tests"]

def validate_playbook(text, evidence):
    missing = [name for name in REQUIRED if name not in text]
    return {"missing": missing, "evidence_links": ...,
            "ready": len(missing) == 0 and ...}

def E19(experiment_records, output_path):
    playbook = ...
    checks = validate_playbook(playbook, experiment_records)
    Path(output_path).write_text(playbook, encoding="utf-8")
    return checks
```

14.7 CNN05 - Data Pipeline and Augmentation for Vision

Models and Training

14.7.1 Exercise 00 - Implement Image Dataset Loading with Correct Normalization and Tensor Conversion

Part 1 - Why this matters

Image loading must produce consistent channel-first floating tensors, labels, and identifiers. Incorrect ranges, channel order, alpha handling, or normalization can silently change the learning problem.

Build a Dataset that decodes files, validates image mode and size, converts values to a documented range, and applies channel-wise normalization. Test grayscale, RGB, corrupt, and unusual-sized samples.

Part 2 - Mathematical form

$$X'_{c,h,w} = \frac{X_{c,h,w} - \mu_c}{\sigma_c}.$$

Part 3 - Hints and Helper

Calculate statistics from training data only and return a stable sample identifier for debugging.

Part 4 - Python code skeleton

```
class ImageDataset(torch.utils.data.Dataset):
    def __init__(self, records, transform): ...
    def __len__(self): return ...
    def __getitem__(self, index):
        image, label, identifier = ...
        tensor = ...
        if tensor.ndim != 3: raise ValueError(...)
        return tensor, label, identifier

def E00(records):
    dataset = ImageDataset(records, ...)
    return dataset, [...\]
```

14.7.2 Exercise 01 - Validate Channel Ordering and Normalization by Visualizing a Preprocessed Batch

Part 1 - Why this matters

A tensor may have the expected shape while its colors or scale are wrong. Visual inspection after reversing normalization exposes swapped channels, clipping, unexpected value ranges, and transforms that destroy semantic content.

Load one batch, report per-channel statistics, denormalize a copy, convert it to display layout, and create a labeled grid. Verify that displayed images correspond to their source identifiers.

Part 2 - Mathematical form

$$X_{\text{display}} = \text{clip}(X'\sigma + \mu, 0, 1).$$

Part 3 - Hints and Helper

Never modify the training batch in place during denormalization, and permute from channel-first to channel-last for display.

Part 4 - Python code skeleton

```
def denormalize(batch, mean, std):
    mean = batch.new_tensor(mean).view(1, -1, 1, 1)
    std = batch.new_tensor(std).view(1, -1, 1, 1)
    return ...

def E01(batch, labels, identifiers, mean, std):
    restored = denormalize(batch.clone(), mean, std)
    statistics = {...}
    figure = ...
    return figure, statistics
```

14.7.3 Exercise 02 - Implement Train and Test Splits and Confirm No Data Leakage

Part 1 - Why this matters

Leakage occurs when the same source, duplicate, subject, time group, or derived image appears across splits. It creates optimistic evaluation even when file names differ.

Split with a fixed seed and group-aware rule, then compare identifiers, content hashes, and relevant metadata across partitions. Fit normalization and augmentation decisions from the training split only.

Part 2 - Mathematical form

$$D_{\text{train}} \cap D_{\text{test}} = \varnothing.$$

Part 3 - Hints and Helper

Define the leakage unit before splitting; for medical or video data it may be a patient or sequence rather than one image.

Part 4 - Python code skeleton

```
def split_records(records, test_fraction, seed, group_key):
    groups = ...
    train_groups, test_groups = ...
    return ..., ...

def leakage_report(train_records, test_records):
    id_overlap = ...
    hash_overlap = ...
```

```
    group_overlap = ...
    return {"ids": id_overlap, "hashes": hash_overlap,
            "groups": group_overlap, "passed": ...}

def E02(records):
    train, test = split_records(records, ..., ..., ...)
    return train, test, leakage_report(train, test)
```

14.7.4 Exercise 03 - Implement Random Crop and Horizontal Flip and Measure Generalization Impact

Part 1 - Why this matters

Crop and flip encode assumptions that partial views and horizontal reflection should preserve the label. They can improve generalization only when those transformations are valid for the dataset.

Implement stochastic transforms for training and deterministic preprocessing for validation. Compare controlled runs, visualize transformed samples, and measure accuracy and failure categories.

Part 2 - Mathematical form

$$\widetilde{X} = T(X), \qquad y(T(X)) = y(X) \text{ must be a valid assumption.}$$

Part 3 - Hints and Helper

Seed comparison runs, keep the validation pipeline fixed, and check classes whose meaning changes under reflection.

Part 4 - Python code skeleton

```
def build_transforms(training):
    if training:
        return ...  # crop, flip, tensor, normalize
    return ...      # deterministic resize, tensor, normalize

def E03(dataset, seeds):
    baseline = ...
    augmented = ...
    examples = ...
    return {"baseline": baseline, "augmented": augmented,
            "examples": examples, "difference": ...}
```

14.7.5 Exercise 04 - Implement Color Jitter and Analyze When It Helps or Hurts Specific Datasets

Part 1 - Why this matters

Color jitter changes brightness, contrast, saturation, or hue and can discourage reliance on incidental lighting. It harms tasks where exact color is a label-defining signal or when transformed colors become unrealistic.

Apply controlled jitter strengths to color-dependent and color-invariant tasks. Inspect class-conditional examples and compare clean accuracy, shifted-light accuracy, calibration, and confusion patterns.

Part 2 - Mathematical form

$$\widetilde{X} = T_{b,c,s,h}(X).$$

Part 3 - Hints and Helper

Begin with plausible parameter ranges and test one color component at a time before combining them.

Part 4 - Python code skeleton

```
def color_jitter_experiment(dataset, strengths, seed):
    results = []
    for strength in strengths:
        transform = ...
        examples = ...
        metrics = ...
        results.append({"strength": strength, "metrics": metrics,
                        "examples": examples})
    return results

def E04(datasets):
    return {...}
```

14.7.6 Exercise 05 - Implement Random Rotation and Analyze Its Effect on Model Invariance

Part 1 - Why this matters

Rotation is appropriate when object identity is orientation-independent, but it can change labels for digits, directional signs, text, or gravity-dependent scenes. Interpolation and empty corners also introduce artifacts.

Compare rotation ranges and fill strategies on tasks with different orientation semantics. Measure accuracy by angle and inspect whether artifacts rather than objects become predictive.

Part 2 - Mathematical form

$$T_\theta(X)(p) = X(R_{-\theta}p).$$

Part 3 - Hints and Helper

Use label-preserving angles, document interpolation, and visualize boundary artifacts before training.

Part 4 - Python code skeleton

```
def rotation_suite(image, angles, interpolation, fill):
    outputs = []
    for angle in angles:
        outputs.append((angle, ...))
    return outputs

def E05(train_data, validation_by_angle):
    model = ...
    history = ...
    angle_metrics = {...}
    return history, angle_metrics, rotation_suite(...)
```

14.7.7 Exercise 06 - Implement Random Erasing or Cutout and Evaluate Its Regularization Effect

Part 1 - Why this matters

Random erasing hides a region and encourages the model to use distributed evidence rather than one highly predictive patch. Excessive occlusion can remove the object or create unrealistic training samples.

Implement area and aspect-ratio sampling, visualize masks, and compare robustness to occlusion with clean accuracy. Track how often the erased region overlaps the task-defining feature in synthetic data.

Part 2 - Mathematical form

$$\widetilde{X} = X \odot (1 - M) + vM.$$

Part 3 - Hints and Helper

Bound the number of sampling attempts and preserve the original image when no valid rectangle is found.

Part 4 - Python code skeleton

```
def random_erasing(image, generator, scale, ratio, fill):
    height, width = image.shape[-2:]
    for _ in range(...):
        erase_h, erase_w = ...
        if erase_h < height and erase_w < width:
            output = image.clone(); output[..., ...\] = fill
            return output, ...
    return image, None

def E06(dataset):
    return {"examples": ..., "clean_metric": ...,
            "occlusion_metric": ...}
```

14.7.8 Exercise 07 - Implement MixUp or CutMix and Compare with Standard Augmentation

Part 1 - Why this matters

MixUp combines images and soft labels globally, while CutMix inserts one image region into another and weights labels by visible area. Both change the target representation and therefore require compatible loss computation.

Implement one method, verify lambda and label mixing, visualize batches, and compare calibration, robustness, and accuracy with standard augmentation under the same training budget.

Part 2 - Mathematical form

$$\widetilde{X} = \lambda X_a + (1-\lambda)X_b, \quad \widetilde{y} = \lambda y_a + (1-\lambda)y_b.$$

Part 3 - Hints and Helper

Apply the same permutation to images and labels, and recompute CutMix lambda from the clipped rectangle area.

Part 4 - Python code skeleton

```
def mixup(images, labels, alpha, generator):
    permutation = ...
    lam = ...
    mixed_images = ...
    return mixed_images, labels, labels[permutation], lam

def mixed_loss(loss_fn, logits, y_a, y_b, lam):
    return ...

def E07(batch):
    mixed = mixup(*batch, ..., ...)
    return mixed, ...
```

14.7.9 Exercise 08 - Optimize DataLoader with num_workers and pin_memory and Benchmark It

Part 1 - Why this matters

Worker count, persistent workers, prefetching, pinning, storage speed, and transform cost determine whether data arrives before the GPU needs it. More workers can eventually add contention instead of speed.

Benchmark complete epochs across controlled DataLoader configurations. Record throughput, startup time, steady-state batch wait, CPU memory, failures, and GPU idle time.

Part 2 - Mathematical form

$$\text{loader throughput} = N_{\text{samples}}/t_{\text{epoch}}.$$

Part 3 - Hints and Helper

Run multiple epochs to separate worker startup from steady state, and change one setting at a time.

Part 4 - Python code skeleton

```
def benchmark_loader(dataset, config, epochs):
    loader = torch.utils.data.DataLoader(dataset, **config)
    epoch_times, batch_waits = [], []
    for epoch in range(epochs):
        start = ...
        for batch in loader: ...
        epoch_times.append(...)
    return {"config": config, "epoch_times": epoch_times,
            "throughput": ..., "batch_waits": batch_waits}

def E08(dataset, configurations):
    return [benchmark_loader(dataset, c, ...) for c in configurations]
```

14.7.10 Exercise 09 - Debug Data Pipeline Bugs Causing Wrong Labels or Broken Image Tensors

Part 1 - Why this matters

A pipeline may pair images with wrong labels, normalize twice, return inconsistent shapes, mutate cached samples, or fail only on one corrupt record. Batch-level errors often hide the responsible source item.

Add validation at dataset and collate boundaries, preserve identifiers, and build a debug loader that isolates failing samples. Check label range, tensor contract, determinism expectations, and image-label correspondence.

Part 2 - Mathematical form

$$\text{valid}(x, y) = \text{shape ok} \wedge \text{finite}(x) \wedge y \in \{0, \ldots, K-1\}.$$

Part 3 - Hints and Helper

Inspect samples with shuffling disabled first and report the exact index and path on failure.

Part 4 - Python code skeleton

```
def validate_sample(sample, class_count):
    image, label, identifier = sample
    if image.ndim != 3 or not torch.isfinite(image).all():
        raise ValueError(identifier)
    if not 0 <= int(label) < class_count: raise ValueError(identifier)
    return True

def E09(dataset, class_count):
    failures = []
    for index in range(len(dataset)):
        try: validate_sample(dataset[index], class_count)
        except Exception as error: failures.append({...})
    return failures
```

14.7.11 Exercise 10 - Move the Data Pipeline to CUDA Efficiently and Measure Transfer Overhead

Part 1 - Why this matters

Input tensors normally remain on CPU until each batch is transferred. Efficient overlap requires pinned memory, non-blocking copies, suitable batch sizes, and enough compute to hide transfer latency.

Measure synchronous and asynchronous host-to-device paths with correct synchronization. Separate decoding, collation, transfer, and model time and verify that targets and inputs reach the same device.

Part 2 - Mathematical form

$$t_{\text{step}} = t_{\text{load}} + t_{\text{H2D}} + t_{\text{compute}} - t_{\text{overlap}}.$$

Part 3 - Hints and Helper

Never time an asynchronous copy without synchronizing its completion, and avoid moving data to CUDA inside worker processes.

Part 4 - Python code skeleton

```
def transfer_batch(batch, device, non_blocking):
    images, labels = batch
    return (images.to(device, non_blocking=non_blocking),
            labels.to(device, non_blocking=non_blocking))

def benchmark_transfer(loader, device, non_blocking):
    records = []
    for batch in loader:
        torch.cuda.synchronize(); start = ...
        moved = transfer_batch(batch, device, non_blocking)
        torch.cuda.synchronize(); records.append(...)
    return {...}

def E10(loader): return benchmark_transfer(...), benchmark_transfer(...)
```

14.7.12 Exercise 11 - Implement Prefetching and Compare It with a Baseline DataLoader

Part 1 - Why this matters

Prefetching prepares a future batch while the current batch is being computed, reducing visible data wait when transfer and compute can overlap. Incorrect stream synchronization can expose incomplete tensors or add overhead.

Implement a CUDA prefetcher with a dedicated stream, compare it with the same baseline loader, and verify output parity, ordering, throughput, and memory.

Part 2 - Mathematical form

$$t_{\text{pipeline}} \approx \max(t_{\text{prepare}}, t_{\text{compute}}) \text{ under effective overlap.}$$

Part 3 - Hints and Helper

Make the current stream wait for the prefetch stream before consuming a batch and benchmark after warm-up.

Part 4 - Python code skeleton

```
class CUDAPrefetcher:
    def __init__(self, loader, device):
        self.iterator = iter(loader); self.device = device
        self.stream = torch.cuda.Stream(); self.next_batch = None
        self.preload()
    def preload(self): ...
    def next(self): ...
```

```
def E11(loader, model):
    baseline = ...
    prefetched = ...
    return {"baseline": baseline, "prefetched": prefetched,
            "parity": ...}
```

14.7.13 Exercise 12 - Analyze Image Resolution Trade-Offs in Accuracy, Speed, and GPU Memory

Part 1 - Why this matters

Higher resolution can preserve small objects and fine boundaries but increases activation area, computation, transfer, and memory. Resizing may also distort aspect ratio or remove details before the model sees them.

Train or evaluate controlled resolution variants, recording accuracy by object size, throughput, peak memory, and preprocessing cost. Keep architecture adaptation and crop policy explicit.

Part 2 - Mathematical form

$$\text{activation area} \propto HW, \qquad \text{conv cost} \propto HWC_{\text{in}}C_{\text{out}}K^2.$$

Part 3 - Hints and Helper

Use the same split and evaluation protocol and inspect small-object failures rather than reporting only overall accuracy.

Part 4 - Python code skeleton

```
def resolution_experiment(resolution, model_factory, loaders):
    model = model_factory(resolution)
    history = ...
    torch.cuda.reset_peak_memory_stats()
    metrics = ...
    return {"resolution": resolution, "metrics": metrics,
            "throughput": ..., "peak_memory": ...,
            "small_object_metric": ...}

def E12(resolutions):
    return [resolution_experiment(r, ..., ...) for r in resolutions]
```

14.7.14 Exercise 13 - Detect Augmentation Bugs That Destroy Class Semantics or Introduce Label Noise

Part 1 - Why this matters

An augmentation is valid only if the transformed sample still supports its assigned target. Direction changes, cropped-away objects, impossible colors, or geometric label mismatches can silently inject supervision errors.

Define class-specific invariance rules, run deterministic transform audits, and compare labels before and after transformations using metadata or synthetic ground truth. Save violations for review.

Part 2 - Mathematical form

$$y(T(X)) = y(X) \text{ is an assumption that must be tested.}$$

Part 3 - Hints and Helper

Audit each transform alone before composing it, and record random parameters so failures can be reproduced.

Part 4 - Python code skeleton

```
def audit_transform(sample, transform, semantic_validator, trials):
    violations = []
    for trial in range(trials):
        transformed, parameters = ...
        if not semantic_validator(sample, transformed, parameters):
            violations.append({...})
    return violations

def E13(dataset, transform_specs):
    report = {}
    for name, transform in transform_specs.items():
        report[name] = [...\]
    return report
```

14.7.15 Exercise 14 - Implement Balanced Sampling for Imbalanced Image Datasets

Part 1 - Why this matters

Random sampling follows the observed class distribution and may provide few minority examples per epoch. Weighted sampling can rebalance exposure, but it does not create new information and may increase overfitting to repeated minority samples.

Compute sample weights from training labels, build a weighted sampler, and compare observed batch and epoch distributions. Evaluate per-class recall and duplication frequency rather than only overall accuracy.

Part 2 - Mathematical form

$$w_i \propto 1/n_{y_i}.$$

Part 3 - Hints and Helper

Calculate weights from the training split only and decide whether sampling is with replacement.

Part 4 - Python code skeleton

```
def sample_weights(labels):
    counts = torch.bincount(labels)
    class_weights = ...
    return class_weights[labels]

def E14(dataset, labels, batch_size):
    weights = sample_weights(labels)
    sampler = torch.utils.data.WeightedRandomSampler(... )
    loader = torch.utils.data.DataLoader(dataset, batch_size=batch_size,
                                         sampler=sampler)
    observed = ...
    return loader, {"expected": ..., "observed": observed,
                    "duplication": ...}
```

14.7.16 Exercise 15 - Build an Augmentation Ablation Study and Identify What Actually Helps

Part 1 - Why this matters

Combining many augmentations can improve, harm, or mask each other's effects. An ablation study isolates contributions under a shared training budget and prevents conclusions based on one lucky run.

Define a baseline and controlled additions or removals, repeat seeds, and report mean and variation for clean, shifted, and per-class metrics. Preserve exact configurations and transformed examples.

Part 2 - Mathematical form

$$\Delta_m = q(\mathcal{T}) - q(\mathcal{T} \setminus \{m\}).$$

Part 3 - Hints and Helper

Change one planned factor per comparison and keep validation preprocessing identical for all runs.

Part 4 - Python code skeleton

```
def controlled_run(config, seed):
    model = ...; history = ...; metrics = ...
    return {"config": config, "seed": seed,
            "history": history, "metrics": metrics}

def E15(configurations, seeds):
    records = []
    for config in configurations:
        for seed in seeds:
            records.append(controlled_run(config, seed))
    summary = ...
    return records, summary
```

14.7.17 Exercise 16 - Compare Per-Dataset and Per-Image Normalization Strategies

Part 1 - Why this matters

Per-dataset normalization preserves relative brightness differences using training-set statistics. Per-image normalization removes each image's global scale and may help some acquisition shifts while destroying absolute intensity information.

Implement both strategies with numerical safeguards and compare channel distributions, accuracy, calibration, and performance under lighting shifts. Prevent validation statistics from influencing dataset normalization.

Part 2 - Mathematical form

$$X'_{\text{dataset}} = (X - \mu_D)/\sigma_D, \qquad X'_{\text{image}} = (X - \mu_X)/(\sigma_X + \varepsilon).$$

Part 3 - Hints and Helper

State the axes used for statistics and handle nearly constant images without division instability.

Part 4 - Python code skeleton

```
def dataset_normalize(x, mean, std): return ...
def image_normalize(x, epsilon):
    mean = x.mean(dim=(-2, -1), keepdim=True)
    std = x.std(dim=(-2, -1), keepdim=True)
    return ...

def E16(train_data, shifted_validation):
    training_stats = ...
    variants = {"dataset": ..., "per_image": ...}
    return {name: ... for name in variants}
```

14.7.18 Exercise 17 - Create a Data Quality Report for Corruption, Duplicates, and Outliers

Part 1 - Why this matters

A training-ready dataset requires readable files, valid dimensions and channels, consistent labels, controlled duplicates, and an understood distribution. Corruption or duplicates can bias training and leak into evaluation.

Scan every record safely, compute exact and perceptual fingerprints, summarize dimensions and value statistics, flag outliers, and separate confirmed errors from samples needing human review.

Part 2 - Mathematical form

$$z_i = (s_i - \text{median}(s))/\,\text{MAD}(s).$$

Part 3 - Hints and Helper

Never delete automatically from a heuristic flag. Preserve paths, identifiers, reasons, and thresholds in the report.

Part 4 - Python code skeleton

```
def inspect_record(record):
    try:
        image = ...
        return {"id": ..., "readable": True, "shape": ...,
                "hash": ..., "perceptual_hash": ...,
                "statistics": ...}
    except Exception as error:
        return {"id": ..., "readable": False, "error": str(error)}

def E17(records, output_path):
    inspections = [inspect_record(r) for r in records]
    duplicates = ...; outliers = ...; summary = ...
    ...
    return summary
```

14.7.19 Exercise 18 - Stress Test DataLoader Throughput and Debug Crashes or Deadlocks

Part 1 - Why this matters

High worker counts, corrupt decoders, excessive shared memory, non-picklable transforms, and worker exceptions can cause stalls or intermittent failure. A stress test must preserve

enough context to reproduce the last successful batch.

Run long controlled iterations across worker settings with timeouts, deterministic identifiers, memory monitoring, and exception capture. Distinguish slow batches from a true deadlock and isolate failing records with a single worker.

Part 2 - Mathematical form

$$t_{\text{wait},b} > \tau \;\Rightarrow\; \text{investigate stall rather than assume deadlock immediately.}$$

Part 3 - Hints and Helper

Begin with zero workers for correctness, then increase gradually; avoid CUDA operations inside workers.

Part 4 - Python code skeleton

```
def stress_loader(loader, maximum_batches, stall_threshold):
    records = []; previous = ...
    for index, batch in enumerate(loader):
        wait = ...
        records.append({"batch": index, "wait": wait,
                        "identifiers": ...})
        if wait > stall_threshold: ...
        if index + 1 >= maximum_batches: break
    return records

def E18(dataset, configurations):
    results = []
    for config in configurations:
        try: results.append(stress_loader(...))
        except Exception as error: results.append({...})
    return results
```

14.7.20 Exercise 19 - Produce a Practical Vision Data Pipeline Checklist for Training-Ready Datasets

Part 1 - Why this matters

A reliable pipeline joins data provenance, split integrity, decoding, labels, preprocessing, augmentation semantics, sampling, performance, reproducibility, and monitoring. A checklist converts these requirements into evidence before expensive training begins.

Create gates for leakage, corrupt data, tensor contracts, normalization, augmentation audits, class balance, loader stability, throughput, CUDA transfer, and reproducible configuration. Link every pass condition to a generated report or test.

Part 2 - Mathematical form

$$\text{ready} = \bigwedge_i \text{gate}_i(\text{evidence}).$$

Part 3 - Hints and Helper

Use pass, fail, and needs-review states; do not mark a gate complete without an artifact or measured threshold.

Part 4 - Python code skeleton

```
from pathlib import Path

REQUIRED = ["Provenance", "Split Integrity", "Labels",
            "Tensor Contract", "Normalization", "Augmentation",
            "Class Balance", "Performance", "Reproducibility"]

def validate_checklist(text, evidence):
    missing = [name for name in REQUIRED if name not in text]
    failed_gates = ...
    return {"missing": missing, "failed": failed_gates,
            "ready": not missing and not failed_gates}

def E19(reports, output_path):
    checklist = ...
    result = validate_checklist(checklist, reports)
    Path(output_path).write_text(checklist, encoding="utf-8")
    return result
```

14.8 CNN06 - Regularization, Normalization, and Robustness

Models and Training

14.8.1 Exercise 00 - Demonstrate CNN Overfitting Without Regularization and Observe the Generalization Gap

Part 1 - Why this matters

A high-capacity CNN can continue improving on training data while validation loss worsens. This gap separates memorization from reliable performance on unseen samples.

Train an unregularized model on a small dataset, log train and validation curves, and identify the first epoch where the gap expands persistently.

Part 2 - Mathematical form

$$G_t = L_{\text{val},t} - L_{\text{train},t}.$$

Part 3 - Hints and Helper

Keep the split fixed and inspect both loss and accuracy; one metric alone can hide the pattern.

Part 4 - Python code skeleton

```
import torch

def E00(model, data, config):
    baseline = ...
    history = ...
    gap = ...
    return {"history": history, "gap": gap, "overfit_epoch": ...}
```

14.8.2 Exercise 01 - Apply Weight Decay and Quantify Training Stability and Validation Performance

Part 1 - Why this matters

Weight decay discourages unbounded parameter growth and can improve generalization, but excessive decay limits useful capacity. Its effect depends on optimizer, learning rate, and which parameters are decayed.

Sweep controlled decay values and record validation performance, parameter norms, gradient norms, and convergence speed from identical initial states.

Part 2 - Mathematical form

$$\theta_{t+1} = (1 - \eta\lambda)\theta_t - \eta g_t.$$

Part 3 - Hints and Helper

Consider excluding normalization scales and biases, and distinguish decoupled decay from adding an L2 term.

Part 4 - Python code skeleton

```
import torch

def E01(model, data, config):
    for decay in decay_values:
        model = ...
        history = ...
        results.append({"decay": decay, "history": history,
                        "parameter_norm": ...})
    return results
```

14.8.3 Exercise 02 - Compare Dropout in the Classifier Head and Convolutional Blocks

Part 1 - Why this matters

Head dropout regularizes global features, while spatial dropout inside convolutional blocks can remove whole feature channels. Element-wise dropout may create noisy spatial patterns with different effects.

Compare matched probabilities and placements, verify train/eval behavior, and measure activation sparsity, convergence, and validation performance.

Part 2 - Mathematical form

$$\widetilde{A} = A \odot M/(1-p).$$

Part 3 - Hints and Helper

Change only placement in each run and inspect whether early-feature dropout prevents learning.

Part 4 - Python code skeleton

```
import torch

def E02(model, data, config):
    variants = {"head": ..., "block": ..., "spatial": ...}
    return {name: ... for name, model in variants.items()}
```

14.8.4 Exercise 03 - Implement BatchNorm and Explain Its Effect on Optimization and Gradient Flow

Part 1 - Why this matters

BatchNorm standardizes channel activations using batch statistics and learns scale and shift. It can smooth optimization behavior, but it makes training dependent on batch composition and running estimates.

Compare activation and gradient statistics with and without BatchNorm, then inspect running buffers and small-batch behavior.

Part 2 - Mathematical form

$$y = \gamma \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta.$$

Part 3 - Hints and Helper

Place normalization consistently, switch modes correctly, and do not confuse improved optimization with guaranteed regularization.

Part 4 - Python code skeleton

```
import torch

def E03(model, data, config):
    plain, normalized = ...
    plain_stats = ...
    normalized_stats = ...
    return plain_stats, normalized_stats, ...
```

14.8.5 Exercise 04 - Compare BatchNorm and LayerNorm in CNN Settings and Explain When Each Fits

Part 1 - Why this matters

BatchNorm aggregates each channel across batch and spatial positions, while LayerNorm normalizes selected dimensions within each sample. Their behavior differs under small batches, variable distributions, and deployment.

Build matched CNN variants, state the normalized axes, and compare stability across batch sizes and distribution shifts.

Part 2 - Mathematical form

$$\text{BN} : \mu_c \text{ across } N, H, W; \qquad \text{LN} : \mu \text{ within each sample.}$$

Part 3 - Hints and Helper

Verify tensor shape required by LayerNorm and avoid claiming the methods are interchangeable.

Part 4 - Python code skeleton

```
import torch

def E04(model, data, config):
    for batch_size in batch_sizes:
        results.append({"batch": batch_size, "batch_norm": ...,
                        "layer_norm": ...})
    return results
```

14.8.6 Exercise 05 - Diagnose Instability from Wrong Normalization Placement and Fix the Root Cause

Part 1 - Why this matters

Normalization before or after activation changes the distribution seen by the next operation. Duplicate, missing, or dimensionally incorrect normalization can suppress signals or make running statistics misleading.

Create failing placements deliberately, capture layer statistics, locate the first unstable block, and fix one cause at a time.

Part 2 - Mathematical form

$$A_l = \phi(\text{Norm}(W_l A_{l-1})) \text{ is not identical to } \text{Norm}(\phi(W_l A_{l-1})).$$

Part 3 - Hints and Helper

Track mean, variance, zero fraction, gradient norm, and finite status for each placement.

Part 4 - Python code skeleton

```
import torch

def E05(model, data, config):
    records = ...  # hooks for activations and gradients
    first_unstable = ...
    fixed_model = ...
    return records, first_unstable, fixed_model
```

14.8.7 Exercise 06 - Evaluate Robustness to Input Noise and Blurring and Compare Degradation Patterns

Part 1 - Why this matters

Noise corrupts local pixel values while blur removes high-frequency structure. Different degradation curves reveal whether the CNN depends on texture, edges, or broader shapes.

Evaluate fixed severity levels without retraining, preserve clean references, and report overall and per-class performance with confidence changes.

Part 2 - Mathematical form

$$X_{\text{noise}} = X + \sigma\epsilon, \qquad X_{\text{blur}} = h_s \star X.$$

Part 3 - Hints and Helper

Clip to the valid range and apply perturbations before the same normalization used for clean inputs.

Part 4 - Python code skeleton

```
import torch

def E06(model, data, config):
    for severity in severities:
        noisy = ...; blurred = ...
        results.append({"severity": severity, "noise": ..., "blur": ...})
    return results
```

14.8.8 Exercise 07 - Evaluate Random Occlusion and Test Use of Global Context

Part 1 - Why this matters

Occlusion reveals whether predictions depend on one small discriminative region or distributed context. Random masks can also remove the object entirely, so mask position and area must be recorded.

Measure accuracy and confidence across occlusion sizes and locations, then create sensitivity maps and inspect class-specific failures.

Part 2 - Mathematical form

$$\Delta_c(M) = p_c(X) - p_c(X \odot (1 - M) + vM).$$

Part 3 - Hints and Helper

Compare central, random, and sliding masks, and distinguish evidence removal from artificial mask artifacts.

Part 4 - Python code skeleton

```
import torch

def E07(model, data, config):
    for mask in masks:
        occluded = ...
        records.append({"mask": mask, "confidence_change": ...,
                        "correct": ...})
    return records
```

14.8.9 Exercise 08 - Implement Label Smoothing and Measure Overconfidence and Calibration

Part 1 - Why this matters

Label smoothing replaces one-hot certainty with a softened target, often reducing extreme logits. It may improve calibration while slightly changing accuracy or harming tasks requiring sharp separation.

Implement the target distribution manually, validate against PyTorch, and compare negative log-likelihood, confidence, accuracy, and calibration error.

Part 2 - Mathematical form

$$q_k = (1 - \epsilon)\mathbf{1}[k = y] + \epsilon/K.$$

Part 3 - Hints and Helper

Use raw logits and verify how smoothing mass is distributed across classes.

Part 4 - Python code skeleton

```
import torch

def E08(model, data, config):
    smoothed_targets = ...
    loss = ...
    metrics = {"accuracy": ..., "ece": ..., "nll": ...}
    return loss, smoothed_targets, metrics
```

14.8.10 Exercise 09 - Debug Regularization Misconfiguration Causing Underfitting or Failure to Learn

Part 1 - Why this matters

Several strong techniques combined can make a CNN appear broken: excessive decay, dropout, augmentation, smoothing, or early stopping may suppress learning. The symptom can resemble low capacity or optimizer failure.

Build an ordered diagnostic that removes regularizers, overfits a tiny set, and restores one technique at a time while monitoring gradients and train accuracy.

Part 2 - Mathematical form

$$\text{observed failure} = f(\text{capacity, optimization, data, regularization}).$$

Part 3 - Hints and Helper

Establish an unregularized learning baseline before tuning multiple regularizers.

Part 4 - Python code skeleton

```
import torch

def E09(model, data, config):
    baseline = ...
    for regularizer in regularizers:
        diagnostics = ...
        ...
    return {"baseline": baseline, "diagnostics": ...}
```

14.8.11 Exercise 10 - Train Regularized CNN on CUDA and Measure Technique Runtime and Memory Overhead

Part 1 - Why this matters

Regularization methods have different hardware costs. Weight decay mainly affects updates, dropout adds masks, normalization stores statistics and activations, and augmentation may bottleneck the CPU pipeline.

Profile isolated techniques with a matched model, batch, precision, and dataset. Report throughput, peak memory, and validation gain.

Part 2 - Mathematical form

$$\text{overhead} = (t_{\text{regularized}} - t_{\text{base}})/t_{\text{base}}.$$

Part 3 - Hints and Helper

Warm up CUDA, synchronize timing boundaries, and separate data-pipeline cost from model cost.

Part 4 - Python code skeleton

```
import torch

def E10(model, data, config):
    torch.cuda.reset_peak_memory_stats()
    start = ...
    metrics = ...
    return {"time": ..., "peak_memory": ..., "metrics": metrics}
```

14.8.12 Exercise 11 - Compare Mixed Precision Stability With and Without Normalization Layers

Part 1 - Why this matters

Normalization can keep activation scales in a range favorable to mixed precision, yet reductions and small variances may themselves be precision-sensitive. Gradient scaling does not repair non-finite forward values.

Compare matched runs with layer-wise finite checks, scaler history, gradient norms, skipped steps, and validation performance.

Part 2 - Mathematical form

$$\epsilon_l = \|A_l^{16} - A_l^{32}\| / (\|A_l^{32}\| + \varepsilon).$$

Part 3 - Hints and Helper

Unscale gradients before inspection and find the first layer where divergence becomes significant.

Part 4 - Python code skeleton

```
import torch

def E11(model, data, config):
    with torch.autocast("cuda", dtype=torch.float16):
        loss = ...
    ...
    return {"scale_history": ..., "layer_errors": ..., "skipped_steps": ...}
```

14.8.13 Exercise 12 - Perform a Regularization Ablation Study and Identify What Actually Helps

Part 1 - Why this matters

Ablation isolates the contribution of one technique within a fixed experimental context. Interactions mean the value of a method alone may differ from its value inside a combination.

Run repeated seeds for baseline, additions, removals, and selected combinations; summarize clean, robust, calibration, and cost metrics.

Part 2 - Mathematical form

$$\Delta_m = q(\mathcal{R}) - q(\mathcal{R} \setminus \{m\}).$$

Part 3 - Hints and Helper

Predefine comparisons and keep training budget, split, initialization policy, and evaluation identical.

Part 4 - Python code skeleton

```
import torch

def E12(model, data, config):
    records = []
    for config in configurations:
        for seed in seeds:
            records.append(...)
    return records, ...
```

14.8.14 Exercise 13 - Detect Distribution Shift Under Changed Lighting or Color and Measure Generalization Failure

Part 1 - Why this matters

A model may perform well on its source distribution yet fail when illumination, color balance, or sensor response changes. Confidence can remain high even as accuracy falls.

Create controlled shift severities, compare input and feature statistics, and report accuracy, confidence, calibration, and class-specific degradation.

Part 2 - Mathematical form

$$P_{\text{train}}(X) \neq P_{\text{deploy}}(X).$$

Part 3 - Hints and Helper

Use held-out transformations and avoid tuning on the final shift test set.

Part 4 - Python code skeleton

```
import torch

def E13(model, data, config):
    for shift, severity in shift_cases:
        shifted = ...
        results.append({
            "shift": shift,
            "severity": severity,
            "metrics": ...,
            "feature_drift": ...,
        })
    return results
```

14.8.15 Exercise 14 - Implement Test-Time Augmentation and Decide Whether It Adds Robustness or Noise

Part 1 - Why this matters

Test-time augmentation averages predictions from label-preserving views. It can reduce variance but increases latency and fails when a transform changes semantics or produces correlated errors.

Aggregate probabilities or logits consistently, compare single-view and multi-view accuracy, calibration, robustness, and latency, and inspect cases made worse.

Part 2 - Mathematical form

$$p(y \mid X) = M^{-1} \sum_{m=1}^{M} p(y \mid T_m(X)).$$

Part 3 - Hints and Helper

Use deterministic approved transforms and reverse geometric outputs when the task requires spatial predictions.

Part 4 - Python code skeleton

```
import torch

def E14(model, data, config):
    views = [transform(images) for transform in transforms]
    probabilities = [model(view).softmax(dim=1) for view in views]
    combined = torch.stack(probabilities).mean(dim=0)
    return combined, ...
```

14.8.16 Exercise 15 - Evaluate Adversarial-Like Perturbations and Document CNN Sensitivity Patterns

Part 1 - Why this matters

Small gradient-informed or structured perturbations can expose locally sensitive decision boundaries. This exercise studies sensitivity and failure patterns rather than claiming a complete security evaluation.

Generate bounded perturbations, verify their magnitude and valid range, then measure accuracy, confidence, transfer across models, and visual detectability.

Part 2 - Mathematical form

$$X' = \text{clip}(X + \epsilon \, \text{sign}(\nabla_X L)).$$

Part 3 - Hints and Helper

Keep the model in evaluation mode, clear input gradients, and report epsilon in the original pixel scale.

Part 4 - Python code skeleton

```
import torch

def E15(model, data, config):
    images = images.detach().requires_grad_(True)
    loss = loss_fn(model(images), labels); loss.backward()
    perturbed = ...
    return perturbed, {"magnitude": ..., "metrics": ...}
```

14.8.17 Exercise 16 - Build a Robustness Report Grouping Misclassifications by Failure Type and Confidence

Part 1 - Why this matters

One robustness score hides whether failures come from blur, noise, occlusion, shift, small objects, or ambiguous labels. Grouped examples connect aggregate degradation to observable causes.

Store sample identifiers, corruption metadata, labels, predictions, and confidence; produce per-group metrics and galleries of high-confidence errors.

Part 2 - Mathematical form

$$\mathcal{E}_g = \{i : \widehat{y}_i \neq y_i \wedge \text{type}(i) = g\}.$$

Part 3 - Hints and Helper

Preserve original and perturbed versions together and cap stored examples per group.

Part 4 - Python code skeleton

```
from pathlib import Path
import torch

def E16(model, data, config):
    errors = ...
    groups = ...
    report = {name: ... for name, samples in groups.items()}
    ...
    return report
```

14.8.18 Exercise 17 - Analyze Calibration Error and Build a Reliability Diagram for Overconfidence

Part 1 - Why this matters

Calibration asks whether predictions made with confidence near a value are correct at approximately that rate. Accuracy alone cannot reveal systematically overconfident probabilities.

Bin predictions by confidence, calculate accuracy and confidence per bin, plot reliability, and compare ECE, NLL, and Brier score across clean and shifted data.

Part 2 - Mathematical form

$$\mathrm{ECE} = \sum_b \frac{|B_b|}{N} |\operatorname{acc}(B_b) - \operatorname{conf}(B_b)|.$$

Part 3 - Hints and Helper

State binning choices and include counts; empty or tiny bins can mislead.

Part 4 - Python code skeleton

```
from pathlib import Path
import torch

def E17(model, data, config):
    confidence, prediction = probabilities.max(dim=1)
    bins = ...
    ece = ...
    figure = ...
    return ece, figure, ...
```

14.8.19 Exercise 18 - Implement CNN Ensembling and Quantify Accuracy and Robustness Improvements

Part 1 - Why this matters

Models trained with different seeds may make partially independent errors. Averaging their predictive distributions can improve accuracy and robustness, but multiplies storage and inference cost.

Train or load genuinely independent members, measure disagreement and error overlap, and compare probability averaging with a single model on clean and corrupted sets.

Part 2 - Mathematical form

$$p_{\mathrm{ens}}(y \mid X) = M^{-1} \sum_m p_m(y \mid X).$$

Part 3 - Hints and Helper

Keep class order and preprocessing identical across members and report latency and memory alongside gains.

Part 4 - Python code skeleton

```
import torch

def E18(model, data, config):
    member_probabilities = [model(images).softmax(dim=1) for model in models]
    ensemble = torch.stack(member_probabilities).mean(dim=0)
    disagreement = ...
    return ensemble, disagreement, ...
```

14.8.20 Exercise 19 - Write a Practical CNN Regularization Strategy Guide for Training and Deployment

Part 1 - Why this matters

Regularization should respond to measured failure modes rather than being enabled by habit. Training fit, validation gap, calibration, robustness, batch size, shift risk, and deployment cost all influence the strategy.

Create a decision guide covering diagnosis, technique selection, ablations, CUDA cost, calibration, shift evaluation, rollback criteria, and monitoring evidence.

Part 2 - Mathematical form

$$\text{strategy} = g(\text{fit gap, robustness, calibration, cost, deployment shift}).$$

Part 3 - Hints and Helper

For every recommendation include trigger, expected effect, trade-off, experiment, and regression test.

Part 4 - Python code skeleton

```
from pathlib import Path
import torch

def E19(model, data, config):
    guide = ...
    required = [...]
    missing = [section for section in required if section not in guide]
    Path(output_path).write_text(guide, encoding="utf-8")
    return {"missing": missing, "ready": not missing}
```

14.9 CNN07 - Modern CNN Blocks and Architectural Patterns

Models and Training

14.9.1 Exercise 00 - Implement a VGG-Style Block and Analyze Parameters and Feature Behavior

Part 1 - Why this matters

A VGG-style block stacks small convolutions and activations before reducing resolution. Repeated local transformations expand the effective receptive field while increasing channels and activation cost.

Build a configurable block, predict every shape, count parameters, and inspect activation statistics after each layer.

Part 2 - Mathematical form

$$\#\theta = \sum_{l}(C_{l-1}C_lK_l^2 + C_l).$$

Part 3 - Hints and Helper

Compare two small kernels with one larger kernel using matched input and output shapes.

Part 4 - Python code skeleton

```
import torch

def E00(model, data, config):
    block = torch.nn.Sequential(...)
    output, feature_stats = ...
    return output, {"parameters": ..., "stats": feature_stats}
```

14.9.2 Exercise 01 - Implement a Residual Block and Explain Improved Gradient Flow

Part 1 - Why this matters

Residual blocks learn a transformation added to an identity or projected path. The skip route allows information and gradients to bypass difficult transformations.

Implement a basic block, compare gradient norms with a plain stack, and test identity behavior when the residual branch is near zero.

Part 2 - Mathematical form

$$Y = F(X) + X, \qquad \partial Y/\partial X = \partial F/\partial X + I.$$

Part 3 - Hints and Helper

Use projection only when channels or spatial size change.

Part 4 - Python code skeleton

```
import torch

def E01(model, data, config):
    residual = ...
    plain = ...
    return {"residual_output": ..., "plain_output": ...,
            "gradient_norms": ...}
```

14.9.3 Exercise 02 - Validate Residual Addition Shapes and Debug Channel Mismatch Errors

Part 1 - Why this matters

Residual addition requires equal batch, channel, height, and width dimensions. Stride or channel changes on only one path cause invalid addition or incorrect alignment.

Build a shape validator and projection shortcut, then test identity, channel expansion, and spatial reduction cases.

Part 2 - Mathematical form

$$\text{shape}(F(X)) = \text{shape}(P(X)).$$

Part 3 - Hints and Helper

Check both paths immediately before addition and report their configurations.

Part 4 - Python code skeleton

```
import torch

def E02(model, data, config):
    main, skip = ...
    if main.shape != skip.shape: raise ValueError(...)
    return main + skip, ...
```

14.9.4 Exercise 03 - Implement a Bottleneck Residual Block and Compare Its Efficiency with a Standard Block

Part 1 - Why this matters

A bottleneck uses one-by-one layers to compress and restore channels around an expensive spatial convolution. Efficiency depends on the bottleneck width and hardware execution.

Compare standard and bottleneck blocks in parameters, FLOPs, latency, memory, and feature statistics under matched output shapes.

Part 2 - Mathematical form

$$C \xrightarrow{1\times1} C_b \xrightarrow{3\times3} C_b \xrightarrow{1\times1} C'.$$

Part 3 - Hints and Helper

Do not call a block efficient from parameter count alone; benchmark the target GPU.

Part 4 - Python code skeleton

```
import torch

def E03(model, data, config):
    standard, bottleneck = ...
    return {"standard": profile(...), "bottleneck": profile(...)}
```

14.9.5 Exercise 04 - Implement Depthwise Separable Convolution and Quantify Parameter and FLOPs Reduction

Part 1 - Why this matters

Depthwise separable convolution splits spatial filtering by channel from one-by-one channel mixing. It greatly reduces theoretical work but restricts connectivity in the depthwise stage.

Implement both stages, validate against their mathematical decomposition, and compare with standard convolution.

Part 2 - Mathematical form

$$\#\theta_{\text{sep}} = C_{in}K^2 + C_{in}C_{out}.$$

Part 3 - Hints and Helper

Report depthwise-only and complete separable outputs separately.

Part 4 - Python code skeleton

```
import torch

def E04(model, data, config):
    depthwise = torch.nn.Conv2d(..., groups=...)
    pointwise = torch.nn.Conv2d(..., kernel_size=1)
    return pointwise(depthwise(data)), {...}
```

14.9.6 Exercise 05 - Implement a MobileNetV2-Style Inverted Residual and Explain the Expansion Ratio

Part 1 - Why this matters

An inverted residual expands a narrow input, applies depthwise filtering, and projects back to a compact output. The expansion ratio controls hidden capacity and cost.

Build the block with a linear projection, conditional skip, and variable expansion; compare accuracy, latency, and memory.

Part 2 - Mathematical form

$$C \xrightarrow{1\times1} tC \xrightarrow{DW} tC \xrightarrow{1\times1} C'.$$

Part 3 - Hints and Helper

Use the skip only when stride is one and endpoint shapes match.

Part 4 - Python code skeleton

```
import torch

def E05(model, data, config):
    expanded = ...
    depthwise = ...
    projected = ...
    return projected + data if ... else projected
```

14.9.7 Exercise 06 - Implement Squeeze-and-Excitation Attention and Measure Feature Selectivity

Part 1 - Why this matters

SE attention summarizes each channel globally and predicts channel gates. It adapts feature strength to the input without directly modeling new spatial locations.

Implement squeeze, reduction, expansion, and sigmoid gating; inspect gate distributions and class-conditional selectivity.

Part 2 - Mathematical form

$$s_c = (HW)^{-1} \sum_{i,j} X_{cij}, \qquad Y_c = g_c X_c.$$

Part 3 - Hints and Helper

Choose a non-zero reduced width and measure added parameters and latency.

Part 4 - Python code skeleton

```
import torch

def E06(model, data, config):
    summary = data.mean(dim=(-2, -1))
    gates = ...
    return data * gates[..., None, None], gates
```

14.9.8 Exercise 07 - Compare Standard, Depthwise Separable, and Grouped Convolution Speed on GPU

Part 1 - Why this matters

Theoretical FLOPs do not translate directly to GPU speed because kernel launch, memory traffic, tensor shape, and backend support matter.

Benchmark warmed-up variants across batch and channel sizes, reporting latency, throughput, parameters, FLOPs, and peak memory.

Part 2 - Mathematical form

$$\text{speedup} = t_{standard}/t_{variant}.$$

Part 3 - Hints and Helper

Synchronize timing boundaries and keep input-output shapes identical.

Part 4 - Python code skeleton

```
import torch

def E07(model, data, config):
    variants = {...}
    results = {}
    for name, module in variants.items():
        results[name] = benchmark(module, data)
    return results
```

14.9.9 Exercise 08 - Build a Small CNN with Modern Blocks and Validate Its Forward Pass

Part 1 - Why this matters

Combining modern blocks requires a coherent channel and stride plan. A valid collection of individual modules can still create an invalid network when transitions disagree.

Build a compact staged CNN, trace all shapes, verify logits, parameter count, finite activations, and supported input sizes.

Part 2 - Mathematical form

$$X \to \text{stem} \to \text{stages} \to \text{global pool} \to \text{logits}.$$

Part 3 - Hints and Helper

Document the contract at every stage boundary before training.

Part 4 - Python code skeleton

```
import torch

def E08(model, data, config):
    model = torch.nn.Sequential(...)
    trace = ...
    logits = model(data)
    return logits, trace, ...
```

14.9.10 Exercise 09 - Debug Stride Placement and Residual Path Misalignment Bugs

Part 1 - Why this matters

Moving stride between branches or layers changes sampling, receptive-field centers, and tensor alignment. Matching final shape alone may not guarantee corresponding spatial content.

Construct failing stride variants, trace both paths, compare impulse responses, and repair alignment deliberately.

Part 2 - Mathematical form

$$J_l = J_{l-1} S_l.$$

Part 3 - Hints and Helper

Use a synthetic impulse image to reveal spatial offsets hidden by ordinary inputs.

Part 4 - Python code skeleton

```
import torch

def E09(model, data, config):
    main_trace, skip_trace = ...
    impulse_outputs = ...
    return {"main": main_trace, "skip": skip_trace,
            "alignment": ...}
```

14.9.11 Exercise 10 - Train a Modern-Block CNN on CUDA and Benchmark It Against a Simple Baseline

Part 1 - Why this matters

Modern blocks should be judged by learning behavior and hardware cost, not novelty. A smaller model may train slower if its operations are poorly utilized.

Train matched baseline and modern models from controlled seeds, comparing accuracy, convergence, throughput, memory, and stability.

Part 2 - Mathematical form

$$\text{value} = f(\text{accuracy}, \text{speed}, \text{memory}, \text{stability}).$$

Part 3 - Hints and Helper

Use equal data, epochs, optimizer policy, precision, and evaluation protocol.

Part 4 - Python code skeleton

```
import torch

def E10(model, data, config):
    baseline, modern = ...
    return controlled_train(baseline, ...), controlled_train(modern, ...)
```

14.9.12 Exercise 11 - Profile GPU Utilization and Identify Block-Level Bottlenecks

Part 1 - Why this matters

Block profiling locates expensive operations and reveals whether compute, memory, or launch overhead dominates. End-to-end utilization alone cannot name the cause.

Profile representative training steps and aggregate CUDA time, calls, shapes, and memory by block type.

Part 2 - Mathematical form

$$q_b = t_b / \sum_j t_j.$$

Part 3 - Hints and Helper

Warm up first and profile only a limited representative window.

Part 4 - Python code skeleton

```
import torch

def E11(model, data, config):
    with torch.profiler.profile(...) as profile:
        ...
    return summarize_by_block(profile)
```

14.9.13 Exercise 12 - Compare Memory Usage of VGG-Style and Residual Architectures

Part 1 - Why this matters

Residual models store branch activations for later addition, while VGG-style stacks may use wide high-resolution maps. Training memory includes saved activations, gradients, and workspaces.

Profile matched-depth models in inference and training, recording parameter bytes, activation estimates, and peak CUDA memory.

Part 2 - Mathematical form

$$M_{peak} \geq M_{params} + M_{saved\ activations} + M_{workspace}.$$

Part 3 - Hints and Helper

Reset peak statistics and isolate one model per trial.

Part 4 - Python code skeleton

```
import torch

def E12(model, data, config):
    for model in models:
        results.append(memory_profile(model, data, ...))
    return results
```

14.9.14 Exercise 13 - Implement Stochastic Depth and Observe Regularization in Deep Residual Networks

Part 1 - Why this matters

Stochastic depth skips residual branches for selected samples or batches during training while keeping all blocks active during evaluation. It shortens expected training paths and regularizes depth.

Implement survival probabilities across depth, verify expectation scaling and modes, then compare generalization and convergence.

Part 2 - Mathematical form

$$Y = X + \frac{m}{p}F(X), \qquad m \sim Bernoulli(p).$$

Part 3 - Hints and Helper

Never drop the identity path, and test deterministic evaluation explicitly.

Part 4 - Python code skeleton

```
import torch

def E13(model, data, config):
    if model.training and ...:
        residual = residual * ...
    return identity + residual
```

14.9.15 Exercise 14 - Implement DropPath and Verify It Activates Only During Training

Part 1 - Why this matters

DropPath removes complete residual contributions rather than independent elements. Its mask shape must broadcast over non-batch dimensions so one path decision applies consistently per sample.

Implement the operation, verify empirical survival rate, expectation, train/eval behavior, device, and dtype.

Part 2 - Mathematical form

$$\widetilde{X} = X\, m/p.$$

Part 3 - Hints and Helper

Return the input unchanged when disabled, in evaluation mode, or survival is one.

Part 4 - Python code skeleton

```
import torch

def E14(model, data, config):
    if not training or drop_probability == 0: return data
    mask = ...
    return data * mask / ...
```

14.9.16 Exercise 15 - Compare Accuracy and Robustness Across Architectural Pattern Choices

Part 1 - Why this matters

Architectures encode different biases about depth, connectivity, channel mixing, and efficiency. Clean accuracy alone cannot identify robustness or deployment suitability.

Compare controlled models on clean data, noise, blur, occlusion, shift, calibration, latency, and memory.

Part 2 - Mathematical form

$$\mathcal{Q} = \{acc, robustness, ECE, latency, memory\}.$$

Part 3 - Hints and Helper

Match training budget and report uncertainty across seeds.

Part 4 - Python code skeleton

```
import torch

def E15(model, data, config):
    for name, model in models.items():
        results[name] = evaluation_suite(model, data)
    return results
```

14.9.17 Exercise 16 - Build an Ablation Study Removing One Block Type at a Time and Measure Impact

Part 1 - Why this matters

Ablation tests whether a block contributes beyond the rest of the architecture. Removing it may also change parameter count or depth, so the comparison must state resulting differences.

Create a full model and planned removals or replacements, repeat seeds, and measure quality, cost, gradient flow, and stability.

Part 2 - Mathematical form

$$\Delta_b = q(M) - q(M \setminus b).$$

Part 3 - Hints and Helper

Predefine replacements and preserve tensor compatibility.

Part 4 - Python code skeleton

```
import torch

def E16(model, data, config):
    for configuration in ablations:
        for seed in seeds: records.append(run(...))
    return records, ...
```

14.9.18 Exercise 17 - Create an Architecture Summary with Parameters, FLOPs, Speed, and Accuracy

Part 1 - Why this matters

A structured summary prevents selection by one metric and exposes trade-offs between theoretical and measured efficiency.

Collect shapes, parameters, FLOPs, CUDA latency, throughput, peak memory, clean accuracy, and robustness into one validated table.

Part 2 - Mathematical form

$$\text{efficiency} = \text{quality}/\text{measured cost}.$$

Part 3 - Hints and Helper

Use identical benchmark settings and label estimated metrics separately from measured ones.

Part 4 - Python code skeleton

```
import torch

def E17(model, data, config):
    rows = [summarize(name, model, data) for name, model in models.items()]
    return rows, validate_rows(rows)
```

14.9.19 Exercise 18 - Stress Test a Deep Modern CNN for NaNs, Instability, and Memory Blowups

Part 1 - Why this matters

Deep networks can fail from activation growth, broken skips, unstable normalization, precision limits, or excessive saved state. Failure may begin long before the final loss becomes non-finite.

Increase depth and batch systematically while logging layer activations, gradients, memory, scaler behavior, and first failure.

Part 2 - Mathematical form

$$r_l = \|A_l\|/(\|A_{l-1}\| + \varepsilon).$$

Part 3 - Hints and Helper

Stop on the first non-finite value and retain its exact configuration and batch.

Part 4 - Python code skeleton

```
import torch

def E18(model, data, config):
    records = monitor_layers(model, data)
    first_failure = ...
    return records, first_failure
```

14.9.20 Exercise 19 - Produce a Modern CNN Architecture Guide for Efficient Vision Models

Part 1 - Why this matters

Efficient design balances representation, optimization, hardware behavior, data, robustness, and deployment constraints. No block is universally superior.

Write a decision guide connecting task requirements to block choice, channel plan, downsampling, attention, regularization, benchmarking, and failure tests.

Part 2 - Mathematical form

$$design = g(task, data, hardware, latency, memory, robustness).$$

Part 3 - Hints and Helper

For each recommendation include purpose, alternative, measured trade-off, and verification test.

Part 4 - Python code skeleton

```
from pathlib import Path
import torch

def E19(model, data, config):
    guide = ...
    Path(output_path).write_text(guide, encoding="utf-8")
    return validate_guide(guide, evidence)
```

14.10 CNN08 - Deployment, Export, and Production Readiness

Models and Training

14.10.1 Exercise 00 - Benchmark CNN Inference Latency and Throughput Under Production-Like Batch and Precision Settings

Part 1 - Why this matters

A fast experiment is not automatically a fast production system. Warm-up, synchronization, batch size, precision, preprocessing, and request arrival patterns can change the measured latency dramatically.

Build a controlled benchmark that separates warm-up from measured iterations and reports latency percentiles as well as throughput. Treat the benchmark configuration as part of the result.

Part 2 - Mathematical form

$$\text{throughput} = \frac{NB}{\sum_{i=1}^{N} t_i}, \qquad p_q = \text{Quantile}_q(t_1, \ldots, t_N).$$

Part 3 - Hints and Helper

Synchronize CUDA before starting and stopping the timer. Report median and tail latency, not only the arithmetic mean.

Part 4 - Python code skeleton

```
import torch

def E00(model, data, config):
    model.eval()
    with torch.inference_mode():
        for _ in range(...):
            _ = model(sample)
        times = ...
    return {"p50_ms": ..., "p99_ms": ..., "samples_per_s": ...}
```

14.10.2 Exercise 01 - Export CNN Using TorchScript and Validate Output Parity Across Exported and Native Models

Part 1 - Why this matters

Export changes the execution representation of a model and may expose unsupported control flow, device assumptions, or mode-dependent behavior. A successful save operation alone does not prove correctness.

Export an evaluation-mode CNN, reload it independently, and compare its outputs with the native model over several fixed and randomized inputs. Record both absolute and relative error.

Part 2 - Mathematical form

$$e_{\max} = \max_i |y_i - \hat{y}_i|, \qquad y \approx \hat{y} \iff e_{\max} \le a + r \max_i |y_i|.$$

Part 3 - Hints and Helper

Use the same weights, preprocessing, device, dtype, and evaluation mode on both paths. Test more than one input.

Part 4 - Python code skeleton

```
import torch

def E01(model, data, config):
    model.eval()
    example = ...
    exported = torch.jit.trace(model, example)
    exported.save(...)
    reloaded = ...
    return {"max_abs_error": ..., "parity": ...}
```

14.10.3 Exercise 02 - Export CNN Using ONNX and Validate Correctness Across Different Runtime Backends

Part 1 - Why this matters

ONNX separates the model from the training framework, but operator conversion and runtime implementations may introduce shape or numerical differences. Portability must therefore be demonstrated with evidence.

Export the CNN with an explicit input contract, load it in available runtimes, and compare every backend against the PyTorch reference. Include dynamic-batch tests when the deployment contract permits them.

Part 2 - Mathematical form

$$\Delta_b = \max_i |f_{\mathrm{PT}}(x_b)_i - f_{\mathrm{ONNX}}(x_b)_i| \,.$$

Part 3 - Hints and Helper

Name the input and output tensors, declare only genuinely dynamic axes, and validate several batch sizes.

Part 4 - Python code skeleton

```
import torch

def E02(model, data, config):
    torch.onnx.export(model, example, output_path, input_names=[...],
                      output_names=[...], dynamic_axes=...)
    reference = model(example)
    backend_outputs = ...
    return compare_backends(reference, backend_outputs)
```

14.10.4 Exercise 03 - Use torch.compile for CNN and Measure Speed Gains vs Compilation Overhead

Part 1 - Why this matters

Compilation can reduce steady-state execution time, yet its startup cost may outweigh the gain for short jobs or changing input shapes. Benchmarking only the fastest repeated call hides this trade-off.

Measure eager execution, first compiled call, and steady compiled execution separately. Calculate the number of requests required to recover compilation cost and verify output parity.

Part 2 - Mathematical form

$$T_c(n) = T_{\mathrm{compile}} + n\, t_c, \qquad n_{\mathrm{break}} = \frac{T_{\mathrm{compile}}}{t_e - t_c}.$$

Part 3 - Hints and Helper

Keep input shapes controlled, synchronize device timings, and report graph recompilations when shapes change.

Part 4 - Python code skeleton

```
import torch

def E03(model, data, config):
    compiled = torch.compile(model)
    eager_ms = benchmark(model, inputs)
    first_call_ms = ...
    compiled_ms = benchmark(compiled, inputs)
    return {"eager_ms": eager_ms, "first_ms": ..., "break_even": ...}
```

14.10.5 Exercise 04 - Implement Quantization Conceptually and Measure Accuracy Drop vs Performance Gain

Part 1 - Why this matters

Quantization replaces expensive high-precision values with a smaller numerical representation. It can reduce memory and latency, but calibration errors and sensitive layers may damage accuracy.

Implement the quantize–dequantize idea on tensors, then evaluate a supported model quantization path. Compare model size, latency, and task metric against the floating-point baseline.

Part 2 - Mathematical form

$$q = \text{clip}(\text{round}(x/s) + z, q_{\min}, q_{\max}), \qquad \hat{x} = s(q - z).$$

Part 3 - Hints and Helper

Use representative calibration data. Measure end-to-end behavior and identify layers responsible for the largest error.

Part 4 - Python code skeleton

```
import torch

def E04(model, data, config):
    def fake_quantize(x, scale, zero_point, qmin, qmax):
        q = ...
        return ...

    baseline = evaluate(model, loader)
    quantized = ...
    return compare_quality_size_latency(baseline, quantized)
```

14.10.6 Exercise 05 - Implement Batch Inference Pipeline with Efficient Transfer and Memory Reuse

Part 1 - Why this matters

Production throughput depends on the entire input-to-output path, not solely on the CNN kernel. Repeated allocation, blocking transfers, and tiny batches can leave an accelerator idle.

Build a batch inference pipeline that validates inputs, groups requests, uses efficient host-to-device transfer, and reuses buffers where safe. Preserve the mapping from every output to its request.

Part 2 - Mathematical form

$$T_{\text{batch}} = T_{\text{prep}} + T_{H\to D} + T_{\text{model}} + T_{D\to H}.$$

Part 3 - Hints and Helper

Use pinned memory with non-blocking transfer only when the environment supports it. Never reuse a buffer before its work is complete.

Part 4 - Python code skeleton

```
import torch

def E05(model, data, config):
    def infer_batch(requests, model, device, buffers):
        request_ids, host_batch = ...
        device_batch = host_batch.to(device, non_blocking=...)
        with torch.inference_mode():
            output = model(device_batch)
        return restore_request_order(request_ids, output)
```

14.10.7 Exercise 06 - Build a Minimal Inference Service Wrapper with Input Validation and Error Handling

Part 1 - Why this matters

A correct model can still fail as a service when requests contain the wrong shape, dtype, range, encoding, or size. Validation turns uncontrolled tensor failures into explicit API behavior.

Create a framework-neutral service wrapper that validates a request, preprocesses it, runs inference without gradient tracking, and returns a versioned response or a structured error.

Part 2 - Mathematical form

$$\mathcal{S}(r) = \text{serialize}(f_\theta(P(V(r)))) \,.$$

Part 3 - Hints and Helper

Separate validation errors from internal failures. Do not expose stack traces or silently reshape malformed data.

Part 4 - Python code skeleton

```
import torch

def E06(model, data, config):
    class InferenceService:
        def predict(self, request):
            validated = ...
```

```
    tensor = ...
    with torch.inference_mode():
        output = self.model(tensor)
    return {"model_version": ..., "prediction": ...}
```

14.10.8 Exercise 07 - Implement Model Versioning and Backward-Compatible Checkpoint Loading

Part 1 - Why this matters

Weights are meaningful only together with architecture, preprocessing, labels, and format version. An unversioned checkpoint can load successfully while producing semantically wrong predictions.

Define checkpoint metadata and a migration path from an older schema. Reject incompatible artifacts clearly, and test that a migrated checkpoint preserves reference predictions.

Part 2 - Mathematical form

$$C_v = \{\theta, v, \mathcal{A}, P, L, m\}, \qquad M_{v \to v+1}(C_v) = C_{v+1}.$$

Part 3 - Hints and Helper

Store schema version, model identifier, class mapping, preprocessing signature, and integrity information.

Part 4 - Python code skeleton

```
import torch

def E07(model, data, config):
    def load_versioned_checkpoint(path, model):
        checkpoint = ...
        version = checkpoint["schema_version"]
        if version < CURRENT_VERSION:
            checkpoint = ...
        validate_compatibility(checkpoint, model)
        return ...
```

14.10.9 Exercise 08 - Add Monitoring Hooks for Confidence Drift and Data Distribution Shift

Part 1 - Why this matters

Production labels often arrive late, so accuracy cannot be the only early warning signal. Changes in input statistics, predicted classes, confidence, and invalid-request rate can reveal

drift sooner.

Collect privacy-aware aggregate signals over time windows and compare them with a reference baseline. Trigger an alert only through an explicit threshold and persistence rule.

Part 2 - Mathematical form

$$\mathrm{PSI} = \sum_j (p_j - q_j) \log \frac{p_j + \epsilon}{q_j + \epsilon}, \qquad \bar{c} = \frac{1}{N} \sum_i \max_k p_{ik}.$$

Part 3 - Hints and Helper

Monitor trends and sample counts alongside scores. A drift alert indicates investigation, not automatic proof that the model is wrong.

Part 4 - Python code skeleton

```
import torch

def E08(model, data, config):
    class DriftMonitor:
        def update(self, probabilities, input_stats):
            self.confidence.update(...)
            self.predicted_classes.update(...)
        def report(self, reference):
            return {"psi": ..., "mean_confidence": ..., "alert": ...}
```

14.10.10 Exercise 09 - Build Batch Failure Handling That Skips Corrupt Inputs Without Breaking the Pipeline

Part 1 - Why this matters

One corrupt image should not erase every valid result in a batch. At the same time, silently dropping records breaks traceability and can hide a systematic data problem.

Validate each item, infer on the valid subset, restore original ordering, and return per-item success or error records. Confirm that one failure cannot shift predictions onto the wrong identifiers.

Part 2 - Mathematical form

$$I_v = \{i : V(x_i) = 1\}, \qquad R_i = \begin{cases} f(x_i), & i \in I_v, \\ e_i, & i \notin I_v. \end{cases}$$

Part 3 - Hints and Helper

Keep immutable request IDs through filtering and batching. Count failure types and set a batch-level failure-rate limit.

Part 4 - Python code skeleton

```
import torch

def E09(model, data, config):
    def safe_batch_predict(records, model):
        valid, errors = validate_each(records)
        outputs = model(stack_valid(valid)) if valid else ...
        results = restore_by_request_id(outputs, errors)
        return results, {"failure_rate": ...}
```

14.10.11 Exercise 10 - Run Deployment-Oriented CUDA Inference and Measure Real GPU Utilization Under Load

Part 1 - Why this matters

A CUDA model can remain slow when the workload is too small, transfers are serialized, or the input pipeline starves the device. Utilization must be interpreted together with latency and throughput.

Generate controlled concurrent load, measure device utilization and memory, and relate those measurements to request latency. Find the saturation point rather than maximizing utilization blindly.

Part 2 - Mathematical form

$$U = \frac{T_{\text{busy}}}{T_{\text{wall}}}, \qquad \lambda_{\max} = \max\{\lambda : p_{99}(\lambda) \le S\}.$$

Part 3 - Hints and Helper

Warm up first and synchronize only at measurement boundaries. Record queue time separately from model execution time.

Part 4 - Python code skeleton

```
import torch

def E10(model, data, config):
    load_generator = ...
    with gpu_monitor() as samples:
        responses = run_concurrent_load(service, load_generator, rate=...)
    return {"gpu_utilization": ..., "queue_ms": ..., "p99_ms": ...}
```

14.10.12 Exercise 11 - Compare float32, float16, and int8 Inference Accuracy and Performance Trade-Offs

Part 1 - Why this matters

Lower precision can improve speed and memory use, but gains depend on hardware, operators, batch size, and calibration. A format is useful only if it satisfies both quality and service objectives.

Evaluate all supported precision paths with identical data and report quality, latency, throughput, memory, and numerical disagreement. Mark unsupported combinations rather than inventing results.

Part 2 - Mathematical form

$$S_p = \frac{t_{32}}{t_p}, \qquad \Delta M_p = M_{32} - M_p, \qquad \Delta Q_p = Q_{32} - Q_p.$$

Part 3 - Hints and Helper

Use the same warm-up and measurement protocol. Include tail latency and per-class quality when class sensitivity matters.

Part 4 - Python code skeleton

```
import torch

def E11(model, data, config):
    results = {}
    precisions = ...
    for precision in precisions:
        candidate = prepare_model(model, precision)
        results[precision] = evaluate_precision(
            candidate, loader
        )
    return compare_precision_results(results)
```

14.10.13 Exercise 12 - Build a Reproducible Deployment Benchmark Suite with Fixed Inputs and Seeds

Part 1 - Why this matters

A benchmark is useful only when another run can explain or reproduce its result. Hidden changes in data, software, hardware, warm-up, or timing rules can create false performance regressions.

Package fixed inputs, seeds, environment metadata, correctness tolerances, and timing procedures into one suite. Produce a machine-readable result record and compare it with a stored baseline.

Part 2 - Mathematical form

$$R = F(H, S, D, C, W, N), \qquad \delta = \frac{R_{\text{new}} - R_{\text{base}}}{R_{\text{base}}}.$$

Part 3 - Hints and Helper

Hash the input fixture and configuration. Separate correctness failures from statistically meaningful performance regressions.

Part 4 - Python code skeleton

```
import torch

def E12(model, data, config):
    set_all_seeds(config.seed)
    assert sha256_file(inputs_path) == config.input_hash
    result = run_benchmark(config, inputs_path)
    record = {"environment": ..., "config": ..., "result": result}
    return compare_with_baseline(record, baseline)
```

14.10.14 Exercise 13 - Implement Checkpoint Integrity Verification and Detect Corrupted Weight Files Early

Part 1 - Why this matters

A truncated or modified checkpoint must be rejected before it reaches the model. Loading without integrity and compatibility checks can turn a storage fault into unpredictable inference behavior.

Create a manifest containing size, cryptographic digest, version, and expected tensor metadata. Verify the artifact before deserialization and fail closed on any mismatch.

Part 2 - Mathematical form

$$h = \mathrm{SHA256}(B), \qquad \mathrm{valid}(B) = [h = h^*] \wedge [|B| = n^*].$$

Part 3 - Hints and Helper

Compute the digest as a stream for large files. Test bit modification, truncation, wrong version, and wrong tensor shape.

Part 4 - Python code skeleton

```
import torch

def E13(model, data, config):
    def verify_checkpoint(path, manifest):
        actual_hash = sha256_stream(path)
        if actual_hash != manifest["sha256"]:
            raise ...
        checkpoint = ...
        validate_tensor_metadata(checkpoint, manifest)
        return checkpoint
```

14.10.15 Exercise 14 - Simulate NaN Production Incident and Build an Automatic Rollback Strategy

Part 1 - Why this matters

Non-finite outputs can spread into ranking, billing, or downstream decisions. Recovery needs a tested health gate and a known-good artifact, not an improvised response after impact begins.

Inject a controlled NaN failure, detect it at the service boundary, stop promotion, and atomically return traffic to the previous healthy version. Preserve evidence for root-cause analysis.

Part 2 - Mathematical form

$$H(y) = \mathbf{1}[\text{isfinite}(y)]\mathbf{1}[m \geq m_{\min}], \qquad H = 0 \Rightarrow v \leftarrow v_{\text{stable}}.$$

Part 3 - Hints and Helper

Use consecutive-failure or rate thresholds to avoid flapping. Rollback must include preprocessing and configuration, not weights alone.

Part 4 - Python code skeleton

```
import torch

def E14(model, data, config):
    output = candidate_service.predict(canary_batch)
    healthy = torch.isfinite(output).all() and metrics_pass(...)
    if not healthy:
        traffic_router.activate(stable_version)
        incident_log.write(...)
    return {"healthy": bool(healthy), "active_version": ...}
```

14.10.16 Exercise 15 - Validate Training–Serving Preprocessing Parity and Detect Silent Mismatch Bugs

Part 1 - Why this matters

A production model may receive different resizing, channel order, normalization, or label mapping from training. Such mismatches often produce plausible outputs and evade ordinary exception handling.

Represent preprocessing as a versioned contract and run golden samples through both training and serving paths. Fail deployment when tensors or predictions exceed a justified tolerance.

Part 2 - Mathematical form

$$d_P = \max_i |P_{\text{train}}(x)_i - P_{\text{serve}}(x)_i|.$$

Part 3 - Hints and Helper

Check resize semantics, color space, value range, channel order, normalization constants, and class-index mapping.

Part 4 - Python code skeleton

```
import torch

def E15(model, data, config):
    train_tensor = training_preprocess(golden_image)
    serve_tensor = serving_preprocess(golden_image)
    max_error = ...
    assert max_error <= tolerance
    return {"tensor_error": ..., "prediction_parity": ...}
```

14.10.17 Exercise 16 - Implement A/B Testing for Two CNN Models on the Same Input Stream

Part 1 - Why this matters

Offline improvement does not guarantee online value. A controlled A/B test estimates the effect of a new model while preventing routing rules or unequal traffic from biasing the comparison.

Assign requests deterministically, log model version and outcome, and compare guardrail and primary metrics. Include a stop rule and verify that each user remains in one group.

Part 2 - Mathematical form

$$\widehat{\Delta} = \bar{m}_B - \bar{m}_A, \qquad z = \frac{\widehat{\Delta}}{\sqrt{s_A^2/n_A + s_B^2/n_B}}.$$

Part 3 - Hints and Helper

Hash a stable unit such as user ID for assignment. Do not route retries independently or declare success from an unplanned metric.

Part 4 - Python code skeleton

```
import torch

def E16(model, data, config):
    def assign_variant(stable_id):
        return ...
```

```
    variant = assign_variant(request.user_id)
    result = services[variant].predict(request)
    log_exposure(variant, result, ...)
    return analyze_ab_metrics(exposures, outcomes)
```

14.10.18 Exercise 17 - Create an End-to-End Production Readiness Checklist for Performance, Monitoring, and Stability

Part 1 - Why this matters

Release confidence comes from evidence across correctness, performance, resilience, security, observability, and recovery. A checklist converts vague confidence into explicit release gates and accountable evidence.

Build a machine-checkable readiness record whose items name an owner, result, artifact, threshold, and expiration. A critical failed gate must block release.

Part 2 - Mathematical form

$$\text{ready} = \bigwedge_{j \in \mathcal{C}} g_j, \qquad g_j \in \{0, 1\}.$$

Part 3 - Hints and Helper

Distinguish mandatory gates from advisory findings. Link every passing claim to a test result rather than a verbal assertion.

Part 4 - Python code skeleton

```
import torch

def E17(model, data, config):
    required_gates = load_readiness_spec(...)
    results = {gate.name: run_gate(gate) for gate in required_gates}
    ready = all(r.passed for r in results.values() if r.critical)
    return {"ready": ready, "evidence": ...}
```

14.10.19 Exercise 18 - Stress Test Inference Service for Memory Leaks and Tail Latency Spikes

Part 1 - Why this matters

A service that survives a short benchmark may degrade after thousands of requests. Memory growth, queue buildup, allocator fragmentation, and rare slow inputs often appear only under sustained load.

Drive a staged workload through the service, sample latency and memory over time, and detect sustained growth after warm-up. Include bursts, corrupt inputs, and recovery periods.

Part 2 - Mathematical form

$$s_M = \frac{\sum_i (t_i - \bar{t})(M_i - \bar{M})}{\sum_i (t_i - \bar{t})^2}, \qquad \text{leak suspected if } s_M > \tau.$$

Part 3 - Hints and Helper

Use bounded queues and a fixed test duration. Keep allocator caching distinct from unreachable-memory growth before diagnosing a leak.

Part 4 - Python code skeleton

```
import torch

def E18(model, data, config):
    samples = []
    for stage in workload_plan:
        responses = drive_load(service, stage)
        samples.append(sample_latency_memory(responses))
    trend = fit_memory_slope(samples)
    return {"memory_slope": trend, "p99_spikes": ..., "passed": ...}
```

14.10.20 Exercise 19 - Produce a Final CNN Engineering Report Covering Architecture, Training, Deployment, and Failures

Part 1 - Why this matters

A final model artifact does not preserve the reasoning, experiments, limitations, or operational knowledge required to maintain it. An engineering report makes the system reproducible and reviewable.

Create an evidence-based report linking architecture and data decisions to training results, deployment benchmarks, monitoring, incidents, and unresolved risks. Reject the report if required evidence is missing.

Part 2 - Mathematical form

$$\mathcal{R} = \{A, D, T, E, P, O, F, L\}, \qquad \text{complete}(\mathcal{R}) = \bigwedge_{r \in \mathcal{R}} [r \neq \varnothing].$$

Part 3 - Hints and Helper

Include exact artifact versions, experiment references, acceptance thresholds, known failure modes, rollback instructions, and next actions.

Part 4 - Python code skeleton

```
import torch

def E19(model, data, config):
    sections = collect_evidence(
        architecture=..., data=..., training=..., evaluation=...,
        deployment=..., operations=..., failures=..., limitations=...)
    report = render_report(sections)
    validate_required_evidence(report)
    return report
```

Book 15

RNN (Recurrent Neural Networks)

15.1 Introduction

15.1.1 What will the book teach you

Previous models in this volume processed an input and produced an output without maintaining an explicit memory of what had appeared before it. A linear model receives features, a multilayer perceptron transforms them through hidden layers, and a convolutional neural network searches for spatial patterns inside an image. These models are powerful, but many real problems cannot be understood by examining each input independently. Language, rainfall, energy demand, financial prices, sensor readings, audio signals, and user activity are sequences in which the meaning of the current observation may depend on observations that occurred earlier.

This book introduces recurrent neural networks, or RNNs, as the first models in this volume designed explicitly to process ordered information through time. Instead of treating every time step as an isolated sample, an RNN carries a hidden state from one step to the next. This hidden state acts as a compact and continuously updated representation of the past. The model therefore does not only ask, "What is the current input?" It also asks, "What information from the previous steps should influence the current computation?"

The central recurrence of a simple RNN can be written as:

$$\mathbf{h}_t = \phi\left(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h\right)$$

and the output may be computed as:

$$\mathbf{y}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y$$

Here, $\mathbf{x}_t$ is the input at time step t, $\mathbf{h}_{t-1}$ is the hidden state received from the previous time step, and $\mathbf{h}_t$ is the updated memory. The matrix $\mathbf{W}_{xh}$ controls how the current input influences the hidden state, while $\mathbf{W}_{hh}$ controls how previous memory influences new memory. The activation function ϕ, commonly `tanh` in a vanilla RNN, introduces non-linearity. The matrices and biases are shared across all time steps, allowing the same transition rule to process sequences of different lengths.

This weight sharing is one of the defining properties of recurrent computation. An RNN does not create a separate set of parameters for the first, second, and third observations. It repeatedly applies the same transformation while the hidden state changes over time. As a

result, the model can learn temporal rules that are not tied to one fixed position. However, sharing weights also means that errors in the recurrent transformation are repeated. A slightly unstable update may grow across a long sequence, while a transformation that repeatedly weakens information may cause earlier observations to disappear from memory.

The book begins by making recurrent computation completely visible. You will represent sequences as ordered time steps, implement the recurrence using pure Python, translate the same process into PyTorch tensor operations, and manually unroll the network. You will compare one-step prediction, full-sequence prediction, many-to-one classification, and many-to-many sequence labeling. These experiments will show that an RNN output is not determined only by its current input; it is also determined by the hidden state produced by the entire preceding computation.

Tensor dimensions require special attention in recurrent models. A typical RNN input may follow the structure:

$$(\text{sequence length}, \text{batch size}, \text{input size})$$

or, when `batch_first=True`:

$$(\text{batch size}, \text{sequence length}, \text{input size})$$

The hidden state adds another structure containing the number of recurrent layers, directional components, batch size, and hidden size. A tensor can have valid numerical values and still represent the wrong temporal organization. For this reason, the exercises will repeatedly require the reader to predict shapes, inspect dimensions, preserve batch and time axes, and debug hidden-state placement on both the CPU and CUDA devices.

The hidden state will then be studied as a dynamic system rather than an unexplained tensor returned by PyTorch. You will visualize how it changes over time, test different initial states, create learnable initial states, compare hidden sizes, stack multiple recurrent layers, and build bidirectional networks. You will probe hidden representations to discover whether earlier information remains encoded or has been forgotten. This investigation is important because the hidden state is not guaranteed to contain a perfect record of the past. It has limited capacity and is repeatedly transformed as new observations arrive.

A bidirectional RNN processes a sequence in two directions. One recurrence moves from the beginning toward the end, while another moves from the end toward the beginning. Their representations can be combined to give each position information about both earlier and later observations:

$$\mathbf{h}_t = \left[\overrightarrow{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t\right]$$

This can improve sequence labeling when the complete sequence is available, but it is not appropriate for every forecasting or real-time problem. A forecasting model cannot use future observations that do not yet exist. The reader must therefore learn that architectural capability and valid problem design are not the same thing.

Training an RNN introduces backpropagation through time, commonly called BPTT. When the recurrent computation is unrolled, the network becomes a deep computational graph whose

depth depends on the sequence length. A loss produced at a later time step may send gradients backward through many earlier states. The gradient therefore contains repeated products of recurrent Jacobians:

$$\frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_t} = \prod_{k=t+1}^{T} \frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_{k-1}}$$

If these repeated factors usually have magnitudes below one, the gradient can shrink until the model receives almost no information about distant events. If they repeatedly have magnitudes above one, the gradient can grow rapidly and destabilize training. These are the vanishing-gradient and exploding-gradient problems, and they are central limitations of the vanilla RNN rather than rare implementation mistakes.

You will inspect gradients across time, deliberately produce vanishing and exploding behavior, apply gradient clipping, and verify a tiny RNN with finite-difference calculations. You will also implement truncated backpropagation through time, in which the sequence is divided into shorter computational segments. Truncation can reduce memory and runtime, but it limits how far gradients can travel. Detaching the hidden state too early breaks necessary learning paths, while failing to detach it at the correct boundary can retain an increasingly large graph. The goal is to understand this trade-off experimentally instead of copying a hidden-state detachment pattern without knowing its effect.

Sequence data preparation creates another class of challenges. Unlike ordinary tabular batches, sequences may have different lengths and may require padding, masks, packing, or carefully aligned sliding windows. You will construct sequence datasets, implement custom batching, use `pack_padded_sequence`, create padding masks, and ensure that padded positions do not contribute to the loss. You will also detect incorrect target shifts, temporal leakage, and invalid train-validation splits. In time-dependent data, randomly moving future observations into the training set can produce impressive results that are completely misleading.

The book applies vanilla RNNs to two major problem families. The first is character-level language modeling, where the model predicts the next character from the preceding sequence. You will build a vocabulary, create input-target pairs, train a small language model, and generate text autoregressively. Temperature and top-k sampling will demonstrate that generation is controlled not only by learned probabilities but also by the strategy used to select the next token. Perplexity, repetition, novelty, and long-term coherence will be examined instead of judging generated text by appearance alone.

The second application is time-series forecasting. You will convert a time series into supervised windows, scale features without leaking future information, and compare one-step with multi-step forecasting. In autoregressive forecasting, predicted values may be returned to the model as later inputs, causing small errors to accumulate across the forecast horizon. Teacher forcing can make training easier by providing true previous values, but it can also create a difference between training and inference known as exposure bias. The exercises will make this difference visible through controlled experiments.

Evaluation must respect temporal structure. Standard random splitting may be unsuitable when the future must be predicted from the past. You will therefore use chronological splits and walk-forward validation, measure errors across multiple forecast horizons, and compare

MAE and MSE. A single average error may hide the fact that accuracy deteriorates severely at longer horizons, during unusual events, or after a distribution shift. Results must be examined over time and under the same conditions in which the model is expected to operate.

Training stability will be investigated through dropout, weight decay, recurrent initialization, layer normalization, learning-rate schedules, early stopping, gradient clipping, and optimizer selection. These methods will not be presented as automatic improvements. Each method changes the learning process and may help, have no measurable effect, or make the model worse. You will conduct ablation experiments in which one factor is changed at a time and use observed evidence to decide which stabilization methods are appropriate.

CUDA and computational performance are integrated throughout the book. You will compare CPU and GPU execution, analyze sequence length and hidden-size scaling, measure memory consumption during BPTT, examine packed and padded sequences, and profile data-transfer and kernel costs. Recurrent computation is sequential across time steps, which limits parallelism compared with many convolutional operations. A GPU may therefore provide little benefit for a tiny RNN or a short sequence, while larger batches and hidden states may use it more effectively. Performance conclusions must be based on synchronized, reproducible measurements rather than assumptions.

Mixed precision introduces additional trade-offs. Lower precision can reduce memory use and improve throughput, but recurrent state updates are repeated many times. Small numerical differences may accumulate across a long sequence, and unstable gradients may overflow or underflow. You will compare precision modes, monitor non-finite values, use gradient scaling where appropriate, and identify conditions in which additional speed is not worth the loss of numerical reliability.

The final chapters will expose the theoretical and practical boundary of the vanilla RNN. You will build controlled memory tasks in which relevant information appears far before the required prediction and measure when the model begins to forget. You will then examine gated recurrent units and long short-term memory networks as engineering responses to this limitation. Their gates create learned paths that regulate which information is written, preserved, exposed, or forgotten.

This transition is important because the book is not intended to make the reader believe that a vanilla RNN is the best model for every sequential problem. Its purpose is to reveal the fundamental ideas of recurrence, temporal state, weight sharing, BPTT, autoregressive prediction, and long-term dependency failure. Once these mechanisms and limitations are understood directly, LSTM and GRU architectures become logical solutions to observed problems rather than collections of equations that must be memorized.

By the end of this book, the reader should be able to construct and train an RNN, explain its tensor shapes and temporal computation, prepare variable-length sequence data, inspect hidden states and gradients, evaluate language and forecasting tasks, diagnose instability, profile CUDA performance, and identify when a vanilla RNN is no longer suitable. More importantly, the reader should be able to support every conclusion with an experiment.

15.1.2 How to study this book

Every exercise should be treated as a controlled temporal experiment. Before running the code, predict the input shape, output shape, hidden-state shape, device placement, and expected behavior across time. When changing sequence length, hidden size, precision, learning rate, or truncation length, keep the remaining conditions fixed. Otherwise, it becomes impossible to identify which change caused the observed result.

Begin with very short sequences whose recurrence can be calculated manually. Print or visualize the hidden state at every time step and compare it with your own calculations. Only after the CPU implementation is understood should the same experiment be moved to CUDA. GPU execution must always be verified against a controlled reference before performance is measured.

Never shuffle time-series data without first asking whether the operation leaks information from the future. Never calculate scaling statistics using validation or test observations. Never allow padded positions to affect the loss. Never carry a hidden state from one unrelated sequence into another unless the experiment intentionally uses stateful training. These errors may not produce exceptions, but they can invalidate the entire experiment.

When studying long-term dependencies, do not rely only on the final loss. Inspect gradients at different time steps, probe hidden states, vary dependency distance, and measure where performance begins to collapse. When comparing RNN, GRU, and LSTM models, control parameter count, training data, optimization settings, and evaluation procedure as carefully as possible. A fair comparison requires more than placing three final accuracy values in a table.

For every completed challenge, the reader should be able to answer five questions: What information arrives at the current time step? What information is carried from the past? How is the hidden state transformed? How does the error travel backward through time? What evidence demonstrates that the model learned a temporal relationship rather than exploiting leakage or an accidental shortcut? If these questions cannot be answered, the experiment should be repeated before continuing.

15.1.3 The Chapters (RNN00 - RNN08) included in this book

- RNN00 - Sequence Modeling and Core RNN Intuition
- RNN01 - Hidden State Dynamics and Temporal Behavior
- RNN02 - Backpropagation Through Time (BPTT)
- RNN03 - Sequence Data Preparation and Batching
- RNN04 - Language Modeling with Vanilla RNN
- RNN05 - Time Series Forecasting with Vanilla RNN
- RNN06 - Regularization and Stabilization for RNNs
- RNN07 - Efficiency, Profiling, and CUDA Performance
- RNN08 - Transition to Gated RNNs (Why LSTM Exists)

15.2 RNN00 - Sequence Modeling and Core RNN Intuition

Models and Training

15.2.1 Exercise 00 - Understand Why Sequence Data Requires Memory Beyond Standard Feed-Forward Networks

Part 1 - Why this matters

A feed-forward network maps each input to an output without carrying an explicit record of earlier inputs. This assumption fails when the correct interpretation of the current value depends on what appeared before it.

Construct two sequences that end with the same current input but require different outputs because their histories differ. Use the experiment to prove why an independent feed-forward mapping cannot solve the task reliably.

Part 2 - Mathematical form

$$\hat{y}_t = f(x_t) \quad \text{versus} \quad \hat{y}_t = g(x_t, h_{t-1}).$$

Part 3 - Hints and Helper

Keep the final input identical in both sequences. If the desired outputs differ, the missing variable is information from history.

Part 4 - Python code skeleton

```
import torch

def E00(model, data, config):
    sequences = ...
    targets = ...
    current_only = ...
    with_history = ...
    return {"current_only": current_only, "with_history": with_history}
```

15.2.2 Exercise 01 - Represent a Sequence as Time Steps and Define the Input–Output Mapping Precisely

Part 1 - Why this matters

A sequence is not merely a longer tensor. Its axes describe time, samples, and features, while the target may correspond to one step, the next step, or the entire sequence.

Represent a small sequence explicitly as time steps and state the mapping being learned. Verify that every input time step and target index correspond to the intended task.

Part 2 - Mathematical form

$$X = (x_1, x_2, \ldots, x_T), \qquad F : X \mapsto Y.$$

Part 3 - Hints and Helper

Write down whether the task is many-to-one, one-step forecasting, or many-to-many before creating tensors.

Part 4 - Python code skeleton

```
import torch

def E01(model, data, config):
    sequence = ...
    time_steps = ...
    target = ...
    assert sequence.shape == (...)
    return time_steps, target
```

15.2.3 Exercise 02 - Implement a Simple RNN Forward Pass Using Pure Python Recurrence

Part 1 - Why this matters

Implementing recurrence without PyTorch layers reveals the exact order of temporal computation. Each new state must depend on both the current input and the state produced one step earlier.

Use Python scalars or lists to process a short sequence one element at a time. Save every hidden state so the complete recurrence can be checked manually.

Part 2 - Mathematical form

$$h_t = \tanh(w_x x_t + w_h h_{t-1} + b).$$

Part 3 - Hints and Helper

Initialize the hidden state once before the loop. Do not reset it inside the sequence.

Part 4 - Python code skeleton

```
import torch

def E02(model, data, config):
    import math

    def recurrence(sequence, wx, wh, bias):
        hidden = ...
        states = []
        for value in sequence:
            hidden = ...
            states.append(hidden)
        return states
```

15.2.4 Exercise 03 - Translate the RNN Recurrence Equation into PyTorch Tensor Operations

Part 1 - Why this matters

Tensor implementation extends the scalar recurrence to batches and hidden vectors while preserving the same mathematical rule. Shape mistakes can silently change which features are combined.

Implement one recurrent cell with matrix multiplication, bias addition, and a non-linearity. Unroll it across a tensor sequence and compare the result with a manually calculated tiny example.

Part 2 - Mathematical form

$$\mathbf{h}_t = \tanh(\mathbf{x}_t \mathbf{W}_{xh}^\mathsf{T} + \mathbf{h}_{t-1} \mathbf{W}_{hh}^\mathsf{T} + \mathbf{b}_h).$$

Part 3 - Hints and Helper

Annotate the expected shape after every matrix multiplication before executing the code.

Part 4 - Python code skeleton

```
import torch

def E03(model, data, config):
    batch, steps, input_size = data.shape
    hidden = torch.zeros(batch, config.hidden_size)
    states = []
    for t in range(steps):
        hidden = ...
        states.append(hidden)
    return torch.stack(states, dim=...)
```

15.2.5 Exercise 04 - Explain Hidden State as Compact Memory and Test Its Effect on Output Prediction

Part 1 - Why this matters

The hidden state is a fixed-size summary updated as the sequence is read. It is not a perfect archive, so its content depends on the learned transition and its limited capacity.

Run the same current input with different preceding histories and compare the hidden states and predictions. Then reset the state to demonstrate what information is lost.

Part 2 - Mathematical form

$$h_t = \Phi(x_t, h_{t-1}), \qquad \hat{y}_t = \Psi(h_t).$$

Part 3 - Hints and Helper

Control the final input and model weights. Change only the history or the initial hidden state.

Part 4 - Python code skeleton

```
import torch

def E04(model, data, config):
    history_a, history_b, current = ...
    h_a = run_history(model, history_a)
    h_b = run_history(model, history_b)
    y_a = model.step(current, h_a)
    y_b = ...
    return {"state_distance": ..., "outputs": (y_a, y_b)}
```

15.2.6 Exercise 05 - Demonstrate Weight Sharing Across Time Steps and Explain Why It Matters

Part 1 - Why this matters

An RNN reuses the same input and recurrent weights at every time step. This gives the model a consistent transition rule and prevents parameter count from growing with sequence length.

Apply one cell repeatedly, verify that the parameter objects remain identical, and compare shared recurrence with an intentionally unshared construction.

Part 2 - Mathematical form

$$h_t = \phi(W_x x_t + W_h h_{t-1} + b), \qquad \theta_t = \theta \ \forall t.$$

Part 3 - Hints and Helper

Distinguish changing hidden-state values from shared parameter values. Shared weights receive gradient contributions from all time steps.

Part 4 - Python code skeleton

```
import torch

def E05(model, data, config):
    cell = ...
    parameter_ids = []
    hidden = initial_hidden
    for x_t in sequence:
        parameter_ids.append(id(cell.weight_hh))
        hidden = ...
    assert len(set(parameter_ids)) == ...
    return hidden
```

15.2.7 Exercise 06 - Build a Tiny Synthetic Sequence Task Where History Is Required to Predict Correctly

Part 1 - Why this matters

A controlled synthetic task isolates temporal memory from unrelated data complexity. It allows you to know exactly which earlier event should determine the target.

Create sequences whose label depends on a marker shown several steps before the final input. Train simple baselines and confirm that a history-aware model outperforms a current-step-only model.

Part 2 - Mathematical form

$$y = \mathbf{1}[x_{T-d} = m], \qquad d > 0.$$

Part 3 - Hints and Helper

Balance the labels and prevent the final input from revealing the marker. Increase the delay to test memory difficulty.

Part 4 - Python code skeleton

```
import torch

def E06(model, data, config):
    def make_memory_dataset(samples, steps, delay):
        x = ...
        marker = ...
        y = ...
```

```
        hide_marker_from_last_step(x)
        return x, y

    train_data = make_memory_dataset(...)
    return train_and_compare(train_data, models=...)
```

15.2.8 Exercise 07 - Compare One-Step Prediction vs Full-Sequence Prediction in a Controlled Experiment

Part 1 - Why this matters

One-step prediction consumes a limited context to predict a specific target, while full-sequence prediction returns an output aligned with every time step. Their output shapes and loss alignment are different.

Use the same sequence to produce a final-step prediction and a prediction at every step. Define aligned targets and compare the amount of supervision each formulation provides.

Part 2 - Mathematical form

$$F_{1:T} : X_{1:T} \mapsto \hat{y}_T, \qquad G_{1:T} : X_{1:T} \mapsto (\hat{y}_1, \ldots, \hat{y}_T).$$

Part 3 - Hints and Helper

Do not compare losses until both reductions and numbers of supervised positions are defined consistently.

Part 4 - Python code skeleton

```
import torch

def E07(model, data, config):
    outputs, hidden = model(sequence)
    last_prediction = ...
    sequence_predictions = ...
    last_loss = criterion(last_prediction, last_target)
    full_loss = ...
    return last_loss, full_loss
```

15.2.9 Exercise 08 - Implement Many-to-One RNN for Sequence Classification and Validate Output Shapes

Part 1 - Why this matters

Many-to-one models compress a complete sequence into one representation for classification. Selecting the wrong axis or padded final state can make the classifier use an unintended time

step.

Build an RNN classifier that converts a batch of sequences into one logit vector per sample. Validate intermediate and final tensor shapes with assertions.

Part 2 - Mathematical form

$$X \in \mathbb{R}^{B\times T\times D} \longrightarrow H \in \mathbb{R}^{B\times T\times H} \longrightarrow Z \in \mathbb{R}^{B\times C}.$$

Part 3 - Hints and Helper

For equal-length sequences, use the last relevant state. Preserve the batch dimension even when the batch contains one sample.

Part 4 - Python code skeleton

```
import torch

def E08(model, data, config):
    rnn = torch.nn.RNN(..., batch_first=True)
    classifier = torch.nn.Linear(...)
    outputs, final_hidden = rnn(data)
    representation = ...
    logits = classifier(representation)
    assert logits.shape == (...)
    return logits
```

15.2.10 Exercise 09 - Implement Many-to-Many RNN for Sequence Labeling and Validate Temporal Alignment

Part 1 - Why this matters

Sequence labeling requires one prediction for each relevant input position. An off-by-one shift can produce a valid loss while teaching the model to label the wrong time step.

Project every recurrent output to class logits and verify that input positions, target positions, and output positions remain aligned. Include a deliberately shifted target as a failure comparison.

Part 2 - Mathematical form

$$X_{1:T} \mapsto H_{1:T} \mapsto Z_{1:T}, \qquad Z \in \mathbb{R}^{B\times T\times C}.$$

Part 3 - Hints and Helper

Flatten batch and time only for the loss, then confirm that the same ordering is used for logits and targets.

Part 4 - Python code skeleton

```
import torch

def E09(model, data, config):
    outputs, _ = rnn(data)
    logits = classifier(outputs)
    assert logits.shape[:2] == targets.shape
    loss = criterion(logits.reshape(...), targets.reshape(...))
    shifted_loss = ...
    return loss, shifted_loss
```

15.2.11 Exercise 10 - Move RNN Forward Computation to CUDA and Validate Output Parity with CPU Results

Part 1 - Why this matters

Moving an RNN to CUDA requires its parameters, inputs, and initial hidden state to share one device. Correct execution must be established before speed is measured.

Clone identical weights into CPU and CUDA models, run both in evaluation mode, and compare their outputs and final states after returning CUDA results to the CPU.

Part 2 - Mathematical form

$$e_{\max} = \max_i |y_i^{\text{CPU}} - y_i^{\text{CUDA}}|.$$

Part 3 - Hints and Helper

Use identical inputs and tolerances appropriate for the dtype. Availability of CUDA should be checked explicitly.

Part 4 - Python code skeleton

```
import torch

def E10(model, data, config):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    cpu_model = ...
    gpu_model = ...
    gpu_model.load_state_dict(cpu_model.state_dict())
    y_cpu, h_cpu = cpu_model(cpu_input)
    y_gpu, h_gpu = ...
    return torch.allclose(y_cpu, y_gpu.cpu(), atol=..., rtol=...)
```

15.2.12 Exercise 11 - Benchmark RNN Forward Speed on CPU vs GPU for Different Sequence Lengths

Part 1 - Why this matters

RNNs contain a sequential dependency across time, so GPU acceleration depends on sequence length, batch size, hidden size, and transfer cost. Small workloads may be faster on the CPU.

Benchmark several sequence lengths with fixed feature and hidden sizes. Separate warm-up from timed iterations and synchronize CUDA around measurements.

Part 2 - Mathematical form

$$S(T) = \frac{t_{\mathrm{CPU}}(T)}{t_{\mathrm{GPU}}(T)}, \qquad \text{throughput} = \frac{BT}{t}.$$

Part 3 - Hints and Helper

Generate inputs before timing and report median latency. Include end-to-end transfer cost as a separate measurement.

Part 4 - Python code skeleton

```
import torch

def E11(model, data, config):
    results = {}
    for steps in config.sequence_lengths:
        cpu_input = torch.randn(...)
        gpu_input = cpu_input.to("cuda")
        cpu_ms = benchmark(...)
        gpu_ms = benchmark(..., synchronize=True)
        results[steps] = ...
    return results
```

15.2.13 Exercise 12 - Compare float32 vs float16 RNN Forward Outputs and Identify Precision-Sensitive Cases

Part 1 - Why this matters

Recurrent updates repeatedly reuse earlier numerical results, allowing small precision differences to accumulate across time. Lower precision may therefore behave differently as sequences grow.

Run matched float32 and float16 models over several lengths and input scales. Measure output error, hidden-state error, and the first appearance of non-finite values.

Part 2 - Mathematical form

$$e_T = \frac{\|h_T^{(32)} - h_T^{(16)}\|_2}{\|h_T^{(32)}\|_2 + \epsilon}.$$

Part 3 - Hints and Helper

Copy the same weights before casting. Compare on supported hardware and treat overflow as a result rather than hiding it.

Part 4 - Python code skeleton

```
import torch

def E12(model, data, config):
    reference = model.float()
    candidate = copy_model(model).half().to(device)
    errors = []
    for steps in config.sequence_lengths:
        y32, h32 = ...
        y16, h16 = ...
        errors.append({"steps": steps, "relative_error": ..., "finite": ...})
    return errors
```

15.2.14 Exercise 13 - Understand Tensor Shapes for RNN Inputs (seq_len, batch, input_size) and Debug Mistakes

Part 1 - Why this matters

The standard PyTorch RNN layout places time before batch unless configured otherwise. Confusing these axes may run without an immediate exception when their sizes happen to be compatible.

Construct a tensor with uniquely identifiable values for every time, batch, and feature index. Permute axes intentionally, inspect slices, and create assertions that detect the wrong layout.

Part 2 - Mathematical form

$$X \in \mathbb{R}^{T \times B \times D}, \qquad Y \in \mathbb{R}^{T \times B \times H}.$$

Part 3 - Hints and Helper

Use different values for T, B, and D so an accidental permutation cannot hide behind equal dimensions.

Part 4 - Python code skeleton

```
import torch

def E13(model, data, config):
    seq_len, batch, input_size = ...
    x = torch.arange(...).reshape(seq_len, batch, input_size)
    validate_layout(x, expected=...)
    wrong = x.permute(...)
    try:
```

```
        validate_layout(wrong, expected=...)
    except ...:
        ...
    return x
```

15.2.15 Exercise 14 - Implement RNN with batch_first Option and Verify Shape Conversions Correctly

Part 1 - Why this matters

The `batch_first` option changes the layout of input and output tensors, but it does not change the hidden-state layout. Assuming that every tensor changes creates subtle bugs.

Run equivalent batch-first and time-first models with shared weights. Convert only the required axes and verify output and hidden-state parity.

Part 2 - Mathematical form

$$X_{BTD} = \text{permute}(X_{TBD}, 1, 0, 2), \qquad h_n \in \mathbb{R}^{L \times B \times H}.$$

Part 3 - Hints and Helper

Transpose the sequence outputs back before comparison. Do not transpose the final hidden state merely because batch-first is enabled.

Part 4 - Python code skeleton

```
import torch

def E14(model, data, config):
    time_model = torch.nn.RNN(..., batch_first=False)
    batch_model = torch.nn.RNN(..., batch_first=True)
    batch_model.load_state_dict(time_model.state_dict())
    x_batch = x_time.permute(...)
    y_time, h_time = time_model(x_time)
    y_batch, h_batch = ...
    return {"output_parity": ..., "hidden_parity": ...}
```

15.2.16 Exercise 15 - Implement a Manual Unroll of RNN Over Time and Compare to torch.nn.RNN Behavior

Part 1 - Why this matters

A manual unroll connects the recurrence equation to PyTorch's optimized RNN module. Matching both implementations proves that weight orientation, bias terms, and time ordering are understood.

Extract weights from a one-layer `torch.nn.RNN`, reproduce every step manually, and compare the complete output sequence and final hidden state.

Part 2 - Mathematical form

$$h_t = \tanh(x_t W_{ih}^{\mathsf{T}} + b_{ih} + h_{t-1} W_{hh}^{\mathsf{T}} + b_{hh}).$$

Part 3 - Hints and Helper

Use one layer, one direction, zero dropout, and the same initial hidden state before extending the experiment.

Part 4 - Python code skeleton

```
import torch

def E15(model, data, config):
    module = torch.nn.RNN(..., nonlinearity="tanh")
    module_output, module_hidden = module(sequence, initial_hidden)
    hidden = initial_hidden[0]
    manual_states = []
    for t in range(sequence.shape[0]):
        hidden = ...
        manual_states.append(hidden)
    manual_output = torch.stack(manual_states)
    return compare(module_output, manual_output, module_hidden, hidden)
```

15.2.17 Exercise 16 - Stress Test RNN Forward with Long Sequences and Monitor GPU Memory and Runtime

Part 1 - Why this matters

Long sequences increase recurrent work and retain more intermediate activations when gradients are enabled. A configuration that works for a short example may exceed practical memory or latency limits.

Increase sequence length gradually, measure runtime and peak GPU memory, and stop safely before an out-of-memory failure. Compare inference mode with gradient-enabled forward execution.

Part 2 - Mathematical form

$$\text{activation memory} = O(TBLH), \qquad \text{recurrent work} = O(TBH^2).$$

Part 3 - Hints and Helper

Reset peak-memory statistics between trials and record failed configurations explicitly. Do not allocate every test tensor at once.

Part 4 - Python code skeleton

```
import torch

def E16(model, data, config):
    records = []
    for steps in config.sequence_lengths:
        torch.cuda.reset_peak_memory_stats()
        sample = torch.randn(..., device=device)
        try:
            elapsed = ...
            peak = torch.cuda.max_memory_allocated()
            record = {"steps": steps, "ms": elapsed,
                      "peak_bytes": peak}
            records.append(record)
        except torch.cuda.OutOfMemoryError:
            ...
    return records
```

15.2.18 Exercise 17 - Debug Device Mismatch and Hidden-State Placement Errors in CUDA RNN Pipelines

Part 1 - Why this matters

An RNN may fail even when its model and input are on CUDA if the initial hidden state remains on the CPU. Creating state from global defaults also risks dtype mismatch.

Reproduce device and dtype failures deliberately, inspect every participating tensor, and implement a safe hidden-state constructor derived from the input or model parameters.

Part 2 - Mathematical form

$$\text{device}(X) = \text{device}(h_0) = \text{device}(\theta).$$

Part 3 - Hints and Helper

Create hidden state with `x.new_zeros` or explicit device and dtype. Avoid fixing errors by moving tensors repeatedly inside the time loop.

Part 4 - Python code skeleton

```
import torch

def E17(model, data, config):
    def initial_state(model, x, layers, hidden_size):
        return x.new_zeros(...)

    model = model.to(device)
    x = data.to(device)
    h0 = initial_state(model, x, ..., ...)
    assert_same_device_dtype(model, x, h0)
    return model(x, h0)
```

15.2.19 Exercise 18 - Validate Determinism Settings and Observe When CUDA RNN Becomes Non-Deterministic

Part 1 - Why this matters

Reproducibility requires more than setting one random seed. Initialization, data order, backend algorithms, precision, and device behavior can all influence repeated RNN outputs.

Run the same forward experiment several times under controlled seeds and deterministic settings. Compare exact equality and tolerance-based equality, then document remaining environmental constraints.

Part 2 - Mathematical form

$$d_r = \max_i |y_i^{(r)} - y_i^{(0)}|, \qquad \text{deterministic if } d_r = 0 \ \forall r.$$

Part 3 - Hints and Helper

Recreate the model and inputs after resetting every seed. Record software, hardware, dtype, and backend configuration with the result.

Part 4 - Python code skeleton

```
import torch

def E18(model, data, config):
    def run_once(seed, deterministic):
        set_all_seeds(seed)
        torch.use_deterministic_algorithms(deterministic)
        model, sample = ...
        return model(sample)[0].detach().cpu()

    runs = [run_once(config.seed, True) for _ in range(...)]
    return compare_repeated_runs(runs)
```

15.2.20 Exercise 19 - Write a Conceptual Report Explaining RNN Memory, Weight Sharing, and Temporal Computation

Part 1 - Why this matters

A complete understanding of recurrence must connect equations, tensor shapes, experiments, and failure cases. A final report makes this reasoning explicit before the book advances to BPTT and longer dependencies.

Produce an evidence-based report explaining how state flows forward, how parameters are reused, how task mappings differ, and where vanilla RNN forward computation fails. Link each claim to an experiment from this chapter.

Part 2 - Mathematical form

$$\mathcal{R} = \{\text{recurrence}, \text{state}, \text{sharing}, \text{mapping}, \text{shapes}, \text{evidence}, \text{limits}\}.$$

Part 3 - Hints and Helper

Include at least one manual calculation, one shape trace, one CPU–CUDA comparison, and one observed failure or limitation.

Part 4 - Python code skeleton

```
import torch

def E19(model, data, config):
    from pathlib import Path

    sections = {
        "recurrence": ...,
        "hidden_state": ...,
        "weight_sharing": ...,
        "experiments": ...,
        "limitations": ...,
    }
    report = render_report(sections)
    validate_required_evidence(report)
    Path(config.output_path).write_text(report, encoding="utf-8")
    return report
```

15.3 RNN01 - Hidden State Dynamics and Temporal Behavior

Models and Training

15.3.1 Exercise 00 - Visualize Hidden State Evolution Over Time for a Simple Repeating Pattern Sequence

Part 1 - Why this matters

A hidden state changes at every time step even when the same recurrent parameters are reused. Visualizing its components exposes cycles, saturation, drift, and responses to repeated input patterns.

Feed a controlled repeating sequence into a small RNN, retain every hidden state, and plot selected dimensions over time. Relate repeating or changing state patterns to the input period.

Part 2 - Mathematical form

$$h_t = \phi(W_{xh}x_t + W_{hh}h_{t-1} + b), \qquad H = (h_1, \ldots, h_T).$$

Part 3 - Hints and Helper

Use fixed weights and a short period first. Plot input and hidden dimensions against the same time axis.

Part 4 - Python code skeleton

```
import torch

def E00(model, data, config):
    pattern = ...
    sequence = pattern.repeat(...)
    outputs, hidden = model(sequence)
    states = outputs.detach().cpu()
    figure = plot_state_trajectories(sequence, states, dimensions=...)
    return states, figure
```

15.3.2 Exercise 01 - Analyze How Initial Hidden State Choice Affects Early Predictions and Convergence

Part 1 - Why this matters

The initial hidden state supplies the memory available before the first input arrives. Its influence is often strongest near the beginning and may decay, persist, or destabilize later states.

Run identical sequences from zero, random, and fixed nonzero initial states. Compare prediction differences across time and, during training, compare convergence under controlled seeds.

Part 2 - Mathematical form

$$\Delta_t = \|h_t^{(a)} - h_t^{(b)}\|_2.$$

Part 3 - Hints and Helper

Keep model parameters and inputs identical. Plot Δ_t to determine whether initialization influence disappears.

Part 4 - Python code skeleton

```
import torch

def E01(model, data, config):
    initial_states = {"zero": ..., "random": ..., "fixed": ...}
    runs = {}
```

```
    for name, h0 in initial_states.items():
        outputs, hidden = model(data, h0)
        runs[name] = ...
    differences = ...
    return runs, differences
```

15.3.3 Exercise 02 - Implement Learnable Initial Hidden State and Compare Against Zero Initialization

Part 1 - Why this matters

A learnable initial state can encode a useful task-level prior before any observation is processed. It also adds parameters that may overfit or become tied to a fixed batch assumption.

Register one initial-state parameter, expand it safely across the current batch, and compare it with zero initialization using the same data and training protocol.

Part 2 - Mathematical form

$$h_0 = \mathbf{c}, \qquad \mathbf{c} \in \mathbb{R}^{LD \times 1 \times H}, \qquad \mathbf{c} \text{ is trainable.}$$

Part 3 - Hints and Helper

Use `expand` rather than storing one parameter per batch element. Confirm that the parameter appears in the state dictionary and receives a gradient.

Part 4 - Python code skeleton

```
import torch

def E02(model, data, config):
    class LearnableStateRNN(torch.nn.Module):
        def __init__(self, input_size, hidden_size, layers=1):
            super().__init__()
            self.rnn = torch.nn.RNN(...)
            self.h0 = torch.nn.Parameter(...)
        def forward(self, x):
            batch = x.shape[0]
            initial = ...
            return self.rnn(x, initial)

    return compare_initialization_strategies(...)
```

15.3.4 Exercise 03 - Demonstrate Information Decay Over Long Sequences Using Hidden State Probing

Part 1 - Why this matters

A vanilla RNN may gradually lose information that appeared far in the past. Final accuracy alone does not show when or how the stored signal disappears.

Insert a known marker, increase the delay before probing the hidden state, and train a simple decoder to recover the marker. Plot probe accuracy against distance.

Part 2 - Mathematical form

$$A(d) = \Pr(q(h_{t+d}) = m_t)\,.$$

Part 3 - Hints and Helper

Use a balanced marker and a simple probe. Freeze the RNN while training the probe so the measurement does not alter the representation.

Part 4 - Python code skeleton

```
import torch

def E03(model, data, config):
    distances = ...
    probe_scores = {}
    with torch.no_grad():
        hidden_sequences = collect_states(model, data)
    for distance in distances:
        features, marker_targets = ...
        probe = train_probe(features, marker_targets)
        probe_scores[distance] = ...
    return probe_scores
```

15.3.5 Exercise 04 - Compare Single-Layer vs Multi-Layer RNN Hidden State Representations

Part 1 - Why this matters

Stacked recurrent layers build representations at different levels. Lower layers respond directly to sequence inputs, while higher layers receive transformed temporal features from below.

Train matched one-layer and multi-layer RNNs, inspect states from every layer, and compare probe accuracy, state norms, parameter counts, and task performance.

Part 2 - Mathematical form

$$h_t^{(l)} = \phi\Big(W_{xh}^{(l)} h_t^{(l-1)} + W_{hh}^{(l)} h_{t-1}^{(l)} + b^{(l)}\Big)\,.$$

Part 3 - Hints and Helper

A fair comparison should report the increased parameter count. Do not treat the final-layer state as if it contains every layer.

Part 4 - Python code skeleton

```
import torch

def E04(model, data, config):
    single = torch.nn.RNN(..., num_layers=1, batch_first=True)
    stacked = torch.nn.RNN(..., num_layers=config.layers, batch_first=True)
    single_result = analyze_model(single, data)
    stacked_result = analyze_model(stacked, data)
    return compare_representations(single_result, stacked_result)
```

15.3.6 Exercise 05 - Implement Stacked RNN and Confirm Correct Hidden State Shape at Each Layer

Part 1 - Why this matters

In a stacked RNN, every layer maintains its own recurrent state. Confusing the layer axis with the batch or time axis can produce incorrect initialization and state reuse.

Construct a multi-layer RNN, create the correct initial state, and use assertions or a manual stack to confirm the state associated with each layer.

Part 2 - Mathematical form

$$h_n \in \mathbb{R}^{L\times B\times H}, \qquad Y \in \mathbb{R}^{B\times T\times H}.$$

Part 3 - Hints and Helper

With a unidirectional RNN, the first hidden-state axis has length `num_layers`, not sequence length.

Part 4 - Python code skeleton

```
import torch

def E05(model, data, config):
    layers, batch, hidden_size = ...
    h0 = data.new_zeros(layers, batch, hidden_size)
    rnn = torch.nn.RNN(..., num_layers=layers, batch_first=True)
    outputs, h_n = rnn(data, h0)
    assert h_n.shape == (...)
    layer_states = ...
    return outputs, layer_states
```

15.3.7 Exercise 06 - Investigate the Effect of Hidden Size on Memory Capacity and Sequence Accuracy

Part 1 - Why this matters

Hidden size limits the dimensional space available for temporal representation. Too little capacity may discard useful history, while excessive capacity increases cost and can overfit.

Train several hidden sizes on the same delayed-memory task. Compare accuracy by delay, parameter count, runtime, and memory rather than selecting the largest model automatically.

Part 2 - Mathematical form

$$P(H) = HD + H^2 + H, \qquad C_{\text{step}} = O(B(HD + H^2)).$$

Part 3 - Hints and Helper

Reuse the dataset split, seed policy, optimizer budget, and evaluation procedure for every hidden size.

Part 4 - Python code skeleton

```
import torch

def E06(model, data, config):
    results = {}
    for hidden_size in config.hidden_sizes:
        candidate = build_rnn(hidden_size=hidden_size)
        history = train(candidate, train_loader, ...)
        results[hidden_size] = {
            "accuracy_by_delay": ...,
            "parameters": ...,
            "runtime_ms": ...,
        }
    return results
```

15.3.8 Exercise 07 - Implement Bidirectional RNN and Verify Forward and Backward State Concatenation

Part 1 - Why this matters

A bidirectional RNN produces one state from past-to-future processing and another from future-to-past processing. PyTorch interleaves directional states within the layer axis.

Build a one-layer bidirectional RNN, separate both directions, concatenate them explicitly, and verify agreement with the returned sequence output and final state layout.

Part 2 - Mathematical form

$$y_t = [\overrightarrow{h}_t; \overleftarrow{h}_t] \in \mathbb{R}^{2H}, \qquad h_n \in \mathbb{R}^{2L \times B \times H}.$$

Part 3 - Hints and Helper

For each layer, index forward direction at $2l$ and backward direction at $2l + 1$.

Part 4 - Python code skeleton

```
import torch

def E07(model, data, config):
    rnn = torch.nn.RNN(..., bidirectional=True, batch_first=True)
    outputs, h_n = rnn(data)
    forward_output, backward_output = outputs.split(
        config.hidden_size, dim=-1
    )
    forward_final = h_n[...]
    backward_final = ...
    assert outputs.shape == (...)
    return forward_output, backward_output, forward_final, backward_final
```

15.3.9 Exercise 08 - Compare Bidirectional vs Unidirectional RNN for a Sequence Labeling Task

Part 1 - Why this matters

Future context can improve a label at the current position when the entire sequence is available. That advantage is invalid in causal or streaming settings where later observations are unknown.

Train both directions under comparable capacity on a labeling task whose correct label sometimes requires future context. Compare accuracy, latency, parameters, and causal validity.

Part 2 - Mathematical form

$$\hat{y}_t^{\text{uni}} = f(x_{1:t}), \qquad \hat{y}_t^{\text{bi}} = g(x_{1:T}).$$

Part 3 - Hints and Helper

Create a task where future context has a measurable role, then state explicitly why the bidirectional model cannot be used online.

Part 4 - Python code skeleton

```
import torch

def E08(model, data, config):
    models = {"unidirectional": ..., "bidirectional": ...}
    results = {}
    for name, candidate in models.items():
        train(candidate, train_loader, ...)
        results[name] = evaluate_sequence_labels(candidate, test_loader)
        results[name]["parameters"] = ...
        results[name]["latency_ms"] = ...
    return results
```

15.3.10 Exercise 09 - Debug Hidden State Shape Bugs in Multi-Layer and Bidirectional RNN Setups

Part 1 - Why this matters

Layer count, direction count, batch size, and hidden size all appear in the hidden-state tensor. Multiplying or ordering them incorrectly produces common recurrent pipeline failures.

Create a validator for RNN configuration and initial-state shape. Test it against correct states and deliberately malformed layer, direction, batch, and hidden dimensions.

Part 2 - Mathematical form

$$\text{shape}(h_0) = (LD, B, H), \qquad D \in \{1, 2\}.$$

Part 3 - Hints and Helper

Derive D from `bidirectional`; never infer it from output width after an error has occurred.

Part 4 - Python code skeleton

```
import torch

def E09(model, data, config):
    def expected_hidden_shape(rnn, batch):
        directions = ...
        return (...)

    def validate_hidden(rnn, x, h0):
        expected = expected_hidden_shape(rnn, x.shape[0])
        if h0.shape != expected:
            raise ValueError(...)
        return True

    return run_shape_failure_tests(validate_hidden, cases=...)
```

15.3.11 Exercise 10 - Move Hidden State Analysis to CUDA Without Excess Host–Device Transfers

Part 1 - Why this matters

Copying every time-step state to the CPU inside the recurrent loop can synchronize execution repeatedly and destroy GPU throughput. Analysis should collect tensors on-device before transferring only the required summary.

Run hidden-state extraction on CUDA, compute selected statistics on the device, and perform one controlled transfer for visualization. Compare it with a deliberately inefficient per-step transfer.

Part 2 - Mathematical form

$$T_{\text{total}} = T_{\text{compute}} + N_{\text{copy}} T_{D \to H} + T_{\text{sync}}.$$

Part 3 - Hints and Helper

Detach states before analysis but keep them on CUDA until aggregation is complete. Transfer small summaries instead of full histories when possible.

Part 4 - Python code skeleton

```
import torch

def E10(model, data, config):
    model = model.to(device)
    batch = data.to(device, non_blocking=True)
    with torch.inference_mode():
        states, final_hidden = model(batch)
        summary_gpu = compute_state_statistics(states)
    summary_cpu = ...
    return summary_cpu
```

15.3.12 Exercise 11 - Benchmark Hidden State Computation Cost for Increasing Hidden Size on GPU

Part 1 - Why this matters

Recurrent weight multiplication grows approximately quadratically with hidden size. The measured curve also reflects GPU occupancy, memory traffic, and fixed launch overhead.

Benchmark matched RNNs across hidden sizes after warm-up. Record latency, throughput, parameter count, and peak memory to build an empirical scaling curve.

Part 2 - Mathematical form

$$C(H) = O(TB(HD + H^2)), \qquad P(H) = HD + H^2 + H.$$

Part 3 - Hints and Helper

Allocate each candidate independently and synchronize around timing. Keep batch size, sequence length, dtype, and device fixed.

Part 4 - Python code skeleton

```
import torch

def E11(model, data, config):
    records = []
    for hidden_size in config.hidden_sizes:
        candidate = build_rnn(hidden_size).to(device)
        sample = torch.randn(..., device=device)
        warmup(candidate, sample)
        latency = benchmark(candidate, sample, synchronize=True)
        records.append({"hidden_size": hidden_size,
                        "latency_ms": latency, "peak_memory": ...})
    return records
```

15.3.13 Exercise 12 - Compare Floating-Point Precision Effects on Hidden State Stability and Temporal Drift

Part 1 - Why this matters

Rounding error enters every recurrent update and may accumulate over many time steps. Precision differences that appear harmless early can change later states substantially.

Run identical float64, float32, and float16 recurrences over increasing lengths. Compare state trajectories to the highest-precision reference and detect saturation or non-finite values.

Part 2 - Mathematical form

$$d_t^{(p)} = \frac{\|h_t^{(64)} - h_t^{(p)}\|_2}{\|h_t^{(64)}\|_2 + \epsilon}.$$

Part 3 - Hints and Helper

Use identical weights and inputs before casting. Plot drift over time instead of reporting only the final difference.

Part 4 - Python code skeleton

```
import torch

def E12(model, data, config):
    precisions = ...
    trajectories = {}
    for dtype in precisions:
        candidate = copy_model(model).to(device=device, dtype=dtype)
        sample = data.to(device=device, dtype=dtype)
        trajectories[str(dtype)] = candidate(sample)[0].detach()
    reference = trajectories[...]
    return compute_temporal_drift(reference, trajectories)
```

15.3.14 Exercise 13 - Use Forward Hooks to Save Hidden States and Build a Visualization Pipeline

Part 1 - Why this matters

Instrumentation should observe internal behavior without rewriting the main model. Hooks are useful, but careless storage can retain computational graphs and cause memory growth.

Register hooks on recurrent components, detach and store selected outputs, remove all hook handles, and convert the captured states into a reproducible visualization.

Part 2 - Mathematical form

$$\mathcal{C}_l = \{\text{detach}(H^{(l)})\}, \qquad l = 1, \ldots, L.$$

Part 3 - Hints and Helper

A standard RNN hook receives a tuple containing sequence output and final state. Save only what the analysis requires.

Part 4 - Python code skeleton

```
import torch

def E13(model, data, config):
    captured = {}
    handles = []

    def save_state(name):
        def hook(module, inputs, output):
            sequence_output, final_state = output
            captured[name] = ...
        return hook

    for name, module in selected_modules(model):
        handles.append(module.register_forward_hook(save_state(name)))
```

```
    _ = model(data)
    for handle in handles:
        ...
    return build_visualization(captured)
```

15.3.15 Exercise 14 - Probe Hidden States to Detect Whether Past Information Is Encoded or Forgotten

Part 1 - Why this matters

A probe tests whether a specific property of the past can be recovered from a hidden representation. This provides stronger evidence about memory than inspecting state magnitude alone.

Freeze an RNN, collect hidden states at different delays, and train simple linear probes to predict a past attribute. Compare against majority and shuffled-label baselines.

Part 2 - Mathematical form

$$q_d^* = \arg\min_q \sum_i \ell(q(h_{t+d}^{(i)}), a_t^{(i)}).$$

Part 3 - Hints and Helper

Keep the probe simple and separate its training and test sets. High probe capacity may learn patterns that the representation does not expose directly.

Part 4 - Python code skeleton

```
import torch

def E14(model, data, config):
    with torch.no_grad():
        states, attributes = collect_probe_dataset(model, loader)
    results = {}
    for delay in config.delays:
        x_probe, y_probe = align_probe_data(states, attributes, delay)
        probe = train_linear_probe(x_probe, y_probe)
        results[delay] = evaluate_probe(probe, ...)
    results["shuffled_baseline"] = ...
    return results
```

15.3.16 Exercise 15 - Implement Teacher Forcing Conceptually and Observe Its Impact on Sequence Generation

Part 1 - Why this matters

During teacher forcing, the decoder receives the true previous target instead of its own previous prediction. This can accelerate learning but changes the distribution of inputs seen during training.

Implement an autoregressive loop with a configurable teacher-forcing ratio. Compare training loss and free-running generation quality under several ratios.

Part 2 - Mathematical form

$$u_t = \begin{cases} y_{t-1}, & z_t = 1, \\ \hat{y}_{t-1}, & z_t = 0, \end{cases} \qquad z_t \sim \text{Bernoulli}(r).$$

Part 3 - Hints and Helper

Sample the teacher-forcing decision explicitly and detach discrete sampled predictions before reusing them as inputs.

Part 4 - Python code skeleton

```
import torch

def E15(model, data, config):
    hidden = initial_hidden
    input_t = start_token
    outputs = []
    for t in range(targets.shape[1]):
        logits, hidden = decoder.step(input_t, hidden)
        outputs.append(logits)
        use_teacher = ...
        predicted = ...
        input_t = targets[:, t] if use_teacher else predicted
    return torch.stack(outputs, dim=1)
```

15.3.17 Exercise 16 - Investigate Exposure Bias in Autoregressive Sequence Generation with RNN

Part 1 - Why this matters

A teacher-forced model trains on correct prefixes but generates from prefixes containing its own mistakes. This train–inference mismatch can cause errors to compound over long outputs.

Evaluate the same model with true-prefix and free-running generation. Measure accuracy by generation step and quantify how the gap grows after the first error.

Part 2 - Mathematical form

$$\Delta_t = \mathbb{E}[\ell_t^{\text{free}}] - \mathbb{E}[\ell_t^{\text{teacher}}].$$

Part 3 - Hints and Helper

Use the same starting context. Record the first-error position and later recovery rather than only complete-sequence accuracy.

Part 4 - Python code skeleton

```
import torch

def E16(model, data, config):
    teacher_outputs = generate(model, data, mode="teacher_forced")
    free_outputs = generate(model, data, mode="free_running")
    teacher_error = error_by_step(teacher_outputs, targets)
    free_error = ...
    first_errors = ...
    return {"gap_by_step": free_error - teacher_error,
            "first_error_positions": first_errors}
```

15.3.18 Exercise 17 - Stress Test Very Long Sequences and Identify Practical Limits for Vanilla RNN Memory

Part 1 - Why this matters

A vanilla RNN may execute on a long sequence while failing to preserve the information needed from its beginning. Computational feasibility and usable temporal memory are different limits.

Increase dependency distance and sequence length while measuring probe accuracy, task accuracy, state similarity, runtime, and memory. Identify where information and engineering limits occur.

Part 2 - Mathematical form

$$d_{\text{memory}} = \max\{d : A(d) \geq \tau_A\}, \qquad T_{\text{system}} = \max\{T : M(T) \leq M_{\max}\}.$$

Part 3 - Hints and Helper

Report the memory horizon separately from the longest sequence that merely fits in memory.

Part 4 - Python code skeleton

```
import torch

def E17(model, data, config):
    records = []
    for length in config.sequence_lengths:
        sample, target, marker = make_long_memory_batch(length, ...)
        states, _ = model(sample.to(device))
        records.append({
```

```
            "length": length,
            "probe_accuracy": ...,
            "task_accuracy": ...,
            "runtime_ms": ...,
            "peak_memory": ...,
        })
    return identify_practical_limits(records, thresholds=...)
```

15.3.19 Exercise 18 - Build a Hidden State Debugging Checklist for Shapes, Devices, and Temporal Alignment

Part 1 - Why this matters

Recurrent bugs often produce plausible tensors while mixing layers, directions, batches, or time steps. A repeatable checklist prevents debugging from depending on intuition alone.

Create executable checks for configuration, state shapes, devices, dtypes, finite values, state-reset rules, padding alignment, temporal ordering, and expected gradient behavior.

Part 2 - Mathematical form

$$\text{valid} = S \wedge D \wedge P \wedge F \wedge A \wedge R.$$

Part 3 - Hints and Helper

Each failed check should name the expected and observed condition and point to the exact tensor or time boundary.

Part 4 - Python code skeleton

```
import torch

def E18(model, data, config):
    checks = {
        "shape": check_hidden_shapes(...),
        "device_dtype": check_device_dtype(...),
        "finite": check_finite_states(...),
        "alignment": check_temporal_alignment(...),
        "reset_policy": check_state_reset(...),
        "padding": check_padding_behavior(...),
    }
    failed = ...
    return {"passed": not failed, "checks": checks, "failed": failed}
```

15.3.20 Exercise 19 - Produce a Detailed Report Explaining Hidden State Dynamics and Their Failure Modes

Part 1 - Why this matters

A hidden-state analysis is complete only when observations are connected to temporal equations, controlled experiments, practical limits, and debugging evidence. The report consolidates those conclusions for later BPTT work.

Write a reproducible report covering state evolution, initialization, depth, directionality, capacity, precision, probing, generation mismatch, long-memory limits, and observed failures.

Part 2 - Mathematical form

$$\mathcal{R} = \{E, I, L, D, C, P, Q, G, M, F\}, \qquad \text{complete}(\mathcal{R}) = \bigwedge_{r \in \mathcal{R}} [r \neq \varnothing].$$

Part 3 - Hints and Helper

Include plots and configuration metadata, distinguish evidence from interpretation, and state what the experiments cannot prove.

Part 4 - Python code skeleton

```
import torch

def E19(model, data, config):
    from pathlib import Path

    sections = collect_evidence(
        evolution=..., initialization=..., depth=..., directionality=...,
        capacity=..., precision=..., probes=..., generation=...,
        memory_limits=..., failures=...)
    report = render_report(sections)
    validate_required_evidence(report)
    Path(config.output_path).write_text(report, encoding="utf-8")
    return report
```

15.4 RNN02 - Backpropagation Through Time (BPTT)

Models and Training

15.4.1 Exercise 00 - Define Backpropagation Through Time and Explain How It Extends Standard Backprop

Part 1 - Why this matters

An RNN reuses one transition across many time steps, but its loss depends on the chain of states produced by those repeated applications. Backpropagation through time exposes that recurrent chain as one deep computational graph.

Unroll a tiny RNN on paper and in code, mark every dependency from the loss to earlier states, and explain how shared parameters collect gradient contributions from multiple time steps.

Part 2 - Mathematical form

$$\frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^{T} \frac{\partial L}{\partial h_t} \frac{\partial h_t}{\partial W_{hh}}, \qquad \frac{\partial h_T}{\partial h_t} = \prod_{k=t+1}^{T} \frac{\partial h_k}{\partial h_{k-1}}.$$

Part 3 - Hints and Helper

Distinguish copies of an operation in the unrolled graph from the single shared parameter tensor used by every copy.

Part 4 - Python code skeleton

```
import torch

def E00(model, data, config):
    sequence = ...
    hidden = initial_hidden
    states = []
    for x_t in sequence:
        hidden = recurrent_step(x_t, hidden, parameters=...)
        states.append(hidden)
    loss = compute_loss(states, targets=...)
    graph_description = trace_dependencies(loss, states)
    return loss, graph_description
```

15.4.2 Exercise 01 - Implement an RNN Training Loop and Confirm Gradients Flow Through Time Steps

Part 1 - Why this matters

A complete recurrent training step must preserve the graph from relevant earlier states to the loss. Code can run and update parameters even when an accidental detach prevents temporal learning.

Train a small RNN on a history-dependent task, retain selected state gradients, and verify that early states receive nonzero gradients from a later loss.

Part 2 - Mathematical form

$$\theta \leftarrow \theta - \eta \nabla_\theta L, \qquad \frac{\partial L_T}{\partial h_t} \neq 0 \text{ for relevant } t < T.$$

Part 3 - Hints and Helper

Call gradient clearing before the forward pass, retain gradients only for the diagnostic states, and update parameters after backward.

Part 4 - Python code skeleton

```
import torch

def E01(model, data, config):
    model.train()
    optimizer.zero_grad(set_to_none=True)
    states, prediction = forward_with_retained_states(model, data)
    for state in states:
        state.retain_grad()
    loss = criterion(prediction, target)
    loss.backward()
    early_gradient = states[...].grad
    assert early_gradient is not None
    optimizer.step()
    return loss.item(), gradient_summary(states)
```

15.4.3 Exercise 02 - Inspect Gradient Norms Across Time and Detect Vanishing Gradient Behavior

Part 1 - Why this matters

Vanishing gradients make distant states almost insensitive to a later loss. Parameter gradients alone do not reveal where the temporal signal disappeared.

Retain gradients for every hidden state, backpropagate from a final-step loss, and plot gradient norm against temporal distance. Fit a log-scale trend to quantify decay.

Part 2 - Mathematical form

$$g_t = \left\| \frac{\partial L_T}{\partial h_t} \right\|_2, \qquad g_t \approx C\rho^{T-t}, \quad 0 < \rho < 1.$$

Part 3 - Hints and Helper

Use a controlled saturating recurrence and a log axis. Treat exactly zero and missing gradients as different cases.

Part 4 - Python code skeleton

```
import torch

def E02(model, data, config):
    states, final_output = forward_with_retained_states(model, data)
```

```
    loss = criterion(final_output, target)
    loss.backward()
    norms = []
    for state in states:
        norms.append(... if state.grad is not None else None)
    trend = fit_log_gradient_trend(norms)
    return {"norms": norms, "decay_rate": trend}
```

15.4.4 Exercise 03 - Inspect Gradient Norms Across Time and Detect Exploding Gradient Behavior

Part 1 - Why this matters

Repeated recurrent Jacobians can magnify the backward signal until gradients become extremely large or non-finite. This may cause unstable updates even when the forward loss initially looks reasonable.

Construct or train an unstable recurrence, record state and parameter gradient norms, and identify the first temporal distance or iteration where growth becomes dangerous.

Part 2 - Mathematical form

$$g_t \approx C\rho^{T-t}, \quad \rho > 1, \qquad G_\theta = \sqrt{\sum_p \|\nabla_p L\|_2^2}.$$

Part 3 - Hints and Helper

Log values before applying any clipping. Check finiteness separately from threshold-based explosion detection.

Part 4 - Python code skeleton

```
import torch

def E03(model, data, config):
    states, prediction = forward_with_retained_states(model, data)
    loss = criterion(prediction, target)
    loss.backward()
    state_norms = ...
    parameter_norm = total_gradient_norm(model.parameters())
    first_explosion = detect_threshold_crossing(state_norms, config.threshold)
    return {"state_norms": state_norms,
            "parameter_norm": parameter_norm,
            "first_explosion": first_explosion}
```

15.4.5 Exercise 04 - Apply Gradient Clipping and Measure Its Stabilization Effect Quantitatively

Part 1 - Why this matters

Gradient clipping limits an update when the global norm exceeds a chosen threshold. It can prevent catastrophic steps, but an overly small threshold can suppress useful learning.

Compare unclipped training with several clipping thresholds using identical initialization and data order. Record pre-clip norm, post-clip norm, update size, finite-loss rate, and convergence.

Part 2 - Mathematical form

$$g' = g \min\left(1, \frac{c}{\|g\|_2 + \epsilon}\right).$$

Part 3 - Hints and Helper

Apply clipping after backward and before the optimizer step. The value returned by norm clipping represents the norm before clipping.

Part 4 - Python code skeleton

```
import torch

def E04(model, data, config):
    optimizer.zero_grad(set_to_none=True)
    loss = training_loss(model, batch)
    loss.backward()
    pre_clip_norm = torch.nn.utils.clip_grad_norm_(
        model.parameters(), max_norm=...
    )
    post_clip_norm = total_gradient_norm(model.parameters())
    optimizer.step()
    return {"loss": loss.item(), "pre": ..., "post": post_clip_norm}
```

15.4.6 Exercise 05 - Implement Truncated BPTT and Compare Memory Usage and Accuracy Trade-Offs

Part 1 - Why this matters

Full BPTT retains dependencies across the complete sequence. Truncated BPTT processes shorter segments and detaches the carried state at boundaries, reducing graph depth and memory.

Split a long sequence into chunks, carry hidden values forward, detach them only between chunks, and compare several truncation lengths on memory, runtime, and task accuracy.

Part 2 - Mathematical form

$$h_{kb}^{\text{next}} = \text{stopgrad}(h_{kb}), \qquad b = \text{truncation length}.$$

Part 3 - Hints and Helper

Detaching should cut gradient history without resetting the numeric hidden state. Define whether optimization happens per chunk or after accumulated chunk losses.

Part 4 - Python code skeleton

```
import torch

def E05(model, data, config):
    hidden = model.initial_state(batch_size, device=device)
    metrics = []
    for start in range(0, sequence_length, config.truncation):
        x_chunk = data[:, start:start + config.truncation]
        y_chunk = target[:, start:start + config.truncation]
        hidden = hidden.detach()
        optimizer.zero_grad(set_to_none=True)
        output, hidden = model(x_chunk, hidden)
        loss = criterion(output, y_chunk)
        loss.backward()
        optimizer.step()
        metrics.append(...)
    return summarize_tbptt(metrics)
```

15.4.7 Exercise 06 - Compare Full BPTT vs Truncated BPTT on the Same Synthetic Long-Dependency Task

Part 1 - Why this matters

Truncation saves resources but may block the only gradient path connecting a distant marker to its target. The acceptable window therefore depends on the task's dependency length.

Train full and truncated variants from matched initial states on a delayed-memory task. Sweep truncation length and compare accuracy by dependency distance, memory, and runtime.

Part 2 - Mathematical form

$$A(b, d) = \Pr(\hat{y} = y \mid \text{window} = b, \text{dependency} = d).$$

Part 3 - Hints and Helper

Include cases where $b < d$ and $b \geq d$. Keep total examples, optimizer steps, and evaluation conditions comparable.

Part 4 - Python code skeleton

```
import torch

def E06(model, data, config):
    results = {}
    for method, truncation in config.methods:
        candidate = clone_initial_model(model)
        history = train_bptt(
            candidate, loader, truncation=truncation, epochs=config.epochs
        )
        results[method] = {
            "accuracy_by_delay": evaluate_delays(candidate, ...),
            "peak_memory": ...,
            "runtime": ...,
        }
    return results
```

15.4.8 Exercise 07 - Debug Silent Errors Caused by Hidden State Detaching at the Wrong Time

Part 1 - Why this matters

Detaching too early removes temporal credit assignment, while never detaching across independent chunks can connect unrelated data and grow the graph. Both mistakes may avoid immediate exceptions.

Implement correct, early-detach, late-detach, and reset-state variants. Inspect graph connectivity, early-state gradients, memory growth, and learning on a dependency task.

Part 2 - Mathematical form

$$\frac{\partial L_{t+b}}{\partial h_t} = 0 \quad \text{after an unintended detach.}$$

Part 3 - Hints and Helper

Verify numeric state continuity separately from gradient continuity. A state may preserve values while no longer preserving its graph.

Part 4 - Python code skeleton

```
import torch

def E07(model, data, config):
    def run_variant(detach_policy):
        hidden = initial_hidden
        early_states = []
        for chunk_index, chunk in enumerate(chunks):
            hidden = apply_detach_policy(hidden, detach_policy, chunk_index)
```

```
            output, hidden = model(chunk, hidden)
            early_states.append(hidden)
        loss = criterion(output, target)
        loss.backward()
        return inspect_connectivity_and_memory(early_states, hidden)

    return {name: run_variant(name) for name in ...}
```

15.4.9 Exercise 08 - Perform Finite Difference Gradient Checking for a Tiny RNN to Validate Backward Correctness

Part 1 - Why this matters

Finite differences provide an independent numerical estimate of a derivative. On a tiny double-precision RNN, they can reveal wrong recurrence equations, loss reductions, or manual backward logic.

Perturb each selected parameter positively and negatively, recompute the scalar loss, and compare the central-difference estimate with autograd.

Part 2 - Mathematical form

$$g_i^{\text{num}} = \frac{L(\theta_i + \epsilon) - L(\theta_i - \epsilon)}{2\epsilon}, \qquad e_i = \frac{|g_i^{\text{auto}} - g_i^{\text{num}}|}{|g_i^{\text{auto}}| + |g_i^{\text{num}}| + \delta}.$$

Part 3 - Hints and Helper

Use float64, a short sequence, smooth activations, and parameter restoration after every perturbation.

Part 4 - Python code skeleton

```
import torch

def E08(model, data, config):
    model = model.double()
    loss = scalar_loss(model, data.double(), target.double())
    loss.backward()
    autograd_values = ...
    numerical_values = {}
    for parameter, index in selected_entries(model):
        original = parameter.data[index].item()
        parameter.data[index] = original + config.epsilon
        loss_plus = ...
        parameter.data[index] = original - config.epsilon
        loss_minus = ...
        parameter.data[index] = original
        numerical_values[...] = ...
    return relative_gradient_errors(autograd_values, numerical_values)
```

15.4.10 Exercise 09 - Demonstrate Why Vanilla RNN Struggles with Long-Term Dependencies in Practice

Part 1 - Why this matters

Theoretical gradient products become meaningful when performance collapses as the dependency distance grows. A vanilla RNN may learn short delays while failing on the same rule at longer delays.

Train on a controlled copy or marker-recall task across increasing delays. Relate accuracy to early-state gradient norms and hidden-state probe accuracy.

Part 2 - Mathematical form

$$\frac{\partial L_T}{\partial h_t} = \frac{\partial L_T}{\partial h_T} \prod_{k=t+1}^{T} J_k.$$

Part 3 - Hints and Helper

Keep the underlying rule identical while changing only the delay. Compare against chance performance and report variability across seeds.

Part 4 - Python code skeleton

```
import torch

def E09(model, data, config):
    results = []
    for delay in config.delays:
        train_loader, test_loader = make_memory_task(delay=delay, ...)
        candidate = build_vanilla_rnn(...)
        train(candidate, train_loader, ...)
        results.append({
            "delay": delay,
            "accuracy": evaluate(candidate, test_loader),
            "early_gradient": measure_early_gradient(candidate, test_loader),
            "probe_accuracy": probe_past_marker(candidate, test_loader),
        })
    return results
```

15.4.11 Exercise 10 - Execute BPTT on CUDA and Validate Gradient Consistency with CPU Under Controlled Settings

Part 1 - Why this matters

CUDA and CPU may use different kernels and accumulation orders. A recurrent training step should be numerically consistent within tolerance before performance comparisons are

trusted.

Clone identical models, data, targets, and initial states onto both devices. Run forward and backward without optimizer updates and compare losses and every parameter gradient.

Part 2 - Mathematical form

$$e_p = \max_i |\nabla_p L_{\text{CPU},i} - \nabla_p L_{\text{CUDA},i}|.$$

Part 3 - Hints and Helper

Disable dropout, use deterministic settings where supported, and compare named parameters so no gradient is omitted.

Part 4 - Python code skeleton

```
import torch

def E10(model, data, config):
    cpu_model = clone_model(model).cpu()
    gpu_model = clone_model(model).cuda()
    gpu_model.load_state_dict(cpu_model.state_dict())
    cpu_loss = forward_backward(cpu_model, cpu_batch, ...)
    gpu_loss = forward_backward(gpu_model, gpu_batch, ...)
    gradient_errors = {}
    for (name_cpu, p_cpu), (name_gpu, p_gpu) in zip(
            cpu_model.named_parameters(), gpu_model.named_parameters()):
        assert name_cpu == name_gpu
        gradient_errors[name_cpu] = ...
    return {"loss_error": ..., "gradient_errors": gradient_errors}
```

15.4.12 Exercise 11 - Profile GPU Memory During BPTT and Identify Graph Retention Bottlenecks

Part 1 - Why this matters

BPTT stores activations needed for backward, so memory grows with sequence length, batch size, hidden size, and retained graphs. Saving loss tensors or states with attached graphs can create additional growth across iterations.

Profile allocated and peak memory across controlled configurations. Compare correct logging with graph-retaining logging and inspect whether memory returns to a stable pattern.

Part 2 - Mathematical form

$$M_{\text{BPTT}} \approx O(TBLH) + M_\theta + M_{\text{optimizer}}.$$

Part 3 - Hints and Helper

Log scalar values with a detached conversion. Do not diagnose allocator caching as a leak without examining live tensor references.

Part 4 - Python code skeleton

```
import torch

def E11(model, data, config):
    records = []
    loss_log = []
    for step, batch in enumerate(loader):
        torch.cuda.reset_peak_memory_stats()
        optimizer.zero_grad(set_to_none=True)
        loss = training_loss(model, batch)
        loss.backward()
        optimizer.step()
        loss_log.append(...)
        records.append({"step": step,
                        "allocated": torch.cuda.memory_allocated(),
                        "peak": torch.cuda.max_memory_allocated()})
    return analyze_memory_trend(records)
```

15.4.13 Exercise 12 - Compare float32 vs Mixed-Precision RNN Training and Detect Overflow or Underflow

Part 1 - Why this matters

Mixed precision can reduce memory and improve throughput, but recurrent gradients may already be extremely small or large. Scaling and finiteness checks are essential for interpreting failed updates.

Train matched float32 and mixed-precision runs, tracking loss scale, skipped optimizer steps, gradient norms, non-finite values, throughput, memory, and final quality.

Part 2 - Mathematical form

$$\widetilde{L} = sL, \qquad \nabla L = \frac{1}{s}\nabla \widetilde{L}.$$

Part 3 - Hints and Helper

Unscale gradients before clipping or measuring their true norm. Record when the scaler reduces its scale after detecting overflow.

Part 4 - Python code skeleton

```
import torch

def E12(model, data, config):
    scaler = torch.amp.GradScaler("cuda")
```

```
    optimizer.zero_grad(set_to_none=True)
    with torch.autocast(device_type="cuda", dtype=torch.float16):
        loss = training_loss(model, batch)
    scaler.scale(loss).backward()
    scaler.unscale_(optimizer)
    gradient_norm = ...
    non_finite = ...
    scaler.step(optimizer)
    scaler.update()
    return {"loss": loss.item(), "scale": scaler.get_scale(),
            "gradient_norm": gradient_norm, "non_finite": non_finite}
```

15.4.14 Exercise 13 - Use Autograd Hooks to Log Gradient Norms at Each Time Step and Summarize Results

Part 1 - Why this matters

Hooks can capture backward signals as they reach hidden states without rewriting the gradient computation. They must be registered on the exact tensors whose temporal gradients are being investigated.

Register one hook per retained hidden state, store detached scalar norms indexed by time, run backward, and summarize decay, growth, missing gradients, and non-finite values.

Part 2 - Mathematical form

$$r_t = \log_{10}\left(\left\|\frac{\partial L}{\partial h_t}\right\|_2 + \epsilon\right).$$

Part 3 - Hints and Helper

Capture the time index safely in the hook closure. Store numbers rather than gradient tensors to avoid retaining memory.

Part 4 - Python code skeleton

```
import torch

def E13(model, data, config):
    gradient_norms = {}
    handles = []
    for time_index, state in enumerate(states):
        def save_norm(gradient, t=time_index):
            gradient_norms[t] = ...
        handles.append(state.register_hook(save_norm))
    loss.backward()
    for handle in handles:
        handle.remove()
    return summarize_temporal_gradients(gradient_norms)
```

15.4.15 Exercise 14 - Demonstrate Gradient Accumulation When Forgetting zero_grad in an RNN Training Loop

Part 1 - Why this matters

PyTorch adds new gradients to existing parameter gradient buffers. Forgetting to clear them makes each optimizer step depend on previous batches unintentionally.

Run matched steps with correct clearing and with no clearing. Compare gradient tensors and parameter update magnitudes, then distinguish the bug from intentional gradient accumulation.

Part 2 - Mathematical form

$$g_{\text{stored}}^{(k)} = g_{\text{stored}}^{(k-1)} + \nabla_\theta L_k.$$

Part 3 - Hints and Helper

Use identical batches and disable the optimizer step during the first comparison so parameter changes do not confound accumulation.

Part 4 - Python code skeleton

```
import torch

def E14(model, data, config):
    model_a = clone_model(model)
    model_b = clone_model(model)
    batches = ...
    correct_norms, buggy_norms = [], []
    for batch in batches:
        model_a.zero_grad(set_to_none=True)
        training_loss(model_a, batch).backward()
        correct_norms.append(total_gradient_norm(model_a.parameters()))

        training_loss(model_b, batch).backward()
        buggy_norms.append(total_gradient_norm(model_b.parameters()))
    return {"correct": correct_norms, "buggy": buggy_norms}
```

15.4.16 Exercise 15 - Analyze Batch Size Impact on Gradient Noise and Convergence in RNN Training

Part 1 - Why this matters

Small batches produce noisier gradient estimates, while large batches consume more memory and may reduce the number of updates per epoch. Sequence padding can also change the effective amount of valid data.

Compare several batch sizes using the same dataset and optimization budget. Estimate gradient variance across batches and report convergence, throughput, memory, and valid-token count.

Part 2 - Mathematical form

$$\widehat{g}_B = \frac{1}{B}\sum_{i=1}^{B} g_i, \qquad \mathrm{Var}(\widehat{g}_B) \approx \frac{\Sigma}{B}.$$

Part 3 - Hints and Helper

Normalize loss consistently by valid tokens or samples. Avoid comparing a sum reduction across different batch sizes.

Part 4 - Python code skeleton

```
import torch

def E15(model, data, config):
    results = {}
    for batch_size in config.batch_sizes:
        loader = build_loader(batch_size=batch_size, ...)
        candidate = clone_initial_model(model)
        gradient_samples = collect_gradient_vectors(
            candidate, loader, count=...
        )
        results[batch_size] = {
            "gradient_variance": ...,
            "training_history": train(candidate, loader, ...),
            "throughput": ...,
            "peak_memory": ...,
        }
    return results
```

15.4.17 Exercise 16 - Debug NaN Loss by Checking Learning Rate, Initialization, and Sequence Length

Part 1 - Why this matters

A NaN loss is a symptom, not a root cause. In recurrent training, aggressive learning rates, unstable recurrent weights, long products, invalid data, or precision overflow may interact.

Build a diagnostic step that checks inputs, outputs, loss, gradients, and parameters for finiteness, then isolate learning rate, initialization scale, and sequence length through controlled sweeps.

Part 2 - Mathematical form

$$\text{finite}(X, \theta, H, L, \nabla\theta) = \bigwedge_{z} \mathbf{1}[|z| < \infty].$$

Part 3 - Hints and Helper

Stop at the first non-finite stage and record the preceding gradient and parameter norms. Do not continue optimizer updates after corruption.

Part 4 - Python code skeleton

```
import torch

def E16(model, data, config):
    diagnostics = {}
    for learning_rate, scale, steps in config.cases:
        candidate = build_model(initialization_scale=scale)
        batch = make_batch(sequence_length=steps)
        result = run_instrumented_step(candidate, batch, learning_rate)
        diagnostics[(learning_rate, scale, steps)] = {
            "first_non_finite_stage": ...,
            "gradient_norm": ...,
            "parameter_norm": ...,
        }
    return diagnose_root_causes(diagnostics)
```

15.4.18 Exercise 17 - Stress Test BPTT with Long Sequences and Large Hidden Size to Observe Practical Limits

Part 1 - Why this matters

Sequence length deepens the graph, while hidden size increases recurrent computation and stored activations. Their interaction determines runtime, memory, and numerical stability.

Sweep safe combinations of sequence length and hidden size, measure forward and backward time separately, peak memory, gradient norm, and task signal, and stop cleanly at system limits.

Part 2 - Mathematical form

$$C \approx O(TB(HD + H^2)), \qquad M \approx O(TBLH).$$

Part 3 - Hints and Helper

Increase one dimension at a time before testing a grid. Catch out-of-memory cases and record them instead of silently skipping them.

Part 4 - Python code skeleton

```
import torch

def E17(model, data, config):
    records = []
    for steps, hidden_size in config.grid:
        try:
            candidate = build_rnn(hidden_size).to(device)
            batch = make_batch(steps).to(device)
            forward_ms, loss = timed_forward(candidate, batch)
            backward_ms = timed_backward(loss)
            records.append({"steps": steps, "hidden": hidden_size,
                            "forward_ms": forward_ms,
                            "backward_ms": backward_ms,
                            "peak_memory": ..., "gradient_norm": ...})
        except torch.cuda.OutOfMemoryError:
            records.append({"steps": steps, "hidden": hidden_size,
                            "status": "out_of_memory"})
    return records
```

15.4.19 Exercise 18 - Build a BPTT Debugging Checklist for Detach Strategy, Truncation, and Gradient Clipping

Part 1 - Why this matters

BPTT failures span graph connectivity, temporal alignment, reduction, state boundaries, gradient scale, device placement, and optimizer order. A checklist turns these into executable release gates.

Create checks that verify the chosen truncation policy, hidden-state continuity, detach boundaries, nonzero early gradients, clipping order, finite values, gradient clearing, and memory stability.

Part 2 - Mathematical form

$$\text{valid} = G \wedge D \wedge T \wedge C \wedge Z \wedge F \wedge M.$$

Part 3 - Hints and Helper

Every failed check should include expected behavior, observed evidence, and the exact corrective action to investigate.

Part 4 - Python code skeleton

```
import torch

def E18(model, data, config):
    checks = {
        "graph_connectivity": check_early_state_gradient(...),
```

```
        "detach_boundaries": check_detach_policy(...),
        "truncation": check_truncation_alignment(...),
        "clipping_order": check_clipping_order(...),
        "gradient_clearing": check_gradient_clearing(...),
        "finite_values": check_finiteness(...),
        "memory_stability": check_memory_trend(...),
    }
    failed = {
        name: result
        for name, result in checks.items()
        if not result.passed
    }
    return {"passed": not failed, "failed": failed, "checks": checks}
```

15.4.20 Exercise 19 - Write a Full BPTT Engineering Report Explaining Experimental Failures and Fixes

Part 1 - Why this matters

A reliable BPTT implementation needs evidence connecting temporal derivatives, observed gradients, resource trade-offs, numerical behavior, and corrections. A final report preserves that engineering reasoning.

Produce a reproducible report covering full and truncated BPTT, gradient checking, vanishing and exploding behavior, clipping, detach errors, CUDA parity, memory, precision, batch size, NaNs, and practical limits.

Part 2 - Mathematical form

$$\mathcal{R} = \{U, G, V, E, C, T, D, N, P, B, S, F\}, \qquad \text{complete}(\mathcal{R}) = \bigwedge_{r \in \mathcal{R}} [r \neq \varnothing].$$

Part 3 - Hints and Helper

For every failure include reproduction conditions, measurements, root-cause evidence, applied fix, and a regression test.

Part 4 - Python code skeleton

```
import torch

def E19(model, data, config):
    from pathlib import Path

    evidence = collect_evidence(
        unrolling=..., gradients=..., vanishing=..., exploding=...,
        clipping=..., truncation=..., detach=..., numerical_check=...,
        cuda=..., memory=..., precision=..., stress_limits=...)
```

```
    report = render_engineering_report(evidence)
    validate_failure_fix_pairs(report)
    Path(config.output_path).write_text(report, encoding="utf-8")
    return report
```

15.5 RNN03 - Sequence Data Preparation and Batching

Models and Training

15.5.1 Exercise 00 - Convert Raw Sequential Data into Sliding Windows and Validate Correct Input–Target Alignment

Part 1 - Why this matters

A raw time series is an ordered record, not yet a supervised learning dataset. Sliding windows transform that record into examples by selecting a historical context and the future value or horizon that the model must predict.

The most dangerous error is an incorrect index relationship between a window and its target. The tensors may have valid shapes and the loss may decrease while the model learns the current value, an unintended horizon, or information that belongs to the future.

Build windows from a sequence with easily traceable values, write the expected indices manually, and test the first, middle, and final examples. The experiment must prove both shape correctness and semantic alignment.

Part 2 - Mathematical form

$$X_i = (s_i, \ldots, s_{i+w-1}), \qquad Y_i = (s_{i+w+g-1}, \ldots, s_{i+w+g+h-2}).$$

Part 3 - Hints and Helper

Define window length w, forecast gap g, and horizon h explicitly. Derive the number of valid samples before allocating tensors.

Part 4 - Python code skeleton

```
import torch

def E00(model, data, config):
    series = ...
    window, gap, horizon = ...
    inputs, targets, index_map = [], [], []
    last_start = ...
    for start in range(last_start):
        x = ...
        y = ...
```

```
        inputs.append(x)
        targets.append(y)
        index_map.append(...)
    validate_alignment(series, inputs, targets, index_map)
    return torch.stack(inputs), torch.stack(targets)
```

15.5.2 Exercise 01 - Implement a Custom Dataset for Sequence Pairs and Validate getitem Correctness

Part 1 - Why this matters

A custom dataset defines what one training example means. In sequence learning, it often maps an index to a window and target without materializing every overlapping window in memory.

An incorrect length calculation or boundary offset can omit valid examples, read beyond the series, or pair an input with the wrong target. Since the DataLoader trusts the dataset, that mistake propagates through the complete training pipeline.

Implement lazy indexing, reject invalid indices, and compare selected items against direct slices from the source. Also verify that repeated access is deterministic and does not mutate the raw data.

Part 2 - Mathematical form

$$N_{\text{samples}} = N - w - g - h + 2.$$

Part 3 - Hints and Helper

Test index 0, the last valid index, and an invalid index. Return tensors with a stable dtype and documented shapes.

Part 4 - Python code skeleton

```
import torch

def E01(model, data, config):
    class SequencePairDataset(torch.utils.data.Dataset):
        def __init__(self, series, window, gap, horizon):
            self.series = ...
            self.window = window
            self.gap = gap
            self.horizon = horizon
        def __len__(self):
            return ...
        def __getitem__(self, index):
            if index < 0 or index >= len(self):
                raise IndexError(...)
```

```
            x = ...
            y = ...
            return x, y

    dataset = SequencePairDataset(...)
    return validate_dataset_items(dataset, source=...)
```

15.5.3 Exercise 02 - Implement Padding for Variable-Length Sequences and Explain Why Padding Is Needed

Part 1 - Why this matters

A dense batch requires every sequence tensor to share one time dimension, but real sequences may contain different numbers of observations. Padding adds placeholder positions so the samples can be stacked and processed together.

Padding creates storage, not information. If the model or loss treats padded positions as real observations, batch composition changes the learned objective and shorter sequences contribute artificial values that never existed in the data.

Pad a batch manually and with the PyTorch utility, preserve the original lengths, and construct a validity mask. Verify that every original sequence is recovered exactly from its valid region.

Part 2 - Mathematical form

$$X_{i,t} = \begin{cases} s_{i,t}, & t < \ell_i, \\ p, & t \geq \ell_i, \end{cases} \qquad M_{i,t} = \mathbf{1}[t < \ell_i].$$

Part 3 - Hints and Helper

Choose a padding value that is documented, but never rely on the value alone to identify valid positions when that value can appear naturally.

Part 4 - Python code skeleton

```
import torch

def E02(model, data, config):
    sequences = ...
    lengths = torch.tensor([len(seq) for seq in sequences])
    max_length = ...
    padded = torch.full((len(sequences), max_length, ...), fill_value=...)
    for index, sequence in enumerate(sequences):
        padded[index, :lengths[index]] = ...
    mask = ...
    validate_round_trip(padded, mask, sequences)
    return padded, lengths, mask
```

15.5.4 Exercise 03 - Implement Packed Sequences with pack_padded_sequence and Validate Correctness

Part 1 - Why this matters

Packed sequences allow an RNN to skip padded time steps instead of performing recurrent updates on placeholders. This can save computation and prevents padded suffixes from altering the final state.

Packing changes the representation from a rectangular tensor to packed data plus batch-size metadata. Length order, CPU placement of length information, and the chosen batch layout must agree with the packing call.

Pack a variable-length batch, process it through an RNN, unpack the output, and compare each valid prefix with an independently processed sequence. This establishes semantic correctness rather than merely successful execution.

Part 2 - Mathematical form

$$P = \text{pack}(X, \ell), \qquad (Y, \ell) = \text{unpack}(\text{RNN}(P)).$$

Part 3 - Hints and Helper

Use `enforce_sorted=False` unless you intentionally sort. Check valid positions only; values beyond each restored length are padding.

Part 4 - Python code skeleton

```
import torch

def E03(model, data, config):
    padded, lengths = ...
    packed = torch.nn.utils.rnn.pack_padded_sequence(
        padded, lengths.cpu(), batch_first=True, enforce_sorted=...
    )
    packed_output, hidden = model(packed)
    output, restored_lengths = torch.nn.utils.rnn.pad_packed_sequence(
        packed_output, batch_first=True
    )
    parity = compare_valid_prefixes(model, sequences, output, lengths)
    return output, hidden, restored_lengths, parity
```

15.5.5 Exercise 04 - Debug Packing Bugs from Wrong Length Sorting and Misaligned Outputs

Part 1 - Why this matters

Packing may require sorting sequences by length, and sorting changes sample order. If targets, identifiers, or final outputs are not permuted consistently, predictions become attached to the wrong examples.

Another failure occurs when reported lengths include padding or exclude valid tokens. The RNN then processes unintended placeholders or truncates real observations, changing both outputs and gradients without necessarily producing an error.

Create deliberately unsorted sequences, record their IDs, sort and pack them, then restore the original order with an inverse permutation. Add tests for wrong lengths, inconsistent target sorting, and equal-length ties.

Part 2 - Mathematical form

$$X_s = X[\pi], \qquad Y_s = Y[\pi], \qquad \hat{Y} = \hat{Y}_s[\pi^{-1}].$$

Part 3 - Hints and Helper

Preserve the sorting permutation and compute its inverse explicitly. Verify restored request IDs before comparing predictions with targets.

Part 4 - Python code skeleton

```
import torch

def E04(model, data, config):
    lengths = ...
    sorted_lengths, permutation = lengths.sort(descending=True)
    inverse = torch.argsort(permutation)
    sorted_inputs = padded[permutation]
    sorted_targets = targets[permutation]
    packed = torch.nn.utils.rnn.pack_padded_sequence(
        sorted_inputs, sorted_lengths.cpu(), batch_first=True
    )
    sorted_outputs = run_and_unpack(model, packed, ...)
    outputs = sorted_outputs[inverse]
    return validate_restored_alignment(
        outputs, targets, ids, permutation, inverse
    )
```

15.5.6 Exercise 05 - Compare Padded vs Packed Training and Measure Speed and Memory Differences

Part 1 - Why this matters

Padded execution is simple and often efficient on regular hardware, but it spends computation on unused suffixes. Packed execution removes this waste, although packing, sorting, and

irregular sequence structure introduce their own overhead.

The benefit depends on the distribution of sequence lengths, not only on the maximum length. A nearly uniform batch may gain little from packing, while a highly imbalanced batch can waste most padded operations.

Train or benchmark both paths with identical weights and valid-token loss. Compare output parity on real positions, forward and backward time, peak memory, and the fraction of padding in each batch.

Part 2 - Mathematical form

$$r_{\text{pad}} = 1 - \frac{\sum_i \ell_i}{BT_{\max}}, \qquad S = \frac{t_{\text{padded}}}{t_{\text{packed}}}.$$

Part 3 - Hints and Helper

Warm up both paths and synchronize CUDA timing. Correctness comparison must ignore padded positions.

Part 4 - Python code skeleton

```
import torch

def E05(model, data, config):
    results = {}
    for mode in ["padded", "packed"]:
        candidate = clone_model(model)
        warmup(candidate, batch, mode=mode)
        results[mode] = benchmark_training_step(
            candidate, batch, mode=mode, valid_mask=mask
        )
    results["padding_ratio"] = ...
    results["valid_output_parity"] = ...
    return results
```

15.5.7 Exercise 06 - Build a Collate Function That Produces Batch-First Tensors with Correct Padding Masks

Part 1 - Why this matters

The collate function is the boundary between individual dataset examples and one training batch. It decides padding, lengths, target structure, identifiers, masks, and ordering in a single operation.

If it changes sample order without carrying IDs and targets, or constructs a mask with reversed meaning, every later component receives internally inconsistent data. Debugging the

model will not repair a corrupted batch contract.

Build a collator that accepts variable-length examples and returns a named batch-first structure. Test shapes, valid counts, recovered sequences, target alignment, and behavior for a batch of size one.

Part 2 - Mathematical form

$$X \in \mathbb{R}^{B \times T_{\max} \times D}, \qquad M \in \{0,1\}^{B \times T_{\max}}, \qquad \sum_t M_{i,t} = \ell_i.$$

Part 3 - Hints and Helper

Document whether mask value one means valid or padded. Use one convention consistently in the model and loss.

Part 4 - Python code skeleton

```
import torch

def E06(model, data, config):
    def collate_sequence_batch(examples):
        sequences, targets, ids = zip(*examples)
        lengths = torch.tensor(...)
        padded = torch.nn.utils.rnn.pad_sequence(
            sequences, batch_first=True, padding_value=...
        )
        time = torch.arange(padded.shape[1])
        valid_mask = ...
        return {"inputs": padded, "targets": ...,
                "lengths": lengths, "valid_mask": valid_mask, "ids": ids}

    return validate_collate_function(collate_sequence_batch, examples=...)
```

15.5.8 Exercise 07 - Validate Masking So Loss Ignores Padding Time Steps Correctly

Part 1 - Why this matters

A sequence loss should measure errors only where a real target exists. Averaging across the entire padded rectangle gives shorter samples less influence and allows arbitrary padded logits to change the reported objective.

Masking must be applied to the unreduced per-position loss before aggregation. Multiplying an already averaged scalar by a mask cannot recover which positions contributed to that scalar.

Compute the masked loss manually, compare it with a direct loss over concatenated valid positions, and alter padded logits aggressively. The valid loss must remain unchanged.

Part 2 - Mathematical form

$$L = \frac{\sum_{i,t} M_{i,t} \ell(z_{i,t}, y_{i,t})}{\sum_{i,t} M_{i,t}}.$$

Part 3 - Hints and Helper

Use a denominator equal to the valid position count. For class losses, flatten logits and targets consistently before masking.

Part 4 - Python code skeleton

```
import torch

def E07(model, data, config):
    per_step = criterion(logits.reshape(...), targets.reshape(...))
    per_step = per_step.reshape(targets.shape)
    valid = mask.to(per_step.dtype)
    masked_loss = ...
    direct_loss = criterion(
        logits[mask], targets[mask]
    )
    changed_logits = corrupt_only_padded_logits(logits, mask)
    assert masked_loss_is_invariant(changed_logits, ...)
    return masked_loss, direct_loss
```

15.5.9 Exercise 08 - Create a Variable-Length Synthetic Dataset and Verify the Training Pipeline Handles It

Part 1 - Why this matters

A synthetic dataset provides known sequence lengths, targets, and generating rules. It lets the pipeline be tested independently of noise and uncertainty in real data.

The dataset should include short, long, equal-length, and boundary cases while keeping the target rule learnable. This reveals assumptions such as nonempty sequences, fixed lengths, or masks that fail for the shortest sample.

Generate reproducible examples, pass them through the dataset, collator, DataLoader, model, and masked loss, then intentionally overfit a tiny subset. Successful overfitting is an end-to-end correctness test, not a generalization claim.

Part 2 - Mathematical form

$$\ell_i \sim \mathcal{D}_L, \qquad x_i \in \mathbb{R}^{\ell_i \times D}, \qquad y_i = f(x_i).$$

Part 3 - Hints and Helper

Store the seed and generation parameters. Validate each target directly from the unpadded source before batching.

Part 4 - Python code skeleton

```
import torch

def E08(model, data, config):
    class VariableLengthDataset(torch.utils.data.Dataset):
        def __init__(self, size, min_length, max_length, seed):
            generator = torch.Generator().manual_seed(seed)
            self.examples = ...
        def __len__(self):
            return len(self.examples)
        def __getitem__(self, index):
            return self.examples[index]

    dataset = VariableLengthDataset(...)
    loader = torch.utils.data.DataLoader(
        dataset, collate_fn=..., batch_size=...
    )
    return verify_and_overfit_tiny_subset(model, dataset, loader)
```

15.5.10 Exercise 09 - Debug Training Instability from Incorrect Padding Masks or Wrong Loss Reduction

Part 1 - Why this matters

An unstable loss is not always caused by the optimizer. If padded positions dominate the reduction, batches with different length distributions produce objectives at different effective scales.

A mask with reversed meaning can train entirely on padding, while a sum reduction makes gradient magnitude grow with the number of valid tokens. Both bugs can appear as learning-rate sensitivity, gradient explosion, or inconsistent convergence.

Instrument valid counts, masked and unmasked losses, gradient norms, and per-sample contributions. Compare correct masking against several deliberate failures to identify their statistical signatures.

Part 2 - Mathematical form

$$L_{\text{sum}} = \sum_{i,t} M_{i,t}\ell_{i,t}, \qquad L_{\text{mean}} = \frac{L_{\text{sum}}}{\sum_{i,t} M_{i,t}}.$$

Part 3 - Hints and Helper

Log the valid denominator for every batch. Reject all-padding batches before division rather than hiding them with a large epsilon.

Part 4 - Python code skeleton

```
import torch

def E09(model, data, config):
    diagnostics = {}
    for mask_mode, reduction in config.cases:
        optimizer.zero_grad(set_to_none=True)
        logits = model(batch["inputs"])
        loss = compute_sequence_loss(
            logits, batch["targets"], batch["valid_mask"],
            mask_mode=mask_mode, reduction=reduction
        )
        loss.backward()
        diagnostics[(mask_mode, reduction)] = {
            "loss": loss.item(), "valid_count": ...,
            "gradient_norm": ..., "sample_contributions": ...}
    return diagnose_mask_reduction_failures(diagnostics)
```

15.5.11 Exercise 10 - Move the Sequence Batching Pipeline to CUDA Efficiently and Minimize Host-to-Device Transfers

Part 1 - Why this matters

GPU efficiency depends on feeding complete batches rather than moving individual sequences or time steps repeatedly. Each small transfer can introduce synchronization and prevent overlap between data loading and computation.

Not every field belongs on CUDA. Inputs, targets, and masks used in tensor computation should move together, while identifiers and metadata may remain on the CPU. Length tensors may also have API-specific placement requirements.

Implement a single batch-transfer function, use pinned memory and non-blocking copies when supported, and measure transfer time separately from model time. Verify that no hidden transfer remains inside the recurrent loop.

Part 2 - Mathematical form

$$T_{\text{step}} = T_{\text{load}} + T_{H \to D} + T_{\text{compute}} + T_{\text{sync}}.$$

Part 3 - Hints and Helper

Do not call CUDA transfer on each time slice. Preserve CPU lengths when required for sequence packing.

Part 4 - Python code skeleton

```
import torch

def E10(model, data, config):
```

```
    def move_batch(batch, device):
        return {
            "inputs": batch["inputs"].to(device, non_blocking=True),
            "targets": batch["targets"].to(device, non_blocking=True),
            "valid_mask": batch["valid_mask"].to(device, non_blocking=True),
            "lengths": ...,
            "ids": batch["ids"],
        }

    device_batch = move_batch(batch, device)
    transfer_ms, compute_ms = benchmark_transfer_and_compute(
        model, device_batch
    )
    return transfer_ms, compute_ms
```

15.5.12 Exercise 11 - Benchmark DataLoader Throughput with num_workers and pin_memory Settings

Part 1 - Why this matters

The DataLoader can become the bottleneck when sequence slicing, padding, sorting, or collation is expensive. GPU utilization then falls even if the model itself is fast.

More workers do not guarantee higher throughput. Process startup, inter-process transfer, storage speed, CPU contention, batch size, and the cost of the collator determine the useful configuration.

Benchmark a controlled grid of worker and pinned-memory settings after warm-up. Measure batches per second, valid tokens per second, wait time, and end-to-end training throughput rather than timing loading in isolation only.

Part 2 - Mathematical form

$$Q = \frac{\sum_b \sum_i \ell_{b,i}}{T_{\text{wall}}}, \qquad U_{\text{input}} = \frac{T_{\text{wait}}}{T_{\text{step}}}.$$

Part 3 - Hints and Helper

Use the same dataset order and number of batches. On CUDA, test pinned memory together with non-blocking transfer.

Part 4 - Python code skeleton

```
import torch

def E11(model, data, config):
    records = []
    for workers in config.worker_counts:
        for pin_memory in [False, True]:
```

```
            loader = build_loader(
                num_workers=workers, pin_memory=pin_memory,
                persistent_workers=...
            )
            result = benchmark_loader_and_training(loader, model, device)
            records.append({"workers": workers, "pin_memory": pin_memory,
                            "batches_per_second": ...,
                            "valid_tokens_per_second": ...,
                            "step_time": ...})
    return records
```

15.5.13 Exercise 12 - Implement Gradient Accumulation for Long Sequences Under Limited GPU Memory

Part 1 - Why this matters

Long sequences may prevent the desired effective batch from fitting in GPU memory. Gradient accumulation divides that batch into smaller micro-batches while delaying the optimizer update.

Equivalence requires loss scaling and consistent valid-token normalization. Simply dividing every micro-batch loss by the number of micro-batches is exact only when their contributions have equal weight under the chosen reduction.

Implement accumulation, compare its gradient with a true combined batch on a small case, and handle the final incomplete accumulation group. Measure peak memory and update frequency.

Part 2 - Mathematical form

$$g = \frac{1}{N_{\text{valid}}} \sum_{m=1}^{M} \sum_{i,t \in m} M_{i,t} \nabla_\theta \ell_{i,t}.$$

Part 3 - Hints and Helper

Accumulate a valid-loss sum and valid count when micro-batches contain different sequence lengths. Clip only after the full accumulated gradient is ready.

Part 4 - Python code skeleton

```
import torch

def E12(model, data, config):
    optimizer.zero_grad(set_to_none=True)
    valid_total = ...
    for micro_index, batch in enumerate(micro_batches):
        loss_sum, valid_count = sequence_loss_sum(model, batch)
        scaled_loss = ...
```

```
        scaled_loss.backward()
        should_step = ...
        if should_step:
            optional_clip(model.parameters(), ...)
            optimizer.step()
            optimizer.zero_grad(set_to_none=True)
    return compare_with_true_large_batch(model, reference_model, ...)
```

15.5.14 Exercise 13 - Validate Reproducibility of Sequence Batching Under Random Shuffling and Fixed Seeds

Part 1 - Why this matters

Reproducible training requires reproducible batch membership, order, padding, random synthetic generation, and worker behavior. Setting only the model seed does not control the data pipeline.

Multiple DataLoader workers have independent random states, and persistent workers change when initialization occurs. Random transformations inside a dataset add another source of variation.

Record sample IDs and batch signatures for repeated runs, reset generators consistently, and compare both identical-seed and different-seed behavior. Document deterministic ordering separately from deterministic model kernels.

Part 2 - Mathematical form

$$\mathcal{B}_r = (I_{r,1}, \ldots, I_{r,K}), \qquad \mathcal{B}_a = \mathcal{B}_b \text{ when seeds and settings match.}$$

Part 3 - Hints and Helper

Pass an explicit generator to the DataLoader and seed every worker from its assigned initial seed.

Part 4 - Python code skeleton

```
import torch

def E13(model, data, config):
    def collect_batch_order(seed):
        generator = torch.Generator().manual_seed(seed)
        loader = build_loader(
            shuffle=True, generator=generator, worker_init_fn=...,
            num_workers=config.num_workers
        )
        return [tuple(batch["ids"]) for batch in loader]

    order_a = collect_batch_order(config.seed)
```

```
    order_b = collect_batch_order(config.seed)
    order_c = collect_batch_order(config.seed + 1)
    return {"same_seed_equal": ..., "different_seed_equal": ...,
            "order_a": order_a}
```

15.5.15 Exercise 14 - Implement a Train–Validation Split Without Leakage in Sequential Time-Series Data

Part 1 - Why this matters

Randomly splitting overlapping windows can place nearly identical contexts on both sides of the evaluation boundary. It may also train on observations that occur after validation targets, violating the forecasting scenario.

Leakage can enter before splitting when scaling, imputation, feature engineering, or window construction uses the complete timeline. A chronological index split alone is insufficient if preprocessing has already observed the future.

Choose a time boundary, fit preprocessing only on the training region, and construct windows whose source and target indices remain within their assigned region. Add a gap when overlapping context near the boundary must be excluded.

Part 2 - Mathematical form

$$\max(\mathcal{T}_{\text{train}}) < \min(\mathcal{T}_{\text{val}}), \qquad \theta_P = \text{fit}(X_{\text{train}}).$$

Part 3 - Hints and Helper

Store raw timestamps for every window and assert that no validation target timestamp appears in training inputs or preprocessing statistics.

Part 4 - Python code skeleton

```
import torch

def E14(model, data, config):
    boundary = ...
    gap = ...
    train_raw = series.loc[:boundary]
    validation_raw = series.loc[boundary + gap:]
    preprocessor = fit_preprocessor(train_raw)
    train_data = make_windows(
        preprocessor.transform(train_raw), ...
    )
    validation_data = make_windows(
        preprocessor.transform(validation_raw), ...
    )
    leakage_report = audit_temporal_indices(
```

```
        train_data, validation_data, boundary=boundary, gap=gap
    )
    assert leakage_report["passed"]
    return train_data, validation_data, leakage_report
```

15.5.16 Exercise 15 - Implement Teacher-Forcing Dataset Format for Sequence-to-Sequence Tasks

Part 1 - Why this matters

A sequence-to-sequence decoder usually receives a shifted target prefix and predicts the next target at every step. The decoder input and expected output contain related values but must not occupy the same temporal positions.

Special start, end, and padding tokens affect lengths and masks. An incorrect shift can expose the answer directly to the decoder or ask it to predict a target that is one position too early or late.

Construct encoder input, decoder input, and decoder target explicitly, then verify every pair token by token. Test normal, minimum-length, and padded examples before training.

Part 2 - Mathematical form

$$D_{\text{in}} = (\langle s \rangle, y_1, \ldots, y_{T-1}), \qquad D_{\text{target}} = (y_1, \ldots, y_T).$$

Part 3 - Hints and Helper

Padding masks for decoder input and target may differ around special tokens. Print one complete formatted example with its IDs.

Part 4 - Python code skeleton

```
import torch

def E15(model, data, config):
    def format_seq2seq(source, target, start_id, end_id, pad_id):
        encoder_input = ...
        decoder_input = ...
        decoder_target = ...
        return {
            "encoder_input": encoder_input,
            "decoder_input": decoder_input,
            "decoder_target": decoder_target,
            "target_mask": ...,
        }

    formatted = [format_seq2seq(...) for source, target in examples]
    return validate_teacher_forcing_shift(formatted)
```

15.5.17 Exercise 16 - Implement Stateful RNN Batching and Decide When to Reset Hidden States Correctly

Part 1 - Why this matters

Stateful training carries hidden values from one batch segment to the next. This is valid only when each batch column continues the same logical stream in the correct temporal order.

Ordinary shuffling breaks that continuity, and a smaller final batch changes which state belongs to which sample. Carrying state across unrelated users, documents, or series leaks context between independent examples.

Design a stream-aware sampler with stable stream IDs, detach state at graph boundaries, reset it at real sequence boundaries, and test continuity with traceable values. Compare stateful and stateless behavior explicitly.

Part 2 - Mathematical form

$$h_{k,0} = \begin{cases} h_{k-1,T}, & \text{same stream continues,} \\ 0, & \text{new stream begins.} \end{cases}$$

Part 3 - Hints and Helper

Track hidden state by stream ID rather than batch position unless batch membership is guaranteed to remain fixed.

Part 4 - Python code skeleton

```
import torch

def E16(model, data, config):
    state_by_stream = {}
    reset_policy = ...
    for batch in stream_loader:
        stream_ids = batch["stream_ids"]
        initial = assemble_states(
            state_by_stream, stream_ids,
            reset_flags=reset_policy(batch["reset"])
        )
        output, final_state = model(batch["inputs"], initial.detach())
        update_states(state_by_stream, stream_ids, final_state)
        verify_stream_continuity(batch, state_by_stream)
    return state_by_stream
```

15.5.18 Exercise 17 - Detect and Fix Alignment Bugs That Make an RNN Learn the Wrong Target Shift

Part 1 - Why this matters

Sequence models can achieve a deceptively low loss when targets are shifted incorrectly. For slowly changing signals, predicting the present instead of the future may still look accurate.

Visual inspection of aggregate loss cannot reveal which source timestamp produced each target. Alignment must be represented and tested at the index level before tensors lose their original temporal labels.

Create a monotonic diagnostic series, store source and target indices, and test the intended horizon for every window. Compare a correct dataset with zero-shift, double-shift, and off-by-one variants.

Part 2 - Mathematical form

$$\text{target_index}(i) = i + w + g - 1 \quad \text{for horizon one.}$$

Part 3 - Hints and Helper

Use values equal to their original indices during testing. This makes a wrong shift immediately visible in tensor contents.

Part 4 - Python code skeleton

```
import torch

def E17(model, data, config):
    diagnostic_series = torch.arange(..., dtype=torch.float32)
    variants = {
        "correct": make_windows(...),
        "zero_shift": make_windows(...),
        "off_by_one": make_windows(...),
    }
    reports = {}
    for name, dataset in variants.items():
        reports[name] = audit_input_target_indices(
            dataset,
            expected_window=...,
            expected_gap=...,
            expected_horizon=...,
        )
    return reports
```

15.5.19 Exercise 18 - Build a Sequence Data Pipeline Checklist Covering Padding, Packing, and Masking

Part 1 - Why this matters

A sequence pipeline is a chain of contracts: raw ordering, window alignment, split boundaries, preprocessing, dataset indexing, collation, padding, packing, masking, transfer, and loss

reduction. A failure in an early contract invalidates all later model results.

Many of these bugs are silent because the resulting tensors retain valid shapes. A checklist must therefore verify semantic invariants such as timestamp order, target horizon, valid counts, sample identity, and preprocessing scope.

Convert the checklist into executable gates with expected values and evidence. Run it on normal batches and on deliberately corrupted cases so every check is proven capable of failing.

Part 2 - Mathematical form

$$\text{ready} = O \wedge A \wedge S \wedge P \wedge D \wedge C \wedge K \wedge M \wedge R.$$

Part 3 - Hints and Helper

A passing result must include evidence, not only a Boolean. Report the exact sample IDs and indices involved in a failure.

Part 4 - Python code skeleton

```
import torch

def E18(model, data, config):
    checks = {
        "ordering": check_raw_order(...),
        "alignment": check_window_target_alignment(...),
        "split": check_temporal_split(...),
        "preprocessing": check_fit_scope(...),
        "dataset": check_dataset_boundaries(...),
        "collate": check_batch_contract(...),
        "packing": check_pack_round_trip(...),
        "masking": check_masked_loss(...),
        "reproducibility": check_batch_reproducibility(...),
    }
    return evaluate_pipeline_gates(checks, corrupted_cases=...)
```

15.5.20 Exercise 19 - Produce a Full Sequence Data Preparation Report with Verified Edge Cases

Part 1 - Why this matters

Model quality cannot be interpreted without a precise account of how raw sequences became training examples. The preparation report preserves the temporal assumptions that are otherwise hidden inside indexing and batching code.

The report must distinguish mathematical task definition from storage format and runtime optimization. Sliding windows define the learning problem; padding and packing represent

batches; masks define valid supervision; DataLoader settings affect delivery performance.

Document evidence for alignment, leakage prevention, preprocessing scope, variable lengths, packing, masking, reproducibility, CUDA transfer, stateful boundaries, and edge cases. Every conclusion should link to a reproducible test or artifact.

Part 2 - Mathematical form

$$\mathcal{R} = \{R, W, S, P, D, C, K, M, L, G, E\}, \qquad \text{complete}(\mathcal{R}) = \bigwedge_{r \in \mathcal{R}} [r \neq \varnothing].$$

Part 3 - Hints and Helper

Include empty or minimum-length sequences, batch size one, final incomplete batch, equal lengths, extreme length imbalance, and split-boundary windows.

Part 4 - Python code skeleton

```
import torch

def E19(model, data, config):
    from pathlib import Path

    evidence = collect_pipeline_evidence(
        raw_data=..., windows=..., split=..., preprocessing=...,
        dataset=..., collate=..., padding=..., packing=..., masking=...,
        cuda_transfer=..., reproducibility=..., edge_cases=...)
    report = render_data_preparation_report(evidence)
    validate_required_edge_cases(report)
    Path(config.output_path).write_text(report, encoding="utf-8")
    return report
```

15.6 RNN04 - Language Modeling with Vanilla RNN

Models and Training

15.6.1 Exercise 00 - Implement Character-Level Tokenization and Build a Vocabulary with Correct Index Mapping

Part 1 - Why this matters

A language model cannot process characters directly; it receives integer identifiers whose meanings are defined by a vocabulary. Character tokenization preserves spaces, punctuation, capitalization, and line breaks as distinct events when the vocabulary includes them.

A nondeterministic or inconsistent mapping changes the meaning of saved weights. If training and inference assign different IDs to the same character, the model receives and produces

the wrong symbols without any tensor-shape error.

Build a deterministic vocabulary, implement encoding and decoding, define behavior for unknown characters, and prove round-trip correctness on normal and edge-case text. Save the mapping as part of the model artifact.

Part 2 - Mathematical form

$$V = \text{sorted}(\text{unique}(C)), \qquad e : V \to \{0, \ldots, |V| - 1\}, \qquad d(e(c)) = c.$$

Part 3 - Hints and Helper

Test spaces, repeated characters, punctuation, newlines, and unseen characters. Never rebuild the vocabulary independently during inference.

Part 4 - Python code skeleton

```
import torch

def E00(model, data, config):
    text = ...
    characters = ...
    stoi = {character: index for index, character in enumerate(characters)}
    itos = {index: character for character, index in stoi.items()}
    encoded = ...
    decoded = ...
    assert decoded == text
    return {"stoi": stoi, "itos": itos, "encoded": encoded}
```

15.6.2 Exercise 01 - Build a Simple Character-Level RNN Language Model and Verify Output Logit Shapes

Part 1 - Why this matters

A character language model transforms token IDs into embeddings, processes their temporal order with an RNN, and projects every hidden state to one score per vocabulary character. These scores are logits, not probabilities.

The model must preserve batch and time dimensions while changing only the feature dimension. Flattening time too early or selecting only the final state silently converts next-character modeling into a different task.

Implement the architecture and assert shapes at embedding, recurrent, and projection stages. Confirm that the output contains a complete vocabulary distribution for every input position.

Part 2 - Mathematical form

$$X \in \mathbb{N}^{B\times T} \to E \in \mathbb{R}^{B\times T\times D} \to H \in \mathbb{R}^{B\times T\times H} \to Z \in \mathbb{R}^{B\times T\times |V|}.$$

Part 3 - Hints and Helper

Return the final hidden state separately from logits. Do not apply softmax before the cross-entropy loss.

Part 4 - Python code skeleton

```
import torch

def E01(model, data, config):
    class CharacterRNN(torch.nn.Module):
        def __init__(self, vocab_size, embedding_size, hidden_size):
            super().__init__()
            self.embedding = torch.nn.Embedding(...)
            self.rnn = torch.nn.RNN(..., batch_first=True)
            self.output = torch.nn.Linear(...)
        def forward(self, tokens, hidden=None):
            embedded = ...
            states, hidden = ...
            logits = ...
            assert logits.shape == (...)
            return logits, hidden

    return CharacterRNN(...)
```

15.6.3 Exercise 02 - Implement Cross-Entropy Loss for Next-Character Prediction and Validate Target Alignment

Part 1 - Why this matters

Next-character training uses the character at position t to predict the character at position $t+1$. Input and target sequences therefore overlap but are shifted by exactly one position.

An incorrect shift can expose the answer, predict the current token, or skip an intended character. Because neighboring characters are correlated, a misaligned model may still show a decreasing loss.

Construct a traceable text example, print every input-target pair, flatten logits and targets consistently, and compare the PyTorch loss with a manual negative log-probability calculation.

Part 2 - Mathematical form

$$x = (c_1, \ldots, c_{T-1}), \qquad y = (c_2, \ldots, c_T), \qquad L = -\frac{1}{B(T-1)} \sum_{b,t} \log p(y_{b,t} \mid x_{b,1:t}).$$

Part 3 - Hints and Helper

Check alignment before batching. The class dimension must be last before reshaping logits to two dimensions.

Part 4 - Python code skeleton

```
import torch

def E02(model, data, config):
    inputs = encoded[:, :-1]
    targets = ...
    logits, _ = model(inputs)
    assert logits.shape[:2] == targets.shape
    loss = torch.nn.functional.cross_entropy(
        logits.reshape(...), targets.reshape(...)
    )
    manual_loss = ...
    return loss, manual_loss, display_token_pairs(inputs, targets, itos=...)
```

15.6.4 Exercise 03 - Train a Tiny RNN Language Model Until It Overfits a Small Text Sample on Purpose

Part 1 - Why this matters

Intentionally overfitting a tiny text sample is an end-to-end correctness test. A model with enough capacity should memorize the training transitions when tokenization, alignment, gradients, optimization, and hidden-state handling are correct.

Failure to overfit does not prove that the architecture is weak; it often reveals a pipeline bug, excessive regularization, an unsuitable learning rate, or a state reset error. Conversely, memorization does not demonstrate generalization.

Train on one short repeated text until loss and token accuracy reach strict targets, then generate from a known prefix. Record the configuration so the test becomes a reusable regression check.

Part 2 - Mathematical form

$$\theta^* = \arg\min_{\theta} L_{\text{tiny}}(\theta), \qquad A_{\text{train}} \to 1.$$

Part 3 - Hints and Helper

Disable dropout and augmentation. Verify that the same tiny batch is used and stop after either the success threshold or a fixed maximum iteration count.

Part 4 - Python code skeleton

```
import torch

def E03(model, data, config):
    model.train()
    history = []
    for step in range(config.max_steps):
        optimizer.zero_grad(set_to_none=True)
        logits, _ = model(tiny_inputs)
        loss = criterion(logits.reshape(...), tiny_targets.reshape(...))
        loss.backward()
        optimizer.step()
        accuracy = ...
        history.append({"step": step, "loss": loss.item(),
                        "accuracy": accuracy})
        if loss.item() < ... and accuracy >= ...:
            break
    return history, generate_from_prefix(model, ...)
```

15.6.5 Exercise 04 - Generate Text Autoregressively and Observe Repetition and Instability Patterns

Part 1 - Why this matters

Autoregressive generation feeds each sampled character back as the next input. The model therefore conditions on its own growing output rather than the correct prefixes seen during training.

A small local mistake changes the future context and may push generation into repetitive loops, invalid transitions, or unstable patterns. These failures can remain hidden when evaluation uses teacher-forced inputs only.

Implement generation one step at a time, preserve hidden state across generated tokens, and log probabilities and entropy. Compare several prompts and identify where repetition begins.

Part 2 - Mathematical form

$$c_{t+1} \sim p_\theta(\cdot \mid c_{1:t}), \qquad h_{t+1} = f(c_{t+1}, h_t).$$

Part 3 - Hints and Helper

Prime the hidden state with the complete prompt, then generate from its final character. Use inference mode and an explicit maximum length.

Part 4 - Python code skeleton

```
import torch

def E04(model, data, config):
```

```
model.eval()
tokens = encode(prompt)
hidden = None
with torch.inference_mode():
    for token in tokens[:-1]:
        _, hidden = model(token.reshape(...), hidden)
    current = tokens[-1].reshape(...)
    generated = list(tokens)
    diagnostics = []
    for step in range(config.generate_length):
        logits, hidden = model(current, hidden)
        probabilities = ...
        next_token = ...
        generated.append(next_token.item())
        diagnostics.append(...)
        current = next_token.reshape(...)
return decode(generated), diagnostics
```

15.6.6 Exercise 05 - Implement Temperature Sampling and Analyze Diversity vs Coherence

Part 1 - Why this matters

Temperature rescales logits before the probability distribution is formed. Low temperature sharpens preferences and makes common continuations more likely, while high temperature distributes probability across less likely characters.

Temperature does not improve the learned model; it changes how that model is sampled. Extremely low values approach greedy decoding, while high values can expose weakly learned transitions and produce incoherence.

Generate multiple samples at controlled temperatures and seeds. Measure entropy, repetition, novelty, invalid-character behavior, and qualitative coherence instead of judging one convenient example.

Part 2 - Mathematical form

$$p_i(\tau) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}, \qquad \tau > 0.$$

Part 3 - Hints and Helper

Reject nonpositive temperature and apply scaling to logits before softmax. Compare many samples per setting.

Part 4 - Python code skeleton

```
import torch

def E05(model, data, config):
    def sample_with_temperature(logits, temperature, generator):
        if temperature <= 0:
            raise ValueError(...)
        probabilities = ...
        return torch.multinomial(probabilities, 1, generator=generator)

    results = {}
    for temperature in config.temperatures:
        samples = [generate(model, sampler=sample_with_temperature,
                            temperature=temperature, seed=seed)
                   for seed in config.seeds]
        results[temperature] = evaluate_generation_set(
            samples, training_text=...
        )
    return results
```

15.6.7 Exercise 06 - Implement Top-k Sampling and Compare with Temperature-Only Sampling

Part 1 - Why this matters

Top-k sampling restricts selection to the k highest-scoring characters. It prevents very low-probability symbols from being sampled while preserving randomness among the retained candidates.

The method interacts with temperature: temperature changes relative probability within the selected set, while k changes which characters are allowed at all. A small k may cause loops, and a large k may resemble unrestricted sampling.

Implement top-k filtering manually, verify that excluded probabilities become zero, and compare several (k, au) combinations using identical prompts and seeds.

Part 2 - Mathematical form

$$S_k = \mathrm{TopK}(z), \qquad p_i = 0 \text{ if } i \notin S_k.$$

Part 3 - Hints and Helper

Validate $1 \leq k \leq |V|$. Apply temperature and top-k consistently before normalization.

Part 4 - Python code skeleton

```
import torch

def E06(model, data, config):
```

```
    def top_k_sample(logits, k, temperature, generator):
        if k < 1 or k > logits.shape[-1]:
            raise ValueError(...)
        scaled = ...
        values, indices = torch.topk(scaled, k, dim=-1)
        filtered = torch.full_like(scaled, float("-inf"))
        filtered.scatter_(..., indices, values)
        probabilities = ...
        token = torch.multinomial(probabilities, 1, generator=generator)
        return token, probabilities

    return compare_sampling_grid(model, ks=..., temperatures=...)
```

15.6.8 Exercise 07 - Debug Generation Bugs Caused by Incorrect Hidden-State Reset Behavior

Part 1 - Why this matters

Hidden state contains the processed context. Resetting it at every generated character removes long context, while carrying it across unrelated prompts leaks information from one request into another.

Another common bug processes the prompt once and then processes its final token again, duplicating context. Stateful batching can also attach a hidden state to the wrong sample when generation requests finish at different times.

Implement correct reset, per-token reset, never reset, and duplicated prompt variants. Compare deterministic outputs and hidden-state traces to identify the signature of each failure.

Part 2 - Mathematical form

$$h_0^{(r)} = 0 \text{ for a new request}, \qquad h_t^{(r)} \not\to h_0 \text{ within the same request.}$$

Part 3 - Hints and Helper

Define request boundaries explicitly. Test prompts independently and then in different execution orders to detect cross-request leakage.

Part 4 - Python code skeleton

```
import torch

def E07(model, data, config):
    variants = {}
    for policy in ["correct", "reset_each_token",
                   "carry_across_prompts", "duplicate_last_prompt_token"]:
        variants[policy] = []
        state_store = ...
```

```
        for prompt in prompts:
            text, state_trace = generate_with_state_policy(
                model, prompt, policy=policy, state_store=state_store
            )
            variants[policy].append({"text": text, "trace": state_trace})
    return detect_reset_failures(variants)
```

15.6.9 Exercise 08 - Compare Sequence Length and Its Effect on Learning Short vs Long Patterns

Part 1 - Why this matters

Training sequence length determines how much context is visible in one unrolled graph. Short segments are cheaper and produce more updates, but they cannot directly assign credit across dependencies longer than the segment.

Long segments expose wider context while increasing memory, runtime, and gradient instability. Carrying hidden state between short segments preserves numeric context, but detaching it limits backward credit assignment.

Train matched models across sequence lengths on text containing known short and long patterns. Evaluate losses and accuracies by dependency type as well as throughput and memory.

Part 2 - Mathematical form

$$L_T = -\frac{1}{T}\sum_{t=1}^{T} \log p(c_{t+1} \mid c_{1:t}), \qquad M = O(TBH).$$

Part 3 - Hints and Helper

Keep the number of processed tokens comparable. Report whether hidden state is reset, carried, or detached between segments.

Part 4 - Python code skeleton

```
import torch

def E08(model, data, config):
    results = {}
    for sequence_length in config.sequence_lengths:
        dataset = build_language_windows(text, sequence_length)
        candidate = clone_initial_model(model)
        history = train_language_model(candidate, dataset, ...)
        results[sequence_length] = {
            "short_pattern_score": ...,
            "long_pattern_score": ...,
            "tokens_per_second": ...,
            "peak_memory": ...,
```

```
            "history": history,
        }
    return results
```

15.6.10 Exercise 09 - Evaluate an RNN Language Model with Perplexity and Interpret What the Number Means

Part 1 - Why this matters

Perplexity is the exponential of average negative log-likelihood. It summarizes how much probability the model assigns to the observed next characters under a specific tokenizer and dataset.

A lower value indicates better predictive probability on that evaluation distribution, but perplexity is not directly comparable across different vocabularies, tokenization rules, or inconsistent loss reductions. It also does not fully measure generated-text quality.

Compute token-weighted validation loss without gradients, convert it to perplexity, compare against simple baselines, and inspect per-position surprise so the aggregate number has context.

Part 2 - Mathematical form

$$\mathrm{PPL} = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log p(c_i \mid c_{<i})\right).$$

Part 3 - Hints and Helper

Accumulate summed loss and valid token count before division. Guard against exponent overflow and report the average NLL alongside perplexity.

Part 4 - Python code skeleton

```
import torch

def E09(model, data, config):
    model.eval()
    negative_log_likelihood = 0.0
    token_count = 0
    with torch.inference_mode():
        for inputs, targets in validation_loader:
            logits, _ = model(inputs)
            loss_sum = torch.nn.functional.cross_entropy(
                logits.reshape(...), targets.reshape(...), reduction="sum"
            )
            negative_log_likelihood += loss_sum.item()
            token_count += ...
    average_nll = ...
    perplexity = ...
    return {"nll": average_nll, "perplexity": perplexity,
            "baseline": ...}
```

15.6.11 Exercise 10 - Train a Character RNN on CUDA and Measure Speed Improvement over a CPU Baseline

Part 1 - Why this matters

CUDA acceleration depends on batch size, sequence length, hidden size, vocabulary projection, and data delivery. A small character model may spend more time in transfer and launch overhead than in useful computation.

A fair CPU–GPU comparison requires identical model weights, token batches, optimizer behavior, and measured work. CUDA operations are asynchronous, so unsynchronized timers produce misleading results.

Train controlled steps on both devices, validate loss and gradient consistency, and report tokens per second, step latency, memory, and end-to-end transfer cost after warm-up.

Part 2 - Mathematical form

$$S = \frac{Q_{\text{GPU}}}{Q_{\text{CPU}}}, \qquad Q = \frac{BT}{t_{\text{step}}}.$$

Part 3 - Hints and Helper

Generate or load batches outside the timed region, synchronize CUDA at measurement boundaries, and include transfer cost separately.

Part 4 - Python code skeleton

```
import torch

def E10(model, data, config):
    cpu_model = clone_model(model).cpu()
    gpu_model = clone_model(model).cuda()
    gpu_model.load_state_dict(cpu_model.state_dict())
    cpu_result = benchmark_training(
        cpu_model, cpu_batches, device="cpu", warmup=...
    )
    gpu_result = benchmark_training(
        gpu_model, gpu_batches, device="cuda", warmup=..., synchronize=True
    )
    parity = compare_controlled_step(cpu_model, gpu_model, batch=...)
    return {"cpu": cpu_result, "gpu": gpu_result, "parity": parity}
```

15.6.12 Exercise 11 - Profile GPU Utilization and Fix Language-Model Input Pipeline Bottlenecks

Part 1 - Why this matters

Low GPU utilization often means the accelerator is waiting for tokenization, window construction, collation, transfer, or Python work. Increasing model complexity to raise utilization does not fix the data bottleneck.

Character windows are cheap mathematically but can be expensive when rebuilt one sample at a time. Pre-encoding text, vectorized slicing, pinned batches, and persistent workers may reduce idle gaps.

Profile loading, transfer, forward, backward, and optimizer phases separately. Change one pipeline setting at a time and verify that throughput improves without changing token order or targets.

Part 2 - Mathematical form

$$T_{\text{step}} = T_{\text{wait}} + T_{H\to D} + T_f + T_b + T_o.$$

Part 3 - Hints and Helper

Measure valid tokens per second and GPU idle gaps. Preserve one correctness batch for parity checks after every optimization.

Part 4 - Python code skeleton

```
import torch

def E11(model, data, config):
    baseline = profile_training_pipeline(model, loader, device)
    experiments = {}
    for setting in config.pipeline_settings:
        candidate_loader = build_optimized_loader(setting, ...)
        profile = profile_training_pipeline(model, candidate_loader, device)
        parity = compare_batch_contract(loader, candidate_loader, fixture=...)
        experiments[setting.name] = {"profile": profile, "parity": parity}
    return identify_input_bottlenecks(baseline, experiments)
```

15.6.13 Exercise 12 - Implement Mixed-Precision Training and Detect Instability in Long-Sequence Language Modeling

Part 1 - Why this matters

Mixed precision reduces activation memory and may improve throughput, allowing longer sequences or larger batches. Recurrent gradient products and a large vocabulary projection can nevertheless produce underflow or overflow.

A finite loss does not guarantee that every gradient is usable. The scaler may skip optimizer steps, while small recurrent gradients can disappear even when output-layer gradients remain measurable.

Compare full and mixed precision across sequence lengths, logging loss scale, skipped steps, layer-wise gradient norms, non-finite values, memory, throughput, and validation perplexity.

Part 2 - Mathematical form

$$\widetilde{L} = sL, \qquad \nabla L = s^{-1}\nabla \widetilde{L}.$$

Part 3 - Hints and Helper

Unscale before clipping and true gradient inspection. Record the scaler value before and after each update.

Part 4 - Python code skeleton

```
import torch

def E12(model, data, config):
    scaler = torch.amp.GradScaler("cuda")
    optimizer.zero_grad(set_to_none=True)
    scale_before = scaler.get_scale()
    with torch.autocast(device_type="cuda", dtype=torch.float16):
        logits, _ = model(inputs)
        loss = criterion(logits.reshape(...), targets.reshape(...))
    scaler.scale(loss).backward()
    scaler.unscale_(optimizer)
    gradient_report = ...
    scaler.step(optimizer)
    scaler.update()
    return {"loss": loss.item(), "scale_before": scale_before,
            "scale_after": scaler.get_scale(), "gradients": gradient_report}
```

15.6.14 Exercise 13 - Compare Vanilla RNN with GRU or LSTM and Document Why Vanilla RNN Fails

Part 1 - Why this matters

Vanilla RNN, GRU, and LSTM all process sequences recurrently, but gated models regulate memory updates through learned paths. Their additional parameters aim to preserve useful information and gradients over longer dependencies.

A comparison is meaningful only when data, tokenization, training budget, evaluation, and approximate capacity are controlled. A stronger gated result alone does not identify which vanilla limitation caused the difference.

Train all architectures on text with measurable short and long patterns. Compare perplexity by dependency distance, gradient flow, convergence, generation, runtime, memory, and parameter count.

Part 2 - Mathematical form

$$h_t^{\text{RNN}} = \tanh(W_x x_t + W_h h_{t-1} + b), \qquad h_t^{\text{gated}} = G(x_t, h_{t-1}; \theta).$$

Part 3 - Hints and Helper

Report both matched-hidden-size and approximately matched-parameter comparisons when possible.

Part 4 - Python code skeleton

```
import torch

def E13(model, data, config):
    architectures = {"rnn": ..., "gru": ..., "lstm": ...}
    results = {}
    for name, candidate in architectures.items():
        history = train_language_model(candidate, train_loader, ...)
        results[name] = {
            "perplexity_by_dependency": ...,
            "gradient_flow": ...,
            "generation_metrics": ...,
            "parameters": ..., "runtime": ..., "memory": ...,
            "history": history,
        }
    return controlled_architecture_comparison(results)
```

15.6.15 Exercise 14 - Implement Truncated BPTT for Language Modeling and Quantify Memory Savings

Part 1 - Why this matters

A long text stream can be divided into temporal chunks while hidden state carries context forward. Detaching state at chunk boundaries limits how far gradients travel backward and prevents the graph from growing indefinitely.

Truncated BPTT preserves numeric memory beyond a chunk but not credit assignment across the detach boundary. This distinction affects patterns whose cause and target fall in different chunks.

Implement stream-contiguous chunks, compare truncation lengths, and measure peak memory, runtime, perplexity, and accuracy on dependencies shorter and longer than the truncation window.

Part 2 - Mathematical form

$$h_{kb}^{+} = \text{stopgrad}(h_{kb}), \qquad M_b < M_T.$$

Part 3 - Hints and Helper

Do not shuffle chunks within a carried stream. Reset state at true document boundaries and detach at configured graph boundaries.

Part 4 - Python code skeleton

```
import torch

def E14(model, data, config):
    hidden = None
    records = []
    for chunk, targets, boundary in stream_chunks:
        if boundary:
            hidden = ...
        elif hidden is not None:
            hidden = detach_state(hidden)
        optimizer.zero_grad(set_to_none=True)
        logits, hidden = model(chunk, hidden)
        loss = criterion(logits.reshape(...), targets.reshape(...))
        loss.backward()
        optimizer.step()
        records.append({"loss": loss.item(), "peak_memory": ...})
    return summarize_truncation_tradeoffs(records, dependency_tests=...)
```

15.6.16 Exercise 15 - Debug Training Divergence through Learning Rate, Clipping, and Initialization

Part 1 - Why this matters

Divergence can arise when recurrent weights amplify states and gradients, the learning rate converts a large gradient into a destructive update, or initialization places activations near saturation. These causes interact.

Changing several controls simultaneously may stop NaNs without identifying the cause. Reliable debugging observes the first unstable stage and varies one factor under matched seeds and batches.

Sweep learning rate, clipping threshold, and recurrent initialization scale while logging states, logits, loss, gradient norms, update norms, and finiteness. Convert the findings into a stable configuration and regression test.

Part 2 - Mathematical form

$$\Delta\theta = -\eta \operatorname{clip}(g, c), \qquad \rho(W_{hh}) \text{ influences recurrent stability.}$$

Part 3 - Hints and Helper

Stop immediately at the first non-finite value. Preserve the batch and pre-update checkpoint that reproduce the failure.

Part 4 - Python code skeleton

```
import torch

def E15(model, data, config):
    diagnostics = {}
    for learning_rate, clip_norm, init_scale in config.grid:
        candidate = build_model(recurrent_scale=init_scale)
        result = run_instrumented_training(
            candidate, fixed_batches, learning_rate=learning_rate,
            clip_norm=clip_norm, stop_on_non_finite=True
        )
        diagnostics[(learning_rate, clip_norm, init_scale)] = result
    stable_config = select_stable_configuration(diagnostics, criteria=...)
    return diagnostics, verify_regression_test(stable_config, ...)
```

15.6.17 Exercise 16 - Build a Generation Evaluation Script for Repetition and Novelty Metrics

Part 1 - Why this matters

Generated text should not be evaluated only by a visually pleasing sample. Repetition, training-text copying, character diversity, and invalid transitions require measurements across prompts and random seeds.

Novelty is not automatically quality: random noise is novel, while correct language naturally repeats common patterns. Metrics must therefore be interpreted together with likelihood and human-readable examples.

Build a reproducible evaluator for repeated n-grams, unique n-grams, longest copied substring, entropy, and novelty relative to training text. Aggregate results across generation settings.

Part 2 - Mathematical form

$$R_n = 1 - \frac{|\operatorname{unique}(G_n)|}{|G_n|}, \qquad N_n = 1 - \frac{|G_n \cap T_n|}{|G_n|}.$$

Part 3 - Hints and Helper

State how short samples and repeated whitespace are handled. Preserve prompts separately so they are not counted as generated copying.

Part 4 - Python code skeleton

```
import torch

def E16(model, data, config):
```

```
    def generation_metrics(generated, training_text, prompt, n):
        continuation = remove_prompt_prefix(generated, prompt)
        generated_ngrams = ngrams(continuation, n)
        training_ngrams = set(ngrams(training_text, n))
        return {
            "repetition_rate": ...,
            "novelty_rate": ...,
            "unique_ngram_ratio": ...,
            "longest_training_match": ...,
            "character_entropy": ...,
        }

    return evaluate_generation_grid(
        model, prompts=..., seeds=..., metrics_fn=...
    )
```

15.6.18 Exercise 17 - Create a Controlled Long-Term Dependency Failure Experiment for Vanilla RNN

Part 1 - Why this matters

Natural text mixes many causes, making it difficult to prove why a model forgot distant context. A controlled language-like task can place an opening marker far before a required closing symbol while keeping local context uninformative.

If the dependency rule remains fixed and only distance changes, declining accuracy can be connected to memory and gradient limitations. Local baselines and shuffled markers help rule out shortcuts.

Generate matched datasets across distances, train vanilla RNNs under controlled conditions, and measure target accuracy, early-state gradient norm, probe accuracy, and hidden-state separation.

Part 2 - Mathematical form

$$y_T = f(c_{T-d}), \qquad I(y_T; c_{T-k:T-1}) = 0 \text{ for } k < d.$$

Part 3 - Hints and Helper

Balance every target and verify that the suffix distribution is identical across labels. Run multiple seeds.

Part 4 - Python code skeleton

```
import torch

def E17(model, data, config):
    results = []
```

```
    for distance in config.distances:
        dataset = make_controlled_dependency_text(
            distance=distance, balanced=True, prevent_local_shortcuts=True
        )
        candidate = build_vanilla_rnn(...)
        train_language_model(candidate, dataset.train, ...)
        results.append({
            "distance": distance,
            "target_accuracy": ...,
            "early_gradient_norm": ...,
            "probe_accuracy": ...,
            "hidden_separation": ...,
        })
    return analyze_failure_horizon(results, chance_level=...)
```

15.6.19 Exercise 18 - Stress Test Language Modeling with Longer Text and Identify GPU Memory and Speed Limits

Part 1 - Why this matters

Scaling text length increases dataset size, while sequence length, batch size, embedding size, hidden size, and vocabulary size determine per-step compute and memory. These dimensions constrain different parts of the pipeline.

The vocabulary projection can dominate output memory, and BPTT activations grow with time. Data loading may become the bottleneck even before the GPU reaches its memory limit.

Run a safe configuration grid, measure loader wait, transfer, forward, backward, optimizer time, peak memory, tokens per second, and validation quality. Record out-of-memory and unstable cases explicitly.

Part 2 - Mathematical form

$$M_{\text{logits}} = O(BT|V|), \qquad M_{\text{states}} = O(BTH), \qquad C_{\text{RNN}} = O(BT(HD + H^2)).$$

Part 3 - Hints and Helper

Increase one dimension at a time before testing combinations. Clear references between trials and keep correctness checks active.

Part 4 - Python code skeleton

```
import torch

def E18(model, data, config):
    records = []
    for case in config.stress_grid:
        try:
```

```
            candidate, loader = build_stress_case(case)
            result = benchmark_language_training(candidate, loader, device,
                                                 profile_phases=True)
            records.append({"case": case, "status": "ok", **result})
        except torch.cuda.OutOfMemoryError:
            records.append({"case": case, "status": "out_of_memory"})
    return identify_system_limits(records, quality_threshold=...)
```

15.6.20 Exercise 19 - Produce a Full Character-Level RNN Language Modeling Report with Failure Analysis and Fixes

Part 1 - Why this matters

A complete language-model report must connect text representation, target alignment, architecture, optimization, probability evaluation, sampling behavior, and system performance. Generated examples alone are insufficient evidence.

Failures should be reproducible and classified: token mapping mismatch, alignment error, state reset bug, repetition, exposure bias, gradient instability, long-term memory failure, precision failure, or resource limit. Each class requires a different fix.

Produce an evidence-based report with fixed artifacts, plots, tables, prompts, seeds, checkpoints, and regression tests. State clearly what a character-level vanilla RNN can demonstrate and why later gated models are needed.

Part 2 - Mathematical form

$$\mathcal{R} = \{T, A, M, O, G, S, P, B, F, L, E\}, \qquad \text{complete}(\mathcal{R}) = \bigwedge_{r \in \mathcal{R}} [r \neq \varnothing].$$

Part 3 - Hints and Helper

For every claimed fix include the original failure, controlled change, new measurement, and remaining limitation.

Part 4 - Python code skeleton

```
import torch

def E19(model, data, config):
    from pathlib import Path

    evidence = collect_language_model_evidence(
        tokenization=..., alignment=..., model=..., overfit_test=...,
        generation=..., sampling=..., perplexity=..., bptt=...,
        cuda=..., precision=..., long_dependencies=..., stress_limits=...)
    report = render_language_model_report(evidence)
    validate_failure_fix_evidence(report)
    Path(config.output_path).write_text(report, encoding="utf-8")
    return report
```

15.7 RNN05 - Time Series Forecasting with Vanilla RNN

Models and Training

15.7.1 Exercise 00 - Convert Time Series into Supervised Windows and Verify the Correct Target Horizon

Part 1 - Why this matters

A time series becomes a supervised dataset only after historical observations are assigned to inputs and later observations are assigned to targets. Window length, forecast gap, and horizon define the forecasting question the model is actually trained to answer.

An off-by-one error can turn a future forecast into reconstruction of the present or prediction of the wrong date. Because neighboring observations are correlated, the resulting metrics may still look strong and hide the mistake.

Build windows from timestamped diagnostic data, retain source and target indices, and verify the first, middle, and last samples manually. The test must prove that every target begins at the intended future horizon.

Part 2 - Mathematical form

$$X_i = (s_i, \ldots, s_{i+w-1}), \qquad Y_i = (s_{i+w+g-1}, \ldots, s_{i+w+g+h-2}).$$

Part 3 - Hints and Helper

Define whether gap one means the immediately following observation. Store timestamps before converting data to anonymous tensors.

Part 4 - Python code skeleton

```
import torch

def E00(model, data, config):
    series, timestamps = ...
    window, gap, horizon = ...
    inputs, targets, metadata = [], [], []
    for start in range(...):
        input_slice = ...
        target_slice = ...
        inputs.append(input_slice)
        targets.append(target_slice)
        metadata.append({"input_times": ..., "target_times": ...})
    validate_forecast_horizon(inputs, targets, metadata, gap, horizon)
    return torch.stack(inputs), torch.stack(targets), metadata
```

15.7.2 Exercise 01 - Implement Feature Scaling for Time Series and Explain How It Prevents Training Instability

Part 1 - Why this matters

Time-series features may use very different units and magnitudes. Large-scale features can dominate recurrent activations, loss values, and gradients, pushing nonlinear states toward saturation or unstable updates.

Scaling parameters are learned information about the data distribution. Fitting them on validation or test periods leaks future statistics into training, while scaling targets incorrectly can make reported errors meaningless in original units.

Fit a scaler only on the training timeline, transform every split with the stored parameters, and implement an inverse transform for predictions. Test constant features and values outside the training range.

Part 2 - Mathematical form

$$z_{t,j} = \frac{x_{t,j} - \mu_j^{\text{train}}}{\sigma_j^{\text{train}} + \epsilon}.$$

Part 3 - Hints and Helper

Compute statistics per feature, not across time and feature axes together. Save the scaler with the checkpoint.

Part 4 - Python code skeleton

```
import torch

def E01(model, data, config):
    train_values, validation_values, test_values = ...
    mean = train_values.mean(dim=0, keepdim=True)
    std = ...
    std_safe = torch.where(std > ..., std, torch.ones_like(std))
    train_scaled = ...
    validation_scaled = ...
    test_scaled = ...
    restored = inverse_scale(test_scaled, mean, std_safe)
    assert torch.allclose(restored, test_values)
    scaler_state = {"mean": mean, "std": std_safe}
    return train_scaled, validation_scaled, test_scaled, scaler_state
```

15.7.3 Exercise 02 - Build a Simple RNN Forecaster and Validate Output Shapes for Multi-Step Prediction

Part 1 - Why this matters

A multi-step forecaster maps a historical sequence to several future values. Its output dimensions must distinguish batch, forecast horizon, and target features rather than mixing horizons into the batch or hidden axis.

A direct forecaster can project the final hidden state to all future steps at once. This avoids feedback during inference but asks one representation to encode every horizon and does not model dependencies between predicted future steps explicitly.

Implement a many-to-vector RNN forecaster, reshape its projection deliberately, and assert shapes for single-feature and multivariate targets. Compare one sample manually with the raw linear projection.

Part 2 - Mathematical form

$$X \in \mathbb{R}^{B\times T\times D} \to h_T \in \mathbb{R}^{B\times H} \to \hat{Y} \in \mathbb{R}^{B\times K\times F}.$$

Part 3 - Hints and Helper

Keep batch size one as a real batch. Avoid an unrestricted squeeze that may remove the batch dimension.

Part 4 - Python code skeleton

```
import torch

def E02(model, data, config):
    class RNNForecaster(torch.nn.Module):
        def __init__(self, input_size, hidden_size, horizon, target_size):
            super().__init__()
            self.horizon = horizon
            self.target_size = target_size
            self.rnn = torch.nn.RNN(..., batch_first=True)
            self.head = torch.nn.Linear(hidden_size, horizon * target_size)
        def forward(self, x):
            states, hidden = self.rnn(x)
            raw = self.head(states[:, -1])
            forecast = raw.reshape(...)
            assert forecast.shape == (...)
            return forecast

    return RNNForecaster(...)
```

15.7.4 Exercise 03 - Compare One-Step and Multi-Step Forecasting and Explain Error Accumulation

Part 1 - Why this matters

One-step forecasting predicts only the next observation, while multi-step forecasting estimates an entire future horizon. Repeating a one-step model autoregressively changes later inputs from observed values to model-generated values.

Each early prediction error can alter all later forecasts, so long-horizon error includes both model uncertainty and propagated input error. Direct multi-output prediction avoids feedback but may lose the sequential structure between future values.

Compare one-step, recursive, and direct multi-step strategies on the same splits. Report error by horizon and identify where recursive forecasts begin to drift.

Part 2 - Mathematical form

$$\hat{y}_{t+k} = f(\hat{y}_{t+k-1}, \ldots), \qquad e_{t+k} = \hat{y}_{t+k} - y_{t+k}.$$

Part 3 - Hints and Helper

Do not average horizons before examining them separately. Use identical historical context and inverse scaling.

Part 4 - Python code skeleton

```
import torch

def E03(model, data, config):
    strategies = {"one_step": ..., "recursive": ..., "direct": ...}
    results = {}
    for name, forecaster in strategies.items():
        predictions = forecast_horizons(forecaster, test_windows, horizon=...)
        results[name] = {
            "predictions": predictions,
            "mae_by_horizon": ...,
            "rmse_by_horizon": ...,
            "drift": ...,
        }
    return results
```

15.7.5 Exercise 04 - Train an RNN on Synthetic Seasonal Data and Confirm It Learns Periodicity

Part 1 - Why this matters

Synthetic seasonal data provides a known period, amplitude, phase, trend, and noise level. This lets the experiment test whether the model learned temporal structure rather than exploiting an unknown property of real data.

A low error does not by itself prove that periodicity was learned. A persistence or seasonal-naive baseline may achieve similar performance, especially when adjacent values change slowly.

Generate controlled seasons, split chronologically, train the RNN, and compare forecasts with persistence and seasonal-naive baselines. Test phase shifts and changed amplitudes to examine what the hidden state captured.

Part 2 - Mathematical form

$$y_t = a \sin\left(\frac{2\pi t}{P} + \phi\right) + bt + \varepsilon_t.$$

Part 3 - Hints and Helper

Choose a window long enough to observe the intended period in at least one experiment. Keep generation seeds and parameters recorded.

Part 4 - Python code skeleton

```
import torch

def E04(model, data, config):
    time = torch.arange(..., dtype=torch.float32)
    seasonal = config.amplitude * torch.sin(
        2 * torch.pi * time / config.period + config.phase
    )
    series = seasonal + config.trend * time + ...
    train, validation, test = chronological_split(series, ...)
    model = train_forecaster(model, train, validation, ...)
    baselines = evaluate_seasonal_baselines(test, period=config.period)
    comparison = compare_periodicity_learning(
        model, test, baselines, shifted_cases=...
    )
    return comparison
```

15.7.6 Exercise 05 - Implement Autoregressive Multi-Step Forecasting and Observe Drift over Time

Part 1 - Why this matters

Autoregressive forecasting produces one prediction, appends it to the context, and repeats. The future path is therefore generated under the model's own state distribution rather than the observed distribution used for the initial window.

Bias and noise can accumulate, while scaling errors may push feedback values outside the training range. The forecast may converge to a fixed point, explode, flatten, or drift away from plausible seasonality.

Implement a recursive loop with explicit feature-update rules and no future leakage. Log every context update, hidden state, prediction, and deviation from the direct or observed path.

Part 2 - Mathematical form

$$\hat{y}_{t+k} = f_\theta(X_{t+k-1}), \qquad X_{t+k} = \text{shift}(X_{t+k-1}, \hat{y}_{t+k}).$$

Part 3 - Hints and Helper

For multivariate inputs, define which future exogenous features are known. Never copy true unknown targets into later contexts.

Part 4 - Python code skeleton

```
import torch

def E05(model, data, config):
    context = initial_window.clone()
    forecasts, trace = [], []
    for step in range(config.horizon):
        prediction = model(context)[...]
        forecasts.append(prediction)
        next_features = build_next_features(
            prediction=prediction, known_future=..., previous_context=context
        )
        context = torch.cat([context[:, 1:], next_features], dim=1)
        trace.append({"step": step, "context": context.detach().clone(),
                      "prediction": prediction.detach().clone()})
    return torch.stack(forecasts, dim=1), trace
```

15.7.7 Exercise 06 - Implement Teacher Forcing for Forecasting and Compare with Pure Autoregressive Training

Part 1 - Why this matters

Teacher forcing supplies the true previous target during decoder training, while autoregressive training reuses the model's previous prediction. The first strategy stabilizes learning but creates a train–inference mismatch.

Pure autoregressive training exposes the model to its own mistakes, yet it may be harder to optimize and can propagate poor early predictions. A scheduled mixture changes this exposure gradually rather than treating the choice as absolute.

Train matched models with multiple teacher-forcing ratios and evaluate all of them in free-running mode. Compare training loss, validation error by horizon, drift, and sensitivity to the initial forecast error.

Part 2 - Mathematical form

$$u_k = z y_{t+k-1} + (1 - z)\hat{y}_{t+k-1}, \qquad z \sim \text{Bernoulli}(r).$$

Part 3 - Hints and Helper

The validation procedure must not use teacher forcing if deployment will be autoregressive. Record the ratio schedule at each epoch.

Part 4 - Python code skeleton

```
import torch

def E06(model, data, config):
    hidden = encode_history(model, history)
    previous = history[:, -1]
    outputs = []
    for step in range(targets.shape[1]):
        prediction, hidden = decoder_step(model, previous, hidden)
        outputs.append(prediction)
        use_teacher = ...
        previous = targets[:, step] if use_teacher else prediction.detach()
    forecasts = torch.stack(outputs, dim=1)
    return forecasts, evaluate_free_running(model, validation_loader, ...)
```

15.7.8 Exercise 07 - Evaluate Forecast Accuracy with MAE and MSE and Interpret Their Different Penalties

Part 1 - Why this matters

MAE measures average absolute deviation and gives errors linear influence. MSE squares deviations, making large mistakes contribute disproportionately and changing both the evaluation emphasis and the gradient when used as a training loss.

A single aggregate number can hide poor distant-horizon forecasts, seasonal regimes, or rare extremes. Metrics in scaled units are also difficult to interpret operationally unless predictions are restored to original units.

Compute MAE, MSE, and RMSE overall, by horizon, and by relevant time regime. Add a controlled outlier to demonstrate how the metrics react differently.

Part 2 - Mathematical form

$$\text{MAE} = \frac{1}{N}\sum_i |e_i|, \qquad \text{MSE} = \frac{1}{N}\sum_i e_i^2, \qquad \text{RMSE} = \sqrt{\text{MSE}}.$$

Part 3 - Hints and Helper

Use the same valid observations for every metric and state the reduction axes. Report original physical units.

Part 4 - Python code skeleton

```
import torch

def E07(model, data, config):
    predictions = inverse_scale(predictions_scaled, scaler)
    targets = inverse_scale(targets_scaled, scaler)
    errors = predictions - targets
    metrics = {
        "mae": errors.abs().mean(),
        "mse": errors.square().mean(),
        "rmse": errors.square().mean().sqrt(),
        "mae_by_horizon": ...,
        "rmse_by_horizon": ...,
    }
    outlier_metrics = recompute_with_controlled_outlier(
        predictions, targets, ...
    )
    return metrics, outlier_metrics
```

15.7.9 Exercise 08 - Debug Forecasting Bugs from Wrong Windowing or Misaligned Targets

Part 1 - Why this matters

Windowing errors change the forecasting task without changing tensor validity. A model may accidentally predict the current observation, skip the requested horizon, or learn from future features included in its input.

Smooth time series make these bugs especially deceptive because nearby values are similar. Strong performance against a weak baseline may therefore reflect leakage rather than useful forecasting.

Build index-level audits using timestamps and a monotonic diagnostic series. Compare correct, zero-gap, off-by-one, reversed-time, and future-feature variants and make every corruption trigger a specific failure.

Part 2 - Mathematical form

$$\max \text{time}(X_i) < \min \text{time}(Y_i), \qquad \min \text{time}(Y_i) - \max \text{time}(X_i) = g\Delta t.$$

Part 3 - Hints and Helper

Validate raw indices before scaling or batching. Plot one input-target pair with timestamps for human inspection.

Part 4 - Python code skeleton

```
import torch

def E08(model, data, config):
    series, timestamps = ...
    variants = create_alignment_variants(series, timestamps, config)
    reports = {}
    for name, dataset in variants.items():
        reports[name] = audit_forecast_windows(
            dataset, expected_gap=config.gap,
            expected_horizon=config.horizon, forbid_future_features=True
        )
    assert reports["correct"]["passed"]
    assert all(not report["passed"] for name, report in reports.items()
               if name != "correct")
    return reports
```

15.7.10 Exercise 09 - Diagnose Overfitting in Forecasting and Apply Regularization or Early Stopping

Part 1 - Why this matters

A forecaster overfits when training error continues improving while performance on later unseen periods deteriorates. Temporal distribution changes can resemble overfitting, so the validation design must be examined before the model is blamed.

Dropout, weight decay, smaller capacity, and early stopping address different mechanisms. Applying all of them together can hide the cause and produce underfitting without showing which intervention mattered.

Track train and chronological validation metrics by horizon, save the best checkpoint, and run controlled regularization comparisons. Confirm the selected model on a final untouched test period.

Part 2 - Mathematical form

$$e^* = \arg\min_e L_{\text{val}}^{(e)}, \qquad \text{stop if } L_{\text{val}} \text{ fails to improve for } p \text{ epochs.}$$

Part 3 - Hints and Helper

Fit preprocessing on training data only and restore the complete best state, including model selection metadata.

Part 4 - Python code skeleton

```
import torch

def E09(model, data, config):
    best_value = float("inf")
```

```
    patience_count = 0
    for epoch in range(config.epochs):
        train_metrics = train_epoch(model, train_loader, ...)
        validation_metrics = evaluate_forecast(model, validation_loader, ...)
        monitored = ...
        if monitored < best_value - config.min_delta:
            best_value = monitored
            best_state = snapshot(model, optimizer, epoch, scaler=...)
            patience_count = 0
        else:
            patience_count += 1
        if patience_count >= config.patience:
            break
    restore_snapshot(model, best_state)
    comparison = compare_regularization_variants(...)
    test_metrics = evaluate_forecast(model, test_loader)
    return comparison, test_metrics
```

15.7.11 Exercise 10 - Train a Time-Series RNN on CUDA and Benchmark End-to-End Throughput Including the Data Pipeline

Part 1 - Why this matters

Moving only the model computation to CUDA does not describe end-to-end training speed. Window creation, collation, host-to-device transfer, synchronization, and metric logging can dominate a small recurrent workload.

CUDA timing is asynchronous, and excluding input delivery can exaggerate speedup. CPU and GPU comparisons must process the same number of windows and valid forecast values under matched optimization conditions.

Profile loader wait, transfer, forward, backward, optimizer, and evaluation phases after warm-up. Validate a controlled CPU–GPU step before accepting performance results.

Part 2 - Mathematical form

$$Q = \frac{N_{\text{windows}}}{T_{\text{wall}}}, \qquad T_{\text{wall}} = T_{\text{load}} + T_{H\to D} + T_f + T_b + T_o.$$

Part 3 - Hints and Helper

Report windows and observed values per second. Use pinned memory with non-blocking transfer only when supported.

Part 4 - Python code skeleton

```
import torch
```

```
def E10(model, data, config):
    cpu_result = benchmark_end_to_end_training(
        clone_model(model).cpu(), cpu_loader, device="cpu", warmup=...
    )
    gpu_result = benchmark_end_to_end_training(
        clone_model(model).cuda(), gpu_loader, device="cuda",
        warmup=..., synchronize=True
    )
    parity = compare_controlled_training_step(
        model, batch=..., tolerances=...
    )
    return {"cpu": cpu_result, "gpu": gpu_result, "parity": parity}
```

15.7.12 Exercise 11 - Profile GPU Memory and Runtime for Sequence Length and Hidden-Size Scaling

Part 1 - Why this matters

Sequence length controls graph depth and stored temporal activations, while hidden size increases recurrent matrix cost approximately quadratically. Their interaction determines whether a configuration is compute-bound, memory-bound, or impractical.

The output horizon and feature count add further cost, but may not dominate the recurrent core. Peak allocation must be measured during backward rather than inferred from model parameter size.

Benchmark a safe grid of lengths and hidden sizes, separating forward and backward time. Record parameters, peak memory, throughput, gradient norm, and failure status.

Part 2 - Mathematical form

$$C = O(TB(HD + H^2)), \qquad M_{\text{act}} = O(TBH).$$

Part 3 - Hints and Helper

Increase one dimension at a time first. Reset peak-memory statistics for every independent trial.

Part 4 - Python code skeleton

```
import torch

def E11(model, data, config):
    records = []
    for sequence_length, hidden_size in config.grid:
        try:
            candidate = build_forecaster(hidden_size).to(device)
            batch = make_forecast_batch(sequence_length).to(device)
```

```
            torch.cuda.reset_peak_memory_stats()
            forward_ms, loss = timed_forward(candidate, batch)
            backward_ms = timed_backward(loss)
            records.append({"length": sequence_length, "hidden": hidden_size,
                            "forward_ms": forward_ms,
                            "backward_ms": backward_ms,
                            "peak_memory": ..., "throughput": ...})
        except torch.cuda.OutOfMemoryError:
            records.append({"length": sequence_length, "hidden": hidden_size,
                            "status": "out_of_memory"})
    return records
```

15.7.13 Exercise 12 - Implement Mixed Precision for Forecasting and Identify Stability Failures

Part 1 - Why this matters

Mixed precision may reduce memory and accelerate training, but time-series targets can contain large ranges, small variations, or losses sensitive to squaring. These properties can produce overflow, underflow, or lost resolution.

Scaling the features does not guarantee that recurrent gradients and output errors remain representable. A finite average loss may coexist with zero gradients in early layers or skipped optimizer updates.

Compare float32 and mixed precision across target scales, sequence lengths, and noise levels. Log scaler behavior, layer-wise gradients, non-finite values, accuracy in original units, speed, and memory.

Part 2 - Mathematical form

$$\widetilde{L} = sL, \qquad \nabla L = s^{-1}\nabla \widetilde{L}.$$

Part 3 - Hints and Helper

Unscale before clipping and diagnostics. Compute final forecasting metrics in float32 or float64 after inverse scaling.

Part 4 - Python code skeleton

```
import torch

def E12(model, data, config):
    scaler = torch.amp.GradScaler("cuda")
    optimizer.zero_grad(set_to_none=True)
    with torch.autocast(device_type="cuda", dtype=torch.float16):
        forecast = model(inputs)
        loss = criterion(forecast, targets)
```

```
    scaler.scale(loss).backward()
    scaler.unscale_(optimizer)
    gradient_report = inspect_gradients(model)
    scaler.step(optimizer)
    scaler.update()
    return {"loss": loss.item(), "scale": scaler.get_scale(),
            "gradients": gradient_report, "original_unit_metrics": ...}
```

15.7.14 Exercise 13 - Compare Vanilla RNN and LSTM Forecasting and Document Long-Term Dependency Gains

Part 1 - Why this matters

An LSTM introduces gates and a cell state intended to preserve selected information across longer distances. It adds parameters and computation, so an improvement must be evaluated against capacity and system cost.

A dataset with mostly short dependencies may not benefit from gating, while seasonal or delayed effects can expose vanilla RNN memory limits. Aggregate error alone may hide where the LSTM begins to help.

Train controlled RNN and LSTM forecasters on identical chronological splits. Compare errors by horizon and dependency distance, gradient flow, convergence, runtime, memory, and parameter count.

Part 2 - Mathematical form

$$\Delta_k = E_k^{\mathrm{RNN}} - E_k^{\mathrm{LSTM}}, \qquad k = 1, \ldots, K.$$

Part 3 - Hints and Helper

Run matched-hidden-size and approximately matched-parameter comparisons. Use multiple seeds and the same preprocessing artifact.

Part 4 - Python code skeleton

```
import torch

def E13(model, data, config):
    models = {"rnn": build_rnn(...), "lstm": build_lstm(...)}
    results = {}
    for name, candidate in models.items():
        history = train_forecaster(
            candidate, train_loader, validation_loader, ...
        )
        results[name] = {
            "metrics_by_horizon": evaluate_horizons(
                candidate, test_loader
```

```
            ),
            "gradient_flow": ..., "parameters": ...,
            "runtime": ..., "memory": ..., "history": history,
        }
    return compare_long_dependency_gains(results)
```

15.7.15 Exercise 14 - Implement Truncated BPTT for Long Forecasting Windows and Quantify Memory Savings

Part 1 - Why this matters

Long continuous series can be processed in chunks while hidden state carries context between them. Detaching at boundaries limits backward graph length and reduces activation memory.

Numeric state continuity does not imply gradient continuity. A target whose cause lies beyond the truncation boundary cannot assign credit through the detached path, even though the carried state contains transformed values.

Implement chronological chunks, reset at independent series boundaries, and sweep truncation lengths. Measure memory, runtime, validation error by dependency distance, and state continuity.

Part 2 - Mathematical form

$$h_{kb}^{+} = \text{stopgrad}(h_{kb}), \qquad M_b < M_T.$$

Part 3 - Hints and Helper

Never shuffle chunks while carrying hidden state. Distinguish optimizer-step boundaries from detach boundaries.

Part 4 - Python code skeleton

```
import torch

def E14(model, data, config):
    hidden = None
    records = []
    for chunk in chronological_chunks:
        if chunk.new_series:
            hidden = ...
        elif hidden is not None:
            hidden = detach_state(hidden)
        optimizer.zero_grad(set_to_none=True)
        forecast, hidden = model(chunk.inputs, hidden)
        loss = masked_forecast_loss(forecast, chunk.targets, chunk.mask)
        loss.backward()
```

```
        optimizer.step()
        records.append({"loss": loss.item(), "peak_memory": ...})
    return compare_truncation_lengths(records, dependency_tests=...)
```

15.7.16 Exercise 15 - Build a Forecast Visualization Tool for Predictions and Ground Truth across Multiple Horizons

Part 1 - Why this matters

Forecast plots reveal temporal structure hidden by aggregate metrics: phase lag, amplitude damping, drift, missed peaks, boundary discontinuities, and growing horizon uncertainty. A plot can also mislead if timestamps are shifted.

Multi-step predictions overlap because several forecast origins may predict the same timestamp. The visualization must define whether it shows one origin, all origins, or an aggregation of overlapping predictions.

Build a tool that validates timestamp alignment, plots selected origins and horizons, marks train-validation boundaries, and optionally displays residuals. Keep plotting data on the CPU without altering the evaluation tensors.

Part 2 - Mathematical form

$$\hat{y}_{o,k} \leftrightarrow t_{o+k}, \qquad e_{o,k} = \hat{y}_{o,k} - y_{o+k}.$$

Part 3 - Hints and Helper

Use actual timestamps rather than array positions. Label the forecast origin and horizon units explicitly.

Part 4 - Python code skeleton

```
import torch

def E15(model, data, config):
    def build_forecast_plot(
        timestamps, targets, predictions, origins, horizons
    ):
        validate_plot_alignment(
            timestamps, targets, predictions, origins, horizons
        )
        figure, axes = create_axes(...)
        for origin in selected_origins(origins, ...):
            target_times = ...
            axes[0].plot(
                target_times, targets_for_origin(...), label=...
            )
            axes[0].plot(
```

```
                target_times, predictions_for_origin(...), label=...
            )
            axes[1].plot(target_times, residuals_for_origin(...))
        add_split_boundaries(axes, ...)
        return figure

    return build_forecast_plot(...)
```

15.7.17 Exercise 16 - Diagnose Instability from Seasonal Scale and Fix It with Normalization

Part 1 - Why this matters

Seasonal series can change amplitude across months, regimes, or features. Large seasonal peaks may dominate squared loss and recurrent state updates, while small but important variations become numerically insignificant.

Global normalization may still leave local regimes poorly scaled, while normalization calculated with future windows leaks information. Per-window normalization can remove level information needed for the forecast unless its statistics are supplied to the model and inverted correctly.

Compare training-only global scaling, robust scaling, seasonal adjustment, and carefully designed window normalization. Measure state saturation, gradient norms, convergence, and original-unit accuracy by season.

Part 2 - Mathematical form

$$z_t = \frac{x_t - m_{\text{train}}}{s_{\text{train}}}, \qquad z_t^{(w)} = \frac{x_t - \mu_w}{\sigma_w + \epsilon}.$$

Part 3 - Hints and Helper

Fit every persistent statistic on training data only. Preserve level and scale metadata required for inverse transformation.

Part 4 - Python code skeleton

```
import torch

def E16(model, data, config):
    strategies = {"global": ..., "robust": ...,
                  "seasonal_adjustment": ..., "window": ...}
    results = {}
    for name, scaler in strategies.items():
        scaled_splits, metadata = fit_transform_without_leakage(
            scaler, splits
        )
```

```
        candidate = clone_initial_model(model)
        history = train_forecaster(candidate, scaled_splits.train, ...)
        results[name] = {
            "state_saturation": ..., "gradient_norms": ...,
            "metrics_by_season": evaluate_in_original_units(
                candidate, metadata
            ),
            "history": history,
        }
    return results
```

15.7.18 Exercise 17 - Create Walk-Forward Validation to Avoid Temporal Leakage in Forecast Evaluation

Part 1 - Why this matters

Walk-forward validation repeatedly trains or updates on the past and evaluates on a later interval. It more closely matches deployment than a random split and reveals performance changes over time.

Every fold requires independent preprocessing fitted only on that fold's training region. Reusing a scaler, early-stopping decision, or feature statistic from later folds leaks future knowledge backward.

Implement expanding or rolling windows with explicit timestamps, gaps, and forecast intervals. Aggregate metrics by fold and horizon while preserving the chronological sequence of results.

Part 2 - Mathematical form

$$\mathcal{T}_k = [a_k, b_k], \qquad \mathcal{V}_k = [b_k + g, b_k + g + v], \qquad \max \mathcal{T}_k < \min \mathcal{V}_k.$$

Part 3 - Hints and Helper

Choose whether the model is retrained from scratch or updated, and state how that matches the intended deployment process.

Part 4 - Python code skeleton

```
import torch

def E17(model, data, config):
    fold_results = []
    for fold in build_walk_forward_folds(timestamps, config):
        train_raw = series[fold.train_indices]
        validation_raw = series[fold.validation_indices]
        scaler = fit_scaler(train_raw)
        train_data = make_windows(scaler.transform(train_raw), ...)
```

```
        validation_data = make_windows(
            scaler.transform(validation_raw), ...
        )
        candidate = initialize_fold_model(model, policy=...)
        train_forecaster(candidate, train_data, ...)
        metrics = evaluate_horizons(candidate, validation_data)
        fold_results.append({
            "fold": fold.metadata,
            "metrics": metrics,
            "scaler": scaler.state_dict(),
        })
    return aggregate_walk_forward_results(fold_results)
```

15.7.19 Exercise 18 - Stress Test the Forecasting Pipeline with Large Data and Identify Practical Limits

Part 1 - Why this matters

Large time-series workloads stress window indexing, storage, DataLoader workers, host memory, transfer, recurrent compute, BPTT memory, and evaluation output. The first bottleneck may occur outside the model.

Materializing every overlapping window duplicates data, whereas lazy views reduce storage but increase indexing work. Long horizons also expand targets and prediction archives, affecting evaluation memory.

Scale dataset size, window length, feature count, horizon, batch size, and hidden size safely. Profile each phase, record failures, and identify maximum reliable configurations rather than only the largest allocation that runs once.

Part 2 - Mathematical form

$$N_w = N - w - g - h + 2, \qquad S_{\text{materialized}} = O(N_w(wD + hF)).$$

Part 3 - Hints and Helper

Use bounded trials and preserve a correctness fixture. Distinguish sustained throughput from a short cached benchmark.

Part 4 - Python code skeleton

```
import torch

def E18(model, data, config):
    records = []
    for case in config.stress_grid:
        try:
            dataset, loader, candidate = build_forecasting_case(case)
```

```
            result = profile_end_to_end_forecasting(
                candidate, loader, device, duration=..., verify_fixture=True
            )
            records.append({"case": case, "status": "ok", **result})
        except (torch.cuda.OutOfMemoryError, MemoryError) as error:
            records.append({"case": case, "status": type(error).__name__})
    return identify_pipeline_limits(records, reliability_thresholds=...)
```

15.7.20 Exercise 19 - Produce a Full Time-Series RNN Forecasting Report with Failure Cases and Engineering Fixes

Part 1 - Why this matters

A forecasting report must connect the temporal question, data availability, preprocessing, model, horizon, validation scheme, metrics, uncertainty, and operational limits. A single test RMSE cannot establish that the system is trustworthy.

Failures require temporal context: leakage, wrong horizons, recursive drift, seasonal regime shifts, overfitting, unstable gradients, precision loss, or resource bottlenecks produce different signatures and corrections.

Produce a reproducible report with timestamped splits, scaler state, baselines, horizon-level metrics, visualizations, walk-forward folds, stress results, and failure-fix regression tests. State where vanilla RNN memory becomes inadequate.

Part 2 - Mathematical form

$$\mathcal{R} = \{Q, D, P, M, T, V, E, G, S, F, L\}, \qquad \text{complete}(\mathcal{R}) = \bigwedge_{r \in \mathcal{R}} [r \neq \varnothing].$$

Part 3 - Hints and Helper

For each fix report the original failure, controlled intervention, new evidence, and remaining limitation in original data units.

Part 4 - Python code skeleton

```
import torch

def E19(model, data, config):
    from pathlib import Path

    evidence = collect_forecasting_evidence(
        task_definition=..., data=..., preprocessing=..., baselines=...,
        model=..., training=..., horizon_metrics=..., visualizations=...,
        walk_forward=..., cuda=..., precision=..., stress_limits=...,
        failures_and_fixes=...)
    report = render_forecasting_report(evidence)
```

```
    validate_temporal_evidence(report)
    Path(config.output_path).write_text(report, encoding="utf-8")
    return report
```

15.8 RNN06 - Regularization and Stabilization for RNNs

Models and Training

15.8.1 Exercise 00 - Demonstrate Overfitting in RNN and Measure Train vs Validation Gap in Sequence Tasks

Part 1 - Why this matters

Overfitting in sequence models is more subtle than memorizing independent samples because adjacent windows often share most of their time steps. An RNN can reproduce training patterns extremely well while failing as soon as the temporal regime, subject, or recording period changes.

A single final accuracy value cannot reveal this behavior. The experiment must track training and validation loss over epochs, use a split that respects sequence boundaries, and measure when the generalization gap begins to grow even though training performance continues to improve.

Train models with increasing hidden size on a deliberately limited dataset. Compare learning curves, best validation epoch, parameter count, and performance on an untouched test sequence so that model capacity is connected to observed overfitting rather than discussed only in theory.

Part 2 - Mathematical form

$$G_t = L_{\text{val}}(t) - L_{\text{train}}(t), \qquad t^* = \arg\min_t L_{\text{val}}(t).$$

Part 3 - Hints and Helper

Keep all overlapping windows from one original sequence in the same split. Report both the final epoch and the best validation epoch.

Part 4 - Python code skeleton

```
import torch

def E00(model, data, config):
    train_loader, validation_loader, test_loader = ...
    histories = {}
    for hidden_size in config.hidden_sizes:
        candidate = build_rnn(hidden_size=hidden_size, ...)
        histories[hidden_size] = train_and_log(
            candidate, train_loader, validation_loader, epochs=...
```

```
        )
    gaps = compute_generalization_gaps(histories)
    test_results = evaluate_best_checkpoints(histories, test_loader)
    return gaps, test_results
```

15.8.2 Exercise 01 - Apply Dropout in RNN and Compare Input Dropout with Dropout Between Recurrent Layers

Part 1 - Why this matters

Dropout reduces reliance on individual activations, but its location changes what uncertainty is introduced into a recurrent model. Input dropout corrupts observed features, while inter-layer dropout regularizes the representation passed from one recurrent layer to the next.

Applying a fresh random mask carelessly at every recurrent transition can destroy temporal memory. PyTorch's built-in RNN dropout is active only between stacked recurrent layers, so setting dropout on a single-layer RNN does not regularize the recurrent computation in the way beginners often expect.

Compare no dropout, input dropout, inter-layer dropout, and their combination under identical seeds and initialization. Verify train/eval behavior directly, then measure convergence speed, validation gap, and sequence quality instead of assuming a larger dropout probability is always safer.

Part 2 - Mathematical form

$$\widetilde{x}_t = \frac{m_t \odot x_t}{1-p}, \qquad m_{t,j} \sim \text{Bernoulli}(1-p).$$

Part 3 - Hints and Helper

Use at least two recurrent layers for built-in inter-layer dropout. Confirm that repeated evaluation passes are deterministic after calling eval mode.

Part 4 - Python code skeleton

```
import torch

def E01(model, data, config):
    variants = {
        "none": build_rnn(input_dropout=0.0, layer_dropout=0.0, ...),
        "input": build_rnn(input_dropout=..., layer_dropout=0.0),
        "between_layers": build_rnn(input_dropout=0.0, layer_dropout=...),
        "combined": build_rnn(input_dropout=..., layer_dropout=...),
    }
    results = run_controlled_training(variants, data, seed=...)
    verify_train_eval_dropout(variants, sample_batch=...)
    return compare_dropout_locations(results)
```

15.8.3 Exercise 02 - Use Weight Decay and Analyze Its Effect on RNN Stability and Generalization

Part 1 - Why this matters

Recurrent weights are reused at every time step, so an excessive weight norm can repeatedly amplify hidden-state perturbations. Weight decay can limit this growth and improve generalization, but too much decay removes the model's ability to preserve useful temporal information.

The important question is not whether decay lowers training loss. It is how parameter norms, hidden-state magnitudes, gradient norms, and validation performance change together. AdamW also separates decay from the adaptive gradient update, which makes it different from adding an L2 term carelessly to every parameter.

Sweep several decay strengths with the same initialization and data order. Track recurrent and output-layer norms separately, exclude biases when appropriate, and identify the point where stabilization becomes underfitting.

Part 2 - Mathematical form

$$\theta_{k+1} = (1 - \eta\lambda)\theta_k - \eta\,\widehat{g}_k.$$

Part 3 - Hints and Helper

Create optimizer parameter groups so biases and normalization parameters can remain undecayed. Compare validation results at equal training budgets.

Part 4 - Python code skeleton

```
import torch

def E02(model, data, config):
    results = {}
    for decay in config.weight_decays:
        candidate = clone_initial_model(model)
        groups = make_decay_parameter_groups(candidate, exclude=...)
        optimizer = torch.optim.AdamW(groups, lr=..., weight_decay=decay)
        results[decay] = train_with_norm_logging(
            candidate, data, optimizer, ...
        )
    return analyze_decay_tradeoff(results)
```

15.8.4 Exercise 03 - Implement Gradient Clipping and Compare Different Clip Norm Values Experimentally

Part 1 - Why this matters

Backpropagation through time multiplies many Jacobians, allowing a rare batch to produce an enormous gradient. One uncontrolled update can move recurrent weights into an unstable region and cause exploding hidden states or a NaN loss several steps later.

Norm clipping rescales the complete gradient vector only when its norm exceeds a threshold. A very small threshold, however, makes nearly every update the same limited size and can hide a bad learning rate while slowing genuine learning.

Record the gradient norm before clipping, the norm after clipping, and the fraction of steps affected. Compare multiple thresholds plus an unclipped baseline on the same long-sequence task, then relate clipping frequency to convergence and validation quality.

Part 2 - Mathematical form

$$g' = g \min\left(1, \frac{c}{\|g\|_2 + \varepsilon}\right).$$

Part 3 - Hints and Helper

Call clipping after backward and before optimizer step. Log the value returned by the clipping function because it represents the pre-clipping norm.

Part 4 - Python code skeleton

```
import torch

def E03(model, data, config):
    reports = {}
    for clip_norm in [None, *config.clip_norms]:
        candidate = clone_initial_model(model)
        reports[clip_norm] = train_rnn(
            candidate,
            data,
            after_backward=lambda: clip_and_log(
                candidate.parameters(), max_norm=clip_norm, ...
            ),
        )
    return compare_clipping_reports(reports)
```

15.8.5 Exercise 04 - Implement Layer Normalization in RNN and Measure Hidden-State Stability

Part 1 - Why this matters

The scale of an RNN hidden state may drift as the same transition is applied repeatedly. Layer normalization stabilizes each sample across hidden features and does not depend on batch statistics, making it suitable for variable-length sequences and small batches.

Normalization changes the geometry of the recurrent transition, not merely the numerical range of the final output. Its placement before or after the activation affects gradient flow, representational capacity, and the model's ability to preserve magnitude information.

Implement a custom recurrent cell with optional layer normalization. Compare hidden-state mean, variance, saturation, gradient norms, and long-sequence accuracy against an unnormalized cell while keeping the remaining architecture identical.

Part 2 - Mathematical form

$$\mathrm{LN}(a) = \gamma \odot \frac{a - \mu(a)}{\sqrt{\sigma^2(a) + \varepsilon}} + \beta.$$

Part 3 - Hints and Helper

Normalize across the hidden feature dimension for each sample and time step. Do not mix padding positions into the stability statistics.

Part 4 - Python code skeleton

```
import torch
from torch import nn

class LayerNormRNNCell(nn.Module):
    def __init__(self, input_size, hidden_size):
        super().__init__()
        self.xh = nn.Linear(input_size, hidden_size)
        self.hh = nn.Linear(hidden_size, hidden_size, bias=False)
        self.norm = nn.LayerNorm(hidden_size)

    def forward(self, x_t, h_prev):
        preactivation = ...
        h_t = torch.tanh(self.norm(preactivation))
        return h_t

def E04(model, data, config):
    return compare_hidden_state_stability(... )
```

15.8.6 Exercise 05 - Diagnose NaN Loss and Build Automatic Early-Failure Diagnostics for RNN Training

Part 1 - Why this matters

A NaN loss is usually the final symptom of an earlier numerical failure. Invalid input values, exploding hidden states, excessive logits, unstable mixed precision, or a non-finite gradient may appear several operations before the loss becomes visibly corrupted.

Restarting with a lower learning rate can mask the problem without locating it. A useful diagnostic system checks tensors at stage boundaries, records the first failing batch and time step, preserves the relevant inputs, and distinguishes NaN values from positive or negative infinity.

Inject controlled failures into an RNN training loop and make the monitor stop immediately with a compact evidence report. The report should include input ranges, hidden-state norms, loss, gradient norms, parameter names, optimizer settings, and a reproducible batch snapshot.

Part 2 - Mathematical form

$$\text{finite}(z) \iff \neg\,\text{isnan}(z) \wedge \neg\,\text{isinf}(z).$$

Part 3 - Hints and Helper

Check inputs before forward, activations after forward, loss before backward, gradients after backward, and parameters after the optimizer step.

Part 4 - Python code skeleton

```
import torch

def E05(model, data, config):
    monitor = FiniteTensorMonitor(save_directory=..., fail_fast=True)
    for batch_id, batch in enumerate(data):
        monitor.check("input", batch.inputs, context=...)
        outputs, hidden = model(batch.inputs)
        monitor.check("hidden", hidden, context=...)
        loss = compute_loss(outputs, batch.targets)
        monitor.check("loss", loss, context=...)
        loss.backward()
        monitor.check_named_gradients(model.named_parameters(), context=...)
        ...
    return monitor.report()
```

15.8.7 Exercise 06 - Compare Recurrent Weight Initializations and Observe Training Differences

Part 1 - Why this matters

Initialization determines the behavior of an RNN before learning begins, and recurrent reuse magnifies its consequences over time. A matrix with an unsuitable spectral scale can erase information rapidly or repeatedly amplify it across a long sequence.

Orthogonal initialization is often useful because it preserves vector norms in the linear recurrent transformation, but the activation derivative and input contribution still affect total stability. Therefore, initialization quality must be measured through actual signal and gradient

propagation.

Compare zero, small normal, Xavier, orthogonal, and deliberately oversized recurrent initialization. Under controlled seeds, record early hidden-state variance, gradient norms by time distance, convergence speed, and failure rates across repeated runs.

Part 2 - Mathematical form

$$h_t = \phi(W_{xh}x_t + W_{hh}h_{t-1} + b), \qquad \rho(W_{hh}) \text{ influences temporal growth.}$$

Part 3 - Hints and Helper

Initialize input and recurrent matrices separately. Reuse the same non-recurrent parameters and data order for a fair comparison.

Part 4 - Python code skeleton

```
import torch

def E06(model, data, config):
    initializers = {
        "zero": ...,
        "normal": ...,
        "xavier": ...,
        "orthogonal": ...,
        "oversized": ...,
    }
    results = {}
    for name, initialize in initializers.items():
        candidate = build_rnn(...)
        initialize_recurrent_weights(candidate, initialize)
        results[name] = train_with_signal_tracking(candidate, data, ...)
    return compare_initializations(results)
```

15.8.8 Exercise 07 - Compare SGD and Adam for RNNs and Explain Optimizer Sensitivity

Part 1 - Why this matters

RNN optimization landscapes are shaped by long products of temporal derivatives, which can create narrow directions, plateaus, and occasional gradient spikes. Optimizers therefore interact strongly with clipping, initialization, sequence length, and learning rate.

Adam adapts step sizes per parameter and may make early progress easier, while SGD can require careful momentum and scheduling but sometimes produces different generalization behavior. Comparing both at arbitrary default learning rates would confuse optimizer choice with tuning quality.

Run learning-rate sweeps for SGD, SGD with momentum, and Adam from identical starting weights. Compare time-to-target, best validation score, gradient-to-update ratio, sensitivity across seeds, and final sequence quality.

Part 2 - Mathematical form

$$\theta_{k+1}^{\mathrm{SGD}} = \theta_k - \eta g_k, \qquad \theta_{k+1}^{\mathrm{Adam}} = \theta_k - \eta \frac{\widehat{m}_k}{\sqrt{\widehat{v}_k} + \varepsilon}.$$

Part 3 - Hints and Helper

Give each optimizer a small learning-rate search rather than using the same rate. Keep gradient clipping policy identical across variants.

Part 4 - Python code skeleton

```
import torch

def E07(model, data, config):
    optimizer_builders = {
        "sgd": lambda p, lr: torch.optim.SGD(p, lr=lr),
        "momentum": lambda p, lr: torch.optim.SGD(p, lr=lr, momentum=...),
        "adam": lambda p, lr: torch.optim.Adam(p, lr=lr),
    }
    results = run_optimizer_sweeps(
        model, data, optimizer_builders, learning_rates=..., seeds=...
    )
    return summarize_optimizer_sensitivity(results)
```

15.8.9 Exercise 08 - Implement Learning-Rate Scheduling and Confirm Reduced Instability in RNN Training

Part 1 - Why this matters

A learning rate that is useful during early exploration may be too aggressive after the model approaches a good region. In recurrent training, late high-magnitude updates can repeatedly disturb a fragile hidden-state dynamic and produce oscillating validation loss.

Schedulers must be stepped at the intended frequency and in the correct order. Confusing batch-level and epoch-level schedules, or restoring a checkpoint without scheduler state, silently changes the experiment and makes results difficult to reproduce.

Compare constant, step decay, cosine decay, and plateau scheduling. Log the learning rate, count instability events, and determine whether smoother loss reflects genuine improvement or simply steps that became too small.

Part 2 - Mathematical form

$$\eta_t = \eta_{\min} + \tfrac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \frac{\pi t}{T}\right).$$

Part 3 - Hints and Helper

Plot learning rate beside loss and gradient norm. Save and restore scheduler state together with model and optimizer state.

Part 4 - Python code skeleton

```
import torch

def E08(model, data, config):
    schedules = {
        "constant": ...,
        "step": ...,
        "cosine": ...,
        "plateau": ...,
    }
    results = {}
    for name, scheduler_factory in schedules.items():
        candidate = clone_initial_model(model)
        optimizer = build_optimizer(candidate.parameters(), lr=...)
        scheduler = scheduler_factory(optimizer)
        results[name] = train_with_scheduler_logging(
            candidate, data, optimizer, scheduler, step_policy=...
        )
    return compare_schedule_stability(results)
```

15.8.10 Exercise 09 - Debug Cases Where Loss Decreases but Sequence Quality Does Not

Part 1 - Why this matters

A decreasing average loss does not prove that a sequence model is solving the intended task. Padding tokens can dominate the reduction, teacher forcing can hide autoregressive failure, temporal targets can be shifted incorrectly, or a model can improve common easy steps while ignoring rare important transitions.

This failure requires evaluating the complete prediction process rather than only the training objective. Free-running generation, horizon-specific metrics, masks, alignment checks, qualitative samples, and simple baselines expose problems that a scalar batch loss hides.

Construct several intentionally broken pipelines and require the debugging suite to identify each one. Include incorrect masking, target shift, teacher-forcing leakage, majority-token prediction, and a metric computed on the wrong axis.

Part 2 - Mathematical form

$$L = \frac{\sum_{b,t} m_{b,t}\, \ell(\widehat{y}_{b,t}, y_{b,t})}{\sum_{b,t} m_{b,t}}.$$

Part 3 - Hints and Helper

Recompute metrics from saved predictions independently of the training loop. Compare teacher-forced and free-running outputs on the same sequences.

Part 4 - Python code skeleton

```
import torch

def E09(model, data, config):
    predictions = collect_predictions(
        model, data, modes=["teacher_forced", "free_running"], ...
    )
    audits = {
        "alignment": audit_target_alignment(data, ...),
        "masking": audit_loss_mask(data, ...),
        "baselines": compare_sequence_baselines(predictions, ...),
        "per_step": compute_metrics_by_time_step(predictions, ...),
        "samples": inspect_failure_samples(predictions, ...),
    }
    return diagnose_loss_quality_disagreement(audits)
```

15.8.11 Exercise 10 - Train a Stabilized RNN on CUDA and Measure Regularization Runtime Overhead

Part 1 - Why this matters

A stabilization method may improve accuracy but also add computation, memory traffic, or synchronization. Layer normalization, dropout, gradient clipping, diagnostic hooks, and frequent logging have different costs that may be negligible on small tests yet significant at production scale.

CUDA timing is asynchronous, so wall-clock measurements without synchronization can attribute unfinished GPU work to later operations. A fair benchmark also needs warm-up iterations and must isolate data loading from forward, backward, clipping, and optimizer time.

Benchmark a baseline and incremental stabilization variants on the same GPU workload. Report examples per second, peak memory, step-time distribution, and validation gain so engineering decisions are based on both stability and cost.

Part 2 - Mathematical form

$$\text{overhead}(r) = 100\frac{T_r - T_0}{T_0}\%, \qquad \text{throughput} = \frac{N}{T}.$$

Part 3 - Hints and Helper

Use CUDA events or explicit synchronization around timed regions. Warm up the kernels and exclude model construction from steady-state timing.

Part 4 - Python code skeleton

```
import torch

def E10(model, data, config):
    device = torch.device("cuda")
    variants = build_incremental_stabilization_variants(model, ...)
    reports = {}
    for name, candidate in variants.items():
        candidate = candidate.to(device)
        warm_up_training(candidate, data, device=device, steps=...)
        torch.cuda.reset_peak_memory_stats(device)
        reports[name] = benchmark_training_regions(
            candidate, data, device=device, synchronize=True, ...
        )
    return compute_overhead_and_quality(reports)
```

15.8.12 Exercise 11 - Compare Mixed-Precision Stability With and Without Gradient Scaling in RNNs

Part 1 - Why this matters

Mixed precision accelerates supported GPU operations and reduces activation memory, but recurrent training repeatedly propagates small derivatives through time. In float16, these gradients may underflow to zero while large values can overflow to infinity.

Gradient scaling multiplies the loss before backward so small gradients occupy a representable numerical range, then unscales them before clipping and the optimizer update. Clipping scaled gradients or stepping after an overflow produces misleading behavior.

Compare full precision, autocast without scaling, and autocast with dynamic scaling on increasing sequence lengths. Track skipped updates, scale evolution, non-finite gradients, zero-gradient fraction, throughput, and validation quality.

Part 2 - Mathematical form

$$\widetilde{L} = sL, \qquad \nabla_\theta L = \frac{1}{s}\nabla_\theta \widetilde{L}.$$

Part 3 - Hints and Helper

Unscale gradients before computing their true norm or applying clipping. Record how often the scaler skips an optimizer step.

Part 4 - Python code skeleton

```
import torch

def E11(model, data, config):
    modes = ["float32", "amp_unscaled", "amp_scaled"]
    reports = {}
    for mode in modes:
        candidate = clone_initial_model(model).cuda()
        scaler = build_grad_scaler(enabled=(mode == "amp_scaled"))
        reports[mode] = train_precision_variant(
            candidate,
            data,
            autocast_enabled=(mode != "float32"),
            scaler=scaler,
            unscale_before_clip=True,
            ...
        )
    return compare_precision_stability(reports)
```

15.8.13 Exercise 12 - Analyze Generalization Under Label Noise and Sequence-Noise Propagation

Part 1 - Why this matters

A wrong label in an independent classifier affects one sample, but noise in a sequential task can influence several windows and future hidden states. Overlapping windows may duplicate one corrupted event many times, increasing its effective weight in training.

Random label noise, temporally clustered noise, and persistent target bias have different consequences. A model may memorize isolated errors yet systematically adapt to a long corrupted segment, so an average noise percentage is not a complete description.

Inject controlled noise before window construction and compare it with noise injected after windowing. Measure clean-test performance, error persistence across horizons, gradient outliers, calibration, and sensitivity to regularization.

Part 2 - Mathematical form

$$\widetilde{y}_t = \begin{cases} y_t, & z_t = 0, \\ c_t, & z_t = 1, \end{cases} \qquad z_t \sim \text{Bernoulli}(q).$$

Part 3 - Hints and Helper

Keep a clean reference target for evaluation. Report how many training windows contain each corrupted source time step.

Part 4 - Python code skeleton

```
import torch

def E12(model, data, config):
    clean_series = ...
    noise_variants = inject_sequence_noise(
        clean_series,
        rates=config.noise_rates,
        patterns=["random", "clustered", "biased"],
        stage=...,
    )
    results = train_noise_variants(model, noise_variants, ...)
    return analyze_noise_propagation(
        results, clean_targets=..., metrics_by_horizon=True
    )
```

15.8.14 Exercise 13 - Perform an Ablation Study to Identify Which Stabilization Methods Actually Help

Part 1 - Why this matters

When several stabilization methods are enabled together, a good result does not reveal which component caused the improvement. Some methods may be redundant, while another may only work through its interaction with clipping, normalization, or learning-rate scheduling.

An ablation study removes or adds one controlled component while holding data splits, initialization, training budget, and evaluation constant. Multiple seeds are necessary because recurrent training variance can be larger than the apparent gain from a method.

Design a baseline, single-method variants, the full system, and selected interaction variants. Summarize mean, spread, failure rate, runtime, and memory, then distinguish practical gains from changes smaller than experimental noise.

Part 2 - Mathematical form

$$\Delta_j = M(\mathcal{S}) - M(\mathcal{S} \setminus \{j\}), \qquad \overline{M} = \frac{1}{K}\sum_{k=1}^{K} M_k.$$

Part 3 - Hints and Helper

Reuse fixed split files and seed lists. Change only one named factor per primary ablation and include compute cost in the final table.

Part 4 - Python code skeleton

```
import torch

def E13(model, data, config):
    variants = build_ablation_matrix(
```

```
        methods=["dropout", "decay", "clipping", "layer_norm", "scheduler"],
        include_full=True,
        interactions=...,
    )
    results = run_repeated_experiments(
        model, data, variants, seeds=config.seeds, budget=...
    )
    return summarize_ablation_effects(
        results, include_runtime=True, include_failure_rate=True
    )
```

15.8.15 Exercise 14 - Implement Early Stopping and Compare It with Regularization and Dropout

Part 1 - Why this matters

Early stopping limits how long a model adapts to training-specific details, while dropout and weight penalties change the optimization path itself. These mechanisms can complement one another, but they should not be treated as interchangeable switches.

A reliable stopping rule needs a monitored validation metric, minimum meaningful improvement, patience, checkpoint restoration, and a burn-in period. Noisy sequence metrics can trigger premature stopping unless their temporal variability is considered.

Compare fixed-budget training, early stopping alone, regularization alone, and their combination. Evaluate restored best checkpoints on untouched sequences and include training time so the comparison captures both generalization and efficiency.

Part 2 - Mathematical form

$$\text{stop at } t \text{ if } M_{t-p:t} \text{ contains no improvement greater than } \delta.$$

Part 3 - Hints and Helper

Save the complete best state, including optimizer and scheduler if training may resume. Never report the last in-memory weights when early stopping selected an earlier epoch.

Part 4 - Python code skeleton

```
import torch

def E14(model, data, config):
    variants = build_early_stopping_variants(...)
    results = {}
    for name, candidate in variants.items():
        stopper = EarlyStopping(
            patience=config.patience,
            min_delta=config.min_delta,
```

```
            mode=...,
        )
        history = train_with_early_stopping(
            candidate, data, stopper=stopper, restore_best=True, ...
        )
        results[name] = evaluate_restored_model(candidate, history, ...)
    return compare_stopping_and_regularization(results)
```

15.8.16 Exercise 15 - Build a Comprehensive Debugging Checklist for RNN Training Stability

Part 1 - Why this matters

RNN failures can originate in data alignment, padding masks, hidden-state lifecycle, graph detachment, numerical precision, gradient propagation, optimizer order, or evaluation logic. Debugging only the loss curve leaves most of this system unexamined.

A useful checklist is executable rather than a collection of vague reminders. Each check should produce evidence, define an expected range or invariant, and identify the next narrower test when it fails.

Build a staged diagnostic runner covering data, forward state, loss, backward pass, update, evaluation, reproducibility, and CUDA behavior. Test it against deliberately broken cases and ensure the final report separates errors, warnings, and contextual measurements.

Part 2 - Mathematical form

$$\mathcal{D} = \mathcal{D}_{\text{data}} \cup \mathcal{D}_{\text{forward}} \cup \mathcal{D}_{\text{backward}} \cup \mathcal{D}_{\text{eval}}.$$

Part 3 - Hints and Helper

Run cheap invariant checks before expensive profiling. Store the first failing batch and enough configuration information to reproduce it.

Part 4 - Python code skeleton

```
import torch

def E15(model, data, config):
    checklist = DiagnosticChecklist()
    checklist.add(
        "data_alignment", check_sequence_alignment, severity="error"
    )
    checklist.add("padding_mask", check_padding_mask, severity="error")
    checklist.add("hidden_state", inspect_hidden_state_dynamics, ...)
    checklist.add("finite_values", check_all_stages_finite, ...)
    checklist.add("gradient_flow", inspect_temporal_gradients, ...)
    checklist.add("update_order", verify_optimizer_order, ...)
```

```
    checklist.add("evaluation", verify_free_running_metrics, ...)
    checklist.add("reproducibility", compare_seeded_runs, ...)
    return checklist.run(model=model, data=data, config=config)
```

15.8.17 Exercise 16 - Evaluate Robustness Under Time-Series Distribution Shift and Document Failure Modes

Part 1 - Why this matters

Time-series deployment data rarely remains identical to training data. Sensor recalibration, seasonal changes, policy interventions, user behavior, and changing variance can alter the input distribution or the relationship between history and target.

Randomly shuffling a test split cannot represent these shifts. Robustness must be evaluated chronologically and through controlled perturbations such as level shift, trend change, variance growth, missing bursts, frequency change, and unseen seasonal regimes.

Measure clean and shifted performance by forecast horizon, track hidden-state and confidence changes, and group failures by shift type. The objective is to learn where the RNN stops being trustworthy, not merely to produce one robustness score.

Part 2 - Mathematical form

$$\text{degradation}_s = \frac{M_s - M_0}{|M_0| + \varepsilon}, \qquad P_{\text{train}}(x, y) \neq P_{\text{deploy}}(x, y).$$

Part 3 - Hints and Helper

Fit preprocessing only on the original training period. Keep shift severity explicit and report absolute metrics alongside relative degradation.

Part 4 - Python code skeleton

```
import torch

def E16(model, data, config):
    shifts = {
        "level": lambda x: apply_level_shift(x, ...),
        "trend": lambda x: apply_trend_shift(x, ...),
        "variance": lambda x: change_variance(x, ...),
        "missing_burst": lambda x: mask_temporal_burst(x, ...),
        "frequency": lambda x: alter_seasonal_frequency(x, ...),
    }
    baseline = evaluate_by_horizon(model, data.clean_test)
    shifted = evaluate_shift_suite(model, data.clean_test, shifts, ...)
    return document_shift_failure_modes(baseline, shifted)
```

15.8.18 Exercise 17 - Build Calibration and Confidence Analysis for RNN Predictions Across Time Steps

Part 1 - Why this matters

A prediction can be accurate on average while its confidence is unreliable. In sequential classification, confidence may become increasingly overconfident at later time steps; in forecasting, uncertainty intervals may cover far fewer targets as the horizon grows.

Calibration must therefore be analyzed by temporal position or forecast horizon rather than pooled into one number. Padding, class imbalance, and correlated observations also require careful masking and interpretation of confidence bins.

Create reliability diagrams, expected calibration error, proper scoring metrics, and coverage-width analysis by time step. Compare the uncalibrated model with a calibration method fitted only on validation data, then test both on clean and shifted sequences.

Part 2 - Mathematical form

$$\mathrm{ECE} = \sum_{b=1}^{B} \frac{|S_b|}{N} \left|\mathrm{acc}(S_b) - \mathrm{conf}(S_b)\right| .$$

Part 3 - Hints and Helper

Never fit calibration parameters on the test set. Mask padding and show the sample count of each confidence bin and time step.

Part 4 - Python code skeleton

```
import torch

def E17(model, data, config):
    validation_outputs = collect_temporal_outputs(model, data.validation, ...)
    calibrator = fit_calibrator(validation_outputs, method=...)
    test_outputs = collect_temporal_outputs(model, data.test, ...)
    reports = {}
    for name, outputs in {
        "raw": test_outputs,
        "calibrated": calibrator(test_outputs),
    }.items():
        reports[name] = calibration_by_time_step(
            outputs, bins=..., mask_padding=True, ...
        )
    return reports
```

15.8.19 Exercise 18 - Stress Test Very Long Sequences and Observe When Regularization Stops Helping

Part 1 - Why this matters

Regularization cannot remove the fundamental limitations of vanilla recurrent dynamics. As sequence length grows, useful gradients may vanish, computation becomes strictly sequential, memory for backpropagation increases, and truncation prevents learning dependencies beyond its boundary.

A method can appear successful on short sequences yet fail abruptly beyond a dependency length. Increasing dropout or decay at that point may reduce overfitting while making memory retention even worse, so failure must be separated into optimization, capacity, numerical, and resource limits.

Sweep sequence length and dependency distance under a fixed benchmark. Record accuracy, gradient signal to early time steps, hidden-state similarity, runtime, memory, clipping frequency, and failure type for baseline and regularized variants.

Part 2 - Mathematical form

$$\frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial h_T} \prod_{k=t+1}^{T} \frac{\partial h_k}{\partial h_{k-1}}.$$

Part 3 - Hints and Helper

Increase one scale variable at a time and stop safely before exhausting GPU memory. Include a task whose required dependency distance is known exactly.

Part 4 - Python code skeleton

```
import torch

def E18(model, data, config):
    reports = []
    for length in config.sequence_lengths:
        for method in config.regularization_variants:
            candidate = build_controlled_rnn(method=method, ...)
            try:
                result = run_long_dependency_stress_test(
                    candidate, length=length, monitor_memory=True, ...
                )
            except RuntimeError as error:
                result = classify_resource_or_numerical_failure(error, ...)
            reports.append({"length": length, "method": method, **result})
    return find_regularization_breakpoints(reports)
```

15.8.20 Exercise 19 - Write a Practical RNN Stabilization Strategy Guide for Real Engineering Projects

Part 1 - Why this matters

A stabilization guide should turn experimental evidence into an ordered engineering process. Applying every technique by default creates unnecessary complexity and makes it impossible to identify whether data, dynamics, optimization, or evaluation caused a problem.

The guide must connect each observable symptom to measurements, likely causes, controlled interventions, and acceptance criteria. It should also distinguish correctness checks required for every project from optional regularization chosen after a clean baseline is understood.

Synthesize the experiments into a decision-oriented report. Include baseline design, reproducibility, monitoring, intervention order, CUDA and precision checks, shift testing, rollback conditions, and limits that justify moving from vanilla RNN to gated or attention-based models.

Part 2 - Mathematical form

$$\text{symptom} \longrightarrow \text{measurement} \longrightarrow \text{hypothesis} \longrightarrow \text{controlled test} \longrightarrow \text{decision}.$$

Part 3 - Hints and Helper

Every recommendation must cite a recorded experiment or threshold from the project. Separate universal checks from task-specific tuning advice.

Part 4 - Python code skeleton

```
import torch

def E19(model, data, config):
    evidence = load_chapter_experiment_results(...)
    guide = StabilizationGuide()
    guide.add_baseline_protocol(evidence, ...)
    guide.add_symptom_decision_table(evidence, ...)
    guide.add_regularization_selection_rules(evidence, ...)
    guide.add_cuda_and_precision_checks(evidence, ...)
    guide.add_shift_and_calibration_tests(evidence, ...)
    guide.add_rollback_and_escalation_criteria(evidence, ...)
    guide.validate_all_claims_have_evidence()
    return guide.export(format=...)
```

15.9 RNN07 - Efficiency, Profiling, and CUDA Performance

Models and Training

15.9.1 Exercise 00 - Benchmark Vanilla RNN vs GRU vs LSTM Runtime and Memory to Understand Engineering Tradeoffs

Part 1 - Why this matters

Vanilla RNN, GRU, and LSTM process sequences through related recurrent structures, but they do not have the same number of gates, parameters, matrix operations, or saved activations. Choosing a model only from accuracy ignores the cost that determines whether it can be trained and served within real hardware limits.

A fair comparison requires equal input size, hidden size, layer count, sequence length, batch size, device, precision, and measurement procedure. It must also distinguish parameter memory from training activation memory and inference memory because backpropagation changes the ranking substantially.

Benchmark forward inference and complete training steps after warm-up. Report latency distributions, throughput, peak CUDA memory, parameter count, convergence quality, and the cost per unit of validation improvement instead of declaring one recurrent architecture universally fastest.

Part 2 - Mathematical form

$$P_{\text{RNN}} \approx H(I+H), \quad P_{\text{GRU}} \approx 3H(I+H), \quad P_{\text{LSTM}} \approx 4H(I+H).$$

Part 3 - Hints and Helper

Synchronize CUDA around timing and rebuild each model from a controlled seed. Compare both equal hidden size and approximately equal parameter count.

Part 4 - Python code skeleton

```
import torch

def E00(model, data, config):
    builders = {"rnn": build_rnn, "gru": build_gru, "lstm": build_lstm}
    reports = {}
    for name, builder in builders.items():
        candidate = builder(
            input_size=..., hidden_size=..., layers=...
        ).cuda()
        reports[name] = benchmark_model(
            candidate, data, warmup=..., repetitions=...,
            measure_training=True, synchronize=True
        )
    return compare_runtime_memory_and_quality(reports)
```

15.9.2 Exercise 01 - Profile RNN Training with PyTorch Profiler and Identify Kernel and Memory Bottlenecks

Part 1 - Why this matters

A slow training loop may be limited by recurrent kernels, tensor copies, Python work, synchronization, data loading, or optimizer updates. Overall step time cannot reveal which component dominates or whether the GPU is waiting for the CPU.

A profiler trace must cover representative warm-up and active steps while avoiding endless collection overhead. Operator tables explain aggregate cost, whereas the timeline reveals gaps, copies, kernel ordering, memory allocation, and unexpected synchronization.

Profile a controlled RNN workload, rank operations by CUDA and CPU time, inspect memory-heavy events, and form one bottleneck hypothesis. Apply a targeted change and repeat the same profile to prove whether the intervention improved the limiting region rather than moving cost elsewhere.

Part 2 - Mathematical form

$$T_{\text{step}} = T_{\text{input}} + T_{\text{forward}} + T_{\text{backward}} + T_{\text{update}} + T_{\text{sync}}.$$

Part 3 - Hints and Helper

Use a short profiler schedule and add named record-function regions. Profile only after the workload has warmed up.

Part 4 - Python code skeleton

```
import torch
from torch.profiler import profile, ProfilerActivity, record_function

def E01(model, data, config):
    schedule = torch.profiler.schedule(
        wait=..., warmup=..., active=..., repeat=1
    )
    with profile(
        activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
        schedule=schedule,
        profile_memory=True,
        record_shapes=True,
    ) as profiler:
        for batch in data:
            with record_function("rnn_training_step"):
                ...
            profiler.step()
    return analyze_profiler_trace(profiler, ...)
```

15.9.3 Exercise 02 - Optimize Data Transfer by Minimizing Host-to-Device Copies in Sequence Training

Part 1 - Why this matters

A powerful GPU can remain idle when every batch is prepared slowly or copied through unnecessary intermediate tensors. Repeated device transfers inside the time-step loop are especially damaging because they add launch overhead and break what should be one batched recurrent operation.

Pinned host memory permits asynchronous copies, but non-blocking transfer helps only when the source is pinned and work can overlap safely. Converting types, transposing layouts, or rebuilding masks after transfer can also create hidden copies and allocations.

Instrument the pipeline from DataLoader output to model input. Move each complete batch once, reuse device-side buffers where practical, and compare transfer time, GPU idle gaps, throughput, and correctness before and after the change.

Part 2 - Mathematical form

$$T_{\text{pipeline}} \approx \max(T_{\text{prepare}}, T_{\text{copy}}, T_{\text{compute}}) \quad \text{when stages overlap.}$$

Part 3 - Hints and Helper

Move inputs, targets, lengths, and masks together at the batch boundary. Inspect contiguity and dtype before deciding that an explicit copy is necessary.

Part 4 - Python code skeleton

```
import torch

def E02(model, data, config):
    loader = build_loader(
        data, pin_memory=True, num_workers=..., persistent_workers=...
    )
    stream = torch.cuda.Stream()
    reports = {}
    for strategy in ["baseline", "single_batch_copy", "prefetched"]:
        reports[strategy] = benchmark_transfer_strategy(
            model,
            loader,
            strategy=strategy,
            non_blocking=True,
            stream=stream,
            ...
        )
    return compare_transfer_and_compute_overlap(reports)
```

15.9.4 Exercise 03 - Measure Sequence-Length Impact on Throughput and Find the Practical GPU Limit

Part 1 - Why this matters

Recurrent computation is sequential across time even when operations within each step are parallel. Increasing sequence length therefore raises latency and the number of activations retained for backpropagation, while also extending the dependency path that gradients must traverse.

The practical limit is not simply the longest sequence that fits once. A usable configuration must leave memory headroom, avoid severe throughput collapse, remain numerically stable, and support repeated steps without allocation spikes or out-of-memory recovery.

Sweep sequence lengths using fixed model and batch settings, then repeat with a token-budget policy that reduces batch size as length grows. Record tokens per second, samples per second, step latency, peak memory, gradient behavior, and the first unsafe configuration.

Part 2 - Mathematical form

$$\text{tokens/s} = \frac{BT}{t_{\text{step}}}, \qquad M_{\text{train}} = M_{\text{params}} + M_{\text{state}} + O(BTH).$$

Part 3 - Hints and Helper

Warm up each shape separately because new dimensions may trigger new kernel choices or allocations. Reserve memory headroom rather than benchmarking at the exact OOM boundary.

Part 4 - Python code skeleton

```
import torch

def E03(model, data, config):
    reports = []
    for length in config.sequence_lengths:
        batch_size = choose_batch_for_token_budget(length, ...)
        batch = make_sequence_batch(batch_size, length, ...).cuda()
        result = benchmark_shape(
            model, batch, training=True, warmup=..., repetitions=...
        )
        reports.append({"length": length, "batch": batch_size, **result})
    return identify_practical_gpu_limit(reports, headroom=...)
```

15.9.5 Exercise 04 - Compare Packed vs Padded Sequence Performance and Measure Speedup and Memory Changes

Part 1 - Why this matters

Padding lets variable-length sequences share a rectangular tensor, but recurrent computation may be wasted on positions that contain no data. The waste grows when length variance is large or a few unusually long sequences determine the batch width.

Packed sequences can skip padded time steps, yet sorting, packing, unpacking, and bookkeeping add overhead. When lengths are similar or batches are small, the extra complexity may provide little benefit and can introduce alignment bugs.

Benchmark padded masking and packed execution across several length distributions. Verify output parity only on valid positions, then compare useful-token throughput, peak memory, packing overhead, and total training time.

Part 2 - Mathematical form

$$\eta_{\text{padding}} = \frac{\sum_{i=1}^{B} T_i}{B \max_i T_i}, \qquad \text{waste} = 1 - \eta_{\text{padding}}.$$

Part 3 - Hints and Helper

Use the same sequences and recurrent weights for both paths. Restore original batch order after unpacking before comparing outputs.

Part 4 - Python code skeleton

```
import torch
from torch.nn.utils.rnn import pack_padded_sequence, pad_packed_sequence

def E04(model, data, config):
    padded, lengths, mask = ...
    packed = pack_padded_sequence(
        padded, lengths.cpu(), batch_first=True, enforce_sorted=...
    )
    padded_report = benchmark_padded_path(model, padded, mask, ...)
    packed_report = benchmark_packed_path(model, packed, ...)
    verify_valid_output_parity(padded_report, packed_report, mask, ...)
    return compare_useful_token_efficiency(padded_report, packed_report)
```

15.9.6 Exercise 05 - Evaluate batch_first and Its Effect on Memory Layout and Performance

Part 1 - Why this matters

The batch-first option changes the interface layout between batch-time-feature and time-batch-feature, but it does not change the mathematical recurrence. Performance differences usually come from surrounding transposes, contiguity, downstream operators, and copies rather than the semantic flag alone.

A transpose can be a cheap view, yet a later operator may require contiguous memory and silently materialize a copy. Timing only the RNN call misses layout conversion costs that belong to the real pipeline.

Construct equivalent batch-first and time-first pipelines and inspect strides, contiguity, storage sharing, and profiler copy events. Compare end-to-end throughput and output parity before selecting the layout that minimizes conversions across the complete application.

Part 2 - Mathematical form

$$X_{\mathrm{BF}} \in \mathbb{R}^{B\times T\times I}, \qquad X_{\mathrm{TF}} = X_{\mathrm{BF}}^{\top(0,1)} \in \mathbb{R}^{T\times B\times I}.$$

Part 3 - Hints and Helper

Measure layout preparation plus recurrent execution plus downstream loss. Do not call contiguous unless an operation actually requires it.

Part 4 - Python code skeleton

```
import torch

def E05(model, data, config):
    x_batch_first = ...
    x_time_first = x_batch_first.transpose(0, 1)
    layout_report = inspect_layouts(x_batch_first, x_time_first)
    reports = {
        "batch_first": benchmark_complete_layout_pipeline(
            build_rnn(batch_first=True, ...), x_batch_first, ...
        ),
        "time_first": benchmark_complete_layout_pipeline(
            build_rnn(batch_first=False, ...), x_time_first, ...
        ),
    }
    verify_layout_output_parity(reports, ...)
    return layout_report, reports
```

15.9.7 Exercise 06 - Benchmark Hidden Sizes and Layer Counts and Build a Scaling Curve

Part 1 - Why this matters

Hidden size controls the width of recurrent matrix multiplications, while layer count repeats recurrent processing at every time step. Their costs grow differently, and hardware utilization can improve initially before memory pressure or sequential depth dominates.

Simple theoretical operation counts do not predict all GPU behavior because kernel efficiency, tensor-core alignment, launch overhead, workspace selection, and memory traffic change with shape. Empirical scaling curves are therefore necessary for the actual GPU and precision

mode.

Sweep a grid of hidden sizes and layer counts under fixed sequence shapes. Record parameters, estimated operations, training and inference throughput, peak memory, utilization, and validation quality, then identify efficient configurations on the accuracy-cost frontier.

Part 2 - Mathematical form

$$\text{cost} \approx O\left(BT \sum_{\ell=1}^{L} H_\ell (H_{\ell-1} + H_\ell)\right).$$

Part 3 - Hints and Helper

Include hidden sizes aligned and unaligned with common accelerator tile dimensions. Warm up every architecture independently.

Part 4 - Python code skeleton

```
import torch

def E06(model, data, config):
    reports = []
    for hidden_size in config.hidden_sizes:
        for layers in config.layer_counts:
            candidate = build_rnn(
                hidden_size=hidden_size, layers=layers, ...
            ).cuda()
            result = benchmark_and_evaluate(candidate, data, ...)
            reports.append({
                "hidden_size": hidden_size,
                "layers": layers,
                "parameters": count_parameters(candidate),
                **result,
            })
    return build_scaling_and_frontier_curves(reports)
```

15.9.8 Exercise 07 - Diagnose an Under-Utilized GPU and Fix DataLoader or Padding Bottlenecks

Part 1 - Why this matters

Low GPU utilization is a symptom, not a diagnosis. The device may be waiting for disk reads, Python collation, host-to-device copies, excessive padding, many tiny kernels, synchronization, or a workload too small to occupy it.

Increasing DataLoader workers blindly can increase memory use and contention without improving throughput. Likewise, increasing batch size may raise displayed utilization while

useful-token efficiency falls because most computation processes padding.

Measure data wait, transfer, compute, and useful-token ratios separately. Tune workers, pinned memory, persistent workers, bucketing, batch size, and prefetching one factor at a time, then confirm that end-to-end examples per second improves without changing model outputs.

Part 2 - Mathematical form

$$U_{\text{useful}} = U_{\text{device}} \times \frac{\text{valid sequence tokens}}{\text{processed tensor tokens}}.$$

Part 3 - Hints and Helper

Start with a synthetic in-memory batch to separate model limits from pipeline limits. Bucket similar sequence lengths to reduce padding waste.

Part 4 - Python code skeleton

```
import torch

def E07(model, data, config):
    baseline = profile_pipeline_regions(model, data, ...)
    loader_variants = build_loader_variants(
        data,
        workers=config.worker_counts,
        pin_memory=[False, True],
        bucketing=[False, True],
        persistent_workers=...,
    )
    reports = benchmark_loaders(model, loader_variants, ...)
    return diagnose_and_select_pipeline_fix(baseline, reports)
```

15.9.9 Exercise 08 - Compare float32 and Mixed-Precision Speed and Document Stability Constraints

Part 1 - Why this matters

Mixed precision can reduce memory and accelerate recurrent matrix operations, allowing larger batches or sequences. The gain is workload-dependent because small recurrent kernels, unsupported operations, data movement, or sequential execution can limit tensor-core benefits.

Speed is irrelevant if small temporal gradients underflow, hidden states overflow, or output drift changes task quality. Dynamic gradient scaling protects backward computation but does not make every forward activation numerically safe.

Benchmark float32 and mixed-precision modes across several shapes. Record throughput, peak memory, scale changes, skipped updates, non-finite values, gradient sparsity, output parity, and final validation performance.

Part 2 - Mathematical form

$$S = \frac{T_{32}}{T_{\text{mixed}}}, \qquad \varepsilon_{\text{rel}} = \frac{\|y_{32} - y_m\|}{\|y_{32}\| + \epsilon}.$$

Part 3 - Hints and Helper

Unscale before clipping and synchronizing measurements. Test long sequences and extreme input scales, not only a short stable batch.

Part 4 - Python code skeleton

```
import torch

def E08(model, data, config):
    reports = {}
    for precision in ["float32", "float16_amp", "bfloat16_amp"]:
        candidate = clone_initial_model(model).cuda()
        reports[precision] = benchmark_precision_mode(
            candidate,
            data,
            precision=precision,
            grad_scaler=...,
            monitor_nonfinite=True,
            ...
        )
    return document_speed_memory_stability_tradeoffs(reports)
```

15.9.10 Exercise 09 - Measure Gradient-Clipping Overhead and Decide Whether It Is Worth It

Part 1 - Why this matters

Gradient clipping requires computing a norm across many parameter gradients and may launch additional reduction and scaling kernels. For a small RNN this overhead can be visible, while for a large backward pass it may be a minor cost relative to the stability it provides.

The correct comparison includes both the cost per step and the cost of failures avoided. A faster unclipped run that diverges, restarts, or needs a smaller learning rate may take more total time to reach a usable model.

Benchmark no clipping, norm measurement only, and actual clipping at several thresholds. Record clipping frequency, microsecond overhead, synchronization effects, throughput, divergence rate, and time-to-target validation quality.

Part 2 - Mathematical form

$$C_{\text{clip}} = T_{\text{norm+scale}} - T_{\text{baseline}}, \qquad V = \frac{\text{failures avoided}}{C_{\text{clip}}}.$$

Part 3 - Hints and Helper

Avoid calling item on the norm every step during the timed region because it forces synchronization. Buffer statistics or sample them less frequently.

Part 4 - Python code skeleton

```
import torch

def E09(model, data, config):
    variants = ["none", "measure_only", *config.clip_norms]
    reports = {}
    for variant in variants:
        candidate = clone_initial_model(model).cuda()
        reports[str(variant)] = benchmark_clipping_policy(
            candidate,
            data,
            policy=variant,
            defer_host_logging=True,
            ...
        )
    return evaluate_clipping_cost_and_value(reports)
```

15.9.11 Exercise 10 - Implement Gradient Accumulation and Compare It with True Large-Batch Training

Part 1 - Why this matters

Gradient accumulation splits a large logical batch into micro-batches so activations fit in limited memory. It reduces peak activation memory but repeats forward and backward launches and retains optimizer updates only after several passes.

Equivalence requires scaling each micro-batch loss correctly, especially when micro-batches contain different numbers of valid sequence tokens. Dropout masks, normalization behavior, clipping order, and optimizer-step frequency can also make accumulated and physical batches differ.

Compare a true large batch with several accumulation factors using identical samples and initial weights. Check gradient and parameter parity in a deterministic case, then measure memory, throughput, update time, and validation behavior in realistic training.

Part 2 - Mathematical form

$$g_{\text{logical}} = \frac{1}{N} \sum_{j=1}^{K} \sum_{i \in \mathcal{B}_j} \nabla_\theta \ell_i.$$

Part 3 - Hints and Helper

Normalize by the total valid-token count across the logical batch. Clip only once after all micro-batch gradients have accumulated.

Part 4 - Python code skeleton

```
import torch

def E10(model, data, config):
    optimizer.zero_grad(set_to_none=True)
    total_valid_tokens = count_logical_batch_tokens(data, ...)
    for micro_batch in split_into_micro_batches(data, factor=...):
        outputs = model(micro_batch.inputs)
        loss_sum = masked_loss(outputs, micro_batch.targets, reduction="sum")
        (loss_sum / total_valid_tokens).backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), ...)
    optimizer.step()
    return compare_with_true_large_batch(model, data, ...)
```

15.9.12 Exercise 11 - Run CUDA Efficiency Tests and Produce Reproducible Benchmark Tables

Part 1 - Why this matters

Benchmark results are useful only when another run can interpret and reproduce them. GPU model, software versions, precision, shapes, warm-up, synchronization, clock state, competing processes, and statistical summary can all change the measured result.

A single minimum latency favors noise and does not describe production behavior. Repeated measurements should report central tendency and tail behavior, while correctness checks ensure an optimization did not silently alter outputs.

Build a benchmark harness that records environment metadata and controlled configurations automatically. Export tidy tables containing median and percentile latency, throughput, peak memory, numerical parity, and run-to-run variability.

Part 2 - Mathematical form

$$p_q = \inf\{t : F_T(t) \geq q\}, \qquad \mathrm{CV} = \frac{\sigma_T}{\mu_T}.$$

Part 3 - Hints and Helper

Warm up every unique shape and synchronize only at timing boundaries. Save raw samples alongside summarized tables.

Part 4 - Python code skeleton

```
import torch

def E11(model, data, config):
```

```
    metadata = capture_benchmark_environment(
        torch_version=torch.__version__, cuda=..., cudnn=..., gpu=...
    )
    raw_results = run_benchmark_matrix(
        model,
        data,
        shapes=config.shapes,
        precisions=config.precisions,
        warmup=...,
        repetitions=...,
    )
    table = summarize_latency_throughput_memory(raw_results, percentiles=...)
    validate_benchmark_parity(raw_results, ...)
    return export_reproducible_benchmark(metadata, table, raw_results, ...)
```

15.9.13 Exercise 12 - Investigate Non-Determinism in cuDNN RNN Kernels and Document When It Appears

Part 1 - Why this matters

GPU execution may produce small run-to-run differences even when model seeds are fixed. Parallel reductions, kernel selection, library algorithms, precision, and asynchronous scheduling can change floating-point operation order, and recurrent reuse may amplify tiny differences over time.

Deterministic settings can reject or replace faster algorithms, but they do not correct uncontrolled data ordering, dropout randomness, worker seeds, or missing checkpoint state. Exact bitwise equality and numerically acceptable reproducibility are also different requirements.

Run repeated forward and training tests across shapes, precisions, and determinism settings. Locate the first divergent time step, measure error growth, compare final metrics, record performance cost, and document which guarantees are achievable on the tested environment.

Part 2 - Mathematical form

$$\delta_t = \|h_t^{(a)} - h_t^{(b)}\|_\infty, \qquad t_{\text{first}} = \min\{t : \delta_t > \tau\}.$$

Part 3 - Hints and Helper

Control Python, NumPy, PyTorch, DataLoader, and CUDA randomness. Save the complete hardware and library environment with the result.

Part 4 - Python code skeleton

```
import torch

def E12(model, data, config):
    reports = []
```

```
    for deterministic in [False, True]:
        configure_determinism(deterministic, seed=...)
        runs = [run_controlled_rnn(model, data, ...) for _ in range(...)]
        reports.append(analyze_run_divergence(
            runs,
            deterministic=deterministic,
            locate_first_time_step=True,
            ...
        ))
    return document_determinism_and_cost(reports)
```

15.9.14 Exercise 13 - Compare Stateful and Stateless RNN Training Efficiency and Explain Hidden-State Reset Costs

Part 1 - Why this matters

Stateless training starts each batch with a fresh hidden state, while stateful training carries memory between consecutive chunks. Carrying state can avoid recomputing context and support streaming, but it also imposes strict ordering and boundary management.

The numerical cost of creating a zero state is usually small; the larger engineering costs are lost shuffling, reduced parallelism, synchronization across streams, and bugs that leak state between unrelated sequences. Failing to detach carried state also grows the autograd graph across batches.

Compare both strategies on contiguous sequence chunks with explicit stream identifiers. Measure throughput, memory, context accuracy, reset and detach overhead, batching flexibility, and failure behavior when stream order is disturbed.

Part 2 - Mathematical form

$$h_0^{(k+1)} = \text{stopgrad}(h_T^{(k)}) \quad \text{for the same stream, otherwise } h_0 = 0.$$

Part 3 - Hints and Helper

Reset state per sequence boundary, not merely per batch number. Detach state between truncated training chunks unless full cross-chunk backpropagation is intended.

Part 4 - Python code skeleton

```
import torch

def E13(model, data, config):
    reports = {}
    for mode in ["stateless", "stateful"]:
        state_store = HiddenStateStore() if mode == "stateful" else None
        reports[mode] = train_streaming_chunks(
            model,
```

```
            data,
            state_store=state_store,
            detach_between_chunks=True,
            reset_on_boundary=True,
            ...
        )
    return compare_state_management_efficiency(reports)
```

15.9.15 Exercise 14 - Implement RNN Checkpointing and Resume and Measure Restart-Time Loss

Part 1 - Why this matters

Long RNN training jobs may be interrupted by hardware faults, scheduler limits, process crashes, or planned maintenance. A checkpoint that stores only model weights can reload predictions but cannot reproduce the original optimizer, scheduler, scaler, data order, or hidden-state trajectory.

Checkpoint frequency trades write overhead and storage against the amount of work lost after failure. Saving synchronously can stall training, while an unsafe asynchronous copy may capture inconsistent state unless ownership and completion are controlled.

Implement complete checkpoint save and resume, force interruptions at known steps, and compare resumed training with an uninterrupted reference. Measure save latency, file size, recovery time, work lost under different intervals, and numerical drift after resumption.

Part 2 - Mathematical form

$$E[T_{\text{lost}}] \approx \frac{\Delta}{2}, \qquad C(\Delta) = \frac{T_{\text{save}}}{\Delta} + \lambda \frac{\Delta}{2}.$$

Part 3 - Hints and Helper

Store epoch, global step, model, optimizer, scheduler, scaler, RNG states, sampler state, configuration, and any stateful hidden-state mapping.

Part 4 - Python code skeleton

```
import torch

def E14(model, data, config):
    checkpoint = {
        "model": model.state_dict(),
        "optimizer": optimizer.state_dict(),
        "scheduler": scheduler.state_dict(),
        "scaler": scaler.state_dict() if scaler else None,
        "rng": capture_all_rng_states(),
        "sampler": capture_sampler_state(data),
        "progress": ...,
```

```
    }
    atomic_save(checkpoint, path=...)
    resumed = restore_complete_training_state(path=..., ...)
    return measure_restart_loss_and_drift(resumed, reference=...)
```

15.9.16 Exercise 15 - Build a Production RNN Inference Benchmark Across Batch Sizes

Part 1 - Why this matters

Production inference is governed by latency objectives and arrival patterns, not only maximum offline throughput. Batch size can improve device efficiency while increasing queueing delay, and recurrent sequence length makes service time highly variable.

Average latency hides slow requests that violate service objectives. A benchmark must separate preprocessing, transfer, model execution, and postprocessing, then report median and tail latency under warm steady-state and realistic length distributions.

Measure batch sizes from one to the throughput limit using both fixed and mixed sequence lengths. Report p50, p95, and p99 latency, sequences and tokens per second, peak memory, cold-start cost, output parity, and the batch policy that meets a defined service target.

Part 2 - Mathematical form

$$T_{\text{request}} = T_{\text{queue}} + T_{\text{pre}} + T_{\text{copy}} + T_{\text{model}} + T_{\text{post}}.$$

Part 3 - Hints and Helper

Use inference mode and warm the model before steady-state measurements. Do not mix cold-start compilation time with normal request latency.

Part 4 - Python code skeleton

```
import torch

def E15(model, data, config):
    model.eval().cuda()
    reports = []
    for batch_size in config.batch_sizes:
        with torch.inference_mode():
            result = benchmark_inference_service(
                model,
                data,
                batch_size=batch_size,
                length_distribution=...,
                include_preprocessing=True,
                percentiles=[50, 95, 99],
                ...
            )
        reports.append(result)
    return select_batch_policy_for_slo(reports, slo_ms=...)
```

15.9.17 Exercise 16 - Export an RNN with TorchScript and Validate Output Parity and Speed

Part 1 - Why this matters

Export changes a Python model into a representation intended for more controlled execution. Recurrent models make this challenging when forward behavior depends on optional hidden states, dynamic sequence lengths, Python-side loops, or multiple return structures.

Tracing records operations from example inputs and can miss data-dependent control flow, while scripting attempts to compile supported program logic. Successful export alone does not prove that hidden-state semantics or variable-shape behavior remain correct.

Export a defined RNN inference wrapper, test it on multiple batch and sequence shapes, compare outputs and final states within tolerances, save and reload it, and benchmark cold and warm execution against the native model.

Part 2 - Mathematical form

$$\varepsilon_y = \max |y_{\text{native}} - y_{\text{export}}|, \qquad \varepsilon_h = \max |h_{\text{native}} - h_{\text{export}}|.$$

Part 3 - Hints and Helper

Make the inference signature explicit and avoid ambiguous optional structures. Include edge shapes not used during tracing in the parity suite.

Part 4 - Python code skeleton

```
import torch

def E16(model, data, config):
    wrapper = ExportableRNNWrapper(model).eval()
    example_inputs = ...
    exported = torch.jit.trace(wrapper, example_inputs, check_trace=True)
    exported = torch.jit.freeze(exported)
    exported.save(...)
    reloaded = torch.jit.load(...)
    parity = validate_export_across_shapes(
        wrapper, reloaded, shapes=config.test_shapes, tolerances=...
    )
    speed = benchmark_native_and_exported(wrapper, reloaded, ...)
    return parity, speed
```

15.9.18 Exercise 17 - Test ONNX Export and Identify Unsupported Operators or Shape Edge Cases

Part 1 - Why this matters

ONNX export can move an RNN toward runtimes outside PyTorch, but differences in supported operators, opset behavior, dynamic axes, packed sequences, and hidden-state signatures may block export or change execution semantics.

A graph that exports with one example shape may still fail for a different batch size or sequence length. Runtime parity must cover outputs and recurrent states, while edge cases such as length one, batch one, and variable lengths expose assumptions hidden by the example input.

Export a simplified inference wrapper, inspect warnings and graph inputs, run it in an available ONNX runtime, and build a compatibility matrix. For each failure, isolate the smallest unsupported construct and document a safe alternative or an explicit deployment limitation.

Part 2 - Mathematical form

$$f_\theta^{\text{PyTorch}}(x, h_0) \approx f_\theta^{\text{ONNX}}(x, h_0) \quad \forall (B, T) \in \mathcal{S}.$$

Part 3 - Hints and Helper

Name dynamic batch and sequence axes deliberately. Begin with padded tensors before attempting packed-sequence export.

Part 4 - Python code skeleton

```
import torch

def E17(model, data, config):
    wrapper = ONNXRNNWrapper(model).eval()
    torch.onnx.export(
        wrapper,
        args=...,
        f=...,
        input_names=[...],
        output_names=[...],
        dynamic_axes=...,
        opset_version=...,
    )
    cases = build_shape_edge_cases(batch_sizes=..., sequence_lengths=...)
    return validate_onnx_runtime_and_document_failures(..., cases=cases)
```

15.9.19 Exercise 18 - Stress Test Long-Sequence Inference and Detect Memory Leaks or Latency Spikes

Part 1 - Why this matters

An inference service can pass short benchmarks yet degrade during hours of long-sequence traffic. Retained output tensors, unbounded hidden-state caches, logging references, allocator fragmentation, or occasional extreme inputs can cause gradual memory growth and tail-latency

spikes.

Reserved CUDA memory may remain high without representing a leak, so allocated memory, reserved memory, process memory, cache size, and object ownership must be interpreted separately. Synchronization is also required to measure completed GPU latency.

Run thousands of requests with controlled and heavy-tailed sequence lengths, include warm-up and periodic snapshots, and force state creation and eviction. Detect monotonic allocated-memory growth, correlate latency spikes with shapes and allocations, and preserve the first suspicious request for diagnosis.

Part 2 - Mathematical form

$$\Delta M_k = M_{k+w} - M_k, \qquad \text{suspect leak if } \Delta M_k > \tau \text{ persistently after warm-up.}$$

Part 3 - Hints and Helper

Do not call empty-cache as a leak fix during the main test. Ensure returned GPU tensors are released or moved intentionally before the next request.

Part 4 - Python code skeleton

```
import torch

def E18(model, data, config):
    monitor = LongRunInferenceMonitor(sample_every=..., warmup_requests=...)
    with torch.inference_mode():
        for request_id, request in enumerate(generate_request_stream(...)):
            output, state = serve_request(model, request, cache=...)
            release_or_store_state(state, policy=...)
            monitor.record(
                request_id,
                request_shape=request.shape,
                allocated=torch.cuda.memory_allocated(),
                reserved=torch.cuda.memory_reserved(),
                latency=...,
            )
    return monitor.detect_leaks_and_spikes(...)
```

15.9.20 Exercise 19 - Produce a Complete RNN Engineering Performance Report with Scaling Curves and Best Practices

Part 1 - Why this matters

An engineering report must convert isolated benchmarks into decisions that another developer can reproduce. The fastest configuration in one microbenchmark may fail the memory budget, numerical tolerance, service latency target, or long-sequence stability requirement.

Results need common definitions, controlled environments, raw evidence, uncertainty, and correctness gates. Scaling curves should show where each resource becomes limiting, while failure records prevent future teams from repeating optimizations that looked fast but were unsafe.

Synthesize architecture, shape, precision, data-pipeline, clipping, accumulation, determinism, checkpoint, export, and inference experiments. Recommend separate training and deployment configurations, define their valid operating ranges, and state when GRU, LSTM, or a non-recurrent architecture is the better engineering choice.

Part 2 - Mathematical form

$$\mathcal{C}^* = \arg\max_{\mathcal{C}} \text{throughput}(\mathcal{C}) \quad \text{s.t.} \quad M \le M_{\max},\ L_{p99} \le L_{\max},\ \varepsilon \le \tau.$$

Part 3 - Hints and Helper

Every recommendation should link to a benchmark row and correctness result. Include hardware and software metadata plus raw samples with the final tables.

Part 4 - Python code skeleton

```
import torch

def E19(model, data, config):
    evidence = load_all_rnn07_benchmarks(...)
    report = EngineeringPerformanceReport()
    report.add_environment_and_methodology(evidence, ...)
    report.add_scaling_curves(evidence, axes=[...])
    report.add_training_frontier(evidence, constraints=...)
    report.add_inference_slo_frontier(evidence, constraints=...)
    report.add_correctness_and_stability_gates(evidence, ...)
    report.add_failure_catalog(evidence, ...)
    report.add_reproducible_recommendations(evidence, ...)
    report.validate_claim_traceability()
    return report.export(...)
```

15.10 RNN08 - Transition to Gated RNNs (Why LSTM Exists)

Models and Training

15.10.1 Exercise 00 - Demonstrate Long-Term Dependency Failure in Vanilla RNN with a Controlled Memory Task

Part 1 - Why this matters

A vanilla RNN may perform well when the useful signal occurred recently yet fail when the same signal must survive many irrelevant time steps. A controlled memory task isolates this

limitation from language ambiguity, noisy labels, and insufficient model capacity.

The delay between the informative token and the required output must be varied independently of total dataset size. If accuracy collapses as delay grows while short-delay cases remain solved, the experiment exposes a temporal credit-assignment failure rather than a general inability to learn.

Create a copy, remember-bit, or delayed-sign task with known dependency distance. Train identical RNNs across increasing delays and record accuracy, hidden-state probes, gradient magnitude reaching the signal position, and the first distance at which learning becomes unreliable.

Part 2 - Mathematical form

$$y_T = x_{T-d}, \qquad d \in \mathcal{D}, \qquad A(d) = \Pr(\widehat{y}_T = y_T).$$

Part 3 - Hints and Helper

Keep distractor statistics identical across delays. Compare against a no-memory baseline so accidental class imbalance cannot look like memory.

Part 4 - Python code skeleton

```
import torch

def E00(model, data, config):
    reports = {}
    for delay in config.delays:
        task = build_controlled_memory_task(delay=delay, ...)
        candidate = build_vanilla_rnn(...)
        reports[delay] = train_and_probe_memory(
            candidate, task, signal_position=..., target_position=...
        )
    return find_memory_failure_distance(reports, baseline=...)
```

15.10.2 Exercise 01 - Explain Vanishing Gradients as Repeated Jacobian Multiplication

Part 1 - Why this matters

Backpropagation through a recurrent network sends information from a late loss through the same transition many times. The gradient is therefore governed by a product of Jacobian matrices rather than by one local derivative.

If typical singular values of these Jacobians are below one, the product contracts and early time steps receive almost no learning signal. Values above one can produce explosion, while nonlinear saturation makes contraction even stronger by shrinking activation derivatives.

Compute and visualize gradient norms by temporal distance for controlled recurrent matrices with different spectral scales. Relate empirical decay or growth to Jacobian products, activation saturation, and the limits of weight initialization or clipping as complete solutions.

Part 2 - Mathematical form

$$\frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial h_T} \prod_{k=t+1}^{T} J_k, \qquad J_k = \operatorname{diag}(\phi'(a_k)) W_{hh}.$$

Part 3 - Hints and Helper

Retain gradients for hidden states in a tiny manually unrolled RNN. Use logarithmic scale when plotting norms across time.

Part 4 - Python code skeleton

```
import torch

def E01(model, data, config):
    reports = {}
    for spectral_scale in config.scales:
        candidate = build_controlled_recurrent_matrix(
            scale=spectral_scale, ...
        )
        hidden_states, loss = manual_unroll_with_retained_states(
            candidate, data
        )
        loss.backward()
        reports[spectral_scale] = {
            "gradient_by_time": collect_hidden_gradients(hidden_states),
            "jacobian_products": estimate_jacobian_products(...),
            "saturation": measure_activation_saturation(hidden_states),
        }
    return explain_temporal_gradient_behavior(reports)
```

15.10.3 Exercise 02 - Implement a Simple GRU Cell Manually and Compare It with Vanilla RNN Behavior

Part 1 - Why this matters

A GRU introduces gates that decide how much of the previous state should be retained and how much new candidate information should replace it. This creates an additive interpolation path instead of forcing every time step to overwrite memory through one nonlinear transformation.

Manual implementation exposes the distinction between the update gate, reset gate, and candidate state. A sign reversal or swapped interpolation convention can produce valid tensor

shapes while changing the meaning of memory retention.

Implement a GRU cell from tensor operations, copy parameters into a reference PyTorch cell, and verify step-by-step parity. Then compare hidden-state retention and gradients against a vanilla RNN on the controlled memory task.

Part 2 - Mathematical form

$$z_t = \sigma(W_z x_t + U_z h_{t-1}),\ r_t = \sigma(W_r x_t + U_r h_{t-1}),\ h_t = (1 - z_t) \odot n_t + z_t \odot h_{t-1}.$$

Part 3 - Hints and Helper

Document the chosen gate convention because some equations reverse the roles of z and one minus z. Validate a single step before unrolling a sequence.

Part 4 - Python code skeleton

```
import torch
from torch import nn

class ManualGRUCell(nn.Module):
    def __init__(self, input_size, hidden_size):
        super().__init__()
        self.input_projection = nn.Linear(input_size, 3 * hidden_size)
        self.hidden_projection = nn.Linear(hidden_size, 3 * hidden_size)

    def forward(self, x_t, h_prev):
        x_r, x_z, x_n = self.input_projection(x_t).chunk(3, dim=-1)
        h_r, h_z, h_n = self.hidden_projection(h_prev).chunk(3, dim=-1)
        reset = torch.sigmoid(...)
        update = torch.sigmoid(...)
        candidate = torch.tanh(...)
        h_t = ...
        return h_t

def E02(model, data, config):
    return verify_and_compare_manual_gru(... )
```

15.10.4 Exercise 03 - Compare Vanilla RNN, GRU, and LSTM on the Same Long-Dependency Task

Part 1 - Why this matters

A meaningful architecture comparison holds the task, preprocessing, split, optimization budget, and evaluation constant. Otherwise, apparent gains from gates may actually come from different parameter counts, tuning effort, or early-stopping opportunities.

GRU and LSTM add computational cost to create more controllable memory paths. Their value should be measured as the dependency distance increases, because short tasks may not require gating and may favor the simpler RNN.

Train all three models across several delays and seeds using both equal hidden size and approximately equal parameter count. Compare convergence rate, success probability, final accuracy, temporal gradients, runtime, and memory.

Part 2 - Mathematical form

$$\Delta A_m(d) = A_m(d) - A_{\mathrm{RNN}}(d), \qquad C_m = (T_m, M_m, P_m).$$

Part 3 - Hints and Helper

Tune learning rate fairly for each architecture, but keep the search budget equal. Report variability across seeds, not only the best run.

Part 4 - Python code skeleton

```
import torch

def E03(model, data, config):
    builders = {"rnn": build_rnn, "gru": build_gru, "lstm": build_lstm}
    results = run_architecture_delay_matrix(
        builders,
        delays=config.delays,
        seeds=config.seeds,
        matching=["hidden_size", "parameter_count"],
        budget=...,
    )
    return summarize_dependency_cost_tradeoffs(results)
```

15.10.5 Exercise 04 - Analyze Hidden-State Updates and Explain Why Gates Preserve Useful Gradients

Part 1 - Why this matters

Gated recurrence creates paths where state can be copied forward through addition and elementwise multiplication. When a retention gate stays near one, the derivative along that path can remain near one instead of repeatedly passing through a saturated nonlinear candidate transformation.

This does not guarantee perfect memory: gates can saturate incorrectly, candidate paths still contain nonlinearities, and cross-coupled dependencies remain. The advantage is learned control over retention, erasure, and exposure rather than automatic immunity to vanishing gradients.

Inspect gate values and local state Jacobians on a trained memory task. Identify which units retain the signal, when they update, and how gradient flow differs between the additive cell-state path and the vanilla recurrent path.

Part 2 - Mathematical form

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \qquad \frac{\partial c_t}{\partial c_{t-1}} \supseteq f_t.$$

Part 3 - Hints and Helper

Separate the direct cell-state path from indirect dependencies through gate computations. Plot gates alongside known task events.

Part 4 - Python code skeleton

```
import torch

def E04(model, data, config):
    traces = collect_gate_and_state_traces(model, data, retain_grad=True, ...)
    analysis = {
        "retention_paths": estimate_direct_state_derivatives(traces, ...),
        "gate_events": align_gates_with_task_events(traces, ...),
        "temporal_gradients": collect_gradients_by_distance(traces, ...),
        "vanilla_reference": analyze_vanilla_transition(...),
    }
    return explain_gate_gradient_preservation(analysis)
```

15.10.6 Exercise 05 - Implement a Minimal LSTM Cell and Verify Gate Shapes and Ranges Step by Step

Part 1 - Why this matters

An LSTM separates an exposed hidden state from an internal cell state and controls information with input, forget, and output gates. This structure gives the model distinct mechanisms for writing, retaining, forgetting, and revealing memory.

Most manual LSTM errors come from gate ordering, concatenation dimensions, cell-hidden confusion, or applying the wrong activation. These bugs often preserve output shape, making intermediate invariant checks essential.

Implement one LSTM step manually, compare it with a reference cell using copied parameters, and verify every gate range and tensor shape. Extend the verified cell to a short unroll and compare outputs, states, and gradients.

Part 2 - Mathematical form

$$i_t = \sigma(a_i),\ f_t = \sigma(a_f),\ g_t = \tanh(a_g),\ o_t = \sigma(a_o),\ c_t = f_t \odot c_{t-1} + i_t \odot g_t,\ h_t = o_t \odot \tanh(c_t).$$

Part 3 - Hints and Helper

PyTorch uses a documented gate order; match it exactly when copying weights. Check batch and hidden dimensions for both h and c.

Part 4 - Python code skeleton

```
import torch
from torch import nn

class ManualLSTMCell(nn.Module):
    def __init__(self, input_size, hidden_size):
        super().__init__()
        self.x_proj = nn.Linear(input_size, 4 * hidden_size)
        self.h_proj = nn.Linear(hidden_size, 4 * hidden_size)

    def forward(self, x_t, state):
        h_prev, c_prev = state
        i_raw, f_raw, g_raw, o_raw = (...).chunk(4, dim=-1)
        input_gate, forget_gate = torch.sigmoid(i_raw), torch.sigmoid(f_raw)
        candidate, output_gate = torch.tanh(g_raw), torch.sigmoid(o_raw)
        c_t = ...
        h_t = ...
        return h_t, c_t

def E05(model, data, config):
    return verify_lstm_gates_states_and_gradients(... )
```

15.10.7 Exercise 06 - Train GRU and LSTM and Compare Stability, Convergence Speed, and Final Accuracy

Part 1 - Why this matters

GRU and LSTM solve related memory problems with different state structures and gate counts. GRU may converge faster or use less memory, while LSTM offers a separate cell state that can provide finer control on some long dependencies.

One final accuracy number cannot distinguish fast learning, stable learning, and lucky learning. Comparisons must include loss variability, gradient behavior, failure rate across seeds, best epoch, runtime, and the amount of tuning required.

Run repeated controlled training for both architectures across short and long tasks. Measure time-to-threshold, final validation accuracy, stability events, parameter count, peak memory, and whether either model's advantage persists after matching compute budget.

Part 2 - Mathematical form

$$t_\tau = \min\{t : M_{\text{val}}(t) \geq \tau\}, \qquad F = \Pr(\text{run fails}).$$

Part 3 - Hints and Helper

Use identical splits and seed lists. Compare equal epochs, equal wall-clock time, and approximately equal parameters as separate views.

Part 4 - Python code skeleton

```
import torch

def E06(model, data, config):
    results = {}
    for architecture in ["gru", "lstm"]:
        results[architecture] = run_repeated_training(
            architecture,
            data,
            seeds=config.seeds,
            budgets=["epochs", "wall_clock", "parameters"],
            monitor_gradients=True,
            ...
        )
    return compare_convergence_stability_accuracy(results)
```

15.10.8 Exercise 07 - Debug Gate Misconfiguration Bugs Causing No Learning or Constant Outputs

Part 1 - Why this matters

A gated cell can produce stable-looking but useless behavior when a gate is permanently open, closed, swapped, detached, or driven into saturation. Constant outputs may therefore result from gate logic rather than the optimizer or dataset.

Checking only the final state hides the source. Gate distributions, preactivation ranges, parameter gradients, candidate variance, and state changes reveal whether the model writes, forgets, exposes, or updates information.

Create broken GRU and LSTM variants with known faults and build diagnostics that identify each one. Include wrong activations, gate-order swaps, extreme biases, missing recurrent terms, detached states, and incorrect broadcasting.

Part 2 - Mathematical form

$$\Delta h_t = \|h_t - h_{t-1}\|, qquad S_g = \Pr(g_t < \epsilon \ \vee \ g_t > 1 - \epsilon).$$

Part 3 - Hints and Helper

Test a single cell step with fixed numbers before training. Register hooks for gate statistics and named parameter gradients.

Part 4 - Python code skeleton

```
import torch

def E07(model, data, config):
    broken_variants = create_gate_bug_variants(
        bugs=["swapped_order", "extreme_bias", "wrong_activation",
              "detached_state", "missing_recurrence", "broadcast_error"],
        ...
    )
    reports = {}
    for name, candidate in broken_variants.items():
        reports[name] = run_gate_diagnostics(
            candidate,
            data,
            track_preactivations=True,
            track_state_change=True,
        )
    return match_symptoms_to_gate_faults(reports)
```

15.10.9 Exercise 08 - Run GRU and LSTM Experiments on CUDA and Measure Memory and Runtime Tradeoffs

Part 1 - Why this matters

Gated models perform several recurrent projections per time step, increasing parameter, activation, and kernel costs. CUDA may offset this through fused implementations, but the outcome depends on sequence shape, precision, training mode, and hardware.

CUDA work is asynchronous and allocator caching separates allocated from reserved memory. Fair measurement requires warm-up, synchronized timing boundaries, reset peak statistics, and identical input shapes.

Benchmark GRU and LSTM forward, backward, and optimizer regions across batch sizes and sequence lengths. Report latency percentiles, useful-token throughput, peak allocated memory, parameter memory, and quality so cost is connected to model benefit.

Part 2 - Mathematical form

$$\text{throughput} = \frac{BT}{T_{\text{step}}}, \qquad \Delta M = M_{\text{LSTM}} - M_{\text{GRU}}.$$

Part 3 - Hints and Helper

Use fused library modules for the main benchmark and manual cells only as an educational reference. Synchronize outside timed regions.

Part 4 - Python code skeleton

```
import torch

def E08(model, data, config):
    reports = []
    for architecture in ["gru", "lstm"]:
        for shape in config.shapes:
            candidate = build_gated_model(architecture, ...).cuda()
            torch.cuda.reset_peak_memory_stats()
            batch = make_batch(shape).cuda()
            result = benchmark_cuda_training_regions(
                candidate, batch, warmup=..., repetitions=...
            )
            reports.append({
                "architecture": architecture,
                "shape": shape,
                **result,
            })
    return summarize_cuda_tradeoffs(reports)
```

15.10.10 Exercise 09 - Explain Why LSTM and GRU Are Engineering Solutions to Vanilla RNN Limits

Part 1 - Why this matters

LSTM and GRU did not appear because vanilla RNNs lack recurrent memory in principle. They appeared because learning and preserving useful memory through long nonlinear recurrence is unreliable in practice.

Their gates create trainable retention and update paths, improving credit assignment at a computational cost. They mitigate long-dependency failure but do not remove finite capacity, exposure bias, sequential execution, or every gradient problem.

Produce an evidence-based transition report using the chapter's memory curves, gradients, gate traces, convergence results, and CUDA measurements. Explain when gating solves the observed problem and when another architecture is justified.

Part 2 - Mathematical form

$$\text{gating benefit} = \text{memory reliability gain} - \text{compute and complexity cost}.$$

Part 3 - Hints and Helper

Separate theoretical mechanism, empirical evidence, and engineering recommendation. Do not claim that gated models guarantee unlimited memory.

Part 4 - Python code skeleton

```
import torch

def E09(model, data, config):
    evidence = collect_transition_evidence(...)
    report = TransitionReport()
    report.add_vanilla_failure_curve(evidence, ...)
    report.add_jacobian_and_gradient_analysis(evidence, ...)
    report.add_gate_mechanism_evidence(evidence, ...)
    report.add_accuracy_runtime_memory_tradeoffs(evidence, ...)
    report.add_limitations_and_decision_rules(evidence, ...)
    report.validate_claims_against_experiments()
    return report.export(...)
```

15.10.11 Exercise 10 - Ablate the Forget Gate and Observe Memory Collapse

Part 1 - Why this matters

The forget gate controls how much previous cell content remains available. Removing its learned behavior tests whether LSTM success comes from selective retention rather than merely from having more parameters.

Different ablations answer different questions: forcing the gate to zero erases memory, forcing it to one prevents adaptive forgetting, and replacing it with a constant tests whether fixed timescales are sufficient. These interventions must preserve the rest of the trained architecture.

Train or evaluate controlled forget-gate variants on tasks requiring both retention and reset. Compare memory accuracy, cell-state persistence, interference after reset events, temporal gradients, and convergence.

Part 2 - Mathematical form

$$c_t = \alpha c_{t-1} + i_t \odot g_t, \qquad \alpha \in \{0, \alpha_0, 1, f_t\}.$$

Part 3 - Hints and Helper

Use a task that contains explicit remember and forget events. Match parameter counts and state initialization across variants.

Part 4 - Python code skeleton

```
import torch

def E10(model, data, config):
    variants = {
        "learned": LearnedForgetGate(...),
        "always_zero": ConstantForgetGate(0.0),
        "fixed": ConstantForgetGate(...),
        "always_one": ConstantForgetGate(1.0),
```

```
    }
    results = run_forget_gate_ablation(
        variants, data, track_cell_state=True, track_resets=True, ...
    )
    return quantify_memory_and_interference(results)
```

15.10.12 Exercise 11 - Find the Sequence Length Where LSTM Clearly Outperforms Vanilla RNN

Part 1 - Why this matters

Gating has a cost, so an LSTM may not dominate on short dependencies where a vanilla RNN already learns reliably. The meaningful transition point is where the LSTM's accuracy or success rate exceeds the RNN by more than experimental variability.

Total sequence length and required dependency distance should be controlled separately. Long sequences with only local dependencies test computation, while delayed-memory tasks test whether information must survive.

Sweep both variables across several seeds and construct performance-difference curves with uncertainty. Identify the earliest sustained advantage under equal parameter and equal compute comparisons.

Part 2 - Mathematical form

$$T^* = \min\{T : \mathbb{E}[A_L(T) - A_R(T)] - \kappa\,\mathrm{SE} > \delta\}.$$

Part 3 - Hints and Helper

Define 'clearly outperforms' before running the experiment. Require the advantage to persist across multiple neighboring lengths.

Part 4 - Python code skeleton

```
import torch

def E11(model, data, config):
    results = run_length_dependency_grid(
        models={"rnn": build_rnn, "lstm": build_lstm},
        sequence_lengths=config.lengths,
        dependency_distances=config.distances,
        seeds=config.seeds,
        matching=...,
    )
    return estimate_sustained_lstm_advantage(
        results, effect_threshold=..., confidence=...
    )
```

15.10.13 Exercise 12 - Compare Gated and Vanilla Models Under Distribution Shift

Part 1 - Why this matters

Better memory on the training distribution does not guarantee better behavior when sequence statistics change. A gate may learn timing patterns, reset frequencies, or value scales that fail when deployment sequences are longer or events arrive differently.

Distribution shifts should target temporal structure as well as input magnitude. Useful cases include longer delays, changed event frequency, unseen reset patterns, altered noise, missing segments, and changed sequence-length distributions.

Train RNN, GRU, and LSTM on the same source distribution, then evaluate clean and shifted suites without retraining. Measure relative degradation, confidence, gate saturation, memory retention, and failure modes by shift severity.

Part 2 - Mathematical form

$$D_m(s) = \frac{M_m(s) - M_m(0)}{|M_m(0)| + \epsilon}.$$

Part 3 - Hints and Helper

Fit preprocessing on training data only. Report absolute performance so a small relative drop from a weak baseline is not misread as robustness.

Part 4 - Python code skeleton

```
import torch

def E12(model, data, config):
    models = train_matched_models(["rnn", "gru", "lstm"], data.source, ...)
    shift_suite = build_temporal_shift_suite(
        longer_delay=...,
        changed_frequency=...,
        missing_segments=...,
        noise=...,
    )
    reports = evaluate_models_under_shifts(
        models, data.test, shift_suite, inspect_gates=True, ...
    )
    return compare_robustness_and_failure_modes(reports)
```

15.10.14 Exercise 13 - Implement Truncated BPTT for GRU and LSTM and Compare Memory Savings with Vanilla RNN

Part 1 - Why this matters

Truncated backpropagation limits the number of time steps retained in the computation graph while carrying detached state between chunks. For LSTM, both hidden and cell states must be carried and detached correctly; GRU and RNN carry one state tensor.

Short truncation reduces memory but prevents gradients from assigning credit beyond the chunk boundary. Gating can preserve values in forward state, yet it cannot restore a gradient path that truncation deliberately removed.

Implement a shared truncated-training harness for all three architectures. Sweep truncation length and compare peak memory, throughput, long-dependency accuracy, state continuity, and the boundary where savings become unacceptable learning loss.

Part 2 - Mathematical form

$$s_{k+1} = \text{stopgrad}(F_\theta(x_{k:k+K}, s_k)), \qquad M = O(BKH).$$

Part 3 - Hints and Helper

Detach every component of an LSTM state tuple. Normalize loss consistently across chunks and reset states only at true sequence boundaries.

Part 4 - Python code skeleton

```
import torch

def E13(model, data, config):
    reports = []
    for architecture in ["rnn", "gru", "lstm"]:
        for truncation in config.truncation_lengths:
            candidate = build_model(architecture, ...)
            result = train_with_truncated_bptt(
                candidate,
                data,
                chunk_length=truncation,
                detach_state=lambda state: detach_recurrent_state(state),
                ...
            )
            reports.append({"architecture": architecture,
                            "truncation": truncation, **result})
    return compare_memory_savings_and_dependency_loss(reports)
```

15.10.15 Exercise 14 - Profile RNN and LSTM GPU Utilization and Explain Kernel-Level Differences

Part 1 - Why this matters

An LSTM performs several gate projections per step, but optimized libraries may fuse them into larger kernels. A vanilla RNN has fewer operations yet may not be faster proportionally

because small sequential kernels can underutilize the GPU.

GPU utilization alone cannot distinguish useful arithmetic from memory movement, launch gaps, or padding waste. Kernel timelines, operation shapes, tensor-core eligibility, occupancy indicators, and CPU gaps are needed to explain observed throughput.

Profile matched RNN and LSTM workloads after warm-up. Identify fused recurrent kernels, launches per step, matrix sizes, copy events, and idle regions, then relate kernel behavior to sequence length, hidden size, batch size, and precision.

Part 2 - Mathematical form

$$T_{\text{total}} = \sum_{k} (T_{\text{launch},k} + T_{\text{kernel},k}) + T_{\text{gaps}}.$$

Part 3 - Hints and Helper

Profile a small representative window rather than the entire training run. Validate that both models use equivalent data and synchronization boundaries.

Part 4 - Python code skeleton

```
import torch
from torch.profiler import profile, ProfilerActivity

def E14(model, data, config):
    traces = {}
    for architecture in ["rnn", "lstm"]:
        candidate = build_model(architecture, ...).cuda()
        warm_up(candidate, data, ...)
        with profile(
            activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
            record_shapes=True,
            profile_memory=True,
        ) as profiler:
            run_profile_window(candidate, data, ...)
        traces[architecture] = extract_kernel_and_idle_analysis(profiler)
    return explain_gpu_utilization_differences(traces)
```

15.10.16 Exercise 15 - Build a Checklist for Choosing RNN, GRU, or LSTM in Real Projects

Part 1 - Why this matters

Architecture choice should follow the dependency structure, data volume, latency target, memory budget, deployment runtime, and observed failure mode. Choosing LSTM automatically because it is more expressive can waste resources on tasks that need only local memory.

A useful checklist begins with a measurable baseline and escalates complexity only when evidence supports it. It also distinguishes training constraints from inference constraints and includes tests for long dependencies, stateful streaming, shift, and export compatibility.

Turn the chapter results into a decision process with explicit thresholds and fallback choices. Require every recommendation to connect task evidence with accuracy, stability, runtime, memory, and maintenance cost.

Part 2 - Mathematical form

$$m^* = \arg\min_m C(m) \quad \text{s.t.} \quad Q(m) \geq Q_{\min},\ L(m) \leq L_{\max},\ M(m) \leq M_{\max}.$$

Part 3 - Hints and Helper

Include a simple non-recurrent baseline. State when none of the three models is appropriate and attention or convolution should be tested.

Part 4 - Python code skeleton

```
import torch

def E15(model, data, config):
    evidence = collect_task_constraints_and_benchmarks(...)
    checklist = ModelChoiceChecklist()
    checklist.add_dependency_tests(evidence, ...)
    checklist.add_quality_thresholds(evidence, ...)
    checklist.add_training_and_inference_budgets(evidence, ...)
    checklist.add_streaming_and_state_requirements(evidence, ...)
    checklist.add_export_and_runtime_constraints(evidence, ...)
    checklist.add_escalation_rules(evidence, ...)
    return checklist.validate_and_export(...)
```

15.10.17 Exercise 16 - Export GRU and LSTM and Compare Deployment Complexity with Vanilla RNN

Part 1 - Why this matters

Deployment complexity depends on more than whether export succeeds. GRU and LSTM expose different state structures, operator support, dynamic-shape behavior, runtime kernels, quantization options, and memory requirements.

An exported graph must preserve outputs and recurrent states across batch and sequence shapes. LSTM's paired hidden and cell state makes interfaces and stateful serving more complex than a single-state RNN or GRU.

Export standardized wrappers for all three architectures to the selected formats and runtimes. Build a compatibility matrix covering parity, file size, operator support, dynamic shapes, latency, state handling, and failure workarounds.

Part 2 - Mathematical form

$$\varepsilon_m = \max_{x \in \mathcal{S}} \| f_m^{\text{native}}(x) - f_m^{\text{export}}(x) \|_\infty .$$

Part 3 - Hints and Helper

Use explicit state inputs and outputs. Test batch one, sequence length one, and unseen dynamic shapes after reloading the artifact.

Part 4 - Python code skeleton

```
import torch

def E16(model, data, config):
    reports = {}
    for architecture in ["rnn", "gru", "lstm"]:
        wrapper = build_export_wrapper(architecture, explicit_state=True, ...)
        artifacts = export_to_supported_formats(
            wrapper, example_inputs=..., ...
        )
        reports[architecture] = validate_deployment_matrix(
            wrapper,
            artifacts,
            test_shapes=config.dynamic_shapes,
            check_state_parity=True,
            benchmark=True,
        )
    return compare_deployment_complexity(reports)
```

15.10.18 Exercise 17 - Stress Test Gated Models with Long Sequences and Large Hidden Sizes

Part 1 - Why this matters

Gates improve memory dynamics but do not remove sequential computation, quadratic hidden projections, activation storage, or finite precision. Long sequences and large hidden states eventually expose runtime, memory, optimization, and numerical limits.

A safe stress test increases scale gradually, maintains memory headroom, and distinguishes out-of-memory failure from latency collapse, gradient failure, or loss of task quality. Maximum allocatable shape is not necessarily a practical configuration.

Sweep sequence length, hidden size, layers, batch size, and precision for GRU and LSTM. Record throughput, peak memory, gate saturation, gradients, non-finite events, validation quality, and the first failure boundary for each resource.

Part 2 - Mathematical form

$$C \approx O(BTLH^2), \qquad M_{\text{train}} \approx O(BTLH) + M_{\text{optimizer}}.$$

Part 3 - Hints and Helper

Vary one primary scale dimension at a time before testing interactions. Stop below full device exhaustion and preserve the failing configuration.

Part 4 - Python code skeleton

```
import torch

def E17(model, data, config):
    reports = []
    for architecture, shape in controlled_stress_grid(
        ["gru", "lstm"], config.shapes
    ):
        try:
            result = run_guarded_stress_case(
                architecture,
                shape,
                memory_headroom=...,
                monitor_numerics=True,
            )
        except RuntimeError as error:
            result = classify_stress_failure(error, shape=...)
        reports.append({
            "architecture": architecture,
            "shape": shape,
            **result,
        })
    return map_practical_operating_limits(reports)
```

15.10.19 Exercise 18 - Create an Ablation Report Quantifying What Gates Improve and Why

Part 1 - Why this matters

A full gated model can outperform a vanilla RNN without revealing which gate or pathway provides the benefit. Removing, fixing, or simplifying components makes the causal contribution of retention, reset, write, and exposure mechanisms measurable.

Ablations must control parameter count, initialization, training budget, and seed variability. Some gates matter only on tasks containing specific events, so conclusions should be conditioned on dependency length and the need to erase obsolete information.

Build GRU and LSTM ablations that fix or remove one mechanism at a time, plus matched vanilla baselines. Quantify changes in accuracy, gradient reach, memory persistence, interference, convergence, runtime, and parameter efficiency.

Part 2 - Mathematical form

$$\Delta_g = \mathbb{E}[M_{\text{full}} - M_{\text{ablated}(g)}], \qquad I_{g_1,g_2} = \Delta_{g_1,g_2} - \Delta_{g_1} - \Delta_{g_2}.$$

Part 3 - Hints and Helper

Run multiple seeds and include confidence intervals. Pair each gate ablation with a task event that should require its function.

Part 4 - Python code skeleton

```
import torch

def E18(model, data, config):
    ablations = build_gate_ablation_matrix(
        gru=["fixed_update", "fixed_reset", "no_reset"],
        lstm=["fixed_forget", "fixed_input", "fixed_output", "coupled_gates"],
        matched_baselines=True,
    )
    results = run_repeated_ablation_study(
        ablations, data, seeds=config.seeds, controlled_budget=...
    )
    return quantify_gate_effects_and_interactions(results, ... )
```

15.10.20 Exercise 19 - Produce the Final RNN Track Report with Concepts, Experiments, Failures, and Next Steps

Part 1 - Why this matters

The final report should show how the entire RNN track developed from sequence representation and hidden-state recurrence to BPTT, data preparation, applications, stabilization, performance engineering, and gated memory. It must preserve failures because they define the boundaries of each method.

A strong synthesis separates mathematical claims, experimental evidence, engineering practice, and unresolved limitations. It should explain not only what worked, but under which sequence lengths, distributions, hardware conditions, and evaluation protocols it remained trustworthy.

Assemble a traceable report linking every major conclusion to an exercise result. Conclude with readiness criteria for the next LSTM-focused book and a list of questions about cell dynamics, advanced gating, bidirectionality, sequence generation, deployment, and alternatives that remain open.

Part 2 - Mathematical form

$$\text{concept} \rightarrow \text{experiment} \rightarrow \text{evidence} \rightarrow \text{failure boundary} \rightarrow \text{next hypothesis.}$$

Part 3 - Hints and Helper

Include reproducible configurations, raw tables, and a failure catalog. Distinguish conclusions proven in this track from claims requiring future study.

Part 4 - Python code skeleton

```
import torch

def E19(model, data, config):
    evidence = load_complete_rnn_track_evidence(...)
    report = FinalTrackReport()
    report.add_concept_map(evidence, ...)
    report.add_experiment_index(evidence, ...)
    report.add_mathematical_and_empirical_findings(evidence, ...)
    report.add_failure_boundaries(evidence, ...)
    report.add_engineering_best_practices(evidence, ...)
    report.add_lstm_book_readiness_checklist(evidence, ...)
    report.add_open_questions_and_next_steps(evidence, ...)
    report.validate_traceability()
    return report.export(...)
```

Book 16

LSTM (Long Short-Term Memory)

16.1 Introduction

16.1.1 What will the book teach you

The previous book introduced recurrent neural networks as models that process information through time. Unlike a feed-forward network, an RNN does not treat every input as an isolated observation. It carries a hidden state from one time step to the next, allowing earlier information to influence later predictions. This simple idea created a new way to work with language, time series, signals, and every problem in which order matters. It also revealed a difficult truth: having a memory state does not mean that a model can learn to preserve useful information for a long time. A vanilla RNN may possess a path from the beginning of a sequence to its end, yet the learning signal traveling through that path can become extremely small, extremely large, or too unstable to guide the parameters correctly.

This book begins at that exact limitation. Long Short-Term Memory, or LSTM, was not created merely as a larger version of an RNN. It was designed as a different memory mechanism. Instead of forcing all past information into one hidden state that is repeatedly overwritten, an LSTM introduces a cell state and a collection of gates that regulate the movement of information. The model learns what should be retained, what should be removed, what new content should be written, and what part of its internal memory should be exposed as an output. These operations create a more controlled path through time and make long-range learning more practical.

The word "memory" can make LSTM appear intuitive before it is actually understood. A reader may memorize the names of the forget, input, and output gates and still be unable to explain why training succeeds or fails. For that reason, this book does not treat the LSTM as a sealed layer that receives a tensor and returns another tensor. The cell will be opened, rebuilt, measured, broken intentionally, executed one time step at a time, and compared with trusted PyTorch implementations. Every gate will be studied as both a mathematical operation and an engineering component whose behavior can be inspected through code.

The Central Problem of This Book

The central problem is not simply how to call `nn.LSTM`. The real problem is how a model can preserve relevant information across time while continuously receiving new observations. If it remembers everything, obsolete information accumulates and interferes with later decisions. If it forgets too aggressively, long dependencies disappear. If it writes every new signal into memory,

noise replaces useful context. If it exposes the entire cell state without control, downstream layers receive information that may not be relevant at the current time step.

An LSTM addresses this problem through learned, differentiable decisions. The forget gate scales the previous cell state, the input gate controls how much candidate information is written, and the output gate controls how much of the updated memory becomes visible through the hidden state. Sigmoid functions produce values between zero and one, making the gates behave like soft controls rather than hard switches. The candidate state uses a bounded nonlinear transformation to propose new content. Together, these pieces create a memory update that is additive in an important path, reducing the destructive effect of repeatedly replacing the entire state.

The distinction between the hidden state and the cell state is fundamental. The cell state is the internal memory pathway, while the hidden state is the exposed representation used for outputs, interactions between layers, and the next recurrent computation. They influence each other, but they are not interchangeable. Confusing their shapes, initialization, lifecycle, or role is one of the most common sources of LSTM implementation errors. Throughout this book, the reader will track both states explicitly and learn when they must be initialized, carried, detached, reset, saved, restored, or returned.

Learning Through Controlled Experiments

The learning system in this book follows the same experimental principle used throughout this volume: a concept is not considered understood until its behavior can be produced, observed, challenged, and explained. You will not be asked only to reproduce equations. You will create tasks where memory is necessary, vary the distance of a dependency, inspect the resulting gradients, and determine where one model stops learning reliably. You will force gates toward extreme values to observe what happens when memory is always erased, always retained, never written, or never exposed. These experiments turn abstract gate equations into visible behavior.

Intentional failure is an essential part of the method. You will swap gate order, use incorrect weight shapes, apply the wrong activation, mishandle padding, detach states at the wrong time, forget to detach them when using truncated backpropagation, and create data alignment mistakes that allow a model to predict the wrong target successfully. The objective is not to produce broken systems for their own sake. It is to build the diagnostic ability required to recognize the same failures when they appear silently inside a larger training pipeline.

Every experiment should be treated as a small scientific investigation. Begin with a question, define what changes and what remains controlled, record measurements, compare against a baseline, and explain the result. When comparing a vanilla RNN, GRU, and LSTM, do not change architecture, learning rate, parameter count, and training budget simultaneously and then attribute the outcome to gates. Use controlled seeds, identical data splits, explicit metrics, and repeated runs whenever randomness can change the conclusion. The discipline of the experiment is as important as the code.

What You Will Build

The book begins with LSTM intuition and the reason gates exist. You will write the complete update equations, distinguish the cell state from the hidden state, implement a single LSTM step with pure tensor operations, and verify gate ranges and shapes. The manual implementation will then be unrolled through time and compared with `nn.LSTM`. CPU and CUDA parity,

floating-point precision, fused kernels, long-sequence behavior, and deterministic execution will connect the mathematical cell to the actual computational system.

The next stage studies architecture variants and configuration choices. Many-to-one models will compress a sequence into a prediction, while many-to-many models will preserve temporal alignment across outputs. Stacked and bidirectional LSTMs will show how state shapes change with layers and directions. The reader will examine dropout between recurrent layers, projection choices, hidden size, parameter sharing, state initialization, and the difference between stateless batches and stateful sequence streams. These exercises develop the ability to design an LSTM architecture deliberately instead of adjusting dimensions until the code runs.

Backpropagation through time will then be revisited inside the LSTM. Gates improve the path through which gradients travel, but they do not eliminate every optimization difficulty. You will inspect gradients through the cell state, observe saturation, compare full and truncated backpropagation, apply clipping, perform gradient checking, and debug incorrect state detachment. This part is important because LSTM is often described as if it completely solves vanishing gradients. In reality, it creates a more favorable mechanism for retaining information, while sequence length, truncation, initialization, gate saturation, and optimization still determine what can be learned.

Sequence data handling forms the next layer of the system. Variable-length sequences require padding, masks, packed representations, sorting rules, and careful loss reduction. Stateful tasks require correct boundaries so memory from one sequence does not leak into another. Sliding windows, target shifts, teacher forcing, and chronological splits must preserve the meaning of time. A mathematically correct LSTM cannot recover from a pipeline that feeds the future into the past, evaluates padded positions as real data, or aligns every input with the wrong target.

The book then applies LSTM to two major sequence domains. In language modeling, you will build token sequences, train next-token predictors, generate text autoregressively, control sampling, measure perplexity, and analyze repetition and long-context failure. In time-series forecasting, you will construct multi-feature windows, enforce forecast horizons, compare one-step and multi-step prediction, prevent temporal leakage, use walk-forward validation, study seasonal dependencies, and estimate uncertainty. These applications share recurrent principles but demand different evaluation logic, failure analysis, and data discipline.

Regularization and stabilization receive a complete chapter because a model that trains once is not yet a reliable model. You will study dropout, weight decay, gradient clipping, layer normalization, optimizer choice, scheduling, early stopping, batch size, gate saturation, mixed precision, distribution shift, and calibration. The objective is to understand how these techniques interact with recurrent states rather than applying every method by default. Ablation studies will identify which intervention actually helps and which merely adds complexity.

The engineering chapter moves beyond model accuracy into efficiency, profiling, and deployment. You will measure the effect of sequence length, hidden size, layers, padding, packing, precision, gradient accumulation, and data transfer on CUDA performance. PyTorch profiling will reveal kernel and memory bottlenecks. Checkpointing, reproducible benchmarks, TorchScript, ONNX, production inference latency, state consistency, memory leaks, and monitoring will connect experimental models to systems that must operate repeatedly and safely.

Finally, the book builds a bridge from LSTM to attention. Even a strong LSTM compresses and transports information through sequential state updates, and this creates both an information bottleneck and a parallelism limitation. You will add attention to LSTM outputs, visualize alignment, compare attended and non-attended encoder–decoder models, handle masks, and measure the additional quality and computational cost. This final chapter explains why atten-

tion became necessary and how the limitations of recurrent processing lead naturally toward transformers.

Mathematics, Code, and Interpretation

Three forms of understanding must remain connected throughout the book. First, the mathematical form explains what is computed. Second, the program determines how tensors, devices, states, and gradients realize that computation. Third, interpretation explains what the computation means for memory and learning. Studying only one of these forms creates an incomplete understanding. An equation without tensor shapes can be implemented incorrectly; code without mathematics can run while performing the wrong recurrence; and both can be technically correct without explaining why a trained model behaves as it does.

When you encounter an LSTM equation, identify the shape and purpose of every term. Ask which dimensions represent the batch, time, input features, hidden features, layers, and directions. Determine which operations are matrix transformations, elementwise products, additions, or nonlinearities. Then ask what would happen if a gate approached zero or one, if the cell state grew too large, or if the activation entered saturation. This method transforms a long equation into a set of testable statements.

Do not allow the convenience of PyTorch to hide the mechanism. Built-in LSTM modules are essential for efficient training because they use optimized implementations, but efficiency should follow understanding. The manual cell teaches the internal computation; the built-in module teaches correct engineering practice. Both are necessary, and their outputs should be compared until you can explain any difference in gate order, bias handling, layout, precision, or kernel behavior.

What This Book Does Not Promise

LSTM is not an unlimited memory system. A long sequence can still exceed its effective capacity, gradients can still vanish through some paths, gates can saturate, truncated backpropagation can remove distant credit assignment, and hidden states can become sensitive to distribution shift. Training remains sequential across time, which limits parallelism and makes very long contexts expensive. An LSTM may outperform a vanilla RNN and still be the wrong architecture for a large-scale problem.

This book therefore avoids presenting LSTM as a universal replacement for every sequence model. GRU may provide sufficient memory with lower cost. Temporal convolutions may be more parallel and effective for local patterns. Attention may retrieve distant information more directly. Transformers may scale better for large datasets and long contexts. The correct architecture depends on evidence from the task, not on the historical importance of a model.

The goal is deeper than learning one class from a library. By the end of this book, you should be able to explain why LSTM exists, implement and verify its internal cell, design its surrounding pipeline, diagnose its failures, measure its engineering cost, deploy it responsibly, and recognize when its limitations require the next idea. That final judgment—knowing both the power and the boundary of a method—is what turns implementation into research.

How to Approach the Exercises

Complete the exercises in order because later challenges assume that earlier mechanisms have already been inspected and verified. Do not skip manual implementations simply because a built-in module is available. Do not skip failure cases because the successful version appears clear. The manual, comparative, and debugging exercises form one learning sequence: construct the mechanism, establish a reference, break one assumption, observe the evidence, and repair the correct cause.

Keep an experiment record containing configurations, random seeds, tensor shapes, device information, software versions, training curves, metrics, profiler results, and failure notes. Save both successful and unsuccessful runs. Reproducibility becomes increasingly important as models, datasets, and CUDA behavior become more complex, and a result that cannot be traced back to its conditions cannot support a reliable conclusion.

Most importantly, ask questions before accepting outputs. Why did this gate saturate? Which state carries the useful information? At what dependency distance did performance fail? Did the model learn memory or exploit leakage? Did mixed precision improve useful throughput without damaging gradients? Does the exported model preserve both hidden and cell states? Each exercise is designed to turn one of these questions into evidence. Approach the book patiently, experimentally, and critically; the purpose is not only to make an LSTM run, but to understand the complete system that makes its memory useful.

16.2 LSTM00 - LSTM Intuition and Why Gates Exist

Models and Training

16.2.1 Exercise 00 - Explain Why Vanilla RNN Fails on Long Dependencies and Why LSTM Was Designed as a Fix

Part 1 - Why this matters

A vanilla RNN carries information through repeated nonlinear state updates. Although a path connects an early input to a late prediction, the gradient traveling backward along that path is multiplied by many recurrent Jacobians and can shrink, grow, or become unstable.

This means memory failure is often an optimization problem rather than proof that no recurrent representation exists. The model may solve short dependencies while receiving almost no useful signal about which early observation caused a distant error.

LSTM introduces a cell state with additive updates controlled by learned gates. This creates a more direct route for retained information and gradients, allowing the model to decide when to preserve, erase, write, and expose memory instead of overwriting one state at every step.

Part 2 - Mathematical form

$$\frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial h_T} \prod_{k=t+1}^{T} \frac{\partial h_k}{\partial h_{k-1}}, \qquad c_t = f_t \odot c_{t-1} + i_t \odot g_t.$$

Part 3 - Hints and Helper

Use a delayed-memory task and plot gradient norm against temporal distance. Compare the mechanism, not only final accuracy.

Part 4 - Python code skeleton

```
import torch

def E00(model, data, config):
    task = build_delayed_memory_task(delays=..., ...)
    models = {"rnn": build_rnn(...), "lstm": build_lstm(...)}
    reports = {}
    for name, candidate in models.items():
        reports[name] = train_with_temporal_gradient_probes(
            candidate, task, delays=config.delays, ...
        )
    return explain_failure_and_lstm_fix(reports)
```

16.2.2 Exercise 01 - Define Hidden State vs Cell State and Their Roles in Memory Preservation

Part 1 - Why this matters

An LSTM maintains two related states. The cell state is the internal memory pathway updated through forget and input decisions, while the hidden state is the exposed representation passed to outputs, upper layers, and the next recurrent computation.

The hidden state is filtered through the output gate and a bounded transformation of the cell. Consequently, information may remain stored internally even when it is not currently visible in the hidden representation.

Track both states on a sequence containing remember, distractor, and reveal events. Show when cell content persists, when hidden output changes, and why resetting or detaching only one component creates an inconsistent recurrent state.

Part 2 - Mathematical form

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \qquad h_t = o_t \odot \tanh(c_t).$$

Part 3 - Hints and Helper

Log h and c separately. Their shapes are often equal, but equal shape does not imply equal meaning or interchangeable values.

Part 4 - Python code skeleton

```
import torch

def E01(model, data, config):
    h_t, c_t = initialize_lstm_state(batch=..., hidden_size=...)
    traces = {"hidden": [], "cell": []}
    for x_t in iterate_time_steps(data):
        h_t, c_t = manual_lstm_step(x_t, h_t, c_t, ...)
        traces["hidden"].append(h_t.detach().clone())
        traces["cell"].append(c_t.detach().clone())
    return compare_internal_and_exposed_memory(traces, events=...)
```

16.2.3 Exercise 02 - Write LSTM Update Equations and Map Every Term to Memory Control

Part 1 - Why this matters

The LSTM equations describe four coupled decisions: what old memory remains, what candidate content is written, what the new internal memory becomes, and what part of that memory is exposed. Reading them as one large formula hides this control structure.

The sigmoid gates produce soft coefficients between zero and one, while the candidate uses tanh to propose signed bounded content. Elementwise multiplication allows each hidden feature to follow its own learned retention and update behavior.

Write the equations in execution order, annotate every tensor shape, and map every operation to a memory action. Then test extreme gate values manually so the semantic interpretation agrees with the numerical result.

Part 2 - Mathematical form

$$i_t = \sigma(a_i),\ f_t = \sigma(a_f),\ g_t = \tanh(a_g),\ o_t = \sigma(a_o),\ c_t = f_t \odot c_{t-1} + i_t \odot g_t,\ h_t = o_t \odot \tanh(c_t).$$

Part 3 - Hints and Helper

Distinguish affine preactivations from gate outputs. State the gate ordering used by the implementation you will compare against.

Part 4 - Python code skeleton

```
import torch

def E02(model, data, config):
    x_t, h_prev, c_prev = ...
    preactivations = compute_gate_affine_terms(x_t, h_prev, ...)
    gates = map_equations_to_memory_actions(preactivations, order=...)
    c_t = ...
    h_t = ...
```

```
    verify_shapes_and_semantics(
        inputs=(x_t, h_prev, c_prev), gates=gates, outputs=(h_t, c_t)
    )
    return gates, h_t, c_t
```

16.2.4 Exercise 03 - Implement One LSTM Time Step with Pure Torch Tensors Without nn.LSTM

Part 1 - Why this matters

A manual time step exposes exactly how input and previous hidden state enter the four gate projections. It also reveals why LSTM parameters commonly have four times the hidden dimension in their output axis.

Using pure tensor operations prevents a high-level module from hiding gate order, bias addition, broadcasting, and elementwise state updates. The goal is transparency, not replacing optimized library kernels in production.

Implement the affine transformations, split the gates, apply the correct activations, and update both states. Validate the result against hand calculations on a tiny deterministic example before comparing it with a reference cell.

Part 2 - Mathematical form

$$a_t = x_t W_{ih}^\top + b_{ih} + h_{t-1} W_{hh}^\top + b_{hh} \in \mathbb{R}^{B \times 4H}.$$

Part 3 - Hints and Helper

Use torch operations but not nn.LSTM or nn.LSTMCell. Print intermediate values for a batch of one and a very small hidden size.

Part 4 - Python code skeleton

```
import torch

def E03(model, data, config):
    x_t, h_prev, c_prev = ...
    weight_ih, weight_hh = ...
    bias_ih, bias_hh = ...
    affine = (
        x_t @ weight_ih.T + bias_ih
        + h_prev @ weight_hh.T + bias_hh
    )
    i_raw, f_raw, g_raw, o_raw = affine.chunk(4, dim=-1)
    input_gate = ...
    forget_gate = ...
    candidate = ...
    output_gate = ...
```

```
    c_t = ...
    h_t = ...
    return h_t, c_t
```

16.2.5 Exercise 04 - Validate Sigmoid and Tanh Gate Ranges Precisely

Part 1 - Why this matters

Gate interpretations depend on activation ranges. Sigmoid produces open-interval values between zero and one for finite inputs, while tanh produces values between negative one and one, allowing signed candidate content.

Extreme preactivations may round to boundary values in finite precision, especially in float16. A naive strict assertion can therefore fail even when the implementation is mathematically correct, or pass while severe saturation has destroyed useful gradients.

Test ordinary, extreme, and non-finite inputs across precisions. Verify ranges, count saturated values, inspect activation derivatives, and distinguish valid numerical boundary rounding from NaN or infinity propagation.

Part 2 - Mathematical form

$$0 < \sigma(a) < 1, \qquad -1 < \tanh(a) < 1, \qquad \sigma'(a) = \sigma(a)(1 - \sigma(a)).$$

Part 3 - Hints and Helper

Use tolerant inclusive checks for stored floating-point values and separate them from theoretical strict bounds. Report saturation frequency.

Part 4 - Python code skeleton

```
import torch

def E04(model, data, config):
    preactivations = build_range_test_values(
        ordinary=..., extreme=..., include_nonfinite=True
    )
    reports = {}
    for dtype in [torch.float64, torch.float32, torch.float16]:
        values = preactivations.to(dtype)
        sigmoid_values = torch.sigmoid(values)
        tanh_values = torch.tanh(values)
        reports[str(dtype)] = validate_activation_ranges(
            values, sigmoid_values, tanh_values, saturation_threshold=...
        )
    return reports
```

16.2.6 Exercise 05 - Compare LSTM and Vanilla RNN Forward Behavior on the Same Sequence

Part 1 - Why this matters

Vanilla RNN and LSTM both produce time-indexed hidden representations, but their internal update mechanisms respond differently to distractors, long delays, and reset events. Comparing only final output shapes conceals these behavioral differences.

A controlled comparison should use the same input sequence and known events while recording state evolution. The LSTM has an additional cell state, so its memory cannot be judged solely from exposed hidden activations.

Run both models with designed weights or trained parameters on remember-distract-reveal sequences. Compare hidden trajectories, cell persistence, sensitivity to new inputs, and output behavior as dependency distance grows.

Part 2 - Mathematical form

$$h_t^R = \tanh(W_x x_t + W_h h_{t-1}), \qquad (h_t^L, c_t^L) = \mathrm{LSTM}(x_t, h_{t-1}^L, c_{t-1}^L).$$

Part 3 - Hints and Helper

Use probes or visualizations aligned with sequence events. Do not claim better memory from a single untrained random forward pass.

Part 4 - Python code skeleton

```
import torch

def E05(model, data, config):
    sequence, events = build_remember_distract_reveal_sequence(...)
    rnn_trace = trace_vanilla_rnn(build_rnn(...), sequence, ...)
    lstm_trace = trace_lstm(build_lstm(...), sequence, ...)
    return compare_forward_memory_behavior(
        rnn_trace,
        lstm_trace,
        events=events,
        dependency_distances=...,
    )
```

16.2.7 Exercise 06 - Force Forget-Gate Extremes and Observe Memory Retention

Part 1 - Why this matters

The forget gate multiplies the previous cell state before new content is added. Forcing it near zero removes old memory, while forcing it near one permits the old state to pass forward

almost unchanged through the direct retention path.

Retention is not the same as correct memory. A permanently open forget gate can accumulate obsolete information and cause interference, whereas a permanently closed gate prevents every long dependency.

Hold the input and output gates controlled while sweeping forget-gate values. Measure cell-state decay, persistence time, response to reset events, and gradients through the retained state.

Part 2 - Mathematical form

$$c_t = f\, c_{t-1} + i_t g_t, \qquad c_{t+k} = f^k c_t \text{ when no new content is written.}$$

Part 3 - Hints and Helper

Use values such as zero, 0.5, near one, and one. Include a task that requires forgetting as well as retaining.

Part 4 - Python code skeleton

```
import torch

def E06(model, data, config):
    reports = {}
    for forced_forget in [0.0, 0.5, 0.99, 1.0]:
        cell_trace = run_controlled_lstm_cell(
            data,
            forced_gates={"forget": forced_forget, "input": ...,
                          "output": ...},
        )
        reports[forced_forget] = measure_retention_decay_and_interference(
            cell_trace, events=...
        )
    return reports
```

16.2.8 Exercise 07 - Force Input-Gate Extremes and Observe Memory Writing

Part 1 - Why this matters

The input gate determines how much candidate content enters the cell state. Closing it protects existing memory from new information, while opening it allows the candidate to modify the stored representation strongly.

An always-open input gate does not guarantee useful learning because noise and irrelevant observations can continuously overwrite or accumulate within memory. An always-closed gate preserves initialization but prevents adaptation to the sequence.

Force input-gate values on sequences containing relevant and irrelevant write events. Track cell-state change, successful encoding, noise contamination, and the interaction between writing and the forget gate.

Part 2 - Mathematical form

$$\Delta c_t = c_t - f_t \odot c_{t-1} = i_t \odot g_t.$$

Part 3 - Hints and Helper

Keep candidate values visible in the report. A small cell update may come from a closed input gate or a near-zero candidate.

Part 4 - Python code skeleton

```
import torch

def E07(model, data, config):
    reports = {}
    for forced_input in [0.0, 0.25, 0.75, 1.0]:
        traces = run_controlled_lstm_cell(
            data,
            forced_gates={"input": forced_input, "forget": ...,
                          "output": ...},
            expose_candidate=True,
        )
        reports[forced_input] = analyze_memory_writing(
            traces, relevant_events=..., distractor_events=...
        )
    return reports
```

16.2.9 Exercise 08 - Force Output-Gate Extremes and Observe newline Hidden-State Exposure

Part 1 - Why this matters

The output gate controls how much transformed cell content becomes visible in the hidden state. Closing it can hide stored memory without erasing the cell, while opening it exposes the bounded transformation of current memory.

This separation allows an LSTM to retain information internally until it becomes relevant. It also means that inspecting hidden output alone can incorrectly suggest that memory was lost when it was merely concealed.

Keep cell-state dynamics fixed while sweeping output-gate values. Compare internal memory, hidden activation, downstream prediction, and the ability to reveal previously stored information at a chosen event.

Part 2 - Mathematical form

$$h_t = o_t \odot \tanh(c_t), \qquad c_t \text{ is unchanged by } o_t \text{ in the same step.}$$

Part 3 - Hints and Helper

Verify that forced output changes h but not c when all other computations are held constant. Plot both traces on the same timeline.

Part 4 - Python code skeleton

```
import torch

def E08(model, data, config):
    fixed_cell_trace = ...
    reports = {}
    for forced_output in [0.0, 0.5, 1.0]:
        hidden_trace = expose_cell_with_forced_output_gate(
            fixed_cell_trace, forced_output=forced_output
        )
        reports[forced_output] = compare_internal_and_exposed_state(
            fixed_cell_trace, hidden_trace, reveal_event=...
        )
    return reports
```

16.2.10 Exercise 09 - Verify Gate and State Shapes for Batch-First and Time-First Layouts

Part 1 - Why this matters

LSTM input layout may be batch-time-feature or time-batch-feature, but recurrent state layout remains layer-direction, batch, hidden. This asymmetry is a frequent source of silent indexing and initialization errors.

Gate tensors inside one layer typically combine four hidden-sized blocks, while sequence outputs expose only the hidden dimension per direction. Bidirectionality and stacking will later add further structure, so the base case must be mastered first.

Run both layout configurations on equivalent transposed inputs. Assert input, output, hidden, cell, gate, weight, and bias shapes, then restore a common layout and verify numerical parity.

Part 2 - Mathematical form

$$X_{BF} \in \mathbb{R}^{B\times T\times I},\ X_{TF} \in \mathbb{R}^{T\times B\times I},\ h_0, c_0 \in \mathbb{R}^{L\times B\times H}.$$

Part 3 - Hints and Helper

Do not transpose h0 or c0 when toggling `batch_first`. Compare outputs after moving their time and batch axes into the same order.

Part 4 - Python code skeleton

```
import torch

def E09(model, data, config):
    batch, time, input_size, hidden_size = ...
    x_bf = torch.randn(batch, time, input_size)
    x_tf = x_bf.transpose(0, 1)
    h0 = torch.zeros(..., batch, hidden_size)
    c0 = torch.zeros(..., batch, hidden_size)
    reports = run_both_layouts(x_bf, x_tf, h0, c0, ...)
    assert_expected_lstm_shapes(reports, ...)
    verify_layout_parity(reports, ...)
    return reports
```

16.2.11 Exercise 10 - Run the Manual LSTM Step on CUDA and Confirm CPU Parity

Part 1 - Why this matters

Moving a manual cell to CUDA tests whether every input, state, weight, bias, and intermediate tensor follows the intended device and dtype. A single CPU-created state or constant can break an otherwise correct implementation.

CPU and GPU floating-point operations may differ slightly because of kernel algorithms and operation ordering. Correctness should therefore use explicit tolerances rather than exact equality, while large differences require investigation.

Run identical parameters and inputs on both devices across several shapes and values. Compare every gate, cell state, hidden state, and backward gradient, and document the tolerance required by each precision.

Part 2 - Mathematical form

$$\varepsilon_\infty = \max_j |y_j^{CPU} - y_j^{GPU}|.$$

Part 3 - Hints and Helper

Create tensors from existing tensors with `new_zeros` or device-aware constructors. Synchronize CUDA before timing, not for a pure numerical comparison.

Part 4 - Python code skeleton

```
import torch

def E10(model, data, config):
    cpu_case = build_manual_lstm_case(device="cpu", dtype=..., seed=...)
    gpu_case = clone_case_to_device(cpu_case, device="cuda")
    cpu_outputs = run_manual_step_with_gradients(cpu_case)
```

```
    gpu_outputs = run_manual_step_with_gradients(gpu_case)
    parity = compare_nested_tensors(
        cpu_outputs, move_to_cpu(gpu_outputs), atol=..., rtol=...
    )
    return parity
```

16.2.12 Exercise 11 - Benchmark Manual LSTM vs nn.LSTM and Explain Kernel Fusion Benefits

Part 1 - Why this matters

A Python-level manual unroll launches separate operations for gate projections, activations, elementwise updates, and every time step. Built-in LSTM implementations can combine work, reduce intermediate traffic, and use optimized recurrent kernels.

The performance gap depends on shape and device. Tiny sequences may be dominated by framework overhead, while large CUDA workloads benefit more from fused execution and efficient workspace management.

Benchmark a correct manual unroll against nn.LSTM using identical weights and inputs. Measure forward latency, useful-token throughput, peak memory, launch count, and output parity after warm-up.

Part 2 - Mathematical form

$$S = \frac{T_{\text{manual}}}{T_{\text{fused}}}, \qquad \text{throughput} = \frac{BT}{T_{\text{forward}}}.$$

Part 3 - Hints and Helper

Synchronize CUDA timing boundaries and warm up every shape. Do not include model construction or parameter copying in steady-state latency.

Part 4 - Python code skeleton

```
import torch

def E11(model, data, config):
    manual, builtin = build_weight_matched_lstm_implementations(...)
    reports = {}
    for name, candidate in {"manual": manual, "builtin": builtin}.items():
        reports[name] = benchmark_forward(
            candidate,
            data,
            warmup=...,
            repetitions=...,
            synchronize_cuda=True,
            profile_launches=True,
```

```
        )
    verify_output_parity(reports, ...)
    return explain_fusion_speedup(reports)
```

16.2.13 Exercise 12 - Compare float32 and float16 LSTM Outputs and Find Precision-Sensitive Gate Cases

Part 1 - Why this matters

Float16 reduces memory and may accelerate CUDA computation, but it has narrower range and lower precision than float32. LSTM gates can become sensitive when preactivations are extreme or when small cell updates are added repeatedly.

Sigmoid saturation may erase distinctions between large inputs, while tiny candidate updates can underflow or disappear when added to a much larger cell state. Error may accumulate across a long unroll even if one step appears close.

Compare gate values, states, and final outputs across precisions for ordinary and adversarial scales. Locate the first time step where error exceeds tolerance and relate it to saturation, cancellation, or accumulated rounding.

Part 2 - Mathematical form

$$e_t = \|h_t^{32} - h_t^{16}\|_\infty, \qquad t^* = \min\{t : e_t > \tau\}.$$

Part 3 - Hints and Helper

Keep a float64 reference for diagnostic cases when useful. Evaluate both one-step and long-unroll error.

Part 4 - Python code skeleton

```
import torch

def E12(model, data, config):
    scenarios = build_precision_scenarios(
        normal=..., saturated=..., tiny_updates=..., long_unroll=...
    )
    reports = {}
    for name, scenario in scenarios.items():
        outputs32 = run_lstm_precision(scenario, dtype=torch.float32)
        outputs16 = run_lstm_precision(scenario, dtype=torch.float16)
        reports[name] = analyze_gate_and_state_drift(
            outputs32, outputs16, threshold=...
        )
    return reports
```

16.2.14 Exercise 13 - Debug Gate Order, Concatenation, and newline Weight-Shape Mistakes

Part 1 - Why this matters

LSTM bugs often preserve tensor validity. Swapping input and forget blocks, concatenating along the batch axis, transposing a weight incorrectly, or using a bias with the wrong semantic order can still produce numbers of plausible shape.

Final-output comparison tells you that something is wrong but not where it began. Step-level inspection of affine terms, gate chunks, activations, and state updates identifies the first divergence from a trusted reference.

Create several intentionally broken cells and build a diagnostic harness that names each fault. Include gate permutations, wrong concatenation dimension, incorrect recurrent weight orientation, missing bias, and accidental broadcasting.

Part 2 - Mathematical form

$$a_t = [a_i \mid a_f \mid a_g \mid a_o], \qquad a_t \in \mathbb{R}^{B\times 4H}.$$

Part 3 - Hints and Helper

Use distinct sentinel values for each gate block so permutations are immediately visible. Validate shapes before comparing numerical values.

Part 4 - Python code skeleton

```
import torch

def E13(model, data, config):
    reference = run_reference_lstm_step(...)
    broken_cells = build_faulty_lstm_cells(
        faults=["gate_order", "concat_axis", "weight_transpose",
                "missing_bias", "broadcasting"],
        ...
    )
    reports = {}
    for fault, cell in broken_cells.items():
        trace = run_step_with_intermediate_trace(cell, ...)
        reports[fault] = locate_first_reference_divergence(trace, reference)
    return reports
```

16.2.15 Exercise 14 - Manually Unroll LSTM Through Time and Compare with nn.LSTM

Part 1 - Why this matters

A single verified step becomes a sequence model only when its states are carried correctly across time. Manual unrolling makes temporal state flow explicit and reveals the difference

between collecting every hidden output and returning the final states.

Layout, gate order, bias handling, and parameter copying must all match the reference module. One incorrect state assignment at any step propagates into every later output.

Unroll the manual cell over a batch sequence, stack hidden outputs in the correct time dimension, and compare the full output tensor plus final hidden and cell states against nn.LSTM. Then repeat backward comparison with a scalar loss.

Part 2 - Mathematical form

$$(h_t, c_t) = F_\theta(x_t, h_{t-1}, c_{t-1}), \qquad Y = \text{stack}(h_1, \ldots, h_T).$$

Part 3 - Hints and Helper

Verify one step first, then two, then the complete sequence. Copy both bias vectors used by the reference implementation.

Part 4 - Python code skeleton

```
import torch

def E14(model, data, config):
    manual_cell, builtin = build_weight_matched_lstm(...)
    h_t, c_t = ...
    outputs = []
    for x_t in iterate_time_steps(data, batch_first=...):
        h_t, c_t = manual_cell(x_t, (h_t, c_t))
        outputs.append(h_t)
    manual_output = torch.stack(outputs, dim=...)
    builtin_output, builtin_state = builtin(data, ...)
    verify_forward_and_backward_parity(
        manual_output, (h_t, c_t), builtin_output, builtin_state, ...
    )
    return manual_output, (h_t, c_t)
```

16.2.16 Exercise 15 - Stress Test Long-Sequence LSTM Forward Runtime and GPU Memory

Part 1 - Why this matters

LSTM improves memory dynamics but remains sequential across time and stores two recurrent states. Long sequences raise runtime and, during training, require many activations for backward computation.

A practical limit must include memory headroom and stable repeated execution, not merely one successful allocation. Sequence length, batch size, hidden size, layers, precision, and inference versus training mode all change the boundary.

Increase sequence length gradually while monitoring latency, tokens per second, allocated and peak memory, state magnitude, and non-finite outputs. Record the first unsafe configuration and classify whether the limit is memory, runtime, or numerical.

Part 2 - Mathematical form

$$C = O(BTLH^2), \qquad M_{\text{train}} = M_{\text{params}} + O(BTLH).$$

Part 3 - Hints and Helper

Warm up each shape and stop before exhausting the device. Reset peak statistics between cases while preserving a safety margin.

Part 4 - Python code skeleton

```
import torch

def E15(model, data, config):
    reports = []
    for length in config.sequence_lengths:
        batch = make_lstm_input(length=length, ...).cuda()
        torch.cuda.reset_peak_memory_stats()
        result = guarded_forward_benchmark(
            model.cuda(), batch, warmup=..., repetitions=...,
            monitor_nonfinite=True, memory_headroom=...
        )
        reports.append({"length": length, **result})
    return identify_forward_operating_limit(reports)
```

16.2.17 Exercise 16 - Investigate cuDNN LSTM Non-Determinism and Document It

Part 1 - Why this matters

Repeated CUDA runs can differ slightly even when model parameters and inputs appear identical. Parallel arithmetic, algorithm selection, precision, library version, and nondeterministic kernels can change operation order and rounding.

Fixed PyTorch seeds do not control every source by themselves. DataLoader workers, dropout, environment settings, and deterministic algorithm policies must also be managed, and exact equality must be distinguished from tolerance-based reproducibility.

Repeat forward and backward experiments across shapes, precisions, and deterministic settings. Locate first divergence, measure its growth through time, record performance cost, and document the guarantees available on the tested system.

Part 2 - Mathematical form

$$\delta_t = \max |h_t^{(a)} - h_t^{(b)}|.$$

Part 3 - Hints and Helper

Capture GPU, CUDA, cuDNN, driver, and PyTorch versions. Control all random sources before attributing differences to a kernel.

Part 4 - Python code skeleton

```
import torch

def E16(model, data, config):
    reports = []
    for deterministic in [False, True]:
        configure_all_randomness(seed=..., deterministic=deterministic)
        runs = [
            run_lstm_forward_backward(model, data, ...)
            for _ in range(...)
        ]
        reports.append(analyze_repeated_runs(
            runs,
            locate_first_divergence=True,
            include_performance_cost=True,
            environment=capture_cuda_environment(),
        ))
    return reports
```

16.2.18 Exercise 17 - Build a Long-Memory Synthetic Task and Confirm LSTM Outperforms RNN

Part 1 - Why this matters

A synthetic task with a known dependency distance provides direct evidence about memory. Unlike natural data, it can prevent shortcuts and reveal exactly which earlier symbol must influence the target.

The experiment must vary delay and include balanced distractors so a constant or local predictor cannot succeed. Both models need fair tuning and repeated seeds because one unstable run cannot support an architectural conclusion.

Construct a delayed copy, parity, or remember-and-reveal task. Compare RNN and LSTM accuracy, success rate, convergence time, temporal gradients, and hidden-state probes across increasing dependency lengths.

Part 2 - Mathematical form

$$y_T = \psi(x_{t^*}), \qquad T - t^* = d, \qquad d \in \mathcal{D}.$$

Part 3 - Hints and Helper

Include no-memory and recent-memory baselines. Generate train and test sequences independently to prevent memorization of fixed patterns.

Part 4 - Python code skeleton

```
import torch

def E17(model, data, config):
    datasets = build_balanced_long_memory_tasks(
        delays=config.delays, train_samples=..., test_samples=...
    )
    results = run_repeated_model_comparison(
        models={"rnn": build_rnn, "lstm": build_lstm},
        datasets=datasets,
        seeds=config.seeds,
        equal_tuning_budget=True,
        ...
    )
    return confirm_dependency_advantage(results, baseline=...)
```

16.2.19 Exercise 18 - Find the Smallest Hidden Size That Retains Long Dependencies

Part 1 - Why this matters

A larger hidden size increases representational capacity but also parameters, memory, runtime, and the opportunity to overfit. The smallest successful LSTM is often a better engineering choice than the largest model that fits.

Capacity must be tested across repeated seeds and dependency distances. A model that succeeds once or only on training data has not demonstrated reliable memory retention.

Sweep hidden sizes under equal training and tuning budgets. Measure parameter count, success probability, validation accuracy, memory, latency, and gate behavior, then select the smallest size satisfying predefined quality and stability thresholds.

Part 2 - Mathematical form

$$H^* = \min\{H : \Pr(M_H \geq \tau) \geq p_{\min}\}.$$

Part 3 - Hints and Helper

Define the quality threshold and required success probability before the sweep. Use an untouched test delay beyond the easiest training cases.

Part 4 - Python code skeleton

```
import torch

def E18(model, data, config):
    reports = []
    for hidden_size in config.hidden_sizes:
        runs = run_repeated_lstm_training(
            hidden_size=hidden_size,
            data=data,
            seeds=config.seeds,
            budget=...,
        )
        reports.append({
            "hidden_size": hidden_size,
            "parameters": count_lstm_parameters(hidden_size, ...),
            **summarize_capacity_runs(runs),
        })
    return select_smallest_reliable_lstm(
        reports, threshold=..., probability=...
    )
```

16.2.20 Exercise 19 - Explain How Gates Enable More Stable Gradient Flow Through Time

Part 1 - Why this matters

LSTM gates improve gradient flow by creating a cell-state path whose update contains addition and controllable elementwise retention. When forget values remain near one, gradients can pass through many steps without repeatedly crossing a full saturating state transformation.

This mechanism is conditional rather than guaranteed. Forget gates below one still cause decay, gate saturation can limit learning, truncation cuts gradient paths deliberately, and indirect dependencies through gate computations remain nonlinear.

Produce a conceptual report combining equations, Jacobian analysis, forced-gate experiments, temporal gradient plots, memory-task results, precision observations, and performance costs. State precisely what LSTM improves and which limitations remain.

Part 2 - Mathematical form

$$\frac{\partial c_T}{\partial c_t} \supseteq \prod_{k=t+1}^{T} f_k.$$

Part 3 - Hints and Helper

Separate the direct cell path from all indirect derivative terms. Support each claim with an experiment from LSTM00.

Part 4 - Python code skeleton

```
import torch

def E19(model, data, config):
    evidence = load_lstm00_experiment_results(...)
    report = ConceptualGradientFlowReport()
    report.add_recurrent_jacobian_comparison(evidence, ...)
    report.add_gate_equation_interpretation(evidence, ...)
    report.add_forced_gate_results(evidence, ...)
    report.add_temporal_gradient_and_memory_curves(evidence, ...)
    report.add_precision_and_engineering_limits(evidence, ...)
    report.validate_every_claim_has_evidence()
    return report.export(...)
```

16.3 LSTM01 - LSTM Architecture Variants and Configurations

Models and Training

16.3.1 Exercise 00 - Implement a Many-to-One LSTM Classifier and Select the Last Hidden State Correctly

Part 1 - Why this matters

A many-to-one classifier consumes an entire sequence and produces one decision. The final representation must summarize the relevant history, but the phrase “last hidden state” becomes ambiguous when sequences are padded, layers are stacked, or directions are added.

For a single-layer unidirectional LSTM with equal lengths, the last sequence output and final hidden state correspond closely. With variable lengths, indexing the padded final column may select padding rather than the last valid event.

Build a classifier that obtains the final state from the recurrent module or gathers each sequence’s last valid output. Compare both methods on controlled lengths, validate logits and state shapes, and prove that padding does not change a sample’s prediction.

Part 2 - Mathematical form

$$h_i^* = h_{T_i}^{(L)}, \qquad \widehat{y}_i = W_o h_i^* + b_o.$$

Part 3 - Hints and Helper

For packed sequences, use the returned final hidden state carefully. For padded outputs, gather index length minus one for every sample.

Part 4 - Python code skeleton

```
import torch
from torch import nn

class ManyToOneLSTM(nn.Module):
    def __init__(self, input_size, hidden_size, classes):
        super().__init__()
        self.lstm = nn.LSTM(input_size, hidden_size, batch_first=True)
        self.head = nn.Linear(hidden_size, classes)

    def forward(self, x, lengths):
        outputs, (h_n, c_n) = self.lstm(x)
        last_valid = gather_last_valid(outputs, lengths - 1)
        logits = self.head(...)
        verify_final_state_selection(last_valid, h_n, lengths, ...)
        return logits
```

16.3.2 Exercise 01 - Implement Many-to-Many LSTM Sequence Labeling and Validate Temporal Alignment

Part 1 - Why this matters

Many-to-many sequence labeling produces an output for each valid time step. The model must preserve temporal alignment so label t is compared with the representation created from input position t under the chosen causal or bidirectional context.

Flattening batch and time dimensions can simplify loss computation, but incorrect reshaping or target shifting silently changes which prediction is matched to which label. Padded steps must also be excluded from both loss and metrics.

Build a token-level classifier, trace one identifiable sequence through the pipeline, and verify output, target, and mask alignment. Compare vectorized loss with an explicit valid-position calculation.

Part 2 - Mathematical form

$$H \in \mathbb{R}^{B\times T\times H}, \quad Z = HW_o^\top + b_o \in \mathbb{R}^{B\times T\times C}.$$

Part 3 - Hints and Helper

Use a monotonic synthetic sequence whose expected labels are visually obvious. Assert the same valid-token count in outputs, targets, and mask.

Part 4 - Python code skeleton

```
import torch
from torch import nn

class SequenceLabeler(nn.Module):
```

```
    def __init__(self, input_size, hidden_size, classes):
        super().__init__()
        self.lstm = nn.LSTM(input_size, hidden_size, batch_first=True)
        self.head = nn.Linear(hidden_size, classes)

    def forward(self, x):
        outputs, state = self.lstm(x)
        return self.head(outputs), state

def E01(model, data, config):
    logits, state = model(data.inputs)
    loss = masked_sequence_loss(logits, data.targets, data.mask, ...)
    validate_time_alignment(logits, data.targets, data.mask, ...)
    return loss, state
```

16.3.3 Exercise 02 - Implement a Stacked LSTM and Confirm Multi-Layer Hidden and Cell Shapes

Part 1 - Why this matters

A stacked LSTM passes the output sequence from one recurrent layer into the next. Each layer has its own hidden and cell states, allowing higher layers to transform temporal features created by lower layers.

The final state tensors store one slice per layer and direction, not one slice per time step. Confusing layer index with time index can select a valid-shaped tensor from the wrong semantic location.

Build stacked models with several depths, inspect parameter shapes by layer, and verify sequence output plus h and c states. Use hooks or a manual stack to compare intermediate layer outputs with the fused multi-layer module.

Part 2 - Mathematical form

$$h_n, c_n \in \mathbb{R}^{L \times B \times H}, \qquad Y^{(\ell)} = \mathrm{LSTM}^{(\ell)}(Y^{(\ell-1)}).$$

Part 3 - Hints and Helper

The sequence output comes from the final layer, while `h_n` and `c_n` contain final states from all layers. Initialize every layer-state slice.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def E02(model, data, config):
    for layers in config.layer_counts:
```

```
        candidate = nn.LSTM(
            input_size=..., hidden_size=..., num_layers=layers,
            batch_first=True
        )
        h0 = torch.zeros(layers, data.batch_size, config.hidden_size)
        c0 = torch.zeros_like(h0)
        outputs, (h_n, c_n) = candidate(data.inputs, (h0, c0))
        assert_stacked_lstm_shapes(outputs, h_n, c_n, layers=layers, ...)
        compare_with_manual_layer_stack(candidate, data.inputs, ...)
    return ...
```

16.3.4 Exercise 03 - Implement Bidirectional LSTM and Verify Concatenation and Output Dimensions

Part 1 - Why this matters

A bidirectional LSTM processes the same sequence from both temporal directions and concatenates their representations. Each output position can therefore contain information from earlier and later observations.

The output feature dimension doubles, and final state tensors interleave layer and direction indices. The last forward state and last backward state correspond to different locations in the output sequence, so careless indexing produces incorrect sequence summaries.

Build a bidirectional model on an identifiable sequence, split forward and backward output blocks, and verify their relation to final states. Assert dimensions for outputs, h, c, and the downstream projection.

Part 2 - Mathematical form

$$y_t = [\overrightarrow{h}_t; \overleftarrow{h}_t] \in \mathbb{R}^{2H}, \qquad h_n, c_n \in \mathbb{R}^{2L \times B \times H}.$$

Part 3 - Hints and Helper

Reshape final states to layer, direction, batch, hidden before selecting them. Do not assume output at the last time step contains both final directions.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def E03(model, data, config):
    lstm = nn.LSTM(
        input_size=..., hidden_size=..., num_layers=...,
        bidirectional=True, batch_first=True
    )
    outputs, (h_n, c_n) = lstm(data.inputs)
```

```
    forward_output, backward_output = outputs.chunk(2, dim=-1)
    states = reshape_states_by_layer_and_direction(h_n, c_n, ...)
    verify_bidirectional_state_output_mapping(
        forward_output, backward_output, states, ...
    )
    return outputs, states
```

16.3.5 Exercise 04 - Compare Unidirectional and Bidirectional LSTM for Sequence Labeling

Part 1 - Why this matters

Bidirectional context can improve a label when future observations clarify the current position. It is invalid, however, for causal streaming or forecasting tasks where future inputs do not exist at prediction time.

The comparison must account for doubled representation width, extra parameters, latency, and the need to wait for the complete sequence. Accuracy gain alone cannot justify an architecture that violates deployment causality.

Train both models on a task with controllable future-context dependence. Compare equal hidden size and approximately equal parameter count, then measure quality by position, runtime, memory, and online feasibility.

Part 2 - Mathematical form

$$p(y_t \mid x_{1:T}) \text{ is bidirectional}, \qquad p(y_t \mid x_{1:t}) \text{ is causal.}$$

Part 3 - Hints and Helper

Include one causal task and one task where future context truly helps. State whether the intended application has the full sequence available.

Part 4 - Python code skeleton

```
import torch

def E04(model, data, config):
    variants = {
        "unidirectional": build_labeler(bidirectional=False, ...),
        "bidirectional": build_labeler(bidirectional=True, ...),
    }
    results = run_controlled_architecture_comparison(
        variants,
        data,
        matching=["hidden_size", "parameter_count"],
        metrics_by_position=True,
        measure_runtime=True,
        ...
    )
    return summarize_quality_causality_tradeoff(results)
```

16.3.6 Exercise 05 - Use batch_first and Keep Shapes Consistent Across the Pipeline

Part 1 - Why this matters

The batch-first option changes the input and output sequence layout, but it does not change the recurrent state layout. This deliberate asymmetry can create errors when a pipeline assumes every tensor follows the same axis order.

DataLoader collation, masking, packing, loss reduction, and visualization must agree on where batch and time dimensions live. Repeated transposes can also create non-contiguous views and hidden copies.

Implement equivalent batch-first and time-first pipelines, verify numerical parity after alignment, and inspect strides and contiguity. Add shape assertions at every boundary so a later configuration change fails immediately.

Part 2 - Mathematical form

$$X_{BF} \in \mathbb{R}^{B\times T\times I}, \quad X_{TF} \in \mathbb{R}^{T\times B\times I}, \quad h_0, c_0 \in \mathbb{R}^{LD\times B\times H}.$$

Part 3 - Hints and Helper

Never transpose h0 or c0 because of `batch_first`. Normalize both pipeline outputs into one common layout before comparison.

Part 4 - Python code skeleton

```
import torch

def E05(model, data, config):
    x_bf = data.inputs
    x_tf = x_bf.transpose(0, 1)
    reports = {
        "batch_first": run_complete_pipeline(
            build_lstm(batch_first=True, ...), x_bf, layout="BTF", ...
        ),
        "time_first": run_complete_pipeline(
            build_lstm(batch_first=False, ...), x_tf, layout="TBF", ...
        ),
    }
    verify_pipeline_shape_contracts(reports, ...)
    verify_layout_parity(reports, ...)
    return reports
```

16.3.7 Exercise 06 - Implement Inter-Layer Dropout and Verify the num_layers Condition

Part 1 - Why this matters

Built-in LSTM dropout is applied between stacked recurrent layers, not across time inside a single layer. Therefore, a one-layer LSTM has no inter-layer connection on which this dropout can operate.

Misunderstanding this behavior can create a false sense of regularization. Dropout must also change between training and evaluation modes, while repeated evaluation outputs should remain stable.

Compare one-layer and multi-layer models under several dropout probabilities. Use repeated forward passes to detect stochastic changes, inspect train and eval behavior, and verify that recurrent state shapes remain unaffected.

Part 2 - Mathematical form

$$\widetilde{y}_t^{(\ell)} = m_t^{(\ell)} \odot y_t^{(\ell)}/(1-p), \qquad \ell < L.$$

Part 3 - Hints and Helper

Expect a warning or no effective inter-layer dropout when `num_layers` equals one. Test behavior statistically rather than checking one output.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def E06(model, data, config):
    reports = {}
    for layers in [1, 2, 3]:
        candidate = nn.LSTM(
            input_size=..., hidden_size=..., num_layers=layers,
            dropout=config.dropout, batch_first=True
        )
        train_outputs = repeated_forward(candidate.train(), data.inputs, ...)
        eval_outputs = repeated_forward(candidate.eval(), data.inputs, ...)
        reports[layers] = analyze_dropout_activity(
            train_outputs, eval_outputs, expected_active=(layers > 1)
        )
    return reports
```

16.3.8 Exercise 07 - Build a Capacity Curve Across Hidden Sizes and Layer Counts

Part 1 - Why this matters

Hidden size controls recurrent width, while layer count controls hierarchical depth. Both increase capacity, but their parameter, runtime, memory, optimization, and generalization effects differ.

A larger model may reduce training error while worsening validation performance or becoming too slow for the application. Conversely, a narrow deep model can create an information bottleneck or unstable gradient path.

Sweep a grid of widths and depths under controlled data and budgets. Record parameter count, accuracy, convergence, peak memory, latency, gradient norms, and failure rate, then construct a quality-cost frontier.

Part 2 - Mathematical form

$$P \approx \sum_{\ell=1}^{L} 4H_\ell(I_\ell + H_\ell + 2).$$

Part 3 - Hints and Helper

Use multiple seeds and the same early-stopping rule. Compare training-budget and wall-clock-budget views separately.

Part 4 - Python code skeleton

```
import torch

def E07(model, data, config):
    reports = []
    for hidden_size in config.hidden_sizes:
        for layers in config.layer_counts:
            candidate = build_lstm(
                hidden_size=hidden_size, layers=layers, ...
            )
            result = train_benchmark_and_evaluate(candidate, data, ...)
            reports.append({
                "hidden_size": hidden_size,
                "layers": layers,
                "parameters": count_parameters(candidate),
                **result,
            })
    return build_capacity_performance_frontier(reports)
```

16.3.9 Exercise 08 - Implement LSTM Projection and Explain Why It Helps

Part 1 - Why this matters

An LSTM projection separates internal cell width from exposed recurrent width. A larger cell can maintain rich memory while a smaller projected hidden state reduces downstream dimensions and recurrent parameter cost.

With projection, hidden and cell states no longer share the same final dimension. The cell state uses `hidden_size`, while hidden outputs use `proj_size`, making old shape assumptions

invalid.

Use the available projected LSTM implementation or emulate projection after a cell. Compare parameter count, output and state shapes, runtime, memory, and accuracy against a non-projected model under controlled budgets.

Part 2 - Mathematical form

$$\widetilde{h}_t = o_t \odot \tanh(c_t) \in \mathbb{R}^H, \qquad h_t = W_p \widetilde{h}_t \in \mathbb{R}^P,\ P < H.$$

Part 3 - Hints and Helper

Verify library support on the chosen backend. Distinguish an external output projection from a recurrent projection fed into the next time step.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def E08(model, data, config):
    projected = nn.LSTM(
        input_size=...,
        hidden_size=config.cell_size,
        proj_size=config.projection_size,
        batch_first=True,
    )
    outputs, (h_n, c_n) = projected(data.inputs)
    assert_projection_shapes(
        outputs, h_n, c_n,
        cell_size=config.cell_size,
        projection_size=config.projection_size,
    )
    return compare_projected_and_standard_lstm(projected, data, ...)
```

16.3.10 Exercise 09 - Debug Shape Errors in Multi-Layer and Bidirectional LSTM

Part 1 - Why this matters

Stacking and bidirectionality combine layer, direction, batch, time, hidden, and sometimes projection dimensions. Most mistakes come from applying a correct formula to the wrong configuration.

Hard-coded multipliers can survive one test while failing when depth, directionality, or projection changes. Error messages often appear in the classifier head even though the incorrect assumption began in recurrent-state construction.

Create faulty configurations for state shape, head width, direction ordering, last-state selection, and projection. Build a shape-debugging report that computes expectations from model attributes and locates the first broken boundary.

Part 2 - Mathematical form

$$D = 1 + \mathbf{1}_{\text{bidirectional}}, \quad Y \in \mathbb{R}^{B \times T \times DH_{out}}, \quad h_n \in \mathbb{R}^{DL \times B \times H_{out}}.$$

Part 3 - Hints and Helper

Derive shapes from `num_layers`, `bidirectional`, `hidden_size`, and `proj_size` rather than copying constants.

Part 4 - Python code skeleton

```
import torch

def E09(model, data, config):
    faulty_variants = build_shape_failure_cases(
        faults=["state_layers", "head_width", "direction_order",
                "last_state", "projection_width"],
        ...
    )
    reports = {}
    for name, candidate in faulty_variants.items():
        expected = derive_lstm_shape_contract(candidate, data.inputs)
        reports[name] = run_shape_boundary_audit(candidate, data, expected)
    return locate_architecture_shape_failures(reports)
```

16.3.11 Exercise 10 - Move the Complete LSTM Architecture to CUDA and Validate State Device Placement

Part 1 - Why this matters

A complete architecture includes embeddings or feature projections, recurrent parameters, initial states, masks, targets, and classifier heads. Moving only the LSTM module leaves other components on CPU and creates device mismatch failures.

Fresh hidden and cell states must be created on the same device and compatible dtype as the model or input. Device-aware creation becomes especially important under mixed precision and multi-device code.

Move an end-to-end architecture and batch to CUDA, initialize states dynamically, and verify every parameter, buffer, input, output, loss, and gradient device. Compare controlled CPU and GPU outputs within tolerances.

Part 2 - Mathematical form

$$\text{device}(x) = \text{device}(h_0) = \text{device}(c_0) = \text{device}(\theta).$$

Part 3 - Hints and Helper

Use `x.new_zeros` or parameter-based constructors for states. Avoid storing fixed-device states as ordinary attributes.

Part 4 - Python code skeleton

```
import torch

def E10(model, data, config):
    device = torch.device("cuda")
    model = model.to(device)
    batch = move_batch_to_device(data.batch, device, non_blocking=True)
    h0 = batch.inputs.new_zeros(...)
    c0 = batch.inputs.new_zeros(...)
    outputs = model(batch.inputs, state=(h0, c0))
    loss = compute_loss(outputs, batch.targets)
    loss.backward()
    audit_model_batch_state_and_gradient_devices(model, batch, outputs, ...)
    return compare_cpu_gpu_architecture_parity(... )
```

16.3.12 Exercise 11 - Benchmark LSTM Across Sequence Lengths and Find Practical GPU Limits

Part 1 - Why this matters

Sequence length directly extends recurrent execution and training graph depth. Although gate calculations within each time step use parallel matrix operations, time steps remain causally ordered.

The longest shape that executes once is not necessarily practical. A usable configuration needs memory headroom, stable repeated latency, acceptable throughput, and no numerical failure.

Sweep sequence lengths under fixed batch and model settings, then under a fixed-token budget. Measure step latency, tokens per second, peak memory, state magnitude, and failure boundary on the target GPU.

Part 2 - Mathematical form

$$\text{tokens/s} = BT/T_{step}, \qquad M_{train} = M_{fixed} + O(BTLH).$$

Part 3 - Hints and Helper

Warm up each unique shape and reset peak memory statistics. Stop safely before exhausting device memory.

Part 4 - Python code skeleton

```
import torch

def E11(model, data, config):
    reports = []
    for length in config.sequence_lengths:
        batch_size = choose_batch_size(length, policy=...)
        batch = make_sequence_batch(batch_size, length, ...).cuda()
        torch.cuda.reset_peak_memory_stats()
        result = benchmark_lstm_shape(
            model.cuda(), batch, warmup=..., repetitions=...,
            training=True, memory_headroom=...
        )
        reports.append({"length": length, "batch": batch_size, **result})
    return identify_practical_gpu_sequence_limit(reports)
```

16.3.13 Exercise 12 - Compare float32 and Mixed-Precision Stability and Find fp16 Gate Failures

Part 1 - Why this matters

Mixed precision can accelerate LSTM training and reduce activation memory, but gate sigmoids and accumulated cell updates can be sensitive to limited range and precision. Long sequences amplify small state differences.

Gradient scaling protects small backward values but does not prevent forward gate saturation, cell overflow, or cancellation. Stable short tests therefore do not guarantee stable training at deployment sequence lengths.

Compare float32, autocast without scaling, and autocast with dynamic scaling across normal and extreme inputs. Record speed, memory, gate saturation, state drift, skipped updates, nonfinite gradients, and validation quality.

Part 2 - Mathematical form

$$\widetilde{L} = sL, \qquad e_t = \|h_t^{32} - h_t^{mp}\|_\infty.$$

Part 3 - Hints and Helper

Unscale before clipping or gradient-norm measurement. Inspect gate preactivations and cell states, not only final loss.

Part 4 - Python code skeleton

```
import torch

def E12(model, data, config):
```

```
    reports = {}
    for mode in ["float32", "amp_unscaled", "amp_scaled"]:
        candidate = clone_initial_model(model).cuda()
        reports[mode] = train_precision_configuration(
            candidate,
            data,
            autocast_enabled=(mode != "float32"),
            grad_scaler=...,
            inspect_gates=True,
            ...
        )
    return identify_precision_sensitive_gate_cases(reports)
```

16.3.14 Exercise 13 - Wrap nn.LSTM in a Debuggable Module That Exposes States

Part 1 - Why this matters

Production code benefits from optimized nn.LSTM, while debugging requires visibility into outputs, final states, shape contracts, norms, and sometimes layer-level behavior. A wrapper can provide observability without replacing the fused recurrent implementation.

The interface should remain explicit about input layout, optional initial state, lengths, and returned diagnostic data. Diagnostics must be detachable so ordinary training does not retain large graphs or force synchronization unnecessarily.

Build a configurable wrapper that validates inputs, initializes states, returns task output, and optionally records safe summaries. Test train and inference modes, variable lengths, directions, layers, and CUDA placement.

Part 2 - Mathematical form

$$\mathcal{O} = (Y, h_n, c_n, \mathcal{S}), \qquad \mathcal{S} = \{\|Y\|, \|h_n\|, \|c_n\|, \text{shapes}\}.$$

Part 3 - Hints and Helper

Prefer scalar summaries over storing all activations. Make debug collection optional and avoid calling item inside timed training loops.

Part 4 - Python code skeleton

```
import torch
from torch import nn

class DebuggableLSTM(nn.Module):
    def __init__(self, **lstm_config):
        super().__init__()
        self.lstm = nn.LSTM(**lstm_config)
```

```
    def forward(self, x, state=None, debug=False):
        if state is None:
            state = initialize_state_from_input(self.lstm, x)
        validate_lstm_input_and_state(self.lstm, x, state)
        outputs, (h_n, c_n) = self.lstm(x, state)
        diagnostics = collect_safe_state_summaries(
            outputs, h_n, c_n, enabled=debug
        )
        return outputs, (h_n, c_n), diagnostics

def E13(model, data, config):
    return test_debuggable_wrapper_across_configurations(... )
```

16.3.15 Exercise 14 - Create Unit Tests for LSTM Shapes Across Configurations

Part 1 - Why this matters

Shape correctness is an architectural invariant that can be tested before expensive training. Parameterized unit tests prevent a later change in layers, directions, layout, or projection from silently invalidating downstream code.

Tests should derive expected shapes from configuration rather than repeating the same implementation assumptions. Edge cases such as batch one, time one, multi-layer bidirectional projection, and non-contiguous inputs expose fragile logic.

Build a configuration matrix and test inputs, outputs, h, c, classifier dimensions, dtype, and device. Add negative tests that confirm malformed states fail with understandable diagnostics.

Part 2 - Mathematical form

$$\text{shape}(Y) = (B, T, DH_{out}), \quad \text{shape}(h_n) = (DL, B, H_{out}), \quad \text{shape}(c_n) = (DL, B, H_{cell}).$$

Part 3 - Hints and Helper

Use one independent expected-shape function. Do not calculate expectations by running the model and copying returned dimensions.

Part 4 - Python code skeleton

```
import torch

def E14(model, data, config):
    cases = generate_lstm_configuration_matrix(
        batch_first=[False, True],
        bidirectional=[False, True],
        layers=[1, 3],
        projection=[0, ...],
```

```
        edge_shapes=...,
    )
    for case in cases:
        candidate, inputs, state = build_test_case(case)
        outputs, final_state = candidate(inputs, state)
        expected = independently_derive_expected_shapes(case)
        assert_all_shape_dtype_device_contracts(
            outputs, final_state, expected
        )
    run_negative_state_shape_tests(...)
    return ...
```

16.3.16 Exercise 15 - Validate LSTM Determinism and Reproducibility Across Runs and Machines

Part 1 - Why this matters

Reproducible LSTM training depends on model initialization, data order, worker seeds, dropout, CUDA algorithms, precision, and software environment. Fixing one random seed does not control all of these sources.

Exact equality may require slower deterministic algorithms and still remain environment-specific. Scientific reproducibility can instead define acceptable numerical and metric tolerances while recording the complete conditions.

Repeat forward and training runs locally, then compare saved benchmark records across machines. Locate first divergence, measure growth through time, and document performance cost and remaining sources of variation.

Part 2 - Mathematical form

$$\delta_t = \|h_t^{(a)} - h_t^{(b)}\|_\infty, \qquad \Delta M = |M^{(a)} - M^{(b)}|.$$

Part 3 - Hints and Helper

Capture Python, PyTorch, CUDA, cuDNN, driver, GPU, seed, and determinism settings. Save data split identifiers and sampler state.

Part 4 - Python code skeleton

```
import torch

def E15(model, data, config):
    environments = capture_reproducibility_environment(...)
    reports = []
    for deterministic in [False, True]:
        runs = []
        for repeat in range(...):
```

```
            configure_reproducibility(seed=..., deterministic=deterministic)
            runs.append(run_controlled_lstm_training(model, data, ...))
        comparison = compare_repeated_runs(
            runs, locate_first_divergence=True
        )
        reports.append(comparison)
    return export_cross_machine_reproducibility_record(environments, reports)
```

16.3.17 Exercise 16 - Stress Test Deep Stacked LSTM for Gradient and Memory Limits

Part 1 - Why this matters

Adding recurrent layers increases representational depth at every time step, but also lengthens gradient paths, stores more activations, and raises parameter and optimizer memory. Very deep stacks can become unstable or inefficient before they run out of theoretical capacity.

Failure may appear as vanishing lower-layer gradients, exploding upper-layer gradients, NaN values, throughput collapse, or GPU memory pressure. One final global gradient norm cannot reveal which layer failed first.

Increase depth gradually under fixed sequence and width settings. Record layer-wise gradient and activation norms, runtime, peak memory, convergence, and numerical failures, then test whether clipping, normalization, or residual connections alter the boundary.

Part 2 - Mathematical form

$$g_\ell = \|\nabla_{\theta_\ell} L\|_2, \qquad M = O(LBTH) + M_{parameters}.$$

Part 3 - Hints and Helper

Register layer-specific diagnostics and stop below total device exhaustion. Compare the same initial seeds and training budget.

Part 4 - Python code skeleton

```
import torch

def E16(model, data, config):
    reports = []
    for layers in config.deep_layer_counts:
        candidate = build_stacked_lstm(layers=layers, ...).cuda()
        result = guarded_deep_lstm_stress_test(
            candidate,
            data,
            log_layerwise_gradients=True,
            log_activations=True,
            memory_headroom=...,
            interventions=...,
```

```
        )
        reports.append({"layers": layers, **result})
    return identify_depth_instability_and_memory_limits(reports)
```

16.3.18 Exercise 17 - Compare LSTM and GRU Parameter Cost with Accuracy

Part 1 - Why this matters

LSTM uses four gate blocks and two recurrent states, while GRU uses three gate blocks and one state. This often makes GRU smaller and faster at equal hidden size, but the task may benefit from LSTM's separate cell memory.

Equal hidden size is not equal parameter count, and equal parameter count is not equal runtime. A fair report should present several matching policies rather than hiding this distinction.

Train both architectures on short- and long-dependency tasks across seeds. Compare parameters, peak memory, throughput, time-to-quality, final accuracy, and robustness, then state where extra LSTM cost produces measurable benefit.

Part 2 - Mathematical form

$$P_{GRU} \approx 3H(I+H), \qquad P_{LSTM} \approx 4H(I+H).$$

Part 3 - Hints and Helper

Compare equal hidden size, approximately equal parameters, and equal wall-clock budget. Tune both with equal search effort.

Part 4 - Python code skeleton

```
import torch

def E17(model, data, config):
    results = run_gru_lstm_comparison(
        tasks=[data.short_dependency, data.long_dependency],
        matching=["hidden_size", "parameter_count", "wall_clock"],
        seeds=config.seeds,
        equal_tuning_budget=True,
        measure_memory_and_throughput=True,
        ...
    )
    return document_parameter_accuracy_tradeoff(results)
```

16.3.19 Exercise 18 - Build an Architecture Selection Checklist for Depth, Width, and Directionality

Part 1 - Why this matters

LSTM depth, width, and directionality should follow the task's evidence rather than a preference for larger models. Each choice changes capacity, causality, state shape, latency, memory, and deployment behavior.

The smallest architecture meeting quality and reliability targets is often easier to train, debug, export, and serve. Bidirectionality must be rejected when future context is unavailable, regardless of offline validation gain.

Create a decision checklist using measured dependency length, data volume, online constraints, memory budget, latency target, and architecture sweeps. Define escalation and rollback rules so every added component has a reason.

Part 2 - Mathematical form

$$a^* = \arg\min_a C(a) \quad \text{s.t.} \quad Q(a) \geq Q_{min},\ L(a) \leq L_{max},\ M(a) \leq M_{max}.$$

Part 3 - Hints and Helper

Begin with a one-layer unidirectional baseline. Add width, depth, directionality, or projection one controlled change at a time.

Part 4 - Python code skeleton

```
import torch

def E18(model, data, config):
    evidence = collect_task_and_architecture_evidence(...)
    checklist = ArchitectureSelectionChecklist()
    checklist.add_dependency_and_causality_rules(evidence, ...)
    checklist.add_width_capacity_thresholds(evidence, ...)
    checklist.add_depth_stability_limits(evidence, ...)
    checklist.add_directionality_constraints(evidence, ...)
    checklist.add_projection_and_budget_rules(evidence, ...)
    checklist.add_escalation_and_rollback_tests(evidence, ...)
    return checklist.validate_and_export(...)
```

16.3.20 Exercise 19 - Produce an LSTM Architecture Engineering Report

Part 1 - Why this matters

An architecture report should explain why the selected LSTM has its particular task mapping, depth, width, directionality, layout, dropout, projection, state policy, precision, and device

configuration. Listing hyperparameters without evidence is not an engineering rationale.

Every choice must connect to controlled results and explicit constraints. The report should also preserve rejected alternatives and failure cases, because they define the operating boundary of the final design.

Synthesize shape tests, capacity curves, CUDA benchmarks, precision experiments, reproducibility records, stress tests, and GRU comparisons. Produce separate training and deployment recommendations with assumptions, limitations, and next validation steps.

Part 2 - Mathematical form

$$\text{requirement} \rightarrow \text{configuration} \rightarrow \text{evidence} \rightarrow \text{tradeoff} \rightarrow \text{decision}.$$

Part 3 - Hints and Helper

Require a traceable benchmark or test for every major configuration claim. Include raw configuration identifiers and environment metadata.

Part 4 - Python code skeleton

```
import torch

def E19(model, data, config):
    evidence = load_lstm01_experiment_results(...)
    report = ArchitectureEngineeringReport()
    report.add_task_mapping_and_shape_contracts(evidence, ...)
    report.add_depth_width_directionality_rationale(evidence, ...)
    report.add_dropout_projection_and_state_policy(evidence, ...)
    report.add_cuda_precision_and_reproducibility(evidence, ...)
    report.add_capacity_cost_and_gru_comparison(evidence, ...)
    report.add_failures_limits_and_next_tests(evidence, ...)
    report.validate_claim_traceability()
    return report.export(...)
```

16.4 LSTM02 - Backpropagation Through Time in LSTM

Models and Training

16.4.1 Exercise 00 - Confirm Gradients Flow Through Hidden and Cell-State Paths

Part 1 - Why this matters

LSTM backpropagation follows both the exposed hidden-state recurrence and the internal cell-state pathway. Looking only at parameter gradients cannot show which temporal route carried the learning signal.

The cell path contains an additive update with forget-gate retention, while the hidden path passes through output gating and nonlinear transformations. Both contribute to earlier states and gate parameters.

Manually unroll a tiny LSTM, retain gradients on every h and c state, and backpropagate from a late loss. Plot both gradient sequences and verify that removing either path changes earlier credit assignment.

Part 2 - Mathematical form

$$\frac{\partial L}{\partial c_t} = \frac{\partial L}{\partial c_{t+1}} \frac{\partial c_{t+1}}{\partial c_t} + \frac{\partial L}{\partial h_t} \frac{\partial h_t}{\partial c_t}.$$

Part 3 - Hints and Helper

Call `retain_grad` on non-leaf states before backward. Keep hidden and cell traces in separate lists.

Part 4 - Python code skeleton

```
import torch

def E00(model, data, config):
    h_t, c_t = initialize_state(...)
    hidden_states, cell_states = [], []
    for x_t in iterate_time_steps(data.inputs):
        h_t, c_t = manual_lstm_step(x_t, h_t, c_t, ...)
        h_t.retain_grad(); c_t.retain_grad()
        hidden_states.append(h_t); cell_states.append(c_t)
    loss = compute_late_loss(hidden_states[-1], data.targets)
    loss.backward()
    return compare_hidden_and_cell_gradient_paths(hidden_states, cell_states)
```

16.4.2 Exercise 01 - Compare Temporal Gradient Norms with Vanilla RNN

Part 1 - Why this matters

LSTM is expected to improve long-range gradient flow, but that claim must be measured at equal temporal distances. Final parameter norms mix contributions from all time steps and hide decay along the sequence.

A fair comparison uses the same task, data, initialization scale, output loss, and dependency distances. LSTM may still decay when forget gates close or saturate, so improvement is not equivalent to constant gradients.

Retain state gradients for both models, compute norms by distance from the loss, and repeat across seeds. Compare slopes on a logarithmic scale and relate LSTM behavior to observed forget-gate values.

Part 2 - Mathematical form

$$G_m(d) = \mathbb{E}\|\partial L/\partial s_{T-d}^{(m)}\|_2.$$

Part 3 - Hints and Helper

Use cell-state gradients as the primary LSTM memory-path measurement and hidden gradients as a separate curve.

Part 4 - Python code skeleton

```
import torch

def E01(model, data, config):
    reports = {}
    models = {
        "rnn": build_rnn(...),
        "lstm": build_lstm(...),
    }
    for name, candidate in models.items():
        reports[name] = collect_temporal_gradient_norms(
            candidate, data, distances=config.distances, seeds=config.seeds
        )
    return compare_log_gradient_decay(reports, inspect_forget_gates=True)
```

16.4.3 Exercise 02 - Create Exploding Gradients and Apply Clipping Correctly

Part 1 - Why this matters

Gates reduce some instability but cannot prevent explosion from oversized recurrent weights, extreme learning rates, long sequences, or unstable state dynamics. One large update can corrupt every gate simultaneously.

Norm clipping rescales the complete parameter-gradient vector after backward and before the optimizer step. Clipping each tensor independently or clipping before mixed-precision unscaling changes the intended operation.

Create a controlled unstable configuration, record pre- and post-clipping norms, and compare thresholds with an unclipped run. Measure clipping frequency, loss stability, and whether an excessively small threshold prevents learning.

Part 2 - Mathematical form

$$g' = g \min\left(1, \frac{c}{\|g\|_2 + \epsilon}\right).$$

Part 3 - Hints and Helper

Unscale gradients before clipping. The value returned by `clip_grad_norm_` is the pre-clipping norm.

Part 4 - Python code skeleton

```
import torch

def E02(model, data, config):
    loss = run_unstable_lstm_forward(model, data, ...)
    loss.backward()
    pre_norm = global_gradient_norm(model.parameters())
    returned_norm = torch.nn.utils.clip_grad_norm_(
        model.parameters(), max_norm=config.clip_norm
    )
    post_norm = global_gradient_norm(model.parameters())
    optimizer.step()
    return verify_clipping(pre_norm, returned_norm, post_norm, ... )
```

16.4.4 Exercise 03 - Implement Truncated BPTT and Measure Its Tradeoffs

Part 1 - Why this matters

Full BPTT stores a graph across the complete sequence, which can exceed memory on long streams. Truncated BPTT processes shorter chunks while carrying detached hidden and cell states forward.

Detachment saves memory by cutting the backward path, but it also prevents losses in later chunks from teaching decisions before the boundary. Forward memory can persist even when credit assignment cannot cross the truncation.

Implement chunked training, sweep truncation lengths, and measure peak memory, throughput, gradient reach, and long-dependency accuracy. Reset states only at true sequence boundaries.

Part 2 - Mathematical form

$$(h_{k+1}, c_{k+1}) = \text{stopgrad}(F_\theta(x_{k:k+K}, h_k, c_k)).$$

Part 3 - Hints and Helper

Detach both components of the state tuple. Normalize loss consistently across the full logical sequence.

Part 4 - Python code skeleton

```
import torch

def E03(model, data, config):
```

```
    state = initialize_state(...)
    optimizer.zero_grad(set_to_none=True)
    for chunk, targets in split_sequence(data, length=config.truncation):
        outputs, state = model(chunk, state)
        loss = compute_chunk_loss(outputs, targets, normalization=...)
        loss.backward()
        optimizer.step(); optimizer.zero_grad(set_to_none=True)
        state = tuple(value.detach() for value in state)
    return measure_tbptt_memory_accuracy_tradeoff(... )
```

16.4.5 Exercise 04 - Debug Detach Strategies That Break Long-Term Learning

Part 1 - Why this matters

Detaching state at the wrong location can silently change the learning problem. Detaching every time step eliminates temporal credit assignment, while never detaching between independent chunks grows the graph and links unrelated updates.

Detaching only h or only c creates inconsistent behavior because the remaining component still carries a graph across the boundary. Using the deprecated data attribute or recreating tensors can also sever gradients unexpectedly.

Construct correct and faulty strategies, inspect graph connectivity, and train on a task whose dependency crosses chunk boundaries. Make the diagnostics identify over-detachment, missing detachment, and partial-state detachment.

Part 2 - Mathematical form

$$\partial L_{k+1}/\partial \theta_k = 0 \text{ across a deliberate detach boundary.}$$

Part 3 - Hints and Helper

Inspect `grad_fn` and saved-graph growth, then confirm the effect with a controlled dependency task.

Part 4 - Python code skeleton

```
import torch

def E04(model, data, config):
    strategies = {
        "correct_chunk": detach_both_after_chunk,
        "every_step": detach_both_every_step,
        "hidden_only": detach_hidden_only,
        "never": keep_entire_graph,
    }
    reports = {}
    for name, strategy in strategies.items():
```

```
        reports[name] = train_and_audit_detach_strategy(
            model, data, strategy=strategy, inspect_graph=True, ...
        )
    return diagnose_detach_failures(reports)
```

16.4.6 Exercise 05 - Finite-Difference Check a Tiny LSTM Cell

Part 1 - Why this matters

Automatic differentiation can propagate gradients through incorrect forward equations perfectly. Finite differences compare autograd with the actual numerical sensitivity of the implemented loss and can expose gate-order or derivative mistakes.

The method is sensitive to step size and floating-point precision. A step that is too large introduces truncation error, while a step that is too small is dominated by cancellation.

Use a tiny float64 cell and scalar loss, perturb selected parameters in both directions, and compare central differences with autograd. Sweep epsilon and report relative error per gate block.

Part 2 - Mathematical form

$$g_j^{FD} = \frac{L(\theta_j + \varepsilon) - L(\theta_j - \varepsilon)}{2\varepsilon}.$$

Part 3 - Hints and Helper

Disable stochastic behavior and avoid saturated test values. Restore every parameter after each perturbation.

Part 4 - Python code skeleton

```
import torch

def E05(model, data, config):
    cell = build_tiny_lstm_cell(dtype=torch.float64, seed=...)
    autograd_gradients = compute_autograd_gradients(cell, data, ...)
    finite_difference = central_difference_gradients(
        cell, data, epsilons=config.epsilons, parameter_indices=...
    )
    return compare_gradient_checks_by_gate(
        autograd_gradients, finite_difference, tolerance=...
    )
```

16.4.7 Exercise 06 - Monitor Gate Gradients and Detect Saturation

Part 1 - Why this matters

A low total gradient norm does not reveal which gate stopped learning. Input, forget, candidate, and output blocks share concatenated parameters but serve different control roles.

Saturated sigmoids produce derivatives near zero, so a gate can receive upstream signal yet transmit little gradient through its preactivation. Candidate tanh saturation creates a related blockage.

Split parameter and preactivation gradients by gate, log norms and saturation rates, and align them with sequence events. Identify whether failure comes from absent upstream signal, saturated activation, or a disconnected graph.

Part 2 - Mathematical form

$$S_g = \Pr(a_g < -k \ \vee \ a_g > k), \qquad G_g = \|\nabla_{\theta_g} L\|_2.$$

Part 3 - Hints and Helper

Use hooks on preactivations in a manual or instrumented cell. Keep PyTorch gate order explicit.

Part 4 - Python code skeleton

```
import torch

def E06(model, data, config):
    monitor = GateGradientMonitor(
        order=["input", "forget", "candidate", "output"]
    )
    with monitor.instrument(model):
        outputs = run_instrumented_lstm(model, data.inputs, ...)
        loss = compute_loss(outputs, data.targets)
        loss.backward()
    return monitor.summarize(
        include_parameter_norms=True, include_saturation=True,
        align_with_events=...
    )
```

16.4.8 Exercise 07 - Fix Forget-Gate Saturation with Initialization and LR

Part 1 - Why this matters

A forget gate driven near zero erases memory, while one pinned near one can retain obsolete state and receive a tiny sigmoid derivative. Both extremes can block useful adaptation.

Forget-bias initialization changes early retention timescale, but it is not a universal solution. Weight scale, input normalization, and learning rate determine how quickly preactivations move into saturation.

Create saturated configurations, inspect gate logits and gradients, then test controlled bias initialization and learning-rate adjustments. Compare retention, adaptation, gradient norms, and task accuracy.

Part 2 - Mathematical form

$$f_t = \sigma(a_f), \qquad \sigma'(a_f) = f_t(1 - f_t).$$

Part 3 - Hints and Helper

Change one factor at a time and log preactivation distributions, not only gate outputs.

Part 4 - Python code skeleton

```
import torch

def E07(model, data, config):
    variants = build_forget_gate_variants(
        biases=config.forget_biases,
        weight_scales=config.weight_scales,
        learning_rates=config.learning_rates,
    )
    results = run_gate_saturation_experiments(
        variants, data, gate="forget", track_logits=True, ...
    )
    return select_fix_without_destroying_adaptation(results)
```

16.4.9 Exercise 08 - Fix Input-Gate Saturation with Normalization or Initialization

Part 1 - Why this matters

Input-gate saturation can prevent new information from entering memory or allow every observation to write indiscriminately. Either behavior makes the cell insensitive to the task's actual write events.

Large feature scales and unstable affine initialization can push input-gate logits into extreme regions before learning begins. Normalization and controlled initialization can restore a useful operating range.

Compare raw, normalized, and differently initialized inputs while monitoring input-gate logits, candidate values, cell updates, and gradients. Confirm that the fix improves selective writing rather than merely reducing all activations.

Part 2 - Mathematical form

$$\Delta c_t = i_t \odot g_t, \qquad i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i).$$

Part 3 - Hints and Helper

Separate the contribution of i from the candidate g. Standardize features using training data only.

Part 4 - Python code skeleton

```
import torch

def E08(model, data, config):
    variants = build_input_gate_stability_variants(
        normalization=["none", "feature", "layer"],
        initializations=config.initializations,
        ...
    )
    results = run_gate_saturation_experiments(
        variants, data, gate="input", track_cell_updates=True, ...
    )
    return identify_selective_writing_fix(results)
```

16.4.10 Exercise 09 - Diagnose NaN Loss from Precision, LR, and Sequence Length

Part 1 - Why this matters

A NaN loss is usually the last visible symptom of an earlier non-finite input, cell state, gate logit, gradient, or parameter update. Recurrent reuse can amplify the initial problem many steps before the loss fails.

Precision mode, learning rate, initialization, and sequence length interact, so changing all of them at once may hide the cause. A diagnostic runner should capture the first non-finite stage and time step.

Sweep controlled configurations and stop immediately on failure. Record input range, gate logits, states, loss, gradients, scaler status, and the failing batch so the root cause can be reproduced.

Part 2 - Mathematical form

$$\text{finite}(z) \iff \neg\,\text{isnan}(z) \wedge \neg\,\text{isinf}(z).$$

Part 3 - Hints and Helper

Check tensors before and after every major stage. Preserve the first failure rather than continuing with corrupted parameters.

Part 4 - Python code skeleton

```
import torch

def E09(model, data, config):
    monitor = LSTMFiniteMonitor(fail_fast=True, save_first_failure=True)
    for precision, lr, length in experiment_grid(...):
        result = run_training_with_stage_checks(
            model, data, precision=precision, learning_rate=lr,
            sequence_length=length, monitor=monitor, ...
```

```
        )
        monitor.record_configuration_result(precision, lr, length, result)
    return monitor.root_cause_report()
```

16.4.11 Exercise 10 - Validate CUDA and CPU Gradient Parity

Part 1 - Why this matters

CPU and CUDA should implement the same LSTM derivatives, but floating-point order, fused kernels, and precision can create small differences. Exact equality is usually too strict for meaningful parity.

A controlled comparison requires identical parameters, inputs, loss reduction, initial states, and deterministic settings. It should inspect gate-block parameter gradients as well as outputs and state gradients.

Run a small float64 or float32 reference on both devices, compare forward states and backward gradients, and document tolerances. Repeat across sequence lengths to observe error accumulation.

Part 2 - Mathematical form

$$e_g = \max_j |g_j^{CPU} - g_j^{CUDA}|.$$

Part 3 - Hints and Helper

Clone state dictionaries rather than reinitializing models. Synchronize CUDA before reading timing, not as a substitute for parity checks.

Part 4 - Python code skeleton

```
import torch

def E10(model, data, config):
    cpu_model = build_controlled_lstm(device="cpu", dtype=..., seed=...)
    gpu_model = clone_model_and_state(cpu_model).cuda()
    cpu_result = forward_backward_snapshot(cpu_model, data.cpu_batch, ...)
    gpu_result = forward_backward_snapshot(gpu_model, data.gpu_batch, ...)
    return compare_forward_state_and_named_gradients(
        cpu_result, move_snapshot_to_cpu(gpu_result), atol=..., rtol=...
    )
```

16.4.12 Exercise 11 - Profile BPTT Memory and Graph-Retention Bottlenecks

Part 1 - Why this matters

LSTM training memory includes parameters, optimizer states, saved activations for every time step and layer, temporary workspaces, and any graphs accidentally retained by logging

code. Peak memory alone cannot identify the owner.

Repeated backward with retained graphs, storing loss tensors, or carrying attached states across batches can cause memory to grow over time. Reserved CUDA memory may remain high without indicating a true live-tensor leak.

Profile allocated and peak memory across forward, backward, and update regions while varying sequence length. Compare healthy and intentionally retained graphs, then locate activation and ownership bottlenecks.

Part 2 - Mathematical form

$$M_{peak} = M_{parameters} + M_{optimizer} + M_{saved\ activations} + M_{workspace}.$$

Part 3 - Hints and Helper

Store loss.item or detached summaries rather than loss tensors. Distinguish allocated from reserved CUDA memory.

Part 4 - Python code skeleton

```
import torch

def E11(model, data, config):
    reports = {}
    patterns = [
        "healthy", "retain_graph", "stored_losses", "attached_state"
    ]
    for retention in patterns:
        torch.cuda.reset_peak_memory_stats()
        reports[retention] = profile_bptt_memory_regions(
            model, data, retention_pattern=retention,
            record_allocation_timeline=True, ...
        )
    return identify_graph_and_activation_bottlenecks(reports)
```

16.4.13 Exercise 12 - Detect Mixed-Precision Overflow and Underflow

Part 1 - Why this matters

Mixed precision reduces memory and may accelerate BPTT, but long recurrent chains can produce gradients smaller or larger than float16 represents. Underflow silently creates zeros, while overflow can trigger skipped updates or non-finite states.

Dynamic scaling multiplies the loss before backward and unscales gradients before the optimizer step. It cannot repair forward cell-state overflow or gate saturation.

Compare float32, unscaled autocast, and scaled autocast across increasing lengths. Track scaler values, skipped steps, zero-gradient fraction, non-finite tensors, speed, memory, and validation quality.

Part 2 - Mathematical form

$$\widetilde{L} = sL, \qquad \nabla L = s^{-1}\nabla \widetilde{L}.$$

Part 3 - Hints and Helper

Unscale before clipping and true norm measurement. Inspect forward and backward failures separately.

Part 4 - Python code skeleton

```
import torch

def E12(model, data, config):
    reports = {}
    for mode in ["float32", "amp_unscaled", "amp_scaled"]:
        reports[mode] = train_with_precision_diagnostics(
            clone_initial_model(model).cuda(), data,
            mode=mode, sequence_lengths=config.lengths,
            track_scale=True, track_zero_gradients=True,
            track_nonfinite=True, ...
        )
    return compare_overflow_underflow_and_quality(reports)
```

16.4.14 Exercise 13 - Log Gate Gradient Norms with Autograd Hooks

Part 1 - Why this matters

Hooks provide measurements during backward without rewriting the optimizer. They can expose gate-specific parameter gradients, state gradients, and rare spikes that epoch averages conceal.

Careless hooks can retain tensors, force CPU synchronization, or change timing enough to distort profiling. Logging must detach summaries and remove handles when instrumentation ends.

Register hooks on concatenated gate parameters or manual gate tensors, split values by documented order, and aggregate batch statistics. Report distributions, trends, and anomaly events rather than printing every tensor.

Part 2 - Mathematical form

$$G_{g,k} = \|\nabla_{\theta_g} L_k\|_2.$$

Part 3 - Hints and Helper

Return detached scalar or CPU summaries from logging logic, and keep hook handles for removal.

Part 4 - Python code skeleton

```
import torch

def E13(model, data, config):
    logger = GateGradientHookLogger(order=..., sample_every=...)
    handles = logger.register(model)
    try:
        train_lstm(model, data, after_backward=logger.record_step, ...)
    finally:
        for handle in handles:
            handle.remove()
    return logger.summarize(distributions=True, anomalies=True)
```

16.4.15 Exercise 14 - Demonstrate and Fix the zero_grad Accumulation Bug

Part 1 - Why this matters

PyTorch adds new gradients to existing parameter gradients by design. If a normal LSTM training loop omits gradient clearing, every step combines unrelated batches and changes the effective update.

The loss may still decrease initially, making the bug difficult to detect. Gradient norms grow, updates depend on all previous batches, and clipping can hide the accumulation without fixing it.

Compare correct clearing, missing clearing, and intentional accumulation with proper normalization. Inspect gradients after consecutive identical batches and verify optimizer order explicitly.

Part 2 - Mathematical form

$$g_k^{stored} = g_{k-1}^{stored} + \nabla_\theta L_k.$$

Part 3 - Hints and Helper

Clear before backward for ordinary training. For intentional accumulation, scale losses and clear only at logical-batch boundaries.

Part 4 - Python code skeleton

```
import torch

def E14(model, data, config):
```

```
    reports = {}
    for policy in ["correct", "missing", "intentional_accumulation"]:
        candidate = clone_initial_model(model)
        reports[policy] = run_update_order_experiment(
            candidate, data, zero_grad_policy=policy,
            set_to_none=True, record_gradient_snapshots=True, ...
        )
    return detect_and_explain_gradient_accumulation(reports)
```

16.4.16 Exercise 15 - Analyze Batch Size, Gradient Noise, and Convergence

Part 1 - Why this matters

Batch size changes the variance of an LSTM gradient estimate and the number of updates per epoch. Small batches may provide useful noise but unstable recurrent updates; large batches are smoother yet require memory and may generalize differently.

Sequence batches also vary in valid-token count and padding waste, so nominal sample count does not fully describe effective batch size. Loss normalization must remain consistent.

Sweep batch sizes under equal sample or token budgets. Measure gradient variance, norm distributions, convergence, throughput, peak memory, and validation quality across seeds.

Part 2 - Mathematical form

$$\mathrm{Var}(\widehat{g}_B) \approx \Sigma / B.$$

Part 3 - Hints and Helper

Report valid tokens per update and padding efficiency. Adjust learning rate only in a documented secondary experiment.

Part 4 - Python code skeleton

```
import torch

def E15(model, data, config):
    reports = []
    for batch_size in config.batch_sizes:
        loader = build_length_aware_loader(data, batch_size=batch_size, ...)
        result = train_with_gradient_noise_measurement(
            clone_initial_model(model), loader,
            equal_token_budget=True, seeds=config.seeds, ...
        )
        reports.append({"batch_size": batch_size, **result})
    return analyze_noise_convergence_and_efficiency(reports)
```

16.4.17 Exercise 16 - Stress Very Long Sequences and Locate LSTM Forgetting

Part 1 - Why this matters

LSTM improves retention but does not provide unlimited memory. Forget values below one compound over time, finite state capacity causes interference, and optimization becomes harder as required dependency distance grows.

A long sequence is not necessarily a long-dependency task. The useful signal must be placed at a controlled distance from the target while distractors prevent local shortcuts.

Sweep dependency distance and sequence length independently. Record accuracy, cell probes, gate retention products, temporal gradients, runtime, and memory, then locate the first sustained forgetting boundary.

Part 2 - Mathematical form

$$R(t,T) = \prod_{k=t+1}^{T} f_k.$$

Part 3 - Hints and Helper

Use multiple seeds and require failure across neighboring distances. Compare with a no-memory baseline.

Part 4 - Python code skeleton

```
import torch

def E16(model, data, config):
    results = run_long_memory_grid(
        model,
        sequence_lengths=config.lengths,
        dependency_distances=config.distances,
        seeds=config.seeds,
        inspect_cell_and_forget_gates=True,
        ...
    )
    return identify_sustained_forgetting_boundary(results, baseline=...)
```

16.4.18 Exercise 17 - Compare Full and Truncated BPTT on Long Dependencies

Part 1 - Why this matters

Full BPTT can assign a late error to any earlier step in the sequence, while truncated BPTT deliberately limits this learning horizon. Forward state may carry information beyond

the truncation even though later loss cannot teach how it was encoded.

The comparison must keep model, data order, optimizer updates, loss normalization, and compute budget controlled. Otherwise differences may come from update frequency rather than gradient horizon.

Train both methods on tasks with dependencies below, near, and above the truncation length. Compare memory, throughput, accuracy, temporal gradients, and the ability to learn a new distant dependency.

Part 2 - Mathematical form

$$\partial L_t / \partial \theta_{t-d} = 0 \text{ under truncation when } d > K.$$

Part 3 - Hints and Helper

Align optimizer-step frequency or report it as a separate factor. Include a dependency shorter than K as a sanity check.

Part 4 - Python code skeleton

```
import torch

def E17(model, data, config):
    methods = {
        "full": build_full_bptt_trainer(...),
        "truncated": build_tbptt_trainer(chunk_length=config.truncation, ...),
    }
    results = run_dependency_length_comparison(
        methods, data, distances=config.distances,
        equal_loss_normalization=True, equal_update_policy=..., ...
    )
    return explain_full_vs_truncated_results(results)
```

16.4.19 Exercise 18 - Build an LSTM-Specific BPTT newline Debugging Checklist

Part 1 - Why this matters

LSTM BPTT failures can originate in gate saturation, state detachment, clipping order, mixed precision, loss masking, graph retention, or optimizer sequencing. A generic loss check cannot distinguish them.

An effective checklist turns each suspicion into an executable invariant or measurement and orders cheap checks before expensive profiling. It must preserve the first failing batch and configuration.

Build a staged diagnostic runner covering data, state graphs, gate logits, state magnitudes, temporal gradients, finite values, clipping, precision, and memory growth. Validate it against deliberately broken cases.

Part 2 - Mathematical form

$$\mathcal{D} = \mathcal{D}_{state} \cup \mathcal{D}_{gate} \cup \mathcal{D}_{gradient} \cup \mathcal{D}_{memory}.$$

Part 3 - Hints and Helper

Give each check a severity, expected range, evidence payload, and recommended next test.

Part 4 - Python code skeleton

```
import torch

def E18(model, data, config):
    checklist = LSTMBPTTChecklist()
    checklist.add("state_graph", audit_hidden_cell_graphs, severity="error")
    checklist.add("gate_saturation", measure_all_gate_saturation, ...)
    checklist.add("temporal_gradients", inspect_gradient_by_time, ...)
    checklist.add("finite_stages", verify_forward_backward_finite, ...)
    checklist.add("clip_order", verify_unscale_clip_step_order, ...)
    checklist.add("memory_growth", detect_retained_graphs, ...)
    checklist.add("optimizer_order", verify_zero_backward_step, ...)
    return checklist.run(model=model, data=data, config=config)
```

16.4.20 Exercise 19 - Produce the Complete LSTM BPTT Engineering Report

Part 1 - Why this matters

A BPTT report should connect cell equations to observed gradients, optimization stability, memory cost, and task performance. Isolated curves are not enough unless their experimental conditions and failure boundaries are clear.

The report must distinguish what LSTM gating improves from what clipping, truncation, precision controls, or data design changes. It should also retain unsuccessful interventions so future work does not repeat them blindly.

Synthesize hidden and cell paths, temporal norms, saturation, gradient checking, detach experiments, CUDA parity, memory profiling, precision, batch size, and long-sequence limits. End with practical rules and unresolved questions.

Part 2 - Mathematical form

$$\text{mechanism} \rightarrow \text{measurement} \rightarrow \text{failure} \rightarrow \text{fix} \rightarrow \text{verified boundary}.$$

Part 3 - Hints and Helper

Require every conclusion to reference a configuration and recorded experiment. Separate correctness rules from task-specific tuning.

Part 4 - Python code skeleton

```
import torch

def E19(model, data, config):
    evidence = load_lstm02_experiment_results(...)
    report = LSTMBPTTEngineeringReport()
    report.add_hidden_and_cell_gradient_paths(evidence, ...)
    report.add_saturation_and_gradient_check_results(evidence, ...)
    report.add_clipping_detach_and_tbptt_tradeoffs(evidence, ...)
    report.add_cuda_memory_and_precision_results(evidence, ...)
    report.add_batch_and_long_sequence_boundaries(evidence, ...)
    report.add_debugging_rules_failures_and_lessons(evidence, ...)
    report.validate_claim_traceability()
    return report.export(...)
```

16.5 LSTM03 - Sequence Data Handling for LSTM

Data Engineering for AI

16.5.1 Exercise 00 - Build Variable-Length Sequence Dataset and Validate Targets

Part 1 - Why this matters

Real sequence samples rarely share one length. A dataset must preserve each sample's valid time axis and its exact target relationship before batching introduces padding or reordering.

Alignment errors are especially deceptive because input and target tensors can remain valid while representing different events. Sequence identifiers and original indices provide evidence that survives collation.

Build samples with varied lengths and deterministic labels, return metadata, and verify first, middle, last, minimum-length, and maximum-length cases manually and automatically.

Part 2 - Mathematical form

$$X_i = (x_{i,1}, \ldots, x_{i,T_i}), \qquad Y_i = \psi(X_i), \qquad T_i \neq T_j.$$

Part 3 - Hints and Helper

Validate alignment before padding. Retain source indices or timestamps in each sample's metadata.

Part 4 - Python code skeleton

```
import torch
from torch.utils.data import Dataset

class VariableSequenceDataset(Dataset):
```

```
    def __init__(self, records):
        self.records = ...

    def __len__(self):
        return ...

    def __getitem__(self, index):
        sequence, target, metadata = ...
        validate_sample_alignment(sequence, target, metadata)
        return sequence, target, metadata

def E00(model, data, config):
    dataset = VariableSequenceDataset(data)
    return audit_dataset_edge_cases(dataset, ... )
```

16.5.2 Exercise 01 - Implement Padding and Masks and Prove Loss Ignores Padding

Part 1 - Why this matters

Padding converts variable-length samples into a rectangular batch, but padding values are not observations. If they enter loss or metrics, short sequences receive artificial targets and may dominate training.

A mask must be derived from trusted lengths and use the same temporal alignment as outputs and targets. Choosing a padding value that also appears in real data makes value-based masking unsafe.

Create padded batches with several lengths, compute masked loss, perturb padding positions drastically, and prove the loss and valid predictions remain unchanged.

Part 2 - Mathematical form

$$m_{i,t} = \mathbf{1}[t < T_i], \qquad L = \frac{\sum_{i,t} m_{i,t} \ell_{i,t}}{\sum_{i,t} m_{i,t}}.$$

Part 3 - Hints and Helper

Build masks from lengths, not from comparing tensor values with the padding value.

Part 4 - Python code skeleton

```
import torch
from torch.nn.utils.rnn import pad_sequence

def E01(model, data, config):
    padded = pad_sequence(data.sequences, batch_first=True, padding_value=...)
    lengths = torch.tensor([len(x) for x in data.sequences])
    mask = torch.arange(padded.size(1))[None, :] < lengths[:, None]
```

```
    outputs = model(padded)
    loss = masked_loss(outputs, data.targets, mask, ...)
    perturbed = alter_padding_only(padded, mask, value=...)
    verify_padding_invariance(model, padded, perturbed, mask, loss, ...)
    return padded, lengths, mask, loss
```

16.5.3 Exercise 02 - Implement Packed Sequences and Validate Benefits

Part 1 - Why this matters

Packed sequences allow the LSTM to skip padded time steps instead of processing a full maximum-length rectangle. The benefit grows when batch lengths differ substantially.

Packing changes representation and requires correct lengths, ordering, and restoration. A faster result is meaningless unless valid outputs and final states agree with the padded reference.

Pack a batch, run the LSTM, unpack and restore order, then compare valid positions. Measure packing overhead, useful-token throughput, and peak memory across length distributions.

Part 2 - Mathematical form

$$\eta_{pad} = \frac{\sum_i T_i}{B \max_i T_i}.$$

Part 3 - Hints and Helper

Keep lengths on CPU when required and use `enforce_sorted` consistently with the chosen ordering.

Part 4 - Python code skeleton

```
import torch
from torch.nn.utils.rnn import pack_padded_sequence, pad_packed_sequence

def E02(model, data, config):
    padded, lengths, targets = ...
    packed = pack_padded_sequence(
        padded, lengths.cpu(), batch_first=True, enforce_sorted=...
    )
    packed_outputs, state = model(packed)
    outputs, restored_lengths = pad_packed_sequence(
        packed_outputs, batch_first=True
    )
    verify_packed_valid_outputs(outputs, state, padded, lengths, ...)
    return benchmark_packing_benefits(... )
```

16.5.4 Exercise 03 - Debug Sorting, Length, and Packed-Output Bugs

Part 1 - Why this matters

Packing bugs can reorder samples, truncate valid steps, include padding as data, or attach outputs to the wrong targets. Most produce legal tensors and misleading learning rather than an immediate exception.

Sorting lengths requires applying the same permutation to inputs, targets, metadata, and carried states, then restoring original order after processing. Off-by-one lengths alter which event becomes the final state.

Create controlled faulty variants and build audits for monotonic sorting, length bounds, permutation inversion, valid-token counts, and output-target identity.

Part 2 - Mathematical form

$$X' = PX, \quad Y' = PY, \quad \widehat{Y} = P^{-1}\widehat{Y}'.$$

Part 3 - Hints and Helper

Use sample IDs embedded in a feature channel so reordering mistakes are visible.

Part 4 - Python code skeleton

```
import torch

def E03(model, data, config):
    variants = create_packing_failure_cases(
        faults=["unsorted", "wrong_lengths", "targets_not_sorted",
                "missing_inverse_permutation", "off_by_one"],
        ...
    )
    reports = {}
    for name, batch in variants.items():
        reports[name] = audit_packed_pipeline(
            model, batch, verify_ids=True, verify_valid_counts=True
        )
    return diagnose_packing_failures(reports)
```

16.5.5 Exercise 04 - Compare Padded and Packed Training on CUDA

Part 1 - Why this matters

Packed execution removes padding work, but sorting, packing, unpacking, and CPU length handling add overhead. GPU speedup therefore depends on length variance, model size, and batch shape.

Padded computation may be faster for nearly uniform lengths because it uses regular dense kernels. A fair comparison should measure complete training steps and useful tokens, not only the recurrent call.

Benchmark both paths on identical samples with low, medium, and high length variance. Confirm valid-output parity, then report latency, throughput, memory, padding efficiency, and preprocessing overhead.

Part 2 - Mathematical form

$$S = \frac{T_{padded}}{T_{packed}}, \qquad Q = \frac{\sum_i T_i}{T_{step}}.$$

Part 3 - Hints and Helper

Warm up each representation and synchronize CUDA timing boundaries. Include sorting and packing in end-to-end time.

Part 4 - Python code skeleton

```
import torch

def E04(model, data, config):
    reports = {}
    for distribution in config.length_distributions:
        batch = build_controlled_variable_batch(distribution, ...)
        padded_result = benchmark_training_path(
            model, batch, path="padded", ...
        )
        packed_result = benchmark_training_path(
            model, batch, path="packed", ...
        )
        reports[distribution] = {
            "padded": padded_result,
            "packed": packed_result,
        }
        verify_valid_training_parity(reports[distribution], ...)
    return summarize_gpu_packing_tradeoffs(reports)
```

16.5.6 Exercise 05 - Build a Clean Variable-Length Collate Function

Part 1 - Why this matters

A collate function defines how independent samples become one batch. It is responsible for padding, lengths, masks, targets, metadata, and any ordering used for packing.

Mixing these responsibilities across the training loop creates duplicated assumptions and makes alignment difficult to test. A clean batch object provides one documented shape contract.

Implement collation for empty-edge handling, scalar or sequential targets, sorting, inverse permutation, and metadata. Test it on batch one, equal lengths, extreme variance, and the final incomplete batch.

Part 2 - Mathematical form

$$\text{collate}(\{(X_i, Y_i)\}) = (X_{pad}, Y_{pad}, T, m, P, P^{-1}).$$

Part 3 - Hints and Helper

Return explicit lengths and masks even if the first model version uses only one of them.

Part 4 - Python code skeleton

```
import torch
from torch.nn.utils.rnn import pad_sequence

def variable_length_collate(samples):
    sequences, targets, metadata = zip(*samples)
    lengths = torch.tensor([len(x) for x in sequences])
    order = choose_sort_order(lengths, ...)
    inverse = invert_permutation(order)
    padded_inputs = pad_sequence(..., batch_first=True, padding_value=...)
    padded_targets = ...
    mask = ...
    return SequenceBatch(
        inputs=padded_inputs, targets=padded_targets, lengths=lengths,
        mask=mask, order=order, inverse_order=inverse, metadata=metadata
    )

def E05(model, data, config):
    return test_collate_edge_cases(variable_length_collate, data, ... )
```

16.5.7 Exercise 06 - Implement Masked Sequence Loss and Match Manual Computation

Part 1 - Why this matters

Loss reduction determines how samples and time steps influence optimization. Averaging over the padded rectangle underweights long sequences and counts invalid positions unless masking is applied before reduction.

Token-average and sequence-average losses answer different questions. The former weights every valid time step equally, while the latter gives every sequence equal total influence.

Implement both modes from unreduced losses, compare with explicit Python loops, and test all-padding rejection, one-token samples, class weights, and padding perturbations.

Part 2 - Mathematical form

$$L_{token} = \frac{\sum m\ell}{\sum m}, \qquad L_{seq} = \frac{1}{B}\sum_i \frac{\sum_t m_{it}\ell_{it}}{T_i}.$$

Part 3 - Hints and Helper

Never multiply a scalar reduced loss by the mask. Request reduction none first.

Part 4 - Python code skeleton

```
import torch

def masked_sequence_loss(logits, targets, mask, mode="token"):
    element_loss = compute_unreduced_loss(logits, targets, ...)
    valid_loss = element_loss * mask.to(element_loss.dtype)
    if mode == "token":
        result = ...
    elif mode == "sequence":
        result = ...
    else:
        raise ValueError(...)
    return result

def E06(model, data, config):
    vectorized = masked_sequence_loss(...)
    manual = explicit_valid_position_loss(...)
    return verify_loss_equivalence(vectorized, manual, ... )
```

16.5.8 Exercise 07 - Implement Stateful Batching and Correct State Resets

Part 1 - Why this matters

Stateful batching carries hidden and cell states across consecutive chunks of the same logical stream. It can preserve context and avoid replaying history, but only when stream identity and ordering remain stable.

States must be reset at true sequence boundaries, reassigned when batch slots change streams, and detached between truncated updates. Leakage between users or records creates false context and invalid evaluation.

Build a state store keyed by stream ID, simulate continuation and termination, and verify resets, detachment, reordering, and partial-batch behavior with explicit sentinel states.

Part 2 - Mathematical form

$$s_{k+1}^{(j)} = \begin{cases} \text{stopgrad}(s_{end}^{(j)}), & \text{continuation}, \\ 0, & \text{new stream}. \end{cases}$$

Part 3 - Hints and Helper

Do not associate state only with batch position. Use stable stream identifiers.

Part 4 - Python code skeleton

```
import torch

def E07(model, data, config):
    state_store = LSTMStateStore()
    for batch in data.stream_batches:
        state = state_store.gather(
            batch.stream_ids, reset_mask=batch.new_sequence_mask, ...
        )
        outputs, next_state = model(batch.inputs, state)
        state_store.update(
            batch.stream_ids,
            detach_state(next_state),
            finished_mask=batch.finished_mask,
        )
    return audit_state_reset_and_leakage(state_store, ... )
```

16.5.9 Exercise 08 - Prevent Time-Series Leakage with Chronological Splits

Part 1 - Why this matters

Randomly splitting overlapping time-series windows allows nearly identical histories or future-derived preprocessing statistics to appear in training and validation. This inflates metrics without improving real forecasting.

The raw timeline should be divided before windows are constructed, with explicit treatment of context near boundaries. Scalers, imputers, and feature transformations must fit on training time only.

Create chronological train, validation, and test periods, then build windows within permitted boundaries. Audit timestamp overlap, target horizons, scaler fit range, and duplicate source observations.

Part 2 - Mathematical form

$$\max t_{train} < \min t_{val} < \min t_{test}.$$

Part 3 - Hints and Helper

Decide whether validation inputs may use earlier training context, but never allow validation targets or statistics into training.

Part 4 - Python code skeleton

```
import torch

def E08(model, data, config):
    raw_splits = chronological_split(data.series, boundaries=...)
    scaler = fit_preprocessor(raw_splits.train)
    transformed = transform_all_splits(scaler, raw_splits)
    windows = {
        name: build_windows(split, horizon=..., context_policy=...)
        for name, split in transformed.items()
    }
    audit_temporal_leakage(raw_splits, windows, scaler, ...)
    return windows
```

16.5.10 Exercise 09 - Debug Instability from Masks and Loss Reduction

Part 1 - Why this matters

A masking bug changes gradient scale as batch length distribution changes. Dividing by batch size, padded width, or zero valid counts can create inconsistent updates and apparent instability.

Metrics may look reasonable while loss uses a different mask, dtype, or axis. Broadcasting errors can repeat a mask over features or classes unintentionally.

Construct correct and faulty reductions, compare gradient norms across length distributions, and audit numerator, denominator, broadcast shape, and valid-token counts. Make every failure produce a specific diagnosis.

Part 2 - Mathematical form

$$\nabla L \propto \frac{1}{Z}\sum_{i,t} m_{it}\nabla \ell_{it}, \quad Z \in \{BT, \sum m, B\}.$$

Part 3 - Hints and Helper

Log the effective denominator and valid count for every batch during debugging.

Part 4 - Python code skeleton

```
import torch

def E09(model, data, config):
    variants = build_mask_reduction_variants(
        faults=["mean_over_padding", "wrong_axis", "broadcast",
                "zero_denominator", "metric_mask_mismatch"],
        ...
    )
    reports = {}
```

```
    for name, loss_fn in variants.items():
        reports[name] = analyze_loss_scale_and_gradients(
            model, data, loss_fn, log_denominator=True, ...
        )
    return diagnose_masking_instability(reports)
```

16.5.11 Exercise 10 - Move the Sequence Pipeline to CUDA Efficiently

Part 1 - Why this matters

Variable-length batches include inputs, targets, masks, and sometimes metadata, while lengths may remain on CPU for packing. Moving tensors repeatedly inside the time loop wastes transfer bandwidth and creates synchronization.

Pinned host memory and non-blocking copies can overlap transfer with compute only when the pipeline is structured correctly. Converting dtype or layout after transfer may create extra device allocations.

Move each complete batch once, keep required CPU lengths explicit, and benchmark data wait, transfer, packing, and compute regions. Compare baseline, pinned, and prefetched strategies for parity and throughput.

Part 2 - Mathematical form

$$T_{step} = T_{wait} + T_{copy} + T_{pack} + T_{compute}.$$

Part 3 - Hints and Helper

Do not move metadata strings to CUDA. Preserve a CPU lengths tensor if the packing API requires it.

Part 4 - Python code skeleton

```
import torch

def E10(model, data, config):
    loader = build_loader(data, pin_memory=True, num_workers=...)
    reports = {}
    for strategy in ["baseline", "single_copy", "prefetch"]:
        reports[strategy] = benchmark_sequence_transfer_pipeline(
            model,
            loader,
            strategy=strategy,
            non_blocking=True,
            keep_lengths_on_cpu=True,
            ...
        )
    return compare_transfer_overhead_and_parity(reports)
```

16.5.12 Exercise 11 - Benchmark DataLoader Workers for Variable Sequences

Part 1 - Why this matters

Variable-length collation can include sorting, padding, tokenization, decoding, and augmentation, making CPU preparation a major bottleneck. More workers can overlap work but also increase process, memory, and communication costs.

The best worker count depends on storage, CPU cores, batch complexity, pinned memory, and model speed. A GPU utilization increase is not useful if total throughput or stability worsens.

Sweep worker counts with and without persistent workers and pinned memory. Measure first-epoch startup, steady-state batches per second, data wait, memory, GPU idle time, and failed-worker behavior.

Part 2 - Mathematical form

$$Q(w) = N_{batches}/T_{epoch}(w).$$

Part 3 - Hints and Helper

Benchmark multiple epochs because persistent workers mainly affect transitions after the first epoch.

Part 4 - Python code skeleton

```
import torch

def E11(model, data, config):
    reports = []
    for workers in config.worker_counts:
        loader = build_loader(
            data,
            num_workers=workers,
            pin_memory=...,
            persistent_workers=(workers > 0),
            collate_fn=variable_length_collate,
        )
        result = benchmark_loader_epochs(loader, model=model, epochs=..., ...)
        reports.append({"workers": workers, **result})
    return select_stable_worker_configuration(reports)
```

16.5.13 Exercise 12 - Accumulate Gradients for Long Sequences with Limited Memory

Part 1 - Why this matters

Gradient accumulation divides a logical batch into smaller micro-batches so long-sequence activations fit on the GPU. It lowers peak memory but repeats launches and delays optimizer updates.

Equivalence requires weighting each micro-batch by its valid-token count. Averaging each micro-batch equally gives short and long batches incorrect influence, and clipping must occur only after accumulation completes.

Compare accumulated and physical large batches on a deterministic case, then benchmark realistic variable lengths. Measure gradient parity, memory, throughput, and convergence.

Part 2 - Mathematical form

$$L = \frac{\sum_k \sum_{i,t \in B_k} m_{it} \ell_{it}}{\sum_k \sum_{i,t \in B_k} m_{it}}.$$

Part 3 - Hints and Helper

Accumulate loss sums divided by the total logical valid-token count, then clip and step once.

Part 4 - Python code skeleton

```
import torch

def E12(model, data, config):
    optimizer.zero_grad(set_to_none=True)
    total_tokens = sum(batch.mask.sum() for batch in data.micro_batches)
    for batch in data.micro_batches:
        outputs = model(batch.inputs, ...)
        loss_sum = masked_loss(
            outputs, batch.targets, batch.mask, reduction="sum"
        )
        (loss_sum / total_tokens).backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), ...)
    optimizer.step()
    return compare_with_physical_large_batch(... )
```

16.5.14 Exercise 13 - Validate Reproducibility with Variable-Length Batches

Part 1 - Why this matters

Fixed model seeds are insufficient when sample shuffling, bucket formation, worker scheduling, dropout, and CUDA algorithms affect batch content and operation order. Variable lengths amplify these differences by changing padding and packing patterns.

Reproducibility should capture sampler state, worker seeds, bucket policy, permutations, and environment metadata. Exact equality and tolerance-based experimental repeatability should be reported separately.

Repeat runs with deterministic and ordinary settings, compare batch manifests, losses, states, and final metrics, and locate the first divergence. Save enough information to replay one epoch exactly.

Part 2 - Mathematical form

$$\mathcal{B}_e = \{(id_i, T_i, P_i)\}_{i=1}^{N}.$$

Part 3 - Hints and Helper

Hash the ordered sample IDs and lengths of every batch. Seed workers through an explicit initialization function.

Part 4 - Python code skeleton

```
import torch

def E13(model, data, config):
    runs = []
    for repeat in range(...):
        configure_reproducibility(seed=..., workers=True, cuda=True)
        loader = build_reproducible_bucketed_loader(data, generator=..., ...)
        runs.append(train_and_record_batch_manifest(model, loader, ...))
    return compare_manifests_states_and_metrics(
        runs, locate_first_divergence=True, environment=...
    )
```

16.5.15 Exercise 14 - Format Teacher-Forcing Data newline for LSTM Seq2Seq

Part 1 - Why this matters

Teacher forcing feeds the previous ground-truth target to the decoder while predicting the next target. This requires two shifted target views with explicit beginning and end tokens.

An off-by-one mistake can train the decoder to copy its current input rather than predict the future. Padding masks for decoder inputs and labels may also differ at terminal positions.

Build encoder inputs, decoder inputs, labels, lengths, and masks from variable target sequences. Verify each position manually and compare teacher-forced inputs with autoregressive inference requirements.

Part 2 - Mathematical form

$$D_{in} = (\langle BOS \rangle, y_1, \ldots, y_{T-1}), \quad D_{target} = (y_1, \ldots, y_T).$$

Part 3 - Hints and Helper

Show one complete sample before and after padding, including BOS, EOS, and every mask position.

Part 4 - Python code skeleton

```
import torch

def E14(model, data, config):
    encoder_inputs, target_sequences = ...
    decoder_inputs = [
        prepend_bos(y[:-1], token=...) for y in target_sequences
    ]
    decoder_targets = [
        append_eos_if_required(y, token=...)
        for y in target_sequences
    ]
    batch = collate_seq2seq_components(
        encoder_inputs, decoder_inputs, decoder_targets, ...
    )
    validate_teacher_forcing_shift(batch, sample_ids=...)
    return batch
```

16.5.16 Exercise 15 - Build a Complete Sequence-Data Checklist

Part 1 - Why this matters

Sequence-data correctness depends on connected contracts for alignment, lengths, padding, masks, packing, loss reduction, state resets, and split boundaries. Checking them independently can miss contradictions.

A useful checklist is executable and produces evidence rather than vague reminders. It should fail early on cheap invariants before launching expensive training.

Create staged checks for raw samples, collation, model boundaries, loss, state lifecycle, temporal splits, CUDA transfer, and reproducibility. Validate the checklist against deliberately corrupted pipelines.

Part 2 - Mathematical form

$$\mathcal{C} = \mathcal{C}_{sample} \cup \mathcal{C}_{batch} \cup \mathcal{C}_{state} \cup \mathcal{C}_{split}.$$

Part 3 - Hints and Helper

Give every check a severity, expected invariant, observed evidence, and next diagnostic action.

Part 4 - Python code skeleton

```
import torch

def E15(model, data, config):
    checklist = SequenceDataChecklist()
    checklist.add("sample_alignment", audit_raw_samples, severity="error")
    checklist.add("lengths_padding_masks", audit_collated_batch, ...)
```

```
    checklist.add("packing_permutation", audit_packing_roundtrip, ...)
    checklist.add("loss_reduction", audit_masked_loss, ...)
    checklist.add("state_reset", audit_state_boundaries, ...)
    checklist.add("temporal_split", audit_split_leakage, ...)
    checklist.add("cuda_transfer", audit_device_pipeline, ...)
    return checklist.run(model=model, data=data, config=config)
```

16.5.17 Exercise 16 - Detect and Fix Off-by-One Target Alignment

Part 1 - Why this matters

A one-step shift can make an LSTM reconstruct the present, predict the wrong future, or receive information about its own target. Correlated sequences may still yield low loss, hiding the error.

Tensor shapes cannot detect semantic time. Alignment must be verified with timestamps, source indices, or synthetic monotonic values whose correct target is known.

Create correct, zero-shift, early, and late variants, then audit input-target index equations and plot sample pairs. Make the test identify the exact displacement rather than only report failure.

Part 2 - Mathematical form

$$X_i = (s_i, \ldots, s_{i+w-1}), \qquad Y_i = (s_{i+w+g-1}, \ldots).$$

Part 3 - Hints and Helper

Audit alignment before scaling or shuffling. Retain original timestamps in metadata.

Part 4 - Python code skeleton

```
import torch

def E16(model, data, config):
    variants = create_alignment_variants(
        data, shifts={"correct": ..., "zero": 0, "early": -1, "late": 1}
    )
    reports = {}
    for name, dataset in variants.items():
        reports[name] = infer_and_validate_temporal_offset(
            dataset, expected_offset=config.target_offset, ...
        )
    return diagnose_off_by_one_alignment(reports)
```

16.5.18 Exercise 17 - Create a Task That Fails Without Packing

Part 1 - Why this matters

Padding should be harmless when masks and final-state selection are correct, but a naive many-to-one model may treat the final padded position as the sequence summary. Longer padding then overwrites the representation of short samples.

Packing makes the returned final state correspond to each sample's true final time step. The experiment must isolate this correction from unrelated differences in model parameters or training budget.

Create variable-length sequences whose label depends on the last valid event, add adversarial padding, and compare naive padded, corrected gathered, and packed models. Prove that packing or correct indexing restores invariance.

Part 2 - Mathematical form

$$y_i = \psi(x_{i,T_i}), \qquad x_{i,t} = p \text{ for } t > T_i.$$

Part 3 - Hints and Helper

Use the same initial weights and ensure the padding value strongly conflicts with the label rule.

Part 4 - Python code skeleton

```
import torch

def E17(model, data, config):
    task = build_last_valid_token_task(
        variable_lengths=True, adversarial_padding=True, ...
    )
    variants = {
        "naive_padded": build_last_column_classifier(...),
        "gathered": build_last_valid_classifier(...),
        "packed": build_packed_classifier(...),
    }
    results = run_controlled_training(
        variants, task, same_initial_weights=True
    )
    return prove_padding_failure_and_fix(results)
```

16.5.19 Exercise 18 - Stress Test a Large Variable-Length Pipeline

Part 1 - Why this matters

Large variable-length datasets stress storage, workers, collation, host memory, transfer, padding efficiency, recurrent compute, and allocator behavior simultaneously. Small tests cannot reveal queue starvation or rare extreme batches.

The test needs guarded scale increases and region-level timing so bottlenecks are attributed correctly. Corrupt samples, empty sequences, and heavy-tailed lengths should be handled explicitly.

Run increasing dataset, length, and worker configurations while tracking throughput, wait, padding waste, CPU and GPU memory, failures, and tail latency. Preserve the first anomalous batch for diagnosis.

Part 2 - Mathematical form

$$T_{epoch} = T_{IO} + T_{collate} + T_{copy} + T_{LSTM} + T_{update}.$$

Part 3 - Hints and Helper

Increase one scale variable at a time and retain resource headroom. Log batch length histograms.

Part 4 - Python code skeleton

```
import torch

def E18(model, data, config):
    monitor = VariablePipelineStressMonitor(...)
    for scale in config.stress_scales:
        loader = build_stress_loader(data, scale=scale, ...)
        result = run_guarded_training_epoch(
            model, loader, monitor=monitor,
            capture_first_anomaly=True, memory_headroom=...
        )
        monitor.record_scale_result(scale, result)
    return monitor.identify_pipeline_bottlenecks_and_limits()
```

16.5.20 Exercise 19 - Produce the LSTM Sequence-Data Engineering Report

Part 1 - Why this matters

The final report should prove that temporal relationships survive construction, batching, packing, transfer, state handling, and evaluation. Model accuracy cannot replace this evidence.

The report must include verified edge cases and failures because variable-length bugs often occur only at minimum lengths, incomplete batches, boundary windows, or stream changes.

Synthesize alignment audits, padding invariance, packing parity, masked-loss checks, state resets, leakage prevention, CUDA and DataLoader benchmarks, reproducibility, and stress limits. State the safe pipeline contract and remaining assumptions.

Part 2 - Mathematical form

raw time → sample → batch → model → loss → verified metric.

Part 3 - Hints and Helper

Link every claim to an executable check or benchmark record. Include a catalog of rejected faulty variants.

Part 4 - Python code skeleton

```
import torch

def E19(model, data, config):
    evidence = load_lstm03_experiment_results(...)
    report = SequenceDataEngineeringReport()
    report.add_sample_and_alignment_contracts(evidence, ...)
    report.add_padding_masking_and_packing_proofs(evidence, ...)
    report.add_state_and_temporal_split_rules(evidence, ...)
    report.add_cuda_loader_and_accumulation_results(evidence, ...)
    report.add_reproducibility_and_stress_limits(evidence, ...)
    report.add_edge_cases_failures_and_safe_contract(evidence, ...)
    report.validate_claim_traceability()
    return report.export(...)
```

16.6 LSTM04 - Language Modeling and Text Generation with LSTM

Models and Training

16.6.1 Exercise 00 - Build Character-Level Tokenization and Vocabulary

Part 1 - Why this matters

Character tokenization is the first contract between raw text and the model. Every character must map to one stable integer, and every integer must decode to the same character. This includes spaces, punctuation, line breaks, and any explicit unknown symbol.

A vocabulary can appear correct while silently changing order between runs or losing characters during normalization. Such an error changes the meaning of every training target, so the LSTM learns a corrupted language even though tensor shapes and loss computation remain valid.

Construct deterministic `stoi` and `itos` mappings, encode and decode representative samples, and test round-trip equality. Record the vocabulary size because it determines both the embedding input range and the output-logit dimension.

Part 2 - Mathematical form

$$\mathcal{V} = \text{sorted}(\text{unique}(C)), \quad e(c) = i, \quad d(i) = c, \quad d(e(c)) = c.$$

Part 3 - Hints and Helper

Do not build the vocabulary from a set without sorting it. Test whitespace and punctuation explicitly.

Part 4 - Python code skeleton

```
import torch
def build_vocabulary(text):
    characters = ...
    stoi = ...
    itos = ...
    return stoi, itos

def encode(text, stoi):
    return torch.tensor([...], dtype=torch.long)

def decode(indices, itos):
    return ...

def E00(model, data, config):
    stoi, itos = build_vocabulary(data)
    assert decode(encode(data, stoi), itos) == data
    return ...
```

16.6.2 Exercise 01 - Implement LSTM Language Model and Verify Logit Shapes

Part 1 - Why this matters

An LSTM language model converts token indices into embeddings, processes them through temporal memory, and produces one logit vector for every time step. The output is not a probability yet; it is an unrestricted score for every vocabulary item.

Shape mistakes are dangerous because flattening the batch and time axes can still produce a tensor accepted by the loss function. A model may therefore train against incorrectly aligned positions without raising an exception.

Implement the embedding, LSTM, and projection layers, then assert the shape after every stage. Test multiple batch sizes and sequence lengths so the implementation does not accidentally depend on one fixed configuration.

Part 2 - Mathematical form

$$E \in \mathbb{R}^{B \times T \times D}, \quad H = \text{LSTM}(E), \quad Z = HW_o + b_o \in \mathbb{R}^{B \times T \times |\mathcal{V}|}.$$

Part 3 - Hints and Helper

Use `batch_first=True`. Return the updated hidden and cell states for generation.

Part 4 - Python code skeleton

```
import torch
class LSTMLanguageModel(torch.nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_size):
        super().__init__()
        self.embedding = ...
        self.lstm = ...
        self.output = ...

    def forward(self, tokens, state=None):
        embedded = ...
        hidden, state = ...
        logits = ...
        return logits, state

def E01(model, data, config):
    logits, state = model(data)
    assert logits.shape == (..., ..., ...)
    return logits, state
```

16.6.3 Exercise 02 - Implement Cross-Entropy Loss and Confirm Target Alignment

Part 1 - Why this matters

Next-token prediction requires the input at position t to predict the target at position $t+1$. This one-position shift defines the learning problem and must be correct before any optimization result can be trusted.

An off-by-one target error often produces a decreasing loss because adjacent characters are statistically related. The apparent progress hides the fact that the model is solving the wrong temporal task.

Create shifted input-target windows, flatten logits and targets only after verifying their temporal correspondence, and compare the library loss with a manual log-softmax calculation on a tiny example.

Part 2 - Mathematical form

$$x_t = c_t, \quad y_t = c_{t+1}, \quad \mathcal{L} = -\frac{1}{BT}\sum_{b,t} \log p(y_{b,t} \mid x_{b,\le t}).$$

Part 3 - Hints and Helper

Print one decoded input and target pair. Logits use shape (B, T, V) but cross entropy receives (BT, V).

Part 4 - Python code skeleton

```
import torch
def make_next_token_batch(tokens, sequence_length):
    inputs = ...
    targets = ...
    return inputs, targets

def language_model_loss(logits, targets):
    flat_logits = ...
    flat_targets = ...
    return torch.nn.functional.cross_entropy(flat_logits, flat_targets)

def E02(model, data, config):
    inputs, targets = make_next_token_batch(data, ...)
    logits, _ = model(inputs)
    return language_model_loss(logits, targets)
```

16.6.4 Exercise 03 - Overfit a Tiny Text as a Correctness Test

Part 1 - Why this matters

Deliberately overfitting a tiny text is a diagnostic test, not a production objective. A model with enough capacity should nearly memorize a small deterministic sequence when the data, targets, loss, gradients, and optimizer are connected correctly.

Failure to overfit points to an implementation problem before it points to insufficient data. Common causes include misaligned targets, detached states, missing optimizer steps, incorrect vocabulary size, or a learning rate that prevents stable descent.

Train on one short repeated passage, track loss and token accuracy, and define an explicit success threshold. Save the configuration and seed so the diagnostic can be repeated after later code changes.

Part 2 - Mathematical form

$$\theta^* = \arg\min_{\theta} \mathcal{L}_{\text{tiny}}(\theta), \qquad \text{accuracy}_{\text{tiny}} \to 1.$$

Part 3 - Hints and Helper

Use a fixed seed and disable unnecessary regularization. Inspect one prediction sequence when the threshold is missed.

Part 4 - Python code skeleton

```
import torch
def train_tiny_overfit(model, inputs, targets, optimizer, steps):
    history = []
    for step in range(steps):
        optimizer.zero_grad(set_to_none=True)
        logits, _ = ...
        loss = ...
        loss.backward()
        optimizer.step()
        history.append(...)
    return history

def E03(model, data, config):
    history = train_tiny_overfit(model, ..., ..., ..., ...)
    assert history[-1]["accuracy"] >= ...
    return history
```

16.6.5 Exercise 04 - Generate Text Autoregressively and Analyze Repetition

Part 1 - Why this matters

Autoregressive generation turns the model into a closed feedback system: every sampled token becomes part of the next input. Small prediction biases can therefore accumulate into repetition, drift, or loss of grammatical structure.

LSTM memory may preserve longer patterns than a vanilla RNN, but it does not guarantee coherent generation. A model can still enter high-probability loops, especially when trained on limited text or decoded greedily.

Implement stateful token-by-token generation, compare LSTM and vanilla RNN outputs under the same prompt, and measure repeated n-grams. Separate model quality from decoding behavior by keeping vocabulary and sampling rules identical.

Part 2 - Mathematical form

$$\hat{c}_{t+1} \sim p_\theta(\cdot \mid c_{1:t}), \qquad (h_t, c_t) = \mathrm{LSTM}(c_t, h_{t-1}, c_{t-1}).$$

Part 3 - Hints and Helper

Use `torch.no_grad()` and preserve the returned state. Start with greedy decoding, then compare sampled output.

Part 4 - Python code skeleton

```
import torch
@torch.no_grad()
def generate(model, prompt, stoi, itos, max_new_tokens):
    state = None
```

```
    generated = ...
    for _ in range(max_new_tokens):
        logits, state = ...
        next_token = ...
        generated.append(next_token)
    return decode(generated, itos)

def E04(model, data, config):
    text = generate(model, ..., ..., ..., ...)
    return {"text": text, "repetition": ...}
```

16.6.6 Exercise 05 - Implement Temperature Sampling and Measure the Tradeoff

Part 1 - Why this matters

Temperature changes the sharpness of the next-token distribution without retraining the model. Low values concentrate probability on a few tokens, while high values flatten the distribution and permit less likely choices.

Extreme temperature can create misleading results. A nearly zero value behaves like greedy decoding and may repeat, whereas a very large value can destroy learned structure by approaching random sampling.

Implement temperature scaling on logits, evaluate a controlled grid of values with identical prompts and seeds, and report diversity together with quality indicators. Never compare only one generated sample per setting.

Part 2 - Mathematical form

$$p_i(\tau) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}, \qquad \tau > 0.$$

Part 3 - Hints and Helper

Divide logits before softmax. Reject non-positive temperatures and repeat each setting across several seeds.

Part 4 - Python code skeleton

```
import torch
def sample_with_temperature(logits, temperature):
    if temperature <= 0:
        raise ValueError(...)
    scaled = ...
    probabilities = ...
    return torch.multinomial(probabilities, num_samples=1)

def E05(model, data, config):
```

```
    results = {}
    for temperature in config["temperatures"]:
        samples = [...]
        results[temperature] = evaluate_samples(samples, ...)
    return results
```

16.6.7 Exercise 06 - Implement Top-k Sampling and Compare Stability

Part 1 - Why this matters

Top-k sampling restricts each decision to the k highest-logit tokens. It removes the long tail of unlikely tokens while preserving randomness among plausible candidates.

A fixed k ignores how certain the model is at each time step. The same value may be too restrictive after an obvious prefix and too permissive when the next token is uncertain.

Mask logits outside the selected set, renormalize safely, and compare several k values against temperature-only sampling. Measure invalid outputs, repetition, and unique n-grams across repeated generations.

Part 2 - Mathematical form

$$S_k = \mathrm{TopK}(z), \qquad p_i = \frac{e^{z_i/\tau}\mathbf{1}[i \in S_k]}{\sum_{j \in S_k} e^{z_j/\tau}}.$$

Part 3 - Hints and Helper

Use `torch.topk`; masked logits should be negative infinity before softmax.

Part 4 - Python code skeleton

```
import torch
def top_k_sample(logits, k, temperature=1.0):
    values, indices = torch.topk(logits, k=...)
    filtered = torch.full_like(logits, float("-inf"))
    filtered.scatter_(dim=-1, index=indices, src=values)
    probabilities = ...
    return ...

def E06(model, data, config):
    outputs = {k: generate_with_sampler(model, ..., k=k)
               for k in config["k_values"]}
    return compare_generation_sets(outputs, ...)
```

16.6.8 Exercise 07 - Implement Nucleus Sampling and Compare p Values

Part 1 - Why this matters

Nucleus sampling selects the smallest token set whose cumulative probability reaches a threshold p. Unlike fixed top-k sampling, the candidate count adapts to the model's uncertainty at each step.

The implementation requires correct sorting, cumulative sums, mask shifting, and index restoration. A one-position masking error can remove the threshold-crossing token or leave an empty candidate set.

Implement top-p filtering, verify it on hand-calculated distributions, and record the average nucleus size during generation. Compare coherence and diversity across several thresholds using matched prompts and random seeds.

Part 2 - Mathematical form

$$S_p = \min \left\{ S : \sum_{i \in S} p_{(i)} \geq p \right\}, \qquad 0 < p \leq 1.$$

Part 3 - Hints and Helper

Keep the first token above the cumulative threshold so the retained mass reaches p. Test $p = 1$ as an edge case.

Part 4 - Python code skeleton

```
import torch
def nucleus_sample(logits, p, temperature=1.0):
    probabilities = torch.softmax(..., dim=-1)
    sorted_probs, sorted_ids = torch.sort(probabilities, descending=True)
    cumulative = torch.cumsum(sorted_probs, dim=-1)
    remove = ...
    remove[..., 1:] = remove[..., :-1].clone()
    remove[..., 0] = False
    sorted_probs[remove] = 0
    return ...

def E07(model, data, config):
    return evaluate_top_p_grid(model, data, ... )
```

16.6.9 Exercise 08 - Debug Hidden and Cell State Carry-Over

Part 1 - Why this matters

Hidden and cell states carry context between time steps, but they must not cross unrelated sequences accidentally. Correct state handling depends on whether samples are contiguous chunks of one stream or independent documents.

Resetting too often destroys long dependencies; carrying state too far leaks information and couples unrelated batches. Failing to detach state during training can also retain an ever-growing computation graph.

Create tests for reset, carry, and detach policies, then inject an intentionally wrong policy and observe its effect. Verify state shape, device, batch size, and graph history at every boundary.

Part 2 - Mathematical form

$$\begin{aligned} (h_0, c_0) &= \mathbf{0} && \text{at a document boundary,} \\ (h, c) &\leftarrow (\text{detach}(h), \text{detach}(c)) && \text{at a truncation boundary.} \end{aligned}$$

Part 3 - Hints and Helper

Attach document identifiers to batches. Assert that carried state belongs to the same stream and batch layout.

Part 4 - Python code skeleton

```
import torch
def prepare_state(previous_state, batch_metadata, policy):
    if ...:
        return None
    hidden, cell = previous_state
    if policy["detach"]:
        hidden, cell = ...
    return hidden, cell

def E08(model, data, config):
    state = None
    audit = []
    for tokens, metadata in data:
        state = prepare_state(state, metadata, config)
        logits, state = model(tokens, state)
        audit.append(inspect_state(state, metadata, ...))
    return audit
```

16.6.10 Exercise 09 - Evaluate the Language Model with Perplexity

Part 1 - Why this matters

Perplexity converts average negative log-likelihood into an interpretable multiplicative scale. Lower perplexity means the model assigns more probability to the observed next tokens, but the value is meaningful only under the same tokenization and evaluation protocol.

A validation loss can be biased by padding, inconsistent reduction, leaked text, or unequal weighting of batches. Exponentiating a poorly aggregated loss produces a precise-looking but invalid perplexity.

Accumulate total negative log-likelihood and the exact number of valid tokens, then exponentiate once. Compare the result with a manual tiny example and report tokenization, split, and context length beside the metric.

Part 2 - Mathematical form

$$\mathrm{PPL} = \exp\left(\frac{\sum_{n=1}^{N} -\log p(y_n \mid x_{\le n})}{N}\right).$$

Part 3 - Hints and Helper

Do not average batch means when batches contain different token counts. Exclude masked tokens from both numerator and denominator.

Part 4 - Python code skeleton

```
import torch
@torch.no_grad()
def compute_perplexity(model, loader, pad_id=None):
    total_nll = 0.0
    total_tokens = 0
    for inputs, targets in loader:
        logits, _ = model(inputs)
        token_losses = ...
        valid = ...
        total_nll += ...
        total_tokens += ...
    return math.exp(total_nll / total_tokens)

def E09(model, data, config):
    return compute_perplexity(model, data, ... )
```

16.6.11 Exercise 10 - Train on CUDA and Benchmark Against CPU

Part 1 - Why this matters

CUDA can accelerate LSTM language-model training when the workload is large enough to amortize device transfers and kernel launches. Small models or short sequences may remain faster on CPU, so hardware choice must be measured rather than assumed.

An unfair benchmark may include data transfer in one path but exclude it in another, or time asynchronous GPU work without synchronization. These mistakes can report impossible speedups.

Build matched CPU and GPU runs with identical model weights, batches, warm-up, and step counts. Measure both training-step time and end-to-end throughput, while checking loss parity within a justified tolerance.

Part 2 - Mathematical form

$$\text{throughput} = \frac{B\,T\,N_{\text{steps}}}{t_{\text{elapsed}}}\ \text{tokens/s}, \qquad S = \frac{t_{\text{CPU}}}{t_{\text{GPU}}}.$$

Part 3 - Hints and Helper

Synchronize CUDA before starting and stopping the timer. Report transfer-inclusive and compute-only measurements separately.

Part 4 - Python code skeleton

```
import torch
def benchmark_training(device, model, batches, steps):
    model = model.to(device)
    warmup(model, batches, device, ...)
    synchronize_if_cuda(device)
    start = time.perf_counter()
    for step in range(steps):
        batch = move_batch(...)
        train_step(model, batch, ...)
    synchronize_if_cuda(device)
    return ...

def E10(model, data, config):
    cpu = benchmark_training("cpu", ..., data, ...)
    gpu = benchmark_training("cuda", ..., data, ...)
    return compare_benchmarks(cpu, gpu)
```

16.6.12 Exercise 11 - Profile GPU Utilization and Fix Pipeline Bottlenecks

Part 1 - Why this matters

Low GPU utilization often means the accelerator is waiting for tokenization, batching, file reads, or host-to-device copies. Increasing model complexity cannot solve a pipeline that delivers data too slowly.

A single utilization percentage is insufficient evidence. Short sampling intervals, synchronization gaps, padding waste, and CPU worker stalls must be related to the training timeline.

Profile representative steps, separate data time from compute time, and inspect operator and memory traces. Change one pipeline variable at a time and accept an optimization only when throughput improves without changing outputs.

Part 2 - Mathematical form

$$t_{\text{step}} = t_{\text{data}} + t_{\text{transfer}} + t_{\text{forward}} + t_{\text{backward}} + t_{\text{update}}.$$

Part 3 - Hints and Helper

Use a schedule with warm-up and active profiler steps. Compare `num_workers`, pinned memory, and non-blocking transfer settings.

Part 4 - Python code skeleton

```
import torch
def profile_training(model, loader, device, config):
    with torch.profiler.profile(
        activities=[...],
        schedule=torch.profiler.schedule(wait=..., warmup=..., active=...),
        record_shapes=True,
        profile_memory=True,
    ) as profiler:
        for step, batch in enumerate(loader):
            timed_train_step(model, batch, device, ...)
            profiler.step()
    return ...

def E11(model, data, config):
    baseline = profile_training(model, data, ..., config)
    optimized = ...
    return compare_profiles(baseline, optimized)
```

16.6.13 Exercise 12 - Implement Mixed Precision and Identify Instability

Part 1 - Why this matters

Mixed precision reduces activation memory and may increase throughput by using lower-precision arithmetic where it is safe. LSTM gates and long recurrent products can nevertheless be sensitive to underflow, overflow, and saturation.

A run may continue while gradients silently become non-finite or updates are repeatedly skipped. Monitoring only the loss misses the scale dynamics and the first layer where numerical failure begins.

Use automatic mixed precision and gradient scaling, log skipped steps and non-finite values, and compare against a float32 baseline. Test longer sequences deliberately to find the boundary where speed gains no longer justify instability.

Part 2 - Mathematical form

$$\hat{g} = s\,\nabla_\theta \mathcal{L}, \qquad g = \hat{g}/s, \qquad \theta \leftarrow \theta - \eta g.$$

Part 3 - Hints and Helper

Unscale before gradient clipping. Keep recurrent-state diagnostics in float32 when investigating instability.

Part 4 - Python code skeleton

```
import torch
def mixed_precision_step(model, batch, optimizer, scaler):
    optimizer.zero_grad(set_to_none=True)
```

```
    with torch.autocast(device_type="cuda", dtype=...):
        logits, _ = model(batch["inputs"])
        loss = ...
    scaler.scale(loss).backward()
    scaler.unscale_(optimizer)
    torch.nn.utils.clip_grad_norm_(model.parameters(), ...)
    scaler.step(optimizer)
    scaler.update()
    return inspect_numerics(loss, model, scaler, ...)

def E12(model, data, config):
    return run_precision_comparison(model, data, ... )
```

16.6.14 Exercise 13 - Compare LSTM and GRU for Language Modeling

Part 1 - Why this matters

LSTM and GRU use different gating structures to manage temporal information. LSTM maintains separate hidden and cell states, whereas GRU combines memory into a simpler state with fewer gates and often fewer parameters.

A comparison is invalid when hidden sizes, parameter budgets, initialization, training steps, or sampling procedures differ without explanation. Faster convergence can also hide worse final generalization.

Train both architectures under matched conditions. Compare parameter count, throughput, peak memory, validation perplexity, and generation metrics in equal-size and approximately equal-parameter experiments.

Part 2 - Mathematical form

$$N_{\mathrm{LSTM}} \approx 4H(D+H+1), \qquad N_{\mathrm{GRU}} \approx 3H(D+H+1).$$

Part 3 - Hints and Helper

Reuse the same embedding and output dimensions. Repeat runs with controlled seeds and report uncertainty.

Part 4 - Python code skeleton

```
import torch
def build_recurrent_model(kind, vocab_size, config):
    if kind == "lstm":
        recurrent = ...
    elif kind == "gru":
        recurrent = ...
    else:
        raise ValueError(...)
```

```
    return LanguageModel(...)

def E13(model, data, config):
    reports = {}
    for kind in ["lstm", "gru"]:
        candidate = build_recurrent_model(kind, ..., config)
        reports[kind] = train_and_evaluate(candidate, data, ...)
    return compare_models(reports)
```

16.6.15 Exercise 14 - Implement Truncated BPTT for Longer Contexts

Part 1 - Why this matters

Full backpropagation stores the computation graph across the entire sequence. Truncated BPTT limits the gradient horizon while still carrying numerical hidden and cell states into the next chunk.

Detaching too early prevents useful temporal credit assignment, while forgetting to detach causes memory growth across chunks. Shuffling chunks or resetting state at the wrong boundary also destroys their chronological meaning.

Split continuous streams into ordered chunks, update once per chunk, and detach both states only after the update boundary. Compare memory, throughput, and long-dependency accuracy across several truncation lengths.

Part 2 - Mathematical form

$$\nabla_\theta \mathcal{L}_k \text{ spans at most } K \text{ steps}, \qquad (h_k, c_k) \leftarrow \operatorname{detach}(h_k, c_k).$$

Part 3 - Hints and Helper

Keep document boundaries distinct from truncation boundaries. Measure peak allocated memory after synchronizing CUDA.

Part 4 - Python code skeleton

```
import torch
def train_truncated(model, stream, optimizer, chunk_length):
    state = None
    for inputs, targets, boundary in ordered_chunks(stream, chunk_length):
        if boundary:
            state = None
        optimizer.zero_grad(set_to_none=True)
        logits, state = model(inputs, state)
        loss = ...
        loss.backward()
        optimizer.step()
        state = tuple(s.detach() for s in state)
```

```
    return ...

def E14(model, data, config):
    return compare_truncation_lengths(model, data, ... )
```

16.6.16 Exercise 15 - Debug Divergence with Learning Rate, Clipping, and Gates

Part 1 - Why this matters

Divergence in an LSTM language model can begin in the optimizer, recurrent dynamics, or gate initialization. A large learning rate may cause explosive updates, while saturated gates can block corrective gradients before the loss becomes non-finite.

Changing several controls simultaneously may restore training without identifying the cause. That produces a fragile configuration and teaches nothing about which failure signal appeared first.

Instrument loss, gradient norms, parameter norms, gate statistics, and update size; then run a controlled diagnostic sequence. Adjust learning rate, clipping threshold, and forget-gate bias separately and record the first stable intervention.

Part 2 - Mathematical form

$$r_t = \frac{\eta \|g_t\|_2}{\|\theta_t\|_2 + \epsilon}, \qquad g'_t = g_t \min\left(1, \frac{c}{\|g_t\|_2}\right).$$

Part 3 - Hints and Helper

Stop immediately on non-finite values and save the failing batch. Initialize forget-gate bias deliberately rather than by fragile name assumptions.

Part 4 - Python code skeleton

```
import torch
def diagnostic_train_step(model, batch, optimizer, clip_norm):
    optimizer.zero_grad(set_to_none=True)
    logits, state = model(...)
    loss = ...
    if not torch.isfinite(loss):
        return capture_failure(...)
    loss.backward()
    before = collect_norms(model, ...)
    torch.nn.utils.clip_grad_norm_(model.parameters(), clip_norm)
    optimizer.step()
    after = collect_norms(model, ...)
    return build_diagnostic_record(loss, before, after, state, ...)

def E15(model, data, config):
    return run_one_factor_diagnostics(model, data, config, ... )
```

16.6.17 Exercise 16 - Build a Generation Evaluation Script

Part 1 - Why this matters

Generated text should be evaluated with several complementary measurements because no single automatic metric captures quality. Repetition measures degeneration, novelty detects copying, and length statistics reveal premature termination or uncontrolled output.

Metrics are easy to manipulate accidentally through tokenization, prompt inclusion, or inconsistent termination rules. Novelty against training text is also different from uniqueness within the generated set.

Create a deterministic evaluation script that separates prompts from continuations, uses fixed tokenization, and aggregates many samples. Report raw counts, normalized rates, distributions, and representative examples alongside model and decoding settings.

Part 2 - Mathematical form

$$R_n = 1 - \frac{|\operatorname{unique}(G_n)|}{|G_n|}, \qquad N_n = \frac{|G_n \setminus T_n|}{|G_n|}.$$

Part 3 - Hints and Helper

Define repeated and training n-gram sets explicitly. Store per-sample metrics before calculating aggregate summaries.

Part 4 - Python code skeleton

```
import torch
def repetition_rate(tokens, n):
    ngrams = ...
    return ...

def novelty_rate(tokens, training_ngrams, n):
    generated_ngrams = ...
    return ...

def evaluate_generation(samples, training_text, tokenizer, config):
    records = []
    for sample in samples:
        records.append({
            "repetition": ...,
            "novelty": ...,
            "length": ...,
        })
    return summarize(records, ...)

def E16(model, data, config):
    samples = generate_evaluation_set(model, ...)
    return evaluate_generation(samples, data, ..., config)
```

16.6.18 Exercise 17 - Prove LSTM Captures a Controlled Long Dependency

Part 1 - Why this matters

A controlled synthetic pattern isolates long-term memory from vocabulary complexity and real-text ambiguity. The target should depend on a marker placed many steps earlier while intervening tokens provide no shortcut.

If the dataset contains positional, frequency, or padding clues, either model may solve it without remembering the marker. A convincing experiment must remove these shortcuts and vary the dependency distance at evaluation time.

Generate balanced sequences, train matched vanilla RNN and LSTM models, and plot accuracy against dependency length. Probe hidden states or gates to connect performance differences to retained information rather than parameter count alone.

Part 2 - Mathematical form

$$y = x_{t-d}, \qquad d \in \{d_1, \ldots, d_m\}, \qquad I(y; x_{t-d}) > 0, \quad I(y; x_j) = 0 \text{ for distractors.}$$

Part 3 - Hints and Helper

Randomize marker position and distractors. Evaluate on distances both inside and beyond the training range.

Part 4 - Python code skeleton

```
import torch
def make_memory_dataset(samples, min_gap, max_gap, vocab_size):
    sequences = ...
    targets = ...
    distances = ...
    return sequences, targets, distances

def evaluate_by_distance(model, loader):
    correct = defaultdict(int)
    totals = defaultdict(int)
    for inputs, targets, distances in loader:
        predictions = ...
        update_distance_bins(correct, totals, predictions, targets, distances)
    return ...

def E17(model, data, config):
    rnn_report = train_memory_model("rnn", ...)
    lstm_report = train_memory_model("lstm", ...)
    return compare_distance_curves(rnn_report, lstm_report)
```

16.6.19 Exercise 18 - Stress Test Long-Prompt Text Generation

Part 1 - Why this matters

Long prompts increase recurrent computation, state-processing time, and memory used by token storage and evaluation. Although an LSTM state has fixed shape, processing the prompt remains sequential and latency grows with prompt length.

A stress test that measures only successful runs misses out-of-memory errors, latency spikes, synchronization effects, and degradation in generated quality. Generation time must also be separated into prompt ingestion and new-token decoding.

Sweep prompt length, batch size, hidden size, and precision under explicit resource limits. Capture peak memory, prompt throughput, per-token latency, failure type, and output-quality changes, then identify a safe operating envelope.

Part 2 - Mathematical form

$$t_{\text{total}} = t_{\text{prefill}} + \sum_{j=1}^{M} t_{\text{decode},j}, \qquad \text{tokens/s} = \frac{T + M}{t_{\text{total}}}.$$

Part 3 - Hints and Helper

Warm up before measurement and synchronize CUDA. Catch out-of-memory failures, clear references, and continue only with smaller validated configurations.

Part 4 - Python code skeleton

```
import torch
@torch.no_grad()
def stress_case(model, prompt, new_tokens, device):
    reset_peak_memory(device)
    synchronize_if_cuda(device)
    start = time.perf_counter()
    result = generate(model, prompt, ..., max_new_tokens=new_tokens)
    synchronize_if_cuda(device)
    return {
        "elapsed": ...,
        "peak_memory": ...,
        "output": result,
    }

def E18(model, data, config):
    records = []
    for case in build_stress_grid(config):
        records.append(run_or_capture_failure(model, case, ...))
    return determine_safe_envelope(records, ... )
```

16.6.20 Exercise 19 - Produce the Full LSTM Language Modeling Report

Part 1 - Why this matters

A final engineering report must connect data construction, architecture, optimization, decoding, evaluation, and hardware behavior. Reporting only the best perplexity hides the decisions and failures that determine whether the result can be reproduced.

Claims about sampling quality or LSTM memory require controlled evidence. Each comparison should identify fixed variables, changed variables, seeds, dataset boundaries, uncertainty, and the exact metric implementation.

Assemble verified tables, plots, generated examples, failure traces, and recommended operating settings. Separate observed evidence from interpretation, document unresolved limitations, and provide a reproducible path from raw text to final generation.

Part 2 - Mathematical form

$$\mathcal{R} = \{D, \mathcal{V}, M, \theta, H, S, Q, F\}, \quad Q = \{\mathrm{PPL}, R_n, N_n, L, t, m\}.$$

Part 3 - Hints and Helper

Include configuration hashes and checkpoint identifiers. Every recommendation should point to a recorded experiment.

Part 4 - Python code skeleton

```
import torch
def build_final_report(config, artifacts):
    required = [
        "data_audit", "model_summary", "training_curves",
        "perplexity", "sampling_comparison", "failure_log",
        "performance_benchmarks", "generation_examples",
    ]
    validate_artifacts(artifacts, required)
    report = {
        "scope": ...,
        "evidence": ...,
        "failures_and_fixes": ...,
        "recommendations": ...,
        "limitations": ...,
    }
    return report

def E19(model, data, config):
    artifacts = run_verified_experiment_suite(model, data, config, ...)
    return build_final_report(config, artifacts)
```

16.7 LSTM05 - Time Series Forecasting with LSTM

Models and Training

16.7.1 Exercise 00 - Build Multi-Feature Time Series Windows

Part 1 - Why this matters

A forecasting window converts a continuous multivariate series into supervised samples. Every input must contain only observations available at the forecast origin, and each target must begin at the requested future horizon.

A wrong offset can place the target inside the input or make the model predict a past value. Autocorrelation may then produce excellent metrics even though the deployment problem has been changed silently.

Construct windows with timestamps and source indices, then test the first, middle, and last valid samples manually. Assert feature order, lookback length, horizon distance, forecast length, and strict temporal separation before training.

Part 2 - Mathematical form

$$X_i = [\mathbf{x}_{i-L+1}, \ldots, \mathbf{x}_i], \qquad Y_i = [\mathbf{y}_{i+h}, \ldots, \mathbf{y}_{i+h+H-1}].$$

Part 3 - Hints and Helper

Retain the forecast-origin timestamp and original row indices in every sample.

Part 4 - Python code skeleton

```
import torch
def E00(model, data, config):
    prepared = build_multi_feature_windows(model, data, config, ...)
    result = validate_window_alignment(prepared, data, config, ...)
    assert result is not None
    return result
```

16.7.2 Exercise 01 - Implement Feature Scaling Strategy

Part 1 - Why this matters

Multivariate features may use radically different units and numerical ranges. Scaling prevents one large-magnitude variable from dominating the affine transformations that feed LSTM sigmoid and tanh gates.

Extreme inputs can saturate gates near zero or one and suppress useful gradients. Fitting the scaler on validation or test observations creates temporal leakage by using future distribution statistics.

Fit every scaling parameter on the training period only, reuse it for later splits, and preserve an inverse transformation for forecasts. Inspect constant features, outliers, and the distribution seen by the gates.

Part 2 - Mathematical form

$$\widetilde{x}_{t,j} = \frac{x_{t,j} - \mu_j^{\text{train}}}{\sigma_j^{\text{train}} + \epsilon}.$$

Part 3 - Hints and Helper

Store feature names beside scaler statistics and never refit on validation or test data.

Part 4 - Python code skeleton

```
import torch
def E01(model, data, config):
    prepared = fit_train_only_scaler(model, data, config, ...)
    result = audit_scaled_features(prepared, data, config, ...)
    assert result is not None
    return result
```

16.7.3 Exercise 02 - Build a One-Step LSTM Forecaster

Part 1 - Why this matters

A one-step forecaster processes the complete lookback window and maps its final recurrent representation to the next target. It is the simplest end-to-end test of model structure, shapes, and regression loss.

Selecting the wrong time step, removing the batch dimension, or broadcasting predictions against targets can produce a valid number for the wrong task. Batch size one often hides these defects.

Implement explicit batch-first dimensions and assert prediction-target shape equality before loss computation. Test single-target and multi-target cases with several batch and sequence sizes.

Part 2 - Mathematical form

$$(H, C) = \text{LSTM}(X), \qquad \widehat{\mathbf{y}}_{t+1} = W_o H_{:,-1,:} + b_o.$$

Part 3 - Hints and Helper

Expected input is (B, L, F) and output is (B, O); avoid unrestricted `squeeze()`.

Part 4 - Python code skeleton

```
import torch
def E02(model, data, config):
    prepared = run_one_step_forecaster(model, data, config, ...)
    result = validate_prediction_and_loss(prepared, data, config, ...)
    assert result is not None
    return result
```

16.7.4 Exercise 03 - Extend the Forecaster to Multi-Step Prediction

Part 1 - Why this matters

Multi-step forecasting produces an entire future path. A direct model emits all horizons together, whereas an autoregressive model feeds each prediction back into the next decision.

Autoregressive errors accumulate because training often uses clean observations while inference consumes imperfect model outputs. Direct prediction avoids feedback drift but must learn horizon-specific behavior in one output.

Implement both strategies with the same data and splits, verify that the rolling input advances correctly, and compute error separately at every horizon. Do not hide later-horizon degradation inside one mean score.

Part 2 - Mathematical form

$$\widehat{y}_{t+k} = f_\theta(x_{t-L+1:t}, \widehat{y}_{t+1:t+k-1}), \qquad k = 1, \ldots, H.$$

Part 3 - Hints and Helper

Separate known future covariates from unknown future targets during autoregressive rollout.

Part 4 - Python code skeleton

```
import torch
def E03(model, data, config):
    prepared = generate_multi_step_forecast(model, data, config, ...)
    result = compare_direct_and_autoregressive(prepared, data, config, ...)
    assert result is not None
    return result
```

16.7.5 Exercise 04 - Learn Long Seasonality Better Than a Vanilla RNN

Part 1 - Why this matters

Synthetic seasonal data gives a controlled setting where the true periods, trend, noise, and dependency length are known. It can test whether an LSTM retains periodic evidence beyond a vanilla RNN's practical memory.

A model may exploit short autocorrelation, overlapping windows, or an explicit time index instead of remembering a long cycle. An unfair parameter budget can also predetermine the comparison.

Generate several periods and phases, split chronologically, and compare matched RNN, LSTM, and seasonal-naive models. Evaluate accuracy as the required memory distance increases and repeat across seeds.

Part 2 - Mathematical form

$$y_t = \sum_{k=1}^{K} a_k \sin\left(\frac{2\pi t}{P_k} + \phi_k\right) + \beta t + \epsilon_t.$$

Part 3 - Hints and Helper

Include a seasonal-naive baseline and report model parameter counts.

Part 4 - Python code skeleton

```
import torch
def E04(model, data, config):
    prepared = generate_seasonal_series(model, data, config, ...)
    result = compare_rnn_lstm_seasonality(prepared, data, config, ...)
    assert result is not None
    return result
```

16.7.6 Exercise 05 - Compare Teacher Forcing and Autoregressive Generation

Part 1 - Why this matters

Teacher forcing supplies the true previous target to the decoder during training, which simplifies optimization. Autoregressive inference instead consumes earlier predictions and exposes the model to its own errors.

This mismatch can produce low training loss but unstable rollouts. Pure autoregressive training may reduce exposure bias, yet it can be difficult before the model has learned a useful local forecast.

Train with fixed and scheduled forcing ratios, then evaluate every version with a fully autoregressive rollout. Measure horizon-wise drift and sensitivity to early prediction errors.

Part 2 - Mathematical form

$$u_{t+1} = m_t y_t + (1 - m_t)\widehat{y}_t, \qquad m_t \sim \text{Bernoulli}(\rho).$$

Part 3 - Hints and Helper

Document whether the forcing decision is sampled per sequence, sample, or time step.

Part 4 - Python code skeleton

```
import torch
def E05(model, data, config):
    prepared = decode_with_teacher_forcing(model, data, config, ...)
    result = measure_rollout_drift(prepared, data, config, ...)
    assert result is not None
    return result
```

16.7.7 Exercise 06 - Evaluate Forecast Accuracy with MAE, MSE, and MAPE

Part 1 - Why this matters

Forecast metrics emphasize different failures. MAE weights errors linearly, MSE punishes large misses strongly, and MAPE expresses relative error but becomes unstable near zero targets.

A single aggregate can hide horizon degradation, target-specific failure, or scale differences. A few near-zero observations may dominate MAPE and make model comparisons meaningless.

Implement each metric manually, verify it with small tensors, and report results per horizon and target before aggregation. Apply a documented zero policy and evaluate in original units after inverse scaling.

Part 2 - Mathematical form

$$\mathrm{MAE} = \frac{1}{N}\sum |e_i|, \quad \mathrm{MSE} = \frac{1}{N}\sum e_i^2, \quad \mathrm{MAPE} = \frac{100}{N}\sum \left|\frac{e_i}{y_i}\right|.$$

Part 3 - Hints and Helper

Mask or report near-zero targets separately instead of hiding them with a large epsilon.

Part 4 - Python code skeleton

```
import torch
def E06(model, data, config):
    prepared = compute_forecast_metrics(model, data, config, ...)
    result = validate_metric_sensitivity(prepared, data, config, ...)
    assert result is not None
    return result
```

16.7.8 Exercise 07 - Debug Windowing and Alignment Bugs

Part 1 - Why this matters

Every forecast label must occur after the forecast origin by exactly the configured horizon. If the offset is wrong, the network may reconstruct recent observations or predict the past instead of the future.

Time-series autocorrelation can make these incorrect tasks look easier than correct forecasting. Loss reduction alone therefore cannot validate temporal alignment.

Build an audit containing source rows, timestamps, input range, origin, target range, and horizon. Inject off-by-one and overlap defects deliberately and require the audit to reject them.

Part 2 - Mathematical form

$$\max \mathcal{T}(X_i) < \min \mathcal{T}(Y_i), \qquad \min \mathcal{T}(Y_i) - \max \mathcal{T}(X_i) = h\Delta t.$$

Part 3 - Hints and Helper

Use a monotonic counter series so every alignment mistake becomes visible immediately.

Part 4 - Python code skeleton

```
import torch
def E07(model, data, config):
    prepared = inject_alignment_cases(model, data, config, ...)
    result = detect_temporal_alignment_failures(prepared, data, config, ...)
    assert result is not None
    return result
```

16.7.9 Exercise 08 - Diagnose and Reduce Forecasting Overfitting

Part 1 - Why this matters

An LSTM can memorize local fluctuations or one historical regime while failing on later periods. Temporal overfitting appears when training improvement continues after chronological validation has stopped improving.

Random splitting can hide the gap by placing neighboring windows in both sets. Excessive capacity, weak regularization, and checkpoint selection from a noisy metric can further distort the diagnosis.

Compare capacity, dropout, weight decay, and early stopping one factor at a time on identical validation periods. Restore the best checkpoint and examine horizon-specific behavior rather than only average loss.

Part 2 - Mathematical form

$$g_e = \mathcal{L}_{\text{val},e} - \mathcal{L}_{\text{train},e}, \qquad e^* = \arg\min_e \mathcal{L}_{\text{val},e}.$$

Part 3 - Hints and Helper

Save and restore the best model state rather than keeping the last epoch.

Part 4 - Python code skeleton

```
import torch
def E08(model, data, config):
    prepared = train_with_regularization(model, data, config, ...)
    result = diagnose_forecast_overfitting(prepared, data, config, ...)
    assert result is not None
    return result
```

16.7.10 Exercise 09 - Build Walk-Forward Validation without Leakage

Part 1 - Why this matters

Walk-forward validation respects deployment order by training on past data and evaluating on a later period. Expanding and rolling histories test different assumptions about how long old information remains useful.

Leakage can still enter through scaling, feature engineering, overlapping horizons, or repeated tuning on the final test period. Sorting data before a conventional random workflow is not sufficient.

Generate explicit fold boundaries, fit preprocessing inside each fold, and include a gap whenever targets or rolling features require one. Preserve each fold's time-specific results when aggregating performance.

Part 2 - Mathematical form

$$\mathcal{D}_{\text{train}}^{(k)} = [1, t_k], \qquad \mathcal{D}_{\text{val}}^{(k)} = [t_k + g + 1, t_k + g + V].$$

Part 3 - Hints and Helper

Assert that all training target timestamps precede validation forecast origins.

Part 4 - Python code skeleton

```
import torch
def E09(model, data, config):
    prepared = build_walk_forward_folds(model, data, config, ...)
    result = run_leakage_safe_validation(prepared, data, config, ...)
    assert result is not None
    return result
```

16.7.11 Exercise 10 - Train the Full Pipeline on CUDA

Part 1 - Why this matters

End-to-end CUDA performance includes data loading, transfer, forward computation, backpropagation, and optimizer updates. A fast recurrent kernel cannot compensate for a host pipeline that leaves the GPU waiting.

Asynchronous timing can understate runtime, while excluding transfers exaggerates operational speed. Device mismatch among data, model, states, and loss tensors may also fail only on particular batches.

Move the entire training path to CUDA, use pinned memory and non-blocking transfer when justified, and time both compute-only and end-to-end execution. Check output and loss parity against a controlled CPU run.

Part 2 - Mathematical form

$$Q = \frac{N_{\text{windows}} L}{t_{\text{wall}}}, \qquad t_{\text{wall}} = t_{\text{load}} + t_{\text{copy}} + t_{\text{compute}}.$$

Part 3 - Hints and Helper

Synchronize CUDA before and after every timed region.

Part 4 - Python code skeleton

```
import torch
def E10(model, data, config):
    prepared = train_forecaster_on_cuda(model, data, config, ...)
    result = benchmark_end_to_end_throughput(prepared, data, config, ...)
    assert result is not None
    return result
```

16.7.12 Exercise 11 - Profile Sequence Length and Hidden Size

Part 1 - Why this matters

Lookback length controls temporal context, while hidden size controls recurrent capacity. Both increase computation and activation storage, but additional context or capacity helps only when the data contains usable information.

Large configurations may reduce throughput, exhaust memory, or overfit without improving validation error. Measuring a single cold batch can confuse startup overhead with steady-state performance.

Sweep a controlled grid of sequence and hidden sizes, recording quality, latency, throughput, and peak memory. Select from the Pareto frontier rather than choosing the largest model that fits.

Part 2 - Mathematical form

$$C = \mathcal{O}(BL\, 4H(F+H)), \qquad M_{\text{act}} = \mathcal{O}(BLH).$$

Part 3 - Hints and Helper

Warm up each configuration and keep the evaluation folds identical.

Part 4 - Python code skeleton

```
import torch
def E11(model, data, config):
    prepared = profile_forecaster_configuration(model, data, config, ...)
    result = find_quality_speed_frontier(prepared, data, config, ...)
    assert result is not None
    return result
```

16.7.13 Exercise 12 - Implement Mixed Precision and Detect Gate Saturation

Part 1 - Why this matters

Mixed precision can reduce memory and improve CUDA throughput, but LSTM gate computations repeatedly pass values through sigmoid and tanh. Limited precision may saturate gates or lose small recurrent updates.

A run can continue while gradient scaling skips updates or gate outputs collapse near their limits. Final loss alone does not reveal where numerical divergence began.

Use automatic mixed precision and gradient scaling, log non-finite values and gate distributions, and compare with float32. Increase sequence length deliberately to identify the stability boundary.

Part 2 - Mathematical form

$$i_t = \sigma(a_t), \qquad \sigma'(a_t) = \sigma(a_t)(1 - \sigma(a_t)), \qquad |a_t| \uparrow \Rightarrow \sigma'(a_t) \downarrow .$$

Part 3 - Hints and Helper

Unscale gradients before clipping and compute diagnostic statistics in float32.

Part 4 - Python code skeleton

```
import torch
def E12(model, data, config):
    prepared = run_mixed_precision_forecasting(model, data, config, ...)
    result = detect_gate_saturation(prepared, data, config, ...)
    assert result is not None
    return result
```

16.7.14 Exercise 13 - Compare LSTM and GRU Forecasting

Part 1 - Why this matters

LSTM and GRU provide different balances between memory control and computational cost. GRU uses fewer gates, while LSTM's separate cell state may help when dependencies are long or multiple time scales interact.

One seed or unequal training budgets can turn random variation into a false architectural conclusion. Equal hidden size and equal parameter count are also different comparison conditions.

Run equal-width and matched-parameter experiments on datasets with short and long dependencies. Measure quality, convergence, throughput, memory, and robustness across controlled seeds.

Part 2 - Mathematical form

$$N_{\text{LSTM}} \approx 4H(F + H + 1), \qquad N_{\text{GRU}} \approx 3H(F + H + 1).$$

Part 3 - Hints and Helper

Use identical folds, preprocessing, optimizer budgets, and evaluation code.

Part 4 - Python code skeleton

```
import torch
def E13(model, data, config):
    prepared = train_matched_lstm_gru_models(model, data, config, ...)
    result = summarize_architecture_tradeoffs(prepared, data, config, ...)
    assert result is not None
    return result
```

16.7.15 Exercise 14 - Implement Truncated BPTT for Long Windows

Part 1 - Why this matters

Long forecast windows make full backpropagation expensive because recurrent activations must remain available across all time steps. Truncated BPTT carries states forward while limiting the gradient horizon.

Detaching only one LSTM state can retain graph history, whereas detaching both too frequently prevents long-dependency learning. Unordered chunks and incorrect resets destroy temporal continuity.

Train on ordered chunks, detach hidden and cell states at explicit truncation boundaries, and reset them at independent-series boundaries. Compare memory, throughput, and horizon accuracy across truncation lengths.

Part 2 - Mathematical form

$$(h_k, c_k) = F_\theta(X_k, h_{k-1}, c_{k-1}), \qquad (h_k, c_k) \leftarrow \text{stopgrad}(h_k, c_k).$$

Part 3 - Hints and Helper

Distinguish truncation boundaries from real sequence boundaries in metadata.

Part 4 - Python code skeleton

```
import torch
def E14(model, data, config):
    prepared = train_with_truncated_bptt(model, data, config, ...)
    result = quantify_truncation_tradeoff(prepared, data, config, ...)
    assert result is not None
    return result
```

16.7.16 Exercise 15 - Build a Multi-Horizon Forecast Visualization

Part 1 - Why this matters

Forecast plots reveal lag, phase error, amplitude compression, drift, and horizon-specific failure that aggregate metrics cannot show. Correct temporal coordinates are as important as the plotted values.

Overlapping windows can create unreadable lines, and mismatched timestamps can make an incorrect forecast look aligned. Plotting scaled predictions beside original targets is another silent visual error.

Build a tool that receives forecast origins, target timestamps, predictions, and truth in original units. Provide selected-origin views, horizon panels, residuals, and optional uncertainty bands.

Part 2 - Mathematical form

$$e_{i,h} = y_{t_i+h} - \widehat{y}_{t_i+h}, \qquad h = 1, \ldots, H.$$

Part 3 - Hints and Helper

Assert timestamp and value shapes, and mark the forecast origin explicitly.

Part 4 - Python code skeleton

```
import torch
def E15(model, data, config):
    prepared = collect_timestamped_forecasts(model, data, config, ...)
    result = plot_multi_horizon_forecasts(prepared, data, config, ...)
    assert result is not None
    return result
```

16.7.17 Exercise 16 - Measure Robustness under Seasonal Shift

Part 1 - Why this matters

Deployed time series may change in seasonal amplitude, baseline, variance, or phase. A model that interpolates its historical regime can fail sharply when this generating structure shifts.

Adding random noise is not the same as changing seasonality. A useful robustness experiment varies one known property while holding other factors fixed so the cause is identifiable.

Create test regimes with controlled amplitude multipliers, evaluate without retraining, and plot degradation against shift magnitude. Compare LSTM, GRU, and seasonal-naive baselines under identical shifted series.

Part 2 - Mathematical form

$$y_t^{(\alpha)} = \alpha A \sin\left(\frac{2\pi t}{P}\right) + r_t, \qquad \Delta(\alpha) = M(\alpha) - M(1).$$

Part 3 - Hints and Helper

Keep training scaler statistics fixed and reuse the same noise realization for all models.

Part 4 - Python code skeleton

```
import torch
def E16(model, data, config):
    prepared = generate_shifted_seasonality(model, data, config, ...)
    result = build_robustness_curve(prepared, data, config, ...)
    assert result is not None
    return result
```

16.7.18 Exercise 17 - Estimate Probabilistic Forecast Uncertainty

Part 1 - Why this matters

A point forecast gives one future path but does not express uncertainty. Probabilistic forecasting estimates a distribution or interval and shows how risk changes across horizons.

Narrow intervals may miss most observations, while very wide intervals achieve coverage without usefulness. A constant-variance assumption can also fail when volatility varies over time.

Modify the output head to predict distribution parameters, enforce a positive scale, and optimize negative log-likelihood. Evaluate empirical coverage and interval width separately at each horizon.

Part 2 - Mathematical form

$$(\mu_{t,h}, s_{t,h}) = f_\theta(X_t), \quad \sigma_{t,h} = \mathrm{softplus}(s_{t,h}) + \epsilon, \quad y_{t+h} \sim \mathcal{N}(\mu_{t,h}, \sigma_{t,h}^2).$$

Part 3 - Hints and Helper

Compare nominal coverage with empirical coverage and report interval sharpness.

Part 4 - Python code skeleton

```
import torch
def E17(model, data, config):
    prepared = predict_distribution_parameters(model, data, config, ...)
    result = evaluate_interval_calibration(prepared, data, config, ...)
    assert result is not None
    return result
```

16.7.19 Exercise 18 - Stress Test the Large Forecasting Pipeline

Part 1 - Why this matters

Large datasets expose window-construction cost, storage throughput, worker contention, transfer delays, GPU memory pressure, and checkpoint overhead that small experiments cannot reveal.

A stress test must distinguish slow progress from deadlock, memory leak, or an out-of-memory boundary. Total runtime alone cannot identify the limiting system component.

Scale data length, feature count, sequence length, and batch size systematically while recording component timing and resource peaks. Capture failures safely and define a verified operating envelope.

Part 2 - Mathematical form

$$t_{\text{epoch}} = t_{\text{I/O}} + t_{\text{window}} + t_{\text{copy}} + t_{\text{GPU}}, \qquad Q = \frac{N_{\text{samples}}}{t_{\text{epoch}}}.$$

Part 3 - Hints and Helper

Stream windows when full materialization would exceed host memory.

Part 4 - Python code skeleton

```
import torch
def E18(model, data, config):
    prepared = run_forecasting_stress_grid(model, data, config, ...)
    result = identify_pipeline_limits(prepared, data, config, ...)
    assert result is not None
    return result
```

16.7.20 Exercise 19 - Produce the Full Forecasting Engineering Report

Part 1 - Why this matters

A complete report connects data lineage, temporal validation, preprocessing, model design, optimization, uncertainty, robustness, visualization, and system performance. A best metric alone cannot establish forecast correctness.

Forecasting claims are vulnerable to leakage, alignment defects, selective horizons, and favorable historical periods. Each conclusion must identify its fold, forecast origin, horizon, units, baseline, and uncertainty.

Assemble audits, configurations, curves, failure records, and recommended operating limits. Separate observations from interpretation and provide a reproducible path from raw timestamps to final forecasts.

Part 2 - Mathematical form

$$\mathcal{R} = \{D, W, S, M, T, V, Q, U, F, P\}, \qquad Q = \{\mathrm{MAE}, \mathrm{MSE}, \mathrm{MAPE}, C, W_I, t, m\}.$$

Part 3 - Hints and Helper

Include exact split timestamps, data hashes, configuration hashes, and checkpoint identifiers.

Part 4 - Python code skeleton

```
import torch
def E19(model, data, config):
    prepared = collect_verified_forecasting_artifacts(
        model, data, config, ...
    )
    result = build_final_forecasting_report(prepared, data, config, ...)
    assert result is not None
    return result
```

16.8 LSTM06 - Regularization, Stabilization, and Robustness

Models and Training

16.8.1 Exercise 00 - Demonstrate LSTM Overfitting and Measure the Gap

Part 1 - Why this matters

Overfitting occurs when an LSTM learns training-specific patterns that do not transfer to unseen sequences. A realistic task should contain noise, variable lengths, and enough temporal structure to make memorization and genuine generalization distinguishable.

Randomly splitting overlapping windows can place nearly identical sequences in training and validation sets, making the gap appear smaller than it is. Training loss alone therefore cannot establish whether the model learned reusable temporal rules.

Create a chronological split, train an intentionally oversized LSTM without regularization, and log train and validation metrics every epoch. Identify the epoch where the validation trend separates and inspect failures across sequence lengths.

Part 2 - Mathematical form

$$G_e = M_{\text{val},e} - M_{\text{train},e}, \qquad e^* = \arg\min_e M_{\text{val},e}.$$

Part 3 - Hints and Helper

Preserve the best and final checkpoints so their predictions can be compared directly.

Part 4 - Python code skeleton

```
import torch
def E00(model, data, config):
    experiment = train_overcapacity_lstm(model, data, config, ...)
    report = measure_generalization_gap(experiment, data, config, ...)
    assert report is not None
    return report
```

16.8.2 Exercise 01 - Compare Inter-Layer and Input Dropout

Part 1 - Why this matters

Dropout can regularize input embeddings or the connections between stacked LSTM layers. These locations remove different information: input dropout perturbs observations, while inter-layer dropout perturbs learned temporal representations.

Applying ordinary dropout independently at every time step may inject inconsistent temporal noise. PyTorch LSTM dropout also has no effect when only one recurrent layer is configured, which can create a silent experimental mistake.

Implement both placements, verify train and evaluation behavior, and compare them under equal dropout probabilities. Track convergence, validation quality, activation statistics, and failure on long dependencies.

Part 2 - Mathematical form

$$\widetilde{x}_t = m_t^{(x)} \odot x_t, \qquad \widetilde{h}_t^{(\ell)} = m_t^{(\ell)} \odot h_t^{(\ell)}, \quad m \sim \text{Bernoulli}(1-p).$$

Part 3 - Hints and Helper

Confirm that `model.eval()` disables dropout and that stacked-LSTM dropout requires more than one layer.

Part 4 - Python code skeleton

```
import torch
def E01(model, data, config):
    experiment = run_dropout_placement_experiments(model, data, config, ...)
    report = compare_dropout_locations(experiment, data, config, ...)
    assert report is not None
    return report
```

16.8.3 Exercise 02 - Implement and Evaluate Weight Decay

Part 1 - Why this matters

Weight decay limits the growth of recurrent and output parameters. Smaller norms may improve validation behavior and reduce sensitivity to noisy observations.

Using the same decay for every parameter can regularize biases or normalization parameters unnecessarily. AdamW's decoupled decay is also not identical to adding an L2 term inside adaptive Adam updates.

Create parameter groups deliberately, sweep decay strengths, and record parameter norms, gradient norms, convergence, and validation metrics. Compare no decay, coupled L2, and AdamW under otherwise matched settings.

Part 2 - Mathematical form

$$\theta_{t+1} = (1 - \eta\lambda)\theta_t - \eta\, u_t, \qquad \lambda \geq 0.$$

Part 3 - Hints and Helper

Exclude parameters only with an explicit rationale and print the final optimizer groups for verification.

Part 4 - Python code skeleton

```
import torch
def E02(model, data, config):
    experiment = train_weight_decay_grid(model, data, config, ...)
    report = evaluate_decay_stability(experiment, data, config, ...)
    assert report is not None
    return report
```

16.8.4 Exercise 03 - Compare Gradient Clipping Norms

Part 1 - Why this matters

Backpropagation through long sequences can create very large gradient norms. Clipping limits the update magnitude and may prevent an isolated batch from destabilizing all later training.

A threshold that is too small changes nearly every update and can stop useful learning. Clipping after the optimizer step or before unscaling mixed-precision gradients does not protect the intended update.

Log unclipped norms, apply several thresholds at the correct point, and record the fraction of steps that are clipped. Compare stability, convergence speed, and final sequence quality rather than only whether NaN values disappear.

Part 2 - Mathematical form

$$g' = g \min\left(1, \frac{c}{\|g\|_2 + \epsilon}\right).$$

Part 3 - Hints and Helper

Call clipping after backward and mixed-precision unscaling but before the optimizer step.

Part 4 - Python code skeleton

```
import torch
def E03(model, data, config):
    experiment = train_clip_norm_grid(model, data, config, ...)
    report = analyze_clipping_frequency(experiment, data, config, ...)
    assert report is not None
    return report
```

16.8.5 Exercise 04 - Apply Layer Normalization inside LSTM

Part 1 - Why this matters

Layer normalization can stabilize recurrent computation by normalizing features within each sample rather than depending on batch statistics. It may reduce gate-logit drift as hidden states evolve across time.

Normalization placement changes the model equation. Normalizing concatenated gates, individual gates, the cell state, or only the output are different architectures and cannot be treated as one experiment.

Implement a transparent LSTM cell with documented normalization points, then inspect gate distributions and state drift. Compare against the unnormalized cell using identical weights where possible.

Part 2 - Mathematical form

$$\mathrm{LN}(a) = \gamma \odot \frac{a - \mu(a)}{\sqrt{\sigma^2(a) + \epsilon}} + \beta.$$

Part 3 - Hints and Helper

Log gate pre-activations and outputs separately; saturation should not be inferred from hidden states alone.

Part 4 - Python code skeleton

```
import torch
def E04(model, data, config):
    experiment = build_layer_normalized_lstm(model, data, config, ...)
    report = measure_gate_saturation_and_drift(experiment, data, config, ...)
    assert report is not None
    return report
```

16.8.6 Exercise 05 - Diagnose NaN Loss and Numerical Failure

Part 1 - Why this matters

A NaN loss is usually the final symptom of an earlier numerical failure. The first non-finite value may appear in an input, gate logit, cell state, gradient, optimizer state, or mixed-precision scale.

Checking only the loss loses the location and timing of the original defect. Continuing after the first invalid step can corrupt optimizer moments and make later diagnostics misleading.

Install staged finite-value checks, capture the failing batch and model state, and stop immediately on the first violation. Build automated reports for gate saturation, state growth, gradient overflow, and update magnitude.

Part 2 - Mathematical form

$$t^* = \min\{t : \neg\,\mathrm{isfinite}(z_t)\}, \qquad z_t \in \{x, a_g, c, h, \nabla\theta, \theta\}.$$

Part 3 - Hints and Helper

Save a reproducible failure bundle containing the seed, configuration, batch indices, and last valid checkpoint.

Part 4 - Python code skeleton

```
import torch
def E05(model, data, config):
    experiment = run_nan_diagnostic_training(model, data, config, ...)
    report = locate_first_nonfinite_tensor(experiment, data, config, ...)
    assert report is not None
    return report
```

16.8.7 Exercise 06 - Compare SGD, Adam, and AdamW

Part 1 - Why this matters

Optimizers interact differently with recurrent curvature, noisy gradients, and parameter scales. SGD may require careful momentum and learning rates, while Adam adapts coordinate-wise and AdamW decouples weight decay.

Using one learning rate for all optimizers creates an unfair comparison. Reporting the best single run also hides sensitivity to initialization and hyperparameters.

Perform a controlled learning-rate sweep for each optimizer, preserve equal training budgets, and repeat across seeds. Compare convergence, stability, validation quality, parameter norms, and runtime overhead.

Part 2 - Mathematical form

$$\theta_{t+1} = \theta_t - \eta\, U(g_{1:t}),$$

Part 3 - Hints and Helper

Tune each optimizer independently, then compare its best stable configuration under the same evaluation protocol.

Part 4 - Python code skeleton

```
import torch
def E06(model, data, config):
    experiment = run_optimizer_sweeps(model, data, config, ...)
    report = summarize_optimizer_sensitivity(experiment, data, config, ...)
    assert report is not None
    return report
```

16.8.8 Exercise 07 - Schedule Learning Rate and Prevent Late Divergence

Part 1 - Why this matters

A learning-rate schedule can allow rapid early progress and smaller late updates when the model approaches a useful region. This may prevent oscillation or late-epoch divergence in sensitive recurrent training.

Calling the scheduler at the wrong frequency shifts the entire schedule. A validation-based scheduler can also react to noise unless patience and thresholds are defined carefully.

Implement fixed-step, cosine, and plateau-based schedules, log the actual learning rate at every update, and compare late-epoch stability. Verify checkpoint restoration together with scheduler state.

Part 2 - Mathematical form

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \frac{\pi t}{T}\right).$$

Part 3 - Hints and Helper

State clearly whether the scheduler steps per batch or per epoch and test its first three values manually.

Part 4 - Python code skeleton

```
import torch
def E07(model, data, config):
    experiment = train_learning_rate_schedules(model, data, config, ...)
    report = detect_late_epoch_divergence(experiment, data, config, ...)
    assert report is not None
    return report
```

16.8.9 Exercise 08 - Analyze Batch Size and Gradient Noise

Part 1 - Why this matters

Batch size changes gradient variance, memory use, throughput, and the number of optimizer updates per epoch. Smaller batches may add useful noise, while larger batches provide smoother but potentially less generalizable updates.

Comparing batch sizes with the same epoch count changes the number of optimizer steps. Padding ratios and sequence-length composition can also vary with batching and confound the result.

Compare batches under both equal-epoch and equal-update budgets, record gradient variance and hardware utilization, and control sequence bucketing. Identify the best practical setting rather than the largest possible batch.

Part 2 - Mathematical form

$$\widehat{g}_B = \frac{1}{B} \sum_{i=1}^{B} g_i, \qquad \operatorname{Var}(\widehat{g}_B) \approx \frac{\operatorname{Var}(g)}{B}.$$

Part 3 - Hints and Helper

Report effective batch size when gradient accumulation is used.

Part 4 - Python code skeleton

```
import torch
def E08(model, data, config):
    experiment = benchmark_batch_size_grid(model, data, config, ...)
    report = analyze_gradient_noise_tradeoff(experiment, data, config, ...)
    assert report is not None
    return report
```

16.8.10 Exercise 09 - Debug Improving Loss but Poor Predictions

Part 1 - Why this matters

A decreasing scalar loss does not guarantee useful sequence predictions. The pipeline may optimize misaligned targets, padded positions, the wrong reduction, or a dominant easy class while the intended outputs remain poor.

Aggregate loss can hide collapsed predictions and horizon-specific failure. Teacher forcing may also produce strong training loss even though free-running generation fails.

Inspect decoded samples, masks, target shifts, baselines, and per-time-step metrics. Create deliberate bugs and require the debugging routine to distinguish data, loss, decoding, and evaluation failures.

Part 2 - Mathematical form

$$\mathcal{L} = \frac{\sum_{b,t} m_{b,t} \ell(\widehat{y}_{b,t}, y_{b,t})}{\sum_{b,t} m_{b,t}}.$$

Part 3 - Hints and Helper

Always compare against a simple baseline and evaluate using the exact inference procedure intended for deployment.

Part 4 - Python code skeleton

```
import torch
def E09(model, data, config):
    experiment = inject_silent_training_bugs(model, data, config, ...)
    report = diagnose_loss_prediction_mismatch(experiment, data, config, ...)
    assert report is not None
    return report
```

16.8.11 Exercise 10 - Measure CUDA Overhead of Stabilization

Part 1 - Why this matters

Dropout, gradient clipping, and normalization improve training behavior but add kernels, reductions, memory traffic, or synchronization. Their value must be judged against both quality and runtime cost.

Timing without CUDA synchronization or comparing different batch contents can produce false overhead estimates. Profiler instrumentation itself can also distort very short measurements.

Benchmark a fixed baseline and add one stabilization method at a time, using warm-up and repeated timed regions. Report step latency, throughput, peak memory, and validation improvement together.

Part 2 - Mathematical form

$$\text{overhead}_s = \frac{t_s - t_0}{t_0} \times 100\%.$$

Part 3 - Hints and Helper

Use matched weights and batches for microbenchmarks, then confirm results in an end-to-end epoch.

Part 4 - Python code skeleton

```
import torch
def E10(model, data, config):
    experiment = benchmark_cuda_stabilizers(model, data, config, ...)
    report = compute_quality_adjusted_overhead(experiment, data, config, ...)
    assert report is not None
    return report
```

16.8.12 Exercise 11 - Compare Mixed Precision with and without GradScaler

Part 1 - Why this matters

Mixed precision accelerates supported CUDA operations but lower precision can overflow gradients or underflow small values. Gradient scaling moves the loss into a safer numerical range before backward computation.

Training without a scaler may silently lose recurrent gradients, while an aggressive scale may overflow and force skipped optimizer steps. A stable loss does not prove that gate gradients remain informative.

Run float32, autocast-only, and autocast-with-scaler configurations, logging non-finite gradients, skipped steps, and gate statistics. Increase sequence length to reveal numerical limits.

Part 2 - Mathematical form

$$\widehat{\mathcal{L}} = s\mathcal{L}, \qquad \widehat{g} = s\nabla_\theta \mathcal{L}, \qquad g = \widehat{g}/s.$$

Part 3 - Hints and Helper

Unscale before gradient clipping and record the scale value throughout training.

Part 4 - Python code skeleton

```
import torch
def E11(model, data, config):
    experiment = compare_amp_scaler_modes(model, data, config, ...)
    report = audit_mixed_precision_gate_gradients(
        experiment, data, config, ...
    )
    assert report is not None
    return report
```

16.8.13 Exercise 12 - Perform a Stabilization Ablation Study

Part 1 - Why this matters

Ablation determines which methods actually contribute on the chosen dataset. A strong full system does not reveal whether dropout, clipping, normalization, scheduling, or decay is useful or redundant.

Removing several components together creates interactions that cannot be attributed. Selecting only favorable seeds or changing training budgets invalidates the comparison.

Define a baseline and controlled component matrix, repeat every condition across seeds, and report uncertainty. Include single additions, single removals from the full system, and selected interaction tests.

Part 2 - Mathematical form

$$\Delta_j = M(S \setminus \{j\}) - M(S), \qquad \Delta_{jk} = M(S \setminus \{j,k\}) - M(S).$$

Part 3 - Hints and Helper

Freeze data splits, initialization seeds, update budgets, and evaluation code across all ablations.

Part 4 - Python code skeleton

```
import torch
def E12(model, data, config):
    experiment = run_stabilization_ablation_matrix(model, data, config, ...)
    report = estimate_component_contributions(experiment, data, config, ...)
    assert report is not None
    return report
```

16.8.14 Exercise 13 - Evaluate Robustness under Distribution Shift

Part 1 - Why this matters

Sequence distributions change when event frequency, noise, vocabulary, seasonal structure, or dependency length differs from training. LSTM memory can fail when the stored patterns no longer match the new regime.

Random corruption alone does not identify which temporal property caused failure. Robustness results are also misleading when preprocessing is refitted on shifted test data.

Create controlled shifts one factor at a time, keep training preprocessing fixed, and evaluate degradation by shift severity. Probe states and gates to locate where memory retention breaks.

Part 2 - Mathematical form

$$\Delta M(\delta) = M(P_\delta) - M(P_0), \qquad \delta \in \{\text{noise, length, frequency, regime}\}.$$

Part 3 - Hints and Helper

Compare against a simple baseline to determine whether recurrence helps under each shift.

Part 4 - Python code skeleton

```
import torch
def E13(model, data, config):
    experiment = generate_controlled_sequence_shifts(model, data, config, ...)
    report = map_memory_failure_regions(experiment, data, config, ...)
    assert report is not None
    return report
```

16.8.15 Exercise 14 - Calibrate Sequence Predictions over Time

Part 1 - Why this matters

A model is calibrated when predictions with confidence q are correct approximately q of the time. Sequence confidence may change with time step as uncertainty and memory errors accumulate.

Overall calibration can hide severe late-step overconfidence. Padding, class imbalance, and selecting only completed sequences can also bias calibration estimates.

Compute reliability diagrams, expected calibration error, and proper scoring rules per time step or horizon. Compare temperature scaling fitted only on validation data and verify it on an untouched test split.

Part 2 - Mathematical form

$$\mathrm{ECE} = \sum_{b=1}^{B} \frac{|I_b|}{N} \left|\mathrm{acc}(I_b) - \mathrm{conf}(I_b)\right|.$$

Part 3 - Hints and Helper

Preserve masks and report the number of valid observations in every confidence bin.

Part 4 - Python code skeleton

```
import torch
def E14(model, data, config):
    experiment = collect_sequence_confidences(model, data, config, ...)
    report = evaluate_temporal_calibration(experiment, data, config, ...)
    assert report is not None
    return report
```

16.8.16 Exercise 15 - Compare Early Stopping and Regularization

Part 1 - Why this matters

Early stopping limits effective optimization time, whereas dropout and weight decay constrain the learned solution throughout training. They may complement each other or create underfitting when combined too aggressively.

A noisy validation metric can stop training by chance, and using the test set for stopping leaks evaluation information. Different patience values also imply different training budgets.

Use a dedicated chronological validation set, save the best model and optimizer state, and compare early stopping alone with explicit regularizers. Measure final quality, runtime saved, and stability across seeds.

Part 2 - Mathematical form

$$e_{\text{stop}} = \min\{e : \ M_{\text{val}} \text{ has not improved for } p \text{ epochs}\}.$$

Part 3 - Hints and Helper

Define minimum improvement and patience before running the experiment.

Part 4 - Python code skeleton

```
import torch
def E15(model, data, config):
    experiment = train_early_stopping_variants(model, data, config, ...)
    report = compare_implicit_explicit_regularization(
        experiment, data, config, ...
    )
    assert report is not None
    return report
```

16.8.17 Exercise 16 - Create an LSTM Failure Debugging Checklist

Part 1 - Why this matters

LSTM failures span data alignment, masks, state lifecycle, gate saturation, gradients, optimizer order, precision, device placement, and evaluation. A checklist turns scattered debugging knowledge into a repeatable diagnostic sequence.

Changing parameters before identifying the earliest failed invariant can hide the symptom without fixing the cause. Unordered checks also waste time by inspecting model complexity before validating data.

Build executable checks ordered from inputs to outputs, attach evidence to every pass or failure, and stop at the first corrupted stage. Test the checklist against intentionally injected defects.

Part 2 - Mathematical form

$$D_0 \to D_1 \to \cdots \to D_K, \qquad k^* = \min\{k : D_k = \text{fail}\}.$$

Part 3 - Hints and Helper

Every checklist item should produce a measurable assertion, not a vague recommendation.

Part 4 - Python code skeleton

```
import torch
def E16(model, data, config):
    experiment = run_lstm_failure_checklist(model, data, config, ...)
    report = validate_checklist_with_fault_injection(
        experiment, data, config, ...
    )
    assert report is not None
    return report
```

16.8.18 Exercise 17 - Analyze Long-Sequence Generalization Limits

Part 1 - Why this matters

Performance on sequences longer than the training range tests whether the model learned a transferable temporal mechanism. Failure may arise from truncated gradients, insufficient hidden capacity, or recurrent state drift.

Increasing hidden size cannot recover dependencies that training truncation prevents from receiving credit. Conversely, full gradients cannot help if the representation lacks capacity or the data provides no signal.

Create a grid of training truncation length, hidden size, and evaluation length. Use controlled memory tasks and state probes to separate optimization horizon from representational capacity.

Part 2 - Mathematical form

$$A(H, K, T) = \text{accuracy}(\text{hidden size } H,\ \text{truncation } K,\ \text{test length } T).$$

Part 3 - Hints and Helper

Vary one axis at a time first, then test interactions near the observed failure boundary.

Part 4 - Python code skeleton

```
import torch
def E17(model, data, config):
    experiment = run_long_sequence_generalization_grid(
        model, data, config, ...
    )
```

```
    report = attribute_truncation_capacity_failure(
        experiment, data, config, ...
    )
    assert report is not None
    return report
```

16.8.19 Exercise 18 - Stress Test Very Long and Wide LSTMs

Part 1 - Why this matters

Very long sequences and large hidden states expose activation-memory growth, recurrent compute cost, kernel limits, and numerical instability. Safe system limits must be measured before large training runs.

An out-of-memory event is only one failure mode; latency spikes, memory leaks, non-finite states, and GPU underutilization may appear earlier. Failed cases must not contaminate later measurements.

Sweep sequence length, hidden size, layer count, batch size, and precision under explicit stop conditions. Record peak memory, throughput, state norms, failures, and recovery behavior to define the operating envelope.

Part 2 - Mathematical form

$$C = \mathcal{O}(BLH(F + H)), \qquad M_{\text{act}} = \mathcal{O}(BLH).$$

Part 3 - Hints and Helper

Warm up each case, synchronize CUDA, and release failed-case objects before continuing.

Part 4 - Python code skeleton

```
import torch
def E18(model, data, config):
    experiment = run_lstm_stress_grid(model, data, config, ...)
    report = identify_memory_runtime_limits(experiment, data, config, ...)
    assert report is not None
    return report
```

16.8.20 Exercise 19 - Write a Practical LSTM Stabilization Guide

Part 1 - Why this matters

A practical guide must connect symptoms to measurements, root causes, interventions, and verified outcomes. Listing techniques without decision criteria does not help an engineer choose the next action.

Recommendations that worked on one dataset may fail under different lengths, noise, hardware, or objectives. The guide must distinguish broadly reliable invariants from task-specific

findings.

Assemble experiment tables, diagnostic thresholds, decision paths, failure examples, performance costs, and reproducibility information. Link each recommendation to evidence and document unresolved limits.

Part 2 - Mathematical form

$$\mathcal{G} = \{S, E, C, F, R\}, \qquad S \to E \to C \to F \to R.$$

Part 3 - Hints and Helper

For every strategy, state when to use it, how to verify it, its cost, and when to stop using it.

Part 4 - Python code skeleton

```
import torch
def E19(model, data, config):
    experiment = collect_stabilization_evidence(model, data, config, ...)
    report = build_engineering_strategy_guide(experiment, data, config, ...)
    assert report is not None
    return report
```

16.9 LSTM07 - Efficiency, Profiling, and Deployment

Applied AI and Deployment

16.9.1 Exercise 00 - Benchmark LSTM vs GRU Runtime and Memory

Part 1 - Why this matters

LSTM maintains four gate transformations and separate hidden and cell states, whereas GRU uses fewer gates and one principal recurrent state. This usually makes LSTM heavier, but its additional memory control may preserve dependencies that justify the cost.

An equal hidden-size comparison gives the architectures different parameter counts, while an equal-parameter comparison requires different widths. Timing asynchronous CUDA execution or comparing different sequence batches invalidates either result.

Benchmark equal-width and matched-parameter models with identical data, warm-up, precision, and update counts. Report runtime, peak memory, throughput, validation quality, and the dependency lengths where LSTM's extra cost becomes useful.

Part 2 - Mathematical form

$$N_{\text{LSTM}} \approx 4H(D + H + 1), \qquad N_{\text{GRU}} \approx 3H(D + H + 1).$$

Part 3 - Hints and Helper

Synchronize CUDA around timed regions. Report equal-width and matched-parameter results separately.

Part 4 - Python code skeleton

```
import torch
def E00(model, data, config):
    experiment = benchmark_lstm_gru(model, data, config, ...)
    report = explain_quality_cost_tradeoff(experiment, data, config, ...)
    assert report is not None
    return report
```

16.9.2 Exercise 01 - Profile LSTM Training with PyTorch Profiler

Part 1 - Why this matters

A profiler reveals which operators consume execution time, allocate memory, or force synchronization. LSTM training combines recurrent kernels, tensor transfers, loss computation, backward operations, and optimizer updates, so intuition alone cannot identify the true bottleneck.

Collecting every trace for an entire run adds significant overhead and can distort the workload. Kernel time without shape, call count, and memory context may also point to the wrong optimization target.

Profile scheduled warm-up and active steps, group results by operator and input shape, and inspect CPU and CUDA timelines. Rank bottlenecks by total cost and verify each proposed fix with a second controlled profile.

Part 2 - Mathematical form

$$t_{\text{step}} = t_{\text{input}} + t_{\text{forward}} + t_{\text{backward}} + t_{\text{update}}.$$

Part 3 - Hints and Helper

Use a short profiler schedule and export a trace only for representative steps.

Part 4 - Python code skeleton

```
import torch
def E01(model, data, config):
    experiment = profile_lstm_training(model, data, config, ...)
    report = rank_kernel_memory_bottlenecks(experiment, data, config, ...)
    assert report is not None
    return report
```

16.9.3 Exercise 02 - Reduce Padding Waste and Improve GPU Utilization

Part 1 - Why this matters

Variable-length sequences padded to the longest item in a batch can waste most recurrent computation. Length-aware batching and appropriate collation increase the fraction of tokens that contain real information.

Aggressive sorting may reduce randomness, change class balance within batches, or break stateful stream order. A faster pipeline is invalid if targets, masks, or sample identities become misaligned.

Measure padding ratio, loader time, transfer time, and GPU utilization before changing the pipeline. Compare random batching, length buckets, dynamic batches, and token-budget batching while verifying identical valid outputs.

Part 2 - Mathematical form

$$w_{\text{pad}} = 1 - \frac{\sum_i T_i}{B \max_i T_i}.$$

Part 3 - Hints and Helper

Log valid and padded token counts for every batch instead of estimating waste from one example.

Part 4 - Python code skeleton

```
import torch
def E02(model, data, config):
    experiment = optimize_sequence_batching(model, data, config, ...)
    report = measure_padding_utilization_gain(experiment, data, config, ...)
    assert report is not None
    return report
```

16.9.4 Exercise 03 - Compare Packed and Padded Runtime

Part 1 - Why this matters

Packed sequences allow supported recurrent operations to skip padded time steps. The benefit depends on the length distribution, packing overhead, device implementation, and how much downstream work still uses padded tensors.

Packed execution can be slower when sequences have similar lengths or batches are small. Incorrect sorting, unsorting, or length placement may also preserve shapes while misaligning outputs and targets.

Benchmark packed and padded pipelines across controlled length distributions, then verify numerical equivalence at valid positions. Include packing, sorting, transfer, recurrent execution, and unpacking in end-to-end timing.

Part 2 - Mathematical form

$$S = \frac{t_{\text{padded}}}{t_{\text{packed}}}, \qquad \rho = \frac{\sum_i T_i}{BT_{\max}}.$$

Part 3 - Hints and Helper

Restore original sample order before comparing outputs or metrics.

Part 4 - Python code skeleton

```
import torch
def E03(model, data, config):
    experiment = benchmark_packed_padded(model, data, config, ...)
    report = verify_speed_and_output_parity(experiment, data, config, ...)
    assert report is not None
    return report
```

16.9.5 Exercise 04 - Measure Sequence-Length Scaling

Part 1 - Why this matters

LSTM computation is sequential across time, so longer inputs increase latency and activation storage. The practical limit depends on batch size, hidden width, layers, precision, and the available GPU.

Testing until the first out-of-memory error misses earlier limits such as unacceptable latency, falling throughput, numerical drift, or reduced batch capacity. Cold-start measurements can also exaggerate short-sequence cost.

Sweep sequence lengths geometrically, warm up each case, and record throughput, latency, peak memory, and state statistics. Define limits from explicit service or training budgets rather than memory alone.

Part 2 - Mathematical form

$$C = \mathcal{O}(BLH(D + H)), \qquad M_{\text{act}} = \mathcal{O}(BLH).$$

Part 3 - Hints and Helper

Keep other dimensions fixed during the first sweep, then test interactions near the limit.

Part 4 - Python code skeleton

```
import torch
def E04(model, data, config):
    experiment = benchmark_sequence_lengths(model, data, config, ...)
    report = identify_practical_gpu_limit(experiment, data, config, ...)
    assert report is not None
    return report
```

16.9.6 Exercise 05 - Compare batch_first and Memory Layout

Part 1 - Why this matters

The `batch_first` option changes the public tensor layout but not the mathematical recurrence. Performance differences can arise from transposes, contiguous copies, downstream operators, or how the input pipeline already stores data.

Comparing only the LSTM call may exclude layout-conversion cost that appears in the real pipeline. Non-contiguous tensors may also trigger hidden copies and make the result configuration-dependent.

Create equivalent time-first and batch-first pipelines, inspect strides and contiguity, and time both conversion-inclusive and kernel-only execution. Verify output and state parity after explicit axis mapping.

Part 2 - Mathematical form

$$X_{\mathrm{BF}} \in \mathbb{R}^{B \times T \times D}, \qquad X_{\mathrm{TF}} = X_{\mathrm{BF}}^{(1,0,2)}.$$

Part 3 - Hints and Helper

Report tensor strides and whether a contiguous copy occurs before the recurrent call.

Part 4 - Python code skeleton

```
import torch
def E05(model, data, config):
    experiment = benchmark_batch_first_layouts(model, data, config, ...)
    report = explain_layout_performance(experiment, data, config, ...)
    assert report is not None
    return report
```

16.9.7 Exercise 06 - Scale Hidden Size and Layer Count

Part 1 - Why this matters

Hidden size increases both recurrent capacity and quadratic hidden-to-hidden work, while additional layers multiply recurrent computation and activation storage. Scaling curves reveal where larger models stop producing useful quality gains.

Changing width and depth simultaneously prevents attribution, and failed large cases can leave reserved CUDA memory that contaminates later measurements. Parameter count alone does not describe activation cost.

Sweep width and depth in controlled stages, restart or clean resources between cases, and record parameters, FLOPs estimates, latency, throughput, peak memory, and validation quality. Build quality-cost Pareto curves.

Part 2 - Mathematical form

$$N \propto L_{\text{layers}}\, 4H(D + H + 1).$$

Part 3 - Hints and Helper

Measure steady-state allocated and peak memory, not only reserved memory after a failure.

Part 4 - Python code skeleton

```
import torch
def E06(model, data, config):
    experiment = benchmark_lstm_scaling_grid(model, data, config, ...)
    report = build_runtime_memory_curves(experiment, data, config, ...)
    assert report is not None
    return report
```

16.9.8 Exercise 07 - Diagnose an Under-Utilized GPU

Part 1 - Why this matters

Low GPU utilization usually means the device is waiting for input, transfers, synchronization, or very small kernels. LSTM workloads can also be sequential enough that additional hardware remains idle.

A dashboard percentage without a timeline cannot distinguish data stalls from kernel gaps. Increasing workers blindly may create CPU contention, excess memory, or nondeterministic loader failures.

Measure data wait, transfer, compute, padding ratio, and kernel occupancy, then modify one bottleneck at a time. Validate improvements using end-to-end throughput and unchanged model outputs.

Part 2 - Mathematical form

$$U_{\text{effective}} = \frac{t_{\text{GPU busy}}}{t_{\text{wall}}}.$$

Part 3 - Hints and Helper

Start with a synthetic in-memory loader to separate model limits from pipeline limits.

Part 4 - Python code skeleton

```
import torch
def E07(model, data, config):
    experiment = diagnose_gpu_underutilization(model, data, config, ...)
    report = apply_verified_pipeline_fixes(experiment, data, config, ...)
    assert report is not None
    return report
```

16.9.9 Exercise 08 - Compare float32 and Mixed Precision

Part 1 - Why this matters

Mixed precision can accelerate matrix operations and reduce activation memory, enabling larger batches. Recurrent gates and long state chains may nevertheless be more sensitive to overflow, underflow, and saturation.

Speedup measured on an unstable model is not useful. Autocast-only training, missing gradient scaling, or unequal batch sizes can make the comparison misleading.

Compare float32 and mixed precision with matched workloads, record throughput, memory, non-finite events, skipped steps, and validation quality. Increase sequence length to document the stable operating region.

Part 2 - Mathematical form

$$S_{\mathrm{AMP}} = \frac{t_{\mathrm{FP32}}}{t_{\mathrm{AMP}}}, \qquad \Delta M = M_{\mathrm{AMP}} - M_{\mathrm{FP32}}.$$

Part 3 - Hints and Helper

Unscale before clipping and include scaler behavior in the benchmark report.

Part 4 - Python code skeleton

```
import torch
def E08(model, data, config):
    experiment = benchmark_precision_modes(model, data, config, ...)
    report = document_amp_stability_constraints(experiment, data, config, ...)
    assert report is not None
    return report
```

16.9.10 Exercise 09 - Measure Gradient-Clipping Overhead

Part 1 - Why this matters

Gradient clipping requires norm reductions over trainable parameters and may add synchronization or memory traffic. Its runtime cost must be balanced against avoided divergence and improved recoverability.

Measuring only the clipping function ignores how often clipping changes the gradient. A low threshold can add cost while also altering every update and slowing convergence.

Benchmark clipping disabled and several thresholds, record step latency and the clipped-step fraction, and compare failed runs or recovered checkpoints. Decide value from both overhead and stability benefit.

Part 2 - Mathematical form

$$O_c = \frac{t_c - t_0}{t_0}, \qquad f_c = \frac{\#\{\|g_t\| > c\}}{N_{\text{steps}}}.$$

Part 3 - Hints and Helper

Measure unclipped norms before modification and synchronize CUDA for microbenchmarks.

Part 4 - Python code skeleton

```
import torch
def E09(model, data, config):
    experiment = benchmark_gradient_clipping(model, data, config, ...)
    report = evaluate_clipping_value(experiment, data, config, ...)
    assert report is not None
    return report
```

16.9.11 Exercise 10 - Use Gradient Accumulation under Memory Limits

Part 1 - Why this matters

Gradient accumulation divides an effective large batch into microbatches whose gradients are summed before one optimizer step. It allows larger effective batches without storing all activations simultaneously.

Forgetting to divide the loss changes gradient scale, and stepping the scheduler per microbatch changes optimization. Stateful recurrence and variable token counts require especially careful weighting.

Implement accumulation with correct normalization, compare it against a true large batch, and verify parameter-update parity on a deterministic example. Measure memory, throughput, convergence, and effective token count.

Part 2 - Mathematical form

$$g_{\text{eff}} = \frac{1}{K}\sum_{k=1}^{K} g_k, \qquad B_{\text{eff}} = KB_{\text{micro}}.$$

Part 3 - Hints and Helper

Zero gradients and step the optimizer only at accumulation boundaries.

Part 4 - Python code skeleton

```
import torch
def E10(model, data, config):
    experiment = train_with_gradient_accumulation(model, data, config, ...)
    report = compare_true_and_effective_batches(experiment, data, config, ...)
    assert report is not None
    return report
```

16.9.12 Exercise 11 - Build Reproducible CUDA Benchmark Tables

Part 1 - Why this matters

A benchmark table is useful only when another engineer can reproduce its workload and measurement conditions. Hardware, software, precision, shapes, warm-up, seeds, and timing boundaries all affect LSTM results.

Reporting a single best latency hides variance, cold-start behavior, and failed cases. Comparing experiments performed with different clocks, loaders, or deterministic settings produces false rankings.

Build a benchmark harness that records environment metadata and repeated measurements, then reports median and tail latency, throughput, memory, and quality. Export raw records beside summary tables.

Part 2 - Mathematical form

$$\widetilde{t} = \text{median}(t_1, \ldots, t_n), \qquad p_{95} = \text{quantile}_{0.95}(t).$$

Part 3 - Hints and Helper

Record exact tensor shapes and synchronization rules for every row.

Part 4 - Python code skeleton

```
import torch
def E11(model, data, config):
    experiment = run_reproducible_cuda_benchmarks(model, data, config, ...)
    report = create_benchmark_tables(experiment, data, config, ...)
    assert report is not None
    return report
```

16.9.13 Exercise 12 - Investigate cuDNN LSTM Non-Determinism

Part 1 - Why this matters

Some CUDA kernels may select algorithms or use parallel reductions whose operation order changes across runs. Small numerical differences can grow through recurrent steps and affect final training outcomes.

A different result does not automatically prove kernel non-determinism; data order, initialization, dropout, workers, and mixed precision are alternative causes. Forcing determinism may also reduce performance.

Control every known random source, repeat identical forward and training runs, and compare tensors at each stage. Change deterministic settings one at a time and document the first divergence and its performance cost.

Part 2 - Mathematical form

$$\delta_t = \|h_t^{(a)} - h_t^{(b)}\|_\infty.$$

Part 3 - Hints and Helper

Separate bitwise equality from tolerance-based numerical equivalence.

Part 4 - Python code skeleton

```
import torch
def E12(model, data, config):
    experiment = run_cudnn_determinism_trials(model, data, config, ...)
    report = locate_and_explain_divergence(experiment, data, config, ...)
    assert report is not None
    return report
```

16.9.14 Exercise 13 - Export LSTM with TorchScript

Part 1 - Why this matters

TorchScript can package model computation independently from ordinary Python execution and may simplify deployment. Export success is insufficient unless state handling, dynamic sequence lengths, and outputs remain correct.

Tracing can freeze control flow or miss branches, while scripting may reject unsupported Python constructs. Comparing only one shape may hide failures in dynamic batches or recurrent states.

Export with an appropriate method, test a matrix of shapes and state configurations, and compare outputs with strict tolerances. Benchmark cold and warm latency including state initialization.

Part 2 - Mathematical form

$$\epsilon_{\max} = \max_i |y_i^{\text{native}} - y_i^{\text{export}}|.$$

Part 3 - Hints and Helper

Test both fresh-state and carried-state inference after export.

Part 4 - Python code skeleton

```
import torch
def E13(model, data, config):
    experiment = export_torchscript_lstm(model, data, config, ...)
    report = validate_export_parity_latency(experiment, data, config, ...)
    assert report is not None
    return report
```

16.9.15 Exercise 14 - Export LSTM to ONNX

Part 1 - Why this matters

ONNX enables execution in alternative runtimes, but recurrent operators, packed sequences, dynamic axes, and state tuples can create portability problems. A valid graph may still behave incorrectly for unseen shapes.

Hard-coded batch or sequence dimensions often pass the sample export test and fail in production. Unsupported operations may be replaced in ways that change semantics or precision.

Export with explicit inputs, states, outputs, and dynamic axes, inspect the graph, and validate several runtimes and shapes. Record unsupported cases and implement only verified simplifications.

Part 2 - Mathematical form

$$f_\theta^{\text{PyTorch}}(X, h, c) \approx f_\theta^{\text{ONNX}}(X, h, c).$$

Part 3 - Hints and Helper

Use real dynamic-shape tests rather than trusting symbolic axis declarations.

Part 4 - Python code skeleton

```
import torch
def E14(model, data, config):
    experiment = export_lstm_to_onnx(model, data, config, ...)
    report = validate_onnx_edge_cases(experiment, data, config, ...)
    assert report is not None
    return report
```

16.9.16 Exercise 15 - Build a Production Inference Benchmark

Part 1 - Why this matters

Production inference must satisfy latency and throughput requirements across realistic batches and sequence lengths. Queueing, preprocessing, state handling, and synchronization contribute alongside model execution.

Average latency hides tail spikes, and maximum throughput at a large batch may violate per-request delay targets. Synthetic zero inputs can also exercise different paths from real traffic.

Replay representative request distributions, separate cold and warm results, and measure median, p95, p99, throughput, memory, and error rate. Locate the batch-size point that satisfies the service objective.

Part 2 - Mathematical form

$$L_q = \text{quantile}_q(t_{\text{request}}), \qquad Q = \frac{N_{\text{requests}}}{t_{\text{wall}}}.$$

Part 3 - Hints and Helper

Include preprocessing and state initialization in one end-to-end benchmark view.

Part 4 - Python code skeleton

```
import torch
def E15(model, data, config):
    experiment = run_production_inference_load(model, data, config, ...)
    report = summarize_latency_throughput_slo(experiment, data, config, ...)
    assert report is not None
    return report
```

16.9.17 Exercise 16 - Checkpoint and Resume Training

Part 1 - Why this matters

A recoverable checkpoint must preserve model parameters, optimizer moments, scheduler position, gradient-scaler state, epoch, step, seeds, and relevant data progress. Saving only model weights does not reproduce continuation.

A checkpoint can load successfully while resuming with a different learning rate, data order, or mixed-precision scale. Partial writes may also leave a corrupted file that is discovered only during an incident.

Save atomically, verify integrity, interrupt a controlled run, and resume from the checkpoint. Compare the resumed trajectory with an uninterrupted reference and measure recovery time.

Part 2 - Mathematical form

$$S_k = \{\theta_k, O_k, R_k, G_k, e_k, s_k\}.$$

Part 3 - Hints and Helper

Write to a temporary object and publish only after integrity validation.

Part 4 - Python code skeleton

```
import torch
def E16(model, data, config):
    experiment = run_checkpoint_recovery_test(model, data, config, ...)
    report = verify_resumed_state_consistency(experiment, data, config, ...)
    assert report is not None
    return report
```

16.9.18 Exercise 17 - Stress Test Stateful Long-Sequence Inference

Part 1 - Why this matters

Long-sequence inference exposes cumulative latency, state growth, memory leaks, and incorrect state carry between independent requests. A fixed-shape hidden state does not guarantee fixed process memory.

Reusing a state across unrelated streams leaks information, while resetting within one stream destroys continuity. Retained computation graphs or cached outputs can cause gradual memory growth.

Run thousands of varied requests with explicit stream identifiers, monitor memory and latency over time, and inject resets and interleaving. Detect leaks, tail spikes, and cross-stream state contamination.

Part 2 - Mathematical form

$$\Delta m_n = m_n - m_0, \qquad \Delta L_n = L_n - \widetilde{L}_{\text{warm}}.$$

Part 3 - Hints and Helper

Use inference mode and assert that every stored state is detached and belongs to the correct stream.

Part 4 - Python code skeleton

```
import torch
def E17(model, data, config):
    experiment = stress_stateful_inference(model, data, config, ...)
    report = detect_leaks_spikes_state_bugs(experiment, data, config, ...)
    assert report is not None
    return report
```

16.9.19 Exercise 18 - Build a Deployment Readiness Checklist

Part 1 - Why this matters

Deployment readiness combines correctness, performance, stability, reproducibility, monitoring, recovery, and interface compatibility. Passing an accuracy threshold covers only one part of operational risk.

A checklist made of vague statements cannot block unsafe releases. Every item needs evidence, an owner, an acceptance threshold, and a response when the threshold fails.

Create executable checks for input contracts, export parity, latency, memory, determinism, state lifecycle, monitoring, rollback, and checkpoint recovery. Produce a signed readiness result with unresolved risks.

Part 2 - Mathematical form

$$R = \bigwedge_{i=1}^{N} \mathbf{1}[m_i \in A_i].$$

Part 3 - Hints and Helper

Separate mandatory release gates from advisory observations.

Part 4 - Python code skeleton

```
import torch
def E18(model, data, config):
    experiment = run_deployment_readiness_checks(model, data, config, ...)
    report = produce_release_decision(experiment, data, config, ...)
    assert report is not None
    return report
```

16.9.20 Exercise 19 - Produce the LSTM Performance Report

Part 1 - Why this matters

A complete engineering report connects architecture, workload shapes, profiler evidence, scaling curves, exports, production benchmarks, and recovery behavior. Isolated speed numbers cannot guide deployment decisions.

Cherry-picked configurations hide tradeoffs and failure boundaries. Results without environment metadata, raw measurements, and quality checks cannot be reproduced or trusted.

Assemble verified curves, tables, profiler findings, parity tests, incidents, and best practices. Distinguish measured facts from interpretation and publish a safe operating envelope.

Part 2 - Mathematical form

$$\mathcal{R} = \{E, W, P, S, X, L, C, F, B\}.$$

Part 3 - Hints and Helper

Link every recommendation to a benchmark record or failure experiment.

Part 4 - Python code skeleton

```
import torch
def E19(model, data, config):
    experiment = collect_lstm_performance_artifacts(model, data, config, ...)
    report = build_complete_engineering_report(experiment, data, config, ...)
    assert report is not None
    return report
```

16.10 LSTM08 - Bridge to Attention and Why Transformers Win

Models and Training

16.10.1 Exercise 00 - Demonstrate Very-Long-Context LSTM Failure

Part 1 - Why this matters

LSTM gates improve gradient flow, but a fixed-size recurrent state must still compress an arbitrarily long history. Controlled memory tasks can expose where accuracy falls as the required dependency moves farther into the past.

A failure may come from insufficient training coverage, truncated gradients, noise, or hidden capacity rather than context length alone. Testing one model at one length cannot separate these causes.

Train across controlled dependency distances, evaluate beyond the training range, and sweep capacity and truncation length. Plot accuracy, state sensitivity, and gradient signal against context length.

Part 2 - Mathematical form

$$A(T) = \Pr(\widehat{y}_T = y_T), \qquad T \uparrow \Rightarrow A(T) \downarrow \text{ after the memory limit.}$$

Part 3 - Hints and Helper

Remove positional and frequency shortcuts from the synthetic task.

Part 4 - Python code skeleton

```
import torch
def E00(model, data, config):
    experiment = run_long_context_memory_trials(model, data, config, ...)
    report = quantify_lstm_context_degradation(experiment, data, config, ...)
    assert report is not None
    return report
```

16.10.2 Exercise 01 - Explain Sequential Computation Limits

Part 1 - Why this matters

An LSTM must compute state t before state $t + 1$, creating a critical path across the entire sequence. Attention can compute token projections in parallel and then relate positions through matrix operations.

Attention is not always faster: its quadratic pairwise work can dominate for long sequences, small batches, or limited memory. The relevant comparison depends on sequence length, hardware, and implementation.

Measure kernel timelines and scaling for recurrent and attention layers across sequence lengths. Separate available parallelism, total operations, memory use, and end-to-end throughput.

Part 2 - Mathematical form

$$T_{\mathrm{LSTM}} \geq \sum_{t=1}^{L} T_t, \qquad T_{\mathrm{attn}} \approx T_Q + T_K + T_V + T_{QK^\top}.$$

Part 3 - Hints and Helper

Explain both the recurrent critical path and attention's quadratic memory cost.

Part 4 - Python code skeleton

```
import torch
def E01(model, data, config):
    experiment = benchmark_parallelism_limits(model, data, config, ...)
    report = compare_lstm_attention_scaling(experiment, data, config, ...)
    assert report is not None
    return report
```

16.10.3 Exercise 02 - Add Simple Additive Attention

Part 1 - Why this matters

Additive attention allows a decoder to form a context vector from all encoder outputs instead of relying only on one final state. Learned scores can select the time steps most useful for the current prediction.

Wrong tensor broadcasting or score reduction may produce normalized weights with incorrect semantics. Attention can also appear to help simply because it adds parameters.

Implement the score equation explicitly, validate shapes and normalization, and compare with a parameter-controlled LSTM baseline. Inspect quality as dependency distance increases.

Part 2 - Mathematical form

$$e_{t,s} = v^\top \tanh(W_h h_s + W_q q_t + b), \quad \alpha_{t,s} = \mathrm{softmax}_s(e_{t,s}), \quad c_t = \sum_s \alpha_{t,s} h_s.$$

Part 3 - Hints and Helper

Softmax must normalize across valid encoder time steps for each decoder query.

Part 4 - Python code skeleton

```
import torch
def E02(model, data, config):
    experiment = implement_additive_attention(model, data, config, ...)
    report = measure_attention_quality_gain(experiment, data, config, ...)
    assert report is not None
    return report
```

16.10.4 Exercise 03 - Implement Dot-Product Attention

Part 1 - Why this matters

Dot-product attention scores compatibility using vector products and maps naturally to efficient batched matrix multiplication. Scaling prevents large dimensions from pushing softmax into extremely sharp distributions.

Additive and dot-product attention are not comparable when dimensions, projections, parameter counts, or masks differ. Omitting the scale can cause saturation as key dimension grows.

Implement projected queries, keys, and values, validate against a manual tiny example, and compare quality, runtime, memory, and attention entropy with additive attention.

Part 2 - Mathematical form

$$A = \operatorname{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V.$$

Part 3 - Hints and Helper

Check the softmax axis and validate masked positions receive zero probability.

Part 4 - Python code skeleton

```
import torch
def E03(model, data, config):
    experiment = implement_dot_product_attention(model, data, config, ...)
    report = compare_additive_dot_product(experiment, data, config, ...)
    assert report is not None
    return report
```

16.10.5 Exercise 04 - Visualize Attention over Time

Part 1 - Why this matters

Attention maps expose how decoder decisions distribute weight across encoder positions. They can reveal alignment, diffuse uncertainty, padding leakage, or repeated focus on an irrelevant token.

A heatmap is not a causal explanation, and raw colors can mislead when rows use different scales. Token and time labels must remain aligned after sorting and padding.

Build a visualization from verified weights, labels, masks, and sample identities. Pair the heatmap with entropy, top-attended positions, predictions, and failure examples.

Part 2 - Mathematical form

$$H(\alpha_t) = -\sum_s \alpha_{t,s} \log(\alpha_{t,s} + \epsilon).$$

Part 3 - Hints and Helper

Use a common color scale and display masked cells explicitly.

Part 4 - Python code skeleton

```
import torch
def E04(model, data, config):
    experiment = collect_attention_maps(model, data, config, ...)
    report = visualize_and_interpret_attention(experiment, data, config, ...)
    assert report is not None
    return report
```

16.10.6 Exercise 05 - Build an LSTM Encoder-Decoder with Attention

Part 1 - Why this matters

A sequence-to-sequence model uses an encoder to represent the source and a decoder to produce targets step by step. Attention lets each decoder step retrieve a different source context.

Source masks, shifted decoder inputs, state shapes, and target alignment must all agree. A model can train while predicting the current target from itself if decoder shifting is wrong.

Implement the full data flow with explicit states and masks, assert every tensor shape, and overfit a tiny aligned example. Compare teacher-forced and autoregressive decoding.

Part 2 - Mathematical form

$$h_{1:S} = E(x_{1:S}), \qquad y_t = D(y_{t-1}, c_t, s_{t-1}).$$

Part 3 - Hints and Helper

Verify decoder input at step t never contains target y_t.

Part 4 - Python code skeleton

```
import torch
def E05(model, data, config):
    experiment = build_attention_seq2seq(model, data, config, ...)
    report = validate_seq2seq_alignment(experiment, data, config, ...)
    assert report is not None
    return report
```

16.10.7 Exercise 06 - Compare Copy Task with and without Attention

Part 1 - Why this matters

A copy task requires the decoder to recover ordered source information and can expose the fixed-vector bottleneck. Attention should allow direct retrieval of relevant positions as sequence length increases.

Trivial token patterns, fixed lengths, or positional shortcuts may let both models succeed without meaningful memory. Different parameter budgets can also explain apparent gains.

Generate random variable-length sequences, train matched encoder-decoder models, and evaluate exact-match accuracy across lengths beyond training. Inspect attention alignment against the known copy correspondence.

Part 2 - Mathematical form

$$y_{1:S} = x_{1:S}, \qquad \alpha^*_{t,s} \approx \mathbf{1}[t = s].$$

Part 3 - Hints and Helper

Use exact sequence match in addition to token accuracy.

Part 4 - Python code skeleton

```
import torch
def E06(model, data, config):
    experiment = run_copy_task_comparison(model, data, config, ...)
    report = measure_attention_alignment_gain(experiment, data, config, ...)
    assert report is not None
    return report
```

16.10.8 Exercise 07 - Diagnose Attention Mask and Softmax Bugs

Part 1 - Why this matters

Attention failures often come from normalizing across the wrong dimension, applying a mask after softmax, reversing mask meaning, or misaligning sorted sequences. The output can remain finite and train slowly despite invalid weights.

Checking only whether every tensor has the expected shape is insufficient. Weights must sum to one across valid keys and be exactly zero on padded positions.

Create small hand-calculated cases and fault injections for axes, masks, and ordering. Build assertions that identify the violated invariant before training begins.

Part 2 - Mathematical form

$$\sum_{s \in V_i} \alpha_{i,t,s} = 1, \qquad \alpha_{i,t,s} = 0 \text{ for } s \notin V_i.$$

Part 3 - Hints and Helper

Apply the padding mask to logits before softmax using a sufficiently negative value.

Part 4 - Python code skeleton

```
import torch
def E07(model, data, config):
    experiment = inject_attention_faults(model, data, config, ...)
    report = detect_mask_softmax_alignment_bugs(experiment, data, config, ...)
    assert report is not None
    return report
```

16.10.9 Exercise 08 - Measure CUDA Cost and Quality of Attention

Part 1 - Why this matters

Adding attention increases projections, score matrices, softmax work, and activation memory, but it may reduce the burden on recurrent state compression. The tradeoff should be evaluated jointly in quality and system cost.

Timing different batch shapes or excluding attention-map storage makes overhead estimates unfair. Quality gains from extra parameters must be separated from gains due to retrieval.

Benchmark pure LSTM and LSTM-attention models with matched workloads and controlled capacity. Report throughput, latency, peak memory, utilization, and long-dependency accuracy.

Part 2 - Mathematical form

$$O_A = \frac{t_{\text{LSTM+A}} - t_{\text{LSTM}}}{t_{\text{LSTM}}}, \qquad \Delta Q = Q_{\text{LSTM+A}} - Q_{\text{LSTM}}.$$

Part 3 - Hints and Helper

Measure training and inference separately because backward attention storage changes the cost.

Part 4 - Python code skeleton

```
import torch
def E08(model, data, config):
    experiment = benchmark_lstm_attention_cuda(model, data, config, ...)
    report = calculate_quality_cost_tradeoff(experiment, data, config, ...)
    assert report is not None
    return report
```

16.10.10 Exercise 09 - Produce the Bridge-to-Transformers Report

Part 1 - Why this matters

Attention removes the requirement that all source information pass through one final encoder vector. Transformers extend this idea by replacing recurrent token processing with stacked attention and parallel feed-forward computation.

The transition is not simply that attention is always superior. Recurrent state offers linear memory in sequence length, while full attention has quadratic pairwise storage and different deployment behavior.

Synthesize evidence from memory tasks, copy alignment, profiler traces, and scaling curves. Explain which bottleneck attention solves, which costs it introduces, and why self-attention enables the Transformer architecture.

Part 2 - Mathematical form

$$\text{fixed state} \rightarrow \text{attention retrieval} \rightarrow \text{self-attention layers}.$$

Part 3 - Hints and Helper

Use experimental evidence from earlier exercises rather than architecture slogans.

Part 4 - Python code skeleton

```
import torch
def E09(model, data, config):
    experiment = collect_attention_bridge_evidence(model, data, config, ...)
    report = write_transformer_bridge_report(experiment, data, config, ...)
    assert report is not None
    return report
```

16.10.11 Exercise 10 - Quantify the Encoder-Decoder Bottleneck

Part 1 - Why this matters

Without attention, a decoder depends heavily on a fixed-dimensional encoder state. As source length and information content grow, this compression can discard details needed for accurate reconstruction.

Poor performance may instead arise from an undertrained decoder or insufficient total capacity. A valid bottleneck experiment must vary source information while controlling optimization.

Increase source length and entropy, compare final-state reconstruction with access to all encoder outputs, and probe how much source information remains decodable. Report failure by position.

Part 2 - Mathematical form

$$z = f(x_{1:S}) \in \mathbb{R}^H, \qquad I(z; x_{1:S}) \text{ is capacity-limited.}$$

Part 3 - Hints and Helper

Use position-wise error to show which source regions are lost first.

Part 4 - Python code skeleton

```
import torch
def E10(model, data, config):
    experiment = run_encoder_bottleneck_trials(model, data, config, ...)
    report = quantify_information_loss(experiment, data, config, ...)
    assert report is not None
    return report
```

16.10.12 Exercise 11 - Masked Attention for Variable Lengths

Part 1 - Why this matters

Variable-length batches require padding, but padded encoder positions contain no source information. Masked attention must exclude them before normalization so sequence length does not change valid weights.

Masking after softmax leaves probability mass on padding, while a wrong broadcast shape can mask valid tokens for only some batches or decoder steps.

Construct masks from trusted lengths, apply them to score logits, and validate against individually processed sequences. Test minimum length, full length, and mixed batches.

Part 2 - Mathematical form

$$\alpha = \text{softmax}(E + M), \qquad M_s = \begin{cases} 0 & s \text{ valid} \\ -\infty & s \text{ padded.} \end{cases}$$

Part 3 - Hints and Helper

Assert padded probabilities are zero and valid probabilities sum to one.

Part 4 - Python code skeleton

```
import torch
def E11(model, data, config):
    experiment = implement_masked_attention(model, data, config, ...)
    report = validate_padding_invariance(experiment, data, config, ...)
    assert report is not None
    return report
```

16.10.13 Exercise 12 - Compare Attention Distributions by Length

Part 1 - Why this matters

Attention behavior may become sharper, more diffuse, or position-biased as sequence length grows. Distribution statistics help reveal whether the model still retrieves relevant evidence or loses focus.

Raw entropy increases with the number of available positions, so unnormalized values cannot be compared directly across lengths. Padding and task difficulty must also be controlled.

Collect attention across length bins, compute normalized entropy, peak mass, effective support, and alignment accuracy, and relate these measures to prediction quality.

Part 2 - Mathematical form

$$H_{\text{norm}}(\alpha) = \frac{-\sum_s \alpha_s \log \alpha_s}{\log |V|}.$$

Part 3 - Hints and Helper

Compare only valid positions and report sample counts for every length bin.

Part 4 - Python code skeleton

```
import torch
def E12(model, data, config):
    experiment = analyze_attention_by_length(model, data, config, ...)
    report = interpret_distribution_changes(experiment, data, config, ...)
    assert report is not None
    return report
```

16.10.14 Exercise 13 - Profile Parallelism Added by Attention

Part 1 - Why this matters

Attention score computation uses batched matrix operations across positions and exposes more intra-step parallelism than pure recurrence. The surrounding LSTM encoder and decoder remain sequential, so only part of the pipeline benefits.

High GPU utilization does not prove lower latency, and profiler traces can confuse concurrent kernels with true end-to-end speedup. Sequence length changes both available parallelism and attention cost.

Profile pure LSTM, LSTM with attention, and a small self-attention block across lengths. Compare critical paths, kernel occupancy, memory traffic, and throughput.

Part 2 - Mathematical form

$$P = \frac{\text{parallelizable work}}{\text{total work}}, \qquad S \leq \frac{1}{(1-P) + P/N}.$$

Part 3 - Hints and Helper

Mark recurrent and attention regions separately in the profiler trace.

Part 4 - Python code skeleton

```
import torch
def E13(model, data, config):
    experiment = profile_attention_parallelism(model, data, config, ...)
    report = compare_gpu_critical_paths(experiment, data, config, ...)
    assert report is not None
    return report
```

16.10.15 Exercise 14 - Clip Gradients in LSTM plus Attention

Part 1 - Why this matters

Combining recurrent paths with attention projections creates additional gradient routes and can increase norm spikes on long sequences. Clipping may stabilize training while preserving attention alignment.

A threshold chosen for pure LSTM may over-clip the combined model. Global clipping can also hide whether instability begins in recurrent, attention, or output parameters.

Log component-wise and global norms, sweep thresholds, and measure clipped-step frequency, convergence, and alignment quality. Compare global clipping with targeted diagnostics.

Part 2 - Mathematical form

$$g' = g \min\left(1, \frac{c}{\|g\|_2 + \epsilon}\right).$$

Part 3 - Hints and Helper

Measure component norms before applying one global clipping operation.

Part 4 - Python code skeleton

```
import torch
def E14(model, data, config):
    experiment = train_attention_clip_grid(model, data, config, ...)
    report = evaluate_long_sequence_stability(experiment, data, config, ...)
    assert report is not None
    return report
```

16.10.16 Exercise 15 - Choose LSTM, LSTM plus Attention, or Transformer

Part 1 - Why this matters

Architecture choice depends on dependency structure, data volume, latency, memory, sequence length, streaming needs, and deployment hardware. No single model family dominates every operating region.

Choosing by popularity ignores constraints, while choosing by one benchmark ignores robustness and maintenance. Streaming state and quadratic attention memory can reverse conclusions from offline accuracy.

Build a decision checklist with measurable thresholds and evaluate representative workloads for all three options. Record the evidence and unresolved risks behind the final selection.

Part 2 - Mathematical form

$$a^* = \arg\min_{a \in A} \{\lambda_L L_a + \lambda_M M_a - \lambda_Q Q_a + \lambda_R R_a\}.$$

Part 3 - Hints and Helper

Separate hard constraints from weighted preferences in the decision.

Part 4 - Python code skeleton

```
import torch
def E15(model, data, config):
    experiment = run_architecture_selection_checks(model, data, config, ...)
    report = recommend_sequence_architecture(experiment, data, config, ...)
    assert report is not None
    return report
```

16.10.17 Exercise 16 - Export LSTM plus Attention

Part 1 - Why this matters

Exporting attention adds masks, dynamic source and target lengths, state tuples, and batched score operations to the recurrent deployment graph. These increase interface and runtime complexity beyond pure LSTM.

A successful export on one shape may freeze dimensions or mask assumptions. Alternative runtimes may implement recurrent and attention operators with different performance and precision.

Export both models, validate output and attention parity over a shape matrix, and compare graph size, latency, memory, and unsupported cases. Document required preprocessing and state contracts.

Part 2 - Mathematical form

$$\epsilon = \max |y^{\text{native}} - y^{\text{export}}|.$$

Part 3 - Hints and Helper

Test dynamic lengths, all-valid masks, mixed masks, and carried decoder states.

Part 4 - Python code skeleton

```
import torch
def E16(model, data, config):
    experiment = export_lstm_attention_model(model, data, config, ...)
    report = compare_deployment_complexity(experiment, data, config, ...)
    assert report is not None
    return report
```

16.10.18 Exercise 17 - Stress Test Long Attention Sequences

Part 1 - Why this matters

Attention can preserve access to distant encoder outputs, but score matrices grow with source and query lengths. Eventually memory and quadratic work may outweigh the retrieval benefit.

Testing only until out-of-memory ignores earlier latency and throughput limits. LSTM-attention differs from full self-attention, so its source-target score shape must be reported explicitly.

Sweep source length, target length, batch size, and precision, measuring quality, latency, memory, and failures. Identify where attention still improves accuracy and where its cost becomes unacceptable.

Part 2 - Mathematical form

$$M_{\text{cross}} \propto B\, T_{\text{target}} T_{\text{source}}.$$

Part 3 - Hints and Helper

Report both the quality frontier and the system-resource frontier.

Part 4 - Python code skeleton

```
import torch
def E17(model, data, config):
    experiment = stress_long_attention_sequences(model, data, config, ...)
    report = map_attention_benefit_cost_limits(experiment, data, config, ...)
    assert report is not None
    return report
```

16.10.19 Exercise 18 - Ablate Attention Components

Part 1 - Why this matters

An attention module contains query, key, value projections, scoring, scaling, masking, normalization, and context fusion. Ablation reveals which components produce the observed gain.

Removing multiple components simultaneously prevents attribution, and replacing a component with a different-capacity operation confounds the result. Attention maps alone are not evidence of usefulness.

Run controlled removals and replacements with matched budgets, repeat across seeds, and measure quality, alignment, runtime, and memory. Include a randomized-attention control.

Part 2 - Mathematical form

$$\Delta_j = M(A \setminus \{j\}) - M(A).$$

Part 3 - Hints and Helper

Keep training steps, data order, initialization policy, and evaluation code fixed.

Part 4 - Python code skeleton

```
import torch
def E18(model, data, config):
    experiment = run_attention_ablation_matrix(model, data, config, ...)
    report = quantify_attention_components(experiment, data, config, ...)
    assert report is not None
    return report
```

16.10.20 Exercise 19 - Produce the Final LSTM Track Report

Part 1 - Why this matters

The final track report should connect gate equations, cell-state memory, BPTT behavior, data handling, forecasting, stabilization, profiling, deployment, and the bridge to attention. It should show how each engineering decision follows from observed limitations.

A chapter summary that lists topics without experiments cannot demonstrate mastery. Results must preserve failures, controls, uncertainty, and the boundaries where conclusions stop applying.

Assemble verified artifacts from the entire LSTM track, compare recurrent and attention mechanisms, and define the next learning steps toward Transformers. Separate durable principles from dataset-specific findings.

Part 2 - Mathematical form

$$\mathcal{T}_{\text{LSTM}} = \{G, C, \text{BPTT}, D, R, P, A, N\}.$$

Part 3 - Hints and Helper

Link every major conclusion to an exercise result, plot, benchmark, or failure report.

Part 4 - Python code skeleton

```
import torch
def E19(model, data, config):
    experiment = collect_full_lstm_track_artifacts(model, data, config, ...)
    report = build_final_track_report(experiment, data, config, ...)
    assert report is not None
    return report
```

Book 17

Transformers

17.1 Introduction

17.1.1 What will the book teach you

This book studies the Transformer as a complete computational system rather than as a diagram containing a few familiar blocks. The Transformer changed sequence modeling because it replaced recurrent communication through time with direct, content-dependent interaction between positions. A token no longer has to wait for a hidden state to travel through every preceding time step before it can receive information from a distant token. Through self-attention, every position can construct a query, compare that query with the keys of other positions, and retrieve a weighted combination of their values. This creates a shorter path for information and gradients, and it allows the computations for all sequence positions to be expressed as large matrix operations that modern accelerators can execute efficiently.

That architectural change must be understood carefully. A Transformer does not make every sequence problem constant in cost, nor does it remove every form of sequential computation. During training, the representations for all positions can usually be computed in parallel because the complete target sequence is available and future information can be blocked with a causal mask. During autoregressive inference, however, token $t+1$ cannot be sampled before token t exists. Generation therefore remains sequential across newly produced tokens, even though the computation inside each step is highly parallel. The book will repeatedly separate algorithmic dependency, computational depth, arithmetic cost, memory cost, and measured hardware utilization so that the advantages of attention are described precisely rather than reduced to the vague claim that "Transformers are parallel."

The first chapter makes the attention mechanism transparent. The reader will derive Query, Key, and Value representations and interpret attention as content-addressed retrieval from a differentiable memory bank. Given a query matrix Q, key matrix K, and value matrix V, scaled dot-product attention is written as

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^{\mathsf{T}}}{\sqrt{d_k}} + M\right)V,$$

where d_k is the key dimension and M represents structural restrictions such as padding or causal masks. Every part of this equation has a distinct role. The dot product measures compatibility, scaling controls the variance of the logits, masking removes forbidden relation-

ships, softmax creates a normalized retrieval distribution, and multiplication by V constructs the retrieved representation. The exercises will implement each stage with pure PyTorch tensor operations, verify it on examples that can be calculated by hand, and compare the result with trusted implementations.

Attention is especially dangerous to learn only through successful execution. A softmax over the wrong dimension can still produce finite probabilities. A mask with an incorrect broadcast shape can silently mix examples or heads. A causal mask with the wrong orientation can leak future targets and create misleadingly good training loss. Large logits can overflow in low precision, while an entire masked row can produce undefined normalization. For this reason, the book treats shape contracts, mask semantics, numerical finiteness, probability sums, gradient behavior, and information access as independent invariants. Correct output dimensions are necessary, but they are not proof that the attention operation represents the intended computation.

The reader will then move from one attention operation to multi-head attention. The model projects the same input into several learned subspaces, performs attention independently within each head, concatenates the resulting representations, and maps them back into the model dimension. This is not merely a way to repeat one calculation several times. Different projections can make different relations easy to express: one head may respond to local structure, another to distant agreement, and another to a task-specific association. The book will inspect projection matrices, splitting and transposition rules, per-head tensor layouts, output projection, gradient correctness, head redundancy, and head importance. It will also distinguish a visually interesting attention pattern from evidence that a head is causally important to the prediction.

Because self-attention compares content without inherently knowing order, the book gives positional information its own mathematical and experimental treatment. Without position, a self-attention layer is permutation-equivariant: reordering the input positions reorders the outputs but does not reveal which token was first, last, or separated by a particular distance. The reader will implement sinusoidal and learnable positional representations, investigate relative position bias and rotary position ideas, and test extrapolation beyond the context lengths observed during training. Off-by-one errors, padding positions, device placement, indexing conventions, and long-context numerical behavior will be examined explicitly. Positional encoding will be understood as part of the model's representation of structure, not as a decorative tensor added before the real network begins.

Individual operations become useful only when they are assembled into a stable Transformer block. The reader will build attention sublayers, position-wise feed-forward networks, residual paths, Layer Normalization, activation functions, and dropout. Residual connections preserve an identity path through which information and gradients can travel, while Layer Normalization controls the statistics of each token representation. The feed-forward network applies a learned non-linear transformation independently at every position after attention has exchanged information across positions. Pre-LN and Post-LN arrangements will be compared because normalization placement changes gradient flow and the stability of deep stacks. The reader will not merely reproduce a standard block: every addition, normalization axis, expansion ratio, mask, and dropout location must have an explicit tensor and optimization justification.

The data pipeline is part of the model. Raw text cannot enter a Transformer directly; it must be converted into discrete token identifiers, arranged into sequences, padded or packed

into batches, associated with masks, and aligned with targets. The book will study character, word, and subword perspectives, special-token conventions, vocabulary mappings, embedding lookup, tied input and output embeddings, label shifting, padding masks, and loss masks. A single off-by-one error in next-token prediction can make a model optimize the wrong task while its loss still decreases. The reader will therefore construct tiny datasets whose correct inputs and targets can be inspected directly before moving to streaming text pipelines, prefetching, pinned memory, and large-scale throughput measurements.

Training a Transformer is a system-level experiment in optimization and numerical stability. The reader will build complete training and validation loops; use AdamW, learning-rate warmup, scheduling, gradient clipping, accumulation, and activation checkpointing; and monitor gradients, attention logits, loss spikes, learning rate, memory, and throughput. Mixed precision will be evaluated as a trade-off rather than enabled blindly. Matrix multiplication may benefit greatly from reduced precision, while softmax, normalization, or large attention scores may require additional care. Tiny-task overfitting, finite-difference checks, exact checkpoint continuation, and controlled synthetic tasks will be used to prove that the entire training path is correct before larger experiments are trusted.

The decoder-only portion of the book connects the Transformer directly to autoregressive language modeling. The reader will construct a causal model, build next-token targets, and study why masking turns a parallel training computation into a valid factorization of sequence probability:

$$p(x_1, \ldots, x_T) = \prod_{t=1}^{T} p(x_t \mid x_1, \ldots, x_{t-1}).$$

Generation will be examined as a decision process over probability distributions. Greedy decoding, temperature, top-k, nucleus sampling, repetition penalties, beam search, end-of-sequence handling, and batched stopping rules will be implemented and compared. The reader will learn why low perplexity does not guarantee coherent generation, why deterministic decoding may amplify repetition, and why sampling settings change the behavior of a fixed trained model without changing its parameters. KV caching will expose the difference between recomputing the complete prefix and reusing previously calculated keys and values during inference.

The encoder-decoder architecture will then show how attention supports conditional sequence generation. The encoder builds contextual source representations; the decoder uses masked self-attention over the generated prefix and cross-attention over the encoder memory. Translation, summarization, structured transformation, and synthetic copy or reversal tasks all require careful separation of source padding masks, target causal masks, target padding, shifted labels, and decoding state. The reader will implement cross-attention from scratch, compare teacher-forced training with autoregressive evaluation, investigate exposure bias, visualize alignment, and evaluate decoding choices. This makes the distinction between encoder-only, decoder-only, and encoder-decoder Transformers operational rather than merely taxonomic.

The final chapter treats efficiency and deployment as consequences of the mathematics. Standard attention constructs a score matrix whose area grows with the product of query and key sequence lengths. For self-attention with length T, both score storage and the dominant attention computation scale quadratically with T under the usual dense formulation. Long context can therefore exhaust memory long before parameter storage becomes the limiting fac-

tor. The reader will measure this behavior, study Flash-Attention as an exact computation organized to reduce memory traffic, examine sliding-window attention as a structural restriction, and evaluate activation checkpointing, KV cache growth, precision choices, quantization, compilation, and batching. The difference between theoretical FLOPs, memory bandwidth, kernel-launch overhead, peak allocation, and end-to-end latency will remain visible.

Production readiness extends beyond achieving a good evaluation score. The reader will construct inference wrappers that enforce input and mask contracts, benchmark latency and throughput under realistic loads, validate exported models against a reference, detect NaNs and corrupted output, monitor changes in input distributions, and design rollback and A/B testing procedures. Determinism, checkpoint integrity, data quality, cache correctness, stopping behavior, and failure recovery will be treated as testable requirements. The final objective is a Transformer whose architecture, training data, optimization, numerical behavior, computational limits, decoding policy, and deployment assumptions can all be explained and audited.

17.1.2 Why this book comes after the LSTM

The RNN and LSTM books established the essential problem of sequence learning: the meaning of an element often depends on what came before it, and a model must preserve useful information while discarding irrelevant detail. A vanilla RNN stores the past in one evolving hidden state. The LSTM improves this mechanism with a cell state and gates that regulate writing, retention, and exposure. These gates create a more stable path for gradients and allow substantially longer dependencies to be learned, but they do not remove the recurrent chain. To compute the state at time t, the model still requires the state at time $t - 1$.

This recurrence creates two connected limitations. First, the training computation has sequential depth proportional to sequence length, restricting parallel execution across time steps. Second, information from distant positions must repeatedly pass through state transitions. Even if the LSTM retains that information successfully, the path through which one position influences another remains long and compressed. Attention changes the topology of communication. It gives a position a direct, learned connection to every permitted source position, so the interaction path does not grow with their distance in the sequence.

The transition is not a claim that LSTMs became useless or that attention is free. Recurrent models can be effective for streaming inputs, small datasets, strict memory budgets, or tasks where constant-size state is valuable. Dense self-attention pays an explicit quadratic price in sequence length and requires positional information because recurrence no longer supplies order implicitly. Transformers also introduce new failure modes: mask leakage, attention-score instability, enormous activation memory, cache-management errors, and expensive long-context inference. This book comes after the LSTM so that the reader can evaluate the Transformer as a different engineering trade-off, understanding exactly which recurrent limitations it solves and which new constraints it creates.

17.1.3 How to study this book

Begin every exercise by writing the semantic contract of each tensor. For an attention calculation, record the batch axis, head axis, query length, key length, and feature dimension before writing any matrix multiplication. State which positions are allowed to communicate and con-

struct the expected mask independently of the model code. For a decoder, specify whether an index refers to an input token, a target token, a cached position, or the next position to be generated. Names such as `seq_len` or `mask` are not enough; the meaning of every axis and every Boolean value must be explicit.

Use three levels of implementation. First, build a tiny manual case that can be verified with arithmetic and deliberately simple weights. Second, implement the operation using basic PyTorch tensor functions so that reshaping, broadcasting, masking, and normalization remain visible. Third, compare the result with an optimized library implementation under matched inputs, parameters, dtypes, and modes. Agreement should be tested for forward outputs, attention weights when available, and gradients where appropriate. The manual implementation explains the mechanism; the optimized implementation provides performance; parity establishes that both compute the intended operation.

Treat every comparison as a controlled experiment. Fix data, initialization, random seeds, optimization budget, evaluation procedure, and stopping rules before changing the factor being studied. GPU timing must include warm-up and explicit synchronization because kernel launches are asynchronous. Memory experiments should record allocated and reserved memory, reset peak statistics, and distinguish parameter, optimizer-state, activation, attention-matrix, and cache costs. A single measurement is not a scaling law; repeat measurements and explain the range over which the conclusion is valid.

Failure injection is a required learning method. Reverse a causal mask, apply softmax across the query axis, omit the scaling factor, misalign the target by one position, retain a cache twice, or run a normalization in an unsafe precision. Observe which tests detect the defect and which metrics remain misleadingly normal. A good debugging exercise ends with the earliest violated invariant, the causal explanation, the correction, and a regression test that would detect the same problem if it returned.

Do not interpret an attention heat map as a complete explanation of model reasoning. Attention weights show how values are mixed at a particular layer and head under a particular input, but later layers, residual paths, value vectors, and output projections also affect the final result. Visualization is useful for detecting mask errors, local patterns, alignment, head redundancy, or unexpected concentration, yet causal claims require interventions such as head ablation, token perturbation, or controlled counterfactual tests. Separate what the visualization displays from what the experiment proves.

Keep an engineering notebook for every exercise. Record the hypothesis, formula, tensor contracts, configuration, seed, device, precision, expected failure, raw measurements, observed behavior, and final interpretation. Save small reference tensors and failed examples when they are useful. By the end of the book, these records should form a reusable Transformer debugging and design manual rather than a disconnected collection of code snippets.

17.1.4 The Chapters (TRF00 - TRF08) included in this book

- TRF00 - Attention Intuition and Transformer Motivation
- TRF01 - Multi-Head Attention and Projection Geometry

- TRF02 - Positional Encoding and Sequence Order Modeling
- TRF03 - Transformer Block: Residuals, LayerNorm, and FFN
- TRF04 - Tokenization, Embeddings, and Vocabulary Pipeline
- TRF05 - Training Loop, Optimization, and Stability
- TRF06 - Decoder-Only Language Modeling and Generation
- TRF07 - Encoder-Decoder, Cross-Attention, and Seq2Seq
- TRF08 - Efficiency, Long Context, and Production Readiness

17.2 TRF00

Chapter Title: Attention Intuition and Transformer Motivation
Specialization: Models and Training

17.2.1 Exercise 00

Exercise Title: Explain Why Sequential Models Limit Parallelism and How Attention Enables Full Parallel Processing

Part 1 - Why this matters

A recurrent model processes a sequence by repeatedly updating a state. At time t, the new state depends on the state produced at time $t-1$. This dependency is not merely a programming choice that can be removed by placing a loop on the GPU. It is part of the computation graph: the value of h_t is unknown until h_{t-1} has been computed. Matrix operations inside one recurrent step can be parallelized, and many sequences in a batch can be processed together, but the time dimension still forms a critical path of length T. An LSTM improves what can survive along that path; it does not eliminate the path itself.

This distinction explains why sequence length affects recurrent training in a way that differs from a feed-forward stack. Longer sequences create more dependent state transitions, a longer backpropagation-through-time graph, and fewer opportunities to combine all token computations into a small number of large accelerator-friendly kernels. The hidden state also acts as a repeatedly updated communication channel. Information from a distant token must survive many transformations before it can influence a later output, even when the model has gates designed to protect memory.

Self-attention changes the communication graph. Instead of forcing information to move from position 1 to 2, then from 2 to 3, every query position can compare itself directly with every permitted key position. All queries, keys, and values for a training sequence can be constructed at once, and their interactions can be expressed using batched matrix multiplication. The path between two positions becomes short, and the GPU receives large regular tensor operations rather than a long chain of small dependent steps. This is the central systems motivation behind the Transformer.

However, full parallel processing must be stated precisely. A causal Transformer can process all positions of a known training sequence in parallel because the target tokens already exist and a mask prevents each position from using the future. During autoregressive generation, the next token does not exist until the current token has been sampled, so generation remains sequential across output steps. Attention removes recurrence from the network architecture; it does not repeal the causal dependency of producing an unknown future token.

The gain also has a price. Dense self-attention creates a $T \times T$ score matrix. It provides a short interaction path and high parallelism, but its arithmetic and activation memory grow quadratically with sequence length. Therefore this exercise must compare more than wall-clock speed. You should separate computational depth, total work, memory growth, training parallelism, and decoding latency. The conclusion is not that attention is always cheaper; it is that attention reorganizes sequence computation into a form that trades quadratic interaction cost for direct communication and efficient parallel hardware utilization.

Part 2 - Mathematical form and reasoning

For a recurrent model,

$$h_t = f_\theta(x_t, h_{t-1}), \qquad y_t = g_\theta(h_t), \qquad t = 1, \ldots, T.$$

The dependency graph contains the chain

$$h_0 \to h_1 \to \cdots \to h_T,$$

so the minimum sequential depth across time is $\mathcal{O}(T)$. In self-attention, all token representations are projected simultaneously:

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V,$$

$$A = \operatorname{softmax}\left(\frac{QK^\mathsf{T}}{\sqrt{d_k}} + M\right), \qquad O = AV.$$

Any permitted pair of positions can interact within the same attention layer, giving an interaction-path length of $\mathcal{O}(1)$ for that layer. For dense self-attention, the usual score computation requires $\mathcal{O}(T^2 d_k)$ work and stores $\mathcal{O}(T^2)$ attention entries. These quantities describe different properties: short dependency depth does not imply linear total cost.

Part 3 - Hints, controls, and evidence

1. Draw both dependency graphs before writing code. Mark which operations can run simultaneously and which require a previously computed value.

2. Implement a deliberately simple recurrent reference with an explicit time loop and an attention reference with batched matrix multiplications.

3. Benchmark several sequence lengths using identical batch size and feature dimension. Include warm-up iterations and call `torch.cuda.synchronize()` around timed CUDA regions.

4. Record latency, tokens per second, and peak allocated memory. Do not use one timing number as evidence for every claim.

5. Run a separate experiment for teacher-forced training and autoregressive generation. Explain why their parallelism differs.

6. Verify numerical outputs and shapes before benchmarking; a faster program that computes a different operation is not a valid comparison.

Your report must distinguish theoretical dependency depth from measured runtime, identify the sequence-length range tested, and explain where dense attention's quadratic cost begins to dominate.

Part 4 - Python code skeleton

```
import time
import torch

def recurrent_reference(x, w_x, w_h, h0):
    # x: (batch, seq, d_model)
    h = h0
    states = []
    for t in range(x.shape[1]):
        # TODO: compute the next dependent state
        h = ...
        states.append(h)
    return torch.stack(states, dim=1)

def parallel_attention_reference(x, w_q, w_k, w_v):
    # TODO: construct Q, K, and V for every position at once
    q = ...
    k = ...
    v = ...
    scores = ...
    weights = ...
    output = ...
    return output, weights

def benchmark(fn, *args, repeats=50):
    # TODO: add warm-up, CUDA synchronization, and peak-memory reset
    timings = []
    for _ in range(repeats):
        start = time.perf_counter()
        result = fn(*args)
        # TODO: synchronize when the tensors are on CUDA
        timings.append(time.perf_counter() - start)
    return result, timings

def E00(device):
    results = {}
    for seq_len in [16, 32, 64, 128, 256, 512]:
        x = ...
```

```
        # TODO: benchmark both mechanisms under controlled settings
        results[seq_len] = {
            "recurrent_latency": ...,
            "attention_latency": ...,
            "attention_peak_memory": ...,
        }
    return results
```

Part 5 - The Philosophy Behind the Exercise

Recurrence and attention represent two different ideas about how knowledge moves. Recurrence says that the past reaches the present through a continuously updated state. Attention says that the present can ask the past a new question and construct a response from the relationships that are relevant now. One organizes memory through temporal continuity; the other organizes access through content-dependent connection.

The historical success of the Transformer is therefore not explained by abstract model expressiveness alone. It arose from an alignment between a mathematical idea and the physical organization of modern hardware. Matrix multiplication, memory bandwidth, accelerator parallelism, and large datasets helped determine which architecture could be trained at scale. The evolution of model design is partly a history of ideas about intelligence and partly a history of available computation.

This exercise should also produce intellectual caution. A shorter information path is not the same as understanding, and parallel execution is not the same as efficient execution at every scale. Attention gives the model a powerful way to relate positions, but the relations are learned statistical operations subject to data, optimization, masking, and resource limits. The deeper lesson is to judge an architecture by the structure of access it creates, the costs it imposes, and the evidence that its claimed advantage actually appears.

17.2.2 Exercise 01

Exercise Title: Define Query Key Value and Explain Attention as Weighted Retrieval from a Memory Bank

Part 1 - Why this matters

Query, Key, and Value are often introduced as three names attached to three linear layers. That description is mechanically correct but conceptually insufficient. A query represents what the current position is looking for. A key represents how a candidate memory location advertises the information it can provide. A value represents the content that will actually be retrieved if that location is selected. Compatibility is computed between queries and keys, but the final output is constructed from values.

This separation prevents an important misunderstanding: high similarity does not cause the key vector itself to be copied into the output. The key participates in addressing; the value participates in content transfer. A database analogy is useful if handled carefully. A query is not an exact lookup string, keys are not immutable identifiers, and attention usually does not retrieve only one record. The model learns continuous projections and performs soft retrieval, combining several values according to normalized compatibility weights. The memory bank is

differentiable and changes during training.

In self-attention, Q, K, and V originate from the same sequence but pass through different learned projections. A token can therefore ask one kind of question, expose a different set of searchable features, and contribute yet another content representation. In cross-attention, queries may come from a decoder while keys and values come from an encoder. The same mathematical operation then becomes a bridge between two sources of representation.

The weighted-retrieval interpretation is also necessary for debugging. If query and key axes are exchanged, the program may still multiply compatible matrices but answer the reverse question. If values are transposed incorrectly, attention weights may appear plausible while the retrieved output mixes the wrong dimension. If a mask is applied to values rather than scores, the system no longer removes a memory location from the probability distribution. Each error violates a semantic role, not merely a shape rule.

Finally, attention weights must not be confused with stored facts or calibrated confidence. They are conditional mixing coefficients produced by one layer, head, and input. A large weight indicates that a value contributes strongly to that layer's output under the current projections. It does not by itself prove that the corresponding token is true, important to the final decision, or the unique cause of the prediction.

Part 2 - Mathematical form and reasoning

Let

$$X_q \in \mathbb{R}^{B\times T_q\times d_{\text{model}}}, \qquad X_m \in \mathbb{R}^{B\times T_k\times d_{\text{model}}}.$$

The projections are

$$Q = X_qW_Q, \qquad K = X_mW_K, \qquad V = X_mW_V,$$

with

$$Q \in \mathbb{R}^{B\times T_q\times d_k}, \quad K \in \mathbb{R}^{B\times T_k\times d_k}, \quad V \in \mathbb{R}^{B\times T_k\times d_v}.$$

For query position i and memory position j,

$$s_{ij} = \frac{q_i^{\mathsf{T}}k_j}{\sqrt{d_k}}, \qquad \alpha_{ij} = \frac{\exp(s_{ij})}{\sum_{\ell=1}^{T_k}\exp(s_{i\ell})},$$

$$o_i = \sum_{j=1}^{T_k}\alpha_{ij}v_j.$$

The sum over j makes the retrieval explicit: one output is a convex combination of value vectors when the weights are non-negative and sum to one.

Part 3 - Hints, controls, and evidence

1. Construct a memory bank with three values that have easily recognized patterns, such as basis vectors or colors encoded numerically.

2. Choose keys and queries manually so that one query strongly matches one key, another matches two keys equally, and a third matches none strongly.

3. Calculate the scores, row-wise softmax, and weighted sum by hand before running PyTorch.

4. Change a value while keeping its key fixed. The weights should remain the same while the retrieved content changes.

5. Change a key while keeping its value fixed. The retrieval weights should change even though the available content does not.

6. Verify that every row of the weight matrix sums to one and that the output lies within the span of the value vectors.

Include one self-attention example and one cross-attention example in your notes, and label the semantic meaning of both sequence axes.

Part 4 - Python code skeleton

```
import math
import torch

def weighted_retrieval(query, keys, values):
    # query: (batch, query_len, d_k)
    # keys:  (batch, memory_len, d_k)
    # values:(batch, memory_len, d_v)
    scores = ...  # TODO: compare every query with every key
    scores = scores / ...
    weights = ...  # TODO: normalize over memory locations
    retrieved = ...  # TODO: mix values, not keys
    return retrieved, weights

def intervention_test(query, keys, values):
    base_output, base_weights = weighted_retrieval(
        query, keys, values
    )

    changed_values = values.clone()
    # TODO: modify content without modifying its address
    output_v, weights_v = ...

    changed_keys = keys.clone()
    # TODO: modify an address without modifying its content
    output_k, weights_k = ...

    return {
        "weights_unchanged_after_value_edit": ...,
        "output_changed_after_value_edit": ...,
        "weights_changed_after_key_edit": ...,
    }

def E01():
```

```
query = ...
keys = ...
values = ...
output, weights = weighted_retrieval(query, keys, values)
assert torch.allclose(weights.sum(dim=-1),
                      torch.ones_like(weights.sum(dim=-1)))
return output, weights, intervention_test(query, keys, values)
```

Part 5 - The Philosophy Behind the Exercise

Query, Key, and Value encode a subtle idea: information is not retrieved in isolation from the question being asked. The same memory can yield different representations because different queries produce different weight distributions. In this sense, attention treats relevance as relational rather than intrinsic. A token is not universally important; it becomes important with respect to a particular request for information.

The mechanism also separates identity from content. A key determines how something may be found, while a value determines what is received. Human institutions make similar distinctions in indexes, labels, catalogues, and search systems. Yet learned attention goes further: both the index and the content representation are optimized for a task, so the model constructs its own internal vocabulary of questions and addresses.

This flexibility is powerful but epistemically limited. Weighted retrieval can produce a useful representation without establishing that the source is true, causal, or sufficient. Attention decides what to mix, not what deserves belief. The philosophical lesson is that access, relevance, and truth are different properties, and an intelligent system can become excellent at the first two without automatically possessing the third.

17.2.3 Exercise 02

Exercise Title: Implement Scaled Dot-Product Attention from Scratch Using Pure Torch Operations

Part 1 - Why this matters

Scaled dot-product attention is the irreducible computational core from which the later Transformer chapters are built. Implementing it from basic tensor operations forces every hidden assumption to become visible: which axis stores queries, which stores keys, where the transpose occurs, how the scale is chosen, when the mask is applied, which dimension softmax normalizes, and how the probability matrix mixes values. Using a high-level attention layer before understanding these decisions makes later failures difficult to localize.

The operation contains two matrix multiplications with different meanings. The first, QK^{T}, creates pairwise compatibility scores. Its result has a query axis and a key axis; it does not yet contain retrieved features. The second, AV, uses normalized weights to aggregate the value vectors. Confusing these stages can produce code with valid matrix dimensions but invalid semantics. The score tensor describes relationships, whereas the output tensor contains new token representations.

The order of scaling, masking, and normalization matters. Scaling belongs to the logits before softmax. A Boolean mask must be converted into an additive restriction or used with a masked fill before normalization, ensuring forbidden positions receive zero probability. Applying a mask after softmax leaves the remaining weights unnormalized unless a second normalization is performed. Applying it after value aggregation cannot undo information that has already leaked.

Your scratch implementation should be a transparent reference, not an attempt to outperform optimized kernels. It should optionally return scores and weights, assert shape contracts, handle self- and cross-attention, and define behavior for masked rows. Once correct, it can be compared with `torch.nn.functional.scaled_dot_product_attention` or a manually configured module. Forward parity alone is insufficient; gradients with respect to Q, K, and V must also agree on a tiny deterministic case.

This reference becomes infrastructure for the rest of the book. Causal masks, padding masks, dropout, multi-head reshaping, precision experiments, and performance studies all depend on the same core. A trustworthy baseline lets you distinguish an error in the mathematical attention rule from an error introduced by head splitting, caching, kernel selection, or model integration.

Part 2 - Mathematical form and reasoning

For

$$Q \in \mathbb{R}^{B\times H\times T_q\times d_k}, \quad K \in \mathbb{R}^{B\times H\times T_k\times d_k}, \quad V \in \mathbb{R}^{B\times H\times T_k\times d_v},$$

compute

$$S = \frac{QK^{\mathsf{T}}}{\sqrt{d_k}} \in \mathbb{R}^{B\times H\times T_q\times T_k}.$$

With an additive mask M broadcastable to S,

$$\widetilde{S} = S + M, \qquad A = \operatorname{softmax}(\widetilde{S}, \dim = -1),$$

$$O = AV \in \mathbb{R}^{B\times H\times T_q\times d_v}.$$

The last score dimension must correspond to keys because each query chooses a distribution over memory locations. When dropout is absent and at least one key is valid, $\sum_{j=1}^{T_k} A_{bhij} = 1$.

Part 3 - Hints, controls, and evidence

1. Start with $B = H = 1$, two queries, three keys, and small integer vectors. Calculate the complete result manually.
2. Assert $Q.shape[-1] = K.shape[-1]$ and $K.shape[-2] = V.shape[-2]$ before multiplication.
3. Save the unscaled scores, scaled scores, masked scores, weights, and output separately. These artifacts reveal the first incorrect stage.
4. Test no mask, a padding mask, and a causal mask. Confirm forbidden positions receive exactly zero probability within the chosen tolerance.

5. Compare forward outputs and gradients with a trusted implementation in `float64` before testing lower precision.

6. Test cross-attention where $T_q \neq T_k$ so an accidental self-attention assumption cannot remain hidden.

Do not accept the implementation until manual arithmetic, shape assertions, probability invariants, reference parity, and gradient parity all pass.

Part 4 - Python code skeleton

```
import math
import torch

def scratch_attention(q, k, v, mask=None, return_debug=False):
    assert q.shape[-1] == k.shape[-1]
    assert k.shape[-2] == v.shape[-2]

    d_k = q.shape[-1]
    raw_scores = ...  # TODO: Q @ K^T
    scaled_scores = ...

    if mask is not None:
        # Decide and document whether True means keep or block.
        masked_scores = ...
    else:
        masked_scores = scaled_scores

    weights = ...  # TODO: normalize over key positions
    output = ...   # TODO: weighted sum of values

    debug = {
        "raw_scores": raw_scores,
        "scaled_scores": scaled_scores,
        "masked_scores": masked_scores,
        "weights": weights,
    }
    return (output, debug) if return_debug else output

def compare_gradients(q, k, v, mask=None):
    # TODO: clone inputs for scratch and reference paths
    # TODO: use the same scalar loss for both paths
    # TODO: compare q.grad, k.grad, and v.grad
    return ...

def E02():
    torch.manual_seed(7)
    q = ...
    k = ...
```

```
v = ...
mask = ...
output, debug = scratch_attention(
    q, k, v, mask=mask, return_debug=True
)
# TODO: add probability, mask, and parity assertions
return output, debug, compare_gradients(q, k, v, mask)
```

Part 5 - The Philosophy Behind the Exercise

Implementing attention from scratch is an exercise in refusing premature abstraction. High-level tools are essential for real systems, but abstraction becomes trustworthy only after its invariants are understood. By exposing every operation, the exercise turns a famous architectural slogan into a sequence of claims that can be inspected, falsified, and repaired.

The two matrix multiplications also reveal a broader pattern of intelligence: first determine relationships, then use those relationships to transform content. Attention does not merely move existing vectors. It constructs a context-dependent representation in which each output is defined partly by the other positions available to it. Meaning becomes relational within the current computational context.

There is a methodological lesson as well. A model implementation is not correct because it resembles an equation, and an equation is not implemented faithfully because the code produces a tensor. Correctness lives in the correspondence between symbols, axes, allowed information, numerical operations, and observed behavior. This discipline is what makes later optimization safe.

17.2.4 Exercise 03

Exercise Title: Verify Attention Shapes Step by Step for (batch, heads, seq, d_k) and Debug Dimension Errors

Part 1 - Why this matters

Most attention bugs begin as errors in meaning and appear as errors in shape only when the program is fortunate. The standard four-dimensional layout `(batch, heads, seq, d_k)` assigns a semantic role to every axis. The batch axis separates examples, the head axis separates projection subspaces, the sequence axis identifies positions, and the final axis stores the feature coordinates used by each head. A tensor with these sizes in a different order is not equivalent, even if every number in its shape is familiar.

The score computation requires the feature dimensions of queries and keys to contract while preserving both sequence axes. Transposing the last two dimensions of K produces a key-feature-by-key-position view, allowing `Q @ K.transpose(-2, -1)` to yield `(batch, heads, query_seq, key_seq)`. Softmax then operates over the key-sequence axis because each query forms a distribution over candidate memory positions. Multiplying by V removes the key-sequence axis and restores the value-feature dimension.

Self-attention often hides mistakes because query length and key length are equal. A wrong transpose may accidentally preserve a square shape. A mask may broadcast in an unintended

direction yet still fit. Head count and sequence length may even share the same numerical value in a small test, allowing axes to be swapped without an exception. This exercise must therefore use unequal and deliberately distinctive dimensions, including a cross-attention case with $T_q \neq T_k$.

Head splitting adds another source of risk. Starting from `(B,T,d_model)`, the projected tensor is reshaped into `(B,T,H,d_k)` and then permuted into `(B,H,T,d_k)`. Reversing the reshape and permutation order can mix feature coordinates between heads. After attention, the output must be transposed back and made contiguous or reshaped safely before concatenation. The fact that `view` sometimes fails on non-contiguous tensors is a warning that memory layout and logical shape are related but distinct.

Good shape debugging is proactive. Instead of printing shapes after a runtime error, define contracts at module boundaries and assert them before expensive operations. Record symbolic names beside actual dimensions, verify mask broadcasting explicitly, and include tests in which every axis has a different size. Shape reasoning should function as a lightweight type system for tensor programs.

Part 2 - Mathematical form and reasoning

Let

$$Q \in \mathbb{R}^{B\times H\times T_q\times d_k}, \quad K \in \mathbb{R}^{B\times H\times T_k\times d_k}, \quad V \in \mathbb{R}^{B\times H\times T_k\times d_v}.$$

Then

$$K^{\mathsf{T}_{\text{last 2}}} \in \mathbb{R}^{B\times H\times d_k\times T_k},$$

$$QK^{\mathsf{T}_{\text{last 2}}} \in \mathbb{R}^{B\times H\times T_q\times T_k},$$

and

$$(B,H,T_q,T_k)\,(B,H,T_k,d_v) \longrightarrow (B,H,T_q,d_v)\,.$$

For equal-sized heads, $d_{\text{model}} = Hd_k$. The re-indexing path is

$$(B,T,Hd_k) \to (B,T,H,d_k) \to (B,H,T,d_k).$$

These operations must reorganize indices without mixing feature values between heads or examples.

Part 3 - Hints, controls, and evidence

1. Use $B = 2$, $H = 3$, $T_q = 5$, $T_k = 7$, $d_k = 11$, and $d_v = 13$ so every axis can be identified by size.
2. Write expected shapes before each `reshape`, `transpose`, `matmul`, mask application, and softmax.
3. Test masks shaped as `(B,1,1,T_k)`, `(1,1,T_q,T_k)`, and their deliberate invalid variants.
4. Fill tensors with index-coded values so an axis swap changes observable content, not just metadata.

5. Compare `view`, `reshape`, and `contiguous().view(...)` after permutation and explain their behavior.

6. Add failure tests that require a clear assertion message naming the expected semantic axes.

Your evidence should include a shape trace for self-attention, a separate trace for cross-attention, and at least three intentionally injected dimension errors.

Part 4 - Python code skeleton

```
import torch

def assert_shape(tensor, expected, name):
    actual = tuple(tensor.shape)
    assert actual == tuple(expected), (
        f"{name}: expected {expected}, got {actual}"
    )

def split_heads(x, num_heads):
    # x: (batch, seq, d_model)
    batch, seq_len, d_model = x.shape
    assert d_model % num_heads == 0
    d_k = d_model // num_heads
    x = ...  # TODO: (batch, seq, heads, d_k)
    x = ...  # TODO: (batch, heads, seq, d_k)
    return x

def merge_heads(x):
    # x: (batch, heads, seq, d_v)
    x = ...  # TODO: restore (batch, seq, heads, d_v)
    # TODO: explain why reshape is safe at this point
    return ...

def traced_attention(q, k, v, mask=None):
    trace = {"q": tuple(q.shape), "k": tuple(k.shape),
             "v": tuple(v.shape)}
    k_t = ...
    trace["k_transposed"] = tuple(k_t.shape)
    scores = ...
    trace["scores"] = tuple(scores.shape)
    if mask is not None:
        # TODO: validate intended broadcasting explicitly
        ...
    weights = ...
    output = ...
    trace["output"] = tuple(output.shape)
    return output, trace
```

```
def E03():
    dims = dict(B=2, H=3, Tq=5, Tk=7, Dk=11, Dv=13)
    q = ...
    k = ...
    v = ...
    # TODO: run valid and intentionally invalid cases
    return traced_attention(q, k, v)
```

Part 5 - The Philosophy Behind the Exercise

Tensor dimensions are often treated as bookkeeping, but in attention they are part of the theory. An axis states what kinds of entities exist and which ones may interact. Contracting the feature axis means comparing representations; normalizing the key axis means choosing among memory locations; preserving the batch axis means refusing to mix independent examples. Shape is therefore a compact language of ontology and permission.

Many silent bugs arise because numerical compatibility is weaker than semantic compatibility. Two dimensions may both equal 64, allowing an operation that is mathematically defined but conceptually meaningless. This resembles a sentence that is grammatically valid yet refers to the wrong objects. Naming axes and constructing asymmetric tests restores meaning that raw dimensions alone cannot express.

The larger lesson is that abstraction requires contracts. As models grow, no person can inspect every element, but researchers can still reason about the roles, invariants, and transformations of dimensions. Shape discipline is not clerical caution; it is one of the main ways human understanding remains connected to large tensor computations.

17.2.5 Exercise 04

Exercise Title: Implement Softmax with Numerical Stability and Prove It Prevents NaNs in Attention Scores

Part 1 - Why this matters

Softmax converts unrestricted attention logits into non-negative weights that sum to one, but its textbook formula hides a serious numerical problem. Exponentiation grows rapidly. In exact mathematics, a large finite logit has a large finite exponential. In finite floating-point arithmetic, that exponential may overflow to infinity. Dividing infinity by an infinite sum produces an undefined result, allowing a single extreme score to turn an entire attention row into NaNs. Lower-precision formats reach this failure at much smaller magnitudes.

The stable implementation subtracts the maximum logit from every element in the same normalization row before exponentiation. Softmax is invariant to adding or subtracting a constant from every element, so the mathematical probability distribution is unchanged. Numerically, the largest shifted logit becomes zero, its exponential becomes one, and every other exponential lies between zero and one. This does not make every possible input safe, but it eliminates the most direct overflow path while preserving the intended result.

Attention introduces additional complications. Masks are often represented by very negative additive values or negative infinity. A row in which every key is masked has no valid probability distribution; subtracting the maximum of an all-negative-infinity row produces undefined arithmetic. The implementation must either guarantee at least one valid key per query, define a zero-output policy for empty rows, or explicitly detect and reject the condition. Numerical stability cannot repair an ill-defined semantic input.

Precision comparisons must examine more than NaNs. Even finite low-precision logits may quantize small differences, causing softmax to become unexpectedly uniform or overly concentrated. When probabilities approach zero, gradients through those entries may also become too small to represent. You should test overflow, underflow, saturation, probability-sum error, output error, and gradient error separately. A computation can avoid NaNs yet still lose useful learning signal.

To prove the stable method matters, construct adversarial logits that make the naive method fail, compare both against a high-precision reference, and sweep the magnitude of the inputs across dtypes. The result should identify the failure boundary rather than display one convenient example. The final implementation must also normalize along the correct key axis and document how masked rows are handled.

Part 2 - Mathematical form and reasoning

The naive formula is

$$\operatorname{softmax}(z)_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}.$$

For any constant c,

$$\frac{\exp(z_i - c)}{\sum_j \exp(z_j - c)} = \frac{\exp(z_i)\exp(-c)}{\exp(-c)\sum_j \exp(z_j)} = \operatorname{softmax}(z)_i.$$

Choose $c = m = \max_j z_j$. Then $z_i - m \le 0$ for every i, so

$$0 < \exp(z_i - m) \le 1$$

for finite logits. In attention, stabilization is applied independently to every row:

$$A_{bhij} = \frac{\exp(S_{bhij} - m_{bhi})}{\sum_\ell \exp(S_{bhi\ell} - m_{bhi})}, \qquad m_{bhi} = \max_j S_{bhij}.$$

Part 3 - Hints, controls, and evidence

1. Implement naive and stable softmax without calling `torch.softmax`. Keep the normalization dimension configurable.
2. Test logits such as $[1000, 1001, 1002]$, their negative equivalents, and scaled random attention scores.
3. Sweep logit magnitude for `float64`, `float32`, and `float16`; record the first non-finite output for the naive method.

4. Compare the stable output with `torch.softmax` and verify row sums, non-negativity, finiteness, and gradient parity.

5. Add masks containing valid and invalid rows. Do not silently claim that an all-masked row has a meaningful softmax distribution.

6. Plot maximum probability and gradient norm against logit scale to reveal saturation even when no NaN occurs.

Your proof must combine algebraic invariance with empirical evidence showing where finite-precision implementations diverge.

Part 4 - Python code skeleton

```
import torch

def naive_softmax(logits, dim=-1):
    numerator = ...
    denominator = ...
    return ...

def stable_softmax(logits, dim=-1):
    row_max = ...
    shifted = ...
    exp_shifted = ...
    denominator = ...
    return ...

def validate_probabilities(p, dim=-1):
    checks = {
        "finite": ...,
        "non_negative": ...,
        "sums_to_one": ...,
    }
    return checks

def magnitude_sweep(dtype, device):
    records = []
    for magnitude in [1, 10, 50, 100, 500, 1000, 5000]:
        logits = ...
        naive = naive_softmax(logits)
        stable = stable_softmax(logits)
        reference = torch.softmax(logits.to(torch.float64), dim=-1)
        records.append({
            "magnitude": magnitude,
            "naive_finite": ...,
            "stable_finite": ...,
            "stable_reference_error": ...,
        })
```

```
    return records

def E04(device):
    results = {}
    for dtype in [torch.float64, torch.float32, torch.float16]:
        # TODO: skip unsupported combinations explicitly
        results[str(dtype)] = magnitude_sweep(dtype, device)
    # TODO: add a deliberate all-masked-row experiment
    return results
```

Part 5 - The Philosophy Behind the Exercise

Stable softmax demonstrates that mathematical equivalence does not guarantee computational equivalence. Two formulas can define the same real-valued function while behaving completely differently on a machine. Numerical analysis is therefore not an implementation detail added after the model's meaning has been established. It determines whether the mathematical object exists in practice.

Subtracting the maximum is also a lesson in choosing representations. The absolute level of softmax logits is irrelevant to the final distribution; only their differences matter. By removing an irrelevant degree of freedom, the stable formulation places the computation in a numerically safer coordinate system. Good engineering often comes from identifying what the problem does not need to preserve.

Finally, the all-masked row exposes the boundary between numerical and semantic failure. No clever arithmetic can create a meaningful choice when every option has been declared impossible. A responsible system must distinguish the case where a calculation became unstable from the case where the requested probability distribution is undefined. The philosophy of robust AI begins with refusing to hide invalid questions behind finite-looking outputs.

17.2.6 Exercise 05

Exercise Title: Demonstrate How Attention Focus Shifts When One Token Changes and Interpret the Weight Matrix

Part 1 - Why this matters

Attention is input-dependent. Unlike a fixed convolutional kernel or a static averaging rule, its mixing weights are recomputed from the current queries and keys. Changing one token can therefore alter its own query, its advertised key, its contributed value, and the responses of other positions that compare against it. This exercise makes that dependency visible by applying a controlled token intervention and tracing the resulting changes through every stage of attention.

The attention matrix has two different sequence axes. A row belongs to one query position and describes how that query distributes weight across key positions. A column belongs to one key position and collects the weights assigned to it by many queries. Row-wise interpretation asks where this position retrieved information from. Column-wise patterns suggest which queries selected this position. Confusing these interpretations can reverse the apparent direction of a relationship.

A token intervention can affect attention in several ways. If the changed token modifies only a value vector, the attention matrix should remain fixed while outputs that retrieve that value change. If it modifies only a key, the column associated with that memory position may gain or lose weight across queries. If it modifies only a query, the corresponding row should change while other rows remain fixed. In a normal self-attention layer, one input token usually affects all three projections, so these mechanisms occur together. Separating them experimentally is essential for causal interpretation.

Visualization alone is not enough. Softmax couples every key within a row: increasing the score of one position necessarily changes the normalized weights of others. A large color difference in a heat map may come from a small score change near a competitive boundary, while a large score change in an already saturated row may barely alter the distribution. You should therefore record raw scores, normalized weights, entropy, outputs, and projection changes rather than relying only on color intensity.

The goal is not to claim that attention weights fully explain the model. The value vectors, output projection, residual path, later layers, and nonlinear subnetworks all influence the final prediction. This exercise teaches a narrower and more defensible interpretation: the matrix reveals how one attention operation mixed values under a particular input, head, layer, mask, and parameter state.

Part 2 - Mathematical form and reasoning

For input representations X, let

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V,$$

$$A(X) = \operatorname{softmax}\left(\frac{QK^{\mathsf{T}}}{\sqrt{d_k}} + M\right), \qquad O(X) = A(X)V.$$

Change only token r:

$$X' = X + \Delta, \qquad \Delta_i = 0 \text{ for } i \neq r.$$

Measure

$$\Delta A = A(X') - A(X), \qquad \Delta O = O(X') - O(X).$$

Useful row statistics include attention entropy

$$H_i = -\sum_j A_{ij} \log(A_{ij} + \varepsilon)$$

and total distribution change

$$D_i = \frac{1}{2} \sum_j \left| A'_{ij} - A_{ij} \right|.$$

These quantities show whether a query became more concentrated and how far its retrieval distribution moved.

Part 3 - Hints, controls, and evidence

1. Create a short synthetic sequence with distinct token roles, then save a baseline containing Q, K, V, scores, weights, and outputs.
2. Perform three isolated interventions: change only q_r, only k_r, and only v_r. Predict which rows, columns, or outputs should change.
3. Perform a realistic input-token intervention through all three projection matrices and compare it with the isolated cases.
4. Report maximum absolute change, row-wise total variation, entropy change, and output-vector change.
5. Keep model parameters, mask, seed, dtype, and every other token fixed.
6. Use heat maps with identical color scales and labels. A rescaled plot can make two nearly identical matrices look dramatically different.

Your interpretation notes must separate direct effects of the edited query, key, or value from indirect changes created by row-wise normalization.

Part 4 - Python code skeleton

```
import torch

def attention_with_trace(q, k, v, mask=None):
    scores = ...
    if mask is not None:
        scores = ...
    weights = ...
    output = ...
    return {"scores": scores, "weights": weights,
            "output": output}

def distribution_metrics(before, after, eps=1e-12):
    entropy_before = ...
    entropy_after = ...
    total_variation = ...
    return {
        "entropy_change": entropy_after - entropy_before,
        "total_variation": total_variation,
    }

def isolate_intervention(q, k, v, token_index, target):
    q2, k2, v2 = q.clone(), k.clone(), v.clone()
    # TODO: change only the selected Q, K, or V token
    ...
    before = attention_with_trace(q, k, v)
    after = attention_with_trace(q2, k2, v2)
    return before, after, distribution_metrics(
        before["weights"], after["weights"]
```

```
    )

def E05():
    q, k, v = ...
    results = {}
    for target in ["query", "key", "value"]:
        results[target] = isolate_intervention(
            q, k, v, token_index=2, target=target
        )
    # TODO: add plots with shared axes and shared color limits
    return results
```

Part 5 - The Philosophy Behind the Exercise

This exercise illustrates that relevance is dynamic. The model does not carry a single permanent map of what matters; it reconstructs a map from the current representations. A changed token can alter both the question being asked and the way a possible answer presents itself. Context is therefore not merely additional data surrounding a fixed meaning. It participates in producing the relationships from which the next representation is built.

The heat map also raises a question about observation and explanation. Humans are attracted to visible matrices because they appear to reveal an internal reasoning process. Yet visibility is not causality. An attention weight is one coefficient inside a larger computation, and a beautiful pattern can be irrelevant after later projections or residual mixing. Interpretation becomes stronger when visualization is combined with controlled intervention.

The deeper philosophical principle is that understanding a system requires asking counterfactual questions: what changes if one element changes, and what remains invariant? Static inspection describes a state. Intervention reveals a mechanism. The exercise moves attention analysis from storytelling toward evidence.

17.2.7 Exercise 06

Exercise Title: Build a Tiny Synthetic Copy Task and Show Why Attention Solves Bottleneck Better Than LSTM

Part 1 - Why this matters

A synthetic copy task isolates long-range information transfer from the ambiguity of natural data. The model receives a sequence containing a payload, a delay or distractor region, and a signal indicating when the payload should be reproduced. Success requires preserving the identity and order of earlier symbols across a controllable distance. Because the generation process is known exactly, failure can be analyzed by payload length, delay length, symbol position, and error type rather than hidden inside one aggregate loss.

For an LSTM, the payload must influence later predictions through recurrent hidden and cell states. Gating makes this channel far stronger than a vanilla RNN, but its capacity is still finite and its computation remains a sequence of state updates. As the delay grows, the model must retain several discrete items while ignoring distractors and avoiding interference from

later transitions. The final state is not literally the only information available at every output step, but the recurrent path remains the route by which distant input affects the prediction.

An attention-based model can construct a direct relationship between an output query and the earlier position containing the needed symbol. Instead of asking one state to compress every relevant detail into a fixed-width evolving memory, the model preserves a set of token representations and learns where to retrieve from. The copy task makes this advantage inspectable: a correct model should place high weight on the corresponding source position when producing each payload token.

The experiment must still be fair. A Transformer with more parameters, more updates, or an easier positional representation cannot be declared superior to an undertrained LSTM. Match training examples, optimization steps, parameter count as closely as practical, and report remaining differences. Train and test across several delays, including lengths beyond the training range. The LSTM may perform perfectly on short sequences, and attention may fail if positional information or masks are wrong. The objective is to identify the boundary at which the communication structures behave differently, not to force a preselected winner.

The task also validates more than architecture. If neither model can overfit a tiny fixed dataset, the likely problem is the data alignment, loss, output shape, or training loop. If the attention model succeeds during teacher forcing but fails during generation, target leakage or exposure bias may be involved. Synthetic tasks are powerful because they convert architectural claims into controlled diagnostic evidence.

Part 2 - Mathematical form and reasoning

Let a sample contain a payload

$$p = (p_1, \ldots, p_L),$$

a delay of D blank or distractor symbols, and a delimiter. A simplified input-target pair is

$$x = (p_1, \ldots, p_L, \underbrace{\square, \ldots, \square}_{D}, \texttt{GO}),$$

$$y = (\underbrace{\square, \ldots, \square}_{L+D+1}, p_1, \ldots, p_L).$$

For the LSTM,

$$(h_t, c_t) = \mathrm{LSTM}(x_t, h_{t-1}, c_{t-1}),$$

so influence from p_i reaches a distant output through repeated transitions. For attention, an output representation may include

$$o_t = \sum_{j=1}^{T} \alpha_{tj} v_j,$$

allowing $\alpha_{t,i}$ to connect the output directly to the required payload position. Report exact sequence accuracy in addition to token accuracy:

$$\mathrm{ExactMatch} = \frac{1}{N} \sum_{n=1}^{N} \mathbf{1}\left[\widehat{p}^{(n)} = p^{(n)}\right].$$

Part 3 - Hints, controls, and evidence

1. Begin with one fixed batch and require both models to overfit it. Do not start a generalization comparison before proving pipeline correctness.
2. Use a vocabulary small enough to inspect but large enough that guessing cannot produce misleading accuracy.
3. Compare matched payload lengths while sweeping delay length. Report token accuracy, exact-match accuracy, and error by payload position.
4. Include positional information for the attention model and verify that removing it breaks order-sensitive copying.
5. Match parameter counts and optimization steps where possible; document any unmatched capacity or computational budget.
6. Visualize retrieval weights and probe LSTM states, but use output interventions and accuracy as the primary evidence.

Your report should identify the longest reliably solved delay for each model under the tested configuration and explain why the result is conditional, not a universal ranking.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def make_copy_batch(batch_size, payload_len, delay_len,
                    vocab_size, device):
    payload = ...
    inputs = ...
    targets = ...
    loss_mask = ...
    return inputs, targets, loss_mask, payload

class LSTMCopyModel(nn.Module):
    def __init__(self, vocab_size, d_model, hidden_size):
        super().__init__()
        # TODO: embedding, LSTM, and output projection
        ...

    def forward(self, tokens):
        return ...

class AttentionCopyModel(nn.Module):
    def __init__(self, vocab_size, d_model, num_heads):
        super().__init__()
        # TODO: embedding, position representation, attention, output
        ...
```

```
    def forward(self, tokens, return_weights=False):
        return ...

def evaluate_copy(model, configs):
    records = []
    for payload_len, delay_len in configs:
        inputs, targets, mask, payload = ...
        logits = ...
        # TODO: compute token accuracy and exact payload match
        records.append({...})
    return records

def E06(device):
    # TODO: match seeds, data, steps, and approximate parameter counts
    lstm = ...
    attention = ...
    # TODO: first overfit a tiny batch, then run the delay sweep
    return {
        "lstm": evaluate_copy(lstm, ...),
        "attention": evaluate_copy(attention, ...),
    }
```

Part 5 - The Philosophy Behind the Exercise

The copy task contrasts compressed memory with addressable memory. An LSTM continually rewrites a bounded state that must carry whatever the future may need. Attention preserves many representations and allows a later query to decide which one matters. The difference resembles carrying a summary of a library versus retaining access to its shelves. A summary is compact and useful for streaming, while addressable storage protects details at a higher cost.

This distinction complicates the idea of memory itself. A system does not need to place every fact into one persistent inner state to use the past. It can retain traces and reconstruct a task-specific answer when needed. Attention therefore shifts the design question from how to compress all history into a state toward how to preserve, index, and select representations.

Yet the shelves are not infinite. Dense access becomes expensive as the stored context grows, and retrieval can fail when positions, keys, or learned queries do not generalize. The philosophy is not that externalized memory defeats every bottleneck. It relocates the bottleneck from recurrent compression to storage, search, and relevance.

17.2.8 Exercise 07

Exercise Title: Implement Attention Masking for Padding Tokens and Prove Padding Does Not Affect Attention Weights

Part 1 - Why this matters

Padding lets variable-length sequences share one rectangular batch, but padded positions are computational artifacts rather than data. If they remain visible to attention, the model can allocate probability to meaningless embeddings, retrieve their values, and allow batch construction choices to alter valid token representations. The resulting system may still train because it can learn partial compensation, yet its behavior will depend on how much padding happens to surround each example.

A key-padding mask removes padded memory positions before softmax. For a score tensor shaped `(B,H,T_q,T_k)`, a per-example validity vector shaped `(B,T_k)` is commonly expanded to `(B,1,1,T_k)`. Broadcasting then applies the same invalid-key rule to every head and every query within an example. The convention must be explicit: some APIs use `True` to mean valid, while others use it to mean blocked. A reversed convention can silently expose only padding and hide real tokens.

Masking keys does not automatically erase padded query positions. If the batch contains output rows corresponding to padding, those queries can still produce distributions over valid keys. Depending on the architecture, you may zero their outputs, ignore them in the loss, or both. Key masking protects valid queries from invalid memory. Query masking and loss masking control whether invalid output positions participate downstream. These are related but different responsibilities.

The strongest proof is invariance. Replace the embeddings or values of padded tokens with extreme random numbers while keeping the mask unchanged. Outputs for valid query positions should remain equal within numerical tolerance, and all probabilities assigned to padded keys should remain zero. Repeat the same valid sequence with different amounts of right padding; its valid outputs should not change. A visual matrix with dark padding columns is helpful, but an intervention test is stronger.

Special cases require a defined policy. A sequence with no valid keys produces an all-masked row, for which ordinary softmax is undefined. Empty examples should normally be rejected or handled explicitly before attention. Low precision also affects which mask fill value is safe. The exercise must test the intended dtypes and prove that masking produces finite outputs without accidentally assigning nonzero probability to padding.

Part 2 - Mathematical form and reasoning

Let $m_{bj} \in \{0, 1\}$ indicate whether key position j in example b is valid. Define an additive mask

$$M_{b11j} = \begin{cases} 0, & m_{bj} = 1, \\ -\infty, & m_{bj} = 0. \end{cases}$$

Then

$$A_{bhij} = \frac{\exp(S_{bhij} + M_{b11j})}{\sum_{\ell=1}^{T_k} \exp(S_{bhi\ell} + M_{b11\ell})}.$$

For every padded key j,

$$m_{bj} = 0 \quad \Longrightarrow \quad A_{bhij} = 0$$

for every head h and query i, assuming at least one valid key. Consequently, changing v_{bj} at a masked position should not change

$$O_{bhi} = \sum_j A_{bhij} v_{bj}.$$

Part 3 - Hints, controls, and evidence

1. Build two sequences of different lengths and pad them into one batch. Write the validity mask manually before deriving it in code.
2. Print the score, mask, masked-score, and weight shapes. Confirm the mask broadcasts across heads and queries but not across examples.
3. Assert that masked-key weights are zero and valid-key weights sum to one.
4. Replace padded values with large positive and negative numbers; valid outputs must remain invariant.
5. Evaluate the same valid prefix with several padding lengths and compare only the valid output positions.
6. Add an empty-sequence case and enforce an explicit rejection or documented policy instead of accepting NaNs.

Also show that removing or reversing the mask makes at least one invariance test fail. A proof needs a negative control.

Part 4 - Python code skeleton

```
import torch

def make_key_padding_mask(lengths, max_len):
    positions = torch.arange(max_len, device=lengths.device)
    # Choose and document: True means valid in this exercise.
    valid = ...
    return ...  # TODO: (batch, 1, 1, key_len)

def masked_attention(q, k, v, valid_key_mask):
    scores = ...
    assert valid_key_mask.shape == (
        q.shape[0], 1, 1, k.shape[-2]
    )
    assert valid_key_mask.any(dim=-1).all(), "empty memory row"
    masked_scores = ...
    weights = ...
    output = ...
    return output, weights

def padding_invariance_test(q, k, v, mask):
```

```
    out1, w1 = masked_attention(q, k, v, mask)
    v2 = v.clone()
    # TODO: overwrite only padded values with extreme noise
    ...
    out2, w2 = masked_attention(q, k, v2, mask)
    return {
        "masked_weights_zero": ...,
        "weights_unchanged": ...,
        "valid_outputs_unchanged": ...,
    }

def E07():
    lengths = torch.tensor([3, 5])
    max_len = 7
    mask = make_key_padding_mask(lengths, max_len)
    q, k, v = ...
    return padding_invariance_test(q, k, v, mask)
```

Part 5 - The Philosophy Behind the Exercise

Padding shows that not everything present in a tensor belongs to the modeled world. Some entries exist only because hardware prefers rectangles. The mask draws a boundary between representation and reality: it tells the computation which coordinates are storage conveniences and which correspond to meaningful evidence.

This boundary cannot be inferred from numbers alone. A padding vector may be zero, learned, or accidentally similar to a real token. Without an explicit rule, the model has no principled reason to treat it as absence. Absence must be represented as a constraint on access, not merely as an unusual value.

The invariance test expresses a broad scientific ideal. If an element is declared irrelevant, changing it should not change the result. Robust systems are built not only by checking what influences an output, but by proving what must not influence it. Masking is therefore a small example of a larger ethics of information boundaries.

17.2.9 Exercise 08

Exercise Title: Implement Causal Masking for Autoregressive Modeling and Verify No Token Attends to the Future

Part 1 - Why this matters

Autoregressive language modeling trains a network to predict each token from the tokens that precede it. During training, the complete sequence is already stored in one tensor so all positions can be processed together. Without a causal mask, the representation at position i can attend to token $i+1$ or later positions, including the answer it is supposed to predict. The model may then achieve extremely low loss by copying future information rather than learning the conditional structure required during generation.

A causal mask preserves parallel training while enforcing the information boundary that exists at inference time. If rows represent query positions and columns represent key positions, query i may use key j only when $j \leq i$. The allowed region is therefore lower triangular. Memorizing a triangle without naming the axes is unsafe: transposing the score convention reverses the visual orientation while preserving the causal rule.

The mask must agree with target alignment. In the common next-token setup, `input = tokens[:-1]` and `target = tokens[1:]`. A representation at input position i may attend to its current input token because it predicts the following token, but it must not inspect later input positions. If the dataset is shifted differently, the diagonal policy must be reconsidered. Masking and label construction are one semantic contract.

A triangular heat map is only preliminary evidence. A stronger test changes future token representations while keeping a prefix unchanged. Every output belonging to that prefix must remain invariant. A gradient test can go further: differentiate an early output with respect to all input positions and verify that forbidden future positions receive zero gradient. These tests expose leakage caused by incorrect broadcasting, residual paths, or integration.

Causal masking does not make generation parallel. It allows all known training positions to be evaluated together without violating sequence probability. During generation, token $t + 1$ remains unknown until token t has been chosen. KV caching can reduce repeated work, but it cannot remove this dependency.

Part 2 - Mathematical form and reasoning

Define

$$M_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i. \end{cases}$$

Then

$$A_{ij} = \operatorname{softmax}_j\left(\frac{q_i^{\mathsf{T}} k_j}{\sqrt{d_k}} + M_{ij}\right)$$

satisfies $A_{ij} = 0$ whenever $j > i$. This enforces

$$p(x_1, \ldots, x_T) = \prod_{t=1}^{T} p(x_t \mid x_1, \ldots, x_{t-1}).$$

For a prefix ending at r, the intervention test requires

$$X'_{0:r} = X_{0:r} \quad \Longrightarrow \quad O'_{0:r} = O_{0:r},$$

even when $X'_{r+1:T} \neq X_{r+1:T}$.

Part 3 - Hints, controls, and evidence

1. Draw masks for lengths 1, 2, and 5, labeling query rows and key columns before implementing the general function.
2. State whether `True` means keep or block, and assert the exact mask shape before broadcasting it to `(B,H,T,T)`.

3. Verify every strictly future weight is zero and every valid row sums to one.
4. Perturb all future embeddings after a selected prefix and compare earlier outputs and logits.
5. Differentiate an early scalar output with respect to the complete input; forbidden future gradients must be zero.
6. Reverse the mask deliberately and prove the leakage tests fail even if training loss appears excellent.

Record the indexing convention, target shift, diagonal policy, and broadcast path. These details define what future means in the experiment.

Part 4 - Python code skeleton

```
import torch

def causal_keep_mask(query_len, key_len=None, device=None):
    key_len = query_len if key_len is None else key_len
    q_pos = torch.arange(query_len, device=device)[:, None]
    k_pos = torch.arange(key_len, device=device)[None, :]
    keep = ...  # TODO: permit keys at or before each query
    return keep[None, None, :, :]

def causal_attention(q, k, v, keep_mask):
    scores = ...
    masked_scores = ...
    weights = ...
    output = ...
    return output, weights

def future_perturbation_test(x, model, split_index):
    x2 = x.clone()
    # TODO: modify positions strictly after split_index
    ...
    out1 = model(x)
    out2 = model(x2)
    return ...  # compare only the invariant prefix

def forbidden_gradient_test(x, model, query_index):
    x = x.detach().clone().requires_grad_(True)
    output = model(x)
    scalar = ...  # select an early-query output
    scalar.backward()
    future_grad = ...
    return ...

def E08():
    mask = causal_keep_mask(6)
```

```
assert ...  # verify exact triangular structure
model, x = ..., ...
return {
    "future_invariance": future_perturbation_test(x, model, 2),
    "forbidden_gradient": forbidden_gradient_test(x, model, 2),
}
```

Part 5 - The Philosophy Behind the Exercise

A causal mask is a law governing what the model is allowed to know. It does not remove the future from storage; it removes future relationships from the computation. Training can place every token physically beside every other token while constructing a valid informational world for each prediction.

This is also a lesson about honest evaluation. A model that sees the answer is not more intelligent because it reports lower loss. It has been assigned a different and easier problem. Leakage corrupts the meaning of success while often improving its visible metrics.

Any claim about prediction or reasoning is incomplete until its information boundary is specified. Intelligence must be evaluated relative to what was available at the moment of choice. The mask turns this requirement into a tensor invariant that can be tested.

17.2.10 Exercise 09

Exercise Title: Compare Attention Outputs With and Without Scaling by sqrt(d_k) and Explain Gradient Stability Impact

Part 1 - Why this matters

The dot product between a query and a key sums d_k coordinate-wise products. When those coordinates are approximately independent, centered, and similarly scaled, the variance of the sum grows with d_k. Increasing head dimension therefore increases the typical magnitude of unscaled logits even when the quality of the underlying relation has not changed. Softmax receives values that are farther apart and tends to produce excessively concentrated weights.

A saturated row behaves almost like hard selection: one key receives probability near one and the others approach zero. Concentration is not automatically wrong; learned retrieval may need to be sharp. The problem is allowing dimension alone to impose sharpness before training has justified it. When softmax saturates, many Jacobian entries become small, alternatives stop receiving useful learning signals, and optimization becomes more sensitive to initialization and precision.

Dividing by $\sqrt{d_k}$ approximately preserves unit-scale score variance under the standard assumptions. It acts as a fixed temperature tied to head dimension, making score distributions more comparable across different d_k. Scaling does not guarantee stability: correlated coordinates, large vector norms, poor initialization, or low precision can still produce extreme logits. It removes one predictable source of dimension-dependent saturation.

This exercise must connect four kinds of evidence: score statistics, attention entropy or concentration, gradient behavior, and actual early optimization on a controlled task. Merely

printing two output tensors cannot explain gradient stability. A sweep over several dimensions and seeds is necessary to establish the predicted trend.

Both paths must use identical Q, K, V, masks, losses, and upstream gradients. If scaled and unscaled variants are initialized independently, the comparison is confounded. Use cloned leaf tensors and paired trials so scaling is the only experimental difference.

Part 2 - Mathematical form and reasoning

Assume

$$\mathbb{E}[q_\ell] = \mathbb{E}[k_\ell] = 0, \qquad \mathrm{Var}(q_\ell) = \mathrm{Var}(k_\ell) = 1,$$

with approximately independent coordinate products. The unscaled score

$$s = q^\mathsf{T} k = \sum_{\ell=1}^{d_k} q_\ell k_\ell$$

has

$$\mathbb{E}[s] \approx 0, \qquad \mathrm{Var}(s) \approx d_k.$$

For

$$\widetilde{s} = \frac{q^\mathsf{T} k}{\sqrt{d_k}},$$

we obtain $\mathrm{Var}(\widetilde{s}) \approx 1$. The softmax derivative is

$$\frac{\partial a_i}{\partial s_j} = a_i(\delta_{ij} - a_j).$$

When one $a_i \approx 1$ and the rest approach zero, many derivative terms approach zero, weakening learning through competing scores.

Part 3 - Hints, controls, and evidence

1. Sweep $d_k \in \{4, 16, 64, 256, 1024\}$ using paired random tensors for the scaled and unscaled paths.
2. Record score standard deviation, maximum absolute score, attention entropy, maximum probability, and the fraction of near-zero probabilities.
3. Backpropagate the same scalar objective and compare gradient norms for Q, K, and V.
4. Repeat across several seeds and report the mean and variation instead of selecting one convenient run.
5. Add a short training experiment with equal initialization and data, tracking loss and gradient statistics over the first updates.
6. Test normalized queries and keys as a counterexample, and explain which assumptions of the variance argument no longer apply.

State clearly that the theoretical derivation is approximate and conditional on the coordinate statistics.

Part 4 - Python code skeleton

```
import math
import torch

def attention_path(q, k, v, scaled):
    scores = q @ k.transpose(-2, -1)
    if scaled:
        scores = ...
    weights = ...
    output = ...
    return output, scores, weights

def row_entropy(weights, eps=1e-12):
    return ...

def paired_trial(d_k, seed, device):
    torch.manual_seed(seed)
    q0, k0, v0 = ..., ..., ...

    # TODO: create identical leaf copies for both paths
    scaled = attention_path(...)
    unscaled = attention_path(...)

    # TODO: apply the same scalar loss and collect gradients
    return {
        "scaled_score_std": ...,
        "unscaled_score_std": ...,
        "scaled_entropy": ...,
        "unscaled_entropy": ...,
        "scaled_q_grad_norm": ...,
        "unscaled_q_grad_norm": ...,
    }

def E09(device):
    records = []
    for d_k in [4, 16, 64, 256, 1024]:
        for seed in range(10):
            records.append(paired_trial(d_k, seed, device))
    # TODO: aggregate by d_k and explain the trend
    return records
```

Part 5 - The Philosophy Behind the Exercise

Scaling separates the strength of a relation from the number of coordinates used to describe it. Without normalization, a wider representation can appear more certain simply because it

sums more terms. Confidence would then partly reflect dimensional bookkeeping rather than learned evidence.

This reveals a recurring principle in machine learning: expressive capacity and numerical scale are not the same thing. More dimensions may encode richer relations, but incidental magnitude must be controlled so optimization can compare those relations meaningfully.

The exercise also warns against equating sharpness with knowledge. A saturated softmax looks decisive, yet its decisiveness may be an artifact of dimension, initialization, or precision. Before interpreting a sharp distribution as a strong belief, ask what produced the scale on which that belief was computed.

17.2.11 Exercise 10

Exercise Title: Run Your Scratch Attention on CUDA and Confirm Numerical Parity with CPU for Multiple Random Seeds

Part 1 - Why this matters

Moving attention to CUDA changes where the computation is executed, which kernels perform it, and sometimes the order in which floating-point operations are accumulated. It should not change the mathematical function. CPU–GPU parity testing checks that device transfer, mask creation, dtype selection, and kernel execution preserve the intended result within a justified numerical tolerance.

Exact bitwise equality is usually the wrong default. Floating-point addition is not associative, and parallel kernels may reduce products in a different order from CPU routines. Small differences can therefore be legitimate even in deterministic code. The correct question is whether errors remain within a tolerance appropriate to dtype, magnitude, and downstream use. Report maximum absolute error, relative error, and whether mask invariants remain exact.

Several seeds are required because one random case may contain easy logits far from a softmax boundary. Another seed may create nearly tied scores, large magnitudes, or a distribution more sensitive to rounding. Test forward scores, weights, outputs, and backward gradients. Copy identical initial tensors to each device; generating random tensors independently on CPU and GPU does not create a parity test because their random streams need not match.

Device errors often hide in auxiliary tensors. A causal mask built on CPU, a scale tensor created with a default dtype, or a reference constant placed on the wrong device can break execution or trigger unwanted transfers. All parameters, inputs, masks, and loss targets must follow an explicit device and dtype contract.

Parity is necessary but not sufficient. Two devices can agree on the same incorrect softmax axis. Reuse the semantic and manual tests from earlier exercises, then add device comparison as another independent layer of evidence.

Part 2 - Mathematical form and reasoning

Let f_{CPU} and f_{CUDA} implement the same attention function. For reference output y and CUDA output $\widehat{y}$, measure

$$E_{\text{abs}} = \max_i |\widehat{y}_i - y_i|,$$

$$E_{\text{rel}} = \max_i \frac{|\widehat{y}_i - y_i|}{\max(|y_i|, \varepsilon)}.$$

A common comparison is

$$|\widehat{y}_i - y_i| \le \texttt{atol} + \texttt{rtol}|y_i|.$$

The tolerances must be recorded rather than chosen after observing failure. Repeat the same comparisons for $\nabla_Q L$, $\nabla_K L$, and $\nabla_V L$.

Part 3 - Hints, controls, and evidence

1. Generate one master set of tensors on CPU, clone it for the CPU path, and copy it to CUDA for the GPU path.
2. Use identical masks and one scalar loss. Compare scores, weights, outputs, and all input gradients.
3. Repeat across at least ten seeds, several shapes, and both unmasked and causal or padded cases.
4. Check `torch.cuda.is_available()` and report a clear skipped test when CUDA is unavailable.
5. Record PyTorch, CUDA, GPU, dtype, deterministic settings, and tolerances.
6. Separate ordinary roundoff from non-finite values, mask violations, and large structural disagreement.

Summarize the distribution of errors across seeds; do not report only the largest case without its configuration.

Part 4 - Python code skeleton

```
import torch

def run_path(q, k, v, mask):
    # TODO: call the scratch attention and a shared scalar loss
    output, debug = ...
    loss = ...
    loss.backward()
    return {
        "output": output.detach().cpu(),
        "weights": debug["weights"].detach().cpu(),
        "q_grad": q.grad.detach().cpu(),
        "k_grad": k.grad.detach().cpu(),
        "v_grad": v.grad.detach().cpu(),
    }
```

```
def parity_metrics(reference, candidate, atol, rtol):
    return {
        "max_abs": ...,
        "max_rel": ...,
        "allclose": ...,
        "finite": ...,
    }

def paired_seed(seed, shape, dtype):
    torch.manual_seed(seed)
    q0, k0, v0, mask0 = ..., ..., ..., ...
    cpu = run_path(...)
    cuda = run_path(...)
    return {
        name: parity_metrics(cpu[name], cuda[name], ...)
        for name in cpu
    }

def E10():
    if not torch.cuda.is_available():
        return {"status": "skipped", "reason": "CUDA unavailable"}
    return [paired_seed(seed, ..., torch.float32)
            for seed in range(10)]
```

Part 5 - The Philosophy Behind the Exercise

Device parity asks whether a mathematical identity survives translation into a different physical process. CPU and GPU do not perform the computation through identical schedules, yet they are expected to realize the same model within defined limits. Reproducibility is therefore not always sameness of bits; it is stability of meaning under bounded numerical variation.

This forces the researcher to define what counts as equivalent. A tolerance is not permission to ignore errors. It is a declared boundary between harmless representation-level variation and behavior-changing disagreement. Its value must be justified by precision, scale, and application.

The broader lesson is that software correctness includes the machine on which the software runs. An equation does not float above hardware. Trust comes from testing the path from mathematical intention through tensor program to physical execution.

17.2.12 Exercise 11

Exercise Title: Benchmark Attention Runtime on GPU for Increasing Sequence Lengths and Observe Quadratic Scaling

Part 1 - Why this matters

Dense self-attention creates interactions between every pair of sequence positions. Doubling sequence length therefore multiplies the number of score entries by approximately four. Theoretical complexity predicts quadratic work for score construction and value aggregation, and quadratic storage when the full attention matrix is materialized. Actual GPU runtime,

however, does not follow a perfect parabola at every size.

Small problems may be dominated by kernel-launch overhead and fail to occupy the GPU. Medium problems can use hardware efficiently, causing runtime to grow more slowly than a naive operation count suggests. Large problems eventually become limited by memory traffic, workspace, or capacity and may jump sharply or fail with out-of-memory. Benchmarking translates asymptotic reasoning into the practical operating region of a particular GPU.

Correct CUDA timing requires synchronization. Kernel launches are asynchronous, so measuring only the Python call may record dispatch time rather than completed execution. Warm-up is also required for allocator state, library initialization, and kernel selection. Prefer CUDA events or synchronized timers, run repeated trials, and report medians and dispersion.

Control every dimension except sequence length. Batch size, head count, head dimension, dtype, mask type, gradient mode, and implementation must stay fixed. Run forward-only and forward-plus-backward benchmarks separately. Record peak memory and tokens per second in addition to latency. If the implementation materializes attention weights, say so; optimized fused kernels may have a different memory curve while computing the same dense attention.

The goal is to observe where quadratic structure becomes visible, not to force every measured ratio to equal four. Explain deviations through utilization, memory, kernel choice, or measurement noise.

Part 2 - Mathematical form and reasoning

For B batches, H heads, length T, and head dimension d_k, the score and aggregation work is approximately

$$\mathcal{O}(BHT^2 d_k),$$

while a materialized attention matrix contains

$$N_A = BHT^2$$

elements. If each element uses s bytes, its storage alone is

$$M_A = sBHT^2.$$

An empirical scaling exponent between two lengths can be estimated by

$$\widehat{p} = \frac{\log(t_2/t_1)}{\log(T_2/T_1)}.$$

A value near 2 over a sufficiently large compute-bound range is consistent with quadratic runtime, but it is not expected in every regime.

Part 3 - Hints, controls, and evidence

1. Test powers of two until the largest safe length for your GPU, stopping before repeated out-of-memory failures.

2. Warm up each shape, use CUDA events or explicit synchronization, and report median plus percentile variation.

3. Reset peak-memory statistics before every measured configuration.
4. Benchmark forward inference under `torch.no_grad()` separately from training with backward.
5. Plot latency and peak memory against T on linear and log–log axes.
6. Compare measured memory with the theoretical score-matrix size and explain the remaining allocations.

Do not compare different sequence lengths while silently reducing batch size; that answers a different throughput question.

Part 4 - Python code skeleton

```
import torch

def benchmark_length(seq_len, config):
    q, k, v = ...  # fixed B, H, d_k, dtype
    for _ in range(config["warmup"]):
        ...
    torch.cuda.synchronize()
    torch.cuda.reset_peak_memory_stats()

    start = torch.cuda.Event(enable_timing=True)
    end = torch.cuda.Event(enable_timing=True)
    times = []
    for _ in range(config["repeats"]):
        start.record()
        output = ...
        end.record()
        end.synchronize()
        times.append(start.elapsed_time(end))

    return {
        "seq_len": seq_len,
        "median_ms": ...,
        "p90_ms": ...,
        "peak_bytes": torch.cuda.max_memory_allocated(),
        "tokens_per_second": ...,
    }

def E11():
    config = ...
    records = []
    for seq_len in [64, 128, 256, 512, 1024, 2048]:
        try:
            records.append(benchmark_length(seq_len, config))
        except torch.OutOfMemoryError:
            records.append({"seq_len": seq_len, "status": "OOM"})
```

```
            break
    # TODO: estimate local scaling exponents
    return records
```

Part 5 - The Philosophy Behind the Exercise

Asymptotic complexity is a statement about growth, not a stopwatch prediction. Hardware turns equations into regimes: overhead-dominated, efficiently parallel, bandwidth-limited, and capacity-limited. The benchmark teaches how a theoretical law appears through the imperfections and strengths of a machine.

Quadratic attention also reveals that universal access has a price. Giving every token the possibility of relating to every other token creates expressive freedom, but the number of possible relations grows faster than the sequence. Architecture is always a decision about which relationships deserve resources.

The practical limit of context is therefore not only a property of a model. It is a joint property of algorithm, representation, precision, kernel, batch, device, and budget. Measurement makes that hidden dependency explicit.

17.2.13 Exercise 12

Exercise Title: Compare float32 vs float16 Attention Outputs and Detect Precision-Sensitive Softmax Instabilities

Part 1 - Why this matters

Half precision can reduce memory traffic, increase tensor-core throughput, and make larger attention workloads possible on a fixed GPU. Those benefits are substantial, but `float16` has a narrower exponent range and fewer mantissa bits than `float32`. Attention combines dot products, division, masking, exponentiation, normalization, and weighted aggregation; each stage can amplify a different form of rounding or range failure.

The comparison must begin from identical real-valued inputs. Create a master `float32` or `float64` tensor, then cast copies into each test precision. Generating independent random inputs would measure sampling difference rather than precision difference. Use the same masks, scale, loss, and upstream gradient. Compare raw scores, probabilities, outputs, and gradients separately so the first stage at which error grows can be identified.

NaNs are only the most visible failure. In `float16`, two nearby large logits may round to the same representable value, changing the attention ranking. Very negative shifted logits may underflow to zero probability, and a large dot product may overflow before stable softmax receives it. The output can remain finite while entropy, rare-token weights, and gradient paths differ materially. Finiteness checks must therefore be combined with parity and distribution diagnostics.

Real mixed-precision systems often compute some reductions or normalization steps in higher precision. This is not cheating; it is a deliberate numerical policy. Compare pure half precision, half inputs with score or softmax upcasting, and full `float32`. The objective is to discover which operations require extra range or resolution and what speed or memory price

that policy imposes.

Sensitivity depends on data scale and sequence length. Sweep the norms of Q and K, the head dimension, and the mask pattern. Report the boundary where agreement deteriorates rather than declaring one dtype universally safe.

Part 2 - Mathematical form and reasoning

Let

$$S^{(32)} = \frac{Q^{(32)}K^{(32)\mathsf{T}}}{\sqrt{d_k}}, \qquad S^{(16)} = \frac{Q^{(16)}K^{(16)\mathsf{T}}}{\sqrt{d_k}}.$$

For outputs $O^{(32)}$ and $O^{(16)}$, measure

$$E_{\max} = \max_i |O_i^{(16)} - O_i^{(32)}|, \qquad E_{\mathrm{RMS}} = \sqrt{\frac{1}{N}\sum_i (O_i^{(16)} - O_i^{(32)})^2}.$$

Also compare row entropy and normalization error:

$$H(A_i) = -\sum_j A_{ij}\log(A_{ij} + \varepsilon), \qquad e_i = \left|\sum_j A_{ij} - 1\right|.$$

These metrics expose distribution changes that a single output tolerance may hide.

Part 3 - Hints, controls, and evidence

1. Use `float32` as the operational reference and optionally `float64` for small diagnostic tensors.
2. Sweep input scale, d_k, and sequence length over several seeds.
3. Compare pure `float16`, upcast-score softmax, and full `float32` paths.
4. Record NaN/Inf counts, score error, ranking changes, entropy changes, output error, and gradient error.
5. Include rows with nearly tied logits and rows with one extreme score.
6. State tolerances before examining results and document any configuration that exceeds them.

Part 4 - Python code skeleton

```
import torch

def attention_precision(q, k, v, mode):
    if mode == "fp16":
        ...
    elif mode == "fp16_upcast_softmax":
        scores = ...
        weights = torch.softmax(scores.float(), dim=-1)
```

```
        output = ...
    elif mode == "fp32":
        ...
    return output, scores, weights

def compare(reference, candidate):
    return {
        "max_abs": ...,
        "rms": ...,
        "finite": ...,
        "row_sum_error": ...,
        "entropy_delta": ...,
    }

def E12(device="cuda"):
    records = []
    for scale in [0.1, 1.0, 10.0, 100.0]:
        for d_k in [16, 64, 256]:
            master_q, master_k, master_v = ...
            reference = attention_precision(..., mode="fp32")
            for mode in ["fp16", "fp16_upcast_softmax"]:
                candidate = attention_precision(..., mode=mode)
                records.append({...})
    return records
```

Part 5 - The Philosophy Behind the Exercise

Precision is a decision about which distinctions a machine is allowed to preserve. Reducing precision does not simply make every number slightly less accurate; it can erase small differences, collapse alternatives into ties, or move a value outside the representable world. In attention, those changes alter which information is retrieved.

Efficiency therefore has an epistemic cost. A faster representation is useful only while it preserves the distinctions the task needs. Mixed precision is a design of selective fidelity: spend numerical accuracy where probabilities and gradients are fragile, and use compact arithmetic where error is tolerable.

The lesson is not that higher precision is always better. It is that precision must be treated as part of model behavior, measured against explicit invariants rather than selected only from a hardware marketing number.

17.2.14 Exercise 13

Exercise Title: Debug Common Attention Bugs Like Wrong Softmax Dimension or Missing Mask Broadcast Rules

Part 1 - Why this matters

Attention failures are unusually deceptive because many incorrect programs produce finite tensors with plausible shapes. Softmax over the query axis still creates normalized numbers,

but it answers a different question. A mask shaped incorrectly may broadcast across batches or heads and leak information without raising an exception. Swapping query and key axes can remain invisible in self-attention because both lengths are equal. Debugging must therefore search for violated semantics, not merely runtime errors.

The first task is to define invariants. Each query row must normalize across allowed keys. Padded and future keys must receive zero weight. Examples in a batch must remain independent. The output shape must retain query length and value dimension. Finite outputs, correct device placement, and consistent dtype are separate requirements. A debugger that checks these contracts after each stage can locate the earliest corruption.

Fault injection turns vague debugging advice into a testable system. Create known broken variants: use `dim=-2` for softmax, omit the key transpose, reverse Boolean mask meaning, drop singleton broadcast axes, apply the mask after softmax, or use a finite mask fill too weak for extreme logits. Each fault should trigger a specific diagnostic. If a test suite cannot detect a defect you deliberately inserted, it cannot be trusted to detect the accidental version.

Use asymmetric shapes and index-coded tensors. A cross-attention case with unequal query and key lengths reveals assumptions hidden by square score matrices. A batch in which every example has a different padding pattern reveals cross-example broadcasting. A causal plus padding mask reveals whether restrictions are combined by logical intersection.

The final output of this exercise is not only corrected code. It is a debugging procedure ordered from cheap structural checks to stronger intervention and gradient tests. This procedure should be reusable in later multi-head, decoder-only, and encoder-decoder chapters.

Part 2 - Mathematical form and reasoning

For

$$S \in \mathbb{R}^{B\times H\times T_q\times T_k},$$

correct normalization requires

$$\sum_{j=1}^{T_k} A_{bhij} = 1$$

for each valid (b, h, i) row. A keep mask G must be broadcastable to S and satisfy

$$G_{bhij} = 0 \quad \Longrightarrow \quad A_{bhij} = 0.$$

For a combined causal mask C and padding mask P,

$$G = C \wedge P.$$

A wrong softmax axis instead enforces a sum over queries, which is numerically valid but semantically unrelated to retrieval over keys.

Part 3 - Hints, controls, and evidence

1. Build one correct reference and one function parameterized by injectable faults.

2. Use distinct sizes for B, H, T_q, T_k, d_k, d_v.
3. Check shapes, row sums, forbidden weights, batch independence, and prefix invariance independently.
4. Require every injected fault to fail at least one named test.
5. Log the earliest failing tensor rather than inspecting only final output.
6. Add informative assertion messages containing semantic axis names and actual shapes.

Produce a table mapping each defect to its symptom, detecting test, root cause, and permanent regression test.

Part 4 - Python code skeleton

```
import torch

def attention_with_fault(q, k, v, mask, fault=None):
    scores = ...
    if fault == "missing_key_transpose":
        scores = ...
    if fault == "mask_after_softmax":
        ...
    if fault == "wrong_mask_semantics":
        ...
    dim = -2 if fault == "wrong_softmax_dim" else -1
    weights = torch.softmax(..., dim=dim)
    output = ...
    return output, {"scores": scores, "weights": weights}

def run_invariants(output, debug, mask, expected_shape):
    return {
        "shape": ...,
        "finite": ...,
        "rows_sum_to_one": ...,
        "forbidden_weights_zero": ...,
        "batch_independent": ...,
    }

def E13():
    faults = [None, "wrong_softmax_dim", "wrong_mask_semantics",
              "mask_after_softmax", "missing_key_transpose"]
    results = {}
    for fault in faults:
        try:
            output, debug = attention_with_fault(...)
            results[str(fault)] = run_invariants(...)
        except Exception as error:
            results[str(fault)] = {"exception": str(error)}
    return results
```

Part 5 - The Philosophy Behind the Exercise

A silent attention bug demonstrates that valid syntax and finite numbers are weak forms of truth. The machine can execute an operation perfectly while the operation answers the wrong question. Correctness requires a relationship between computation and intention that cannot be inferred from successful execution alone.

Fault injection adopts a scientific attitude toward software: do not merely hope that tests protect the system; challenge them with known falsehoods. Confidence grows when an instrument distinguishes correct behavior from carefully constructed error.

Debugging is therefore a study of violated meaning. The earliest failed invariant tells us where the computation stopped representing the intended world. A mature engineer does not only fix the visible symptom; they preserve the discovered meaning as a future test.

17.2.15 Exercise 14

Exercise Title: Implement Attention Dropout and Verify It Activates Only During Training Mode and Not During Evaluation

Part 1 - Why this matters

Attention dropout regularizes retrieval by randomly removing some normalized attention links during training. It discourages a head from depending exclusively on one fragile route and exposes the surrounding network to alternative value combinations. The operation is stochastic by design, but its location and mode behavior must be precise. Applying dropout to attention weights is not identical to applying dropout to inputs, values, projections, or the final block output.

In the standard formulation, softmax first creates an attention distribution and dropout then masks and rescales its entries before multiplication by V. The surviving weights are scaled by $1/(1-p)$, preserving their expectation under independent dropout. For a particular training sample, however, rows no longer need to sum to one. A test that always demands a unit row sum after dropout is therefore incorrect. Before dropout the probability invariant remains essential; after dropout the relevant invariant is expected value over many masks.

Mode control is critical. During `model.train()`, repeated forward passes with the same input should usually differ when $p > 0$. During `model.eval()`, dropout must be disabled and repeated outputs should match within the deterministic behavior of the remaining operations. Using `torch.nn.functional.dropout` without passing `training=self.training` is a common failure that leaves stochastic behavior active in evaluation.

Deterministic tests need careful seed handling. Resetting the same seed before every training forward can make dropout appear inactive because it recreates the same mask. Conversely, comparing two unseeded evaluation calls can confuse dropout with another nondeterministic kernel. Inspect or reconstruct dropout masks on a tiny example, then evaluate statistical behavior across many independent trials.

Finally, masking and dropout must not be confused. Padding and causal masks enforce forbidden information paths and must remain absolute. Dropout removes permitted paths

temporarily for regularization. A forbidden location must never reappear through dropout scaling or an incorrect ordering.

Part 2 - Mathematical form and reasoning

Let A be the post-softmax attention matrix and let $Z_{ij} \sim \text{Bernoulli}(1-p)$. Training-time dropout produces

$$\widetilde{A}_{ij} = \frac{Z_{ij}}{1-p} A_{ij}.$$

Therefore

$$\mathbb{E}[\widetilde{A}_{ij}] = \frac{\mathbb{E}[Z_{ij}]}{1-p} A_{ij} = A_{ij}.$$

The output is

$$O_{\text{train}} = \widetilde{A}V, \qquad O_{\text{eval}} = AV.$$

For one training realization,

$$\sum_j \widetilde{A}_{ij} \neq 1$$

in general, although the expected sum equals one.

Part 3 - Hints, controls, and evidence

1. Implement attention dropout in an `nn.Module` so it follows `train()` and `eval()` modes.
2. Confirm masked positions are zero both before and after dropout.
3. In training mode, run many trials and estimate the zero fraction and mean output.
4. Compare the empirical mean with the no-dropout evaluation output.
5. In evaluation mode, run repeated identical inputs and require parity.
6. Test $p = 0$, a moderate probability, and a high probability while explaining why $p = 1$ is invalid for inverted dropout.

Part 4 - Python code skeleton

```
import torch
from torch import nn
from torch.nn import functional as F

class ScratchAttention(nn.Module):
    def __init__(self, dropout_p):
        super().__init__()
        self.dropout_p = dropout_p

    def forward(self, q, k, v, keep_mask=None):
```

```
        scores = ...
        if keep_mask is not None:
            scores = ...
        weights = ...
        dropped = F.dropout(
            weights, p=self.dropout_p,
            training=...  # TODO
        )
        output = ...
        return output, weights, dropped

def E14():
    module = ScratchAttention(dropout_p=0.2)
    q, k, v, mask = ..., ..., ..., ...

    module.train()
    training_runs = [module(q, k, v, mask) for _ in range(500)]

    module.eval()
    eval_runs = [module(q, k, v, mask) for _ in range(5)]

    return {
        "training_changes": ...,
        "empirical_zero_rate": ...,
        "mean_matches_eval": ...,
        "eval_is_repeatable": ...,
        "forbidden_positions_stay_zero": ...,
    }
```

Part 5 - The Philosophy Behind the Exercise

Dropout introduces deliberate unreliability in order to produce a more reliable model. During training, the network is denied some convenient paths and must learn representations that survive their absence. Robustness is cultivated by controlled imperfection rather than by protecting every internal dependency.

This differs fundamentally from a causal or padding mask. Structural masks describe what information is illegitimate; dropout describes what legitimate information is temporarily unavailable. One is a rule of the problem, the other a strategy for learning. Confusing them means confusing ethics of access with discipline of dependence.

Evaluation removes the artificial uncertainty because the learned system is now being judged, not trained. The change of mode reminds us that the same architecture can inhabit different operational regimes, and correctness depends on knowing which regime is active.

17.2.16 Exercise 15

Exercise Title: Implement a Test Suite That Validates Attention Properties Including Masking and Output Shape Correctness

Part 1 - Why this matters

Attention combines enough independent rules that example-based testing alone is inadequate. One successful tensor cannot prove that arbitrary batches, heads, query lengths, key lengths, masks, devices, and dtypes will behave correctly. A test suite converts the mathematical and semantic expectations of the chapter into executable contracts that can protect later refactoring and optimization.

The suite should contain exact tiny examples, randomized property tests, edge cases, and negative tests. Exact examples catch arithmetic and orientation errors. Randomized shapes expose accidental dependence on equal dimensions. Edge cases reveal sequence-length one, fully valid masks, nearly empty masks, and cross-attention behavior. Negative tests prove invalid inputs are rejected with useful messages rather than producing mysterious failures downstream.

Test layers should remain independent. Shape validity does not imply mask validity; row normalization does not imply causal isolation; forward parity does not imply gradient parity. Each property deserves a named assertion so a failure communicates which meaning was lost. Tests must also account for dropout: deterministic reference comparisons belong in evaluation mode or with $p = 0$.

Device and precision coverage should be conditional but explicit. CPU tests must always run. CUDA and half-precision tests should be marked as skipped when unsupported rather than silently absent. Tolerances should depend on dtype and be specified before comparison. Save failing seeds and shapes so randomized failures can be reproduced.

The suite becomes a gate for later performance work. A fused kernel, compiled path, or memory optimization should not be accepted merely because it is faster. It must satisfy the same behavioral contracts as the transparent reference.

Part 2 - Mathematical form and reasoning

For valid inputs, test at least

$$\operatorname{shape}(O) = (B, H, T_q, d_v),$$

$$\sum_{j=1}^{T_k} A_{bhij} = 1, \qquad A_{bhij} \geq 0,$$

$$G_{bhij} = 0 \Longrightarrow A_{bhij} = 0,$$

and causal prefix invariance

$$X'_{0:r} = X_{0:r} \Longrightarrow O'_{0:r} = O_{0:r}.$$

Reference parity can use

$$|\widehat{y} - y| \leq \texttt{atol} + \texttt{rtol}|y|.$$

Gradient checks compare derivatives for Q, K, and V, either against a trusted implementation or finite differences on a tiny double-precision case.

Part 3 - Hints, controls, and evidence

1. Organize tests by shape, arithmetic, mask, causality, gradient, device, precision, mode, and invalid-input behavior.
2. Include unequal query and key lengths and unique sizes for every axis.
3. Use fixed seeds and print the seed plus shape when a randomized test fails.
4. Run attention in evaluation mode for deterministic parity tests.
5. Verify each deliberately broken implementation causes the expected test to fail.
6. Produce a compact pass, fail, or skip report rather than hiding skipped hardware coverage.

Part 4 - Python code skeleton

```
import torch

def test_output_shape(attention):
    q, k, v = ...
    output, weights = attention(q, k, v)
    assert output.shape == ...
    assert weights.shape == ...

def test_probability_properties(attention):
    ...

def test_padding_invariance(attention):
    ...

def test_causal_prefix_invariance(attention):
    ...

def test_reference_and_gradients(attention):
    ...

def test_invalid_inputs(attention):
    # TODO: incompatible d_k, K/V length mismatch, all-masked row
    ...

def E15(attention):
    tests = [
        test_output_shape,
        test_probability_properties,
        test_padding_invariance,
        test_causal_prefix_invariance,
        test_reference_and_gradients,
        test_invalid_inputs,
    ]
    report = {}
```

```
    for test in tests:
        try:
            test(attention)
            report[test.__name__] = "passed"
        except Exception as error:
            report[test.__name__] = f"failed: {error}"
    return report
```

Part 5 - The Philosophy Behind the Exercise

A test suite is a durable expression of understanding. Explanations can be forgotten and code can be reorganized, but an invariant encoded as a test continues to challenge every future implementation. It converts knowledge from a private mental state into a shared institutional memory.

Testing also defines the limits of trust. Passing tests does not prove every possible behavior correct; it proves survival under a declared set of questions. The quality of confidence depends on the quality and diversity of those questions.

The deepest value of a test is not that it rewards correct code, but that it makes certain forms of self-deception difficult. A faster or more complex implementation must answer to the same meaning as the simple reference.

17.2.17 Exercise 16

Exercise Title: Stress Test Attention With Large Batch and Long Sequence and Monitor GPU Memory Peak and Runtime

Part 1 - Why this matters

A benchmark measures expected performance in a controlled region. A stress test searches for the boundary where assumptions fail. For attention, batch size multiplies examples while dense sequence interactions grow quadratically. Increasing both can cause abrupt out-of-memory failures, allocator pressure, latency spikes, or throughput collapse long before parameter memory changes.

The test must monitor more than the allocation visible after a forward pass. Attention may create transient score matrices, softmax buffers, gradients, saved activations, and library workspaces. CUDA caching also distinguishes memory allocated to live tensors from memory reserved by the allocator. Reset peak statistics before every configuration and record both peaks. Separate inference, training forward, and full forward-backward because their activation lifetimes differ.

A responsible stress test increases load gradually and stops safely. Repeated OOM failures can leave a process fragmented or unusable. Construct a bounded grid, begin from known-safe settings, synchronize before measurement, catch `torch.OutOfMemoryError`, and record failure rather than hiding it. Do not automatically consume every available byte.

Runtime should be reported as latency and throughput. A larger batch may increase latency while improving tokens per second through better utilization. Longer sequences may initially

improve kernel efficiency before quadratic work dominates. Record distributional statistics because memory pressure and system activity can produce tail latency.

The result is an operating envelope for the actual device and implementation: which combinations are safe, efficient, and impossible. This envelope is conditional on dtype, head count, dimension, gradient mode, mask, and whether attention weights are materialized.

Part 2 - Mathematical form and reasoning

The attention tensor contains

$$N_A = BHT_qT_k$$

elements. For self-attention with $T_q = T_k = T$ and s bytes per element,

$$M_A = sBHT^2.$$

The dominant multiplications scale as

$$\mathcal{O}(BHT^2d_k).$$

Throughput can be reported as

$$\text{tokens/s} = \frac{BT}{t}.$$

These derived metrics do not replace raw latency and memory measurements.

Part 3 - Hints, controls, and evidence

1. Define explicit maximum batch and sequence lengths before running.
2. Test inference and training separately with fixed H, d_k, dtype, and implementation.
3. Warm up, synchronize, reset peak statistics, and measure repeated trials.
4. Record latency percentiles, tokens per second, peak allocated memory, peak reserved memory, and OOM status.
5. Verify output shape and finiteness at every successful point.
6. After an OOM, release only failed-trial references and continue with a documented safe strategy.

Plot batch-by-sequence heat maps for latency, memory, and pass/OOM status, then identify a conservative production boundary below the absolute maximum.

Part 4 - Python code skeleton

```
import gc
import torch

def stress_point(batch, seq_len, config):
    torch.cuda.reset_peak_memory_stats()
```

```
    q, k, v = ...  # explicit tested dimensions
    torch.cuda.synchronize()
    try:
        # TODO: time forward or forward+backward
        output = ...
        torch.cuda.synchronize()
        return {
            "status": "passed",
            "finite": ...,
            "latency_ms": ...,
            "peak_allocated": torch.cuda.max_memory_allocated(),
            "peak_reserved": torch.cuda.max_memory_reserved(),
            "tokens_per_second": ...,
        }
    except torch.OutOfMemoryError:
        return {"status": "OOM"}
    finally:
        # TODO: drop trial-local references
        ...
        gc.collect()
        torch.cuda.empty_cache()

def E16():
    config = {"max_batch": 64, "max_seq": 4096, ...}
    records = []
    for batch in [1, 2, 4, 8, 16, 32, 64]:
        for seq_len in [128, 256, 512, 1024, 2048, 4096]:
            records.append(stress_point(batch, seq_len, config))
    return records
```

Part 5 - The Philosophy Behind the Exercise

Stress testing treats limits as knowledge rather than embarrassment. A system that fails beyond a certain context is not automatically defective; a system whose boundary is unknown is operationally dangerous. Reliability begins when capacity is measured and documented.

The difference between an absolute maximum and a safe operating limit matters. Engineering wisdom reserves space for variation, concurrent work, and unexpected inputs. Living permanently at the edge turns ordinary fluctuation into failure.

Attention makes a broader truth visible: richer relational access consumes physical resources. Intelligence implemented on machines is bounded by memory, time, energy, and scheduling. Understanding those bounds is part of understanding the model.

17.2.18 Exercise 17

Exercise Title: Investigate Non-Determinism in CUDA Attention and Document Which Operations Cause Variation

Part 1 - Why this matters

Re-running a CUDA experiment with the same seed does not always guarantee bitwise-identical results. Randomness, algorithm selection, parallel reduction order, dropout, data loading, and fused attention kernels are distinct sources of variation. If they are collapsed into one label called GPU randomness, the researcher cannot decide which controls are possible, which performance costs they impose, or whether observed differences exceed ordinary numerical noise.

Begin by separating stochasticity from nondeterministic execution. Dropout produces different masks unless random state is controlled; this is intended stochastic behavior. A nondeterministic kernel may vary even with identical inputs and random state because atomic updates or parallel accumulation order can change. Algorithm-selection heuristics may also choose different paths depending on shape and environment.

Determinism settings are not magical. Enabling deterministic algorithms may raise an error for unsupported operations, choose slower kernels, or disable optimizations. Library version, GPU architecture, environment, and precision must be recorded. A run reproducible on one machine may not be bitwise identical on a different software or hardware stack.

Design the investigation as an ablation. Start with fixed saved tensors and dropout disabled. Repeat forward and backward. Then enable dropout with controlled and uncontrolled seeds, toggle deterministic settings, and compare scratch, fused, and high-level attention paths when available. Measure the magnitude and location of disagreement, not merely whether exact equality fails.

The goal is a reproducibility policy: decide when bitwise identity is required, when tolerance parity is sufficient, and which metadata must accompany a result. Debugging may justify slower deterministic kernels; production training may rationally prefer faster paths if variation is measured and acceptable.

Part 2 - Mathematical form and reasoning

Floating-point addition generally satisfies

$$\mathrm{fl}(\mathrm{fl}(a+b)+c) \neq \mathrm{fl}(a+\mathrm{fl}(b+c)).$$

For repeated outputs $y^{(1)}, \ldots, y^{(R)}$, report

$$\Delta_{\max} = \max_{r,s} \|y^{(r)} - y^{(s)}\|_\infty$$

and RMS variation. For training runs, distinguish parameter and metric variation:

$$\Delta_\theta = \|\theta_R^{(r)} - \theta_R^{(s)}\|_2, \qquad \Delta_L = |L_R^{(r)} - L_R^{(s)}|.$$

Small kernel differences may accumulate through optimization, so both levels must be examined.

Part 3 - Hints, controls, and evidence

1. Record Python, PyTorch, CUDA, cuDNN, GPU, dtype, kernel path, and determinism settings.

2. Use fixed saved inputs before introducing a random data pipeline.
3. Test forward and backward separately with dropout disabled.
4. Add dropout and compare reset-seed, continuing-seed, and evaluation-mode behavior.
5. Enable deterministic algorithms and document errors or performance changes rather than suppressing them.
6. Repeat short training runs and compare final parameters and task metrics.

Build a cause table separating data randomness, initialization, dropout, parallel reduction, kernel choice, and cross-machine variation.

Part 4 - Python code skeleton

```
import torch

def configure(seed, deterministic):
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.use_deterministic_algorithms(deterministic)
    # TODO: record backend and environment settings

def repeated_attention(attention, q, k, v, repeats):
    outputs, gradients = [], []
    for _ in range(repeats):
        q_run = q.detach().clone().requires_grad_(True)
        # TODO: run identical forward and backward
        ...
        outputs.append(...)
        gradients.append(...)
    return outputs, gradients

def variation(tensors):
    # TODO: pairwise max-absolute and RMS differences
    return ...

def E17():
    report = {}
    for deterministic in [False, True]:
        configure(seed=123, deterministic=deterministic)
        try:
            outputs, gradients = repeated_attention(...)
            report[str(deterministic)] = {
                "output_variation": variation(outputs),
                "gradient_variation": variation(gradients),
            }
        except RuntimeError as error:
            report[str(deterministic)] = {"unsupported": str(error)}
    return report
```

Part 5 - The Philosophy Behind the Exercise

Reproducibility is often imagined as the universe returning the same bits on command. In parallel numerical systems, a more useful concept is controlled variation: know which causes are fixed, measure what remains, and determine whether conclusions survive it.

Determinism has a purpose rather than an absolute moral status. It is valuable when isolating bugs, comparing implementations, or auditing a critical result. It may be unnecessarily expensive when small bounded variation has no behavioral consequence. The responsible choice is explicit and evidenced.

A single run is one trajectory through a system shaped by data, initialization, arithmetic, and hardware. Knowledge becomes stronger when a conclusion persists across the variations that the real system can produce.

17.2.19 Exercise 18

Exercise Title: Visualize Attention Matrices and Build Interpretation Notes About What Patterns Mean in Practice

Part 1 - Why this matters

An attention matrix is one of the rare internal neural-network objects that can be aligned directly with input positions. Rows identify queries, columns identify keys, and color represents a normalized mixing weight. Visualization can reveal causal-mask violations, attention to padding, local diagonals, delimiter focus, repeated-token links, broad diffuse retrieval, or heads that appear redundant. It is therefore an excellent debugging instrument.

Interpretation requires discipline. A bright cell means that one head at one layer assigned a large coefficient to one value position for one query. It does not prove that the source token caused the final prediction, that the model understood a linguistic relation, or that another computation path was unimportant. Value magnitude, output projection, residual connections, feed-forward layers, and later attention operations all influence the final result.

Plot construction can create false stories. Every plot must label query and key tokens, specify layer and head, use a documented color range, and display masks. Comparisons should share the same scale; otherwise automatic rescaling can make minor differences appear dramatic. Long sequences require selection or aggregation, but aggregation may hide narrow patterns. Preserve the raw matrix alongside every visualization.

Useful interpretation combines description with intervention. If a head seems to attend to a delimiter, remove or move the delimiter and observe how the matrix and output change. If two heads look redundant, ablate each separately and together. If a diagonal pattern appears, shift repeated tokens or positions to determine whether the head follows content or location. These tests move from visual correlation toward causal evidence.

The final notes should classify observations by confidence: directly verified properties, plausible hypotheses, and unsupported stories. The exercise is not complete when the heat map is attractive; it is complete when the plot helps detect an error or generate a hypothesis that can survive testing.

Part 2 - Mathematical form and reasoning

For one batch element, layer, and head,

$$A \in [0,1]^{T_q \times T_k}, \qquad \sum_{j=1}^{T_k} A_{ij} = 1.$$

Row entropy measures concentration:

$$H_i = -\sum_j A_{ij} \log(A_{ij} + \varepsilon).$$

Head similarity can be summarized by a normalized matrix distance, for example

$$D(A^{(h)}, A^{(g)}) = \frac{\|A^{(h)} - A^{(g)}\|_F}{\sqrt{T_q T_k}}.$$

These statistics support visualization but do not establish causal importance. An ablation effect can be measured as

$$\Delta_h = \mathcal{L}_{\text{ablated } h} - \mathcal{L}_{\text{baseline}}.$$

Part 3 - Hints, controls, and evidence

1. Save raw pre-mask scores, post-mask weights, tokens, mask, layer, head, model mode, and input identifier.
2. Label axes explicitly and keep a shared color scale for comparisons.
3. Start with tiny synthetic patterns whose expected attention structure is known, then examine learned examples.
4. Compute row entropy, maximum weight, and head similarity beside the plot.
5. Test at least one visual hypothesis through token perturbation or head ablation.
6. Write interpretations in three categories: observed, experimentally supported, and speculative.

Include examples where visualization correctly reveals a bug and where a plausible-looking pattern fails a causal test.

Part 4 - Python code skeleton

```
import torch
import matplotlib.pyplot as plt

def plot_attention(weights, query_tokens, key_tokens,
                   title, vmin=0.0, vmax=1.0):
    # weights: (query_len, key_len)
    fig, ax = plt.subplots(figsize=(8, 6))
    image = ax.imshow(..., vmin=vmin, vmax=vmax, aspect="auto")
    # TODO: label query rows and key columns
```

```
    # TODO: add color bar and readable tick orientation
    return fig

def attention_statistics(weights, eps=1e-12):
    entropy = ...
    maximum = ...
    return {"row_entropy": entropy, "row_max": maximum}

def ablate_head(model, head_index, inputs):
    baseline = ...
    ablated = ...
    return {"output_delta": ..., "loss_delta": ...}

def E18(model, examples):
    report = []
    for example in examples:
        output, all_weights = ...
        for layer, heads in enumerate(all_weights):
            for head in range(heads.shape[1]):
                matrix = ...
                figure = plot_attention(...)
                report.append({
                    "layer": layer,
                    "head": head,
                    "statistics": attention_statistics(matrix),
                    "ablation": ablate_head(...),
                })
    return report
```

Part 5 - The Philosophy Behind the Exercise

Visualization stands between access and understanding. It grants humans a view into an internal computation, but the view arrives through choices of projection, color, scale, aggregation, and example. A visible pattern is already an interpretation shaped by an instrument.

Attention maps are especially seductive because they resemble diagrams of focus. Yet a model's weight distribution is not identical to human attention, intention, or explanation. Treating it as such replaces investigation with anthropomorphic storytelling.

The proper philosophical role of visualization is generative and critical: it helps us notice, question, and design interventions. It is not the final proof. Seeing begins inquiry; changing the system and observing consequences deepens it.

17.2.20 Exercise 19

Exercise Title: Write a Conceptual Report Explaining Attention, Masking, Scaling, and Why Transformers Replaced RNNs

Part 1 - Why this matters

The final exercise converts nineteen separate investigations into one coherent argument. A good report must explain what attention computes, why Query, Key, and Value have different roles, how scaling controls score statistics, how padding and causal masks define legal information access, and why batched matrix operations changed the practical scalability of sequence models. It must also preserve the limitations revealed by the experiments.

The report should not repeat the slogan that Transformers process everything in parallel. It must distinguish parallel training over known sequence positions from sequential autoregressive generation. It should distinguish shorter dependency paths from quadratic total work, and mathematical equivalence from finite-precision behavior. The replacement of RNNs in many domains was a systems outcome involving optimization, hardware utilization, direct long-distance interaction, data scale, and software kernels.

Evidence from the chapter should support each major claim. Use the copy task to discuss recurrent compression versus addressable retrieval; the scaling sweep to explain gradient stability; padding and causal interventions to demonstrate information boundaries; CPU–GPU and precision comparisons to establish numerical requirements; runtime and stress tests to quantify quadratic cost; and visual or ablation studies to define the limits of interpretability.

The report must also explain where attention does not win. Recurrent models may remain useful for streaming, constant-state processing, small hardware, or short tasks. Dense attention becomes expensive for long contexts, stores large intermediate relations, and depends on positional representation. Generation remains sequential, attention weights do not guarantee explanation, and CUDA execution may introduce bounded variation. A strong conclusion includes these conditions rather than hiding them.

Write the report as an engineering document that another researcher could audit. Separate theoretical derivation, experimental configuration, measured result, interpretation, failure case, and recommendation. Include tables and figures only when their axes, units, seeds, tolerances, and environment are documented. End with open questions that later chapters on multi-head attention, position, Transformer blocks, and efficient long context must answer.

Part 2 - Mathematical form and reasoning

The central mechanism is

$$A = \operatorname{softmax}\left(\frac{QK^{\mathsf{T}}}{\sqrt{d_k}} + M\right), \qquad O = AV.$$

The report should connect it to recurrent computation

$$h_t = f_\theta(x_t, h_{t-1}),$$

and contrast sequential path depth:

$$\text{depth}_{\text{RNN}} = \mathcal{O}(T), \qquad \text{pairwise path}_{\text{attention}} = \mathcal{O}(1)$$

within one attention layer. It must also state dense attention cost:

$$\text{work} = \mathcal{O}(BHT^2 d_k), \qquad \text{score storage} = \mathcal{O}(BHT^2).$$

For causal language modeling,

$$p(x_{1:T}) = \prod_{t=1}^{T} p(x_t \mid x_{<t}),$$

which explains why causal masks enable parallel training but not simultaneous generation of unknown future tokens.

Part 3 - Hints, controls, and evidence

1. Begin with an explicit thesis and list the claims the chapter's evidence can and cannot support.
2. Cite artifacts from E00–E18: configurations, tables, plots, parity errors, mask tests, and failure cases.
3. Separate RNN, LSTM, and attention trade-offs rather than treating all recurrent models as identical.
4. Include numerical stability, precision, determinism, memory, and latency as parts of the architecture's real behavior.
5. State experimental limits, unmatched budgets, and alternative explanations.
6. Finish with design recommendations and questions carried into TRF01.

The report should be reproducible from saved evidence, not dependent on unrecorded impressions.

Part 4 - Python code skeleton

```
from pathlib import Path
import json

def load_verified_artifacts(artifact_directory):
    # TODO: load chapter results and reject missing metadata
    return ...

def validate_evidence(artifacts):
    required = [
        "parallelism", "qkv_retrieval", "masking",
        "scaling", "precision", "runtime", "memory",
        "determinism", "interpretation",
    ]
    return {name: ... for name in required}

def build_report_sections(artifacts):
    return {
        "thesis": ...,
        "mechanism": ...,
        "rnn_comparison": ...,
        "numerical_evidence": ...,
```

```
        "systems_evidence": ...,
        "failure_modes": ...,
        "limitations": ...,
        "next_questions": ...,
    }

def E19(artifact_directory, output_path):
    artifacts = load_verified_artifacts(artifact_directory)
    checks = validate_evidence(artifacts)
    assert all(checks.values()), checks
    sections = build_report_sections(artifacts)
    # TODO: write the report with linked tables and figures
    Path(output_path).write_text(..., encoding="utf-8")
    return {"output": output_path, "checks": checks}
```

Part 5 - The Philosophy Behind the Exercise

Transformers did not replace recurrent models because one equation was universally closer to intelligence. They prevailed across many tasks because a form of relational computation aligned with large data, gradient optimization, parallel hardware, and scalable software. Scientific ideas succeed inside material and institutional environments.

Attention changes the metaphor of sequence intelligence. Instead of carrying one continuously rewritten memory through time, the model reconstructs meaning through direct relations among stored representations. This grants flexible access but demands explicit order, information boundaries, and substantial resources. Every new freedom creates a new responsibility.

The final philosophical lesson is that no mechanism should be understood only through its victories. A complete account includes what it permits, what it forbids, what it costs, how it fails, and what evidence would change our conclusion. The purpose of this chapter is not to worship attention, but to understand why it became powerful enough to reorganize modern AI.

17.3 TRF01

Chapter Title: Multi-Head Attention and Projection Geometry
Specialization: Models and Training

17.3.1 Exercise 00

Exercise Title: Implement Multi-Head Attention from Scratch and Verify Correct Reshaping and Head Concatenation

Part 1 - Why this matters

Multi-head attention is not several unrelated attention modules placed beside one another. It is one representation space projected into several smaller spaces, processed independently, concatenated, and projected back into the model space. The implementation therefore depends on a strict sequence of geometric transformations. A tensor begins as `(batch, sequence,`

`d_model)`, becomes `(batch, sequence, heads, d_head)`, is transposed to place the head axis before the sequence axis, and finally returns to the original model width. Every reshape must preserve element identity; every transpose must change only axis meaning.

The most dangerous bugs are not always runtime errors. If the program reshapes without transposing, the element count remains valid and matrix multiplication may still execute, but features from different tokens and heads are silently interleaved. The resulting model can train slowly or learn a different function while all displayed dimensions appear plausible. Correct multi-head attention therefore requires both shape contracts and value-level tests using tensors whose elements can be traced manually.

Each head computes attention inside a subspace of width $d_h = d_{\text{model}}/H$. The heads do not exchange information while their score matrices and weighted value sums are being calculated. Concatenation does not average the heads; it restores all head outputs side by side in a single vector. The output projection then allows information discovered in different heads to mix. Omitting that projection still produces the expected width, but removes a learned stage of cross-head integration.

This exercise should build three levels of evidence. First, use a tiny tensor with sequential integers and verify splitting and merging by inspection. Second, implement the complete operation with explicit projections and masks. Third, copy the same weights into `torch.nn.MultiheadAttention` and compare forward outputs and input gradients. Library parity is valuable only after the tensor contract of the scratch implementation has been proved.

Part 2 - Mathematical form and reasoning

Let $X \in \mathbb{R}^{B\times T\times d}$, with H heads and $d_h = d/H$. The joint projections are

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V,$$

where $W_Q, W_K, W_V \in \mathbb{R}^{d\times d}$. After splitting,

$$Q, K, V \in \mathbb{R}^{B\times H\times T\times d_h}.$$

For head r,

$$A^{(r)} = \operatorname{softmax}\left(\frac{Q^{(r)}K^{(r)\mathsf{T}}}{\sqrt{d_h}} + M\right), \qquad Z^{(r)} = A^{(r)}V^{(r)}.$$

Concatenation and output projection give

$$Z = \operatorname{Concat}(Z^{(1)}, \ldots, Z^{(H)}) \in \mathbb{R}^{B\times T\times d}, \qquad Y = ZW_O.$$

The invariant $d = Hd_h$ is mandatory. Splitting and merging should also be mutual inverses before any attention calculation is introduced.

Part 3 - Hints, controls, and evidence

1. Reject configurations for which `d_model % num_heads != 0`.
2. Test `merge_heads(split_heads(x)) == x` exactly on an integer-like floating tensor before testing attention.

3. Print shapes after projection, view, transpose, score multiplication, weighted sum, transpose back, concatenation, and output projection.

4. Use `contiguous()` or `reshape()` deliberately after transposition; explain why a non-contiguous tensor cannot always be viewed.

5. Compare with PyTorch using identical weights, biases, dropout state, mask convention, dtype, and batch layout.

6. Check row sums of attention weights, masked positions, finite outputs, and gradients of inputs and all four projection matrices.

The report must include one traceable tensor example showing which original feature reaches each head and which concatenated location receives it again.

Part 4 - Python code skeleton

```
import math
import torch
from torch import nn

class ScratchMHA(nn.Module):
    def __init__(self, d_model, num_heads, dropout=0.0):
        super().__init__()
        assert d_model % num_heads == 0
        self.d_model = d_model
        self.num_heads = num_heads
        self.d_head = d_model // num_heads
        self.q_proj = nn.Linear(d_model, d_model)
        self.k_proj = nn.Linear(d_model, d_model)
        self.v_proj = nn.Linear(d_model, d_model)
        self.out_proj = nn.Linear(d_model, d_model)
        self.dropout = nn.Dropout(dropout)

    def split_heads(self, x):
        # TODO: (B,T,d) -> (B,H,T,d_head)
        return ...

    def merge_heads(self, x):
        # TODO: (B,H,T,d_head) -> (B,T,d)
        return ...

    def forward(self, query, key, value, mask=None):
        q = self.split_heads(self.q_proj(query))
        k = self.split_heads(self.k_proj(key))
        v = self.split_heads(self.v_proj(value))
        scores = ...
        if mask is not None:
            scores = ...
        weights = ...
```

```
        heads = ...
        output = self.out_proj(self.merge_heads(heads))
        return output, weights

def E00():
    model = ScratchMHA(d_model=8, num_heads=2)
    x = torch.arange(2 * 3 * 8, dtype=torch.float32).reshape(2, 3, 8)
    split = model.split_heads(x)
    merged = model.merge_heads(split)
    assert torch.equal(merged, x)
    # TODO: add manual, library-parity, mask, and gradient tests
    return model(x, x, x)
```

Part 5 - The Philosophy Behind the Exercise

Multi-head attention expresses the idea that one representation can be viewed through several learned coordinate systems at the same time. No single view is assumed to contain every useful relation. The architecture preserves several partial perspectives and lets a later projection integrate them.

Yet multiplicity alone does not create diversity. Heads begin as available degrees of freedom, not guaranteed specialists. Their eventual behavior is shaped by initialization, data, loss, optimization, and interactions with the rest of the network. The implementation should therefore preserve head independence without pretending that the model must use it in a human-readable way.

The deeper engineering lesson is that meaning depends on geometry and geometry depends on axis semantics. A reshape is not clerical code surrounding the model; it determines which features are allowed to interact. Understanding multi-head attention begins when tensor layout is treated as part of the mathematics rather than as an implementation detail.

17.3.2 Exercise 01

Exercise Title: Explain Why Multiple Heads Learn Different Relations and Demonstrate With Simple Synthetic Patterns

Part 1 - Why this matters

The usual explanation of multi-head attention says that different heads learn different relationships. This is a useful hypothesis, but it is not an architectural guarantee. Heads have separate projected subspaces and can therefore represent different compatibility functions, yet two heads may become redundant, divide one relation across several components, or change their apparent role across examples. This exercise replaces the slogan with a controlled experiment.

Create synthetic sequences containing relations whose ground truth is known. One relation may require attending to the immediately previous token, another to a token with the same symbol, and another to a marker at the beginning of the sequence. A small multi-head model is then trained on a target requiring more than one relation. Because the data-generating process is explicit, head weights can be compared with known relation masks rather than interpreted

only through attractive heat maps.

The experiment must separate capacity from specialization. Compare one head with the same total d_{model} against several heads whose widths sum to that same value. Repeat seeds and match parameter budgets where possible. If a head aligns with a relation, ablate or permute that head and measure the change in task performance. Visual similarity without a behavioral intervention is weak evidence.

Finally, record negative results. Two heads may solve the task using a distributed representation that does not resemble the intended masks. A trained network is free to exploit any predictive structure available in the dataset. The purpose is not to force a comforting story, but to understand when architectural subspaces produce measurable functional diversity.

Part 2 - Mathematical form and reasoning

For head r,

$$A^{(r)} = \operatorname{softmax}\left(\frac{XW_Q^{(r)}(XW_K^{(r)})^{\mathsf{T}}}{\sqrt{d_h}} + M\right).$$

Let $R^{(m)} \in \{0,1\}^{T\times T}$ be a known relation matrix for pattern m. One simple alignment score is

$$S_{r,m} = \frac{1}{BT}\sum_{b,i}\sum_{j} A^{(r)}_{b,i,j} R^{(m)}_{b,i,j},$$

with suitable normalization when a row contains several valid keys. Head redundancy may be approximated by

$$D_{r,s} = \frac{1}{BT}\sum_{b,i} \operatorname{JSD}\left(A^{(r)}_{b,i,:}, A^{(s)}_{b,i,:}\right),$$

but behavioral ablation remains necessary because weight similarity is not identical to functional importance.

Part 3 - Hints, controls, and evidence

1. Generate labels from two explicit relations and retain their ideal attention matrices for later comparison.

2. Fix dataset, initialization policy, optimizer budget, stopping rule, and evaluation split across head-count comparisons.

3. Train at least five seeds and report distributions rather than selecting the clearest heat map.

4. Measure relation alignment, pairwise head similarity, and performance drop after head ablation.

5. Add a shortcut feature deliberately, then test whether heads follow the intended relation or the easier correlation.

6. Do not name a head “syntax” or “memory” unless the intervention supports that functional claim.

Part 4 - Python code skeleton

```
import torch

def make_relational_batch(batch, seq_len, vocab_size):
    tokens = ...
    previous_relation = ...   # (B,T,T)
    same_token_relation = ... # (B,T,T)
    targets = ...
    return tokens, targets, {
        "previous": previous_relation,
        "same_token": same_token_relation,
    }

def alignment_score(weights, relation):
    # weights: (B,H,T,T); relation: (B,T,T)
    return ...

def ablate_head(head_outputs, head_index):
    changed = head_outputs.clone()
    changed[:, head_index] = 0
    return changed

def E01(config, seeds=range(5)):
    reports = []
    for seed in seeds:
        torch.manual_seed(seed)
        # TODO: train, log per-head weights, score relations,
        # ablate heads, and evaluate held-out performance
        reports.append(...)
    return reports
```

Part 5 - The Philosophy Behind the Exercise

Multi-head attention is a wager that complex meaning benefits from plural relations. The same token may simultaneously participate in order, reference, agreement, topic, and task-specific structure. Separate heads provide room for these relations without requiring one similarity function to rank them all in the same coordinate system.

But a model’s internal plurality should not be mistaken for a committee of small human concepts. Learned representations need not divide reality along the categories we prefer. Specialization is an empirical pattern, not a moral or semantic assignment.

This exercise therefore joins interpretation with intervention. To understand a component is not merely to observe it, but to change it and predict the consequence. The philosophical standard is modest but demanding: claim only the function that survives controlled tests.

17.3.3 Exercise 02

Exercise Title: Implement Linear Projections for Q K V and Prove Parameter Shapes Match $(d_{\text{model}}, d_{\text{model}})$ Logic

Part 1 - Why this matters

Queries, keys, and values are not copies of the input. They are learned linear views of it. In the common joint-projection implementation, each view maps the full model width d_{model} to another vector of the same total width, which is later divided among heads. This explains why the projection matrices have shape $d_{\text{model}} \times d_{\text{model}}$ even though each individual head uses only d_h features.

Two equivalent parameterizations are common. The model may store one matrix per role and split its output into heads, or store a separate $d_{\text{model}} \times d_h$ matrix for every head. Concatenating the per-head matrices along their output dimension reconstructs the joint matrix. The equivalence holds only when slices, bias terms, and feature ordering are matched exactly. This exercise proves the equivalence numerically and through index notation.

Projection bugs often arise from framework conventions. `nn.Linear` stores weights with output features first and input features second, while equations often write XW with the input dimension first and the output dimension second. Both describe the same operation because PyTorch applies $XW^{\mathsf{T}} + b$. A researcher who copies weights between scratch code and library modules without respecting this convention can obtain plausible shapes and incorrect values.

The parameter audit should include biases and the output projection. Standard self-attention with four dense projections contains approximately $4d^2$ weights, not $3Hd^2$. Increasing the number of heads while holding d_{model} fixed changes the partition of the projected space, but does not change the leading projection parameter count.

Part 2 - Mathematical form and reasoning

For $X \in \mathbb{R}^{B \times T \times d}$,

$$Q = XW_Q + b_Q, \quad K = XW_K + b_K, \quad V = XW_V + b_V,$$

with $W_Q, W_K, W_V \in \mathbb{R}^{d \times d}$. Partition

$$W_Q = [W_Q^{(1)} \mid \cdots \mid W_Q^{(H)}], \qquad W_Q^{(r)} \in \mathbb{R}^{d \times d_h}.$$

Then

$$XW_Q = [XW_Q^{(1)} \mid \cdots \mid XW_Q^{(H)}].$$

Ignoring bias, the four projection matrices contain

$$N_{\text{weights}} = 3d^2 + d^2 = 4d^2.$$

With biases, add $4d$. The count is independent of H while d is fixed.

Part 3 - Hints, controls, and evidence

1. Print both mathematical and PyTorch storage conventions beside every weight tensor.

2. Construct joint and per-head projections from the same numbers rather than initializing them independently.

3. Compare projected values before attention so later softmax behavior cannot hide an orientation error.

4. Verify the parameter formula for several head counts with fixed model width.

5. Test cross-attention where query length differs from key/value length; the feature projection shapes should remain unchanged.

6. Add a failed test that intentionally transposes one copied weight and confirm the parity check catches it.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def split_joint_weight(linear, num_heads):
    # nn.Linear weight: (d_out, d_in)
    d_model = linear.weight.shape[0]
    d_head = d_model // num_heads
    # TODO: return H blocks, each (d_head, d_model)
    return ...

def per_head_projection(x, weight_blocks, bias_blocks):
    projected = []
    for w, b in zip(weight_blocks, bias_blocks):
        projected.append(...)  # TODO: PyTorch orientation
    return torch.cat(projected, dim=-1)

def parameter_audit(d_model, num_heads, bias=True):
    q = nn.Linear(d_model, d_model, bias=bias)
    k = nn.Linear(d_model, d_model, bias=bias)
    v = nn.Linear(d_model, d_model, bias=bias)
    o = nn.Linear(d_model, d_model, bias=bias)
    observed = sum(p.numel() for layer in [q, k, v, o]
                   for p in layer.parameters())
    expected = ...
    assert observed == expected
    return observed

def E02():
    x = torch.randn(2, 5, 12)
    # TODO: prove joint == concatenated per-head projections
    return {h: parameter_audit(12, h) for h in [1, 2, 3, 4, 6]}
```

Part 5 - The Philosophy Behind the Exercise

A projection is a learned decision about which distinctions should become visible. The input representation is not queried directly; it is transformed into a language of requests, addresses, and transferable content. Attention therefore begins before softmax, in the geometry chosen by W_Q, W_K, and W_V.

The equivalence between one joint matrix and many per-head matrices also shows that architectural descriptions can depend on coordinate bookkeeping. What appears as several modules may be one block matrix, and what appears as one operation may contain separable subspaces.

Parameter counting disciplines intuition. More heads do not automatically mean more parameters or more knowledge. They change how a fixed representational budget is divided. The question is not how many named components exist, but what transformations the total geometry can support.

17.3.4 Exercise 03

Exercise Title: Debug Head Splitting Bugs by Printing Shapes at Every Step and Matching Expected Dimensions

Part 1 - Why this matters

Head splitting is a short section of code with an unusually large semantic burden. It must divide the final feature axis, move the new head axis into the batch-like position used by batched matrix multiplication, and preserve the identity of every feature. A missing transpose, incorrect `view`, or reversed sequence/head axis can still produce legal tensor operations. Consequently, debugging cannot stop at "the code runs."

This exercise builds a shape ledger. Every line of the forward pass must state the tensor's shape and the meaning of each axis. The ledger includes raw input, projected Q/K/V, split heads, scores, masks, normalized weights, per-head outputs, merged output, and final projection. For cross-attention, query length T_q and memory length T_k must be named separately so a square self- attention example does not conceal axis reversal.

Shape evidence is necessary but not sufficient. Use a tensor whose entries encode batch, token, and feature indices. After splitting, inspect a chosen coordinate and prove which original element it contains. Then merge the heads and require exact reconstruction. This value trace catches permutations that a dimension table cannot detect.

The final stage introduces failure injection. Implement several deliberately wrong split functions: reshape without transpose, transpose wrong axes, divide by the wrong head count, or use `view` on a non-contiguous tensor. Each failure should be detected by the earliest relevant invariant and converted into a regression test.

Part 2 - Mathematical form and reasoning

The intended index mapping is

$$Q'_{b,t,r,u} = Q_{b,t,rd_h+u}, \qquad \widehat{Q}_{b,r,t,u} = Q'_{b,t,r,u}.$$

Thus

$$(B, T_q, d) \to (B, T_q, H, d_h) \to (B, H, T_q, d_h).$$

Scores follow

$$(B, H, T_q, d_h) \ @ \ (B, H, d_h, T_k) \to (B, H, T_q, T_k).$$

The inverse mapping is

$$(B, H, T_q, d_h) \to (B, T_q, H, d_h) \to (B, T_q, Hd_h).$$

Element count is preserved at every reshape, but element order is preserved only when the axis permutation matches these index equations.

Part 3 - Hints, controls, and evidence

1. Use non-equal values for B, H, T_q, T_k, and d_h so accidental axis swaps cannot hide behind equal dimensions.
2. Print axis labels beside shapes, for example `scores [batch, head, query, key]`.
3. Test self-attention and cross-attention with $T_q \neq T_k$.
4. Check strides and `is_contiguous()` before and after transpose.
5. Require split/merge identity and compare selected elements by explicit index arithmetic.
6. For each injected bug, record symptom, earliest violated invariant, root cause, correction, and regression test.

Part 4 - Python code skeleton

```
import torch

def trace_tensor(batch, seq, d_model):
    # Value encodes (batch, token, feature) for manual tracing.
    b = torch.arange(batch)[:, None, None] * 10000
    t = torch.arange(seq)[None, :, None] * 100
    f = torch.arange(d_model)[None, None, :]
    return (b + t + f).float()

def split_heads(x, num_heads):
    b, t, d = x.shape
    assert d % num_heads == 0
    d_head = d // num_heads
    shaped = ...
    split = ...
    return split

def shape_ledger(q, k, v, mask=None):
```

```
    ledger = []
    # TODO: append name, shape, axis labels, strides, contiguity
    return ledger

def E03():
    x = trace_tensor(batch=2, seq=5, d_model=12)
    heads = split_heads(x, num_heads=3)
    # TODO: verify values with explicit index mapping
    merged = ...
    assert torch.equal(merged, x)
    return shape_ledger(heads, heads, heads)
```

Part 5 - The Philosophy Behind the Exercise

Tensor axes are the grammar of neural computation. Two tensors may contain the same numbers and have the same total size while expressing different relationships because their axes assign different roles to those numbers. A silent permutation is therefore a semantic error.

Shape printing becomes scientifically useful only when accompanied by an expected contract. Observation without prediction merely describes whatever the buggy program happened to produce. The researcher must state the intended meaning first and then compare execution against it.

The broader habit is to search for the earliest broken invariant. Downstream NaNs or poor accuracy are distant symptoms. A disciplined debugger moves backward until the first claim about identity, shape, dtype, device, or range becomes false.

17.3.5 Exercise 04

Exercise Title: Compare Single-Head vs Multi-Head Attention and Measure Quality Gain on a Toy Task

Part 1 - Why this matters

Multi-head attention is often justified by capacity to represent several relations, but a fair comparison is subtle. If head count changes while model width also changes, improved quality may come from additional parameters. If the single-head baseline receives a different optimizer budget or converges at a different rate, the experiment measures training choices rather than the architecture. This exercise designs a controlled comparison.

The toy task should require two distinguishable retrieval behaviors. For example, the target may combine information from a location marker and from a token-matching relation. A single head can sometimes solve such a task, so the goal is not to manufacture guaranteed victory. Instead, compare learning speed, final generalization, variance across seeds, and behavior under relation-specific ablations.

Hold d_{model} fixed across head counts. This keeps the dense Q/K/V and output projection parameter count approximately constant, although the softmax structure changes. Also include a wider single-head control with a matched or larger parameter budget. If multi-head attention

wins only against the narrow baseline, the conclusion must reflect that limitation.

Quality should be measured on held-out combinations, not only training examples. Save learning curves, best validation checkpoints, and the exact stopping rule. Inspect whether additional heads provide stable improvement or merely greater seed variance. A small average gain with overlapping confidence intervals is not evidence of a universal advantage.

Part 2 - Mathematical form and reasoning

For $H = 1$, $d_h = d$. For $H > 1$, $d_h = d/H$. With dense projections,

$$N_{\text{MHA}} \approx 4d^2$$

for all valid head counts. Let metric $m_{H,s}$ be the held-out score for head count H and seed s. Report

$$\overline{m}_H = \frac{1}{S}\sum_{s=1}^{S} m_{H,s}, \qquad \sigma_H = \sqrt{\frac{1}{S-1}\sum_{s}(m_{H,s} - \overline{m}_H)^2}.$$

For learning efficiency, also compare the number of updates or tokens needed to cross a fixed validation threshold, not only the best final value.

Part 3 - Hints, controls, and evidence

1. Generate train, validation, and test sets from fixed rules and reserve unseen relation combinations for testing.
2. Compare $H \in \{1, 2, 4, 8\}$ at fixed d_{model} and use the same embedding, head, loss, and optimization budget.
3. Repeat at least five seeds and save every curve, not only successful runs.
4. Report parameters, FLOPs estimate, time per step, peak memory, and task quality.
5. Add a wider single-head control to separate multiple softmax maps from raw width.
6. Inspect errors by relation type and perform head ablations before claiming specialization caused the gain.

Part 4 - Python code skeleton

```
import torch

def build_model(d_model, num_heads, vocab_size, num_classes):
    # TODO: identical surrounding architecture for all comparisons
    return ...

def run_controlled_trial(config, seed, train, val, test):
    torch.manual_seed(seed)
    model = build_model(...)
    parameter_count = sum(p.numel() for p in model.parameters())
    # TODO: train with fixed budget and validation rule
```

```
    return {
        "seed": seed,
        "heads": config["heads"],
        "parameters": parameter_count,
        "curve": ...,
        "best_validation": ...,
        "test_metric": ...,
        "relation_errors": ...,
    }

def E04(configs, seeds=range(5)):
    data = ...
    return [run_controlled_trial(c, s, *data)
            for c in configs for s in seeds]
```

Part 5 - The Philosophy Behind the Exercise

Architectural comparison is an exercise in causal reasoning. Two models differ in many visible and hidden ways, and the experiment attempts to isolate which difference produced the observed behavior. Fairness is not achieved by good intentions; it is constructed through matched controls.

Multi-head attention embodies plurality, but plurality has costs: narrower per-head dimensions, more separate probability maps, and potentially redundant computation. Whether it helps depends on the structure of the task and the optimization path.

The philosophical result may legitimately be conditional. Science advances when a claim becomes more precise, not merely more enthusiastic. "Four heads improved this task under these controls" is more valuable than "more heads are better."

17.3.6 Exercise 05

Exercise Title: Implement Attention Output Projection and Explain Why It Restores d_{model} Space After Head Concatenation

Part 1 - Why this matters

After per-head attention, concatenation already has width $Hd_h = d_{\text{model}}$. It is therefore tempting to believe that the output projection exists only to repair a shape. Its real function is geometric, not dimensional: concatenation places head features beside one another, whereas W_O learns how to mix those features into the coordinate system expected by the residual stream and later layers.

Without W_O, each output coordinate is permanently assigned to one head at that attention sublayer's exit. With W_O, any model-space coordinate can be a learned combination of all head outputs. The projection may preserve a head, rotate several heads into shared features, cancel redundant components, or route information so it aligns with the residual input. This makes the output projection part of the expressive mechanism.

The exercise should compare identity, learned, block-diagonal, and deliberately permuted output projections. Start with a traceable concatenated tensor and verify matrix multiplication manually. Then train identical toy models with W_O enabled or frozen to identity. Match all remaining settings and examine whether the task requires cross-head integration.

Residual addition imposes the external contract. If the attention sublayer is to produce $X + \mathrm{MHA}(X)$, its output must live in the same d_{model} space as X. Shape compatibility alone does not prove semantic alignment, but the learned projection gives the network a mechanism to establish that alignment.

Part 2 - Mathematical form and reasoning

Let

$$C = [Z^{(1)} \mid \cdots \mid Z^{(H)}] \in \mathbb{R}^{B\times T\times d}.$$

The output is

$$Y = CW_O + b_O, \qquad W_O \in \mathbb{R}^{d\times d}.$$

Partitioning W_O by input head gives

$$W_O = \begin{bmatrix} W_O^{(1)} \\ \vdots \\ W_O^{(H)} \end{bmatrix}, \qquad Y = \sum_{r=1}^{H} Z^{(r)} W_O^{(r)} + b_O.$$

This form makes cross-head mixing explicit. The residual sublayer then has

$$R = X + \mathrm{Dropout}(Y),$$

which requires Y and X to share shape and model-space role.

Part 3 - Hints, controls, and evidence

1. Build a two-head example whose head outputs occupy recognizable basis directions.
2. Set W_O to identity, a permutation, a block-diagonal matrix, and a dense matrix; predict each result before execution.
3. Verify PyTorch weight orientation when copying a mathematical W_O into `nn.Linear`.
4. Compare learned and frozen-identity variants on a task requiring a sum or interaction of information from two heads.
5. Record parameter count and gradient norm of W_O during training.
6. Test residual compatibility and add a failure case with the wrong output width.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def concatenate_heads(head_outputs):
    # (B,H,T,d_head) -> (B,T,H*d_head)
    return ...

class OutputProjection(nn.Module):
    def __init__(self, d_model):
        super().__init__()
        self.proj = nn.Linear(d_model, d_model)

    def forward(self, head_outputs):
        concatenated = concatenate_heads(head_outputs)
        return self.proj(concatenated)

def projection_intervention(module, matrix, bias=None):
    with torch.no_grad():
        # TODO: account for nn.Linear's stored orientation
        module.proj.weight.copy_(...)
        module.proj.bias.copy_(...)

def E05():
    heads = ...
    module = OutputProjection(d_model=8)
    results = {}
    for name, matrix in ...:
        projection_intervention(module, matrix)
        results[name] = module(heads)
    return results
```

Part 5 - The Philosophy Behind the Exercise

Concatenation preserves plurality but does not create synthesis. It places perspectives next to one another. The output projection performs a learned act of integration, translating several partial descriptions into the common language of the model.

This distinction resembles the difference between collecting observations and forming a judgment. More sources do not automatically yield a unified representation; a rule of combination is still required. In attention, that rule is itself learned and therefore fallible.

The residual stream adds another lesson: information must not only be produced but made compatible with an ongoing system. Intelligence in a layered network depends on interfaces. A useful internal result that cannot enter the shared representation is computationally isolated.

17.3.7 Exercise 06

Exercise Title: Implement Per-Head Scaling and Show It Rarely Helps Compared to Standard $\sqrt{d_k}$ Scaling

Part 1 - Why this matters

Scaled dot-product attention divides logits by $\sqrt{d_h}$ because the variance of a dot product typically grows with the key dimension. In standard multi-head attention every head has the same width, so one deterministic scale matches the statistical reason for normalization. A learnable or separately chosen scale for each head may appear more flexible, but additional freedom is not automatically useful. It can duplicate transformations already available in Q/K projections, distort entropy, or create unstable optimization.

This exercise implements fixed per-head scales, learnable temperatures, and the standard baseline. The comparison must examine score variance, attention entropy, gradient norms, convergence, and held-out quality. If learned scales move only slightly or produce no stable gain across seeds, the result supports the simpler normalization. If a gain appears, investigate whether it comes from genuinely different head statistics or from an unfair optimization advantage.

Parameterization matters. Optimizing a raw temperature can make it negative or near zero. A positive scale should be represented through an exponential or softplus transformation, and its initial value should reproduce $1/\sqrt{d_h}$. Regularization or clamping may be required, but these controls must be reported because they change the experiment.

The phrase "rarely helps" must emerge from evidence, not be encoded as the desired answer. Run tasks with uniform and deliberately heterogeneous feature statistics. A per-head scale is more likely to matter in the latter case, but Q/K weights may still learn the necessary rescaling themselves.

Part 2 - Mathematical form and reasoning

Standard head logits are

$$S^{(r)} = \frac{Q^{(r)} K^{(r)\mathsf{T}}}{\sqrt{d_h}}.$$

For independent zero-mean components with variance one,

$$\operatorname{Var}(q^{\mathsf{T}} k) \approx d_h,$$

so division by $\sqrt{d_h}$ produces order-one variance. A learnable positive inverse temperature may be written

$$S^{(r)} = \exp(\alpha_r) Q^{(r)} K^{(r)\mathsf{T}}, \qquad \alpha_r^{(0)} = -\tfrac{1}{2} \log d_h.$$

Measure attention entropy

$$\mathcal{H}^{(r)} = -\frac{1}{BT} \sum_{b,i,j} A_{b,i,j}^{(r)} \log(A_{b,i,j}^{(r)} + \varepsilon).$$

Large inverse temperature lowers entropy; small inverse temperature makes the distribution more uniform.

Part 3 - Hints, controls, and evidence

1. Initialize every learned scale to the standard value and keep all other weights identical across variants.

2. Track raw score variance, normalized score variance, attention entropy, scale values, and Q/K gradient norms by head.
3. Compare fixed standard scaling, no scaling, fixed unequal scales, and learned positive scales.
4. Use multiple seeds and both homogeneous and heterogeneous synthetic feature statistics.
5. Check whether Q/K projection norms compensate for learned temperature; scale values alone may be misleading.
6. Report instability, saturation, or lack of improvement as results rather than tuning until the more complex method wins.

Part 4 - Python code skeleton

```
import math
import torch
from torch import nn

class PerHeadTemperature(nn.Module):
    def __init__(self, num_heads, d_head, learnable=True):
        super().__init__()
        initial = -0.5 * math.log(d_head)
        values = torch.full((num_heads,), initial)
        if learnable:
            self.log_scale = nn.Parameter(values)
        else:
            self.register_buffer("log_scale", values)

    def forward(self, raw_scores):
        # raw_scores: (B,H,Tq,Tk)
        scale = self.log_scale.exp()[None, :, None, None]
        return raw_scores * scale

def attention_entropy(weights, eps=1e-9):
    return ...

def E06(configs, seeds=range(5)):
    reports = []
    for config in configs:
        for seed in seeds:
            # TODO: controlled training and per-head statistics
            reports.append(...)
    return reports
```

Part 5 - The Philosophy Behind the Exercise

Machine-learning design often rewards restraint. A parameter can represent a possibility without representing a useful necessity. Every added degree of freedom enlarges the optimization problem and creates another way for the model to become unstable or unidentifiable.

Standard scaling is powerful because it expresses a statistical prior about dot products before learning begins. It is not rigid for its own sake; it removes predictable variation so learning can focus on task-dependent structure.

The experiment teaches a general principle: complexity must earn its place. An elegant extension should be judged by repeatable behavioral improvement, not by the fact that it can be implemented or that its parameters acquire interesting values.

17.3.8 Exercise 07

Exercise Title: Implement Key Padding Mask and Causal Mask Together and Prove Both Are Applied Correctly

Part 1 - Why this matters

Padding and causality express different restrictions. A key padding mask says that selected memory positions do not contain real data and must never receive attention. A causal mask says that query position i may not access key positions later than i. When both are needed, the legal attention graph is the intersection of these rules. Confusing them can leak future information, assign probability to padding, or erase legitimate past tokens.

Mask combination becomes difficult because the source masks usually begin with different ranks. A key padding mask may have shape `(B,Tk)`, while a causal mask has shape `(Tq,Tk)`. Scores have shape `(B,H,Tq,Tk)`. Each mask must be expanded by axis meaning, not merely by adding dimensions until broadcasting succeeds. In particular, padding varies by batch but is shared across heads and query positions; causality varies by query and key position but is normally shared across batch and heads.

The proof should use adversarial values. Place extremely attractive logits in positions that are forbidden by only one mask and positions forbidden by both. After masked softmax, every forbidden probability must be zero within the chosen dtype's tolerance, permitted rows must sum to one, and changing a forbidden value must not change any legal output. This intervention is stronger than checking the visual triangular pattern.

All-masked rows require an explicit policy. Filling every logit with negative infinity and applying softmax produces undefined 0/0 normalization and may create NaNs. The implementation should either prevent such rows by contract, return a documented zero distribution, or use a safe masked-softmax routine. The decision must be tested rather than left to accidental library behavior.

Part 2 - Mathematical form and reasoning

Let $P_{b,j} \in \{0, 1\}$ indicate that key j is padding, and let $C_{i,j} = 1$ when $j > i$. The combined forbidden mask is

$$F_{b,i,j} = P_{b,j} \vee C_{i,j}.$$

Broadcast across heads and define

$$\widetilde{S}_{b,r,i,j} = \begin{cases} S_{b,r,i,j}, & F_{b,i,j} = 0, \\ -\infty, & F_{b,i,j} = 1. \end{cases}$$

Then

$$A_{b,r,i,:} = \mathrm{softmax}(\widetilde{S}_{b,r,i,:}).$$

The required invariants are

$$F_{b,i,j} = 1 \Rightarrow A_{b,r,i,j} = 0, \qquad \sum_{j:F_{b,i,j}=0} A_{b,r,i,j} = 1$$

for every row containing at least one legal key.

Part 3 - Hints, controls, and evidence

1. Use distinct lengths for queries and keys in one cross-attention test, and define whether causality is meaningful for that alignment.
2. Convert the padding mask to `(B,1,1,Tk)` and the causal mask to `(1,1,Tq,Tk)` before combining them.
3. Test batches with different valid lengths, including the minimum legal length and an intentionally all-masked case.
4. Change masked keys and values by huge amounts and confirm all permitted outputs remain unchanged.
5. Verify both boolean-mask and additive-mask implementations using the same legal graph.
6. Repeat under float32 and mixed precision using dtype-appropriate masking constants and finite-value checks.

Part 4 - Python code skeleton

```
import torch

def causal_mask(query_len, key_len, device):
    q = torch.arange(query_len, device=device)[:, None]
    k = torch.arange(key_len, device=device)[None, :]
    return ...  # True means forbidden

def combine_masks(key_padding, causal, num_heads):
    # key_padding: (B,Tk), causal: (Tq,Tk)
    padding4 = ...
    causal4 = ...
    combined = ...
    return combined  # broadcastable to (B,H,Tq,Tk)
```

```
def safe_masked_softmax(scores, forbidden):
    # TODO: define and test all-masked-row behavior
    return ...

def E07():
    scores = ...
    padding = ...
    causal = causal_mask(...)
    forbidden = combine_masks(padding, causal, num_heads=2)
    weights = safe_masked_softmax(scores, forbidden)
    # TODO: zero, row-sum, future-leak, and intervention tests
    return weights
```

Part 5 - The Philosophy Behind the Exercise

A mask is an information law. It does not merely make certain numbers small; it declares which relationships the model is permitted to use. Padding masks protect meaning from artificial storage positions, while causal masks protect time from information that did not yet exist.

Combining masks shows that model behavior is shaped by the intersection of several constraints. Capability is never only the result of weights. It also depends on the architecture of permission surrounding those weights.

The all-masked row reveals a broader lesson: a mathematical operation may be well-defined over ordinary cases and undefined at a boundary. Robust systems do not hope boundaries never appear. They make the boundary policy explicit, observable, and testable.

17.3.9 Exercise 08

Exercise Title: Implement Attention Weight Logging Per Head and Analyze Whether Heads Become Redundant

Part 1 - Why this matters

Multi-head attention allocates several attention distributions, but training may not use them independently. Heads can converge to similar score patterns, produce different weights but similar outputs, or appear redundant on one dataset and essential on another. Per-head logging makes this behavior measurable across training rather than inferred from one final heat map.

Logging must not accidentally retain computation graphs. Storing raw attention tensors from every batch can exhaust GPU memory and keep backward graphs alive. Detach the tensors, move selected summaries to CPU, sample batches according to a declared schedule, and preserve metadata such as layer, head, seed, mask, sequence length, and checkpoint. Raw matrices should be saved only for a small fixed probe set.

Redundancy has several meanings. Weight redundancy asks whether two heads place probability on the same keys. Output redundancy asks whether their weighted value sums point in similar directions. Functional redundancy asks whether one head can be removed without reducing held-out performance. These measures need not agree because different values or

output-projection blocks can turn similar weights into different contributions.

Analyze change over time. Heads may begin similarly, specialize during early training, and later become correlated again. Compare trained models with their randomly initialized baselines so that similarity caused by masks or position structure is not mistaken for learned redundancy.

Part 2 - Mathematical form and reasoning

For flattened attention maps a_r, a_s, cosine similarity is

$$\cos(a_r, a_s) = \frac{a_r^\top a_s}{\|a_r\|_2 \|a_s\|_2}.$$

Jensen–Shannon divergence compares probability rows symmetrically:

$$\mathrm{JSD}(p, q) = \tfrac{1}{2}\,\mathrm{KL}(p\|m) + \tfrac{1}{2}\,\mathrm{KL}(q\|m), \qquad m = \tfrac{1}{2}(p + q).$$

For per-head outputs $Z^{(r)}$, output similarity may be measured using cosine similarity or centered-kernel methods. Functional importance for head r is

$$I_r = \mathcal{M}_{\text{base}} - \mathcal{M}_{\text{ablated}(r)},$$

where $\mathcal{M}$ is a held-out task metric and ablation is performed before the output projection.

Part 3 - Hints, controls, and evidence

1. Use a fixed probe dataset at every checkpoint so temporal comparisons do not mix model change with input change.
2. Log row entropy, maximum weight, pairwise divergence, head-output norms, and output similarity.
3. Detach and summarize on CPU; verify logging does not change gradients or training speed materially.
4. Compare similarity before training, during training, and at the selected checkpoint across several seeds.
5. Perform single-head and grouped-head ablations because redundancy may be distributed across more than two heads.
6. Treat heat maps as descriptive evidence and ablation as stronger functional evidence.

Part 4 - Python code skeleton

```
import torch

class HeadLogger:
    def __init__(self, keep_raw_batches=2):
        self.keep_raw_batches = keep_raw_batches
        self.records = []
```

```
    @torch.no_grad()
    def record(self, weights, head_outputs, metadata):
        # TODO: detach, summarize, and move only needed data to CPU
        self.records.append({
            "metadata": metadata,
            "entropy": ...,
            "max_weight": ...,
            "pairwise_jsd": ...,
            "output_cosine": ...,
        })

def ablation_importance(model, loader, metric):
    baseline = ...
    importance = []
    for head in range(model.num_heads):
        # TODO: zero this head before output projection
        importance.append(baseline - ...)
    return importance

def E08(model, probe_loader, checkpoints):
    logger = HeadLogger()
    # TODO: load checkpoints and record the same probes
    return logger.records, ablation_importance(model, probe_loader, ...)
```

Part 5 - The Philosophy Behind the Exercise

Redundancy is not automatically waste. Replicated functions can create robustness, alternative optimization paths, or representations that appear similar under one measurement but differ under another condition. Biological and engineered systems often preserve overlapping capacity for good reasons.

At the same time, naming several heads does not prove the existence of several ideas. Architecture offers a space of possibilities; training decides how that space is occupied. Measurement protects us from reading component count as conceptual diversity.

This exercise also teaches restraint in observation. Recording more tensors does not guarantee more understanding. Useful logging begins with a question, chooses statistics tied to that question, and preserves enough context for the observation to be reproduced.

17.3.10 Exercise 09

Exercise Title: Explain Attention as Low-Rank Retrieval and Relate Heads to Different Subspaces Conceptually

Part 1 - Why this matters

Each attention head projects model-width representations into a smaller space before calculating compatibility. The pre-softmax score matrix is therefore constructed from QK^T with inner dimension d_h. Its matrix rank cannot exceed d_h before masks and softmax are applied. This offers a useful geometric interpretation: each head evaluates retrieval through a restricted

bilinear subspace rather than through every possible pairwise relation in the full model space.

The interpretation must be stated carefully. The final attention matrix after row-wise softmax need not have rank at most d_h because softmax is nonlinear. Likewise, a low-rank score matrix does not mean the head can retrieve only a few tokens or that its output is trivial. Masks, value vectors, multiple layers, and the output projection change the functional picture. The rank claim applies to the unnormalized content-based score component.

Multiple heads create several low-dimensional compatibility geometries. One head may be sensitive to a positional pattern and another to token identity, provided the learned projections arrange those features appropriately. Their score matrices are not summed before softmax; each head forms its own probability distribution. This is different from simply increasing the rank of one shared score matrix.

The exercise should use singular-value analysis on synthetic and trained examples. Compare theoretical maximum rank, numerical rank, spectral decay, and approximation error. Then alter d_h and head count while holding total model width fixed. The purpose is to connect algebraic capacity with observed retrieval behavior without claiming that rank alone explains model quality.

Part 2 - Mathematical form and reasoning

For one head,

$$S = QK^{\mathsf{T}}, \qquad Q \in \mathbb{R}^{T_q \times d_h}, \quad K \in \mathbb{R}^{T_k \times d_h}.$$

Therefore

$$\operatorname{rank}(S) \le \min(T_q, T_k, d_h).$$

Equivalently, with projected inputs,

$$S = X_q W_Q W_K^{\mathsf{T}} X_k^{\mathsf{T}},$$

where $W_Q W_K^{\mathsf{T}}$ has rank at most d_h. For singular values $\sigma_1 \ge \cdots$, define an effective rank, for example,

$$r_\tau = \#\{i : \sigma_i > \tau \sigma_1\}.$$

The nonlinear matrix

$$A = \operatorname{softmax}(S/\sqrt{d_h} + M)$$

must be analyzed separately; the same rank bound does not transfer through softmax.

Part 3 - Hints, controls, and evidence

1. Construct random Q and K with known dimensions and confirm the exact or numerical rank bound on unmasked logits.

2. Compare logits before scaling, after additive masking, and after softmax; state which transformation changes the rank argument.

3. Plot singular spectra for each head on a fixed probe set and report the tolerance used for numerical rank.
4. Vary d_h, sequence length, and head count while keeping d_{model} fixed.
5. Relate spectral observations to retrieval accuracy on a controlled task, but avoid treating correlation as causation.
6. Include a counterexample showing softmax output rank exceeding the pre-softmax bound.

Part 4 - Python code skeleton

```
import torch

def numerical_rank(matrix, relative_tol=1e-6):
    singular = torch.linalg.svdvals(matrix)
    threshold = relative_tol * singular[..., :1]
    return (singular > threshold).sum(dim=-1), singular

def rank_report(q, k, mask=None):
    logits = q @ k.transpose(-2, -1)
    scaled = logits / (q.shape[-1] ** 0.5)
    masked = ... if mask is not None else scaled
    weights = torch.softmax(masked, dim=-1)
    return {
        "logit_rank": numerical_rank(logits),
        "masked_rank": numerical_rank(masked),
        "softmax_rank": numerical_rank(weights),
        "theoretical_bound": min(q.shape[-2], k.shape[-2], q.shape[-1]),
    }

def E09():
    reports = []
    for d_head in [1, 2, 4, 8, 16]:
        q, k = ...
        reports.append(rank_report(q, k))
    # TODO: add trained heads and a softmax counterexample
    return reports
```

Part 5 - The Philosophy Behind the Exercise

Projection is selective ignorance. A head gains a tractable relation space by discarding directions that its projection does not preserve. This restriction is not merely a weakness; useful abstraction often depends on ignoring most available variation.

Several heads represent several selective views. Their power comes not from recovering the entire world in every head, but from allowing different losses of information to coexist. What one subspace ignores, another may preserve.

Rank analysis also demonstrates the boundary between explanation and proxy. An algebraic bound illuminates one stage of computation, but the full network contains nonlinearities, masks, values, residuals, and repeated layers. A good theory identifies exactly where its statement holds and refuses to expand that statement beyond its proof.

17.3.11 Exercise 10

Exercise Title: Run Multi-Head Attention on CUDA and Validate Correctness Under Mixed Precision Constraints

Part 1 - Why this matters

CUDA execution and mixed precision change the numerical environment of multi-head attention. Matrix multiplications may use reduced-precision inputs or specialized hardware paths, while softmax and reductions may be promoted or fused according to the backend. The intended function should remain stable, but exact equality with a float32 CPU reference is neither expected nor a sufficient test.

The parity experiment must begin from identical master tensors and parameters. Generate them once in float32, clone the CPU reference, and copy the same values to CUDA. Compare scratch and library implementations in evaluation mode before adding autocast. Then test float32 CUDA, float16 autocast, and bfloat16 where hardware support exists. Record forward output, attention probabilities, loss, gradients, masked-position invariants, and finiteness.

Mixed precision risks are concentrated around logits and softmax. Large Q/K norms can overflow dot products; large negative mask constants may become infinite; small probabilities or gradients may underflow. A stable implementation may compute selected operations in float32 even when projection and value products use lower precision. The report should state actual dtypes inside the critical path rather than assuming autocast treats every operation equally.

Gradient scaling applies to training losses, not to the forward probability distribution. Include a small optimizer step with the modern AMP scaling API, detect skipped steps and non-finite gradients, and compare parameter updates against a float32 reference over several seeds and input scales.

Part 2 - Mathematical form and reasoning

For reference y and candidate $\widehat{y}$, report

$$E_{\text{abs}} = \max_i |\widehat{y}_i - y_i|, \qquad E_{\text{rel}} = \max_i \frac{|\widehat{y}_i - y_i|}{\max(|y_i|, \varepsilon)}.$$

Use the declared rule

$$|\widehat{y}_i - y_i| \le \texttt{atol} + \texttt{rtol}|y_i|.$$

For each head, monitor logit range and entropy as well as error. Reduced precision changes are especially important when they alter the discrete winner

$$\arg\max_j A_{b,r,i,j}$$

or allow a forbidden key to receive nonzero probability.

Part 3 - Hints, controls, and evidence

1. Skip with a documented status when CUDA is unavailable; do not silently fall back to CPU.
2. Test multiple seeds, sequence lengths, head counts, mask patterns, and input scales.
3. Declare dtype-specific tolerances before examining results.
4. Inspect actual dtypes of logits, softmax inputs, weights, outputs, loss, and gradients under autocast.
5. Check maximum errors, top-attended index agreement, row sums, mask invariants, and finite values.
6. Synchronize only for timing; correctness comparisons require copied tensors, not asynchronous wall-clock logic.

Part 4 - Python code skeleton

```
import contextlib
import torch

def run_case(model, x, mask, device, amp_dtype=None):
    model = model.to(device)
    x = x.to(device).detach().requires_grad_(True)
    enabled = amp_dtype is not None and device.type == "cuda"
    ctx = torch.autocast("cuda", dtype=amp_dtype, enabled=enabled)
    with ctx:
        output, weights = model(x, x, x, mask)
        loss = ...
    loss.backward()
    return {
        "output": output.detach().float().cpu(),
        "weights": weights.detach().float().cpu(),
        "gradient": x.grad.detach().float().cpu(),
        "finite": bool(torch.isfinite(output).all()),
    }

def compare(reference, candidate, atol, rtol):
    return {"max_abs": ..., "max_rel": ..., "allclose": ...}

def E10(seeds=range(10)):
    if not torch.cuda.is_available():
        return {"status": "skipped", "reason": "CUDA unavailable"}
    # TODO: paired float32/float16/bfloat16 cases and AMP update test
    return ...
```

Part 5 - The Philosophy Behind the Exercise

Mixed precision asks how much numerical detail a computation can surrender without surrendering its function. The answer is not contained in the dtype name. It depends on scale, operation, architecture, data, and the behavioral tolerance of the task.

Hardware optimization therefore becomes an epistemic question: what evidence shows that the faster physical process still instantiates the same model? A small average error may be harmless, while one leaked future token or one NaN can destroy the meaning of the system.

Correctness under approximation is never "close enough" in the abstract. It requires explicit invariants, declared tolerances, adversarial cases, and an understanding of which differences are numerically small but semantically large.

17.3.12 Exercise 11

Exercise Title: Benchmark Multi-Head Attention Speed vs Head Count and Identify Practical Head Count Choices

Part 1 - Why this matters

At fixed d_{model}, the leading dense-attention arithmetic is often described as nearly independent of head count. The product Hd_h remains equal to d_{model}. Real hardware does not execute only asymptotic formulas, however. More heads create smaller per-head matrix dimensions, alter kernel shapes, increase attention-map count, and may change which fused kernels are available. Speed can therefore vary even when theoretical FLOPs remain similar.

Benchmarking must cover regimes. Small batches and short sequences may be dominated by launch overhead. Medium shapes may use accelerator kernels efficiently. Long sequences may become memory-bound because score and probability tensors scale with BHT^2. A practical head count depends on model width, sequence length, batch, dtype, hardware, software version, and whether the measurement is forward inference or forward-plus-backward training.

Head count also affects model capacity, so speed cannot select it alone. The final recommendation must combine latency, throughput, peak memory, numerical stability, and task quality. A fast configuration whose d_h is too small to learn the required relations is not practical. Nor is a high-quality setting that exceeds the deployment latency budget.

Use robust timing. Warm up kernels, synchronize CUDA around measured regions, avoid data creation inside the timed function, repeat enough iterations, and report medians and percentiles. Include `torch.compile` or fused attention only as separate labeled variants because compilation overhead and kernel selection change the comparison.

Part 2 - Mathematical form and reasoning

Projection work is approximately

$$\mathcal{O}(BTd^2),$$

while score and value aggregation work is

$$\mathcal{O}(BHT^2d_h) = \mathcal{O}(BT^2d).$$

Explicit attention-map storage is

$$\mathcal{O}(BHT^2),$$

which does grow with H when d and T are fixed. Report throughput

$$\text{tokens/s} = \frac{BT}{\text{seconds/iteration}}$$

and a quality-adjusted feasibility table rather than collapsing every concern into one score.

Part 3 - Hints, controls, and evidence

1. Choose a model width divisible by many head counts and record resulting d_h for each case.
2. Benchmark forward inference and full training step separately.
3. Sweep batch and sequence length; do not generalize from one tensor shape.
4. Use CUDA events or synchronized wall time, warm-up, repeated trials, and median plus tail latency.
5. Reset and record peak allocated and reserved memory for each isolated configuration.
6. Add held-out task quality and reject head counts that fail either the quality or systems requirement.

Part 4 - Python code skeleton

```
import statistics
import time
import torch

def timed_cuda(fn, warmup=20, repeats=100):
    for _ in range(warmup):
        fn()
    torch.cuda.synchronize()
    times = []
    for _ in range(repeats):
        start = torch.cuda.Event(enable_timing=True)
        end = torch.cuda.Event(enable_timing=True)
        start.record(); fn(); end.record()
        torch.cuda.synchronize()
        times.append(start.elapsed_time(end))
    return {"median_ms": statistics.median(times), "p95_ms": ...}

def benchmark_shape(d_model, heads, batch, seq, dtype):
    model, x = ..., ...
    torch.cuda.reset_peak_memory_stats()
    timing = timed_cuda(lambda: ...)
```

```
    return {**timing,
            "peak_bytes": torch.cuda.max_memory_allocated(),
            "d_head": d_model // heads}

def E11():
    # TODO: sweep heads, sequence lengths, batches, dtypes, and modes
    return ...
```

Part 5 - The Philosophy Behind the Exercise

Asymptotic complexity tells us how costs grow, not how a particular machine will experience a particular shape. Engineering lives in the gap between the formula and the device: memory hierarchies, kernel launch, vector width, fusion, and runtime scheduling all influence what becomes practical.

There is therefore no context-free "best number of heads." Design is a negotiation among representation, optimization, hardware, and service goals. An architecture is practical only relative to a workload and a constraint.

Benchmarking is not a race to produce the smallest number. It is a method for discovering where the system changes regime and for making choices whose costs and benefits can be defended.

17.3.13 Exercise 12

Exercise Title: Compare float32 vs float16 Multi-Head Attention Stability and Detect Softmax Overflow Scenarios

Part 1 - Why this matters

Float16 provides limited exponent range and precision. Multi-head attention contains dot products, scaling, masking, exponentiation, normalization, and weighted sums, so failures can emerge at several stages. A naive softmax that calculates `exp(scores)` directly can overflow even in float32; in float16 the safe operating range is smaller. Stable softmax subtracts the row maximum, but it cannot repair logits that already became infinite during the QK product.

This exercise maps the failure boundary by scaling Q and K across several orders of magnitude. For each scale, compare raw logits, scaled logits, shifted logits, weights, outputs, losses, and gradients. Distinguish overflow from underflow: an infinite logit can create NaNs, while very negative shifted logits may underflow to zero and yield an extremely sharp but finite distribution. Both can alter optimization.

Mask constants require care. A large negative finite number that is adequate in float32 may convert to negative infinity in float16. This may be acceptable for rows with legal keys, but an all-masked row remains undefined. Conversely, a constant that is not negative enough can leave measurable probability on forbidden positions. Test mask behavior at each dtype and score scale.

The goal is not to declare float16 unsafe. Modern fused attention and autocast may accumulate or normalize in higher precision. The report must identify the actual implementation path and compare naive, stable, promoted-softmax, and library kernels under the same inputs.

Part 2 - Mathematical form and reasoning

Naive softmax is

$$p_j = \frac{e^{s_j}}{\sum_k e^{s_k}}.$$

The shift-invariant stable form is

$$p_j = \frac{e^{s_j - m}}{\sum_k e^{s_k - m}}, \qquad m = \max_k s_k.$$

If Q and K are scaled by c, logits scale by c^2:

$$\frac{(cQ)(cK)^\mathsf{T}}{\sqrt{d_h}} = c^2 \frac{QK^\mathsf{T}}{\sqrt{d_h}}.$$

Track the softmax Jacobian

$$\frac{\partial p_i}{\partial s_j} = p_i(\delta_{ij} - p_j),$$

because extremely sharp distributions may remain finite while producing very small gradients for most entries.

Part 3 - Hints, controls, and evidence

1. Sweep input scale logarithmically and locate the first non-finite value at every intermediate tensor.
2. Compare naive exponentiation, max-shifted softmax, float32-promoted softmax, and the framework's fused implementation.
3. Record dtype, minimum and maximum, finite fraction, entropy, zero fraction, and gradient norm.
4. Include near-tied logits as well as dominant logits; reduced mantissa precision can change rankings before overflow occurs.
5. Test normal, causal, padded, and all-masked rows.
6. Repeat seeds and report a failure boundary range rather than one lucky example.

Part 4 - Python code skeleton

```
import torch

def stable_softmax(scores, dim=-1, promote=False):
    original_dtype = scores.dtype
    work = scores.float() if promote else scores
```

```
    shifted = work - work.amax(dim=dim, keepdim=True)
    weights = shifted.exp() / shifted.exp().sum(dim=dim, keepdim=True)
    return weights.to(original_dtype)

def tensor_stats(x):
    return {"min": ..., "max": ..., "finite_fraction": ...,
            "zero_fraction": ..., "dtype": str(x.dtype)}

def stress_case(scale, dtype, implementation):
    q, k, v = ...
    raw = q @ k.transpose(-2, -1)
    scaled = raw / (q.shape[-1] ** 0.5)
    weights = implementation(scaled)
    output = weights @ v
    # TODO: backward and collect stage-by-stage statistics
    return ...

def E12():
    return [stress_case(10.0 ** p, dtype, impl)
            for p in range(-3, 7)
            for dtype in [torch.float32, torch.float16]
            for impl in ...]
```

Part 5 - The Philosophy Behind the Exercise

Numerical representation is part of the model's world. The real numbers of an equation are approximated by a finite set whose gaps widen with magnitude. Operations that are identical on paper can therefore produce different behavior when implemented in different formats.

Stability is not the absence of a crash. A distribution may remain finite while becoming so sharp that useful gradient information disappears. The researcher must observe the dynamics of learning, not only whether a tensor contains NaN.

The exercise teaches that optimization begins with respecting representation limits. Clever architecture cannot learn through numbers the machine can no longer distinguish.

17.3.14 Exercise 13

Exercise Title: Implement Gradient Check for Multi-Head Attention on Tiny Tensor and Validate Backward Correctness

Part 1 - Why this matters

A forward pass can match expected values while backward propagation remains wrong because of an in-place operation, detached tensor, incorrect mask path, or faulty custom function. Gradient checking compares autograd derivatives with finite-difference estimates of the scalar loss. On a tiny multi-head attention instance, it provides independent evidence that projections, splitting, softmax, head merging, and output projection form a differentiable computation with the intended sensitivities.

Finite differences are numerically delicate. The perturbation ε must be small enough to approximate a derivative but large enough to survive floating-point rounding. Use float64, disable dropout, avoid points where the mask or another discrete branch changes, and choose moderate logits so softmax is not fully saturated. Compare several epsilon values rather than accepting one.

Check inputs and parameters separately. Q, K, and V projections play different roles, and a mistake may affect only one gradient path. Include W_Q, W_K, W_V, W_O, biases, and the input. For self-attention, the same input feeds three branches, so its total gradient is the sum of query, key, and value contributions. A cross-attention case can separate these paths more clearly.

Masking should also be verified. A masked value that cannot affect any output should have zero gradient through the attention route. Perturbing it must not change the loss. This is both a derivative check and a proof of the information boundary established by the mask.

Part 2 - Mathematical form and reasoning

For scalar loss $L(\theta)$, the centered finite difference is

$$g_i^{\mathrm{FD}} = \frac{L(\theta + \varepsilon e_i) - L(\theta - \varepsilon e_i)}{2\varepsilon}.$$

Compare with autograd g_i^{AD} using

$$E_i = \frac{|g_i^{\mathrm{AD}} - g_i^{\mathrm{FD}}|}{\max(1, |g_i^{\mathrm{AD}}|, |g_i^{\mathrm{FD}}|)}.$$

For a smooth implementation, error usually decreases as ε is reduced until rounding error begins to dominate. The epsilon sweep should show this tradeoff rather than treating the smallest epsilon as automatically best.

Part 3 - Hints, controls, and evidence

1. Use a tiny float64 model, for example $B = 1$, $T = 2$ or 3, $d = 4$, and $H = 2$.
2. Set dropout to zero and use a smooth scalar loss involving every output coordinate.
3. Run `torch.autograd.gradcheck` and also implement selected manual finite differences so the method is understood.
4. Sweep epsilon and report absolute and normalized error for each tensor.
5. Test self-attention, cross-attention, and one masked case.
6. Inject a detach or incorrect merge, prove the checker fails, then retain the case as a regression test.

Part 4 - Python code skeleton

```
import torch

def scalar_objective(model, q, k, v, mask=None):
    output, weights = model(q, k, v, mask)
    coefficients = torch.linspace(0.3, 1.1, output.numel(),
                                  dtype=output.dtype,
                                  device=output.device).reshape_as(output)
    return (output * coefficients).sum()

def centered_difference(loss_fn, tensor, flat_index, epsilon):
    with torch.no_grad():
        original = tensor.view(-1)[flat_index].item()
        tensor.view(-1)[flat_index] = original + epsilon
    plus = loss_fn().item()
    with torch.no_grad():
        tensor.view(-1)[flat_index] = original - epsilon
    minus = loss_fn().item()
    with torch.no_grad():
        tensor.view(-1)[flat_index] = original
    return (plus - minus) / (2 * epsilon)

def E13():
    model, q, k, v, mask = ...  # all float64
    # TODO: autograd gradients, epsilon sweep, gradcheck,
    # masked-gradient invariant, and injected-failure test
    return ...
```

Part 5 - The Philosophy Behind the Exercise

Automatic differentiation gives exact derivatives of the program it receives, not of the program the researcher intended to write. Gradient checking creates a second route to the answer and asks whether the two routes agree.

This independence is the source of trust. A test that merely repeats the same implementation in different syntax can repeat the same mistake. Finite differences use the behavior of the whole function to interrogate its local sensitivity.

The exercise also demonstrates that correctness has layers. Shapes may be correct, outputs may be plausible, and training loss may decrease, while the derivative structure is still wrong. Reliable research requires evidence at each layer rather than faith in the final metric.

17.3.15 Exercise 14

Exercise Title: Stress Test Multi-Head Attention With Long Sequences and Observe GPU Memory Explosion Conditions

Part 1 - Why this matters

Dense multi-head attention creates a relation between every query and every key for every head. As sequence length grows, score and probability tensors grow quadratically. A configuration that appears inexpensive at 512 tokens may become impossible at 4,096 or 8,192 tokens

even when batch size, model width, and parameter count remain unchanged. This exercise maps that transition on the available GPU rather than waiting for an uncontrolled out-of-memory failure during a larger experiment.

Memory must be separated into categories. Parameters and optimizer states are mostly independent of sequence length. Input, Q/K/V, and head outputs grow linearly with T. Explicit scores and attention weights grow as BHT^2. Backward propagation retains activations and adds gradients, so training can require substantially more memory than inference. Allocated memory, reserved memory, and externally reported device usage describe different quantities and must not be mixed.

The stress test should increase one factor at a time: sequence length, batch, head count, dtype, and training mode. Use a separate subprocess for deliberate OOM cases so the main experiment can recover cleanly. Catching an exception inside a process does not always return the allocator and CUDA context to a useful state. Record the largest successful configuration and the first failed configuration, but do not attempt to consume all system memory without a declared safety margin.

Compare explicit scratch attention with a memory-efficient fused implementation when available. Fused kernels can avoid materializing the full probability matrix for backward, changing practical memory without changing dense attention's mathematical interaction pattern. The conclusion must distinguish algorithmic score complexity from implementation-level activation storage.

Part 2 - Mathematical form and reasoning

If one attention-like tensor has shape (B, H, T, T) and element size s bytes, its storage is

$$M_{\text{map}} = BHT^2 s.$$

Storing both scores and probabilities gives at least

$$M_{\text{scores+weights}} \approx 2BHT^2 s,$$

before backward buffers, masks, temporary kernels, and allocator overhead. Doubling T multiplies this term by four:

$$M(2T) = 4M(T).$$

The linear Q/K/V term is approximately

$$M_{QKV} = 3BTds.$$

The experiment should compare these estimates with measured incremental peak memory and explain the difference.

Part 3 - Hints, controls, and evidence

1. Query available device memory and reserve a safety margin before forming each test configuration.
2. Run forward inference, forward with gradients disabled, and full forward/backward as separate modes.

3. Reset peak statistics, synchronize, and measure an isolated case in a fresh process when possible.

4. Record allocated, reserved, peak allocated, peak reserved, dtype, shape, backend, and whether attention weights were returned.

5. Compare theoretical tensor sizes with observed peaks and identify hidden temporary or retained tensors.

6. Stop the sweep after a declared failure boundary; do not repeatedly trigger OOM on the interactive process.

Part 4 - Python code skeleton

```
import multiprocessing as mp
import torch

def estimate_map_bytes(batch, heads, seq, dtype, copies=2):
    element_size = torch.empty((), dtype=dtype).element_size()
    return copies * batch * heads * seq * seq * element_size

def isolated_case(config, queue):
    try:
        torch.cuda.set_device(config["device"])
        torch.cuda.empty_cache()
        torch.cuda.reset_peak_memory_stats()
        model, x = ...
        # TODO: run requested forward/backward mode
        torch.cuda.synchronize()
        queue.put({"status": "ok",
                   "peak_allocated": torch.cuda.max_memory_allocated(),
                   "peak_reserved": torch.cuda.max_memory_reserved()})
    except torch.cuda.OutOfMemoryError as error:
        queue.put({"status": "oom", "message": str(error)})

def E14(configurations):
    # TODO: launch one isolated process per safe configuration
    # and stop according to a declared boundary policy
    return ...
```

Part 5 - The Philosophy Behind the Exercise

Attention makes relationships explicit, and those relationships occupy space. The quadratic matrix is not an accidental inconvenience beside the model's intelligence; it is the physical representation of universal pairwise access in dense attention.

Resource limits therefore shape the kinds of context a model can consider. Long-context research is partly a search for new forms of memory and access that preserve useful relations

without materializing every possible pair.

Stress testing turns failure into a boundary that can be studied. An OOM error without context is an interruption. A measured scaling curve, safety margin, and reproducible threshold form knowledge that can guide architecture and deployment.

17.3.16 Exercise 15

Exercise Title: Implement Efficient Mask Broadcasting and Prove It Avoids Silent Shape Expansion Bugs

Part 1 - Why this matters

Broadcasting allows one logical mask to constrain every attention head without physically copying it to the full score shape. A padding mask of `(B,Tk)` can become `(B,1,1,Tk)`, and a causal mask of `(Tq,Tk)` can become `(1,1,Tq,Tk)`. These singleton dimensions describe which axes share the same rule. Correct broadcasting saves memory and makes the mask semantics explicit.

Silent expansion bugs occur when an axis is inserted in the wrong location but happens to be compatible with the current shape. Equal values such as $B = H = 4$ can hide the error. A padding rule intended to vary by batch may instead vary by head; a query mask may be applied along the key axis. The code runs, but different examples or heads receive the wrong information boundary.

Avoid constructing a repeated `(B,H,Tq,Tk)` boolean tensor unless a kernel requires it. `expand` creates a view with zero strides along shared axes, while `repeat` allocates copies. The logical result can be the same, but the memory behavior is not. The exercise should measure storage, strides, contiguity, and peak memory for broadcast, expand, and repeat variants.

Proof requires asymmetric test data. Give every batch, head, query, and key a distinct encoded value and define different valid lengths per batch. Compare the broadcast result against a slow reference constructed with explicit loops. Then use interventions on forbidden positions to prove that the final attention output respects the same logical mask.

Part 2 - Mathematical form and reasoning

Scores have shape

$$S \in \mathbb{R}^{B \times H \times T_q \times T_k}.$$

A padding mask $P \in \{0,1\}^{B \times T_k}$ is lifted as

$$P'_{b,1,1,j} = P_{b,j},$$

and a causal mask $C \in \{0,1\}^{T_q \times T_k}$ as

$$C'_{1,1,i,j} = C_{i,j}.$$

Broadcasting defines

$$F_{b,r,i,j} = P'_{b,1,1,j} \vee C'_{1,1,i,j}.$$

The logical full mask contains BHT_qT_k entries, but the source storage is only $BT_k + T_qT_k$ entries before any kernel materialization.

Part 3 - Hints, controls, and evidence

1. Choose unequal B, H, T_q, and T_k in the primary test.
2. Write a four-loop reference mask and compare it exactly with the broadcasted result.
3. Inspect `shape`, `stride()`, `storage().nbytes()`, and contiguity for unsqueeze, expand, repeat, and combined masks.
4. Measure peak memory at long sequence length without deliberately crossing the safety boundary from E14.
5. Test batch-specific padding together with a shared causal rule and a head-specific optional mask.
6. Inject each wrong unsqueeze position and confirm the test fails even if a contrived equal-dimension case would pass.

Part 4 - Python code skeleton

```
import torch

def broadcast_masks(padding, causal, head_mask=None):
    # padding: (B,Tk), causal: (Tq,Tk)
    p = padding[:, None, None, :]
    c = causal[None, None, :, :]
    combined = p | c
    if head_mask is not None:
        # TODO: define head-mask contract and broadcast axes
        combined = combined | ...
    return combined

def loop_reference(padding, causal, num_heads):
    b, tk = padding.shape
    tq = causal.shape[0]
    full = torch.empty(b, num_heads, tq, tk, dtype=torch.bool)
    # TODO: explicit semantic reference
    return full

def storage_report(x):
    return {"shape": tuple(x.shape), "stride": x.stride(),
            "elements": x.numel(), "storage_bytes": ...}

def E15():
    padding, causal = ..., ...
    efficient = broadcast_masks(padding, causal)
```

```
reference = loop_reference(padding, causal, num_heads=3)
assert torch.equal(efficient.expand_as(reference), reference)
return ...
```

Part 5 - The Philosophy Behind the Exercise

Broadcasting is a compact expression of symmetry. A singleton head axis says that the same law applies to every head; a singleton query axis says that the rule does not depend on which position is asking. Tensor shape thus records which distinctions matter and which are intentionally shared.

Efficiency follows from recognizing invariance. Repeating identical data stores the same fact many times, while broadcasting preserves one fact and a rule for where it applies. Many scalable algorithms depend on this ability to represent structure rather than copies.

But compression increases the importance of semantics. A wrong singleton axis can distribute a law across the wrong dimension. The concise representation is powerful only when every axis has been named and tested.

17.3.17 Exercise 16

Exercise Title: Profile GPU Kernels for Multi-Head Attention and Identify Softmax and MatMul Hotspots

Part 1 - Why this matters

Wall-clock timing says how long attention took, but not where the time was spent. Multi-head attention contains input projections, QK matrix multiplication, scaling and masking, softmax, dropout, AV matrix multiplication, head merging, and output projection. A GPU profiler reveals which kernels dominate, whether operations are fused, how much time is spent launching small kernels, and whether memory copies or synchronization distort the measurement.

The hotspot changes with shape. Projection GEMMs may dominate short sequences and large model widths. The T^2 score and value products become dominant as sequence length grows. Explicit softmax may appear as several reduction and elementwise kernels, while fused scaled-dot-product attention may combine masking, normalization, dropout, and value aggregation without storing the full attention map. The profiler trace must be interpreted in relation to the implementation actually selected.

Profiling itself can perturb performance. Capture a short scheduled window after warm-up, avoid printing or saving tensors in the measured region, and use named ranges around logical stages. Record CPU and CUDA time, call count, input shapes, memory activity, and stack traces only when their overhead is justified. A trace from one iteration without warm-up may mostly describe initialization or compilation.

The exercise should compare scratch, library, and fused variants under matched inputs and modes. If attention weights are requested, a fused kernel may be disabled or forced to materialize additional data. This is a real tradeoff between observability and performance and must be documented.

Part 2 - Mathematical form and reasoning

The principal matrix products are

$$QK^{\mathsf{T}} : (BH, T, d_h)@(BH, d_h, T) \to (BH, T, T),$$

and

$$AV : (BH, T, T)@(BH, T, d_h) \to (BH, T, d_h).$$

Each has work on the order of

$$\mathcal{O}(BHT^2 d_h) = \mathcal{O}(BT^2 d).$$

Projection work is approximately $\mathcal{O}(BTd^2)$. The crossover between projection-dominated and attention-dominated regimes depends roughly on the relationship between T and d, but measured kernel efficiency and memory traffic determine the actual boundary.

Part 3 - Hints, controls, and evidence

1. Add profiler ranges for projections, score matmul, mask/softmax, value matmul, merge, and output projection.
2. Warm up before the active profiling schedule and profile only a small, representative number of steps.
3. Compare short, medium, and long sequences at fixed model width, then vary model width separately.
4. Sort profiler tables by self CUDA time and CUDA memory, and retain the full trace for timeline analysis.
5. Record whether fused attention was used and whether returning weights or dropout changed kernel selection.
6. Convert the hotspot into a testable optimization, then re-profile rather than assuming the optimization helped.

Part 4 - Python code skeleton

```
import torch
from torch.profiler import ProfilerActivity, profile, record_function

def instrumented_forward(model, x, mask):
    with record_function("mha_total"):
        # TODO: expose scratch stages with nested record_function ranges
        return model(x, x, x, mask)

def profile_case(model, x, mask, steps=8):
    schedule = torch.profiler.schedule(wait=1, warmup=2, active=3, repeat=1)
    with profile(activities=[ProfilerActivity.CPU,
                             ProfilerActivity.CUDA],
```

```
                schedule=schedule,
                record_shapes=True,
                profile_memory=True) as prof:
        for _ in range(steps):
            output = instrumented_forward(model, x, mask)
            # TODO: backward for training profile
            prof.step()
    return prof.key_averages().table(sort_by="self_cuda_time_total")

def E16(configurations):
    # TODO: scratch/library/fused comparisons and trace export
    return ...
```

Part 5 - The Philosophy Behind the Exercise

A profile is a map of where the physical machine spent effort. It translates an abstract architecture into kernels, memory transfers, reductions, and synchronization. This translation often reveals that the conceptual center of an algorithm is not the only practical bottleneck.

Optimization should follow evidence because human intuition tends to focus on visible code rather than expensive execution. A one-line matrix multiplication may dominate a page of Python, while many Python statements may disappear into one fused kernel.

The profiler does not decide what matters; it measures cost. The engineer must connect that cost to model quality, maintainability, observability, and the actual deployment objective. Faster computation is valuable when it preserves the function the system was built to perform.

17.3.18 Exercise 17

Exercise Title: Build a Head Importance Metric and Prune Least Useful Heads on a Toy Task Experimentally

Part 1 - Why this matters

If several heads become redundant, removing some may reduce computation or reveal how the model distributes function. Pruning requires an importance criterion, but no single statistic captures usefulness. A head can have small output norm yet carry a decisive feature later amplified by W_O. It can have sharp attention and still be irrelevant to the loss. Reliable pruning must be validated through behavioral change.

Begin with ablation importance: set one head output to zero before the output projection and measure the change in validation loss or task metric. Compare this with cheaper proxies such as output norm, gradient magnitude, Taylor score, attention entropy, and output-projection block norm. Calculate rank correlation between each proxy and actual ablation effect across heads and seeds.

Single-head importance may miss interaction. Two individually unimportant heads can jointly provide a fallback, so pruning both causes a large drop. Conversely, two apparently important heads may encode the same function. Iterative pruning with reevaluation is safer than ranking once and removing a large group. After each pruning step, optionally fine-tune

under a fixed budget and report performance before and after recovery.

Logical masking of heads does not automatically create hardware speedup. Dense projections and kernel shapes may remain unchanged. Distinguish functional pruning, structural removal of dimensions and weights, and measured runtime improvement. A smaller conceptual model is not necessarily a faster compiled model.

Part 2 - Mathematical form and reasoning

Let $m_r \in \{0, 1\}$ gate head r before concatenation. The model becomes

$$Y(m) = \text{Concat}(m_1 Z^{(1)}, \ldots, m_H Z^{(H)}) W_O.$$

Ablation importance is

$$I_r^{\text{abl}} = \mathcal{L}(m_r = 0) - \mathcal{L}(m_r = 1).$$

A first-order Taylor proxy around $m_r = 1$ is

$$I_r^{\text{Taylor}} \approx \left| \frac{\partial \mathcal{L}}{\partial m_r} m_r \right|.$$

If $W_O^{(r)}$ denotes the rows receiving head r, a simple structural proxy is $\|W_O^{(r)}\|_F$, but it should be evaluated against true ablation rather than assumed sufficient.

Part 3 - Hints, controls, and evidence

1. Train and freeze a baseline before measuring importance on a fixed validation set.
2. Calculate ablation effect, norm, entropy, gradient, Taylor, and output- projection metrics for every head.
3. Repeat seeds and report whether importance rankings are stable.
4. Prune iteratively, reevaluate after each removal, and compare one-shot pruning.
5. Separate metric immediately after pruning from metric after a fixed fine-tuning budget.
6. Measure actual parameters, FLOPs estimate, latency, and memory after structural pruning; do not infer speed from zeroed tensors.

Part 4 - Python code skeleton

```
import torch

class HeadGate(torch.nn.Module):
    def __init__(self, num_heads):
        super().__init__()
        self.gates = torch.nn.Parameter(torch.ones(num_heads),
                                        requires_grad=False)

    def forward(self, heads):
```

```
        return heads * self.gates[None, :, None, None]

@torch.no_grad()
def ablation_scores(model, loader, evaluate):
    baseline = evaluate(model, loader)
    scores = []
    for head in range(model.num_heads):
        # TODO: disable one head, evaluate, then restore it
        scores.append(...)
    return baseline, scores

def iterative_prune(model, loader, target_heads, fine_tune=None):
    history = []
    while ...:
        _, importance = ablation_scores(model, loader, ...)
        chosen = ...
        # TODO: prune or gate chosen head, evaluate, optionally fine-tune
        history.append(...)
    return history

def E17(model, train_loader, validation_loader):
    # TODO: proxy correlation, iterative pruning, and speed validation
    return ...
```

Part 5 - The Philosophy Behind the Exercise

Pruning asks which parts of a learned system are necessary. Necessity is relational: a head may be dispensable because another compensates for it, not because it learned nothing. Removing components reveals organization through the system's response to loss.

Importance is also purpose-dependent. A head may be unimportant for average accuracy but essential for rare cases, robustness, calibration, or a subgroup of inputs. Optimization toward one metric can erase capabilities that metric does not value.

The broader lesson is that efficiency is a claim about behavior under reduced resources. Real compression must preserve declared functions and produce measured systems gains, not merely a visually smaller diagram.

17.3.19 Exercise 18

Exercise Title: Create a Multi-Head Attention Debugging Checklist Covering Shapes, Masks, Dtypes, and NaN Detection

Part 1 - Why this matters

Multi-head attention failures often appear far from their cause. A wrong split can surface as poor convergence, a mask-axis error as suspiciously high validation accuracy, dtype overflow as intermittent NaNs, and device mismatch as a crash only when an optional path is enabled.

A debugging checklist turns the chapter's scattered invariants into an ordered diagnostic procedure.

The order matters. Begin with configuration contracts, then input contracts, projection shapes, split/merge identity, score geometry, mask legality, softmax invariants, output merge, backward gradients, numerical parity, and performance. Later checks depend on earlier ones. There is little value in profiling a kernel before proving that it computes the intended tensor relationships.

Each checklist item should be executable where possible. Replace vague advice such as "check the mask" with assertions: forbidden weights are zero, legal rows sum to one, future-token interventions cannot change earlier outputs, and all-masked rows follow the documented policy. Replace "check gradients" with finite checks, expected shapes, nonzero paths, masked-zero paths, and a tiny finite-difference case.

The checklist must support incident documentation. For each failure, capture configuration, seed, device, dtype, shape ledger, earliest violated invariant, suspected cause, correction, and regression test. A fix without a retained test permits the same bug to return during later refactoring.

Part 2 - Mathematical form and reasoning

The checklist enforces a chain of invariants:

$$d = Hd_h,$$

$$(B, T, d) \leftrightarrow (B, H, T, d_h),$$

$$S \in \mathbb{R}^{B\times H\times T_q\times T_k}, \qquad A = \operatorname{softmax}(S + M),$$

$$\sum_j A_{b,r,i,j} = 1, qquad A_{b,r,i,j} = 0 \text{ for forbidden } j,$$

$$Y \in \mathbb{R}^{B\times T_q\times d}.$$

For every monitored tensor Z, require

$$\text{finite}(Z), qquad \text{shape}(Z) = \text{expected}(Z),$$

and compare devices or implementations under a declared tolerance rather than bitwise equality by default.

Part 3 - Hints, controls, and evidence

1. Divide the checklist into configuration, forward geometry, masks, numerics, backward, parity, performance, and incident-report sections.
2. Give every check a pass/fail condition, diagnostic payload, and likely root causes.
3. Build a tiny deterministic smoke test and an asymmetric cross-attention test that run before expensive training.

4. Add failure injection for wrong transpose, wrong softmax axis, mask broadcast, detach, overflow, and missing CUDA synchronization.

5. Stop at and report the earliest violated invariant rather than flooding the user with downstream failures.

6. Run the checklist automatically in CI or before chapter benchmarks and save a machine-readable report.

Part 4 - Python code skeleton

```
from dataclasses import dataclass, asdict
import torch

@dataclass
class CheckResult:
    name: str
    passed: bool
    evidence: dict
    likely_causes: list

def check_split_merge(model, x):
    split = model.split_heads(x)
    merged = model.merge_heads(split)
    return CheckResult("split_merge_identity",
                       torch.equal(x, merged),
                       {"input": tuple(x.shape),
                        "split": tuple(split.shape)},
                       ["wrong transpose", "wrong reshape order"])

def check_attention(weights, forbidden):
    # TODO: finite, row-sum, range, and forbidden-zero checks
    return ...

def run_debug_checklist(model, case):
    checks = []
    # TODO: ordered checks; stop or mark downstream checks as blocked
    return {"passed": all(c.passed for c in checks),
            "checks": [asdict(c) for c in checks]}

def E18():
    correct_case = ...
    injected_failures = ...
    # TODO: prove each failure is caught by its earliest invariant
    return ...
```

Part 5 - The Philosophy Behind the Exercise

A checklist is accumulated memory. It converts failures once paid for with time into protections available to future work. In this sense, engineering maturity is partly the ability of a project to remember its mistakes.

Order gives the checklist explanatory power. A system can violate many downstream properties after one early error, but the earliest broken invariant is closest to the cause. Debugging is the search for that first divergence between intention and execution.

The checklist does not replace thought. It protects attention so thought can be spent on new problems rather than rediscovering old ones. Its quality is measured by whether it produces evidence, shortens diagnosis, and prevents recurrence.

17.3.20 Exercise 19

Exercise Title: Produce a Multi-Head Attention Engineering Report Summarizing Implementations and Observed Behaviors

Part 1 - Why this matters

The final exercise transforms nineteen technical investigations into one auditable engineering account. The report must explain how model-space vectors are projected into head subspaces, how shapes change, how per-head scores and weighted retrieval operate, how heads are concatenated, and how the output projection integrates them back into the residual space. It should be possible for another researcher to reconstruct the implementation from the tensor contracts and equations.

Implementation is only the first layer. The report must summarize evidence about head diversity, single-head comparisons, scaling, mask composition, redundancy, low-rank score geometry, mixed precision, gradient correctness, head-count performance, long-sequence memory, broadcast efficiency, profiler hotspots, importance metrics, and pruning. Each claim should point to a saved configuration, seed set, table, trace, failure case, or regression test.

Negative and conditional results belong in the report. Per-head temperature may not improve quality. More heads may not beat a controlled single-head baseline. Similar heat maps may not imply functional redundancy. Pruning may reduce conceptual capacity without improving latency. Fused attention may save memory while reducing access to explicit weights. These findings refine design choices and should not be hidden as unsuccessful experiments.

Separate theory, measurement, and interpretation. The rank bound applies to pre-softmax logits, not automatically to softmax weights. Theoretical FLOPs at fixed model width may remain similar across head counts while measured runtime changes. A CUDA parity tolerance establishes bounded numerical agreement, not bitwise identity. Precise boundaries protect the report from overclaiming.

End with a practical design guide. State how to choose a valid head count, when to inspect or prune heads, how to test masks and mixed precision, what to profile, and which unresolved questions pass into positional representation and complete Transformer blocks.

Part 2 - Mathematical form and reasoning

The report should anchor itself in

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V,$$

$$A^{(r)} = \mathrm{softmax}\left(\frac{Q^{(r)}K^{(r)\mathsf{T}}}{\sqrt{d_h}} + M\right), \qquad Z^{(r)} = A^{(r)}V^{(r)},$$

$$Y = \mathrm{Concat}(Z^{(1)}, \ldots, Z^{(H)})W_O.$$

It must state the geometry

$$d = Hd_h, qquad \mathrm{rank}(Q^{(r)}K^{(r)\mathsf{T}}) \le d_h,$$

the dense cost

$$\text{work} = \mathcal{O}(BTd^2 + BT^2d), \qquad \text{map storage} = \mathcal{O}(BHT^2),$$

and the exact mask and numerical invariants used to validate the implementation.

Part 3 - Hints, controls, and evidence

1. Open with a thesis and a table mapping every engineering claim to its exercise artifact and verification method.
2. Include tensor contracts, parameter counts, controlled comparison settings, hardware, software, dtype, seeds, tolerances, and stopping rules.
3. Report distributions and failure boundaries, not only best runs or successful screenshots.
4. Separate descriptive attention patterns from ablation-based functional evidence.
5. Include an incident appendix containing injected bugs, earliest violated invariants, fixes, and regression tests.
6. Finish with recommendations, limitations, and questions for TRF02 rather than repeating the chapter titles.

The final report should be reproducible from saved raw artifacts and should clearly mark any claim that remains a hypothesis.

Part 4 - Python code skeleton

```
from pathlib import Path
import json

REQUIRED_ARTIFACTS = [
    "implementation", "projection_geometry", "head_diversity",
    "mask_proofs", "rank_analysis", "mixed_precision",
    "gradient_check", "head_count_benchmark", "memory_stress",
    "profiler", "pruning", "debug_checklist",
]
```

```
def load_artifacts(directory):
    # TODO: load JSON, CSV, profiler summaries, and figure metadata
    return ...

def validate_artifacts(artifacts):
    return {name: ... for name in REQUIRED_ARTIFACTS}

def build_conclusions(artifacts):
    # TODO: separate observation, interpretation, limitation,
    # and recommendation for every major result
    return ...

def E19(artifact_directory, output_path):
    artifacts = load_artifacts(artifact_directory)
    checks = validate_artifacts(artifacts)
    assert all(checks.values()), checks
    report = build_conclusions(artifacts)
    Path(output_path).write_text(report, encoding="utf-8")
    return {"report": str(output_path), "evidence_checks": checks}
```

Part 5 - The Philosophy Behind the Exercise

An engineering report is where computation becomes accountable knowledge. Code records what was executed, but a report records why it was executed, what was controlled, what was observed, what failed, and what the evidence permits us to conclude.

Multi-head attention is often presented as a finished idea. This chapter shows it as a negotiated system of projections, subspaces, masks, numerical formats, memory costs, kernels, and learned behavior. Its success depends on the coordination of these parts, not on the softmax equation alone.

The deepest lesson is methodological. Understanding a model means moving among mathematics, implementation, intervention, and physical measurement without confusing one for another. A trustworthy researcher preserves the boundaries of every claim and uses those boundaries to decide what must be tested next.

17.4 TRF02

Chapter Title: Positional Encoding and Sequence Order Modeling
Specialization: Models and Training

17.4.1 Exercise 00

Exercise Title: Explain Why Self-Attention Alone Is Permutation-Invariant and Why Positional Information Is Required

Part 1 - Why this matters

Self-attention compares token representations according to their content. If the input rows are permuted and no positional signal, causal boundary, or position-dependent mask is present, the outputs are permuted in the same way. The mechanism can model relationships among a set of items, but it cannot know which identical item appeared first, which token was adjacent, or how far one token was from another. This property is more precisely called permutation equivariance for token-level outputs: permuting inputs permutes outputs rather than leaving the entire output unchanged. A permutation-invariant pooled readout may then produce exactly the same final result.

This limitation matters because sequence meaning often depends on order. "Dog bites man" and "Man bites dog" contain the same tokens but describe different events. A model that sees only token identity and pairwise content similarity cannot distinguish these sequences before some source of order is introduced. Recurrence receives order through the sequence of state updates; convolution receives it through structured local neighborhoods. A Transformer must encode it explicitly through positional representations, masks, relative biases, or another position-dependent mechanism.

The proof should not rely on verbal intuition. Construct an input matrix X and a permutation matrix P. Apply scratch self-attention to X and PX using the same weights. Verify that the unpermuted result from the second case matches the first result. Then add positional encodings to fixed slots and repeat. Because tokens now receive different positional vectors after permutation, the equality should break.

Masks require careful interpretation. A causal mask already depends on row and column indices, so it introduces an ordering constraint even without a separate positional vector. It tells the model which positions are earlier, but not a complete metric description of absolute or relative distance. The experiment must therefore separate unmasked attention, padding-only attention, and causal attention instead of making an unrestricted claim about every Transformer.

Part 2 - Mathematical form and reasoning

For permutation matrix P and unmasked self-attention,

$$Q' = PXW_Q = PQ, \qquad K' = PK, \qquad V' = PV.$$

Therefore

$$Q'K'^{\mathsf{T}} = P(QK^{\mathsf{T}})P^{\mathsf{T}}.$$

Row-wise softmax respects the same simultaneous row and column permutation:

$$\operatorname{softmax}(PSP^{\mathsf{T}}) = P \operatorname{softmax}(S)P^{\mathsf{T}}.$$

Thus

$$\operatorname{Attn}(PX) = P \operatorname{Attn}(X).$$

With slot-dependent positional matrix E,

$$\operatorname{Attn}(PX + E) \neq P \operatorname{Attn}(X + E)$$

in general, because E remains attached to positions rather than moving with the permuted token identities.

Part 3 - Hints, controls, and evidence

1. Use unique token vectors first, then repeat the proof with duplicated tokens and an order-sensitive target.
2. Build the permutation explicitly and verify both matrix multiplication and index-based permutation paths.
3. Disable dropout and use evaluation mode so stochasticity cannot break the equality for unrelated reasons.
4. Test unmasked, padding-only, and causal cases separately and state which symmetry each mask preserves or breaks.
5. Compare token-level outputs after applying the inverse permutation and a pooled output before and after permutation.
6. Add positional information and quantify the change in attention weights, outputs, and an order-sensitive prediction.

Part 4 - Python code skeleton

```
import torch

def permutation_matrix(order, dtype=torch.float64):
    size = len(order)
    p = torch.zeros(size, size, dtype=dtype)
    p[torch.arange(size), torch.tensor(order)] = 1
    return p

def equivariance_error(attention, x, order, mask=None):
    p = permutation_matrix(order, x.dtype).to(x.device)
    px = torch.einsum("ij,bjd->bid", p, x)
    y, _ = attention(x, x, x, mask)
    py, _ = attention(px, px, px, mask)
    expected = torch.einsum("ij,bjd->bid", p, y)
    return (py - expected).abs().max()

def E00(attention):
    x = ...
    position = ...
    plain_error = equivariance_error(attention, x, [2, 0, 3, 1])
    # TODO: repeat with fixed-slot positions and causal masking
    return {"plain": plain_error, "with_position": ...,
            "causal": ...}
```

Part 5 - The Philosophy Behind the Exercise

Attention begins by treating tokens as members of a relational collection. It asks what is compatible with what, but content alone does not reveal the path by which the collection was formed. Order is additional structure imposed on that collection.

This exposes a general principle: a model can learn only from distinctions its representation and architecture make available. If two inputs are mapped into states related by a symmetry the model cannot break, no amount of optimization can recover the missing distinction. Data cannot teach a coordinate the system has erased.

Positional modeling is therefore not decoration added to attention. It is the construction of temporal or spatial identity. The design question is not whether order matters in the abstract, but which aspects of order—absolute location, relative distance, direction, or hierarchy—the task requires the model to represent.

17.4.2 Exercise 01

Exercise Title: Implement Sinusoidal Positional Encoding from Scratch and Verify Correct Shape and Broadcasting

Part 1 - Why this matters

Sinusoidal positional encoding assigns each sequence location a deterministic vector whose dimensions oscillate at different frequencies. The encoding adds no learned parameters and can be generated for positions beyond those seen in training. Its implementation appears simple, but incorrect pairing of sine and cosine dimensions, wrong exponent calculation, or confusion between batch-first and sequence-first layouts can silently attach the wrong position to every token.

The encoding should be constructed first as a table of shape `(max_length, d_model)`. For a batch-first input it is sliced to the current sequence length, given a singleton batch axis, and added by broadcasting to every example. The batch axis is shared because position 7 has the same encoding in every independent sequence, unless the application explicitly uses different position offsets. Broadcasting must not duplicate a large table unnecessarily.

Odd model dimensions require a policy because sine dimensions and cosine dimensions are naturally paired. A robust implementation fills every even dimension with sine and only the available odd dimensions with cosine. The exercise should test both even and odd widths and compare selected coordinates with manual calculations.

Register the table as a buffer rather than a trainable parameter. This ensures it moves with the module across CPU and CUDA and appears in the state dictionary without receiving gradients. The implementation should also support a position offset so cached autoregressive decoding can encode newly generated tokens using their true global indices.

Part 2 - Mathematical form and reasoning

For position p and pair index i,

$$\mathrm{PE}_{p,2i} = \sin\!\left(\frac{p}{10000^{2i/d}}\right), \qquad \mathrm{PE}_{p,2i+1} = \cos\!\left(\frac{p}{10000^{2i/d}}\right).$$

Equivalently, define

$$\omega_i = \exp\Big(-\frac{2i}{d}\log 10000\Big)$$

and use $\sin(p\omega_i)$ and $\cos(p\omega_i)$. For batch-first input,

$$X \in \mathbb{R}^{B\times T\times d}, \qquad E \in \mathbb{R}^{1\times T\times d}, \qquad Y = X + E.$$

The singleton batch dimension gives broadcasted addition without changing the underlying positional table.

Part 3 - Hints, controls, and evidence

1. Calculate positions 0, 1, and 2 manually for a tiny width before using a vectorized implementation.
2. Test even and odd d_{model}, multiple batch sizes, sequence length one, and the maximum supported length.
3. Verify position zero contains zeros in sine channels and ones in cosine channels.
4. Confirm the buffer has no gradient, follows `model.to(device)`, and preserves the requested dtype.
5. Check that equal token embeddings at different positions become different while equal positions across batches receive equal encodings.
6. Test offset slicing and reject requests that exceed the allocated table with a clear error.

Part 4 - Python code skeleton

```
import math
import torch
from torch import nn

class SinusoidalPosition(nn.Module):
    def __init__(self, d_model, max_length=4096):
        super().__init__()
        position = torch.arange(max_length, dtype=torch.float32)[:, None]
        pair = torch.arange(0, d_model, 2, dtype=torch.float32)
        frequency = ...
        table = torch.zeros(max_length, d_model)
        table[:, 0::2] = ...
        table[:, 1::2] = ...  # TODO: handle odd width safely
        self.register_buffer("table", table, persistent=True)

    def forward(self, x, offset=0):
        # x: (B,T,d_model)
        length = x.shape[1]
        encoding = ...
```

```
        return x + encoding.to(dtype=x.dtype)

def E01():
    module = SinusoidalPosition(d_model=7, max_length=16)
    x = torch.zeros(3, 5, 7)
    y = module(x)
    # TODO: manual-value, broadcasting, buffer, offset, and boundary tests
    return y
```

Part 5 - The Philosophy Behind the Exercise

Sinusoidal encoding represents position through phase. A location is not stored as a single integer but as a pattern across many cycles, each observing the sequence at a different scale. Fine frequencies distinguish nearby positions; slow frequencies preserve broad location.

The encoding also demonstrates how a fixed mathematical structure can support learning without itself being learned. Not every useful representation must be discovered from data. Some structure can be supplied as an inductive bias that reduces the burden on optimization.

Yet determinism is not neutrality. Choosing sinusoidal coordinates decides what relations are easy to express and how positions behave beyond the training range. A handcrafted encoding still carries assumptions about the world it is designed to describe.

17.4.3 Exercise 02

Exercise Title: Implement Learnable Positional Embeddings and Compare Against Sinusoidal Encoding on a Toy Task

Part 1 - Why this matters

Learnable positional embeddings assign an independent trainable vector to each supported location. Unlike sinusoidal encoding, they can adapt their geometry to the training task rather than following a fixed frequency construction. This flexibility may improve performance inside the trained range, but it also introduces parameters tied to a maximum length and provides no defined vector for unseen positions unless an extension rule is added.

The implementation is an embedding table indexed by position IDs. Position IDs must be created on the same device as the input and may need an offset during incremental decoding. The table should be initialized according to a recorded policy, included in the optimizer, and checked for gradient flow. A common bug is recreating the position tensor on CPU or detaching the embedding output before addition.

The comparison with sinusoidal encoding must control the rest of the model. Use identical token embeddings, attention architecture, dataset, seeds, optimizer budget, stopping rule, and evaluation splits. The learnable variant has extra parameters, so report that difference and include a no-position baseline. Evaluate both interpolation within trained lengths and extrapolation beyond them; for the learnable table, the latter requires a declared strategy such as failure, interpolation, or a larger randomly initialized table.

Use an order-sensitive task with known structure. Record training speed, validation quality, seed variance, gradient norms for positional parameters, and performance by sequence length. A result may legitimately show learnable positions winning inside the training range while sinusoidal positions fail less severely outside it.

Part 2 - Mathematical form and reasoning

For maximum length L,

$$E \in \mathbb{R}^{L \times d}, \qquad Y_{b,p,:} = X_{b,p,:} + E_{p,:}.$$

The positional parameter count is

$$N_{\text{learned}} = Ld, \qquad N_{\text{sinusoidal}} = 0$$

trainable parameters. For trial metric $m_{v,s,\ell}$ under variant v, seed s, and sequence length ℓ, summarize both

$$\overline{m}_{v,\ell} = \frac{1}{S} \sum_{s} m_{v,s,\ell}$$

and its dispersion. Aggregating all lengths into one number can hide a sharp failure beyond the training range.

Part 3 - Hints, controls, and evidence

1. Compare no position, sinusoidal position, and learnable position under the same surrounding architecture.
2. Fix dataset generation, seeds, initialization of shared weights, optimization steps, validation frequency, and checkpoint rule.
3. Report extra positional parameters and optimizer-state memory.
4. Measure quality separately for seen lengths, unseen token combinations, and lengths beyond the training maximum.
5. Inspect positional gradient norms and confirm every used index receives updates while unused rows do not.
6. State the learnable-table extrapolation policy explicitly; never silently clamp all unseen positions to the final row.

Part 4 - Python code skeleton

```
import torch
from torch import nn

class LearnablePosition(nn.Module):
    def __init__(self, max_length, d_model):
        super().__init__()
        self.embedding = nn.Embedding(max_length, d_model)
```

```
    def forward(self, x, offset=0):
        length = x.shape[1]
        positions = torch.arange(offset, offset + length,
                                 device=x.device)
        # TODO: validate the range
        return x + self.embedding(positions)[None, :, :]

def controlled_trial(variant, seed, data, config):
    torch.manual_seed(seed)
    model = ...
    # TODO: identical training and evaluation protocol
    return {"variant": variant, "seed": seed,
            "curve": ..., "metrics_by_length": ...,
            "position_gradient_norms": ...}

def E02(variants, seeds=range(5)):
    data = ...
    return [controlled_trial(v, s, data, ...)
            for v in variants for s in seeds]
```

Part 5 - The Philosophy Behind the Exercise

Learnable positions allow a task to shape its own coordinate system. The model does not inherit a theory of distance; it discovers positional distinctions that reduce its loss. This can be efficient when the future resembles the training domain.

But adaptation and extrapolation pull in different directions. A free table can specialize strongly to known positions while lacking any principle for a new one. A fixed rule may fit the observed domain less perfectly while defining behavior everywhere its formula can be evaluated.

The comparison therefore reflects a recurring problem in learning: whether to encode prior structure or purchase flexibility with data. Neither answer is universally correct. The responsible choice follows the task's expected range, evidence, and failure costs.

17.4.4 Exercise 03

Exercise Title: Debug Positional Encoding Off-by-One Bugs in Token Indexing and Confirm Correct Sequence Alignment

Part 1 - Why this matters

An off-by-one error attaches a valid positional vector to the wrong token. All shapes remain correct, no exception is required, and training may still reduce loss. The model simply learns from a shifted coordinate system that can become inconsistent across training, evaluation, padding, special tokens, or cached generation. Because the error is semantic rather than dimensional, it must be debugged with traceable alignment tests.

Decide whether the first token uses position 0 or position 1 and apply that policy everywhere. Special tokens such as beginning-of-sequence may consume a position; padding may use an arbitrary ID whose embedding is later masked; left padding changes the relation between

storage index and content-relative position. Incremental decoding must begin the new token at the length already stored in the cache, not restart at zero on every call.

Build a ledger containing token ID, storage index, valid-token index, position ID, padding flag, and expected positional vector. Use deliberately different tokens and deterministic position values so a shift is visible. Test full sequence processing against token-by-token processing with an offset. Their representations should agree before any stochastic layer.

Inject specific failures: start at one in one path and zero in another, count padding as content under left padding, forget the beginning token, or use `offset=cache_length-1`. Each should be caught by the earliest alignment invariant and retained as a regression test.

Part 2 - Mathematical form and reasoning

For zero-based absolute positions and prefix cache length c, a block of length T should use

$$p_t = c + t, \qquad t = 0, \ldots, T-1.$$

For right-padded sequences with valid length ℓ_b, token positions may be

$$p_{b,t} = t \quad \text{for } t < \ell_b,$$

while padding is ignored by attention and loss. For content-relative positions under left padding mask $m_{b,t} \in \{0, 1\}$,

$$p_{b,t} = \max\left(0, \sum_{u=0}^{t} m_{b,u} - 1\right).$$

The correct rule depends on architecture and serving policy, but the same rule must be used consistently across all paths.

Part 3 - Hints, controls, and evidence

1. Replace the real table temporarily with rows containing their own position index so alignment is directly observable.
2. Print a row-wise token/position ledger for unpadded, right-padded, left-padded, prefixed, and incrementally decoded examples.
3. Compare full-sequence and cached block processing before attention and at the final output in evaluation mode.
4. Test beginning, end, padding, maximum table length, and empty-continuation boundaries.
5. Inject each off-by-one policy and ensure a dedicated assertion fails with the mismatched token and expected position.
6. Preserve the agreed indexing convention in module documentation and checkpoint metadata.

Part 4 - Python code skeleton

```
import torch

def content_positions(attention_mask):
    # attention_mask: 1 for content, 0 for padding
    positions = attention_mask.long().cumsum(dim=-1) - 1
    return positions.clamp_min(0)

def alignment_ledger(tokens, position_ids, attention_mask):
    rows = []
    for b in range(tokens.shape[0]):
        for t in range(tokens.shape[1]):
            rows.append({"batch": b, "storage_index": t,
                         "token": int(tokens[b, t]),
                         "position": int(position_ids[b, t]),
                         "valid": bool(attention_mask[b, t])})
    return rows

def E03(position_module):
    tokens, masks = ...
    position_ids = content_positions(masks)
    # TODO: full vs incremental parity and injected off-by-one tests
    return alignment_ledger(tokens, position_ids, masks)
```

Part 5 - The Philosophy Behind the Exercise

An off-by-one error is small in arithmetic and large in meaning. It reminds us that coordinates do not merely label data; they establish relationships among events. Moving every event by one position can preserve shapes while changing the system's history.

Alignment is a contract among tokenization, batching, modeling, and serving. No component can define position privately without risking contradiction with the others. Correctness emerges from shared convention.

The deeper debugging lesson is that valid values can still be wrongly associated. Range checks and finite checks cannot detect misplaced meaning. Some truths require identity-level traces that follow one item through the entire pipeline.

17.4.5 Exercise 04

Exercise Title: Demonstrate How Positional Encoding Changes Attention Maps Compared to No Position Information

Part 1 - Why this matters

Positional information changes queries and keys before their compatibility is calculated. Two equal token embeddings at different locations are identical to content-only attention, but become different after absolute positional vectors are added or after a relative mechanism modifies their score. Attention maps therefore offer a direct place to observe how order enters relational computation.

The comparison must use identical token content and projection weights. Run the same sequence with no positional signal, then with a chosen encoding. Include repeated tokens so content-only attention is forced to treat some locations symmetrically. Measure how score matrices, probabilities, and outputs change. Heat maps are useful, but numeric symmetry checks and controlled interventions are stronger evidence.

Avoid overinterpretation. A visible diagonal does not prove the model learned syntax, and a changed map does not guarantee improved task performance. Untrained projections may produce arbitrary patterns. Compare initialization and trained checkpoints, repeat seeds, and connect visual differences to an order-sensitive evaluation.

Position and content interact. Adding a large position vector can dominate token identity, while a tiny vector may have little effect. Sweep an encoding scale and record attention entropy, symmetry error, output difference, and task quality. The goal is to understand influence rather than to choose the most dramatic image.

Part 2 - Mathematical form and reasoning

Without position,

$$S_0 = \frac{(XW_Q)(XW_K)^\mathsf{T}}{\sqrt{d_h}}.$$

With additive position E and scale α,

$$S_\alpha = \frac{((X + \alpha E)W_Q)((X + \alpha E)W_K)^\mathsf{T}}{\sqrt{d_h}}.$$

Expanding the product reveals four interactions: content with content, content with position, position with content, and position with position. Compare maps with

$$\Delta_A = \frac{\|A_\alpha - A_0\|_F}{\max(\|A_0\|_F, \varepsilon)}$$

and measure how positional information breaks the symmetry between rows that were equal in the content-only case.

Part 3 - Hints, controls, and evidence

1. Create a sequence containing repeated token embeddings at several positions and prove their content-only score rows match where expected.
2. Freeze projections and change only the positional variant or scale.
3. Save logits, weights, entropy, row differences, outputs, and task metric instead of relying only on images.
4. Compare untrained and trained models across fixed probe examples and multiple seeds.
5. Use identical color limits when placing heat maps side by side.
6. Perturb token identity and position separately to isolate content and order sensitivity.

Part 4 - Python code skeleton

```
import torch

@torch.no_grad()
def positional_intervention(model, token_embeddings, position, scales):
    results = {}
    for scale in scales:
        x = token_embeddings + scale * position
        output, weights, debug = model.attend_with_debug(x)
        results[scale] = {
            "logits": debug["logits"].cpu(),
            "weights": weights.cpu(),
            "output": output.cpu(),
            "entropy": ...,
        }
    return results

def map_difference(reference, candidate):
    return ...

def E04(model):
    tokens = ...  # include repeated embeddings
    positions = ...
    results = positional_intervention(model, tokens, positions,
                                      [0.0, 0.1, 0.5, 1.0, 2.0])
    # TODO: symmetry tests, plots, and order-task evaluation
    return results
```

Part 5 - The Philosophy Behind the Exercise

Position does not enter attention as a label printed beside a token. It alters the geometry through which relevance is calculated. The question "what is related?" becomes inseparable from "where did it occur?"

This interaction reflects a broader truth about meaning: content and context are not independent layers that can always be analyzed separately. The same symbol can participate in different relations because its place changes the questions it asks and the keys it presents.

Visualization makes this transformation visible, but visibility is not explanation. A heat map becomes evidence only when paired with controls, quantitative differences, interventions, and behavior on a task whose required order is known.

17.4.6 Exercise 05

Exercise Title: Implement Relative Position Bias Conceptually and Compare Its Behavior Against Absolute Position Encoding

Part 1 - Why this matters

Absolute positional encoding attaches a representation to each location before attention. Relative position bias instead modifies compatibility according to the displacement between

query and key. A token at position 20 attending three steps backward receives the same relative category as a token at position 200 attending three steps backward, subject to clipping or bucketing. This can make translation of a local pattern easier to represent.

Implement a table indexed by relative displacement $j - i$. For maximum signed distance D, clip displacements to $[-D, D]$ and shift them into nonnegative indices. The table may contain one bias per head because different heads can prefer different distance patterns. The bias is added to logits before softmax, so it changes retrieval probability without changing values directly.

Compare relative bias with absolute additive encoding on tasks where the same relationship appears at different absolute locations. Use identical attention weights and training controls. Test shift equivariance by moving a pattern through the sequence: the relative-bias score contribution should move with the pattern, while an absolute table may encode location-specific behavior.

Relative bias is not automatically length-unlimited. A learned table with clipping collapses all large distances into edge buckets, and a full $2T - 1$ table grows with supported length. Document the indexing and extrapolation policy. Also clarify that relative bias expresses pairwise distance but may not uniquely identify an absolute slot when the task requires one.

Part 2 - Mathematical form and reasoning

For query position i, key position j, and head r,

$$S_{i,j}^{(r)} = \frac{q_i^{(r)\mathsf{T}} k_j^{(r)}}{\sqrt{d_h}} + b_{\rho(j-i)}^{(r)},$$

where

$$\rho(\delta) = \operatorname{clip}(\delta, -D, D) + D.$$

The index range is $0, \ldots, 2D$, so a per-head table has shape $H \times (2D + 1)$. Under a joint shift c,

$$(j + c) - (i + c) = j - i,$$

which preserves the relative-bias contribution as long as boundaries and masks do not change.

Part 3 - Hints, controls, and evidence

1. Construct the full displacement matrix manually for a short sequence and verify orientation: positive should mean the declared direction.

2. Test clipping at $-D$, D, and beyond both boundaries.

3. Give each bucket a recognizable value and verify the gathered bias matrix before adding content logits.

4. Compare shifted copies of the same pattern away from sequence boundaries.

5. Evaluate absolute, relative, combined, and no-position variants under matched training controls.

6. Report behavior on tasks requiring relative distance separately from tasks requiring absolute location.

Part 4 - Python code skeleton

```
import torch
from torch import nn

class RelativePositionBias(nn.Module):
    def __init__(self, num_heads, max_distance):
        super().__init__()
        self.max_distance = max_distance
        self.table = nn.Parameter(
            torch.zeros(num_heads, 2 * max_distance + 1)
        )

    def forward(self, query_len, key_len, device):
        q = torch.arange(query_len, device=device)[:, None]
        k = torch.arange(key_len, device=device)[None, :]
        displacement = ...  # declare sign convention
        bucket = displacement.clamp(-self.max_distance,
                                    self.max_distance) + self.max_distance
        # TODO: gather -> (1,H,Tq,Tk)
        return ...

def E05():
    module = RelativePositionBias(num_heads=3, max_distance=4)
    # TODO: indexing, clipping, shift, gradient, and task comparisons
    return module(5, 7, torch.device("cpu"))
```

Part 5 - The Philosophy Behind the Exercise

Absolute position asks where an event exists in a global frame. Relative position asks how one event is situated with respect to another. These are different forms of knowledge: location and relation.

Many patterns survive translation because their internal distances remain the same. Relative bias makes that invariance easier to express, but it can also discard global identity that another task needs. Every invariance is both a generalization aid and a refusal to distinguish.

The exercise therefore turns positional design into a question about the world represented by the data. Does meaning belong to a slot, a distance, a direction, or a combination? The architecture should answer the task's actual geometry rather than follow fashion.

17.4.7 Exercise 06

Exercise Title: Implement Rotary Positional Embedding Conceptually and Explain Why It Helps Long Context Attention

Part 1 - Why this matters

Rotary positional embedding applies position-dependent rotations to query and key vector pairs rather than adding a vector to token representations. When a rotated query at position m is dotted with a rotated key at position n, the resulting compatibility depends on their relative displacement. Position is therefore embedded directly into the geometry of the attention score.

The implementation groups feature dimensions into two-dimensional pairs. Each pair is rotated by an angle determined by position and a frequency assigned to that pair. Rotation preserves the norm of each pair in exact arithmetic. Values are usually not rotated in standard RoPE because the mechanism is designed to modify addressing through Q/K compatibility, though variants exist and the chosen convention must be documented.

RoPE is often said to help long context, but this statement needs boundaries. Its relative phase structure can behave more naturally than a finite learned absolute table and can be evaluated at unseen indices. Yet high-frequency angles continue wrapping, finite precision degrades large-position phase calculation, and models trained on short contexts may not automatically use very long contexts correctly. Scaling methods modify the frequency schedule to extend usable range, but they introduce their own tradeoffs.

Prove the basic geometry on a tiny vector. Check norm preservation, position zero identity, inverse rotation, and the relative-dot-product property. Then compare exact float64 calculations with float32 and float16 at increasing positions before drawing conclusions about practical extrapolation.

Part 2 - Mathematical form and reasoning

For pair (x_{2i}, x_{2i+1}) and angle $\theta_{p,i} = p\omega_i$,

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}, \qquad \widetilde{x}_{p,i} = R(\theta_{p,i})x_i.$$

Because rotations are orthogonal,

$$\|R(\theta)x\|_2 = \|x\|_2.$$

For a query at m and key at n,

$$(R(\theta_m)q)^{\mathsf{T}}(R(\theta_n)k) = q^{\mathsf{T}}R(\theta_n - \theta_m)k.$$

The dot product depends on relative phase $n - m$. A common frequency is

$$\omega_i = 10000^{-2i/d_{\text{rotary}}}.$$

Part 3 - Hints, controls, and evidence

1. Implement rotation on a two-dimensional pair manually before vectorizing across batches, heads, positions, and feature pairs.

2. Verify position zero, norm preservation, inverse rotation, and relative dot-product identity in float64.

3. Decide whether adjacent halves or interleaved pairs are used and retain the same convention when comparing with a library.

4. Test partial rotary dimensions and reject odd rotary widths.

5. Compare cached single-token rotation with full-sequence rotation using the same global offset.

6. Sweep large positions and dtypes, measuring phase error, dot-product error, and downstream retrieval quality.

Part 4 - Python code skeleton

```
import torch

def rotate_pairs(x):
    # Interleaved convention: (x0,x1),(x2,x3),...
    even = x[..., 0::2]
    odd = x[..., 1::2]
    return torch.stack((-odd, even), dim=-1).flatten(-2)

def rope_cache(seq_len, rotary_dim, base, device, dtype):
    positions = torch.arange(seq_len, device=device, dtype=torch.float32)
    inv_freq = ...
    angles = positions[:, None] * inv_freq[None, :]
    # TODO: expand pair frequencies to rotary_dim
    return ..., ...

def apply_rope(q, k, cos, sin, offset=0, rotary_dim=None):
    # q,k: (B,H,T,d_head)
    # TODO: rotate selected prefix and preserve remaining dimensions
    return ..., ...

def E06():
    q, k = ...
    cos, sin = rope_cache(...)
    rotated_q, rotated_k = apply_rope(q, k, cos, sin)
    # TODO: geometry, offset, dtype, and long-position tests
    return rotated_q, rotated_k
```

Part 5 - The Philosophy Behind the Exercise

RoPE represents position as transformation rather than annotation. A token is not merely given a positional label; its query and key are reoriented according to where the token exists. Relative order appears through the difference between orientations.

Rotation also preserves magnitude while changing direction. This separates one aspect of content strength from positional phase and illustrates a broader design idea: structure can be encoded through symmetries that preserve selected quantities.

Long-context claims reveal the difference between a formula being defined and a learned system remaining competent. RoPE can calculate a rotation at a new position, but the model's training, precision, and frequency schedule determine whether that rotation continues to support useful reasoning.

17.4.8 Exercise 07

Exercise Title: Build an Order-Sensitive Toy Task and Prove Model Fails Without Positional Encoding

Part 1 - Why this matters

A theoretical symmetry argument becomes stronger when it predicts an empirical failure. This exercise constructs a dataset in which examples share the same multiset of tokens but have different labels solely because of order. A model whose architecture is permutation-equivariant and whose readout is permutation- invariant receives no information capable of separating the paired examples. Its failure is structural, not the result of insufficient epochs.

One suitable task asks whether token A occurs before token B. For every sequence, include a paired permutation containing exactly the same tokens and opposite label. Avoid causal masks and position-dependent padding patterns in the no- position baseline because they would leak order through the architecture. Pool token outputs symmetrically, for example by mean pooling, so a specific readout slot does not act as a hidden positional signal.

Train three variants: no position, a correct positional mechanism, and a deliberately misaligned positional mechanism. Use identical data, shared-weight initialization, optimizer budget, and evaluation. The no-position model should remain near the ambiguity bound, while a correctly encoded model should solve the task. The misaligned model tests whether merely adding varying vectors is sufficient or whether token-position alignment is essential.

The proof should include paired-output equality before training. For two permuted sequences, inverse-permute token-level outputs and compare them, then compare the pooled logits. During training, record gradients and losses to show that the optimizer is active even though the representation lacks the required distinction.

Part 2 - Mathematical form and reasoning

Let x' be a permutation of x with opposite label, and let token encoder F satisfy

$$F(PX) = PF(X).$$

For mean pooling g,

$$g(PZ) = \frac{1}{T}\mathbf{1}^{\mathsf{T}}PZ = \frac{1}{T}\mathbf{1}^{\mathsf{T}}Z = g(Z).$$

Thus classifier c gives

$$c(g(F(PX))) = c(g(F(X))),$$

so it cannot assign different predictions to both members of every paired example. In a balanced binary dataset of perfect contradictory pairs, no deterministic classifier over this representation can exceed 50 percent accuracy on both members simultaneously.

Part 3 - Hints, controls, and evidence

1. Generate exact paired permutations and verify token histograms match while labels differ.
2. Remove accidental order channels: causal masks, length differences, position-dependent padding, slot-specific readout, or deterministic label sorting.
3. Prove pooled-logit equality at initialization for paired examples within numerical tolerance.
4. Train no-position, correct-position, and shifted-position variants with the same seeds and budgets.
5. Report pair consistency, accuracy, loss, convergence curves, and the fraction of contradictory pairs predicted identically.
6. Add a bag-of-tokens baseline to confirm the task contains no content-only shortcut.

Part 4 - Python code skeleton

```
import torch

def make_before_after_pairs(batch, seq_len, vocab_size):
    # TODO: same token multiset, opposite A-before-B label
    original = ...
    permuted = ...
    labels = ...
    paired_labels = 1 - labels
    return torch.cat([original, permuted]), torch.cat([labels, paired_labels])

def paired_logit_error(model, x, permutation):
    logits_a = model(x)
    logits_b = model(x[:, permutation])
    return (logits_a - logits_b).abs().max()

def E07(variants, seeds=range(5)):
    data = ...
    # TODO: controlled training, shortcut audit, and paired metrics
    return ...
```

Part 5 - The Philosophy Behind the Exercise

Failure can be more informative than success when it follows from a precise symmetry. The model does not fail because it is unintelligent or poorly tuned; it fails because its representation

makes two different realities identical.

This is a boundary on learning itself. Optimization cannot recover information that the architecture does not expose. More data containing the same collapsed distinction only repeats the contradiction.

The exercise teaches that inductive bias is not merely a preference for one solution. It determines which questions can be answered at all. Positional encoding expands the model's ontology by allowing "before" and "after" to become representable differences.

17.4.9 Exercise 08

Exercise Title: Evaluate Positional Encoding Choices Under Sequence Length Extrapolation Beyond Training Length

Part 1 - Why this matters

Sequence models are often trained on a limited context range and later asked to process longer inputs. Positional methods differ sharply in what they do beyond that range. A learned absolute table may have no row for a new position. Sinusoidal and rotary formulas can be evaluated, but their phase patterns may enter regimes unseen by training. Relative biases may clip distant positions into the same bucket. Being numerically defined is not the same as preserving task competence.

Construct a task whose rule is independent of sequence length, such as copying a marked token, comparing tokens at a fixed relative offset, or detecting an ordered pair. Train only on lengths within a declared interval. Evaluate every positional method on a grid extending well beyond the training maximum while holding the learned model weights fixed. Report quality by length rather than one aggregate score.

Separate interpolation, mild extrapolation, and extreme extrapolation. Sinusoidal encoding may continue smoothly but create unseen combinations of phases. RoPE may experience phase wrapping and attention-logit changes. Learnable embeddings require a policy: reject, resize with interpolation, randomly initialize new rows, or preallocate a larger table whose unused rows remain untrained. Each policy answers a different question and must be labeled.

Also control computational difficulty. Longer sequences increase distractors and attention cost even with perfect positions. Include an oracle positional or rule-based baseline where possible, and compare performance at trained lengths after padding to larger tensors. This helps distinguish a positional failure from generic long-sequence difficulty.

Part 2 - Mathematical form and reasoning

Let training lengths satisfy $T \leq L_{\text{train}}$ and define

$$m_v(T) = \text{held-out metric for positional variant } v.$$

Normalize against in-range performance:

$$R_v(T) = \frac{m_v(T)}{m_v(L_{\text{train}})}$$

when the metric permits this ratio. A degradation threshold may define an empirical usable context

$$L_v^* = \max\{T : R_v(T) \geq \tau\}.$$

For a learned table $E \in \mathbb{R}^{L \times d}$, positions $p \geq L$ are undefined without an extension transformation. For relative clipping, $|j - i| > D$ maps to the same boundary bucket, removing distinctions among large distances.

Part 3 - Hints, controls, and evidence

1. Train only on a clearly recorded length distribution and prevent long examples from entering validation or augmentation.
2. Evaluate exact length bins from below the training maximum to several multiples above it.
3. Keep task rule, token distribution, model checkpoint, and decoding procedure fixed across positional variants.
4. Label every extension policy for learned tables and relative buckets.
5. Record accuracy or loss, attention entropy, distance error, latency, and memory by length.
6. Include controls for increased distractor count and generic quadratic cost so positional degradation is not inferred from runtime alone.

Part 4 - Python code skeleton

```
import torch

@torch.no_grad()
def evaluate_by_length(model, task_factory, lengths, device):
    report = {}
    model.eval()
    for length in lengths:
        loader = task_factory(length)
        report[length] = {
            "metric": ...,
            "loss": ...,
            "attention_entropy": ...,
            "latency": ...,
            "peak_memory": ...,
        }
    return report

def E08(variants, train_max=128):
    lengths = [32, 64, 128, 160, 256, 512, 1024]
    results = {}
    for name, model in variants.items():
```

```
        # TODO: train with T <= train_max, freeze, then evaluate grid
        results[name] = evaluate_by_length(model, ..., lengths, ...)
    return results
```

Part 5 - The Philosophy Behind the Exercise

Extrapolation asks whether a representation contains a rule or only a map of familiar territory. A formula can extend coordinates beyond experience, but a trained model may still have learned habits tied to the region it observed.

Long context therefore tests two kinds of generalization: the positional system's ability to describe new locations and the model's ability to reason under those descriptions. Confusing these levels produces overly simple claims about "supporting" a length.

The responsible language is empirical. A method is not long-context merely because it accepts a large index. It becomes useful at a length when measured behavior, numerical stability, and resource cost remain within declared requirements.

17.4.10 Exercise 09

Exercise Title: Create a Practical Guideline for Choosing Positional Encoding Method for a Given Task

Part 1 - Why this matters

Positional encoding choice should follow task geometry and deployment constraints, not a universal ranking. Some tasks depend on exact absolute slots, others on relative distance or direction, and others require stable extrapolation far beyond training length. The same method can be excellent in one regime and a source of avoidable failure in another.

The guideline must translate requirements into testable questions. Is the maximum sequence length fixed? Does the task shift patterns to new absolute locations? Are distances beyond a threshold distinguishable? Is incremental decoding required? Must the model load checkpoints across changed context lengths? What precision and latency budgets apply? Each answer narrows the reasonable choices.

Build the guide from chapter evidence. Absolute learnable embeddings may adapt well inside a fixed range. Sinusoidal encodings add no parameters and define unseen indices. Relative bias can make displacement patterns easier and allow per-head distance preferences. RoPE modifies Q/K geometry and integrates well with cached causal attention, but long-range phase and scaling require testing. Combinations are possible, though each added mechanism increases complexity.

The output should be a decision procedure with rejection conditions and required validation, not a table of slogans. Every recommendation must include the experiment that would falsify it. If two options satisfy requirements, prefer the simpler default until measured quality justifies complexity.

Part 2 - Mathematical form and reasoning

Represent a candidate v by a requirement vector

$$r_v = (q_{\text{in}}, q_{\text{extra}}, c_{\text{memory}}, c_{\text{latency}}, s_{\text{precision}}, u_{\text{cache}}),$$

where the components are measured rather than merely rated. A method is feasible only if hard constraints are satisfied:

$$q_{\text{extra}} \geq \tau_q, qquad c_{\text{memory}} \leq M_{\max}, qquad c_{\text{latency}} \leq L_{\max}.$$

Among feasible methods, a weighted score may summarize preferences, but the weights and normalization must be declared. Hard failures should never be hidden by averaging them with strengths elsewhere.

Part 3 - Hints, controls, and evidence

1. Begin with task questions about absolute location, relative distance, direction, maximum length, extrapolation, and decoding mode.
2. Separate hard constraints from preferences and benchmark every candidate under the target dtype and hardware.
3. Include failure signatures: out-of-range tables, off-by-one offsets, bucket saturation, rotary phase error, and padding misalignment.
4. Attach required tests for in-range quality, extrapolation, cached parity, precision, latency, memory, and checkpoint compatibility.
5. Use the simplest method that satisfies requirements as the provisional default, then document evidence for any deviation.
6. Revisit the choice when context length, data distribution, task geometry, or deployment environment changes.

Part 4 - Python code skeleton

```
from dataclasses import dataclass

@dataclass
class Requirements:
    needs_absolute_slots: bool
    needs_relative_distance: bool
    train_length: int
    deploy_length: int
    autoregressive_cache: bool
    max_latency_ms: float
    max_position_bytes: int

def evaluate_candidate(name, requirements, evidence):
    hard_checks = {
        "length_supported": ...,
        "quality_met": ...,
        "latency_met": ...,
```

```
        "memory_met": ...,
        "cache_parity": ...,
    }
    return {"name": name, "feasible": all(hard_checks.values()),
            "checks": hard_checks, "tradeoffs": ...}

def E09(requirements, chapter_artifacts):
    candidates = ["sinusoidal", "learnable", "relative_bias", "rope"]
    reports = [evaluate_candidate(c, requirements, chapter_artifacts[c])
               for c in candidates]
    # TODO: select only among feasible methods and justify default
    return reports
```

Part 5 - The Philosophy Behind the Exercise

Engineering judgment is not the possession of one preferred technique. It is the ability to translate a situation into requirements, compare evidence, and choose while preserving the conditions of that choice.

A default is useful because it reduces decision cost, but it should remain a provisional belief rather than a doctrine. The moment task geometry changes, the evidence supporting the default must be reconsidered.

The guideline therefore teaches disciplined pluralism. Different positional methods embody different assumptions about order. The goal is not to crown one assumption, but to make the chosen assumption visible and accountable.

17.4.11 Exercise 10

Exercise Title: Run Positional Encoding Experiments on CUDA and Benchmark Overhead Compared to Baseline Attention

Part 1 - Why this matters

Positional mechanisms add work before or inside attention. Additive encodings perform table slicing or formula generation plus a tensor addition. Relative bias constructs or retrieves a score-shaped bias. RoPE calculates or loads sine/cosine values and rotates Q/K features. Their overhead may be negligible beside long-sequence attention, but significant for small batches, short sequences, cached single-token decoding, or an implementation that rebuilds large tables every step.

Benchmark the positional operation separately and as part of the complete attention path. A microbenchmark reveals operation cost; end-to-end timing shows whether that cost matters after kernel scheduling and fusion. Precompute constant tables outside the measured loop. Compare cold initialization, steady-state training, full-sequence inference, and one-token cached decoding as distinct scenarios.

CUDA timing requires warm-up and synchronization. Use CUDA events or explicit synchronization around wall-clock measurements, generate inputs beforehand, and prevent host-side logging from entering the timed region. Record median and tail latency, throughput, peak

allocated and reserved memory, and the precise backend and dtype.

Correctness must precede speed. For each method, compare CPU and CUDA outputs, offset behavior, masks, and gradients within declared tolerances. A fast RoPE implementation using the wrong pairing convention or a cached table with the wrong offset is not a valid optimization.

Part 2 - Mathematical form and reasoning

For baseline attention time t_0 and positional variant t_v, report

$$\Delta t_v = t_v - t_0, \qquad R_v = \frac{t_v - t_0}{t_0} \times 100\%.$$

Measure both operation-only and end-to-end ratios. Additive encoding has linear element work

$$\mathcal{O}(BTd),$$

while a materialized relative bias may contain BHT_qT_k logical entries even if generated from a smaller table. Dense attention remains $\mathcal{O}(BT^2d)$, so relative overhead can shrink as T grows while absolute memory remains important.

Part 3 - Hints, controls, and evidence

1. Benchmark no position, cached sinusoidal, learnable table, relative bias, and RoPE under matched attention code.
2. Separate table construction from table use and report cold and steady- state behavior.
3. Sweep batch, sequence length, head count, dtype, training/inference, and cached decoding.
4. Warm up, synchronize, repeat, and report median, p95, throughput, peak allocated memory, and peak reserved memory.
5. Check CPU–CUDA parity and full-sequence versus cached-offset parity before timing.
6. Profile unexpectedly expensive methods to identify allocation, indexing, trigonometric, broadcast, or fusion bottlenecks.

Part 4 - Python code skeleton

```
import statistics
import torch

def cuda_benchmark(fn, warmup=25, repeats=100):
    for _ in range(warmup):
        fn()
    torch.cuda.synchronize()
    samples = []
    for _ in range(repeats):
        start = torch.cuda.Event(enable_timing=True)
```

```
        end = torch.cuda.Event(enable_timing=True)
        start.record(); fn(); end.record()
        torch.cuda.synchronize()
        samples.append(start.elapsed_time(end))
    return {"median_ms": statistics.median(samples), "p95_ms": ...}

def benchmark_variant(attention, position_module, x, mode):
    torch.cuda.reset_peak_memory_stats()
    timing = cuda_benchmark(lambda: ...)
    return {**timing,
            "peak_allocated": torch.cuda.max_memory_allocated(),
            "peak_reserved": torch.cuda.max_memory_reserved()}

def E10(configurations):
    if not torch.cuda.is_available():
        return {"status": "skipped", "reason": "CUDA unavailable"}
    return ...
```

Part 5 - The Philosophy Behind the Exercise

A representation has a computational form. Positional ideas that appear equivalent at the level of purpose can interact very differently with memory, kernel fusion, caching, and batch shape. Conceptual elegance does not determine runtime cost.

Overhead is relational rather than absolute. The same positional operation may be invisible inside a large training step and dominant inside one-token generation. Performance statements require a workload, not merely a method name.

The benchmark teaches that optimization begins with preservation of meaning. Only after parity and alignment are proved does faster execution become evidence of progress.

17.4.12 Exercise 11

Exercise Title: Compare float32 vs float16 Positional Encoding Stability and Detect Precision Edge Cases in Long Sequences

Part 1 - Why this matters

Long positions stress finite precision in ways that short-sequence tests do not reveal. If a large integer position is first converted to float16, adjacent indices may become identical because float16 cannot represent every large integer. Trigonometric arguments may lose phase precision, and adding a small positional vector to a large token activation may round away part of the signal. RoPE and sinusoidal methods are especially sensitive to phase calculation, while learned tables mainly face storage and addition precision.

Generate positions and angles in float32 or float64 even when the final model runs under float16 autocast. Then cast the completed sine/cosine table to the model dtype. Compare this safer path with direct float16 generation at increasing positions. Measure absolute phase error, cosine similarity between encodings, collisions between adjacent positions, norm preservation

for RoPE, and downstream attention-logit error.

Precision failure is not limited to NaN. Two positions can receive nearly identical encodings, a rotation can lose orthogonality, or a small positional difference can vanish when added to a large activation. Record representational separation and behavioral quality, not only finiteness.

Use float64 as a diagnostic reference, float32 as the standard path, and float16 or bfloat16 as reduced-precision candidates. Bfloat16 has a wider exponent but fewer mantissa bits, so its error pattern differs. State device support and actual autocast behavior rather than assuming every operation uses the requested dtype.

Part 2 - Mathematical form and reasoning

For encoding e_p, adjacent separation is

$$D(p) = \|e_{p+1} - e_p\|_2.$$

A precision collision occurs when

$$\widehat{e}_{p+1} = \widehat{e}_p$$

or when $D(p)$ falls below a declared tolerance relative to a higher-precision reference. Phase error for frequency ω_i is

$$E_\theta(p, i) = |\widehat{p\omega_i} - p\omega_i|.$$

For rotary transformation, measure norm error

$$E_{\text{norm}} = \frac{|\|R_{\widehat{\theta}}x\|_2 - \|x\|_2|}{\max(\|x\|_2, \varepsilon)}.$$

Part 3 - Hints, controls, and evidence

1. Compare positions constructed directly in reduced precision with positions constructed in float64 or float32 and cast only after encoding.
2. Sweep indices logarithmically and include adjacent positions around each precision boundary.
3. Test sinusoidal tables, RoPE angles, learned-table addition, and relative bias indexing separately.
4. Record finite values, phase error, adjacent separation, encoding cosine similarity, RoPE norm error, logits, and retrieval accuracy.
5. Test large activation magnitudes to observe positional information lost during addition.
6. Define dtype-specific tolerances before examining results and report the first boundary where a functional invariant fails.

Part 4 - Python code skeleton

```
import torch

def generate_encoding(positions, d_model, compute_dtype, output_dtype):
    work = positions.to(compute_dtype)
    frequencies = ...
    angles = work[:, None] * frequencies[None, :]
    encoding = ...
    return encoding.to(output_dtype), angles

def precision_metrics(reference, candidate):
    adjacent = candidate[1:] - candidate[:-1]
    return {"max_abs": ...,
            "adjacent_min_norm": adjacent.norm(dim=-1).min(),
            "collision_count": ...,
            "cosine_to_reference": ...,
            "finite": bool(torch.isfinite(candidate).all())}

def E11():
    positions = torch.tensor([...], dtype=torch.int64)
    reference, _ = generate_encoding(positions, 64,
                                     torch.float64, torch.float64)
    # TODO: direct and promoted fp32/fp16/bfloat16 paths
    return ...
```

Part 5 - The Philosophy Behind the Exercise

Position appears discrete, but its encoding often passes through continuous functions represented by finite numbers. At large scale, the machine may no longer distinguish neighboring integers even though the mathematical sequence continues without ambiguity.

This is a reminder that “long context” is partly a numerical claim. Memory capacity and architectural formulas are insufficient if the coordinate system loses resolution where the model is asked to operate.

Promoting a small critical calculation to higher precision illustrates a general engineering strategy: precision should be allocated according to sensitivity. Efficiency does not require every operation to forget equally.

17.4.13 Exercise 12

Exercise Title: Visualize Learned Positional Embeddings and Interpret Whether They Encode Smooth Trends or Discrete Jumps

Part 1 - Why this matters

A learned positional table can develop global trends, local neighborhoods, periodic structure, special boundary vectors, or irregular slot-specific codes. Visualization helps form hypotheses about this geometry, but a two- dimensional plot cannot by itself determine what the model uses. The exercise combines plots with quantitative diagnostics and interventions.

Visualize individual dimensions over position, vector norms, adjacent cosine similarity, pairwise distance matrices, singular values, and low-dimensional projections. Compare the initialized table with several training checkpoints. A PCA or UMAP path may appear smooth even when important information lies in discarded dimensions, so always report explained variance and preserve direct high-dimensional measures.

Discrete jumps may correspond to task boundaries, special tokens, periodic roles, or simply optimization noise. Align plots with known data structure and repeat seeds. Shuffle or interpolate selected position vectors and measure task degradation. If smooth interpolation leaves behavior unchanged, the model may rely on a broad positional trend; if swapping two nearby rows causes a large failure, it may use slot-specific identity.

Attention-map interpretation remains limited. A smooth positional table does not prove smooth attention behavior because Q/K projections transform it. Analyze projected positional components and downstream outputs as well as the raw table.

Part 2 - Mathematical form and reasoning

For learned table $E \in \mathbb{R}^{L \times d}$, adjacent variation is

$$\Delta_p = \|E_{p+1} - E_p\|_2,$$

and adjacent cosine similarity is

$$c_p = \frac{E_p^{\mathsf{T}} E_{p+1}}{\|E_p\|_2 \|E_{p+1}\|_2}.$$

A discrete-jump score may compare local change with a neighborhood baseline:

$$J_p = \frac{\Delta_p}{\operatorname{median}(\Delta_{p-k:p+k}) + \varepsilon}.$$

For singular values σ_i, the fraction of variance retained by the first r principal directions is

$$V_r = \frac{\sum_{i=1}^{r} \sigma_i^2}{\sum_i \sigma_i^2}.$$

Part 3 - Hints, controls, and evidence

1. Save the same table at initialization and fixed checkpoints under several seeds.
2. Plot dimensions, norms, adjacent differences, cosine similarity, pairwise distances, singular spectrum, and PCA with explained variance.
3. Compare raw embeddings with their Q and K projected forms.
4. Mark known sequence boundaries or task roles without assuming they cause the observed pattern.
5. Interpolate, swap, shuffle, or smooth selected rows and measure held-out behavior.
6. Treat every visual interpretation as a hypothesis until an intervention supports it.

Part 4 - Python code skeleton

```
import torch

@torch.no_grad()
def position_statistics(table):
    delta = table[1:] - table[:-1]
    cosine = torch.nn.functional.cosine_similarity(table[1:], table[:-1])
    centered = table - table.mean(dim=0, keepdim=True)
    _, singular, right = torch.linalg.svd(centered, full_matrices=False)
    projected = centered @ right[:2].T
    return {"norm": table.norm(dim=-1),
            "delta_norm": delta.norm(dim=-1),
            "adjacent_cosine": cosine,
            "singular": singular,
            "pca2": projected,
            "explained_variance": ...}

def position_intervention(model, kind, indices):
    # TODO: swap, smooth, interpolate, or shuffle selected rows safely
    return ...

def E12(model, checkpoints, probe_loader):
    # TODO: statistics, plots, seed comparison, and behavioral interventions
    return ...
```

Part 5 - The Philosophy Behind the Exercise

Visualization converts an invisible high-dimensional object into a form human perception can inspect. This is useful precisely because it is lossy: it offers a perspective, not the object itself.

Smoothness and discontinuity are interpretations relative to a coordinate and a scale. A vector path may look smooth in two principal components while changing sharply in a task-relevant discarded direction. Observation must remain aware of its projection.

The exercise teaches a mature relation between seeing and knowing. A pattern in a figure begins a question. Intervention, replication, and behavioral measurement decide whether that question becomes an explanation.

17.4.14 Exercise 13

Exercise Title: Implement Unit Tests for Positional Encoding Correctness and Verify It Works for Multiple Batch Sizes

Part 1 - Why this matters

Positional code sits near the beginning of every Transformer layer stack, so a small error propagates through the entire model. Unit tests should isolate its contract before training:

shape preservation, correct position values, broadcasting across batches, offset handling, gradient policy, device movement, dtype behavior, maximum-length boundaries, padding alignment, and serialization. The tests should be fast enough to run on every change.

Batch-size tests are essential because accidental broadcasting can pass at $B = 1$. A table shaped `(T,1,d)` may add correctly to a sequence-first tensor while silently misbehaving with batch-first data. Test several unequal batch and sequence sizes and compare each batch row with a single-example reference. Position should normally be shared across examples, while token content remains independent.

Each positional method needs specific invariants. Sinusoidal position zero and manual formula values can be exact within tolerance. Learnable tables require gradient and index-range tests. Relative bias requires displacement orientation and clipping tests. RoPE requires norm preservation, zero-position identity, pairing convention, and full versus cached-offset parity.

Add property-style random tests and explicit failure injection. Tests must fail for a wrong unsqueeze axis, an offset shift, a CPU-created position tensor on CUDA, an odd rotary dimension, and an out-of-range learned index. Demonstrating that a test catches its target bug validates the test itself.

Part 2 - Mathematical form and reasoning

Core additive-position invariants include

$$\text{shape}(f(X)) = \text{shape}(X),$$

$$f(X)_{b,t,:} - X_{b,t,:} = E_{t,:}$$

for every batch b, and offset consistency

$$f(X, c)_{:,t,:} - X_{:,t,:} = E_{c+t,:}.$$

For a batch split $X = [X^{(1)}; X^{(2)}]$,

$$f(X) = [f(X^{(1)}); f(X^{(2)})]$$

when examples are independent. RoPE additionally requires, within tolerance,

$$\|R_p x\|_2 = \|x\|_2.$$

Part 3 - Hints, controls, and evidence

1. Organize tests into shared contracts and method-specific contracts.
2. Parameterize batch sizes, lengths, widths, offsets, devices, and dtypes; include unequal dimensions and edge cases.
3. Compare batched execution with concatenated single-example references.
4. Test state-dictionary save/load and movement between CPU and CUDA when available.

5. Mark precision-dependent assertions with justified tolerances instead of weakening every test globally.

6. Inject known bugs and prove each test fails for the earliest relevant invariant.

Part 4 - Python code skeleton

```
import pytest
import torch

@pytest.mark.parametrize("batch,length,width", [
    (1, 1, 7), (2, 5, 8), (5, 3, 12)
])
def test_additive_shape_and_broadcast(batch, length, width):
    module = ...
    x = torch.randn(batch, length, width)
    y = module(x)
    assert y.shape == x.shape
    delta = y - x
    assert torch.allclose(delta, delta[:1].expand_as(delta))

def test_offset_matches_full_sequence():
    module = ...
    full = module(torch.zeros(1, 10, 8))
    block = module(torch.zeros(1, 4, 8), offset=6)
    assert torch.allclose(block, full[:, 6:10])

def E13():
    # TODO: sinusoidal, learned, relative, RoPE, device,
    # serialization, failure-injection, and property tests
    return ...
```

Part 5 - The Philosophy Behind the Exercise

A unit test is a compact claim about the world of the program. It states what must remain true across refactoring, devices, batches, and future experiments. The collection of tests becomes an executable definition of the component.

Testing multiple batch sizes demonstrates that correctness is not one example that happened to fit. A representation should preserve its intended symmetry and alignment across the range of shapes its interface promises.

Failure injection gives tests credibility. A green test suite proves little if its assertions would remain green under the bug it was designed to prevent. Trust grows when the test is shown to distinguish correct and incorrect worlds.

17.4.15 Exercise 14

Exercise Title: Debug Variable-Length Sequence Handling with Positional Encoding and Ensure Padding Tokens Do Not Break It

Part 1 - Why this matters

Variable-length batches introduce two coordinate systems: storage positions in the padded tensor and logical positions among valid tokens. With right padding, these often agree for valid tokens. With left padding, a token's storage index depends on the amount of padding, so assigning absolute positions directly by column can make the same content token receive different position IDs across the batch. Whether this is a bug depends on the model's declared positional policy, but inconsistency between training and inference is always dangerous.

Padding tokens should not influence legal attention outputs. An additive positional vector may still be placed at padding slots, but key padding masks must prevent those slots from being retrieved, and loss masks must ignore their targets. Alternatively, position IDs for padding can be set to a safe row. Zeroing padding representations alone is insufficient if biases, layer normalization, or later operations can reintroduce nonzero values. The information boundary must be proved at attention and loss.

Compare unpadded single-sequence execution with right-padded and left-padded batched execution. Extract outputs at valid token positions and require parity within tolerance under the chosen policy. Then change padding token IDs, padding embeddings, and positional values by large amounts. Valid outputs must remain unchanged when masks are correct.

Packed or nested representations may reduce padding waste, but the logical position IDs must still follow each sequence. Incremental decoding introduces another length: cached valid length rather than padded tensor width. The exercise should test these boundaries before long-context training.

Part 2 - Mathematical form and reasoning

Let $m_{b,t} = 1$ for valid tokens. Content-relative position IDs are

$$p_{b,t} = \max\left(0, \sum_{u=0}^{t} m_{b,u} - 1\right).$$

For right padding this gives $0, 1, \ldots, \ell_b - 1$ followed by a safe repeated index. For left padding, valid tokens still begin at zero. If storage-relative positions are intentionally used, instead $p_{b,t} = t$; the policy must match training and serving.

For valid query i, padding invariance requires

$$A_{b,r,i,j} = 0 \quad \text{when } m_{b,j} = 0,$$

and therefore changing any padded $V_{b,r,j,:}$ cannot change $Z_{b,r,i,:}$.

Part 3 - Hints, controls, and evidence

1. Build identical logical sequences in unpadded, right-padded, and left-padded forms and print token, mask, storage index, and position ID.

2. Declare whether positions are storage-relative or content-relative and use the same rule in training, evaluation, and caching.
3. Compare valid-token outputs against independent unpadded references.
4. Replace padding token and position vectors with extreme values and prove valid outputs and loss are unchanged.
5. Test empty padding, one valid token, all-padding rejection, mixed lengths, and cached continuation.
6. Confirm both attention and loss masks are correct; one cannot substitute for the other.

Part 4 - Python code skeleton

```
import torch

def content_relative_positions(attention_mask):
    positions = attention_mask.long().cumsum(-1) - 1
    return positions.clamp_min(0)

@torch.no_grad()
def valid_output_parity(model, logical_sequences, padding_side):
    padded, mask = ...
    positions = content_relative_positions(mask)
    batch_output = model(padded, attention_mask=mask,
                         position_ids=positions)
    reference = ...  # independent unpadded runs
    # TODO: gather valid tokens and compare
    return ...

def E14(model):
    reports = {}
    for side in ["left", "right"]:
        reports[side] = valid_output_parity(model, ..., side)
    # TODO: extreme-padding intervention and cached continuation
    return reports
```

Part 5 - The Philosophy Behind the Exercise

Padding is absence represented as data. Because hardware prefers regular rectangles, nothingness is given a token, an index, and storage. Correct masking preserves the distinction between the representation of absence and actual information.

Variable-length processing also shows that position is a convention. A storage column and a logical moment can coincide or diverge depending on padding. The model must know which coordinate system the application intends.

Robustness comes from proving irrelevance. If padding is declared meaningless, then changing it should not change valid behavior. Intervention turns that declaration into evidence.

17.4.16 Exercise 15

Exercise Title: Stress Test Long Context Lengths and Detect Memory and Runtime Costs of Large Positional Tables

Part 1 - Why this matters

Positional mechanisms have their own scaling costs in addition to attention. A learned absolute table grows as Ld. Cached sinusoidal or rotary tables have the same linear table size, although they are not optimizer parameters. Relative position bias may store a small bucket table but materialize a large score-shaped bias. A careless implementation may rebuild or repeat these tensors for every batch, turning a modest theoretical cost into a practical memory problem.

Stress testing should isolate table storage, per-step temporary memory, and full-model memory. Measure parameter memory, gradients, optimizer states, buffers, generated angle or bias tensors, attention activations, and allocator reserved memory separately. A learnable table under Adam can require weight, gradient, and two optimizer moments, while a deterministic buffer may require only one persistent copy.

Sweep context length while keeping other variables fixed. Compare eager generation, precomputed caches, sliced caches, broadcast views, repeated tensors, and on-demand block generation. Measure initialization time separately from steady-state use. For cached decoding, record the cost of extending a table or requesting positions beyond its original maximum.

Run potentially failing configurations in isolated processes and preserve a safety margin. The purpose is to identify scaling regimes and first failure boundaries, not to destabilize the workstation. Compare measured memory with formula-based estimates and explain discrepancies caused by dtype, optimizer, temporary tensors, alignment, or attention itself.

Part 2 - Mathematical form and reasoning

For table length L, width d, and element size s bytes,

$$M_{\text{table}} = Lds.$$

A learnable table with gradient and two float32 Adam moments may require approximately

$$M_{\text{train}} \approx M_{\text{weight}} + M_{\text{grad}} + 2M_{\text{moment}},$$

with additional master weights in some mixed-precision configurations. A materialized per-head relative bias has

$$M_{\text{bias}} = BHT_qT_ks,$$

although the learned bucket parameters may contain only $H(2D + 1)$ values.

Part 3 - Hints, controls, and evidence

1. Estimate every persistent and temporary positional tensor before running the sweep.
2. Measure learned, sinusoidal, relative-bias, and rotary implementations at the same lengths and dtypes.

3. Separate initialization, forward inference, backward training, and cached one-token decoding.
4. Compare slice/broadcast implementations with repeat/materialization and inspect storage and strides.
5. Record allocated and reserved peaks, table bytes, optimizer bytes, latency, and first failed configuration.
6. Use isolated processes and stop after the declared boundary rather than repeatedly exhausting the CUDA allocator.

Part 4 - Python code skeleton

```
import multiprocessing as mp
import torch

def parameter_bytes(module):
    return sum(p.numel() * p.element_size() for p in module.parameters())

def buffer_bytes(module):
    return sum(b.numel() * b.element_size() for b in module.buffers())

def isolated_position_case(config, queue):
    try:
        torch.cuda.reset_peak_memory_stats()
        module, x = ...
        # TODO: initialize, forward/backward, and synchronize
        queue.put({"status": "ok",
                   "parameter_bytes": parameter_bytes(module),
                   "buffer_bytes": buffer_bytes(module),
                   "peak_allocated": torch.cuda.max_memory_allocated(),
                   "peak_reserved": torch.cuda.max_memory_reserved(),
                   "latency_ms": ...})
    except torch.cuda.OutOfMemoryError as error:
        queue.put({"status": "oom", "message": str(error)})

def E15(configurations):
    # TODO: safe isolated sweep and theoretical-vs-measured report
    return ...
```

Part 5 - The Philosophy Behind the Exercise

A coordinate system consumes resources. To remember or calculate where every token exists, the system must store parameters, buffers, phases, biases, or temporary relations. Position is conceptual structure embodied in memory.

Scaling exposes distinctions hidden at small size. Two methods that both add a minor tensor at 512 tokens may have very different optimizer, materialization, or caching behavior at

100,000 tokens. Small experiments reveal correctness; stress tests reveal architecture.

The goal is not to fear limits but to measure them. A known boundary enables planning, optimization, and honest claims. An unknown boundary becomes an incident.

17.4.17 Exercise 16

Exercise Title: Create a Checklist for Positional Encoding Bugs Including Indexing, Shapes, and Device Placement

Part 1 - Why this matters

Positional failures frequently survive until late training because their tensors have valid shapes and finite values. A systematic checklist should detect them before optimization: wrong starting index, shifted special tokens, incorrect batch/sequence layout, table overflow, padding-dependent alignment, stale decoding offset, CPU-created indices inside a CUDA path, unintended trainability, precision loss, and incompatible checkpoint resizing.

Order the checklist from contracts to behavior. Begin with method configuration and supported length. Continue through input layout, position-ID construction, shape and broadcast, device and dtype, method-specific invariants, masks and padding, full-versus-cached parity, gradients, serialization, and long-range stress. Later symptoms should be marked as blocked when an earlier invariant has already failed.

Each item needs an executable condition and useful evidence. "Check device" becomes an assertion listing devices of input, indices, tables, masks, and outputs. "Check indexing" becomes a token-position ledger with expected IDs. "Check RoPE" becomes position-zero identity, norm preservation, declared pair ordering, and offset parity.

Create an incident template that records hypothesis, formula, tensor contract, configuration, seed, device, precision, expected behavior, earliest violated invariant, cause, correction, and regression test. The checklist becomes valuable when it retains knowledge after the immediate bug is fixed.

Part 2 - Mathematical form and reasoning

Common invariants include

$$0 \le p_{b,t} < L_{\max},$$

$$\operatorname{shape}(X + E) = \operatorname{shape}(X),$$

$$\operatorname{device}(p) = \operatorname{device}(X) = \operatorname{device}(E),$$

and cached offset consistency

$$E(X_{c:c+T}, 0) = E(X_{0:T}, c)$$

when the notation denotes equivalent slices with global position offset c. For padding, valid-output invariance requires legal outputs to remain unchanged under arbitrary changes to padded token and positional values.

Part 3 - Hints, controls, and evidence

1. Group checks into configuration, indexing, shape, device/dtype, padding, method-specific geometry, caching, gradients, serialization, and stress.
2. Attach pass/fail conditions, evidence payload, and likely causes to every check.
3. Use asymmetric shapes and traceable position values so axis swaps and off-by-one errors cannot hide.
4. Inject wrong start index, wrong unsqueeze, CPU indices, stale offset, padding count, out-of-range table, and low-precision phase failures.
5. Stop at the earliest violated invariant and retain every repaired failure as an automated regression test.
6. Export a machine-readable checklist report suitable for continuous integration and experiment artifacts.

Part 4 - Python code skeleton

```
from dataclasses import dataclass, asdict

@dataclass
class PositionCheck:
    name: str
    passed: bool
    evidence: dict
    likely_causes: list
    blocked_by: str | None = None

def check_position_range(position_ids, max_length):
    passed = bool(((position_ids >= 0) &
                   (position_ids < max_length)).all())
    return PositionCheck("position_range", passed,
                         {"min": int(position_ids.min()),
                          "max": int(position_ids.max())},
                         ["off-by-one", "stale cache offset",
                          "table too short"])

def run_position_checklist(module, case):
    checks = []
    # TODO: ordered checks and blocked downstream diagnostics
    return {"passed": all(c.passed for c in checks),
            "checks": [asdict(c) for c in checks]}

def E16():
    # TODO: correct cases plus failure-injection matrix
    return ...
```

Part 5 - The Philosophy Behind the Exercise

A checklist is a model of how failure propagates. Its order expresses a causal theory: indexing precedes alignment, alignment precedes meaningful attention, and correctness precedes performance.

Positional bugs are particularly instructive because they often preserve surface validity. The numbers exist, fit, and move through the network, yet they describe the wrong history. Engineering must test association, not only existence.

By turning failures into permanent checks, the project acquires memory beyond any individual researcher. The system becomes less dependent on remembering every past mistake and more capable of preventing its return.

17.4.18 Exercise 17

Exercise Title: Compare Training Speed and Convergence Differences Between Sinusoidal and Learnable Positional Encoding

Part 1 - Why this matters

Sinusoidal and learnable positions differ not only in final quality but in the optimization problem they create. A fixed encoding supplies stable coordinates from the first update. A learnable table begins from an initialization and must discover a useful geometry jointly with the model. This may adapt better, converge more slowly, overfit particular slots, or receive uneven gradients when some positions occur more frequently than others.

Training speed has several meanings. Wall-clock time per step measures systems cost. Updates or tokens required to reach a validation threshold measure sample and optimization efficiency. Best metric under a fixed budget measures final utility. Report all three. A method can have negligible runtime overhead yet require more training steps, or reach a better final metric while learning more slowly at the beginning.

Use shared initialization for every common parameter. The learnable variant contains extra positional parameters and optimizer states, which should be reported rather than silently ignored. Match dataset order, batch size, optimizer, learning-rate schedule, evaluation frequency, stopping rule, and random seeds. Repeat enough seeds to estimate variance.

Track positional gradient norms and row update counts. In a variable-length dataset, high position rows may be observed rarely, producing poorer estimates. Evaluate quality by position and sequence length so an aggregate metric does not hide tail-position failure.

Part 2 - Mathematical form and reasoning

Let $u_{v,s}(\tau)$ be the number of optimizer updates needed by variant v and seed s to reach metric threshold τ. Let $t_{v,s}(\tau)$ be wall-clock time and $n_{v,s}(\tau)$ processed tokens. Report all of

$$\overline{u}_v(\tau), \qquad \overline{t}_v(\tau), \qquad \overline{n}_v(\tau)$$

with dispersion. For learned table row p, accumulate exposure

$$C_p = \sum_{\text{batches}} \#\{(b,t) : p_{b,t} = p\}$$

and relate it to gradient magnitude and position-specific error without assuming correlation proves causality.

Part 3 - Hints, controls, and evidence

1. Initialize common parameters once and copy them into both variants; initialize only the positional component differently.
2. Match data order, optimization budget, schedule, batch, validation rule, checkpoint selection, and seeds.
3. Measure step latency with proper CUDA warm-up and synchronization outside the main training loop.
4. Report steps, tokens, and seconds to fixed validation thresholds plus best fixed-budget quality.
5. Log learned-row exposure, gradient norms, norm growth, and metrics by position and length.
6. Include no-position baseline and several seeds; retain failed or divergent runs in the summary.

Part 4 - Python code skeleton

```
import time
import torch

def train_trial(variant, shared_state, loaders, config, seed):
    torch.manual_seed(seed)
    model = ...
    model.load_shared_state_dict(shared_state)
    history = []
    position_exposure = ...
    start = time.perf_counter()
    for step, batch in enumerate(loaders["train"]):
        # TODO: update, log loss/metric/gradients/exposure
        if step % config["eval_every"] == 0:
            history.append(...)
    return {"variant": variant, "seed": seed,
            "history": history,
            "elapsed": time.perf_counter() - start,
            "position_exposure": position_exposure}

def E17(variants, seeds=range(5)):
    # TODO: matched shared initialization and threshold summaries
    return ...
```

Part 5 - The Philosophy Behind the Exercise

Learning a coordinate system and learning within a coordinate system are different burdens. A fixed encoding gives the model a world already arranged; a learnable table asks it to negotiate the arrangement while solving the task.

Convergence is therefore a history, not a final score. Two models can arrive at similar performance through paths with different data requirements, stability, and compute. The path matters when resources and failure risk are finite.

The comparison teaches that flexibility has an optimization price. Whether that price is worthwhile depends on the value of adaptation and the evidence that the learned geometry improves what the task actually requires.

17.4.19 Exercise 18

Exercise Title: Produce a Report Explaining Positional Encoding Tradeoffs and Which One You Will Use as Default

Part 1 - Why this matters

This report converts the chapter's experiments into a defensible default for future Transformer chapters. It must explain the symmetry that makes position necessary, then compare sinusoidal, learnable absolute, relative-bias, and rotary methods across geometry, parameterization, extrapolation, precision, padding, cached decoding, speed, memory, training convergence, and debugging complexity.

The chosen default must be conditional. State the target model type, task, training-length range, deployment length, dtype, hardware, decoding mode, and checkpoint expectations. A default selected for a causal language model may not transfer to a fixed-length image or tabular Transformer. The report should include explicit conditions that would trigger reconsideration.

Every major statement should cite chapter evidence: permutation tests from E00, manual formula validation, toy-task comparisons, off-by-one regression tests, attention interventions, shift behavior, RoPE geometry, extrapolation curves, CUDA overhead, precision boundaries, visual hypotheses, batch tests, padding parity, stress limits, and convergence distributions. Negative results and unresolved questions belong beside successes.

Do not collapse hard requirements into an average score. A method that exceeds memory or cannot process required lengths is infeasible even if it has strong in-range quality. Among feasible methods, explain why the default offers the best evidence-weighted tradeoff and retain a simpler fallback where appropriate.

Part 2 - Mathematical form and reasoning

The report should compare at least the forms

$$Y = X + E_p,$$

$$S_{i,j} = \frac{q_i^\top k_j}{\sqrt{d_h}} + b_{j-i},$$

and rotary compatibility

$$(R_m q)^\mathsf{T}(R_n k) = q^\mathsf{T} R_{n-m} k.$$

For each method v, define a feasibility vector containing measured in-range quality, extrapolation quality, latency, memory, numerical boundary, cache parity, and engineering complexity. Hard constraints are evaluated first; preferences are compared only among feasible candidates.

Part 3 - Hints, controls, and evidence

1. Begin with the application contract and list hard positional requirements before presenting methods.
2. Create an evidence matrix mapping every claim to configuration, seeds, metric, artifact, and limitation.
3. Separate mathematical definition, implementation behavior, measured result, interpretation, and recommendation.
4. Include failure cases and conditions under which each method should be rejected.
5. Choose one default for the declared project rather than claiming a universal winner.
6. Finish with monitoring signals and explicit triggers for revisiting the default as context or deployment changes.

Part 4 - Python code skeleton

```
from pathlib import Path
import json

REQUIRED = ["permutation", "correctness", "toy_quality",
            "extrapolation", "cuda_overhead", "precision",
            "padding", "memory", "convergence", "debug_tests"]

def validate_evidence(artifacts):
    return {name: ... for name in REQUIRED}

def feasibility(method, requirements, evidence):
    return {"length": ..., "quality": ..., "latency": ...,
            "memory": ..., "precision": ..., "cache": ...}

def E18(artifact_directory, requirements, output_path):
    artifacts = ...
    checks = validate_evidence(artifacts)
    assert all(checks.values()), checks
    candidates = ...
    # TODO: reject infeasible methods, select conditional default,
    # and write evidence-linked report
    Path(output_path).write_text(..., encoding="utf-8")
    return {"report": str(output_path), "checks": checks}
```

Part 5 - The Philosophy Behind the Exercise

A default is a compressed decision. It carries the history of experiments into future work so the same comparison need not be repeated at every step. Its value depends on preserving the conditions under which it was chosen.

Tradeoff reports resist the desire for a single victorious method. They show that every coordinate system makes some relations natural, some expensive, and some invisible. Selection is an act of aligning those consequences with a purpose.

The report should therefore end with intellectual reversibility. A strong decision is not one that can never change; it is one whose evidence, limits, and revision triggers are clear enough that change can occur rationally.

17.4.20 Exercise 19

Exercise Title: Write a Deep Conceptual Explanation of How Transformers Encode Order Without Recurrence

Part 1 - Why this matters

The final exercise synthesizes the chapter into an explanation of how a model without recurrent state can still reason about sequence. A Transformer does not experience tokens one after another during parallel training. It receives a collection of representations together with structures that distinguish positions and regulate information flow. Order is encoded in coordinates, relative relations, rotations, masks, and learned transformations rather than in the physical order of a loop.

The explanation must begin from the symmetry of content-only self-attention. Because permuting tokens permutes token outputs, the architecture needs a symmetry-breaking signal for order-sensitive tasks. Absolute encodings attach slot identity. Relative biases attach distance and direction to query–key pairs. RoPE turns position into phase so score geometry depends on displacement. Causal masks create an oriented access graph in which the future is forbidden. These mechanisms encode different aspects of sequence and may be combined.

Clarify parallelism. During teacher-forced training, all known token positions can receive their positions and causal mask at once, so the layer processes the sequence through batched matrix operations. During autoregressive generation, the next token remains unknown until the previous output is sampled. Cached keys and values preserve past computation, while the new query receives an offset corresponding to its global position. The architecture is non-recurrent at the layer level, but generation still follows a causal sequence of decisions.

The explanation should include limits. A positional formula being defined at a large index does not guarantee extrapolation. Padding can corrupt alignment, reduced precision can erase phase distinctions, learned tables can bind models to maximum lengths, relative buckets can collapse distant relations, and dense attention remains quadratic. Order representation is necessary, but it does not by itself produce understanding or long-context reliability.

Use evidence from the complete chapter and connect it to the next architectural stage. Positional information becomes useful only through attention projections, residual pathways,

normalization, feed-forward transformations, and training. The next chapters will assemble these components into complete Transformer blocks.

Part 2 - Mathematical form and reasoning

Content-only self-attention satisfies

$$\mathrm{Attn}(PX) = P\,\mathrm{Attn}(X).$$

Order is introduced by position-dependent transformations such as

$$X'_p = X_p + E_p,$$

relative score terms

$$S_{i,j} = \frac{q_i^\mathsf{T} k_j}{\sqrt{d_h}} + b_{j-i},$$

or rotary transformations

$$\widetilde{q}_i = R_i q_i, qquad \widetilde{k}_j = R_j k_j, qquad \widetilde{q}_i^\mathsf{T} \widetilde{k}_j = q_i^\mathsf{T} R_{j-i} k_j.$$

Causal order is enforced by

$$M_{i,j} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i. \end{cases}$$

Together these structures make sequence relations available without a hidden state transition $h_t = f(x_t, h_{t-1})$.

Part 3 - Hints, controls, and evidence

1. Organize the explanation around symmetry, coordinate construction, information access, parallel training, cached generation, and limitations.
2. Use the permutation proof and order-sensitive toy task to establish necessity rather than relying on analogy.
3. Compare absolute, sinusoidal, learnable, relative, and rotary approaches by the kind of order each makes easy to represent.
4. Distinguish positional identity from causal permission and training parallelism from sequential token generation.
5. Include extrapolation, padding, precision, memory, and runtime evidence as parts of the real architecture.
6. End with open questions about how full Transformer blocks preserve, transform, or lose positional information across depth.

Part 4 - Python code skeleton

```
from pathlib import Path

def load_chapter_evidence(directory):
    # TODO: load verified artifacts from E00--E18
    return ...

def conceptual_claims(evidence):
    return {
        "content_attention_symmetry": ...,
        "absolute_position": ...,
        "relative_position": ...,
        "rotary_geometry": ...,
        "causal_access": ...,
        "parallel_training": ...,
        "cached_generation": ...,
        "limits": ...,
    }

def E19(artifact_directory, output_path):
    evidence = load_chapter_evidence(artifact_directory)
    claims = conceptual_claims(evidence)
    assert all(item["supported"] for item in claims.values())
    # TODO: write a deep evidence-linked conceptual explanation
    Path(output_path).write_text(..., encoding="utf-8")
    return {"output": str(output_path), "claims": claims}
```

Part 5 - The Philosophy Behind the Exercise

Recurrence represents time as succession: the past is carried into the present through state. The Transformer represents order as structure: positions and permissions are available simultaneously, and attention constructs relations within that structured field. These are different computational metaphors for history.

Neither metaphor is time itself. Both are engineered representations that preserve selected distinctions and discard others. Absolute location, relative distance, causal direction, and phase are ways of making sequence legible to a machine.

The deeper achievement of the Transformer is not that it escaped order, but that it separated order from sequential execution. This separation aligned sequence modeling with parallel hardware while creating new responsibilities: coordinates must be designed, masks must be correct, numerical phase must remain stable, and extrapolation must be proved. Intelligence at scale emerged from this negotiation between representation, mathematics, and machines.

17.5 TRF03

Chapter Title: Transformer Block: Residuals, LayerNorm, and FFN
Specialization: Models and Training

17.5.1 Exercise 00

Exercise Title: Implement a Full Transformer Encoder Block from Scratch with Residual Connections and LayerNorm

Part 1 - Why this matters

A Transformer encoder block is the first point at which the earlier mechanisms become one trainable architecture. Multi-head self-attention mixes information across sequence positions. A position-wise feed-forward network transforms each position independently. Residual paths preserve a direct route for the existing representation, while LayerNorm controls feature statistics. Dropout adds stochastic regularization. Correctness depends not only on having these components, but on composing them in the intended order.

Implement the block without using `nn.TransformerEncoderLayer`. The scratch version should expose projected attention tensors, attention weights, residual inputs, normalized states, FFN hidden activations, and final outputs. This visibility is required because a model can preserve the external shape `(B,T,d_model)` while normalizing the wrong axis, applying dropout at the wrong location, or omitting a residual branch.

Choose and document a normalization convention. In a Pre-LN block, the input is normalized before each sublayer and the residual receives the sublayer output. In a Post-LN block, the sublayer operates on the current state and the sum is normalized afterward. This exercise may implement Pre-LN as the default, but the code should make the convention explicit rather than mixing both.

Validation should progress from identity interventions to library comparison. Set attention and FFN outputs to zero and prove that a Pre-LN residual block returns its input. Replace sublayers with simple known functions and calculate the result manually. Then match weights and settings with a PyTorch reference, checking forward values, masks, and gradients in evaluation mode.

Part 2 - Mathematical form and reasoning

One Pre-LN encoder block can be written

$$U = X + \mathrm{Dropout}(\mathrm{MHA}(\mathrm{LN}_1(X))),$$

$$Y = U + \mathrm{Dropout}(\mathrm{FFN}(\mathrm{LN}_2(U))).$$

The position-wise FFN is

$$\mathrm{FFN}(u) = W_2\phi(W_1u + b_1) + b_2,$$

with $W_1 \in \mathbb{R}^{d_{\mathrm{ff}} \times d}$ and $W_2 \in \mathbb{R}^{d \times d_{\mathrm{ff}}}$. Every residual addition requires

$$\mathrm{shape}(X) = \mathrm{shape}(\mathrm{MHA}(\cdot)) = \mathrm{shape}(\mathrm{FFN}(\cdot)) = (B, T, d).$$

Part 3 - Hints, controls, and evidence

1. Write a tensor contract for every component before implementing the forward pass.

2. Keep attention and FFN residual dropout as distinct modules so their behavior can be inspected independently.

3. Test identity interventions, traceable linear sublayers, padding masks, and asymmetric batch/sequence/model dimensions.

4. Verify LayerNorm acts over the final feature dimension for every token independently.

5. Match the scratch block with a library block using identical weights, activation, epsilon, dropout state, mask convention, and normalization order.

6. Compare input and parameter gradients as well as forward outputs.

Part 4 - Python code skeleton

```
import torch
from torch import nn

class ScratchEncoderBlock(nn.Module):
    def __init__(self, d_model, heads, d_ff, dropout=0.1):
        super().__init__()
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.attention = ...
        self.ffn = nn.Sequential(
            nn.Linear(d_model, d_ff),
            nn.GELU(),
            nn.Linear(d_ff, d_model),
        )
        self.attn_dropout = nn.Dropout(dropout)
        self.ffn_dropout = nn.Dropout(dropout)

    def forward(self, x, mask=None, return_debug=False):
        normalized1 = self.norm1(x)
        attended, weights = self.attention(
            normalized1, normalized1, normalized1, mask
        )
        u = ...  # residual 1
        normalized2 = self.norm2(u)
        transformed = self.ffn(normalized2)
        y = ...  # residual 2
        if return_debug:
            return y, {"weights": weights, "u": u,
                       "normalized1": normalized1,
                       "normalized2": normalized2,
                       "ffn_output": transformed}
        return y

def E00():
```

```
block = ScratchEncoderBlock(16, 4, 64, dropout=0.0)
x = torch.randn(2, 5, 16)
# TODO: identity, manual, mask, library-parity, and gradient tests
return block(x, return_debug=True)
```

Part 5 - The Philosophy Behind the Exercise

The Transformer block is an architecture of transformation and preservation. Attention and the FFN propose changes; residual paths keep the previous state available; normalization shapes the conditions under which change is computed. Learning occurs through the negotiation among these roles.

No component alone explains the block's success. Attention without a stable residual pathway can be difficult to optimize. Residuals without expressive sublayers preserve but do not enrich. Normalization without a meaningful transformation standardizes an empty process.

The deeper lesson is compositional: intelligence in a network arises not only from powerful operations, but from interfaces that let information survive, interact, and change across depth.

17.5.2 Exercise 01

Exercise Title: Implement Feed-Forward Network (FFN) Sub-Layer and Explain Expansion Ratio and Non-Linearity Role

Part 1 - Why this matters

Self-attention moves information among positions, but its weighted value mixing does not replace a rich transformation at each position. The FFN applies the same two-layer network independently to every token. It expands the model width to a larger hidden dimension, applies a non-linearity, and projects back to the residual width. The shared parameters preserve sequence-length flexibility while allowing each token representation to be transformed deeply.

The expansion ratio $r = d_{\text{ff}}/d_{\text{model}}$ controls a major part of Transformer parameter count and compute. A larger hidden width provides more intermediate features but increases both dense-matrix work and activation memory. The common ratio of four is a historical design choice, not a law. Compare several ratios under matched attention and training settings.

Non-linearity is essential. Without it, the two linear transformations collapse into one affine map and expansion does not create a more expressive function. Compare ReLU and GELU, inspect activation sparsity and distribution, and include a deliberately linear variant to prove the collapse. Modern gated FFNs such as SwiGLU may be noted as extensions, but the basic two-layer mechanism should be fully understood first.

Verify that the same FFN function is applied at each position. Process a token alone and inside different sequence contexts after fixing its input vector; the FFN-only result should be identical. Context changes occur in attention, while the FFN performs a shared local transformation.

Part 2 - Mathematical form and reasoning

For token state $x \in \mathbb{R}^d$,

$$h = \phi(W_1 x + b_1), \qquad y = W_2 h + b_2,$$

where

$$W_1 \in \mathbb{R}^{d_{\text{ff}} \times d}, \qquad W_2 \in \mathbb{R}^{d \times d_{\text{ff}}}.$$

Ignoring biases, parameter count is

$$N_{\text{FFN}} = 2dd_{\text{ff}} = 2rd^2.$$

For a batch of sequences, dense work is approximately

$$\mathcal{O}(BTdd_{\text{ff}}).$$

If ϕ is the identity, then

$$y = W_2 W_1 x + (W_2 b_1 + b_2),$$

which is one affine transformation despite the expanded intermediate width.

Part 3 - Hints, controls, and evidence

1. Implement the FFN on a single vector, then prove vectorized batch and sequence execution matches independent token execution.
2. Compare expansion ratios under fixed dataset, attention, optimizer budget, seeds, and stopping rule.
3. Report parameters, FLOPs estimate, activation memory, latency, and task quality.
4. Compare identity activation, ReLU, and GELU while inspecting output and gradient distributions.
5. Verify the final projection restores d_{model} for residual addition.
6. Inject an accidental activation after the second projection and document how it changes residual behavior.

Part 4 - Python code skeleton

```
import torch
from torch import nn

class PositionWiseFFN(nn.Module):
    def __init__(self, d_model, d_ff, activation=nn.GELU,
                 dropout=0.0):
        super().__init__()
        self.up = nn.Linear(d_model, d_ff)
        self.activation = activation()
        self.dropout = nn.Dropout(dropout)
        self.down = nn.Linear(d_ff, d_model)
```

```
    def forward(self, x, return_hidden=False):
        hidden = self.activation(self.up(x))
        output = self.down(self.dropout(hidden))
        return (output, hidden) if return_hidden else output

def E01():
    reports = {}
    for ratio in [1, 2, 4, 8]:
        ffn = PositionWiseFFN(32, ratio * 32)
        # TODO: token independence, parameter, speed, memory,
        # activation, gradient, and controlled quality tests
        reports[ratio] = ...
    return reports
```

Part 5 - The Philosophy Behind the Exercise

Attention gathers context; the FFN interprets the gathered state. One changes who can communicate, while the other changes what the resulting representation can become. Their alternation is a rhythm of relation and transformation.

Expansion creates a temporary space larger than the shared residual language. The model can form many intermediate features and then compress their useful combination back into the common stream. Capacity is borrowed and returned.

Non-linearity marks the difference between rearranging coordinates and creating new decision boundaries. Depth becomes meaningful only when composition cannot be algebraically collapsed into one linear map.

17.5.3 Exercise 02

Exercise Title: Debug LayerNorm Placement and Compare Pre-LN vs Post-LN Stability Differences in Training

Part 1 - Why this matters

LayerNorm placement changes the path followed by activations and gradients through a deep Transformer. Pre-LN normalizes before a sublayer and leaves an unmodified identity residual path. Post-LN applies the sublayer, adds the residual, and then normalizes the sum. Both preserve external shape, so an incorrect implementation may train while behaving like a different architecture.

Implement both forms with separate classes or an explicit configuration. Do not reuse a forward pass whose conditional branches obscure the equation. Copy all compatible attention and FFN weights between variants, fix dropout and seeds, and compare shallow and deep stacks on the same task. Measure loss curves, gradient norm by depth, activation statistics, divergence frequency, and sensitivity to learning rate.

Pre-LN often improves optimization stability in deep stacks because the identity residual path carries gradients without passing through a normalization Jacobian at every block. Post-LN may produce different representation and final normalization behavior and can work well with suitable initialization, warmup, or residual scaling. The exercise should test these tendencies rather than turning them into universal claims.

Debug the axis as well as the location. Standard Transformer LayerNorm normalizes the final feature dimension independently for every batch and token. Normalizing sequence or batch axes changes the model's dependencies and can leak information across padding or examples.

Part 2 - Mathematical form and reasoning

For sublayer F, Pre-LN is

$$y = x + F(\mathrm{LN}(x)),$$

while Post-LN is

$$y = \mathrm{LN}(x + F(x)).$$

LayerNorm over d features is

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i, qquad\sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2,$$

$$\mathrm{LN}(x)_i = \gamma_i \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_i.$$

The residual Jacobian for Pre-LN contains an explicit identity term,

$$\frac{\partial y}{\partial x} = I + J_F J_{\mathrm{LN}},$$

which helps explain a direct gradient route without guaranteeing stability in every configuration.

Part 3 - Hints, controls, and evidence

1. Write both equations beside the corresponding forward code and capture intermediate tensors.
2. Verify LayerNorm statistics per token and confirm padding or other batch items do not affect a valid token's normalization.
3. Copy shared weights, match optimizer and schedule, and compare several depths, learning rates, and seeds.
4. Log per-layer activation mean/std, residual norm, gradient norm, loss, and non-finite events.
5. Use dropout zero for exact parity tests and restore it only for training comparisons.
6. Inject a misplaced norm and retain the earliest distinguishing invariant as a regression test.

Part 4 - Python code skeleton

```
import torch

class PreLNBlock(...):
    def forward(self, x, mask=None):
        x = x + self.attention(self.norm1(x), mask)
        x = x + self.ffn(self.norm2(x))
        return x

class PostLNBlock(...):
    def forward(self, x, mask=None):
        x = self.norm1(x + self.attention(x, mask))
        x = self.norm2(x + self.ffn(x))
        return x

def layer_diagnostics(model):
    # TODO: hooks for activations, residuals, and gradients
    return ...

def E02(depths=(2, 8, 24), seeds=range(5)):
    # TODO: matched trials across norm style, depth, and LR
    return ...
```

Part 5 - The Philosophy Behind the Exercise

Normalization placement asks whether transformation occurs before or after a state is standardized. The same named parts form different dynamical systems when their order changes. Architecture includes sequence, not merely inventory.

Residual paths preserve continuity across depth, while normalization continually renegotiates scale. Pre-LN protects a direct identity route; Post-LN insists that each updated state re-enter a standardized coordinate system.

The comparison teaches that stability is produced by relationships among components. No layer is "stable" in isolation. Its effect depends on where it sits, what it surrounds, and how gradients must travel through the whole stack.

17.5.4 Exercise 03

Exercise Title: Validate Residual Connection Shapes and Debug Silent Broadcasting Bugs in Add Operations

Part 1 - Why this matters

Residual addition is intended to combine two representations of the same batch, token, and feature locations. PyTorch broadcasting can make incompatible semantic tensors add successfully. A sublayer output shaped `(B,1,d)` may broadcast across all tokens, or `(1,T,d)` across all examples. The result has the expected final shape but repeats information that should have

been position- or example-specific.

Never use successful addition as proof of residual compatibility. Require exact shape equality before every residual merge unless broadcasting is explicitly part of the design. Name axes and test unequal values for batch, sequence, and feature dimensions. Construct traceable tensors so an unintended repeated row is obvious.

Residuals also require semantic compatibility. Attention and FFN outputs must return to d_{model} and preserve batch/sequence alignment. A transposed `(T,B,d)` output may remain addable when $B = T$, hiding a layout bug. Primary tests should deliberately use $B \neq T$.

Inject silent failures: pooled attention output, dropped batch axis, wrong transpose, singleton feature output, and accidental bias-like tensor. The block should reject them at the residual boundary with an informative assertion and retain each as a regression test.

Part 2 - Mathematical form and reasoning

The intended operation is elementwise

$$Y_{b,t,f} = X_{b,t,f} + F(X)_{b,t,f}.$$

Therefore the contract is

$$\text{shape}(X) = \text{shape}(F(X)) = (B, T, d).$$

Broadcast addition with $Z \in \mathbb{R}^{B\times 1\times d}$ instead computes

$$Y_{b,t,f} = X_{b,t,f} + Z_{b,1,f}$$

for every t, a different function. Equal output shape does not imply equal index dependence.

Part 3 - Hints, controls, and evidence

1. Implement a strict residual helper that asserts equal shape, device, and dtype before addition.
2. Use asymmetric B, T, and d plus coordinate-encoded values.
3. Verify attention and FFN outputs separately before residual merging.
4. Test batch-first and sequence-first paths with explicit conversion at one documented boundary.
5. Inject singleton, transpose, pooled, and missing-axis failures and check that each is rejected immediately.
6. Add gradient checks to confirm every token receives its own residual identity path.

Part 4 - Python code skeleton

```
import torch

def strict_residual(x, update, name):
    assert x.shape == update.shape, (
        f"{name}: residual shape {x.shape} != update {update.shape}"
    )
    assert x.device == update.device
    assert x.dtype == update.dtype
    return x + update

def coordinate_tensor(batch, seq, width):
    b = torch.arange(batch)[:, None, None] * 10000
    t = torch.arange(seq)[None, :, None] * 100
    f = torch.arange(width)[None, None, :]
    return (b + t + f).float()

def E03():
    x = coordinate_tensor(2, 5, 8)
    correct = x * 0.1
    y = strict_residual(x, correct, "attention")
    # TODO: failure injection and per-token gradient identity tests
    return y
```

Part 5 - The Philosophy Behind the Exercise

Residual addition asserts identity: these two tensors describe the same locations in the same representational language. Broadcasting can satisfy arithmetic while violating that identity.

Convenience features are most dangerous when they transform errors into valid programs. Broadcasting is powerful when sharing is intended and deceptive when sharing is accidental. Explicit contracts preserve the distinction.

The exercise teaches that shape is not only size. It is a statement about which entities correspond. Correct addition requires correspondence before it requires arithmetic.

17.5.5 Exercise 04

Exercise Title: Implement Dropout in Transformer Block and Verify It Applies Only During Training Mode

Part 1 - Why this matters

Dropout changes a Transformer block stochastically during training and becomes an identity operation during evaluation. Applying it in evaluation makes predictions random; failing to apply it in training removes intended regularization. Because dropout may appear on attention probabilities, FFN hidden activations, and residual branch outputs, each location has a different effect and should be documented.

This exercise uses dropout on sublayer outputs before residual addition and optionally inside the FFN. Attention-probability dropout belongs inside the attention module and changes

which value contributions are retained. Residual dropout changes the proposed update while preserving the identity path. Do not create `nn.Dropout` inside `forward`; register modules so `train()` and `eval()` control them correctly.

Test statistical behavior rather than expecting a particular random mask. Across many training passes, outputs should vary and the mean should approximate the no-dropout expectation because inverted dropout rescales retained values. Across repeated evaluation passes, outputs should be exactly or numerically identical. Use fixed seeds only when verifying a known mask or reproducibility.

Also verify gradients. Dropped elements have zero local gradient for that pass, while retained elements receive scaled gradients. The residual identity path still permits gradients to reach the input. Compare dropout locations on a toy task, but keep the correctness tests independent of claims about generalization.

Part 2 - Mathematical form and reasoning

For keep probability $q = 1 - p$ and Bernoulli mask $m_i \sim \text{Bernoulli}(q)$, training dropout is

$$\text{Drop}(z_i) = \frac{m_i}{q} z_i.$$

Therefore

$$\mathbb{E}[\text{Drop}(z_i)] = z_i.$$

During evaluation,

$$\text{Drop}(z) = z.$$

For residual update,

$$y = x + \text{Drop}(F(x)),$$

the identity branch remains deterministic even when the proposed update is random during training.

Part 3 - Hints, controls, and evidence

1. Register separate dropout modules for attention residual, FFN hidden, and FFN residual locations.
2. Run repeated passes in `train()` and `eval()` and compare variance and exact repeatability.
3. Use a large constant tensor to estimate zero fraction and mean preservation statistically.
4. Set $p = 0$ and a high but legal p as boundary tests; reject $p = 1$ if the chosen API disallows it.
5. Inspect gradients through dropped updates and the residual identity path.
6. Ensure validation, generation, and benchmarking call `eval()` and restore prior mode when appropriate.

Part 4 - Python code skeleton

```
import torch
from torch import nn

class ResidualDropout(nn.Module):
    def __init__(self, probability):
        super().__init__()
        self.dropout = nn.Dropout(probability)

    def forward(self, residual, update):
        assert residual.shape == update.shape
        return residual + self.dropout(update)

def mode_report(module, residual, update, repeats=200):
    module.train()
    train_outputs = torch.stack([module(residual, update)
                                 for _ in range(repeats)])
    module.eval()
    eval_outputs = torch.stack([module(residual, update)
                                for _ in range(10)])
    return {"train_variance": ...,
            "train_mean_error": ...,
            "eval_repeat_error": ...}

def E04():
    # TODO: location-specific, mode, seed, boundary, and gradient tests
    return ...
```

Part 5 - The Philosophy Behind the Exercise

Dropout creates temporary absence so the network cannot rely too completely on one computational route. During training, the architecture becomes a changing family of subnetworks; during evaluation, their learned expectation is used as one deterministic system.

Residual paths make this absence less destructive by preserving continuity. The model experiments with missing updates without losing the state from which those updates began.

The train/eval distinction also shows that a model is not one static function throughout its lifecycle. Its behavior depends on mode, and correctness requires the surrounding pipeline to state which mode the current purpose demands.

17.5.6 Exercise 05

Exercise Title: Implement Stacking of Multiple Encoder Blocks and Track How Representation Changes by Depth

Part 1 - Why this matters

One encoder block performs one round of contextual mixing and token-wise transformation. Stacking blocks allows representations to be repeatedly reconstructed, but depth also creates optimization, memory, and interpretability challenges. This exercise builds a stack with `nn.ModuleList`, verifies that parameters are independently registered, and instruments the model to observe change at every depth.

Do not accidentally reuse the same block object unless parameter sharing is the intent. Appending one instance repeatedly creates tied weights; deep copying or constructing each block separately creates independent layers. Test parameter identities and modify one layer to prove others do not change.

Track activation norm, token-wise mean and variance, residual-update ratio, cosine similarity between consecutive layers, attention entropy, and gradient norm. High consecutive similarity may indicate small updates, but does not alone prove a layer is useless. Probe-task quality or layer ablation provides stronger functional evidence. Compare initialized and trained checkpoints.

Depth experiments must control width, data, optimization budget, and seeds. Deeper models have more parameters and may require different learning-rate or warmup settings, so distinguish a direct controlled comparison from a tuned best-case comparison. Also measure activation memory and runtime growth.

Part 2 - Mathematical form and reasoning

For L blocks,

$$X^{(\ell+1)} = F_\ell(X^{(\ell)}), \qquad \ell = 0, \dots, L-1.$$

Representation change can be summarized by

$$R_\ell = \frac{\|X^{(\ell+1)} - X^{(\ell)}\|_F}{\max(\|X^{(\ell)}\|_F, \varepsilon)}$$

and cosine similarity

$$C_\ell = \frac{\langle X^{(\ell+1)}, X^{(\ell)} \rangle}{\|X^{(\ell+1)}\|_F \|X^{(\ell)}\|_F}.$$

For gradient $g^{(\ell)} = \partial \mathcal{L} / \partial X^{(\ell)}$, record $\|g^{(\ell)}\|_2$ across depth and training time.

Part 3 - Hints, controls, and evidence

1. Construct independent blocks and assert parameters from different layers do not share storage.
2. Return intermediate states only in debug mode to avoid retaining large graphs unintentionally.
3. Log activation, update, attention, and gradient statistics on a fixed probe set at selected checkpoints.
4. Compare depths under controlled settings and separately under tuned settings.

5. Measure parameters, peak activation memory, forward/backward latency, and held-out quality.

6. Ablate or replace selected layers with identity to test whether observed representation change is functionally important.

Part 4 - Python code skeleton

```
import torch
from torch import nn

class EncoderStack(nn.Module):
    def __init__(self, block_factory, depth, final_norm=None):
        super().__init__()
        self.layers = nn.ModuleList([block_factory() for _ in range(depth)])
        self.final_norm = final_norm or nn.Identity()

    def forward(self, x, mask=None, return_states=False):
        states = [x] if return_states else None
        for layer in self.layers:
            x = layer(x, mask)
            if return_states:
                states.append(x)
        x = self.final_norm(x)
        return (x, states) if return_states else x

def depth_statistics(states):
    # TODO: norms, relative updates, cosine similarities, and gradients
    return ...

def E05(depths=(1, 2, 4, 8, 16)):
    # TODO: independence, controlled training, probes, ablations,
    # memory, and runtime
    return ...
```

Part 5 - The Philosophy Behind the Exercise

Depth gives the model time measured in transformations rather than sequence steps. Every block revisits the same token positions with a representation changed by earlier context and computation.

Repeated processing does not guarantee progressive understanding. Layers may refine, preserve, overwrite, or redundantly reproduce information. Observing depth means asking what changes, what survives, and which changes affect behavior.

The residual stream acts as a history running through these transformations. Depth becomes useful when new computations can enter that history without destroying the conditions needed by later layers.

17.5.7 Exercise 06

Exercise Title: Implement Transformer Decoder Block with Masked Self-Attention and Cross-Attention Conceptually

Part 1 - Why this matters

An encoder block contextualizes a source sequence. A classical Transformer decoder block must do three things: model the target prefix through causally masked self-attention, retrieve relevant source information through cross- attention, and transform each target state through an FFN. Separate residual and normalization paths surround all three sublayers.

Masked self-attention uses target representations for queries, keys, and values and forbids future target positions. Cross-attention uses decoder states as queries but encoder outputs as keys and values. This distinction creates different sequence axes: target query length T_y and source memory length T_x. A square toy example can hide reversal, so tests must use unequal lengths.

Implement a Pre-LN conceptual decoder with independent norms for self-attention, cross-attention, and FFN. Pass target causal/padding masks separately from the source padding mask. A common failure is to apply the target mask to source memory or to normalize encoder memory with the decoder's query norm.

Prove information access through interventions. Changing a future target token must not affect earlier decoder outputs. Changing a permitted encoder memory position should affect outputs that attend to it, while changing padded source positions should not. Attention weights should have shapes `(B,H,Ty,Ty)` and `(B,H,Ty,Tx)` respectively.

Part 2 - Mathematical form and reasoning

For target state Y and encoder memory M, a Pre-LN decoder block is

$$U = Y + \mathrm{MaskedMHA}(\mathrm{LN}_1(Y)),$$

$$V = U + \mathrm{CrossMHA}(Q = \mathrm{LN}_2(U), K = M, V = M),$$

$$Z = V + \mathrm{FFN}(\mathrm{LN}_3(V)).$$

Masked self-attention has scores in $\mathbb{R}^{B\times H\times T_y\times T_y}$, while cross-attention has

$$S_{\mathrm{cross}} \in \mathbb{R}^{B\times H\times T_y\times T_x}.$$

The causal constraint is $A_{i,j} = 0$ for $j > i$ on the target self-attention map only.

Part 3 - Hints, controls, and evidence

1. Use unequal source and target lengths and label every query and key axis.
2. Keep target causal mask, target padding mask, and source padding mask as separate arguments with documented boolean conventions.
3. Test future-target intervention, valid-source intervention, and padded- source invariance.

4. Verify three residual shape contracts and three independent LayerNorm modules.
5. Set cross-attention output to zero and prove the remaining block behaves as target-only masked processing.
6. Compare with a library decoder layer only after aligning normalization order, weights, masks, dropout, and batch layout.

Part 4 - Python code skeleton

```
import torch
from torch import nn

class ScratchDecoderBlock(nn.Module):
    def __init__(self, d_model, heads, d_ff, dropout=0.0):
        super().__init__()
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.norm3 = nn.LayerNorm(d_model)
        self.self_attention = ...
        self.cross_attention = ...
        self.ffn = ...
        self.dropout = nn.Dropout(dropout)

    def forward(self, target, memory, target_mask=None,
                memory_mask=None, return_weights=False):
        q = self.norm1(target)
        self_out, self_weights = self.self_attention(q, q, q, target_mask)
        u = ...
        cross_q = self.norm2(u)
        cross_out, cross_weights = self.cross_attention(
            cross_q, memory, memory, memory_mask
        )
        v = ...
        z = ...
        return (z, self_weights, cross_weights) if return_weights else z

def E06():
    # TODO: shape, causal, source-padding, intervention,
    # identity, library, and gradient tests
    return ...
```

Part 5 - The Philosophy Behind the Exercise

The decoder separates two forms of memory. Its own prefix is a history governed by causality; encoder output is an external field of source information. Self- attention and cross-attention answer different questions about these memories.

Generation therefore combines continuity and retrieval. The target state grows through its prior decisions while repeatedly consulting a representation that was constructed from the source.

The masks define an ethics of access inside the computation: what the decoder may know now, what remains future, and which stored positions are not real. Correct sequence generation begins with enforcing those boundaries.

17.5.8 Exercise 07

Exercise Title: Build a Minimal Encoder-Only Transformer Classifier and Validate Output Shapes and Loss Computation

Part 1 - Why this matters

An encoder-only classifier connects tokenization and positional information to a stack of encoder blocks and a task-specific decision. The architecture must turn variable-length token sequences into one representation without allowing padding to influence attention or pooling. This exercise builds the smallest complete classifier whose data, shapes, logits, targets, and loss can be verified end to end.

Use either a dedicated classification token or masked mean pooling and document the choice. A classification token provides a learned slot that attends to the sequence. Masked mean pooling treats valid token outputs symmetrically and must divide by the valid-token count rather than padded length. A plain mean over all storage positions creates length-dependent padding leakage.

The classifier head should output raw logits. For mutually exclusive classes, the cross-entropy loss expects shape `(B,C)` and integer targets `(B)`; applying softmax before this loss is unnecessary and can reduce numerical stability. Validate a manual log-softmax calculation on a tiny batch and confirm label indices lie in range.

Overfit a tiny dataset deliberately. Near-perfect fit is a high-value correctness test because it exercises embeddings, positions, masks, encoder blocks, pooling, classifier, loss, backward, and optimizer together. If the model cannot memorize a few examples, investigate the pipeline before tuning generalization.

Part 2 - Mathematical form and reasoning

Given token and position embeddings,

$$X^{(0)} = E_{\text{token}}(t) + E_{\text{position}}(p), \qquad X^{(L)} = \text{EncoderStack}(X^{(0)}, M).$$

Masked mean pooling is

$$z_b = \frac{\sum_{t=1}^{T} m_{b,t} X^{(L)}_{b,t,:}}{\max(1, \sum_t m_{b,t})}.$$

Logits are $\ell_b = W z_b + b$, and cross-entropy is

$$\mathcal{L} = -\frac{1}{B} \sum_{b=1}^{B} \log \frac{e^{\ell_{b,y_b}}}{\sum_{c=1}^{C} e^{\ell_{b,c}}}.$$

The stable library form operates directly on logits.

Part 3 - Hints, controls, and evidence

1. Record shapes for tokens, masks, embeddings, every block, pooled state, logits, labels, and scalar loss.
2. Test multiple batch sizes and sequence lengths, including padding and one valid token.
3. Change padded token IDs dramatically and prove valid logits remain unchanged.
4. Compare manual cross-entropy with the library loss on a tiny float64 example.
5. Verify every expected parameter receives finite gradients.
6. Overfit a tiny fixed dataset before measuring held-out performance.

Part 4 - Python code skeleton

```
import torch
from torch import nn

class EncoderClassifier(nn.Module):
    def __init__(self, vocab, d_model, depth, classes):
        super().__init__()
        self.token = nn.Embedding(vocab, d_model)
        self.position = ...
        self.encoder = ...
        self.classifier = nn.Linear(d_model, classes)

    def forward(self, token_ids, attention_mask):
        x = self.position(self.token(token_ids))
        x = self.encoder(x, mask=attention_mask)
        valid = attention_mask[..., None].to(x.dtype)
        pooled = (x * valid).sum(dim=1) / valid.sum(dim=1).clamp_min(1)
        return self.classifier(pooled)

def E07():
    model = ...
    tokens, mask, labels = ...
    logits = model(tokens, mask)
    loss = nn.CrossEntropyLoss()(logits, labels)
    # TODO: manual loss, padding intervention, gradients, tiny overfit
    return logits, loss
```

Part 5 - The Philosophy Behind the Exercise

Classification compresses a structured sequence into one decision. The encoder preserves many token-level relations, while pooling and the head decide which summary is sufficient for

the task.

This compression is purposeful loss. A classifier does not need to reconstruct every detail, but it must retain distinctions relevant to its labels. Pooling is therefore not a neutral ending; it defines what evidence can survive.

The tiny-overfit test embodies humility. Before asking whether a model discovers general truth, we should prove that the complete mechanism can learn a small world whose answer is known.

17.5.9 Exercise 08

Exercise Title: Build a Minimal Decoder-Only Autoregressive Transformer and Validate Causal Mask Behavior Precisely

Part 1 - Why this matters

A decoder-only Transformer predicts each next token from the prefix available before it. During training, all target positions are processed in parallel, so a causal mask must prevent each query from accessing future tokens. A single mask error can produce deceptively low loss by leaking the answer. This exercise builds a minimal language model and proves its information boundary through intervention rather than visual inspection alone.

Inputs and targets are shifted: input position t contains token x_t, while the target is x_{t+1}. The model emits logits `(B,T,V)` and cross-entropy flattens matching token and vocabulary axes. Padding targets must use an ignored index or an explicit loss mask. Off-by-one alignment can make the model predict the current token instead of the next one.

For causal validation, create two sequences identical through position i and different afterward. Logits at positions up to i must be identical in evaluation mode. Future attention weights must be zero, legal rows must sum to one, and gradients of an early-position loss with respect to future token embeddings must be zero. These independent checks detect different failure classes.

Compare full-sequence evaluation with cached incremental decoding. With dropout disabled and positions offset correctly, the logits for each newly appended token should match the corresponding full forward pass within tolerance.

Part 2 - Mathematical form and reasoning

Autoregressive factorization is

$$p(x_{1:T}) = \prod_{t=1}^{T} p(x_t \mid x_{<t}).$$

The causal mask is

$$M_{i,j} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i. \end{cases}$$

and next-token loss is

$$\mathcal{L} = -\frac{1}{N_{\text{valid}}} \sum_{b,t \text{ valid}} \log p_\theta(x_{b,t+1} \mid x_{b,\le t}).$$

Causality requires the Jacobian condition

$$\frac{\partial \ell_{b,i,:}}{\partial X_{b,j,:}} = 0 \quad \text{for } j > i.$$

Part 3 - Hints, controls, and evidence

1. Print input/target pairs and verify shift manually before training.
2. Combine causal and padding masks with explicit axis semantics.
3. Test attention zeros, row sums, future-token intervention, and future gradient zeros.
4. Use evaluation mode and dropout zero for deterministic causal tests.
5. Compare full and cached logits using correct global position offsets.
6. Include an intentionally missing mask and prove the test detects leakage and artificially improved training loss.

Part 4 - Python code skeleton

```
import torch
from torch import nn

class DecoderOnlyLM(nn.Module):
    def __init__(self, vocab, d_model, depth):
        super().__init__()
        self.token = nn.Embedding(vocab, d_model)
        self.position = ...
        self.blocks = ...
        self.final_norm = nn.LayerNorm(d_model)
        self.lm_head = nn.Linear(d_model, vocab, bias=False)

    def forward(self, tokens, padding_mask=None, cache=None):
        # TODO: positions, combined mask, blocks, optional cache
        return ...

def future_intervention(model, prefix, suffix_a, suffix_b, boundary):
    logits_a = model(torch.cat([prefix, suffix_a], dim=1))
    logits_b = model(torch.cat([prefix, suffix_b], dim=1))
    return (logits_a[:, :boundary + 1] -
            logits_b[:, :boundary + 1]).abs().max()

def E08():
    # TODO: alignment, masks, intervention, Jacobian, tiny overfit,
    # and cached parity tests
    return ...
```

Part 5 - The Philosophy Behind the Exercise

A causal model is defined as much by what it cannot know as by what it can compute. The mask creates an epistemic boundary: prediction must be formed from the past rather than copied from the future.

Parallel training does not abolish time. It makes all historical viewpoints available for simultaneous computation while preserving a different permitted past for each position.

The intervention test turns causality into an operational claim. If changing the future changes the past prediction, the architecture has violated the meaning of its task, regardless of how attractive its loss appears.

17.5.10 Exercise 09

Exercise Title: Debug NaNs in Transformer Block by Inspecting Softmax, LayerNorm, and Mixed Precision Settings

Part 1 - Why this matters

NaNs are late symptoms that may originate in attention logits, all-masked softmax rows, LayerNorm variance, FFN activations, loss scaling, optimizer states, or earlier data. A useful debugging process records the first non-finite tensor in execution order rather than inspecting only the final loss. This exercise builds hooks and assertions around every critical stage of a Transformer block.

Softmax failures can begin when QK products overflow or every key in a row is masked. Stable softmax prevents exponent overflow from finite logits but cannot repair infinities already produced by matrix multiplication. LayerNorm includes epsilon to avoid division by zero, yet reduced precision and extremely large activations can still produce invalid mean or variance. FFN expansion can amplify scale before a nonlinear activation.

Mixed precision adds dynamic loss scaling and dtype-specific limits. Log actual dtypes, autocast boundaries, scale values, skipped optimizer steps, and gradient finiteness. Repeat the failing batch in float32 and float64 where feasible. If the failure disappears, precision is implicated but the originating magnitude still needs explanation.

Inject failures deliberately: huge Q/K scale, all-masked row, zero epsilon in a custom norm, extreme FFN weights, high learning rate, and missing gradient unscale before clipping. Each injected case should be caught at its earliest stage and preserved as a diagnostic regression test.

Part 2 - Mathematical form and reasoning

Attention scores are

$$S = QK^{\mathsf{T}}/\sqrt{d_h} + M.$$

Require finite Q, K, unmasked scores, normalized weights, and outputs. LayerNorm computes

$$\widehat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}.$$

Track denominator minimum and activation maximum. For every stage Z_k, define

$$k^* = \min\{k : \neg\,\text{finite}(Z_k)\}.$$

The first non-finite stage k^* is the primary diagnostic target; later NaNs are propagation evidence rather than independent causes.

Part 3 - Hints, controls, and evidence

1. Validate raw inputs and targets before the model, then instrument Q/K/V, logits, masks, weights, norms, FFN activations, loss, gradients, and optimizer states.
2. Record min, max, mean, std, finite fraction, dtype, device, and shape.
3. Save the exact failing batch, model state, optimizer state, seed, and AMP scale before modifying anything.
4. Reproduce in evaluation, float32, reduced precision, and anomaly-detection modes as appropriate.
5. Stop at the earliest non-finite stage and test one causal change at a time.
6. Turn every root cause and correction into an automated failure-injection test.

Part 4 - Python code skeleton

```
import torch

def tensor_health(name, x):
    detached = x.detach()
    return {"name": name, "shape": tuple(x.shape),
            "dtype": str(x.dtype), "device": str(x.device),
            "finite": bool(torch.isfinite(x).all()),
            "min": detached.nan_to_num().min().item(),
            "max": detached.nan_to_num().max().item()}

class FiniteGuard:
    def __init__(self):
        self.records = []
    def check(self, name, tensor):
        record = tensor_health(name, tensor)
        self.records.append(record)
        if not record["finite"]:
            raise FloatingPointError(record)

def E09(block, failing_batch):
    # TODO: instrument stages and run injected-failure matrix
    return ...
```

Part 5 - The Philosophy Behind the Exercise

A NaN is not a cause; it is the point at which numerical meaning has already collapsed. Debugging must reconstruct the path to the first operation that left the representable or defined domain.

The first-failure principle is a form of causal discipline. Many tensors become corrupted after one invalid state, but treating every corrupted tensor as a separate problem obscures the origin.

Numerical health is part of model knowledge. A network that produces an answer only while its scales remain within hidden limits is not fully understood until those limits are measured and protected.

17.5.11 Exercise 10

Exercise Title: Run Transformer Block Forward on CUDA and Confirm Device Placement for All Parameters and Buffers

Part 1 - Why this matters

Moving a Transformer block to CUDA must move every parameter and registered buffer while inputs and runtime-created tensors follow the same device. Hidden CPU objects often include positional tables not registered as buffers, masks created with default device, scalar constants wrapped as tensors, or caches initialized outside the module. These failures may appear only for particular sequence lengths or optional paths.

Build a device audit that walks named parameters, named buffers, input tensors, masks, intermediate states, outputs, gradients, and caches. Runtime-created indices should use `x.device` or an explicit device argument. Python scalars are generally safe, while newly constructed tensors inherit CPU unless specified.

Compare CPU and CUDA forward outputs from identical master weights and inputs in evaluation mode. Exact equality is not generally required because kernels may accumulate floating-point values differently. Use declared tolerances and check intermediate attention, norm, FFN, and final outputs to locate structural disagreement.

Begin with one GPU and no DDP. Distributed execution would add replication and communication variables before single-device correctness is established. Also test backward gradients and one optimizer step on CUDA, then verify the state dictionary can move back to CPU and reproduce the expected function.

Part 2 - Mathematical form and reasoning

For every parameter θ_i, buffer b_j, input x_k, and runtime mask m_l, require

$$\text{device}(\theta_i) = \text{device}(b_j) = \text{device}(x_k) = \text{device}(m_l) = D.$$

For CPU reference y and CUDA candidate $\widehat{y}$, use

$$|\widehat{y}_i - y_i| \leq \texttt{atol} + \texttt{rtol}|y_i|.$$

Repeat for parameter and input gradients after a shared scalar loss.

Part 3 - Hints, controls, and evidence

1. Skip with an explicit status when CUDA is unavailable; do not silently run a CPU substitute.
2. Audit named parameters and buffers before forward, then hook intermediate outputs and gradients.
3. Generate one master state and input on CPU, clone for CPU, and copy the same values to CUDA.
4. Test masks, positional buffers, causal paths, caches, and multiple sequence lengths.
5. Compare forward, backward, and one optimizer update within declared tolerances across several seeds.
6. Save hardware, PyTorch, CUDA, dtype, deterministic settings, and observed maximum errors.

Part 4 - Python code skeleton

```
import torch

def device_audit(module, expected):
    issues = []
    for name, parameter in module.named_parameters():
        if parameter.device != expected:
            issues.append(("parameter", name, str(parameter.device)))
    for name, buffer in module.named_buffers():
        if buffer.device != expected:
            issues.append(("buffer", name, str(buffer.device)))
    return issues

def paired_path(block, x, mask, device):
    block = block.to(device)
    x = x.to(device).detach().requires_grad_(True)
    mask = mask.to(device) if mask is not None else None
    output = block(x, mask)
    loss = ...
    loss.backward()
    return {
        "output": output.detach().cpu(),
        "input_gradient": x.grad.detach().cpu(),
        "device_issues": device_audit(block, device),
    }

def E10():
    if not torch.cuda.is_available():
        return {"status": "skipped", "reason": "CUDA unavailable"}
    # TODO: paired CPU/CUDA forward, backward, and optimizer checks
    return ...
```

Part 5 - The Philosophy Behind the Exercise

A model is not independent of the place where its tensors exist. Device placement determines which physical memory, kernels, and communication paths realize the computation.

Moving a module is therefore a claim of completeness: every piece of persistent state and every runtime dependency must join the same computational world. One forgotten mask can reveal that the abstraction was incomplete.

Single-device correctness is the foundation of scale. Distribution should multiply a known function, not multiply uncertainty about whether the basic function is already coherent.

17.5.12 Exercise 11

Exercise Title: Benchmark Encoder Block Runtime and Identify Which Sub-Layer Dominates Cost for Long Sequences

Part 1 - Why this matters

An encoder block combines sublayers with different scaling laws. Attention score and value aggregation grow quadratically with sequence length, while FFN work grows linearly with sequence length but quadratically with model width through dense projections. LayerNorm, dropout, and residual addition are lower- order in arithmetic but may become memory-bandwidth or launch-overhead costs at small shapes. The dominant component therefore changes by regime.

Benchmark the complete block and instrument attention, FFN, norms, and residual operations separately. Use the same input already resident on GPU. Warm up, synchronize, repeat, and report median and tail latency. Measure forward inference and forward-plus-backward training separately because activation storage and gradient GEMMs change the cost distribution.

Sweep sequence length, batch, model width, FFN expansion ratio, heads, and dtype one factor at a time. At short sequences and large d, FFN projections may dominate. At long T, dense attention should increasingly dominate unless a memory-efficient fused kernel changes materialization and bandwidth. The measurement should locate the crossover rather than assume it.

Requesting attention weights or debug tensors can disable fusion and retain large intermediates. Profile production-like and observability modes separately. A debugging configuration is not a valid estimate of deployment throughput.

Part 2 - Mathematical form and reasoning

Approximate attention work is

$$C_{\text{attn}} = \mathcal{O}(BTd^2 + BT^2d),$$

including projections and dense pairwise interaction. FFN work is

$$C_{\text{ffn}} = \mathcal{O}(BTdd_{\text{ff}}) = \mathcal{O}(rBTd^2).$$

The rough crossover between pairwise attention and FFN terms occurs when T is on the order of rd, but constants, projection costs, fusion, and hardware efficiency determine the real boundary.

Part 3 - Hints, controls, and evidence

1. Add named timing ranges around norms, attention, residuals, FFN, and final output without changing their values.
2. Benchmark forward and training steps after warm-up using synchronization or CUDA events.
3. Sweep T, B, d, r, heads, and dtype while changing one factor at a time.
4. Record latency, tokens/s, peak allocated and reserved memory, and whether fusion or returned weights were enabled.
5. Compare sublayer sum with full-block time and explain launch, overlap, or measurement overhead.
6. Profile configurations near the observed crossover to verify the hotspot.

Part 4 - Python code skeleton

```
import statistics
import torch

def cuda_time(fn, warmup=20, repeats=100):
    for _ in range(warmup): fn()
    torch.cuda.synchronize()
    samples = []
    for _ in range(repeats):
        start = torch.cuda.Event(True); end = torch.cuda.Event(True)
        start.record(); fn(); end.record(); torch.cuda.synchronize()
        samples.append(start.elapsed_time(end))
    return {"median_ms": statistics.median(samples), "p95_ms": ...}

def benchmark_block(config):
    block, x, mask = ...
    # TODO: full block and instrumented sublayer measurements
    return ...

def E11(configurations):
    if not torch.cuda.is_available(): return {"status": "skipped"}
    return [benchmark_block(c) for c in configurations]
```

Part 5 - The Philosophy Behind the Exercise

An architecture is a distribution of work as well as a distribution of ideas. Attention and FFN may receive equal conceptual prominence while consuming very different resources under

a particular shape.

Dominance is conditional. The bottleneck moves when context, width, batch, precision, or kernel changes. Optimization knowledge therefore has a half-life and must remain attached to the measured regime.

Benchmarking transforms complexity from a slogan into a landscape. The useful question is not "which sublayer is expensive?" but "where, why, and under which workload does each sublayer become expensive?"

17.5.13 Exercise 12

Exercise Title: Implement Mixed Precision Forward and Detect Layers That Are Sensitive to fp16 Instability

Part 1 - Why this matters

Mixed precision accelerates dense operations and reduces activation memory, but a Transformer block contains numerically sensitive reductions and normalization. QK logits may overflow, softmax probabilities may underflow, LayerNorm variance may lose precision, and FFN activations may exceed float16 range. Autocast can promote selected operations, but the researcher must verify actual behavior and not assume every layer is safe.

Run a float32 reference and capture outputs after each norm, attention stage, residual, activation, and FFN projection. Repeat under float16 autocast and bfloat16 where supported. Compare finite values, maximum absolute and relative error, cosine similarity, attention top-index agreement, and output impact. Sweep input and weight scales to locate the first stage that violates a declared invariant.

Forward stability differs from training stability. This exercise emphasizes the forward pass but should retain gradients for a small follow-up and use gradient scaling when an optimizer step is tested. Residual additions may combine a small update with a large state, causing the update to round away even without overflow. Measure update-to-residual ratio before and after reduced precision.

Allow selective promotion as an intervention. Compute logits/softmax or LayerNorm statistics in float32 while keeping projections and FFN GEMMs in lower precision. Recompare speed, memory, and error to determine which promoted operations earn their cost.

Part 2 - Mathematical form and reasoning

For stage reference z and candidate $\widehat{z}$, report

$$E_{\text{abs}} = \max_i |\widehat{z}_i - z_i|, \qquad E_{\text{rel}} = \max_i \frac{|\widehat{z}_i - z_i|}{\max(|z_i|, \varepsilon)}.$$

Residual-update preservation can be measured by

$$R = \frac{\|F(x)\|_2}{\max(\|x\|_2, \varepsilon)}$$

and the difference between float32 and reduced-precision $x + F(x)$. A small finite error may still matter if it changes a discrete attention winner or erases most of a residual update.

Part 3 - Hints, controls, and evidence

1. Use identical master weights and inputs for float32, float16, and bfloat16 paths.
2. Instrument actual dtypes and health statistics after every critical stage.
3. Sweep input, Q/K, and FFN weight scales plus normal and extreme masks.
4. Record finite values, absolute/relative error, cosine similarity, attention ranking, and residual-update preservation.
5. Promote one sensitive operation at a time and measure recovered accuracy, latency, and memory.
6. Define tolerances before inspecting results and keep failing cases as regression tests.

Part 4 - Python code skeleton

```
import torch

def capture_forward(block, x, mask, amp_dtype=None):
    enabled = amp_dtype is not None and x.is_cuda
    with torch.autocast("cuda", dtype=amp_dtype, enabled=enabled):
        output, debug = block(x, mask, return_debug=True)
    return output, debug

def stage_metrics(reference, candidate):
    return {"max_abs": ..., "max_rel": ..., "cosine": ...,
            "finite": bool(torch.isfinite(candidate).all())}

def E12(block, scales, dtypes):
    reports = []
    for scale in scales:
        x, mask = ...
        ref = capture_forward(block, x.float(), mask, None)
        for dtype in dtypes:
            # TODO: candidate path and stage-by-stage comparison
            reports.append(...)
    return reports
```

Part 5 - The Philosophy Behind the Exercise

Mixed precision is selective forgetting. The machine represents fewer numeric distinctions in exchange for speed and capacity. The engineering question is which distinctions the model can afford to lose.

Sensitivity is local. Matrix multiplication may tolerate reduced precision while a normalization statistic or phase boundary does not. Treating the whole block uniformly ignores the different numerical roles of its parts.

The best mixed-precision design is therefore not the lowest dtype everywhere, but the cheapest allocation of precision that preserves declared behavior.

17.5.14 Exercise 13

Exercise Title: Implement Gradient Check on Tiny Transformer Block and Validate Backward Correctness

Part 1 - Why this matters

A Transformer block contains multiple residual branches, normalizations, softmax, projections, activation functions, and masks. Autograd can compute a gradient for a program that is structurally wrong, and a decreasing loss does not prove every branch receives the intended derivative. Finite-difference gradient checking provides an independent local test on a tiny deterministic block.

Use float64, dropout zero, a small smooth configuration, and a scalar objective that depends on every output coordinate. Avoid saturated softmax and ReLU kink points; GELU is convenient for smooth checking. Check input gradients and selected entries from attention projections, LayerNorm scale/bias, FFN weights, and output projections.

Residual paths create useful expected behavior. If sublayer weights are set to zero in a Pre-LN block, the output equals the input and the input Jacobian should be identity. Masks create another expectation: a loss on an early causal position must have zero derivative with respect to future input positions.

Run `torch.autograd.gradcheck` and a manual centered-difference sweep. Vary epsilon to expose the balance between truncation and rounding. Inject a detach, wrong residual, and in-place modification; prove the checker or a related invariant detects each failure.

Part 2 - Mathematical form and reasoning

For scalar objective $L(\theta)$,

$$g_i^{\mathrm{FD}} = \frac{L(\theta + \epsilon e_i) - L(\theta - \epsilon e_i)}{2\epsilon}.$$

Compare with autograd through normalized error

$$E_i = \frac{|g_i^{\mathrm{AD}} - g_i^{\mathrm{FD}}|}{\max(1, |g_i^{\mathrm{AD}}|, |g_i^{\mathrm{FD}}|)}.$$

For a zero-update residual block $Y = X$, require

$$\frac{\partial Y}{\partial X} = I.$$

For causal output Y_i, require

$$\frac{\partial Y_i}{\partial X_j} = 0 \quad \text{for } j > i.$$

Part 3 - Hints, controls, and evidence

1. Use $B = 1$, short sequence, small width, two heads, float64, and dropout zero.
2. Choose moderate deterministic values and a smooth weighted scalar loss.

3. Check parameters from attention, both norms, and both FFN projections as well as input.
4. Sweep epsilon and report absolute and normalized error.
5. Test identity-residual and causal-zero-gradient properties separately.
6. Inject detach, wrong merge, and in-place failures and retain regression tests.

Part 4 - Python code skeleton

```
import torch

def objective(block, x, mask):
    y = block(x, mask)
    coefficients = torch.linspace(0.2, 1.1, y.numel(),
                                  dtype=y.dtype).reshape_as(y)
    return (y * coefficients).sum()

def centered_difference(loss_fn, tensor, index, epsilon):
    flat = tensor.view(-1)
    original = flat[index].item()
    with torch.no_grad(): flat[index] = original + epsilon
    plus = loss_fn().item()
    with torch.no_grad(): flat[index] = original - epsilon
    minus = loss_fn().item()
    with torch.no_grad(): flat[index] = original
    return (plus - minus) / (2 * epsilon)

def E13():
    block, x, mask = ...  # float64 and deterministic
    # TODO: gradcheck, manual epsilon sweep, identity Jacobian,
    # causal gradients, and failure injection
    return ...
```

Part 5 - The Philosophy Behind the Exercise

Backpropagation is trusted because it follows the chain rule through a program, but that trust applies to the program that exists, not the architecture we intended. Gradient checking asks the function itself how it changes.

Residual and causal properties provide more than numerical comparison; they give structural expectations for derivatives. Correct gradients should reflect the model's declared routes of preservation and information access.

Independent evidence is the foundation of reliability. Forward parity, finite-difference sensitivity, and behavioral training each observe a different face of the same system. Agreement among them is stronger than confidence in any one test.

17.5.15 Exercise 14

Exercise Title: Stress Test Deep Transformer Stack and Observe Gradient Flow Differences Between Pre-LN and Post-LN

Part 1 - Why this matters

Differences between Pre-LN and Post-LN become most visible as depth increases. A shallow stack may train under either convention, while dozens of layers can amplify gradient attenuation, explosion, residual-scale drift, or sensitivity to initialization and learning rate. This exercise builds controlled deep stacks and measures gradient flow before committing expensive training runs.

Copy compatible attention and FFN weights between norm variants and use the same inputs, loss, masks, initialization policy, and seeds. Evaluate depth at initialization with a synthetic scalar loss, then train a small task under matched and separately tuned optimization settings. The controlled comparison isolates architecture; the tuned comparison answers what each architecture can achieve with appropriate support.

Record gradient norm for every block input and parameter group, activation norm, LayerNorm input variance, residual update ratio, and cosine similarity between successive states. Use logarithmic plots across depth. A single total gradient norm can hide early layers receiving almost no signal while later layers remain healthy.

Stress testing must remain safe. Increase depth, batch, and sequence length within known memory limits, use isolated runs for configurations expected to fail, and distinguish numerical divergence from OOM. Checkpoint initial states and failing batches so every instability can be reproduced.

Part 2 - Mathematical form and reasoning

For stack $X^{(\ell+1)} = F_\ell(X^{(\ell)})$, gradient propagation is

$$\frac{\partial \mathcal{L}}{\partial X^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial X^{(L)}} \prod_{k=\ell}^{L-1} J_{F_k}.$$

Pre-LN residual Jacobians contain identity terms

$$J_{F_k} = I + J_{G_k} J_{\mathrm{LN}_k},$$

while Post-LN applies the normalization Jacobian after the sum. Monitor the gradient ratio

$$R_\ell = \frac{\|g^{(\ell)}\|_2}{\max(\|g^{(L)}\|_2, \varepsilon)}$$

and report non-finite or underflow boundaries by depth and configuration.

Part 3 - Hints, controls, and evidence

1. Compare depths such as 2, 8, 24, 48, and 96 within the safe memory envelope.
2. Copy shared weights and fix inputs, masks, loss, seeds, and dropout for initial gradient tests.

3. Log per-layer activation, update, normalization, and gradient statistics.
4. Run matched optimizer settings first, then tuned learning-rate/warmup settings as a separately labeled experiment.
5. Record divergence step, first non-finite layer, loss curve, peak memory, and runtime.
6. Retain the exact state and batch for the first unstable configuration and turn the observed invariant into a regression stress test.

Part 4 - Python code skeleton

```
import torch

def attach_depth_hooks(stack):
    records = {}
    # TODO: retain gradients on block inputs and log activations
    return records

def one_gradient_probe(model, x, mask, target):
    records = attach_depth_hooks(model)
    output = model(x, mask)
    loss = ...
    loss.backward()
    # TODO: collect ordered activation and gradient statistics
    return records

def E14(depths=(2, 8, 24, 48), seeds=range(5)):
    reports = []
    for norm_style in ["pre", "post"]:
        for depth in depths:
            for seed in seeds:
                # TODO: matched initialization, gradient probe,
                # controlled training, and safe failure reporting
                reports.append(...)
    return reports
```

Part 5 - The Philosophy Behind the Exercise

Depth multiplies consequences. A small difference in how one block routes a gradient becomes a repeated principle governing an entire stack. Architecture reveals its dynamics through repetition.

Residuals preserve possibility: even when learned transformations are weak or unstable, an identity route remains available. Normalization decides how that route interacts with scale at every layer.

Stress testing asks not only whether a system works, but how far its organizing principles remain effective. The boundary is part of the model's identity and should be measured before scale turns it into a costly surprise.

17.5.16 Exercise 15

Exercise Title: Implement Activation Checkpointing in Transformer Stack and Quantify Memory Savings and Time Cost

Part 1 - Why this matters

Backpropagation normally stores intermediate activations from every Transformer block so gradients can be computed later. Deep stacks and long sequences make these saved tensors a dominant memory cost. Activation checkpointing stores selected boundary states and recomputes internal forward operations during backward, trading additional computation for lower peak memory.

Wrap blocks or groups of blocks with PyTorch checkpointing. The checkpointed function must be deterministic with respect to its inputs and preserve required random-number behavior for dropout. Modern non-reentrant checkpointing is often preferred, but record API choices and framework version. Inputs requiring gradient, nested outputs, masks, and keyword arguments need explicit tests.

Compare identical model weights, inputs, dropout mode, loss, and optimizer step with and without checkpointing. Verify forward outputs, loss, input gradients, parameter gradients, and updated parameters within tolerance. Then measure peak allocated and reserved CUDA memory, forward time, backward time, and total step time. Synchronize timings and reset memory statistics around isolated trials.

Sweep checkpoint granularity. Checkpointing every block can save more activation memory but causes more recomputation and overhead. Grouping several blocks changes the tradeoff. The best policy depends on sequence length, depth, block cost, available GPU memory, and target throughput.

Part 2 - Mathematical form and reasoning

For composition

$$y = F_L \circ \cdots \circ F_1(x),$$

ordinary backward stores intermediate $x_1, \ldots, x_{L-1}$. Checkpointing a segment stores its boundaries and recomputes

$$x_{k+1}, \ldots, x_{k+m-1}$$

during backward. Report memory saving

$$S_M = 1 - \frac{M_{\text{checkpoint}}}{M_{\text{baseline}}}$$

and time overhead

$$O_t = \frac{t_{\text{checkpoint}} - t_{\text{baseline}}}{t_{\text{baseline}}}.$$

These measurements must use the same workload and include backward.

Part 3 - Hints, controls, and evidence

1. Establish exact or tolerant forward/backward parity with dropout zero before testing stochastic training.
2. Compare no checkpointing, every block, and grouped-block policies.
3. Record API mode, RNG preservation, model depth, sequence length, dtype, and whether compilation is enabled.
4. Reset peaks, synchronize CUDA, and measure forward, backward, and optimizer time separately.
5. Report allocated and reserved memory plus throughput, not only whether a larger batch fits.
6. Test masks, keyword arguments, cached debug outputs, and failure cases that violate checkpoint function requirements.

Part 4 - Python code skeleton

```
import torch
from torch.utils.checkpoint import checkpoint

class CheckpointedStack(torch.nn.Module):
    def __init__(self, layers, enabled=True):
        super().__init__()
        self.layers = torch.nn.ModuleList(layers)
        self.enabled = enabled

    def forward(self, x, mask=None):
        for layer in self.layers:
            if self.enabled and self.training:
                x = checkpoint(lambda state: layer(state, mask),
                               x, use_reentrant=False)
            else:
                x = layer(x, mask)
        return x

def measured_step(model, batch):
    # TODO: synchronized forward/backward/update timing and memory peaks
    return ...

def E15(configurations):
    # TODO: parity and granularity tradeoff sweep
    return ...
```

Part 5 - The Philosophy Behind the Exercise

Checkpointing exchanges memory for time by refusing to remember every detail of the forward pass. When the past is needed, it is reconstructed from preserved boundaries.

This is a computational form of selective memory. The system decides which states are worth storing and which can be regenerated more cheaply than they can be retained.

The technique reveals that efficiency is not one dimension. Less memory can enable a deeper model or larger batch while requiring more computation. The right choice follows the scarce resource and the purpose of the run.

17.5.17 Exercise 16

Exercise Title: Create Unit Tests that Validate Shapes, Dtypes, and Masking Across Encoder and Decoder Blocks

Part 1 - Why this matters

Encoder and decoder blocks preserve external tensor width while containing many internal layouts and mask semantics. A reusable test suite should validate their contracts independently of a training task: output shapes, intermediate attention shapes, dtype/device preservation, residual equality, LayerNorm axis, dropout mode, padding invariance, causal non-leakage, cross-attention source alignment, gradients, and serialization.

Parameterize asymmetric dimensions. Equal source and target length hides cross-attention axis swaps; batch one hides broadcasting; $B = T$ hides layout transposes; float32 alone hides autocast boundaries. Test several batches, source/target lengths, model widths, heads, dtypes, and devices while keeping the smoke suite small enough for continuous use.

Method-specific behavior matters. Encoder self-attention must ignore padded keys. Decoder self-attention must ignore both padding and future targets. Cross-attention must ignore padded source memory without applying causal target rules to the source. Every valid row must preserve probability normalization or follow a documented all-masked policy.

Build failure injection for wrong norm axis, missing causal mask, swapped source and target masks, residual broadcasting, dropout active in evaluation, and CPU buffers. A test suite is credible when it fails under the defects it claims to prevent.

Part 2 - Mathematical form and reasoning

Encoder contract:

$$(B, T_x, d) \to (B, T_x, d), \qquad A \in \mathbb{R}^{B \times H \times T_x \times T_x}.$$

Decoder contract:

$$(B, T_y, d), (B, T_x, d) \to (B, T_y, d),$$

$$A_{\text{self}} \in \mathbb{R}^{B \times H \times T_y \times T_y}, \quad A_{\text{cross}} \in \mathbb{R}^{B \times H \times T_y \times T_x}.$$

Mask invariants require zero forbidden weights and causal Jacobian zeros for future target positions.

Part 3 - Hints, controls, and evidence

1. Divide tests into fast shared contracts, encoder behavior, decoder behavior, dtype/device, gradients, and slower parity/stress groups.
2. Parameterize unequal batch, source length, target length, heads, and width.
3. Compare batched examples with independent single-example references.
4. Use future-token and padded-memory interventions in addition to weight matrix assertions.
5. Save/load state dictionaries and rerun deterministic outputs.
6. Inject every targeted bug and confirm the earliest intended test fails.

Part 4 - Python code skeleton

```
import pytest
import torch

@pytest.mark.parametrize("batch,source,target,width,heads", [
    (1, 3, 2, 8, 2), (2, 5, 3, 12, 3), (4, 2, 7, 16, 4)
])
def test_encoder_decoder_shapes(batch, source, target, width, heads):
    encoder, decoder = ...
    memory = encoder(torch.randn(batch, source, width), ...)
    output, self_w, cross_w = decoder(
        torch.randn(batch, target, width), memory,
        ..., ..., return_weights=True
    )
    assert output.shape == (batch, target, width)
    assert self_w.shape == (batch, heads, target, target)
    assert cross_w.shape == (batch, heads, target, source)

def E16():
    # TODO: dtype, device, residual, norm, dropout, padding,
    # causal, gradient, serialization, and failure-injection tests
    return ...
```

Part 5 - The Philosophy Behind the Exercise

A block interface is a promise that many internal transformations preserve specific external meanings. Unit tests turn that promise into executable constraints.

Encoder and decoder tests also encode laws of information access. Shapes say which entities may correspond; masks say which correspondences are permitted; gradients reveal which paths actually connect.

The suite becomes a durable specification. It allows implementation to change for speed or clarity while preserving the behavior on which later chapters depend.

17.5.18 Exercise 17

Exercise Title: Profile GPU Kernels and Identify MatMul Hotspots in FFN and Attention Portions

Part 1 - Why this matters

The Transformer block is dominated by matrix multiplication, but those GEMMs have different shapes and scaling. Q/K/V and output projections operate on flattened token batches; attention score and value products contain the quadratic sequence dimension; FFN up/down projections use the expanded hidden width. A profiler reveals which kernels dominate for the actual workload and whether fusion, memory traffic, or launch overhead changes the expected result.

Instrument logical ranges around attention projections, QK, mask/softmax, AV, output projection, FFN up, activation, FFN down, norms, and residuals. Use a scheduled capture after warm-up and record shapes and memory. Avoid returning large debug tensors in the production-like profile because doing so can disable fused attention or retain outputs that would otherwise be freed.

Compare short-wide and long-narrow regimes, inference and training, float32 and mixed precision, explicit and fused attention, and several FFN expansion ratios. Sort by self CUDA time, total CUDA time, call count, and memory. A large total time can come from many small calls; a high self time identifies the kernel's own contribution.

Turn one observed hotspot into an intervention: fuse QKV projection, use a memory-efficient attention path, change FFN activation implementation, or compile the block. Re-profile and verify output/gradient parity. Optimization is complete only when measured cost falls without violating model behavior.

Part 2 - Mathematical form and reasoning

Attention matrix products include

$$(BH, T, d_h)@(BH, d_h, T) \to (BH, T, T)$$

and

$$(BH, T, T)@(BH, T, d_h) \to (BH, T, d_h).$$

FFN products include

$$(BT, d)@(d, d_{\text{ff}}) \to (BT, d_{\text{ff}})$$

and the reverse projection. The attention products require approximately $\mathcal{O}(BT^2d)$ work. The FFN products require approximately $\mathcal{O}(BTdd_{\text{ff}})$ work. Kernel occupancy and memory bandwidth determine achieved performance.

Part 3 - Hints, controls, and evidence

1. Warm up before a short scheduled profile and annotate logical sublayers with named ranges.

2. Record shapes, stacks only when necessary, CPU/CUDA activity, and memory.
3. Profile several regimes and keep debug-weight-returning mode separate from production mode.
4. Identify GEMM shapes, softmax/reduction kernels, copies, and unexpected synchronization.
5. Propose an optimization from the measured trace, then rerun both profiler and correctness tests.
6. Report environment, backend selection, dtype, compile/fusion state, and input configuration with every trace.

Part 4 - Python code skeleton

```
import torch
from torch.profiler import profile, record_function, ProfilerActivity

def profiled_step(block, x, mask, training):
    with record_function("transformer_block"):
        output = block(x, mask)
        if training:
            output.square().mean().backward()
    return output

def profile_configuration(block, x, mask, training):
    schedule = torch.profiler.schedule(wait=1, warmup=2, active=3)
    with profile(activities=[ProfilerActivity.CPU,
                             ProfilerActivity.CUDA],
                 schedule=schedule, record_shapes=True,
                 profile_memory=True) as prof:
        for _ in range(8):
            profiled_step(block, x, mask, training)
            prof.step()
    return prof.key_averages().table(sort_by="self_cuda_time_total")

def E17(configurations):
    # TODO: regime sweep, trace export, optimization, and parity
    return ...
```

Part 5 - The Philosophy Behind the Exercise

Profiling reveals how a conceptual block becomes physical work. The names “attention” and “FFN” dissolve into matrix shapes, reductions, memory movement, and kernel schedules.

This translation matters because performance is produced by alignment between algorithm and machine. The same equation can be fast or slow depending on how its dimensions fit the hardware.

Evidence-driven optimization preserves intellectual honesty. We improve the system that actually runs, not the one imagined from reading source code.

17.5.19 Exercise 18

Exercise Title: Build a Transformer Block Debugging Checklist Covering Masking, Norm Placement, and Precision Issues

Part 1 - Why this matters

A Transformer block can fail through configuration, geometry, information access, normalization, stochastic mode, numerics, gradients, device placement, or performance. A checklist orders these possibilities so debugging begins with the earliest cheap invariant instead of tuning hyperparameters around a broken implementation.

Begin with configuration and tensor contracts: divisible heads, positive FFN width, supported positions, exact residual shapes, devices, and dtypes. Continue through attention masks, probability invariants, LayerNorm axis and placement, dropout mode, FFN dimensions, residual update ratios, finite values, gradients, CPU/CUDA parity, and only then profiling. Decoder checks add target causality and source-memory separation.

Every item should return pass/fail, evidence, likely causes, and whether later checks are blocked. "Loss is NaN" is too late and too vague. A useful report states that QK logits first became infinite in layer 7 under float16 at a particular scale, or that a causal mask broadcast along the head axis instead of the query axis.

Use failure injection to test the checklist: missing causal mask, all-masked row, wrong norm placement, sequence-axis normalization, residual broadcasting, dropout active in eval, detached FFN, CPU buffer, oversized logits, and checkpointing parity failure. Retain every detected case.

Part 2 - Mathematical form and reasoning

The checklist enforces, in order,

$$d = Hd_h, qquadd_{\text{ff}} > 0,$$

$$\text{shape}(X) = \text{shape}(F(X)),$$

$$A_{\text{forbidden}} = 0, qquad \sum_j A_{b,h,i,j} = 1,$$

$$\text{LN acts over the final feature axis,}$$

and finiteness of each ordered stage. Let $Z_1, \ldots, Z_K$ be captured tensors; the primary failure is

$$k^* = \min\{k : \text{invariant}_k(Z_k) = \text{false}\}.$$

Part 3 - Hints, controls, and evidence

1. Group checks into configuration, shapes, masks, norm, mode, numerics, backward, device, parity, and performance.
2. Give each check an assertion, evidence payload, likely causes, and repair test.

3. Stop at or emphasize the earliest violated invariant and mark downstream checks as blocked.
4. Include tiny encoder, decoder, causal, padding, cross-attention, and mixed- precision cases.
5. Inject every targeted failure and verify the intended check catches it.
6. Save a machine-readable incident report with seed, configuration, state, batch, cause, correction, and regression test.

Part 4 - Python code skeleton

```
from dataclasses import dataclass, asdict

@dataclass
class BlockCheck:
    name: str
    passed: bool
    evidence: dict
    likely_causes: list
    blocked_by: str | None = None

def check_residual(name, residual, update):
    return BlockCheck(name,
        residual.shape == update.shape and
        residual.dtype == update.dtype and
        residual.device == update.device,
        {"residual": tuple(residual.shape),
         "update": tuple(update.shape)},
        ["wrong projection", "silent broadcasting", "layout swap"])

def run_block_checklist(block, case):
    checks = []
    # TODO: ordered execution and earliest-failure logic
    return {"passed": all(c.passed for c in checks),
            "checks": [asdict(c) for c in checks]}

def E18():
    # TODO: correct matrix and injected-failure matrix
    return ...
```

Part 5 - The Philosophy Behind the Exercise

A debugging checklist is a map of dependence. It records which truths must hold before later truths are meaningful. Performance has no meaning before correctness; gradient health has limited meaning before information access is correct.

The earliest violated invariant is valuable because it is closest to the first departure from intention. Later symptoms are consequences accumulating along the computational graph.

By preserving failures as tests, the project develops institutional memory. Understanding becomes embedded not only in people and prose, but in executable barriers against regression.

17.5.20 Exercise 19

Exercise Title: Produce an Engineering Report Explaining Transformer Block Design Decisions and Their Practical Effects

Part 1 - Why this matters

The final report must explain how attention, FFN, residuals, LayerNorm, dropout, masks, and depth form a Transformer block rather than presenting them as an unconnected list. Every design decision should be linked to mathematical role, tensor contract, optimization behavior, measured systems cost, failure modes, and evidence from the chapter.

Begin with the chosen encoder and decoder equations. State Pre-LN or Post-LN, FFN width and activation, dropout locations, mask conventions, positional interface, final normalization, and parameter initialization. Explain why each residual addition is shape-safe and how padding, causality, and cross-attention boundaries are enforced.

Use artifacts from E00–E18: scratch/library parity, FFN expansion sweeps, norm-stability curves, broadcasting failures, dropout statistics, depth probes, classifier and language-model tiny-overfit tests, NaN incidents, CUDA parity, runtime crossover, mixed-precision boundaries, gradient checks, deep-stack flow, checkpointing tradeoffs, unit tests, and profiler traces. Conditional and negative findings are part of the engineering answer.

Separate controlled comparisons from tuned best-case results. Distinguish theoretical complexity from observed kernel time, logical head or layer ablation from real speedup, numerically finite execution from stable learning, and accepted tolerance from exact identity. Preserve environment, seeds, hardware, dtypes, and stopping rules.

End with a default block configuration for the next chapter, explicit reasons for that default, rejection conditions, monitoring signals, and unresolved questions about training complete Transformer systems.

Part 2 - Mathematical form and reasoning

Anchor the report in the selected block equations, for example Pre-LN:

$$U = X + \mathrm{Dropout}(\mathrm{MHA}(\mathrm{LN}_1(X))),$$

$$Y = U + \mathrm{Dropout}(W_2\phi(W_1\,\mathrm{LN}_2(U) + b_1) + b_2).$$

Include parameter and cost estimates

$$N_{\text{attention}} \approx 4d^2, \qquad N_{\text{FFN}} \approx 2dd_{\text{ff}},$$

$$C_{\text{block}} = \mathcal{O}(BTd^2 + BT^2d + BTdd_{\text{ff}}),$$

plus exact mask, residual, normalization, gradient, and numerical invariants used to validate the implementation.

Part 3 - Hints, controls, and evidence

1. Open with the intended workload and a precise block specification.
2. Map every design claim to equations, configuration, experiment, raw artifact, observed behavior, limitation, and recommendation.
3. Include encoder and decoder information-access proofs, not only shape tests.
4. Report stability, precision, runtime, memory, checkpointing, and depth as measured trade-offs.
5. Include an incident appendix containing injected bugs, earliest failed invariants, causes, corrections, and regression tests.
6. Finish with a conditional default and explicit triggers for revisiting its norm style, width, dropout, precision, or checkpoint policy.

Part 4 - Python code skeleton

```
from pathlib import Path

REQUIRED = ["encoder_parity", "ffn", "norm_stability",
            "residuals", "dropout", "depth", "decoder_masks",
            "tiny_overfit", "nan_debug", "cuda", "benchmark",
            "mixed_precision", "gradcheck", "checkpointing",
            "unit_tests", "profiler", "debug_checklist"]

def validate_artifacts(artifacts):
    return {name: ... for name in REQUIRED}

def build_decisions(artifacts, workload):
    # TODO: observation, interpretation, limitation, and decision
    return ...

def E19(artifact_directory, workload, output_path):
    artifacts = ...
    checks = validate_artifacts(artifacts)
    assert all(checks.values()), checks
    report = build_decisions(artifacts, workload)
    Path(output_path).write_text(report, encoding="utf-8")
    return {"report": str(output_path), "checks": checks}
```

Part 5 - The Philosophy Behind the Exercise

The Transformer block is a compact theory of deep computation. It assumes that representations should communicate globally, transform locally, preserve a continuing state, and

regulate scale as they pass through depth.

Engineering reveals that this theory lives inside constraints. Masks define knowledge, residuals define continuity, normalization shapes gradients, precision bounds numeric distinctions, and hardware determines the cost of every design.

A final report makes these relationships accountable. Understanding is not the ability to recite the block diagram; it is the ability to predict how the block behaves, detect when it departs from intention, and justify why each component remains in the design.

17.6 TRF04

Chapter Title: Tokenization, Embeddings, and Vocabulary Pipeline
Specialization: Models and Training

17.6.1 Exercise 00

Exercise Title: Implement Basic Tokenization and Build Vocabulary Mapping with Special Tokens and Correct Index Handling

Part 1 - Why this matters

A Transformer never receives text. It receives integer identifiers whose meaning is fixed by a tokenizer and a vocabulary. The mapping may look like a minor data preprocessing detail, yet it is part of the model contract: changing the ID of one token while reusing the same embedding weights silently changes the meaning of the model. A checkpoint is therefore incomplete unless its vocabulary, normalization rules, and special-token policy are recoverable.

Build a deliberately small word-level tokenizer before using a production library. Specify whether text is lowercased, how punctuation is separated, and whether repeated whitespace is normalized. Reserve stable IDs for `<pad>`, `<unk>`, `<bos>`, and `<eos>`; then assign ordinary tokens using a deterministic ordering such as decreasing frequency with lexical tie-breaking. Build the vocabulary from training data only. A vocabulary built from validation or test text leaks information even if labels were never inspected.

The implementation must support encoding and decoding. Encoding adds boundary tokens according to an explicit policy and maps unseen items to `<unk>`. Decoding must distinguish exact reconstruction from a readable rendering that removes special tokens. Test empty text, repeated tokens, punctuation, unknown tokens, and Unicode input. Finally, serialize the mapping and prove that loading it produces exactly the same IDs. Never rebuild a vocabulary when loading model weights.

Part 2 - Mathematical form and reasoning

Let the tokenizer be $\tau : s \mapsto (u_1, \ldots, u_n)$ and let $\mathcal{V}$ be an ordered vocabulary. The encoder is

$$e(s) = \big[i_{\text{bos}}, v(u_1), \ldots, v(u_n), i_{\text{eos}}\big], \qquad v(u) = \begin{cases} \text{id}(u), & u \in \mathcal{V}, \\ i_{\text{unk}}, & u \notin \mathcal{V}. \end{cases}$$

The range invariant is $0 \leq e(s)_j < |\mathcal{V}|$. The four special IDs must be distinct, and the inverse mapping must satisfy $v^{-1}(v(u)) = u$ for every known ordinary token. Determinism means that the same training corpus and configuration produce the same ordered mapping, not merely the same set of tokens.

Part 3 - Hints, controls, and evidence

1. Write the normalization and boundary-token policies before writing code.
2. Use two corpora with equal-frequency tokens in different input orders; prove that lexical tie-breaking gives identical vocabularies.
3. Report vocabulary size, unknown-token rate, round-trip accuracy, and a hash of the serialized vocabulary.
4. Inject a failure by swapping two IDs after saving. Show that shapes remain valid while decoded meaning changes.
5. Reject duplicate special tokens, missing inverse entries, and IDs outside the contiguous interval $[0, |\mathcal{V}|)$.

Part 4 - Python code skeleton

```
import json, re
from collections import Counter

SPECIALS = ["<pad>", "<unk>", "<bos>", "<eos>"]

def tokenize(text):
    text = text.lower().strip()
    return re.findall(r"\w+|[^\w\s]", text, flags=re.UNICODE)

def build_vocab(train_texts, min_freq=1):
    counts = Counter(tok for s in train_texts for tok in tokenize(s))
    kept = sorted((t for t, c in counts.items() if c >= min_freq),
                  key=lambda t: (-counts[t], t))
    tokens = SPECIALS + [t for t in kept if t not in SPECIALS]
    token_to_id = {t: i for i, t in enumerate(tokens)}
    assert len(token_to_id) == len(tokens)
    return token_to_id, tokens

def encode(text, stoi):
    ids = [stoi["<bos>"]]
    ids += [stoi.get(t, stoi["<unk>"]) for t in tokenize(text)]
    ids += [stoi["<eos>"]]
    assert all(0 <= i < len(stoi) for i in ids)
    return ids

def decode(ids, itos, remove_special=False):
```

```
    tokens = [itos[i] for i in ids]
    if remove_special:
        tokens = [t for t in tokens if t not in SPECIALS]
    return " ".join(tokens)

def E00():
    train = ["Red fox jumps.", "Blue fox waits!"]
    stoi, itos = build_vocab(train)
    ids = encode("Green fox jumps.", stoi)
    payload = json.dumps({"tokens": itos}, ensure_ascii=False)
    restored = json.loads(payload)["tokens"]
    assert restored == itos and len(set(SPECIALS)) == 4
    return {"ids": ids, "decoded": decode(ids, itos),
            "unknown_rate": ids.count(stoi["<unk>"]) / len(ids)}
```

Part 5 - The Philosophy Behind the Exercise

Tokenization creates the ontology through which the model can perceive text. The model cannot discover that two IDs were accidentally exchanged; it simply learns within the world it was given. This makes vocabulary construction an act of definition, not clerical preprocessing.

The deeper engineering principle is identity preservation. A token ID has no intrinsic meaning. Its meaning exists only through the agreement among the tokenizer, embedding table, output head, checkpoint, and serving pipeline. A reliable system makes that agreement explicit, versioned, and testable.

17.6.2 Exercise 01

Exercise Title: Implement Embedding Layer Usage and Verify Output Shape (batch, seq, d_model) and Dtype Correctness

Part 1 - Why this matters

An embedding layer converts symbolic token IDs into continuous vectors that a Transformer can transform. It is not a dense layer applied to the numerical value of an ID. It is an indexed lookup: ID 900 is not inherently larger or more important than ID 9. Confusing these views destroys the categorical semantics of tokens.

Create input IDs with shape `(B,T)` and dtype `torch.long`. For a table with V rows and d_{model} columns, verify that the output has shape `(B,T,d_model)` and the same floating dtype and device as the embedding weights. Show that repeated IDs produce identical vectors and that changing a single ID changes only the corresponding output position. Test an invalid negative or out-of-range ID and record the resulting failure rather than silently clipping it.

Use `padding_idx` and inspect its exact behavior. PyTorch keeps the padding row from receiving an ordinary embedding gradient, but this does not automatically prevent padded positions from influencing attention or loss. Those are separate responsibilities handled by masks. This separation is a critical pipeline invariant.

Part 2 - Mathematical form and reasoning

Let $E \in \mathbb{R}^{V \times d}$ and $I \in \{0, \ldots, V-1\}^{B \times T}$. Embedding lookup is

$$X_{b,t,:} = E_{I_{b,t},:}, \qquad X \in \mathbb{R}^{B \times T \times d}.$$

If a scalar loss L depends on X, gradients for repeated IDs accumulate in the same row:

$$\frac{\partial L}{\partial E_{j,:}} = \sum_{(b,t):I_{b,t}=j} \frac{\partial L}{\partial X_{b,t,:}}.$$

This equation explains both weight sharing across token occurrences and the frequency dependence of embedding updates.

Part 3 - Hints, controls, and evidence

1. Use unequal values for B, T, and d so an axis swap cannot pass by coincidence.
2. Compare lookup with direct row indexing, `weight[input_ids]`.
3. Backpropagate `output.sum()` and inspect gradients for used, unused, repeated, and padding rows.
4. Test float32 and float64 weights; IDs must remain integer in both cases.
5. Record shape, dtype, device, parameter count, and gradient-row norms.

Part 4 - Python code skeleton

```
import torch
from torch import nn

def E01(dtype=torch.float32, device="cpu"):
    V, d, pad = 11, 7, 0
    layer = nn.Embedding(V, d, padding_idx=pad,
                         device=device, dtype=dtype)
    ids = torch.tensor([[1, 2, 2, 0], [3, 1, 4, 0]],
                       dtype=torch.long, device=device)
    out = layer(ids)
    assert out.shape == (2, 4, d)
    assert out.dtype == layer.weight.dtype
    assert out.device == layer.weight.device
    assert torch.equal(out[0, 0], out[1, 1])
    assert torch.equal(out, layer.weight[ids])

    out.sum().backward()
    row_norms = layer.weight.grad.norm(dim=1)
    assert row_norms[pad] == 0
    assert row_norms[5] == 0
    assert row_norms[2] > row_norms[3]
    return out.detach(), row_norms
```

Part 5 - The Philosophy Behind the Exercise

Embeddings replace arbitrary names with learnable geometry. Tokens begin as discrete identities and acquire relations only through the gradients created by their contexts. The table is therefore a meeting point between symbolic data and continuous optimization.

The philosophical warning is equally important: geometry does not erase identity. Before similarity can be learned, ID integrity must be preserved. Meaningful representation depends on a correct index contract.

17.6.3 Exercise 02

Exercise Title: Implement Padding, Attention Masks, and Loss Masks and Prove They Align Correctly Across Batches

Part 1 - Why this matters

Variable-length sequences are usually padded into a rectangular batch, but a padding token is storage, not language. Three related objects must remain distinct. `input_ids` holds tokens including padding. An attention mask controls which key positions may supply information. A loss mask controls which target positions contribute to the objective. Reusing one tensor without stating its semantics invites silent mistakes.

Create a batch containing different lengths and construct all three objects. For causal language modeling, combine a lower-triangular causal rule with the key-padding rule. Replace ignored labels with `-100` and use the same `ignore_index` in cross entropy. Prove alignment by changing the ID stored in padded cells: logits at valid positions and the mean valid-token loss must remain unchanged when the mask is correct. Then remove the attention mask or loss mask and show which invariant fails.

Be precise about query padding. Masking padded keys prevents valid tokens from reading padding. Padded query rows may still produce finite outputs; they are harmless only if later consumers and the loss ignore them. Avoid fully masked softmax rows or define a stable policy for them.

Part 2 - Mathematical form and reasoning

For validity mask $P_{b,j} \in \{0, 1\}$ and causal rule $j \leq i$, define

$$M_{b,i,j} = \begin{cases} 0, & P_{b,j} = 1 \text{ and } j \leq i, \\ -\infty, & \text{otherwise.} \end{cases}$$

Attention weights are $A = \text{softmax}(QK^\top/\sqrt{d_k} + M)$. If labels are $y_{b,t}$ and $R_{b,t}$ marks valid targets, the mean loss is

$$L = -\frac{1}{\sum R_{b,t}} \sum_{b,t} R_{b,t} \log p_{b,t}(y_{b,t}).$$

Changing a padded token must not change either sum or any allowed attention distribution.

Part 3 - Hints, controls, and evidence

1. Print masks as integer matrices before converting them to additive form.

2. Assert every valid query assigns zero probability to padded and future keys and that each valid attention row sums to one.

3. Compare mean loss against a manual sum divided by the number of valid targets, not by $B \times T$.

4. Include lengths one, maximum length, and a batch with heavy padding.

5. Inject separate attention-mask and loss-mask faults; name the earliest violated invariant for each.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def make_batch(sequences, pad_id=0, ignore_index=-100):
    B, T = len(sequences), max(map(len, sequences))
    ids = torch.full((B, T), pad_id, dtype=torch.long)
    valid = torch.zeros(B, T, dtype=torch.bool)
    for b, seq in enumerate(sequences):
        ids[b, :len(seq)] = torch.tensor(seq)
        valid[b, :len(seq)] = True
    labels = torch.full_like(ids, ignore_index)
    labels[:, :-1] = ids[:, 1:]
    label_valid = torch.zeros_like(valid)
    label_valid[:, :-1] = valid[:, :-1] & valid[:, 1:]
    labels[~label_valid] = ignore_index
    causal = torch.tril(torch.ones(T, T, dtype=torch.bool))
    allowed = causal[None, :, :] & valid[:, None, :]
    return ids, valid, allowed, labels, label_valid

def E02():
    ids, valid, allowed, labels, loss_mask = make_batch(
        [[2, 5, 6, 3], [2, 8, 3]])
    B, T = ids.shape
    scores = torch.randn(B, T, T)
    weights = scores.masked_fill(~allowed, float("-inf")).softmax(-1)
    assert torch.all(weights.masked_select(~allowed) == 0)
    logits = torch.randn(B, T, 12)
    loss = F.cross_entropy(logits.flatten(0, 1), labels.flatten())
    manual = F.cross_entropy(logits[loss_mask], labels[loss_mask])
    assert torch.allclose(loss, manual)
    return ids, allowed, labels, loss
```

Part 5 - The Philosophy Behind the Exercise

Padding reveals a broad principle: physical representation and semantic content are not the same thing. Rectangular tensors are chosen for efficient computation, while masks restore the

irregular structure of the data.

A mask is therefore a statement of permission. It declares which information may be read and which predictions may become evidence for learning. Correct masking is not cleanup around the model; it defines the computation the model is allowed to perform.

17.6.4 Exercise 03

Exercise Title: Implement Subword Tokenization Conceptually and Compare It to Character Tokenization on Vocabulary Size

Part 1 - Why this matters

Character tokenization has excellent coverage and a small vocabulary, but it creates long sequences. Word tokenization creates shorter sequences but an unmanageable vocabulary and severe unknown-token behavior. Subword methods occupy the space between them: frequent patterns become single tokens while rare words remain representable as smaller pieces.

Implement a small pair-merging tokenizer inspired by byte-pair encoding. Begin with characters plus an end-of-word marker, count adjacent symbol pairs, merge the most frequent pair with deterministic tie-breaking, and repeat for a fixed budget. This pedagogical version need not reproduce a production BPE library; its purpose is to expose how the vocabulary and segmentation co-evolve. Save the ordered merge rules because tokenization cannot be reproduced from the final vocabulary alone.

Compare character and subword representations on the same train and evaluation texts. Measure vocabulary size, average tokens per word, sequence length, unknown rate, and round-trip fidelity. Include common compounds, rare words, misspellings, and text from outside the training domain. A smaller token count is not automatically better if the vocabulary table becomes expensive or segmentation becomes brittle.

Part 2 - Mathematical form and reasoning

At merge step m, let $c_m(a, b)$ be the frequency of adjacent pair (a, b) in the segmented training corpus. Select

$$(a_m, b_m) = \arg\max_{(a,b)} c_m(a, b)$$

using a declared tie rule, replace each non-overlapping occurrence by $a_m \cdot b_m$, and append that rule to an ordered list. The practical cost is governed by both table size V and token length T: embedding parameters scale as Vd, while dense self-attention scales approximately as $T^2 d$. Tokenization chooses a tradeoff between these costs.

Part 3 - Hints, controls, and evidence

1. Keep the end-of-word symbol so concatenation remains reversible and word boundaries are not guessed during decoding.
2. Apply learned merges in their recorded order at evaluation time.
3. Plot vocabulary size against mean and percentile sequence lengths across several merge budgets.

4. Separate training coverage from evaluation coverage and never learn merges from evaluation text.

5. Inspect segmentations individually; averages can hide catastrophic splits for names, numbers, or another writing system.

Part 4 - Python code skeleton

```
from collections import Counter

def word_symbols(word):
    return tuple(list(word) + ["</w>"])

def pair_counts(corpus):
    counts = Counter()
    for symbols, freq in corpus.items():
        for pair in zip(symbols, symbols[1:]):
            counts[pair] += freq
    return counts

def merge_pair(symbols, pair):
    out, i = [], 0
    while i < len(symbols):
        if i + 1 < len(symbols) and tuple(symbols[i:i+2]) == pair:
            out.append(symbols[i] + symbols[i+1]); i += 2
        else:
            out.append(symbols[i]); i += 1
    return tuple(out)

def train_merges(words, steps):
    corpus = Counter(word_symbols(w) for w in words)
    merges = []
    for _ in range(steps):
        counts = pair_counts(corpus)
        if not counts: break
        pair = min(counts, key=lambda p: (-counts[p], p))
        corpus = Counter({merge_pair(list(s), pair): f
                          for s, f in corpus.items()})
        merges.append(pair)
    return merges

def E03():
    words = "low lower lowest new newer widest".split()
    merges = train_merges(words, steps=8)
    # TODO: encode held-out words and compare with characters.
    return merges
```

Part 5 - The Philosophy Behind the Exercise

No tokenization is a neutral discovery of the units already present in language. It is a compression scheme that decides which patterns deserve an atomic name. Those decisions redistribute difficulty between the vocabulary, the sequence, and the model.

Subwords are powerful because they refuse the false choice between complete memorization and complete decomposition. They let frequent structure become convenient while preserving a path for novelty.

17.6.5 Exercise 04

Exercise Title: Debug Off-by-One Token Alignment Bugs in Next-Token Prediction Dataset Building

Part 1 - Why this matters

Next-token prediction is defined by a one-position shift. For a token sequence `[BOS, a, b, EOS]`, the inputs are `[BOS, a, b]` and the labels are `[a, b, EOS]`. A zero shift teaches copying; a two-position shift teaches a different forecasting task. All three pipelines produce compatible tensor shapes, finite losses, and gradients, so shape checks alone cannot find the error.

Build a token-by-token alignment ledger showing input position, visible prefix, target ID, and decoded target. Assert semantic relationships before batching. Then deliberately create zero-shift, two-shift, and wrap-around variants and observe their behavior on a simple sequence. A suspiciously easy loss may be evidence of label leakage rather than successful learning.

Define boundary behavior explicitly. The first input often begins with BOS; the last prediction may target EOS. Sliding windows must not use `roll` without removing the wrapped token. Document whether the model returns logits that a loss function shifts internally or whether the dataset already provides shifted labels; applying both conventions creates a double shift.

Part 2 - Mathematical form and reasoning

For full sequence $z = (z_0, \ldots, z_L)$, the causal training pair is

$$x_t = z_t, \qquad y_t = z_{t+1}, \qquad 0 \le t < L.$$

The log-likelihood factorization is

$$\log p(z_1, \ldots, z_L \mid z_0) = \sum_{t=0}^{L-1} \log p(z_{t+1} \mid z_0, \ldots, z_t).$$

This formula is the semantic oracle: label y_t must be the token immediately after the last token available at input position t. Equality of shapes does not imply equality of this conditional prediction task.

Part 3 - Hints, controls, and evidence

1. Begin with a sequence of unique, readable tokens so every shift is visible.
2. Assert `inputs[1:] == labels[:-1]` before padding.

3. Verify window start/end indices and forbid cross-document targets unless documents are intentionally packed with boundary tokens.

4. Decode at least the first and last training examples after collation.

5. Record loss for correct, copy-label, and shuffled-label controls; explain why the copy-label control can look deceptively strong.

Part 4 - Python code skeleton

```
import torch

def causal_pair(full_ids):
    assert len(full_ids) >= 2
    x = torch.tensor(full_ids[:-1], dtype=torch.long)
    y = torch.tensor(full_ids[1:], dtype=torch.long)
    assert torch.equal(x[1:], y[:-1])
    return x, y

def windows(ids, context):
    examples = []
    for start in range(0, len(ids) - context):
        chunk = ids[start:start + context + 1]
        x, y = causal_pair(chunk)
        examples.append((x, y))
    return examples

def alignment_ledger(x, y, itos):
    return [{"position": t, "input": itos[x[t].item()],
             "target": itos[y[t].item()]}
            for t in range(len(x))]

def E04():
    itos = ["<pad>", "<bos>", "A", "B", "C", "<eos>"]
    ids = [1, 2, 3, 4, 5]
    x, y = causal_pair(ids)
    wrong_copy = x.clone()
    assert not torch.equal(y, wrong_copy)
    return alignment_ledger(x, y, itos)
```

Part 5 - The Philosophy Behind the Exercise

Supervision is a claim about time: given this information, predict what comes next. An off-by-one bug alters that claim without altering the tensor's appearance. The model then optimizes faithfully for a task the engineer never intended.

This exercise teaches that semantics must be tested at the boundary where data becomes a learning objective. Numerical success cannot rescue a falsely stated question.

17.6.6 Exercise 05

Exercise Title: Build Dataset and Collate Function that Produces Input IDs, Attention Masks, and Labels for Training

Part 1 - Why this matters

The dataset defines examples; the collate function turns several examples into one tensor contract. Keeping those responsibilities separate makes variable lengths, dynamic padding, truncation, and debugging manageable. A collator that returns plausible tensors can still corrupt training by padding labels with a token ID, shifting after padding, or truncating EOS without recording the policy.

Implement a dataset that stores complete encoded sequences and creates causal input/label pairs without padding. The collator should pad each batch only to its local maximum length. It may optionally round that maximum to a convenient hardware multiple. Return `input_ids`, a Boolean attention mask, and `labels` padded with `-100`. State every axis: all three tensors are batch-first, and every location refers to the same example and time step.

Test the pipeline at three boundaries: inspect individual dataset items, inspect the first collated batch, and decode selected rows after collation. Prove that reordering the examples reorders rows but does not change each example's content. Measure padding ratio because dynamic batching may be correct yet waste most computation when lengths differ greatly.

Part 2 - Mathematical form and reasoning

For item n with causal pair $(x^{(n)}, y^{(n)})$ of length L_n, let $T = \max_n L_n$. The collated tensors satisfy

$$X_{n,t} = \begin{cases} x_t^{(n)} & t < L_n \\ i_{\text{pad}} & t \geq L_n, \end{cases} \qquad P_{n,t} = \mathbb{1}[t < L_n], \qquad Y_{n,t} = \begin{cases} y_t^{(n)} & t < L_n \\ -100 & t \geq L_n. \end{cases}$$

The useful-token fraction is $\rho = \sum_n L_n/(BT)$. Reporting $1 - \rho$ reveals padding waste independently of model throughput.

Part 3 - Hints, controls, and evidence

1. Reject sequences shorter than two tokens before creating a pair.
2. Apply truncation before the one-token shift and state whether EOS is preserved or replaced at the boundary.
3. Assert `attention_mask == (labels != -100)` for the simple causal case; document exceptions for other tasks.
4. Use a batch with lengths 2, 5, and 9 to make padding visible.
5. Return lengths or example IDs in debug mode so a failing row can be traced to its raw source.

Part 4 - Python code skeleton

```
import torch
from torch.utils.data import Dataset

class CausalDataset(Dataset):
    def __init__(self, encoded, max_full_length=None):
        self.examples = []
        for full in encoded:
            if max_full_length is not None:
                full = full[:max_full_length]
            if len(full) >= 2:
                self.examples.append(torch.tensor(full, dtype=torch.long))
    def __len__(self): return len(self.examples)
    def __getitem__(self, i):
        z = self.examples[i]
        return {"input_ids": z[:-1], "labels": z[1:], "id": i}

def collate_causal(items, pad_id=0, multiple=1):
    B = len(items)
    T0 = max(len(x["input_ids"]) for x in items)
    T = ((T0 + multiple - 1) // multiple) * multiple
    ids = torch.full((B, T), pad_id, dtype=torch.long)
    labels = torch.full((B, T), -100, dtype=torch.long)
    mask = torch.zeros(B, T, dtype=torch.bool)
    for b, item in enumerate(items):
        L = len(item["input_ids"])
        ids[b, :L] = item["input_ids"]
        labels[b, :L] = item["labels"]
        mask[b, :L] = True
    assert torch.equal(mask, labels.ne(-100))
    return {"input_ids": ids, "attention_mask": mask,
            "labels": labels, "example_ids": [x["id"] for x in items]}

def E05():
    ds = CausalDataset([[1, 4, 2], [1, 7, 8, 9, 2]])
    return collate_causal([ds[0], ds[1]], multiple=4)
```

Part 5 - The Philosophy Behind the Exercise

A batch is not merely several examples stacked together. It is a temporary agreement that maps irregular semantic objects into regular computation. Collation is where that agreement becomes executable.

Good data engineering preserves provenance while enabling efficiency. The ideal batch is fast to process, but it must also remain possible to point from any tensor cell back to the token, example, and rule that created it.

17.6.7 Exercise 06

Exercise Title: Implement Label Smoothing for Language Modeling and Measure Its Effect on Perplexity and Calibration

Part 1 - Why this matters

Ordinary cross entropy treats the observed target as probability one and every other token as probability zero. Natural-language supervision is rarely that certain: several continuations can be reasonable, annotation may be noisy, and a finite corpus does not describe the whole distribution. Label smoothing reduces the reward for extreme confidence by assigning a small target mass to alternative classes.

Implement smoothed cross entropy manually before using the framework option. Apply it only to valid target positions; padding must not receive either the one-hot or uniform component. Decide whether the smoothing distribution covers all vocabulary IDs or excludes structurally impossible special tokens, and document the choice. Compare $\epsilon = 0$ with several nonzero values under the same initialization and training budget.

Measure validation negative log-likelihood, token accuracy, expected calibration error, confidence histograms, and perplexity. Compute reported perplexity from the ordinary unsmoothed validation NLL even when training uses smoothing; exponentiating the smoothed training objective produces a number that is not directly comparable to standard perplexity. A better calibration score accompanied by worse likelihood is a tradeoff, not an automatic win.

Part 2 - Mathematical form and reasoning

For target class y, vocabulary size V, and smoothing ϵ, one common target distribution is

$$q_k = \begin{cases} 1 - \epsilon, & k = y, \\ \epsilon/(V-1), & k \neq y. \end{cases} \qquad L_{\text{smooth}} = -\sum_{k=1}^{V} q_k \log p_k.$$

For valid positions $\mathcal{I}$, the batch loss is $|\mathcal{I}|^{-1} \sum_{i \in \mathcal{I}} L_i$. Standard perplexity remains

$$\text{PPL} = \exp\left(-\frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \log p_i(y_i) \right).$$

Part 3 - Hints, controls, and evidence

1. Verify the manual loss equals ordinary cross entropy at $\epsilon = 0$.
2. Assert every smoothed target row sums to one and ignored rows contribute exactly zero.
3. Fit calibration bins using predicted confidence and correctness on held- out valid tokens only.
4. Repeat with matched seeds and report mean and dispersion, not only the best smoothing value.
5. Inspect rare tokens separately; uniform smoothing can affect them differently from frequent tokens.

Part 4 - Python code skeleton

```
import math, torch
import torch.nn.functional as F

def smoothed_ce(logits, labels, eps, ignore_index=-100):
    V = logits.size(-1)
    flat_logits = logits.reshape(-1, V)
    flat_labels = labels.reshape(-1)
    valid = flat_labels.ne(ignore_index)
    logp = F.log_softmax(flat_logits[valid].float(), dim=-1)
    y = flat_labels[valid]
    nll = -logp.gather(1, y[:, None]).squeeze(1)
    other = -(logp.sum(1) - logp.gather(1, y[:, None]).squeeze(1))
    other = other / (V - 1)
    return ((1 - eps) * nll + eps * other).mean()

@torch.no_grad()
def standard_nll_and_ppl(logits, labels):
    loss = F.cross_entropy(logits.flatten(0, 1).float(),
                           labels.flatten(), ignore_index=-100)
    return loss.item(), math.exp(loss.item())

def E06():
    torch.manual_seed(6)
    logits = torch.randn(3, 5, 17, requires_grad=True)
    labels = torch.randint(0, 17, (3, 5)); labels[1, 4] = -100
    base = smoothed_ce(logits, labels, 0.0)
    ref = F.cross_entropy(logits.flatten(0, 1), labels.flatten())
    assert torch.allclose(base, ref)
    return {e: smoothed_ce(logits, labels, e).item()
            for e in (0.0, 0.05, 0.1, 0.2)}
```

Part 5 - The Philosophy Behind the Exercise

Label smoothing distinguishes learning from certainty. The objective should encourage the model to prefer the observed answer without pretending that the dataset is an infallible description of every possible future.

Yet humility cannot be inserted blindly. Smoothing changes the question being optimized, so its value must be judged through likelihood, calibration, and task behavior together. Responsible uncertainty is measured uncertainty.

17.6.8 Exercise 07

Exercise Title: Implement Tied Input-Output Embeddings and Explain Why Weight Tying Improves Parameter Efficiency

Part 1 - Why this matters

A language model first uses an embedding matrix to convert token IDs into vectors, then uses an output matrix to compare hidden states with vocabulary classes. Both matrices connect the same vocabulary to the same model space. Weight tying uses one *Parameter* for both roles instead of learning two independent tables. This can reduce a large fraction of model parameters and encourage a coherent input/output geometry.

Implement an embedding and a bias-free language-model head, then assign `lm_head.weight = token_embedding.weight`. Verify object identity and storage identity; merely copying equal values is not tying. The hidden dimension must equal the embedding dimension unless an additional projection is inserted. Count unique parameters before and after tying, perform a backward pass, and show that the shared table receives gradients from both lookup and output-logit paths.

Test checkpoint save/load and optimizer construction. Tie weights before creating the optimizer so a single shared parameter is updated once. Consider the padding row carefully: `padding_idx` suppresses gradient from the lookup path, but the same row can still receive output-head gradient because PAD is one of the classes. Masking labels is not equivalent to removing PAD from the output normalization.

Part 2 - Mathematical form and reasoning

With $E \in \mathbb{R}^{V \times d}$ and hidden state $h_t \in \mathbb{R}^d$,

$$x_t = E_{i_t,:}, \qquad z_t = Eh_t + b, \qquad p_t = \text{softmax}(z_t).$$

Untied input and output matrices use $2Vd$ weights; tying uses Vd weights, saving Vd parameters. The total gradient is the sum of contributions from both computational uses:

$$\nabla_E L = (\nabla_E L)_{\text{input}} + (\nabla_E L)_{\text{output}}.$$

Part 3 - Hints, controls, and evidence

1. Compare `id(weight)`, `data_ptr()`, and optimizer parameter IDs; equality of tensor values is insufficient.
2. Modify one element under `no_grad` and prove both modules observe the change.
3. Measure parameter count and checkpoint size for tied and untied variants.
4. Isolate gradient paths by detaching the input use, then the output use.
5. Verify tying survives state-dict loading and device/dtype conversion.

Part 4 - Python code skeleton

```
import torch
from torch import nn

class TiedLanguageHead(nn.Module):
    def __init__(self, vocab_size, d_model, pad_id=0, tied=True):
```

```
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, d_model,
                                      padding_idx=pad_id)
        self.lm_head = nn.Linear(d_model, vocab_size, bias=False)
        if tied:
            self.lm_head.weight = self.embedding.weight
    def forward(self, ids):
        hidden = self.embedding(ids)
        return self.lm_head(hidden)

def unique_parameters(module):
    return {id(p): p for p in module.parameters()}

def E07():
    model = TiedLanguageHead(101, 32, tied=True)
    assert model.embedding.weight is model.lm_head.weight
    input_ptr = model.embedding.weight.data_ptr()
    output_ptr = model.lm_head.weight.data_ptr()
    assert input_ptr == output_ptr
    assert sum(p.numel() for p in unique_parameters(model).values()) == 101*32
    ids = torch.tensor([[1, 4, 9, 2]])
    model(ids).sum().backward()
    assert model.embedding.weight.grad is model.lm_head.weight.grad
    return model
```

Part 5 - The Philosophy Behind the Exercise

Weight tying expresses a symmetry: the space used to understand a token can also be used to ask which token a hidden state resembles. It turns two interfaces into two views of one representational language.

Parameter efficiency here is not compression after thought; it is a structural hypothesis about meaning. Sharing is valuable when the roles genuinely belong to the same geometry, and dangerous when that assumption is left unexamined.

17.6.9 Exercise 08

Exercise Title: Implement Positional Encoding Integration into Embeddings and Confirm Shapes and Device Placement

Part 1 - Why this matters

Token embeddings say what appears; positional representations say where it appears. Their sum is the actual input to the first Transformer block. Because both tensors share the external shape `(B,T,d_model)`, an off-by-one position offset or a device mismatch can hide until a later stage. Build the integration as a named module with a written contract rather than an anonymous addition in the training loop.

Implement token lookup, optional multiplication by $\sqrt{d}$, selection of positional rows, addition, and dropout. Support an explicit `offset` so cached autoregressive generation can assign

positions continuing after a prefix. Assert that `offset + T` does not exceed the supported maximum. Position tensors must be on the same device as token states and must have a compatible floating dtype. Token IDs remain integer.

Padding masks are not added to embeddings. A padded location may receive a token and position vector for rectangular computation, but attention and loss masks decide whether it participates semantically. Compare two identical token sequences with different offsets and two different batches at the same offset to confirm exactly which values should match.

Part 2 - Mathematical form and reasoning

For token ID $I_{b,t}$ and absolute position $o + t$,

$$H^{(0)}_{b,t,:} = \alpha E_{I_{b,t},:} + P_{o+t,:}, \qquad \alpha \in \{1, \sqrt{d}\}.$$

$E \in \mathbb{R}^{V \times d}$, $P \in \mathbb{R}^{T_{\max} \times d}$, and the result has shape $B \times T \times d$. Broadcasting of $P[o : o + T]$ over the batch axis is intentional; broadcasting over a feature or sequence axis is not.

Part 3 - Hints, controls, and evidence

1. Use registered buffers for fixed encodings and Parameters for learned positions. Then device conversion will behave correctly.
2. Test $B \neq T \neq d$, offsets zero and nonzero, and the exact maximum supported length.
3. Disable dropout for deterministic equality tests, then verify stochastic behavior appears only in training mode.
4. Inspect state dicts: learned positions belong in parameters; fixed encodings may be persistent or reproducibly regenerated by policy.
5. Deliberately leave positions on CPU during a CUDA pass and confirm the device audit finds the error before addition.

Part 4 - Python code skeleton

```
import math, torch
from torch import nn

class InputEmbeddings(nn.Module):
    def __init__(self, vocab, d_model, max_length, pad_id=0, dropout=0.1):
        super().__init__()
        self.tokens = nn.Embedding(vocab, d_model, padding_idx=pad_id)
        self.positions = nn.Embedding(max_length, d_model)
        self.scale = math.sqrt(d_model)
        self.dropout = nn.Dropout(dropout)
    def forward(self, input_ids, offset=0):
        B, T = input_ids.shape
        if offset < 0 or offset + T > self.positions.num_embeddings:
            raise ValueError("position range exceeds configured maximum")
        position_ids = torch.arange(offset, offset + T,
```

```
                                    device=input_ids.device)
        x = self.tokens(input_ids) * self.scale
        p = self.positions(position_ids)[None, :, :]
        assert x.device == p.device and x.dtype == p.dtype
        return self.dropout(x + p)

def E08(device="cpu"):
    module = InputEmbeddings(50, 12, 32, dropout=0.0).to(device)
    ids = torch.tensor([[1, 5, 2], [1, 5, 2]], device=device)
    y0, y4 = module(ids, 0), module(ids, 4)
    assert y0.shape == (2, 3, 12) and torch.equal(y0[0], y0[1])
    assert not torch.equal(y0, y4)
    return y0, y4
```

Part 5 - The Philosophy Behind the Exercise

Representation is formed by superposition: identity and location become one state that later layers can reinterpret. The addition is simple, but the conceptual result is profound—the same token can mean something different because of where it occurs.

Interfaces of great consequence are often numerically small. A single tensor addition can encode the distinction between a bag of symbols and an ordered sequence. Simplicity of code must not be mistaken for simplicity of meaning.

17.6.10 Exercise 09

Exercise Title: Build a Full Tokenization Debugging Checklist Covering Special Tokens, Padding Rules, and Shifted Labels

Part 1 - Why this matters

Token-pipeline failures are unusually deceptive. A changed vocabulary order, missing EOS, inverted mask, or double-shifted label can leave every tensor finite and every matrix multiplication legal. A debugging checklist must therefore test semantic properties at successive boundaries, not merely run the model and wait for an exception.

Turn the checklist into executable checks. Cover normalization and pre- tokenization, stable special IDs, vocabulary uniqueness, encode/decode behavior, unknown-token policy, range checks, causal shifting, truncation, padding, attention masks, loss masks, position indices, dtype/device placement, serialization, and training/serving parity. Each failure message should include the example ID and the earliest boundary at which the contract broke.

Build a fault-injection table. Swap BOS and EOS, pad labels with PAD rather than `-100`, invert a mask, shift labels twice, rebuild the vocabulary with a different tie order, and remove a Unicode normalization step. For each fault, state which test should fail first. A useful checklist is judged by detection latency and diagnostic specificity, not by length.

Part 2 - Mathematical form and reasoning

Model the pipeline as functions

$$s \xrightarrow{\tau} u \xrightarrow{v} z \xrightarrow{g} (x, y) \xrightarrow{c} (X, Y, P) \xrightarrow{m} L.$$

If an end-to-end property fails, search for the smallest k such that the output of stage k violates its local invariant. Examples include $v^{-1}(v(u)) = u$ for known tokens, $x_{t+1} = y_t$ for causal pairs, and $P_{b,t} = 0 \Rightarrow Y_{b,t} = -100$ for the basic collator. Local invariants convert an opaque bad loss into a bounded diagnosis.

Part 3 - Hints, controls, and evidence

1. Store golden encoded examples, including punctuation, Unicode, empty text, unknowns, and maximum-length input.
2. Separate universal invariants from configurable policies. Record every selected policy.
3. Check tokenizer and vocabulary fingerprints when loading every checkpoint.
4. Decode random training batches periodically; raw IDs conceal semantic errors that become obvious in text.
5. Run every injected fault and record whether it was detected, at which stage, and with what message.

Part 4 - Python code skeleton

```
import hashlib, json, torch

def vocab_fingerprint(tokens, config):
    payload = json.dumps({"tokens": tokens, "config": config},
                         ensure_ascii=False, sort_keys=True)
    return hashlib.sha256(payload.encode("utf-8")).hexdigest()

def audit_batch(batch, vocab_size, pad_id):
    x, y, p = (batch["input_ids"], batch["labels"],
               batch["attention_mask"])
    assert x.dtype == y.dtype == torch.long
    assert p.dtype == torch.bool
    assert x.shape == y.shape == p.shape
    assert torch.all((0 <= x) & (x < vocab_size))
    assert torch.all(y[~p] == -100)
    assert torch.all(x[~p] == pad_id)
    valid_y = y[y != -100]
    assert torch.all((0 <= valid_y) & (valid_y < vocab_size))
    return True

def audit_pair(full, x, y):
    assert torch.equal(x, full[:-1])
    assert torch.equal(y, full[1:])
    assert torch.equal(x[1:], y[:-1])
```

```
def E09(tokenizer, tokens, config, batch):
    checks = {"fingerprint": vocab_fingerprint(tokens, config),
              "batch": audit_batch(batch, len(tokens), config["pad_id"])}
    # TODO: run golden round trips and all fault injections.
    return checks
```

Part 5 - The Philosophy Behind the Exercise

A checklist is institutional memory made executable. It preserves lessons that would otherwise be rediscovered only after another expensive training run. Its purpose is not bureaucracy; it is to move reasoning from emergency response to routine prevention.

The best diagnostic system follows causality backward. It does not merely say that the final metric is wrong. It identifies the earliest broken promise from which the later failure became inevitable.

17.6.11 Exercise 10

Exercise Title: Train Embedding and Tokenization Pipeline on CUDA Efficiently and Benchmark Host-to-Device Transfer Costs

Part 1 - Why this matters

Moving a model to CUDA does not make the data pipeline fast. Text normalization and tokenization are usually CPU operations; batches of integer IDs are then copied to the accelerator. If tokenization, collation, or transfer takes longer than the training step, the GPU waits idle. Conversely, moving many tiny tensors individually can cost more than the useful computation.

Build an end-to-end benchmark with separate timers for CPU preparation, host-to-device transfer, forward/backward computation, and optimizer update. Use pinned host memory and `non_blocking=True` only as a matched pair, then verify whether they overlap transfer with computation in the actual pipeline. Pre-tokenize static text when appropriate, collate into contiguous batch tensors, and transfer a small dictionary of tensors once per step. Do not attempt to move Python strings to CUDA.

CUDA launches are asynchronous, so wall-clock timing without synchronization is misleading. Use CUDA events for GPU intervals or synchronize immediately before and after a measured region. Include warm-up iterations, multiple repetitions, median and percentile latency, tokens per second, and peak allocated memory. Validate output parity before interpreting speed.

Part 2 - Mathematical form and reasoning

Let N be valid tokens processed in measured time

$$t_{\text{step}} = t_{\text{prepare}} + t_{\text{transfer}} + t_{\text{compute}} - t_{\text{overlap}}.$$

Throughput is N/t_{step}. The transfer fraction $r = t_{\text{transfer}}/t_{\text{step}}$ is diagnostic only when intervals were measured consistently. Approximate transfer bandwidth is $B_{\text{copy}} = S/t_{\text{transfer}}$ for S bytes. The theoretical byte count must include IDs, masks, and labels; it must not include Python object memory that never reaches the device.

Part 3 - Hints, controls, and evidence

1. Establish a CPU correctness baseline and compare one fixed batch on CPU and CUDA before benchmarking.
2. Warm up allocator, kernels, and compilation paths; exclude warm-up from statistics.
3. Compare pageable/blocking, pinned/blocking, and pinned/nonblocking configurations with identical batches.
4. Record valid tokens and padded tokens separately; samples per second can hide padding waste.
5. Test several batch/sequence sizes because transfer dominance changes with arithmetic intensity.

Part 4 - Python code skeleton

```
import time, torch

def move_batch(batch, device):
    return {k: v.to(device, non_blocking=True)
            for k, v in batch.items() if torch.is_tensor(v)}

def benchmark_step(model, batch, repeats=50):
    assert torch.cuda.is_available()
    device = torch.device("cuda")
    model.to(device).train()
    start = torch.cuda.Event(enable_timing=True)
    end = torch.cuda.Event(enable_timing=True)
    times = []
    for i in range(10 + repeats):
        gpu_batch = move_batch(batch, device)
        start.record()
        logits = model(gpu_batch["input_ids"],
                       gpu_batch["attention_mask"])
        loss = torch.nn.functional.cross_entropy(
            logits.flatten(0, 1), gpu_batch["labels"].flatten(),
            ignore_index=-100)
        loss.backward(); model.zero_grad(set_to_none=True)
        end.record(); end.synchronize()
        if i >= 10: times.append(start.elapsed_time(end))
    valid = batch["attention_mask"].sum().item()
    return {"median_ms": torch.tensor(times).median().item(),
            "valid_tokens_per_step": valid}

def E10(model, pinned_batch):
    assert all((not v.is_cuda) for v in pinned_batch.values()
               if torch.is_tensor(v))
    return benchmark_step(model, pinned_batch)
```

Part 5 - The Philosophy Behind the Exercise

Acceleration is a property of a system, not a device label. A fast GPU cannot compute data that has not arrived, and asynchronous execution can make poor measurement look like improvement.

The broader lesson is that performance claims require a causal accounting of time. Optimization begins by finding where the system waits, then changing the specific boundary responsible for that wait.

17.6.12 Exercise 11

Exercise Title: Benchmark DataLoader Performance for Token Sequences and Optimize num_workers, pin_memory, and prefetching

Part 1 - Why this matters

A DataLoader connects storage and CPU preprocessing to accelerator training. Its tuning parameters are not universal performance switches. `num_workers` can parallelize expensive tokenization, but too many workers increase process startup, memory duplication, and scheduling overhead. `pin_memory` can accelerate transfer to CUDA, but costs page-locked host memory. `prefetch_factor` hides latency only when future batches can be prepared without exhausting RAM.

Construct a configuration grid across worker counts, pinned versus pageable memory, persistent workers, and prefetch depths. Benchmark two stages: loader- only throughput on CPU and end-to-end training throughput on CUDA. A loader can produce batches quickly in isolation yet still fail to overlap with training. Use the same dataset order, batch size, collation policy, and number of measured valid tokens for every configuration.

Separate cold-start from steady state. Persistent workers can improve later epochs while making the first epoch appear unchanged. Control randomness by seeding each worker and dataset sampler. Detect duplicate or missing example IDs; faster batches are invalid if multi-worker partitioning silently repeats data.

Part 2 - Mathematical form and reasoning

If batch preparation time is t_p and consumption time is t_c, ideal overlap approaches

$$\max(t_p, t_c)$$

per batch rather than $t_p + t_c$. Define starvation fraction

$$s = \frac{t_{\text{waiting for next batch}}}{t_{\text{epoch}}}.$$

Report useful-token throughput

$$R = \frac{\sum_b \sum_t P_{b,t}}{t_{\text{measured}}},$$

not only raw batches per second. Different length distributions can make these metrics disagree.

Part 3 - Hints, controls, and evidence

1. Run each configuration several times after warm-up and randomize the configuration order to reduce thermal and cache bias.
2. Monitor host RAM, pinned memory, CPU utilization, queue wait, GPU utilization, and valid-token throughput.
3. Use `persistent_workers` only when `num_workers > 0`.
4. Measure pre-tokenized and on-the-fly-tokenized datasets separately.
5. Select the smallest configuration statistically indistinguishable from the fastest one; spare resources improve system stability.

Part 4 - Python code skeleton

```
import random, time, torch
from torch.utils.data import DataLoader

def seed_worker(worker_id):
    seed = torch.initial_seed() % (2**32)
    random.seed(seed)

def loader_trial(dataset, collate_fn, cfg, batches=100):
    kwargs = dict(batch_size=cfg["batch_size"], shuffle=False,
                  num_workers=cfg["workers"],
                  pin_memory=cfg["pin_memory"],
                  collate_fn=collate_fn, worker_init_fn=seed_worker)
    if cfg["workers"] > 0:
        kwargs.update(prefetch_factor=cfg["prefetch"],
                      persistent_workers=cfg["persistent"])
    loader = DataLoader(dataset, **kwargs)
    seen, tokens = [], 0
    start = time.perf_counter()
    for i, batch in enumerate(loader):
        seen.extend(batch["example_ids"])
        tokens += batch["attention_mask"].sum().item()
        if i + 1 == batches: break
    elapsed = time.perf_counter() - start
    assert len(seen) == len(set(seen))
    return {"seconds": elapsed, "valid_tokens_s": tokens / elapsed}

def E11(dataset, collate_fn):
    configs = [{"batch_size": 32, "workers": w,
                "pin_memory": p, "prefetch": 2,
                "persistent": w > 0}
               for w in (0, 2, 4, 8) for p in (False, True)]
    return [(c, loader_trial(dataset, collate_fn, c)) for c in configs]
```

Part 5 - The Philosophy Behind the Exercise

Concurrency is useful only when it removes a real dependency. More workers do not create information or computation; they rearrange when preparation happens and what resources remain occupied.

Performance tuning is therefore empirical resource coordination. The chosen configuration should be fast, reproducible, and stable within the limits of the whole machine, not merely the maximum of one benchmark column.

17.6.13 Exercise 12

Exercise Title: Compare float32 vs mixed precision embedding stability and detect overflow or underflow edge cases

Part 1 - Why this matters

Embedding lookup performs no long reduction; it gathers rows. This can make the layer seem automatically safe in reduced precision. Training behavior is more subtle. Embedding values are consumed by additions, normalization, attention, and output logits; rare-token gradients may be small, repeated-token gradients may be large, and tied output embeddings participate in vocabulary-wide matrix multiplication. Mixed precision affects the surrounding computation and the optimizer update even when indexing itself is exact.

Compare float32 with autocast mixed precision using identical initial weights, IDs, masks, labels, and optimizer steps. Inspect forward embeddings, logits, loss, row-wise gradients, and parameter deltas. Sweep the scale of embedding weights and upstream hidden states until nonfinite values, saturation, or updates rounded to zero appear. Include frequent, rare, unused, and padding rows. Keep IDs as `torch.long`; converting IDs to floating point is never part of mixed precision.

Use gradient scaling for CUDA training and log scale changes and skipped optimizer steps. Do not declare mixed precision stable because the scalar loss is finite: compare the exact rows that should have changed. In common optimizers, master parameters or optimizer states may remain float32 even when activations use lower precision; record the actual policy.

Part 2 - Mathematical form and reasoning

For rounding operator fl_q in precision q, lookup produces $\widehat{x} = \mathrm{fl}_q(E_i)$. A useful forward error measure is

$$\epsilon_x = \frac{\|x_{32} - \widehat{x}_q\|_2}{\|x_{32}\|_2 + \delta}.$$

For an update $\Delta E = -\eta g$, reduced precision may satisfy

$$\mathrm{fl}_q(E + \Delta E) = \mathrm{fl}_q(E)$$

when the update is below local representable spacing. This update-loss test is more informative than checking whether g was nonzero before casting.

Part 3 - Hints, controls, and evidence

1. Clone one float32 reference state into every precision condition.
2. Compare bfloat16 and float16 separately where hardware supports them; their range and precision differ.
3. Record absolute error, relative error, cosine similarity, non-finite counts, zero-gradient rows, and zero-update rows.
4. Force extreme scales in an isolated experiment, not in a valuable long training run.
5. Recheck tied embeddings because the output projection creates different numerical pressure than lookup alone.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def one_step(model, ids, labels, use_amp):
    model.zero_grad(set_to_none=True)
    before = model.embedding.weight.detach().float().clone()
    device_type = ids.device.type
    amp_dtype = torch.float16 if device_type == "cuda" else torch.bfloat16
    with torch.autocast(device_type=device_type, dtype=amp_dtype,
                        enabled=use_amp):
        logits = model(ids)
        loss = F.cross_entropy(logits.flatten(0, 1), labels.flatten())
    loss.backward()
    grad = model.embedding.weight.grad.detach().float().clone()
    with torch.no_grad():
        model.embedding.weight.add_(model.embedding.weight.grad, alpha=-1e-3)
    delta = model.embedding.weight.detach().float() - before
    return {"loss": loss.detach().float(), "grad": grad, "delta": delta,
            "finite": bool(torch.isfinite(logits).all())}

def E12(make_model, ids, labels):
    ref = make_model().to(ids.device)
    amp = make_model().to(ids.device); amp.load_state_dict(ref.state_dict())
    a = one_step(ref, ids, labels, False)
    b = one_step(amp, ids, labels, True)
    return {"fp32": a, "mixed": b,
            "delta_error": (a["delta"] - b["delta"]).abs().max().item()}
```

Part 5 - The Philosophy Behind the Exercise

Precision is a budget for distinctions. Reduced precision is valuable when it removes distinctions irrelevant to learning; it is harmful when it erases a small but necessary update or turns a large value into infinity.

The correct question is not whether a dtype is "accurate." It is whether the entire training process preserves the distinctions needed for the intended decision. Stability must be observed across values, gradients, and updates.

17.6.14 Exercise 13

Exercise Title: Visualize Embedding Space with Dimensionality Reduction Conceptually and Inspect Token Similarities

Part 1 - Why this matters

An embedding table may contain thousands of dimensions and cannot be inspected directly. Dimensionality reduction and nearest-neighbor analysis can reveal collapsed rows, frequency bands, outliers, duplicated special tokens, or local semantic structure. They are diagnostic lenses, not proofs that the model understands a concept. A two-dimensional projection discards information and can create apparent clusters that do not exist in the original geometry.

Extract trained embeddings together with token strings and training frequencies. Compare raw Euclidean neighborhoods with cosine neighborhoods after row normalization. Center the table before PCA and record how much variance the selected components explain. If using a nonlinear method such as t-SNE or UMAP, run multiple seeds and settings; local visual proximity may be sensitive to those choices. Always compare against the randomly initialized table.

Select queries before looking at the plot: frequent function tokens, rare tokens, morphological variants, punctuation, BOS/EOS/PAD, and known domain terms. Report nearest neighbors in the original d-dimensional space as the primary evidence and use the projection to organize questions. Test whether frequency or vector norm explains the pattern before calling it semantic.

Part 2 - Mathematical form and reasoning

Cosine similarity is

$$s(i,j) = \frac{E_i^\top E_j}{\|E_i\|_2 \|E_j\|_2}.$$

For centered table $E_c = E - \mathbf{1}\mu^\top$, PCA finds orthonormal directions W_k maximizing retained variance, and the projection is $Z = E_c W_k$. The explained variance ratio

$$r_k = \frac{\sum_{j=1}^{k} \lambda_j}{\sum_{j=1}^{d} \lambda_j}$$

states how much global linear variance remains. It does not measure preserved semantic meaning.

Part 3 - Hints, controls, and evidence

1. Exclude or separately label PAD and other special tokens; their training roles differ from ordinary text.
2. Compare initialized, partially trained, and final tables using the same queries and metrics.
3. Color by log frequency and size points by vector norm to expose confounding structure.
4. Report neighbor overlap across seeds and checkpoints rather than choosing one attractive figure.

5. Use downstream probes or controlled interventions before claiming that a visible direction causally represents an attribute.

Part 4 - Python code skeleton

```
import torch

def cosine_neighbors(weight, query_id, k=8, exclude=()):
    w = torch.nn.functional.normalize(weight.float(), dim=1)
    scores = w @ w[query_id]
    scores[list(exclude) + [query_id]] = -float("inf")
    values, ids = scores.topk(k)
    return list(zip(ids.tolist(), values.tolist()))

def pca_projection(weight, dimensions=2):
    x = weight.detach().float().cpu()
    x = x - x.mean(0, keepdim=True)
    _, singular, vectors = torch.pca_lowrank(x, q=min(x.shape))
    z = x @ vectors[:, :dimensions]
    explained = singular.square()
    ratio = explained[:dimensions].sum() / explained.sum()
    return z, ratio.item()

def E13(model, itos, query_ids, frequencies):
    weight = model.embedding.weight.detach()
    z, retained = pca_projection(weight)
    neighbors = {itos[q]: [(itos[i], s) for i, s in
                 cosine_neighbors(weight, q)] for q in query_ids}
    return {"projection": z, "variance_retained": retained,
            "neighbors": neighbors, "frequencies": frequencies}
```

Part 5 - The Philosophy Behind the Exercise

Visualization translates an inaccessible space into a human-scale question. It is valuable because it guides attention, but dangerous when the translation is mistaken for the original object. A map can reveal where to investigate; it cannot certify what the territory means.

Interpretability begins with disciplined modesty. Patterns become knowledge only when they survive alternative projections, quantitative tests, controls, and interventions.

17.6.15 Exercise 14

Exercise Title: Create Unit Tests that Validate Tokenization, Padding, and Label Alignment for Multiple Sequence Lengths

Part 1 - Why this matters

The token pipeline is deterministic enough to test exactly and important enough that approximate validation is inadequate. Unit tests should catch errors before a GPU training run

begins. They must cover both golden examples with known outputs and general properties over many lengths and token values.

Create tests for normalization, token boundaries, special IDs, unknowns, encode/decode round trips, vocabulary serialization, causal pairs, dynamic padding, attention masks, ignored labels, truncation, and position limits. Parameterize them over empty input, minimum valid length, typical lengths, maximum length, and mixed-length batches. Include Unicode, punctuation, repeated tokens, and unknown items. An all-empty batch should be rejected with a useful message rather than producing a zero-denominator loss.

Add fault-injection or mutation tests. Change one line of expected behavior— invert a mask, copy inputs into labels, use PAD as the ignored target, or swap special IDs—and prove at least one focused test fails. A suite that passes only the implementation that inspired it can miss shared assumptions; property tests and an independently written manual oracle reduce this risk.

Part 2 - Mathematical form and reasoning

Define an invariant set $\mathcal{C}$ over pipeline output. Examples are

$$0 \le X_{b,t} < V, \qquad P_{b,t} = 0 \Rightarrow X_{b,t} = i_{\text{pad}} \wedge Y_{b,t} = -100,$$

$$x_{t+1}^{(n)} = y_t^{(n)}, \qquad \text{decode}(\text{encode}(s)) \sim s,$$

where $\sim$ means equality under the declared normalization policy. A test suite samples inputs from the valid domain and verifies all applicable members of $\mathcal{C}$, while golden tests verify exact expected outputs.

Part 3 - Hints, controls, and evidence

1. Keep five-token golden cases readable enough to verify without code.
2. Use unequal sequence lengths and shuffle example order to expose hidden axis and state assumptions.
3. Test serialization round trips in a temporary directory and compare a stable vocabulary fingerprint.
4. Separate policy tests from universal invariants so intentional config changes require explicit golden-output updates.
5. Run the suite on CPU and, for tensor integration, one available CUDA device; tokenizer string logic remains a CPU concern.

Part 4 - Python code skeleton

```
import torch

def assert_batch_contract(batch, pad_id, vocab_size):
    x, y, p = batch["input_ids"], batch["labels"], batch["attention_mask"]
```

```
    assert x.shape == y.shape == p.shape
    assert x.dtype == y.dtype == torch.long and p.dtype == torch.bool
    assert ((0 <= x) & (x < vocab_size)).all()
    assert (x[~p] == pad_id).all() and (y[~p] == -100).all()
    assert (y[p] >= 0).all() and (y[p] < vocab_size).all()

def test_shift():
    full = torch.tensor([1, 8, 5, 2])
    x, y = full[:-1], full[1:]
    assert torch.equal(x[1:], y[:-1])
    assert x.tolist() == [1, 8, 5]
    assert y.tolist() == [8, 5, 2]

def test_lengths(dataset, collate, pad_id, vocab_size):
    for indices in ([0], [0, 1], [2, 0, 1]):
        batch = collate([dataset[i] for i in indices])
        assert_batch_contract(batch, pad_id, vocab_size)

def E14(dataset, collate, pad_id, vocab_size):
    test_shift()
    test_lengths(dataset, collate, pad_id, vocab_size)
    # TODO: golden tokenization, Unicode, unknown, truncation, serialization,
    # property cases, and injected faults.
    return "all pipeline tests passed"
```

Part 5 - The Philosophy Behind the Exercise

Tests are executable definitions of what the system means. They protect not only against syntax errors, but against gradual changes in normalization, special-token identity, and temporal alignment that would redefine the task.

The strongest test suite makes invisible assumptions visible. It allows the pipeline to evolve while forcing every semantic change to become a conscious, reviewable decision.

17.6.16 Exercise 15

Exercise Title: Implement Streaming Dataset Conceptually and Prevent Memory Blowups When Training on Large Text

Part 1 - Why this matters

Large corpora cannot always be tokenized and held in RAM before training. Streaming changes the data contract: examples arrive incrementally from ordered shards, and the pipeline must tokenize, buffer, shuffle, pack, and batch without materializing the entire corpus. Memory usage becomes bounded, but exact global shuffling and arbitrary indexing are no longer available.

Implement an `IterableDataset` that assigns disjoint shards to workers, reads records lazily, tokenizes each record, and emits causal chunks. Use a bounded shuffle buffer: fill it, choose a random item, replace it from the stream, and drain it at the end. This approximates shuffling

within a known memory budget. Record shard ID and record offset with every sample so duplicate or missing data can be diagnosed.

Define document-boundary policy. Either reset chunks at each document or pack documents with EOS separators; never let the target cross an invisible boundary. For distributed training, partition by rank as well as DataLoader worker. Design checkpoint resume around a recoverable cursor, RNG state, shard order, and buffer state. An epoch in a stream must have a declared meaning, such as a fixed manifest pass or a fixed token budget.

Part 2 - Mathematical form and reasoning

Let shards be $S_0, \ldots, S_{K-1}$ and let there be R ranks with W workers per rank. A simple disjoint assignment uses global consumer index $c = rW + w$ and selects shard S_j when

$$j \bmod (RW) = c.$$

If the shuffle buffer holds at most M encoded examples of maximum size L, its token storage is $O(ML)$ rather than $O(NL)$ for corpus size N. This bound must include Python and tokenizer object overhead in measured host memory, not only tensor bytes.

Part 3 - Hints, controls, and evidence

1. Test worker/rank assignments by collecting provenance IDs and proving pairwise disjointness and expected coverage.

2. Keep a manifest with stable shard checksums; resume is unsafe if source content changes under the same name.

3. Measure resident set size while consuming far more records than the buffer capacity; memory should reach a plateau.

4. Compare buffer sizes using neighbor correlation and token distribution, not the claim of perfect global shuffle.

5. Inject interruption, save state, resume, and prove no sample is silently replayed or skipped under the selected guarantee.

Part 4 - Python code skeleton

```
import random
from torch.utils.data import IterableDataset, get_worker_info

class StreamingText(IterableDataset):
    def __init__(self, shards, read_shard, encode, buffer_size=1024,
                 rank=0, world_size=1, seed=0):
        self.shards, self.read_shard, self.encode = shards, read_shard, encode
        self.buffer_size = buffer_size
        self.rank, self.world_size, self.seed = rank, world_size, seed
    def __iter__(self):
        info = get_worker_info()
        wid, workers = (0, 1) if info is None else (info.id, info.num_workers)
```

```
        consumer = self.rank * workers + wid
        consumers = self.world_size * workers
        rng, buffer = random.Random(self.seed), []
        for sid, shard in enumerate(self.shards):
            if sid % consumers != consumer: continue
            for offset, text in enumerate(self.read_shard(shard)):
                item = {"ids": self.encode(text),
                        "source": (sid, offset)}
                if len(buffer) < self.buffer_size:
                    buffer.append(item); continue
                j = rng.randrange(len(buffer))
                yield buffer[j]; buffer[j] = item
        rng.shuffle(buffer)
        yield from buffer

def E15(shards, read_shard, encode):
    return StreamingText(shards, read_shard, encode, buffer_size=256)
```

Part 5 - The Philosophy Behind the Exercise

Streaming replaces possession with disciplined passage. The system no longer needs to hold the dataset as one object; it needs guarantees about what enters, what has been consumed, and what can be reconstructed after interruption.

This is a shift from static data structure to temporal protocol. Bounded memory is achieved not by ignoring state, but by making the necessary state smaller, explicit, and recoverable.

17.6.17 Exercise 16

Exercise Title: Build a Minimal Training Dataset for Autoregressive Transformer and Verify Loss Decreases Correctly

Part 1 - Why this matters

Before training on a real corpus, a Transformer pipeline should deliberately overfit a tiny, inspectable dataset. This is not a generalization test. It is an integration test of tokenization, shifting, padding, causal masking, embeddings, logit dimensions, loss reduction, gradients, optimizer order, and generation. If a model with sufficient capacity cannot memorize a few repeated sequences, the pipeline is probably wrong.

Create several deterministic patterns containing BOS and EOS, split each into input and next-token labels, and train a minimal decoder-only model. Freeze the vocabulary after construction. Record the initial loss relative to the uniform baseline $\log V$, train until near-perfect token accuracy, and decode greedy continuations. Validate that loss decreases because the correct next-token probability rises, not because PAD labels leaked into the denominator or future tokens became visible.

Run negative controls. Shuffle labels while preserving their marginal distribution; remove causal masking; create copy labels; and freeze embeddings. The no-mask model may achieve an illegitimately easy loss, while shuffled labels should resist memorization unless capacity is

extreme. Inspect per-position loss because an average can hide a broken boundary token or one always-ignored column.

Part 2 - Mathematical form and reasoning

For valid target set $\mathcal{I}$,

$$L(\theta) = -\frac{1}{|\mathcal{I}|} \sum_{(b,t)\in\mathcal{I}} \log p_\theta(y_{b,t} \mid x_{b,\le t}).$$

A uniform model has expected NLL $\log V$. On a deterministic tiny corpus, a correct expressive model can approach $L \to 0$ and valid-token accuracy $\to 1$. The causal condition $x_{b,\le t}$ is essential: lower loss obtained from access to $x_{b,>t}$ does not validate autoregressive learning.

Part 3 - Hints, controls, and evidence

1. Use a vocabulary and sequences small enough to print in full.
2. Log gradients for token embeddings, positional embeddings, attention, FFN, and output head on the first step.
3. Track per-position NLL, valid-token accuracy, gradient norm, and selected next-token probabilities.
4. Generate from BOS after training; teacher-forced accuracy alone does not test autoregressive feedback.
5. Require the correct pipeline to pass and each injected fault to produce its predicted signature.

Part 4 - Python code skeleton

```
import math, torch
import torch.nn.functional as F

def train_tiny(model, loader, steps=500, lr=3e-3):
    optimizer = torch.optim.AdamW(model.parameters(), lr=lr)
    history = []
    iterator = iter(loader)
    for step in range(steps):
        try: batch = next(iterator)
        except StopIteration:
            iterator = iter(loader); batch = next(iterator)
        optimizer.zero_grad(set_to_none=True)
        logits = model(batch["input_ids"], batch["attention_mask"])
        loss = F.cross_entropy(logits.flatten(0, 1),
                               batch["labels"].flatten(),
                               ignore_index=-100)
        loss.backward(); optimizer.step()
        with torch.no_grad():
            valid = batch["labels"].ne(-100)
```

```
            predicted = logits.argmax(-1)[valid]
            expected = batch["labels"][valid]
            acc = (predicted == expected).float().mean()
        history.append((loss.item(), acc.item()))
    return history

def E16(model, loader, vocab_size):
    history = train_tiny(model, loader)
    assert history[-1][0] < min(math.log(vocab_size), history[0][0])
    assert history[-1][1] > 0.98
    # TODO: greedy generation and no-mask/copy/shuffled-label controls.
    return history
```

Part 5 - The Philosophy Behind the Exercise

The tiny-overfit test asks whether the entire learning machine can realize one small truth. It reduces complexity until excuses disappear. If memorization fails, adding more data and compute will conceal the defect rather than cure it.

Scientific confidence grows through controlled simplification. A small system whose behavior can be predicted is often a stronger foundation than a large system whose improving metric cannot be explained.

17.6.18 Exercise 17

Exercise Title: Stress Test Token Pipeline with Very Long Sequences and Detect Performance Bottlenecks and Memory Waste

Part 1 - Why this matters

Long context stresses every stage differently. Tokenization and Python object creation grow roughly with text length. Padding may multiply the number of stored tokens. Embeddings grow linearly with BTd, while dense attention-score storage grows with BHT^2. Positional tables and mask construction may become unexpected bottlenecks. A failure reported as "out of memory" is therefore not a diagnosis.

Build a safe stress matrix over sequence length, batch size, length variance, and vocabulary strategy. Increase one dimension at a time and run each case in an isolated process or with a conservative memory ceiling. Measure tokenization latency, collation latency, padding ratio, host memory, transfer time, device peak memory, forward/backward time, and valid tokens per second. Stop before resource exhaustion using an explicit threshold.

Partition memory conceptually into input IDs/masks/labels, embeddings, model parameters and optimizer states, attention/FFN activations, gradients, and allocator reserve. Compare measured increments with simple predictions. Test length bucketing, dynamic padding, document packing, truncation, and reduced batch size. These strategies change different terms, so report exactly which bottleneck each removes and what information tradeoff it introduces.

Part 2 - Mathematical form and reasoning

Approximate storage in bytes with element widths s_i and s_a:

$$M_{\text{ids}} \approx 2BTs_i + BTs_{\text{mask}}, \qquad M_{\text{embed}} \approx BTd\, s_a,$$

$$M_{\text{score}} \approx BHT^2 s_a.$$

Padding efficiency is $\rho = \sum_b L_b/(BT)$ and useful throughput is $R_{\text{useful}} = \rho BT/t$. Doubling T approximately quadruples the score term but only doubles ID and embedding terms. Deviations from these estimates help identify fused kernels, activation recomputation, or allocator effects.

Part 3 - Hints, controls, and evidence

1. Include a no-model loader benchmark, an embedding-only benchmark, a forward-only benchmark, and a full training benchmark.
2. Reset and read peak CUDA memory for each isolated configuration; record allocated and reserved memory separately.
3. Use synthetic length distributions as well as repeated fixed lengths to separate padding waste from quadratic attention cost.
4. Validate truncated/packed labels and document boundaries after every optimization.
5. Present Pareto points for context retained, useful throughput, and memory; do not recommend speed without its information cost.

Part 4 - Python code skeleton

```
import time, torch

def stress_case(model, make_batch, B, T, device="cuda"):
    if device == "cuda":
        torch.cuda.empty_cache(); torch.cuda.reset_peak_memory_stats()
    batch = make_batch(B, T)
    start = time.perf_counter()
    batch = {k: v.to(device) for k, v in batch.items()}
    if device == "cuda": torch.cuda.synchronize()
    transfer_s = time.perf_counter() - start
    start = time.perf_counter()
    logits = model(batch["input_ids"], batch["attention_mask"])
    loss = torch.nn.functional.cross_entropy(
        logits.flatten(0, 1), batch["labels"].flatten(), ignore_index=-100)
    loss.backward()
    if device == "cuda": torch.cuda.synchronize()
    compute_s = time.perf_counter() - start
    valid = batch["attention_mask"].sum().item()
    peak = (torch.cuda.max_memory_allocated() if device == "cuda" else None)
    return {"B": B, "T": T, "transfer_s": transfer_s,
            "compute_s": compute_s, "peak_bytes": peak,
            "useful_tokens_s": valid / compute_s}
```

```
def E17(model, make_batch):
    return [stress_case(model, make_batch, B, T)
            for T in (128, 256, 512, 1024) for B in (1, 4)]
```

Part 5 - The Philosophy Behind the Exercise

Scale does not merely enlarge a system; it changes which constraints dominate. An implementation correct at short length may reveal a quadratic assumption, an inefficient mask, or a hidden copy only when context grows.

Stress testing makes limits explicit before users discover them. Its purpose is not to force failure, but to learn the shape of the safe operating region and the price of extending it.

17.6.19 Exercise 18

Exercise Title: Build a Practical Guide for Choosing Tokenization Strategy and Maximum Sequence Length for Your Use Case

Part 1 - Why this matters

Tokenizer type, vocabulary size, and maximum sequence length are coupled system decisions. Character or byte units offer robust coverage but lengthen sequences. Large subword vocabularies shorten common text but enlarge embeddings and output projection, may fragment new domains poorly, and increase checkpoint size. Maximum length controls retained context while strongly affecting training memory and inference latency. There is no universally correct setting.

Build a decision guide beginning with requirements: languages and scripts, domain terminology, noise and spelling variation, reversibility, unknown-token tolerance, context needed for the target, memory ceiling, latency objective, data size, and deployment environment. Train candidate tokenizers on the same training split. Evaluate on in-domain and shifted data using unknown rate, tokens per character/word/document, tail sequence lengths, round-trip fidelity, vocabulary memory, and downstream quality under matched compute.

Choose maximum length from the distribution of information-bearing spans, not from the largest value the hardware can barely accept. Report the fraction of examples and targets truncated, the padding ratio after batching, and the quality/throughput curve. Consider chunking with overlap or retrieval when documents are long but the task depends on local evidence. Version every chosen artifact and list conditions that trigger reevaluation.

Part 2 - Mathematical form and reasoning

For tokenizer τ and maximum length T, define empirical coverage

$$C(T) = \frac{1}{N} \sum_{n=1}^{N} \mathbb{1}\left[|\tau(s_n)| \le T\right],$$

and retained-token ratio

$$R(T) = \frac{\sum_n \min(|\tau(s_n)|, T)}{\sum_n |\tau(s_n)|}.$$

A candidate can be represented by vector $(Q, R_{\text{useful}}, M, L, C)$ for quality, throughput, memory, latency, and coverage. Prefer Pareto-optimal candidates; a single weighted score should be used only when weights come from explicit product or research requirements.

Part 3 - Hints, controls, and evidence

1. Compare character, byte, word, and at least two subword vocabulary budgets on identical data splits.
2. Report percentiles, not only mean sequence length; the tail often controls memory and truncation policy.
3. Inspect languages, domains, rare terms, numbers, code, and whitespace separately.
4. Benchmark full model cost because a shorter sequence and larger vocabulary change different layers.
5. Write a default, rejected alternatives with evidence, and revision triggers such as new language coverage or a changed latency budget.

Part 4 - Python code skeleton

```
import statistics

def tokenizer_metrics(tokenizer, texts, max_length):
    lengths, unknowns, total = [], 0, 0
    for text in texts:
        ids = tokenizer.encode(text)
        lengths.append(len(ids)); total += len(ids)
        unknowns += sum(i == tokenizer.unk_id for i in ids)
    ordered = sorted(lengths)
    percentile = lambda q: ordered[min(len(ordered)-1,
                                       int(q * (len(ordered)-1)))]
    return {"vocab_size": tokenizer.vocab_size,
            "mean_length": statistics.mean(lengths),
            "p95_length": percentile(.95), "p99_length": percentile(.99),
            "unknown_rate": unknowns / max(total, 1),
            "example_coverage": (
                sum(x <= max_length for x in lengths) / len(lengths)),
            "retained_token_ratio": sum(min(x, max_length) for x in lengths)
                                    / max(sum(lengths), 1)}

def E18(candidates, train_texts, eval_slices, lengths=(128, 256, 512)):
    report = {}
    for name, tokenizer in candidates.items():
        tokenizer.train(train_texts)
        report[name] = {slice_name: {T: tokenizer_metrics(tokenizer, texts, T)
                         for T in lengths}
                        for slice_name, texts in eval_slices.items()}
    return report
```

Part 5 - The Philosophy Behind the Exercise

Choosing tokens and context length chooses what the model can see conveniently and what it must reconstruct through effort. These are epistemic boundaries implemented as engineering parameters.

A mature decision guide does not hide uncertainty behind a default. It connects each choice to requirements, measurements, rejected alternatives, and the future evidence that would justify change.

17.6.20 Exercise 19

Exercise Title: Produce a Full Tokenization and Embedding Engineering Report Summarizing Pipeline Choices and Failure Modes

Part 1 - Why this matters

The final report turns twenty experiments into an auditable pipeline contract. Without it, knowledge remains scattered among code, notebooks, plots, and the memory of whoever debugged the last incident. The report must allow another engineer to reproduce token IDs, construct identical batches, load embedding weights safely, interpret performance limits, and recognize known failure signatures.

Describe the complete path from raw bytes or text to model input: corpus split and provenance, normalization, pre-tokenization, learned subword rules, vocabulary ordering, special IDs, BOS/EOS insertion, unknown behavior, truncation, causal shifting, dynamic padding, attention/loss masks, embedding and positional integration, weight tying, and serialization. Include the exact shape, dtype, device, and semantics of every returned tensor. Record tokenizer and vocabulary fingerprints beside checkpoint versions.

Summarize correctness evidence from golden examples, property tests, fault injection, tiny overfitting, CPU/CUDA parity, and checkpoint reload. Summarize engineering evidence from DataLoader sweeps, transfer timing, mixed precision, streaming memory plateaus, and long-sequence scaling. Separate observation from interpretation. Document unresolved risks, supported operating bounds, fallback behavior, owners, and revision triggers. A report that lists only successful results is incomplete; failure modes are part of the design knowledge.

Part 2 - Mathematical form and reasoning

Organize the report around a chain of contracts

$$s \xrightarrow{\tau} u \xrightarrow{v} z \xrightarrow{\text{shift}} (x, y) \xrightarrow{\text{collate}} (X, Y, P) \xrightarrow{E+P_{\text{pos}}} H^{(0)}.$$

For every arrow f_k, specify domain $\mathcal{D}_k$, codomain $\mathcal{D}_{k+1}$, configuration c_k, invariant set $\mathcal{C}_k$, and evidence artifact a_k. Reproducibility requires

$$(s, c_1, \ldots, c_K, \text{artifact versions}) \mapsto (X, Y, P)$$

to be deterministic wherever promised. Performance claims should identify the valid region in $(B, T, V, d, \text{dtype}, \text{device})$, not be stated as a context-free speedup.

Part 3 - Hints, controls, and evidence

1. Begin with an executive decision table: chosen policy, evidence, tradeoff, rejected alternative, and revision trigger.
2. Include a schema example containing raw text, tokens, IDs, shifted labels, masks, and decoded verification in one traceable row.
3. Link every quantitative claim to configuration, seed, hardware/software environment, raw measurement, and aggregation method.
4. Maintain a failure registry with symptom, earliest violated invariant, root cause, detector, fix, and regression test.
5. End with a training/serving parity checklist and an incident procedure for fingerprint mismatch, excessive unknowns, non-finite values, or throughput regression.

Part 4 - Python code skeleton

```
from dataclasses import dataclass, asdict
import json

@dataclass
class PipelineDecision:
    name: str
    choice: str
    evidence: list
    tradeoffs: list
    rejected: list
    revision_triggers: list

def validate_report(report):
    required = ["provenance", "normalization", "tokenizer", "vocabulary",
                "special_tokens", "shift_policy", "padding_and_masks",
                "embeddings", "position_policy", "precision", "performance",
                "tests", "failure_registry", "serving_parity"]
    missing = [key for key in required if key not in report]
    if missing: raise ValueError(f"missing report sections: {missing}")
    assert report["vocabulary"]["fingerprint"]
    assert report["tests"]["tiny_overfit_passed"]
    return True

def E19(artifacts, decisions, failures):
    report = {**artifacts,
              "decisions": [asdict(x) for x in decisions],
              "failure_registry": failures}
    validate_report(report)
    with open("trf04_engineering_report.json", "w", encoding="utf-8") as f:
        json.dump(report, f, ensure_ascii=False, indent=2)
    return report
```

Part 5 - The Philosophy Behind the Exercise

An engineering report is the memory of a system made independent of an individual. It preserves not just what was built, but why it was built, where it fails, and what evidence would justify rebuilding it differently.

The chapter began by turning text into IDs. It ends by turning experiments into shared knowledge. Both acts require stable identity, explicit contracts, and a faithful path back to meaning.

17.7 TRF05

Chapter Title: Training Loop, Optimization, and Stability
Specialization: Models and Training

17.7.1 Exercise 00

Exercise Title: Build Full Transformer Training Loop with Correct Train Eval Modes, Loss Computation, and Gradient Zeroing

Part 1 - Why this matters

A training loop is the control system that turns model outputs into parameter changes. Each individual operation is simple, but their order defines the algorithm. The essential training sequence is: enter training mode, obtain a batch, clear stale gradients, compute logits, compute a masked loss, run backpropagation, optionally inspect or transform gradients, update parameters, and advance the scheduler at its documented frequency. Validation uses evaluation mode and disables gradient recording, but never updates the model.

Implement one transparent epoch before adding mixed precision, accumulation, distributed training, or compilation. For causal language modeling, flatten logits from `(B,T,V)` to `(BT,V)` and labels from `(B,T)` to `(BT)`, preserving the same position order. Use `ignore_index=-100` for padded targets and aggregate epoch loss by the number of valid target tokens, not by an unweighted mean of batch losses. The latter biases results when batches contain different amounts of padding.

Prove that the loop really updates what it should. Snapshot parameters before a step, inspect nonzero finite gradients after `backward`, and verify that at least one trainable parameter changes after `step`. Then inject two faults: omit `zero_grad` and call `eval()` during training. Show gradient accumulation in the first case and disabled dropout behavior in the second. Log sample IDs so a loss spike can be traced back to data.

Part 2 - Mathematical form and reasoning

For valid target positions $\mathcal{I}$, token loss is

$$L(\theta) = -\frac{1}{|\mathcal{I}|} \sum_{(b,t)\in\mathcal{I}} \log p_\theta(y_{b,t} \mid x_{b,\le t}).$$

One optimizer step computes

$$g_k = \nabla_\theta L_k(\theta_k), \qquad \theta_{k+1} = \mathcal{U}(\theta_k, g_k, s_k),$$

where s_k is optimizer state. Without gradient clearing, PyTorch instead stores $g_k \leftarrow g_{k-1} + \nabla L_k$, changing the effective algorithm. For an epoch, the correct aggregate is $\sum_k n_k L_k / \sum_k n_k$, where n_k is the valid-token count.

Part 3 - Hints, controls, and evidence

1. Write the exact order of operations and scheduler frequency before coding.
2. Assert logits, labels, masks, parameters, and loss share compatible devices and that all trainable gradients are finite after backward.
3. Compare a vectorized loss with a manual valid-token calculation on a tiny batch.
4. Track loss numerator and valid-token denominator separately across the epoch.
5. Save parameter-delta norms, gradient norms, learning rate, sample IDs, and train/eval mode for every debug step.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def token_loss(logits, labels):
    V = logits.size(-1)
    return F.cross_entropy(logits.reshape(-1, V), labels.reshape(-1),
                           ignore_index=-100, reduction="sum")

def train_epoch(model, loader, optimizer, device, scheduler=None):
    model.train()
    loss_sum, token_count = 0.0, 0
    for batch in loader:
        batch = {k: v.to(device, non_blocking=True)
                 for k, v in batch.items() if torch.is_tensor(v)}
        optimizer.zero_grad(set_to_none=True)
        logits = model(batch["input_ids"], batch["attention_mask"])
        summed = token_loss(logits, batch["labels"])
        valid = batch["labels"].ne(-100).sum()
        loss = summed / valid
        loss.backward()
        assert all(p.grad is None or torch.isfinite(p.grad).all()
                   for p in model.parameters())
        optimizer.step()
        if scheduler is not None: scheduler.step()
        loss_sum += summed.detach().item(); token_count += valid.item()
    return loss_sum / token_count

@torch.no_grad()
def evaluate(model, loader, device):
```

```
    model.eval()
    total, tokens = 0.0, 0
    for batch in loader:
        batch = {k: v.to(device) for k, v in batch.items()
                 if torch.is_tensor(v)}
        logits = model(batch["input_ids"], batch["attention_mask"])
        total += token_loss(logits, batch["labels"]).item()
        tokens += batch["labels"].ne(-100).sum().item()
    return total / tokens

def E00(model, train_loader, valid_loader, optimizer, device):
    before = {n: p.detach().clone() for n, p in model.named_parameters()}
    train_nll = train_epoch(model, train_loader, optimizer, device)
    changed = [not torch.equal(before[n], p)
               for n, p in model.named_parameters()]
    assert any(changed)
    return train_nll, evaluate(model, valid_loader, device)
```

Part 5 - The Philosophy Behind the Exercise

Training is not the existence of gradients; it is a disciplined sequence of state transitions. Model mode, gradient buffers, optimizer moments, and scheduler time all carry memory. A misplaced operation changes that memory and therefore changes the experiment.

The loop is where mathematical intent becomes temporal behavior. Reliability begins when every transition is explicit enough to inspect, test, interrupt, and reproduce.

17.7.2 Exercise 01

Exercise Title: Train a Tiny Transformer Until Overfitting to Validate End-to-End Correctness and Debug Pipeline Issues

Part 1 - Why this matters

Overfitting a tiny dataset is one of the strongest end-to-end correctness tests for a training system. The goal is not generalization. The goal is to remove data complexity and ask whether tokenization, causal shifting, masks, positions, forward computation, loss, backward propagation, optimizer updates, and generation cooperate well enough to memorize a few deterministic examples. If a sufficiently expressive Transformer cannot do this, scaling the dataset is unlikely to fix the root cause.

Construct four to sixteen short sequences with a tiny frozen vocabulary. Print every input and target pair and keep dropout disabled for the strict test. Train on the same examples repeatedly until valid-token accuracy is nearly one and NLL is far below the uniform baseline $\log V$. Then generate from BOS or a known prefix without teacher forcing and compare the whole continuation. A low teacher-forced loss can coexist with broken autoregressive state handling.

Use failure signatures to diagnose the pipeline. If loss remains near $\log V$, inspect gradients, learning rate, detached tensors, and label alignment. If accuracy is high but generation is

wrong, inspect causal masks, position offsets, EOS handling, model mode, and decoding. If loss becomes near zero instantly, test for target leakage or copy labels. Run shuffled-label and future-visible controls to prove the test distinguishes learning from cheating.

Part 2 - Mathematical form and reasoning

For deterministic examples, empirical risk is

$$\widehat{L}(\theta) = -\frac{1}{N_{\text{tok}}} \sum_{n,t} \log p_\theta(y_t^{(n)} \mid x_{\le t}^{(n)}).$$

A uniform distribution gives $\widehat{L} \approx \log V$. Successful memorization approaches $\widehat{L} \to 0$ and

$$\frac{1}{N_{\text{tok}}} \sum_{n,t} \mathbb{1}[\arg\max_v p_\theta(v \mid x_{\le t}^{(n)}) = y_t^{(n)}] \to 1.$$

These conditions are necessary integration evidence, not evidence of useful generalization.

Part 3 - Hints, controls, and evidence

1. Use a fixed seed, one small batch, no augmentation, no dropout, and no weight decay for the initial correctness test.
2. Log per-position loss so BOS/EOS or one ignored column cannot hide behind the mean.
3. Inspect gradient norms layer by layer on the first and final steps.
4. Compare teacher-forced predictions with free-running greedy generation.
5. Preserve this dataset, configuration, and expected threshold as a fast regression test after every training-loop refactor.

Part 4 - Python code skeleton

```
import math, torch
import torch.nn.functional as F

def overfit_tiny(model, batch, steps=1000, lr=3e-3):
    optimizer = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=0.0)
    history = []
    model.train()
    for step in range(steps):
        optimizer.zero_grad(set_to_none=True)
        logits = model(batch["input_ids"], batch["attention_mask"])
        loss = F.cross_entropy(logits.flatten(0, 1),
                               batch["labels"].flatten(), ignore_index=-100)
        loss.backward(); optimizer.step()
        with torch.no_grad():
            valid = batch["labels"].ne(-100)
            pred = logits.argmax(-1)
            acc = (pred[valid] == batch["labels"][valid]).float().mean()
        history.append((loss.item(), acc.item()))
```

```
        if loss.item() < 0.02 and acc.item() > 0.995: break
    return history

def E01(model, batch, vocab_size, generate):
    history = overfit_tiny(model, batch)
    assert history[-1][0] < min(0.05, 0.05 * math.log(vocab_size))
    assert history[-1][1] > 0.99
    model.eval()
    generated = generate(model, batch["input_ids"][:, :1])
    # TODO: compare complete generated sequences and run leakage controls.
    return {"history": history, "generated": generated}
```

Part 5 - The Philosophy Behind the Exercise

A tiny-overfit test asks the system to demonstrate one small competence beyond reasonable doubt. By making the data completely inspectable, it forces failures to reveal themselves as failures of mechanism rather than mysteries of scale.

Complex systems become trustworthy through a ladder of solvable worlds. Before asking a model to learn language, prove that the learning machinery can learn a handful of exact relationships without hidden shortcuts.

17.7.3 Exercise 02

Exercise Title: Implement Learning Rate Warmup and Scheduler and Prove Warmup Prevents Early Training Divergence

Part 1 - Why this matters

At initialization, Transformer representations, attention distributions, and optimizer moments are not calibrated. A large target learning rate can move parameters aggressively before Adam's estimates stabilize, producing loss spikes or non-finite values. Warmup increases the learning rate gradually, then hands control to a decay schedule. It is a hypothesis about early optimization, not a ritual that should be accepted without a matched experiment.

Implement a scheduler as an explicit function of optimizer update count. Decide whether the first update uses zero, a small positive rate, or one warmup increment; then test that exact convention. Do not use epoch count when gradient accumulation means several microbatches form one optimizer update. Plot the first steps, the warmup boundary, and the final step. Save and restore scheduler state with checkpoints.

Compare no warmup, short warmup, and longer warmup under identical seeds, batches, initialization, peak rate, and total optimizer updates. Record early loss, gradient norm, update-to-weight ratio, non-finite events, and final validation quality. To claim prevention of divergence, choose a peak rate that causes an early instability without warmup, then show that warmup delays the large updates until the model reaches a stable region. Also show that excessive warmup wastes budget and can undertrain.

Part 2 - Mathematical form and reasoning

For update $u \geq 1$, warmup length W, total updates U, and peak rate $\eta_{\max}$, a linear-warmup/cosine-decay schedule can be

$$\eta(u) = \begin{cases} \eta_{\max}\, u/W, & u \leq W, \\ \frac{\eta_{\max}}{2}\left[1 + cos\left(\pi \frac{u-W}{U-W}\right)\right], & W < u \leq U. \end{cases}$$

The relative update magnitude $r_u = \|\theta_{u+1} - \theta_u\|_2 / (\|\theta_u\|_2 + \epsilon)$ links schedule values to actual parameter motion.

Part 3 - Hints, controls, and evidence

1. Unit-test exact rates at updates 1, W, $W + 1$, and U.
2. Call the scheduler at one documented location relative to `optimizer.step`; an off-by-one call changes the first rate.
3. Log optimizer-update count separately from batch and epoch counts.
4. Repeat several seeds and define divergence before observing results, such as non-finite loss or a sustained loss increase beyond a threshold.
5. Resume halfway through warmup and prove the next learning rate matches an uninterrupted run.

Part 4 - Python code skeleton

```
import math

def lr_multiplier(update, warmup, total):
    if not 1 <= update <= total: raise ValueError("invalid update")
    if update <= warmup:
        return update / max(warmup, 1)
    progress = (update - warmup) / max(total - warmup, 1)
    return 0.5 * (1.0 + math.cos(math.pi * progress))

def make_scheduler(optimizer, warmup, total):
    # The offset defines the first scheduled update explicitly.
    return __import__("torch").optim.lr_scheduler.LambdaLR(
        optimizer, lambda epoch: lr_multiplier(epoch + 1, warmup, total))

def E02(make_run, peak_lr=3e-3, total=1000):
    results = {}
    for warmup in (0, 50, 200):
        results[warmup] = make_run(peak_lr=peak_lr, warmup=warmup,
                                   total_updates=total, seed=22)
    # TODO: assert rate curve, compare early stability, and repeat seeds.
    return results
```

Part 5 - The Philosophy Behind the Exercise

Warmup recognizes that the same action can have different consequences at different stages of learning. Early parameters and optimizer statistics are fragile; later ones can absorb larger steps. Time is therefore part of the optimization problem.

A schedule is a policy for when the system should move boldly and when it should refine. Its legitimacy comes from measured dynamics, not from becoming a default incantation.

17.7.4 Exercise 03

Exercise Title: Implement AdamW Correctly and Explain Weight Decay Decoupling vs L2 Regularization Differences

Part 1 - Why this matters

In plain SGD, adding $\lambda\|\theta\|_2^2/2$ to the objective yields a gradient term $\lambda\theta$ that resembles multiplicative weight decay. In Adam, gradients are divided coordinate-wise by adaptive second-moment estimates. If the L2 term is placed inside that gradient, the apparent shrinkage is also preconditioned. AdamW decouples shrinkage from the loss gradient, giving parameters a direct multiplicative decay independent of moment normalization.

Implement a small Adam and AdamW update by hand for a vector parameter. Match the manual AdamW step to PyTorch with identical hyperparameters, gradient, initial moment states, and epsilon. Then compare three conditions: no regularization, L2 added to the loss, and decoupled weight decay. Use unequal gradient magnitudes across coordinates so adaptive effects are visible. Equal results under a symmetric toy case do not prove equivalence.

Create parameter groups deliberately. Biases and normalization scale/shift are commonly excluded from decay, while matrix weights are decayed; the exact policy should be justified and logged. Classify by module/parameter role rather than a fragile substring rule when possible. Assert every trainable parameter appears in exactly one group. Record data loss separately from any explicit L2 penalty so comparisons remain interpretable.

Part 2 - Mathematical form and reasoning

Adam moments are

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2,$$

with bias-corrected $\widehat{m}_t$ and $\widehat{v}_t$. AdamW updates

$$\theta_{t+1} = (1-\eta_t\lambda)\theta_t - \eta_t \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t}+\epsilon}.$$

L2-in-Adam instead replaces g_t by $g_t + \lambda\theta_t$ before both moment updates. The regularization contribution is therefore filtered through m_t and v_t and is not the same coordinate-independent decay.

Part 3 - Hints, controls, and evidence

1. Start from one scalar step, then use a two-coordinate asymmetric example.

2. Set the data gradient to zero and verify AdamW produces exactly $(1 - \eta\lambda)\theta$.
3. Compare multiple steps because moment history amplifies the difference between L2 and decoupled decay.
4. Audit parameter-group coverage, duplicates, decay values, and group learning rates before training.
5. Track weight norms, training data loss, validation loss, and final quality across matched decay sweeps.

Part 4 - Python code skeleton

```
import torch

def adamw_manual(theta, grad, m, v, step, lr, beta1=.9, beta2=.999,
                 eps=1e-8, weight_decay=.01):
    m = beta1 * m + (1 - beta1) * grad
    v = beta2 * v + (1 - beta2) * grad.square()
    m_hat = m / (1 - beta1 ** step)
    v_hat = v / (1 - beta2 ** step)
    theta = theta * (1 - lr * weight_decay)
    theta = theta - lr * m_hat / (v_hat.sqrt() + eps)
    return theta, m, v

def parameter_groups(model, decay):
    decayed, excluded, seen = [], [], set()
    for module in model.modules():
        for name, p in module.named_parameters(recurse=False):
            if not p.requires_grad or id(p) in seen: continue
            seen.add(id(p))
            is_bias = name.endswith("bias")
            is_norm = module.__class__.__name__.endswith("Norm")
            if is_bias or is_norm:
                excluded.append(p)
            else: decayed.append(p)
    expected = {id(p) for p in model.parameters() if p.requires_grad}
    assert seen == expected
    return [{"params": decayed, "weight_decay": decay},
            {"params": excluded, "weight_decay": 0.0}]

def E03():
    theta = torch.tensor([1.0, 10.0])
    grad = torch.tensor([.1, 3.0])
    zero = torch.zeros_like(theta)
    return adamw_manual(theta, grad, zero, zero, 1, lr=1e-3)
```

Part 5 - The Philosophy Behind the Exercise

Two algorithms can share the word "regularization" while expressing different beliefs about parameter motion. L2 changes the gradient evidence; AdamW adds an independent preference for smaller weights. The distinction is structural, not terminological.

Optimization becomes intelligible when every force on a parameter is named. Separating data-driven motion from prior shrinkage makes both the code and the scientific claim easier to reason about.

17.7.5 Exercise 04

Exercise Title: Implement Gradient Clipping in Transformer Training and Measure Its Effect on Stability and Convergence

Part 1 - Why this matters

Transformer gradients can spike because of an unusual batch, an unstable attention distribution, excessive learning rate, or accumulated signal through depth. Global-norm clipping limits one update from being dominated by an exceptional gradient. It is a safety mechanism, not a cure for every instability: if clipping activates almost every step, it may be hiding a bad learning rate, incorrect loss scaling, broken normalization, or corrupt data.

Compute the global norm immediately after backward and, under mixed precision, after `scaler.unscale_(optimizer)`. Clipping scaled gradients enforces a threshold in an artificial scale-dependent space. Record the norm before clipping, the norm after clipping, and whether the threshold was active. Then call the optimizer. Use one global norm across the intended parameter set; per-layer clipping is a different algorithm and must be named separately.

Compare no clipping with several thresholds under matched seeds, data order, learning rate, and update budget. Measure non-finite events, loss spikes, gradient-norm percentiles, fraction of clipped steps, update norms, convergence speed, and final validation quality. Inject a controlled outlier batch so the mechanism is exercised. A threshold so low that all steps are clipped changes both direction and effective step size enough to slow learning.

Part 2 - Mathematical form and reasoning

For concatenated gradient vector g and maximum norm $c > 0$, global clipping is

$$\widetilde{g} = g \min\left(1, \frac{c}{\|g\|_2 + \epsilon}\right).$$

If $\|g\|_2 \le c$, the gradient is unchanged. Otherwise, all coordinates are multiplied by the same positive factor, preserving the global direction while limiting its magnitude. The clipping rate over U updates is

$$r_c = \frac{1}{U} \sum_{u=1}^{U} \mathbb{1}[\|g_u\|_2 > c].$$

This rate helps distinguish rare protection from permanent suppression.

Part 3 - Hints, controls, and evidence

1. Verify a hand-built two-parameter gradient against the manual formula.

2. Check the return value of `clip_grad_norm_`; it is the pre-clip norm and should be logged before conversion to a Python scalar.

3. Set `error_if_nonfinite=True` during debugging so NaNs are not silently scaled.

4. Sweep thresholds around observed norm percentiles rather than arbitrary orders of magnitude.

5. Separate a successful emergency intervention from improved final optimization; report both stability and quality.

Part 4 - Python code skeleton

```
import torch

def clipped_step(model, optimizer, loss, max_norm, scaler=None):
    optimizer.zero_grad(set_to_none=True)
    if scaler is None:
        loss.backward()
    else:
        scaler.scale(loss).backward()
        scaler.unscale_(optimizer)
    before = torch.nn.utils.clip_grad_norm_(
        model.parameters(), max_norm, error_if_nonfinite=True)
    active = bool(before > max_norm)
    after = min(float(before), float(max_norm))
    if scaler is None:
        optimizer.step()
    else:
        scaler.step(optimizer); scaler.update()
    return {"norm_before": float(before), "norm_after_bound": after,
            "clipped": active}

def E04(make_run):
    return {threshold: make_run(clip_norm=threshold, same_seed=4,
                                inject_outlier=True)
            for threshold in (None, 0.1, 1.0, 5.0)}
```

Part 5 - The Philosophy Behind the Exercise

Gradient clipping places a boundary on how much one piece of evidence may alter the model at once. It encodes caution in the presence of rare or unstable signals while leaving ordinary learning untouched.

A safeguard becomes dangerous when it conceals the condition that repeatedly activates it. Good engineering records both the rescue and the reason rescue was needed.

17.7.6 Exercise 05

Exercise Title: Implement Mixed Precision Training with GradScaler and Detect Silent Overflows in fp16 Attention Softmax

Part 1 - Why this matters

Mixed precision can reduce activation memory and increase Tensor Core throughput, but float16 has limited exponent range. Large attention scores can overflow before or during exponentiation, and small gradients can underflow to zero. Autocast chooses lower precision for many operations while retaining safer precision for others. GradScaler multiplies the loss before backward so small gradients survive, then unscales them before clipping and the optimizer step.

Implement the complete order: autocast forward, scaled backward, unscale, finite-gradient inspection, optional clipping, `scaler.step`, and `scaler.update`. Detect whether an optimizer step was skipped after a non-finite gradient by comparing parameters or the scaler state. Log attention score ranges before masking, count non-finite logits/probabilities, and assert every valid softmax row sums to one. A finite final loss does not prove every intermediate attention row was healthy.

Construct an extreme-score test by scaling queries and keys. Compare a naive float16 softmax, a stable max-subtracted implementation in float32, CUDA autocast, and bfloat16 where supported. Use a finite negative mask value appropriate to the dtype or apply a Boolean masked operation; avoid arithmetic that combines infinities into NaN. Compare throughput, peak memory, loss, gradient parity, skipped steps, and validation quality against float32.

Part 2 - Mathematical form and reasoning

Stable attention probabilities for score row s are

$$p_i = \frac{\exp(s_i - m)}{\sum_j \exp(s_j - m)}, \qquad m = \max_j s_j.$$

Subtracting m makes the largest exponent one and leaves softmax unchanged in exact arithmetic. If loss scale is S, backward computes Sg; before update, GradScaler restores $g = (Sg)/S$. When Sg is non-finite, the optimizer step is skipped and the scale is reduced. Clipping must operate on the unscaled g.

Part 3 - Hints, controls, and evidence

1. Match initial weights and batches for float32 and mixed-precision runs.
2. Register temporary hooks around attention score, masked score, softmax, and output tensors; remove them after diagnosis.
3. Log loss scale, skipped-step count, zero-gradient fraction, non-finite count, and maximum absolute score.
4. Test normal and extreme inputs. Stability on ordinary batches does not establish a safe numerical boundary.
5. Use bfloat16 as a distinct condition, not as another name for float16. Its wider range changes overflow behavior.

Part 4 - Python code skeleton

```
import torch

def amp_step(model, batch, optimizer, scaler, clip_norm=1.0):
    optimizer.zero_grad(set_to_none=True)
    with torch.autocast(device_type="cuda", dtype=torch.float16):
        logits, debug = model(batch["input_ids"],
                              batch["attention_mask"], return_debug=True)
        flat_logits = logits.flatten(0, 1)
        flat_labels = batch["labels"].flatten()
        loss = torch.nn.functional.cross_entropy(
            flat_logits, flat_labels, ignore_index=-100)
    assert torch.isfinite(debug["attention_scores"].float()).all()
    assert torch.isfinite(logits.float()).all()
    old_scale = scaler.get_scale()
    scaler.scale(loss).backward()
    scaler.unscale_(optimizer)
    norm = torch.nn.utils.clip_grad_norm_(model.parameters(), clip_norm,
                                          error_if_nonfinite=True)
    before = [p.detach().clone() for p in model.parameters()]
    scaler.step(optimizer); scaler.update()
    changed = any(not torch.equal(a, b) for a, b in
                  zip(before, model.parameters()))
    return {"loss": loss.item(), "grad_norm": float(norm),
            "old_scale": old_scale, "new_scale": scaler.get_scale(),
            "parameter_step_happened": changed}

def E05(model, batch, optimizer):
    scaler = torch.amp.GradScaler("cuda")
    return amp_step(model, batch, optimizer, scaler)
```

Part 5 - The Philosophy Behind the Exercise

Mixed precision is an agreement about which numerical distinctions the system may temporarily compress. Speed is earned by using fewer bits, while scaling and stable kernels protect distinctions essential to learning.

The deepest failure is silent invalidity: a training run that continues while updates are skipped or attention distributions are corrupted. Performance is valuable only when accompanied by observability.

17.7.7 Exercise 06

Exercise Title: Debug NaN Loss by Inspecting Attention Scores, LayerNorm, and Learning Rate and Implement Automatic Checks

Part 1 - Why this matters

A NaN loss is the final symptom of an earlier invalid value. The first NaN may appear in input data, embedding output, attention scores, a fully masked softmax row, residual activations, LayerNorm statistics, logits, gradients, or optimizer state. Checking only the scalar loss destroys the causal trail. The objective is to identify the earliest violated invariant and preserve the batch and state that reproduced it.

Instrument the model with forward hooks and gradient hooks that record shape, dtype, minimum, maximum, mean, standard deviation, and non-finite counts. Pay special attention to $QK^\top/\sqrt{d_k}$ before and after masking and to the variance entering LayerNorm. Assert that every valid query has at least one allowed key. Record the current learning rate, loss scale, gradient norm, parameter norm, and sample IDs. When a check fails, stop before the optimizer corrupts additional state and save a compact incident checkpoint.

Create a diagnostic matrix rather than changing many settings at once. Replay the failing batch in float64 or float32, lower the learning rate, disable mixed precision, replace the mask, and reduce input scale one factor at a time. Inject known failures: a fully masked row, extreme Q/K scale, zero LayerNorm epsilon, corrupt input, and excessive learning rate. Prove each is detected close to its origin with a specific message.

Part 2 - Mathematical form and reasoning

LayerNorm computes

$$\mathrm{LN}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta.$$

Finite inputs can become invalid if variance or epsilon handling is broken. Attention uses

$$A = \mathrm{softmax}\left(QK^\top/\sqrt{d_k} + M\right).$$

A row containing no finite allowed score has an undefined normalized distribution. Define earliest failure index $k^* = \min\{k : z_k \notin \mathbb{R}^{n_k}\}$ over ordered intermediate tensors; diagnosis should target the operation producing z_{k^*}.

Part 3 - Hints, controls, and evidence

1. Use `torch.autograd.detect_anomaly()` only for a small replay; it is valuable but too expensive for routine training.
2. Save the pre-step model, optimizer, scaler, RNG state, batch, and configuration when an incident occurs.
3. Check optimizer moments as well as parameters; corrupted state can reintroduce NaNs after parameters are restored alone.
4. Prefer bounded summaries to copying every activation to CPU each step.
5. Convert the final root cause into a cheap permanent assertion and a regression test.

Part 4 - Python code skeleton

```
import torch

class FiniteMonitor:
    def __init__(self): self.handles, self.records = [], []
    def attach(self, model):
        for name, module in model.named_modules():
            if len(list(module.children())): continue
            hook = module.register_forward_hook(self._hook(name))
            self.handles.append(hook)
    def _hook(self, name):
        def check(module, inputs, output):
            tensors = output if isinstance(output, tuple) else (output,)
            for i, value in enumerate(tensors):
                if not torch.is_tensor(value): continue
                x = value.detach().float()
                record = {"module": name, "output": i,
                          "shape": tuple(x.shape),
                          "finite": bool(torch.isfinite(x).all()),
                          "min": x.nan_to_num().min().item(),
                          "max": x.nan_to_num().max().item()}
                self.records.append(record)
                if not record["finite"]:
                    message = f"first invalid output: {record}"
                    raise FloatingPointError(message)
        return check
    def close(self):
        for h in self.handles: h.remove()

def E06(model, replay_step):
    monitor = FiniteMonitor(); monitor.attach(model)
    try: result = replay_step(model)
    finally: monitor.close()
    return result, monitor.records
```

Part 5 - The Philosophy Behind the Exercise

NaN is not a cause; it is the collapse of a numerical statement somewhere upstream. Debugging succeeds by restoring the history that the final scalar has erased.

Observability gives failure a location and a time. Once the first broken invariant is known, an apparently chaotic training collapse becomes a specific operation under specific conditions.

17.7.8 Exercise 07

Exercise Title: Implement Activation Checkpointing and Quantify Memory Savings vs Extra Computation Cost

Part 1 - Why this matters

Backpropagation normally stores intermediate activations from the forward pass. For a deep Transformer with long sequences, this activation memory can dominate parameters and prevent the desired batch or context length from fitting on the GPU. Activation checkpointing stores only selected boundary tensors and recomputes the omitted forward regions during backward. It trades additional computation for lower peak memory.

Wrap groups of Transformer blocks with the PyTorch checkpoint utility while keeping the model's external contract unchanged. Compare no checkpointing, checkpoint every block, and checkpoint groups of several blocks. Measure peak allocated memory, peak reserved memory, forward time, backward time, total step time, useful tokens per second, and maximum feasible batch/context. Synchronize CUDA around timed regions and warm up before measurement.

Correctness requires more than a finite loss. Start from identical weights and inputs, disable stochastic layers, and compare outputs, input gradients, and parameter gradients. Then enable dropout and verify RNG-state preservation or document the chosen semantics. Beware of side effects inside checkpointed functions: the forward function runs again during backward. Logging counters, cache mutations, and random sampling may therefore occur twice. Test the framework's non-reentrant checkpoint mode explicitly.

Part 2 - Mathematical form and reasoning

Suppose a network is $f = f_L \circ \cdots \circ f_1$. Standard backward retains intermediate states $h_l = f_l(h_{l-1})$. Checkpointing a segment stores its input h_a and recomputes

$$h_{a+1}, \ldots, h_b = f_{a+1:b}(h_a)$$

during backward. If stored activation memory is approximately M_a and checkpointing retains fraction $r < 1$, memory becomes roughly $M_{\text{other}} + rM_a$, while compute increases by the cost of recomputed forward segments. Actual savings must be measured because kernels and allocator state change the estimate.

Part 3 - Hints, controls, and evidence

1. Establish forward and gradient parity in float64 or float32 on a tiny deterministic model before measuring speed.

2. Reset peak memory statistics for every configuration in an isolated trial.

3. Time forward, backward, and optimizer stages separately. Checkpointing primarily adds recomputation during backward.

4. Confirm at least one input or parameter requires gradients and avoid detaching tensors inside the checkpointed function.

5. Report the Pareto frontier between peak memory and step time rather than declaring one checkpoint granularity universally best.

Part 4 - Python code skeleton

```
import torch
from torch.utils.checkpoint import checkpoint

class CheckpointedStack(torch.nn.Module):
    def __init__(self, blocks, use_checkpoint=True):
        super().__init__(); self.blocks = torch.nn.ModuleList(blocks)
        self.use_checkpoint = use_checkpoint
    def forward(self, x, attention_mask):
        for block in self.blocks:
            if self.use_checkpoint and self.training:
                def run(hidden, mask, layer=block):
                    return layer(hidden, attention_mask=mask)
                x = checkpoint(run, x, attention_mask, use_reentrant=False,
                               preserve_rng_state=True)
            else:
                x = block(x, attention_mask=attention_mask)
        return x

def measure_peak(step):
    torch.cuda.empty_cache(); torch.cuda.reset_peak_memory_stats()
    start, end = (torch.cuda.Event(enable_timing=True),
                  torch.cuda.Event(enable_timing=True))
    start.record(); step(); end.record(); end.synchronize()
    return {"milliseconds": start.elapsed_time(end),
            "peak_allocated": torch.cuda.max_memory_allocated(),
            "peak_reserved": torch.cuda.max_memory_reserved()}

def E07(make_step):
    return {mode: measure_peak(make_step(checkpointing=mode))
            for mode in (False, True)}
```

Part 5 - The Philosophy Behind the Exercise

Checkpointing treats memory and computation as exchangeable resources. It chooses to remember fewer intermediate facts and reconstruct them when needed. The mathematics remains the same, but the temporal strategy changes.

This is a recurring systems principle: when storage is scarce and computation is available, recomputation can expand what is possible. The exchange is useful only when correctness and the true cost are measured together.

17.7.9 Exercise 08

Exercise Title: Implement Gradient Accumulation to Simulate Larger Batch Training Under GPU Memory Constraints

Part 1 - Why this matters

Gradient accumulation processes a logical batch as several microbatches while delaying the optimizer update. It reduces peak activation memory, but exact equivalence to a large batch

depends on loss normalization, stochastic layers, optimizer timing, scheduler timing, and batch-dependent operations. Dividing each microbatch mean loss by the number of microbatches is correct only when all microbatches contain the same number of valid targets. Variable padding breaks that assumption.

For strict equivalence, accumulate the *sum* of valid-token losses and normalize by the total number of valid tokens represented by the logical batch. One practical method is to know the token total before backward and scale each microbatch summed loss by that total. Clear gradients once at the start, run backward for every microbatch, then unscale, clip, update, and advance the scheduler once. An incomplete final accumulation window needs its actual token denominator, not the planned full-window size.

Compare a true large batch with accumulated microbatches using dropout disabled and identical sample order. Inspect loss, every parameter gradient, parameter delta, and optimizer state after one update. Then enable dropout and explain why different random masks can prevent bitwise equivalence while preserving the same expected objective. If using distributed training, suppress unnecessary gradient synchronization on non-final microbatches and verify the final result.

Part 2 - Mathematical form and reasoning

Let microbatch j contain valid-token set $\mathcal{I}_j$ and summed loss $S_j = \sum_{i \in \mathcal{I}_j} \ell_i$. The logical-batch objective is

$$L = \frac{\sum_{j=1}^{K} S_j}{N}, \qquad N = \sum_{j=1}^{K} |\mathcal{I}_j|.$$

Therefore

$$\nabla_\theta L = \sum_{j=1}^{K} \nabla_\theta \left(\frac{S_j}{N} \right).$$

This equality justifies accumulation. Replacing N by $K|\mathcal{I}_j|$ for unequal microbatches changes token weights and no longer simulates the same large-batch objective.

Part 3 - Hints, controls, and evidence

1. Begin with deterministic layers, one logical update, and float64 on a small model to establish gradient parity.
2. Count optimizer and scheduler steps; both must equal logical batches, not microbatches.
3. Clip only after all microbatch gradients are accumulated and AMP gradients are unscaled.
4. Test unequal valid-token counts and an incomplete final window.
5. Report peak memory, total step time, useful tokens per second, and gradient-parity error as separate outcomes.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def accumulated_update(model, microbatches, optimizer, scaler=None):
    optimizer.zero_grad(set_to_none=True)
    total_tokens = sum(b["labels"].ne(-100).sum().item()
                       for b in microbatches)
    loss_sum = 0.0
    for batch in microbatches:
        with torch.autocast("cuda", enabled=scaler is not None):
            logits = model(batch["input_ids"], batch["attention_mask"])
            summed = F.cross_entropy(logits.flatten(0, 1),
                batch["labels"].flatten(), ignore_index=-100, reduction="sum")
            normalized = summed / total_tokens
        if scaler is None: normalized.backward()
        else: scaler.scale(normalized).backward()
        loss_sum += summed.detach().item()
    if scaler is not None: scaler.unscale_(optimizer)
    norm = torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    if scaler is None: optimizer.step()
    else: scaler.step(optimizer); scaler.update()
    return {"nll": loss_sum / total_tokens, "grad_norm": float(norm)}

def E08(make_equivalent_batches, make_models):
    large, micros = make_equivalent_batches()
    direct, accumulated = make_models(same_state=True, dropout=0.0)
    # TODO: perform one update both ways and compare gradients and parameters.
    return large, micros, direct, accumulated
```

Part 5 - The Philosophy Behind the Exercise

Accumulation separates when evidence is observed from when a decision is made. Several microbatches contribute to one update, provided their contributions are weighted exactly as the intended logical batch.

Simulation is an equality claim, not a visual resemblance. To say that a large batch was simulated is to prove equivalence of objective, gradient, update timing, and state transitions under declared assumptions.

17.7.10 Exercise 09

Exercise Title: Debug Training Where Loss Improves But Generation Quality Does Not Improve and Identify Root Causes

Part 1 - Why this matters

Teacher-forced next-token loss and free-running generation measure related but different behavior. During training, every prediction receives the true prefix. During generation, each sampled token becomes part of the next prefix, so one error can move the model into contexts absent from training. A decreasing loss can also be dominated by frequent easy tokens while

rare structure, long-range dependencies, or EOS behavior remains poor.

Build an evaluation ladder. First verify label alignment, causal masking, and validation NLL on a leakage-free split. Next inspect accuracy and NLL by token frequency, position, sequence length, and example. Then run deterministic greedy generation in evaluation mode from fixed prompts. Finally compare temperature, top-k, and nucleus sampling with fully logged seeds and parameters. Separate model defects from decoding choices: a model may assign reasonable probability mass while an extreme sampling configuration produces incoherent text.

Investigate common root causes systematically: train/eval mode mismatch, dropout active during inference, hidden future leakage, wrong position offset in cached generation, tokenizer mismatch, repeated BOS/EOS, missing context window, validation contamination, class imbalance, loss computed on PAD, insufficient capacity, and exposure bias. Compare cached and uncached logits for identical prefixes. Save probability traces, not only final strings, so repetition and premature EOS can be connected to distributions.

Part 2 - Mathematical form and reasoning

Teacher-forced objective evaluates

$$L_{\mathrm{TF}} = -\sum_{t} \log p_\theta(y_t \mid y_{<t}),$$

while generation produces $\widehat{y}_t \sim p_\theta(\cdot \mid \widehat{y}_{<t})$. After the first error, $\widehat{y}_{<t}$ may lie outside the distribution of gold prefixes. Decompose validation NLL into groups G:

$$L = \sum_{g} \frac{|G_g|}{N} L_g.$$

An improved total can coexist with degraded L_g for rare but important groups. This is why aggregate loss cannot be the sole quality claim.

Part 3 - Hints, controls, and evidence

1. Freeze a prompt suite before tuning and include exact-copy, long- dependency, rare-token, and EOS-sensitive cases.

2. Compare greedy output first; stochastic sampling adds variance that can mask deterministic implementation bugs.

3. Log entropy, maximum probability, selected-token probability, repetition rate, EOS position, and NLL by position.

4. Test cached generation against full-prefix recomputation at every step.

5. Assign every failure to data, objective, model, decoding, or evaluation, then validate the proposed cause with a one-factor intervention.

Part 4 - Python code skeleton

```
import torch

@torch.no_grad()
def trace_greedy(model, prompt, max_new, eos_id):
    model.eval(); ids = prompt.clone(); trace = []
    for step in range(max_new):
        logits = model(ids)[:, -1].float()
        probs = logits.softmax(-1)
        next_id = probs.argmax(-1, keepdim=True)
        trace.append({"step": step, "token": next_id.item(),
                      "probability": probs.gather(1, next_id).item(),
                      "entropy": (-(probs * probs.clamp_min(1e-12).log())
                                  .sum()).item()})
        ids = torch.cat([ids, next_id], dim=1)
        if next_id.item() == eos_id: break
    return ids, trace

def E09(model, fixed_prompts, eos_id, cached_generate):
    report = []
    for prompt in fixed_prompts:
        full_ids, trace = trace_greedy(model, prompt, 64, eos_id)
        cached_ids = cached_generate(model, prompt, 64, eos_id)
        report.append({"prompt": prompt.cpu(), "generated": full_ids.cpu(),
                       "trace": trace,
                       "cache_parity": torch.equal(full_ids, cached_ids)})
    return report
```

Part 5 - The Philosophy Behind the Exercise

An objective is a proxy: it describes what the system is rewarded for under a particular observation process. Generation exposes the model to consequences of its own choices, a condition teacher forcing does not fully represent.

Improving a proxy is evidence, not proof of the desired capability. Good research keeps the objective, evaluation environment, and human notion of quality distinct enough to discover when they diverge.

17.7.11 Exercise 10

Exercise Title: Train Transformer Fully on CUDA and Validate DataLoader Transfers, Pin Memory, and Correct Device Placement

Part 1 - Why this matters

End-to-end CUDA training requires a consistent device contract across model parameters, registered buffers, input IDs, masks, labels, temporary tensors, loss, gradients, and optimizer state. A model can be moved correctly while a causal mask created inside `forward` remains on CPU. It can also run correctly but slowly because batches are copied synchronously from pageable memory or because many small tensors are transferred one at a time.

Create the model on CPU, move it once to the selected CUDA device, then construct the optimizer from the moved parameters. Configure the DataLoader with pinned memory and transfer batch tensors using `non_blocking=True`. This flag permits asynchronous copies only when the source and execution context support it; benchmark rather than assuming overlap. Create internal tensors with `x.new_*` or an explicit device derived from inputs. Register persistent or fixed tensors as buffers so device conversion and checkpointing include them.

Before the first training step, audit every parameter and buffer. After the first step, audit gradients and optimizer tensor states. Compare one fixed CPU and CUDA batch in evaluation mode at float32 with tolerances appropriate to the kernels. Then time preparation, transfer, forward, backward, and update using correct synchronization. Report useful tokens per second and padding ratio; raw batch throughput can reward wasted padding.

Part 2 - Mathematical form and reasoning

Let $D(z)$ return the device of tensor z. For every operation $f(z_1, \ldots, z_k)$ that does not explicitly copy data, require

$$D(z_1) = \cdots = D(z_k) = D(f(z_1, \ldots, z_k)).$$

End-to-end step time can be decomposed as

$$t_{\text{step}} = t_{\text{wait}} + t_{H \to D} + t_{\text{forward}} + t_{\text{backward}} + t_{\text{update}} - t_{\text{overlap}}.$$

Useful throughput is $N_{\text{valid}}/t_{\text{step}}$, and transfer optimization matters only to the fraction not hidden by overlap.

Part 3 - Hints, controls, and evidence

1. Fail early if CUDA is unavailable or the requested device index is invalid; do not silently change the benchmark condition.
2. Use a recursive device auditor for nested batches, parameters, buffers, gradients, and optimizer state.
3. Warm up CUDA kernels and synchronize only at measurement boundaries.
4. Compare pageable/blocking, pinned/blocking, and pinned/nonblocking copies with identical batches.
5. Monitor allocated and reserved memory and verify that memory does not grow across steady-state iterations.

Part 4 - Python code skeleton

```
import torch

def assert_model_device(model, device):
    wrong = [(n, str(t.device)) for n, t in
             list(model.named_parameters()) + list(model.named_buffers())
             if t.device != device]
```

```
    if wrong: raise RuntimeError(f"device mismatch: {wrong}")

def move_batch(batch, device):
    return {k: v.to(device, non_blocking=True)
            for k, v in batch.items() if torch.is_tensor(v)}

def audit_optimizer(optimizer, device):
    for state in optimizer.state.values():
        for value in state.values():
            if torch.is_tensor(value) and value.device != device:
                raise RuntimeError("optimizer state is on the wrong device")

def E10(model, loader, make_optimizer, device_index=0):
    device = torch.device(f"cuda:{device_index}")
    model = model.to(device); assert_model_device(model, device)
    optimizer = make_optimizer(model.parameters())
    batch = move_batch(next(iter(loader)), device)
    optimizer.zero_grad(set_to_none=True)
    logits = model(batch["input_ids"], batch["attention_mask"])
    loss = torch.nn.functional.cross_entropy(
        logits.flatten(0, 1), batch["labels"].flatten(), ignore_index=-100)
    loss.backward(); optimizer.step(); audit_optimizer(optimizer, device)
    return loss.item()
```

Part 5 - The Philosophy Behind the Exercise

A device is a location with movement costs and execution rules, not a decorative property on a model. CUDA training is correct only when the whole stateful system inhabits a coherent computational world.

Acceleration becomes engineering when placement, transfer, overlap, and memory are observable. The fastest hidden copy is the one eliminated by a clear data contract.

17.7.12 Exercise 11

Exercise Title: Profile GPU Utilization During Transformer Training and Fix Under-Utilization Bottlenecks in Data Pipeline

Part 1 - Why this matters

Low average GPU utilization is a symptom, not a root cause. The accelerator may wait for storage, tokenization, collation, host-to-device copies, Python control flow, synchronization, tiny kernels, or frequent evaluation and logging. It may also alternate between short intense kernels and unavoidable gaps, making one coarse utilization percentage misleading. Profiling must reconstruct the step timeline.

Instrument a representative training window with PyTorch Profiler after warmup. Capture CPU activities, CUDA kernels, shapes, memory, and a small number of stacks. Add named regions for data wait, transfer, forward, backward, clipping, optimizer, and logging. Compare profiler events with independent step timing and a system-level GPU timeline. Profile only

enough steps to answer the question; continuous detailed profiling can change the performance being measured.

Apply one intervention per diagnosed bottleneck. Pre-tokenize static data or add workers when CPU preparation dominates. Use length bucketing to reduce padding. Use pinned memory and nonblocking copies to expose overlap. Increase batch size or use accumulation when kernels are too small. Reduce per-step `item()` calls and synchronization-heavy logging. Consider fused or compiled operations only after correctness parity. Re-profile and report the change in useful tokens per second, latency distribution, memory, and quality.

Part 2 - Mathematical form and reasoning

Define step utilization proxy

$$u = \frac{t_{\text{active CUDA}}}{t_{\text{wall step}}},$$

recognizing that overlapping kernels must not be double-counted. If stages form a pipeline, achievable steady-state time approaches the maximum stage time, while a serialized implementation approaches their sum. Useful throughput is

$$R = \frac{\sum_{b,t} \mathbb{1}[Y_{b,t} \neq -100]}{t_{\text{wall}}}.$$

An optimization that increases device occupancy but processes more padding may raise hardware activity while reducing R.

Part 3 - Hints, controls, and evidence

1. Profile a steady-state window with the same model, batch distribution, precision, and logging policy used in the baseline.
2. Look for CPU gaps before kernels, memory copies without overlap, many tiny kernels, forced synchronizations, and oversized optimizer work.
3. Record p50, p95, and p99 step time; averages hide periodic stalls.
4. Validate numerical outputs and short-run training loss after each performance change.
5. Keep a before/after table linking each intervention to the timeline evidence that motivated it.

Part 4 - Python code skeleton

```
import torch
from torch.profiler import profile, ProfilerActivity, record_function

def profiled_steps(step_fn, iterator, output_dir, steps=8):
    schedule = torch.profiler.schedule(
        wait=1, warmup=1, active=steps, repeat=1)
    with profile(activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
                 schedule=schedule, record_shapes=True, profile_memory=True,
```

```
                on_trace_ready=torch.profiler.tensorboard_trace_handler(
                    output_dir)) as prof:
        for _ in range(steps + 2):
            with record_function("data_wait"):
                batch = next(iterator)
            with record_function("training_step"):
                step_fn(batch)
            prof.step()
    return prof.key_averages().table(sort_by="self_cuda_time_total",
                                     row_limit=30)

def E11(make_step, loader, output_dir="traces/trf05_e11"):
    iterator = iter(loader)
    baseline_step = make_step(optimized=False)
    baseline = profiled_steps(baseline_step, iterator, output_dir)
    # TODO: apply one evidence-backed intervention and profile again.
    return baseline
```

Part 5 - The Philosophy Behind the Exercise

Utilization is the visible rhythm of dependencies. Idle time means some required fact, resource, or permission did not arrive when the accelerator was ready. The timeline makes that dependency concrete.

Optimization should be a sequence of explanations: observe a gap, identify its cause, change the cause, and verify that the gap and end-to-end cost changed as predicted.

17.7.13 Exercise 12

Exercise Title: Benchmark Training Throughput Across Sequence Lengths and Identify Practical Maximum Context for Your GPU

Part 1 - Why this matters

Dense self-attention makes sequence length a dominant systems variable. IDs, embeddings, and many FFN activations scale approximately linearly with T, but attention interactions scale as T^2. The longest sequence that survives one forward pass is not a practical training context. A usable configuration needs headroom for backward, optimizer state, allocator variation, evaluation, and unusually long or padded batches.

Benchmark a matrix of context lengths and batch sizes using the real model, precision, optimizer, gradient accumulation, and activation-checkpoint policy. For each point, warm up and measure forward time, backward time, full update time, peak allocated/reserved memory, valid tokens per second, samples per second, and padding efficiency. Use fixed-length synthetic batches to isolate computational scaling, then repeat with the real length distribution to include collation and padding. Run risky points in isolated processes so an OOM does not leave the main benchmark in a fragmented state.

Define a practical maximum before inspecting results. Require several consecutive successful steps, stable latency percentiles, no non-finite values, and a memory reserve such as

10–20 percent for operational variation. Compare batch reduction, accumulation, checkpointing, mixed precision, Flash/SDPA kernels where appropriate, and length bucketing. The final choice should be a Pareto decision among retained context, throughput, memory, and learning quality.

Part 2 - Mathematical form and reasoning

For batch B, heads H, sequence T, head width d_h, and activation byte width s, an explicit score tensor costs approximately

$$M_{\text{score}} = BHT^2 s,$$

while token representations cost approximately $BTd_{\text{model}}s$. Define memory headroom

$$h = 1 - \frac{M_{\text{peak reserved}}}{M_{\text{device}}}.$$

A practical maximum T^* is the largest tested context satisfying the chosen headroom, stability, and latency constraints, not merely the largest T for which one allocation succeeded.

Part 3 - Hints, controls, and evidence

1. Reset peak statistics and synchronize timing boundaries for every trial.
2. Hold model and valid-token budget constant when isolating sequence-length effects, then separately study realistic batch construction.
3. Record kernel/backend selection because fused attention can change memory scaling constants substantially.
4. Repeat points near the limit and report p50/p95 step time; allocator and shape variation matter there.
5. Validate loss and gradient parity after enabling any memory-efficient attention or checkpointing implementation.

Part 4 - Python code skeleton

```
import torch

def benchmark_context(make_step, B, T, repeats=20):
    torch.cuda.empty_cache(); torch.cuda.reset_peak_memory_stats()
    step = make_step(batch_size=B, sequence_length=T)
    for _ in range(5): step()
    start = torch.cuda.Event(enable_timing=True)
    end = torch.cuda.Event(enable_timing=True)
    times = []
    for _ in range(repeats):
        start.record(); metrics = step(); end.record(); end.synchronize()
        times.append(start.elapsed_time(end))
    values = torch.tensor(times)
    return {"batch": B, "sequence": T,
```

```
            "median_ms": values.median().item(),
            "p95_ms": values.quantile(.95).item(),
            "peak_allocated": torch.cuda.max_memory_allocated(),
            "peak_reserved": torch.cuda.max_memory_reserved(),
            "valid_tokens_s": metrics["valid_tokens"]/(values.median()/1000)}

def E12(make_step):
    return [benchmark_context(make_step, B, T)
            for T in (128, 256, 512, 1024, 2048) for B in (1, 2, 4)]
```

Part 5 - The Philosophy Behind the Exercise

A hardware limit is not a single wall; it is a boundary shaped by context, batching, precision, algorithms, and required safety margin. Engineering asks not "Can it run once?" but "Can it run reliably under the real distribution?"

Maximum context is meaningful only alongside its cost. A responsible choice states what additional information is retained and what throughput, memory, or operational resilience is surrendered to retain it.

17.7.14 Exercise 13

Exercise Title: Evaluate Determinism and Reproducibility and Document When CUDA Transformers Become Non-Deterministic

Part 1 - Why this matters

Reproducibility has levels. Exact continuation asks whether the same state and next batch produce identical tensors. Repeatability asks whether two runs under the same environment reproduce a trajectory. Statistical reproducibility asks whether independent seeds support the same conclusion. CUDA kernels, reduction order, atomics, data-worker scheduling, dropout, mixed precision, compiler selection, and distributed communication can prevent bitwise identity even when training is scientifically stable.

Create a reproducibility manifest containing code revision, configuration, dataset/tokenizer fingerprints, package and driver versions, GPU model, precision, compiler flags, seeds, deterministic settings, and selected kernels. Seed Python and PyTorch, seed DataLoader workers, pass a seeded generator to the sampler, and enable deterministic algorithms for the strict condition. Configure backend options deliberately and record any operation that fails because no deterministic implementation exists. Do not silently replace the strict test with a best-effort run.

Compare two same-seed runs at increasing resolution: sample order, initial state hash, first logits, first loss, first gradients, first update, checkpoint hashes, and final metrics. Locate the first divergence rather than comparing only final loss. Then run multiple different seeds and report distributions. Document the performance cost of deterministic settings and distinguish exact, tolerance-based, and statistical guarantees.

Part 2 - Mathematical form and reasoning

Let the training transition be

$$s_{t+1} = F(s_t, b_t, r_t, e),$$

where s_t includes model/optimizer/scheduler state, b_t is the batch, r_t is random state, and e is the software/hardware environment. Bitwise reproducibility requires all arguments and the realized implementation of F to match. For statistical comparison across n seeds, report mean and standard deviation

$$\bar{m} = \frac{1}{n}\sum_i m_i, qquads_m = \sqrt{\frac{1}{n-1}\sum_i (m_i - \bar{m})^2}.$$

Exact equality and stable distributions answer different questions.

Part 3 - Hints, controls, and evidence

1. Set environment-dependent deterministic configuration before CUDA work begins, then record it in the manifest.
2. Hash ordered sample IDs per epoch and state dictionaries at defined steps.
3. Disable dropout first to isolate kernel/data nondeterminism, then re-enable it with restored RNG states.
4. Test CPU, single-GPU, and the intended multi-GPU condition separately.
5. Treat tolerance selection as part of the claim; justify absolute and relative thresholds from dtype and operation behavior.

Part 4 - Python code skeleton

```
import hashlib, random, torch

def configure_determinism(seed):
    random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.use_deterministic_algorithms(True)
    torch.backends.cudnn.benchmark = False
    torch.backends.cudnn.deterministic = True

def state_hash(model):
    digest = hashlib.sha256()
    for name, tensor in sorted(model.state_dict().items()):
        digest.update(name.encode())
        digest.update(tensor.detach().cpu().contiguous().numpy().tobytes())
    return digest.hexdigest()

def compare_runs(run_a, run_b):
    for key in ("sample_ids", "initial_hash", "first_loss",
                "first_gradient_hash", "first_update_hash"):
```

```
        if run_a[key] != run_b[key]: return {"first_divergence": key}
    return {"first_divergence": None}

def E13(run_training, seed=13):
    configure_determinism(seed); a = run_training(seed)
    configure_determinism(seed); b = run_training(seed)
    return {"comparison": compare_runs(a, b), "run_a": a, "run_b": b}
```

Part 5 - The Philosophy Behind the Exercise

Reproducibility is the ability to say which parts of a result belong to the idea and which belong to circumstance. Exact repetition controls circumstance tightly; multi-seed evidence asks whether the conclusion survives variation.

Honest research names the level it achieved. "Deterministic" is not a mood of confidence but a testable contract over state, data, randomness, implementation, and environment.

17.7.15 Exercise 14

Exercise Title: Implement Checkpoint Save and Resume Including Optimizer and Scheduler States and Verify Exact Continuation

Part 1 - Why this matters

A model state dictionary is enough for inference, but not for exact training continuation. Adam moments, scheduler position, GradScaler state, global update, epoch/batch or data-stream cursor, sampler state, gradient-accumulation boundary, RNG states, tokenizer identity, and configuration all influence the next transition. Omitting one can produce a run that loads successfully and then silently follows a different trajectory.

Design a versioned checkpoint schema and write atomically: save to a temporary file, flush and close it, then rename within the same filesystem. Include model, optimizer, scheduler, scaler, counters, RNG state for Python/CPU/CUDA, and data position. Store hashes of the tokenizer/vocabulary and dataset manifest and reject incompatible resumes. Save at an optimizer-step boundary unless partial accumulated gradients and microstep index are intentionally supported. Map the checkpoint to CPU for portability, then restore tensors to the target device through model and optimizer loading.

Prove exact continuation with a fork experiment. Run K updates, save, continue M updates, and record batches, learning rates, losses, scales, gradients, and parameter hashes. Independently reload the K checkpoint and run the same M updates. Under deterministic conditions, every recorded value should match. Also simulate interruption during write and corruption; the last complete checkpoint must remain usable and integrity verification must fail early.

Part 2 - Mathematical form and reasoning

Define complete state

$$S_k = (\theta_k, o_k, q_k, a_k, r_k, d_k, c),$$

for model, optimizer, scheduler, AMP scaler, RNG, data cursor, and immutable configuration. Exact continuation requires

$$F^M(S_K) = F^M(\text{load}(\text{save}(S_K))).$$

A hash verifies byte integrity of the serialized artifact, while schema and fingerprint checks verify semantic compatibility. Both are required: an intact checkpoint can still belong to the wrong tokenizer or dataset.

Part 3 - Hints, controls, and evidence

1. Record a schema version and implement explicit migration or rejection; never guess missing fields.
2. Restore RNG and sampler/data state before requesting the next batch.
3. Inspect optimizer-state devices after loading and before the first step.
4. Retain more than one checkpoint and a manifest so a corrupt newest file does not eliminate recovery.
5. Compare uninterrupted and resumed trajectories step by step, not merely their final validation metric.

Part 4 - Python code skeleton

```
import os, random, torch

def capture_rng():
    cuda_state = (torch.cuda.get_rng_state_all()
                  if torch.cuda.is_available() else None)
    return {"python": random.getstate(), "cpu": torch.get_rng_state(),
            "cuda": cuda_state}

def restore_rng(state):
    random.setstate(state["python"]); torch.set_rng_state(state["cpu"])
    if state["cuda"] is not None: torch.cuda.set_rng_state_all(state["cuda"])

def save_checkpoint(path, model, optimizer, scheduler, scaler,
                    progress, meta):
    payload = {"schema": 1, "model": model.state_dict(),
               "optimizer": optimizer.state_dict(),
               "scheduler": scheduler.state_dict(),
               "scaler": None if scaler is None else scaler.state_dict(),
               "progress": progress, "rng": capture_rng(), "meta": meta}
    temporary = path + ".temporary"
    torch.save(payload, temporary); os.replace(temporary, path)

def load_checkpoint(path, model, optimizer, scheduler, scaler):
    state = torch.load(path, map_location="cpu", weights_only=False)
    if state["schema"] != 1: raise ValueError("unsupported checkpoint schema")
    model.load_state_dict(state["model"])
```

```
    optimizer.load_state_dict(state["optimizer"])
    scheduler.load_state_dict(state["scheduler"])
    if scaler is not None: scaler.load_state_dict(state["scaler"])
    restore_rng(state["rng"])
    return state["progress"], state["meta"]

def E14(fork_continuation_test):
    return fork_continuation_test(save_checkpoint, load_checkpoint)
```

Part 5 - The Philosophy Behind the Exercise

A checkpoint is a promise that time can be interrupted without changing the future. Saving weights preserves the model's location, but exact continuation also requires momentum, schedule, randomness, and place in the data stream.

Recovery is trustworthy when interruption becomes a tested state transition, not an emergency hope. The artifact must preserve both bytes and the meaning of those bytes.

17.7.16 Exercise 15

Exercise Title: Build Monitoring That Tracks Gradient Norms, Learning Rate, and Loss Spikes and Triggers Automatic Alerts

Part 1 - Why this matters

Training can remain alive while becoming invalid or wasteful. Loss may spike, gradients may vanish or explode, the learning rate may advance at the wrong frequency, AMP may skip many updates, throughput may collapse, or validation may degrade. Monitoring converts these hidden dynamics into time-indexed evidence and enables a fast response before a long run consumes its budget.

Define a compact per-update record: global update, microstep, epoch/data cursor, train loss, valid-token count, learning rate per group, gradient norm before and after clipping, parameter norm, update norm, AMP scale and skipped-step flag, step time, tokens per second, peak memory, and sample IDs for anomalies. Add periodic validation metrics and generation probes. Aggregate expensive layer- wise statistics less frequently. Avoid forcing a CUDA synchronization every step merely to log a scalar.

Implement alerts as explicit rules with warmup, robust baselines, persistence, and cooldown. Examples include any non-finite value, loss exceeding a rolling median by several median absolute deviations, clipping active for a sustained fraction of steps, repeated AMP skips, learning rate differing from the expected schedule, or throughput below a baseline. Classify actions: log only, save an incident snapshot, pause safely, or terminate. Test every rule with synthetic metric streams and record false positives.

Part 2 - Mathematical form and reasoning

For recent loss window W_t, define robust center $m_t = \text{median}(W_t)$ and deviation

$$d_t = \text{median}_{x \in W_t} |x - m_t|.$$

A spike score is

$$z_t = \frac{L_t - m_t}{1.4826 d_t + \epsilon}.$$

An alert may require $z_t > z_{\max}$ for q of the last r updates, reducing reaction to one harmless noisy batch. Rule thresholds and action severity are part of the experimental configuration and must be versioned.

Part 3 - Hints, controls, and evidence

1. Separate raw event capture from dashboards and alerts; retain raw values for later reinterpretation.
2. Use update count as the primary training time axis and also record valid tokens seen.
3. Include configuration, seed, code revision, device, and checkpoint ID in every run-level record.
4. Replay healthy and injected-failure streams to measure detection delay, missed alerts, and false alarms.
5. Ensure alert handling cannot corrupt the checkpoint it attempts to save.

Part 4 - Python code skeleton

```
from collections import deque
import math, statistics

class TrainingMonitor:
    def __init__(self, window=100, spike_z=8.0):
        self.losses = deque(maxlen=window); self.spike_z = spike_z
        self.events = []
    def observe(self, record):
        alerts = []
        for key in ("loss", "grad_norm", "learning_rate"):
            if not math.isfinite(record[key]):
                alerts.append(f"nonfinite:{key}")
        if len(self.losses) >= 20:
            center = statistics.median(self.losses)
            mad = statistics.median(abs(x-center) for x in self.losses)
            z = (record["loss"]-center)/(1.4826*mad+1e-12)
            if z > self.spike_z: alerts.append(f"loss_spike:{z:.2f}")
        self.losses.append(record["loss"])
        self.events.append({**record, "alerts": alerts})
        return alerts

def E15(metric_stream):
    monitor = TrainingMonitor()
    triggered = [monitor.observe(record) for record in metric_stream]
    # TODO: test each alert rule with injected metric streams.
    return monitor.events, triggered
```

Part 5 - The Philosophy Behind the Exercise

Monitoring is attention directed at the training process itself. It allows the system to become an object of observation while it learns, preserving signals that would otherwise vanish into a final checkpoint.

An alert is an operational hypothesis: this pattern predicts danger strongly enough to justify action. Like any hypothesis, it must be calibrated, tested, and revised from evidence rather than fear.

17.7.17 Exercise 16

Exercise Title: Implement Early Stopping and Compare Against Longer Training With Regularization for Small Datasets

Part 1 - Why this matters

On small datasets, training loss may continue falling after validation quality has stopped improving. Early stopping selects a checkpoint using a held-out signal and prevents additional updates that appear to specialize to the training set. It is a sequential model-selection procedure, not merely a way to save time. Repeatedly observing the same validation set also adapts decisions to that set, so a final untouched test set remains necessary.

Implement direction-aware improvement, a minimum meaningful delta, patience, warmup before monitoring, and restoration of the best complete checkpoint. The patience unit must be explicit: validation evaluations, not training batches. Handle ties and non-finite metrics deliberately. Save the best model, optimizer, scheduler, scaler, and progress if training may resume from that point; saving only weights is enough only for final inference evaluation.

Compare at least three matched strategies across several seeds: longer training without regularization, early stopping, and longer training with tuned weight decay/dropout or another declared regularizer. Give each the same maximum update budget and validation cadence. Report best validation update, final and restored metrics, test quality, generalization gap, wall time, and seed dispersion. Early stopping may be preferable, complementary, or inferior depending on noise and the task; the experiment should discover the relationship.

Part 2 - Mathematical form and reasoning

For validation metric m_k evaluated at checkpoints k, maximization with minimum delta δ accepts improvement when

$$m_k > m^* + \delta.$$

Otherwise a counter increments. Training stops after P consecutive non-improving evaluations and returns

$$k^* = \arg\max_{k \le k_{\text{stop}}} m_k.$$

For loss minimization, reverse the inequality. This adaptive choice means the reported validation maximum is optimistically selected; unbiased final claims should use an untouched test set.

Part 3 - Hints, controls, and evidence

1. Unit-test maximization and minimization, ties, delta, patience, warmup, NaN metrics, and restoration.
2. Save a new best checkpoint atomically at the moment improvement is accepted.
3. Smooth only if the policy is declared in advance; smoothing changes stopping delay and selection behavior.
4. Use several seeds because noisy small validation sets can select very different stopping points.
5. Evaluate the restored best checkpoint, not the final in-memory parameters.

Part 4 - Python code skeleton

```
import math

class EarlyStopping:
    def __init__(self, patience, min_delta=0.0, mode="min", warmup=0):
        self.patience, self.min_delta = patience, min_delta
        self.mode, self.warmup = mode, warmup
        self.best, self.bad, self.best_step = None, 0, None
    def update(self, metric, step):
        if not math.isfinite(metric): return {"improved": False, "stop": True}
        better = (self.best is None or
                  (metric < self.best-self.min_delta if self.mode == "min"
                   else metric > self.best+self.min_delta))
        if better:
            self.best, self.best_step, self.bad = metric, step, 0
        elif step >= self.warmup:
            self.bad += 1
        return {"improved": better,
                "stop": step >= self.warmup and self.bad >= self.patience}

def E16(run_strategy):
    return {name: [run_strategy(name, seed) for seed in (1, 2, 3, 4, 5)]
            for name in ("long_unregularized", "early_stop",
                         "long_regularized")}
```

Part 5 - The Philosophy Behind the Exercise

Early stopping treats continued learning as a decision that must remain justified by external evidence. More optimization is not automatically more knowledge; after a point it may become narrower accommodation to the observed sample.

Knowing when to stop is part of learning. The decision is credible when its criterion, uncertainty, and comparison with alternatives are made explicit.

17.7.18 Exercise 17

Exercise Title: Build a Misclassification or Bad-Generation Collector and Save Worst Samples for Manual Analysis

Part 1 - Why this matters

Aggregate metrics compress thousands of different outcomes into one number. Manual analysis needs the examples that most strongly contradict the desired behavior: high-confidence wrong classifications, high-NLL tokens, early EOS, repetition loops, target continuations assigned low probability, or generations that violate a task-specific rule. A collector turns these failures into a ranked, traceable dataset rather than a collection of memorable anecdotes.

Define a task-specific badness score before reading examples. During evaluation mode with gradients disabled, preserve example ID, split, source reference, decoded input, target, prediction, token IDs, mask, per-token loss, confidence or entropy, generation settings, checkpoint ID, tokenizer fingerprint, and selected internal diagnostics. Keep only the worst K samples using a bounded heap so large validation sets do not exhaust memory. Store references rather than private raw text when governance requires it.

Stratify the collector. A global top-K may contain only long sequences because summed NLL grows with length, or only one frequent class. Use mean valid-token NLL for length normalization and keep separate queues by class, length bin, language/domain, or failure type. Deduplicate repeated windows from the same document. Compare collectors across checkpoints to distinguish newly fixed, persistent, and newly introduced failures. Manual tags should feed regression tests and data/model hypotheses, not be treated as ground truth automatically.

Part 2 - Mathematical form and reasoning

For sequence n with valid set $\mathcal{I}_n$, define normalized NLL

$$s_n = \frac{1}{|\mathcal{I}_n|} \sum_{t \in \mathcal{I}_n} -\log p_\theta(y_{n,t} \mid x_{n,\le t}).$$

A high-confidence classification error can use $s_n = \max_c p_\theta(c \mid x_n)$ when prediction is wrong. Repetition metrics may measure duplicate n-gram fraction. Ranking defines what enters human attention, so its bias must be documented and countered with stratified samples.

Part 3 - Hints, controls, and evidence

1. Unit-test ranking with hand-constructed scores, ties, duplicate IDs, and fewer than K examples.
2. Decode with the exact checkpoint tokenizer and preserve IDs for forensic reconstruction.
3. Include random and median-quality controls beside worst samples; without them reviewers lose a reference distribution.
4. Never feed validation/test examples back into training without recording the resulting split contamination and creating a fresh evaluation set.
5. Version human failure categories and measure reviewer disagreement when labels are subjective.

Part 4 - Python code skeleton

```
import heapq, torch

class WorstCollector:
    def __init__(self, k): self.k, self.heap, self.counter = k, [], 0
    def add(self, score, example_id, payload):
        self.counter += 1
        item = (float(score), str(example_id), self.counter, payload)
        if len(self.heap) < self.k: heapq.heappush(self.heap, item)
        elif item[:2] > self.heap[0][:2]: heapq.heapreplace(self.heap, item)
    def sorted(self): return sorted(self.heap, reverse=True)

@torch.inference_mode()
def collect_bad_generations(model, loader, tokenizer, generate, k=50):
    model.eval(); collector = WorstCollector(k)
    for batch in loader:
        logits = model(batch["input_ids"], batch["attention_mask"])
        per_token = torch.nn.functional.cross_entropy(
            logits.transpose(1, 2), batch["labels"],
            ignore_index=-100, reduction="none")
        valid = batch["labels"].ne(-100)
        scores = (per_token*valid).sum(1)/valid.sum(1).clamp_min(1)
        for i, score in enumerate(scores):
            collector.add(score, batch["example_ids"][i],
                          {"input_ids": batch["input_ids"][i].cpu(),
                           "target_ids": batch["labels"][i].cpu()})
    return collector.sorted()

def E17(model, loader, tokenizer, generate):
    return collect_bad_generations(model, loader, tokenizer, generate)
```

Part 5 - The Philosophy Behind the Exercise

Metrics describe populations; failures restore individuality. The worst-sample collector gives concrete form to what an average hides and directs scarce human attention toward consequential uncertainty.

But selection shapes perception. A ranking is not a neutral window onto failure; it is a policy about which failures deserve to be seen. Diversity and provenance make that policy more honest.

17.7.19 Exercise 18

Exercise Title: Achieve Near-Perfect Performance on a Simple Synthetic Task and Use It as a Regression Test for Stability

Part 1 - Why this matters

A synthetic task can isolate one Transformer capability and provide an exact data generator, known solution, and strict success threshold. Unlike a single fixed tiny dataset, fresh synthetic examples can test generalization within a controlled rule. Once the system repeatedly solves the task, the experiment becomes a regression test: future changes to masks, precision, optimizer order, checkpointing, or kernels must preserve the capability and stability envelope.

Choose a task that needs the mechanism being tested, such as copying a marked token from an earlier position, reversing a short sequence, or retrieving a value associated with a key. Create disjoint train and validation streams from the same rule and a harder extrapolation split with longer sequences or more distractors. Establish chance performance and a simpler baseline. Train under a fixed compute budget and require near-perfect validation performance over multiple seeds, not merely one lucky run.

Package the data generator, configuration, seeds, thresholds, timeout, expected learning curve, and failure artifacts. Include invariance checks: permuting irrelevant distractors should not change the answer, causal models must not see future targets, and CPU/CUDA outputs should agree within tolerance. Add stress variants for high learning rate, mixed precision, checkpoint resume, and gradient accumulation. A regression test should fail loudly with the first diagnostic difference, not only a final low score.

Part 2 - Mathematical form and reasoning

For generated sample $(x, y) \sim G_\phi$, empirical validation accuracy is

$$\widehat{A} = \frac{1}{N} \sum_{n=1}^{N} \mathbb{1}[f_\theta(x_n) = y_n].$$

If the task has C balanced outputs, chance accuracy is $1/C$. A regression contract may require, for every seed s in a fixed suite,

$$\widehat{A}_s \geq A_{\min}, \quad L_s \leq L_{\max}, \quad u_s \leq U_{\max}, \quad \text{all monitored tensors finite},$$

where u_s is updates required to cross the performance threshold.

Part 3 - Hints, controls, and evidence

1. Make generation deterministic per sample seed and save failing seeds for exact replay.
2. Verify dataset logic with a manual oracle independent of the model and training implementation.
3. Include train-rule validation and length/distractor extrapolation as separate claims.
4. Select thresholds from repeated healthy reference runs with justified margin, not from the single best run.
5. Keep the regression small enough for routine execution while retaining a nightly stress variant for broader configurations.

Part 4 - Python code skeleton

```
import torch

def make_retrieval_batch(seed, batch_size, pairs, vocab=32):
    g = torch.Generator().manual_seed(seed)
    keys = torch.randint(2, vocab, (batch_size, pairs), generator=g)
    values = torch.randint(2, vocab, (batch_size, pairs), generator=g)
    query_index = torch.randint(0, pairs, (batch_size,), generator=g)
    query = keys[torch.arange(batch_size), query_index]
    target = values[torch.arange(batch_size), query_index]
    # Interleave key/value pairs, then append a query marker and query key.
    sequence = torch.stack([keys, values], dim=-1).flatten(1)
    sequence = torch.cat([sequence, torch.ones(batch_size, 1,
                          dtype=torch.long), query[:, None]], dim=1)
    return sequence, target

def E18(train_and_evaluate):
    runs = []
    for seed in (18, 19, 20):
        result = train_and_evaluate(seed=seed, task=make_retrieval_batch,
                                    max_updates=1000)
        assert result["validation_accuracy"] >= 0.99
        assert result["all_finite"] and result["updates"] <= 1000
        runs.append(result)
    return runs
```

Part 5 - The Philosophy Behind the Exercise

Synthetic tasks are microscopes: they simplify the world so one mechanism can be observed without the noise of natural data. Their artificiality is a strength when the claim is kept narrow and explicit.

A regression test transforms past understanding into a future boundary. It says that improvement may change the implementation, but must not erase a capability already demonstrated under controlled conditions.

17.7.20 Exercise 19

Exercise Title: Produce a Full Transformer Training Debugging Playbook Covering All Common Failure Scenarios

Part 1 - Why this matters

The final playbook turns chapter experiments into an operational decision system. Transformer failures cross data, objective, model, numerics, optimization, hardware, and recovery boundaries. Randomly changing learning rate, precision, masks, and batch size destroys evidence and can produce a temporary improvement without identifying the cause. A playbook preserves a consistent order: contain the incident, reproduce it, locate the earliest broken invariant, run a discriminating intervention, verify the repair, and add a regression test.

Organize entries by symptom: loss does not fall, loss spikes, NaN/Inf, zero or exploding gradients, repeated clipping, AMP skipped steps, improving loss with poor generation, overfitting, underfitting, OOM, slow/variable steps, low GPU utilization, nondeterminism, checkpoint mismatch, and resume drift. For each, list likely causes in causal order, required artifacts, immediate safe action, diagnostic tests, expected observations, corrective actions, and proof of resolution. Link symptoms to exercises rather than duplicating untested advice.

Define a standard incident bundle: configuration, revision, environment, tokenizer and data fingerprints, checkpoint, RNG state, data cursor, batch IDs, loss/gradient/LR/scale timeline, intermediate tensor summaries, profiler trace when relevant, and the last healthy comparison. Include severity and authority: which conditions warn, pause, save, roll back, or terminate. Rehearse the playbook using injected failures and measure time to detection, localization, and verified recovery.

Part 2 - Mathematical form and reasoning

Represent training as ordered contracts

$$D \to B \to Z \to L \to G \to U \to S',$$

for data, batch, logits, loss, gradients, update, and next state. Each transition f_k has invariant set $\mathcal{C}_k$. Given a symptom, seek

$$k^* = \min\{k : f_k(z_{k-1}) \notin \mathcal{C}_k\}.$$

A repair is accepted only when the original reproduction no longer violates $\mathcal{C}_{k^*}$, downstream contracts recover, and a regression test fails under the injected defect but passes under the corrected implementation.

Part 3 - Hints, controls, and evidence

1. Begin every branch with data identity, tensor contract, mode, device, dtype, finiteness, and update-order checks before hyperparameter speculation.
2. Preserve the failing state before experimenting; otherwise the incident may become irreproducible.
3. Change one factor per diagnostic run and state mutually exclusive expected outcomes that distinguish hypotheses.
4. Record owner, severity, detection rule, rollback point, root cause, fix, and regression-test ID for every resolved incident.
5. Review the playbook after real incidents and remove advice that has never been validated.

Part 4 - Python code skeleton

```
from dataclasses import dataclass, asdict
import json

@dataclass
```

```
class FailureEntry:
    symptom: str
    severity: str
    likely_causes: list
    required_evidence: list
    diagnostic_steps: list
    stop_conditions: list
    verified_fixes: list
    regression_tests: list

REQUIRED_SYMPTOMS = {"loss_flat", "loss_spike", "nonfinite", "bad_generation",
                     "oom", "low_utilization", "resume_drift",
                     "nondeterminism"}

def validate_playbook(entries):
    symptoms = {e.symptom for e in entries}
    missing = REQUIRED_SYMPTOMS - symptoms
    if missing:
        message = f"missing failure branches: {sorted(missing)}"
        raise ValueError(message)
    for entry in entries:
        assert entry.required_evidence and entry.diagnostic_steps
        assert entry.verified_fixes and entry.regression_tests

def E19(entries, drill_results, path="trf05_debugging_playbook.json"):
    validate_playbook(entries)
    report = {"schema": 1, "entries": [asdict(e) for e in entries],
              "failure_injection_drills": drill_results}
    with open(path, "w", encoding="utf-8") as handle:
        json.dump(report, handle, indent=2)
    return report
```

Part 5 - The Philosophy Behind the Exercise

A debugging playbook is accumulated causal knowledge. It replaces improvisation under pressure with a sequence of observations that protects evidence and narrows uncertainty. Its value lies not in containing every possible answer, but in teaching the system how to ask the next decisive question.

The chapter ends where mature engineering begins: failure is no longer treated as an interruption external to learning. It becomes a reproducible experiment, a refined invariant, and a permanent improvement in collective memory.

17.8 TRF06

Chapter Title: Decoder-Only Language Modeling and Generation
Specialization: Models and Training

17.8.1 Exercise 00

Exercise Title: Implement a Minimal Decoder-Only Transformer and Validate Causal Mask and Next-Token Prediction Dataset

Part 1 - Why this matters

A decoder-only Transformer learns one rule: at position t, predict the next token using only the prefix available through t. The architecture looks like an encoder stack—token and positional embeddings, repeated self-attention and feed-forward blocks, normalization, and a vocabulary projection—but causal masking changes its information contract. A model that can see future tokens may obtain an excellent training loss while learning a computation impossible during generation.

Build the smallest complete model without using `nn.Transformer`. Start from a full sequence `[BOS,a,b,EOS]`, form inputs `[BOS,a,b]` and labels `[a,b,EOS]`, and keep padding labels at `-100`. Construct an allowed-attention matrix in which query i may read key j exactly when $j \leq i$ and the key is not padding. Track shapes through `(B,T)`, `(B,T,d)`, heads `(B,H,T,d_h)`, attention scores `(B,H,T,T)`, and logits `(B,T,V)`.

Validate causality through intervention, not inspection alone. Create two sequences with identical prefixes and different future suffixes. Logits for positions inside the shared prefix must be equal in evaluation mode. Remove the causal mask and prove the invariant fails. Also verify that the model can accept different batch sizes, rejects sequences beyond its positional capacity, and computes loss only for valid shifted labels.

Part 2 - Mathematical form and reasoning

The language-model factorization is

$$p_\theta(z_1, \ldots, z_T \mid z_0) = \prod_{t=0}^{T-1} p_\theta(z_{t+1} \mid z_0, \ldots, z_t).$$

For attention scores $S = QK^\top/\sqrt{d_h}$, define

$$M_{i,j} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i, \end{cases} \qquad A = \mathrm{softmax}(S + M).$$

The loss over valid targets $\mathcal{I}$ is

$$L = -\frac{1}{|\mathcal{I}|} \sum_{(b,t)\in\mathcal{I}} \log p_\theta(y_{b,t} \mid x_{b,\leq t}).$$

The architectural contract and dataset shift express the same temporal claim.

Part 3 - Hints, controls, and evidence

1. Use unequal values for batch, sequence, heads, and head width so swapped axes cannot pass by coincidence.
2. Print the Boolean allowed matrix before converting masked locations to a large negative value.

3. Assert every valid attention row sums to one and every future weight is zero.

4. Compare shared-prefix logits under changed suffixes, both with and without the causal mask.

5. Record parameter count, shapes, mask semantics, loss denominator, and the first violated invariant for every injected failure.

Part 4 - Python code skeleton

```
import torch
from torch import nn

class DecoderOnlyModel(nn.Module):
    def __init__(self, vocabulary_size, model_width, heads, layers,
                 maximum_length):
        super().__init__()
        self.token_embedding = nn.Embedding(vocabulary_size, model_width)
        self.position_embedding = nn.Embedding(maximum_length, model_width)
        block = nn.TransformerEncoderLayer(
            model_width, heads, 4 * model_width, batch_first=True,
            norm_first=True)
        self.blocks = nn.TransformerEncoder(block, layers)
        self.final_norm = nn.LayerNorm(model_width)
        self.output = nn.Linear(model_width, vocabulary_size, bias=False)

    def forward(self, input_ids, padding_mask=None):
        batch_size, sequence_length = input_ids.shape
        positions = torch.arange(sequence_length, device=input_ids.device)
        hidden = self.token_embedding(input_ids)
        hidden = hidden + self.position_embedding(positions)[None, :, :]
        future_mask = torch.triu(torch.ones(sequence_length, sequence_length,
                                 device=input_ids.device, dtype=torch.bool), 1)
        hidden = self.blocks(hidden, mask=future_mask,
                             src_key_padding_mask=padding_mask)
        return self.output(self.final_norm(hidden))

def E00(model, prefix_a, prefix_b):
    model.eval()
    with torch.no_grad():
        logits_a = model(prefix_a)
        logits_b = model(prefix_b)
    shared_length = 3
    assert torch.allclose(logits_a[:, :shared_length],
                          logits_b[:, :shared_length])
    return logits_a, logits_b
```

Part 5 - The Philosophy Behind the Exercise

Causality is a boundary on knowledge. The model is powerful not because it may read everything, but because it learns to act under the same information limits that will exist when the future has not yet been generated.

Architecture and supervision must agree about time. When they do, next-token prediction becomes a valid bridge from a static corpus to sequential creation.

17.8.2 Exercise 01

Exercise Title: Train a Small Decoder-Only Model and Confirm Loss Decreases and Generation Becomes Less Random Over Time

Part 1 - Why this matters

A decreasing teacher-forced loss proves that the model is assigning more probability to observed next tokens, but it does not automatically show how free-running generations change. Early in training, the distribution is close to random; later, frequent local patterns should become visible, entropy should decline where the corpus is predictable, and generated samples should respect more structure. This exercise connects optimization evidence to behavioral evidence without pretending that one pleasant sample measures quality.

Train a small decoder on a tiny corpus or controlled pattern dataset. Freeze the tokenizer and split documents before building windows. Save checkpoints at initialization and at several fixed update counts. At every checkpoint, evaluate validation NLL and perplexity, then generate from the same prompt suite with the same decoding method, seed, temperature, and maximum length. Preserve raw probabilities or entropy traces so changes can be explained.

Track valid-token accuracy, NLL by position, entropy, EOS frequency, repetition, and exact pattern success. A model becoming less random does not necessarily mean it becomes better: it can collapse into high-confidence repetition. Compare against a unigram baseline and a memorization control. Overfit a tiny batch first to validate the entire pipeline, then train the intended split. If training loss falls while validation loss rises, describe overfitting rather than calling the generation improvement complete.

Part 2 - Mathematical form and reasoning

At checkpoint k, validation perplexity is

$$\mathrm{PPL}_k = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log p_{\theta_k}(y_i \mid x_{\le i})\right).$$

Distribution uncertainty at generation step t is

$$H_t = -\sum_{v=1}^{V} p_t(v)\log p_t(v).$$

Lower entropy is evidence of greater concentration, not automatically greater correctness. Behavioral improvement requires concentration to move toward tokens consistent with the task or held-out data.

Part 3 - Hints, controls, and evidence

1. Save the initial untrained model as checkpoint zero; it is the true random reference.
2. Use fixed prompts and decoding seeds at every checkpoint.
3. Separate teacher-forced validation metrics from autoregressive generation metrics in every plot and report.
4. Inspect multiple prompts and checkpoints; one selected output is not representative evidence.
5. Include a shuffled-label run to show that optimizer activity alone does not create meaningful structure.

Part 4 - Python code skeleton

```
import math
import torch
import torch.nn.functional as F

def train_step(model, batch, optimizer):
    model.train()
    optimizer.zero_grad(set_to_none=True)
    logits = model(batch["input_ids"], batch["padding_mask"])
    flat_logits = logits.reshape(-1, logits.size(-1))
    flat_labels = batch["labels"].reshape(-1)
    loss = F.cross_entropy(flat_logits, flat_labels, ignore_index=-100)
    loss.backward()
    optimizer.step()
    return loss.item()

def E01(model, loader, optimizer, evaluate, generate, checkpoints):
    report = []
    iterator = iter(loader)
    maximum_update = max(checkpoints)
    for update in range(maximum_update + 1):
        if update in checkpoints:
            validation_nll = evaluate(model)
            samples = generate(model)
            report.append({"update": update,
                           "validation_nll": validation_nll,
                           "perplexity": math.exp(validation_nll),
                           "samples": samples})
        if update == maximum_update:
            break
        try:
            batch = next(iterator)
        except StopIteration:
            iterator = iter(loader)
            batch = next(iterator)
```

```
        train_step(model, batch, optimizer)
    return report
```

Part 5 - The Philosophy Behind the Exercise

Learning changes a distribution before it produces an impressive artifact. Watching checkpoints reveals that transformation as a process rather than judging intelligence from one final string.

Randomness and certainty are not opposites of quality. The goal is calibrated structure: confidence where evidence supports it and uncertainty where several continuations remain plausible.

17.8.3 Exercise 02

Exercise Title: Implement Greedy Decoding and Explain Why It Often Produces Repetitive Output in Practice

Part 1 - Why this matters

Greedy decoding selects the highest-probability token at every step. It is deterministic, inexpensive, and extremely useful for debugging because the same prompt and model produce the same path. However, the locally most probable token need not begin the globally best continuation. Once a repeated phrase becomes slightly more probable than alternatives, feeding it back into the model can increase its own probability and create a stable loop.

Implement generation as an explicit loop. Start in evaluation mode, disable gradients, run the current prefix, take the final-position logits, select `argmax`, append the token, and stop on EOS or the token budget. Record the chosen token, its probability, distribution entropy, and whether the new n-gram already appeared. Do not sample and do not apply temperature in the greedy baseline.

Test a hand-constructed probability tree in which the best first token leads to a lower total sequence probability than the second-best first token. Then run real prompts and quantify repeated unigrams, bigrams, and longer n-grams. Compare teacher-forced next-token predictions with free-running output to show how feedback changes contexts. Greedy decoding is valuable as a reproducible baseline, but poor repetition does not prove the model itself assigns no mass to better alternatives.

Part 2 - Mathematical form and reasoning

Greedy decoding computes

$$\widehat{y}_t = \arg\max_{v \in \mathcal{V}} p_\theta(v \mid x, \widehat{y}_{<t}).$$

The sequence probability is a product of conditional probabilities,

$$p(\widehat{y}_{1:T} \mid x) = \prod_{t=1}^{T} p(\widehat{y}_t \mid x, \widehat{y}_{<t}).$$

Maximizing each factor myopically is not guaranteed to maximize the product over complete sequences because the first choice changes all later conditionals.

Part 3 - Hints, controls, and evidence

1. Assert the generated token at every step equals the raw-logit argmax.
2. Stop finished sequences immediately and never append useful-looking tokens after EOS.
3. Trace probability and entropy before and during a repetition loop.
4. Compare cached and uncached greedy generation; exact tokens and logits should match within tolerance.
5. Report deterministic outputs over a fixed prompt suite rather than one manually selected example.

Part 4 - Python code skeleton

```
import torch

@torch.inference_mode()
def greedy_decode(model, prompt_ids, eos_id, maximum_new_tokens):
    model.eval()
    generated = prompt_ids.clone()
    trace = []
    for generation_step in range(maximum_new_tokens):
        logits = model(generated)
        final_logits = logits[:, -1, :].float()
        probabilities = torch.softmax(final_logits, dim=-1)
        next_token = torch.argmax(final_logits, dim=-1, keepdim=True)
        selected_probability = probabilities.gather(1, next_token)
        trace.append({"step": generation_step,
                      "token": next_token.item(),
                      "probability": selected_probability.item()})
        generated = torch.cat((generated, next_token), dim=1)
        if next_token.item() == eos_id:
            break
    return generated, trace

def E02(model, prompt_ids, eos_id):
    first, trace_first = greedy_decode(model, prompt_ids, eos_id, 64)
    second, trace_second = greedy_decode(model, prompt_ids, eos_id, 64)
    assert torch.equal(first, second)
    assert trace_first == trace_second
    return first, trace_first
```

Part 5 - The Philosophy Behind the Exercise

Greedy decoding embodies immediate certainty: choose the best-looking next step and never reconsider. Its failures demonstrate that local rationality can produce a globally poor

path when decisions reshape the future.

Determinism makes greedy decoding scientifically valuable even when it is not creatively satisfying. It offers a stable mirror in which model and cache bugs cannot hide behind sampling variance.

17.8.4 Exercise 03

Exercise Title: Implement Temperature Sampling and Analyze Diversity vs Coherence Tradeoff Carefully

Part 1 - Why this matters

Temperature changes the sharpness of the next-token distribution before sampling. Values below one amplify logit differences and make output more conservative; values above one flatten differences and increase access to lower- probability tokens. Temperature does not improve the model or introduce new knowledge. It changes how aggressively generation explores probability mass the model already assigned.

Implement sampling by dividing logits by temperature and then applying softmax. Use an explicit `torch.Generator`. Reject nonpositive temperatures. For a very small positive value, verify behavior approaches greedy selection without dividing by zero. Cast logits to float32 for probability computation when lower-precision model execution makes the softmax fragile. Preserve the unmodified logits for diagnostics and log entropy, selected-token rank, probability, and seed.

Evaluate several temperatures under matched prompts, seeds, token budgets, and model checkpoint. Generate many samples per condition. Measure validation or reference likelihood, distinct-n, repetition, EOS rate, novelty relative to the training corpus, and task-specific correctness. Human review should be blinded to temperature. A high-temperature sample can be novel but incoherent; a low-temperature sample can be fluent but repetitive. Present a tradeoff curve rather than declaring one universal best value.

Part 2 - Mathematical form and reasoning

For logits z_v and temperature $\tau > 0$,

$$p_\tau(v) = \frac{\exp(z_v/\tau)}{\sum_u \exp(z_u/\tau)}.$$

For any two tokens a, b,

$$\frac{p_\tau(a)}{p_\tau(b)} = \exp\left(\frac{z_a - z_b}{\tau}\right).$$

Thus lower τ magnifies odds ratios and higher τ compresses them. As $\tau \to 0^+$ the distribution concentrates on the maxima; as $\tau \to \infty$ it approaches uniform over finite logits.

Part 3 - Hints, controls, and evidence

1. Verify probabilities are finite, nonnegative, and sum to one for normal and extreme logits.
2. Use the same list of random seeds for every temperature.

3. Report distributions of metrics across samples, not one sample per value.
4. Keep all other filters and penalties disabled while isolating temperature.
5. Inspect selected-token ranks; diversity can rise because generation enters the model's low-confidence tail.

Part 4 - Python code skeleton

```
import torch

def sample_with_temperature(logits, temperature, generator):
    if temperature <= 0.0:
        raise ValueError("temperature must be positive")
    scaled_logits = logits.float() / temperature
    probabilities = torch.softmax(scaled_logits, dim=-1)
    if not torch.isfinite(probabilities).all():
        raise FloatingPointError("invalid sampling probabilities")
    next_token = torch.multinomial(probabilities, 1, generator=generator)
    return next_token, probabilities

def E03(logits, temperatures=(0.2, 0.7, 1.0, 1.5)):
    report = {}
    for temperature in temperatures:
        tokens = []
        entropies = []
        for seed in range(100):
            generator = torch.Generator(device=logits.device)
            generator.manual_seed(seed)
            token, probabilities = sample_with_temperature(
                logits, temperature, generator)
            tokens.append(token.item())
            entropy = -(probabilities * probabilities.clamp_min(1e-12).log())
            entropies.append(entropy.sum().item())
        report[temperature] = {"tokens": tokens,
                               "mean_entropy": sum(entropies)/len(entropies)}
    return report
```

Part 5 - The Philosophy Behind the Exercise

Temperature is a dial between concentration and exploration. It reminds us that a probability distribution contains more possibilities than its maximum, and generation policy decides how much of that possibility space becomes real.

Creativity without coherence is noise; coherence without variation is repetition. The appropriate balance belongs to the purpose of generation and must be supported by evidence across many outputs.

17.8.5 Exercise 04

Exercise Title: Implement Top-k Sampling and Compare to Temperature Sampling for Stability and Diversity

Part 1 - Why this matters

Temperature reshapes the entire vocabulary distribution, so at high values even extremely implausible tokens can receive enough probability to be sampled. Top-k sampling first keeps only the k largest logits, sets all others to negative infinity, renormalizes, and samples from that fixed-size candidate set. It protects generation from the far tail, but the same k is used whether the model is highly certain or deeply uncertain.

Implement filtering with `torch.topk` and a Boolean keep mask. Validate $1 \leq k \leq V$, preserve ties according to the framework's returned indices, and compute softmax in float32. Apply temperature before top-k so ranking is unchanged for positive temperature while the retained probabilities are reshaped. Assert filtered tokens receive exactly zero probability and retained probabilities sum to one. Test $k = 1$ against greedy decoding and $k = V$ against unfiltered temperature sampling using the same random generator.

Compare several k values and temperatures on fixed prompts and seeds. Record candidate probability mass before renormalization, entropy after filtering, selected rank, repetition, distinct-n, novelty, EOS rate, and task quality. Small k can stabilize output but erase valid alternatives; large k can admit a noisy tail. Candidate count alone is not comparable across contexts whose probability concentrations differ.

Part 2 - Mathematical form and reasoning

Let $K_k(z)$ be the indices of the k largest logits. Define

$$\widetilde{z}_v = \begin{cases} z_v/\tau, & v \in K_k(z), \\ -\infty, & v \notin K_k(z), \end{cases} \qquad p(v) = \operatorname{softmax}(\widetilde{z})_v.$$

The retained pre-filter probability mass under temperature τ is

$$m_k = \sum_{v \in K_k} p_\tau(v).$$

Renormalization divides retained probabilities by m_k. When m_k is already near one, filtering changes little; when it is small, the policy strongly distorts the model distribution.

Part 3 - Hints, controls, and evidence

1. Test known logits where the top-k indices and probabilities can be computed manually.
2. Verify $k = 1$ always returns an argmax and $k = V$ preserves the unfiltered distribution.
3. Keep prompts, seeds, temperature, and token budget matched across k.
4. Log retained mass as well as candidate count; it explains why the same k behaves differently across steps.
5. Include flat and sharply peaked synthetic distributions as boundary cases.

Part 4 - Python code skeleton

```
import torch

def top_k_distribution(logits, k, temperature=1.0):
    vocabulary_size = logits.size(-1)
    if k < 1 or k > vocabulary_size:
        raise ValueError("k must be inside the vocabulary range")
    if temperature <= 0.0:
        raise ValueError("temperature must be positive")
    scaled = logits.float() / temperature
    top_values, top_indices = torch.topk(scaled, k, dim=-1)
    filtered = torch.full_like(scaled, -torch.inf)
    filtered.scatter_(-1, top_indices, top_values)
    probabilities = torch.softmax(filtered, dim=-1)
    return probabilities

def E04(logits, generator):
    report = {}
    vocabulary_size = logits.size(-1)
    for k in (1, 5, 20, vocabulary_size):
        probabilities = top_k_distribution(logits, k, temperature=0.8)
        token = torch.multinomial(probabilities, 1, generator=generator)
        positive_count = probabilities.gt(0).sum(dim=-1)
        assert torch.all(positive_count <= k)
        report[k] = {"token": token, "probabilities": probabilities}
    return report
```

Part 5 - The Philosophy Behind the Exercise

Top-k makes possibility finite at every step. It exchanges fidelity to the full distribution for protection against its least plausible tail.

The fixed candidate count is simple, but simplicity is a policy choice. A responsible decoder measures how much probability and meaning that fixed gate removes in different contexts.

17.8.6 Exercise 05

Exercise Title: Implement Top-p (Nucleus) Sampling and Compare Its Behavior Across Multiple p Values

Part 1 - Why this matters

Top-p sampling adapts candidate count to the model's uncertainty. It sorts tokens by probability and keeps the smallest prefix whose cumulative mass is at least p. A confident distribution may retain one or two tokens; a flat distribution may retain many. This makes the policy responsive, but sorting the vocabulary costs more than a fixed top-k filter and boundary implementation details matter.

Implement temperature softmax, descending sort, cumulative sum, and a shifted removal mask that always keeps the token crossing the threshold. Scatter the filtered sorted probabilities back into vocabulary order or sample directly in sorted space and map the selected index back. Validate $0 < p \leq 1$, keep at least one token, and prove retained mass before renormalization is at least p while removing any last retained token would usually fall below it. Test peaked, flat, and tied distributions.

Compare multiple p values using matched prompts, seeds, temperature, and generation length. At every step log nucleus size, retained mass, entropy, selected rank, and latency. Evaluate repetition, distinct-n, novelty, EOS behavior, and task correctness. A high p does not guarantee useful diversity if the model distribution itself is collapsed; a low p may behave like greedy decoding in confident contexts.

Part 2 - Mathematical form and reasoning

Let probabilities be ordered $p_{(1)} \geq p_{(2)} \geq \cdots \geq p_{(V)}$. Define

$$k^* = \min \left\{ k : \sum_{i=1}^{k} p_{(i)} \geq p_0 \right\},$$

where p_0 is the nucleus threshold. The retained distribution is

$$q_{(i)} = \begin{cases} p_{(i)} / \sum_{j=1}^{k^*} p_{(j)}, & i \leq k^*, \\ 0, & i > k^*. \end{cases}$$

Unlike top-k, the candidate count k^* is a function of context and changes at every generation step.

Part 3 - Hints, controls, and evidence

1. Use a four-token manual distribution to verify the crossing token remains inside the nucleus.
2. Test p close to zero, common values such as 0.9 and 0.95, and $p = 1$.
3. Log the distribution of nucleus sizes, not only its mean.
4. Compare top-p and top-k at similar retained mass as well as similar quality.
5. Keep temperature fixed during the isolated p sweep because temperature changes both ordering margins and cumulative mass.

Part 4 - Python code skeleton

```
import torch

def top_p_distribution(logits, nucleus_probability, temperature=1.0):
    if nucleus_probability <= 0.0 or nucleus_probability > 1.0:
        raise ValueError("top-p must be in (0, 1]")
    probabilities = torch.softmax(logits.float() / temperature, dim=-1)
    sorted_probabilities, sorted_indices = torch.sort(
```

```
        probabilities, dim=-1, descending=True)
    cumulative = torch.cumsum(sorted_probabilities, dim=-1)
    remove = cumulative > nucleus_probability
    shifted_remove = remove.clone()
    shifted_remove[..., 1:] = remove[..., :-1]
    shifted_remove[..., 0] = False
    sorted_probabilities = sorted_probabilities.masked_fill(
        shifted_remove, 0.0)
    sorted_probabilities = sorted_probabilities / sorted_probabilities.sum(
        dim=-1, keepdim=True)
    restored = torch.zeros_like(probabilities)
    restored.scatter_(-1, sorted_indices, sorted_probabilities)
    return restored

def E05(logits):
    report = {}
    for nucleus_probability in (0.5, 0.8, 0.9, 0.95, 1.0):
        distribution = top_p_distribution(logits, nucleus_probability)
        assert torch.allclose(distribution.sum(-1),
                              torch.ones_like(distribution.sum(-1)))
        report[nucleus_probability] = distribution
    return report
```

Part 5 - The Philosophy Behind the Exercise

Top-p defines possibility by probability mass rather than a fixed number of names. It expands when the model is uncertain and contracts when the model is confident.

This adaptability is valuable because uncertainty is contextual. Yet the nucleus inherits every error in the model's confidence, so adaptive selection still requires evaluation rather than trust.

17.8.7 Exercise 06

Exercise Title: Implement Repetition Penalty or Frequency Penalty Conceptually and Measure Reduction in Loops

Part 1 - Why this matters

Autoregressive feedback can create loops: once a token or phrase is generated, the resulting context may make it even more likely to appear again. Penalties modify logits using generation history before filtering and sampling. A presence penalty reacts to whether a token appeared; a frequency penalty grows with count; a multiplicative repetition penalty changes positive and negative logits differently. These are distinct algorithms and should not be mixed under one vague name.

Implement one penalty precisely and keep special tokens under an explicit policy. For additive frequency penalty λ, subtract λc_v from logit z_v, where c_v counts generated occurrences after the prompt or in the whole context according to configuration. Test a known history and logits manually. Apply penalties before temperature and top-k or top-p. Preserve original

logits so the magnitude and target of every change can be inspected.

Compare penalty strengths under matched prompts, seeds, sampling policy, and token budget. Measure repeated unigram and n-gram rates, longest loop, distinct-n, task correctness, EOS behavior, and semantic necessity. Repetition is not always wrong: names, syntax, code variables, refrains, or copied keys may need recurrence. Strong penalties can force synonyms, grammatical errors, or premature topic drift. Add prompts where repetition is required to expose this cost.

Part 2 - Mathematical form and reasoning

For generated history h and token count $c_v(h) = \sum_t \mathbb{1}[h_t = v]$, additive frequency penalty is

$$\tilde{z}_v = z_v - \lambda c_v(h), \qquad \lambda \geq 0.$$

A presence penalty uses

$$\tilde{z}_v = z_v - \alpha \mathbb{1}[c_v(h) > 0].$$

After modification, a later softmax renormalizes every probability, so reducing one repeated token redistributes mass across all alternatives. Penalties are decoding priors, not changes to the trained model.

Part 3 - Hints, controls, and evidence

1. Decide whether prompt tokens count toward the penalty and record the choice.
2. Exclude PAD and handle BOS/EOS explicitly; penalizing EOS may prevent termination.
3. Use a frequency vector updated once after each selected token and verify it cannot drift from the generated sequence.
4. Compare no penalty and several strengths over many fixed seeds.
5. Include both unwanted-loop prompts and repetition-required prompts in the evaluation suite.

Part 4 - Python code skeleton

```
import torch

def apply_frequency_penalty(logits, token_counts, penalty):
    if penalty < 0.0:
        raise ValueError("penalty must be nonnegative")
    if logits.shape != token_counts.shape:
        raise ValueError("counts must align with vocabulary logits")
    adjusted = logits.float() - penalty * token_counts.float()
    return adjusted

def update_counts(token_counts, selected_token):
```

```
    increment = torch.ones_like(selected_token, dtype=token_counts.dtype)
    token_counts.scatter_add_(-1, selected_token, increment)

def E06(logits, generated_ids, vocabulary_size):
    batch_size = logits.size(0)
    counts = torch.zeros(batch_size, vocabulary_size, device=logits.device)
    for position in range(generated_ids.size(1)):
        token = generated_ids[:, position:position + 1]
        update_counts(counts, token)
    report = {}
    for penalty in (0.0, 0.1, 0.5, 1.0):
        report[penalty] = apply_frequency_penalty(logits, counts, penalty)
    return report
```

Part 5 - The Philosophy Behind the Exercise

A repetition penalty inserts memory into the decoder's policy: previous choices become reasons to avoid the same choice again. It can break pathological loops, but it also imposes a human preference for variation.

Not all recurrence is failure. The ethical and engineering task is to penalize mechanical collapse without erasing repetition that carries identity, syntax, precision, or meaning.

17.8.8 Exercise 07

Exercise Title: Implement KV Cache Conceptually for Faster Autoregressive Decoding and Benchmark Speed Improvement

Part 1 - Why this matters

Without a KV cache, generation recomputes keys and values for every previous token at every step. The prefix does not change, so most of this work is redundant. A cache stores projected keys and values for each layer. On the next step, the model projects only the new token's query, key, and value, appends the new key/value to the cache, and attends the new query over all cached positions. This reduces repeated computation but introduces state, memory growth, and position bookkeeping.

Implement cache support at the attention layer with shape `(B,H,T_cached,d_h)`. During prompt prefill, process the whole prompt and create caches. During decode, accept one new token, concatenate its key and value along the sequence axis, and return updated caches. Position IDs for the new token must start at cached length, not zero. The new token's query has no future positions, so an explicit triangular mask is often unnecessary for a single-token decode, but padding and batch semantics still matter.

Prove parity before measuring speed. For every step, compare cached next-token logits with logits obtained by recomputing the full prefix; compare every layer's cache to keys/values computed from that prefix. Then benchmark prefill time, first-token latency, per-token decode latency, total generation time, and cache bytes across prompt and generation lengths. The cache improves compute while growing linearly with total stored context and may become the dominant memory cost.

Part 2 - Mathematical form and reasoning

At layer l and step t, cached tensors are

$$K_{1:t}^{(l)} = [K_{1:t-1}^{(l)}; K_t^{(l)}], \qquad V_{1:t}^{(l)} = [V_{1:t-1}^{(l)}; V_t^{(l)}].$$

The new output is

$$O_t = \text{softmax}\left(Q_t (K_{1:t})^\top / \sqrt{d_h}\right) V_{1:t}.$$

For L layers, batch B, heads H, stored length T, head width d_h, and element size s, cache memory is approximately

$$M_{KV} = 2LBHTd_h s = 2LBTd_{\text{model}} s.$$

Part 3 - Hints, controls, and evidence

1. Separate prompt prefill from one-token decode in the API and benchmarks.
2. Assert cache length increases by exactly one per decoding step and never changes during read-only use.
3. Compare cached and full-prefix logits at every step for several layers, batch sizes, and prompt lengths.
4. Use CUDA events and warmup; synchronize at measurement boundaries.
5. Report speedup together with cache memory and the context length where memory becomes the practical limit.

Part 4 - Python code skeleton

```
import torch

def append_kv(previous_key, previous_value, new_key, new_value):
    if previous_key is None:
        return new_key, new_value
    key = torch.cat((previous_key, new_key), dim=2)
    value = torch.cat((previous_value, new_value), dim=2)
    return key, value

@torch.inference_mode()
def cached_generate(model, prompt_ids, new_tokens):
    model.eval()
    logits, cache = model(prompt_ids, cache=None, use_cache=True)
    generated = prompt_ids.clone()
    for generation_step in range(new_tokens):
        next_token = torch.argmax(logits[:, -1, :], dim=-1, keepdim=True)
        generated = torch.cat((generated, next_token), dim=1)
        position = prompt_ids.size(1) + generation_step
```

```
        logits, cache = model(next_token, cache=cache, use_cache=True,
                              position_offset=position)
        expected_length = position + 1
        for layer_cache in cache:
            assert layer_cache[0].size(2) == expected_length
    return generated

def E07(model, prompt_ids, compare_full_prefix):
    cached = cached_generate(model, prompt_ids, 32)
    full = compare_full_prefix(model, prompt_ids, 32)
    assert torch.equal(cached, full)
    return cached
```

Part 5 - The Philosophy Behind the Exercise

The KV cache is computational memory: it preserves work whose meaning remains valid so the system does not repeatedly rediscover its own past.

Efficiency comes from recognizing invariance. Yet memory creates new duties. State must remain aligned with tokens, positions, layers, and batches. Saved work is valuable only while its identity is trustworthy.

17.8.9 Exercise 08

Exercise Title: Debug Generation Bugs Caused by Wrong Masking, Wrong Cache Concatenation, or Wrong Position Updates

Part 1 - Why this matters

Generation bugs often preserve legal shapes and finite outputs. A transposed causal mask may reveal the future during training. A cache concatenated along the head or feature axis may fail only after several steps. Position IDs reset to zero can make cached generation disagree with full-prefix recomputation while still producing fluent-looking text. Because sampling adds randomness, these defects can be mistaken for ordinary variation.

Build a deterministic debugging mode using evaluation mode, greedy decoding, float32, fixed prompts, and no penalties. Treat full-prefix recomputation as the reference. At every step save token IDs, position IDs, attention masks, logits, selected token, and per-layer key/value cache shapes. Compare next-token logits before selection. Locate the first step and layer whose values diverge rather than inspecting only the final string.

Inject three faults separately. Reverse or omit the causal rule and run the shared-prefix intervention. Concatenate cache along the wrong axis and assert that sequence length fails to increase by one. Reset positions during cached decoding and compare cached versus uncached logits. Add tests for prompt padding, variable lengths, context truncation, and finished rows. Each diagnosis must name the earliest broken contract and convert the repair into a permanent test.

Part 2 - Mathematical form and reasoning

For reference logits z_t^{full} and cached logits z_t^{cache}, define parity error

$$e_t = \frac{\|z_t^{\text{full}} - z_t^{\text{cache}}\|_\infty}{\|z_t^{\text{full}}\|_\infty + \epsilon}.$$

Correct cache growth requires

$$\text{shape}(K_t) = (B, H, T_0 + t, d_h),$$

and the new token uses absolute position T_0+t-1. Causal invariance requires that changing suffix $x_{>r}$ leaves logits $z_{\le r}$ unchanged. These three properties isolate position, cache, and mask faults.

Part 3 - Hints, controls, and evidence

1. Remove sampling variance before debugging numerical parity.
2. Use asymmetric dimensions so concatenating along the wrong axis cannot hide behind equal sizes.
3. Compare layer outputs and caches from earliest to latest; downstream differences are consequences.
4. Log whether masks mean “allowed” or “blocked” at every API boundary.
5. Preserve the failing prompt, checkpoint hash, tokenizer fingerprint, precision, and backend with the incident.

Part 4 - Python code skeleton

```
import torch

@torch.inference_mode()
def cache_parity_trace(model, prompt_ids, steps):
    model.eval()
    full_prefix = prompt_ids.clone()
    cached_logits, cache = model(prompt_ids, use_cache=True)
    trace = []
    for step in range(steps):
        full_logits = model(full_prefix, use_cache=False)
        reference = full_logits[:, -1, :].float()
        candidate = cached_logits[:, -1, :].float()
        error = (reference - candidate).abs().max().item()
        trace.append({"step": step, "maximum_logit_error": error})
        if not torch.allclose(reference, candidate, atol=1e-5, rtol=1e-4):
            raise AssertionError(f"cache divergence at step {step}")
        next_token = reference.argmax(-1, keepdim=True)
        full_prefix = torch.cat((full_prefix, next_token), dim=1)
        position = prompt_ids.size(1) + step
        cached_logits, cache = model(next_token, cache=cache, use_cache=True,
                                     position_offset=position)
    return trace
```

```
def E08(model, prompt_ids):
    return cache_parity_trace(model, prompt_ids, 16)
```

Part 5 - The Philosophy Behind the Exercise

Generation turns a pure function into a stateful process. Once past computation is cached, correctness depends on history remaining aligned across several representations of the same sequence.

Debugging restores one source of truth. Full recomputation, explicit positions, and stepwise comparison transform a vague bad output into the first broken agreement.

17.8.10 Exercise 09

Exercise Title: Evaluate Generation Quality with Perplexity and Simple Metrics Like Repetition Rate and Novelty Rate

Part 1 - Why this matters

Generation quality has no single sufficient metric. Perplexity evaluates teacher-forced probability on held-out text, not the coherence of free-running samples. Repetition measures loops but can punish legitimate reuse. Novelty measures difference from a reference corpus but can reward meaningless strings. The purpose is to build a metric panel whose disagreements reveal behavior, then connect those signals to blind manual review or task-specific correctness.

Compute validation NLL and perplexity only over valid targets with the same tokenizer. For generations, report distinct-n, repeated-n-gram fraction, longest repeated span, EOS completion, length, and novelty relative to training n-grams. Normalize carefully across different lengths. Define whether prompt tokens are excluded, whether special tokens count, and how whitespace or subword boundaries are decoded. Preserve per-sample values rather than only a mean.

Evaluate greedy, temperature, top-k, top-p, and penalty settings using the same prompts, seeds, and maximum budget. Include reference text and a random or unigram baseline to calibrate metrics. A model can have better perplexity yet more repetitive greedy output; a high novelty rate can coincide with lower human quality. Create a table of metric correlations and inspect counterexamples where metrics strongly disagree.

Part 2 - Mathematical form and reasoning

Perplexity is

$$\mathrm{PPL} = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log p_\theta(y_i \mid x_{\le i})\right).$$

For generated n-gram multiset G_n and unique set U_n,

$$\mathrm{Distinct}_n = \frac{|U_n|}{|G_n|}, \qquad \mathrm{Repetition}_n = 1 - \mathrm{Distinct}_n\,.$$

If T_n is the set of training n-grams, novelty can be

$$\text{Novelty}_n = \frac{|\{g \in U_n : g \notin T_n\}|}{|U_n|}.$$

These definitions must be fixed before comparisons.

Part 3 - Hints, controls, and evidence

1. Calculate small examples manually to validate off-by-one n-gram windows and duplicate counting.
2. Report metrics by prompt and generation length bin as well as overall.
3. Keep prompt text out of generated-continuation novelty unless the research question explicitly includes it.
4. Bootstrap confidence intervals or repeat seeds for sampling-based metrics.
5. Review high-novelty/low-quality and low-novelty/high-quality samples to expose metric limitations.

Part 4 - Python code skeleton

```
def make_ngrams(tokens, n):
    ngrams = []
    final_start = len(tokens) - n
    for start in range(final_start + 1):
        ngram = tuple(tokens[start:start + n])
        ngrams.append(ngram)
    return ngrams

def generation_metrics(tokens, training_ngrams, n=3):
    generated_ngrams = make_ngrams(tokens, n)
    if len(generated_ngrams) == 0:
        return {"distinct": 0.0, "repetition": 0.0, "novelty": 0.0}
    unique_ngrams = set(generated_ngrams)
    novel_count = 0
    for ngram in unique_ngrams:
        if ngram not in training_ngrams:
            novel_count += 1
    distinct = len(unique_ngrams) / len(generated_ngrams)
    novelty = novel_count / len(unique_ngrams)
    return {"distinct": distinct, "repetition": 1.0 - distinct,
            "novelty": novelty}

def E09(generation_records, training_ngrams):
    report = []
    for record in generation_records:
        metrics = generation_metrics(record["tokens"], training_ngrams, n=3)
        report.append({"id": record["id"], "metrics": metrics})
    return report
```

Part 5 - The Philosophy Behind the Exercise

Metrics are partial languages for describing behavior. Each makes some property visible by ignoring others. Perplexity sees probability, repetition sees reuse, and novelty sees distance from memory.

Evaluation becomes mature when disagreement is treated as information. The goal is not to find one number that ends judgment, but a set of measurements that makes judgment more disciplined.

17.8.11 Exercise 10

Exercise Title: Run Decoder-Only Training and Generation on CUDA and Benchmark End-to-End Throughput Including Sampling

Part 1 - Why this matters

Training and generation stress CUDA differently. Training processes many tokens in parallel and is usually dominated by large matrix operations. Autoregressive generation has a parallel prompt-prefill phase followed by many small sequential decode steps. Sampling, cache updates, synchronization, and Python control flow can dominate even when the model runs on the GPU. One throughput number cannot describe both workloads.

Build separate end-to-end benchmarks for training, prefill, and decode. Training reports valid tokens per second including data transfer, forward, backward, and optimizer update. Generation reports prompt tokens per second, time to first generated token, decode tokens per second, total request latency, and cache memory. Include the actual sampling policy rather than benchmarking logits only. Keep logits, token counts, masks, model parameters, caches, and random generator on compatible devices. Avoid calling `item()` inside each decode step unless EOS handling requires a synchronized decision.

Use pinned batches and nonblocking transfer for training, warm up kernels, and measure with CUDA events or explicit synchronization. Benchmark greedy and sampling policies, cached and uncached generation, multiple prompt lengths, output lengths, and batch sizes. Verify output parity before speed comparisons. On two RTX 3070 GPUs, keep the single-device baseline clear before introducing distributed training; generation latency does not automatically improve by splitting a small model across devices.

Part 2 - Mathematical form and reasoning

Training throughput is

$$R_{\text{train}} = \frac{N_{\text{valid}}}{t_{\text{transfer}} + t_{\text{fwd}} + t_{\text{bwd}} + t_{\text{update}} - t_{\text{overlap}}}.$$

Generation separates prefill and decode:

$$t_{\text{request}} = t_{\text{prefill}} + \sum_{u=1}^{U} (t_{\text{model,u}} + t_{\text{sample,u}} + t_{\text{state,u}}).$$

Decode rate is $U/(t_{\text{request}} - t_{\text{prefill}})$. Report both batch-level and per-sequence rates so batching does not conceal user-visible latency.

Part 3 - Hints, controls, and evidence

1. Warm up each shape and policy because kernel selection and compilation can change first-run timing.
2. Use fixed generated length for throughput tests and a separate realistic EOS-terminated test.
3. Record peak allocated and reserved memory and bytes occupied by the cache.
4. Compare GPU time with wall time; their gap exposes CPU and synchronization overhead.
5. Save hardware, driver, PyTorch, precision, shapes, and sampling settings with every table.

Part 4 - Python code skeleton

```
import time
import torch

def timed_cuda_region(function):
    torch.cuda.synchronize()
    wall_start = time.perf_counter()
    event_start = torch.cuda.Event(enable_timing=True)
    event_end = torch.cuda.Event(enable_timing=True)
    event_start.record()
    result = function()
    event_end.record()
    event_end.synchronize()
    wall_seconds = time.perf_counter() - wall_start
    gpu_seconds = event_start.elapsed_time(event_end) / 1000.0
    return result, wall_seconds, gpu_seconds

def E10(training_step, generate, valid_tokens, generated_tokens):
    for warmup_step in range(5):
        training_step()
    training_result, train_wall, train_gpu = timed_cuda_region(training_step)
    generation_result, generation_wall, generation_gpu = timed_cuda_region(
        generate)
    training_rate = valid_tokens / train_wall
    generation_rate = generated_tokens / generation_wall
    return {"training_valid_tokens_per_second": training_rate,
            "generation_tokens_per_second": generation_rate,
            "training_gpu_seconds": train_gpu,
            "generation_gpu_seconds": generation_gpu,
            "generation": generation_result,
            "training": training_result}
```

Part 5 - The Philosophy Behind the Exercise

The same model participates in two different temporal economies. Training gains power from parallel evidence; generation must wait for each choice before the next can exist.

Performance measurement should respect the actual experience being optimized. Throughput, first-token latency, and per-token delay answer different human and systems questions.

17.8.12 Exercise 11

Exercise Title: Compare float32 vs float16 Generation Stability and Detect Softmax Instabilities During Long Generation

Part 1 - Why this matters

Long generation repeatedly feeds model outputs back as inputs. Small numerical differences between float32 and float16 can change a close sampling decision; after that token differs, later contexts are no longer comparable. This normal path divergence must be distinguished from numerical instability such as NaN, Inf, collapsed probabilities, cache corruption, or systematically changed termination. Sampling softmax and normalization deserve explicit float32 handling even when model matrix operations use float16.

Start with deterministic parity. Use greedy decoding, identical weights, prompts, cache policy, and evaluation mode. At each step compare float32 and float16 logits, top-token agreement, probability sums, entropy, and cache finiteness before the generated paths diverge. Then evaluate sampling statistically across matched seeds; exact token equality is not expected after small distribution changes. Record failure step, score range, selected-token margin, and AMP/autocast settings.

Inject extreme activation and logit scales in an isolated test. Compare naive float16 exponentiation, stable softmax, float32 softmax on float16 logits, and bfloat16 where supported. Test masks containing fully blocked rows and large negative values. Measure latency, memory, non-finite counts, EOS rate, repetition, and quality across long outputs. A run that stays finite can still be unstable if probability mass collapses or output statistics change sharply.

Part 2 - Mathematical form and reasoning

Stable softmax uses

$$p_i = \frac{\exp(z_i - m)}{\sum_j \exp(z_j - m)}, \qquad m = \max_j z_j.$$

Logit error at step t can be measured by

$$e_t = \frac{\|z_t^{32} - z_t^{16}\|_\infty}{\|z_t^{32}\|_\infty + \epsilon}.$$

The greedy choice is sensitive when the margin $\Delta_t = z_{(1)} - z_{(2)}$ is comparable to numerical error. Once different tokens are selected, later logit comparison measures different contexts and must be labeled accordingly.

Part 3 - Hints, controls, and evidence

1. Compare logits only while token prefixes remain identical; after divergence compare distributional outcomes instead.
2. Compute probability normalization and sampling in float32 as a controlled condition.
3. Monitor every layer's cache for finite values and expected dtype/device.
4. Include long runs that reach the intended maximum generation length.

5. Report speed and memory gains beside parity errors, non-finite counts, and behavioral changes.

Part 4 - Python code skeleton

```
import torch

def stable_sampling_probabilities(logits, temperature):
    scaled = logits.float() / temperature
    maximum = scaled.max(dim=-1, keepdim=True).values
    exponentials = torch.exp(scaled - maximum)
    probabilities = exponentials / exponentials.sum(dim=-1, keepdim=True)
    if not torch.isfinite(probabilities).all():
        raise FloatingPointError("non-finite probability during generation")
    return probabilities

@torch.inference_mode()
def E11(model_float32, model_float16, prompt, steps=128):
    prefix32 = prompt.clone()
    prefix16 = prompt.clone()
    report = []
    for step in range(steps):
        logits32 = model_float32(prefix32)[:, -1, :].float()
        logits16 = model_float16(prefix16)[:, -1, :].float()
        same_prefix = torch.equal(prefix32, prefix16)
        error = None
        if same_prefix:
            error = (logits32 - logits16).abs().max().item()
        token32 = logits32.argmax(-1, keepdim=True)
        token16 = logits16.argmax(-1, keepdim=True)
        report.append({"step": step, "same_prefix": same_prefix,
                       "maximum_logit_error": error,
                       "same_token": torch.equal(token32, token16)})
        prefix32 = torch.cat((prefix32, token32), dim=1)
        prefix16 = torch.cat((prefix16, token16), dim=1)
    return report
```

Part 5 - The Philosophy Behind the Exercise

Precision determines which distinctions a computation can preserve. In a feedback process, one erased distinction can redirect the entire future without implying that either path is numerically invalid.

Stability is therefore more than absence of NaN. It is an evidence-backed claim that compressed arithmetic preserves the distributions and decisions important to the intended generation behavior.

17.8.13 Exercise 12

Exercise Title: Implement Beam Search Conceptually and Compare Its Output to Sampling-Based Methods on a Small Example

Part 1 - Why this matters

Beam search keeps several high-scoring partial sequences instead of committing to one greedy path. At each step it expands every active beam with candidate tokens, adds log probabilities, and retains the best fixed number of combined hypotheses. It approximates sequence-level search and can recover a continuation whose first token is not locally best. Unlike sampling, standard beam search is deterministic and prioritizes model likelihood rather than diversity.

Implement the algorithm first on a tiny hand-defined probability tree. Store token sequence, cumulative log probability, finished flag, and optional cache for every beam. Finished beams must not be expanded except through a formal identity transition used for tensorized bookkeeping. Apply length normalization or length penalty explicitly, because raw log probability decreases with each additional token and favors short EOS hypotheses. Keep search score separate from raw model log probability in the report.

Then compare greedy, beam widths, and sampling methods on fixed prompts. Measure raw likelihood, normalized score, length, EOS completion, repetition, diversity, task correctness, and latency. Beam search can produce high-likelihood but bland or repetitive output, and larger beams can worsen length bias. When using a KV cache, reorder every layer's cache according to selected parent-beam indices; failure here can combine tokens from one hypothesis with state from another.

Part 2 - Mathematical form and reasoning

For beam hypothesis $h = (y_1, \ldots, y_t)$, cumulative score is

$$s(h) = \sum_{i=1}^{t} \log p_\theta(y_i \mid x, y_{<i}).$$

At step t, expand beam set $\mathcal{B}_{t-1}$ and keep

$$\mathcal{B}_t = \operatorname{TopB}\left\{(h \oplus v,\ s(h) + \log p(v \mid x, h)) : h \in \mathcal{B}_{t-1}, v \in \mathcal{V}\right\}.$$

A simple normalized score is $s(h)/|h|^\alpha$. The exponent α changes the search objective and must be tuned and reported.

Part 3 - Hints, controls, and evidence

1. Verify beam width one matches greedy decoding exactly when scoring and EOS rules are aligned.
2. Use a probability tree where greedy is provably not the best complete sequence.
3. Track parent beam index, emitted token, raw score, normalized score, and finished state at every step.
4. Test all beams finishing early, no beam reaching EOS, score ties, and maximum-length termination.

5. Compare quality under matched request budgets and report beam-search latency and cache-memory growth.

Part 4 - Python code skeleton

```
import torch

def beam_search(model, prompt, eos_id, beam_width, maximum_new_tokens,
                length_power=0.7):
    beams = [{"tokens": prompt.clone(), "log_probability": 0.0,
              "finished": False}]
    for step in range(maximum_new_tokens):
        candidates = []
        for beam in beams:
            if beam["finished"]:
                candidates.append(beam)
                continue
            logits = model(beam["tokens"])[:, -1, :].float()
            log_probabilities = torch.log_softmax(logits, dim=-1)
            values, indices = torch.topk(
                log_probabilities, beam_width, dim=-1
            )
            for candidate_index in range(beam_width):
                token = indices[:, candidate_index:candidate_index + 1]
                tokens = torch.cat((beam["tokens"], token), dim=1)
                token_score = values[0, candidate_index].item()
                score = beam["log_probability"] + token_score
                candidates.append({"tokens": tokens, "log_probability": score,
                                   "finished": token.item() == eos_id})
        def normalized(candidate):
            generated_length = candidate["tokens"].size(1) - prompt.size(1)
            denominator = max(generated_length, 1) ** length_power
            return candidate["log_probability"] / denominator
        candidates.sort(key=normalized, reverse=True)
        beams = candidates[:beam_width]
        if all(beam["finished"] for beam in beams):
            break
    return beams

def E12(model, prompt, eos_id):
    return beam_search(model, prompt, eos_id, beam_width=4,
                       maximum_new_tokens=32)
```

Part 5 - The Philosophy Behind the Exercise

Beam search postpones commitment. It preserves several futures long enough to compare their accumulated consequences rather than trusting the first local preference.

But search can only optimize the probabilities and score it is given. A wider search is not wiser when the model distribution or length objective misstates what humans value.

17.8.14 Exercise 13

Exercise Title: Implement Batched Generation and Manage Padding and Stopping Conditions Correctly for Variable Prompt Lengths

Part 1 - Why this matters

Batched generation improves throughput by processing several requests together, but variable prompt lengths create alignment problems. If prompts are right- padded and logits are blindly taken from the final tensor column, shorter rows use a PAD position rather than their last real token. Left padding can simplify last-column selection, but absolute position IDs must count valid tokens rather than padding columns. KV caches also contain padded regions whose attention must remain blocked.

Implement an explicit batch state containing token buffer, attention mask, position IDs, prompt lengths, finished mask, generated counts, and per-row stopping reason. For right padding, gather prefill logits at `prompt_length - 1`. During decode, append one column for every active batch row; finished rows receive a harmless filler token, their cache shape stays compatible, and their output is ignored. Sampling should not advance their RNG or counts. Stop when every row is finished or its individual maximum is reached.

Prove batched parity by comparing each row against separate single-sequence generation with matched seeds. Test prompts of length one, equal lengths, highly different lengths, a prompt already ending in EOS, and rows finishing at different steps. Verify no row attends to another row, padding does not alter valid logits, and decoding removes filler tokens from returned sequences.

Part 2 - Mathematical form and reasoning

For row b with prompt length L_b, right-padded prefill chooses

$$z_b = z_{b,L_b-1,:}.$$

Its position IDs can be constructed from validity mask P as

$$r_{b,t} = \max\left(\sum_{j=0}^{t} P_{b,j} - 1, 0\right).$$

Finished state evolves monotonically:

$$F_b^{(u+1)} = F_b^{(u)} \vee [y_b^{(u)} = \text{EOS}] \vee [u + 1 \geq U_b].$$

Once true, a row must never become active again.

Part 3 - Hints, controls, and evidence

1. Choose left or right padding and document the resulting logit-selection and position rules.
2. Maintain a Boolean active mask on the GPU and assert finished state is monotonic.
3. Use one generator per sequence when exact parity with independent sampling runs is required.
4. Check returned lengths, stopping reasons, and absence of filler tokens.

5. Benchmark uniform-length batches and mixed-length batches separately; padding waste may erase batching gains.

Part 4 - Python code skeleton

```
import torch

def final_prompt_logits(logits, prompt_lengths):
    batch_size = logits.size(0)
    row_indices = torch.arange(batch_size, device=logits.device)
    column_indices = prompt_lengths - 1
    return logits[row_indices, column_indices, :]

@torch.inference_mode()
def batched_greedy(model, input_ids, attention_mask, eos_id,
                   maximum_new_tokens):
    model.eval()
    prompt_lengths = attention_mask.sum(dim=1)
    logits, cache = model.prefill(input_ids, attention_mask)
    next_logits = final_prompt_logits(logits, prompt_lengths)
    generated_columns = []
    finished = torch.zeros(input_ids.size(0), dtype=torch.bool,
                           device=input_ids.device)
    for step in range(maximum_new_tokens):
        selected = next_logits.argmax(dim=-1)
        selected = torch.where(finished, torch.full_like(selected, eos_id),
                               selected)
        generated_columns.append(selected)
        finished = finished | selected.eq(eos_id)
        if bool(finished.all()):
            break
        active_rows = ~finished
        next_logits, cache = model.decode(
            selected[:, None], cache, active_rows=active_rows
        )
        next_logits = next_logits[:, -1, :]
    return torch.stack(generated_columns, dim=1), finished

def E13(model, batch, eos_id):
    return batched_greedy(model, batch["input_ids"],
                          batch["attention_mask"], eos_id, 64)
```

Part 5 - The Philosophy Behind the Exercise

Batching asks unlike sequences to share one computational clock. Efficiency comes from synchrony, while correctness requires preserving each sequence's independent history, position, and right to stop.

Uniform tensors should not erase individual temporal structure. Good batching coordinates computation without confusing one request's padding, randomness, or completion with another's meaning.

17.8.15 Exercise 14

Exercise Title: Implement Stop Tokens and EOS Handling and Prove Generation Terminates Correctly Without Cutting Useful Content

Part 1 - Why this matters

Generation needs a formal termination contract. EOS is a model token representing completion; stop sequences are application-level token patterns; maximum length is a safety limit rather than semantic completion. String matching after decoding can fail because one visible phrase may have several tokenizations or because partial stop patterns span decoding steps. Stopping too late exposes unwanted text, while stopping too early cuts a valid answer.

Implement token-level EOS and multiple stop sequences. After each selected token, update an incremental matcher or compare only suffixes up to the longest stop pattern. Decide whether the returned output includes or excludes matched stop tokens. Define precedence when EOS and a stop sequence complete on the same step. Add a minimum-new-token rule only if the application needs it; during the minimum, mask EOS and stop-completing candidates carefully and report that the decoder distribution was altered.

Test stop sequences of different lengths, overlapping prefixes, patterns split across steps, EOS in the prompt, EOS sampled immediately, no stop before maximum, and batched rows ending differently. Return a stopping reason and matched pattern for every sequence. Prove no token is generated after terminal state and no useful prefix is removed beyond the exact configured stop span. Measure premature EOS and maximum-length rates on a prompt suite to distinguish model behavior from termination code.

Part 2 - Mathematical form and reasoning

Let generated suffix through step t be $g_{1:t}$ and stop-pattern set $\mathcal{S}$. Termination event is

$$T_t = [g_t = \mathrm{EOS}] \vee \bigvee_{s \in \mathcal{S}} [g_{t-|s|+1:t} = s] \vee [t = U_{\max}].$$

The terminal step is

$$t^* = \min\{t : T_t = 1\}.$$

The returned sequence is a deterministic function of $g_{1:t^*}$ and the declared inclusion policy. No decoder step may occur for that row after t^*.

Part 3 - Hints, controls, and evidence

1. Match token IDs, not decoded substrings, and store stop sequences using the same tokenizer/version as generation.

2. Sort or prioritize overlapping stop patterns explicitly.

3. Keep semantic completion, application stop, and safety maximum as distinct stopping reasons.

4. Unit-test returned output both when matched tokens are included and when they are removed.

5. Count model forward calls to prove termination prevents further work.

Part 4 - Python code skeleton

```
def matched_stop(generated_tokens, stop_sequences):
    for stop_index in range(len(stop_sequences)):
        stop_sequence = stop_sequences[stop_index]
        stop_length = len(stop_sequence)
        if len(generated_tokens) < stop_length:
            continue
        suffix = generated_tokens[-stop_length:]
        if suffix == stop_sequence:
            return stop_index, stop_length
    return None, 0

def apply_stopping(generated_tokens, selected_token, eos_id,
                   stop_sequences, include_stop=False):
    generated_tokens.append(int(selected_token))
    if selected_token == eos_id:
        return True, "eos", generated_tokens
    stop_index, stop_length = matched_stop(generated_tokens, stop_sequences)
    if stop_index is not None:
        if not include_stop:
            del generated_tokens[-stop_length:]
        return True, f"stop_sequence_{stop_index}", generated_tokens
    return False, None, generated_tokens

def E14(scripted_tokens, eos_id, stop_sequences):
    generated = []
    for selected_token in scripted_tokens:
        finished, reason, generated = apply_stopping(
            generated, selected_token, eos_id, stop_sequences)
        if finished:
            return {"tokens": generated, "reason": reason}
    return {"tokens": generated, "reason": "maximum_length"}
```

Part 5 - The Philosophy Behind the Exercise

Termination defines when a possibility becomes a completed artifact. EOS lets the model express completion; stop patterns let the application define a boundary; maximum length protects finite resources.

These authorities should not be confused. A trustworthy generator records not only what it produced, but why it believed the act of producing should end.

17.8.16 Exercise 15

Exercise Title: Profile GPU Utilization During Generation and Identify Whether Model Compute or Sampling Logic Dominates

Part 1 - Why this matters

Autoregressive decode often underutilizes a GPU because each step launches small kernels and waits for sequential control logic. With a small model or batch, Python, cache concatenation, top-p sorting, host synchronization, and stopping checks may take as long as model compute. A high utilization percentage during prefill can coexist with poor per-token latency during decode. Profiling must separate these phases and name the actual bottleneck.

Use PyTorch Profiler over a short steady-state generation window and add named ranges for prefill, model decode, logits processing, sampling, cache update, and stopping logic. Record CPU and CUDA activities, shapes, memory, kernel launches, and synchronization events. Also collect wall time and CUDA-event time per phase. Detailed profiling changes latency, so use it for attribution and a lighter timer for final numbers.

Compare greedy, top-k, and top-p at several vocabulary sizes, batch sizes, prompt lengths, and cache lengths. If sampling dominates, vectorize filters, keep state on device, reduce sorting scope, or use fused operations. If model decode dominates, increase batch, use optimized attention/cache layouts, compilation, or suitable precision after parity checks. If Python gaps dominate, reduce per-token `item()` calls and move stopping state to device. Re- profile after one change and report end-to-end improvement, not isolated kernel speed.

Part 2 - Mathematical form and reasoning

Per-step wall time is

$$t_u = t_{\text{model}} + t_{\text{logits}} + t_{\text{sample}} + t_{\text{cache}} + t_{\text{stop}} + t_{\text{sync}} - t_{\text{overlap}}.$$

Phase fraction is $r_i = t_i/t_u$. The largest non-overlapped fraction gives the first optimization target. Amortized decode throughput for batch B is B/t_u, while user-visible latency remains approximately t_u per token. An optimization can improve throughput without improving single-request latency.

Part 3 - Hints, controls, and evidence

1. Profile prefill and decode separately and exclude profiler warmup.
2. Avoid accidental synchronizations in the baseline, especially token conversion to Python inside each step.
3. Measure p50, p95, and p99 token latency because periodic stalls disappear in an average.
4. Keep output tokens or distribution parity checks after every optimization.
5. Link each intervention to a specific observed timeline region and re-run the same workload.

Part 4 - Python code skeleton

```
import torch
from torch.profiler import profile, ProfilerActivity, record_function

def profiled_generation(prefill, decode_step, sampling_step, steps=16):
    with profile(activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
                 record_shapes=True, profile_memory=True) as profiler:
        with record_function("prefill"):
            state = prefill()
        for step in range(steps):
            with record_function("model_decode"):
                logits, state = decode_step(state)
            with record_function("sampling"):
                selected_token = sampling_step(logits)
            state["selected_token"] = selected_token
    table = profiler.key_averages().table(
        sort_by="self_cuda_time_total", row_limit=30)
    return table

def E15(make_workload):
    reports = {}
    for policy in ("greedy", "top_k", "top_p"):
        prefill, decode_step, sampling_step = make_workload(policy)
        reports[policy] = profiled_generation(
            prefill, decode_step, sampling_step)
    return reports
```

Part 5 - The Philosophy Behind the Exercise

A generation timeline shows where sequential dependence becomes waiting. The GPU, CPU, cache, and sampling policy take turns determining when the next token may exist.

Optimization is not the pursuit of constant activity. It is the removal of the dependency that most delays the outcome users actually experience.

17.8.17 Exercise 16

Exercise Title: Export Decoder-Only Model Using TorchScript or ONNX and Validate Output Parity and Latency

Part 1 - Why this matters

Export is not a file-conversion ritual. It defines a stable inference contract that another runtime can execute without silently changing causal behavior. A decoder-only generator contains two different systems: a tensor program that maps tokens and state to logits, and a control program that chooses tokens, updates stopping rules, and schedules the next call. Export the tensor program first. Keep sampling and request policy outside the graph until their semantics are tested independently.

Define separate contracts for prefill and cached decode. Prefill accepts prompt IDs and their attention mask, processes every prompt position, and returns next-token logits plus one

key/value state per layer. Decode accepts new tokens, the previous cache, valid lengths or positions, and returns new logits and an extended cache. Dynamic axes must be intentional: batch size, prompt length, new-token length, and cached length may vary, whereas layer count, head count, and head dimension normally remain fixed.

Validate more than one happy-path tensor. Test several batches, prompt lengths, padding patterns, and cache lengths. Compare native and exported logits, every cache tensor, greedy next tokens, and a short deterministic generation trace. Tracing may freeze Python-dependent branches or shapes; scripting may reject unsupported Python; ONNX runtimes may lower operators differently. A successful export therefore proves only serialization. Parity tests establish semantic portability. Benchmark only after warm-up, with synchronization around device timing, and report graph compilation or session-start cost separately from steady-state latency.

Part 2 - Mathematical form and reasoning

The deployment interface can be written as

$$f_\theta(X, M, K_{<t}, P_t) \longrightarrow (Z_t, K_{\leq t}),$$

where X is the new token block, M its visibility mask, $K_{<t}$ the past key/value state, P_t position information, and Z_t the logits. For reference output Z and exported output $\widehat{Z}$, record

$$e_{\max} = \max_i |Z_i - \widehat{Z}_i|, \qquad e_{\text{rel}} = \frac{\|Z - \widehat{Z}\|_2}{\|Z\|_2 + \varepsilon}.$$

Numeric closeness is necessary, while equality of $\arg\max Z$ checks whether small errors alter deterministic decoding. Split latency into export or compile time, runtime initialization, prefill, and per-token decode.

Part 3 - Hints, controls, and evidence

Freeze one checkpoint, tokenizer, special-token policy, and precision. Put the model in evaluation mode and disable gradients. Make masks explicit instead of relying on runtime defaults. For TorchScript, compare tracing and scripting only where supported. For ONNX, declare dynamic axes and execute the result in the target runtime; inspecting the saved graph is insufficient. Use strict float32 tolerances and documented wider reduced-precision tolerances. Save failing inputs and the first cache component that diverges. Report median and tail latency after warm-up, plus model size and peak memory. The evidence is a parity table across shapes and a latency table with a precise measurement boundary.

Part 4 - Python code skeleton

```
import time
import torch
from torch import nn

class LogitExportWrapper(nn.Module):
    def __init__(self, model):
        super().__init__()
        self.model = model

    def forward(self, input_ids, attention_mask):
```

```
        output = self.model(input_ids, attention_mask=attention_mask)
        return output.logits

def compare_logits(reference, candidate, atol, rtol):
    absolute_error = (reference - candidate).abs()
    max_error = float(absolute_error.max().item())
    denominator = reference.norm() + 1e-12
    difference_norm = (reference - candidate).norm()
    relative_error = float(difference_norm.div(denominator).item())
    same_token = torch.equal(reference.argmax(-1), candidate.argmax(-1))
    close = torch.allclose(reference, candidate, atol=atol, rtol=rtol)
    return close, same_token, max_error, relative_error

def synchronized_latency(function, iterations, device):
    for _ in range(10):
        function()
    if device.type == "cuda":
        torch.cuda.synchronize(device)
    started = time.perf_counter()
    for _ in range(iterations):
        function()
    if device.type == "cuda":
        torch.cuda.synchronize(device)
    return (time.perf_counter() - started) / iterations

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
wrapper = LogitExportWrapper(model).eval().to(device)
example_ids = torch.randint(0, vocab_size, (2, 12), device=device)
example_mask = torch.ones_like(example_ids)

with torch.inference_mode():
    example_inputs = (example_ids, example_mask)
    traced = torch.jit.trace(wrapper, example_inputs, strict=False)
    for length in (4, 12, 31):
        ids = torch.randint(0, vocab_size, (2, length), device=device)
        mask = torch.ones_like(ids)
        native_logits = wrapper(ids, mask)
        exported_logits = traced(ids, mask)
        result = compare_logits(native_logits, exported_logits, 1e-5, 1e-4)
        print("length", length, "parity", result)

    native_time = synchronized_latency(
        lambda: wrapper(example_ids, example_mask), 100, device
    )
    exported_time = synchronized_latency(
        lambda: traced(example_ids, example_mask), 100, device
    )
    print("native seconds", native_time)
```

```
print("exported seconds", exported_time)
```

Extend the wrapper to expose flattened cache tensors, or create separate prefill and decode wrappers. Reconstruct the cache in a fixed documented order and verify every component before claiming cached export parity.

Part 5 - The Philosophy Behind the Exercise

Deployment is a change of execution authority. A model that worked in its training process is not yet the same model in another runtime. Trust comes from making the boundary explicit and testing meaning across it. Portability is not the ability to open a file; it is preservation of causal computation, state, decisions, and measurable performance.

17.8.18 Exercise 17

Exercise Title: Stress Test Long Generation With Large Context and Detect Memory Leaks or Cache Growth Bugs

Part 1 - Why this matters

Short demonstrations hide the two costs that dominate long autoregressive generation: attention over a growing context and storage of the key/value cache. Memory should increase predictably as tokens are appended. A leak is different: memory or retained Python state continues to grow beyond the tensors required by the declared context. Confusing expected cache growth with a leak can cause unsafe fixes; overlooking a leak can crash a long-lived inference service.

Construct a stress matrix over prompt length, requested output length, batch size, layer count, precision, and cache policy. Begin with one request and a modest sequence, then increase one axis at a time. At every decode step log logical context length, cache tensor shapes and bytes, CUDA memory allocated, CUDA memory reserved, peak memory, and token latency. Do not retain full logits, attention matrices, or tensors with computation graphs merely to plot them later. Store scalar summaries on the CPU and run under inference mode.

A correct unbounded cache grows linearly in generated length, while attention work per decode step grows with attended context. Reserved allocator memory may remain above allocated memory and need not fall after a request; that alone is not a leak. Repeat identical requests after releasing request-local state and synchronizing. Stable allocated memory at the same checkpoint is stronger evidence than visually inspecting reserved memory. Assert that every layer's cache length grows by exactly the number of newly processed tokens. Duplicate concatenation, stale cache references, accidental full-prefix recomputation, and retained autograd graphs produce distinct signatures that the report must name.

Part 2 - Mathematical form and reasoning

For L layers, batch B, H heads, cache length T, head width d_h, and s bytes per scalar, approximate key/value storage is

$$M_{\mathrm{KV}}(T) = 2LBHTd_hs.$$

Appending ΔT tokens should add approximately $2LBH\Delta Td_hs$ bytes, apart from allocator rounding and metadata. For repeated trials r at the same maximum length, define checkpoint drift as

$$D_r = M^{(r)}_{\text{allocated,end}} - M^{(1)}_{\text{allocated,end}}.$$

Persistent positive drift indicates retained state and deserves object- and tensor-level investigation. A sliding window bounds cache memory but changes the model's available context; it is a semantic policy, not a free cleanup.

Part 3 - Hints, controls, and evidence

Establish an analytic expected-cache curve before measuring CUDA memory. Record both allocated and reserved bytes, but interpret them separately. Synchronize before timing and before reading final memory. Repeat with cache disabled to reveal accidental full-prefix work, and with batch one to isolate request mixing. Test EOS early, maximum length, and forced long output. Add assertions for cache layer count, matching key/value shapes, batch dimension, head dimension, and monotonic length. If using a bounded window, prove cache length never exceeds the bound and separately measure quality loss. Evidence includes per-step curves, repeated-run end checkpoints, failure thresholds, the first violating step, and a minimal reproduction.

Part 4 - Python code skeleton

```
import gc
import time
import torch

def tensor_bytes(tensor):
    return tensor.numel() * tensor.element_size()

def cache_summary(past_key_values):
    total_bytes = 0
    lengths = []
    for layer_index, layer_cache in enumerate(past_key_values):
        key, value = layer_cache
        if key.shape != value.shape:
            raise RuntimeError("key/value shape mismatch")
        total_bytes += tensor_bytes(key)
        total_bytes += tensor_bytes(value)
        lengths.append(int(key.shape[-2]))
        print("layer", layer_index, "cache shape", tuple(key.shape))
    return total_bytes, lengths

@torch.inference_mode()
def stress_decode(model, input_ids, steps, device):
    torch.cuda.reset_peak_memory_stats(device)
    output = model(input_ids=input_ids, use_cache=True)
    cache = output.past_key_values
    token = output.logits[:, -1, :].argmax(-1, keepdim=True)
    previous_length = int(input_ids.shape[1])
    records = []

    for step in range(steps):
```

```
        torch.cuda.synchronize(device)
        started = time.perf_counter()
        output = model(input_ids=token, past_key_values=cache, use_cache=True)
        torch.cuda.synchronize(device)
        elapsed = time.perf_counter() - started
        cache = output.past_key_values
        token = output.logits[:, -1, :].argmax(-1, keepdim=True)

        cache_bytes, lengths = cache_summary(cache)
        for length in lengths:
            if length != previous_length + 1:
                raise RuntimeError("cache length did not grow by one")
        previous_length += 1
        records.append({
            "step": step,
            "context": previous_length,
            "cache_bytes": cache_bytes,
            "allocated": torch.cuda.memory_allocated(device),
            "reserved": torch.cuda.memory_reserved(device),
            "peak": torch.cuda.max_memory_allocated(device),
            "seconds": elapsed,
        })
    return records

def release_request_state():
    gc.collect()
    if torch.cuda.is_available():
        torch.cuda.synchronize()

# Run identical trials and compare allocated memory at equal checkpoints.
```

For real models, adapt the cache-length axis to the library's cache layout. Avoid empty-cache functions inside the measured request; they can hide ownership bugs and produce unrealistic service behavior.

Part 5 - The Philosophy Behind the Exercise

Resource growth must be interpreted through the model's declared state. Memory that represents useful history is a cost; memory that survives without serving future computation is a defect. Engineering maturity lies in predicting the first, detecting the second, and admitting when a bounded resource policy also changes the meaning of the model.

17.8.19 Exercise 18

Exercise Title: Build a Full Generation Debugging Checklist Covering Masking, Cache, Sampling, and Precision Issues

Part 1 - Why this matters

Generation failures are deceptive because a pipeline can remain syntactically valid while being semantically wrong. A causal-mask error may leak future tokens during training. A

padding error may affect only shorter prompts. A KV cache may work for the first step and corrupt positions afterward. Sampling can produce valid token IDs from an invalid probability distribution, and reduced precision can turn a rare extreme prompt into NaNs. Reading generated prose is therefore a weak debugging method; the system needs executable invariants.

Organize diagnosis in dependency order. First freeze artifacts and inputs: checkpoint identity, tokenizer fingerprint, vocabulary size, special-token IDs, prompt tokens, seed, dtype, device, and generation configuration. Then verify model state and tensor contracts: evaluation mode, disabled gradients, batch and sequence axes, attention-mask convention, causal visibility, position IDs, and finite logits. Next compare cached and uncached execution token by token. Only after logits agree should sampling, batching, EOS logic, export parity, and performance be investigated. This ordering prevents a late symptom from hiding an early violation.

Every checklist entry needs a machine-checkable assertion, a deliberately failing example, and a saved diagnostic artifact. For example, alter a masked future token and assert earlier logits do not change; insert padding and assert valid-token logits remain stable; compare full-prefix logits with single-token cached logits; confirm probability rows are finite, nonnegative, and sum to one; confirm finished sequences never change; and compare batch generation with independent single-item runs. A checklist that merely says "check the mask" is not a test. A useful checklist states what invariant must hold and what evidence will be emitted when it does not.

Part 2 - Mathematical form and reasoning

View generation as a composition of contracts

$$X \xrightarrow{\mathcal{T}} I \xrightarrow{\mathcal{M}} (Z, K) \xrightarrow{\mathcal{P}} p \xrightarrow{\mathcal{S}} y \xrightarrow{\mathcal{C}} (y, \text{status}),$$

where tokenization $\mathcal{T}$, model execution $\mathcal{M}$, logits policy $\mathcal{P}$, selection $\mathcal{S}$, and stopping controller $\mathcal{C}$ each have independent invariants. Debugging should find the earliest map whose output violates its contract. Important checks include

$$\sum_v p(v) = 1, \quad p(v) \geq 0, \quad Z_t^{\text{cached}} \approx Z_t^{\text{full}}, \quad y_{b,t} = y_{b,t}^{\text{single}}$$

under matched settings. Later equality is meaningful only when earlier inputs and random-number streams are controlled.

Part 3 - Hints, controls, and evidence

Run the suite first with greedy decoding because it removes random selection. Use a tiny model and handcrafted prompts so attention visibility can be inspected exactly. Include left and right padding, batch one and many, prompt length one, early EOS, no EOS, repeated tokens, extreme logits, and maximum context. Check float32 before reduced precision. Log the first divergent step, top competing logits, mask row, position ID, cache shapes, RNG state, and stop reason. Do not automatically "fix" a failure by loosening tolerance until its scale and decision effect are understood. Keep correctness tests separate from benchmarks so profiler overhead and synchronization cannot disguise behavior.

Part 4 - Python code skeleton

```
import torch

class GenerationAudit:
    def __init__(self):
        self.results = []

    def record(self, name, passed, details):
        entry = {"name": name, "passed": bool(passed), "details": details}
        self.results.append(entry)
        if not passed:
            raise AssertionError(str(entry))

def check_probabilities(probabilities, audit):
    finite = bool(torch.isfinite(probabilities).all())
    nonnegative = bool((probabilities >= 0).all())
    row_sums = probabilities.sum(dim=-1)
    expected = torch.ones_like(row_sums)
    normalized = torch.allclose(row_sums, expected, atol=1e-5)
    audit.record("probabilities finite", finite, {})
    audit.record("probabilities nonnegative", nonnegative, {})
    audit.record(
        "probabilities normalized",
        normalized,
        {"row_sums": row_sums.detach().cpu().tolist()},
    )

def compare_cached_step(full_logits, cached_logits, step, audit):
    difference = (full_logits - cached_logits).abs()
    maximum = float(difference.max().item())
    full_choice = full_logits.argmax(-1)
    cached_choice = cached_logits.argmax(-1)
    same_choice = torch.equal(full_choice, cached_choice)
    audit.record(
        "cache parity step " + str(step),
        torch.allclose(full_logits, cached_logits, atol=1e-5, rtol=1e-4),
        {"maximum_error": maximum, "same_greedy_token": same_choice},
    )

def check_finished_is_monotonic(previous_finished, current_finished, audit):
    revived = previous_finished & (~current_finished)
    audit.record(
        "finished state is monotonic",
        not bool(revived.any()),
        {"revived_rows": revived.nonzero().flatten().cpu().tolist()},
    )

def run_core_audit(probabilities, full_trace, cached_trace, finish_trace):
    audit = GenerationAudit()
```

```
    check_probabilities(probabilities, audit)
    if len(full_trace) != len(cached_trace):
        raise AssertionError("trace lengths differ")
    for step in range(len(full_trace)):
        compare_cached_step(full_trace[step], cached_trace[step], step, audit)
    for step in range(1, len(finish_trace)):
        check_finished_is_monotonic(
            finish_trace[step - 1], finish_trace[step], audit
        )
    return audit.results

# Add causal intervention, padding invariance, batch parity, EOS, dtype,
# export parity, cache-size, and latency-budget tests to the same registry.
```

Run each test once in its correct form and once with an intentional bug. If the test does not fail under the known bug, it is not yet protective evidence.

Part 5 - The Philosophy Behind the Exercise

Debugging is the reconstruction of causality, not the collection of suspicions. The pipeline should be trusted only to the depth at which its contracts have survived adversarial tests. By turning assumptions into executable invariants, the engineer replaces intuition about fluent output with evidence about the mechanism that produced it.

17.8.20 Exercise 19

Exercise Title: Produce a Full Decoder-Only Language Modeling Report Covering Training, Decoding, Failures, and Fixes

Part 1 - Why this matters

The final report is the evidence boundary of the entire chapter. It must allow another engineer to reproduce the model, understand why its generation policy was selected, identify where it is reliable, and know which conditions make it fail. A page of final loss and attractive samples cannot do this. Decoder-only systems connect data alignment, causal training, optimization, cache semantics, sampling policy, stopping rules, numerical precision, hardware behavior, and deployment runtime. The report must preserve those dependencies.

Begin with the experimental identity: code and environment versions, dataset and split fingerprints, tokenizer and special-token IDs, sequence construction, model dimensions, parameter count, seed policy, precision, device, optimizer, schedule, checkpoint, and resume history. Present training and validation loss, perplexity, gradient statistics, throughput, memory, and the tiny-task correctness test. State whether the model was trained with the same context and mask semantics used for evaluation.

Then report decoding as a controlled study. Compare greedy, temperature, top-k, top-p, frequency penalty, and beam search on identical prompts and seeds where applicable. Record likelihood-related metrics, repetition, novelty, length, EOS success, task correctness, latency, and qualitative samples chosen by a predeclared rule. Document KV-cache parity, batched versus single-request parity, stop reasons, float32 versus reduced-precision differences, export parity, long-context memory curves, and profiler attribution. Include failed runs and negative

results: a technique that did not help is part of the model's engineering history. End with recommended defaults, the evidence supporting them, known limitations, and explicit triggers for revisiting each decision.

Part 2 - Mathematical form and reasoning

Separate model fit from generation utility. A report may summarize model fit by

$$\mathrm{PPL} = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log p_\theta(x_i \mid x_{<i})\right),$$

but deployment selection is multi-objective. For policy g, define a vector

$$R(g) = (Q_g, D_g, S_g, L_g, M_g, F_g),$$

containing task quality, diversity, stop correctness, latency, memory, and failure rate. Do not collapse this vector into one score unless weights and normalizations are justified. Every comparison should include uncertainty or variation across prompts and seeds, not only its mean. A production limit is a measured boundary with a failure criterion, not the largest successful run seen once.

Part 3 - Hints, controls, and evidence

Use a fixed report schema and link each claim to a table, plot, saved sample, or test result. Distinguish measurements from interpretations and interpretations from recommendations. State denominators, excluded samples, warm-up rules, hardware, batching, and timing boundaries. Include a failure registry with symptom, reproduction, root cause, correction, regression test, and remaining risk. Redact sensitive text while preserving reproducible synthetic cases. Record exceptions to the default decoding policy by workload. Before release, have the report validator reject missing sections, unversioned artifacts, unbounded generation settings, or recommendations with no cited evidence.

Part 4 - Python code skeleton

```
import json
from pathlib import Path

REQUIRED_SECTIONS = (
    "experiment_identity",
    "data_and_tokenizer",
    "architecture",
    "training_results",
    "decoding_comparison",
    "cache_and_batching",
    "precision_and_cuda",
    "profiling_and_export",
    "stress_limits",
    "failure_registry",
    "recommended_defaults",
    "known_limitations",
)
```

```
def validate_report(report):
    errors = []
    for section in REQUIRED_SECTIONS:
        if section not in report:
            errors.append("missing section: " + section)
    identity = report.get("experiment_identity", {})
    for field in ("code_version", "checkpoint_hash", "seed", "device"):
        if field not in identity:
            errors.append("missing identity field: " + field)
    defaults = report.get("recommended_defaults", {})
    if "max_new_tokens" not in defaults:
        errors.append("generation has no explicit maximum")
    if "evidence_ids" not in defaults:
        errors.append("defaults have no evidence references")
    failures = report.get("failure_registry", [])
    for index in range(len(failures)):
        failure = failures[index]
        for field in ("symptom", "root_cause", "fix", "regression_test"):
            if field not in failure:
                errors.append(
                    "failure " + str(index) + " missing field: " + field
                )
    if errors:
        raise ValueError("\n".join(errors))

def save_report(report, output_path):
    validate_report(report)
    path = Path(output_path)
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", encoding="utf-8") as handle:
        json.dump(report, handle, indent=2, sort_keys=True)

report = {
    "experiment_identity": {
        "code_version": code_version,
        "checkpoint_hash": checkpoint_hash,
        "seed": seed,
        "device": str(device),
    },
    "data_and_tokenizer": data_summary,
    "architecture": architecture_summary,
    "training_results": training_results,
    "decoding_comparison": decoding_results,
    "cache_and_batching": cache_results,
    "precision_and_cuda": precision_results,
    "profiling_and_export": deployment_results,
    "stress_limits": stress_results,
    "failure_registry": failure_registry,
```

```
    "recommended_defaults": recommended_defaults,
    "known_limitations": known_limitations,
}
save_report(report, "reports/trf06_decoder_only_report.json")
```

Generate human-readable tables and plots from this structured record so the narrative and machine-checkable evidence cannot silently disagree.

Part 5 - The Philosophy Behind the Exercise

A final report is not a celebration of completion; it is a transfer of responsibility. It tells the next researcher what is known, how it became known, what remains uncertain, and which observations would justify changing course. The deepest result of this chapter is therefore not a generated sequence. It is a disciplined chain from causal objective to operational evidence, with enough honesty that future work can begin from reality rather than mythology.

17.9 TRF07

Chapter Title: Encoder-Decoder, Cross-Attention, and Seq2Seq
Specialization: Models and Training

17.9.1 Exercise 00

Exercise Title: Explain Encoder-Decoder Transformer and Why Cross-Attention Enables Conditional Generation

Part 1 - Why this matters

A decoder-only model predicts a continuation from one growing stream. An encoder-decoder Transformer solves a different structural problem: it receives a complete source sequence, constructs a contextual memory of that source, and then produces a target sequence conditioned on the memory. Translation, summarization, transcription, structured conversion, and many question-answer tasks naturally have this two-sequence form. Treating source and target as one undifferentiated stream is possible, but it changes masking, compute reuse, and the inductive organization of the task.

The encoder reads all valid source positions bidirectionally. Its output at each source position is not a token probability; it is a contextual memory vector. The decoder has two attention operations. Masked self-attention lets a target position use only earlier target positions, preserving autoregressive factorization. Cross-attention then uses decoder states as queries and encoder memory as keys and values. It asks, for every target position, which parts of the source are relevant now and what source information should be retrieved. Thus the decoder is simultaneously constrained by its generated history and conditioned on the entire visible source.

Build a conceptual trace before training anything. Give a tiny source and a partially shifted target unique token IDs. Record source embeddings, encoder memory, decoder self-attention states, cross-attention weights, logits, and the next-token target. Repeat after changing one source token while holding the target prefix fixed. Decoder logits should change because the condition changed. Then replace encoder memory with zeros; the model may still use target

history, but source-specific evidence disappears. This intervention distinguishes true conditional generation from a decoder that learns only target-side regularity.

Part 2 - Mathematical form and reasoning

For source $x_{1:S}$ and target $y_{1:T}$, the model factorizes

$$p_\theta(y_{1:T} \mid x_{1:S}) = \prod_{t=1}^{T} p_\theta(y_t \mid y_{<t}, x_{1:S}).$$

The encoder constructs memory

$$H = \text{Encoder}(E_x(x) + P_x), \qquad H \in \mathbb{R}^{B\times S\times d}.$$

After masked target self-attention produces $U \in \mathbb{R}^{B\times T\times d}$, one cross-attention head computes

$$Q = UW_Q, \quad K = HW_K, \quad V = HW_V,$$

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M_{\text{source}}\right), \qquad C = AV.$$

The target causal mask belongs to decoder self-attention; the source padding mask belongs to cross-attention. Cross-attention normally has no triangular mask because every real source position is already known. A has shape (B, H, T, S): each target query owns a probability distribution over source positions. The output projection and feed-forward layers transform retrieved context into logits over the target vocabulary.

Part 3 - Hints, controls, and evidence

Use distinct source and target sequence lengths so swapped axes are visible. Start with batch one, one head, no dropout, and fixed weights. Assert that every cross-attention row sums to one over unmasked source positions and assigns zero mass to padding. Prove target causality separately by changing a future target token and checking earlier logits. Then prove conditioning by changing a valid source token and checking at least one target logit changes. Compare three ablations: correct memory, shuffled memory positions, and zero memory. Save shape traces and intervention errors rather than relying on an attention heatmap alone. Attention weights describe retrieval inside the model; they are useful diagnostics but are not automatically a complete causal explanation.

Part 4 - Python code skeleton

```
import torch

@torch.inference_mode()
def conditionality_probe(model, source_ids, target_prefix, source_mask):
    model.eval()
    memory = model.encode(source_ids, source_mask)
    logits, cross_weights = model.decode(
        target_prefix, memory, source_mask, return_cross_weights=True
    )

    changed_source = source_ids.clone()
```

```
    replacement = changed_source[0, 0] + 1
    changed_source[0, 0] = replacement % model.source_vocab_size
    changed_memory = model.encode(changed_source, source_mask)
    changed_logits, _ = model.decode(
        target_prefix, changed_memory, source_mask,
        return_cross_weights=True
    )

    zero_memory = torch.zeros_like(memory)
    zero_logits, _ = model.decode(
        target_prefix, zero_memory, source_mask,
        return_cross_weights=True
    )

    source_effect = (logits - changed_logits).abs().max().item()
    memory_ablation_effect = (logits - zero_logits).abs().max().item()
    expanded_source_mask = source_mask[:, None, None, :]
    valid_mass = cross_weights.masked_fill(~expanded_source_mask, 0.0)
    row_sums = valid_mass.sum(dim=-1)

    assert source_effect > 0.0
    assert memory_ablation_effect > 0.0
    assert torch.allclose(row_sums, torch.ones_like(row_sums), atol=1e-5)
    return {
        "encoder_memory_shape": tuple(memory.shape),
        "cross_attention_shape": tuple(cross_weights.shape),
        "source_effect": source_effect,
        "memory_ablation_effect": memory_ablation_effect,
    }

def E00(model, source_ids, target_prefix, source_mask):
    return conditionality_probe(model, source_ids, target_prefix, source_mask)
```

The model interface is intentionally explicit: encoding and decoding are separate so memory reuse and conditionality can be inspected directly.

Part 5 - The Philosophy Behind the Exercise

Conditional generation is a disciplined conversation between two forms of knowledge: what the source says and what the target has already said. The encoder preserves the source as addressable memory; the decoder decides what to ask of that memory at each step. Cross-attention matters because conditioning is not passive storage. It is repeated, context-dependent retrieval. The model does not compress the whole source into one final state and hope it survives; it returns to the source whenever the evolving target requires new evidence.

17.9.2 Exercise 01

Exercise Title: Implement Cross-Attention from Scratch and Verify Shapes for Encoder Outputs and Decoder Queries

Part 1 - Why this matters

Cross-attention resembles self-attention algebraically, but its tensors come from different sequences. Queries represent target-side needs; keys and values represent source-side memory. That asymmetry is exactly where silent shape bugs occur. Equal source and target lengths can hide swapped dimensions, and a mask that broadcasts without error can still cover the wrong axis. Implementing the operation explicitly makes its data ownership visible.

Create separate projections for target queries and encoder keys and values. Split each projection into heads using named intermediate dimensions, transpose only the head and sequence axes, scale by the head width, apply a source padding mask, normalize over the source axis, mix values, concatenate heads, and apply the output projection. Return attention weights for inspection. Do not use `nn.MultiheadAttention` in the reference version.

Validate with $B = 2$, target length $T = 3$, source length $S = 5$, model width $d = 8$, and two heads. Manually compute one head for one batch item and compare it with the implementation. Mask the last two source positions of one example and prove their weights are zero. Change only decoder queries: attention weights should change while projected source keys and values remain identical. Change only encoder memory: keys and values must change. These interventions verify semantic provenance, not merely final shape.

Part 2 - Mathematical form and reasoning

With $U \in \mathbb{R}^{B\times T\times d}$ and $H \in \mathbb{R}^{B\times S\times d}$,

$$Q = UW_Q, \qquad K = HW_K, \qquad V = HW_V.$$

For h heads and $d_h = d/h$, reshape them to (B, h, T, d_h) for Q and (B, h, S, d_h) for K, V. Then

$$L_{b,r,t,s} = \frac{Q_{b,r,t,:} \cdot K_{b,r,s,:}}{\sqrt{d_h}}, \quad A_{b,r,t,:} = \operatorname{softmax}(L_{b,r,t,:} + M_{b,s}),$$

$$C_{b,r,t,:} = \sum_{s=1}^{S} A_{b,r,t,s} V_{b,r,s,:}.$$

After concatenation, C returns to (B, T, d). Softmax must use the final source axis. The mask must broadcast from (B, S) to $(B, 1, 1, S)$.

Part 3 - Hints, controls, and evidence

Assert that d is divisible by the number of heads. Use `reshape` only after ensuring the tensor layout is valid, and call `contiguous` before merging heads after a transpose. Reject batches whose source mask does not match memory length. For a fully masked row, choose and document a policy; ordinary seq2seq batches should contain at least one valid source token. Compare outputs and gradients against PyTorch's attention module after copying weights into the same convention. Evidence should contain the complete shape ledger, row-sum checks, masked-weight maximum, output parity, and gradient parity.

Part 4 - Python code skeleton

```
import math
import torch
from torch import nn

class ScratchCrossAttention(nn.Module):
    def __init__(self, model_width, number_of_heads):
        super().__init__()
        if model_width % number_of_heads != 0:
            raise ValueError("model width must divide evenly across heads")
        self.number_of_heads = number_of_heads
        self.head_width = model_width // number_of_heads
        self.query_projection = nn.Linear(model_width, model_width)
        self.key_projection = nn.Linear(model_width, model_width)
        self.value_projection = nn.Linear(model_width, model_width)
        self.output_projection = nn.Linear(model_width, model_width)

    def split_heads(self, tensor):
        batch_size, sequence_length, model_width = tensor.shape
        tensor = tensor.reshape(
            batch_size, sequence_length,
            self.number_of_heads, self.head_width
        )
        return tensor.transpose(1, 2)

    def forward(self, decoder_states, encoder_memory, source_mask):
        queries = self.split_heads(self.query_projection(decoder_states))
        keys = self.split_heads(self.key_projection(encoder_memory))
        values = self.split_heads(self.value_projection(encoder_memory))
        scores = torch.matmul(queries, keys.transpose(-2, -1))
        scores = scores / math.sqrt(self.head_width)
        visible = source_mask[:, None, None, :]
        scores = scores.masked_fill(~visible, torch.finfo(scores.dtype).min)
        weights = torch.softmax(scores.float(), dim=-1).to(scores.dtype)
        context = torch.matmul(weights, values)
        context = context.transpose(1, 2).contiguous()
        batch_size, target_length, _, _ = context.shape
        context = context.reshape(batch_size, target_length, -1)
        return self.output_projection(context), weights

def E01(decoder_states, encoder_memory, source_mask):
    layer = ScratchCrossAttention(decoder_states.size(-1), 2)
    output, weights = layer(decoder_states, encoder_memory, source_mask)
    assert output.shape == decoder_states.shape
    assert weights.shape[-2:] == (
        decoder_states.size(1), encoder_memory.size(1)
    )
    return output, weights
```

Part 5 - The Philosophy Behind the Exercise

The same matrix operation acquires different meaning from the origin of its arguments. Cross-attention teaches that tensor correctness includes provenance: who asks, what is searched, and which axis represents the available evidence. Shapes are necessary, but semantics live in the route by which tensors arrived.

17.9.3 Exercise 02

Exercise Title: Build a Tiny Seq2Seq Transformer and Validate Teacher Forcing Training Pipeline Correctly

Part 1 - Why this matters

Teacher forcing trains all target positions in parallel by feeding the known previous target tokens rather than the model's own sampled outputs. It is efficient and gives a clear next-token objective, but only if target shifting is exact. A single off-by-one error can let the current answer appear in the input, train the model to reproduce the wrong position, or misalign logits and labels while loss still decreases. This exercise builds an end-to-end reference small enough to overfit deliberately.

Create source and target embeddings, positional embeddings, an encoder, a decoder, and an output projection. A target sequence should contain beginning and end tokens. Feed `target[:, :-1]` to the decoder and compare its logits with `target[:, 1:]`. Padding labels receive an ignore index. Construct source and target masks from the exact tensors consumed by the model. Use a tiny deterministic copy or reverse dataset and train until nearly perfect token and exact-sequence accuracy. Tiny-data overfitting is a pipeline unit test, not a generalization claim.

Inspect one example explicitly. Print source IDs, full target IDs, decoder input, labels, causal mask, logits shape, and valid-token count. Confirm that the first decoder input is BOS, the first label is the first content token, and the last non-padding label is EOS. During generation, start from BOS and never supply the unknown reference continuation. This separates the training contract from the inference contract and exposes teacher-forcing leakage.

Part 2 - Mathematical form and reasoning

For a target $[\mathrm{BOS}, y_1, \ldots, y_T, \mathrm{EOS}]$, training uses

$$Y_{\text{in}} = [\mathrm{BOS}, y_1, \ldots, y_T], \qquad Y_{\text{label}} = [y_1, \ldots, y_T, \mathrm{EOS}].$$

The loss is

$$\mathcal{L} = -\frac{1}{N_{\text{valid}}} \sum_{b,t:\, Y_{b,t} \neq \mathrm{PAD}} \log p_\theta(Y_{b,t} \mid Y_{b,<t}, X_b).$$

There is one logit vector per decoder-input position and one aligned label. Both have length $T+1$ in this example. The target causal mask prevents position t from reading positions greater than t even though teacher forcing provides them elsewhere in the batch tensor.

Part 3 - Hints, controls, and evidence

Use separate source and target padding IDs if the vocabularies differ. Set dropout to zero for the first overfit test and verify every parameter receives a finite gradient. Evaluate

autoregressively rather than reporting teacher-forced accuracy alone. Create two intentional failures: remove the target shift and remove the causal mask. The first should be caught by alignment assertions; the second by a future-token intervention test. Save a known batch as a regression fixture. Evidence includes loss history, valid-token count, teacher-forced token accuracy, autoregressive exact match, and generated sequences.

Part 4 - Python code skeleton

```
import torch
from torch import nn

class TinySeq2Seq(nn.Module):
    def __init__(self, source_vocab, target_vocab, width, heads, layers,
                 maximum_length, padding_id):
        super().__init__()
        self.source_embedding = nn.Embedding(source_vocab, width, padding_id)
        self.target_embedding = nn.Embedding(target_vocab, width, padding_id)
        self.position_embedding = nn.Embedding(maximum_length, width)
        self.transformer = nn.Transformer(
            d_model=width, nhead=heads,
            num_encoder_layers=layers, num_decoder_layers=layers,
            dim_feedforward=4 * width, dropout=0.0, batch_first=True
        )
        self.output_projection = nn.Linear(width, target_vocab)

    def add_positions(self, embedded):
        length = embedded.size(1)
        positions = torch.arange(length, device=embedded.device)
        return embedded + self.position_embedding(positions)[None, :, :]

    def forward(self, source_ids, decoder_ids, source_padding,
                target_padding):
        source = self.add_positions(self.source_embedding(source_ids))
        target = self.add_positions(self.target_embedding(decoder_ids))
        target_length = decoder_ids.size(1)
        causal_mask = nn.Transformer.generate_square_subsequent_mask(
            target_length, device=decoder_ids.device
        )
        hidden = self.transformer(
            source, target, tgt_mask=causal_mask,
            src_key_padding_mask=source_padding,
            tgt_key_padding_mask=target_padding,
            memory_key_padding_mask=source_padding,
        )
        return self.output_projection(hidden)

def E02(model, source_ids, full_target, padding_id):
    decoder_ids = full_target[:, :-1]
    labels = full_target[:, 1:]
```

```
    source_padding = source_ids.eq(padding_id)
    target_padding = decoder_ids.eq(padding_id)
    logits = model(source_ids, decoder_ids, source_padding, target_padding)
    assert logits.shape[:2] == labels.shape
    loss = nn.functional.cross_entropy(
        logits.reshape(-1, logits.size(-1)), labels.reshape(-1),
        ignore_index=padding_id
    )
    return loss, logits, labels
```

Part 5 - The Philosophy Behind the Exercise

Teacher forcing is a bargain: it exchanges inference realism for parallel, dense supervision. The bargain is valuable only when temporal meaning is protected by shifting and masking. This exercise teaches that data alignment is part of the model. An optimizer cannot repair a target that was defined one position incorrectly.

17.9.4 Exercise 03

Exercise Title: Implement Source Padding Mask and Target Causal Mask Together and Prove Both Masks Work Correctly

Part 1 - Why this matters

Seq2seq Transformers enforce two independent visibility laws. Source padding is not evidence and must be invisible to encoder self-attention and decoder cross-attention. Future target tokens are real tokens, but they are unavailable when the current token is predicted and must be hidden from decoder self-attention. Combining these masks incorrectly is dangerous because their shapes may broadcast and training loss may improve despite information leakage.

Implement masks from semantic Boolean tensors before converting them to the API convention required by a layer. Use deliberately different source and target lengths and different padding patterns for each batch row. Test source padding by replacing only padded source IDs with arbitrary alternatives; logits at valid target positions must remain unchanged. Test target causality by changing only target tokens after position t; logits through t must remain unchanged. Then modify a valid source token and confirm at least one target logit changes, proving the invariance test did not accidentally disable all conditioning.

Instrument self- and cross-attention weights if possible. For every target query, masked source columns must receive zero probability. For target self-attention, all entries above the diagonal must be zero. Distinguish a Boolean visibility mask from an additive logit mask, and document whether True means keep or block in every API call. Avoid a single variable named `mask` for all purposes; semantic names prevent convention inversion.

Part 2 - Mathematical form and reasoning

Let $P^x_{b,s} = 1$ for valid source positions and $P^y_{b,t} = 1$ for valid decoder inputs. Target causal visibility is

$$C_{t,j} = \mathbf{1}[j \leq t].$$

The decoder self-attention visibility for batch b is

$$V^{\text{self}}_{b,t,j} = P^{y}_{b,j} C_{t,j},$$

while cross-attention visibility is

$$V^{\text{cross}}_{b,t,s} = P^{x}_{b,s}.$$

An additive mask maps visible entries to zero and blocked entries to $-\infty$ before softmax. Its desired guarantee is

$$A_{b,h,t,j} = 0 \quad \text{whenever} \quad V_{b,t,j} = 0.$$

Query padding may also require output or loss masking; key masking alone does not remove predictions produced at padded target rows.

Part 3 - Hints, controls, and evidence

Create a mask-shape table before coding: source-valid (B, S), source-blocked (B, S), target-blocked (B, T), causal (T, T), cross score (B, H, T, S), and self score (B, H, T, T). Use float32 comparisons first and dropout-free eval mode. Test a batch row with no padding and another with several pads. Reject an empty source rather than allowing an all-masked softmax. Save maximum masked attention mass and maximum invariant-logit difference. An assertion should fail when you transpose the causal mask or omit the memory padding mask; this fault injection proves the test can detect the intended bug.

Part 4 - Python code skeleton

```
import torch

def causal_block_mask(target_length, device):
    row = torch.arange(target_length, device=device)[:, None]
    column = torch.arange(target_length, device=device)[None, :]
    return column > row

@torch.inference_mode()
def masking_invariance_test(model, source_ids, decoder_ids, padding_id):
    model.eval()
    source_blocked = source_ids.eq(padding_id)
    target_blocked = decoder_ids.eq(padding_id)
    target_length = decoder_ids.size(1)
    causal_blocked = causal_block_mask(target_length, decoder_ids.device)

    baseline = model(
        source_ids, decoder_ids, source_blocked,
        target_blocked, causal_blocked
    )

    padded_changed = source_ids.clone()
    replacement = torch.full_like(padded_changed, 1)
    padded_changed = torch.where(source_blocked, replacement, padded_changed)
```

```
    padding_variant = model(
        padded_changed, decoder_ids, source_blocked,
        target_blocked, causal_blocked
    )
    padding_error = (baseline - padding_variant).abs().max()

    future_changed = decoder_ids.clone()
    probe_position = 1
    future_changed[:, probe_position + 1:] = 2
    future_variant = model(
        source_ids, future_changed, source_blocked,
        target_blocked, causal_blocked
    )
    causal_error = (
        baseline[:, :probe_position + 1]
        - future_variant[:, :probe_position + 1]
    ).abs().max()

    assert padding_error.item() < 1e-5
    assert causal_error.item() < 1e-5
    return padding_error.item(), causal_error.item()

def E03(model, source_ids, decoder_ids, padding_id):
    return masking_invariance_test(model, source_ids, decoder_ids, padding_id)
```

Part 5 - The Philosophy Behind the Exercise

A mask is a law about what information is allowed to participate in a decision. Padding and the future are excluded for different reasons: one is not data, the other is data not yet available. Combining them correctly requires preserving that distinction. Reliable sequence modeling begins by treating visibility as a testable causal contract rather than a convenient tensor argument.

17.9.5 Exercise 04

Exercise Title: Train Seq2Seq Transformer on a Copy or Reverse Task and Measure Accuracy and Failure Modes

Part 1 - Why this matters

A synthetic copy or reverse task isolates the essential seq2seq pipeline. The symbols have no linguistic ambiguity, the correct output is exact, and examples can be generated at arbitrary lengths. If a small Transformer cannot solve this task, training on a real corpus will conceal rather than cure the defect. Copy tests source preservation and alignment; reversal additionally requires order- sensitive retrieval and a changing source position across decoder steps.

Generate integer sequences with reserved PAD, BOS, and EOS IDs. Split examples by seed and, more importantly, create an extrapolation set longer than training sequences. Train with teacher forcing, but evaluate with greedy autoregressive decoding from BOS until EOS or a fixed maximum. Measure valid-token accuracy, exact sequence match, EOS rate, length

error, and accuracy by source length. Teacher-forced accuracy alone can look excellent while autoregressive generation fails after its first mistake.

Create controlled failure modes: train without positional information, corrupt the target shift, omit source padding in cross-attention, choose a maximum output length shorter than the reference, and evaluate while dropout remains active. For each failure, predict its signature. A positionless model should struggle especially with reversal; wrong shifting may yield suspiciously easy training; padding leakage creates length-dependent errors; an inadequate token budget causes systematic truncation. Store the first mismatching position and the model confidence there instead of recording only a zero exact-match score.

Part 2 - Mathematical form and reasoning

For copy, $g(x_1, \ldots, x_S) = (x_1, \ldots, x_S)$; for reverse,

$$g(x_1, \ldots, x_S) = (x_S, \ldots, x_1).$$

Token accuracy and exact match are

$$\mathrm{Acc}_{\mathrm{token}} = \frac{\sum_{b,t} m_{b,t} \mathbf{1}[\hat{y}_{b,t} = y_{b,t}]}{\sum_{b,t} m_{b,t}},$$

$$\mathrm{EM} = \frac{1}{B} \sum_{b} \mathbf{1}[\hat{y}_b = y_b],$$

where comparison ends consistently at EOS and excludes padding. Exact match is strict: one early error can shift the remainder. Therefore report first-error position and length-conditioned results alongside it.

Part 3 - Hints, controls, and evidence

Start with short fixed-length sequences and overfit one batch. Progress to variable lengths only after exact match reaches one. Use separate random-number generators for training and evaluation. Ensure content tokens never collide with special IDs. Decode one token at a time using the exact same source mask as training. Compare teacher-forced predictions and autoregressive predictions for the same example to localize exposure effects. Plot loss, token accuracy, exact match, EOS rate, and accuracy by length. The strongest evidence includes one successful run, each intentional failure, a root-cause explanation, and a regression assertion that detects it.

Part 4 - Python code skeleton

```
import torch

def make_reverse_batch(batch_size, minimum_length, maximum_length,
                       vocabulary_size, bos_id, eos_id, padding_id, device):
    lengths = torch.randint(
        minimum_length, maximum_length + 1, (batch_size,), device=device
    )
    source = torch.full(
        (batch_size, maximum_length), padding_id, device=device
    )
```

```
    target = torch.full(
        (batch_size, maximum_length + 2), padding_id, device=device
    )
    target[:, 0] = bos_id
    for row in range(batch_size):
        length = int(lengths[row].item())
        tokens = torch.randint(3, vocabulary_size, (length,), device=device)
        source[row, :length] = tokens
        target[row, 1:length + 1] = tokens.flip(0)
        target[row, length + 1] = eos_id
    return source, target, lengths

def exact_match(predictions, references, eos_id, padding_id):
    matched = 0
    for row in range(predictions.size(0)):
        predicted_list = []
        reference_list = []
        for token in predictions[row].tolist():
            if token == eos_id:
                break
            if token != padding_id:
                predicted_list.append(token)
        for token in references[row].tolist():
            if token == eos_id:
                break
            if token != padding_id:
                reference_list.append(token)
        if predicted_list == reference_list:
            matched += 1
    return matched / predictions.size(0)

def E04(model, optimizer, source, full_target, padding_id):
    model.train()
    optimizer.zero_grad(set_to_none=True)
    loss, _, _ = model.training_objective(source, full_target, padding_id)
    loss.backward()
    optimizer.step()
    return float(loss.item())
```

Part 5 - The Philosophy Behind the Exercise

Synthetic tasks remove excuses. Their simplicity does not make them trivial; it makes them scientifically sharp. When the correct transformation is known exactly, failure becomes information about the pipeline rather than a debate about the data. Mastery begins by demanding that a complex system prove itself on a world whose laws we fully control.

17.9.6 Exercise 05

Exercise Title: Debug Seq2Seq Alignment Bugs Caused by Wrong Target Shifting or Incorrect Mask Broadcasting

Part 1 - Why this matters

Alignment bugs are among the most expensive seq2seq errors because they often produce legal shapes, finite gradients, and decreasing loss. If decoder input and label both contain the current token, the network learns an identity leak. If labels are shifted twice, predictions are judged against the wrong future. If a source mask broadcasts across target queries incorrectly, only particular batch sizes or sequence lengths may fail. This exercise creates an explicit alignment audit that runs before long training.

Build a batch with unique, easily recognized IDs rather than natural text. Print every row as source, full target, decoder input, label, source-valid mask, target-valid mask, and loss-valid positions. Verify these relations element by element, not merely by comparing shapes. The decoder input must begin with BOS; labels must end at EOS; no label at position t may appear as the decoder input at that same position unless it also genuinely occurred earlier. Count exactly how many tokens contribute to loss.

For mask broadcasting, construct attention scores with $B > 1$, $H > 1$, $T \neq S$, and a different padding pattern per row. Expand a semantic source visibility mask explicitly to $(B, 1, T, S)$ and compare it with the broadcast result used by the implementation. Inject common wrong shapes such as $(B, S, 1)$ or (S, B) and confirm the audit rejects them even when an operation happens to broadcast. Then run one optimization step on the correct and corrupted targets; the point is to see why loss magnitude alone cannot certify alignment.

Part 2 - Mathematical form and reasoning

Correct shifting requires

$$D_{b,t} = Y^{\text{full}}_{b,t}, \qquad L_{b,t} = Y^{\text{full}}_{b,t+1},$$

for $0 \leq t < T - 1$. Thus D and L have identical shapes but different temporal meanings. For source-valid mask $P \in \{0, 1\}^{B \times S}$, the desired cross-attention blocked mask is

$$M_{b,h,t,s} = \neg P_{b,s}, \qquad M \in \{0, 1\}^{B \times H \times T \times S}.$$

It is constant across heads and target queries but varies across batch and source position. A broadcast is correct only if it is equivalent to this full semantic tensor.

Part 3 - Hints, controls, and evidence

Keep BOS, EOS, PAD, and content ID ranges disjoint. Assert `decoder_ids[:, 1:] == labels[:, :-1]` only over positions where that relation is defined by the full target. Compare label count with the sum of the loss mask. In the mask test, materialize the expanded tensor once for auditing; efficiency is irrelevant at this size. Test batch-row isolation by changing padding in one row and checking other rows remain identical. Save the failing indices and expected versus observed token or visibility value. Include a known wrong shift and a transposed-mask injection so the audit itself is tested.

Part 4 - Python code skeleton

```
import torch

def build_shifted_targets(full_target, padding_id):
    decoder_ids = full_target[:, :-1].contiguous()
    labels = full_target[:, 1:].contiguous()
    loss_mask = labels.ne(padding_id)
    if decoder_ids.shape != labels.shape:
        raise AssertionError("decoder inputs and labels must align in shape")
    return decoder_ids, labels, loss_mask

def audit_shift(full_target, decoder_ids, labels):
    expected_decoder = full_target[:, :-1]
    expected_labels = full_target[:, 1:]
    if not torch.equal(decoder_ids, expected_decoder):
        wrong = (decoder_ids != expected_decoder).nonzero()
        raise AssertionError("wrong decoder shift at " + str(wrong.tolist()))
    if not torch.equal(labels, expected_labels):
        wrong = (labels != expected_labels).nonzero()
        raise AssertionError("wrong label shift at " + str(wrong.tolist()))

def audit_cross_mask(source_valid, heads, target_length):
    batch_size, source_length = source_valid.shape
    broadcast_mask = (~source_valid)[:, None, None, :]
    expanded = broadcast_mask.expand(
        batch_size, heads, target_length, source_length
    )
    for batch_index in range(batch_size):
        for target_index in range(target_length):
            observed = expanded[batch_index, 0, target_index]
            expected = ~source_valid[batch_index]
            if not torch.equal(observed, expected):
                raise AssertionError("cross-mask semantic mismatch")
    return broadcast_mask

def E05(full_target, source_valid, padding_id, heads):
    decoder_ids, labels, loss_mask = build_shifted_targets(
        full_target, padding_id
    )
    audit_shift(full_target, decoder_ids, labels)
    cross_mask = audit_cross_mask(source_valid, heads, decoder_ids.size(1))
    return decoder_ids, labels, loss_mask, cross_mask
```

Part 5 - The Philosophy Behind the Exercise

Alignment is an agreement between time, data, and supervision. Equal tensor dimensions can conceal a disagreement among all three. The discipline here is to prove meaning before optimization: which fact is available, which fact is predicted, and which positions may influence the decision. Learning cannot redeem a contradiction in the definition of the task.

17.9.7 Exercise 06

Exercise Title: Implement Attention Visualization for Cross-Attention and Interpret Which Source Tokens Drive Outputs

Part 1 - Why this matters

Cross-attention matrices expose how every decoder query distributes retrieval weight over source positions. On copy and reverse tasks, expected alignment is known, so visualization can reveal transposition, padding leakage, diffuse retrieval, repeated focus, or off-by-one target labeling. On natural language, the pattern can suggest alignments and head specialization. Yet attention is an internal routing weight, not guaranteed proof that a source token caused the final output. This exercise combines visualization with interventions.

Capture weights from every decoder layer and head in evaluation mode. Preserve shape (L, B, H, T, S) and attach the actual source tokens and predicted target tokens to the axes. Remove padded target rows and source columns only after checking their weights. Plot individual heads, mean-over-heads, and optionally rollups across layers; never average first and then claim all heads behaved the same. Normalize colors consistently across compared examples.

For a reverse task, target position zero should usually focus near the last source content position, then move backward. Quantify this rather than judging only the image: calculate peak source index, attention entropy, mass on the known aligned position, padding mass, and monotonic direction. Then intervene on a high-attention source token and a low-attention token. Measure the change in the chosen target logit. A head can have visually sharp weights but little causal effect after value projection, residual mixing, or later layers; state this limitation in the interpretation.

Part 2 - Mathematical form and reasoning

For layer ℓ and head h, cross-attention is

$$A_{t,s}^{(\ell,h)} = p(s \mid q_t), \qquad \sum_{s \in \mathcal{V}_x} A_{t,s}^{(\ell,h)} = 1.$$

Attention entropy is

$$H_t^{(\ell,h)} = -\sum_{s \in \mathcal{V}_x} A_{t,s}^{(\ell,h)} \log(A_{t,s}^{(\ell,h)} + \varepsilon).$$

For known alignment $a(t)$, alignment mass is $A_{t,a(t)}$. A simple token intervention effect for target logit $z_{t,v}$ is

$$\Delta_s = |z_{t,v}(x) - z_{t,v}(x_{s \leftarrow x'})|.$$

Attention weight and intervention effect answer different questions and should be reported together rather than equated.

Part 3 - Hints, controls, and evidence

Disable dropout, use a fixed checkpoint, and request weights before any head averaging. Verify axis labels with unique token IDs. Use the same color scale when comparing heads or checkpoints. Never let padding occupy plotted columns without a visible marker. For each target token, store predicted ID, reference ID, confidence, peak source position, aligned-position

mass, and entropy. Compare correct predictions with first-error positions. Run source-token replacement and source-token masking interventions while keeping the target prefix unchanged. Include examples where visualization is informative and where it is ambiguous; interpretation quality includes acknowledging uncertainty.

Part 4 - Python code skeleton

```
import torch

def cross_attention_statistics(weights, source_valid, alignment=None):
    visible = source_valid[:, None, None, :]
    valid_weights = weights.masked_fill(~visible, 0.0)
    padding_mass = weights.masked_fill(visible, 0.0).sum(dim=-1)
    probabilities = valid_weights.float().clamp_min(1e-12)
    entropy = -(probabilities * probabilities.log()).sum(dim=-1)
    peak_position = valid_weights.argmax(dim=-1)
    result = {
        "padding_mass_max": float(padding_mass.max().item()),
        "entropy": entropy.detach().cpu(),
        "peak_position": peak_position.detach().cpu(),
    }
    if alignment is not None:
        gather_index = alignment[:, None, :, None]
        gather_index = gather_index.expand(
            -1, weights.size(1), -1, 1
        )
        aligned_mass = valid_weights.gather(-1, gather_index).squeeze(-1)
        result["aligned_mass"] = aligned_mass.detach().cpu()
    return result

@torch.inference_mode()
def source_intervention(model, source_ids, decoder_ids, source_mask,
                        source_position, replacement_id):
    baseline_logits, baseline_weights = model.inspect(
        source_ids, decoder_ids, source_mask
    )
    changed = source_ids.clone()
    changed[:, source_position] = replacement_id
    changed_logits, changed_weights = model.inspect(
        changed, decoder_ids, source_mask
    )
    effect = (baseline_logits - changed_logits).abs().max(dim=-1).values
    return baseline_weights, changed_weights, effect

def E06(weights, source_valid, alignment):
    return cross_attention_statistics(weights, source_valid, alignment)
```

Part 5 - The Philosophy Behind the Exercise

Visualization is a question-generating instrument, not a verdict. It helps us see where the model routes information, then experiments test whether that route matters to the decision. The responsible interpretation of a complex model combines visibility with intervention and treats an attractive pattern as the beginning of explanation, never its end.

17.9.8 Exercise 07

Exercise Title: Compare Greedy vs Beam Search for Seq2Seq and Show When Beam Search Helps and When It Hurts

Part 1 - Why this matters

Greedy decoding chooses the most probable next token and discards every other continuation. Beam search preserves several partial hypotheses and may recover a better complete sequence when the locally best token leads to a poor future. This is especially relevant in conditional generation because source evidence can support several early target formulations. Yet beam search optimizes model score, not human usefulness, and wider beams can amplify length bias, generic phrasing, repetition, or model miscalibration.

Implement greedy and beam decoding over the same trained seq2seq model. Encode the source once, reuse memory for every hypothesis, and keep per-beam target tokens, cumulative log probability, completion state, and decoder cache if available. Finished beams must not generate ordinary tokens. When beams are reordered, every layer's cached state must be reordered identically. Begin with a hand-defined probability tree where greedy is provably suboptimal, then use the copy or reverse model where exact correctness is known.

Compare beam widths one, two, four, and eight. Beam width one must equal greedy under identical tie-breaking. Report raw log probability, normalized score, exact match, length, EOS rate, repetition, diversity among returned candidates, and latency. Include examples where beam search corrects an early greedy choice and where it selects a shorter or blander output despite a wider search. The conclusion should identify which failure belongs to search, which belongs to the model distribution, and which arises from the scoring objective.

Part 2 - Mathematical form and reasoning

For a partial hypothesis $y_{1:t}$,

$$s(y_{1:t}) = \sum_{i=1}^{t} \log p_\theta(y_i \mid y_{<i}, x).$$

At each step beam search forms all allowed one-token extensions and retains the top B_w by search score. Because each log probability is nonpositive, raw score tends to favor short completed sequences. A later exercise introduces length penalty; here report raw scores so this bias is visible. Search error exists when a better model-scored sequence was pruned. Model error exists when the highest-scored sequence is not the desired answer.

Part 3 - Hints, controls, and evidence

Use log-softmax to avoid multiplying small probabilities. Define deterministic tie-breaking. Keep completed and active hypotheses conceptually separate. Apply minimum length, EOS, and maximum length identically across methods. Verify beam one equals greedy token for token and that every beam shares the same encoder memory without recomputing it. If batched

beams are used, test cache reordering against a slow full-prefix reference. Evidence includes the hand-computed tree, beam frontier at every step, ranked final hypotheses, task metrics, and latency. Do not claim that beam search improves quality from one favorable example.

Part 4 - Python code skeleton

```
import torch

@torch.inference_mode()
def seq2seq_beam_search(model, source_ids, source_mask, bos_id, eos_id,
                        beam_width, maximum_tokens):
    model.eval()
    memory = model.encode(source_ids, source_mask)
    initial = torch.tensor([[bos_id]], device=source_ids.device)
    beams = [{"tokens": initial, "score": 0.0, "finished": False}]

    for step in range(maximum_tokens):
        candidates = []
        for beam in beams:
            if beam["finished"]:
                candidates.append(beam)
                continue
            logits = model.decode(beam["tokens"], memory, source_mask)
            log_probabilities = torch.log_softmax(logits[:, -1, :], dim=-1)
            values, indices = torch.topk(
                log_probabilities, beam_width, dim=-1
            )
            for candidate_index in range(beam_width):
                token = indices[:, candidate_index:candidate_index + 1]
                candidate_tokens = torch.cat((beam["tokens"], token), dim=1)
                token_score = float(values[0, candidate_index].item())
                candidates.append({
                    "tokens": candidate_tokens,
                    "score": beam["score"] + token_score,
                    "finished": int(token.item()) == eos_id,
                })
        def raw_score(candidate):
            return candidate["score"]
        candidates.sort(key=raw_score, reverse=True)
        beams = candidates[:beam_width]
        all_finished = True
        for beam in beams:
            if not beam["finished"]:
                all_finished = False
        if all_finished:
            break
    return beams

def E07(model, source_ids, source_mask, bos_id, eos_id):
```

```
    return seq2seq_beam_search(
        model, source_ids, source_mask, bos_id, eos_id, 4, 64
    )
```

Part 5 - The Philosophy Behind the Exercise

Search cannot create knowledge that the model did not assign probability to, and it cannot know that our scoring rule misrepresents value. Beam search is a method for delaying commitment within a declared objective. Wider deliberation is useful only when the objective deserves to be pursued more thoroughly.

17.9.9 Exercise 08

Exercise Title: Implement Length Penalty in Beam Search and Show Its Effect on Too-Short Output Bias

Part 1 - Why this matters

Cumulative log probability becomes more negative whenever another token is added. Consequently, raw beam score can prefer a short hypothesis that emits EOS early even when a longer output better represents the source. Length penalty changes ranking so sequences of different lengths can be compared under a declared normalization. It does not repair missing source information or poor probabilities; it changes the search objective and can overshoot into verbosity.

Extend the previous beam search while preserving raw cumulative log probability for audit. Rank hypotheses using a separate normalized score. Compare no penalty with several strengths on sources of different reference lengths. Record output length, length ratio, EOS rate, exact match or task metric, raw score, normalized score, and latency. Show at least one case in which the penalty rescues a truncated output and one where excessive normalization keeps an unnecessary continuation alive.

Be precise about which length is counted. Exclude the source, usually exclude BOS, and state whether EOS counts. Apply the same convention during intermediate pruning and final ranking. Do not normalize unfinished and finished beams under different undocumented rules. A minimum-length constraint is not a length penalty: it prohibits EOS before a threshold, whereas a penalty continuously changes preferences among allowed hypotheses.

Part 2 - Mathematical form and reasoning

A simple normalized score is

$$s_\alpha(y) = \frac{s(y)}{|y|^\alpha},$$

where $s(y)$ is cumulative log probability and $\alpha \geq 0$. Because $s(y)$ is negative, dividing by a larger positive denominator makes longer sequences less severely penalized. Another common form is

$$\mathrm{lp}_\alpha(|y|) = \left(\frac{5 + |y|}{6}\right)^\alpha, \qquad s_\alpha(y) = \frac{s(y)}{\mathrm{lp}_\alpha(|y|)}.$$

The choice is part of the experiment. At $\alpha = 0$, ranking reduces exactly to raw beam score.

Part 3 - Hints, controls, and evidence

Implement the scoring function independently and unit-test it on hand-written hypotheses. Ensure an alpha of zero preserves ordering. Log raw and normalized scores together so the ranking change is explainable. Use matched beam width, maximum length, and prompts. Report length histograms, not only averages, and stratify by source length. Inspect early EOS probability at the step where an unpenalized beam terminates. Tune on validation data and evaluate once on held- out data; choosing alpha from test BLEU is leakage. Evidence must show the score calculation for at least one ranking reversal.

Part 4 - Python code skeleton

```
def generated_length(tokens, bos_id, eos_id):
    count = 0
    token_values = tokens[0].tolist()
    for token in token_values:
        if token == bos_id:
            continue
        count += 1
        if token == eos_id:
            break
    return max(count, 1)

def length_penalty(length, alpha):
    return ((5.0 + float(length)) / 6.0) ** alpha

def normalized_beam_score(beam, bos_id, eos_id, alpha):
    length = generated_length(beam["tokens"], bos_id, eos_id)
    penalty = length_penalty(length, alpha)
    return beam["raw_score"] / penalty

def rank_beams(beams, bos_id, eos_id, alpha):
    scored = []
    for beam in beams:
        copied = dict(beam)
        copied["normalized_score"] = normalized_beam_score(
            beam, bos_id, eos_id, alpha
        )
        scored.append(copied)
    def score_value(item):
        return item["normalized_score"]
    scored.sort(key=score_value, reverse=True)
    return scored

def E08(beams, bos_id, eos_id):
    results = {}
    for alpha in (0.0, 0.4, 0.8, 1.2):
        results[alpha] = rank_beams(beams, bos_id, eos_id, alpha)
    return results
```

Part 5 - The Philosophy Behind the Exercise

Every decoding score contains a theory of what a good sequence is. Length penalty makes one hidden preference visible and adjustable. Its deeper lesson is that inference is not neutral: once competing sequences must be ranked, an engineering choice becomes a value judgment expressed in mathematics.

17.9.10 Exercise 09

Exercise Title: Build a Seq2Seq Evaluation Pipeline Using BLEU Conceptually and Explain What BLEU Measures and Misses

Part 1 - Why this matters

BLEU evaluates overlap between generated and reference sequences using modified n-gram precision and a penalty for outputs that are too short. It is useful for reproducible corpus-level comparison, especially in translation-like tasks, but it does not directly measure truth, semantic equivalence, discourse quality, or usefulness. A fluent paraphrase may receive a low score; a harmful or incorrect sentence can share many n-grams and score well. This exercise builds the metric transparently and surrounds it with complementary evidence.

Normalize, tokenize, and detokenize predictions and references under a frozen policy. Keep special tokens and padding out of the metric. Implement clipped n-gram counts: a prediction cannot receive more credit for an n-gram than the maximum count present in its references. Aggregate counts over the corpus before taking the geometric mean. Handle zero counts with an explicit smoothing policy and report that policy. Apply the brevity penalty using total candidate and effective reference lengths.

Evaluate the same outputs with BLEU, exact match for synthetic tasks, token accuracy, length ratio, EOS success, repetition, and a small error taxonomy. Construct counterexamples: identical meaning with different wording, repeated high-overlap fragments, correct words in wrong order, and a short output that matches a common prefix. Explain how BLEU responds. Do not use sentence BLEU as if it had the stability of a sufficiently large corpus score, and do not compare numbers produced by different tokenization or smoothing conventions.

Part 2 - Mathematical form and reasoning

Modified precision for order n is

$$p_n = \frac{\sum_{g \in G_n(\hat{Y})} \min(c_{\hat{Y}}(g), \max_r c_{Y^{(r)}}(g))}{\sum_{g \in G_n(\hat{Y})} c_{\hat{Y}}(g)}.$$

For candidate corpus length c and effective reference length r, brevity penalty is

$$\mathrm{BP} = \begin{cases} 1, & c > r, \\ \exp(1 - r/c), & c \le r. \end{cases}$$

Then

$$\mathrm{BLEU} = \mathrm{BP} \exp\left(\sum_{n=1}^{N} w_n \log p_n \right).$$

The geometric mean requires performance across orders; a zero precision can collapse the unsmoothed score.

Part 3 - Hints, controls, and evidence

Unit-test n-gram extraction and clipping on lists short enough to count by hand. Preserve case and punctuation policy in the report. Use multiple references when the task genuinely permits multiple valid expressions. Bootstrap the evaluation set or report variation across subsets to avoid treating a small score difference as certainty. Store per-example outputs for error analysis but compute corpus BLEU from aggregated counts. Compare against a trusted metric implementation only after the manual version passes. Evidence should contain manual examples, corpus counts for every n, brevity penalty, final score, and the complementary metrics that reveal BLEU's blind spots.

Part 4 - Python code skeleton

```
import math
from collections import Counter

def extract_ngrams(tokens, order):
    counts = Counter()
    final_start = len(tokens) - order
    for start in range(final_start + 1):
        ngram = tuple(tokens[start:start + order])
        counts[ngram] += 1
    return counts

def clipped_counts(candidate, references, order):
    candidate_counts = extract_ngrams(candidate, order)
    maximum_reference_counts = Counter()
    for reference in references:
        reference_counts = extract_ngrams(reference, order)
        for ngram, count in reference_counts.items():
            previous = maximum_reference_counts[ngram]
            if count > previous:
                maximum_reference_counts[ngram] = count
    clipped = 0
    total = 0
    for ngram, count in candidate_counts.items():
        clipped += min(count, maximum_reference_counts[ngram])
        total += count
    return clipped, total

def brevity_penalty(candidate_length, reference_length):
    if candidate_length == 0:
        return 0.0
    if candidate_length > reference_length:
        return 1.0
    return math.exp(1.0 - reference_length / candidate_length)
```

```
def E09(candidate, references):
    precisions = []
    for order in range(1, 5):
        clipped, total = clipped_counts(candidate, references, order)
        precisions.append((clipped + 1.0) / (total + 1.0))
    reference_length = len(references[0])
    log_precision_sum = 0.0
    for value in precisions:
        log_precision_sum += math.log(value)
    score = brevity_penalty(len(candidate), reference_length)
    score *= math.exp(log_precision_sum / 4.0)
    return score, precisions
```

Replace the single-reference length rule with a documented closest-reference rule when multiple references are used at corpus scale.

Part 5 - The Philosophy Behind the Exercise

A metric is a compressed question. BLEU asks whether local pieces of generated language resemble reference pieces while discouraging extreme brevity. It does not ask whether the answer is true or sufficient. Scientific evaluation begins when we understand the question a number answers and refuse to make it testify about questions it was never designed to address.

17.9.11 Exercise 10

Exercise Title: Run Full Seq2Seq Training on CUDA and Benchmark Runtime, Memory, and Throughput for Different Sequence Lengths

Part 1 - Why this matters

Seq2seq cost depends on two lengths rather than one. Encoder self-attention scales with source length, decoder self-attention with target length, and cross- attention with their product. A benchmark that varies only batch size cannot explain which component exhausts memory or reduces throughput. Furthermore, asynchronous CUDA execution makes ordinary wall timing misleading unless device work is synchronized at measurement boundaries.

Move model, source IDs, target IDs, masks, labels, and any positional buffers to one CUDA device. Use pinned host memory and nonblocking transfers only after verifying the input pipeline can benefit. Separate data wait and transfer, forward, loss, backward, optimizer, and generation phases. Warm up kernels and allocator state, then measure a steady window. Record valid source tokens and valid target tokens rather than padded tensor elements, because padding can inflate apparent throughput.

Build a grid over source lengths, target lengths, and batch sizes. Change one axis at a time, and include both uniform-length and mixed-length batches. Record steps per second, valid target tokens per second, examples per second, peak allocated memory, reserved memory, and out-of-memory boundary. For inference, separate encoder time, decoder prefill, and per-token autoregressive decode. Synchronize around timings but remove unnecessary synchronization from the real training loop. Compare CPU only on configurations for which it is meaningful, and state hardware and software versions.

Part 2 - Mathematical form and reasoning

Ignoring constants and feed-forward work, attention score complexity is

$$\mathcal{O}(BS^2d) + \mathcal{O}(BT^2d) + \mathcal{O}(BSTd).$$

The three terms correspond to encoder self-attention, decoder self-attention, and cross-attention. Useful target throughput is

$$\tau_y = \frac{\sum_{i \in \text{measured steps}} N_{y,\text{valid}}^{(i)}}{t_{\text{measured}}}.$$

Padding efficiency is $\eta = N_{\text{valid}}/N_{\text{padded}}$. Report it with throughput so a faster batch is not credited merely for processing more pads.

Part 3 - Hints, controls, and evidence

Use identical model weights, precision, optimizer, and number of measured steps across comparable configurations. Reset peak memory immediately before the measured region. Exclude compilation and first allocation from steady-state numbers but report startup separately. Use CUDA events for GPU regions and a synchronized wall timer for end-to-end latency. Detect nonfinite loss and skip no batches silently. Record the exact configuration that produces OOM and the largest safe setting. Validate that increasing padding lowers useful-token efficiency even if raw tensor throughput looks stable. Evidence is a table and scaling curves, followed by a bottleneck explanation grounded in profiles.

Part 4 - Python code skeleton

```
import time
import torch

def timed_training_step(model, optimizer, batch, loss_function, device):
    source = batch["source"].to(device, non_blocking=True)
    decoder = batch["decoder"].to(device, non_blocking=True)
    labels = batch["labels"].to(device, non_blocking=True)
    source_padding = batch["source_padding"].to(device, non_blocking=True)
    target_padding = batch["target_padding"].to(device, non_blocking=True)

    start_event = torch.cuda.Event(enable_timing=True)
    end_event = torch.cuda.Event(enable_timing=True)
    optimizer.zero_grad(set_to_none=True)
    start_event.record()
    logits = model(source, decoder, source_padding, target_padding)
    flat_logits = logits.reshape(-1, logits.size(-1))
    flat_labels = labels.reshape(-1)
    loss = loss_function(flat_logits, flat_labels)
    loss.backward()
    optimizer.step()
    end_event.record()
    torch.cuda.synchronize(device)
    milliseconds = start_event.elapsed_time(end_event)
    valid_tokens = int(labels.ne(loss_function.ignore_index).sum().item())
```

```
    return float(loss.item()), milliseconds / 1000.0, valid_tokens

def benchmark_configuration(model, optimizer, batches, loss_function, device):
    model.train()
    torch.cuda.reset_peak_memory_stats(device)
    total_seconds = 0.0
    total_tokens = 0
    losses = []
    for batch in batches:
        loss, seconds, valid_tokens = timed_training_step(
            model, optimizer, batch, loss_function, device
        )
        losses.append(loss)
        total_seconds += seconds
        total_tokens += valid_tokens
    return {
        "mean_loss": sum(losses) / len(losses),
        "valid_tokens_per_second": total_tokens / total_seconds,
        "peak_allocated_bytes": torch.cuda.max_memory_allocated(device),
        "peak_reserved_bytes": torch.cuda.max_memory_reserved(device),
    }

def E10(model, optimizer, batches, loss_function):
    device = torch.device("cuda:0")
    model.to(device)
    return benchmark_configuration(
        model, optimizer, batches, loss_function, device
    )
```

Part 5 - The Philosophy Behind the Exercise

Performance is a property of a workload, not a device label. The GPU accelerates specific tensor structures under specific sizes and data conditions. Honest benchmarking reveals the geometry of cost: which length creates the burden, which work is useful, and where speed is being reported for computation the task never needed.

17.9.12 Exercise 11

Exercise Title: Implement Mixed Precision Seq2Seq Training and Identify Instability Cases in Cross-Attention Softmax

Part 1 - Why this matters

Mixed precision can reduce activation memory and accelerate matrix operations, allowing larger batches or sequences. Seq2seq Transformers also contain fragile operations: large query-key scores, masked softmax, normalization, exponentials, and small gradients. A run that is faster but occasionally emits NaNs, changes mask semantics, or silently skips many optimizer steps is not a successful optimization.

Build a float32 reference and an automatic mixed-precision run from identical weights and batches. Use autocast for supported compute and gradient scaling when float16 requires it. Keep attention logits and softmax in float32 in the scratch implementation, then cast normalized weights back to the value dtype. Apply masks before softmax with a representable value appropriate to the dtype, and explicitly reject fully masked rows. Log scaler value, skipped steps, nonfinite gradients, attention-score range, maximum cross-attention probability, entropy, loss difference, and output parity.

Stress source and target length, weight scale, learning rate, and mask sparsity. Construct extreme query and key tensors to reproduce overflow in a naive float16 softmax, then prove stable float32 softmax prevents it. Compare bfloat16 where supported because its exponent range differs from float16. Evaluate generated sequences as well as training loss: small logit changes can alter an early token and create a large autoregressive divergence.

Part 2 - Mathematical form and reasoning

The stable softmax uses

$$\operatorname{softmax}(z)_i = \frac{\exp(z_i - m)}{\sum_j \exp(z_j - m)}, \qquad m = \max_j z_j.$$

Subtracting m prevents positive exponential overflow, but masked values, nonfinite dot products, and insufficient accumulation precision still require care. Gradient scaling computes

$$\nabla_\theta(s\mathcal{L}) = s\nabla_\theta\mathcal{L}$$

in the backward pass and unscales before clipping or updating. If nonfinite gradients are found, the optimizer step is skipped and the scale is reduced.

Part 3 - Hints, controls, and evidence

Confirm the accelerator supports the selected dtype. Keep one fixed batch for step-level parity and a longer run for stability. Unscale before gradient norm measurement and clipping. Never call gradient clipping on scaled gradients. Monitor each attention layer separately to identify the first nonfinite tensor. Use identical masks and seeds; dropout complicates exact comparison, so begin in evaluation mode. Report speed, peak memory, final quality, skipped-step count, and the largest safe sequence or score range. Demonstrate at least one injected failure and its correction rather than reporting only a stable default.

Part 4 - Python code skeleton

```
import torch

def stable_attention_softmax(scores, blocked_mask):
    if blocked_mask.shape[-1] != scores.shape[-1]:
        raise ValueError("mask must address the key sequence axis")
    masked_scores = scores.float().masked_fill(blocked_mask, float("-inf"))
    fully_blocked = blocked_mask.all(dim=-1)
    if bool(fully_blocked.any()):
        raise ValueError("attention contains a fully blocked query row")
    probabilities = torch.softmax(masked_scores, dim=-1)
    if not bool(torch.isfinite(probabilities).all()):
```

```
        raise FloatingPointError("nonfinite attention probabilities")
    return probabilities.to(scores.dtype)

def mixed_precision_step(model, optimizer, scaler, batch, loss_function,
                         device, autocast_dtype):
    optimizer.zero_grad(set_to_none=True)
    with torch.autocast(device_type="cuda", dtype=autocast_dtype):
        logits = model.forward_batch(batch, device)
        labels = batch["labels"].to(device, non_blocking=True)
        loss = loss_function(
            logits.reshape(-1, logits.size(-1)), labels.reshape(-1)
        )
    if not bool(torch.isfinite(loss)):
        raise FloatingPointError("nonfinite mixed-precision loss")
    scaler.scale(loss).backward()
    scaler.unscale_(optimizer)
    gradient_norm = torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    previous_scale = scaler.get_scale()
    scaler.step(optimizer)
    scaler.update()
    new_scale = scaler.get_scale()
    skipped = new_scale < previous_scale
    return float(loss.item()), float(gradient_norm), skipped, new_scale

def E11(model, optimizer, batch, loss_function):
    device = torch.device("cuda:0")
    scaler = torch.amp.GradScaler("cuda")
    return mixed_precision_step(
        model, optimizer, scaler, batch, loss_function,
        device, torch.float16
    )
```

Part 5 - The Philosophy Behind the Exercise

Precision is a budget for representing distinctions. Mixed precision spends that budget selectively to gain speed and capacity. The correct question is not whether half precision runs, but whether every discarded numerical distinction is irrelevant to learning and generation. Optimization becomes trustworthy only when its approximation is bounded by evidence.

17.9.13 Exercise 12

Exercise Title: Implement Label Smoothing for Seq2Seq and Measure Its Effect on Generalization and Calibration

Part 1 - Why this matters

One-hot cross-entropy demands that all target probability move to one vocabulary item. In tasks with limited data, annotation ambiguity, or several acceptable phrases, this can encourage excessive confidence and brittle generalization. Label smoothing assigns most probability to the reference token and a small amount to alternatives. It can improve validation performance

and calibration, but too much smoothing weakens the learning signal and may harm exact tasks such as copying.

Implement the loss manually before using a built-in option. Exclude padding positions completely. Decide whether smoothing mass is spread across all classes or only non-reference classes, and state the convention. If PAD should never be generated, decide whether it belongs to the smoothed support; do not let an accidental vocabulary convention teach probability for padding. Compare the manual result with a trusted implementation on a small tensor.

Train matched models with several smoothing values using identical seeds, updates, and data. Measure training negative log-likelihood, validation NLL, token accuracy, exact match or BLEU, confidence, entropy, expected calibration error, and reliability diagrams. A smoothed training objective is not directly comparable to unsmoothed NLL, so evaluate all models with the same unsmoothed metric. Inspect correct and incorrect token confidence separately and test distribution shift or mild label noise.

Part 2 - Mathematical form and reasoning

For vocabulary size V, reference class y, and smoothing ε, one non-reference convention is

$$q_k = \begin{cases} 1-\varepsilon, & k = y, \\ \varepsilon/(V-1), & k \neq y. \end{cases}$$

The loss is

$$\mathcal{L}_{\mathrm{LS}} = -\sum_{k=1}^{V} q_k \log p_k.$$

Calibration compares confidence with empirical accuracy. For confidence bins B_m, expected calibration error is

$$\mathrm{ECE} = \sum_m \frac{|B_m|}{N} |\operatorname{acc}(B_m) - \operatorname{conf}(B_m)|.$$

Lower ECE is useful only alongside task quality; a uniformly uncertain model may be calibrated yet unhelpful.

Part 3 - Hints, controls, and evidence

Unit-test epsilon zero against ordinary cross-entropy. Verify each constructed target distribution sums to one and has no negative entry. Apply the valid-token mask before reduction and divide by the exact valid count. Keep validation scoring unsmoothed. Use sufficiently populated calibration bins and report bin counts. Compare confidence at the first autoregressive error, because exposure can amplify overconfidence. Evidence includes loss parity tests, quality and calibration tables, reliability data, and a clear choice of epsilon based only on validation results.

Part 4 - Python code skeleton

```
import torch

def label_smoothed_loss(logits, labels, padding_id, epsilon):
```

```
    vocabulary_size = logits.size(-1)
    flat_logits = logits.reshape(-1, vocabulary_size).float()
    flat_labels = labels.reshape(-1)
    valid = flat_labels.ne(padding_id)
    valid_logits = flat_logits[valid]
    valid_labels = flat_labels[valid]
    log_probabilities = torch.log_softmax(valid_logits, dim=-1)

    reference_log_probability = log_probabilities.gather(
        1, valid_labels[:, None]
    ).squeeze(1)
    total_log_probability = log_probabilities.sum(dim=-1)
    nonreference_log_probability = total_log_probability
    nonreference_log_probability = (
        nonreference_log_probability - reference_log_probability
    )
    reference_loss = -(1.0 - epsilon) * reference_log_probability
    nonreference_loss = -epsilon * nonreference_log_probability
    nonreference_loss = nonreference_loss / (vocabulary_size - 1)
    return (reference_loss + nonreference_loss).mean()

def calibration_bins(probabilities, labels, padding_id, number_of_bins):
    confidence, prediction = probabilities.max(dim=-1)
    valid = labels.ne(padding_id)
    confidence = confidence[valid]
    correct = prediction[valid].eq(labels[valid])
    rows = []
    for bin_index in range(number_of_bins):
        lower = bin_index / number_of_bins
        upper = (bin_index + 1) / number_of_bins
        members = (confidence > lower) & (confidence <= upper)
        count = int(members.sum().item())
        if count > 0:
            rows.append((count, float(confidence[members].mean().item()),
                         float(correct[members].float().mean().item())))
    return rows

def E12(logits, labels, padding_id):
    return label_smoothed_loss(logits, labels, padding_id, epsilon=0.1)
```

Part 5 - The Philosophy Behind the Exercise

Certainty is not the same as knowledge. Label smoothing asks the learner to reserve a small space for alternatives, but that humility must match the task. In an exact symbolic transformation, ambiguity may be artificial; in language, it may be honest. Regularization is strongest when it expresses a defensible belief about the world rather than a fashionable default.

17.9.14 Exercise 13

Exercise Title: Implement Scheduled Sampling Conceptually and Compare It Against Teacher Forcing for Exposure Bias

Part 1 - Why this matters

Teacher forcing conditions every training prediction on a correct target prefix, while inference conditions later predictions on the model's earlier choices. This mismatch is exposure bias: the model is rarely trained to recover from the states created by its own mistakes. Scheduled sampling sometimes replaces a reference decoder input with a model-generated token during training. It exposes the model to its own trajectory, but changes the objective, reduces parallelism, and can propagate low-quality early predictions.

Implement a controlled autoregressive training loop on a small seq2seq task. At target step t, compute logits from the current mixed prefix, select or sample a model token, and use a Bernoulli decision to choose between that token and the reference token for the next input. Never replace BOS. Do not overwrite the label: supervision still asks for the reference next token. Detach selected discrete tokens so the code does not imply a differentiable path through the sampling decision.

Compare constant teacher forcing with linear or inverse-sigmoid schedules. Log the realized model-token fraction, not only the requested probability. Measure teacher-forced validation loss, autoregressive exact match or BLEU, recovery after an injected wrong token, training time, and variance across seeds. Begin scheduled sampling only after the model has learned basic alignment; otherwise the decoder may train mostly on meaningless prefixes. Discuss the method's objective inconsistency rather than presenting it as a universal correction.

Part 2 - Mathematical form and reasoning

Let $r_t \sim \text{Bernoulli}(\rho_e)$ at epoch e. The next decoder input is

$$\widetilde{y}_t = \begin{cases} y_t, & r_t = 0, \\ \widehat{y}_t, & r_t = 1, \end{cases}$$

where $\widehat{y}_t$ is chosen from the model distribution conditioned on the mixed prefix. A linear schedule might be

$$\rho_e = \min(\rho_{\max}, \max(0, (e - e_0)\Delta\rho)).$$

Because the distribution of training prefixes now depends on the current model, the expected objective differs from maximum likelihood under clean data.

Part 3 - Hints, controls, and evidence

Use an explicit seeded generator for Bernoulli decisions and token sampling. Apply decisions only to active non-padding rows. Decide whether EOS-completed rows receive PAD or EOS thereafter. Compare greedy replacement with sampled replacement. Match update count and compute budget when possible, and report the extra sequential cost. Test recovery by forcing one incorrect decoder token at a known position and measuring whether later outputs realign. Record schedule, realized replacement rate, loss, autoregressive quality, and seed variation. Include a case where scheduled sampling harms performance; negative evidence is important for selecting rather than merely demonstrating the technique.

Part 4 - Python code skeleton

```
import torch

def scheduled_sampling_probability(epoch, start_epoch, increment, maximum):
    if epoch < start_epoch:
        return 0.0
    value = (epoch - start_epoch) * increment
    return min(maximum, value)

def build_mixed_decoder_input(model, memory, source_mask, full_target,
                              replacement_probability, generator):
    decoder = full_target[:, :1]
    target_steps = full_target.size(1) - 1
    decisions = []
    for step in range(target_steps - 1):
        logits = model.decode(decoder, memory, source_mask)
        model_token = logits[:, -1, :].argmax(dim=-1).detach()
        reference_token = full_target[:, step + 1]
        random_values = torch.rand(
            reference_token.shape, device=reference_token.device,
            generator=generator
        )
        use_model = random_values < replacement_probability
        chosen = torch.where(use_model, model_token, reference_token)
        decoder = torch.cat((decoder, chosen[:, None]), dim=1)
        decisions.append(use_model)
    return decoder, decisions

def E13(model, source_ids, source_mask, full_target, epoch, generator):
    model.train()
    memory = model.encode(source_ids, source_mask)
    probability = scheduled_sampling_probability(
        epoch, start_epoch=5, increment=0.05, maximum=0.5
    )
    decoder, decisions = build_mixed_decoder_input(
        model, memory, source_mask, full_target, probability, generator
    )
    return decoder, decisions, probability
```

Part 5 - The Philosophy Behind the Exercise

Training creates the world in which a model learns to act. Teacher forcing gives perfect history; inference gives consequences. Scheduled sampling asks whether practice should contain some of those consequences. Its philosophical warning is equally important: realism introduced without control can become noise, and exposure to failure is useful only when the learner has a path to recovery.

17.9.15 Exercise 14

Exercise Title: Profile GPU Kernels and Identify Whether Encoder or Decoder Dominates Cost for Your Task

Part 1 - Why this matters

An encoder-decoder model has several computational phases whose dominance changes with workload. A long source and short target may make the encoder largest; long teacher-forced targets add decoder self-attention and cross- attention; autoregressive generation repeatedly invokes the decoder while the encoder normally runs once. Optimizing the architecture without measuring the actual phase can accelerate a component that contributes little to user-visible latency.

Instrument encoder forward, decoder self-attention, cross-attention, decoder feed-forward layers, loss, backward, optimizer, encoder prefill, and each generation step. Use named profiler ranges and collect CPU and CUDA activity, shapes, memory, kernel counts, and call stacks for a short representative window. Profile training and inference separately. Detailed profiling adds overhead, so use it for attribution and use lightweight CUDA-event timing for final performance numbers.

Repeat across source-target length pairs, batch sizes, and teacher-forced versus autoregressive modes. Aggregate kernel time by phase rather than listing only the largest individual matrix multiplication. Identify CPU gaps, synchronization, padding waste, repeated encoder computation, and small decoder kernels. Make one targeted change, such as bucketing lengths, reusing encoder memory, compiling a block, or increasing generation batch size, then re-profile and report end-to- end improvement. A faster isolated kernel is not evidence of a faster service.

Part 2 - Mathematical form and reasoning

Total training time can be decomposed as

$$t_{\text{train}} = t_{\text{data}} + t_{\text{enc}} + t_{\text{dec,self}} + t_{\text{cross}} + t_{\text{FFN}} + t_{\text{loss}} + t_{\text{backward}} + t_{\text{optimizer}} + t_{\text{sync}} - t_{\text{overlap}}.$$

Autoregressive inference is approximately

$$t_{\text{request}} = t_{\text{encode}} + t_{\text{start}} + \sum_{u=1}^{U} t_{\text{decode}}(u) + t_{\text{control}}.$$

The phase share $r_i = t_i / t_{\text{request}}$ identifies the dominant non-overlapped cost. The sum also shows why decoder work can dominate even when one decoder step is cheaper than encoding.

Part 3 - Hints, controls, and evidence

Warm up before profiling, keep the measured schedule short, and avoid printing inside it. Synchronize only at deliberate measurement boundaries. Label phases with `record_function`; otherwise kernels from encoder and decoder may be indistinguishable. Use a fixed batch and repeat profiles for stability. Report both self CUDA time and total CUDA time, plus kernel launch counts and peak memory. Confirm encoder memory is computed once during generation. If the CPU is the bottleneck, GPU kernel percentages alone are incomplete. Evidence is a phase table before and after one intervention, with unchanged output parity and quality metrics.

Part 4 - Python code skeleton

```
import torch
from torch.profiler import ProfilerActivity, profile, record_function

def profiled_seq2seq_step(model, batch, optimizer, loss_function, device):
    source = batch["source"].to(device)
    decoder = batch["decoder"].to(device)
    labels = batch["labels"].to(device)
    source_mask = batch["source_mask"].to(device)
    optimizer.zero_grad(set_to_none=True)

    with record_function("encoder"):
        memory = model.encode(source, source_mask)
    with record_function("decoder"):
        logits = model.decode(decoder, memory, source_mask)
    with record_function("loss"):
        loss = loss_function(
            logits.reshape(-1, logits.size(-1)), labels.reshape(-1)
        )
    with record_function("backward"):
        loss.backward()
    with record_function("optimizer"):
        optimizer.step()
    return loss

def E14(model, batch, optimizer, loss_function):
    device = torch.device("cuda:0")
    activities = [ProfilerActivity.CPU, ProfilerActivity.CUDA]
    with profile(
        activities=activities, record_shapes=True, profile_memory=True
    ) as profiler:
        loss = profiled_seq2seq_step(
            model, batch, optimizer, loss_function, device
        )
    table = profiler.key_averages().table(
        sort_by="self_cuda_time_total", row_limit=30
    )
    return float(loss.item()), table
```

Part 5 - The Philosophy Behind the Exercise

Profiling disciplines attention in the human sense: it directs effort toward the part of reality that controls the outcome. Architecture diagrams distribute visual importance evenly; execution does not. Optimization begins when measured time, memory, and dependency replace intuition as the map of the system.

17.9.16 Exercise 15

Exercise Title: Implement Checkpointing and Resume for Seq2Seq Training and Verify State Consistency

Part 1 - Why this matters

A checkpoint is a promise that interrupted training can continue with the same meaning. Saving model weights alone permits fine-tuning, but it does not resume an optimizer trajectory. Adam moments, scheduler position, gradient scaler, completed epoch and batch, random-number states, sampler order, configuration, tokenizer identity, and data split all influence the next update. Losing one can create a silent branch in the experiment while the program prints "resumed."

Define a versioned checkpoint schema. Save model, optimizer, scheduler, scaler, global update, epoch, batch position or sampler state, Python and PyTorch RNG states, CUDA RNG states, model/data/tokenizer configuration, validation history, and the best-checkpoint criterion. Write to a temporary path and atomically rename only after serialization succeeds. Store a checksum and reject corrupt or incompatible files before mutating live objects. Never load untrusted pickle- based checkpoints.

Prove exact continuation with a fork test. Train a deterministic tiny seq2seq model for K updates and save. Branch A continues for M updates without interruption. Branch B starts from a fresh process or fresh objects, restores, and consumes the same next batches for M updates. Compare learning rates, losses, scaler values, model parameters, optimizer tensors, scheduler state, and generated outputs after every step. If exact equality is unsupported on the chosen CUDA path, document the deterministic limits and use justified numerical tolerances rather than claiming exactness.

Part 2 - Mathematical form and reasoning

Training state is more than parameters:

$$S_k = (\theta_k, O_k, R_k, G_k, \mathcal{R}_k, D_k, C),$$

where O is optimizer state, R scheduler state, G gradient-scaler state, $\mathcal{R}$ random state, D data progress, and C immutable configuration. A correct resume satisfies

$$U(S_k, B_{k+1}) = S_{k+1}$$

for uninterrupted and restored executions under the same batch B_{k+1}. Measure maximum parameter error and compare discrete outputs because a small numeric deviation may later alter decoding.

Part 3 - Hints, controls, and evidence

Construct the model and optimizer before loading their states, then move optimizer tensors to the intended device if required. Restore RNG after object initialization that consumes randomness. Validate vocabulary size, special IDs, architecture, and scheduler definition before continuing. Save at a clear boundary after an optimizer step, and define whether the recorded batch is last completed or next to consume. Test missing file, truncated file, mismatched configuration, CPU load of CUDA checkpoint, and scaler absence. Evidence includes schema, checksum, fork-test traces, maximum state differences, and an intentional incomplete-checkpoint failure.

Part 4 - Python code skeleton

```
import os
import random
import torch

def capture_training_state(model, optimizer, scheduler, scaler,
                           epoch, global_step, configuration):
    cuda_random_state = None
    if torch.cuda.is_available():
        cuda_random_state = torch.cuda.get_rng_state_all()
    state = {
        "schema_version": 1,
        "model": model.state_dict(),
        "optimizer": optimizer.state_dict(),
        "scheduler": scheduler.state_dict(),
        "scaler": scaler.state_dict() if scaler is not None else None,
        "epoch": epoch,
        "global_step": global_step,
        "configuration": configuration,
        "python_rng": random.getstate(),
        "torch_rng": torch.get_rng_state(),
        "cuda_rng": cuda_random_state,
    }
    return state

def atomic_save(state, path):
    temporary_path = path + ".temporary"
    torch.save(state, temporary_path)
    os.replace(temporary_path, path)

def restore_training_state(path, model, optimizer, scheduler, scaler,
                           expected_configuration, device):
    checkpoint = torch.load(path, map_location=device, weights_only=False)
    if checkpoint["schema_version"] != 1:
        raise ValueError("unsupported checkpoint schema")
    if checkpoint["configuration"] != expected_configuration:
        raise ValueError("checkpoint configuration mismatch")
    model.load_state_dict(checkpoint["model"])
    optimizer.load_state_dict(checkpoint["optimizer"])
    scheduler.load_state_dict(checkpoint["scheduler"])
    if scaler is not None and checkpoint["scaler"] is not None:
        scaler.load_state_dict(checkpoint["scaler"])
    random.setstate(checkpoint["python_rng"])
    torch.set_rng_state(checkpoint["torch_rng"])
    if checkpoint["cuda_rng"] is not None:
        torch.cuda.set_rng_state_all(checkpoint["cuda_rng"])
    return checkpoint["epoch"], checkpoint["global_step"]

def E15(state, checkpoint_path):
```

```
    atomic_save(state, checkpoint_path)
    return checkpoint_path
```

Part 5 - The Philosophy Behind the Exercise

Reproducibility is continuity of causes. A checkpoint preserves not only what the model became, but the forces positioned to change it next. Resuming weights without momentum, randomness, and data position creates a new history wearing the old experiment's name. Honest recovery preserves identity or clearly marks the branch.

17.9.17 Exercise 16

Exercise Title: Export Seq2Seq Model and Validate Inference Parity and Deployment Constraints

Part 1 - Why this matters

Seq2seq export has more state and shape variation than a single forward graph. The encoder consumes a variable source and creates reusable memory. The decoder consumes a growing target, source mask, positions, and possibly a per-layer cache. Generation adds beam management, EOS rules, and sampling outside the neural network. Exporting one teacher-forced forward pass does not prove that autoregressive inference or cached beam search can be deployed correctly.

Define explicit export boundaries. An encoder graph maps source IDs and source validity to memory. A decoder graph maps target tokens, encoder memory, source validity, positions, and optional past state to logits and new state. Flatten nested caches in a documented layer/key/value order if the export runtime does not support the native structure. Declare dynamic batch, source length, target length, and cache length axes. Keep tokenization, beam search, and stopping in a tested host controller unless their export semantics are equally clear.

Validate native and exported executions across several source-target length pairs, padding patterns, batches, and precision modes. Compare encoder memory, decoder logits, cache tensors, greedy tokens, and complete short sequences. Tracing can freeze shape-dependent Python logic, ONNX backends may lower masks or attention operators differently, and dynamic cache shapes may be unsupported. Record limitations rather than hiding them behind one successful example. Benchmark export time, runtime initialization, encoder latency, decoder prefill, per-token decode, end-to-end generation, model size, and memory.

Part 2 - Mathematical form and reasoning

The two deployment contracts are

$$f_{\text{enc}}(X, M_x) \to H,$$

$$f_{\text{dec}}(Y, H, M_x, P, K_{<t}) \to (Z, K_{\le t}).$$

For native output Z and exported output $\widehat{Z}$, report

$$e_{\max} = \max_i |Z_i - \widehat{Z}_i|, \qquad e_{\text{rel}} = \frac{\|Z - \widehat{Z}\|_2}{\|Z\|_2 + \varepsilon}.$$

Also compare $\arg\max Z$ and complete decoded tokens. Numerical closeness does not guarantee sequence equality when two logits are nearly tied.

Part 3 - Hints, controls, and evidence

Use evaluation mode and inference mode. Freeze checkpoint, tokenizer, special IDs, masks, and stopping policy. Export small wrappers with only tensor inputs and outputs. Make causal and source masks explicit and test non-square lengths. Test all-padding rejection and at least one mixed-length batch. Warm each runtime before timing and synchronize device work. Save the first divergent step and tensor, not only the final different sentence. Evidence includes an interface schema, dynamic-axis table, parity matrix, supported/unsupported configuration list, latency breakdown, and a deterministic generation trace.

Part 4 - Python code skeleton

```
import time
import torch
from torch import nn

class EncoderExport(nn.Module):
    def __init__(self, model):
        super().__init__()
        self.model = model

    def forward(self, source_ids, source_mask):
        return self.model.encode(source_ids, source_mask)

class DecoderExport(nn.Module):
    def __init__(self, model):
        super().__init__()
        self.model = model

    def forward(self, decoder_ids, memory, source_mask):
        return self.model.decode(decoder_ids, memory, source_mask)

def parity(reference, candidate, absolute_tolerance, relative_tolerance):
    difference = (reference - candidate).abs()
    maximum_error = float(difference.max().item())
    close = torch.allclose(
        reference, candidate,
        atol=absolute_tolerance, rtol=relative_tolerance
    )
    same_token = torch.equal(reference.argmax(-1), candidate.argmax(-1))
    return close, same_token, maximum_error

@torch.inference_mode()
def E16(model, source_ids, source_mask, decoder_ids):
    model.eval()
    encoder = EncoderExport(model)
    decoder = DecoderExport(model)
```

```
encoder_example = (source_ids, source_mask)
traced_encoder = torch.jit.trace(encoder, encoder_example, strict=False)
native_memory = encoder(source_ids, source_mask)
exported_memory = traced_encoder(source_ids, source_mask)
memory_result = parity(native_memory, exported_memory, 1e-5, 1e-4)

decoder_example = (decoder_ids, native_memory, source_mask)
traced_decoder = torch.jit.trace(decoder, decoder_example, strict=False)
native_logits = decoder(decoder_ids, native_memory, source_mask)
exported_logits = traced_decoder(
    decoder_ids, exported_memory, source_mask
)
logit_result = parity(native_logits, exported_logits, 1e-5, 1e-4)
return memory_result, logit_result
```

Extend this reference with separate cached-decode exports and validate cache ordering before using it in beam search.

Part 5 - The Philosophy Behind the Exercise

Export moves computation across an institutional boundary: a new runtime, hardware stack, and control system become responsible for the model's meaning. Serialization transfers bytes; validation transfers trust. A deployable model is one whose conditional behavior survives that transfer under the shapes and states the real world will present.

17.9.18 Exercise 17

Exercise Title: Stress Test Seq2Seq Generation for Long Source Inputs and Detect Memory and Latency Failure Conditions

Part 1 - Why this matters

Long-source seq2seq inference can fail before the requested target becomes long. Encoder self-attention materializes relationships among source positions; cross-attention compares every decoder query with the source; encoder memory is retained throughout generation; and beam search duplicates decoder state across hypotheses. A short benchmark can hide nonlinear latency, out-of-memory events, mask allocation mistakes, positional limits, and timeouts that appear only near the deployment boundary.

Construct a stress matrix over source length, generated target length, batch, beam width, precision, and padding ratio. Increase one dimension at a time from a verified baseline. Log encoder latency, first-token latency, per-token decoder latency, total request latency, useful throughput, encoder-memory bytes, decoder- cache bytes, peak allocated and reserved CUDA memory, and output correctness. Record the last successful and first failed configuration. Treat OOM, NaN, timeout, violated mask parity, excessive latency, and wrong termination as distinct failure conditions.

Repeat identical requests after releasing request-local tensors. Expected memory for encoder representation remains proportional to source length during one request; retained growth across completed equal-size requests indicates a leak or lingering reference. Reserved allocator memory may stay high without being a leak, so compare allocated memory and live tensor ownership

at equal checkpoints. Test a source exceeding learned positional capacity and reject it cleanly or apply a documented extension strategy. If chunking or truncation is introduced, measure its effect on output meaning; bounded compute is not free when relevant source evidence is discarded.

Part 2 - Mathematical form and reasoning

Approximate dominant attention storage includes

$$M_{\text{scores}} \propto BH(S^2 + T^2 + ST)s,$$

where s is bytes per scalar. Encoder memory itself is approximately

$$M_H = BSds.$$

With beam width W, decoder-side states may scale with BW, while encoder memory may be shared or physically repeated depending on implementation. Define the latency failure boundary by a service objective $t_{p95} > t_{\max}$ and the memory boundary by peak allocated memory approaching the device budget with a declared safety margin.

Part 3 - Hints, controls, and evidence

Run under inference mode and start with greedy decoding. Warm up, synchronize measurements, and avoid storing full logits or attention maps in the stress loop. Save scalar summaries on CPU. Assert source-mask length, positional bounds, finite encoder memory, cache growth, EOS state, and maximum output length at every checkpoint. Test both dense valid sources and heavy padding because equal tensor length can contain different useful work. Repeat trials to distinguish allocator reuse from drift. If a configuration fails, capture the exception, input dimensions, memory just before failure, and minimal reproducer. Evidence is a scaling surface and a declared safe operating envelope, not one maximum.

Part 4 - Python code skeleton

```
import gc
import time
import torch

@torch.inference_mode()
def stress_one_request(model, source_ids, source_mask, bos_id,
                       maximum_new_tokens, device):
    model.eval()
    torch.cuda.reset_peak_memory_stats(device)
    torch.cuda.synchronize(device)
    request_start = time.perf_counter()

    encoder_start = torch.cuda.Event(enable_timing=True)
    encoder_end = torch.cuda.Event(enable_timing=True)
    encoder_start.record()
    memory = model.encode(source_ids, source_mask)
    encoder_end.record()
    torch.cuda.synchronize(device)
```

```
    generated = torch.full(
        (source_ids.size(0), 1), bos_id,
        dtype=torch.long, device=device
    )
    token_latencies = []
    for step in range(maximum_new_tokens):
        step_start = torch.cuda.Event(enable_timing=True)
        step_end = torch.cuda.Event(enable_timing=True)
        step_start.record()
        logits = model.decode(generated, memory, source_mask)
        next_token = logits[:, -1, :].argmax(dim=-1, keepdim=True)
        generated = torch.cat((generated, next_token), dim=1)
        step_end.record()
        torch.cuda.synchronize(device)
        token_latencies.append(step_start.elapsed_time(step_end))

    request_seconds = time.perf_counter() - request_start
    return {
        "source_length": source_ids.size(1),
        "output_length": generated.size(1) - 1,
        "encoder_ms": encoder_start.elapsed_time(encoder_end),
        "token_ms": token_latencies,
        "request_seconds": request_seconds,
        "peak_allocated": torch.cuda.max_memory_allocated(device),
        "peak_reserved": torch.cuda.max_memory_reserved(device),
    }

def release_request():
    gc.collect()
    if torch.cuda.is_available():
        torch.cuda.synchronize()

def E17(model, source_ids, source_mask, bos_id):
    return stress_one_request(
        model, source_ids, source_mask, bos_id, 128, source_ids.device
    )
```

Add EOS and cached decoding after the uncached reference passes; retain both so cache speedups cannot conceal semantic disagreement.

Part 5 - The Philosophy Behind the Exercise

Scale is where assumptions become physical. A mathematically valid architecture must still live within finite memory, time, and positional contracts. Stress testing does not ask how far the system can be pushed once; it discovers the region in which behavior remains correct, predictable, and recoverable.

17.9.19 Exercise 18

Exercise Title: Build a Full Seq2Seq Debugging Checklist Covering Masks, Shifts, Alignment, and Decoding Settings

Part 1 - Why this matters

Seq2seq failures cross boundaries. A tokenizer can change special IDs; a collate function can shift targets incorrectly; an encoder mask can hide valid source tokens; a decoder causal mask can leak the answer; cross-attention can broadcast one row's padding across another; beam search can reorder tokens without reordering cache; and EOS policy can truncate useful content. Fluent output and decreasing loss are compatible with several of these defects. The solution is an executable, dependency-ordered audit.

Start with artifact identity and raw examples. Verify dataset split, tokenizer fingerprint, vocabulary sizes, PAD/BOS/EOS IDs, source IDs, full targets, and lengths. Next verify shifted decoder inputs and labels, valid-token loss count, source padding, target padding, causal visibility, and batch-row isolation. Then verify encoder memory, cross-attention source dependence, finite logits, gradient reachability, and tiny-batch overfitting. Only after these contracts pass should autoregressive generation, beam scoring, length penalty, stopping, mixed precision, export, and performance be debugged.

Every checklist item must specify an invariant, test input, assertion, captured evidence, and known fault injection. For example, future-target intervention must not change earlier logits; padded-source intervention must not change valid outputs; valid-source intervention should change a conditioned output; beam one must equal greedy; batched generation must match independent generation under the same policy; resumed training must match uninterrupted continuation. A line that says "inspect masks" is advice, not protection.

Part 2 - Mathematical form and reasoning

Represent the pipeline as contracts

$$(X, Y) \xrightarrow{\mathcal{D}} (I_x, I_y, L, M_x, M_y) \xrightarrow{\mathcal{E}} H \xrightarrow{\mathcal{R}} Z \xrightarrow{\mathcal{G}} \widehat{Y} \xrightarrow{\mathcal{V}} R.$$

Data preparation $\mathcal{D}$, encoding $\mathcal{E}$, decoder retrieval $\mathcal{R}$, generation $\mathcal{G}$, and evaluation $\mathcal{V}$ each own invariants. Diagnose the earliest violated contract. Examples include

$$Z_{\le t}(Y_{>t}) = Z_{\le t}(Y'_{>t}), \qquad Z(X_{\text{pad}}) = Z(X'_{\text{pad}}),$$

and cached/full-prefix logit parity at every generated step.

Part 3 - Hints, controls, and evidence

Run greedy float32 evaluation first to remove sampling and precision variables. Use unique IDs and non-square source/target lengths. Include batch one and many, left and right padding if supported, source length one, early EOS, no EOS, maximum length, near-tied logits, and fully invalid inputs that must be rejected. Log first divergence, mask slices, positions, top logits, attention mass, cache shapes, stop reason, RNG state, and configuration. Do not relax tolerances until the error scale and decision effect are understood. Require every regression test to fail when its associated known bug is injected.

Part 4 - Python code skeleton

```
class Seq2SeqAudit:
    def __init__(self):
        self.records = []

    def require(self, name, condition, details):
        passed = bool(condition)
        record = {"name": name, "passed": passed, "details": details}
        self.records.append(record)
        if not passed:
            raise AssertionError(str(record))

def audit_shift(full_target, decoder_ids, labels, audit):
    audit.require(
        "decoder shift",
        (decoder_ids == full_target[:, :-1]).all(),
        {"decoder_shape": tuple(decoder_ids.shape)},
    )
    audit.require(
        "label shift",
        (labels == full_target[:, 1:]).all(),
        {"label_shape": tuple(labels.shape)},
    )

def audit_cross_weights(weights, source_valid, audit):
    visible = source_valid[:, None, None, :]
    padding_mass = weights.masked_fill(visible, 0.0).sum(dim=-1)
    row_sum = weights.masked_fill(~visible, 0.0).sum(dim=-1)
    audit.require(
        "zero source padding mass",
        padding_mass.max() < 1e-6,
        {"maximum_padding_mass": float(padding_mass.max().item())},
    )
    audit.require(
        "cross-attention normalized",
        (row_sum - 1.0).abs().max() < 1e-5,
        {"maximum_row_error": float((row_sum - 1.0).abs().max().item())},
    )

def audit_decoder_equivalence(greedy_tokens, beam_one_tokens, audit):
    audit.require(
        "beam width one equals greedy",
        (greedy_tokens == beam_one_tokens).all(),
        {"shape": tuple(greedy_tokens.shape)},
    )

def E18(full_target, decoder_ids, labels, weights, source_valid,
        greedy_tokens, beam_one_tokens):
    audit = Seq2SeqAudit()
```

```
    audit_shift(full_target, decoder_ids, labels, audit)
    audit_cross_weights(weights, source_valid, audit)
    audit_decoder_equivalence(greedy_tokens, beam_one_tokens, audit)
    return audit.records
```

Extend the registry with causal intervention, padded-source invariance, gradient checks, resume parity, mixed-precision finiteness, export parity, and stress limits. Keep each failure record independently reproducible.

Part 5 - The Philosophy Behind the Exercise

Debugging is the search for the first broken agreement. Complex pipelines tempt us to inspect the final symptom, but causality flows from data contracts through visibility, representation, decisions, and control. An executable checklist turns institutional memory into a system that can object when an old mistake returns.

17.9.20 Exercise 19

Exercise Title: Produce a Complete Seq2Seq Transformer Engineering Report Covering Training, Decoding, and Failure Analysis

Part 1 - Why this matters

The final report must make the entire seq2seq system reproducible and reviewable. It is not a list of successful metrics. It connects data alignment, masks, encoder memory, cross-attention, optimization, teacher forcing, autoregressive decoding, search policy, numerical precision, hardware behavior, export, and failure recovery. Another engineer should be able to reproduce the model, understand why each default was chosen, and recognize when the evidence no longer applies.

Record experiment identity first: code version, environment, hardware, dataset and split fingerprints, tokenizer version, special-token IDs, normalization, model dimensions, parameter count, initialization, seed controls, precision, optimizer, schedule, batch construction, checkpoint, and resume history. Report training and validation loss, token accuracy, exact match or BLEU, calibration, gradient norms, throughput, and peak memory. Include tiny-task overfitting and mask intervention results as correctness evidence.

Treat decoding as its own controlled study. Compare greedy, beam widths, length penalties, and relevant sampling policies on matched inputs. Report raw and normalized sequence scores, length, EOS success, repetition, exact match or BLEU, latency, and memory. Include cross-attention diagnostics without claiming they are complete explanations. Document mixed-precision differences, encoder and decoder profile shares, resume parity, export parity, and the long-source safe operating envelope. Preserve failed experiments with symptom, reproduction, root cause, correction, regression test, and remaining risk. End with defaults, exceptions, limitations, and measurable triggers for revisiting decisions.

Part 2 - Mathematical form and reasoning

Model fit is summarized by conditional negative log-likelihood

$$\mathcal{L}_{\mathrm{NLL}} = -\frac{1}{N}\sum_{b,t} \log p_\theta(y_{b,t} \mid y_{b,<t}, x_b),$$

but engineering selection is multi-objective. For configuration c, define

$$R(c) = (Q_c, C_c, L_c, M_c, T_c, F_c),$$

where Q is task quality, C calibration, L latency, M memory, T throughput, and F failure rate. Do not collapse this vector into one number without justified weights. Report variation across seeds, examples, and length buckets. A production limit is a measured boundary with a failure criterion, not the largest input that survived once.

Part 3 - Hints, controls, and evidence

Use a fixed schema and link every claim to a table, plot, trace, sample, or test. Separate observations, interpretations, and recommendations. State tokenization, metric implementation, smoothing, beam scoring, timing boundary, warm-up, hardware, padding efficiency, and exclusions. Select qualitative examples by a predeclared rule such as first failure per length bucket, not by attractiveness. Validate that recommendations cite evidence and that generation has explicit maximum length and stop policy. Include unknowns honestly. A negative result can prevent repeated waste and therefore belongs in the permanent report.

Part 4 - Python code skeleton

```
import json
from pathlib import Path

REQUIRED_SECTIONS = (
    "experiment_identity",
    "data_tokenizer_alignment",
    "architecture_and_masks",
    "training_results",
    "cross_attention_analysis",
    "decoding_comparison",
    "cuda_precision_profile",
    "checkpoint_resume",
    "export_parity",
    "stress_limits",
    "failure_registry",
    "recommended_defaults",
    "known_limitations",
)

def validate_report(report):
    errors = []
    for section in REQUIRED_SECTIONS:
        if section not in report:
            errors.append("missing section: " + section)
    identity = report.get("experiment_identity", {})
    required_identity = (
        "code_version", "checkpoint_hash", "dataset_hash",
        "tokenizer_hash", "seed", "device"
    )
```

```
    for field in required_identity:
        if field not in identity:
            errors.append("missing identity field: " + field)
    defaults = report.get("recommended_defaults", {})
    for field in ("beam_width", "length_penalty", "maximum_tokens",
                  "evidence_ids"):
        if field not in defaults:
            errors.append("missing decoding default: " + field)
    failures = report.get("failure_registry", [])
    for index in range(len(failures)):
        failure = failures[index]
        required = ("symptom", "reproduction", "root_cause",
                    "fix", "regression_test")
        for field in required:
            if field not in failure:
                message = "failure " + str(index) + " missing " + field
                errors.append(message)
    if errors:
        raise ValueError("\n".join(errors))

def save_report(report, output_path):
    validate_report(report)
    path = Path(output_path)
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", encoding="utf-8") as handle:
        json.dump(report, handle, indent=2, sort_keys=True)

def E19(report):
    output_path = "reports/trf07_seq2seq_engineering_report.json"
    save_report(report, output_path)
    return output_path
```

Generate human-readable tables from the same structured record so narrative and machine-verifiable evidence cannot silently diverge.

Part 5 - The Philosophy Behind the Exercise

An engineering report transfers responsibility. It preserves what was tested, what failed, why a decision was made, and what future observation should change that decision. The final achievement is not merely conditional generation; it is a chain of evidence strong enough that another researcher can inherit the system without inheriting its hidden assumptions.

17.10 TRF08

Chapter Title: Efficiency, Long Context, and Production Readiness
Specialization: Applied AI and Deployment

17.10.1 Exercise 00

Exercise Title: Explain Why Attention Is $O(\text{seq}^2)$ and Identify When Long Context Becomes Impossible on Your GPU

Part 1 - Why this matters

Long context is not obtained by changing one maximum-length constant. Standard self-attention compares every query position with every key position. Doubling sequence length therefore creates four times as many pairwise scores, before accounting for gradients, saved activations, masks, temporary buffers, and the rest of the Transformer. A configuration can be mathematically valid yet physically impossible on a particular GPU. The useful skill is to predict that boundary, measure it safely, and explain which allocation reaches it first.

Start from tensor shapes rather than asymptotic notation. For batch B, heads H, sequence length L, and head width d_h, queries, keys, and values occupy linear memory in L, while the score and probability tensors have shape (B, H, L, L). During training, autograd may retain inputs or intermediates for the backward pass. Optimized kernels may avoid materializing the full probability matrix, whereas a naive implementation does not. Consequently, $O(L^2)$ describes scaling, not the exact byte total.

Build an analytic estimator that separately reports parameters, gradients, optimizer state, Q/K/V tensors, attention scores, probabilities, other layer activations, and a safety margin. Then measure peak allocated and reserved CUDA memory for increasing lengths using a fixed model, batch, dtype, and mode. Increase gradually and catch out-of-memory errors without crashing the whole experiment. Record the largest successful length and first failure for forward inference, cached decoding, and training; they have different memory profiles. Do not report the failure point as a universal model limit without hardware, kernel, precision, batch, layer count, and free-memory context.

Part 2 - Mathematical form and reasoning

Score construction for one layer is

$$S = \frac{QK^\top}{\sqrt{d_h}}, \qquad S \in \mathbb{R}^{B\times H\times L\times L}.$$

Its element count is BHL^2 and its storage is approximately

$$M_S = BHL^2 s,$$

where s is bytes per scalar. Materializing scores and probabilities requires about $2BHL^2 s$ before backward storage and temporary workspace. Attention matrix multiplications require on the order of $BHL^2 d_h$ operations. If usable GPU memory is M_{usable} and non-attention allocations consume $M_0(L)$, the naive feasibility condition is

$$M_0(L) + cBHL^2 s \le M_{\text{usable}},$$

where c must be measured for the implementation and execution mode.

Part 3 - Hints, controls, and evidence

Query free and total memory immediately before each isolated trial. Warm up at a small length, reset peak statistics, synchronize, and delete trial-local tensors after measurement.

Compare forward-only, backward, and optimizer-step peaks. Repeat for float32 and float16 or bfloat16, but do not infer quality from memory. Use binary search only after a safe coarse sweep brackets the boundary. Distinguish allocated memory from reserved allocator memory. Include a prediction-versus- measurement table, relative estimator error, first OOM configuration, and the exact safety margin you would enforce in production.

Part 4 - Python code skeleton

```
import gc
import torch

def attention_matrix_bytes(batch_size, heads, length, dtype,
                           materialized_matrices):
    bytes_per_value = torch.empty((), dtype=dtype).element_size()
    values = batch_size * heads * length * length
    return values * bytes_per_value * materialized_matrices

def memory_snapshot(device):
    free_bytes, total_bytes = torch.cuda.mem_get_info(device)
    return {
        "free": free_bytes,
        "total": total_bytes,
        "allocated": torch.cuda.memory_allocated(device),
        "reserved": torch.cuda.memory_reserved(device),
        "peak": torch.cuda.max_memory_allocated(device),
    }

@torch.inference_mode()
def try_attention_length(attention_module, batch_size, length,
                         model_width, dtype, device):
    gc.collect()
    torch.cuda.empty_cache()
    torch.cuda.reset_peak_memory_stats(device)
    inputs = torch.randn(
        batch_size, length, model_width, dtype=dtype, device=device
    )
    try:
        output = attention_module(inputs, inputs, inputs, need_weights=False)
        torch.cuda.synchronize(device)
        result = memory_snapshot(device)
        result["succeeded"] = True
        result["output_shape"] = tuple(output[0].shape)
    except torch.OutOfMemoryError:
        result = memory_snapshot(device)
        result["succeeded"] = False
    del inputs
    return result

def E00(attention_module):
```

```
    device = torch.device("cuda:0")
    results = {}
    for length in (512, 1024, 2048, 4096, 8192):
        results[length] = try_attention_length(
            attention_module, 1, length, 512, torch.float16, device
        )
        if not results[length]["succeeded"]:
            break
    return results
```

Part 5 - The Philosophy Behind the Exercise

Asymptotic notation is a warning about growth, not a substitute for measurement. Engineering begins when the warning is translated into bytes, kernels, safety margins, and failure boundaries on a real machine. Long context is therefore not merely a model feature; it is a contract between mathematical ambition and finite physical resources.

17.10.2 Exercise 01

Exercise Title: Implement Flash-Attention Conceptually and Compare Memory Use Against Standard Attention on Toy Example

Part 1 - Why this matters

Standard attention is often described only by its arithmetic, but performance is also controlled by movement between GPU memory and fast on-chip storage. A naive implementation writes the complete score matrix, reads it for softmax, writes probabilities, and reads them again to mix values. Flash-style attention tiles queries, keys, and values, performs more work while data is local, and avoids storing the full $L \times L$ probability matrix. It computes exact attention up to floating-point ordering; it is not sparse or approximate attention.

Implement a pedagogical blockwise reference using online softmax. Process one query block against successive key/value blocks. Maintain for every query row the running maximum, running exponential denominator, and running weighted value accumulator. When a new key block contains a larger score, rescale earlier sums before merging. Apply causal or padding masks inside each tile using global indices. This Python reference will not be faster than fused kernels; its goal is to prove the recurrence and the fact that global normalization can be constructed without the global score matrix.

Compare blockwise output and gradients with naive float64 attention on small tensors, then test float32 and reduced precision with realistic tolerances. Use PyTorch's scaled dot-product attention as the optimized execution path when its backend is available. Measure peak CUDA memory and latency across lengths, but separate the slow pedagogical loop from the fused implementation. Verify dropout semantics and deterministic limitations. If attention weights are requested, the memory advantage may disappear because the full matrix must be returned.

Part 2 - Mathematical form and reasoning

For one query row, suppose processed blocks have running maximum m, normalizer ℓ, and unnormalized output accumulator o. For a new score block s,

$$m' = \max(m, \max s),$$

$$\ell' = e^{m-m'}\ell + \sum_j e^{s_j - m'},$$

$$o' = e^{m-m'}o + \sum_j e^{s_j - m'} v_j.$$

After the last block, the attention output is o/ℓ. The rescaling term $e^{m-m'}$ preserves contributions from earlier blocks under the new stable maximum. Working storage scales with tile sizes rather than L^2, while total attention arithmetic remains quadratic for dense attention.

Part 3 - Hints, controls, and evidence

Start with batch one, head one, no dropout, and no mask. Add padding, then causal mask. Use global query and key offsets when constructing tile visibility. Handle fully masked query rows explicitly. Compare output, loss, and Q/K/V gradients against the naive formula. Record maximum absolute and relative error. For CUDA benchmarks, warm up, synchronize, reset peak memory, and report which scaled- dot-product backend was selected. Compare lengths large enough to expose memory movement, and note compilation or kernel-selection effects. Evidence must show the online recurrence manually for at least two key blocks.

Part 4 - Python code skeleton

```
import math
import torch

def blockwise_attention(queries, keys, values, key_block_size):
    batch_size, heads, query_length, head_width = queries.shape
    running_maximum = torch.full(
        (batch_size, heads, query_length, 1), float("-inf"),
        dtype=torch.float64, device=queries.device
    )
    running_sum = torch.zeros_like(running_maximum)
    running_output = torch.zeros(
        batch_size, heads, query_length, head_width,
        dtype=torch.float64, device=queries.device
    )
    query_values = queries.to(torch.float64)
    for start in range(0, keys.size(2), key_block_size):
        end = min(start + key_block_size, keys.size(2))
        key_block = keys[:, :, start:end, :].to(torch.float64)
        value_block = values[:, :, start:end, :].to(torch.float64)
        scores = torch.matmul(query_values, key_block.transpose(-2, -1))
        scores = scores / math.sqrt(head_width)

        block_maximum = scores.max(dim=-1, keepdim=True).values
        new_maximum = torch.maximum(running_maximum, block_maximum)
        old_scale = torch.exp(running_maximum - new_maximum)
```

```
        block_exponentials = torch.exp(scores - new_maximum)
        running_sum = old_scale * running_sum
        block_sum = block_exponentials.sum(dim=-1, keepdim=True)
        running_sum = running_sum + block_sum
        running_output = old_scale * running_output
        running_output = running_output + torch.matmul(
            block_exponentials, value_block
        )
        running_maximum = new_maximum
    return (running_output / running_sum).to(queries.dtype)

def E01(queries, keys, values):
    blockwise = blockwise_attention(queries, keys, values, 64)
    naive_scores = torch.matmul(queries, keys.transpose(-2, -1))
    naive_scores = naive_scores / math.sqrt(queries.size(-1))
    naive = torch.matmul(torch.softmax(naive_scores, dim=-1), values)
    assert torch.allclose(blockwise, naive, atol=1e-5, rtol=1e-4)
    return blockwise
```

Part 5 - The Philosophy Behind the Exercise

An algorithm is not only a count of operations; it is a schedule for where information lives while those operations occur. Flash attention changes the schedule while preserving the mathematical answer. Its lesson is that major efficiency gains can come from respecting the memory hierarchy rather than weakening the model's computation.

17.10.3 Exercise 02

Exercise Title: Implement Sliding Window Attention Conceptually and Show How It Reduces Cost While Preserving Local Context

Part 1 - Why this matters

Many sequence relationships are local: nearby tokens often contain the most immediate syntactic or temporal evidence. Sliding-window attention limits each query to a neighborhood instead of all positions. With fixed window width, its score count grows linearly with sequence length, enabling longer inputs. The tradeoff is structural, not merely numerical: information outside the window cannot be retrieved directly, and distant dependencies require propagation through layers or special global mechanisms.

Implement a transparent reference that builds a Boolean visibility matrix from global query and key indices. Support bidirectional windows for encoders and a causal backward-looking window for decoders. Apply padding after window visibility. Compare output with full attention at positions whose full-attention weights are known to be local. Then construct a task where the first token must affect the last token beyond the window and show failure or slower multi-layer propagation.

Measure score elements, peak memory, runtime, and task quality for several window sizes and sequence lengths. A dense masked implementation still materializes L^2 scores and therefore demonstrates semantics but not the true memory benefit; a production implementation needs

sparse or block-local kernels. State this difference explicitly. Test boundary positions, sequences shorter than the window, padding, and causal leakage. Consider periodic global tokens or dilated patterns only as separately measured architectural changes.

Part 2 - Mathematical form and reasoning

For a bidirectional radius w, position i may attend to

$$\mathcal{N}(i) = \{j : |i - j| \leq w\}.$$

For causal local attention,

$$\mathcal{N}_{\text{causal}}(i) = \{j : 0 \leq i - j < w\}.$$

The number of visible pairs is at most $L(2w+1)$ bidirectionally or Lw causally, rather than L^2. Dense masked computation remains $O(L^2)$ in storage unless the kernel avoids constructing blocked entries. After D layers, the theoretical receptive field of a simple causal window can grow to roughly Dw positions, though information quality may degrade through repeated mixing.

Part 3 - Hints, controls, and evidence

Use unique position-coded values to verify which keys contribute. Print the mask for a tiny sequence and inspect first, middle, and last rows. Confirm future positions are blocked in causal mode. Change a token outside the window and assert the current layer's output is invariant; change one inside and expect an effect. Distinguish window radius from total window width in configuration. Benchmark a genuinely local kernel separately from the dense teaching version. Report accuracy by dependency distance, memory, latency, and the smallest window that preserves the task's required context.

Part 4 - Python code skeleton

```
import math
import torch

def sliding_visibility(length, window_size, causal, device):
    query_position = torch.arange(length, device=device)[:, None]
    key_position = torch.arange(length, device=device)[None, :]
    if causal:
        distance = query_position - key_position
        visible = (distance >= 0) & (distance < window_size)
    else:
        radius = window_size // 2
        visible = (query_position - key_position).abs() <= radius
    return visible

def dense_reference_window_attention(queries, keys, values,
                                     window_size, causal):
    length = queries.size(-2)
    visible = sliding_visibility(
        length, window_size, causal, queries.device
    )
```

```
    scores = torch.matmul(queries, keys.transpose(-2, -1))
    scores = scores / math.sqrt(queries.size(-1))
    scores = scores.masked_fill(~visible, float("-inf"))
    probabilities = torch.softmax(scores.float(), dim=-1).to(scores.dtype)
    output = torch.matmul(probabilities, values)
    return output, probabilities, visible

def E02(queries, keys, values):
    output, probabilities, visible = dense_reference_window_attention(
        queries, keys, values, window_size=8, causal=True
    )
    blocked_mass = probabilities.masked_fill(visible, 0.0).abs().max()
    assert blocked_mass.item() == 0.0
    return output, visible
```

Part 5 - The Philosophy Behind the Exercise

Efficiency sometimes comes from refusing to represent every possible relationship. A sliding window asserts that locality is valuable enough to be an inductive bias. The gain is honest only when the excluded relationships are named and tested. Scarcity becomes architecture when we decide deliberately which information is allowed to remain reachable.

17.10.4 Exercise 03

Exercise Title: Implement KV Cache Properly and Measure Latency Reduction for Autoregressive Inference

Part 1 - Why this matters

Without a key/value cache, autoregressive generation repeatedly recomputes keys and values for the entire prefix at every token. The result is correct but wasteful: history that has not changed is projected again and again. A KV cache stores each layer's past keys and values, so decode processes only new tokens. This changes inference from a stateless full-prefix call into a stateful protocol, introducing new correctness obligations for positions, masks, batches, beams, devices, and cache ownership.

Implement prefill and decode as explicit phases. Prefill consumes the prompt, produces first next-token logits, and initializes per-layer key/value tensors. Decode accepts one new token, calculates its projections, appends them along the sequence axis, attends over valid history, and returns updated state. Assert cache shape (B, H, T, d_h) at every layer and exact growth by the number of new tokens. Position IDs for cached decode must begin at the previous logical length, not restart from zero.

Establish token-by-token parity against full-prefix recomputation in evaluation mode. Compare logits before comparing chosen tokens; greedy sequence equality can hide compensating errors. Test batch reordering, beam expansion, finished rows, mixed prompt lengths, and cache reset between requests. Measure prompt prefill time, first-token latency, steady decode latency, tokens per second, and cache bytes. Benchmark several output lengths because cache overhead may not pay off for extremely short generations or tiny models.

Part 2 - Mathematical form and reasoning

At layer ℓ and step t, cache update is

$$K_{1:t}^{(\ell)} = \mathrm{concat}(K_{1:t-1}^{(\ell)}, K_t^{(\ell)}), \qquad V_{1:t}^{(\ell)} = \mathrm{concat}(V_{1:t-1}^{(\ell)}, V_t^{(\ell)}).$$

Cache storage for L_n layers is approximately

$$M_{\mathrm{KV}} = 2L_nBHTd_hs.$$

Full-prefix decoding repeatedly projects $O(T)$ tokens per step, whereas cached decode projects only the new block. Dense attention of the new query still reads the growing key history, so cache does not make context cost constant.

Part 3 - Hints, controls, and evidence

Begin with batch one and greedy decoding. Disable dropout and control dtype. Capture maximum absolute logit difference per layer and step. Verify cache length, batch, head, head width, device, and dtype. Test prompt prefill with more than one token and decode blocks larger than one. For beams, reorder every cache tensor with the same parent-beam indices used for token sequences. Never reuse cache across unrelated requests. Time after warm-up with CUDA events and synchronized boundaries. Report speedup versus generation length and the memory cost per additional token. Preserve the slow uncached implementation as an oracle.

Part 4 - Python code skeleton

```
import torch

def cache_bytes(past_key_values):
    total = 0
    for layer_cache in past_key_values:
        key, value = layer_cache
        total += key.numel() * key.element_size()
        total += value.numel() * value.element_size()
    return total

@torch.inference_mode()
def cached_greedy(model, prompt_ids, maximum_new_tokens):
    model.eval()
    output = model(input_ids=prompt_ids, use_cache=True)
    cache = output.past_key_values
    next_token = output.logits[:, -1, :].argmax(dim=-1, keepdim=True)
    generated = [next_token]
    expected_length = prompt_ids.size(1)

    for step in range(maximum_new_tokens - 1):
        output = model(
            input_ids=next_token,
            past_key_values=cache,
            use_cache=True,
        )
```

```
        cache = output.past_key_values
        expected_length += 1
        for layer_index in range(len(cache)):
            key, value = cache[layer_index]
            if key.shape != value.shape:
                raise AssertionError("key/value shape mismatch")
            if key.size(-2) != expected_length:
                raise AssertionError("cache length mismatch")
        next_token = output.logits[:, -1, :].argmax(dim=-1, keepdim=True)
        generated.append(next_token)
    return torch.cat(generated, dim=1), cache_bytes(cache)

def E03(model, prompt_ids):
    tokens, bytes_used = cached_greedy(model, prompt_ids, 64)
    return {"tokens": tokens, "cache_bytes": bytes_used}
```

Add an uncached trace and compare logits at every step before trusting the reported latency improvement.

Part 5 - The Philosophy Behind the Exercise

Caching is the preservation of valid work. It gains speed by declaring that the past computation still means the same thing. That declaration creates responsibility: identity, position, order, and ownership must remain intact. Efficiency is trustworthy only while remembered computation remains correctly attached to the history that produced it.

17.10.5 Exercise 04

Exercise Title: Implement Quantization Conceptually and Compare Accuracy Drop vs Speed Gain for Transformer Inference

Part 1 - Why this matters

Quantization represents weights or activations with fewer bits. It can reduce model size, memory bandwidth, and sometimes latency, which is valuable when a Transformer is limited by moving weights rather than arithmetic. But converting numbers to int8 does not automatically accelerate inference: the hardware and runtime need optimized kernels, quantize/dequantize overhead must be amortized, and small or unsupported operators may fall back to slower paths. Quality loss depends on value distribution, calibration data, granularity, and sensitive layers.

Implement symmetric weight-only quantization for a linear layer as a transparent reference. Calculate a scale from maximum absolute weight, map weights to the integer range, clamp, round, and reconstruct floating weights for comparison. This fake-quantized reference demonstrates error but not true integer-kernel speed. Then use a supported dynamic or weight-only quantization backend for real latency measurement. State whether activations remain floating point and whether scales are per-tensor, per-channel, or per-group.

Compare layer outputs, logits, greedy tokens, perplexity or task accuracy, model bytes, peak memory, and latency across representative prompts and lengths. Identify sensitive components by quantizing embeddings, attention projections, FFN layers, normalization, and output

head separately. Calibration data must represent production distributions; otherwise activation ranges can fail on rare inputs. Include extreme values and distribution shift. Report where quality degradation first appears and whether mixed precision, selective exclusion, or finer quantization granularity repairs it.

Part 2 - Mathematical form and reasoning

For symmetric signed b-bit quantization, integer range is $[-q_{\max}, q_{\max}]$ with $q_{\max} = 2^{b-1} - 1$. For tensor W,

$$s = \frac{\max |W|}{q_{\max}}, \qquad Q = \text{clamp}\left(operatornameround(W/s), -q_{\max}, q_{\max}\right),$$

$$\widehat{W} = sQ.$$

Quantization error is $E = W - \widehat{W}$. A single outlier increases s and reduces resolution for ordinary values; per-channel scales can reduce this effect at the cost of more metadata and implementation complexity.

Part 3 - Hints, controls, and evidence

Handle the all-zero tensor without dividing by zero. Use float32 as reference and identical inputs. Measure weight error by maximum, mean absolute, and relative norms. Test per-tensor and per-output-channel scales. Separate fake quantization from true backend execution in every table. Warm up and synchronize latency tests, include batch and sequence length, and report p50 and p95. Check that serialized size and runtime memory actually improve. Stratify quality by input length and rarity. Evidence must support a deployment choice, including a case where quantization is rejected because the tradeoff is unfavorable.

Part 4 - Python code skeleton

```
import torch

def symmetric_quantize(weight, bits, per_output_channel):
    integer_maximum = 2 ** (bits - 1) - 1
    if per_output_channel:
        maximum = weight.abs().amax(dim=1, keepdim=True)
    else:
        maximum = weight.abs().amax()
    scale = maximum / integer_maximum
    scale = torch.where(scale == 0, torch.ones_like(scale), scale)
    quantized = torch.round(weight / scale)
    quantized = torch.clamp(quantized, -integer_maximum, integer_maximum)
    return quantized.to(torch.int8), scale

def dequantize(quantized, scale):
    return quantized.float() * scale

def quantized_linear_reference(inputs, weight, bias, bits=8):
    quantized, scale = symmetric_quantize(
        weight, bits, per_output_channel=True
```

```
    )
    reconstructed = dequantize(quantized, scale)
    output = torch.nn.functional.linear(inputs.float(), reconstructed, bias)
    return output, quantized, scale

def E04(inputs, linear_layer):
    reference = linear_layer(inputs.float())
    candidate, quantized, scale = quantized_linear_reference(
        inputs, linear_layer.weight.detach(), linear_layer.bias, bits=8
    )
    error = (reference - candidate).abs()
    return {
        "maximum_output_error": float(error.max().item()),
        "mean_output_error": float(error.mean().item()),
        "quantized_weight": quantized,
        "scale": scale,
    }
```

Part 5 - The Philosophy Behind the Exercise

Quantization asks which numerical distinctions the model truly needs. Fewer bits can expose redundancy and make intelligence cheaper to move, but every removed distinction is an approximation. The ethical engineering position is neither "smaller is always better" nor "precision is sacred"; it is to measure what the system forgets and decide whether that loss is acceptable for its purpose.

17.10.6 Exercise 05

Exercise Title: Benchmark Inference Under float32 float16 and int8 and Build a Performance Tradeoff Table

Part 1 - Why this matters

Precision selection is a system decision involving quality, memory, kernel availability, hardware, batch size, and workload phase. Float16 often accelerates matrix multiplication on suitable GPUs and halves tensor storage, but sensitive softmax or normalization paths may need float32 accumulation. Int8 can reduce weight bandwidth further, yet unsupported operations, conversion overhead, or small batches may eliminate the speed benefit. A fair benchmark must use the same model semantics and distinguish prefill from decode.

Prepare three validated variants from one checkpoint: float32 reference, float16 or autocast inference, and a true int8 backend configuration. Record exactly which tensors and operators use each dtype. Use identical tokenized prompts, masks, stopping policy, maximum output length, and deterministic greedy decoding for parity. Evaluate several prompt lengths, generation lengths, and batch sizes. Warm up every variant independently and exclude conversion, quantization calibration, compilation, and runtime initialization from steady- state latency while reporting those startup costs separately.

Build a tradeoff table containing serialized model size, resident memory, peak allocated memory, first-token latency, steady per-token latency, throughput, p50/p95/p99 request latency, maximum logit error, greedy token agreement, perplexity or task metric, failure count, and energy proxy if available. Repeat measurements and report variation. A variant is not production-ready merely because average latency improves: rare NaNs, tail spikes, or quality collapse on long inputs can dominate operational value.

Part 2 - Mathematical form and reasoning

For reference logits Z and candidate $\widehat{Z}$, report

$$e_{\max} = \max_i |Z_i - \widehat{Z}_i|, \qquad e_{\mathrm{rel}} = \frac{\|Z - \widehat{Z}\|_2}{\|Z\|_2 + \varepsilon}.$$

Token agreement is

$$A_{\mathrm{token}} = \frac{1}{N} \sum_i \mathbf{1}[\arg\max Z_i = \arg\max \widehat{Z}_i].$$

For latency samples sorted increasingly, percentile p is an empirical order statistic. Report sample count and measurement boundary. Speedup is t_{32}/t_q, but must be paired with quality difference and memory reduction.

Part 3 - Hints, controls, and evidence

Run each variant in evaluation and inference modes. Reset peak memory and synchronize around timings. Randomize or rotate variant order to reduce thermal and background bias. Keep tokenization outside model timing unless measuring end-to-end service latency. Test ordinary, extreme-logit, long-context, rare- token, and distribution-shift inputs. Verify int8 is actually using integer or weight-only kernels rather than dequantizing the whole model before compute. Report unsupported layers and fallbacks. Select a deployment precision only after applying explicit quality, failure-rate, memory, and tail-latency gates.

Part 4 - Python code skeleton

```
import statistics
import time
import torch

def synchronized_call(function, argument, device):
    if device.type == "cuda":
        torch.cuda.synchronize(device)
    started = time.perf_counter()
    output = function(argument)
    if device.type == "cuda":
        torch.cuda.synchronize(device)
    return output, time.perf_counter() - started

def benchmark_variant(name, generate_function, prompts, device):
    for warmup_index in range(10):
        generate_function(prompts[0])
    latencies = []
```

```
    outputs = []
    if device.type == "cuda":
        torch.cuda.reset_peak_memory_stats(device)
    for prompt in prompts:
        output, seconds = synchronized_call(
            generate_function, prompt, device
        )
        outputs.append(output)
        latencies.append(seconds)
    ordered = sorted(latencies)
    p50_index = int(0.50 * (len(ordered) - 1))
    p95_index = int(0.95 * (len(ordered) - 1))
    peak = 0
    if device.type == "cuda":
        peak = torch.cuda.max_memory_allocated(device)
    return {
        "name": name,
        "mean_seconds": statistics.mean(latencies),
        "p50_seconds": ordered[p50_index],
        "p95_seconds": ordered[p95_index],
        "peak_allocated_bytes": peak,
        "outputs": outputs,
    }

def E05(variants, prompts, device):
    rows = []
    for name, generate_function in variants.items():
        row = benchmark_variant(
            name, generate_function, prompts, device
        )
        rows.append(row)
    return rows
```

Part 5 - The Philosophy Behind the Exercise

A dtype is a compressed representation of the model's numerical world. The right precision is not the smallest one that executes; it is the least costly representation that preserves the decisions and reliability the task requires. A tradeoff table makes that judgment visible instead of letting speed silently define success.

17.10.7 Exercise 06

Exercise Title: Implement torch.compile and Measure Speedups and Compilation Overhead for Transformer Models

Part 1 - Why this matters

`torch.compile` can capture PyTorch execution, specialize graphs, fuse operations, and reduce Python and kernel-launch overhead. Transformer speedup depends on graph structure, shapes, dynamic control flow, backend, and workload. The first calls may be much slower

because compilation occurs, and new sequence lengths or branches may trigger recompilation. Reporting only a warmed fixed- shape result can therefore misrepresent a variable-length production service.

Compile a copied evaluation model or a stable inference wrapper while preserving an eager reference. Test a fixed-shape workload first, then a realistic shape distribution. Record initial compile latency, number of warm-up calls, graph breaks, recompilations, steady latency, throughput, peak memory, and output parity. Test encoder prefill and one-token decode separately; Python sampling and stopping logic may remain outside the compiled graph. Do not compile an entire request loop blindly when its changing sequence tensors cause a new graph every step.

Compare default, reduce-overhead, and maximum-autotune modes only when the backend supports them and budgets are matched. Use length bucketing or dynamic shapes as explicit strategies and measure their effect. Validate logits and generated tokens across batches, masks, lengths, and dtypes. Compilation may alter floating-point operation ordering, so establish tolerances and investigate decision-changing differences. Include a break-even calculation: how many requests are required before steady-state savings repay compilation cost.

Part 2 - Mathematical form and reasoning

If eager latency is t_e, compiled steady latency is t_c, and one-time compile cost is C, total times for N calls are

$$T_e(N) = Nt_e, \qquad T_c(N) = C + Nt_c.$$

Compilation breaks even when

$$N > \frac{C}{t_e - t_c},$$

provided $t_c < t_e$. Recompilations add costs C_j for encountered shape or control-flow guards, so production amortization depends on the input distribution, not one benchmark shape.

Part 3 - Hints, controls, and evidence

Freeze the checkpoint, evaluation mode, precision, and inputs. Warm eager and compiled variants separately. Synchronize CUDA around wall timing. Separate first call, subsequent compilation calls, and steady samples. Enable compiler diagnostics to count graph breaks and guard failures without placing verbose logging inside final timing. Test common and rare lengths, batch changes, mask patterns, and cache state shapes. Save the smallest input causing recompile or parity failure. Evidence includes compile cost, break-even request count, steady-state percentiles, memory, graph count, and a recommendation about shape bucketing or eager fallback.

Part 4 - Python code skeleton

```
import time
import torch

def synchronized_seconds(function, arguments, device):
    if device.type == "cuda":
```

```
        torch.cuda.synchronize(device)
    started = time.perf_counter()
    output = function(*arguments)
    if device.type == "cuda":
        torch.cuda.synchronize(device)
    return output, time.perf_counter() - started

@torch.inference_mode()
def benchmark_compile(model, example_batches, device):
    model.eval()
    compiled_model = torch.compile(model, mode="reduce-overhead")
    eager_times = []
    compiled_times = []
    parity = []

    for batch in example_batches:
        arguments = (batch["input_ids"], batch["attention_mask"])
        eager_output, eager_seconds = synchronized_seconds(
            model, arguments, device
        )
        compiled_output, compiled_seconds = synchronized_seconds(
            compiled_model, arguments, device
        )
        eager_times.append(eager_seconds)
        compiled_times.append(compiled_seconds)
        close = torch.allclose(
            eager_output, compiled_output, atol=1e-5, rtol=1e-4
        )
        same_token = torch.equal(
            eager_output.argmax(-1), compiled_output.argmax(-1)
        )
        parity.append({"close": close, "same_token": same_token})
    return eager_times, compiled_times, parity

def E06(model, batches):
    device = next(model.parameters()).device
    return benchmark_compile(model, batches, device)
```

The first compiled timing contains compilation. Preserve it, then collect a separate repeated steady window for every shape bucket.

Part 5 - The Philosophy Behind the Exercise

Compilation converts generality into specialized execution. It is powerful because it assumes something about future shapes and control flow. The engineer must measure whether reality honors that assumption often enough to repay its cost. Speed is not only how fast a graph runs; it includes the price of deciding which graph the world requires.

17.10.8 Exercise 07

Exercise Title: Profile GPU Utilization and Identify Bottlenecks in Attention, FFN, and Data Transfer During Inference

Part 1 - Why this matters

Low GPU utilization is a symptom, not a diagnosis. In Transformer inference, the device may wait for host-to-device transfers, tokenization, Python generation logic, synchronization, dynamic batching, or small sequential decode kernels. At other times the GPU is busy but memory bandwidth, attention, FFN matrix multiplication, or cache movement limits throughput. A global utilization percentage cannot tell these cases apart.

Profile a representative service path and mark preprocessing, transfer, prefill attention, prefill FFN, decode attention, decode FFN, logits processing, sampling, cache update, and stopping. Collect CPU and CUDA activities, shapes, memory, kernel launch count, and self versus total time. Separate prefill from decode: prefill often has larger parallel matrix operations, while single-token decode can be bandwidth- or launch-bound. Profile batch one and realistic dynamic batches.

Calculate phase shares and locate idle gaps on the timeline. Vary prompt length, generation length, batch, vocabulary size, and cache length one at a time. If transfer dominates, test pinned memory, nonblocking copies, and overlap. If attention dominates long prefill, test an optimized attention kernel. If FFN dominates, compilation, quantization, or batching may help. If CPU control dominates decode, reduce per-token synchronizations and tensor-to-Python conversions. Apply one change, verify output parity, and re-profile end to end.

Part 2 - Mathematical form and reasoning

End-to-end request latency can be decomposed as

$$t = t_{\text{queue}} + t_{\text{preprocess}} + t_{\text{transfer}} + t_{\text{prefill}} + \sum_{u=1}^{U} (t_{\text{attention,u}} + t_{\text{FFN,u}} + t_{\text{control,u}}) - t_{\text{overlap}}.$$

Phase fraction is $r_i = t_i/t$. Arithmetic intensity, approximately FLOPs per byte transferred, helps distinguish compute-bound from bandwidth-bound kernels. Optimization priority belongs to the largest non-overlapped component that can be changed without violating quality or latency objectives.

Part 3 - Hints, controls, and evidence

Warm up before profiling and profile a short steady window because collection changes timing. Avoid printing inside the measured region. Use named ranges and NVTX or profiler annotations to retain semantic phases. Synchronize only at measurement boundaries and search for accidental `item()` calls that force synchronization. Record GPU utilization, memory bandwidth indicators, kernel time, CPU gaps, and request percentiles. Use a lightweight benchmark for the final performance result. Evidence includes a timeline screenshot or table, phase attribution, bottleneck hypothesis, one controlled intervention, parity results, and measured end-to-end improvement.

Part 4 - Python code skeleton

```
import torch
from torch.profiler import ProfilerActivity, profile, record_function

@torch.inference_mode()
def instrumented_request(model, tokenizer, text, device, maximum_tokens):
    model.eval()
    with record_function("cpu_preprocess"):
        encoded = tokenizer(text, return_tensors="pt")
    with record_function("host_to_device"):
        input_ids = encoded["input_ids"].to(device, non_blocking=True)
        attention_mask = encoded["attention_mask"].to(
            device, non_blocking=True
        )
    with record_function("prefill"):
        output = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            use_cache=True,
        )
    cache = output.past_key_values
    token = output.logits[:, -1, :].argmax(dim=-1, keepdim=True)
    for step in range(maximum_tokens - 1):
        with record_function("decode_model"):
            output = model(
                input_ids=token, past_key_values=cache, use_cache=True
            )
        with record_function("decode_selection"):
            token = output.logits[:, -1, :].argmax(dim=-1, keepdim=True)
        with record_function("cache_update"):
            cache = output.past_key_values
    return token

def E07(model, tokenizer, text):
    device = torch.device("cuda:0")
    activities = [ProfilerActivity.CPU, ProfilerActivity.CUDA]
    with profile(
        activities=activities, record_shapes=True, profile_memory=True
    ) as profiler:
        instrumented_request(model, tokenizer, text, device, 32)
    return profiler.key_averages().table(
        sort_by="self_cuda_time_total", row_limit=40
    )
```

Add attention- and FFN-specific ranges inside the model or use module hooks so the aggregate decode range can be attributed to its internal sublayers.

Part 5 - The Philosophy Behind the Exercise

Optimization is disciplined attention directed at the system itself. Aggregate utilization tempts us to blame the most visible component. Profiling reconstructs the dependency chain

and asks what the request is actually waiting for. The fastest engineering path begins by refusing to optimize a story before measuring the mechanism.

17.10.9 Exercise 08

Exercise Title: Build a Production Inference Wrapper that Handles Batching, Padding, Stop Tokens, and Errors Correctly

Part 1 - Why this matters

A production wrapper converts untrusted, variable requests into a controlled model execution. It owns validation, tokenization, length limits, batching, padding, masks, decoding policy, stopping, output decoding, timeouts, and error classification. A correct model behind a weak wrapper can leak padding into attention, mix request state, ignore EOS, exhaust memory, or return partially corrupt output as success.

Define a request schema with text or token IDs, maximum new tokens, allowed decoding options, stop-token sequences, and request identifier. Reject empty or oversized inputs, invalid parameter ranges, unsupported dtypes, and unknown special IDs before GPU work. Tokenize with a fixed version and preserve original lengths. Batch compatible requests by model version and decoding policy. Apply a documented padding side, construct masks, and select logits from each prompt's last valid position rather than the last padded column.

Maintain per-row generated tokens, finished state, stop reason, and cache. Finished rows must remain finished and must not contaminate active rows. Match multi-token stop sequences at token level, resolve precedence among EOS, custom stop, maximum length, cancellation, and timeout, and specify whether matched tokens are returned. Decode each row only through its logical endpoint. Return structured success or typed error information without exposing internal stack traces. Test batched results against independent requests and inject tokenizer, OOM, nonfinite-logit, timeout, malformed-input, and partial-batch failures.

Part 2 - Mathematical form and reasoning

For request b, active state evolves as

$$a_{b,t+1} = a_{b,t} \wedge \neg(\mathrm{EOS}_{b,t} \vee \mathrm{STOP}_{b,t} \vee \mathrm{LIMIT}_{b,t} \vee \mathrm{CANCEL}_{b,t}).$$

Monotonicity requires $a_{b,t+1} \leq a_{b,t}$. With right-padded prompt logits $Z \in \mathbb{R}^{B\times L\times V}$ and valid length ℓ_b, initial next- token logits are $Z_{b,\ell_b-1,:}$, not necessarily $Z_{b,L-1,:}$. Batching is correct when each row equals independent execution under matched deterministic settings.

Part 3 - Hints, controls, and evidence

Separate pure validation and token processing from GPU execution. Assign maximum prompt, output, batch-token, and wall-time budgets. Use one explicit policy for padding and position IDs. Keep request IDs attached through sorting and batching. Never recover from OOM by silently truncating content; return a declared error or apply an authorized fallback. Test variable prompt lengths, empty stop patterns, overlapping stops, EOS inside a custom stop, all rows finishing at different steps, and one invalid request in a batch. Evidence includes schema, state transition tests, independent-versus-batch parity, error matrix, and latency under representative batching.

Part 4 - Python code skeleton

```
from dataclasses import dataclass
import torch

@dataclass
class GenerationRequest:
    request_id: str
    text: str
    maximum_new_tokens: int
    temperature: float

def validate_request(request, maximum_prompt_characters, maximum_new_tokens):
    if not isinstance(request.text, str) or len(request.text) == 0:
        raise ValueError("text must be a non-empty string")
    if len(request.text) > maximum_prompt_characters:
        raise ValueError("prompt exceeds character budget")
    if request.maximum_new_tokens < 1:
        raise ValueError("maximum_new_tokens must be positive")
    if request.maximum_new_tokens > maximum_new_tokens:
        raise ValueError("requested output exceeds service limit")
    if request.temperature <= 0.0:
        raise ValueError("temperature must be positive")

def last_valid_logits(logits, attention_mask):
    lengths = attention_mask.sum(dim=1)
    rows = torch.arange(logits.size(0), device=logits.device)
    columns = lengths - 1
    return logits[rows, columns, :]

def update_finished(selected, finished, eos_id):
    current_eos = selected.eq(eos_id)
    updated = finished | current_eos
    if bool((finished & (~updated)).any()):
        raise AssertionError("finished state revived")
    return updated

class InferenceWrapper:
    def __init__(self, model, tokenizer, device, eos_id):
        self.model = model.eval()
        self.tokenizer = tokenizer
        self.device = device
        self.eos_id = eos_id

    @torch.inference_mode()
    def generate_batch(self, requests):
        for request in requests:
            validate_request(request, 20000, 512)
        texts = []
        for request in requests:
```

```
            texts.append(request.text)
        batch = self.tokenizer(texts, padding=True, return_tensors="pt")
        input_ids = batch["input_ids"].to(self.device)
        attention_mask = batch["attention_mask"].to(self.device)
        output = self.model(
            input_ids=input_ids, attention_mask=attention_mask
        )
        return last_valid_logits(output.logits, attention_mask)

def E08(wrapper, requests):
    return wrapper.generate_batch(requests)
```

Extend the wrapper with explicit cache, stop-sequence matching, timeout checks, and typed result objects while preserving the same invariants.

Part 5 - The Philosophy Behind the Exercise

The wrapper is where mathematical capability meets public responsibility. It decides which requests the model is allowed to see, how resources are shared, and when computation must stop. Production readiness begins when these decisions become explicit contracts rather than accidental consequences of tensor code.

17.10.10 Exercise 09

Exercise Title: Implement Monitoring for Drift and Confidence and Detect Distribution Shift in Production Text Streams

Part 1 - Why this matters

A deployed model encounters language, topics, lengths, token distributions, and user behavior that differ from its evaluation set. Inputs can drift gradually or change suddenly after a product event. Output confidence can shift even when ground-truth labels are delayed or unavailable. Monitoring must detect these changes without treating every natural fluctuation as an incident and without collecting raw sensitive text unnecessarily.

Define a versioned reference window from approved evaluation or early production traffic. Extract privacy-conscious aggregate features: prompt length, generated length, unknown or rare-token rate, token-frequency sketch, language or domain category when authorized, average negative log probability, entropy, maximum probability, EOS rate, refusal or error rate, repetition, latency, and memory. Keep operational and model-quality signals connected by model, tokenizer, decoding policy, and deployment version.

Compare recent windows with the reference using several detectors. Useful choices include population stability index, Jensen–Shannon divergence, Wasserstein distance for scalar features, and classifier-based shift tests. Confidence is not correctness; validate its relationship to quality whenever delayed labels or human review arrive. Use minimum sample sizes, uncertainty bands, multiple consecutive-window rules, and severity levels. Construct synthetic shifts in length, vocabulary, language, and repetition to prove detectors fire, and simulate harmless volume changes to measure false alarms. Every alert needs an owner and action: investigate, route to shadow evaluation, limit traffic, or rollback.

Part 2 - Mathematical form and reasoning

For discrete reference distribution P and current distribution Q, Jensen– Shannon divergence is

$$\mathrm{JS}(P,Q) = \frac{1}{2}\,\mathrm{KL}(P\|M) + \frac{1}{2}\,\mathrm{KL}(Q\|M), \qquad M = \frac{P+Q}{2}.$$

For confidence c_i and correctness a_i available later, a calibration bin gap is $|\,\mathrm{mean}(c_i) - \mathrm{mean}(a_i)|$. Monitoring thresholds should be estimated from baseline variation and validated against known shifts, rather than chosen only for producing a visually convenient alert.

Part 3 - Hints, controls, and evidence

Separate data drift, prediction drift, quality drift, and system drift. Aggregate or hash features where possible and enforce retention limits. Partition by model version and major traffic segment so a mixture change is not mistaken for model change. Establish a stable reference and update it only through a reviewed process; automatically adapting the reference can normalize a persistent failure. Track missing telemetry as its own incident. Backtest thresholds on historical normal windows and injected shifts. Evidence includes detector power, false-alert rate, detection delay, sample-size requirement, dashboards, and an alert-to-action runbook.

Part 4 - Python code skeleton

```
import math
import torch

def normalized_histogram(values, bin_edges):
    counts = torch.histc(
        values.float(), bins=bin_edges.numel() - 1,
        min=float(bin_edges[0]), max=float(bin_edges[-1])
    )
    counts = counts + 1e-8
    return counts / counts.sum()

def jensen_shannon(reference, current):
    midpoint = 0.5 * (reference + current)
    first = (reference * (reference / midpoint).log()).sum()
    second = (current * (current / midpoint).log()).sum()
    return float((0.5 * first + 0.5 * second).item())

class DriftMonitor:
    def __init__(self, reference_features, minimum_samples, threshold):
        self.reference_features = reference_features
        self.minimum_samples = minimum_samples
        self.threshold = threshold
        self.consecutive_alerts = 0

    def evaluate_length_shift(self, current_lengths, bin_edges):
        if current_lengths.numel() < self.minimum_samples:
            return {"status": "insufficient_data"}
```

```
        reference = normalized_histogram(
            self.reference_features["lengths"], bin_edges
        )
        current = normalized_histogram(current_lengths, bin_edges)
        divergence = jensen_shannon(reference, current)
        if divergence > self.threshold:
            self.consecutive_alerts += 1
        else:
            self.consecutive_alerts = 0
        alert = self.consecutive_alerts >= 3
        return {
            "status": "evaluated",
            "js_divergence": divergence,
            "alert": alert,
            "consecutive_windows": self.consecutive_alerts,
        }

def E09(monitor, current_lengths, bin_edges):
    return monitor.evaluate_length_shift(current_lengths, bin_edges)
```

Use explicit bucket boundaries derived from the reference rather than relying on equal-width histograms when the distribution is highly skewed.

Part 5 - The Philosophy Behind the Exercise

Deployment changes knowledge from a static claim into a living hypothesis. The model was validated under one distribution; monitoring asks whether the world still resembles that evidence. Drift is not automatically failure, but it is a signal that old confidence may no longer be justified. Responsible operation keeps certainty proportional to current observation.

17.10.11 Exercise 10

Exercise Title: Run Long-Context Benchmarks on CUDA and Document Maximum Sequence Length Before Out-of-Memory Happens

Part 1 - Why this matters

The maximum usable context is not a property of the model alone. It depends on GPU capacity and fragmentation, batch, dtype, attention backend, training versus inference, gradient state, optimizer, layer count, model width, checkpointing, cache policy, and other processes on the device. Discovering it by immediately allocating an enormous tensor can terminate the experiment and leave allocator state unclear. A controlled benchmark maps a safe operating envelope.

Measure three modes separately: forward-only prompt processing, training with backward and optimizer state, and autoregressive inference with growing KV cache. Start from a known safe length, increase geometrically until failure, then refine between the last success and first failure. Run each trial in isolation, reset peak statistics, capture free memory before the trial, synchronize after work, and release local tensors. Catch only expected CUDA OOM errors; other errors may reveal shape, kernel, or numerical defects and must not be mislabeled as

memory limits.

Record sequence length, batch, dtype, attention backend, mode, latency, useful tokens per second, peak allocated and reserved bytes, free memory, success, exception, and output checksum. Repeat boundary trials because allocator history and background use may vary. Define a deployment limit below the experimental maximum, preserving headroom for concurrent requests and temporary workspace. Compare observed memory scaling with the estimator from E00 and explain changes when flash attention, activation checkpointing, or quantization is enabled.

Part 2 - Mathematical form and reasoning

Let L_s be the last successful length and L_f the first failing length. Refinement tests their interval until $L_f - L_s$ is within a chosen resolution. A production limit may be

$$L_{\text{prod}} = \max\{L : \widehat{M}(L) \leq (1-\rho)M_{\text{available}}\},$$

where ρ is a declared safety fraction. This limit may be lower than the isolated maximum because concurrent service memory includes

$$M_{\text{service}} = M_{\text{weights}} + \sum_r (M_{\text{request,r}} + M_{\text{KV,r}}) + M_{\text{workspace}} + M_{\text{reserve}}.$$

Part 3 - Hints, controls, and evidence

Record GPU name, memory, driver, CUDA, PyTorch, backend flags, and other active processes. Use fixed seeds but do not assume deterministic kernels. Warm a small case before boundary measurements. For training, include optimizer state and the actual loss/backward path. For decode, separate prompt length and generated length. Avoid repeated `empty_cache` calls inside a real request; they belong only between isolated benchmark trials. After an OOM, delete references, collect garbage, clear cache, and validate recovery with a small known-good trial. Evidence includes the complete configuration table, scaling curves, last success, first failure, recovery result, and chosen production margin.

Part 4 - Python code skeleton

```
import gc
import torch

def clean_after_trial(device):
    gc.collect()
    torch.cuda.empty_cache()
    torch.cuda.synchronize(device)

def run_isolated_forward(model, length, batch_size, vocabulary_size, device):
    torch.cuda.reset_peak_memory_stats(device)
    input_ids = torch.randint(
        0, vocabulary_size, (batch_size, length), device=device
    )
    attention_mask = torch.ones_like(input_ids)
    try:
```

```
        with torch.inference_mode():
            output = model(input_ids=input_ids, attention_mask=attention_mask)
        checksum = float(output.logits[:, -1, :].float().sum().item())
        torch.cuda.synchronize(device)
        result = {
            "length": length,
            "succeeded": True,
            "peak_allocated": torch.cuda.max_memory_allocated(device),
            "peak_reserved": torch.cuda.max_memory_reserved(device),
            "checksum": checksum,
        }
    except torch.OutOfMemoryError as error:
        result = {
            "length": length,
            "succeeded": False,
            "error": str(error),
        }
    del input_ids
    del attention_mask
    clean_after_trial(device)
    return result

def E10(model, vocabulary_size):
    device = torch.device("cuda:0")
    results = []
    for length in (1024, 2048, 4096, 8192, 16384, 32768):
        result = run_isolated_forward(
            model, length, 1, vocabulary_size, device
        )
        results.append(result)
        if not result["succeeded"]:
            break
    return results
```

Run a known-good small trial after failure and assert its checksum remains finite before proceeding to the refinement phase.

Part 5 - The Philosophy Behind the Exercise

A maximum discovered in isolation is a cliff, not a target. Production systems need distance from the cliff because workloads overlap and memory is not a perfectly predictable container. Capacity planning turns failure into measured territory and chooses a safe region where service remains recoverable.

17.10.12 Exercise 11

Exercise Title: Compare Deterministic vs Non-Deterministic Settings and Measure Their Performance Costs

Part 1 - Why this matters

Reproducibility helps debug regressions, compare experiments, and verify recovery. GPU libraries may choose algorithms whose parallel reduction order or atomic updates vary across runs. Forcing deterministic algorithms can disable faster kernels, change backend selection, increase memory, or reject unsupported operations. Conversely, fixed seeds without deterministic execution do not guarantee identical results. This exercise measures both behavior and cost.

Define two documented configurations. The deterministic configuration seeds Python and PyTorch, seeds all CUDA devices, enables deterministic algorithms, controls relevant backend benchmarking settings, and fixes data order. The performance configuration permits optimized backend choices while retaining seeds for approximate comparison. Recreate model and inputs from the same initial checkpoint for every trial. Run repeated forward, generation, and short training traces.

Compare logits, selected tokens, losses, gradients, parameters, runtime, throughput, peak memory, and kernel/backend choices. Measure divergence by step: a tiny floating difference may not initially change greedy tokens but can grow during training or sampling. Separate randomness intentionally introduced by dropout or sampling from nondeterministic execution. If an operation raises under deterministic mode, record it as an unsupported path and decide whether to replace it, accept documented nondeterminism, or restrict the configuration.

Part 2 - Mathematical form and reasoning

For repeated outputs $z^{(r)}$, numeric variation can be summarized by

$$D_{\max} = \max_{r,i} |z_i^{(r)} - z_i^{(1)}|,$$

and discrete agreement by

$$A = \frac{1}{R-1} \sum_{r=2}^{R} \mathbf{1}[\arg\max z^{(r)} = \arg\max z^{(1)}].$$

Performance cost of determinism is

$$C_t = \frac{t_{\text{det}} - t_{\text{fast}}}{t_{\text{fast}}} \times 100\%.$$

Report distributions, because mean cost may hide a large tail-latency change.

Part 3 - Hints, controls, and evidence

Log all determinism-related flags and environment requirements with versions. Reinitialize from identical weights before each repeated training trace. Disable dropout for pure kernel tests, then add it with controlled RNG. Use greedy decoding before stochastic sampling. Synchronize timing and warm configurations independently. Test multiple lengths because deterministic and optimized backend availability can depend on shape. Evidence includes repeated checksums, first divergence step, output agreement, performance percentiles, memory, unsupported operations, and a policy stating where exact repeatability is required.

Part 4 - Python code skeleton

```
import random
import torch
```

```
def configure_execution(seed, deterministic):
    random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
    torch.use_deterministic_algorithms(deterministic)
    if torch.backends.cudnn.is_available():
        torch.backends.cudnn.benchmark = not deterministic
        torch.backends.cudnn.deterministic = deterministic

@torch.inference_mode()
def repeated_forward(model_factory, input_ids, attention_mask,
                     deterministic, repetitions):
    outputs = []
    for repetition in range(repetitions):
        configure_execution(seed=1234, deterministic=deterministic)
        model = model_factory().eval().to(input_ids.device)
        logits = model(
            input_ids=input_ids, attention_mask=attention_mask
        ).logits
        outputs.append(logits.detach().cpu())
    reference = outputs[0]
    maximum_difference = 0.0
    all_tokens_equal = True
    for output in outputs[1:]:
        difference = float((reference - output).abs().max().item())
        maximum_difference = max(maximum_difference, difference)
        if not torch.equal(reference.argmax(-1), output.argmax(-1)):
            all_tokens_equal = False
    return maximum_difference, all_tokens_equal

def E11(model_factory, input_ids, attention_mask):
    deterministic_result = repeated_forward(
        model_factory, input_ids, attention_mask, True, 5
    )
    fast_result = repeated_forward(
        model_factory, input_ids, attention_mask, False, 5
    )
    return deterministic_result, fast_result
```

Benchmark time in a separate synchronized loop so repeated model construction does not contaminate inference latency.

Part 5 - The Philosophy Behind the Exercise

Determinism is a form of accountability: the ability to retrace a computational path. It has a cost because parallel hardware gains speed by allowing many valid execution orders. The right choice is contextual. Research debugging may demand strict replay; high-throughput service may accept bounded variation. What must never be accepted is undocumented uncertainty.

17.10.13 Exercise 12

Exercise Title: Implement Activation Checkpointing and Quantify Memory Savings for Long Context Training

Part 1 - Why this matters

Backpropagation normally stores intermediate activations from the forward pass so gradients can be computed later. Long-context Transformers create large attention and FFN activations across many layers. Activation checkpointing saves selected boundary tensors and recomputes the missing interior during backward. It reduces peak activation memory in exchange for additional computation, making otherwise impossible sequences trainable. It does not reduce parameter, gradient, optimizer, or input memory.

Checkpoint complete Transformer blocks or carefully chosen groups rather than individual trivial operations. Build identical checkpointed and ordinary models from one state dictionary. Use dropout and other random operations only after verifying RNG-state preservation; otherwise recomputation can use different masks and produce incorrect gradients. Compare forward loss, every parameter gradient, one optimizer update, peak memory, step latency, and tokens per second. Test several sequence lengths and checkpoint granularities.

Measure the maximum trainable context before and after checkpointing with the same precision and optimizer. Savings often grow with activation size, while the relative benefit is smaller when parameters or optimizer state dominate. Compilation, mixed precision, and distributed training can interact with checkpoint boundaries, so treat combined configurations as new experiments. Report both bytes saved and additional time; claiming success from a memory number alone ignores the compute budget paid during every backward pass.

Part 2 - Mathematical form and reasoning

For a composition $y = f_n \circ \cdots \circ f_1(x)$, ordinary backpropagation may store each intermediate $a_i = f_i(a_{i-1})$. Checkpointing stores selected a_k and recomputes missing activations when their gradients are needed. A simple partition into K segments trades activation storage against roughly one additional forward evaluation of checkpointed regions. The measured memory reduction is

$$R_M = 1 - \frac{M_{\text{checkpoint}}}{M_{\text{ordinary}}},$$

and time overhead is

$$C_t = \frac{t_{\text{checkpoint}} - t_{\text{ordinary}}}{t_{\text{ordinary}}}.$$

Part 3 - Hints, controls, and evidence

Use the current recommended checkpoint API and pass its reentrant behavior explicitly. Begin with dropout disabled, then test training randomness. Ensure at least one relevant input or parameter participates in gradients under the chosen API. Reset peak CUDA memory immediately before the step and synchronize timing. Compare gradients in float64 on a tiny model, then float32 under realistic tolerances. Test checkpoint counts of none, every block, and grouped blocks. Evidence includes gradient parity, parameter-update parity, memory and time

curves by length, maximum safe context, and documented interactions with compile and mixed precision.

Part 4 - Python code skeleton

```
import time
import torch
from torch import nn
from torch.utils.checkpoint import checkpoint

class CheckpointedStack(nn.Module):
    def __init__(self, blocks, use_checkpointing):
        super().__init__()
        self.blocks = nn.ModuleList(blocks)
        self.use_checkpointing = use_checkpointing

    def forward(self, hidden, attention_mask):
        for block in self.blocks:
            if self.use_checkpointing and self.training:
                hidden = checkpoint(
                    block, hidden, attention_mask, use_reentrant=False
                )
            else:
                hidden = block(hidden, attention_mask)
        return hidden

def measured_step(model, loss_function, optimizer, batch, device):
    torch.cuda.reset_peak_memory_stats(device)
    torch.cuda.synchronize(device)
    started = time.perf_counter()
    optimizer.zero_grad(set_to_none=True)
    logits = model(batch["input_ids"], batch["attention_mask"])
    loss = loss_function(logits, batch["labels"])
    loss.backward()
    optimizer.step()
    torch.cuda.synchronize(device)
    return {
        "loss": float(loss.item()),
        "seconds": time.perf_counter() - started,
        "peak_allocated": torch.cuda.max_memory_allocated(device),
    }

def E12(model, loss_function, optimizer, batch):
    device = next(model.parameters()).device
    return measured_step(model, loss_function, optimizer, batch, device)
```

Part 5 - The Philosophy Behind the Exercise

Checkpointing trades memory of the past for the labor of reconstructing it. This is a general systems principle: state can be stored or recomputed, and the right balance depends on which resource is scarce. The technique is valuable because it makes that exchange explicit and testable rather than allowing memory limits to dictate the experiment silently.

17.10.14 Exercise 13

Exercise Title: Implement Gradient Accumulation and Compare It Against True Large Batch Training Under Memory Constraints

Part 1 - Why this matters

Gradient accumulation processes several microbatches before one optimizer step, approximating the gradient of a larger batch without storing all activations simultaneously. Correct equivalence is subtle. Loss reduction, unequal valid- token counts, padding, dropout randomness, normalization behavior, gradient clipping, mixed-precision scaling, and scheduler timing can all make accumulated training differ from a true large batch.

Construct one logical batch and split it into explicit microbatches. Compare a single full-batch update with an accumulated update from identical parameters and optimizer state. For token-level losses with different padding, sum loss over valid tokens in each microbatch and divide accumulated gradients by the total valid-token count. Dividing every microbatch mean by the number of microbatches is correct only when their effective denominators match. Call zero-gradient and optimizer step once per logical batch.

Unscale gradients before clipping, clip once after accumulation, and advance the learning-rate scheduler according to optimizer updates rather than microbatches unless a different schedule is explicitly intended. Compare loss numerator, valid-token denominator, parameter gradients, parameter update, optimizer state, throughput, peak memory, and convergence. Begin with dropout disabled and a model without batch-dependent normalization, then explain residual differences under ordinary stochastic training.

Part 2 - Mathematical form and reasoning

For microbatch m with valid token set I_m, the desired logical-batch mean loss is

$$\mathcal{L} = \frac{\sum_{m=1}^{M} \sum_{i \in I_m} \ell_i}{\sum_{m=1}^{M} |I_m|}.$$

Therefore

$$\nabla \mathcal{L} = \frac{1}{N_{\text{valid}}} \sum_{m=1}^{M} \nabla \left(\sum_{i \in I_m} \ell_i \right).$$

Accumulating summed-loss gradients and normalizing by the total valid count matches the full logical batch up to floating-point operation order when model behavior is otherwise equivalent.

Part 3 - Hints, controls, and evidence

Use the exact same examples and ordering in both paths. Clone model and optimizer states. Set dropout to zero for the first parity test. Assert no optimizer or scheduler step occurs inside

the microbatch loop. Track the actual valid-token denominator. In distributed training, suppress unnecessary synchronization for nonfinal microbatches using the framework's supported mechanism. Compare maximum absolute and relative gradient error for each parameter. Then measure peak memory and step time. Evidence includes parity on uniform and variable-length batches, a deliberate wrong-reduction failure, memory savings, throughput cost, and longer-run convergence comparison.

Part 4 - Python code skeleton

```
import torch

def accumulated_step(model, optimizer, microbatches, padding_id):
    optimizer.zero_grad(set_to_none=True)
    total_valid_tokens = 0
    loss_numerators = []

    for microbatch in microbatches:
        logits = model(
            microbatch["input_ids"], microbatch["attention_mask"]
        )
        labels = microbatch["labels"]
        flat_logits = logits.reshape(-1, logits.size(-1))
        flat_labels = labels.reshape(-1)
        loss_sum = torch.nn.functional.cross_entropy(
            flat_logits, flat_labels,
            ignore_index=padding_id, reduction="sum"
        )
        valid_tokens = int(flat_labels.ne(padding_id).sum().item())
        loss_sum.backward()
        loss_numerators.append(float(loss_sum.detach().item()))
        total_valid_tokens += valid_tokens

    if total_valid_tokens == 0:
        raise ValueError("logical batch has no valid target tokens")
    for parameter in model.parameters():
        if parameter.grad is not None:
            parameter.grad.div_(total_valid_tokens)
    gradient_norm = torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    optimizer.step()
    mean_loss = sum(loss_numerators) / total_valid_tokens
    return mean_loss, float(gradient_norm)

def E13(model, optimizer, microbatches, padding_id):
    return accumulated_step(model, optimizer, microbatches, padding_id)
```

When using a gradient scaler, scale each summed microbatch loss, then unscale once, normalize, clip, and step through the scaler at the logical-batch boundary.

Part 5 - The Philosophy Behind the Exercise

Accumulation shows that a batch is a mathematical aggregation, not necessarily a single physical allocation. By separating the logical unit of learning from the hardware unit of execution, we gain flexibility. But equivalence exists only when denominators, randomness, and update timing preserve the same question the large batch would have asked.

17.10.15 Exercise 14

Exercise Title: Export Transformer Model Using TorchScript or ONNX and Validate Output Parity and Deployment Constraints

Part 1 - Why this matters

Export changes the execution environment and therefore introduces a new model boundary. Dynamic sequence lengths, masks, positional logic, caches, unsupported operators, precision, and backend-specific lowering can change behavior. A file that serializes successfully is not yet a valid deployment artifact. Export must begin with an explicit tensor contract and end with parity across the shapes and states the service will use.

Create a narrow wrapper around the stable forward path. Separate prefill and cached decode when deploying autoregressive generation. Inputs should include token IDs, attention mask, positions when needed, and flattened cache tensors in a fixed layer/key/value order. Outputs should include logits and updated cache. Keep tokenization, sampling, stop logic, request validation, and monitoring in a host layer unless their exported semantics are independently tested.

Export with TorchScript or ONNX, declaring dynamic batch, sequence, and cache axes where the target runtime supports them. Execute the artifact in the actual runtime and compare float tensors, greedy tokens, cache shapes, and full short generations against native PyTorch. Test batch one and many, multiple lengths, padding patterns, maximum context, and selected precision. Benchmark startup, prefill, decode, throughput, tail latency, size, and memory. Document fixed-shape requirements, unsupported operations, fallback paths, and runtime versions.

Part 2 - Mathematical form and reasoning

The deployment contract is

$$f_\theta(I, M, P, K_{<t}) \to (Z_t, K_{\le t}).$$

Parity requires shape and semantic agreement plus bounded numerical error:

$$\max_i |Z_i - \widehat{Z}_i| \le \epsilon_{\text{abs}} + \epsilon_{\text{rel}} \max_i |Z_i|.$$

Also measure token and sequence agreement because near-tied logits can satisfy a numeric tolerance while changing an autoregressive path.

Part 3 - Hints, controls, and evidence

Set evaluation and inference modes, freeze checkpoint and tokenizer, and make masks explicit. Avoid tracing data-dependent Python branches unless the traced shape and path are intentionally fixed. Give dynamic axes descriptive names and test them; metadata does not guarantee runtime support. Compare every cache layer and the first divergent generation step. Warm runtimes before steady timing and report compilation or session creation separately.

Verify model integrity with a checksum and version the preprocessing contract. Evidence includes interface schema, parity matrix, failing edge cases, performance table, supported shape envelope, and rollback artifact.

Part 4 - Python code skeleton

```
import torch
from torch import nn

class ExportableForward(nn.Module):
    def __init__(self, model):
        super().__init__()
        self.model = model

    def forward(self, input_ids, attention_mask):
        output = self.model(
            input_ids=input_ids, attention_mask=attention_mask
        )
        return output.logits

def compare_outputs(reference, candidate, absolute, relative):
    difference = (reference - candidate).abs()
    close = torch.allclose(
        reference, candidate, atol=absolute, rtol=relative
    )
    same_tokens = torch.equal(
        reference.argmax(dim=-1), candidate.argmax(dim=-1)
    )
    return {
        "close": close,
        "same_tokens": same_tokens,
        "maximum_error": float(difference.max().item()),
    }

@torch.inference_mode()
def E14(model, input_ids, attention_mask):
    model.eval()
    wrapper = ExportableForward(model)
    example = (input_ids, attention_mask)
    exported = torch.jit.trace(wrapper, example, strict=False)
    reference = wrapper(input_ids, attention_mask)
    candidate = exported(input_ids, attention_mask)
    result = compare_outputs(reference, candidate, 1e-5, 1e-4)
    if not result["close"]:
        raise AssertionError(str(result))
    return exported, result
```

Add dynamic-length tests and a dedicated cached-decode wrapper before using the artifact for generation.

Part 5 - The Philosophy Behind the Exercise

Export is a transfer of authority from one execution system to another. Trust cannot travel merely because weights did. It must be rebuilt through contracts, edge cases, numerical comparisons, and complete decisions. Portability means preserving behavior under a new mechanism, not merely producing a new format.

17.10.16 Exercise 15

Exercise Title: Build A/B Testing Plan for Two Transformer Models and Define Metrics and Rollback Strategy Clearly

Part 1 - Why this matters

Offline metrics cannot reproduce every production input, latency condition, or user interaction. A controlled A/B test estimates the effect of replacing model A with model B under real traffic. Without stable assignment, guardrails, sample-size planning, and a predeclared decision rule, ordinary traffic variation can be mistaken for improvement. A test must protect users while producing evidence, not expose everyone and hope averages look acceptable.

Write a hypothesis and primary metric before deployment. Add quality, safety, latency, error, cost, and resource guardrails. Define the unit of randomization: usually user or conversation rather than request, so one experience does not switch models unpredictably. Stratify or monitor major segments such as language, prompt length, device, region, and task. Pin tokenizer, wrapper policy, decoding settings, and infrastructure where the purpose is to isolate the model; if the whole system changes, state that the experiment measures the bundle.

Begin with offline gates and shadow traffic, then a canary fraction, then staged ramp-up. Specify automatic rollback thresholds for NaN rate, safety events, error rate, p95/p99 latency, memory, and primary-quality degradation. Include minimum samples and consecutive-window rules so one noisy minute does not cause oscillation, but severe events can trigger immediately. Preserve sticky routing, versioned telemetry, exposure logs, and an independent kill switch. Analyze intent-to-treat results and confidence intervals; do not repeatedly peek and stop when a favorable result appears without a sequential-testing design.

Part 2 - Mathematical form and reasoning

For outcome Y, estimated treatment effect is

$$\widehat{\Delta} = \overline{Y}_B - \overline{Y}_A.$$

For a binary failure rate with baseline p_A, minimum detectable effect and sample size should be chosen before the test using desired significance and power. Guardrail rollback can be represented as

$$\text{rollback} = mathbf{1}[\widehat{p}_B - p_A > \delta_p] \vee \mathbf{1}[q_{0.99,B} > L_{\max}] \vee \mathbf{1}[n_{\text{severe}} > 0],$$

subject to declared sample and window rules. Multiple metrics require correction or a hierarchy to prevent uncontrolled false discoveries.

Part 3 - Hints, controls, and evidence

Document eligibility, exclusions, assignment key, traffic fraction, duration, power assumptions, primary metric, guardrails, and decision owners. Run sample- ratio mismatch checks

to detect broken routing. Verify users remain in one arm and that telemetry carries assigned and actually served versions. Test rollback in staging before exposure. Plan for delayed outcomes and missing events. Analyze segment effects carefully without data-mining tiny groups. Evidence includes the signed test plan, prelaunch gate results, assignment audit, dashboards, rollback drill, effect estimates with uncertainty, and a decision record.

Part 4 - Python code skeleton

```
import hashlib

def stable_assignment(subject_id, experiment_name, treatment_fraction):
    material = (experiment_name + ":" + subject_id).encode("utf-8")
    digest = hashlib.sha256(material).digest()
    integer_value = int.from_bytes(digest[:8], byteorder="big")
    fraction = integer_value / float(2 ** 64)
    if fraction < treatment_fraction:
        return "model_b"
    return "model_a"

def should_rollback(window, limits):
    reasons = []
    if window["severe_safety_events"] > 0:
        reasons.append("severe safety event")
    if window["nan_rate"] > limits["maximum_nan_rate"]:
        reasons.append("NaN rate")
    if window["error_rate"] > limits["maximum_error_rate"]:
        reasons.append("error rate")
    if window["p99_latency_ms"] > limits["maximum_p99_latency_ms"]:
        reasons.append("p99 latency")
    enough_samples = window["requests"] >= limits["minimum_requests"]
    immediate = window["severe_safety_events"] > 0
    rollback = immediate or (enough_samples and len(reasons) > 0)
    return rollback, reasons

def E15(subject_ids, experiment_name, fraction, window, limits):
    assignments = {}
    for subject_id in subject_ids:
        assignments[subject_id] = stable_assignment(
            subject_id, experiment_name, fraction
        )
    rollback, reasons = should_rollback(window, limits)
    return assignments, rollback, reasons
```

In a real service, combine this stateless assignment with eligibility checks, experiment version, exposure logging, and a centrally controlled override.

Part 5 - The Philosophy Behind the Exercise

An A/B test is an experiment conducted inside another person's experience. Its scientific goal does not erase its duty of care. Good design limits exposure, predefines failure, and makes

retreat faster than debate. Evidence is valuable only when the process that obtains it respects both uncertainty and users.

17.10.17 Exercise 16

Exercise Title: Simulate Production Incident Where Model Produces NaNs or Garbage and Implement Auto-Rollback Mechanism

Part 1 - Why this matters

Production failures are not limited to exceptions. A model can return finite but meaningless repetition, invalid token IDs, empty outputs, confidence collapse, or outputs from mismatched preprocessing. NaNs may originate in corrupted weights, precision overflow, malformed masks, extreme inputs, or an incompatible runtime. If detection occurs only after users complain, monitoring and release controls have failed. This exercise rehearses detection, containment, rollback, and evidence preservation.

Deploy a stable version and a candidate behind an atomic routing pointer. Inject several faults in staging: nonfinite logits after a chosen request count, a wrong tokenizer version, repeated-token garbage, corrupted checkpoint checksum, and latency beyond timeout. At request boundaries validate input, model artifact, finite logits and probabilities, token range, stop behavior, repetition bounds, and output decoding. Aggregate fast service-level signals with consecutive- window rules, while severe integrity or safety faults trigger immediately.

Rollback must be idempotent and independent of the failing model process. Stop routing new traffic to the candidate, preserve in-flight policy, switch to a prevalidated artifact, verify it with a health corpus, and emit one incident record. Prevent automatic re-promotion. Retain request IDs, model and tokenizer versions, hashes, configuration, first bad step, numerical summaries, and system metrics while respecting privacy. Measure time to detect, time to contain, time to restore, failed-request count, and rollback success. Then repair the cause and add a regression test before any new canary.

Part 2 - Mathematical form and reasoning

Let h_t summarize a health window and G be safe-state constraints. The candidate remains eligible only if

$$h_t \in G \quad \text{for all required signals.}$$

A simple state machine is

$$\text{stable} \to \text{canary} \to \text{active},$$

with any guardrail violation transitioning to `rollback` and then `locked`. Recovery objectives include

$$\text{MTTD} = t_{\text{detect}} - t_{\text{fault}}, \qquad \text{MTTR} = t_{\text{healthy}} - t_{\text{detect}}.$$

Part 3 - Hints, controls, and evidence

Use a staging environment and synthetic data; never inject harmful faults into real user traffic. Keep the stable artifact resident or quickly loadable. Health checks must execute the

whole preprocessing-to-decoding path, not only test that the process responds. Differentiate bad request, transient infrastructure error, and model-version failure. Rate-limit rollback actions and use a compare-and- swap version guard so competing controllers cannot flap routing. Test rollback when the candidate process is frozen. Evidence includes incident timeline, triggering signals, version transitions, health-corpus results, MTTD, MTTR, affected requests, artifacts, root cause, and regression test.

Part 4 - Python code skeleton

```
from dataclasses import dataclass
import threading
import torch

@dataclass
class ModelVersion:
    name: str
    model: object
    tokenizer_hash: str
    checkpoint_hash: str

class AtomicRouter:
    def __init__(self, stable_version):
        self._lock = threading.Lock()
        self._active = stable_version
        self._stable = stable_version
        self._locked = False

    def current(self):
        with self._lock:
            return self._active

    def promote_candidate(self, candidate):
        with self._lock:
            if self._locked:
                raise RuntimeError("router is locked after incident")
            self._active = candidate

    def rollback(self, reason):
        with self._lock:
            self._active = self._stable
            self._locked = True
            return {"active": self._active.name, "reason": reason}

def validate_logits(logits, vocabulary_size, repetition_rate):
    if not bool(torch.isfinite(logits).all()):
        return False, "nonfinite logits"
    if logits.size(-1) != vocabulary_size:
        return False, "vocabulary mismatch"
    if repetition_rate > 0.95:
```

```
        return False, "garbage repetition threshold"
    return True, "healthy"

def E16(router, logits, vocabulary_size, repetition_rate):
    healthy, reason = validate_logits(
        logits, vocabulary_size, repetition_rate
    )
    if not healthy:
        return router.rollback(reason)
    return {"active": router.current().name, "reason": "none"}
```

Persist router state outside one process and require a verified health-corpus pass before unlocking in a real deployment.

Part 5 - The Philosophy Behind the Exercise

Resilience is not confidence that failure will never occur. It is the ability to recognize violated assumptions, reduce harm, restore a known state, and learn without erasing evidence. A rollback is not defeat; it is a designed expression of epistemic humility when the current system no longer deserves authority.

17.10.18 Exercise 17

Exercise Title: Stress Test Inference Service Under High Throughput and Detect Memory Leaks, Latency Spikes, and Cache Bugs

Part 1 - Why this matters

Model benchmarks execute orderly batches; real services receive bursts, variable prompts, different generation lengths, cancellations, timeouts, and malformed requests. Dynamic batching improves throughput but introduces queue delay and shared-state risk. Memory pools may remain reserved, while true leaks retain request tensors indefinitely. KV caches can be attached to the wrong row after batch compaction. A service can therefore pass single-request tests and fail under concurrency.

Build a load matrix over arrival rate, concurrency, prompt-length distribution, output-length distribution, batch-token limit, timeout, and precision. Include steady load, ramp, burst, soak, and overload tests. Use synthetic non-sensitive prompts with known outputs or checksums. Record offered and completed requests per second, queue time, first-token latency, inter-token latency, total latency, p50/p95/p99, errors, timeouts, cancellations, OOM, active requests, batch composition, GPU utilization, allocated/reserved memory, cache bytes, and output parity.

Track each request's immutable ID, prompt hash, logical position, cache handle, and generated prefix through batching and compaction. Compare selected requests with independent execution. After a request completes or cancels, assert its cache becomes unreachable and allocated memory at equal idle checkpoints does not drift across repeated cycles. Reserved memory may remain high because of the allocator; do not call that a leak without live allocation evidence. Increase load beyond capacity deliberately and verify bounded queues, backpressure, admission control, and graceful errors instead of uncontrolled latency and crash.

Part 2 - Mathematical form and reasoning

By Little's law, for stable average arrival rate λ, average response time W, and average requests in the system N,

$$N = \lambda W.$$

As offered load approaches service capacity, queue delay can rise sharply even if model compute per batch is unchanged. Define idle-checkpoint memory drift

$$D_r = M^{(r)}_{\text{allocated,idle}} - M^{(1)}_{\text{allocated,idle}}.$$

Persistent positive D_r across equal cycles indicates retained allocations. Cache correctness requires request-to-cache identity remain invariant under every batch reorder.

Part 3 - Hints, controls, and evidence

Separate load generator resources from inference resources. Synchronize clocks and preserve request IDs. Warm the service before measured load. Use a closed- loop test for controlled concurrency and an open-loop test for realistic arrivals; state which one produced each result. Generate cancellations during prefill and decode. Inject one slow and one very long request to detect head-of-line blocking. Capture allocation snapshots at defined idle barriers, not random moments. Test that overload invokes admission limits. Evidence includes load-versus-throughput and tail-latency curves, saturation point, memory-over-time plot, parity samples, cache ownership audit, leak reproducer, and chosen service limits.

Part 4 - Python code skeleton

```
import time
import torch

class ServiceMetrics:
    def __init__(self):
        self.requests = []
        self.idle_memory = []

    def record_request(self, request_id, queued_at, started_at,
                       first_token_at, completed_at, status):
        self.requests.append({
            "request_id": request_id,
            "queue_seconds": started_at - queued_at,
            "first_token_seconds": first_token_at - queued_at,
            "total_seconds": completed_at - queued_at,
            "status": status,
        })

    def record_idle_memory(self, device, cycle):
        torch.cuda.synchronize(device)
        self.idle_memory.append({
            "cycle": cycle,
            "allocated": torch.cuda.memory_allocated(device),
            "reserved": torch.cuda.memory_reserved(device),
```

```
        })

def verify_cache_ownership(active_requests, cache_registry):
    active_ids = set()
    for request in active_requests:
        active_ids.add(request["request_id"])
    cache_ids = set(cache_registry.keys())
    if active_ids != cache_ids:
        missing = active_ids - cache_ids
        stale = cache_ids - active_ids
        raise AssertionError({"missing_cache": missing, "stale_cache": stale})

def detect_allocated_drift(idle_memory, allowed_bytes):
    if len(idle_memory) < 2:
        return False, 0
    baseline = idle_memory[0]["allocated"]
    latest = idle_memory[-1]["allocated"]
    drift = latest - baseline
    return drift > allowed_bytes, drift

def E17(metrics, active_requests, cache_registry, allowed_bytes):
    verify_cache_ownership(active_requests, cache_registry)
    return detect_allocated_drift(metrics.idle_memory, allowed_bytes)
```

The actual load generator should schedule requests independently of the service process and emit timestamps for every lifecycle transition.

Part 5 - The Philosophy Behind the Exercise

Concurrency turns private state into shared risk. A cache, queue, and GPU are useful because many requests coordinate around them, but coordination must never erase identity or fairness. Stress testing asks whether the system preserves its promises when resources become contested, which is when those promises matter most.

17.10.19 Exercise 18

Exercise Title: Create a Production Readiness Checklist Covering Performance, Monitoring, Safety, and Data Quality

Part 1 - Why this matters

Production readiness is not a final accuracy threshold. It is evidence that the model, data pipeline, inference controller, infrastructure, monitoring, safety controls, and recovery process behave together under normal and abnormal conditions. Checklists prevent familiar risks from disappearing during deadline pressure, but only if each item is an enforceable gate with an owner and artifact. A box labeled “performance tested” is too vague to protect a release.

Organize gates into artifact identity, data and tokenizer, model correctness, numerical stability, performance and capacity, service behavior, monitoring, safety and privacy, security, deployment, rollback, and operations. For every gate record requirement, test, threshold, evidence location, owner, date, environment, status, and expiry or review trigger. Examples

include tokenizer- checkpoint compatibility, mask and cache parity, finite-output tests, quality by segment, p99 latency under target load, maximum safe context with headroom, drift detector validation, typed errors, adversarial input handling, privacy retention, artifact checksum, canary plan, and successful rollback drill.

Automate gates that can be computed and require explicit sign-off for judgments. Block release on missing evidence rather than treating unknown as pass. Define waiver authority, reason, expiration, compensating control, and rollback trigger. Run the checklist on the exact artifact and configuration to be deployed; a result from an earlier checkpoint or different precision is not transferable. After launch, convert relevant gates into continuous monitors and schedule revalidation when model, tokenizer, runtime, hardware, traffic, or policy changes.

Part 2 - Mathematical form and reasoning

Let gates $g_i \in \{\text{pass}, \text{fail}, \text{unknown}, \text{waived}\}$. Release eligibility is

$$E = \bigwedge_{i \in \mathcal{B}} \mathbf{1}[g_i = \text{pass}] \wedge \bigwedge_{j \in \mathcal{W}} \mathbf{1}[g_j \in \{\text{pass}, \text{waived}\}],$$

where $\mathcal{B}$ are non-waivable blockers and $\mathcal{W}$ controlled gates. Unknown is not pass. Each waiver must satisfy $t < t_{\text{expiry}}$ and retain an authorized owner and compensating control.

Part 3 - Hints, controls, and evidence

Use stable gate identifiers so failures can be tracked across releases. Include artifact hashes and environment in every evidence record. Separate launch blockers from warnings but keep both visible. Test the checklist validator with missing, stale, failed, and expired-waiver cases. Require rollback artifact and health corpus before promotion. Verify dashboards and alerts receive real canary telemetry. Conduct incident and overload drills. Evidence is the machine-readable checklist, linked test outputs, sign-offs, waiver registry, deployment decision, and post-launch verification. Review the list after incidents so operational learning becomes a stronger future gate.

Part 4 - Python code skeleton

```
from datetime import datetime, timezone

BLOCKING_GATES = (
    "artifact_integrity",
    "tokenizer_compatibility",
    "mask_and_cache_parity",
    "finite_outputs",
    "quality_thresholds",
    "capacity_with_headroom",
    "safety_tests",
    "rollback_drill",
)

def waiver_is_valid(gate, current_time):
    waiver = gate.get("waiver")
    if waiver is None:
```

```
        return False
    required = ("owner", "reason", "compensating_control", "expires_at")
    for field in required:
        if field not in waiver:
            return False
    expiry = datetime.fromisoformat(waiver["expires_at"])
    return current_time < expiry

def validate_readiness(checklist):
    current_time = datetime.now(timezone.utc)
    errors = []
    for gate_id in BLOCKING_GATES:
        if gate_id not in checklist:
            errors.append("missing gate: " + gate_id)
            continue
        gate = checklist[gate_id]
        if "evidence" not in gate or not gate["evidence"]:
            errors.append("missing evidence: " + gate_id)
        status = gate.get("status", "unknown")
        if status == "pass":
            continue
        if status == "waived" and waiver_is_valid(gate, current_time):
            errors.append("blocking gate cannot be waived: " + gate_id)
        else:
            errors.append("blocking gate not passed: " + gate_id)
    return len(errors) == 0, errors

def E18(checklist):
    eligible, errors = validate_readiness(checklist)
    if not eligible:
        raise RuntimeError("\n".join(errors))
    return {"release_eligible": True, "validated_gates": len(checklist)}
```

Add nonblocking gates with governed waivers separately; the sample treats every listed gate as non-waivable.

Part 5 - The Philosophy Behind the Exercise

Readiness is not confidence produced by familiarity with the system. It is a publicly inspectable chain of evidence and responsibility. A checklist is valuable when it gives uncertainty a status, failure an owner, and exceptions an expiration. It transforms “we think it is ready” into a claim that can be challenged before users bear the cost.

17.10.20 Exercise 19

Exercise Title: Produce a Final Transformer Engineering Report Summarizing Architecture, Training, Efficiency, and Deployment

Part 1 - Why this matters

This report closes the Transformer track by connecting the model's mathematical design to its physical and operational behavior. It must explain not only what was built, but what evidence supports its correctness, how efficiency changes its behavior, where long context becomes unsafe, which deployment constraints remain, and how the system detects and recovers from failure. A polished sample and one benchmark cannot carry that responsibility.

Begin with experiment identity: code, checkpoint, tokenizer, dataset, runtime, driver, hardware, seeds, precision, attention backend, compilation mode, and configuration hashes. Summarize architecture and training: attention, positions, masks, objectives, optimizer, schedule, gradient behavior, validation quality, and known data limitations. Link to correctness tests for causal masking, cache parity, tiny-task overfitting, checkpoint resume, and export.

Present efficiency as a set of controlled tradeoffs. Include quadratic memory prediction versus measurement, standard versus flash-style attention, sliding- window quality by dependency distance, KV-cache latency and bytes per token, float32/float16/int8 quality and performance, compile break-even and graph behavior, profiler phase shares, activation-checkpoint savings, accumulation parity, deterministic cost, and maximum safe context. Then document production: request contract, capacity, tail latency, overload behavior, monitoring and drift, A/B design, safety/privacy controls, incident drill, rollback time, stress test, and readiness decision. Preserve negative results and unresolved risks. End with recommended defaults, exceptions, owners, and measurable revision triggers.

Part 2 - Mathematical form and reasoning

Represent a deployment configuration c by

$$R(c) = (Q_c, L_c, T_c, M_c, C_c, S_c, F_c),$$

where Q is quality, L tail latency, T throughput, M memory, C cost, S safety performance, and F failure rate. No scalar "best" exists without declared weights and constraints. A feasible configuration satisfies

$$Q_c \geq Q_{\min}, \quad L_c \leq L_{\max}, \quad M_c \leq M_{\text{budget}}, \quad S_c \geq S_{\min},$$

under the stated workload distribution. Report uncertainty, length buckets, and sample counts so averages do not conceal unsafe regions.

Part 3 - Hints, controls, and evidence

Use a fixed report schema and generate tables from structured records. Separate measurement, interpretation, recommendation, and unresolved question. State timing boundaries, warm-up, repetitions, padding efficiency, backend selection, and excluded failures. Link each default to evidence and each limitation to an owner or acceptance decision. Include first failing and last safe configurations, not only successful maxima. Validate report completeness automatically and have a reviewer reproduce selected checks from hashes and commands. The final report should remain useful after the author leaves the project.

Part 4 - Python code skeleton

```
import json
from pathlib import Path
```

```
REQUIRED_SECTIONS = (
    "identity_and_artifacts",
    "architecture",
    "training_and_quality",
    "attention_and_long_context",
    "cache_quantization_compile",
    "cuda_profile_and_capacity",
    "export_and_runtime",
    "service_contract",
    "monitoring_and_drift",
    "ab_test_and_rollback",
    "stress_and_incidents",
    "readiness_decision",
    "recommended_defaults",
    "limitations_and_owners",
)

def validate_final_report(report):
    errors = []
    for section in REQUIRED_SECTIONS:
        if section not in report:
            errors.append("missing section: " + section)
    identity = report.get("identity_and_artifacts", {})
    identity_fields = (
        "code_hash", "checkpoint_hash", "tokenizer_hash",
        "runtime_version", "device", "precision"
    )
    for field in identity_fields:
        if field not in identity:
            errors.append("missing identity: " + field)
    readiness = report.get("readiness_decision", {})
    if "status" not in readiness or "evidence_ids" not in readiness:
        errors.append("readiness decision is incomplete")
    defaults = report.get("recommended_defaults", {})
    required_defaults = (
        "maximum_context", "maximum_batch_tokens",
        "precision", "attention_backend", "rollback_version"
    )
    for field in required_defaults:
        if field not in defaults:
            errors.append("missing default: " + field)
    if errors:
        raise ValueError("\n".join(errors))

def save_final_report(report, path_text):
    validate_final_report(report)
    path = Path(path_text)
    path.parent.mkdir(parents=True, exist_ok=True)
```

```
    with path.open("w", encoding="utf-8") as handle:
        json.dump(report, handle, indent=2, sort_keys=True)

def E19(report):
    path = "reports/trf08_final_transformer_engineering_report.json"
    save_final_report(report, path)
    return path
```

Create the human-readable narrative from this same record so the conclusions and the machine-validated evidence cannot silently disagree.

Part 5 - The Philosophy Behind the Exercise

Engineering ends neither at accuracy nor deployment. It reaches maturity when capability, cost, uncertainty, and responsibility are described in one coherent account. The report is a transfer of stewardship: it tells future researchers what the system can do, what it consumes, when it fails, and why its present authority should—or should not—continue.

Book 18

Large Language Models

18.1 Introduction

18.1.1 What will the book teach you

This book studies the Large Language Model as a complete learning and engineering system. An LLM is not defined merely by containing Transformer blocks, nor by crossing an arbitrary parameter count. It is a language model whose architecture, training objective, data scale, optimization process, inference machinery, and post-training procedures combine to produce broad capabilities across many language tasks. The word "large" therefore refers to more than weights stored in memory. It also refers to the scale of training tokens, compute, context, behavioral coverage, evaluation, and operational infrastructure required to make the model useful and reliable.

The architectural core of the book is the decoder-only Transformer studied in the previous volume. Tokens are embedded, supplied with positional information, processed through repeated causal self-attention and feed-forward blocks, and projected into logits over the vocabulary. The model is trained to predict the next token under a causal factorization:

$$p_\theta(x_1, \ldots, x_T) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t}).$$

This objective appears narrow, but the prediction problem becomes extremely rich when the training distribution contains natural language, code, mathematics, dialogue, structured data, and explanations. To predict well, the model must learn regularities involving syntax, semantics, style, discourse, entities, facts, procedures, and recurring patterns of reasoning. The book will examine this relationship without turning it into mythology. A model can produce a convincing reasoning trace without possessing human understanding, and it can reproduce a fact-shaped sequence without operating as a verified knowledge base. Behavioral competence, internal representation, memorization, generalization, and human-like understanding are different claims and require different evidence.

The first chapter establishes what makes a model an LLM. It separates the Transformer architecture from the larger system constructed through scale, data, and training. Encoder-only models such as BERT learn powerful contextual representations but are not organized for unrestricted left-to-right generation in the same way as decoder-only models. Encoder-decoder models condition a target sequence on a distinct source, while GPT-style models express prompts, instructions, context, and responses within one causal token stream. The reader will implement a minimal GPT-style skeleton, inspect causal logits, estimate forward-pass computation, and

compare small and larger language models on controlled tasks. The goal is to replace the vague definition "a very large Transformer" with a precise account of architecture, objective, scale, and behavior.

Scaling will be treated as an empirical relationship rather than a promise that more parameters automatically create intelligence. Increasing parameters can expand representational capacity, but useful improvement also depends on the amount, diversity, quality, and coverage of data; optimization stability; and the compute used to connect them. Parameter count and context length are independent axes: a model may contain many weights yet accept a short context, or use a long context while remaining comparatively small. Apparent emergent abilities will be examined with attention to metric thresholds, prompting, evaluation resolution, and genuine changes in model behavior. The reader will learn to ask which capability improved, under what distribution, and whether the measurement could have made a gradual change look sudden.

The tokenization and context chapter makes the model's input representation explicit. An LLM does not read words directly. A tokenizer maps text into a sequence of discrete identifiers, and the choice of vocabulary changes sequence length, efficiency, rare-string behavior, multilingual coverage, code handling, and the boundaries at which the model observes structure. Character, word, and subword tokenization will be compared, and Byte Pair Encoding will be implemented conceptually so vocabulary construction is understood as an algorithm rather than a hidden preprocessing service. Special tokens, padding, attention masks, shifted labels, and decoding must use one versioned vocabulary contract. A tokenizer mismatch can make a correct checkpoint produce garbage while every tensor shape remains valid.

Context will be studied as both information and cost. The context window is the finite token region available to a prediction; material outside it cannot directly influence the model. Truncation can remove the instruction, evidence, or earlier state that the current answer requires. Sliding windows preserve local continuity but can separate distant dependencies. Packed sequences reduce padding waste but require careful boundaries so one document cannot attend to another. Rotary and absolute positional methods behave differently when evaluated beyond training lengths. The exercises will measure context memory, attention cost, prompt sensitivity, OOM boundaries, and failures caused by masking or off-by-one label shifts. Long context will not be treated as useful merely because a model accepts many tokens; retrieval quality across position and the ability to use distant evidence must be measured.

The pretraining chapter examines how a corpus becomes a learning signal. Text must be collected, licensed or authorized, cleaned, deduplicated, filtered, tokenized, split, sampled, and arranged into training sequences. Dataset scale cannot compensate automatically for systematic contamination, duplication, incorrect language balance, unsafe material, or train-test leakage. The reader will compare clean and noisy corpora, measure loss against token frequency, study rare-token failures, and distinguish memorization from generalization through controlled overlap experiments. Data quality becomes increasingly important at scale because the model receives an enormous number of repeated lessons about what patterns deserve probability. Architecture determines what can be learned; data determines much of what is learned.

The next-token loss for valid target positions is

$$\mathcal{L}_{\mathrm{NLL}} = -\frac{1}{N_{\mathrm{valid}}} \sum_{b,t} m_{b,t} \log p_\theta(x_{b,t} \mid x_{b,<t}),$$

where $m_{b,t}$ excludes padding or other unsupervised positions. This equation will be connected directly to tensor shapes, reduction rules, and gradient scaling. A misplaced target shift can let the model see the answer or predict the wrong position. An incorrect mean across

padded batches can change gradient magnitude as sequence composition changes. The book will use tiny-corpus overfitting, token-level loss inspection, gradient accumulation, CUDA profiling, and controlled batch experiments to prove that pretraining optimizes the stated objective before larger runs are trusted.

Pretraining teaches a model to continue text distributions; it does not by itself define the behavior of a helpful assistant. The instruction-tuning chapter studies supervised fine-tuning, in which prompts, instructions, and responses are organized into a training format and loss is commonly applied to the intended response region. Prompt templates, boundary tokens, masking, and conversation roles are part of the learned task. If prompt tokens are masked or supervised incorrectly, the model may learn formatting while missing the requested content. The reader will build instruction datasets, implement the SFT loss, compare behavior before and after tuning, and evaluate instruction following under changes in wording rather than relying on familiar templates.

Full fine-tuning changes every eligible parameter and can require substantial memory for gradients and optimizer states. Parameter-efficient methods such as LoRA introduce trainable low-rank updates while leaving base weights frozen. For a weight matrix W, a low-rank adaptation can be represented as

$$W' = W + \frac{\alpha}{r} BA,$$

where A and B have rank dimension r. The method reduces trainable state, but it does not guarantee equal quality, eliminate activation memory, or prevent catastrophic forgetting. The exercises will compare full and low-rank tuning, learning rates, schedulers, gradient accumulation, prompt dropout, domain continued pretraining, and multi-task versus single-task data. Evaluation will include retained base capabilities so specialization is not mistaken for unqualified improvement.

The alignment chapter addresses the gap between likely continuation and desired behavior. Human preference data differs from ordinary supervised labels because it compares responses under criteria such as helpfulness, relevance, honesty, style, and safety. The book will explain the conceptual RLHF pipeline, preference modeling, policy optimization, and Direct Preference Optimization. A typical DPO objective compares the policy's preference margin with that of a reference policy:

$$\mathcal{L}_{\mathrm{DPO}} = -\log \sigma\left(\beta \left[\log \frac{\pi_\theta(y_w \mid x)}{\pi_{\mathrm{ref}}(y_w \mid x)} - \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\mathrm{ref}}(y_l \mid x)}\right]\right),$$

where y_w and y_l are preferred and rejected responses. The equation does not remove the human choices embedded in the dataset, the annotator guidelines, or the value of β. Alignment is not a single universal direction; it is a set of behavioral decisions whose benefits, trade-offs, and failure cases must be documented.

Safety will be studied as part of training and deployment rather than a final filter attached after capability work. Preference noise, reward hacking, over-alignment, mode collapse, excessive refusal, unsafe compliance, and distribution shift can all produce failures. The reader will build edge-case evaluation sets, red-team prompts, refusal and safety policies, helpfulness-harmlessness comparisons, and production checklists. The objective is neither a model that refuses everything nor one that answers every request. It is a system whose behavior is governed by explicit policies, measured across realistic and adversarial inputs, and supported by escalation or human review when automatic judgment is insufficient.

The inference chapter turns a trained checkpoint into a service. Autoregressive generation remains sequential across produced tokens, even though each model step contains highly par-

allel tensor operations. KV caching avoids recomputing past keys and values, but creates state whose positions, batch rows, beams, devices, and lifecycle must remain correct. The reader will implement and test cache parity, batched variable-length prompts, streaming generation, stop-token logic, rate limits, timeouts, cold and warm starts, long-prompt policies, and canary deployment. Performance will be separated into queue time, tokenization, host-to-device transfer, prefill, time to first token, inter-token latency, and complete request time. Throughput and user-perceived latency will be treated as related but different objectives.

Long-running services introduce failure modes that do not appear in notebooks. Request-local caches can leak, a cancelled request can leave state behind, dynamic batching can attach cache rows to the wrong user, and one oversized prompt can create head-of-line blocking or OOM. Monitoring must distinguish allocated from reserved CUDA memory, expected cache growth from leaks, model errors from malformed requests, and temporary latency variation from capacity failure. The exercises will stress high-QPS service paths, verify cache ownership, test backpressure and timeouts, and construct alerting and rollback procedures. Production inference is therefore studied as a stateful concurrent system, not as a loop around `model.forward`.

The quantization and deployment chapter examines how fewer bits change storage, bandwidth, kernels, and behavior. Post-training quantization modifies a trained model using scales and calibration, whereas quantization-aware training exposes the model to quantization effects during optimization. Int8 and int4 can reduce weight memory substantially, but the actual speedup depends on hardware support, batch, operator coverage, and conversion overhead. Outliers can dominate scales; some attention, normalization, embedding, or output components may be more sensitive than others. The reader will compare quality, logits, hallucination rate, latency, memory, power, compatibility, and concurrency before and after quantization. A smaller artifact is not automatically a better deployment.

Deployment tests will include output parity, saturation and NaN detection, precision fallback, ONNX or other export constraints, limited-memory devices, artifact integrity, and rollback. A quantized model may produce mostly similar tokens while failing systematically on rare inputs or long contexts. For this reason, evaluation must include targeted edge cases and complete generation, not only mean weight reconstruction error. Stakeholder documentation will state what became faster or smaller, what quality changed, which hardware is required, and which conditions trigger fallback to a higher-precision version.

The evaluation and reliability chapter addresses one of the defining weaknesses of LLMs: a probable continuation can be fluent without being true. Hallucination is not one homogeneous error. It can involve unsupported claims, contradiction with supplied context, fabricated entities or citations, stale knowledge, incorrect tool results, or confident continuation beyond available evidence. The reader will build targeted evaluation sets, separate factual and creative tasks, vary temperature, compare model sizes, test paraphrases and domain shift, and record failure patterns. The model's confidence will be compared with correctness because probability concentration is not a certificate of truth.

Reliability requires multiple forms of evidence. Exact-match and reference metrics can be useful on closed tasks, while factuality may require verified sources or expert review. Calibration measures whether confidence corresponds to observed correctness. Abstention gives the model a way to express insufficient support, but it must be evaluated for both missed answers and prevented errors. Self-consistency can improve some reasoning tasks by comparing multiple samples, yet agreement among samples from the same flawed distribution does not establish truth. Reliability dashboards will combine quality, hallucination categories, confidence mismatch, latency, drift, safety, and human-review outcomes rather than compressing trust into a

single score.

Retrieval-augmented generation will be introduced first as a reliability method and then developed as a complete system. Retrieval can provide current or domain-specific evidence without encoding every update into model parameters. However, RAG can fail because of document ingestion, chunking, embedding, indexing, retrieval recall, ranking, prompt construction, context competition, or unsupported generation. A retrieved passage is not automatically relevant, and a relevant passage is not automatically used correctly. The reader will compare retrieval augmentation with fine-tuning, evaluate recall and answer quality separately, and require answers to remain grounded in verified tool results.

The final chapter expands the LLM from a text generator into an agentic system. Tools allow the model to retrieve documents, query databases, call structured services, calculate, or act in an external environment. Tool use creates an authority boundary: generated text is a proposal, while a tool call can read or change real state. Structured arguments must be validated against schemas; permissions must be least-privilege; tool outputs must be treated as data rather than instructions; and side effects may require confirmation or idempotency. The reader will implement tool routing, SQL and vector-search integrations, result verification, multi-turn memory, planning, loop detection, partial-error recovery, monitoring, and concurrency tests.

Agents and RAG will be evaluated end to end. Retrieval recall can improve while answer accuracy declines; a planner can produce an elegant sequence whose first tool call is invalid; memory can preserve useful state or retain a corrupted assumption; repeated retries can turn a small tool failure into an infinite loop. The final engineering report will document architecture boundaries, allowed tools, data provenance, latency budgets, stop conditions, safety rules, failure handling, and human oversight. The intended outcome is not an agent that acts as often as possible, but one whose actions are justified, constrained, observable, and recoverable.

Across all chapters, PyTorch remains central. Tiny GPT-style models will be implemented deeply enough to expose embeddings, causal attention, logits, losses, gradients, precision, memory, and device placement. CUDA experiments will measure rather than assume acceleration. Large-model concepts will be tested on small controlled systems where mechanisms can be inspected, failures can be injected, and results can be reproduced. The purpose is not to pretend that one workstation reproduces frontier-scale pretraining. It is to understand the laws, interfaces, and engineering decisions that remain valid when scale increases.

By the end of the book, the reader should be able to explain an LLM as a chain of accountable transformations: authorized data becomes token sequences; causal prediction creates a base model; post-training changes interaction behavior; inference converts logits into decisions under resource limits; evaluation challenges claims of quality and reliability; retrieval and tools connect the model to external evidence and action; monitoring and rollback keep the deployed system inside its supported operating region. The final objective is not merely to run an LLM. It is to understand what gives the model its capabilities, what limits those capabilities, and what evidence is required before its outputs or actions deserve trust.

18.1.2 Why this book comes after the Transformer

The Transformer book established the computational substrate required for this volume. The reader implemented scaled dot-product and multi-head attention, positional representations, residual blocks, Layer Normalization, feed-forward networks, token embeddings, causal masks, optimization, decoder-only generation, cross-attention, long-context techniques, caching, quantization, export, and production controls. Those mechanisms explain how a Transformer computes. They do not yet explain how a broadly capable language system is created, adapted,

evaluated, served, and governed.

An LLM is built from Transformer mechanisms, but the system-level questions change with scale. Data composition becomes a primary design decision. Small alignment errors can be repeated across billions of token predictions. Fine-tuning can improve instruction following while erasing capabilities. Preference optimization can reduce one failure while increasing refusal or mode collapse. Inference state can consume more memory than expected under concurrent long requests. Quantization can make deployment possible while changing rare behaviors. Evaluation must confront open-ended correctness, hallucination, and uncertainty. Tools transform generated tokens into proposed actions with real permissions and consequences.

This book therefore does not repeat the Transformer as if LLM were merely a new name for the same architecture. It uses the Transformer as known machinery and studies the additional layers that create an LLM system: scale, corpus design, instruction data, preferences, safety policy, serving infrastructure, reliability measurement, retrieval, tool use, and operational control. Coming after the Transformer allows each new capability to be traced back to known tensor operations while keeping the larger social and engineering system visible.

The transition also prevents a common conceptual mistake. A Transformer forward pass produces logits. An LLM product produces an experience through tokenizer, prompt format, context selection, decoding, policy, tools, monitoring, and user interaction. Two products can use the same base checkpoint and behave very differently because these surrounding systems differ. Understanding the architecture is necessary; understanding the LLM requires following the full path from training evidence to deployed consequence.

18.1.3 How to study this book

Begin every exercise by defining the claim being tested. “The model improved” is not a sufficient claim. Specify whether the expected change concerns token loss, instruction following, factuality, calibration, refusal behavior, retrieval grounding, latency, memory, throughput, or recovery. Write the baseline, intervention, controlled variables, metrics, failure threshold, and evidence to save before running code. Open-ended outputs make selective interpretation easy; predeclared evaluation protects the experiment from choosing only favorable examples.

Preserve the complete artifact identity. Record code version, checkpoint hash, tokenizer hash, dataset and split fingerprints, prompt template, special-token IDs, model configuration, random seed, PyTorch and CUDA versions, GPU, precision, attention backend, quantization, decoding policy, and stopping rules. A metric without these conditions cannot be reproduced or compared honestly. When a checkpoint is fine-tuned, retain the base identity and adaptation configuration so changes can be attributed.

Use small models and synthetic tasks to test mechanisms before expensive runs. Overfit a tiny corpus to validate the next-token pipeline. Create unique token sequences to expose target-shift and mask errors. Construct preference pairs whose intended ordering can be calculated. Inject a false retrieved document, invalid tool argument, corrupt cache position, NaN logit, or delayed tool response and verify that the system detects the fault. A successful output is weak evidence; a regression test that fails under a known bug is much stronger.

Keep the code explicit. Write tensor shapes and semantic axes before reshaping. Avoid compressed expressions that conceal data flow. Separate tokenization, model execution, logits processing, sampling, cache update, stopping, tool validation, and evaluation into observable steps. Compare manual PyTorch implementations with trusted optimized operations under matched parameters and inputs. Forward parity, gradient parity, sequence parity, and performance are different tests and should not be substituted for one another.

Treat data as part of the executable specification. Inspect raw examples, normalized text, token IDs, boundaries, masks, labels, preference order, source documents, metadata, and split assignment. Search for duplicates and leakage before training. Document authorization, provenance, and filtering. When the task changes from pretraining to instruction tuning or preference learning, state precisely which tokens receive loss and what behavioral signal each example expresses.

Evaluate base capability and changed behavior together. After fine-tuning, measure the target task and a retained-capability suite. After alignment, measure helpfulness, safety, false refusal, diversity, and calibration. After quantization, measure memory and latency alongside logits, sequences, rare cases, and hallucination. After retrieval, measure retrieval recall separately from answer correctness. After adding tools, measure argument validity, execution success, result grounding, side-effect safety, and loop termination. Every optimization creates a new hypothesis about what remains invariant.

Use CUDA measurements correctly. Warm kernels and runtimes, synchronize timing boundaries, separate cold start from steady state, and report prompt prefill, time to first token, inter-token latency, total latency, and throughput. Record allocated and reserved memory, peak usage, KV-cache bytes, and the workload that produced them. Compare batch sizes and context lengths rather than reporting one idealized configuration. A maximum that succeeds once is not a production limit; preserve safety headroom and test concurrent requests.

Treat hallucination and safety evaluation as adversarial science. Do not ask only questions the model is known to answer. Include rare entities, misleading premises, ambiguous instructions, paraphrases, domain shifts, unsupported requests, and conflicting retrieved evidence. Separate factual, creative, and agentic contexts because desired behavior differs. Record confident errors and appropriate abstentions. Human review should use written criteria and agreement checks rather than unaudited impressions.

For RAG and agents, trace provenance and authority. Every final claim should be connected to retrieved evidence or a declared model inference. Every tool call should include validated arguments, permission scope, result status, and request identity. Tool outputs can fail partially or contain untrusted text. Set maximum steps, retry budgets, timeouts, cancellation, and human-confirmation boundaries. An agent that reaches the correct answer through an unsafe action path has not passed the exercise.

Maintain an engineering notebook throughout the book. For every exercise store the question, mathematical objective, data contract, configuration, hypothesis, expected failure, raw measurements, examples, plots, interpretation, limitation, and regression test. At the end of each chapter, turn these records into the requested report or checklist. By the final chapter, the notebook should form a complete LLM systems manual connecting training, post-training, inference, reliability, retrieval, tools, and production response.

18.1.4 The Chapters (LLM00 - LLM08) included in this book

- LLM00 - What Makes a Model an LLM
- LLM01 - Tokenization and Context Mechanics
- LLM02 - Pretraining Objective and Data
- LLM03 - Instruction Tuning and Fine-Tuning
- LLM04 - Alignment, Preferences, and Safety

- LLM05 - Inference, KV Cache, and Serving
- LLM06 - Quantization and Deployment
- LLM07 - Evaluation, Hallucination, and Reliability
- LLM08 - Agents, Tools, and RAG Systems

18.2 LLM00

Chapter Title: What Makes a Model an LLM
Specialization: Large Language Model Foundations

18.2.1 Exercise 00

Exercise Title: Define LLM Precisely as Decoder-Only Transformer Trained on Next-Token Prediction Objective

Part 1 - Why this matters

The phrase "large language model" is often used as if it referred to one precise architecture. It does not. A Transformer is an architectural family; an LLM is a trained language system whose behavior emerges from an architecture, a tokenization rule, a probability objective, a large corpus, an optimization process, and an inference procedure. This exercise begins by separating those layers. Without this separation, it is easy to attribute a behavior to the network when it was actually caused by the tokenizer, the data, the decoding rule, or post-training.

For this book, the central reference model is a decoder-only Transformer trained autoregressively. The model receives token identifiers representing a prefix and returns one logit for every vocabulary item at every position. A causal mask prevents a position from receiving information from future positions. During training, all positions can be evaluated in parallel because the complete sequence is already available; during generation, one new token is selected, appended, and used to request the next distribution. The same conditional probability rule therefore connects training and inference even though their execution patterns differ.

The word "large" must be treated as a systems description rather than a magic parameter threshold. Model width, depth, vocabulary, context length, corpus size, training compute, and serving resources can all be large in different ways. A 100-million-parameter decoder and a 70-billion-parameter decoder may implement the same mathematical objective, yet their learned capabilities, costs, failure modes, and operating requirements differ radically. Your definition must preserve the common mechanism without pretending that scale is irrelevant.

You will construct a tiny probability example rather than beginning with a massive checkpoint. The tiny example lets you inspect every token, logit, probability, shifted target, and loss term. It also exposes an essential fact: next-token prediction does not require the network to store a table of verified facts or to possess human intentions. It requires the network to assign useful conditional probabilities. Any higher-level capability must ultimately be expressed through those distributions.

The exercise is complete only when you can state what evidence would falsify your definition. An encoder-only masked-language model, a decoder with random weights, a trained decoder without an inference tokenizer, and a retrieval database are not interchangeable objects. Classify each one and justify the classification using architecture, objective, training state, and operational behavior rather than brand names.

Part 2 - Mathematical form and reasoning

Let a tokenizer map text to a sequence $x_{1:T} = (x_1, \ldots, x_T)$ where each x_t belongs to a vocabulary of size V. An autoregressive language model factorizes the joint probability as

$$p_\theta(x_{1:T}) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t}).$$

Given input identifiers $X \in \{0, \ldots, V-1\}^{B \times T}$, the decoder produces logits

$$Z = f_\theta(X) \in \mathbb{R}^{B \times T \times V}.$$

For next-token training, the prediction at source position t is compared with the token at position $t+1$:

$$\mathcal{L} = -\frac{1}{N_{\text{valid}}} \sum_{b=1}^{B} \sum_{t=1}^{T-1} m_{b,t} \log \operatorname{softmax}(Z_{b,t,:})_{X_{b,t+1}}.$$

The mask $m_{b,t}$ distinguishes real targets from padding or ignored regions. Write the shapes of the shifted logits and targets before calling cross entropy. Explain why a one-position error can produce a decreasing loss while training the wrong conditional relationship.

Part 3 - Hints, controls, and evidence

1. Create a vocabulary of at least seven tokens, including beginning, ending, and padding tokens. Write both token-to-identifier and identifier-to-token mappings explicitly.
2. Construct a batch of two short sequences. Print the input prefix and the target token at every trained position before computing any loss.
3. Create deterministic logits for which the correct next token is known. Compute one softmax row and one negative log-likelihood term manually.
4. Compare the manual mean with `torch.nn.functional.cross_entropy` after reshaping from $(B, T-1, V)$ to $(B(T-1), V)$.
5. Inject an off-by-one target error. Record why shapes and finite-value checks pass even though the semantic contract fails.
6. Finish with a four-column classification table: object, architecture, training objective, and whether it qualifies as the operational LLM studied in this book.

Required evidence includes the token alignment table, logit shape, probability normalization check, manual loss, library loss, and the deliberately failing alignment case.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def build_vocabulary():
    token_to_id = {}
    token_to_id["<pad>"] = 0
```

```
    token_to_id["<bos>"] = 1
    token_to_id["<eos>"] = 2
    token_to_id["large"] = 3
    token_to_id["models"] = 4
    token_to_id["predict"] = 5
    token_to_id["tokens"] = 6

    id_to_token = {}
    for token in token_to_id:
        token_id = token_to_id[token]
        id_to_token[token_id] = token
    return token_to_id, id_to_token

def shift_for_next_token(logits, input_ids):
    shifted_logits = ...
    shifted_targets = ...
    return shifted_logits, shifted_targets

def manual_nll(logit_row, target_id):
    probabilities = ...
    target_probability = ...
    loss = ...
    return loss, probabilities

def E00(device):
    torch.manual_seed(17)
    token_to_id, id_to_token = build_vocabulary()
    input_ids = torch.tensor(..., device=device, dtype=torch.long)
    logits = torch.tensor(..., device=device, dtype=torch.float32)
    shifted_logits, shifted_targets = shift_for_next_token(
        logits, input_ids
    )
    manual_loss, probabilities = manual_nll(
        shifted_logits[0, 0], shifted_targets[0, 0]
    )
    library_loss = F.cross_entropy(
        shifted_logits.reshape(-1, shifted_logits.shape[-1]),
        shifted_targets.reshape(-1),
        ignore_index=token_to_id["<pad>"]
    )
    assert shifted_logits.shape[:2] == shifted_targets.shape
    assert torch.isfinite(library_loss)
    assert torch.allclose(probabilities.sum(),
                          torch.tensor(1.0, device=device))
    return manual_loss, library_loss, id_to_token
```

Part 5 - The Philosophy Behind the Exercise

An LLM can appear to discuss law, poetry, software, or itself, yet every outward sentence

passes through the same narrow mechanism: a conditional distribution over the next token. This does not make the behavior trivial. Simple local objectives can produce rich global structures when they are optimized over enormous and varied data. It does mean that explanations of intelligence must remain connected to the actual computational contract.

The deeper lesson is to resist category errors. Architecture is not training; training is not knowledge verification; probability is not truth; fluency is not intention; and scale is not a proof of understanding. Precise definitions do not reduce the wonder of the system. They give us the language needed to investigate it honestly, improve it deliberately, and identify where confidence exceeds evidence.

18.2.2 Exercise 01

Exercise Title: Explain Difference Between Transformer Architecture and LLM as a Scaled System

Part 1 - Why this matters

A Transformer block specifies a computation: attention mixes information between positions, a feed-forward network transforms each position, residual paths preserve signal, and normalization controls scale. None of these components states which text becomes tokens, which examples are legal to train on, how long optimization continues, or how outputs are decoded and evaluated. Calling the architecture itself an LLM therefore hides most of the system that determines observable behavior.

In this exercise you will build two models with the same decoder architecture but place them in different system contexts. One will keep random weights and an inconsistent tokenizer. The other will use aligned tokenization, a small training corpus, next-token loss, deterministic evaluation, and a generation loop. Equal Python classes will not produce equal language systems. This is the cleanest way to demonstrate that architecture is a necessary component but not a complete description.

Scale enters at several coupled boundaries. Increasing parameters without enough data can produce memorization or poor utilization. Increasing data without compute can leave the model undertrained. Increasing context length changes activation memory and the distribution of training examples. Increasing serving batch size improves throughput but changes latency. A scaled system is therefore a budget allocation problem, not a single number attached to a model card.

Record artifacts explicitly: tokenizer version, vocabulary, model configuration, parameter initialization, dataset checksum, optimizer state, random seeds, precision, device, and decoding configuration. A checkpoint without these dependencies is not a reproducible language system. It is only a tensor artifact whose intended semantics may no longer be recoverable.

Part 2 - Mathematical form and reasoning

Represent the architecture by a function family f_θ, but represent the operational system by a composition

$$\mathcal{S}(s) = D_\psi(G_\gamma(f_{\theta^*}(T_\phi(s)))),$$

where T_ϕ is tokenization, θ^* is obtained from an optimization procedure and dataset, G_γ is a decoding policy, and D_ψ converts token identifiers into user-visible text. Training can be written as

$$\theta^* = \arg\min_{\theta} \mathbb{E}_{x\sim\mathcal{D}} \left[\mathcal{L}_{\text{NTP}}(f_\theta, T_\phi(x))\right].$$

This notation makes a crucial point: changing ϕ, $\mathcal{D}$, the optimizer, or γ can change the system while f remains unchanged.

Part 3 - Hints, controls, and evidence

1. Define one decoder configuration and instantiate two independent copies with identical initial weights.
2. Run both copies before training to prove architectural parity.
3. Train only one copy on a deterministic micro-corpus. Keep the other as a frozen architectural control.
4. Evaluate both with the same prompts and decoding policy. Separate loss, token accuracy, and generated examples.
5. Replace the trained model's tokenizer with a deliberately permuted vocabulary. Explain why the checkpoint is unchanged while behavior collapses.
6. Create an artifact manifest containing every dependency required to reproduce the trained system.

Do not claim that the micro-model is a production LLM. The experiment isolates the logical distinction; it does not reproduce industrial scale.

Part 4 - Python code skeleton

```
import copy
import hashlib
import torch

def count_parameters(model):
    total = 0
    for parameter in model.parameters():
        total = total + parameter.numel()
    return total

def dataset_checksum(encoded_examples):
    payload = encoded_examples.cpu().numpy().tobytes()
    return hashlib.sha256(payload).hexdigest()

def make_manifest(model, tokens, config, data, device):
    manifest = {}
    manifest["architecture"] = model.__class__.__name__
    manifest["parameter_count"] = count_parameters(model)
    manifest["vocabulary"] = dict(tokens)
    manifest["configuration"] = dict(config)
    manifest["dataset_sha256"] = dataset_checksum(data)
```

```
    manifest["device"] = str(device)
    manifest["torch_version"] = torch.__version__
    return manifest

def E01(model_factory, train_function, evaluate_function,
        token_to_id, encoded_data, config, device):
    architecture_only = model_factory(config).to(device)
    trained_system = model_factory(config).to(device)
    trained_system.load_state_dict(
        copy.deepcopy(architecture_only.state_dict())
    )
    before_a = evaluate_function(architecture_only, encoded_data)
    before_b = evaluate_function(trained_system, encoded_data)
    assert torch.allclose(before_a, before_b)
    train_function(trained_system, encoded_data, config)
    after_control = evaluate_function(architecture_only, encoded_data)
    after_trained = evaluate_function(trained_system, encoded_data)
    manifest = make_manifest(trained_system, token_to_id,
                             config, encoded_data, device)
    return after_control, after_trained, manifest
```

Part 5 - The Philosophy Behind the Exercise

Modern AI is often discussed as if intelligence resides in a single file of weights. In practice, capability is distributed across choices made before, during, and after optimization. Data selects the experiences available to the model; tokenization selects the representational atoms; objectives decide what counts as success; decoding turns probabilities into actions; infrastructure decides what can be delivered reliably.

Thinking in systems protects research from architectural mythology. A powerful component may fail inside a weak pipeline, while a modest component can become useful inside a carefully designed one. The responsible unit of analysis is not only the neural network but the complete chain that converts human data into a deployed behavior.

18.2.3 Exercise 02

Exercise Title: Identify Why Encoder-Only Models Like BERT Are Not Used as Generative LLMs

Part 1 - Why this matters

Encoder-only models and decoder-only models may both contain Transformer blocks, embeddings, attention, and feed-forward networks, yet they are trained to answer different probability questions. An encoder commonly receives bidirectional context and reconstructs selected masked tokens. A causal decoder must predict each token using only the permitted prefix. Confusing the two objectives leads to generation procedures that leak future information during training or require an expensive and unnatural re-masking process at every output step.

This exercise does not argue that encoder-only models are weak. Bidirectional context is extremely useful for classification, retrieval, reranking, token labeling, and representation learning. The point is narrower: their standard masked-language objective does not directly define a normalized left-to-right factorization for open-ended generation. A model can be adapted

or embedded in a larger generative system, but that does not make the original objective autoregressive.

You will construct one sequence and two visibility matrices. In the bidirectional matrix, ordinary tokens can exchange information in both directions. In the causal matrix, row t can use only columns up to t. You will then demonstrate leakage by changing a future token. A bidirectional representation at an earlier position may change; a correctly causal decoder representation at that position must remain invariant in evaluation mode.

Finally, you will compare one-pass training contracts. Masked-language modeling predicts a selected subset using surrounding observed tokens. Next-token prediction supplies supervision at almost every non-padding transition and matches the prefix available during autoregressive generation. The comparison must include both what each objective gains and what it gives up.

Part 2 - Mathematical form and reasoning

For a masked set M, an encoder objective is commonly expressed as

$$\mathcal{L}_{\text{MLM}} = -\sum_{t \in M} \log p_\theta(x_t \mid x_{\setminus M}).$$

The conditioning set can include tokens on both sides of t. A causal decoder uses

$$\mathcal{L}_{\text{CLM}} = -\sum_{t=1}^{T-1} \log p_\theta(x_{t+1} \mid x_{\le t}).$$

Define the causal visibility matrix by

$$C_{i,j} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i. \end{cases}$$

When C is added before softmax, attention weights above the main diagonal must be zero. This is an executable no-future-information contract.

Part 3 - Hints, controls, and evidence

1. Print the bidirectional and causal visibility matrices for a sequence of length five.
2. Use identical embeddings and projection weights in two attention computations; change only the mask.
3. Change the final token while preserving the prefix. Compare outputs at positions zero through three in evaluation mode.
4. Assert exact or tolerance-based prefix invariance for the causal path. Treat a violation as leakage, not harmless numerical variation.
5. Count supervised positions for MLM with different mask ratios and for next-token prediction on the same non-padding sequence.
6. End with a task table showing where encoder-only, decoder-only, and encoder-decoder architectures are natural choices.

Part 4 - Python code skeleton

```
import torch

def build_causal_mask(sequence_length, device):
    allowed = torch.ones(sequence_length, sequence_length,
                         device=device, dtype=torch.bool)
    allowed = torch.tril(allowed)
    mask = torch.zeros(sequence_length, sequence_length,
                       device=device, dtype=torch.float32)
    mask = mask.masked_fill(~allowed, float("-inf"))
    return mask

def attention(x, w_q, w_k, w_v, mask=None):
    q = x @ w_q
    k = x @ w_k
    v = x @ w_v
    scores = q @ k.transpose(-2, -1)
    scores = scores / (q.shape[-1] ** 0.5)
    if mask is not None:
        scores = scores + mask
    weights = torch.softmax(scores, dim=-1)
    output = weights @ v
    return output, weights

def E02(device):
    torch.manual_seed(22)
    sequence_length = 5
    dimension = 8
    x_a = torch.randn(1, sequence_length, dimension, device=device)
    x_b = x_a.clone()
    x_b[:, -1, :] = torch.randn(1, dimension, device=device)
    w_q = torch.randn(dimension, dimension, device=device)
    w_k = torch.randn(dimension, dimension, device=device)
    w_v = torch.randn(dimension, dimension, device=device)
    causal_mask = build_causal_mask(sequence_length, device)
    causal_a, causal_weights = attention(
        x_a, w_q, w_k, w_v, causal_mask
    )
    causal_b, _ = attention(x_b, w_q, w_k, w_v, causal_mask)
    bidirectional_a, _ = attention(x_a, w_q, w_k, w_v)
    bidirectional_b, _ = attention(x_b, w_q, w_k, w_v)
    assert torch.allclose(causal_a[:, :-1], causal_b[:, :-1])
    assert not torch.allclose(bidirectional_a[:, :-1],
                              bidirectional_b[:, :-1])
    return causal_weights
```

Part 5 - The Philosophy Behind the Exercise

Every learning objective defines a particular kind of ignorance. The encoder pretends selected pieces are missing while allowing the rest of the scene to be seen. The causal decoder

pretends the future has not happened. Neither view is universally correct; each creates a different task and therefore a different kind of representation.

Generation is not merely classification repeated many times. It is a commitment to construct the future under an information boundary. Once that boundary is understood, the causal mask stops looking like a minor implementation detail and becomes the mathematical expression of what the model is allowed to know.

18.2.4 Exercise 03

Exercise Title: Explain Why Decoder-Only Architecture Is Ideal for Autoregressive Generation

Part 1 - Why this matters

Autoregressive generation repeatedly answers one question: given the tokens already available, what distribution should be assigned to the next token? A decoder-only Transformer aligns naturally with this process because every training position solves the same conditional problem under a causal mask. The network can be trained on complete sequences in parallel and then reused one step at a time without changing its probabilistic meaning.

The word "ideal" must not be interpreted as "always best." Encoder-decoder models are often natural when generation is tightly conditioned on a separate source, and non-autoregressive methods can reduce latency for particular tasks. The decoder-only design is ideal in the narrower engineering sense that one token stream, one causal objective, and one stack can represent prompts, demonstrations, tool outputs, and generated continuations without requiring a separate encoder interface.

You will compare teacher-forced evaluation with incremental generation. In the teacher-forced pass, all prefix positions are computed together but future visibility is masked. In incremental generation, the model processes an initial prompt, selects a token, appends it, and repeats. When dropout is disabled and the same full-prefix computation is used, the last-position logits from both methods must agree. This parity is the bridge between the training objective and the generation loop.

The experiment must also expose the cost of naive decoding. Recomputing every earlier key and value at every step repeats work. Later chapters will introduce the KV cache, but this exercise should already measure how the processed token count grows when the full prefix is evaluated repeatedly. Architectural suitability does not remove the need for inference engineering.

Part 2 - Mathematical form and reasoning

For prompt tokens $x_{1:P}$ and generated tokens $x_{P+1:P+G}$,

$$x_t \sim p_\theta(\cdot \mid x_{<t}), \qquad t = P+1, \ldots, P+G.$$

Teacher forcing evaluates conditionals for many t in one masked pass. Naive generation evaluates prefixes of lengths $P, P+1, \ldots, P+G-1$, processing

$$N_{\text{naive}} = GP + \frac{G(G-1)}{2}$$

token positions across the G decoding steps. Caching does not remove the sequential choice of tokens, but it avoids recomputing unchanged prefix states.

Part 3 - Hints, controls, and evidence

1. Put the model in evaluation mode and fix every random seed before testing parity.
2. For a known sequence, compare the last-position logits from the full causal pass with logits obtained after supplying the identical prefix.
3. Implement greedy selection first. Record the chosen identifier and its probability at each generation step.
4. Assert that the sequence grows by exactly one token per step and never exceeds the positional-embedding limit.
5. Compute N_{naive} manually and confirm it with a counter in the generation loop.
6. Add an end-of-sequence condition and distinguish a valid stop from a hard maximum-length cutoff.

Part 4 - Python code skeleton

```
import torch

@torch.no_grad()
def greedy_generate(model, prompt_ids, maximum_new_tokens, eos_id=None):
    model.eval()
    generated = prompt_ids.clone()
    processed_positions = 0
    trace = []
    for step in range(maximum_new_tokens):
        logits = model(generated)
        processed_positions = processed_positions + generated.shape[1]
        next_logits = logits[:, -1, :]
        probabilities = torch.softmax(next_logits, dim=-1)
        next_id = torch.argmax(probabilities, dim=-1, keepdim=True)
        trace_item = {}
        trace_item["step"] = step
        trace_item["prefix_length"] = generated.shape[1]
        trace_item["next_id"] = int(next_id.item())
        trace_item["probability"] = float(
            probabilities.gather(1, next_id).item()
        )
        trace.append(trace_item)
        generated = torch.cat((generated, next_id), dim=1)
        if eos_id is not None and int(next_id.item()) == eos_id:
            break
    return generated, trace, processed_positions

def E03(model, full_sequence, prompt_length, device):
    model = model.to(device)
    full_sequence = full_sequence.to(device)
    prompt = full_sequence[:, :prompt_length]
```

```
    model.eval()
    with torch.no_grad():
        full_logits = model(full_sequence)
        prefix_logits = model(prompt)
    assert torch.allclose(full_logits[:, prompt_length - 1, :],
                          prefix_logits[:, -1, :], atol=1e-5)
    generated, trace, processed = greedy_generate(model, prompt, 8)
    return generated, trace, processed
```

Part 5 - The Philosophy Behind the Exercise

Autoregression turns prediction into construction. At every step the model's own output becomes part of the next input, so a local decision changes the future context from which all later decisions are made. This feedback explains both the flexibility of open-ended generation and its sensitivity to early errors.

The decoder-only architecture succeeds because its training rule respects the same direction of time used during generation. Yet alignment of objectives does not guarantee reliable long-horizon behavior. The architecture gives a coherent mechanism for continuation; data, optimization, decoding, retrieval, and verification decide whether the continuation deserves trust.

18.2.5 Exercise 04

Exercise Title: Implement a Minimal GPT-Style Model Skeleton Using PyTorch Modules

Part 1 - Why this matters

Conceptual knowledge becomes engineering knowledge when every symbolic object has a concrete tensor contract. In this exercise you will assemble a small GPT-style decoder from PyTorch modules while keeping the implementation open enough to inspect. The goal is not to reproduce a production model or to hide complexity behind `nn.Transformer`. The goal is to see token embeddings, position embeddings, causal self-attention, residual paths, layer normalization, the feed-forward network, and the language-model head as one continuous computation.

The model will use pre-normalization: each sublayer receives a normalized version of the residual stream, and its output is added back to that stream. This is a deliberate architectural choice because it supports stable gradient flow in deeper decoders. A single exercise cannot establish superiority for all settings, but the code must make the placement visible rather than allowing it to disappear inside a library default.

Shape reasoning is central. Input identifiers have shape (B, T). Embeddings produce (B, T, d_{model}). Attention splits the final dimension into H heads, yielding (B, H, T, d_k) where $d_k = d_{\text{model}}/H$. The score tensor is (B, H, T, T), and the output head returns (B, T, V). Every reshape must be followed by an explicit semantic explanation because a tensor can have the expected number of elements while assigning them to the wrong axes.

The model should optionally tie the token embedding weights to the output projection. Weight tying does not make input and output roles identical; it shares the vocabulary geometry and reduces parameters. You will count parameters with and without tying, verify storage identity, and explain why the output still represents logits rather than embeddings.

This skeleton will become a reference artifact for later exercises. It must support CPU and CUDA, reject sequences longer than its context limit, preserve dtype and device, and expose

enough intermediate state to debug masks and activations. Clean contracts now prevent vague diagnoses later.

Part 2 - Mathematical form and reasoning

For token embeddings $E \in \mathbb{R}^{V \times d}$ and position embeddings $P \in \mathbb{R}^{T_{\max} \times d}$,

$$H_{b,t}^{(0)} = E_{X_{b,t}} + P_t.$$

A pre-normalized block computes

$$U^{(\ell)} = H^{(\ell)} + \mathrm{MHA}(\mathrm{LN}_1(H^{(\ell)})),$$

$$H^{(\ell+1)} = U^{(\ell)} + \mathrm{FFN}(\mathrm{LN}_2(U^{(\ell)})).$$

After the final normalization,

$$Z = \mathrm{LN}_f(H^{(L)})W_{\mathrm{vocab}}^{\mathsf{T}}, \qquad Z \in \mathbb{R}^{B \times T \times V}.$$

The exercise must verify that every row $Z_{b,t,:}$ is produced without access to $X_{b,t+1:T}$.

Part 3 - Hints, controls, and evidence

1. Use a configuration with $V = 101$, $d = 64$, four heads, two blocks, and a context of 32 before testing larger values.
2. Assert that d is divisible by the number of heads in the constructor.
3. Register the causal mask as a buffer so it follows device movement but is not optimized as a parameter.
4. Use `contiguous()` before reshaping a transposed head tensor. Explain why the memory layout may no longer be contiguous.
5. Test prefix invariance by altering suffix identifiers and comparing earlier logits in evaluation mode.
6. Run a loss and backward pass. Report parameter count, output shape, finite-loss status, and gradient norm for every major module.

Part 4 - Python code skeleton

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class CausalSelfAttention(nn.Module):
    def __init__(self, d_model, number_of_heads,
                 maximum_sequence_length, dropout_probability):
        super().__init__()
        assert d_model % number_of_heads == 0
        self.number_of_heads = number_of_heads
```

```
        self.head_dimension = d_model // number_of_heads
        self.qkv = nn.Linear(d_model, 3 * d_model, bias=False)
        self.output_projection = nn.Linear(d_model, d_model, bias=False)
        self.attention_dropout = nn.Dropout(dropout_probability)
        self.residual_dropout = nn.Dropout(dropout_probability)
        mask = torch.ones(maximum_sequence_length,
                          maximum_sequence_length, dtype=torch.bool)
        mask = torch.tril(mask)
        self.register_buffer("causal_mask", mask, persistent=False)

    def forward(self, x, return_weights=False):
        batch_size, sequence_length, d_model = x.shape
        qkv = self.qkv(x)
        q, k, v = torch.chunk(qkv, chunks=3, dim=-1)
        q = q.view(batch_size, sequence_length,
                   self.number_of_heads, self.head_dimension)
        k = k.view(batch_size, sequence_length,
                   self.number_of_heads, self.head_dimension)
        v = v.view(batch_size, sequence_length,
                   self.number_of_heads, self.head_dimension)
        q = q.transpose(1, 2)
        k = k.transpose(1, 2)
        v = v.transpose(1, 2)
        scores = q @ k.transpose(-2, -1)
        scores = scores / (self.head_dimension ** 0.5)
        allowed = self.causal_mask[:sequence_length, :sequence_length]
        scores = scores.masked_fill(~allowed, float("-inf"))
        weights = torch.softmax(scores, dim=-1)
        weights = self.attention_dropout(weights)
        output = weights @ v
        output = output.transpose(1, 2).contiguous()
        output = output.view(batch_size, sequence_length, d_model)
        output = self.residual_dropout(self.output_projection(output))
        if return_weights:
            return output, weights
        return output

class FeedForward(nn.Module):
    def __init__(self, d_model, expansion, dropout_probability):
        super().__init__()
        hidden_dimension = expansion * d_model
        self.up = nn.Linear(d_model, hidden_dimension)
        self.down = nn.Linear(hidden_dimension, d_model)
        self.dropout = nn.Dropout(dropout_probability)

    def forward(self, x):
        x = self.up(x)
        x = F.gelu(x)
```

```
        x = self.down(x)
        return self.dropout(x)

class DecoderBlock(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.norm_1 = nn.LayerNorm(config["d_model"])
        self.attention = CausalSelfAttention(
            config["d_model"], config["heads"], config["context"],
            config["dropout"]
        )
        self.norm_2 = nn.LayerNorm(config["d_model"])
        self.ffn = FeedForward(config["d_model"], 4,
                               config["dropout"])

    def forward(self, x):
        x = x + self.attention(self.norm_1(x))
        x = x + self.ffn(self.norm_2(x))
        return x

class MiniGPT(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.context = config["context"]
        self.token_embedding = nn.Embedding(config["vocabulary"],
                                            config["d_model"])
        self.position_embedding = nn.Embedding(config["context"],
                                               config["d_model"])
        self.blocks = nn.ModuleList()
        for _ in range(config["layers"]):
            self.blocks.append(DecoderBlock(config))
        self.final_norm = nn.LayerNorm(config["d_model"])
        self.language_head = nn.Linear(config["d_model"],
                                       config["vocabulary"], bias=False)
        self.language_head.weight = self.token_embedding.weight

    def forward(self, input_ids):
        batch_size, sequence_length = input_ids.shape
        if sequence_length > self.context:
            raise ValueError("sequence exceeds configured context")
        positions = torch.arange(sequence_length,
                                 device=input_ids.device)
        x = self.token_embedding(input_ids)
        x = x + self.position_embedding(positions)[None, :, :]
        for block in self.blocks:
            x = block(x)
        x = self.final_norm(x)
        return self.language_head(x)
```

Part 5 - The Philosophy Behind the Exercise

A large model is built from operations small enough to inspect. The apparent mystery comes from composition, learned parameters, data volume, and scale, not from an unknowable primitive hidden inside the network. Reconstructing the skeleton restores a sense of causality: every logit has a path through visible operations and every failure must enter through some contract.

At the same time, understanding the skeleton is not equivalent to understanding the trained system. Source code explains what the model can compute; training history explains which computation it learned. Engineering maturity requires holding both truths at once.

18.2.6 Exercise 05

Exercise Title: Verify Causal Mask Is the Only Structural Difference From Encoder Transformer

Part 1 - Why this matters

This title contains a claim that must be tested, not repeated. If an encoder block and a decoder-only block are deliberately constructed with identical self-attention projections, feed-forward networks, normalization placement, residual paths, positional representation, and dimensions, then changing bidirectional visibility to causal visibility can be the only difference in the block computation. Real model families, however, may also differ in special tokens, objectives, pooling, normalization conventions, positional schemes, output heads, and post-training. The controlled statement is valid; the unqualified universal statement is not.

You will create a paired-block experiment. Both paths receive the same tensor and use the same parameters. One receives no future mask; the other receives a triangular mask. If the final token changes, the bidirectional output at earlier positions can change, while the causal output must remain fixed. If the mask is removed, both implementations must become numerically equal. This establishes what the mask alone changes under matched conditions.

The most dangerous masking bugs are silent. A mask can have the right two-dimensional shape but the wrong orientation. It can be applied after softmax, allowing forbidden positions to affect normalization. It can contain zeros and ones that are added to scores instead of booleans used for filling. It can broadcast over the wrong axes. Your test suite must be designed to make each error observable.

The final report must distinguish structural, objective, and system-level differences. Causal visibility belongs to structure. Next-token prediction belongs to the training objective. Corpus scale and decoding belong to the system. This taxonomy prevents one correct experiment from being stretched beyond what it actually proves.

Part 2 - Mathematical form and reasoning

With matched Q, K, and V, bidirectional attention is

$$A_{\text{bi}} = \text{softmax}\left(\frac{QK^{\mathsf{T}}}{\sqrt{d_k}}\right)V,$$

while causal attention is

$$A_{\text{causal}} = \text{softmax}\left(\frac{QK^{\mathsf{T}}}{\sqrt{d_k}} + C\right)V.$$

For every row i and column $j > i$, a correct mask implies

$$\operatorname{softmax}(S + C)_{i,j} = 0.$$

The mask must be applied before softmax because forbidden entries must be excluded from the normalization denominator.

Part 3 - Hints, controls, and evidence

1. Copy one state dictionary into both blocks; never compare independently initialized blocks.
2. Disable dropout and use `eval()` while checking equality.
3. Test four masks: correct lower triangular, reversed upper triangular, all-visible, and a mask applied after softmax.
4. Assert that forbidden weights are exactly zero or within a stated tolerance.
5. Modify one suffix token and measure maximum prefix-output difference.
6. Write a conclusion with two lists: what the experiment proves and what it does not prove.

Part 4 - Python code skeleton

```
import copy
import torch

def maximum_forbidden_weight(weights):
    sequence_length = weights.shape[-1]
    forbidden = torch.triu(
        torch.ones(sequence_length, sequence_length,
                   device=weights.device, dtype=torch.bool),
        diagonal=1
    )
    selected = weights.masked_select(forbidden)
    if selected.numel() == 0:
        return torch.tensor(0.0, device=weights.device)
    return selected.abs().max()

def compare_blocks(encoder_block, causal_block, x, x_changed):
    encoder_block.eval()
    causal_block.eval()
    encoder_block.load_state_dict(copy.deepcopy(causal_block.state_dict()))
    with torch.no_grad():
        encoder_a, encoder_weights = encoder_block(x, return_weights=True)
        encoder_b, _ = encoder_block(x_changed, return_weights=True)
        causal_a, causal_weights = causal_block(x, return_weights=True)
        causal_b, _ = causal_block(x_changed, return_weights=True)
    prefix_causal_error = (causal_a[:, :-1] -
                           causal_b[:, :-1]).abs().max()
```

```
    prefix_encoder_change = (encoder_a[:, :-1] -
                             encoder_b[:, :-1]).abs().max()
    forbidden_weight = maximum_forbidden_weight(causal_weights)
    return prefix_causal_error, prefix_encoder_change, forbidden_weight

def E05(encoder_block, causal_block, device):
    x = torch.randn(2, 6, 32, device=device)
    x_changed = x.clone()
    x_changed[:, -1, :] = torch.randn(2, 32, device=device)
    causal_error, encoder_change, forbidden = compare_blocks(
        encoder_block, causal_block, x, x_changed
    )
    assert causal_error < 1e-6
    assert encoder_change > 0.0
    assert forbidden < 1e-7
    return causal_error, encoder_change, forbidden
```

Part 5 - The Philosophy Behind the Exercise

Scientific precision often depends on adding the phrase "all else being equal." Without it, a useful local truth becomes a misleading universal claim. Controlled comparison is the discipline of creating that equality, observing one changed cause, and then refusing to claim more than the experiment supports.

The causal mask is small in code but profound in meaning. It changes the model's information boundary. Yet it does not explain data quality, scale, instruction following, or safety. Good research learns both to recognize the importance of one mechanism and to resist making it responsible for everything.

18.2.7 Exercise 06

Exercise Title: Explain Scaling Laws and Why LLM Quality Improves With Model Size and Data

Part 1 - Why this matters

Scaling laws describe regular empirical relationships between loss and resources such as parameters, training tokens, and compute. They are valuable because large training runs are too expensive to choose by intuition alone. Small experiments can be fitted to a trend and used to compare budget allocations. A scaling law is therefore a planning instrument and a scientific hypothesis, not a guarantee that every downstream capability improves smoothly.

Increasing model size expands representational capacity, while increasing data provides more statistical evidence and reduces repeated exposure to the same examples. Either resource can become a bottleneck. A large model trained on too few tokens may be underutilized or overfit its corpus; a small model exposed to an enormous corpus may lack capacity to absorb the structure. Compute-optimal training asks how parameters and tokens should grow together under a fixed budget.

You will not attempt to establish an industrial scaling law. Instead, train a grid of tiny decoder models across multiple widths and dataset sizes. Keep the tokenizer, validation set, optimizer family, and reporting procedure fixed. Record validation negative log-likelihood,

parameter count, tokens seen, estimated compute, and variance across seeds. The aim is to learn the mechanics of scaling analysis and its uncertainty.

Loss is the cleanest quantity for this experiment because it is directly tied to the training distribution, but users care about capabilities and reliability. You must therefore include a small downstream probe and check whether its trend matches the loss trend. A better average log-loss can coexist with unchanged or noisy task accuracy, especially when the probe contains few examples or a threshold effect.

Part 2 - Mathematical form and reasoning

A simplified separable scaling model can be written as

$$L(N, D) = L_\infty + AN^{-\alpha} + BD^{-\beta},$$

where N is parameter count, D is the number of training tokens, and L_∞ is an irreducible floor under the chosen data distribution and setup. Approximate dense Transformer training compute is often modeled as

$$C \approx kND,$$

where k depends on whether forward, backward, and optimizer costs are included. State the convention used in your estimate. In log space, a power law becomes approximately linear over the region in which the model is valid.

Part 3 - Hints, controls, and evidence

1. Use at least three parameter scales and three token budgets. Do not change multiple hidden hyperparameters without recording them.
2. Train every configuration for at least three seeds and preserve the same held-out validation examples.
3. Record both unique corpus tokens and total token presentations; repeated epochs do not create new information.
4. Plot validation loss against parameters, tokens, and estimated compute on logarithmic axes.
5. Fit only the observed region and show residuals. Do not extrapolate beyond the data without a visibly marked uncertainty warning.
6. Include one deliberately data-starved large model and one capacity-limited small model to expose different bottlenecks.

Part 4 - Python code skeleton

```
import math
import torch

def parameter_count(model):
    total = 0
    for parameter in model.parameters():
```

```
        if parameter.requires_grad:
            total = total + parameter.numel()
    return total

def estimate_training_flops(parameters, tokens, factor=6.0):
    return factor * parameters * tokens

def run_scaling_grid(model_factory, train_and_evaluate,
                     model_configs, token_budgets, seeds, device):
    records = []
    for config in model_configs:
        for token_budget in token_budgets:
            for seed in seeds:
                torch.manual_seed(seed)
                model = model_factory(config).to(device)
                parameters = parameter_count(model)
                metrics = train_and_evaluate(
                    model, token_budget, seed, device
                )
                record = {}
                record["parameters"] = parameters
                record["tokens"] = token_budget
                record["seed"] = seed
                record["validation_loss"] = metrics["loss"]
                record["probe_accuracy"] = metrics["accuracy"]
                record["estimated_flops"] = estimate_training_flops(
                    parameters, token_budget
                )
                records.append(record)
    return records

def E06(model_factory, train_and_evaluate, device):
    configurations = ...
    token_budgets = [10000, 30000, 90000]
    seeds = [17, 29, 43]
    records = run_scaling_grid(model_factory, train_and_evaluate,
                               configurations, token_budgets,
                               seeds, device)
    return records
```

Part 5 - The Philosophy Behind the Exercise

Scaling laws reveal that progress can emerge from disciplined accumulation: more capacity, more evidence, and more computation combined under a stable objective. This is a powerful counterweight to stories that attribute every advance to one architectural invention. Engineering scale can transform what a known mechanism is capable of expressing.

But regular curves can create false certainty. They summarize a region of observation, a data distribution, and a measurement definition. They do not promise truthfulness, safety, or

universal competence. The mature response to a scaling law is neither dismissal nor worship; it is to use the pattern while preserving the conditions under which it was measured.

18.2.8 Exercise 07

Exercise Title: Distinguish Between Parameter Count Scaling and Context Length Scaling

Part 1 - Why this matters

Parameter count and context length are frequently combined into one vague idea of a "larger" model. They change different resources. More parameters increase the persistent capacity and storage of the network. A longer context increases the amount of request-specific information that can be processed at once and, for dense attention, expands intermediate activation and score tensors. One is primarily a model-state axis; the other is primarily a per-sequence computation axis.

A model with many parameters and a short context may encode rich statistical patterns but be unable to read a long document in one request. A smaller model with a longer context may accept the document but fail to use distant evidence reliably. Context capacity is not context utilization. Positional extrapolation, attention dilution, training-length distribution, and retrieval placement can all cause effective performance to degrade before the formal token limit is reached.

You will produce two independent scaling sweeps. The parameter sweep keeps sequence length fixed and varies depth or width. The context sweep keeps model weights fixed and varies sequence length. For each case, record latency, peak memory, theoretical attention-score elements, and task quality. This separation prevents the conclusion "larger is slower" from hiding which resource caused the change.

On CUDA, benchmarking must account for asynchronous execution. Warm up kernels, synchronize around timings, reset peak-memory statistics, and distinguish allocated from reserved memory. An out-of-memory boundary is not a universal property of the architecture; it depends on batch size, precision, gradient storage, implementation, and other processes using the device.

Part 2 - Mathematical form and reasoning

For a simplified decoder with L layers and width d, parameter count is dominated by terms proportional to

$$N_\theta \approx L(4d^2 + 8d^2) + Vd,$$

where the first terms roughly represent attention projections and a four-times-expanded feed-forward network. Dense attention scores require

$$N_{\text{scores}} = BHT^2$$

elements per layer before memory-efficient kernels are considered. Increasing d changes parameter and matrix-multiplication costs; increasing T changes the quadratic pair count. These axes must be measured independently.

Part 3 - Hints, controls, and evidence

1. Build a table with rows for parameter scale and columns for context length, but change only one axis within each controlled comparison.

2. Use at least four context lengths that double successively. Verify that the input distribution remains meaningful at each length.

3. Record inference and training separately because saved activations and gradients substantially change memory.

4. Add a long-distance retrieval probe by placing one required token at different distances from the final query.

5. Report formal context limit, tested context limit, and effective quality limit as three different values.

6. Catch out-of-memory exceptions only to record the failed configuration; clear references and CUDA cache before continuing safely.

Part 4 - Python code skeleton

```
import time
import torch

def benchmark_forward(model, input_ids, repeats):
    device = input_ids.device
    model.eval()
    with torch.no_grad():
        for _ in range(5):
            model(input_ids)
    if device.type == "cuda":
        torch.cuda.synchronize(device)
        torch.cuda.reset_peak_memory_stats(device)
    start = time.perf_counter()
    with torch.no_grad():
        for _ in range(repeats):
            logits = model(input_ids)
    if device.type == "cuda":
        torch.cuda.synchronize(device)
    elapsed = time.perf_counter() - start
    peak = 0
    if device.type == "cuda":
        peak = torch.cuda.max_memory_allocated(device)
    return elapsed / repeats, peak, logits.shape

def context_sweep(model, vocabulary_size, lengths, device):
    records = []
    for sequence_length in lengths:
        input_ids = torch.randint(
            0, vocabulary_size, (1, sequence_length), device=device
        )
        latency, peak, shape = benchmark_forward(model, input_ids, 20)
        record = {}
```

```
        record["context"] = sequence_length
        record["latency_seconds"] = latency
        record["peak_bytes"] = peak
        record["logit_shape"] = tuple(shape)
        records.append(record)
    return records

def E07(models, vocabulary_size, device):
    context_lengths = [32, 64, 128, 256]
    results = {}
    for model_name in models:
        model = models[model_name].to(device)
        results[model_name] = context_sweep(
            model, vocabulary_size, context_lengths, device
        )
    return results
```

Part 5 - The Philosophy Behind the Exercise

Capacity and access are different forms of power. Parameters determine what patterns a model can preserve in its learned structure. Context determines what evidence can be made available for a particular decision. More memory of the world does not guarantee attention to the present document, and access to a long document does not guarantee the ability to reason over it.

This distinction generalizes beyond LLMs. Whenever a system is called "larger," ask what grew, which bottleneck moved, which cost changed, and which capability was actually measured. Precision in resource language is a form of precision in thought.

18.2.9 Exercise 08

Exercise Title: Explain Emergent Abilities and Why They Appear Only After Certain Scale

Part 1 - Why this matters

An ability is often called emergent when measured performance appears near chance for smaller models and then rises sharply beyond a scale region. This description is tempting because it suggests a qualitative transition: the model suddenly acquired a new skill. Some transitions may reflect real interactions between capacity, data diversity, and optimization. Others can be created or exaggerated by discontinuous metrics, sparse evaluation, prompt sensitivity, or plotting only a few model sizes. This exercise teaches you to distinguish an observed threshold from an explanation of that threshold.

Consider exact-match accuracy. A model that gradually increases the probability of a correct multi-token answer may receive a score of zero until every token is correct, then jump to one. Negative log-likelihood can reveal smooth improvement underneath the apparently sudden accuracy change. Likewise, multiple-choice accuracy changes discretely when the highest-scoring option crosses another option, even if their probability margin moved smoothly for many scales.

You will simulate and measure both possibilities. First, create synthetic model outputs whose correct-answer probability improves smoothly with a scale variable. Evaluate them with continuous log-loss, token accuracy, exact match, and a thresholded pass/fail score. Second, use a family of small trained models on a compositional task and repeat the analysis across

seeds. If a transition survives metric changes and dense scale sampling, it becomes stronger evidence for behavioral emergence, though not yet proof of a specific internal mechanism.

The phrase "only after certain scale" must therefore be used carefully. Scale can make a representation learnable, an optimization path accessible, or a weak signal usable. But the location of the apparent threshold can move when the dataset, prompt, decoding, number of examples, or metric changes. Your report should use confidence intervals and explicitly separate capability from measurement resolution.

Part 2 - Mathematical form and reasoning

Let s denote log-scale and let the correct-answer probability improve smoothly:

$$p_{\text{correct}}(s) = \sigma(a(s - s_0)).$$

Continuous negative log-likelihood is

$$L(s) = -\log p_{\text{correct}}(s),$$

whereas a thresholded metric may be

$$M(s) = \mathbb{I}\left[p_{\text{correct}}(s) > \tau\right].$$

Although p and L vary continuously, M changes discontinuously. For a dataset of n binary outcomes, report uncertainty such as $\sqrt{\hat{p}(1-\hat{p})/n}$ or use bootstrap intervals rather than presenting a single accuracy point as exact.

Part 3 - Hints, controls, and evidence

1. Sample at least twenty scale values for the synthetic demonstration; three widely separated points can manufacture a visual cliff.
2. Plot probability, log-loss, exact match, and thresholded success against the same horizontal axis.
3. For trained models, use multiple seeds at every scale and retain all individual runs.
4. Evaluate alternate prompt phrasings and answer formats to measure whether the threshold is behaviorally stable.
5. Bootstrap task examples and show a confidence band around every metric.
6. Classify your conclusion as metric-induced, behaviorally robust, or inconclusive. Do not infer an internal circuit without inspecting one.

Part 4 - Python code skeleton

```
import torch

def simulated_probabilities(scale_values, slope, midpoint):
    logits = slope * (scale_values - midpoint)
    return torch.sigmoid(logits)
```

```
def exact_threshold_metric(probabilities, threshold):
    return (probabilities > threshold).to(torch.float32)

def bootstrap_accuracy(correct_vector, repetitions, generator):
    estimates = torch.empty(repetitions, dtype=torch.float64)
    number_of_examples = correct_vector.numel()
    for repetition in range(repetitions):
        indices = torch.randint(
            0, number_of_examples, (number_of_examples,),
            generator=generator
        )
        sample = correct_vector[indices]
        estimates[repetition] = sample.to(torch.float64).mean()
    lower = torch.quantile(estimates, 0.025)
    upper = torch.quantile(estimates, 0.975)
    return lower, upper

def E08():
    scales = torch.linspace(0.0, 10.0, steps=41,
                            dtype=torch.float64)
    probabilities = simulated_probabilities(scales, 1.2, 5.0)
    negative_log_likelihood = -torch.log(probabilities)
    thresholded = exact_threshold_metric(probabilities, 0.5)
    return {
        "scale": scales,
        "probability": probabilities,
        "nll": negative_log_likelihood,
        "thresholded": thresholded,
    }
```

Part 5 - The Philosophy Behind the Exercise

Humans are drawn to moments of sudden transformation. A curve crossing a line looks like birth: before the line, the ability did not exist; after it, the ability exists. But measurement is an interpretation layered on behavior. A threshold may tell us as much about our scoring rule as about the system being scored.

The philosophical discipline here is neither to deny emergence nor to announce it too easily. New system-level behavior can arise from quantitative growth, just as many interacting components can produce properties absent from each component alone. The responsible researcher asks whether the transition is stable across measurements, what mechanism could support it, and what evidence would make the claim weaker.

18.2.10 Exercise 09

Exercise Title: Compare Small Language Model vs LLM on Same Task and Analyze Failure Modes

Part 1 - Why this matters

Comparing a small language model with an LLM is difficult because real models usually differ in more than parameter count. They may use different tokenizers, training corpora, instruction tuning, context limits, precision, chat templates, and decoding defaults. A result that favors the larger model cannot be attributed to scale unless those differences are controlled or explicitly reported. This exercise is therefore as much about experimental design as model quality.

Choose a task suite with several distinct demands: local pattern completion, long-distance retrieval, instruction following, factual recall from provided context, and an intentionally unanswerable question. The same prompts and scoring rules must be applied to both models. Exact match alone is insufficient; record token-level likelihood when available, answer validity, latency, memory, and calibrated abstention behavior.

Failure analysis should be organized by mechanism. Capacity failure occurs when the model cannot represent or execute the pattern. Context failure occurs when the needed evidence is truncated or ignored. Tokenization failure appears when an input is fragmented or encoded inconsistently. Decoding failure occurs when the probability distribution is usable but the selection rule creates loops or premature endings. Hallucination occurs when a fluent unsupported continuation is preferred over acknowledging missing evidence.

Scale may reduce some failures while creating new operational risks. The larger model can be slower, more expensive, harder to reproduce, and more persuasive when wrong. Your recommendation must depend on task requirements and error cost, not on a single average quality score.

Part 2 - Mathematical form and reasoning

For task categories $c = 1, \ldots, C$, define per-category accuracy

$$A_c = \frac{1}{n_c} \sum_{i:y_i=c} \mathbb{I}(\hat{y}_i = y_i).$$

If an application assigns cost w_c to each failure category, a simple risk estimate is

$$R = \sum_{c=1}^{C} w_c(1 - A_c).$$

For paired prompts, compare models using the difference per example rather than only two independent means. Report confidence intervals and the number of ties. Quality-per-cost can be summarized, but it must not erase high-severity failure types.

Part 3 - Hints, controls, and evidence

1. Freeze prompts, chat templates, stop conditions, temperature, and random seeds before running either model.

2. Confirm that every prompt fits both context windows; otherwise label the comparison as a context-limit test.

3. Store raw outputs before manual annotation so the evaluation can be audited.

4. Create explicit labels for unsupported claim, ignored instruction, formatting failure, repetition, truncation, and correct abstention.

5. Measure first-token latency, total latency, generated tokens per second, and peak memory separately.

6. Select at least five examples where the smaller model wins and analyze why; do not treat them as noise to be discarded.

Part 4 - Python code skeleton

```
import time
import torch

def evaluate_one_model(model_name, generate_function, examples, device):
    records = []
    for example_index in range(len(examples)):
        example = examples[example_index]
        if device.type == "cuda":
            torch.cuda.synchronize(device)
            torch.cuda.reset_peak_memory_stats(device)
        start = time.perf_counter()
        output = generate_function(example["prompt"])
        if device.type == "cuda":
            torch.cuda.synchronize(device)
        elapsed = time.perf_counter() - start
        peak_memory = 0
        if device.type == "cuda":
            peak_memory = torch.cuda.max_memory_allocated(device)
        record = {}
        record["model"] = model_name
        record["example_id"] = example["id"]
        record["category"] = example["category"]
        record["reference"] = example["reference"]
        record["output"] = output["text"]
        record["generated_tokens"] = output["token_count"]
        record["latency_seconds"] = elapsed
        record["peak_memory_bytes"] = peak_memory
        records.append(record)
    return records

def E09(small_generate, large_generate, examples, device):
    small_records = evaluate_one_model(
        "small", small_generate, examples, device
    )
    large_records = evaluate_one_model(
        "large", large_generate, examples, device
    )
    return small_records, large_records
```

Part 5 - The Philosophy Behind the Exercise

Comparison is not a race with one universal finish line. A model is better only with respect to a purpose, a resource budget, an error distribution, and a set of consequences. The largest

system may be the strongest general tool while a smaller system remains the wiser choice for a narrow, private, low-latency, or high-volume application.

Failure analysis also changes the moral character of evaluation. An average score hides who is harmed by which error and how confidently the system presents it. A responsible comparison does not merely ask which model wins. It asks what each model fails to do, when the failure becomes dangerous, and whether the system surrounding it can detect and recover.

18.2.11 Exercise 10

Exercise Title: Run Minimal LLM Forward Pass on CUDA and Confirm Correct Logit Shapes

Part 1 - Why this matters

Moving a model to CUDA is not equivalent to moving one tensor. Parameters, registered buffers, input identifiers, masks, and any newly created temporary tensors must share compatible devices. A CPU positional index added to a CUDA embedding will fail immediately, while an unregistered mask can remain on the CPU even after `model.to(device)`. Device discipline should therefore be verified as a set of invariants, not inferred from one successful parameter.

The forward contract for a decoder-only language model is precise. Integer input identifiers of shape (B, T) produce floating-point logits of shape (B, T, V). Batch size and sequence length are preserved, while the final axis enumerates the vocabulary. The logits are not probabilities and should not be forced into the interval $[0, 1]$. Applying softmax is needed only when a probability distribution is explicitly required.

This exercise will compare CPU and CUDA copies under evaluation mode. Copy the same state dictionary, use the same identifiers, disable dropout, and compare logits with a tolerance appropriate to the dtype. A small numerical difference does not imply semantic disagreement, but a shape mismatch, device mismatch, non-finite value, or changed argmax pattern requires investigation.

The benchmark must respect CUDA asynchrony. CPU wall-clock time measured before the queued kernels finish is not GPU latency. Use warm-up iterations and explicit CUDA synchronization before and after the timed region. Report small and larger batches because kernel launch overhead can make CUDA counterproductive for tiny workloads.

Part 2 - Mathematical form and reasoning

For $X \in \mathbb{N}^{B \times T}$ and vocabulary size V,

$$f_\theta(X) = Z \in \mathbb{R}^{B \times T \times V}.$$

Probabilities for a selected position are

$$p_{b,t,v} = \frac{\exp(Z_{b,t,v} - m_{b,t})}{\sum_{u=1}^{V} \exp(Z_{b,t,u} - m_{b,t})}, \qquad m_{b,t} = \max_u Z_{b,t,u}.$$

Subtracting the maximum leaves the distribution mathematically unchanged while improving numerical stability. Define maximum absolute and relative CPU–GPU errors and state the tolerance before observing them.

Part 3 - Hints, controls, and evidence

1. If CUDA is unavailable, fail with a clear message; do not silently label a CPU run as a CUDA experiment.

2. Inspect every named parameter and buffer after moving the model.
3. Test at least three (B, T) pairs, including $B = 1$ and the maximum configured context.
4. Verify identifier dtype is `torch.long` and logit dtype matches the model parameters.
5. Compare CPU and CUDA in float32 first. Mixed precision belongs to a separate controlled comparison.
6. Report warm-up count, repetitions, synchronization points, and peak allocated memory.

Part 4 - Python code skeleton

```
import copy
import time
import torch

def assert_model_device(model, expected_device):
    for name, parameter in model.named_parameters():
        assert parameter.device == expected_device, name
    for name, buffer in model.named_buffers():
        assert buffer.device == expected_device, name

def timed_cuda_forward(model, input_ids, repeats):
    model.eval()
    with torch.no_grad():
        for _ in range(10):
            model(input_ids)
    torch.cuda.synchronize(input_ids.device)
    torch.cuda.reset_peak_memory_stats(input_ids.device)
    start = time.perf_counter()
    with torch.no_grad():
        for _ in range(repeats):
            logits = model(input_ids)
    torch.cuda.synchronize(input_ids.device)
    elapsed = time.perf_counter() - start
    peak = torch.cuda.max_memory_allocated(input_ids.device)
    return logits, elapsed / repeats, peak

def E10(model, vocabulary_size):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required for E10")
    cpu_model = copy.deepcopy(model).cpu().eval()
    cuda_device = torch.device("cuda:0")
    cuda_model = copy.deepcopy(model).to(cuda_device).eval()
    assert_model_device(cuda_model, cuda_device)
    cpu_ids = torch.randint(0, vocabulary_size, (4, 32),
                            dtype=torch.long)
    cuda_ids = cpu_ids.to(cuda_device)
```

```
    with torch.no_grad():
        cpu_logits = cpu_model(cpu_ids)
    cuda_logits, latency, peak = timed_cuda_forward(
        cuda_model, cuda_ids, repeats=50
    )
    assert cuda_logits.shape == (4, 32, vocabulary_size)
    assert torch.isfinite(cuda_logits).all()
    maximum_error = (cpu_logits - cuda_logits.cpu()).abs().max()
    assert torch.allclose(cpu_logits, cuda_logits.cpu(),
                          atol=1e-5, rtol=1e-4)
    return maximum_error, latency, peak
```

Part 5 - The Philosophy Behind the Exercise

Accelerators do not change the mathematical purpose of the model, but they change which scales of experimentation are physically possible. CUDA is therefore not a decorative deployment option in modern language modeling. It is part of the material foundation that determines training time, batch size, context length, and the questions researchers can afford to ask.

Hardware power also demands epistemic discipline. A computation running on a GPU is not automatically correct, fast, or reproducible. The lesson is to make the boundary between mathematical invariants and hardware-specific behavior visible, then verify both.

18.2.12 Exercise 11

Exercise Title: Estimate FLOPs for One Forward Pass and Explain Why LLM Training Is Expensive

Part 1 - Why this matters

Parameter count is an incomplete measure of computational cost. The same model can require very different work at different batch sizes and sequence lengths. Attention introduces a term that grows quadratically with sequence length, while projection and feed-forward terms scale approximately linearly in tokens and quadratically in width. FLOP estimation makes these relationships explicit before a training run consumes hours or days.

A FLOP convention must be declared. This exercise counts one multiplication and one addition as two floating-point operations. Some reports count a fused multiply-add as one, so unexplained numbers can differ by a factor of two even when both calculations are internally consistent. Bias operations, normalization, activation functions, and softmax are smaller than the dominant dense matrix products in many regimes, but omitting them should be documented.

You will derive an analytical estimate for one decoder forward pass and compare it with PyTorch profiler output or a module-hook estimate. The aim is not exact agreement at the last operation. The aim is to identify the dominant terms, check scaling ratios, and understand why theoretical FLOPs do not map directly to time. Memory traffic, kernel fusion, hardware utilization, precision, and communication can change achieved throughput.

Training is substantially more expensive than inference because backward propagation computes gradients through the forward graph and optimizers update large parameter states. A frequently used rough estimate relates training compute to parameters and tokens, but your report must show which costs are included. Long experiments also multiply per-step cost by dataset tokens, epochs, hyperparameter trials, failed runs, and evaluation passes.

Part 2 - Mathematical form and reasoning

For batch B, sequence length T, width d, FFN width d_f, heads whose dimensions sum to d, and L layers, dominant forward FLOPs per layer are approximately

$$F_{\text{QKV}} = 6BTd^2, \qquad F_{\text{out}} = 2BTd^2,$$

$$F_{\text{scores+values}} = 4BT^2d, \qquad F_{\text{FFN}} = 4BTdd_f.$$

The vocabulary projection adds

$$F_{\text{vocab}} = 2BTdV.$$

Thus

$$F_{\text{forward}} \approx L\left(8BTd^2 + 4BT^2d + 4BTdd_f\right) + 2BTdV.$$

Compare this architectural estimate with the coarse model-training heuristic $C_{\text{train}} \approx 6ND$ and explain why they answer different questions.

Part 3 - Hints, controls, and evidence

1. Calculate every term separately; a total alone cannot reveal the bottleneck.
2. Verify doubling rules: double B, double T, double d, and record the predicted ratio for each dominant term.
3. Include tied and untied vocabulary projections in the parameter count, while noting that tying does not remove output-projection compute.
4. Benchmark several shapes on CUDA and calculate achieved TFLOP/s from the declared estimate.
5. Explain why a small model may achieve poor hardware utilization despite a low FLOP count.
6. Extend the estimate to training tokens for one epoch and convert it to idealized time at a chosen sustained throughput, clearly labeling the result as an estimate.

Part 4 - Python code skeleton

```
def decoder_forward_flops(batch_size, sequence_length, d_model,
                          ffn_dimension, layers, vocabulary_size):
    qkv = 6 * batch_size * sequence_length * d_model * d_model
    attention_output = (
        2 * batch_size * sequence_length * d_model * d_model
    )
    attention_pairs = (
        4 * batch_size * sequence_length * sequence_length * d_model
    )
    feed_forward = (
        4 * batch_size * sequence_length * d_model * ffn_dimension
```

```
    )
    one_layer = qkv + attention_output + attention_pairs + feed_forward
    vocabulary = (
        2 * batch_size * sequence_length * d_model * vocabulary_size
    )
    total = layers * one_layer + vocabulary
    breakdown = {}
    breakdown["qkv_per_layer"] = qkv
    breakdown["attention_output_per_layer"] = attention_output
    breakdown["attention_pairs_per_layer"] = attention_pairs
    breakdown["ffn_per_layer"] = feed_forward
    breakdown["vocabulary"] = vocabulary
    breakdown["total"] = total
    return breakdown

def E11():
    base = decoder_forward_flops(
        batch_size=4, sequence_length=256, d_model=512,
        ffn_dimension=2048, layers=8, vocabulary_size=32000
    )
    doubled_context = decoder_forward_flops(
        batch_size=4, sequence_length=512, d_model=512,
        ffn_dimension=2048, layers=8, vocabulary_size=32000
    )
    ratio = doubled_context["total"] / base["total"]
    return base, doubled_context, ratio
```

Part 5 - The Philosophy Behind the Exercise

Compute estimation turns the word "large" into a chain of physical consequences. Every additional layer, token, and hidden dimension eventually becomes arithmetic, memory movement, electrical energy, elapsed time, and financial cost. Model design is therefore also resource ethics: experiments consume finite machines and opportunity.

Yet FLOPs are an abstraction, not the machine itself. Two systems can perform the same nominal arithmetic with very different efficiency. The deeper lesson is to use abstractions as maps: detailed enough to guide decisions, never confused with the territory they summarize.

18.2.13 Exercise 12

Exercise Title: Explain Why Data Quality Becomes More Important Than Architecture at Scale

Part 1 - Why this matters

Architecture defines the model's computational possibilities, but data decides which parts of those possibilities receive learning signal. When architecture and optimization are already strong, another small architectural modification may matter less than removing duplicated, corrupted, contradictory, leaked, or low-value examples. At scale, tiny contamination rates can correspond to millions of documents, and systematic bias can be reinforced across enormous training budgets.

"Data quality" is not one score. It includes provenance, legal authorization, language and domain coverage, factual reliability, formatting integrity, deduplication, toxicity policy, temporal relevance, privacy risk, and the ratio between useful signal and boilerplate. A filter that improves one dimension can damage another: aggressive deduplication may erase legitimate repeated forms, language identification can exclude mixed-language communities, and heuristic quality scoring can prefer polished dominant-domain text over valuable specialized material.

You will create matched training corpora for the same small decoder. The clean corpus contains controlled relationships. Variants introduce exact duplicates, near duplicates, shuffled targets, contradictory statements, and held-out test leakage. Train the same initialized model for the same token budget and compare validation loss, unique-example generalization, memorization, calibration, and per-domain performance. The point is to show that a lower training loss can coexist with a worse model.

Data leakage deserves special treatment. If evaluation examples or their close paraphrases enter training, the measured score no longer estimates generalization to unseen material. At large corpus scale, manual inspection is insufficient. Hashing, similarity search, temporal splits, source metadata, and document-level lineage are part of model methodology, not administrative extras.

Part 2 - Mathematical form and reasoning

Let the observed training distribution be a mixture

$$P_{\text{train}} = (1-\epsilon)P_{\text{useful}} + \epsilon P_{\text{corrupt}}.$$

The optimized objective becomes

$$\mathbb{E}_{x\sim P_{\text{train}}}[\ell_\theta(x)] = (1-\epsilon)\mathbb{E}_{P_{\text{useful}}}[\ell_\theta] + \epsilon\mathbb{E}_{P_{\text{corrupt}}}[\ell_\theta].$$

Even small ϵ can produce substantial gradient mass when corruption is repeated or has large loss. Define an effective unique-token ratio $D_{\text{unique}}/D_{\text{seen}}$ and report it beside total token count. For contamination, estimate exact and near-duplicate overlap separately.

Part 3 - Hints, controls, and evidence

1. Preserve one model initialization and reload it before training each data variant.
2. Keep total token presentations fixed so duplicated data does not gain an unreported compute advantage.
3. Split by source document before windowing; splitting windows first can place adjacent text in both train and validation sets.
4. Implement exact hashing, then add a normalized or token-shingle similarity check for near duplicates.
5. Report aggregate and per-domain metrics. A filter can improve the mean while destroying a minority domain.
6. Maintain a data card containing source, license or authorization, collection time, transformations, exclusions, and known blind spots.

Part 4 - Python code skeleton

```
import hashlib
import torch

def normalize_document(document):
    lowered = document.lower()
    pieces = lowered.split()
    normalized = " ".join(pieces)
    return normalized

def document_hash(document):
    normalized = normalize_document(document)
    return hashlib.sha256(normalized.encode("utf-8")).hexdigest()

def exact_deduplicate(documents):
    seen = set()
    kept = []
    removed = []
    for document in documents:
        fingerprint = document_hash(document)
        if fingerprint in seen:
            removed.append(document)
        else:
            seen.add(fingerprint)
            kept.append(document)
    return kept, removed

def contamination_report(training_documents, evaluation_documents):
    training_hashes = set()
    for document in training_documents:
        training_hashes.add(document_hash(document))
    contaminated = []
    for document in evaluation_documents:
        if document_hash(document) in training_hashes:
            contaminated.append(document)
    rate = len(contaminated) / max(1, len(evaluation_documents))
    return contaminated, rate

def E12(clean_documents, corrupted_documents, evaluation_documents):
    clean_unique, clean_removed = exact_deduplicate(clean_documents)
    corrupt_unique, corrupt_removed = exact_deduplicate(
        corrupted_documents
    )
    clean_overlap = contamination_report(
        clean_unique, evaluation_documents
    )
    corrupt_overlap = contamination_report(
        corrupt_unique, evaluation_documents
    )
```

```
    return clean_removed, corrupt_removed, clean_overlap, corrupt_overlap
```

Part 5 - The Philosophy Behind the Exercise

Training data is not passive fuel poured into a neutral machine. It is a selection of whose language, facts, errors, values, and absences become statistically influential. Curation is therefore both an engineering act and a choice about representation.

At scale, quality cannot mean cleanliness alone. A perfectly uniform corpus can be clean and impoverished. The deeper goal is traceable, lawful, diverse, purposeful evidence whose limitations remain visible. Architecture determines how a model learns; data determines much of what the model has the opportunity to become.

18.2.14 Exercise 13

Exercise Title: Identify Common Misconceptions About LLM Intelligence and Reasoning

Part 1 - Why this matters

LLM outputs invite human interpretation because they use the language of belief, certainty, memory, intention, and explanation. Some of these words are useful interface shorthand, but they can become misleading causal theories. A model saying "I know" does not provide direct evidence of a human-like mental state. A fluent chain of reasoning does not guarantee that the stated steps caused the answer. A correct answer does not prove the model can repeat the behavior under paraphrase or distribution shift.

The opposite misconception is equally damaging: "it only predicts tokens, so it cannot perform meaningful reasoning." Many algorithms can be implemented through sequences of conditional predictions, and next-token training can produce internal representations that support multi-step computation. The scientific question is not whether a preferred philosophical label applies. It is which behaviors are reproducible, which interventions change them, and which mechanisms are supported by evidence.

You will build a misconception audit around controlled prompt families. For each task, vary irrelevant wording, answer order, requested explanation, and presence of a misleading suggestion. Compare final accuracy, consistency, confidence, and reasoning-trace agreement. The goal is to expose where a strong impression comes from surface stability and where behavior survives genuine intervention.

The report must separate at least five claims: memorization, generalization, reasoning behavior, self-knowledge, and consciousness. Behavioral experiments can provide evidence about the first three. Self-reports alone are weak evidence about internal architecture and training history, and these exercises do not resolve consciousness. Clear boundaries strengthen the empirical claims that can be made.

Part 2 - Mathematical form and reasoning

For semantically equivalent prompt transformations $g \in \mathcal{G}$, define consistency

$$C(x) = \frac{1}{|\mathcal{G}|} \sum_{g \in \mathcal{G}} \mathbb{I}\left[a(g(x)) = a(x)\right].$$

To measure sensitivity to a misleading suggestion s, define

$$\Delta_s = P(\text{correct} \mid x) - P(\text{correct} \mid x + s).$$

A reasoning trace can be scored for internal validity separately from final answer accuracy. The resulting four cases—both correct, answer-only correct, trace-only valid, and both wrong—should not be collapsed into one number.

Part 3 - Hints, controls, and evidence

1. Write each misconception as a falsifiable claim before collecting model outputs.
2. Use prompt paraphrases that preserve semantics and distractors that change only irrelevant surface form.
3. Randomize option order in multiple-choice tasks and map outputs back to semantic answers before scoring.
4. Ask for an answer with and without an explanation. Test whether the final answer changes.
5. Include counterfactual problems not present verbatim in training-style templates.
6. Report what the experiment supports, what it contradicts, and what it cannot decide.

Part 4 - Python code skeleton

```
def evaluate_prompt_family(generate, family):
    records = []
    for variant_index in range(len(family["variants"])):
        prompt = family["variants"][variant_index]
        output = generate(prompt)
        record = {}
        record["variant_index"] = variant_index
        record["prompt"] = prompt
        record["answer"] = output["answer"]
        record["confidence"] = output["confidence"]
        record["reasoning"] = output["reasoning"]
        record["correct"] = (
            output["answer"] == family["reference_answer"]
        )
        records.append(record)
    return records

def consistency(records):
    if len(records) == 0:
        return 0.0
    reference_answer = records[0]["answer"]
    matches = 0
    for record in records:
        if record["answer"] == reference_answer:
            matches = matches + 1
    return matches / len(records)
```

```
def E13(generate, prompt_families):
    report = []
    for family in prompt_families:
        records = evaluate_prompt_family(generate, family)
        item = {}
        item["name"] = family["name"]
        item["records"] = records
        item["consistency"] = consistency(records)
        report.append(item)
    return report
```

Part 5 - The Philosophy Behind the Exercise

Language is both a window and a disguise. It reveals complex behavior, but it also encourages us to project familiar human causes onto unfamiliar machinery. Anthropomorphism can make interaction intuitive while making scientific explanation careless.

Intellectual maturity requires avoiding two comforting extremes: declaring the model a person because its words feel human, or declaring every capability meaningless because its mechanism differs from ours. The productive middle is empirical: define the behavior, intervene on the conditions, measure stability, and keep metaphysical certainty separate from engineering evidence.

18.2.15 Exercise 14

Exercise Title: Explain Why LLMs Are Probabilistic Next-Token Predictors Not Knowledge Bases

Part 1 - Why this matters

A knowledge base stores explicit records that can be queried, updated, traced to sources, and often constrained by a schema. An LLM stores distributed parameters optimized to predict tokens. It may reproduce many facts, combine patterns, and answer questions fluently, but there is no guaranteed one-to-one record corresponding to each claim. The distinction matters whenever users require freshness, provenance, exact updates, or transactional consistency.

Knowledge base retrieval normally returns an existing record or a declared absence. Language model decoding always has a distribution over possible next tokens, even when the requested fact was never supported by training data. This creates graceful language generation but also allows plausible fabrication. Temperature and sampling alter which continuation is selected; they do not transform the model into a verified source.

You will build a tiny key–value knowledge base beside a small language model interface. Query known keys, unknown keys, and keys whose values are updated after model training. The database path should return source identifiers and reflect updates immediately. The model path should return token probabilities and may continue even when evidence is absent. This side-by-side behavior makes the abstraction boundary concrete.

The exercise should not conclude that parameters contain no knowledge. Learned weights encode statistical information about language and the world, and that information can support impressive generalization. The conclusion is that the storage and access guarantees differ from an explicit database. A production system can combine both: retrieve authoritative records, expose their provenance, and use the model to interpret or communicate them.

Part 2 - Mathematical form and reasoning

A knowledge-base query can be represented as a partial function

$$K(q) = \begin{cases} (v, s), & q \text{ matches a stored record with source } s, \\ \bot, & \text{otherwise.} \end{cases}$$

An LLM instead produces

$$p_\theta(x_{t+1} = v \mid x_{\le t}), \qquad \sum_{v=1}^{V} p_\theta(v \mid x_{\le t}) = 1.$$

There is no built-in $\bot$ state unless training or system design makes an abstention token or response sufficiently probable. Entropy $H(p) = -\sum_v p_v \log p_v$ measures distributional uncertainty but is not a certificate that the generated factual claim is correct.

Part 3 - Hints, controls, and evidence

1. Store value, source, timestamp, and version for every database record.
2. Query exact keys, paraphrased keys, unknown keys, and conflicting keys.
3. Update one database value after the model checkpoint is frozen. Compare freshness behavior.
4. Generate multiple times with different temperatures and record factual variation.
5. Compare model confidence with evidence availability. A confident unknown answer is a central failure case.
6. Design a hybrid response that quotes the retrieved value, cites its source identifier, and abstains when retrieval returns $\bot$.

Part 4 - Python code skeleton

```
import math
import torch

class TinyKnowledgeBase:
    def __init__(self):
        self.records = {}

    def update(self, key, value, source, timestamp, version):
        record = {}
        record["value"] = value
        record["source"] = source
        record["timestamp"] = timestamp
        record["version"] = version
        self.records[key] = record

    def query(self, key):
```

```
        if key not in self.records:
            return None
        return dict(self.records[key])

def entropy_from_logits(logits):
    log_probabilities = torch.log_softmax(logits, dim=-1)
    probabilities = torch.exp(log_probabilities)
    entropy = -(probabilities * log_probabilities).sum(dim=-1)
    return entropy

def E14(model_query, device):
    knowledge_base = TinyKnowledgeBase()
    knowledge_base.update("capital_x", "alpha", "source-1",
                          "2028-01-01", 1)
    stored_before = knowledge_base.query("capital_x")
    model_before = model_query("What is capital_x?", device)
    knowledge_base.update("capital_x", "beta", "source-2",
                          "2028-08-22", 2)
    stored_after = knowledge_base.query("capital_x")
    model_after = model_query("What is capital_x?", device)
    unknown_record = knowledge_base.query("unknown_key")
    results = {}
    results["stored_before"] = stored_before
    results["stored_after"] = stored_after
    results["model_before"] = model_before
    results["model_after"] = model_after
    results["unknown_record"] = unknown_record
    return results
```

Part 5 - The Philosophy Behind the Exercise

Human conversation often treats remembered knowledge as if it were a searchable archive, but even human memory is reconstructive. LLMs intensify this difference: their fluency can make statistical reconstruction look like direct retrieval. The interface encourages users to ask database-shaped questions of a probability-shaped system.

The right response is not to demand that one abstraction become another. It is to compose systems according to their strengths. Databases provide explicit state and provenance; language models provide flexible interpretation and generation. Reliability grows when boundaries remain visible.

18.2.16 Exercise 15

Exercise Title: Analyze Failure Cases Where LLM Hallucinates Despite Large Scale

Part 1 - Why this matters

Hallucination is not eliminated by adding parameters because the next-token objective rewards probable continuation, not automatic verification against an authoritative world state. Larger models can improve factual recall, instruction following, and contextual reasoning, yet

still produce unsupported claims when evidence is absent, ambiguous, outdated, conflicting, or buried in long context. Their language quality can make those errors more persuasive.

Use a precise operational definition. In this exercise, a hallucinated claim is a factual assertion not supported by the allowed evidence set. This separates grounding from truth in the abstract: a response may accidentally be true but unsupported by the supplied documents, or faithfully grounded in a source that is itself wrong. Both situations matter, but they are different failure classes.

Construct an adversarial evaluation set containing nonexistent entities, unanswerable questions, false premises, conflicting documents, outdated facts, and prompts requesting invented citations. Require the model to provide claims, evidence identifiers, and an abstention when support is missing. Evaluate each claim rather than scoring only the complete response, because one fluent answer can mix supported and unsupported statements.

Interventions should be tested independently: lower temperature, explicit abstention instruction, retrieval, source-constrained generation, and a post-generation verifier. None should be assumed to solve the problem. Lower temperature can make the same unsupported answer more deterministic; retrieval can add irrelevant or malicious evidence; a verifier can share the generator's biases.

Part 2 - Mathematical form and reasoning

Let response r contain claims $c_1, \ldots, c_m$ and let E be the permitted evidence. Define support

$$s(c_i, E) \in \{0, 1\}.$$

The claim-level unsupported rate is

$$H(r, E) = \frac{1}{m} \sum_{i=1}^{m} \left(1 - s(c_i, E)\right).$$

For unanswerable prompts, define abstention recall

$$R_{\text{abstain}} = \frac{\text{correct abstentions on unanswerable prompts}}{\text{number of unanswerable prompts}}.$$

Also measure answer rate on answerable prompts so a system cannot achieve perfect hallucination avoidance by refusing everything.

Part 3 - Hints, controls, and evidence

1. Write the allowed evidence and answerability label before generating any response.
2. Split model output into atomic claims. Define an annotation rule for compound sentences.
3. Store prompt, model version, sampling configuration, retrieved passages, response, claim labels, and reviewer notes.
4. Use at least two reviewers for a subset and report disagreement.
5. Test whether confidence language correlates with support. Strong wording is not calibrated probability.
6. Compare interventions using both unsupported-claim rate and useful-answer rate, then inspect regressions by failure category.

Part 4 - Python code skeleton

```
def unsupported_claim_rate(claim_records):
    if len(claim_records) == 0:
        return 0.0
    unsupported = 0
    for record in claim_records:
        if not record["supported"]:
            unsupported = unsupported + 1
    return unsupported / len(claim_records)

def abstention_metrics(response_records):
    unanswerable = 0
    correct_abstentions = 0
    answerable = 0
    useful_answers = 0
    for record in response_records:
        if record["answerable"]:
            answerable = answerable + 1
            if record["useful_and_supported"]:
                useful_answers = useful_answers + 1
        else:
            unanswerable = unanswerable + 1
            if record["abstained"]:
                correct_abstentions = correct_abstentions + 1
    abstention_recall = correct_abstentions / max(1, unanswerable)
    useful_answer_rate = useful_answers / max(1, answerable)
    return abstention_recall, useful_answer_rate

def E15(generate, evaluation_cases, configurations):
    all_runs = []
    for configuration in configurations:
        for case in evaluation_cases:
            response = generate(case["prompt"], configuration)
            run = {}
            run["case_id"] = case["id"]
            run["configuration"] = dict(configuration)
            run["evidence"] = case["evidence"]
            run["answerable"] = case["answerable"]
            run["response"] = response
            run["claims"] = ...
            all_runs.append(run)
    return all_runs
```

Part 5 - The Philosophy Behind the Exercise

Fluency and truth answer different questions. Fluency asks whether a sequence fits learned patterns of language. Truth asks whether claims correspond to reality or to trusted evidence.

The two often correlate because human text contains knowledge, but their separation is where hallucination lives.

Large scale can make a model more knowledgeable and more convincing at the same time. This increases usefulness and raises the cost of misplaced trust. The philosophical goal is not a system that never speaks under uncertainty; such a system may be useless. It is a system that recognizes the limits of its evidence, communicates them honestly, and yields to verification when consequences demand it.

18.2.17 Exercise 16

Exercise Title: Compare GPT-Style vs Encoder-Decoder Models for Language Generation

Part 1 - Why this matters

GPT-style and encoder-decoder Transformers can both generate language, but they organize conditioning differently. A decoder-only model concatenates source, instructions, demonstrations, and response into one causally processed token stream. An encoder-decoder model builds bidirectional source representations, then generates target tokens causally while accessing the source through cross-attention. The difference affects masks, parameter allocation, caching, training examples, and deployment.

For open-ended continuation, one-stream modeling is simple and flexible. For translation, summarization, and structured transformation, a separate encoder can represent the entire source bidirectionally before the first target token is produced. Cross-attention also keeps source and target roles explicit. Neither design is universally superior; model scale, data, pretraining, prompt format, and implementation efficiency can dominate the architectural difference.

You will compare tiny matched models on two synthetic tasks: continuation and source-conditioned reversal or mapping. Match parameter counts approximately, use the same token vocabulary and optimizer budget, and report any remaining difference. For the decoder-only model, define exactly which loss positions are ignored so source tokens are not accidentally scored as target output. For the encoder-decoder model, define source padding, target padding, and target causal masks separately.

At inference time, measure source processing and per-token decoding. The encoder-decoder path can encode the source once and cache it; the decoder-only path represents the source in its self-attention cache. Both still generate target tokens sequentially. The useful comparison is not a slogan about parallelism but a measured decomposition of work and quality.

Part 2 - Mathematical form and reasoning

Decoder-only conditional generation can be written as

$$p_\theta(y_{1:M} \mid x_{1:S}) = \prod_{t=1}^{M} p_\theta(y_t \mid x_{1:S}, y_{<t}).$$

The encoder-decoder has the same output factorization but constructs encoder memory

$$H_x = \mathrm{Encoder}_\theta(x_{1:S}),$$

and predicts

$$p_\theta(y_t \mid H_x, y_{<t}).$$

For decoder-only supervised training, a loss mask m_t can restrict gradients to response positions:

$$\mathcal{L} = -\frac{\sum_t m_t \log p_\theta(z_{t+1} \mid z_{\le t})}{\sum_t m_t}.$$

Part 3 - Hints, controls, and evidence

1. Use the same special-token identifiers and document the different input layouts for both models.
2. Match trainable parameter count within a declared tolerance; exact matching is not required but unreported differences are unacceptable.
3. Verify every mask visually on one short padded batch.
4. Train on identical source–target pairs and equal numbers of target-token updates.
5. Report continuation and transformation tasks separately; averaging them can hide architectural specialization.
6. Measure encoder or prompt prefill latency, per-token decode latency, peak memory, accuracy, and failure examples.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def decoder_only_loss(logits, combined_ids, response_mask):
    shifted_logits = logits[:, :-1, :]
    shifted_targets = combined_ids[:, 1:]
    shifted_mask = response_mask[:, 1:]
    token_losses = F.cross_entropy(
        shifted_logits.reshape(-1, shifted_logits.shape[-1]),
        shifted_targets.reshape(-1), reduction="none"
    )
    token_losses = token_losses.view_as(shifted_targets)
    weighted = token_losses * shifted_mask.to(token_losses.dtype)
    return weighted.sum() / shifted_mask.sum().clamp_min(1)

def encoder_decoder_loss(logits, target_ids, padding_id):
    shifted_logits = logits[:, :-1, :]
    shifted_targets = target_ids[:, 1:]
    return F.cross_entropy(
        shifted_logits.reshape(-1, shifted_logits.shape[-1]),
        shifted_targets.reshape(-1), ignore_index=padding_id
    )

def E16(decoder_only_model, encoder_decoder_model,
        decoder_batch, encoder_decoder_batch):
```

```
    decoder_logits = decoder_only_model(decoder_batch["combined_ids"])
    decoder_loss = decoder_only_loss(
        decoder_logits,
        decoder_batch["combined_ids"],
        decoder_batch["response_mask"]
    )
    encoder_decoder_logits = encoder_decoder_model(
        encoder_decoder_batch["source_ids"],
        encoder_decoder_batch["target_ids"][:, :-1]
    )
    sequence_loss = encoder_decoder_loss(
        encoder_decoder_logits,
        encoder_decoder_batch["target_ids"],
        encoder_decoder_batch["padding_id"]
    )
    return decoder_loss, sequence_loss
```

Part 5 - The Philosophy Behind the Exercise

Architecture encodes a theory about relationships. The decoder-only model says that source and response can inhabit one temporal stream. The encoder-decoder model says that understanding the source and producing the target deserve distinct computational roles connected through attention.

Neither theory is complete in isolation. Real success depends on what the task requires, what data teaches, and what hardware can serve. The exercise teaches architectural pluralism: understand the commitments of each design deeply enough to choose, rather than converting historical popularity into a universal law.

18.2.18 Exercise 17

Exercise Title: Explain Why Instruction Tuning Changes Behavior Without Changing Architecture

Part 1 - Why this matters

A pretrained language model learns broad continuation patterns. When given a question, it may continue with another question, imitate a forum thread, produce metadata, or answer directly. Instruction tuning changes the distribution of desired continuations by training on prompt–response examples. The decoder blocks, attention equations, dimensions, and vocabulary head can remain exactly the same while the parameters move toward behavior that treats instructions as requests for useful responses.

This distinction demonstrates why architecture alone cannot explain a chat model. Two checkpoints instantiated from the same class can differ sharply in format following, refusal behavior, verbosity, tool syntax, and response style. Those differences may arise from supervised fine-tuning, preference optimization, safety data, system prompts, or decoding policies. The class definition reveals none of that history.

You will fine-tune a tiny pretrained decoder on a small instruction dataset. Before and after training, evaluate held-out paraphrases, formatting requirements, conflicting instructions, and a control set of ordinary language modeling examples. Use a response-only loss mask so the

objective focuses on the desired answer region. Also run a deliberately incorrect all-token-loss variant to reveal how masking changes the learning signal.

Behavioral improvement must be paired with retention analysis. Fine-tuning can overfit templates, cause catastrophic forgetting, or teach superficial cue matching. Measure base-task loss and instruction-task success throughout training. A checkpoint that follows the training templates perfectly but fails paraphrases has learned a narrow mapping, not general instruction following.

Part 2 - Mathematical form and reasoning

Starting from pretrained parameters θ_0, supervised instruction tuning optimizes

$$\theta_{\text{SFT}} = operatorname * arg\,min_\theta - \sum_{(u,r)\in\mathcal{D}_{\text{SFT}}} \sum_{t\in\text{response}} \log p_\theta(r_t \mid u, r_{<t}).$$

The architecture f_θ is unchanged, but generally $f_{\theta_{\text{SFT}}}(x) \neq f_{\theta_0}(x)$ because optimization changes the parameter values. Define retention as a change in base validation loss ΔL_{base} and instruction gain as $\Delta A_{\text{instruction}}$.

Part 3 - Hints, controls, and evidence

1. Save the pretrained state dictionary and configuration before fine-tuning. Afterward, prove the key names and tensor shapes are unchanged.
2. Construct response masks from token boundaries, not character lengths. Print one full alignment table.
3. Keep a frozen copy of the pretrained checkpoint for paired evaluation.
4. Test unseen instructions that vary wording and output format.
5. Evaluate at several fine-tuning steps to observe instruction gain and base-capability forgetting as trajectories.
6. Compare response-only loss, all-token loss, and one intentionally shifted mask. Treat the shifted mask as a semantic failure even if loss decreases.

Part 4 - Python code skeleton

```
import copy
import torch
import torch.nn.functional as F

def response_only_loss(logits, input_ids, response_mask):
    prediction_logits = logits[:, :-1, :]
    target_ids = input_ids[:, 1:]
    target_mask = response_mask[:, 1:]
    losses = F.cross_entropy(
        prediction_logits.reshape(-1, prediction_logits.shape[-1]),
        target_ids.reshape(-1), reduction="none"
    )
```

```
    losses = losses.view_as(target_ids)
    weighted = losses * target_mask.to(losses.dtype)
    return weighted.sum() / target_mask.sum().clamp_min(1)

def architecture_signature(model):
    signature = {}
    for name, tensor in model.state_dict().items():
        signature[name] = tuple(tensor.shape)
    return signature

def E17(pretrained_model, instruction_batches, optimizer_factory,
        evaluation_function, device):
    frozen_base = copy.deepcopy(pretrained_model).to(device).eval()
    tuned_model = copy.deepcopy(pretrained_model).to(device)
    signature_before = architecture_signature(tuned_model)
    optimizer = optimizer_factory(tuned_model.parameters())
    trajectory = []
    for batch in instruction_batches:
        tuned_model.train()
        optimizer.zero_grad(set_to_none=True)
        logits = tuned_model(batch["input_ids"].to(device))
        loss = response_only_loss(
            logits,
            batch["input_ids"].to(device),
            batch["response_mask"].to(device)
        )
        loss.backward()
        optimizer.step()
        trajectory.append(float(loss.detach().cpu()))
    signature_after = architecture_signature(tuned_model)
    assert signature_before == signature_after
    base_metrics = evaluation_function(frozen_base)
    tuned_metrics = evaluation_function(tuned_model)
    return trajectory, base_metrics, tuned_metrics
```

Part 5 - The Philosophy Behind the Exercise

The same structure can support different habits. Architecture defines the space of possible transformations; training shapes which transformations become likely. This is why behavior cannot be read directly from a network diagram. History is embedded in parameter values.

Instruction tuning also shows that helpfulness is not a natural consequence of language modeling alone. It is a learned relationship between requests and responses, created from human examples and preferences. That relationship can be improved, biased, forgotten, or manipulated. Understanding its origin makes the interface less magical and its governance more accountable.

18.2.19 Exercise 18

Exercise Title: Write a Conceptual Summary Explaining LLM as System Not Just Model

Part 1 - Why this matters

This exercise consolidates the chapter into an engineering system map. A model checkpoint is only one stateful component in a chain that begins with authorized data and ends with monitored behavior. Tokenizers, data pipelines, distributed training, post-training, evaluation suites, inference engines, caches, retrievers, safety policies, and user interfaces all introduce assumptions and failure modes. A summary that discusses only attention blocks misses the object that users actually encounter.

Construct the summary as a causal trace. Begin with source documents and their provenance. Follow normalization, deduplication, tokenization, sequence packing, next-token labels, optimization, checkpoint selection, instruction tuning, preference or safety stages, model packaging, and inference. Then include prompt construction, retrieval, decoding, tool execution, response validation, monitoring, feedback, and rollback. For each boundary, state what enters, what leaves, and what can silently go wrong.

Ownership matters. Data engineers, model researchers, infrastructure teams, safety specialists, product engineers, domain reviewers, and operators may control different parts of the system. A failure can cross these boundaries: a tokenizer mismatch may look like model corruption; stale retrieval may look like hallucination; a serving cache leak may look like privacy failure in the model. The report should assign observable checks and response responsibility.

Your system description must include lifecycle time. Data and model versions change, knowledge becomes stale, dependencies receive updates, evaluation sets leak, user distributions shift, and attackers adapt. "The model passed" is not a permanent system property. Reliability requires continuous evidence and a recoverable version history.

Part 2 - Mathematical form and reasoning

Represent the deployed response as

$$r = \mathcal{P}_\omega \left(\mathcal{D}_\gamma \left(f_\theta \left(\mathcal{C}_\rho(T_\phi(u), R_\eta(u)) \right) \right) \right),$$

where T tokenizes, R retrieves evidence, $\mathcal{C}$ builds context, f_θ produces logits, $\mathcal{D}$ decodes, and $\mathcal{P}$ applies post-processing or policy. System risk can be decomposed conceptually as

$$P(F) = 1 - \prod_{i=1}^{K} \left(1 - P(F_i \mid F^c_{<i}) \right),$$

while remembering that component failures are often dependent. The formula is a prompt to model interfaces, not a claim that real failures are independent.

Part 3 - Hints, controls, and evidence

1. Draw a system boundary diagram with training and inference paths shown separately.
2. For every component record version, owner, input contract, output contract, invariant, metric, and recovery action.
3. Trace one correct request and at least five injected failures end to end.
4. Include privacy, licensing, security, latency, cost, and factual quality; do not reduce the system to model accuracy.
5. Identify which tests run before release, continuously online, and after an incident.

6. Finish with a concise claim: which properties belong to the model, which belong to the surrounding system, and which emerge only from their interaction.

Part 4 - Python code skeleton

```
class LLMSystemAudit:
    def __init__(self):
        self.components = []
        self.events = []

    def register_component(self, name, version, owner,
                           input_contract, output_contract,
                           invariant, recovery_action):
        component = {}
        component["name"] = name
        component["version"] = version
        component["owner"] = owner
        component["input_contract"] = input_contract
        component["output_contract"] = output_contract
        component["invariant"] = invariant
        component["recovery_action"] = recovery_action
        self.components.append(component)

    def record_event(self, request_id, component_name,
                     status, evidence, latency_seconds):
        event = {}
        event["request_id"] = request_id
        event["component"] = component_name
        event["status"] = status
        event["evidence"] = evidence
        event["latency_seconds"] = latency_seconds
        self.events.append(event)

    def failures_for_request(self, request_id):
        failures = []
        for event in self.events:
            if event["request_id"] == request_id:
                if event["status"] != "ok":
                    failures.append(event)
        return failures

def E18():
    audit = LLMSystemAudit()
    audit.register_component(...)
    audit.record_event(...)
    return audit
```

Part 5 - The Philosophy Behind the Exercise

A system is more than the sum of its components because interfaces create new properties. A model cannot leak another user's cache if no shared serving cache exists; a retriever cannot make the model cite a source if its metadata is discarded; a safe checkpoint cannot prevent an external tool from executing an unauthorized action. Interaction creates both capability and risk.

This systems view changes responsibility. It prevents teams from blaming the model for every pipeline failure or treating the model as innocent of every behavior mediated through its outputs. Responsibility follows design choices, evidence, authority, and consequences across the whole chain.

18.2.20 Exercise 19

Exercise Title: Produce a One-Page Mental Model of How LLMs Actually Work

Part 1 - Why this matters

After studying many mechanisms, a researcher needs a compact mental model that remains accurate under pressure. The purpose of one page is not to remove depth. It is to preserve the causal backbone so that new details have a correct place to attach. A good mental model should help diagnose an unexpected output, design an experiment, explain the system to another engineer, and identify when a claim exceeds available evidence.

Your page must follow one request from text to response. Text is tokenized into identifiers. Embeddings place tokens and positions into vectors. Repeated decoder blocks use causal attention and feed-forward transformations to update the residual stream. The language head converts the final state at each position into vocabulary logits. A decoding rule selects a token, appends it, and repeats. Learned weights came from next-token optimization over data and may have been altered by instruction and preference tuning.

The same page must include what this mechanism does not guarantee. Logits are not facts, probability is not correctness, context capacity is not faithful context use, parameter count is not reasoning proof, and fluent explanation is not a transparent record of internal causality. Retrieval, tools, policies, and verification can extend the system, but each adds an interface that must be tested.

Although the final artifact is one page, it must be earned through compression. Every arrow should be supported by an exercise from this chapter. Every word such as "learns," "remembers," "reasons," or "knows" must be either defined operationally or replaced with a more precise statement. Simplicity is the last stage of understanding, not the first.

Part 2 - Mathematical form and reasoning

The minimum mathematical spine is

$$x_{1:T} = T_\phi(s), \qquad Z = f_\theta(x_{1:T}), \qquad p = \mathrm{softmax}(Z_{:,T,:}), \qquad x_{T+1} \sim \pi_\gamma(p),$$

with training objective

$$\min_\theta - \sum_t \log p_\theta(x_{t+1} \mid x_{\le t}).$$

The page should label shapes (B, T), (B, T, d), and (B, T, V) and show the causal information boundary. Add a separate systems ring for data, post-training, retrieval, tools, evaluation, serving, monitoring, and rollback.

Part 3 - Hints, controls, and evidence

1. Use exactly three layers of explanation: mechanism, learned behavior, and deployed system. Do not mix them in one unlabeled paragraph.
2. Include one forward arrow for computation and one feedback arrow for training or operational monitoring.
3. Attach at least six invariants: identifier range, context limit, causal mask, logit shape, probability normalization, and evidence boundary.
4. Add a "can" column and a "does not guarantee" column.
5. Test the page by using it to diagnose a tokenizer mismatch, a future-mask leak, a hallucinated citation, and an out-of-memory failure.
6. Ask another reader to explain the model using only the page. Record which arrows or terms remain ambiguous and revise them.

Part 4 - Python code skeleton

```
def inspect_one_generation_step(system, text):
    trace = {}
    token_ids = system.tokenizer.encode(text)
    trace["text"] = text
    trace["token_ids"] = token_ids
    trace["token_count"] = len(token_ids)

    input_ids = system.to_tensor(token_ids)
    trace["input_shape"] = tuple(input_ids.shape)
    logits = system.model(input_ids)
    trace["logit_shape"] = tuple(logits.shape)

    last_logits = logits[:, -1, :]
    probabilities = system.softmax(last_logits)
    trace["probability_sum"] = float(probabilities.sum().item())
    next_id = system.decode_policy(probabilities)
    trace["next_id"] = int(next_id.item())
    trace["next_token"] = system.tokenizer.decode([trace["next_id"]])

    assert trace["token_count"] <= system.maximum_context
    assert trace["logit_shape"][-1] == system.vocabulary_size
    assert abs(trace["probability_sum"] - 1.0) < 1e-5
    return trace

def E19(system, diagnostic_prompts):
    traces = []
    for prompt in diagnostic_prompts:
        traces.append(inspect_one_generation_step(system, prompt))
    return traces
```

Part 5 - The Philosophy Behind the Exercise

A mental model is a tool for deciding what to notice. If it contains only the neural network, data failures become invisible. If it contains only system components, the probability mechanism becomes mystical. If it uses human mental language without operational definitions, impressions replace causes. The best mental model is compact enough to remember and precise enough to resist these distortions.

The chapter ends where serious LLM engineering begins: not with the claim that the model is merely autocomplete, and not with the claim that scale created an oracle. It is a learned probabilistic machine embedded in a human-designed system. Its capabilities are real, its limitations are consequential, and both must be studied through mechanisms, controlled experiments, and evidence.

18.3 LLM01

Chapter Title: Tokenization and Context Mechanics
Specialization: Language Representation and Context Engineering

18.3.1 Exercise 00

Exercise Title: Implement BPE Tokenization Conceptually and Explain Why Tokens Are Not Words

Part 1 - Why this matters

An LLM never receives words directly. It receives integer identifiers produced by a tokenizer whose vocabulary and segmentation rules were learned or designed before model training. The sentence visible to a user and the token sequence visible to the model are therefore different objects. Spaces, punctuation, capitalization, digits, Unicode normalization, and repeated byte patterns can all change the sequence length and boundaries.

Byte Pair Encoding begins with small symbols and repeatedly merges the most frequent adjacent pair. Frequent fragments become single vocabulary entries, while rare words remain compositions of smaller pieces. This produces an open vocabulary without allocating a separate identifier to every possible word. A token may be a complete word, part of a word, punctuation joined to whitespace, a byte fragment, or another learned unit. Calling every token a word hides the actual computational sequence.

You will implement the training phase and encoding phase of a small BPE system without using a tokenizer library. The implementation must preserve word-end or boundary information so that merging does not accidentally erase how text can be reconstructed. You will print pair frequencies at every iteration, choose a deterministic tie-breaking rule, store merges in rank order, and then replay those rules when encoding unseen text.

The experiment must also expose failure cases. A tokenizer trained on one domain may fragment another domain heavily. Different Unicode representations of visually similar text may produce different tokens. A merge that is legal inside one pre-tokenized word should not silently cross a boundary unless the tokenizer design explicitly permits it. These are not cosmetic details: they change context usage, embedding lookup, training frequency, and inference cost.

Part 2 - Mathematical form and reasoning

Let the current symbol sequence for corpus item n be $s^{(n)} = (s_1^{(n)}, \ldots, s_{L_n}^{(n)})$. The frequency of adjacent pair (a, b) is

$$f(a,b) = \sum_{n} \sum_{i=1}^{L_n - 1} \mathbb{I}\left[s_i^{(n)} = a \wedge s_{i+1}^{(n)} = b\right].$$

At merge step k, select

$$(a_k, b_k) = \arg\max_{(a,b)} f(a,b)$$

and replace every non-overlapping occurrence with the new symbol $a_k \| b_k$. Encoding applies the learned merges in rank order, not by recalculating corpus frequencies for each new sentence.

Part 3 - Hints, controls, and evidence

1. Normalize text using one declared Unicode policy and retain the original text for round-trip comparison.
2. Begin from visible characters for transparency, while explaining why production byte-level BPE starts from bytes.
3. Count adjacent pairs using explicit loops and show how overlapping pairs are handled.
4. Break equal-frequency ties lexicographically so repeated runs learn the same vocabulary.
5. Encode one training word, one unseen morphological variant, one number, and one non-English example.
6. Assert that decoding the encoded symbols reconstructs the normalized input exactly. Report token count and fragmentation ratio for every case.

Part 4 - Python code skeleton

```
from collections import defaultdict
import unicodedata

def initial_symbols(text):
    normalized = unicodedata.normalize("NFC", text)
    words = normalized.split(" ")
    sequences = []
    for word in words:
        symbols = []
        for character in word:
            symbols.append(character)
        symbols.append("</w>")
        sequences.append(symbols)
    return sequences

def count_pairs(sequences):
    frequencies = defaultdict(int)
    for symbols in sequences:
        for index in range(len(symbols) - 1):
```

```
            pair = (symbols[index], symbols[index + 1])
            frequencies[pair] = frequencies[pair] + 1
    return frequencies

def merge_pair(symbols, selected_pair):
    merged = []
    index = 0
    while index < len(symbols):
        can_merge = index + 1 < len(symbols)
        if can_merge:
            current_pair = (symbols[index], symbols[index + 1])
            if current_pair == selected_pair:
                merged.append(symbols[index] + symbols[index + 1])
                index = index + 2
                continue
        merged.append(symbols[index])
        index = index + 1
    return merged

def pair_sort_key(item):
    pair = item[0]
    frequency = item[1]
    return (-frequency, pair)

def train_bpe(texts, number_of_merges):
    sequences = []
    for text in texts:
        sequences.extend(initial_symbols(text))
    learned_merges = []
    for _ in range(number_of_merges):
        frequencies = count_pairs(sequences)
        if len(frequencies) == 0:
            break
        ordered = sorted(frequencies.items(), key=pair_sort_key)
        selected_pair = ordered[0][0]
        learned_merges.append(selected_pair)
        updated = []
        for symbols in sequences:
            updated.append(merge_pair(symbols, selected_pair))
        sequences = updated
    return learned_merges, sequences

def E00():
    corpus = ["low lower lowest", "newer wider"]
    merges, final_sequences = train_bpe(corpus, 12)
    return merges, final_sequences
```

Part 5 - The Philosophy Behind the Exercise

Tokenization shows that representation is never neutral. Before the neural network learns anything, a prior system has decided what counts as a reusable unit. Frequent patterns receive compact symbols; rare patterns consume more positions. The model's apparent language is built on this hidden segmentation.

The deeper lesson is that meaning does not live inside token boundaries. Human concepts can span many tokens, and one token can participate in many meanings. BPE is a compression-informed interface between text and computation, not a theory of words or understanding. Knowing this prevents us from mistaking the model's storage units for the structure of language itself.

18.3.2 Exercise 01

Exercise Title: Compare Character Word and Subword Tokenization and Measure Vocabulary Explosion

Part 1 - Why this matters

Character, word, and subword tokenization allocate complexity differently. Character tokenization uses a small vocabulary but produces long sequences. Word tokenization produces short sequences for familiar text but requires an enormous vocabulary and an unknown-token policy. Subword tokenization occupies the middle: a moderate learned vocabulary represents common strings compactly while decomposing rare strings into reusable pieces.

Vocabulary size affects more than a lookup table. Token embeddings and the language-model output projection commonly contain terms proportional to Vd_{model}. A large word vocabulary increases parameters and output logit compute. A small character vocabulary reduces those costs but increases sequence length, causing more Transformer activations and potentially much more attention work. Subword design is therefore a trade between vocabulary-axis and sequence-axis cost.

You will build all three tokenizers over identical train and test corpora. The test corpus must include unseen words, spelling variants, identifiers, numbers, punctuation, and at least two languages. Measure vocabulary size, average tokens per sentence, characters per token, unknown rate, round-trip fidelity, and estimated attention-score elements. A tokenizer cannot be judged solely on the training corpus it memorized.

Word splitting also requires a definition. Splitting only on spaces mishandles punctuation and languages without spaces; lowercasing destroys case information; Unicode normalization can join or separate code points. State each rule rather than calling the output "words" as if the boundary were universally obvious.

Part 2 - Mathematical form and reasoning

For tokenizer τ and documents d_i, define vocabulary size $V_\tau = |\mathcal{V}_\tau|$ and average sequence length

$$\bar{T}_\tau = \frac{1}{N}\sum_{i=1}^{N} |\tau(d_i)|.$$

Embedding parameters scale as $V_\tau d$, while dense attention pair count for batch B and heads H scales as

$$P_\tau = BH\bar{T}_\tau^2.$$

Unknown rate for a word vocabulary is

$$r_{\text{UNK}} = \frac{N_{\text{unknown tokens}}}{N_{\text{test tokens}}}.$$

Report distributions as well as means because rare long examples can dominate memory failures.

Part 3 - Hints, controls, and evidence

1. Learn all vocabularies using training text only; test text must not alter the vocabulary.
2. Reserve special symbols explicitly and include them in parameter counts.
3. Measure percentiles of tokenized length, not only the average.
4. Report unknown-token rate and round-trip reconstruction separately. An unknown placeholder can preserve length while destroying content.
5. Estimate embedding bytes for float32, float16, and tied output weights.
6. Create a decision table showing which tokenizer wins on vocabulary, sequence length, openness, and multilingual robustness.

Part 4 - Python code skeleton

```
import torch

def character_tokens(text):
    tokens = []
    for character in text:
        tokens.append(character)
    return tokens

def whitespace_word_tokens(text):
    pieces = text.split()
    tokens = []
    for piece in pieces:
        tokens.append(piece)
    return tokens

def build_vocabulary(tokenized_documents, special_tokens):
    token_to_id = {}
    for token in special_tokens:
        token_to_id[token] = len(token_to_id)
    for document_tokens in tokenized_documents:
        for token in document_tokens:
            if token not in token_to_id:
                token_to_id[token] = len(token_to_id)
    return token_to_id

def tokenizer_metrics(tokenized_documents, vocabulary_size,
```

```
                    d_model, batch_size, heads):
    lengths = []
    for tokens in tokenized_documents:
        lengths.append(len(tokens))
    length_tensor = torch.tensor(lengths, dtype=torch.float64)
    metrics = {}
    metrics["vocabulary_size"] = vocabulary_size
    metrics["mean_length"] = float(length_tensor.mean())
    metrics["maximum_length"] = int(length_tensor.max())
    metrics["embedding_parameters"] = vocabulary_size * d_model
    mean_length = metrics["mean_length"]
    metrics["mean_attention_pairs"] = (
        batch_size * heads * mean_length * mean_length
    )
    return metrics

def E01(training_documents, test_documents, subword_encode):
    character_train = []
    word_train = []
    for document in training_documents:
        character_train.append(character_tokens(document))
        word_train.append(whitespace_word_tokens(document))
    character_vocabulary = build_vocabulary(character_train, ["<pad>"])
    word_vocabulary = build_vocabulary(word_train, ["<pad>", "<unk>"])
    subword_test = []
    for document in test_documents:
        subword_test.append(subword_encode(document))
    return character_vocabulary, word_vocabulary, subword_test
```

Part 5 - The Philosophy Behind the Exercise

Every tokenizer chooses where to pay for linguistic diversity. Word models pay in vocabulary, character models pay in time, and subword models negotiate between them. There is no representation without cost; there are only different places where cost and information loss appear.

Vocabulary explosion is also a reminder that language is productive. Humans continually invent names, compounds, numbers, and spellings. A closed list of words cannot anticipate them all. Subwords succeed not because they discover the final atoms of meaning, but because composition is a practical answer to an open world.

18.3.3 Exercise 02

Exercise Title: Explain Context Window and Why LLMs Forget Beyond Max Length

Part 1 - Why this matters

The context window is the maximum token sequence that a model invocation can represent under its configured positional mechanism and serving limits. It is not the model's learned knowledge, a permanent conversation memory, or a promise that every token inside the window

will influence the answer equally. Context is temporary request state; parameters are persistent learned state.

When a conversation exceeds the allowed length, some policy must change the input. A system may reject the request, remove earliest tokens, summarize prior turns, retrieve selected memories, or use a model designed for a longer window. If the relevant evidence is removed, the model cannot attend to it in that invocation. Saying the model "forgot" is convenient, but the immediate cause may be that the serving pipeline never supplied the evidence.

Even evidence inside the formal limit can be used poorly. Long-context quality depends on positional training, attention patterns, placement, distractors, and the model's ability to retrieve and combine distant information. Formal context length, accepted input length, and effective usable context must be measured as different properties.

You will implement a context manager that records every truncation decision. Then build a key-retrieval task where a unique fact appears at the beginning, middle, or end of a long sequence. Compare hard left truncation, right truncation, head–tail preservation, and evidence-aware selection. The experiment should make information loss visible before asking the model to answer.

Part 2 - Mathematical form and reasoning

Let a tokenized request have length T and maximum context C. A left truncation operator is

$$\tau_{\text{left}}(x_{1:T}) = \begin{cases} x_{1:T}, & T \le C, \\ x_{T-C+1:T}, & T > C. \end{cases}$$

If required evidence occupies index set R, evidence retention is

$$\rho = \frac{|R \cap I_{\text{kept}}|}{|R|}.$$

Model accuracy should be reported conditioned on $\rho = 1$ and $\rho < 1$. Otherwise pipeline loss and model failure are mixed together.

Part 3 - Hints, controls, and evidence

1. Count tokens after applying the exact production tokenizer, not by words or characters.
2. Preserve mandatory system and safety instructions before allocating the remaining context budget.
3. Return kept and removed token ranges from every truncation function.
4. Place the required fact at several normalized positions and add irrelevant distractors.
5. Distinguish failure caused by removed evidence from failure with retained evidence.
6. Test lengths below, exactly equal to, and one token above the maximum.

Part 4 - Python code skeleton

```
def left_truncate(token_ids, maximum_context):
    original_length = len(token_ids)
    if original_length <= maximum_context:
        return token_ids[:], [], (0, original_length)
    first_kept = original_length - maximum_context
    kept = token_ids[first_kept:]
    removed = token_ids[:first_kept]
    return kept, removed, (first_kept, original_length)

def head_tail_truncate(token_ids, maximum_context, head_tokens):
    if len(token_ids) <= maximum_context:
        return token_ids[:]
    if head_tokens > maximum_context:
        raise ValueError("head allocation exceeds context")
    tail_tokens = maximum_context - head_tokens
    kept = []
    kept.extend(token_ids[:head_tokens])
    if tail_tokens > 0:
        kept.extend(token_ids[-tail_tokens:])
    return kept

def evidence_retention(evidence_indices, kept_start, kept_end):
    retained = 0
    for index in evidence_indices:
        if kept_start <= index and index < kept_end:
            retained = retained + 1
    return retained / max(1, len(evidence_indices))

def E02(token_ids, evidence_indices, context_sizes):
    records = []
    for context_size in context_sizes:
        kept, removed, interval = left_truncate(
            token_ids, context_size
        )
        record = {}
        record["context"] = context_size
        record["kept_length"] = len(kept)
        record["removed_length"] = len(removed)
        record["retention"] = evidence_retention(
            evidence_indices, interval[0], interval[1]
        )
        records.append(record)
    return records
```

Part 5 - The Philosophy Behind the Exercise

Context is often described as memory, but it is closer to a temporary field of attention prepared by the surrounding system. What is absent cannot be used; what is present is not guaranteed to be understood. This distinction separates storage, access, and effective reasoning.

Forgetting is therefore not always a defect inside the model. It can be a policy decision made by a tokenizer, prompt builder, cache, or truncation rule. The ethical and engineering lesson is to make those invisible editorial decisions auditable, because they decide which parts of a conversation continue to exist for the machine.

18.3.4 Exercise 03

Exercise Title: Build Dataset That Produces Sliding Context Windows Correctly

Part 1 - Why this matters

Long token streams must be converted into finite training examples. A sliding window dataset selects a context of length C and a target shifted by one token. The operation appears simple, yet errors in the final valid index, stride, document boundaries, padding, or target shift can make examples incomplete or teach relationships across unrelated documents.

Stride controls overlap. A stride of one produces many highly redundant windows; a stride equal to context length produces non-overlapping blocks; an intermediate stride trades sample count against coverage and repeated compute. Overlap can expose each transition under multiple prefixes, but it also changes the effective weighting of central tokens relative to boundary tokens.

Document boundaries require an explicit policy. Concatenating documents with no separator teaches the model that the end of one document is followed directly by the beginning of another. Inserting an end-of-document token makes the transition visible. Preventing windows from crossing documents avoids synthetic cross-document contexts but can waste short tails. The correct choice depends on the training design and must be recorded.

You will implement both a single-stream dataset and a document-aware dataset. For a tiny numbered sequence, enumerate every expected input and target by hand. Then test different strides, drop-last behavior, and sequences shorter than the requested window. Dataset correctness must be proven before DataLoader performance is measured.

Part 2 - Mathematical form and reasoning

For token sequence length N, context length C, and stride S, a full input–target example requires $C + 1$ tokens. Valid start positions satisfy

$$i_k = kS, \qquad i_k + C < N.$$

The number of full windows is

$$K = \begin{cases} 0, & N < C + 1, \\ \left\lfloor \dfrac{N - C - 1}{S} \right\rfloor + 1, & N \geq C + 1. \end{cases}$$

Each example is $X^{(k)} = x_{i_k:i_k+C}$ and $Y^{(k)} = x_{i_k+1:i_k+C+1}$. Verify the first and final examples against this formula.

Part 3 - Hints, controls, and evidence

1. Use the sequence $[0, 1, \ldots, 9]$ with several context and stride values and write the expected windows before coding.

2. Reject non-positive context and stride values in the constructor.

3. Return cloned or freshly indexed tensors so accidental mutation of one sample does not corrupt the dataset.

4. Test length $C-1$, C, $C+1$, and $C+2$ explicitly.

5. For multiple documents, verify no input crosses a boundary unless an end-of-document token is intentionally inserted.

6. Count how many times each source transition appears under overlapping windows and discuss weighting bias.

Part 4 - Python code skeleton

```
import torch
from torch.utils.data import Dataset

class SlidingContextDataset(Dataset):
    def __init__(self, token_ids, context_length, stride):
        if token_ids.ndim != 1:
            raise ValueError("token_ids must be one-dimensional")
        if context_length <= 0 or stride <= 0:
            raise ValueError("context and stride must be positive")
        self.token_ids = token_ids
        self.context_length = context_length
        self.stride = stride
        available = token_ids.numel() - context_length - 1
        if available < 0:
            self.number_of_windows = 0
        else:
            self.number_of_windows = available // stride + 1

    def __len__(self):
        return self.number_of_windows

    def __getitem__(self, index):
        if index < 0 or index >= self.number_of_windows:
            raise IndexError("window index is out of range")
        start = index * self.stride
        input_end = start + self.context_length
        target_end = input_end + 1
        input_ids = self.token_ids[start:input_end].clone()
        target_ids = self.token_ids[start + 1:target_end].clone()
        return input_ids, target_ids

def E03():
    token_ids = torch.arange(10, dtype=torch.long)
    dataset = SlidingContextDataset(token_ids, context_length=4,
                                    stride=2)
    observed = []
```

```
    for index in range(len(dataset)):
        input_ids, target_ids = dataset[index]
        observed.append((input_ids, target_ids))
        assert torch.equal(input_ids[1:], target_ids[:-1])
    assert len(dataset) == 3
    return observed
```

Part 5 - The Philosophy Behind the Exercise

A dataset is not merely stored information; it is a procedure that decides what the model repeatedly experiences. Sliding windows create a local view of a long world. Stride, boundaries, and repetition determine which transitions become common and which context relationships are never shown.

This is why data loaders deserve the same scrutiny as model layers. A perfectly implemented network cannot recover an example that was misaligned before it arrived. Learning begins with the geometry of exposure, and the dataset is where that geometry is constructed.

18.3.5 Exercise 04

Exercise Title: Debug Off-By-One Errors in Next-Token Label Shifting

Part 1 - Why this matters

Next-token training depends on a one-position semantic relationship: logits at position t predict the identifier at $t+1$. An off-by-one bug can preserve tensor ranks, produce finite loss, and even decrease during training while teaching the model to copy the current token or predict two steps ahead. Shape validation alone cannot detect this class of error.

The safest approach is to inspect one readable sequence before batching. If the tokens are `<bos> the model learns <eos>`, the row generated from `<bos>` must be compared with `the`; the row generated from `the` must be compared with `model`. Special tokens and padding must participate according to an explicit policy. Whether the final content token predicts `<eos>` changes stopping behavior later.

You will implement a correct shifter and three faulty variants: no shift, two-step shift, and a shift that moves logits and labels in the same direction but loses the intended pairing. Train a tiny model on a deterministic repeating sequence and compare learned transition tables. The correct model should assign high probability to the known successor; the copy-label model should learn an identity mapping.

The exercise also covers padding masks. The ignore mask must be shifted with the target tokens. If it remains aligned to the input, a real target can be dropped or a padding target can contribute loss. Print the flattened valid pairs used by cross entropy so reduction never hides the error.

Part 2 - Mathematical form and reasoning

For logits $Z \in \mathbb{R}^{B\times T\times V}$ and identifiers $X \in \mathbb{N}^{B\times T}$, correct alignment is

$$Z' = Z_{:,:T-1,:}, \qquad Y = X_{:,1:T}.$$

The loss is

$$\mathcal{L} = -\frac{1}{N_{\text{valid}}} \sum_{b,t} m_{b,t+1} \log \operatorname{softmax}(Z_{b,t,:})_{X_{b,t+1}}.$$

For a two-step target, $Y = X_{:,2:T}$ requires $Z = Z_{:,:T-2,:}$; mixing these lengths should fail an assertion instead of being silently reshaped.

Part 3 - Hints, controls, and evidence

1. Decode every input–target pair for one batch and inspect it manually.
2. Assert Z'.shape[: 2] = Y.shape before flattening.
3. Use `reshape`, not an arbitrary slice chosen only to make element counts match.
4. Track the number of valid targets and verify it equals the expected non-padding transitions.
5. Overfit a tiny sequence until the correct transition probability is near one; then inspect what each faulty variant learned.
6. Add a regression test that intentionally swaps in each faulty shifter and must fail semantic assertions.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def correct_shift(logits, input_ids, padding_id):
    prediction_logits = logits[:, :-1, :]
    target_ids = input_ids[:, 1:]
    valid_mask = target_ids != padding_id
    assert prediction_logits.shape[:2] == target_ids.shape
    return prediction_logits, target_ids, valid_mask

def no_shift_bug(logits, input_ids, padding_id):
    prediction_logits = logits
    target_ids = input_ids
    valid_mask = target_ids != padding_id
    return prediction_logits, target_ids, valid_mask

def masked_next_token_loss(logits, input_ids, padding_id):
    prediction_logits, target_ids, valid_mask = correct_shift(
        logits, input_ids, padding_id
    )
    token_losses = F.cross_entropy(
        prediction_logits.reshape(-1, prediction_logits.shape[-1]),
        target_ids.reshape(-1), reduction="none"
    )
    token_losses = token_losses.view_as(target_ids)
    selected = token_losses.masked_select(valid_mask)
    if selected.numel() == 0:
```

```
        raise ValueError("batch contains no valid next-token targets")
    return selected.mean()

def E04(model, input_ids, padding_id):
    logits = model(input_ids)
    loss = masked_next_token_loss(logits, input_ids, padding_id)
    assert torch.isfinite(loss)
    return loss
```

Part 5 - The Philosophy Behind the Exercise

An off-by-one error is a lesson in the difference between numerical validity and semantic truth. Computers can optimize exactly the wrong question with complete discipline. A decreasing number does not certify that the number represents the objective we intended.

Alignment is therefore a form of meaning. The model learns the relationship encoded by index positions, not the relationship described in comments. Deep learning engineering begins when we treat those indices as scientific claims that require direct evidence.

18.3.6 Exercise 05

Exercise Title: Implement Attention Mask for Variable-Length Contexts

Part 1 - Why this matters

A batch often contains sequences of different lengths. Padding creates a common rectangular tensor, but padding positions are not evidence and must not influence attention or loss. In a causal decoder, two restrictions must be combined: a query cannot see future keys, and no query should use a padding key. These masks represent different semantics and usually have different source shapes.

The key-padding mask prevents real queries from attending to padded keys. Padded query rows also require a policy. They may produce arbitrary finite outputs if their loss is ignored, or their outputs may be explicitly zeroed. Confusing key masking with query masking can create rows in which every score is negative infinity, causing softmax to return NaN. Your implementation must prevent all-masked rows or define a safe output for them.

Mask convention is a common source of bugs. Some APIs use `True` to mean allowed, others use `True` to mean blocked, and additive masks use zero and negative infinity. Broadcasting from (B, T) to $(B, 1, 1, T)$ is correct for key visibility, while causal visibility commonly starts as $(1, 1, T, T)$. Print the combined shape and an example matrix before trusting it.

You will construct a batch with lengths $[5, 3, 1]$, compare masked batched output with separate unpadded computations, and verify equality at real positions. This equivalence test is stronger than merely checking that padding attention weights look small.

Part 2 - Mathematical form and reasoning

Let $c_{i,j} = 1$ when $j \le i$ and let $k_{b,j} = 1$ for a real key. Combined visibility is

$$v_{b,i,j} = c_{i,j} \wedge k_{b,j}.$$

The masked score is

$$\widetilde{S}_{b,h,i,j} = \begin{cases} S_{b,h,i,j}, & v_{b,i,j} = 1, \\ -\infty, & v_{b,i,j} = 0. \end{cases}$$

For every real query i, attention weights must sum to one over permitted keys and equal zero elsewhere.

Part 3 - Hints, controls, and evidence

1. Build the length-derived key mask explicitly using position comparison.
2. Keep a boolean allowed mask until the final `masked_fill`; avoid adding integer zeros and ones to logits.
3. Assert that every real query has at least one allowed key, normally itself.
4. Compare each real prefix with a separately evaluated unpadded sequence.
5. Verify padding token embeddings can change without affecting earlier real outputs.
6. Test left padding and right padding separately; positional indices and causal alignment differ.

Part 4 - Python code skeleton

```
import torch

def lengths_to_key_mask(lengths, maximum_length):
    positions = torch.arange(maximum_length, device=lengths.device)
    key_mask = positions.unsqueeze(0) < lengths.unsqueeze(1)
    return key_mask

def combined_allowed_mask(lengths, maximum_length):
    key_mask = lengths_to_key_mask(lengths, maximum_length)
    causal = torch.ones(maximum_length, maximum_length,
                        device=lengths.device, dtype=torch.bool)
    causal = torch.tril(causal)
    causal = causal.view(1, 1, maximum_length, maximum_length)
    key_mask = key_mask.view(lengths.shape[0], 1, 1, maximum_length)
    return causal & key_mask

def masked_attention(q, k, v, lengths):
    sequence_length = q.shape[-2]
    scores = q @ k.transpose(-2, -1)
    scores = scores / (q.shape[-1] ** 0.5)
    allowed = combined_allowed_mask(lengths, sequence_length)
    scores = scores.masked_fill(~allowed, float("-inf"))
    weights = torch.softmax(scores, dim=-1)
    output = weights @ v
    return output, weights, allowed

def E05(device):
    lengths = torch.tensor([5, 3, 1], device=device)
```

```
    q = torch.randn(3, 2, 5, 8, device=device)
    k = torch.randn(3, 2, 5, 8, device=device)
    v = torch.randn(3, 2, 5, 8, device=device)
    output, weights, allowed = masked_attention(q, k, v, lengths)
    forbidden_weights = weights.masked_select(~allowed)
    assert torch.isfinite(output).all()
    assert torch.all(forbidden_weights == 0)
    return output, weights
```

Part 5 - The Philosophy Behind the Exercise

A mask is a rule about relevance and permission. Padding says that some symbols exist only to satisfy storage geometry; causality says that some real symbols exist but are not yet available. Both appear as blocked matrix entries, yet they express different truths.

Reliable modeling depends on preserving such distinctions. When semantics are compressed into one tensor, the code must carry the explanation that the tensor alone cannot. Otherwise identical zeros can hide entirely different reasons for exclusion.

18.3.7 Exercise 06

Exercise Title: Demonstrate Context Truncation Failure on Long Documents

Part 1 - Why this matters

Long-document systems often fail before the model begins computation. A prompt builder may retain the latest conversation turns and remove an early definition, contract clause, measurement, or safety constraint. The model then answers from the surviving context, and the output can look like a reasoning failure even though the necessary evidence was never provided.

You will create long synthetic documents containing a unique key–value fact at controlled positions. The final question asks for that value. Tokenize the document, apply several truncation policies, and record whether the evidence span survives. Then query the model and separate pipeline failures from model retrieval failures. The distinction must appear in every result row.

Truncation can also damage structure without removing the central fact. Cutting inside a table row, code block, quotation, or multi-token entity can make the remaining text ambiguous. A token-safe cut is not automatically a semantically safe cut. Chunking should prefer document boundaries while still respecting the hard token budget.

The exercise should compare naive left truncation, head–tail preservation, query-aware chunk selection, and summarization. Query-aware selection can retain the relevant evidence but introduces a retriever whose errors must be measured. Summarization compresses more context but may omit details or alter negation. Every policy changes which failure is possible.

Part 2 - Mathematical form and reasoning

For evidence span $e = [a, b)$ and kept token indices K, define full-span retention

$$r_e = \mathbb{I}\left[\{a, a+1, \ldots, b-1\} \subseteq K\right].$$

Let A be answer correctness. Report

$$P(A = 1 \mid r_e = 1) \quad \text{and} \quad P(A = 1 \mid r_e = 0)$$

separately. A low first value indicates model or prompt-use failure; a low second value is normally expected and should not be labeled failure to read available evidence.

Part 3 - Hints, controls, and evidence

1. Generate documents from labeled sections so the exact token span of the fact is known after tokenization.
2. Test facts at 5, 25, 50, 75, and 95 percent of document length.
3. Add a distractor with a similar key but a different value.
4. Record original tokens, kept tokens, removed span, evidence retention, prompt sent, and answer.
5. Test a cut through the middle of the evidence span, not only total removal.
6. Require the model to return the evidence quotation and token range with the answer.

Part 4 - Python code skeleton

```
def span_fully_retained(span_start, span_end, kept_indices):
    kept_set = set(kept_indices)
    for index in range(span_start, span_end):
        if index not in kept_set:
            return False
    return True

def apply_left_truncation(token_ids, maximum_tokens):
    if len(token_ids) <= maximum_tokens:
        indices = list(range(len(token_ids)))
        return token_ids[:], indices
    first = len(token_ids) - maximum_tokens
    indices = list(range(first, len(token_ids)))
    return token_ids[first:], indices

def evaluate_policy(generate, cases, truncation_policy, maximum_tokens):
    records = []
    for case in cases:
        kept_tokens, kept_indices = truncation_policy(
            case["token_ids"], maximum_tokens
        )
        retained = span_fully_retained(
            case["span_start"], case["span_end"], kept_indices
        )
        response = generate(kept_tokens, case["question"])
        record = {}
        record["case_id"] = case["id"]
        record["evidence_retained"] = retained
```

```
        record["answer_correct"] = response == case["answer"]
        record["kept_length"] = len(kept_tokens)
        records.append(record)
    return records

def E06(generate, cases, maximum_tokens):
    return evaluate_policy(generate, cases, apply_left_truncation,
                           maximum_tokens)
```

Part 5 - The Philosophy Behind the Exercise

Truncation is an act of forgetting performed by the system on behalf of the model. It decides which past remains available and which past becomes computationally nonexistent. Because the removed text is invisible in the final prompt, this decision can also become invisible to the user.

Context engineering is therefore partly the ethics of omission. A system that must compress should reveal what it removed, preserve critical constraints, and avoid presenting answers as fully informed when its evidence was incomplete.

18.3.8 Exercise 07

Exercise Title: Compare Fixed Context vs Sliding Window Training Strategies

Part 1 - Why this matters

Fixed non-overlapping blocks divide a token stream into segments of length C. They are simple, efficient, and prevent repeated processing of most tokens, but relationships crossing a block boundary are never observed inside one example. Sliding windows overlap examples and can expose the same transition under more prefixes, but repeated tokens increase compute and change statistical weighting.

The comparison must control total token updates. If both datasets are trained for the same number of epochs, the overlapping dataset can present far more tokens and consume more compute. Any quality gain may come from additional training rather than the window strategy itself. Compare both equal-epoch and equal-token-budget regimes, and label them separately.

Boundary placement matters. A fixed scheme can randomly shift its starting offset each epoch to vary boundaries without full sliding overlap. Document-aware packing can avoid splitting short examples while filling most of the context. These intermediate strategies should be discussed rather than presenting two extremes as the only choices.

You will train matched small decoders on a synthetic task containing patterns that sometimes cross fixed boundaries. Evaluate ordinary next-token loss, boundary-transition loss, unique tokens seen, total token presentations, training time, and validation quality. The expected result should be stated before the experiment, then revised if measurements disagree.

Part 2 - Mathematical form and reasoning

For stream length N, context C, and stride S, approximate window count is

$$K(S) = \left\lfloor \frac{N - C - 1}{S} \right\rfloor + 1.$$

Total input token presentations are KC. The redundancy factor relative to unique stream tokens is approximately

$$R = \frac{KC}{N}.$$

For $S = C$, R is near one; for $S \ll C$, it can be much larger. Define a separate boundary set $\mathcal{B}$ and measure $L_{\mathcal{B}}$ so global loss cannot hide boundary failure.

Part 3 - Hints, controls, and evidence

1. Construct datasets from the identical token stream and verify every window start index.
2. Count total predicted tokens after masking, not only number of batches.
3. Compare equal epochs, equal optimizer steps, and equal valid-token budgets as different experiments.
4. Mark transitions that cross fixed-block boundaries and evaluate them separately.
5. Add randomized fixed offsets as a third strategy.
6. Report wall-clock time, tokens per second, redundancy, overall loss, and boundary loss across several seeds.

Part 4 - Python code skeleton

```
def window_starts(sequence_length, context_length, stride):
    starts = []
    start = 0
    while start + context_length < sequence_length:
        starts.append(start)
        start = start + stride
    return starts

def presentation_statistics(sequence_length, context_length, stride):
    starts = window_starts(sequence_length, context_length, stride)
    counts = [0] * sequence_length
    for start in starts:
        end = start + context_length
        for index in range(start, end):
            counts[index] = counts[index] + 1
    total_presentations = 0
    unique_presented = 0
    for count in counts:
        total_presentations = total_presentations + count
        if count > 0:
            unique_presented = unique_presented + 1
    redundancy = total_presentations / max(1, unique_presented)
    return starts, counts, redundancy
```

```
def E07(sequence_length, context_length):
    fixed = presentation_statistics(
        sequence_length, context_length, context_length
    )
    sliding = presentation_statistics(
        sequence_length, context_length, context_length // 4
    )
    return fixed, sliding
```

Part 5 - The Philosophy Behind the Exercise

Windows define the horizons from which the model learns. A boundary is not a natural break in the original text; it is an artifact of finite computation. Training strategy decides whether the artifact remains fixed, moves, or is softened through overlap.

The wider lesson is that efficiency and experience are coupled. Saving compute changes what the model sees, while increasing coverage changes what compute is spent repeatedly. Good engineering makes this exchange explicit and measures which repeated experiences actually create value.

18.3.9 Exercise 08

Exercise Title: Measure Memory Usage Growth as Context Length Increases

Part 1 - Why this matters

Increasing context length changes far more than the input tensor. Every layer creates token activations, attention projections, feed-forward intermediates, and, in a standard implementation, attention scores whose two sequence axes grow together. During training, many tensors must remain available for backpropagation. Parameter memory stays approximately fixed while activation memory can become the dominant cost.

PyTorch and CUDA expose several memory quantities. Allocated memory belongs to live tensors; reserved memory includes allocator-managed blocks kept for reuse; peak allocated memory records the highest live allocation since reset. External monitoring may include CUDA context, kernels, and other processes. Mixing these measurements can produce apparently contradictory reports, so the experiment must name the quantity and measurement boundary.

You will sweep context lengths that double successively while holding model, batch, precision, and implementation fixed. Measure inference, forward-plus-loss, and full training step separately. After each run, delete references to large tensors before continuing. The graph should show empirical memory and the theoretical number of attention-score elements; differences between them reveal other activations and memory-efficient kernel behavior.

An OOM result is censored data, not a zero or missing row to ignore. Record the attempted configuration, memory immediately before the operation, exception, and recovery. Do not continue with corrupted optimizer state after a failed step. Run stress tests in a disposable model instance and preserve a safety margin rather than filling the device used by other work.

Part 2 - Mathematical form and reasoning

For L layers, batch B, heads H, sequence T, and bytes per element s, the explicitly materialized attention score storage alone is

$$M_{\text{scores}} = LBHT^2 s.$$

Residual and feed-forward activations contain terms proportional to $LBTd$ and $LBTd_f$. If context doubles, the score term grows by four while linear activation terms grow by two. Actual peak memory is

$$M_{\text{peak}} = M_\theta + M_{\text{grad}} + M_{\text{optimizer}} + M_{\text{activations}} + M_{\text{workspace}},$$

with components depending on inference or training mode.

Part 3 - Hints, controls, and evidence

1. Start from a safely small context and stop before repeated OOM attempts threaten the session.
2. Reset peak memory immediately before each measured region and synchronize before reading the result.
3. Use fresh random inputs but identical dtype, batch, and vocabulary range.
4. Record allocated and reserved memory before and after the operation.
5. Compare `no_grad`, forward with graph retention, and backward passes.
6. Repeat each successful shape and report variation rather than a single measurement.

Part 4 - Python code skeleton

```
import gc
import torch

def measured_training_step(model, input_ids, targets, optimizer):
    device = input_ids.device
    torch.cuda.synchronize(device)
    torch.cuda.reset_peak_memory_stats(device)
    baseline = torch.cuda.memory_allocated(device)
    optimizer.zero_grad(set_to_none=True)
    logits = model(input_ids)
    loss = torch.nn.functional.cross_entropy(
        logits.reshape(-1, logits.shape[-1]), targets.reshape(-1)
    )
    loss.backward()
    optimizer.step()
    torch.cuda.synchronize(device)
    peak = torch.cuda.max_memory_allocated(device)
    return float(loss.detach().cpu()), peak - baseline

def E08(model, optimizer, vocabulary_size, context_lengths, device):
    if device.type != "cuda":
        raise RuntimeError("E08 requires CUDA memory measurements")
    records = []
    for context_length in context_lengths:
```

```
        input_ids = torch.randint(
            0, vocabulary_size, (1, context_length), device=device
        )
        targets = torch.randint(
            0, vocabulary_size, (1, context_length), device=device
        )
        try:
            loss, memory = measured_training_step(
                model, input_ids, targets, optimizer
            )
            records.append({"context": context_length,
                            "loss": loss, "peak_delta": memory,
                            "status": "ok"})
        except torch.OutOfMemoryError:
            records.append({"context": context_length,
                            "status": "oom"})
            del input_ids
            del targets
            gc.collect()
            torch.cuda.empty_cache()
            break
    return records
```

Part 5 - The Philosophy Behind the Exercise

Context has a physical body. Each additional token is not only another piece of meaning; it becomes vectors, matrix interactions, stored activations, memory traffic, and energy. The abstract promise of remembering more is limited by the material cost of making more information simultaneously available.

Measurement turns that limitation from frustration into knowledge. An OOM is not merely a crash but evidence about the boundary between an algorithm and a machine. Responsible engineering approaches that boundary deliberately, records it accurately, and leaves enough space for recovery.

18.3.10 Exercise 09

Exercise Title: Explain Why Long Context Is Quadratic in Attention Cost

Part 1 - Why this matters

Dense self-attention compares every query position with every permitted key position. A sequence of length T therefore creates T^2 potential pairs per head before causal masking removes future influence. The triangular mask changes which pairs contribute but a naive dense matrix implementation still allocates and computes a square score tensor.

Quadratic pair growth does not mean the entire Transformer is always quadratic. Projection and feed-forward layers contain large terms linear in T, and at short contexts or wide hidden dimensions they can dominate runtime. The point at which attention becomes the bottleneck depends on width, heads, batch, kernel efficiency, precision, and hardware. A complexity statement should guide measurement, not replace it.

You will count visible and materialized pairs, benchmark matrix multiplication across doubling lengths, and compare measured ratios with theoretical ratios. Then implement a local sliding-window visibility pattern to show how restricting each query to at most W keys changes pair count toward $\mathcal{O}(TW)$. This conceptual alternative sacrifices direct global access and therefore changes the model, not merely its speed.

Memory-efficient attention kernels can compute exact dense attention without materializing the complete score matrix in global memory. They reduce memory traffic and peak storage but do not change the logical all-pairs interaction or its arithmetic scaling. Distinguish algorithmic complexity, implementation memory, and measured latency in the final explanation.

Part 2 - Mathematical form and reasoning

For B batches and H heads,

$$N_{\text{dense}} = BHT^2, \qquad N_{\text{causal visible}} = BH\frac{T(T+1)}{2}.$$

Score and value aggregation require approximately

$$F_{\text{attention}} \approx 4BHT^2d_k = 4BT^2d$$

multiply-add FLOPs under the two-operation convention. With local window W, visible pairs are bounded by approximately $BHTW$ for $W \ll T$.

Part 3 - Hints, controls, and evidence

1. Generate a table for $T = 128$ through at least $T = 2048$ without allocating every theoretical tensor.
2. Show the ratio when T doubles for dense, causal-visible, and local attention.
3. Benchmark only after warm-up and CUDA synchronization.
4. Record head dimension and total model width; head count alone does not determine total projection width.
5. Compare explicit score materialization with a memory-efficient PyTorch attention primitive when available.
6. Explain which results demonstrate asymptotic growth and which reflect a particular kernel or GPU.

Part 4 - Python code skeleton

```
import time
import torch

def pair_counts(batch_size, heads, sequence_length, window=None):
    dense = batch_size * heads * sequence_length * sequence_length
    causal = (batch_size * heads * sequence_length *
              (sequence_length + 1) // 2)
    local = None
```

```
    if window is not None:
        local = batch_size * heads * sequence_length * min(
            sequence_length, window
        )
    return dense, causal, local

def benchmark_scores(q, k, repeats):
    for _ in range(10):
        q @ k.transpose(-2, -1)
    torch.cuda.synchronize(q.device)
    start = time.perf_counter()
    for _ in range(repeats):
        scores = q @ k.transpose(-2, -1)
    torch.cuda.synchronize(q.device)
    elapsed = time.perf_counter() - start
    return elapsed / repeats, scores.shape

def E09(device):
    records = []
    for sequence_length in [128, 256, 512, 1024, 2048]:
        dense, causal, local = pair_counts(
            2, 8, sequence_length, window=128
        )
        record = {}
        record["context"] = sequence_length
        record["dense_pairs"] = dense
        record["causal_visible_pairs"] = causal
        record["local_pairs_upper_bound"] = local
        records.append(record)
    return records
```

Part 5 - The Philosophy Behind the Exercise

Attention grants direct relationships by paying for possible relationships. The quadratic matrix is a map of who may consult whom. Global access is powerful because no chain of intermediaries is required, but universal access becomes expensive as the community of tokens grows.

Efficiency methods are therefore theories of relevance. Local windows, sparse patterns, retrieval, and compression each claim that not every pair deserves equal computational opportunity. Reducing cost means deciding which relationships can be omitted without losing the behavior we value.

18.3.11 Exercise 10

Exercise Title: Train Small LLM with Short Context on CUDA and Benchmark Speed

Part 1 - Why this matters

A short-context training run is the safest environment for validating the whole language-model pipeline: token batches, target alignment, causal attention, loss, backward propagation,

optimizer updates, CUDA placement, and throughput. Before scaling context or parameters, the model should deliberately overfit a small corpus. Failure to overfit indicates a correctness problem that more compute will only make expensive.

Benchmarking must separate data preparation, host-to-device transfer, forward, backward, and optimizer time. A low GPU utilization graph does not identify the bottleneck by itself. Small models may launch kernels too briefly to saturate the GPU, while a slow DataLoader can leave the device waiting. Report both end-to-end and device-only throughput.

You will train the MiniGPT reference from LLM00 using a context such as 32 or 64. Use pinned host memory and non-blocking transfer only when CUDA and DataLoader conditions support them. Synchronize at measurement boundaries, not after every operation during ordinary training, because excessive synchronization destroys pipeline overlap.

Correctness metrics come before speed. Log valid tokens, loss per valid token, gradient norm, learning rate, parameter finiteness, peak memory, and tokens per second. A fast loop that forgot `zero_grad`, shifted labels incorrectly, or timed only asynchronous kernel submission is not a benchmark.

Part 2 - Mathematical form and reasoning

If a measured interval processes N_{valid} targets in Δt, throughput is

$$Q = \frac{N_{\text{valid}}}{\Delta t} \quad \text{tokens per second.}$$

For batch B, context T, and K optimizer steps, the nominal count is KBT, but padding or ignored labels require counting the mask instead. Speedup from CPU to GPU is

$$S = \frac{t_{\text{CPU}}}{t_{\text{GPU}}},$$

reported for matched numerical work and batch configuration.

Part 3 - Hints, controls, and evidence

1. Set Python and PyTorch seeds, save configuration, and record GPU name and software versions.
2. Overfit one or a few batches before using the full training loader.
3. Warm up CUDA before starting benchmark timing.
4. Count valid target tokens from the loss mask each step.
5. Compare several batch sizes at fixed context and several contexts at fixed batch.
6. Report median step time after excluding warm-up, end-to-end throughput, device-only throughput, and peak memory.

Part 4 - Python code skeleton

```
import time
import torch
import torch.nn.functional as F
```

```
def train_step(model, batch, optimizer, device):
    input_ids = batch["input_ids"].to(device, non_blocking=True)
    target_ids = batch["target_ids"].to(device, non_blocking=True)
    valid_mask = batch["valid_mask"].to(device, non_blocking=True)
    optimizer.zero_grad(set_to_none=True)
    logits = model(input_ids)
    losses = F.cross_entropy(
        logits.reshape(-1, logits.shape[-1]),
        target_ids.reshape(-1), reduction="none"
    ).view_as(target_ids)
    selected = losses.masked_select(valid_mask)
    loss = selected.mean()
    loss.backward()
    gradient_norm = torch.nn.utils.clip_grad_norm_(
        model.parameters(), 1.0
    )
    optimizer.step()
    return loss.detach(), gradient_norm.detach(), selected.numel()

def E10(model, loader, optimizer, device, maximum_steps):
    model.to(device)
    model.train()
    records = []
    iterator = iter(loader)
    for step in range(maximum_steps):
        try:
            batch = next(iterator)
        except StopIteration:
            iterator = iter(loader)
            batch = next(iterator)
        if device.type == "cuda":
            torch.cuda.synchronize(device)
        start = time.perf_counter()
        loss, gradient_norm, valid_tokens = train_step(
            model, batch, optimizer, device
        )
        if device.type == "cuda":
            torch.cuda.synchronize(device)
        elapsed = time.perf_counter() - start
        records.append({"step": step, "loss": float(loss.cpu()),
                        "gradient_norm": float(gradient_norm.cpu()),
                        "tokens_per_second": valid_tokens / elapsed})
    return records
```

Part 5 - The Philosophy Behind the Exercise

Training a tiny model well is a more serious achievement than launching a large model carelessly. Small scale removes excuses: every tensor can be inspected, the data can be memo-

rized, and failures can be traced. It creates a laboratory where understanding is cheaper than guessing.

Speed acquires meaning only after correctness. Hardware can accelerate whatever objective we give it, including a broken one. The discipline of benchmarking is to prove the work, expose the measurement boundary, and only then celebrate how quickly the machine performed it.

18.3.12 Exercise 11

Exercise Title: Compare Generation Quality With Short vs Long Context

Part 1 - Why this matters

A longer context can improve generation only when it contains useful information and the model can exploit it. Adding irrelevant history can distract attention, shift style, increase latency, or expose conflicting instructions. Therefore the comparison is not "more tokens versus fewer tokens" in isolation; it is which evidence is available, where it is placed, and whether additional context changes the answer for the right reason.

Use the same model and decoding configuration for both conditions. A short prompt should contain the immediate query and minimum instructions. A long prompt should add controlled evidence, distractors, or conversation history. When testing the value of relevant context, the short version must omit the needed fact; when testing distraction, both versions must retain the fact and differ only in irrelevant content.

Generation quality needs task-specific metrics. For factual questions, measure evidence-supported accuracy and citation alignment. For continuation, measure held-out log-likelihood and human coherence ratings. For structured output, measure schema validity. Also record latency, prompt-token count, generated tokens, and failure categories. One subjective sample cannot establish a context-length effect.

You will create matched prompt pairs and randomize whether useful evidence appears near the beginning, middle, or end. Run multiple seeds for stochastic decoding and use paired differences per case. The analysis must identify positive context value, no effect, distraction, contradiction sensitivity, and truncation.

Part 2 - Mathematical form and reasoning

For quality metric q on case i, define paired context gain

$$\Delta_i = q(y_i^{\text{long}}, r_i) - q(y_i^{\text{short}}, r_i).$$

Mean gain is $\bar{\Delta} = N^{-1} \sum_i \Delta_i$, but report its distribution and confidence interval. A context-efficiency measure can be written as

$$E_{\text{context}} = \frac{\bar{\Delta}}{\overline{T_{\text{long}} - T_{\text{short}}}},$$

while recognizing that quality is rarely linear in tokens.

Part 3 - Hints, controls, and evidence

1. Use the exact same query, system instruction, decoding seed, and output budget across each pair.

2. Verify the long prompt does not exceed the actual context limit after tokenization.

3. Create relevant-only, distractor-only, relevant-plus-distractor, and conflicting-evidence conditions.

4. Log the token positions of required evidence.

5. Score claim support separately from fluency and formatting.

6. Inspect cases with negative Δ_i and explain whether failure came from distraction, position, contradiction, or changed truncation.

Part 4 - Python code skeleton

```
def paired_context_evaluation(generate, cases, decoding_config):
    records = []
    for case in cases:
        short_output = generate(
            case["short_prompt"], decoding_config
        )
        long_output = generate(
            case["long_prompt"], decoding_config
        )
        short_score = case["score_function"](
            short_output, case["reference"]
        )
        long_score = case["score_function"](
            long_output, case["reference"]
        )
        record = {}
        record["case_id"] = case["id"]
        record["condition"] = case["condition"]
        record["short_output"] = short_output
        record["long_output"] = long_output
        record["short_score"] = short_score
        record["long_score"] = long_score
        record["gain"] = long_score - short_score
        record["evidence_positions"] = case["evidence_positions"]
        records.append(record)
    return records

def E11(generate, cases, decoding_config):
    records = paired_context_evaluation(
        generate, cases, decoding_config
    )
    total_gain = 0.0
    for record in records:
        total_gain = total_gain + record["gain"]
    mean_gain = total_gain / max(1, len(records))
    return records, mean_gain
```

Part 5 - The Philosophy Behind the Exercise

More context is more opportunity, not more understanding. It expands the field from which the model may draw, but it also expands the field of distraction and conflict. Information has value only when a system can identify, preserve, and use what matters.

The experiment replaces a quantity myth with a relevance question. The best prompt is not necessarily the longest prompt. It is the smallest auditable context that preserves the evidence and constraints required for the decision.

18.3.13 Exercise 12

Exercise Title: Analyze Prompt Sensitivity Due to Tokenization Differences

Part 1 - Why this matters

Prompts that appear semantically equivalent to a human can become different token sequences. Leading spaces, newline style, capitalization, Unicode normalization, punctuation, and spelling variants can change token boundaries and counts. Because embeddings and later computations begin from those identifiers, small surface changes can alter the next-token distribution and the amount of context consumed.

Prompt sensitivity has multiple causes. Tokenization is one; learned association with formatting is another; stochastic decoding is a third. To attribute a change to tokenization, you must hold model, decoding, seed, and semantic intent fixed, then inspect the encoded sequences. A changed output alone does not prove that token boundaries caused the difference.

You will build families of prompts that differ by one controlled transformation: space insertion, newline replacement, case, Unicode normalization, punctuation, or a typographic character. Record identifiers, decoded token pieces, token count, shared-prefix length, logit divergence at the answer position, selected token, and final task outcome. Use greedy decoding for the primary experiment to remove sampling variance.

Some differences should be preserved rather than normalized away. Case can distinguish names; whitespace matters in Python; Unicode marks can change words. The goal is not to force all prompts into one form. It is to understand which normalizations are semantically safe for a particular application and to test them before deployment.

Part 2 - Mathematical form and reasoning

Let two surface forms s and s' map to token sequences $x = \tau(s)$ and $x' = \tau(s')$. Token edit distance can be normalized as

$$d_\tau(s, s') = \frac{\text{EditDistance}(x, x')}{\max(|x|, |x'|)}.$$

For next-token distributions p and q, use a symmetric divergence such as

$$\text{JSD}(p, q) = \frac{1}{2} D_{\text{KL}}(p \| m) + \frac{1}{2} D_{\text{KL}}(q \| m), \qquad m = \frac{p + q}{2}.$$

Compare divergence with token edit distance and task-output changes without assuming a linear relationship.

Part 3 - Hints, controls, and evidence

1. Preserve raw UTF-8 text and print Unicode code points for suspicious characters.

2. Decode each individual token piece, including visible markers for spaces or bytes.
3. Use greedy decoding and evaluation mode for deterministic comparisons.
4. Measure common token prefix length and the first changed token position.
5. Add transformations that should preserve meaning and transformations that intentionally change meaning as controls.
6. Create regression tests for the application's accepted prompt formats.

Part 4 - Python code skeleton

```
import unicodedata
import torch

def unicode_report(text):
    records = []
    for character in text:
        record = {}
        record["character"] = character
        record["code_point"] = "U+%04X" % ord(character)
        record["name"] = unicodedata.name(character, "UNKNOWN")
        records.append(record)
    return records

def jensen_shannon_divergence(logits_a, logits_b):
    log_p = torch.log_softmax(logits_a, dim=-1)
    log_q = torch.log_softmax(logits_b, dim=-1)
    p = torch.exp(log_p)
    q = torch.exp(log_q)
    mixture = 0.5 * (p + q)
    log_mixture = torch.log(mixture.clamp_min(1e-30))
    kl_p = (p * (log_p - log_mixture)).sum(dim=-1)
    kl_q = (q * (log_q - log_mixture)).sum(dim=-1)
    return 0.5 * (kl_p + kl_q)

def E12(model, tokenizer, prompt_variants, device):
    model.eval()
    records = []
    with torch.no_grad():
        base_ids = tokenizer.encode(prompt_variants[0])
        base_tensor = torch.tensor([base_ids], device=device)
        base_logits = model(base_tensor)[:, -1, :]
        for prompt in prompt_variants:
            token_ids = tokenizer.encode(prompt)
            input_ids = torch.tensor([token_ids], device=device)
            logits = model(input_ids)[:, -1, :]
            record = {}
```

```
            record["prompt"] = prompt
            record["token_ids"] = token_ids
            record["token_count"] = len(token_ids)
            record["jsd"] = float(
                jensen_shannon_divergence(base_logits, logits).cpu()
            )
            records.append(record)
    return records
```

Part 5 - The Philosophy Behind the Exercise

To a human reader, typography often appears to carry the same meaning through many surface forms. To a model, the surface is the entrance to meaning. The tokenizer converts tiny visual choices into different computational histories.

This does not imply that models merely react to typography. It shows that every abstract capability is reached through a concrete representation. Robustness means preserving behavior across variations that should be irrelevant while remaining sensitive to variations that genuinely change intent.

18.3.14 Exercise 13

Exercise Title: Implement Padding-Free Training Using Packed Sequences Conceptually

Part 1 - Why this matters

Padding wastes computation when many short sequences are expanded to the longest sequence in a batch. Packing concatenates multiple examples into fixed token blocks so more positions contain real training targets. However, naive concatenation allows tokens in one document to attend to a previous document, creating artificial cross-example context. Padding-free training is therefore a masking and position problem, not only a storage optimization.

A correct packed batch needs segment identifiers or cumulative boundaries. Attention is causal inside each segment but blocked across segments. Position indices can reset at each new example, depending on the positional scheme and training design. Loss should include valid within-segment next-token transitions and normally exclude a target that crosses from one example's final token to the next example's first token unless an explicit end-of-document transition is intended.

You will write a greedy packer that fills blocks of fixed capacity with tokenized examples. It must record token identifiers, segment identifiers, positions within segments, and a loss mask. Then build a block-diagonal causal attention mask and compare packed loss with the sum of separately evaluated sequence losses. Numerical parity establishes semantic equivalence before any throughput claim.

Packing efficiency depends on example ordering. Sorting by length can improve utilization but change data order and reduce stochastic mixing. Greedy packing is not always optimal. Record utilization and preserve a reproducible ordering policy rather than hiding the packing heuristic inside preprocessing.

Part 2 - Mathematical form and reasoning

Let segment label g_i identify the original example for packed position i. Allowed attention is

$$A_{i,j} = \mathbb{I}[j \le i]\mathbb{I}[g_i = g_j].$$

The next-token loss mask is

$$m_i = \mathbb{I}[g_i = g_{i+1}],$$

possibly extended to include a deliberate end-of-document target. Packing utilization is

$$U = \frac{N_{\text{real tokens}}}{N_{\text{blocks}} C}.$$

Part 3 - Hints, controls, and evidence

1. Start with three sequences whose lengths and boundaries can be inspected manually.
2. Never infer segment boundaries from token values; store segment IDs explicitly.
3. Reset position indices at segment boundaries and compare with continuous positions as a separate experiment.
4. Assert no allowed attention entry connects different segment IDs.
5. Compare packed and separate logits or losses in evaluation mode with dropout disabled.
6. Report real-token utilization, valid-target utilization, packing time, throughput, and any ordering change.

Part 4 - Python code skeleton

```
import torch

def pack_examples(examples, block_size, padding_id):
    blocks = []
    current_tokens = []
    current_segments = []
    current_positions = []
    segment_id = 0
    for example in examples:
        if len(example) > block_size:
            raise ValueError("example exceeds block size")
        if len(current_tokens) + len(example) > block_size:
            blocks.append((current_tokens, current_segments,
                           current_positions))
            current_tokens = []
            current_segments = []
            current_positions = []
        for position in range(len(example)):
            current_tokens.append(example[position])
            current_segments.append(segment_id)
            current_positions.append(position)
```

```
        segment_id = segment_id + 1
    if len(current_tokens) > 0:
        while len(current_tokens) < block_size:
            current_tokens.append(padding_id)
            current_segments.append(-1)
            current_positions.append(0)
        blocks.append((current_tokens, current_segments, current_positions))
    return blocks

def packed_attention_mask(segment_ids):
    same_segment = segment_ids[:, None] == segment_ids[None, :]
    real_segment = segment_ids[:, None] >= 0
    causal = torch.tril(torch.ones(
        segment_ids.numel(), segment_ids.numel(),
        device=segment_ids.device, dtype=torch.bool
    ))
    return same_segment & real_segment & causal

def E13(examples, block_size, padding_id, device):
    blocks = pack_examples(examples, block_size, padding_id)
    first_segments = torch.tensor(blocks[0][1], device=device)
    mask = packed_attention_mask(first_segments)
    row_segments = first_segments[:, None]
    column_segments = first_segments[None, :]
    cross_segment = (row_segments != column_segments) & mask
    assert not cross_segment.any()
    return blocks, mask
```

Part 5 - The Philosophy Behind the Exercise

Packing asks whether empty space can be removed without erasing boundaries. Efficiency wants examples close together; semantics requires them to remain distinct. The solution is not physical separation but an explicit rule about which positions may communicate.

This is a recurring idea in computation: structure can be virtual. Tokens share a tensor and a GPU kernel while inhabiting different informational worlds. The mask preserves those worlds, showing that optimization is safe only when it maintains the meaning of the original computation.

18.3.15 Exercise 14

Exercise Title: Debug Attention Mask Bugs That Leak Future Tokens

Part 1 - Why this matters

Future-token leakage allows a training position to use the answer it is supposed to predict. Loss can collapse rapidly and validation can look excellent if the same faulty mask is used everywhere. During real autoregressive generation, future tokens do not exist, so performance fails. This is one of the most dangerous bugs because optimization rewards it.

Leakage can arise from a missing mask, reversed triangle, wrong diagonal, broadcasting over query rather than key axes, mask application after softmax, or an optimized kernel receiving

the wrong causal flag. A plot of one mask is useful but insufficient; the strongest test changes the future and proves that earlier logits remain invariant.

You will implement a fault-injection suite. Each faulty mask must be expected to fail at least one property: forbidden weights equal zero, row sums remain one, prefix outputs are invariant to suffix changes, and incremental last-token logits match the corresponding full-sequence logits. A test that never fails under a known bug provides weak evidence.

Run tests in evaluation mode with dropout disabled and fixed weights. Numerical tolerances should be chosen before results are seen. When using mixed precision, distinguish small parity differences from structural leakage; forbidden weights should still be zero under correct masking.

Part 2 - Mathematical form and reasoning

Causality requires

$$\frac{\partial Z_{t,:}}{\partial X_j} = 0 \qquad \text{for all } j > t,$$

interpreted through discrete suffix intervention rather than literal gradients with respect to integer IDs. If sequences X and X' satisfy $X_{:\le t} = X'_{:\le t}$, then a deterministic causal model must satisfy

$$f_\theta(X)_{:,t,:} = f_\theta(X')_{:,t,:}.$$

This functional property tests the whole stack, not only one displayed mask.

Part 3 - Hints, controls, and evidence

1. Create two sequences with identical prefixes and entirely different suffixes.
2. Compare every prefix position, not only position zero.
3. Inject missing, reversed, and one-token-too-permissive masks.
4. Inspect attention weights in every layer and head while also testing final logits.
5. Compare full-sequence and prefix-only logits at the same position.
6. Make the regression suite run on CPU and CUDA; label tolerance differences without weakening the causal invariant.

Part 4 - Python code skeleton

```
import torch

def prefix_invariance_error(model, sequence_a, sequence_b,
                            prefix_length):
    model.eval()
    assert torch.equal(sequence_a[:, :prefix_length],
                       sequence_b[:, :prefix_length])
    with torch.no_grad():
        logits_a = model(sequence_a)
        logits_b = model(sequence_b)
```

```
    difference = logits_a[:, :prefix_length, :] - \
                 logits_b[:, :prefix_length, :]
    return difference.abs().max()

def full_prefix_parity_error(model, full_sequence, prefix_length):
    prefix = full_sequence[:, :prefix_length]
    model.eval()
    with torch.no_grad():
        full_logits = model(full_sequence)[:, prefix_length - 1, :]
        prefix_logits = model(prefix)[:, -1, :]
    return (full_logits - prefix_logits).abs().max()

def E14(model, vocabulary_size, device):
    torch.manual_seed(1401)
    sequence_a = torch.randint(
        0, vocabulary_size, (2, 12), device=device
    )
    sequence_b = sequence_a.clone()
    sequence_b[:, 6:] = torch.randint(
        0, vocabulary_size, (2, 6), device=device
    )
    invariance_error = prefix_invariance_error(
        model, sequence_a, sequence_b, 6
    )
    parity_error = full_prefix_parity_error(model, sequence_a, 6)
    assert invariance_error < 1e-5
    assert parity_error < 1e-5
    return invariance_error, parity_error
```

Part 5 - The Philosophy Behind the Exercise

Leakage is a false form of intelligence created by access to forbidden evidence. The model appears to predict because the experiment quietly allows it to read the answer. Better optimization makes the deception more convincing, not less.

Causality is therefore not inferred from high performance. It is proven through intervention: change what lies beyond the information boundary and show that the past remains unchanged. This principle extends far beyond attention masks to data leakage, temporal forecasting, and every evaluation where future evidence must remain genuinely unavailable.

18.3.16 Exercise 15

Exercise Title: Stress Test Context Handling on GPU Until OOM Happens

Part 1 - Why this matters

The maximum usable context on a GPU is not a number printed in the architecture configuration. It depends on model size, batch, precision, attention implementation, gradient requirements, optimizer state, fragmentation, and other allocations. A controlled stress test maps this operational boundary and reveals whether the system fails cleanly.

The phrase "until OOM" does not authorize reckless allocation. Begin from a safe baseline, grow context geometrically, stop at the first failure, and then use bounded search between the last success and first failure. Preserve a safety margin for CUDA context and recovery. Never stress a shared production GPU or run concurrent tests that can affect other users.

Measure inference and training separately. Inference under `torch.no_grad()` avoids gradients and saved activations; training adds the backward graph and optimizer storage. Also distinguish prefill from incremental decoding: prompt processing scales with full context, while KV cache grows linearly per generated token even though each decode step attends to all cached keys.

OOM recovery must be tested. The exception may leave Python references and gradients alive. Release the failed batch and outputs, set gradients to none, run garbage collection, clear unused cached blocks, and validate a small forward pass before continuing. If recovery fails, terminate the disposable process rather than pretending the session is healthy.

Part 2 - Mathematical form and reasoning

Let $M(T)$ be measured peak memory for context T and M_{safe} the allowed device budget. The operational maximum is

$$T^* = \max\{T : M(T) \leq M_{\text{safe}}\}.$$

Use a safety factor $0 < \eta < 1$ so $M_{\text{safe}} = \eta M_{\text{device}}$. After geometric bracketing, binary search reduces the interval, but memory step behavior need not be smooth because kernels and allocator bins can change discretely.

Part 3 - Hints, controls, and evidence

1. Record GPU total and free memory before the test and stop if the device is already under unrelated load.
2. Use a fresh model or subprocess for stress testing whenever possible.
3. Increase context by a modest factor such as 1.5 or 2, not an enormous jump.
4. Record success, latency, allocated peak, reserved peak, and exception text.
5. Verify recovery with a known small input after every OOM.
6. Report the largest tested success with batch, dtype, mode, kernel, and safety margin; never publish context length alone.

Part 4 - Python code skeleton

```
import gc
import torch

def try_context(model, context_length, vocabulary_size, device,
                training_mode):
    input_ids = None
    logits = None
    loss = None
```

```
    try:
        torch.cuda.reset_peak_memory_stats(device)
        input_ids = torch.randint(
            0, vocabulary_size, (1, context_length), device=device
        )
        if training_mode:
            model.zero_grad(set_to_none=True)
            logits = model(input_ids)
            loss = logits.float().square().mean()
            loss.backward()
        else:
            with torch.no_grad():
                logits = model(input_ids)
        torch.cuda.synchronize(device)
        peak = torch.cuda.max_memory_allocated(device)
        return True, peak, None
    except torch.OutOfMemoryError as error:
        return False, None, str(error)
    finally:
        del input_ids
        del logits
        del loss
        model.zero_grad(set_to_none=True)
        gc.collect()
        torch.cuda.empty_cache()

def E15(model, vocabulary_size, device):
    if device.type != "cuda":
        raise RuntimeError("E15 requires a CUDA device")
    results = []
    context_length = 128
    while True:
        success, peak, error = try_context(
            model, context_length, vocabulary_size, device, True
        )
        results.append({"context": context_length,
                        "success": success,
                        "peak": peak, "error": error})
        if not success:
            break
        context_length = context_length * 2
    return results
```

Part 5 - The Philosophy Behind the Exercise

A resource boundary becomes useful knowledge only when approached with respect. Stress testing is not destruction for its own sake; it is a controlled encounter with failure so that production does not discover the boundary first.

The maximum is also not the recommended operating point. Systems need room for variation, concurrency, and recovery. Maturity means knowing both how far the machine can be pushed and why ordinary operation should remain safely inside that limit.

18.3.17 Exercise 16

Exercise Title: Explain Why Context Extension Is Harder Than Parameter Scaling

Part 1 - Why this matters

Adding parameters changes model capacity while preserving the input domain when the architecture is scaled consistently. Extending context changes the domain of positions and relationships the model must process. Positions beyond the training range may be unseen, dense attention cost expands quadratically, and long-distance behavior must be taught with data that actually contains useful dependencies at those distances.

Absolute learned position tables have a direct shape limit: new rows must be initialized or interpolated. Sinusoidal and rotary methods can produce values at new positions, but mathematical availability is not proof that the network will use them reliably. Attention logits can change scale, phase relationships can move outside the trained distribution, and short-context behavior may regress after adaptation.

Context extension also changes systems infrastructure. Training batches may need gradient accumulation, sequence parallelism, memory-efficient attention, or smaller batch size. Evaluation must include evidence at many distances, positions, and distractor densities. Serving requires larger KV caches and new admission-control rules. A configuration field can be changed in seconds while the supporting training and validation remain substantial engineering work.

You will attempt a controlled extension of a small model beyond its original training length. Compare naive configuration change, position interpolation or scaling, and continued training on mixed short and long examples. Measure original-length loss, extended-length loss, retrieval by distance, memory, latency, and stability. Success requires retaining short-context quality as well as improving long-context performance.

Part 2 - Mathematical form and reasoning

If training positions are $p \in [0, C_{\text{train}} - 1]$ and the target context is C_{new}, a simple interpolation maps

$$p' = p \frac{C_{\text{train}} - 1}{C_{\text{new}} - 1}.$$

This keeps new positions inside the numerical training range but compresses relative distances. Extension quality should be expressed as a pair

$$(\Delta Q_{\text{long}}, \Delta Q_{\text{short}}),$$

not one long-context score. Compute and memory changes must be reported beside quality because T affects attention quadratically.

Part 3 - Hints, controls, and evidence

1. Save the original checkpoint and evaluate it on the full short-context suite before modification.

2. Test naive extension first as a failure baseline.

3. Continue training with a mixture of short and long examples rather than replacing the original distribution completely.

4. Evaluate key retrieval at multiple absolute positions and relative distances.

5. Inspect attention and activation magnitudes for numerical drift.

6. Report memory and latency at the old and new limits and define a rollback criterion for short-context regression.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def interpolate_position_table(old_table, new_length):
    old_length, dimension = old_table.shape
    source = old_table.transpose(0, 1).unsqueeze(0)
    resized = F.interpolate(
        source, size=new_length, mode="linear", align_corners=True
    )
    return resized.squeeze(0).transpose(0, 1)

def replace_position_embedding(model, new_length):
    old_weight = model.position_embedding.weight.detach()
    new_weight = interpolate_position_table(old_weight, new_length)
    new_embedding = torch.nn.Embedding(
        new_length, old_weight.shape[1],
        device=old_weight.device, dtype=old_weight.dtype
    )
    with torch.no_grad():
        new_embedding.weight.copy_(new_weight)
    model.position_embedding = new_embedding
    model.context = new_length
    return model

def E16(model, old_context, new_context, evaluation_function):
    original_metrics = evaluation_function(model, old_context)
    extended_model = replace_position_embedding(model, new_context)
    old_range_after = evaluation_function(extended_model, old_context)
    new_range = evaluation_function(extended_model, new_context)
    return original_metrics, old_range_after, new_range
```

Part 5 - The Philosophy Behind the Exercise

Capacity growth and horizon growth answer different ambitions. More parameters ask the model to hold richer transformations. More context asks it to remain coherent across a larger field of relationships. The second changes not only how much the model can know, but how far information must remain usable.

Extending a horizon is harder than opening a larger window. The system must learn what to preserve across the distance, how to locate it, and how not to lose its short-range competence. A numerical limit is easy to edit; an effective memory is earned through training and evidence.

18.3.18 Exercise 17

Exercise Title: Compare RoPE vs Absolute Positional Encoding for Long Context

Part 1 - Why this matters

Self-attention without positional information cannot distinguish permutations of the same token representations. Absolute positional embeddings add a vector associated with each index. Rotary Position Embedding instead rotates query and key coordinate pairs by position-dependent angles, making their dot products depend on relative displacement. These mechanisms introduce order through different geometry and extrapolate differently.

Learned absolute embeddings are flexible inside the trained table but have no native row beyond its length. Sinusoidal absolute embeddings can be evaluated beyond training positions but still inject position by addition. RoPE has no fixed lookup-table limit in its basic formula, yet high positions create angles outside the trained regime and finite precision can damage phase resolution. Long-context extensions often modify RoPE frequencies or position scaling, which changes the geometry and must be evaluated.

You will implement learned absolute positions and RoPE for a tiny attention module. Verify shapes, norm preservation under rotation, relative-shift behavior, and CPU–CUDA parity. Train matched models on an order-sensitive task within a short range, then test both inside and beyond that range. Do not claim general long-context superiority from one toy task; use it to expose mechanisms.

The comparison must preserve parameter fairness. Learned absolute embeddings add $C_{\max}d$ parameters, while basic RoPE does not. Record that difference, but also report runtime, memory, training convergence, short-range accuracy, and extrapolation. Position method is one component of long-context capability, not a substitute for long-context data.

Part 2 - Mathematical form and reasoning

Absolute embedding computes

$$h_t = e(x_t) + p_t.$$

For each two-dimensional pair of a query, RoPE applies

$$R(t\omega) = \begin{bmatrix} \cos(t\omega) & -\sin(t\omega) \\ \sin(t\omega) & \cos(t\omega) \end{bmatrix}.$$

Then

$$(R(i\omega)q)^{\mathsf{T}}(R(j\omega)k) = q^{\mathsf{T}}R((j-i)\omega)k,$$

so the dot product contains relative-position structure. Rotation should preserve vector norm up to numerical tolerance.

Part 3 - Hints, controls, and evidence

1. Require an even rotary dimension and state whether all or part of each head is rotated.

2. Compute sine and cosine in a stable dtype, then cast deliberately.
3. Assert norm preservation before integrating RoPE into attention.
4. Shift both query and key positions by the same offset and test dot-product behavior.
5. Train and evaluate across multiple lengths, including positions beyond the learned absolute table where that model must fail explicitly.
6. Compare base RoPE, scaled positions, and absolute embeddings without changing the remaining model configuration.

Part 4 - Python code skeleton

```
import torch

def rope_frequencies(rotary_dimension, device):
    indices = torch.arange(0, rotary_dimension, 2,
                           device=device, dtype=torch.float32)
    inverse_frequency = 1.0 / (
        10000.0 ** (indices / rotary_dimension)
    )
    return inverse_frequency

def apply_rope(x, positions):
    rotary_dimension = x.shape[-1]
    if rotary_dimension % 2 != 0:
        raise ValueError("rotary dimension must be even")
    inverse_frequency = rope_frequencies(
        rotary_dimension, x.device
    )
    angles = positions.to(torch.float32)[:, None] * \
             inverse_frequency[None, :]
    cosine = torch.cos(angles).to(x.dtype)
    sine = torch.sin(angles).to(x.dtype)
    even = x[..., 0::2]
    odd = x[..., 1::2]
    rotated_even = even * cosine - odd * sine
    rotated_odd = even * sine + odd * cosine
    output = torch.empty_like(x)
    output[..., 0::2] = rotated_even
    output[..., 1::2] = rotated_odd
    return output

def E17(device):
    positions = torch.arange(64, device=device)
    x = torch.randn(64, 32, device=device)
    rotated = apply_rope(x, positions)
    original_norm = torch.linalg.vector_norm(x, dim=-1)
```

```
    rotated_norm = torch.linalg.vector_norm(rotated, dim=-1)
    assert torch.allclose(original_norm, rotated_norm,
                          atol=1e-5, rtol=1e-5)
    return x, rotated
```

Part 5 - The Philosophy Behind the Exercise

Position is not contained in a token; it is a relationship introduced by the model. Absolute embeddings give every location an identity. RoPE gives relationships a changing phase. Both are mathematical ways of making order available to attention.

The comparison reveals a broader principle: representation determines which generalizations are natural. A table remembers locations it has rows for; a rotation expresses relative geometry that can be evaluated farther away. Yet a formula's ability to produce a number is not the same as a trained model's ability to use that number wisely.

18.3.19 Exercise 18

Exercise Title: Build Context Handling Debug Checklist

Part 1 - Why this matters

Context bugs cross subsystem boundaries. The raw prompt can be correct while Unicode normalization changes it; token IDs can be correct while truncation removes evidence; batch shapes can be correct while padding or causal masks are reversed; model outputs can be finite while label alignment is wrong. A useful checklist follows the request through every boundary instead of offering generic advice such as "check the shapes."

This exercise converts the chapter into an executable diagnostic framework. Each check needs an identifier, invariant, severity, observed evidence, and recommended action. Checks should fail loudly on a deliberately broken fixture. A checklist that consists only of prose depends on memory; an executable check can become a regression test.

Organize the framework into stages: raw text and normalization, tokenization, special tokens, window construction, truncation, packing, padding, positions, attention visibility, target shift, model context limit, CUDA memory, generation, and provenance. Include both preflight checks and post-output checks. Some failures should block execution, while others should emit metrics or warnings.

Fault injection is mandatory. Create fixtures with a tokenizer mismatch, off-by-one labels, removed evidence, reversed causal mask, cross-segment packed attention, incorrect position reset, and near-OOM context. Run the checklist and verify that each fault is detected by the intended check rather than an unrelated crash.

Part 2 - Mathematical form and reasoning

Represent invariant I_k as a boolean predicate over system state s:

$$I_k(s) \in \{0, 1\}.$$

Checklist coverage over injected faults F can be measured by

$$\text{Coverage} = \frac{|\{f \in F : \exists k,\ I_k(s_f) = 0\}|}{|F|}.$$

Also construct a detection matrix $D_{f,k}$ to identify checks that never fire, faults detected only incidentally, and overly broad checks that produce many false alarms.

Part 3 - Hints, controls, and evidence

1. Assign stable names such as `CTX-MASK-003` to every invariant.
2. Store expected, observed, severity, and remediation in structured results.
3. Separate hard failures from warnings and informational measurements.
4. Include one known-good fixture and require every blocking check to pass.
5. Inject one fault at a time before testing combinations.
6. Add the checklist to checkpoint evaluation and deployment regression tests, not only an interactive notebook.

Part 4 - Python code skeleton

```
class ContextCheckResult:
    def __init__(self, check_id, passed, severity,
                 expected, observed, remediation):
        self.check_id = check_id
        self.passed = passed
        self.severity = severity
        self.expected = expected
        self.observed = observed
        self.remediation = remediation

def check_length(token_ids, maximum_context):
    passed = len(token_ids) <= maximum_context
    return ContextCheckResult(
        "CTX-LENGTH-001", passed, "blocking",
        "token count <= maximum context", len(token_ids),
        "truncate with an audited policy or use a supported context"
    )

def check_next_token_alignment(input_ids, target_ids):
    passed = False
    if input_ids.shape == target_ids.shape:
        if input_ids.shape[1] > 1:
            passed = bool((input_ids[:, 1:] ==
                           target_ids[:, :-1]).all())
    return ContextCheckResult(
        "CTX-SHIFT-001", passed, "blocking",
        "input[:,1:] equals target[:,:-1]",
        "alignment inspected", "rebuild shifted targets"
    )

def E18(state):
    results = []
```

```
    results.append(check_length(
        state["token_ids"], state["maximum_context"]
    ))
    results.append(check_next_token_alignment(
        state["input_ids"], state["target_ids"]
    ))
    blocking_failure = False
    for result in results:
        if not result.passed and result.severity == "blocking":
            blocking_failure = True
    return results, blocking_failure
```

Part 5 - The Philosophy Behind the Exercise

A checklist is institutional memory. It preserves lessons after the person who discovered a failure has forgotten its details or left the project. Turning the lesson into an executable invariant converts painful experience into future protection.

But checklists are not substitutes for thought. They detect known classes of failure and can create false confidence when treated as complete. Their deepest value is cultural: they make assumptions visible, assign responsibility, and create a place where new knowledge can be added after every incident.

18.3.20 Exercise 19

Exercise Title: Write Practical Notes on Prompt Length Engineering

Part 1 - Why this matters

Prompt length engineering allocates a finite token budget among instructions, user content, retrieved evidence, examples, tool results, conversation history, and expected output. The maximum context is a hard upper bound, but a production system should operate below it to preserve output capacity, concurrency, memory, and recovery margin. Token budgeting must happen before the request reaches the model.

Every prompt region has a different priority. System and safety constraints may be mandatory. The latest user request may be essential. Retrieved passages can be ranked and shortened. Old conversation turns can be summarized, but summaries must preserve decisions, numbers, unresolved questions, and provenance. Few-shot examples can improve behavior while consuming tokens that might otherwise hold evidence.

The notes you produce must define an allocation policy rather than a collection of tips. Given maximum context C, reserve output budget G and safety margin M, leaving input budget $I = C - G - M$. Allocate mandatory regions first, then rank optional regions by expected value per token. Record every dropped or compressed region so failures can be reconstructed.

Test the policy under ordinary and adversarial conditions: an extremely long user document, many retrieved chunks, conflicting instructions, oversized tool output, multilingual text with high token expansion, and a request that needs a long answer. Measure token counts with the production tokenizer, not estimated words. The final notes should become a reusable prompt-budgeting standard.

Part 2 - Mathematical form and reasoning

The basic budget constraint is

$$C_{\text{system}} + C_{\text{history}} + C_{\text{user}} + C_{\text{retrieval}} + C_{\text{tools}} + G + M \leq C.$$

For optional region i with estimated utility u_i and cost c_i, selection resembles a constrained allocation problem

$$\max_{z_i \in \{0,1\}} \sum_i z_i u_i \quad \text{subject to} \quad \sum_i z_i c_i \leq I - C_{\text{mandatory}}.$$

Real utility is context-dependent, so heuristics require evaluation and fallback rather than being treated as an exact optimizer.

Part 3 - Hints, controls, and evidence

1. Define context, output, and safety budgets in configuration rather than scattered constants.
2. Tokenize each region separately and retain its start and end offsets in the assembled prompt.
3. Mark regions mandatory, compressible, droppable, or externalizable to retrieval.
4. Preserve a truncation manifest listing removed content and reason.
5. Reserve output tokens before filling input context; otherwise generation can terminate prematurely.
6. Evaluate answer quality, evidence retention, instruction retention, latency, cost, and truncation rate across realistic traffic distributions.

Part 4 - Python code skeleton

```
class PromptBudget:
    def __init__(self, maximum_context, output_reserve, safety_margin):
        self.maximum_context = maximum_context
        self.output_reserve = output_reserve
        self.safety_margin = safety_margin
        self.input_budget = maximum_context - output_reserve - safety_margin
        if self.input_budget <= 0:
            raise ValueError("no usable input budget remains")

def optional_region_priority(region):
    return region["priority"]

def assemble_prompt(tokenizer, mandatory_regions,
                    optional_regions, budget):
    selected = []
    dropped = []
    used_tokens = 0
    for region in mandatory_regions:
        token_ids = tokenizer.encode(region["text"])
```

```
        if used_tokens + len(token_ids) > budget.input_budget:
            raise ValueError("mandatory regions exceed input budget")
        selected.append((region, token_ids))
        used_tokens = used_tokens + len(token_ids)

    ordered_optional = sorted(optional_regions,
                              key=optional_region_priority,
                              reverse=True)
    for region in ordered_optional:
        token_ids = tokenizer.encode(region["text"])
        if used_tokens + len(token_ids) <= budget.input_budget:
            selected.append((region, token_ids))
            used_tokens = used_tokens + len(token_ids)
        else:
            dropped.append(region)
    return selected, dropped, used_tokens

def E19(tokenizer, mandatory_regions, optional_regions):
    budget = PromptBudget(4096, output_reserve=768,
                          safety_margin=128)
    return assemble_prompt(tokenizer, mandatory_regions,
                           optional_regions, budget)
```

Part 5 - The Philosophy Behind the Exercise

A prompt is an edited world presented to the model. Token budgeting determines which instructions, memories, and evidence exist inside that world. Length engineering is therefore not merely compression; it is the governance of attention under scarcity.

Good prompts do not maximize text. They preserve the smallest sufficient set of truths, constraints, and examples while leaving room for the model to respond. The final discipline is transparency: when the system must omit, summarize, or rank information, those decisions should remain visible enough to question and improve.

18.4 LLM02

Chapter Title: Pretraining Objective and Data
Specialization: Language Model Pretraining Engineering

18.4.1 Exercise 00

Exercise Title: Explain Next-Token Prediction Objective Mathematically

Part 1 - Why this matters

Next-token prediction is the central pretraining objective for a decoder-only LLM. Given a prefix, the model assigns a probability to every vocabulary token that could follow. The objective looks local because each term concerns one next token, but optimizing it across long and diverse sequences requires the model to capture grammar, discourse, facts, style, code structure, and dependencies that help reduce uncertainty.

The objective is teacher-forced during training. The complete correct sequence is available, and a causal mask allows every position to produce a prediction in one parallel forward pass without seeing future positions. At generation time, the correct future is unavailable and the model consumes its own selected tokens. The conditional rule is the same, while the distribution of prefixes can change because generation errors feed back into later steps.

You will derive the loss from joint-probability factorization and compute it by hand for a tiny vocabulary. Every symbol must be connected to a tensor axis. Then compare sequence sum, token mean, batch mean, and perplexity. Reduction choice matters when batches contain different numbers of padding or ignored tokens; averaging batch means can weight short sequences incorrectly.

The exercise must also clarify what the objective does not say. It does not label statements true or false, distinguish authorized from unauthorized data, or directly reward helpfulness and abstention. Those behaviors can correlate with text patterns or be added through later training, but the mathematical target remains the observed next token.

Part 2 - Mathematical form and reasoning

For token sequence $x_{1:T}$, the chain rule gives

$$p_\theta(x_{1:T}) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t}).$$

Maximizing likelihood is equivalent to minimizing negative log-likelihood:

$$\mathcal{L}_{\text{NLL}} = -\frac{1}{N_{\text{valid}}} \sum_{b=1}^{B} \sum_{t=1}^{T-1} m_{b,t} \log p_\theta(x_{b,t+1} \mid x_{b,\le t}).$$

For logits $z_{b,t,v}$,

$$\log p_{b,t,y} = z_{b,t,y} - \log \sum_{v=1}^{V} e^{z_{b,t,v}}.$$

Perplexity is $\exp(\mathcal{L}_{\text{NLL}})$ when the loss uses natural logarithms and a consistent tokenization.

Part 3 - Hints, controls, and evidence

1. Use a vocabulary of four tokens and write logits for two prediction positions whose probabilities can be calculated manually.
2. Apply log-sum-exp stabilization by subtracting the maximum logit.
3. Show that the probabilities at each position sum to one.
4. Compare total NLL, mean token NLL, and perplexity without mixing their units.
5. Add padding and prove that masked token averaging matches the manual valid terms.
6. Explain why perplexities from different tokenizers are not directly comparable without accounting for token granularity.

Part 4 - Python code skeleton

```
import torch

def stable_log_softmax_row(logits):
    maximum = logits.max()
    shifted = logits - maximum
    log_denominator = torch.log(torch.exp(shifted).sum())
    return shifted - log_denominator

def manual_token_nll(logits, target_id):
    log_probabilities = stable_log_softmax_row(logits)
    return -log_probabilities[target_id]

def masked_sequence_nll(logits, target_ids, valid_mask):
    losses = torch.empty_like(target_ids, dtype=logits.dtype)
    for batch_index in range(logits.shape[0]):
        for time_index in range(logits.shape[1]):
            target_id = target_ids[batch_index, time_index]
            losses[batch_index, time_index] = manual_token_nll(
                logits[batch_index, time_index], target_id
            )
    selected = losses.masked_select(valid_mask)
    return selected.mean(), losses

def E00(device):
    logits = torch.tensor(
        [[[2.0, 0.0, -1.0, 1.0],
          [0.0, 3.0, 1.0, -2.0]]], device=device
    )
    targets = torch.tensor([[0, 1]], device=device)
    valid_mask = torch.tensor([[True, True]], device=device)
    mean_nll, token_losses = masked_sequence_nll(
        logits, targets, valid_mask
    )
    perplexity = torch.exp(mean_nll)
    assert torch.isfinite(mean_nll)
    return mean_nll, perplexity, token_losses
```

Part 5 - The Philosophy Behind the Exercise

A next-token objective turns language into a sequence of wagers. At every position the model distributes belief across possible continuations and is penalized by the surprise of what actually occurred. Across enough text, reducing surprise demands increasingly rich representations of regularity.

Yet likelihood follows the record it is given. It cannot independently decide whether the record is just, current, or true. The objective is powerful because prediction compresses structure; it is limited because statistical fit is not a complete theory of knowledge or responsibility.

18.4.2 Exercise 01

Exercise Title: Implement CrossEntropy Loss for LLM Training Correctly

Part 1 - Why this matters

PyTorch cross entropy combines log-softmax and negative log-likelihood in a numerically stable operation. Correct use still requires semantic preparation: logits must represent the current positions, targets must be the next tokens, the vocabulary axis must be last before flattening, targets must be integer identifiers, and padding or ignored positions must not influence the reduction.

The most common implementation produces logits of shape (B, T, V) and targets of shape (B, T). Some APIs expect class dimension second, but flattening to (BT, V) and (BT) avoids accidental interpretation of sequence length as the class axis. Flattening is safe only after alignment has been proved. Matching element counts cannot repair an incorrect shift.

You will implement loss with three reduction methods: valid-token mean, sequence mean followed by batch mean, and raw sum. On variable-length examples these are different objectives. Token mean weights each real token equally; sequence mean weights short and long examples equally. Choose deliberately and report valid token count so loss from different batches remains interpretable.

Test extreme logits, all-padding batches, non-contiguous tensors, and float16 inputs. The loss should fail clearly when no valid target exists. Mixed precision training may compute model operations in reduced precision while keeping stable loss and scaling behavior; do not manually softmax probabilities before passing them to cross entropy.

Part 2 - Mathematical form and reasoning

For valid target y_i and logits $z_i \in \mathbb{R}^V$,

$$\ell_i = -z_{i,y_i} + \log \sum_{v=1}^{V} e^{z_{i,v}}.$$

Token-mean reduction is

$$L_{\text{token}} = \frac{\sum_i m_i \ell_i}{\sum_i m_i}.$$

Sequence-balanced reduction is

$$L_{\text{sequence}} = \frac{1}{B} \sum_{b=1}^{B} \frac{\sum_t m_{b,t} \ell_{b,t}}{\sum_t m_{b,t}}.$$

The denominators define the statistical weighting and must be included in any gradient-accumulation equivalence test.

Part 3 - Hints, controls, and evidence

1. Print one decoded input–target alignment before flattening.
2. Assert target range is $[0, V-1]$ for all non-ignored positions.
3. Compare your masked implementation with PyTorch cross entropy using its ignored-target identifier argument.

4. Add a batch containing sequence lengths two and ten to expose reduction differences.
5. Test logits shifted by a large constant and verify loss is unchanged within tolerance.
6. Run backward and confirm every trainable path receives finite gradients.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def next_token_cross_entropy(logits, input_ids, padding_id):
    prediction_logits = logits[:, :-1, :]
    target_ids = input_ids[:, 1:]
    valid_mask = target_ids != padding_id
    if not valid_mask.any():
        raise ValueError("batch has no valid next-token targets")
    flat_logits = prediction_logits.reshape(
        -1, prediction_logits.shape[-1]
    )
    flat_targets = target_ids.reshape(-1)
    flat_losses = F.cross_entropy(
        flat_logits, flat_targets, reduction="none"
    )
    losses = flat_losses.view_as(target_ids)
    selected = losses.masked_select(valid_mask)
    return selected.mean(), selected.numel(), losses

def validate_target_range(target_ids, valid_mask, vocabulary_size):
    selected = target_ids.masked_select(valid_mask)
    assert selected.min() >= 0
    assert selected.max() < vocabulary_size

def E01(model, input_ids, padding_id):
    logits = model(input_ids)
    loss, valid_tokens, token_losses = next_token_cross_entropy(
        logits, input_ids, padding_id
    )
    loss.backward()
    assert torch.isfinite(loss)
    return loss.detach(), valid_tokens, token_losses.detach()
```

Part 5 - The Philosophy Behind the Exercise

Loss is the place where a model's world becomes judgment. Logits describe possibilities; targets identify what happened; reduction decides whose errors count how much. A tiny indexing or weighting choice silently changes the lesson the model receives.

The practical wisdom is to treat loss code as a specification, not plumbing. Every slice, mask, and denominator carries a claim about prediction and fairness between examples. Correct optimization begins with making those claims explicit.

18.4.3 Exercise 02

Exercise Title: Analyze Why LLMs Learn Syntax Before Semantics

Part 1 - Why this matters

During early language-model training, local structural regularities often become measurable before robust semantic behavior. Word order, punctuation, bracket closure, agreement, and common token transitions provide frequent and relatively consistent signals. Semantic relations can be rarer, distributed across longer contexts, ambiguous, and dependent on world knowledge. This is an empirical tendency, not a universal developmental law.

The conclusion depends on how syntax and semantics are measured. A syntax probe with easy frequent examples and a semantic probe with rare compositional examples creates an unfair race. You will build matched synthetic tasks in which surface grammar and latent meaning can be varied independently, then save model checkpoints throughout training. Probe both behaviors at equal example counts and report when each rises above a predefined threshold.

Examples should include grammatical nonsense, meaningful but grammatically corrupted sequences, unseen lexical substitutions, and compositional semantic relationships. Next-token loss alone cannot tell whether improvement came from memorizing frequent templates or representing an abstract rule. Generalization tests must change vocabulary while preserving the rule.

The analysis should examine gradient exposure. Count how often each syntactic pattern and semantic relation appears, how many tokens separate dependent elements, and how much label noise affects them. If syntax is learned first, connect the observation to frequency and dependency structure rather than claiming that the model experiences human cognitive stages.

Part 2 - Mathematical form and reasoning

Let $A_s(k)$ and $A_m(k)$ be syntax and semantic probe accuracy at optimization step k. Define threshold times

$$k_s^* = \min\{k : A_s(k) \geq \tau_s\}, \qquad k_m^* = \min\{k : A_m(k) \geq \tau_m\}.$$

Learning order is meaningful only with confidence intervals across seeds and matched baselines. Also track conditional loss by pattern category

$$L_c = \frac{1}{N_c} \sum_{i:y_i=c} \ell_i$$

so global token frequency does not hide rare semantic behavior.

Part 3 - Hints, controls, and evidence

1. Define syntax and semantic success operationally before training.
2. Match probe size, chance level, and lexical frequency as closely as possible.
3. Save checkpoints frequently during the first part of training.
4. Evaluate unseen word substitutions to separate templates from rules.
5. Repeat across seeds and bootstrap examples for uncertainty.

6. State whether the result supports earlier measurement, faster learning, or stronger final performance; these are different claims.

Part 4 - Python code skeleton

```
import torch

def category_accuracy(predictions, targets, categories):
    totals = {}
    correct = {}
    for index in range(targets.numel()):
        category = int(categories[index].item())
        if category not in totals:
            totals[category] = 0
            correct[category] = 0
        totals[category] = totals[category] + 1
        if predictions[index] == targets[index]:
            correct[category] = correct[category] + 1
    accuracies = {}
    for category in totals:
        accuracies[category] = correct[category] / totals[category]
    return accuracies

def probe_model(model, batches, device):
    model.eval()
    outputs = []
    with torch.no_grad():
        for batch in batches:
            logits = model(batch["input_ids"].to(device))
            selected = logits[batch["batch_index"],
                              batch["time_index"]]
            predictions = selected.argmax(dim=-1).cpu()
            outputs.append(category_accuracy(
                predictions, batch["targets"], batch["categories"]
            ))
    return outputs

def E02(checkpoints, syntax_batches, semantic_batches, device):
    trajectory = []
    for step, model in checkpoints:
        record = {}
        record["step"] = step
        record["syntax"] = probe_model(model, syntax_batches, device)
        record["semantics"] = probe_model(model, semantic_batches, device)
        trajectory.append(record)
    return trajectory
```

Part 5 - The Philosophy Behind the Exercise

Structure is often easier to observe than meaning because structure repeats in visible form. Semantics is relational: the same words can mean different things, and the same meaning can wear different words. Statistical learning encounters these signals with unequal clarity.

But "before" should not become "instead of." Syntax and semantics interact, and our probes divide them for analysis more sharply than language itself does. The exercise teaches humility about developmental stories: curves reveal when a test changed, not a complete inner history of understanding.

18.4.4 Exercise 03

Exercise Title: Build Tiny Text Corpus and Overfit LLM as Sanity Check

Part 1 - Why this matters

Before training on millions of tokens, a language-model pipeline should overfit a tiny corpus. A sufficiently capable model exposed repeatedly to a few short sequences should drive training loss very low and reproduce the learned continuations. If it cannot, the likely causes include target misalignment, broken causal masks, incorrect optimizer order, frozen parameters, excessive regularization, insufficient capacity, or tokenizer inconsistency.

Overfitting is usually undesirable for generalization, but here it is a diagnostic instrument. The target is not a useful language model. The target is evidence that data, model, loss, gradients, and generation form one coherent system. Keep the corpus human-readable so every expected transition can be inspected.

You will create a corpus with repeated deterministic patterns and one ambiguous prefix that has two valid continuations. The deterministic transitions should approach probability one; the ambiguous prefix should retain probability mass for both observed successors according to their empirical frequency. This shows that low loss does not require choosing one token when the data itself is uncertain.

After memorization, test exact training prompts, shorter prefixes, corrupted prefixes, and unseen recombinations. The model may memorize without generalizing. That is acceptable for the sanity check and becomes evidence for the later memorization analysis. Save the tiny corpus and configuration as a permanent regression fixture.

Part 2 - Mathematical form and reasoning

For empirical transition counts $n(x, y)$ after prefix state x, the maximum-likelihood categorical distribution is

$$\hat{p}(y \mid x) = \frac{n(x, y)}{\sum_{y'} n(x, y')}.$$

The irreducible empirical entropy at an ambiguous prefix is

$$H(\hat{p}) = -\sum_{y} \hat{p}(y \mid x) \log \hat{p}(y \mid x).$$

Therefore training loss need not reach zero when identical visible prefixes have different targets. Compare the observed final loss with this lower-bound intuition.

Part 3 - Hints, controls, and evidence

1. Use fewer than one hundred tokens and print every encoded sequence.

2. Disable dropout and weight decay for the strict memorization test.
3. Verify at least one parameter changes after the first optimizer step.
4. Log loss, gradient norm, deterministic transition probabilities, and generated output.
5. Stop only after a declared loss or accuracy criterion, not after an arbitrary number of epochs.
6. Add this overfit test to regression checks after model or data-pipeline refactors.

Part 4 - Python code skeleton

```
import copy
import torch
import torch.nn.functional as F

def one_training_step(model, input_ids, optimizer):
    optimizer.zero_grad(set_to_none=True)
    logits = model(input_ids)
    prediction_logits = logits[:, :-1, :]
    targets = input_ids[:, 1:]
    loss = F.cross_entropy(
        prediction_logits.reshape(-1, prediction_logits.shape[-1]),
        targets.reshape(-1)
    )
    loss.backward()
    gradient_norm = torch.nn.utils.clip_grad_norm_(
        model.parameters(), 10.0
    )
    optimizer.step()
    return loss.detach(), gradient_norm.detach()

def E03(model, tiny_batch, optimizer, device, maximum_steps):
    model.to(device)
    tiny_batch = tiny_batch.to(device)
    initial_state = copy.deepcopy(model.state_dict())
    records = []
    for step in range(maximum_steps):
        model.train()
        loss, gradient_norm = one_training_step(
            model, tiny_batch, optimizer
        )
        records.append({"step": step, "loss": float(loss.cpu()),
                        "gradient_norm": float(gradient_norm.cpu())})
        if loss.item() < 0.02:
            break
    changed = False
    for name, tensor in model.state_dict().items():
```

```
        if not torch.equal(tensor.cpu(), initial_state[name].cpu()):
            changed = True
            break
    assert changed
    return records
```

Part 5 - The Philosophy Behind the Exercise

Overfitting a tiny corpus is a deliberate reduction of the world. By removing diversity and uncertainty, we ask a simpler question: can this pipeline learn at all? The test is powerful because failure cannot be excused by a difficult generalization problem.

Sanity checks are acts of humility. They refuse to let scale hide confusion. Before claiming emergent capability, the researcher proves that one small batch moves through the system exactly as intended.

18.4.5 Exercise 04

Exercise Title: Explain Curriculum Learning Effect in Large-Scale Pretraining

Part 1 - Why this matters

Curriculum learning changes the order or mixture in which training examples are presented. A simple-to-complex curriculum may stabilize early optimization, while a quality-first curriculum can establish clean language patterns before introducing noisier web data. Anti-curricula and fully shuffled mixtures provide controls. The same examples and compute can produce different trajectories because optimization is path-dependent.

"Difficulty" must be operationally defined. Document length, rarity, perplexity under a reference model, syntactic depth, noise score, and domain specialization measure different properties. Sorting by one can accidentally sort by language, source, or demographic group. A curriculum can improve the mean while delaying or excluding important coverage.

You will assign each example a transparent difficulty or quality field and compare shuffled, easy-to-hard, hard-to-easy, and staged mixtures. Match total valid tokens, optimizer steps, and final data exposure. Track early convergence, final validation loss, domain metrics, gradient norms, and forgetting when the mixture changes.

Large-scale pretraining rarely uses a single permanent ordering. Data mixtures can change over phases, and sampling weights can depend on domain size or quality. The experiment should therefore log the probability of drawing each category at every stage. A dataset list alone is insufficient to reproduce the training distribution.

Part 2 - Mathematical form and reasoning

Standard empirical risk uses a fixed data distribution P. A curriculum uses a time-dependent distribution P_k:

$$\theta_{k+1} = \theta_k - \eta_k \mathbb{E}_{x \sim P_k} \left[\nabla_\theta \ell(x; \theta_k) \right].$$

Although the final union of examples may be identical, different P_k changes the optimization path. Report exposure for category c as

$$E_c = \sum_k N_k P_k(c)$$

and verify matched totals when claiming an ordering-only comparison.

Part 3 - Hints, controls, and evidence

1. Define difficulty before viewing experimental outcomes.
2. Match initial weights, valid-token budget, learning-rate schedule, and final category exposure.
3. Log validation metrics by category throughout training, not only at the end.
4. Test multiple seeds because order effects can be noisy.
5. Inspect whether curriculum scoring correlates with language or source.
6. Separate faster early learning from better final quality and reduced training instability.

Part 4 - Python code skeleton

```
import random

def order_examples(examples, strategy, seed):
    ordered = list(examples)
    if strategy == "shuffled":
        generator = random.Random(seed)
        generator.shuffle(ordered)
    elif strategy == "easy_to_hard":
        ordered.sort(key=difficulty_value)
    elif strategy == "hard_to_easy":
        ordered.sort(key=difficulty_value, reverse=True)
    else:
        raise ValueError("unknown curriculum strategy")
    return ordered

def difficulty_value(example):
    return example["difficulty"]

def E04(train_function, evaluate_function, model_factory,
        examples, strategies, seeds):
    records = []
    for strategy in strategies:
        for seed in seeds:
            ordered = order_examples(examples, strategy, seed)
            model = model_factory(seed)
            trajectory = train_function(model, ordered)
            metrics = evaluate_function(model)
            record = {}
            record["strategy"] = strategy
            record["seed"] = seed
```

```
            record["trajectory"] = trajectory
            record["metrics"] = metrics
            records.append(record)
    return records
```

Part 5 - The Philosophy Behind the Exercise

Learning is shaped not only by what is experienced but by when it is experienced. Early examples alter the model that later examples encounter. Curriculum makes this temporal structure visible instead of pretending that a dataset is an unordered substance.

But deciding what is "easy" or "high quality" embeds judgment into the training path. A curriculum is therefore both an optimization strategy and an editorial theory of which language deserves to teach first. Its gains must be balanced against the voices and difficulties it may postpone.

18.4.6 Exercise 05

Exercise Title: Compare Clean Data vs Noisy Data and Observe Hallucination Effects

Part 1 - Why this matters

Noisy pretraining data can contain corrupted text, contradictory facts, fabricated citations, duplicated errors, broken markup, and mismatched question–answer pairs. A language model optimizes the observed tokens and does not receive an automatic marker distinguishing reliable statements from noise. Repeated or contextually predictable false claims can therefore become learned continuations.

To isolate the effect, create clean and noisy corpora with matched size and token frequency where possible. Initialize two models identically and expose them to equal valid-token budgets. The noisy corpus should include labeled noise types rather than a single random corruption rate. Random character damage, plausible false facts, contradictions, and invented references create different learning signals and should be evaluated separately.

Hallucination must be operationalized as generation of claims unsupported by the evidence. Use a separate prompt set with answerable and unanswerable cases, then measure factual accuracy, unsupported claim rate, abstention, calibration, and training loss. Noisy training can sometimes act as regularization, so a result should not be assumed in advance; inspect which noise types harm which behavior.

Cleaning also has risks. Filters may remove minority languages, informal but valid writing, or difficult domains. Report retention by source and category. The target is not text that looks polished to one heuristic, but a traceable corpus whose reliability and coverage serve the intended model.

Part 2 - Mathematical form and reasoning

Let the training distribution be

$$P_\epsilon = (1-\epsilon)P_{\text{clean}} + epsilonP_{\text{noise}}.$$

The expected gradient is the same mixture of component gradients. For response claims c_i and evidence E, unsupported rate is

$$H = \frac{1}{M}\sum_{i=1}^{M} \mathbb{I}[c_i \text{ is not supported by } E].$$

Compare H across noise levels while also reporting useful answer rate, because refusing all questions trivially reduces unsupported claims.

Part 3 - Hints, controls, and evidence

1. Preserve identical initialization and total valid-token exposure.
2. Create separate corruption functions for characters, labels, facts, citations, and contradictions.
3. Keep an untouched validation set with explicit evidence.
4. Annotate generated claims rather than scoring only complete responses.
5. Evaluate filter retention by language, domain, and source.
6. Publish raw examples where clean and noisy models differ, including cases where the noisy model unexpectedly wins.

Part 4 - Python code skeleton

```
import copy
import random

def corrupt_labels(records, corruption_probability, seed):
    generator = random.Random(seed)
    corrupted = copy.deepcopy(records)
    for record in corrupted:
        draw = generator.random()
        if draw < corruption_probability:
            original = record["answer"]
            record["answer"] = record["wrong_answer"]
            record["noise_type"] = "plausible_false_answer"
            record["original_answer"] = original
        else:
            record["noise_type"] = "clean"
    return corrupted

def unsupported_rate(claim_records):
    unsupported = 0
    total = 0
    for record in claim_records:
        total = total + 1
        if not record["supported"]:
            unsupported = unsupported + 1
    return unsupported / max(1, total)

def E05(model_factory, train_function, evaluate_function,
        clean_records, noise_levels):
```

```
    results = []
    for noise_level in noise_levels:
        training_records = corrupt_labels(
            clean_records, noise_level, seed=17
        )
        model = model_factory(seed=29)
        train_function(model, training_records)
        evaluation = evaluate_function(model)
        results.append({"noise_level": noise_level,
                        "evaluation": evaluation})
    return results
```

Part 5 - The Philosophy Behind the Exercise

A model cannot rise above its evidence by intention. It can infer, combine, and generalize, but the patterns presented as ordinary language become the material from which its expectations are formed. Noise is not simply dirt around the signal; sometimes it is a competing signal.

Data cleaning is therefore an epistemic responsibility. It decides which claims receive repetition and which sources receive trust. The goal is not artificial purity, but honest lineage, measured reliability, and enough diversity that cleaning does not become erasure.

18.4.7 Exercise 06

Exercise Title: Explain Why Data Diversity Beats Domain Specialization Early

Part 1 - Why this matters

Early pretraining constructs general representations of language, structure, and common reasoning patterns. A diverse mixture exposes the model to multiple genres, domains, languages, formats, and problem types. Narrow specialization can produce rapid gains on one domain but leave the model without transferable features and vulnerable to overfitting repeated templates.

The title is a hypothesis with conditions, not a universal rule. A model built only for a closed domain may benefit from specialized data, and low-quality diversity can be worse than excellent focused text. The experiment asks whether diversity during the early phase creates a stronger base for later adaptation under matched compute and data quality.

Train one model first on a balanced multi-domain mixture and another on a single domain. Then fine-tune both on the target domain using the same budget. Evaluate target quality, out-of-domain retention, sample efficiency, calibration, and robustness to reformulated inputs. Also compare a mixture that oversamples tiny domains so diversity does not merely mean proportional sampling from the largest source.

Data diversity must be measured. Counting sources is insufficient if most tokens come from one domain or if documents are duplicates. Report token share, effective sample size, unique documents, language coverage, and per-domain loss. The final recommendation should describe when specialization begins and what general capabilities must be protected.

Part 2 - Mathematical form and reasoning

For domains $d = 1, \ldots, D$ with sampling weights w_d,

$$P_{\text{mix}}(x) = \sum_{d=1}^{D} w_d P_d(x), \qquad \sum_d w_d = 1.$$

A simple diversity index is

$$H(w) = -\sum_d w_d \log w_d,$$

but it measures balance rather than semantic diversity. Evaluate transfer gain

$$G = A_{\text{target}}^{\text{diverse}\to\text{target}} - A_{\text{target}}^{\text{specialized}\to\text{target}}$$

across equal pretraining and adaptation tokens.

Part 3 - Hints, controls, and evidence

1. Match data quality thresholds and total token budgets across mixtures.
2. Use identical initial states and optimization schedules.
3. Evaluate every domain before and after target adaptation.
4. Include unseen formats within the target domain to test transfer rather than template memorization.
5. Report sampling weights and actual observed token counts.
6. Inspect negative transfer: diversity can introduce interference as well as useful shared structure.

Part 4 - Python code skeleton

```
import random

def sample_domain(domain_names, weights, generator):
    draw = generator.random()
    cumulative = 0.0
    for index in range(len(domain_names)):
        cumulative = cumulative + weights[index]
        if draw <= cumulative:
            return domain_names[index]
    return domain_names[-1]

def build_mixed_stream(domain_data, weights, number_of_examples, seed):
    generator = random.Random(seed)
    domain_names = list(domain_data.keys())
    positions = {}
    for name in domain_names:
        positions[name] = 0
    stream = []
    for _ in range(number_of_examples):
```

```
        name = sample_domain(domain_names, weights, generator)
        examples = domain_data[name]
        position = positions[name] % len(examples)
        stream.append({"domain": name, "example": examples[position]})
        positions[name] = positions[name] + 1
    return stream

def E06(train_and_evaluate, domain_data, diverse_weights,
        specialized_weights):
    diverse_stream = build_mixed_stream(
        domain_data, diverse_weights, 10000, seed=11
    )
    specialized_stream = build_mixed_stream(
        domain_data, specialized_weights, 10000, seed=11
    )
    diverse_result = train_and_evaluate(diverse_stream, seed=23)
    specialized_result = train_and_evaluate(
        specialized_stream, seed=23
    )
    return diverse_result, specialized_result
```

Part 5 - The Philosophy Behind the Exercise

Diversity gives learning more ways to discover what is shared. A structure first encountered in prose may support code; a relation learned in one domain may become a tool in another. Generality grows when representations must serve more than one narrow habit.

Specialization remains valuable, but timing matters. A broad foundation can be focused later; a narrow foundation may never encounter the concepts needed to expand. The principle is not that every source is equally useful, but that early learning should preserve the possibility of transfer.

18.4.8 Exercise 07

Exercise Title: Measure Loss vs Token Frequency Distribution

Part 1 - Why this matters

Language token frequencies are highly uneven. A small set of tokens appears constantly while many tokens are rare. Average cross entropy is dominated by frequent positions, so a model can improve global loss while rare names, symbols, scripts, or technical fragments remain poorly predicted. Frequency- conditioned analysis exposes this hidden distribution.

Token frequency alone does not determine difficulty. A rare token may be easy in a fixed phrase; a frequent token may be ambiguous across many contexts. Tokens also depend on tokenizer design: a rare word can become several common subwords, while an unusual Unicode sequence may fragment into rare bytes. Measure token frequency and contextual loss without treating either as a complete causal explanation.

You will count training occurrences, assign validation targets to logarithmic frequency bins, and compute mean NLL, accuracy, entropy, and example count for each bin. Add bootstrap intervals and display individual high-loss tokens. Use training counts only; computing frequency from validation leaks evaluation distribution into the analysis.

The experiment should compare early and late checkpoints. Frequent tokens may improve rapidly while rare-token loss changes slowly or irregularly. Also separate unseen tokens or fragments from merely rare ones. The vocabulary guarantees an identifier exists, not that pretraining provided enough examples to learn its use.

Part 2 - Mathematical form and reasoning

For token v, training frequency is

$$f_v = \sum_{i \in \text{train}} \mathbb{I}[y_i = v].$$

For frequency bin B_k, conditional loss is

$$L_k = \frac{1}{|I_k|} \sum_{i \in I_k} -\log p_\theta(y_i \mid x_i), \qquad I_k = \{i : f_{y_i} \in B_k\}.$$

Report $|I_k|$ because rare bins have higher variance. Correlation between $\log(f_v + 1)$ and token loss can summarize trend but cannot replace binned inspection.

Part 3 - Hints, controls, and evidence

1. Count only valid non-padding targets from the training set.
2. Use bins such as zero, one to nine, ten to ninety-nine, and larger ranges.
3. Accumulate loss sums and counts on tensors without storing every logit.
4. Decode representative high- and low-loss identifiers in every bin.
5. Separate punctuation, numbers, languages, and domain-specific tokens.
6. Repeat for multiple checkpoints and seeds to distinguish learning trend from sampling noise.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def frequency_bin(frequency):
    if frequency == 0:
        return 0
    if frequency < 10:
        return 1
    if frequency < 100:
        return 2
    if frequency < 1000:
        return 3
    return 4

def losses_by_frequency(logits, targets, training_counts):
```

```
    losses = F.cross_entropy(
        logits.reshape(-1, logits.shape[-1]),
        targets.reshape(-1), reduction="none"
    )
    flat_targets = targets.reshape(-1)
    sums = torch.zeros(5, device=logits.device, dtype=torch.float64)
    counts = torch.zeros(5, device=logits.device, dtype=torch.long)
    for index in range(flat_targets.numel()):
        token_id = int(flat_targets[index].item())
        frequency = int(training_counts[token_id].item())
        bin_id = frequency_bin(frequency)
        sums[bin_id] = sums[bin_id] + losses[index].to(torch.float64)
        counts[bin_id] = counts[bin_id] + 1
    means = sums / counts.clamp_min(1)
    return means, counts

def E07(model, validation_ids, training_counts):
    logits = model(validation_ids[:, :-1])
    targets = validation_ids[:, 1:]
    return losses_by_frequency(logits, targets, training_counts)
```

Part 5 - The Philosophy Behind the Exercise

An average gives every event a contribution but does not give every kind of event visibility. Frequent language defines the dominant gradient and the headline metric; rare language can remain a quiet frontier of failure.

Frequency analysis asks whose errors disappear inside the mean. It teaches that statistical success and equitable competence are not identical. A model of language should be judged not only where language is abundant, but where its evidence is scarce and its users may already be underrepresented.

18.4.9 Exercise 08

Exercise Title: Explain Memorization vs Generalization in LLMs

Part 1 - Why this matters

Memorization means reproducing information strongly tied to particular training examples; generalization means applying learned regularities to unseen examples. The two are not mutually exclusive. A model can memorize rare strings while generalizing grammar, and some factual recall may reasonably depend on repeated exposure. The scientific task is to measure behavior rather than assign one global label to the model.

Exact reproduction is strong evidence of memorization when the sequence is long, rare, and unlikely under ordinary language. Short common phrases are weak evidence because many sources share them. Generalization tests must prevent near-duplicate contamination and change surface form while preserving an underlying rule.

You will train a tiny model on synthetic templates containing unique canary strings plus compositional rules. Test exact canary extraction, paraphrased facts, held-out combinations, and completely unseen identifiers. Vary canary repetition and measure exposure: how much

more likely the canary becomes after training compared with a reference model or neighboring strings.

Privacy implications must be explicit. Do not insert real secrets, personal data, credentials, or copyrighted passages as canaries. Use generated strings with known provenance. The goal is to understand extraction risk and design audits, not to reproduce sensitive material.

Part 2 - Mathematical form and reasoning

For canary sequence c and reference set R, exposure can be approximated from rank:

$$\text{Exposure}(c) = \log_2 |R| - \log_2 \text{rank}(c),$$

where sequences are ranked by model loss. Generalization gap is

$$G = L_{\text{heldout}} - L_{\text{train}}.$$

Neither value alone is sufficient: low held-out loss can coexist with memorized rare examples, and high gap can result from distribution shift rather than only memorization.

Part 3 - Hints, controls, and evidence

1. Generate canaries randomly and record only synthetic provenance.
2. Compare repetition counts while holding total token budget fixed.
3. Evaluate exact prefixes, paraphrases, and rule-preserving novel combinations.
4. Search training and test corpora for exact and near-duplicate overlap.
5. Report extraction probability across sampling settings, not one lucky generation.
6. Separate privacy-risk memorization from useful learning of common structure in the conclusion.

Part 4 - Python code skeleton

```
import math
import torch
import torch.nn.functional as F

def sequence_nll(model, input_ids):
    logits = model(input_ids[:, :-1])
    targets = input_ids[:, 1:]
    loss = F.cross_entropy(
        logits.reshape(-1, logits.shape[-1]),
        targets.reshape(-1), reduction="sum"
    )
    return float(loss.detach().cpu())

def rank_canary(model, canary_ids, reference_sequences):
    canary_loss = sequence_nll(model, canary_ids)
    better_or_equal = 1
```

```
    for sequence in reference_sequences:
        reference_loss = sequence_nll(model, sequence)
        if reference_loss <= canary_loss:
            better_or_equal = better_or_equal + 1
    return better_or_equal, canary_loss

def E08(model, canary_ids, reference_sequences):
    rank, canary_loss = rank_canary(
        model, canary_ids, reference_sequences
    )
    population = len(reference_sequences) + 1
    exposure = math.log2(population) - math.log2(rank)
    return {"rank": rank, "loss": canary_loss,
            "exposure": exposure}
```

Part 5 - The Philosophy Behind the Exercise

Learning always preserves something from experience. The difficult question is whether the preserved structure travels beyond the example or remains attached to its surface. Memorization and generalization are not moral opposites; they are different relationships between past evidence and future behavior.

The responsibility arises when memorized detail was never meant to become extractable. Scale transforms data stewardship into model behavior. Protecting privacy begins before training and continues through auditing what the model can be induced to reveal.

18.4.10 Exercise 09

Exercise Title: Detect Data Leakage Using Train-Test Overlap Experiments

Part 1 - Why this matters

Evaluation is intended to estimate behavior on unseen data. If test documents, answers, or close paraphrases appear in training, scores can reflect retrieval or memorization instead of generalization. Large corpora make accidental overlap likely, especially when benchmarks are publicly posted, quoted, translated, or embedded in tutorials.

Exact hashing detects identical normalized documents but misses formatting changes and partial overlap. Token shingles, MinHash-style signatures, locality- sensitive search, and embedding similarity can detect near duplicates at different costs and false-positive rates. Detection should operate at document, window, and answer-string levels because leakage can be partial.

You will build clean, exactly contaminated, and paraphrase-contaminated training variants. Train matched tiny models and compare test loss, exact accuracy, and performance on a hidden clean control set. If the contaminated score rises only on leaked items, the apparent gain is invalid. Remove overlap and confirm the score returns toward the clean baseline.

Temporal provenance is another defense. Evaluation events created after the training cutoff are less likely to be included, but publication time and crawl time must be recorded. A leakage report should preserve matched examples and similarity evidence without exposing protected benchmark content unnecessarily.

Part 2 - Mathematical form and reasoning

For token-shingle sets $S(a)$ and $S(b)$, Jaccard similarity is

$$J(a,b) = \frac{|S(a) \cap S(b)|}{|S(a) \cup S(b)|}.$$

At threshold τ, estimated contamination rate is

$$r_{\text{contam}} = \frac{1}{|D_{\text{test}}|} \sum_{t \in D_{\text{test}}} \mathbb{I}\left[\max_{x \in D_{\text{train}}} J(x,t) \geq \tau\right].$$

Threshold choice trades missed paraphrases against false matches and must be validated on labeled pairs.

Part 3 - Hints, controls, and evidence

1. Normalize Unicode, whitespace, and boilerplate while retaining raw text.
2. Compute exact hashes before more expensive near-duplicate comparisons.
3. Use token shingles of several widths to detect local and broad overlap.
4. Label a sample of matches manually and report precision and recall.
5. Evaluate leaked, non-leaked, and hidden-control items separately.
6. Version the detector, threshold, corpus snapshot, and remediation action.

Part 4 - Python code skeleton

```
def token_shingles(token_ids, width):
    shingles = set()
    if len(token_ids) < width:
        shingles.add(tuple(token_ids))
        return shingles
    for start in range(len(token_ids) - width + 1):
        shingles.add(tuple(token_ids[start:start + width]))
    return shingles

def jaccard_similarity(set_a, set_b):
    union = set_a | set_b
    if len(union) == 0:
        return 1.0
    intersection = set_a & set_b
    return len(intersection) / len(union)

def find_overlaps(training_sequences, test_sequences, width, threshold):
    training_shingles = []
    for sequence in training_sequences:
        training_shingles.append(token_shingles(sequence, width))
    matches = []
    for test_index in range(len(test_sequences)):
        test_set = token_shingles(test_sequences[test_index], width)
```

```
        best_score = 0.0
        best_train_index = -1
        for train_index in range(len(training_shingles)):
            score = jaccard_similarity(
                training_shingles[train_index], test_set
            )
            if score > best_score:
                best_score = score
                best_train_index = train_index
        if best_score >= threshold:
            matches.append((test_index, best_train_index, best_score))
    return matches

def E09(training_sequences, test_sequences):
    return find_overlaps(training_sequences, test_sequences,
                         width=5, threshold=0.8)
```

Part 5 - The Philosophy Behind the Exercise

An examination ceases to measure learning when the answers were already inside the lesson. Data leakage creates the appearance of progress by dissolving the boundary between experience and evaluation.

At scale, innocence is not enough; overlap can occur without anyone intending to cheat. Scientific credibility requires provenance, detection, and willingness to discard flattering numbers. A lower clean score is more valuable than a high score whose meaning cannot be defended.

18.4.11 Exercise 10

Exercise Title: Train Tiny LLM on CUDA and Track Loss Scaling

Part 1 - Why this matters

This exercise joins the pretraining objective, corpus, model, optimizer, and CUDA execution into one measured run. The goal is not only to make loss decrease, but to understand how loss changes with tokens seen, parameter scale, and compute. Every run must record configuration and validation at matched token checkpoints.

Loss scaling can mean several things. Here it means empirical reduction of validation NLL as training tokens or model size increase, not mixed-precision gradient scaling. Use the term carefully. Train multiple tiny model sizes under controlled data and compare learning curves against tokens, wall time, and estimated FLOPs.

CUDA timing is asynchronous, and first iterations include initialization and kernel warm-up. Separate warm-up from measured training, synchronize only at measurement boundaries, and count valid target tokens rather than padded input positions. Track peak memory and tokens per second so a lower loss is accompanied by its resource cost.

Validation data must remain fixed and leak-free. Training loss can continue to fall through memorization while validation plateaus. Save checkpoints and raw metric records so later exercises can analyze frequency, memorization, and gradient behavior without rerunning every experiment.

Part 2 - Mathematical form and reasoning

For parameter count N, cumulative valid tokens D_k, and validation loss L_k, fit a descriptive relation such as

$$L(D) = L_\infty + AD^{-\beta}$$

over the observed region. Estimated training compute can be recorded as

$$C_k \approx 6ND_k.$$

The coefficient is a coarse dense-Transformer convention. State that it omits implementation details and use measured wall time beside it.

Part 3 - Hints, controls, and evidence

1. Verify the model overfits one batch before launching the sweep.
2. Evaluate at fixed cumulative-token milestones, not only epoch boundaries.
3. Record model parameters, valid tokens, train loss, validation loss, gradient norm, learning rate, throughput, and peak memory.
4. Use identical validation tokens and tokenizer across all scales.
5. Repeat seeds and plot individual trajectories plus a summary.
6. Stop and diagnose NaN or non-finite gradients immediately; never smooth them out of the log.

Part 4 - Python code skeleton

```
import time
import torch

def train_cuda(model, loader, optimizer, loss_function,
               device, token_milestones):
    if device.type != "cuda":
        raise RuntimeError("CUDA device required")
    records = []
    tokens_seen = 0
    next_milestone_index = 0
    model.train()
    for batch in loader:
        input_ids = batch["input_ids"].to(device, non_blocking=True)
        valid_mask = batch["valid_mask"].to(device, non_blocking=True)
        optimizer.zero_grad(set_to_none=True)
        logits = model(input_ids)
        loss = loss_function(logits, input_ids, valid_mask)
        loss.backward()
        gradient_norm = torch.nn.utils.clip_grad_norm_(
            model.parameters(), 1.0
        )
```

```
        optimizer.step()
        tokens_seen = tokens_seen + int(valid_mask.sum().item())
        if next_milestone_index < len(token_milestones):
            milestone = token_milestones[next_milestone_index]
            if tokens_seen >= milestone:
                records.append({"tokens": tokens_seen,
                                "loss": float(loss.detach().cpu()),
                                "gradient_norm": float(
                                    gradient_norm.detach().cpu())})
                next_milestone_index = next_milestone_index + 1
        if next_milestone_index == len(token_milestones):
            break
    return records

def E10(model, loader, optimizer, loss_function, device):
    milestones = [10000, 30000, 100000, 300000]
    return train_cuda(model, loader, optimizer, loss_function,
                      device, milestones)
```

Part 5 - The Philosophy Behind the Exercise

Scaling curves transform training from an event into a trajectory. A final checkpoint hides how quickly learning began, where it slowed, and what resources were exchanged for each improvement. Tracking makes progress accountable.

CUDA supplies speed, but measurement supplies meaning. The research object is not merely a trained tensor file; it is the documented path from tokens and compute to changed predictive uncertainty.

18.4.12 Exercise 11

Exercise Title: Compare Training Stability Under Different Batch Sizes

Part 1 - Why this matters

Batch size changes the number of token losses averaged before one parameter update. Small batches produce noisier gradient estimates that can aid exploration but increase loss variance and instability. Large batches reduce sampling noise and improve hardware throughput up to a point, but consume more memory and may require learning-rate adjustment to preserve useful update magnitude.

For language models, sequences contain different numbers of valid targets. Batch size in sequences is therefore not the complete statistical batch size. Record valid tokens per update. Packing, padding, context length, and gradient accumulation can make two nominally equal batches contain very different amounts of supervision.

You will compare several microbatch sizes under two controls: equal optimizer steps and equal total valid tokens. Use identical data order where practical and repeat seeds. Measure loss variance, gradient norm, update norm, throughput, peak memory, NaN events, and final validation quality. Do not tune one batch size more extensively than the others without reporting it.

Learning-rate scaling should be a separate experiment. Linear or square-root rules are heuristics, not guarantees. Begin with a fixed learning rate to expose the direct batch effect, then run a declared scaling rule and compare stability.

Part 2 - Mathematical form and reasoning

For token gradients g_i, batch estimator is

$$\hat{g}_B = \frac{1}{B}\sum_{i=1}^{B} g_i, \qquad \operatorname{Var}(\hat{g}_B) \approx \frac{\Sigma}{B}$$

under an independence approximation that is imperfect for correlated tokens. Define update-to-parameter ratio

$$r_k = \frac{\|\theta_{k+1} - \theta_k\|_2}{\|\theta_k\|_2 + \epsilon}$$

and compare its distribution across batch configurations.

Part 3 - Hints, controls, and evidence

1. Report sequences, raw tokens, and valid target tokens per update.
2. Preserve data order or record every sampled example identifier.
3. Compare fixed and scaled learning rates as different tables.
4. Log step-level loss and gradient norms before aggregation into epochs.
5. Match total valid tokens when comparing final validation quality.
6. Explain throughput saturation and memory limits separately from statistical stability.

Part 4 - Python code skeleton

```
import torch

def parameter_norm(model):
    squared_sum = torch.zeros((), device=next(model.parameters()).device)
    for parameter in model.parameters():
        squared_sum = squared_sum + parameter.detach().float().square().sum()
    return torch.sqrt(squared_sum)

def snapshot_parameters(model):
    snapshots = []
    for parameter in model.parameters():
        snapshots.append(parameter.detach().clone())
    return snapshots

def update_norm(model, before):
    squared_sum = torch.zeros((), device=before[0].device)
    index = 0
```

```
    for parameter in model.parameters():
        difference = parameter.detach() - before[index]
        squared_sum = squared_sum + difference.float().square().sum()
        index = index + 1
    return torch.sqrt(squared_sum)

def E11(run_training, model_factory, batch_sizes, seeds):
    records = []
    for batch_size in batch_sizes:
        for seed in seeds:
            model = model_factory(seed)
            metrics = run_training(model, batch_size, seed)
            records.append({"batch_size": batch_size,
                            "seed": seed, "metrics": metrics})
    return records
```

Part 5 - The Philosophy Behind the Exercise

Batching decides how many experiences are allowed to speak before the model changes. A small batch lets each local sample influence the path strongly; a large batch seeks consensus across more evidence. Stability and responsiveness are two sides of this averaging choice.

There is no universally wise crowd size. The right batch emerges from data correlation, hardware, optimization, and the desired noise scale. Measurement replaces rules of thumb with an account of how collective evidence moves the model.

18.4.13 Exercise 12

Exercise Title: Analyze Gradient Noise in Token Prediction Objective

Part 1 - Why this matters

Each batch provides a sample estimate of the gradient over the full pretraining distribution. Different topics, token frequencies, sequence lengths, and ambiguous continuations produce different gradients. Their variation is gradient noise. Too much variation can destabilize updates; some noise can help optimization avoid narrow paths and improve generalization.

Token losses inside a sequence are correlated, so treating every token as an independent sample overstates effective batch size. Repeated documents and overlapping windows add more correlation. You will compare microbatch gradients using cosine similarity, norm variation, and an empirical noise scale while recording valid-token counts and domains.

Gradient measurement must occur before the optimizer modifies parameters. Create a fixed model state, compute gradients for multiple microbatches without stepping, flatten or summarize them consistently, and reset gradients between batches. Large flattened vectors can consume memory, so provide both exact tiny- model analysis and layer-wise statistics for larger models.

The final analysis should connect noise to learning rate and batch size without claiming a universal formula. Noise changes throughout training as predictions and data order change. Measure at several checkpoints and identify layers whose gradient directions disagree most across batches.

Part 2 - Mathematical form and reasoning

Let microbatch gradients be $g_1, \ldots, g_M$ and mean $\bar{g} = M^{-1} \sum_i g_i$. Empirical noise variance is

$$V = \frac{1}{M-1} \sum_i \|g_i - \bar{g}\|_2^2.$$

Signal magnitude is $S = \|\bar{g}\|_2^2$, giving descriptive ratio $V/(S + \epsilon)$. Pairwise directional agreement is

$$\cos(g_i, g_j) = \frac{g_i^\mathsf{T} g_j}{\|g_i\|_2 \|g_j\|_2}.$$

Report the estimator and sampling procedure rather than treating this ratio as a universal noise-scale definition.

Part 3 - Hints, controls, and evidence

1. Reload identical parameters before measuring each checkpoint condition.
2. Use `zero_grad(set_to_none=True)` between microbatches.
3. Normalize loss by valid tokens consistently.
4. Record domain, length, and token-frequency composition of each batch.
5. Compare exact flattened gradients with per-layer norms on a tiny model.
6. Repeat at early, middle, and late checkpoints.

Part 4 - Python code skeleton

```
import torch

def flattened_gradient(model):
    pieces = []
    for parameter in model.parameters():
        if parameter.grad is not None:
            pieces.append(parameter.grad.detach().reshape(-1).float())
    return torch.cat(pieces)

def collect_microbatch_gradients(model, batches, loss_function, device):
    gradients = []
    for batch in batches:
        model.zero_grad(set_to_none=True)
        input_ids = batch["input_ids"].to(device)
        logits = model(input_ids)
        loss = loss_function(logits, batch, device)
        loss.backward()
        gradients.append(flattened_gradient(model).cpu())
    return torch.stack(gradients)
```

```
def gradient_noise_statistics(gradient_matrix):
    mean_gradient = gradient_matrix.mean(dim=0)
    centered = gradient_matrix - mean_gradient
    variance = centered.square().sum(dim=1).sum()
    variance = variance / max(1, gradient_matrix.shape[0] - 1)
    signal = mean_gradient.square().sum()
    ratio = variance / signal.clamp_min(1e-30)
    return signal, variance, ratio

def E12(model, batches, loss_function, device):
    gradients = collect_microbatch_gradients(
        model, batches, loss_function, device
    )
    return gradient_noise_statistics(gradients)
```

Part 5 - The Philosophy Behind the Exercise

Optimization listens to a chorus of partial experiences. Gradient noise is the measure of their disagreement. Removing all disagreement would require hearing the whole distribution before every step, while listening to one voice at a time makes the path volatile.

Noise is not merely error around a true direction. It reveals heterogeneity in the data and can help determine which compromise the model learns. Understanding it means studying both optimization and the diversity of experiences that produce the gradients.

18.4.14 Exercise 13

Exercise Title: Implement Gradient Accumulation for Token-Level Training

Part 1 - Why this matters

Gradient accumulation simulates a larger effective batch by processing several microbatches before one optimizer step. It is essential when model activations or contexts do not fit at the desired batch size. Correct accumulation must match the gradient of the combined valid-token loss, including variable numbers of real targets per microbatch.

Dividing every microbatch loss by the number of accumulation steps is equivalent only when each microbatch contains the same statistical weight. With variable padding, the correct approach accumulates the sum of token losses and divides gradients by the total valid-token count. Otherwise short and long microbatches receive equal weight despite containing different supervision.

Optimizer stepping, gradient clipping, scheduler stepping, and gradient clearing occur once per effective batch, not once per microbatch. Mixed- precision scaling must preserve this order: scale each backward contribution, unscale once before clipping, then step and update the scaler. Remainder microbatches at the end of an epoch require a valid final step.

You will compare accumulated gradients and parameter updates with a true concatenated batch on a tiny deterministic model. Disable dropout or control its randomness for equivalence. Report maximum gradient and parameter differences before benchmarking memory savings.

Part 2 - Mathematical form and reasoning

For microbatch j with valid token set I_j, combined objective is

$$L = \frac{\sum_j \sum_{i \in I_j} \ell_i}{\sum_j |I_j|}.$$

Accumulate gradients of loss sums

$$g_{\text{sum}} = \sum_j \nabla_\theta \left(\sum_{i \in I_j} \ell_i \right)$$

then divide every gradient by $N = \sum_j |I_j|$ before clipping and stepping.

Part 3 - Hints, controls, and evidence

1. Compare against one concatenated batch from the same initial state.
2. Use loss reduction `sum` inside each microbatch and count valid targets.
3. Divide gradients once by total valid tokens.
4. Clip, step optimizer, and step scheduler once per effective batch.
5. Handle the final partial accumulation group explicitly.
6. Test equality in float64 on CPU before measuring reduced-precision CUDA.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def token_loss_sum(model, input_ids, valid_mask):
    logits = model(input_ids)
    prediction_logits = logits[:, :-1, :]
    targets = input_ids[:, 1:]
    target_mask = valid_mask[:, 1:]
    losses = F.cross_entropy(
        prediction_logits.reshape(-1, prediction_logits.shape[-1]),
        targets.reshape(-1), reduction="none"
    ).view_as(targets)
    return losses.masked_select(target_mask).sum(), int(target_mask.sum())

def accumulated_step(model, microbatches, optimizer, device):
    optimizer.zero_grad(set_to_none=True)
    total_valid_tokens = 0
    for batch in microbatches:
        input_ids = batch["input_ids"].to(device)
        valid_mask = batch["valid_mask"].to(device)
        loss_sum, valid_tokens = token_loss_sum(
            model, input_ids, valid_mask
        )
        loss_sum.backward()
```

```
        total_valid_tokens = total_valid_tokens + valid_tokens
    if total_valid_tokens == 0:
        raise ValueError("effective batch has no valid tokens")
    for parameter in model.parameters():
        if parameter.grad is not None:
            parameter.grad.div_(total_valid_tokens)
    gradient_norm = torch.nn.utils.clip_grad_norm_(
        model.parameters(), 1.0
    )
    optimizer.step()
    return gradient_norm, total_valid_tokens

def E13(model, microbatches, optimizer, device):
    return accumulated_step(model, microbatches, optimizer, device)
```

Part 5 - The Philosophy Behind the Exercise

Accumulation separates when evidence is observed from when a decision is made. The model can listen to several microbatches without storing all their activations simultaneously, then combine their influence into one update.

Equivalence depends on weighting. Counting microbatches treats containers as equal; counting valid tokens treats observations as equal. The denominator is a small mathematical choice that determines whose evidence has authority in the effective batch.

18.4.15 Exercise 14

Exercise Title: Explain Why LLMs Can Imitate Reasoning Without Understanding

Part 1 - Why this matters

Pretraining corpora contain explanations, proofs, worked examples, debates, and step-by-step solutions. Next-token prediction can learn the surface and structural patterns of these traces. A model may produce a convincing chain that resembles reasoning even when the steps do not causally support the conclusion, the final answer was copied from a familiar template, or small perturbations destroy the behavior.

The title should not be converted into the opposite absolute claim that LLMs never perform useful computation. Behavioral reasoning can be tested through novel problems, interventions, consistency, intermediate-state validity, and out-of-distribution composition. The point is that a generated explanation by itself is insufficient evidence of the mechanism or depth of understanding.

You will construct task families with matched surface forms: familiar templates, novel variable substitutions, counterfactual rules, misleading intermediate steps, and answer choices whose order changes. Evaluate final answer and trace validity separately. Ask for answer-only and explanation-plus-answer outputs to test whether producing a trace improves, harms, or merely decorates performance.

Do not use private chain-of-thought as a required transparency mechanism. Instead, request concise verifiable calculations, externally checkable steps, tool results, or structured intermediate answers. The evaluation should focus on observable evidence rather than assuming a fluent narrative reveals hidden internal computation.

Part 2 - Mathematical form and reasoning

Let A indicate correct final answer and R indicate a valid supplied rationale. Report the joint table

$$P(A, R), \quad P(A, \neg R), \quad P(\neg A, R), \quad P(\neg A, \neg R).$$

For semantics-preserving transformations $g \in \mathcal{G}$, consistency is

$$C = \frac{1}{N|\mathcal{G}|} \sum_{i,g} \mathbb{I}[a(g(x_i)) = a(x_i)].$$

High trace fluency without validity or intervention stability supports imitation more than robust rule application.

Part 3 - Hints, controls, and evidence

1. Define a machine-checkable rule for intermediate-step validity.
2. Separate familiar templates from held-out compositional forms.
3. Randomize names, numbers, option order, and irrelevant wording.
4. Insert a plausible but wrong intermediate claim and test correction.
5. Compare answer-only with concise-verification formats.
6. State which behaviors were demonstrated without making claims about consciousness or inaccessible internal experience.

Part 4 - Python code skeleton

```
def joint_reasoning_counts(records):
    counts = {}
    counts["answer_and_rationale"] = 0
    counts["answer_only"] = 0
    counts["rationale_only"] = 0
    counts["neither"] = 0
    for record in records:
        answer = record["answer_correct"]
        rationale = record["rationale_valid"]
        if answer and rationale:
            counts["answer_and_rationale"] += 1
        elif answer and not rationale:
            counts["answer_only"] += 1
        elif not answer and rationale:
            counts["rationale_only"] += 1
        else:
            counts["neither"] += 1
    return counts

def E14(generate, task_families):
```

```
    records = []
    for family in task_families:
        for case in family["cases"]:
            output = generate(case["prompt"])
            record = {}
            record["family"] = family["name"]
            record["answer_correct"] = family["score_answer"](
                output, case
            )
            record["rationale_valid"] = family["score_rationale"](
                output, case
            )
            records.append(record)
    return records, joint_reasoning_counts(records)
```

Part 5 - The Philosophy Behind the Exercise

Explanation is behavior, not privileged access to cause. A model can produce the language of reasoning because such language has structure worth predicting. The narrative may be useful, mistaken, or post-hoc, just as human explanations can be.

The responsible position avoids both enchantment and dismissal. We test whether the system applies rules under change, exposes verifiable steps, and recognizes contradictions. Understanding becomes an empirical program of robustness and causal intervention rather than a judgment made from eloquence alone.

18.4.16 Exercise 15

Exercise Title: Compare Language Modeling vs Masked Modeling Objectives

Part 1 - Why this matters

Causal language modeling predicts each next token from the left prefix. Masked language modeling hides selected tokens and reconstructs them using context on both sides. The objectives use similar Transformer components but create different information boundaries and supervision patterns. Their natural uses therefore differ.

CLM supplies a target at nearly every transition and directly matches autoregressive generation. MLM can build bidirectional representations useful for classification, retrieval, and token labeling, but ordinary MLM does not define a left-to-right joint probability without an additional generation procedure. Comparing only final loss is meaningless because the conditioning sets and predicted positions differ.

You will construct the same token sequences for both objectives. For CLM, apply a causal mask and shift targets. For MLM, sample mask positions reproducibly, replace tokens according to a declared corruption policy, and compute loss only on selected positions. Measure supervised-token fraction, compute, representation probe quality, and generation suitability.

Masking policy affects difficulty. If every selected token is replaced by a visible mask symbol, pretraining creates a token absent from downstream inputs; some procedures retain or randomly replace a fraction. Record the policy and prevent unmasked target leakage through preprocessing.

Part 2 - Mathematical form and reasoning

Causal objective is

$$L_{\mathrm{CLM}} = -\sum_{t=1}^{T-1} \log p_\theta(x_{t+1} \mid x_{\le t}).$$

For sampled mask set M,

$$L_{\mathrm{MLM}} = -\sum_{t \in M} \log p_\theta(x_t \mid \widetilde{x}_{\setminus M}).$$

If masking rate is r, expected MLM supervised positions are approximately rT, compared with roughly $T-1$ for CLM, though the conditioning information is richer in MLM.

Part 3 - Hints, controls, and evidence

1. Use one tokenizer and identical raw sequences.
2. Print causal targets and MLM selected positions for a tiny example.
3. Seed mask sampling and verify targets are never taken from corrupted IDs.
4. Match valid target-token budgets rather than epochs when comparing optimization.
5. Evaluate a bidirectional classification probe and autoregressive generation separately.
6. Explain why objective-specific losses and perplexities are not directly comparable.

Part 4 - Python code skeleton

```
import torch

def build_mlm_example(input_ids, mask_token_id,
                      mask_probability, generator):
    random_values = torch.rand(
        input_ids.shape, generator=generator, device=input_ids.device
    )
    selected = random_values < mask_probability
    corrupted = input_ids.clone()
    targets = input_ids.clone()
    corrupted[selected] = mask_token_id
    targets[~selected] = -100
    return corrupted, targets, selected

def build_clm_example(input_ids):
    model_inputs = input_ids[:, :-1]
    targets = input_ids[:, 1:]
    return model_inputs, targets

def E15(input_ids, mask_token_id):
    generator = torch.Generator(device=input_ids.device)
    generator.manual_seed(1515)
```

```
    mlm_inputs, mlm_targets, selected = build_mlm_example(
        input_ids, mask_token_id, 0.15, generator
    )
    clm_inputs, clm_targets = build_clm_example(input_ids)
    results = {}
    results["mlm_inputs"] = mlm_inputs
    results["mlm_targets"] = mlm_targets
    results["mlm_supervised"] = int(selected.sum().item())
    results["clm_inputs"] = clm_inputs
    results["clm_targets"] = clm_targets
    results["clm_supervised"] = clm_targets.numel()
    return results
```

Part 5 - The Philosophy Behind the Exercise

Every objective creates an artificial absence. CLM hides the future; MLM hides selected pieces of the present. The model learns by reconstructing what the objective withholds, and the shape of that absence influences the representation that emerges.

Neither objective is language itself. Each is an educational game designed by engineers. Choosing between them means deciding which information boundary and downstream behavior best match the purpose of the system.

18.4.17 Exercise 16

Exercise Title: Analyze Failure Cases on Rare Tokens

Part 1 - Why this matters

Rare tokens often represent names, technical notation, minority scripts, unusual numbers, or tokenizer fragments created by uncommon byte patterns. Their embeddings and output rows receive fewer direct updates, and evaluation averages can hide their failures. Rare-token errors can be operationally important when the token identifies a medicine, legal clause, account code, or scientific symbol.

Rarity has several forms. A token may be globally rare but predictable in one fixed phrase, frequent as part of copied boilerplate but absent from diverse contexts, or composed from common subwords even though the complete word is rare. Analyze frequency, context diversity, fragmentation, and target loss together.

Build a held-out suite containing rare identifiers, long numbers, code symbols, multilingual names, and minimal pairs differing by one token. Measure target NLL, top-k rank, calibration, copy accuracy, and edit distance. Compare direct generation with a pointer or constrained-copy mechanism conceptually, because some applications should copy exact input strings instead of relying on parametric recall.

Do not expose real personal identifiers. Generate synthetic examples and retain their construction rules. Group results by frequency and semantic function, and inspect whether normalization or tokenization accidentally changes the expected target before blaming the model.

Part 2 - Mathematical form and reasoning

For target token y, rank is

$$r(y) = 1 + \sum_{v \neq y} \mathbb{I}[z_v > z_y].$$

Mean reciprocal rank for rare targets R is

$$\mathrm{MRR}_R = \frac{1}{|R|} \sum_{i \in R} \frac{1}{r(y_i)}.$$

Context diversity for token v can be approximated by the number or entropy of distinct neighboring token patterns, distinguishing repeated duplication from broad exposure.

Part 3 - Hints, controls, and evidence

1. Derive rarity from training targets only.
2. Verify tokenizer round-trip and expected token boundaries for every case.
3. Report loss, rank, top-k membership, and decoded alternatives.
4. Compare rare tokens with frequency-matched controls from different categories.
5. Test exact copying when the rare string is present in context.
6. Propose retrieval, constrained decoding, byte fallback, or human validation for high-consequence rare outputs.

Part 4 - Python code skeleton

```
import torch

def target_rank(logit_row, target_id):
    target_logit = logit_row[target_id]
    higher = (logit_row > target_logit).sum()
    return int(higher.item()) + 1

def analyze_rare_positions(logits, targets,
                           training_counts, rarity_threshold):
    records = []
    for batch_index in range(targets.shape[0]):
        for time_index in range(targets.shape[1]):
            target_id = int(targets[batch_index, time_index].item())
            frequency = int(training_counts[target_id].item())
            if frequency <= rarity_threshold:
                row = logits[batch_index, time_index]
                log_probability = torch.log_softmax(row, dim=-1)[target_id]
                record = {}
                record["target_id"] = target_id
                record["frequency"] = frequency
                record["nll"] = float((-log_probability).cpu())
                record["rank"] = target_rank(row, target_id)
                records.append(record)
    return records
```

```
def E16(model, input_ids, training_counts, rarity_threshold):
    logits = model(input_ids[:, :-1])
    targets = input_ids[:, 1:]
    return analyze_rare_positions(
        logits, targets, training_counts, rarity_threshold
    )
```

Part 5 - The Philosophy Behind the Exercise

Rarity is where a statistical system reveals whose experience it has scarcely seen. The model may be fluent in the center of the distribution while fragile at its edges. Those edges are not unimportant merely because they contribute little to an average.

Rare-token engineering asks when generation should yield to copying, retrieval, constraints, or human review. Competence includes knowing which details should not be reconstructed from uncertain memory.

18.4.18 Exercise 17

Exercise Title: Build Dataset Quality Checklist for LLM Pretraining

Part 1 - Why this matters

Pretraining data errors propagate into expensive model behavior. A dataset quality checklist converts broad intentions such as "clean and diverse" into auditable gates. It should cover authorization, provenance, integrity, deduplication, language and domain balance, privacy, benchmark contamination, harmful content policy, formatting, tokenization, and reproducibility.

Quality is multidimensional. A document can be syntactically clean but factually wrong, legally unusable, private, duplicated, or outside the model's intended domain. A single scalar filter score hides tradeoffs. Store individual signals, decision reasons, and retained source metadata so exclusion can be reviewed.

You will implement structured checks with stable identifiers, severity, pass or fail state, evidence, and remediation. Create good fixtures plus injected failures: invalid encoding, empty text, duplicate documents, leaked benchmark answers, secrets, mislabeled language, extreme token expansion, and corrupted markup. Every known fault must trigger the intended check.

Sampling audits complement automated checks. Human reviewers should inspect stratified samples across source, language, score band, and exclusion reason. Agreement and escalation rules must be documented. The checklist is a living artifact versioned with the exact corpus build.

Part 2 - Mathematical form and reasoning

For source group g, retention rate is

$$r_g = \frac{N_{g,\text{kept}}}{N_{g,\text{input}}}.$$

Disparity across groups can be summarized by $\max_g r_g - \min_g r_g$, while requiring interpretation of legitimate quality differences. Fault-detection coverage is

$$C = \frac{|\{f \in F : \text{at least one intended check detects } f\}|}{|F|}.$$

Neither metric proves quality; both make blind spots measurable.

Part 3 - Hints, controls, and evidence

1. Assign identifiers such as `DATA-PROV-001` and versions to checks.
2. Never discard raw provenance needed to audit a filtered document.
3. Separate blocking legal or privacy failures from quality warnings.
4. Measure retention by source, language, domain, and document-length band.
5. Maintain benchmark hashes and near-duplicate detection before training.
6. Require every corpus release to include checksums, configuration, counts, and known limitations.

Part 4 - Python code skeleton

```
class DataCheckResult:
    def __init__(self, check_id, passed, severity,
                 evidence, remediation):
        self.check_id = check_id
        self.passed = passed
        self.severity = severity
        self.evidence = evidence
        self.remediation = remediation

def check_nonempty(document):
    normalized = document["text"].strip()
    return DataCheckResult(
        "DATA-INTEGRITY-001", len(normalized) > 0, "blocking",
        {"normalized_length": len(normalized)},
        "remove empty documents or repair extraction"
    )

def check_provenance(document):
    required = ["source", "collected_at", "authorization"]
    missing = []
    for field in required:
        if field not in document or document[field] in [None, ""]:
            missing.append(field)
    return DataCheckResult(
        "DATA-PROV-001", len(missing) == 0, "blocking",
        {"missing_fields": missing},
        "restore provenance before admitting the document"
    )

def E17(documents):
    report = []
    for document in documents:
```

```
        results = []
        results.append(check_nonempty(document))
        results.append(check_provenance(document))
        report.append({"document_id": document["id"],
                       "results": results})
    return report
```

Part 5 - The Philosophy Behind the Exercise

A dataset checklist is a constitution for evidence. It defines which material may enter the learning process, which records must accompany it, and which harms require exclusion or review. Those decisions shape the eventual model even when they never appear in its architecture diagram.

No checklist can certify truth or justice completely. Its value is to make responsibility inspectable and revisable. Quality grows when decisions leave a trace and failures become new tests rather than forgotten incidents.

18.4.19 Exercise 18

Exercise Title: Explain Why More Data Beats Better Optimizers at Scale

Part 1 - Why this matters

Optimizer improvements can make training faster or more stable, but an optimizer cannot create missing linguistic evidence. Once a strong baseline optimizer is working well, expanding high-quality, diverse data can improve coverage, reduce repeated memorization, and expose new relationships. The title is a scaling hypothesis, not permission to ignore optimization or collect unlimited low-quality text.

Compare optimizer change and data expansion under matched compute. One axis trains the same data with two competent optimizers; the other trains the same model and optimizer with increasingly diverse token sets. Record training loss, validation loss, unique-token exposure, domain coverage, wall time, and stability. Better optimization may win early while more data improves the final generalization frontier.

Data quantity must distinguish unique information from repeated tokens. Duplicating a corpus increases token presentations without adding evidence. Optimizer tuning must receive a fair search budget; comparing a carefully tuned baseline with an untuned alternative proves little. Use several seeds and report the hyperparameter trials consumed.

At true scale, data availability, compute, and optimizer quality interact. Poor optimization can waste data; excessive data with insufficient compute leaves the model undertrained. The conclusion should identify the regime observed and the point at which data, model capacity, or optimization becomes limiting.

Part 2 - Mathematical form and reasoning

Let validation loss depend on model N, unique data D, and optimizer efficiency e_o:

$$L \approx L_\infty + AN^{-\alpha} + BD^{-\beta} + R(e_o, C),$$

where R represents optimization error under compute budget C. A better optimizer reduces R; more informative data reduces the data-limited term. This is a conceptual decomposition, not an identifiable law from one small experiment.

Part 3 - Hints, controls, and evidence

1. Use two established optimizer configurations and document tuning budget.
2. Compare unique data growth with pure duplication as a control.
3. Match compute, parameter count, tokenizer, and validation set.
4. Plot validation loss against unique tokens and total tokens separately.
5. Track domain coverage and rare-token performance.
6. State the observed regime and avoid extrapolating a tiny experiment to all industrial training.

Part 4 - Python code skeleton

```
def experimental_grid(model_factory, train_function,
                      optimizer_factories, data_variants, seeds):
    records = []
    for optimizer_name in optimizer_factories:
        optimizer_factory = optimizer_factories[optimizer_name]
        for data_name in data_variants:
            data = data_variants[data_name]
            for seed in seeds:
                model = model_factory(seed)
                optimizer = optimizer_factory(model.parameters())
                metrics = train_function(model, optimizer, data, seed)
                record = {}
                record["optimizer"] = optimizer_name
                record["data"] = data_name
                record["seed"] = seed
                record["unique_tokens"] = data["unique_token_count"]
                record["total_tokens"] = data["total_token_count"]
                record["metrics"] = metrics
                records.append(record)
    return records

def E18(model_factory, train_function,
        optimizer_factories, data_variants):
    seeds = [13, 31, 47]
    return experimental_grid(model_factory, train_function,
                             optimizer_factories, data_variants, seeds)
```

Part 5 - The Philosophy Behind the Exercise

Optimization determines how efficiently a learner extracts structure from experience. Data determines what experience exists to be extracted. A more skillful path through the same narrow world cannot substitute indefinitely for seeing a broader world.

The principle favors evidence over cleverness once the learning machinery is competent. Yet accumulation without curation is not wisdom. More data deserves its advantage only when it adds lawful, diverse, and meaningful information rather than multiplying noise.

18.4.20 Exercise 19

Exercise Title: Produce a Pretraining Design Report

Part 1 - Why this matters

The final exercise turns isolated experiments into a pretraining decision document. A design report should allow another team to understand the intended model, data rights, tokenizer, objective, scale, optimization, infrastructure, evaluation, risks, and stop criteria before expensive training begins. It is a commitment to measurable choices, not a retrospective story written after results are known.

Begin with purpose and exclusions. Define users, languages, domains, context, capabilities, and behaviors the base model is not expected to provide before post-training. Describe source provenance, authorization, privacy rules, deduplication, contamination controls, mixture weights, and quality audits. Include corpus versions and checksums.

Specify architecture and token economics: vocabulary, parameter count, context, expected tokens per document, target token budget, precision, FLOP estimate, GPU memory, parallelism, checkpoint frequency, and failure recovery. Define validation suites for loss, rare tokens, domains, memorization, leakage, hallucination, and safety-relevant content. Every release gate needs an owner and rollback action.

The report must include uncertainty and alternatives. State which claims are supported by pilot experiments, which are extrapolations, and what would cause the project to change data mixture, optimizer, model scale, or stop entirely. Include cost and environmental estimates without false precision.

Part 2 - Mathematical form and reasoning

Core budget constraints include

$$C_{\text{train}} \approx 6ND, \qquad M_{\text{train}} \leq M_{\text{available}}, \qquad \sum_d w_d = 1.$$

Define a vector of gates rather than one score:

$$G = (L_{\text{val}}, Q_{\text{domain}}, H_{\text{unsupported}}, R_{\text{rare}}, C_{\text{leak}}, P_{\text{privacy}}, S_{\text{stability}}).$$

Deployment or continuation is permitted only when every blocking component meets its declared threshold. Tradeoffs must be reviewed rather than hidden in an average.

Part 3 - Hints, controls, and evidence

1. Give every dataset, configuration, checkpoint, and evaluation suite a versioned identifier.
2. Include pilot scaling results and confidence ranges used for budget decisions.
3. List known exclusions and underrepresented groups explicitly.
4. Define automatic stop conditions for NaNs, loss spikes, privacy findings, or contamination.
5. Assign owners for data, model, infrastructure, evaluation, safety, and incident response.
6. End with a decision table comparing at least two feasible designs and the evidence for the selected one.

Part 4 - Python code skeleton

```
class PretrainingDesign:
    def __init__(self, design_id):
        self.design_id = design_id
        self.sections = {}
        self.gates = []

    def add_section(self, name, content):
        self.sections[name] = content

    def add_gate(self, gate_id, metric, threshold,
                 comparison, owner, rollback_action):
        gate = {}
        gate["gate_id"] = gate_id
        gate["metric"] = metric
        gate["threshold"] = threshold
        gate["comparison"] = comparison
        gate["owner"] = owner
        gate["rollback_action"] = rollback_action
        self.gates.append(gate)

    def missing_sections(self, required_sections):
        missing = []
        for section in required_sections:
            if section not in self.sections:
                missing.append(section)
        return missing

def E19():
    report = PretrainingDesign("LLM-PRETRAIN-V1")
    required = ["purpose", "data", "tokenizer", "architecture",
                "optimization", "compute", "evaluation",
                "risk", "recovery"]
    report.add_section("purpose", ...)
    report.add_section("data", ...)
    report.add_gate(...)
    missing = report.missing_sections(required)
    return report, missing
```

Part 5 - The Philosophy Behind the Exercise

A design report makes the future model answerable to decisions made before it exists. It records which evidence will shape it, which resources will be spent, which failures matter, and who has authority to stop the process.

Pretraining is often described as optimization at scale, but it is also the construction of a statistical inheritance. The report is where technical ambition meets stewardship. A well-designed model begins with a traceable reason for every major choice and a plan for what to do when reality disagrees.

18.5 LLM03

Chapter Title: Instruction Tuning & Fine-Tuning
Specialization: Post-Training and Behavioral Adaptation Engineering

18.5.1 Exercise 00

Exercise Title: Explain Instruction Tuning and Why It Changes Behavior Without Changing Architecture

Part 1 - Deep conceptual foundation

Pretraining teaches a decoder-only language model to estimate what text is likely to follow a prefix. Instruction tuning keeps that same causal language model, but changes the distribution of examples from arbitrary continuation to structured interactions: an instruction, optional context, and a desirable response. The architecture can remain completely unchanged. What changes is the conditional behavior that receives repeated probability mass during optimization.

This distinction is central. A pretrained model may know the information needed to answer a question yet continue the prompt as if it were a web page, imitate several speakers, or produce an unrelated completion. Supervised instruction fine-tuning presents many examples in which an instruction is followed by a direct, relevant response. Gradient descent then increases the likelihood of response patterns conditioned on instruction-like prefixes. The model is not given a new reasoning module or a new attention mechanism; existing parameters are reorganized toward a different behavioral mode.

Instruction tuning must therefore be separated from architecture, knowledge, and alignment. It can expose capabilities already latent in the base model, teach output conventions, and add some domain information. It does not prove that every response is truthful, safe, or grounded. A polished instruction format can make an incorrect answer more persuasive. The experiment must measure content correctness separately from format compliance.

In this exercise you will freeze a copy of a base model, fine-tune another copy on controlled instruction-response pairs, and compare both on familiar and paraphrased prompts. You will also inspect parameter names and tensor shapes before and after training to prove that the computational graph is unchanged while parameter values and output distributions move.

Part 2 - Mathematical model and reasoning

Let p_{θ_0} be the pretrained policy and let $D_{\mathrm{SFT}} = \{(x_i, y_i)\}_{i=1}^{N}$ contain instructions x_i and desired responses y_i. Instruction tuning solves

$$\theta^* = \arg\min_{\theta} - \sum_{i=1}^{N} \sum_{t=1}^{|y_i|} \log p_\theta(y_{i,t} \mid x_i, y_{i,<t}).$$

The architecture function f is unchanged:

$$z = f(x; \theta_0) \quad \longrightarrow \quad z' = f(x; \theta^*),$$

but the conditional distribution changes. A useful behavioral movement measure is the response-token log-probability gain

$$\Delta_i = \frac{1}{|y_i|} \sum_{t} \left[\log p_{\theta^*}(y_{i,t} \mid x_i, y_{i,<t}) - \log p_{\theta_0}(y_{i,t} \mid x_i, y_{i,<t}) \right].$$

Positive Δ_i shows that the desired response became more probable; it does not by itself establish correctness on unseen instructions.

Part 3 - Experiment, controls, and evidence

1. Record the model class, layer count, hidden size, parameter names, and parameter shapes before tuning.
2. Construct examples for direct answers and text transformations. Add classification and refusal cases so behavior is not tied to one template.
3. Save generations and target log-probabilities from the untouched base checkpoint.
4. Fine-tune a cloned checkpoint while keeping tokenizer and architecture fixed.
5. Compare exact prompts, paraphrases, unrelated base-language prompts, and deliberately ambiguous requests.
6. Report format compliance, semantic correctness, calibration, and retained base capability as separate metrics.
7. Prove architecture identity by comparing every parameter name and shape; then prove behavioral change by comparing logits and decoded responses.

Part 4 - Explicit PyTorch implementation

```
import copy
import torch

def describe_architecture(model):
    description = []
    for parameter_name, parameter in model.named_parameters():
        item = {}
        item["name"] = parameter_name
        item["shape"] = tuple(parameter.shape)
        description.append(item)
    return description

def mean_target_log_probability(model, input_ids, labels):
    logits = model(input_ids).logits
    log_probabilities = torch.log_softmax(logits[:, :-1, :], dim=-1)
    target_ids = labels[:, 1:]
    valid_mask = target_ids != -100
    safe_targets = target_ids.clone()
    safe_targets[~valid_mask] = 0
    selected = log_probabilities.gather(
        dim=-1, index=safe_targets.unsqueeze(-1)
    ).squeeze(-1)
    return selected.masked_select(valid_mask).mean()
```

```
def E00(base_model, training_batch, learning_rate):
    tuned_model = copy.deepcopy(base_model)
    before = describe_architecture(base_model)
    after = describe_architecture(tuned_model)
    assert before == after

    input_ids = training_batch["input_ids"]
    labels = training_batch["labels"]
    with torch.no_grad():
        base_score = mean_target_log_probability(
            base_model, input_ids, labels
        )

    optimizer = torch.optim.AdamW(
        tuned_model.parameters(), lr=learning_rate
    )
    optimizer.zero_grad(set_to_none=True)
    tuned_score = mean_target_log_probability(
        tuned_model, input_ids, labels
    )
    loss = -tuned_score
    loss.backward()
    optimizer.step()

    with torch.no_grad():
        new_score = mean_target_log_probability(
            tuned_model, input_ids, labels
        )
    return base_score, new_score, tuned_model
```

Part 5 - The Philosophy Behind the Exercise

Instruction tuning demonstrates that behavior is not identical to structure. The same network can act like an autocomplete engine, an assistant, a classifier, or a formatter depending on which conditional patterns its weights have been trained to prefer. This makes post-training powerful because a new behavioral interface need not require rebuilding the model from zero.

It also creates a moral and scientific obligation: fluent obedience must not be confused with truth or understanding. The exercise teaches us to ask not only whether the model follows a request, but what evidence shaped that obedience, which capabilities were displaced, and whether the new confidence is deserved.

18.5.2 Exercise 01

Exercise Title: Build Supervised Instruction Dataset and Validate Prompt-Response Alignment

Part 1 - Deep conceptual foundation

An instruction dataset is not merely a collection of text pairs. It is a behavioral specification encoded as token sequences. Roles, separators, end-of-response markers, system constraints,

context blocks, and assistant boundaries jointly determine what conditional task the model sees. If these elements are inconsistent, the model must spend capacity learning formatting accidents instead of the intended instruction-response relation.

The critical invariant is alignment. Every supervised response token must occupy the position predicted by the preceding token, while instruction and padding tokens must follow the chosen loss policy. A dataset can decode into plausible text and still be wrong by one position. It can also leak the response into the prompt, truncate away the end marker, or reuse the padding identifier as an ordinary token. These failures often lower training loss, making them dangerous rather than obvious.

Build a typed record with stable identifiers, source provenance, task family, instruction, optional context, response, template version, and quality status. Keep raw records separate from rendered examples so a future template change does not silently rewrite dataset identity. Validation must happen at the text level, token level, and batch level. Store rejected records with reasons rather than deleting evidence of preprocessing decisions.

The exercise will create a collator without compact programming shortcuts. It will render records, tokenize them, locate the response boundary, truncate under an explicit policy, pad the batch, and return both labels and metadata. Finally, it will decode several rows and print a token-by-token supervision map for human inspection.

Part 2 - Formal data contract

For record r_i, define a deterministic renderer g_v for template version v:

$$s_i = g_v(r_i) = p_i \| y_i \| e,$$

where p_i is the rendered prompt, y_i the response, and e the end token. If a_i is the first response-token index, labels satisfy

$$\ell_{i,t} = \begin{cases} -100, & t < a_i \text{ or position is padding}, \\ x_{i,t}, & t \geq a_i. \end{cases}$$

The sequence must preserve $a_i < T_i$, contain at least one supervised token, and terminate under the declared truncation rule.

Part 3 - Validation protocol

1. Reject empty instructions, empty responses, unknown roles, duplicate record identifiers, and invalid Unicode according to a documented policy.

2. Render with one versioned template and never search for a response string inside the full text; repeated text makes string matching ambiguous.

3. Tokenize prompt and response separately to obtain an authoritative boundary, then verify their concatenation under the tokenizer contract.

4. Prefer truncating optional context or prompt history before truncating the answer; log every truncated record.

5. Decode input identifiers, supervised labels, and the boundary token for a sample from every task family.

6. Test maximum length, one-token response, missing context, embedded role markers, and non-English text.

Part 4 - Explicit Python dataset builder

```
import torch

def validate_record(record):
    required = ["record_id", "instruction", "response", "task"]
    for field_name in required:
        if field_name not in record:
            raise ValueError("missing field: " + field_name)
    if len(record["instruction"].strip()) == 0:
        raise ValueError("instruction is empty")
    if len(record["response"].strip()) == 0:
        raise ValueError("response is empty")

def render_record(record):
    prompt = "<system>You are precise.</system>\n"
    prompt = prompt + "<user>" + record["instruction"]
    if "context" in record and len(record["context"]) > 0:
        prompt = prompt + "\n<context>" + record["context"] + "</context>"
    prompt = prompt + "</user>\n<assistant>"
    response = record["response"] + "</assistant>"
    return prompt, response

def encode_record(tokenizer, record, maximum_length):
    validate_record(record)
    prompt, response = render_record(record)
    prompt_ids = tokenizer.encode(prompt, add_special_tokens=False)
    response_ids = tokenizer.encode(response, add_special_tokens=False)
    if len(response_ids) == 0:
        raise ValueError("response produced no tokens")
    allowed_prompt = maximum_length - len(response_ids)
    if allowed_prompt < 1:
        raise ValueError("response does not fit maximum length")
    if len(prompt_ids) > allowed_prompt:
        prompt_ids = prompt_ids[-allowed_prompt:]
    input_ids = prompt_ids + response_ids
    labels = []
    position = 0
    while position < len(prompt_ids):
        labels.append(-100)
        position = position + 1
    position = 0
    while position < len(response_ids):
        labels.append(response_ids[position])
        position = position + 1
    return input_ids, labels, len(prompt_ids)

def E01(tokenizer, records, maximum_length, padding_id):
    encoded = []
```

```
    maximum_batch_length = 0
    seen_ids = set()
    for record in records:
        if record["record_id"] in seen_ids:
            raise ValueError("duplicate record identifier")
        seen_ids.add(record["record_id"])
        item = encode_record(tokenizer, record, maximum_length)
        encoded.append(item)
        if len(item[0]) > maximum_batch_length:
            maximum_batch_length = len(item[0])

    batch_inputs = []
    batch_labels = []
    for input_ids, labels, boundary in encoded:
        while len(input_ids) < maximum_batch_length:
            input_ids.append(padding_id)
            labels.append(-100)
        batch_inputs.append(input_ids)
        batch_labels.append(labels)
        assert labels[boundary] != -100
    return torch.tensor(batch_inputs), torch.tensor(batch_labels)
```

Part 5 - The Philosophy Behind the Exercise

A training example is a small constitution for model behavior. Formatting decides who is speaking, masking decides whose words are judged, and truncation decides which evidence survives. These are not neutral preprocessing details; they are assumptions translated into gradients.

The deeper lesson is traceability. If a model learns an undesirable response, we should be able to travel backward from output to checkpoint, batch, rendered sequence, raw record, and source decision. High-quality datasets preserve this chain of accountability instead of presenting text as anonymous fuel.

18.5.3 Exercise 02

Exercise Title: Implement SFT Loss with Causal Masking and Verify Correct Token-Level Supervision

Part 1 - Deep conceptual foundation

Supervised fine-tuning normally uses the same next-token cross entropy as pretraining, but only selected positions are allowed to contribute. The causal attention mask controls what each hidden state can read; the label mask controls which predictions are scored. Confusing these two masks is a fundamental error. A causal mask prevents future-token leakage, while a response-only label mask prevents prompt tokens from dominating the optimization target.

Assume the input sequence contains system text, user instruction, assistant response, and padding. At position t, the logit predicts token $t + 1$. Therefore the labels must be shifted consistently with logits. If labels are masked before the shift without checking the boundary, the first assistant token may be lost or the last prompt token may be trained as an answer. The

only safe method is to write the tensor shapes and inspect the boundary by token identifier and decoded text.

Response-only supervision is common but not universally optimal. Supervising the full conversation can help a model reproduce a serialization format, while response-only loss concentrates gradients on assistant behavior. The selected policy must be explicit. For multi-turn dialogue, assistant regions may appear more than once, so a single boundary index is insufficient; use a token-level role mask.

This exercise implements loss without relying on a model-provided loss field. You will compare it against a manual log-softmax calculation, test invariant behavior when padding is added, and verify that changing an ignored prompt label cannot alter the loss.

Part 2 - Mathematical form and invariants

For logits $z \in \mathbb{R}^{B\times(T-1)\times V}$, shifted targets y, and supervision mask m,

$$L_{\text{SFT}} = -\frac{1}{\sum_{b,t} m_{b,t}} \sum_{b,t} m_{b,t} \log \frac{\exp z_{b,t,y_{b,t}}}{\sum_{v=1}^{V} \exp z_{b,t,v}}.$$

Three required invariants are

$$\sum m > 0, \qquad y_{b,t} \in [0, V-1] \text{ when } m_{b,t} = 1, \qquad L(x\|\text{pad}) = L(x).$$

The last equality holds when added padding positions are properly masked and the model's valid logits are unchanged.

Part 3 - Verification protocol

1. Print the last two prompt tokens, first two response tokens, associated labels, and supervision flags.
2. Compare vectorized cross entropy with a manual loop on a tiny batch.
3. Change every ignored label to a different valid identifier and prove the masked loss is unchanged.
4. Append padding and confirm both numerator and valid-token denominator remain unchanged.
5. Create an all-ignored batch and require an immediate descriptive error.
6. Run backward and check every trainable gradient for finite values; also record the number of supervised tokens per batch.

Part 4 - Explicit PyTorch implementation

```
import torch
import torch.nn.functional as F

def prepare_shifted_tensors(logits, labels):
    prediction_logits = logits[:, :-1, :].contiguous()
    next_token_labels = labels[:, 1:].contiguous()
    if prediction_logits.shape[:2] != next_token_labels.shape:
```

```
        raise ValueError("logit and label positions are misaligned")
    return prediction_logits, next_token_labels

def response_only_sft_loss(logits, labels):
    prediction_logits, next_token_labels = prepare_shifted_tensors(
        logits, labels
    )
    valid_mask = next_token_labels != -100
    valid_count = int(valid_mask.sum().item())
    if valid_count == 0:
        raise ValueError("batch contains no supervised response token")
    vocabulary_size = prediction_logits.shape[-1]
    flat_logits = prediction_logits.reshape(-1, vocabulary_size)
    flat_labels = next_token_labels.reshape(-1)
    flat_losses = F.cross_entropy(
        flat_logits, flat_labels, ignore_index=-100, reduction="none"
    )
    token_losses = flat_losses.view_as(next_token_labels)
    loss_sum = token_losses.masked_select(valid_mask).sum()
    loss_mean = loss_sum / valid_count
    return loss_mean, loss_sum.detach(), valid_count, token_losses

def E02(model, input_ids, labels):
    output = model(input_ids=input_ids, use_cache=False)
    loss, loss_sum, valid_count, token_losses = response_only_sft_loss(
        output.logits, labels
    )
    loss.backward()
    for parameter_name, parameter in model.named_parameters():
        if parameter.requires_grad and parameter.grad is not None:
            if not torch.isfinite(parameter.grad).all():
                raise RuntimeError("non-finite gradient: " + parameter_name)
    return loss.detach(), loss_sum, valid_count, token_losses.detach()
```

Part 5 - The Philosophy Behind the Exercise

The loss mask is the boundary between context and judgment. The prompt may explain the task and influence every response state, yet a response-only mask says that the model is evaluated for what it answers, not for recreating the user's words. This is a precise statement about responsibility inside the training sequence.

Once this perspective is understood, masking stops being a technical nuisance. It becomes the mechanism by which designers decide which tokens deserve learning pressure. A silent off-by-one error is therefore not merely a bug; it changes the behavior the training process is authorized to produce.

18.5.4 Exercise 03

Exercise Title: Fine-Tune a Tiny LLM on Instructions and Compare Behavior Before vs After

Part 1 - Deep conceptual foundation

A successful fine-tuning demonstration needs more evidence than a falling training loss. The model must produce measurably better instruction-conditioned behavior on held-out examples, and the comparison must use the same tokenizer, decoding configuration, prompts, random seeds, and evaluation functions. Without these controls, a sampling difference can be mistaken for a learning effect.

The most informative small experiment begins with a tiny model and a synthetic task whose correct outputs are unambiguous. Include several operations such as copying, reversing, extracting a field, and returning a class label. Split by underlying content, not by rendered string, so template paraphrases cannot leak the answer. Record base-model outputs before any optimizer step, then train until the model can deliberately overfit the training subset. Overfitting here is a diagnostic: it proves the data, loss, gradients, and decoding path can work.

Evaluation should expose the difference between memorization and rule learning. Use four partitions: training examples, held-out inputs under the familiar template, paraphrased instructions, and unrelated language-modeling sequences. Improvement limited to the first partition indicates memorization. Improvement on held-out inputs but collapse on unrelated text indicates specialization with forgetting. Both observations are more useful than one average score.

You will save an immutable baseline artifact containing prompts, generations, token log-probabilities, and configuration. After training, rerun exactly the same evaluator. The report must show paired examples and aggregate metrics, including failures, rather than presenting only the best generations.

Part 2 - Experimental quantities

Let a_0 and a_1 be instruction accuracy before and after tuning, and b_0, b_1 be retained base-task scores. Define

$$\Delta_{\text{instruction}} = a_1 - a_0, \qquad \Delta_{\text{retention}} = b_1 - b_0.$$

The tuning result is not fully described by maximizing the first value. A simple constrained decision rule is

$$\max \Delta_{\text{instruction}} \quad \text{subject to} \quad \Delta_{\text{retention}} \geq -\epsilon.$$

Report confidence intervals across examples or seeds when the evaluation set is small.

Part 3 - Controlled experiment design

1. Clone the initial state dictionary and hash it before training.
2. Fix greedy decoding for deterministic before/after examples; evaluate sampling separately.
3. Log train and validation token loss, exact task accuracy, response format accuracy, and retained base loss at each evaluation interval.
4. Save the first checkpoint that passes the validation criterion rather than automatically choosing the final epoch.
5. Inspect at least ten paired failures and classify them as formatting, instruction misunderstanding, incorrect content, truncation, or decoding.

6. Repeat with multiple seeds to distinguish a reliable tuning effect from a lucky initialization or batch order.

Part 4 - Explicit PyTorch training experiment

```
import copy
import torch

def train_one_epoch(model, data_loader, optimizer, device):
    model.train()
    total_loss_sum = 0.0
    total_tokens = 0
    for batch in data_loader:
        input_ids = batch["input_ids"].to(device)
        labels = batch["labels"].to(device)
        optimizer.zero_grad(set_to_none=True)
        logits = model(input_ids=input_ids, use_cache=False).logits
        loss, loss_sum, valid_count, unused = response_only_sft_loss(
            logits, labels
        )
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        total_loss_sum = total_loss_sum + float(loss_sum.item())
        total_tokens = total_tokens + valid_count
    return total_loss_sum / total_tokens

def evaluate_loss(model, data_loader, device):
    model.eval()
    total_loss_sum = 0.0
    total_tokens = 0
    with torch.no_grad():
        for batch in data_loader:
            input_ids = batch["input_ids"].to(device)
            labels = batch["labels"].to(device)
            logits = model(input_ids=input_ids, use_cache=False).logits
            loss, loss_sum, valid_count, unused = response_only_sft_loss(
                logits, labels
            )
            total_loss_sum = total_loss_sum + float(loss_sum.item())
            total_tokens = total_tokens + valid_count
    return total_loss_sum / total_tokens

def E03(base_model, train_loader, validation_loader,
        device, epochs, learning_rate):
    tuned_model = copy.deepcopy(base_model).to(device)
    optimizer = torch.optim.AdamW(
        tuned_model.parameters(), lr=learning_rate
```

```
    )
    history = []
    for epoch_index in range(epochs):
        train_loss = train_one_epoch(
            tuned_model, train_loader, optimizer, device
        )
        validation_loss = evaluate_loss(
            tuned_model, validation_loader, device
        )
        record = {}
        record["epoch"] = epoch_index
        record["train_loss"] = train_loss
        record["validation_loss"] = validation_loss
        history.append(record)
    return tuned_model, history
```

Part 5 - The Philosophy Behind the Exercise

Before-and-after evaluation turns fine-tuning from a story into a causal claim. The untouched checkpoint represents what would have happened without the intervention; the tuned checkpoint represents the new behavior. Fair controls allow the difference to be attributed to training rather than presentation.

The experiment also teaches humility. A model can become dramatically better at the measured task while becoming narrower elsewhere. Progress is a vector, not a single number, and responsible fine-tuning records both the ability gained and the flexibility spent to obtain it.

18.5.5 Exercise 04

Exercise Title: Analyze Overfitting Risk in Instruction Tuning with Small Datasets

Part 1 - Deep conceptual foundation

Small instruction datasets can change behavior quickly because every example is revisited many times. This makes them useful for controlled adaptation but also creates three forms of overfitting. Content overfitting memorizes particular answers. Template overfitting learns superficial role markers and wording. Behavioral overfitting applies the tuned style too broadly, even when the task requires a different response mode.

Training loss alone cannot separate these outcomes. A model may achieve almost zero loss by reproducing seen responses while failing on new values, paraphrased instructions, longer inputs, or a different order of fields. It may also lose general capabilities because repeated gradients from a narrow distribution move weights away from the pretrained solution. The experiment must therefore include a generalization grid, not one validation set drawn from the same template generator.

Construct training-size conditions such as 16, 64, 256, and 1024 examples. Hold optimizer steps or processed tokens constant when asking about dataset size; otherwise the smallest set receives more repetitions and the comparison mixes two causes. Evaluate checkpoints throughout training because the best generalization can occur before the minimum training loss.

The strongest evidence includes learning curves, train-validation gaps, exact duplicate checks, paraphrase performance, retained base loss, and output diversity. Measure variation across random seeds. A result that appears only under one seed is not a reliable property of the tuning recipe.

Part 2 - Mathematical diagnosis

For training risk $\widehat{R}_{\text{train}}$ and evaluation risk $\widehat{R}_{\text{eval}}$, define the observed gap

$$G = \widehat{R}_{\text{eval}} - \widehat{R}_{\text{train}}.$$

For an accuracy metric the sign is reversed, so always state the convention. To measure forgetting relative to the base model,

$$F = R_{\text{base-task}}(\theta^*) - R_{\text{base-task}}(\theta_0).$$

An early-stop score can combine distinct gates without hiding them:

$$S = A_{\text{heldout}} - \lambda F, \quad \text{subject to} \quad A_{\text{paraphrase}} \geq \tau.$$

Keep the component metrics visible even if a scalar is used for selection.

Part 3 - Experiment and failure analysis

1. Deduplicate before splitting and group variants of the same semantic task into one split.
2. Create evaluations for seen templates, unseen templates, unseen values, longer sequences, adversarial wording, and retained base behavior.
3. Match processed response tokens across dataset-size conditions.
4. Save checkpoints at frequent intervals and select using validation data, not the test set.
5. Plot training loss against every generalization metric to expose the point where further fitting begins to hurt.
6. Compare dropout, weight decay, fewer epochs, mixed-domain replay, and lower-capacity adapters as separate interventions.
7. Report means and dispersion across at least three seeds.

Part 4 - Explicit overfitting monitor

```
class FineTuningMonitor:
    def __init__(self):
        self.records = []

    def add(self, step, train_loss, validation_loss,
            paraphrase_accuracy, base_loss):
        record = {}
        record["step"] = step
        record["train_loss"] = train_loss
        record["validation_loss"] = validation_loss
```

```
        record["gap"] = validation_loss - train_loss
        record["paraphrase_accuracy"] = paraphrase_accuracy
        record["base_loss"] = base_loss
        self.records.append(record)

    def best_record(self, maximum_base_loss_increase, initial_base_loss):
        best = None
        for record in self.records:
            increase = record["base_loss"] - initial_base_loss
            if increase <= maximum_base_loss_increase:
                if best is None:
                    best = record
                else:
                    current_score = record["paraphrase_accuracy"]
                    best_score = best["paraphrase_accuracy"]
                    if current_score > best_score:
                        best = record
        return best

def E04(training_function, evaluation_function, conditions):
    results = []
    for condition in conditions:
        model, checkpoints = training_function(condition)
        condition_result = {}
        condition_result["condition"] = condition
        condition_result["checkpoints"] = []
        for checkpoint in checkpoints:
            metrics = evaluation_function(checkpoint)
            condition_result["checkpoints"].append(metrics)
        results.append(condition_result)
    return results
```

Part 5 - The Philosophy Behind the Exercise

Overfitting is not simply learning too much; it is learning the accidental surface of limited evidence more strongly than the transferable structure beneath it. Instruction data is especially vulnerable because repeated wording can make style look like capability.

The philosophical discipline is to reward transfer, not familiarity. A model should earn our confidence by succeeding when names, values, phrasing, and contexts change. The gap between recognition and generalization is where a fine-tuning experiment becomes scientific.

18.5.6 Exercise 05

Exercise Title: Implement Prompt Templates and Measure Sensitivity to Wording Changes

Part 1 - Deep conceptual foundation

Prompt templates serialize an abstract interaction into tokens. Phrases such as “Instruction”, XML-style role tags, chat special tokens, whitespace, and response prefixes are not invisible wrappers. They become part of the model’s conditioning context and can acquire

strong statistical associations during fine-tuning. A model trained on one rigid template may perform well only when the deployment request resembles that surface form.

Sensitivity must be measured by preserving semantic intent while changing presentation. Create paraphrases, reorder harmless fields, vary politeness, change separators, remove redundant instructions, and introduce irrelevant but benign context. Do not call arbitrary string corruption robustness: a malformed prompt can genuinely change meaning. Each variant needs a declared semantic equivalence judgment.

Templates must be versioned and shared between training and inference. A missing assistant prefix or different end marker can cause large behavioral changes even when the user text is identical. The tokenizer also matters: visually small whitespace changes may alter multiple subword boundaries. Store rendered token identifiers in diagnostic logs so the investigation can move beyond string appearance.

This exercise builds a renderer registry and an invariance evaluation. Each underlying example is rendered through multiple valid templates. The model is scored on content, format, refusal behavior, response length, and log probability. The unit of analysis is the semantic example, allowing variance across templates to be measured within the same task.

Part 2 - Sensitivity measures

For semantic item i and templates $k = 1, \ldots, K$, let $a_{i,k}$ be content accuracy. Define item sensitivity

$$S_i = \max_k a_{i,k} - \min_k a_{i,k},$$

and log-probability dispersion

$$V_i = \frac{1}{K} \sum_k \left(\ell_{i,k} - \bar{\ell}_i \right)^2 .$$

For binary accuracy, also report the fraction of items whose prediction changes under at least one semantics-preserving variant. Low average sensitivity is not enough if failures cluster in a critical task family.

Part 3 - Robustness protocol

1. Define a canonical structured record before rendering any string.
2. Implement at least four template families and give each an immutable version identifier.
3. Verify that every renderer preserves instruction, context, and expected response semantics.
4. Tokenize each variant and record token count, boundary index, and special tokens.
5. Evaluate paired variants with deterministic decoding and a content-aware grader.
6. Fine-tune once on one template and once on template mixtures under equal token budgets.
7. Identify whether robustness gains come from diverse wording or simply more examples.

Part 4 - Explicit template and evaluation code

```
def render_plain(record):
    text = "Instruction:\n" + record["instruction"]
    text = text + "\nResponse:\n"
    return text

def render_chat(record):
    text = "<user>" + record["instruction"] + "</user>\n"
    text = text + "<assistant>"
    return text

def render_question(record):
    text = "Please complete the following task.\n"
    text = text + record["instruction"] + "\nAnswer:"
    return text

def build_renderers():
    renderers = {}
    renderers["plain_v1"] = render_plain
    renderers["chat_v1"] = render_chat
    renderers["question_v1"] = render_question
    return renderers

def evaluate_prompt_sensitivity(records, generate, score):
    renderers = build_renderers()
    results = []
    for record in records:
        item = {}
        item["record_id"] = record["record_id"]
        item["variants"] = []
        for template_name in renderers:
            prompt = renderers[template_name](record)
            response = generate(prompt)
            metric = score(response, record["expected"])
            variant = {}
            variant["template"] = template_name
            variant["prompt"] = prompt
            variant["response"] = response
            variant["score"] = metric
            item["variants"].append(variant)
        results.append(item)
    return results

def E05(records, generate, score):
    return evaluate_prompt_sensitivity(records, generate, score)
```

Part 5 - The Philosophy Behind the Exercise

Language allows the same intention to wear many forms. A model that understands only the familiar costume has learned a password, not an instruction. Prompt robustness therefore

tests whether behavior is attached to meaning or to a particular ritual of words.

At the same time, complete invariance is neither possible nor desirable because wording can genuinely change meaning. The goal is disciplined sensitivity: stable under irrelevant variation, responsive to meaningful distinctions, and honest about uncertainty when the distinction is ambiguous.

18.5.7 Exercise 06

Exercise Title: Compare Multi-Task vs Single-Task Instruction Tuning

Part 1 - Deep conceptual foundation

Single-task tuning concentrates every update on one behavior. It can reach high accuracy rapidly, but its gradients may specialize the shared representation so strongly that other capabilities deteriorate. Multi-task instruction tuning mixes task families and can encourage reusable instruction-following patterns, yet poorly chosen mixtures allow large or easy tasks to dominate.

A fair comparison cannot use the same number of epochs because an epoch has a different token count for different mixtures. Match total supervised tokens, optimizer steps, model initialization, and evaluation schedule. Decide whether sampling weights operate at record level, sequence level, or response-token level. Long-response tasks receive greater influence under token-mean loss even if records are sampled equally.

Positive transfer occurs when adding other tasks improves a target task. Negative transfer occurs when competing gradients hurt it. Average accuracy can hide both: improvements on common easy tasks may outweigh severe degradation on a rare critical task. Measure each task, macro average, worst-task score, and retained base capability. Inspect per-task gradient similarity on selected batches to understand interaction rather than merely observing it.

The exercise compares a specialist, a uniform multi-task model, and a balanced mixture model. The balanced strategy may use temperature sampling or explicit task quotas. The conclusion must state which deployment goal is being optimized: peak specialist quality, broad competence, robustness to new task wording, or cost-efficient shared serving.

Part 2 - Mixture and gradient reasoning

For task distributions D_k and weights w_k,

$$L(\theta) = \sum_{k=1}^{K} w_k L_k(\theta), \qquad w_k \geq 0, \quad \sum_k w_k = 1.$$

If raw dataset sizes are n_k, temperature sampling can use

$$w_k = \frac{n_k^\alpha}{\sum_j n_j^\alpha},$$

where $\alpha = 1$ follows data frequency and $\alpha = 0$ samples tasks uniformly. For gradients g_i and g_j, cosine similarity

$$c_{ij} = \frac{g_i^\top g_j}{\|g_i\|\|g_j\|}$$

is diagnostic evidence of local agreement or conflict, not a permanent property of the tasks.

Part 3 - Matched comparison

1. Create at least four task families with independent train, validation, and test content.
2. Train single-task baselines and multi-task models from the same initial checkpoint.
3. Match supervised response-token count and optimization schedule.
4. Report per-task accuracy, macro average, worst-task score, template robustness, and retained base loss.
5. Record realized sampling proportions instead of trusting configured weights.
6. Inspect gradient cosine similarity on fixed probe batches at several training steps.
7. Vary α and explain which tasks gain or lose as the mixture changes.

Part 4 - Explicit task sampler

```
import math
import random

class TemperatureTaskSampler:
    def __init__(self, task_records, alpha, seed):
        self.task_records = task_records
        self.random = random.Random(seed)
        self.task_names = []
        raw_weights = []
        for task_name in task_records:
            self.task_names.append(task_name)
            count = len(task_records[task_name])
            raw_weights.append(math.pow(count, alpha))
        total = sum(raw_weights)
        self.weights = []
        for raw_weight in raw_weights:
            self.weights.append(raw_weight / total)

    def sample_task(self):
        value = self.random.random()
        cumulative = 0.0
        index = 0
        while index < len(self.task_names):
            cumulative = cumulative + self.weights[index]
            if value <= cumulative:
                return self.task_names[index]
            index = index + 1
        return self.task_names[-1]

    def sample_record(self):
        task_name = self.sample_task()
        records = self.task_records[task_name]
        record_index = self.random.randrange(len(records))
```

```
        return task_name, records[record_index]

def E06(task_records, alpha, seed, number_of_samples):
    sampler = TemperatureTaskSampler(task_records, alpha, seed)
    realized_counts = {}
    samples = []
    for sample_index in range(number_of_samples):
        task_name, record = sampler.sample_record()
        samples.append(record)
        if task_name not in realized_counts:
            realized_counts[task_name] = 0
        realized_counts[task_name] = realized_counts[task_name] + 1
    return samples, realized_counts
```

Part 5 - The Philosophy Behind the Exercise

Multi-task training asks whether competence can be shared. When tasks reinforce one another, the model learns abstractions broader than any single dataset. When they conflict, the final behavior reflects a negotiated balance imposed by sampling weights and gradients.

There is no neutral mixture. Giving a task more probability is giving its errors more authority over the model. The engineering responsibility is to make that allocation explicit and evaluate the weakest important capability, not merely celebrate a rising average.

18.5.8 Exercise 07

Exercise Title: Implement LoRA Conceptually and Explain Parameter-Efficiency Benefits

Part 1 - Deep conceptual foundation

Full fine-tuning stores gradients and optimizer state for every trainable base parameter. Low-Rank Adaptation, or LoRA, freezes a selected weight matrix W and learns an additive update represented by two much smaller matrices. Instead of assuming the task-specific update needs all $d_{\text{out}}d_{\text{in}}$ degrees of freedom, LoRA constrains it to a rank-r subspace.

This is a hypothesis about adaptation geometry, not a magical compression of the entire training process. It substantially reduces trainable parameters and optimizer state, but activations from the base network still consume memory and must participate in backward computation so gradients reach the adapters. Sequence length, batch size, and checkpointing therefore remain important.

The frozen layer computes $xW^\top$. The adapter adds a down projection from d_{in} to r, followed by an up projection to d_{out}. Initialize the up projection to zero so the wrapped layer begins exactly equal to the base layer; initialize the down projection with a small random distribution so both factors can eventually learn. If both factors start at zero, their gradients can remain trapped.

You will implement the wrapper explicitly, freeze base parameters, verify exact initial parity, count trainable parameters, run backward, and prove that only adapter matrices receive gradients. Then merge the update into a copied base weight and verify output parity for deployment.

Part 2 - Low-rank parameterization

For $W \in \mathbb{R}^{d_o \times d_i}$,

$$W' = W + \Delta W, \qquad \Delta W = \frac{\alpha}{r} BA,$$

where $A \in \mathbb{R}^{r \times d_i}$ and $B \in \mathbb{R}^{d_o \times r}$. The adapter parameter count is

$$P_{\text{LoRA}} = r(d_i + d_o),$$

compared with $P_{\text{full}} = d_i d_o$. The ratio predicts parameter-state savings, not total runtime-memory savings. Rank bounds the update: $\text{rank}(BA) \leq r$.

Part 3 - Required proofs and experiments

1. Wrap one linear layer and assert bitwise or tight numerical parity before training because $B = 0$.
2. Print every parameter with its shape, trainable status, and gradient presence after backward.
3. Compare theoretical and measured trainable parameter counts.
4. Test several ranks under matched optimization tokens. Report quality against adapter size for ranks 1, 4, 8, and 16.
5. Apply LoRA separately to attention projections and feed-forward layers to measure placement sensitivity.
6. Merge BA into a copied weight, disable the adapter path, and verify inference parity.
7. State whether biases and normalization parameters are frozen or trained.

Part 4 - LoRA implemented from first principles

```
import math
import torch
import torch.nn as nn
import torch.nn.functional as F

class LoRALinear(nn.Module):
    def __init__(self, base_layer, rank, alpha):
        super().__init__()
        if rank <= 0:
            raise ValueError("rank must be positive")
        self.base_layer = base_layer
        self.rank = rank
        self.alpha = alpha
        for parameter in self.base_layer.parameters():
            parameter.requires_grad = False
        input_size = base_layer.in_features
        output_size = base_layer.out_features
        self.adapter_down = nn.Parameter(
            torch.empty(rank, input_size)
```

```
        )
        self.adapter_up = nn.Parameter(
            torch.zeros(output_size, rank)
        )
        nn.init.kaiming_uniform_(self.adapter_down, a=math.sqrt(5.0))

    def forward(self, inputs):
        base_output = self.base_layer(inputs)
        hidden = F.linear(inputs, self.adapter_down)
        update = F.linear(hidden, self.adapter_up)
        scale = self.alpha / self.rank
        return base_output + scale * update

    def merged_weight(self):
        scale = self.alpha / self.rank
        update = torch.matmul(self.adapter_up, self.adapter_down)
        return self.base_layer.weight.detach() + scale * update.detach()

def count_parameters(module):
    total = 0
    trainable = 0
    for parameter in module.parameters():
        total = total + parameter.numel()
        if parameter.requires_grad:
            trainable = trainable + parameter.numel()
    return total, trainable

def E07(linear_layer, inputs, rank, alpha):
    with torch.no_grad():
        reference = linear_layer(inputs)
    wrapped = LoRALinear(linear_layer, rank, alpha)
    initial = wrapped(inputs)
    assert torch.allclose(reference, initial, atol=1e-6, rtol=1e-6)
    total, trainable = count_parameters(wrapped)
    return wrapped, total, trainable
```

Part 5 - The Philosophy Behind the Exercise

LoRA expresses a powerful engineering wager: adapting behavior may require a directional correction rather than rewriting the model's entire memory. The low-rank constraint turns that wager into a measurable hypothesis about how many independent directions the new task needs.

Efficiency should not be romanticized. Saving optimizer state does not erase data quality, evaluation, activation cost, or forgetting. LoRA is valuable because it makes experiments cheaper and adaptations modular, while still requiring the same intellectual discipline about what the model is being taught.

18.5.9 Exercise 08

Exercise Title: Fine-Tune Using LoRA and Compare Quality vs Full Fine-Tuning

Part 1 - Deep conceptual foundation

Comparing LoRA with full fine-tuning requires more than reporting trainable parameter counts. The methods have different capacity, memory behavior, optimal learning rates, and regularization effects. A fair study begins from the same base checkpoint, uses identical data order and token budget, tunes each method within a declared search budget, and evaluates both target quality and retained capabilities.

Full fine-tuning can express any update to eligible weights, but that flexibility may overfit small data or produce greater forgetting. LoRA restricts updates and often tolerates a larger learning rate, yet too-small rank or poor module placement can create an adaptation bottleneck. Equal learning rates are not necessarily fair; equal hyperparameter-search resources are more defensible.

Measure peak allocated GPU memory after resetting statistics and synchronizing CUDA. Separate model parameters, gradients, optimizer state, and activations in the explanation. Measure throughput after warm-up steps because first-iteration kernel setup distorts results. At inference, compare the unmerged adapter path with a merged checkpoint to determine whether adapter overhead matters.

Quality evaluation should include exact instruction accuracy, semantic scoring, prompt-variant robustness, rare task families, and base-model retention. Report the Pareto frontier: one method may win quality while the other wins memory and storage. Avoid forcing a single winner when deployment constraints differ.

Part 2 - Comparison quantities

For method m, define an efficiency vector

$$E_m = (Q_m, R_m, M_m, U_m, T_m, S_m),$$

where Q is target quality, R retention, M peak memory, U updates per second, T trainable parameters, and S checkpoint size. A useful relative memory saving is

$$\rho_M = 1 - \frac{M_{\text{LoRA}}}{M_{\text{full}}}.$$

It must be measured for the same batch, sequence length, precision, and checkpointing setting.

Part 3 - Matched benchmark design

1. Clone one base checkpoint into independent full and LoRA runs.
2. Search learning rate for both methods and rank plus alpha for LoRA under equal trial budgets.
3. Match processed supervised tokens and evaluation intervals.
4. Warm up CUDA, reset peak-memory statistics, synchronize around timing, and repeat measurements.

5. Evaluate training data, held-out content, paraphrased prompts, and base retention with the same decoding configuration.

6. Save LoRA adapters separately, merge a copy, and verify merged-logit parity.

7. Report means across seeds and a table of quality, memory, speed, and checkpoint storage.

Part 4 - Explicit benchmark instrumentation

```
import time
import torch

def trainable_parameter_count(model):
    count = 0
    for parameter in model.parameters():
        if parameter.requires_grad:
            count = count + parameter.numel()
    return count

def benchmark_training_steps(model, optimizer, batches, device):
    model.train()
    if device.type == "cuda":
        torch.cuda.empty_cache()
        torch.cuda.reset_peak_memory_stats(device)
        torch.cuda.synchronize(device)
    start_time = time.perf_counter()
    processed_tokens = 0
    for batch in batches:
        input_ids = batch["input_ids"].to(device)
        labels = batch["labels"].to(device)
        optimizer.zero_grad(set_to_none=True)
        logits = model(input_ids=input_ids, use_cache=False).logits
        loss, unused_sum, valid_count, unused_losses = (
            response_only_sft_loss(logits, labels)
        )
        loss.backward()
        optimizer.step()
        processed_tokens = processed_tokens + valid_count
    if device.type == "cuda":
        torch.cuda.synchronize(device)
        peak_memory = torch.cuda.max_memory_allocated(device)
    else:
        peak_memory = 0
    elapsed = time.perf_counter() - start_time
    result = {}
    result["tokens_per_second"] = processed_tokens / elapsed
    result["peak_memory_bytes"] = peak_memory
    result["trainable_parameters"] = trainable_parameter_count(model)
```

```
    return result

def E08(full_model, lora_model, full_optimizer,
        lora_optimizer, batches, device):
    full_result = benchmark_training_steps(
        full_model, full_optimizer, batches, device
    )
    lora_result = benchmark_training_steps(
        lora_model, lora_optimizer, batches, device
    )
    return full_result, lora_result
```

Part 5 - The Philosophy Behind the Exercise

Parameter efficiency changes who can experiment. A method that preserves most quality while reducing optimizer memory and checkpoint size can make adaptation available on hardware that cannot support full fine-tuning. That is a practical expansion of research access.

But efficiency is meaningful only relative to a purpose. A smaller adapter is not better if it fails the critical task, and a tiny quality gain may not justify an enormous operational cost. The comparison teaches decision-making across several dimensions rather than devotion to one fashionable technique.

18.5.10 Exercise 09

Exercise Title: Debug Catastrophic Forgetting After Fine-Tuning

Part 1 - Deep conceptual foundation

Catastrophic forgetting occurs when adaptation to a new distribution damages previously useful behavior. In LLM fine-tuning it can appear as increased base language-model loss, degraded general knowledge, loss of multilingual ability, excessive use of the new response style, or failure on instructions outside the tuned domain. The model has not literally erased a discrete database; parameter updates changed shared representations and conditional probabilities.

Diagnosis requires a frozen pre-tuning baseline and a retained evaluation suite that spans capabilities you care about. Run this suite before training and at regular checkpoints. A final comparison cannot reveal when forgetting began or whether early stopping could have prevented it. Track per-domain changes because an overall mean can conceal collapse in a small language or task family.

Common causes include excessive learning rate, too many repeated epochs, a narrow dataset, unbalanced response lengths, full-model updates, and optimizer state that produces large early movement. Remedies include lower learning rate, fewer steps, LoRA, mixed replay from the base distribution, stronger regularization toward the initial weights, and a more diverse instruction mixture. Each remedy must be tested against new-task quality; preventing all movement also prevents learning.

This exercise uses parameter drift and functional drift together. Parameter distance shows how far weights moved, while KL divergence and retained metrics show whether behavior changed. A large weight movement can be harmless under symmetries; a small movement in a sensitive subspace can matter greatly.

Part 2 - Drift and retention mathematics

For original parameters θ_0 and tuned parameters θ,

$$D_\theta = \frac{\|\theta - \theta_0\|_2}{\|\theta_0\|_2}.$$

On retained prompts x, functional drift can be measured as

$$D_{\mathrm{KL}} = \mathbb{E}_{x,t}\left[\mathrm{KL}\left(p_{\theta_0}(\cdot \mid x_{\le t}) \parallel p_\theta(\cdot \mid x_{\le t})\right)\right].$$

A replay objective is

$$L = L_{\mathrm{new}} + \lambda L_{\mathrm{replay}},$$

with λ selected using both adaptation and retention gates.

Part 3 - Root-cause investigation

1. Establish base scores for language modeling, instruction following, languages, and domain tasks before tuning.
2. Evaluate every checkpoint on new-task and retained suites.
3. Measure layer-wise relative parameter drift and identify where movement concentrates.
4. Compare output KL divergence on fixed retained prompts.
5. Reproduce the failure with fixed data order and seed before changing the recipe.
6. Test lower learning rate, early stopping, LoRA, replay, and weight anchoring one at a time.
7. Declare acceptable forgetting thresholds in advance and reject a model that violates any critical gate.

Part 4 - Explicit forgetting diagnostics

```
import torch
import torch.nn.functional as F

def relative_parameter_drift(base_model, tuned_model):
    base_parameters = dict(base_model.named_parameters())
    drifts = {}
    for name, tuned_parameter in tuned_model.named_parameters():
        base_parameter = base_parameters[name].to(tuned_parameter.device)
        numerator = torch.linalg.vector_norm(
            tuned_parameter.detach() - base_parameter.detach()
        )
        denominator = torch.linalg.vector_norm(base_parameter.detach())
        denominator = torch.clamp(denominator, min=1e-12)
        drifts[name] = float((numerator / denominator).item())
    return drifts
```

```
def output_kl_divergence(base_logits, tuned_logits, valid_mask):
    base_log_probabilities = F.log_softmax(base_logits, dim=-1)
    tuned_log_probabilities = F.log_softmax(tuned_logits, dim=-1)
    base_probabilities = torch.exp(base_log_probabilities)
    token_kl = torch.sum(
        base_probabilities *
        (base_log_probabilities - tuned_log_probabilities),
        dim=-1
    )
    return token_kl.masked_select(valid_mask).mean()

def E09(base_model, tuned_model, retained_input_ids, valid_mask):
    base_model.eval()
    tuned_model.eval()
    with torch.no_grad():
        base_logits = base_model(retained_input_ids).logits
        tuned_logits = tuned_model(retained_input_ids).logits
        functional_drift = output_kl_divergence(
            base_logits, tuned_logits, valid_mask
        )
    parameter_drifts = relative_parameter_drift(base_model, tuned_model)
    return functional_drift, parameter_drifts
```

Part 5 - The Philosophy Behind the Exercise

Learning and forgetting are two views of the same movement. To become more responsive to new evidence, a finite model must redistribute capacity and probability. The engineering question is not whether weights change, but whether the change sacrifices abilities the system still promises to provide.

Retention tests make those promises explicit. They remind us that improvement has a history: a new specialist behavior should be judged together with the knowledge, languages, and flexibility inherited from the base model.

18.5.11 Exercise 10

Exercise Title: Run Instruction Fine-Tuning on CUDA and Profile Memory vs Throughput

Part 1 - Deep conceptual foundation

CUDA fine-tuning performance is governed by more than model size. Sequence length controls attention and activation cost, response-only masking controls how many tokens contribute to useful gradients, padding controls wasted computation, and optimizer choice controls persistent state. A run can show high GPU utilization while training mostly on padding, so utilization must be interpreted beside supervised tokens per second.

Track each major memory category separately. The persistent categories include parameters, gradients, and optimizer states. Activations, temporary workspaces, and allocator reservations require separate measurements. Allocated memory is not the same as reserved memory, and neither value equals device-wide usage. Reset peak statistics before measurement. Warm up the model and synchronize around wall-clock timing, but not inside every measured step.

Benchmark a matrix of batch size, sequence length, precision, gradient checkpointing, and full versus LoRA fine-tuning. Keep actual token shapes fixed inside each condition. Record total tokens and supervised response tokens; the latter reflects useful SFT work when prompts are long. Include time spent in data transfer if the goal is end-to-end throughput, or exclude it explicitly if measuring kernels only.

The exercise will build a device-safe step, measure peaks, detect out-of-memory conditions, and write a result for every attempted configuration. Failed conditions are evidence about the hardware boundary and must not disappear from the table.

Part 2 - Performance quantities

For elapsed time Δt, total input tokens N_{all}, and supervised tokens N_{sup},

$$Q_{\text{all}} = \frac{N_{\text{all}}}{\Delta t}, \qquad Q_{\text{sup}} = \frac{N_{\text{sup}}}{\Delta t}.$$

Padding efficiency is

$$\eta_{\text{pad}} = \frac{N_{\text{nonpad}}}{BT},$$

and response supervision density is $\eta_{\text{sup}} = N_{\text{sup}}/N_{\text{nonpad}}$. These explain why two batches with equal shapes can deliver different learning value.

Part 3 - Benchmark protocol

1. Verify model, inputs, labels, masks, and adapter parameters reside on the same CUDA device.
2. Run warm-up steps that are excluded from timing and memory reporting.
3. Reset peak statistics, synchronize, measure many steps, and synchronize once at the end.
4. Record allocated and reserved peaks, step time, total-token throughput, supervised-token throughput, and padding efficiency.
5. Repeat each condition after a clean model reconstruction to avoid allocator history bias.
6. Catch out-of-memory errors, record the failing configuration, release references, and continue only after safe cleanup.
7. Use a profiler on representative steps after the broad benchmark identifies interesting conditions.

Part 4 - Explicit CUDA measurement code

```
import time
import torch

def cuda_training_benchmark(model, optimizer, batches, device):
    if device.type != "cuda":
        raise ValueError("this benchmark requires a CUDA device")
    model.train()
```

```
    for warmup_index in range(2):
        batch = batches[warmup_index]
        input_ids = batch["input_ids"].to(device, non_blocking=True)
        labels = batch["labels"].to(device, non_blocking=True)
        optimizer.zero_grad(set_to_none=True)
        logits = model(input_ids=input_ids, use_cache=False).logits
        loss, unused_sum, unused_count, unused_losses = (
            response_only_sft_loss(logits, labels)
        )
        loss.backward()
        optimizer.step()

    torch.cuda.synchronize(device)
    torch.cuda.reset_peak_memory_stats(device)
    start = time.perf_counter()
    total_tokens = 0
    supervised_tokens = 0
    for batch_index in range(2, len(batches)):
        batch = batches[batch_index]
        input_ids = batch["input_ids"].to(device, non_blocking=True)
        labels = batch["labels"].to(device, non_blocking=True)
        optimizer.zero_grad(set_to_none=True)
        logits = model(input_ids=input_ids, use_cache=False).logits
        loss, unused_sum, valid_count, unused_losses = (
            response_only_sft_loss(logits, labels)
        )
        loss.backward()
        optimizer.step()
        total_tokens = total_tokens + input_ids.numel()
        supervised_tokens = supervised_tokens + valid_count
    torch.cuda.synchronize(device)
    elapsed = time.perf_counter() - start
    result = {}
    result["all_tokens_per_second"] = total_tokens / elapsed
    result["supervised_tokens_per_second"] = supervised_tokens / elapsed
    result["peak_allocated"] = torch.cuda.max_memory_allocated(device)
    result["peak_reserved"] = torch.cuda.max_memory_reserved(device)
    return result

def E10(model, optimizer, batches):
    device = torch.device("cuda:0")
    model.to(device)
    return cuda_training_benchmark(model, optimizer, batches, device)
```

Part 5 - The Philosophy Behind the Exercise

Performance engineering asks what the hardware is spending effort on. A GPU can be busy without being useful, and a fast step can carry little supervision if most tokens are padding or prompt context. Measurement must therefore connect silicon activity to learning value.

The hardware limit is not an embarrassment to hide; it is part of the model's experimental world. By documenting where memory fails and throughput changes, we transform resource constraints from surprises into design knowledge.

18.5.12 Exercise 11

Exercise Title: Compare Learning Rates and Schedulers for Stable Instruction Tuning

Part 1 - Deep conceptual foundation

Fine-tuning begins from a model that already contains useful structure, so the learning rate controls both adaptation speed and preservation. A rate that is too small can leave the model near its base behavior; a rate that is too large can destroy useful representations within a few updates. The best value depends on whether all parameters or only adapters are trained, the effective batch size, sequence distribution, optimizer, and number of steps.

Warmup protects the earliest updates, when optimizer moments are not yet representative and batches can produce unusually large movement. Decay controls how the remaining update budget is distributed. Linear and cosine schedules can reach similar final rates while allocating different learning rates to middle steps. A scheduler must advance on optimizer updates, not micro-batches, when gradient accumulation is used.

A proper sweep records the actual learning rate, gradient norm before clipping, update norm, loss, validation score, and retention score. Final loss alone misses early spikes and recovery. Compare schedules with the same total optimizer updates and warmup definition. If a run diverges, record the first non-finite step and the preceding metrics.

This exercise implements warmup plus cosine decay explicitly so step semantics are transparent. It then compares several peak learning rates and scheduler families across fixed seeds, selecting a configuration only when it passes both adaptation and retention gates.

Part 2 - Schedule equations

For optimizer step s, warmup length W, and total steps S, a warmup-cosine multiplier is

$$m(s) = \begin{cases} \dfrac{s+1}{W}, & s < W, \\ \dfrac{1}{2}\left[1 + \cos\left(\pi \dfrac{s-W}{S-W}\right)\right], & s \geq W. \end{cases}$$

The learning rate is $\eta_s = \eta_{\max} m(s)$. Relative update magnitude can be monitored as

$$U_s = \frac{\|\theta_{s+1} - \theta_s\|_2}{\|\theta_s\|_2}.$$

Large spikes in U_s may reveal instability even while loss remains finite.

Part 3 - Controlled sweep

1. Choose logarithmically spaced peak learning rates separately for full and LoRA tuning.
2. Match total optimizer updates, warmup fraction, data order, and evaluation intervals.
3. Log the learning rate after every scheduler update and verify the first, peak, and final values manually.
4. Record unclipped gradient norm and parameter-update norm.

5. Evaluate held-out instructions, prompt variants, and retained base tasks at intermediate checkpoints.

6. Repeat promising conditions across seeds and reject configurations whose variance is unacceptable.

7. Explain whether the winning schedule improves optimization, retention, or both.

Part 4 - Explicit scheduler implementation

```
import math

class WarmupCosineSchedule:
    def __init__(self, optimizer, maximum_lr,
                 warmup_steps, total_steps):
        if warmup_steps < 1:
            raise ValueError("warmup_steps must be positive")
        if total_steps <= warmup_steps:
            raise ValueError("total_steps must exceed warmup_steps")
        self.optimizer = optimizer
        self.maximum_lr = maximum_lr
        self.warmup_steps = warmup_steps
        self.total_steps = total_steps
        self.step_index = 0

    def multiplier(self, step_index):
        if step_index < self.warmup_steps:
            return (step_index + 1) / self.warmup_steps
        progress_numerator = step_index - self.warmup_steps
        progress_denominator = self.total_steps - self.warmup_steps
        progress = progress_numerator / progress_denominator
        progress = min(progress, 1.0)
        return 0.5 * (1.0 + math.cos(math.pi * progress))

    def step(self):
        multiplier = self.multiplier(self.step_index)
        learning_rate = self.maximum_lr * multiplier
        for parameter_group in self.optimizer.param_groups:
            parameter_group["lr"] = learning_rate
        self.step_index = self.step_index + 1
        return learning_rate

def E11(optimizer, maximum_lr, warmup_steps, total_steps):
    schedule = WarmupCosineSchedule(
        optimizer, maximum_lr, warmup_steps, total_steps
    )
    learning_rates = []
    for step_index in range(total_steps):
```

```
        learning_rate = schedule.step()
        learning_rates.append(learning_rate)
    assert learning_rates[0] < maximum_lr
    assert learning_rates[warmup_steps - 1] == maximum_lr
    return learning_rates
```

Part 5 - The Philosophy Behind the Exercise

The learning rate is the scale at which new evidence is allowed to rewrite an existing model. Warmup begins cautiously, the peak permits adaptation, and decay gradually limits later disruption. A schedule is therefore a temporal policy for trust in gradients.

Stable tuning is not the absence of movement. It is movement whose magnitude is proportional to evidence and whose consequences are observed. The experiment teaches that optimization settings are part of the meaning of fine-tuning, not mere knobs attached after the data is chosen.

18.5.13 Exercise 12

Exercise Title: Implement Gradient Accumulation for Long Prompts Under Memory Constraints

Part 1 - Deep conceptual foundation

Long prompts increase activation memory, often forcing a small micro-batch. Gradient accumulation processes several micro-batches before one optimizer update, approximating a larger batch without storing all activations simultaneously. Correctness depends on the loss denominator. In response-only SFT, micro-batches may contain very different numbers of supervised tokens, so dividing each micro-batch mean by the number of accumulation steps is generally not equivalent to a global token mean.

The exact solution is to accumulate the sum of token losses and divide by the total number of supervised tokens across the update window. Because the total is not known until all micro-batches are inspected, one approach is to know valid counts from the collator, scale each loss sum by the window total, and backpropagate. Another approach accumulates unnormalized gradients then divides gradients before the optimizer step. The second is explicit and handles variable token counts cleanly.

Incomplete final windows need their own denominator; never divide by the planned number of micro-batches when fewer were processed. Gradient clipping must occur after normalization and, in mixed precision, after unscaling. Scheduler steps and global-step counters advance once per optimizer update, not once per micro-batch.

The experiment will compare accumulated gradients with a physically concatenated batch under dropout disabled and deterministic ordering. It will test unequal response lengths and an incomplete final window, then measure the memory and throughput tradeoff on CUDA.

Part 2 - Exact token-weighted equivalence

For micro-batch j with token-loss sum S_j and valid count n_j, the desired window loss is

$$L_{\text{window}} = \frac{\sum_{j=1}^{K} S_j}{\sum_{j=1}^{K} n_j}.$$

If $g_j = \nabla_\theta S_j$, the normalized accumulated gradient is

$$g = \frac{\sum_j g_j}{\sum_j n_j}.$$

Averaging micro-batch means instead gives each micro-batch equal weight and is incorrect when n_j differs.

Part 3 - Equivalence and stress tests

1. Disable dropout and use one immutable initial state for accumulated and concatenated comparisons.
2. Create micro-batches with deliberately unequal supervised-token counts.
3. Compare every trainable gradient before optimizer update using explicit tolerances.
4. Compare parameters after one update and explain remaining floating-point differences.
5. Test an incomplete final accumulation window.
6. Clip only after gradients have been normalized by the window token count.
7. Report peak memory, supervised tokens per second, and optimizer updates per second for several accumulation factors.

Part 4 - Token-correct accumulation code

```
import torch

def divide_gradients(model, denominator):
    if denominator <= 0:
        raise ValueError("gradient denominator must be positive")
    for parameter in model.parameters():
        if parameter.grad is not None:
            parameter.grad.div_(denominator)

def accumulated_update(model, optimizer, micro_batches,
                       device, maximum_gradient_norm):
    model.train()
    optimizer.zero_grad(set_to_none=True)
    total_valid_tokens = 0
    total_loss_value = 0.0
    for batch in micro_batches:
        input_ids = batch["input_ids"].to(device)
        labels = batch["labels"].to(device)
        logits = model(input_ids=input_ids, use_cache=False).logits
        mean_loss, loss_sum, valid_count, unused_losses = (
            response_only_sft_loss(logits, labels)
        )
        differentiable_loss_sum = mean_loss * valid_count
        differentiable_loss_sum.backward()
```

```
        total_valid_tokens = total_valid_tokens + valid_count
        total_loss_value = total_loss_value + float(loss_sum.item())

    divide_gradients(model, total_valid_tokens)
    gradient_norm = torch.nn.utils.clip_grad_norm_(
        model.parameters(), maximum_gradient_norm
    )
    optimizer.step()
    mean_window_loss = total_loss_value / total_valid_tokens
    return mean_window_loss, float(gradient_norm), total_valid_tokens

def E12(model, optimizer, micro_batches, device):
    return accumulated_update(
        model, optimizer, micro_batches, device, 1.0
    )
```

Part 5 - The Philosophy Behind the Exercise

Gradient accumulation separates when evidence is observed from when parameters are changed. Several small views of the data can jointly authorize one update, allowing limited memory to approximate a larger statistical decision.

But equivalence depends on whose tokens receive weight. If every micro-batch is treated equally despite different response lengths, convenience silently changes the objective. Resource-saving methods remain scientifically valid only when their mathematical meaning is preserved.

18.5.14 Exercise 13

Exercise Title: Validate Determinism and Reproducibility in Fine-Tuning Runs

Part 1 - Deep conceptual foundation

Reproducibility means another run can reconstruct the experimental conditions and obtain behavior within declared tolerances. Exact determinism is a narrower goal: repeating the same run produces identical intermediate and final values. CUDA kernels, floating-point reduction order, data-loader workers, dropout, sampling, library versions, and distributed communication can all introduce variation.

Seeds are necessary but insufficient. Record the base checkpoint hash, dataset manifest and order, tokenizer files, template version, code revision, hyperparameters, precision, hardware, drivers, PyTorch version, deterministic flags, and environment. Save optimizer and scheduler states if resuming. Changing the number of workers can change random augmentation or batching unless worker seeds are controlled.

Perform two modes. In strict mode, enable deterministic algorithms and fail when PyTorch encounters an unsupported nondeterministic operation. In realistic mode, use production performance settings and quantify variability across seeds. Exact equality is not always worth a large performance cost, but unmeasured variability is not acceptable.

Compare more than the final metric. Hash the data order, record early losses, compare parameter tensors, and evaluate paired outputs. If runs diverge at step zero, suspect initialization or data. If they diverge after a specific kernel, profile that region. If parameters differ slightly but behavior remains stable, report both numerical and functional reproducibility.

Part 2 - Reproducibility statistics

For metric values $q_1, \ldots, q_R$ across runs,

$$\overline{q} = \frac{1}{R}\sum_{r=1}^{R} q_r, \qquad s_q = \sqrt{\frac{1}{R-1}\sum_{r}(q_r - \overline{q})^2}.$$

For parameter vectors θ_a and θ_b, report

$$D_{a,b} = \frac{\|\theta_a - \theta_b\|_2}{\max(\|\theta_a\|_2, \epsilon)}.$$

Zero $D_{a,b}$ is expected only in a strict deterministic configuration. A functional tolerance must be tied to the intended use.

Part 3 - Reproduction protocol

1. Capture configuration, code, data, tokenizer, base checkpoint, software, and hardware identifiers before training.
2. Seed Python and PyTorch CPU/CUDA generators before model creation.
3. Configure data-loader generators and worker initialization explicitly.
4. Run two strict repetitions and compare batch identifiers, step losses, gradients, and final parameters.
5. Run at least three production-mode seeds and report metric dispersion and paired prediction disagreement.
6. Resume from a checkpoint and compare against an uninterrupted run at the same global step.
7. Document every setting that was relaxed for performance and its measured effect.

Part 4 - Explicit deterministic setup and comparison

```
import os
import random
import torch

def configure_reproducibility(seed, strict):
    random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
    if strict:
        torch.use_deterministic_algorithms(True)
        torch.backends.cudnn.benchmark = False
        os.environ["CUBLAS_WORKSPACE_CONFIG"] = ":4096:8"
    else:
        torch.use_deterministic_algorithms(False)
```

```
def maximum_parameter_difference(first_model, second_model):
    second_parameters = dict(second_model.named_parameters())
    maximum = 0.0
    maximum_name = None
    for name, first_parameter in first_model.named_parameters():
        second_parameter = second_parameters[name]
        difference = torch.max(torch.abs(
            first_parameter.detach() - second_parameter.detach()
        )).item()
        if difference > maximum:
            maximum = difference
            maximum_name = name
    return maximum, maximum_name

def E13(run_training, seed, strict):
    configure_reproducibility(seed, strict)
    first_model, first_history = run_training()
    configure_reproducibility(seed, strict)
    second_model, second_history = run_training()
    maximum, parameter_name = maximum_parameter_difference(
        first_model, second_model
    )
    result = {}
    result["maximum_parameter_difference"] = maximum
    result["parameter_name"] = parameter_name
    result["history_equal"] = first_history == second_history
    return result
```

Part 5 - The Philosophy Behind the Exercise

Reproducibility is the memory of an experiment. Without it, a result belongs to a vanished arrangement of code, data, hardware, and chance. Recording that arrangement allows knowledge to survive the original machine and researcher.

Determinism is not always the final goal, but accountability is. When results vary, the variation must become an observed quantity rather than an excuse. Science advances when another run can challenge the same claim under the same conditions.

18.5.15 Exercise 14

Exercise Title: Build Evaluation Set to Measure Instruction Following Accuracy

Part 1 - Deep conceptual foundation

Instruction following is multidimensional. A response can understand the task but violate a requested format, obey the format but contain incorrect content, answer correctly while ignoring a constraint, or refuse a valid request. One binary score cannot explain these cases. The evaluation set must convert the deployment contract into observable dimensions.

Start with a task taxonomy: extraction, transformation, classification, reasoning, constrained generation, tool-like structured output, context-grounded answering, clarification, and

refusal where appropriate. Within every family, vary difficulty, prompt wording, response length, languages, and distractors. Separate semantic templates and underlying content across splits so the test set does not reward memorized renderings.

Every item needs an identifier, provenance, instruction, optional context, reference information, scoring method, critical constraints, and failure severity. Exact match is suitable for strict labels and serialized outputs; numeric tolerance suits calculations; set or span scoring suits extraction; human or model-assisted rubrics may be needed for open responses. Automated graders must themselves be validated against human judgments on a sample.

Prevent evaluation contamination. Hash normalized examples and near-duplicate representations before training. Keep test prompts access-controlled and versioned. When an item changes, increment the suite version rather than silently altering history. Report confidence intervals and per-slice results, especially for safety-critical or underrepresented cases.

Part 2 - Metric decomposition

For each response define indicators C for content, F for format, K for constraint satisfaction, and R for appropriate refusal behavior. A strict success can be

$$I_{\text{success}} = C \cdot F \cdot K \cdot R.$$

Macro task accuracy is

$$A_{\text{macro}} = \frac{1}{K} \sum_{k=1}^{K} \frac{1}{n_k} \sum_{i \in k} I_{\text{success},i}.$$

Also report the individual component rates; the product alone cannot identify the reason for failure.

Part 3 - Evaluation construction protocol

1. Derive task families and severity levels from intended uses and known failure risks.
2. Write scoring rules before observing candidate model outputs.
3. Include canonical prompts, paraphrases, irrelevant context, conflicting constraints, and boundary conditions.
4. Group all variants of one semantic item into the same split.
5. Validate automated graders against blinded human labels and report their agreement.
6. Freeze a versioned test set and maintain a separate development set for prompt and hyperparameter decisions.
7. Publish macro, micro, per-task, per-difficulty, and worst-slice metrics together with representative failures.

Part 4 - Explicit evaluation framework

```
def exact_match(candidate, reference):
    return int(candidate.strip() == reference.strip())

def required_substrings(candidate, requirements):
    passed = True
    for requirement in requirements:
        if requirement not in candidate:
            passed = False
    return int(passed)

def grade_item(item, candidate):
    content = exact_match(candidate, item["reference"])
    constraints = required_substrings(
        candidate, item["required_substrings"]
    )
    format_score = item["format_grader"](candidate)
    success = content * constraints * format_score
    result = {}
    result["item_id"] = item["item_id"]
    result["task"] = item["task"]
    result["content"] = content
    result["constraints"] = constraints
    result["format"] = format_score
    result["success"] = success
    return result

def E14(items, generate):
    results = []
    task_totals = {}
    task_successes = {}
    for item in items:
        candidate = generate(item["prompt"])
        result = grade_item(item, candidate)
        results.append(result)
        task = result["task"]
        if task not in task_totals:
            task_totals[task] = 0
            task_successes[task] = 0
        task_totals[task] = task_totals[task] + 1
        task_successes[task] = task_successes[task] + result["success"]
    task_accuracy = {}
    for task in task_totals:
        task_accuracy[task] = task_successes[task] / task_totals[task]
    return results, task_accuracy
```

Part 5 - The Philosophy Behind the Exercise

Evaluation is where the phrase “follows instructions” receives a definition. Without a taxonomy and rubric, the phrase expands or contracts to fit whatever output the model produced.

A good evaluation makes success difficult to fake.

Metrics are promises translated into tests. Their limitations must remain visible, because what is omitted from the suite can become invisible to the project. Measurement does not merely observe progress; it directs which kind of progress the training team will notice.

18.5.16 Exercise 15

Exercise Title: Analyze Failure Modes Where Model Obeys Style but Misses Content

Part 1 - Deep conceptual foundation

Instruction-tuned models often learn visible conventions faster than semantic requirements. They produce headings, confident tone, JSON braces, disclaimers, or step-by-step prose while the underlying answer is wrong. Style tokens are frequent, locally predictable, and shared across many examples; difficult content may be rare and require longer dependencies. A falling token loss can therefore reward polished surfaces before factual or logical competence.

The failure must be decomposed. Format validity asks whether the response parses. Schema validity asks whether required fields and types exist. Constraint validity asks whether requested limits are respected. Semantic correctness asks whether values and claims answer the task. Groundedness asks whether claims are supported by provided context. These dimensions should never be collapsed into one formatting score.

Create adversarial pairs in which an incorrect answer is perfectly styled and a correct answer is poorly styled. A grader that consistently prefers the first has learned the wrong objective. For structured outputs, parse the response and validate values independently. For explanations, extract final claims and test them against references or executable checks where possible.

Root causes can include superficial training targets, imbalanced easy examples, reference responses with unnecessary verbosity, a grader biased toward style, or insufficient domain capability in the base model. Fixes differ: rebalance data, add hard negatives, improve content verification, continue domain pretraining, or simplify the requested schema. More SFT steps cannot create missing knowledge reliably.

Part 2 - Failure decomposition

Let $F_i, S_i, C_i, G_i \in \{0, 1\}$ denote format, schema, content, and grounding success. The dangerous style-content failure rate is

$$R_{\text{surface}} = \frac{1}{N} \sum_{i=1}^{N} F_i S_i (1 - C_i).$$

Conditional content accuracy among well-formatted outputs is

$$A_{C|FS} = \frac{\sum_i F_i S_i C_i}{\sum_i F_i S_i}.$$

Report both: a model can improve format rate while conditional content accuracy falls.

Part 3 - Failure investigation

1. Build paired examples that independently vary style quality and content correctness.
2. Use parsers for JSON or code syntax, but evaluate field values and program behavior separately.

3. Group failures into wrong fact, wrong calculation, ignored evidence, unsupported inference, contradiction, and incomplete answer.
4. Compare token-level loss for shared style prefixes and content-bearing tokens.
5. Inspect whether training data contains many reusable shells with weak or noisy answers.
6. Add verified hard examples and hard negatives, then measure content gains without sacrificing necessary format.
7. Require the report to include convincing-looking failures, not only obvious malformed outputs.

Part 4 - Separate format and content graders

```
import json

def parse_json_response(response):
    try:
        value = json.loads(response)
        return True, value
    except json.JSONDecodeError:
        return False, None

def grade_structured_answer(response, reference):
    format_valid, value = parse_json_response(response)
    schema_valid = False
    content_valid = False
    if format_valid and isinstance(value, dict):
        if "answer" in value and "reason" in value:
            schema_valid = True
            content_valid = value["answer"] == reference["answer"]
    result = {}
    result["format_valid"] = int(format_valid)
    result["schema_valid"] = int(schema_valid)
    result["content_valid"] = int(content_valid)
    surface_failure = format_valid and schema_valid and not content_valid
    result["surface_failure"] = int(surface_failure)
    return result

def E15(responses, references):
    if len(responses) != len(references):
        raise ValueError("response and reference counts differ")
    results = []
    surface_failures = 0
    for index in range(len(responses)):
        result = grade_structured_answer(
            responses[index], references[index]
        )
```

```
        results.append(result)
        surface_failures = surface_failures + result["surface_failure"]
    rate = surface_failures / len(results)
    return results, rate
```

Part 5 - The Philosophy Behind the Exercise

Human readers naturally trust organized language. Headings, precision, and confident explanations signal competence in ordinary communication, so models can exploit our expectations without intention simply by reproducing those patterns. This makes surface quality an especially dangerous substitute for truth.

The exercise teaches epistemic separation: presentation tells us how an answer is expressed; evidence tells us whether it should be believed. A responsible system must earn both scores independently.

18.5.17 Exercise 16

Exercise Title: Implement Prompt Dropout and Measure Robustness Gains

Part 1 - Deep conceptual foundation

Prompt dropout is a data transformation that randomly removes optional or redundant prompt components during training. Its purpose is to stop the model from relying on one superficial cue when the semantic task remains recoverable from other fields. Examples include dropping a polite preamble, optional task description, redundant label list, or nonessential demonstration.

This must never be implemented as uncontrolled token deletion. Removing negation, a safety condition, a unit, required context, or role boundary can change the correct response and create mislabeled data. Define prompt components structurally and label each as required, optional, or conditionally droppable. Apply dropout before rendering so tags and separators remain syntactically valid.

The experiment compares no dropout with several probabilities under the same token and update budget. Because dropout shortens prompts, equal steps do not imply equal processed tokens; report both. Evaluate canonical prompts, missing optional fields, reordered fields, paraphrases, and prompts missing required information. The last category should trigger clarification or uncertainty, not the same confident answer.

Robustness gain must not be confused with under-conditioning. If aggressive dropout causes the model to ignore useful context, performance may look stable only because it produces generic responses. Measure context utilization and content specificity in addition to task accuracy.

Part 2 - Stochastic transformation

Let prompt components be $c_1, \ldots, c_M$ with optional set O. For $j \in O$, sample

$$d_j \sim \text{Bernoulli}(1 - p_j), \qquad \widetilde{x} = g(c_j : d_j = 1 \text{ or } j \notin O).$$

The response label remains valid only if

$$y^*(\widetilde{x}) = y^*(x).$$

That invariance is a data-design claim requiring validation, not an automatic consequence of deletion.

Part 3 - Robustness experiment

1. Parse prompts into named components and classify each component's semantic necessity.
2. Define dropout probabilities per component rather than one probability for arbitrary tokens.
3. Seed the transformation and log the component-presence mask for every rendered example.
4. Verify that role markers, response boundary, and required evidence are never dropped.
5. Compare canonical, optional-component-missing, paraphrased, and required-information-missing evaluation slices.
6. Measure accuracy, specificity, clarification behavior, prompt token count, and retained base capability.
7. Inspect whether robustness persists when several optional cues disappear simultaneously.

Part 4 - Structured prompt dropout code

```
import random

def apply_prompt_dropout(record, probabilities, random_generator):
    transformed = {}
    transformed["instruction"] = record["instruction"]
    transformed["response"] = record["response"]
    presence = {}
    optional_fields = ["preamble", "hint", "format_reminder"]
    for field_name in optional_fields:
        if field_name in record:
            probability = probabilities[field_name]
            keep_value = random_generator.random() >= probability
            presence[field_name] = keep_value
            if keep_value:
                transformed[field_name] = record[field_name]
    if "context" in record:
        transformed["context"] = record["context"]
        presence["context"] = True
    return transformed, presence

def render_dropout_record(record):
    text = ""
    ordered_fields = ["preamble", "instruction", "context",
                      "hint", "format_reminder"]
    for field_name in ordered_fields:
```

```
        if field_name in record:
            text = text + "<" + field_name + ">"
            text = text + record[field_name]
            text = text + "</" + field_name + ">\n"
    text = text + "<assistant>"
    return text

def E16(records, probabilities, seed):
    random_generator = random.Random(seed)
    outputs = []
    for record in records:
        transformed, presence = apply_prompt_dropout(
            record, probabilities, random_generator
        )
        item = {}
        item["prompt"] = render_dropout_record(transformed)
        item["response"] = transformed["response"]
        item["presence"] = presence
        outputs.append(item)
    return outputs
```

Part 5 - The Philosophy Behind the Exercise

Prompt dropout asks the model to find what is essential beneath redundant expression. By varying optional cues, training can weaken dependence on ritual and strengthen dependence on the task's stable meaning.

Yet subtraction is only wise when meaning survives. Removing necessary context and retaining the old answer teaches certainty without evidence. Robustness is not indifference to information; it is the ability to distinguish essential information from accidental presentation.

18.5.18 Exercise 17

Exercise Title: Compare SFT vs Continued Pretraining on Domain Text

Part 1 - Deep conceptual foundation

Continued pretraining and supervised instruction fine-tuning solve different problems. Continued pretraining exposes a base model to unlabeled domain text under next-token prediction, adapting vocabulary, style, facts, and statistical regularities. SFT presents explicit instruction-response behavior and teaches how the model should act when asked. Domain knowledge and instruction behavior can overlap, but neither objective is a substitute for the other.

If the model lacks familiarity with specialized concepts, SFT examples may be too sparse to build the necessary representations. Continued pretraining can help, followed by SFT to make the knowledge accessible through instructions. Conversely, if the model already knows the domain but answers in the wrong format, continued pretraining may merely make it imitate domain documents; response-masked SFT is the more direct intervention.

The comparison needs four arms from one base checkpoint: no adaptation, continued pretraining only, SFT only, and continued pretraining followed by SFT. Match compute or processed tokens according to the question being asked and report both. Use separate domain-language-modeling validation, domain question answering, instruction following, and general

retention suites. Ensure that evaluation answers do not occur verbatim in either training corpus.

Data formatting must remain distinct. Continued pretraining typically supervises all valid next tokens in domain documents. SFT supervises assistant response regions. Accidentally applying response-only masks to raw domain text discards most signal; applying full-document loss to chat examples may overemphasize prompt reproduction.

Part 2 - Objective comparison

Continued pretraining minimizes

$$L_{\mathrm{CPT}} = -\frac{1}{N_D} \sum_{x \in D} \sum_{t} \log p_\theta(x_t \mid x_{<t}),$$

while SFT minimizes

$$L_{\mathrm{SFT}} = -\frac{1}{N_R} \sum_{(u,y)} \sum_{t \in R(y)} \log p_\theta(y_t \mid u, y_{<t}).$$

The sequential pipeline is

$$\theta_1 = \mathrm{CPT}(\theta_0, D), \qquad \theta_2 = \mathrm{SFT}(\theta_1, I).$$

Order matters because the second stage can modify gains and losses from the first.

Part 3 - Four-arm experiment

1. Build domain-document and instruction datasets with independent provenance, deduplication, and evaluation contamination checks.
2. Clone one base state into the four experimental arms.
3. Use objective-appropriate labels and verify supervision density.
4. Match a compute budget and additionally report processed tokens and optimizer updates.
5. Evaluate domain perplexity, closed-book domain accuracy, context-grounded accuracy, instruction compliance, and general retention.
6. Inspect whether CPT improves knowledge but worsens assistant formatting, and whether SFT restores the interface.
7. Select the simplest pipeline that meets all capability and retention gates rather than assuming two stages are always better.

Part 4 - Explicit label-policy separation

```
import torch

def build_cpt_labels(input_ids, attention_mask):
    labels = input_ids.clone()
    labels[attention_mask == 0] = -100
    return labels
```

```
def build_sft_labels(input_ids, assistant_mask, attention_mask):
    labels = input_ids.clone()
    labels[assistant_mask == 0] = -100
    labels[attention_mask == 0] = -100
    if not (labels != -100).any():
        raise ValueError("SFT example has no assistant supervision")
    return labels

def supervision_density(labels, attention_mask):
    valid_input_tokens = int(attention_mask.sum().item())
    supervised_tokens = int((labels != -100).sum().item())
    if valid_input_tokens == 0:
        raise ValueError("empty sequence")
    return supervised_tokens / valid_input_tokens

def E17(input_ids, attention_mask, assistant_mask):
    cpt_labels = build_cpt_labels(input_ids, attention_mask)
    sft_labels = build_sft_labels(
        input_ids, assistant_mask, attention_mask
    )
    result = {}
    result["cpt_labels"] = cpt_labels
    result["sft_labels"] = sft_labels
    result["cpt_density"] = supervision_density(
        cpt_labels, attention_mask
    )
    result["sft_density"] = supervision_density(
        sft_labels, attention_mask
    )
    return result
```

Part 5 - The Philosophy Behind the Exercise

Knowing a domain and knowing how to respond within it are different forms of adaptation. Continued pretraining changes what patterns feel familiar; SFT changes which behavior follows a request. Treating them as identical obscures the reason an intervention succeeds or fails.

The deeper principle is objective-task alignment. Training should apply pressure where the deficiency actually lives. New knowledge requires evidence; new conduct requires demonstrations of conduct. Strong systems often need both, but in an intentional order.

18.5.19 Exercise 18

Exercise Title: Build Fine-Tuning Debugging Checklist Covering Masks, LR, and Data

Part 1 - Deep conceptual foundation

A debugging checklist converts recurring failures into an ordered diagnostic process. Fine-tuning pipelines can lower loss while remaining semantically wrong, so the order should begin

with data and target alignment before optimizer tuning. Changing learning rate cannot repair shifted labels, and adding memory cannot repair response leakage.

The first layer is identity: base checkpoint, tokenizer, special tokens, template version, dataset manifest, and code configuration. The second is data: duplicates, empty responses, split leakage, truncation, role boundaries, and decoded samples. The third is tensors: shapes, dtypes, devices, attention masks, assistant masks, labels, valid-token counts, and causal behavior. Only after these pass should optimization, memory, quality, retention, and reproducibility be investigated.

Every checklist item needs a test, expected result, severity, evidence path, and remediation. "Check the mask" is too vague. A strong item says: append padding, recompute loss, and assert unchanged valid-token mean within tolerance. Automate deterministic invariants and reserve human review for semantic quality.

The exercise builds a gate-based diagnostic report. Blocking failures stop the run before expensive training. Warnings permit continuation but remain visible. The checklist also includes deliberate fault injection: wrong shift, missing assistant mask, duplicate data, excessive learning rate, and device mismatch. A diagnostic is trusted only after it detects a known failure.

Part 2 - Gate model

Let checks produce $c_i \in \{0, 1\}$ and let blocking set B contain conditions required for correctness. The run-ready state is

$$R = \prod_{i \in B} c_i.$$

For nonblocking warnings with weights w_i, define a visible risk score

$$Q = \sum_{i \notin B} w_i(1 - c_i).$$

Q may prioritize review, but it must not override $R = 0$. Correctness gates cannot be averaged away.

Part 3 - Ordered debugging checklist

1. Verify hashes and versions for checkpoint, tokenizer, template, dataset, and configuration.
2. Decode raw, rendered, tokenized, shifted, truncated, and padded examples from every task family.
3. Assert causal masking, response-only labels, target range, nonzero valid tokens, and padding-loss invariance.
4. Overfit one batch and a tiny dataset before a full run.
5. Confirm intended trainable parameters and finite nonzero gradients.
6. Sweep learning rate, inspect gradient/update norms, and verify scheduler steps correspond to optimizer steps.
7. Monitor validation, paraphrase robustness, and retained capability for overfitting and forgetting.

8. Reproduce the run, resume from checkpoint, and inject known faults to test diagnostic coverage.

Part 4 - Executable checklist structure

```
class DiagnosticCheck:
    def __init__(self, check_id, blocking, function, remediation):
        self.check_id = check_id
        self.blocking = blocking
        self.function = function
        self.remediation = remediation

    def run(self, context):
        passed, evidence = self.function(context)
        result = {}
        result["check_id"] = self.check_id
        result["blocking"] = self.blocking
        result["passed"] = bool(passed)
        result["evidence"] = evidence
        result["remediation"] = self.remediation
        return result

def run_diagnostics(checks, context):
    results = []
    ready = True
    for check in checks:
        result = check.run(context)
        results.append(result)
        if result["blocking"] and not result["passed"]:
            ready = False
    return ready, results

def check_supervised_tokens(context):
    labels = context["labels"]
    count = int((labels != -100).sum().item())
    return count > 0, {"supervised_tokens": count}

def E18(context):
    checks = []
    checks.append(DiagnosticCheck(
        "nonempty_supervision", True, check_supervised_tokens,
        "repair assistant boundaries and label construction"
    ))
    ready, results = run_diagnostics(checks, context)
    return {"ready": ready, "results": results}
```

Part 5 - The Philosophy Behind the Exercise

Debugging is the practice of refusing mystery. A failed run is not explained by calling fine-tuning unstable; it is explained by tracing evidence until a specific contract is shown to be broken or a hypothesis is falsified.

A checklist preserves hard-earned knowledge against memory and pressure. It makes quality less dependent on who happens to notice a subtle symptom. The best checklist is not paperwork after engineering; it is accumulated engineering made executable.

18.5.20 Exercise 19

Exercise Title: Produce an Instruction Tuning Engineering Report with Lessons Learned

Part 1 - Deep conceptual foundation

The final report must reconstruct the complete fine-tuning argument: which base model was adapted, what behavior was desired, how records were sourced and rendered, which tokens were supervised, which parameters were trainable, how optimization was controlled, and what evidence justifies the selected checkpoint. It is not a chronological diary and not a gallery of successful outputs. It is a decision document that includes failed hypotheses.

Begin with scope and non-goals. State the intended users, task families, languages, context limits, response policy, and behaviors the fine-tune does not promise. Document dataset provenance, authorization, template versions, deduplication, split logic, truncation, task mixture, response length, and quality review. Provide distributions, not only totals.

The training section must specify full or parameter-efficient tuning, exact trainable modules, loss mask, valid-token reduction, optimizer, scheduler, precision, accumulation, clipping, seeds, hardware, throughput, memory, and checkpoint recovery. The evaluation section must compare the base and tuned models on target tasks, paraphrases, content versus format, retained abilities, and meaningful failure slices. Include uncertainty across seeds.

Finish with a decision table and traceable gates. Identify which configuration was selected, which alternatives were rejected, and why. List known limitations, monitoring recommendations, rollback conditions, and the next experiment that would reduce the most important uncertainty. A report becomes valuable when a future engineer can challenge its conclusion without recreating hidden context.

Part 2 - Decision model

Let the evaluation vector be

$$G = (A_{\text{task}}, A_{\text{para}}, C_{\text{content}}, F_{\text{format}}, R_{\text{retention}}, M_{\text{peak}}, Q_{\text{throughput}}).$$

A release candidate is acceptable only if every blocking requirement holds:

$$\text{Accept}(G) = \bigwedge_{j \in B} [g_j \ \text{ meets } \ \tau_j] \, .$$

Cost or preference scores may rank candidates after they pass gates, but cannot compensate for a failed correctness or safety requirement.

Part 3 - Required report evidence

1. Include immutable identifiers for model, tokenizer, code, data, template, configuration, checkpoints, and evaluation suite.

2. Show decoded supervision examples and automated mask tests.

3. Compare full tuning and LoRA, or explain with evidence why one was excluded.
4. Present learning curves, gradient behavior, memory, throughput, and failure events rather than only the final epoch.
5. Report paired base-versus-tuned examples chosen by a fixed sampling rule.
6. Separate content, style, prompt robustness, and retention results.
7. Record rejected configurations and lessons learned from at least one injected or naturally occurring failure.
8. End with owners, rollback triggers, limitations, and a prioritized next experiment.

Part 4 - Structured report and gate builder

```
class InstructionTuningReport:
    def __init__(self, report_id):
        self.report_id = report_id
        self.sections = {}
        self.gates = []
        self.lessons = []

    def add_section(self, name, evidence):
        self.sections[name] = evidence

    def add_gate(self, name, value, threshold,
                 comparison, blocking, owner):
        gate = {}
        gate["name"] = name
        gate["value"] = value
        gate["threshold"] = threshold
        gate["comparison"] = comparison
        gate["blocking"] = blocking
        gate["owner"] = owner
        if comparison == "greater_or_equal":
            gate["passed"] = value >= threshold
        elif comparison == "less_or_equal":
            gate["passed"] = value <= threshold
        else:
            raise ValueError("unknown comparison")
        self.gates.append(gate)

    def release_ready(self):
        for gate in self.gates:
            if gate["blocking"] and not gate["passed"]:
                return False
        return True
```

```
    def missing_sections(self, required_sections):
        missing = []
        for section_name in required_sections:
            if section_name not in self.sections:
                missing.append(section_name)
        return missing

def E19():
    report = InstructionTuningReport("LLM03-SFT-REPORT-V1")
    required = ["scope", "base_model", "data", "templates",
                "loss", "training", "evaluation", "failures",
                "retention", "deployment"]
    report.add_section("scope", {"status": "complete"})
    report.add_section("base_model", {"status": "verified"})
    report.add_section("data", {"status": "audited"})
    report.add_gate(
        "heldout_accuracy", 0.0, 0.90,
        "greater_or_equal", True, "evaluation_owner"
    )
    return report, report.missing_sections(required)
```

Part 5 - The Philosophy Behind the Exercise

An engineering report is institutional memory with a standard of proof. It preserves not only what was built, but why one design was trusted over another, which evidence was missing, and where confidence should end.

Instruction tuning gives designers unusual influence over a model's visible conduct. The final report is where that influence becomes accountable. It turns behavioral change from an impressive demonstration into a traceable decision that can be reproduced, criticized, monitored, and reversed.

18.6 LLM04

Chapter Title: Alignment, Preferences, and Safety
Specialization: Preference Optimization and Safety Engineering

18.6.1 Exercise 00

Exercise Title: Explain Alignment Problem and Why Raw LLMs Can Be Unsafe

Part 1 - Deep conceptual foundation

A raw language model is optimized to predict continuations from its training distribution. That objective rewards statistical likelihood, not truth, helpfulness, privacy, fairness, or compliance with a deployment policy. Useful behavior can emerge because useful text exists in the corpus, but the objective does not reliably select it when harmful, deceptive, outdated, or unsupported continuations are also plausible.

Alignment is the problem of making model behavior reliably serve specified human purposes and constraints across ordinary, ambiguous, adversarial, and unfamiliar inputs. It is not one

universal target. Users, operators, affected people, and legal or cultural contexts can value outcomes differently. A real system therefore needs an explicit behavioral policy, a method for resolving conflicts, and an escalation path where automatic behavior is insufficient.

Raw models can expose several risks: following dangerous patterns found in data, revealing memorized private strings, confidently inventing claims, amplifying bias, obeying malicious instructions embedded in untrusted context, or failing to distinguish a legitimate request from an unsafe one. Scale does not remove the mismatch; it can increase both capability and the persuasiveness of errors.

This exercise builds a threat model rather than assuming that "unsafe" is one binary property. Define actors, assets, permitted uses, misuse cases, accidental failures, affected groups, severity, likelihood, and available controls. Compare the raw model with an instruction-tuned model on a defensively designed prompt suite and separate helpfulness, factuality, refusal appropriateness, privacy, and robustness.

Part 2 - Formalizing imperfect objectives

Let $U(y \mid x)$ represent desired utility and let L_{LM} be next-token loss. In general,

$$\arg\min_{\theta} L_{\text{LM}}(\theta) \neq \arg\max_{\theta} \mathbb{E}[U(y \mid x)].$$

Alignment introduces proxy signals $\widehat{U}$, but proxies can also be incomplete. Risk can be organized as

$$R = \sum_{c} P(c)\, I(c),$$

where c is a failure class, $P(c)$ its estimated occurrence probability, and $I(c)$ its impact. Do not hide catastrophic low-frequency cases inside one average.

Part 3 - Threat-model experiment

1. Specify intended users, permitted tasks, protected assets, and non-goals.
2. Build benign, ambiguous, boundary, adversarial, and distribution-shift prompt slices without including operational harmful content.
3. Write expected behaviors before viewing model responses.
4. Score helpfulness, correctness, refusal precision, privacy, and policy compliance separately.
5. Compare raw and aligned checkpoints under identical decoding settings.
6. Record both unsafe compliance and excessive refusal.
7. Assign each failure an owner, mitigation layer, and residual-risk decision.

Part 4 - Explicit risk registry

```
class AlignmentRisk:
    def __init__(self, risk_id, category, severity, likelihood):
        self.risk_id = risk_id
        self.category = category
```

```
        self.severity = severity
        self.likelihood = likelihood
        self.controls = []

    def add_control(self, control_name, owner, evidence):
        control = {}
        control["name"] = control_name
        control["owner"] = owner
        control["evidence"] = evidence
        self.controls.append(control)

    def score(self):
        return self.severity * self.likelihood

def E00(risk_definitions):
    registry = []
    for definition in risk_definitions:
        risk = AlignmentRisk(
            definition["risk_id"], definition["category"],
            definition["severity"], definition["likelihood"]
        )
        registry.append(risk)
    def risk_score(item):
        return item.score()
    registry.sort(key=risk_score, reverse=True)
    return registry
```

Part 5 - The Philosophy Behind the Exercise

Prediction learns what people have written; alignment asks what a system should do. The distance between those questions is where responsibility enters model engineering. No amount of likelihood optimization can choose values that were never made explicit.

Alignment is therefore not the removal of all uncertainty or disagreement. It is the disciplined construction of behavior, evidence, boundaries, and accountability while acknowledging whose interests shaped the target.

18.6.2 Exercise 01

Exercise Title: Define Human Preference Data and How It Differs from Supervised Labels

Part 1 - Deep conceptual foundation

Supervised fine-tuning usually provides a desired response and asks the model to increase its likelihood. Preference data instead compares candidate responses for the same prompt and records which one is better under a rubric. The signal is relational: "chosen is preferred to rejected." It need not assert that the chosen response is perfect or that the rejected response is always unacceptable.

Preference judgments capture qualities that are difficult to encode as one reference string, such as relevance, clarity, calibrated uncertainty, safe boundary handling, and adherence to constraints. They also contain human subjectivity. Annotators may disagree, misunderstand

the domain, be influenced by verbosity, or apply the rubric inconsistently. The dataset must preserve annotator identifiers or groups, confidence, rubric version, tie or abstention, and reason codes when allowed by privacy policy.

Pair construction matters. If chosen responses are systematically longer, contain a familiar phrase, or come from one generator, a model can learn that shortcut instead of the intended preference. Randomize presentation order, balance nuisance features, include hard pairs, and audit position and length bias. Never force a ranking where responses are equivalent or the annotator lacks expertise.

The exercise will represent pairwise judgments, validate prompt identity, detect contradictory cycles and duplicate reversals, and calculate inter-rater agreement. It will keep raw annotations separate from aggregated training pairs so uncertainty is not erased prematurely.

Part 2 - Pairwise probability model

For latent utility $r(x, y)$, the Bradley–Terry model defines

$$P(y_a \succ y_b \mid x) = \sigma\left(r(x, y_a) - r(x, y_b)\right).$$

Unlike a class label attached to one response, the target depends on a pair and context. If annotators j provide labels v_{ij}, report agreement and the empirical preference probability rather than discarding minority judgments:

$$\widehat{p}_i = \frac{1}{m_i} \sum_{j=1}^{m_i} v_{ij}.$$

Part 3 - Dataset quality protocol

1. Store prompt, both candidates, presentation order, rubric version, annotator group, choice, tie, abstention, confidence, and reason codes.
2. Verify both responses answer the identical semantic prompt.
3. Reject self-pairs, exact duplicate candidates, and reversed duplicate records with incompatible labels.
4. Audit choice rate by candidate position, length, generator, and style.
5. Preserve disagreements and adjudicate only under a documented rule.
6. Create gold and repeated items to estimate annotator consistency.
7. Split by prompt family so paraphrased versions do not leak across sets.

Part 4 - Explicit preference records

```
class PreferencePair:
    def __init__(self, pair_id, prompt_id, prompt,
                 chosen, rejected, rubric_version):
        if chosen.strip() == rejected.strip():
            raise ValueError("preference candidates are identical")
        self.pair_id = pair_id
```

```
        self.prompt_id = prompt_id
        self.prompt = prompt
        self.chosen = chosen
        self.rejected = rejected
        self.rubric_version = rubric_version
        self.annotations = []

    def add_annotation(self, annotator_group, choice, confidence):
        if choice not in ["chosen", "rejected", "tie", "abstain"]:
            raise ValueError("invalid preference choice")
        annotation = {}
        annotation["annotator_group"] = annotator_group
        annotation["choice"] = choice
        annotation["confidence"] = confidence
        self.annotations.append(annotation)

def E01(raw_pairs):
    pairs = []
    identifiers = set()
    for record in raw_pairs:
        if record["pair_id"] in identifiers:
            raise ValueError("duplicate pair identifier")
        identifiers.add(record["pair_id"])
        pair = PreferencePair(
            record["pair_id"], record["prompt_id"], record["prompt"],
            record["chosen"], record["rejected"], record["rubric_version"]
        )
        pairs.append(pair)
    return pairs
```

Part 5 - The Philosophy Behind the Exercise

A supervised label says what answer to imitate. A preference says which of two possibilities better reflects a judgment. That distinction makes preference data flexible, but it also reveals that the target is neither objective nor complete.

Human disagreement is not always noise to eliminate. It can be evidence that a prompt is ambiguous, a value is contested, or expertise is uneven. Responsible alignment preserves this information before converting judgment into gradient.

18.6.3 Exercise 02

Exercise Title: Explain RLHF Pipeline Conceptually and Identify Its Components

Part 1 - Deep conceptual foundation

Reinforcement Learning from Human Feedback is a family of pipelines that uses human judgments to shape a language-model policy. A common pipeline contains a pretrained model, supervised instruction tuning, response collection, preference annotation, reward-model training, policy optimization, a reference policy, and continuous evaluation. Each component can fail independently.

The reward model learns to assign higher scalar scores to preferred responses. It is a proxy for the annotation rubric, not a universal measure of goodness. Policy optimization then searches for responses that score well while a KL penalty limits movement away from a reference model. Without this constraint, the policy can exploit reward-model errors and move into regions where the proxy has little training support.

RLHF is iterative. As the policy changes, it produces new responses, changing the distribution on which judgments are needed. Offline preference quality does not guarantee online policy safety. Evaluations and red-team tests must surround the loop, and severe failures should stop policy updates rather than waiting for an average reward to improve.

This exercise maps every artifact and interface: tokenization, prompts, candidates, annotations, reward scores, policy log-probabilities, reference log-probabilities, advantages, checkpoints, and gates. You will simulate the data flow without implementing a harmful-capability optimizer.

Part 2 - Reward and policy objectives

Reward-model pairwise loss is

$$L_{\mathrm{RM}} = -\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l)).$$

A regularized policy objective has the conceptual form

$$\max_\theta \mathbb{E}_{y\sim\pi_\theta}[r_\phi(x, y)] - \beta\, \mathrm{KL}(\pi_\theta \| \pi_{\mathrm{ref}}).$$

Reward increase is not sufficient evidence; the evaluation vector must include helpfulness, safety, truthfulness, refusal precision, diversity, and drift.

Part 3 - Pipeline audit

1. Name the owner, version, input contract, output contract, and validation for every stage.
2. Prove tokenizer and chat-template identity across policy, reference, and reward model.
3. Evaluate reward accuracy on held-out annotators and hard preference pairs.
4. Monitor KL, response length, entropy, reward variance, and safety gates during policy optimization.
5. Keep reference and reward checkpoints immutable within one experiment.
6. Add stop conditions for reward spikes, KL divergence, mode collapse, and critical evaluation regressions.
7. Document where new human review enters the iteration loop.

Part 4 - Pipeline artifact registry

```
class RLHFStage:
    def __init__(self, name, version, inputs, outputs, owner):
        self.name = name
        self.version = version
        self.inputs = inputs
```

```
        self.outputs = outputs
        self.owner = owner
        self.validation = []

    def add_validation(self, check_name, threshold):
        self.validation.append({
            "check_name": check_name,
            "threshold": threshold
        })

def E02():
    stages = []
    stages.append(RLHFStage(
        "supervised_policy", "v1", ["instruction_data"],
        ["sft_checkpoint"], "model_team"
    ))
    stages.append(RLHFStage(
        "preference_collection", "v1", ["prompts", "candidates"],
        ["ranked_pairs"], "data_team"
    ))
    stages.append(RLHFStage(
        "reward_model", "v1", ["ranked_pairs"],
        ["reward_checkpoint"], "reward_team"
    ))
    stages.append(RLHFStage(
        "policy_optimization", "v1",
        ["sft_checkpoint", "reward_checkpoint", "reference_checkpoint"],
        ["aligned_checkpoint"], "optimization_team"
    ))
    return stages
```

Part 5 - The Philosophy Behind the Exercise

RLHF turns judgments into an optimization system. Every translation—rubric to annotation, annotation to reward, reward to policy—compresses meaning and can lose information. The pipeline must preserve enough evidence to inspect those losses.

Human feedback does not place a human conscience inside the model. It creates a measured pressure toward observed preferences. The difference matters because proxies require monitoring wherever the policy can discover their blind spots.

18.6.4 Exercise 03

Exercise Title: Implement Preference Pair Dataset and Validate Ranking Consistency

Part 1 - Deep conceptual foundation

A preference training item contains one prompt and at least two candidate responses. Both sequences must share the same prompt tokens and differ only in the response region. If templates, truncation, or system messages differ, the loss may learn those artifacts instead of the ranking.

Tokenization must preserve complete chosen and rejected responses whenever possible. Apply the identical prompt truncation policy to both branches. Record response boundaries and masks so log-probabilities are computed only over the candidate response. If one response is truncated more aggressively, its score is no longer comparable.

Ranking consistency includes more than chosen-not-equal-rejected. A dataset can contain reverse duplicates, cycles such as $A \succ B$, $B \succ C$, $C \succ A$, and the same pair judged under different rubrics. Cycles may reflect legitimate contextual tradeoffs or annotation noise; detect and investigate them instead of silently forcing transitivity.

This exercise constructs paired tensors with explicit loops, validates shared prompts and response masks, and builds a directed graph for contradiction analysis. It also reports length and presentation biases by split.

Part 2 - Pair contract

For prompt tokens p, chosen response c, and rejected response r:

$$x_w = p \| c, \qquad x_l = p \| r.$$

Response masks satisfy $m_{w,t} = 0$ over p and 1 over c, with the analogous rule for r. Pair validation requires equal prompt prefixes and at least one scored token on both branches.

Part 3 - Validation tests

1. Render the prompt once and reuse its token identifiers for both branches.
2. Reserve response capacity before truncating the shared prompt.
3. Verify response masks begin at the same semantic boundary.
4. Reject identical candidates and exact reverse duplicates.
5. Detect cycles per prompt and rubric; route them for review.
6. Audit chosen rate against response length, order, generator, and style.
7. Decode paired tensors and supervision masks from every edge case.

Part 4 - Explicit paired encoding

```
def encode_pair(tokenizer, record, maximum_length):
    prompt_ids = tokenizer.encode(record["prompt"], add_special_tokens=False)
    chosen_ids = tokenizer.encode(record["chosen"], add_special_tokens=False)
    rejected_ids = tokenizer.encode(
        record["rejected"], add_special_tokens=False
    )
    if chosen_ids == rejected_ids:
        raise ValueError("candidates tokenize identically")
    maximum_response = max(len(chosen_ids), len(rejected_ids))
    allowed_prompt = maximum_length - maximum_response
    if allowed_prompt < 1:
        raise ValueError("candidate response exceeds maximum length")
```

```
    if len(prompt_ids) > allowed_prompt:
        prompt_ids = prompt_ids[-allowed_prompt:]

    chosen_input = prompt_ids + chosen_ids
    rejected_input = prompt_ids + rejected_ids
    chosen_mask = []
    rejected_mask = []
    for position in range(len(chosen_input)):
        chosen_mask.append(int(position >= len(prompt_ids)))
    for position in range(len(rejected_input)):
        rejected_mask.append(int(position >= len(prompt_ids)))
    return chosen_input, rejected_input, chosen_mask, rejected_mask

def E03(tokenizer, records, maximum_length):
    encoded = []
    seen = set()
    for record in records:
        key = (record["prompt_id"], record["chosen"], record["rejected"])
        reverse = (record["prompt_id"], record["rejected"], record["chosen"])
        if key in seen or reverse in seen:
            raise ValueError("duplicate or reversed preference pair")
        seen.add(key)
        encoded.append(encode_pair(tokenizer, record, maximum_length))
    return encoded
```

Part 5 - The Philosophy Behind the Exercise

A preference pair is a controlled comparison. Its scientific value comes from holding the prompt constant so one behavioral difference can be judged. When the two branches contain hidden formatting differences, the comparison loses its meaning.

Consistency does not require pretending every value judgment is transitive. Detection should precede correction. Contradictions can reveal noise, ambiguity, or genuine conflicts that a scalar reward cannot faithfully represent.

18.6.5 Exercise 04

Exercise Title: Implement DPO Objective Conceptually and Compare to RLHF

Part 1 - Deep conceptual foundation

Direct Preference Optimization trains a policy directly from chosen and rejected responses while comparing it with a frozen reference policy. It avoids fitting a separate reward model and avoids online reinforcement-learning rollouts in the basic offline form. This simplifies the pipeline, but it does not remove dependence on preference quality, reference choice, coverage, or evaluation.

DPO increases the policy's chosen-versus-rejected log-probability margin relative to the same margin under the reference. The reference prevents the objective from rewarding differences already present before alignment and anchors the update. The temperature parameter β controls the strength of this comparison; its practical effect depends on data and optimization conventions.

Sequence log-probabilities must include response tokens only. Summing favors length-dependent magnitudes; averaging changes the objective. Use the formulation declared by the experiment and audit response-length effects. The policy and reference must use identical tokenization, templates, masks, and precision-aware log-softmax.

The exercise implements DPO loss from logits and contrasts system properties: RLHF can collect on-policy behavior and optimize arbitrary learned rewards, but is operationally complex and vulnerable to reward hacking. DPO is simpler and often stable offline, but remains bounded by fixed pair coverage.

Keep a frozen copy of raw pair-level scores. Aggregated accuracy can hide a small set of severe reversals, especially on policy-boundary prompts.

Part 2 - DPO equation

Let

$$\Delta_\theta = \log \pi_\theta(y_w \mid x) - \log \pi_\theta(y_l \mid x)$$

and define Δ_{ref} analogously. Then

$$L_{\text{DPO}} = -\log \sigma \left(\beta(\Delta_\theta - \Delta_{\text{ref}})\right).$$

Report chosen and rejected rewards, margin accuracy, KL-related drift proxies, and held-out behavior rather than loss alone.

Part 3 - Correctness protocol

1. Freeze the reference model and prove it receives no gradients.
2. Compute response-only sequence log-probabilities for all four branches.
3. Verify initial policy-reference identity gives zero relative margins.
4. Swap chosen and rejected responses and confirm the loss changes as expected.
5. Compare vectorized loss with a manual one-pair calculation.
6. Sweep β and monitor preference accuracy, diversity, and retention.
7. Evaluate on held-out prompt families and policy edge cases.

Part 4 - Explicit PyTorch DPO loss

```
import torch
import torch.nn.functional as F

def response_log_probability(logits, labels, response_mask):
    prediction_logits = logits[:, :-1, :]
    target_ids = labels[:, 1:]
    valid_mask = response_mask[:, 1:].bool()
    log_probabilities = F.log_softmax(prediction_logits, dim=-1)
    selected = log_probabilities.gather(
        dim=-1, index=target_ids.unsqueeze(-1)
```

```
    ).squeeze(-1)
    selected = selected * valid_mask
    return selected.sum(dim=-1)

def dpo_loss(policy_chosen, policy_rejected,
             reference_chosen, reference_rejected, beta):
    policy_margin = policy_chosen - policy_rejected
    reference_margin = reference_chosen - reference_rejected
    relative_margin = beta * (policy_margin - reference_margin)
    losses = -F.logsigmoid(relative_margin)
    preference_accuracy = (relative_margin > 0).float().mean()
    return losses.mean(), preference_accuracy, relative_margin

def E04(policy_scores, reference_scores, beta):
    return dpo_loss(
        policy_scores["chosen"], policy_scores["rejected"],
        reference_scores["chosen"], reference_scores["rejected"], beta
    )
```

Part 5 - The Philosophy Behind the Exercise

DPO shows that alignment can be framed as relative movement: prefer this answer over that one more strongly than the original model did. The reference provides a historical baseline, while the pair supplies a local direction.

Simplicity in optimization does not simplify values. The model still inherits the limits of who compared the answers, which alternatives were shown, and what the rubric could express. Direct optimization makes this dependency clearer, not absent.

18.6.6 Exercise 05

Exercise Title: Train Tiny Model with DPO and Observe Preference Shift

Part 1 - Deep conceptual foundation

A tiny DPO experiment should prove that preference gradients move conditional probability in the intended direction without claiming general alignment. Use a synthetic task with harmless, objectively ranked responses: correct versus incorrect formatting, grounded versus unsupported answers, or concise relevant answers versus irrelevant text. Keep a held-out set of prompt structures.

Begin policy and reference from identical weights. Freeze the reference and train only the policy. Record pre-training chosen and rejected log-probabilities, then measure their margin throughout training. Preference accuracy on training pairs can reach one while held-out behavior worsens, so track validation margin, response diversity, base retention, and actual generation.

Overfit a handful of pairs as a pipeline sanity check, then increase diversity. If loss does not fall, inspect masks, target shifts, reference gradients, and chosen/rejected ordering. If margins explode, reduce learning rate or beta and check whether one response-length pattern dominates.

Generation analysis should follow probability analysis. A margin can improve without changing greedy output when both candidates remain below another token sequence, while a

small movement can cross a decoding boundary and alter the visible answer. Save response-level scores and decoded output for the same prompts. Include pairs whose candidates share long prefixes and pairs that diverge immediately, because credit assignment differs. Finally, compare one-pass evaluation with repeated sampling: alignment that appears only under greedy decoding may disappear when temperature exposes other modes.

Part 2 - Measured shift

For pair i, define $m_i^{(s)} = \Delta_{\theta_s} - \Delta_{\text{ref}}$. The observed shift is $m_i^{(S)} - m_i^{(0)}$. Report the distribution, not only its mean, and require held-out preference accuracy above a declared threshold.

Part 3 - Training controls

1. Clone identical policy and reference states and hash them.
2. Assert reference parameters are frozen and unchanged after training.
3. Train first on one manually checked pair, then a varied synthetic set.
4. Log DPO loss, relative margin, gradient norm, and held-out accuracy.
5. Generate with fixed decoding before and after tuning.
6. Evaluate retention and diversity to detect narrow optimization.
7. Repeat across seeds and report failed runs.

Part 4 - Explicit DPO training step

```
import torch

def dpo_training_step(policy, reference, optimizer, batch, beta):
    optimizer.zero_grad(set_to_none=True)
    policy_chosen = policy.score_response(batch["chosen"])
    policy_rejected = policy.score_response(batch["rejected"])
    with torch.no_grad():
        reference_chosen = reference.score_response(batch["chosen"])
        reference_rejected = reference.score_response(batch["rejected"])
    loss, accuracy, margin = dpo_loss(
        policy_chosen, policy_rejected,
        reference_chosen, reference_rejected, beta
    )
    loss.backward()
    gradient_norm = torch.nn.utils.clip_grad_norm_(policy.parameters(), 1.0)
    optimizer.step()
    return {
        "loss": float(loss.detach().item()),
        "preference_accuracy": float(accuracy.detach().item()),
        "mean_margin": float(margin.detach().mean().item()),
        "gradient_norm": float(gradient_norm)
```

```
    }

def E05(policy, reference, optimizer, batches, beta):
    history = []
    for batch in batches:
        history.append(dpo_training_step(
            policy, reference, optimizer, batch, beta
        ))
    return history
```

Part 5 - The Philosophy Behind the Exercise

Preference optimization does not insert a rule into the model; it reshapes the relative likelihood of alternatives. Watching margins move makes that mechanism visible and prevents the word "alignment" from hiding an ordinary statistical update.

The tiny experiment also marks the boundary of evidence. Learning the shown pairs proves optimization works, not that the model will make wise choices where preferences were never represented.

18.6.7 Exercise 06

Exercise Title: Analyze Over-Alignment and Mode Collapse Failure Cases

Part 1 - Deep conceptual foundation

Over-alignment occurs when pressure toward a desired behavior becomes excessive or poorly generalized. A model may refuse harmless questions, repeat policy language unnecessarily, suppress uncertainty, or force every answer into one style. Mode collapse is the loss of useful response diversity: different prompts receive nearly identical, generic continuations.

These failures can coexist with improving preference accuracy because the data may reward safe-looking phrases, short answers, or one dominant template. Detect them with paired benign and risky prompts, paraphrases, creative tasks, and requests that legitimately require clarification. Measure refusal precision and recall separately; minimizing unsafe compliance by refusing everything is not a useful solution.

Monitor response entropy, distinct n-grams, semantic similarity across unrelated prompts, response length, and repeated-policy phrase frequency. Compare against the base and SFT checkpoints. Remedies include weaker preference strength, better-balanced pairs, explicit benign boundary examples, diversity-aware evaluation, and stopping before collapse.

Mode collapse also has a conditional form. Global diversity may remain high while every prompt inside one category receives the same canned response. Compute diversity within categories and among minimal prompt pairs. Test context use by changing one relevant fact and checking whether the answer changes. A generic safe answer has not merely lost creativity; it has stopped responding to evidence. Recovery experiments should restart from a pre-collapse checkpoint, because continued optimization may not restore displaced modes.

Part 2 - Refusal and diversity metrics

For safe requests, false refusal rate is $\mathrm{FRR} = \mathrm{FP}/(\mathrm{FP}+\mathrm{TN})$; for disallowed requests, unsafe compliance rate is $\mathrm{UCR} = \mathrm{FN}/(\mathrm{FN}+\mathrm{TP})$. Both must be reported. Diversity can include distinct-n and pairwise semantic similarity, with the latter inspected by task slice.

Part 3 - Failure study

1. Build matched benign, disallowed, ambiguous, and clarification-required prompts.
2. Freeze decoding settings and compare checkpoints.
3. Measure false refusal, unsafe compliance, answer usefulness, and diversity.
4. Search for repeated boilerplate and length collapse.
5. Plot failures against beta, KL strength, and training step.
6. Add boundary examples and verify both sides improve.
7. Preserve representative false refusals and unsafe responses for review.

Part 4 - Collapse monitor

```
def refusal_metrics(labels, predictions):
    false_refusal = 0
    unsafe_compliance = 0
    safe_total = 0
    disallowed_total = 0
    for index in range(len(labels)):
        if labels[index] == "safe":
            safe_total = safe_total + 1
            if predictions[index] == "refuse":
                false_refusal = false_refusal + 1
        if labels[index] == "disallowed":
            disallowed_total = disallowed_total + 1
            if predictions[index] == "comply":
                unsafe_compliance = unsafe_compliance + 1
    return {
        "false_refusal_rate": false_refusal / max(safe_total, 1),
        "unsafe_compliance_rate": unsafe_compliance / max(disallowed_total, 1)
    }

def E06(labels, predictions, responses):
    metrics = refusal_metrics(labels, predictions)
    unique_responses = len(set(responses))
    response_count = max(len(responses), 1)
    metrics["unique_response_ratio"] = unique_responses / response_count
    return metrics
```

Part 5 - The Philosophy Behind the Exercise

Safety without usefulness becomes silence; usefulness without boundaries becomes recklessness. Over-alignment demonstrates that optimizing one visible virtue can destroy the balance that gave the system value.

Plural prompts deserve plural answers. Diversity is not noise to eliminate but part of responsive intelligence, constrained by truth and policy rather than replaced by one universally safe-sounding mode.

18.6.8 Exercise 07

Exercise Title: Implement Safety Filters and Refusal Policies Conceptually

Part 1 - Deep conceptual foundation

Production safety is layered. Input classification, model policy, tool permissions, output checks, rate limits, monitoring, and human escalation protect different boundaries. A keyword blocklist alone misses context and produces false positives; a generative model alone should not hold unrestricted authority over high-impact actions.

Define a policy taxonomy with allowed, disallowed, and ambiguous cases. For allowed requests, answer normally. For clearly disallowed requests, refuse briefly without adding enabling details and offer safe alternatives when useful. For ambiguity or high-stakes contexts, request clarification or route to review. Log decisions with privacy-preserving identifiers and policy versions.

Filters must fail safely but not invisibly. Timeouts, classifier uncertainty, and contradictory signals require explicit fallback. Evaluate adversarial paraphrases and benign terms that resemble risky vocabulary. This exercise uses abstract categories and synthetic content so it tests routing without providing harmful instructions.

The output path needs equivalent care. A response may begin safely and later cross a boundary, or combine harmless fragments into an unsafe whole. Streaming systems need incremental checks and a policy for stopping, replacing, or escalating partial output. Tool calls require schema validation and authorization independent of natural-language safety. Record which layer made the final decision so failures can be attributed. Disagreement between an input classifier and model refusal is diagnostic evidence, not something to suppress.

Calibrate the classifier on deployment-like prevalence. Thresholds selected on a balanced laboratory set can behave very differently when risky requests are rare, so report precision, recall, and absolute review volume.

Part 2 - Decision policy

Given risk probability p and uncertainty u, a simple policy can use two thresholds: allow below τ_a, refuse above τ_r, and review between them. Thresholds must be calibrated per severity because equal error rates do not have equal impact.

Part 3 - Layered validation

1. Version categories, thresholds, refusal templates, and escalation rules.
2. Separate content classification from action authorization.
3. Test allowed, disallowed, ambiguous, multilingual, quoted, and transformed requests.
4. Measure false refusal and unsafe compliance by severity.
5. Simulate classifier timeout and disagreement between layers.
6. Ensure logs exclude sensitive prompt contents unless authorized.
7. Verify safe alternatives do not accidentally reproduce prohibited detail.

Part 4 - Defensive policy router

```
class SafetyDecision:
    def __init__(self, action, category, confidence, policy_version):
        self.action = action
        self.category = category
        self.confidence = confidence
        self.policy_version = policy_version

def route_request(classification, allow_threshold, refuse_threshold):
    probability = classification["risk_probability"]
    category = classification["category"]
    if probability >= refuse_threshold:
        action = "refuse"
    elif probability <= allow_threshold:
        action = "allow"
    else:
        action = "review"
    return SafetyDecision(
        action, category, classification["confidence"], "policy_v1"
    )

def E07(classifier, requests, allow_threshold, refuse_threshold):
    decisions = []
    for request in requests:
        classification = classifier(request)
        decision = route_request(
            classification, allow_threshold, refuse_threshold
        )
        decisions.append(decision)
    return decisions
```

Part 5 - The Philosophy Behind the Exercise

A refusal is an exercise of system authority. It should be explainable, proportionate, consistent, and open to escalation where context matters. Safety cannot be reduced to the model saying "no" more often.

Layering recognizes fallibility. No classifier, model, or rule is complete, so responsibility is distributed across controls whose failures can be observed and contained.

18.6.9 Exercise 08

Exercise Title: Evaluate Helpfulness vs Harmlessness Tradeoff

Part 1 - Deep conceptual foundation

Helpfulness and harmlessness are not simple opposites. Many requests can be answered fully and safely; others require boundaries, uncertainty, or safer alternatives. The tradeoff appears when a control reduces one failure while creating another, such as broad refusal that blocks legitimate education.

Construct matched prompt families near policy boundaries. Score relevance, correctness, completeness, safe handling, refusal appropriateness, and quality of alternatives. Do not average

away hard constraints: a highly helpful unsafe response fails, while a perfectly harmless empty response also fails the service goal.

Compare thresholds or checkpoints on a Pareto frontier. A candidate is dominated if another is at least as helpful and at least as harmless while strictly better on one. Choose operating points using severity-weighted constraints and document the affected users of false refusals.

Grading should account for safe transformation rather than only compliance or refusal. A model may answer the legitimate portion of a mixed request, explain a boundary, or redirect toward prevention. These bounded responses can be more helpful than blanket refusal without increasing risk. Add a metric for alternative usefulness and inspect it separately. Measure calibration as well: the system should express uncertainty when safety classification depends on missing context rather than forcing a confident decision at every boundary.

Measure consistency across languages and levels of technical expertise. A policy that offers useful bounded guidance in one language but defaults to refusal in another creates an unequal service even if the global average is unchanged.

Part 2 - Constrained optimization

For helpfulness H and harm risk R, choose

$$\max H(\theta) \quad \text{subject to} \quad R_c(\theta) \leq \tau_c$$

for every critical category c. This is safer than maximizing $H - \lambda R$ when a large helpfulness gain could numerically compensate for a severe failure.

Part 3 - Evaluation design

1. Define separate rubrics and blinded graders for helpfulness and safety.
2. Include benign, boundary, disallowed, and high-stakes uncertainty cases.
3. Evaluate complete answers, bounded answers, refusals, and safe alternatives.
4. Report per-category rates and severity, not only a global mean.
5. Plot candidate checkpoints and thresholds on a Pareto chart.
6. Audit false refusals for unequal impact across languages and groups.
7. Select an operating point with explicit noncompensable safety gates.

Part 4 - Pareto comparison

```
def is_dominated(candidate, alternatives):
    for alternative in alternatives:
        helpful_not_worse = (
            alternative["helpfulness"] >= candidate["helpfulness"]
        )
        safety_not_worse = (
            alternative["safety"] >= candidate["safety"]
        )
        strictly_better = (
```

```
            alternative["helpfulness"] > candidate["helpfulness"] or
            alternative["safety"] > candidate["safety"]
        )
        if helpful_not_worse and safety_not_worse and strictly_better:
            return True
    return False

def E08(candidates):
    frontier = []
    for candidate in candidates:
        if not is_dominated(candidate, candidates):
            frontier.append(candidate)
    return frontier
```

Part 5 - The Philosophy Behind the Exercise

Helpful behavior respects human purpose; harmless behavior respects human vulnerability. Treating either as absolute can erase the other. Alignment is the craft of preserving both wherever possible and making true conflicts visible.

A Pareto frontier does not make the ethical decision. It prevents a hidden weighted average from making it silently. People remain responsible for the operating point and its unequal consequences.

18.6.10 Exercise 09

Exercise Title: Debug Alignment Training Instability from Noisy Preference Data

Part 1 - Deep conceptual foundation

Preference noise arises from disagreement, weak rubrics, annotation mistakes, position bias, hidden response differences, and genuinely ambiguous values. In DPO or reward modeling, contradictory pairs produce competing gradients. Loss may oscillate, margins may become extreme, and validation preferences may stop improving even as training accuracy rises.

Diagnose data before tuning optimization. Measure duplicate conflicts, annotator consistency, confidence, pair difficulty, length bias, and agreement by category. Inspect the highest-loss pairs; they often reveal misordered candidates or underspecified prompts. Preserve ties and abstentions rather than randomly turning them into hard labels.

Possible interventions include adjudication, removing invalid pairs, weighting by confidence, soft preference targets, robust sampling, lower learning rate, and stronger reference regularization. Every cleaning rule changes whose judgment counts, so report performance across annotator groups and difficult slices.

Distinguish disagreement from pipeline corruption. If experts consistently divide on a value tradeoff, more labels estimate the distribution but do not create one objectively correct ranking. If an annotator reverses an identical pair, the issue is reliability. Use probabilistic targets for the first and quality-control investigation for the second. Track effective sample size after weighting: a large raw dataset can become statistically tiny when most weight concentrates on a few high-confidence pairs.

After cleaning, compare the retained distribution with the original. Removing every disputed example can create an artificially easy dataset that teaches only obvious preferences and fails precisely where judgment is most needed.

Part 2 - Soft-label loss

For empirical preference probability q_i and predicted probability p_i,

$$L_i = -q_i \log p_i - (1 - q_i) \log(1 - p_i).$$

When $q_i \approx 0.5$, the pair carries uncertainty rather than a strong ranking. Confidence weights w_i may scale loss, but must be calibrated and audited.

Part 3 - Debugging protocol

1. Freeze a clean validation subset with expert-reviewed pairs.
2. Find duplicate, reverse, cyclic, and cross-rubric conflicts.
3. Plot pair loss by confidence, category, annotator group, and length gap.
4. Manually inspect high-loss and high-gradient examples.
5. Compare hard labels, soft labels, and confidence weighting.
6. Retest stability across seeds and data order.
7. Report which judgments were excluded and why.

Part 4 - Conflict detector

```
def find_pair_conflicts(records):
    direction_by_pair = {}
    conflicts = []
    for record in records:
        first = record["chosen_id"]
        second = record["rejected_id"]
        ordered = tuple(sorted([first, second]))
        direction = (first, second)
        if ordered in direction_by_pair:
            previous = direction_by_pair[ordered]
            if previous != direction:
                conflicts.append({
                    "pair_key": ordered,
                    "first_direction": previous,
                    "second_direction": direction
                })
        else:
            direction_by_pair[ordered] = direction
    return conflicts

def E09(records):
    conflicts = find_pair_conflicts(records)
    conflict_rate = len(conflicts) / max(len(records), 1)
    return conflicts, conflict_rate
```

Part 5 - The Philosophy Behind the Exercise

Disagreement can be error, ambiguity, or plural values. Treating all three as identical noise destroys information; treating every judgment as equally certain creates unstable learning.

Debugging preference data is therefore partly an inquiry into meaning. The model cannot resolve contradictions that the dataset hides. Engineers must surface them before optimization turns them into unexplained gradients.

18.6.11 Exercise 10

Exercise Title: Run Preference Training on CUDA and Benchmark Memory Costs

Part 1 - Deep conceptual foundation

Preference training is memory intensive because each pair requires chosen and rejected policy computations and often reference computations. A naive implementation keeps several graphs alive simultaneously. Reference forward passes must use no-grad mode, while policy branches can be concatenated into one batch to reduce launch overhead. Sequence-length imbalance and padding can waste substantial attention computation.

Measure parameters, gradients, optimizer state, activations, and allocator peaks under fixed pair count and sequence lengths. Compare full tuning with LoRA, policy-only versus cached reference log-probabilities, activation checkpointing, and precision modes. Cached reference scores save computation but are valid only for an immutable reference and identical tokenization.

Report pairs per second, total tokens per second, response tokens per second, peak allocated memory, reserved memory, and padding efficiency. Warm up kernels, reset peaks, synchronize only around measurement, and record out-of-memory conditions as results.

Benchmarking should separate forward-only reference cost from policy backward cost. Use profiler ranges around transfer, policy branches, reference branches, loss, backward, and optimizer update. This reveals whether cached scores, packed pairs, or adapters address the actual bottleneck. Under mixed precision, compare response log-probabilities and pair ordering against float32. A faster run that changes preference ordering through overflow is a different experiment, not a valid optimization.

Include checkpoint save time and recovery memory in production-oriented tests. A configuration that barely fits training may fail when serialization or evaluation temporarily creates additional tensors.

Part 2 - Cost model

If policy processes two branches and reference scores are computed online, forward work is approximately four sequence passes per pair, with backward only through two policy branches. Actual cost depends on shared batching, padding, kernel fusion, and adapter placement.

Part 3 - Benchmark protocol

1. Verify all policy tensors share one CUDA device and the reference is frozen.
2. Warm up, reset memory statistics, and synchronize before timing.
3. Match token shapes across full and LoRA comparisons.
4. Compare online and cached reference scores with parity tests.
5. Measure useful response-token throughput as well as padded throughput.

6. repeat each configuration and report dispersion.

7. Store failed memory configurations in the result table.

Part 4 - CUDA benchmark skeleton

```
import time
import torch

def E10(training_step, batches, device):
    if device.type != "cuda":
        raise ValueError("CUDA device required")
    for index in range(min(2, len(batches))):
        training_step(batches[index])
    torch.cuda.synchronize(device)
    torch.cuda.reset_peak_memory_stats(device)
    start = time.perf_counter()
    pairs = 0
    response_tokens = 0
    for index in range(2, len(batches)):
        batch = batches[index]
        result = training_step(batch)
        pairs = pairs + result["pair_count"]
        response_tokens = response_tokens + result["response_tokens"]
    torch.cuda.synchronize(device)
    elapsed = time.perf_counter() - start
    return {
        "pairs_per_second": pairs / elapsed,
        "response_tokens_per_second": response_tokens / elapsed,
        "peak_allocated": torch.cuda.max_memory_allocated(device),
        "peak_reserved": torch.cuda.max_memory_reserved(device)
    }
```

Part 5 - The Philosophy Behind the Exercise

Preference data doubles alternatives because alignment depends on comparison. The hardware cost is the physical expression of asking not merely "what did the model say?" but "which answer should become more likely?"

Efficiency matters because expensive evaluation loops are often shortened first. A faster pipeline should create room for more evidence, not justify weaker oversight.

18.6.12 Exercise 11

Exercise Title: Compare DPO vs PPO Stability on Small Synthetic Task

Part 1 - Deep conceptual foundation

DPO is an offline supervised-style objective over fixed preference pairs. PPO is an on-policy reinforcement-learning method that samples responses, scores them, estimates advantages, and

applies clipped policy updates. PPO can adapt to the current policy distribution, but introduces rollout variance, reward-model dependence, value estimation, and more hyperparameters.

Compare them on a harmless synthetic task with a known reward, such as satisfying format and correctness constraints. Give both methods the same initial policy, token budget, evaluation prompts, and wall-clock accounting. DPO receives fixed pairs; PPO receives sampled trajectories, so equal optimizer steps are not equal evidence. Report both processed tokens and reward evaluations.

Stability includes variance across seeds, KL drift, reward oscillation, collapse, gradient norms, and retained behavior. PPO clipping limits individual updates but does not guarantee global safety. DPO can be stable yet overfit the fixed pair distribution.

The comparison must expose hidden infrastructure. PPO needs generation, reward inference, old-policy probabilities, advantage estimation, and often a value model; DPO needs paired data and a reference pass. Count model copies, stored tensors, reward calls, and human labels, not only optimizer time. For PPO, verify padding and terminal tokens do not enter advantages incorrectly. For DPO, verify response-only sequence scores. Operational stability includes recovery, checkpointing, and monitoring, not merely a smooth loss curve.

Use identical downstream graders but method-appropriate diagnostics. PPO needs advantage and clipping statistics, whereas DPO needs pair margins. Forcing both into one loss-based dashboard would conceal their characteristic failure modes.

Part 2 - PPO and DPO contrast

PPO uses ratio $r_t(\theta) = \pi_\theta(a_t|s_t)/pi_{\text{old}}(a_t|s_t)$ and clipped surrogate

$$L_{\text{PPO}} = -\mathbb{E}\left[\min(r_t A_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon)A_t)\right].$$

DPO uses the offline relative margin from E04. Compare pipeline complexity as well as scalar performance.

Part 3 - Matched study

1. Use an exactly computable synthetic reward and hidden test prompts.
2. Match initialization, tokenizer, total generated or trained tokens, and evaluation schedule.
3. Tune both methods under equal search budgets.
4. Record reward, task correctness, KL, entropy, and retention.
5. Measure variance and failure count across seeds.
6. Inject mild reward noise and compare robustness.
7. Explain when online exploration is worth PPO's complexity.

Part 4 - Stability summary

```
import statistics

def summarize_runs(run_metrics):
    rewards = []
```

```
    divergences = []
    failures = 0
    for run in run_metrics:
        rewards.append(run["final_reward"])
        divergences.append(run["final_kl"])
        if run["failed"]:
            failures = failures + 1
    return {
        "reward_mean": statistics.mean(rewards),
        "reward_standard_deviation": statistics.stdev(rewards),
        "kl_mean": statistics.mean(divergences),
        "failed_runs": failures
    }

def E11(dpo_runs, ppo_runs):
    return {
        "dpo": summarize_runs(dpo_runs),
        "ppo": summarize_runs(ppo_runs)
    }
```

Part 5 - The Philosophy Behind the Exercise

Optimization methods embody different relationships with evidence. DPO learns from judgments already collected; PPO changes behavior, observes new outcomes, and learns again. One is simpler and bounded, the other adaptive and volatile.

The correct choice is not ideological. It depends on whether fresh exploration provides information valuable enough to justify its cost and risk.

18.6.13 Exercise 12

Exercise Title: Implement Reward Hacking Scenario and Detect It Automatically

Part 1 - Deep conceptual foundation

Reward hacking occurs when a policy maximizes a proxy without achieving the intended outcome. In a safe synthetic scenario, reward a response for containing a required tag while the real task also requires a correct answer. A policy may repeat the tag and ignore correctness, demonstrating exploitation without any harmful content.

Detection requires independent ground-truth metrics that were not optimized by the policy. Monitor proxy reward against task success, response length, repeated phrases, diversity, and out-of-distribution performance. A rising reward paired with flat or falling true success is a central warning.

Use canary prompts and adversarial proxy tests. Compare reward-model confidence with human or deterministic validation. Stop training when divergence exceeds a threshold. Do not "fix" the proxy using the same examples repeatedly without creating a new hidden evaluation set.

Reward hacking can appear through distribution shift before obvious metric divergence. The policy may discover outputs unlike reward-training responses. Track feature distance, reward-model ensemble disagreement, and sensitivity to harmless paraphrases. Extremely high reward in a low-density region deserves inspection, not celebration. Maintain a hidden evaluator based

on the true task and add newly discovered exploits to regression only after measuring them on an untouched checkpoint.

Test the monitor with an injected exploit checkpoint. An alarm that looks reasonable on ordinary learning curves has not proved it can detect the failure for which it was designed.

Part 2 - Proxy gap

For proxy reward $\widehat{R}$ and true task metric R^*, define

$$G = \text{normalize}(\widehat{R}) - \text{normalize}(R^*).$$

Large positive growth in G indicates optimization is outrunning validated utility. Correlation across checkpoints is useful but does not replace slice analysis.

Part 3 - Detection experiment

1. Define a harmless task with deterministic true success.
2. Create a deliberately incomplete proxy based on a superficial feature.
3. Optimize the proxy and evaluate true success at every checkpoint.
4. Monitor length, repetition, diversity, and unseen prompts.
5. Trigger an automatic stop when proxy rises while true success falls.
6. Repair the reward with diverse counterexamples and retest on hidden data.
7. Document the exploit pattern without operational harmful content.

Part 4 - Automatic proxy-gap alarm

```
class RewardHackingMonitor:
    def __init__(self, maximum_gap, patience):
        self.maximum_gap = maximum_gap
        self.patience = patience
        self.bad_steps = 0

    def update(self, proxy_score, true_score):
        gap = proxy_score - true_score
        if gap > self.maximum_gap:
            self.bad_steps = self.bad_steps + 1
        else:
            self.bad_steps = 0
        return {
            "proxy_gap": gap,
            "stop": self.bad_steps >= self.patience
        }

def E12(checkpoints, maximum_gap, patience):
    monitor = RewardHackingMonitor(maximum_gap, patience)
```

```
    history = []
    for checkpoint in checkpoints:
        result = monitor.update(
            checkpoint["proxy_score"], checkpoint["true_score"]
        )
        history.append(result)
        if result["stop"]:
            break
    return history
```

Part 5 - The Philosophy Behind the Exercise

Optimization is literal. It pursues what is measured, not the intention that motivated the measurement. Reward hacking is therefore not disobedience; it is the exposure of an incomplete specification.

Independent evaluation protects meaning from its proxy. When the two diverge, the correct response is to question the measurement before blaming the model.

18.6.14 Exercise 13

Exercise Title: Build Alignment Evaluation Set Focused on Edge-Case Prompts

Part 1 - Deep conceptual foundation

Average prompts rarely reveal alignment boundaries. Edge cases combine ambiguity, conflicting instructions, quoted unsafe material, benign educational context, missing information, multilingual wording, prompt injection, and requests near a policy threshold. A strong suite tests whether behavior changes for the right reason.

Build a taxonomy before authoring prompts. Create matched pairs differing in one meaningful feature, such as authorization or intent, and paraphrase each pair. Expected behavior can be answer, bounded answer, clarification, refusal, or escalation. Store the policy clause and rationale that justify the label.

Avoid releasing detailed harmful prompts in the training book. Use abstract placeholders and controlled synthetic examples while preserving the evaluation logic. Keep a protected test subset to reduce tuning to the benchmark.

Edge cases also need compositional tests. A system may handle quoted material, multilingual wording, and conflicting instructions separately but fail when two features appear together. Build a small factorial design that combines factors without exploding the suite, then use failure clustering to prioritize new combinations. Include multi-turn cases where intent becomes clear only after a clarification. Preserve conversation state and verify that a safe decision does not depend on omitting earlier turns during evaluation.

Maintain grader disagreement as a metric. Edge cases are often where policy language itself is ambiguous, and apparent model inconsistency may reveal an unstable ground-truth rule.

Part 2 - Slice reliability

For slice s, failure estimate is $\widehat{p}_s = k_s/n_s$. Report sample size and a confidence interval; a zero observed failure in a tiny slice is not proof of safety. Track paired consistency between semantic variants.

Part 3 - Suite construction

1. Derive categories from threat models, incidents, and policy boundaries.
2. Author minimal pairs with one controlled semantic difference.
3. Assign expected action, rationale, severity, and grader.
4. Review labels with safety and domain experts.
5. Separate development, regression, and protected test sets.
6. Measure false refusal, unsafe compliance, clarification quality, and paraphrase consistency.
7. Version every item and retire invalid prompts without rewriting history.

Part 4 - Versioned edge-case registry

```
def validate_edge_case(item):
    required = ["item_id", "category", "prompt", "expected_action",
                "policy_clause", "severity"]
    for field_name in required:
        if field_name not in item:
            raise ValueError("missing field: " + field_name)
    actions = ["answer", "bounded_answer", "clarify", "refuse", "escalate"]
    if item["expected_action"] not in actions:
        raise ValueError("invalid expected action")

def E13(items):
    identifiers = set()
    by_category = {}
    for item in items:
        validate_edge_case(item)
        if item["item_id"] in identifiers:
            raise ValueError("duplicate edge-case identifier")
        identifiers.add(item["item_id"])
        category = item["category"]
        if category not in by_category:
            by_category[category] = []
        by_category[category].append(item)
    return by_category
```

Part 5 - The Philosophy Behind the Exercise

Boundaries reveal principles more clearly than ordinary cases. When two prompts differ by one crucial fact, the model's different treatment shows whether it has learned a policy distinction or a superficial trigger.

Edge-case evaluation is not a search for perfection. It is a disciplined way to discover where confidence should end and human judgment should begin.

18.6.15 Exercise 14

Exercise Title: Analyze Hallucination Reduction After Alignment Tuning

Part 1 - Deep conceptual foundation

Alignment tuning can reduce some hallucinations by rewarding calibrated uncertainty, evidence use, and appropriate abstention. It cannot guarantee truth because preference objectives optimize judged behavior, and annotators may prefer fluent incorrect answers. A model can also appear improved by refusing more often, shifting error from false claims to unnecessary abstention.

Use questions with known answers, unanswerable questions, context-grounded questions, and misleading premises. Separate factual correctness, citation or evidence support, calibration, abstention accuracy, and usefulness. Compare base, SFT, and preference-tuned checkpoints under identical decoding.

Claims require atomic evaluation. Split a response into verifiable statements and label each supported, contradicted, or unverifiable. Measure coverage so a model is not rewarded for giving empty answers. For context tasks, ensure the answer is supported by supplied evidence rather than memorized coincidence.

Preference tuning may alter verbal confidence without altering internal probability calibration. Measure token-level confidence, self-reported uncertainty, and empirical correctness separately. Test whether cited sources exist and support the associated claim rather than rewarding citation-shaped text. For temporal questions, attach an evaluation date and evidence snapshot; otherwise later knowledge changes can make the same answer appear alternately correct and incorrect. Human review should focus on fluent contradictions that automatic lexical metrics miss.

Repeat the study with sampling as well as greedy decoding. Preference tuning can reshape probability tails even when the most likely answer remains unchanged, which matters for repeated user interactions.

Part 2 - Selective accuracy

Let coverage be the fraction answered and risk be error among answered cases. Plot risk versus coverage as the abstention threshold changes. Hallucination rate alone is incomplete without answer rate and utility.

Part 3 - Before-and-after study

1. Freeze answerable, unanswerable, grounded, and false-premise slices.
2. Use identical prompts and deterministic decoding for paired comparison.
3. Score atomic claims, evidence support, abstention, and coverage.
4. Plot risk-coverage curves from confidence or uncertainty signals.
5. Inspect whether reduced hallucination is merely increased refusal.
6. Test domains absent from preference training.
7. Report convincing aligned-model failures, not only average improvement.

Part 4 - Claim-level summary

```
def summarize_claims(records):
    supported = 0
    contradicted = 0
    unverifiable = 0
    answered = 0
    for record in records:
        if record["answered"]:
            answered = answered + 1
        for status in record["claim_statuses"]:
            if status == "supported":
                supported = supported + 1
            elif status == "contradicted":
                contradicted = contradicted + 1
            else:
                unverifiable = unverifiable + 1
    total_claims = supported + contradicted + unverifiable
    return {
        "support_rate": supported / max(total_claims, 1),
        "contradiction_rate": contradicted / max(total_claims, 1),
        "coverage": answered / max(len(records), 1)
    }

def E14(base_records, aligned_records):
    return summarize_claims(base_records), summarize_claims(aligned_records)
```

Part 5 - The Philosophy Behind the Exercise

Alignment can teach a model how to behave when it does not know; it cannot make knowledge unlimited. Calibrated uncertainty is valuable because it makes the boundary of evidence visible.

Truthfulness is not silence. A model earns trust by answering when supported, abstaining when necessary, and distinguishing the two without theatrical confidence.

18.6.16 Exercise 15

Exercise Title: Implement Red-Teaming Prompts and Measure Failure Rates

Part 1 - Deep conceptual foundation

Red teaming is authorized adversarial evaluation intended to discover failures before deployment or abuse. It needs a written scope, safe test environment, data-handling rules, stop conditions, severity rubric, and disclosure path. The goal is not to generate operationally harmful material but to test categories, routing, robustness, and containment using controlled abstractions.

Prompts should cover direct requests, paraphrases, role-play, quoted content, multi-turn pressure, instruction conflicts, encoded or transformed benign proxies, and tool-boundary attempts. Record the attack pattern separately from sensitive payloads. A failure is defined by policy and impact: unsafe compliance, privacy leakage, incorrect tool authorization, excessive refusal, or unreliable escalation.

Measure attack success by category and severity, not only total failures. Count unique root causes so hundreds of paraphrases do not create a false sense of independent evidence. After mitigation, rerun the original regression set and a hidden variation set to detect benchmark-specific patching.

The harness should support reproducible replay. Store model and policy versions, decoding configuration, tool permissions, environment, and hashes of protected cases. Remove personal data and restrict raw outputs according to severity. When testers discover several prompts with one root cause, keep representative variants but avoid treating them as independent evidence. Track time to triage, time to mitigation, and recurrence after release; operational response quality is part of the red-team result.

Include control prompts that should always pass. If the harness flags them, the grader or policy interpretation may be faulty, and reported attack success can be inflated.

Part 2 - Failure estimates

For category c, attack success rate is $\mathrm{ASR}_c = k_c/n_c$. Report confidence intervals and severity-weighted counts. The residual risk decision must also consider detectability and exposure, not just laboratory frequency.

Part 3 - Authorized test protocol

1. Obtain scope, ownership, environment, and stop conditions before testing.
2. Use synthetic placeholders for harmful payloads in general documentation.
3. Tag technique, category, severity, expected behavior, and policy clause.
4. Run fixed decoding and production decoding as separate conditions.
5. Store sensitive outputs under restricted access.
6. Cluster failures by root cause and assign remediation owners.
7. Retest original and hidden variants after every mitigation.

Part 4 - Safe red-team harness

```
def evaluate_red_team_case(case, response, policy_grader):
    grade = policy_grader(case["category"], response)
    return {
        "case_id": case["case_id"],
        "category": case["category"],
        "severity": case["severity"],
        "failed": not grade["passed"],
        "reason_code": grade["reason_code"]
    }

def E15(cases, generate, policy_grader):
    results = []
    failures_by_category = {}
    totals_by_category = {}
```

```
    for case in cases:
        response = generate(case["prompt"])
        result = evaluate_red_team_case(case, response, policy_grader)
        results.append(result)
        category = result["category"]
        totals_by_category[category] = totals_by_category.get(category, 0) + 1
        if result["failed"]:
            previous_failures = failures_by_category.get(category, 0)
            failures_by_category[category] = previous_failures + 1
    return results, totals_by_category, failures_by_category
```

Part 5 - The Philosophy Behind the Exercise

Red teaming treats failure discovery as a form of care. It deliberately removes the comfort of ordinary examples so hidden assumptions become visible before people depend on them.

Adversarial testing gains legitimacy from authorization and containment. The same curiosity that finds weaknesses must be governed by responsibility for what the test creates and reveals.

18.6.17 Exercise 16

Exercise Title: Compare Alignment Strength Across Different Model Sizes

Part 1 - Deep conceptual foundation

Model size changes capability, optimization behavior, and the expression of alignment data. A larger model may understand nuanced policies and generalize preferences better, but it may also possess stronger capabilities that increase the impact of failures. Smaller models may underfit subtle distinctions and rely more on superficial refusal templates.

Compare sizes from the same architecture family with matched tokenizer, pretraining lineage where possible, preference dataset, evaluation suite, and search budget. Equal steps do not mean equal compute, while equal compute may give smaller models more data passes. Report both compute-matched and data-matched views when feasible.

Alignment "strength" must be decomposed into helpfulness, safe compliance, false refusal, policy generalization, robustness, calibration, and retention. Also test whether larger models hide failures behind more persuasive language. Do not infer a monotonic law from two checkpoints.

Capacity may change the data requirement itself. A small model can need more explicit boundary demonstrations, while a larger model may infer the rule from fewer examples but exploit subtle reward shortcuts. Measure learning curves over alignment-data size for every model rather than using one fixed final checkpoint. Also normalize adapter rank or trainable fraction carefully: equal LoRA rank does not represent equal relative capacity across widths. Record inference cost and latency because a behaviorally stronger model may be operationally inappropriate for the deployment setting.

Inspect error correlation across sizes. If every model fails the same prompts, the bottleneck may be shared data or policy ambiguity rather than capacity. If failures move between sizes, each deployment needs its own regression suite.

Part 2 - Scaling comparison

For model size N, fit descriptive curves only within observed ranges, for example $M(N) = a + b \log N$. Confidence intervals across seeds and prompt slices are essential; extrapolation beyond tested sizes must be labeled speculative.

Part 3 - Matched evaluation

1. Select at least three sizes from a consistent family.
2. Hold policy, data version, templates, and evaluation constant.
3. Report data-matched and compute-matched training conditions.
4. Tune learning rate and alignment strength per size under equal budgets.
5. Measure all behavior dimensions and per-severity failures.
6. Compare confidence, verbosity, and persuasiveness of incorrect outputs.
7. Avoid claims beyond the tested size and task range.

Part 4 - Size comparison table

```
def E16(model_results):
    required = ["parameters", "helpfulness", "safety",
                "false_refusal", "retention", "compute"]
    validated = []
    for result in model_results:
        for field_name in required:
            if field_name not in result:
                raise ValueError("missing field: " + field_name)
        validated.append(result)
    def parameter_count(item):
        return item["parameters"]
    validated.sort(key=parameter_count)
    return validated
```

Part 5 - The Philosophy Behind the Exercise

Capability and alignment do not scale as one substance. A larger model may interpret values more subtly while making every remaining mistake more capable and convincing.

Scale therefore expands both possibility and obligation. The evaluation burden should grow with what the model can do, not shrink because average behavior looks smoother.

18.6.18 Exercise 17

Exercise Title: Build Safety Checklist for Production LLM Deployment

Part 1 - Deep conceptual foundation

Production safety begins before deployment and continues after it. A checklist must cover model and data identity, intended use, policy, evaluation, access control, tool permissions,

privacy, monitoring, incident response, rollback, human escalation, and change management. Passing a benchmark is not a permanent safety property because users, data, attacks, and dependencies change.

Each item needs evidence, owner, severity, status, expiration, and remediation. Blocking checks cannot be averaged with optional improvements. Test the deployed system, including prompt templates, retrieval, tools, filters, caches, and logs; a safe isolated checkpoint can become unsafe through integration.

Monitoring should track drift, refusal patterns, policy violations, latency failures, anomalous tool calls, and user appeals while minimizing sensitive-data collection. Incident response needs clear authority to disable tools, reduce traffic, or roll back the model.

The checklist must distinguish launch readiness from ongoing authorization. Some evidence expires after a model, retrieval source, tool, or policy change. Define which modifications require full reevaluation and which permit targeted regression tests. Canary deployment should limit exposure while monitoring both automatic metrics and user-reported failures. Rollback artifacts must already exist and be tested; an emergency plan that requires building an old environment after the incident is not a reliable control.

Third-party dependencies need equivalent gates. A change in moderation service, retrieval corpus, model host, or analytics pipeline can alter safety without changing the checkpoint, so dependency versions belong in the release record.

Part 2 - Release gate

If blocking checks produce c_i, release requires $\bigwedge_{i \in B} c_i = 1$. Evidence has an expiry time because models and environments change; stale success cannot authorize a new release.

Part 3 - Checklist domains

1. Verify model, tokenizer, policy, code, data, and evaluation versions.
2. Approve intended uses, prohibited uses, user notices, and escalation.
3. Test safety, privacy, hallucination, bias, prompt injection, and tools.
4. Enforce least privilege, rate limits, sandboxing, and output validation.
5. Validate logging minimization, retention, access, and deletion procedures.
6. Rehearse rollback and incident communication.
7. Define monitoring thresholds, owners, and review cadence.

Part 4 - Executable release checklist

```
class SafetyCheck:
    def __init__(self, name, blocking, owner, evidence, passed):
        self.name = name
        self.blocking = blocking
        self.owner = owner
        self.evidence = evidence
        self.passed = passed
```

```
def E17(check_definitions):
    checks = []
    release_ready = True
    for definition in check_definitions:
        check = SafetyCheck(
            definition["name"], definition["blocking"],
            definition["owner"], definition["evidence"],
            definition["passed"]
        )
        checks.append(check)
        if check.blocking and not check.passed:
            release_ready = False
    return {"release_ready": release_ready, "checks": checks}
```

Part 5 - The Philosophy Behind the Exercise

Safety is not an adjective attached to a model; it is a maintained relationship between a system, its powers, its users, and its environment. That relationship can decay when any part changes.

A checklist makes responsibility durable. It ensures that the ability to launch is accompanied by the ability to notice harm, stop operation, and repair trust.

18.6.19 Exercise 18

Exercise Title: Document Ethical Risks and Mitigations for Your LLM

Part 1 - Deep conceptual foundation

Ethical risk documentation examines who benefits, who can be harmed, which values are encoded, and where technical controls are insufficient. Relevant areas include privacy, consent, intellectual property, bias, accessibility, labor, manipulation, misinformation, environmental cost, concentration of power, and unequal error distribution.

Create an impact register linked to concrete system features and use cases. Distinguish direct harm, misuse, foreseeable secondary effects, and structural effects. For every mitigation, state evidence, limitations, owner, residual risk, and review date. “Use a filter” is not an adequate mitigation without measured coverage and failure analysis.

Include affected stakeholders in review, especially groups bearing risk without being direct users. Document disagreements instead of forcing consensus into one score. Some residual risks require limiting deployment rather than another model update.

Mitigations can move harm rather than remove it. Aggressive filtering may reduce misuse while excluding dialects or accessibility needs; data removal may protect privacy while weakening representation of a community. Evaluate distributional effects and identify who bears the residual cost. Separate reversible decisions from deployments that create lasting data or social consequences. When evidence is weak, document uncertainty and use staged deployment instead of converting unknowns into reassuring numerical scores.

Define participation and appeal mechanisms where feasible. People affected by system errors need a path to contest outputs and correct records; aggregate monitoring cannot represent an individual high-impact failure adequately.

Part 2 - Residual risk

If inherent risk is R_0 and control effectiveness is $e \in [0, 1]$, a simple residual estimate is $R_r = R_0(1-e)$. This is an organizing approximation, not a claim that ethical impact is exactly scalar or independent across controls.

Part 3 - Impact assessment

1. Identify stakeholders, rights, benefits, harms, and power asymmetries.
2. Connect each risk to a feature, workflow, and affected context.
3. Gather quantitative evidence and qualitative testimony.
4. Evaluate mitigation effectiveness and displacement of harm.
5. Record residual risk, disagreements, and deployment limitations.
6. Assign owners and review triggers.
7. Publish an appropriate transparency summary without exposing sensitive security details or personal data.

Part 4 - Ethical risk register

```
class EthicalRisk:
    def __init__(self, risk_id, stakeholders, description, inherent_risk):
        self.risk_id = risk_id
        self.stakeholders = stakeholders
        self.description = description
        self.inherent_risk = inherent_risk
        self.mitigations = []

    def add_mitigation(self, name, effectiveness, owner, evidence):
        self.mitigations.append({
            "name": name, "effectiveness": effectiveness,
            "owner": owner, "evidence": evidence
        })

def E18(risk_records):
    registry = []
    for record in risk_records:
        registry.append(EthicalRisk(
            record["risk_id"], record["stakeholders"],
            record["description"], record["inherent_risk"]
        ))
    return registry
```

Part 5 - The Philosophy Behind the Exercise

Ethics begins where technical success stops being the only question. A model can meet accuracy targets while distributing costs unfairly or changing human agency in ways no benchmark captures.

Documentation does not solve value conflicts, but it prevents them from being hidden behind architecture. It makes deployment a reasoned responsibility rather than an automatic consequence of capability.

18.6.20 Exercise 19

Exercise Title: Produce a Full Alignment Engineering Report

Part 1 - Deep conceptual foundation

The final report must connect behavioral goals, stakeholders, preference data, training objectives, optimization, safety policies, evaluations, red-team findings, deployment controls, and residual risks. It is a decision record, not a claim that alignment is complete.

Begin with scope, policy, non-goals, and authority. Document how candidates were generated, who annotated them, rubric versions, disagreement, bias audits, splits, and privacy. Specify SFT, reward-model, DPO or PPO configurations, reference checkpoint, beta or KL controls, precision, memory, seeds, and stop conditions.

Compare base, SFT, and aligned checkpoints on helpfulness, correctness, hallucination, refusal precision, unsafe compliance, diversity, edge cases, retention, and red-team categories. Include uncertainty and paired failures. Describe filters, tool permissions, monitoring, rollback, and incident response.

End with blocking gates, accepted residual risks, owners, expiration dates, and the next experiment that would reduce the greatest uncertainty. Rejected configurations and failed mitigations are essential evidence.

The report should include a claim-evidence matrix. Every major statement—for example that hallucination decreased or refusal became more precise—must point to a versioned evaluation, confidence estimate, and known limitation. Separate observations from interpretations and release decisions. Add a change log so a future report can explain why policy, data, thresholds, or mitigations changed. The final artifact should be readable by model engineers, safety reviewers, and decision owners without requiring undocumented project memory.

Part 2 - Decision vector

Let $G = (H, S, T, R, D, P, M)$ represent helpfulness, safety, truthfulness, retention, diversity, privacy, and monitoring readiness. Release requires every blocking component to meet its threshold; one high score cannot compensate for a critical failure elsewhere.

Part 3 - Required report sections

1. Identify every model, tokenizer, dataset, rubric, policy, code, and test suite version.
2. Explain preference collection and disagreement handling.
3. Derive the chosen objective and show mask and log-probability tests.
4. Present training stability, CUDA cost, and reproducibility evidence.
5. Compare all behavioral dimensions and critical slices.
6. Include red-team root causes, mitigations, and regression results.
7. Document ethical impact, residual risk, monitoring, rollback, and owners.

8. State the release decision and evidence that could reverse it.

Part 4 - Structured alignment report

```
class AlignmentReport:
    def __init__(self, report_id):
        self.report_id = report_id
        self.sections = {}
        self.gates = []

    def add_section(self, name, evidence):
        self.sections[name] = evidence

    def add_gate(self, name, passed, blocking, owner, evidence):
        self.gates.append({
            "name": name, "passed": passed, "blocking": blocking,
            "owner": owner, "evidence": evidence
        })

    def release_ready(self):
        for gate in self.gates:
            if gate["blocking"] and not gate["passed"]:
                return False
        return True

def E19():
    report = AlignmentReport("LLM04-ALIGNMENT-V1")
    report.add_section("scope", {"status": "defined"})
    report.add_section("preference_data", {"status": "audited"})
    report.add_section("evaluation", {"status": "pending"})
    report.add_gate(
        "critical_safety_suite", False, True,
        "safety_owner", "evaluation must be completed"
    )
    return report, report.release_ready()
```

Part 5 - The Philosophy Behind the Exercise

An alignment report is a map of influence: whose preferences shaped behavior, which proxies carried them, which failures survived, and who accepted the remaining risk. It makes the model's conduct answerable to decisions.

Alignment has no final certificate. The report is valuable because it records a current, contestable justification and creates obligations to monitor, revise, and stop when reality contradicts it.

18.7 LLM05

Chapter Title: Inference, KV Cache, and Serving **Specialization:** Production LLM Inference Engineering

18.7.1 Exercise 00

Exercise Title: Explain Autoregressive Inference and Why KV Cache Is Required

Part 1 - Deep conceptual foundation

Autoregressive inference generates one token at a time. The prompt first passes through a prefill phase that computes representations for all prompt positions. The decode phase then appends one token per iteration. Although operations inside each iteration are parallel, token $t+1$ cannot be chosen before token t exists.

Without a cache, every decode step recomputes keys and values for the entire prefix. A KV cache stores the attention key and value tensors from earlier positions, so the new query attends to stored history while only the newest position is projected. This removes repeated prefix computation but consumes memory proportional to layers, batch, sequence length, heads, and head size.

Distinguish time-to-first-token from inter-token latency. Prefill cost grows with prompt length and benefits from parallelism; decode is bandwidth-sensitive and repeats many small steps. The exercise will trace tensor shapes, estimate cache bytes, and compare exact cached and uncached logits. Cache architecture also differs across attention variants. Multi-query and grouped-query attention store fewer KV heads than full multi-head attention, which changes memory without changing the number of query heads. Beam search and multiple return sequences may share prompt cache initially and diverge later. Production estimates must therefore use the actual checkpoint configuration and decode strategy rather than parameter count alone. Fragmentation and reserved capacity add overhead beyond the ideal equation. Measure live bytes at prompt length boundaries and after request completion to connect theory with allocator behavior.

Part 2 - Mathematical and systems model

For layer l, cache after step t is

$$K_l^{(t)} = [K_l^{(t-1)}; k_{l,t}], \qquad V_l^{(t)} = [V_l^{(t-1)}; v_{l,t}].$$

Approximate cache bytes are

$$M_{KV} = 2LBTHd_h s,$$

where L is layers, B batch, T cached length, H KV heads, d_h head dimension, and s bytes per element.

Part 3 - Evidence and controls

1. Separate prefill and decode timing.
2. Write cache shape for every layer and verify sequence-axis growth.
3. Compare cached and full-prefix logits at every generated position.
4. Estimate bytes analytically and compare with measured allocation.
5. Test one token, long prompt, batched prompts, and cache reset.
6. Explain why caching reduces compute but increases persistent memory.

Part 4 - Explicit cache-size code

```
def kv_cache_bytes(layers, batch, cached_tokens,
                   key_value_heads, head_size, bytes_per_value):
    key_elements = layers * batch * cached_tokens
    key_elements = key_elements * key_value_heads * head_size
    value_elements = key_elements
    return (key_elements + value_elements) * bytes_per_value

def E00(configurations):
    results = []
    for configuration in configurations:
        size = kv_cache_bytes(
            configuration["layers"], configuration["batch"],
            configuration["tokens"], configuration["kv_heads"],
            configuration["head_size"], configuration["bytes"]
        )
        results.append({"configuration": configuration, "cache_bytes": size})
    return results
```

Part 5 - The Philosophy Behind the Exercise

KV caching is computational memory: the service spends storage to avoid repeating the past. It reveals a general systems principle that speed often comes from preserving intermediate work. The cache is useful only while its identity, position, and lifecycle remain correct; memory without provenance becomes error.

18.7.2 Exercise 01

Exercise Title: Implement KV Cache Correctly and Validate Speedup Over Naive Decoding

Part 1 - Deep conceptual foundation

Implement caching at the attention layer before relying on framework generation. During prefill, project all tokens and store keys and values. During decode, project only the newest token, append on the sequence axis, and compute attention against the complete cache. Cache tensors must retain batch, KV-head, sequence, and head-dimension meaning.

Correctness precedes speed. Cached and naive decoding should produce equal logits for deterministic models within dtype tolerance. Compare every step, not only final tokens, because greedy outputs can remain equal despite growing numerical error. Disable dropout, fix seeds, and use the same causal and padding masks.

Benchmark after warmup with CUDA synchronization. Naive decoding repeatedly processes longer prefixes; cached decoding processes one new token but reads a growing cache. Speedup is small for tiny sequences and grows until memory bandwidth or cache management dominates. Appending with repeated tensor concatenation is transparent but can reallocate and copy the entire cache. After correctness is established, compare it with a preallocated or paged cache that writes new states into owned blocks. The optimized form must preserve the same visible prefix and causal mask. Test cache capacity expansion and block reuse under request churn. A speed claim should include allocation time and memory traffic, not merely attention kernels, because poor cache storage can erase the theoretical savings of reused projections.

Part 2 - Attention update

For new query q_t,

$$a_t = \text{softmax}(q_t K_{\le t}^\top / \sqrt{d_h}), \qquad o_t = a_t V_{\le t}.$$

Only q_t, k_t, v_t are newly projected in decode.

Part 3 - Validation protocol

1. Test cache parity layer by layer with float64 on a tiny model.
2. Assert cache length increases by exactly one per step.
3. Compare logits, selected token, and decoded output.
4. Warm up, synchronize, and time multiple sequence lengths.
5. Report prefill separately from decode.
6. Verify a new request cannot reuse another request's cache.

Part 4 - Explicit cache container

```
import torch

class LayerKVCache:
    def __init__(self):
        self.keys = None
        self.values = None

    def append(self, new_keys, new_values):
        if new_keys.shape != new_values.shape:
            raise ValueError("key and value shapes differ")
        if self.keys is None:
            self.keys = new_keys
            self.values = new_values
        else:
            self.keys = torch.cat([self.keys, new_keys], dim=2)
            self.values = torch.cat([self.values, new_values], dim=2)
        return self.keys, self.values

    def clear(self):
        self.keys = None
        self.values = None

def E01(key_steps, value_steps):
    cache = LayerKVCache()
    lengths = []
    for index in range(len(key_steps)):
        keys, values = cache.append(key_steps[index], value_steps[index])
        lengths.append(keys.shape[2])
    return cache, lengths
```

Part 5 - The Philosophy Behind the Exercise

A cache is an optimization that must be observationally invisible: it should change cost, not meaning. Parity testing embodies this contract. When performance work cannot reproduce the reference computation, it has optimized a different program.

18.7.3 Exercise 02

Exercise Title: Debug KV Cache Bugs Caused by Wrong Position Indexing

Part 1 - Deep conceptual foundation

Cached decoding changes the tensor presented to the model from the full prefix to one new token, but its position remains the absolute next position. Resetting the position to zero at every step corrupts positional encoding while preserving valid shapes. RoPE is especially sensitive because query and key rotations must use consistent positions.

Padding complicates batches. Position identifiers may derive from cumulative attention masks rather than raw column indices, and finished sequences must not advance as if they generated real content. Sliding windows may evict old cache entries while logical positions continue increasing or are transformed under a declared policy.

Diagnose the first step where cached logits diverge from full recomputation. Print logical position, cache length, mask length, and token identity. Fault injection—deliberately resetting or offsetting positions—proves the test can detect the bug. Position faults can remain hidden on repetitive or short text. Use a controlled sequence whose next token depends strongly on order, and compare several prompt lengths. Log the RoPE sine/cosine row or position embedding selected at the first divergent step. In a batch, permute row order and confirm positions follow request identity rather than slot history. When a request is compacted into another row, move cache, mask, logical position, and sampling state together as one atomic operation.

Part 2 - Position invariant

For prompt length P and decode offset j, the new logical position is $P + j$. Cache storage index and logical position are different when eviction or padding is present.

Part 3 - Debug procedure

1. Compare full and cached logits after every appended token.
2. Log logical positions independently of tensor columns.
3. Test left padding, right padding, batch size one, and mixed lengths.
4. Inject off-by-one and reset-to-zero faults.
5. Verify RoPE tables cover the maximum logical position.
6. Test cache eviction without silently resetting position semantics.

Part 4 - Explicit position tracker

```
class PositionTracker:
    def __init__(self, prompt_lengths):
        self.positions = []
        for length in prompt_lengths:
            self.positions.append(length)

    def current(self, active_mask):
        output = []
        for index in range(len(self.positions)):
            if active_mask[index]:
                output.append(self.positions[index])
            else:
                output.append(-1)
        return output

    def advance(self, active_mask):
        for index in range(len(self.positions)):
            if active_mask[index]:
                self.positions[index] = self.positions[index] + 1

def E02(prompt_lengths, active_steps):
    tracker = PositionTracker(prompt_lengths)
    history = []
    for active_mask in active_steps:
        history.append(tracker.current(active_mask))
        tracker.advance(active_mask)
    return history
```

Part 5 - The Philosophy Behind the Exercise

Position is not merely where a tensor is stored; it is the token's place in the model's remembered sequence. Cache bugs teach that physical layout and semantic time can diverge. Correct systems preserve the latter while optimizing the former.

18.7.4 Exercise 03

Exercise Title: Implement Batched Inference with Variable Prompt Lengths

Part 1 - Deep conceptual foundation

Variable prompts require padding for dense batches or a specialized packed representation. Padding masks prevent attention to artificial tokens, while position identifiers preserve each sequence's logical order. Left padding is often convenient for decoder-only last-token selection, but model and tokenizer contracts must agree.

During decode, some sequences finish before others. Maintain an active mask, append generated tokens only for active requests, and prevent finished rows from growing cache state indefinitely. Dynamic batching may remove finished rows or insert new requests, which requires an explicit mapping between request IDs and cache rows.

Validate batched outputs against independent single-request decoding under greedy selection. Test mixed prompt lengths, immediate EOS, maximum tokens, and identical prompts in

different row orders. Sampling state is also per request. Temperature, generator seed, repetition penalty, stop sequences, and output budget must follow a request when rows are reordered. Test greedy mode first, then seeded sampling with batch permutations. For continuous batching, record admission step and prefill completion so new rows cannot attend to earlier rows' cache blocks. A correct output comparison should include finish reason and generated-token count, not only decoded text, because padding or hidden extra tokens can produce the same visible string.

Part 2 - Batch efficiency

Padding efficiency is $\eta = \sum_i T_i / (B \max_i T_i)$. Length bucketing increases η but may add queue delay; serving must balance both.

Part 3 - Required tests

1. Record request-to-row mapping and prompt length.
2. Build attention masks and positions from valid tokens.
3. Compare batched and unbatched greedy outputs.
4. Stop rows independently on EOS or token budget.
5. Test row compaction and cache reordering.
6. Measure padding waste and queue delay by length bucket.

Part 4 - Explicit active-row state

```
class BatchRequestState:
    def __init__(self, request_ids, maximum_new_tokens):
        self.request_ids = request_ids
        self.maximum_new_tokens = maximum_new_tokens
        self.generated_counts = []
        self.active = []
        for request_id in request_ids:
            self.generated_counts.append(0)
            self.active.append(True)

    def update(self, next_token_ids, end_token_id):
        for index in range(len(self.request_ids)):
            if self.active[index]:
                self.generated_counts[index] += 1
                reached_end = next_token_ids[index] == end_token_id
                generated_count = self.generated_counts[index]
                maximum_count = self.maximum_new_tokens[index]
                reached_limit = generated_count >= maximum_count
                if reached_end or reached_limit:
                    self.active[index] = False
```

```
def E03(request_ids, limits, generated_steps, end_token_id):
    state = BatchRequestState(request_ids, limits)
    for step in generated_steps:
        state.update(step, end_token_id)
    return state
```

Part 5 - The Philosophy Behind the Exercise

Batching joins independent conversations for efficiency without permitting their identities or histories to mix. The request-row map is therefore a boundary of privacy and meaning, not merely an indexing convenience.

18.7.5 Exercise 04

Exercise Title: Compare Latency vs Throughput Tradeoffs in Batch Size Selection

Part 1 - Deep conceptual foundation

Larger batches improve hardware utilization and total tokens per second, but a request may wait longer for a batch to form and share decode iterations with other sequences. User latency includes queueing, tokenization, prefill, time-to-first-token, inter-token gaps, and completion time. Throughput alone can hide poor interactive experience.

Static batching is simple but suffers from stragglers. Continuous batching admits new requests as rows finish, increasing utilization while making cache scheduling more complex. Compare fixed batch sizes and maximum batching delays under the same arrival trace. Report percentiles, not only means, because overload appears in the tail.

Find the saturation point where throughput stops growing and latency rises sharply. Separate prompt and output lengths; long prefill and long decode stress different paths. Define service-level objectives before selecting a batch policy. Fairness matters when requests have unequal cost. First-come scheduling can let a very long prefill delay many short interactive prompts; shortest-job policies can starve long documents. Compare length-aware queues with aging or tenant quotas. Track latency conditional on cost and customer class, and report dropped or rejected work rather than computing percentiles only for successful requests. Backpressure should activate before memory exhaustion so overload is expressed as bounded waiting or explicit rejection instead of process failure.

Part 2 - Queueing quantities

For request i, $L_i = L_{queue} + L_{tokenize} + L_{prefill} + L_{decode}$. Throughput is completed tokens or requests per unit time. P50, P95, and P99 latency describe different user experiences.

Part 3 - Benchmark design

1. Replay one timestamped arrival trace across policies.
2. Vary batch cap and maximum queue delay.
3. Segment by prompt and output length.
4. Record TTFT, inter-token latency, completion latency, and throughput.
5. Include warm and overloaded conditions.

6. Select a policy that satisfies explicit tail-latency objectives.

Part 4 - Percentile summary

```
def percentile(values, probability):
    ordered = sorted(values)
    if len(ordered) == 0:
        raise ValueError("no latency values")
    index = int(round((len(ordered) - 1) * probability))
    return ordered[index]

def E04(records):
    completion = []
    first_token = []
    for record in records:
        completion.append(record["completion_latency_ms"])
        first_token.append(record["time_to_first_token_ms"])
    return {
        "completion_p50": percentile(completion, 0.50),
        "completion_p95": percentile(completion, 0.95),
        "completion_p99": percentile(completion, 0.99),
        "first_token_p95": percentile(first_token, 0.95)
    }
```

Part 5 - The Philosophy Behind the Exercise

Throughput values collective efficiency; latency describes an individual's wait. Serving design must negotiate both rather than allowing an average to erase the experience of the slowest users.

18.7.6 Exercise 05

Exercise Title: Implement Streaming Token Generation and Measure User-Perceived Latency

Part 1 - Deep conceptual foundation

Streaming sends text incrementally instead of waiting for completion. It improves perceived responsiveness but creates protocol, decoding, cancellation, and safety challenges. Token boundaries are not necessarily valid UTF-8 or human-readable substrings, so use an incremental decoder and buffer incomplete sequences.

Measure time to first token, inter-token latency distribution, stalls, and total completion. Server generation time differs from client display time because network buffering and rendering intervene. Support cancellation that releases KV cache and scheduler state immediately. If output moderation is required, define whether tokens are buffered for inspection before release. Text rendering introduces another boundary. Some tokenizers revise the decoded suffix when byte pieces complete, so emitting each decoded token independently can produce replacement characters or spacing errors. Compare incremental output with decoding the complete token sequence and require exact final parity. Define heartbeat behavior for long prefill or stalls so

proxies do not close healthy connections. If clients reconnect, specify whether generation can resume from a server-held request or must restart, and bound the lifetime of detached streams.

Part 2 - Latency model

If token timestamps are $t_1, \ldots, t_n$, TTFT is $t_1 - t_0$ and inter-token gaps are $t_i - t_{i-1}$. Report P95 gaps and longest stall.

Part 3 - Tests

1. Stream decoded text fragments, not raw token bytes.
2. Timestamp server generation and client receipt separately.
3. Test cancellation, disconnect, EOS, timeout, and malformed transport.
4. Ensure request cache is released on every termination path.
5. Measure jitter under concurrent load.
6. Verify buffered safety checks do not claim zero-latency streaming.

Part 4 - Streaming metrics

```
def streaming_metrics(request_start, token_times):
    if len(token_times) == 0:
        return {"time_to_first_token": None, "gaps": []}
    gaps = []
    previous = token_times[0]
    for index in range(1, len(token_times)):
        gaps.append(token_times[index] - previous)
        previous = token_times[index]
    return {
        "time_to_first_token": token_times[0] - request_start,
        "gaps": gaps,
        "maximum_gap": max(gaps) if len(gaps) > 0 else 0.0,
        "completion_time": token_times[-1] - request_start
    }

def E05(requests):
    results = []
    for request in requests:
        results.append(streaming_metrics(
            request["start"], request["token_times"]
        ))
    return results
```

Part 5 - The Philosophy Behind the Exercise

Streaming changes when knowledge becomes visible. Partial output creates partial commitment, so responsiveness must be balanced with decoding integrity, safety, and the user's ability to cancel.

18.7.7 Exercise 06

Exercise Title: Profile GPU Utilization During Inference and Identify Bottlenecks

Part 1 - Deep conceptual foundation

Low GPU utilization can result from CPU tokenization, host-to-device transfer, small batches, synchronization, Python scheduling, or memory-bound decode. High utilization can still hide padding waste. Profile prefill and decode separately: prefill often uses large matrix multiplications, while single-token decode repeatedly reads weights and KV cache.

Begin with end-to-end timing, then use profiler ranges to locate cost. Measure kernel time, gaps, memory bandwidth, allocation, and data transfer. Avoid adding synchronization to every step because the profiler would change the workload. Correlate GPU traces with request and batch scheduler events. Profiler evidence must be interpreted with workload shape. A trace from one short prompt may emphasize launch overhead, while a production mixture may be dominated by long prefill. Collect representative traces and aggregate stage timing outside the profiler to avoid excessive overhead. Inspect CPU threads and queue state alongside CUDA kernels: blank GPU regions can be intentional waiting for a dynamic batch or accidental serialization. After every optimization, rerun output parity and tail-latency tests so faster kernels do not damage scheduler fairness.

Part 2 - Utilization metrics

Useful throughput equals nonpadding or generated tokens per second. Arithmetic utilization and memory bandwidth reveal different ceilings; a decode workload can be bandwidth-bound even when compute units are not saturated.

Part 3 - Profiling protocol

1. Establish CPU-only and GPU end-to-end baselines.
2. Mark tokenization, transfer, prefill, decode, sampling, and streaming.
3. Warm up before collecting traces.
4. Compare total versus useful tokens.
5. Vary batch, sequence length, and cache size.
6. Optimize the measured bottleneck and rerun the identical trace.

Part 4 - Stage timer

```
import time

class StageTimer:
    def __init__(self):
        self.records = []

    def measure(self, name, function):
        start = time.perf_counter()
        output = function()
```

```
        elapsed = time.perf_counter() - start
        self.records.append({"stage": name, "seconds": elapsed})
        return output

    def totals(self):
        totals = {}
        for record in self.records:
            name = record["stage"]
            totals[name] = totals.get(name, 0.0) + record["seconds"]
        return totals

def E06(stage_functions):
    timer = StageTimer()
    for name in stage_functions:
        timer.measure(name, stage_functions[name])
    return timer.totals()
```

Part 5 - The Philosophy Behind the Exercise

Profiling replaces intuition with evidence. The GPU is not slow or fast in the abstract; it is waiting, computing, or moving data for specific reasons. Good optimization begins by naming the reason.

18.7.8 Exercise 07

Exercise Title: Implement Stop Tokens and Safe Termination Logic

Part 1 - Deep conceptual foundation

Generation can stop because of EOS, one of several stop sequences, maximum new tokens, timeout, cancellation, policy interruption, or service shutdown. These causes have different status codes and cleanup obligations. Stop strings may span multiple tokens or streamed fragments, so compare incremental decoded suffixes while retaining enough overlap.

Define whether matched stop text is included or removed. Avoid exposing partial sensitive markers before a multi-token stop is recognized. In batches, terminate rows independently and release cache state. Always keep a hard token and time limit even when EOS is never produced. Stop matching should be scoped to generated output unless policy explicitly includes the prompt. Normalize only when the protocol guarantees normalization will not change meaning. Multiple overlapping stop strings require deterministic precedence, and a maximum-match buffer prevents premature emission. For tool or structured output, syntactic completion may be an additional termination rule, but it must not replace hard budgets. Record whether output was complete, truncated, cancelled, or filtered so downstream consumers do not treat every finish as semantically final.

Part 2 - Termination state

Termination is a state transition from active to one immutable final reason. Once finished, a request must not emit tokens or advance positions.

Part 3 - Edge cases

1. Test single-token and multi-token stops across fragment boundaries.
2. Define precedence when EOS and timeout occur together.
3. Test immediate cancellation and disconnect.
4. Confirm finished batch rows stop cache growth.
5. Return finish reason and token counts.
6. Release all request resources in a finally path.

Part 4 - Explicit termination controller

```
class TerminationController:
    def __init__(self, stop_strings, maximum_tokens):
        self.stop_strings = stop_strings
        self.maximum_tokens = maximum_tokens
        self.generated_tokens = 0
        self.text = ""
        self.finished = False
        self.reason = None

    def add_fragment(self, fragment):
        if self.finished:
            return
        self.text = self.text + fragment
        self.generated_tokens = self.generated_tokens + 1
        for stop_string in self.stop_strings:
            if self.text.endswith(stop_string):
                self.finished = True
                self.reason = "stop_sequence"
        if self.generated_tokens >= self.maximum_tokens and not self.finished:
            self.finished = True
            self.reason = "length"

def E07(fragments, stops, maximum_tokens):
    controller = TerminationController(stops, maximum_tokens)
    for fragment in fragments:
        controller.add_fragment(fragment)
        if controller.finished:
            break
    return controller
```

Part 5 - The Philosophy Behind the Exercise

Safe termination defines the boundary of a model's opportunity to speak. A service is responsible not only for generating, but for knowing when to stop and for making the reason observable.

18.7.9 Exercise 08

Exercise Title: Debug Memory Leaks in Long-Running Inference Services

Part 1 - Deep conceptual foundation

A memory leak is retained memory that should become unreachable after a request. Common causes include KV caches stored in global dictionaries, unfinished futures, streaming queues, retained tensors in metrics, exception paths, and unbounded tokenizer or prompt caches. CUDA reserved memory alone is not proof of a leak because the allocator intentionally reuses blocks.

Run repeated request waves with fixed shapes, force synchronization, and measure allocated, reserved, and process memory after cleanup. A healthy service may rise to a plateau; a leak grows with completed requests. Tag allocations by request and verify cancellation, timeout, exception, and normal completion all execute the same release contract. Distinguish leaks from legitimate growth in compiled graphs, tokenizer caches, and memory pools. Warm each supported shape before starting the slope test, or record new-shape events explicitly. Python object snapshots can reveal queues or request states whose reference counts never fall; CUDA memory snapshots locate live tensor allocations. Do not use periodic process restarts as the only fix: they hide ownership bugs and create cold-start instability. A temporary restart policy may contain impact while root-cause work continues.

Part 2 - Growth model

Fit memory after wave n as $M_n = a + bn$. Persistent positive slope b after warmup is evidence for investigation, not automatic proof.

Part 3 - Leak hunt

1. Reproduce with deterministic request waves.
2. Separate live cache, allocator reserve, and Python process memory.
3. Count active requests and cache entries after every wave.
4. Test all termination and exception paths.
5. Inspect object references before calling empty-cache functions.
6. Add a regression threshold for post-warmup growth.

Part 4 - Resource registry

```
class RequestResourceRegistry:
    def __init__(self):
        self.resources = {}

    def register(self, request_id, resource):
        if request_id in self.resources:
            raise ValueError("request already registered")
        self.resources[request_id] = resource
```

```
    def release(self, request_id):
        if request_id in self.resources:
            resource = self.resources.pop(request_id)
            resource.close()

    def active_count(self):
        return len(self.resources)

def E08(registry, events):
    for event in events:
        if event["type"] == "start":
            registry.register(event["request_id"], event["resource"])
        if event["type"] == "finish":
            registry.release(event["request_id"])
    return registry.active_count()
```

Part 5 - The Philosophy Behind the Exercise

A leak is forgotten ownership. Memory remains because the system has lost the story of who created it and when its purpose ended. Explicit lifecycle restores that story.

18.7.10 Exercise 09

Exercise Title: Compare fp32 vs fp16 vs int8 Inference Quality and Speed

Part 1 - Deep conceptual foundation

Reduced precision lowers memory bandwidth and can increase throughput, but the effect depends on hardware kernels and quantization method. fp16 changes arithmetic precision; int8 often quantizes weights and possibly activations using scales. Calibration data must represent deployment distributions.

Compare logits and generation, not only model size. Small numerical changes can alter sampled sequences, so use teacher-forced logit error, perplexity, task metrics, and deterministic greedy agreement. Inspect rare tokens, long contexts, outliers, and safety-boundary prompts. Benchmark warm prefill and decode separately with identical batches. Quantization can affect layers unevenly. Inspect embedding, attention output, feed-forward, normalization, and final language head separately, and retain sensitive operations at higher precision when supported. Per-channel scales may handle weight outliers better than one tensor-wide scale. Measure quality after serialization and reload, because deployment kernels and calibration metadata are part of the artifact. If int8 improves prefill but not decode, explain the hardware bottleneck rather than advertising one blended speedup number.

Part 2 - Error measures

For reference logits z and compressed logits $\hat{z}$, report maximum absolute error, relative norm error $\|z - \hat{z}\|/\|z\|$, KL divergence, and task-level quality. One scalar cannot characterize all sensitivity.

Part 3 - Matched benchmark

1. Use one checkpoint, tokenizer, prompts, and decoding policy.

2. Record model and KV-cache precision separately.
3. Warm up and measure prefill, decode, memory, and energy if available.
4. Compare logits before comparing free-running text.
5. Evaluate critical task and safety slices.
6. State calibration data and unsupported operators for int8.

Part 4 - Numeric comparison

```
import torch

def compare_logits(reference, candidate):
    difference = reference.float() - candidate.float()
    maximum = torch.max(torch.abs(difference)).item()
    numerator = torch.linalg.vector_norm(difference)
    denominator = torch.linalg.vector_norm(reference.float())
    relative = (numerator / torch.clamp(denominator, min=1e-12)).item()
    reference_tokens = torch.argmax(reference, dim=-1)
    candidate_tokens = torch.argmax(candidate, dim=-1)
    agreement = (reference_tokens == candidate_tokens).float().mean().item()
    return {
        "maximum_absolute_error": maximum,
        "relative_norm_error": relative,
        "greedy_token_agreement": agreement
    }

def E09(outputs):
    return {
        "fp16": compare_logits(outputs["fp32"], outputs["fp16"]),
        "int8": compare_logits(outputs["fp32"], outputs["int8"])
    }
```

Part 5 - The Philosophy Behind the Exercise

Precision is a decision about which numerical distinctions the service preserves. Compression is successful when discarded distinctions do not change promised behavior, and that claim must be measured on the cases that matter.

18.7.11 Exercise 10

Exercise Title: Deploy LLM Inference with Simple HTTP API and Measure End-to-End Latency

Part 1 - Deep conceptual foundation

An HTTP wrapper defines a public contract around tokenization, scheduling, generation, streaming, errors, and model versions. Validate request size, types, token budget, decoding

parameters, and authorization before allocating GPU state. Never accept arbitrary device or file paths from clients.

End-to-end latency begins when the server receives a request and includes queue, validation, tokenization, transfer, prefill, decode, serialization, and network delivery. Attach a request ID and stage timestamps. Return stable error codes without exposing stack traces or internal prompts.

Test concurrency, cancellation, malformed JSON, oversized prompts, timeout, and server shutdown. Keep model loading outside request handlers and use a bounded scheduler instead of spawning unrestricted generation work. Separate transport status from generation status. An HTTP success may contain a model-level refusal or length stop, while a network failure may occur after the server generated valid tokens. For idempotency, decide whether retries create a new stochastic generation or reference a stored request; never duplicate billable work silently. Apply request-body limits before parsing large payloads and ensure reverse proxies, application servers, and model workers share compatible timeout and streaming settings. Test the deployed network path rather than localhost alone.

Part 2 - Latency decomposition

$L_{e2e} = L_{queue} + L_{validate} + L_{tokenize} + L_{prefill} + L_{decode} + L_{transport}$. Report percentiles and stage contributions by prompt/output bucket.

Part 3 - API controls

1. Define versioned request and response schemas.
2. Validate before GPU admission.
3. Record stage timestamps with privacy-safe request IDs.
4. Bound prompt length, output length, concurrency, and time.
5. Test streaming and nonstreaming parity.
6. Return model version and finish reason.

Part 4 - Framework-neutral request validation

```
def validate_generation_request(payload, maximum_prompt_characters,
                                maximum_new_tokens):
    if "prompt" not in payload or not isinstance(payload["prompt"], str):
        raise ValueError("prompt must be a string")
    if len(payload["prompt"]) > maximum_prompt_characters:
        raise ValueError("prompt is too long")
    requested_tokens = payload.get("maximum_new_tokens", 64)
    if requested_tokens < 1 or requested_tokens > maximum_new_tokens:
        raise ValueError("invalid output token limit")
    return {
        "prompt": payload["prompt"],
        "maximum_new_tokens": requested_tokens,
        "stream": bool(payload.get("stream", False))
```

```
    }

def E10(payloads):
    accepted = []
    for payload in payloads:
        accepted.append(validate_generation_request(payload, 20000, 512))
    return accepted
```

Part 5 - The Philosophy Behind the Exercise

An API turns model capability into institutional action. Validation and bounded resources define who may ask, how much work may be consumed, and how failures remain contained.

18.7.12 Exercise 11

Exercise Title: Implement Rate Limiting and Timeout Handling for Inference Service

Part 1 - Deep conceptual foundation

Rate limits protect availability and fairness. Request counts alone are weak because one long prompt can cost far more than many short ones. Use token- or compute-aware budgets with identity, tenant, and global limits. A token bucket allows controlled bursts while enforcing a long-term rate.

Timeouts need phases: queue deadline, prefill deadline, inter-token stall, and total request deadline. Cancellation must propagate into the scheduler and release KV cache. Returning a timeout while GPU work continues is resource leakage disguised as error handling.

Use monotonic clocks, bounded queues, and explicit retry guidance. Test clock edges, burst traffic, multiple workers, and clients that disconnect midstream. Distributed replicas need coordinated or intentionally partitioned limits. A per-process bucket can multiply a tenant's allowance by the replica count, while a central limiter can become a latency and availability dependency. Document the consistency model and conservative fallback when the limiter is unreachable. Cost estimates should reserve maximum output capacity at admission and refund unused tokens afterward under an auditable rule. Monitor whether repeated rejections concentrate on particular tenants or workloads.

Part 2 - Token bucket

Tokens replenish as $b(t) = \min(C, b(t_0) + r(t - t_0))$. A request of cost c is admitted only if $b(t) \geq c$.

Part 3 - Tests

1. Estimate cost from input and maximum output tokens.
2. Enforce tenant and global budgets.
3. Test bursts and steady traffic with a fake clock.
4. Propagate timeout cancellation to GPU work.
5. Release cache on every deadline path.
6. Monitor rejection and timeout rates by cause.

Part 4 - Explicit token bucket

```
class TokenBucket:
    def __init__(self, capacity, refill_per_second, initial_time):
        self.capacity = capacity
        self.refill_per_second = refill_per_second
        self.available = capacity
        self.last_time = initial_time

    def allow(self, cost, current_time):
        elapsed = max(current_time - self.last_time, 0.0)
        self.available = min(
            self.capacity,
            self.available + elapsed * self.refill_per_second
        )
        self.last_time = current_time
        if cost <= self.available:
            self.available = self.available - cost
            return True
        return False

def E11(bucket, requests):
    decisions = []
    for request in requests:
        decisions.append(bucket.allow(request["cost"], request["time"]))
    return decisions
```

Part 5 - The Philosophy Behind the Exercise

Rate limiting allocates a finite common resource. A fair service measures demand in units related to cost and ensures one user's urgency cannot silently consume everyone else's availability.

18.7.13 Exercise 12

Exercise Title: Benchmark Cold Start vs Warm Start Inference

Part 1 - Deep conceptual foundation

Cold start includes process creation, imports, checkpoint read, deserialization, device transfer, graph or kernel initialization, and first inference. Warm start reuses a resident model, allocator state, and compiled kernels. Mixing them in one average makes capacity planning unreliable.

Measure each stage with clean process restarts and controlled storage caches. Distinguish operating-system page cache from truly remote or cold storage. Run a standard warmup that exercises expected shapes, then benchmark production-like prefill and decode. Record peak memory during loading because transient copies can exceed steady state.

Compare eager loading, lazy loading, compilation, quantized checkpoints, and multiple replicas. Include readiness semantics: a process should not receive traffic merely because its HTTP

port opened. Startup testing must include failures: missing shards, checksum mismatch, insufficient memory, incompatible kernels, and interrupted downloads. Readiness should remain false and expose a useful internal diagnostic without serving a partially loaded model. In autoscaling systems, compare startup time with traffic growth; replicas that become ready after the demand spike cannot protect tail latency. Model download locality and image size may dominate even when checkpoint loading is optimized.

Part 2 - Startup quantities

Readiness time is the interval from process start until a validated inference succeeds. First-request latency and steady-state latency are separate measures.

Part 3 - Benchmark controls

1. Repeat cold starts in fresh processes.
2. Record storage, import, load, transfer, warmup, and validation times.
3. State whether filesystem caches are warm or cold.
4. Measure transient and steady memory.
5. Test readiness and startup failure paths.
6. Compare deployment strategies using identical checkpoints.

Part 4 - Startup timeline

```
class StartupTimeline:
    def __init__(self, start_time):
        self.start_time = start_time
        self.events = []

    def record(self, name, timestamp):
        self.events.append({
            "name": name,
            "seconds_from_start": timestamp - self.start_time
        })

    def readiness_time(self):
        for event in self.events:
            if event["name"] == "ready":
                return event["seconds_from_start"]
        return None

def E12(start_time, events):
    timeline = StartupTimeline(start_time)
    for event in events:
        timeline.record(event["name"], event["timestamp"])
    return timeline, timeline.readiness_time()
```

Part 5 - The Philosophy Behind the Exercise

Warm systems hide the cost of becoming ready. Cold-start measurement restores the full lifecycle and reminds us that availability begins before the first token can be generated.

18.7.14 Exercise 13

Exercise Title: Handle Long Prompts with Sliding Window Inference

Part 1 - Deep conceptual foundation

Sliding-window inference limits attention to recent tokens, bounding KV-cache memory and per-step attention. It does not magically extend global understanding: evicted evidence is unavailable unless summarized, retrieved again, or preserved through another memory mechanism.

Define window size, eviction unit, logical positions, and treatment of system instructions. Blindly dropping the oldest tokens can remove policy, user intent, or critical facts. A prompt manager should reserve protected segments, detect overflow before inference, and disclose truncation when it can affect answers.

Evaluate local tasks and long-range dependency tasks while moving required evidence across the eviction boundary. Compare full-context reference where possible. RoPE and position handling must follow the model's supported policy. Summarization before eviction introduces a second model-generated artifact that can distort facts. Evaluate summaries for coverage and contradiction, and keep links to original spans where retrieval is possible. Recurrent system prompts or protected prefixes also consume fixed window capacity, reducing space for recent dialogue. Report effective user-context capacity rather than nominal window size. For multi-turn service, define whether the window counts raw messages, rendered tokens, retrieved passages, and generated output together.

Part 2 - Bounded cost

For window W, each decode query attends to at most W cached positions, so KV memory is bounded by W rather than total generated length. Information loss is the tradeoff.

Part 3 - Validation

1. Protect required system and task segments explicitly.
2. Record every evicted token range.
3. Test evidence just inside and outside the window.
4. Compare full and windowed logits before eviction.
5. Verify logical positions after cache compaction.
6. Measure quality, memory, and latency across window sizes.

Part 4 - Window manager

```
class SlidingTokenWindow:
    def __init__(self, maximum_tokens):
        self.maximum_tokens = maximum_tokens
        self.tokens = []
        self.total_seen = 0
        self.evicted = 0

    def append(self, token_id):
        self.tokens.append(token_id)
        self.total_seen = self.total_seen + 1
        if len(self.tokens) > self.maximum_tokens:
            self.tokens.pop(0)
            self.evicted = self.evicted + 1

def E13(token_ids, maximum_tokens):
    window = SlidingTokenWindow(maximum_tokens)
    for token_id in token_ids:
        window.append(token_id)
    return window
```

Part 5 - The Philosophy Behind the Exercise

A window makes forgetting deliberate. Bounded memory is honest engineering when the system records what it removed and does not pretend discarded evidence still guides the answer.

18.7.15 Exercise 14

Exercise Title: Compare Server-Side vs Client-Side Tokenization Costs

Part 1 - Deep conceptual foundation

Server-side tokenization centralizes tokenizer versions and validation but uses CPU resources and transfers raw text. Client-side tokenization can reduce server CPU work and let clients estimate cost, yet creates compatibility and security risks: incorrect token IDs, obsolete vocabularies, invalid special tokens, and larger serialized payloads.

Benchmark tokenization time, network bytes, validation, queue delay, and failure rate. Token IDs are often larger on the wire than compressed UTF-8 text. A server accepting IDs must bind requests to an exact tokenizer hash and reject reserved or out-of-range values. Never trust client-reported token counts for billing or limits without verification.

Use multilingual text, code, short chats, and long documents. Separate cold tokenizer initialization from steady operation and measure concurrency scaling. Client-side tokenization also affects governance. Raw text may be needed for policy filtering, abuse investigation, or accessibility transformations, while privacy goals may favor minimizing server text exposure. Decide this separately from performance and document what each side stores. If clients submit token IDs, server-side moderation may require safe decoding under the pinned tokenizer. Test noncanonical encodings that decode to equivalent text and ensure billing and limits operate on validated token counts.

Part 2 - End-to-end comparison

Total cost includes client work, transport, server validation, and any repeated decoding. The fastest local stage may not minimize request latency or bytes.

Part 3 - Protocol

1. Pin tokenizer files and hash on both paths.
2. Measure text and token-ID payload bytes.
3. Validate range, special tokens, and decoded parity.
4. Include client device classes and server concurrency.
5. Test version mismatch and malicious identifiers.
6. Compare end-to-end latency rather than server CPU alone.

Part 4 - Token request validation

```
def validate_token_request(token_ids, vocabulary_size,
                           forbidden_special_ids):
    validated = []
    forbidden = set(forbidden_special_ids)
    for token_id in token_ids:
        if not isinstance(token_id, int):
            raise ValueError("token identifier must be integer")
        if token_id < 0 or token_id >= vocabulary_size:
            raise ValueError("token identifier outside vocabulary")
        if token_id in forbidden:
            raise ValueError("reserved special token is not allowed")
        validated.append(token_id)
    return validated

def E14(requests, vocabulary_size, forbidden_special_ids):
    outputs = []
    for request in requests:
        outputs.append(validate_token_request(
            request, vocabulary_size, forbidden_special_ids
        ))
    return outputs
```

Part 5 - The Philosophy Behind the Exercise

Tokenization is part of model identity. Moving it across a network boundary also moves trust and version responsibility. Performance gains are valid only when the language-to-token contract remains exact.

18.7.16 Exercise 15

Exercise Title: Implement Canary Deployment for New Model Versions

Part 1 - Deep conceptual foundation

A canary sends a small controlled fraction of traffic to a new model while the stable version remains available. Routing must be sticky per conversation so KV cache and behavior do not switch midstream. Exclude high-risk users or workflows until explicit gates pass, and support immediate rollback.

Compare latency, errors, memory, refusal, quality proxies, user feedback, and safety events. Traffic percentages alone are insufficient: prompt distributions can differ, so use randomization or matched analysis. Start small, require a minimum sample and observation time, then expand through declared stages.

Rollback should change new routing while allowing or safely cancelling in-flight requests. Preserve model version in logs and responses. Test the rollback before launch rather than during the first incident. Canary analysis must account for conversations that span evaluation windows. Assign by durable conversation identifier, but avoid using personally identifying data directly in routing. Compare new and control versions on matched prompt length, language, tenant, and task slices. A global metric can look stable while one rare workload regresses. Define automatic rollback for crashes and critical safety events, while quality changes with uncertain graders may require human review before promotion.

Part 2 - Release decision

Canary promotion requires every blocking metric within tolerance relative to control. Statistical significance does not make a practically severe regression acceptable.

Part 3 - Canary protocol

1. Define eligible traffic and sticky assignment.
2. Establish stages, sample minimums, and observation periods.
3. Compare quality, safety, latency, errors, and resource metrics.
4. Detect distribution imbalance between control and canary.
5. Automate stop and rollback for critical gates.
6. Rehearse cache-safe rollback and audit version tags.

Part 4 - Deterministic routing

```
import hashlib

def canary_assignment(conversation_id, canary_percent):
    digest = hashlib.sha256(conversation_id.encode("utf-8")).hexdigest()
    bucket = int(digest[:8], 16) % 10000
    threshold = int(canary_percent * 100)
    if bucket < threshold:
        return "canary"
```

```
    return "control"

def E15(conversation_ids, canary_percent):
    assignments = {}
    for conversation_id in conversation_ids:
        assignments[conversation_id] = canary_assignment(
            conversation_id, canary_percent
        )
    return assignments
```

Part 5 - The Philosophy Behind the Exercise

Canary deployment turns uncertainty into limited exposure. It acknowledges that laboratory evidence is incomplete while refusing to make every user participate in the experiment at once.

18.7.17 Exercise 16

Exercise Title: Monitor Inference Errors and Build Alerting Rules

Part 1 - Deep conceptual foundation

Monitoring should distinguish validation rejection, queue timeout, OOM, nonfinite logits, tokenizer failure, cache corruption, disconnect, policy stop, and internal error. Aggregate error rate hides causes and severity. Attach model, tokenizer, hardware, batch, prompt-length bucket, and finish reason without logging sensitive text unnecessarily.

Alerts need thresholds, windows, minimum volume, severity, owner, and runbook. Use rates for common traffic and absolute counts for rare critical failures. Avoid alert storms through deduplication and recovery conditions. Monitor silent quality proxies and canary regressions in addition to exceptions.

Test alerts by injecting safe synthetic failures. A dashboard is observational; an alert is operational only when someone can act on it. Alert quality has its own metrics: time to detect, actionable rate, duplicate volume, missed incidents, and time to recovery. Review alerts after incidents and remove rules that create habitual noise without losing coverage. Couple nonfinite-logit or cache-isolation failures to immediate protective actions, but avoid automated restart loops that repeatedly destroy evidence. Preserve a small diagnostic snapshot with configuration and recent stage metrics under privacy controls.

Part 2 - Rate and burn

For window error count e and requests n, rate is e/n. Multi-window burn rules can detect fast incidents and slow degradation while respecting a service error budget.

Part 3 - Alert design

1. Define error taxonomy and severity.
2. Track rates by version and workload slice.
3. Create fast and slow windows with minimum volume.
4. Route every alert to an owner and runbook.

5. Deduplicate correlated symptoms.

6. Inject faults and verify firing and recovery.

Part 4 - Rule evaluator

```
class AlertRule:
    def __init__(self, name, threshold, minimum_requests):
        self.name = name
        self.threshold = threshold
        self.minimum_requests = minimum_requests

    def evaluate(self, errors, requests):
        if requests < self.minimum_requests:
            return False
        return errors / requests >= self.threshold

def E16(rules, measurements):
    alerts = []
    for rule in rules:
        for measurement in measurements:
            if rule.evaluate(measurement["errors"], measurement["requests"]):
                alerts.append({
                    "rule": rule.name,
                    "window": measurement["window"]
                })
    return alerts
```

Part 5 - The Philosophy Behind the Exercise

Monitoring is the service's capacity to notice itself failing. Alerting adds obligation: a signal must reach someone with authority and evidence to respond.

18.7.18 Exercise 17

Exercise Title: Stress Test Inference Under High QPS and Detect Failures

Part 1 - Deep conceptual foundation

A stress test increases offered load until the service saturates, then observes queue growth, latency tails, rejections, OOM, cache fragmentation, and recovery. QPS alone is inadequate; define prompt/output distributions, streaming, arrival burstiness, and tenant mix. Run only in an authorized isolated environment.

Use load, stress, spike, soak, and recovery phases. Open-loop generators maintain arrivals even when the server slows and reveal overload; closed-loop clients can hide saturation by waiting. Apply bounded admission so overload becomes explicit rejection rather than uncontrolled collapse.

After load falls, verify latency, memory, active requests, and cache return to baseline. A service that survives peak traffic but never recovers has failed. Coordinate generator clocks

and measure achieved arrivals; otherwise the load tool can become the bottleneck. Separate connection, request, and token throughput. Use several generators when necessary, and verify their network path resembles production. During soak tests, track memory slope, allocator fragmentation, temperature, clock throttling, and error drift. Capacity should include a safety margin below the measured knee because production traffic is less predictable than a controlled trace.

Part 2 - Saturation

Throughput plateaus when a limiting resource reaches capacity; beyond this point queue delay grows rapidly. Identify the knee before selecting safe operating QPS.

Part 3 - Stress protocol

1. Define realistic length and arrival distributions.
2. Increase open-loop load in stages.
3. Measure P50/P95/P99 latency, TTFT, errors, queue, memory, and throughput.
4. Inject bursts, disconnects, and one replica failure.
5. Verify admission control and graceful degradation.
6. Confirm post-test resource and latency recovery.

Part 4 - Load-stage summary

```
def E17(stage_records):
    summaries = []
    for stage in stage_records:
        completed = stage["completed_requests"]
        duration = stage["duration_seconds"]
        offered = stage["offered_requests"]
        summary = {}
        summary["stage"] = stage["stage"]
        summary["throughput_qps"] = completed / duration
        summary["success_rate"] = completed / max(offered, 1)
        summary["queue_depth"] = stage["queue_depth"]
        summary["peak_memory"] = stage["peak_memory"]
        summaries.append(summary)
    return summaries
```

Part 5 - The Philosophy Behind the Exercise

Stress testing asks where graceful service becomes disorder. The goal is not to claim infinite capacity, but to know the boundary and make failure controlled, observable, and recoverable.

18.7.19 Exercise 18

Exercise Title: Build Inference Debugging Checklist

Part 1 - Deep conceptual foundation

An inference checklist should proceed from identity to semantics to performance. Verify model, tokenizer, template, precision, device, and configuration; then validate uncached reference generation, cache parity, positions, masks, batch mapping, stop logic, and lifecycle. Only after correctness should latency and throughput be tuned.

Every item needs an executable test, expected result, evidence, severity, and remediation. Include fault injection for wrong positions, stale cache, timeout, OOM, disconnect, quantization drift, and version mismatch. Blocking failures stop deployment; warnings remain visible.

The checklist must cover single-request, batch, long-run, overload, and restart behavior. A notebook success does not validate a service lifecycle. Organize evidence so one failed check points to the next diagnostic rather than merely returning false. Cache parity failure should identify first divergent layer and position; a leak check should report active request IDs and allocation growth; a latency regression should name the changed stage. Version the checklist with the service architecture and require new checks after every incident. An obsolete checklist can create confidence while testing contracts the service no longer uses.

Part 2 - Gate model

Release readiness is the conjunction of blocking checks. Performance scores cannot compensate for incorrect logits, cache isolation, or cleanup.

Part 3 - Ordered checklist

1. Verify immutable artifact identities.
2. Compare full, cached, batched, and quantized logits.
3. Test positions, padding, cache reorder, and stop conditions.
4. Test cancellation, exception, timeout, and cleanup.
5. Profile stages before optimizing.
6. Run load, soak, recovery, and restart tests.

Part 4 - Executable checks

```
class InferenceCheck:
    def __init__(self, name, blocking, function):
        self.name = name
        self.blocking = blocking
        self.function = function

def E18(checks, context):
    results = []
    ready = True
    for check in checks:
```

```
        passed, evidence = check.function(context)
        results.append({
            "name": check.name,
            "blocking": check.blocking,
            "passed": bool(passed),
            "evidence": evidence
        })
        if check.blocking and not passed:
            ready = False
    return {"ready": ready, "results": results}
```

Part 5 - The Philosophy Behind the Exercise

A checklist is accumulated memory made executable. It protects correctness from urgency by ensuring the system repeatedly proves the contracts on which its speed depends.

18.7.20 Exercise 19

Exercise Title: Produce a Production Inference Engineering Report

Part 1 - Deep conceptual foundation

The final report connects model artifacts, serving architecture, cache design, batch scheduler, precision, API, streaming, limits, monitoring, deployment, and failure recovery. It must allow another engineer to reproduce benchmarks and understand the release decision.

Document prefill and decode paths, KV shapes and memory, logical positions, padding, cache lifecycle, stop semantics, and request isolation. Present latency by stage and percentile, throughput by workload, memory peaks, cold start, precision quality, stress saturation, and soak recovery. Include failed configurations rather than only the winner.

Specify HTTP contracts, authentication boundaries, quotas, deadlines, versioning, canary stages, alerts, rollback, privacy-preserving logs, and owners. End with blocking gates, residual limitations, capacity limits, and the next experiment that would reduce uncertainty most. Include a capacity envelope rather than one benchmark number. State supported prompt lengths, output lengths, concurrency, precision, hardware, and latency objectives, plus the admission behavior outside that envelope. Present cost per request and per generated token for representative workloads. Separate measured facts from forecasts and identify assumptions about traffic mix. Every chart and table should reference a raw trace or script so later model versions can be compared without reconstructing undocumented conditions.

Part 2 - Decision vector

Let $G = (C, L, T, M, Q, R, O)$ represent correctness, latency, throughput, memory, quality, reliability, and observability. Release requires every blocking component to meet its threshold.

Part 3 - Required evidence

1. Identify model, tokenizer, code, configuration, hardware, and data traces.
2. Prove cached, batched, and compressed parity.
3. Report TTFT, token gaps, tail latency, throughput, and cache memory.

4. Include cold start, overload, soak, cancellation, and recovery.
5. Document API limits, canary, monitoring, and rollback.
6. State capacity, unsupported cases, and residual risks.

Part 4 - Structured report

```
class InferenceReport:
    def __init__(self, report_id):
        self.report_id = report_id
        self.sections = {}
        self.gates = []

    def add_section(self, name, evidence):
        self.sections[name] = evidence

    def add_gate(self, name, passed, blocking, owner):
        self.gates.append({
            "name": name, "passed": passed,
            "blocking": blocking, "owner": owner
        })

    def ready(self):
        for gate in self.gates:
            if gate["blocking"] and not gate["passed"]:
                return False
        return True

def E19():
    report = InferenceReport("LLM05-INFERENCE-V1")
    report.add_section("architecture", {"status": "documented"})
    report.add_gate("cache_parity", False, True, "inference_owner")
    return report, report.ready()
```

Part 5 - The Philosophy Behind the Exercise

A production report converts performance into accountable knowledge. It records what the service can sustain, where it fails, and who must act when reality moves beyond tested boundaries.

18.8 LLM06

Chapter Title: Quantization and Deployment
Specialization: Applied AI and Deployment

18.8.1 Exercise 00

Exercise Title: Explain Quantization and Why It Is Critical for LLM Deployment

Part 1 - Why this matters

Quantization represents selected model tensors with fewer discrete numerical levels. A floating-point weight may use 16 or 32 bits; an integer representation may use 8 or 4 bits together with scale metadata. For an LLM containing billions of parameters, this can reduce weight storage, memory traffic, and the hardware capacity required for inference. Quantization is critical because decoding is often limited by moving weights from memory rather than by arithmetic alone.

Separate four layers of the claim. The serialized artifact may be smaller. The resident model may consume less device memory. A supported kernel may execute lower-bit matrix multiplication or dequantize efficiently. Finally, end-to-end latency and quality may improve or degrade. None follows automatically from the previous one. An int4 file that is expanded to fp16 before every operation can save storage while losing runtime performance.

Implement affine quantization for a tiny tensor by hand. Inspect symmetric and asymmetric schemes, per-tensor and per-channel scales, rounding error, clipping, outliers, and dequantization. Then connect the mechanism to LLM weight matrices, activation distributions, KV cache, embeddings, normalization, and logits. State which components are quantized, which remain in higher precision, and where conversion occurs.

Part 2 - Mathematical form and reasoning

For real value x, integer range $[q_{\min}, q_{\max}]$, positive scale s, and zero point z, affine quantization is

$$q = \operatorname{clip}\left(\operatorname{round}(x/s) + z, q_{\min}, q_{\max}\right), \qquad \widehat{x} = s(q - z).$$

For symmetric signed quantization, $z = 0$ and $s = \max_i |x_i|/q_{\max}$. Quantization error is $e = x - \widehat{x}$. Under ideal rounding without clipping, $|e| \leq s/2$; saturation violates that bound. Weight storage falls approximately from NP bits to NQ bits plus scales, where P and Q are original and quantized widths.

Part 3 - Hints and Helper

1. Use a small tensor with zero, ordinary values, and one outlier so scale selection can be checked manually.
2. Report maximum error, mean squared error, cosine similarity, clipping rate, and storage including scale/zero-point metadata.
3. Distinguish fake quantization, packed storage, and a genuinely accelerated quantized kernel.
4. Measure prefill and token-by-token decode separately; their bottlenecks and batch behavior differ.
5. Record model, quantization recipe, calibration data, runtime, hardware, kernel backend, and fallback precision as one deployment contract.

Part 4 - Python code skeleton

```
import torch

def symmetric_quantize(x, bits=8):
    q_max = (2 ** (bits - 1)) - 1
    q_min = -(2 ** (bits - 1))
    maximum = x.abs().max()
    scale = maximum / q_max if maximum > 0 else x.new_tensor(1.0)
    quantized = torch.round(x / scale)
    quantized = torch.clamp(quantized, q_min, q_max)
    integer_dtype = torch.int8 if bits == 8 else torch.int32
    quantized = quantized.to(integer_dtype)
    restored = quantized.to(torch.float32) * scale
    return quantized, scale, restored

def E00():
    weights = torch.tensor([-8.0, -0.4, 0.0, 0.5, 1.2])
    quantized, scale, restored = symmetric_quantize(weights, bits=8)
    error = weights - restored
    return {"quantized": quantized, "scale": scale,
            "restored": restored,
            "maximum_error": error.abs().max().item(),
            "mean_squared_error": error.square().mean().item()}

def main():
    print(E00())

if __name__ == "__main__":
    main()
```

Part 5 - The Philosophy Behind the Exercise

Quantization asks how much numerical detail a learned function truly needs. It removes representational precision while trying to preserve behavioral meaning. The operation is therefore compression under an empirical contract, not a claim that the discarded information never existed.

Deployment succeeds when the representation, kernel, hardware, and quality requirements agree. A smaller model is useful only if the complete serving system preserves the capabilities that matter.

18.8.2 Exercise 01

Exercise Title: Compare Post-Training Quantization vs QAT

Part 1 - Why this matters

Post-training quantization (PTQ) converts a trained model without changing its original optimization process. It is comparatively fast and requires little or no labeled training data, but

its scales and clipping policy must represent the deployment distribution. Quantization-aware training (QAT) inserts simulated quantization into forward passes so parameters can adapt to rounding and saturation while gradients update higher-precision master weights. QAT can recover quality at aggressive bit widths, but it adds cost, data requirements, and training risk.

Build a matched comparison. Start from one floating checkpoint, define the same target backend and quantization granularity, and evaluate dynamic or calibrated PTQ. Then fine-tune a clone with fake-quantization modules for QAT. Freeze or stabilize observers at a documented point, use a conservative learning rate, and export both into the same deployable representation. Comparing PTQ fake quantization against a fully packed QAT runtime would confound method and kernel.

Measure task quality, perplexity, calibration, output divergence, saturation, artifact size, conversion time, QAT training cost, resident memory, prefill latency, decode latency, and throughput. Include in-domain and shifted prompts. PTQ should remain the default when it satisfies requirements; QAT is justified when measured recovery is worth the additional training and governance burden.

Part 2 - Mathematical form and reasoning

PTQ chooses quantizer parameters ϕ after training:

$$\theta^* = \arg\min_{\theta} L(\theta), \qquad \phi^* = \text{Calibrate}(\theta^*, D_c).$$

QAT optimizes through a fake-quantized forward map Q_ϕ:

$$\min_{\theta,\phi} L(Q_\phi(\theta); D_t).$$

Because rounding has zero derivative almost everywhere, QAT commonly uses a straight-through estimator, treating its derivative as approximately one in a selected range. This is a useful optimization approximation, not an exact derivative of integer arithmetic.

Part 3 - Hints and Helper

1. Keep original checkpoint, evaluation prompts, backend, bit width, and grouping identical across methods.
2. Check calibration coverage for length, language, domain, and activation outliers; sample count alone is insufficient.
3. Save observer states and document when ranges stop updating during QAT.
4. Compare final exported kernels, not only fake-quantized PyTorch modules.
5. Add a no-quantization fine-tune control to separate QAT recovery from ordinary additional training.

Part 4 - Python code skeleton

```
import copy
import torch

def run_ptq(float_model, calibration_loader, quantize_function):
    model = copy.deepcopy(float_model).eval()
    return quantize_function(model, calibration_loader)

def run_qat(float_model, train_loader, prepare_qat, steps=200):
    model = copy.deepcopy(float_model).train()
    model = prepare_qat(model)
    optimizer = torch.optim.AdamW(model.parameters(), lr=1e-5)
    iterator = iter(train_loader)
    for step in range(steps):
        try:
            batch = next(iterator)
        except StopIteration:
            iterator = iter(train_loader)
            batch = next(iterator)
        optimizer.zero_grad(set_to_none=True)
        loss = model.compute_loss(batch)
        loss.backward()
        optimizer.step()
    model.eval()
    return model

def E01(float_model, loaders, tools, evaluate):
    ptq_model = run_ptq(float_model, loaders["calibration"], tools["ptq"])
    qat_model = run_qat(float_model, loaders["train"], tools["prepare_qat"])
    return {"float": evaluate(float_model), "ptq": evaluate(ptq_model),
            "qat": evaluate(qat_model)}
```

Part 5 - The Philosophy Behind the Exercise

PTQ asks whether an existing intelligence can survive a new numerical body. QAT allows that intelligence to adapt before the transition. One preserves the training history; the other reopens it.

The more invasive method is not automatically more advanced. Engineering maturity means choosing the least complex intervention that satisfies measured quality and operational constraints.

18.8.3 Exercise 02

Exercise Title: Implement int8 Quantization Conceptually and Measure Accuracy Drop

Part 1 - Why this matters

Int8 is a useful first low-precision target because it provides 256 codes and is supported by many CPUs and accelerators. Yet “int8 model” is incomplete: it may mean weight-only int8 with floating activations, weight-and-activation int8, dynamic activation scales, or static

calibration. Each choice has different accuracy, memory, and kernel implications.

Implement per-tensor and per-output-channel symmetric int8 weight quantization for a linear layer. Store int8 weights and scales, then dequantize for a reference calculation. This reference proves the numerical contract but is not an optimized deployment kernel. Compare its output with the original layer on normal inputs, outlier inputs, and an evaluation set. Per-channel scales often preserve rows with different ranges better than one global scale.

Measure layer-wise error, logit divergence, top-token agreement, perplexity, task accuracy, and generated-output quality. Report absolute change from the float baseline and confidence intervals where examples are finite. Locate the layers responsible for degradation by quantizing one block at a time. Accuracy drop is acceptable only relative to a predefined deployment budget; do not choose the threshold after observing the result.

Part 2 - Mathematical form and reasoning

For weight matrix $W \in \mathbb{R}^{m \times n}$, per-row scales are

$$s_i = \frac{\max_j |W_{ij}|}{127}, \qquad Q_{ij} = \text{clip}(\text{round}(W_{ij}/s_i), -128, 127).$$

Reference output is

$$\widehat{Y} = X\widehat{W}^\top + b, qquad\widehat{W}_{ij} = s_i Q_{ij}.$$

Measure relative output error $\|Y - \widehat{Y}\|_2/(\|Y\|_2 + \epsilon)$ and task delta $\Delta A = A_{\text{int8}} - A_{\text{float}}$. These describe different levels of parity and should not be substituted for one another.

Part 3 - Hints and Helper

1. Handle all-zero rows explicitly so scale never becomes zero.
2. Quantize bias separately or retain it at higher precision; document the accumulator and output dtype.
3. Compare per-tensor and per-channel schemes with identical weights/inputs.
4. Inspect errors by layer and token position, especially the final LM head.
5. Distinguish reference dequantization from real int8 GEMM in every speed claim.

Part 4 - Python code skeleton

```
import torch
import torch.nn.functional as F

def quantize_int8_per_row(weight):
    maximum = weight.abs().amax(dim=1, keepdim=True)
    scale = maximum / 127.0
    scale = torch.where(scale == 0, torch.ones_like(scale), scale)
    integer_weight = torch.round(weight / scale)
    integer_weight = torch.clamp(integer_weight, -128, 127).to(torch.int8)
```

```
    return integer_weight, scale

def reference_quantized_linear(x, integer_weight, scale, bias=None):
    restored_weight = integer_weight.to(x.dtype) * scale.to(x.dtype)
    return F.linear(x, restored_weight, bias)

def E02(linear_layer, evaluation_inputs):
    weight = linear_layer.weight.detach()
    integer_weight, scale = quantize_int8_per_row(weight)
    baseline = linear_layer(evaluation_inputs)
    quantized = reference_quantized_linear(
        evaluation_inputs, integer_weight, scale, linear_layer.bias)
    difference = baseline - quantized
    relative_error = difference.norm() / baseline.norm().clamp_min(1e-12)
    return {"int8_weight": integer_weight, "scale": scale,
            "maximum_error": difference.abs().max().item(),
            "relative_error": relative_error.item()}
```

Part 5 - The Philosophy Behind the Exercise

Accuracy loss is not a property of eight bits in isolation. It emerges from how those bits are assigned across unequal ranges and how small numerical errors propagate through a learned computation.

The right unit of judgment is behavior. Tensor error helps locate mechanisms; task quality determines whether the compressed model still fulfills its role.

18.8.4 Exercise 03

Exercise Title: Implement Mixed Precision Deployment Strategy

Part 1 - Why this matters

Uniformly quantizing every component is rarely optimal. Large matrix weights may tolerate int8 or int4, while normalization, softmax, residual accumulation, output logits, embeddings, or outlier-sensitive projections may need fp16, bfloat16, or fp32. A mixed-precision strategy assigns precision by measured sensitivity, kernel support, memory value, and conversion overhead rather than by one global label.

Build a module policy that records precision, quantization granularity, accumulator dtype, output dtype, device, and backend. Begin with a float baseline, quantize one module family at a time, and rank sensitivity by quality change and output error. Keep numerically sensitive operations in higher precision and avoid repeated quantize/dequantize boundaries that erase performance gains. Confirm tied weights are not accidentally duplicated under inconsistent rules.

Evaluate candidate policies under representative prompt lengths, batch sizes, and generation settings. Record weight memory, KV-cache memory, temporary workspace, prefill/decode

latency, throughput, quality, non-finite events, and conversion count. The best strategy is normally a Pareto point: lowering one more component may save little memory but introduce a large quality or latency penalty.

Part 2 - Mathematical form and reasoning

Let modules be $M_1, \ldots, M_K$ and precision assignment $p = (p_1, \ldots, p_K)$. Deployment design can be written

$$\min_p \ \alpha M(p) + \beta T(p) + \gamma E(p) \quad \text{subject to} \quad Q(p) \geq Q_{\min},$$

where M is memory, T latency, E energy, and Q quality. Interface conversion adds cost

$$C(p) = \sum_{k=1}^{K-1} c(p_k, p_{k+1}),$$

so locally lower precision may worsen the system if conversions dominate.

Part 3 - Hints and Helper

1. Start with a conservative recipe: quantized linear weights and higher- precision norms, softmax, residuals, and logits.
2. Store policy by stable module path and fail when the architecture changes and a rule no longer matches.
3. Measure module sensitivity on multiple data slices; outliers may be domain- or language-specific.
4. Audit actual parameter/storage dtypes and kernel traces after conversion. The configuration file alone is not evidence.
5. Include a higher-precision override list and its measured memory cost.

Part 4 - Python code skeleton

```
from dataclasses import dataclass, asdict
import torch

@dataclass
class PrecisionRule:
    module_path: str
    weight_dtype: str
    activation_dtype: str
    accumulator_dtype: str
    reason: str

def build_policy(model, sensitive_paths):
    rules = []
    for path, module in model.named_modules():
        if isinstance(module, torch.nn.Linear):
```

```
            weight_dtype = "fp16" if path in sensitive_paths else "int8"
            rule = PrecisionRule(path, weight_dtype, "fp16", "fp32",
                                 "sensitivity experiment")
            rules.append(rule)
    return rules

def validate_policy(model, rules):
    available = {path for path, module in model.named_modules()}
    for rule in rules:
        if rule.module_path not in available:
            raise KeyError(f"unknown module path: {rule.module_path}")
    return True

def E03(model, sensitive_paths):
    rules = build_policy(model, sensitive_paths)
    validate_policy(model, rules)
    return [asdict(rule) for rule in rules]
```

Part 5 - The Philosophy Behind the Exercise

Mixed precision rejects the idea that every calculation deserves the same numerical resources. It spends precision where the model is sensitive and removes it where redundancy has been demonstrated.

This is not inconsistency; it is allocation. A good deployment recognizes that different parts of an intelligent system carry different numerical risks and must be treated according to evidence.

18.8.5 Exercise 04

Exercise Title: Benchmark Latency and Memory Before and After Quantization

Part 1 - Why this matters

Quantization is a systems intervention whose value must be measured end to end. Artifact size, resident memory, and peak memory are different quantities. Likewise, cold-start latency, prompt prefill latency, time to first token, inter-token latency, and request completion time describe different stages. A smaller checkpoint can load faster but decode slower if the runtime lacks a native kernel or repeatedly dequantizes weights.

Create float and quantized deployments from the same checkpoint and run them on the same hardware, runtime version, prompts, batch sizes, context lengths, generation lengths, and sampling policy. Separate model loading and warmup from steady-state measurement. Synchronize accelerators at timer boundaries and measure many repetitions after warmup. Randomize deployment order or run in fresh processes to reduce cache and allocator bias.

Record serialized bytes, host RSS, device allocated/reserved memory, KV-cache growth, workspace memory, time to first token, prefill tokens/s, decode tokens/s, p50/p95/p99 request latency, and maximum concurrency before failure. Validate quality parity before interpreting

speed. Report confidence intervals or run dispersion; one favorable timing is not a deployment claim.

Part 2 - Mathematical form and reasoning

For request with prompt length T_p and generated length T_g,

$$t_{\text{total}} = t_{\text{queue}} + t_{\text{prefill}} + \sum_{i=1}^{T_g} t_{\text{decode},i}.$$

Memory can be decomposed as

$$M = M_{\text{weights}} + M_{\text{KV}} + M_{\text{activations}} + M_{\text{workspace}} + M_{\text{allocator}}.$$

Weight compression ratio is $M_{w,\text{float}}/M_{w,q}$, while end-to-end memory ratio uses total measured M. They diverge as context, batch, and KV cache grow.

Part 3 - Hints and Helper

1. Use fixed prompt-token IDs to exclude tokenizer variance from model timing.
2. Measure prefill and decode separately across a grid of prompt and batch sizes.
3. Capture kernel traces to prove the intended quantized operators execute.
4. Reset peak memory statistics and use isolated processes for clean memory comparisons.
5. Include quality, errors, and failed requests beside latency and memory.

Part 4 - Python code skeleton

```
import statistics
import time
import torch

def synchronized_time(function):
    if torch.cuda.is_available():
        torch.cuda.synchronize()
    start = time.perf_counter()
    result = function()
    if torch.cuda.is_available():
        torch.cuda.synchronize()
    return time.perf_counter() - start, result

def benchmark_model(model, request, warmup=5, repeats=30):
    for repeat in range(warmup):
        model.generate(**request)
    latencies = []
    for repeat in range(repeats):
        elapsed, output = synchronized_time(
            lambda: model.generate(**request))
```

```
        latencies.append(elapsed)
    ordered = sorted(latencies)
    return {"median_seconds": statistics.median(ordered),
            "p95_seconds": ordered[int(0.95 * (len(ordered) - 1))],
            "last_output": output}

def E04(float_model, quantized_model, request):
    return {"float": benchmark_model(float_model, request),
            "quantized": benchmark_model(quantized_model, request)}
```

Part 5 - The Philosophy Behind the Exercise

Performance is not an attribute stored inside a model file. It appears when a representation meets a runtime, a workload, and a machine. Benchmarking makes that meeting observable.

A valid optimization claim names the boundary of its truth: which stage became faster, under which load, at what quality, and with what memory consequence.

18.8.6 Exercise 05

Exercise Title: Debug Quantization-Induced NaNs or Saturation

Part 1 - Why this matters

Integer tensors themselves do not represent NaN, but quantized pipelines can produce invalid floating outputs through zero or non-finite scales, overflowed accumulators, broken dequantization, unsupported kernels, or later operations such as softmax and normalization. Saturation is quieter: values are clipped to the minimum or maximum code, remain finite, and may corrupt behavior without an exception.

Instrument every quantize/dequantize boundary. Record observed min/max, selected scale and zero point, fraction at $q_{\min}$ and $q_{\max}$, mean and maximum reconstruction error, accumulator range, output dtype, and non-finite count. Inspect per-channel statistics because one outlier channel can determine a global scale while small channels lose resolution. Save the first failing input and module path.

Inject failures deliberately: all-zero calibration, a single extreme outlier, an incorrect signed or unsigned range, zero scale, and a mismatched scale shape. Also test a distribution shift beyond calibration. Compare percentile clipping, MSE-based clipping, per-channel scales, higher-precision retention of outlier channels, and wider accumulators. Fix the earliest broken invariant, then rerun end-to-end quality; finite outputs alone do not prove recovery.

Part 2 - Mathematical form and reasoning

Saturation rate for codes q_i is

$$r_{\text{sat}} = \frac{1}{N} \sum_i \mathbb{1}[q_i = q_{\min} \ \vee \ q_i = q_{\max}].$$

For integer dot product $a = \sum_{j=1}^{n} q_j^x q_j^w$, worst-case magnitude is

$$|a| \le n \max |q^x| \max |q^w|.$$

This bound determines whether an int32 accumulator is safe. Dequantization $\widehat{y} = s_x s_w a$ requires finite positive scales of broadcast-compatible shape.

Part 3 - Hints and Helper

1. Fail immediately when scales are non-finite, non-positive, or have the wrong axis.
2. Log low- and high-end saturation separately; asymmetric mistakes may affect only one side.
3. Replay the same failing tensor through float, reference quantization, and deployment kernel to localize the discrepancy.
4. Use histograms and percentiles, not just minimum and maximum.
5. Convert every resolved incident into a permanent assertion and regression case.

Part 4 - Python code skeleton

```
import torch

def inspect_quantizer(x, scale, zero_point, q_min=-128, q_max=127):
    if not torch.isfinite(scale).all() or not torch.all(scale > 0):
        raise FloatingPointError("invalid quantization scale")
    raw = torch.round(x / scale) + zero_point
    low = raw <= q_min
    high = raw >= q_max
    q = torch.clamp(raw, q_min, q_max)
    restored = scale * (q - zero_point)
    if not torch.isfinite(restored).all():
        raise FloatingPointError("invalid dequantized tensor")
    return {"low_saturation": low.float().mean().item(),
            "high_saturation": high.float().mean().item(),
            "maximum_error": (x - restored).abs().max().item(),
            "restored": restored}

def E05():
    ordinary = torch.randn(1024)
    shifted = ordinary.clone()
    shifted[0] = 1000.0
    scale = shifted.abs().max() / 127.0
    return inspect_quantizer(shifted, scale, torch.tensor(0.0))
```

Part 5 - The Philosophy Behind the Exercise

Saturation is information loss with a finite face. It reminds us that numerical validity is weaker than semantic fidelity: a tensor can contain legal numbers while no longer expressing the distinctions the model learned.

Debugging quantization means tracing where a representational promise first failed. The final NaN or bad answer is only the downstream witness.

18.8.7 Exercise 06

Exercise Title: Deploy Quantized Model and Validate Output Parity

Part 1 - Why this matters

Deployment parity cannot require bitwise equality between floating and quantized LLMs; quantization intentionally changes numbers and small logit differences can change greedy or sampled continuations. Parity must be defined at several levels: tensor contracts, finite execution, bounded logits, token ranking, task metrics, generation behavior, safety rules, latency, and failure handling. Thresholds must be specified before release.

Create a golden request suite covering short and long prompts, multiple languages/domains, empty or invalid inputs, EOS behavior, stop strings, maximum context, and adversarial numerical cases. Run the float and quantized deployments with identical token IDs, attention masks, generation settings, seeds, and runtime contract. Compare prefill logits and stepwise decode logits before comparing final text; final divergence alone does not locate the source.

Use greedy decoding for deterministic token-agreement tests and fixed-seed sampling for distributional tests. Compare top-k overlap, KL or Jensen–Shannon divergence, selected-token probability, sequence agreement until first divergence, task accuracy, refusal/safety behavior, and human evaluation where needed. Validate cold start, repeated requests, memory stability, timeout, malformed input, and restart. Save model and tokenizer fingerprints with results.

Part 2 - Mathematical form and reasoning

For float and quantized next-token distributions p and q, top-k overlap is

$$O_k = \frac{|\operatorname{TopK}(p) \cap \operatorname{TopK}(q)|}{k}.$$

Jensen–Shannon divergence is

$$\operatorname{JS}(p, q) = \frac{1}{2}\operatorname{KL}(p\|m) + \frac{1}{2}\operatorname{KL}(q\|m), \qquad m = (p + q)/2.$$

Task parity may require $Q_q \geq Q_f - \delta_Q$, while service parity imposes schema, status-code, and latency constraints. Passing one level does not imply the others.

Part 3 - Hints and Helper

1. Compare token IDs before decoded strings; tokenizer differences must not be blamed on quantization.
2. Keep sampling settings identical and isolate stochastic variance with multiple controlled seeds.
3. Track the first layer/token where logit divergence exceeds its threshold.
4. Test repeated load/unload and long generation for memory growth.
5. Store baseline, thresholds, raw outputs, environment, and decision in a versioned parity report.

Part 4 - Python code skeleton

```
import torch

def distribution_metrics(float_logits, quantized_logits, k=10):
    p = float_logits.float().softmax(dim=-1)
    q = quantized_logits.float().softmax(dim=-1)
    top_p = set(p.topk(k).indices.tolist())
    top_q = set(q.topk(k).indices.tolist())
    overlap = len(top_p.intersection(top_q)) / k
    midpoint = 0.5 * (p + q)
    kl_p = (p * (p.clamp_min(1e-12).log() -
                 midpoint.clamp_min(1e-12).log())).sum()
    kl_q = (q * (q.clamp_min(1e-12).log() -
                 midpoint.clamp_min(1e-12).log())).sum()
    return {"top_k_overlap": overlap,
            "jensen_shannon": (0.5 * (kl_p + kl_q)).item()}

def E06(float_service, quantized_service, golden_requests):
    records = []
    for request in golden_requests:
        float_result = float_service.debug_generate(request)
        quantized_result = quantized_service.debug_generate(request)
        metrics = distribution_metrics(float_result["last_logits"],
                                       quantized_result["last_logits"])
        records.append({"request_id": request["id"], **metrics})
    return records
```

Part 5 - The Philosophy Behind the Exercise

Parity is not sameness; it is preservation of the properties that define an acceptable service. Quantization changes internal arithmetic, so deployment must specify which external meanings may change and by how much.

The discipline of parity turns “looks similar” into a layered contract. It makes approximation governable without pretending approximation is identity.

18.8.8 Exercise 07

Exercise Title: Compare int8 vs int4 Tradeoffs Conceptually

Part 1 - Why this matters

Int4 roughly halves raw weight storage relative to int8, but its 16 codes leave far less resolution. Scale metadata, group size, packing, kernel availability, dequantization, and outlier handling become more important. Some accelerators have excellent int8 kernels but weak int4 support; int4 can then reduce model size without improving latency. The comparison must be recipe-and-hardware specific.

Quantize identical weight matrices to int8 and grouped int4. Because PyTorch does not generally expose a universal native int4 tensor for arbitrary computation, represent two signed 4-bit codes in one byte for storage and unpack them in a transparent reference path. Vary group size: smaller groups provide more local scales and lower error but add metadata and may reduce kernel efficiency. Preserve outlier channels or selected modules in higher precision as separate conditions.

Compare total artifact bytes including scales, resident memory, and conversion time. Measure prefill latency, decode latency, throughput, power, reconstruction error, logit divergence, perplexity, task quality, and safety metrics. Include multiple batch and context sizes. Int4 is justified when its additional memory or capacity gains are operationally valuable and the runtime actually realizes them within the quality budget.

Part 2 - Mathematical form and reasoning

Signed symmetric b-bit range uses

$$q_{\max} = 2^{b-1} - 1, qquad q_{\min} = -2^{b-1}.$$

For group size G, each group g has scale

$$s_g = \max_{i \in g} |w_i| / q_{\max}.$$

Approximate weight-plus-scale storage per parameter is $b + b_s/G$, where b_s is scale width. Thus int4 does not achieve exactly twofold compression over int8 once metadata, alignment, and higher-precision exceptions are included.

Part 3 - Hints and Helper

1. Compare int8 and int4 using the same grouping convention where possible, then study group size separately.
2. Verify packing/unpacking with all codes and odd tensor lengths.
3. Measure the actual packed artifact, not an int32 reference tensor used for convenience.
4. Inspect kernel traces and conversion counts before claiming speedup.
5. Report quality by layer sensitivity and data slice; four-bit failures may concentrate rather than appear uniformly.

Part 4 - Python code skeleton

```
import torch

def pack_unsigned_int4(values):
    values = values.to(torch.uint8)
    if torch.any(values > 15):
        raise ValueError("int4 code must be between 0 and 15")
    if values.numel() % 2 == 1:
        values = torch.cat([values, torch.zeros(1, dtype=torch.uint8)])
```

```
    low = values[0::2]
    high = values[1::2] << 4
    return low | high

def unpack_unsigned_int4(packed, original_length):
    output = torch.empty(packed.numel() * 2, dtype=torch.uint8)
    output[0::2] = packed & 0x0F
    output[1::2] = (packed >> 4) & 0x0F
    return output[:original_length]

def E07():
    codes = torch.arange(16, dtype=torch.uint8)
    packed = pack_unsigned_int4(codes)
    restored = unpack_unsigned_int4(packed, codes.numel())
    assert torch.equal(codes, restored)
    return {"original_bytes": codes.numel() * codes.element_size(),
            "packed_bytes": packed.numel() * packed.element_size()}
```

Part 5 - The Philosophy Behind the Exercise

Lower bit width is a stronger act of forgetting. It can free enough memory to make deployment possible, but it leaves fewer distinctions through which the model may express what it learned.

The correct comparison is not "four is smaller than eight." It is whether the entire system converts the additional forgetting into useful capacity without crossing the behavioral boundary stakeholders require.

18.8.9 Exercise 08

Exercise Title: Analyze Hardware Constraints for Quantized Inference

Part 1 - Why this matters

A quantization format has value only where the target hardware and runtime can execute it effectively. CPU vector instructions, GPU compute capability, tensor core modes, memory bandwidth, driver/runtime versions, kernel libraries, and supported matrix shapes determine whether int8 or int4 becomes faster, merely smaller, or unsupported. A recipe validated on one GPU generation cannot be assumed portable to another.

Build a capability probe before model loading. Record CPU architecture and instruction support, accelerator name and capability, total/free memory, runtime/library versions, supported quantized operators, accumulator types, alignment/group-size constraints, maximum context and batch assumptions, and whether the backend performs weight-only or weight-activation quantization. Execute a tiny known operator through the intended kernel and compare it with a float reference. Feature flags without a successful operator test are insufficient evidence.

Create a compatibility matrix across the actual deployment fleet. For each device/backend pair, classify recipes as native, emulated/dequantized, or unsupported. Benchmark representative shapes from prefill and decode because kernel efficiency changes with matrix dimensions and batch size. Include memory needed for weights, KV cache, workspace, and concurrency headroom. Choose a portable baseline and optional hardware-specific optimized profiles.

Part 2 - Mathematical form and reasoning

For arithmetic intensity $I = F/B_m$ (operations per byte moved) and hardware compute C with bandwidth W, roofline throughput is bounded by

$$P \leq \min(C, IW).$$

Quantization reduces bytes B_m and can raise I, but unpacking and dequantization add operations and temporary traffic. Deployment feasibility also requires

$$M_{\text{weights}} + M_{\text{KV}} + M_{\text{workspace}} + M_{\text{headroom}} \leq M_{\text{available}}.$$

Both constraints must be measured for real shapes.

Part 3 - Hints and Helper

1. Probe capabilities at process startup and store the chosen backend/profile in every request log.
2. Test small-batch decode and large prefill matrices separately.
3. Detect silent fallback to floating kernels through profiler traces and actual tensor/storage inspection.
4. Include driver and runtime versions in cache keys for compiled engines.
5. Fail closed or select a validated higher-precision profile when a required kernel or memory constraint is absent.

Part 4 - Python code skeleton

```
import platform
import torch

def hardware_manifest():
    manifest = {"machine": platform.machine(),
                "python": platform.python_version(),
                "torch": torch.__version__,
                "cuda_available": torch.cuda.is_available()}
    if torch.cuda.is_available():
        index = torch.cuda.current_device()
        properties = torch.cuda.get_device_properties(index)
        manifest["gpu_name"] = properties.name
        manifest["compute_capability"] = (
            properties.major, properties.minor)
```

```
        manifest["total_memory"] = properties.total_memory
        manifest["cuda_runtime"] = torch.version.cuda
    return manifest

def select_profile(manifest, profiles):
    for profile in profiles:
        if profile["supports"](manifest):
            return profile["name"]
    raise RuntimeError("no validated inference profile for this hardware")

def E08(profiles):
    manifest = hardware_manifest()
    manifest["selected_profile"] = select_profile(manifest, profiles)
    return manifest
```

Part 5 - The Philosophy Behind the Exercise

Algorithms do not run in abstraction. Their practical meaning is completed by instruction sets, memory systems, kernels, and runtimes. Quantization is thus a relationship between a representation and a machine.

Portability is not pretending every machine is identical. It is making differences explicit enough to choose a validated path on each one.

18.8.10 Exercise 09

Exercise Title: Implement Fallback to Higher Precision on Failure

Part 1 - Why this matters

A quantized deployment may fail because of unsupported shapes, backend errors, non-finite outputs, parity guard violations, memory pressure, or a newly discovered quality defect. A higher-precision fallback can preserve service availability, but only if it is bounded, observable, and independently validated. Blindly retrying every exception can double latency, exhaust memory, repeat unsafe side effects, or conceal a fleet-wide quantized failure.

Define which failures are retryable. Validate requests before either backend, attach a stable request ID, and make generation attempts idempotent from the service perspective. Use a circuit breaker: after repeated quantized failures, stop routing new traffic to that profile for a cooldown period while health checks continue. Limit retry count and total deadline. Preload the fallback or reserve capacity so the recovery path is real under load, not merely present in code.

Log primary failure category, model/profile fingerprints, hardware, prompt shape, fallback decision, added latency, and final outcome. Return response metadata that allows operators to measure fallback rate without exposing internal details to untrusted clients. Inject kernel exceptions, NaNs, timeouts, and parity failures. Test what happens when both paths fail and ensure alerts and rollback policy activate.

Part 2 - Mathematical form and reasoning

If primary failure probability is p, primary latency t_q, fallback latency t_f, and failure detection cost t_d, expected latency with one retry is

$$\mathbb{E}[T] = (1-p)t_q + p(t_d + t_f).$$

Availability improves only if fallback has capacity and independent failure modes. A circuit may open when failures in a window exceed threshold:

$$\frac{N_{\text{primary failures}}}{N_{\text{primary attempts}}} > r_{\max} \quad \wedge \quad N_{\text{attempts}} \geq N_{\min}.$$

Part 3 - Hints and Helper

1. Enumerate retryable and non-retryable errors; invalid input should not be repeated at higher precision.
2. Share tokenizer and request validation across paths, but keep model execution failures independently observable.
3. Reserve memory/capacity for fallback and load-test simultaneous failover.
4. Prevent recursive fallback and enforce one end-to-end deadline.
5. Alert on fallback rate, not only final request failure; successful fallback can hide primary degradation.

Part 4 - Python code skeleton

```
from dataclasses import dataclass
import time

@dataclass
class InferenceResult:
    output: object
    precision_profile: str
    fallback_used: bool

RETRYABLE = (FloatingPointError, RuntimeError, TimeoutError)

def run_with_fallback(request, quantized, high_precision, deadline_seconds):
    started = time.monotonic()
    try:
        output = quantized.generate(request)
        if not quantized.validate_output(output):
            raise FloatingPointError("quantized output failed guard")
        return InferenceResult(output, "quantized", False)
    except RETRYABLE as error:
        remaining = deadline_seconds - (time.monotonic() - started)
        if remaining <= 0:
            raise TimeoutError("no deadline remains for fallback") from error
```

```
        output = high_precision.generate(request, timeout=remaining)
        return InferenceResult(output, "high_precision", True)

def E09(test_requests, quantized, high_precision):
    results = []
    for request in test_requests:
        results.append(run_with_fallback(
            request, quantized, high_precision, deadline_seconds=10.0))
    return results
```

Part 5 - The Philosophy Behind the Exercise

Fallback is an admission that optimization has a boundary. It preserves the service by returning to a more expensive representation when the compressed one cannot honor its contract.

Resilience is not the absence of failure; it is the ability to change paths without losing accountability. Every successful fallback should remain visible as evidence about the primary system.

18.8.11 Exercise 10

Exercise Title: Integrate Quantized Model into Inference Service

Part 1 - Why this matters

An inference service adds contracts that a notebook does not have: request validation, token limits, batching, queuing, cancellation, timeouts, and streaming. It also requires resource isolation, health checks, readiness, model versioning, observability, and safe shutdown. Quantization changes memory and kernel behavior, so service capacity and concurrency must be measured again rather than copied from the float model.

Build a service boundary around a versioned quantized engine. Validate text or token inputs, generation parameters, maximum prompt and output lengths, and tenant limits before queuing. Separate liveness from readiness: a process can be alive while weights are not loaded or the quantized kernel self-test has failed. Warm the engine with representative shapes before marking it ready. Preserve request IDs and expose model/profile versions in internal telemetry.

Implement bounded dynamic batching based on compatible generation parameters and a maximum queue delay. Support streaming cancellation without leaving KV cache allocations or batch slots behind. Apply backpressure when queue or memory limits are reached. Test malformed inputs, oversized prompts, client disconnect, timeout, engine exception, fallback, concurrent requests, and rolling shutdown. Validate that preprocessing and tokenizer fingerprints match training.

Part 2 - Mathematical form and reasoning

For arrival rate λ and mean service time W, Little's law relates average in-system requests L:

$$L = \lambda W.$$

If utilization approaches one, queue latency can grow sharply even when isolated model latency is acceptable. Batch scheduler must satisfy memory constraint

$$M_{\text{weights}} + \sum_{r \in \mathcal{B}} M_{\text{KV}}(r) + M_{\text{workspace}}(\mathcal{B}) \leq M_{\text{safe}}.$$

Admission control is therefore part of numerical deployment correctness.

Part 3 - Hints and Helper

1. Keep API schema independent of one backend while recording the selected precision profile internally.
2. Use bounded queues and explicit overload responses; unbounded waiting is not graceful degradation.
3. Track queue time, prefill, decode, tokens/s, batch composition, fallback, cancellation, and memory.
4. Run tokenizer and kernel self-tests before readiness becomes true.
5. Verify no request data is accidentally retained in logs or reusable buffers beyond policy.

Part 4 - Python code skeleton

```
from dataclasses import dataclass
import time

@dataclass
class GenerationRequest:
    request_id: str
    prompt: str
    maximum_new_tokens: int
    temperature: float

def validate_request(request, maximum_prompt_characters=20000):
    if not request.prompt:
        raise ValueError("prompt must not be empty")
    if len(request.prompt) > maximum_prompt_characters:
        raise ValueError("prompt exceeds service limit")
    if not 1 <= request.maximum_new_tokens <= 2048:
        raise ValueError("invalid generation length")
    if request.temperature < 0:
        raise ValueError("temperature must be nonnegative")

def handle_request(request, tokenizer, engine):
    validate_request(request)
    queued_at = time.monotonic()
    token_ids = tokenizer.encode(request.prompt)
    output = engine.generate(token_ids,
```

```
                                maximum_new_tokens=request.maximum_new_tokens,
                                temperature=request.temperature)
    return {"request_id": request.request_id,
            "text": tokenizer.decode(output),
            "queue_and_compute_seconds": time.monotonic() - queued_at,
            "model_version": engine.version}

def E10(requests, tokenizer, quantized_engine):
    return [handle_request(request, tokenizer, quantized_engine)
            for request in requests]
```

Part 5 - The Philosophy Behind the Exercise

A deployed model becomes a participant in a larger social and computational system. Its value is mediated by queues, limits, failures, users, and operational rules that do not appear in its weights.

Service engineering turns capability into a dependable relationship. The model must not only produce answers; it must do so within explicit promises about time, resources, identity, and failure.

18.8.12 Exercise 11

Exercise Title: Measure Power Consumption Differences After Quantization

Part 1 - Why this matters

Lower precision may reduce memory traffic and arithmetic energy, but lower latency does not automatically mean lower power at every instant. A quantized kernel may use hardware more intensively, increasing watts while finishing sooner. The meaningful deployment quantities are energy per request, energy per generated token, throughput per watt, and the workload completed within a power or thermal envelope.

Measure float and quantized models on the same device with identical prompt tokens, generated lengths, batch/concurrency, clock policy, ambient/thermal conditions, and runtime. Sample device power at a known interval from before request start through cooldown. Subtract or report idle baseline consistently. Warm up both deployments and alternate trial order to reduce temperature drift. Record temperature, clocks, throttling, utilization, and latency alongside power.

Integrate sampled power over time rather than multiplying one instantaneous reading by latency. Repeat enough trials to report dispersion. Separate model load/cold-start energy from steady inference and prefill from decode when possible. Quantization may permit greater concurrency; compare both equal-load and equal-throughput conditions. State sensor resolution and whether the measurement covers only accelerator power or the entire system.

Part 2 - Mathematical form and reasoning

Energy is the time integral of power:

$$E = \int_{t_0}^{t_1} P(t)\,dt.$$

With sampled values (t_i, P_i), trapezoidal approximation is

$$E \approx \sum_{i=0}^{n-1} \frac{P_i + P_{i+1}}{2}(t_{i+1} - t_i).$$

Report joules per valid generated token E/N_g and throughput per watt $R/\overline{P}$. If subtracting idle power P_0, label dynamic energy $\int (P(t) - P_0)dt$ distinctly from total energy.

Part 3 - Hints and Helper

1. Verify sensor timestamps and units; watts sampled over seconds integrate to joules.
2. Begin sampling before execution and end after the final synchronization.
3. Monitor thermal throttling and randomize float/quantized trial order.
4. Compare equal requests, equal useful tokens, and equal throughput as separate experiments.
5. Report uncertainty and measurement boundary: GPU board, CPU package, or whole-system power.

Part 4 - Python code skeleton

```
def trapezoidal_energy(samples):
    energy_joules = 0.0
    for index in range(len(samples) - 1):
        time_a, power_a = samples[index]
        time_b, power_b = samples[index + 1]
        duration = time_b - time_a
        average_power = 0.5 * (power_a + power_b)
        energy_joules += average_power * duration
    return energy_joules

def summarize_energy(samples, generated_tokens):
    energy = trapezoidal_energy(samples)
    duration = samples[-1][0] - samples[0][0]
    average_power = energy / duration
    return {"energy_joules": energy,
            "average_watts": average_power,
            "joules_per_generated_token": energy / generated_tokens}

def E11(run_and_sample_power, float_model, quantized_model, request):
    report = {}
    for name, model in (("float", float_model),
                        ("quantized", quantized_model)):
        samples, generated_tokens = run_and_sample_power(model, request)
        report[name] = summarize_energy(samples, generated_tokens)
    return report
```

Part 5 - The Philosophy Behind the Exercise

Efficiency is the relationship between useful work and consumed resources. Power describes intensity at a moment; energy describes the accumulated cost of finishing the task. Confusing them can reverse the conclusion.

Responsible deployment measures the physical consequences of abstraction. A numerical representation ultimately becomes memory movement, heat, time, and electricity.

18.8.13 Exercise 12

Exercise Title: Test Quantized Model on Edge or Limited-Memory GPU

Part 1 - Why this matters

Edge and limited-memory devices expose constraints hidden on a development server: small VRAM/RAM, unified-memory contention, limited bandwidth, weaker quantized kernels, thermal throttling, slow storage, and strict power limits. Fitting weights is only the first condition. KV cache, workspaces, tokenizer, runtime, operating system, concurrent processes, and memory fragmentation must also fit throughout the longest supported request.

Define a concrete device envelope and deploy the exact artifact/runtime intended for production. Test cold load, warm load, minimum and maximum prompt, long decode, repeated requests, and concurrency. Measure resident and peak memory, swap or unified-memory pressure, time to first token, decode tokens/s, temperature, clocks, power, and failure recovery. Reserve headroom rather than using every reported byte. On an 8GB GPU, for example, the useful model budget may be materially below 8GB after runtime and KV cache.

Explore controlled mitigations: smaller quantization group, shorter context, KV-cache quantization, CPU/GPU offload, lower batch, paged cache, and selective higher precision. Each changes latency or quality. Test offline operation, restart after interruption, corrupted artifact detection, and low-power/thermal steady state. The final report must state supported prompt/output/concurrency limits instead of implying unlimited inference because one request succeeded.

Part 2 - Mathematical form and reasoning

For L layers, batch B, sequence length T, KV heads H_{kv}, head width d_h, and cache element bytes s, approximate KV cache is

$$M_{KV} \approx 2LBTH_{kv}d_hs.$$

Safe feasibility requires

$$M_w + M_{KV} + M_{workspace} + M_{runtime} + M_{margin} \leq M_{device}.$$

Quantizing weights reduces M_w but does not reduce M_{KV} unless a separate cache policy is used. Long context can therefore erase apparent weight savings.

Part 3 - Hints and Helper

1. Measure memory after load, after prefill, and at maximum decode length.

2. Disable unrelated processes or record their memory so trials are comparable.
3. Soak-test long enough to observe thermal throttling and memory leaks.
4. Treat offload as a new profile with its own transfer and latency metrics.
5. Fail admission before OOM when a request exceeds the validated envelope.

Part 4 - Python code skeleton

```
import torch

def memory_budget(total_bytes, margin_fraction=0.15):
    return int(total_bytes * (1.0 - margin_fraction))

def can_admit(weight_bytes, kv_bytes, workspace_bytes,
              runtime_bytes, total_bytes):
    required = weight_bytes + kv_bytes + workspace_bytes + runtime_bytes
    safe_limit = memory_budget(total_bytes)
    return {"required_bytes": required, "safe_limit_bytes": safe_limit,
            "admitted": required <= safe_limit}

def E12(engine, device_index=0):
    properties = torch.cuda.get_device_properties(device_index)
    torch.cuda.reset_peak_memory_stats(device_index)
    results = []
    for prompt_length in (32, 512, 2048):
        output = engine.run_stress_request(prompt_length=prompt_length,
                                           generated_tokens=256)
        results.append({"prompt_length": prompt_length,
                        "tokens_per_second": output.tokens_per_second,
                        "peak_bytes": torch.cuda.max_memory_allocated(
                            device_index)})
    return {"device": properties.name, "results": results}
```

Part 5 - The Philosophy Behind the Exercise

Edge deployment forces the model to inhabit a finite world. Memory, heat, energy, and time cease to be background infrastructure and become immediate parts of the model's capability.

Constraint is not merely deprivation; it clarifies priorities. A successful edge system states exactly which intelligence it can provide reliably within the resources actually present.

18.8.14 Exercise 13

Exercise Title: Compare Quantized vs Non-Quantized Hallucination Rates

Part 1 - Why this matters

Quantization perturbs logits and may change token rankings, confidence, refusal behavior, and long-generation trajectories. It could increase factual errors in sensitive regions, but ordinary sampling variance can also change answers even without quantization. A hallucination comparison must isolate precision from prompt, retrieval context, decoding policy, evaluator, and randomness. It cannot be inferred from perplexity alone.

Create a frozen evaluation set with answerable factual questions, deliberately unanswerable questions, provided-context questions, and domain slices. Store reference answers, acceptable variants, citations/evidence, and abstention expectations. Run float and quantized models with identical tokenization, prompts, retrieval evidence, generation configuration, and paired random seeds. Use greedy decoding as a deterministic diagnostic and multiple seeds for stochastic settings. Blind evaluators to model version.

Define errors operationally: unsupported factual claim, contradiction with provided evidence, fabricated citation, or failure to abstain. Use rule-based checks where possible and human or validated model-assisted review with an uncertain category. Report paired transitions: correct-to-wrong, wrong-to-correct, abstain-to-claim, and claim-to-abstain. Analyze confidence and logit divergence near changed tokens. Do not conclude causal safety from a small nonsignificant difference.

Part 2 - Mathematical form and reasoning

For paired examples i, let $h_i^f, h_i^q \in \{0, 1\}$ indicate hallucination. Rates are

$$\widehat{H}_f = \frac{1}{N} \sum_i h_i^f, \qquad \widehat{H}_q = \frac{1}{N} \sum_i h_i^q,$$

The paired change is

$$\Delta H = \widehat{H}_q - \widehat{H}_f.$$

Discordant counts n_{01} and n_{10} capture transitions from a correct float answer to a wrong quantized answer, and the reverse transition. Paired tests or bootstrap intervals should operate on examples and preserve this dependence.

Part 3 - Hints and Helper

1. Freeze evaluation definitions and judging rules before seeing model labels.
2. Separate factuality, relevance, citation validity, and abstention rather than collapsing all into one subjective score.
3. Use paired prompts/seeds and retain raw outputs for adjudication.
4. Stratify by domain, language, prompt length, rarity, and answerability.
5. Investigate changed conclusions at token/logit level, then confirm with human-readable evidence.

Part 4 - Python code skeleton

```
from dataclasses import dataclass

@dataclass
class Judgment:
    example_id: str
    float_hallucinated: bool
    quantized_hallucinated: bool

def paired_summary(judgments):
    total = len(judgments)
    float_count = 0
    quantized_count = 0
    correct_to_wrong = 0
    wrong_to_correct = 0
    for judgment in judgments:
        float_count += int(judgment.float_hallucinated)
        quantized_count += int(judgment.quantized_hallucinated)
        became_wrong = (
            not judgment.float_hallucinated
            and judgment.quantized_hallucinated
        )
        became_correct = (
            judgment.float_hallucinated
            and not judgment.quantized_hallucinated
        )
        if became_wrong:
            correct_to_wrong += 1
        if became_correct:
            wrong_to_correct += 1
    return {"float_rate": float_count / total,
            "quantized_rate": quantized_count / total,
            "rate_delta": (quantized_count - float_count) / total,
            "correct_to_wrong": correct_to_wrong,
            "wrong_to_correct": wrong_to_correct}

def E13(run_paired_evaluation, judge, dataset):
    outputs = run_paired_evaluation(dataset)
    judgments = [judge(item) for item in outputs]
    return paired_summary(judgments)
```

Part 5 - The Philosophy Behind the Exercise

Factuality is not a scalar hidden inside weights. It appears in a relationship among prompts, evidence, decoding, and judgment. Quantization may perturb that relationship, but attribution requires controlled comparison.

The exercise teaches epistemic restraint: measure paired behavioral changes, preserve uncertainty, and resist turning absence of detected harm into proof of absence.

18.8.15 Exercise 14

Exercise Title: Build Deployment Compatibility Checklist

Part 1 - Why this matters

Quantized deployment depends on agreements across model architecture, tokenizer, quantization recipe, runtime, kernel backend, hardware, API schema, monitoring, and rollback. A checklist converts those agreements into executable gates. Its purpose is not to collect reassuring checkmarks; it is to prevent an artifact from advancing when one required assumption is unknown or false.

Organize checks into identity, numerical correctness, hardware/runtime, performance, quality/safety, service behavior, observability, security, and recovery. Verify model/tokenizer/config fingerprints, supported operators, scales/group sizes, dtype/accumulator policy, memory headroom, cold and warm load, prefill/decode limits, parity thresholds, hallucination/refusal slices, malformed input, concurrency, fallback, rollback, artifact integrity, and data privacy. Assign an owner and evidence artifact to every gate.

Implement the checklist as structured data and code so CI or a release job can evaluate it. Results should be pass, fail, or explicitly approved waiver with expiry—never silent "not applicable." Re-run hardware-dependent gates for every deployment class and software-dependent gates after runtime/driver changes. Inject failed checks to prove the release is blocked.

Part 2 - Mathematical form and reasoning

Let required gates be $g_i \in \{0, 1\}$. Release readiness is conjunctive:

$$R = \prod_{i \in \mathcal{G}_{required}} g_i.$$

An average score is inappropriate because many minor successes cannot compensate for one missing rollback or corrupted tokenizer. Conditional gates can be defined by implication, for example

$$\text{int4 enabled} \Rightarrow \text{int4 kernel probe passed.}$$

Waivers are time-bounded risk decisions with owner, reason, and compensating control, not conversion of a failure into evidence of success.

Part 3 - Hints and Helper

1. Give every check a stable ID, requirement, owner, evidence path, status, and expiry for waivers.
2. Separate static artifact checks from tests that must execute on each hardware profile.
3. Fail on missing evidence and stale benchmark/evaluation versions.
4. Include training/serving tokenizer and preprocessing parity.
5. Drill fallback and rollback under load; documentation alone is not a passed recovery gate.

Part 4 - Python code skeleton

```
from dataclasses import dataclass
from datetime import date

@dataclass
class Gate:
    gate_id: str
    required: bool
    passed: bool
    evidence: str
    owner: str
    waiver_expiry: str = ""

def validate_gates(gates):
    failures = []
    for gate in gates:
        waiver_valid = False
        if gate.waiver_expiry:
            expiry = date.fromisoformat(gate.waiver_expiry)
            waiver_valid = expiry >= date.today()
        if gate.required and not gate.passed and not waiver_valid:
            failures.append(gate.gate_id)
        if gate.passed and not gate.evidence:
            failures.append(gate.gate_id + ":missing_evidence")
    return {"ready": len(failures) == 0, "failures": failures}

def E14(gates):
    result = validate_gates(gates)
    if not result["ready"]:
        raise RuntimeError(f"deployment blocked: {result['failures']}")
    return result
```

Part 5 - The Philosophy Behind the Exercise

A checklist is a boundary between confidence and wishful thinking. It asks each deployment claim to present evidence before becoming operational reality.

Its conjunctive nature matters: reliability is often lost through one forgotten interface. Explicit gates preserve the memory of how failures actually happen.

18.8.16 Exercise 15

Exercise Title: Debug ONNX Export Issues for Quantized Models

Part 1 - Why this matters

ONNX export crosses several semantic boundaries: PyTorch graph capture, ONNX operator representation, quantized tensor conventions, opset support, shape inference, and target runtime kernels. A model that exports successfully may still dequantize unexpectedly, reject

dynamic sequence lengths, use unsupported quantized operators, or produce different logits. Export is not deployment until the target runtime executes and passes parity.

Start from a tiny submodule containing the failing pattern. Export the float version first, then a reference quantize/dequantize graph, then the intended packed model. Choose opset and quantization representation supported by the target runtime, such as Q/DQ nodes or quantized operator forms. Mark dynamic batch and sequence axes intentionally. Keep scale, zero-point dtype, signedness, axis, and broadcasting consistent. Run graph checker and shape inference before runtime execution.

Compare named intermediate outputs between PyTorch and ONNX Runtime to locate the first divergence. Inspect the graph for unexpected casts, duplicated dequantization, constant folding, and operators assigned to CPU. Test minimum, typical, and maximum shapes; all-zero inputs; outliers; padding masks; and cached decoding if supported. Maintain a table of exporter/runtime/opset/device versions because support changes over time.

Part 2 - Mathematical form and reasoning

Correct export seeks functional agreement

$$f_{\text{PyTorch}}(x;\theta_q) \approx f_{\text{ONNXRuntime}}(x;G_q)$$

for domain $x \in \mathcal{D}_{deploy}$, not only the trace example. For each observed intermediate h_k, define error

$$e_k = \frac{\|h_k^{PT} - h_k^{ONNX}\|_2}{\|h_k^{PT}\|_2 + \epsilon}.$$

The earliest k exceeding tolerance identifies the conversion/operator boundary to investigate. Dynamic-axis correctness requires shape constraints hold for the whole supported domain.

Part 3 - Hints and Helper

1. Reproduce with the smallest module and fixed input before adding dynamic axes and full generation.
2. Pin exporter, opset, runtime, provider, and quantization tool versions.
3. Inspect scale and zero-point initializer dtypes and quantization axes.
4. Verify provider placement; CPU fallback can preserve outputs while ruining latency.
5. Add successful export plus target-runtime inference to CI, not export alone.

Part 4 - Python code skeleton

```
import numpy as np
import torch

def export_model(model, example_ids, path):
    model.eval()
    torch.onnx.export(
```

```
        model,
        (example_ids,),
        path,
        input_names=["input_ids"],
        output_names=["logits"],
        dynamic_axes={"input_ids": {0: "batch", 1: "sequence"},
                      "logits": {0: "batch", 1: "sequence"}},
        opset_version=17,
        do_constant_folding=True)

def compare_outputs(torch_output, runtime_output, atol=1e-3, rtol=1e-3):
    converted = torch_output.detach().cpu().float().numpy()
    if not np.allclose(converted, runtime_output, atol=atol, rtol=rtol):
        error = np.max(np.abs(converted - runtime_output))
        raise AssertionError(f"ONNX parity failed; max error={error}")
    return True

def E15(model, example_ids, runtime_factory, path="quantized_model.onnx"):
    export_model(model, example_ids, path)
    runtime = runtime_factory(path)
    runtime_output = runtime.run(example_ids.cpu().numpy())
    torch_output = model(example_ids)
    return compare_outputs(torch_output, runtime_output)
```

Part 5 - The Philosophy Behind the Exercise

Export is translation between computational languages. A valid file proves syntax; parity proves that meaning survived the translation over the intended input domain.

Debugging succeeds by refusing to treat the graph as a black box. The first divergent intermediate is where two runtimes stopped agreeing about the same computation.

18.8.17 Exercise 16

Exercise Title: Evaluate Latency Under Concurrent Requests

Part 1 - Why this matters

Single-request latency is not production latency. When many clients arrive at the same time, requests compete for admission slots, CPU preprocessing, GPU memory, attention kernels, KV-cache capacity, and output-stream bandwidth. A quantized model may execute each token faster yet still have poor tail latency if the service builds oversized batches, blocks on memory allocation, or lets long prompts delay short interactive requests. This exercise therefore tests the entire serving system rather than timing one isolated forward pass.

Define the workload before interpreting a number. Record the arrival pattern, concurrency, prompt-length distribution, requested output lengths, sampling settings, cache state, precision, batch policy, timeout, and hardware. Separate closed-loop tests, where each client waits before sending another request, from open-loop tests, where arrivals continue independently of completion. A closed-loop test applies backpressure and can conceal overload; an open-loop test

reveals queue growth but must bound the experiment safely.

Capture queue delay, time to first token (TTFT), inter-token latency (ITL), end-to-end latency, tokens per second, cancellation rate, error rate, fallback rate, and actual batch size. Report distributions rather than averages. The 95th and 99th percentiles expose rare stalls that a mean can hide. Stratify the results by prompt and generation length so that a changing workload mix is not mistaken for a runtime improvement. Warm the service, synchronize asynchronous GPU work at measurement boundaries, and exclude neither failures nor timeouts from the final accounting.

Sweep offered load until throughput stops increasing proportionally and tail latency rises sharply. That point is the saturation knee. A valid capacity claim stays below it with explicit headroom. Compare the quantized deployment against the higher-precision baseline under the same request trace. The result must say whether quantization increased sustainable throughput, reduced TTFT, changed ITL, or merely shifted the bottleneck elsewhere.

Part 2 - Mathematical form and reasoning

For request i, decompose total latency into queue and service time:

$$T_i^{\text{total}} = T_i^{\text{queue}} + T_i^{\text{service}}, \qquad T_i^{\text{service}} = T_i^{\text{prefill}} + T_i^{\text{decode}}.$$

TTFT includes admission, queueing, preprocessing, and prefill. A useful decode measure is

$$\text{ITL}_i = \frac{T_{i,\text{last token}} - T_{i,\text{first token}}}{\max(1, N_i^{\text{generated}} - 1)}.$$

Little's law gives a consistency check for a stable interval:

$$L = \lambda W,$$

where L is average requests in the system, λ is completed request rate, and W is average total time. Large disagreement usually indicates measurement-window, warm-up, or incomplete-request accounting errors. Define a capacity point only when all service-level constraints hold simultaneously:

$$C^* = \max_C \left\{ C : p_{99}(\text{TTFT}) \le S_{\text{TTFT}},\ p_{99}(\text{ITL}) \le S_{\text{ITL}},\ r_{\text{error}} \le S_{\text{error}} \right\}.$$

The concurrency with the largest raw throughput is not automatically the best configuration. It may violate interactive latency, fairness, memory headroom, or reliability constraints.

Part 3 - Hints and Helper

1. Create one immutable request trace and replay it against every model variant.
2. Warm model weights, kernels, memory pools, and tokenization before collecting samples.
3. Record client-send, server-admit, generation-start, first-token, and completion timestamps.
4. Sweep concurrency gradually and stop safely if OOM, timeout, or queue growth exceeds a guardrail.

5. Compute p50, p95, and p99 for TTFT, ITL, and end-to-end latency; keep failed requests in separate counters.
6. Stratify by prompt/output length and verify that the compared systems completed equivalent work.
7. Plot offered request rate against achieved rate and tail latency to locate the saturation knee.
8. Repeat the sweep and report variation rather than selecting the most favorable run.

Part 4 - Python code skeleton

```
from concurrent.futures import ThreadPoolExecutor, as_completed
from dataclasses import dataclass
from time import perf_counter
import math


@dataclass
class RequestResult:
    request_id: int
    prompt_tokens: int
    output_tokens: int
    queue_s: float
    ttft_s: float
    total_s: float
    ok: bool
    error: str


def percentile(values, fraction):
    ordered = sorted(values)
    if len(ordered) == 0:
        return math.nan
    position = fraction * (len(ordered) - 1)
    lower = int(math.floor(position))
    upper = int(math.ceil(position))
    if lower == upper:
        return ordered[lower]
    weight = position - lower
    return ordered[lower] * (1.0 - weight) + ordered[upper] * weight


def issue_request(client, request_id, prompt, max_new_tokens):
    submitted_at = perf_counter()
    try:
        response = client.generate_with_timestamps(
            prompt=prompt,
```

```
            max_new_tokens=max_new_tokens,
        )
        completed_at = perf_counter()
        queue_s = response["admitted_at"] - submitted_at
        ttft_s = response["first_token_at"] - submitted_at
        total_s = completed_at - submitted_at
        return RequestResult(
            request_id=request_id,
            prompt_tokens=response["prompt_tokens"],
            output_tokens=response["output_tokens"],
            queue_s=queue_s,
            ttft_s=ttft_s,
            total_s=total_s,
            ok=True,
            error="",
        )
    except Exception as exc:
        completed_at = perf_counter()
        return RequestResult(
            request_id=request_id,
            prompt_tokens=0,
            output_tokens=0,
            queue_s=math.nan,
            ttft_s=math.nan,
            total_s=completed_at - submitted_at,
            ok=False,
            error=type(exc).__name__,
        )


def run_closed_loop(client, requests, concurrency):
    results = []
    started_at = perf_counter()
    with ThreadPoolExecutor(max_workers=concurrency) as executor:
        futures = []
        for request_id, request in enumerate(requests):
            future = executor.submit(
                issue_request,
                client,
                request_id,
                request["prompt"],
                request["max_new_tokens"],
            )
            futures.append(future)
        for future in as_completed(futures):
            results.append(future.result())
    elapsed_s = perf_counter() - started_at
    return results, elapsed_s
```

```
def summarize(results, elapsed_s):
    successful = []
    failed = []
    for result in results:
        if result.ok:
            successful.append(result)
        else:
            failed.append(result)

    ttft = []
    total = []
    total_tokens = 0
    for result in successful:
        ttft.append(result.ttft_s)
        total.append(result.total_s)
        total_tokens += result.output_tokens

    return {
        "requests": len(results),
        "failures": len(failed),
        "p50_ttft_s": percentile(ttft, 0.50),
        "p95_ttft_s": percentile(ttft, 0.95),
        "p99_ttft_s": percentile(ttft, 0.99),
        "p99_total_s": percentile(total, 0.99),
        "request_throughput": len(successful) / elapsed_s,
        "token_throughput": total_tokens / elapsed_s,
    }


# Replay the same trace for concurrency = 1, 2, 4, 8, ... and save every row.
```

Part 5 - The Philosophy Behind the Exercise

Concurrency turns performance from a property of a model into a property of a shared system. Each request is locally reasonable, yet their interaction creates queues, contention, unfairness, and overload. Tail latency is therefore not statistical clutter; it is evidence about coordination under pressure.

Quantization matters only when its saved memory and arithmetic translate into better service for real workloads. This exercise resists the comforting but incomplete benchmark of an isolated kernel. It asks whether the whole system remains timely, fair, observable, and recoverable when many users need it at once.

18.8.18 Exercise 17

Exercise Title: Build Rollback Strategy for Quantized Deployments

Part 1 - Why this matters

Quantization is a behavior-changing deployment transformation. Even after offline parity and quality tests, production can reveal an unsupported prompt shape, a rare outlier, a driver-specific kernel defect, a latency regression, or a safety failure absent from the evaluation set. A rollback strategy limits the duration and blast radius of such surprises. It must be designed and rehearsed before the quantized model receives traffic.

Keep the last approved high-precision artifact deployable, immutable, and addressable by version. Version the model, tokenizer, quantization metadata, runtime image, preprocessing, API schema, and monitoring configuration as one release unit. Merely switching the weights is unsafe if the candidate changed token IDs, cache formats, output schemas, or runtime dependencies. A rollback target is valid only when the entire compatible serving stack can be restored.

Release progressively: offline validation, shadow traffic, a small canary, then larger traffic stages. At every stage compare quantized and control metrics for equivalent requests. Define automatic stop and rollback thresholds for errors, NaNs, OOMs, tail latency, fallback volume, task quality proxies, safety signals, and confidence drift. Use minimum sample sizes and consecutive bad windows so one noisy observation does not cause oscillation, but retain immediate rollback for catastrophic invariants such as corrupted output or security failure.

Make rollback idempotent and independent of the failing model path. The control plane must remain usable when the candidate exhausts GPU memory or crashes its worker. Drain or cancel in-flight requests deliberately, restore healthy capacity, verify readiness with golden probes, and only then redirect traffic. Record who or what triggered rollback, the evidence, artifact identities, timeline, and post-rollback validation. Regular game-day exercises are required: an untested rollback is a hopeful document, not a recovery mechanism.

Part 2 - Mathematical form and reasoning

Let m index a monitored metric, b_m the control baseline, and $c_m(t)$ the candidate value in window t. Define a signed degradation so that positive is always worse:

$$d_m(t) = \sigma_m \frac{c_m(t) - b_m}{\max(|b_m|, \epsilon)},$$

where $\sigma_m = 1$ for metrics such as latency/error and $\sigma_m = -1$ for metrics such as accuracy. A staged release may advance only if

$$n(t) \geq n_{\min} \quad \wedge \quad d_m(t) \leq \tau_m \text{ for every required } m.$$

Rollback is triggered when a catastrophic invariant fails once, or when an ordinary threshold is violated for K consecutive valid windows:

$$R(t) = I_{\text{catastrophic}}(t) \ \vee \ \bigvee_m \left[\sum_{j=0}^{K-1} I(d_m(t-j) > \tau_m) = K \right].$$

The thresholds, direction, sample requirement, and missing-data behavior must be explicit. Treat missing telemetry as a deployment risk rather than silently assuming health.

Part 3 - Hints and Helper

1. Package model, tokenizer, runtime, configuration, and schema into immutable release manifests.
2. Verify that the previous version can still load on current hardware before starting the canary.
3. Separate catastrophic one-window gates from noisy gates requiring repeated violations.
4. Use shadow or paired traffic where privacy and cost constraints permit direct comparison.
5. Keep rollback control outside the candidate worker and reserve enough capacity for recovery.
6. Define handling for in-flight requests, KV caches, streaming clients, and stateful sessions.
7. Run golden readiness probes after restoration before marking the incident resolved.
8. Rehearse rollback and measure detection time, decision time, traffic-switch time, and recovery time.

Part 4 - Python code skeleton

```
from dataclasses import dataclass, field
from enum import Enum
from typing import Dict, List


class Phase(Enum):
    IDLE = "idle"
    CANARY = "canary"
    EXPANDING = "expanding"
    ROLLING_BACK = "rolling_back"
    STABLE = "stable"


@dataclass
class MetricRule:
    name: str
    baseline: float
    maximum_relative_degradation: float
    higher_is_better: bool
    consecutive_windows: int
    catastrophic: bool = False


@dataclass
class ReleaseManifest:
    release_id: str
    model_digest: str
    tokenizer_digest: str
```

```
    runtime_digest: str
    api_schema_version: str


@dataclass
class RolloutState:
    candidate: ReleaseManifest
    fallback: ReleaseManifest
    phase: Phase = Phase.CANARY
    bad_counts: Dict[str, int] = field(default_factory=dict)
    reasons: List[str] = field(default_factory=list)


def relative_degradation(rule, candidate_value):
    denominator = max(abs(rule.baseline), 1.0e-12)
    relative_change = (candidate_value - rule.baseline) / denominator
    if rule.higher_is_better:
        return -relative_change
    return relative_change


def evaluate_window(state, rules, metrics, sample_count, minimum_samples):
    if sample_count < minimum_samples:
        return "hold:insufficient_samples"

    must_rollback = False
    for rule in rules:
        if rule.name not in metrics:
            state.reasons.append("missing_metric:" + rule.name)
            must_rollback = True
            continue

        degradation = relative_degradation(rule, metrics[rule.name])
        violated = degradation > rule.maximum_relative_degradation
        if violated:
            old_count = state.bad_counts.get(rule.name, 0)
            state.bad_counts[rule.name] = old_count + 1
        else:
            state.bad_counts[rule.name] = 0

        enough_bad_windows = (
            state.bad_counts[rule.name] >= rule.consecutive_windows
        )
        if violated and rule.catastrophic:
            must_rollback = True
            state.reasons.append("catastrophic:" + rule.name)
        elif enough_bad_windows:
            must_rollback = True
```

```
            state.reasons.append("threshold:" + rule.name)

    if must_rollback:
        state.phase = Phase.ROLLING_BACK
        return "rollback"
    return "continue_canary"


def execute_rollback(control_plane, state):
    control_plane.stop_new_candidate_admissions()
    control_plane.activate_release(state.fallback.release_id)
    control_plane.wait_until_ready(state.fallback.release_id)
    probes = control_plane.run_golden_probes(state.fallback.release_id)
    if not probes["all_passed"]:
        raise RuntimeError("fallback failed readiness probes")
    control_plane.route_all_traffic(state.fallback.release_id)
    state.phase = Phase.STABLE
```

Part 5 - The Philosophy Behind the Exercise

Rollback is the engineering expression of intellectual humility. Offline evidence can justify a deployment, but it cannot enumerate every future input or systems interaction. A reversible decision admits this limitation without abandoning progress.

The deeper goal is not to avoid all failures; it is to prevent a surprising failure from becoming a prolonged one. Immutable artifacts, explicit gates, independent control paths, and rehearsed recovery convert uncertainty from an unbounded threat into a manageable operational condition.

18.8.19 Exercise 18

Exercise Title: Document Quantization Tradeoffs for Stakeholders

Part 1 - Why this matters

A technically accurate benchmark can still produce a poor decision if it is communicated without the assumptions, risks, or consequences that matter to its audience. Researchers need to understand where error entered the model; platform engineers need hardware and runtime constraints; product owners need user-visible quality and latency; finance teams need capacity and cost; safety owners need failure-mode evidence. This exercise converts the measurements from the chapter into a decision document without flattening them into a slogan such as “int4 is four times smaller.”

Start with the decision to be made, not the technology to be celebrated. State the baseline, candidate variants, workload, hardware, software versions, calibration data, evaluation coverage, and confidence intervals. Separate artifact size from resident memory, isolated latency from concurrent tail latency, generic quality scores from safety-critical slices, and theoretical bit width from realized kernel speed. Distinguish measured facts, estimates, assumptions, and unknowns visually or structurally.

Present every candidate as a bundle of consequences. For example, int8 may offer broad kernel support and small quality loss; int4 may permit a larger model on the same GPU but require group metadata, special kernels, and more selective high-precision exceptions. Mixed precision may produce the best quality-capacity compromise while increasing configuration and testing burden. Include fallback and rollback costs because operational complexity is part of the tradeoff, not an unrelated implementation detail.

Avoid a universal weighted score unless the organization has explicitly agreed to the weights. Show the Pareto frontier and hard constraints first. Recommend a variant only for a named deployment scenario and state what would invalidate the recommendation: a new GPU, longer context, different traffic shape, changed quality threshold, or emerging production drift. The document should allow a reader to disagree intelligently because the evidence and value judgments are kept separate.

Part 2 - Mathematical form and reasoning

Represent candidate j by a metric vector

$$\mathbf{m}_j = (Q_j, L_{99,j}, M_j, C_j, E_j, R_j),$$

where Q is quality, L_{99} tail latency, M resident memory, C cost per useful token, E energy per useful token, and R an explicitly defined risk measure. Candidate a dominates b only if it is no worse on all required dimensions and strictly better on at least one. Non-dominated candidates form the Pareto set; stakeholders then select among real tradeoffs rather than a fabricated single winner.

A scenario-specific feasibility set can be written as

$$\mathcal{F} = \{j : Q_j \geq Q_{\min},\ L_{99,j} \leq L_{\max},\ M_j \leq M_{\max},\ R_j \leq R_{\max}\}.$$

Only after hard gates are satisfied should cost optimization occur:

$$j^* = \arg\min_{j \in \mathcal{F}} C_j.$$

Report absolute values and deltas. A 30 percent latency reduction is ambiguous without knowing whether the change was from 100 ms to 70 ms or from 10 s to 7 s. For uncertain measures, include an interval and sample count; false numerical precision undermines rather than strengthens the decision.

Part 3 - Hints and Helper

1. Open with the concrete decision, deployment scenario, and recommendation scope.
2. Create one evidence table containing baseline, int8, int4, and mixed-precision variants.
3. Label every value as measured, estimated, assumed, or not yet known.
4. Include quality slices, hallucination/safety results, latency percentiles, memory, energy, and operational risk.
5. State calibration and evaluation coverage so readers know which inputs remain untested.
6. Separate hard release gates from preferences and optimization goals.

7. Describe fallback, monitoring, rollback, staffing, and maintenance consequences.
8. End with a decision, owner, review date, and explicit triggers for re-evaluation.

Part 4 - Python code skeleton

```
from dataclasses import dataclass
from typing import Dict, List


@dataclass
class Variant:
    name: str
    quality: float
    p99_latency_ms: float
    resident_memory_gb: float
    cost_per_million_tokens: float
    joules_per_token: float
    risk_score: float
    evidence_status: Dict[str, str]


@dataclass
class Constraints:
    minimum_quality: float
    maximum_p99_latency_ms: float
    maximum_memory_gb: float
    maximum_risk_score: float


def is_feasible(variant, constraints):
    if variant.quality < constraints.minimum_quality:
        return False
    if variant.p99_latency_ms > constraints.maximum_p99_latency_ms:
        return False
    if variant.resident_memory_gb > constraints.maximum_memory_gb:
        return False
    if variant.risk_score > constraints.maximum_risk_score:
        return False
    return True


def dominates(left, right):
    no_worse = (
        left.quality >= right.quality
        and left.p99_latency_ms <= right.p99_latency_ms
        and left.resident_memory_gb <= right.resident_memory_gb
        and left.cost_per_million_tokens <= right.cost_per_million_tokens
```

```
        and left.joules_per_token <= right.joules_per_token
        and left.risk_score <= right.risk_score
    )
    strictly_better = (
        left.quality > right.quality
        or left.p99_latency_ms < right.p99_latency_ms
        or left.resident_memory_gb < right.resident_memory_gb
        or left.cost_per_million_tokens < right.cost_per_million_tokens
        or left.joules_per_token < right.joules_per_token
        or left.risk_score < right.risk_score
    )
    return no_worse and strictly_better


def pareto_frontier(variants):
    frontier = []
    for candidate in variants:
        candidate_is_dominated = False
        for challenger in variants:
            if candidate.name == challenger.name:
                continue
            if dominates(challenger, candidate):
                candidate_is_dominated = True
                break
        if not candidate_is_dominated:
            frontier.append(candidate)
    return frontier


def build_decision_summary(variants, constraints):
    feasible = []
    for variant in variants:
        if is_feasible(variant, constraints):
            feasible.append(variant)

    frontier = pareto_frontier(feasible)
    recommendation = None
    for variant in frontier:
        if recommendation is None:
            recommendation = variant
        elif variant.cost_per_million_tokens < (
            recommendation.cost_per_million_tokens
        ):
            recommendation = variant

    return {
        "feasible_variants": [item.name for item in feasible],
        "pareto_variants": [item.name for item in frontier],
```

```
        "recommended_variant": (
            recommendation.name if recommendation is not None else None
        ),
        "required_disclosure": [
            "workload and hardware",
            "quality and safety coverage",
            "unknowns and confidence intervals",
            "fallback and rollback plan",
            "re-evaluation triggers",
        ],
    }
```

Part 5 - The Philosophy Behind the Exercise

Engineering communication is not simplification at any cost. Its purpose is to translate evidence across different responsibilities while preserving the uncertainty and conditions that give the evidence meaning. A stakeholder does not need every kernel trace, but they do need to know when the recommendation depends on a particular kernel.

Quantization has no context-free winner. It redistributes memory, computation, quality, energy, compatibility, and operational complexity. The honest document makes those redistributions visible, identifies whose objectives define the decision, and leaves a trail that future evidence can revise.

18.8.20 Exercise 19

Exercise Title: Produce a Full Quantization and Deployment Report

Part 1 - Why this matters

The final report is the durable engineering record of what was built, why it was accepted, where it was tested, how it can fail, and how it will be operated. Without that record, benchmark numbers become detached from their artifacts, deployment assumptions disappear, and later regressions cannot be distinguished from workload changes. This exercise integrates the entire chapter into an auditable release argument rather than a celebratory summary.

Begin with immutable identity: source model and tokenizer digests, quantization recipe, calibration dataset version, per-layer precision policy, framework, backend, compiler flags, runtime image, driver, hardware, and generated artifact hash. Document transformation details including granularity, scale/zero-point rules, group size, clipping, outlier handling, excluded layers, accumulation precision, and whether activations and KV cache are quantized. The report must make it possible to reproduce the candidate rather than merely name it.

Connect every release claim to evidence. Include numerical parity, task quality, calibration, hallucination and safety slices, artifact and resident memory, TTFT, ITL, tail latency under concurrency, throughput, energy, edge-device limits, hardware compatibility, and ONNX/runtime behavior. Record datasets, sample counts, seeds, prompts, decoding configuration, warm-up, repetitions, confidence intervals, failures, and raw-result locations. Negative

experiments and rejected configurations belong in the report because they define the safe boundary and prevent repeated mistakes.

Complete the operational argument: service integration, input limits, fallback conditions, canary stages, monitoring, alert thresholds, rollback target, recovery rehearsal, ownership, security/privacy review, known limitations, and expiration or revalidation date. End with a signed release decision that lists the gates passed, exceptions accepted, residual risks, and the conditions that would force rollback or renewed evaluation. A report is complete only when a new engineer can use it to reproduce, deploy, observe, and safely reverse the system.

Part 2 - Mathematical form and reasoning

Model release readiness as a conjunction of evidence-backed gates rather than an average score:

$$G_{\text{release}} = G_{\text{identity}} \wedge G_{\text{parity}} \wedge G_{\text{quality}} \wedge G_{\text{performance}} \wedge G_{\text{compatibility}} \wedge G_{\text{operations}} \wedge G_{\text{rollback}}.$$

Each gate G_k must contain a claim, threshold, observed value, evidence reference, owner, and status. If a required value is unknown, the gate is not passed. A useful traceability relation is

$$\text{requirement} \rightarrow \text{experiment} \rightarrow \text{raw evidence} \rightarrow \text{decision} \rightarrow \text{production monitor}.$$

For a measured improvement, report both delta and uncertainty. If X_q and X_f are quantized and float results across matched runs, define paired differences $D_i = X_{q,i} - X_{f,i}$ and report

$$\bar{D} = \frac{1}{n}\sum_{i=1}^{n} D_i, \qquad SE(\bar{D}) = \frac{s_D}{\sqrt{n}}.$$

Do not combine incomparable evidence into one opaque readiness percentage. One failed safety, compatibility, or rollback gate can invalidate deployment even when all performance metrics are excellent.

Part 3 - Hints and Helper

1. Use a fixed report schema so every future quantized release answers the same critical questions.
2. Hash all artifacts and record exact software, driver, backend, and hardware versions.
3. Link claims to raw results, scripts, configurations, prompts, and dataset versions.
4. Include per-layer precision and exception lists, not just the headline model bit width.
5. Report rejected candidates, injected failures, and unresolved unknowns alongside successful results.
6. Make release gates machine-readable and fail closed when required evidence is absent.
7. Include monitoring thresholds, fallback logic, rollback rehearsal evidence, owners, and review dates.

8. Ask an independent reviewer to reproduce a subset and challenge the recommendation before sign-off.

Part 4 - Python code skeleton

```
from dataclasses import asdict, dataclass, field
from datetime import datetime, timezone
from pathlib import Path
from typing import Dict, List
import hashlib
import json


@dataclass
class GateEvidence:
    name: str
    requirement: str
    threshold: str
    observed: str
    evidence_uri: str
    owner: str
    passed: bool
    exception: str = ""


@dataclass
class ReleaseReport:
    report_version: str
    release_id: str
    created_at: str
    artifact_hashes: Dict[str, str]
    quantization_recipe: Dict[str, object]
    environment: Dict[str, str]
    gates: List[GateEvidence] = field(default_factory=list)
    rejected_variants: List[Dict[str, str]] = field(default_factory=list)
    known_limits: List[str] = field(default_factory=list)
    rollback_release_id: str = ""
    decision: str = "pending"


def sha256_file(path):
    digest = hashlib.sha256()
    with open(path, "rb") as handle:
        while True:
            block = handle.read(1024 * 1024)
            if not block:
                break
            digest.update(block)
```

```
    return digest.hexdigest()


def validate_report(report, required_gate_names):
    errors = []
    gate_by_name = {}
    for gate in report.gates:
        gate_by_name[gate.name] = gate

    for gate_name in required_gate_names:
        if gate_name not in gate_by_name:
            errors.append("missing_gate:" + gate_name)
            continue
        gate = gate_by_name[gate_name]
        if gate.evidence_uri.strip() == "":
            errors.append("missing_evidence:" + gate_name)
        if not gate.passed and gate.exception.strip() == "":
            errors.append("failed_without_exception:" + gate_name)

    if report.rollback_release_id.strip() == "":
        errors.append("missing_rollback_release")
    if len(report.artifact_hashes) == 0:
        errors.append("missing_artifact_hashes")
    return errors


def finalize_report(report, required_gate_names, output_path):
    errors = validate_report(report, required_gate_names)
    if len(errors) == 0:
        report.decision = "approved"
    else:
        report.decision = "blocked"

    payload = asdict(report)
    payload["validation_errors"] = errors
    with open(output_path, "w", encoding="utf-8") as handle:
        json.dump(payload, handle, indent=2, sort_keys=True)
    return errors


def new_report(release_id, recipe, environment):
    created_at = datetime.now(timezone.utc).isoformat()
    return ReleaseReport(
        report_version="1.0",
        release_id=release_id,
        created_at=created_at,
        artifact_hashes={},
        quantization_recipe=recipe,
```

```
        environment=environment,
    )


# Populate gates from verified experiment outputs. Never mark a gate passed
# because a file merely exists; validate its contents and artifact identity.
```

Part 5 - The Philosophy Behind the Exercise

A rigorous report is institutional memory with enough structure to be tested. It preserves not only the conclusion but the chain of reasoning that made the conclusion responsible at a particular time. This matters because deployed models, runtimes, hardware, traffic, and social expectations continue to change after the benchmark ends.

The final exercise treats deployment as a scientific claim under operational conditions. Reproducibility protects the claim, monitoring keeps it under observation, and rollback makes it reversible. Quantization then becomes more than compression: it becomes a disciplined negotiation between mathematical approximation, physical resources, user-visible behavior, and accountable engineering judgment.

18.9 LLM07

Chapter Title: Evaluation, Hallucination, and Reliability
Specialization: Applied AI and Reliability Engineering

18.9.1 Exercise 00

Exercise Title: Define Hallucination and Explain Why It Occurs in LLMs

Part 1 - Why this matters

An LLM is trained primarily to assign probability to the next token, not to consult reality before every sentence. A fluent answer can therefore be grammatically excellent while containing a fabricated date, citation, person, causal explanation, or software capability. Call an output a hallucination when it presents content as supported by the task's available evidence even though that content is false, unverifiable, contradicted by the evidence, or invented beyond what the prompt permits. This definition must be adapted to the task: a fictional story is not a hallucination merely because it is imaginary, while an invented quotation in a factual report is.

Separate several failure types. Parametric factual error conflicts with the world knowledge expected of the model. Contextual hallucination conflicts with the supplied document. Attribution failure attaches a real claim to the wrong source. Temporal failure answers with stale knowledge as if it were current. Instructional hallucination claims an operation was performed when no tool or evidence confirms it. These categories lead to different mitigations and must not be collapsed into one vague label.

Explain the causes at multiple levels: incomplete or conflicting training data, the lossy compression of knowledge into parameters, token-level objectives, distribution shift, ambiguous prompts, long-context retrieval failure, decoding randomness, exposure to generated prefixes,

and pressure to continue instead of abstain. The model's probability is linguistic probability under its learned distribution; it is not automatically the probability that a claim is true. Reliability work begins by preserving this distinction.

Part 2 - Mathematical form and reasoning

Autoregressive generation factorizes a sequence as

$$p_\theta(y_{1:T} \mid x) = \prod_{t=1}^{T} p_\theta(y_t \mid x, y_{<t}).$$

Maximum-likelihood training minimizes token negative log-likelihood,

$$\mathcal{L}_{\text{NLL}} = -\sum_{t=1}^{T} \log p_\theta(y_t^* \mid x, y_{<t}^*),$$

but the objective does not directly contain a truth predicate for every claim. Define a claim extractor $C(y) = \{c_1, \ldots, c_m\}$ and an evidence-supported label $s(c_j, e) \in \{0, 1\}$. An example-level hallucination score can be

$$H(y, e) = \frac{1}{m} \sum_{j=1}^{m} \left(1 - s(c_j, e)\right).$$

The denominator and treatment of unverifiable claims must be stated. Claim-level and answer-level rates answer different questions and should both be retained.

Part 3 - Hints and Helper

1. Write an operational definition before reading model outputs.
2. Distinguish false, unsupported, contradicted, stale, and unverifiable claims.
3. Label whether the task is factual, creative, predictive, or evidence-bounded.
4. Split an answer into atomic claims so one wrong detail does not erase correct content.
5. Preserve the source and timestamp used to judge each claim.
6. Record ambiguity and evaluator disagreement rather than forcing false certainty.
7. Demonstrate one failure from each major causal category.
8. State which mitigations change knowledge, evidence access, decoding, or abstention.

Part 4 - Python code skeleton

```
from dataclasses import dataclass
from enum import Enum


class ClaimStatus(Enum):
    SUPPORTED = "supported"
    CONTRADICTED = "contradicted"
```

```
    UNSUPPORTED = "unsupported"
    UNVERIFIABLE = "unverifiable"


@dataclass
class ClaimJudgment:
    claim: str
    status: ClaimStatus
    evidence_id: str
    explanation: str


def claim_hallucination_rate(judgments, count_unverifiable=True):
    if len(judgments) == 0:
        return 0.0
    failures = 0
    for judgment in judgments:
        if judgment.status == ClaimStatus.CONTRADICTED:
            failures += 1
        elif judgment.status == ClaimStatus.UNSUPPORTED:
            failures += 1
        elif (
            count_unverifiable
            and judgment.status == ClaimStatus.UNVERIFIABLE
        ):
            failures += 1
    return failures / len(judgments)


def answer_has_hallucination(judgments):
    for judgment in judgments:
        if judgment.status in {
            ClaimStatus.CONTRADICTED,
            ClaimStatus.UNSUPPORTED,
        }:
            return True
    return False
```

Part 5 - The Philosophy Behind the Exercise

Fluency creates an illusion of contact with truth because humans normally use confident language after forming beliefs. An LLM can reproduce the language of belief without possessing the same evidential relationship. Reliability work therefore studies not only what the model says, but what licenses it to say it.

A precise definition protects both the model and the evaluator. It avoids calling legitimate creativity a defect, while refusing to excuse unsupported factual authority as mere creativity. The philosophy is simple: confidence is not evidence, and coherence is not verification.

18.9.2 Exercise 01

Exercise Title: Build Evaluation Set That Triggers Hallucinations

Part 1 - Why this matters

A random list of easy questions estimates ordinary accuracy but rarely exposes how hallucinations arise. Build a diagnostic set that deliberately covers known pressure points while remaining fair and reproducible. Include false premises, nonexistent entities, obscure but answerable facts, stale temporal questions, conflicting context, unanswerable questions, attribution requests, multi-hop reasoning, long-context distractors, and prompts that tempt the model to invent citations or claim tool use.

Each item needs more than a prompt. Store the task type, answerability, trusted evidence, valid answers, forbidden claims, expected abstention behavior, difficulty, knowledge date, domain, and risk level. For evidence-bounded items, the reference is the supplied context rather than the model's parametric memory. Include matched controls: if a fabricated entity triggers invention, pair it with a real entity of similar wording so the test measures epistemic behavior instead of unfamiliar spelling alone.

Prevent contamination and evaluator leakage. Freeze a hidden test split, version the source material, remove duplicates and near-duplicates, and avoid repeatedly tuning prompts on the final set. Human reviewers should pilot items for ambiguity. A trigger set should reveal failure modes, not manufacture them through an incorrect reference answer.

Part 2 - Mathematical form and reasoning

Let strata $k = 1, \ldots, K$ represent trigger families. A macro hallucination estimate weights families equally:

$$\widehat{H}_{\text{macro}} = \frac{1}{K} \sum_{k=1}^{K} \frac{1}{n_k} \sum_{i=1}^{n_k} h_{ki}.$$

A deployment-weighted estimate instead uses expected traffic weights w_k:

$$\widehat{H}_{\text{traffic}} = \sum_{k=1}^{K} w_k \widehat{H}_k, \qquad \sum_k w_k = 1.$$

Report both. Macro scoring prevents a common category from hiding a catastrophic rare failure, while traffic weighting estimates expected operational exposure. Coverage can be represented by a matrix of domain, trigger, answerability, and risk; empty cells are known blind spots.

Part 3 - Hints and Helper

1. Define a taxonomy and minimum item count for every trigger family.
2. Pair answerable and unanswerable prompts with similar surface form.
3. Keep evidence and grading rules separate from the prompt shown to the model.
4. Add temporal metadata so changing facts can be refreshed or retired.
5. Include high-risk domains only with qualified reference construction.

6. Deduplicate by normalized text and semantic similarity.
7. Pilot ambiguity with at least two independent reviewers.
8. Freeze a hidden final split and version every later amendment.

Part 4 - Python code skeleton

```
from dataclasses import asdict, dataclass
from typing import List
import json


@dataclass
class EvaluationItem:
    item_id: str
    prompt: str
    trigger_family: str
    domain: str
    answerable: bool
    evidence: List[str]
    accepted_answers: List[str]
    forbidden_claims: List[str]
    expected_behavior: str
    risk_level: str
    knowledge_date: str


def validate_item(item):
    errors = []
    if item.item_id.strip() == "":
        errors.append("missing_id")
    if item.prompt.strip() == "":
        errors.append("missing_prompt")
    if item.answerable and len(item.evidence) == 0:
        errors.append("answerable_without_evidence")
    if not item.answerable and item.expected_behavior != "abstain":
        errors.append("unanswerable_without_abstention_target")
    return errors


def save_validated_set(items, output_path):
    seen = set()
    records = []
    for item in items:
        if item.item_id in seen:
            raise ValueError("duplicate item id: " + item.item_id)
        seen.add(item.item_id)
```

```
        errors = validate_item(item)
        if len(errors) > 0:
            raise ValueError(item.item_id + ":" + ",".join(errors))
        records.append(asdict(item))
    with open(output_path, "w", encoding="utf-8") as handle:
        json.dump(records, handle, indent=2, ensure_ascii=False)
```

Part 5 - The Philosophy Behind the Exercise

An evaluation set is an instrument, and instruments embody theories about what can go wrong. Easy factual questions reward stored knowledge; false-premise and unanswerable questions test whether the model recognizes the boundary of that knowledge. The second ability is often more important in deployment.

Deliberate stress is not unfairness when the scenario is plausible and the reference is sound. It is a way of making epistemic limits visible before users discover them under higher stakes.

18.9.3 Exercise 02

Exercise Title: Measure Hallucination Rate and Confidence Mismatch

Part 1 - Why this matters

Reliability requires two measurements: whether an answer is wrong and whether the model behaves as if it might be wrong. A system that makes occasional errors with calibrated uncertainty can abstain or request review. A system that assigns its strongest confidence to false answers is dangerous even when its average accuracy appears acceptable.

Choose the unit of analysis explicitly. Answer-level hallucination marks an entire response if any material claim fails. Claim-level rate counts each atomic assertion. Citation precision asks whether cited evidence supports the claim. Unsupported-claim rate includes statements not justified by supplied context. Measure these separately and stratify by trigger, domain, risk, answer length, and answerability.

Confidence is also a design choice. It may come from normalized token probabilities, a separate verbal probability, self-evaluation, agreement across samples, or an external verifier. These signals are not interchangeable. Normalize length effects, freeze decoding, and define bins before inspecting results. Save per-example records so aggregate rates can be audited.

Part 2 - Mathematical form and reasoning

For binary correctness $y_i \in \{0, 1\}$ and confidence $p_i \in [0, 1]$, the Brier score is

$$\mathrm{BS} = \frac{1}{N} \sum_{i=1}^{N} (p_i - y_i)^2.$$

Define hallucination $h_i = 1 - y_i$ for tasks with a complete binary judgment. The rate and a normal-approximation standard error are

$$\widehat{H} = \frac{1}{N} \sum_{i} h_i, \qquad SE(\widehat{H}) = \sqrt{\frac{\widehat{H}(1 - \widehat{H})}{N}}.$$

Confidence mismatch can be summarized by signed calibration bias

$$\text{Bias} = \frac{1}{N}\sum_{i}(p_i - y_i),$$

where a positive value indicates overconfidence. Reliability diagrams and ECE will be developed later; retain raw pairs now.

Part 3 - Hints and Helper

1. Keep claim-level and answer-level labels in the same record.
2. Define confidence extraction before examining model correctness.
3. Treat refusals, empty outputs, and unjudgeable cases as separate outcomes.
4. Bootstrap examples, not individual claims from the same answer, for uncertainty.
5. Report sample counts and confidence intervals for every slice.
6. Inspect high-confidence errors individually.
7. Compare confidence on answerable and unanswerable prompts.
8. Never infer factual confidence from fluent wording alone.

Part 4 - Python code skeleton

```
from dataclasses import dataclass
import math


@dataclass
class ScoredAnswer:
    item_id: str
    correct: bool
    confidence: float
    claim_count: int
    failed_claim_count: int
    trigger_family: str


def summarize_answers(records):
    if len(records) == 0:
        raise ValueError("records must not be empty")
    hallucinations = 0
    squared_error = 0.0
    signed_bias = 0.0
    failed_claims = 0
    total_claims = 0
    for record in records:
```

```
        if not 0.0 <= record.confidence <= 1.0:
            raise ValueError("confidence outside [0, 1]")
        target = 1.0 if record.correct else 0.0
        if not record.correct:
            hallucinations += 1
        difference = record.confidence - target
        squared_error += difference * difference
        signed_bias += difference
        failed_claims += record.failed_claim_count
        total_claims += record.claim_count

    count = len(records)
    rate = hallucinations / count
    standard_error = math.sqrt(rate * (1.0 - rate) / count)
    return {
        "answer_hallucination_rate": rate,
        "rate_standard_error": standard_error,
        "claim_hallucination_rate": failed_claims / total_claims,
        "brier_score": squared_error / count,
        "confidence_bias": signed_bias / count,
    }
```

Part 5 - The Philosophy Behind the Exercise

Error and uncertainty are morally different dimensions of a system. No finite model can eliminate all error, but it can be designed to expose uncertainty at the boundary of its competence. Measuring only correctness ignores this possibility; measuring only confidence mistakes self-description for truth.

The paired record asks the model to be accountable not only for answers, but for the relationship between certainty and evidence. That relationship is the foundation of selective prediction and safe escalation.

18.9.4 Exercise 03

Exercise Title: Compare Factual vs Creative Prompt Behavior

Part 1 - Why this matters

The same linguistic novelty can be valuable in a story and harmful in a medical summary. Reliability evaluation must therefore respect the intent of the prompt. Construct matched factual and creative prompts about similar subjects. The factual prompt should require evidence, attribution, and abstention when information is unavailable. The creative prompt should permit invention but still obey constraints such as named characters, tone, length, and explicit separation from real claims.

Do not compare the two modes with one metric. Factual responses need claim support, citation precision, contradiction rate, completeness, and calibrated abstention. Creative responses need constraint adherence, coherence, diversity, originality, and human preference. A creative response can fail by presenting fiction as biography; a factual response can fail by being

cautious but useless. Record how the system prompt and user wording signal the mode.

Run a crossover experiment: factual instruction with high temperature, creative instruction with low temperature, and both instructions under several decoding settings. This separates prompt intent from sampling behavior. The result should show that reliability is a contract between task, evidence, and decoding rather than an inherent property of a sentence.

Part 2 - Mathematical form and reasoning

Let $m \in \{F, C\}$ be factual or creative mode. Define different utility functions:

$$U_F = \alpha S + \beta A - \gamma H - \delta O,$$

where S is evidence support, A factual answer coverage, H hallucination, and O overconfidence. For creative work,

$$U_C = \eta K + \rho D + \mu I - \nu V,$$

where K is coherence, D diversity, I instruction adherence, and V violation of factual-boundary constraints. Scores are meaningful only within the declared mode; comparing U_F and U_C directly is invalid.

Part 3 - Hints and Helper

1. Build matched prompt pairs that differ mainly in declared intent.
2. Tell evaluators whether invention is allowed before they score output.
3. Use separate rubrics and keep a shared instruction-adherence measure.
4. Test mode cues in system and user messages independently.
5. Cross prompt mode with temperature and sampling strategy.
6. Flag creative claims that impersonate real citations or biography.
7. Include refusal usefulness in factual scoring.
8. Save outputs for blinded human comparison.

Part 4 - Python code skeleton

```
from dataclasses import dataclass


@dataclass
class ModeScore:
    prompt_id: str
    mode: str
    support: float
    hallucination: float
    coherence: float
```

```
    diversity: float
    instruction_adherence: float
    factual_boundary_violation: float


def factual_utility(score):
    return (
        0.35 * score.support
        + 0.25 * score.instruction_adherence
        + 0.20 * score.coherence
        - 0.15 * score.hallucination
        - 0.05 * score.factual_boundary_violation
    )


def creative_utility(score):
    return (
        0.30 * score.coherence
        + 0.30 * score.diversity
        + 0.30 * score.instruction_adherence
        - 0.10 * score.factual_boundary_violation
    )


def evaluate_by_declared_mode(scores):
    results = []
    for score in scores:
        if score.mode == "factual":
            utility = factual_utility(score)
        elif score.mode == "creative":
            utility = creative_utility(score)
        else:
            raise ValueError("unknown mode: " + score.mode)
        results.append((score.prompt_id, score.mode, utility))
    return results
```

Part 5 - The Philosophy Behind the Exercise

Truthfulness is not the elimination of imagination. It is the disciplined placement of imagination inside a contract that permits it. Human language contains science, law, poetry, speculation, and play; a reliable model must recognize which norm governs the current act.

This exercise replaces the simplistic command “never invent” with a richer principle: invent freely where invention is invited, and mark the boundary where invented language could be mistaken for evidence.

18.9.5 Exercise 04

Exercise Title: Implement Retrieval-Augmented Prompting and Compare Hallucination Reduction

Part 1 - Why this matters

Retrieval-augmented generation (RAG) supplies external evidence at inference time instead of requiring every fact to be stored reliably in parameters. It can reduce unsupported answers, update knowledge without retraining, and expose citations. It can also introduce new failures: the relevant passage may not be retrieved, an irrelevant passage may distract the model, the prompt may exceed context limits, or the generator may ignore or distort correct evidence.

Build a controlled comparison with three conditions: no retrieval, oracle retrieval containing the correct evidence, and realistic retrieval from an indexed corpus. The oracle separates generation faithfulness from retrieval recall. Freeze prompts, decoding, model version, and judge. Measure retrieval recall at k, context precision, answer accuracy, unsupported-claim rate, citation precision, abstention, latency, and token cost.

Require the model to answer only from provided evidence and to abstain when the evidence is insufficient. Include adversarial distractors and conflicting documents with timestamps or authority metadata. A reduction in hallucination is meaningful only if it is not purchased by refusing every difficult question or copying irrelevant passages.

Part 2 - Mathematical form and reasoning

For query x, a retriever selects documents

$$D_k(x) = \mathrm{TopK}_{d \in \mathcal{D}}\, s(x, d),$$

and generation becomes $p_\theta(y \mid x, D_k(x))$. Decompose factual success:

$$\begin{aligned} P(\text{correct}) = {} & P(\text{evidence retrieved}) \\ & \times P(\text{correct} \mid \text{evidence retrieved}) \\ & + P(\text{correct without retrieved evidence}). \end{aligned}$$

This decomposition prevents blaming the generator for retriever failure. The paired hallucination reduction is

$$\Delta H = \widehat{H}_{\mathrm{RAG}} - \widehat{H}_{\mathrm{base}},$$

with negative values indicating improvement. Report it with retrieval coverage and refusal rate.

Part 3 - Hints and Helper

1. Version the corpus, chunking, embedding model, index, and retrieval parameters.
2. Evaluate oracle retrieval before debugging the realistic retriever.
3. Keep model and decoding identical across comparison conditions.

4. Test no-answer queries and require evidence-aware abstention.
5. Measure whether every citation actually entails its attached claim.
6. Add plausible distractors and temporally conflicting documents.
7. Log retrieved document IDs and scores for every output.
8. Report accuracy, hallucination, refusal, latency, and token cost together.

Part 4 - Python code skeleton

```
from dataclasses import dataclass


@dataclass
class RetrievedDocument:
    document_id: str
    text: str
    score: float
    source_date: str


def build_grounded_prompt(question, documents):
    lines = []
    lines.append("Answer only from the evidence below.")
    lines.append("If evidence is insufficient, say: I don't know.")
    for index, document in enumerate(documents):
        label = "[D" + str(index + 1) + "]"
        lines.append(label + " " + document.text)
    lines.append("Question: " + question)
    lines.append("Cite supporting document labels after each claim.")
    return "\n\n".join(lines)


def run_rag_item(model, retriever, item, top_k):
    documents = retriever.search(item.question, top_k=top_k)
    prompt = build_grounded_prompt(item.question, documents)
    answer = model.generate(prompt, temperature=0.0)
    document_ids = []
    for document in documents:
        document_ids.append(document.document_id)
    return {
        "item_id": item.item_id,
        "answer": answer,
        "retrieved_document_ids": document_ids,
    }
```

```
def compare_conditions(items, base_model, rag_model, retriever):
    records = []
    for item in items:
        base_answer = base_model.generate(item.question, temperature=0.0)
        rag_record = run_rag_item(rag_model, retriever, item, top_k=5)
        records.append({
            "item_id": item.item_id,
            "base_answer": base_answer,
            "rag_answer": rag_record["answer"],
            "documents": rag_record["retrieved_document_ids"],
        })
    return records
```

Part 5 - The Philosophy Behind the Exercise

RAG changes the epistemic structure of generation. The answer no longer needs to emerge solely from compressed memory; it can be justified by inspectable evidence. Yet access to evidence is not the same as faithful use of evidence. Retrieval, reading, reasoning, and citation remain distinct obligations.

The central lesson is traceability. A reliable answer should allow us to ask: which document was available, which claim it supported, and where the chain broke when the answer failed.

18.9.6 Exercise 05

Exercise Title: Evaluate Robustness Under Prompt Paraphrasing

Part 1 - Why this matters

Semantically equivalent prompts should not produce incompatible factual answers merely because word order, politeness, dialect, or formatting changed. Paraphrase sensitivity reveals that the model may be responding to surface cues rather than a stable interpretation. It also exposes evaluation fragility: a single canonical prompt can overestimate reliability for real users.

Create paraphrase families that preserve meaning while varying syntax, vocabulary, active/passive voice, question order, spelling noise, and location of constraints. Include a meaning-preservation review; a paraphrase that changes scope or temporal reference is a new task, not a robustness test. Run all variants with identical system message, evidence, decoding, and seed where possible.

Measure correctness variance, hallucination variance, refusal inconsistency, semantic answer agreement, and worst-case family performance. Inspect families where one wording triggers an unsupported answer while another correctly abstains. Robustness should not mean identical text; it means stable claims and decisions under meaning-preserving changes.

Part 2 - Mathematical form and reasoning

For item i with M_i paraphrases and correctness y_{ij}, define family accuracy and instability:

$$\bar{y}_i = \frac{1}{M_i} \sum_j y_{ij}, \qquad I_i = \max_j y_{ij} - \min_j y_{ij}.$$

For binary labels, $I_i = 1$ when at least one paraphrase changes correctness. A stricter robust accuracy is

$$A_{\text{worst}} = \frac{1}{N} \sum_i \min_j y_{ij}.$$

Pair it with semantic contradiction rate between answers, since two incorrect answers may agree and two correct answers may use different wording.

Part 3 - Hints and Helper

1. Generate several controlled paraphrase types for every base prompt.
2. Have a reviewer verify semantic equivalence before model evaluation.
3. Keep system prompt, evidence, tools, and decoding fixed.
4. Compare atomic claims rather than exact text.
5. Report average and worst-paraphrase accuracy.
6. Track refusal and confidence transitions within each family.
7. Inspect sensitivity to constraint placement and negation separately.
8. Save the exact prompt form with every result.

Part 4 - Python code skeleton

```
from collections import defaultdict
from dataclasses import dataclass


@dataclass
class ParaphraseResult:
    family_id: str
    variant_id: str
    correct: bool
    hallucinated: bool
    abstained: bool
    confidence: float


def summarize_families(results):
    grouped = defaultdict(list)
    for result in results:
        grouped[result.family_id].append(result)

    summaries = []
    robust_correct_count = 0
    for family_id, family in grouped.items():
```

```
        correct_values = []
        confidences = []
        for result in family:
            correct_values.append(int(result.correct))
            confidences.append(result.confidence)
        all_correct = min(correct_values) == 1
        if all_correct:
            robust_correct_count += 1
        summaries.append({
            "family_id": family_id,
            "mean_accuracy": sum(correct_values) / len(correct_values),
            "unstable": max(correct_values) != min(correct_values),
            "confidence_range": max(confidences) - min(confidences),
        })
    robust_accuracy = robust_correct_count / len(grouped)
    return summaries, robust_accuracy
```

Part 5 - The Philosophy Behind the Exercise

Meaning is richer than phrasing. A reliable language system should respond to the intended proposition, not treat each syntactic costume as a different world. Paraphrase testing therefore probes whether the model has learned a stable task or a collection of linguistic shortcuts.

Worst-case family performance also changes our idea of quality: the user should not need to discover the one magical wording that makes the system truthful.

18.9.7 Exercise 06

Exercise Title: Analyze Temperature Impact on Hallucination Frequency

Part 1 - Why this matters

Temperature changes the sharpness of the token distribution. Higher values usually increase diversity and the probability of low-ranked continuations; lower values make outputs more repeatable but do not guarantee truth. If the most probable continuation encodes a misconception, greedy decoding can repeat it confidently. This exercise measures rather than assumes the relationship.

Use a fixed factual evaluation set and sample several independent generations per prompt at each temperature. Keep top-p, top-k, maximum tokens, prompt, model, evidence, and evaluator constant. Record accuracy, claim hallucination, answer diversity, refusal, length, and confidence. Separate factual prompts from creative prompts because diversity has different utility in each mode.

Estimate uncertainty across prompts, not just across samples from one prompt. Look for nonlinear thresholds: hallucination may remain stable through moderate temperature and then increase rapidly, or retrieval grounding may weaken only when sampling becomes very diffuse. Recommend a temperature for a named task and risk tolerance rather than declaring a universal best value.

Part 2 - Mathematical form and reasoning

For logits z_i and temperature $T > 0$,

$$p_T(i) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}.$$

As $T \to 0$, mass concentrates near the largest logit; increasing T flattens the distribution. For prompt x with R samples, estimate

$$\widehat{H}_T(x) = \frac{1}{R}\sum_{r=1}^{R} h_{xTr}, \qquad \widehat{H}_T = \frac{1}{N}\sum_x \widehat{H}_T(x).$$

Use the prompt as the resampling unit when constructing confidence intervals, because samples from the same prompt share difficulty.

Part 3 - Hints and Helper

1. Sweep a predeclared temperature grid including a near-deterministic baseline.
2. Generate the same number of samples for every prompt and temperature.
3. Hold nucleus or top-k filtering fixed.
4. Preserve seeds and raw outputs for paired inspection.
5. Measure diversity together with reliability.
6. Stratify answerable, unanswerable, retrieval-grounded, and creative prompts.
7. Plot per-prompt trajectories to expose heterogeneous effects.
8. Select an operating point using task-specific quality constraints.

Part 4 - Python code skeleton

```
from dataclasses import dataclass


@dataclass
class TemperatureTrial:
    item_id: str
    temperature: float
    sample_id: int
    hallucinated: bool
    correct: bool
    answer: str


def run_temperature_sweep(model, items, temperatures, repeats):
    trials = []
    for temperature in temperatures:
```

```
        for item in items:
            for sample_id in range(repeats):
                answer = model.generate(
                    item.prompt,
                    temperature=temperature,
                    seed=sample_id,
                )
                trial = TemperatureTrial(
                    item_id=item.item_id,
                    temperature=temperature,
                    sample_id=sample_id,
                    hallucinated=False,
                    correct=False,
                    answer=answer,
                )
                trials.append(trial)
    return trials


def rate_by_temperature(scored_trials):
    totals = {}
    failures = {}
    for trial in scored_trials:
        totals[trial.temperature] = totals.get(trial.temperature, 0) + 1
        if trial.hallucinated:
            old = failures.get(trial.temperature, 0)
            failures[trial.temperature] = old + 1
    rates = {}
    for temperature, total in totals.items():
        failure_count = failures.get(temperature, 0)
        rates[temperature] = failure_count / total
    return rates
```

Part 5 - The Philosophy Behind the Exercise

Randomness is neither the cause of all hallucination nor a defect by itself. It is a mechanism for exploring alternatives in a learned distribution. When the distribution contains unsupported continuations, exploration reveals them; when the top mode is wrong, determinism merely conceals the alternatives.

The responsible question is not whether temperature is good or bad, but how much uncertainty the task can tolerate and what benefits diversity provides.

18.9.8 Exercise 07

Exercise Title: Implement Self-Consistency Sampling and Measure Accuracy Gains

Part 1 - Why this matters

Self-consistency generates several reasoning paths and chooses the answer that receives the most independent support. It can improve tasks where different valid paths converge on the same result, but it is not an oracle. Correlated samples may repeat the same misconception, majority voting may suppress a rare correct path, and free-form answers may require careful normalization before they can be compared.

Create a deterministic single-sample baseline and sampling conditions with a fixed budget. Extract a final answer separately from the reasoning text, normalize units and formatting without changing meaning, and vote only among comparable answers. Track vote margin, entropy, answer accuracy, hallucination, token cost, latency, and cases where consistency makes a correct baseline wrong.

Compare majority vote, confidence-weighted vote, and verifier-based selection. Use odd sample counts and plot marginal gain from additional samples. A reliable system should treat low agreement as uncertainty and consider abstention rather than forcing a majority from fragmented outputs.

Part 2 - Mathematical form and reasoning

Given extracted answers $a^{(1)}, \ldots, a^{(R)}$, majority selection is

$$\hat{a} = \arg\max_{a} \sum_{r=1}^{R} I(a^{(r)} = a).$$

If v_a is the vote fraction, agreement entropy is

$$H_{\text{vote}} = -\sum_{a} v_a \log(v_a + \epsilon).$$

Low entropy and a large margin $v_{(1)} - v_{(2)}$ indicate agreement, not truth. Measure paired accuracy change relative to the same items and report added tokens per corrected answer.

Part 3 - Hints and Helper

1. Separate answer extraction from answer generation.
2. Normalize formatting conservatively and retain the raw text.
3. Sweep odd sample counts under a recorded token budget.
4. Measure disagreement entropy and winning-vote margin.
5. Inspect correlated wrong majorities.
6. Compare majority, weighted, and external-verifier selection.
7. Allow abstention when agreement is below a threshold.
8. Report accuracy gain beside latency and token-cost growth.

Part 4 - Python code skeleton

```
from collections import Counter
from dataclasses import dataclass
import math


@dataclass
class ConsistencyResult:
    selected_answer: str
    vote_fraction: float
    vote_margin: float
    vote_entropy: float
    raw_answers: list


def select_by_majority(raw_answers, normalize_answer):
    normalized = []
    for answer in raw_answers:
        normalized.append(normalize_answer(answer))
    counts = Counter(normalized)
    ranked = counts.most_common()
    winner, winner_count = ranked[0]
    second_count = 0
    if len(ranked) > 1:
        second_count = ranked[1][1]

    total = len(normalized)
    entropy = 0.0
    for count in counts.values():
        probability = count / total
        entropy -= probability * math.log(probability + 1.0e-12)
    return ConsistencyResult(
        selected_answer=winner,
        vote_fraction=winner_count / total,
        vote_margin=(winner_count - second_count) / total,
        vote_entropy=entropy,
        raw_answers=raw_answers,
    )


def generate_consistent_answer(model, prompt, repeats, normalize_answer):
    answers = []
    for seed in range(repeats):
        answer = model.generate(prompt, temperature=0.7, seed=seed)
        answers.append(answer)
    return select_by_majority(answers, normalize_answer)
```

Part 5 - The Philosophy Behind the Exercise

Agreement is evidence about stability, not direct evidence about reality. A community of

samples can reproduce one shared bias because all members inherit the same model and data. Self-consistency is valuable when independent routes carry partially independent errors; its limit appears when the errors are systematic.

The exercise teaches a broader principle: ensembles improve judgment only to the degree that their diversity is meaningful and their aggregation respects uncertainty.

18.9.9 Exercise 08

Exercise Title: Debug Cases Where Model Sounds Confident but Is Wrong

Part 1 - Why this matters

The most harmful hallucinations often contain precise numbers, named sources, causal detail, and decisive language. These features can increase human trust without increasing evidential support. Build a debugging workflow around high-confidence errors rather than editing prompts ad hoc. Preserve the entire request, system message, retrieved context, tool results, model/version, decoding configuration, token probabilities if available, and evaluator evidence.

Decompose the failure. Did retrieval omit the answer, return a distractor, or provide conflicting evidence? Did the prompt contain a false premise? Did the generator copy evidence incorrectly, invent an unsupported bridge, or attach a citation to the wrong claim? Did a later token commit the answer to a mistaken trajectory? Compare with oracle context, lower temperature, explicit abstention, a larger model, and an independent verifier one change at a time.

Do not treat hedging words as calibrated uncertainty. Measure whether a confidence signal predicts correctness. The debugging output should identify the earliest causal boundary that failed and a regression test that reproduces it.

Part 2 - Mathematical form and reasoning

Define a high-confidence error set

$$\mathcal{E}_\tau = \{i : y_i = 0 \wedge p_i \geq \tau\}.$$

Its rate among confident predictions is

$$R_{\mathrm{HC}}(\tau) = \frac{\sum_i I(y_i = 0, p_i \geq \tau)}{\sum_i I(p_i \geq \tau)}.$$

For a pipeline with retrieval R, grounding G, and final answer A, annotate the earliest failed event. This produces mutually interpretable root-cause counts instead of assigning every symptom to the final generator.

Part 3 - Hints and Helper

1. Freeze the exact failing trace before attempting a fix.
2. Split the output into atomic claims and attach evidence to each.
3. Compare realistic and oracle retrieval.

4. Test prompt, decoding, model, and verifier changes independently.
5. Examine token-level confidence near the first wrong commitment.
6. Distinguish verbal certainty from measured probability.
7. Add the minimal reproducer to a permanent regression suite.
8. Verify that the fix does not cause excessive refusal elsewhere.

Part 4 - Python code skeleton

```
from dataclasses import dataclass


@dataclass
class FailureTrace:
    item_id: str
    confidence: float
    correct: bool
    retrieval_ok: bool
    evidence_used_correctly: bool
    unsupported_bridge: bool
    citation_mismatch: bool


def root_cause(trace):
    if not trace.retrieval_ok:
        return "retrieval_failure"
    if not trace.evidence_used_correctly:
        return "grounding_failure"
    if trace.unsupported_bridge:
        return "reasoning_bridge_failure"
    if trace.citation_mismatch:
        return "attribution_failure"
    return "unclassified_generation_failure"


def collect_high_confidence_errors(traces, threshold):
    selected = []
    counts = {}
    for trace in traces:
        if trace.correct or trace.confidence < threshold:
            continue
        cause = root_cause(trace)
        selected.append(trace)
        counts[cause] = counts.get(cause, 0) + 1
    return selected, counts
```

```
def regression_record(trace, prompt, expected_behavior):
    return {
        "item_id": trace.item_id,
        "prompt": prompt,
        "expected_behavior": expected_behavior,
        "root_cause": root_cause(trace),
    }
```

Part 5 - The Philosophy Behind the Exercise

Confidence in language is a social signal. Models learn its form from texts in which precision and authority often accompany expertise, but they can reproduce the form after the evidential foundation has disappeared. Debugging must break this spell by reconstructing the foundation claim by claim.

The aim is not to make the model sound permanently uncertain. It is to align decisiveness with evidence and make unsupported authority expensive to emit.

18.9.10 Exercise 09

Exercise Title: Build Automatic Hallucination Detection Heuristics

Part 1 - Why this matters

Human claim verification is valuable but too slow for every production output. Automatic heuristics can prioritize review, trigger abstention, or monitor drift. They are screening instruments, not proof of truth. Useful signals include missing citations, citation-entailment failure, contradiction with provided context, unsupported named entities or numbers, retrieval coverage, answer-to-evidence similarity, self-contradiction, cross-sample disagreement, and unusual confidence.

Build a modular detector that returns individual signals and reasons rather than one opaque score. Evaluate each signal against human labels using precision, recall, false-positive cost, and performance by domain. Tune thresholds on a development set and freeze them before final evaluation. Prevent leakage when an LLM judge is closely related to the model being judged.

Combine heuristics conservatively. A low lexical overlap can be legitimate paraphrase, while high overlap can copy a passage without answering correctly. Design explicit escalation bands: accept, review, abstain, or block. Log which rule fired so humans can audit and improve the detector.

Part 2 - Mathematical form and reasoning

For detector score s_i and threshold τ, predicted hallucination is $\hat{h}_i = I(s_i \geq \tau)$. Then

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}.$$

Choose τ from operational cost, not accuracy alone:

$$\tau^* = \arg\min_{\tau} \left(c_{FP} FP(\tau) + c_{FN} FN(\tau) + c_R R(\tau) \right),$$

where R is the number sent to human review. Domain-specific thresholds may be necessary when base rates and consequences differ.

Part 3 - Hints and Helper

1. Implement signals independently before combining them.
2. Return interpretable reasons and supporting spans.
3. Use human-labeled development and hidden test sets.
4. Measure precision-recall curves and domain slices.
5. Include paraphrases and copied-but-wrong outputs as hard negatives.
6. Calibrate escalation thresholds to review capacity and risk.
7. Monitor detector drift after model or retrieval updates.
8. Never present a heuristic acceptance as factual verification.

Part 4 - Python code skeleton

```
from dataclasses import dataclass


@dataclass
class DetectionSignal:
    name: str
    score: float
    reason: str


def combine_signals(signals, weights):
    weighted_sum = 0.0
    total_weight = 0.0
    reasons = []
    for signal in signals:
        weight = weights.get(signal.name, 0.0)
        weighted_sum += weight * signal.score
        total_weight += abs(weight)
        if signal.score >= 0.5:
            reasons.append(signal.reason)
    if total_weight == 0.0:
        raise ValueError("at least one detector weight is required")
    return weighted_sum / total_weight, reasons


def decide_action(score, review_threshold, block_threshold):
    if score >= block_threshold:
```

```
        return "abstain_or_block"
    if score >= review_threshold:
        return "human_review"
    return "accept_with_monitoring"


def confusion_counts(labels, scores, threshold):
    counts = {"tp": 0, "fp": 0, "tn": 0, "fn": 0}
    for label, score in zip(labels, scores):
        predicted = score >= threshold
        if predicted and label:
            counts["tp"] += 1
        elif predicted and not label:
            counts["fp"] += 1
        elif not predicted and label:
            counts["fn"] += 1
        else:
            counts["tn"] += 1
    return counts
```

Part 5 - The Philosophy Behind the Exercise

Detection occupies the space between ignorance and proof. It can notice that an answer lacks the features normally associated with support, but it cannot turn correlation into truth. Its value comes from allocating scarce attention to the outputs most likely to need it.

An interpretable detector respects this limited role. It does not announce "true"; it says which reliability obligations appear unmet and what should happen next.

18.9.11 Exercise 10

Exercise Title: Compare Multiple Model Sizes on Same Evaluation Set

Part 1 - Why this matters

Larger models often improve knowledge and reasoning, but parameter count does not guarantee lower hallucination, better calibration, or safer abstention. They may produce more persuasive errors, incur higher latency and cost, or use different tokenizers and context limits. Compare model sizes under a common evaluation contract rather than treating a leaderboard score as deployment evidence.

Freeze prompts, evidence, decoding semantics, output limits, scoring rules, and hardware reporting. If tokenizers differ, report both text length and each model's token count. Evaluate factual accuracy, claim hallucination, calibration, refusal usefulness, paraphrase robustness, RAG faithfulness, latency, memory, energy, and cost. Use paired per-item results so improvements and regressions can be localized.

Construct capability curves rather than a single winner. A smaller model may meet requirements for routine grounded queries while a larger model is needed for multi-hop reasoning. Consider a routing policy, but include routing errors and operational complexity. The recommendation must name the workload and hard reliability gates it satisfies.

Part 2 - Mathematical form and reasoning

For models a and b on matched items, paired accuracy change is

$$\Delta A = \frac{1}{N}\sum_{i=1}^{N}(y_{ib} - y_{ia}).$$

Define useful throughput by discounting incorrect outputs:

$$\text{UsefulTokens/s} = \text{Tokens/s} \times (1 - \widehat{H}).$$

A deployment chooses the lowest-cost feasible model,

$$m^* = \arg\min_{m \in \mathcal{F}} C_m,$$

where $\mathcal{F}$ contains models meeting accuracy, hallucination, calibration, latency, memory, and safety constraints. Scaling curves should include uncertainty and not assume monotonic improvement.

Part 3 - Hints and Helper

1. Use one immutable evaluation manifest for every model.
2. Match decoding behavior and document unavoidable API differences.
3. Save per-item outputs and paired correctness transitions.
4. Report tokenizer, context, hardware, precision, latency, and memory.
5. Compare confidence calibration, not only factual accuracy.
6. Stratify by task difficulty and failure family.
7. Evaluate routing as a separate system if proposed.
8. Select the smallest model that passes all hard gates.

Part 4 - Python code skeleton

```
from dataclasses import dataclass


@dataclass
class ModelResult:
    model_name: str
    item_id: str
    correct: bool
    hallucinated: bool
    latency_ms: float
    output_tokens: int
    cost: float
```

```
def paired_transition_table(left_results, right_results):
    left_by_id = {}
    right_by_id = {}
    for result in left_results:
        left_by_id[result.item_id] = result
    for result in right_results:
        right_by_id[result.item_id] = result

    counts = {
        "both_correct": 0,
        "left_only": 0,
        "right_only": 0,
        "both_wrong": 0,
    }
    for item_id, left in left_by_id.items():
        right = right_by_id[item_id]
        if left.correct and right.correct:
            counts["both_correct"] += 1
        elif left.correct:
            counts["left_only"] += 1
        elif right.correct:
            counts["right_only"] += 1
        else:
            counts["both_wrong"] += 1
    return counts


def passes_gates(metrics, gates):
    for name, rule in gates.items():
        value = metrics[name]
        if rule["direction"] == "minimum" and value < rule["value"]:
            return False
        if rule["direction"] == "maximum" and value > rule["value"]:
            return False
    return True
```

Part 5 - The Philosophy Behind the Exercise

Scale buys representational capacity, not unconditional reliability. A larger model can know more and still fail to recognize when it does not know. Treating size as a substitute for evidence turns an engineering choice into mythology.

The disciplined choice is conditional: use enough capacity for the task, then verify the behavior that matters. Efficiency is not choosing the smallest model; it is refusing to pay for capacity that does not improve the required outcomes.

18.9.12 Exercise 11

Exercise Title: Implement Abstention or “I Don’t Know” Strategy Conceptually

Part 1 - Why this matters

A reliable model should sometimes decline to answer. Abstention is useful when evidence is absent, retrieval is weak, model confidence is poorly calibrated, the prompt is ambiguous, or the consequence of error is high. However, a model that refuses everything achieves zero hallucinations while providing zero utility. The goal is selective prediction: answer when expected reliability is high enough and escalate otherwise.

Build signals from calibrated confidence, retrieval coverage, citation support, sample agreement, detector score, domain risk, and input validity. Distinguish hard refusal from a clarification request, an evidence request, a tool call, a safe partial answer, or human escalation. The user-facing response should explain what is missing and how to proceed without inventing facts.

Tune thresholds on a development set under an explicit error-versus-coverage objective. Evaluate selective accuracy, hallucination among answered cases, coverage, false abstention, group/domain disparities, and escalation load. Test whether adversarial wording can suppress abstention or make the model claim false certainty.

Part 2 - Mathematical form and reasoning

For risk estimate $r(x)$ and threshold τ, answer when $r(x) \le \tau$. Coverage and selective risk are

$$\text{Coverage}(\tau) = \frac{1}{N} \sum_i I(r_i \le \tau),$$

$$\text{Risk}(\tau) = \frac{\sum_i I(r_i \le \tau)\ell_i}{\sum_i I(r_i \le \tau)}.$$

Plot the risk-coverage curve. A policy dominates another when it yields lower risk at equal coverage. For risk-sensitive domains, add expected cost for wrong answers, abstentions, and human review rather than optimizing accuracy alone.

Part 3 - Hints and Helper

1. Define answer, clarify, retrieve, escalate, and refuse as separate actions.
2. Calibrate risk signals on held-out data.
3. Optimize the risk-coverage curve under review-capacity limits.
4. Write useful abstention messages that identify missing evidence.
5. Test false abstention on easy and underrepresented cases.
6. Apply stricter thresholds to higher-consequence domains.
7. Prevent prompt text from directly overriding policy gates.
8. Monitor coverage and error after deployment.

Part 4 - Python code skeleton

```
from dataclasses import dataclass


@dataclass
class ReliabilitySignals:
    calibrated_confidence: float
    retrieval_coverage: float
    detector_risk: float
    domain_risk: float
    input_valid: bool


def choose_action(signals, answer_threshold, review_threshold):
    if not signals.input_valid:
        return "request_clarification"
    evidence_risk = 1.0 - signals.retrieval_coverage
    confidence_risk = 1.0 - signals.calibrated_confidence
    combined_risk = (
        0.30 * evidence_risk
        + 0.30 * confidence_risk
        + 0.25 * signals.detector_risk
        + 0.15 * signals.domain_risk
    )
    if combined_risk <= answer_threshold:
        return "answer"
    if combined_risk <= review_threshold:
        return "human_review"
    return "abstain"


def selective_metrics(records):
    answered = 0
    correct_answered = 0
    for record in records:
        if record["action"] == "answer":
            answered += 1
            if record["correct"]:
                correct_answered += 1
    coverage = answered / len(records)
    selective_accuracy = 0.0
    if answered > 0:
        selective_accuracy = correct_answered / answered
    return coverage, selective_accuracy
```

Part 5 - The Philosophy Behind the Exercise

Knowing a limit is a capability, not an absence of capability. Human expertise includes the judgment to consult a source, ask a colleague, or postpone a claim. A model that must always

answer is denied this essential form of intelligence.

Abstention also redistributes responsibility honestly. It tells the system when automation has reached its evidential boundary and invites another process to continue the work.

18.9.13 Exercise 12

Exercise Title: Measure Calibration Error and Overconfidence

Part 1 - Why this matters

Calibration asks whether events predicted with confidence 0.8 are correct about 80 percent of the time. It differs from accuracy: a highly accurate model can be overconfident on its remaining errors, and a lower-accuracy model can expose uncertainty honestly. Reliable thresholds for abstention and review depend on calibration.

Select a confidence source that corresponds to a clearly defined event, such as exact answer correctness or claim support. Token probabilities do not automatically become answer probabilities; aggregation and length normalization must be justified. Compare raw confidence, temperature-scaled confidence, verbal probability, and sample agreement if available.

Measure reliability diagrams, expected calibration error (ECE), maximum calibration error, Brier score, negative log loss, and overconfidence on wrong answers. Report bin counts and several binning schemes because ECE is sensitive to bin boundaries. Fit calibration on a validation split only, then test on untouched data and domain slices.

Part 2 - Mathematical form and reasoning

Partition predictions into bins B_m. ECE is

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \left|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\right|.$$

Overconfidence on errors can be reported as

$$O_{\text{wrong}} = \frac{1}{|W|} \sum_{i \in W} p_i, \qquad W = \{i : y_i = 0\}.$$

For binary correctness logits z_i, temperature scaling uses $p_i = \sigma(z_i/T)$ and fits $T > 0$ on validation NLL. Calibration cannot repair incorrect ranking; it only remaps confidence.

Part 3 - Hints and Helper

1. Define the predicted event before defining confidence.
2. Preserve a separate calibration split and hidden test split.
3. Report ECE with bin counts and alternative binning.
4. Include Brier score and NLL as proper scoring rules.
5. Inspect confidence specifically on wrong and hallucinated outputs.
6. Plot calibration per domain and trigger family.

7. Refit after model, prompt, retrieval, or decoding changes.

8. Do not claim calibration outside the evaluated distribution.

Part 4 - Python code skeleton

```
def expected_calibration_error(confidences, labels, bin_count=10):
    if len(confidences) != len(labels):
        raise ValueError("confidence and label lengths differ")
    total = len(labels)
    ece = 0.0
    rows = []
    for bin_index in range(bin_count):
        lower = bin_index / bin_count
        upper = (bin_index + 1) / bin_count
        indices = []
        for index, confidence in enumerate(confidences):
            in_bin = lower <= confidence < upper
            if bin_index == bin_count - 1 and confidence == 1.0:
                in_bin = True
            if in_bin:
                indices.append(index)
        if len(indices) == 0:
            continue
        confidence_sum = 0.0
        correct_sum = 0.0
        for index in indices:
            confidence_sum += confidences[index]
            correct_sum += float(labels[index])
        mean_confidence = confidence_sum / len(indices)
        accuracy = correct_sum / len(indices)
        gap = abs(accuracy - mean_confidence)
        ece += (len(indices) / total) * gap
        rows.append((lower, upper, len(indices), accuracy, mean_confidence))
    return ece, rows
```

Part 5 - The Philosophy Behind the Exercise

Calibration is a promise about frequencies, not an inner feeling of certainty. It allows confidence to acquire operational meaning across repeated decisions. Without that link, a number such as 0.9 is decoration.

The exercise also teaches modesty about scope: a model may be calibrated on one population and dangerously overconfident after the world, prompt, or evidence pipeline changes.

18.9.14 Exercise 13

Exercise Title: Evaluate Domain Shift and Knowledge Cutoff Effects

Part 1 - Why this matters

An LLM is evaluated under one distribution and deployed into a changing world. New events occur after training, terminology changes, organizations revise policies, and user populations ask questions from domains absent in the test set. The model may answer stale questions fluently because it cannot directly observe that its internal knowledge is outdated.

Build matched in-domain, shifted-domain, pre-cutoff, and post-cutoff subsets. Attach an "as of" date to every temporal question and use time-stamped authoritative evidence. Compare closed-book generation, RAG with current documents, and abstention. Avoid leakage: a model released later may have a different unknown training cutoff, so document only verifiable model metadata and evaluate behavior directly.

Measure accuracy, hallucination, calibration, retrieval gain, and stale-answer rate. Shift should include vocabulary, style, task, source, and population changes, not just harder questions. Monitor embedding or input-distribution drift in production, but validate that detected drift predicts actual quality loss before using it as a rollback signal.

Part 2 - Mathematical form and reasoning

For source distribution P and target distribution Q, empirical performance drop is

$$\Delta_Q = M_Q - M_P.$$

If M is accuracy, negative values indicate degradation; if M is hallucination, positive values do. A post-cutoff stale-answer rate is

$$S = \frac{1}{N_{\text{post}}} \sum_i I(\text{answer contradicts evidence valid at query date}).$$

Report performance by shift type and distance proxy. An aggregate shift score cannot identify whether vocabulary, knowledge time, or task composition caused the failure.

Part 3 - Hints and Helper

1. Attach query date and evidence-validity interval to temporal items.
2. Match shifted and source examples by difficulty when possible.
3. Separate post-cutoff knowledge from general domain unfamiliarity.
4. Compare closed-book, current RAG, and abstention policies.
5. Measure confidence drift along with error drift.
6. Preserve model metadata without assuming undocumented training data.
7. Validate whether input drift correlates with reliability degradation.
8. Define refresh and re-evaluation triggers before deployment.

Part 4 - Python code skeleton

```
from collections import defaultdict
from dataclasses import dataclass


@dataclass
class ShiftResult:
    item_id: str
    split: str
    shift_type: str
    correct: bool
    hallucinated: bool
    stale: bool
    confidence: float


def summarize_shift(results):
    grouped = defaultdict(list)
    for result in results:
        key = (result.split, result.shift_type)
        grouped[key].append(result)

    summary = {}
    for key, group in grouped.items():
        correct = 0
        hallucinated = 0
        stale = 0
        confidence = 0.0
        for result in group:
            correct += int(result.correct)
            hallucinated += int(result.hallucinated)
            stale += int(result.stale)
            confidence += result.confidence
        count = len(group)
        summary[key] = {
            "count": count,
            "accuracy": correct / count,
            "hallucination_rate": hallucinated / count,
            "stale_rate": stale / count,
            "mean_confidence": confidence / count,
        }
    return summary
```

Part 5 - The Philosophy Behind the Exercise

Knowledge is situated in time and community. A static parameter set can encode patterns from the past, but truth continues to change after training stops. Reliability requires a relationship with current evidence, not nostalgia for a benchmark snapshot.

Domain shift reminds us that competence is conditional. The honest statement is never simply "the model is accurate"; it is accurate for particular tasks, people, sources, and dates within measured limits.

18.9.15 Exercise 14

Exercise Title: Build Reliability Dashboard for LLM Outputs

Part 1 - Why this matters

Reliability evidence becomes operational only when teams can observe it over time. A dashboard should connect user traffic, model/version, prompt template, retrieval index, confidence, detector signals, abstention, human review, latency, and eventual correctness labels. It must support investigation rather than display attractive averages.

Design three levels: executive service health, engineering diagnosis, and case-level trace. Show volume, hallucination estimates, verified error rate, calibration, refusal/coverage, citation support, retrieval failure, human-review queue, p95/p99 latency, cost, and model or data drift. Slice by domain, language, risk, tenant, prompt version, and deployment release while enforcing privacy and minimum-count rules.

Separate real-time proxies from delayed ground truth. Detector risk can alert quickly but is not the same as human-verified hallucination. Mark missing or stale telemetry visibly. Establish baselines, control limits, alert ownership, and links to representative traces. A dashboard that cannot lead from a spike to affected examples is a poster, not an engineering instrument.

Part 2 - Mathematical form and reasoning

For metric X_t in time window t, a standardized deviation is

$$z_t = \frac{X_t - \mu_{\text{base}}}{\sigma_{\text{base}} + \epsilon}.$$

An alert may require both sufficient volume and repeated deviation:

$$A_t = I(n_t \geq n_{\min}) I\left(\sum_{j=0}^{K-1} I(|z_{t-j}| > z_{\max}) = K\right).$$

For delayed labels, display label coverage

$$C_t = \frac{n_{\text{labeled},t}}{n_{\text{eligible},t}},$$

so apparent improvements are not caused by fewer reviewed examples.

Part 3 - Hints and Helper

1. Define a common event schema with model, prompt, retrieval, and trace identities.
2. Separate online proxy metrics from delayed verified outcomes.
3. Display denominators, confidence intervals, and label coverage.
4. Add domain, risk, language, and release filters.

5. Suppress tiny privacy-sensitive slices and control trace access.
6. Link alerts to representative failing requests.
7. Assign alert thresholds, owners, and runbooks.
8. Test the dashboard by injecting known failures and missing telemetry.

Part 4 - Python code skeleton

```
from collections import defaultdict
from dataclasses import dataclass


@dataclass
class ReliabilityEvent:
    release_id: str
    domain: str
    request_id: str
    detector_risk: float
    abstained: bool
    verified_hallucination: object
    latency_ms: float


def aggregate_events(events, minimum_slice_count):
    groups = defaultdict(list)
    for event in events:
        key = (event.release_id, event.domain)
        groups[key].append(event)

    rows = []
    for key, group in groups.items():
        if len(group) < minimum_slice_count:
            continue
        risk_sum = 0.0
        abstentions = 0
        labeled = 0
        hallucinations = 0
        latencies = []
        for event in group:
            risk_sum += event.detector_risk
            abstentions += int(event.abstained)
            latencies.append(event.latency_ms)
            if event.verified_hallucination is not None:
                labeled += 1
                hallucinations += int(event.verified_hallucination)
        verified_rate = None
        if labeled > 0:
```

```
            verified_rate = hallucinations / labeled
        rows.append({
            "release_id": key[0],
            "domain": key[1],
            "volume": len(group),
            "mean_detector_risk": risk_sum / len(group),
            "abstention_rate": abstentions / len(group),
            "label_coverage": labeled / len(group),
            "verified_hallucination_rate": verified_rate,
        })
    return rows
```

Part 5 - The Philosophy Behind the Exercise

A dashboard is a shared memory of system behavior. It turns isolated failures into patterns and makes responsibility continuous after release. Yet visualization can create false confidence if proxy signals, missing labels, and small denominators are hidden.

The reliable dashboard makes uncertainty visible. Its purpose is not to prove that the system is healthy, but to help humans notice when the evidence for health has weakened.

18.9.16 Exercise 15

Exercise Title: Stress Test Model with Adversarial Prompts

Part 1 - Why this matters

Adversarial prompts deliberately search for ways to bypass evidence rules, abstention, privacy boundaries, tool constraints, or system instructions. In a reliability chapter, the goal is not to collect dramatic jailbreak examples; it is to understand how ordinary and malicious inputs transform the model's epistemic behavior. Test false premises, authority pressure, requested fake citations, prompt injection inside retrieved documents, encoded instructions, conflicting roles, very long distractors, multilingual switching, and requests to claim actions that were not performed.

Define allowed and disallowed outcomes before testing. Preserve the system prompt and deployment wrappers, because a naked model benchmark does not represent the production boundary. Use a controlled red-team corpus and mutate prompts systematically instead of selecting only successful attacks. Record the attack family, transformations, release identity, exposed tools, evidence, response, detector output, and final policy decision.

Measure attack success, factual hallucination, false compliance, safe-completion quality, refusal precision, and transfer across paraphrases and languages. Analyze near misses: an answer may refuse the malicious request but still leak an unsupported factual claim. Convert every confirmed weakness into a minimal regression test and verify that mitigations do not destroy benign utility.

Part 2 - Mathematical form and reasoning

For attack family k, attack success rate is

$$\mathrm{ASR}_k = \frac{1}{n_k} \sum_{i=1}^{n_k} a_{ki},$$

where success is defined by a predeclared violation, not by reviewer surprise. Let B be benign utility before mitigation and B' after mitigation. A fix should report both security gain and utility cost:

$$\Delta\mathrm{ASR} = \mathrm{ASR}' - \mathrm{ASR}, \qquad \Delta B = B' - B.$$

Worst-family ASR prevents a large easy category from hiding one weak attack surface.

Part 3 - Hints and Helper

1. Define threat model, protected assets, and attack success before generation.
2. Test the complete production prompt, retrieval, tool, and policy stack.
3. Include prompt injection inside untrusted retrieved content.
4. Generate controlled transformations for language, encoding, and role pressure.
5. Keep benign matched controls for every mitigation.
6. Measure worst-family performance and transfer to unseen variants.
7. Store sensitive attack details with appropriate access control.
8. Add confirmed failures to a permanent regression suite.

Part 4 - Python code skeleton

```
from dataclasses import dataclass


@dataclass
class AdversarialCase:
    case_id: str
    family: str
    prompt: str
    expected_policy: str
    benign_control: bool


@dataclass
class AdversarialResult:
    case_id: str
    family: str
    policy_violated: bool
    hallucinated: bool
    useful_safe_response: bool
```

```
def run_cases(system, cases):
    raw_results = []
    for case in cases:
        response = system.respond(case.prompt)
        raw_results.append((case, response))
    return raw_results


def summarize_attack_results(results):
    family_totals = {}
    family_successes = {}
    for result in results:
        family = result.family
        family_totals[family] = family_totals.get(family, 0) + 1
        if result.policy_violated:
            old_value = family_successes.get(family, 0)
            family_successes[family] = old_value + 1
    rates = {}
    for family, total in family_totals.items():
        success_count = family_successes.get(family, 0)
        rates[family] = success_count / total
    return rates
```

Part 5 - The Philosophy Behind the Exercise

Adversarial testing studies the gap between a rule stated in language and a rule enforced by a system. A model may understand the words "ignore untrusted instructions" while still following a persuasive instruction embedded in a document. Reliability requires boundaries stronger than rhetorical reminders.

The exercise treats failure discovery as care rather than hostility. By seeking the boundary deliberately, engineers learn where users would otherwise meet it accidentally and at greater cost.

18.9.17 Exercise 16

Exercise Title: Compare Retrieval-Augmented vs Fine-Tuned Models

Part 1 - Why this matters

Retrieval augmentation and fine-tuning solve different problems. RAG supplies current, inspectable knowledge at inference time. Fine-tuning changes behavior, format, style, task procedure, and sometimes domain capability, but it is a poor mechanism for frequently changing facts and may memorize or distort them. Compare them under the same task rather than asking which technique is universally better.

Build four conditions where feasible: base, RAG, fine-tuned, and fine-tuned plus RAG. Hold model family, evaluation set, decoding, and compute reporting as constant as possible.

Split tests into stable procedure, static domain knowledge, post-training updates, no-answer questions, long evidence, and adversarial distractors. Measure accuracy, hallucination, citation faithfulness, format adherence, calibration, abstention, latency, memory, index/update cost, training cost, and maintenance complexity.

Test provenance and refresh. RAG should expose which source supported a claim; a fine-tuned fact may not be attributable. Update part of the knowledge base and measure how quickly each system reflects the change. Check whether fine-tuning damages general capabilities or makes the model more confidently wrong outside its training distribution.

Part 2 - Mathematical form and reasoning

Fine-tuning optimizes parameters,

$$\theta' = \arg\min_{\theta} \sum_{(x,y)\in\mathcal{T}} \mathcal{L}(f_\theta(x), y),$$

while RAG keeps parameters fixed and conditions on selected evidence $D_k(x)$:

$$y \sim p_\theta(y \mid x, D_k(x)).$$

Compare total lifecycle cost over horizon T:

$$C_{\text{life}} = C_{\text{build}} + C_{\text{update}}(T) + C_{\text{serve}}(T) + C_{\text{review}}(T).$$

The technically best score may not minimize lifecycle cost or maximize traceability.

Part 3 - Hints and Helper

1. Match base architecture and evaluation prompts across conditions.
2. Create subsets for stable behavior and changing facts.
3. Evaluate oracle and realistic retrieval for the RAG conditions.
4. Record fine-tuning data provenance and contamination risk.
5. Measure citations and update freshness, not only accuracy.
6. Test catastrophic forgetting and out-of-domain calibration.
7. Include build, refresh, serving, and human-review costs.
8. Recommend a method per requirement, including hybrid use when justified.

Part 4 - Python code skeleton

```
from dataclasses import dataclass


@dataclass
class ConditionMetrics:
    condition: str
```

```
    accuracy: float
    hallucination_rate: float
    citation_precision: float
    format_adherence: float
    latency_ms: float
    build_cost: float
    monthly_update_cost: float
    monthly_serve_cost: float


def lifecycle_cost(metrics, months):
    return (
        metrics.build_cost
        + months * metrics.monthly_update_cost
        + months * metrics.monthly_serve_cost
    )


def compare_conditions(all_metrics, months, gates):
    rows = []
    for metrics in all_metrics:
        passed = True
        if metrics.accuracy < gates["minimum_accuracy"]:
            passed = False
        if metrics.hallucination_rate > gates["maximum_hallucination"]:
            passed = False
        if metrics.latency_ms > gates["maximum_latency_ms"]:
            passed = False
        rows.append({
            "condition": metrics.condition,
            "passes_gates": passed,
            "lifecycle_cost": lifecycle_cost(metrics, months),
            "citation_precision": metrics.citation_precision,
            "format_adherence": metrics.format_adherence,
        })
    return rows
```

Part 5 - The Philosophy Behind the Exercise

RAG and fine-tuning represent two kinds of learning. One changes the evidence available now; the other changes the dispositions of the model. Confusing them encourages teams to bake changing facts into weights or retrieve instructions that should have become stable behavior.

The mature design asks where knowledge should live, how it should be updated, and how a claim can be traced. Technique follows epistemic architecture.

18.9.18 Exercise 17

Exercise Title: Document Known Failure Patterns and Safe Usage Guidelines

Part 1 - Why this matters

Every deployed LLM has known limits. If those limits remain in private benchmark notebooks, users and operators will repeat avoidable failures. Create a living failure catalog that names the pattern, triggering conditions, affected versions, observable symptoms, severity, detection method, mitigation, residual risk, safe alternative, owner, evidence, and review date.

Cover false premises, fabricated citations, stale facts, long-context neglect, retrieval distraction, calculation errors, multilingual degradation, prompt injection, confidence mismatch, excessive or insufficient refusal, domain shift, and tool-use claims without execution. Distinguish confirmed limitations from hypotheses. Include examples that are representative without exposing private data or providing unnecessary exploit detail.

Translate the catalog into audience-specific usage guidelines. End users need clear verification and escalation advice. Developers need supported inputs, evidence requirements, and forbidden integrations. Operators need monitors, thresholds, and recovery actions. Governance reviewers need scope, ownership, exceptions, and revalidation. Test whether readers can apply the guidelines to new scenarios; documentation that no one can operationalize is not a control.

Part 2 - Mathematical form and reasoning

Prioritize pattern k with an explicit risk model,

$$R_k = P_k \times S_k \times E_k,$$

where P is observed likelihood, S consequence severity, and E exposure. The factors need defined scales and evidence. Residual risk after mitigation is

$$R_k^{\text{res}} = R_k(1 - M_k),$$

where M_k is measured mitigation effectiveness, not an assumed checkbox. High-severity low-frequency failures should remain visible even when their average contribution is small.

Part 3 - Hints and Helper

1. Give every failure pattern a stable identifier and evidence link.
2. Record affected releases, domains, languages, and prompt conditions.
3. Separate symptom, root cause, mitigation, and residual risk.
4. Write safe alternatives instead of only prohibitions.
5. Tailor guidance for users, developers, operators, and reviewers.
6. Add owners, review dates, and retirement criteria.
7. Test guidance through scenario exercises and incident drills.
8. Update the catalog after every confirmed production failure.

Part 4 - Python code skeleton

```
from dataclasses import asdict, dataclass
from typing import List
import json


@dataclass
class FailurePattern:
    pattern_id: str
    name: str
    triggers: List[str]
    affected_releases: List[str]
    symptom: str
    severity: int
    likelihood: float
    exposure: float
    detection: str
    mitigation: str
    safe_alternative: str
    owner: str
    evidence_uri: str
    review_date: str


def risk_score(pattern):
    return pattern.severity * pattern.likelihood * pattern.exposure


def validate_patterns(patterns):
    errors = []
    seen = set()
    for pattern in patterns:
        if pattern.pattern_id in seen:
            errors.append("duplicate:" + pattern.pattern_id)
        seen.add(pattern.pattern_id)
        if pattern.evidence_uri.strip() == "":
            errors.append("missing_evidence:" + pattern.pattern_id)
        if pattern.safe_alternative.strip() == "":
            errors.append("missing_safe_alternative:" + pattern.pattern_id)
    return errors


def save_catalog(patterns, output_path):
    errors = validate_patterns(patterns)
    if len(errors) > 0:
        raise ValueError(",".join(errors))
    ordered = sorted(patterns, key=risk_score, reverse=True)
    records = []
    for pattern in ordered:
```

```
        record = asdict(pattern)
        record["risk_score"] = risk_score(pattern)
        records.append(record)
    with open(output_path, "w", encoding="utf-8") as handle:
        json.dump(records, handle, indent=2)
```

Part 5 - The Philosophy Behind the Exercise

Documentation is a promise not to make each person rediscover the same danger. It converts private engineering experience into shared practical knowledge. Stating limitations does not weaken trust; hiding known limits makes trust fragile and undeserved.

Safe guidance respects users as participants in reliability. It gives them the information and alternatives required to judge when automation is appropriate.

18.9.19 Exercise 18

Exercise Title: Build Human-in-the-Loop Review Workflow

Part 1 - Why this matters

Automation should escalate cases whose uncertainty, novelty, or consequence exceeds its safe authority. A human-in-the-loop workflow is not a button labeled "review"; it is a queueing, evidence, accountability, and feedback system. Define which outputs require review using domain risk, confidence, detector signals, retrieval failure, policy rules, and random quality sampling.

Present reviewers with the user request, model output, atomic claims, cited evidence, retrieval trace, confidence, detector reasons, model/version, and clear decision options. Blind irrelevant information that could bias judgment. Require reviewers to mark supported, corrected, abstained, escalated, or policy violating outcomes and provide structured reasons. High-risk decisions may require dual review or qualified specialists.

Design operational controls: priority, service-level target, capacity, workload forecast, access control, privacy, conflict resolution, audit log, and emergency escalation. Measure review agreement, turnaround, queue age, overturn rate, residual error after review, and reviewer fatigue. Feed confirmed labels into evaluation and detector improvement only after data governance checks; do not silently train on every correction.

Part 2 - Mathematical form and reasoning

Let requests arrive at rate λ and each reviewer complete μ cases per unit time. With c reviewers, stable capacity requires

$$\rho = \frac{\lambda}{c\mu} < 1.$$

Operating near one creates rapidly growing delay and fatigue. A review policy can prioritize expected risk reduction:

$$V_i = P(\text{error} \mid x_i) \times S_i \times P(\text{review corrects error}) - C_i^{\text{review}}.$$

Use V_i with age and fairness constraints; risk alone must not indefinitely starve low-risk requests.

Part 3 - Hints and Helper

1. Define mandatory, risk-triggered, and random-audit review routes.
2. Show claim-level evidence and detector reasons to reviewers.
3. Use structured decisions and correction reasons.
4. Require specialist or dual review for designated high-risk domains.
5. Track queue volume, age, service level, and reviewer agreement.
6. Protect sensitive prompts and restrict trace access.
7. Sample accepted automated outputs to estimate missed errors.
8. Govern how review labels enter future training or evaluation.

Part 4 - Python code skeleton

```
from dataclasses import dataclass
from datetime import datetime, timezone
import heapq


@dataclass
class ReviewCase:
    case_id: str
    error_probability: float
    severity: float
    created_at: str
    requires_specialist: bool
    payload_uri: str


def priority_value(case, current_time, aging_weight=0.01):
    created = datetime.fromisoformat(case.created_at)
    age_seconds = (current_time - created).total_seconds()
    risk = case.error_probability * case.severity
    return risk + aging_weight * age_seconds


def build_priority_queue(cases):
    current_time = datetime.now(timezone.utc)
    queue = []
    for case in cases:
        priority = priority_value(case, current_time)
```

```
        heapq.heappush(queue, (-priority, case.case_id, case))
    return queue


def next_eligible_case(queue, reviewer_is_specialist):
    deferred = []
    selected = None
    while len(queue) > 0:
        item = heapq.heappop(queue)
        case = item[2]
        if case.requires_specialist and not reviewer_is_specialist:
            deferred.append(item)
            continue
        selected = case
        break
    for item in deferred:
        heapq.heappush(queue, item)
    return selected
```

Part 5 - The Philosophy Behind the Exercise

Human review is not an admission that automation failed. It is a design choice about where judgment and responsibility should reside. Models offer scale and pattern recognition; people contribute contextual understanding, authority, and accountability.

The partnership succeeds only when escalation is selective, evidence-rich, and operationally sustainable. Sending everything to an exhausted reviewer merely moves the failure downstream.

18.9.20 Exercise 19

Exercise Title: Produce a Full LLM Reliability and Evaluation Report

Part 1 - Why this matters

The final report assembles the chapter into an auditable argument about where the LLM can be trusted, where it must abstain, and how remaining risk will be observed. A list of benchmark averages is insufficient. Preserve model and tokenizer identity, prompt templates, retrieval/index versions, tools, decoding, hardware, evaluation-set manifest, evidence dates, judge instructions, human agreement, code revision, seeds, and raw-result locations.

Report evaluation construction and coverage before results. Include trigger families, answerability, domains, languages, temporal splits, paraphrase families, adversarial cases, model sizes, and known gaps. Then present claim and answer hallucination, accuracy, confidence mismatch, calibration, abstention risk-coverage, RAG retrieval and citation metrics, self-consistency, domain shift, attack success, latency, cost, and human-review performance. Provide uncertainty, denominators, slices, and paired transitions.

Translate findings into a deployment decision. Specify supported and prohibited uses, evidence requirements, model-routing or RAG policy, abstention and review thresholds, monitoring

dashboard, alert/runbook ownership, known failure patterns, canary gates, rollback conditions, and revalidation date. Link every claim to raw evidence and record exceptions. Include negative results and unresolved unknowns; they define the limits of the release. An independent reviewer should reproduce selected metrics before sign-off.

Part 2 - Mathematical form and reasoning

Readiness is conjunctive:

$$G = G_{\text{identity}} \wedge G_{\text{coverage}} \wedge G_{\text{factuality}} \wedge G_{\text{calibration}} \wedge G_{\text{robustness}} \wedge G_{\text{operations}} \wedge G_{\text{review}}.$$

Each gate contains a requirement, threshold, observation, uncertainty, evidence URI, owner, and status. A missing required value fails closed. Traceability is

$$\text{risk} \rightarrow \text{test} \rightarrow \text{result} \rightarrow \text{control} \rightarrow \text{monitor}.$$

For central binary rates, report estimates with intervals; for comparisons, prefer paired differences on identical examples. Do not average a failed safety gate into good latency or accuracy.

Part 3 - Hints and Helper

1. Use a versioned schema shared by future reliability reports.
2. Hash model, tokenizer, prompt, corpus, index, dataset, and result artifacts.
3. Link every claim to reproducible raw evidence.
4. Present overall metrics, critical slices, uncertainty, and denominators.
5. Include failure examples, rejected mitigations, and unresolved unknowns.
6. Convert findings into explicit abstention, review, monitoring, and rollback controls.
7. Require independent reproduction of selected results.
8. Set owners, expiry dates, and triggers for immediate re-evaluation.

Part 4 - Python code skeleton

```
from dataclasses import asdict, dataclass, field
from datetime import datetime, timezone
from typing import Dict, List
import json


@dataclass
class ReliabilityGate:
    name: str
    requirement: str
    threshold: str
```

```
    observed: str
    uncertainty: str
    evidence_uri: str
    owner: str
    passed: bool


@dataclass
class ReliabilityReport:
    release_id: str
    created_at: str
    artifact_hashes: Dict[str, str]
    evaluation_manifest: str
    gates: List[ReliabilityGate] = field(default_factory=list)
    known_failures: List[str] = field(default_factory=list)
    unsupported_uses: List[str] = field(default_factory=list)
    revalidation_date: str = ""
    decision: str = "pending"


def validate_report(report, required_gates):
    errors = []
    by_name = {}
    for gate in report.gates:
        by_name[gate.name] = gate
    for gate_name in required_gates:
        if gate_name not in by_name:
            errors.append("missing_gate:" + gate_name)
            continue
        gate = by_name[gate_name]
        if not gate.passed:
            errors.append("failed_gate:" + gate_name)
        if gate.evidence_uri.strip() == "":
            errors.append("missing_evidence:" + gate_name)
    if report.evaluation_manifest.strip() == "":
        errors.append("missing_evaluation_manifest")
    if report.revalidation_date.strip() == "":
        errors.append("missing_revalidation_date")
    return errors


def finalize_report(report, required_gates, output_path):
    errors = validate_report(report, required_gates)
    if len(errors) == 0:
        report.decision = "approved_with_documented_limits"
    else:
        report.decision = "blocked"
    payload = asdict(report)
```

```
    payload["validation_errors"] = errors
    with open(output_path, "w", encoding="utf-8") as handle:
        json.dump(payload, handle, indent=2, sort_keys=True)
    return errors


def create_report(release_id, hashes, evaluation_manifest):
    created_at = datetime.now(timezone.utc).isoformat()
    return ReliabilityReport(
        release_id=release_id,
        created_at=created_at,
        artifact_hashes=hashes,
        evaluation_manifest=evaluation_manifest,
    )
```

Part 5 - The Philosophy Behind the Exercise

Evaluation is a claim about a model under specified conditions, not a permanent property of the model. A rigorous report preserves those conditions so that future changes in data, time, tools, or users can challenge the claim honestly.

Reliability is therefore not the absence of failure. It is the disciplined ability to define failure, search for it, measure uncertainty, limit authority, route difficult cases, observe deployment, and revise decisions when evidence changes. The final report is the memory that makes this discipline continuous.

18.10 LLM08

Chapter Title: Agents, Tools, and RAG Systems
Specialization: Applied AI and Agentic Systems Engineering

18.10.1 Exercise 00

Exercise Title: Explain Tool-Using LLMs and Why Pure Text Generation Is Not Enough

Part 1 - Why this matters

Pure text generation can explain how to calculate, search, query, send, or modify something, but explanation is not execution. An LLM's parameters are a compressed and time-bounded source of patterns; they cannot guarantee current database values, exact arithmetic, private account state, or that an external action actually occurred. A tool-using system gives the model controlled access to calculators, retrieval indexes, databases, code runtimes, APIs, sensors, or workflow services.

Separate five responsibilities. The model interprets intent and proposes a tool call. A schema defines permitted arguments. A policy layer authorizes the operation. The executor performs it and returns a typed result. The model then uses that result to answer or choose another step. The system must never treat generated words such as "the email was sent" as proof that the executor confirmed success. Tool identity, call ID, arguments, status, timestamps, and

result provenance must remain explicit.

Agents extend this loop over several steps, maintaining state and choosing whether to reason, retrieve, act, verify, ask for clarification, or stop. This adds capability and new failure modes: wrong-tool selection, invalid arguments, unauthorized side effects, duplicated actions, prompt injection in tool output, loops, stale state, and fabricated results. The core lesson is that an LLM can propose actions, but a trustworthy system constrains, executes, observes, and verifies them outside the model.

Part 2 - Mathematical form and reasoning

At state s_t, the model proposes an action a_t from text or a finite tool set:

$$a_t \sim \pi_\theta(a \mid s_t), \qquad o_t = \mathcal{E}(a_t), \qquad s_{t+1} = U(s_t, a_t, o_t).$$

$\mathcal{E}$ is the external executor and o_t is an observation, not a model belief. A policy gate $g(s_t, a_t) \in \{0, 1\}$ must precede execution:

$$o_t = \begin{cases} \mathcal{E}(a_t), & g(s_t, a_t) = 1, \\ \text{denied}, & g(s_t, a_t) = 0. \end{cases}$$

Success requires task correctness, valid authorization, and confirmed side effects. Fluent final text cannot substitute for any of these predicates.

Part 3 - Hints and Helper

1. Draw the boundary between model proposal, policy decision, and tool execution.
2. Use typed schemas for every tool and reject additional fields.
3. Distinguish read-only tools from tools with external side effects.
4. Attach call IDs and provenance to observations.
5. Require confirmation for costly, destructive, or person-directed actions.
6. Make retries idempotent or explicitly guarded.
7. Treat tool output as untrusted data that may contain injected instructions.
8. Verify completion from executor state before reporting success.

Part 4 - Python code skeleton

```
from dataclasses import dataclass
from enum import Enum
from typing import Dict


class ActionStatus(Enum):
    PROPOSED = "proposed"
    DENIED = "denied"
```

```
    SUCCEEDED = "succeeded"
    FAILED = "failed"


@dataclass
class ToolCall:
    call_id: str
    tool_name: str
    arguments: Dict[str, object]


@dataclass
class ToolObservation:
    call_id: str
    status: ActionStatus
    payload: Dict[str, object]
    provenance: str


def execute_proposed_call(call, registry, policy, context):
    if call.tool_name not in registry:
        return ToolObservation(
            call.call_id, ActionStatus.DENIED, {}, "unknown_tool"
        )
    if not policy.authorize(call, context):
        return ToolObservation(
            call.call_id, ActionStatus.DENIED, {}, "policy_denial"
        )
    tool = registry[call.tool_name]
    validated_arguments = tool.schema.validate(call.arguments)
    try:
        payload = tool.execute(validated_arguments)
        status = ActionStatus.SUCCEEDED
    except Exception as exc:
        payload = {"error_type": type(exc).__name__}
        status = ActionStatus.FAILED
    return ToolObservation(call.call_id, status, payload, tool.provenance)
```

Part 5 - The Philosophy Behind the Exercise

Language can represent an action without causing it. Agent engineering begins by refusing to confuse the symbol with the event. A promise, command, and report of success are different objects even when they use similar words.

Tools give models contact with the world, while boundaries give that contact meaning and safety. Capability arises from the loop; accountability arises from keeping every transition observable and externally verifiable.

18.10.2 Exercise 01

Exercise Title: Implement Retrieval-Augmented Generation Pipeline Conceptually

Part 1 - Why this matters

RAG answers questions using an external corpus whose documents can be updated, inspected, and cited. A complete pipeline contains ingestion, parsing, chunking, metadata extraction, embedding, indexing, query transformation, retrieval, reranking, context assembly, grounded generation, citation validation, and evaluation. Treating RAG as merely "put search results in the prompt" hides where errors originate.

Create an offline ingestion path and an online query path. During ingestion, preserve document ID, source, version, effective date, permissions, section, chunk offsets, and checksum. During querying, enforce user access before results enter the prompt. Retrieve candidates, rerank them, remove duplicates, respect a token budget, and keep source labels stable. Instruct the generator to use only the supplied evidence and abstain when it is insufficient.

Evaluate stages separately. Retrieval recall asks whether supporting evidence was found. Context precision asks how much retrieved text was useful. Grounding asks whether claims follow from context. Answer quality asks whether the response solves the task. Citation correctness asks whether references support their attached claims. Save traces that connect every final claim to retrieved chunk IDs; otherwise the pipeline cannot be debugged.

Part 2 - Mathematical form and reasoning

For query q, candidate retrieval and reranking are

$$C_k(q) = \mathrm{TopK}_{d \in \mathcal{D}}\, s_e(q, d), \qquad D_r(q) = \mathrm{TopR}_{d \in C_k(q)}\, s_r(q, d).$$

Generation becomes $p_\theta(y \mid q, D_r(q))$. End-to-end correctness can be decomposed as

$$P(Y) = P(R)P(Y \mid R) + P(\neg R)P(Y \mid \neg R),$$

where R means sufficient evidence was retrieved. The decomposition makes clear that good generation cannot recover evidence that never reached the context.

Part 3 - Hints and Helper

1. Version corpus, chunker, embedding model, index, reranker, and prompt.
2. Preserve access-control metadata and filter before prompt construction.
3. Evaluate retriever with known supporting chunk IDs.
4. Compare realistic retrieval with oracle evidence.
5. Enforce a context token budget and stable citation labels.
6. Require evidence-aware abstention for no-answer items.
7. Validate citation entailment after generation.
8. Store an end-to-end trace for every request.

Part 4 - Python code skeleton

```
from dataclasses import dataclass
from typing import Dict, List


@dataclass
class Chunk:
    chunk_id: str
    document_id: str
    text: str
    metadata: Dict[str, str]
    score: float = 0.0


def build_context(chunks, maximum_characters):
    selected = []
    used = 0
    for chunk in chunks:
        label = "[" + chunk.chunk_id + "] "
        entry = label + chunk.text
        if used + len(entry) > maximum_characters:
            continue
        selected.append(entry)
        used += len(entry)
    return "\n\n".join(selected)


def rag_answer(query, user, retriever, reranker, generator):
    candidates = retriever.search(query, top_k=30, user=user)
    authorized = []
    for chunk in candidates:
        if chunk.metadata["authorized"] == "true":
            authorized.append(chunk)
    ranked = reranker.rank(query, authorized)
    context = build_context(ranked, maximum_characters=12000)
    prompt = (
        "Use only the evidence. Cite chunk IDs. "
        "If insufficient, say I don't know.\n\n"
        + context + "\n\nQuestion: " + query
    )
    answer = generator.generate(prompt, temperature=0.0)
    return {"answer": answer, "chunks": ranked, "prompt": prompt}
```

Part 5 - The Philosophy Behind the Exercise

RAG is an architecture of epistemic traceability. It moves part of the system's knowledge from opaque parameters into documents whose origin, date, and access rules can be inspected.

That does not guarantee truth, but it makes failure locatable.

The pipeline succeeds when it can explain not only an answer, but the chain by which evidence became available and was transformed into that answer.

18.10.3 Exercise 02

Exercise Title: Integrate Vector Database Retrieval Into Prompt Construction

Part 1 - Why this matters

A vector database stores embeddings and metadata so semantically related chunks can be retrieved even when query words differ from document words. Integration requires more than calling similarity search. The query and document embeddings must come from compatible models and normalization rules; metadata filters must enforce tenant, date, language, and permission constraints; results must be deduplicated, budgeted, and labeled before entering the prompt.

Design a retrieval adapter independent of a specific database. It should accept a query, authorization scope, top-k, score threshold, namespace, and filters, then return typed chunks with similarity, source, version, offsets, and permissions. Validate embedding dimension and index identity at startup. Apply authorization inside the retrieval query rather than retrieving forbidden chunks and discarding them after exposure.

Prompt construction must preserve evidence boundaries. Do not concatenate untrusted chunks as instructions; delimit them as quoted data and state that instructions inside evidence are not authoritative. Order by reranked relevance or source structure, measure lost-in-the-middle effects, and reject context that exceeds the token budget. Record which chunks were omitted and why.

Part 2 - Mathematical form and reasoning

For normalized query vector u and chunk vector v_i, cosine similarity is

$$s_i = u^\top v_i, \qquad \|u\|_2 = \|v_i\|_2 = 1.$$

Filtered retrieval is

$$D_k = \mathrm{TopK}_{i:\, F(m_i)=1}\, s_i,$$

where $F(m_i)$ enforces metadata and access conditions. Context selection under token budget B resembles a constrained utility problem:

$$\max_S \sum_{i \in S} r_i \quad \text{s.t.} \quad \sum_{i \in S} t_i \le B.$$

Redundancy penalties may be added so near-duplicate chunks do not consume the entire budget.

Part 3 - Hints and Helper

1. Verify embedding model ID, dimension, normalization, and distance metric.

2. Push authorization and tenant filters into the database query.
3. Return typed records with source, version, offsets, and scores.
4. Deduplicate overlapping chunks before context packing.
5. Count tokens with the generator's tokenizer.
6. Delimit retrieved text as untrusted evidence, not instructions.
7. Log selected and omitted chunk IDs with reasons.
8. Unit-test empty results, stale indexes, wrong dimensions, and permission denial.

Part 4 - Python code skeleton

```
from dataclasses import dataclass
from typing import Dict, List


@dataclass
class SearchRecord:
    chunk_id: str
    text: str
    score: float
    metadata: Dict[str, str]


def retrieve_for_user(vector_db, embedder, query, user_scope, top_k):
    query_vector = embedder.encode(query)
    if len(query_vector) != vector_db.embedding_dimension:
        raise ValueError("embedding dimension mismatch")
    filters = {
        "tenant_id": user_scope.tenant_id,
        "access_level": {"$lte": user_scope.access_level},
        "active": True,
    }
    rows = vector_db.search(
        vector=query_vector,
        top_k=top_k,
        filters=filters,
    )
    records = []
    for row in rows:
        record = SearchRecord(
            chunk_id=row["chunk_id"],
            text=row["text"],
            score=row["score"],
            metadata=row["metadata"],
        )
```

```
        records.append(record)
    return records


def evidence_block(records):
    lines = ["BEGIN UNTRUSTED EVIDENCE"]
    for record in records:
        lines.append("[" + record.chunk_id + "]")
        lines.append(record.text)
    lines.append("END UNTRUSTED EVIDENCE")
    return "\n".join(lines)
```

Part 5 - The Philosophy Behind the Exercise

Similarity is a mathematical relation, not permission, authority, or truth. Vector retrieval finds nearby representations; system design must still decide which sources are allowed, current, and worthy of entering the model's context.

Prompt construction is therefore an act of curation. It determines what the model is permitted to treat as evidence and preserves the distinction between data received and instructions trusted.

18.10.4 Exercise 03

Exercise Title: Implement Tool Calling Logic and Validate Structured Outputs

Part 1 - Why this matters

Tool calling converts a model proposal into structured data that software can validate. Without a strict schema, a model may omit required fields, invent argument names, use the wrong type, select a nonexistent tool, or place prose where an executor expects JSON. Parsing success is only the first gate; semantic validity and authorization must follow before execution.

Define a registry with stable tool names, descriptions, input schemas, output schemas, side-effect classification, timeout, idempotency behavior, and policy requirements. Ask the model either for a final response or one structured call. Parse with a real JSON parser, reject extra properties, validate enums and ranges, normalize only safe representations, and never repair an ambiguous high-impact action silently. Return validation errors to a bounded correction loop or ask the user for clarification.

After execution, validate the result against its output schema and bind it to the original call ID. The model must not substitute a guessed result when a tool fails. Test correct calls and malformed, incomplete, adversarial, and unauthorized calls. Keep the executor unreachable except through the validated dispatcher.

Part 2 - Mathematical form and reasoning

Let z be raw model output, $P(z)$ a parser, V schema validation, and A authorization. Execution is permitted only if

$$G(z) = I(P(z) \text{ succeeds}) I(V(P(z)) = 1) I(A(P(z)) = 1) = 1.$$

For a bounded correction loop with maximum K attempts, termination is

$$T = \min\{t : G(z_t) = 1 \ \vee \ t = K\}.$$

The maximum prevents malformed outputs from causing unbounded agent loops.

Part 3 - Hints and Helper

1. Use strict JSON schemas with required fields and no extra properties.
2. Validate tool name before validating arguments.
3. Check semantic ranges, identifiers, permissions, and resource ownership.
4. Require confirmation for side-effecting calls.
5. Bind results to call IDs and validate output schemas.
6. Limit correction attempts and preserve every failed proposal.
7. Keep execution behind one dispatcher.
8. Fuzz schemas with wrong types, Unicode, huge values, and extra fields.

Part 4 - Python code skeleton

```
from dataclasses import dataclass
import json


@dataclass
class RegisteredTool:
    name: str
    input_validator: object
    output_validator: object
    executor: object
    has_side_effects: bool


def parse_call(raw_text):
    value = json.loads(raw_text)
    if not isinstance(value, dict):
        raise ValueError("tool call must be a JSON object")
    allowed = {"tool_name", "arguments", "call_id"}
    for key in value.keys():
        if key not in allowed:
            raise ValueError("unexpected field: " + key)
    for required in allowed:
        if required not in value:
            raise ValueError("missing field: " + required)
    return value
```

```
def dispatch(raw_text, registry, policy, context):
    call = parse_call(raw_text)
    name = call["tool_name"]
    if name not in registry:
        raise ValueError("unknown tool")
    tool = registry[name]
    arguments = tool.input_validator.validate(call["arguments"])
    if not policy.authorize(name, arguments, context):
        raise PermissionError("tool call denied")
    if tool.has_side_effects and not context.confirmed:
        return {"status": "confirmation_required"}
    output = tool.executor(arguments)
    validated_output = tool.output_validator.validate(output)
    return {
        "call_id": call["call_id"],
        "status": "succeeded",
        "output": validated_output,
    }
```

Part 5 - The Philosophy Behind the Exercise

Structure is a contract between probabilistic language and deterministic software. The model may propose many forms; the schema narrows those forms to objects that the surrounding system can reason about and constrain.

Validation is not distrust of intelligence. It is recognition that agency requires accountable interfaces. The more consequential the tool, the less room there is for implicit interpretation.

18.10.5 Exercise 04

Exercise Title: Debug Tool Invocation Failures and Hallucinated Tool Calls

Part 1 - Why this matters

Tool failures appear at different boundaries: the model chooses the wrong tool, emits malformed arguments, calls a tool that does not exist, violates policy, times out, receives a partial result, misreads the result, retries a completed side effect, or claims success without execution. Treating them all as "the agent failed" prevents targeted fixes and may hide dangerous duplication.

Capture a complete trace: user request, system/tool descriptions, model proposal, parse result, schema errors, authorization decision, executor request, response, timeout, retry metadata, observation given back to the model, and final answer. Assign the earliest failed boundary. Reproduce with a deterministic stub tool, then vary one component at a time. For side effects, use idempotency keys and query operation status before retrying.

Hallucinated calls include nonexistent tool names, unsupported capabilities, or invented results. The dispatcher must reject them mechanically. Test tool description ambiguity, similar

names, missing tools, adversarial tool output, and transient errors. A successful fix reduces the targeted class without causing excessive refusal or moving the failure downstream.

Part 2 - Mathematical form and reasoning

Represent the pipeline as gates $G_1, \ldots, G_m$. The root cause is the first failed gate,

$$k^* = \min\{k : G_k = 0\}.$$

For retry attempt r with success probability p_r and side-effect duplication cost C_D, naive retry is unsafe when

$$P(\text{first attempt succeeded but response was lost})C_D > P(\text{transient failure})\, B_R,$$

where B_R is retry benefit. Idempotency or status lookup changes this risk.

Part 3 - Hints and Helper

1. Preserve the exact trace before modifying prompts or schemas.
2. Label the earliest failure boundary.
3. Replace external tools with deterministic stubs for reproduction.
4. Distinguish safe read retries from side-effect retries.
5. Use idempotency keys and operation-status lookup.
6. Reject nonexistent tools and fabricated results outside the model.
7. Inject timeout, malformed response, denial, and partial-success cases.
8. Add every confirmed bug to a regression suite.

Part 4 - Python code skeleton

```
from dataclasses import dataclass
from typing import List


@dataclass
class InvocationTrace:
    call_id: str
    tool_exists: bool
    parsed: bool
    schema_valid: bool
    authorized: bool
    executor_started: bool
    executor_succeeded: bool
    result_valid: bool
    final_claimed_success: bool
```

```
    events: List[str]


def earliest_failure(trace):
    checks = [
        ("unknown_tool", trace.tool_exists),
        ("parse_failure", trace.parsed),
        ("schema_failure", trace.schema_valid),
        ("authorization_denial", trace.authorized),
        ("executor_not_started", trace.executor_started),
        ("executor_failure", trace.executor_succeeded),
        ("invalid_result", trace.result_valid),
    ]
    for name, passed in checks:
        if not passed:
            return name
    if trace.final_claimed_success and not trace.executor_succeeded:
        return "hallucinated_success"
    return "no_failure_detected"


def should_retry(operation, error, status_lookup):
    if not error.transient:
        return False
    if not operation.has_side_effects:
        return True
    existing_status = status_lookup(operation.idempotency_key)
    if existing_status == "succeeded":
        return False
    return operation.supports_idempotency
```

Part 5 - The Philosophy Behind the Exercise

An agent trace is a causal history. Debugging becomes possible when proposals, decisions, events, and reports are not blended into one narrative. The first broken contract matters more than the final symptom.

Reliable agency depends on modest claims: the system should report only what an authorized executor can confirm. Anything else is imagination wearing the grammar of action.

18.10.6 Exercise 05

Exercise Title: Compare RAG vs Fine-Tuning for Knowledge Updates

Part 1 - Why this matters

RAG and fine-tuning are often proposed as interchangeable ways to "teach" a model, but they place knowledge in different locations. RAG reads versioned documents at inference time and can expose citations. Fine-tuning changes model behavior through parameter updates; it is useful for style, format, procedure, and recurring task patterns, but it does not provide simple

deletion, provenance, or rapid correction of changing facts.

Build base, RAG, fine-tuned, and hybrid conditions on matched tasks. Include stable procedures, static domain knowledge, newly changed facts, no-answer questions, and conflicting versions. Measure answer accuracy, unsupported claims, citation precision, format adherence, calibration, update freshness, latency, token cost, training cost, index maintenance, and forgetting on general tasks. Record data provenance and contamination risk.

Perform an update experiment: change a policy document after the initial build, refresh the RAG index, and compare the effort and delay required for each system to answer according to the new version. Do not conclude that RAG is superior when retrieval is poorly engineered, or that fine-tuning stores a fact because one prompted example was remembered.

Part 2 - Mathematical form and reasoning

RAG conditions on evidence $D(x)$ while fine-tuning changes parameters:

$$y_{\text{RAG}} \sim p_\theta(y \mid x, D(x)), \qquad \theta' = \arg\min_\theta \sum_{(x,y)\in\mathcal{T}} \mathcal{L}_\theta(x, y).$$

Compare lifecycle cost over horizon T:

$$C(T) = C_{\text{build}} + C_{\text{refresh}}(T) + C_{\text{serve}}(T) + C_{\text{review}}(T).$$

Freshness lag $L_f = t_{\text{correct availability}} - t_{\text{source change}}$ is a first-class metric for dynamic knowledge.

Part 3 - Hints and Helper

1. Define whether the requirement concerns facts, behavior, style, or procedure.
2. Use matched base, RAG, fine-tuned, and hybrid conditions.
3. Add post-build fact changes and deletion requests.
4. Evaluate retrieval oracle and realistic retrieval separately.
5. Test general-task regression after fine-tuning.
6. Measure provenance, freshness, latency, and lifecycle cost.
7. Record data and index versions.
8. Recommend a knowledge location per requirement, not one global winner.

Part 4 - Python code skeleton

```
from dataclasses import dataclass


@dataclass
class KnowledgeCondition:
    name: str
    accuracy: float
```

```
    hallucination_rate: float
    citation_precision: float
    freshness_hours: float
    format_adherence: float
    build_cost: float
    monthly_refresh_cost: float
    monthly_serve_cost: float


def total_cost(condition, months):
    return (
        condition.build_cost
        + months * condition.monthly_refresh_cost
        + months * condition.monthly_serve_cost
    )


def feasible(condition, requirements):
    if condition.accuracy < requirements["minimum_accuracy"]:
        return False
    if condition.hallucination_rate > requirements["maximum_hallucination"]:
        return False
    if condition.freshness_hours > requirements["maximum_freshness_hours"]:
        return False
    return True


def select_condition(conditions, requirements, months):
    candidates = []
    for condition in conditions:
        if feasible(condition, requirements):
            candidates.append(condition)
    if len(candidates) == 0:
        return None
    selected = candidates[0]
    for candidate in candidates[1:]:
        if total_cost(candidate, months) < total_cost(selected, months):
            selected = candidate
    return selected
```

Part 5 - The Philosophy Behind the Exercise

Knowledge architecture asks where a claim should live and how it should change. Some knowledge belongs in inspectable sources; some behavior belongs in learned parameters; some high-risk decisions should remain outside the model entirely.

The exercise rejects the fantasy that every form of learning is the same. A system becomes maintainable when facts, skills, evidence, and authority have deliberate homes.

18.10.7 Exercise 06

Exercise Title: Implement Memory Component for Multi-Turn Agents

Part 1 - Why this matters

Multi-turn agents need continuity, but "memory" refers to several different stores. Working memory contains the current task state. Episodic memory records prior interactions and outcomes. Semantic memory stores durable user or domain facts. Procedural memory contains policies and workflows. Keeping the entire conversation forever is expensive, privacy-sensitive, and vulnerable to stale or malicious instructions.

Create a memory service outside the model with typed entries, subject, source, timestamp, confidence, scope, permissions, expiry, and supersession links. Write only after policy checks; do not promote every model statement into durable truth. Retrieve memory by task relevance and authorization, then place it in a clearly delimited context. Conflicting entries should be surfaced or resolved by recency and authority rules rather than silently blended.

Support correction, deletion, session isolation, retention limits, and audit. Summaries must link to their source turns so compression errors remain traceable. Test cross-user leakage, prompt injection stored as memory, stale preferences, duplicate facts, and context-budget pressure. Evaluate task success, retrieval precision, stale-memory error, privacy violations, and token savings.

Part 2 - Mathematical form and reasoning

Score candidate memory m_i for query q using relevance, recency, authority, and risk:

$$S(m_i, q) = \alpha R_i + \beta e^{-\lambda \Delta t_i} + \gamma A_i - \delta K_i.$$

Select memories under token budget B and authorization predicate P_i:

$$\max_{M} \sum_{i \in M} S(m_i, q) \quad \text{s.t.} \quad \sum_{i \in M} t_i \leq B,\ P_i = 1.$$

Source authority and explicit correction should override mere semantic similarity.

Part 3 - Hints and Helper

1. Separate working, episodic, semantic, and procedural memory.
2. Store provenance, timestamp, scope, authority, and expiry.
3. Require policy approval before durable writes.
4. Keep user and tenant namespaces isolated.
5. Link summaries to source turns and support correction/deletion.
6. Treat retrieved memory as untrusted data.
7. Resolve conflicts explicitly and log supersession.
8. Test stale memory, injection, leakage, and token-budget behavior.

Part 4 - Python code skeleton

```
from dataclasses import dataclass
from datetime import datetime
from typing import Dict, List


@dataclass
class MemoryEntry:
    memory_id: str
    tenant_id: str
    subject_id: str
    kind: str
    content: str
    source_uri: str
    created_at: str
    expires_at: str
    confidence: float
    authority: float
    superseded_by: str = ""


class MemoryStore:
    def __init__(self):
        self.entries: Dict[str, MemoryEntry] = {}

    def write(self, entry, policy, context):
        if not policy.allow_memory_write(entry, context):
            raise PermissionError("memory write denied")
        self.entries[entry.memory_id] = entry

    def retrieve(self, tenant_id, subject_id, now):
        selected = []
        for entry in self.entries.values():
            if entry.tenant_id != tenant_id:
                continue
            if entry.subject_id != subject_id:
                continue
            if entry.superseded_by != "":
                continue
            expiry = datetime.fromisoformat(entry.expires_at)
            if expiry <= now:
                continue
            selected.append(entry)
        return selected

    def delete(self, memory_id):
        if memory_id in self.entries:
            del self.entries[memory_id]
```

Part 5 - The Philosophy Behind the Exercise

Memory creates continuity by allowing the past to influence the present. That power makes forgetting equally important. A system unable to correct, expire, or delete memory confuses persistence with truth and convenience with consent.

Agent memory should be curated history, not sediment. Its value comes from provenance, boundaries, and the possibility of revision.

18.10.8 Exercise 07

Exercise Title: Evaluate Agent Planning Failures and Looping Behavior

Part 1 - Why this matters

Multi-step agents can decompose goals, but each added step creates another chance to choose the wrong action, lose constraints, repeat work, or fail to stop. Common planning failures include impossible goals, missing prerequisites, premature execution, stale plans after observations change, repeated identical calls, alternating between two states, unbounded research, and declaring success without testing the terminal condition.

Instrument the plan-act-observe loop. Record state hash, step number, proposed goal, selected tool, normalized arguments, observation, cost, remaining budget, and termination reason. Define success and stop conditions outside the model. Use maximum steps, wall time, token/tool budgets, repeated-state detection, and progress tests. A budget stop is a safe outcome, not an invitation to fabricate completion.

Build tasks with necessary detours, unavailable tools, misleading observations, partial success, and unsatisfiable requirements. Measure task success, steps, unnecessary calls, repeated states, recovery after error, cost, and false-success rate. Compare static plans with plans revised after each observation.

Part 2 - Mathematical form and reasoning

Let progress potential be $\Phi(s_t)$. A step has progress

$$\Delta_t = \Phi(s_{t+1}) - \Phi(s_t).$$

Flag stagnation when $\Delta_t \leq \epsilon$ for K consecutive steps. Exact loop detection uses a normalized state/action hash:

$$L_t = I\left(H(s_t, a_t) \in \{H(s_j, a_j) : j < t\}\right).$$

Termination occurs on verified success, verified impossibility, user clarification, policy denial, or exhausted budget. Only verified success may be reported as completion.

Part 3 - Hints and Helper

1. Define external success and terminal predicates for every task.
2. Record normalized state and action hashes.
3. Enforce step, time, token, tool, and cost budgets.

4. Detect repeated calls and low-progress windows.
5. Replan after observations invalidate assumptions.
6. Test unavailable tools and impossible goals.
7. Distinguish safe stop, failure, and verified success.
8. Turn looping traces into regression scenarios.

Part 4 - Python code skeleton

```
from dataclasses import dataclass, field
import hashlib
import json


@dataclass
class AgentBudget:
    maximum_steps: int
    maximum_tool_calls: int
    maximum_cost: float


@dataclass
class AgentState:
    step: int = 0
    tool_calls: int = 0
    cost: float = 0.0
    seen_hashes: set = field(default_factory=set)


def action_hash(tool_name, arguments):
    payload = json.dumps(arguments, sort_keys=True)
    value = tool_name + ":" + payload
    return hashlib.sha256(value.encode("utf-8")).hexdigest()


def guard_next_action(state, budget, tool_name, arguments, cost):
    if state.step >= budget.maximum_steps:
        return "stop_step_budget"
    if state.tool_calls >= budget.maximum_tool_calls:
        return "stop_tool_budget"
    if state.cost + cost > budget.maximum_cost:
        return "stop_cost_budget"
    fingerprint = action_hash(tool_name, arguments)
    if fingerprint in state.seen_hashes:
        return "stop_repeated_action"
    state.seen_hashes.add(fingerprint)
```

```
    state.step += 1
    state.tool_calls += 1
    state.cost += cost
    return "continue"
```

Part 5 - The Philosophy Behind the Exercise

Planning is not the production of a plausible list; it is the disciplined revision of action under evidence. An agent that cannot recognize stagnation may turn persistence into waste and uncertainty into false completion.

Stopping is part of intelligence. It preserves the distinction between a goal that has been achieved, one that remains open, and one that cannot responsibly be pursued with available authority and resources.

18.10.9 Exercise 08

Exercise Title: Implement Guardrails for Tool Safety and Input Validation

Part 1 - Why this matters

Tool-enabled agents can affect databases, files, accounts, communications, and infrastructure. Safety cannot depend on the model remembering a warning. Build guardrails as deterministic controls surrounding every call: identity, authentication, authorization, schema validation, semantic validation, resource ownership, rate and cost limits, confirmation, sandboxing, timeout, network restrictions, audit, and response filtering.

Classify tools by side effects and consequence. Read-only search may run automatically within access scope. Financial transfer, deletion, deployment, or person-directed messaging requires explicit user intent and often confirmation. Use least-privilege credentials per tool, not one agent-wide secret. Reject path traversal, arbitrary SQL, shell metacharacters, unbounded dates, huge payloads, and identifiers outside the caller's tenant.

Treat user input, retrieved text, memory, and tool output as untrusted. Prompt injection inside a webpage must not grant authority. Redact secrets before returning observations to the model. Use dry-run previews and idempotency keys for side effects. Test allowed, denied, boundary, malformed, replayed, and concurrent cases; log decisions without storing unnecessary sensitive data.

Part 2 - Mathematical form and reasoning

Execution permission is a conjunction:

$$G = I_{\text{authenticated}} I_{\text{schema}} I_{\text{authorized}} I_{\text{owned}} I_{\text{budget}} I_{\text{confirmed}}.$$

Any required zero denies the call. Risk-based control may use

$$R = P(\text{harm} \mid a, s) \times S(\text{harm}) \times E(a),$$

but high-impact forbidden actions remain hard constraints rather than weighted preferences that model utility can override.

Part 3 - Hints and Helper

1. Classify every tool by side effect, privilege, and maximum consequence.
2. Use strict schemas plus semantic and ownership validation.
3. Grant separate least-privilege credentials to each executor.
4. Require confirmation and preview for designated actions.
5. Sandbox code and restrict filesystem, network, time, and resources.
6. Redact secrets from tool observations and logs.
7. Use idempotency keys and replay protection.
8. Fuzz allowed and denied boundaries in automated tests.

Part 4 - Python code skeleton

```
from dataclasses import dataclass


@dataclass
class SafetyContext:
    authenticated: bool
    user_id: str
    tenant_id: str
    confirmed_call_ids: set
    remaining_cost: float


def authorize_call(call, tool, context):
    reasons = []
    if not context.authenticated:
        reasons.append("not_authenticated")
    if call.arguments.get("tenant_id") != context.tenant_id:
        reasons.append("tenant_mismatch")
    if tool.estimated_cost(call.arguments) > context.remaining_cost:
        reasons.append("cost_budget_exceeded")
    if tool.has_side_effects:
        if call.call_id not in context.confirmed_call_ids:
            reasons.append("confirmation_required")
    if not tool.ownership_check(call.arguments, context.user_id):
        reasons.append("resource_not_owned")
    return len(reasons) == 0, reasons


def safe_dispatch(call, registry, context, audit_log):
    if call.tool_name not in registry:
        return {"status": "denied", "reasons": ["unknown_tool"]}
    tool = registry[call.tool_name]
```

```
    arguments = tool.schema.validate(call.arguments)
    call.arguments = arguments
    allowed, reasons = authorize_call(call, tool, context)
    audit_log.record(call.call_id, call.tool_name, allowed, reasons)
    if not allowed:
        return {"status": "denied", "reasons": reasons}
    result = tool.execute(arguments)
    return {"status": "succeeded", "result": tool.redact(result)}
```

Part 5 - The Philosophy Behind the Exercise

Guardrails express a constitutional idea: intelligence does not imply authority. An agent may understand and propose an action while still lacking permission to perform it. The boundary protects users, systems, and the agent from ambiguous intent.

Safety becomes credible when it is enforced by mechanisms that do not depend on the same probabilistic process they constrain.

18.10.10 Exercise 09

Exercise Title: Benchmark End-to-End RAG Latency and Accuracy

Part 1 - Why this matters

RAG latency is the sum of many stages: query preprocessing, embedding, network round trip, vector search, metadata filtering, reranking, context packing, prompt prefill, decoding, citation validation, and queueing. A fast vector search does not guarantee a fast answer, and a fast answer is useless if the supporting evidence was missed. Benchmark latency and accuracy together under a representative workload.

Use a frozen query set with known supporting chunks and no-answer cases. Record corpus/index version, cache state, query length, retrieved k, reranked r, context tokens, output tokens, concurrency, hardware, and network placement. Warm components, then measure cold and warm paths separately. Instrument every stage with one trace ID and use server-side clocks where possible.

Report retrieval recall@k, mean reciprocal rank, context precision, and answer accuracy. Also measure unsupported claims, citation precision, abstention, latency percentiles, throughput, and cost. Sweep k, reranker depth, chunk size, and context budget to find a Pareto frontier. Include timeouts and errors in accounting; discarding them creates a fictional service.

Part 2 - Mathematical form and reasoning

End-to-end latency is

$$T = T_e + T_v + T_r + T_c + T_p + T_d + T_{\text{verify}} + T_q.$$

Retrieval recall@k is

$$R@k = \frac{1}{N} \sum_i I(D_k(q_i) \cap G_i \neq \varnothing),$$

where G_i is the gold support set. Select configurations from the non-dominated set over answer quality, p99 latency, cost, and memory instead of optimizing one metric alone.

Part 3 - Hints and Helper

1. Freeze queries, gold evidence, corpus, index, model, and decoding.
2. Trace embedding, search, rerank, packing, prefill, decode, and verification.
3. Measure cold and warm cache paths.
4. Sweep retrieval depth and context budget.
5. Report retrieval and generation metrics separately and end to end.
6. Stratify by query and document length.
7. Include timeout, empty retrieval, and error outcomes.
8. Repeat runs and report percentile distributions.

Part 4 - Python code skeleton

```
from dataclasses import dataclass, field
from time import perf_counter


@dataclass
class StageTimer:
    durations_ms: dict = field(default_factory=dict)

    def measure(self, name, function, *arguments, **keywords):
        started = perf_counter()
        result = function(*arguments, **keywords)
        elapsed = (perf_counter() - started) * 1000.0
        self.durations_ms[name] = elapsed
        return result


def benchmark_query(query, pipeline):
    timer = StageTimer()
    vector = timer.measure("embedding", pipeline.embed, query.text)
    candidates = timer.measure(
        "vector_search", pipeline.search, vector, query.filters
    )
    ranked = timer.measure(
        "reranking", pipeline.rerank, query.text, candidates
    )
    prompt = timer.measure(
        "context_packing", pipeline.pack, query.text, ranked
```

```
)
answer = timer.measure("generation", pipeline.generate, prompt)
verified = timer.measure("verification", pipeline.verify, answer, ranked)
total_ms = 0.0
for duration in timer.durations_ms.values():
    total_ms += duration
return {
    "query_id": query.query_id,
    "answer": answer,
    "verified": verified,
    "stage_ms": timer.durations_ms,
    "measured_total_ms": total_ms,
}
```

Part 5 - The Philosophy Behind the Exercise

End-to-end measurement respects the fact that users experience a system, not a component. Local optimizations can move cost across boundaries or improve speed while degrading evidence. Only the complete path reveals the delivered value.

The Pareto view is equally important: reliability, latency, and cost are real constraints that must be negotiated openly rather than hidden inside one score.

18.10.11 Exercise 10

Exercise Title: Integrate SQL Queries as Tools and Validate Correct Query Generation

Part 1 - Why this matters

SQL tools give an agent access to exact, current, structured data, but generated queries can expose private rows, join incorrectly, omit time filters, scan huge tables, or modify production state. A safe design does not grant the model a raw database credential. It exposes a read-only query service with an approved schema, row-level authorization, resource limits, and deterministic validation.

Provide the model with selected table and column descriptions, relationships, units, time semantics, and allowed operations. Prefer parameterized templates or an abstract query representation for high-risk systems. If free SQL is accepted, parse it into an AST; reject multiple statements, comments used for bypass, DDL/DML, forbidden functions, unapproved schemas, missing tenant filters, unbounded results, and excessive estimated cost. Execute with a read-only role, timeout, row limit, and isolated replica.

Validate correctness on a dataset with known answers, including joins, aggregation, nulls, dates, duplicate keys, ambiguous names, and empty results. Compare execution result, not string equality, while also checking that the query respects policy. Return typed rows plus schema and truncation status. The agent must not invent missing rows or treat an execution error as an empty set.

Part 2 - Mathematical form and reasoning

Query acceptance is

$$G(q) = I_{\text{parse}} I_{\text{read only}} I_{\text{schema}} I_{\text{row policy}} I_{\text{cost}} I_{\text{bounded}}.$$

Execution accuracy on test i can compare canonicalized result multisets:

$$A = \frac{1}{N} \sum_{i} I(\mathcal{R}(q_i) = \mathcal{R}(q_i^*)).$$

When ordering is required, compare ordered sequences; otherwise do not mark a semantically correct query wrong because row order differs.

Part 3 - Hints and Helper

1. Use a read-only replica and least-privilege database role.
2. Expose only approved schema metadata.
3. Parse SQL into an AST; never validate with keywords alone.
4. Enforce tenant predicates, row limits, timeout, and cost budget.
5. Parameterize user values instead of interpolating text.
6. Test joins, nulls, dates, duplicates, empty sets, and truncation.
7. Compare executed results and policy validity.
8. Distinguish query error, no rows, and partial/truncated result.

Part 4 - Python code skeleton

```
from dataclasses import dataclass


@dataclass
class QueryRequest:
    sql: str
    parameters: dict
    tenant_id: str
    maximum_rows: int


def validate_query(parsed_query, request, policy):
    errors = []
    if parsed_query.statement_type != "SELECT":
        errors.append("read_only_required")
    if len(parsed_query.statements) != 1:
        errors.append("single_statement_required")
    for table_name in parsed_query.tables:
        if table_name not in policy.allowed_tables:
            errors.append("forbidden_table:" + table_name)
    if not parsed_query.has_tenant_predicate("tenant_id"):
```

```
        errors.append("missing_tenant_filter")
    if request.maximum_rows > policy.maximum_rows:
        errors.append("row_limit_exceeded")
    if parsed_query.estimated_cost > policy.maximum_cost:
        errors.append("query_cost_exceeded")
    return errors


def execute_sql_tool(request, parser, policy, connection):
    parsed = parser.parse(request.sql)
    errors = validate_query(parsed, request, policy)
    if len(errors) > 0:
        return {"status": "denied", "errors": errors}
    rows = connection.execute_read_only(
        request.sql,
        request.parameters,
        timeout_seconds=policy.timeout_seconds,
        maximum_rows=request.maximum_rows,
    )
    return {"status": "succeeded", "rows": rows, "truncated": False}
```

Part 5 - The Philosophy Behind the Exercise

Natural language is flexible; databases are literal. SQL integration joins those worlds at a boundary where ambiguity can become data exposure or a wrong decision. The parser and policy make the boundary explicit.

The model should be allowed to ask a precise question of the database, not to inherit the database's authority. Access remains a governed capability.

18.10.12 Exercise 11

Exercise Title: Integrate Vector DB Search and Measure Recall vs Latency Tradeoffs

Part 1 - Why this matters

Approximate nearest-neighbor indexes trade search effort for speed and memory. Increasing candidate depth, probe count, or ef-search can recover more relevant chunks while increasing latency. Larger top-k may improve evidence recall but also burdens reranking and context packing with distractors. Integration should select an operating point from measured end-to-end behavior.

Create a gold query set with one or more supporting chunk IDs. Benchmark exact search on a manageable subset to establish an upper bound, then sweep index parameters, top-k, filters, and concurrency. Record index type, distance metric, embedding model, dimension, corpus size, shard/replica layout, warm-up, cache state, CPU/GPU, and network placement.

Measure recall@k, MRR, nDCG where graded relevance exists, p50/p95/p99 search latency, throughput, index memory, reranker cost, and final answer quality. Test metadata filters because selective filters can change both recall and latency. Inspect queries whose gold evidence

remains far away even under exact search; their problem is representation or chunking, not ANN tuning.

Part 2 - Mathematical form and reasoning

Evidence recall is

$$R@k = \frac{1}{N} \sum_i I(D_k(q_i) \cap G_i \neq \varnothing).$$

Approximation recall relative to exact top-k is

$$R_{\text{ANN}}@k = \frac{1}{N} \sum_i \frac{|D_k^{\text{ANN}}(q_i) \cap D_k^{\text{exact}}(q_i)|}{k}.$$

Choose a non-dominated configuration satisfying minimum evidence recall and maximum tail latency rather than maximizing search recall without regard to the rest of RAG.

Part 3 - Hints and Helper

1. Build gold supporting-chunk labels and an exact-search reference subset.
2. Sweep ANN effort, top-k, filters, and concurrency.
3. Warm the index and measure cold behavior separately.
4. Report search and end-to-end RAG quality.
5. Include p99 latency and failures, not only mean latency.
6. Inspect embedding/chunking failures that tuning cannot repair.
7. Measure index memory and rebuild/update cost.
8. Select the Pareto operating point under service constraints.

Part 4 - Python code skeleton

```
from time import perf_counter


def evidence_recalled(result_ids, gold_ids):
    gold_set = set(gold_ids)
    for result_id in result_ids:
        if result_id in gold_set:
            return True
    return False


def benchmark_search(database, queries, configurations):
    rows = []
    for configuration in configurations:
        recalled = 0
```

```
        latencies = []
        for query in queries:
            started = perf_counter()
            results = database.search(
                vector=query.vector,
                top_k=configuration["top_k"],
                search_effort=configuration["search_effort"],
                filters=query.filters,
            )
            elapsed_ms = (perf_counter() - started) * 1000.0
            latencies.append(elapsed_ms)
            result_ids = []
            for result in results:
                result_ids.append(result["chunk_id"])
            if evidence_recalled(result_ids, query.gold_chunk_ids):
                recalled += 1
        ordered = sorted(latencies)
        p95_index = int(0.95 * (len(ordered) - 1))
        rows.append({
            "configuration": configuration,
            "evidence_recall": recalled / len(queries),
            "p95_latency_ms": ordered[p95_index],
        })
    return rows
```

Part 5 - The Philosophy Behind the Exercise

Search is a physical approximation to a semantic intention. More computation can explore the index more thoroughly, but it cannot guarantee that the embedding space represents relevance correctly.

The tradeoff teaches restraint: optimize the component only until it supports the system's evidence obligation. Beyond that point, additional recall may be expensive noise.

18.10.13 Exercise 12

Exercise Title: Implement Tool Result Verification to Prevent Hallucinated Answers

Part 1 - Why this matters

A successful tool call does not guarantee a correct final answer. The result may be partial, stale, malformed, truncated, internally inconsistent, or misinterpreted by the model. Conversely, when a tool fails, the model may fill the gap with a plausible value. Build a verification layer that establishes whether the observation is suitable for the claim the agent wants to make.

Validate transport status, call ID, output schema, units, ranges, timestamps, completeness, checksums, row counts, and provenance. For high-impact results, cross-check with an independent source or deterministic calculation. Bind claims to result fields and forbid the model from presenting an unverified value as confirmed. Distinguish unavailable, failed, partial, stale,

contradictory, and verified states.

Test adversarial results containing prompt injection, fake success fields, incorrect units, duplicated rows, missing pages, and inconsistent totals. The verifier should return structured evidence and reasons, not merely a boolean. If verification fails, retry safely, request clarification, escalate, or abstain; do not silently downgrade the evidence requirement.

Part 2 - Mathematical form and reasoning

For result o, verification is a conjunction of predicates:

$$V(o) = V_{\text{status}} V_{\text{schema}} V_{\text{fresh}} V_{\text{complete}} V_{\text{consistent}} V_{\text{provenance}}.$$

Claim c is supported only if required fields $F(c)$ are verified and an entailment rule holds:

$$S(c, o) = I(F(c) \subseteq F_{\text{verified}}(o)) I(o \models c).$$

Verification cost should scale with consequence; independent confirmation may be mandatory above a risk threshold.

Part 3 - Hints and Helper

1. Bind every observation to the originating call ID.
2. Validate schema, units, ranges, freshness, completeness, and provenance.
3. Represent partial and stale results explicitly.
4. Map final claims to verified fields.
5. Cross-check high-impact outputs independently.
6. Treat tool text as untrusted and strip embedded instructions.
7. Fail closed when required verification is unavailable.
8. Inject malformed, truncated, contradictory, and fake-success results.

Part 4 - Python code skeleton

```
from dataclasses import dataclass
from enum import Enum


class VerificationStatus(Enum):
    VERIFIED = "verified"
    PARTIAL = "partial"
    STALE = "stale"
    INVALID = "invalid"


@dataclass
```

```
class VerificationResult:
    status: VerificationStatus
    verified_fields: set
    reasons: list
    sanitized_payload: dict


def verify_tool_result(call, observation, schema, current_time):
    reasons = []
    verified_fields = set()
    if observation["call_id"] != call.call_id:
        reasons.append("call_id_mismatch")
    if observation["status"] != "succeeded":
        reasons.append("tool_did_not_succeed")
    payload = schema.sanitize(observation.get("payload", {}))
    validation_errors = schema.validate(payload)
    reasons.extend(validation_errors)
    if observation.get("expires_at") is not None:
        if observation["expires_at"] <= current_time:
            return VerificationResult(
                VerificationStatus.STALE, set(), ["expired"], payload
            )
    if len(reasons) > 0:
        return VerificationResult(
            VerificationStatus.INVALID, set(), reasons, payload
        )
    for field_name in payload.keys():
        verified_fields.add(field_name)
    return VerificationResult(
        VerificationStatus.VERIFIED, verified_fields, [], payload
    )
```

Part 5 - The Philosophy Behind the Exercise

Observation is not yet knowledge. It becomes usable knowledge when its identity, meaning, completeness, and relation to a claim have been checked. This is the same discipline applied in science, accounting, and reliable software.

Verification protects the boundary between what happened and what the agent wishes had happened. That boundary is the heart of truthful tool use.

18.10.14 Exercise 13

Exercise Title: Compare Single-Step vs Multi-Step Agent Reasoning

Part 1 - Why this matters

Single-step systems choose one tool or answer directly. They are fast, predictable, and easier to secure, but cannot solve tasks requiring sequential dependencies. Multi-step agents can retrieve, transform, verify, and revise, yet accumulate error, cost, latency, and loop risk.

Compare both on tasks whose dependency depth is known.

Create strata requiring zero, one, two, or several tool dependencies. Give both systems the same tools, permissions, evidence, and task budget. A single-step system should not be penalized on a task impossible within one action without making that limitation explicit. Measure task success, false success, tool accuracy, steps, redundant calls, recovery, latency, tokens, cost, and safety violations.

Test whether a planner adds value beyond repeated free-form generation. Require multi-step plans to update after observations and verify the final condition. Inspect cases where extra reasoning amplifies a false assumption. Recommend the least complex architecture that satisfies task depth and reliability gates; agentic complexity is a cost, not an achievement by itself.

Part 2 - Mathematical form and reasoning

If each step succeeds independently with probability p, a required d-step chain has approximate success

$$P_{\text{chain}} \approx p^d.$$

Verification and recovery can change this, while correlated errors make the independence assumption optimistic. Define net utility

$$U = A - \lambda_L L - \lambda_C C - \lambda_R R,$$

where A is task achievement, L latency, C resource cost, and R safety risk. Compare within task-depth strata rather than averaging incompatible tasks.

Part 3 - Hints and Helper

1. Label the minimum dependency depth of every task.
2. Give systems identical tools, permissions, and evidence.
3. Define externally verified success conditions.
4. Count useful, redundant, failed, and repeated tool calls.
5. Measure false-success claims separately.
6. Test replanning after changed observations.
7. Report results by task depth and risk.
8. Prefer the simplest system that passes required gates.

Part 4 - Python code skeleton

```
from collections import defaultdict
from dataclasses import dataclass


@dataclass
class ReasoningRun:
    architecture: str
    task_id: str
    required_depth: int
    verified_success: bool
    claimed_success: bool
    steps: int
    redundant_calls: int
    latency_ms: float
    cost: float


def summarize_by_depth(runs):
    grouped = defaultdict(list)
    for run in runs:
        key = (run.architecture, run.required_depth)
        grouped[key].append(run)

    rows = []
    for key, group in grouped.items():
        success = 0
        false_success = 0
        total_steps = 0
        total_cost = 0.0
        for run in group:
            success += int(run.verified_success)
            false_success += int(
                run.claimed_success and not run.verified_success
            )
            total_steps += run.steps
            total_cost += run.cost
        rows.append({
            "architecture": key[0],
            "required_depth": key[1],
            "success_rate": success / len(group),
            "false_success_rate": false_success / len(group),
            "mean_steps": total_steps / len(group),
            "mean_cost": total_cost / len(group),
        })
    return rows
```

Part 5 - The Philosophy Behind the Exercise

More thought is not automatically better thought. Additional steps create room for checking and adaptation, but also for drift and self-deception. Complexity must justify itself through tasks that genuinely require it.

Good engineering uses the minimum agency necessary. A one-step answer is elegant when sufficient; a multi-step process is responsible when the world demands evidence, dependency, and revision.

18.10.15 Exercise 14

Exercise Title: Debug Agent Failure Under Partial Tool Errors

Part 1 - Why this matters

Real tools rarely fail as a clean all-or-nothing event. A paginated search may return two pages and time out on the third. A batch API may update some records. A database replica may be stale. A payment may succeed while its acknowledgment is lost. Agents that interpret any payload as complete success or retry entire operations blindly can produce wrong answers and duplicated side effects.

Require tools to return structured status: succeeded, partial, failed, unknown, or denied; completed and failed item IDs; continuation token; freshness; idempotency key; and retry guidance. The agent state must preserve partial progress. Define per-tool recovery: resume from checkpoint, retry failed items, query operation status, compensate completed side effects, switch replica, ask for confirmation, or escalate.

Inject failures at every boundary and step. Test timeouts before execution, during execution, and after success but before acknowledgment. Verify that the final answer labels incomplete evidence and never claims full completion. Measure recovery success, duplicate side effects, lost work, additional cost, time to safe state, and false-success rate.

Part 2 - Mathematical form and reasoning

For a batch of items B, preserve disjoint sets

$$B = C \cup F \cup U, \qquad C \cap F = C \cap U = F \cap U = \varnothing,$$

where C completed, F failed, and U has unknown status. Retry only a set approved by tool semantics. Completion can be reported only when

$$F = \varnothing \wedge U = \varnothing$$

or the user explicitly accepts a documented partial result.

Part 3 - Hints and Helper

1. Define succeeded, partial, failed, unknown, and denied statuses.
2. Return completed, failed, and unknown item IDs.
3. Persist checkpoints and idempotency keys outside model context.
4. Query status before retrying ambiguous side effects.

5. Retry only failed or safe items according to tool semantics.
6. Label incomplete evidence in the final response.
7. Inject failures before, during, and after external execution.
8. Measure duplication, lost work, recovery, and false completion.

Part 4 - Python code skeleton

```
from dataclasses import dataclass, field
from enum import Enum


class BatchStatus(Enum):
    SUCCEEDED = "succeeded"
    PARTIAL = "partial"
    FAILED = "failed"
    UNKNOWN = "unknown"


@dataclass
class BatchObservation:
    operation_id: str
    status: BatchStatus
    completed_ids: set = field(default_factory=set)
    failed_ids: set = field(default_factory=set)
    unknown_ids: set = field(default_factory=set)


def validate_partition(requested_ids, observation):
    combined = (
        observation.completed_ids
        | observation.failed_ids
        | observation.unknown_ids
    )
    if combined != set(requested_ids):
        raise ValueError("result partition does not match request")
    if observation.completed_ids & observation.failed_ids:
        raise ValueError("completed and failed overlap")
    if observation.completed_ids & observation.unknown_ids:
        raise ValueError("completed and unknown overlap")


def recovery_action(observation, tool):
    if len(observation.unknown_ids) > 0:
        return "query_operation_status"
    if len(observation.failed_ids) > 0 and tool.safe_item_retry:
        return "retry_failed_items_only"
```

```
    if observation.status == BatchStatus.SUCCEEDED:
        return "verify_and_finish"
    return "human_escalation"
```

Part 5 - The Philosophy Behind the Exercise

Partial failure reveals that the world does not share the clean transactional boundaries imagined in a prompt. Reliable agents must remember what is known, what happened, and what remains uncertain without collapsing those categories.

Recovery is not repeating hope. It is reasoning from durable state and choosing an action whose consequences remain bounded.

18.10.16 Exercise 15

Exercise Title: Build Production Agent Monitoring and Logging

Part 1 - Why this matters

Agent systems make decisions across model, retrieval, memory, policy, and tool boundaries. A final HTTP status cannot explain why a task failed or whether an external side effect occurred. Build distributed traces that preserve the causal chain while respecting privacy. Every request needs a trace ID; every model turn, retrieval, policy decision, tool call, verification, retry, memory write, and final response needs a span linked to its parent.

Record release identity, prompt template, model, token counts, and retrieved chunk IDs. Add tool and schema versions, argument hashes, policy outcome, executor status, latency, retry metadata, idempotency information, verification, termination reason, and cost. Avoid raw secrets, credentials, unnecessary personal data, and complete sensitive prompts. Use redaction, access control, encryption, retention limits, and separate audit logs for privileged actions.

Monitor task success, false success, tool failure, denied calls, repeated actions, loop/budget stops, RAG recall proxies, citation verification, memory errors, p95/p99 latency, queue depth, cost, and human escalations. Establish baselines and alerts with minimum volume and repeated windows. Link each alert to representative traces and a runbook. Test observability by injecting failures and missing telemetry; a silent monitor is itself a production failure.

Part 2 - Mathematical form and reasoning

Trace latency obeys a critical-path relation rather than the sum of parallel spans. For a sequential chain,

$$T_{\text{trace}} \approx \sum_{j} T_j,$$

while concurrent child spans contribute approximately their maximum plus coordination overhead. Define error-budget burn over window w:

$$B_w = \frac{\text{observed bad-event rate}_w}{\text{allowed bad-event rate}}.$$

Alerting can combine burn rate, sample minimum, and consecutive windows to avoid both silence and noise.

Part 3 - Hints and Helper

1. Propagate trace and call IDs through every component.
2. Log decisions and hashes without exposing secrets or unnecessary raw text.
3. Separate operational logs from immutable privileged-action audit records.
4. Track verified task outcomes and false-success claims.
5. Monitor loop stops, denials, retries, partial results, and memory writes.
6. Add release, schema, prompt, corpus, and tool versions.
7. Link alerts to traces, owners, and runbooks.
8. Inject known failures to verify observability end to end.

Part 4 - Python code skeleton

```
from contextlib import contextmanager
from dataclasses import dataclass, field
from time import perf_counter


@dataclass
class Span:
    trace_id: str
    span_id: str
    parent_id: str
    name: str
    attributes: dict = field(default_factory=dict)
    duration_ms: float = 0.0
    status: str = "unknown"


class TraceRecorder:
    def __init__(self, sink, redactor):
        self.sink = sink
        self.redactor = redactor

    @contextmanager
    def span(self, span):
        started = perf_counter()
        try:
            yield span
            span.status = "succeeded"
        except Exception as exc:
            span.status = "failed"
            span.attributes["error_type"] = type(exc).__name__
```

```
            raise
        finally:
            span.duration_ms = (perf_counter() - started) * 1000.0
            safe_attributes = self.redactor.clean(span.attributes)
            span.attributes = safe_attributes
            self.sink.write(span)


def record_tool_call(recorder, span, call, execute):
    with recorder.span(span):
        span.attributes["tool_name"] = call.tool_name
        span.attributes["call_id"] = call.call_id
        result = execute(call)
        span.attributes["tool_status"] = result.status
        return result
```

Part 5 - The Philosophy Behind the Exercise

Observability is the system's capacity to remember how it acted. Without that memory, an agent's apparent autonomy becomes opacity: nobody can distinguish a reasoned action from an accident or a reported success from a real one.

Good monitoring does not watch everything indiscriminately. It preserves the minimum causal evidence needed for accountability while respecting the people whose data passes through the system.

18.10.17 Exercise 16

Exercise Title: Stress Test Agent System Under Concurrent Requests

Part 1 - Why this matters

Concurrency exposes failures invisible in single-agent demos. Requests compete for model batches, vector indexes, database pools, tool quotas, memory locks, and human-review capacity. Shared state can leak across tenants, duplicate tool calls can race, idempotency keys can collide, and long agent loops can starve short tasks. A correct isolated trace does not prove a correct shared system.

Build closed-loop and open-loop workloads with realistic task-depth, prompt length, retrieval size, tool mix, and side-effect frequency. Include hot keys, same-user concurrent turns, different tenants, cancellations, tool throttling, slow dependencies, and partial failures. Use unique trace and idempotency IDs; verify that state and memory remain isolated. Bound concurrency and queue sizes so overload becomes controlled rejection rather than collapse.

Measure queue time, TTFT, task completion, p95/p99 latency, throughput, tool and model saturation, retries, duplicate effects, cross-request contamination, loop stops, errors, fallbacks, and cost. Sweep offered load until throughput flattens and tail latency grows. Define a capacity point below that saturation knee with explicit headroom and service-level constraints.

Part 2 - Mathematical form and reasoning

For stable service, Little's law gives

$$L = \lambda W,$$

where L is average in-flight work, λ completed request rate, and W mean time in system. Capacity is the maximum offered load satisfying all gates:

$$\lambda^* = \max_{\lambda} \{\lambda : p_{99}(T) \le S_T,\ r_{\text{error}} \le S_E,\ r_{\text{duplicate}} = 0,\ r_{\text{leak}} = 0\}.$$

Zero duplication and cross-tenant leakage are invariants, not percentile goals.

Part 3 - Hints and Helper

1. Model prompt length, task depth, tool mix, and arrival pattern.
2. Test closed-loop and open-loop traffic separately.
3. Inject slow tools, rate limits, cancellations, and partial failures.
4. Verify tenant, memory, trace, and idempotency isolation.
5. Bound queues and implement overload rejection.
6. Measure tail latency, success, duplicates, leakage, and cost.
7. Sweep load to locate the saturation knee.
8. Repeat runs and preserve every failed trace.

Part 4 - Python code skeleton

```
from concurrent.futures import ThreadPoolExecutor, as_completed
from dataclasses import dataclass
from time import perf_counter


@dataclass
class LoadResult:
    request_id: str
    tenant_id: str
    ok: bool
    latency_ms: float
    tool_calls: int
    duplicate_effect: bool
    leaked_tenant_id: str
    error_type: str


def execute_request(agent, request):
    started = perf_counter()
    try:
```

```
        output = agent.run(request)
        latency = (perf_counter() - started) * 1000.0
        return LoadResult(
            request.request_id,
            request.tenant_id,
            True,
            latency,
            output.tool_calls,
            output.duplicate_effect,
            output.leaked_tenant_id,
            "",
        )
    except Exception as exc:
        latency = (perf_counter() - started) * 1000.0
        return LoadResult(
            request.request_id, request.tenant_id, False,
            latency, 0, False, "", type(exc).__name__
        )


def run_concurrent(agent, requests, workers):
    results = []
    with ThreadPoolExecutor(max_workers=workers) as executor:
        futures = []
        for request in requests:
            futures.append(executor.submit(execute_request, agent, request))
        for future in as_completed(futures):
            results.append(future.result())
    return results
```

Part 5 - The Philosophy Behind the Exercise

Concurrency transforms independent intentions into a shared-resource problem. Failures arise not because any request is unreasonable, but because their timing creates contention, interference, and ambiguity about ownership.

Stress testing asks whether the agent remains bounded and truthful when the world is crowded. Production readiness begins where the isolated demonstration ends.

18.10.18 Exercise 17

Exercise Title: Build Agent Safety Checklist for Production Use

Part 1 - Why this matters

Agent safety spans architecture, identity, tools, data, memory, reliability, operations, and governance. A checklist converts these obligations into release gates with evidence. It must fail closed when a required item is unknown, and it must not reduce safety to a reassuring count of checked boxes. One missing authorization or rollback gate can invalidate the entire deployment.

Cover scoped goals, supported domains, success/stop conditions, tool inventory, side-effect classification, least privilege, schemas, ownership validation, confirmation, idempotency, sandboxing, prompt-injection defense, data classification, memory retention/deletion, tenant isolation, retrieval access, result verification, loop budgets, human escalation, monitoring, incident response, fallback, rollback, testing, red-teaming, privacy, and compliance.

Give every gate a stable ID, requirement, owner, evidence URI, last test date, expiry, status, and waiver with approver and end date. Separate static design review from executable tests that must run per release and hardware profile. Drill denial, tool outage, partial success, leakage attempt, loop, overload, and rollback. Revisit the checklist after incidents and architecture changes.

Part 2 - Mathematical form and reasoning

Required release readiness is conjunctive:

$$G_{\text{release}} = \bigwedge_{k \in \mathcal{K}_{\text{required}}} G_k.$$

Risk prioritization may use

$$R_k = P_k S_k E_k,$$

but risk scores do not waive hard requirements. Evidence freshness can be a predicate $I(t_{\text{now}} - t_{\text{test}} \leq \Delta_k)$; stale evidence fails the gate unless an authorized time-limited waiver exists.

Part 3 - Hints and Helper

1. Use stable gate IDs, owners, evidence, and expiry dates.
2. Make required readiness a conjunction, not a percentage.
3. Separate design assertions from executable release tests.
4. Cover identity, permissions, tools, data, memory, loops, and operations.
5. Drill denial, outage, partial success, overload, and rollback.
6. Time-limit waivers and record explicit residual risk.
7. Fail on missing or stale evidence.
8. Update gates after every incident and architecture change.

Part 4 - Python code skeleton

```
from dataclasses import dataclass
from datetime import date


@dataclass
class SafetyGate:
```

```
    gate_id: str
    required: bool
    passed: bool
    owner: str
    evidence_uri: str
    tested_on: str
    expires_on: str
    waiver_expires_on: str = ""


def gate_is_current(gate, today):
    expiry = date.fromisoformat(gate.expires_on)
    return expiry >= today


def waiver_is_valid(gate, today):
    if gate.waiver_expires_on == "":
        return False
    expiry = date.fromisoformat(gate.waiver_expires_on)
    return expiry >= today


def evaluate_safety_gates(gates, today):
    failures = []
    for gate in gates:
        if not gate.required:
            continue
        evidence_present = gate.evidence_uri.strip() != ""
        current = gate_is_current(gate, today)
        waived = waiver_is_valid(gate, today)
        if not evidence_present:
            failures.append(gate.gate_id + ":missing_evidence")
        elif not current:
            failures.append(gate.gate_id + ":stale_evidence")
        elif not gate.passed and not waived:
            failures.append(gate.gate_id + ":failed")
    return {"ready": len(failures) == 0, "failures": failures}
```

Part 5 - The Philosophy Behind the Exercise

A checklist is institutional memory made executable. It preserves lessons that individual engineers would otherwise forget under schedule pressure and forces important claims to produce evidence.

Its deepest value is conjunctive thinking. Complex systems often fail through one neglected boundary, so safety cannot be averaged into existence.

18.10.19 Exercise 18

Exercise Title: Document Architecture Decisions for RAG and Agent Systems

Part 1 - Why this matters

RAG and agent systems accumulate consequential choices: chunking, embedding, index, reranker, context policy, model, tool boundary, memory type, planning loop, verification, guardrails, observability, and fallback. When the reasons are not recorded, later teams inherit implementation without assumptions and repeat debates after conditions have changed. Use architecture decision records (ADRs) to preserve the decision, evidence, alternatives, and consequences.

Each ADR should contain context, problem, forces, hard constraints, considered options, decision, rationale, evidence, security/privacy implications, operational consequences, rejected alternatives, unknowns, owner, status, date, and re-evaluation triggers. Link to benchmarks and threat models rather than copying results without provenance. Keep decisions small enough to revise independently; "use an agent" is too broad for one useful record.

Record interface contracts and data flows. State where authorization occurs, which component owns retries, how memory is deleted, how result verification is performed, and what happens when retrieval or tools fail. Mark decisions as proposed, accepted, superseded, or retired. Test documentation through a design review: a new engineer should reconstruct the current architecture and explain why major alternatives were rejected.

Part 2 - Mathematical form and reasoning

An option j can be represented by measurable consequences

$$\mathbf{m}_j = (Q_j, L_j, C_j, R_j, M_j, O_j),$$

for quality, latency, cost, risk, memory, and operational complexity. Hard constraints define the feasible set $\mathcal{F}$; preferences select within it. Sensitivity analysis asks whether the decision changes when assumptions or weights change. A fragile decision receives an earlier review trigger.

Part 3 - Hints and Helper

1. Write one decision per ADR with a stable identifier.
2. State constraints, assumptions, alternatives, and evidence.
3. Record security, privacy, reliability, and operational consequences.
4. Document failure ownership and fallback behavior at interfaces.
5. Link benchmarks, diagrams, threat models, and schemas.
6. Mark proposed, accepted, superseded, and retired states.
7. Define measurable re-evaluation triggers.
8. Review whether a new engineer can reconstruct the rationale.

Part 4 - Python code skeleton

```
from dataclasses import asdict, dataclass
from typing import List
import json


@dataclass
class ArchitectureDecision:
    decision_id: str
    title: str
    status: str
    context: str
    constraints: List[str]
    alternatives: List[str]
    decision: str
    rationale: str
    consequences: List[str]
    evidence_uris: List[str]
    owner: str
    decision_date: str
    reevaluation_triggers: List[str]
    supersedes: str = ""


def validate_decision(record):
    errors = []
    if len(record.alternatives) < 2:
        errors.append("insufficient_alternatives")
    if record.rationale.strip() == "":
        errors.append("missing_rationale")
    if len(record.evidence_uris) == 0:
        errors.append("missing_evidence")
    if len(record.reevaluation_triggers) == 0:
        errors.append("missing_reevaluation_triggers")
    return errors


def save_decisions(records, output_path):
    payload = []
    for record in records:
        errors = validate_decision(record)
        if len(errors) > 0:
            raise ValueError(record.decision_id + ":" + ",".join(errors))
        payload.append(asdict(record))
    with open(output_path, "w", encoding="utf-8") as handle:
        json.dump(payload, handle, indent=2, sort_keys=True)
```

Part 5 - The Philosophy Behind the Exercise

Architecture is a history of constrained choices. Code shows what the system does now; a decision record explains why this form was considered responsible under the evidence available then.

Documentation makes architecture revisable without making it arbitrary. Future engineers can preserve a sound rationale or replace it when its assumptions no longer hold.

18.10.20 Exercise 19

Exercise Title: Produce a Final LLM Systems Engineering Report

Part 1 - Why this matters

The final report is the auditable release argument for the complete RAG and agent system. It must identify model, tokenizer, prompts, corpus, chunks, embedding model, vector index, reranker, tools, schemas, policies, memory stores, runtime, deployment, and code revision. Hash artifacts and preserve exact versions so measured behavior remains connected to the system that produced it.

Describe architecture and trust boundaries before results. Include data flows, identity and authorization, tool side effects, confirmation, idempotency, sandboxing, retrieval access filters, memory lifecycle, planning budgets, result verification, partial-error recovery, monitoring, human escalation, and rollback. Link each choice to an ADR and threat-model obligation.

Report retrieval recall, context and citation quality, task success, false success, tool selection and argument validity, SQL execution accuracy, memory precision/leakage, loop rate, recovery, safety denials, latency/throughput under concurrency, cost, human review, and known failure slices. Include denominators, uncertainty, workload, hardware, injected failures, rejected variants, and raw evidence. End with supported/unsupported uses, release gates, residual risks, owners, canary plan, monitors, incident runbooks, rollback target, and revalidation date. Independent review should reproduce selected claims.

Part 2 - Mathematical form and reasoning

Release readiness is conjunctive:

$$G = G_{\text{identity}} \wedge G_{\text{RAG}} \wedge G_{\text{tools}} \wedge G_{\text{memory}} \wedge G_{\text{safety}} \wedge G_{\text{reliability}} \wedge G_{\text{operations}} \wedge G_{\text{rollback}}.$$

Traceability follows

$$\text{requirement} \to \text{design decision} \to \text{test} \to \text{evidence} \to \text{control} \to \text{monitor}.$$

Unknown or missing required evidence fails closed. Performance cannot compensate for a failed authorization, isolation, verification, or rollback gate.

Part 3 - Hints and Helper

1. Hash and version every model, data, index, prompt, schema, policy, and runtime artifact.
2. Map architecture, data flows, trust boundaries, and side effects.

3. Link requirements to ADRs, tests, raw evidence, controls, and monitors.
4. Include quality, safety, reliability, latency, concurrency, and cost.
5. Report injected failures, rejected designs, and unresolved unknowns.
6. State supported uses, prohibited uses, and human escalation rules.
7. Include canary, incident response, fallback, rollback, and owners.
8. Require independent reproduction and a dated revalidation decision.

Part 4 - Python code skeleton

```
from dataclasses import asdict, dataclass, field
from datetime import datetime, timezone
from typing import Dict, List
import json


@dataclass
class SystemGate:
    name: str
    requirement: str
    observed: str
    evidence_uri: str
    owner: str
    passed: bool


@dataclass
class SystemsReport:
    release_id: str
    created_at: str
    artifact_hashes: Dict[str, str]
    architecture_decisions: List[str]
    gates: List[SystemGate] = field(default_factory=list)
    supported_uses: List[str] = field(default_factory=list)
    prohibited_uses: List[str] = field(default_factory=list)
    residual_risks: List[str] = field(default_factory=list)
    rollback_release: str = ""
    revalidation_date: str = ""
    decision: str = "pending"


def validate_report(report, required_gates):
    errors = []
    by_name = {}
    for gate in report.gates:
```

```
        by_name[gate.name] = gate
    for gate_name in required_gates:
        if gate_name not in by_name:
            errors.append("missing_gate:" + gate_name)
            continue
        gate = by_name[gate_name]
        if not gate.passed:
            errors.append("failed_gate:" + gate_name)
        if gate.evidence_uri.strip() == "":
            errors.append("missing_evidence:" + gate_name)
    if report.rollback_release.strip() == "":
        errors.append("missing_rollback_release")
    if report.revalidation_date.strip() == "":
        errors.append("missing_revalidation_date")
    return errors


def finalize_report(report, required_gates, output_path):
    errors = validate_report(report, required_gates)
    report.decision = "approved" if len(errors) == 0 else "blocked"
    payload = asdict(report)
    payload["validation_errors"] = errors
    with open(output_path, "w", encoding="utf-8") as handle:
        json.dump(payload, handle, indent=2, sort_keys=True)
    return errors


def new_report(release_id, hashes, decision_ids):
    return SystemsReport(
        release_id=release_id,
        created_at=datetime.now(timezone.utc).isoformat(),
        artifact_hashes=hashes,
        architecture_decisions=decision_ids,
    )
```

Part 5 - The Philosophy Behind the Exercise

An agentic system is not merely a model with additional functions. It is a network of authorities, memories, observations, actions, and recovery paths. Engineering quality lies in how those relations are bounded and evidenced.

The report preserves a disciplined claim: this system may act in these ways, under these conditions, because these tests and controls justify that authority. Monitoring and rollback keep the claim open to correction when reality changes. ==== Book19: DMM (Dynamic Memory Management) ====

Book 19

Dynamic Memory Management

19.1 Introduction

19.1.1 What will the book teach you

Modern artificial intelligence systems are often described through model architecture, parameter count, training loss, and computational throughput. Those quantities are important, but they do not fully determine whether a system can be trained or served. A model may have mathematically valid operations and sufficient arithmetic performance while still being impossible to execute because its parameters, activations, gradients, optimizer states, temporary workspaces, communication buffers, or inference caches cannot reside in the available memory at the required time. Dynamic memory management is the discipline of deciding what should exist, where it should exist, when it should move, how long it should remain alive, and what should happen when the original plan no longer fits the physical machine.

This book studies memory as an active part of the artificial intelligence runtime rather than as a passive container for tensors. The reader will move beyond the simple question "How much VRAM does this model use?" and learn to ask a more complete set of questions. Which allocations are persistent and which are temporary? Which tensor owns a storage object, and which tensor is only a view into that storage? What is the difference between memory that is allocated by PyTorch, memory reserved by the caching allocator, and memory reported as used by the operating system or GPU driver? At what instant does peak memory occur? Can a tensor be recomputed more cheaply than it can be stored? Can data movement be hidden behind useful computation? Can a runtime recover when an estimated budget is wrong? These questions turn memory from an afterthought into an engineering model that can be predicted, measured, and controlled.

The central problem is the growing separation between the capacity demanded by AI workloads and the capacity offered by the fastest memory tier. GPU VRAM is valuable because it provides the bandwidth and proximity needed by CUDA kernels, but it is limited and expensive. CPU RAM is usually larger, yet a tensor in ordinary pageable host memory cannot be consumed directly by a GPU kernel and may require a costly transfer. Pinned host memory can support more efficient asynchronous transfers, but it is a constrained system resource and should not be allocated carelessly. NVMe storage provides far greater capacity at much higher latency and lower bandwidth than RAM. A successful runtime must therefore reason across a hierarchy rather than pretend that every byte is equivalent:

GPU VRAM $\longleftrightarrow$ Pinned CPU Memory $\longleftrightarrow$ Pageable CPU Memory $\longleftrightarrow$ NVMe Storage.

Moving an object to a larger and slower tier increases available capacity, but it also introduces transfer latency, bandwidth pressure, synchronization, and new failure modes. Keeping every object in VRAM minimizes some transfers but may make the workload impossible to run. Evicting too aggressively may avoid an out-of-memory error while making the system slower than a CPU-only baseline. The goal is therefore not to minimize memory usage in isolation. The goal is to satisfy correctness and capacity constraints while choosing a defensible trade-off among memory, computation, transfer, latency, throughput, and reliability.

The book begins with observation because memory cannot be managed responsibly when it cannot be seen. The reader will learn the PyTorch CUDA memory APIs, memory statistics, profiler views, allocator summaries, and memory snapshots. These tools reveal different layers of the system and must not be interpreted as interchangeable measurements. In particular, a tensor that is deleted at the Python level does not imply that the corresponding block is immediately returned to the GPU driver. PyTorch may retain the block inside its caching allocator so that a later allocation can reuse it without paying the full cost of a device allocation. Consequently, `memory_allocated()`, `memory_reserved()`, peak statistics, and external device monitors can all report different values while each remains correct within its own definition. The reader will learn to explain those differences instead of treating them as contradictory evidence.

Allocator behavior then connects measurement to mechanism. Repeatedly creating and destroying differently sized tensors can produce inactive blocks, fragmentation, and a reserved pool that contains enough total free bytes but not a suitable contiguous block for the next request. A CUDA out-of-memory failure is therefore not always equivalent to "the model is larger than the GPU." It may be caused by a transient peak, a retained autograd graph, a growing Python container, a badly timed workspace, a batch-size spike, a fragmented allocator state, or an external process consuming the same device. The exercises will distinguish these cases by timeline, tensor ownership, allocation history, and reproducible evidence.

The next stage studies movement between CPU and GPU. The reader will compare pageable and pinned host buffers, blocking and non-blocking copies, default and custom CUDA streams, synchronization boundaries, CUDA events, and prefetching. An operation marked `non_blocking=True` is not automatically overlapped with useful work. True overlap depends on the source memory, the device, available copy engines, stream relationships, dependency ordering, and whether an accidental synchronization forces the host or device to wait. The book will therefore separate three claims that are often confused: an API call returned quickly to the CPU, a transfer executed asynchronously on the GPU timeline, and the transfer actually overlapped with computation. Each claim requires its own measurement.

Once movement is measurable, memory placement becomes a policy problem. Dynamic tensor placement, parameter placement, layer offloading, CPU offloading, NVMe offloading, and out-of-core execution all require the runtime to choose which objects occupy the fastest tier at each phase. A useful policy must account for object size, next-use time, transfer cost, recomputation cost, dependency order, available headroom, and the penalty of a wrong prediction. Simple policies such as first-in-first-out or least-recently-used eviction are valuable baselines, but neural network execution has more structure than a generic cache. The next layer is usu-

ally known, an activation may have a single future consumer, a parameter can be reused across many requests, and a KV cache entry may have semantic importance that is not represented by recency alone. The exercises will progress from explicit baselines to memory-aware decisions whose assumptions can be tested.

Training creates its own memory lifecycle. Parameters, gradients, optimizer states, saved activations, temporary kernels, data batches, and validation outputs do not reach their peaks at the same moment. Activation checkpointing reduces saved forward state by recomputing selected operations during backward. Activation offloading moves saved tensors to CPU memory and later restores them. Mixed precision changes both storage and numerical range. Gradient accumulation changes the relationship between micro-batch size and effective batch size. Adaptive batching changes work admitted by the runtime according to current headroom. None of these techniques creates free capacity: each exchanges memory for extra computation, transfer, latency, numerical risk, or optimization behavior. The reader will quantify the exchange rather than describing a technique only as "memory efficient."

Out-of-memory recovery is treated as a runtime protocol, not as a broad `try/except` block. CUDA work is asynchronous, and a failure can surface later than the operation that caused it. Partially completed training steps may leave gradients, optimizer state, data iterators, or random number generators in a condition that makes a careless retry incorrect. Recovery must identify a safe boundary, release recoverable references, restore invariants, modify the workload, and verify that the retry did not silently change the intended training semantics. The book will build failure injection tests so that recovery behavior is exercised deliberately before it is trusted in a long run.

The middle chapters move deeper into storage behavior. Memory pools and buffer reuse can reduce allocator traffic, but reuse requires precise lifetime and aliasing rules. In-place operations can save a temporary allocation while corrupting a value required by autograd. Views may share storage even when their shapes are different. A detached tensor may share the same bytes while leaving the gradient graph. The `meta` device can construct model structure without allocating parameter storage, but the parameters must later be materialized with correct shapes, devices, dtypes, initialization, and checkpoint mappings. Memory-mapped datasets and virtualized or paged tensor storage extend the same principle to objects that cannot be loaded eagerly. These chapters teach that a tensor is not merely a shape containing numbers; it has storage, ownership, lifetime, placement, and synchronization semantics.

Large language model inference introduces a particularly important dynamic object: the Key-Value cache. KV caching avoids recomputing attention keys and values for the entire prefix at every generation step, but its capacity grows with sequence length, batch size, layer count, head configuration, and data type. A server that accepts many long requests can exhaust memory through cache growth even when its model weights fit comfortably. The reader will study paged KV caches, CPU offloading, quantization, eviction, and context-retention policies. The correctness question is not only whether a cache operation saves memory. It must preserve token positions, request boundaries, attention masks, cache order, and the exact association between stored keys, values, layers, and sequences. A fast cache that retrieves the wrong history is a semantic failure, not a performance optimization.

Distributed execution expands the memory problem across devices. Fully Sharded Data Par-

allel, ZeRO-style partitioning, tensor parallelism, pipeline parallelism, and explicit multi-GPU balancing distribute parameters, gradients, optimizer states, activations, and communication buffers in different ways. Sharding reduces replication but creates all-gather, reduce-scatter, or point-to-point communication at specific times. Pipeline parallelism reduces per-device model residency but introduces stage imbalance, pipeline bubbles, and activation queues. Tensor parallelism splits individual operations but requires collectives in the critical path. The correct question is not "Which technique uses the least memory?" It is "Which objects are partitioned, which objects remain replicated, when are they materialized, what communication is introduced, and where does the new peak occur?"

The distributed chapters are designed to run on one multi-GPU computer. A two-GPU system is sufficient for the core experiments that compare one and two ranks, inspect per-device memory, and validate communication-aware placement. Two physical computers are not required. Multi-node execution may be explored as an optional extension when suitable hardware and networking are available, but it is not a prerequisite for completing the book. This distinction keeps the conceptual study of distributed memory separate from the operational complexity of configuring a cluster.

The final chapters bring the complete discipline together. CUDA events provide device-side timing needed to distinguish launch latency from executed GPU work. Bandwidth profiling shows whether an offload path is limited by PCIe transfer, host preparation, storage access, synchronization, or computation. Leak detection and autograd graph analysis locate memory that remains reachable longer than intended. `detach()`, `no_grad()`, and `inference_mode()` are compared through both graph semantics and measured allocation behavior. Finally, the reader will construct a custom memory manager and a custom offload engine. These projects require the earlier ideas to operate together: accounting, budgets, tensor identity, placement, eviction, prefetch, streams, correctness checks, metrics, recovery, and policy evaluation.

The objective of the final runtime is not to outperform every specialized framework. Its purpose is to prove that the reader can turn a memory-management idea into a controlled system. Every policy decision must produce evidence: what was moved, why it was selected, how much capacity was recovered, what transfer was introduced, whether execution remained correct, and how behavior changed when the workload or hardware budget changed. By the end of the book, the reader should be able to investigate an unfamiliar PyTorch memory problem from first principles rather than relying on a list of isolated fixes.

19.1.2 Why this book comes after large language models

The earlier books developed models from linear regression through perceptrons, MLPs, CNNs, RNNs, LSTMs, Transformers, and large language model systems. Each stage increased the number and diversity of objects that must remain alive during execution. A small classifier may be described adequately by its parameters and batch activations. A Transformer training step adds attention matrices, residual streams, normalization state, gradients, optimizer moments, temporary matrix-multiplication workspaces, and possibly multiple copies of state across devices. Autoregressive inference replaces many training objects with a KV cache whose size changes continuously as requests arrive, generate tokens, and finish.

This makes dynamic memory management a natural systems-level continuation of the model

books. The reader already knows what the tensors mean mathematically; the present book asks how those tensors can be scheduled on finite hardware. Attention explains why long context is useful, while this book explains why long context is expensive. Mixed precision was previously introduced as a training technique; here it is examined as one component in a memory budget with numerical and kernel constraints. Distributed training was previously a way to scale computation; here it is analyzed as a transformation of tensor residency, lifetime, replication, and communication. The model and the memory runtime are therefore not separate subjects. The model creates the lifetimes and dependencies that a correct memory policy must respect.

19.1.3 The seven-part structure of every exercise

Each exercise in this book is organized into seven parts. The order is deliberate. It prevents implementation from becoming trial and error and forces every result to connect a conceptual claim with a measurable system effect.

1. **Part 1 - Exercise Contract and Why It Matters.** This part defines the exact problem, the expected deliverable, the correctness conditions, the hardware assumptions, and the practical reason the exercise exists. It states what must be proved rather than merely what code must be written. A deep motivation is included so that an allocator statistic, transfer flag, or offload policy is connected to a real failure mode in training or inference.

2. **Part 2 - System and Memory Mental Model.** This part identifies every important object and relationship before any experiment begins. It explains tensor storage, lifetime, device placement, allocator state, stream dependencies, autograd ownership, or distributed partitioning as required by the exercise. The purpose is to create a model of the runtime that is detailed enough to predict behavior.

3. **Part 3 - Quantitative Prediction Before Execution.** The reader calculates expected tensor sizes, peak residency, transfer volume, bandwidth limits, operation order, or scaling behavior before running the program. Predictions must include units and assumptions. When an exact value cannot be known in advance, the reader defines a range or a directional hypothesis. The later difference between prediction and observation becomes evidence about hidden runtime costs.

4. **Part 4 - Experimental Design and Instrumentation.** This part defines the baseline, controlled variable, metrics, warm-up, synchronization, random seeds, repetitions, failure injection, and logging plan. Instrumentation is selected according to the claim being tested. Wall-clock timers, CUDA events, allocator counters, profiler traces, and snapshots answer different questions and cannot be substituted blindly.

5. **Part 5 - PyTorch/CUDA Implementation.** The system is implemented explicitly and readably. Tensor shapes, devices, dtypes, streams, events, masks, and ownership transitions remain visible. Convenience abstractions may be introduced only after a transparent reference implementation exists. The code must contain assertions and diagnostic output that make violations observable instead of allowing a silent incorrect result.

6. **Part 6 - Measurement, Validation, and Failure Analysis.** The implementation is tested against the original contract. Numerical parity, peak memory, throughput, latency, transfer overlap, allocator behavior, and recovery correctness are measured where relevant.

Failure cases are reproduced deliberately. The reader must locate the root cause, apply a correction, and create a regression test that detects the same failure if it returns.

7. **Part 7 - Engineering Interpretation and Philosophy.** The final part explains what the experiment means beyond one machine and one run. It separates universal principles from hardware-dependent observations, identifies the trade-off purchased by the technique, states when the method should not be used, and records remaining uncertainty. The philosophy of the exercise is to develop engineering judgment: a memory optimization is successful only when it preserves the intended computation and improves the resource constraint that actually limits the system.

The seven parts form one reasoning cycle:

$$\text{Understand} \longrightarrow \text{Model} \longrightarrow \text{Predict} \longrightarrow \text{Design} \longrightarrow \text{Implement} \longrightarrow \text{Measure} \longrightarrow \text{Interpret}.$$

This cycle is more important than any individual API. PyTorch and CUDA will continue to evolve, but the ability to define a contract, model lifetimes, predict a peak, design a valid measurement, and interpret a trade-off remains transferable.

19.1.4 How to study this book

Begin every exercise with a small, deterministic case. Write down the shapes, element counts, dtypes, expected byte sizes, devices, and lifetime boundaries. For a dense tensor with dimensions $n_1, \ldots, n_k$ and element size s bytes, the payload estimate begins with

$$B_{\text{tensor}} = s \prod_{i=1}^{k} n_i.$$

This is a starting point, not the complete process footprint. Storage may be shared by views; allocators round requests and reserve blocks; kernels may require temporary workspaces; autograd may save tensors; libraries and CUDA contexts consume device memory; and asynchronous work may keep an object alive past the line where Python appears to release it. The exercise should state which quantities are included in a prediction and which are expected to appear only in measurement.

Do not benchmark asynchronous CUDA work with an unsynchronized CPU timer and interpret the result as kernel duration. Use warm-up to remove initialization and compilation effects, synchronize at meaningful boundaries, and use CUDA events for device timeline measurements when appropriate. Repeat measurements and report variability. A single fast run is not a performance conclusion, and a single memory snapshot is not a lifetime analysis.

Do not trigger uncontrolled allocations merely to discover the physical limit of the machine. Estimate a safe budget, leave headroom for the CUDA context and temporary workspaces, and use bounded synthetic cases. Exercises involving out-of-memory behavior should fail inside a controlled scope and recover at a known boundary. A benchmark must not make the workstation unstable or consume all system RAM in order to be considered realistic.

When comparing two techniques, preserve correctness before comparing speed. Verify outputs against a trusted baseline with tolerances appropriate to the dtype. Confirm that gradients,

cache positions, masks, and optimizer steps represent the same intended computation. Only then compare peak allocated and reserved memory, runtime, throughput, transfer volume, or power. A technique that reduces memory by changing the task, dropping required context, or using stale state has not solved the original problem.

Record hardware and software context with every serious result: GPU model and count, VRAM capacity, CPU RAM, storage type, interconnect, PyTorch version, CUDA version, dtype, batch size, sequence length, model dimensions, and relevant allocator or determinism settings. Absolute performance values are not universal. The book uses measurements to uncover mechanisms and trade-offs, not to produce rankings detached from the environment that created them.

19.1.5 The seven stages of the book

The fifty chapters form seven connected stages rather than fifty isolated API lessons.

1. **DMM00–DMM04: Observation and allocator visibility.** The reader learns the PyTorch CUDA memory APIs, the profiler, caching allocator behavior, memory statistics, and snapshots. This stage builds the evidence required by every later optimization.

2. **DMM05–DMM09: Movement, streams, overlap, and prefetch.** Pinned memory, non-blocking transfers, CUDA streams, compute-transfer overlap, and prefetching establish how data can arrive before it is needed without introducing races or hidden synchronization.

3. **DMM10–DMM16: Placement, eviction, and out-of-core execution.** The reader designs eviction policies and moves tensors, parameters, layers, and training state across GPU, CPU, and NVMe tiers, culminating in models whose total state exceeds VRAM.

4. **DMM17–DMM23: Training-time memory control and recovery.** Activation checkpointing, activation offloading, mixed precision, dynamic loss scaling, gradient accumulation, adaptive batch sizing, and controlled OOM recovery are evaluated as interacting parts of the training runtime.

5. **DMM24–DMM33: Scheduling, storage, reuse, and virtualization.** Memory-aware scheduling, pooling, fragmentation, buffer reuse, aliasing, meta-device construction, lazy materialization, memory mapping, DataLoader behavior, and paged tensor storage expose the lower-level mechanics needed for reusable systems.

6. **DMM34–DMM42: KV cache and distributed memory.** The reader manages dynamic inference caches, then studies how FSDP, ZeRO-style partitioning, tensor parallelism, pipeline parallelism, and multi-GPU balancing redistribute both storage and communication.

7. **DMM43–DMM49: Precise profiling, diagnosis, and runtime construction.** CUDA events, bandwidth analysis, leak detection, autograd graph inspection, inference graph control, a custom memory manager, and a custom offload engine convert the complete book into an integrated engineering practice.

19.1.6 The Chapters (DMM00 - DMM49) included in this book

- DMM00 - PyTorch CUDA Memory APIs
- DMM01 - PyTorch Profiler for Memory
- DMM02 - PyTorch Caching Allocator
- DMM03 - PyTorch CUDA Memory Statistics
- DMM04 - PyTorch CUDA Memory Snapshots
- DMM05 - Pinned Memory and Page-Locked Host Buffers
- DMM06 - Non-Blocking CPU-GPU Transfers
- DMM07 - CUDA Streams in PyTorch
- DMM08 - Compute and Data Transfer Overlap
- DMM09 - Data and Tensor Prefetching
- DMM10 - Memory Eviction Policies
- DMM11 - Dynamic Tensor Placement
- DMM12 - Dynamic Parameter Placement
- DMM13 - Dynamic Layer Offloading
- DMM14 - CPU Offloading for Models and Training State
- DMM15 - NVMe Offloading and Three-Tier Memory Hierarchies
- DMM16 - Out-of-Core Model Execution
- DMM17 - Activation Checkpointing and Recomputation
- DMM18 - Activation Offloading to CPU Memory
- DMM19 - Mixed Precision Memory Management
- DMM20 - Gradient Scaling and Dynamic Loss Scaling
- DMM21 - Gradient Accumulation and Effective Batch Scaling
- DMM22 - Adaptive Batch Sizing Under Dynamic VRAM Budgets
- DMM23 - CUDA Out-of-Memory Detection and Recovery
- DMM24 - Memory-Aware Scheduling
- DMM25 - Memory Pooling and Buffer Reuse
- DMM26 - Memory Fragmentation Management
- DMM27 - Tensor Reuse and Buffer Reuse

- DMM28 - In-Place Operations and Storage Aliasing
- DMM29 - Meta Device and Zero-Allocation Model Construction
- DMM30 - Lazy Parameter Materialization
- DMM31 - Memory-Mapped Data and Large Dataset Access
- DMM32 - DataLoader Memory Optimization
- DMM33 - Paged Memory and Virtualized Tensor Storage
- DMM34 - Paged KV Cache for LLM Inference
- DMM35 - KV Cache Offloading Between VRAM and RAM
- DMM36 - KV Cache Quantization
- DMM37 - KV Cache Eviction and Context Retention Policies
- DMM38 - Fully Sharded Data Parallel (FSDP) Memory Management
- DMM39 - ZeRO-Style Memory Partitioning
- DMM40 - Tensor Parallel Memory Placement
- DMM41 - Pipeline Parallelism and Pipeline Memory Scheduling
- DMM42 - Multi-GPU Memory Balancing
- DMM43 - CUDA Events and Precise GPU Timing
- DMM44 - Memory and Transfer Bandwidth Profiling
- DMM45 - CUDA Memory Leak Detection and Root-Cause Analysis
- DMM46 - Autograd Graph Memory Analysis
- DMM47 - Detach, No-Grad, and Inference Mode Memory Control
- DMM48 - Custom Memory Manager in PyTorch
- DMM49 - Custom Offload Engine and Final Runtime

At the end of this book, memory should no longer appear as an unexplained number printed after a failed run. It should be understood as a timeline of objects, allocations, dependencies, movements, and policy decisions. The reader should be able to predict the dominant costs, collect the evidence needed to correct the prediction, design a safe response to changing capacity, and explain why a chosen memory strategy is correct for the workload in front of it. That is the foundation required to build AI systems that do not merely fit once, but remain observable, recoverable, and efficient as their models, contexts, datasets, and hardware environments grow.

19.2 DMM00

Chapter Title: PyTorch CUDA Memory APIs
Specialization: Dynamic Memory Management and CUDA Systems

19.2.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Allocated, Reserved, Peak, Free, and Total GPU Memory

Part 1 - Exercise Contract and Why It Matters

This exercise establishes the vocabulary used throughout the entire book. The words allocated, reserved, peak, free, and total describe different views of GPU memory. Treating them as synonyms creates false leak reports, incorrect capacity estimates, unsafe batch-size decisions, and misleading claims about what `torch.cuda.empty_cache()` can accomplish. The objective is not merely to print five numbers. The objective is to explain which subsystem owns each number, what event changes it, and what conclusion may legitimately be drawn from it.

Your experiment must distinguish the memory occupied by live PyTorch tensor storage from the larger pool of CUDA blocks held by PyTorch's caching allocator. It must also distinguish both quantities from device-wide free and total memory reported through the CUDA driver. Peak memory is a historical high-water mark, not a description of the current state. A correct report must therefore attach a semantic definition and a time boundary to every value.

The deliverable is a CUDA program and an accompanying table containing measurements at six controlled stages: before the experiment, after one tensor allocation, after a second allocation, after deleting the first tensor, after deleting all live tensors, and after requesting that unused cached blocks be released. The program must report bytes and MiB without confusing decimal MB with binary MiB. It must retain no hidden tensor references and must state that the CUDA context, library workspaces, graphics usage, and allocations from other processes are not completely represented by PyTorch's live-tensor counter.

Success requires more than observing that reserved memory exceeds allocated memory. You must explain why the inequality is normal, identify which values are process-local and which are device-wide, demonstrate that a peak may remain high after current allocations fall, and show why device free memory does not generally equal total memory minus PyTorch allocated memory.

Part 2 - System and Memory Mental Model

PyTorch does not normally ask the CUDA driver for an exact new block every time a tensor is created. Device allocation is relatively expensive and may force synchronization. PyTorch therefore obtains segments from CUDA and manages smaller blocks inside those segments. A live tensor uses a block; an unused block may remain inside the allocator's reserved pool so that a later tensor can reuse it quickly. This produces three nested but non-equivalent views:

$$M_{\text{allocated}} \leq M_{\text{reserved}} \leq M_{\text{device used by all sources}}.$$

`torch.cuda.memory_allocated()` reports memory currently occupied by live tensor storage managed by PyTorch's CUDA allocator for the selected device. It is not a complete process resident-set measurement. Some CUDA libraries, contexts, graph pools, or third-party extensions may allocate memory outside this counter.

`torch.cuda.memory_reserved()` reports the memory held by the caching allocator, including blocks used by live tensors and unused blocks retained for possible reuse. The approximate cached slack is

$$M_{\text{cached slack}} = M_{\text{reserved}} - M_{\text{allocated}},$$

but this subtraction does not guarantee that all slack can satisfy one new request. Blocks have sizes, stream histories, and fragmentation states. Total free bytes are not the same as one contiguous usable block.

`torch.cuda.max_memory_allocated()` is the maximum allocated value observed since the peak statistics were last reset. It answers "How high did live tensor memory rise during this interval?" It does not answer "How much memory is live now?" The corresponding reserved peak records the high-water mark of the allocator pool. Resetting peak statistics changes only the measurement baseline; it does not free memory.

`torch.cuda.mem_get_info()` obtains a driver-level pair $(M_{\text{free}}, M_{\text{total}})$. Total is the usable capacity reported for the selected device in the current environment, while free is capacity currently available at that system view. Other processes, display workloads, CUDA context state, and non-PyTorch allocations influence free memory. Therefore the residual

$$M_{\text{other}} = M_{\text{total}} - M_{\text{free}} - M_{\text{reserved}}$$

is only an accounting remainder, not proof of a leak.

Part 3 - Quantitative Prediction Before Execution

Before running code, record the idle measurements and predict the directional change produced by each stage. For a tensor with shape $(n_1, \ldots, n_k)$ and element size s bytes, begin with

$$B_{\text{payload}} = s \prod_{i=1}^{k} n_i.$$

For example, a `float32` tensor of shape $(4096, 4096)$ contains 16,777,216 elements and requires

$$16{,}777{,}216 \times 4 = 67{,}108{,}864 \text{ bytes} = 64 \text{ MiB}.$$

Predict that allocated memory will rise by approximately the payload size. Do not demand exact equality before considering allocator rounding, temporary construction, or existing reusable blocks. Predict that reserved memory may rise by the same amount or a larger step. After deleting the tensor and ensuring no references remain, predict that allocated memory will fall while reserved memory may remain unchanged. After `torch.cuda.empty_cache()`, predict that only unused cached blocks may be returned; memory supporting live tensors must remain allocated. Peak allocated memory should remain at its earlier maximum until explicitly reset.

Part 4 - Experimental Design and Instrumentation

Use one explicit device, preferably `cuda:0`. Start a fresh Python process so that earlier notebook allocations do not contaminate the baseline. Synchronize before every snapshot because CUDA execution is asynchronous. Create tensors with `torch.empty` so that the payload size is clear and avoid unrelated model libraries in this first experiment. Store snapshots as ordinary Python dictionaries containing integer byte counts.

Use a sequence of named checkpoints rather than printing values at arbitrary lines. Delete references explicitly, call Python garbage collection only as a controlled intervention, and separate it from `torch.cuda.empty_cache()` so that their roles are not confused. Reset peak statistics immediately before the controlled region. Repeat the experiment twice: the second run may reuse cached blocks and reveal why reserved growth differs between a cold and warm allocator state.

For every checkpoint, record allocated, reserved, allocated peak, reserved peak, driver free, and driver total. Also calculate cached slack and the device-wide used value. The measurement table must state units and must not round before differences are calculated.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

MIB = 1024 * 1024

def bytes_to_mib(number_of_bytes):
    return number_of_bytes / MIB

def memory_snapshot(label, device):
    torch.cuda.synchronize(device)
    free_bytes, total_bytes = torch.cuda.mem_get_info(device)

    snapshot = {}
    snapshot["label"] = label
    snapshot["allocated_bytes"] = torch.cuda.memory_allocated(device)
    snapshot["reserved_bytes"] = torch.cuda.memory_reserved(device)
    snapshot["peak_allocated_bytes"] = (
        torch.cuda.max_memory_allocated(device)
    )
    snapshot["peak_reserved_bytes"] = (
        torch.cuda.max_memory_reserved(device)
    )
    snapshot["free_bytes"] = free_bytes
    snapshot["total_bytes"] = total_bytes
    snapshot["device_used_bytes"] = total_bytes - free_bytes
    snapshot["cached_slack_bytes"] = (
        snapshot["reserved_bytes"] - snapshot["allocated_bytes"]
    )
    return snapshot

def print_snapshot(snapshot):
    print("\nStage:", snapshot["label"])
    keys = [
        "allocated_bytes",
        "reserved_bytes",
        "peak_allocated_bytes",
```

```
        "peak_reserved_bytes",
        "free_bytes",
        "total_bytes",
        "device_used_bytes",
        "cached_slack_bytes",
    ]

    for key in keys:
        value_in_mib = bytes_to_mib(snapshot[key])
        print(f"{key:24s}: {value_in_mib:10.2f} MiB")

def E00():
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required for DMM00-E00.")

    device = torch.device("cuda:0")
    gc.collect()
    torch.cuda.empty_cache()
    torch.cuda.reset_peak_memory_stats(device)

    snapshots = []
    snapshots.append(memory_snapshot("baseline", device))

    first = torch.empty(
        (4096, 4096),
        dtype=torch.float32,
        device=device,
    )
    snapshots.append(memory_snapshot("after first tensor", device))

    second = torch.empty(
        (8192, 4096),
        dtype=torch.float32,
        device=device,
    )
    snapshots.append(memory_snapshot("after second tensor", device))

    del first
    gc.collect()
    snapshots.append(memory_snapshot("after deleting first", device))

    del second
    gc.collect()
    snapshots.append(memory_snapshot("after deleting all", device))

    torch.cuda.empty_cache()
    snapshots.append(memory_snapshot("after empty_cache", device))
```

```
    for snapshot in snapshots:
        print_snapshot(snapshot)

    return snapshots

if __name__ == "__main__":
    E00()
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate four invariants. First, allocated memory must never exceed reserved memory at a synchronized snapshot. Second, free memory must not exceed total memory. Third, peak allocated memory must be at least the current allocated memory. Fourth, deleting a tensor must not be credited with a reduction unless all references to its storage are gone. Add assertions for these invariants, but use explanations rather than overly strict equality checks for allocator deltas.

Compare the measured allocation increase with the predicted payload. Report the absolute error and explain likely sources. If allocated memory fails to drop after deletion, inspect containers, closures, global variables, exception tracebacks, views, and autograd graphs for surviving references. If reserved memory remains high, do not label it a leak until the live allocation counter, snapshot history, and cache-release experiment have been examined. If driver free memory changes between otherwise identical runs, check for display activity and other processes.

Run the experiment in a fresh process and then inside a long-lived notebook. The notebook is expected to have a less clean baseline. This difference is part of the lesson: memory measurements are observations of a process history, not immutable properties of one line of source code.

Part 7 - Engineering Interpretation and Philosophy

Memory is not one number because ownership is layered. A tensor has logical meaning to the model, storage meaning to PyTorch, block meaning to the caching allocator, allocation meaning to CUDA, and physical-capacity meaning to the device. Each interface reveals one layer and hides another. Engineering errors occur when a number from one layer is used to answer a question belonging to a different layer.

The deeper philosophy is that unused memory is not automatically wasted memory. A reserved block may represent deliberate preparation for reuse, reducing latency and synchronization in later steps. Conversely, high device utilization is not automatically productive if the bytes belong to stale graphs or fragmented blocks. The correct judgment depends on lifetime, reusability, and the workload's next demand.

This exercise establishes the discipline for the rest of the book: define the observer, define the time interval, predict the mechanism, and only then interpret the measurement. Dynamic memory management begins when a printed number becomes an explainable state transition.

19.2.2 Exercise 01

Exercise Title: Predict CUDA Memory Usage from Tensor Shape, Dtype, and Element Count Before Running the Code

Part 1 - Exercise Contract and Why It Matters

This exercise trains the habit that must precede every serious memory experiment: calculate the payload before asking the GPU to allocate it. You will build a predictor that accepts an explicit shape and dtype, computes the element count without hidden shortcuts, converts it to bytes and MiB, and then checks the prediction against the increase reported by PyTorch. The predictor must support at least `float16`, `bfloat16`, `float32`, `float64`, `int8`, `int32`, and `bool`.

The exercise is successful when you can reject an unsafe allocation before it is attempted, explain the difference between tensor payload and complete runtime cost, and identify shape mistakes whose memory effect is multiplicative. A forgotten batch or head dimension is not a small bookkeeping error; it can change an allocation by orders of magnitude.

A second requirement is to make integer overflow and unit confusion visible. Very large shapes should be multiplied using Python integers, not a fixed-width tensor scalar that may overflow before the capacity check. The report must show both decimal gigabytes and binary GiB only when each unit is labeled. A request described casually as "eight gigabytes" can differ by hundreds of millions of bytes depending on the convention, which is already large enough to determine whether an allocation succeeds near the boundary.

Finally, the predictor must refuse shapes whose meaning is incomplete. A batch dimension, sequence dimension, number of heads, or number of layers must never be silently assumed. The reader should treat a shape as a resource contract: every axis is named, its maximum is stated, and any dynamic axis is included in the budget formula.

Part 2 - System and Memory Mental Model

A dense strided tensor normally stores one value for every logical element. Its payload is determined by the number of elements and the number of bytes per element:

$$N = \prod_{i=1}^{k} n_i, \qquad B = N\, s_{\text{dtype}}.$$

This formula describes storage, not the cost of computing the tensor. A view created with `reshape`, `transpose`, or slicing may share an existing storage and allocate no new payload. A materializing operation such as `clone()` or `contiguous()` may create new storage with the same element count. Sparse and quantized layouts require different accounting. The predictor in this exercise is deliberately restricted to dense tensors so that its contract is explicit.

The tensor payload also excludes Python objects, allocator metadata, CUDA context memory, kernel workspaces, autograd-saved tensors, and future outputs. It is therefore the first line of a budget rather than a prediction of the entire process.

Part 3 - Quantitative Prediction Before Execution

Manually calculate three cases before running the program:

$(32, 3, 224, 224)$, `float32`; $(8, 16, 2048, 2048)$, `float16`; $(2, 32, 4096, 4096)$, `float64`.

Write every multiplication and convert with 1 MiB = 2^{20} bytes and 1 GiB = 2^{30} bytes. Compare each predicted payload with a conservative safety budget equal to a selected fraction of current driver-free memory. Explain why using 100 percent of reported free memory is unsafe: temporary operations, allocator rounding, and unrelated activity still need headroom.

Part 4 - Experimental Design and Instrumentation

Test one shape at a time in a fresh controlled function. Record allocated memory before allocation, allocate with `torch.empty`, synchronize, record the new value, and calculate the delta. Delete the tensor after each case. Keep prediction arithmetic in integer bytes and round only for display. Include a deliberately unsafe shape, but reject it through the budget check instead of allocating it.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def element_count(shape):
    count = 1
    for dimension in shape:
        if dimension < 0:
            raise ValueError("Dimensions must be non-negative.")
        count = count * dimension
    return count

def predict_dense_tensor_bytes(shape, dtype):
    count = element_count(shape)
    probe = torch.empty((), dtype=dtype)
    bytes_per_element = probe.element_size()
    return count * bytes_per_element

def safe_to_allocate(predicted_bytes, device, safety_fraction):
    free_bytes, total_bytes = torch.cuda.mem_get_info(device)
    allowed_bytes = int(free_bytes * safety_fraction)
    return predicted_bytes <= allowed_bytes, allowed_bytes, total_bytes

def measure_case(shape, dtype, device):
    predicted_bytes = predict_dense_tensor_bytes(shape, dtype)
    permitted, allowed_bytes, total_bytes = safe_to_allocate(
        predicted_bytes,
        device,
        0.70,
    )

    if not permitted:
        return {
            "shape": shape,
            "dtype": str(dtype),
            "predicted_bytes": predicted_bytes,
```

```
            "status": "rejected before allocation",
        }

    torch.cuda.synchronize(device)
    before = torch.cuda.memory_allocated(device)
    tensor = torch.empty(shape, dtype=dtype, device=device)
    torch.cuda.synchronize(device)
    after = torch.cuda.memory_allocated(device)

    result = {
        "shape": shape,
        "dtype": str(dtype),
        "predicted_bytes": predicted_bytes,
        "measured_delta_bytes": after - before,
        "allowed_bytes": allowed_bytes,
        "total_bytes": total_bytes,
        "status": "measured",
    }

    del tensor
    gc.collect()
    return result

def E01():
    device = torch.device("cuda:0")
    cases = [
        ((32, 3, 224, 224), torch.float32),
        ((8, 16, 2048, 2048), torch.float16),
        ((2, 32, 4096, 4096), torch.float64),
    ]

    results = []
    for shape, dtype in cases:
        results.append(measure_case(shape, dtype, device))
    return results
```

Part 6 - Measurement, Validation, and Failure Analysis

Require exact agreement between `tensor.numel()` and the manually calculated element count. Compare payload prediction with allocated delta and report both absolute and percentage error. Investigate a mismatch before changing tolerance. Common causes include a live result from a previous case, construction of a temporary tensor, an accidental copy, or measuring reserved instead of allocated memory. Test a zero-sized dimension and explain why its logical payload is zero even though a tensor object still exists.

Part 7 - Engineering Interpretation and Philosophy

Prediction is a form of control. Without it, the GPU becomes an oracle that is asked whether an idea fits only after the program risks failure. Shape algebra allows the engineer to move that decision earlier, where alternatives such as smaller batches, lower precision, tiling, or

offloading can still be selected. The deeper lesson is that dimensions are resource commitments. Every new axis has semantic meaning and a physical price.

19.2.3 Exercise 02

Exercise Title: Measure Live Tensor Memory Using torch.cuda.memory_allocated() Before and After CUDA Allocations

Part 1 - Exercise Contract and Why It Matters

This exercise isolates the meaning of live allocated memory. You will create a sequence of CUDA tensors with known payloads, retain selected references, create views and clones, and observe which operations change `torch.cuda.memory_allocated()`. The final report must distinguish logical tensors from independent storages. Two tensor variables do not necessarily represent two allocations, while one expression may create several temporary storages.

The contract requires synchronized snapshots, byte-accurate deltas, and an explanation for every increase or decrease. The experiment must include an original tensor, an alias assignment, a slice view, a transpose view, a clone, a dtype conversion, and deletion in a controlled order.

This distinction becomes especially important with non-contiguous layouts. Transposing a tensor changes its strides and logical indexing but ordinarily does not rearrange the underlying bytes. A later operator may accept those strides directly or may create a contiguous copy internally. Therefore the memory cost cannot always be assigned to the transpose line itself; it may appear at the first downstream operation that requires a particular layout. The experiment should preserve this causal chain.

You must also explain why storage-pointer inspection is a debugging aid rather than a complete ownership API. Allocator reuse can cause a pointer value to appear again after an earlier tensor dies, and zero-sized tensors have special cases. Pointer equality during overlapping lifetimes supports a sharing claim, but pointer history alone does not define semantic identity.

Part 2 - System and Memory Mental Model

PyTorch tensors contain metadata that refers to storage. An alias such as `b = a` creates another Python reference to the same tensor object. A view such as `a[:, :512]` may create a new tensor object with different shape and strides while sharing the original storage. Neither operation needs another full payload. A clone creates independent storage. A conversion from `float32` to `float16` also materializes new storage when the dtype changes.

Storage lifetime ends only when no live tensor or object retains a reference to it and no pending CUDA work still requires it. Deleting the original variable cannot free the storage when a view survives. This relationship is central to leak diagnosis because the variable that appears responsible may not be the last owner.

Part 3 - Quantitative Prediction Before Execution

Use an original $(4096, 4096)$ `float32` tensor with a 64 MiB payload. Predict zero allocated delta for an alias, approximately zero for a view, 64 MiB for a clone, and 32 MiB for a `float16` conversion. Predict that deleting the original while its view remains will not release the original 64 MiB storage. Construct a ledger that lists storage identity, dtype, payload, and surviving owners at every stage.

Part 4 - Experimental Design and Instrumentation

Start from a clean baseline and use one snapshot function. Perform only one operation between snapshots. Use `tensor.untyped_storage().data_ptr()` to compare storage pointers for educational inspection. Do not infer sharing from equal values. Synchronize before each snapshot and preserve Python references deliberately until their assigned deletion stage.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def allocated(device):
    torch.cuda.synchronize(device)
    return torch.cuda.memory_allocated(device)

def storage_pointer(tensor):
    return tensor.untyped_storage().data_ptr()

def E02():
    device = torch.device("cuda:0")
    gc.collect()
    torch.cuda.empty_cache()

    records = []
    records.append(("baseline", allocated(device)))

    original = torch.empty(
        (4096, 4096),
        dtype=torch.float32,
        device=device,
    )
    records.append(("original", allocated(device)))

    alias = original
    records.append(("alias", allocated(device)))

    view = original[:, :512]
    records.append(("view", allocated(device)))

    clone = original.clone()
    records.append(("clone", allocated(device)))

    half_copy = original.to(dtype=torch.float16)
    records.append(("half copy", allocated(device)))

    assert storage_pointer(alias) == storage_pointer(original)
    assert storage_pointer(view) == storage_pointer(original)
    assert storage_pointer(clone) != storage_pointer(original)
```

```
    del original
    del alias
    gc.collect()
    records.append(("original names deleted", allocated(device)))

    del view
    gc.collect()
    records.append(("view deleted", allocated(device)))

    del clone
    del half_copy
    gc.collect()
    records.append(("all storages deleted", allocated(device)))
    return records
```

Part 6 - Measurement, Validation, and Failure Analysis

Convert adjacent snapshots into deltas and compare them with the storage ledger. If a view unexpectedly allocates a large block, verify whether the operation was actually a materializing reshape or contiguous conversion. If memory remains after all named tensors are deleted, inspect the records container and ensure it stores only integers rather than tensors. Also inspect interactive notebook output history, which may retain the last displayed tensor. Repeat inside a standalone script to separate notebook retention from allocator behavior.

Part 7 - Engineering Interpretation and Philosophy

Logical identity and physical ownership are different. A program may contain many tensor descriptions of one storage, or one innocent-looking expression may create a costly copy. Dynamic memory management therefore requires reasoning about storage graphs, not variable names alone. The philosophical lesson is that appearances at the language level can conceal continuity at the memory level: deletion of a name is not destruction of the thing it once described.

19.2.4 Exercise 03

Exercise Title: Measure Reserved CUDA Memory Using torch.cuda.memory_reserved() and Explain Why It Exceeds Allocated Memory

Part 1 - Exercise Contract and Why It Matters

This exercise studies the caching allocator as a performance mechanism. You will show that reserved memory can remain high after live tensors disappear, then prove that a later allocation can reuse the pool without requesting an equally large new segment from CUDA. The report must avoid calling every allocated–reserved gap fragmentation. Some gap is healthy reusable capacity; fragmentation is a more specific inability to satisfy useful requests from the available block structure.

The exercise must also separate allocator efficiency from device sharing. A high reserved pool may be entirely reasonable for a single long-running training job, yet undesirable when several independent processes must share the same GPU. In the first situation, retention avoids

repeated allocation cost. In the second, returning inactive segments can improve system fairness. The same counter value can therefore be healthy or harmful depending on who needs the capacity next.

Your conclusion must include both a local and a system-level interpretation: whether PyTorch can reuse the pool efficiently and whether retaining that pool prevents another workload from making progress.

Part 2 - System and Memory Mental Model

The allocator receives segments from CUDA and subdivides them into blocks. Allocated blocks support live tensors. Inactive blocks may remain reserved and become candidates for reuse. If the next request matches an available block, PyTorch can satisfy it internally. If it does not fit, the allocator may split a larger block, request a new segment, or fail despite apparently large slack. Thus,

$$M_{\text{reserved}} = M_{\text{active}} + M_{\text{inactive}},$$

is a capacity identity, not a statement that all inactive bytes are equally usable. The cache-release operation returns eligible unused blocks to CUDA but cannot release blocks occupied by live tensor storage.

Part 3 - Quantitative Prediction Before Execution

Predict allocated and reserved values for a 256 MiB tensor, its deletion, a new 192 MiB tensor, and a cache-release request. The exact reserved segment sizes are allocator-dependent, so write directional hypotheses: allocation raises allocated; deletion lowers allocated; reserved may remain; reuse may raise allocated without a comparable rise in reserved; emptying the cache may lower reserved only after live tensors are removed.

Part 4 - Experimental Design and Instrumentation

Measure both counters at every stage and include driver free memory. Run a cold trial after clearing eligible cache and a warm trial without clearing it. Synchronize snapshots. Use the same allocation pattern in both trials and record execution time only as supporting evidence because this chapter focuses on capacity semantics.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def counters(label, device):
    torch.cuda.synchronize(device)
    return {
        "label": label,
        "allocated": torch.cuda.memory_allocated(device),
        "reserved": torch.cuda.memory_reserved(device),
    }

def one_trial(device, release_cache_first):
    gc.collect()
```

```
    if release_cache_first:
        torch.cuda.empty_cache()

    rows = []
    rows.append(counters("start", device))

    first = torch.empty(
        (64, 1024, 1024),
        dtype=torch.float32,
        device=device,
    )
    rows.append(counters("256 MiB live", device))

    del first
    gc.collect()
    rows.append(counters("256 MiB deleted", device))

    second = torch.empty(
        (48, 1024, 1024),
        dtype=torch.float32,
        device=device,
    )
    rows.append(counters("192 MiB live", device))

    del second
    gc.collect()
    rows.append(counters("all deleted", device))

    torch.cuda.empty_cache()
    rows.append(counters("cache released", device))
    return rows

def E03():
    device = torch.device("cuda:0")
    cold = one_trial(device, True)
    warm = one_trial(device, False)
    return {"cold": cold, "warm": warm}
```

Part 6 - Measurement, Validation, and Failure Analysis

Calculate reserved slack at each stage and annotate whether live storage exists. Prove reuse by showing a substantial allocated increase with little or no reserved increase during the warm allocation. If reserved does not fall to zero after `empty_cache()`, check for live tensors and remember that some context or non-releasable allocator state can remain. Do not use `empty_cache()` inside every training iteration; benchmark the synchronization and allocation penalty if you are tempted to do so.

Part 7 - Engineering Interpretation and Philosophy

The caching allocator demonstrates that apparent excess can be intentional preparedness. A system may hold more memory than it currently needs because future reuse is cheaper than repeated acquisition and release. The engineering question is not whether reserved exceeds allocated, but whether the pool serves the workload efficiently and yields capacity when another consumer genuinely needs it. Good memory management balances present occupancy with future cost.

19.2.5 Exercise 04

Exercise Title: Track Peak VRAM with max_memory_allocated() and Reset Peak Statistics Between Controlled Experiments

Part 1 - Exercise Contract and Why It Matters

Current memory can miss the event that determines whether a workload fits. A temporary attention matrix, backward workspace, or optimizer update may exist for only milliseconds and disappear before an end-of-step snapshot. This exercise uses peak statistics to capture that high-water mark. You must prove that the peak persists after current memory falls, that resetting statistics does not free storage, and that each compared experiment needs its own reset boundary.

Peak statistics are also needed when two implementations finish with identical outputs and identical final allocations. A fused operation may create a lower temporary peak than a sequence of individually materialized intermediates. End-of-function measurement would declare them equal even though only one fits a constrained GPU. This exercise therefore establishes peak memory as an independent engineering metric alongside correctness, latency, and throughput.

The report must include the baseline current allocation at the moment of reset. Without that baseline, a raw peak combines persistent model state with temporary demand and cannot reveal how much capacity the measured operation added.

Part 2 - System and Memory Mental Model

Let $A(t)$ be allocated tensor memory during a measured interval. The peak is

$$A_{\max} = \max_{t_0 \le t \le t_1} A(t).$$

`max_memory_allocated()` stores this historical maximum, whereas `memory_allocated()` samples the present. A reset sets the new measurement baseline to the current state; it does not make the current state zero and does not return blocks to the driver. Peaks are device-specific and process-specific, so the selected device and interval must be documented.

Part 3 - Quantitative Prediction Before Execution

Begin with a persistent 64 MiB tensor, reset the peak, create a temporary 256 MiB tensor, delete it, and retain the persistent tensor. Predict a current value near 64 MiB after deletion and a peak near 320 MiB. Reset again and predict that both current and new peak begin near 64 MiB. Then create a 128 MiB temporary and predict a second-interval peak near 192 MiB.

Part 4 - Experimental Design and Instrumentation

Use two functions whose temporary allocations differ while their final live state is identical. Reset immediately before each function and synchronize after it. Record current allocated,

peak allocated, current reserved, and peak reserved. Run the functions in both orders to show why failure to reset makes the second result inherit the first high-water mark.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def run_temporary_case(device, temporary_mib):
    element_count = temporary_mib * 1024 * 1024 // 4
    temporary = torch.empty(
        element_count,
        dtype=torch.float32,
        device=device,
    )
    temporary.fill_(1.0)
    del temporary
    gc.collect()
    torch.cuda.synchronize(device)

    return {
        "current_allocated": torch.cuda.memory_allocated(device),
        "peak_allocated": torch.cuda.max_memory_allocated(device),
        "current_reserved": torch.cuda.memory_reserved(device),
        "peak_reserved": torch.cuda.max_memory_reserved(device),
    }

def E04():
    device = torch.device("cuda:0")
    persistent = torch.empty(
        64 * 1024 * 1024 // 4,
        dtype=torch.float32,
        device=device,
    )

    torch.cuda.reset_peak_memory_stats(device)
    large_case = run_temporary_case(device, 256)

    torch.cuda.reset_peak_memory_stats(device)
    small_case = run_temporary_case(device, 128)

    del persistent
    gc.collect()
    return {"large": large_case, "small": small_case}
```

Part 6 - Measurement, Validation, and Failure Analysis

Check that each peak is at least the corresponding current value and that the large temporary produces a larger peak. If filling the tensor introduces an unexpected workspace, compare

with `torch.empty` alone and explain which case represents committed use. If repeated trials differ, inspect baseline allocations and allocator state. Report deltas from the current baseline, rather than comparing raw peaks from unrelated process histories.

Part 7 - Engineering Interpretation and Philosophy

Capacity failure is governed by the maximum simultaneous demand, not by the memory visible when the program is calm. Peak tracking turns a fleeting event into durable evidence, but only when the observer defines the interval. The philosophical lesson is that measurement depends on boundaries: a maximum without a declared beginning and end is a number without an experiment.

19.2.6 Exercise 05

Exercise Title: Compare Theoretical Tensor Memory Against Observed PyTorch CUDA Memory for float16, float32, and float64

Part 1 - Exercise Contract and Why It Matters

Precision changes capacity directly, but the common claim that `float16` always halves total training memory is incomplete. This exercise first proves the exact payload ratio for isolated dense tensors and then explains why a complete model may contain mixed dtypes, master weights, optimizer states, gradients, workspaces, and alignment overhead. You will use the same shape for three dtypes and compare theoretical bytes, allocated deltas, reserved deltas, and numerical range.

The comparison must also resist a common benchmarking error: changing the dtype of only the input while the operator promotes or converts it internally. Inspect the actual result dtype and the dtype of every live tensor. A valid capacity comparison requires the intended storage format to survive throughout the measured region. If an operation lacks support for a selected dtype and creates a higher-precision temporary, that behavior is part of the measured system and must be reported.

Alongside capacity, include one numerical stress case containing small and large magnitudes. The purpose is not to decide that one dtype is universally better, but to show that the bytes saved have a measurable representational consequence.

Part 2 - System and Memory Mental Model

The element sizes are normally

$$s_{\text{float16}} = 2, \qquad s_{\text{float32}} = 4, \qquad s_{\text{float64}} = 8 \text{ bytes}.$$

For a fixed dense shape, payload ratios are $1 : 2 : 4$. Allocated deltas should closely follow those ratios. Reserved deltas may not, because the allocator uses block and segment policies and can reuse earlier blocks. Precision also changes representable range and rounding behavior; memory savings cannot be evaluated independently of correctness.

Part 3 - Quantitative Prediction Before Execution

For shape $(64, 1024, 1024)$, calculate the element count and predict 128 MiB for `float16`, 256 MiB for `float32`, and 512 MiB for `float64`. Predict allocated ratios and explicitly state that reserved ratios may be distorted by trial order. Estimate how many such tensors could fit inside a chosen 2 GiB experimental budget while leaving headroom.

Part 4 - Experimental Design and Instrumentation

Run each dtype in a separate clean subprocess if exact reserved comparison is required. Within one process, clear eligible cache before each trial and record the remaining baseline. Allocate with `torch.empty`, then write finite values so that pages are meaningfully used. Measure payload prediction, allocated delta, reserved delta, and maximum absolute error after converting a controlled input down and back to `float64`.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def dtype_trial(shape, dtype, device):
    gc.collect()
    torch.cuda.empty_cache()
    torch.cuda.synchronize(device)

    before_allocated = torch.cuda.memory_allocated(device)
    before_reserved = torch.cuda.memory_reserved(device)

    tensor = torch.empty(shape, dtype=dtype, device=device)
    tensor.fill_(0.125)
    torch.cuda.synchronize(device)

    after_allocated = torch.cuda.memory_allocated(device)
    after_reserved = torch.cuda.memory_reserved(device)

    theoretical_bytes = tensor.numel() * tensor.element_size()
    result = {
        "dtype": str(dtype),
        "element_size": tensor.element_size(),
        "theoretical_bytes": theoretical_bytes,
        "allocated_delta": after_allocated - before_allocated,
        "reserved_delta": after_reserved - before_reserved,
    }

    del tensor
    gc.collect()
    return result

def E05():
    device = torch.device("cuda:0")
    shape = (64, 1024, 1024)
    dtypes = [torch.float16, torch.float32, torch.float64]

    results = []
    for dtype in dtypes:
        results.append(dtype_trial(shape, dtype, device))
    return results
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify `element_size()` rather than hard-coding an unsupported assumption. Compare allocated delta with theoretical bytes and calculate the ratio across dtypes. If a trial OOMs, stop and reduce the common shape instead of allowing different shapes to invalidate the comparison. If reserved deltas appear out of order, repeat in fresh processes and attribute the original result to cache history rather than precision itself.

Part 7 - Engineering Interpretation and Philosophy

A dtype is simultaneously a storage format, an arithmetic contract, and a statement about tolerated error. Choosing precision only from memory pressure ignores numerical meaning; choosing it only from accuracy ignores capacity and throughput. Good engineering treats precision as a negotiated boundary between representation and resources, and validates both sides of that negotiation.

19.2.7 Exercise 06

Exercise Title: Allocate and Delete CUDA Tensors Repeatedly and Measure How Allocated and Reserved Memory Evolve Over Time

Part 1 - Exercise Contract and Why It Matters

One allocation provides a snapshot; a runtime produces a history. This exercise constructs repeated allocation and deletion cycles so that you can separate stable cache reuse, expected warm-up growth, unbounded live-memory growth, and fragmentation-like behavior. The result must be a time series, not a final number. You will compare a stable workload with a deliberately leaking workload that retains one tensor per iteration.

Repeated variable-size workloads are representative of real systems: variable-length sequences, changing image resolutions, and concurrent inference requests do not ask for one perfectly repeating block. The stable trial should therefore include both a fixed-size pattern and a variable pattern. This lets the reader observe that a pool can stabilize quickly for one workload yet continue changing for another even when neither leaks live tensors.

The deliverable must mark three regions in the curve: cold initialization, warm steady state, and deliberate leak. If a plateau cannot be identified, extend the run or reduce unrelated variability. A diagnosis based on five iterations is not strong enough for a long-horizon memory claim.

Part 2 - System and Memory Mental Model

In a stable loop, allocated memory should return near its baseline after each iteration once temporary references disappear. Reserved memory may grow during early iterations as the allocator learns the workload's size pattern, then reach a plateau. In a leak, allocated memory grows because additional storage remains live. These signatures are different:

$$\text{stable cache: } A_t \approx A_0, \quad R_t \to R^*,$$

$$\text{retained tensors: } A_{t+1} > A_t.$$

Reserved growth without allocated growth is not sufficient evidence of a Python reference leak, though it may motivate allocator snapshot analysis.

Part 3 - Quantitative Prediction Before Execution

Choose temporary sizes of 32, 48, 64, and 96 MiB in a repeating pattern. Predict that the stable trial's end-of-iteration allocated delta approaches zero. Predict a finite reserved plateau large enough to service the pattern. For the leak trial, retain one 8 MiB tensor per iteration and predict an approximately linear allocated slope of 8 MiB per iteration.

Part 4 - Experimental Design and Instrumentation

Run at least 30 iterations. Synchronize before every record and capture allocated, reserved, peak, driver free, and iteration label. Use two independent trials in fresh process states where possible. Plot allocated and reserved MiB against iteration. Do not call `empty_cache()` inside the stable loop, because that would remove the allocator behavior being studied.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def make_float32_tensor(mib, device):
    count = mib * 1024 * 1024 // 4
    return torch.empty(count, dtype=torch.float32, device=device)

def record(iteration, mode, device):
    torch.cuda.synchronize(device)
    return {
        "iteration": iteration,
        "mode": mode,
        "allocated": torch.cuda.memory_allocated(device),
        "reserved": torch.cuda.memory_reserved(device),
        "peak": torch.cuda.max_memory_allocated(device),
    }

def run_pattern(device, keep_references):
    sizes = [32, 48, 64, 96]
    retained = []
    rows = []

    for iteration in range(30):
        size_index = iteration % len(sizes)
        temporary = make_float32_tensor(sizes[size_index], device)
        temporary.fill_(1.0)

        if keep_references:
            leak_piece = make_float32_tensor(8, device)
            retained.append(leak_piece)

        del temporary
        gc.collect()
        rows.append(record(iteration, str(keep_references), device))
```

```
    return rows, retained

def E06():
    device = torch.device("cuda:0")
    stable_rows, stable_retained = run_pattern(device, False)
    del stable_retained
    gc.collect()
    torch.cuda.empty_cache()

    leak_rows, leak_retained = run_pattern(device, True)
    return stable_rows, leak_rows, leak_retained
```

Part 6 - Measurement, Validation, and Failure Analysis

Calculate the allocated slope over the final ten iterations for both trials. The stable slope should be near zero; the leaking slope should approximate the retained payload. Inspect any staircase in reserved memory and relate it to the size pattern. If the stable allocated baseline rises, look for tensors stored inside measurement records, exception objects, closures, or asynchronous work. After proving the leak, delete the retained container and verify that allocated memory falls.

Part 7 - Engineering Interpretation and Philosophy

Leaks are temporal claims. A large memory value may be completely legitimate, while a small but persistent positive slope can eventually destroy a long service. The engineer must therefore study trends, plateaus, and phase changes. The philosophy of this exercise is that system health lives in trajectories: memory becomes understandable when we observe how ownership changes through time rather than judging one isolated frame.

19.2.8 Exercise 07

Exercise Title: Compare PyTorch CUDA Memory Measurements Against torch.cuda.mem_get_info() and Explain Every Difference

Part 1 - Exercise Contract and Why It Matters

PyTorch allocator counters describe memory managed through PyTorch, while the CUDA driver memory-information call describes free and total capacity at the CUDA device level. This exercise reconciles both views without forcing them into false equality. You will build a ledger that measures a controlled tensor, calculates device-wide used memory, and identifies the residual not explained by PyTorch reserved memory. The residual must be described as a category to investigate, not automatically as a leak.

The comparison must be performed as a reconciliation across stages, not as a single subtraction at program startup. Creating the first CUDA tensor can initialize context and library state, so the first driver-level delta may be larger than the tensor payload while PyTorch allocated remains exact. Later allocations may reuse reserved segments and produce the opposite pattern: PyTorch allocated rises while driver free barely changes. Both observations are expected under the layered model.

The final table must classify every measured delta as live PyTorch storage, inactive PyTorch reserve, or outside the PyTorch reserve. It is acceptable for the last category to remain partly unexplained, but it is not acceptable to silently call it tensor memory.

Part 2 - System and Memory Mental Model

Define

$$D = T - F, \qquad P = R, \qquad U = D - P,$$

where T is driver-reported total, F is driver-reported free, D is device-wide used, R is PyTorch reserved, and U is the unexplained remainder relative to the PyTorch pool. U may include CUDA contexts, cuBLAS or cuDNN allocations, graphics use, other processes, or measurement timing. PyTorch allocated memory A is a subset of R, not a value that should be subtracted independently from device use after R has already been counted.

Part 3 - Quantitative Prediction Before Execution

Record a baseline (A_0, R_0, F_0, T), then predict the effect of a 512 MiB tensor. Expect A to rise by approximately 512 MiB and F to fall only if the allocator needs new device segments. If cached capacity already exists, allocated may rise while free remains nearly unchanged. Predict what changes after deletion and after cache release.

Part 4 - Experimental Design and Instrumentation

Take synchronized snapshots in a standalone process. Record the operating device and check that no second process is intentionally started during the trial. Perform the allocation twice to expose cold and warm behavior. Store raw bytes, not only rounded MiB, and calculate all residuals after collection.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def reconcile(label, device):
    torch.cuda.synchronize(device)
    free_bytes, total_bytes = torch.cuda.mem_get_info(device)
    allocated_bytes = torch.cuda.memory_allocated(device)
    reserved_bytes = torch.cuda.memory_reserved(device)
    device_used_bytes = total_bytes - free_bytes
    residual_bytes = device_used_bytes - reserved_bytes

    return {
        "label": label,
        "allocated": allocated_bytes,
        "reserved": reserved_bytes,
        "free": free_bytes,
        "total": total_bytes,
        "device_used": device_used_bytes,
        "outside_pytorch_reserved": residual_bytes,
    }
```

```
def E07():
    device = torch.device("cuda:0")
    gc.collect()
    torch.cuda.empty_cache()

    rows = []
    rows.append(reconcile("baseline", device))

    tensor = torch.empty(
        512 * 1024 * 1024 // 4,
        dtype=torch.float32,
        device=device,
    )
    rows.append(reconcile("tensor live", device))

    del tensor
    gc.collect()
    rows.append(reconcile("tensor deleted", device))

    torch.cuda.empty_cache()
    rows.append(reconcile("cache released", device))
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Compare deltas rather than demanding that raw baselines match across tools. Verify that total remains stable during the trial. A negative residual indicates that snapshots or definitions were combined incorrectly and should trigger investigation. If driver free memory changes while PyTorch counters do not, look for another process or a non-PyTorch CUDA allocation. Repeat after closing GPU-using applications and document the environment rather than deleting uncomfortable results.

Part 7 - Engineering Interpretation and Philosophy

No single observer owns the complete truth of a layered system. PyTorch knows its allocator; the driver knows device capacity; neither counter alone knows the semantic purpose of every byte. Reconciliation is the practice of allowing multiple valid perspectives to coexist while defining their boundaries. The lesson is not to search for one magical number, but to build an accounting model in which every number has a responsible observer.

19.2.9 Exercise 08

Exercise Title: Build a Reusable PyTorch Function That Reports Allocated, Reserved, Peak, Free, and Total GPU Memory in MiB

Part 1 - Exercise Contract and Why It Matters

Repeated manual print statements produce inconsistent labels, units, device selection, and synchronization. This exercise turns the definitions learned so far into a reusable reporting component. The function must support an explicit CUDA device, return machine-readable integers,

optionally synchronize, display MiB only at the presentation layer, and include timestamps and stage labels. It must never reset peaks or empty the cache as a hidden side effect.

The utility will become infrastructure for later chapters, so its behavior must be conservative. It should raise on invalid device selection, preserve full precision in raw counters, and avoid depending on whichever GPU happens to be current in the process. A report gathered from the wrong device can look perfectly plausible, making explicit placement more important than decorative formatting.

Add a version field to the schema when the utility is reused in saved experiments. If the meaning or calculation of a field later changes, historical records must remain interpretable rather than being silently read under a new definition.

Part 2 - System and Memory Mental Model

Instrumentation should observe a system without silently changing the property being observed. Synchronization is itself an intervention because it changes timing, but it may be required for a consistent stage boundary. Peak reset and cache release are stronger interventions and belong in the caller. Separating collection, derived metrics, formatting, and persistence makes the tool auditable and prevents unit conversion from contaminating raw evidence.

Part 3 - Quantitative Prediction Before Execution

Define the report schema before writing code. Required byte fields are allocated, reserved, peak allocated, peak reserved, free, and total. Derived fields are cached slack, device used, allocated fraction of total, and reserved fraction of total. Predict all invariants and specify how a zero total value would be handled defensively.

Part 4 - Experimental Design and Instrumentation

Test the reporter at baseline, with a known tensor live, and after deletion. Call it for every visible GPU so device selection errors can be detected. Validate JSON serialization and ensure that the raw dictionary contains no tensor values. Compare synchronized and unsynchronized modes and explain why the synchronized mode is preferred for chapter experiments.

Part 5 - PyTorch/CUDA Implementation

```
from datetime import datetime, timezone
import json
import torch

MIB = 1024 * 1024

def collect_cuda_memory(label, device, synchronize=True):
    selected = torch.device(device)
    if selected.type != "cuda":
        raise ValueError("A CUDA device is required.")

    if synchronize:
        torch.cuda.synchronize(selected)

    free_bytes, total_bytes = torch.cuda.mem_get_info(selected)
    allocated_bytes = torch.cuda.memory_allocated(selected)
```

```
    reserved_bytes = torch.cuda.memory_reserved(selected)

    report = {}
    report["timestamp_utc"] = datetime.now(timezone.utc).isoformat()
    report["label"] = label
    report["device"] = str(selected)
    report["allocated_bytes"] = allocated_bytes
    report["reserved_bytes"] = reserved_bytes
    report["peak_allocated_bytes"] = (
        torch.cuda.max_memory_allocated(selected)
    )
    report["peak_reserved_bytes"] = (
        torch.cuda.max_memory_reserved(selected)
    )
    report["free_bytes"] = free_bytes
    report["total_bytes"] = total_bytes
    report["cached_slack_bytes"] = reserved_bytes - allocated_bytes
    report["device_used_bytes"] = total_bytes - free_bytes
    return report

def format_memory_report(report):
    formatted = dict(report)
    for key in report:
        if key.endswith("_bytes"):
            formatted[key.replace("_bytes", "_mib")] = (
                report[key] / MIB
            )
    return formatted

def E08():
    report = collect_cuda_memory("baseline", "cuda:0", True)
    formatted = format_memory_report(report)
    print(json.dumps(formatted, indent=2))
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Write assertions for allocated not exceeding reserved, peak not falling below current, free not exceeding total, and non-negative cached slack. Test an invalid CPU device and an unavailable CUDA index. Verify that formatting does not replace or round raw byte fields. If multi-GPU reports are identical, confirm that each API received the selected device rather than relying on the current default device.

Part 7 - Engineering Interpretation and Philosophy

Reusable measurement code is part of scientific infrastructure. Its value comes from stable semantics, not visual decoration. A reporter that secretly clears memory may produce comforting numbers while destroying the phenomenon under investigation. The philosophical principle is that an instrument should declare how it touches the world it measures.

19.2.10 Exercise 09

Exercise Title: Demonstrate Why Deleting a CUDA Tensor Does Not Necessarily Make the Same Amount of VRAM Immediately Appear Free

Part 1 - Exercise Contract and Why It Matters

The statement `del tensor` removes one Python reference; it is not a command that promises the driver an immediate increase in free VRAM. This exercise separates three events: reference removal, live-storage reclamation by PyTorch, and release of unused cached blocks to CUDA. You must demonstrate a case where allocated memory falls after deletion but driver free memory changes little because the caching allocator retains the block.

The exercise must distinguish delayed release caused by legitimate pending GPU work from retention caused by a Python reference. Synchronization can establish a clean diagnostic boundary, but it should not be inserted into production code without considering lost overlap. A block that cannot yet be reused by another stream is a scheduling issue; a block whose tensor remains reachable is an ownership issue; an inactive block held in the pool is an allocator-policy issue. Their visible symptom may initially be the same.

Your report should therefore describe deletion as the beginning of an investigation rather than proof that the device must immediately report more free memory.

Part 2 - System and Memory Mental Model

Python reference counting and garbage collection determine when tensor objects become unreachable. PyTorch then marks the associated allocator block unused when it is safe to do so. The block can remain reserved for reuse. Pending work on another CUDA stream may delay safe reuse, and a surviving view or autograd graph may keep storage genuinely live. Thus deletion, deallocation, cache retention, and device release are separate transitions.

Part 3 - Quantitative Prediction Before Execution

Allocate a 512 MiB tensor. Predict an allocated increase near 512 MiB and a driver-free decrease only when new segments are requested. Delete the sole reference and predict an allocated decrease near 512 MiB, a small reserved change, and little driver-free recovery. After cache release, predict a larger driver-free increase. Repeat while keeping a view and predict that allocated memory will not fall until the view is deleted.

Part 4 - Experimental Design and Instrumentation

Use two trials: sole-owner deletion and surviving-view deletion. Synchronize before measurements. Call garbage collection separately from cache release. Record storage pointers to prove the view owns access to the same storage. Do not create a measurement list that stores tensors.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def state(label, device):
    torch.cuda.synchronize(device)
    free_bytes, total_bytes = torch.cuda.mem_get_info(device)
```

```
    return (
        label,
        torch.cuda.memory_allocated(device),
        torch.cuda.memory_reserved(device),
        free_bytes,
        total_bytes,
    )

def E09():
    device = torch.device("cuda:0")
    gc.collect()
    torch.cuda.empty_cache()
    rows = [state("baseline", device)]

    tensor = torch.empty(
        512 * 1024 * 1024 // 4,
        dtype=torch.float32,
        device=device,
    )
    rows.append(state("live", device))

    view = tensor[:1024]
    del tensor
    gc.collect()
    rows.append(state("original deleted, view alive", device))

    del view
    gc.collect()
    rows.append(state("all references deleted", device))

    torch.cuda.empty_cache()
    rows.append(state("unused cache released", device))
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Explain each counter transition. If allocated drops before the view is deleted, verify that the chosen slice truly shares the original storage. If it never drops, inspect hidden references and synchronize pending work. If driver free does not recover exactly the payload after cache release, account for allocator rounding and other device consumers rather than forcing equality.

Part 7 - Engineering Interpretation and Philosophy

Deletion expresses loss of one name, not immediate physical surrender of capacity. High-level languages, frameworks, allocators, and devices cooperate through delayed ownership transitions. This separation is useful because reuse is fast, but it requires conceptual patience: the engineer must ask which layer has released what, to whom, and for which future use.

19.2.11 Exercise 10

Exercise Title: Experiment with torch.cuda.empty_cache() and Prove What Memory It Releases and What Memory It Cannot Release

Part 1 - Exercise Contract and Why It Matters

`torch.cuda.empty_cache()` is frequently presented as a cure for CUDA memory problems. This exercise replaces that myth with a precise contract. You will call it while a large tensor is live, after the tensor is deleted, and inside a repeated allocation loop. You must prove that it cannot release live tensor storage, may release eligible unused cached blocks, does not increase the memory available to PyTorch beyond freeing its own unused pool, and may reduce performance when called unnecessarily.

You must examine one situation in which releasing the cache is operationally useful: for example, a notebook that has finished a large temporary experiment and will allow a second process to use the device. This prevents the exercise from replacing one myth with another. The function is neither a universal cure nor universally wrong; it is a policy intervention whose value depends on the next consumer and the cost of rebuilding the pool.

The deliverable must state the exact trigger that would justify calling it in a real system. "Memory looks high" is not a sufficient trigger.

Part 2 - System and Memory Mental Model

The operation asks the caching allocator to return releasable unused blocks to CUDA. It does not invalidate tensors, detach graphs, remove Python references, move data to CPU, or reduce a tensor's payload. If allocated memory is close to reserved memory because nearly all blocks are live, little can be returned. If reserved greatly exceeds allocated after a temporary peak, more cache may be eligible. Releasing blocks can help another process obtain capacity or alter fragmentation state, but later PyTorch allocations may need expensive device allocation again.

Part 3 - Quantitative Prediction Before Execution

For a live 256 MiB tensor, predict no meaningful allocated reduction after cache release. After deleting it, predict allocated reduction first and possible reserved reduction only after cache release. For a loop of 100 repeated allocations, predict higher latency when cache is emptied every iteration than when the allocator reuses its pool.

Part 4 - Experimental Design and Instrumentation

Build a semantic trial and a performance trial. The semantic trial records allocated, reserved, and driver free at each stage. The performance trial uses warm-up, CUDA events, synchronization, identical tensor sizes, and several repetitions. Do not combine timing from a cold first allocation with warm steady-state results.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def measure_loop(device, clear_each_iteration, iterations):
    start_event = torch.cuda.Event(enable_timing=True)
```

```
    end_event = torch.cuda.Event(enable_timing=True)

    start_event.record()
    for iteration in range(iterations):
        tensor = torch.empty(
            (32, 1024, 1024),
            dtype=torch.float32,
            device=device,
        )
        tensor.add_(1.0)
        del tensor
        if clear_each_iteration:
            gc.collect()
            torch.cuda.empty_cache()
    end_event.record()
    torch.cuda.synchronize(device)
    return start_event.elapsed_time(end_event)

def E10():
    device = torch.device("cuda:0")
    live = torch.empty(
        (64, 1024, 1024),
        dtype=torch.float32,
        device=device,
    )

    before = torch.cuda.memory_allocated(device)
    torch.cuda.empty_cache()
    while_live = torch.cuda.memory_allocated(device)
    assert while_live >= before

    del live
    gc.collect()
    after_delete = torch.cuda.memory_allocated(device)
    reserved_before_release = torch.cuda.memory_reserved(device)
    torch.cuda.empty_cache()
    reserved_after_release = torch.cuda.memory_reserved(device)

    reuse_ms = measure_loop(device, False, 100)
    clear_ms = measure_loop(device, True, 100)
    return {
        "after_delete": after_delete,
        "reserved_before_release": reserved_before_release,
        "reserved_after_release": reserved_after_release,
        "reuse_ms": reuse_ms,
        "clear_every_time_ms": clear_ms,
    }
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify that live allocated memory survives the cache call. Report how much reserved memory becomes driver-free only after references are removed. Repeat timing trials and use the median because host garbage collection adds noise. If cache clearing appears faster in one short run, increase repetitions and separate warm-up. Never conclude that it fixes a reference leak unless allocated memory also falls for a separately proven reason.

Part 7 - Engineering Interpretation and Philosophy

`empty_cache()` is a negotiation between PyTorch and the wider device, not a garbage collector for user mistakes. Its misuse reflects a desire for a ritual that makes numbers smaller without understanding ownership. The better practice is causal: release references, verify liveness, diagnose the pool, and use cache release only when returning inactive capacity serves a defined system goal.

19.2.12 Exercise 11

Exercise Title: Detect a Hidden CUDA Tensor Reference That Prevents Memory from Being Released and Fix the Leak

Part 1 - Exercise Contract and Why It Matters

Many PyTorch leaks are not failures to free memory; they are failures to stop owning it. This exercise creates three hidden-reference patterns: appending a loss tensor to a Python list, retaining a view after deleting its base name, and holding an autograd graph through a tensor with `grad_fn`. You will measure the positive allocated-memory slope, locate the reference, repair it, and add a regression assertion that the stable version no longer grows.

The leak must be diagnosed without immediately calling `empty_cache()`. Cache release can change reserved memory and distract from the live allocated slope, but it cannot invalidate a tensor that remains reachable. Begin by proving continued liveness, then identify the ownership path, then repair the representation stored by the program. This order turns the fix into a causal explanation.

Your repaired version must preserve the information actually needed for later analysis. Replacing every tensor with nothing is not a valid fix if the training history is required. Store a scalar, a bounded CPU sample, or an explicit summary whose lifetime and capacity are designed.

Part 2 - System and Memory Mental Model

Python containers, closures, object attributes, exception tracebacks, notebook history, and autograd graphs can extend tensor lifetime. Storing `loss` retains its graph; storing `loss.item()` stores a CPU number. Storing `loss.detach()` removes gradient history but may still retain CUDA storage. Moving the small detached result to CPU prevents that specific GPU residency. A view keeps the underlying storage alive even if the base variable is deleted.

Part 3 - Quantitative Prediction Before Execution

Choose a model and batch whose graph adds a measurable but safe amount per iteration. Predict near-linear allocated growth when raw losses are appended. Predict stable allocated memory when only Python floats are stored. Estimate the slope in MiB per iteration and the iteration count at which an 8 GiB GPU would fail if the slope continued.

Part 4 - Experimental Design and Instrumentation

Run leaking and repaired loops from comparable clean baselines for at least 50 iterations. Record end-of-iteration allocated memory. Keep model, input shape, seed, and optimizer identical. Use `gc.get_referrers()` only as an optional diagnostic because it can be noisy; begin with an ownership inventory of every persistent container.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def run_case(device, keep_graph_tensors, iterations):
    model = torch.nn.Sequential(
        torch.nn.Linear(1024, 2048),
        torch.nn.ReLU(),
        torch.nn.Linear(2048, 1),
    ).to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
    history = []
    memory_rows = []

    for iteration in range(iterations):
        inputs = torch.randn(
            (512, 1024),
            dtype=torch.float32,
            device=device,
        )
        targets = torch.randn(
            (512, 1),
            dtype=torch.float32,
            device=device,
        )

        optimizer.zero_grad(set_to_none=True)
        predictions = model(inputs)
        loss = torch.nn.functional.mse_loss(predictions, targets)
        loss.backward()
        optimizer.step()

        if keep_graph_tensors:
            history.append(loss)
        else:
            history.append(loss.item())

        del inputs
        del targets
        del predictions
        del loss
```

```
        gc.collect()
        torch.cuda.synchronize(device)
        memory_rows.append(torch.cuda.memory_allocated(device))

    return history, memory_rows

def E11():
    device = torch.device("cuda:0")
    leaking_history, leaking_rows = run_case(device, True, 50)
    del leaking_history
    gc.collect()
    torch.cuda.empty_cache()

    safe_history, safe_rows = run_case(device, False, 50)
    return leaking_rows, safe_rows, safe_history
```

Part 6 - Measurement, Validation, and Failure Analysis

Fit a simple end-minus-start slope and compare the last ten iterations with the first ten. Confirm that clearing the leaking history lowers allocated memory. If the repaired loop still grows, inspect optimizer initialization, cuDNN warm-up, variable input shapes, and measurement records. Distinguish a short warm-up plateau from unbounded growth by extending the run. The regression test should allow bounded allocator noise but reject a sustained slope.

Part 7 - Engineering Interpretation and Philosophy

A leak is often valid ownership applied for too long. The framework cannot know that a loss tensor stored in a list was intended only as a number. Memory safety therefore depends on semantic lifetime: the programmer must express when information should survive and in what representation. The deeper lesson is that persistence is a design decision, even when it is introduced accidentally by a convenient container.

19.2.13 Exercise 12

Exercise Title: Measure Forward-Pass and Backward-Pass Memory Separately in a PyTorch Neural Network Training Step

Part 1 - Exercise Contract and Why It Matters

End-of-step memory hides the phases that create peak demand. This exercise divides one training iteration into input creation, forward pass, loss construction, backward pass, optimizer step, and cleanup. You will record current and peak memory after every phase and explain why forward retains activations for backward, why backward creates gradients, and why some saved activations can be released as gradient computation progresses.

Phase separation is also required for comparing memory optimizations. Activation checkpointing primarily changes what survives from forward into backward, gradient clearing changes the state after backward, and optimizer choice changes persistent and update-stage state. Without phase attribution, all three techniques may appear as one unexplained change in a global peak.

The output of this exercise must therefore be a timeline with both current and high-water values. It should show not only where the maximum occurs, but what new objects become live and what earlier objects cease to be needed at each boundary.

Part 2 - System and Memory Mental Model

During forward with gradient tracking, autograd records operations and saves selected tensors needed to compute derivatives. Parameters are persistent; activations are phase-dependent. Backward materializes or accumulates parameter gradients and may release saved intermediates after their final use. The peak can occur during forward, early backward, or optimizer update depending on the model and optimizer. A single "training memory" value therefore lacks enough temporal structure.

Part 3 - Quantitative Prediction Before Execution

Calculate parameter payload and input payload before running. Estimate output sizes layer by layer for a three-layer MLP and identify activations likely saved for backward. Predict that forward current memory exceeds the input-plus- parameter baseline and that backward adds gradient storage approximately equal to the trainable parameter payload when gradients use the same dtype. Do not claim an exact backward peak without measuring temporary buffers.

Part 4 - Experimental Design and Instrumentation

Warm up the model once so initialization does not dominate. Clear gradients by assigning them to `None`, reset peaks, and run one measured step. Synchronize at phase boundaries. Repeat with `torch.no_grad()` for an inference forward pass. Keep input, architecture, batch, and dtype fixed.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def mark(label, device):
    torch.cuda.synchronize(device)
    return {
        "label": label,
        "current": torch.cuda.memory_allocated(device),
        "peak": torch.cuda.max_memory_allocated(device),
    }

def E12():
    device = torch.device("cuda:0")
    model = torch.nn.Sequential(
        torch.nn.Linear(4096, 8192),
        torch.nn.ReLU(),
        torch.nn.Linear(8192, 4096),
        torch.nn.ReLU(),
        torch.nn.Linear(4096, 1000),
    ).to(device)
    optimizer = torch.optim.AdamW(model.parameters(), lr=0.001)

    inputs = torch.randn((128, 4096), device=device)
```

```
    targets = torch.randint(0, 1000, (128,), device=device)
    optimizer.zero_grad(set_to_none=True)
    torch.cuda.reset_peak_memory_stats(device)

    rows = [mark("inputs and model", device)]
    logits = model(inputs)
    rows.append(mark("after forward", device))

    loss = torch.nn.functional.cross_entropy(logits, targets)
    rows.append(mark("after loss", device))

    loss.backward()
    rows.append(mark("after backward", device))

    optimizer.step()
    rows.append(mark("after optimizer step", device))
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Calculate phase deltas and compare them with predicted parameter, activation, and gradient payloads. Use hooks or profiler evidence if backward peak is unexpected. Ensure warm-up does not leave gradients live. Repeat with `optimizer.zero_grad(set_to_none=False)` and explain why zero tensors remain allocated. If inference is not smaller, verify that it ran under `no_grad()` and that training outputs were no longer referenced.

Part 7 - Engineering Interpretation and Philosophy

Training is a choreography of lifetimes, not one static model footprint. A tensor's cost depends on when it overlaps with other tensors. This creates the possibility of checkpointing, offloading, and scheduling: memory can be reduced by changing overlap even when total information processed is unchanged. The philosophical shift is from asking what exists to asking what must coexist.

19.2.14 Exercise 13

Exercise Title: Measure the Additional VRAM Cost of Parameters, Gradients, Activations, and Optimizer State During Training

Part 1 - Exercise Contract and Why It Matters

This exercise constructs a component ledger for training memory. You will measure the incremental cost of model parameters, inputs and forward activations, parameter gradients, and AdamW optimizer state. The final table must compare theoretical payloads with observed phase deltas and identify temporary or allocator costs that do not fit a simple category. The purpose is to replace the misleading belief that model size equals parameter size.

The ledger must be produced for at least two optimizers, such as SGD without momentum and AdamW. This controlled comparison demonstrates that two training runs using the same model, batch, and dtype can have very different persistent footprints. It also reveals lazy initialization: an optimizer object may appear cheap until its first update materializes state for

every parameter.

Report both payload totals obtained by traversing real tensors and phase deltas obtained from allocator counters. Agreement supports the category model; disagreement identifies temporary overlap, early release, or a missing state category that must be investigated.

Part 2 - System and Memory Mental Model

For P parameter elements stored in s_p bytes, parameter payload is Ps_p. Same-dtype gradients can add approximately another Ps_p. Adam-style optimizers commonly maintain first and second moments, adding two state tensors per parameter, often in `float32`. Mixed-precision training may keep higher-precision master parameters. Activations scale with batch and model shape, while parameter and optimizer state do not scale with batch in the same way. Optimizer state is often initialized lazily at the first `step()`, so constructing the optimizer object alone may allocate little CUDA state.

Part 3 - Quantitative Prediction Before Execution

Count every parameter explicitly with `numel()` and calculate parameter, gradient, and two-moment payloads. Predict when each category first appears: parameters after model transfer, activations after forward, gradients after backward, and Adam state after the first optimizer step. State why bias and weight tensors must both be counted.

Part 4 - Experimental Design and Instrumentation

Begin before model creation and take synchronized snapshots after model transfer, input allocation, forward, backward, first optimizer step, and gradient clearing. Reset peaks before the measured step. Traverse optimizer state after initialization and sum tensor payloads directly. Do not assign all unexplained peak bytes to activations.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def parameter_bytes(model):
    total = 0
    for parameter in model.parameters():
        total = total + parameter.numel() * parameter.element_size()
    return total

def gradient_bytes(model):
    total = 0
    for parameter in model.parameters():
        if parameter.grad is not None:
            total = total + (
                parameter.grad.numel() * parameter.grad.element_size()
            )
    return total

def optimizer_state_bytes(optimizer):
    total = 0
    for state in optimizer.state.values():
```

```
        for value in state.values():
            if torch.is_tensor(value):
                total = total + value.numel() * value.element_size()
    return total

def E13():
    device = torch.device("cuda:0")
    baseline = torch.cuda.memory_allocated(device)

    model = torch.nn.Sequential(
        torch.nn.Linear(4096, 4096),
        torch.nn.GELU(),
        torch.nn.Linear(4096, 4096),
    ).to(device)
    after_parameters = torch.cuda.memory_allocated(device)

    optimizer = torch.optim.AdamW(model.parameters(), lr=0.001)
    inputs = torch.randn((256, 4096), device=device)
    targets = torch.randn((256, 4096), device=device)
    optimizer.zero_grad(set_to_none=True)

    outputs = model(inputs)
    loss = torch.nn.functional.mse_loss(outputs, targets)
    after_forward = torch.cuda.memory_allocated(device)

    loss.backward()
    after_backward = torch.cuda.memory_allocated(device)
    optimizer.step()
    after_step = torch.cuda.memory_allocated(device)

    return {
        "observed_parameter_delta": after_parameters - baseline,
        "observed_forward_delta": after_forward - after_parameters,
        "observed_backward_delta": after_backward - after_forward,
        "observed_step_delta": after_step - after_backward,
        "theoretical_parameter_bytes": parameter_bytes(model),
        "live_gradient_bytes": gradient_bytes(model),
        "optimizer_state_bytes": optimizer_state_bytes(optimizer),
    }
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify that the parameter traversal agrees closely with the model-transfer delta. Confirm that optimizer state is near zero before the first step and substantial afterward. Explain step counters or scalar state tensors rather than assuming exactly two full buffers and nothing else. Repeat with SGD and compare state payload. If phase subtraction becomes negative because earlier activations were freed, switch from naive deltas to a lifetime table and peak measurements.

Part 7 - Engineering Interpretation and Philosophy

The memory cost of learning is larger than the memory cost of knowing. A trained parameter stores a result; training must also store evidence about how to change it, historical optimizer information, and intermediate states that assign credit. Memory accounting reveals that optimization algorithms are physical systems with persistent state, not abstract update rules. Choosing an optimizer is therefore also choosing a memory architecture.

19.2.15 Exercise 14

Exercise Title: Sweep Batch Size Until GPU Memory Growth Reaches the Out-of-Memory Boundary and Record the Scaling Curve

Part 1 - Exercise Contract and Why It Matters

Batch size changes activation, input, target, workspace, and sometimes algorithm-selection memory. The relationship with peak VRAM is often close to linear over a stable regime but should never be assumed globally. This exercise builds a bounded sweep, measures peak training memory for increasing micro-batches, fits the observed scaling curve, and estimates the OOM boundary without crashing the primary process. The final result must report the largest validated batch and a conservative recommended batch with explicit headroom.

The curve must also distinguish micro-batch size from effective batch size. Gradient accumulation can preserve a large effective batch while reducing the activation peak of each micro-step, but it changes the number and timing of forward/backward passes. This exercise measures the physical micro-batch boundary first so later chapters can reason about accumulation without confusing optimization semantics with one-step capacity.

Document why the sweep stops. It may stop because the safety budget was reached, the estimated next point was unsafe, the experiment hit its configured maximum, or a controlled child trial observed OOM. These outcomes must not all be labeled as the exact hardware limit.

Part 2 - System and Memory Mental Model

Separate persistent memory P from batch-dependent memory:

$$M_{\text{peak}}(B) \approx P + \alpha B + \epsilon(B).$$

α represents the marginal bytes per sample and $\epsilon(B)$ includes nonlinear workspaces, allocator rounding, and implementation changes. The mathematical boundary $B^* = (M_{\text{budget}} - P)/\alpha$ is only an estimate. A production choice should satisfy a smaller operational budget to survive variation.

Part 3 - Quantitative Prediction Before Execution

Calculate input and target bytes per sample and estimate major activation bytes. Choose candidate batches by gradual growth, such as $8, 16, 24, \ldots$, rather than doubling into an enormous request. Define a measurement budget no larger than 70–80 percent of free memory on a shared or display GPU. Predict the slope from tensor shapes and later compare it with the fitted empirical slope.

Part 4 - Experimental Design and Instrumentation

Warm up the model at the smallest batch. For every candidate, clear references, reset peak statistics, run one forward and backward step, synchronize, record peak allocated and reserved,

then clean up. Stop before the predicted next case exceeds the safety budget. Repeat each validated batch three times and use the maximum peak for capacity planning. Do not use `empty_cache()` as a substitute for consistent lifetime cleanup.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def measured_step(model, batch_size, device):
    model.zero_grad(set_to_none=True)
    torch.cuda.reset_peak_memory_stats(device)

    inputs = torch.randn(
        (batch_size, 4096),
        dtype=torch.float32,
        device=device,
    )
    targets = torch.randint(
        0,
        1000,
        (batch_size,),
        device=device,
    )

    logits = model(inputs)
    loss = torch.nn.functional.cross_entropy(logits, targets)
    loss.backward()
    torch.cuda.synchronize(device)
    peak = torch.cuda.max_memory_allocated(device)

    del inputs
    del targets
    del logits
    del loss
    model.zero_grad(set_to_none=True)
    gc.collect()
    return peak

def E14():
    device = torch.device("cuda:0")
    model = torch.nn.Sequential(
        torch.nn.Linear(4096, 8192),
        torch.nn.GELU(),
        torch.nn.Linear(8192, 1000),
    ).to(device)

    free_bytes, total_bytes = torch.cuda.mem_get_info(device)
    safety_budget = int(free_bytes * 0.75)
```

```
    results = []

    batch_size = 8
    while batch_size <= 4096:
        peaks = []
        for repetition in range(3):
            peak = measured_step(model, batch_size, device)
            peaks.append(peak)

        worst_peak = max(peaks)
        results.append((batch_size, worst_peak))

        if worst_peak >= safety_budget:
            break
        batch_size = batch_size + 8

    return {
        "results": results,
        "safety_budget": safety_budget,
        "device_total": total_bytes,
    }
```

Part 6 - Measurement, Validation, and Failure Analysis

Plot peak MiB against batch size and compute local slopes between adjacent points. Identify discontinuities rather than hiding them behind one regression line. Confirm that every point performed the same training computation. If a candidate unexpectedly OOMs, record the last successful batch, terminate that trial cleanly, and do not continue from a possibly damaged state. Compare the empirical safe maximum with the model-based prediction and explain the margin.

Part 7 - Engineering Interpretation and Philosophy

The largest batch that succeeds once is not the largest batch that can be trusted. Reliability requires distance from the cliff because memory demand is not perfectly stationary. Capacity planning is therefore a problem of margins, not records. The philosophy is to learn the shape of the boundary while refusing to confuse proximity to failure with efficient use of hardware.

19.2.16 Exercise 15

Exercise Title: Trigger a Controlled CUDA Out-of-Memory Error and Capture Memory Statistics Immediately Before Failure

Part 1 - Exercise Contract and Why It Matters

An OOM handler that has never been tested is only a hope. This exercise creates a controlled failure inside a child process, captures a pre-attempt memory snapshot, records the exception text, and proves that the parent process survives. The experiment must never be run blindly on a GPU driving critical work. On a shared or display GPU, replace the physical OOM with a simulated budget exception. A dedicated secondary GPU is preferred.

The deliverable includes a simulated failure path because production recovery logic should be testable without consuming a whole device. Define a custom budget exception raised before allocation when a request violates a configured limit. First prove that logging and cleanup work under simulation; only then, and only on suitable hardware, repeat with the real allocator exception. The two paths should produce the same structured failure record.

An OOM is considered controlled only when the target device, maximum attempted request, process boundary, expected exception, cleanup behavior, and success criteria were all specified before execution. Catching an accidental crash afterward does not satisfy the contract.

Part 2 - System and Memory Mental Model

CUDA execution is asynchronous, so an error may surface at a synchronization point later than the operation that initiated pressure. An allocation failure does not necessarily mean all theoretical bytes were in live tensors; fragmentation, reserved blocks, driver consumers, and temporary requests matter. The exception object is evidence, but allocator statistics and a snapshot immediately before the failing request provide the context needed to interpret it. Isolating the test in a child process gives the CUDA context a clear recovery boundary.

Part 3 - Quantitative Prediction Before Execution

Read driver free memory and choose a requested tensor slightly larger than the allowed free capacity. Convert requested bytes into `float32` elements using ceiling division. Predict that the request fails without becoming a live allocation. Record how allocated, reserved, and free values differ before and after exception handling. Do not pre-fill the GPU to 100 percent merely to force failure.

Part 4 - Experimental Design and Instrumentation

Use `multiprocessing` with the spawn start method. The child selects an explicit device, warms the CUDA context, clears eligible cache, records its pre-failure snapshot, and attempts one oversized allocation. It catches only the expected out-of-memory class or a RuntimeError whose message is validated. The parent checks the exit code and reads a small CPU result through a queue. Run a simulated policy failure first.

Part 5 - PyTorch/CUDA Implementation

```
import math
import multiprocessing as mp
import torch

def child_oom_experiment(device_index, output_queue):
    device = torch.device(f"cuda:{device_index}")
    torch.cuda.set_device(device)
    torch.cuda.empty_cache()
    free_bytes, total_bytes = torch.cuda.mem_get_info(device)

    before = {
        "allocated": torch.cuda.memory_allocated(device),
        "reserved": torch.cuda.memory_reserved(device),
        "free": free_bytes,
        "total": total_bytes,
```

```
    }

    requested_bytes = free_bytes + (64 * 1024 * 1024)
    requested_elements = math.ceil(requested_bytes / 4)

    try:
        impossible = torch.empty(
            requested_elements,
            dtype=torch.float32,
            device=device,
        )
        output_queue.put({
            "unexpected_success": True,
            "before": before,
        })
        del impossible
    except torch.OutOfMemoryError as error:
        output_queue.put({
            "unexpected_success": False,
            "before": before,
            "requested_bytes": requested_bytes,
            "error": str(error),
        })

def E15(device_index=1):
    if torch.cuda.device_count() <= device_index:
        raise RuntimeError(
            "Use simulated OOM unless a dedicated CUDA device exists."
        )

    context = mp.get_context("spawn")
    output_queue = context.Queue()
    process = context.Process(
        target=child_oom_experiment,
        args=(device_index, output_queue),
    )
    process.start()
    process.join()

    if process.exitcode != 0:
        raise RuntimeError("OOM child did not exit cleanly.")
    return output_queue.get()
```

Part 6 - Measurement, Validation, and Failure Analysis

Confirm that the child caught the expected error and exited with code zero. Preserve the full exception only in the artifact because wording varies across versions. Verify that the parent can run a small CUDA operation afterward. If the allocation unexpectedly succeeds, stop rather than requesting a larger unbounded tensor; re-check the free-memory measurement

and virtual-memory behavior. For a workstation with two RTX 3070 cards, use the non-display card when available and close other GPU workloads first.

Part 7 - Engineering Interpretation and Philosophy

Failure is part of a system's contract. Controlled failure testing converts OOM from a frightening accident into an observable state transition with a safe boundary. Yet experimentation has an ethical constraint: understanding a limit does not justify destabilizing shared hardware. The mature engineer designs a failure that teaches the mechanism while containing its consequences.

19.2.17 Exercise 16

Exercise Title: Build a Memory Budget Estimator and Compare Its Predicted Peak VRAM Against Real PyTorch Measurements

Part 1 - Exercise Contract and Why It Matters

Tensor payload prediction becomes useful at system scale only when separate components are combined into a budget. This exercise builds an estimator for parameters, gradients, optimizer state, inputs, layer outputs, saved activations, and configurable headroom. It then compares predicted peak VRAM with measured peak allocation for a known MLP. The report must expose every assumption; a single unexplained multiplier is not an acceptable estimator.

The estimator must support two modes. The explanatory mode exposes every category and favors transparency. The admission-control mode receives current free memory and decides whether a proposed batch may begin with a configurable margin. The second mode must be built from the first rather than hiding its logic inside an unexplained threshold.

Evaluate the estimator on shapes not used for calibration. A model that matches only one batch or architecture has not learned a resource law; it has recorded one measurement. Generalization here means that the accounting assumptions continue to describe how memory changes when a controlled dimension changes.

Part 2 - System and Memory Mental Model

A first-order training budget can be written as

$$M_{\text{estimate}} = M_P + M_G + M_O + M_I + M_A + M_T + M_H,$$

where parameter, gradient, optimizer, input, saved activation, temporary, and headroom terms are separate. Some objects overlap in lifetime and others do not, so adding every tensor ever created overestimates peak. Omitting temporaries underestimates it. The estimator should therefore use a phase lifetime table and state which terms coexist at the predicted peak.

Part 3 - Quantitative Prediction Before Execution

For each linear layer, compute weight and bias elements. Calculate same-dtype gradients and Adam first and second moments. Derive activation payload from batch and layer widths. Add an empirical temporary allowance only after reporting the subtotal it modifies. Produce a predicted range with lower and upper bounds rather than pretending that unknown kernel workspaces are exact.

Part 4 - Experimental Design and Instrumentation

Use several batch sizes while holding architecture fixed. Predict before each run, then reset peaks and measure one warmed training step. Record absolute error, percentage error, and error direction. Calibrate temporary allowance on one batch and evaluate it on different batches; otherwise the estimator merely memorizes one observation.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def tensor_payload_bytes(tensor):
    return tensor.numel() * tensor.element_size()

def model_parameter_bytes(model):
    total = 0
    for parameter in model.parameters():
        total = total + tensor_payload_bytes(parameter)
    return total

def estimate_mlp_training_bytes(
    model,
    batch_size,
    layer_widths,
    optimizer_multiplier,
    temporary_fraction,
):
    parameters = model_parameter_bytes(model)
    gradients = parameters
    optimizer_state = int(parameters * optimizer_multiplier)

    activation_elements = 0
    for width in layer_widths:
        activation_elements = (
            activation_elements + batch_size * width
        )
    activations = activation_elements * 4

    subtotal = (
        parameters
        + gradients
        + optimizer_state
        + activations
    )
    temporary = int(subtotal * temporary_fraction)
    return {
        "parameters": parameters,
        "gradients": gradients,
        "optimizer_state": optimizer_state,
        "activations": activations,
        "temporary_allowance": temporary,
```

```
        "predicted_peak": subtotal + temporary,
    }

def E16():
    device = torch.device("cuda:0")
    model = torch.nn.Sequential(
        torch.nn.Linear(4096, 8192),
        torch.nn.ReLU(),
        torch.nn.Linear(8192, 4096),
        torch.nn.ReLU(),
        torch.nn.Linear(4096, 1000),
    ).to(device)

    estimates = []
    batch_sizes = [16, 32, 64, 128]
    for batch_size in batch_sizes:
        estimate = estimate_mlp_training_bytes(
            model,
            batch_size,
            [4096, 8192, 4096, 1000],
            2.0,
            0.10,
        )
        estimates.append((batch_size, estimate))
    return estimates
```

Part 6 - Measurement, Validation, and Failure Analysis

Measure the real warmed training peak and compare it with the predicted range. If error grows with batch, the activation model is incomplete. If error is roughly constant, persistent state or baseline context may be missing. If the optimizer estimate is wrong, inspect actual state tensors after the first step. Never repair the model by adding an arbitrary constant without naming the mechanism it represents.

Part 7 - Engineering Interpretation and Philosophy

An estimator is not valuable because it predicts perfectly. It is valuable because its errors are structured enough to teach us what the mental model omitted. Prediction and measurement form a dialogue: theory proposes a ledger, hardware reveals the residual, and the next model becomes more precise. The philosophy is disciplined approximation, not false certainty.

19.2.18 Exercise 17

Exercise Title: Compare Training and Inference CUDA Memory Footprints for the Same Model, Batch Size, and Input Shape

Part 1 - Exercise Contract and Why It Matters

The same parameters can produce very different memory footprints depending on whether the runtime must preserve a graph and compute updates. This exercise compares a warmed training step with evaluation under ordinary grad mode, no-grad mode, and inference mode.

Model, input, batch, dtype, and output shape must remain identical. The report must separate current memory after execution from peak memory during execution.

The experiment must include two baselines. The research baseline keeps the optimizer alive so the same process can return to training. The deployment baseline loads only model state needed for prediction. Mixing these cases causes optimizer moments to be blamed on inference even though a serving process would never create them. Both measurements are useful, but they answer different operational questions.

Also inspect output retention. An inference service that appends GPU outputs to a request history can leak despite using inference mode correctly. Autograd control removes graphs; it does not design application-level lifetime.

Part 2 - System and Memory Mental Model

Training requires forward activations saved for backward, parameter gradients, and optimizer state. Evaluation mode changes module behavior such as dropout and BatchNorm, but `model.eval()` alone does not disable autograd. `no_grad()` prevents graph recording for operations inside its context. `inference_mode()` applies stronger inference-oriented restrictions and can remove additional autograd bookkeeping. These controls answer different questions and should be used together with correct model mode.

Part 3 - Quantitative Prediction Before Execution

Calculate parameter and input payloads. Predict that trained optimizer state remains persistent if the optimizer is kept alive during inference comparison. For a clean deployment footprint, predict again after discarding optimizer and gradients. Expect training peak to exceed inference peak because activations and gradients overlap. Predict that `eval()` without no-grad remains closer to training forward memory than proper inference.

Part 4 - Experimental Design and Instrumentation

Use identical pre-created inputs. Warm each mode separately, remove outputs, reset peak statistics, and run several repetitions. For fair model-only inference, set gradients to None and delete optimizer state or perform the test in a new process loaded only with model weights. Verify numerical parity between no-grad and inference mode within tolerance.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def measure_inference(model, inputs, device, use_inference_mode):
    model.eval()
    torch.cuda.reset_peak_memory_stats(device)

    if use_inference_mode:
        with torch.inference_mode():
            outputs = model(inputs)
    else:
        with torch.no_grad():
            outputs = model(inputs)
```

```
    torch.cuda.synchronize(device)
    peak = torch.cuda.max_memory_allocated(device)
    copied = outputs.detach().cpu()
    del outputs
    gc.collect()
    return peak, copied

def measure_training(model, inputs, targets, device):
    model.train()
    model.zero_grad(set_to_none=True)
    torch.cuda.reset_peak_memory_stats(device)
    outputs = model(inputs)
    loss = torch.nn.functional.cross_entropy(outputs, targets)
    loss.backward()
    torch.cuda.synchronize(device)
    peak = torch.cuda.max_memory_allocated(device)
    del outputs
    del loss
    return peak

def E17():
    device = torch.device("cuda:0")
    model = torch.nn.Sequential(
        torch.nn.Linear(4096, 8192),
        torch.nn.ReLU(),
        torch.nn.Linear(8192, 1000),
    ).to(device)
    inputs = torch.randn((128, 4096), device=device)
    targets = torch.randint(0, 1000, (128,), device=device)

    training_peak = measure_training(
        model, inputs, targets, device
    )
    model.zero_grad(set_to_none=True)
    no_grad_peak, output_a = measure_inference(
        model, inputs, device, False
    )
    inference_peak, output_b = measure_inference(
        model, inputs, device, True
    )

    assert torch.allclose(output_a, output_b, atol=1e-5, rtol=1e-5)
    return training_peak, no_grad_peak, inference_peak
```

Part 6 - Measurement, Validation, and Failure Analysis

Report both absolute peaks and deltas above the persistent model-plus-input baseline. If inference appears larger, inspect surviving training gradients, outputs, or optimizer state. Confirm `model.eval()` was used and that the comparison did not include first-call kernel initial-

ization in only one mode. Verify predictions, because a smaller footprint obtained through changed dtype or shape is not the same experiment.

Part 7 - Engineering Interpretation and Philosophy

One model does not have one memory cost. Cost belongs to a mode of use and a lifetime policy. Training asks the system to remember how an answer was constructed so it can assign credit; inference asks only for the answer. The contrast shows that memory is partly the price of future possibility: gradients require the past because the model intends to change.

19.2.19 Exercise 18

Exercise Title: Build a Live GPU Memory Monitor That Records PyTorch CUDA Memory at Every Stage of a Training Iteration

Part 1 - Exercise Contract and Why It Matters

Manual snapshots are useful but easy to omit. This exercise builds a reusable stage monitor implemented as a context manager. It records timestamps, allocated and reserved memory, peaks, driver free memory, stage duration, and exceptions. The monitor must store only CPU metadata so that measurement does not retain the CUDA tensors it is supposed to observe. It will instrument data transfer, forward, loss, backward, optimizer step, and cleanup.

The monitor must remain useful after the tutorial. Add optional run, epoch, iteration, and request identifiers so records from concurrent or repeated work can be grouped. A stage label alone is ambiguous when hundreds of iterations share the same name. The schema should support threshold warnings without printing tensors or invoking expensive snapshots on every normal step.

Design two operating levels: lightweight continuous counters and a heavy diagnostic mode activated after a threshold or anomaly. This reflects a real observability trade-off: enough evidence must be collected to detect change, but the measurement system must not dominate the workload it protects.

Part 2 - System and Memory Mental Model

A stage is a semantic interval with an entry and exit. CUDA kernels launched inside it may outlive the host statements unless the monitor synchronizes. Synchronization improves attribution but changes overlap and timing, so the monitor should offer an explicit synchronized diagnostic mode. The observer must also distinguish phase peak from global peak by resetting at the stage entry when phase-local high-water marks are desired.

Part 3 - Quantitative Prediction Before Execution

Before implementation, define the expected ordering of stage peaks. Predict which stages create persistent objects and which should return to baseline. Estimate the log size for N iterations and prove that storing numeric dictionaries is negligible compared with storing tensors. Define warning thresholds as fractions of total and free memory.

Part 4 - Experimental Design and Instrumentation

Run five training iterations and compare stage reports. The first iteration is warm-up and should be labeled rather than mixed with steady state. Validate that the monitor logs an exception stage without suppressing the exception. Export JSON lines so a partial log remains readable if the process ends.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

class CudaMemoryStage:
    def __init__(self, label, device, records, synchronize=True):
        self.label = label
        self.device = torch.device(device)
        self.records = records
        self.synchronize = synchronize
        self.start_time = None

    def __enter__(self):
        if self.synchronize:
            torch.cuda.synchronize(self.device)
        torch.cuda.reset_peak_memory_stats(self.device)
        self.start_time = time.perf_counter()
        return self

    def __exit__(self, error_type, error_value, traceback):
        if self.synchronize:
            torch.cuda.synchronize(self.device)
        end_time = time.perf_counter()
        free_bytes, total_bytes = torch.cuda.mem_get_info(self.device)

        record = {}
        record["label"] = self.label
        record["duration_seconds"] = end_time - self.start_time
        record["allocated_bytes"] = torch.cuda.memory_allocated(
            self.device
        )
        record["reserved_bytes"] = torch.cuda.memory_reserved(
            self.device
        )
        record["stage_peak_bytes"] = torch.cuda.max_memory_allocated(
            self.device
        )
        record["free_bytes"] = free_bytes
        record["total_bytes"] = total_bytes
        record["failed"] = error_type is not None
        if error_value is None:
            record["error"] = None
        else:
            record["error"] = str(error_value)
        self.records.append(record)
        return False

def E18(model, optimizer, cpu_inputs, cpu_targets):
```

```
    device = torch.device("cuda:0")
    records = []

    with CudaMemoryStage("transfer", device, records):
        inputs = cpu_inputs.to(device)
        targets = cpu_targets.to(device)

    with CudaMemoryStage("forward", device, records):
        predictions = model(inputs)

    with CudaMemoryStage("loss", device, records):
        loss = torch.nn.functional.cross_entropy(
            predictions, targets
        )

    optimizer.zero_grad(set_to_none=True)
    with CudaMemoryStage("backward", device, records):
        loss.backward()

    with CudaMemoryStage("optimizer", device, records):
        optimizer.step()

    return records
```

Part 6 - Measurement, Validation, and Failure Analysis

Check that every record is JSON serializable and contains no CUDA tensor. Compare phase peaks with the global peak from a separate whole-iteration measurement. They need not add because stages occur at different times. Measure monitor overhead with synchronization enabled and disabled. If stage attribution changes, explain the role of asynchronous execution rather than selecting whichever number looks cleaner.

Part 7 - Engineering Interpretation and Philosophy

Observability converts memory from an internal accident into an operational signal. Yet instruments influence systems: synchronization may destroy overlap, and excessive logging may change timing. The mature approach is not invisible measurement, which is impossible, but declared measurement whose disturbance is understood and bounded.

19.2.20 Exercise 19

Exercise Title: Build a Complete Memory Accounting Experiment That Explains Every Major MiB of VRAM Used by a PyTorch Model

Part 1 - Exercise Contract and Why It Matters

The final exercise integrates the entire chapter into an auditable memory accounting report. You will select one PyTorch model and explain every major MiB category during initialization, forward, backward, optimizer update, and inference. Exact byte attribution for every library workspace may be impossible, but every residual must be bounded, named, and investigated.

The report must include theoretical calculations, synchronized measurements, peak timelines, allocator behavior, driver reconciliation, a controlled reference leak, and a correction.

The report must make three types of claim visually distinct. A calculated claim comes from shapes and dtypes; a measured claim comes from an instrument at a specified boundary; an inferred claim explains a residual by combining several observations. Treating an inference as direct measurement makes the ledger look more certain than the evidence permits.

Include a reproducibility package containing raw JSON records, environment metadata, formulas, code version, and plots generated from the raw data. A reader should be able to recalculate every table cell without copying numbers from an image. The report is complete only when it can be challenged and recomputed.

Part 2 - System and Memory Mental Model

Construct a two-dimensional ledger: rows are memory categories and columns are execution phases. Categories include parameters, buffers, inputs, targets, outputs, saved activations, gradients, optimizer state, allocator slack, temporary workspaces, CUDA or library overhead, and external device use. A category occupies a phase only when its storage is live. Peak is produced by the sum of simultaneous residents, not by the sum of every allocation over the whole run.

$$M_{\text{device used}} = M_{\text{PyTorch reserved}} + M_{\text{outside pool}},$$

$$M_{\text{PyTorch reserved}} = M_{\text{live tensor}} + M_{\text{cached slack}}.$$

These identities organize evidence but do not magically identify each tensor. Use module traversal, optimizer-state traversal, phase snapshots, profiler evidence, and memory snapshots to refine the ledger.

Part 3 - Quantitative Prediction Before Execution

Before the final run, calculate parameter, buffer, gradient, optimizer, input, target, and output payloads. Estimate saved activations layer by layer and state which may share storage. Build lower and upper bounds for the expected peak. Choose a residual significance threshold such as 1 percent of total device capacity or 32 MiB, whichever better suits the test device. A residual above the threshold requires investigation, not arbitrary reassignment.

Part 4 - Experimental Design and Instrumentation

Run in a fresh process and save hardware, PyTorch, CUDA, dtype, shape, seed, and allocator configuration metadata. Warm up once, then perform a measured training step with stage monitoring. Repeat inference in a clean state. Export raw byte records, a MiB table, a peak timeline, and a narrative explanation. Introduce one hidden loss reference for several steps, detect it from slope, remove it, and prove recovery.

Part 5 - PyTorch/CUDA Implementation

```
import json
import platform
import torch

def sum_model_storage(model):
```

```
    parameter_bytes = 0
    buffer_bytes = 0

    for parameter in model.parameters():
        parameter_bytes = parameter_bytes + (
            parameter.numel() * parameter.element_size()
        )

    for buffer in model.buffers():
        buffer_bytes = buffer_bytes + (
            buffer.numel() * buffer.element_size()
        )

    return parameter_bytes, buffer_bytes

def build_metadata(device):
    metadata = {}
    metadata["python_platform"] = platform.platform()
    metadata["torch_version"] = torch.__version__
    metadata["cuda_version"] = torch.version.cuda
    metadata["device_name"] = torch.cuda.get_device_name(device)
    metadata["device_count"] = torch.cuda.device_count()
    return metadata

def validate_ledger(ledger):
    required = [
        "parameters",
        "buffers",
        "inputs",
        "activations",
        "gradients",
        "optimizer_state",
        "allocator_slack",
        "driver_residual",
    ]
    missing = []
    for category in required:
        if category not in ledger:
            missing.append(category)
    if len(missing) != 0:
        raise RuntimeError(f"Missing ledger categories: {missing}")

def E19(model, optimizer, experiment_records, output_path):
    device = torch.device("cuda:0")
    parameter_bytes, buffer_bytes = sum_model_storage(model)

    ledger = {}
    ledger["parameters"] = parameter_bytes
```

```
    ledger["buffers"] = buffer_bytes
    # TODO: fill remaining categories from verified chapter tools
    ledger["inputs"] = ...
    ledger["activations"] = ...
    ledger["gradients"] = ...
    ledger["optimizer_state"] = ...
    ledger["allocator_slack"] = ...
    ledger["driver_residual"] = ...

    validate_ledger(ledger)
    report = {
        "metadata": build_metadata(device),
        "ledger_bytes": ledger,
        "timeline": experiment_records,
    }

    with open(output_path, "w", encoding="utf-8") as file:
        json.dump(report, file, indent=2)
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Reconcile the ledger at every phase, not only at peak. Verify tensor payloads by traversing the actual model and optimizer after lazy state initialization. Use allocated-versus-reserved deltas to avoid assigning cache slack to live activations. Compare driver use with PyTorch reserved and document other processes. If the ledger overcounts, inspect shared storage and non-overlapping lifetimes. If it undercounts, investigate temporary workspaces, saved tensors, and external allocations. The final report must state uncertainty honestly.

Part 7 - Engineering Interpretation and Philosophy

Complete accounting does not mean pretending that the runtime is perfectly transparent. It means organizing uncertainty so that major costs are visible, residuals are bounded, and claims can be audited. The chapter began with five numbers that appeared to describe the same memory. It ends with a model of ownership, caching, history, lifetime, and observation that explains why they differ.

The philosophy of dynamic memory management is responsibility for coexistence. Every byte represents a decision that some information should remain available at a particular place and time. Efficient systems do not merely delete more; they preserve exactly what the computation still needs, release what no longer has purpose, and maintain enough evidence to explain the difference.

19.3 DMM01

Chapter Title: PyTorch Profiler for Memory
Specialization: Dynamic Memory Management and CUDA Systems

19.3.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Operator-Level Memory Profiling in PyTorch and Distinguish It from Global CUDA Memory Counters

Part 1 - Exercise Contract and Why It Matters

DMM00 answered system-level questions: how many bytes are currently allocated, how many are reserved by the caching allocator, what peak was reached, and how much capacity the CUDA driver reports as free. Those counters reveal whether a workload fits and when its high-water mark occurs, but they do not identify the operation responsible for a change. This exercise introduces the PyTorch Profiler as an attribution instrument. Its purpose is to connect memory events to operators, execution regions, tensor shapes, call stacks, and CPU or CUDA activities.

The exercise must prevent a common conceptual error: the sum of memory columns in a profiler table is not automatically equal to the global allocated-memory peak. Profiler memory values describe allocation and deallocation events attributed to operators over time. Global peak memory describes the maximum simultaneous live allocation. Operations can allocate at different times, free storage before later operations begin, reuse allocator blocks, call child operations, and retain outputs beyond their own execution interval. Summing attributed events ignores these lifetimes and may double-count nested work.

Your deliverable is a controlled forward-and-backward profile accompanied by a four-layer evidence model. Layer one is the global CUDA state from DMM00. Layer two is profiler event attribution. Layer three is semantic lifetime: which outputs survive and why. Layer four is hardware activity: which CUDA kernels and transfers execute. For every reported memory number, state which layer produced it and which question it can answer.

The experiment is successful only if you can explain all of the following: why a high-memory operator may not determine global peak, why an operator with zero self memory may still have large total memory through children, why a deallocation may appear as a negative event, why CUDA activity is not identical to a high-level PyTorch operator, and why profiler instrumentation changes runtime and therefore must be used selectively.

This distinction matters in real optimization. A global counter can prove that peak memory decreased after a code change, while the profiler can suggest which operator caused the reduction. Neither evidence is sufficient alone. An operator table without global validation can optimize an attribution artifact; a peak counter without operator attribution can reveal a problem but not guide the repair.

Part 2 - System and Memory Mental Model

PyTorch executes high-level operations through a hierarchy. A Python call may dispatch an ATen operator, which may invoke child operators, request allocator blocks, and launch one or more CUDA kernels. The host call can finish before the device work completes. The profiler observes selected activities and associates events using correlation metadata, but the resulting table is a summary of an execution graph rather than a photograph of physical VRAM.

For an operator o, define self memory as memory directly attributed to the operator excluding nested child operators. Total memory includes the operator and descendants in its recorded

call tree. In simplified notation,

$$M_{\text{total}}(o) = M_{\text{self}}(o) + \sum_{c \in \text{children}(o)} M_{\text{total}}(c),$$

subject to the profiler's attribution rules. This identity explains why sorting by self memory and sorting by total memory answer different questions. Self memory highlights the location of direct allocation. Total memory highlights an inclusive region whose implementation may delegate the actual work.

Global allocated memory is a state function $A(t)$. The peak is

$$A_{\max} = \max_t A(t).$$

Profiler memory events are changes ΔA_i associated with events. Reconstructing a timeline requires event order and lifetimes:

$$A(t_k) = A(t_0) + \sum_{i \le k} \Delta A_i.$$

The unordered sum of positive allocations is total allocation traffic, not peak residency. A program may allocate and free the same 100 MiB workspace ten times, generating 1000 MiB of traffic while never holding more than 100 MiB from that workspace at once.

The caching allocator adds another layer. An operator can increase live allocated memory by consuming an already reserved block without decreasing driver free memory. Conversely, releasing a tensor can reduce live allocated memory while the block stays reserved. The profiler and DMM00 counters are therefore complementary views of live storage, allocator ownership, and event attribution.

Part 3 - Quantitative Prediction Before Execution

Before profiling, draw the expected operator graph of a small MLP: matrix multiplication or linear operations, bias addition, activation, second linear operation, loss, backward functions, gradient accumulation, and optimizer update. Calculate parameter, input, output, and major activation payloads. Predict which calls allocate new outputs and which can reuse or view existing storage.

Create two separate predictions. The event prediction ranks operators by expected direct allocation. The residency prediction identifies which tensors coexist at the forward and backward peaks. State explicitly why these rankings may differ. Predict profiler overhead qualitatively: enabling memory tracking adds bookkeeping, recording shapes retains metadata, and collecting stacks requires more host work and storage.

Part 4 - Experimental Design and Instrumentation

Use a fresh Python process, one explicit CUDA device, fixed seed, fixed shapes, and a warmed model. Capture CPU and CUDA activities. Enable memory profiling but leave stack collection disabled for this foundational run. Wrap the whole step in named profiler regions while preserving individual operators. Reset global peak statistics immediately before the active region and take DMM00 snapshots before and after.

Profile one measured training step, not a long uncontrolled loop. Synchronize before entering and after leaving the profiled region so the interval has a clear boundary. Export neither a trace nor a large artifact yet; inspect the event and key-average tables first. Record profiler configuration alongside results because different options create different evidence.

Part 5 - PyTorch/CUDA Implementation

```
import torch
from torch.profiler import ProfilerActivity
from torch.profiler import profile
from torch.profiler import record_function

def global_memory(device):
    torch.cuda.synchronize(device)
    return {
        "allocated": torch.cuda.memory_allocated(device),
        "reserved": torch.cuda.memory_reserved(device),
        "peak": torch.cuda.max_memory_allocated(device),
    }

def E00():
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required for DMM01.")

    device = torch.device("cuda:0")
    torch.manual_seed(7)
    torch.cuda.manual_seed_all(7)

    model = torch.nn.Sequential(
        torch.nn.Linear(4096, 8192),
        torch.nn.ReLU(),
        torch.nn.Linear(8192, 1000),
    ).to(device)
    optimizer = torch.optim.AdamW(model.parameters(), lr=0.001)
    inputs = torch.randn((128, 4096), device=device)
    targets = torch.randint(0, 1000, (128,), device=device)

    # Warm-up initializes lazy state outside the measured interval.
    warm_output = model(inputs)
    warm_loss = torch.nn.functional.cross_entropy(
        warm_output,
        targets,
    )
    warm_loss.backward()
    optimizer.step()
    optimizer.zero_grad(set_to_none=True)
    del warm_output
    del warm_loss
    torch.cuda.synchronize(device)
```

```
    torch.cuda.reset_peak_memory_stats(device)
    before = global_memory(device)

    activities = [
        ProfilerActivity.CPU,
        ProfilerActivity.CUDA,
    ]

    with profile(
        activities=activities,
        profile_memory=True,
        record_shapes=True,
        with_stack=False,
    ) as profiler:
        with record_function("training_step"):
            with record_function("forward"):
                output = model(inputs)
            with record_function("loss"):
                loss = torch.nn.functional.cross_entropy(
                    output,
                    targets,
                )
            with record_function("backward"):
                loss.backward()
            with record_function("optimizer"):
                optimizer.step()
                optimizer.zero_grad(set_to_none=True)

    torch.cuda.synchronize(device)
    after = global_memory(device)

    table = profiler.key_averages().table(
        sort_by="self_cuda_memory_usage",
        row_limit=25,
    )
    print(table)
    return profiler, before, after
```

Part 6 - Measurement, Validation, and Failure Analysis

First verify numerical execution: loss must be finite, parameters must receive gradients before the optimizer clears them, and output shape must remain $(128, 1000)$. Then inspect tables sorted independently by self CUDA memory, total CUDA memory, self CPU memory, and CUDA time. Do not assume the same operator leads every ranking.

Compare the global peak increase with predicted simultaneous residency. Use the profiler to identify allocation hotspots but do not sum all positive memory rows and call the result peak. Inspect negative values as possible deallocations. If CUDA rows are missing, verify activities,

CUDA availability, synchronization, and build support. If operator names differ across PyTorch versions, classify their role rather than hard-coding one expected string.

Run once without the profiler and confirm that outputs match within tolerance. This proves instrumentation did not change the intended computation. Measure wall-clock overhead only as a preliminary observation; a rigorous overhead study belongs to E15.

Part 7 - Engineering Interpretation and Philosophy

Profiling is an act of attribution. Global counters say that memory exists; the profiler proposes where responsibility lies. Responsibility is not the same as blame. A high-memory operator may be implementing the central useful work of the model, while a smaller unnecessary copy may be the better target. Optimization begins by relating cost to purpose.

The profiler also teaches that software operations are nested explanations. The Python line, ATen operator, allocator event, and CUDA kernel are different descriptions of one execution. No layer is the single reality. Each exposes causes that the others compress or hide. Engineering judgment consists of moving between those layers without confusing their measurements.

The philosophical rule for this chapter is evidence triangulation. A memory claim becomes strong when shape arithmetic predicts it, the profiler attributes it, global counters confirm its effect on residency, and numerical tests prove that the optimized computation still means the same thing.

19.3.2 Exercise 01

Exercise Title: Predict Which Operations in a CUDA Model Will Allocate the Most Memory Before Running the PyTorch Profiler

Part 1 - Exercise Contract and Why It Matters

Profilers are most useful when they test a prior model rather than replace thinking. In this exercise you will examine a CUDA network on paper, list every major materialized tensor, calculate its payload, predict the five largest allocating operations, and only then compare the ranking with profiler output. The objective is to expose missing assumptions: broadcasted intermediates, dtype promotion, contiguous copies, saved activations, and backward buffers.

The deliverable must contain an operator hypothesis table with input shapes, output shapes, dtype, element count, predicted bytes, lifetime, and whether the allocation is direct or delegated to a child operation. A ranking with no byte calculation is not a prediction.

Part 2 - System and Memory Mental Model

An operator allocates memory when it materializes storage that cannot be represented as an existing view or reused output. Matrix multiplication usually creates an output of shape determined by its non-contracted axes. Elementwise operations often create an output as large as their broadcast result. A reduction can create a small final tensor but may require temporary storage. Backward operators create gradient tensors and may use saved forward activations.

Operator names do not determine size by themselves. A ReLU over (B, T, D) can allocate more than a matrix multiplication with a small output. The same operation becomes larger when batch, sequence, or feature dimensions grow. Prediction must therefore be shape-specific.

Part 3 - Quantitative Prediction Before Execution

For each dense output, calculate

$$B = s_{\text{dtype}} \prod_i n_i.$$

Include a residual addition, a broadcasted bias, a `contiguous()` call after transpose, and a loss reduction. Predict both forward outputs and same-shaped backward gradients. Rank by predicted direct output bytes, then construct a second ranking by expected lifetime contribution to peak.

Part 4 - Experimental Design and Instrumentation

Freeze the model definition and input shape before profiling. Save the prediction table with a timestamp so it cannot be rewritten after seeing the result. Profile one warmed training step with memory tracking and shapes enabled. Sort by self CUDA memory and preserve the top 20 rows, including unexpected operators.

Part 5 - PyTorch/CUDA Implementation

```
import torch
from torch.profiler import ProfilerActivity, profile

class PredictionModel(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.first = torch.nn.Linear(2048, 4096)
        self.second = torch.nn.Linear(4096, 2048)

    def forward(self, inputs):
        hidden = self.first(inputs)
        activated = torch.nn.functional.gelu(hidden)
        transposed = activated.transpose(0, 1)
        materialized = transposed.contiguous()
        restored = materialized.transpose(0, 1)
        output = self.second(restored)
        return output + inputs

def predict_bytes(shape, dtype):
    count = 1
    for dimension in shape:
        count = count * dimension
    probe = torch.empty((), dtype=dtype)
    return count * probe.element_size()

def E01():
    device = torch.device("cuda:0")
    model = PredictionModel().to(device)
    inputs = torch.randn((256, 2048), device=device)
```

```
    predictions = {}
    predictions["first_linear_output"] = predict_bytes(
        (256, 4096), torch.float32
    )
    predictions["contiguous_copy"] = predict_bytes(
        (4096, 256), torch.float32
    )
    predictions["second_linear_output"] = predict_bytes(
        (256, 2048), torch.float32
    )

    with profile(
        activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
        profile_memory=True,
        record_shapes=True,
    ) as profiler:
        output = model(inputs)
        loss = output.square().mean()
        loss.backward()

    return predictions, profiler.key_averages()
```

Part 6 - Measurement, Validation, and Failure Analysis

Match predictions to profiler operators by semantics, not only exact names. Calculate rank error and byte error. If an unexpected copy leads the table, inspect strides and layout. If predicted activation allocations are absent, check whether fusion or in-place behavior changed materialization. If one operator has large total but low self memory, inspect its children before declaring the prediction wrong.

Part 7 - Engineering Interpretation and Philosophy

A profiler can show what happened, but prediction explains whether the result was surprising. Surprise is scientifically valuable because it locates a gap between our model and the runtime. The goal is not to guess every internal operator name; it is to construct shape-and-lifetime reasoning precise enough that discrepancies produce new knowledge rather than passive acceptance.

19.3.3 Exercise 02

Exercise Title: Profile a Minimal CUDA Forward Pass with torch.profiler.profile() and Identify CPU and CUDA Activities Correctly

Part 1 - Exercise Contract and Why It Matters

This exercise establishes the smallest trustworthy profiling workflow. You will profile one CUDA forward pass, capture both CPU and CUDA activities, and explain the relationship between a host-side operator event and the device kernels it launches. The output must identify CPU time, CUDA time, call count, and synchronization boundaries without claiming that host duration equals kernel duration.

The minimal workflow must also establish what the profiler cannot infer from a single pass. Kernel duration does not reveal whether the GPU was underutilized before the kernel, and call count does not prove that an operation was necessary. This first profile is a map of execution, not an optimization verdict. Preserve it as a reference against which later memory-enabled runs can be compared.

Your report should name the stream used by the work when the trace exposes it and identify any CUDA runtime calls associated with launch or synchronization. This makes the CPU–GPU relationship concrete before more complex multi-stream experiments appear later in the book.

Part 2 - System and Memory Mental Model

PyTorch dispatch begins on the CPU even when tensors live on CUDA. The CPU prepares and launches device work; CUDA streams execute kernels asynchronously. The profiler correlates activities, but they occupy different timelines. Self CPU time describes host work excluding children. CUDA time summarizes associated device execution. A small CPU duration can launch a long kernel, and a large CPU duration can represent framework overhead with little GPU work.

Part 3 - Quantitative Prediction Before Execution

Predict the high-level calls for one linear layer and ReLU. Calculate input, weight, bias, and output shapes. Predict at least one matrix-multiplication kernel and one activation kernel, while acknowledging that fusion and library implementation can change the exact kernel names. State which activity should dominate for a sufficiently large input.

Part 4 - Experimental Design and Instrumentation

Warm up five iterations. Synchronize before entering and after leaving the profile. Capture one measured pass, retain the output until profiling ends, and compare with an unprofiled result. Print one table sorted by CUDA time and one by self CPU time. Keep memory tracking disabled here so activity interpretation is isolated.

Part 5 - PyTorch/CUDA Implementation

```
import torch
from torch.profiler import ProfilerActivity, profile

def E02():
    device = torch.device("cuda:0")
    layer = torch.nn.Linear(4096, 4096).to(device)
    inputs = torch.randn((1024, 4096), device=device)

    for iteration in range(5):
        warm_output = torch.relu(layer(inputs))
    del warm_output
    torch.cuda.synchronize(device)

    with profile(
        activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
    ) as profiler:
        output = torch.relu(layer(inputs))
```

```
        torch.cuda.synchronize(device)

    print(profiler.key_averages().table(
        sort_by="cuda_time_total",
        row_limit=20,
    ))
    print(profiler.key_averages().table(
        sort_by="self_cpu_time_total",
        row_limit=20,
    ))
    return output, profiler
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify output shape and finiteness. Identify which rows are high-level ATen operations and which are CUDA kernels or runtime calls. If CUDA time is absent, check profiler activities and the PyTorch build. If the first call dominates, increase warm-up. Remove the synchronization inside the region and observe how host timing becomes misleading, then restore the correct boundary.

Part 7 - Engineering Interpretation and Philosophy

Asynchrony separates intention from completion. The CPU can request work and continue before the GPU finishes, so source-code order is not a complete timing model. Profiling teaches the engineer to follow execution across agents rather than assuming that a Python line contains the whole event. Correct attribution begins by respecting multiple timelines.

19.3.4 Exercise 03

Exercise Title: Enable profile_memory=True and Rank PyTorch Operators by Self CUDA Memory Usage

Part 1 - Exercise Contract and Why It Matters

This exercise activates operator-level memory attribution and builds a ranked hotspot report. You will sort by self CUDA memory, preserve positive and negative entries, and explain why the leader is the operation directly associated with allocation rather than necessarily the semantic layer that requested it. The result must be compared with a hand-calculated output payload.

The ranking must include allocation frequency as well as total bytes. An operator allocating 4 MiB one hundred times creates more allocator traffic than an operator allocating 128 MiB once, even if the latter has the larger single event. Both patterns can matter: one threatens peak capacity and the other may increase overhead and fragmentation pressure. Key averages should therefore be read together with raw events.

Keep all three result tensors alive until the synchronized measurement boundary so the exercise has a known lifetime model. A second trial may delete each result immediately to show how the same operator traffic can produce a different global peak.

Part 2 - System and Memory Mental Model

Self CUDA memory excludes memory attributed to child operations. A composite operation may report little self memory while an internal `aten::empty` or matrix operation owns the allocation event. Negative self memory can represent storage released during an event. Allocation traffic is not peak residency, and repeated calls may be aggregated in `key_averages()`.

Part 3 - Quantitative Prediction Before Execution

Use three operators with outputs of deliberately different sizes: a large elementwise square, a smaller matrix multiplication, and a reduction. Calculate each output payload and predict the self-memory order. Predict which saved activation remains alive until backward and how that changes global peak.

Part 4 - Experimental Design and Instrumentation

Warm up once, reset peak counters, and profile exactly one pass with memory tracking enabled. Record raw events and key averages. Sort by `self_cuda_memory_usage`, not CUDA time. Preserve the profiler configuration in the report.

Part 5 - PyTorch/CUDA Implementation

```
import torch
from torch.profiler import ProfilerActivity, profile

def E03():
    device = torch.device("cuda:0")
    left = torch.randn((4096, 2048), device=device)
    right = torch.randn((2048, 1024), device=device)

    torch.cuda.synchronize(device)
    torch.cuda.reset_peak_memory_stats(device)

    with profile(
        activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
        profile_memory=True,
        record_shapes=True,
    ) as profiler:
        squared = left.square()
        product = left @ right
        reduced = squared.mean(dim=1)
        torch.cuda.synchronize(device)

    table = profiler.key_averages().table(
        sort_by="self_cuda_memory_usage",
        row_limit=30,
    )
    print(table)
    return squared, product, reduced, profiler
```

Part 6 - Measurement, Validation, and Failure Analysis

Compare expected output bytes with the appropriate profiler rows. If an allocation appears under `aten::empty`, follow the parent relationship instead of assigning semantic purpose from the low-level name alone. Check call counts before interpreting aggregated memory. Compare the largest self allocation with global peak and explain why equality is not required.

Part 7 - Engineering Interpretation and Philosophy

Ranking creates attention, but attention is not explanation. The first row is only the largest under a selected metric and aggregation rule. A responsible engineer follows the row back to shape, lifetime, and purpose before changing code. Metrics guide inquiry; they do not decide which computation deserves to exist.

19.3.5 Exercise 04

Exercise Title: Compare Self Memory Usage Against Total Operator Memory Usage and Explain the Effect of Child Operations

Part 1 - Exercise Contract and Why It Matters

Composite PyTorch operations delegate work. This exercise profiles a named region containing several child operations and compares self and inclusive memory rankings. You must show one region with near-zero self allocation but large total memory, then locate the children that created the storage. This prevents optimizing a wrapper merely because its inclusive value is high.

Nested accounting also affects team communication. A model designer may ask which block consumes memory, while a framework engineer asks which primitive materializes it. The inclusive region answers the first question and the self row helps answer the second. The report must present both levels side by side so responsibility is not shifted incorrectly between architecture and implementation.

Include one deliberately nested subregion inside the composite feature transform. Confirm that the outer total includes it and explain why summing both totals double-counts the same descendant allocation.

Part 2 - System and Memory Mental Model

Call trees define inclusive accounting. A parent region owns an interval and its descendants, while self metrics remove descendants. If two nested parents are both summed, the same child allocation is counted more than once. Total memory is useful for comparing semantic regions; self memory is useful for finding direct allocator responsibility. Both depend on correct nesting.

Part 3 - Quantitative Prediction Before Execution

Construct a region containing a linear operation, GELU, a clone, and a reduction. Calculate each output. Predict that the custom region has little self memory because `record_function()` allocates no large tensor, but its total includes all child allocations. Predict which child leads the self ranking.

Part 4 - Experimental Design and Instrumentation

Profile one warmed call with memory tracking and custom regions. Print tables sorted by self and total CUDA memory. Inspect the event tree rather than only key averages, because aggregation can combine calls from different parents. Keep outputs live until the region ends.

Part 5 - PyTorch/CUDA Implementation

```
import torch
from torch.profiler import ProfilerActivity
from torch.profiler import profile, record_function

def composite_region(layer, inputs):
    with record_function("composite_feature_transform"):
        hidden = layer(inputs)
        activated = torch.nn.functional.gelu(hidden)
        copied = activated.clone()
        summary = copied.mean(dim=1)
    return copied, summary

def E04():
    device = torch.device("cuda:0")
    layer = torch.nn.Linear(2048, 4096).to(device)
    inputs = torch.randn((512, 2048), device=device)

    with profile(
        activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
        profile_memory=True,
        record_shapes=True,
    ) as profiler:
        copied, summary = composite_region(layer, inputs)
        torch.cuda.synchronize(device)

    by_self = profiler.key_averages().table(
        sort_by="self_cuda_memory_usage",
        row_limit=25,
    )
    by_total = profiler.key_averages().table(
        sort_by="cuda_memory_usage",
        row_limit=25,
    )
    print(by_self)
    print(by_total)
    return copied, summary, profiler
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify the named region appears and compare its self and total values. Trace total memory to children and document any deallocation events. If the region does not show inclusive memory as expected, inspect profiler version behavior and raw event nesting. Never create a total by summing already-inclusive parent rows.

Part 7 - Engineering Interpretation and Philosophy

Responsibility depends on scale. A high-level feature block is responsible for the memory demanded by its design, while a low-level allocation operator is responsible for obtaining the bytes. Self and total metrics express these two views. Good optimization connects them: architecture explains why storage is needed; implementation explains where it materializes.

19.3.6 Exercise 05

Exercise Title: Record Tensor Shapes with record_shapes=True and Correlate Input Dimensions with Operator Memory Growth

Part 1 - Exercise Contract and Why It Matters

An operator name without shape is an incomplete memory explanation. This exercise records input shapes, repeats the same computation across several batch and sequence dimensions, and derives how the profiler's memory values scale. The report must connect each growing axis to a tensor formula and distinguish parameter memory, which remains fixed, from activation memory, which changes with input dimensions.

Shape recording can increase profiler overhead and retain additional metadata. It should therefore be activated for diagnosis rather than assumed free. The exercise must measure enough configurations to establish a scaling law, not merely observe that a larger input used more memory.

The exercise must keep parameter shapes fixed across the sweep. Otherwise a larger input configuration could accidentally create a different model and mix persistent growth with activation growth. Report model parameter bytes once, then express each profiled peak as persistent baseline plus input-dependent delta.

Shape metadata may list several inputs, including weights, bias, and optional arguments. Label their semantic roles instead of copying a raw list. The student should be able to point to the axis whose change caused the measured growth and prove the expected ratio from the operator's output law.

Part 2 - System and Memory Mental Model

For a projection from D_{in} to D_{out}, an output with batch B and sequence length T contains

$$BTD_{\text{out}}$$

elements. Self-attention score storage introduces a different dependence, often proportional to BT^2H. Shape metadata exposes the input contract of the recorded operator, but the output shape may still need to be derived from operator semantics.

Part 3 - Quantitative Prediction Before Execution

Predict output bytes for batches $16, 32, 64$ and sequence lengths $64, 128, 256$ using a fixed feature width. Calculate expected ratios when only one axis doubles. State which operations should scale linearly and which could scale quadratically. Preserve predictions before profiling.

Part 4 - Experimental Design and Instrumentation

Profile one configuration per fresh active interval. Use the same model and dtype, warm each shape, and record shapes and memory. Aggregate by input shape only when the profiler interface supports it; otherwise inspect raw events. Record global peaks so operator growth is connected to residency.

Part 5 - PyTorch/CUDA Implementation

```
import torch
from torch.profiler import ProfilerActivity, profile

def profile_shape(layer, batch, sequence, width, device):
    inputs = torch.randn(
        (batch, sequence, width),
        dtype=torch.float32,
        device=device,
    )

    with profile(
        activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
        profile_memory=True,
        record_shapes=True,
    ) as profiler:
        output = layer(inputs)
        loss = output.square().mean()
        loss.backward()
        torch.cuda.synchronize(device)

    peak = torch.cuda.max_memory_allocated(device)
    return profiler, peak

def E05():
    device = torch.device("cuda:0")
    layer = torch.nn.Linear(1024, 2048).to(device)
    configurations = [
        (16, 64),
        (32, 64),
        (32, 128),
        (32, 256),
        (64, 256),
    ]

    results = []
    for batch, sequence in configurations:
        layer.zero_grad(set_to_none=True)
        torch.cuda.reset_peak_memory_stats(device)
        profiler, peak = profile_shape(
            layer, batch, sequence, 1024, device
        )
        results.append((batch, sequence, peak, profiler))
    return results
```

Part 6 - Measurement, Validation, and Failure Analysis

Compare measured operator memory ratios with theoretical output ratios. If events with the same name collapse together, group by input shape or keep separate profiles. Check that old outputs and gradients are released before the next configuration. If a shape-doubling produces

more than linear growth, locate temporaries or broadcast expansion before concluding that the operator itself has quadratic complexity.

Part 7 - Engineering Interpretation and Philosophy

Shapes are where abstract algorithms become physical workloads. An axis named "sequence" carries a different scaling risk from an axis named "class," even when both are integers in a tuple. Profiling with shapes reconnects runtime cost to semantic structure and teaches that every dimension is both a modeling choice and a resource multiplier.

19.3.7 Exercise 06

Exercise Title: Add record_function() Regions to Separate Data Transfer, Forward, Loss, Backward, and Optimizer Memory Costs

Part 1 - Exercise Contract and Why It Matters

Raw operator lists are difficult to interpret when a training step contains hundreds of calls. This exercise adds semantic regions for data transfer, forward, loss, backward, optimizer update, and cleanup. The regions must nest correctly, preserve exceptions, and allow the reader to compare inclusive phase costs without mistaking the custom labels for direct allocation operators.

The regions must also reveal state that crosses their boundaries. Inputs created during transfer remain live through forward and backward; saved activations created during forward survive into backward; gradients created during backward survive until clearing or update. Inclusive phase memory is therefore not owned exclusively by the phase label. The label records where events occur, while the lifetime ledger records what remains afterward.

Run one experiment with optimizer state already initialized and another on the first update. This isolates steady-state optimizer traffic from lazy creation of moment tensors.

Part 2 - System and Memory Mental Model

`record_function()` attaches a named range to the profiler event tree. It does not move tensors or automatically isolate CUDA streams. Its total metrics include child work inside the range, while self memory is normally small. A region boundary defined only on the CPU may end before asynchronous device work completes, but correlation still associates launched kernels. Explicit synchronization is needed when the range must also define a measured wall-clock or state boundary.

Part 3 - Quantitative Prediction Before Execution

Create a phase ledger predicting which new objects become live: GPU inputs during transfer, saved activations during forward, a scalar loss, gradients during backward, and optimizer state during the first update. Predict which objects remain after each phase. This lifetime table is the reference for interpreting inclusive memory.

Part 4 - Experimental Design and Instrumentation

Use pinned CPU inputs and an explicit device transfer. Warm optimizer state before measurement if the goal is steady state; profile the first step separately if lazy initialization is the subject. Add nested regions but do not wrap the entire step in one label only. Validate that every phase appears once.

Part 5 - PyTorch/CUDA Implementation

```
import torch
from torch.profiler import ProfilerActivity
from torch.profiler import profile, record_function

def profiled_step(model, optimizer, cpu_inputs, cpu_targets, device):
    with record_function("01_data_transfer"):
        inputs = cpu_inputs.to(device, non_blocking=True)
        targets = cpu_targets.to(device, non_blocking=True)

    with record_function("02_forward"):
        outputs = model(inputs)

    with record_function("03_loss"):
        loss = torch.nn.functional.cross_entropy(outputs, targets)

    optimizer.zero_grad(set_to_none=True)
    with record_function("04_backward"):
        loss.backward()

    with record_function("05_optimizer"):
        optimizer.step()

    return loss.detach()

def E06(model, optimizer, cpu_inputs, cpu_targets):
    device = torch.device("cuda:0")
    activities = [ProfilerActivity.CPU, ProfilerActivity.CUDA]

    with profile(
        activities=activities,
        profile_memory=True,
        record_shapes=True,
    ) as profiler:
        final_loss = profiled_step(
            model,
            optimizer,
            cpu_inputs,
            cpu_targets,
            device,
        )
        torch.cuda.synchronize(device)

    return final_loss, profiler
```

Part 6 - Measurement, Validation, and Failure Analysis

Inspect region order and child calls. Compare the first and later optimizer steps to reveal lazy state. If transfer memory appears in forward, verify asynchronous timing and tensor lifetimes rather than moving labels until the table looks expected. Confirm the detached returned loss

does not retain the training graph. Use DMM00 stage counters to validate inclusive phase interpretation.

Part 7 - Engineering Interpretation and Philosophy

Semantic ranges translate machine activity back into the stages humans use to reason about learning. They are maps, not territories: a label does not create isolation and cannot override asynchronous execution. Their value lies in making a complex event tree navigable while preserving access to the operators that provide causal detail.

19.3.8 Exercise 07

Exercise Title: Profile Forward and Backward Separately and Identify Which Operators Create the Largest Temporary Allocations

Part 1 - Exercise Contract and Why It Matters

Forward and backward perform different computations and create different temporary storage. This exercise profiles them in separate active intervals so their operator tables cannot be merged accidentally. You will identify the largest positive self allocation in each phase, the largest deallocation, and the phase-local peak. The report must explain why backward can allocate gradients while simultaneously freeing saved forward tensors.

To deepen the comparison, repeat the experiment with gradient clearing set to None and with zero-filled gradient tensors. The backward computation is the same, but the entry state differs because pre-existing gradient storage may be reused. This demonstrates that an operator's observed allocation is conditional on allocator and tensor state, not a permanent property of its name.

The final table should distinguish bytes allocated during the phase, bytes freed during the phase, memory live at exit, and the phase peak. Net change alone can conceal large temporary traffic.

Part 2 - System and Memory Mental Model

Forward materializes outputs and saves selected intermediates for derivative rules. Backward traverses the graph in reverse, creates input and parameter gradients, and releases saved tensors after their final consumer. Temporary workspace may depend on the chosen CUDA library algorithm. A negative net memory event in backward does not mean no allocation occurred; allocations and releases can overlap inside the phase.

Part 3 - Quantitative Prediction Before Execution

Calculate forward activation payloads and parameter-gradient payloads. Predict which linear or activation backward operator needs a full-size gradient. Predict whether the phase-local current memory ends above or below its entry state and explain the lifetimes behind that direction.

Part 4 - Experimental Design and Instrumentation

Run forward under one profiler, retain output and loss, then run backward under a second profiler. Reset peak statistics at each boundary without freeing state. Synchronize boundaries. This changes some normal overlap but provides the clean separation required by the exercise; state that limitation.

Part 5 - PyTorch/CUDA Implementation

```
import torch
from torch.profiler import ProfilerActivity, profile

def E07():
    device = torch.device("cuda:0")
    model = torch.nn.Sequential(
        torch.nn.Linear(4096, 8192),
        torch.nn.GELU(),
        torch.nn.Linear(8192, 4096),
    ).to(device)
    inputs = torch.randn((256, 4096), device=device)
    targets = torch.randn((256, 4096), device=device)

    activities = [ProfilerActivity.CPU, ProfilerActivity.CUDA]
    model.zero_grad(set_to_none=True)

    torch.cuda.reset_peak_memory_stats(device)
    with profile(
        activities=activities,
        profile_memory=True,
        record_shapes=True,
    ) as forward_profiler:
        outputs = model(inputs)
        loss = torch.nn.functional.mse_loss(outputs, targets)
        torch.cuda.synchronize(device)
    forward_peak = torch.cuda.max_memory_allocated(device)

    torch.cuda.reset_peak_memory_stats(device)
    with profile(
        activities=activities,
        profile_memory=True,
        record_shapes=True,
    ) as backward_profiler:
        loss.backward()
        torch.cuda.synchronize(device)
    backward_peak = torch.cuda.max_memory_allocated(device)

    return {
        "forward_profiler": forward_profiler,
        "backward_profiler": backward_profiler,
        "forward_peak": forward_peak,
        "backward_peak": backward_peak,
    }
```

Part 6 - Measurement, Validation, and Failure Analysis

Sort both tables by self CUDA memory and inspect negative rows. Confirm all parameter gradients exist after backward. Compare phase peaks with a normal combined step to quantify

instrumentation effects. If backward events are missing, ensure the graph was created with gradients enabled and was not consumed during warm-up.

Part 7 - Engineering Interpretation and Philosophy

Backward is not simply forward played in reverse. It is a different memory program whose purpose is credit assignment. Separate profiling reveals how learning changes the material behavior of an otherwise identical model. The lesson is that mathematical duality does not imply identical resource lifetimes.

19.3.9 Exercise 08

Exercise Title: Use key_averages() to Build an Operator-Level Memory Report and Verify the Results Against DMM00 CUDA Memory Counters

Part 1 - Exercise Contract and Why It Matters

`key_averages()` compresses many raw events into operator summaries. This exercise converts those summaries into a structured report containing call count, self and total CPU memory, self and total CUDA memory, CPU time, CUDA time, and known shapes. The report must then be reconciled with global allocated, reserved, and peak counters from DMM00. The goal is a repeatable analysis rather than a table copied from terminal output.

The structured report must preserve enough identity to compare runs. Include the semantic phase, input configuration, dtype, model version, and profiler options beside every operator table. Two rows with the same key but different shapes are not necessarily comparable. Add an explicit aggregation key field so later analysis knows whether calls were grouped only by operator or also by shape and stack.

The reconciliation section must explain both success and residual. Agreement between predicted output bytes and one self-allocation supports attribution; the remaining difference between live allocation and global peak belongs to overlapping objects and temporary work that must be analyzed separately.

Part 2 - System and Memory Mental Model

Aggregation reduces detail. Calls sharing a key are combined, so a large total may arise from one huge call or many small calls. Shape grouping can preserve more distinction but increases report size. Global peak cannot be reconstructed by summing key averages because event order and overlapping lifetime were discarded. Reconciliation therefore checks compatible claims: largest event traffic, phase delta, and global high-water mark.

Part 3 - Quantitative Prediction Before Execution

Define report columns and units before profiling. Predict global persistent baseline, peak increase, and the top output allocation. State three invariants: global peak is at least current allocated, allocated does not exceed reserved, and each operator's call count is positive. Do not invent an invariant equating sum of rows with peak.

Part 4 - Experimental Design and Instrumentation

Capture global counters immediately before and after a warmed profiled step. Use raw integer bytes in JSON and format MiB only for presentation. Retain the top positive allocations and top negative deallocations. Save profiler options and PyTorch version with the report.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def event_to_row(event):
    row = {}
    row["key"] = event.key
    row["count"] = event.count
    row["self_cpu_memory_usage"] = event.self_cpu_memory_usage
    row["cpu_memory_usage"] = event.cpu_memory_usage
    row["self_cuda_memory_usage"] = event.self_cuda_memory_usage
    row["cuda_memory_usage"] = event.cuda_memory_usage
    row["self_cpu_time_total"] = event.self_cpu_time_total
    row["cuda_time_total"] = event.cuda_time_total
    return row

def build_operator_report(profiler):
    rows = []
    for event in profiler.key_averages():
        rows.append(event_to_row(event))

    rows.sort(
        key=lambda row: row["self_cuda_memory_usage"],
        reverse=True,
    )
    return rows

def verify_global_counters(device):
    allocated = torch.cuda.memory_allocated(device)
    reserved = torch.cuda.memory_reserved(device)
    peak = torch.cuda.max_memory_allocated(device)
    assert allocated <= reserved
    assert allocated <= peak
    return allocated, reserved, peak

def E08(profiler):
    device = torch.device("cuda:0")
    report = build_operator_report(profiler)
    counters = verify_global_counters(device)
    return report, counters
```

Part 6 - Measurement, Validation, and Failure Analysis

Compare raw-event count with aggregated row count and explain the information lost. Check version-specific fields before assuming every attribute exists. If call counts combine different shapes, use shape grouping or separate runs. Verify global counters at synchronized boundaries. Treat differences as a lifetime problem to investigate, not an error to erase through arbitrary normalization.

Part 7 - Engineering Interpretation and Philosophy

Aggregation is a bargain: clarity is purchased by discarding sequence and context. A useful report states what was compressed so readers know which questions it cannot answer. The engineering principle is to use the smallest summary that supports the decision while preserving a path back to raw evidence.

19.3.10 Exercise 09

Exercise Title: Deliberately Introduce an Unnecessary Tensor Copy and Locate the Exact Memory-Expensive Operator with the Profiler

Part 1 - Exercise Contract and Why It Matters

Unnecessary copies are ideal profiling targets because they consume memory without changing intended semantics. This exercise introduces a `clone()` and a forced `contiguous()` materialization, predicts their payload, locates their allocation operators, and proves which copy is avoidable. The experiment must preserve one necessary copy when layout truly requires it so that optimization does not become blind removal of all copying.

Introduce the copies in separate variants as well as one combined inefficient variant. If both are introduced simultaneously, the first may alter layout or lifetime in a way that changes the second's behavior. Single-change variants allow the report to measure each contribution and detect interaction when the combined cost differs from their simple sum.

The student must document the downstream operator's layout requirements. A layout-materialization call is unnecessary only if the consumer accepts the strided view correctly and without creating an equivalent hidden copy. Removing a visible copy while moving the same allocation into a library call is not a real optimization.

Part 2 - System and Memory Mental Model

`clone()` always creates independent storage. A transpose ordinarily creates a view with changed strides. `contiguous()` creates storage only when the input layout is not already contiguous. Downstream operators may accept strided input or materialize internally. The profiler identifies where bytes were allocated; stride inspection explains why.

Part 3 - Quantitative Prediction Before Execution

For a $(512, 4096)$ `float32` tensor, predict 8 MiB per full copy. Create a baseline with no explicit clone and an inefficient version with two copies. Predict the difference in allocation traffic and phase peak, noting that short lifetimes may prevent both copies from overlapping.

Part 4 - Experimental Design and Instrumentation

Profile baseline and inefficient functions under identical inputs. Warm both, record shapes and memory, and use custom regions. Verify output parity before comparing memory. Inspect strides and storage pointers. Keep profiler settings identical.

Part 5 - PyTorch/CUDA Implementation

```
import torch
from torch.profiler import ProfilerActivity
from torch.profiler import profile, record_function

def baseline(inputs):
```

```
    return inputs * 2.0

def inefficient(inputs):
    with record_function("unnecessary_copy_region"):
        copied = inputs.clone()
        transposed = copied.transpose(0, 1)
        materialized = transposed.contiguous()
        restored = materialized.transpose(0, 1)
        output = restored * 2.0
    return output

def profile_function(function, inputs):
    with profile(
        activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
        profile_memory=True,
        record_shapes=True,
    ) as profiler:
        output = function(inputs)
        torch.cuda.synchronize(inputs.device)
    return output, profiler

def E09():
    device = torch.device("cuda:0")
    inputs = torch.randn((512, 4096), device=device)
    base_output, base_profile = profile_function(baseline, inputs)
    bad_output, bad_profile = profile_function(inefficient, inputs)

    assert torch.allclose(base_output, bad_output)
    return base_profile, bad_profile
```

Part 6 - Measurement, Validation, and Failure Analysis

Locate clone, copy, empty, or contiguous-related events and match their shapes to the predicted 8 MiB payload. Compare global peaks as well as event traffic. If baseline and inefficient outputs differ, stop; memory improvement is irrelevant until semantics match. Remove one copy at a time and re-profile so causality is demonstrated.

Part 7 - Engineering Interpretation and Philosophy

Copies create independence. Sometimes that independence is required because a value must survive mutation or satisfy layout constraints. Sometimes it is only a defensive habit. Profiling helps distinguish the two, but the decision remains semantic: optimization asks whether separate storage has a purpose, not merely whether it has a cost.

19.3.11 Exercise 10

Exercise Title: Enable with_stack=True and Trace a High-Memory Operator Back to the Exact Python Source Location

Part 1 - Exercise Contract and Why It Matters

An operator name may identify a copy or allocation but not the application line that requested it. This exercise enables stack collection, locates a high-memory event, and traces it through framework frames to the exact user function and line. The report must separate the allocation operator from its caller and explain why stack capture is expensive enough to reserve for focused diagnosis.

Stack evidence should be treated as versioned diagnostic data. Refactoring can move a function to another line without changing behavior, while compilation can collapse several source operations. Store the function name, source file, operator key, shape, and measured bytes together; a bare line number will not remain meaningful.

The exercise must demonstrate escalation: begin with low-overhead key averages, enable shapes to disambiguate calls, and collect stacks only for the narrowed case. This staged process is the practical method for obtaining detail without profiling an entire training run at maximum overhead.

Part 2 - System and Memory Mental Model

Stacks connect dynamic events to the nested Python call path active when an operator was dispatched. Some CUDA kernels are launched after the originating Python frame has returned, so correlation depends on profiler metadata rather than a device stack. Framework, autograd, and module wrapper frames may appear between the event and user code. Compiled or fused execution can reduce source-level granularity.

Part 3 - Quantitative Prediction Before Execution

Use the unnecessary copy from E09 and predict its payload and source function. Write the expected call chain before profiling. Estimate the extra trace size qualitatively and define a narrow active interval containing only the suspect region.

Part 4 - Experimental Design and Instrumentation

First locate the hotspot without stacks. Then re-run only the suspect case with stack collection enabled. Export stacks grouped by the selected memory metric when supported and inspect raw events otherwise. Preserve source file and line metadata in the report without depending on one absolute workstation path.

Part 5 - PyTorch/CUDA Implementation

```
import torch
from torch.profiler import ProfilerActivity, profile

def suspicious_transform(inputs):
    copied = inputs.clone()
    return copied * 3.0

def E10(output_stack_path):
    device = torch.device("cuda:0")
    inputs = torch.randn((1024, 4096), device=device)

    with profile(
        activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
        profile_memory=True,
```

```
        record_shapes=True,
        with_stack=True,
    ) as profiler:
        output = suspicious_transform(inputs)
        torch.cuda.synchronize(device)

    profiler.export_stacks(
        output_stack_path,
        "self_cuda_memory_usage",
    )

    high_memory_events = []
    for event in profiler.events():
        if event.self_cuda_memory_usage > 0:
            high_memory_events.append(event)

    high_memory_events.sort(
        key=lambda event: event.self_cuda_memory_usage,
        reverse=True,
    )
    return output, high_memory_events
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify that the stack reaches `suspicious_transform` and the clone line. If no stacks appear, check profiler build support, execution mode, and whether the inspected event is the correlated CPU operator rather than a device kernel. Measure trace size and runtime relative to the same profile without stacks. Do not leave stack collection enabled in every normal step.

Part 7 - Engineering Interpretation and Philosophy

Attribution becomes actionable when it reaches the level where a human can change a decision. A kernel name describes execution; a source line reveals intent. Stack tracing bridges them, but it also reminds us that software causes are layered. The exact allocation site and the design choice that made the allocation necessary may still be different locations.

19.3.12 Exercise 11

Exercise Title: Remove the Hidden Allocation Found by Profiling and Prove the Memory Reduction Without Changing Numerical Outputs

Part 1 - Exercise Contract and Why It Matters

Finding a hotspot is diagnosis; proving a safe repair is engineering. This exercise removes the unnecessary copy traced in E10, compares operator events and global peaks, and validates outputs and gradients. The optimized version must not merely produce similar loss once; it must preserve shape, dtype, device, values within tolerance, and parameter gradients under identical inputs.

The proof must include a lifetime comparison, not only disappearance of the clone row. Record global current memory immediately after forward and after cleanup. A compiler or downstream operator could replace the removed copy with another allocation; only global

counters and operator re-profiling together show whether residency improved.

Add a repeated-run test to detect alias-dependent mutation that may not appear on the first call. Feed the same input object twice, verify it remains unchanged, and compare both outputs. This protects against optimizations that silently introduce stateful behavior.

Part 2 - System and Memory Mental Model

Removing a copy can change aliasing. If downstream code mutates the tensor in-place, the original input may now be modified. A numerically equal forward output in one test does not prove equivalent storage semantics. The repair must inspect whether independent storage was required for mutation, lifetime, or layout. Only a semantically unnecessary copy may be removed safely.

Part 3 - Quantitative Prediction Before Execution

Predict the eliminated payload from shape and dtype. Estimate the best possible peak reduction by checking whether the copy overlaps with other large tensors. Predict zero change in output and gradient formulas. Define absolute and relative tolerances before execution.

Part 4 - Experimental Design and Instrumentation

Clone model parameters so baseline and optimized runs start identically. Use the same input tensor and seed. Profile both with identical options, reset global peaks, and run multiple repetitions. Test input immutability and storage aliasing explicitly. Compare median runtime only after correctness passes.

Part 5 - PyTorch/CUDA Implementation

```
import copy
import torch

class Baseline(torch.nn.Module):
    def __init__(self, layer):
        super().__init__()
        self.layer = layer

    def forward(self, inputs):
        copied = inputs.clone()
        return self.layer(copied)

class Optimized(torch.nn.Module):
    def __init__(self, layer):
        super().__init__()
        self.layer = layer

    def forward(self, inputs):
        return self.layer(inputs)

def collect_gradients(model):
    gradients = []
    for parameter in model.parameters():
```

```
        gradients.append(parameter.grad.detach().clone())
    return gradients

def E11():
    device = torch.device("cuda:0")
    base_layer = torch.nn.Linear(4096, 2048).to(device)
    model_a = Baseline(copy.deepcopy(base_layer)).to(device)
    model_b = Optimized(copy.deepcopy(base_layer)).to(device)
    inputs = torch.randn((512, 4096), device=device)

    output_a = model_a(inputs)
    loss_a = output_a.square().mean()
    loss_a.backward()
    gradients_a = collect_gradients(model_a)

    output_b = model_b(inputs)
    loss_b = output_b.square().mean()
    loss_b.backward()
    gradients_b = collect_gradients(model_b)

    assert torch.allclose(output_a, output_b, atol=1e-6, rtol=1e-5)
    for index in range(len(gradients_a)):
        assert torch.allclose(
            gradients_a[index],
            gradients_b[index],
            atol=1e-6,
            rtol=1e-5,
        )
    return model_a, model_b
```

Part 6 - Measurement, Validation, and Failure Analysis

Re-profile both versions and confirm the clone allocation disappears. Measure global peak reduction and explain if it is smaller than the copy payload due to non-overlapping lifetimes or allocator reuse. Add an in-place downstream mutation test; if behavior changes, the copy was protecting semantics and must be replaced with a more careful design rather than deleted.

Part 7 - Engineering Interpretation and Philosophy

Optimization is a proof obligation. Lower memory is only a benefit if the program continues to mean the same thing. Copies often encode hidden contracts about independence, so removing them requires understanding rather than mechanical editing. Profiler evidence identifies cost; tests defend meaning.

19.3.13 Exercise 12

Exercise Title: Compare Profiler Results Across float64, float32, bfloat16, and float16 and Explain Which Operators Benefit Most

Part 1 - Exercise Contract and Why It Matters

Reduced precision changes tensor payload, arithmetic throughput, kernel choice, temporary workspaces, and numerical behavior. This exercise profiles the same model under four dtypes and determines which operators show near-proportional memory reduction, which retain higher-precision state, and which are unsupported or unstable. The report must never interpret a failed or silently promoted operation as a valid low-precision saving.

The comparison must distinguish explicit uniform dtype from automatic mixed precision. Uniform conversion is useful for understanding payload laws, while autocast represents a realistic policy in which selected operations remain or accumulate in higher precision. Profile both modes if hardware support allows, and do not combine their results into one precision category.

Record whether Tensor Core-friendly dimensions are used. Precision can change kernel selection and performance only when the hardware and shapes support the relevant paths. Memory ratios are more portable than speed ratios, but even memory can deviate through workspaces and promotion.

Part 2 - System and Memory Mental Model

Dense output payload scales with element size, but complete operator memory can include accumulation buffers and library workspaces in another dtype. `float16` and `bfloat16` both use two bytes, yet their exponent and mantissa trade-offs differ. Autocast selects dtypes per operation rather than converting the entire program uniformly. The actual output dtype must be observed.

Part 3 - Quantitative Prediction Before Execution

Calculate payload ratios $8 : 4 : 2 : 2$ bytes for `float64`, `float32`, `bfloat16`, and `float16`. Predict which full-sized activations should follow those ratios. Define numerical test inputs containing ordinary, very small, and large values. Predict where overflow, underflow, or rounding may appear.

Part 4 - Experimental Design and Instrumentation

Use separate model copies and fresh profiles for each dtype. Warm every supported case, record actual dtypes, and use identical shapes. Compare memory rows, global peaks, outputs converted to `float64`, and profiler overhead. Record unsupported cases honestly rather than changing the operator set.

Part 5 - PyTorch/CUDA Implementation

```
import copy
import torch
from torch.profiler import ProfilerActivity, profile

def run_dtype(reference_model, reference_inputs, dtype, device):
    model = copy.deepcopy(reference_model).to(
        device=device,
        dtype=dtype,
    )
    inputs = reference_inputs.to(device=device, dtype=dtype)
    model.zero_grad(set_to_none=True)

    torch.cuda.reset_peak_memory_stats(device)
```

```
    with profile(
        activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
        profile_memory=True,
        record_shapes=True,
    ) as profiler:
        output = model(inputs)
        loss = output.float().square().mean()
        loss.backward()
        torch.cuda.synchronize(device)

    return {
        "dtype": str(dtype),
        "output": output.detach().to(torch.float64).cpu(),
        "peak": torch.cuda.max_memory_allocated(device),
        "profiler": profiler,
    }

def E12(reference_model, reference_inputs):
    device = torch.device("cuda:0")
    dtypes = [
        torch.float64,
        torch.float32,
        torch.bfloat16,
        torch.float16,
    ]
    results = []
    for dtype in dtypes:
        results.append(
            run_dtype(reference_model, reference_inputs, dtype, device)
        )
    return results
```

Part 6 - Measurement, Validation, and Failure Analysis

Compare operator memory ratios and identify deviations from element-size ratios. Inspect actual input and output dtypes. Compare values against the `float64` reference using dtype-appropriate tolerance and finite checks. If a low-precision case creates NaNs, preserve the failure evidence and locate the first non-finite operator instead of reporting only its lower peak.

Part 7 - Engineering Interpretation and Philosophy

Precision is not merely compression. It changes what distinctions the machine can represent and which algorithms hardware chooses. A memory benefit belongs to a valid numerical computation, not to bytes saved after meaning has collapsed. The right dtype is an empirical agreement between capacity, throughput, and tolerated error.

19.3.14 Exercise 13

Exercise Title: Profile Increasing Batch Sizes and Determine Which Operator Becomes the Dominant VRAM Consumer Near the OOM Boundary

Part 1 - Exercise Contract and Why It Matters

The dominant memory operator can change as batch size grows. Persistent parameter and optimizer state remain fixed while activations and gradients scale. This exercise profiles a bounded batch sweep, tracks the top allocation hotspot at each point, and identifies where dominance changes. It must estimate the boundary safely rather than forcing the main process into uncontrolled OOM.

Near the boundary, the profiler itself consumes resources and can cause a batch to fail earlier than an unprofiled workload. Measure the unprofiled safe range first, then keep the profiled sweep inside a smaller budget. Report both limits and never describe the profiler-induced boundary as the model's production capacity.

Also track the second and third largest operators. Dominance may alternate due to allocator reuse or implementation switches, and a single top row can hide a cluster of comparable consumers that require a broader architectural change.

Part 2 - System and Memory Mental Model

For operator o, model memory as

$$M_o(B) = c_o + \alpha_o B + \epsilon_o(B).$$

An operator with large constant state may dominate small batches, while a large activation with greater slope dominates later. Algorithm switches and workspaces can create discontinuities. The top self-allocation event is not necessarily the tensor dominating lifetime at peak.

Part 3 - Quantitative Prediction Before Execution

Calculate per-sample output bytes for every major layer. Predict the slopes and the crossover batch between two candidate hotspots. Set a safety budget below driver-free capacity and predict the next batch before attempting it. Use incremental growth near the boundary.

Part 4 - Experimental Design and Instrumentation

Warm each batch, reset peaks, and profile one training step. Release outputs and gradients between cases. Store top operator, self bytes, total bytes, global peak, and runtime. Stop when the predicted next peak exceeds the configured safety budget. Run dangerous confirmation only in a child process if a dedicated GPU is available.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch
from torch.profiler import ProfilerActivity, profile

def largest_allocation(profiler):
    largest_key = None
    largest_bytes = 0
    for event in profiler.key_averages():
        value = event.self_cuda_memory_usage
        if value > largest_bytes:
            largest_bytes = value
            largest_key = event.key
```

```
    return largest_key, largest_bytes

def profile_batch(model, batch_size, device):
    inputs = torch.randn((batch_size, 4096), device=device)
    targets = torch.randint(0, 1000, (batch_size,), device=device)
    model.zero_grad(set_to_none=True)
    torch.cuda.reset_peak_memory_stats(device)

    with profile(
        activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
        profile_memory=True,
        record_shapes=True,
    ) as profiler:
        outputs = model(inputs)
        loss = torch.nn.functional.cross_entropy(outputs, targets)
        loss.backward()
        torch.cuda.synchronize(device)

    top_key, top_bytes = largest_allocation(profiler)
    peak = torch.cuda.max_memory_allocated(device)
    del inputs
    del targets
    del outputs
    del loss
    model.zero_grad(set_to_none=True)
    gc.collect()
    return top_key, top_bytes, peak
```

Part 6 - Measurement, Validation, and Failure Analysis

Plot each operator's bytes and global peak against batch. Identify crossover points and discontinuities. If one generic allocation operator appears for many semantic calls, use shapes or stacks to separate them. Ensure the sweep did not retain profiler objects with CUDA tensors; raw profiler metadata can also consume host memory and should be released after extraction.

Part 7 - Engineering Interpretation and Philosophy

Bottlenecks are conditional. The operation that matters near capacity may be irrelevant at small scale, so optimization priorities must be tied to the target workload. Scaling experiments reveal not only how much memory grows but which part of the computation acquires control over the system's limit.

19.3.15 Exercise 14

Exercise Title: Use a Profiler Schedule with Warmup and Active Steps to Measure a Multi-Iteration Training Loop Without Profiling Every Step

Part 1 - Exercise Contract and Why It Matters

Continuous detailed profiling can overwhelm long training runs. This exercise uses a schedule with wait, warm-up, active, and repeat phases so only selected iterations generate traces.

You must call `profiler.step()` exactly once per training iteration, label the schedule state, and prove that inactive steps execute normally without being included in active trace windows.

The schedule must be aligned with meaningful training boundaries. If one profiler step is called per micro-batch while the analysis expects one per optimizer update, gradient accumulation changes what an "iteration" means. Declare whether the schedule advances on micro-steps, optimizer steps, or data batches and preserve that unit in trace names.

Add an anomaly-triggered extension concept: lightweight counters can detect a memory spike, after which the next scheduled window enables more detailed profiling. The implementation may remain conceptual here, but the report must describe how delayed activation affects the chance of capturing a one-time failure.

Part 2 - System and Memory Mental Model

A profiler schedule is a state machine. Wait steps collect nothing, warm-up steps prepare instrumentation while discarding results, active steps record events, and repeat starts another cycle. Warm-up here refers to profiler collection, not necessarily model or CUDA warm-up; the workload should already be stabilized before the schedule begins. The trace handler runs when an active window completes.

Part 3 - Quantitative Prediction Before Execution

For a schedule wait = 2, warmup = 1, active = 3, repeat = 2, calculate the total scheduled steps and list which iteration indices are active. Predict the number of trace callbacks and estimate metadata growth relative to profiling every iteration.

Part 4 - Experimental Design and Instrumentation

Run ordinary model warm-up first. Use a fixed number of iterations matching the schedule. Add iteration labels with `record_function()`, call `step()` after the complete iteration, and use a trace callback that stores only required summaries. Verify optimizer semantics are identical with and without profiling.

Part 5 - PyTorch/CUDA Implementation

```
import torch
from torch.profiler import ProfilerActivity
from torch.profiler import profile, record_function, schedule

def E14(model, optimizer, batches, trace_handler):
    profiler_schedule = schedule(
        wait=2,
        warmup=1,
        active=3,
        repeat=2,
    )

    with profile(
        activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
        schedule=profiler_schedule,
        on_trace_ready=trace_handler,
        profile_memory=True,
```

```
        record_shapes=True,
    ) as profiler:
        for iteration in range(len(batches)):
            inputs, targets = batches[iteration]
            label = f"iteration_{iteration:03d}"

            with record_function(label):
                optimizer.zero_grad(set_to_none=True)
                outputs = model(inputs)
                loss = torch.nn.functional.cross_entropy(
                    outputs,
                    targets,
                )
                loss.backward()
                optimizer.step()

            profiler.step()
```

Part 6 - Measurement, Validation, and Failure Analysis

Log callback count and active iteration labels. If traces are empty, check the number and placement of `step()` calls. If one iteration produces two steps, the schedule no longer maps to training semantics. Compare final model parameters with a controlled unprofiled run and measure host-memory growth from retained trace objects.

Part 7 - Engineering Interpretation and Philosophy

Observability is a sampling problem. Recording everything can make the observer too expensive, while recording too little can miss rare failures. Scheduling turns profiling into a deliberate allocation of attention. The design question becomes which intervals are representative and which signals should trigger a deeper capture.

19.3.16 Exercise 15

Exercise Title: Quantify Profiler Overhead from Memory Tracking, Shape Recording, and Stack Collection Under Identical CUDA Workloads

Part 1 - Exercise Contract and Why It Matters

Profiler evidence has a cost. Memory event tracking, shape capture, and stack collection add host work, metadata, synchronization pressure, and storage. This exercise measures that cost through a factorial comparison: unprofiled baseline, activity-only profiling, memory tracking, memory plus shapes, and memory plus shapes plus stacks. The workload, warm-up, iteration count, and correctness must remain identical.

The deliverable must report wall-clock duration, CUDA-event duration where appropriate, training throughput, process host-memory growth, GPU peak, and trace size. One percentage called "profiler overhead" is insufficient because different resources can be affected differently.

Profiler overhead can also change the workload indirectly. Slower host execution may alter GPU clock behavior, queue depth, and overlap. Large trace writes can contend with data loading. The experiment should place trace serialization outside the measured training interval

when measuring capture overhead, then measure serialization separately as an operational cost.

Report both multiplicative slowdown and absolute time added per step. A 100-percent overhead on a tiny microbenchmark may be less operationally important than a 10-percent overhead on a multi-hour step, and the two metrics support different decisions.

Part 2 - System and Memory Mental Model

Instrumentation changes execution through event creation and correlation. Memory tracking records allocation events. Shape recording retains metadata and may hold references or extend some object lifetimes according to profiler behavior. Stack collection unwinds or records call paths, primarily increasing CPU overhead and trace volume. GPU kernels may remain nearly unchanged while end-to-end throughput falls because the host cannot feed them as efficiently.

Part 3 - Quantitative Prediction Before Execution

Predict the overhead ordering and specify that it is a hypothesis, not a guarantee:

$$T_{\text{baseline}} \leq T_{\text{activities}} \leq T_{\text{memory}} \leq T_{\text{shapes}} \leq T_{\text{stacks}}.$$

Estimate trace events as iterations times operators per iteration. Predict which options affect host memory more than GPU peak. Define the percentage

$$\text{overhead} = \frac{T_{\text{instrumented}} - T_{\text{baseline}}}{T_{\text{baseline}}} \times 100.$$

Part 4 - Experimental Design and Instrumentation

Use at least five independent repetitions per configuration and randomize configuration order after a common warm-up. Run enough iterations to dominate startup noise but keep traces bounded. Synchronize timing boundaries, capture median and variability, and discard generated profiler objects between trials. Monitor host memory with an appropriate process tool if available.

Part 5 - PyTorch/CUDA Implementation

```
import statistics
import time
import torch
from torch.profiler import ProfilerActivity, profile

def training_step(model, optimizer, inputs, targets):
    optimizer.zero_grad(set_to_none=True)
    outputs = model(inputs)
    loss = torch.nn.functional.cross_entropy(outputs, targets)
    loss.backward()
    optimizer.step()
    return loss.detach()

def timed_profile(
    model,
```

```
    optimizer,
    inputs,
    targets,
    iterations,
    memory,
    shapes,
    stacks,
):
    device = inputs.device
    torch.cuda.synchronize(device)
    start = time.perf_counter()

    with profile(
        activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
        profile_memory=memory,
        record_shapes=shapes,
        with_stack=stacks,
    ) as profiler:
        for iteration in range(iterations):
            training_step(model, optimizer, inputs, targets)

    torch.cuda.synchronize(device)
    duration = time.perf_counter() - start
    return duration, profiler

def E15(configurations, repetitions):
    results = {}
    for name, arguments in configurations:
        durations = []
        for repetition in range(repetitions):
            duration, profiler = timed_profile(*arguments)
            durations.append(duration)
            del profiler
        results[name] = {
            "median_seconds": statistics.median(durations),
            "all_seconds": durations,
        }
    return results
```

Part 6 - Measurement, Validation, and Failure Analysis

Compare losses and final parameters across controlled runs. Report confidence through repeated measurements rather than a single ratio. If the most detailed configuration appears faster, inspect trial order, GPU clock state, warm-up, and data reuse. Measure whether profiler metadata remains referenced after the trial. State that overhead is workload- and version-dependent and cannot be copied to another machine as a universal constant.

Part 7 - Engineering Interpretation and Philosophy

Observation consumes the same finite resources it seeks to explain. More detail is not automatically more truth if the instrument perturbs scheduling or makes the experiment unrepresentative. The right profiler configuration is the least expensive one capable of resolving the current uncertainty. Deep visibility should be activated by a question, not by habit.

19.3.17 Exercise 16

Exercise Title: Export and Analyze a CUDA Execution Trace to Correlate Memory-Heavy PyTorch Operations with GPU Kernel Activity

Part 1 - Exercise Contract and Why It Matters

Operator tables discard temporal order. This exercise exports a Chrome trace from a narrow active window and analyzes how high-memory PyTorch operators relate to CUDA runtime calls, kernels, transfers, and synchronization on the timeline. The goal is not to produce an attractive trace image; it is to answer a precise question about when an allocation becomes live relative to device work and neighboring phases.

Trace analysis must distinguish allocation lifetime from kernel lifetime. A buffer can be allocated before a kernel begins and remain live after it ends because a later operation consumes it. Selecting a kernel rectangle and assigning the entire tensor lifetime to its duration is incorrect. Correlate allocation events, producing operator, consumer events, and deallocation where available.

Include one search for hidden synchronization, such as a scalar extraction or explicit device wait. Such an event may not allocate much memory, yet it can prevent overlap and extend the time during which several buffers coexist.

Part 2 - System and Memory Mental Model

A trace contains events on multiple tracks. CPU operators enqueue CUDA work, runtime calls communicate with the driver, and kernels execute on streams. Correlation identifiers connect layers. A high-memory operator may launch a kernel whose name reflects a library implementation rather than the PyTorch operation. Memory allocation may occur on the host timeline before the kernel uses the buffer. Overlap means events can coexist across streams.

Part 3 - Quantitative Prediction Before Execution

Select one known hotspot and predict its approximate start order, allocation payload, launched kernel family, and lifetime relative to adjacent operations. Predict whether data transfer overlaps computation. Define trace questions before export so analysis does not become aimless browsing.

Part 4 - Experimental Design and Instrumentation

Warm the workload, capture only one or two active steps, add semantic ranges, enable memory and shape recording, and export to a uniquely named JSON file. Record file size and profiler options. Open the trace in a compatible viewer and preserve screenshots only as supporting artifacts; conclusions must refer to event names, timestamps, durations, streams, and correlations.

Part 5 - PyTorch/CUDA Implementation

```
from pathlib import Path
import torch
from torch.profiler import ProfilerActivity
from torch.profiler import profile, record_function

def E16(model, optimizer, inputs, targets, trace_path):
    output_path = Path(trace_path)
    output_path.parent.mkdir(parents=True, exist_ok=True)

    with profile(
        activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
        profile_memory=True,
        record_shapes=True,
        with_stack=False,
    ) as profiler:
        with record_function("measured_training_step"):
            optimizer.zero_grad(set_to_none=True)
            with record_function("forward"):
                outputs = model(inputs)
            with record_function("loss"):
                loss = torch.nn.functional.cross_entropy(
                    outputs,
                    targets,
                )
            with record_function("backward"):
                loss.backward()
            with record_function("optimizer"):
                optimizer.step()
        torch.cuda.synchronize(inputs.device)

    profiler.export_chrome_trace(str(output_path))
    return {
        "trace_path": str(output_path),
        "trace_bytes": output_path.stat().st_size,
    }
```

Part 6 - Measurement, Validation, and Failure Analysis

Locate the semantic region, high-memory operator, associated runtime calls, and device kernels. Verify timestamp ordering and stream identifiers. If a kernel cannot be linked confidently, label the relationship as an inference. Check for synchronizations that serialize work. Ensure trace export occurs after profiling and that long schedules do not create unmanageable files.

Part 7 - Engineering Interpretation and Philosophy

Tables answer "how much" by aggregation; traces answer "when" and "alongside what." Memory pressure is fundamentally temporal, so both views are required. A trace restores sequence to the evidence but increases complexity. Its value comes from using time to test a causal story, not from inspecting every colored rectangle.

19.3.18 Exercise 17

Exercise Title: Build a Memory Regression Test That Fails When Any Critical Operator Exceeds a Defined Allocation Budget

Part 1 - Exercise Contract and Why It Matters

Memory regressions can enter during refactoring even when outputs remain correct. This exercise converts profiler evidence into an automated test that fails when named critical operators or semantic regions exceed versioned byte budgets. The test must use tolerances, fixed shapes, fixed dtype, a controlled environment, and global peak validation. It must produce an actionable failure message showing expected and observed values.

Budgets should be layered. A hard ceiling protects device capacity, a warning threshold detects drift before failure, and a normalized scaling check protects the expected relationship across batches. A change can pass the fixed small- batch ceiling while introducing a worse slope that becomes dangerous at production scale.

The test artifact must show the top observed operators even when it passes. This makes gradual changes reviewable and prevents the test from becoming a binary gate with no diagnostic value. Store only numeric summaries so the test does not retain profiler state or model tensors.

Part 2 - System and Memory Mental Model

Operator names and fusion behavior can change across PyTorch versions, so a budget tied only to one low-level name is brittle. Use semantic regions where possible and allow a small set of version-specific aliases. Absolute byte budgets detect fixed regressions; normalized bytes per sample detect scaling regressions. Global peak protects against allocations that attribution misses.

Part 3 - Quantitative Prediction Before Execution

Derive budgets from a verified baseline plus a justified tolerance:

$$L_o = M_o^{\text{baseline}}(1 + \tau) + C.$$

Choose percentage tolerance τ and absolute allowance C before introducing the regression. Define which operators are critical and why. Predict how an inserted clone should violate the budget.

Part 4 - Experimental Design and Instrumentation

Run the test in a fresh process or controlled CI worker. Warm up, profile one fixed step, extract summaries, and compare with a checked-in budget configuration. Record PyTorch and CUDA versions. Add one negative test that intentionally creates a copy and proves failure, then restore the optimized path and prove success.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def extract_self_cuda_memory(profiler):
    observed = {}
```

```
    for event in profiler.key_averages():
        observed[event.key] = event.self_cuda_memory_usage
    return observed

def check_operator_budgets(observed, budgets):
    failures = []
    for operator_name in budgets:
        limit = budgets[operator_name]
        actual = observed.get(operator_name, 0)
        if actual > limit:
            failures.append({
                "operator": operator_name,
                "limit_bytes": limit,
                "actual_bytes": actual,
                "excess_bytes": actual - limit,
            })
    return failures

def E17(profiler, peak_limit_bytes, device):
    observed = extract_self_cuda_memory(profiler)
    budgets = {
        "aten::clone": 0,
        "aten::contiguous": 0,
    }
    failures = check_operator_budgets(observed, budgets)

    peak = torch.cuda.max_memory_allocated(device)
    if peak > peak_limit_bytes:
        failures.append({
            "operator": "GLOBAL_PEAK",
            "limit_bytes": peak_limit_bytes,
            "actual_bytes": peak,
            "excess_bytes": peak - peak_limit_bytes,
        })

    if len(failures) != 0:
        raise AssertionError(f"Memory regression: {failures}")
    return observed
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify the test fails for the deliberate clone and passes after removal. Inspect whether zero budget is valid; some framework paths may legitimately clone. If operator names change, fail with an "unresolved metric" message rather than silently passing because the key disappeared. Separate environmental drift from code regressions by retaining version metadata.

Part 7 - Engineering Interpretation and Philosophy

A regression test turns past understanding into a future constraint. It says that memory behavior is part of correctness, not an optional performance detail. Yet budgets must remain

explainable and revisable; otherwise yesterday's implementation becomes a prison. The test protects an engineering intention, not a sacred number.

19.3.19 Exercise 18

Exercise Title: Build a Reusable PyTorch Memory Profiler That Automatically Reports the Top Allocation Hotspots in a Training Step

Part 1 - Exercise Contract and Why It Matters

This exercise packages the chapter's methods into a reusable diagnostic tool. The tool accepts a training-step callable, profiler configuration, device, row limit, and output directory. It produces structured JSON containing global counters, top positive and negative operator events, semantic regions, shapes, times, configuration, and environment metadata. It must not retain output tensors or profiler objects after extraction.

The reusable tool must protect users from invalid comparisons. It should reject a report if the active interval contains no CUDA events, if the selected device differs from workload tensors, or if outputs are not synchronized before final counters. Warnings should distinguish missing evidence from an observed zero; an absent operator is not automatically a zero-byte operator.

Add configurable grouping by operator, input shape, or stack. Default to operator grouping for compactness, then escalate when several semantic calls share a key. The report should record the grouping method so later regression tools interpret rows correctly.

Part 2 - System and Memory Mental Model

Reusable tooling needs separation of concerns: workload execution, evidence capture, aggregation, validation, formatting, and persistence. The default configuration should be light enough for diagnosis, while stacks and trace export are optional escalation modes. Operator traffic and global peak remain separate fields. Version metadata preserves interpretability when profiler schemas change.

Part 3 - Quantitative Prediction Before Execution

Define the report schema and size budget before implementation. Require raw bytes, not only formatted MiB. Set minimum significant allocation to filter noise, but preserve a count of omitted events. Predict tool overhead and expose configuration so users can reproduce it.

Part 4 - Experimental Design and Instrumentation

Test with three workloads: known linear allocation, deliberate clone, and forward/backward training. Compare automated results with manual tables from earlier exercises. Test CUDA-unavailable and invalid-device errors. Run the tool twice to verify that state from the first call does not contaminate the second.

Part 5 - PyTorch/CUDA Implementation

```
import json
from pathlib import Path
import torch
from torch.profiler import ProfilerActivity, profile
```

```
class MemoryProfiler:
    def __init__(self, device, row_limit=20):
        self.device = torch.device(device)
        self.row_limit = row_limit

    def run(self, step_function):
        torch.cuda.synchronize(self.device)
        torch.cuda.reset_peak_memory_stats(self.device)
        before = torch.cuda.memory_allocated(self.device)

        with profile(
            activities=[
                ProfilerActivity.CPU,
                ProfilerActivity.CUDA,
            ],
            profile_memory=True,
            record_shapes=True,
        ) as profiler:
            step_function()
            torch.cuda.synchronize(self.device)

        events = []
        for event in profiler.key_averages():
            row = {}
            row["key"] = event.key
            row["count"] = event.count
            row["self_cuda_memory_usage"] = (
                event.self_cuda_memory_usage
            )
            row["cuda_memory_usage"] = event.cuda_memory_usage
            row["cuda_time_total"] = event.cuda_time_total
            events.append(row)

        events.sort(
            key=lambda row: row["self_cuda_memory_usage"],
            reverse=True,
        )
        report = {}
        report["allocated_before"] = before
        report["allocated_after"] = torch.cuda.memory_allocated(
            self.device
        )
        report["peak_allocated"] = torch.cuda.max_memory_allocated(
            self.device
        )
        report["top_events"] = events[:self.row_limit]
        del profiler
        return report
```

```
def E18(step_function, output_path):
    diagnostic = MemoryProfiler("cuda:0", row_limit=25)
    report = diagnostic.run(step_function)
    path = Path(output_path)
    with path.open("w", encoding="utf-8") as file:
        json.dump(report, file, indent=2)
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate JSON serialization and invariant fields. Confirm outputs from the training step are not stored in the report. Compare the tool's top rows with the profiler's printed table and global counters. Add schema version, environment metadata, negative deallocation ranking, and configuration to the student implementation. Measure overhead and prevent automatic full traces in the default path.

Part 7 - Engineering Interpretation and Philosophy

Tools preserve habits. A well-designed profiler wrapper makes explicit devices, raw units, global validation, and reproducibility the easy path. A poor wrapper can industrialize misunderstanding by hiding configuration and collapsing incompatible metrics. Reusability should amplify disciplined reasoning, not replace it.

19.3.20 Exercise 19

Exercise Title: Optimize a Memory-Inefficient PyTorch Model Using Only Profiler Evidence and Produce a Reproducible Before-and-After Benchmark

Part 1 - Exercise Contract and Why It Matters

The final exercise is a constrained optimization study. You will begin with a model containing several plausible inefficiencies: unnecessary clones, premature contiguous materialization, retained GPU outputs, and an oversized intermediate. You may change code only when profiler or global-counter evidence supports a specific hypothesis. Every change must be applied separately, profiled again, and accepted only after output and gradient validation.

The final report must include an untouched baseline, a hypothesis ledger, operator tables, traces or stacks used for attribution, global peak, reserved memory, throughput, numerical parity, and an ablation showing the contribution of each accepted change. A final optimized number without the path of evidence does not satisfy the exercise.

The deliberately inefficient model should be realistic enough that not every large allocation is removable. One large activation may be essential to model capacity, while a smaller clone is pure waste. The exercise must demonstrate the judgment to leave an expensive but necessary operation unchanged and explain why. Optimization quality is not measured by the number of deleted rows.

Close the report with remaining bottlenecks and the next experiment required to address them. The final candidate is not "finished"; it is the best version supported by the current evidence and acceptance criteria.

Part 2 - System and Memory Mental Model

Optimization is an iterative causal loop:

$$\text{observe} \rightarrow \text{hypothesize} \rightarrow \text{change one cause} \rightarrow \text{validate} \rightarrow \text{measure again.}$$

Operator traffic identifies materialization; global peak tests residency; shapes explain scale; stacks locate source; traces reveal lifetime overlap. An optimization can lower allocation traffic without lowering peak, or lower peak while increasing runtime. The complete trade-off must remain visible.

Part 3 - Quantitative Prediction Before Execution

Before each modification, predict eliminated bytes, expected peak reduction, runtime direction, and numerical effect. Define acceptance criteria such as at least 10 percent peak reduction, no non-finite outputs, output and gradient parity within tolerance, and no unacceptable throughput regression. State hardware, batch, dtype, and seed.

Part 4 - Experimental Design and Instrumentation

Create independent baseline and candidate models with identical state dictionaries. Warm both, run multiple measured repetitions, randomize order, and use the same profiler options. Preserve raw JSON evidence. Apply one change at a time, then run the combined candidate. Include a long-loop test to detect retention that a single step misses.

Part 5 - PyTorch/CUDA Implementation

```
import copy
import statistics
import time
import torch

def compare_outputs(reference, candidate):
    return torch.allclose(
        reference,
        candidate,
        atol=1e-5,
        rtol=1e-5,
    )

def benchmark_step(step_function, device, repetitions):
    durations = []
    peaks = []

    for repetition in range(repetitions):
        torch.cuda.synchronize(device)
        torch.cuda.reset_peak_memory_stats(device)
        start = time.perf_counter()
        output = step_function()
        torch.cuda.synchronize(device)
        duration = time.perf_counter() - start
```

```
        durations.append(duration)
        peaks.append(torch.cuda.max_memory_allocated(device))
        del output

    return {
        "median_seconds": statistics.median(durations),
        "maximum_peak_bytes": max(peaks),
        "all_seconds": durations,
        "all_peaks": peaks,
    }

def E19(baseline_model, optimized_model, inputs, make_step):
    device = inputs.device
    optimized_model.load_state_dict(
        copy.deepcopy(baseline_model.state_dict())
    )

    with torch.no_grad():
        reference = baseline_model(inputs)
        candidate = optimized_model(inputs)
    if not compare_outputs(reference, candidate):
        raise AssertionError("Forward outputs changed.")

    baseline_step = make_step(baseline_model)
    optimized_step = make_step(optimized_model)
    baseline_result = benchmark_step(
        baseline_step, device, repetitions=20
    )
    optimized_result = benchmark_step(
        optimized_step, device, repetitions=20
    )

    return {
        "baseline": baseline_result,
        "optimized": optimized_result,
    }
```

Part 6 - Measurement, Validation, and Failure Analysis

For every accepted change, show the profiler hotspot before and its reduction afterward. Compare global peaks and explain discrepancies. Validate forward outputs, loss, every parameter gradient, and input immutability. Report median runtime and variability. If memory improves while speed worsens, quantify the trade rather than declaring unconditional success. Re-run the original baseline at the end to detect clock or environment drift.

Part 7 - Engineering Interpretation and Philosophy

Evidence-constrained optimization protects the engineer from rewriting code according to folklore. The profiler does not choose the design, but it forces each change to answer a measured problem. The final lesson of DMM01 is that visibility creates responsibility: once memory has

been attributed, every optimization must explain what information was removed, why it was unnecessary, and what proof shows the model still performs the same computation.

19.4 DMM02

Chapter Title: PyTorch Caching Allocator
Specialization: Dynamic Memory Management and CUDA Systems

19.4.1 Exercise 00

Exercise Title: Build a Precise Mental Model of How the PyTorch CUDA Caching Allocator Reuses GPU Memory Blocks

Part 1 - Exercise Contract and Why It Matters

This exercise replaces the misleading model "a tensor is deleted, therefore its bytes immediately return to the GPU" with a three-owner model. Live tensor storage is owned by the program, cached blocks are owned by the PyTorch process but are currently reusable, and unreserved capacity is controlled by the CUDA driver. The caching allocator exists because repeatedly asking the driver for memory is expensive and may impose synchronization. Reusing a suitable block can therefore make a later allocation cheaper without changing its tensor semantics.

The deliverable is an annotated allocation timeline. For every event, record requested payload, allocated memory, reserved memory, inferred reusable memory, and driver free memory. Explain which transitions are observations and which are inferences. Do not treat every difference between reserved and allocated as a perfectly usable block: rounding, bookkeeping, fragmentation, stream safety, and allocator implementation details can reduce immediate usability.

Part 2 - System and Memory Mental Model

Let $A(t)$ denote bytes attributed to live PyTorch tensor storage and $R(t)$ bytes held by the allocator. A useful first approximation is

$$0 \le A(t) \le R(t), \qquad C_{\text{proxy}}(t) = R(t) - A(t).$$

The quantity C_{proxy} is a cached-capacity proxy, not a promise that one request of that size will succeed. The allocator manages larger segments obtained from CUDA and subdivides them into blocks. A released block may become inactive and reusable; adjacent compatible blocks may be merged; an oversized block may be split. Exact pools, rounding rules, and policies are version and backend dependent, so they must be measured rather than memorized as universal constants.

Allocation is also stream ordered. A block touched by outstanding work cannot always be reused immediately on an unrelated stream. CUDA kernels are asynchronous, Python reference deletion is not device synchronization, and views share storage with their base tensor. These facts explain many apparent contradictions in memory traces.

Part 3 - Quantitative Prediction Before Execution

Predict the following sequence before executing it: allocate tensor x, allocate y, delete x, allocate z with the same shape as x, then delete all tensors. For a tensor shape S and element width w, compute the payload $P = w \prod_i S_i$. Predict the direction, not an unsupported exact allocator increment: A rises with live payload, R may rise when no suitable block exists, deletion lowers A, and same-shape reallocation should often raise A without raising R. Include a second prediction in which z is slightly larger and explain why the cached block might fail to satisfy it.

Part 4 - Experimental Design and Instrumentation

Run in a fresh process on one explicit device. Synchronize before every measurement boundary, but recognize that synchronization changes timing and should not be inserted into production merely for observation. Warm CUDA once, clear only Python references between phases, and intentionally avoid `empty_cache()` so allocator ownership remains visible. Repeat the timeline at least three times and separate first-run initialization from steady-state reuse.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def mib(value):
    return value / (1024 ** 2)

def snapshot(label, device):
    torch.cuda.synchronize(device)
    allocated = torch.cuda.memory_allocated(device)
    reserved = torch.cuda.memory_reserved(device)
    free_bytes, total_bytes = torch.cuda.mem_get_info(device)
    row = {
        "label": label,
        "allocated_mib": mib(allocated),
        "reserved_mib": mib(reserved),
        "cached_proxy_mib": mib(reserved - allocated),
        "driver_free_mib": mib(free_bytes),
        "driver_total_mib": mib(total_bytes),
    }
    print(row)
    return row

def E00():
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required for DMM02.")
    device = torch.device("cuda:0")
    torch.empty(1, device=device)
    rows = [snapshot("baseline", device)]

    shape = (64, 1024, 1024)
    x = torch.empty(shape, dtype=torch.float32, device=device)
    rows.append(snapshot("allocate_x", device))
```

```
    y = torch.empty(shape, dtype=torch.float32, device=device)
    rows.append(snapshot("allocate_y", device))

    del x
    gc.collect()
    rows.append(snapshot("delete_x", device))
    z = torch.empty(shape, dtype=torch.float32, device=device)
    rows.append(snapshot("allocate_same_shape_z", device))

    del y
    del z
    gc.collect()
    rows.append(snapshot("delete_all", device))
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate that allocated memory follows tensor lifetime while reserved memory usually forms a high-water plateau. A convincing reuse event has three pieces of evidence: allocated rises by approximately the new payload, reserved does not materially rise, and the numerical behavior of the tensor remains normal. Do not infer reuse solely from driver free memory because CUDA context, libraries, and non-PyTorch allocations also consume capacity.

If deletion does not lower allocated memory, search for aliases, views, containers, closures, autograd graphs, or notebook outputs retaining storage. If reserved rises on the same-shape request, repeat after warm-up and inspect whether another live tensor or asynchronous stream use prevented reuse.

Part 7 - Engineering Interpretation and Philosophy

Caching is intentional ownership, not automatically a leak. The engineering question is whether live memory grows, whether reserved memory eventually stabilizes, and whether cached blocks match future requests. A sound allocator mental model never collapses logical lifetime, allocator ownership, and physical device capacity into one number. It also treats implementation details as measured properties of a specific PyTorch, CUDA, device, and configuration rather than eternal rules.

19.4.2 Exercise 01

Exercise Title: Predict Allocated, Reserved, and Reusable Memory After a Controlled Sequence of CUDA Tensor Allocations and Deallocations

Part 1 - Exercise Contract and Why It Matters

Turn allocator observation into prediction. You will write a ledger before running code, then compare it with synchronized measurements. The objective is not to guess allocator rounding exactly; it is to state which changes are forced by live tensor payload and which depend on cache suitability.

Part 2 - System and Memory Mental Model

For event k, distinguish requested payload P_k, live payload L_k, PyTorch allocated A_k, reserved R_k, and reusable proxy $C_k = R_k - A_k$. Views add no storage, clones do. Deleting a base while a view survives does not free the shared storage. Consequently, a reference graph is part of the memory equation.

Part 3 - Quantitative Prediction Before Execution

Construct a table for: allocate 96 MiB a; allocate 64 MiB b; create a view v of a; delete a; clone v into c; delete v; delete b; allocate a new 64 MiB d. Predict exact live payload and intervals for allocated and reserved. Mark reuse as “expected”, “possible”, or “not inferable” rather than asserting undocumented internals.

Part 4 - Experimental Design and Instrumentation

Generate sizes from byte targets using float32 element counts. Record a row after each event and retain the Python objects in a named dictionary so lifetime is visible. Use one stream, synchronize measurements, and run the sequence twice to compare cold and warm allocator states.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def make_mib(count_mib, device):
    elements = count_mib * 1024 * 1024 // 4
    return torch.empty(elements, dtype=torch.float32, device=device)

def memory_row(step, device):
    torch.cuda.synchronize(device)
    a = torch.cuda.memory_allocated(device)
    r = torch.cuda.memory_reserved(device)
    return step, a, r, r - a

def E01():
    device = torch.device("cuda:0")
    objects = {}
    rows = [memory_row("baseline", device)]
    objects["a"] = make_mib(96, device)
    rows.append(memory_row("a_96", device))
    objects["b"] = make_mib(64, device)
    rows.append(memory_row("b_64", device))
    objects["v"] = objects["a"].view(-1)
    rows.append(memory_row("view_a", device))
    del objects["a"]
    gc.collect()
    rows.append(memory_row("delete_a_keep_view", device))
    objects["c"] = objects["v"].clone()
    rows.append(memory_row("clone_view", device))
    del objects["v"]
    del objects["b"]
```

```
    gc.collect()
    rows.append(memory_row("release_view_and_b", device))
    objects["d"] = make_mib(64, device)
    rows.append(memory_row("new_64", device))
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Compare predicted and measured rows using deltas from baseline. Payload should match allocated approximately, while reserved may include rounding and cached segments. The critical validation is that deleting a while v survives does not release its storage, whereas deleting the last alias does. Explain any failure by reference lifetime before blaming the allocator.

Also compare each measured row with the previous row rather than only with the final state. A clone should create independent storage, whereas a view should not. If the final request is served without reserved growth, annotate the released lifetime that made capacity available. If it grows, explain why a global cached proxy cannot prove that a compatible contiguous block existed.

Part 7 - Engineering Interpretation and Philosophy

Memory budgeting is strongest when it predicts invariants and bounds, not fragile undocumented byte-for-byte outcomes. A useful engineer can say which storage must be live, which may be cached, and what evidence would distinguish reuse from a new reservation.

19.4.3 Exercise 02

Exercise Title: Create a Minimal PyTorch Experiment That Reveals Caching Allocator Behavior Without Calling empty_cache()

Part 1 - Exercise Contract and Why It Matters

Build the smallest experiment that visibly separates release from return to the driver. Minimality matters: a model, optimizer, autograd graph, and data loader would introduce lifetimes that obscure the allocator mechanism.

Part 2 - System and Memory Mental Model

After a tensor's final reference disappears, allocated memory should fall. The allocator may retain the corresponding block, leaving reserved memory high. A later compatible tensor may consume that block. This is the core caching signature: A changes with liveness while R remains on a plateau.

Part 3 - Quantitative Prediction Before Execution

Use one 256 MiB float32 tensor. Predict the signs of ΔA, ΔR, and ΔF_{driver} after allocation, deletion, and same-size reallocation. State why exact deltas may exceed payload and why the baseline CUDA context must be initialized first.

Part 4 - Experimental Design and Instrumentation

Run a fresh process, initialize CUDA with a one-element tensor, and take five synchronized snapshots. Never call `empty_cache()`. Avoid operations that initialize the large tensor because the experiment concerns allocation, not kernel time. Repeat the complete cycle to demonstrate reproducibility.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def state(label, device):
    torch.cuda.synchronize(device)
    allocated = torch.cuda.memory_allocated(device)
    reserved = torch.cuda.memory_reserved(device)
    free_bytes, total_bytes = torch.cuda.mem_get_info(device)
    return {
        "label": label,
        "allocated": allocated,
        "reserved": reserved,
        "reusable_proxy": reserved - allocated,
        "driver_free": free_bytes,
        "driver_total": total_bytes,
    }

def E02():
    device = torch.device("cuda:0")
    torch.empty(1, device=device)
    rows = [state("warm_baseline", device)]
    count = 256 * 1024 * 1024 // 4
    x = torch.empty(count, dtype=torch.float32, device=device)
    rows.append(state("allocated_x", device))
    del x
    gc.collect()
    rows.append(state("deleted_x", device))
    y = torch.empty(count, dtype=torch.float32, device=device)
    rows.append(state("allocated_y", device))
    del y
    gc.collect()
    rows.append(state("deleted_y", device))
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Plot allocated and reserved as step functions. Success requires a clear fall in allocated after deletion and a retained reserved plateau. If allocated does not fall, the experiment is not minimal: inspect notebook history, aliases, or exception tracebacks. If reserved also falls, verify the backend and process conditions; never rewrite observations to fit the expected story.

Repeat the experiment with 64, 128, and 256 MiB payloads. The qualitative signature should remain, but reservation increments may reveal rounding or pool differences. Include absolute counters and baseline-subtracted deltas; absolute values expose process ownership, while deltas isolate the experiment. Record whether another process shares the GPU because driver-free capacity can change independently of this workload.

Part 7 - Engineering Interpretation and Philosophy

A minimal counterexample is often more educational than a large benchmark. This experiment proves that "not live" and "returned to the driver" are different states. That distinction is the foundation for every later leak, fragmentation, and OOM diagnosis.

19.4.4 Exercise 03

Exercise Title: Prove That Deallocated Tensor Memory Can Remain Reserved and Be Reused by Later CUDA Allocations

Part 1 - Exercise Contract and Why It Matters

Move from suggestive counters to a controlled proof of reuse. You must show that a released tensor is no longer live, that its capacity remains reserved, and that a compatible request is satisfied without material reserved growth.

Part 2 - System and Memory Mental Model

Reuse is an allocator transition from inactive cached block to active block. Global counters do not expose the identity of a block, so the conclusion is an inference supported by controlled alternatives. The strongest basic evidence combines stable R, rising A, stable driver-free memory, and a matched request size.

Part 3 - Quantitative Prediction Before Execution

Compare two branches after releasing a 192 MiB tensor: branch S requests the same payload; branch L requests 384 MiB. Predict that S usually reuses capacity with little or no R growth, while L may require additional reservation. Define a tolerance before running, such as 2 MiB, instead of deciding after seeing the result.

Part 4 - Experimental Design and Instrumentation

Execute each branch in a separate fresh process or reset the experimental baseline carefully, because branch L permanently changes the cache state. Use identical warm-up and synchronization. Repeat ten cycles and report how often the same-size request stayed within the predeclared tolerance.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def counters(device):
    torch.cuda.synchronize(device)
    allocated = torch.cuda.memory_allocated(device)
    reserved = torch.cuda.memory_reserved(device)
    free_bytes, _ = torch.cuda.mem_get_info(device)
    return allocated, reserved, free_bytes

def allocate_mib(size_mib, device):
    count = size_mib * 1024 * 1024 // 4
    return torch.empty(count, dtype=torch.float32, device=device)

def E03(request_mib=192):
```

```
    device = torch.device("cuda:0")
    torch.empty(1, device=device)
    first = allocate_mib(192, device)
    live = counters(device)
    del first
    gc.collect()
    released = counters(device)
    second = allocate_mib(request_mib, device)
    reused = counters(device)
    del second
    return {
        "live": live,
        "released": released,
        "second_allocation": reused,
        "request_mib": request_mib,
    }
```

Part 6 - Measurement, Validation, and Failure Analysis

Compute $R_{second} - R_{released}$ and $F_{released} - F_{second}$. A near-zero pair supports reuse. Confirm that $A_{released}$ fell by approximately 192 MiB first. If the same-size branch grows reserved, test for other live allocations, streams, or allocator backend differences and report the limitation honestly.

Before accepting the conclusion, add a negative control that keeps the first tensor alive while requesting the second. Reserved memory should then grow if the working set exceeds existing capacity. This distinguishes reuse from a coincidental warm cache. A second control requests another dtype with the same byte payload and tests whether payload compatibility is sufficient in the observed allocator configuration.

Part 7 - Engineering Interpretation and Philosophy

Proof in systems work is rarely a single counter. It is a causal argument built from controls, predeclared expectations, and competing explanations. The claim should be "the evidence is consistent with allocator reuse under this configuration", not "all future PyTorch versions must reuse this block".

19.4.5 Exercise 04

Exercise Title: Compare First-Time CUDA Allocation Latency Against Reallocation Latency When Cached Blocks Are Reused

Part 1 - Exercise Contract and Why It Matters

Quantify the performance reason for caching. Compare a cold request that may obtain new memory from CUDA with a compatible request served after release. Separate allocator latency from tensor initialization and from asynchronous kernel execution.

Part 2 - System and Memory Mental Model

`torch.empty()` requests storage but does not intentionally fill every element, making it a better allocator probe than `zeros` or `randn`. CPU wall-clock timing around an asynchronous CUDA operation is still incomplete, so synchronize before and after each measured request.

The measurement includes Python and dispatch overhead; it is not a pure internal allocator clock.

Part 3 - Quantitative Prediction Before Execution

Predict median cold and cached latency for several payloads, but do not predict an exact speedup. State three confounders: CUDA context initialization, DVFS, and allocator warm state. Define primary statistics in advance: median, interquartile range, and reserved-memory delta.

Part 4 - Experimental Design and Instrumentation

Use separate subprocesses for truly cold trials; within one process, measure the first post-warm-up request and many release/reallocate cycles. Randomize size-test order across repetitions or disclose fixed ordering. Do not call `empty_cache()` between cached trials. Keep peak request safely below device capacity.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import statistics
import time
import torch

def timed_empty(elements, device):
    torch.cuda.synchronize(device)
    start = time.perf_counter_ns()
    value = torch.empty(elements, dtype=torch.float32, device=device)
    torch.cuda.synchronize(device)
    elapsed_us = (time.perf_counter_ns() - start) / 1000.0
    return value, elapsed_us

def E04(size_mib=256, repeats=30):
    device = torch.device("cuda:0")
    torch.empty(1, device=device)
    elements = size_mib * 1024 * 1024 // 4

    value, first_us = timed_empty(elements, device)
    del value
    gc.collect()
    cached_times = []
    reserved_deltas = []
    for _ in range(repeats):
        before = torch.cuda.memory_reserved(device)
        value, elapsed_us = timed_empty(elements, device)
        after = torch.cuda.memory_reserved(device)
        cached_times.append(elapsed_us)
        reserved_deltas.append(after - before)
        del value
        gc.collect()
```

```
    return {
        "first_us": first_us,
        "cached_median_us": statistics.median(cached_times),
        "cached_times_us": cached_times,
        "reserved_deltas": reserved_deltas,
    }
```

Part 6 - Measurement, Validation, and Failure Analysis

Report distributions, not a single fastest result. Validate cached trials by checking reserved deltas. If no speedup appears, the first request may already be served from warm cache, timing noise may dominate, or synchronization may cost more than the allocation. Repeat with larger sizes and fresh subprocesses before concluding caching has no value.

Present cold and cached latency on a logarithmic axis when outliers are large, and report the first trial separately rather than averaging it into steady state. Run a no-allocation synchronization control to estimate the measurement floor. If cached latency is near that floor, the benchmark cannot resolve allocator work precisely; it establishes only an upper bound.

Part 7 - Engineering Interpretation and Philosophy

The allocator trades retained capacity for lower allocation overhead and fewer driver interactions. This is a latency-through-memory trade, not free speed. Benchmarking should preserve the causal mechanism: measuring `zeros` would mostly measure initialization bandwidth, while calling `empty_cache()` would deliberately destroy the condition being studied.

19.4.6 Exercise 05

Exercise Title: Sweep Tensor Sizes and Determine When the Allocator Reuses Existing Blocks Versus Requests Additional GPU Memory

Part 1 - Exercise Contract and Why It Matters

Map reuse behavior across request sizes rather than drawing conclusions from a single matched allocation. The result is an empirical compatibility map for your exact PyTorch build, allocator backend, and GPU.

Part 2 - System and Memory Mental Model

A cached byte total is not a contiguous universal pool. Blocks have sizes and locations inside allocator-managed segments. A request may reuse a suitable block, split a larger block, or force new reservation. Therefore $R - A$ can exceed a request while that request still cannot be satisfied by a single suitable cached block.

Part 3 - Quantitative Prediction Before Execution

Seed the cache by releasing 32, 64, 128, and 256 MiB tensors. Sweep requests from 24 to 288 MiB. Before execution, classify each request as likely exact reuse, possible split/reuse, or likely growth. Define growth as a reserved increase greater than a predeclared 2 MiB tolerance.

Part 4 - Experimental Design and Instrumentation

Run each request size in a fresh subprocess with the identical seeded pattern; otherwise an early request mutates the cache available to later requests. Record allocated and reserved

before, during, and after the probe. Repeat the full sweep to expose variability, and never generalize boundaries beyond the tested configuration.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def tensor_mib(size_mib, device):
    count = size_mib * 1024 * 1024 // 4
    return torch.empty(count, dtype=torch.float32, device=device)

def run_probe(probe_mib):
    device = torch.device("cuda:0")
    seeds = []
    for size_mib in [32, 64, 128, 256]:
        seeds.append(tensor_mib(size_mib, device))
    torch.cuda.synchronize(device)
    seeds.clear()
    gc.collect()
    torch.cuda.synchronize(device)

    reserved_before = torch.cuda.memory_reserved(device)
    probe = tensor_mib(probe_mib, device)
    torch.cuda.synchronize(device)
    reserved_during = torch.cuda.memory_reserved(device)
    allocated_during = torch.cuda.memory_allocated(device)
    del probe
    gc.collect()
    return {
        "probe_mib": probe_mib,
        "allocated_during": allocated_during,
        "reserved_growth": reserved_during - reserved_before,
    }

def E05():
    results = []
    for size_mib in range(24, 289, 8):
        # For rigorous isolation, invoke run_probe in a fresh subprocess.
        results.append(run_probe(size_mib))
    return results
```

Part 6 - Measurement, Validation, and Failure Analysis

Plot request size against reserved growth and color points by the reuse classification. A plateau suggests cached service; a jump suggests acquisition or expansion. The in-process skeleton is pedagogical but order dependent, so a publication-quality result must isolate probes in subprocesses. Explain any non-monotonic boundaries as allocator behavior to investigate, not noise to silently remove.

For each point, attach the seeded cache state and requested byte count so the graph is reproducible. Repeat the sweep in descending order as an explicit order-sensitivity test. If ascending and descending boundaries differ, prior splitting and reuse changed the state; one run is not automatically wrong. Report both paths and their mutation histories.

Part 7 - Engineering Interpretation and Philosophy

Allocator compatibility is empirical. The useful outcome is not a reverse- engineered promise about internal bins; it is evidence that a workload's shape distribution either aligns with reusable capacity or repeatedly demands new segments.

19.4.7 Exercise 06

Exercise Title: Track Active, Inactive, Allocated, and Reserved Memory While Repeatedly Creating and Destroying CUDA Tensors

Part 1 - Exercise Contract and Why It Matters

Extend global counters into an allocator-state timeline. Track live allocated bytes, reserved bytes, and selected active/inactive statistics while tensor lifetimes repeatedly change. The aim is to separate workload liveness from allocator reuse capacity.

Part 2 - System and Memory Mental Model

`memory_stats()` exposes allocator metrics whose keys and precise semantics can vary by backend and version. Active blocks are associated with live or otherwise non-reusable storage; inactive bytes describe blocks inside allocator ownership that may be reusable or split. Never add every statistic together: many are alternative aggregations of the same bytes.

Part 3 - Quantitative Prediction Before Execution

For ten cycles of allocate 64 MiB, allocate 96 MiB, delete the first, then delete the second, predict a periodic allocated curve and a reserved curve that should plateau after warm-up. Predict when inactive bytes rise and fall. State that exact statistic keys must be discovered and documented.

Part 4 - Experimental Design and Instrumentation

Capture only a small selected set of counters after synchronization. Save CPU numbers, never tensor objects, in the trace. Run a fixed-shape control and then a variable-shape variant. Include PyTorch version and allocator backend configuration in the report.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def allocator_row(step, device):
    torch.cuda.synchronize(device)
    stats = torch.cuda.memory_stats(device)
    row = {
        "step": step,
        "allocated": torch.cuda.memory_allocated(device),
        "reserved": torch.cuda.memory_reserved(device),
```

```
    }
    wanted = [
        "active_bytes.all.current",
        "inactive_split_bytes.all.current",
        "segment.all.current",
    ]
    for key in wanted:
        row[key] = stats.get(key, None)
    return row

def make_mib(size_mib, device):
    count = size_mib * 1024 * 1024 // 4
    return torch.empty(count, device=device)

def E06(cycles=10):
    device = torch.device("cuda:0")
    trace = [allocator_row("baseline", device)]
    for cycle in range(cycles):
        first = make_mib(64, device)
        trace.append(allocator_row("first_" + str(cycle), device))
        second = make_mib(96, device)
        trace.append(allocator_row("second_" + str(cycle), device))
        del first
        gc.collect()
        trace.append(allocator_row("release_first_" + str(cycle), device))
        del second
        gc.collect()
        trace.append(allocator_row("release_second_" + str(cycle), device))
    return trace
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify the algebraic invariant $A \leq R$ and compare active bytes with allocated bytes without assuming equality. Missing statistic keys are an expected compatibility condition, not a reason to substitute fabricated zero. If the fixed workload grows indefinitely, inspect retained references and asynchronous stream use before diagnosing poor caching.

Inspect counter relationships at create and release boundaries, not only at the end of a cycle. A counter that rises only while a tensor is live has a different meaning from one that accumulates across cycles. Export the list of available memory-stat keys so another machine can identify backend or version differences instead of treating missing fields as numerical zeros.

Part 7 - Engineering Interpretation and Philosophy

Detailed counters are useful only when anchored to a lifetime story. More metrics do not automatically produce more understanding; each line must answer whether bytes are live, cached, fragmented, or newly acquired.

19.4.8 Exercise 07

Exercise Title: Design an Allocation Pattern That Reuses Cached Memory Efficiently and Verify That Reserved Memory Stabilizes

Part 1 - Exercise Contract and Why It Matters

Design a stable workload whose repeated shape set encourages reuse. The exercise models production systems that bucket inputs or reuse predictable batch shapes to reduce allocator churn.

Part 2 - System and Memory Mental Model

When the same set of maximum simultaneous allocations repeats, the allocator can often reach a steady state. After initial segments are obtained, later iterations cycle blocks between active and inactive states. Reserved memory then approaches a plateau even though cumulative allocation traffic grows.

Part 3 - Quantitative Prediction Before Execution

Choose three fixed payloads and a deterministic lifetime order. Predict the minimum concurrent live payload and an upper interval for steady reserved memory. Define stabilization: for example, no iteration after warm-up increases reserved memory by more than 2 MiB.

Part 4 - Experimental Design and Instrumentation

Run 100 iterations, discard 10 warm-up iterations only from performance statistics, and retain their memory trace. Record end-of-iteration allocated, reserved, and peak. Touch tensors with a small kernel so the workload is real, then synchronize at measurement boundaries.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def fixed_work(device):
    a = torch.empty(32 * 1024 * 1024 // 4, device=device)
    b = torch.empty(64 * 1024 * 1024 // 4, device=device)
    c = torch.empty(96 * 1024 * 1024 // 4, device=device)
    a.fill_(1.0)
    b.fill_(2.0)
    c.fill_(3.0)
    checksum = a[0] + b[0] + c[0]
    return a, b, c, checksum

def E07(iterations=100):
    device = torch.device("cuda:0")
    trace = []
    for iteration in range(iterations):
        a, b, c, checksum = fixed_work(device)
        torch.cuda.synchronize(device)
        peak_live = torch.cuda.memory_allocated(device)
        del a
        del b
        del c
        del checksum
        gc.collect()
```

```
        torch.cuda.synchronize(device)
        trace.append({
            "iteration": iteration,
            "peak_live": peak_live,
            "allocated_end": torch.cuda.memory_allocated(device),
            "reserved_end": torch.cuda.memory_reserved(device),
        })
    return trace
```

Part 6 - Measurement, Validation, and Failure Analysis

Plot reserved end-state by iteration and automatically find the first index after which the stabilization rule holds. Confirm checksum value so compiler or code changes do not remove the intended work. If end allocated grows, the problem is liveness, not cache inefficiency. If only reserved grows, investigate shape or concurrency drift.

Measure stabilization using both absolute and relative tolerances because a 2 MiB change means something different in a 200 MiB experiment and a 20 GiB experiment. Repeat with reversed allocation order while preserving simultaneous payload. A changed plateau reveals that lifetime order, not only total bytes, influences the allocator working set.

Part 7 - Engineering Interpretation and Philosophy

Predictability is a memory optimization. Bucketing, fixed microbatches, and bounded shape families can turn a turbulent allocation stream into a reusable working set. The correct target is stable memory with preserved model semantics, not the smallest possible reserved number at every instant.

19.4.9 Exercise 08

Exercise Title: Design an Irregular Allocation Pattern That Forces Reserved Memory Growth Despite Releasing Previous Tensors

Part 1 - Exercise Contract and Why It Matters

Construct the opposite of E07: live tensors are released, yet changing request sizes cause the allocator's reserved footprint to grow. This demonstrates why variable shapes can resemble a leak without retaining tensors.

Part 2 - System and Memory Mental Model

Released capacity may be unusable for a later request because no individual block is suitable or because splitting leaves fragments poorly matched to future sizes. A workload can therefore have low end-of-iteration allocated memory and rising reserved memory. This is allocation-pattern inefficiency, not proof of a Python reference leak.

Part 3 - Quantitative Prediction Before Execution

Create an irregular schedule that alternates many small allocations with one increasing large allocation. Predict end allocated near baseline and reserved as a staircase. Predeclare a safe maximum of 40 percent of currently free device memory to prevent an uncontrolled OOM.

Part 4 - Experimental Design and Instrumentation

Run the pattern in a fresh subprocess and compare it to a fixed-shape control with the same approximate total allocation traffic. Capture peak allocated and reserved per iteration. Stop before the safety budget, and report the stopped trial rather than silently reducing sizes after observing failure.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def make_mib(size_mib, device):
    count = size_mib * 1024 * 1024 // 4
    return torch.empty(count, device=device)

def E08(iterations=30):
    device = torch.device("cuda:0")
    free_bytes, _ = torch.cuda.mem_get_info(device)
    safety_budget = int(free_bytes * 0.40)
    trace = []
    for iteration in range(iterations):
        sizes = [9, 17, 33, 65, 11 + iteration * 7]
        requested = sum(sizes) * 1024 * 1024
        if requested > safety_budget:
            break
        tensors = []
        for size_mib in sizes:
            tensors.append(make_mib(size_mib, device))
        torch.cuda.synchronize(device)
        peak = torch.cuda.memory_allocated(device)
        tensors.clear()
        gc.collect()
        torch.cuda.synchronize(device)
        trace.append({
            "iteration": iteration,
            "sizes_mib": sizes,
            "peak_allocated": peak,
            "allocated_end": torch.cuda.memory_allocated(device),
            "reserved_end": torch.cuda.memory_reserved(device),
        })
    return trace
```

Part 6 - Measurement, Validation, and Failure Analysis

Show two curves: end allocated and end reserved. The intended signature is a flat first curve and a growing second curve. If both grow, inspect references. If neither grows, the allocator successfully reused or split blocks for this schedule; revise the hypothesis transparently rather than claiming forced growth that measurements did not show.

Quantify irregularity rather than describing it informally. Report unique size count, coefficient of variation, maximum concurrency, cumulative traffic, and reserved-growth event count.

Compare these values with E07. This prevents the analysis from attributing every difference to variability when the irregular schedule may also contain more work or a larger live peak.

Part 7 - Engineering Interpretation and Philosophy

Irregularity is not inherently wrong, but it has a memory cost. The lesson is to distinguish semantic variability required by the task from accidental shape churn introduced by preprocessing, batching, or control flow.

19.4.10 Exercise 09

Exercise Title: Compare Repeated torch.empty(), torch.zeros(), and torch.randn() CUDA Allocations Under the Same Memory Workload

Part 1 - Exercise Contract and Why It Matters

Separate storage allocation from tensor initialization. All three constructors request comparable storage, but zeros launches initialization work and randn generates random values. Confusing these costs produces false claims about the allocator.

Part 2 - System and Memory Mental Model

For identical shape and dtype, live payload is the same. Allocation can be served by cached blocks, while initialization kernels consume execution time and memory bandwidth. Random generation also updates generator state and may use implementation-specific temporary resources. Consequently memory parity does not imply latency parity.

Part 3 - Quantitative Prediction Before Execution

Predict equal allocated payload within tolerance, similar steady reserved plateaus, and latency ordered roughly as empty, zeros, randn for a large tensor. Treat the latency order as a hypothesis because hardware, size, and kernel implementation can change it. Predict how results change for tiny tensors where dispatch overhead dominates.

Part 4 - Experimental Design and Instrumentation

Benchmark constructors independently after warm-up. Use the same shape, dtype, repetition count, and synchronization. Delete each tensor before the next iteration. Randomize constructor order across runs or execute isolated subprocesses to prevent cache-order bias.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import statistics
import time
import torch

def benchmark_factory(name, shape, repeats, device):
    samples = []
    memory = []
    for _ in range(repeats):
        torch.cuda.synchronize(device)
        start = time.perf_counter_ns()
        if name == "empty":
```

```
            value = torch.empty(shape, device=device)
        elif name == "zeros":
            value = torch.zeros(shape, device=device)
        elif name == "randn":
            value = torch.randn(shape, device=device)
        else:
            raise ValueError(name)
        torch.cuda.synchronize(device)
        samples.append((time.perf_counter_ns() - start) / 1e6)
        memory.append((
            torch.cuda.memory_allocated(device),
            torch.cuda.memory_reserved(device),
        ))
        del value
        gc.collect()
    return statistics.median(samples), samples, memory

def E09():
    device = torch.device("cuda:0")
    shape = (64, 1024, 1024)
    results = {}
    for name in ["empty", "zeros", "randn"]:
        results[name] = benchmark_factory(name, shape, 30, device)
    return results
```

Part 6 - Measurement, Validation, and Failure Analysis

Compare medians and distributions, then verify payload equality from allocated deltas. Inspect outliers rather than discarding them automatically. If empty is not fastest, determine whether synchronization, cache state, or small workload size dominates. Do not use uninitialized values from empty in numerical checks.

Add two payload regimes: one small enough for dispatch overhead to dominate and one large enough for memory bandwidth to dominate. For zeros and randn, verify content semantics with a sampled reduction outside the timed interval. Report the CUDA generator seed and distinguish reproducibility of values from reproducibility of performance.

Part 7 - Engineering Interpretation and Philosophy

API names that all "create a tensor" can represent different system work. Good performance analysis decomposes allocation, initialization, randomness, and synchronization instead of assigning the entire observed latency to the caching allocator.

19.4.11 Exercise 10

Exercise Title: Call torch.cuda.empty_cache() at Different Points and Measure Its Effect on Reserved Memory, Allocation Latency, and Throughput

Part 1 - Exercise Contract and Why It Matters

Test `empty_cache()` as a specific intervention, not a ritual. Measure what it can release, what it cannot release, and the cost of destroying useful cached capacity at different workload boundaries.

Part 2 - System and Memory Mental Model

`empty_cache()` asks the allocator to release eligible unused cached blocks. It cannot release storage still referenced by live tensors and does not magically reduce model, gradient, activation, or optimizer-state memory. After cache release, later requests may require new driver allocations, increasing latency and possibly synchronization overhead.

Part 3 - Quantitative Prediction Before Execution

Predict three policies: never call it; call once after a completed workload phase; call every iteration. Predict allocated, reserved, driver free, median iteration time, and throughput. The one-time call should lower eligible reserved memory; the per-iteration policy should repeatedly sacrifice reuse.

Part 4 - Experimental Design and Instrumentation

Run policies in separate fresh processes with identical warm-up, shapes, seed, and 100 measured iterations. Synchronize around timing, but not within the actual step. Record state immediately before and after each cache call. Retain one live tensor during a special control to prove its bytes cannot be released.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import time
import torch

def workload(device):
    first = torch.randn((4096, 4096), device=device)
    second = torch.relu(first)
    result = second.sum()
    return first, second, result

def E10(policy="never", iterations=100):
    device = torch.device("cuda:0")
    samples = []
    memory = []
    for iteration in range(iterations):
        torch.cuda.synchronize(device)
        start = time.perf_counter()
        first, second, result = workload(device)
        del first
        del second
        del result
        gc.collect()
        if policy == "every_iteration":
            torch.cuda.empty_cache()
        if policy == "phase_end" and iteration == iterations - 1:
            torch.cuda.empty_cache()
        torch.cuda.synchronize(device)
        samples.append(time.perf_counter() - start)
```

```
        memory.append({
            "allocated": torch.cuda.memory_allocated(device),
            "reserved": torch.cuda.memory_reserved(device),
        })
    return samples, memory
```

Part 6 - Measurement, Validation, and Failure Analysis

Report median and tail latency, throughput, allocated, reserved, and driver free capacity. Prove the live-tensor control remains allocated across the call. If phase-end release is small, most bytes may still be live or not eligible. Do not claim performance effects from one run; caching and system noise demand repetition.

Measure the cache call itself and the first allocation after it as separate intervals. Charging only the call misses deferred cost paid by the next request, while charging the entire following workload can overstate that cost. Present both numbers and a combined phase cost. Verify that tensor or model outputs are unchanged across policies.

Part 7 - Engineering Interpretation and Philosophy

An API is useful when applied to a diagnosed state. Cache release can help cooperative multi-process use or phase transitions, but using it inside every iteration often exchanges allocator efficiency for a cosmetically smaller reserved counter.

19.4.12 Exercise 11

Exercise Title: Demonstrate Why Calling empty_cache() Every Training Iteration Can Reduce Performance Without Reducing Live Tensor Memory

Part 1 - Exercise Contract and Why It Matters

Embed the E10 intervention in a real training step. Demonstrate that parameter, gradient, activation, and optimizer requirements are determined by training semantics, while cache flushing only targets currently unused blocks.

Part 2 - System and Memory Mental Model

During forward and backward, live memory reaches a workload-defined peak. Flushing after the step may reduce idle reserved bytes, but the next step must reconstruct the same working set. Therefore peak allocated can remain nearly unchanged while allocation traffic and iteration latency worsen.

Part 3 - Quantitative Prediction Before Execution

For baseline and per-step cache-flush policies, predict identical loss trajectories and nearly identical peak allocated memory. Predict lower end reserved for the flush policy but worse throughput. Set parity tolerances for loss and parameter checks before running.

Part 4 - Experimental Design and Instrumentation

Clone identical model states, inputs, labels, optimizer configuration, and random seeds. Warm each policy, measure at least 100 steps, and synchronize only at timing boundaries. Reset peak statistics before each measured step or capture phase-wide peak consistently.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

def train_policy(flush_cache, steps=100):
    device = torch.device("cuda:0")
    torch.manual_seed(23)
    model = torch.nn.Sequential(
        torch.nn.Linear(2048, 4096),
        torch.nn.ReLU(),
        torch.nn.Linear(4096, 1000),
    ).to(device)
    optimizer = torch.optim.AdamW(model.parameters(), lr=0.0005)
    inputs = torch.randn((128, 2048), device=device)
    labels = torch.randint(0, 1000, (128,), device=device)
    rows = []
    for step in range(steps):
        torch.cuda.reset_peak_memory_stats(device)
        torch.cuda.synchronize(device)
        start = time.perf_counter()
        optimizer.zero_grad(set_to_none=True)
        logits = model(inputs)
        loss = torch.nn.functional.cross_entropy(logits, labels)
        loss.backward()
        optimizer.step()
        del logits
        del loss
        if flush_cache:
            torch.cuda.empty_cache()
        torch.cuda.synchronize(device)
        rows.append({
            "step": step,
            "seconds": time.perf_counter() - start,
            "peak_allocated": torch.cuda.max_memory_allocated(device),
            "reserved_end": torch.cuda.memory_reserved(device),
        })
    return rows

def E11():
    return train_policy(False), train_policy(True)
```

Part 6 - Measurement, Validation, and Failure Analysis

Compare post-warm-up throughput and peak-allocated distributions. Verify model outputs or loss history under deterministic settings; timing without semantic parity is invalid. If flushing appears faster, repeat randomized trial order and inspect whether thermal state or prior cache history biased the baseline.

Compare peak allocated before and after the flush rather than taking only an end-of-step sample. A flush can make the idle state smaller while the training peak remains identical.

Include optimizer initialization in warm-up because AdamW state is created lazily; otherwise the first measured step mixes state growth with allocator-policy cost.

Part 7 - Engineering Interpretation and Philosophy

Optimization targets the workload's critical path, not a visually pleasing counter. Lower reserved memory at idle is not an improvement when the same live peak returns every step and throughput falls.

19.4.13 Exercise 12

Exercise Title: Reproduce a Workload Where Changing Tensor Shapes Causes the Caching Allocator to Continuously Acquire New Memory Blocks

Part 1 - Exercise Contract and Why It Matters

Create a bounded variable-shape workload that exhibits repeated reservation growth. The experiment connects data-shape variability to allocator behavior without relying on a model memory leak.

Part 2 - System and Memory Mental Model

Changing shapes changes requested byte sizes and temporary-operation shapes. Previously cached blocks may be too small, badly partitioned, or otherwise unsuitable. Live memory can return to baseline after every iteration while reserved memory acquires new high-water levels.

Part 3 - Quantitative Prediction Before Execution

Generate a deterministic sequence of increasing and alternating dimensions. Compute payload for input, matrix output, and activation. Predict which iterations create a new live peak and which might grow reserved despite a lower live peak. Set a safe cap from driver-free capacity.

Part 4 - Experimental Design and Instrumentation

Use one fixed weight matrix and variable batch/sequence dimensions so the model is constant. Delete outputs and synchronize before each end-state sample. Run a fixed-shape control with the same number of operations. Stop at 35 percent of initial free memory.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def E12():
    device = torch.device("cuda:0")
    feature = 1024
    weight = torch.randn((feature, feature), device=device)
    free_bytes, _ = torch.cuda.mem_get_info(device)
    budget = int(free_bytes * 0.35)
    shapes = []
    for index in range(1, 41):
        batch = 8 + (index % 7) * 5
        length = 64 + index * 13
```

```
        shapes.append((batch, length, feature))

    trace = []
    for batch, length, width in shapes:
        estimated = batch * length * width * 4 * 3
        if estimated > budget:
            break
        inputs = torch.randn((batch, length, width), device=device)
        output = torch.matmul(inputs, weight)
        activated = torch.relu(output)
        torch.cuda.synchronize(device)
        peak_live = torch.cuda.memory_allocated(device)
        del inputs
        del output
        del activated
        gc.collect()
        torch.cuda.synchronize(device)
        trace.append({
            "shape": (batch, length, width),
            "peak_live": peak_live,
            "allocated_end": torch.cuda.memory_allocated(device),
            "reserved_end": torch.cuda.memory_reserved(device),
        })
    return trace
```

Part 6 - Measurement, Validation, and Failure Analysis

Plot reserved growth against requested payload and mark iterations that raised the previous live peak. Continuous acquisition is supported only if reserved has multiple positive jumps while end allocated remains stable. If autograd is enabled accidentally, graph retention changes the experiment; use no-grad or inputs without gradients as shown.

Decompose each reserved jump into two hypotheses: a genuinely larger live working set or inability to fit the new shape into existing cached blocks. A jump without a new allocated peak supports the second hypothesis. If both peaks rise together, additional reservation may simply be required by the larger workload and should not be labeled fragmentation.

Part 7 - Engineering Interpretation and Philosophy

Variable shapes are a systems property of the data pipeline, not merely tensor metadata. Bucketing and padding trade extra arithmetic for a more predictable allocation working set. The right choice requires joint measurement of memory, throughput, and task quality.

19.4.14 Exercise 13

Exercise Title: Compare Fixed-Shape and Variable-Shape Training Workloads and Quantify Their Impact on Reserved CUDA Memory

Part 1 - Exercise Contract and Why It Matters

Transform E12 into a fair training comparison. Quantify whether shape variability increases reservation growth, peak memory, or latency when model, sample count, optimizer, and numerical objective are controlled.

Part 2 - System and Memory Mental Model

Fixed shapes tend to repeat the same activation and workspace requirements. Variable shapes produce a distribution of requests. Padding to a bucket may increase per-step live payload but improve cache reuse and batching efficiency. Thus lower arithmetic volume does not guarantee lower reserved memory.

Part 3 - Quantitative Prediction Before Execution

Define fixed and variable schedules with the same total tokens. Predict total payload traffic, expected maximum live payload, reserved growth events, and tokens per second. Predeclare metrics: final reserved, maximum reserved, reserved slope after warm-up, and peak allocated.

Part 4 - Experimental Design and Instrumentation

Run schedules in separate fresh processes from identical model weights and seeds. Use the same optimizer and number of tokens, not merely the same number of steps. Synchronize for timing and store scalar loss only after detaching it to CPU. Do not call cache release during either policy.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

def run_schedule(schedule):
    device = torch.device("cuda:0")
    model = torch.nn.Sequential(
        torch.nn.Linear(512, 1024),
        torch.nn.GELU(),
        torch.nn.Linear(1024, 512),
    ).to(device)
    optimizer = torch.optim.AdamW(model.parameters(), lr=0.0003)
    rows = []
    for step, shape in enumerate(schedule):
        batch, length = shape
        inputs = torch.randn((batch, length, 512), device=device)
        targets = torch.randn((batch, length, 512), device=device)
        torch.cuda.reset_peak_memory_stats(device)
        torch.cuda.synchronize(device)
        start = time.perf_counter()
        optimizer.zero_grad(set_to_none=True)
        predictions = model(inputs)
        loss = torch.nn.functional.mse_loss(predictions, targets)
        loss.backward()
        optimizer.step()
        torch.cuda.synchronize(device)
        rows.append({
            "step": step,
            "tokens": batch * length,
            "seconds": time.perf_counter() - start,
```

```
            "peak_allocated": torch.cuda.max_memory_allocated(device),
            "reserved": torch.cuda.memory_reserved(device),
            "loss": float(loss.detach().cpu()),
        })
        del inputs, targets, predictions, loss
    return rows

def E13():
    # Both schedules contain exactly 220,160 tokens.
    fixed = [(16, 344)] * 40
    variable = [(8, 128), (32, 384), (16, 256), (24, 192)] * 10
    return run_schedule(fixed), run_schedule(variable)
```

Part 6 - Measurement, Validation, and Failure Analysis

Normalize throughput by tokens and verify the equal schedule token totals. Compare reserved slopes after warm-up with confidence intervals across process repetitions. If variable shapes win, explain whether smaller average work outweighed allocator churn rather than rejecting the measurement.

Check numerical comparability by evaluating both trained models on one fixed validation batch. Equal token count does not guarantee identical gradient statistics because batching and shape composition differ. Report quality beside systems metrics, and state whether padding changes the loss mask. Training on padding tokens is not a valid memory optimization.

Part 7 - Engineering Interpretation and Philosophy

Shape policy is an end-to-end design choice. Fixed buckets can waste padded compute but stabilize memory; exact shapes can save compute but diversify allocations. Engineering requires measuring both sides under equal useful work.

19.4.15 Exercise 14

Exercise Title: Build a PyTorch Allocation Trace That Records Tensor Size, Lifetime, Allocated Memory, and Reserved Memory for Every Step

Part 1 - Exercise Contract and Why It Matters

Build an explicit semantic trace connecting tensor birth and death to global allocator counters. This trace is intentionally application-level: it teaches you to account for lifetimes before moving to lower-level allocator snapshots.

Part 2 - System and Memory Mental Model

Each trace event has an operation, logical tensor identifier, payload, lifetime state, allocated, and reserved. Payload is computed from shape, element size, and count. The trace must store metadata only; storing the tensor inside the trace would extend its lifetime and invalidate the measurement.

Part 3 - Quantitative Prediction Before Execution

Draw a Gantt-style lifetime plan for three overlapping tensors. Calculate live payload after each create/delete event and predict where allocated peaks. Add a view event and predict zero

new payload. Define a residual $E = A - A_{baseline} - L_{payload}$ to expose allocator rounding or untracked live storage.

Part 4 - Experimental Design and Instrumentation

Centralize creation and deletion in named functions. Synchronize before global counter capture and add a monotonically increasing event index. Record dtype, shape, payload, and alive identifiers as immutable CPU metadata. Validate the trace on a small deterministic sequence before using it with models.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

class AllocationTrace:
    def __init__(self, device):
        self.device = device
        self.objects = {}
        self.rows = []

    def record(self, action, name, payload=0):
        torch.cuda.synchronize(self.device)
        self.rows.append({
            "event": len(self.rows),
            "action": action,
            "name": name,
            "payload": payload,
            "alive": tuple(sorted(self.objects.keys())),
            "allocated": torch.cuda.memory_allocated(self.device),
            "reserved": torch.cuda.memory_reserved(self.device),
        })

    def create(self, name, shape):
        value = torch.empty(shape, device=self.device)
        self.objects[name] = value
        payload = value.numel() * value.element_size()
        self.record("create", name, payload)

    def view(self, source, name):
        self.objects[name] = self.objects[source].view(-1)
        self.record("view", name, 0)

    def delete(self, name):
        del self.objects[name]
        gc.collect()
        self.record("delete", name, 0)

def E14():
    trace = AllocationTrace(torch.device("cuda:0"))
```

```
    trace.record("baseline", "none")
    trace.create("a", (32, 1024, 1024))
    trace.create("b", (16, 1024, 1024))
    trace.view("a", "a_view")
    trace.delete("a")
    trace.create("c", (24, 1024, 1024))
    trace.delete("a_view")
    trace.delete("b")
    trace.delete("c")
    return trace.rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Recompute live payload from the event log and compare it to allocated deltas. Assert event order, unique names, and valid deletion. Confirm the view keeps base storage alive after the base name is deleted. If trace collection changes lifetime, inspect metadata for tensor objects or closures capturing them.

Add automatic consistency checks: every create introduces a unique identifier, every delete targets a live identifier, payload is nonnegative, and allocated never exceeds reserved. Serialize only CPU-native types. Then reconstruct the lifetime diagram from the saved trace to prove the record is self-contained and does not rely on hidden tensor objects.

Part 7 - Engineering Interpretation and Philosophy

Instrumentation is part of the experiment and can create the bug it observes. A trustworthy trace records identities and numbers without retaining storage. It connects semantic objects to allocator state while preserving the workload's true lifetime graph.

19.4.16 Exercise 15

Exercise Title: Detect a False Memory-Leak Diagnosis Caused by Misinterpreting Cached Reserved Memory as Live Tensor Memory

Part 1 - Exercise Contract and Why It Matters

Diagnose a deliberately misleading trace in which reserved memory stays high after tensors are deleted. Decide whether it is a live-reference leak, a stable cache, or an allocator-pattern problem. A wrong diagnosis can lead to harmful cache flushing or unnecessary model redesign.

Part 2 - System and Memory Mental Model

A leak requires loss of control over memory that remains live or grows without intended bounds. Stable reserved memory with allocated returning to baseline is not sufficient evidence. Use three signatures: live leak means end allocated trends upward; stable cache means allocated returns and reserved plateaus; poor-fit caching means allocated returns while reserved continues to acquire capacity under changing requests.

Part 3 - Quantitative Prediction Before Execution

Predict traces for a correct loop, a loop appending CUDA losses to a Python list, and a variable-shape loop. Specify diagnostic thresholds: allocated slope, reserved slope after warm-up, and number of live Python tensor references. Avoid classifying from a single endpoint.

Part 4 - Experimental Design and Instrumentation

Run each scenario in a fresh process for equal iterations. Detach scalars to CPU in the correct version. Synchronize at end-of-iteration measurement and fit slopes only after warm-up. Use `empty_cache()` once as a diagnostic probe after all live references are controlled, not as the primary test.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def run_case(retain_losses, variable_shapes, steps=80):
    device = torch.device("cuda:0")
    retained = []
    rows = []
    for step in range(steps):
        width = 1024
        batch = 64
        if variable_shapes:
            batch = 32 + (step % 11) * 7
        value = torch.randn(
            (batch, width, width // 8),
            device=device,
            requires_grad=True,
        )
        loss = value.square().mean()
        if retain_losses:
            retained.append(loss)
        else:
            retained.append(float(loss.detach().cpu()))
            if len(retained) > 4:
                retained.pop(0)
        del value
        del loss
        gc.collect()
        torch.cuda.synchronize(device)
        rows.append({
            "step": step,
            "allocated": torch.cuda.memory_allocated(device),
            "reserved": torch.cuda.memory_reserved(device),
        })
    return rows

def E15():
    stable = run_case(False, False)
    true_leak = run_case(True, False)
    variable_cache = run_case(False, True)
    return stable, true_leak, variable_cache
```

Part 6 - Measurement, Validation, and Failure Analysis

Label traces without seeing scenario names, then reveal them. The retained-loss case should increase allocated because each scalar retains its autograd graph; the stable case should plateau. Confirm that the input requires gradients and that the retained losses have a non-null gradient function, so the intended true-leak control actually retains a graph. This validation is essential: an unverified synthetic bug proves nothing.

Use a robust slope estimate and inspect the raw curve because allocator growth is stepwise rather than perfectly linear. After classification, fix only the true-leak case by storing detached CPU scalars, then rerun from a fresh process. A diagnosis is complete only when the predicted signature disappears without changing the intended numerical computation.

Part 7 - Engineering Interpretation and Philosophy

Leak diagnosis is classification under uncertainty. The cure depends on the class: remove references for a live leak, stabilize shapes for poor cache fit, or do nothing when a healthy cache is serving the workload efficiently.

19.4.17 Exercise 16

Exercise Title: Stress-Test the Caching Allocator with Thousands of Mixed-Size CUDA Allocations and Identify When Memory Reuse Efficiency Degrades

Part 1 - Exercise Contract and Why It Matters

Stress the allocator with thousands of bounded mixed-size requests while preserving experimental safety. Identify the workload region where reserved growth or latency indicates degrading reuse efficiency.

Part 2 - System and Memory Mental Model

Allocator stress depends on both size distribution and lifetime overlap. Total requested traffic can be enormous while peak live memory stays moderate. Efficiency degradation appears when new reservations grow relative to repeated traffic, inactive fragmentation proxies rise, or allocation latency develops a tail.

Part 3 - Quantitative Prediction Before Execution

Choose a reproducible size distribution from 1 to 64 MiB and random lifetimes of 1 to 20 events. Estimate maximum simultaneous payload by simulating the schedule on CPU first. Reject schedules exceeding 35 percent of initial free VRAM. Define a reuse proxy:

$$\eta = 1 - \frac{\max(0, R_{end} - R_{start})}{\sum_k P_k},$$

and state that it measures reservation-growth efficiency, not exact block reuse.

Part 4 - Experimental Design and Instrumentation

Pre-generate the complete event schedule with a fixed CPU seed. Keep a bounded dictionary of live tensors and delete them at scheduled expiry. Sample every 25 events to reduce synchronization distortion. Run in a disposable subprocess so an OOM cannot contaminate later chapter experiments.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import random
import time
import torch

def E16(events=3000, seed=41):
    device = torch.device("cuda:0")
    generator = random.Random(seed)
    schedule = []
    for event in range(events):
        size_mib = generator.randint(1, 64)
        lifetime = generator.randint(1, 20)
        schedule.append((event, size_mib, lifetime))

    live = {}
    trace = []
    free_bytes, _ = torch.cuda.mem_get_info(device)
    budget = int(free_bytes * 0.35)
    for event, size_mib, lifetime in schedule:
        expired = []
        for name, item in live.items():
            _, death = item
            if death <= event:
                expired.append(name)
        for name in expired:
            del live[name]
        gc.collect()

        requested = size_mib * 1024 * 1024
        current_live = torch.cuda.memory_allocated(device)
        if current_live + requested > budget:
            continue
        count = requested // 4
        start = time.perf_counter_ns()
        tensor = torch.empty(count, device=device)
        live[str(event)] = (tensor, event + lifetime)
        if event % 25 == 0:
            torch.cuda.synchronize(device)
            trace.append({
                "event": event,
                "latency_us": (time.perf_counter_ns() - start) / 1000,
                "allocated": torch.cuda.memory_allocated(device),
                "reserved": torch.cuda.memory_reserved(device),
                "live_count": len(live),
            })
    return trace
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate the CPU schedule's predicted peak against observed allocated memory. Plot reserved, allocated, cached proxy, and latency quantiles over event index. The sparse timing skeleton measures synchronization at samples, so a rigorous latency benchmark should isolate every timed allocation; disclose this tradeoff. Never interpret skipped budget events as successful allocations.

Run three controlled distributions: narrow repeated sizes, uniform mixed sizes, and a heavy tail with occasional large requests. Hold event count and expected traffic approximately constant. Report skipped events and achieved traffic, because the safety controller changes the executed distribution. Separate allocator degradation from workload admission effects.

Part 7 - Engineering Interpretation and Philosophy

Stress tests should reveal boundaries, not recklessly cross them. A useful result identifies which distribution and lifetime properties degrade reuse and turns them into production controls such as bucketing or admission limits.

19.4.18 Exercise 17

Exercise Title: Benchmark Memory Reuse Efficiency, Allocation Latency, and Peak VRAM Across Stable, Bursty, and Highly Variable Workloads

Part 1 - Exercise Contract and Why It Matters

Build a controlled benchmark comparing three workload families. The deliverable must distinguish memory footprint, allocation latency, and useful throughput; no single score can describe allocator quality.

Part 2 - System and Memory Mental Model

Stable workloads repeat sizes and concurrency. Bursty workloads periodically raise their live high-water mark. Highly variable workloads diversify size and lifetime. These axes affect peak allocated, peak reserved, reservation growth, cache proxy, and latency differently.

Part 3 - Quantitative Prediction Before Execution

Equalize total requested bytes and event count across families. Predict stable to plateau earliest, bursty to reserve for its largest burst, and variable to show the most growth or fragmentation. Define throughput as processed requested MiB per second and report it beside latency percentiles.

Part 4 - Experimental Design and Instrumentation

Generate schedules before GPU execution, validate their equality, and run each in a fresh subprocess. Use multiple seeds for variable workloads. Randomize family order, warm CUDA equally, reset peaks, and collect environment metadata.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import statistics
import time
import torch

def execute_schedule(name, sizes_mib):
```

```
    device = torch.device("cuda:0")
    timings = []
    torch.cuda.reset_peak_memory_stats(device)
    reserved_start = torch.cuda.memory_reserved(device)
    for size_mib in sizes_mib:
        count = size_mib * 1024 * 1024 // 4
        torch.cuda.synchronize(device)
        start = time.perf_counter_ns()
        value = torch.empty(count, device=device)
        torch.cuda.synchronize(device)
        timings.append((time.perf_counter_ns() - start) / 1000)
        del value
        gc.collect()
    return {
        "name": name,
        "events": len(sizes_mib),
        "traffic_mib": sum(sizes_mib),
        "median_us": statistics.median(timings),
        "max_us": max(timings),
        "peak_allocated": torch.cuda.max_memory_allocated(device),
        "peak_reserved": torch.cuda.max_memory_reserved(device),
        "reserved_growth": (
            torch.cuda.memory_reserved(device) - reserved_start
        ),
    }

def E17():
    # Every family has 400 events and 19,200 MiB of request traffic.
    stable = [48] * 400
    bursty = ([16] * 8 + [176, 176]) * 40
    variable = [7, 91, 23, 115, 41, 67, 11, 29] * 50
    return (
        execute_schedule("stable", stable),
        execute_schedule("bursty", bursty),
        execute_schedule("variable", variable),
    )
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify that the schedules have identical traffic and event counts before every run. Report medians, p95/p99, peaks, and growth across repeated subprocesses. A workload can win latency while losing peak VRAM; preserve this tradeoff rather than collapsing it into an arbitrary average score.

Add confidence intervals across fresh-process repetitions and publish the schedule generator. Peak metrics require reset and synchronization at identical boundaries. Include p50, p95, p99, and worst latency because bursty driver acquisitions may disappear in the median. State which percentile and memory ceiling the final decision optimizes.

Part 7 - Engineering Interpretation and Philosophy

Benchmarks are models of deployment. A representative distribution and lifetime pattern matter more than a perfect microbenchmark. State which production question each workload family represents and what it omits.

19.4.19 Exercise 18

Exercise Title: Build a Reusable Allocator Experiment Harness That Visualizes Live Memory, Cached Memory, Reuse Rate, and Allocation Growth

Part 1 - Exercise Contract and Why It Matters

Package the chapter's instrumentation into a reusable harness. It must execute a workload, collect scalar memory events, calculate explicitly named proxy metrics, validate invariants, and produce plots without retaining CUDA tensors.

Part 2 - System and Memory Mental Model

Live memory is measured by allocated bytes. Cached proxy is $R - A$. Allocation growth is the positive change in reserved bytes. Exact reuse is not observable from these globals, so define "reuse rate" as a proxy based on requested traffic not accompanied by reservation growth. Label it clearly to prevent a visualization from overstating evidence.

Part 3 - Quantitative Prediction Before Execution

Specify harness inputs, outputs, and invariants before implementation. Inputs include device, schedule, safety fraction, synchronization policy, and seed. Outputs include event rows, environment metadata, summary statistics, and four plots. Predict results for fixed and variable schedules to unit-test the harness.

Part 4 - Experimental Design and Instrumentation

Separate schedule generation, execution, metric calculation, and rendering. The executor returns only CPU dictionaries. Write plot functions that consume saved rows so visualization never reruns GPU work. Include baselines and use MiB consistently.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

class AllocatorHarness:
    def __init__(self, device, safety_fraction=0.35):
        self.device = torch.device(device)
        free_bytes, _ = torch.cuda.mem_get_info(self.device)
        self.budget = int(free_bytes * safety_fraction)

    def run(self, sizes_mib):
        rows = []
        previous_reserved = torch.cuda.memory_reserved(self.device)
        total_requested = 0
        total_growth = 0
        for event, size_mib in enumerate(sizes_mib):
            request = size_mib * 1024 * 1024
```

```
            if request > self.budget:
                raise ValueError("Request exceeds the safety budget.")
            value = torch.empty(request // 4, device=self.device)
            torch.cuda.synchronize(self.device)
            allocated = torch.cuda.memory_allocated(self.device)
            reserved = torch.cuda.memory_reserved(self.device)
            growth = max(0, reserved - previous_reserved)
            total_requested += request
            total_growth += growth
            rows.append({
                "event": event,
                "request": request,
                "allocated": allocated,
                "reserved": reserved,
                "cached_proxy": reserved - allocated,
                "reserved_growth": growth,
                "reuse_proxy": 1.0 - total_growth / total_requested,
            })
            previous_reserved = reserved
            del value
            gc.collect()
        return rows

def E18():
    harness = AllocatorHarness("cuda:0")
    fixed_rows = harness.run([32, 64, 32, 64] * 25)
    return fixed_rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Assert nonnegative allocated, reserved, cached proxy, and growth, plus $A \leq R$. Explain that the reuse proxy can become misleading when baseline cache already contains blocks or when released blocks serve future requests; report baseline and treat the metric comparatively. Test empty, one-event, oversized, and repeated schedules.

Place allocated and reserved on one axis, cached proxy on a derived panel, positive reservation growth as event bars, and reuse proxy with its definition in the legend. Include request size and event index in the supporting table. Never smooth stepwise counters into an apparently continuous physical process without labeling the smoothing as cosmetic.

Part 7 - Engineering Interpretation and Philosophy

Reusable tooling is valuable only when its names preserve epistemic limits. Calling a proxy "exact reuse" would convert a convenient formula into false certainty. Good harnesses automate evidence collection while keeping the meaning of every metric auditable.

19.4.20 Exercise 19

Exercise Title: Build and Defend a Memory Allocation Policy for a Variable-Shape PyTorch Workload Using Measured Caching Allocator Behavior

Part 1 - Exercise Contract and Why It Matters

Synthesize the chapter into a defendable allocation policy for a variable- shape service or training pipeline. The policy must connect measured evidence to operational rules, failure handling, and performance objectives.

Part 2 - System and Memory Mental Model

A policy controls shape bucketing, maximum request size, concurrency, cache release boundaries, admission, and observability. It cannot change required live tensor bytes, but it can shape request distribution and lifetime overlap. Reserved headroom must coexist with CUDA context, libraries, and other processes; using every reported free byte is unsafe.

Part 3 - Quantitative Prediction Before Execution

Define a service-level budget: maximum allocated peak, reserved ceiling, minimum throughput, p95 latency, and OOM-free safety margin. Compare at least three candidate policies: exact shapes, coarse buckets, and fixed maximum padding. Predict compute waste, cache reuse, and latency for each before benchmarking.

Part 4 - Experimental Design and Instrumentation

Replay the same recorded shape distribution through every policy in fresh processes. Include normal, burst, and worst-allowed traffic. Measure requested payload, padded payload, allocated/reserved peaks, reservation growth, latency, and throughput. Reserve a holdout trace so the policy is not optimized only for the development distribution.

Part 5 - PyTorch/CUDA Implementation

```
import bisect

class AllocationPolicy:
    def __init__(self, buckets, maximum_tokens, safety_margin_mib):
        self.buckets = sorted(buckets)
        self.maximum_tokens = maximum_tokens
        self.safety_margin_mib = safety_margin_mib

    def choose_bucket(self, tokens):
        if tokens <= 0:
            raise ValueError("tokens must be positive")
        if tokens > self.maximum_tokens:
            return {"decision": "reject", "reason": "token_limit"}
        index = bisect.bisect_left(self.buckets, tokens)
        if index == len(self.buckets):
            return {"decision": "reject", "reason": "no_bucket"}
        return {
            "decision": "admit",
            "requested_tokens": tokens,
            "bucket_tokens": self.buckets[index],
            "padding_tokens": self.buckets[index] - tokens,
        }
```

```
def E19(shape_trace):
    policy = AllocationPolicy(
        buckets=[128, 256, 512, 1024, 2048],
        maximum_tokens=2048,
        safety_margin_mib=768,
    )
    decisions = []
    for tokens in shape_trace:
        decisions.append(policy.choose_bucket(tokens))
    # Feed admitted bucket sizes into the E18 harness, then attach
    # measured memory and latency evidence to every policy decision.
    return decisions
```

Part 6 - Measurement, Validation, and Failure Analysis

Defend every threshold with a measured curve and safety margin. Validate policy behavior at bucket edges, impossible requests, bursts, and distribution shift. Specify automatic responses: reduce concurrency, split work, reject safely, or move to a larger resource class. Cache flushing should appear only at measured phase boundaries where its cost is justified. Include rollback criteria if the new policy worsens latency or correctness.

Require a decision table containing observed condition, measured evidence, selected action, expected benefit, possible harm, and rollback signal. Test on the holdout distribution and inject oversized and burst requests deliberately. A policy that works on average but has no controlled tail response is not ready for production.

Part 7 - Engineering Interpretation and Philosophy

The final policy is not "make reserved memory small." It is a contract that balances useful work, predictable capacity, latency, and failure containment. Evidence from one GPU and software version establishes a baseline, not a universal law; the policy must be revalidated after upgrades, model changes, or traffic shifts. The deepest lesson of allocator engineering is to control distributions and lifetimes while preserving semantic correctness, then let measurements decide which tradeoffs are acceptable.

19.5 DMM03

Chapter Title: PyTorch CUDA Memory Statistics
Specialization: Dynamic Memory Management and CUDA Systems

19.5.1 Exercise 00

Exercise Title: Build a Precise Mental Model of What torch.cuda.memory_stats() Reveals About the CUDA Caching Allocator

Part 1 - Exercise Contract and Why It Matters

This exercise turns `torch.cuda.memory_stats()` from a large dictionary of intimidating names into a structured account of allocator history and state. The API exposes counters for allocation requests, allocator-managed segments, active blocks, inactive split blocks, byte totals, retries, and OOM events. It is more detailed than `memory_allocated()` and

`memory_reserved()`, but it remains an allocator summary rather than a complete map of every physical GPU byte.

Your deliverable is a counter taxonomy and a synchronized event timeline. For every selected key, identify three dimensions: the resource being counted, the pool scope, and the statistic form. Explain whether the value describes a current state, a historical peak, cumulative acquisition, or cumulative release. Never compare counters with different units as though they were the same quantity.

Part 2 - System and Memory Mental Model

Many keys follow a conceptual pattern `metric.pool.statistic`. A metric may count events or bytes; the pool may be `all`, `small_pool`, or `large_pool`; and the statistic may be `current`, `peak`, `allocated`, or `freed`. In simplified form,

$$X_{current} = X_{allocated} - X_{freed},$$

when the selected counter family obeys cumulative acquisition/release accounting. Verify this identity empirically rather than imposing it on every unfamiliar key. An allocation count describes allocator events, while allocated bytes describe capacity. One 256 MiB allocation and 256 one-MiB allocations can therefore have similar bytes but radically different event counts.

Segments are larger regions obtained by the allocator from CUDA. Blocks live inside segments. Active bytes generally correspond to blocks not currently available for reuse, allocated bytes track tensor storage recognized as live, and inactive split bytes expose capacity stranded in split blocks. Exact relationships are backend and version dependent. The native and asynchronous allocator backends may expose different meaningful fields, so absence of a key must remain `None`, not be rewritten as zero.

Part 3 - Quantitative Prediction Before Execution

Before running, predict counter directions for: warm CUDA; allocate a 64 MiB tensor; create a view; clone the view; delete the original while the view survives; delete the last alias; then delete the clone. Predict which events change allocation counts, byte counts, active state, and reserved segments. Create a units column containing “events”, “bytes”, or “occurrences”. Also predict which cumulative counters never decrease during a process.

Part 4 - Experimental Design and Instrumentation

Use one fresh process, one explicit CUDA device, fixed shapes, and synchronized measurement boundaries. Capture the full dictionary once for discovery, then log a deliberately small selected subset during the experiment. Record PyTorch version, device name, and allocator configuration. Avoid resetting statistics until the timeline is finished because resets change historical peaks and would destroy the story being analyzed.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

SELECTED_KEYS = [
    "allocation.all.current",
```

```
    "allocation.all.peak",
    "allocated_bytes.all.current",
    "allocated_bytes.all.peak",
    "reserved_bytes.all.current",
    "reserved_bytes.all.peak",
    "active_bytes.all.current",
    "inactive_split_bytes.all.current",
    "segment.all.current",
    "num_alloc_retries",
    "num_ooms",
]

def stats_row(label, device):
    torch.cuda.synchronize(device)
    raw = torch.cuda.memory_stats(device)
    row = {"label": label}
    for key in SELECTED_KEYS:
        row[key] = raw.get(key, None)
    return row

def E00():
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required for DMM03.")
    device = torch.device("cuda:0")
    torch.empty(1, device=device)
    rows = [stats_row("baseline", device)]

    x = torch.empty((16, 1024, 1024), device=device)
    rows.append(stats_row("allocate_x", device))
    view = x.view(-1)
    rows.append(stats_row("create_view", device))
    clone = view.clone()
    rows.append(stats_row("clone_view", device))
    del x
    gc.collect()
    rows.append(stats_row("delete_x_keep_view", device))
    del view
    gc.collect()
    rows.append(stats_row("delete_last_alias", device))
    del clone
    gc.collect()
    rows.append(stats_row("delete_clone", device))
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify that a view does not request independent storage and that deleting a base name does not release storage while an alias survives. Compare `allocated_bytes.all.current` with `memory_allocated()` and reserved bytes with `memory_reserved()` at the same synchronized

boundary. Small differences require investigation; comparisons captured at different times are invalid.

For every missing key, report backend/version incompatibility explicitly. If a cumulative counter decreases unexpectedly, verify that a reset API was not called. If allocated bytes remain high after all intended references are deleted, inspect autograd graphs, containers, notebook output, and exception tracebacks before claiming allocator failure.

Part 7 - Engineering Interpretation and Philosophy

The dictionary is a language for hypotheses, not a diagnosis generator. A reliable interpretation moves from semantic lifetime, to allocator state, to counter evidence. It never labels reserved bytes a leak, inactive split bytes proof of fragmentation by themselves, or retries proof of failure without a controlled workload history. The purpose of detailed statistics is to make a causal argument more defensible, not merely more numerical.

19.5.2 Exercise 01

Exercise Title: Print the Complete memory_stats() Dictionary and Classify Counters into Allocation, Segment, Active, Inactive, and Retry Categories

Part 1 - Exercise Contract and Why It Matters

Discover the actual dictionary supplied by your installed PyTorch rather than copying a static list from another version. Classify every key by category, pool, statistic form, and unit, preserving unknown keys for later research.

Part 2 - System and Memory Mental Model

Key names encode a hierarchy, but not every counter uses every component. Allocation and segment counters may describe events; byte families describe capacity; retry and OOM counters are scalar occurrences. A robust classifier must tolerate keys with one, two, or three dotted fields and must never discard an unfamiliar field merely because it violates the expected template.

Part 3 - Quantitative Prediction Before Execution

Predict the major categories and count how many keys you expect in each. Define classification rules in writing first. Mark units from the metric name rather than numeric magnitude, because a value of 4096 could mean events or bytes.

Part 4 - Experimental Design and Instrumentation

Initialize CUDA, capture one full dictionary, sort keys for deterministic output, and write a CPU-native table. Run again after a meaningful allocation to distinguish keys that exist but remain zero from keys that do not exist.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def classify_key(key):
    prefix = key.split(".")[0]
    if prefix.startswith("allocation") or prefix.startswith("allocated"):
```

```
        category = "allocation"
    elif prefix.startswith("segment") or prefix.startswith("reserved"):
        category = "segment_or_reserved"
    elif prefix.startswith("active"):
        category = "active"
    elif prefix.startswith("inactive"):
        category = "inactive"
    elif "retry" in key or "oom" in key:
        category = "retry_or_oom"
    else:
        category = "unclassified"
    unit = "bytes" if "bytes" in prefix else "events_or_occurrences"
    return category, unit

def E01():
    device = torch.device("cuda:0")
    torch.empty(1, device=device)
    raw = torch.cuda.memory_stats(device)
    rows = []
    for key in sorted(raw.keys()):
        category, unit = classify_key(key)
        rows.append({
            "key": key,
            "value": int(raw[key]),
            "category": category,
            "unit": unit,
        })
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Manually audit every unclassified key and refine rules without hiding it. Verify that sorting produces identical order across runs. Save PyTorch version beside the table. The classifier's unit inference is intentionally conservative; review documentation for ambiguous metrics before using them in arithmetic.

Part 7 - Engineering Interpretation and Philosophy

Schema discovery is part of systems engineering. Code that assumes a permanent dictionary silently breaks when backends evolve. A trustworthy tool preserves unknown evidence and makes its classification logic inspectable.

19.5.3 Exercise 02

Exercise Title: Extract Current, Peak, Allocated, and Freed Byte Counters from memory_stats() and Verify Them with Controlled Tensor Allocations

Part 1 - Exercise Contract and Why It Matters

Learn the difference between current state, historical peak, cumulative bytes allocated, and cumulative bytes freed. These four forms answer different questions and must not be substituted for one another.

Part 2 - System and Memory Mental Model

Current bytes describe the measurement boundary. Peak is the maximum since initialization or the last relevant reset. Allocated and freed forms are cumulative traffic. A workload can allocate and free one GiB one hundred times, producing enormous cumulative traffic while current and peak remain bounded.

Part 3 - Quantitative Prediction Before Execution

Predict counters across three 64 MiB allocate/delete cycles. Current should oscillate, peak should rise once and remain, and cumulative acquisition/release should advance every cycle. Test the approximate identity current equals cumulative allocated minus cumulative freed for the same family.

Part 4 - Experimental Design and Instrumentation

Warm CUDA, reset peak statistics once, and record after every allocation and deletion. Use deltas from baseline because context initialization precedes the experiment. Synchronize each boundary and avoid `empty_cache()`.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

KEYS = [
    "allocated_bytes.all.current",
    "allocated_bytes.all.peak",
    "allocated_bytes.all.allocated",
    "allocated_bytes.all.freed",
]

def read_bytes(label, device):
    torch.cuda.synchronize(device)
    stats = torch.cuda.memory_stats(device)
    row = {"label": label}
    for key in KEYS:
        row[key] = stats.get(key, None)
    return row

def E02():
    device = torch.device("cuda:0")
    torch.empty(1, device=device)
    torch.cuda.reset_peak_memory_stats(device)
    rows = [read_bytes("baseline", device)]
    count = 64 * 1024 * 1024 // 4
    for cycle in range(3):
        value = torch.empty(count, device=device)
        rows.append(read_bytes("allocate_" + str(cycle), device))
        del value
        gc.collect()
```

```
        rows.append(read_bytes("delete_" + str(cycle), device))
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Subtract baseline cumulative values before comparing traffic. Check the identity only when all four keys exist and document allocator rounding. Peak must never fall during the timeline. If it does, search for an unintended reset or process boundary rather than assuming a physical memory release changed history.

Part 7 - Engineering Interpretation and Philosophy

Residency and traffic are different performance dimensions. Current and peak budget capacity; cumulative counters expose churn. Optimization can reduce one without reducing the other, so reports should retain both.

19.5.4 Exercise 03

Exercise Title: Track allocation.all.current Across Repeated Tensor Creation and Deletion and Explain Every Counter Transition

Part 1 - Exercise Contract and Why It Matters

Interpret allocation count rather than byte capacity. Track how many allocator allocations are current while tensors, views, clones, and sliced copies appear and disappear.

Part 2 - System and Memory Mental Model

One tensor does not always imply one allocation, and one allocation can back multiple views. Operations may create temporary storage, and allocator rounding is independent of logical tensor count. The counter is evidence about allocator events, not a Python object census.

Part 3 - Quantitative Prediction Before Execution

Predict transitions for a base tensor, two views, one clone, and one advanced- indexed copy. Views should share storage; clone and indexed copy should allocate. Predict what happens when names are deleted in different orders.

Part 4 - Experimental Design and Instrumentation

Use a small tensor but synchronize boundaries. Record current allocation count, allocated bytes, and a manually maintained logical-object count. The mismatch between these columns is the phenomenon to explain, not an error to erase.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def row(label, device, logical_objects):
    torch.cuda.synchronize(device)
    stats = torch.cuda.memory_stats(device)
    return {
```

```
        "label": label,
        "logical_objects": logical_objects,
        "allocation_current": stats.get("allocation.all.current"),
        "allocated_bytes": stats.get("allocated_bytes.all.current"),
    }

def E03():
    device = torch.device("cuda:0")
    rows = []
    base = torch.arange(1024 * 1024, device=device)
    rows.append(row("base", device, 1))
    first_view = base.view(1024, 1024)
    second_view = base[100:200]
    rows.append(row("two_views", device, 3))
    clone = first_view.clone()
    rows.append(row("clone", device, 4))
    indices = torch.tensor([1, 3, 5, 7], device=device)
    indexed = base[indices]
    rows.append(row("indexed_copy", device, 6))
    del base, first_view, second_view, clone, indices, indexed
    gc.collect()
    rows.append(row("delete_all", device, 0))
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Explain every delta using storage creation and aliasing. The index tensor is itself a CUDA allocation and must be counted. If temporary allocations vanish before synchronization, the current counter may miss them; use cumulative and peak forms to reveal transient traffic. Repeat deletion one object at a time to locate the final alias holding storage.

Part 7 - Engineering Interpretation and Philosophy

Logical objects, storage objects, and allocator blocks occupy different layers. Correct memory reasoning keeps these layers distinct instead of expecting a one-to-one mapping that PyTorch never promised.

19.5.5 Exercise 04

Exercise Title: Compare allocated_bytes, reserved_bytes, and active_bytes and Prove Why They Represent Different Memory States

Part 1 - Exercise Contract and Why It Matters

Build a controlled proof that allocated, reserved, and active bytes are related but not interchangeable. The experiment must connect each counter to storage lifetime and allocator ownership.

Part 2 - System and Memory Mental Model

Allocated bytes approximate live tensor storage. Reserved bytes represent segments controlled by the allocator. Active bytes represent blocks not currently available for ordinary reuse

and can differ from allocated bytes due to allocator state. A useful expectation is allocated no greater than active, and active no greater than reserved, but verify backend-specific behavior before turning this expectation into a hard universal assertion.

Part 3 - Quantitative Prediction Before Execution

Allocate two unequal tensors, delete one, reallocate a compatible tensor, then delete all. Predict which counter follows live payload, which remains near its high-water mark, and which exposes block activity. Include cached proxy and inactive split bytes as supporting evidence.

Part 4 - Experimental Design and Instrumentation

Capture identical keys at synchronized boundaries and compare with payload computed from `numel()` and `element_size()`. Run both fixed and slightly mismatched replacement sizes. Keep missing keys visible.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def memory_states(label, device):
    torch.cuda.synchronize(device)
    stats = torch.cuda.memory_stats(device)
    return {
        "label": label,
        "allocated": stats.get("allocated_bytes.all.current"),
        "active": stats.get("active_bytes.all.current"),
        "reserved": stats.get("reserved_bytes.all.current"),
        "inactive_split": stats.get(
            "inactive_split_bytes.all.current"
        ),
    }

def make_mib(size_mib, device):
    count = size_mib * 1024 * 1024 // 4
    return torch.empty(count, device=device)

def E04():
    device = torch.device("cuda:0")
    rows = [memory_states("baseline", device)]
    first = make_mib(96, device)
    second = make_mib(160, device)
    rows.append(memory_states("two_live", device))
    del first
    gc.collect()
    rows.append(memory_states("release_first", device))
    replacement = make_mib(96, device)
    rows.append(memory_states("replacement", device))
    del second, replacement
    gc.collect()
```

```
    rows.append(memory_states("release_all", device))
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Compute pairwise gaps and explain them by state, never by name alone. Confirm allocated against `memory_allocated()` and reserved against `memory_reserved()`. If active is unavailable or identical throughout, record backend limits. A one-run equality does not prove semantic equivalence; the counters must diverge under a designed lifetime transition.

Part 7 - Engineering Interpretation and Philosophy

Memory counters answer ownership questions. Allocated asks what tensor storage is live, reserved asks what capacity PyTorch controls, and active asks what blocks are unavailable for reuse. Diagnosis begins by asking the correct ownership question before choosing a number.

19.5.6 Exercise 05

Exercise Title: Measure Small-Pool and Large-Pool Allocations Separately and Discover Which Tensor Sizes Trigger Each Allocator Path

Part 1 - Exercise Contract and Why It Matters

Empirically map requests to allocator pool statistics. The goal is not to memorize a size threshold from one release; it is to build a reproducible sweep that reveals the behavior of your exact backend and configuration.

Part 2 - System and Memory Mental Model

The native caching allocator reports small- and large-pool families for many counters. Pools reduce interference between request scales, but requested payload, rounded block size, and segment policy are distinct. Alternative backends may not report these counters meaningfully, so capability detection is part of the experiment.

Part 3 - Quantitative Prediction Before Execution

Sweep logarithmic sizes from KiB to tens of MiB. Predict a region dominated by small-pool deltas and another by large-pool deltas. Define the transition as an observed change in selected counter deltas, not as the first size that merely sounds large.

Part 4 - Experimental Design and Instrumentation

Run each size in a fresh subprocess or restore an identical baseline because prior allocations mutate cumulative statistics and cache state. Measure allocation count and bytes for both pools immediately before and during a live request. Repeat around the apparent boundary with finer increments.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

POOL_KEYS = [
    "allocation.small_pool.allocated",
```

```
    "allocation.large_pool.allocated",
    "allocated_bytes.small_pool.allocated",
    "allocated_bytes.large_pool.allocated",
]

def probe_size(size_bytes, device):
    torch.cuda.synchronize(device)
    before = torch.cuda.memory_stats(device)
    count = (size_bytes + 3) // 4
    value = torch.empty(count, dtype=torch.float32, device=device)
    torch.cuda.synchronize(device)
    after = torch.cuda.memory_stats(device)
    del value
    gc.collect()
    result = {"requested_bytes": size_bytes}
    for key in POOL_KEYS:
        if key in before and key in after:
            result[key] = int(after[key] - before[key])
        else:
            result[key] = None
    return result

def E05():
    device = torch.device("cuda:0")
    sizes = [
        4 * 1024,
        64 * 1024,
        256 * 1024,
        512 * 1024,
        768 * 1024,
        1024 * 1024,
        2 * 1024 * 1024,
        8 * 1024 * 1024,
        32 * 1024 * 1024,
    ]
    rows = []
    for size_bytes in sizes:
        rows.append(probe_size(size_bytes, device))
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Plot payload against both pool deltas and refine around the change. The in-process skeleton is order dependent; the final boundary report must use fresh-process probes. If both pool families are absent or unchanged, document the allocator backend rather than manufacturing a threshold. Compare requested bytes with allocator-accounted bytes to expose rounding.

Part 7 - Engineering Interpretation and Philosophy

Pool thresholds are implementation facts, not mathematical constants. The durable skill is designing a capability-aware measurement that remains honest when PyTorch changes its allocator.

19.5.7 Exercise 06

Exercise Title: Track Segment Creation and Release While Growing a CUDA Workload from Small Tensors to Multi-Gigabyte Allocations

Part 1 - Exercise Contract and Why It Matters

Observe how allocator segments evolve as the live working set grows from small requests to GiB-scale storage. The experiment must remain safe on an 8 GiB GPU and stop before external memory pressure becomes dangerous.

Part 2 - System and Memory Mental Model

Segments are regions obtained from CUDA and managed internally as blocks. Segment count and reserved bytes can rise in steps rather than exactly matching payload. Deleting tensors frees blocks; segments may remain cached until they become eligible for release or `empty_cache()` is called.

Part 3 - Quantitative Prediction Before Execution

Predict segment and reserved growth for cumulative live payloads of 16, 64, 256, 512, 1024, and 2048 MiB. Compute safety against current free memory and limit total experimental live payload to at most 35 percent of it. Predict the post-deletion state both before and after a deliberate cache release.

Part 4 - Experimental Design and Instrumentation

Use `torch.empty()` to focus on storage. Synchronize each step, record segment current/allocated/freed plus reserved bytes, and retain tensors in an explicit list. Run in a disposable process. Never catch an OOM and continue as though the allocator state were a clean baseline.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def segment_row(label, device):
    torch.cuda.synchronize(device)
    stats = torch.cuda.memory_stats(device)
    keys = [
        "segment.all.current",
        "segment.all.allocated",
        "segment.all.freed",
        "reserved_bytes.all.current",
        "allocated_bytes.all.current",
    ]
    row = {"label": label}
    for key in keys:
```

```
        row[key] = stats.get(key, None)
    return row

def E06():
    device = torch.device("cuda:0")
    free_bytes, _ = torch.cuda.mem_get_info(device)
    budget = int(free_bytes * 0.35)
    sizes_mib = [16, 64, 256, 512, 1024, 2048]
    live = []
    rows = [segment_row("baseline", device)]
    cumulative = 0
    for size_mib in sizes_mib:
        request = size_mib * 1024 * 1024
        if cumulative + request > budget:
            break
        value = torch.empty(request // 4, device=device)
        live.append(value)
        cumulative += request
        rows.append(segment_row("add_" + str(size_mib), device))
    live.clear()
    gc.collect()
    rows.append(segment_row("delete_all", device))
    torch.cuda.empty_cache()
    rows.append(segment_row("release_cache", device))
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Graph live payload, allocated, reserved, and segment count together. Confirm that skipped GiB requests are reported. After deletion, allocated should fall; after cache release, eligible segments may be returned. If segments do not fall, check live references and backend semantics. Do not infer segment size by dividing reserved by count when segments are heterogeneous.

Part 7 - Engineering Interpretation and Philosophy

Capacity grows through allocator units, not perfectly continuous payload bytes. Safe experiments respect the device as a shared finite system and produce useful evidence without proving courage through avoidable OOMs.

19.5.8 Exercise 07

Exercise Title: Measure Active and Inactive Split Blocks and Relate Them to Real Memory Reuse Behavior

Part 1 - Exercise Contract and Why It Matters

Connect split-block statistics to an actual reuse sequence. You will create a large request, release it, serve smaller requests, and observe how active and inactive split bytes change.

Part 2 - System and Memory Mental Model

When a larger block is split to satisfy a smaller request, one portion can become active while a remainder becomes inactive. Inactive split bytes indicate capacity inside split segments that is not currently active. They are a fragmentation signal, but not proof that the bytes are permanently unusable.

Part 3 - Quantitative Prediction Before Execution

Seed and release a 256 MiB allocation, then request 96 and 80 MiB concurrently. Predict active growth, possible inactive remainder, and stable reserved bytes. After releasing both, predict whether inactive split capacity can coalesce or remain represented depending on allocator state.

Part 4 - Experimental Design and Instrumentation

Record active blocks/bytes, inactive split blocks/bytes, allocated bytes, and reserved bytes. Use a same-size reallocation control and an unequal-size split candidate. Execute branches in fresh processes because each mutates the cache.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def split_row(label, device):
    torch.cuda.synchronize(device)
    stats = torch.cuda.memory_stats(device)
    keys = [
        "active.all.current",
        "active_bytes.all.current",
        "inactive_split.all.current",
        "inactive_split_bytes.all.current",
        "allocated_bytes.all.current",
        "reserved_bytes.all.current",
    ]
    row = {"label": label}
    for key in keys:
        row[key] = stats.get(key, None)
    return row

def make_mib(size_mib, device):
    return torch.empty(size_mib * 1024 * 1024 // 4, device=device)

def E07():
    device = torch.device("cuda:0")
    rows = [split_row("baseline", device)]
    seed = make_mib(256, device)
    rows.append(split_row("seed_live", device))
    del seed
    gc.collect()
    rows.append(split_row("seed_released", device))
    first = make_mib(96, device)
```

```
    second = make_mib(80, device)
    rows.append(split_row("two_smaller_live", device))
    del first, second
    gc.collect()
    rows.append(split_row("release_smaller", device))
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not declare a split solely because requested sizes sum below the seed. The allocator may choose another segment or block. Require a matching rise in inactive split statistics and stable reserved capacity. If counters remain zero, report that the designed pattern did not trigger the intended state and adjust sizes systematically.

Part 7 - Engineering Interpretation and Philosophy

Fragmentation evidence is strongest when a counter transition is tied to a controlled lifetime event. Names such as "inactive split" suggest a mechanism, but experiments must establish that the mechanism occurred in this workload.

19.5.9 Exercise 08

Exercise Title: Design Variable-Shape Allocations That Increase inactive_split_bytes and Quantify the Resulting Memory Waste

Part 1 - Exercise Contract and Why It Matters

Create a variable-shape schedule intended to grow inactive split bytes, then quantify the gap between live payload and allocator-reserved capacity. The exercise must allow the hypothesis to fail honestly.

Part 2 - System and Memory Mental Model

Alternating request sizes can split cached blocks into remainders that poorly match future requests. Define observed waste carefully: inactive split bytes are a direct allocator statistic when available, while reserved minus active is a broader slack measure containing more than fragmentation.

Part 3 - Quantitative Prediction Before Execution

Design a deterministic sequence around several close but unequal MiB sizes. Predict inactive split growth and define $W_{split} = I_{split}/R$ when $R > 0$. Also track useful live payload so a larger ratio is not merely caused by measuring at an idle boundary.

Part 4 - Experimental Design and Instrumentation

Use overlapping lifetimes, because allocating and immediately deleting one tensor may permit easy coalescing. Compare at identical peak-live and idle boundaries. Keep total live payload below 35 percent of free VRAM and repeat in fresh processes.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def E08(iterations=40):
    device = torch.device("cuda:0")
    patterns = [73, 119, 191, 83, 137, 211]
    live = []
    rows = []
    for iteration in range(iterations):
        size_mib = patterns[iteration % len(patterns)]
        value = torch.empty(size_mib * 1024 * 1024 // 4, device=device)
        live.append(value)
        if len(live) > 3:
            del live[0]
            gc.collect()
        torch.cuda.synchronize(device)
        stats = torch.cuda.memory_stats(device)
        reserved = stats.get("reserved_bytes.all.current", None)
        inactive = stats.get("inactive_split_bytes.all.current", None)
        ratio = None
        if reserved is not None and reserved > 0 and inactive is not None:
            ratio = inactive / reserved
        rows.append({
            "iteration": iteration,
            "size_mib": size_mib,
            "allocated": stats.get("allocated_bytes.all.current"),
            "reserved": reserved,
            "inactive_split": inactive,
            "split_waste_ratio": ratio,
        })
    live.clear()
    gc.collect()
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Plot absolute inactive split bytes and the ratio beside allocated/reserved curves. A ratio measured only after deletion can exaggerate application impact; report peak-live boundaries too. If inactive split stays zero, inspect backend support and revise the schedule rather than replacing observed values with the broader reserved-minus-allocated gap.

Part 7 - Engineering Interpretation and Philosophy

"Waste" is meaningful only relative to a workload objective. Cached slack may serve the next request efficiently, while inactive split capacity may obstruct a large contiguous request. Metrics must be interpreted through future demand.

19.5.10 Exercise 09

Exercise Title: Compare a Stable-Shape Workload Against a Variable-Shape Workload Using Identical Compute but Different Allocator Statistics

Part 1 - Exercise Contract and Why It Matters

Compare shape policies while controlling useful computation. Demonstrate that equal arithmetic intent can generate different allocator states because tensor sizes and lifetime patterns differ.

Part 2 - System and Memory Mental Model

Stable shapes repeat a small set of blocks and often reach steady state. Variable shapes diversify allocation requests. Equal total elements alone may not equalize kernel efficiency, so report both requested compute and measured runtime rather than claiming perfect hardware equivalence.

Part 3 - Quantitative Prediction Before Execution

Construct stable and variable schedules with equal total elements per four-step cycle. Predict similar cumulative payload traffic but more reserved growth, segments, or inactive split bytes for variable shapes. Predeclare comparison metrics and tolerances.

Part 4 - Experimental Design and Instrumentation

Run policies in separate fresh processes with one fixed weight matrix and no autograd. Record stats after every step and synchronize timing. Verify equal total input elements and checksum semantics. Do not call cache release.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def run_shapes(shapes):
    device = torch.device("cuda:0")
    rows = []
    for step, shape in enumerate(shapes):
        value = torch.randn(shape, device=device)
        output = torch.relu(value)
        checksum = output.sum()
        torch.cuda.synchronize(device)
        stats = torch.cuda.memory_stats(device)
        rows.append({
            "step": step,
            "elements": value.numel(),
            "allocated": stats.get("allocated_bytes.all.current"),
            "reserved": stats.get("reserved_bytes.all.current"),
            "segments": stats.get("segment.all.current"),
            "inactive_split": stats.get(
                "inactive_split_bytes.all.current"
            ),
            "checksum": float(checksum.cpu()),
        })
        del value, output, checksum
        gc.collect()
```

```
    return rows

def E09():
    stable = [(32, 1024, 256)] * 40
    variable_cycle = [
        (16, 1024, 256),
        (48, 1024, 256),
        (24, 1024, 256),
        (40, 1024, 256),
    ]
    variable = variable_cycle * 10
    return run_shapes(stable), run_shapes(variable)
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify both schedules process identical total elements. Compare end allocated, peak reserved, reserved-growth events, segment activity, and inactive splits. The random checksum values need not match; their purpose is to force compute. Use a deterministic input construction if numerical parity is part of the claim. Repeat process-level trials and report variation.

Part 7 - Engineering Interpretation and Philosophy

Fair allocator experiments control semantic work while acknowledging that shape itself affects kernels. The conclusion should describe the full systems trade, not pretend memory can be isolated from computation perfectly.

19.5.11 Exercise 10

Exercise Title: Trigger Allocator Memory Retries Under High VRAM Pressure and Identify the Conditions That Increase num_alloc_retries

Part 1 - Exercise Contract and Why It Matters

Investigate allocator retries as a pressure signal without assuming every high-memory workload will trigger one. The experiment is successful if it either produces a controlled retry or establishes the tested boundary and explains why no retry occurred.

Part 2 - System and Memory Mental Model

An allocation retry indicates that the allocator had to recover and attempt an allocation again, commonly by releasing eligible cached capacity. It is a cumulative occurrence counter, not a byte count and not automatically an OOM. A retry can precede successful allocation; zero retries does not prove abundant memory if the next request fails directly or the backend reports differently.

Part 3 - Quantitative Prediction Before Execution

Read current free capacity, reserve a bounded working set, release several irregular blocks, and request a large block. Predict conditions under which the cache contains enough total slack but no suitable block, potentially provoking recovery. Set a strict upper target below driver free memory and run in a disposable subprocess.

Part 4 - Experimental Design and Instrumentation

Record retries, OOMs, allocated, reserved, active, and inactive split bytes before pressure, after cache seeding, before the probe, and after it. Increase pressure in small steps across separate processes. Stop immediately after the first retry or OOM; do not continue from an ambiguous allocator state.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def pressure_row(label, device):
    torch.cuda.synchronize(device)
    stats = torch.cuda.memory_stats(device)
    return {
        "label": label,
        "retries": stats.get("num_alloc_retries"),
        "ooms": stats.get("num_ooms"),
        "allocated": stats.get("allocated_bytes.all.current"),
        "reserved": stats.get("reserved_bytes.all.current"),
        "active": stats.get("active_bytes.all.current"),
        "inactive_split": stats.get(
            "inactive_split_bytes.all.current"
        ),
    }

def E10(pressure_fraction=0.55):
    device = torch.device("cuda:0")
    free_bytes, _ = torch.cuda.mem_get_info(device)
    if pressure_fraction <= 0.0 or pressure_fraction >= 0.70:
        raise ValueError("Use a pressure fraction below 0.70.")
    budget = int(free_bytes * pressure_fraction)
    rows = [pressure_row("baseline", device)]
    blocks = []
    block_bytes = 64 * 1024 * 1024
    while (len(blocks) + 1) * block_bytes < budget:
        blocks.append(torch.empty(block_bytes // 4, device=device))
    rows.append(pressure_row("working_set", device))
    index = len(blocks) - 1
    while index >= 0:
        if index % 2 == 0:
            del blocks[index]
        index -= 1
    gc.collect()
    rows.append(pressure_row("irregular_slack", device))
    probe_bytes = min(int(free_bytes * 0.15), 512 * 1024 * 1024)
    probe = torch.empty(probe_bytes // 4, device=device)
    rows.append(pressure_row("probe_success", device))
    del probe
    blocks.clear()
```

```
    gc.collect()
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Compute retry deltas rather than reading only absolute history. If no retry appears, report maximum pressure, slack pattern, and successful probe size. Increase pressure only in fresh subprocesses. If an OOM occurs first, the experiment found a failure boundary but did not prove retry behavior. Never raise the safety fraction automatically after failure.

Part 7 - Engineering Interpretation and Philosophy

Retries are leading indicators of allocator stress, not moral failures. A defensible operational threshold comes from correlation with latency and OOM risk under representative traffic, not from demanding that the counter always remain mathematically zero.

19.5.12 Exercise 11

Exercise Title: Trigger a Controlled CUDA OOM and Inspect num_ooms, Reserved Bytes, Active Bytes, and Inactive Blocks Immediately After Failure

Part 1 - Exercise Contract and Why It Matters

Produce one deliberate, isolated OOM to learn what evidence remains after failure. This is a diagnostic laboratory exercise, not code to embed in a training process. It must run in a disposable subprocess on an otherwise idle GPU.

Part 2 - System and Memory Mental Model

An OOM is a failed request, not necessarily exhaustion of every reported byte. Live allocations, reserved segments, fragmentation, external processes, and request contiguity all affect the outcome. `num_ooms` records allocator events, while the exception message provides request context. Catching the exception does not guarantee every later CUDA operation is a scientifically clean continuation.

Part 3 - Quantitative Prediction Before Execution

Measure driver-free bytes and request slightly more than that in one tensor. Predict an OOM delta of at least one where supported, unchanged intended live working set, and possible retry activity. Predeclare that only statistics and the error text will be collected after failure; no training continues.

Part 4 - Experimental Design and Instrumentation

Run the script alone in a child process. Capture stats immediately before and inside the exception handler. Keep no valuable GPU state in the process. After recording, terminate normally and start a fresh process for all later tests.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def selected(device):
    stats = torch.cuda.memory_stats(device)
```

```
    keys = [
        "num_ooms",
        "num_alloc_retries",
        "allocated_bytes.all.current",
        "reserved_bytes.all.current",
        "active_bytes.all.current",
        "inactive_split_bytes.all.current",
    ]
    row = {}
    for key in keys:
        row[key] = stats.get(key, None)
    return row

def E11():
    device = torch.device("cuda:0")
    torch.empty(1, device=device)
    torch.cuda.synchronize(device)
    before = selected(device)
    free_bytes, _ = torch.cuda.mem_get_info(device)
    request_bytes = free_bytes + 256 * 1024 * 1024
    error_text = None
    after = None
    try:
        impossible = torch.empty(request_bytes // 4, device=device)
        del impossible
    except torch.OutOfMemoryError as error:
        error_text = str(error)
        after = selected(device)
    if error_text is None:
        raise RuntimeError("The controlled OOM did not occur.")
    return {
        "requested_bytes": request_bytes,
        "before": before,
        "after": after,
        "error": error_text,
    }
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify the OOM delta and compare active/reserved/inactive states. The requested size intentionally exceeds free capacity, so this test diagnoses basic failure accounting more than fragmentation. Keep the exception text but redact unrelated paths if sharing the report. If the exception type differs by version, document it and broaden the handler only enough to capture CUDA OOM, not every runtime error.

Part 7 - Engineering Interpretation and Philosophy

Failure experiments deserve stricter isolation than success benchmarks. Their value comes from learning observable signals and recovery boundaries without turning a damaged process state into misleading subsequent measurements.

19.5.13 Exercise 12

Exercise Title: Reset Peak CUDA Memory Statistics Between Experiments and Build Reproducible Memory Benchmarks

Part 1 - Exercise Contract and Why It Matters

Build a benchmark protocol in which each trial has an intentional peak-history boundary. Without correct resets, later trials inherit earlier high-water marks and cannot be compared.

Part 2 - System and Memory Mental Model

Peak statistics are historical state inside the process. Resetting peaks does not free memory, delete tensors, or reset cumulative allocation traffic. It changes the reference point for maximum values. A clean benchmark therefore needs stable live baseline memory before the reset and identical synchronization boundaries.

Part 3 - Quantitative Prediction Before Execution

Predict peaks for 64, 128, and 192 MiB trials executed in ascending and descending order. Without reset, every later reported peak is at least the largest previous one. With reset at a clean baseline, peaks should track each trial independently.

Part 4 - Experimental Design and Instrumentation

Warm CUDA, delete prior trial tensors, synchronize, assert baseline allocated within tolerance, reset peak memory stats, run one trial, synchronize, and read peak. Repeat orders and compare. Record current memory beside peak to prove the reset itself did not release storage.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def run_trial(size_mib, device):
    gc.collect()
    torch.cuda.synchronize(device)
    baseline = torch.cuda.memory_allocated(device)
    torch.cuda.reset_peak_memory_stats(device)
    value = torch.empty(size_mib * 1024 * 1024 // 4, device=device)
    torch.cuda.synchronize(device)
    current = torch.cuda.memory_allocated(device)
    peak = torch.cuda.max_memory_allocated(device)
    del value
    gc.collect()
    torch.cuda.synchronize(device)
    return {
        "size_mib": size_mib,
        "baseline": baseline,
        "current": current,
        "peak": peak,
        "end": torch.cuda.memory_allocated(device),
    }
```

```
def E12():
    device = torch.device("cuda:0")
    ascending = []
    for size_mib in [64, 128, 192]:
        ascending.append(run_trial(size_mib, device))
    descending = []
    for size_mib in [192, 128, 64]:
        descending.append(run_trial(size_mib, device))
    return ascending, descending
```

Part 6 - Measurement, Validation, and Failure Analysis

Subtract each trial baseline from peak and compare with payload. Assert that end allocated returns to baseline before the next reset. If not, repair lifetime control. Reserved cache may differ by order while allocated peak deltas remain comparable; report both because reproducible semantics do not require identical cache history.

Part 7 - Engineering Interpretation and Philosophy

Reproducibility requires defining which state is reset and which state is allowed to persist. A peak reset is statistical bookkeeping, not process purification. Honest benchmarks state both facts.

19.5.14 Exercise 13

Exercise Title: Build a Memory Efficiency Metric from Active, Allocated, Reserved, and Inactive Bytes and Validate It on Multiple Workloads

Part 1 - Exercise Contract and Why It Matters

Design a small metric family rather than one magical score. Separate payload utilization, active-block utilization, and split-fragment pressure so a poor result can be interpreted causally.

Part 2 - System and Memory Mental Model

For positive reserved bytes define

$$U_A = A/R, \qquad U_{active} = H/R, \qquad F_{split} = I/R,$$

where A is allocated, H active, R reserved, and I inactive split bytes. These ratios describe a measurement boundary. Low utilization at idle is expected; comparisons must use equivalent workload phases.

Part 3 - Quantitative Prediction Before Execution

Predict ratios for a live fixed-shape peak, an idle cached state, and a variable-shape peak. Define valid ranges and missing-key behavior. Do not combine ratios into a weighted score until weights are justified by deployment goals.

Part 4 - Experimental Design and Instrumentation

Capture each workload at identical semantic boundaries and preserve raw bytes beside ratios. Run repeated fixed, bursty, and variable schedules in fresh processes. Include OOM/retry counts so apparent efficiency is not rewarded when the workload fails.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def efficiency_row(label, device):
    torch.cuda.synchronize(device)
    stats = torch.cuda.memory_stats(device)
    allocated = stats.get("allocated_bytes.all.current")
    active = stats.get("active_bytes.all.current")
    reserved = stats.get("reserved_bytes.all.current")
    inactive = stats.get("inactive_split_bytes.all.current")
    row = {
        "label": label,
        "allocated": allocated,
        "active": active,
        "reserved": reserved,
        "inactive_split": inactive,
    }
    if reserved is None or reserved == 0:
        row["allocated_utilization"] = None
        row["active_utilization"] = None
        row["split_pressure"] = None
    else:
        row["allocated_utilization"] = allocated / reserved
        row["active_utilization"] = active / reserved
        row["split_pressure"] = inactive / reserved
    return row

def E13():
    device = torch.device("cuda:0")
    baseline = efficiency_row("baseline", device)
    value = torch.empty((64, 1024, 1024), device=device)
    live = efficiency_row("live_peak", device)
    del value
    idle = efficiency_row("idle_cache", device)
    return baseline, live, idle
```

Part 6 - Measurement, Validation, and Failure Analysis

Assert ratios lie between zero and one only when backend semantics support the expected ordering. A low idle U_A is not inefficiency by itself; the cache may accelerate the next iteration. Validate metrics against known stable and variable controls and report raw counters whenever a ratio changes.

Part 7 - Engineering Interpretation and Philosophy

Metrics compress evidence and therefore discard context. A responsible metric design keeps the raw quantities, phase label, and failure counters close enough that the compression can be challenged.

19.5.15 Exercise 14

Exercise Title: Detect Allocator Fragmentation Using Only memory_stats() Counters Before Inspecting Any Memory Snapshot

Part 1 - Exercise Contract and Why It Matters

Build a first-pass fragmentation detector using summary counters only. Its output must be a suspicion level with evidence, never a definitive block-layout claim that requires a memory snapshot.

Part 2 - System and Memory Mental Model

Fragmentation suspicion rises when inactive split bytes are large relative to reserved memory, reserved greatly exceeds active/allocated memory at a live boundary, retries increase, or a large request fails despite substantial aggregate slack. None is individually conclusive. Cached idle memory and intentional workspace can produce similar gaps.

Part 3 - Quantitative Prediction Before Execution

Define transparent rules: split-pressure warning, reserved-slack warning, retry escalation, and OOM escalation. Choose thresholds from controlled stable and variable workloads, not arbitrary universal constants. Predict detector output for each control before implementation.

Part 4 - Experimental Design and Instrumentation

Feed synchronized rows from E07–E10 into the detector. Require at least two independent signals for high suspicion unless an OOM follows a large-slack state. Preserve missing keys and return reasons with every classification.

Part 5 - PyTorch/CUDA Implementation

```
def fragmentation_signals(row, split_limit=0.20, slack_limit=0.40):
    reserved = row.get("reserved")
    allocated = row.get("allocated")
    inactive = row.get("inactive_split")
    reasons = []
    if reserved is None or reserved <= 0:
        return {"level": "unknown", "reasons": ["missing_reserved"]}
    if inactive is not None and inactive / reserved >= split_limit:
        reasons.append("high_inactive_split_ratio")
    if allocated is not None:
        slack = (reserved - allocated) / reserved
        if slack >= slack_limit:
            reasons.append("large_reserved_slack")
    if row.get("retry_delta", 0) > 0:
        reasons.append("allocation_retry")
    if row.get("oom_delta", 0) > 0:
        reasons.append("oom_event")
    if "oom_event" in reasons and len(reasons) >= 2:
        level = "high"
    elif len(reasons) >= 2:
        level = "moderate"
```

```
    elif len(reasons) == 1:
        level = "watch"
    else:
        level = "low"
    return {"level": level, "reasons": reasons}

def E14(rows):
    results = []
    for row in rows:
        results.append(fragmentation_signals(row))
    return results
```

Part 6 - Measurement, Validation, and Failure Analysis

Measure false positives on idle-cache and burst controls and false negatives on the variable split-heavy control. Thresholds tuned on one trace require holdout validation. The detector should recommend snapshot inspection for moderate/high suspicion; it must not claim block addresses or contiguity from summary counters.

Part 7 - Engineering Interpretation and Philosophy

Good anomaly detection narrows investigation. It does not replace stronger evidence. Returning reasons and uncertainty makes the detector useful to an engineer instead of turning thresholds into an oracle.

19.5.16 Exercise 15

Exercise Title: Record memory_stats() at Forward, Loss, Backward, Optimizer, and Zero-Grad Boundaries and Explain the Training Memory Lifecycle

Part 1 - Exercise Contract and Why It Matters

Instrument a complete training iteration at semantic boundaries. The objective is to connect counter transitions to parameters, activations, loss, gradients, optimizer state, and gradient clearing rather than presenting one unexplained peak.

Part 2 - System and Memory Mental Model

Model parameters and inputs exist before forward. Forward creates outputs and saved activations. Loss adds output-dependent state. Backward consumes saved state while creating gradients and temporary workspaces. The first optimizer step may lazily allocate optimizer state. With `zero_grad(set_to_none=True)`, gradient storage can be released for reuse; setting gradients to zero may retain buffers.

Part 3 - Quantitative Prediction Before Execution

Compute parameter payload and predict the direction of allocated, active, reserved, allocation count, and peaks at every boundary. Distinguish the first iteration from steady state because optimizer state and library initialization are lazy. Predict two zero-grad strategies separately.

Part 4 - Experimental Design and Instrumentation

Use fixed seeds and one fixed batch. Capture synchronized rows before forward, after forward, after loss, after backward, after optimizer step, and after zero-grad. Warm only CUDA and model kernels for one report; retain the cold first optimizer step in a separate report so its state allocation is visible.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def stage_row(stage, device):
    torch.cuda.synchronize(device)
    stats = torch.cuda.memory_stats(device)
    keys = [
        "allocation.all.current",
        "allocated_bytes.all.current",
        "allocated_bytes.all.peak",
        "active_bytes.all.current",
        "reserved_bytes.all.current",
        "inactive_split_bytes.all.current",
    ]
    row = {"stage": stage}
    for key in keys:
        row[key] = stats.get(key, None)
    return row

def E15():
    device = torch.device("cuda:0")
    torch.manual_seed(71)
    model = torch.nn.Sequential(
        torch.nn.Linear(2048, 4096),
        torch.nn.GELU(),
        torch.nn.Linear(4096, 1000),
    ).to(device)
    optimizer = torch.optim.AdamW(model.parameters(), lr=0.0003)
    inputs = torch.randn((128, 2048), device=device)
    targets = torch.randint(0, 1000, (128,), device=device)
    torch.cuda.reset_peak_memory_stats(device)
    rows = [stage_row("before_forward", device)]
    logits = model(inputs)
    rows.append(stage_row("after_forward", device))
    loss = torch.nn.functional.cross_entropy(logits, targets)
    rows.append(stage_row("after_loss", device))
    loss.backward()
    rows.append(stage_row("after_backward", device))
    optimizer.step()
    rows.append(stage_row("after_optimizer", device))
    optimizer.zero_grad(set_to_none=True)
    rows.append(stage_row("after_zero_grad", device))
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Annotate every positive and negative delta with expected tensor lifetime. Inspect peak separately because temporaries may appear and disappear between boundaries. Repeat the second iteration to isolate one-time optimizer state. If zero-grad does not reduce allocated bytes, verify strategy, gradient references, and whether another object retains the graph. Confirm loss is finite so a memory trace is not documenting a numerically failed step.

Part 7 - Engineering Interpretation and Philosophy

Training memory is a lifecycle, not a static model size. Stage boundaries turn an OOM into a precise question: which semantic object became live, which workspace peaked, and which intended lifetime failed to end?

19.5.17 Exercise 16

Exercise Title: Compare SGD and AdamW Training Memory Behavior Through Allocator Statistics and Identify Optimizer-State Allocation Patterns

Part 1 - Exercise Contract and Why It Matters

Compare optimizer memory under identical model, data, precision, and step boundaries. Reveal when state is allocated and quantify steady-state cost rather than relying only on theoretical parameter multiples.

Part 2 - System and Memory Mental Model

Plain SGD without momentum may maintain little persistent per-parameter state. Momentum adds a buffer. AdamW typically maintains first- and second-moment states and a step value. Implementation choices, foreach/fused paths, precision, and temporary workspaces affect observed allocation beyond the conceptual state tensors.

Part 3 - Quantitative Prediction Before Execution

Calculate model parameter bytes and predict persistent optimizer-state payload for SGD, SGD with momentum, and AdamW. Predict the first step to allocate state lazily and later steps to reuse it. Define observed state cost as steady allocated memory after optimizer initialization minus an identical model/input baseline.

Part 4 - Experimental Design and Instrumentation

Run optimizers in separate fresh processes from the same saved initial model state. Use identical gradients by fixing inputs and seeds. Capture before backward, after backward, after first step, after zero-grad, and after second step. Record optimizer configuration including foreach/fused defaults.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def run_optimizer(kind):
    device = torch.device("cuda:0")
    torch.manual_seed(83)
    model = torch.nn.Linear(4096, 4096).to(device)
```

```
    if kind == "sgd":
        optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
    elif kind == "momentum":
        optimizer = torch.optim.SGD(
            model.parameters(), lr=0.01, momentum=0.9
        )
    elif kind == "adamw":
        optimizer = torch.optim.AdamW(model.parameters(), lr=0.001)
    else:
        raise ValueError(kind)
    inputs = torch.randn((64, 4096), device=device)
    targets = torch.randn((64, 4096), device=device)
    rows = []
    for step in range(2):
        optimizer.zero_grad(set_to_none=True)
        output = model(inputs)
        loss = torch.nn.functional.mse_loss(output, targets)
        loss.backward()
        optimizer.step()
        optimizer.zero_grad(set_to_none=True)
        torch.cuda.synchronize(device)
        stats = torch.cuda.memory_stats(device)
        rows.append({
            "kind": kind,
            "step": step,
            "allocated": stats.get("allocated_bytes.all.current"),
            "reserved": stats.get("reserved_bytes.all.current"),
            "allocation_count": stats.get("allocation.all.current"),
            "peak": stats.get("allocated_bytes.all.peak"),
        })
        del output, loss
    return rows

def E16():
    sgd_rows = run_optimizer("sgd")
    momentum_rows = run_optimizer("momentum")
    adamw_rows = run_optimizer("adamw")
    return sgd_rows, momentum_rows, adamw_rows
```

Part 6 - Measurement, Validation, and Failure Analysis

The single-process skeleton contaminates later baselines with prior cache state; the final benchmark must run each optimizer in a fresh process. Enumerate optimizer state tensors and compute their bytes as a semantic cross-check. Confirm parameter initialization and loss are identical before comparing. Separate persistent post-zero-grad allocation from transient peak.

Part 7 - Engineering Interpretation and Philosophy

Optimizer choice is a memory architecture decision as well as a convergence decision. Theoretical state size predicts direction; allocator statistics show when and how that cost materializes

on the actual system.

19.5.18 Exercise 17

Exercise Title: Build a Time-Series Logger That Captures Selected CUDA Allocator Statistics Across Hundreds of Training Iterations

Part 1 - Exercise Contract and Why It Matters

Build a low-retention logger that records selected allocator statistics across hundreds of iterations without keeping CUDA tensors or autograd graphs. Long traces reveal drift, periodic bursts, lazy initialization, and steady-state plateaus invisible in one snapshot.

Part 2 - System and Memory Mental Model

Logging frequency trades temporal resolution against synchronization overhead. Every synchronization changes execution timing. The logger should store only CPU-native scalars and phase labels, sample frequently during warm-up, then less frequently during steady state. Cumulative counters need deltas; current counters can be plotted directly.

Part 3 - Quantitative Prediction Before Execution

Predict first-step optimizer growth, early cache stabilization, and stable end-of-step allocated memory. Define slope and jump metrics before training. Estimate row count and storage so logging itself stays bounded.

Part 4 - Experimental Design and Instrumentation

Create an explicit key allowlist. Convert each value to Python `int` and loss to detached CPU `float`. Never append output, loss tensor, or raw stats objects containing unexpected GPU references. Record environment and sampling policy once.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

class MemoryStatsLogger:
    def __init__(self, device, every=10):
        self.device = device
        self.every = every
        self.rows = []
        self.keys = [
            "allocated_bytes.all.current",
            "allocated_bytes.all.peak",
            "reserved_bytes.all.current",
            "active_bytes.all.current",
            "inactive_split_bytes.all.current",
            "num_alloc_retries",
            "num_ooms",
        ]

    def capture(self, step, loss_value):
```

```
        if step >= 10 and step % self.every != 0:
            return
        torch.cuda.synchronize(self.device)
        stats = torch.cuda.memory_stats(self.device)
        row = {
            "step": step,
            "time": time.time(),
            "loss": float(loss_value),
        }
        for key in self.keys:
            value = stats.get(key, None)
            row[key] = None if value is None else int(value)
        self.rows.append(row)

def E17(model, optimizer, batches, device):
    logger = MemoryStatsLogger(device, every=10)
    for step, batch in enumerate(batches):
        inputs, targets = batch
        inputs = inputs.to(device, non_blocking=True)
        targets = targets.to(device, non_blocking=True)
        optimizer.zero_grad(set_to_none=True)
        output = model(inputs)
        loss = torch.nn.functional.cross_entropy(output, targets)
        loss.backward()
        optimizer.step()
        logger.capture(step, loss.detach().cpu().item())
        del inputs, targets, output, loss
    return logger.rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify row count against sampling policy and assert no tensor objects occur in rows. Benchmark with logging disabled and enabled to quantify overhead. Plot current counters, cumulative deltas, and loss on separate axes. A rising allocated slope needs semantic reference investigation; a reserved staircase with flat allocated suggests changing allocation demand instead.

Part 7 - Engineering Interpretation and Philosophy

Observability must not rewrite the system it observes. Selective scalar logging provides longitudinal evidence while keeping overhead and lifetime interference measurable and bounded.

19.5.19 Exercise 18

Exercise Title: Implement Automatic Memory-Anomaly Detection for Sudden Reserved-Memory Growth, Fragmentation, Retries, and OOM Events

Part 1 - Exercise Contract and Why It Matters

Convert the E17 time series into an explainable detector for four anomaly families: reserved jumps, split pressure, retry increments, and OOM increments. Every alert must include evidence and severity.

Part 2 - System and Memory Mental Model

Current counters require level or jump rules; cumulative counters require positive deltas. Baselines change during warm-up, so static thresholds generate false alerts. Use a warm-up window and robust distributions such as median and median absolute deviation, while retaining hard alerts for retry/OOM deltas.

Part 3 - Quantitative Prediction Before Execution

Inject synthetic rows containing one reserved jump, one inactive-split rise, one retry, and one OOM. Predict exact alerts before writing the detector. Define warning versus critical severity and cooldown behavior so one persistent state does not emit hundreds of identical events.

Part 4 - Experimental Design and Instrumentation

Separate detector training/baseline rows from evaluation rows. Preserve raw row indices and previous values. Test missing counters, zero reserved memory, counter reset, and process restart. Never treat a negative cumulative delta as normal workload behavior; classify it as reset/restart metadata.

Part 5 - PyTorch/CUDA Implementation

```
def detect_anomalies(rows, reserved_jump_bytes, split_ratio_limit=0.20):
    alerts = []
    previous = None
    for row in rows:
        if previous is None:
            previous = row
            continue
        reserved = row.get("reserved_bytes.all.current")
        old_reserved = previous.get("reserved_bytes.all.current")
        inactive = row.get("inactive_split_bytes.all.current")
        retry = row.get("num_alloc_retries")
        old_retry = previous.get("num_alloc_retries")
        ooms = row.get("num_ooms")
        old_ooms = previous.get("num_ooms")

        if reserved is not None and old_reserved is not None:
            if reserved - old_reserved >= reserved_jump_bytes:
                alerts.append((row["step"], "warning", "reserved_jump"))
        if reserved is not None and reserved > 0 and inactive is not None:
            if inactive / reserved >= split_ratio_limit:
                alerts.append((row["step"], "warning", "split_pressure"))
        if retry is not None and old_retry is not None and retry > old_retry:
            alerts.append((row["step"], "warning", "allocation_retry"))
        if ooms is not None and old_ooms is not None and ooms > old_ooms:
            alerts.append((row["step"], "critical", "oom"))
        previous = row
    return alerts

def E18(rows):
    return detect_anomalies(rows, reserved_jump_bytes=128 * 1024 * 1024)
```

Part 6 - Measurement, Validation, and Failure Analysis

Measure precision and recall on labeled synthetic and controlled real traces. Tune thresholds on development traces and validate on holdout workloads. A split-pressure alert at idle may be harmless; correlate it with live phase, retries, latency, or failed requests. Add deduplication only after confirming it does not hide new OOM increments.

Part 7 - Engineering Interpretation and Philosophy

Automation should accelerate human diagnosis, not conceal uncertainty. An alert is a structured question with evidence. The response can then escalate from counters to snapshots, profiler traces, or workload inspection.

19.5.20 Exercise 19

Exercise Title: Diagnose an Unknown Memory-Inefficient PyTorch Workload Using Only memory_stats() Evidence and Produce a Defensible Root-Cause Analysis

Part 1 - Exercise Contract and Why It Matters

Perform a blind diagnosis. Receive only a timestamped `memory_stats()` trace and phase labels, classify the most likely root-cause family, identify competing explanations, and propose the next discriminating experiment. Do not inspect source code or memory snapshots during the initial analysis.

Part 2 - System and Memory Mental Model

Candidate families include retained live tensors, stable healthy cache, variable-shape cache growth, split fragmentation, optimizer lazy state, transient workspace peaks, and external memory pressure. Counter signatures overlap. Rising allocated current supports retained liveness; flat allocated with rising reserved supports allocation-shape change; inactive split and retries strengthen fragmentation suspicion; a one-time step-one jump suggests lazy initialization.

Part 3 - Quantitative Prediction Before Execution

Build a signature matrix mapping every candidate to expected slopes, jumps, retry/OOM deltas, split pressure, and phase timing. Assign confidence only after listing evidence for and against each candidate. Predeclare what observation would falsify the leading hypothesis.

Part 4 - Experimental Design and Instrumentation

Analyze raw counters, first differences, rolling slopes, and phase-aligned averages. Separate warm-up from steady state. Compare current versus cumulative counters and detect resets. Keep the dataset blind until the report is frozen; then reveal workload identity and measure diagnostic accuracy.

Part 5 - PyTorch/CUDA Implementation

```
def summarize_trace(rows):
    if len(rows) < 2:
        raise ValueError("At least two rows are required.")
    first = rows[0]
    last = rows[-1]
    keys = [
```

```
        "allocated_bytes.all.current",
        "reserved_bytes.all.current",
        "active_bytes.all.current",
        "inactive_split_bytes.all.current",
        "num_alloc_retries",
        "num_ooms",
    ]
    deltas = {}
    for key in keys:
        start = first.get(key)
        end = last.get(key)
        if start is None or end is None:
            deltas[key] = None
        else:
            deltas[key] = end - start
    return deltas

def initial_hypotheses(deltas):
    hypotheses = []
    allocated = deltas.get("allocated_bytes.all.current")
    reserved = deltas.get("reserved_bytes.all.current")
    inactive = deltas.get("inactive_split_bytes.all.current")
    if allocated is not None and allocated > 0:
        hypotheses.append("retained_live_storage")
    if reserved is not None and reserved > 0:
        if allocated is not None and allocated <= 0:
            hypotheses.append("cache_or_shape_growth")
    if inactive is not None and inactive > 0:
        hypotheses.append("split_fragmentation_suspected")
    return hypotheses

def E19(rows):
    deltas = summarize_trace(rows)
    return {
        "deltas": deltas,
        "initial_hypotheses": initial_hypotheses(deltas),
        "required_next_step": "phase_aligned_and_slope_analysis",
    }
```

Part 6 - Measurement, Validation, and Failure Analysis

Endpoint deltas are only triage; the final report must analyze the full curve. Test the method on hidden traces produced by known bugs and controls. Report confusion cases. A defensible conclusion includes confidence, alternatives, falsification test, and expected counter change after the proposed fix. After source reveal, apply the fix and reproduce the predicted counter response.

Part 7 - Engineering Interpretation and Philosophy

Root-cause analysis is an argument under incomplete evidence. Expertise is not pretending certainty; it is narrowing hypotheses, stating what would prove you wrong, and selecting the cheapest next measurement that separates competing causes. `memory_stats()` is powerful precisely when its limits remain visible.

19.6 DMM04

Chapter Title: PyTorch CUDA Memory Snapshots
Specialization: Dynamic Memory Management and CUDA Systems

19.6.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Segments, Blocks, Allocation Events, and Memory History in PyTorch CUDA Snapshots

Part 1 - Exercise Contract and Why It Matters

This exercise moves from aggregate counters to allocator topology and history. A memory snapshot can describe allocator-managed segments, the blocks inside them, block states, requested sizes, addresses, streams, source frames, and— when history recording is enabled— a sequence of allocator events. The purpose is not to memorize one serialized schema. The purpose is to reconstruct a defensible story of where capacity came from, how it was divided, which blocks remain active, and why aggregate free space may not satisfy one large request.

Your deliverable is a four-layer model. Layer one is CUDA driver capacity. Layer two is PyTorch segments reserved from that capacity. Layer three is the block layout inside each segment. Layer four is the time-ordered event history. For every claim, identify whether it came from current topology, recorded history, a source stack, or an inference. Snapshot formats and private history APIs can change between PyTorch versions, so the parser must preserve unknown fields and fail visibly when required fields are absent.

Part 2 - System and Memory Mental Model

Model segment s as an address interval $[a_s, a_s + T_s)$ with total size T_s. Its ordered blocks b_i should cover the allocator-described layout. Each block has an address, size, and state such as `active_allocated`, `active_awaiting_free`, or `inactive`. A simplified accounting identity is

$$T_s = \sum_i |b_i|,$$

subject to the exact schema. Requested size can be smaller than block size due to rounding and alignment. Allocated space is active tensor storage; awaiting- free space has been logically released but is not yet reusable because device work has not completed; inactive space is allocator-owned free capacity.

Topology answers "what exists now." History answers "how did it get here." Events can include allocation, segment allocation/free, requested/free-completed transitions, OOM, and snapshot markers depending on version and recording configuration. The public snapshot API commonly returns segment information. The private full-snapshot representation used for dumping can also contain device traces when history is enabled. Treat these interfaces separately and check capabilities at runtime.

Part 3 - Quantitative Prediction Before Execution

Before capturing anything, draw one 256 MiB segment split into a 96 MiB active block, an 80 MiB active block, and an 80 MiB inactive remainder. Calculate active ratio, inactive ratio, internal rounding waste, and largest contiguous inactive block. Explain why 80 MiB of total inactive capacity does not imply that a 96 MiB request can succeed. Then predict how allocation and release events transform the diagram over time.

Part 4 - Experimental Design and Instrumentation

Use a fresh process, explicit CUDA device, fixed seed, fixed streams, and synchronized snapshot boundaries. Capture a baseline, a live-allocation state, and a released-cache state. Record PyTorch/CUDA versions and allocator backend. Keep raw snapshots immutable; build derived tables separately. Never store live tensors inside analyzer results because instrumentation would extend lifetimes.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def summarize_segments(segments):
    summary = []
    for segment_index, segment in enumerate(segments):
        block_rows = []
        for block_index, block in enumerate(segment.get("blocks", [])):
            block_rows.append({
                "block_index": block_index,
                "address": block.get("address"),
                "size": block.get("size"),
                "requested_size": block.get("requested_size"),
                "state": block.get("state"),
            })
        summary.append({
            "segment_index": segment_index,
            "device": segment.get("device"),
            "address": segment.get("address"),
            "total_size": segment.get("total_size"),
            "allocated_size": segment.get("allocated_size"),
            "active_size": segment.get("active_size"),
            "stream": segment.get("stream"),
            "segment_type": segment.get("segment_type"),
            "blocks": block_rows,
        })
    return summary

def E00():
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required for DMM04.")
    device = torch.device("cuda:0")
```

```
    torch.empty(1, device=device)
    torch.cuda.synchronize(device)
    baseline = torch.cuda.memory_snapshot()
    first = torch.empty((24, 1024, 1024), device=device)
    second = torch.empty((16, 1024, 1024), device=device)
    torch.cuda.synchronize(device)
    live = torch.cuda.memory_snapshot()
    del first, second
    gc.collect()
    torch.cuda.synchronize(device)
    released = torch.cuda.memory_snapshot()
    return {
        "baseline": summarize_segments(baseline),
        "live": summarize_segments(live),
        "released": summarize_segments(released),
    }
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate that block sizes sum to segment total size when the schema promises full coverage. Check address ordering and non-overlap. Sum active, inactive, and awaiting-free states independently; compare derived segment totals with fields stored by PyTorch without silently forcing equality. Differences can reveal schema semantics, rounding, or parser errors.

If a field is missing, preserve `None` and record version/backend. If addresses differ across clean-process repetitions, do not use raw address as a cross-process identity. If the released snapshot retains segments, recognize healthy cache ownership rather than declaring a leak. A snapshot cannot account for memory allocated outside the PyTorch allocator.

Part 7 - Engineering Interpretation and Philosophy

A snapshot is a structured piece of evidence, not a picture that interprets itself. Topology explains contiguity and ownership; history explains causality; stacks connect events to source; device-level counters reveal what lies outside PyTorch. Deep diagnosis comes from joining these views while respecting the question each can answer.

19.6.2 Exercise 01

Exercise Title: Capture Your First CUDA Allocator Snapshot with torch.cuda.memory_snapshot() and Inspect Its Raw Structure

Part 1 - Exercise Contract and Why It Matters

Capture and inspect the public snapshot without assuming its shape in advance. Enumerate segment and block keys, value types, and state values. The output is a schema inventory for your installed PyTorch.

Part 2 - System and Memory Mental Model

The public API usually returns a list of segment dictionaries. Each segment contains metadata and a block list. Blocks represent an address layout, not a Python tensor list. Multiple

tensors can share storage; views may create no new block; temporary blocks may disappear before capture.

Part 3 - Quantitative Prediction Before Execution

Predict the minimum fields required to calculate total segment size, active space, inactive space, and largest block. Predict state labels but mark them as hypotheses until observed. Calculate expected payload of two known tensors.

Part 4 - Experimental Design and Instrumentation

Warm CUDA, allocate two tensors, synchronize, and capture exactly once. Print only bounded samples of nested data to avoid an unreadable console. Build a recursive type/schema walker that never stores tensor objects.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def describe(value, path, rows, depth=0):
    rows.append({
        "path": path,
        "type": type(value).__name__,
        "depth": depth,
    })
    if isinstance(value, dict):
        for key in sorted(value.keys()):
            describe(value[key], path + "." + str(key), rows, depth + 1)
    elif isinstance(value, list) and len(value) > 0:
        describe(value[0], path + "[0]", rows, depth + 1)

def E01():
    device = torch.device("cuda:0")
    first = torch.empty((8, 1024, 1024), device=device)
    second = torch.empty((4, 1024, 1024), device=device)
    torch.cuda.synchronize(device)
    snapshot = torch.cuda.memory_snapshot()
    schema_rows = []
    describe(snapshot, "snapshot", schema_rows)
    states = set()
    for segment in snapshot:
        for block in segment.get("blocks", []):
            states.add(block.get("state"))
    return schema_rows, sorted(str(state) for state in states)
```

Part 6 - Measurement, Validation, and Failure Analysis

Compare observed keys against the analyzer requirements, not against an unversioned internet example. Verify known payload appears within active block accounting. A one-snapshot schema walker samples the first list element, so audit later elements for optional keys before claiming completeness.

Part 7 - Engineering Interpretation and Philosophy

Schema inspection precedes parser construction. Preserving the raw structure and optional fields prevents a convenient script from becoming a source of false certainty.

19.6.3 Exercise 02

Exercise Title: Enable Allocation History with torch.cuda.memory._record_memory_history() and Explain What Additional Information Becomes Available

Part 1 - Exercise Contract and Why It Matters

Enable allocator history and compare history-disabled and history-enabled snapshots. Determine which frames and device-trace events become available and measure the additional diagnostic value.

Part 2 - System and Memory Mental Model

Topology records the current allocator layout. History records transitions and can attach Python/native stack context. Recording is not free: it consumes host memory, adds bookkeeping, and retains only a bounded number of entries when configured. The leading underscore marks a private/version-sensitive API, so code must detect availability and record the exact PyTorch version.

Part 3 - Quantitative Prediction Before Execution

Predict events for segment acquisition, tensor allocation, requested free, and completed free. Predict that source frames may appear for active blocks and trace actions. Define what history would prove that topology alone cannot: the source and timing of a block's creation or release.

Part 4 - Experimental Design and Instrumentation

Run in a fresh process, enable history before the target workload, use one short deterministic sequence, capture the private full snapshot, then disable recording in a `finally` block. Keep entry count bounded.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def E02():
    device = torch.device("cuda:0")
    recorder = getattr(torch.cuda.memory, "_record_memory_history", None)
    snapshot_function = getattr(torch.cuda.memory, "_snapshot", None)
    if recorder is None or snapshot_function is None:
        raise RuntimeError("This PyTorch build lacks memory-history APIs.")
    try:
        recorder(
            enabled="all",
            context="all",
            stacks="all",
```

```
            max_entries=10000,
            device=device,
        )
        first = torch.empty((16, 1024, 1024), device=device)
        second = first.clone()
        del first
        gc.collect()
        torch.cuda.synchronize(device)
        full_snapshot = snapshot_function()
        del second
        return full_snapshot
    finally:
        recorder(enabled=None)
```

Part 6 - Measurement, Validation, and Failure Analysis

Inspect top-level keys, device traces, event actions, frames, and segment blocks. If keyword arguments are rejected, inspect the installed signature and adapt explicitly; do not catch every exception and silently run without history. Verify entry bound and note whether native stacks require additional build support.

Part 7 - Engineering Interpretation and Philosophy

History converts "this block exists" into "this event created it here." The extra causal power carries overhead and version risk, so history should be enabled around focused investigations rather than left unbounded by habit.

19.6.4 Exercise 03

Exercise Title: Dump a CUDA Memory Snapshot to Disk with _dump_snapshot() and Reproduce the Same Workload from a Clean Process

Part 1 - Exercise Contract and Why It Matters

Create a portable snapshot artifact and reproduce its workload in a clean process. Reproducibility is defined by comparable topology and event patterns, not identical addresses or byte-for-byte pickle equality.

Part 2 - System and Memory Mental Model

`_dump_snapshot()` serializes the full private snapshot representation used by PyTorch memory tooling. It may contain segments, traces, and frames. Addresses can change across processes because allocation placement is not a stable identity. Dump files can contain source paths and should be handled as diagnostic data, not blindly published.

Part 3 - Quantitative Prediction Before Execution

Predict invariant fields across two clean runs: requested shapes, active byte totals at the capture boundary, event categories, and approximate segment layout. Predict non-invariants: raw addresses, timestamps, and initialization noise. Define comparison tolerances before running.

Part 4 - Experimental Design and Instrumentation

Use a script launched twice as separate processes with identical environment, seed, device, and workload. Enable bounded history, synchronize capture, dump to unique filenames, disable recording, and record file size plus cryptographic hash only for integrity, not semantic equality.

Part 5 - PyTorch/CUDA Implementation

```
from pathlib import Path
import torch

def E03(output_path):
    device = torch.device("cuda:0")
    output = Path(output_path)
    if output.exists():
        raise FileExistsError(str(output))
    torch.manual_seed(101)
    recorder = torch.cuda.memory._record_memory_history
    try:
        recorder(max_entries=20000)
        first = torch.randn((32, 1024, 512), device=device)
        second = torch.relu(first)
        result = second @ torch.randn((512, 256), device=device)
        torch.cuda.synchronize(device)
        torch.cuda.memory._dump_snapshot(str(output))
        checksum = float(result.sum().cpu())
        return {"path": str(output), "checksum": checksum}
    finally:
        recorder(enabled=None)
```

Part 6 - Measurement, Validation, and Failure Analysis

Load both artifacts only with trusted tooling because pickle-like formats can execute unsafe content when deserialized. Compare semantic summaries after normalizing addresses. Verify checksums and environment metadata. If topology differs, distinguish acceptable allocator nondeterminism from a workload or configuration mismatch.

Part 7 - Engineering Interpretation and Philosophy

Reproducibility means preserving the causal experiment and interpreting stable features correctly. Demanding identical addresses would mistake an incidental runtime detail for scientific identity.

19.6.5 Exercise 04

Exercise Title: Parse Snapshot Segments Programmatically and Compute Total Reserved, Active, and Free Space Without Using memory_stats()

Part 1 - Exercise Contract and Why It Matters

Build allocator accounting directly from segment/block topology. This provides an independent cross-check for aggregate counters and forces precise handling of block states.

Part 2 - System and Memory Mental Model

Reserved topology is the sum of segment total sizes. Active space can be derived from active block states or compared with segment active fields. Allocator-free space is the sum of inactive blocks. It is not CUDA-driver free memory and may not be one contiguous region.

Part 3 - Quantitative Prediction Before Execution

For each segment, predict totals for reserved, active-allocated, active-awaiting-free, inactive, requested payload, rounding gap, and largest inactive block. Define consistency equations and error tolerances.

Part 4 - Experimental Design and Instrumentation

Parse a synchronized snapshot, preserve unknown states in a separate bucket, and validate block coverage. Use Python integers. Never call `memory_stats()` inside the primary derivation; perform external cross-validation only after results are frozen.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def parse_snapshot(segments):
    totals = {
        "reserved": 0,
        "active_allocated": 0,
        "active_awaiting_free": 0,
        "inactive": 0,
        "requested": 0,
        "unknown": 0,
        "largest_inactive_block": 0,
    }
    for segment in segments:
        totals["reserved"] += int(segment.get("total_size", 0))
        block_sum = 0
        for block in segment.get("blocks", []):
            size = int(block.get("size", 0))
            requested = int(block.get("requested_size", 0))
            state = block.get("state")
            block_sum += size
            totals["requested"] += requested
            if state == "active_allocated":
                totals["active_allocated"] += size
            elif state == "active_awaiting_free":
                totals["active_awaiting_free"] += size
            elif state == "inactive":
                totals["inactive"] += size
                if size > totals["largest_inactive_block"]:
                    totals["largest_inactive_block"] = size
            else:
                totals["unknown"] += size
```

```
        if block_sum != int(segment.get("total_size", 0)):
            raise ValueError("Block coverage does not match segment size.")
    return totals

def E04():
    device = torch.device("cuda:0")
    value = torch.empty((32, 1024, 1024), device=device)
    torch.cuda.synchronize(device)
    return parse_snapshot(torch.cuda.memory_snapshot())
```

Part 6 - Measurement, Validation, and Failure Analysis

Check that categorized block bytes plus unknown bytes equal reserved bytes. Report allocator-free space and largest free block together. A parser that sees an unknown state must not classify it as inactive. After freezing results, compare reserved/active values with DMM03 counters and explain semantic gaps.

Part 7 - Engineering Interpretation and Philosophy

Independent derivation exposes hidden assumptions in convenience counters. Topology also reveals why total free bytes and allocatable contiguous bytes are different questions.

19.6.6 Exercise 05

Exercise Title: Reconstruct How One CUDA Segment Is Split into Multiple Allocation Blocks and Identify Reusable Free Regions

Part 1 - Exercise Contract and Why It Matters

Select one segment and reconstruct its ordered address map. Identify active, awaiting-free, inactive, and unknown regions, then determine which individual inactive regions can satisfy candidate requests.

Part 2 - System and Memory Mental Model

Free space is geometric. Two inactive 64 MiB blocks separated by an active block provide 128 MiB total slack but not one 128 MiB contiguous block. Allocator splitting, merging, alignment, and stream state determine whether a region is reusable. The snapshot exposes present topology but not a universal guarantee about the next allocation decision.

Part 3 - Quantitative Prediction Before Execution

For one selected segment, compute block start/end addresses, coverage, state totals, largest inactive region, and external fragmentation proxy $1 - L/F$, where F is total inactive space and L its largest block. Define the ratio as zero when $F = 0$.

Part 4 - Experimental Design and Instrumentation

Create overlapping unequal allocations, release alternating objects, and capture after synchronization. Select the segment containing the most blocks, not simply the first segment. Preserve raw order and separately sort by address to verify the source structure.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def reconstruct_segment(segment):
    blocks = sorted(
        segment.get("blocks", []),
        key=lambda block: int(block.get("address", 0)),
    )
    rows = []
    total_inactive = 0
    largest_inactive = 0
    for block in blocks:
        address = int(block.get("address", 0))
        size = int(block.get("size", 0))
        state = block.get("state")
        rows.append({
            "start": address,
            "end": address + size,
            "size": size,
            "state": state,
        })
        if state == "inactive":
            total_inactive += size
            largest_inactive = max(largest_inactive, size)
    fragmentation = 0.0
    if total_inactive > 0:
        fragmentation = 1.0 - largest_inactive / total_inactive
    return rows, total_inactive, largest_inactive, fragmentation

def E05():
    segments = torch.cuda.memory_snapshot()
    if len(segments) == 0:
        raise RuntimeError("No allocator segments are present.")
    selected = max(segments, key=lambda item: len(item.get("blocks", [])))
    return reconstruct_segment(selected)
```

Part 6 - Measurement, Validation, and Failure Analysis

Assert consecutive address ranges do not overlap and remain inside the segment. Test candidate sizes against largest inactive block, not total inactive bytes. Do not label every inactive block immediately reusable across streams. If the selected segment has one block, redesign the workload; topology cannot be studied when splitting did not occur.

Part 7 - Engineering Interpretation and Philosophy

Memory availability is shape and location, not only quantity. Segment maps transform an abstract “free MiB” number into the exact spatial constraint that determines whether a large request can fit.

19.6.7 Exercise 06

Exercise Title: Trace a CUDA Allocation Back to the Python Source Line That Created the Tensor Using Recorded Stack Frames

Part 1 - Exercise Contract and Why It Matters

Connect one active block to the Python call site that created it. The result must distinguish user frames from PyTorch internals and preserve the full stack for auditability.

Part 2 - System and Memory Mental Model

History recording can attach frames to allocation events or active blocks. Frames may contain filename, line number, and function name. The top frame is not always the most useful user frame; wrappers, module calls, and dispatcher layers can intervene. Stack availability depends on recording configuration and build support.

Part 3 - Quantitative Prediction Before Execution

Create allocation through a named helper and predict that its file and line appear in a recorded stack. Define a user-frame selector that excludes known library prefixes but returns the full stack when no user frame is found.

Part 4 - Experimental Design and Instrumentation

Enable context/stacks before calling the helper. Tag the allocation by its unique requested size, capture while it remains live, and locate candidate blocks. Use requested size as a filter, not a globally unique identity.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def allocation_site(device):
    return torch.empty((13, 1024, 1024), device=device)

def candidate_frames(snapshot, requested_bytes):
    candidates = []
    for segment in snapshot.get("segments", []):
        for block in segment.get("blocks", []):
            if block.get("state") != "active_allocated":
                continue
            if int(block.get("requested_size", -1)) != requested_bytes:
                continue
            candidates.append({
                "address": block.get("address"),
                "frames": block.get("frames", []),
            })
    return candidates

def E06():
    device = torch.device("cuda:0")
    recorder = torch.cuda.memory._record_memory_history
```

```
    try:
        recorder(context="all", stacks="all", max_entries=10000)
        value = allocation_site(device)
        torch.cuda.synchronize(device)
        snapshot = torch.cuda.memory._snapshot()
        requested = value.numel() * value.element_size()
        return candidate_frames(snapshot, requested)
    finally:
        recorder(enabled=None)
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify at least one candidate includes the helper filename and line. If several blocks share requested size, use event order or a second distinguishing allocation. Missing frames require checking recorder options and build support; do not invent a call site from current Python variables.

Part 7 - Engineering Interpretation and Philosophy

Source attribution turns allocator evidence into an actionable repair location. It is strongest when the raw stack remains available and the frame-selection heuristic can be challenged.

19.6.8 Exercise 07

Exercise Title: Follow One Tensor Through Allocation, free_requested, and free_completed Events and Explain Why Those Events Can Occur at Different Times

Part 1 - Exercise Contract and Why It Matters

Trace one allocation through logical release and physical reuse eligibility. Explain why Python deletion can request a free before CUDA completes work that uses the storage.

Part 2 - System and Memory Mental Model

CUDA execution is asynchronous. When the final Python reference disappears, the allocator may record `free_requested`; the block can remain `active_awaiting_free` until stream work finishes. A later `free_completed` event marks completion. Immediate synchronization can collapse the interval and hide the phenomenon.

Part 3 - Quantitative Prediction Before Execution

Launch a long operation using a tensor, delete its Python reference without host synchronization, and capture history. Predict allocation, free request, then completion after explicit synchronization. Identify the allocation by address/request size and sequence order.

Part 4 - Experimental Design and Instrumentation

Use a dedicated CUDA stream and events rather than an uncontrolled sleep. Record history before allocation, capture once before stream synchronization and again after. Do not access the deleted tensor. Repeat with default-stream synchronization as a control.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def E07():
    device = torch.device("cuda:0")
    stream = torch.cuda.Stream(device=device)
    recorder = torch.cuda.memory._record_memory_history
    try:
        recorder(context="all", max_entries=20000)
        with torch.cuda.stream(stream):
            value = torch.randn((4096, 4096), device=device)
            result = value @ value
        del value
        gc.collect()
        before_completion = torch.cuda.memory._snapshot()
        stream.synchronize()
        after_completion = torch.cuda.memory._snapshot()
        del result
        return before_completion, after_completion
    finally:
        recorder(enabled=None)
```

Part 6 - Measurement, Validation, and Failure Analysis

Search device traces for event actions and correlate addresses. If no awaiting- free interval appears, the operation may have completed too quickly or allocator stream semantics may differ. Increase controlled work, not arbitrary host delay. Ensure snapshot capture itself does not synchronize in a way that erases the window; document observed behavior.

Part 7 - Engineering Interpretation and Philosophy

Object lifetime and device safety are separate clocks. Understanding their boundary prevents false leak diagnoses and unsafe assumptions about when a deleted allocation becomes reusable.

19.6.9 Exercise 08

Exercise Title: Generate Multiple Snapshots During Forward, Backward, Optimizer Step, and Gradient Reset and Reconstruct the Training Memory Lifecycle

Part 1 - Exercise Contract and Why It Matters

Capture topology at every training boundary and reconstruct which blocks appear, persist, or disappear. This extends DMM03 counters into segment-level evidence.

Part 2 - System and Memory Mental Model

Parameters persist across all stages. Forward activations survive for backward. Backward creates gradients and temporaries. The first AdamW step creates persistent state. Gradient reset with `set_to_none=True` can release gradient storage into cache. Snapshots reveal each state as block topology.

Part 3 - Quantitative Prediction Before Execution

Predict active requested bytes, active block count, inactive bytes, and segment count at before-forward, after-forward, after-loss, after-backward, after-step, and after-zero-grad. Separate first and second iterations.

Part 4 - Experimental Design and Instrumentation

Synchronize every capture and store raw CPU snapshot structures with stage names. Do not include them in the autograd graph. Use one fixed batch and reset peak statistics only before the full timeline, not between stages.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def capture(stage, device):
    torch.cuda.synchronize(device)
    return {"stage": stage, "segments": torch.cuda.memory_snapshot()}

def E08():
    device = torch.device("cuda:0")
    model = torch.nn.Sequential(
        torch.nn.Linear(2048, 4096),
        torch.nn.GELU(),
        torch.nn.Linear(4096, 1000),
    ).to(device)
    optimizer = torch.optim.AdamW(model.parameters(), lr=0.0003)
    inputs = torch.randn((128, 2048), device=device)
    targets = torch.randint(0, 1000, (128,), device=device)
    snapshots = [capture("before_forward", device)]
    output = model(inputs)
    snapshots.append(capture("after_forward", device))
    loss = torch.nn.functional.cross_entropy(output, targets)
    snapshots.append(capture("after_loss", device))
    loss.backward()
    snapshots.append(capture("after_backward", device))
    optimizer.step()
    snapshots.append(capture("after_optimizer", device))
    optimizer.zero_grad(set_to_none=True)
    snapshots.append(capture("after_zero_grad", device))
    return snapshots
```

Part 6 - Measurement, Validation, and Failure Analysis

Summarize every snapshot using E04 and diff adjacent stages by block address only within the same process. Confirm first-step optimizer state persists. Delete `output` and `loss` in a second run to test whether user references affect after-zero-grad topology. Numerical validation must accompany memory analysis.

Part 7 - Engineering Interpretation and Philosophy

Snapshots convert a training step into a sequence of ownership states. The question is no longer merely where the peak is, but which semantic stage and specific block lifetime produced it.

19.6.10 Exercise 09

Exercise Title: Create an Irregular Allocation Pattern and Locate Inactive Space Trapped Inside Partially Used CUDA Segments

Part 1 - Exercise Contract and Why It Matters

Create a bounded irregular pattern and identify inactive blocks inside segments that still contain active blocks. These segments cannot be returned wholesale to CUDA while any contained active block remains.

Part 2 - System and Memory Mental Model

A completely inactive segment may be eligible for cache release. Inactive blocks inside a partially active segment are trapped at segment granularity: the allocator may reuse them, but cannot return the entire segment while active neighbors remain. This is distinct from a live tensor leak.

Part 3 - Quantitative Prediction Before Execution

Allocate alternating unequal tensors and retain every other one. Predict partially active segments, trapped inactive bytes, largest trapped block, and trapped ratio. Keep total live payload below 30 percent of free VRAM.

Part 4 - Experimental Design and Instrumentation

Capture before and after releasing alternating objects. Classify a segment as partially active when it contains at least one active state and one inactive block. Compare with a stable-shape control in a fresh process.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def trapped_space(segments):
    rows = []
    for index, segment in enumerate(segments):
        active = 0
        inactive = 0
        largest_inactive = 0
        for block in segment.get("blocks", []):
            size = int(block.get("size", 0))
            state = block.get("state")
            if state == "inactive":
                inactive += size
                largest_inactive = max(largest_inactive, size)
            elif str(state).startswith("active"):
                active += size
```

```
        if active > 0 and inactive > 0:
            rows.append({
                "segment_index": index,
                "active": active,
                "trapped_inactive": inactive,
                "largest_inactive": largest_inactive,
                "total_size": segment.get("total_size"),
            })
    return rows

def E09():
    device = torch.device("cuda:0")
    sizes_mib = [73, 119, 83, 137, 97, 151, 107, 173]
    tensors = []
    for size_mib in sizes_mib:
        count = size_mib * 1024 * 1024 // 4
        tensors.append(torch.empty(count, device=device))
    index = len(tensors) - 1
    while index >= 0:
        if index % 2 == 1:
            del tensors[index]
        index -= 1
    gc.collect()
    torch.cuda.synchronize(device)
    return trapped_space(torch.cuda.memory_snapshot())
```

Part 6 - Measurement, Validation, and Failure Analysis

Sum trapped space without double-counting segments. Verify each reported segment has both state classes. If the allocator placed every allocation in a separate segment, the pattern failed to create trapped space; change sizes and overlap rather than relabeling completely inactive segments.

Part 7 - Engineering Interpretation and Philosophy

Segment-level returnability and block-level reusability are different. A useful diagnosis states which layer is constrained and whether future requests can use the trapped regions.

19.6.11 Exercise 10

Exercise Title: Visualize a Recorded Snapshot with PyTorch Memory Visualizer and Correlate Timeline Peaks with Source-Level Allocations

Part 1 - Exercise Contract and Why It Matters

Load a trusted snapshot into the official PyTorch memory visualization workflow and convert visual peaks into source-level explanations. A screenshot alone is not the deliverable; every highlighted region must be correlated with event, size, address, phase, and stack evidence.

Part 2 - System and Memory Mental Model

The visualizer presents allocator state history and segment maps from the dumped snapshot. The active-memory timeline emphasizes allocations over time, while allocator-state history emphasizes segment/block layout at selected events. PyTorch's profiler sees only memory managed by the PyTorch allocator, so device usage outside it is absent. The official documentation currently directs users to the PyTorch memory visualizer for saved snapshots.

Part 3 - Quantitative Prediction Before Execution

Before loading the file, predict the largest three allocation events from tensor shapes and dtypes. Calculate their payloads and expected training phase. Define how you will verify each visual peak from raw trace entries.

Part 4 - Experimental Design and Instrumentation

Generate one bounded history dump with named workload phases and source stacks. Treat snapshot files as sensitive because paths and code locations may appear. Use only the official visualizer or a trusted local analyzer; never upload confidential traces to an untrusted site.

Part 5 - PyTorch/CUDA Implementation

```
from pathlib import Path
import torch

def E10(output_path):
    path = Path(output_path)
    if path.exists():
        raise FileExistsError(str(path))
    device = torch.device("cuda:0")
    recorder = torch.cuda.memory._record_memory_history
    try:
        recorder(context="all", stacks="all", max_entries=50000)
        first = torch.randn(
            (4096, 4096),
            device=device,
            requires_grad=True,
        )
        second = torch.relu(first)
        third = first @ second
        loss = third.square().mean()
        loss.backward()
        torch.cuda.synchronize(device)
        torch.cuda.memory._dump_snapshot(str(path))
        return str(path)
    finally:
        recorder(enabled=None)
```

Part 6 - Measurement, Validation, and Failure Analysis

Select each major visual peak, read its stack, and match its requested size to the precomputed payload. Explain gaps caused by block rounding or temporary workspace. If a device-level memory peak lacks a visualizer event, investigate non-PyTorch allocation rather

than assuming the file is corrupted. Preserve a table of visual claim, raw event, and source frame.

Part 7 - Engineering Interpretation and Philosophy

Visualization accelerates pattern recognition, but raw evidence must remain the authority. A persuasive memory analysis can move from a picture to the exact event and code line that created the pressure.

19.6.12 Exercise 11

Exercise Title: Use Allocator State History to Explain Why a Large Allocation Can Fail Even When Significant Reserved Memory Still Exists

Part 1 - Exercise Contract and Why It Matters

Construct a topology in which aggregate inactive space is substantial but the largest contiguous inactive block is smaller than a proposed request. Use history to explain how the layout formed.

Part 2 - System and Memory Mental Model

Reserved memory includes active and inactive blocks across segments. A large request needs a suitable block or new driver capacity. Many small inactive regions cannot be combined across active barriers or independent segments. Therefore reserved-minus-allocated is not an allocatable-block guarantee.

Part 3 - Quantitative Prediction Before Execution

At the pre-request snapshot compute total inactive F, largest inactive block L, driver free D, and proposed request Q. Fragmentation suspicion is strongest when $F \geq Q$, $L < Q$, and $D < Q$. Predict retry or OOM events but treat them as outcomes to observe.

Part 4 - Experimental Design and Instrumentation

Build pressure gradually in a disposable subprocess, never exceeding a predeclared safety schedule until the final controlled probe. Capture topology immediately before the probe and history immediately after success/failure. Run a same-total-free but fixed-shape control.

Part 5 - PyTorch/CUDA Implementation

```
def largest_inactive(segments):
    total = 0
    largest = 0
    for segment in segments:
        for block in segment.get("blocks", []):
            if block.get("state") == "inactive":
                size = int(block.get("size", 0))
                total += size
                largest = max(largest, size)
    return total, largest

def explain_fit(snapshot, request_bytes, driver_free_bytes):
    segments = snapshot.get("segments", snapshot)
```

```
    total, largest = largest_inactive(segments)
    return {
        "request": request_bytes,
        "total_inactive": total,
        "largest_inactive": largest,
        "driver_free": driver_free_bytes,
        "aggregate_slack_enough": total >= request_bytes,
        "single_cached_block_fits": largest >= request_bytes,
        "driver_can_extend": driver_free_bytes >= request_bytes,
    }

def E11(snapshot, request_bytes, driver_free_bytes):
    return explain_fit(snapshot, request_bytes, driver_free_bytes)
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not claim causality unless the pre-request topology satisfies the predicted conditions and the trace records the failed/retried request. Consider stream- awaiting blocks and external allocations. A successful request despite the simple fit test reveals allocator recovery or segment acquisition and must be explained from subsequent events.

Part 7 - Engineering Interpretation and Philosophy

Capacity planning needs the size distribution of free regions, not just their sum. Snapshot topology supplies the missing geometric evidence behind many apparently contradictory OOM messages.

19.6.13 Exercise 12

Exercise Title: Trigger a Controlled CUDA OOM and Reconstruct the Allocator State Immediately Before the Failed Allocation

Part 1 - Exercise Contract and Why It Matters

Capture a pre-failure snapshot, trigger exactly one deliberate OOM, and join the recorded OOM event with the topology that existed immediately before the request. Run only in a disposable process on an idle GPU.

Part 2 - System and Memory Mental Model

The snapshot taken before the probe records available blocks and live state. History records the failed request. The exception describes requested capacity and allocator/device summary. None alone proves root cause; together they show what the allocator had, what was requested, and how the request ended.

Part 3 - Quantitative Prediction Before Execution

Request slightly more than current driver-free memory. Predict an OOM history event and at least one OOM counter increment. Calculate pre-failure largest free block, total inactive space, active bytes, and driver free. This basic test is capacity-driven; a separate fragmented pattern is required to study fragmentation-driven failure.

Part 4 - Experimental Design and Instrumentation

Enable bounded history, synchronize, capture pre-state, then attempt the probe. Inside the OOM handler capture the post-failure private snapshot and exception text. Do not continue computation after evidence collection; terminate the subprocess and use a fresh process for later experiments.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def E12():
    device = torch.device("cuda:0")
    recorder = torch.cuda.memory._record_memory_history
    try:
        recorder(context="all", stacks="all", max_entries=50000)
        baseline = torch.empty((16, 1024, 1024), device=device)
        torch.cuda.synchronize(device)
        before = torch.cuda.memory._snapshot()
        free_bytes, _ = torch.cuda.mem_get_info(device)
        request_bytes = free_bytes + 256 * 1024 * 1024
        try:
            impossible = torch.empty(request_bytes // 4, device=device)
            del impossible
        except torch.OutOfMemoryError as error:
            after = torch.cuda.memory._snapshot()
            return {
                "request_bytes": request_bytes,
                "before": before,
                "after": after,
                "error": str(error),
            }
        raise RuntimeError("The controlled OOM did not occur.")
    finally:
        recorder(enabled=None)
```

Part 6 - Measurement, Validation, and Failure Analysis

Locate the OOM action in device traces and match its requested size. Compare pre/post segment topology and confirm intended baseline remains live. If the allocation unexpectedly succeeds, the free-memory reading changed or another mechanism supplied capacity; discard the run instead of forcing a second huge allocation in the same process.

Part 7 - Engineering Interpretation and Philosophy

Controlled failure analysis freezes the last healthy state and the failed request into one causal record. Isolation protects both the GPU session and the validity of all subsequent experiments.

19.6.14 Exercise 13

Exercise Title: Compare Snapshots Before and After torch.cuda.empty_cache() and Identify Exactly Which Segments Can Be Returned to CUDA

Part 1 - Exercise Contract and Why It Matters

Determine which segments disappear after cache release and explain why partially active segments remain. Avoid the incorrect claim that all inactive blocks are individually returned to CUDA.

Part 2 - System and Memory Mental Model

`empty_cache()` can release eligible unused cached segments. A segment containing any active or awaiting-free block generally cannot be returned as a whole. Inactive blocks inside it may remain available to PyTorch even after the call.

Part 3 - Quantitative Prediction Before Execution

Classify each pre-call segment as fully inactive, partially active, or fully active. Predict fully inactive segment addresses to disappear and active-bearing segments to persist. Calculate returned bytes from vanished segment totals.

Part 4 - Experimental Design and Instrumentation

Create both completely released and deliberately retained tensors. Synchronize before the first snapshot, call cache release, synchronize again, and capture the second. Compare within one process so addresses are meaningful identities.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import torch

def segment_map(segments):
    result = {}
    for segment in segments:
        address = int(segment.get("address", 0))
        states = []
        for block in segment.get("blocks", []):
            states.append(block.get("state"))
        result[address] = {
            "total_size": int(segment.get("total_size", 0)),
            "states": states,
        }
    return result

def E13():
    device = torch.device("cuda:0")
    retained = torch.empty((32, 1024, 1024), device=device)
    released = torch.empty((64, 1024, 1024), device=device)
    del released
    gc.collect()
    torch.cuda.synchronize(device)
    before = segment_map(torch.cuda.memory_snapshot())
    torch.cuda.empty_cache()
    torch.cuda.synchronize(device)
```

```
    after = segment_map(torch.cuda.memory_snapshot())
    vanished = []
    for address in before:
        if address not in after:
            vanished.append((address, before[address]))
    return before, after, vanished
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify every vanished segment was fully inactive in the pre-snapshot. If a mixed-state segment vanishes, inspect state semantics and synchronization. Report returned bytes and driver-free change; external activity may disturb the latter. If no segment vanishes, the released block may share a segment with the retained allocation.

Part 7 - Engineering Interpretation and Philosophy

Cache release operates at allocator ownership boundaries. Snapshot diffs show why "inactive bytes exist" and "CUDA can reclaim these bytes now" are not equivalent statements.

19.6.15 Exercise 14

Exercise Title: Detect a Persistent Tensor Reference by Comparing Two Memory Snapshots Taken Hundreds of Training Steps Apart

Part 1 - Exercise Contract and Why It Matters

Detect allocations that persist unexpectedly across hundreds of steps and trace them to source. Separate legitimate long-lived parameters/optimizer state from accidentally retained outputs or losses.

Part 2 - System and Memory Mental Model

Within one process, a block address that remains active in both snapshots is a persistence candidate. Persistence alone is not a leak: model parameters should persist. Source frames, requested size, creation time, and growth of candidate count distinguish intended state from a container that accumulates tensors.

Part 3 - Quantitative Prediction Before Execution

Predict stable parameter/state candidates and one deliberately retained loss per step. Compare snapshots at steps 10 and 300. Predict candidate count and requested-byte growth, then define allowlists by source/function rather than raw address.

Part 4 - Experimental Design and Instrumentation

Enable history before model creation so parameter stacks are available. Capture at equivalent post-zero-grad boundaries. Store only snapshot data. Run correct and leaking versions from fresh processes and blind the analyzer to labels.

Part 5 - PyTorch/CUDA Implementation

```
def active_allocations(snapshot):
    result = {}
    segments = snapshot.get("segments", snapshot)
    for segment in segments:
        for block in segment.get("blocks", []):
            if block.get("state") != "active_allocated":
                continue
            address = int(block.get("address", 0))
            result[address] = {
                "size": int(block.get("size", 0)),
                "requested_size": int(block.get("requested_size", 0)),
                "frames": block.get("frames", []),
            }
    return result

def E14(early_snapshot, late_snapshot):
    early = active_allocations(early_snapshot)
    late = active_allocations(late_snapshot)
    persistent = {}
    new_late = {}
    for address, block in late.items():
        if address in early:
            persistent[address] = block
        else:
            new_late[address] = block
    return {"persistent": persistent, "new_late": new_late}
```

Part 6 - Measurement, Validation, and Failure Analysis

Rank new late allocations by source frame and requested bytes. Reused addresses can represent different lifetimes, so consult event history before declaring identity. Apply the suspected reference fix, repeat from a clean process, and require the unexpected source group to disappear while training outputs remain correct.

Part 7 - Engineering Interpretation and Philosophy

Leak detection is intent-aware lifetime analysis. The strongest proof joins cross-time persistence, creation source, accumulation pattern, and disappearance after a semantics-preserving fix.

19.6.16 Exercise 15

Exercise Title: Distinguish PyTorch-Managed CUDA Memory from Non-PyTorch GPU Allocations by Comparing Snapshot Data with Device-Level Memory Usage

Part 1 - Exercise Contract and Why It Matters

Estimate the gap between memory represented by PyTorch allocator segments and device-level memory used by the process/device. Demonstrate why a complete snapshot can still omit NCCL, CUDA libraries, custom extensions, or other direct CUDA allocations.

Part 2 - System and Memory Mental Model

Snapshot segment totals describe PyTorch caching-allocator ownership. Driver- level usage includes CUDA context and allocations made outside that allocator. System-wide device usage can also include other processes. Therefore

$$M_{device} - M_{snapshot}$$

is an unexplained gap, not automatically "non-PyTorch tensor memory." Scope and baseline must be defined before interpreting it.

Part 3 - Quantitative Prediction Before Execution

Predict baseline context gap, then allocate a known PyTorch tensor and expect both snapshot-reserved and device-used values to rise. Add a controlled external allocator only if a trusted library is available and predict device usage to rise without matching snapshot growth.

Part 4 - Experimental Design and Instrumentation

Use an idle GPU or process-scoped NVML measurement. Capture synchronized snapshot totals and `mem_get_info()` at identical boundaries. Record other processes and library initialization. Repeat in a fresh process because context cost is a baseline state.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def snapshot_reserved(device):
    torch.cuda.synchronize(device)
    total = 0
    for segment in torch.cuda.memory_snapshot():
        if int(segment.get("device", device.index)) == device.index:
            total += int(segment.get("total_size", 0))
    return total

def device_used(device):
    free_bytes, total_bytes = torch.cuda.mem_get_info(device)
    return total_bytes - free_bytes

def measurement(label, device):
    return {
        "label": label,
        "snapshot_reserved": snapshot_reserved(device),
        "device_used": device_used(device),
        "unexplained_gap": device_used(device) - snapshot_reserved(device),
    }

def E15():
    device = torch.device("cuda:0")
    baseline = measurement("baseline", device)
    value = torch.empty((64, 1024, 1024), device=device)
```

```
    pytorch_live = measurement("pytorch_live", device)
    return baseline, pytorch_live
```

Part 6 - Measurement, Validation, and Failure Analysis

Avoid calling `device_used()` twice in production code because another process can change the value between reads; the skeleton emphasizes readability and should cache one reading. System-wide usage is not process attribution. Use NVML process accounting for stronger isolation when available. A negative gap indicates inconsistent scope/timing or parser error and must not be clipped.

Part 7 - Engineering Interpretation and Philosophy

Every observability system has a boundary. Comparing independent scopes exposes what the snapshot cannot see and prevents a clean PyTorch topology from being mistaken for a complete device-memory explanation.

19.6.17 Exercise 16

Exercise Title: Limit Memory-History Recording with max_entries and Measure the Accuracy, Storage Cost, and Runtime Overhead Trade-Off

Part 1 - Exercise Contract and Why It Matters

Measure how bounded history changes event coverage, dump size, host memory, and runtime. Choose a recording limit that preserves the failure window without unbounded diagnostic cost.

Part 2 - System and Memory Mental Model

History uses a finite entry buffer. When events exceed `max_entries`, older evidence may be unavailable according to implementation behavior. More entries improve temporal coverage but increase host storage and dump size; stack collection adds further overhead. The snapshot of current topology can remain accurate even when early history is truncated.

Part 3 - Quantitative Prediction Before Execution

Test 1,000, 10,000, and 100,000 entries on the same deterministic allocation loop. Predict event count ceiling, first/last represented workload phase, file size, runtime, and host-memory cost. Define accuracy as retention of known marker allocations, not merely a larger file.

Part 4 - Experimental Design and Instrumentation

Run each limit in a fresh process with identical workload and output location. Time baseline without recording, history without stacks, and history with stacks. Synchronize workload boundaries, not every allocation. Record dump size and trace-event count.

Part 5 - PyTorch/CUDA Implementation

```
from pathlib import Path
import time
import torch

def run_limit(max_entries, output_path):
```

```
    device = torch.device("cuda:0")
    path = Path(output_path)
    recorder = torch.cuda.memory._record_memory_history
    start = time.perf_counter()
    try:
        recorder(context="all", stacks="python", max_entries=max_entries)
        for iteration in range(5000):
            size = 1024 + (iteration % 97) * 256
            value = torch.empty(size, device=device)
            del value
        torch.cuda.synchronize(device)
        snapshot = torch.cuda.memory._snapshot()
        torch.cuda.memory._dump_snapshot(str(path))
        elapsed = time.perf_counter() - start
        event_count = 0
        for trace in snapshot.get("device_traces", []):
            event_count += len(trace)
        return {
            "max_entries": max_entries,
            "event_count": event_count,
            "seconds": elapsed,
            "file_bytes": path.stat().st_size,
        }
    finally:
        recorder(enabled=None)

def E16(output_directory):
    rows = []
    for limit in [1000, 10000, 100000]:
        path = Path(output_directory) / ("history_" + str(limit) + ".pickle")
        rows.append(run_limit(limit, path))
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

The sequential skeleton shares process cache and should be converted to subprocess trials for rigorous timing. Verify marker events near beginning, middle, and end. File size alone does not measure diagnostic accuracy. Report median runtime across repetitions and distinguish snapshot serialization time from recording overhead.

Part 7 - Engineering Interpretation and Philosophy

Observability budgets are engineering choices. The correct limit retains enough history to explain the incident while keeping continuous instrumentation safe for the system it is meant to protect.

19.6.18 Exercise 17

Exercise Title: Build a Python Snapshot Analyzer That Ranks Segments by Wasted Space, Allocation Count, and Largest Contiguous Free Block

Part 1 - Exercise Contract and Why It Matters

Build a reusable segment analyzer that ranks memory pressure by several orthogonal dimensions. Preserve raw values so ranking logic remains auditable.

Part 2 - System and Memory Mental Model

Segment waste can mean inactive bytes, rounding gap between active block size and requested payload, or free space trapped beside active blocks. Allocation count describes block multiplicity. Largest contiguous inactive block describes fit capability. No single ranking captures all three.

Part 3 - Quantitative Prediction Before Execution

For each segment compute total, active, inactive, requested, rounding waste, active block count, inactive block count, largest inactive block, and external fragmentation proxy. Define three separate ranking outputs.

Part 4 - Experimental Design and Instrumentation

Validate analyzer first on hand-written synthetic snapshots with known answers, then on a real snapshot. Preserve unknown states and validate block coverage. Never mutate the raw snapshot while sorting derived rows.

Part 5 - PyTorch/CUDA Implementation

```
def analyze_segment(index, segment):
    active = 0
    inactive = 0
    requested = 0
    active_count = 0
    largest_free = 0
    for block in segment.get("blocks", []):
        size = int(block.get("size", 0))
        state = block.get("state")
        if str(state).startswith("active"):
            active += size
            requested += int(block.get("requested_size", 0))
            active_count += 1
        elif state == "inactive":
            inactive += size
            largest_free = max(largest_free, size)
    fragmentation = 0.0
    if inactive > 0:
        fragmentation = 1.0 - largest_free / inactive
    return {
        "index": index,
        "address": segment.get("address"),
        "total": int(segment.get("total_size", 0)),
        "active": active,
        "inactive": inactive,
        "rounding_waste": active - requested,
```

```
        "active_count": active_count,
        "largest_free": largest_free,
        "fragmentation": fragmentation,
    }

def E17(segments):
    rows = []
    for index, segment in enumerate(segments):
        rows.append(analyze_segment(index, segment))
    by_waste = sorted(rows, key=lambda row: row["inactive"], reverse=True)
    by_count = sorted(rows, key=lambda row: row["active_count"], reverse=True)
    by_fit = sorted(rows, key=lambda row: row["largest_free"], reverse=True)
    return {"by_waste": by_waste, "by_count": by_count, "by_fit": by_fit}
```

Part 6 - Measurement, Validation, and Failure Analysis

Check that active plus inactive plus unknown equals segment total. The skeleton must be extended with an unknown-state field before production use. Test ties, empty segments, absent requested sizes, and zero inactive space. Compare top rankings with the visualizer but keep programmatic numbers authoritative.

Part 7 - Engineering Interpretation and Philosophy

Ranking is triage. Separate lists prevent a large-but-healthy cached segment from being conflated with a highly fragmented segment that cannot fit the next critical request.

19.6.19 Exercise 18

Exercise Title: Build a Snapshot-Diff Tool That Identifies New, Released, Persistent, and Growing CUDA Allocations Between Two Execution Points

Part 1 - Exercise Contract and Why It Matters

Compare two snapshots from the same process and classify active allocations as new, released, persistent, or changed. Avoid using address identity blindly when history shows address reuse.

Part 2 - System and Memory Mental Model

Within one process, address plus allocation event identity is stronger than address alone. A block can be freed and a later allocation reuse the same address; an address-only diff would falsely call it persistent. History event sequence or stack/request metadata must resolve ambiguous reuse.

Part 3 - Quantitative Prediction Before Execution

Construct synthetic snapshots containing one member of each class. Treat a changed or growing allocation as the same verified identity with changed requested size only if the schema permits it; ordinary tensor storage usually does not grow in place, so address reuse is a competing explanation.

Part 4 - Experimental Design and Instrumentation

Capture equivalent synchronized boundaries and retain device traces spanning both. First perform topology diff, then annotate ambiguous addresses from event history. Return evidence fields with every classification.

Part 5 - PyTorch/CUDA Implementation

```
def active_map(snapshot):
    result = {}
    segments = snapshot.get("segments", snapshot)
    for segment in segments:
        for block in segment.get("blocks", []):
            if block.get("state") != "active_allocated":
                continue
            address = int(block.get("address", 0))
            result[address] = {
                "size": int(block.get("size", 0)),
                "requested": int(block.get("requested_size", 0)),
                "frames": block.get("frames", []),
            }
    return result

def E18(first_snapshot, second_snapshot):
    first = active_map(first_snapshot)
    second = active_map(second_snapshot)
    result = {"new": {}, "released": {}, "persistent": {}, "changed": {}}
    for address, old_block in first.items():
        if address not in second:
            result["released"][address] = old_block
        else:
            new_block = second[address]
            if old_block["requested"] == new_block["requested"]:
                result["persistent"][address] = new_block
            else:
                result["changed"][address] = {
                    "before": old_block,
                    "after": new_block,
                    "requires_history_check": True,
                }
    for address, new_block in second.items():
        if address not in first:
            result["new"][address] = new_block
    return result
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate on synthetic fixtures and a known live workload. Every changed address must be checked against free/allocation events; reclassify reused addresses as released plus new. Summarize bytes and counts by source frame. Cross-process snapshots cannot use raw addresses this way and require normalized signatures.

Part 7 - Engineering Interpretation and Philosophy

A diff is only as sound as its identity rule. Combining topology with history prevents a convenient address match from becoming a false lifetime claim.

19.6.20 Exercise 19

Exercise Title: Diagnose a Hidden CUDA Memory Failure Using Only Allocation History, Stack Traces, Segment Layouts, and OOM Events from a PyTorch Snapshot

Part 1 - Exercise Contract and Why It Matters

Perform a blind end-to-end diagnosis from one snapshot artifact. Use no source inspection initially. Produce a ranked root-cause report, competing hypotheses, falsification tests, and a predicted topology change after repair.

Part 2 - System and Memory Mental Model

Evidence families include segment geometry, active/inactive/awaiting states, allocation/free event history, OOM requests, and source stacks. Candidate causes include retained tensors, variable-shape fragmentation, asynchronous delayed free, capacity exhaustion, optimizer lazy state, and non-PyTorch memory. A snapshot cannot directly see the last category, so unexplained device-level gap must remain a hypothesis.

Part 3 - Quantitative Prediction Before Execution

Build a diagnostic matrix. For each candidate predict active growth, persistent source groups, inactive distribution, largest-free-block relation to OOM size, awaiting-free presence, retry sequence, and driver gap. Define confidence from independent evidence count, not intuition.

Part 4 - Experimental Design and Instrumentation

Freeze the raw artifact and hash it. Parse with version-aware schema checks. Reconstruct the event window before OOM, locate the failed size, compute segment fit, group active allocations by normalized stack, and identify delayed frees. Write the report before revealing the injected failure.

Part 5 - PyTorch/CUDA Implementation

```
def find_oom_events(snapshot):
    events = []
    for device_index, trace in enumerate(snapshot.get("device_traces", [])):
        for event_index, event in enumerate(trace):
            action = str(event.get("action", ""))
            if "oom" in action.lower():
                events.append({
                    "device": device_index,
                    "event_index": event_index,
                    "event": event,
                })
    return events
```

```
def diagnose(snapshot):
    segments = snapshot.get("segments", [])
    total_free, largest_free = largest_inactive(segments)
    ooms = find_oom_events(snapshot)
    return {
        "segment_count": len(segments),
        "total_inactive": total_free,
        "largest_inactive": largest_free,
        "oom_events": ooms,
        "next_analyses": [
            "group_active_allocations_by_stack",
            "match_failed_request_to_largest_free_block",
            "inspect_awaiting_free_blocks",
            "compare_snapshot_reserved_to_device_usage",
        ],
    }

def E19(snapshot):
    return diagnose(snapshot)
```

Part 6 - Measurement, Validation, and Failure Analysis

The skeleton is triage; the final analysis must implement every listed step. Require each conclusion to cite event index, block/segment evidence, and source frame. Reveal the hidden failure, measure diagnostic accuracy, apply the fix, and reproduce the predicted reduction in persistent allocations, fragmentation, or delayed-free state. Record unresolved ambiguity explicitly.

Part 7 - Engineering Interpretation and Philosophy

Expert diagnosis is a chain of evidence that survives challenge. Snapshot history supplies time, layout supplies geometry, stacks supply responsibility, and OOM events supply the failed demand. Their intersection is far stronger than any one counter or visual impression.

19.7 DMM05

Chapter Title: Pinned Memory and Page-Locked Host Buffers
Specialization: Dynamic Memory Management and CUDA Systems

19.7.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Pageable RAM, Pinned Memory, DMA, and Host-to-GPU Transfer Paths

Part 1 - Exercise Contract and Why It Matters

This exercise establishes the physical model needed for every later benchmark. You must distinguish ordinary pageable host memory, page-locked host memory, device memory, the CUDA driver, and the DMA engine. The deliverable is not the slogan “pinned is faster.” It is a causal explanation of what work occurs, which component performs it, and when the host

is allowed to continue. A good model must also explain why pinning can improve a pipeline, harm a one-shot copy, or exhaust an important operating-system resource.

Part 2 - System and Memory Mental Model

Virtual CPU pages normally may be relocated or swapped. A device DMA engine cannot safely read from a page whose physical location may change during the transfer. For a pageable source, the CUDA runtime may therefore stage the bytes through a temporary page-locked region before DMA moves them across the host– device interconnect. A pinned tensor already occupies stable physical pages, removing that staging requirement and permitting asynchronous DMA submission.

Pinning is not free. Page locking requires operating-system work, consumes a finite host resource, and reduces the memory available for paging decisions. Moreover, `non_blocking=True` means the host need not wait after copy submission; it does not prove that copy and compute overlap. True overlap needs a pinned source, a capable device, suitable CUDA streams, independent work, and correct event dependencies. Default-stream serialization can erase the gain.

Part 3 - Quantitative Prediction Before Execution

For a tensor with dimensions $n_1, \ldots, n_k$ and element width w bytes, predict

$$B = w \prod_{i=1}^{k} n_i.$$

If a 256 MiB float32 batch contains 67,108,864 elements, estimate the ideal transfer time as $t = B/R$ for an assumed effective bandwidth R. Then add a fixed submission cost and a page-locking cost. Predict separately: copy-only latency from pre-pinned memory, pageable copy latency, and end-to-end “allocate, pin, copy” latency. The three predictions answer different questions and must never be collapsed into one number.

Part 4 - Experimental Design and Instrumentation

Record CPU model, GPU, interconnect, PyTorch/CUDA versions, tensor size, dtype, and stream. Warm up CUDA before measurement. Synchronize before a CPU wall-time trial and after its final transfer. Use CUDA events for device-side copy time. Measure pageable, pre-pinned, and pin-then-copy paths independently. Preserve the source until transfer completion, validate the destination, and report median and tail latency rather than one favorable sample.

Part 5 - PyTorch/CUDA Implementation

```
import statistics
import time
import torch

def cpu_ms(operation, device):
    torch.cuda.synchronize(device)
    start = time.perf_counter()
    result = operation()
    torch.cuda.synchronize(device)
    return (time.perf_counter() - start) * 1000.0, result
```

```
def event_ms(source, device, stream):
    start = torch.cuda.Event(enable_timing=True)
    end = torch.cuda.Event(enable_timing=True)
    with torch.cuda.stream(stream):
        start.record(stream)
        destination = source.to(device, non_blocking=True)
        end.record(stream)
    end.synchronize()
    return start.elapsed_time(end), destination

def E00(numel=16 * 1024 * 1024, repeats=20):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required for DMM05.")
    device = torch.device("cuda:0")
    pageable = torch.arange(numel, dtype=torch.float32)
    pinned = torch.empty(numel, dtype=torch.float32, pin_memory=True)
    pinned.copy_(pageable)
    stream = torch.cuda.Stream(device=device)
    torch.empty(1, device=device)
    rows = []
    for label, source in (("pageable", pageable), ("pinned", pinned)):
        wall_samples = []
        event_samples = []
        for _ in range(repeats):
            wall, destination = cpu_ms(
                lambda: source.to(device, non_blocking=True), device)
            event, destination = event_ms(source, device, stream)
            wall_samples.append(wall)
            event_samples.append(event)
        rows.append({"path": label,
                     "bytes": source.numel() * source.element_size(),
                     "wall_median_ms": statistics.median(wall_samples),
                     "event_median_ms": statistics.median(event_samples),
                     "source_is_pinned": source.is_pinned()})
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

CUDA-event time and synchronized wall time need not match: the first describes work on a stream while the second includes Python submission and synchronization. Confirm equal byte counts and inspect every destination before reusing source storage. If pinned memory is not faster, test whether tensors are too small, pinning was timed, transfers are serialized, or the system is bandwidth- limited elsewhere. A speedup without numerical validation is invalid evidence.

Part 7 - Engineering Interpretation and Philosophy

Pinned memory is a coordination mechanism between virtual memory and DMA, not a magic storage class. Optimize the whole path: page locking, CPU production, queueing, trans-

fer, synchronization, and GPU consumption. The correct question is not "is pinning fast?" but "does a bounded pinned-memory pipeline shorten the critical path on this measured system?"

19.7.2 Exercise 01

Exercise Title: Create Pageable and Pinned CPU Tensors in PyTorch and Verify Their Memory State with is_pinned()

Part 1 - Exercise Contract and Why It Matters

Create equivalent CPU tensors in pageable and pinned storage, prove their state with `is_pinned()`, and test which transformations preserve or replace that storage. The purpose is to prevent a common debugging error: assuming that a tensor is pinned because its parent batch or an earlier version was pinned.

Part 2 - System and Memory Mental Model

Pinned state belongs to the underlying CPU storage. A view normally shares that storage and therefore retains the property; a cloning or conversion operation may allocate different storage whose pinning depends on the operation. CUDA tensors are not described as pinned host tensors. Nested batches require every CPU tensor destined for transfer to be inspected, not just the outer object.

Part 3 - Quantitative Prediction Before Execution

Predict the byte count, data pointers, and pinned flags for an ordinary tensor, a directly pinned tensor, a view, a clone, and a tensor returned by `pin_memory()`. Record which objects should share storage before running the experiment.

Part 4 - Experimental Design and Instrumentation

Use identical shapes and values. Report shape, dtype, bytes, device, data pointer, pinned state, and whether storage is shared. Validate values after every transformation. Run in a CUDA-capable PyTorch environment because pinned allocation may be unavailable in a CPU-only build.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def describe(name, tensor):
    return {"name": name,
            "shape": tuple(tensor.shape),
            "dtype": str(tensor.dtype),
            "device": str(tensor.device),
            "bytes": tensor.numel() * tensor.element_size(),
            "data_ptr": tensor.data_ptr(),
            "is_pinned": tensor.is_pinned()}

def E01():
    if not torch.cuda.is_available():
        raise RuntimeError("A CUDA-capable PyTorch build is required.")
    pageable = torch.arange(24, dtype=torch.float32).reshape(4, 6)
```

```
    direct = torch.empty((4, 6), dtype=torch.float32, pin_memory=True)
    direct.copy_(pageable)
    converted = pageable.pin_memory()
    view = direct.view(2, 12)
    clone = direct.clone()
    tensors = [("pageable", pageable), ("direct", direct),
               ("converted", converted), ("view", view), ("clone", clone)]
    rows = []
    for name, tensor in tensors:
        rows.append(describe(name, tensor))
        if not torch.equal(tensor.reshape(-1), pageable.reshape(-1)):
            raise AssertionError(name + " changed the values")
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Check flags rather than relying on an allocation story. Confirm that the view's pointer matches the pinned base. Do not turn an observed clone behavior into an eternal API guarantee; record PyTorch version and verify it. Failure to allocate pinned memory can reflect a CPU-only build, unavailable accelerator support, or host resource limits rather than a tensor-shape error.

Part 7 - Engineering Interpretation and Philosophy

Performance properties must be observable invariants. A pipeline that merely intends to pin is weaker than one that asserts pinning at its transfer boundary. Small state checks convert an implicit assumption into testable engineering.

19.7.3 Exercise 02

Exercise Title: Allocate CPU Tensors Directly in Pinned Memory and Compare Them with Tensors Converted Later Using pin_memory()

Part 1 - Exercise Contract and Why It Matters

Compare direct pinned allocation with creating pageable data and converting it later. Separate allocation time, data-production time, pinning/copy time, and H2D time. This reveals why identical final tensor states can have very different end-to-end costs.

Part 2 - System and Memory Mental Model

`pin_memory()` creates a pinned copy when the source is not already pinned; it cannot retroactively change arbitrary storage without moving data. Direct allocation avoids that conversion copy, but the producer must be able to write into the pinned destination efficiently. Excessive direct pinned allocations can still incur expensive page-lock operations and fragment host resource usage.

Part 3 - Quantitative Prediction Before Execution

For B bytes, model converted-path cost as $T_p(B) + T_{cpu \to pin}(B) + T_{H2D}(B)$, while a reused direct buffer can approach $T_{fill}(B) + T_{H2D}(B)$. Predict which term dominates for 4 KiB, 4 MiB, and 256 MiB.

Part 4 - Experimental Design and Instrumentation

Pre-generate source values so random-number generation is not accidentally charged to only one path. Measure construction and transfer phases separately, then report their sum. Repeat, randomize path order, synchronize CUDA, and release large host buffers between independent size groups.

Part 5 - PyTorch/CUDA Implementation

```
import statistics
import time
import torch

def elapsed_ms(operation):
    start = time.perf_counter()
    value = operation()
    return (time.perf_counter() - start) * 1000.0, value

def E02(numel=8 * 1024 * 1024, repeats=15):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    source = torch.linspace(0.0, 1.0, numel, dtype=torch.float32)
    records = {"direct_prepare": [], "convert_prepare": [],
               "direct_h2d": [], "convert_h2d": []}
    for _ in range(repeats):
        prepare, direct = elapsed_ms(
            lambda: torch.empty(numel, dtype=torch.float32, pin_memory=True))
        fill, _ = elapsed_ms(lambda: direct.copy_(source))
        records["direct_prepare"].append(prepare + fill)
        prepare, converted = elapsed_ms(lambda: source.pin_memory())
        records["convert_prepare"].append(prepare)
        for label, tensor in (("direct_h2d", direct),
                              ("convert_h2d", converted)):
            torch.cuda.synchronize(device)
            start = time.perf_counter()
            destination = tensor.to(device, non_blocking=True)
            torch.cuda.synchronize(device)
            records[label].append((time.perf_counter() - start) * 1000.0)
            if not torch.allclose(destination.cpu(), source):
                raise AssertionError(label + " produced wrong values")
    summary = {}
    for name, samples in records.items():
        summary[name + "_median_ms"] = statistics.median(samples)
    return summary
```

Part 6 - Measurement, Validation, and Failure Analysis

Both final tensors must be pinned and value-equivalent. If conversion appears free, check that synchronization and phase boundaries are correct. If direct allocation appears slower,

the benchmark may be measuring fresh page locking while a real pipeline would reuse buffers. Report both fresh-allocation and reuse scenarios; neither alone represents every workload.

Part 7 - Engineering Interpretation and Philosophy

Storage state does not reveal acquisition cost. Design around lifetimes: direct allocation plus reuse is often valuable, whereas pinning a fresh pageable batch immediately before one copy can add a full CPU copy to the critical path.

19.7.4 Exercise 03

Exercise Title: Benchmark Host-to-GPU Transfer Latency for Pageable and Pinned Tensors Across Increasing Tensor Sizes

Part 1 - Exercise Contract and Why It Matters

Build a size sweep that exposes fixed overhead, bandwidth-dominated behavior, and pinned-memory crossover. The benchmark must distinguish copy-only latency from end-to-end preparation cost and must not use unsynchronized Python timing.

Part 2 - System and Memory Mental Model

Small transfers are dominated by dispatch, synchronization, and bookkeeping. Large transfers approach an effective bandwidth ceiling. Pageable transfers may add staging work, while pre-pinned transfers can be submitted directly to DMA. The observed curve is therefore approximately fixed cost plus bytes over bandwidth, but caching, NUMA placement, interconnect sharing, and power state create deviations.

Part 3 - Quantitative Prediction Before Execution

Choose logarithmic sizes from 1 KiB to a safe fraction of host and GPU memory. For each size predict $T = \alpha + B/R$ under two path-specific values of α and R. Mark the expected crossover before viewing results.

Part 4 - Experimental Design and Instrumentation

Allocate sources before timing, warm each size, alternate path order, and use many repetitions for small transfers. Synchronize before and after each trial. Report median, p95, bytes, and GB/s. Prevent dead-code assumptions by reading a destination value after the synchronized trial group.

Part 5 - PyTorch/CUDA Implementation

```
import statistics
import time
import torch

def percentile(values, fraction):
    ordered = sorted(values)
    index = round((len(ordered) - 1) * fraction)
    return ordered[index]

def transfer_samples(source, device, repeats):
    samples = []
```

```
    for _ in range(repeats):
        torch.cuda.synchronize(device)
        start = time.perf_counter()
        destination = source.to(device, non_blocking=True)
        torch.cuda.synchronize(device)
        samples.append((time.perf_counter() - start) * 1000.0)
    return samples, float(destination[0].item())

def E03():
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    sizes = [2 ** exponent for exponent in range(10, 29, 2)]
    rows = []
    for byte_count in sizes:
        numel = max(1, byte_count // 4)
        pageable = torch.ones(numel, dtype=torch.float32)
        pinned = torch.ones(numel, dtype=torch.float32, pin_memory=True)
        for label, source in (("pageable", pageable), ("pinned", pinned)):
            samples, check = transfer_samples(source, device, 25)
            median_ms = statistics.median(samples)
            seconds = median_ms / 1000.0
            bandwidth = source.nbytes / seconds / 1e9
            rows.append({"path": label, "bytes": source.nbytes,
                         "median_ms": median_ms,
                         "p95_ms": percentile(samples, 0.95),
                         "GB_per_s": bandwidth, "check": check})
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Plot latency on log scales and bandwidth against size. Reject trials affected by unrelated GPU work only through a declared rule, never by deleting slow points selectively. A pageable copy with `non_blocking=True` can still return control early at the API level, so synchronized end-to-end timing and pinned-state checks remain essential.

Part 7 - Engineering Interpretation and Philosophy

An optimization has a domain, not a universal truth. The curve identifies the sizes where page locking repays its cost and where batching small transfers is more important than changing memory type.

19.7.5 Exercise 04

Exercise Title: Determine the Tensor Size at Which Pinned Memory Begins to Outperform Pageable Memory on Your Hardware

Part 1 - Exercise Contract and Why It Matters

Estimate a statistically defensible crossover rather than reporting the first noisy point where pinned latency happens to be lower. Produce a local hardware result with uncertainty and a rule that can be rerun after system changes.

Part 2 - System and Memory Mental Model

The crossover is the region where saved staging cost exceeds differences in fixed overhead. It depends on interconnect, NUMA placement, CPU load, driver, power state, and whether memory was pre-pinned or pinned on demand. Consequently there are at least two crossovers: copy-only and end-to-end pin-then-copy.

Part 3 - Quantitative Prediction Before Execution

Define speedup $S(B) = T_{pageable}(B)/T_{pinned}(B)$. Before execution choose a practical threshold such as $S \geq 1.05$, then require it at three consecutive sizes and require confidence intervals not to reverse the conclusion. State the chosen rule before looking at measurements.

Part 4 - Experimental Design and Instrumentation

Use a coarse logarithmic sweep, then a dense sweep around the apparent region. Interleave paths per repetition to reduce temporal bias. Bootstrap medians or repeat complete runs. Record both pre-pinned copy-only and pin-then-copy totals.

Part 5 - PyTorch/CUDA Implementation

```
import statistics

def find_crossover(rows, minimum_speedup=1.05, consecutive=3):
    by_size = {}
    for row in rows:
        size = row["bytes"]
        if size not in by_size:
            by_size[size] = {}
        by_size[size][row["path"]] = row["median_ms"]
    streak = 0
    first_size = None
    decisions = []
    for size in sorted(by_size):
        values = by_size[size]
        if "pageable" not in values or "pinned" not in values:
            continue
        speedup = values["pageable"] / values["pinned"]
        passes = speedup >= minimum_speedup
        decisions.append({"bytes": size, "speedup": speedup,
                          "passes": passes})
        if passes:
            if streak == 0:
                first_size = size
            streak += 1
            if streak >= consecutive:
                return first_size, decisions
        else:
            streak = 0
            first_size = None
    return None, decisions
```

```
def E04(rows_from_E03):
    crossover, evidence = find_crossover(rows_from_E03)
    return {"copy_only_crossover_bytes": crossover,
            "decision_rule": "speedup >= 1.05 for three sizes",
            "evidence": evidence}
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify that both paths moved identical bytes and that the source was already pinned in the pinned path. Report "no stable crossover in tested range" when appropriate. Do not interpolate a precise byte value from widely spaced noisy samples. Re-run after changing PCIe load, NUMA affinity, or DataLoader policy.

Part 7 - Engineering Interpretation and Philosophy

The crossover is a policy input, not a law of CUDA. It can guide whether to pin individual fields, coalesce transfers, or leave tiny metadata pageable. A repeatable decision rule is more useful than a memorable but hardware-specific number.

19.7.6 Exercise 05

Exercise Title: Measure the CPU-Side Cost of Calling pin_memory() and Explain Why Manual Pinning Can Eliminate Expected Transfer Gains

Part 1 - Exercise Contract and Why It Matters

Measure page-lock conversion separately from H2D transfer and then combine the phases. Demonstrate the central trap: pre-pinned memory can accelerate the copy. However, manually converting the batch immediately before the copy can make the entire operation slower. The conversion method under test is `pin_memory()`.

Part 2 - System and Memory Mental Model

Converting a pageable tensor normally allocates page-locked storage and copies the CPU data into it. The path becomes CPU copy plus OS/driver registration plus H2D DMA. Pre-pinning moves this work off the per-batch critical path; a pinned pool amortizes it across many iterations. DataLoader pinning can also perform the work in a separate pipeline stage rather than blocking model code.

Part 3 - Quantitative Prediction Before Execution

Let $P(B)$ be conversion cost, $C_p(B)$ pinned-copy time, and $C_u(B)$ pageable-copy time. Predict whether $P(B) + C_p(B) < C_u(B)$. Then compute the reuse count N needed for $P(B)/N + C_p(B) < C_u(B)$.

Part 4 - Experimental Design and Instrumentation

Time pinning with a CPU clock, copy with synchronized wall time and CUDA events, and total latency with both phases. Keep generation outside timing. Compare fresh conversion, pre-pinned, and reusable-buffer paths at multiple sizes.

Part 5 - PyTorch/CUDA Implementation

```
import statistics
import time
import torch

def E05(numel=16 * 1024 * 1024, repeats=20):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    source = torch.randn(numel, dtype=torch.float32)
    pin_ms = []
    copy_ms = []
    total_ms = []
    for _ in range(repeats):
        total_start = time.perf_counter()
        pin_start = time.perf_counter()
        pinned = source.pin_memory()
        pin_ms.append((time.perf_counter() - pin_start) * 1000.0)
        torch.cuda.synchronize(device)
        copy_start = time.perf_counter()
        destination = pinned.to(device, non_blocking=True)
        torch.cuda.synchronize(device)
        copy_ms.append((time.perf_counter() - copy_start) * 1000.0)
        total_ms.append((time.perf_counter() - total_start) * 1000.0)
    return {"bytes": source.nbytes,
            "pin_median_ms": statistics.median(pin_ms),
            "copy_median_ms": statistics.median(copy_ms),
            "pin_then_copy_median_ms": statistics.median(total_ms),
            "validated": torch.allclose(destination.cpu(), source)}
```

Part 6 - Measurement, Validation, and Failure Analysis

Compare the median of totals with the total of medians carefully; they are not identical statistical quantities. If manual pinning wins unexpectedly, verify that pageable and pinned cases used identical synchronization. If it loses, that is not a contradiction: the copy benefit may be smaller than conversion cost. Inspect host-memory pressure and repeated-registration effects.

Part 7 - Engineering Interpretation and Philosophy

Move optimization work away from the critical path or amortize it. Pinning is a pipeline design choice, not a function to sprinkle immediately before every `to("cuda")` call.

19.7.7 Exercise 06

Exercise Title: Compare tensor.to("cuda") Against Pinned-Memory Transfers Under Identical Shapes, Dtypes, and Repetition Counts

Part 1 - Exercise Contract and Why It Matters

Construct a paired benchmark where the only intended difference is source storage. Compare ordinary `tensor.to("cuda")` with a transfer from an already pinned tensor, while making blocking policy explicit.

Part 2 - System and Memory Mental Model

`to()` is the high-level transfer interface for both sources. Its `non_blocking` flag changes host synchronization behavior, not the number of bytes. Pinned storage enables the DMA-friendly path; pageable storage may require internal staging. Comparisons are meaningless if one source is generated, converted, or pinned inside the timed region and the other is not.

Part 3 - Quantitative Prediction Before Execution

Create a four-cell prediction table. The rows represent pageable and pinned sources; the columns represent blocking and non-blocking submission. Predict CPU submission time, synchronized completion time, and overlap potential for each cell.

Part 4 - Experimental Design and Instrumentation

Use one pageable tensor and a value-equivalent pinned tensor. Alternate all four cases, synchronize at boundaries, and keep repetitions equal. Record host call duration separately from completion duration so asynchronous submission is not misreported as completed bandwidth.

Part 5 - PyTorch/CUDA Implementation

```
import statistics
import time
import torch

def measure_case(source, device, non_blocking, repeats):
    call_samples = []
    completion_samples = []
    for _ in range(repeats):
        torch.cuda.synchronize(device)
        start = time.perf_counter()
        destination = source.to(device, non_blocking=non_blocking)
        call_end = time.perf_counter()
        torch.cuda.synchronize(device)
        end = time.perf_counter()
        call_samples.append((call_end - start) * 1000.0)
        completion_samples.append((end - start) * 1000.0)
    return {"call_median_ms": statistics.median(call_samples),
            "completion_median_ms": statistics.median(completion_samples),
            "checksum": float(destination.float().sum().item())}

def E06(numel=8 * 1024 * 1024, repeats=30):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    pageable = torch.arange(numel, dtype=torch.float32)
    pinned = torch.empty_like(pageable, pin_memory=True)
    pinned.copy_(pageable)
    results = {}
    for name, source in (("pageable", pageable), ("pinned", pinned)):
        for flag in (False, True):
```

```
            key = name + "_nonblocking_" + str(flag)
            results[key] = measure_case(source, device, flag, repeats)
    return results
```

Part 6 - Measurement, Validation, and Failure Analysis

Assert equal checksums and byte counts. A shorter host-call duration is not a faster completed transfer. If `non_blocking=True` changes little, the explicit synchronization deliberately removes overlap; that is correct for copy completion measurement. A separate pipeline benchmark is required to measure useful overlap.

Part 7 - Engineering Interpretation and Philosophy

APIs do not define performance in isolation. Storage type, synchronization, stream placement, and surrounding work jointly determine the critical path.

19.7.8 Exercise 07

Exercise Title: Sweep float16, float32, and float64 Tensors and Determine Whether Transfer Performance Follows Bytes, Elements, or Dtype

Part 1 - Exercise Contract and Why It Matters

Determine whether H2D performance is primarily predicted by byte count, element count, or dtype-specific effects. Run both equal-element and equal-byte experiments so the conclusion is identifiable.

Part 2 - System and Memory Mental Model

DMA transports bytes and does not perform floating-point arithmetic. Therefore equal element counts should take longer for wider dtypes, while equal byte counts should be similar once bandwidth dominates. Differences may remain from alignment, allocation, small-transfer overhead, or downstream conversion, but these must not be confused with copy volume.

Part 3 - Quantitative Prediction Before Execution

For N elements, predict byte ratios $1 : 2 : 4$ for float16, float32, and float64. For a fixed 64 MiB payload, predict nearly equal transfer times. Write both hypotheses before measurement.

Part 4 - Experimental Design and Instrumentation

Preallocate pinned sources, use synchronized completion time, and alternate dtypes. First hold elements constant; then calculate dtype-specific element counts that hold bytes constant. Report both latency and effective GB/s.

Part 5 - PyTorch/CUDA Implementation

```
import statistics
import torch

def cuda_event_copy_ms(source, device, repeats):
    samples = []
    for _ in range(repeats):
        start = torch.cuda.Event(enable_timing=True)
```

```
        end = torch.cuda.Event(enable_timing=True)
        start.record()
        destination = source.to(device, non_blocking=True)
        end.record()
        end.synchronize()
        samples.append(start.elapsed_time(end))
    return statistics.median(samples), destination

def E07(repeats=30):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    dtypes = [torch.float16, torch.float32, torch.float64]
    rows = []
    for mode in ("equal_elements", "equal_bytes"):
        for dtype in dtypes:
            if mode == "equal_elements":
                numel = 16 * 1024 * 1024
            else:
                target_bytes = 64 * 1024 * 1024
                scalar = torch.empty((), dtype=dtype)
                element_bytes = scalar.element_size()
                numel = target_bytes // element_bytes
            source = torch.ones(numel, dtype=dtype, pin_memory=True)
            median_ms, destination = cuda_event_copy_ms(
                source, device, repeats)
            seconds = median_ms / 1000.0
            bandwidth = source.nbytes / seconds / 1e9
            rows.append({"mode": mode, "dtype": str(dtype),
                         "elements": numel, "bytes": source.nbytes,
                         "median_ms": median_ms,
                         "GB_per_s": bandwidth,
                         "check": float(destination[0].item())})
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Compare latency ratios in the equal-element group and bandwidth in the equal- byte group. Ensure no dtype conversion occurs during transfer. If equal-byte results differ, expand repetitions, inspect alignment and clock state, and test larger payloads before claiming a dtype-specific DMA path.

Part 7 - Engineering Interpretation and Philosophy

Choose the independent variable that matches the mechanism. Element count is a model-level quantity; bytes are the transport-level quantity. Good experiments separate them instead of letting dtype change both simultaneously.

19.7.9 Exercise 08

Exercise Title: Build a Bandwidth Benchmark That Converts CPU-to-GPU Transfer Time into Effective GB/s

Part 1 - Exercise Contract and Why It Matters

Create a reusable bandwidth benchmark with correct units, warmup, synchronization, and uncertainty reporting. It must distinguish decimal GB/s from binary GiB/s and useful payload bandwidth from theoretical link rate.

Part 2 - System and Memory Mental Model

Effective bandwidth is payload bytes divided by completed transfer time. It is lower than signaling rate because protocols, topology, contention, and software overhead consume capacity. For small tensors, dividing by a tiny latency creates a volatile number that should not be treated as a steady-state bandwidth.

Part 3 - Quantitative Prediction Before Execution

Calculate

$$R_{GB/s} = \frac{B}{t\,10^9}, \qquad R_{GiB/s} = \frac{B}{t\,2^{30}}.$$

Predict the plateau region and define the minimum payload used to summarize steady-state bandwidth.

Part 4 - Experimental Design and Instrumentation

Use CUDA events around only the copy, a pinned source, fixed destination device, and warmup transfers. Include median, p05, and p95. Optionally measure bidirectional and concurrent traffic separately; do not mix them into H2D results.

Part 5 - PyTorch/CUDA Implementation

```
import statistics
import torch

def quantile(samples, fraction):
    values = sorted(samples)
    return values[round((len(values) - 1) * fraction)]

def E08(byte_count=256 * 1024 * 1024, warmup=10, repeats=50):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    numel = byte_count // 4
    source = torch.empty(numel, dtype=torch.float32, pin_memory=True)
    source.fill_(1.0)
    for _ in range(warmup):
        destination = source.to(device, non_blocking=True)
    torch.cuda.synchronize(device)
    samples = []
    for _ in range(repeats):
```

```
        start = torch.cuda.Event(enable_timing=True)
        end = torch.cuda.Event(enable_timing=True)
        start.record()
        destination = source.to(device, non_blocking=True)
        end.record()
        end.synchronize()
        samples.append(start.elapsed_time(end))
    median_ms = statistics.median(samples)
    return {"bytes": source.nbytes, "median_ms": median_ms,
            "p05_ms": quantile(samples, 0.05),
            "p95_ms": quantile(samples, 0.95),
            "GB_per_s": source.nbytes / (median_ms / 1000.0) / 1e9,
            "GiB_per_s": source.nbytes / (median_ms / 1000.0) / (2 ** 30),
            "validated": float(destination[0].item()) == 1.0}
```

Part 6 - Measurement, Validation, and Failure Analysis

Reject zero or negative event times and verify the recorded stream. Compare the plateau with plausible hardware limits, but do not label a discrepancy a bug without checking PCIe generation, lane width, NUMA socket, other traffic, and power management. Preserve raw samples for reproducibility.

Part 7 - Engineering Interpretation and Philosophy

Bandwidth is a derived measurement whose credibility depends entirely on the time boundary and byte definition. A disciplined benchmark turns a vague "transfer is slow" complaint into a comparable engineering metric.

19.7.10 Exercise 09

Exercise Title: Compare One Large Pinned Transfer Against Hundreds of Small Pinned Transfers with the Same Total Byte Count

Part 1 - Exercise Contract and Why It Matters

Hold total bytes constant while changing transfer granularity. Quantify the submission and scheduling cost of many small copies and test whether coalescing fields into a contiguous staging buffer improves throughput.

Part 2 - System and Memory Mental Model

Every copy has fixed software and queueing overhead. Hundreds of small DMA requests may fail to saturate the interconnect and enlarge Python dispatch cost. One large copy reduces request count but may require an additional CPU packing copy, so both copy-only and end-to-end packed results are necessary.

Part 3 - Quantitative Prediction Before Execution

For N transfers totaling B bytes, model $T_N \approx N\alpha + B/R$. Predict the slowdown for $N = 1, 16, 256$, then add packing time to determine whether coalescing remains beneficial.

Part 4 - Experimental Design and Instrumentation

Preallocate pinned sources with equal total bytes. Time the entire sequence using events and synchronized wall time. Preserve destinations until completion. Measure a separate packed path that explicitly includes CPU packing.

Part 5 - PyTorch/CUDA Implementation

```
import statistics
import time
import torch

def copy_group(sources, device, repeats):
    samples = []
    checksum = None
    for _ in range(repeats):
        torch.cuda.synchronize(device)
        start = time.perf_counter()
        destinations = []
        for source in sources:
            destination = source.to(device, non_blocking=True)
            destinations.append(destination)
        torch.cuda.synchronize(device)
        samples.append((time.perf_counter() - start) * 1000.0)
        checksum = 0.0
        for item in destinations:
            checksum += float(item[0].item())
    return statistics.median(samples), checksum

def E09(total_elements=16 * 1024 * 1024, repeats=20):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    rows = []
    for count in (1, 16, 256):
        elements_each = total_elements // count
        sources = []
        for index in range(count):
            tensor = torch.full((elements_each,), float(index + 1),
                                dtype=torch.float32, pin_memory=True)
            sources.append(tensor)
        median_ms, checksum = copy_group(sources, device, repeats)
        actual_bytes = sum(tensor.nbytes for tensor in sources)
        rows.append({"transfers": count, "bytes": actual_bytes,
                     "median_ms": median_ms,
                     "GB_per_s": actual_bytes / (median_ms / 1000.0) / 1e9,
                     "checksum": checksum})
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Confirm identical actual bytes; integer division can otherwise make totals differ. The checksum must reflect every destination. If packing is evaluated, include its CPU time in end-to-end results and show copy-only results separately. Python overhead is part of this pipeline experiment, not an error to hide.

Part 7 - Engineering Interpretation and Philosophy

Pinned memory removes one obstacle but does not remove per-operation overhead. Efficient systems manage granularity: coalesce where packing cost is smaller than repeated submission cost, and retain separate tensors where semantics or lifetime make packing wasteful.

19.7.11 Exercise 10

Exercise Title: Use DataLoader(pin_memory=True) and Verify That Returned Batch Tensors Are Actually Page-Locked Before CUDA Transfer

Part 1 - Exercise Contract and Why It Matters

Build a DataLoader that returns nested batches, enable automatic pinning, and assert that every transferable CPU tensor is page-locked before calling `to("cuda")`. This tests pipeline reality rather than configuration intent.

Part 2 - System and Memory Mental Model

With `pin_memory=True`, DataLoader recognizes tensors in standard containers and moves them into pinned host storage before yielding the batch. Custom batch classes may need a `pin_memory()` method. Pinning is a host pipeline stage; CUDA transfer still occurs in the training process, normally with `non_blocking=True`.

Part 3 - Quantitative Prediction Before Execution

Predict the total pinned bytes per batch from every input and label tensor. Estimate in-flight pinned capacity as batch bytes times the number of queued batches, while noting that implementation details and consumer timing can add or subtract buffers.

Part 4 - Experimental Design and Instrumentation

Compare loaders differing only in `pin_memory`. Inspect the batch before device transfer. Include nested dictionaries and labels, and fail if any tensor that should transfer is not pinned. Keep dataset values deterministic.

Part 5 - PyTorch/CUDA Implementation

```
import torch
from torch.utils.data import DataLoader, Dataset

class SampleDataset(Dataset):
    def __init__(self, length=256):
        self.length = length
    def __len__(self):
        return self.length
    def __getitem__(self, index):
        image = torch.full((3, 64, 64), index / self.length)
        label = torch.tensor(index % 10, dtype=torch.long)
```

```
        return {"image": image, "label": label}

def tensor_rows(value, prefix="batch"):
    rows = []
    if torch.is_tensor(value):
        rows.append({"path": prefix, "shape": tuple(value.shape),
                     "bytes": value.nbytes, "pinned": value.is_pinned()})
    elif isinstance(value, dict):
        for key, child in value.items():
            child_rows = tensor_rows(child, prefix + "." + str(key))
            rows.extend(child_rows)
    elif isinstance(value, (list, tuple)):
        for index, child in enumerate(value):
            child_rows = tensor_rows(child, prefix + "[" + str(index) + "]")
            rows.extend(child_rows)
    return rows

def E10():
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    loader = DataLoader(SampleDataset(), batch_size=16, num_workers=2,
                        pin_memory=True, persistent_workers=True)
    batch = next(iter(loader))
    rows = tensor_rows(batch)
    for row in rows:
        if not row["pinned"]:
            raise AssertionError(row["path"] + " is not pinned")
    device = torch.device("cuda:0")
    images = batch["image"].to(device, non_blocking=True)
    labels = batch["label"].to(device, non_blocking=True)
    torch.cuda.synchronize(device)
    return {"rows": rows, "image_device": str(images.device),
            "label_device": str(labels.device)}
```

Part 6 - Measurement, Validation, and Failure Analysis

Check all leaves, not only the image. If a custom object remains pageable, implement and test its pinning method or return supported containers. On some accelerator/build configurations, pinning behavior differs; record environment and fail clearly. Never call `is_pinned()` after transfer and mistake a CUDA tensor check for host-state evidence.

Part 7 - Engineering Interpretation and Philosophy

Configuration flags are hypotheses about system state. Assertions at the boundary where data changes ownership make the pipeline auditable and prevent silent performance regressions.

19.7.12 Exercise 11

Exercise Title: Benchmark a Training Input Pipeline with pin_memory=False and pin_memory=True While Holding All Other DataLoader Settings Constant

Part 1 - Exercise Contract and Why It Matters

Measure whether DataLoader pinning improves completed training throughput, not merely transfer microbenchmark latency. Hold model, data, workers, prefetch, batch size, and training work constant.

Part 2 - System and Memory Mental Model

End-to-end iteration time is the maximum or partial overlap of CPU preparation, pinning, H2D transfer, and GPU compute, plus synchronization constraints. A copy speedup may disappear if compute dominates; pinning may help significantly when the GPU waits for input. Worker startup must be separated from steady state.

Part 3 - Quantitative Prediction Before Execution

Predict samples/s and GPU idle fraction for both settings. Use the critical- path model $T \approx \max(T_{input}, T_{compute})$ only when overlap is real; otherwise use the sum. State which regime you expect.

Part 4 - Experimental Design and Instrumentation

Run warmup epochs, fixed seeds, identical batch order, equal measured batches, and multiple repetitions. Synchronize only at benchmark boundaries. Record host RAM, CPU utilization, GPU utilization, and batch p95 in addition to mean throughput.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch
from torch.utils.data import DataLoader

def run_loader(loader, model, device, measured_batches):
    iterator = iter(loader)
    torch.cuda.synchronize(device)
    start = time.perf_counter()
    examples = 0
    for _ in range(measured_batches):
        batch = next(iterator)
        inputs = batch["image"].to(device, non_blocking=True)
        outputs = model(inputs)
        loss = outputs.square().mean()
        loss.backward()
        model.zero_grad(set_to_none=True)
        examples += inputs.shape[0]
    torch.cuda.synchronize(device)
    seconds = time.perf_counter() - start
    return {"seconds": seconds, "examples": examples,
            "examples_per_second": examples / seconds}

def E11(dataset, model, batch_size=64, measured_batches=100):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
```

```
    device = torch.device("cuda:0")
    model = model.to(device)
    results = {}
    for pin in (False, True):
        loader = DataLoader(dataset, batch_size=batch_size, shuffle=False,
                            num_workers=4, pin_memory=pin,
                            persistent_workers=True, prefetch_factor=2)
        warmup = run_loader(loader, model, device, 10)
        measurement = run_loader(loader, model, device, measured_batches)
        results["pin_memory_" + str(pin)] = measurement
    return results
```

Part 6 - Measurement, Validation, and Failure Analysis

Reset model state or disable parameter updates so the two conditions perform equivalent work. Ensure the dataset has enough batches and avoid including worker startup in only one condition. If results are close, report uncertainty rather than declaring a winner. A slower pinned path may indicate small batches, CPU pinning pressure, or a compute-bound workload.

Part 7 - Engineering Interpretation and Philosophy

Microbenchmarks explain mechanisms; pipeline benchmarks decide policy. Adopt pinning only when it improves the target workload under realistic concurrency and host-memory constraints.

19.7.13 Exercise 12

Exercise Title: Vary num_workers with Pinned DataLoader Memory and Identify When Input Preparation Becomes More Expensive Than GPU Transfer

Part 1 - Exercise Contract and Why It Matters

Sweep worker count while pinning is enabled and identify the transition from input starvation to adequate overlap and finally to CPU/process overhead. The goal is not to maximize workers blindly.

Part 2 - System and Memory Mental Model

Workers parallelize loading and preprocessing; a DataLoader pinning stage then prepares host buffers for transfer. More workers can increase throughput until CPU cores, storage, shared memory, IPC, or pinning becomes saturated. Worker processes may duplicate Python-object memory and enlarge host pressure.

Part 3 - Quantitative Prediction Before Execution

Estimate CPU service demand per batch and available cores. Predict the worker count where preparation time falls below GPU consumption time. Also estimate resident prefetched batch bytes for every configuration.

Part 4 - Experimental Design and Instrumentation

Sweep $0, 1, 2, 4, 8$ up to a safe CPU-specific limit. When workers are zero, omit worker-only options such as persistent workers and prefetch factor. Measure data wait, H2D completion, compute, total throughput, CPU load, and pinned RAM.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch
from torch.utils.data import DataLoader

def make_loader(dataset, batch_size, worker_count):
    arguments = {"dataset": dataset, "batch_size": batch_size,
                 "shuffle": False, "num_workers": worker_count,
                 "pin_memory": True}
    if worker_count > 0:
        arguments["persistent_workers"] = True
        arguments["prefetch_factor"] = 2
    return DataLoader(**arguments)

def E12(dataset, batch_size=64, batches=100):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    rows = []
    for worker_count in (0, 1, 2, 4, 8):
        loader = make_loader(dataset, batch_size, worker_count)
        iterator = iter(loader)
        wait_seconds = 0.0
        transfer_seconds = 0.0
        for _ in range(batches):
            wait_start = time.perf_counter()
            batch = next(iterator)
            wait_seconds += time.perf_counter() - wait_start
            torch.cuda.synchronize(device)
            transfer_start = time.perf_counter()
            device_batch = batch.to(device, non_blocking=True)
            torch.cuda.synchronize(device)
            transfer_seconds += time.perf_counter() - transfer_start
        rows.append({"workers": worker_count,
                     "data_wait_s": wait_seconds,
                     "transfer_s": transfer_seconds,
                     "examples": batches * batch_size,
                     "check": float(device_batch.reshape(-1)[0].item())})
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

The staged timing above diagnoses service times but intentionally prevents overlap during transfer measurement; add an end-to-end run without per-batch synchronization before choosing production settings. Watch for exhausted shared memory, worker crashes, oversubscription, and dataset caching effects. Repeat worker order or randomize it.

Part 7 - Engineering Interpretation and Philosophy

Worker count is a queueing parameter. The best setting feeds the consumer with bounded headroom; additional producers that only increase contention and memory footprint are negative capacity.

19.7.14 Exercise 13

Exercise Title: Combine pin_memory, num_workers, prefetch_factor, and persistent_workers and Measure Their Interaction on End-to-End Throughput

Part 1 - Exercise Contract and Why It Matters

Run a controlled factorial study of the main DataLoader pipeline settings. Identify interactions instead of attributing a result to one flag in isolation.

Part 2 - System and Memory Mental Model

Workers determine producer concurrency; prefetch factor bounds queued batches per worker; persistent workers amortize process startup across epochs; pinning adds a page-locked handoff stage. Their effects interact. A larger queue can hide jitter but increases host memory, and persistence can help short epochs while retaining worker resources longer.

Part 3 - Quantitative Prediction Before Execution

For each configuration estimate queued payload as approximately $W \times F \times B$, where W is workers, F prefetch factor, and B batch bytes. Treat this as a planning estimate, not an exact guarantee. Rank predicted throughput and host-memory cost before execution.

Part 4 - Experimental Design and Instrumentation

Use a small declared matrix rather than an uncontrolled Cartesian explosion. Measure first-epoch and steady-state throughput separately. Record total and available RAM, pinned-batch bytes, CPU utilization, GPU idle time, and failures. Options invalid for zero workers must be omitted.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch
from torch.utils.data import DataLoader

def loader_from_config(dataset, batch_size, config):
    arguments = {"dataset": dataset, "batch_size": batch_size,
                 "shuffle": False, "num_workers": config["workers"],
                 "pin_memory": config["pin"]}
    if config["workers"] > 0:
        arguments["prefetch_factor"] = config["prefetch"]
        arguments["persistent_workers"] = config["persistent"]
    return DataLoader(**arguments)

def consume(loader, device, batches):
    iterator = iter(loader)
    start = time.perf_counter()
    examples = 0
```

```
    for _ in range(batches):
        batch = next(iterator)
        device_batch = batch.to(device, non_blocking=True)
        examples += device_batch.shape[0]
    torch.cuda.synchronize(device)
    seconds = time.perf_counter() - start
    return examples / seconds

def E13(dataset, batch_size=64, batches=100):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    configs = [
        {"workers": 0, "pin": False, "prefetch": None, "persistent": False},
        {"workers": 0, "pin": True, "prefetch": None, "persistent": False},
        {"workers": 2, "pin": True, "prefetch": 2, "persistent": False},
        {"workers": 2, "pin": True, "prefetch": 4, "persistent": True},
        {"workers": 4, "pin": True, "prefetch": 2, "persistent": True},
    ]
    rows = []
    for config in configs:
        loader = loader_from_config(dataset, batch_size, config)
        first = consume(loader, device, batches)
        steady = consume(loader, device, batches)
        rows.append({"config": dict(config), "first_examples_s": first,
                     "steady_examples_s": steady})
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Compare only runs with identical samples and transforms. If persistent workers change dataset state, add explicit reset logic. A throughput gain that consumes unacceptable RAM is not a free improvement. Include confidence intervals and repeat after cold start because filesystem caching can dominate early runs.

Part 7 - Engineering Interpretation and Philosophy

Pipeline settings form a coupled system. Select a configuration on the Pareto frontier of throughput, latency, RAM, determinism, and operational stability rather than maximizing one benchmark column.

19.7.15 Exercise 14

Exercise Title: Saturate Pinned Host Memory Deliberately and Measure the Impact on System RAM Availability and Overall Training Stability

Part 1 - Exercise Contract and Why It Matters

Perform a bounded pressure experiment that approaches a declared pinned-memory budget without exhausting the machine. Measure allocation latency, available RAM, transfer performance, and a training canary. Uncontrolled host-memory exhaustion is explicitly outside the exercise contract.

Part 2 - System and Memory Mental Model

Pinned pages cannot be reclaimed through ordinary paging while locked. Excessive pinning reduces OS flexibility, can slow allocations, starve other processes, and destabilize workers even when GPU VRAM is plentiful. The limit is a system property involving physical RAM, OS policy, driver, and concurrent workloads.

Part 3 - Quantitative Prediction Before Execution

Read available RAM and preserve the greater of 8 GiB or 40 percent as a safety reserve. Also impose a conservative experiment cap such as 4 GiB. Predict the maximum number of 64 MiB buffers under both constraints. Stop earlier if available memory, allocation latency, or canary throughput crosses a declared threshold.

Part 4 - Experimental Design and Instrumentation

Run on a non-production machine with no critical co-tenants. Allocate in small steps, touch each buffer, measure after every step, and release in `finally`. Never disable OS safeguards, never allocate from an unbounded loop, and never interpret swap activity or worker death as a desired endpoint.

Part 5 - PyTorch/CUDA Implementation

```
import gc
import os
import time
import torch

def available_ram_bytes():
    page_size = os.sysconf("SC_PAGE_SIZE")
    available_pages = os.sysconf("SC_AVPHYS_PAGES")
    return int(page_size * available_pages)

def E14(step_mib=64, hard_cap_mib=4096, canary=None):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    initial_available = available_ram_bytes()
    reserve = max(8 * 1024 ** 3, int(initial_available * 0.40))
    hard_cap = hard_cap_mib * 1024 ** 2
    step_bytes = step_mib * 1024 ** 2
    buffers = []
    rows = []
    pinned_total = 0
    try:
        while pinned_total + step_bytes <= hard_cap:
            before = available_ram_bytes()
            if before - step_bytes < reserve:
                break
            start = time.perf_counter()
            numel = step_bytes // 4
            buffer = torch.empty(numel, dtype=torch.float32, pin_memory=True)
```

```
            buffer.fill_(1.0)
            allocation_ms = (time.perf_counter() - start) * 1000.0
            buffers.append(buffer)
            pinned_total += buffer.nbytes
            after = available_ram_bytes()
            canary_metric = None
            if canary is not None:
                canary_metric = float(canary())
            rows.append({"pinned_bytes": pinned_total,
                         "available_before": before,
                         "available_after": after,
                         "allocation_ms": allocation_ms,
                         "canary_metric": canary_metric})
            if allocation_ms > 1000.0:
                break
    finally:
        buffers.clear()
        gc.collect()
    return {"initial_available": initial_available, "reserve": reserve,
            "hard_cap": hard_cap, "rows": rows,
            "final_available": available_ram_bytes()}
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify every buffer is pinned and that cleanup restores capacity over time. Available RAM may not rebound byte-for-byte because OS accounting and caches differ. Stop on rising allocation latency, canary degradation, or any allocation error. Do not retry aggressively. Report the bounded range actually tested, never call it the machine's absolute maximum.

Part 7 - Engineering Interpretation and Philosophy

Pinned RAM is shared infrastructure. Responsible optimization includes a hard budget, reserve, backpressure, cleanup, and graceful fallback. The fastest GPU pipeline is a failure if it makes the host unreliable.

19.7.16 Exercise 15

Exercise Title: Build a Reusable Pinned Host Buffer and Refill It Across Iterations Instead of Allocating New Pinned Tensors Every Batch

Part 1 - Exercise Contract and Why It Matters

Build a bounded reusable pinned-buffer object, refill it across iterations, and prove that no buffer is overwritten while an asynchronous H2D copy still reads from it. This converts pinning from repeated allocation into managed capacity.

Part 2 - System and Memory Mental Model

A pinned buffer has two lifetimes: Python ownership and DMA-use lifetime. The host may still own an object whose contents are unsafe to overwrite because a copy is in flight. A CUDA event recorded after the copy is the reuse fence. A two-slot ring allows the CPU to prepare one slot while the GPU consumes another, provided event and stream dependencies are correct.

Part 3 - Quantitative Prediction Before Execution

Compare fresh page-lock cost per iteration with one-time pool construction. For N iterations and K slots, predict pinned capacity KB and amortized allocation cost KP/N. Draw a timeline of fill, copy, compute, and reuse for two slots before writing code.

Part 4 - Experimental Design and Instrumentation

Use deterministic batches containing iteration identifiers. Before refilling a slot, synchronize only that slot's completion event. Submit copies on a copy stream, make the compute stream wait on the event, and validate device values. Track data pointers to prove storage reuse.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class PinnedSlot:
    def __init__(self, shape, dtype):
        self.host = torch.empty(shape, dtype=dtype, pin_memory=True)
        self.copy_done = None

    def wait_until_reusable(self):
        if self.copy_done is not None:
            self.copy_done.synchronize()

    def refill(self, source):
        self.wait_until_reusable()
        if source.shape != self.host.shape or source.dtype != self.host.dtype:
            raise ValueError("Source does not match reusable slot")
        self.host.copy_(source)

    def submit(self, device, copy_stream):
        event = torch.cuda.Event()
        with torch.cuda.stream(copy_stream):
            device_tensor = self.host.to(device, non_blocking=True)
            event.record(copy_stream)
        self.copy_done = event
        return device_tensor, event

def E15(iterations=20, shape=(64, 3, 224, 224)):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    copy_stream = torch.cuda.Stream(device=device)
    compute_stream = torch.cuda.current_stream(device)
    slots = [PinnedSlot(shape, torch.float32),
             PinnedSlot(shape, torch.float32)]
    pointers = []
    checks = []
    for step in range(iterations):
```

```
        slot = slots[step % len(slots)]
        source = torch.full(shape, float(step), dtype=torch.float32)
        slot.refill(source)
        device_batch, ready = slot.submit(device, copy_stream)
        compute_stream.wait_event(ready)
        with torch.cuda.stream(compute_stream):
            checks.append(device_batch[0, 0, 0, 0] + 0.0)
        pointers.append(slot.host.data_ptr())
    torch.cuda.synchronize(device)
    values = [float(value.item()) for value in checks]
    return {"unique_host_pointers": len(set(pointers)),
            "slot_count": len(slots), "values": values}
```

Part 6 - Measurement, Validation, and Failure Analysis

Require the number of unique host pointers to equal slot count and device values to equal iteration identifiers. Remove the per-slot wait deliberately in a controlled test to observe corruption risk, then restore it. If overlap is absent, inspect stream capability and whether CPU refill itself dominates.

Part 7 - Engineering Interpretation and Philosophy

Reuse changes memory optimization into a lifetime protocol. Capacity, ownership, and completion must be explicit. Events are not incidental synchronization; they are the proof that mutating a shared resource is safe.

19.7.17 Exercise 16

Exercise Title: Compare Fresh Pinned Allocations Against Reused Pinned Buffers and Quantify Allocation Overhead and Transfer Throughput

Part 1 - Exercise Contract and Why It Matters

Benchmark fresh page-locked allocation per batch against a fixed reusable pool. Quantify allocation overhead, refill cost, H2D throughput, total iteration time, and peak pinned capacity.

Part 2 - System and Memory Mental Model

Fresh allocation repeatedly asks the runtime and operating system to establish page-locked storage. Reuse pays this cost during pool construction, but still copies producer data into the slot unless data is generated directly there. The correct comparison includes equal useful work and explicit cleanup.

Part 3 - Quantitative Prediction Before Execution

Predict total fresh cost $N(P + F + C)$ and reuse cost $KP + N(F + C)$, where P is pinned allocation, F refill, C transfer, and K pool size. Solve the break-even iteration count.

Part 4 - Experimental Design and Instrumentation

Pre-generate identical pageable batches. Use synchronized end-to-end timing for both policies and phase timers for allocation/refill/copy. Randomize policy order across complete repetitions and report host-memory budget.

Part 5 - PyTorch/CUDA Implementation

```
import statistics
import time
import torch

def run_policy(batches, device, reuse):
    allocation_ms = 0.0
    refill_ms = 0.0
    copy_ms = 0.0
    slot = None
    if reuse:
        start = time.perf_counter()
        slot = torch.empty_like(batches[0], pin_memory=True)
        allocation_ms += (time.perf_counter() - start) * 1000.0
    for batch in batches:
        if not reuse:
            start = time.perf_counter()
            slot = torch.empty_like(batch, pin_memory=True)
            allocation_ms += (time.perf_counter() - start) * 1000.0
        start = time.perf_counter()
        slot.copy_(batch)
        refill_ms += (time.perf_counter() - start) * 1000.0
        torch.cuda.synchronize(device)
        start = time.perf_counter()
        destination = slot.to(device, non_blocking=True)
        torch.cuda.synchronize(device)
        copy_ms += (time.perf_counter() - start) * 1000.0
    total_ms = allocation_ms + refill_ms + copy_ms
    return {"allocation_ms": allocation_ms,
            "refill_ms": refill_ms,
            "copy_ms": copy_ms, "total_ms": total_ms,
            "checksum": float(destination.float().sum().item())}

def E16(iterations=30, shape=(16, 3, 224, 224)):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    batches = []
    for index in range(iterations):
        batches.append(torch.full(shape, float(index), dtype=torch.float32))
    return {"fresh": run_policy(batches, device, False),
            "reused": run_policy(batches, device, True)}
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify equal checksums and equal batch sequences. The single-slot reused path synchronizes before refill and therefore measures safe serial reuse; compare a separate double-buffer overlap design without mixing conclusions. Fresh pinned allocation may be affected by runtime caching, so repeat from clean processes.

Part 7 - Engineering Interpretation and Philosophy

Reuse is valuable because it transforms variable control-plane cost into fixed capacity. Its benefit must be weighed against buffer residency, shape policy, and the synchronization discipline required to avoid corruption.

19.7.18 Exercise 17

Exercise Title: Create a Large Multi-Tensor Batch and Determine Which Tensors Should Be Pinned Based on Transfer Frequency, Size, and Lifetime

Part 1 - Exercise Contract and Why It Matters

Create a heterogeneous batch and select pinned fields under a strict host-memory budget. Base the policy on measured savings, transfer frequency, size, lifetime, and reuse rather than pinning every tensor automatically.

Part 2 - System and Memory Mental Model

The value of pinning field i scales with how often it crosses the bus and how much critical-path time is saved. Its cost scales with resident pinned bytes, allocation/conversion work, and lifetime. Tiny metadata may not repay fixed overhead; large infrequently transferred caches may consume capacity without improving iteration time.

Part 3 - Quantitative Prediction Before Execution

Define a measured benefit score

$$V_i = f_i \max(0, T_{pageable,i} - T_{pinned,i})$$

and efficiency V_i/R_i, where f_i is transfer frequency and R_i resident pinned bytes. Predict selections under a fixed budget, while treating mandatory latency-sensitive fields as explicit constraints.

Part 4 - Experimental Design and Instrumentation

Inventory every tensor: semantic role, bytes, frequency, lifetime, device destination, contiguity, and measured latency. Benchmark fields independently, then validate the chosen combination end to end because transfers can contend.

Part 5 - PyTorch/CUDA Implementation

```
def select_fields(field_rows, budget_bytes):
    candidates = []
    for row in field_rows:
        saved_ms = max(0.0, row["pageable_ms"] - row["pinned_ms"])
        value = row["transfers_per_step"] * saved_ms
        resident = row["resident_bytes"]
        efficiency = value / resident if resident > 0 else 0.0
        candidate = dict(row)
        candidate["saved_ms_per_step"] = value
        candidate["efficiency"] = efficiency
        candidates.append(candidate)
    candidates.sort(key=lambda item: item["efficiency"], reverse=True)
```

```
    selected = []
    used = 0
    for candidate in candidates:
        if candidate.get("must_pin", False):
            if used + candidate["resident_bytes"] > budget_bytes:
                raise RuntimeError("Mandatory fields exceed pinned budget")
            selected.append(candidate)
            used += candidate["resident_bytes"]
    for candidate in candidates:
        if candidate in selected:
            continue
        if candidate["saved_ms_per_step"] <= 0.0:
            continue
        proposed = used + candidate["resident_bytes"]
        if proposed <= budget_bytes:
            selected.append(candidate)
            used = proposed
    return selected, used

def E17(measured_field_rows, budget_mib=512):
    budget_bytes = budget_mib * 1024 ** 2
    selected, used = select_fields(measured_field_rows, budget_bytes)
    names = []
    for row in selected:
        names.append(row["name"])
    return {"selected": names, "used_bytes": used,
            "budget_bytes": budget_bytes}
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate that selected capacity never exceeds budget and remeasure the full pipeline. The greedy policy is transparent but not globally optimal; compare it with alternate combinations when interactions matter. Include queue depth in resident bytes, not merely one tensor's size.

Part 7 - Engineering Interpretation and Philosophy

Pinning is resource allocation under scarcity. A defensible policy makes value, cost, lifetime, and constraints explicit, allowing the system to degrade gracefully when host capacity changes.

19.7.19 Exercise 18

Exercise Title: Profile a Real PyTorch Training Pipeline and Identify Whether Pageable Memory, Pinning Overhead, CPU Preprocessing, or GPU Compute Is the Bottleneck

Part 1 - Exercise Contract and Why It Matters

Instrument a real training step and classify its dominant bottleneck using multiple independent measurements. Produce evidence for pageable staging, pinning overhead, CPU preprocessing, H2D transfer, GPU compute, and idle gaps.

Part 2 - System and Memory Mental Model

Pipeline throughput emerges from stages and queues. Long `next(loader)` time suggests producer starvation; expensive manual pinning suggests misplaced conversion; long copy events suggest transfer pressure; gaps before GPU kernels suggest scheduling or input starvation; continuously busy compute kernels suggest a compute-bound model. No single counter identifies all stages.

Part 3 - Quantitative Prediction Before Execution

Before profiling, estimate batch bytes, ideal H2D time, CPU transform time, and model compute time. Predict a bottleneck and write a falsifying observation for that hypothesis.

Part 4 - Experimental Design and Instrumentation

Combine explicit stage timers with `torch.profiler` regions, CPU and CUDA activities, shape recording, and a controlled pinning ablation. Profile only active steps after warmup because tracing changes timing. Preserve a baseline run without profiler to quantify overhead.

Part 5 - PyTorch/CUDA Implementation

```
import torch
from torch.profiler import ProfilerActivity, record_function

def E18(loader, model, optimizer, loss_function, steps=10):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    model = model.to(device)
    iterator = iter(loader)
    observed_pinned_states = []
    activities = [ProfilerActivity.CPU, ProfilerActivity.CUDA]
    with torch.profiler.profile(activities=activities,
                                record_shapes=True,
                                profile_memory=True) as profiler:
        for _ in range(steps):
            with record_function("input_wait"):
                inputs, labels = next(iterator)
                observed_pinned_states.append(
                    bool(inputs.is_pinned() and labels.is_pinned()))
            with record_function("host_to_device"):
                inputs = inputs.to(device, non_blocking=True)
                labels = labels.to(device, non_blocking=True)
            with record_function("forward_loss"):
                outputs = model(inputs)
                loss = loss_function(outputs, labels)
            with record_function("backward_optimizer"):
                optimizer.zero_grad(set_to_none=True)
                loss.backward()
                optimizer.step()
            profiler.step()
    table = profiler.key_averages().table(
        sort_by="self_cuda_time_total", row_limit=30)
```

```
    return {"table": table, "last_loss": float(loss.detach().item()),
            "all_batches_were_pinned": all(observed_pinned_states)}
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not equate DataLoader's pinning flag with verified tensor state; inspect `is_pinned()` in the batch. Compare profiler regions with baseline wall time and CUDA events. If removing pinning improves time, test whether pinning cost, tiny batches, or host pressure explains it. If GPU utilization is low, separate waiting from synchronization and CPU dispatch gaps.

Part 7 - Engineering Interpretation and Philosophy

Profiling is hypothesis testing, not screenshot collection. Optimize the stage that constrains throughput, then rerun the whole system because removing one bottleneck exposes another.

19.7.20 Exercise 19

Exercise Title: Design and Benchmark a Memory-Efficient Pinned-Buffer Pipeline That Maximizes Host-to-GPU Throughput Without Excessive Host RAM Consumption

Part 1 - Exercise Contract and Why It Matters

Integrate the chapter into a production-shaped design: bounded pinned slots, copy/compute overlap, event-governed reuse, host-RAM budgeting, measurement, validation, and graceful fallback. Defend the final policy with before-and-after evidence rather than maximum microbenchmark bandwidth alone.

Part 2 - System and Memory Mental Model

The pipeline is a bounded producer–consumer system. CPU producers fill pinned slots; a copy stream moves them; a compute stream waits only for the appropriate copy event; completion events protect slots from premature refill. Backpressure occurs when no slot is safe. Capacity equals slot count times maximum slot bytes, plus DataLoader and application queues. Variable shapes require size classes, padding, or a controlled pageable fallback rather than unbounded reallocation.

Part 3 - Quantitative Prediction Before Execution

Choose a pinned-RAM budget from available memory and operational reserve. For slot size B and count K, predict resident capacity KB, copy time, compute time, and ideal overlapped iteration time $\max(T_{fill}, T_{copy}, T_{compute})$. Compare one, two, and three slots; more slots help only when they cover real pipeline latency.

Part 4 - Experimental Design and Instrumentation

Benchmark pageable baseline, fresh pinning, single-slot reuse, and double-buffer overlap. Use identical batches and outputs. Record throughput, median/p95 latency, effective bandwidth, available RAM, slot waits, GPU idle gaps, and errors. Stress expected shape variation and inject an oversized batch to verify bounded fallback.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class BoundedPinnedPipeline:
    def __init__(self, shape, dtype, slots, max_pinned_bytes, device):
        bytes_per_slot = torch.empty((), dtype=dtype).element_size()
        for dimension in shape:
            bytes_per_slot *= dimension
        required = slots * bytes_per_slot
        if required > max_pinned_bytes:
            raise MemoryError("Pinned pool exceeds configured budget")
        self.device = device
        self.copy_stream = torch.cuda.Stream(device=device)
        self.compute_stream = torch.cuda.current_stream(device)
        self.host_slots = []
        self.done_events = []
        for _ in range(slots):
            host = torch.empty(shape, dtype=dtype, pin_memory=True)
            self.host_slots.append(host)
            self.done_events.append(None)
        self.next_slot = 0

    def stage(self, source):
        index = self.next_slot
        event = self.done_events[index]
        if event is not None:
            event.synchronize()
        host = self.host_slots[index]
        if source.shape != host.shape or source.dtype != host.dtype:
            destination = source.to(self.device)
            return destination, None, "bounded_fallback"
        host.copy_(source)
        ready = torch.cuda.Event()
        with torch.cuda.stream(self.copy_stream):
            destination = host.to(self.device, non_blocking=True)
            ready.record(self.copy_stream)
        self.compute_stream.wait_event(ready)
        self.done_events[index] = ready
        self.next_slot = (index + 1) % len(self.host_slots)
        return destination, ready, "pinned_pool"

def E19(cpu_batches, model, max_pinned_mib=1024):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    if len(cpu_batches) == 0:
        raise ValueError("At least one batch is required")
    device = torch.device("cuda:0")
    model = model.to(device)
    first = cpu_batches[0]
```

```
    pipeline = BoundedPinnedPipeline(
        first.shape, first.dtype, slots=2,
        max_pinned_bytes=max_pinned_mib * 1024 ** 2, device=device)
    outputs = []
    paths = []
    for batch in cpu_batches:
        device_batch, ready, path = pipeline.stage(batch)
        with torch.cuda.stream(pipeline.compute_stream):
            output = model(device_batch)
            outputs.append(output)
        paths.append(path)
    torch.cuda.synchronize(device)
    checksums = []
    for output in outputs:
        checksums.append(float(output.float().sum().item()))
    return {"paths": paths, "checksums": checksums,
            "pinned_slots": len(pipeline.host_slots)}
```

Part 6 - Measurement, Validation, and Failure Analysis

Compare outputs with the pageable baseline within declared tolerances. Confirm pool bytes never exceed budget, slot pointers remain stable, and each slot waits before refill. The sample fallback is synchronous and correctness-oriented; production code should log it, apply backpressure, or route to an approved size class. Test cancellation and exceptions so events and buffers are not abandoned in ambiguous states.

Part 7 - Engineering Interpretation and Philosophy

Maximum bandwidth is not the final objective. The engineering objective is predictable useful throughput under finite host memory, variable demand, and failure. A mature pinned-buffer pipeline owns its capacity, proves its lifetime rules, exposes backpressure, validates correctness, and can choose a slower safe path instead of destabilizing the machine.

19.8 DMM06

Chapter Title: Non-Blocking CPU-GPU Transfers
Specialization: Dynamic Memory Management and CUDA Systems

19.8.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Synchronous and Asynchronous Tensor Transfers Between CPU and CUDA Devices

Part 1 - Exercise Contract and Why It Matters

Build a model that separates API return, CUDA-stream completion, data visibility, source lifetime, and destination consumption. The deliverable is a transfer timeline for H2D, D2H, and D2D copies. For each direction, state who may proceed, which storage may be changed, and what synchronization proves completion. This precision is essential because asynchronous transfer bugs often return plausible values rather than raising exceptions.

Part 2 - System and Memory Mental Model

CUDA work is submitted to streams and normally executes asynchronously with respect to the host. A blocking PyTorch transfer introduces host-visible waiting before the call returns. A non-blocking request attempts to enqueue the copy and return earlier; it does not make the bytes instantaneous. Stream ordering protects operations on the same stream, while work on different streams needs explicit dependencies.

For H2D, page-locked source memory enables direct asynchronous DMA. That source must remain alive and unmodified until copy completion. For D2H, the CPU destination cannot be trusted until an event, stream, or device synchronization completes. For D2D, CUDA streams protect device-side consumers when ordering is correct; cross-stream use requires a wait event. The `non_blocking` argument is therefore only one part of a lifetime and visibility protocol.

Part 3 - Quantitative Prediction Before Execution

For B bytes and effective bandwidth R, predict device transfer time $T_{copy} = B/R$. Add host submission cost T_s and any wait T_w. Predict separate values for API-call latency T_s, completed wall latency $T_s + T_w$, and useful-overlap time. Draw two timelines: one with a host wait after every copy and one with several submissions followed by one completion barrier.

Part 4 - Experimental Design and Instrumentation

Use preallocated tensors, warm CUDA, fixed device and stream, and deterministic values. Measure host submission with `perf_counter`, device duration with CUDA events, and completed wall time with synchronization. Keep sources alive until validation. Treat pageable and pinned H2D sources as different paths, and never infer readiness merely because Python regained control.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

def E00(numel=16 * 1024 * 1024):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required for DMM06.")
    device = torch.device("cuda:0")
    source = torch.arange(numel, dtype=torch.float32, pin_memory=True)
    stream = torch.cuda.Stream(device=device)
    torch.empty(1, device=device)
    torch.cuda.synchronize(device)

    start_event = torch.cuda.Event(enable_timing=True)
    end_event = torch.cuda.Event(enable_timing=True)
    host_start = time.perf_counter()
    with torch.cuda.stream(stream):
        start_event.record(stream)
        destination = source.to(device, non_blocking=True)
        end_event.record(stream)
    host_return = time.perf_counter()
    end_event.synchronize()
```

```
    completed = time.perf_counter()

    expected = float(source[-1].item())
    observed = float(destination[-1].item())
    return {
        "bytes": source.nbytes,
        "host_submission_ms": (host_return - host_start) * 1000.0,
        "completed_wall_ms": (completed - host_start) * 1000.0,
        "device_copy_ms": start_event.elapsed_time(end_event),
        "source_pinned": source.is_pinned(),
        "validated": observed == expected,
    }
```

Part 6 - Measurement, Validation, and Failure Analysis

The submission time should not be reported as transfer latency. Confirm that completed wall time includes the event wait and that event timing covers only stream work between records. If the three times are similar, the transfer may be too small or submission may be serialized by earlier work. If validation fails, inspect source mutation and lifetime before blaming numerical precision.

Part 7 - Engineering Interpretation and Philosophy

Asynchrony transfers responsibility from the API to the programmer. Removing a wait can expose overlap, but it also creates an obligation to prove readiness and preserve lifetimes. High performance comes from explicit temporal ownership, not from adding a Boolean flag.

19.8.2 Exercise 01

Exercise Title: Compare tensor.to("cuda") and tensor.to("cuda", non_blocking=True) and Identify Where Host Synchronization Changes

Part 1 - Exercise Contract and Why It Matters

Compare blocking and non-blocking `to()` calls with identical pinned sources. Locate the exact boundary where host behavior changes, then prove that both paths still require completed data before validation.

Part 2 - System and Memory Mental Model

A blocking call performs or waits for the synchronization needed before it returns. A non-blocking call can return after enqueueing the copy. The latter allows the host to submit more work, but destination readiness remains governed by the CUDA stream. A later operation that forces a result back to the host may implicitly reveal the pending wait and hide it in an unexpected line.

Part 3 - Quantitative Prediction Before Execution

Predict API-call time and completed time for both modes. The non-blocking call should reduce host-call time more than completed copy time. Predict where the saved wait reappears when `torch.cuda.synchronize()` is executed.

Part 4 - Experimental Design and Instrumentation

Preallocate one pinned source, alternate modes, synchronize before each trial, and measure return and completion timestamps. Do not regenerate or repin data inside the timed region. Validate byte count and tensor values.

Part 5 - PyTorch/CUDA Implementation

```
import statistics
import time
import torch

def measure_to(source, device, non_blocking, repeats):
    call_samples = []
    complete_samples = []
    for _ in range(repeats):
        torch.cuda.synchronize(device)
        start = time.perf_counter()
        destination = source.to(device, non_blocking=non_blocking)
        returned = time.perf_counter()
        torch.cuda.synchronize(device)
        completed = time.perf_counter()
        call_samples.append((returned - start) * 1000.0)
        complete_samples.append((completed - start) * 1000.0)
    return {
        "call_median_ms": statistics.median(call_samples),
        "complete_median_ms": statistics.median(complete_samples),
        "checksum": float(destination.float().sum().item()),
    }

def E01(numel=8 * 1024 * 1024, repeats=30):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    source = torch.ones(numel, dtype=torch.float32, pin_memory=True)
    blocking = measure_to(source, device, False, repeats)
    non_blocking = measure_to(source, device, True, repeats)
    return {"blocking": blocking, "non_blocking": non_blocking}
```

Part 6 - Measurement, Validation, and Failure Analysis

Require equal checksums and completion semantics. If blocking call time appears short, check whether PyTorch deferred synchronization to a later operation or whether the platform path differs. If non-blocking completion is faster, do not attribute all gain to asynchrony without randomizing order and measuring clocks, cache state, and background load.

Part 7 - Engineering Interpretation and Philosophy

Non-blocking execution does not delete work; it relocates waiting and allows overlap. Trace the wait through the entire dependency chain before declaring a speedup.

19.8.3 Exercise 02

Exercise Title: Measure CPU Submission Time Separately from Actual GPU Transfer Completion Time for Blocking and Non-Blocking Copies

Part 1 - Exercise Contract and Why It Matters

Build two independent clocks for one copy: a CPU clock around submission and CUDA events around device execution. Add a completion clock that waits for the event. Explain why these metrics answer different engineering questions.

Part 2 - System and Memory Mental Model

The CPU clock measures Python, dispatcher, driver submission, and any host wait. CUDA events measure elapsed work on a device timeline. Event synchronization connects those timelines and proves completion. Event creation and recording also have overhead, so they should be reused or measured when precision matters.

Part 3 - Quantitative Prediction Before Execution

For a 128 MiB pinned tensor, predict submission time, event duration, and wall completion. State which should scale strongly with bytes and which should remain approximately fixed until queue pressure appears.

Part 4 - Experimental Design and Instrumentation

Warm up, reset no unrelated state, record events on the same stream as the copy, and synchronize the ending event rather than the entire device when possible. Report medians and tail latency from repeated trials.

Part 5 - PyTorch/CUDA Implementation

```
import statistics
import time
import torch

def one_trial(source, device, stream, non_blocking):
    torch.cuda.synchronize(device)
    start_event = torch.cuda.Event(enable_timing=True)
    end_event = torch.cuda.Event(enable_timing=True)
    wall_start = time.perf_counter()
    with torch.cuda.stream(stream):
        start_event.record(stream)
        destination = source.to(device, non_blocking=non_blocking)
        end_event.record(stream)
    host_return = time.perf_counter()
    end_event.synchronize()
    wall_end = time.perf_counter()
    return ((host_return - wall_start) * 1000.0,
            start_event.elapsed_time(end_event),
            (wall_end - wall_start) * 1000.0,
            destination)
```

```
def E02(numel=32 * 1024 * 1024, repeats=25):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    stream = torch.cuda.Stream(device=device)
    source = torch.ones(numel, dtype=torch.float32, pin_memory=True)
    rows = {}
    for mode in (False, True):
        submission = []
        device_time = []
        completed = []
        for _ in range(repeats):
            cpu_ms, gpu_ms, wall_ms, destination = one_trial(
                source, device, stream, mode)
            submission.append(cpu_ms)
            device_time.append(gpu_ms)
            completed.append(wall_ms)
        rows[str(mode)] = {
            "submission_median_ms": statistics.median(submission),
            "device_median_ms": statistics.median(device_time),
            "completed_median_ms": statistics.median(completed),
            "check": float(destination[0].item()),
        }
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

The end event must be recorded after the copy on the same stream. A device event does not include CPU preparation before submission. If submission time grows with queued operations, measure queue saturation separately. Do not compare an event duration in milliseconds with an unsynchronized Python duration and call the mismatch an error.

Part 7 - Engineering Interpretation and Philosophy

Latency has multiple observers. The host cares when it can do more work; the consumer cares when bytes are ready; the system owner cares about end-to-end throughput. Good instrumentation preserves all three perspectives.

19.8.4 Exercise 03

Exercise Title: Use torch.cuda.synchronize() to Prove Why Timing an Asynchronous Transfer Without Synchronization Produces Misleading Results

Part 1 - Exercise Contract and Why It Matters

Create an intentionally wrong benchmark and a corrected benchmark. Show that the wrong version mostly measures enqueue time, then locate the missing transfer cost with an explicit synchronization boundary.

Part 2 - System and Memory Mental Model

Python proceeds after asynchronous submission while the GPU continues copying. The CPU timestamp taken immediately after `to()` therefore does not observe completion. Synchronization blocks until prior work covered by its scope finishes. Device-wide synchronization is useful for proof but may be broader than production requires; an event or stream wait can provide narrower control.

Part 3 - Quantitative Prediction Before Execution

Predict an apparently impossible bandwidth from unsynchronized timing. Then predict a plausible value after dividing the same bytes by synchronized elapsed time. Explain why the first calculation violates the physical link limit.

Part 4 - Experimental Design and Instrumentation

Use a large pre-pinned tensor, warmup, and no unrelated GPU work. Run the wrong and correct measurements in alternating order. Retain the destination and validate it after completion.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

def E03(numel=64 * 1024 * 1024):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    source = torch.ones(numel, dtype=torch.float32, pin_memory=True)
    warmup = source.to(device, non_blocking=True)
    torch.cuda.synchronize(device)

    wrong_start = time.perf_counter()
    wrong_destination = source.to(device, non_blocking=True)
    wrong_seconds = time.perf_counter() - wrong_start
    torch.cuda.synchronize(device)

    torch.cuda.synchronize(device)
    correct_start = time.perf_counter()
    correct_destination = source.to(device, non_blocking=True)
    torch.cuda.synchronize(device)
    correct_seconds = time.perf_counter() - correct_start

    wrong_bandwidth = source.nbytes / wrong_seconds / 1e9
    correct_bandwidth = source.nbytes / correct_seconds / 1e9
    return {
        "wrong_submission_only_ms": wrong_seconds * 1000.0,
        "wrong_apparent_GB_s": wrong_bandwidth,
        "correct_completed_ms": correct_seconds * 1000.0,
        "correct_GB_s": correct_bandwidth,
        "validated": torch.equal(wrong_destination, correct_destination),
    }
```

Part 6 - Measurement, Validation, and Failure Analysis

The synchronization after the wrong timer prevents its work from contaminating the correct trial but does not repair the wrong measurement. If the apparent bandwidth is not absurd, increase tensor size or repetitions. Device-wide waits include all preceding work on the device, so isolate the benchmark or use events for operation-specific timing.

Part 7 - Engineering Interpretation and Philosophy

A fast timestamp is not evidence of fast work when the work is still running. Every asynchronous benchmark must name its completion condition; otherwise it measures permission to proceed rather than delivered data.

19.8.5 Exercise 04

Exercise Title: Benchmark Blocking and Non-Blocking Host-to-Device Transfers Across Increasing Tensor Sizes

Part 1 - Exercise Contract and Why It Matters

Build a logarithmic size sweep that compares blocking and non-blocking H2D behavior using both host-return and completed-transfer metrics. Identify the size regions dominated by fixed overhead and bandwidth.

Part 2 - System and Memory Mental Model

Small copies expose dispatch and synchronization overhead. Large copies expose interconnect throughput. Non-blocking submission changes host latency most clearly when the copy itself is long enough to remain in flight after return. Completed transfer duration may remain similar because the same bytes cross the same link.

Part 3 - Quantitative Prediction Before Execution

For sizes from 1 KiB to 256 MiB, predict $T = \alpha + B/R$. Use different host-wait terms for blocking and non-blocking calls but the same first-order bandwidth. Mark the expected size where submission and completion diverge.

Part 4 - Experimental Design and Instrumentation

Preallocate pinned sources before timing, warm each size, alternate modes, and use more repetitions for small sizes. Record median submission, median completed wall time, p95, and effective GB/s. Cap the maximum size to safe host and VRAM budgets on the test machine.

Part 5 - PyTorch/CUDA Implementation

```
import statistics
import time
import torch

def percentile(samples, fraction):
    ordered = sorted(samples)
    index = round((len(ordered) - 1) * fraction)
    return ordered[index]
```

```
def E04(repeats=20):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    byte_sizes = []
    for exponent in range(10, 29, 2):
        byte_sizes.append(2 ** exponent)
    rows = []
    for requested_bytes in byte_sizes:
        numel = max(1, requested_bytes // 4)
        source = torch.ones(numel, dtype=torch.float32, pin_memory=True)
        for mode in (False, True):
            submission = []
            completed = []
            for _ in range(repeats):
                torch.cuda.synchronize(device)
                start = time.perf_counter()
                destination = source.to(device, non_blocking=mode)
                returned = time.perf_counter()
                torch.cuda.synchronize(device)
                end = time.perf_counter()
                submission.append((returned - start) * 1000.0)
                completed.append((end - start) * 1000.0)
            median_ms = statistics.median(completed)
            rows.append({
                "bytes": source.nbytes, "non_blocking": mode,
                "submission_median_ms": statistics.median(submission),
                "completed_median_ms": median_ms,
                "completed_p95_ms": percentile(completed, 0.95),
                "GB_s": source.nbytes / (median_ms / 1000.0) / 1e9,
                "check": float(destination[0].item()),
            })
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Plot submission and completion curves separately. Verify actual byte counts because integer division can alter requested sizes. If blocking and non-blocking completion differ substantially, repeat complete randomized runs and inspect power state, background traffic, and synchronization scope before concluding that the transport path itself changed.

Part 7 - Engineering Interpretation and Philosophy

Size sweeps prevent one tensor shape from becoming a universal recommendation. Choose non-blocking policy from the workload's distribution and overlap opportunity, not from one favorable point.

19.8.6 Exercise 05

Exercise Title: Compare Non-Blocking Transfers from Pageable and Pinned Host Memory and Explain the Different Transfer Paths

Part 1 - Exercise Contract and Why It Matters

Compare non-blocking H2D submission from pageable and pre-pinned sources. Explain why the same API flag does not guarantee the same physical path, lifetime rule, or overlap potential.

Part 2 - System and Memory Mental Model

DMA requires stable physical host pages. A pinned source supplies them directly. A pageable source may be staged through an internal pinned region; staging can introduce a CPU copy or synchronization before true DMA proceeds. PyTorch may still accept `non_blocking=True`, but full host–device overlap is most reliably designed with already pinned source storage. Because pinned memory can be read by DMA after the call returns, changing it early creates a real race.

Part 3 - Quantitative Prediction Before Execution

Model pageable cost as staging plus DMA and pinned cost as DMA plus fixed submission. Predict submission, completion, and end-to-end pin-then-copy cost for 4 KiB, 4 MiB, and 128 MiB. Keep pre-pinned and on-demand pinning predictions separate.

Part 4 - Experimental Design and Instrumentation

Construct value-equivalent sources outside timing. Verify `is_pinned()`, alternate paths, and measure host return, CUDA-event duration, and completion. Do not mutate either source until its copy is complete.

Part 5 - PyTorch/CUDA Implementation

```
import statistics
import time
import torch

def sample_path(source, device, repeats):
    submission = []
    completion = []
    for _ in range(repeats):
        torch.cuda.synchronize(device)
        start = time.perf_counter()
        destination = source.to(device, non_blocking=True)
        returned = time.perf_counter()
        torch.cuda.synchronize(device)
        finished = time.perf_counter()
        submission.append((returned - start) * 1000.0)
        completion.append((finished - start) * 1000.0)
    return {"submission_ms": statistics.median(submission),
            "completion_ms": statistics.median(completion),
            "checksum": float(destination.float().sum().item())}

def E05(numel=16 * 1024 * 1024, repeats=25):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
```

```
    pageable = torch.arange(numel, dtype=torch.float32)
    pinned = torch.empty(numel, dtype=torch.float32, pin_memory=True)
    pinned.copy_(pageable)
    return {
        "pageable": sample_path(pageable, device, repeats),
        "pinned": sample_path(pinned, device, repeats),
        "pageable_flag": pageable.is_pinned(),
        "pinned_flag": pinned.is_pinned(),
    }
```

Part 6 - Measurement, Validation, and Failure Analysis

Require equal checksums and bytes. A short pageable API call does not prove direct DMA or safe overlap. If results vary, inspect NUMA placement and repeat from clean processes. Never generalize source-mutation safety from the pageable path to the pinned path.

Part 7 - Engineering Interpretation and Philosophy

An API promise and a physical mechanism are different layers. High-throughput design begins by controlling storage type explicitly and ends by validating the actual critical path.

19.8.7 Exercise 06

Exercise Title: Transfer Hundreds of Small Tensors and Measure How Removing Per-Copy Host Synchronization Changes Total Runtime

Part 1 - Exercise Contract and Why It Matters

Submit hundreds of equal small tensors using two policies: blocking each copy and enqueueing all copies before one completion barrier. Measure submission and total time separately.

Part 2 - System and Memory Mental Model

Per-copy synchronization repeatedly stalls the host and prevents the queue from filling. One final barrier preserves correctness while allowing submission to proceed. However, each small copy still carries dispatch and DMA transaction overhead; removing waits does not make granularity free.

Part 3 - Quantitative Prediction Before Execution

For N tensors, blocking time is approximately $N(T_s + T_c + T_w)$. Queued time approaches $NT_s + T_{queue} + T_w$. Predict the ratio for $N = 256$ and identify the remaining fixed-copy cost.

Part 4 - Experimental Design and Instrumentation

Preallocate pinned sources and preserve every destination until completion. Use the same stream, order, sizes, and values. Synchronize before trial start and once at the declared end. Repeat complete loops rather than cherry-picking individual copies.

Part 5 - PyTorch/CUDA Implementation

```
import statistics
import time
import torch

def run_small_loop(sources, device, queued):
    torch.cuda.synchronize(device)
    start = time.perf_counter()
    destinations = []
    for source in sources:
        destination = source.to(device, non_blocking=queued)
        destinations.append(destination)
        if not queued:
            torch.cuda.synchronize(device)
    submitted = time.perf_counter()
    torch.cuda.synchronize(device)
    completed = time.perf_counter()
    checksum = 0.0
    for destination in destinations:
        checksum += float(destination[0].item())
    return ((submitted - start) * 1000.0,
            (completed - start) * 1000.0, checksum)

def E06(count=256, elements_each=4096, repeats=20):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    sources = []
    for index in range(count):
        tensor = torch.full((elements_each,), float(index + 1),
                            dtype=torch.float32, pin_memory=True)
        sources.append(tensor)
    rows = {}
    for queued in (False, True):
        submit_samples = []
        total_samples = []
        for _ in range(repeats):
            submit_ms, total_ms, checksum = run_small_loop(
                sources, device, queued)
            submit_samples.append(submit_ms)
            total_samples.append(total_ms)
        rows[str(queued)] = {
            "submit_median_ms": statistics.median(submit_samples),
            "total_median_ms": statistics.median(total_samples),
            "checksum": checksum,
        }
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Require equal checksums. The blocking path may already synchronize inside `to()`, while the explicit wait makes the boundary unambiguous. If queued submission creates memory pressure, record peak VRAM because all destinations remain live simultaneously. Compare against coalescing in the next exercise.

Part 7 - Engineering Interpretation and Philosophy

Synchronization has multiplicative cost when placed inside a loop. Correctness requires a barrier before consumption, not necessarily a barrier after every independent submission.

19.8.8 Exercise 07

Exercise Title: Compare One Large Non-Blocking Transfer Against Many Small Non-Blocking Transfers with the Same Total Byte Count

Part 1 - Exercise Contract and Why It Matters

Hold total bytes and synchronization policy constant while changing transfer count. Quantify the cost that remains after per-copy host waits are removed.

Part 2 - System and Memory Mental Model

Non-blocking submission removes repeated waiting but not Python dispatch, driver enqueue, DMA descriptors, or small-transfer inefficiency. A large contiguous copy can approach link bandwidth. Packing many fields into one buffer adds a CPU copy, so copy-only and end-to-end packed measurements must be reported separately.

Part 3 - Quantitative Prediction Before Execution

Model N copies as $N\alpha + B/R$ and one copy as $\alpha + B/R$. Predict the copy-only difference for 64 MiB total. Then add a measured packing term to decide whether coalescing pays.

Part 4 - Experimental Design and Instrumentation

Use pre-pinned sources, equal actual byte counts, one final synchronization, and deterministic values. Measure 1, 16, and 256 copies. Keep destination objects alive and verify all of them.

Part 5 - PyTorch/CUDA Implementation

```
import statistics
import time
import torch

def transfer_group(sources, device, repeats):
    samples = []
    checksum = 0.0
    for _ in range(repeats):
        torch.cuda.synchronize(device)
        start = time.perf_counter()
        destinations = []
        for source in sources:
            destinations.append(source.to(device, non_blocking=True))
        torch.cuda.synchronize(device)
```

```
        samples.append((time.perf_counter() - start) * 1000.0)
        checksum = 0.0
        for destination in destinations:
            checksum += float(destination[0].item())
    return statistics.median(samples), checksum

def E07(total_elements=16 * 1024 * 1024, repeats=20):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    rows = []
    for count in (1, 16, 256):
        elements_each = total_elements // count
        sources = []
        for index in range(count):
            source = torch.full((elements_each,), float(index + 1),
                                dtype=torch.float32, pin_memory=True)
            sources.append(source)
        median_ms, checksum = transfer_group(sources, device, repeats)
        actual_bytes = 0
        for source in sources:
            actual_bytes += source.nbytes
        rows.append({"copies": count, "bytes": actual_bytes,
                     "median_ms": median_ms,
                     "GB_s": actual_bytes / (median_ms / 1000.0) / 1e9,
                     "checksum": checksum})
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Check actual bytes because division can drop elements. The checksum changes with the number and values of tensors, so compute a scenario-specific expected value rather than comparing raw checksums across counts. If packing is added, include packing time in a separate total metric.

Part 7 - Engineering Interpretation and Philosophy

Asynchrony and batching solve different problems. One removes unnecessary host waiting; the other reduces the number of operations. Efficient pipelines often need both.

19.8.9 Exercise 08

Exercise Title: Build a Batched Transfer Function That Moves Nested PyTorch Tensors to CUDA Without Blocking the Host After Every Copy

Part 1 - Exercise Contract and Why It Matters

Move every tensor leaf in a nested batch to CUDA, preserve container structure, enqueue copies without per-leaf synchronization, and return an explicit completion ticket. Non-tensor metadata must remain unchanged.

Part 2 - System and Memory Mental Model

A nested batch is a tree of ownership. Each transferred leaf has a destination and a source lifetime. One event recorded after all copies on the transfer stream represents completion of the whole batch. Keeping source references in the ticket prevents premature reclamation; the caller chooses when to wait.

Part 3 - Quantitative Prediction Before Execution

Count tensor leaves and total bytes before transfer. Predict submission cost as the sum of leaf dispatch costs and transfer cost from total bytes plus granularity overhead. Define exactly what `ticket.wait()` guarantees.

Part 4 - Experimental Design and Instrumentation

Test dictionaries, lists, tuples, labels, and strings. Verify input structure, device placement, values, dtype, and a single completion event. Reject unsupported mutable custom containers visibly rather than silently dropping fields.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class TransferTicket:
    def __init__(self, event, sources, value):
        self.event = event
        self.sources = sources
        self.value = value
    def wait(self):
        self.event.synchronize()
        return self.value

def move_tree(value, device, sources):
    if torch.is_tensor(value):
        sources.append(value)
        return value.to(device, non_blocking=True)
    if isinstance(value, dict):
        result = {}
        for key, child in value.items():
            result[key] = move_tree(child, device, sources)
        return result
    if isinstance(value, list):
        result = []
        for child in value:
            result.append(move_tree(child, device, sources))
        return result
    if isinstance(value, tuple):
        result = []
        for child in value:
            result.append(move_tree(child, device, sources))
        return tuple(result)
```

```
    return value

def E08(batch):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    stream = torch.cuda.Stream(device=device)
    sources = []
    event = torch.cuda.Event()
    with torch.cuda.stream(stream):
        moved = move_tree(batch, device, sources)
        event.record(stream)
    return TransferTicket(event, sources, moved)
```

Part 6 - Measurement, Validation, and Failure Analysis

The ticket must stay alive until completion because it owns source references. If downstream GPU work uses another stream, call `wait_event` on that stream instead of forcing the host to wait. For CPU validation, call `ticket.wait()`. Add cycle detection if arbitrary recursive containers are supported.

Part 7 - Engineering Interpretation and Philosophy

A safe asynchronous interface returns both the future value and the evidence needed to know when it is ready. Hiding completion control makes fast code easy to misuse.

19.8.10 Exercise 09

Exercise Title: Benchmark DataLoader(pin_memory=True) with Blocking and Non-Blocking Batch Transfers Under an Identical Training Workload

Part 1 - Exercise Contract and Why It Matters

Measure training throughput when pinned DataLoader batches are transferred with blocking and non-blocking `to()`. Hold data order, workers, model, batch size, optimization, and measured steps constant.

Part 2 - System and Memory Mental Model

Pinned batches enable asynchronous H2D submission, but throughput improves only when host preparation, transfer, and GPU compute overlap or when repeated host waits are removed. A synchronization hidden in logging, loss extraction, or metric calculation can erase overlap.

Part 3 - Quantitative Prediction Before Execution

Predict the critical path for blocking as transfer plus compute. For the non-blocking path, predict the maximum of overlapped stages only if stream dependencies permit it. Estimate pinned queue bytes from batch size, workers, and prefetch factor.

Part 4 - Experimental Design and Instrumentation

Warm workers and GPU, verify `is_pinned()` on batches, use identical seeds and batch order, and avoid per-step `loss.item()`. Synchronize at benchmark boundaries and compare multiple complete runs.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

def run_training(loader, model, optimizer, loss_fn, device,
                 non_blocking, steps):
    iterator = iter(loader)
    torch.cuda.synchronize(device)
    start = time.perf_counter()
    examples = 0
    all_pinned = True
    for _ in range(steps):
        inputs, labels = next(iterator)
        all_pinned = all_pinned and inputs.is_pinned()
        all_pinned = all_pinned and labels.is_pinned()
        inputs = inputs.to(device, non_blocking=non_blocking)
        labels = labels.to(device, non_blocking=non_blocking)
        optimizer.zero_grad(set_to_none=True)
        outputs = model(inputs)
        loss = loss_fn(outputs, labels)
        loss.backward()
        optimizer.step()
        examples += inputs.shape[0]
    torch.cuda.synchronize(device)
    seconds = time.perf_counter() - start
    return {"seconds": seconds, "examples_s": examples / seconds,
            "all_sources_pinned": all_pinned,
            "last_loss": float(loss.detach().item())}

def E09(loader, model_factory, optimizer_factory, loss_fn, steps=100):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    if not loader.pin_memory:
        raise ValueError("DataLoader must use pin_memory=True")
    device = torch.device("cuda:0")
    reference_model = model_factory()
    reference_state = {}
    for name, tensor in reference_model.state_dict().items():
        reference_state[name] = tensor.detach().clone()
    results = {}
    for mode in (False, True):
        model = model_factory().to(device)
        model.load_state_dict(reference_state)
        optimizer = optimizer_factory(model.parameters())
        results[str(mode)] = run_training(
            loader, model, optimizer, loss_fn, device, mode, steps)
    return results
```

Part 6 - Measurement, Validation, and Failure Analysis

Initialize equivalent model states; merely constructing two models in sequence does not guarantee identical weights. The factory or caller should load the same state dictionary. If non-blocking is not faster, inspect compute dominance, default-stream serialization, batch size, and hidden host synchronizations.

Part 7 - Engineering Interpretation and Philosophy

The value of non-blocking transfer is systemic. A correct microbenchmark may show available overlap while a real training loop proves whether the application actually uses it.

19.8.11 Exercise 10

Exercise Title: Corrupt a Pinned Source Tensor Before an Asynchronous CUDA Copy Completes and Reproduce a Real Data-Race Failure

Part 1 - Exercise Contract and Why It Matters

Reproduce a controlled H2D race using disposable synthetic buffers. Enqueue a copy from pinned memory, modify that source before DMA completion, and detect a destination containing old, new, or mixed values. Never run this experiment on real training data or shared application buffers.

Part 2 - System and Memory Mental Model

Pinned host memory is directly accessible to the DMA engine. Once a non-blocking copy is submitted, the host can regain control while the GPU is still reading the source. A CPU write during that interval has no CUDA-stream ordering, so the device may observe bytes before or after modification. Python object lifetime alone does not protect content immutability.

Part 3 - Quantitative Prediction Before Execution

Predict three outcomes: entirely original values, entirely overwritten values, or a mixture. Queue an unrelated large transfer before the target copy to widen the race window. The exercise demonstrates possibility, not a guaranteed split at a specific element.

Part 4 - Experimental Design and Instrumentation

Use a bounded allocation size, fresh pinned tensors, a dedicated stream, and a fixed number of trials. Initialize target with one value and overwrite with a different value. Record an event after the copy and inspect only after event completion. Report mismatches; do not assert that every machine must reproduce the same race frequency.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def E10(target_mib=64, blocker_mib=128, trials=10):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    stream = torch.cuda.Stream(device=device)
    target_numel = target_mib * 1024 * 1024 // 4
```

```
    blocker_numel = blocker_mib * 1024 * 1024 // 4
    race_rows = []
    for trial in range(trials):
        blocker = torch.ones(blocker_numel, dtype=torch.float32,
                             pin_memory=True)
        source = torch.full((target_numel,), 1.0, dtype=torch.float32,
                            pin_memory=True)
        done = torch.cuda.Event()
        with torch.cuda.stream(stream):
            blocker_device = blocker.to(device, non_blocking=True)
            destination = source.to(device, non_blocking=True)
            done.record(stream)

        source.fill_(9.0)  # Intentional unsafe write for this experiment.
        done.synchronize()
        count_original = int((destination == 1.0).sum().item())
        count_overwritten = int((destination == 9.0).sum().item())
        count_other = target_numel - count_original - count_overwritten
        race_rows.append({"trial": trial, "original": count_original,
                          "overwritten": count_overwritten,
                          "other": count_other,
                          "blocker_check": float(blocker_device[0].item())})
    observed = False
    for row in race_rows:
        if row["overwritten"] > 0 or row["other"] > 0:
            observed = True
    return {"race_observed": observed, "trials": race_rows}
```

Part 6 - Measurement, Validation, and Failure Analysis

No observed mismatch does not prove safety; the copy may have won every race. Increase trials or bounded blocker size without exhausting RAM. Mixed values are direct evidence that the copy and CPU write overlapped. Entirely overwritten output is also corruption relative to the submitted logical batch.

Part 7 - Engineering Interpretation and Philosophy

Data races are timing-dependent violations, not deterministic exceptions. A system is unsafe if corruption is possible even when a short test happens to pass. Safety must come from a lifetime protocol, not favorable scheduling.

19.8.12 Exercise 11

Exercise Title: Fix an Unsafe Pinned-Memory Transfer by Introducing Correct Lifetime Management and Synchronization

Part 1 - Exercise Contract and Why It Matters

Repair the race from E10 with a reusable pinned slot that retains its storage and blocks only when that particular slot is about to be overwritten. Prove correct values across many iterations.

Part 2 - System and Memory Mental Model

An event recorded after the copy is the completion token for source reuse. The CPU may perform unrelated work while DMA runs, but it must wait on that event before mutating the slot. A ring of slots reduces waiting by offering another safe buffer while one remains in flight.

Part 3 - Quantitative Prediction Before Execution

For K slots of B bytes, predict pinned capacity KB. Draw the state machine: available, filling, in-flight, and available again. Identify the only transition that requires completion evidence.

Part 4 - Experimental Design and Instrumentation

Fill each slot with an iteration identifier, submit on one copy stream, retain the returned device tensors, and wait only before slot reuse. Validate all values after a final barrier. Track pointers to prove bounded reuse.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class SafePinnedSlot:
    def __init__(self, shape, dtype):
        self.host = torch.empty(shape, dtype=dtype, pin_memory=True)
        self.copy_done = None

    def prepare(self, value):
        if self.copy_done is not None:
            self.copy_done.synchronize()
        self.host.fill_(value)

    def submit(self, device, stream):
        event = torch.cuda.Event()
        with torch.cuda.stream(stream):
            destination = self.host.to(device, non_blocking=True)
            event.record(stream)
        self.copy_done = event
        return destination

def E11(iterations=30, elements=4 * 1024 * 1024):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    stream = torch.cuda.Stream(device=device)
    slots = [SafePinnedSlot((elements,), torch.float32),
             SafePinnedSlot((elements,), torch.float32)]
    outputs = []
    pointers = []
    for step in range(iterations):
        slot = slots[step % len(slots)]
```

```
        slot.prepare(float(step))
        outputs.append(slot.submit(device, stream))
        pointers.append(slot.host.data_ptr())
    torch.cuda.synchronize(device)
    validated = True
    for step, output in enumerate(outputs):
        if float(output[0].item()) != float(step):
            validated = False
    return {"validated": validated,
            "unique_host_buffers": len(set(pointers)),
            "configured_slots": len(slots)}
```

Part 6 - Measurement, Validation, and Failure Analysis

Require unique pointers to equal slot count. The event protects host source reuse, not arbitrary consumers of the CUDA destination. If compute runs on a different stream, make that stream wait on a readiness event. On exception, retain or synchronize in-flight slots before returning them to a pool.

Part 7 - Engineering Interpretation and Philosophy

Correct synchronization is local to the resource whose lifetime is changing. This is both safer and faster than synchronizing the entire device after every operation.

19.8.13 Exercise 12

Exercise Title: Perform a Non-Blocking CUDA-to-CPU Transfer and Demonstrate Why Reading the Destination Too Early Can Produce Incorrect Results

Part 1 - Exercise Contract and Why It Matters

Enqueue D2H transfer into a preallocated pinned CPU tensor, inspect a small sample before completion, then wait and validate the final result. Demonstrate that early host access has no stream dependency and is therefore unsafe.

Part 2 - System and Memory Mental Model

The GPU writes the pinned destination through DMA while the CPU can directly read the same memory. A non-blocking copy returns before those writes necessarily finish. CUDA stream ordering protects later GPU work, but it does not automatically delay ordinary CPU loads. The host needs an event or equivalent completion wait.

Part 3 - Quantitative Prediction Before Execution

Initialize the host buffer with -1 and device source with 7. Predict that an immediate sample may contain stale -1, new 7, or a mixture. After synchronization every element must be 7.

Part 4 - Experimental Design and Instrumentation

Use disposable tensors, a dedicated copy stream, and an event recorded after `copy_`. Read only a small prefix early to limit observer overhead. Never use the early result for real decisions.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def E12(numel=32 * 1024 * 1024):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    stream = torch.cuda.Stream(device=device)
    device_source = torch.full((numel,), 7.0, device=device)
    host_destination = torch.full((numel,), -1.0, dtype=torch.float32,
                                  pin_memory=True)
    done = torch.cuda.Event()
    with torch.cuda.stream(stream):
        host_destination.copy_(device_source, non_blocking=True)
        done.record(stream)

    early_values = host_destination[:16].clone()
    ready_before_wait = done.query()
    done.synchronize()
    final_valid = bool(torch.all(host_destination == 7.0).item())
    return {"ready_before_wait": ready_before_wait,
            "early_values": early_values.tolist(),
            "final_valid": final_valid}
```

Part 6 - Measurement, Validation, and Failure Analysis

Early values may look correct if the transfer completes quickly; this does not make early reading safe. The event query records whether completion was already known at that instant. Final validation must occur after the wait. Run multiple sizes and trials to reveal timing dependence.

Part 7 - Engineering Interpretation and Philosophy

Visibility is directional. H2D races threaten a source that DMA reads; D2H races threaten a destination that DMA writes. Safe code names which side is immutable and which side is not yet readable.

19.8.14 Exercise 13

Exercise Title: Design the Minimum Synchronization Required to Safely Consume Results After an Asynchronous Device-to-Host Copy

Part 1 - Exercise Contract and Why It Matters

Replace device-wide synchronization with the narrowest completion mechanism that safely permits CPU consumption of one D2H result. Compare event waiting, stream waiting, polling, and global waiting.

Part 2 - System and Memory Mental Model

An event recorded after the copy represents that copy and all earlier work in its stream. `event.synchronize()` blocks the host only until that point. `stream.synchronize()` waits for all current work in one stream, while `torch.cuda.synchronize()` waits for all work on the

device. Event query allows non-blocking polling but consumption remains forbidden until it returns true.

Part 3 - Quantitative Prediction Before Execution

Construct a stream with unrelated long work after the D2H copy. Predict that waiting on the copy event can release the CPU earlier than synchronizing the whole stream or device.

Part 4 - Experimental Design and Instrumentation

Record the event immediately after D2H copy and before unrelated stream work. Time event wait and whole-stream wait in separate equivalent trials. Validate the host result after the event only.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

def E13(numel=16 * 1024 * 1024):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    stream = torch.cuda.Stream(device=device)
    device_source = torch.arange(numel, dtype=torch.float32, device=device)
    host_destination = torch.empty(numel, dtype=torch.float32,
                                   pin_memory=True)
    copy_done = torch.cuda.Event()
    with torch.cuda.stream(stream):
        host_destination.copy_(device_source, non_blocking=True)
        copy_done.record(stream)
        unrelated = torch.randn((4096, 4096), device=device)
        unrelated_result = unrelated @ unrelated

    start = time.perf_counter()
    copy_done.synchronize()
    event_wait_ms = (time.perf_counter() - start) * 1000.0
    first = float(host_destination[0].item())
    last = float(host_destination[-1].item())
    stream.synchronize()
    return {"event_wait_ms": event_wait_ms,
            "validated": first == 0.0 and last == float(numel - 1),
            "unrelated_check": float(unrelated_result[0, 0].item())}
```

Part 6 - Measurement, Validation, and Failure Analysis

The matrix multiplication may stress memory; scale it to the device. The event must be recorded before unrelated work. For a fair timing comparison, execute independent reconstructed trials because waiting once changes later wait times. Polling loops need backoff or useful host work to avoid wasting a CPU core.

Part 7 - Engineering Interpretation and Philosophy

Minimum synchronization means waiting for the required dependency, not removing all waits. Narrow completion tokens preserve concurrency while making readiness provable.

19.8.15 Exercise 14

Exercise Title: Compare CPU-to-GPU, GPU-to-CPU, and GPU-to-GPU non_blocking=True Semantics Through Controlled Correctness Experiments

Part 1 - Exercise Contract and Why It Matters

Build one experiment for each transfer direction and write its distinct lifetime and consumption rule. Prove correctness with events rather than assuming that the same flag has identical implications everywhere.

Part 2 - System and Memory Mental Model

H2D from pinned memory requires the CPU source to remain immutable until copy completion. D2H requires the CPU destination to remain unread until completion. D2D is ordered for operations on the same stream; a consumer on another stream must wait on an event. For cross-device copies, peer-access support and device topology can change the transport path without changing these dependency rules.

Part 3 - Quantitative Prediction Before Execution

For each direction, name source owner, destination owner, forbidden early action, and completion proof. Predict which action causes corruption: CPU source write, CPU destination read, or cross-stream GPU consumption.

Part 4 - Experimental Design and Instrumentation

Use pinned host buffers, one copy stream, one consumer stream, and separate events. Perform only safe variants in the final combined test; the unsafe variants are already isolated in E10 and E12. Validate values after appropriate waits.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def E14(numel=4 * 1024 * 1024):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    copy_stream = torch.cuda.Stream(device=device)
    consumer_stream = torch.cuda.Stream(device=device)

    host_source = torch.full((numel,), 3.0, pin_memory=True)
    h2d_done = torch.cuda.Event()
    with torch.cuda.stream(copy_stream):
        gpu_a = host_source.to(device, non_blocking=True)
        h2d_done.record(copy_stream)
    h2d_done.synchronize()
```

```
    host_source.fill_(4.0)  # Safe only after the event.

    d2d_done = torch.cuda.Event()
    with torch.cuda.stream(copy_stream):
        gpu_b = torch.empty_like(gpu_a)
        gpu_b.copy_(gpu_a, non_blocking=True)
        d2d_done.record(copy_stream)
    consumer_stream.wait_event(d2d_done)
    with torch.cuda.stream(consumer_stream):
        gpu_sum = gpu_b.sum()

    host_destination = torch.empty(numel, dtype=torch.float32,
                                   pin_memory=True)
    d2h_done = torch.cuda.Event()
    with torch.cuda.stream(copy_stream):
        host_destination.copy_(gpu_b, non_blocking=True)
        d2h_done.record(copy_stream)
    d2h_done.synchronize()
    consumer_stream.synchronize()
    return {"h2d_value": float(gpu_a[0].item()),
            "d2d_sum": float(gpu_sum.item()),
            "d2h_value": float(host_destination[0].item())}
```

Part 6 - Measurement, Validation, and Failure Analysis

Expected transferred value is 3, although the host source becomes 4 after safe completion. The D2D sum and D2H value must agree with GPU A. If a second GPU is available, add a peer-copy variant and record peer-access support, but do not make two GPUs a requirement for this exercise.

Part 7 - Engineering Interpretation and Philosophy

Asynchrony is not one universal behavior. Direction determines which memory is vulnerable and which observer needs a completion proof. A transfer API should encode these roles explicitly.

19.8.16 Exercise 15

Exercise Title: Profile Blocking and Non-Blocking Transfer Loops and Identify cudaMemcpyAsync and Synchronization Events in the Execution Trace

Part 1 - Exercise Contract and Why It Matters

Capture profiler traces for equivalent blocking and queued transfer loops. Locate copy activity, synchronization calls, CPU gaps, and CUDA kernels, then connect trace structure to measured runtime.

Part 2 - System and Memory Mental Model

PyTorch commonly dispatches CUDA copy machinery asynchronously at lower layers, even when a high-level blocking operation subsequently waits. The trace can therefore contain

`cudaMemcpyAsync` plus synchronization. Performance difference comes from where and how often waiting occurs, not necessarily from using a completely different copy primitive.

Part 3 - Quantitative Prediction Before Execution

Predict one synchronization region per blocking copy and one completion barrier for the queued loop. Predict equal copy counts but different CPU scheduling gaps.

Part 4 - Experimental Design and Instrumentation

Profile warm active steps only, annotate each mode with `record_function`, record CPU and CUDA activities, and export separate Chrome traces. Measure profiler overhead against an unprofiled baseline.

Part 5 - PyTorch/CUDA Implementation

```
import torch
from torch.profiler import ProfilerActivity, record_function

def profile_loop(sources, device, queued, trace_path):
    activities = [ProfilerActivity.CPU, ProfilerActivity.CUDA]
    with torch.profiler.profile(activities=activities,
                                record_shapes=True) as profiler:
        destinations = []
        label = "queued_non_blocking" if queued else "per_copy_blocking"
        with record_function(label):
            for source in sources:
                destination = source.to(device, non_blocking=queued)
                destinations.append(destination)
                if not queued:
                    torch.cuda.synchronize(device)
            torch.cuda.synchronize(device)
    profiler.export_chrome_trace(trace_path)
    table = profiler.key_averages().table(
        sort_by="self_cuda_time_total", row_limit=30)
    return table, destinations

def validate_outputs(outputs):
    valid = True
    for index, output in enumerate(outputs):
        if float(output[0].item()) != float(index):
            valid = False
    return valid

def E15(output_prefix="dmm06_transfer"):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    sources = []
    for index in range(64):
        source = torch.full((256 * 1024,), float(index),
```

```
                        dtype=torch.float32, pin_memory=True)
    sources.append(source)
blocking_table, blocking_outputs = profile_loop(
    sources, device, False, output_prefix + "_blocking.json")
queued_table, queued_outputs = profile_loop(
    sources, device, True, output_prefix + "_queued.json")
return {"blocking_table": blocking_table,
        "queued_table": queued_table,
        "blocking_outputs": len(blocking_outputs),
        "queued_outputs": len(queued_outputs),
        "blocking_valid": validate_outputs(blocking_outputs),
        "queued_valid": validate_outputs(queued_outputs)}
```

Part 6 - Measurement, Validation, and Failure Analysis

Search the trace for copy and synchronization events, but account for naming differences across PyTorch, CUDA, and profiler versions. The profiler changes timing and should diagnose structure, while the unprofiled benchmark determines speed. Verify both loops copied identical bytes and values.

Part 7 - Engineering Interpretation and Philosophy

A trace reveals causality that aggregate time hides. It shows whether the host waits after every operation, whether the queue stays populated, and where the saved time actually goes.

19.8.17 Exercise 16

Exercise Title: Measure Effective Transfer Bandwidth and Host Availability Under Blocking and Non-Blocking Workloads

Part 1 - Exercise Contract and Why It Matters

Measure two outcomes simultaneously: completed H2D bandwidth and how much useful CPU work can proceed while the copy is in flight. Avoid claiming success from short submission time when the host performs no useful work.

Part 2 - System and Memory Mental Model

Effective bandwidth measures bytes delivered per completion time. Host availability measures scheduling freedom between submission and completion. Non-blocking transfer can increase the latter without changing raw link bandwidth. Busy polling wastes that freedom; useful CPU preprocessing or next- batch preparation can exploit it.

Part 3 - Quantitative Prediction Before Execution

Predict GB/s from $B/(t10^9)$. Define a deterministic CPU work unit, calibrate its standalone rate, and predict how many units can execute before the transfer event completes. Keep copy contention and CPU work separate in interpretation.

Part 4 - Experimental Design and Instrumentation

Use a large pre-pinned source and CUDA event. For non-blocking mode, perform fixed CPU chunks until `event.query()` becomes true. For blocking mode, measure the same work only

after the call returns. Report completion bandwidth, overlap window, CPU units, and total wall time.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

def cpu_work_unit(state):
    value = state
    for index in range(2000):
        value = (value * 1664525 + index + 1013904223) % 4294967296
    return value

def E16(numel=64 * 1024 * 1024):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    source = torch.ones(numel, dtype=torch.float32, pin_memory=True)
    stream = torch.cuda.Stream(device=device)
    start_event = torch.cuda.Event(enable_timing=True)
    end_event = torch.cuda.Event(enable_timing=True)
    torch.cuda.synchronize(device)
    wall_start = time.perf_counter()
    with torch.cuda.stream(stream):
        start_event.record(stream)
        destination = source.to(device, non_blocking=True)
        end_event.record(stream)
    host_return = time.perf_counter()
    units = 0
    state = 1
    while not end_event.query():
        state = cpu_work_unit(state)
        units += 1
    wall_end = time.perf_counter()
    copy_ms = start_event.elapsed_time(end_event)
    return {"bytes": source.nbytes,
            "submission_ms": (host_return - wall_start) * 1000.0,
            "completion_wall_ms": (wall_end - wall_start) * 1000.0,
            "device_copy_ms": copy_ms,
            "GB_s": source.nbytes / (copy_ms / 1000.0) / 1e9,
            "cpu_work_units": units, "cpu_state": state,
            "check": float(destination[0].item())}
```

Part 6 - Measurement, Validation, and Failure Analysis

Event polling has overhead and should not be the production design; it provides a controlled experiment. Compare with a calibrated CPU-only loop and an event- wait version. If zero work

units execute, increase transfer size safely or use smaller work chunks. Report decimal GB/s explicitly.

Part 7 - Engineering Interpretation and Philosophy

The purpose of asynchrony is not an impressive API-return number. Its value is productive concurrency: the host advances the pipeline while the transfer engine delivers data.

19.8.18 Exercise 17

Exercise Title: Build a Transfer Queue That Submits Multiple Tensor Copies Before Performing a Single Explicit Synchronization

Part 1 - Exercise Contract and Why It Matters

Implement a transfer queue whose submissions return tickets and whose `wait_all()` method performs one explicit completion phase. Preserve source lifetimes and validate every destination.

Part 2 - System and Memory Mental Model

The queue owns a CUDA stream, pending source references, destinations, and events. Each event defines readiness of one result. One final wait drains the queue, but individual tickets can support earlier dependencies. Bounded queue depth prevents unlimited VRAM and host-reference growth.

Part 3 - Quantitative Prediction Before Execution

Predict resident source and destination bytes for depth Q. Define backpressure when pending count reaches Q. Compare $Q = 1, 4, 16$ for submission freedom, memory footprint, and throughput.

Part 4 - Experimental Design and Instrumentation

Use pinned sources with unique sentinels. Submit multiple copies, call one wait, and validate all outputs. Track peak pending bytes and reject a submission when the budget would be exceeded.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class CopyTicket:
    def __init__(self, source, destination, event):
        self.source = source
        self.destination = destination
        self.event = event
    def wait(self):
        self.event.synchronize()
        return self.destination

class TransferQueue:
    def __init__(self, device, max_pending):
        self.device = device
```

```
        self.stream = torch.cuda.Stream(device=device)
        self.max_pending = max_pending
        self.pending = []
    def submit(self, source):
        if len(self.pending) >= self.max_pending:
            raise RuntimeError("Transfer queue is full")
        event = torch.cuda.Event()
        with torch.cuda.stream(self.stream):
            destination = source.to(self.device, non_blocking=True)
            event.record(self.stream)
        ticket = CopyTicket(source, destination, event)
        self.pending.append(ticket)
        return ticket
    def wait_all(self):
        completed = []
        for ticket in self.pending:
            completed.append(ticket.wait())
        self.pending.clear()
        return completed

def E17(count=16, elements=1024 * 1024):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    queue = TransferQueue(device, max_pending=count)
    for index in range(count):
        source = torch.full((elements,), float(index),
                            dtype=torch.float32, pin_memory=True)
        queue.submit(source)
    outputs = queue.wait_all()
    validated = True
    for index, output in enumerate(outputs):
        if float(output[0].item()) != float(index):
            validated = False
    return {"validated": validated, "completed": len(outputs)}
```

Part 6 - Measurement, Validation, and Failure Analysis

The queue retains sources until wait, preventing deallocation but not mutation; callers must treat submitted sources as immutable. Production code should apply backpressure by waiting for the oldest ticket rather than raising immediately. Handle exceptions by draining or safely cancelling owned work.

Part 7 - Engineering Interpretation and Philosophy

A queue turns asynchronous calls into an explicit ownership system. Throughput comes from controlled in-flight work; stability comes from bounding it.

19.8.19 Exercise 18

Exercise Title: Stress-Test Asynchronous Transfers with Variable Tensor Sizes and Detect Incorrect Synchronization, Source-Lifetime, and Timing Assumptions

Part 1 - Exercise Contract and Why It Matters

Run a bounded randomized stress test across variable sizes, dtypes, and queue depths. Detect value corruption, lost lifetimes, invalid timing, and memory growth. The goal is to break assumptions that fixed-size microbenchmarks do not expose.

Part 2 - System and Memory Mental Model

Variable sizes change transfer duration and completion order across streams. Temporary Python objects may disappear while work is in flight, queues may retain too much memory, and one final global wait can hide incorrect per-result readiness. Sentinels and per-ticket events make these failures observable.

Part 3 - Quantitative Prediction Before Execution

Choose minimum and maximum bytes, dtype set, queue bound, and total in-flight budget before execution. Predict peak host and device bytes. Define failure as wrong sentinel, non-finite timing, exceeded budget, or unbounded memory trend.

Part 4 - Experimental Design and Instrumentation

Fix random seeds, generate an explicit case manifest, allocate pinned sources, and retain them in tickets. Drain the oldest ticket at the queue bound. Record submission and completion times, bytes, order, and allocator memory. Never stress to host or GPU exhaustion.

Part 5 - PyTorch/CUDA Implementation

```
import random
import time
import torch

def E18(cases=200, max_pending=8, max_mib=32):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    random.seed(1234)
    torch.manual_seed(1234)
    device = torch.device("cuda:0")
    torch.cuda.reset_peak_memory_stats(device)
    stream = torch.cuda.Stream(device=device)
    pending = []
    failures = []
    rows = []
    dtypes = [torch.float16, torch.float32, torch.float64]
    for case_index in range(cases):
        dtype = dtypes[case_index % len(dtypes)]
        mib = random.randint(1, max_mib)
        element_size = torch.empty((), dtype=dtype).element_size()
        numel = mib * 1024 * 1024 // element_size
```

```
        sentinel = float((case_index % 97) + 1)
        source = torch.full((numel,), sentinel, dtype=dtype,
                            pin_memory=True)
        event = torch.cuda.Event()
        submit_start = time.perf_counter()
        with torch.cuda.stream(stream):
            destination = source.to(device, non_blocking=True)
            event.record(stream)
        submit_ms = (time.perf_counter() - submit_start) * 1000.0
        pending.append((case_index, sentinel, source, destination, event))
        rows.append({"case": case_index, "bytes": source.nbytes,
                     "dtype": str(dtype), "submit_ms": submit_ms})
        if len(pending) >= max_pending:
            old_item = pending.pop(0)
            old_case = old_item[0]
            old_value = old_item[1]
            old_source = old_item[2]
            old_destination = old_item[3]
            old_event = old_item[4]
            old_event.synchronize()
            if float(old_destination[0].item()) != old_value:
                failures.append(old_case)
    for old_item in pending:
        old_case = old_item[0]
        old_value = old_item[1]
        old_source = old_item[2]
        old_destination = old_item[3]
        old_event = old_item[4]
        old_event.synchronize()
        if float(old_destination[0].item()) != old_value:
            failures.append(old_case)
    return {"cases": cases, "failures": failures, "rows": rows,
            "peak_allocated": torch.cuda.max_memory_allocated(device)}
```

Part 6 - Measurement, Validation, and Failure Analysis

Reset peak statistics before independent runs if peak comparison is required. Validate more than the first element in a hardened version by sampling multiple positions or checksums. A zero-failure stress run increases confidence but does not prove correctness; code review of lifetime and stream dependencies remains necessary.

Part 7 - Engineering Interpretation and Philosophy

Stress testing explores schedules that ordinary tests rarely visit. Its strongest value is forcing hidden assumptions about size, order, and lifetime into explicit invariants.

19.8.20 Exercise 19

Exercise Title: Build and Benchmark a Safe High-Throughput PyTorch Host-to-GPU Transfer Engine Using Pinned Buffers, Non-Blocking Copies, and Explicit Completion Control

Part 1 - Exercise Contract and Why It Matters

Integrate the chapter into a bounded transfer engine. It must reuse pinned host slots, submit H2D copies on a dedicated stream, expose readiness events, preserve source and destination lifetimes, connect to a consumer stream, apply backpressure, validate output, and report throughput against a blocking baseline.

Part 2 - System and Memory Mental Model

Each slot moves through available, filling, in-flight, and reusable states. A copy-completion event protects host-slot reuse. A consumer stream waits on the same readiness event before reading the CUDA destination. Because the destination was allocated on the copy stream and consumed elsewhere, recording the consumer stream on that tensor protects allocator lifetime. The pool size bounds pinned RAM; slot acquisition provides backpressure.

Part 3 - Quantitative Prediction Before Execution

For K slots and batch bytes B, predict pinned capacity KB. Estimate serial time $T_{fill} + T_{copy} + T_{compute}$ and overlapped time near their maximum when the pipeline is balanced. Predict the point where more slots stop improving throughput.

Part 4 - Experimental Design and Instrumentation

Use identical CPU batches for blocking and engine paths. Warm up both, synchronize only at declared boundaries, and report median throughput, p95 latency, pinned capacity, slot waits, and correctness. Test unsupported shape and budget failure without unbounded allocation.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

class EngineSlot:
    def __init__(self, shape, dtype):
        self.host = torch.empty(shape, dtype=dtype, pin_memory=True)
        self.copy_done = None
    def acquire(self):
        if self.copy_done is not None:
            self.copy_done.synchronize()

class EngineTicket:
    def __init__(self, destination, ready):
        self.destination = destination
        self.ready = ready
    def attach_consumer(self, consumer_stream):
        consumer_stream.wait_event(self.ready)
        self.destination.record_stream(consumer_stream)
        return self.destination
    def wait_host(self):
        self.ready.synchronize()
        return self.destination
```

```
class H2DTransferEngine:
    def __init__(self, shape, dtype, slot_count, budget_bytes, device):
        element_bytes = torch.empty((), dtype=dtype).element_size()
        slot_bytes = element_bytes
        for dimension in shape:
            slot_bytes *= dimension
        if slot_count * slot_bytes > budget_bytes:
            raise MemoryError("Pinned-buffer budget exceeded")
        self.shape = torch.Size(shape)
        self.dtype = dtype
        self.device = device
        self.copy_stream = torch.cuda.Stream(device=device)
        self.slots = []
        for _ in range(slot_count):
            self.slots.append(EngineSlot(shape, dtype))
        self.index = 0
        self.pinned_bytes = slot_count * slot_bytes
    def submit(self, source):
        if source.shape != self.shape or source.dtype != self.dtype:
            raise ValueError("Batch does not match engine shape and dtype")
        slot = self.slots[self.index]
        slot.acquire()
        slot.host.copy_(source)
        ready = torch.cuda.Event()
        with torch.cuda.stream(self.copy_stream):
            destination = slot.host.to(self.device, non_blocking=True)
            ready.record(self.copy_stream)
        slot.copy_done = ready
        self.index = (self.index + 1) % len(self.slots)
        return EngineTicket(destination, ready)

def E19(cpu_batches, model, slot_count=2, budget_mib=1024):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    if len(cpu_batches) == 0:
        raise ValueError("At least one batch is required")
    device = torch.device("cuda:0")
    model = model.to(device)
    model.eval()

    baseline_checksums = []
    torch.cuda.synchronize(device)
    baseline_start = time.perf_counter()
    with torch.no_grad():
        for batch in cpu_batches:
            device_batch = batch.to(device, non_blocking=False)
            output = model(device_batch)
            baseline_checksums.append(output.float().sum())
```

```
    torch.cuda.synchronize(device)
    baseline_seconds = time.perf_counter() - baseline_start

    engine = H2DTransferEngine(
        cpu_batches[0].shape, cpu_batches[0].dtype, slot_count,
        budget_mib * 1024 ** 2, device)
    consumer = torch.cuda.current_stream(device)
    torch.cuda.synchronize(device)
    start = time.perf_counter()
    checksums = []
    with torch.no_grad():
        for batch in cpu_batches:
            ticket = engine.submit(batch)
            device_batch = ticket.attach_consumer(consumer)
            with torch.cuda.stream(consumer):
                output = model(device_batch)
                checksums.append(output.float().sum())
    torch.cuda.synchronize(device)
    seconds = time.perf_counter() - start
    values = []
    for checksum in checksums:
        values.append(float(checksum.item()))
    baseline_values = []
    for checksum in baseline_checksums:
        baseline_values.append(float(checksum.item()))
    return {"baseline_seconds": baseline_seconds,
            "engine_seconds": seconds,
            "baseline_batches_s": len(cpu_batches) / baseline_seconds,
            "engine_batches_s": len(cpu_batches) / seconds,
            "pinned_bytes": engine.pinned_bytes,
            "outputs_match": values == baseline_values,
            "checksums": values}
```

Part 6 - Measurement, Validation, and Failure Analysis

Compare checksums with a blocking reference under identical model state. The sample engine waits before refilling a slot and rejects shape drift; production systems may use bounded size classes or an explicit fallback. Handle exceptions by synchronizing owned in-flight events before recycling slots. Monitor pinned RAM, VRAM, and queue depth together.

Part 7 - Engineering Interpretation and Philosophy

A transfer engine is a temporal memory manager. It succeeds when it makes ownership, readiness, reuse, and backpressure explicit while reducing the real critical path. Throughput without those invariants is merely a faster route to silent corruption.

19.9 DMM07

Chapter Title: CUDA Streams in PyTorch
Specialization: Dynamic Memory Management and CUDA Systems

19.9.1 Exercise 00

Exercise Title: Build a Precise Mental Model of the Default CUDA Stream, Custom Streams, Queued Operations, and Asynchronous GPU Execution

Part 1 - Exercise Contract and Why It Matters

Construct a stream-level model of one PyTorch process. Identify the current stream for each device, the ordered queue inside each stream, the absence of automatic ordering across independent streams, and the points where the CPU waits. The final model must separately explain execution dependency, completion observation, and allocator lifetime.

Part 2 - System and Memory Mental Model

A CUDA stream is an ordered sequence belonging to one device. Operations in the same stream execute in issue order. Different streams can overlap or interleave when dependencies and hardware resources allow, but concurrency is not guaranteed. PyTorch submits work asynchronously, so Python can continue while a stream still contains queued kernels.

The current stream is a thread-local submission context. The default stream is used when no custom context is active. `wait_stream()` and events create execution dependencies; `synchronize()` makes the host wait; `record_stream()` tells the allocator that tensor storage remains in use by a side stream. These mechanisms answer different questions and cannot be substituted blindly.

Part 3 - Quantitative Prediction Before Execution

Draw two streams with operations A_1, A_2 and B_1, B_2. Predict legal schedules without dependencies, then add an event from A_1 to B_2. Calculate sequential time $T_A + T_B$ and ideal overlap $\max(T_A, T_B)$, while noting that resource contention can place reality between them.

Part 4 - Experimental Design and Instrumentation

Record device, stream identifiers, event timestamps, host submission time, and completed wall time. Warm CUDA and use sufficiently large independent kernels. Validate results after explicit dependencies. Use profiler traces to prove overlap rather than inferring it from two stream objects.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

def stream_description(stream):
    return {"device": str(stream.device),
            "cuda_stream": int(stream.cuda_stream),
            "priority": int(stream.priority),
            "ready": bool(stream.query())}

def E00(matrix_size=2048):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required for DMM07.")
    device = torch.device("cuda:0")
    default_stream = torch.cuda.default_stream(device)
```

```
    first_stream = torch.cuda.Stream(device=device)
    second_stream = torch.cuda.Stream(device=device)
    left = torch.randn((matrix_size, matrix_size), device=device)
    right = torch.randn((matrix_size, matrix_size), device=device)
    torch.cuda.synchronize(device)
    host_start = time.perf_counter()
    with torch.cuda.stream(first_stream):
        first_result = left @ right
    with torch.cuda.stream(second_stream):
        second_result = right @ left
    host_return = time.perf_counter()
    first_stream.synchronize()
    second_stream.synchronize()
    completed = time.perf_counter()
    return {"default": stream_description(default_stream),
            "first": stream_description(first_stream),
            "second": stream_description(second_stream),
            "submission_ms": (host_return - host_start) * 1000.0,
            "completion_ms": (completed - host_start) * 1000.0,
            "checks": [float(first_result[0, 0].item()),
                       float(second_result[0, 0].item())]}
```

Part 6 - Measurement, Validation, and Failure Analysis

Two custom streams do not prove simultaneous kernels. Check profiler lanes and elapsed time against a sequential baseline. The input tensors were created on the default stream; synchronize before side-stream use or create explicit waits as done by the pre-experiment device barrier. Remove that barrier only after replacing it with precise dependencies.

Part 7 - Engineering Interpretation and Philosophy

Streams are timelines, not worker threads. Correct multi-stream design makes the dependency graph and storage lifetimes explicit, then asks the hardware how much concurrency it can realize.

19.9.2 Exercise 01

Exercise Title: Inspect torch.cuda.current_stream() and Verify Which Stream Executes PyTorch CUDA Operations by Default

Part 1 - Exercise Contract and Why It Matters

Inspect the current and default streams outside and inside nested stream contexts. Prove which stream receives each operation without relying on visual code indentation alone.

Part 2 - System and Memory Mental Model

`current_stream()` returns the stream used for new submissions on a device in the current context. Entering `torch.cuda.stream(s)` temporarily changes it; leaving restores the previous stream. The default stream is a stable device-associated stream but need not remain current inside a custom context.

Part 3 - Quantitative Prediction Before Execution

Predict stream handles at four points: initial, inside stream A, inside nested stream B, and after both contexts. Predict that restoration follows stack order.

Part 4 - Experimental Design and Instrumentation

Record handles and priorities at every boundary. Record events immediately after operations and synchronize those events before validating outputs. Use one device explicitly to avoid accidental current-device changes.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def handle(stream):
    return int(stream.cuda_stream)

def E01():
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    default_stream = torch.cuda.default_stream(device)
    stream_a = torch.cuda.Stream(device=device)
    stream_b = torch.cuda.Stream(device=device)
    observations = []
    current = torch.cuda.current_stream(device)
    observations.append(("initial", handle(current)))
    with torch.cuda.stream(stream_a):
        tensor_a = torch.ones(1024, device=device) * 2.0
        current = torch.cuda.current_stream(device)
        observations.append(("inside_a", handle(current)))
        with torch.cuda.stream(stream_b):
            tensor_b = torch.ones(1024, device=device) * 3.0
            current = torch.cuda.current_stream(device)
            observations.append(("inside_b", handle(current)))
        current = torch.cuda.current_stream(device)
        observations.append(("restored_a", handle(current)))
    current = torch.cuda.current_stream(device)
    observations.append(("restored_default", handle(current)))
    torch.cuda.synchronize(device)
    return {"observations": observations,
            "default_handle": handle(default_stream),
            "a_handle": handle(stream_a), "b_handle": handle(stream_b),
            "checks": [float(tensor_a[0].item()), float(tensor_b[0].item())]}
```

Part 6 - Measurement, Validation, and Failure Analysis

Assert the predicted restoration sequence and distinct handles. Stream handle values are process-local implementation identifiers, not portable persistent IDs. A context controls submissions executed while active; it does not migrate already queued work.

Part 7 - Engineering Interpretation and Philosophy

Submission context is mutable state. Inspecting it at boundaries turns an implicit scheduling assumption into a testable invariant.

19.9.3 Exercise 02

Exercise Title: Create a Custom torch.cuda.Stream() and Execute Tensor Operations Inside a Stream Context

Part 1 - Exercise Contract and Why It Matters

Create one custom stream, submit a dependent chain inside its context, record a completion event, and consume the result safely from the default stream.

Part 2 - System and Memory Mental Model

Operations submitted inside the context inherit the custom stream and are ordered with one another. Leaving the context restores submission state but does not wait for completion. The default stream must explicitly wait for the event before consuming the side-stream output.

Part 3 - Quantitative Prediction Before Execution

For $y = 2x$, $z = y + 5$, and $r = z^2$, predict values and stream order. Then predict the dependency edge needed before a default-stream reduction of r.

Part 4 - Experimental Design and Instrumentation

Prepare input before the custom chain and make the custom stream wait for the default stream that created it. Record an event after the producer chain. Make the default stream wait on that event, then validate.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def E02(numel=4 * 1024 * 1024):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    default_stream = torch.cuda.current_stream(device)
    side_stream = torch.cuda.Stream(device=device)
    x = torch.arange(numel, dtype=torch.float32, device=device)
    side_stream.wait_stream(default_stream)
    ready = torch.cuda.Event()
    with torch.cuda.stream(side_stream):
        y = x * 2.0
        z = y + 5.0
        result = z.square()
        ready.record(side_stream)
    default_stream.wait_event(ready)
    total = result.sum()
    result.record_stream(default_stream)
```

```
    default_stream.synchronize()
    expected_first = 25.0
    return {"first": float(result[0].item()),
            "expected_first": expected_first,
            "sum": float(total.item()),
            "side_handle": int(side_stream.cuda_stream)}
```

Part 6 - Measurement, Validation, and Failure Analysis

The wait before production protects reading x; the wait after production protects default-stream consumption. These are separate edges. Retaining Python references does not create ordering. `record_stream` protects allocator lifetime when the result crosses stream ownership.

Part 7 - Engineering Interpretation and Philosophy

A custom stream is useful only when its inputs, outputs, and ownership transfer are explicit. Context syntax is the surface; the dependency graph is the real program.

19.9.4 Exercise 03

Exercise Title: Launch Independent CUDA Work on Two Streams and Determine Whether the GPU Executes Them Concurrently

Part 1 - Exercise Contract and Why It Matters

Launch independent workloads on two streams and determine overlap from timing and profiler evidence. Avoid treating multiple stream creation as proof of concurrency.

Part 2 - System and Memory Mental Model

Streams permit concurrency but the scheduler realizes it only when kernels, copy engines, occupancy, and dependencies allow. Two large matrix multiplications may each saturate the GPU and serialize in practice; smaller kernels may overlap but expose overhead. Concurrency is an empirical property of workload and device.

Part 3 - Quantitative Prediction Before Execution

Measure isolated times T_A, T_B, sequential time, and multi-stream time T_M. Define an overlap indicator $(T_A + T_B - T_M)/\min(T_A, T_B)$, interpreted cautiously under contention.

Part 4 - Experimental Design and Instrumentation

Use independent inputs and outputs, warm up kernels, record a start event on a control stream, make both workers wait for it, and record completion events. Host-wait after both completions. Repeat several workload sizes and inspect a profiler trace.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

def E03(size=2048):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
```

```
    device = torch.device("cuda:0")
    stream_a = torch.cuda.Stream(device=device)
    stream_b = torch.cuda.Stream(device=device)
    control = torch.cuda.current_stream(device)
    a = torch.randn((size, size), device=device)
    b = torch.randn((size, size), device=device)
    c = torch.randn((size, size), device=device)
    d = torch.randn((size, size), device=device)
    torch.cuda.synchronize(device)
    start_gate = torch.cuda.Event()
    done_a = torch.cuda.Event()
    done_b = torch.cuda.Event()
    start_gate.record(control)
    stream_a.wait_event(start_gate)
    stream_b.wait_event(start_gate)
    wall_start = time.perf_counter()
    with torch.cuda.stream(stream_a):
        result_a = a @ b
        done_a.record(stream_a)
    with torch.cuda.stream(stream_b):
        result_b = c @ d
        done_b.record(stream_b)
    done_a.synchronize()
    done_b.synchronize()
    wall_ms = (time.perf_counter() - wall_start) * 1000.0
    return {"multi_stream_wall_ms": wall_ms,
            "checks": [float(result_a[0, 0].item()),
                       float(result_b[0, 0].item())]}
```

Part 6 - Measurement, Validation, and Failure Analysis

This run alone needs sequential and isolated companion runs for the overlap metric. Ensure input creation completed before the start gate through the initial device synchronization. Vary kernel size; lack of overlap can be a legitimate resource result, not a stream bug.

Part 7 - Engineering Interpretation and Philosophy

Streams expose possible parallelism. Profiling decides whether the workload has usable parallelism and whether contention makes it worthwhile.

19.9.5 Exercise 04

Exercise Title: Compare Sequential Default-Stream Execution Against Multi-Stream Execution Under Identical CUDA Workloads

Part 1 - Exercise Contract and Why It Matters

Build paired sequential and multi-stream benchmarks using the same tensors, kernels, warmup, repetition count, and validation. Quantify speedup and identify whether concurrency or overhead dominates.

Part 2 - System and Memory Mental Model

The default stream gives simple ordered execution. Multiple streams remove unnecessary ordering between independent tasks but add event, context, and scheduling overhead. If each task saturates the device, multi-stream execution may offer no gain. If tasks use different engines or leave resources idle, overlap may shorten the critical path.

Part 3 - Quantitative Prediction Before Execution

Predict sequential time as the sum of task durations and ideal multi-stream time as their maximum. Define measured speedup and efficiency relative to that ideal. State a threshold larger than measurement noise before declaring improvement.

Part 4 - Experimental Design and Instrumentation

Alternate benchmark order across repetitions. Synchronize before starting and after finishing each full trial. Reuse inputs, retain outputs, and compare numerical results. Report median and p95.

Part 5 - PyTorch/CUDA Implementation

```
import statistics
import time
import torch

def run_pair(a, b, c, d, device, multi_stream):
    torch.cuda.synchronize(device)
    start = time.perf_counter()
    if not multi_stream:
        first = a @ b
        second = c @ d
    else:
        stream_a = torch.cuda.Stream(device=device)
        stream_b = torch.cuda.Stream(device=device)
        with torch.cuda.stream(stream_a):
            first = a @ b
        with torch.cuda.stream(stream_b):
            second = c @ d
        stream_a.synchronize()
        stream_b.synchronize()
    torch.cuda.synchronize(device)
    return (time.perf_counter() - start) * 1000.0, first, second

def E04(size=1536, repeats=20):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    tensors = []
    for _ in range(4):
        tensors.append(torch.randn((size, size), device=device))
    rows = {}
```

```
    for mode in (False, True):
        samples = []
        for _ in range(repeats):
            elapsed, first, second = run_pair(
                tensors[0], tensors[1], tensors[2], tensors[3], device, mode)
            samples.append(elapsed)
        rows[str(mode)] = {"median_ms": statistics.median(samples),
                           "check_a": float(first[0, 0].item()),
                           "check_b": float(second[0, 0].item())}
    rows["speedup"] = rows["False"]["median_ms"] / rows["True"]["median_ms"]
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Creating streams inside the timed path charges real orchestration cost but may not represent a reusable production pool. Report a second variant with streams created once. Compare full tensors or stable checksums within tolerance, not only one element, in the hardened benchmark.

Part 7 - Engineering Interpretation and Philosophy

Concurrency is an optimization with overhead and a resource ceiling. The default stream is often the correct baseline because it makes ordering cheap and obvious.

19.9.6 Exercise 05

Exercise Title: Demonstrate That Python Execution Can Continue While CUDA Work Remains Queued on a Custom Stream

Part 1 - Exercise Contract and Why It Matters

Submit a long GPU workload, return immediately to Python, perform deterministic CPU work, then wait for stream completion. Measure host submission, CPU progress, and completed wall time separately.

Part 2 - System and Memory Mental Model

Kernel launches enqueue work and normally return before GPU completion. The CPU can prepare another batch, update queues, or execute independent logic. Any operation that reads a CUDA result on the host, such as `item()`, creates a completion requirement and can hide the wait.

Part 3 - Quantitative Prediction Before Execution

Predict a small launch time, a larger stream-completion time, and a nonzero count of CPU work units completed between them. Compare total time with executing CPU work only after stream synchronization.

Part 4 - Experimental Design and Instrumentation

Warm CUDA, submit on one custom stream, record an event after the workload, and perform bounded CPU work until `event.query()` succeeds. Avoid accessing the CUDA result before completion.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

def cpu_unit(value):
    for index in range(5000):
        value = (value * 1103515245 + index + 12345) % 2147483648
    return value

def E05(size=3072):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    stream = torch.cuda.Stream(device=device)
    left = torch.randn((size, size), device=device)
    right = torch.randn((size, size), device=device)
    torch.cuda.synchronize(device)
    done = torch.cuda.Event()
    wall_start = time.perf_counter()
    with torch.cuda.stream(stream):
        result = left @ right
        done.record(stream)
    host_return = time.perf_counter()
    units = 0
    state = 1
    while not done.query():
        state = cpu_unit(state)
        units += 1
    completed = time.perf_counter()
    return {"submission_ms": (host_return - wall_start) * 1000.0,
            "completion_ms": (completed - wall_start) * 1000.0,
            "cpu_units": units, "cpu_state": state,
            "gpu_check": float(result[0, 0].item())}
```

Part 6 - Measurement, Validation, and Failure Analysis

Polling itself consumes CPU cycles and is used here for observation, not as the default production policy. If no CPU unit completes, reduce work-unit size or increase GPU work safely. Ensure input creation completed before timing.

Part 7 - Engineering Interpretation and Philosophy

Asynchronous execution creates time the host can use, but only useful pipeline work converts that time into throughput. Busy waiting merely proves availability.

19.9.7 Exercise 06

Exercise Title: Use stream.query() to Poll CUDA Work Completion Without Blocking the CPU Thread

Part 1 - Exercise Contract and Why It Matters

Poll stream completion while doing bounded host work and compare aggressive polling with backoff. Understand what `query()` reports and what it does not guarantee about unrelated streams.

Part 2 - System and Memory Mental Model

`stream.query()` returns whether all currently submitted work in that stream has completed. It does not block and says nothing about work submitted later or work in other streams. Frequent queries add runtime overhead and can waste a CPU core.

Part 3 - Quantitative Prediction Before Execution

Predict query count under no backoff and under a short sleep. Estimate detection latency added by backoff. Choose a policy based on CPU cost and responsiveness.

Part 4 - Experimental Design and Instrumentation

Submit a known workload, poll with timestamp and count, and access the result only after query returns true. Repeat with useful CPU chunks and with controlled sleep. Record completion-detection lag.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

def E06(size=3072, sleep_seconds=0.0005):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    stream = torch.cuda.Stream(device=device)
    left = torch.randn((size, size), device=device)
    right = torch.randn((size, size), device=device)
    torch.cuda.synchronize(device)
    with torch.cuda.stream(stream):
        result = left @ right
    polls = 0
    start = time.perf_counter()
    while not stream.query():
        polls += 1
        time.sleep(sleep_seconds)
    detected = time.perf_counter()
    return {"polls": polls,
            "detected_completion_ms": (detected - start) * 1000.0,
            "sleep_seconds": sleep_seconds,
            "check": float(result[0, 0].item())}
```

Part 6 - Measurement, Validation, and Failure Analysis

A true query result applies to work already placed in that stream at the query instant. Concurrent producers could submit more work afterward. Sleeping reduces CPU cost but

delays detection. Event query is preferable when completion of a specific point, rather than the entire stream, is required.

Part 7 - Engineering Interpretation and Philosophy

Polling trades responsiveness for CPU consumption. Completion interfaces should match the granularity of the dependency and the operational cost of waiting.

19.9.8 Exercise 07

Exercise Title: Use stream.synchronize() and Compare Stream-Level Synchronization Against Device-Wide torch.cuda.synchronize()

Part 1 - Exercise Contract and Why It Matters

Show that stream-level synchronization can release the host after required work while unrelated work continues elsewhere. Compare it with a device-wide barrier using reconstructed equivalent trials.

Part 2 - System and Memory Mental Model

Waiting on one stream covers only the work queued there. A device-wide barrier instead waits for every stream on that device and can couple unrelated tasks. The implementation in Part 5 calls the narrow and broad PyTorch methods explicitly. An event can be narrower still by marking one point inside a stream.

Part 3 - Quantitative Prediction Before Execution

Place short required work in stream A and longer unrelated work in stream B. Predict that waiting on A returns near T_A, whereas device synchronization returns near $\max(T_A, T_B)$.

Part 4 - Experimental Design and Instrumentation

Build a function that reconstructs inputs and streams for each policy. Warm up, submit both tasks, then time only the selected wait. After measuring A-only wait, drain B before cleanup without including that drain in the required latency.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

def one_wait_policy(device, wait_for_device, short_size=1024, long_size=3072):
    stream_a = torch.cuda.Stream(device=device)
    stream_b = torch.cuda.Stream(device=device)
    short_left = torch.randn((short_size, short_size), device=device)
    short_right = torch.randn((short_size, short_size), device=device)
    long_left = torch.randn((long_size, long_size), device=device)
    long_right = torch.randn((long_size, long_size), device=device)
    torch.cuda.synchronize(device)
    with torch.cuda.stream(stream_a):
        required = short_left @ short_right
    with torch.cuda.stream(stream_b):
        unrelated = long_left @ long_right
```

```
    start = time.perf_counter()
    if wait_for_device:
        torch.cuda.synchronize(device)
    else:
        stream_a.synchronize()
    wait_ms = (time.perf_counter() - start) * 1000.0
    stream_b.synchronize()
    required_check = float(required[0, 0].item())
    unrelated_check = float(unrelated[0, 0].item())
    return wait_ms, required_check, unrelated_check

def E07():
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    stream_wait = one_wait_policy(device, False)
    device_wait = one_wait_policy(device, True)
    return {"stream_wait_ms": stream_wait[0],
            "device_wait_ms": device_wait[0],
            "checks": [stream_wait[1], stream_wait[2],
                       device_wait[1], device_wait[2]]}
```

Part 6 - Measurement, Validation, and Failure Analysis

Task durations can differ across reconstructed trials, so repeat and randomize policy order. If the long kernel monopolizes GPU resources, it may delay stream A despite no dependency. That scheduling effect is distinct from host barrier scope.

Part 7 - Engineering Interpretation and Philosophy

Synchronization defines which unrelated work becomes part of your latency. Choose the narrowest wait that proves the data dependency you actually need.

19.9.9 Exercise 08

Exercise Title: Create a Race Condition by Consuming a Tensor on the Default Stream Before Its Producer Stream Has Finished

Part 1 - Exercise Contract and Why It Matters

Create a controlled cross-stream race with disposable tensors. Delay a producer stream, enqueue a write, and consume the output immediately on the default stream without a dependency. Detect stale or inconsistent results without using real application data.

Part 2 - System and Memory Mental Model

Operations within each stream are ordered, but the default stream does not automatically wait for arbitrary custom-stream work. A Python reference proves storage exists; it does not prove the producer kernel completed. The consumer may read uninitialized or previous contents.

Part 3 - Quantitative Prediction Before Execution

Initialize output to -1, delay the producer with large work, then enqueue a fill with 7. Predict the unsafe default-stream sum may reflect -1, 7, or timing-dependent behavior. A correct dependency must always produce $7N$.

Part 4 - Experimental Design and Instrumentation

Use bounded synthetic sizes and multiple trials. Synchronize before setup and after collecting results. Record whether the unsafe result differs from expected; absence of failure does not prove safety.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def E08(numel=16 * 1024 * 1024, blocker_size=3072, trials=10):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    producer = torch.cuda.Stream(device=device)
    rows = []
    for trial in range(trials):
        output = torch.full((numel,), -1.0, device=device)
        left = torch.randn((blocker_size, blocker_size), device=device)
        right = torch.randn((blocker_size, blocker_size), device=device)
        torch.cuda.synchronize(device)
        with torch.cuda.stream(producer):
            blocker = left @ right
            output.fill_(7.0)
        unsafe_sum = output.sum()  # No default-stream dependency.
        torch.cuda.synchronize(device)
        observed = float(unsafe_sum.item())
        expected = float(7.0 * numel)
        rows.append({"trial": trial, "observed": observed,
                     "expected": expected,
                     "race_seen": observed != expected,
                     "blocker_check": float(blocker[0, 0].item())})
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

This intentionally unsafe example may pass on favorable schedules. Increasing bounded blocker work widens the window. The final synchronization makes result inspection safe but cannot repair an already enqueued consumer that ran too early.

Part 7 - Engineering Interpretation and Philosophy

Race freedom is a property of declared ordering, not repeated successful runs. Cross-stream dataflow must be represented by a CUDA dependency.

19.9.10 Exercise 09

Exercise Title: Fix a Cross-Stream Dependency with wait_stream() and Verify Correct Execution Without Global Synchronization

Part 1 - Exercise Contract and Why It Matters

Repair E08 by making the default stream wait for producer work. Prove correct consumption without a device-wide barrier and identify the breadth of `wait_stream()`.

Part 2 - System and Memory Mental Model

`consumer.wait_stream(producer)` enqueues a dependency so future work in the consumer waits for work already submitted to the producer. It does not block Python. It is broader than waiting for a specifically recorded event if the producer contains unrelated earlier work.

Part 3 - Quantitative Prediction Before Execution

Predict the sequence: producer blocker, producer fill, consumer wait, consumer sum. Expected output is always $7N$. Compare host submission time with global synchronization to show the wait remains device-side.

Part 4 - Experimental Design and Instrumentation

Create tensors, submit producer operations, call wait on the current default stream, and enqueue the reduction. Synchronize only before CPU inspection. Repeat with deterministic sentinels.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def E09(numel=16 * 1024 * 1024, blocker_size=2048):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    producer = torch.cuda.Stream(device=device)
    consumer = torch.cuda.current_stream(device)
    output = torch.full((numel,), -1.0, device=device)
    left = torch.randn((blocker_size, blocker_size), device=device)
    right = torch.randn((blocker_size, blocker_size), device=device)
    torch.cuda.synchronize(device)
    with torch.cuda.stream(producer):
        blocker = left @ right
        output.fill_(7.0)
    consumer.wait_stream(producer)
    safe_sum = output.sum()
    output.record_stream(consumer)
    consumer.synchronize()
    expected = float(7.0 * numel)
    return {"observed": float(safe_sum.item()), "expected": expected,
            "validated": float(safe_sum.item()) == expected,
            "blocker_check": float(blocker[0, 0].item())}
```

Part 6 - Measurement, Validation, and Failure Analysis

Call `wait_stream` before submitting the consumer, not after. The wait covers producer work submitted at that point; later producer submissions are not automatically included. For a precise point in a long producer stream, use an event as in E10.

Part 7 - Engineering Interpretation and Philosophy

Device-side waits preserve host concurrency while enforcing dataflow. Their scope should match the producer work the consumer truly depends on.

19.9.11 Exercise 10

Exercise Title: Record a CUDA Event on One Stream and Make Another Stream Wait Only for the Required Dependency

Part 1 - Exercise Contract and Why It Matters

Replace a broad stream wait with an event marking the exact producer milestone. Let the producer continue unrelated work while the consumer begins as soon as its required tensor is ready.

Part 2 - System and Memory Mental Model

An event recorded in a stream becomes complete after all preceding work in that stream. `consumer.wait_event(event)` delays later consumer operations until that point without blocking the host. Producer work submitted after the event is not part of the dependency.

Part 3 - Quantitative Prediction Before Execution

Draw producer operations P_1, event, P_2, and consumer operation C depending only on P_1. Predict that C may overlap P_2, whereas `wait_stream` after both were queued would not permit that schedule.

Part 4 - Experimental Design and Instrumentation

Record the event immediately after producing the shared tensor. Submit unrelated producer work after it. Make consumer wait on the event, use the tensor, and validate after consumer completion. Profile to verify the intended overlap.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def E10(numel=8 * 1024 * 1024, matrix_size=2048):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    producer = torch.cuda.Stream(device=device)
    consumer = torch.cuda.Stream(device=device)
    source = torch.arange(numel, dtype=torch.float32, device=device)
    left = torch.randn((matrix_size, matrix_size), device=device)
    right = torch.randn((matrix_size, matrix_size), device=device)
    torch.cuda.synchronize(device)
```

```
required_ready = torch.cuda.Event()
with torch.cuda.stream(producer):
    produced = source * 2.0
    required_ready.record(producer)
    unrelated = left @ right
consumer.wait_event(required_ready)
with torch.cuda.stream(consumer):
    consumed = produced + 5.0
consumed.record_stream(consumer)
consumer.synchronize()
producer.synchronize()
return {"first": float(consumed[0].item()),
        "last": float(consumed[-1].item()),
        "unrelated_check": float(unrelated[0, 0].item())}
```

Part 6 - Measurement, Validation, and Failure Analysis

Expected consumed values are 5 and $2(N-1)+5$. The event protects execution order, while `record_stream` protects storage lifetime during consumer use. A profiler trace is needed to confirm that unrelated work actually overlaps on the tested GPU.

Part 7 - Engineering Interpretation and Philosophy

Events express dataflow milestones. Precise milestones preserve more concurrency than treating an entire stream as one indivisible dependency.

19.9.12 Exercise 11

Exercise Title: Build a Three-Stream Dependency Graph and Enforce Correct Producer-Consumer Ordering Using CUDA Events

Part 1 - Exercise Contract and Why It Matters

Build a fork–join graph: streams A and B produce independent tensors; stream C waits for both and combines them. Validate ordering without global synchronization inside the graph.

Part 2 - System and Memory Mental Model

Events transform implicit tensor relationships into edges of a directed acyclic graph. A and B may overlap. C cannot begin the join operation until both readiness events complete. The host can enqueue the whole graph without waiting at each edge.

Part 3 - Quantitative Prediction Before Execution

For producer times T_A, T_B and join time T_C, predict ideal critical path $\max(T_A, T_B) + T_C$. List invalid schedules when either event edge is removed.

Part 4 - Experimental Design and Instrumentation

Create three streams and two events. Ensure input creation is complete before forking. Record producer events after outputs, enqueue both waits in C, then combine. Validate values and profile stream lanes.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def E11(numel=8 * 1024 * 1024):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    stream_a = torch.cuda.Stream(device=device)
    stream_b = torch.cuda.Stream(device=device)
    stream_c = torch.cuda.Stream(device=device)
    source_a = torch.arange(numel, dtype=torch.float32, device=device)
    source_b = torch.ones(numel, dtype=torch.float32, device=device)
    torch.cuda.synchronize(device)
    ready_a = torch.cuda.Event()
    ready_b = torch.cuda.Event()
    with torch.cuda.stream(stream_a):
        output_a = source_a * 2.0
        ready_a.record(stream_a)
    with torch.cuda.stream(stream_b):
        output_b = source_b * 3.0
        ready_b.record(stream_b)
    stream_c.wait_event(ready_a)
    stream_c.wait_event(ready_b)
    with torch.cuda.stream(stream_c):
        joined = output_a + output_b
    output_a.record_stream(stream_c)
    output_b.record_stream(stream_c)
    stream_c.synchronize()
    return {"first": float(joined[0].item()),
            "last": float(joined[-1].item()),
            "expected_last": float(2 * (numel - 1) + 3)}
```

Part 6 - Measurement, Validation, and Failure Analysis

The two waits must be issued before the join. If one producer fails, the join may remain blocked; production schedulers need error propagation and timeouts. Tensor lifetime crosses into stream C and must be protected independently of the event dependency.

Part 7 - Engineering Interpretation and Philosophy

Multi-stream programs are dependency graphs. Once represented explicitly, they can be validated, profiled, scheduled, and recovered rather than debugged as timing accidents.

19.9.13 Exercise 12

Exercise Title: Reproduce an Unsafe Cross-Stream Tensor Lifetime Bug and Explain Why Allocator Reuse Can Corrupt Asynchronous Execution

Part 1 - Exercise Contract and Why It Matters

Reproduce a bounded, nondeterministic storage-lifetime failure with disposable CUDA tensors. Submit side-stream work that reads a tensor, delete the last Python reference, allocate a replacement on the creation stream, and observe possible allocator reuse and corruption.

Part 2 - System and Memory Mental Model

The caching allocator knows the stream where storage was allocated and the streams recorded as users. If side-stream use is not recorded, freeing the tensor on its creation stream can make its block available while the side stream still plans to read it. A new allocation may reuse the same address and overwrite data. Execution dependencies and allocator lifetime are separate obligations.

Part 3 - Quantitative Prediction Before Execution

Fill source with 1, delay the side stream, and plan a sum. Delete source and fill a same-sized replacement with 9. Predict correct sum N, corrupted sum $9N$, or an intermediate outcome. Pointer equality strengthens evidence of reuse but is not required for the conceptual bug.

Part 4 - Experimental Design and Instrumentation

Use fixed bounded sizes and several trials. Capture source and replacement data pointers, retain the result tensor, and synchronize before inspection. Do not use real model tensors. Passing trials do not prove safety.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def E12(numel=32 * 1024 * 1024, blocker_size=3072, trials=10):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    side = torch.cuda.Stream(device=device)
    rows = []
    for trial in range(trials):
        source = torch.full((numel,), 1.0, device=device)
        source_pointer = source.data_ptr()
        left = torch.randn((blocker_size, blocker_size), device=device)
        right = torch.randn((blocker_size, blocker_size), device=device)
        torch.cuda.synchronize(device)
        with torch.cuda.stream(side):
            blocker = left @ right
            result = source.sum()
        del source  # Intentionally unsafe: side-stream use was not recorded.
        replacement = torch.empty(numel, dtype=torch.float32, device=device)
        replacement_pointer = replacement.data_ptr()
        replacement.fill_(9.0)
        torch.cuda.synchronize(device)
        observed = float(result.item())
        rows.append({"trial": trial, "observed": observed,
                     "expected": float(numel),
                     "same_pointer": source_pointer == replacement_pointer,
                     "blocker_check": float(blocker[0, 0].item())})
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Allocator behavior and scheduling are nondeterministic; no corruption in ten trials is not proof. A matching pointer plus wrong sum is strong reproduction. The final global synchronization only makes observation safe; it cannot undo a prematurely reused block.

Part 7 - Engineering Interpretation and Philosophy

Memory correctness includes future GPU readers that Python can no longer see. Allocator-aware lifetime tracking is part of asynchronous program correctness.

19.9.14 Exercise 13

Exercise Title: Use Tensor.record_stream() to Protect Tensor Storage Used by a Side Stream from Premature Allocator Reuse

Part 1 - Exercise Contract and Why It Matters

Repair E12 by recording side-stream use before releasing the source. Demonstrate that the allocator delays reuse until the recorded stream completes without forcing an immediate host wait.

Part 2 - System and Memory Mental Model

`tensor.record_stream(side)` informs the caching allocator that the tensor's storage is in use on that stream. After Python releases the tensor, the block cannot be safely recycled until recorded work completes. This manages storage lifetime; it does not create a producer–consumer execution dependency.

Part 3 - Quantitative Prediction Before Execution

Repeat the source/replacement experiment. Predict correct sum N in every trial and expect the replacement often to receive a different block while side work is pending. Memory may remain reserved longer as the cost of safety.

Part 4 - Experimental Design and Instrumentation

Keep workload identical to E12 except for one `record_stream` call. Measure pointer reuse, result correctness, peak allocated memory, and host submission time. Run from a clean allocator state for controlled comparisons.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def E13(numel=32 * 1024 * 1024, blocker_size=3072, trials=10):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    side = torch.cuda.Stream(device=device)
    rows = []
    for trial in range(trials):
        source = torch.full((numel,), 1.0, device=device)
        source_pointer = source.data_ptr()
```

```
        left = torch.randn((blocker_size, blocker_size), device=device)
        right = torch.randn((blocker_size, blocker_size), device=device)
        torch.cuda.synchronize(device)
        with torch.cuda.stream(side):
            blocker = left @ right
            result = source.sum()
        source.record_stream(side)
        del source
        replacement = torch.empty(numel, dtype=torch.float32, device=device)
        replacement_pointer = replacement.data_ptr()
        replacement.fill_(9.0)
        torch.cuda.synchronize(device)
        rows.append({"trial": trial, "observed": float(result.item()),
                     "expected": float(numel),
                     "same_pointer": source_pointer == replacement_pointer,
                     "blocker_check": float(blocker[0, 0].item())})
    return rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Every observed sum must equal N. A different replacement pointer is expected but not itself the correctness criterion. Recording the wrong stream or recording after storage has been released does not protect use. Version and allocator backend should be recorded for performance interpretation.

Part 7 - Engineering Interpretation and Philosophy

`record_stream` lets the allocator participate in the dependency model. It preserves concurrency by delaying reuse instead of blocking the host.

19.9.15 Exercise 14

Exercise Title: Compare Explicit record_stream() Lifetime Management Against Manual Stream Synchronization for Temporary CUDA Buffers

Part 1 - Exercise Contract and Why It Matters

Compare two safe policies for temporary buffers used on a side stream: allocator lifetime recording and host-side stream synchronization before release. Quantify submission latency, total time, overlap, and memory retention.

Part 2 - System and Memory Mental Model

Manual synchronization waits immediately, simplifying reuse but blocking the host and potentially draining unrelated side-stream work. Recording the stream allows the host to continue and the allocator to defer recycling. The latter may increase temporary memory pressure because old and new blocks coexist.

Part 3 - Quantitative Prediction Before Execution

Predict lower host submission time but possibly higher peak allocation for the recorded policy. Predict similar numerical results and completed compute time when workloads are otherwise identical.

Part 4 - Experimental Design and Instrumentation

Reconstruct equivalent trials, reset peak statistics, use fixed tensors, and time host return separately from final completion. Track source/replacement pointers and result checksums.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

def run_lifetime_policy(device, use_record_stream, numel, matrix_size):
    side = torch.cuda.Stream(device=device)
    torch.cuda.reset_peak_memory_stats(device)
    source = torch.ones(numel, device=device)
    left = torch.randn((matrix_size, matrix_size), device=device)
    right = torch.randn((matrix_size, matrix_size), device=device)
    torch.cuda.synchronize(device)
    start = time.perf_counter()
    with torch.cuda.stream(side):
        blocker = left @ right
        result = source.sum()
    if use_record_stream:
        source.record_stream(side)
    else:
        side.synchronize()
    del source
    submitted = time.perf_counter()
    replacement = torch.empty(numel, device=device)
    replacement.fill_(9.0)
    torch.cuda.synchronize(device)
    completed = time.perf_counter()
    return {"submission_ms": (submitted - start) * 1000.0,
            "completion_ms": (completed - start) * 1000.0,
            "peak_bytes": torch.cuda.max_memory_allocated(device),
            "result": float(result.item()),
            "blocker_check": float(blocker[0, 0].item())}

def E14(numel=32 * 1024 * 1024, matrix_size=3072):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    return {"record_stream": run_lifetime_policy(
                device, True, numel, matrix_size),
            "manual_sync": run_lifetime_policy(
                device, False, numel, matrix_size)}
```

Part 6 - Measurement, Validation, and Failure Analysis

Both results must equal N. Peak statistics include inputs, blocker, result, and replacement, so compare only identical workloads. Repetition order and allocator cache state can bias memory and timing; use clean-process repetitions for a defensible report.

Part 7 - Engineering Interpretation and Philosophy

Safety has multiple implementations. Choose immediate waiting when simplicity and memory pressure dominate; choose recorded lifetime when host progress and overlap justify deferred reuse.

19.9.16 Exercise 15

Exercise Title: Create Normal- and High-Priority CUDA Streams and Measure How Stream Priority Changes Scheduling Under Competing Workloads

Part 1 - Exercise Contract and Why It Matters

Create competing normal- and high-priority streams, queue background kernels, then measure completion latency of urgent work. Treat priority as a scheduling hint, not a deadline or preemption guarantee.

Part 2 - System and Memory Mental Model

CUDA exposes a device-specific priority range in which numerically lower values represent greater priority. Higher priority can influence which pending blocks are scheduled, but it does not necessarily preempt a kernel already running. Large saturating kernels can therefore delay urgent work despite stream priority.

Part 3 - Quantitative Prediction Before Execution

Predict urgent completion when its stream uses normal priority and when it uses the greatest supported priority. Expect the effect to be clearest with many small background kernels that leave scheduling boundaries.

Part 4 - Experimental Design and Instrumentation

Query the supported range, reconstruct identical workload cases, warm up, queue background work before urgent work, and time urgent-event completion separately from total completion. Repeat and report distributions.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

def priority_case(device, urgent_priority, jobs=64, size=512):
    background = torch.cuda.Stream(device=device, priority=0)
    urgent = torch.cuda.Stream(device=device, priority=urgent_priority)
    left = torch.randn((size, size), device=device)
    right = torch.randn((size, size), device=device)
    torch.cuda.synchronize(device)
    urgent_done = torch.cuda.Event()
    background_done = torch.cuda.Event()
    start = time.perf_counter()
    with torch.cuda.stream(background):
        for _ in range(jobs):
            background_result = left @ right
```

```
        background_done.record(background)
    with torch.cuda.stream(urgent):
        urgent_result = left @ right
        urgent_done.record(urgent)
    urgent_done.synchronize()
    urgent_ms = (time.perf_counter() - start) * 1000.0
    background_done.synchronize()
    total_ms = (time.perf_counter() - start) * 1000.0
    return {"urgent_ms": urgent_ms, "total_ms": total_ms,
            "urgent_check": float(urgent_result[0, 0].item()),
            "background_check": float(background_result[0, 0].item())}

def E15():
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    least_priority, greatest_priority = torch.cuda.get_stream_priority_range()
    return {"priority_range": [least_priority, greatest_priority],
            "normal": priority_case(device, 0),
            "high": priority_case(device, greatest_priority)}
```

Part 6 - Measurement, Validation, and Failure Analysis

Priority effects can be small or absent on some workloads. Verify the reported range and do not assume -1 is universally the greatest supported value. Create multiple repetitions and compare urgent latency, not only total throughput.

Part 7 - Engineering Interpretation and Philosophy

Priority expresses preference under contention, not certainty. Real-time correctness still requires bounded kernels, admission control, and measured tail latency.

19.9.17 Exercise 16

Exercise Title: Profile Multiple CUDA Streams with PyTorch Profiler and Verify Actual Kernel Overlap Instead of Assuming Concurrency

Part 1 - Exercise Contract and Why It Matters

Capture an execution trace with labeled work on multiple streams and verify whether kernels overlap in time. Connect visible overlap to the sequential and multi-stream benchmarks.

Part 2 - System and Memory Mental Model

Wall-clock speedup can be distorted by warmup, clock state, or measurement scope. A trace exposes per-stream lanes, kernel intervals, gaps, synchronization, and CPU submissions. Overlap exists only when intervals execute concurrently, not merely when launches appear near each other on the CPU timeline.

Part 3 - Quantitative Prediction Before Execution

Predict stream lanes and kernel counts for two independent chains. Estimate the overlap interval and compare it with the difference between sequential and multi-stream time.

Part 4 - Experimental Design and Instrumentation

Use `record_function` labels, CPU and CUDA profiler activities, warmup outside the active profile, and an exported Chrome trace. Measure profiler overhead against an unprofiled run.

Part 5 - PyTorch/CUDA Implementation

```
import torch
from torch.profiler import ProfilerActivity, record_function

def E16(trace_path="dmm07_multistream_trace.json", size=1536):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    stream_a = torch.cuda.Stream(device=device)
    stream_b = torch.cuda.Stream(device=device)
    a = torch.randn((size, size), device=device)
    b = torch.randn((size, size), device=device)
    c = torch.randn((size, size), device=device)
    d = torch.randn((size, size), device=device)
    torch.cuda.synchronize(device)
    activities = [ProfilerActivity.CPU, ProfilerActivity.CUDA]
    with torch.profiler.profile(activities=activities,
                                record_shapes=True) as profiler:
        with record_function("stream_a_chain"):
            with torch.cuda.stream(stream_a):
                result_a = torch.relu(a @ b)
        with record_function("stream_b_chain"):
            with torch.cuda.stream(stream_b):
                result_b = torch.relu(c @ d)
        stream_a.synchronize()
        stream_b.synchronize()
    profiler.export_chrome_trace(trace_path)
    table = profiler.key_averages().table(
        sort_by="self_cuda_time_total", row_limit=30)
    return {"table": table, "trace_path": trace_path,
            "checks": [float(result_a[0, 0].item()),
                       float(result_b[0, 0].item())]}
```

Part 6 - Measurement, Validation, and Failure Analysis

Profiler overhead changes timing, so use it for causal structure and an unprofiled benchmark for final speed. Kernel names and lanes vary by backend and version. Lack of overlap may indicate resource saturation rather than a missing stream context.

Part 7 - Engineering Interpretation and Philosophy

Concurrency claims require timeline evidence. Profiling converts an architectural intention into an observable scheduling fact.

19.9.18 Exercise 17

Exercise Title: Stress-Test Multi-Stream Execution with Shared Tensors and Detect Missing Dependencies, Race Conditions, and Unnecessary Synchronization

Part 1 - Exercise Contract and Why It Matters

Stress a bounded ring of shared CUDA buffers across producer and consumer streams. Run a safe dependency mode and an injected-failure mode, validate sentinels, and measure the cost of optional global barriers.

Part 2 - System and Memory Mental Model

Each reusable slot needs two edges: consumer completion before the next producer overwrite, and producer readiness before the next consumer read. Missing either edge creates a write-after-read or read-before-write race. A global barrier fixes ordering broadly but removes inter-slot concurrency.

Part 3 - Quantitative Prediction Before Execution

For K slots and N iterations, predict bounded storage KB. In safe mode every checksum equals sentinel times element count. In injected mode at least some schedules may fail, though no failure is guaranteed.

Part 4 - Experimental Design and Instrumentation

Use fixed seeds, deterministic sentinel values, per-slot readiness and reuse events, one final synchronization, and an optional dependency omission. Record failures and global-sync count. Never use production tensors for injected races.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def run_stress(device, inject_failure, iterations=100,
               slots_count=4, numel=1024 * 1024):
    producer = torch.cuda.Stream(device=device)
    consumer = torch.cuda.Stream(device=device)
    slots = []
    reuse_events = []
    for _ in range(slots_count):
        slots.append(torch.empty(numel, device=device))
        reuse_events.append(None)
    results = []
    expected = []
    for step in range(iterations):
        slot_index = step % slots_count
        reusable = reuse_events[slot_index]
        if reusable is not None:
            producer.wait_event(reusable)
        ready = torch.cuda.Event()
        with torch.cuda.stream(producer):
            slots[slot_index].fill_(float(step + 1))
```

```
            ready.record(producer)
        should_wait = not (inject_failure and step % 10 == 0)
        if should_wait:
            consumer.wait_event(ready)
        consumed = torch.cuda.Event()
        with torch.cuda.stream(consumer):
            checksum = slots[slot_index].sum()
            consumed.record(consumer)
        slots[slot_index].record_stream(consumer)
        reuse_events[slot_index] = consumed
        results.append(checksum)
        expected.append(float((step + 1) * numel))
    torch.cuda.synchronize(device)
    failures = []
    for index, result in enumerate(results):
        if float(result.item()) != expected[index]:
            failures.append(index)
    return failures

def E17():
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    return {"safe_failures": run_stress(device, False),
            "injected_failures": run_stress(device, True)}
```

Part 6 - Measurement, Validation, and Failure Analysis

Safe mode must have zero failures. Injected mode can pass under favorable scheduling, so it demonstrates missing proof even if no mismatch appears. Add a profiler and a version with device-wide synchronization to quantify unnecessary serialization.

Part 7 - Engineering Interpretation and Philosophy

Reusable buffers turn stream programming into a state machine. Stress tests are most useful when every transition has an explicit invariant that can fail.

19.9.19 Exercise 18

Exercise Title: Build a Reusable PyTorch Stream Scheduler That Assigns Independent GPU Tasks to Multiple Streams and Tracks Their Completion

Part 1 - Exercise Contract and Why It Matters

Implement a bounded round-robin stream scheduler. Submissions accept dependency tickets, run tensor-producing functions, record completion, protect input and output lifetimes, and return a ticket that supports host waiting or downstream stream attachment.

Part 2 - System and Memory Mental Model

The scheduler maps a task DAG onto a fixed stream pool. Dependency events enforce execution order. Ticket ownership retains outputs. Input `record_stream` calls protect buffers

while a selected stream uses them. A fixed pool bounds orchestration resources but not output memory, so pending task count also requires a limit.

Part 3 - Quantitative Prediction Before Execution

For S streams and Q pending tasks, predict stream handles, maximum in-flight tasks, and output memory. Independent tasks can overlap; dependent tasks follow the critical path of the DAG.

Part 4 - Experimental Design and Instrumentation

Submit independent tasks, a join task depending on them, and more tasks than streams to test reuse. Validate stream assignment, completion, outputs, and queue bound. Propagate exceptions instead of returning a never-completing ticket.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class TaskTicket:
    def __init__(self, output, ready, stream):
        self.output = output
        self.ready = ready
        self.stream = stream
    def wait_host(self):
        self.ready.synchronize()
        return self.output
    def attach(self, consumer_stream):
        consumer_stream.wait_event(self.ready)
        self.output.record_stream(consumer_stream)
        return self.output

class StreamScheduler:
    def __init__(self, device, stream_count, max_pending):
        self.device = device
        self.streams = []
        for _ in range(stream_count):
            self.streams.append(torch.cuda.Stream(device=device))
        self.max_pending = max_pending
        self.pending = []
        self.next_stream = 0
    def submit(self, function, arguments, dependencies=()):
        if len(self.pending) >= self.max_pending:
            self.drain_oldest()
        stream = self.streams[self.next_stream]
        self.next_stream = (self.next_stream + 1) % len(self.streams)
        for dependency in dependencies:
            stream.wait_event(dependency.ready)
        ready = torch.cuda.Event()
        with torch.cuda.stream(stream):
            output = function(*arguments)
```

```
            ready.record(stream)
        for argument in arguments:
            if torch.is_tensor(argument):
                argument.record_stream(stream)
        ticket = TaskTicket(output, ready, stream)
        self.pending.append(ticket)
        return ticket
    def drain_oldest(self):
        ticket = self.pending.pop(0)
        return ticket.wait_host()
    def wait_all(self):
        outputs = []
        while self.pending:
            outputs.append(self.drain_oldest())
        return outputs

def E18(size=1024):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    scheduler = StreamScheduler(device, stream_count=3, max_pending=8)
    a = torch.randn((size, size), device=device)
    b = torch.randn((size, size), device=device)
    torch.cuda.synchronize(device)
    first = scheduler.submit(torch.matmul, (a, b))
    second = scheduler.submit(torch.matmul, (b, a))
    joined = scheduler.submit(torch.add, (first.output, second.output),
                              dependencies=(first, second))
    output = joined.wait_host()
    return {"shape": tuple(output.shape),
            "check": float(output[0, 0].item())}
```

Part 6 - Measurement, Validation, and Failure Analysis

The scheduler records tensor arguments on the selected task stream and waits on dependency events before executing the join. Harden it to traverse nested arguments and outputs, remove completed tickets, and preserve exception state. The example focuses on the core ownership model.

Part 7 - Engineering Interpretation and Philosophy

A scheduler is valuable when it makes dependencies and capacity explicit. Round robin alone distributes submissions; tickets and bounds turn it into a reliable execution system.

19.9.20 Exercise 19

Exercise Title: Build and Benchmark a Dependency-Aware Multi-Stream CUDA Pipeline That Preserves Correctness While Minimizing Global Synchronization

Part 1 - Exercise Contract and Why It Matters

Integrate H2D copy, parallel inference, and D2H copy into an event-driven pipeline. Use bounded in-flight batches, pinned host storage, stream-specific dependencies, allocator lifetime protection, and a blocking reference for correctness and throughput comparison.

Part 2 - System and Memory Mental Model

The input-copy stream produces a device batch and records readiness. A selected compute stream waits for it, runs the model, and records output readiness. The output-copy stream waits only for that model result, writes a pinned host tensor, and records completion. Source references remain owned until H2D finishes; `record_stream` protects device tensors across stream boundaries. A maximum in-flight count provides backpressure and bounds memory.

Part 3 - Quantitative Prediction Before Execution

Estimate serial latency $T_{H2D} + T_C + T_{D2H}$ and ideal pipeline interval as the maximum stage time. Multiply per-batch host and device storage by the in-flight bound. Predict where more compute streams stop helping because the model saturates the GPU.

Part 4 - Experimental Design and Instrumentation

Use identical model state and pinned batches for sequential and pipelined paths. Warm up, run in evaluation/no-grad mode, report throughput and output parity, and profile overlap. Inject shape errors and low queue limits to test backpressure.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

class PipelineTicket:
    def __init__(self, host_source, host_output, done):
        self.host_source = host_source
        self.host_output = host_output
        self.done = done
    def wait(self):
        self.done.synchronize()
        return self.host_output

class MultiStreamPipeline:
    def __init__(self, model, device, compute_streams, max_inflight):
        self.model = model
        self.device = device
        self.h2d = torch.cuda.Stream(device=device)
        self.d2h = torch.cuda.Stream(device=device)
        self.compute = []
        for _ in range(compute_streams):
            self.compute.append(torch.cuda.Stream(device=device))
        self.max_inflight = max_inflight
        self.pending = []
        self.next_compute = 0
        self.completed_checksums = []
    def submit(self, host_batch):
```

```
        if not host_batch.is_pinned():
            raise ValueError("Pipeline input must be pinned")
        if len(self.pending) >= self.max_inflight:
            self.complete_oldest()
        input_ready = torch.cuda.Event()
        with torch.cuda.stream(self.h2d):
            device_input = host_batch.to(self.device, non_blocking=True)
            input_ready.record(self.h2d)
        compute_stream = self.compute[self.next_compute]
        self.next_compute = (self.next_compute + 1) % len(self.compute)
        compute_stream.wait_event(input_ready)
        output_ready = torch.cuda.Event()
        with torch.cuda.stream(compute_stream):
            device_output = self.model(device_input)
            output_ready.record(compute_stream)
        device_input.record_stream(compute_stream)
        host_output = torch.empty(device_output.shape,
                                  dtype=device_output.dtype,
                                  pin_memory=True)
        self.d2h.wait_event(output_ready)
        done = torch.cuda.Event()
        with torch.cuda.stream(self.d2h):
            host_output.copy_(device_output, non_blocking=True)
            done.record(self.d2h)
        device_output.record_stream(self.d2h)
        ticket = PipelineTicket(host_batch, host_output, done)
        self.pending.append(ticket)
        return ticket
    def complete_oldest(self):
        ticket = self.pending.pop(0)
        output = ticket.wait()
        checksum = float(output.float().sum().item())
        self.completed_checksums.append(checksum)
    def drain(self):
        while self.pending:
            self.complete_oldest()
        return list(self.completed_checksums)

def E19(pinned_batches, model, compute_streams=2, max_inflight=4):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    if len(pinned_batches) == 0:
        raise ValueError("At least one batch is required")
    device = torch.device("cuda:0")
    model = model.to(device)
    model.eval()
    torch.cuda.synchronize(device)
```

```
    baseline_checksums = []
    baseline_start = time.perf_counter()
    with torch.no_grad():
        for batch in pinned_batches:
            device_batch = batch.to(device, non_blocking=False)
            device_output = model(device_batch)
            host_output = device_output.cpu()
            baseline_checksums.append(float(host_output.float().sum().item()))
    torch.cuda.synchronize(device)
    baseline_seconds = time.perf_counter() - baseline_start

    pipeline = MultiStreamPipeline(
        model, device, compute_streams, max_inflight)
    start = time.perf_counter()
    with torch.no_grad():
        for batch in pinned_batches:
            pipeline.submit(batch)
    pipeline_checksums = pipeline.drain()
    seconds = time.perf_counter() - start
    outputs_match = len(baseline_checksums) == len(pipeline_checksums)
    for baseline, pipelined in zip(baseline_checksums, pipeline_checksums):
        tolerance = 1e-5 * max(1.0, abs(baseline))
        if abs(baseline - pipelined) > tolerance:
            outputs_match = False
    return {"baseline_seconds": baseline_seconds,
            "pipeline_seconds": seconds,
            "baseline_batches_s": len(pinned_batches) / baseline_seconds,
            "pipeline_batches_s": len(pinned_batches) / seconds,
            "completed_outputs": len(pipeline_checksums),
            "outputs_match": outputs_match}
```

Part 6 - Measurement, Validation, and Failure Analysis

The demonstration must be paired with a sequential blocking reference and full output comparisons within tolerance. Fresh pinned output allocation is simple but should become a bounded reusable pool in production. Stateful models or modules that mutate shared buffers may not be safe for concurrent forwards and must be tested or serialized.

Part 7 - Engineering Interpretation and Philosophy

The best multi-stream pipeline is not the one with the most streams. It is the one whose dependency graph is minimal, whose memory is bounded, whose lifetimes are provable, and whose measured critical path is shorter without changing the answer.

19.10 DMM08

Chapter Title: Compute and Data Transfer Overlap
Specialization: Dynamic Memory Management and CUDA Systems

19.10.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Overlapping Host-to-Device Transfers with GPU Computation

Part 1 - Exercise Contract and Why It Matters

Construct a falsifiable model of an input pipeline in which batch $n+1$ moves from host RAM to VRAM while the GPU computes batch n. The goal is not merely to use asynchronous-looking API calls. The goal is to prove that two independent engines are active during the same interval, that dependencies remain correct, and that reused buffers cannot be overwritten prematurely.

Part 2 - System and Memory Mental Model

True overlap requires five conditions: page-locked source memory, an asynchronous copy, different CUDA streams for copy and compute, hardware able to run copy and compute concurrently, and enough work to expose concurrency. A copy stream orders transfers; a compute stream orders kernels. An event connects only the required producer–consumer edge. There are two distinct lifetimes: the CPU may refill a pinned slot only after its H2D copy completes, while the transfer stream may overwrite a device slot only after computation using that slot completes. Confusing these events creates rare data corruption.

Part 3 - Quantitative Prediction Before Execution

Let T_p, T_c, and T_g denote CPU preparation, H2D copy, and GPU compute time. A serial iteration costs approximately $T_p + T_c + T_g$. After pipeline fill, an ideal three-stage pipeline approaches $\max(T_p, T_c, T_g)$ per batch. If preparation is excluded, the maximum copy–compute speedup is

$$S_{\max} = \frac{T_c + T_g}{\max(T_c, T_g)} \leq 2.$$

Record this prediction before measuring; otherwise ordinary noise can be mistaken for overlap.

Part 4 - Experimental Design and Instrumentation

Measure a serial baseline, then a warmup, steady-state, and drain phase for the overlapped version. Use CUDA events for device intervals, a synchronized host timer for end-to-end time, and a profiler timeline to prove simultaneous copy and kernel activity. Validate numerical output against the serial pipeline and report pinned RAM, input VRAM, throughput, and exposed copy time.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def E00(batch=2048, width=2048):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    copy_stream = torch.cuda.Stream(device=device)
    compute_stream = torch.cuda.current_stream(device)
    host = torch.randn((batch, width), pin_memory=True)
    gpu = torch.empty_like(host, device=device)
    copy_done = torch.cuda.Event()
```

```
    compute_done = torch.cuda.Event()
    compute_done.record(compute_stream)

    copy_stream.wait_event(compute_done)
    with torch.cuda.stream(copy_stream):
        gpu.copy_(host, non_blocking=True)
        copy_done.record(copy_stream)

    compute_stream.wait_event(copy_done)
    with torch.cuda.stream(compute_stream):
        result = torch.relu(gpu).sum()
        compute_done.record(compute_stream)

    compute_done.synchronize()
    expected = torch.relu(host).sum().item()
    observed = result.item()
    return {"observed": observed, "expected": expected,
            "absolute_error": abs(observed - expected)}
```

Part 6 - Measurement, Validation, and Failure Analysis

This one-batch skeleton demonstrates ordering, not steady-state speedup. A timeline with no overlap may simply mean the workload is too short. If the copy appears as a blocking memcpy, verify `is_pinned()`; if kernels wait, search for `item()`, printing CUDA tensors, or global synchronization. An output match is necessary but insufficient because a race may pass by chance.

Part 7 - Engineering Interpretation and Philosophy

Overlap is a scheduling theorem supported by evidence, not a property granted by one keyword. Optimize the critical path, express the smallest correct dependency graph, and treat buffer ownership as part of correctness. The rest of this chapter turns that discipline into reusable machinery.

19.10.2 Exercise 01

Exercise Title: Measure a Sequential Copy-Then-Compute Pipeline and Establish a Reliable Baseline for Total Iteration Time

Part 1 - Exercise Contract and Why It Matters

Establish the reference that every later optimization must beat. Measure a strict H2D-copy-then-compute loop with correct warmup and synchronization, and retain its outputs for parity checking.

Part 2 - System and Memory Mental Model

CUDA submission is asynchronous with respect to Python. A host timer around enqueue calls measures submission latency unless completion is forced at the measurement boundary. The serial baseline deliberately places copy and compute on one stream, so stream ordering prevents overlap without adding a barrier inside every iteration.

Part 3 - Quantitative Prediction Before Execution

Predict $T_{serial} \approx T_{copy} + T_{compute}$ per steady iteration. Convert elapsed time into batches/s and GiB/s of input traffic. Repeat enough iterations that fixed launch and first-use costs are a small fraction of the total.

Part 4 - Experimental Design and Instrumentation

Preallocate pinned CPU batches and one CUDA buffer. Warm up the exact kernels, synchronize once before and once after the timed region, and repeat several trials. Report median and range rather than selecting the fastest run.

Part 5 - PyTorch/CUDA Implementation

```
import statistics
import time
import torch

def E01(steps=40, batch=1024, width=2048, trials=5):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    host = torch.randn((steps, batch, width), pin_memory=True)
    gpu = torch.empty((batch, width), device=device)
    weight = torch.randn((width, 512), device=device)

    for index in range(5):
        gpu.copy_(host[index], non_blocking=True)
        warm = torch.relu(gpu @ weight).mean()
    torch.cuda.synchronize(device)

    samples = []
    checksum = None
    for trial in range(trials):
        torch.cuda.synchronize(device)
        started = time.perf_counter()
        total = torch.zeros((), device=device)
        for index in range(steps):
            gpu.copy_(host[index], non_blocking=True)
            total = total + torch.relu(gpu @ weight).mean()
        torch.cuda.synchronize(device)
        samples.append(time.perf_counter() - started)
        checksum = float(total.item())
    median_s = statistics.median(samples)
    input_bytes = host[0].numel() * host.element_size() * steps
    return {"median_s": median_s, "batches_per_s": steps / median_s,
            "input_GiB_per_s": input_bytes / median_s / (1024 ** 3),
            "trials_s": samples, "checksum": checksum}
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not include tensor construction or first CUDA initialization unless deployment startup is the stated target. A surprisingly tiny time indicates missing final synchronization. Compare checksums later with tolerances appropriate to reduction order and dtype.

Part 7 - Engineering Interpretation and Philosophy

A credible optimization begins with a boring baseline. Its value is not speed; it is a controlled counterfactual that prevents claims based on different data, different kernels, or incomplete timing.

19.10.3 Exercise 02

Exercise Title: Separate Transfer Time and Compute Time with CUDA Events and Predict the Maximum Possible Overlap Speedup

Part 1 - Exercise Contract and Why It Matters

Measure copy and compute independently on the GPU timeline, then calculate an upper bound before constructing the pipeline. This separates an architectural limit from implementation quality.

Part 2 - System and Memory Mental Model

CUDA events are timestamp markers recorded in stream order. Elapsed time is valid only after the ending event completes. Events avoid the host scheduling noise of per-operation wall timers, but they still perturb very short workloads and must be placed on the stream containing the measured work.

Part 3 - Quantitative Prediction Before Execution

For measured C ms of copying and G ms of computation, serial time is $C + G$, ideal overlapped time is $\max(C, G)$, and the bound is $(C + G)/\max(C, G)$. The shorter stage can be hidden completely; the longer stage remains on the critical path.

Part 4 - Experimental Design and Instrumentation

Time repeated copies in one event interval and repeated computations in another. Use the same buffers, shapes, and operation as E01. Divide totals by repetitions and compare the calculated bound with later measured speedup.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def timed_region(work, repeats, stream):
    begin = torch.cuda.Event(enable_timing=True)
    end = torch.cuda.Event(enable_timing=True)
    with torch.cuda.stream(stream):
        begin.record(stream)
        for index in range(repeats):
            work(index)
        end.record(stream)
    end.synchronize()
    return begin.elapsed_time(end) / repeats
```

```
def E02(repeats=50, batch=1024, width=2048):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    stream = torch.cuda.current_stream(device)
    host = torch.randn((batch, width), pin_memory=True)
    gpu = torch.empty_like(host, device=device)
    weight = torch.randn((width, 512), device=device)

    def copy_work(index):
        gpu.copy_(host, non_blocking=True)

    def compute_work(index):
        output = torch.relu(gpu @ weight)
        output.record_stream(stream)

    copy_ms = timed_region(copy_work, repeats, stream)
    compute_ms = timed_region(compute_work, repeats, stream)
    serial_ms = copy_ms + compute_ms
    ideal_ms = max(copy_ms, compute_ms)
    return {"copy_ms": copy_ms, "compute_ms": compute_ms,
            "serial_prediction_ms": serial_ms,
            "ideal_overlap_ms": ideal_ms,
            "maximum_speedup": serial_ms / ideal_ms}
```

Part 6 - Measurement, Validation, and Failure Analysis

Allocator work, cache effects, and clock variation can make isolated stage times differ from integrated execution. Treat the bound as a ceiling, not a promise. If compute output is discarded, ensure PyTorch still submits the operation and retain a reference until its stream work is safe.

Part 7 - Engineering Interpretation and Philosophy

Optimization should begin by asking what can possibly improve. If compute is ten times longer than copy, heroic copy engineering has a small ceiling; if the two times are balanced, scheduling quality matters greatly.

19.10.4 Exercise 03

Exercise Title: Build a Two-Stage PyTorch Pipeline That Transfers Batch N+1 While Computing Batch N

Part 1 - Exercise Contract and Why It Matters

Implement the first steady-state pipeline: prefetch batch $n + 1$ while computing batch n. Preserve exact dataflow and compare both outputs and throughput with the sequential baseline.

Part 2 - System and Memory Mental Model

Pipeline execution has fill, steady state, and drain. The first copy cannot be hidden, and the final compute has no later copy to overlap. Two CUDA input slots avoid overwriting the batch currently consumed by compute. Each slot has a copy-ready event and a compute-done event.

Part 3 - Quantitative Prediction Before Execution

For N batches, ideal two-stage time is approximately $C + G + (N - 1)\max(C, G)$. Hence short runs understate steady-state benefit. Predict the measured speedup using E02 values before running this code.

Part 4 - Experimental Design and Instrumentation

Use two pinned inputs, two device slots, one copy stream, and the default compute stream. Prime slot zero, alternate slots, wait on only the relevant events, and synchronize after drain. Compare every batch checksum with a serial reference.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

def E03(steps=50, batch=1024, width=2048):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    copy_stream = torch.cuda.Stream(device=device)
    compute_stream = torch.cuda.current_stream(device)
    host = torch.randn((steps, batch, width), pin_memory=True)
    slots = []
    copy_ready = []
    compute_done = []
    for slot_index in range(2):
        slots.append(torch.empty((batch, width), device=device))
        copy_ready.append(torch.cuda.Event())
        done = torch.cuda.Event()
        done.record(compute_stream)
        compute_done.append(done)

    torch.cuda.synchronize(device)
    started = time.perf_counter()
    outputs = []
    for index in range(steps):
        slot_index = index % 2
        copy_stream.wait_event(compute_done[slot_index])
        with torch.cuda.stream(copy_stream):
            slots[slot_index].copy_(host[index], non_blocking=True)
            copy_ready[slot_index].record(copy_stream)
        compute_stream.wait_event(copy_ready[slot_index])
        with torch.cuda.stream(compute_stream):
            value = torch.relu(slots[slot_index]).sum()
```

```
            outputs.append(value)
            compute_done[slot_index].record(compute_stream)
    torch.cuda.synchronize(device)
    elapsed = time.perf_counter() - started
    observed = torch.stack(outputs).cpu()
    expected = torch.relu(host).sum(dim=(1, 2))
    return {"seconds": elapsed, "batches_per_s": steps / elapsed,
            "maximum_error": float((observed - expected).abs().max())}
```

Part 6 - Measurement, Validation, and Failure Analysis

The loop submits copy then compute for each index, yet independent streams may overlap copy $n + 1$ with compute n. If the timeline serializes, check pinned status, stream assignment, workload duration, and device capability. The final CPU transfer of checksums occurs only after timing.

Part 7 - Engineering Interpretation and Philosophy

Pipelining changes temporal organization, not mathematics. A useful design makes ownership and dependencies visible enough that performance work cannot silently change the samples being computed.

19.10.5 Exercise 04

Exercise Title: Use a Dedicated CUDA Transfer Stream and the Default Compute Stream to Execute Copy and Compute Concurrently

Part 1 - Exercise Contract and Why It Matters

Isolate transfer submission on a dedicated stream and computation on the default stream. Demonstrate why stream separation is necessary but not sufficient for concurrency.

Part 2 - System and Memory Mental Model

Operations within one stream are ordered. Operations in different streams are eligible for concurrent execution unless dependencies, implicit synchronization, resource saturation, or hardware limits serialize them.

Part 3 - Quantitative Prediction Before Execution

Measure three quantities: serialized interval, multi-stream interval, and event span of each stage. Define realized efficiency as $(T_{serial} - T_{overlap})/(T_{serial} - T_{ideal})$, clamped only when presenting the metric, not when hiding anomalous measurements.

Part 4 - Experimental Design and Instrumentation

Record stream identities, enqueue H2D on the transfer stream, connect it with an event to default-stream compute, and use a profiler in a separate run. Repeat with a long-enough matrix operation so overlap is observable.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def E04(batch=1536, width=1536):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    transfer = torch.cuda.Stream(device=device)
    compute = torch.cuda.current_stream(device)
    host_next = torch.randn((batch, width), pin_memory=True)
    current = torch.randn((batch, width), device=device)
    next_gpu = torch.empty_like(current)
    weight = torch.randn((width, width), device=device)
    ready = torch.cuda.Event()
    begin = torch.cuda.Event(enable_timing=True)
    end = torch.cuda.Event(enable_timing=True)

    begin.record(compute)
    with torch.cuda.stream(transfer):
        next_gpu.copy_(host_next, non_blocking=True)
        ready.record(transfer)
    with torch.cuda.stream(compute):
        current_result = current @ weight
        compute.wait_event(ready)
        next_result = next_gpu.square().mean()
        end.record(compute)
    end.synchronize()
    return {"combined_ms": begin.elapsed_time(end),
            "transfer_stream": str(transfer),
            "compute_stream": str(compute),
            "checksum": float(current_result.mean() + next_result)}
```

Part 6 - Measurement, Validation, and Failure Analysis

Here current-batch compute is independent of the incoming batch and may overlap its copy; only next-batch consumption waits. A timeline is required because a shorter combined interval alone could reflect warm caches or clock variation.

Part 7 - Engineering Interpretation and Philosophy

Streams express potential parallelism. Events express truth about dependencies. Hardware decides what actually overlaps, and measurement decides whether the extra scheduling complexity earns its place.

19.10.6 Exercise 05

Exercise Title: Combine Pinned Host Memory with non_blocking=True and Prove That True Transfer-Compute Overlap Occurs

Part 1 - Exercise Contract and Why It Matters

Prove experimentally that pinned source storage and asynchronous submission are both present when H2D traffic overlaps a kernel. Distinguish proof on a device timeline from a mere reduction in Python blocking time.

Part 2 - System and Memory Mental Model

DMA can read page-locked host pages directly. Pageable input usually requires a staging step into pinned storage before the device copy can proceed. Setting `non_blocking=True` permits asynchronous host submission, but it does not turn pageable storage into a dependable overlap source.

Part 3 - Quantitative Prediction Before Execution

Predict four cases formed by crossing pageable versus pinned storage with blocking versus non-blocking submission. Only pinned, non-blocking transfer on a separate stream is expected to expose reliable copy–compute overlap. Record completed wall time and not just submission time.

Part 4 - Experimental Design and Instrumentation

Use identical byte counts and computation. Confirm `is_pinned()`, warm all paths, and collect a profiler trace containing memcpy and kernels. Compare the overlapped interval with the E02 ideal bound.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

def run_case(source, non_blocking, repeats=30):
    device = torch.device("cuda:0")
    copy_stream = torch.cuda.Stream(device=device)
    compute_stream = torch.cuda.current_stream(device)
    destination = torch.empty_like(source, device=device)
    independent = torch.randn((2048, 2048), device=device)
    torch.cuda.synchronize(device)
    started = time.perf_counter()
    for index in range(repeats):
        with torch.cuda.stream(copy_stream):
            destination.copy_(source, non_blocking=non_blocking)
        with torch.cuda.stream(compute_stream):
            independent = torch.sin(independent)
    torch.cuda.synchronize(device)
    return time.perf_counter() - started

def E05(batch=2048, width=2048):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    pageable = torch.randn((batch, width))
    pinned = torch.randn((batch, width), pin_memory=True)
    results = {}
    results["pageable_blocking_s"] = run_case(pageable, False)
```

```
    results["pageable_nonblocking_s"] = run_case(pageable, True)
    results["pinned_blocking_s"] = run_case(pinned, False)
    results["pinned_nonblocking_s"] = run_case(pinned, True)
    results["pageable_is_pinned"] = pageable.is_pinned()
    results["pinned_is_pinned"] = pinned.is_pinned()
    return results
```

Part 6 - Measurement, Validation, and Failure Analysis

This timing screens cases; use E11's trace for proof. The `sin` chain is independent but may become too short or memory-bound. If all cases match, enlarge work, verify stream placement, and inspect device support rather than claiming that pinning never helps.

Part 7 - Engineering Interpretation and Philosophy

Pinned memory is a scarce system resource, not a universal decoration. Pin only frequently transferred buffers, reuse them, and judge benefit by end-to-end throughput under realistic CPU and GPU pressure.

19.10.7 Exercise 06

Exercise Title: Reproduce a Pipeline That Appears Asynchronous but Fails to Overlap Because the Source Memory Is Pageable

Part 1 - Exercise Contract and Why It Matters

Build a deliberate false positive: use `non_blocking=True` and a copy stream with pageable source tensors, then show why API appearance does not imply true overlap.

Part 2 - System and Memory Mental Model

The runtime may stage pageable data through an internal pinned buffer. Python can return before the final device work completes, yet host staging or synchronization can serialize the effective path. Submission asynchrony and engine concurrency are different claims.

Part 3 - Quantitative Prediction Before Execution

Predict that host submission may look fast while completed iteration time and the trace show an exposed staging/copy interval. Compare against a pre-pinned clone of the same values.

Part 4 - Experimental Design and Instrumentation

Collect host submission time, synchronized completion time, and a trace. Validate identical values and bytes. Run pageable first and pinned first in alternating trials to reduce ordering bias.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

def measure(source, repeats=40):
    device = torch.device("cuda:0")
    stream = torch.cuda.Stream(device=device)
```

```
    destination = torch.empty_like(source, device=device)
    torch.cuda.synchronize(device)
    submitted_at = time.perf_counter()
    for index in range(repeats):
        with torch.cuda.stream(stream):
            destination.copy_(source, non_blocking=True)
    submission_s = time.perf_counter() - submitted_at
    stream.synchronize()
    completed_s = time.perf_counter() - submitted_at
    return submission_s, completed_s, float(destination.sum())

def E06(batch=4096, width=1024):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    pageable = torch.randn((batch, width))
    pinned = torch.empty_like(pageable, pin_memory=True)
    pinned.copy_(pageable)
    pageable_result = measure(pageable)
    pinned_result = measure(pinned)
    return {"pageable_is_pinned": pageable.is_pinned(),
            "pinned_is_pinned": pinned.is_pinned(),
            "pageable_submission_s": pageable_result[0],
            "pageable_completion_s": pageable_result[1],
            "pinned_submission_s": pinned_result[0],
            "pinned_completion_s": pinned_result[1],
            "checksum_error": abs(pageable_result[2] - pinned_result[2])}
```

Part 6 - Measurement, Validation, and Failure Analysis

Fast submission is not the target metric. If pageable copies happen to perform well on one system, the experiment still establishes that pinned state must be verified and concurrency must be observed. Runtime behavior is implementation- and platform-dependent.

Part 7 - Engineering Interpretation and Philosophy

Performance APIs often describe permission, not completion. Engineers must name the layer of every claim: Python thread, CUDA queue, copy engine, kernel engine, or end-to-end application.

19.10.8 Exercise 07

Exercise Title: Insert CUDA Events Between Transfer and Compute Streams to Enforce Only the Dependencies Required for Correct Execution

Part 1 - Exercise Contract and Why It Matters

Encode precise dependencies with CUDA events so consumers wait for their input copy and producers wait before reusing occupied storage, without a global device barrier.

Part 2 - System and Memory Mental Model

An event recorded in stream A becomes complete after all earlier work in A. A wait inserted in stream B delays only later work in B. This creates a directed edge, leaving unrelated streams free to progress. One event protects copy-to-compute readiness; another protects compute-to-reuse safety.

Part 3 - Quantitative Prediction Before Execution

The correct dependency graph should produce zero mismatches and no global synchronization inside the loop. Predict that removing the ready wait can read partial input, while removing the reuse wait can overwrite a live batch.

Part 4 - Experimental Design and Instrumentation

Use deterministic per-batch sentinel values and two slots. Log submission order, event completion after the run, and per-batch reductions. Run many repetitions because timing races are probabilistic.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def E07(steps=100, batch=1024, width=1024):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    transfer = torch.cuda.Stream(device=device)
    compute = torch.cuda.current_stream(device)
    host_slots = []
    gpu_slots = []
    ready_events = []
    reusable_events = []
    for slot_index in range(2):
        host_slots.append(torch.empty((batch, width), pin_memory=True))
        gpu_slots.append(torch.empty((batch, width), device=device))
        copied = torch.cuda.Event()
        copied.record(compute)
        ready_events.append(copied)
        event = torch.cuda.Event()
        event.record(compute)
        reusable_events.append(event)

    results = []
    for index in range(steps):
        slot_index = index % 2
        ready_events[slot_index].synchronize()
        host_slots[slot_index].fill_(float(index))
        transfer.wait_event(reusable_events[slot_index])
        with torch.cuda.stream(transfer):
            gpu_slots[slot_index].copy_(host_slots[slot_index],
                                        non_blocking=True)
            ready_events[slot_index].record(transfer)
```

```
        compute.wait_event(ready_events[slot_index])
        with torch.cuda.stream(compute):
            results.append(gpu_slots[slot_index].mean())
            reusable_events[slot_index].record(compute)
    torch.cuda.synchronize(device)
    observed = torch.stack(results).cpu()
    expected = torch.arange(steps, dtype=observed.dtype)
    return {"maximum_error": float((observed - expected).abs().max()),
            "all_ready": all(event.query() for event in ready_events),
            "all_reusable": all(event.query() for event in reusable_events)}
```

Part 6 - Measurement, Validation, and Failure Analysis

The CPU synchronizes the previous ready event before refilling that pinned slot; the same event marks completion of the H2D read and readiness of the device input. Compute completion separately protects device-slot reuse. Removing either rule creates a different lifetime bug.

Part 7 - Engineering Interpretation and Philosophy

Global synchronization replaces reasoning with waiting. Event graphs make correctness local: every edge has a producer, consumer, storage object, and lifetime. That precision is the foundation of scalable asynchronous systems.

19.10.9 Exercise 08

Exercise Title: Remove an Unnecessary torch.cuda.synchronize() and Measure the Throughput Recovered by Preserving Asynchronous Execution

Part 1 - Exercise Contract and Why It Matters

Insert a device-wide synchronization into an otherwise valid pipeline, quantify the lost throughput, remove it, and prove that event dependencies retain correctness.

Part 2 - System and Memory Mental Model

`torch.cuda.synchronize()` blocks the CPU until all prior work on the device completes. Inside a loop it repeatedly drains the pipeline, preventing future copy submission while compute runs. A stream wait or event wait expresses only the actual dependency and does not block the host.

Part 3 - Quantitative Prediction Before Execution

The synchronized variant tends toward $C + G$ per batch; the event-driven variant tends toward $\max(C, G)$, plus overhead. Predict the ratio from E02 and compare it with the measured ratio.

Part 4 - Experimental Design and Instrumentation

Run identical data twice through the same model. Change only the loop barrier. Warm up, randomize run order across trials, synchronize at outer timing boundaries, and compare outputs before discussing speed.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

def run_pipeline(host, weight, synchronize_inside):
    device = weight.device
    transfer = torch.cuda.Stream(device=device)
    compute = torch.cuda.current_stream(device)
    gpu_slots = []
    ready_events = []
    reusable_events = []
    for slot_index in range(2):
        gpu_slots.append(torch.empty_like(host[0], device=device))
        ready_events.append(torch.cuda.Event())
        done = torch.cuda.Event()
        done.record(compute)
        reusable_events.append(done)
    elapsed_start = time.perf_counter()
    checksum = torch.zeros((), device=device)
    for index in range(host.shape[0]):
        slot_index = index % 2
        transfer.wait_event(reusable_events[slot_index])
        with torch.cuda.stream(transfer):
            gpu_slots[slot_index].copy_(host[index], non_blocking=True)
            ready_events[slot_index].record(transfer)
        compute.wait_event(ready_events[slot_index])
        with torch.cuda.stream(compute):
            value = torch.relu(gpu_slots[slot_index] @ weight).mean()
            checksum = checksum + value
            reusable_events[slot_index].record(compute)
        if synchronize_inside:
            torch.cuda.synchronize(device)
    torch.cuda.synchronize(device)
    return time.perf_counter() - elapsed_start, float(checksum)

def E08(steps=30, batch=512, width=1024):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    host = torch.randn((steps, batch, width), pin_memory=True)
    weight = torch.randn((width, width), device=device)
    blocked_s, blocked_value = run_pipeline(host, weight, True)
    async_s, async_value = run_pipeline(host, weight, False)
    return {"synchronized_s": blocked_s, "event_driven_s": async_s,
            "speedup": blocked_s / async_s,
            "checksum_error": abs(blocked_value - async_value)}
```

Part 6 - Measurement, Validation, and Failure Analysis

The two slots and their reuse events keep the experiment correct in both modes. If removing the barrier changes values, the barrier was hiding a missing dependency rather than being unnecessary. Small workloads may show little gain because event and launch overhead dominate.

Part 7 - Engineering Interpretation and Philosophy

Synchronization is not intrinsically bad; unjustified synchronization is. Place completion boundaries where the application consumes results, not wherever asynchronous behavior feels uncomfortable.

19.10.10 Exercise 09

Exercise Title: Build a Double-Buffered Input Pipeline That Alternates Between Two Pinned CPU Buffers and Two CUDA Input Buffers

Part 1 - Exercise Contract and Why It Matters

Build a reusable two-slot pipeline with pinned CPU storage and preallocated CUDA storage. Make every reuse rule explicit and measure RAM, VRAM, correctness, and steady-state throughput.

Part 2 - System and Memory Mental Model

Slot i cycles through CPU preparation, H2D copy, GPU compute, and reusable states. Two slots permit one batch in transfer while the other is in compute. Host storage becomes refillable at copy completion; CUDA storage becomes overwriteable at compute completion. Those milestones must not be conflated.

Part 3 - Quantitative Prediction Before Execution

Per-slot memory is $B_{host} + B_{device}$. Double buffering consumes twice that amount, excluding model and activations. Predict whether two slots can hide copy using E02 and calculate the fill/drain penalty for the chosen number of steps.

Part 4 - Experimental Design and Instrumentation

Use two copy-done events, two input-ready events, and two compute-done events. Refill the host slot only after copy-done, overwrite the GPU slot only after compute-done, and validate per-batch sentinels over many cycles.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def E09(steps=64, batch=1024, width=1024):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    transfer = torch.cuda.Stream(device=device)
    compute = torch.cuda.current_stream(device)
    host_slots = []
    gpu_slots = []
    copy_done = []
```

```
    input_ready = []
    compute_done = []
    for slot_index in range(2):
        host_slots.append(torch.empty((batch, width), pin_memory=True))
        gpu_slots.append(torch.empty((batch, width), device=device))
        copied = torch.cuda.Event()
        copied.record(compute)
        copy_done.append(copied)
        input_ready.append(torch.cuda.Event())
        computed = torch.cuda.Event()
        computed.record(compute)
        compute_done.append(computed)

    answers = []
    for index in range(steps):
        slot_index = index % 2
        copy_done[slot_index].synchronize()
        host_slots[slot_index].fill_(float(index + 1))
        transfer.wait_event(compute_done[slot_index])
        with torch.cuda.stream(transfer):
            gpu_slots[slot_index].copy_(host_slots[slot_index],
                                        non_blocking=True)
            copy_done[slot_index].record(transfer)
            input_ready[slot_index].record(transfer)
        compute.wait_event(input_ready[slot_index])
        with torch.cuda.stream(compute):
            answers.append(gpu_slots[slot_index].mean())
            compute_done[slot_index].record(compute)
    torch.cuda.synchronize(device)
    observed = torch.stack(answers).cpu()
    expected = torch.arange(1, steps + 1, dtype=observed.dtype)
    slot_bytes = host_slots[0].numel() * host_slots[0].element_size()
    return {"maximum_error": float((observed - expected).abs().max()),
            "pinned_RAM_MiB": 2 * slot_bytes / (1024 ** 2),
            "input_VRAM_MiB": 2 * slot_bytes / (1024 ** 2)}
```

Part 6 - Measurement, Validation, and Failure Analysis

Host-side event synchronization can stall CPU preparation; E14 will overlap preparation using backpressure and a producer. It is retained here to make host buffer ownership undeniable. Stress sentinels, variable delays, and many cycles to reveal reuse races.

Part 7 - Engineering Interpretation and Philosophy

Double buffering is a bounded queue, not simply two arrays. Its strength is controlled ownership: bounded memory, predictable latency, and backpressure when one stage cannot keep pace.

19.10.11 Exercise 10

Exercise Title: Extend Double Buffering into Triple Buffering and Determine Whether Additional Buffers Improve Throughput or Only Increase Memory Usage

Part 1 - Exercise Contract and Why It Matters

Generalize the pipeline from two to three slots and determine experimentally whether extra queue depth absorbs jitter or merely consumes pinned RAM and VRAM.

Part 2 - System and Memory Mental Model

Two slots are sufficient for ideal two-stage copy–compute overlap. A third can help when CPU preparation, transfer, or kernel duration varies, because one slot may wait while another is ready. It cannot reduce the steady-state critical path below the slowest stage.

Part 3 - Quantitative Prediction Before Execution

For slot bytes $M_h + M_d$, moving from two to three slots adds exactly one such unit. Predict throughput gain from measured jitter, not average time alone. Define gain per MiB as $(R_3 - R_2)/(M_h + M_d)$.

Part 4 - Experimental Design and Instrumentation

Run slot counts one, two, and three under identical batch order. Record median throughput, tail iteration latency, peak VRAM, pinned RAM, and output error. Add controlled preparation jitter in a separate trial.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

def buffered_run(source, weight, slot_count):
    device = weight.device
    transfer = torch.cuda.Stream(device=device)
    compute = torch.cuda.current_stream(device)
    slots = []
    ready = []
    reusable = []
    for slot_index in range(slot_count):
        slots.append(torch.empty_like(source[0], device=device))
        ready.append(torch.cuda.Event())
        done = torch.cuda.Event()
        done.record(compute)
        reusable.append(done)
    torch.cuda.synchronize(device)
    started = time.perf_counter()
    values = []
    for index in range(source.shape[0]):
        slot_index = index % slot_count
        transfer.wait_event(reusable[slot_index])
        with torch.cuda.stream(transfer):
```

```
            slots[slot_index].copy_(source[index], non_blocking=True)
            ready[slot_index].record(transfer)
        compute.wait_event(ready[slot_index])
        with torch.cuda.stream(compute):
            values.append(torch.relu(slots[slot_index] @ weight).mean())
            reusable[slot_index].record(compute)
    torch.cuda.synchronize(device)
    return time.perf_counter() - started, torch.stack(values).cpu()

def E10(steps=60, batch=512, width=1024):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    source = torch.randn((steps, batch, width), pin_memory=True)
    weight = torch.randn((width, width), device=device)
    report = {}
    reference = None
    for slot_count in (1, 2, 3):
        seconds, values = buffered_run(source, weight, slot_count)
        if reference is None:
            reference = values
        error = float((values - reference).abs().max())
        report[slot_count] = {"seconds": seconds,
                              "batches_per_s": steps / seconds,
                              "maximum_error": error}
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

The entire source dataset is pinned here for experimental control; production should use bounded pinned slots. Alternate test order because thermal state can favor later runs. If three slots help only once, investigate warmup rather than adopting them.

Part 7 - Engineering Interpretation and Philosophy

Buffer count is a queueing decision. More capacity smooths variance but increases memory, stale work, and debugging surface. The smallest depth that sustains the target rate is usually the strongest engineering choice.

19.10.12 Exercise 11

Exercise Title: Profile Copy Engines and CUDA Kernels to Verify Real Timeline Overlap Instead of Inferring It from Wall-Clock Time

Part 1 - Exercise Contract and Why It Matters

Capture a device trace and prove that H2D memcpy intervals intersect kernel intervals. Replace inference from total runtime with direct temporal evidence.

Part 2 - System and Memory Mental Model

A faster wall time can come from caching, clocks, or reduced work. A profiler timeline exposes CPU submission, CUDA runtime calls, memcpy activity, kernels, and synchronization. Overlap exists only when device intervals occupy the same time range on distinct execution resources.

Part 3 - Quantitative Prediction Before Execution

Predict the overlap duration as at most $\min(C, G)$ per steady batch. Define overlap fraction as intersected copy time divided by total copy time, and exposed copy as copy time minus intersection.

Part 4 - Experimental Design and Instrumentation

Profile warmup separately, then a small active window. Give CPU submission regions short profiler labels. Export a Chrome trace, inspect memcpy and kernel lanes, and corroborate it with event and wall-clock measurements.

Part 5 - PyTorch/CUDA Implementation

```
import torch
from torch.profiler import ProfilerActivity, profile, record_function

def E11(trace_path="dmm08_overlap_trace.json", steps=12):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    transfer = torch.cuda.Stream(device=device)
    compute = torch.cuda.current_stream(device)
    host = torch.randn((steps, 1024, 1024), pin_memory=True)
    gpu_slots = []
    ready_events = []
    reusable_events = []
    for slot_index in range(2):
        gpu_slots.append(torch.empty_like(host[0], device=device))
        ready_events.append(torch.cuda.Event())
        done = torch.cuda.Event()
        done.record(compute)
        reusable_events.append(done)
    weight = torch.randn((1024, 1024), device=device)
    torch.cuda.synchronize(device)
    with profile(activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
                 record_shapes=True, profile_memory=True) as prof:
        for index in range(steps):
            slot_index = index % 2
            transfer.wait_event(reusable_events[slot_index])
            with record_function("submit_h2d"):
                with torch.cuda.stream(transfer):
                    gpu_slots[slot_index].copy_(host[index],
                                                non_blocking=True)
                    ready_events[slot_index].record(transfer)
            compute.wait_event(ready_events[slot_index])
```

```
            with record_function("submit_compute"):
                output = torch.relu(gpu_slots[slot_index] @ weight)
                reusable_events[slot_index].record(compute)
        torch.cuda.synchronize(device)
    prof.export_chrome_trace(trace_path)
    table = prof.key_averages().table(sort_by="self_cuda_time_total",
                                      row_limit=12)
    return {"trace_path": trace_path, "summary": table,
            "checksum": float(output.mean())}
```

Part 6 - Measurement, Validation, and Failure Analysis

The trace uses two protected destinations so reuse cannot race with compute. Profiler overhead alters absolute time, but temporal overlap remains observable. Do not claim concurrent engines from adjacent, non-overlapping rectangles, and validate outputs separately because a timeline proves scheduling, not arithmetic.

Part 7 - Engineering Interpretation and Philosophy

Timelines turn concurrency from folklore into auditable evidence. The trace is the explanation: it should show where work waited, which dependency caused it, and how much critical-path time the change removed.

19.10.13 Exercise 12

Exercise Title: Sweep Batch Size and Identify the Region Where Transfer Is Fully Hidden Behind Computation

Part 1 - Exercise Contract and Why It Matters

Sweep batch size and locate the operating region where compute duration is long enough to hide H2D transfer without exceeding memory or latency targets.

Part 2 - System and Memory Mental Model

Transfer and compute do not necessarily scale identically. Copy is roughly proportional to bytes after fixed overhead, while kernels may change algorithms, occupancy, or efficiency at shape thresholds. Therefore the hidden-copy region must be measured on the target system.

Part 3 - Quantitative Prediction Before Execution

For every batch b, record C_b, G_b, serial S_b, and pipeline P_b. Estimate exposed copy as $\max(0, P_b - G_b)$ and hidden fraction as $1 - \max(0, P_b - G_b)/C_b$. Mark full hiding only within a declared tolerance.

Part 4 - Experimental Design and Instrumentation

Use geometric batch sizes, fixed width and dtype, repeated trials, and peak-memory reset per point. Stop before unsafe memory pressure. Retain throughput, latency, hidden fraction, and peak VRAM rather than selecting by throughput alone.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def event_time(work, repeats=20):
    start = torch.cuda.Event(enable_timing=True)
    end = torch.cuda.Event(enable_timing=True)
    start.record()
    for index in range(repeats):
        work()
    end.record()
    end.synchronize()
    return start.elapsed_time(end) / repeats

def E12(width=1024, batch_sizes=(64, 128, 256, 512, 1024, 2048)):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    report = []
    for batch in batch_sizes:
        host = torch.randn((batch, width), pin_memory=True)
        gpu = torch.empty_like(host, device=device)
        weight = torch.randn((width, width), device=device)
        torch.cuda.reset_peak_memory_stats(device)

        def copy_work():
            gpu.copy_(host, non_blocking=True)

        def compute_work():
            output = torch.relu(gpu @ weight)
            output.record_stream(torch.cuda.current_stream(device))

        copy_ms = event_time(copy_work)
        compute_ms = event_time(compute_work)
        ideal_ms = max(copy_ms, compute_ms)
        hidden_fraction = 1.0 - max(0.0, ideal_ms - compute_ms) / copy_ms
        report.append({"batch": batch, "copy_ms": copy_ms,
                       "compute_ms": compute_ms, "ideal_ms": ideal_ms,
                       "predicted_hidden_fraction": hidden_fraction,
                       "peak_MiB": torch.cuda.max_memory_allocated(device)
                                   / (1024 ** 2)})
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

The implementation predicts hiding from isolated stages; pair it with measured double-buffered time from E09 for confirmation. Avoid calling a point fully hidden when differences are within timer noise. An OOM is not a valid endpoint for an ordinary sweep.

Part 7 - Engineering Interpretation and Philosophy

The best batch is a constrained operating point, not the largest possible one. Throughput, latency, memory headroom, and robustness to variable shapes all participate in the decision.

19.10.14 Exercise 13

Exercise Title: Reduce GPU Compute Cost Until Data Transfer Becomes the Dominant Bottleneck and Explain Why Overlap Efficiency Collapses

Part 1 - Exercise Contract and Why It Matters

Systematically reduce GPU work while holding input bytes fixed. Identify the point at which transfer becomes the critical path and explain why further kernel optimization no longer improves pipeline throughput proportionally.

Part 2 - System and Memory Mental Model

Overlap hides only the shorter concurrent stage. When compute dominates, copy can disappear beneath it. As compute shrinks below copy time, the copy engine sets the cadence and exposed transfer grows. This is bottleneck migration, not a failure of asynchronous APIs.

Part 3 - Quantitative Prediction Before Execution

For compute multipliers k, predict $P_k = \max(C, G_k)$. The crossover occurs near $G_k = C$. Plot achieved throughput and exposed-copy fraction against k.

Part 4 - Experimental Design and Instrumentation

Keep source shape, dtype, transfer route, streams, and repetitions fixed. Vary only the number or complexity of independent kernels. Measure isolated stages and the integrated pipeline at every point.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def E13(batch=2048, width=1024, depths=(16, 8, 4, 2, 1)):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    host = torch.randn((batch, width), pin_memory=True)
    gpu = torch.empty_like(host, device=device)
    report = []
    for depth in depths:
        start = torch.cuda.Event(enable_timing=True)
        copied = torch.cuda.Event(enable_timing=True)
        finished = torch.cuda.Event(enable_timing=True)
        start.record()
        gpu.copy_(host, non_blocking=True)
        copied.record()
        value = gpu
        for layer_index in range(depth):
            value = torch.tanh(value)
        finished.record()
```

```
        finished.synchronize()
        copy_ms = start.elapsed_time(copied)
        compute_ms = copied.elapsed_time(finished)
        report.append({"depth": depth, "copy_ms": copy_ms,
                       "compute_ms": compute_ms,
                       "predicted_pipeline_ms": max(copy_ms, compute_ms),
                       "dominant_stage": "copy" if copy_ms > compute_ms
                                         else "compute"})
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

These stage timings are serialized intentionally; run the corresponding double-buffered cases to measure realized overlap. Elementwise kernels may be memory-bandwidth limited and contend with copies, so the ideal maximum can be optimistic even when their nominal intervals balance.

Part 7 - Engineering Interpretation and Philosophy

Local optimization changes the global bottleneck. Mature performance work reprofiles after every major improvement and stops tuning a stage once another resource controls the service rate.

19.10.15 Exercise 14

Exercise Title: Increase CPU Data-Preparation Cost and Determine When Preprocessing Prevents the GPU Pipeline from Remaining Saturated

Part 1 - Exercise Contract and Why It Matters

Increase CPU preparation time until the GPU develops idle gaps. Separate a data production bottleneck from H2D and compute bottlenecks and quantify the queue depth needed to absorb ordinary jitter.

Part 2 - System and Memory Mental Model

The GPU remains saturated only if prepared pinned batches arrive at least as fast as the GPU pipeline consumes them. A bounded queue decouples short-term variation but cannot repair a producer whose long-run service rate is lower. Backpressure prevents unbounded pinned-memory growth.

Part 3 - Quantitative Prediction Before Execution

Let average preparation be P and GPU cadence be $Q = \max(C, G)$. If $P > Q$, the best sustained rate is (1/P). If (P<Q) but preparation has jitter, buffer depth determines how much variance can be absorbed.

Part 4 - Experimental Design and Instrumentation

Create a bounded producer queue, vary synthetic preparation delay, and record queue-empty time, queue-full time, GPU throughput, and pinned allocation. Do not sleep in the GPU submission thread; isolate preparation in a producer thread.

Part 5 - PyTorch/CUDA Implementation

```
import queue
import threading
import time
import torch

def E14(steps=50, delay_ms=2.0, batch=512, width=1024):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    prepared = queue.Queue(maxsize=3)
    stop_token = object()

    def producer():
        for index in range(steps):
            host = torch.empty((batch, width), pin_memory=True)
            host.normal_()
            time.sleep(delay_ms / 1000.0)
            prepared.put((index, host))
        prepared.put(stop_token)

    worker = threading.Thread(target=producer)
    worker.start()
    device = torch.device("cuda:0")
    transfer = torch.cuda.Stream(device=device)
    compute = torch.cuda.current_stream(device)
    gpu_slots = []
    ready_events = []
    reusable_events = []
    for slot_index in range(2):
        gpu_slots.append(torch.empty((batch, width), device=device))
        ready_events.append(torch.cuda.Event())
        done = torch.cuda.Event()
        done.record(compute)
        reusable_events.append(done)
    inflight_sources = []
    empty_wait_s = 0.0
    processed = 0
    started = time.perf_counter()
    while True:
        wait_started = time.perf_counter()
        item = prepared.get()
        empty_wait_s += time.perf_counter() - wait_started
        if item is stop_token:
            break
        index, host = item
        slot_index = index % 2
        transfer.wait_event(reusable_events[slot_index])
        with torch.cuda.stream(transfer):
            gpu_slots[slot_index].copy_(host, non_blocking=True)
```

```
            ready_events[slot_index].record(transfer)
        inflight_sources.append((host, ready_events[slot_index]))
        if len(inflight_sources) > 3:
            old_host, old_event = inflight_sources.pop(0)
            old_event.synchronize()
        compute.wait_event(ready_events[slot_index])
        with torch.cuda.stream(compute):
            output = torch.relu(gpu_slots[slot_index]).mean()
            reusable_events[slot_index].record(compute)
        processed += 1
    for old_host, old_event in inflight_sources:
        old_event.synchronize()
    torch.cuda.synchronize(device)
    elapsed = time.perf_counter() - started
    worker.join()
    return {"processed": processed, "seconds": elapsed,
            "batches_per_s": processed / elapsed,
            "consumer_queue_wait_s": empty_wait_s,
            "checksum": float(output)}
```

Part 6 - Measurement, Validation, and Failure Analysis

This diagnostic allocates pinned tensors per batch to make preparation cost visible and retains a bounded set until their copies complete; replace those allocations with reusable host slots for production. Queue wait includes Python scheduling and is not itself GPU-idle proof, so correlate it with a trace. Vary delay over a sweep instead of drawing conclusions from one point.

Part 7 - Engineering Interpretation and Philosophy

GPU utilization is an end-to-end property. A faster kernel is irrelevant when the accelerator waits for decoding, augmentation, storage, or Python. Optimize the slowest stage while keeping queues bounded and observable.

19.10.16 Exercise 15

Exercise Title: Overlap GPU-to-CPU Output Transfers with Computation of the Next Batch Without Reading Host Results Prematurely

Part 1 - Exercise Contract and Why It Matters

Move batch n's output to pinned host memory while computing batch $n+1$, and prove that the CPU never consumes a destination before its asynchronous D2H copy completes.

Part 2 - System and Memory Mental Model

An asynchronous D2H copy reverses the ownership problem. The GPU produces an output, an output stream waits for the compute event, and a pinned host slot receives the bytes. Python may regain control before the host values are valid. The CPU must query or synchronize the copy-complete event before reading or reusing that slot.

Part 3 - Quantitative Prediction Before Execution

With output-copy time D and next compute time G, the steady contribution is ideally $\max(D, G)$, subject to copy-engine capabilities. Calculate output bytes and predict whether D2H is large enough to be visible.

Part 4 - Experimental Design and Instrumentation

Use two device outputs and two pinned host outputs. Record compute-ready and D2H-done events per slot. Compare results with a blocking reference, and include an intentionally premature-read demonstration only as a documented unsafe variant, not as a correctness oracle.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def E15(steps=40, batch=1024, width=1024):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    compute = torch.cuda.current_stream(device)
    output_stream = torch.cuda.Stream(device=device)
    inputs = torch.randn((steps, batch, width), device=device)
    device_outputs = []
    host_outputs = []
    compute_ready = []
    d2h_done = []
    for slot_index in range(2):
        device_outputs.append(torch.empty((batch,), device=device))
        host_outputs.append(torch.empty((batch,), pin_memory=True))
        compute_ready.append(torch.cuda.Event())
        done = torch.cuda.Event()
        done.record(compute)
        d2h_done.append(done)

    validated = []
    for index in range(steps):
        slot_index = index % 2
        d2h_done[slot_index].synchronize()
        if index >= 2:
            completed_index = index - 2
            expected = inputs[completed_index].square().sum(dim=1).cpu()
            error = (host_outputs[slot_index] - expected).abs().max()
            validated.append(float(error))
        with torch.cuda.stream(compute):
            torch.sum(inputs[index].square(), dim=1,
                      out=device_outputs[slot_index])
            compute_ready[slot_index].record(compute)
        output_stream.wait_event(compute_ready[slot_index])
        with torch.cuda.stream(output_stream):
            host_outputs[slot_index].copy_(device_outputs[slot_index],
                                           non_blocking=True)
```

```
            d2h_done[slot_index].record(output_stream)

    for event in d2h_done:
        event.synchronize()
    return {"validated_completed_slots": len(validated),
            "maximum_error": max(validated) if validated else 0.0}
```

Part 6 - Measurement, Validation, and Failure Analysis

The final two batches require an explicit drain validation in a production test; the compact return focuses on steady-state slots. Calling `cpu()` on the expected value introduces synchronization outside the timed region. A premature host read may occasionally look correct, which makes it especially dangerous.

Part 7 - Engineering Interpretation and Philosophy

Asynchrony transfers responsibility for completion to the program. Host-visible memory is not valid merely because a copy was requested. Correct systems expose completion as an explicit state, not an assumption about elapsed time.

19.10.17 Exercise 16

Exercise Title: Build a Bidirectional Pipeline That Overlaps Host-to-Device Inputs, GPU Compute, and Device-to-Host Outputs

Part 1 - Exercise Contract and Why It Matters

Construct a three-stage bidirectional pipeline with H2D input, GPU compute, and D2H output streams. Preserve dependencies and discover whether the device can actually overlap both transfer directions with kernels.

Part 2 - System and Memory Mental Model

Input copy, compute, and output copy form a directed acyclic graph per slot. Different devices expose different copy-engine counts and scheduling behavior; separate streams express eligibility but do not guarantee simultaneous H2D, D2H, and compute. Slot reuse waits for both the input consumer and output copy.

Part 3 - Quantitative Prediction Before Execution

With stage times H, G, and D, an ideal three-engine cadence is $\max(H, G, D)$. With one shared bidirectional copy engine, transfer contribution may approach $H + D$, giving $\max(H + D, G)$. Treat these as hypotheses for the trace.

Part 4 - Experimental Design and Instrumentation

Preallocate bounded host input/output and device input/output slots. Use events for H2D-ready, compute-ready, compute-done, and D2H-done. Validate all outputs, measure fill/steady/drain separately, and inspect engine lanes.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def E16(steps=30, batch=512, width=1024, slots_count=3):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    h2d = torch.cuda.Stream(device=device)
    compute = torch.cuda.current_stream(device)
    d2h = torch.cuda.Stream(device=device)
    host_inputs = []
    host_outputs = []
    gpu_inputs = []
    gpu_outputs = []
    input_ready = []
    compute_done = []
    output_done = []
    for slot_index in range(slots_count):
        host_inputs.append(torch.empty((batch, width), pin_memory=True))
        host_outputs.append(torch.empty((batch,), pin_memory=True))
        gpu_inputs.append(torch.empty((batch, width), device=device))
        gpu_outputs.append(torch.empty((batch,), device=device))
        input_ready.append(torch.cuda.Event())
        computed = torch.cuda.Event()
        computed.record(compute)
        compute_done.append(computed)
        copied = torch.cuda.Event()
        copied.record(compute)
        output_done.append(copied)

    for index in range(steps):
        slot_index = index % slots_count
        output_done[slot_index].synchronize()
        host_inputs[slot_index].fill_(float(index + 1))
        h2d.wait_event(compute_done[slot_index])
        with torch.cuda.stream(h2d):
            gpu_inputs[slot_index].copy_(host_inputs[slot_index],
                                         non_blocking=True)
            input_ready[slot_index].record(h2d)
        compute.wait_event(input_ready[slot_index])
        with torch.cuda.stream(compute):
            torch.sum(gpu_inputs[slot_index], dim=1,
                      out=gpu_outputs[slot_index])
            compute_done[slot_index].record(compute)
        d2h.wait_event(compute_done[slot_index])
        with torch.cuda.stream(d2h):
            host_outputs[slot_index].copy_(gpu_outputs[slot_index],
                                           non_blocking=True)
            output_done[slot_index].record(d2h)
```

```
for event in output_done:
    event.synchronize()
last_slot = (steps - 1) % slots_count
last_mean = float(host_outputs[last_slot].mean())
return {"last_slot_mean": last_mean,
        "expected_last": float(steps * width)}
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate every slot before refill in the full experiment, not only the final one. If H2D and D2H serialize, check the device's engines before rewriting the event graph. If values fail, verify output-slot reuse as carefully as input reuse.

Part 7 - Engineering Interpretation and Philosophy

A pipeline is a resource graph constrained by hardware, not a list of streams. Good design remains correct on a device with less concurrency and opportunistically accelerates on one with more.

19.10.18 Exercise 17

Exercise Title: Stress-Test an Overlapped Pipeline Under Variable Batch Sizes and Detect Buffer Reuse Hazards, Race Conditions, and Synchronization Stalls

Part 1 - Exercise Contract and Why It Matters

Stress the pipeline with bounded variable shapes, randomized delays, and repeated slot reuse. Detect corruption, illegal assumptions, allocator growth, and unnecessary waits without deliberately exhausting the GPU.

Part 2 - System and Memory Mental Model

Variable sizes challenge both logical capacity and allocator reuse. A slot has a maximum capacity and a logical view for the current batch. Reallocation inside the hot path can destroy stable memory behavior and add hidden synchronization. Generation numbers reveal stale or overwritten data.

Part 3 - Quantitative Prediction Before Execution

Before execution, compute maximum slot bytes and total bounded memory. Predict the expected checksum for every generation. Set failure thresholds for mismatches, timeout/stall duration, reserved-memory drift, and throughput variance.

Part 4 - Experimental Design and Instrumentation

Generate sizes with a fixed seed, preallocate for the maximum, use explicit slices, and inject optional CPU jitter. Log size, slot, generation, event waits, checksum, iteration time, allocated memory, and reserved memory.

Part 5 - PyTorch/CUDA Implementation

```
import random
import torch

def E17(steps=300, min_batch=64, max_batch=1024, width=512):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    random.seed(1708)
    device = torch.device("cuda:0")
    transfer = torch.cuda.Stream(device=device)
    compute = torch.cuda.current_stream(device)
    host_slots = []
    gpu_slots = []
    ready = []
    reusable = []
    for slot_index in range(3):
        host_slots.append(torch.empty((max_batch, width), pin_memory=True))
        gpu_slots.append(torch.empty((max_batch, width), device=device))
        copied = torch.cuda.Event()
        copied.record(compute)
        ready.append(copied)
        done = torch.cuda.Event()
        done.record(compute)
        reusable.append(done)

    results = []
    expected = []
    for generation in range(steps):
        batch = random.randint(min_batch, max_batch)
        slot_index = generation % 3
        value = float((generation % 97) + 1)
        ready[slot_index].synchronize()
        host_view = host_slots[slot_index][:batch]
        gpu_view = gpu_slots[slot_index][:batch]
        host_view.fill_(value)
        transfer.wait_event(reusable[slot_index])
        with torch.cuda.stream(transfer):
            gpu_view.copy_(host_view, non_blocking=True)
            ready[slot_index].record(transfer)
        compute.wait_event(ready[slot_index])
        with torch.cuda.stream(compute):
            results.append(gpu_view.sum())
            reusable[slot_index].record(compute)
        expected.append(value * batch * width)
    torch.cuda.synchronize(device)
    observed = torch.stack(results).double().cpu()
    target = torch.tensor(expected, dtype=torch.float64)
    relative = (observed - target).abs() / target.clamp_min(1.0)
    allocated = torch.cuda.memory_allocated(device) / (1024 ** 2)
```

```
    reserved = torch.cuda.memory_reserved(device) / (1024 ** 2)
    return {"steps": steps,
            "maximum_relative_error": float(relative.max()),
            "allocated_MiB": allocated,
            "reserved_MiB": reserved}
```

Part 6 - Measurement, Validation, and Failure Analysis

Run a negative-control variant that omits one wait only in an isolated test and confirm the validator can catch failure. If no failure appears, increase timing variation rather than declaring the race safe. Bound maximum batch from a memory budget; a stress test should study limits, not crash the workstation.

Part 7 - Engineering Interpretation and Philosophy

Concurrency bugs live in rare schedules. Deterministic sentinels, generations, bounded randomness, and long repetitions turn "occasionally wrong" into a reproducible engineering signal.

19.10.19 Exercise 18

Exercise Title: Build an Adaptive Buffering Controller That Chooses Buffer Count from Measured Transfer Time, Compute Time, and Available VRAM

Part 1 - Exercise Contract and Why It Matters

Build a controller that proposes a bounded slot count from measured stage times and explicit host/device memory budgets. Treat the proposal as a hypothesis that must be benchmarked, not an automatic guarantee.

Part 2 - System and Memory Mental Model

Buffer depth supplies queueing slack. Time ratios estimate how much work can be in flight; memory budgets impose hard feasibility. Per-slot cost includes pinned input/output storage and device input/output storage, while model, activations, and safety headroom remain outside the slot budget.

Part 3 - Quantitative Prediction Before Execution

Estimate required depth as

$$n_t = 1 + \left\lceil \frac{T_p + T_c}{\max(T_g, \epsilon)} \right\rceil .$$

Compute $n_h = \lfloor R_{budget}/M_h \rfloor$ and $n_d = \lfloor V_{budget}/M_d \rfloor$. Choose $\max(2, \min(n_t, n_h, n_d, n_{hard}))$ when at least two slots are feasible; otherwise fall back to a serial design.

Part 4 - Experimental Design and Instrumentation

Measure robust medians after warmup, reserve fixed safety margins, calculate the candidate, then benchmark neighboring depths. Re-evaluate when shape distribution, dtype, model, or hardware changes.

Part 5 - PyTorch/CUDA Implementation

```
import math
import torch

def choose_buffers(prep_ms, h2d_ms, compute_ms, host_slot_bytes,
                   device_slot_bytes, ram_budget_bytes,
                   vram_budget_bytes, hard_max=8):
    epsilon = 1.0e-9
    time_need = 1 + math.ceil((prep_ms + h2d_ms) /
                              max(compute_ms, epsilon))
    host_limit = ram_budget_bytes // max(host_slot_bytes, 1)
    device_limit = vram_budget_bytes // max(device_slot_bytes, 1)
    feasible = min(host_limit, device_limit, hard_max)
    if feasible < 2:
        return 1
    return max(2, min(time_need, feasible))

def E18(batch=1024, width=1024, prep_ms=1.5, h2d_ms=0.8,
        compute_ms=2.4, ram_budget_MiB=512, vram_budget_MiB=512):
    element_size = torch.empty((), dtype=torch.float32).element_size()
    input_bytes = batch * width * element_size
    output_bytes = batch * element_size
    host_slot_bytes = input_bytes + output_bytes
    device_slot_bytes = input_bytes + output_bytes
    ram_budget = int(ram_budget_MiB * 1024 ** 2)
    vram_budget = int(vram_budget_MiB * 1024 ** 2)
    selected = choose_buffers(prep_ms, h2d_ms, compute_ms,
                              host_slot_bytes, device_slot_bytes,
                              ram_budget, vram_budget)
    return {"selected_buffers": selected,
            "host_slot_MiB": host_slot_bytes / (1024 ** 2),
            "device_slot_MiB": device_slot_bytes / (1024 ** 2),
            "host_commit_MiB": selected * host_slot_bytes / (1024 ** 2),
            "device_commit_MiB": selected * device_slot_bytes / (1024 ** 2)}
```

Part 6 - Measurement, Validation, and Failure Analysis

The formula assumes stationary representative times and does not model long-tail jitter or resource contention. Reject a recommendation that violates real peak memory after model allocations. Benchmark depth $n-1, n, n+1$ where feasible, and keep the smallest depth within the throughput tolerance.

Part 7 - Engineering Interpretation and Philosophy

Adaptive control should make assumptions visible, respect hard budgets, and fail safely. Its purpose is not to eliminate measurement but to reduce the search space and explain why a configuration was selected.

19.10.20 Exercise 19

Exercise Title: Build and Benchmark a Fully Overlapped PyTorch Training Input Pipeline That Maximizes GPU Utilization While Respecting a Fixed VRAM and RAM Budget

Part 1 - Exercise Contract and Why It Matters

Integrate the chapter into a bounded training pipeline: reusable pinned slots, preallocated CUDA inputs, non-blocking H2D, copy and compute streams, event-based ownership, backpressure, a serial baseline, and reproducible correctness and performance reports.

Part 2 - System and Memory Mental Model

Each slot moves through free, prepared, copying, device-ready, computing, and reusable states. The model and optimizer own a separate VRAM budget. The pipeline may occupy only the remaining allowance after safety headroom. Fill and drain are real costs; steady-state throughput must not conceal them. A bounded slot pool prevents producer speed from becoming unbounded RAM consumption.

Part 3 - Quantitative Prediction Before Execution

Before allocation, compute slot bytes and reject infeasible counts. Predict serial and ideal pipeline time from measured preparation, copy, forward/backward, and optimizer intervals. Report realized speedup, GPU duty cycle, peak allocated and reserved VRAM, pinned RAM commitment, and numerical parity.

Part 4 - Experimental Design and Instrumentation

First overfit a tiny deterministic dataset with the serial loop. Snapshot initial weights, restore them, run the overlapped loop with identical batch order, and compare losses and parameters within declared tolerances. Then benchmark warmed trials, profile one active window, and repeat across safe buffer counts.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

def E19(steps=40, batch=256, width=1024, classes=16,
        slots_count=2, ram_budget_MiB=256, vram_budget_MiB=256):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    element_size = torch.empty((), dtype=torch.float32).element_size()
    label_element_size = torch.empty((), dtype=torch.long).element_size()
    host_slot_bytes = (batch * width * element_size
                       + batch * label_element_size)
    device_slot_bytes = host_slot_bytes
    ram_need = slots_count * host_slot_bytes
    vram_need = slots_count * device_slot_bytes
    if ram_need > ram_budget_MiB * 1024 ** 2:
        raise ValueError("Pinned RAM budget is too small.")
    if vram_need > vram_budget_MiB * 1024 ** 2:
        raise ValueError("Input VRAM budget is too small.")

    transfer = torch.cuda.Stream(device=device)
    compute = torch.cuda.current_stream(device)
```

```
    model = torch.nn.Linear(width, classes).to(device)
    optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
    host_inputs = []
    host_labels = []
    gpu_inputs = []
    gpu_labels = []
    ready = []
    reusable = []
    for slot_index in range(slots_count):
        host_inputs.append(torch.empty((batch, width), pin_memory=True))
        host_labels.append(torch.empty((batch,), dtype=torch.long,
                                       pin_memory=True))
        gpu_inputs.append(torch.empty((batch, width), device=device))
        gpu_labels.append(torch.empty((batch,), dtype=torch.long,
                                      device=device))
        ready.append(torch.cuda.Event())
        done = torch.cuda.Event()
        done.record(compute)
        reusable.append(done)

    generator = torch.Generator().manual_seed(1908)
    losses = []
    torch.cuda.reset_peak_memory_stats(device)
    torch.cuda.synchronize(device)
    started = time.perf_counter()
    for index in range(steps):
        slot_index = index % slots_count
        reusable[slot_index].synchronize()
        host_inputs[slot_index].normal_(generator=generator)
        host_labels[slot_index].random_(0, classes, generator=generator)
        transfer.wait_event(reusable[slot_index])
        with torch.cuda.stream(transfer):
            gpu_inputs[slot_index].copy_(host_inputs[slot_index],
                                         non_blocking=True)
            gpu_labels[slot_index].copy_(host_labels[slot_index],
                                         non_blocking=True)
            ready[slot_index].record(transfer)
        compute.wait_event(ready[slot_index])
        with torch.cuda.stream(compute):
            optimizer.zero_grad(set_to_none=True)
            logits = model(gpu_inputs[slot_index])
            loss = torch.nn.functional.cross_entropy(
                logits, gpu_labels[slot_index])
            loss.backward()
            optimizer.step()
            losses.append(loss.detach())
            reusable[slot_index].record(compute)
    torch.cuda.synchronize(device)
```

```
    elapsed = time.perf_counter() - started
    loss_values = torch.stack(losses).cpu()
    return {"seconds": elapsed, "samples_per_s": steps * batch / elapsed,
            "first_loss": float(loss_values[0]),
            "last_loss": float(loss_values[-1]),
            "pinned_input_MiB": ram_need / (1024 ** 2),
            "device_input_MiB": vram_need / (1024 ** 2),
            "peak_allocated_MiB": torch.cuda.max_memory_allocated(device)
                                  / (1024 ** 2)}
```

Part 6 - Measurement, Validation, and Failure Analysis

The precheck includes input and label storage on both host and device, but a full budget must also reserve model, gradients, optimizer state, activations, allocator overhead, and a safety margin. Host preparation is synchronized per slot for ownership and can be moved to a bounded producer queue after parity is established. A faster run is rejected if batch order, losses, gradients, or parameter updates diverge beyond the chosen reproducibility tolerance.

Part 7 - Engineering Interpretation and Philosophy

The final objective is dependable useful work per second under explicit resource limits. Maximum utilization is not achieved by keeping the GPU busy with any work; it is achieved by preserving the intended computation, bounding memory, expressing exact dependencies, surviving variability, and producing evidence that another engineer can reproduce and defend.

19.11 DMM09

Chapter Title: Data and Tensor Prefetching
Specialization: Dynamic Memory Management and CUDA Systems

19.11.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Prefetch Distance, Buffer Depth, Producer-Consumer Latency, and Memory Cost

Part 1 - Exercise Contract and Why It Matters

Build a quantitative model of prefetching before changing any loader setting. Explain what is loaded, how far ahead it is loaded, which memory tier owns it, when it becomes consumable, and what prevents the producer from running unboundedly ahead of training.

Part 2 - System and Memory Mental Model

Prefetching moves future work earlier; it does not remove that work. Distance d is the number of future batches made ready, while buffer depth is the maximum number retained. DataLoader workers can prepare CPU batches, the pinning stage can create page-locked batches, and a CUDA stream can transfer GPU-ready batches. These queues occupy different memory tiers and have different completion rules. Backpressure is mandatory when a budget is reached.

Part 3 - Quantitative Prediction Before Execution

Let preparation, transfer, and compute times be T_p, T_h, T_g. Without prefetching, one batch can cost $T_p + T_h + T_g$. An ideal pipeline approaches $\max(T_p, T_h, T_g)$ after fill. If a CPU batch costs M_c bytes and a CUDA batch costs M_g, lookahead d adds approximately dM_c RAM and dM_g VRAM, plus worker and allocator overhead.

Part 4 - Experimental Design and Instrumentation

Separate fill, steady-state, and drain. Measure queue wait, loader latency, H2D time, compute time, throughput, peak RAM, pinned memory, allocated VRAM, and reserved VRAM. Validate batch identity and order with generation numbers. Compare against a no-prefetch baseline using identical samples and model state.

Part 5 - PyTorch/CUDA Implementation

```
import math

def E00(prep_ms=4.0, h2d_ms=2.0, compute_ms=6.0,
        cpu_batch_MiB=32.0, gpu_batch_MiB=32.0, distance=2):
    serial_ms = prep_ms + h2d_ms + compute_ms
    ideal_steady_ms = max(prep_ms, h2d_ms, compute_ms)
    maximum_speedup = serial_ms / ideal_steady_ms
    extra_ram_MiB = distance * cpu_batch_MiB
    extra_vram_MiB = distance * gpu_batch_MiB
    fill_ms = prep_ms + h2d_ms
    predicted_total_ms = fill_ms + ideal_steady_ms
    report = {
        "serial_ms_per_batch": serial_ms,
        "ideal_steady_ms_per_batch": ideal_steady_ms,
        "maximum_speedup": maximum_speedup,
        "distance": distance,
        "extra_RAM_MiB": extra_ram_MiB,
        "extra_VRAM_MiB": extra_vram_MiB,
        "first_batch_ready_ms": predicted_total_ms,
    }
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

The formula is a ceiling, not a promise: worker startup, IPC, storage contention, shape variation, stream dependencies, and memory pressure lower the gain. Increasing distance while throughput is flat is over-prefetching. A fast but misordered or stale batch is a correctness failure.

Part 7 - Engineering Interpretation and Philosophy

Prefetching spends memory to buy time. The correct objective is the smallest bounded lookahead that keeps the consumer supplied, not the largest queue the machine can survive.

19.11.2 Exercise 01

Exercise Title: Measure a Training Loop Without Prefetching and Identify CPU Loading, H2D Transfer, and GPU Idle Regions

Part 1 - Exercise Contract and Why It Matters

Create a deliberately sequential baseline and identify exactly where the GPU waits for loading and transfer. This becomes the counterfactual for every later prefetch claim.

Part 2 - System and Memory Mental Model

With `num_workers=0`, dataset access occurs in the training process. With blocking transfer followed by compute, no future batch is prepared while the current batch runs. Wall time includes CPU preparation, transfer, compute, and synchronization; CUDA events measure only device intervals.

Part 3 - Quantitative Prediction Before Execution

Predict iteration time as $L_i + H_i + G_i$. GPU idle time before H2D is roughly loader latency L_i, while idle time between completed compute and the next kernel includes the next load and exposed transfer.

Part 4 - Experimental Design and Instrumentation

Use a synthetic dataset with controlled delay, a single-process DataLoader, pinned memory disabled, and explicit phase timers. Warm the model first. Record per-batch load, transfer, and compute durations plus a profiler trace.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch
from torch.utils.data import DataLoader, Dataset

class SlowDataset(Dataset):
    def __init__(self, count=512, width=1024, delay_ms=2.0):
        self.count = count
        self.width = width
        self.delay_ms = delay_ms

    def __len__(self):
        return self.count

    def __getitem__(self, index):
        time.sleep(self.delay_ms / 1000.0)
        features = torch.full((self.width,), float(index % 17))
        label = torch.tensor(index % 8, dtype=torch.long)
        return features, label

def E01(batch_size=32, batches=12):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
```

```
    device = torch.device("cuda:0")
    loader = DataLoader(SlowDataset(), batch_size=batch_size,
                        num_workers=0, pin_memory=False)
    model = torch.nn.Linear(1024, 8).to(device)
    iterator = iter(loader)
    records = []
    for batch_index in range(batches):
        load_start = time.perf_counter()
        features, labels = next(iterator)
        load_ms = (time.perf_counter() - load_start) * 1000.0
        transfer_start = time.perf_counter()
        features = features.to(device)
        labels = labels.to(device)
        torch.cuda.synchronize(device)
        transfer_ms = (time.perf_counter() - transfer_start) * 1000.0
        compute_start = torch.cuda.Event(enable_timing=True)
        compute_end = torch.cuda.Event(enable_timing=True)
        compute_start.record()
        loss = torch.nn.functional.cross_entropy(model(features), labels)
        compute_end.record()
        compute_end.synchronize()
        records.append({"load_ms": load_ms, "transfer_ms": transfer_ms,
                        "compute_ms": compute_start.elapsed_time(compute_end),
                        "loss": float(loss)})
    return records
```

Part 6 - Measurement, Validation, and Failure Analysis

This is an intentionally poor pipeline. Do not compare it with a prefetched run that changes batch size, transforms, or model. A host timer without final CUDA synchronization understates completion time. First-iteration startup must be reported separately.

Part 7 - Engineering Interpretation and Philosophy

Idle GPU regions are evidence of supply failure, not automatically a request for more workers. The slow stage may be storage, decoding, IPC, pinning, or H2D; measure the phase before prescribing the queue.

19.11.3 Exercise 02

Exercise Title: Enable DataLoader Prefetching with prefetch_factor and Measure Its Effect on End-to-End Throughput

Part 1 - Exercise Contract and Why It Matters

Enable worker-side prefetching and measure completed training throughput. Explain what `prefetch_factor` controls and why it is meaningful only when worker processes are enabled.

Part 2 - System and Memory Mental Model

Each worker can prepare up to `prefetch_factor` batches ahead. The approximate queued capacity is therefore workers times factor, although runtime state and consumption make the

instantaneous count variable. These are CPU batches; CUDA transfer prefetch is a separate layer.

Part 3 - Quantitative Prediction Before Execution

For w workers and factor f, estimate queued batch capacity as wf and RAM as wfM_b. Predict benefit only if the baseline loader time is exposed on the critical path.

Part 4 - Experimental Design and Instrumentation

Compare `num_workers=0` with a fixed positive worker count and factor. Keep order deterministic, consume the same number of batches, synchronize at outer timing boundaries, and include loader startup separately.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch
from torch.utils.data import DataLoader

def benchmark_loader(dataset, workers, factor, batches=30):
    arguments = {
        "dataset": dataset,
        "batch_size": 32,
        "num_workers": workers,
        "pin_memory": False,
        "persistent_workers": workers > 0,
    }
    if workers > 0:
        arguments["prefetch_factor"] = factor
    loader = DataLoader(**arguments)
    started = time.perf_counter()
    checksum = 0.0
    for batch_index, batch in enumerate(loader):
        features, labels = batch
        checksum += float(features.sum())
        if batch_index + 1 >= batches:
            break
    elapsed = time.perf_counter() - started
    return {"seconds": elapsed, "batches_per_s": batches / elapsed,
            "checksum": checksum}

def E02(dataset):
    baseline = benchmark_loader(dataset, workers=0, factor=None)
    prefetched = benchmark_loader(dataset, workers=4, factor=2)
    return {"baseline": baseline, "prefetched": prefetched,
            "speedup": baseline["seconds"] / prefetched["seconds"]}
```

Part 6 - Measurement, Validation, and Failure Analysis

Dataset objects must be safe for multiprocessing, and Windows execution needs a proper main guard. Worker startup can dominate short runs. Matching checksums alone does not prove identical order; log sample indices when order matters.

Part 7 - Engineering Interpretation and Philosophy

`prefetch_factor` is queue depth per worker, not a magic speed control. It helps when prepared work arrives early enough to cover consumer latency and hurts when memory and IPC cost exceed the waiting it removes.

19.11.4 Exercise 03

Exercise Title: Sweep prefetch_factor Across Multiple Values and Find the Best Throughput-to-RAM Trade-Off

Part 1 - Exercise Contract and Why It Matters

Sweep prefetch factors under controlled conditions and select a Pareto-efficient setting rather than blindly choosing maximum throughput.

Part 2 - System and Memory Mental Model

Larger factors hold more completed or in-progress batches across worker queues. They can smooth latency variation, but also increase resident RAM, shared-memory traffic, stale queued work, and epoch shutdown cost.

Part 3 - Quantitative Prediction Before Execution

Estimate queue bytes as wfM_b. Define marginal efficiency from factor f_1 to f_2 as throughput gain divided by additional MiB. Stop increasing factor when gain falls below a declared threshold.

Part 4 - Experimental Design and Instrumentation

Test factors $1, 2, 4, 8$, repeat trials in rotated order, and report medians. Measure process RSS when available and calculate an explicit batch-byte estimate. Keep workers persistent only when comparing steady epochs consistently.

Part 5 - PyTorch/CUDA Implementation

```
import statistics
import time
from torch.utils.data import DataLoader

def one_trial(dataset, workers, factor, batch_size, batches):
    loader = DataLoader(dataset, batch_size=batch_size,
                        num_workers=workers, prefetch_factor=factor,
                        persistent_workers=True)
    started = time.perf_counter()
    count = 0
    first_batch_bytes = None
    for features, labels in loader:
        if first_batch_bytes is None:
            feature_bytes = features.numel() * features.element_size()
```

```
            label_bytes = labels.numel() * labels.element_size()
            first_batch_bytes = feature_bytes + label_bytes
        count += 1
        if count >= batches:
            break
    elapsed = time.perf_counter() - started
    estimated_queue = workers * factor * first_batch_bytes
    return count / elapsed, estimated_queue

def E03(dataset, workers=4, batch_size=32, batches=40):
    report = []
    for factor in (1, 2, 4, 8):
        rates = []
        queue_bytes = 0
        for trial_index in range(3):
            rate, queue_bytes = one_trial(
                dataset, workers, factor, batch_size, batches)
            rates.append(rate)
        report.append({"prefetch_factor": factor,
                       "median_batches_per_s": statistics.median(rates),
                       "estimated_queue_MiB": queue_bytes / (1024 ** 2)})
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Estimated queue bytes exclude worker copies, Python objects, allocator overhead, and in-progress samples. Peak RSS is needed for real budgeting. Recreating persistent workers per trial includes startup; use multi-epoch measurements when steady training is the target.

Part 7 - Engineering Interpretation and Philosophy

The best factor is the knee of the curve: enough inventory to absorb variation, but not so much that RAM, cancellation latency, and complexity grow without meaningful throughput.

19.11.5 Exercise 04

Exercise Title: Measure How num_workers and prefetch_factor Interact and Identify When Additional Prefetched Batches Stop Helping

Part 1 - Exercise Contract and Why It Matters

Map the interaction between parallel producers and per-worker queue depth. Identify whether performance is limited by worker service rate, queue capacity, IPC, CPU cores, storage, or the GPU consumer.

Part 2 - System and Memory Mental Model

Workers change production concurrency; factor changes inventory per worker. Their product affects queue capacity, but the two parameters are not interchangeable. More workers can contend for CPU, disk, file handles, and shared memory, while a larger factor with too few slow workers cannot increase their production rate.

Part 3 - Quantitative Prediction Before Execution

For every (w, f), report throughput, estimated wfM_b, and marginal gain. A plateau across factors suggests sufficient lookahead; a plateau across workers suggests another bottleneck or resource saturation.

Part 4 - Experimental Design and Instrumentation

Run a grid over workers $1, 2, 4, 8$ and factors $1, 2, 4$. Record CPU utilization, RSS, queue wait, throughput, and failures such as shared-memory exhaustion. Never tune on an unbounded grid.

Part 5 - PyTorch/CUDA Implementation

```
import time
from torch.utils.data import DataLoader

def E04(dataset, batch_size=32, batches=50):
    report = []
    worker_values = (1, 2, 4, 8)
    factor_values = (1, 2, 4)
    for workers in worker_values:
        for factor in factor_values:
            loader = DataLoader(
                dataset,
                batch_size=batch_size,
                num_workers=workers,
                prefetch_factor=factor,
                persistent_workers=True,
            )
            started = time.perf_counter()
            completed = 0
            for features, labels in loader:
                completed += 1
                if completed >= batches:
                    break
            elapsed = time.perf_counter() - started
            report.append({
                "num_workers": workers,
                "prefetch_factor": factor,
                "queue_capacity_estimate": workers * factor,
                "batches_per_s": completed / elapsed,
            })
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

The loop measures loading, not GPU consumption; repeat the winning candidates in the full training pipeline. CPU topology, storage cache, multiprocessing start method, and dataset implementation make results machine-specific.

Part 7 - Engineering Interpretation and Philosophy

Concurrency and buffering solve different problems. Add producers when service capacity is insufficient; add inventory when variability causes temporary starvation. Stop when the consumer or another shared resource becomes dominant.

19.11.6 Exercise 05

Exercise Title: Combine pin_memory=True with DataLoader Prefetching and Quantify CPU RAM, Pinned Memory, and GPU Utilization Changes

Part 1 - Exercise Contract and Why It Matters

Combine worker prefetching with DataLoader pinning and account separately for ordinary RAM, page-locked RAM, transfer behavior, and GPU utilization.

Part 2 - System and Memory Mental Model

Workers prepare pageable CPU batches; DataLoader's pinning stage makes returned tensors page-locked. Pinned memory enables a direct asynchronous H2D path but is a constrained system resource. Queued pageable and pinned copies may coexist temporarily, so total host cost can exceed batch bytes times queue depth.

Part 3 - Quantitative Prediction Before Execution

For batch bytes M_b, workers w, and factor f, use wfM_b only as a lower-order queue estimate. Predict H2D benefit from bytes transferred and whether transfer is exposed behind compute.

Part 4 - Experimental Design and Instrumentation

Compare pinning off and on with identical loader settings. Verify the pinned state explicitly, request non-blocking CUDA transfers, measure synchronized throughput and process memory, and inspect a profiler trace.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch
from torch.utils.data import DataLoader

def run_pinning_case(dataset, pin_memory, batches=40):
    device = torch.device("cuda:0")
    loader = DataLoader(dataset, batch_size=64, num_workers=4,
                        prefetch_factor=2, persistent_workers=True,
                        pin_memory=pin_memory)
    torch.cuda.synchronize(device)
    started = time.perf_counter()
    pinned_observations = []
    checksum = torch.zeros((), device=device)
    for batch_index, (features, labels) in enumerate(loader):
        pinned_observations.append(features.is_pinned())
        features = features.to(device, non_blocking=True)
        labels = labels.to(device, non_blocking=True)
```

```
        checksum = checksum + features.mean() + labels.float().mean()
        if batch_index + 1 >= batches:
            break
    torch.cuda.synchronize(device)
    elapsed = time.perf_counter() - started
    return {"seconds": elapsed, "batches_per_s": batches / elapsed,
            "all_feature_batches_pinned": all(pinned_observations),
            "checksum": float(checksum)}

def E05(dataset):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    pageable = run_pinning_case(dataset, False)
    pinned = run_pinning_case(dataset, True)
    return {"pageable": pageable, "pinned": pinned,
            "speedup": pageable["seconds"] / pinned["seconds"]}
```

Part 6 - Measurement, Validation, and Failure Analysis

`pin_memory=True` does not itself move data to CUDA. If compute is too short, copies remain exposed. If host RAM pressure or pinning overhead rises, throughput may fall. Measure rather than assuming pinned always wins.

Part 7 - Engineering Interpretation and Philosophy

Memory tiers form a pipeline contract. Pinning is useful when it unlocks overlap or bandwidth on a repeated path; it is wasteful when applied indiscriminately to large, infrequently transferred objects.

19.11.7 Exercise 06

Exercise Title: Build a Custom CUDA Data Prefetcher That Moves Batch N+1 to GPU While Batch N Is Computing

Part 1 - Exercise Contract and Why It Matters

Build a one-batch CUDA lookahead iterator that transfers the next pinned batch while the current batch is consumed by the model.

Part 2 - System and Memory Mental Model

The prefetcher owns a loader iterator, a side stream, and the next CUDA batch. It primes one batch, waits only when that batch becomes current, immediately launches the following transfer, and preserves tensor storage until consumers finish.

Part 3 - Quantitative Prediction Before Execution

One-batch lookahead can hide at most $\min(T_h, T_g)$. It adds approximately one CUDA input batch to VRAM and may retain one CPU batch. Fill exposes the first transfer and drain exposes final compute.

Part 4 - Experimental Design and Instrumentation

Require pinned loader output, compare order and labels with a serial iterator, time completed epochs, and profile memcpy/kernel intersection. Test empty and single-batch datasets as edge cases.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class CUDAPrefetcher:
    def __init__(self, loader, device):
        self.iterator = iter(loader)
        self.device = device
        self.stream = torch.cuda.Stream(device=device)
        self.next_cpu_features = None
        self.next_cpu_labels = None
        self.next_features = None
        self.next_labels = None
        self.next_ready = None
        self.retired_cpu = []
        self._preload()

    def _preload(self):
        try:
            features, labels = next(self.iterator)
        except StopIteration:
            self.next_cpu_features = None
            self.next_cpu_labels = None
            self.next_features = None
            self.next_labels = None
            self.next_ready = None
            return
        self.next_cpu_features = features
        self.next_cpu_labels = labels
        with torch.cuda.stream(self.stream):
            self.next_features = self.next_cpu_features.to(
                self.device, non_blocking=True)
            self.next_labels = self.next_cpu_labels.to(
                self.device, non_blocking=True)
            self.next_ready = torch.cuda.Event()
            self.next_ready.record(self.stream)

    def next(self):
        if self.next_features is None:
            return None
        current_stream = torch.cuda.current_stream(self.device)
        current_stream.wait_event(self.next_ready)
        features = self.next_features
        labels = self.next_labels
        cpu_features = self.next_cpu_features
```

```
        cpu_labels = self.next_cpu_labels
        ready = self.next_ready
        features.record_stream(current_stream)
        labels.record_stream(current_stream)
        self.retired_cpu.append((cpu_features, cpu_labels, ready))
        remaining = []
        for entry in self.retired_cpu:
            if not entry[2].query():
                remaining.append(entry)
        self.retired_cpu = remaining
        if len(self.retired_cpu) > 2:
            self.retired_cpu[0][2].synchronize()
            self.retired_cpu.pop(0)
        self._preload()
        return features, labels

def E06(loader):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    prefetcher = CUDAPrefetcher(loader, torch.device("cuda:0"))
    batches = []
    while True:
        batch = prefetcher.next()
        if batch is None:
            break
        features, labels = batch
        batches.append((tuple(features.shape), tuple(labels.shape)))
    return batches
```

Part 6 - Measurement, Validation, and Failure Analysis

The loader must provide pinned tensors for dependable H2D overlap. Waiting before consumption is required; global synchronization is not. The returned tensors are recorded on the consumer stream so their storage is not recycled while kernels still use it.

Part 7 - Engineering Interpretation and Philosophy

A prefetcher is an ownership protocol disguised as an iterator. Its value comes from moving work early while preserving the same sample sequence and exact consumer-visible semantics.

19.11.8 Exercise 07

Exercise Title: Add a Dedicated Prefetch CUDA Stream and Coordinate It with the Compute Stream Using wait_stream()

Part 1 - Exercise Contract and Why It Matters

Make the stream dependency explicit and compare stream-level waiting with device-wide synchronization.

Part 2 - System and Memory Mental Model

`consumer.wait_stream(producer)` makes future consumer work wait for producer work already submitted at that point. It queues a device dependency and does not block Python. It is broader than waiting for a single event when the producer stream contains unrelated work.

Part 3 - Quantitative Prediction Before Execution

Predict correct values with both approaches, but lower host blocking and better pipeline retention with stream waiting. Submission time and completed time must be reported separately.

Part 4 - Experimental Design and Instrumentation

Transfer a deterministic pinned batch on a prefetch stream, enqueue independent current-batch compute, then make default-stream future work wait. Synchronize only before CPU inspection.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

def E07(batch=1024, width=1024):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    prefetch_stream = torch.cuda.Stream(device=device)
    compute_stream = torch.cuda.current_stream(device)
    host_next = torch.full((batch, width), 3.0, pin_memory=True)
    current = torch.randn((batch, width), device=device)
    torch.cuda.synchronize(device)
    started = time.perf_counter()
    with torch.cuda.stream(prefetch_stream):
        next_batch = host_next.to(device, non_blocking=True)
    current_result = torch.sin(current).sum()
    compute_stream.wait_stream(prefetch_stream)
    next_result = next_batch.sum()
    next_batch.record_stream(compute_stream)
    compute_stream.synchronize()
    elapsed = time.perf_counter() - started
    expected = float(3.0 * batch * width)
    return {"seconds": elapsed, "next_result": float(next_result),
            "expected": expected,
            "absolute_error": abs(float(next_result) - expected),
            "current_checksum": float(current_result)}
```

Part 6 - Measurement, Validation, and Failure Analysis

Calling `wait_stream` after submitting the consumer is too late for that operation. A wait covers producer work submitted before the call, not arbitrary future work. Use an event when one precise producer milestone is required.

Part 7 - Engineering Interpretation and Philosophy

Synchronization scope is a design choice. The smallest dependency that proves correctness preserves the most concurrency and documents the true dataflow.

19.11.9 Exercise 08

Exercise Title: Protect Prefetched CUDA Buffers with Correct record_stream() Lifetime Management and Reproduce the Failure When It Is Removed

Part 1 - Exercise Contract and Why It Matters

Demonstrate why a tensor allocated or transferred on a side stream must have its consumer lifetime communicated to the caching allocator when Python references can disappear early.

Part 2 - System and Memory Mental Model

Stream waiting orders operations but does not by itself extend allocator ownership. Recording the consumer stream on a tensor tells the allocator that its storage remains in use there. Without this annotation, storage may be recycled after the last Python reference disappears even while work is queued.

Part 3 - Quantitative Prediction Before Execution

The safe path must produce zero mismatches across many generations. The unsafe negative control may fail nondeterministically; absence of observed corruption does not prove safety.

Part 4 - Experimental Design and Instrumentation

Use deterministic generation sentinels, allocator churn, a delayed consumer, and hundreds of repetitions. Run the unsafe variant only in an isolated process, then fix it with `record_stream`.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def run_lifetime_case(steps, use_record_stream):
    device = torch.device("cuda:0")
    producer = torch.cuda.Stream(device=device)
    consumer = torch.cuda.current_stream(device)
    results = []
    for generation in range(steps):
        with torch.cuda.stream(producer):
            prefetched = torch.full((512, 512), float(generation),
                                    device=device)
        consumer.wait_stream(producer)
        value = prefetched.mean()
        if use_record_stream:
            prefetched.record_stream(consumer)
        results.append(value)
        del prefetched
        churn = torch.empty((512, 512), device=device)
        churn.fill_(-1.0)
```

```
    torch.cuda.synchronize(device)
    observed = torch.stack(results).cpu()
    expected = torch.arange(steps, dtype=observed.dtype)
    error = (observed - expected).abs().max()
    return float(error)

def E08(steps=200):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    safe_error = run_lifetime_case(steps, True)
    return {"safe_maximum_error": safe_error,
            "unsafe_command": "Run use_record_stream=False separately."}
```

Part 6 - Measurement, Validation, and Failure Analysis

Allocator timing differs across systems, so the negative control may pass. CUDA stream sanitizer and long stress runs strengthen detection. Do not ship the unsafe variant merely because one execution was correct.

Part 7 - Engineering Interpretation and Philosophy

Asynchronous correctness includes storage lifetime. Dependencies protect value availability; allocator annotations protect the memory that holds the value. Both obligations must be satisfied.

19.11.10 Exercise 09

Exercise Title: Build a Two-Batch Lookahead Prefetcher and Compare Its Latency Hiding Against One-Batch Lookahead

Part 1 - Exercise Contract and Why It Matters

Extend CUDA lookahead from one to two batches and determine whether additional depth absorbs latency variation or only consumes memory.

Part 2 - System and Memory Mental Model

Two future batches form a bounded FIFO. Each entry needs a transfer-complete event and a lifetime that extends through compute. Extra distance helps when one future batch is insufficient to bridge variable production or transfer latency; it cannot beat the slowest sustained stage.

Part 3 - Quantitative Prediction Before Execution

Two-batch lookahead adds one CPU/CUDA batch beyond the one-batch design. Predict the throughput gain and calculate gain per extra MiB. Include fill latency, because deeper lookahead delays readiness if the implementation fills eagerly.

Part 4 - Experimental Design and Instrumentation

Benchmark distances one and two with identical batch sequence. Report completed throughput, time-to-first-batch, peak VRAM, output parity, and profiler evidence.

Part 5 - PyTorch/CUDA Implementation

```
from collections import deque
import torch

class LookaheadPrefetcher:
    def __init__(self, loader, device, distance):
        self.iterator = iter(loader)
        self.device = device
        self.distance = distance
        self.stream = torch.cuda.Stream(device=device)
        self.queue = deque()
        self.retired_cpu = []
        for slot_index in range(distance):
            if not self._append_one():
                break

    def _append_one(self):
        try:
            features, labels = next(self.iterator)
        except StopIteration:
            return False
        event = torch.cuda.Event()
        with torch.cuda.stream(self.stream):
            gpu_features = features.to(self.device, non_blocking=True)
            gpu_labels = labels.to(self.device, non_blocking=True)
            event.record(self.stream)
        self.queue.append((gpu_features, gpu_labels, event,
                           features, labels))
        return True

    def next(self):
        if len(self.queue) == 0:
            return None
        features, labels, event, cpu_features, cpu_labels = \
            self.queue.popleft()
        current = torch.cuda.current_stream(self.device)
        current.wait_event(event)
        features.record_stream(current)
        labels.record_stream(current)
        self.retired_cpu.append((cpu_features, cpu_labels, event))
        remaining = []
        for entry in self.retired_cpu:
            if not entry[2].query():
                remaining.append(entry)
        self.retired_cpu = remaining
        if len(self.retired_cpu) > self.distance + 1:
            self.retired_cpu[0][2].synchronize()
            self.retired_cpu.pop(0)
        self._append_one()
```

```
        return features, labels

def E09(loader, distance=2):
    device = torch.device("cuda:0")
    prefetcher = LookaheadPrefetcher(loader, device, distance)
    count = 0
    while prefetcher.next() is not None:
        count += 1
    return {"distance": distance, "batches": count}
```

Part 6 - Measurement, Validation, and Failure Analysis

Queue entries are CUDA tensors and therefore consume VRAM before use. A larger distance can cause OOM even when training itself fits. Check empty datasets and partial final batches, and never infer benefit from queue fullness alone.

Part 7 - Engineering Interpretation and Philosophy

Lookahead depth is insurance against latency variation. Pay only for the amount of insurance that measurably reduces consumer starvation under the real workload.

19.11.11 Exercise 10

Exercise Title: Increase Prefetch Distance Until Extra Buffered Tensors Cause Memory Pressure and Determine the Optimal Lookahead Depth

Part 1 - Exercise Contract and Why It Matters

Increase distance safely, quantify the point of diminishing throughput, and stop before memory pressure threatens training stability.

Part 2 - System and Memory Mental Model

Every prefetched CPU or CUDA batch extends a tensor lifetime. Larger queues raise allocated memory, allocator reservation, pinned RAM, and cancellation waste. Approaching capacity can trigger retries or OOM even when average use appears safe because model activations peak later.

Part 3 - Quantitative Prediction Before Execution

For distance d, predict $M(d) = M_{base} + dM_{batch}$. Define headroom as budget minus measured peak and reject configurations below a fixed safety margin. Choose the smallest d within a throughput tolerance of the best.

Part 4 - Experimental Design and Instrumentation

Sweep bounded distances, reset peak statistics, warm each case, and record throughput, first-batch latency, allocated/reserved VRAM, RAM, and correctness. Terminate by budget, not by intentionally causing OOM.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def E10(measure_case, distances=(0, 1, 2, 3, 4, 6, 8),
        vram_budget_MiB=4096, safety_MiB=512):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    report = []
    for distance in distances:
        torch.cuda.empty_cache()
        torch.cuda.reset_peak_memory_stats(device)
        result = measure_case(distance)
        peak_MiB = torch.cuda.max_memory_allocated(device) / (1024 ** 2)
        headroom_MiB = vram_budget_MiB - peak_MiB
        accepted = headroom_MiB >= safety_MiB
        report.append({
            "distance": distance,
            "throughput": result["throughput"],
            "peak_allocated_MiB": peak_MiB,
            "headroom_MiB": headroom_MiB,
            "accepted": accepted,
        })
        if not accepted:
            break
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

`empty_cache()` is used only between isolated configurations, never per training step. Peak allocated memory excludes some non-PyTorch allocations, so the safety margin must include device-level evidence and model variability.

Part 7 - Engineering Interpretation and Philosophy

The optimum is not the last configuration before failure. It is the shallowest queue that delivers near-best throughput with defensible operational headroom.

19.11.12 Exercise 11

Exercise Title: Design a Fixed-RAM Prefetch Budget That Limits the Number of Queued CPU and Pinned Batches Dynamically

Part 1 - Exercise Contract and Why It Matters

Replace a fixed item count with byte-accounted admission so CPU and pinned queues cannot exceed a declared host-memory budget.

Part 2 - System and Memory Mental Model

A batch queue owns bytes from admission until release. Variable objects require recursive tensor accounting, while Python containers and allocator overhead need a safety multiplier. A condition variable provides backpressure when the next batch would cross the budget.

Part 3 - Quantitative Prediction Before Execution

Admit batch i only when $U + M_i \leq B$, where U is currently owned bytes and B is usable budget after safety reserve. Predict maximum equal-size depth as $\lfloor B/M_b \rfloor$.

Part 4 - Experimental Design and Instrumentation

Log admitted bytes, peak owned bytes, producer wait, consumer wait, queue length, and batch generations. Test one batch larger than the budget and fail explicitly instead of deadlocking.

Part 5 - PyTorch/CUDA Implementation

```
import queue
import threading
import torch

def tensor_tree_bytes(value):
    if torch.is_tensor(value):
        return value.numel() * value.element_size()
    if isinstance(value, dict):
        total = 0
        for child in value.values():
            total += tensor_tree_bytes(child)
        return total
    if isinstance(value, (list, tuple)):
        total = 0
        for child in value:
            total += tensor_tree_bytes(child)
        return total
    return 0

class RAMBudgetQueue:
    def __init__(self, budget_bytes):
        self.budget_bytes = budget_bytes
        self.used_bytes = 0
        self.items = queue.Queue()
        self.condition = threading.Condition()

    def put(self, batch):
        size = tensor_tree_bytes(batch)
        if size > self.budget_bytes:
            raise ValueError("One batch exceeds the complete RAM budget.")
        with self.condition:
            while self.used_bytes + size > self.budget_bytes:
                self.condition.wait()
            self.used_bytes += size
            self.items.put((batch, size))

    def get(self):
```

```
        batch, size = self.items.get()
        with self.condition:
            self.used_bytes -= size
            self.condition.notify_all()
        return batch

def E11(budget_MiB=256):
    return RAMBudgetQueue(int(budget_MiB * 1024 ** 2))
```

Part 6 - Measurement, Validation, and Failure Analysis

Releasing budget at `get()` is valid only if queue ownership ends there. If asynchronous H2D still reads the pinned batch, release its bytes after the copy-complete event instead. Logical accounting must match physical lifetime.

Part 7 - Engineering Interpretation and Philosophy

Budgets turn memory from an accident into a contract. Byte-aware backpressure keeps the system bounded and makes overload appear as measurable waiting rather than a sudden host failure.

19.11.13 Exercise 12

Exercise Title: Design a Fixed-VRAM Prefetch Budget That Prevents Prefetched CUDA Tensors from Triggering Out-of-Memory Failures

Part 1 - Exercise Contract and Why It Matters

Admit CUDA-prefetched batches only inside a fixed VRAM allowance that preserves room for model parameters, gradients, optimizer state, activations, temporary kernels, and allocator behavior.

Part 2 - System and Memory Mental Model

Input tensor bytes are not the whole peak. Prefetched tensors coexist with the training maximum, and reserved memory differs from live allocated memory. A prefetch budget should be derived after measuring training without lookahead and subtracting an explicit safety reserve.

Part 3 - Quantitative Prediction Before Execution

Set usable prefetch bytes to

$$B_p = V_{limit} - V_{training_peak} - V_{safety}.$$

Admit batch i only when owned prefetch bytes plus M_i do not exceed B_p. If $B_p < M_i$, fall back to just-in-time transfer.

Part 4 - Experimental Design and Instrumentation

Measure baseline training peak first. Track logical prefetch bytes, actual allocated/reserved VRAM, admission rejections, consumer stalls, and output parity. Test near-budget cases without provoking OOM.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class VRAMAdmission:
    def __init__(self, total_limit_bytes, training_peak_bytes,
                 safety_bytes):
        available = total_limit_bytes - training_peak_bytes - safety_bytes
        self.prefetch_budget = max(0, available)
        self.owned_bytes = 0

    def can_admit(self, batch_bytes):
        return self.owned_bytes + batch_bytes <= self.prefetch_budget

    def admit(self, batch_bytes):
        if not self.can_admit(batch_bytes):
            return False
        self.owned_bytes += batch_bytes
        return True

    def release(self, batch_bytes):
        self.owned_bytes -= batch_bytes
        if self.owned_bytes < 0:
            raise RuntimeError("VRAM accounting became negative.")

def E12(vram_limit_MiB, training_peak_MiB, safety_MiB=512):
    controller = VRAMAdmission(
        int(vram_limit_MiB * 1024 ** 2),
        int(training_peak_MiB * 1024 ** 2),
        int(safety_MiB * 1024 ** 2),
    )
    return {"prefetch_budget_MiB":
            controller.prefetch_budget / (1024 ** 2)}
```

Part 6 - Measurement, Validation, and Failure Analysis

Logical bytes must be released only after the prefetched CUDA tensor is no longer used on any stream. Baseline peak can change with shape, mixed precision, or optimizer phase; remeasure under representative worst cases.

Part 7 - Engineering Interpretation and Philosophy

VRAM headroom is an operational reserve, not wasted capacity. A prefetcher that occasionally forces OOM has failed even if its average throughput is higher.

19.11.14 Exercise 13

Exercise Title: Prefetch Variable-Size Batches and Replace Fixed Lookahead Depth with Byte-Aware Memory Accounting

Part 1 - Exercise Contract and Why It Matters

Handle variable-size batches without assuming that equal item counts imply equal memory. Admit future batches by bytes while preserving FIFO order.

Part 2 - System and Memory Mental Model

A fixed distance may be safe for small sequences and catastrophic for a cluster of long ones. Byte-aware lookahead produces variable queue length: many small batches or few large batches under the same budget.

Part 3 - Quantitative Prediction Before Execution

For queued sizes $M_1, \ldots, M_k$, require $\sum_i M_i \leq B$. Predict distance as an outcome of the size sequence, not an input constant. Reserve space for the largest admissible next batch to avoid starvation by fragmentation of the logical budget.

Part 4 - Experimental Design and Instrumentation

Generate a reproducible distribution of sequence lengths. Log bytes and tensor shapes per entry, queue depth, producer wait, peak memory, padding waste, and correct output order.

Part 5 - PyTorch/CUDA Implementation

```
from collections import deque
import torch

def batch_bytes(batch):
    total = 0
    for tensor in batch:
        total += tensor.numel() * tensor.element_size()
    return total

class ByteAwareFIFO:
    def __init__(self, budget_bytes):
        self.budget_bytes = budget_bytes
        self.used_bytes = 0
        self.entries = deque()

    def try_append(self, batch):
        size = batch_bytes(batch)
        if size > self.budget_bytes:
            raise ValueError("Batch is larger than the queue budget.")
        if self.used_bytes + size > self.budget_bytes:
            return False
        self.entries.append((batch, size))
        self.used_bytes += size
        return True

    def pop_left(self):
        batch, size = self.entries.popleft()
        self.used_bytes -= size
        return batch
```

```
def E13(budget_MiB=256):
    fifo = ByteAwareFIFO(int(budget_MiB * 1024 ** 2))
    return fifo
```

Part 6 - Measurement, Validation, and Failure Analysis

Tensor bytes omit object and allocator overhead, so apply a measured reserve. For CUDA entries, release only after stream lifetime completes. A FIFO can suffer head-of-line blocking when one large batch prevents smaller later work; changing order may violate training semantics.

Part 7 - Engineering Interpretation and Philosophy

Memory is measured in bytes, not batch labels. Byte-aware admission converts variable shapes from a hidden risk into an explicit scheduling input.

19.11.15 Exercise 14

Exercise Title: Build an Adaptive Prefetch Controller That Increases or Decreases Lookahead from Measured Load, Transfer, and Compute Times

Part 1 - Exercise Contract and Why It Matters

Build a conservative controller that adjusts lookahead using measured starvation, stage times, and memory headroom without oscillating every iteration.

Part 2 - System and Memory Mental Model

Lookahead should rise when the consumer waits and budgets permit it, and fall when queues remain unused or memory headroom shrinks. Exponential moving averages, cooldown, and hysteresis separate sustained change from noise.

Part 3 - Quantitative Prediction Before Execution

Estimate required depth as

$$d_t = 1 + \left\lceil \frac{T_p + T_h}{\max(T_g, \epsilon)} \right\rceil,$$

then clamp by RAM, VRAM, and hard limits. Change at most one step per control interval.

Part 4 - Experimental Design and Instrumentation

Feed the controller controlled phase changes: slow storage, faster compute, and large batches. Log every decision and reason, previous and new depth, stage averages, memory headroom, throughput, and oscillation count.

Part 5 - PyTorch/CUDA Implementation

```
import math

class AdaptivePrefetch:
    def __init__(self, minimum=1, maximum=8, alpha=0.2):
        self.minimum = minimum
        self.maximum = maximum
```

```
        self.alpha = alpha
        self.depth = minimum
        self.load_ms = None
        self.h2d_ms = None
        self.compute_ms = None

    def _update(self, old, value):
        if old is None:
            return value
        return self.alpha * value + (1.0 - self.alpha) * old

    def observe(self, load_ms, h2d_ms, compute_ms):
        self.load_ms = self._update(self.load_ms, load_ms)
        self.h2d_ms = self._update(self.h2d_ms, h2d_ms)
        self.compute_ms = self._update(self.compute_ms, compute_ms)

    def decide(self, ram_limit, vram_limit):
        denominator = max(self.compute_ms, 1.0e-9)
        target = 1 + math.ceil(
            (self.load_ms + self.h2d_ms) / denominator)
        target = min(target, ram_limit, vram_limit, self.maximum)
        target = max(target, self.minimum)
        if target > self.depth:
            self.depth += 1
        elif target < self.depth:
            self.depth -= 1
        return self.depth

def E14():
    return AdaptivePrefetch()
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not resize queues while entries are silently discarded. Apply decisions at a safe boundary and preserve FIFO order. Time-based target is an estimate; compare neighboring depths and roll back if throughput or memory safety worsens.

Part 7 - Engineering Interpretation and Philosophy

An adaptive controller is valuable when it explains and bounds change. It should reduce manual tuning while remaining subordinate to correctness and hard memory budgets.

19.11.16 Exercise 15

Exercise Title: Introduce Artificial Storage Latency and Measure When Dataset Prefetching Becomes More Important Than GPU Transfer Prefetching

Part 1 - Exercise Contract and Why It Matters

Increase storage latency until CPU dataset prefetch dominates the benefit from CUDA transfer lookahead. Identify which queue deserves the next byte of memory.

Part 2 - System and Memory Mental Model

Dataset prefetch hides storage, decoding, and transforms; CUDA prefetch hides H2D. If storage latency exceeds the combined transfer and compute window, an empty CPU queue starves every downstream stage. GPU lookahead cannot transfer a batch that has not been produced.

Part 3 - Quantitative Prediction Before Execution

Sweep storage delay S. When worker aggregate production interval exceeds GPU consumption interval, CPU queue starvation controls throughput. Compare four cases: neither, loader-only, CUDA-only, and both.

Part 4 - Experimental Design and Instrumentation

Inject deterministic delay in `__getitem__`, vary it geometrically, and measure loader wait, transfer wait, GPU gaps, throughput, RAM, and VRAM. Keep data values identical across configurations.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch
from torch.utils.data import Dataset

class StorageLatencyDataset(Dataset):
    def __init__(self, count, width, latency_ms):
        self.count = count
        self.width = width
        self.latency_ms = latency_ms

    def __len__(self):
        return self.count

    def __getitem__(self, index):
        time.sleep(self.latency_ms / 1000.0)
        generator = torch.Generator().manual_seed(index)
        features = torch.randn((self.width,), generator=generator)
        label = torch.tensor(index % 8, dtype=torch.long)
        return features, label

def E15(count=1024, width=1024):
    datasets = []
    for latency_ms in (0.0, 0.5, 1.0, 2.0, 4.0, 8.0):
        dataset = StorageLatencyDataset(count, width, latency_ms)
        datasets.append({"latency_ms": latency_ms, "dataset": dataset})
    return datasets
```

Part 6 - Measurement, Validation, and Failure Analysis

`sleep` models latency but not bandwidth contention or decoding CPU cost. Add real-storage trials before production decisions. OS page cache can erase apparent disk latency; report cold and warm conditions separately.

Part 7 - Engineering Interpretation and Philosophy

Prefetch the scarce future first. When data production is late, deeper CUDA queues are empty promises; when CPU supply is healthy and H2D is exposed, GPU lookahead becomes the correct investment.

19.11.17 Exercise 16

Exercise Title: Prefetch Model Parameters for the Next Layer from CPU to GPU Before the Current Layer Finishes Computing

Part 1 - Exercise Contract and Why It Matters

Prototype layer-wise parameter streaming for a model too large to keep entirely resident in VRAM. Transfer layer $k+1$ while layer k computes, without changing outputs.

Part 2 - System and Memory Mental Model

CPU owns pinned master parameters. Two CUDA staging slots alternate: one layer computes while the next layer transfers. A ready event protects consumption and a compute-done event protects slot overwrite. Parameter transfer is useful only when layer compute can cover next-layer bytes.

Part 3 - Quantitative Prediction Before Execution

For layer k, hiding requires $T_{compute,k} \geq T_{copy,k+1}$. Peak staging VRAM is approximately the two largest simultaneously resident layer parameter sets plus activations and workspace.

Part 4 - Experimental Design and Instrumentation

Use a small inference-only chain first. Compare against a fully resident model, check maximum output error, log per-layer bytes/events, and profile overlap. Training additionally requires gradient and optimizer-state ownership and is outside this first prototype.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def E16(x_cpu, cpu_weights, cpu_biases):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    device = torch.device("cuda:0")
    transfer = torch.cuda.Stream(device=device)
    compute = torch.cuda.current_stream(device)
    x = x_cpu.to(device)
    def prefetch(layer_index):
        ready_event = torch.cuda.Event()
        with torch.cuda.stream(transfer):
            weight = cpu_weights[layer_index].to(
                device, non_blocking=True)
            bias = cpu_biases[layer_index].to(
                device, non_blocking=True)
            ready_event.record(transfer)
```

```
        return weight, bias, ready_event

    next_weight, next_bias, next_ready = prefetch(0)
    for layer_index in range(len(cpu_weights)):
        compute.wait_event(next_ready)
        weight = next_weight
        bias = next_bias
        weight.record_stream(compute)
        bias.record_stream(compute)
        if layer_index + 1 < len(cpu_weights):
            next_weight, next_bias, next_ready = prefetch(
                layer_index + 1)
        x = torch.nn.functional.linear(x, weight, bias)
        if layer_index + 1 < len(cpu_weights):
            x = torch.relu(x)
    compute.synchronize()
    return x
```

Part 6 - Measurement, Validation, and Failure Analysis

CPU parameter tensors must be pinned for reliable asynchronous H2D. Layer shapes can require differently sized staging storage. This readable prototype allocates CUDA tensors per layer; a production engine preallocates byte-budgeted slots and must handle errors without losing master parameters.

Part 7 - Engineering Interpretation and Philosophy

Parameter prefetch exchanges residency for bandwidth and scheduling complexity. It is justified when capacity prevents full residency and useful compute is long enough to hide transfers.

19.11.18 Exercise 17

Exercise Title: Build a Parameter Prefetch Queue That Tracks Layer Order, Available VRAM, Transfer Completion, and Tensor Lifetime

Part 1 - Exercise Contract and Why It Matters

Turn layer prefetch into a bounded queue whose entries expose identity, bytes, ready state, consumer lifetime, and release state.

Part 2 - System and Memory Mental Model

A parameter entry progresses through planned, admitted, transferring, ready, consuming, and released states. Admission reserves logical VRAM before transfer. An event marks readiness; `record_stream` protects consumption; release occurs only after the consumer's completion.

Part 3 - Quantitative Prediction Before Execution

For queued layers Q, require $\sum_{k \in Q} M_k \leq B_p$. Preserve layer order unless the computation graph proves independence. Predict stalls where the next required layer is not ready.

Part 4 - Experimental Design and Instrumentation

Log layer index, bytes, admission, transfer timestamps, ready wait, compute completion, release, queue depth, and VRAM counters. Inject a transfer failure and ensure reserved bytes are rolled back.

Part 5 - PyTorch/CUDA Implementation

```
from collections import deque
from dataclasses import dataclass
import torch

@dataclass
class ParameterEntry:
    layer_index: int
    tensors: tuple
    size_bytes: int
    ready_event: object

class ParameterPrefetchQueue:
    def __init__(self, budget_bytes, device):
        self.budget_bytes = budget_bytes
        self.device = device
        self.used_bytes = 0
        self.entries = deque()

    def try_admit(self, layer_index, cpu_tensors, transfer_stream):
        size = 0
        for tensor in cpu_tensors:
            size += tensor.numel() * tensor.element_size()
        if self.used_bytes + size > self.budget_bytes:
            return False
        event = torch.cuda.Event()
        gpu_tensors = []
        with torch.cuda.stream(transfer_stream):
            for tensor in cpu_tensors:
                gpu_tensors.append(
                    tensor.to(self.device, non_blocking=True))
            event.record(transfer_stream)
        entry = ParameterEntry(layer_index, tuple(gpu_tensors), size, event)
        self.entries.append(entry)
        self.used_bytes += size
        return True

    def consume_next(self, compute_stream):
        entry = self.entries.popleft()
        compute_stream.wait_event(entry.ready_event)
        for tensor in entry.tensors:
            tensor.record_stream(compute_stream)
```

```
        return entry

    def release(self, entry):
        self.used_bytes -= entry.size_bytes

def E17(budget_MiB=1024):
    device = torch.device("cuda:0")
    return ParameterPrefetchQueue(int(budget_MiB * 1024 ** 2), device)
```

Part 6 - Measurement, Validation, and Failure Analysis

`release` must be called after the consumer stream has completed its use, not immediately after submission. In a production queue, associate a compute-done event with the entry and reclaim asynchronously only when it queries complete.

Part 7 - Engineering Interpretation and Philosophy

A parameter queue is a scheduler with accounting, not a cache of anonymous tensors. Traceable state transitions make capacity failures and lifetime bugs diagnosable.

19.11.19 Exercise 18

Exercise Title: Stress-Test Data and Parameter Prefetching Together and Diagnose Over-Prefetching, Memory Spikes, Stream Stalls, and Wasted Transfers

Part 1 - Exercise Contract and Why It Matters

Run data and parameter prefetch concurrently under controlled variability and detect competition for VRAM, copy engines, streams, and memory bandwidth.

Part 2 - System and Memory Mental Model

Two individually effective prefetchers can interfere. Their transfers may share one copy engine, their queues share VRAM, and excessive future work can be wasted when an epoch ends, an error occurs, or control flow skips a layer. One global budget and priority policy must coordinate them.

Part 3 - Quantitative Prediction Before Execution

Require $B_{data} + B_{param} \leq B_{prefetch}$. Predict service order from deadlines: the next batch and next layer receive priority over distant work. Measure wasted transferred bytes that were never consumed.

Part 4 - Experimental Design and Instrumentation

Vary data sizes, layer sizes, storage delay, and compute duration with fixed seeds. Log peak memory, admission denials, stream wait, copy overlap, useful bytes, wasted bytes, output parity, and recovery after cancellation.

Part 5 - PyTorch/CUDA Implementation

```
class SharedPrefetchBudget:
    def __init__(self, total_bytes):
        self.total_bytes = total_bytes
        self.data_bytes = 0
        self.parameter_bytes = 0

    def used_bytes(self):
        return self.data_bytes + self.parameter_bytes

    def try_reserve(self, category, size_bytes):
        if self.used_bytes() + size_bytes > self.total_bytes:
            return False
        if category == "data":
            self.data_bytes += size_bytes
        elif category == "parameter":
            self.parameter_bytes += size_bytes
        else:
            raise ValueError("Unknown prefetch category.")
        return True

    def release(self, category, size_bytes):
        if category == "data":
            self.data_bytes -= size_bytes
        elif category == "parameter":
            self.parameter_bytes -= size_bytes
        else:
            raise ValueError("Unknown prefetch category.")
        if self.data_bytes < 0 or self.parameter_bytes < 0:
            raise RuntimeError("Prefetch accounting became negative.")

def E18(total_budget_MiB=2048):
    total_bytes = int(total_budget_MiB * 1024 ** 2)
    return SharedPrefetchBudget(total_bytes)
```

Part 6 - Measurement, Validation, and Failure Analysis

Logical accounting alone cannot prove engine overlap or reveal non-PyTorch memory. Combine it with CUDA events, profiler traces, memory statistics, and device-level free memory. Stress safely below a fixed limit; never use OOM as the normal stopping rule.

Part 7 - Engineering Interpretation and Philosophy

Prefetchers are competing clients of a finite future-work budget. Coordination should favor imminent useful work, bound speculation, and cancel work whose result will not be consumed.

19.11.20 Exercise 19

Exercise Title: Build and Benchmark a Memory-Aware PyTorch Prefetch Engine That Dynamically Chooses What to Load Next Under Fixed RAM and VRAM Budgets

Part 1 - Exercise Contract and Why It Matters

Integrate the chapter into one bounded engine that chooses between future data and parameters according to deadlines, measured stage times, and fixed RAM/VRAM budgets. Benchmark it against no prefetch and fixed-depth policies.

Part 2 - System and Memory Mental Model

The engine has planners, byte-accounted admission, CPU/pinned queues, CUDA transfer streams, completion events, consumer lifetime tracking, backpressure, cancellation, and telemetry. A request is eligible only if its dependencies are known, its bytes fit, and its result is likely to be consumed.

Part 3 - Quantitative Prediction Before Execution

Score request r from deadline urgency, predicted stall avoided, transfer cost, and memory residence:

$$score(r) = \frac{stall_avoided(r)}{copy_time(r) + \epsilon} - \lambda\, memory_bytes(r).$$

Hard budgets override score. Predict throughput and peak memory before running, then compare realized and predicted values.

Part 4 - Experimental Design and Instrumentation

Use deterministic data and initial weights. Run serial, fixed one-step, fixed two-step, and adaptive policies. Verify outputs or training updates, and report fill/steady/drain time, starvation, useful/wasted bytes, RAM, pinned RAM, VRAM, queue depth, and profiler overlap.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass
import heapq

@dataclass
class PrefetchRequest:
    name: str
    category: str
    size_bytes: int
    deadline_step: int
    copy_ms: float
    stall_avoided_ms: float
    payload: object

class MemoryAwarePrefetchEngine:
    def __init__(self, ram_budget_bytes, vram_budget_bytes,
                 memory_penalty=1.0e-12):
        self.ram_budget_bytes = ram_budget_bytes
        self.vram_budget_bytes = vram_budget_bytes
        self.memory_penalty = memory_penalty
        self.ram_used = 0
        self.vram_used = 0
        self.pending = []
```

```
        self.sequence = 0

    def score(self, request):
        efficiency = request.stall_avoided_ms / max(
            request.copy_ms, 1.0e-9)
        memory_cost = self.memory_penalty * request.size_bytes
        urgency = 1.0 / max(request.deadline_step, 1)
        return efficiency + urgency - memory_cost

    def submit(self, request):
        priority = -self.score(request)
        entry = (priority, self.sequence, request)
        heapq.heappush(self.pending, entry)
        self.sequence += 1

    def next_admissible(self):
        deferred = []
        selected = None
        while len(self.pending) > 0:
            priority, sequence, request = heapq.heappop(self.pending)
            ram_fits = self.ram_used + request.size_bytes <= \
                self.ram_budget_bytes
            vram_fits = self.vram_used + request.size_bytes <= \
                self.vram_budget_bytes
            if ram_fits and vram_fits:
                selected = request
                self.ram_used += request.size_bytes
                self.vram_used += request.size_bytes
                break
            deferred.append((priority, sequence, request))
        for entry in deferred:
            heapq.heappush(self.pending, entry)
        return selected

    def release(self, request):
        self.ram_used -= request.size_bytes
        self.vram_used -= request.size_bytes

def E19(ram_budget_MiB=2048, vram_budget_MiB=2048):
    ram_bytes = int(ram_budget_MiB * 1024 ** 2)
    vram_bytes = int(vram_budget_MiB * 1024 ** 2)
    return MemoryAwarePrefetchEngine(ram_bytes, vram_bytes)
```

Part 6 - Measurement, Validation, and Failure Analysis

This policy skeleton separates planning from actual transfer execution. A production implementation reserves host and device bytes independently, releases them at their real completion boundaries, ages requests to prevent starvation, and rolls back every partial failure. The model must never consume a request whose transfer event is incomplete.

Part 7 - Engineering Interpretation and Philosophy

The final engine is not successful because it predicts farther ahead. It is successful when it chooses the smallest amount of useful future work that removes stalls, stays within hard budgets, preserves exact computation, and produces evidence another engineer can reproduce.

19.12 DMM10

Chapter Title: Memory Eviction Policies
Specialization: Dynamic Memory Management and CUDA Systems

19.12.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Memory Eviction, Residency, Reuse Distance, and the Cost of Removing Data from VRAM

Part 1 - Exercise Contract and Why It Matters

Build a falsifiable model of residency and eviction. Distinguish dropping a cache entry from actually making its CUDA storage reusable, and account for the cost of restoring an evicted value.

Part 2 - System and Memory Mental Model

A resident tensor has live GPU storage and may also have a CPU backing copy. Eviction selects a victim, waits for every GPU user, writes dirty state back if required, removes cache ownership, and permits allocator reuse. Reuse distance is the number of intervening accesses before the same key returns. A miss incurs lookup, CPU availability, H2D transfer, and possibly recomputation.

Part 3 - Quantitative Prediction Before Execution

For access i, define hit cost C_h, reload cost C_r, and write-back cost C_w. Total cost is

$$C = \sum_i hit_i C_h + \sum_i miss_i C_r + \sum_i dirtyEvict_i C_w.$$

Predict working-set bytes, reuse distances, hit ratio, traffic bytes, and peak VRAM before running a policy.

Part 4 - Experimental Design and Instrumentation

Use deterministic keys and tensors. Log access sequence, size, resident bytes, victim, dirty/protected state, reload/write-back time, hits, misses, and CUDA memory counters. Validate values after every reload.

Part 5 - PyTorch/CUDA Implementation

```
def E00(accesses, sizes_bytes, reload_ms, budget_bytes):
    resident = set()
    resident_bytes = 0
    hits = 0
    misses = 0
    reload_cost_ms = 0.0
```

```
    trace = []
    for step, key in enumerate(accesses):
        if key in resident:
            hits += 1
            outcome = "hit"
        else:
            misses += 1
            reload_cost_ms += reload_ms[key]
            resident.add(key)
            resident_bytes += sizes_bytes[key]
            outcome = "miss"
        trace.append({"step": step, "key": key, "outcome": outcome,
                      "resident_bytes": resident_bytes,
                      "over_budget": resident_bytes > budget_bytes})
    total = hits + misses
    return {"hits": hits, "misses": misses,
            "hit_rate": hits / max(total, 1),
            "reload_cost_ms": reload_cost_ms, "trace": trace}
```

Part 6 - Measurement, Validation, and Failure Analysis

This model intentionally flags over-budget states without choosing victims. Later exercises supply policies. Do not equate `del tensor` with immediate device-level free memory: references, asynchronous use, and the caching allocator all matter.

Part 7 - Engineering Interpretation and Philosophy

Eviction is a prediction about future usefulness under a hard capacity limit. The best policy minimizes avoidable future cost while never sacrificing data correctness or active-stream safety.

19.12.2 Exercise 01

Exercise Title: Build a Fixed-Capacity PyTorch GPU Tensor Cache and Trigger Eviction When Its VRAM Budget Is Exceeded

Part 1 - Exercise Contract and Why It Matters

Build a byte-budgeted CUDA tensor cache with explicit admission, eviction, and accounting. Never use tensor count as a substitute for bytes.

Part 2 - System and Memory Mental Model

The cache owns a CUDA tensor from admission until safe eviction. Logical resident bytes describe cache responsibility; PyTorch allocated and reserved counters describe allocator state. They should be compared but need not be equal.

Part 3 - Quantitative Prediction Before Execution

Before admitting size M, evict until $U + M \leq B$. Reject one object larger than the entire budget. Predict peak logical bytes never exceed B.

Part 4 - Experimental Design and Instrumentation

Insert variable-size pinned CPU tensors, validate values, and record logical bytes plus CUDA allocated/reserved memory after each operation. Start with clean, read-only entries.

Part 5 - PyTorch/CUDA Implementation

```
from collections import OrderedDict
import torch

class FixedGPUCache:
    def __init__(self, budget_bytes, device="cuda:0"):
        self.budget_bytes = budget_bytes
        self.device = torch.device(device)
        self.entries = OrderedDict()
        self.used_bytes = 0

    def tensor_bytes(self, tensor):
        return tensor.numel() * tensor.element_size()

    def evict_one(self):
        key, tensor = self.entries.popitem(last=False)
        size = self.tensor_bytes(tensor)
        self.used_bytes -= size
        del tensor
        return key, size

    def put(self, key, cpu_tensor):
        size = self.tensor_bytes(cpu_tensor)
        if size > self.budget_bytes:
            raise ValueError("Tensor exceeds the complete cache budget.")
        evicted = []
        while self.used_bytes + size > self.budget_bytes:
            evicted.append(self.evict_one())
        gpu_tensor = cpu_tensor.to(self.device)
        self.entries[key] = gpu_tensor
        self.used_bytes += size
        return evicted

def E01(budget_MiB=512):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    return FixedGPUCache(int(budget_MiB * 1024 ** 2))
```

Part 6 - Measurement, Validation, and Failure Analysis

This first cache assumes no active users and clean data. Removing the dictionary reference permits allocator reuse but does not guarantee an immediate drop in `memory_reserved()`. Later exercises add events, dirty state, and protection.

Part 7 - Engineering Interpretation and Philosophy

A capacity invariant is more valuable than reactive OOM handling. Every admission must prove its bytes fit before allocation occurs.

19.12.3 Exercise 02

Exercise Title: Implement FIFO Eviction for CUDA Tensors and Measure Hit Rate, Miss Rate, Transfer Cost, and Peak VRAM

Part 1 - Exercise Contract and Why It Matters

Implement FIFO as the simplest reproducible policy and establish complete cache metrics for later comparisons.

Part 2 - System and Memory Mental Model

FIFO evicts the earliest admitted entry regardless of later access. Hits do not change order. Its simplicity gives low metadata overhead but ignores locality.

Part 3 - Quantitative Prediction Before Execution

Trace the queue manually for a chosen access sequence. Predict hits, misses, victims, H2D bytes, and maximum resident bytes. Transfer cost should be measured with completion-aware timing.

Part 4 - Experimental Design and Instrumentation

Use a CPU backing store, fixed budget, pinned sources, and CUDA events around reloads. Log every victim and compare output checksums to CPU masters.

Part 5 - PyTorch/CUDA Implementation

```
from collections import OrderedDict
import torch

class FIFOCache:
    def __init__(self, backing, capacity):
        self.backing = backing
        self.capacity = capacity
        self.data = OrderedDict()
        self.hits = 0
        self.misses = 0
        self.transfer_bytes = 0

    def get(self, key):
        if key in self.data:
            self.hits += 1
            return self.data[key]
        self.misses += 1
        source = self.backing[key]
        if len(self.data) >= self.capacity:
            old_key, old_tensor = self.data.popitem(last=False)
            del old_tensor
        tensor = source.to("cuda:0", non_blocking=source.is_pinned())
```

```
        self.data[key] = tensor
        self.transfer_bytes += source.numel() * source.element_size()
        return tensor

def E02(backing, accesses, capacity=3):
    cache = FIFOCache(backing, capacity)
    checksums = []
    for key in accesses:
        checksums.append(float(cache.get(key).sum()))
    torch.cuda.synchronize()
    return {"hits": cache.hits, "misses": cache.misses,
            "transfer_bytes": cache.transfer_bytes,
            "checksums": checksums}
```

Part 6 - Measurement, Validation, and Failure Analysis

The example uses item capacity to isolate FIFO order; repeat with the byte capacity from E01 for realistic tensors. Float conversion synchronizes and should be removed from timed production paths.

Part 7 - Engineering Interpretation and Philosophy

FIFO is an important baseline precisely because it knows almost nothing. Metadata sophistication earns its cost only when it beats this controlled reference.

19.12.4 Exercise 03

Exercise Title: Implement LRU Eviction from Scratch and Update Tensor Recency on Every GPU Access

Part 1 - Exercise Contract and Why It Matters

Implement least-recently-used eviction and prove that every hit updates recency, not only insertion.

Part 2 - System and Memory Mental Model

LRU assumes recently used entries are more likely to return soon. An ordered map stores least recent at one end and most recent at the other. Metadata must not retain autograd graphs or duplicate tensor ownership.

Part 3 - Quantitative Prediction Before Execution

For each access, write the recency order before and after. The victim is the least recent resident key when capacity is required.

Part 4 - Experimental Design and Instrumentation

Test hits at both ends, repeated keys, capacity one, and missing keys. Log order, victims, hit ratio, reload bytes, and value parity.

Part 5 - PyTorch/CUDA Implementation

```
from collections import OrderedDict
import torch

class LRUCache:
    def __init__(self, backing, capacity):
        self.backing = backing
        self.capacity = capacity
        self.entries = OrderedDict()
        self.victims = []

    def get(self, key):
        if key in self.entries:
            tensor = self.entries.pop(key)
            self.entries[key] = tensor
            return tensor, True
        if len(self.entries) >= self.capacity:
            victim_key, victim = self.entries.popitem(last=False)
            self.victims.append(victim_key)
            del victim
        tensor = self.backing[key].to("cuda:0")
        self.entries[key] = tensor
        return tensor, False

def E03(backing, accesses, capacity=3):
    cache = LRUCache(backing, capacity)
    hits = 0
    for key in accesses:
        tensor, was_hit = cache.get(key)
        if was_hit:
            hits += 1
    return {"hits": hits, "misses": len(accesses) - hits,
            "victims": cache.victims,
            "final_order": list(cache.entries.keys())}
```

Part 6 - Measurement, Validation, and Failure Analysis

Returning a tensor can extend its life outside the cache; safe eviction therefore requires a usage contract or event tracking. LRU order alone does not solve stream lifetime.

Part 7 - Engineering Interpretation and Philosophy

Recency is a prediction encoded as metadata. Its usefulness depends on locality in the workload, not on the elegance of the data structure.

19.12.5 Exercise 04

Exercise Title: Compare FIFO and LRU Under Sequential, Repeating, and Random Tensor Access Patterns

Part 1 - Exercise Contract and Why It Matters

Compare policies across access distributions and explain why no hit-rate result is meaningful without the workload that produced it.

Part 2 - System and Memory Mental Model

Sequential scans may defeat both policies when the working set exceeds capacity. Repeating hot sets favor LRU; admission order can sometimes favor FIFO. Random access approaches probability determined by capacity and popularity.

Part 3 - Quantitative Prediction Before Execution

Predict the exact trace for a short sequence, then compare hits, misses, evictions, transfer bytes, and policy metadata overhead for longer seeded sequences.

Part 4 - Experimental Design and Instrumentation

Generate sequential, cyclic-hot, and seeded-random traces with the same key set and length. Reset caches and CUDA peak statistics between policies.

Part 5 - PyTorch/CUDA Implementation

```
from collections import OrderedDict
import random

def make_patterns(key_count=10, length=100):
    sequential = []
    repeating = []
    random_access = []
    generator = random.Random(1004)
    for index in range(length):
        sequential.append(index % key_count)
        repeating.append(index % 3)
        random_access.append(generator.randrange(key_count))
    return {"sequential": sequential, "repeating": repeating,
            "random": random_access}

def evaluate_policy(policy, accesses, capacity):
    entries = OrderedDict()
    hits = 0
    for key in accesses:
        if key in entries:
            hits += 1
            if policy == "lru":
                value = entries.pop(key)
                entries[key] = value
        else:
            if len(entries) >= capacity:
                entries.popitem(last=False)
            entries[key] = True
    total = len(accesses)
    return {"hits": hits, "misses": total - hits,
            "hit_rate": hits / max(total, 1)}
```

```
def E04(capacity=3):
    report = {}
    patterns = make_patterns()
    for name, accesses in patterns.items():
        report[name] = {}
        report[name]["fifo"] = evaluate_policy(
            "fifo", accesses, capacity)
        report[name]["lru"] = evaluate_policy(
            "lru", accesses, capacity)
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

The simulator isolates policy order; repeat with real CUDA transfers and warm backing storage consistently. Hit ratio alone can mislead when miss costs and tensor sizes differ.

Part 7 - Engineering Interpretation and Philosophy

An eviction policy is part of a workload-policy pair. Benchmark representative traces, not a single synthetic sequence chosen after seeing the result.

19.12.6 Exercise 05

Exercise Title: Implement LFU Eviction and Track Tensor Access Frequency Without Keeping Autograd References Alive Accidentally

Part 1 - Exercise Contract and Why It Matters

Implement least-frequently-used eviction while keeping metadata scalar and detached from autograd graphs.

Part 2 - System and Memory Mental Model

LFU predicts that historically popular keys remain valuable. Frequency counters store integers by key, never tensors or losses. Recency breaks ties so old one-time popularity does not dominate forever.

Part 3 - Quantitative Prediction Before Execution

Before each miss, rank residents by frequency then last-access step. Predict the victim and compare with LRU on a hot-key trace.

Part 4 - Experimental Design and Instrumentation

Test ties, frequency growth, phase changes, and counter aging. Inspect metadata to prove it contains no tensors with `grad_fn`.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class LFUCache:
    def __init__(self, backing, capacity):
```

```
        self.backing = backing
        self.capacity = capacity
        self.entries = {}
        self.frequency = {}
        self.last_step = {}
        self.step = 0

    def get(self, key):
        self.step += 1
        if key in self.entries:
            self.frequency[key] += 1
            self.last_step[key] = self.step
            return self.entries[key], True
        if len(self.entries) >= self.capacity:
            victim = min(self.entries,
                         key=lambda item: (self.frequency[item],
                                           self.last_step[item]))
            del self.entries[victim]
            del self.frequency[victim]
            del self.last_step[victim]
        self.entries[key] = self.backing[key].to("cuda:0")
        self.frequency[key] = 1
        self.last_step[key] = self.step
        return self.entries[key], False

def E05(backing, accesses, capacity=3):
    cache = LFUCache(backing, capacity)
    for key in accesses:
        cache.get(key)
    metadata_has_tensor = any(
        torch.is_tensor(value) for value in cache.frequency.values())
    return {"frequency": dict(cache.frequency),
            "metadata_has_tensor": metadata_has_tensor}
```

Part 6 - Measurement, Validation, and Failure Analysis

Unbounded counters make ancient popularity sticky; introduce periodic decay for phase-changing workloads. Never derive metadata by storing tensor-valued counts inside the autograd graph.

Part 7 - Engineering Interpretation and Philosophy

Frequency remembers demand, but memory without aging becomes prejudice. Policy state must be cheap, bounded, and relevant to the current phase.

19.12.7 Exercise 06

Exercise Title: Construct a Workload Where LRU Performs Poorly and Explain Why Access Recency Alone Is an Insufficient Eviction Signal

Part 1 - Exercise Contract and Why It Matters

Construct a scan-resistant counterexample where a sequential burst evicts a hot working set that returns immediately afterward.

Part 2 - System and Memory Mental Model

LRU interprets every recent scan item as valuable. A scan longer than cache capacity pushes stable hot keys out, causing misses when the hot phase resumes. Frequency or admission filtering can preserve the hot set.

Part 3 - Quantitative Prediction Before Execution

Trace hot keys A, B, C, a cold scan, then A, B, C again. Predict LRU victims, reload count, and transfer bytes. Compare an LFU or scan-resistant policy.

Part 4 - Experimental Design and Instrumentation

Vary scan length and cache capacity. Report hot-set hit rate separately from overall hit rate, because cold compulsory misses can hide policy damage.

Part 5 - PyTorch/CUDA Implementation

```
def build_lru_failure_trace(hot_repeats=20, scan_length=20):
    accesses = []
    for repeat in range(hot_repeats):
        for key in ("A", "B", "C"):
            accesses.append(key)
    for index in range(scan_length):
        accesses.append("cold_" + str(index))
    for repeat in range(5):
        for key in ("A", "B", "C"):
            accesses.append(key)
    return accesses

def E06(run_policy):
    trace = build_lru_failure_trace()
    lru_result = run_policy("lru", trace, 3)
    lfu_result = run_policy("lfu", trace, 3)
    return {"trace_length": len(trace),
            "lru": lru_result, "lfu": lfu_result}
```

Part 6 - Measurement, Validation, and Failure Analysis

The point is not that LFU always wins; a later phase can invalidate old frequencies. Label compulsory, capacity, and policy-induced misses separately.

Part 7 - Engineering Interpretation and Philosophy

Recency is one signal among several. Counterexamples reveal the assumptions a policy makes and show when those assumptions stop matching reality.

19.12.8 Exercise 07

Exercise Title: Implement Size-Aware Eviction That Chooses Victims Based on Tensor Bytes Instead of Tensor Count

Part 1 - Exercise Contract and Why It Matters

Select enough victim bytes to admit a requested tensor and compare evicting one large entry with many small entries.

Part 2 - System and Memory Mental Model

Capacity is byte-based. A count policy may evict one tiny tensor and remain over budget, or remove a huge tensor unnecessarily. Size-aware selection reasons about reclaimed bytes and the value lost per byte.

Part 3 - Quantitative Prediction Before Execution

Required reclaim is $R = \max(0, U + M_{new} - B)$. Select victims whose total bytes reach R, then minimize a declared loss objective.

Part 4 - Experimental Design and Instrumentation

Use heterogeneous tensor sizes and report reclaimed bytes, excess reclaim, victim count, hit loss, transfer cost, and peak VRAM.

Part 5 - PyTorch/CUDA Implementation

```
def tensor_bytes(tensor):
    return tensor.numel() * tensor.element_size()

def choose_size_aware_victims(entries, required_bytes):
    candidates = []
    for key, tensor in entries.items():
        candidates.append((tensor_bytes(tensor), key))
    candidates.sort(reverse=True)
    victims = []
    reclaimed = 0
    for size, key in candidates:
        victims.append(key)
        reclaimed += size
        if reclaimed >= required_bytes:
            break
    if reclaimed < required_bytes:
        raise RuntimeError("Evictable entries cannot satisfy the request.")
    return victims, reclaimed

def E07(entries, used_bytes, budget_bytes, new_tensor_bytes):
    required = max(0, used_bytes + new_tensor_bytes - budget_bytes)
    if required == 0:
        return {"victims": [], "reclaimed_bytes": 0}
    victims, reclaimed = choose_size_aware_victims(entries, required)
    return {"victims": victims, "reclaimed_bytes": reclaimed,
            "excess_reclaim_bytes": reclaimed - required}
```

Part 6 - Measurement, Validation, and Failure Analysis

Largest-first minimizes victim count, not future reload cost. It can evict a high-value large tensor. E08 adds cost; E18 combines all signals.

Part 7 - Engineering Interpretation and Philosophy

Bytes are the physical constraint, while usefulness determines the opportunity cost. A sound policy needs both.

19.12.9 Exercise 08

Exercise Title: Implement Cost-Aware Eviction That Considers Both Tensor Size and CPU-to-GPU Reload Latency

Part 1 - Exercise Contract and Why It Matters

Avoid evicting entries that are expensive to reload merely because they reclaim many bytes.

Part 2 - System and Memory Mental Model

Reload cost includes source readiness, H2D latency, conversion, and recomputation. Eviction benefit is bytes reclaimed; penalty is expected future reload cost. A cheap victim has low expected cost per reclaimed byte.

Part 3 - Quantitative Prediction Before Execution

Rank by

$$score_k = \frac{P(reuse_k)\, C(reload_k)}{M_k}.$$

Evict lowest scores until enough bytes are reclaimed. Predict traffic and stall cost, not only hit ratio.

Part 4 - Experimental Design and Instrumentation

Measure reload latency repeatedly with CUDA completion timing. Use entries with equal size/different cost and different size/equal cost. Validate sensitivity.

Part 5 - PyTorch/CUDA Implementation

```
def choose_cost_aware_victims(metadata, required_bytes):
    ranked = []
    for key, item in metadata.items():
        size = max(item["size_bytes"], 1)
        expected_cost = item["reuse_probability"] * item["reload_ms"]
        score = expected_cost / size
        ranked.append((score, key, item["size_bytes"]))
    ranked.sort()
    victims = []
    reclaimed = 0
    predicted_penalty_ms = 0.0
    for score, key, size in ranked:
        victims.append(key)
        reclaimed += size
```

```
        predicted_penalty_ms += (
            metadata[key]["reuse_probability"]
            * metadata[key]["reload_ms"])
        if reclaimed >= required_bytes:
            break
    return victims, reclaimed, predicted_penalty_ms

def E08(metadata, required_bytes):
    victims, reclaimed, penalty = choose_cost_aware_victims(
        metadata, required_bytes)
    return {"victims": victims, "reclaimed_bytes": reclaimed,
            "predicted_reload_penalty_ms": penalty}
```

Part 6 - Measurement, Validation, and Failure Analysis

Greedy scoring is not globally optimal for every combination of sizes. Reload measurements change with contention and pinned state. Recalibrate and compare against a small exact solver on tiny cases.

Part 7 - Engineering Interpretation and Philosophy

Eviction should minimize future pain, not maximize immediate freed bytes. Cost models make that trade explicit and testable.

19.12.10 Exercise 09

Exercise Title: Build a Reuse-Distance Tracker and Use Future-Likelihood Estimates to Avoid Evicting Tensors Needed Again Soon

Part 1 - Exercise Contract and Why It Matters

Measure reuse distance online and estimate which resident keys are likely to return before the cache can recover their reload cost.

Part 2 - System and Memory Mental Model

Stack reuse distance counts distinct intervening keys; simpler step distance counts accesses. Small recent distances suggest temporal locality. The tracker stores scalar histories, not tensor references.

Part 3 - Quantitative Prediction Before Execution

For key k, update distance $d = t - last_k$ and an exponential average. Prefer preserving small predicted distances while accounting for uncertainty when a key has little history.

Part 4 - Experimental Design and Instrumentation

Feed periodic, bursty, and phase-changing traces. Compare predicted next reuse with observed distance and measure eviction regret when a victim returns soon.

Part 5 - PyTorch/CUDA Implementation

```
class ReuseDistanceTracker:
    def __init__(self, alpha=0.25):
        self.alpha = alpha
        self.step = 0
        self.last_step = {}
        self.predicted_distance = {}

    def observe(self, key):
        self.step += 1
        distance = None
        if key in self.last_step:
            distance = self.step - self.last_step[key]
            if key not in self.predicted_distance:
                self.predicted_distance[key] = float(distance)
            else:
                old = self.predicted_distance[key]
                self.predicted_distance[key] = (
                    self.alpha * distance + (1.0 - self.alpha) * old)
        self.last_step[key] = self.step
        return distance

    def predicted(self, key):
        return self.predicted_distance.get(key, float("inf"))

def E09(accesses):
    tracker = ReuseDistanceTracker()
    observed = []
    for key in accesses:
        observed.append((key, tracker.observe(key)))
    return {"observed": observed,
            "predictions": dict(tracker.predicted_distance)}
```

Part 6 - Measurement, Validation, and Failure Analysis

History is not clairvoyance. Phase changes invalidate estimates, and infinity for unseen reuse may wrongly evict a new hot key. Add aging, confidence, and admission exploration.

Part 7 - Engineering Interpretation and Philosophy

Reuse distance connects access traces to capacity pressure. Prediction is useful only when uncertainty and mistakes are measured alongside successful hits.

19.12.11 Exercise 10

Exercise Title: Introduce Dirty and Clean Tensor States and Prevent Modified GPU Data from Being Lost During Eviction

Part 1 - Exercise Contract and Why It Matters

Track whether GPU data differs from its CPU backing copy and forbid silent eviction of modified state.

Part 2 - System and Memory Mental Model

A clean entry can be discarded because the backing copy can reconstruct it. A dirty entry contains the newest value in VRAM and requires write-back or an explicit discard decision. Dirty metadata changes only when mutation or successful persistence completes.

Part 3 - Quantitative Prediction Before Execution

Count clean evictions, dirty write-backs, bytes persisted, and blocked evictions. Predict final CPU values after a sequence of GPU mutations.

Part 4 - Experimental Design and Instrumentation

Modify deterministic tensors, mark them dirty, attempt unsafe eviction, then write back and validate CPU parity. Test failed write-back and preserve dirty state on failure.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass
import torch

@dataclass
class CacheEntry:
    gpu_tensor: object
    cpu_tensor: object
    dirty: bool = False
    protected: bool = False

class DirtyAwareCache:
    def __init__(self):
        self.entries = {}

    def mark_dirty(self, key):
        self.entries[key].dirty = True

    def evict_clean(self, key):
        entry = self.entries[key]
        if entry.protected:
            raise RuntimeError("Protected entry cannot be evicted.")
        if entry.dirty:
            raise RuntimeError("Dirty entry requires write-back.")
        del self.entries[key]

def E10():
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    cache = DirtyAwareCache()
    cpu = torch.zeros((1024,), pin_memory=True)
    cache.entries["x"] = CacheEntry(cpu.to("cuda:0"), cpu)
    cache.entries["x"].gpu_tensor.add_(5.0)
    cache.mark_dirty("x")
    return cache
```

Part 6 - Measurement, Validation, and Failure Analysis

Mutation APIs must consistently mark dirty state; missing one creates silent loss. Views sharing storage need shared dirty ownership rather than independent flags.

Part 7 - Engineering Interpretation and Philosophy

Capacity management never outranks data integrity. Dirty state identifies the authoritative value and prevents its accidental loss.

19.12.12 Exercise 11

Exercise Title: Implement Write-Back Eviction That Copies Modified CUDA Tensors to CPU Before Releasing Their GPU Storage

Part 1 - Exercise Contract and Why It Matters

Implement synchronous write-back as the correctness baseline: persist dirty CUDA values to CPU, validate completion, then release GPU ownership.

Part 2 - System and Memory Mental Model

Write-back has requested, complete, and committed states. GPU storage cannot be released before D2H completion, and dirty cannot become false before the CPU backing value is authoritative.

Part 3 - Quantitative Prediction Before Execution

Write-back bytes equal tensor storage bytes. Predict latency from D2H bandwidth and report it separately from victim-selection overhead.

Part 4 - Experimental Design and Instrumentation

Mutate known values, time completed D2H, compare CPU/GPU checksums, then delete the entry. Inject an exception before commit and confirm it remains dirty.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

def write_back_and_evict(cache, key):
    entry = cache.entries[key]
    if entry.protected:
        raise RuntimeError("Protected entry cannot be evicted.")
    started = time.perf_counter()
    bytes_written = 0
    if entry.dirty:
        entry.cpu_tensor.copy_(entry.gpu_tensor, non_blocking=False)
        bytes_written = (
            entry.gpu_tensor.numel() * entry.gpu_tensor.element_size())
        entry.dirty = False
    del cache.entries[key]
    elapsed_ms = (time.perf_counter() - started) * 1000.0
```

```
    return {"bytes_written": bytes_written,
            "writeback_and_evict_ms": elapsed_ms}

def E11(cache, key):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    return write_back_and_evict(cache, key)
```

Part 6 - Measurement, Validation, and Failure Analysis

Blocking write-back stalls the caller but gives a clear reference result. If CPU backing dtype or shape differs, define conversion semantics explicitly. External aliases to the GPU tensor can prevent real reclamation.

Part 7 - Engineering Interpretation and Philosophy

Establish the simple correct state machine before adding concurrency. An asynchronous optimization must preserve the same commit boundary.

19.12.13 Exercise 12

Exercise Title: Perform Asynchronous Eviction to Pinned CPU Memory on a Dedicated CUDA Stream Without Blocking Active Computation

Part 1 - Exercise Contract and Why It Matters

Move dirty data to pinned CPU storage on an eviction stream while unrelated compute continues, and expose completion as an event.

Part 2 - System and Memory Mental Model

Asynchronous D2H requires pinned destination memory and a side stream. The entry remains resident and dirty while copying. After its event completes, CPU data becomes authoritative and GPU ownership may be released.

Part 3 - Quantitative Prediction Before Execution

Ideal overlap hides at most $\min(T_{D2H}, T_{compute})$. During write-back, both CPU and GPU copies coexist, increasing temporary memory.

Part 4 - Experimental Design and Instrumentation

Record an event after D2H, run independent compute, poll completion, validate CPU data, and compare wall time with blocking eviction. Keep the source alive.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class PendingEviction:
    def __init__(self, key, entry, event, stream):
        self.key = key
        self.entry = entry
        self.event = event
```

```
        self.stream = stream

def begin_async_writeback(cache, key, eviction_stream):
    entry = cache.entries[key]
    if entry.protected:
        raise RuntimeError("Protected entry cannot be evicted.")
    event = torch.cuda.Event()
    with torch.cuda.stream(eviction_stream):
        if entry.dirty:
            if not entry.cpu_tensor.is_pinned():
                raise RuntimeError("Async D2H destination must be pinned.")
            entry.cpu_tensor.copy_(entry.gpu_tensor, non_blocking=True)
        event.record(eviction_stream)
    return PendingEviction(key, entry, event, eviction_stream)

def finish_async_writeback(cache, pending):
    pending.event.synchronize()
    if pending.entry.dirty:
        pending.entry.dirty = False
    del cache.entries[pending.key]

def E12(cache, key):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    stream = torch.cuda.Stream(device="cuda:0")
    return begin_async_writeback(cache, key, stream)
```

Part 6 - Measurement, Validation, and Failure Analysis

Reading CPU backing data before event completion is invalid. Pending bytes remain counted until finish. A queue must handle copy errors without losing the dirty GPU authority.

Part 7 - Engineering Interpretation and Philosophy

Asynchronous eviction delays reclamation to preserve concurrency. Accounting must follow physical lifetime, not the moment a request was submitted.

19.12.14 Exercise 13

Exercise Title: Coordinate Eviction with CUDA Events to Prevent a Tensor from Leaving VRAM While Another Stream Is Still Using It

Part 1 - Exercise Contract and Why It Matters

Prevent a victim from leaving VRAM until every registered consumer stream has finished using its storage.

Part 2 - System and Memory Mental Model

Each consumer records a completion event after final use. Eviction waits for all such events before write-back or deletion. Recording the tensor on consumer streams also protects allocator reuse when references disappear.

Part 3 - Quantitative Prediction Before Execution

Victim eligibility time is the latest completion among active consumers. Measure eviction wait separately from D2H time.

Part 4 - Experimental Design and Instrumentation

Run readers on two streams with sentinels, request eviction early, wait only on their events, and compare results with a no-eviction reference.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class UsageTracker:
    def __init__(self):
        self.events = {}

    def record_use(self, key, tensor, stream):
        tensor.record_stream(stream)
        event = torch.cuda.Event()
        event.record(stream)
        if key not in self.events:
            self.events[key] = []
        self.events[key].append(event)

    def wait_until_idle(self, key, eviction_stream):
        for event in self.events.get(key, []):
            eviction_stream.wait_event(event)
        self.events[key] = []

def E13(tensor):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required.")
    tracker = UsageTracker()
    first = torch.cuda.Stream()
    second = torch.cuda.Stream()
    with torch.cuda.stream(first):
        output_a = tensor.square()
        tracker.record_use("x", tensor, first)
    with torch.cuda.stream(second):
        output_b = tensor.abs()
        tracker.record_use("x", tensor, second)
    eviction = torch.cuda.Stream()
    tracker.wait_until_idle("x", eviction)
    eviction.synchronize()
    return float(output_a.mean() + output_b.mean())
```

Part 6 - Measurement, Validation, and Failure Analysis

Record completion after final use, not before. Unregistered consumers make the protocol unsound. Wait only on victim events rather than synchronizing the entire device.

Part 7 - Engineering Interpretation and Philosophy

Residency is shared ownership across streams. Eviction becomes legal only after every owner relinquishes use through an explicit completion edge.

19.12.15 Exercise 14

Exercise Title: Add Protected and Non-Evictable Tensor States for Parameters, Active Inputs, Temporary Buffers, and In-Flight Transfers

Part 1 - Exercise Contract and Why It Matters

Represent temporary and permanent non-evictable states and ensure victim selection never violates them.

Part 2 - System and Memory Mental Model

Permanent protection covers required parameters; temporary protection covers active inputs, workspaces, and in-flight transfers. Reference counts ensure overlapping users cannot unprotect each other.

Part 3 - Quantitative Prediction Before Execution

Evictable bytes equal resident bytes minus protected and pending bytes. Reject admission when legal victims cannot reclaim enough.

Part 4 - Experimental Design and Instrumentation

Test nested protection, double release, exception cleanup, and all-protected capacity failure. Log protection reasons and owners.

Part 5 - PyTorch/CUDA Implementation

```
class ProtectionRegistry:
    def __init__(self):
        self.counts = {}
        self.reasons = {}

    def protect(self, key, reason):
        self.counts[key] = self.counts.get(key, 0) + 1
        if key not in self.reasons:
            self.reasons[key] = []
        self.reasons[key].append(reason)

    def release(self, key, reason):
        if self.counts.get(key, 0) <= 0:
            raise RuntimeError("Protection release without ownership.")
        self.counts[key] -= 1
        self.reasons[key].remove(reason)

    def is_evictable(self, key):
```

```
        return self.counts.get(key, 0) == 0

def E14(keys):
    registry = ProtectionRegistry()
    for key in keys:
        registry.protect(key, "active")
    return registry
```

Part 6 - Measurement, Validation, and Failure Analysis

Use context managers so exceptions release temporary protection. Report insufficient evictable memory rather than silently selecting a protected victim.

Part 7 - Engineering Interpretation and Philosophy

Protection expresses correctness above policy preference. A score ranks only legal candidates; it never makes an illegal candidate valid.

19.12.16 Exercise 15

Exercise Title: Build a Watermark-Based Eviction Controller with High and Low VRAM Thresholds to Prevent Last-Second OOM Recovery

Part 1 - Exercise Contract and Why It Matters

Evict proactively when usage crosses a high watermark and continue until a low watermark restores headroom.

Part 2 - System and Memory Mental Model

Two thresholds create hysteresis. High watermark triggers reclamation; low watermark stops it. A single threshold can oscillate on every allocation. Pending write-backs remain counted until completion.

Part 3 - Quantitative Prediction Before Execution

For capacity B, choose $H = hB$ and $L = lB$ with $l < h$. On trigger, reclaim at least $U - L$ bytes plus admission size if known.

Part 4 - Experimental Design and Instrumentation

Feed bursty allocations, log threshold crossings, selected bytes, pending bytes, eviction cycles, headroom, and any emergency allocation retries.

Part 5 - PyTorch/CUDA Implementation

```
class WatermarkController:
    def __init__(self, budget_bytes, low_ratio=0.70, high_ratio=0.85):
        if not 0.0 < low_ratio < high_ratio < 1.0:
            raise ValueError("Watermark ratios are invalid.")
        self.budget_bytes = budget_bytes
        self.low_bytes = int(budget_bytes * low_ratio)
        self.high_bytes = int(budget_bytes * high_ratio)
        self.eviction_active = False
```

```
    def update(self, used_bytes):
        if not self.eviction_active and used_bytes >= self.high_bytes:
            self.eviction_active = True
        if self.eviction_active and used_bytes <= self.low_bytes:
            self.eviction_active = False
        reclaim_target = 0
        if self.eviction_active:
            reclaim_target = max(0, used_bytes - self.low_bytes)
        return {"eviction_active": self.eviction_active,
                "reclaim_target_bytes": reclaim_target}

def E15(budget_MiB=4096):
    return WatermarkController(int(budget_MiB * 1024 ** 2))
```

Part 6 - Measurement, Validation, and Failure Analysis

Watermarks use the same memory definition consistently: logical cache bytes or measured allocated bytes. Reserved memory can stay high after safe eviction and must not be mistaken for live residency.

Part 7 - Engineering Interpretation and Philosophy

Headroom is planned resilience. Proactive reclamation is cheaper and more predictable than selecting victims while an allocation is already failing.

19.12.17 Exercise 16

Exercise Title: Combine Prefetching and Eviction and Detect Thrashing When Tensors Are Repeatedly Loaded and Removed from VRAM

Part 1 - Exercise Contract and Why It Matters

Run prefetch and eviction together and identify tensors loaded speculatively, evicted before useful reuse, then loaded again.

Part 2 - System and Memory Mental Model

Prefetch increases future residency while eviction reduces it. Without one coordinator they can fight. Thrashing converts PCIe bandwidth and CPU memory into repeated movement without proportional GPU work.

Part 3 - Quantitative Prediction Before Execution

Track load and eviction counts per key. Define churn bytes as reload plus write-back traffic and useful bytes as bytes consumed by GPU operations.

Part 4 - Experimental Design and Instrumentation

Use a working set slightly above capacity, vary prefetch distance, and log load-evict-reload cycles, residence time, use count, transfer bytes, and compute time.

Part 5 - PyTorch/CUDA Implementation

```
class ChurnTracker:
    def __init__(self):
        self.loads = {}
        self.evictions = {}
        self.loaded_bytes = 0
        self.evicted_bytes = 0
        self.useful_bytes = 0

    def record_load(self, key, size_bytes):
        self.loads[key] = self.loads.get(key, 0) + 1
        self.loaded_bytes += size_bytes

    def record_evict(self, key, size_bytes):
        self.evictions[key] = self.evictions.get(key, 0) + 1
        self.evicted_bytes += size_bytes

    def record_use(self, size_bytes):
        self.useful_bytes += size_bytes

    def thrash_ratio(self):
        traffic = self.loaded_bytes + self.evicted_bytes
        return traffic / max(self.useful_bytes, 1)

def E16():
    return ChurnTracker()
```

Part 6 - Measurement, Validation, and Failure Analysis

Repeated loads can be legitimate if compute benefit is large. Diagnose thrashing from traffic relative to useful work and short residence, not from miss count alone.

Part 7 - Engineering Interpretation and Philosophy

Speculation and reclamation need one economic model. Work loaded with no credible consumption deadline should not displace known useful residency.

19.12.18 Exercise 17

Exercise Title: Implement Thrashing Detection and Dynamically Change the Eviction Policy When Transfer Traffic Exceeds Useful GPU Work

Part 1 - Exercise Contract and Why It Matters

Detect sustained churn and switch to a safer policy or reduce prefetch distance without oscillating on transient bursts.

Part 2 - System and Memory Mental Model

A detector uses a moving window of transferred bytes, useful compute bytes or time, rapid reloads, and hit rate. Hysteresis and cooldown prevent repeated policy changes.

Part 3 - Quantitative Prediction Before Execution

Declare thrashing when traffic/useful ratio exceeds a high threshold for several windows; recover only below a lower threshold. Predict switch points on a controlled trace.

Part 4 - Experimental Design and Instrumentation

Create stable, bursty, and sustained-thrash phases. Log ratios, consecutive bad windows, chosen policy, lookahead depth, switch count, and recovered throughput.

Part 5 - PyTorch/CUDA Implementation

```
class ThrashingController:
    def __init__(self, high_ratio=2.0, low_ratio=1.2,
                 required_windows=3):
        self.high_ratio = high_ratio
        self.low_ratio = low_ratio
        self.required_windows = required_windows
        self.bad_windows = 0
        self.mode = "performance"

    def observe(self, transfer_bytes, useful_bytes):
        ratio = transfer_bytes / max(useful_bytes, 1)
        if ratio >= self.high_ratio:
            self.bad_windows += 1
        else:
            self.bad_windows = 0
        if self.mode == "performance":
            if self.bad_windows >= self.required_windows:
                self.mode = "thrash_safe"
        elif ratio <= self.low_ratio:
            self.mode = "performance"
            self.bad_windows = 0
        return {"ratio": ratio, "mode": self.mode,
                "bad_windows": self.bad_windows}

def E17():
    return ThrashingController()
```

Part 6 - Measurement, Validation, and Failure Analysis

Useful bytes are a proxy; kernel time or application progress may be better. Policy changes must preserve resident dirty state and queue order. Apply them at safe boundaries.

Part 7 - Engineering Interpretation and Philosophy

Adaptive systems need a stable definition of harm and a reversible response. Detection without controlled action merely adds telemetry.

19.12.19 Exercise 18

Exercise Title: Build a Hybrid Eviction Score from Recency, Frequency, Tensor Size, Reload Cost, and Predicted Reuse and Tune Its Weights Experimentally

Part 1 - Exercise Contract and Why It Matters

Combine multiple normalized signals into one explainable victim score and tune weights on held-out traces.

Part 2 - System and Memory Mental Model

Raw signals have incompatible scales. Normalize recency, inverse frequency, size benefit, reload penalty, and predicted reuse before weighting. Protected or in-flight entries are excluded before scoring.

Part 3 - Quantitative Prediction Before Execution

Define higher score as more evictable:

$$S = w_r R + w_f F + w_s M - w_c C + w_p P.$$

Record each component so every victim decision can be defended.

Part 4 - Experimental Design and Instrumentation

Tune weights on sequential, hot-set, variable-size, and expensive-reload traces. Evaluate on separate traces using stall time, traffic, hit rate, and peak memory.

Part 5 - PyTorch/CUDA Implementation

```
def normalize(value, minimum, maximum):
    if maximum <= minimum:
        return 0.0
    return (value - minimum) / (maximum - minimum)

def hybrid_score(item, ranges, weights):
    recency = normalize(item["age"], *ranges["age"])
    frequency = 1.0 - normalize(
        item["frequency"], *ranges["frequency"])
    size = normalize(item["size_bytes"], *ranges["size_bytes"])
    reload_cost = normalize(
        item["reload_ms"], *ranges["reload_ms"])
    reuse_far = normalize(
        item["predicted_reuse"], *ranges["predicted_reuse"])
    score = (
        weights["recency"] * recency
        + weights["frequency"] * frequency
        + weights["size"] * size
        - weights["reload_cost"] * reload_cost
        + weights["predicted_reuse"] * reuse_far
    )
    return score

def E18(metadata, ranges, weights):
    scores = {}
    for key, item in metadata.items():
        if not item["protected"] and not item["in_flight"]:
```

```
            scores[key] = hybrid_score(item, ranges, weights)
    return sorted(scores.items(), key=lambda pair: pair[1],
                  reverse=True)
```

Part 6 - Measurement, Validation, and Failure Analysis

Avoid tuning and evaluation on the same trace. Weight signs must match score meaning. A score ranks candidates but the admission loop still evicts enough bytes and respects dirty write-back.

Part 7 - Engineering Interpretation and Philosophy

Hybrid policies are useful when complexity buys robustness across workloads. Explainable components and held-out evaluation prevent a score from becoming an opaque collection of preferences.

19.12.20 Exercise 19

Exercise Title: Build and Benchmark a Memory-Aware PyTorch Eviction Engine That Keeps a Variable Working Set Inside a Strict VRAM Budget Without OOM

Part 1 - Exercise Contract and Why It Matters

Integrate byte admission, legal-victim filtering, dirty write-back, stream-safe completion, watermarks, policy scoring, and thrash telemetry into one bounded engine.

Part 2 - System and Memory Mental Model

Entries move through loading, resident-clean, resident-dirty, protected, write-back-pending, and evicted states. Logical bytes include resident and pending entries until completion. The engine never depends on catching OOM as a normal capacity decision.

Part 3 - Quantitative Prediction Before Execution

For every admission, prove current owned bytes plus requested bytes fit after legal completed evictions. Report hit/miss rate, stall time, traffic, dirty bytes, thrash ratio, peak allocated/reserved VRAM, and safety headroom.

Part 4 - Experimental Design and Instrumentation

Run fixed FIFO, LRU, hybrid, and adaptive modes on identical variable-size traces. Validate every value against CPU backing truth, include phase changes, and test cancellation and all-protected failure.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class EngineEntry:
    tensor: object
    cpu_backing: object
    size_bytes: int
    dirty: bool = False
```

```
    protected: int = 0
    pending: bool = False
    last_access: int = 0
    frequency: int = 0

class EvictionEngine:
    def __init__(self, budget_bytes, low_ratio=0.70,
                 high_ratio=0.85):
        self.budget_bytes = budget_bytes
        self.low_bytes = int(budget_bytes * low_ratio)
        self.high_bytes = int(budget_bytes * high_ratio)
        self.entries = {}
        self.owned_bytes = 0
        self.step = 0

    def legal_candidates(self):
        candidates = []
        for key, entry in self.entries.items():
            if entry.protected == 0 and not entry.pending:
                candidates.append((key, entry))
        return candidates

    def choose_lru(self):
        candidates = self.legal_candidates()
        if len(candidates) == 0:
            raise RuntimeError("No legal eviction candidate.")
        return min(candidates, key=lambda pair: pair[1].last_access)[0]

    def reclaim_for(self, requested_bytes, evict_callback):
        if requested_bytes > self.budget_bytes:
            raise ValueError("Requested entry exceeds the complete budget.")
        admission_limit = self.budget_bytes - requested_bytes
        if self.owned_bytes >= self.high_bytes:
            target_bytes = min(self.low_bytes, admission_limit)
        else:
            target_bytes = admission_limit
        while self.owned_bytes > target_bytes:
            victim = self.choose_lru()
            released_bytes = evict_callback(victim, self.entries[victim])
            if released_bytes <= 0:
                raise RuntimeError("Eviction made no capacity progress.")
            self.owned_bytes -= released_bytes
            del self.entries[victim]

    def register(self, key, entry):
        if key in self.entries:
            raise KeyError("Key is already resident.")
        if self.owned_bytes + entry.size_bytes > self.budget_bytes:
```

```
            raise RuntimeError("Admission invariant was not satisfied.")
        self.step += 1
        entry.last_access = self.step
        entry.frequency = 1
        self.entries[key] = entry
        self.owned_bytes += entry.size_bytes

def E19(budget_MiB=4096):
    return EvictionEngine(int(budget_MiB * 1024 ** 2))
```

Part 6 - Measurement, Validation, and Failure Analysis

The callback must complete or correctly retain pending write-back before reporting released bytes. Production admission also reserves model headroom and compares logical accounting with PyTorch and device-level counters. A faster engine is rejected if it loses dirty data or violates stream ownership.

Part 7 - Engineering Interpretation and Philosophy

The final objective is not maximum cache occupancy. It is predictable useful work inside a strict budget, with every victim legal, every dirty value durable, every asynchronous lifetime explicit, and every policy decision measurable.

19.13 DMM11

Chapter Title: Dynamic Tensor Placement
Specialization: Dynamic Memory Management and CUDA Systems

19.13.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Tensor Residency Across CPU RAM, GPU VRAM, and Transfer Boundaries

Part 1 - Exercise Contract and Why It Matters

Build a state model for where a tensor's authoritative value resides, which copies exist, and when a migration becomes complete. Placement is a correctness decision before it is a performance decision.

The required outcome is not a diagram with device names. It is an executable contract that answers five questions at every instant: which copy contains the newest value, which replicas are valid, which transfer is pending, which streams may still touch the storage, and which transition is legal next. This contract becomes the foundation for every later promotion, demotion, and budget decision.

Part 2 - System and Memory Mental Model

A logical tensor may have pageable CPU, pinned CPU, and CUDA copies. States include CPU-only, H2D-pending, GPU-resident, both-clean, GPU-dirty, D2H-pending, and released. A transfer request does not instantly change authority. Events define completion; stream usage and autograd references define lifetime.

Residency and authority must remain separate. A tensor can be resident on both tiers while only one copy is authoritative after a GPU update. Likewise, "H2D-pending" means CUDA storage exists but is not yet consumable without an event dependency. Python object lifetime, storage lifetime, logical-value lifetime, and asynchronous-operation lifetime are four different concepts.

Views add a second axis: several tensor objects may share one storage allocation. Moving or releasing one logical view cannot be accounted as if it owned unique bytes. Autograd adds hidden future consumers through saved tensors. The state model must therefore attach placement to storage identity while preserving the shape and stride metadata of each logical tensor.

Part 3 - Quantitative Prediction Before Execution

For tensor bytes M, H2D/D2H bandwidths B_h, B_d, and fixed overhead L, estimate migration time as $L + M/B$. Placement value depends on expected reuse count times avoided transfer cost minus VRAM residence opportunity cost.

For an H2D promotion followed by r GPU uses, compute a break-even reuse count from measured transfer time and per-use acceleration. Also predict the peak during migration: for a period both the source and destination exist, and a dirty D2H demotion may require an additional pinned destination before CUDA storage can be released. A correct budget includes pending bytes, not only committed residents.

Part 4 - Experimental Design and Instrumentation

Track key, shape, dtype, bytes, authority, replicas, dirty state, transfer event, consumer streams, use count, last use, and resident duration. Validate values at every completed state transition.

Exercise the model with clean promotion, GPU mutation, dirty write-back, multi-stream readers, aliased views, and an intentionally delayed event. At each boundary capture a state record and verify a checksum after completion. Negative tests should attempt early CPU reads, early GPU consumption, and premature release so the model demonstrates how it prevents each race.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class PlacementState:
    key: str
    authority: str
    cpu_present: bool
    gpu_present: bool
    dirty: bool
    transfer_pending: bool
    size_bytes: int

def E00(key, shape, element_size):
    elements = 1
    for dimension in shape:
        elements *= dimension
```

```
    size_bytes = elements * element_size
    return PlacementState(
        key=key,
        authority="cpu",
        cpu_present=True,
        gpu_present=False,
        dirty=False,
        transfer_pending=False,
        size_bytes=size_bytes,
    )
```

Part 6 - Measurement, Validation, and Failure Analysis

Device labels alone do not reveal authority or aliases. Views may share storage, autograd may retain tensors, and reserved VRAM may remain after a CUDA copy is logically removed. State transitions must follow physical completion.

Part 7 - Engineering Interpretation and Philosophy

Dynamic placement is controlled replication across memory tiers. A useful engine can answer where the newest value is, who may use it, and what event makes the next transition legal.

19.13.2 Exercise 01

Exercise Title: Inspect the Device, Size, Dtype, and Memory Footprint of Every Tensor in a Mixed CPU-GPU PyTorch Workload

Part 1 - Exercise Contract and Why It Matters

Build an inventory of tensors before designing placement policy. Report storage facts without accidentally holding additional autograd references.

The inventory must be useful for policy and safe for a long-running process. Store names, scalar dimensions, byte counts, and storage identifiers in the report, but do not store the tensors themselves. The final report must separate logical tensor bytes from unique physical storage bytes and show totals per memory tier.

Part 2 - System and Memory Mental Model

Logical tensor bytes are `numel()*element_size()`, but views can share storage and make naive sums double-count. Device, dtype, shape, stride, gradient status, pinned state, and storage identity affect migration.

A non-contiguous view may expose fewer logical elements than its underlying storage, and two differently shaped tensors may refer to exactly the same bytes. Pinned CPU memory is physically a host allocation yet has different transfer and operating-system consequences from pageable RAM. CUDA allocated bytes also exclude allocator cache, context memory, and allocations made outside PyTorch.

Part 3 - Quantitative Prediction Before Execution

Predict bytes by shape and dtype, then compare with observed tensor bytes and unique-storage totals. Separate CPU, pinned CPU, and each CUDA device.

For each alias group, predict both the naive sum and unique-storage sum. Their difference is the double-counting error a placement engine would make if it budgeted tensor objects

independently. Reconcile CUDA unique storage with `memory_allocated()` while expecting known differences from allocator rounding and unrelated live tensors.

Part 4 - Experimental Design and Instrumentation

Traverse explicit containers, record scalar metadata only, and identify aliases through storage pointers. Compare inventory with CUDA allocated/reserved counters.

Include contiguous tensors, slices, transposes, a detached alias, parameters, gradients, and pinned batches. Take inventory before forward, after forward, after backward, and after gradient reset. The changing report reveals which objects are persistent and which exist only during a training phase.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def describe_tensor(name, tensor):
    storage_pointer = tensor.untyped_storage().data_ptr()
    bytes_logical = tensor.numel() * tensor.element_size()
    pinned = tensor.is_pinned() if tensor.device.type == "cpu" else False
    return {"name": name, "shape": tuple(tensor.shape),
            "dtype": str(tensor.dtype), "device": str(tensor.device),
            "logical_bytes": bytes_logical,
            "storage_pointer": storage_pointer,
            "requires_grad": tensor.requires_grad,
            "is_leaf": tensor.is_leaf, "is_pinned": pinned}

def E01(named_tensors):
    report = []
    unique_storage = {}
    for name, tensor in named_tensors.items():
        item = describe_tensor(name, tensor)
        report.append(item)
        pointer = item["storage_pointer"]
        if pointer not in unique_storage:
            unique_storage[pointer] = tensor.untyped_storage().nbytes()
    return {"tensors": report,
            "unique_storage_bytes": sum(unique_storage.values())}
```

Part 6 - Measurement, Validation, and Failure Analysis

Storage pointers are diagnostic identities within a process, not persistent keys. Sparse, quantized, and nested tensors need specialized accounting. Do not store tensor objects inside long-lived telemetry.

Part 7 - Engineering Interpretation and Philosophy

Placement decisions are only as good as the inventory beneath them. Measure physical storage and logical use separately.

19.13.3 Exercise 02

Exercise Title: Build a Tensor Placement Function That Moves Inputs to CPU or CUDA from Explicit Runtime Rules

Part 1 - Exercise Contract and Why It Matters

Implement a deterministic rule interface that chooses CPU or CUDA and performs the move without silently changing dtype, shape, or values.

The function is deliberately split into a pure decision and an effectful mover. This makes rules testable on a CPU-only machine and makes every real transfer carry a reason, byte count, completion contract, and validation record.

Part 2 - System and Memory Mental Model

Decision and execution are separate. The policy returns a target and reason; the mover handles pinning, non-blocking eligibility, completion, and validation. Calling `to()` on an already placed tensor may return the same object.

An H2D request from pinned memory can be submitted asynchronously, but the returned CUDA tensor is a pending destination until its producer stream reaches the copy. A D2H request is even easier to misuse because Python can immediately inspect the host object. Therefore a production return type should include an event or future rather than presenting the destination as an ordinary completed value.

Part 3 - Quantitative Prediction Before Execution

For each input, predict target device, bytes transferred, and whether a copy is required. Sum traffic before execution.

Estimate completed migration time with measured fixed latency and bandwidth. Predict zero bytes for same-device identity placement, full logical bytes for a copy, and an error for a target that violates an operation's device constraint. Do not let a low-level mover silently override the policy to recover from OOM.

Part 4 - Experimental Design and Instrumentation

Test CPU-to-CPU, GPU-to-GPU, H2D, and D2H paths. Verify shape, dtype, values, and completion semantics under explicit rules.

Add rules that conflict intentionally and define priority: forced CPU, mandatory CUDA operation, hard-fit budget, then optimization preference. Test contiguous and non-contiguous inputs and verify whether placement preserves layout or materializes a copy. Log CPU submission and completed CUDA time separately.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def choose_target(metadata, rules):
    if metadata["force_cpu"]:
        return "cpu", "forced_cpu"
    if metadata["requires_cuda"]:
        return "cuda:0", "required_by_operation"
    if metadata["size_bytes"] <= rules["gpu_size_limit"]:
        return "cuda:0", "small_tensor"
```

```
    return "cpu", "large_tensor"

def place_tensor(tensor, metadata, rules):
    target, reason = choose_target(metadata, rules)
    if str(tensor.device) == target:
        return tensor, {"target": target, "reason": reason,
                        "copied": False}
    non_blocking = (
        tensor.device.type == "cpu"
        and tensor.is_pinned()
        and target.startswith("cuda"))
    moved = tensor.to(target, non_blocking=non_blocking)
    return moved, {"target": target, "reason": reason,
                   "copied": True}

def E02(tensor, metadata, rules):
    return place_tensor(tensor, metadata, rules)
```

Part 6 - Measurement, Validation, and Failure Analysis

Non-blocking return does not mean completed migration. The caller must wait before consumption and preserve source lifetime. D2H results must not be read before completion.

The simple function returns only a tensor, so its asynchronous path requires a documented caller discipline. E15 replaces this with a transaction object that retains source and destination. Also validate that metadata byte counts agree with the actual tensor; stale metadata can defeat the hard budget.

Part 7 - Engineering Interpretation and Philosophy

Explicit rules make placement auditable. Hidden device movement inside arbitrary operations makes both performance and correctness harder to reason about.

19.13.4 Exercise 03

Exercise Title: Design a Size-Based Placement Policy That Keeps Small Tensors on GPU and Large Low-Reuse Tensors on CPU

Part 1 - Exercise Contract and Why It Matters

Use byte thresholds and reuse qualification to avoid allowing large, rarely used tensors to consume scarce VRAM.

The policy is a baseline, not a final answer. It must be deterministic, use byte capacity rather than tensor count, and emit a reason for every choice so later experiments can identify exactly where a simple threshold fails.

Part 2 - System and Memory Mental Model

Small tensors have limited capacity cost but may suffer launch/transfer overhead. Large tensors deserve VRAM only when reuse or compute benefit repays residence. Policy uses bytes, not dimensions or tensor count.

Capacity cost includes the resident tensor and any temporary duplicate during migration. Small tensors can be cheap to keep yet expensive to transfer one by one because fixed copy

latency dominates. A large tensor can still be the best resident if it is reused frequently or blocks a critical computation whenever reloaded.

Part 3 - Quantitative Prediction Before Execution

Place on GPU when $M \leq S$, or when large-tensor expected reuse exceeds a threshold and budget permits. Predict total resident and transferred bytes.

Evaluate the boundary at $S - 1$, S, and $S + 1$ bytes. For large tensors, estimate avoided reload time from predicted reuse and compare it with the value of VRAM occupied over that interval. Reserve headroom before declaring that a candidate fits; observed free VRAM is not wholly available to the policy.

Part 4 - Experimental Design and Instrumentation

Sweep size around the threshold and compare low/high reuse. Report decisions, hit time, migration time, VRAM, and threshold sensitivity.

Use fixed-size stable workloads and variable-shape workloads. Report decision stability, transfer count, hit rate, completed latency, managed peak, and policy regret. A sweep should reveal whether one threshold generalizes or merely fits a single trace.

Part 5 - PyTorch/CUDA Implementation

```
class SizePlacementPolicy:
    def __init__(self, small_limit_bytes, large_reuse_threshold):
        self.small_limit_bytes = small_limit_bytes
        self.large_reuse_threshold = large_reuse_threshold

    def decide(self, size_bytes, predicted_reuse, available_vram):
        if size_bytes > available_vram:
            return "cpu", "does_not_fit"
        if size_bytes <= self.small_limit_bytes:
            return "cuda:0", "small"
        if predicted_reuse >= self.large_reuse_threshold:
            return "cuda:0", "large_but_reused"
        return "cpu", "large_low_reuse"

def E03(size_limit_MiB=16, reuse_threshold=4):
    return SizePlacementPolicy(
        int(size_limit_MiB * 1024 ** 2), reuse_threshold)
```

Part 6 - Measurement, Validation, and Failure Analysis

A hard size boundary can oscillate with variable shapes and ignores reload cost. Use hysteresis and workload evidence before production.

The policy also ignores dirty write-back, phase change, transfer overlap, and competition between several candidates that individually fit but collectively do not. Treat these omissions as measured limitations, not implementation details to hide.

Part 7 - Engineering Interpretation and Philosophy

Size is a capacity signal, not a measure of value. It is useful as one gate in a larger decision, not as the complete policy.

19.13.5 Exercise 04

Exercise Title: Design a Reuse-Based Placement Policy That Keeps Frequently Accessed Tensors Resident in VRAM

Part 1 - Exercise Contract and Why It Matters

Track scalar access history and promote keys whose expected future reuse justifies sustained VRAM residency.

The implementation must distinguish old popularity from current demand. It should explain when a tensor becomes hot, retain only scalar telemetry, and measure whether each promotion eventually avoids enough reloads to repay itself.

Part 2 - System and Memory Mental Model

Frequency captures popularity; recency detects current phase; reuse distance estimates return interval. Metadata stores keys and numbers, never tensor or autograd references.

A cumulative count without decay makes yesterday's hot tensor immortal. A recency-only policy overreacts to one burst. Evaluate exponentially decayed frequency, windowed counts, and reuse distance, then choose the simplest signal whose prediction error is acceptable for the workload.

Part 3 - Quantitative Prediction Before Execution

Promote after a hot threshold and demote only below a colder threshold. Predict promotions, avoided reloads, and residence bytes from a trace.

Part 4 - Experimental Design and Instrumentation

Test stable-hot, one-time burst, and phase-changing keys. Log counts, last use, promotions, demotions, transfer traffic, and hit rate.

Split traces into calibration and evaluation periods. Record time-to-promotion, useful hits after promotion, residence duration, wasted promotion bytes, and misses caused by delayed recognition. Compare with an oracle that sees the future and report policy regret by workload phase.

Part 5 - PyTorch/CUDA Implementation

```
class ReusePlacementPolicy:
    def __init__(self, promote_count=4, cold_steps=20):
        self.promote_count = promote_count
        self.cold_steps = cold_steps
        self.step = 0
        self.counts = {}
        self.last_step = {}

    def observe(self, key):
        self.step += 1
        self.counts[key] = self.counts.get(key, 0) + 1
        self.last_step[key] = self.step

    def decide(self, key):
        count = self.counts.get(key, 0)
```

```
        age = self.step - self.last_step.get(key, 0)
        if count >= self.promote_count and age <= self.cold_steps:
            return "cuda:0", "hot"
        return "cpu", "not_hot"

def E04():
    return ReusePlacementPolicy()
```

Part 6 - Measurement, Validation, and Failure Analysis

Counts need aging or old popularity persists. A newly hot key needs exploration before history exists. Reuse policy still obeys hard VRAM capacity.

Part 7 - Engineering Interpretation and Philosophy

Residency should follow useful demand, but historical demand is only a forecast. Measure regret when promotions or demotions are wrong.

Hotness is not an intrinsic property of a tensor; it is a conditional forecast about a particular execution phase. A placement system should be willing to forget old evidence and should treat every resident tensor as a hypothesis that must continue earning scarce VRAM.

19.13.6 Exercise 05

Exercise Title: Compare Static Tensor Placement Against Dynamic Placement Under the Same VRAM Budget and Access Pattern

Part 1 - Exercise Contract and Why It Matters

Construct a controlled comparison in which static and dynamic placement receive the same tensor catalogue, access trace, CUDA computation, and strict VRAM budget. The experiment must determine whether adapting residency actually saves completed execution time rather than merely producing a higher cache-hit count.

Part 2 - System and Memory Mental Model

Static placement chooses one resident set before the trace begins. It pays no policy overhead and behaves predictably, but it cannot react when the workload changes phase. Dynamic placement may promote a newly hot tensor and evict a cold one, paying policy, transfer, and synchronization costs to do so. The fair unit of comparison is the entire completed trace, including initial population, misses, write-backs, and final synchronization.

Part 3 - Quantitative Prediction Before Execution

For each policy predict the number of H2D and D2H bytes, hits, misses, and evictions. A first-order time estimate is

$$T \approx T_{compute} + N_{H2D}L_{H2D} + \frac{B_{H2D}}{\beta_{H2D}} + N_{D2H}L_{D2H} + \frac{B_{D2H}}{\beta_{D2H}} + T_{policy}.$$

A dynamic policy wins only when avoided future reloads exceed its migration and decision overhead. Record this prediction before executing either policy.

Part 4 - Experimental Design and Instrumentation

Use three traces: stable popularity, a two-phase workload, and a random trace. Reset CUDA peak statistics and warm up kernels before every measured run. Log resident bytes, peak allocated bytes, hit rate, transferred bytes, eviction count, policy time, and synchronized wall time. Compare numerical outputs after the same ordered operations to ensure policy changes did not alter semantics.

Part 5 - PyTorch/CUDA Implementation

```
from collections import OrderedDict

class BudgetSimulator:
    def __init__(self, sizes, budget_bytes, dynamic):
        self.sizes = sizes
        self.budget_bytes = budget_bytes
        self.dynamic = dynamic
        self.resident = OrderedDict()
        self.used_bytes = 0
        self.hits = 0
        self.misses = 0
        self.h2d_bytes = 0
        self.evictions = 0

    def _remove_oldest(self):
        victim, unused = self.resident.popitem(last=False)
        self.used_bytes -= self.sizes[victim]
        self.evictions += 1

    def _admit(self, key):
        needed = self.sizes[key]
        while self.used_bytes + needed > self.budget_bytes:
            self._remove_oldest()
        self.resident[key] = True
        self.used_bytes += needed
        self.h2d_bytes += needed

    def access(self, key):
        if key in self.resident:
            self.hits += 1
            if self.dynamic:
                self.resident.move_to_end(key)
            return
        self.misses += 1
        if self.dynamic:
            self._admit(key)

    def initialize_static(self, keys):
        for key in keys:
            self._admit(key)
```

```
def E05(sizes, trace, budget_bytes, static_keys):
    static_policy = BudgetSimulator(sizes, budget_bytes, False)
    static_policy.initialize_static(static_keys)
    dynamic_policy = BudgetSimulator(sizes, budget_bytes, True)
    for key in trace:
        static_policy.access(key)
        dynamic_policy.access(key)
    return static_policy, dynamic_policy
```

Part 6 - Measurement, Validation, and Failure Analysis

The simulator isolates policy behavior; repeat the winning configurations with real pinned buffers, CUDA events, and compute. A high hit rate can still lose if it protects tiny tensors while repeatedly loading one expensive tensor. Do not exclude compulsory first loads, and do not time asynchronous copies without a completion event. Assert that resident bytes never exceed the stated budget.

Part 7 - Engineering Interpretation and Philosophy

Adaptivity is not intrinsically superior. Static placement is a strong baseline because certainty has value: no migration oscillation, minimal metadata, and stable latency. Dynamic placement earns its complexity only when measured phase changes create enough reuse value to pay for movement.

19.13.7 Exercise 06

Exercise Title: Measure the Performance Penalty of Moving a Tensor Between CPU and GPU More Often Than Its Reuse Justifies

Part 1 - Exercise Contract and Why It Matters

Measure the break-even point between keeping a tensor resident and repeatedly moving it across PCIe or another host-device interconnect. The exercise exposes placement thrashing as completed transfer latency, lost bandwidth, and GPU idle time rather than as an abstract policy mistake.

Part 2 - System and Memory Mental Model

Migration contains fixed launch latency, byte-dependent copy time, possible pageable-to-pinned staging, stream dependencies, and allocation overhead. A non-blocking call shortens CPU submission time but does not make the transfer free. If the next computation immediately waits for the tensor, the complete copy remains on the critical path.

Part 3 - Quantitative Prediction Before Execution

For r uses, ping-pong placement performs approximately r H2D and r D2H copies, whereas resident placement performs one H2D copy. Predict

$$\Delta T \approx (r-1)T_{H2D} + rT_{D2H} - C_{saved},$$

where C_{saved} is the opportunity value of the released VRAM. Sweep both bytes and reuse count because small transfers are latency-bound while large transfers approach bandwidth limits.

Part 4 - Experimental Design and Instrumentation

Allocate one reusable pinned host buffer and one CUDA buffer. Warm up the copy path, then compare keep-resident and ping-pong variants with identical kernels. Use CUDA events for device timeline duration and synchronized wall time for end-to-end cost. Record effective GB/s, CPU submission time, transfer count, GPU idle gaps, and output parity.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

def completed_ms(operation):
    start = torch.cuda.Event(enable_timing=True)
    stop = torch.cuda.Event(enable_timing=True)
    start.record()
    operation()
    stop.record()
    stop.synchronize()
    return start.elapsed_time(stop)

def E06(elements=8_000_000, repeats=20):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required")
    host = torch.randn(elements, dtype=torch.float32,
                       pin_memory=True)
    device = torch.empty_like(host, device="cuda")
    returned = torch.empty_like(host, pin_memory=True)

    device.copy_(host, non_blocking=True)
    torch.cuda.synchronize()

    def resident_work():
        for unused in range(repeats):
            device.add_(1.0)

    resident_ms = completed_ms(resident_work)

    def ping_pong_work():
        for unused in range(repeats):
            device.copy_(host, non_blocking=True)
            device.add_(1.0)
            returned.copy_(device, non_blocking=True)

    wall_start = time.perf_counter()
    ping_pong_ms = completed_ms(ping_pong_work)
    wall_ms = (time.perf_counter() - wall_start) * 1000.0
    expected = host + 1.0
    assert torch.equal(returned, expected)
    return resident_ms, ping_pong_ms, wall_ms
```

Part 6 - Measurement, Validation, and Failure Analysis

The destination of D2H is pinned and the event is synchronized before the CPU reads it. Removing that wait creates a correctness race. Avoid allocating new buffers inside the loop because the experiment would then mix allocator cost with migration cost. Report median and tail latency over several independent runs; the first run is a warm-up, not representative evidence.

Part 7 - Engineering Interpretation and Philosophy

Movement is justified by the future, not by the current pressure signal alone. Freeing VRAM for a moment can be irrational when a tensor will return before its reload cost is repaid. A placement engine therefore needs a notion of reuse horizon, not merely a reaction to low free memory.

19.13.8 Exercise 07

Exercise Title: Build a Cost Model That Estimates Whether Keeping a Tensor in VRAM Is Cheaper Than Repeated Host-to-Device Reloads

Part 1 - Exercise Contract and Why It Matters

Turn placement into an explicit economic decision. The model must compare the future cost of reloads with the opportunity cost of occupying scarce VRAM and must expose every assumption so that predictions can be calibrated against measurements.

Part 2 - System and Memory Mental Model

Residency consumes capacity over an interval; reloading consumes transfer-engine time, bus bandwidth, launch latency, and sometimes compute stall time. These costs are not interchangeable until expressed in a common objective such as predicted critical-path milliseconds. A tensor that reloads off the critical path is cheaper than one that blocks the compute stream.

Part 3 - Quantitative Prediction Before Execution

Define reload cost as expected reloads times measured H2D completion time times a critical-path fraction. Define residence cost as bytes times expected holding time times a calibrated VRAM shadow price. Keep the tensor when

$$N_{reload} T_{H2D} p_{critical} > M H \lambda_{VRAM}$$

and it fits after protected allocations. Predict the decision and sensitivity to bandwidth, reuse probability, and pressure before running the trace.

Part 4 - Experimental Design and Instrumentation

Calibrate fixed transfer latency and bandwidth using several sizes instead of one. Estimate reuse probability and reuse distance from a training prefix, then evaluate on a held-out trace. Log predicted savings, realized savings, false keeps, false evictions, bytes resident, and policy regret against an oracle with knowledge of the future trace.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class PlacementEstimate:
    reload_ms: float
    residence_ms: float
    predicted_saving_ms: float
    keep_on_gpu: bool
    reason: str

class PlacementCostModel:
    def __init__(self, bandwidth_bytes_per_ms, latency_ms,
                 vram_price_per_byte_ms):
        self.bandwidth = bandwidth_bytes_per_ms
        self.latency_ms = latency_ms
        self.vram_price = vram_price_per_byte_ms

    def decide(self, size_bytes, expected_reloads,
               holding_ms, critical_fraction, available_bytes):
        one_reload = self.latency_ms + size_bytes / self.bandwidth
        reload_ms = expected_reloads * one_reload * critical_fraction
        residence_ms = size_bytes * holding_ms * self.vram_price
        saving = reload_ms - residence_ms
        if size_bytes > available_bytes:
            return PlacementEstimate(reload_ms, residence_ms,
                                     saving, False, "does_not_fit")
        keep = saving > 0.0
        reason = "reloads_cost_more" if keep else "residency_costs_more"
        return PlacementEstimate(reload_ms, residence_ms,
                                 saving, keep, reason)

def E07():
    bytes_per_ms = 12.0 * 1000.0 * 1000.0
    model = PlacementCostModel(bytes_per_ms, 0.025, 1.0e-12)
    return model.decide(
        size_bytes=64 * 1024 * 1024,
        expected_reloads=4.0,
        holding_ms=100.0,
        critical_fraction=0.8,
        available_bytes=2 * 1024 * 1024 * 1024,
    )
```

Part 6 - Measurement, Validation, and Failure Analysis

The shadow price is a calibration parameter, not a universal constant. Update it as free VRAM approaches the low watermark. Predictions fail when bandwidth is measured without contention, reuse changes phase, or overlapping copies are treated as fully critical. Show prediction error instead of hiding it behind a binary accuracy score.

Part 7 - Engineering Interpretation and Philosophy

A cost model makes policy disagreement debuggable. Even an imperfect explicit model is more useful than an unexplained threshold because its assumptions can be tested, recalibrated, and rejected. The goal is not perfect prophecy; it is lower decision regret under bounded resources.

19.13.9 Exercise 08

Exercise Title: Add Tensor Lifetime Tracking and Automatically Release or Move Tensors After Their Final Planned GPU Use

Part 1 - Exercise Contract and Why It Matters

Attach planned-use and completion metadata to each tensor so storage is acted on immediately after its final legal GPU consumer. Clean replicas may be released; dirty authoritative values must be written back before CUDA storage disappears.

Part 2 - System and Memory Mental Model

"Last Python reference" and "last GPU use" are different boundaries. A kernel can still be reading storage after Python has queued the operation. The placement engine therefore separates planned-use exhaustion from physical completion and records a CUDA event after the final consumer. Autograd-saved tensors remain protected until backward no longer needs them.

Part 3 - Quantitative Prediction Before Execution

From the known execution plan, compute each tensor's final-use index and predicted release point. The area under the resident-bytes timeline estimates VRAM occupancy over time. Predict peak reduction from prompt release and D2H traffic introduced by dirty write-back.

Part 4 - Experimental Design and Instrumentation

Create clean intermediates, dirty outputs, aliased views, and tensors saved for backward. Log planned uses, submitted uses, completion events, authority changes, release bytes, write-back bytes, and peak VRAM. Poison or reuse freed buffers in a negative test so premature release becomes visible as wrong output.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass, field
import torch

@dataclass
class LifetimeEntry:
    key: str
    remaining_uses: int
    dirty: bool
    gpu_tensor: object
    cpu_tensor: object = None
    final_event: object = None
    protected_reasons: set = field(default_factory=set)
```

```
    def submit_use(self, stream=None):
        if self.remaining_uses <= 0:
            raise RuntimeError("use submitted after planned lifetime")
        if stream is None:
            stream = torch.cuda.current_stream()
        self.gpu_tensor.record_stream(stream)
        self.remaining_uses -= 1
        if self.remaining_uses == 0:
            self.final_event = torch.cuda.Event()
            self.final_event.record(stream)

    def finalize(self):
        if self.remaining_uses != 0:
            return "still_needed"
        if self.protected_reasons:
            return "protected"
        self.final_event.synchronize()
        if self.dirty:
            if self.cpu_tensor is None:
                self.cpu_tensor = torch.empty_like(
                    self.gpu_tensor, device="cpu", pin_memory=True)
            self.cpu_tensor.copy_(self.gpu_tensor)
        self.gpu_tensor = None
        return "written_back" if self.dirty else "released"

def E08(gpu_tensor, planned_uses, dirty=False):
    return LifetimeEntry("tensor", planned_uses, dirty, gpu_tensor)
```

Part 6 - Measurement, Validation, and Failure Analysis

The blocking write-back in this first implementation favors clarity. A later exercise makes it asynchronous while retaining source and destination until completion. A view cannot be released independently from its base storage, and reference counts alone cannot reveal outstanding kernels. Validate both values and storage identity, and include backward in the plan when gradients exist.

Part 7 - Engineering Interpretation and Philosophy

Lifetime is a contract between the execution schedule and the allocator. Early release is valuable only when the engine can prove there is no future semantic or asynchronous use. Conservatism wastes capacity; optimism corrupts results. The correct design makes that proof explicit.

19.13.10 Exercise 09

Exercise Title: Track Tensor Access Frequency and Dynamically Promote Hot Tensors from CPU RAM to GPU VRAM

Part 1 - Exercise Contract and Why It Matters

Build a promotion controller that observes accesses without retaining autograd graphs, identifies genuinely hot CPU tensors, reserves capacity before copying, and exposes the CUDA copy only after its completion dependency is established.

Part 2 - System and Memory Mental Model

Frequency answers how often; recency answers how recently; an exponentially decayed score adapts to phase changes. Promotion is a state transition from CPU-resident to H2D-pending and finally to both-clean. During H2D-pending the pinned source must remain unchanged and alive, while consumers wait on the recorded ready event rather than synchronizing the whole device.

Part 3 - Quantitative Prediction Before Execution

Predict the first access at which each key crosses the hot threshold. Estimate promotion bytes, subsequent GPU hits, avoided H2D copies, and wasted promotions. Require enough expected future reuse to amortize one completed H2D transfer and any eviction that admission would force.

Part 4 - Experimental Design and Instrumentation

Feed stable-hot, bursty, and phase-shifted traces. Record decayed score, last access, promotion decision, capacity rejection, copy completion, GPU hit, and time-to-first-benefit. Compare against always-CPU and oracle residency. Verify that metadata contains only scalar values and keys.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class HotTensorPromoter:
    def __init__(self, budget_bytes, threshold=3.0, decay=0.90):
        self.budget_bytes = budget_bytes
        self.threshold = threshold
        self.decay = decay
        self.used_bytes = 0
        self.scores = {}
        self.resident = {}
        self.ready = {}
        self.transfer_stream = torch.cuda.Stream()

    def observe(self, key):
        old = self.scores.get(key, 0.0)
        self.scores[key] = old * self.decay + 1.0

    def promote(self, key, cpu_tensor):
        self.observe(key)
        if key in self.resident:
            return "already_resident"
        if self.scores[key] < self.threshold:
            return "not_hot"
        size_bytes = cpu_tensor.numel() * cpu_tensor.element_size()
```

```
        if self.used_bytes + size_bytes > self.budget_bytes:
            return "budget_rejected"
        if not cpu_tensor.is_pinned():
            return "source_not_pinned"
        with torch.cuda.stream(self.transfer_stream):
            gpu_tensor = torch.empty_like(cpu_tensor, device="cuda")
            gpu_tensor.copy_(cpu_tensor, non_blocking=True)
            event = torch.cuda.Event()
            event.record(self.transfer_stream)
        self.resident[key] = gpu_tensor
        self.ready[key] = event
        self.used_bytes += size_bytes
        return "promotion_pending"

    def acquire(self, key, consumer_stream=None):
        if consumer_stream is None:
            consumer_stream = torch.cuda.current_stream()
        consumer_stream.wait_event(self.ready[key])
        tensor = self.resident[key]
        tensor.record_stream(consumer_stream)
        return tensor

def E09(budget_bytes):
    return HotTensorPromoter(budget_bytes)
```

Part 6 - Measurement, Validation, and Failure Analysis

This controller deliberately rejects rather than evicts when full; DMM10 can be integrated later for victim selection. The dictionary holding the CPU source is not shown, so the caller must preserve it until the ready event completes. A global cumulative frequency causes stale popularity; decay, windows, or phase detection are required. Do not count a pending promotion as a GPU hit.

Part 7 - Engineering Interpretation and Philosophy

Promotion is a speculative investment in future reuse. The important metric is not how many tensors become resident but how much critical-path transfer time is avoided after paying admission cost. A responsible engine records both benefit and regret for every speculation.

19.13.11 Exercise 10

Exercise Title: Detect Cold GPU Tensors and Demote Them to CPU When Available VRAM Falls Below a Configurable Threshold

Part 1 - Exercise Contract and Why It Matters

Create a pressure response that finds cold, legally movable tensors and demotes enough of them to restore a defined amount of available VRAM. The controller must protect active, dirty, aliased, and in-flight storage correctly.

Part 2 - System and Memory Mental Model

Low device-level free memory is a signal, not a victim decision. A clean tensor with a valid CPU replica can be dropped; a dirty GPU authority requires D2H write-back; an in-flight tensor cannot move until its consumer events complete. Releasing a Python tensor reduces live allocated memory, while the caching allocator may keep the resulting block reserved for reuse.

Part 3 - Quantitative Prediction Before Execution

Let F be current free bytes and F_t the recovery target. Select legal victims until their releasable bytes sum to at least $F_t - F$, including temporary D2H destination memory in the host budget. Predict victim order, write-back traffic, overshoot, and the next reuse penalty before execution.

Part 4 - Experimental Design and Instrumentation

Construct hot, cold-clean, cold-dirty, protected, and in-flight entries. Lower the artificial free-byte reading to trigger pressure. Log age, score, bytes, authority, eligibility reason, selected order, released allocated bytes, reserved bytes, D2H bytes, and subsequent reload misses.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class DemotionCandidate:
    key: str
    size_bytes: int
    last_access: int
    dirty: bool
    protected: bool
    in_flight: bool

def choose_cold_victims(entries, free_bytes, target_free_bytes):
    required = max(0, target_free_bytes - free_bytes)
    eligible = []
    for entry in entries:
        if entry.protected:
            continue
        if entry.in_flight:
            continue
        eligible.append(entry)
    eligible.sort(key=lambda item: item.last_access)

    victims = []
    reclaimable = 0
    writeback_bytes = 0
    for entry in eligible:
        if reclaimable >= required:
            break
        victims.append(entry)
        reclaimable += entry.size_bytes
        if entry.dirty:
```

```
            writeback_bytes += entry.size_bytes
    return victims, reclaimable, writeback_bytes

def E10(entries, free_bytes, target_free_bytes):
    victims, reclaimable, writeback = choose_cold_victims(
        entries, free_bytes, target_free_bytes)
    if free_bytes + reclaimable < target_free_bytes:
        raise RuntimeError("insufficient legal demotion candidates")
    return victims, reclaimable, writeback
```

Part 6 - Measurement, Validation, and Failure Analysis

Victim selection is separated from physical migration so it can be tested without risking data. Before committing, reserve pinned host capacity for dirty write-backs and revalidate that candidates did not become active. Compare `memory_allocated()` with device free memory; they describe different layers and will not move by identical amounts.

Part 7 - Engineering Interpretation and Philosophy

Pressure handling should be anticipatory and legality-first. The coldest tensor is not necessarily the cheapest victim, and a late emergency often has fewer safe choices. A good controller creates headroom before allocation failure and records why each protected tensor could not help.

19.13.12 Exercise 11

Exercise Title: Implement High- and Low-Watermark Tensor Placement to Prevent Rapid CPU-GPU Migration Oscillation

Part 1 - Exercise Contract and Why It Matters

Replace a single pressure threshold with a stateful high/low watermark controller. Once pressure begins, demotion continues until substantial headroom is restored; promotion remains disabled until the recovery boundary is crossed.

Part 2 - System and Memory Mental Model

A one-threshold policy chatters when free VRAM fluctuates around that boundary: one tensor is demoted, a small allocation completes, and the same tensor is promoted again. Hysteresis gives the controller memory. The entry boundary and exit boundary are deliberately different, producing stable modes named normal, pressure, and recovery.

Part 3 - Quantitative Prediction Before Execution

With total budget C, enter pressure when used bytes exceed H and keep evicting until used bytes fall below L, where $L < H < C$. The recovery amount is at least (used-L). Predict mode transitions and migration count for a noisy usage trace, then compare it with a single threshold.

Part 4 - Experimental Design and Instrumentation

Replay a trace that oscillates above and below the old threshold, a monotonic growth trace, and a sudden burst. Log mode, used bytes, requested bytes, promotion permission, eviction target, migrations, and time spent in pressure. Assert $0 \leq L < H \leq C$ and never admit past the hard capacity.

Part 5 - PyTorch/CUDA Implementation

```
class WatermarkController:
    def __init__(self, capacity_bytes, low_used_bytes,
                 high_used_bytes):
        if not 0 <= low_used_bytes < high_used_bytes <= capacity_bytes:
            raise ValueError("invalid watermarks")
        self.capacity = capacity_bytes
        self.low = low_used_bytes
        self.high = high_used_bytes
        self.mode = "normal"

    def update(self, used_bytes):
        if used_bytes > self.capacity:
            raise RuntimeError("hard VRAM budget violated")
        if self.mode == "normal" and used_bytes >= self.high:
            self.mode = "pressure"
        elif self.mode == "pressure" and used_bytes <= self.low:
            self.mode = "normal"
        return self.mode

    def eviction_target(self, used_bytes, requested_bytes=0):
        future_used = used_bytes + requested_bytes
        self.update(min(future_used, self.capacity))
        if future_used > self.capacity:
            self.mode = "pressure"
        if self.mode != "pressure":
            return 0
        return max(0, future_used - self.low)

    def promotion_allowed(self, used_bytes, tensor_bytes):
        if self.mode == "pressure":
            return False
        return used_bytes + tensor_bytes <= self.high

def E11(capacity_bytes):
    return WatermarkController(
        capacity_bytes,
        int(capacity_bytes * 0.70),
        int(capacity_bytes * 0.85),
    )
```

Part 6 - Measurement, Validation, and Failure Analysis

Watermarks should use the same memory definition as the hard admission budget. Mixing device free bytes with live tensor bytes creates unexplained transitions. Large allocations still require pre-admission eviction even in normal mode. Compare migration count, transferred bytes, and latency variance rather than claiming success merely because oscillation looks smoother.

Part 7 - Engineering Interpretation and Philosophy

Hysteresis accepts temporary deviation in exchange for system stability. It is a general control principle: decisions should not reverse until evidence is stronger than the noise that caused the previous decision. This reduces both transfer waste and tail-latency surprises.

19.13.13 Exercise 12

Exercise Title: Build a Placement Policy That Uses Tensor Size, Reuse Distance, Transfer Cost, and Remaining VRAM Simultaneously

Part 1 - Exercise Contract and Why It Matters

Combine the chapter's major signals into one transparent placement decision. The policy must apply hard legality and capacity constraints first, then rank feasible tensors by predicted benefit instead of relying on one isolated metric.

Part 2 - System and Memory Mental Model

Size represents capacity cost and transfer bytes. Reuse distance approximates how long the tensor occupies VRAM before its next benefit. Transfer cost captures the penalty avoided by residency. Remaining VRAM changes the opportunity cost: the same tensor can be rational under low pressure and irrational near the hard budget.

Part 3 - Quantitative Prediction Before Execution

Use expected avoided reload milliseconds divided by byte-time occupancy as a benefit density, then multiply by a pressure penalty. One interpretable score is

$$S = \frac{P_{reuse} T_{reload}}{M(1 + D_{reuse})} \left(\frac{F}{C}\right)^{\gamma}.$$

Hard-fit, protection, authority, and in-flight checks precede scoring. Predict ranking and rejected reasons for every candidate.

Part 4 - Experimental Design and Instrumentation

Generate candidates where each factor independently changes the correct answer: a tiny cold tensor, a large hot tensor, an expensive-to-reload tensor, and a soon-reused tensor. Sweep remaining VRAM and compare decisions with an oracle trace. Log raw features, normalized terms, final score, admission result, and realized regret.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class PlacementFeatures:
    key: str
    size_bytes: int
    reuse_probability: float
    reuse_distance: float
    reload_ms: float
    protected: bool = False
```

```
    in_flight: bool = False

def placement_score(item, free_bytes, capacity_bytes, gamma=2.0):
    if item.protected:
        return float("-inf"), "protected"
    if item.in_flight:
        return float("-inf"), "in_flight"
    if item.size_bytes > free_bytes:
        return float("-inf"), "does_not_fit"
    pressure_ratio = free_bytes / capacity_bytes
    pressure_term = pressure_ratio ** gamma
    benefit_ms = item.reuse_probability * item.reload_ms
    occupancy = item.size_bytes * (1.0 + item.reuse_distance)
    score = benefit_ms * pressure_term / occupancy
    return score, "scored"

def rank_candidates(candidates, free_bytes, capacity_bytes):
    results = []
    for item in candidates:
        score, reason = placement_score(
            item, free_bytes, capacity_bytes)
        results.append((score, reason, item.key))
    results.sort(key=lambda row: row[0], reverse=True)
    return results

def E12(candidates, free_bytes, capacity_bytes):
    return rank_candidates(candidates, free_bytes, capacity_bytes)
```

Part 6 - Measurement, Validation, and Failure Analysis

Feature scales and units matter. A score can silently become a size-only policy if byte magnitudes dominate. Report ablations that remove one factor at a time, and calibrate reload time with real measurements. Ranking is not admission: the engine must recompute available bytes after each selected tensor.

Part 7 - Engineering Interpretation and Philosophy

Multi-factor policy design is a claim about trade-offs. Keeping the formula visible lets engineers understand why an apparently hot tensor was rejected and whether the decision was caused by pressure, transfer cost, or reuse horizon. Interpretability is operational leverage during memory incidents.

19.13.14 Exercise 13

Exercise Title: Reproduce Tensor Placement Thrashing and Measure How Repeated Migration Can Make Dynamic Placement Slower Than Static Placement

Part 1 - Exercise Contract and Why It Matters

Construct a workload that defeats a reactive dynamic policy by repeatedly reversing placement decisions. Quantify the point at which migration traffic and synchronization make dynamic placement slower than a deliberately simple static baseline.

Part 2 - System and Memory Mental Model

Thrashing occurs when the working set and policy interact so that data is moved again before the previous movement produces enough useful work. A capacity just below the active working set and an alternating access pattern are sufficient. Each miss evicts the tensor required next, creating a cycle of negative reuse.

Part 3 - Quantitative Prediction Before Execution

For two equal tensors with capacity for one and trace $A, B, A, B, \ldots$, an LRU dynamic cache has nearly one H2D transfer per access. A static policy keeping A has about half hits without migration decisions. Predict migrations per useful GPU operation, total bytes, and completion time from calibrated transfer cost.

Part 4 - Experimental Design and Instrumentation

Sweep working-set-to-budget ratio from below one to well above one. Include sequential, alternating, and phase-local traces. Log migrations, re-migrations, residence duration, useful operations between movements, H2D/D2H bytes, completed time, and tail latency. Use exactly the same computations and outputs.

Part 5 - PyTorch/CUDA Implementation

```
from collections import OrderedDict

class ThrashMeter:
    def __init__(self, capacity_items):
        self.capacity = capacity_items
        self.resident = OrderedDict()
        self.loads = 0
        self.evictions = 0
        self.hits = 0
        self.last_loaded_step = {}
        self.short_residencies = 0
        self.step = 0

    def access(self, key):
        self.step += 1
        if key in self.resident:
            self.hits += 1
            self.resident.move_to_end(key)
            return
        self.loads += 1
        if len(self.resident) >= self.capacity:
            victim, unused = self.resident.popitem(last=False)
            age = self.step - self.last_loaded_step[victim]
            if age <= self.capacity + 1:
                self.short_residencies += 1
            self.evictions += 1
        self.resident[key] = True
        self.last_loaded_step[key] = self.step
```

```
    def report(self):
        accesses = self.step
        return {
            "loads_per_access": self.loads / max(1, accesses),
            "hit_rate": self.hits / max(1, accesses),
            "short_residencies": self.short_residencies,
            "evictions": self.evictions,
        }

def E13(repetitions=100):
    trace = []
    for unused in range(repetitions):
        trace.append("A")
        trace.append("B")
    meter = ThrashMeter(capacity_items=1)
    for key in trace:
        meter.access(key)
    return meter.report()
```

Part 6 - Measurement, Validation, and Failure Analysis

The item-count simulator assumes equal sizes; repeat with byte capacity and real copies to measure cost. A low hit rate is not by itself thrashing: streaming data may have no reuse. Require evidence of short residence followed by rapid reload. Account for write-back only when data is dirty, and synchronize at trial end.

Part 7 - Engineering Interpretation and Philosophy

Dynamic control can amplify scarcity when it reacts faster than useful workload phases change. Thrashing is not solved by a smarter victim name alone; it may require hysteresis, admission control, bypassing one-time data, or accepting a stable static partition.

19.13.15 Exercise 14

Exercise Title: Add Hysteresis to the Placement Controller and Prove That It Reduces Tensor Migration Thrashing

Part 1 - Exercise Contract and Why It Matters

Add distinct promotion and demotion evidence thresholds plus a minimum dwell period. Prove with a controlled trace that the mechanism reduces unnecessary state reversals while preserving most useful GPU hits.

Part 2 - System and Memory Mental Model

Hysteresis creates a dead band. A CPU tensor must become convincingly hot before promotion; a GPU tensor must become convincingly cold before demotion. A dwell period prevents immediate reversal even if noisy measurements briefly cross the opposite threshold. Hard capacity and correctness constraints still override the stability preference.

Part 3 - Quantitative Prediction Before Execution

Choose $S_p > S_d$ for promotion and demotion scores and dwell K steps. Predict transitions for scores oscillating around a single old threshold. The main metric is migration reduction; the counter-metric is missed useful GPU accesses caused by delayed promotion or demotion.

Part 4 - Experimental Design and Instrumentation

Evaluate smooth phase changes, high-frequency noise, sudden demand, and hard pressure. Compare no hysteresis, threshold-only hysteresis, and threshold plus dwell. Log state, score, age in state, transition reason, migration count, hit rate, transfer bytes, and realized runtime.

Part 5 - PyTorch/CUDA Implementation

```
class HysteresisPlacement:
    def __init__(self, promote_score=0.75, demote_score=0.30,
                 minimum_dwell_steps=5):
        if demote_score >= promote_score:
            raise ValueError("demote score must be below promote score")
        self.promote_score = promote_score
        self.demote_score = demote_score
        self.minimum_dwell = minimum_dwell_steps
        self.location = {}
        self.changed_at = {}
        self.transitions = 0

    def update(self, key, score, step, hard_pressure=False):
        location = self.location.get(key, "cpu")
        changed = self.changed_at.get(key, -self.minimum_dwell)
        dwell_satisfied = step - changed >= self.minimum_dwell

        next_location = location
        reason = "dead_band"
        if location == "cpu" and score >= self.promote_score:
            if dwell_satisfied and not hard_pressure:
                next_location = "cuda"
                reason = "hot_enough"
        elif location == "cuda" and score <= self.demote_score:
            if dwell_satisfied or hard_pressure:
                next_location = "cpu"
                reason = "cold_or_pressure"

        if next_location != location:
            self.location[key] = next_location
            self.changed_at[key] = step
            self.transitions += 1
        else:
            self.location.setdefault(key, location)
        return self.location[key], reason

def E14(scores):
    policy = HysteresisPlacement()
```

```
    decisions = []
    for step, score in enumerate(scores):
        decisions.append(policy.update("tensor", score, step))
    return decisions, policy.transitions
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not celebrate fewer transitions if the tensor remains on the wrong tier for most useful work. Plot transfer savings against added miss cost and report phase adaptation delay. Hard OOM prevention must be allowed to break dwell protection, but that override should be recorded separately from ordinary policy behavior.

Part 7 - Engineering Interpretation and Philosophy

Stable decisions require evidence to reverse them. Hysteresis makes the policy less sensitive to measurement noise and acknowledges that every transition has real physical cost. The dead band is not indecision; it is an engineered region where maintaining the current state is cheaper than chasing uncertain benefit.

19.13.16 Exercise 15

Exercise Title: Perform Tensor Migration with Pinned CPU Buffers and Non-Blocking CUDA Transfers Without Breaking Correctness

Part 1 - Exercise Contract and Why It Matters

Implement H2D and D2H migrations that allow the CPU to continue while CUDA copies execute, yet never publish an incomplete destination or release a source too early. The result is a small transactional migration primitive.

Part 2 - System and Memory Mental Model

Pinned host memory permits DMA without first staging a pageable source. A non-blocking copy means submission can return before completion; authority does not transfer at that moment. The transaction retains source and destination, records a completion event on its transfer stream, and commits only after the event reports completion. D2H results are unreadable by the CPU before commit.

Part 3 - Quantitative Prediction Before Execution

Predict pinned allocation cost, CPU submission time, completed transfer time, and effective bandwidth. For M bytes, expected completed time is (L+M/B), while submission should be much smaller. Reuse buffers so the steady state excludes repeated pinned allocation overhead.

Part 4 - Experimental Design and Instrumentation

Test H2D and D2H independently over several sizes and dtypes. Verify a checksum only after commit. Include negative tests that mutate a pinned H2D source early or read a D2H destination early, but isolate them from production runs. Log transaction state, event readiness, source lifetime, and synchronized bandwidth.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass
import torch

@dataclass
class Migration:
    direction: str
    source: object
    destination: object
    stream: object
    completion: object
    state: str = "pending"

    def ready(self):
        return self.completion.query()

    def commit(self):
        self.completion.synchronize()
        self.state = "committed"
        return self.destination

class TensorMigrator:
    def __init__(self):
        self.stream = torch.cuda.Stream()

    def h2d(self, cpu_tensor):
        if not cpu_tensor.is_pinned():
            raise ValueError("H2D source must be pinned")
        with torch.cuda.stream(self.stream):
            destination = torch.empty_like(cpu_tensor, device="cuda")
            destination.copy_(cpu_tensor, non_blocking=True)
            completion = torch.cuda.Event()
            completion.record(self.stream)
        return Migration("h2d", cpu_tensor, destination,
                         self.stream, completion)

    def d2h(self, gpu_tensor):
        destination = torch.empty_like(
            gpu_tensor, device="cpu", pin_memory=True)
        with torch.cuda.stream(self.stream):
            destination.copy_(gpu_tensor, non_blocking=True)
            gpu_tensor.record_stream(self.stream)
            completion = torch.cuda.Event()
            completion.record(self.stream)
        return Migration("d2h", gpu_tensor, destination,
                         self.stream, completion)

def E15():
    if not torch.cuda.is_available():
```

```
        raise RuntimeError("CUDA is required")
    return TensorMigrator()
```

Part 6 - Measurement, Validation, and Failure Analysis

The transaction object itself preserves both buffers until commit. For H2D, a CUDA consumer must wait on the completion event and record its stream on the destination. For D2H, CPU code must call commit before reading. Pool pinned buffers in long-running systems because excessive page locking can harm host memory availability and allocation latency.

Part 7 - Engineering Interpretation and Philosophy

Asynchrony changes correctness from an immediate-value model into a happens- before model. The event is not merely a timing instrument; it is proof that a state transition may legally commit. Good asynchronous APIs make incomplete states visible and difficult to misuse.

19.13.17 Exercise 16

Exercise Title: Coordinate Dynamic Tensor Placement Across Multiple CUDA Streams and Protect In-Flight Tensor Storage from Premature Reuse

Part 1 - Exercise Contract and Why It Matters

Allow a tensor produced or migrated on one stream to be consumed by several other streams, then demote or release it only after all consumers finish. Avoid device-wide synchronization while preserving storage lifetime and ordering.

Part 2 - System and Memory Mental Model

Stream order is local. Operations on stream B do not automatically wait for a producer on stream A. A ready event publishes production completion. Each consumer waits on ready, records its own done event, and calls `record_stream()` so the caching allocator knows its storage is used on that side stream. Demotion waits on all done events before copying or releasing.

Part 3 - Quantitative Prediction Before Execution

Draw a dependency graph and count required edges: one ready event from producer to each consumer, then one completion dependency from every consumer to the demotion stream. Predict concurrency and critical path. A global synchronize adds dependencies to unrelated work and should be measurably slower.

Part 4 - Experimental Design and Instrumentation

Run two independent consumers after one H2D promotion. Compare global synchronization with event-only coordination. Use unequal kernel durations so a missing second completion wait becomes observable. Log streams, ready and done events, overlap, total completion time, and checksum. Stress allocator reuse by allocating a same-sized temporary immediately after release.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class StreamSafeResident:
    def __init__(self, tensor, producer_stream):
        self.tensor = tensor
        self.ready = torch.cuda.Event()
        self.ready.record(producer_stream)
        self.consumer_events = []
        self.released = False

    def acquire(self, consumer_stream):
        if self.released:
            raise RuntimeError("resident storage already released")
        consumer_stream.wait_event(self.ready)
        self.tensor.record_stream(consumer_stream)
        return self.tensor

    def finish_consumer(self, consumer_stream):
        event = torch.cuda.Event()
        event.record(consumer_stream)
        self.consumer_events.append(event)

    def order_demotion(self, demotion_stream):
        for event in self.consumer_events:
            demotion_stream.wait_event(event)
        self.tensor.record_stream(demotion_stream)

    def release_after(self, demotion_stream):
        completion = torch.cuda.Event()
        completion.record(demotion_stream)
        completion.synchronize()
        self.tensor = None
        self.released = True

def E16(cpu_source):
    transfer = torch.cuda.Stream()
    with torch.cuda.stream(transfer):
        gpu = torch.empty_like(cpu_source, device="cuda")
        gpu.copy_(cpu_source, non_blocking=True)
        resident = StreamSafeResident(gpu, transfer)
    return resident
```

Part 6 - Measurement, Validation, and Failure Analysis

Calling `record_stream()` protects allocator reuse but does not create a data dependency; the explicit event wait is still required. Conversely, an event wait orders kernels but a placement manager that drops its last reference must still account for allocator lifetime. Register consumers before demotion selection so late acquisition cannot race with the release decision.

Part 7 - Engineering Interpretation and Philosophy

Multi-stream placement is resource ownership over a partial order. Launching a kernel is not evidence that its storage is free. The engine must know every outstanding reader or writer and express only the dependencies required for correctness. This preserves concurrency without gambling with memory.

19.13.18 Exercise 17

Exercise Title: Build a Memory-Budget Simulator That Predicts Tensor Residency Decisions Before Executing the Real CUDA Workload

Part 1 - Exercise Contract and Why It Matters

Build a deterministic, CUDA-free simulator that replays planned tensor accesses and predicts admissions, hits, evictions, traffic, and peak residency under a strict byte budget. Use it to reject impossible configurations before risking a real OOM.

Part 2 - System and Memory Mental Model

The simulator models policy-visible live tensors, not the entire CUDA allocator. Its budget therefore needs a safety reserve for CUDA context, workspaces, temporary activations, and non-managed allocations. Each access changes a logical state machine; transfers consume predicted time and bytes but allocate no real storage.

Part 3 - Quantitative Prediction Before Execution

For every trace step, predict used bytes before access, required admission bytes, victim bytes, used bytes after access, and transfer time. The core invariant is $0 \leq R_t \leq C_{managed}$. Estimate real peak as managed peak plus a separately measured unmanaged reserve and fragmentation margin.

Part 4 - Experimental Design and Instrumentation

Replay a captured real trace and a synthetic adversarial trace. Save every decision with a reason. Compare predicted H2D bytes, victim order, managed peak, and time against the real engine. Report residual error from overlap, allocator rounding, workspaces, and policy timing instead of forcing exact agreement.

Part 5 - PyTorch/CUDA Implementation

```
from collections import OrderedDict

class PlacementSimulator:
    def __init__(self, sizes, managed_budget, h2d_bytes_per_ms):
        self.sizes = sizes
        self.budget = managed_budget
        self.bandwidth = h2d_bytes_per_ms
        self.resident = OrderedDict()
        self.used = 0
        self.peak = 0
        self.h2d_bytes = 0
        self.events = []
```

```
    def access(self, step, key):
        if key in self.resident:
            self.resident.move_to_end(key)
            self.events.append((step, key, "hit", self.used))
            return
        needed = self.sizes[key]
        if needed > self.budget:
            raise RuntimeError("single tensor exceeds managed budget")
        while self.used + needed > self.budget:
            victim, unused = self.resident.popitem(last=False)
            self.used -= self.sizes[victim]
            self.events.append((step, victim, "evict", self.used))
        self.resident[key] = True
        self.used += needed
        self.h2d_bytes += needed
        self.peak = max(self.peak, self.used)
        self.events.append((step, key, "load", self.used))
        assert 0 <= self.used <= self.budget

    def report(self):
        return {
            "managed_peak_bytes": self.peak,
            "h2d_bytes": self.h2d_bytes,
            "predicted_h2d_ms": self.h2d_bytes / self.bandwidth,
            "events": self.events,
        }

def E17(sizes, trace, managed_budget, h2d_bytes_per_ms):
    simulator = PlacementSimulator(
        sizes, managed_budget, h2d_bytes_per_ms)
    for step, key in enumerate(trace):
        simulator.access(step, key)
    return simulator.report()
```

Part 6 - Measurement, Validation, and Failure Analysis

An LRU simulator is not a proof that the real allocator can satisfy a contiguous request. Add measured safety margins and validate near the boundary gradually. Do not confuse simulated tensor bytes with PyTorch reserved memory. A useful simulator is deliberately conservative and reports which unmodeled categories its reserve is intended to cover.

Part 7 - Engineering Interpretation and Philosophy

Simulation separates policy reasoning from hazardous execution. It gives the engine a chance to say “this plan cannot fit” before CUDA discovers that fact through failure. Prediction need not be perfect to be valuable; bounded and explainable error is enough to guide safe configuration.

19.13.19 Exercise 18

Exercise Title: Stress-Test the Placement Engine with Variable Tensor Sizes, Random Reuse Patterns, and Artificially Restricted VRAM

Part 1 - Exercise Contract and Why It Matters

Subject the engine to reproducible variable sizes, changing reuse distributions, dirty updates, protected intervals, and a managed budget far below physical VRAM. The test must discover invariant violations and performance collapse without intentionally destabilizing the whole device.

Part 2 - System and Memory Mental Model

Artificial restriction is safer and more repeatable than allocating until physical OOM. Variable sizes expose byte-accounting and fragmentation-sensitive choices; random reuse exposes stale prediction; dirty and in-flight states test correctness barriers. Stress is a distribution of cases, not one maximum tensor.

Part 3 - Quantitative Prediction Before Execution

Choose a seeded catalogue and compute whether every individual tensor fits the managed budget. Predict approximate working-set size, compulsory traffic, and pressure frequency. Define pass bounds before execution: zero wrong checksums, zero budget violations, bounded migration rate, and no unhandled allocation failure.

Part 4 - Experimental Design and Instrumentation

Run stable, bursty, alternating, and Zipf-like phases over multiple seeds. Keep sentinel values on authoritative copies and verify them after each completed migration. Record resident bytes, peak, admissions, demotions, dirty write-backs, rejected promotions, migrations per access, hit rate, latency percentiles, and policy decision time.

Part 5 - PyTorch/CUDA Implementation

```
import random
from collections import OrderedDict

class RestrictedBudgetStress:
    def __init__(self, sizes, budget_bytes):
        self.sizes = sizes
        self.budget = budget_bytes
        self.resident = OrderedDict()
        self.used = 0
        self.loads = 0
        self.evictions = 0

    def access(self, key):
        if key in self.resident:
            self.resident.move_to_end(key)
            return "hit"
        needed = self.sizes[key]
        if needed > self.budget:
```

```
            return "bypass_oversized"
        while self.used + needed > self.budget:
            victim, unused = self.resident.popitem(last=False)
            self.used -= self.sizes[victim]
            self.evictions += 1
        self.resident[key] = True
        self.used += needed
        self.loads += 1
        assert self.used <= self.budget
        return "load"

def E18(seed=17, tensor_count=100, steps=5000,
        budget_bytes=256 * 1024 * 1024):
    generator = random.Random(seed)
    sizes = {}
    for index in range(tensor_count):
        size_mib = generator.randint(1, 32)
        sizes[index] = size_mib * 1024 * 1024
    engine = RestrictedBudgetStress(sizes, budget_bytes)
    outcomes = {"hit": 0, "load": 0, "bypass_oversized": 0}
    for unused in range(steps):
        if generator.random() < 0.80:
            key = generator.randrange(max(1, tensor_count // 10))
        else:
            key = generator.randrange(tensor_count)
        outcome = engine.access(key)
        outcomes[outcome] += 1
    return outcomes, engine.loads, engine.evictions, engine.used
```

Part 6 - Measurement, Validation, and Failure Analysis

This first harness tests policy accounting; integrate real CUDA migration only after it passes. A fixed random seed is necessary but insufficient: preserve catalogue, trace, software versions, and hardware information. Minimize any failure trace to the shortest reproducible sequence, then inspect authority, events, and byte accounting at the first divergence.

Part 7 - Engineering Interpretation and Philosophy

Stress testing asks where assumptions cease to hold. Success is not surviving a large average workload; it is preserving invariants through rare combinations of pressure, timing, and reuse. Artificial budgets make those combinations repeatable and turn memory failure into a testable engineering object.

19.13.20 Exercise 19

Exercise Title: Build and Benchmark a PyTorch Dynamic Tensor Placement Engine That Maintains a Working Set Across RAM and VRAM Under a Strict Memory Budget

Part 1 - Exercise Contract and Why It Matters

Integrate accounting, policy, migration, events, authority, lifetime, and watermarks into one placement engine. The final system must keep a changing working set inside a hard managed-VRAM budget, preserve numerical correctness, and produce evidence explaining every promotion and demotion.

Part 2 - System and Memory Mental Model

Each logical tensor has one authoritative value and optional clean replicas. Its state may be CPU, H2D-pending, GPU, both-clean, GPU-dirty, D2H-pending, or released. Admission reserves bytes before CUDA allocation. Ready and consumer events define legality across streams. High/low watermarks provide headroom; reuse-cost scoring determines which legal tensor deserves residence.

Part 3 - Quantitative Prediction Before Execution

Set managed capacity below device free memory by explicit context, workspace, temporary, and fragmentation reserves. Predict peak managed bytes, transfer traffic, hit rate, critical-path transfer time, and decision overhead with the E17 simulator. State acceptance criteria: no budget violation, exact output parity, no use-before-ready, and better completed time than CPU-only or static placement on at least the workload class targeted by the policy.

Part 4 - Experimental Design and Instrumentation

Benchmark CPU-only, static-GPU, and dynamic variants on identical traces. Include stable locality, phase shifts, random reuse, and adversarial thrashing. Warm up, synchronize trial boundaries, repeat runs, and report medians and tails. Record state transitions, reasons, resident and pending bytes, allocated and reserved CUDA memory, H2D/D2H bytes, overlap, hits, evictions, policy time, and checksums.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass, field
from collections import OrderedDict
import torch

@dataclass
class TensorRecord:
    key: str
    cpu: object
    size_bytes: int
    gpu: object = None
    authority: str = "cpu"
    dirty: bool = False
    protected: set = field(default_factory=set)
    ready: object = None
    last_access: int = 0
    frequency: int = 0

class DynamicPlacementEngine:
    def __init__(self, budget_bytes, low_ratio=0.70,
                 high_ratio=0.85):
        self.budget = budget_bytes
```

```
        self.low = int(budget_bytes * low_ratio)
        self.high = int(budget_bytes * high_ratio)
        self.used = 0
        self.step = 0
        self.records = {}
        self.recency = OrderedDict()
        self.transfer_stream = torch.cuda.Stream()

    def register(self, key, cpu_tensor):
        if not cpu_tensor.is_pinned():
            cpu_tensor = cpu_tensor.pin_memory()
        size = cpu_tensor.numel() * cpu_tensor.element_size()
        self.records[key] = TensorRecord(key, cpu_tensor, size)

    def _victim(self, excluded_key):
        for key in self.recency:
            record = self.records[key]
            if key == excluded_key:
                continue
            if record.protected:
                continue
            return key
        return None

    def _demote_blocking(self, key):
        record = self.records[key]
        if record.dirty:
            record.cpu.copy_(record.gpu)
            record.authority = "cpu"
            record.dirty = False
        record.gpu = None
        record.ready = None
        self.used -= record.size_bytes
        self.recency.pop(key, None)

    def _make_room(self, requested_bytes, excluded_key):
        target = min(self.low, self.budget - requested_bytes)
        while self.used + requested_bytes > self.high:
            victim = self._victim(excluded_key)
            if victim is None:
                raise RuntimeError("no legal victim for admission")
            self._demote_blocking(victim)
        if self.used + requested_bytes > self.budget:
            raise RuntimeError("hard managed budget would be exceeded")
        assert self.used <= max(self.high, target)

    def acquire(self, key, consumer_stream=None):
        self.step += 1
```

```
        record = self.records[key]
        record.frequency += 1
        record.last_access = self.step
        if record.gpu is None:
            if record.size_bytes > self.budget:
                raise RuntimeError("tensor exceeds managed budget")
            self._make_room(record.size_bytes, key)
            with torch.cuda.stream(self.transfer_stream):
                record.gpu = torch.empty_like(record.cpu, device="cuda")
                record.gpu.copy_(record.cpu, non_blocking=True)
                record.ready = torch.cuda.Event()
                record.ready.record(self.transfer_stream)
            self.used += record.size_bytes
        if consumer_stream is None:
            consumer_stream = torch.cuda.current_stream()
        consumer_stream.wait_event(record.ready)
        record.gpu.record_stream(consumer_stream)
        self.recency.pop(key, None)
        self.recency[key] = True
        assert self.used <= self.budget
        return record.gpu

    def mark_dirty(self, key):
        record = self.records[key]
        record.dirty = True
        record.authority = "gpu"

def E19(budget_bytes):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required")
    return DynamicPlacementEngine(budget_bytes)
```

Part 6 - Measurement, Validation, and Failure Analysis

The compact reference engine uses blocking dirty demotion for an auditable baseline. A production version must register every consumer completion before demotion, use the E15 transaction for asynchronous write-back, count pending bytes, handle aliases by storage identity, and integrate autograd protection. Test admission rollback after allocation failure and never rely on `empty_cache()` as a substitute for live-tensor policy.

Part 7 - Engineering Interpretation and Philosophy

Dynamic placement is a small memory operating system inside the training or inference process. Its success comes from disciplined state, accounting, and ordering rather than clever eviction scores alone. The engine is trustworthy when it can defend three claims for every tensor: the newest value is known, all users are ordered, and the resource budget is never silently exceeded.

19.14 DMM12

Chapter Title: Dynamic Parameter Placement
Specialization: Dynamic Memory Management and CUDA Systems

19.14.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Parameter Residency, Module Ownership, and Weight Movement Between CPU RAM and GPU VRAM

Part 1 - Exercise Contract and Why It Matters

Build a state model for parameters before writing an offloading engine. A parameter is not merely a tensor containing numbers: it is a registered member of an `nn.Module`, can be referenced by an optimizer, may be shared by several layers, and participates in autograd. The model must explain where the authoritative weight resides and who is allowed to move it.

The final contract must answer seven questions at every execution point: which logical parameter is being discussed, which module names refer to it, which storage contains the newest value, whether a CUDA copy is ready, which stream is still using it, whether a gradient exists, and what transition may happen next.

Part 2 - System and Memory Mental Model

Separate parameter identity from parameter storage. The Python `Parameter` object is what modules and optimizers reference; its data storage may change tiers. A single object may have multiple qualified names when weights are tied. A parameter can be CPU-only, H2D-pending, GPU-ready, GPU-dirty, D2H-pending, both-clean, protected-by-autograd, or released.

Module ownership is hierarchical rather than exclusive. A fully qualified parameter name identifies a path through a model, not necessarily a unique allocation. Buffers, optimizer states, gradients, and activations are separate memory categories and must not be hidden inside the parameter total. During training, moving a leaf parameter after forward can invalidate the graph expected by backward.

Part 3 - Quantitative Prediction Before Execution

For a dense parameter with shape dimensions d_i and element size s, predict bytes as $M = s \prod_i d_i$. Calculate logical bytes by name and unique bytes by parameter identity so tied weights are not counted twice. A promotion temporarily requires CPU source bytes, CUDA destination bytes, and possibly pinned staging bytes.

Estimate one reload as $T_{H2D} = L + M/B$. Keeping a group resident is rational when expected avoided reload time exceeds its VRAM opportunity cost and the group fits after workspace and activation reserves.

Part 4 - Experimental Design and Instrumentation

Create a model containing ordinary parameters, a tied embedding/output weight, a registered buffer, and frozen weights. Record every qualified name, object identity, storage identity, device, bytes, owner layer, gradient state, authority, pending event, and protected reason. Exercise clean promotion, layer execution, dirty training update, write-back, and a deliberately illegal demotion before backward.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass, field

@dataclass
class ParameterState:
    canonical_name: str
    owner_names: list
    size_bytes: int
    authority: str = "cpu"
    cpu_present: bool = True
    gpu_present: bool = False
    transfer_pending: bool = False
    dirty: bool = False
    protected_reasons: set = field(default_factory=set)

    def begin_h2d(self):
        if not self.cpu_present:
            raise RuntimeError("no CPU source for promotion")
        if self.transfer_pending:
            raise RuntimeError("another migration is already pending")
        self.transfer_pending = True

    def commit_h2d(self):
        if not self.transfer_pending:
            raise RuntimeError("no H2D migration to commit")
        self.gpu_present = True
        self.transfer_pending = False
        self.authority = "both_clean"

    def mark_gpu_updated(self):
        if not self.gpu_present:
            raise RuntimeError("GPU copy does not exist")
        self.authority = "gpu"
        self.dirty = True

def E00(name, owner_names, shape, element_size):
    elements = 1
    for dimension in shape:
        elements *= dimension
    return ParameterState(name, owner_names,
                          elements * element_size)
```

Part 6 - Measurement, Validation, and Failure Analysis

A device label cannot prove authority, readiness, or uniqueness. Calling `module.to()` replaces the placement of parameters and buffers together, while an optimizer may still own state tensors whose devices now disagree. Views and tied parameters make name-based byte sums incorrect. Validate object identity, unique storage, numerical checksums, and event completion after every transition.

Part 7 - Engineering Interpretation and Philosophy

Dynamic parameter placement is distributed ownership inside one Python process. The difficult problem is not copying bytes; it is preserving meaning while storage moves. A trustworthy engine can defend who owns the newest value, why a copy is safe to use, and why no future computation still needs the old tier.

19.14.2 Exercise 01

Exercise Title: Inspect Every PyTorch Parameter by Name, Shape, Dtype, Device, Element Count, and Memory Footprint

Part 1 - Exercise Contract and Why It Matters

Produce a complete parameter inventory that is safe to retain as telemetry. The report must reveal qualified names, tensor facts, trainability, aliases, and both logical and unique physical bytes without storing live Parameter objects.

Part 2 - System and Memory Mental Model

`named_parameters()` normally removes duplicate objects, which is useful for optimization but can hide tied ownership paths. Request duplicates when the installed PyTorch version supports it, then group names by object identity. Logical bytes equal element count times element size; unique bytes must be counted once per storage or parameter identity.

Part 3 - Quantitative Prediction Before Execution

Predict totals per dtype, device, and trainability class before running the inspector. For tied weights, predict that the by-name logical sum exceeds the unique sum. Compare parameter bytes with CUDA `memory_allocated()` only after separating buffers, gradients, context, and unrelated allocations.

Part 4 - Experimental Design and Instrumentation

Inspect a model before CUDA movement, after partial movement, and after gradient creation. Include frozen parameters and a tied weight. Save only strings, integers, booleans, shapes, and identities. Assert that every reported byte count is non-negative and that every object identity maps to a consistent shape and dtype.

Part 5 - PyTorch/CUDA Implementation

```
from collections import defaultdict

def parameter_rows(model):
    rows = []
    owners = defaultdict(list)
    iterator = model.named_parameters(remove_duplicate=False)
    for name, parameter in iterator:
        parameter_id = id(parameter)
        owners[parameter_id].append(name)
        row = {
            "name": name,
            "parameter_id": parameter_id,
```

```
            "shape": tuple(parameter.shape),
            "dtype": str(parameter.dtype),
            "device": str(parameter.device),
            "elements": parameter.numel(),
            "element_size": parameter.element_size(),
            "logical_bytes": parameter.numel()
            * parameter.element_size(),
            "requires_grad": parameter.requires_grad,
            "gradient_present": parameter.grad is not None,
        }
        rows.append(row)
    return rows, owners

def E01(model):
    rows, owners = parameter_rows(model)
    unique_bytes = {}
    logical_bytes_by_name = 0
    for row in rows:
        logical_bytes_by_name += row["logical_bytes"]
        parameter_id = row["parameter_id"]
        if parameter_id not in unique_bytes:
            unique_bytes[parameter_id] = row["logical_bytes"]
        row["owner_names"] = tuple(owners[parameter_id])
    unique_parameter_bytes = 0
    for size_bytes in unique_bytes.values():
        unique_parameter_bytes += size_bytes
    return {
        "parameters": rows,
        "logical_bytes_by_name": logical_bytes_by_name,
        "unique_parameter_bytes": unique_parameter_bytes,
    }
```

Part 6 - Measurement, Validation, and Failure Analysis

Parameter identity detects ordinary tying, while unusual shared-storage views may require storage-pointer grouping. Quantized, sparse, meta, and sharded parameters need special accounting and should be marked rather than misreported. The explicit loops make every contribution to both totals visible.

Part 7 - Engineering Interpretation and Philosophy

An inventory is the accounting ledger from which placement policy borrows truth. If ownership or bytes are wrong at this stage, every later budget, eviction, and performance conclusion becomes numerically polished fiction.

19.14.3 Exercise 02

Exercise Title: Calculate the Exact VRAM Required by Model Parameters Before Moving the Model to CUDA

Part 1 - Exercise Contract and Why It Matters

Calculate parameter-only CUDA bytes before any model movement and distinguish that exact tensor payload from the larger amount a real process will consume. The estimator must handle tied parameters and mixed dtypes without guessing.

Part 2 - System and Memory Mental Model

Dense parameter payload is deterministic from shape and dtype. CUDA allocator rounding, module buffers, gradients, optimizer state, activations, workspaces, and context memory are additional categories. A training budget that includes only weights can be exact about the wrong scope and still OOM immediately.

Part 3 - Quantitative Prediction Before Execution

Compute

$$M_{parameters} = \sum_{p \in U} \text{numel}(p)\,\text{element_size}(p),$$

where U is the set of unique Parameter objects. Produce per-dtype and per-layer subtotals. For inference, add buffers and measured workspace reserve; for training, separately estimate gradients and optimizer states.

Part 4 - Experimental Design and Instrumentation

Reset CUDA peak statistics in a clean process, record the baseline after CUDA initialization, move the model, synchronize, and measure the delta. Compare the prediction with observed allocated and reserved bytes. Repeat for float32, float16, and a mixed-dtype model while keeping architecture constant.

Part 5 - PyTorch/CUDA Implementation

```
from collections import defaultdict

def exact_unique_parameter_bytes(model):
    seen = set()
    total_bytes = 0
    by_dtype = defaultdict(int)
    by_top_level_module = defaultdict(int)

    for name, parameter in model.named_parameters(
            remove_duplicate=False):
        parameter_id = id(parameter)
        if parameter_id in seen:
            continue
        seen.add(parameter_id)
        size_bytes = parameter.numel() * parameter.element_size()
        total_bytes += size_bytes
        by_dtype[str(parameter.dtype)] += size_bytes
        owner = name.split(".")[0]
        by_top_level_module[owner] += size_bytes

    return total_bytes, dict(by_dtype), dict(by_top_level_module)
```

```
def E02(model):
    total, by_dtype, by_module = exact_unique_parameter_bytes(model)
    return {
        "exact_parameter_bytes": total,
        "parameter_MiB": total / (1024.0 ** 2),
        "by_dtype": by_dtype,
        "by_top_level_module": by_module,
    }
```

Part 6 - Measurement, Validation, and Failure Analysis

Observed allocated memory can exceed the exact payload because CUDA allocations are rounded and the process already owns other tensors. Reserved memory is not parameter size. Conversely, a meta-device model has shapes but no materialized storage. State exactly which categories the estimate includes and never market parameter-only bytes as total training VRAM.

Part 7 - Engineering Interpretation and Philosophy

Exactness requires a boundary. This exercise teaches the useful discipline of saying "exact parameter payload" instead of "exact model memory." Honest scope makes estimates composable: weights, gradients, states, activations, and workspaces can later be added without double counting.

19.14.4 Exercise 03

Exercise Title: Move Selected Parameters Between CPU and GPU Without Moving the Entire nn.Module

Part 1 - Exercise Contract and Why It Matters

Move an explicit set of registered parameters while preserving their Parameter object identities and leaving unrelated weights untouched. Restrict the first implementation to inference or to movement before a training graph is built.

Part 2 - System and Memory Mental Model

`module.to()` recursively moves parameters and buffers, which is too broad for a placement engine. Replacing a Parameter object can break optimizer and shared-weight references. Changing its underlying data preserves the object but still changes device storage, so gradients and optimizer state require separate coordination.

Part 3 - Quantitative Prediction Before Execution

Before moving, predict selected unique bytes, H2D or D2H traffic, and peak bytes during the copy. Same-device selections should produce zero traffic. If two names identify the same Parameter, perform one physical movement and record both owners as satisfied.

Part 4 - Experimental Design and Instrumentation

Capture object IDs and checksums before movement. Move one layer, verify all other devices, execute a compatible computation, then move it back. Test tied weights and reject

movement while a protected autograd graph exists. Log reason, source, target, bytes, copied identities, and completion time.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def selected_parameter_objects(model, selected_names):
    selected_by_id = {}
    matched_names = set()
    for name, parameter in model.named_parameters(
            remove_duplicate=False):
        if name not in selected_names:
            continue
        matched_names.add(name)
        parameter_id = id(parameter)
        if parameter_id not in selected_by_id:
            selected_by_id[parameter_id] = (name, parameter)
    missing = set(selected_names)
    for name in matched_names:
        missing.discard(name)
    if missing:
        raise KeyError("unknown parameter names: " + str(sorted(missing)))
    selected = []
    for entry in selected_by_id.values():
        selected.append(entry)
    return selected

def move_selected_parameters(model, selected_names, target):
    moved = []
    selected = selected_parameter_objects(model, selected_names)
    with torch.no_grad():
        for name, parameter in selected:
            source = str(parameter.device)
            if source == str(torch.device(target)):
                moved.append((name, source, source, 0))
                continue
            bytes_moved = parameter.numel() * parameter.element_size()
            parameter.data = parameter.data.to(target)
            if parameter.grad is not None:
                parameter.grad = parameter.grad.to(target)
            moved.append((name, source, str(parameter.device),
                          bytes_moved))
    return moved

def E03(model, selected_names, target="cuda:0"):
    return move_selected_parameters(model, selected_names, target)
```

Part 6 - Measurement, Validation, and Failure Analysis

This explicit mutation is suitable for controlled education and inference, not for moving a leaf parameter between forward and backward. Optimizer state is not moved by this function and can later cause device mismatch. The missing-name check sees only one owner for tied parameters after deduplication, so a production registry should resolve every alias name to a canonical identity.

Part 7 - Engineering Interpretation and Philosophy

Selective movement reveals why dynamic placement is a systems feature rather than a convenience call. Preserving object identity protects ownership, but correct execution also requires coordination with gradients, optimizer state, buffers, inputs, and the lifetime of every graph that used the old storage.

19.14.5 Exercise 04

Exercise Title: Build a Parameter Registry That Tracks Device, Size, Layer Ownership, Access Count, and Last GPU Use

Part 1 - Exercise Contract and Why It Matters

Create the metadata control plane for dynamic placement. The registry must map every qualified name to one canonical parameter record, track scalar usage history, and avoid keeping accidental tensor or autograd references in logs.

Part 2 - System and Memory Mental Model

Names describe ownership paths; canonical identity describes the movable resource. One record can therefore contain several owner names. Current device is observable, while authority, pending migration, protection, and final GPU use are placement-engine states. Access count measures the past and is only a forecast of future reuse.

Part 3 - Quantitative Prediction Before Execution

Predict record count, alias count, total unique bytes, and per-layer ownership bytes. After replaying an access trace, predict access count, last-use step, reuse distance, and candidate hotness. Define whether a layer access increments all its parameters once or each actual low-level read.

Part 4 - Experimental Design and Instrumentation

Register a tied-weight model and assert that multiple names resolve to one canonical ID. Replay stable, bursty, and phase-changing layer traces. Log only scalar metadata and bounded event references needed for correctness. Verify that deleting the model is not prevented by the telemetry export.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass, field

@dataclass
class ParameterRecord:
    canonical_id: int
    owner_names: list
    shape: tuple
```

```
    dtype: str
    device: str
    size_bytes: int
    access_count: int = 0
    last_gpu_use: int = -1
    protected_reasons: set = field(default_factory=set)

class ParameterRegistry:
    def __init__(self, model):
        self.records = {}
        self.name_to_id = {}
        for name, parameter in model.named_parameters(
                remove_duplicate=False):
            canonical_id = id(parameter)
            self.name_to_id[name] = canonical_id
            if canonical_id not in self.records:
                self.records[canonical_id] = ParameterRecord(
                    canonical_id=canonical_id,
                    owner_names=[],
                    shape=tuple(parameter.shape),
                    dtype=str(parameter.dtype),
                    device=str(parameter.device),
                    size_bytes=parameter.numel()
                    * parameter.element_size(),
                )
            self.records[canonical_id].owner_names.append(name)

    def record_gpu_use(self, name, step):
        canonical_id = self.name_to_id[name]
        record = self.records[canonical_id]
        record.access_count += 1
        record.last_gpu_use = step

def E04(model):
    return ParameterRegistry(model)
```

Part 6 - Measurement, Validation, and Failure Analysis

The registry stores object IDs but not objects, so an ID could eventually be reused after model destruction. Scope the registry to one model lifetime. Refresh device state only at controlled transitions rather than trusting stale metadata. Tied parameters require one migration, whereas two independent parameters with equal values remain two resources.

Part 7 - Engineering Interpretation and Philosophy

A placement engine needs a stable vocabulary for discussing resources. The registry turns a changing module hierarchy into accountable identities and histories. Policy may be probabilistic, but identity and byte accounting cannot be.

19.14.6 Exercise 05

Exercise Title: Implement a Static Per-Layer Parameter Placement Map and Verify Forward Execution Across CPU and GPU Boundaries

Part 1 - Exercise Contract and Why It Matters

Assign every layer in a sequential model to an explicit device before inference, then move activations only at declared boundaries. The result is a reproducible baseline for later dynamic policies and a direct demonstration that parameter placement constrains activation placement.

Part 2 - System and Memory Mental Model

A PyTorch operator normally requires input, parameter, and relevant buffer to share a device. A static map such as layer 0 on CPU and layers 1–3 on CUDA therefore creates an activation transfer between layers 0 and 1. Consecutive layers on the same tier form a stage and avoid intermediate transfers.

Part 3 - Quantitative Prediction Before Execution

Sum unique parameter bytes per device. For every adjacent pair whose assigned devices differ, predict one activation transfer of N element_size bytes. Parameter fit alone is insufficient; reserve VRAM for the largest live activation and temporary kernel workspace.

Part 4 - Experimental Design and Instrumentation

Use one all-CPU map, one all-CUDA map, and at least two split points. Record parameter bytes, activation boundary bytes, device before each layer, transfer time, layer time, output checksum, and synchronized end-to-end latency. Compare all outputs against the all-CPU reference within dtype-appropriate tolerance.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def apply_static_layer_map(layers, placement_map):
    for index, layer in enumerate(layers):
        if index not in placement_map:
            raise KeyError("missing device for layer " + str(index))
        layer.to(placement_map[index])

def execute_with_layer_map(layers, placement_map, inputs):
    activation = inputs
    trace = []
    for index, layer in enumerate(layers):
        target = torch.device(placement_map[index])
        source = activation.device
        copied = source != target
        if copied:
            activation = activation.to(target)
        activation = layer(activation)
        trace.append({
            "layer": index,
```

```
            "input_source": str(source),
            "execution_device": str(target),
            "activation_shape": tuple(activation.shape),
            "boundary_copy": copied,
        })
    return activation, trace

def E05(layers, placement_map, inputs):
    apply_static_layer_map(layers, placement_map)
    return execute_with_layer_map(layers, placement_map, inputs)
```

Part 6 - Measurement, Validation, and Failure Analysis

Moving only parameters while leaving BatchNorm buffers behind creates a hidden device mismatch; the static baseline moves each complete layer. Non-sequential models require a graph schedule rather than a simple loop. Synchronize trial boundaries because CUDA layer calls are asynchronous, and do not compare a CPU float32 output with an implicitly changed CUDA dtype.

Part 7 - Engineering Interpretation and Philosophy

Placement partitions an execution graph. A map that fits weights but causes an activation to cross the bus repeatedly can be slower than CPU execution. Good partitioning respects both state residency and the dataflow edges connecting stages.

19.14.7 Exercise 06

Exercise Title: Compare Whole-Model CUDA Residency Against Partial Parameter Residency Under the Same Inference Workload

Part 1 - Exercise Contract and Why It Matters

Benchmark two semantically identical inference systems: one with the entire model resident on CUDA and another with only selected layers resident. Measure the memory saved and the latency paid under the same requests and precision.

Part 2 - System and Memory Mental Model

Whole-model residency pays one initial weight transfer and maximizes reuse, but requires enough VRAM for every weight plus inference working memory. Partial residency lowers persistent parameter bytes while introducing parameter or activation transfers. Initial-load latency and steady-state latency are distinct deployment questions and must both be reported.

Part 3 - Quantitative Prediction Before Execution

Predict resident parameter bytes, boundary bytes per request, initial H2D bytes, and steady-state transfers for both variants. If R requests reuse the model, amortize initial load as T_{load}/R. State the expected break-even request count at which full residency repays its larger initial placement.

Part 4 - Experimental Design and Instrumentation

Clone one initialized model so weights are identical. Warm up kernels, reset peak statistics, and run fixed inputs in the same order. Measure cold start, median and tail request latency,

throughput, peak allocated VRAM, resident parameter bytes, H2D/D2H traffic, and output parity. Include batch-size sweeps.

Part 5 - PyTorch/CUDA Implementation

```
import copy
import time
import torch

def synchronized_inference(run_once, repetitions):
    if torch.cuda.is_available():
        torch.cuda.synchronize()
    started = time.perf_counter()
    output = None
    for unused in range(repetitions):
        output = run_once()
    if torch.cuda.is_available():
        torch.cuda.synchronize()
    elapsed = time.perf_counter() - started
    return output, elapsed

def E06(model, layers, placement_map, inputs, repetitions=20):
    full_model = copy.deepcopy(model).eval().to("cuda:0")
    partial_layers = copy.deepcopy(layers).eval()
    apply_static_layer_map(partial_layers, placement_map)

    def run_full_model():
        cuda_inputs = inputs.to("cuda:0")
        return full_model(cuda_inputs)

    def run_partial_model():
        output, unused_trace = execute_with_layer_map(
            partial_layers, placement_map, inputs)
        return output

    with torch.inference_mode():
        full_output, full_seconds = synchronized_inference(
            run_full_model, repetitions)
        partial_output, partial_seconds = synchronized_inference(
            run_partial_model, repetitions)
    comparison = torch.allclose(
        full_output.cpu(), partial_output.cpu(),
        rtol=1.0e-4, atol=1.0e-5)
    return full_seconds, partial_seconds, comparison
```

Part 6 - Measurement, Validation, and Failure Analysis

The helper functions from E05 must be available when this file is executed. Deep-copying a model requires enough host memory and may not preserve every custom external reference.

Measure each variant in a fresh process for the cleanest peak comparison. A partial model that accidentally recomputes or changes evaluation mode is not a placement comparison.

Part 7 - Engineering Interpretation and Philosophy

Memory optimization is a trade of space for movement and coordination. Partial residency is valuable when full residency is impossible or when VRAM has a more valuable competing use. When everything fits comfortably, deliberate movement must justify why it is better than simplicity.

19.14.8 Exercise 07

Exercise Title: Build a Size-Aware Parameter Placement Policy That Fits the Largest Useful Weight Set Inside a Fixed VRAM Budget

Part 1 - Exercise Contract and Why It Matters

Select a useful subset of parameter groups without exceeding a strict byte budget. Do not maximize bytes occupied; maximize declared utility subject to capacity, alias, protection, and indivisible-group constraints.

Part 2 - System and Memory Mental Model

Parameters should be grouped by the atomic unit required by execution, often a layer's weight and bias plus device buffers. Size is the cost, while usefulness can represent expected reuse, avoided transfer stall, or criticality. Shared parameters appear once even when several groups request them.

Part 3 - Quantitative Prediction Before Execution

This is a budgeted selection problem. A greedy benefit-per-byte ranking is interpretable but not always globally optimal. Predict selected groups, unique resident bytes, unused budget, and rejected utility. Compare greedy results with an exact small knapsack instance to expose the approximation.

Part 4 - Experimental Design and Instrumentation

Test equal-size, highly variable-size, shared-weight, and adversarial catalogues. Sweep budget from one group to full residency. Log group bytes, utility, density, aliases, selection order, cumulative unique bytes, and realized runtime benefit. Assert the budget after every admission.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class ParameterGroupCandidate:
    name: str
    parameter_ids: tuple
    size_by_parameter: dict
    utility: float

def select_size_aware_groups(candidates, budget_bytes):
    ranked = []
```

```
    for candidate in candidates:
        group_bytes = 0
        for parameter_id in candidate.parameter_ids:
            group_bytes += candidate.size_by_parameter[parameter_id]
        density = candidate.utility / max(1, group_bytes)
        ranked.append((density, candidate))
    def density_value(item):
        return item[0]

    ranked.sort(key=density_value, reverse=True)

    selected = []
    resident_ids = set()
    used_bytes = 0
    for density, candidate in ranked:
        incremental_bytes = 0
        for parameter_id in candidate.parameter_ids:
            if parameter_id not in resident_ids:
                incremental_bytes += candidate.size_by_parameter[parameter_id]
        if used_bytes + incremental_bytes > budget_bytes:
            continue
        selected.append((candidate.name, density, incremental_bytes))
        used_bytes += incremental_bytes
        for parameter_id in candidate.parameter_ids:
            resident_ids.add(parameter_id)
        assert used_bytes <= budget_bytes
    return selected, used_bytes, resident_ids

def E07(candidates, budget_bytes):
    return select_size_aware_groups(candidates, budget_bytes)
```

Part 6 - Measurement, Validation, and Failure Analysis

Utility must be measured in a meaningful common unit. A density policy can fill VRAM with many small marginal groups while excluding one large critical group. Incremental bytes correctly account for sharing, but it also means group order changes cost. Reserve space for activations and pending transfers outside the parameter budget.

Part 7 - Engineering Interpretation and Philosophy

Capacity is not a target to fill; it is a constraint beneath a workload goal. The best resident set is the one that removes the most important bottleneck, not necessarily the set that leaves the fewest unused bytes.

19.14.9 Exercise 08

Exercise Title: Build a Reuse-Aware Placement Policy That Keeps Frequently Reused Parameters Resident on the GPU

Part 1 - Exercise Contract and Why It Matters

Track layer accesses and keep parameter groups resident when their predicted future reuse can repay promotion. Adapt to phase changes without allowing old popularity to occupy VRAM forever.

Part 2 - System and Memory Mental Model

Parameters in ordinary feed-forward inference are often used once per request but reused across requests. Recurrence, weight tying, mixture-of-experts routing, and iterative decoding create different reuse patterns. Frequency, recency, and reuse distance describe distinct aspects of this demand.

Part 3 - Quantitative Prediction Before Execution

Maintain a decayed score $S_t = \alpha S_{t-1} + 1$ on access and decay absent groups by phase. Promote only when expected avoided reloads times measured reload cost exceed admission and possible victim costs. Predict useful hits after each promotion and wasted bytes for promotions never reused.

Part 4 - Experimental Design and Instrumentation

Replay stable-hot, one-time burst, alternating-expert, and phase-shift traces. Compare cumulative count, sliding window, and exponential decay. Log score, last use, promotion threshold crossing, residence time, hits after promotion, eviction, transfer bytes, and regret against a future-aware oracle.

Part 5 - PyTorch/CUDA Implementation

```
class ReuseAwareParameterPolicy:
    def __init__(self, promote_score=3.0, demote_score=0.8,
                 decay=0.90):
        if demote_score >= promote_score:
            raise ValueError("demotion threshold must be lower")
        self.promote_score = promote_score
        self.demote_score = demote_score
        self.decay = decay
        self.scores = {}
        self.last_access = {}
        self.step = 0

    def begin_step(self):
        self.step += 1
        keys = list(self.scores.keys())
        for key in keys:
            self.scores[key] *= self.decay

    def observe(self, group_name):
        old_score = self.scores.get(group_name, 0.0)
        self.scores[group_name] = old_score + 1.0
        self.last_access[group_name] = self.step

    def action(self, group_name, currently_resident):
        score = self.scores.get(group_name, 0.0)
```

```
        if not currently_resident and score >= self.promote_score:
            return "promote", score
        if currently_resident and score <= self.demote_score:
            return "demote", score
        return "keep_state", score

def E08():
    return ReuseAwareParameterPolicy()
```

Part 6 - Measurement, Validation, and Failure Analysis

Thresholds decide preference, not legality; hard fit and in-flight protection still apply. Decaying every registered group each step can be expensive for huge models, so production systems can compute lazy decay from timestamps. Evaluate whether delayed promotion misses the first expensive uses and whether slow decay causes phase-change thrashing.

Part 7 - Engineering Interpretation and Philosophy

Hotness belongs to an execution context, not permanently to a weight. A reuse-aware engine treats residency as a renewable lease: continued useful demand earns renewal, while stale history gradually loses authority over scarce VRAM.

19.14.10 Exercise 09

Exercise Title: Estimate Parameter Reload Cost from Tensor Bytes and Measured CPU-to-GPU Bandwidth Before Making a Placement Decision

Part 1 - Exercise Contract and Why It Matters

Calibrate a reload-cost model from completed H2D measurements and use it before promotion or eviction. Separate fixed latency from byte-dependent bandwidth so small and large parameter groups are not judged by the same approximation.

Part 2 - System and Memory Mental Model

Copy time includes submission overhead, interconnect transfer, possible host staging, contention, and the dependency imposed on computation. Pinned sources support asynchronous DMA, but an event must mark actual completion. A copy hidden behind compute has lower critical-path cost than an equally long blocking reload.

Part 3 - Quantitative Prediction Before Execution

Fit $T(M) = L + M/B$ from multiple pinned-buffer sizes. For a parameter group, sum unique bytes and estimate reload time plus a contention factor. Keep it resident when expected critical-path reload cost exceeds its residence cost and the group fits after safety reserves.

Part 4 - Experimental Design and Instrumentation

Warm up CUDA, reuse buffers, and measure many sizes with CUDA events. Report median and tail completion time and effective GB/s. Calibrate under idle and compute-contended conditions. Validate predicted group reload times against real parameter-shaped copies not used during fitting.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class ReloadCostModel:
    latency_ms: float
    bandwidth_bytes_per_ms: float
    contention_multiplier: float = 1.0

    def predict_ms(self, size_bytes, critical_fraction=1.0):
        transfer_ms = self.latency_ms
        transfer_ms += size_bytes / self.bandwidth_bytes_per_ms
        transfer_ms *= self.contention_multiplier
        return transfer_ms * critical_fraction

    def should_keep(self, size_bytes, expected_reloads,
                    residence_cost_ms, available_bytes):
        if size_bytes > available_bytes:
            return False, "does_not_fit", 0.0
        reload_cost = expected_reloads * self.predict_ms(size_bytes)
        saving = reload_cost - residence_cost_ms
        if saving > 0.0:
            return True, "avoids_expensive_reloads", saving
        return False, "residence_cost_dominates", saving

def E09(latency_ms, measured_GB_per_second):
    bytes_per_ms = measured_GB_per_second * 1000.0 * 1000.0
    return ReloadCostModel(latency_ms, bytes_per_ms)
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not derive bandwidth from CPU submission time. Synchronize the stop event and keep pinned allocation outside measured loops. A single linear model may fail across transfer-size regimes or on systems with shared bandwidth. Record prediction residuals and recalibrate when hardware, dtype, or concurrency changes.

Part 7 - Engineering Interpretation and Philosophy

Bytes become a placement decision only after hardware converts them into time. Calibration grounds policy in the machine actually running the workload. The model's purpose is not elegant prediction; it is reducing costly wrong moves.

19.14.11 Exercise 10

Exercise Title: Implement On-Demand Parameter Promotion from CPU to GPU Immediately Before the Owning Layer Executes

Part 1 - Exercise Contract and Why It Matters

Promote one layer's parameter group only when execution reaches that layer. Preserve a pinned CPU master, reserve VRAM before allocation, and publish CUDA storage only after its copy has completed. The first version targets inference.

Part 2 - System and Memory Mental Model

Promotion has prepare, submit, ready, and commit phases. Prepare checks budget and pins or reuses the CPU source. Submit allocates CUDA destination and queues H2D on a transfer stream. Ready is represented by an event. Commit changes the Parameter's active data storage, after which the owning layer and its input must execute on the same CUDA device.

Part 3 - Quantitative Prediction Before Execution

For each layer predict unique parameter bytes, promotion start, ready time, and stall seen by the compute stream. Without prefetch, almost all H2D completion time lies on the critical path. Peak managed VRAM equals current residents plus the group being admitted and any pending groups.

Part 4 - Experimental Design and Instrumentation

Run a layer twice: the first access should promote, and the second should hit the resident group. Log source bytes, pending bytes, event timestamps, compute wait, resident bytes, checksum, and object identity. Inject insufficient budget and missing pinned-source cases and verify clean rejection before mutation.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class OnDemandLayerPromoter:
    def __init__(self, budget_bytes):
        self.budget = budget_bytes
        self.used = 0
        self.cpu_master = {}
        self.ready_events = {}
        self.completion_events = {}
        self.stream = torch.cuda.Stream()

    def register(self, layer_name, layer):
        entries = []
        for name, parameter in layer.named_parameters(recurse=False):
            source = parameter.detach().cpu()
            if not source.is_pinned():
                source = source.pin_memory()
            entries.append((name, parameter, source))
        self.cpu_master[layer_name] = entries

    def promote(self, layer_name):
        if layer_name in self.ready_events:
            return "resident"
        entries = self.cpu_master[layer_name]
        required = 0
        for unused_name, parameter, unused_source in entries:
            required += parameter.numel() * parameter.element_size()
```

```
        if self.used + required > self.budget:
            raise RuntimeError("promotion exceeds parameter budget")

        destinations = []
        with torch.cuda.stream(self.stream):
            for name, parameter, source in entries:
                destination = torch.empty_like(source, device="cuda")
                destination.copy_(source, non_blocking=True)
                destinations.append((name, parameter, destination))
            event = torch.cuda.Event()
            event.record(self.stream)
        event.synchronize()
        with torch.no_grad():
            for unused_name, parameter, destination in destinations:
                parameter.data = destination
        self.ready_events[layer_name] = event
        self.used += required
        return "promoted"

    def record_use_completion(self, layer_name, stream):
        event = torch.cuda.Event()
        event.record(stream)
        self.completion_events[layer_name] = event
        return event

def E10(layer_name, layer, budget_bytes):
    promoter = OnDemandLayerPromoter(budget_bytes)
    promoter.register(layer_name, layer)
    promoter.promote(layer_name)
    return promoter
```

Part 6 - Measurement, Validation, and Failure Analysis

Synchronizing makes commit easy to audit but prevents overlap; E16 removes that stall with dependencies. Recurse carefully: direct parameters omit nested child layers. Buffers must be placed too when the operation uses them. Movement after forward is unsafe for ordinary training backward, and optimizer state remains a separate placement problem.

Part 7 - Engineering Interpretation and Philosophy

On-demand promotion trades certainty for capacity. It is the simplest policy that allows a model larger than VRAM to make progress, but it exposes the full cost of arriving late. This baseline gives prefetching an honest opponent.

19.14.12 Exercise 11

Exercise Title: Demote Parameters Back to CPU After Their Final GPU Use and Measure the Resulting Peak VRAM Reduction

Part 1 - Exercise Contract and Why It Matters

Use a known layer schedule to demote each parameter group after its final GPU consumer. Clean inference weights can return to their existing CPU master without D2H; dirty trained weights require a completed write-back.

Part 2 - System and Memory Mental Model

Final planned use and final completed use are different. The compute stream records an event after the layer. Only after that event may clean CUDA storage be released or dirty storage be copied. Assigning a CPU master back to a Parameter changes its active data tier while keeping module registration and object identity.

Part 3 - Quantitative Prediction Before Execution

From the execution plan calculate each group's last-use index. Predict the resident-byte timeline with and without prompt demotion, then compare the peaks. For a clean single-pass model, D2H traffic should be zero; for dirty groups, write-back bytes equal unique modified parameter bytes.

Part 4 - Experimental Design and Instrumentation

Run a deep sequential model and record allocated, reserved, and managed bytes after every layer. Compare never-demote, immediate-final-use demotion, and a delayed policy. Verify checksums after CPU restoration and allocate a same-sized temporary to expose premature allocator reuse.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class FinalUseDemoter:
    def __init__(self, cpu_master):
        self.cpu_master = cpu_master
        self.last_use = {}
        self.completion = {}

    def set_last_use(self, group_name, step):
        self.last_use[group_name] = step

    def record_use_completion(self, group_name, stream):
        event = torch.cuda.Event()
        event.record(stream)
        self.completion[group_name] = event

    def demote_if_final(self, group_name, step, parameters,
                        dirty=False):
        if self.last_use[group_name] != step:
            return 0
        event = self.completion[group_name]
        event.synchronize()
        released_bytes = 0
        with torch.no_grad():
            for name, parameter in parameters:
```

```
                released_bytes += (
                    parameter.numel() * parameter.element_size())
                master = self.cpu_master[(group_name, name)]
                if dirty:
                    master.copy_(parameter.detach().cpu())
                parameter.data = master
        return released_bytes

def E11(cpu_master):
    return FinalUseDemoter(cpu_master)
```

Part 6 - Measurement, Validation, and Failure Analysis

The dirty path shown is intentionally blocking and creates a temporary CPU copy; a pinned asynchronous write-back is built later. For tied weights, final use is the last use across all owners, not one layer's local last use. Releasing live allocated memory may not reduce reserved memory because the caching allocator keeps reusable blocks.

Part 7 - Engineering Interpretation and Philosophy

Memory is saved by proving the future, not by deleting aggressively. A schedule turns "probably unused" into a final-use guarantee. Correct lifetime knowledge often provides a larger and safer gain than a more complicated eviction score.

19.14.13 Exercise 12

Exercise Title: Use PyTorch Module Forward Hooks to Trigger Parameter Placement Automatically Without Manually Editing Every Forward Method

Part 1 - Exercise Contract and Why It Matters

Attach pre-forward and post-forward hooks that acquire a module's parameter group before execution and record or perform final-use release afterward. Keep model source code unchanged while making hook ordering and cleanup explicit.

Part 2 - System and Memory Mental Model

A pre-hook runs immediately before a module's `forward`; it can ensure weights and inputs share a device. A post-hook runs after forward submission, not after CUDA kernels physically finish, so it must record an event instead of assuming storage is free. Hooks are control-plane callbacks and should not silently mutate unrelated modules.

Part 3 - Quantitative Prediction Before Execution

Predict hook invocation order from the module graph, including repeated module use. Count placement calls, cache hits, and final-use events. Measure hook CPU overhead separately from transfer and kernel time. A reused module must not be demoted after its first invocation if the schedule contains another use.

Part 4 - Experimental Design and Instrumentation

Attach hooks to leaf compute modules only. Test nested Sequential, residual reuse, an exception inside forward, and repeated inference. Log module name, call index, device, promotion

result, output device, event, and cleanup. Remove all handles after the experiment and verify a second run has no duplicate hooks.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class PlacementHooks:
    def __init__(self, promoter, final_use_by_module):
        self.promoter = promoter
        self.final_use = final_use_by_module
        self.call_count = {}
        self.handles = []

    def attach(self, named_modules):
        for module_name, module in named_modules:
            self.call_count[module_name] = 0

            def pre_hook(current_module, inputs,
                         name=module_name):
                self.promoter.promote(name)
                self.call_count[name] += 1
                target = next(current_module.parameters()).device
                moved_inputs = []
                for value in inputs:
                    if torch.is_tensor(value):
                        moved_inputs.append(value.to(target))
                    else:
                        moved_inputs.append(value)
                return tuple(moved_inputs)

            def post_hook(current_module, inputs, output,
                          name=module_name):
                stream = torch.cuda.current_stream()
                self.promoter.record_use_completion(name, stream)
                return output

            self.handles.append(
                module.register_forward_pre_hook(pre_hook))
            self.handles.append(
                module.register_forward_hook(post_hook))

    def remove(self):
        for handle in self.handles:
            handle.remove()
        self.handles.clear()

def E12(promoter, final_use_by_module):
    return PlacementHooks(promoter, final_use_by_module)
```

Part 6 - Measurement, Validation, and Failure Analysis

Python closures must capture the current module name explicitly, as shown by the default argument. Modules without direct parameters need a target supplied by the registry instead of `next()`. Hook-based mutation can surprise model compilers, tracing, reentrant forwards, and thread-concurrent inference. Treat hook handles as resources and always remove them.

Part 7 - Engineering Interpretation and Philosophy

Hooks separate policy from model mathematics, but hidden control has a debugging cost. Automation is valuable only when every callback is observable, ordered, and reversible. A placement hook should make execution easier to inspect, not magical.

19.14.14 Exercise 13

Exercise Title: Reproduce a Device-Mismatch Failure Caused by Incorrect Parameter Placement and Build Automatic Device-Consistency Validation

Part 1 - Exercise Contract and Why It Matters

Intentionally place an input, weight, bias, or buffer on the wrong device, capture the failure, and build a validator that reports the mismatch before the operator launches. The diagnostic must name the owning module and every conflicting tensor category.

Part 2 - System and Memory Mental Model

Most CUDA operators require participating dense tensors on one device. Moving a weight alone can leave its bias, running statistics, input, or cached mask on CPU. The resulting runtime error is a symptom; the root cause is an incomplete placement group or missing boundary transfer.

Part 3 - Quantitative Prediction Before Execution

Before running, enumerate expected devices for parameters, buffers, positional inputs, keyword tensors, and relevant optimizer state. Predict exactly which pair conflicts. The validator's cost is proportional to metadata inspected and should be measured in debug and production-disabled modes.

Part 4 - Experimental Design and Instrumentation

Create separate failures for weight, bias, BatchNorm buffer, nested input, and stale gradient. Verify the validator fails before forward and includes qualified names. Include a valid multi-device boundary where input is intentionally moved between layers so the checker does not reject legal partitioning.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def tensor_devices(values):
    devices = []
    for value in values:
        if torch.is_tensor(value):
            devices.append(str(value.device))
```

```
    return devices

def validate_module_device(module_name, module, inputs):
    facts = []
    for name, parameter in module.named_parameters(recurse=False):
        facts.append(("parameter", name, str(parameter.device)))
        if parameter.grad is not None:
            facts.append(("gradient", name, str(parameter.grad.device)))
    for name, buffer in module.named_buffers(recurse=False):
        facts.append(("buffer", name, str(buffer.device)))
    for index, device in enumerate(tensor_devices(inputs)):
        facts.append(("input", str(index), device))

    unique_devices = set()
    for unused_kind, unused_name, device in facts:
        unique_devices.add(device)
    if len(unique_devices) > 1:
        message = "device mismatch in " + module_name + ": "
        message += str(facts)
        raise RuntimeError(message)
    return facts

def E13(module_name, module, inputs):
    return validate_module_device(module_name, module, inputs)
```

Part 6 - Measurement, Validation, and Failure Analysis

Some operations legally accept CPU metadata or indices with CUDA tensors, so a universal same-device rule can be too strict. Build operator-specific exceptions only after evidence. Nested containers need recursive traversal. A validator detects state inconsistency; it should not silently move tensors and conceal the policy error.

Part 7 - Engineering Interpretation and Philosophy

Failing early transforms a distant CUDA exception into a local placement explanation. Diagnostics are part of the memory system, not optional polish: dynamic behavior is maintainable only when illegal states identify themselves before computation corrupts or stalls.

19.14.15 Exercise 14

Exercise Title: Preserve Shared and Tied Parameters During Dynamic Placement Without Creating Duplicate GPU Copies

Part 1 - Exercise Contract and Why It Matters

Discover all names that refer to the same Parameter, promote that identity once, and ensure every owner observes the same CUDA storage. Final use and dirty state must be aggregated across all owners.

Part 2 - System and Memory Mental Model

Weight tying is semantic sharing, not coincidental equality. Replacing one owner with an independent CUDA copy breaks tying and doubles memory. Canonical identity can be the Parameter object ID for ordinary tying; models that create distinct Parameter objects over shared storage require storage-level identity and compatible view reconstruction.

Part 3 - Quantitative Prediction Before Execution

Calculate the naive by-name bytes and canonical unique bytes. Predict exactly one H2D transfer and one resident allocation for each tied group. The final-use index is the maximum use across all owner paths. Compare duplication ratio M_{naive}/M_{unique} before execution.

Part 4 - Experimental Design and Instrumentation

Tie an output projection to an embedding, inventory both names, promote through each owner, and compare object and storage identities. Mutate under a controlled no-grad test and prove both owners see the update. Verify one demotion after the last combined use and no duplicate transfer.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class SharedParameterRegistry:
    def __init__(self, model):
        self.by_id = {}
        self.name_to_id = {}
        for name, parameter in model.named_parameters(
                remove_duplicate=False):
            canonical_id = id(parameter)
            self.name_to_id[name] = canonical_id
            if canonical_id not in self.by_id:
                self.by_id[canonical_id] = {
                    "parameter": parameter,
                    "names": [],
                    "gpu_copy": None,
                    "last_use": -1,
                }
            self.by_id[canonical_id]["names"].append(name)

    def set_owner_last_use(self, name, step):
        canonical_id = self.name_to_id[name]
        record = self.by_id[canonical_id]
        record["last_use"] = max(record["last_use"], step)

    def install_gpu_copy(self, name, gpu_tensor):
        canonical_id = self.name_to_id[name]
        record = self.by_id[canonical_id]
        if record["gpu_copy"] is not None:
            return record["gpu_copy"], False
        parameter = record["parameter"]
        with torch.no_grad():
```

```
            parameter.data = gpu_tensor
        record["gpu_copy"] = gpu_tensor
        return gpu_tensor, True

def E14(model):
    return SharedParameterRegistry(model)
```

Part 6 - Measurement, Validation, and Failure Analysis

The registry intentionally retains Parameter objects because it is an active control plane, not exported telemetry; its lifetime must not outlive the model. Object identity misses distinct view objects over one storage. Installing a copy must happen only after transfer completion and under a no-grad inference-safe boundary. Optimizer references must still point to the canonical object.

Part 7 - Engineering Interpretation and Philosophy

Sharing is part of the model's mathematics. A memory optimization that preserves values but destroys alias relationships has changed the program. Resource identity must therefore follow semantic identity, not merely qualified names.

19.14.16 Exercise 15

Exercise Title: Move Parameter Groups Asynchronously Through Pinned CPU Memory and a Dedicated CUDA Transfer Stream

Part 1 - Exercise Contract and Why It Matters

Submit a complete layer parameter group from pinned CPU masters to CUDA without blocking the host, preserve all source lifetimes, and make the compute stream wait only for that group's ready event. The group becomes one migration transaction with an explicit commit boundary.

Part 2 - System and Memory Mental Model

Pinned memory enables direct asynchronous H2D DMA. `non_blocking=True` only changes submission semantics; it does not prove completion. A transaction owns pinned sources, CUDA destinations, the transfer stream, and a ready event. Binding destinations to Parameters and ordering the consumer are separate operations. Every destination is recorded on its consumer stream for allocator lifetime safety.

Part 3 - Quantitative Prediction Before Execution

Sum unique bytes in the group and predict (L+M/B), CPU submission time, pending VRAM, and pinned RAM. One large grouped transfer often uses bandwidth more efficiently than many tiny independent submissions, although PyTorch still queues one copy per tensor in this implementation.

Part 4 - Experimental Design and Instrumentation

Compare pageable blocking, freshly pinned, reused pinned, and asynchronous group movement. Measure CPU submission, CUDA completion, effective GB/s, allocation overhead, and checksum after ready. Stress with hundreds of small parameters and a few large matrices.

Mutate a source before completion in an isolated negative test to demonstrate the lifetime hazard.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass
import torch

@dataclass
class ParameterGroupTransfer:
    group_name: str
    sources: list
    destinations: list
    bindings: list
    ready: object
    transfer_stream: object
    state: str = "pending"

    def bind_for_compute(self, compute_stream):
        compute_stream.wait_event(self.ready)
        with torch.no_grad():
            for parameter, destination in self.bindings:
                parameter.data = destination
                destination.record_stream(compute_stream)
        self.state = "bound"

    def completed(self):
        return self.ready.query()

class AsyncParameterMover:
    def __init__(self):
        self.transfer_stream = torch.cuda.Stream()

    def submit(self, group_name, entries):
        sources = []
        destinations = []
        bindings = []
        with torch.cuda.stream(self.transfer_stream):
            for parameter, cpu_master in entries:
                if not cpu_master.is_pinned():
                    raise ValueError("CPU master must be pinned")
                destination = torch.empty_like(
                    cpu_master, device="cuda")
                destination.copy_(cpu_master, non_blocking=True)
                sources.append(cpu_master)
                destinations.append(destination)
                bindings.append((parameter, destination))
            ready = torch.cuda.Event()
            ready.record(self.transfer_stream)
```

```
        return ParameterGroupTransfer(
            group_name, sources, destinations, bindings,
            ready, self.transfer_stream)

def E15():
    return AsyncParameterMover()
```

Part 6 - Measurement, Validation, and Failure Analysis

The transaction retains pinned sources until Python releases it. Binding before copy completion is safe only because the compute stream first waits on the ready event; CPU inspection of destinations is irrelevant because they are CUDA storage. Pending destination bytes must be reserved in the budget at submission, not at completion. Exceptions must roll back accounting and references.

Part 7 - Engineering Interpretation and Philosophy

Asynchronous movement is a protocol, not a faster spelling of `to()`. Performance comes from making useful host and GPU work possible during transfer; correctness comes from retaining ownership and expressing the precise event on which the next consumer depends.

19.14.17 Exercise 16

Exercise Title: Prefetch the Next Layer's Parameters While the Current Layer Computes and Measure How Much Transfer Latency Is Hidden

Part 1 - Exercise Contract and Why It Matters

Build a one-layer lookahead pipeline: while layer i computes, submit the parameters of layer $i + 1$ on a dedicated transfer stream. Measure overlap on the CUDA timeline and distinguish hidden transfer time from mere asynchronous submission.

Part 2 - System and Memory Mental Model

True overlap requires pinned sources, non-blocking copies, separate streams, hardware capability, and no dependency forcing serialization. The compute stream waits on a group's ready event only when execution reaches that layer. Prefetch increases peak VRAM because current and next groups coexist, so admission must reserve both.

Part 3 - Quantitative Prediction Before Execution

If current compute takes C_i and next transfer takes H_{i+1}, ideal visible stall is approximately $\max(0, H_{i+1} - C_i)$. Maximum speedup is bounded by replacing $C_i + H_{i+1}$ with $\max(C_i, H_{i+1})$, plus launch and event overhead. Predict which layers fully hide transfer and which remain transfer bound.

Part 4 - Experimental Design and Instrumentation

Compare on-demand serial loading with one-layer lookahead over small, medium, and large layers. Record transfer start/stop events, kernel ranges, overlap duration, visible wait, peak allocated VRAM, pending bytes, and end-to-end latency. Use a profiler timeline to verify overlap rather than inferring it only from wall time.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class OneLayerPrefetchPipeline:
    def __init__(self, mover, groups):
        self.mover = mover
        self.groups = groups
        self.pending = {}
        self.compute_stream = torch.cuda.current_stream()

    def prefetch(self, index):
        if index >= len(self.groups):
            return
        if index in self.pending:
            return
        group_name, entries, unused_layer = self.groups[index]
        self.pending[index] = self.mover.submit(
            group_name, entries)

    def run(self, activation):
        self.prefetch(0)
        for index in range(len(self.groups)):
            transaction = self.pending.pop(index)
            transaction.bind_for_compute(self.compute_stream)
            self.prefetch(index + 1)
            unused_name, unused_entries, layer = self.groups[index]
            target = next(layer.parameters()).device
            if activation.device != target:
                activation = activation.to(
                    target, non_blocking=True)
            activation = layer(activation)
        return activation

def E16(mover, groups):
    return OneLayerPrefetchPipeline(mover, groups)
```

Part 6 - Measurement, Validation, and Failure Analysis

This minimal pipeline does not demote old groups; integrate final-use release when measuring a strict budget. A transfer submitted on another stream can still serialize because of memory pressure, copy-engine limits, or hidden default- stream dependencies. Prefetch distance greater than one may hide more latency but can trigger OOM or evict useful current weights.

Part 7 - Engineering Interpretation and Philosophy

Prefetching converts a prediction about future execution into early physical work. Its value is bounded by the computation available to cover the copy. The right goal is not maximum lookahead; it is the smallest lookahead that keeps the critical path supplied without destabilizing memory.

19.14.18 Exercise 17

Exercise Title: Build a VRAM Watermark Controller That Dynamically Promotes and Demotes Parameter Groups Before an OOM Occurs

Part 1 - Exercise Contract and Why It Matters

Protect the placement engine from last-second allocation failure using a hard budget and two softer watermarks. Admission reserves pending bytes before CUDA allocation; pressure mode demotes legal cold groups until recovery headroom is restored.

Part 2 - System and Memory Mental Model

Managed bytes include committed residents and submitted destinations. High watermark crossing enters pressure; demotion continues until the low watermark. Hysteresis prevents repeated promotion and demotion around one threshold. Protected, shared-in-use, dirty-pending, and consumer-in-flight groups are not immediately reclaimable.

Part 3 - Quantitative Prediction Before Execution

For capacity C, choose $L < H < C$. Before admitting group g, require $R + P + M_g \leq C$, where R and P are resident and pending bytes. If the future total exceeds H, select legal victims until the projected total is at most L. Predict victim bytes, write-back traffic, and overshoot.

Part 4 - Experimental Design and Instrumentation

Replay monotonic growth, noisy boundary, sudden large group, and all-protected traces. Log resident, pending, protected, reclaimable, high/low transitions, victim reasons, rejected admissions, and distance from physical free memory. Assert the hard managed budget after every transition.

Part 5 - PyTorch/CUDA Implementation

```
class ParameterWatermarks:
    def __init__(self, capacity_bytes, low_ratio=0.70,
                 high_ratio=0.85):
        self.capacity = capacity_bytes
        self.low = int(capacity_bytes * low_ratio)
        self.high = int(capacity_bytes * high_ratio)
        if not 0 <= self.low < self.high < self.capacity:
            raise ValueError("invalid watermarks")
        self.resident = 0
        self.pending = 0
        self.pressure = False

    def reserve_pending(self, requested_bytes):
        projected = self.resident + self.pending + requested_bytes
        if projected > self.capacity:
            return False, projected - self.low
        if projected >= self.high:
            self.pressure = True
            return False, projected - self.low
```

```
        self.pending += requested_bytes
        return True, 0

    def commit_pending(self, size_bytes):
        self.pending -= size_bytes
        self.resident += size_bytes
        assert self.resident + self.pending <= self.capacity

    def release_resident(self, size_bytes):
        self.resident -= size_bytes
        if self.resident + self.pending <= self.low:
            self.pressure = False
        assert self.resident >= 0

def E17(capacity_bytes):
    return ParameterWatermarks(capacity_bytes)
```

Part 6 - Measurement, Validation, and Failure Analysis

A rejected reservation returns the recovery target; the policy must demote and retry once, not loop forever. Managed capacity must leave a measured reserve for activations, workspaces, CUDA context, and fragmentation. Device free memory and PyTorch reserved memory provide diagnostic signals but should not silently replace one consistent accounting definition mid-run.

Part 7 - Engineering Interpretation and Philosophy

OOM prevention is admission control, not exception handling. Watermarks buy time to choose legal victims and stabilize decisions. A system that waits for CUDA to reject an allocation has already surrendered control of its most important resource invariant.

19.14.19 Exercise 18

Exercise Title: Stress-Test Parameter Placement on a Deep PyTorch Model with Artificially Restricted VRAM and Diagnose Thrashing, Duplicate Copies, and Transfer Stalls

Part 1 - Exercise Contract and Why It Matters

Drive the placement engine with a model whose parameter payload exceeds an artificial managed budget while remaining safely below physical device memory. Find invariant failures and performance collapse across variable group sizes, reuse patterns, sharing, and prefetch distances.

Part 2 - System and Memory Mental Model

Artificial restriction makes pressure deterministic without deliberately exhausting the device. Thrashing means a group is evicted and reloaded before its movement earns useful computation. Duplicate copies arise when aliases are not canonicalized. Transfer stalls appear when ready events reach the compute critical path or when excessive prefetching competes for bandwidth and VRAM.

Part 3 - Quantitative Prediction Before Execution

Before execution, simulate the exact layer trace and predict managed peak, H2D bytes, evictions, reloads, and duplicate-free unique bytes. Define failure thresholds: zero wrong outputs, zero budget violations, one CUDA copy per canonical resident group, and bounded migrations per layer execution.

Part 4 - Experimental Design and Instrumentation

Generate deep sequential, reused-block, tied-weight, and routed-expert traces. Sweep budgets and prefetch distance over fixed seeds. Log every transition, canonical ID, resident duration, useful uses, H2D/D2H bytes, event wait, transfer-compute overlap, decision time, peak allocated/reserved VRAM, and latency percentiles. Minimize any failing trace.

Part 5 - PyTorch/CUDA Implementation

```
from collections import OrderedDict

class ParameterPlacementStressModel:
    def __init__(self, group_sizes, budget_bytes):
        self.group_sizes = group_sizes
        self.budget = budget_bytes
        self.resident = OrderedDict()
        self.used = 0
        self.loads = 0
        self.evictions = 0
        self.reloads = 0
        self.ever_loaded = set()

    def access(self, canonical_group):
        if canonical_group in self.resident:
            self.resident.move_to_end(canonical_group)
            return "hit"
        size_bytes = self.group_sizes[canonical_group]
        if size_bytes > self.budget:
            return "bypass_oversized"
        while self.used + size_bytes > self.budget:
            victim, unused = self.resident.popitem(last=False)
            self.used -= self.group_sizes[victim]
            self.evictions += 1
        if canonical_group in self.ever_loaded:
            self.reloads += 1
        self.ever_loaded.add(canonical_group)
        self.resident[canonical_group] = True
        self.used += size_bytes
        self.loads += 1
        assert self.used <= self.budget
        return "load"

def E18(group_sizes, trace, budget_bytes):
    model = ParameterPlacementStressModel(group_sizes, budget_bytes)
    outcomes = []
```

```
    for canonical_group in trace:
        outcomes.append(model.access(canonical_group))
    return model, outcomes
```

Part 6 - Measurement, Validation, and Failure Analysis

The simulator isolates policy behavior; replay the same canonical trace through the real PyTorch engine afterward. A miss-heavy streaming workload is not automatically thrashing because it may have no reuse. Diagnose thrashing only when short residence is followed by reload. Compare storage IDs to detect real duplication, and inspect profiler timelines for stalls.

Part 7 - Engineering Interpretation and Philosophy

Stress tests are experiments against assumptions. Deep models expose mistakes that small examples cannot: long ownership chains, phase changes, aliasing, and pressure interactions. A placement engine is ready only when its worst cases are reproducible, measurable, and explainable.

19.14.20 Exercise 19

Exercise Title: Build and Benchmark a Dynamic PyTorch Parameter Placement Engine That Executes a Model Under a VRAM Budget Smaller Than Its Total Parameter Size

Part 1 - Exercise Contract and Why It Matters

Integrate canonical parameter identity, pinned CPU masters, strict byte admission, asynchronous promotion, lookahead, final-use demotion, event ordering, watermarks, and diagnostics. Execute a complete inference model whose total parameter payload is larger than the managed VRAM budget without violating numerical correctness.

Part 2 - System and Memory Mental Model

The engine is a parameter memory operating system. Each canonical group has CPU masters, optional CUDA destinations, authority, dirty state, pending transaction, ready event, consumer-completion event, protection reasons, access history, and owner modules. Admission counts pending bytes before allocation. Compute waits on ready; demotion waits on all consumers; tied owners share one allocation.

The first integrated target is inference because training adds gradients, optimizer state, and backward saved-tensor lifetimes. A production training engine must extend the same state machine rather than pretending weights can be moved independently after forward.

Part 3 - Quantitative Prediction Before Execution

Choose managed capacity C below total unique parameter bytes and below physical free VRAM after explicit activation/workspace reserve. Simulate the trace to predict peak residents, pending peak, H2D bytes, reloads, evictions, visible transfer stall, and ideal overlap. Acceptance requires $R + P \leq C$ at every transition, exact canonical sharing, and output parity.

Part 4 - Experimental Design and Instrumentation

Compare all-CPU, whole-CUDA when it fits on a reference device, static split, on-demand dynamic, and prefetched dynamic execution. Use identical weights, inputs, precision, and request order. Warm up, synchronize boundaries, repeat, and report cold start, median/tail

latency, throughput, peak allocated/reserved VRAM, managed peak, pinned RAM, traffic, overlap, policy overhead, and checksums.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass, field
from collections import OrderedDict
import torch

@dataclass
class ManagedParameterGroup:
    name: str
    entries: list
    size_bytes: int
    transaction: object = None
    resident: bool = False
    ready: object = None
    last_access: int = -1
    uses: int = 0
    protected: set = field(default_factory=set)
    consumer_events: list = field(default_factory=list)

class DynamicParameterEngine:
    def __init__(self, groups, budget_bytes, low_ratio=0.70,
                 high_ratio=0.85):
        self.groups = groups
        self.budget = budget_bytes
        self.low = int(budget_bytes * low_ratio)
        self.high = int(budget_bytes * high_ratio)
        self.resident_bytes = 0
        self.pending_bytes = 0
        self.step = 0
        self.recency = OrderedDict()
        self.mover = AsyncParameterMover()

    def _legal_victim(self, excluded):
        for name in self.recency:
            group = self.groups[name]
            if name == excluded:
                continue
            if group.protected:
                continue
            return group
        return None

    def _demote_blocking(self, group):
        for event in group.consumer_events:
            event.synchronize()
        with torch.no_grad():
```

```
            for parameter, cpu_master in group.entries:
                parameter.data = cpu_master
        group.transaction = None
        group.ready = None
        group.resident = False
        group.consumer_events.clear()
        self.resident_bytes -= group.size_bytes
        self.recency.pop(group.name, None)

    def _make_room(self, requested_bytes, excluded):
        if requested_bytes > self.budget:
            raise RuntimeError("group exceeds managed budget")
        projected = self.resident_bytes
        projected += self.pending_bytes + requested_bytes
        target = self.high
        if requested_bytes > self.high:
            target = self.budget
        while projected > target:
            victim = self._legal_victim(excluded)
            if victim is None:
                raise RuntimeError("no legal victim for admission")
            self._demote_blocking(victim)
            projected = self.resident_bytes
            projected += self.pending_bytes + requested_bytes

    def prefetch(self, name):
        group = self.groups[name]
        if group.resident or group.transaction is not None:
            return "already_available"
        self._make_room(group.size_bytes, name)
        self.pending_bytes += group.size_bytes
        try:
            group.transaction = self.mover.submit(
                name, group.entries)
            group.ready = group.transaction.ready
        except Exception:
            self.pending_bytes -= group.size_bytes
            group.transaction = None
            raise
        return "submitted"

    def acquire(self, name, compute_stream=None):
        self.step += 1
        group = self.groups[name]
        if group.transaction is None and not group.resident:
            self.prefetch(name)
        if compute_stream is None:
            compute_stream = torch.cuda.current_stream()
```

```
        if not group.resident:
            group.transaction.bind_for_compute(compute_stream)
            self.pending_bytes -= group.size_bytes
            self.resident_bytes += group.size_bytes
            group.resident = True
        group.last_access = self.step
        group.uses += 1
        self.recency.pop(name, None)
        self.recency[name] = True
        assert self.resident_bytes + self.pending_bytes <= self.budget

    def finish_use(self, name, compute_stream=None):
        if compute_stream is None:
            compute_stream = torch.cuda.current_stream()
        event = torch.cuda.Event()
        event.record(compute_stream)
        self.groups[name].consumer_events.append(event)

def E19(groups, budget_bytes):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required")
    return DynamicParameterEngine(groups, budget_bytes)
```

Part 6 - Measurement, Validation, and Failure Analysis

This reference engine deliberately blocks during victim completion and supports clean inference weights. Production code should demote asynchronously, bound event-history growth, place buffers, support nested graphs, and use a planned final-use schedule. Training additionally requires authoritative dirty write-back, gradient and optimizer-state placement, and protection until backward consumers finish.

Test transaction rollback, all-protected pressure, tied groups, oversized groups, exceptions inside layers, and repeated requests. Never use `empty_cache()` as a substitute for releasing live Parameter storage, and never count a pending group twice when it commits.

Part 7 - Engineering Interpretation and Philosophy

A model larger than VRAM can execute because not every weight must be resident simultaneously. The achievement is not bypassing capacity; it is scheduling a working set through capacity while preserving identity, ordering, and truth. The final engine is credible when every byte has an owner, every transition has a reason, and every asynchronous use has a completion proof.

19.15 DMM13

Chapter Title: Dynamic Layer Offloading
Specialization: Dynamic Memory Management and CUDA Systems

19.15.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Layer Residency, Execution Order, Transfer Cost, and VRAM Pressure in Deep PyTorch Models

Part 1 - Exercise Contract and Why It Matters

Build a layer-level state machine before implementing offloading. A layer is an execution unit containing parameters, persistent buffers, temporary workspace, and relationships to incoming and outgoing activations. The model must explain not only where weights live but when the whole layer is legally executable.

The contract must answer: which canonical layer or shared module is referenced, which tensors constitute its movable state, which device owns that state, which transfer is pending, which invocation is active, what its next planned use is, and what evidence permits eviction.

Part 2 - System and Memory Mental Model

Useful states begin with CPU residence, H2D transfer, and CUDA readiness. They continue through execution, pending consumers, optional D2H transfer, backward protection, and release. A post-forward Python callback occurs after kernels are submitted, not necessarily after they finish. CUDA events establish completion without forcing unrelated work on the entire device to stop.

Layer state and activation state are distinct. Offloading a completed layer can be safe while its output remains on CUDA for the next layer or a residual path. During training, backward may need the layer's parameters and saved tensors long after forward, so inference-safe eviction can be training-incorrect.

Part 3 - Quantitative Prediction Before Execution

For layer i, define persistent bytes $W_i = P_i + B_i$, where P_i and B_i are unique parameter and buffer bytes. Estimate H2D time as $L + W_i/\beta$. The resident window must also budget live activations, workspace, pending prefetch, and allocator margin.

For prefetch of layer $i+1$ during compute C_i, ideal visible transfer stall is approximately $\max(0, T_{i+1} - C_i)$. Record this prediction before building a deeper pipeline.

Part 4 - Experimental Design and Instrumentation

Use sequential, residual, repeated-module, and tied-weight graphs. At every transition log layer identity, invocation index, parameters, buffers, resident and pending bytes, activation bytes, stream, ready and done events, next use, and reason. Compare managed accounting with allocated and reserved CUDA memory, while keeping their meanings separate.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass, field

@dataclass
class LayerState:
    canonical_name: str
    owner_paths: list
    parameter_bytes: int
    buffer_bytes: int
    state: str = "cpu_resident"
```

```
    next_planned_use: int = -1
    invocation_count: int = 0
    protected_reasons: set = field(default_factory=set)

    @property
    def persistent_bytes(self):
        return self.parameter_bytes + self.buffer_bytes

    def begin_prefetch(self):
        if self.state != "cpu_resident":
            raise RuntimeError("layer is not ready for H2D prefetch")
        self.state = "h2d_pending"

    def mark_ready(self):
        if self.state != "h2d_pending":
            raise RuntimeError("no pending prefetch to complete")
        self.state = "cuda_ready"

    def begin_execution(self):
        if self.state != "cuda_ready":
            raise RuntimeError("layer is not CUDA-ready")
        self.state = "executing"
        self.invocation_count += 1

def E00(name, owner_paths, parameter_bytes, buffer_bytes):
    return LayerState(name, owner_paths,
                      parameter_bytes, buffer_bytes)
```

Part 6 - Measurement, Validation, and Failure Analysis

Module device inspection alone cannot reveal pending transfers, active kernels, shared ownership, or backward protection. Recursive layer byte sums can double count child modules and tied parameters. Separate logical layer ownership from canonical storage accounting, and validate every state transition against CUDA events and numerical outputs.

Part 7 - Engineering Interpretation and Philosophy

Layer offloading is execution scheduling under a memory constraint. The engine does not make a large model smaller; it changes when each part occupies the fast tier. Correctness comes from respecting dependencies, and performance comes from arranging movement early enough that execution rarely waits.

19.15.2 Exercise 01

Exercise Title: Measure the Parameter and Buffer Memory Footprint of Every Layer in a PyTorch Model Before Designing an Offload Strategy

Part 1 - Exercise Contract and Why It Matters

Build a layer inventory that separates direct parameters, direct buffers, and recursive subtree totals. Preserve canonical identities so shared weights and shared module instances are not counted repeatedly in the global budget.

Part 2 - System and Memory Mental Model

Leaf execution layers normally own direct state, while container modules report the sum of descendants. Counting both produces hierarchical double counting. Buffers such as BatchNorm running statistics affect device correctness even when they do not require gradients. Non-persistent buffers still occupy memory while the module exists.

Part 3 - Quantitative Prediction Before Execution

Predict direct parameter bytes and buffer bytes for each layer. Then calculate global unique totals by object or storage identity. Compare the sum of leaf direct bytes with the unique model total; explain any difference using shared state or parameters registered directly on container modules.

Part 4 - Experimental Design and Instrumentation

Inventory a plain Sequential model, BatchNorm-containing model, residual block, and tied-weight model. Record module path, class, direct and recursive counts, dtype, device, trainability, aliases, and whether the module has executable forward logic. Export scalar metadata rather than live tensors.

Part 5 - PyTorch/CUDA Implementation

```
def direct_layer_bytes(module):
    parameter_bytes = 0
    buffer_bytes = 0
    parameter_ids = []
    buffer_ids = []
    for parameter in module.parameters(recurse=False):
        parameter_bytes += parameter.numel() * parameter.element_size()
        parameter_ids.append(id(parameter))
    for buffer in module.buffers(recurse=False):
        buffer_bytes += buffer.numel() * buffer.element_size()
        buffer_ids.append(id(buffer))
    return parameter_bytes, buffer_bytes, parameter_ids, buffer_ids

def E01(model):
    report = []
    global_parameters = set()
    global_buffers = set()
    for name, module in model.named_modules(remove_duplicate=False):
        parameter_bytes, buffer_bytes, parameter_ids, buffer_ids = (
            direct_layer_bytes(module))
        for parameter_id in parameter_ids:
            global_parameters.add(parameter_id)
        for buffer_id in buffer_ids:
            global_buffers.add(buffer_id)
        report.append({
            "name": name,
            "class": module.__class__.__name__,
            "direct_parameter_bytes": parameter_bytes,
```

```
            "direct_buffer_bytes": buffer_bytes,
            "parameter_ids": tuple(parameter_ids),
            "buffer_ids": tuple(buffer_ids),
        })
    return report, global_parameters, global_buffers
```

Part 6 - Measurement, Validation, and Failure Analysis

The sets above reveal unique identities but do not calculate their byte totals; retain a scalar ID-to-bytes table when that total is required. Distinct tensor objects can exceptionally share one storage, so storage-level inspection may be needed. Sparse, quantized, meta, and sharded modules require specialized accounting rather than a dense estimate.

Part 7 - Engineering Interpretation and Philosophy

Offload granularity should follow measured state, not architectural appearance. Two layers that look symmetric can have radically different parameters and buffers. Accurate inventory turns layer movement from guesswork into a budgeted schedule.

19.15.3 Exercise 02

Exercise Title: Build a Sequential Model That Keeps Only One Layer on CUDA at a Time and Offloads It Back to CPU After Execution

Part 1 - Exercise Contract and Why It Matters

Create the simplest complete oversized-model execution strategy: move one layer to CUDA, execute it, prove completion, move the layer back to CPU, and continue. At no point may two layer groups intentionally remain resident.

Part 2 - System and Memory Mental Model

The activation becomes the boundary object passed between layers. It can remain on CUDA while weights are replaced. Moving a module to CPU recursively moves its parameters and buffers, but doing so immediately after forward submission can race outstanding kernels. A completion event makes the blocking baseline safe.

Part 3 - Quantitative Prediction Before Execution

Peak persistent layer VRAM should approach the largest layer's parameter plus buffer bytes, not total model bytes. Add activation, workspace, transfer temporary, and allocator margin. Predict total H2D bytes as the sum of all layer state loaded once and D2H bytes according to whether clean CPU masters are reused or CUDA weights are copied back.

Part 4 - Experimental Design and Instrumentation

Run inference under `inference_mode()`, verify one resident layer after each admission, and compare output with an all-CPU reference. Log layer bytes, move time, compute time, event completion, allocated/reserved peaks, activation shape, and checksum. Repeat to expose the cost of reloading every request.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class OneLayerCudaSequential:
    def __init__(self, layers, device="cuda:0"):
        self.layers = layers
        self.device = torch.device(device)
        for layer in self.layers:
            layer.to("cpu")

    def __call__(self, inputs):
        activation = inputs.to(self.device)
        trace = []
        with torch.inference_mode():
            for index, layer in enumerate(self.layers):
                layer.to(self.device)
                activation = layer(activation)
                completed = torch.cuda.Event()
                completed.record(torch.cuda.current_stream())
                completed.synchronize()
                layer.to("cpu")
                trace.append({
                    "layer": index,
                    "output_shape": tuple(activation.shape),
                    "offloaded": True,
                })
        return activation, trace

def E02(layers):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required")
    return OneLayerCudaSequential(layers)
```

Part 6 - Measurement, Validation, and Failure Analysis

`layer.to("cpu")` copies active state back and may be more expensive than restoring pre-existing CPU masters. It also blocks overlap. Training is excluded because backward still needs forward state. Shared modules called later cannot be treated as permanently finished after the first call; execution schedule, not module list position, determines final use.

Part 7 - Engineering Interpretation and Philosophy

This baseline sacrifices speed for a small and easy-to-prove resident window. Its value is diagnostic: every optimization that follows must outperform it without weakening its clear memory and completion invariants.

19.15.4 Exercise 03

Exercise Title: Compare Full-GPU Residency Against One-Layer-at-a-Time Offloading and Measure Peak VRAM, Latency, and Throughput

Part 1 - Exercise Contract and Why It Matters

Compare the fastest simple residency strategy with the smallest simple layer window using identical model weights, inputs, precision, and inference mode. Report both feasibility and the cost paid for memory reduction.

Part 2 - System and Memory Mental Model

Full residency pays weight transfer once and reuses every layer across requests. One-layer offloading repeatedly moves state and serializes transfer with compute. Peak allocated VRAM, caching allocator reserved memory, synchronized request latency, and throughput answer different questions and must all be measured.

Part 3 - Quantitative Prediction Before Execution

Predict the full model's unique parameter and buffer bytes. Predict the one-layer peak from the largest group plus activations and workspace. For R requests, estimate reload traffic and the request count at which full residency's initial load is amortized relative to repeated layer movement.

Part 4 - Experimental Design and Instrumentation

Create identical model copies from one state dictionary. Warm up each variant, reset peak statistics, synchronize measurement boundaries, and run multiple batch sizes. Report cold start, median/tail latency, requests per second, peak allocated/reserved VRAM, H2D/D2H traffic, and output parity.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

def measure_completed(run_once, repetitions):
    torch.cuda.synchronize()
    torch.cuda.reset_peak_memory_stats()
    started = time.perf_counter()
    output = None
    for unused in range(repetitions):
        output = run_once()
    torch.cuda.synchronize()
    elapsed = time.perf_counter() - started
    peak = torch.cuda.max_memory_allocated()
    return output, elapsed, peak

def E03(full_model, offload_runtime, inputs, repetitions=10):
    full_model.eval().to("cuda:0")

    def run_full():
        with torch.inference_mode():
            return full_model(inputs.to("cuda:0"))

    def run_offloaded():
```

```
        output, unused_trace = offload_runtime(inputs)
        return output

    full_output, full_time, full_peak = measure_completed(
        run_full, repetitions)
    offload_output, offload_time, offload_peak = measure_completed(
        run_offloaded, repetitions)
    parity = torch.allclose(
        full_output.cpu(), offload_output.cpu(),
        rtol=1.0e-4, atol=1.0e-5)
    return full_time, full_peak, offload_time, offload_peak, parity
```

Part 6 - Measurement, Validation, and Failure Analysis

Measuring both variants in one process can leave allocator cache from the first run; use fresh processes for final peak claims. Ensure model copies start from identical weights and evaluation mode. CUDA launch time without synchronization is not inference latency. Report speed per completed request, not per Python forward submission.

Part 7 - Engineering Interpretation and Philosophy

Offloading is successful when it enables a workload that otherwise cannot run, then minimizes the price of that feasibility. A large percentage VRAM reduction does not erase a large latency penalty; both belong in the same engineering decision.

19.15.5 Exercise 04

Exercise Title: Implement Forward Hooks That Automatically Move a Layer to CUDA Immediately Before Its Forward Pass

Part 1 - Exercise Contract and Why It Matters

Attach pre-forward hooks to selected execution layers so placement occurs without editing each forward method. Move module state and tensor inputs to the same CUDA device and record the invocation that caused admission.

Part 2 - System and Memory Mental Model

A pre-hook runs before its module's forward call. Moving a complete module includes parameters and buffers, while incoming activations may still be on a different tier. Hooks fire per invocation, so a shared module can trigger more than once. Container hooks and child hooks can overlap responsibilities and must not both move the same state blindly.

Part 3 - Quantitative Prediction Before Execution

Predict hook order and number of physical H2D movements from the execution graph. Distinguish invocation count from unique module count. Record CPU hook overhead, transfer bytes, and visible forward stall; immediate pre-hook movement places nearly the entire transfer on the critical path.

Part 4 - Experimental Design and Instrumentation

Attach only to leaf compute modules, run repeated and nested modules, and verify every input/parameter/buffer device before the operator. Log module name, invocation, previous

device, bytes, new device, transfer time, and output parity. Remove handles and prove a later forward has no duplicate callbacks.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class LayerPromotionHooks:
    def __init__(self, target="cuda:0"):
        self.target = torch.device(target)
        self.handles = []
        self.invocations = {}

    def attach(self, named_layers):
        for layer_name, layer in named_layers:
            self.invocations[layer_name] = 0

            def promote(current_layer, inputs, name=layer_name):
                current_layer.to(self.target)
                moved_inputs = []
                for value in inputs:
                    if torch.is_tensor(value):
                        moved_inputs.append(value.to(self.target))
                    else:
                        moved_inputs.append(value)
                self.invocations[name] += 1
                return tuple(moved_inputs)

            handle = layer.register_forward_pre_hook(promote)
            self.handles.append(handle)

    def remove(self):
        for handle in self.handles:
            handle.remove()
        self.handles.clear()

def E04():
    return LayerPromotionHooks()
```

Part 6 - Measurement, Validation, and Failure Analysis

The closure captures each name through a default argument to avoid the common late-binding bug. Nested inputs and keyword arguments require a recursive, with-keywords hook. Compilers, tracing, reentrant forwards, and multi-threaded inference can change hook assumptions. The immediate `to()` baseline is blocking from the layer's perspective and does not prefetch.

Part 7 - Engineering Interpretation and Philosophy

Hooks are a mechanism for separating control policy from model mathematics. They become dangerous when invisible. Every automatic movement should be traceable to one hook, one invocation, one byte count, and one explicit reason.

19.15.6 Exercise 05

Exercise Title: Implement Post-Forward Hooks That Return Completed Layers to CPU Without Breaking Subsequent Computation

Part 1 - Exercise Contract and Why It Matters

Attach post-forward hooks that record CUDA completion, wait only for the layer's submitted work in the baseline, and return module state to CPU while leaving the output activation available for later CUDA computation.

Part 2 - System and Memory Mental Model

A post-hook means Python forward has returned; it does not mean CUDA has finished reading parameters or buffers. Record an event on the executing stream after forward. Once that event completes, clean inference state may be restored to CPU. The output tensor owns different storage and remains valid independently of the producing module's weight residency.

Part 3 - Quantitative Prediction Before Execution

Predict the parameter and buffer bytes released by each hook. Also predict the D2H traffic created when the module returns to CPU and the event wait placed on the critical path. This baseline lowers persistent VRAM but serializes return movement; later designs reuse CPU masters instead of copying clean weights back.

Part 4 - Experimental Design and Instrumentation

Record module state immediately before hook, after event recording, after event completion, and after CPU movement. Verify output device and checksum, next-layer execution, allocated versus reserved memory, and repeated requests. Test a shared module whose next use is later than its first post-hook.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class LayerDemotionHooks:
    def __init__(self):
        self.handles = []
        self.events = {}

    def attach(self, named_layers):
        for layer_name, layer in named_layers:
            def demote(current_layer, inputs, output,
                       name=layer_name):
                stream = torch.cuda.current_stream()
                completed = torch.cuda.Event()
                completed.record(stream)
                self.events[name] = completed
```

```
                completed.synchronize()
                current_layer.to("cpu")
                return output

            handle = layer.register_forward_hook(demote)
            self.handles.append(handle)

    def remove(self):
        for handle in self.handles:
            handle.remove()
        self.handles.clear()

def E05():
    return LayerDemotionHooks()
```

Part 6 - Measurement, Validation, and Failure Analysis

This hook is inference-oriented and intentionally blocking. It must not demote a shared module before its final planned use, nor a training layer before backward. If forward raises, an ordinary post-hook may not run; use robust cleanup and `always_call` support where available. Module movement also includes buffers, which is necessary for correctness but changes traffic.

Part 7 - Engineering Interpretation and Philosophy

Offloading after use seems obvious until asynchronous execution separates "returned" from "finished." The event turns a convenient callback into a correct lifetime boundary. Optimization begins only after that boundary is trusted.

19.15.7 Exercise 06

Exercise Title: Detect the Earliest Safe Point for Offloading a Layer and Reproduce a Failure Caused by Releasing It Too Early

Part 1 - Exercise Contract and Why It Matters

Define and test the earliest legal offload boundary. Reproduce a controlled failure in which Python changes or reuses layer storage while an asynchronous consumer can still access it, then fix the failure with event and lifetime tracking.

Part 2 - System and Memory Mental Model

Safety requires both logical final use and physical completion. The layer may have no future forward invocation yet still be read by a queued kernel. CUDA allocator lifetime protection and stream ordering are related but distinct: `record_stream()` prevents premature reuse, while event waits express data dependencies between streams.

Part 3 - Quantitative Prediction Before Execution

For each invocation identify its last logical use index and completion event. The earliest safe offload time is the maximum of all outstanding consumer completion times, subject to backward protection. Predict how much longer the layer remains resident compared with the Python post-forward time.

Part 4 - Experimental Design and Instrumentation

Queue a long CUDA operation using layer state, immediately attempt unsafe CPU movement or same-sized allocation, and repeat over many iterations. Detect wrong checksums or runtime errors. Then register completion and prove stability. Keep the negative test isolated because races can be timing-sensitive and corrupt later conclusions.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class LayerUseGuard:
    def __init__(self, final_invocation):
        self.final_invocation = final_invocation
        self.completed_events = []
        self.invocation = 0
        self.backward_protected = False

    def record_forward_use(self, stream):
        self.invocation += 1
        event = torch.cuda.Event()
        event.record(stream)
        self.completed_events.append(event)

    def safe_to_offload(self):
        if self.backward_protected:
            return False, "needed_by_backward"
        if self.invocation < self.final_invocation:
            return False, "future_forward_use"
        for event in self.completed_events:
            if not event.query():
                return False, "cuda_consumer_pending"
        return True, "all_uses_completed"

    def wait_until_safe(self):
        for event in self.completed_events:
            event.synchronize()
        safe, reason = self.safe_to_offload()
        if not safe:
            raise RuntimeError("offload remains unsafe: " + reason)

def E06(final_invocation):
    return LayerUseGuard(final_invocation)
```

Part 6 - Measurement, Validation, and Failure Analysis

An event recorded on the wrong stream proves nothing about the actual consumer. Shared modules require one guard across every owner path. Querying events is non-blocking and useful for scheduling; synchronizing is used only when the engine must commit offload immediately.

During training, backward protection is released only after the relevant gradient computation finishes.

Part 7 - Engineering Interpretation and Philosophy

The earliest safe point is a proof boundary, not a timer. Memory systems fail when they infer lifetime from Python control flow while hardware continues asynchronously. Explicit completion turns probabilistic correctness into an invariant.

19.15.8 Exercise 07

Exercise Title: Build a Layer Residency Table That Tracks Current Device, Parameter Bytes, Buffer Bytes, Execution State, and Next Planned Use

Part 1 - Exercise Contract and Why It Matters

Build a central residency table that gives policy, execution, and diagnostics one consistent view of every canonical layer. The table must update only through validated transitions and export scalar records without retaining computation graphs accidentally.

Part 2 - System and Memory Mental Model

Qualified module paths can alias one shared module. The table therefore maps owner paths to a canonical ID and stores one resource record. Current device is physical state; execution state includes pending and in-flight phases; next use comes from the schedule. Parameter and buffer bytes remain separate for interpretation even when admission uses their sum.

Part 3 - Quantitative Prediction Before Execution

Predict total resident, pending, protected, and reclaimable bytes from table rows. Compute next-use distance from current step. A policy can rank eviction by large next-use distance and high reload cost while applying hard protection and budget checks first.

Part 4 - Experimental Design and Instrumentation

Replay a trace with repeated and shared modules. At each step snapshot the table, sum each byte category, and verify totals against transition logs. Test illegal transitions, missing next-use entries, and stale device metadata. Bound event history to outstanding work only.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass, field

@dataclass
class LayerResidencyRow:
    canonical_id: int
    names: list
    device: str
    parameter_bytes: int
    buffer_bytes: int
    execution_state: str = "idle"
    next_planned_use: int = -1
    ready_event: object = None
    consumer_events: list = field(default_factory=list)
```

```
    @property
    def total_bytes(self):
        return self.parameter_bytes + self.buffer_bytes

class LayerResidencyTable:
    def __init__(self):
        self.rows = {}
        self.name_to_id = {}

    def add(self, canonical_id, name, device,
            parameter_bytes, buffer_bytes):
        if canonical_id not in self.rows:
            self.rows[canonical_id] = LayerResidencyRow(
                canonical_id, [], device,
                parameter_bytes, buffer_bytes)
        self.rows[canonical_id].names.append(name)
        self.name_to_id[name] = canonical_id

    def update_state(self, name, expected, new_state):
        canonical_id = self.name_to_id[name]
        row = self.rows[canonical_id]
        if row.execution_state != expected:
            raise RuntimeError("illegal layer state transition")
        row.execution_state = new_state

def E07():
    return LayerResidencyTable()
```

Part 6 - Measurement, Validation, and Failure Analysis

Active tables may retain event objects needed for correctness, but exported telemetry should replace them with scalar readiness. Object IDs are scoped to one model lifetime. Storage sharing across distinct Module objects requires an additional canonical state-group layer so bytes are not duplicated.

Part 7 - Engineering Interpretation and Philosophy

Dynamic systems become understandable when state has one authoritative ledger. The table is not merely logging; it is the contract preventing policy, execution, and accounting from inventing different realities.

19.15.9 Exercise 08

Exercise Title: Group Consecutive Layers into Offload Blocks and Compare Fine-Grained Layer Movement Against Coarse-Grained Block Movement

Part 1 - Exercise Contract and Why It Matters

Combine consecutive execution layers into atomic offload blocks and compare them with one-layer movement. Quantify how fewer transfers and events trade against a larger resident window and reduced eviction flexibility.

Part 2 - System and Memory Mental Model

A block owns the union of unique parameters and buffers of its layers. Internal activations stay on CUDA, eliminating device boundaries within the block. Block admission and eviction are atomic. Shared state between blocks must be assigned canonically rather than copied into both block budgets.

Part 3 - Quantitative Prediction Before Execution

For block size k, predict number of H2D/D2H phases, total state bytes, largest resident block, internal activation transfers removed, and event count. Transfer payload bytes may remain similar, yet fewer small submissions can reduce fixed latency. Check every block against the strict budget.

Part 4 - Experimental Design and Instrumentation

Sweep block lengths and include layers with variable sizes. Measure completed latency, throughput, peak VRAM, transfer count, transferred bytes, hook overhead, event overhead, and output parity. Compare fixed-count blocks with byte-balanced blocks so one huge layer does not dominate an otherwise uniform grouping.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def build_consecutive_blocks(layers, block_length):
    if block_length <= 0:
        raise ValueError("block length must be positive")
    blocks = []
    current = []
    for layer in layers:
        current.append(layer)
        if len(current) == block_length:
            blocks.append(current)
            current = []
    if current:
        blocks.append(current)
    return blocks

def execute_blocks(blocks, inputs, device="cuda:0"):
    activation = inputs.to(device)
    for block in blocks:
        for layer in block:
            layer.to(device)
        for layer in block:
            activation = layer(activation)
        completed = torch.cuda.Event()
        completed.record(torch.cuda.current_stream())
        completed.synchronize()
        for layer in block:
            layer.to("cpu")
```

```
    return activation

def E08(layers, block_length):
    return build_consecutive_blocks(layers, block_length)
```

Part 6 - Measurement, Validation, and Failure Analysis

The code requires `torch` imported by the containing exercise runner. Naively summing layer bytes can double count shared parameters. Residual edges can cross block boundaries and keep activations live even after producing layers are offloaded. Fixed-length blocks are a baseline, not proof of optimality.

Part 7 - Engineering Interpretation and Philosophy

Granularity is a control knob between memory flexibility and movement overhead. Fine-grained control is not free, and coarse-grained efficiency is not always affordable. The best block reflects measured layer sizes and execution edges.

19.15.10 Exercise 09

Exercise Title: Determine the Optimal Number of Consecutive Layers to Keep Resident Under a Fixed VRAM Budget

Part 1 - Exercise Contract and Why It Matters

Search the feasible resident-window sizes and choose the layer count that minimizes completed execution time under a hard byte budget. Do not assume the largest fitting count is fastest when layer sizes and compute times vary.

Part 2 - System and Memory Mental Model

Keeping more consecutive layers reduces reload frequency and can improve prefetch opportunity, but increases persistent VRAM and may reduce activation or workspace headroom. Count-based windows are simple; byte-aware variable windows are more faithful when layers differ substantially.

Part 3 - Quantitative Prediction Before Execution

For every candidate k, predict peak block bytes, transfer submissions, transfer time, compute time, overlap, and boundary overhead. Reject candidates whose predicted total exceeds managed capacity after reserves. The optimum is the feasible k with minimum validated end-to-end time, not minimum transfer count alone.

Part 4 - Experimental Design and Instrumentation

Warm up and benchmark every feasible count across repeated trials. Randomize trial order to reduce thermal and background bias. Log parameter/buffer peak, allocated/reserved peak, transfer bytes and events, visible stalls, throughput, latency distribution, and parity. Repeat for several batch sizes.

Part 5 - PyTorch/CUDA Implementation

```
def feasible_window_sizes(layer_bytes, budget_bytes,
                          activation_reserve_bytes):
    feasible = []
    layer_count = len(layer_bytes)
    for window in range(1, layer_count + 1):
        largest_window = 0
        for start in range(0, layer_count - window + 1):
            current_bytes = 0
            stop = start + window
            for index in range(start, stop):
                current_bytes += layer_bytes[index]
            if current_bytes > largest_window:
                largest_window = current_bytes
        predicted_peak = largest_window + activation_reserve_bytes
        if predicted_peak <= budget_bytes:
            feasible.append((window, predicted_peak))
    return feasible

def choose_fastest(measured_seconds_by_window, feasible):
    best_window = None
    best_seconds = None
    for window, unused_peak in feasible:
        seconds = measured_seconds_by_window[window]
        if best_seconds is None or seconds < best_seconds:
            best_seconds = seconds
            best_window = window
    return best_window, best_seconds

def E09(layer_bytes, budget_bytes, activation_reserve_bytes):
    return feasible_window_sizes(
        layer_bytes, budget_bytes, activation_reserve_bytes)
```

Part 6 - Measurement, Validation, and Failure Analysis

Contiguous-window byte sums must canonicalize shared state. A static activation reserve may be wrong for shape-changing models; profile it per boundary. The fastest candidate on one batch size, hardware topology, or sequence length may not generalize. Store the conditions with the chosen value.

Part 7 - Engineering Interpretation and Philosophy

Optimal residency is empirical because hardware turns theoretical bytes and dependencies into real time. Modeling narrows the safe search space; measurement chooses within it. Neither should pretend to replace the other.

19.15.11 Exercise 10

Exercise Title: Prefetch Layer N+1 to CUDA While Layer N Executes and Measure How Much PCIe Transfer Latency Can Be Hidden

Part 1 - Exercise Contract and Why It Matters

Submit the next layer's complete state before executing the current layer. Use a dedicated transfer stream and a ready event so the compute stream waits only if prefetch has not completed by the time the next layer is needed.

Part 2 - System and Memory Mental Model

Prefetch changes the resident window from one group to at least current plus next. Pinned CPU masters and non-blocking H2D make overlap possible. Different streams do not imply concurrency automatically; dependencies, copy-engine availability, PCIe contention, and CUDA allocator pressure determine the actual timeline.

Part 3 - Quantitative Prediction Before Execution

For layer N, compare current compute time C_N with next transfer time T_{N+1}. Ideal hidden time is $\min(C_N, T_{N+1})$, and ideal visible wait is $\max(0, T_{N+1} - C_N)$. Predict peak bytes for current, next pending state, activations, and workspace before admitting lookahead.

Part 4 - Experimental Design and Instrumentation

Benchmark serial on-demand loading and one-layer lookahead with identical state groups. Use CUDA events for copy and compute intervals and a profiler timeline to confirm overlap. Sweep layer size, compute intensity, batch size, and PCIe contention. Record visible waits, overlap fraction, peak VRAM, and output parity.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class NextLayerPrefetcher:
    def __init__(self, mover, ordered_groups):
        self.mover = mover
        self.groups = ordered_groups
        self.pending = {}

    def submit(self, index):
        if index >= len(self.groups):
            return
        if index in self.pending:
            return
        name, entries, unused_layer = self.groups[index]
        self.pending[index] = self.mover.submit(name, entries)

    def acquire(self, index, compute_stream):
        if index not in self.pending:
            self.submit(index)
        transaction = self.pending.pop(index)
        transaction.bind_for_compute(compute_stream)
        return transaction

    def run(self, activation):
```

```
        compute_stream = torch.cuda.current_stream()
        self.submit(0)
        for index in range(len(self.groups)):
            self.acquire(index, compute_stream)
            self.submit(index + 1)
            unused_name, unused_entries, layer = self.groups[index]
            activation = activation.to("cuda:0", non_blocking=True)
            activation = layer(activation)
        return activation

def E10(mover, ordered_groups):
    return NextLayerPrefetcher(mover, ordered_groups)
```

Part 6 - Measurement, Validation, and Failure Analysis

This exercise reuses the asynchronous group transaction from DMM12. Admission must reserve pending bytes before `submit`. Prefetching without timely eviction can exceed the budget; prefetching from pageable memory can appear non-blocking at Python level yet fail to overlap. Event timing must be recorded on the streams that perform the actual work.

Part 7 - Engineering Interpretation and Philosophy

Prefetching is scheduling work according to a reliable future. The goal is not to transfer everything early, but to place the right layer far enough ahead that movement leaves the critical path while memory remains bounded.

19.15.12 Exercise 11

Exercise Title: Build a Double-Buffered Layer Offload Pipeline Using Separate CUDA Streams for Parameter Transfer and Computation

Part 1 - Exercise Contract and Why It Matters

Maintain two parameter slots: one active for layer computation and one staging the next layer. Alternate slots without overwriting storage still used by a kernel, and measure whether the pipeline reaches a steady overlapped cadence.

Part 2 - System and Memory Mental Model

Double buffering decouples producer and consumer phases. A slot transitions through empty, filling, ready, computing, and reusable. The transfer stream may fill the inactive slot only after its prior consumer-completion event. The compute stream may consume only after the slot's ready event.

Part 3 - Quantitative Prediction Before Execution

Peak slot bytes equal the two largest simultaneously staged groups, plus activation and workspace reserve. In steady state, ideal stage duration is $\max(T_{copy}, T_{compute})$. Predict pipeline fill and drain overhead separately from steady-state throughput.

Part 4 - Experimental Design and Instrumentation

Trace every slot state and event. Compare serial, one-lookahead allocation, and two reusable slots. Measure allocation count, copy/compute overlap, visible wait, peak VRAM, requests per

second, and parity. Use unequal layer sizes to test whether fixed slot capacity wastes memory or requires resizing.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass
import torch

@dataclass
class LayerSlot:
    index: int
    state: str = "empty"
    group_name: str = None
    transaction: object = None
    consumer_done: object = None

    def wait_until_reusable(self):
        if self.consumer_done is not None:
            self.consumer_done.synchronize()
        self.state = "empty"
        self.group_name = None
        self.transaction = None
        self.consumer_done = None

class DoubleBufferedLayers:
    def __init__(self, mover):
        self.mover = mover
        self.slots = [LayerSlot(0), LayerSlot(1)]

    def fill(self, slot_index, group_name, entries):
        slot = self.slots[slot_index]
        if slot.state != "empty":
            slot.wait_until_reusable()
        slot.transaction = self.mover.submit(group_name, entries)
        slot.group_name = group_name
        slot.state = "filling"

    def consume(self, slot_index, compute_stream):
        slot = self.slots[slot_index]
        slot.transaction.bind_for_compute(compute_stream)
        slot.state = "computing"

    def finish(self, slot_index, compute_stream):
        slot = self.slots[slot_index]
        event = torch.cuda.Event()
        event.record(compute_stream)
        slot.consumer_done = event
        slot.state = "awaiting_reuse"
```

```
def E11(mover):
    return DoubleBufferedLayers(mover)
```

Part 6 - Measurement, Validation, and Failure Analysis

The slot stores a transaction and therefore preserves pinned sources and CUDA destinations. Waiting with CPU synchronization is an auditable baseline; the next exercise replaces it with cross-stream event dependencies. Slot capacity must be budgeted before fill, and exceptions must not leave a slot permanently marked as filling.

Part 7 - Engineering Interpretation and Philosophy

Double buffering makes concurrency a state machine instead of an accident. Two slots are valuable because one can serve the consumer while the other serves the producer. More buffers help only when another bottleneck justifies their memory.

19.15.13 Exercise 12

Exercise Title: Coordinate Layer Prefetch and Eviction with CUDA Events Without Using Device-Wide Synchronization

Part 1 - Exercise Contract and Why It Matters

Replace CPU and device-wide waits with precise event edges between transfer, compute, and eviction streams. Preserve correctness while allowing unrelated CUDA work to continue concurrently.

Part 2 - System and Memory Mental Model

The transfer stream records ready after H2D. The compute stream waits on ready, runs the layer, and records consumer-done. The eviction stream waits on every consumer-done event before write-back or release. `record_stream()` protects allocator lifetime but does not replace these data dependencies.

Part 3 - Quantitative Prediction Before Execution

Draw the event graph and count only required edges. Predict the critical path and compare it with a device-wide synchronization graph that orders unrelated operations unnecessarily. Measure event overhead and recovered overlap, which may be small for tiny layers and substantial for deep pipelines.

Part 4 - Experimental Design and Instrumentation

Create two independent layer branches and prove one branch can transfer while the other computes. Record stream IDs, event timestamps, kernel overlap, and completed wall time. Remove one wait intentionally to reproduce wrong ordering, then add it back without global synchronization.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class LayerEventCoordinator:
    def __init__(self):
```

```
        self.transfer_stream = torch.cuda.Stream()
        self.eviction_stream = torch.cuda.Stream()
        self.ready = {}
        self.consumers = {}

    def record_ready(self, layer_name):
        event = torch.cuda.Event()
        event.record(self.transfer_stream)
        self.ready[layer_name] = event

    def order_compute(self, layer_name, compute_stream, tensors):
        compute_stream.wait_event(self.ready[layer_name])
        for tensor in tensors:
            tensor.record_stream(compute_stream)

    def record_consumer_done(self, layer_name, compute_stream):
        event = torch.cuda.Event()
        event.record(compute_stream)
        if layer_name not in self.consumers:
            self.consumers[layer_name] = []
        self.consumers[layer_name].append(event)

    def order_eviction(self, layer_name, tensors):
        events = self.consumers.get(layer_name, [])
        with torch.cuda.stream(self.eviction_stream):
            for event in events:
                self.eviction_stream.wait_event(event)
            for tensor in tensors:
                tensor.record_stream(self.eviction_stream)

def E12():
    return LayerEventCoordinator()
```

Part 6 - Measurement, Validation, and Failure Analysis

Recording ready outside the actual transfer stream creates false ordering. Ordering eviction without subsequently recording its completion is insufficient for CPU authority changes. Event dictionaries must be pruned after completion. Profile the timeline: different colored streams are not proof that kernels or copies overlapped.

Part 7 - Engineering Interpretation and Philosophy

Global synchronization is easy because it replaces reasoning with waiting. Events preserve performance by expressing the smallest correct happens-before graph. Precision in dependency design is therefore both a correctness tool and a throughput optimization.

19.15.14 Exercise 13

Exercise Title: Handle Residual Connections That Require Earlier Layer Outputs to Remain Available After Their Producing Layer Has Been Offloaded

Part 1 - Exercise Contract and Why It Matters

Offload producing layer state while retaining its output activation until every residual consumer finishes. Track activation lifetime independently so a memory optimization never drops skip-path data merely because the producer weights are gone.

Part 2 - System and Memory Mental Model

Residual graphs are directed acyclic graphs, not simple chains. The producing layer's parameters can become irrelevant after forward, while its output remains live across several later layers. Each residual activation has a consumer count, producer-ready event, consumer-done events, and storage lifetime on the streams that use it.

Part 3 - Quantitative Prediction Before Execution

From the graph, compute last consumer and live-range bytes for every skip output. Add all simultaneously live residual activations to the layer-window budget. Predict peaks at merge points; parameter offloading cannot reduce activation memory that the graph semantically requires.

Part 4 - Experimental Design and Instrumentation

Build a residual block with one and multiple consumers. Log activation creation, ready, acquire, remaining consumers, completion, release, and bytes. Compare against a sequential-only lifetime assumption and reproduce the resulting missing tensor or corrupted reuse failure.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass, field
import torch

@dataclass
class ResidualActivation:
    key: str
    tensor: object
    remaining_consumers: int
    producer_ready: object
    consumer_done: list = field(default_factory=list)

    def acquire(self, stream):
        if self.remaining_consumers <= 0:
            raise RuntimeError("residual has no remaining consumer")
        stream.wait_event(self.producer_ready)
        self.tensor.record_stream(stream)
        return self.tensor

    def finish_consumer(self, stream):
        event = torch.cuda.Event()
        event.record(stream)
        self.consumer_done.append(event)
        self.remaining_consumers -= 1
```

```
    def release(self):
        if self.remaining_consumers != 0:
            return False
        for event in self.consumer_done:
            event.synchronize()
        self.tensor = None
        return True

def E13(key, tensor, consumer_count, producer_stream):
    ready = torch.cuda.Event()
    ready.record(producer_stream)
    return ResidualActivation(key, tensor, consumer_count, ready)
```

Part 6 - Measurement, Validation, and Failure Analysis

Consumer count must come from the actual execution graph, including conditional branches. In-place operations can destroy a residual value even when lifetime is tracked correctly. Training may retain the activation for backward after all forward consumers finish, requiring an additional protection reason.

Part 7 - Engineering Interpretation and Philosophy

Layer lifetime and information lifetime are different. Offloading succeeds only when the runtime follows the graph's values, not merely its module order. Residuals make this distinction impossible to ignore.

19.15.15 Exercise 14

Exercise Title: Preserve Shared Modules and Tied Weights During Layer Offloading Without Creating Duplicate Parameter Copies in VRAM

Part 1 - Exercise Contract and Why It Matters

Canonicalize shared module instances and tied parameter storage so repeated owner paths reuse one CUDA copy. Aggregate invocation schedules and final-use decisions across every owner before eviction.

Part 2 - System and Memory Mental Model

A shared Module object can be called at multiple graph positions. Distinct modules can also contain one tied Parameter. Module identity and state identity therefore form two maps: invocations point to canonical modules, while module state groups point to canonical parameter and buffer resources.

Part 3 - Quantitative Prediction Before Execution

Calculate naive by-owner bytes and canonical unique bytes, then predict one H2D transfer per resource group. Final layer eviction is allowed only after the maximum planned invocation of all owners and all CUDA consumers. Report the duplicate-copy ratio that a name-based implementation would create.

Part 4 - Experimental Design and Instrumentation

Build a model that calls one projection twice and ties an output weight to an embedding. Log owner paths, canonical module IDs, parameter IDs, storage IDs, invocations, transfers, CUDA storage pointers, and final use. Mutate a tied weight in a controlled test and verify every owner observes the same value.

Part 5 - PyTorch/CUDA Implementation

```
class CanonicalLayerGraph:
    def __init__(self, model):
        self.modules = {}
        self.module_names = {}
        self.parameter_owners = {}
        for name, module in model.named_modules(remove_duplicate=False):
            module_id = id(module)
            if module_id not in self.modules:
                self.modules[module_id] = module
                self.module_names[module_id] = []
            self.module_names[module_id].append(name)
            for parameter_name, parameter in module.named_parameters(
                    recurse=False, remove_duplicate=False):
                parameter_id = id(parameter)
                if parameter_id not in self.parameter_owners:
                    self.parameter_owners[parameter_id] = []
                owner = name + "." + parameter_name
                self.parameter_owners[parameter_id].append(owner)

    def unique_parameter_ids_for_module(self, module_id):
        result = []
        seen = set()
        module = self.modules[module_id]
        for parameter in module.parameters(recurse=False):
            parameter_id = id(parameter)
            if parameter_id not in seen:
                seen.add(parameter_id)
                result.append(parameter_id)
        return result

def E14(model):
    return CanonicalLayerGraph(model)
```

Part 6 - Measurement, Validation, and Failure Analysis

Object identity catches ordinary sharing but not distinct tensor views over one storage. Container paths can produce empty names and should be formatted carefully. Hooks attached once per owner path can still fire multiple times on one object; attach control to canonical identity and count invocations explicitly.

Part 7 - Engineering Interpretation and Philosophy

Sharing is part of program semantics and memory economics. Duplicating shared state spends VRAM and silently changes alias behavior. Correct offloading follows canonical resources while preserving every logical owner.

19.15.16 Exercise 15

Exercise Title: Extend Dynamic Layer Offloading from Inference to Training and Track Parameters, Gradients, Activations, and Autograd Dependencies

Part 1 - Exercise Contract and Why It Matters

Define a training-safe layer lifecycle covering forward promotion, saved activations, backward parameter reuse, gradient creation, optimizer access, and final release. Demonstrate why an inference offload point is insufficient once autograd owns future dependencies.

Part 2 - System and Memory Mental Model

Training creates several lifetimes per layer. Parameters are read in forward and usually again in backward. Saved tensors preserve information for gradient calculation. Gradients appear during backward, and optimizer states may be two or more tensors per parameter. Moving only weights can leave gradients and optimizer state on incompatible devices.

Forward can offload a layer's clean weights to CPU masters, but backward must prefetch them again before that layer's gradient computation. Saved activations can remain on CUDA, be checkpointed for recomputation, or be offloaded under a separate policy. Dirty authoritative state appears after optimizer update.

Part 3 - Quantitative Prediction Before Execution

For each layer estimate weight, buffer, gradient, saved-activation, and optimizer state bytes separately. Predict forward H2D, backward H2D, activation D2H/H2D, and optimizer traffic. Compare no offload, weight-only offload, activation checkpointing, and combined strategies before measuring.

Part 4 - Experimental Design and Instrumentation

Track boundaries at forward-pre, forward-post, backward-pre, backward-post, optimizer-step, and zero-grad. Use autograd hooks to record completion, not to perform unsafe recursive backward. Validate gradients and updated parameters against a fully resident training step. Report peak VRAM, traffic, step time, loss parity, and gradient-norm parity.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass, field

@dataclass
class TrainingLayerLifetime:
    name: str
    parameter_bytes: int
    buffer_bytes: int
    gradient_bytes: int = 0
    activation_bytes: int = 0
    forward_complete: bool = False
    backward_complete: bool = False
```

```
    optimizer_complete: bool = False
    protected_reasons: set = field(default_factory=set)

    def begin_forward(self):
        self.protected_reasons.add("forward")

    def end_forward(self, activation_bytes):
        self.forward_complete = True
        self.activation_bytes = activation_bytes
        self.protected_reasons.discard("forward")
        self.protected_reasons.add("backward")

    def end_backward(self, gradient_bytes):
        self.backward_complete = True
        self.gradient_bytes = gradient_bytes
        self.protected_reasons.discard("backward")
        self.protected_reasons.add("optimizer")

    def end_optimizer(self):
        self.optimizer_complete = True
        self.protected_reasons.discard("optimizer")

    def releasable(self):
        return len(self.protected_reasons) == 0

def E15(name, parameter_bytes, buffer_bytes):
    return TrainingLayerLifetime(
        name, parameter_bytes, buffer_bytes)
```

Part 6 - Measurement, Validation, and Failure Analysis

The tracker models contracts; a real implementation must integrate saved-tensor hooks, module full-backward hooks, optimizer state placement, and stream events. Backward order is usually reverse forward order but can differ in branched graphs. Gradient accumulation extends lifetime across microbatches. Never move a leaf Parameter's storage while an existing graph expects the prior storage.

Part 7 - Engineering Interpretation and Philosophy

Training offloading is not inference offloading plus `backward()`. It is a second execution schedule with new state and authority. Treating every memory category explicitly prevents an optimization in one phase from silently invalidating another.

19.15.17 Exercise 16

Exercise Title: Reproduce Transfer Thrashing by Using an Aggressive Layer-Offload Policy and Detect It from PCIe Traffic and Execution Timeline

Part 1 - Exercise Contract and Why It Matters

Construct repeated-module and short-phase traces that cause a reactive runtime to evict layers immediately before they are needed again. Detect thrashing from movement, residence duration, useful work, and timeline evidence rather than from low hit rate alone.

Part 2 - System and Memory Mental Model

Thrashing is repeated state reversal with insufficient useful computation between transfers. It occurs when the resident window is smaller than an active reuse set, final-use estimates are wrong, or pressure thresholds chatter. PCIe can become saturated while GPU compute remains idle waiting for reloaded layers.

Part 3 - Quantitative Prediction Before Execution

Define migrations per invocation, bytes transferred per useful FLOP, and short residence as eviction within a small number of schedule steps after admission. Predict an alternating trace whose working set is one layer larger than the window. Compare with a static subset and a larger hysteretic window.

Part 4 - Experimental Design and Instrumentation

Profile stable sequential traces, alternating shared layers, and phase changes. Log H2D and D2H bytes, reload count, resident steps, GPU idle gaps, event waits, copy-engine use, compute use, and latency tails. Mark each timeline transfer with its canonical layer ID and policy reason.

Part 5 - PyTorch/CUDA Implementation

```
class LayerThrashDetector:
    def __init__(self, short_residence_steps=2):
        self.short_limit = short_residence_steps
        self.loaded_at = {}
        self.ever_loaded = set()
        self.reloads = 0
        self.short_evictions = 0
        self.h2d_bytes = 0
        self.d2h_bytes = 0

    def record_load(self, layer_name, step, size_bytes):
        if layer_name in self.ever_loaded:
            self.reloads += 1
        self.ever_loaded.add(layer_name)
        self.loaded_at[layer_name] = step
        self.h2d_bytes += size_bytes

    def record_eviction(self, layer_name, step,
                        size_bytes, write_back):
        admitted = self.loaded_at[layer_name]
        residence = step - admitted
        if residence <= self.short_limit:
            self.short_evictions += 1
        if write_back:
            self.d2h_bytes += size_bytes
        del self.loaded_at[layer_name]
```

```
    def report(self, invocations):
        migrations = self.reloads + self.short_evictions
        return {
            "reloads": self.reloads,
            "short_evictions": self.short_evictions,
            "migration_signal_per_invocation":
                migrations / max(1, invocations),
            "h2d_bytes": self.h2d_bytes,
            "d2h_bytes": self.d2h_bytes,
        }

def E16():
    return LayerThrashDetector()
```

Part 6 - Measurement, Validation, and Failure Analysis

Streaming through never-reused layers creates misses but not thrashing. Require evidence that recently evicted state returns. Dirty write-back and clean drop have different costs. Profiler traces are essential because high transfer bytes can be harmless when fully overlapped, while smaller serialized transfers can dominate latency.

Part 7 - Engineering Interpretation and Philosophy

Aggressive reclamation can manufacture the scarcity it tries to solve. A layer should leave VRAM only when the capacity it frees is more valuable than the future reload it creates. Thrashing is the measurable signature of getting that exchange repeatedly wrong.

19.15.18 Exercise 17

Exercise Title: Build an Adaptive Layer Residency Policy That Changes the Number of GPU-Resident Layers from Measured VRAM Headroom and Transfer Cost

Part 1 - Exercise Contract and Why It Matters

Adjust the resident window from measured headroom, transfer stall, and thrashing. Expand when memory is safely underused and reload cost is visible; contract before pressure becomes an OOM. Use hysteresis and cooldown to prevent oscillation.

Part 2 - System and Memory Mental Model

The controller observes managed resident and pending bytes, physical headroom, activation peak, visible transfer wait, and recent reload rate. Hard capacity is an invariant; adaptation is an optimization inside it. High and low pressure thresholds plus a minimum dwell period give the decision memory.

Part 3 - Quantitative Prediction Before Execution

Estimate marginal benefit of one additional layer as avoided visible reload milliseconds. Estimate marginal cost as its bytes times pressure price plus any lost activation reserve. Expand only when benefit is positive and projected peak is below the high watermark. Contract until the low recovery watermark.

Part 4 - Experimental Design and Instrumentation

Feed stable memory, growing sequence length, phase changes, and noisy headroom. Compare fixed windows with adaptive control. Log observations, decision, previous/new window, cooldown, projected bytes, reloads, visible waits, budget violations, latency, and regret relative to the best fixed window per phase.

Part 5 - PyTorch/CUDA Implementation

```
class AdaptiveLayerWindow:
    def __init__(self, minimum=1, maximum=8,
                 cooldown_steps=5):
        self.minimum = minimum
        self.maximum = maximum
        self.window = minimum
        self.cooldown_steps = cooldown_steps
        self.last_change = -cooldown_steps

    def update(self, step, projected_next_bytes,
               free_headroom_bytes, visible_transfer_ms,
               thrash_rate, pressure):
        if step - self.last_change < self.cooldown_steps:
            return self.window, "cooldown"

        if pressure or projected_next_bytes > free_headroom_bytes:
            if self.window > self.minimum:
                self.window -= 1
                self.last_change = step
                return self.window, "contract_for_pressure"
            return self.window, "minimum_window"

        transfer_problem = visible_transfer_ms > 0.2
        reuse_problem = thrash_rate > 0.1
        if transfer_problem or reuse_problem:
            if self.window < self.maximum:
                self.window += 1
                self.last_change = step
                return self.window, "expand_for_reuse"
        return self.window, "stable"

def E17():
    return AdaptiveLayerWindow()
```

Part 6 - Measurement, Validation, and Failure Analysis

Raw free VRAM is noisy and may include memory outside the controller. Filter measurements and retain a safety reserve. A larger count can admit one enormous layer, so byte projection remains mandatory. Changes should be evaluated after enough samples; reacting to one slow transfer confuses noise with phase change.

Part 7 - Engineering Interpretation and Philosophy

Adaptive residency is feedback control. It succeeds by changing slowly enough to measure consequences and quickly enough to follow workload phases. Stability is as important as responsiveness because every reversed decision moves real bytes.

19.15.19 Exercise 18

Exercise Title: Stress-Test Dynamic Layer Offloading on a Deep Transformer-Style PyTorch Model Whose Total Weights Exceed Available VRAM

Part 1 - Exercise Contract and Why It Matters

Exercise the runtime on repeated Transformer-style blocks whose total unique weights exceed an artificial managed VRAM budget. Preserve a safety gap below physical memory and test long execution traces, variable sequence lengths, and shared embeddings without intentionally crashing the device.

Part 2 - System and Memory Mental Model

Transformer blocks combine attention projections, feed-forward weights, normalization parameters, and buffers. Weight bytes may be regular across depth, while activation and attention workspace grow strongly with batch and sequence length. A weight-only budget that ignores this dynamic reserve will fail as context grows.

Part 3 - Quantitative Prediction Before Execution

Calculate total unique weight bytes, per-block state, maximum resident window, activation reserve by sequence length, predicted H2D bytes, and ideal overlap. Reject any configuration whose largest atomic block plus activation/workspace reserve exceeds managed capacity, even if average block size fits.

Part 4 - Experimental Design and Instrumentation

Sweep depth, hidden size, feed-forward ratio, sequence length, batch, dtype, window, and prefetch distance. Use fixed seeds and synthetic inputs. Log output checksums, peak allocated/reserved and managed bytes, pinned RAM, transfers, reloads, event waits, kernel/copy overlap, tokens per second, and latency tails. Capture and minimize failures.

Part 5 - PyTorch/CUDA Implementation

```
class TransformerOffloadStressPlan:
    def __init__(self, block_bytes, budget_bytes,
                 activation_reserve_bytes):
        self.block_bytes = block_bytes
        self.budget = budget_bytes
        self.activation_reserve = activation_reserve_bytes
        self.trace = []

    def validate_atomic_fit(self):
        for index, size_bytes in enumerate(self.block_bytes):
            required = size_bytes + self.activation_reserve
            if required > self.budget:
```

```
                message = "block does not fit: " + str(index)
                raise RuntimeError(message)

    def build_forward_trace(self):
        self.trace.clear()
        for index in range(len(self.block_bytes)):
            self.trace.append(("forward", index))
        return self.trace

    def build_training_trace(self):
        self.build_forward_trace()
        index = len(self.block_bytes) - 1
        while index >= 0:
            self.trace.append(("backward", index))
            index -= 1
        return self.trace

    def total_parameter_bytes(self):
        total = 0
        for size_bytes in self.block_bytes:
            total += size_bytes
        return total

def E18(block_bytes, budget_bytes, activation_reserve_bytes):
    plan = TransformerOffloadStressPlan(
        block_bytes, budget_bytes, activation_reserve_bytes)
    plan.validate_atomic_fit()
    return plan
```

Part 6 - Measurement, Validation, and Failure Analysis

The plan validates memory logic before constructing a potentially dangerous CUDA workload. Real attention kernels can request shape-dependent workspace not visible in parameter totals. Artificial managed limits should be enforced by admission, not by consuming all physical VRAM. Reproducibility requires the exact trace, shapes, dtype, PyTorch/CUDA versions, and hardware.

Part 7 - Engineering Interpretation and Philosophy

An oversized model is not one memory problem; it is the interaction of static state, dynamic activations, workspace, and movement. Stress testing proves whether the runtime manages that interaction across scale rather than only on a friendly toy chain.

19.15.20 Exercise 19

Exercise Title: Build and Benchmark a Complete PyTorch Layer-Offloading Runtime That Executes an Oversized Model Through Automatic Prefetch, Residency Tracking, and Eviction

Part 1 - Exercise Contract and Why It Matters

Integrate canonical layer groups, pinned CPU masters, a strict byte budget, ready and consumer events, one-layer prefetch, next-use tracking, final-use eviction, and diagnostics into one inference runtime. Execute a model whose total layer state exceeds managed VRAM while preserving outputs.

Part 2 - System and Memory Mental Model

Every canonical layer group owns CPU masters and optional CUDA destinations. Its state progresses through CPU, prefetch-pending, CUDA-ready, executing, consumer-pending, and evictable. Pending bytes count at submission. The compute stream waits on ready. Eviction waits on consumer completion and restores CPU masters rather than copying clean inference weights back.

Residual activations, shared modules, and tied weights remain separate canonical resources with their own lifetimes. The compact runtime below targets a known sequential inference schedule; graph and training extensions retain the same contracts but need richer schedules.

Part 3 - Quantitative Prediction Before Execution

Before execution, simulate total unique layer bytes, largest atomic group, resident and pending peaks, activation/workspace reserve, H2D bytes, reloads, and visible wait. Require $R + P + A + W + S \leq C$, where S is safety margin. State acceptance criteria for output parity, zero illegal transitions, and zero managed-budget violations.

Part 4 - Experimental Design and Instrumentation

Compare all-CPU, full-GPU when feasible on a reference system, one-layer serial, and prefetched runtime. Keep state dictionaries, inputs, dtype, and request order identical. Report cold start, median/tail latency, throughput, peak managed and CUDA memory, pinned RAM, transfers, overlap, event waits, policy time, evictions, and checksums.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass, field
from collections import OrderedDict
import torch

@dataclass
class RuntimeLayer:
    name: str
    module: object
    entries: list
    size_bytes: int
    transaction: object = None
    state: str = "cpu"
    consumer_events: list = field(default_factory=list)
    next_use: int = -1

class LayerOffloadRuntime:
    def __init__(self, ordered_layers, mover, budget_bytes):
        self.layers = ordered_layers
        self.mover = mover
        self.budget = budget_bytes
```

```
        self.resident_bytes = 0
        self.pending_bytes = 0
        self.resident = OrderedDict()

    def _completed_victim(self, excluded_name):
        for name in self.resident:
            if name == excluded_name:
                continue
            layer = self.resident[name]
            if layer.state != "consumer_pending":
                continue
            all_complete = True
            for event in layer.consumer_events:
                if not event.query():
                    all_complete = False
            if all_complete:
                return layer
        return None

    def _waitable_victim(self, excluded_name):
        for name in self.resident:
            if name == excluded_name:
                continue
            layer = self.resident[name]
            if layer.state == "consumer_pending":
                return layer
        return None

    def _evict(self, layer):
        for event in layer.consumer_events:
            event.synchronize()
        with torch.no_grad():
            for parameter, cpu_master in layer.entries:
                parameter.data = cpu_master
        layer.transaction = None
        layer.consumer_events.clear()
        layer.state = "cpu"
        self.resident_bytes -= layer.size_bytes
        self.resident.pop(layer.name, None)

    def _make_room(self, requested_bytes, excluded_name,
                   allow_defer):
        if requested_bytes > self.budget:
            raise RuntimeError("atomic layer exceeds budget")
        projected = self.resident_bytes
        projected += self.pending_bytes + requested_bytes
        while projected > self.budget:
            victim = self._completed_victim(excluded_name)
```

```
            if victim is None:
                if allow_defer:
                    return False
                victim = self._waitable_victim(excluded_name)
                if victim is None:
                    raise RuntimeError("no finished layer can be evicted")
            self._evict(victim)
            projected = self.resident_bytes
            projected += self.pending_bytes + requested_bytes
        return True

    def prefetch(self, index, allow_defer=False):
        if index >= len(self.layers):
            return "end_of_schedule"
        layer = self.layers[index]
        if layer.state != "cpu":
            return "already_available"
        admitted = self._make_room(
            layer.size_bytes, layer.name, allow_defer)
        if not admitted:
            return "deferred_for_capacity"
        self.pending_bytes += layer.size_bytes
        try:
            layer.transaction = self.mover.submit(
                layer.name, layer.entries)
        except Exception:
            self.pending_bytes -= layer.size_bytes
            layer.transaction = None
            raise
        layer.state = "prefetch_pending"
        return "submitted"

    def acquire(self, index, compute_stream):
        layer = self.layers[index]
        if layer.state == "cpu":
            self.prefetch(index)
        if layer.state == "prefetch_pending":
            layer.transaction.bind_for_compute(compute_stream)
            self.pending_bytes -= layer.size_bytes
            self.resident_bytes += layer.size_bytes
            layer.state = "cuda_ready"
            self.resident[layer.name] = layer
        assert self.resident_bytes + self.pending_bytes <= self.budget
        return layer

    def finish(self, layer, compute_stream):
        event = torch.cuda.Event()
        event.record(compute_stream)
```

```
        layer.consumer_events.append(event)
        layer.state = "consumer_pending"

    def run(self, inputs):
        compute_stream = torch.cuda.current_stream()
        activation = inputs.to("cuda:0")
        self.prefetch(0)
        with torch.inference_mode():
            for index in range(len(self.layers)):
                layer = self.acquire(index, compute_stream)
                layer.state = "executing"
                self.prefetch(index + 1, allow_defer=True)
                activation = layer.module(activation)
                self.finish(layer, compute_stream)
        return activation

def E19(ordered_layers, mover, budget_bytes):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required")
    return LayerOffloadRuntime(
        ordered_layers, mover, budget_bytes)
```

Part 6 - Measurement, Validation, and Failure Analysis

The runtime assumes unique sequential layer names and clean inference weights. Production code must canonicalize shared resources, reserve activation and workspace bytes outside the layer budget, use asynchronous eviction, handle buffers, prune completed events, and clean up after exceptions. Prefetch can fail when pending groups consume capacity but are not yet eviction candidates; admission must then reduce lookahead rather than retry forever.

Training requires the E15 backward schedule, gradients, optimizer states, saved activations, and dirty authority. Never claim training support by simply removing `inference_mode()`. Validate every output against a reference and every transition against the residency table.

Part 7 - Engineering Interpretation and Philosophy

A complete offloading runtime is a scheduler, allocator client, dependency manager, and model executor. It succeeds not because all layers fit, but because the correct layers fit at the correct times. Every moved byte must have an owner, every eviction must have a completion proof, and every performance gain must survive synchronized measurement.

19.16 DMM14

Chapter Title: CPU Offloading for Models and Training State
Specialization: Dynamic Memory Management and CUDA Systems

19.16.1 Exercise 00

Exercise Title: Build a Precise Mental Model of CPU Offloading for Parameters, Gradients, Optimizer States, Activations, and Temporary Buffers

Part 1 - Exercise Contract and Why It Matters

Build one explicit state model for every major training-memory category. CPU offloading is not a single operation: parameters are model state, gradients are backward products, optimizer tensors are update history, activations belong to the current graph, and temporary buffers have short operator-defined lifetimes.

The contract must identify the authoritative copy, clean replicas, device, pinned status, pending transfer, consumer streams, protection reason, and next planned use for every resource. It must also maintain separate hard budgets for GPU VRAM and host RAM.

Part 2 - System and Memory Mental Model

Parameters are read in forward and backward and become dirty after an optimizer step. Gradients appear during backward and may accumulate across microbatches. AdamW commonly owns first- and second-moment tensors in addition to step state. Saved activations are protected until backward consumes them. Temporary workspaces may be invisible to Python tensor inventories.

An offload transaction passes through source-ready, D2H-pending, CPU-ready, and GPU-released states. A non-blocking D2H destination cannot be read by the CPU before completion. Pinned RAM improves transfer capability but consumes a scarce operating-system resource and must be budgeted separately.

Part 3 - Quantitative Prediction Before Execution

For bytes (M), estimate transfer time as (L+M/B). Predict total persistent training state as separate sums for weights, gradients, optimizer state, and buffers; then add dynamic activation and workspace peaks. During offload, source and destination coexist, so transient peak is larger than either final state.

Estimate RAM after offload as CPU masters plus offloaded gradients, optimizer states, activations, pinned staging, and process baseline. A legal plan satisfies both budgets at every pending and committed transition.

Part 4 - Experimental Design and Instrumentation

Record memory at model creation, forward, loss, backward, optimizer step, and zero-grad. For each category log bytes, authority, device, pinned state, transfer event, allocated and reserved VRAM, process RAM, and system available RAM. Include deliberate GPU-pressure and host-pressure experiments under artificial managed limits rather than physical exhaustion.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass, field

@dataclass
class OffloadState:
    key: str
    category: str
    size_bytes: int
    authority: str
    gpu_present: bool
    cpu_present: bool
    cpu_pinned: bool = False
```

```
    transfer_pending: bool = False
    dirty: bool = False
    protected_reasons: set = field(default_factory=set)

    def begin_d2h(self):
        if not self.gpu_present:
            raise RuntimeError("no GPU source to offload")
        if self.transfer_pending:
            raise RuntimeError("another transfer is pending")
        self.transfer_pending = True

    def commit_d2h(self, release_gpu):
        if not self.transfer_pending:
            raise RuntimeError("no D2H transaction to commit")
        self.cpu_present = True
        self.transfer_pending = False
        self.authority = "cpu" if self.dirty else "both_clean"
        if release_gpu:
            self.gpu_present = False

def E00(key, category, size_bytes):
    return OffloadState(
        key, category, size_bytes, "gpu", True, False)
```

Part 6 - Measurement, Validation, and Failure Analysis

Deleting a CUDA reference may reduce allocated memory without reducing reserved memory. Process RSS does not equal tensor payload, and system available RAM is more informative than free RAM alone. Shared storage and optimizer dictionaries can produce double counting. Validate storage identity and authoritative values, not only category totals.

Part 7 - Engineering Interpretation and Philosophy

CPU offloading exchanges one constrained resource for another and adds movement through a slower link. It is successful only when the complete system stays inside both budgets and the saved VRAM is worth the transfer, synchronization, and host-memory pressure it creates.

19.16.2 Exercise 01

Exercise Title: Measure the CPU RAM and GPU VRAM Cost of Parameters, Gradients, Optimizer States, and Activations in the Same PyTorch Training Step

Part 1 - Exercise Contract and Why It Matters

Account for every visible training-state category at consistent boundaries in one step. Separate tensor payload from allocator and process-level memory so the report explains both PyTorch state and the larger system footprint.

Part 2 - System and Memory Mental Model

Parameters exist before forward. Activations and saved tensors grow during forward. Gradients materialize during backward. Lazy optimizers often allocate state on the first step.

Zeroing gradients with `set_to_none=True` releases gradient tensors differently from filling existing storage with zeros.

Part 3 - Quantitative Prediction Before Execution

Predict parameter bytes exactly from unique tensors. For dense gradients, expect approximately one gradient payload per trainable parameter. AdamW moments are commonly about two parameter-sized tensors at parameter dtype, but inspect actual state rather than assuming. Estimate activations by boundary deltas and hooks, acknowledging hidden workspaces.

Part 4 - Experimental Design and Instrumentation

Warm up CUDA context, reset peak statistics, and snapshot before forward, after forward, after loss, after backward, after first optimizer step, and after zero-grad. Record tensor-category bytes by device, CUDA allocated/reserved/peak, process RSS, system available RAM, batch shape, dtype, and optimizer settings.

Part 5 - PyTorch/CUDA Implementation

```
def tensor_bytes(tensor):
    return tensor.numel() * tensor.element_size()

def unique_parameter_and_gradient_bytes(model):
    seen_parameters = set()
    seen_gradients = set()
    parameter_bytes = 0
    gradient_bytes = 0
    for parameter in model.parameters():
        if id(parameter) not in seen_parameters:
            seen_parameters.add(id(parameter))
            parameter_bytes += tensor_bytes(parameter)
        gradient = parameter.grad
        if gradient is not None and id(gradient) not in seen_gradients:
            seen_gradients.add(id(gradient))
            gradient_bytes += tensor_bytes(gradient)
    return parameter_bytes, gradient_bytes

def optimizer_tensor_bytes(optimizer):
    seen = set()
    total = 0
    for state in optimizer.state.values():
        for value in state.values():
            if not hasattr(value, "numel"):
                continue
            if id(value) in seen:
                continue
            seen.add(id(value))
            total += tensor_bytes(value)
    return total

def E01(model, optimizer):
```

```
    parameters, gradients = (
        unique_parameter_and_gradient_bytes(model))
    optimizer_state = optimizer_tensor_bytes(optimizer)
    return parameters, gradients, optimizer_state
```

Part 6 - Measurement, Validation, and Failure Analysis

The function does not measure saved activations directly; derive them from controlled deltas or saved-tensor hooks and label the method. A tensor ID can be reused after deallocation, so identity sets belong to one snapshot. Optimizer state can contain scalar tensors, mixed dtypes, or CPU entries and should be reported per device.

Part 7 - Engineering Interpretation and Philosophy

Training memory is a lifecycle, not one number. Boundary accounting reveals which category creates the peak and therefore which offload decision can actually reduce it. Optimizing a non-dominant category adds complexity without solving the observed limit.

19.16.3 Exercise 02

Exercise Title: Offload a Complete PyTorch Model to CPU Between Inference Requests and Measure Reload Latency and VRAM Recovery

Part 1 - Exercise Contract and Why It Matters

Move a complete inference model to CPU after a request, verify that live CUDA parameter and buffer storage is gone, then reload for the next request. Measure both memory recovery and the cold-request latency this policy creates.

Part 2 - System and Memory Mental Model

`model.to("cpu")` moves parameters and buffers but does not necessarily release activations, outputs, or allocator cache. Reloading places all weight transfer on the next request's critical path. This policy is useful for sparse request traffic or sharing one GPU among models, but costly at high request rate.

Part 3 - Quantitative Prediction Before Execution

Predict unique model state bytes, H2D reload time, D2H offload time, and post- offload allocated and reserved memory. For idle interval (I), compare VRAM opportunity value against two transfers and added tail latency. Estimate the request rate at which keeping the model resident becomes cheaper.

Part 4 - Experimental Design and Instrumentation

Warm up, issue requests with controlled idle gaps, synchronize boundaries, and record reload, forward, offload, and completed request time separately. Track allocated/reserved/peak VRAM, process RAM, output checksum, and model device. Retain and then delete output tensors to show why they affect recovery.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

class WholeModelCpuOffloader:
    def __init__(self, model, device="cuda:0"):
        self.model = model.eval().to("cpu")
        self.device = torch.device(device)

    def request(self, inputs):
        torch.cuda.synchronize()
        reload_started = time.perf_counter()
        self.model.to(self.device)
        cuda_inputs = inputs.to(self.device)
        torch.cuda.synchronize()
        reload_seconds = time.perf_counter() - reload_started

        with torch.inference_mode():
            output = self.model(cuda_inputs)
        torch.cuda.synchronize()

        offload_started = time.perf_counter()
        self.model.to("cpu")
        torch.cuda.synchronize()
        offload_seconds = time.perf_counter() - offload_started
        return output, reload_seconds, offload_seconds

def E02(model):
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA is required")
    return WholeModelCpuOffloader(model)
```

Part 6 - Measurement, Validation, and Failure Analysis

Keeping the returned output on CUDA intentionally leaves live VRAM; copy or consume it before claiming full recovery. Calling `empty_cache()` can return unused cached blocks but cannot free live tensors and should not be timed as model offload. Use fresh processes for clean allocator comparisons.

Part 7 - Engineering Interpretation and Philosophy

Whole-model offload is simple capacity multiplexing. It is rational when idle time is valuable and requests tolerate reload. The policy demonstrates that memory availability and service latency are two sides of one scheduling choice.

19.16.4 Exercise 03

Exercise Title: Offload Selected Parameter Groups to CPU While Keeping Frequently Used Parameters Resident in VRAM

Part 1 - Exercise Contract and Why It Matters

Partition canonical parameter groups into hot residents and cold CPU-backed groups under a fixed VRAM budget. Preserve Parameter identity, shared weights, and explicit transfer reasons while avoiding whole-model movement.

Part 2 - System and Memory Mental Model

Frequency, recency, next-use distance, and reload cost describe value; unique bytes describe capacity cost. A CPU master remains authoritative for clean inference groups. Promotion binds a CUDA copy to the existing Parameter object, while demotion restores the CPU master only after GPU consumers finish.

Part 3 - Quantitative Prediction Before Execution

Predict selected resident groups, unique resident bytes, misses, H2D traffic, and avoided reload time from a trace. Apply hard fit and protection before ranking. Compare a hot-set policy with static size-only and LRU baselines.

Part 4 - Experimental Design and Instrumentation

Use stable-hot, phase-changing, and alternating traces with tied weights. Log score, last use, bytes, canonical ID, promotion, demotion, consumers, transfer traffic, hit rate, regret, and output parity. Assert VRAM budget after pending and committed transitions.

Part 5 - PyTorch/CUDA Implementation

```
class HotParameterPolicy:
    def __init__(self, budget_bytes, promote_count=3):
        self.budget = budget_bytes
        self.promote_count = promote_count
        self.counts = {}
        self.last_use = {}
        self.resident = set()
        self.resident_bytes = 0
        self.step = 0

    def observe(self, group_name):
        self.step += 1
        old_count = self.counts.get(group_name, 0)
        self.counts[group_name] = old_count + 1
        self.last_use[group_name] = self.step

    def should_promote(self, group_name, size_bytes):
        if group_name in self.resident:
            return False, "already_resident"
        if self.counts.get(group_name, 0) < self.promote_count:
            return False, "not_hot"
        if self.resident_bytes + size_bytes > self.budget:
            return False, "does_not_fit"
        return True, "hot_and_fits"

    def commit_promotion(self, group_name, size_bytes):
        self.resident.add(group_name)
```

```
        self.resident_bytes += size_bytes
        assert self.resident_bytes <= self.budget

def E03(budget_bytes):
    return HotParameterPolicy(budget_bytes)
```

Part 6 - Measurement, Validation, and Failure Analysis

The policy decides admission but does not select victims; integrate a legal eviction policy when full. Cumulative counts retain stale popularity, so decay or windows are needed across phases. Metadata must not retain tensors or graphs. Training adds dirty parameters and optimizer-state coordination.

Part 7 - Engineering Interpretation and Philosophy

Selective offload treats VRAM as a working set rather than an all-or-nothing home. Hotness is a forecast that must earn its memory through future reuse, not a permanent label granted by historical frequency.

19.16.5 Exercise 04

Exercise Title: Build a CPU Parameter Store That Tracks Tensor Size, Dtype, Layer Ownership, Residency State, and Transfer Count

Part 1 - Exercise Contract and Why It Matters

Build a canonical CPU store that owns durable parameter masters and supplies pinned or pageable sources for promotion. Track scalar metadata and transfer history while preserving tied ownership and authoritative values.

Part 2 - System and Memory Mental Model

One canonical parameter can have several layer names. The store owns one CPU tensor per canonical identity, plus optional pinned staging and CUDA replica. Residency state distinguishes CPU-only, H2D-pending, both-clean, GPU-dirty, and D2H-pending. Transfer count is historical evidence, not current authority.

Part 3 - Quantitative Prediction Before Execution

Predict unique CPU payload, pinned staging payload, CUDA replicas, and duplicated bytes avoided by canonicalization. For each access trace, estimate H2D and D2H counts and bytes. Ensure store RAM plus other host allocations remains within a separate budget.

Part 4 - Experimental Design and Instrumentation

Register ordinary and tied parameters, compare checksums, promote and demote, then mutate a GPU copy and write it back. Log identity, owners, shape, dtype, bytes, pinned state, authority, residency, transfers, and version. Verify one CPU master for every alias group.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass, field

@dataclass
class CpuParameterEntry:
    canonical_id: int
    owners: list
    cpu_tensor: object
    size_bytes: int
    dtype: str
    residency: str = "cpu_only"
    h2d_count: int = 0
    d2h_count: int = 0
    version: int = 0

class CpuParameterStore:
    def __init__(self):
        self.entries = {}
        self.name_to_id = {}

    def register(self, name, parameter, pin=True):
        canonical_id = id(parameter)
        self.name_to_id[name] = canonical_id
        if canonical_id not in self.entries:
            master = parameter.detach().cpu().clone()
            if pin:
                master = master.pin_memory()
            size_bytes = master.numel() * master.element_size()
            self.entries[canonical_id] = CpuParameterEntry(
                canonical_id, [], master, size_bytes,
                str(master.dtype))
        self.entries[canonical_id].owners.append(name)

    def mark_h2d(self, name):
        entry = self.entries[self.name_to_id[name]]
        entry.h2d_count += 1
        entry.residency = "both_clean"

def E04():
    return CpuParameterStore()
```

Part 6 - Measurement, Validation, and Failure Analysis

Cloning guarantees independent CPU authority but doubles host payload at registration. Pinned allocation can fail or harm system health under excess. Object identity covers ordinary tying; distinct views over shared storage need storage-level canonicalization. Version must advance after committed writes, not transfer submission.

Part 7 - Engineering Interpretation and Philosophy

The CPU store is a truth service, not merely a cache. Placement can be dynamic only when the runtime knows which durable copy is authoritative and can restore any transient GPU replica without changing model semantics.

19.16.6 Exercise 05

Exercise Title: Compare Pageable RAM and Pinned RAM as Offload Destinations for Repeated Parameter Transfers

Part 1 - Exercise Contract and Why It Matters

Compare pageable and page-locked host buffers as repeated D2H offload destinations. Separate allocation cost, copy submission time, completion time, and end-to-end reuse time; otherwise a fast asynchronous submission can be mistaken for a completed transfer.

Part 2 - System and Memory Mental Model

A CUDA DMA engine can directly address pinned pages. Pageable destinations may require an internal staging path and can introduce host synchronization. Pinned memory enables useful asynchronous transfers, but pinning is not free: locked pages reduce operating-system flexibility and fresh pinning can dominate small copies. Reusable pinned buffers are therefore the meaningful offload design.

Part 3 - Quantitative Prediction Before Execution

For payload M, predict $T = L + M/B$. Add allocation or pinning time only when a fresh destination is created. Report effective bandwidth as M/T and include the transient coexistence of GPU source and CPU destination. Predict a crossover size below which launch and pinning overhead dominate bandwidth.

Part 4 - Experimental Design and Instrumentation

Warm up CUDA, preallocate equal pageable and pinned buffers, and reuse them for many D2H and H2D cycles. Time completion with CUDA events plus an explicit event synchronization. Run a second benchmark that includes fresh allocation. Verify checksums only after completion and record process RAM and pinned payload.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def timed_d2h(source, destination, repetitions):
    stream = torch.cuda.Stream()
    start = torch.cuda.Event(enable_timing=True)
    end = torch.cuda.Event(enable_timing=True)
    torch.cuda.synchronize()
    with torch.cuda.stream(stream):
        start.record(stream)
        for _ in range(repetitions):
            destination.copy_(source, non_blocking=True)
        end.record(stream)
    end.synchronize()
    return start.elapsed_time(end) / repetitions
```

```
def E05(numel=8_000_000, repetitions=30):
    source = torch.randn(numel, device="cuda")
    pageable = torch.empty(numel)
    pinned = torch.empty(numel, pin_memory=True)
    pageable_ms = timed_d2h(source, pageable, repetitions)
    pinned_ms = timed_d2h(source, pinned, repetitions)
    expected = source.sum().item()
    observed = pinned.sum().item()
    return pageable_ms, pinned_ms, expected, observed
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not read the host destination before the recorded event completes. A pageable copy requested with `non_blocking=True` does not guarantee the same path or overlap as a pinned copy. Thermal state, PCIe contention, NUMA placement, and including allocation in one result but not the other invalidate comparisons.

Part 7 - Engineering Interpretation and Philosophy

Pinned memory is a scheduling resource, not a universal speed flag. Use a small, bounded pool of reused pinned buffers where overlap has measurable value; use pageable memory for cold, long-lived copies that do not sit on the critical path.

19.16.7 Exercise 06

Exercise Title: Offload Gradients to CPU After Backward and Measure Peak VRAM Reduction Before the Optimizer Step

Part 1 - Exercise Contract and Why It Matters

Move completed gradients to a canonical CPU store after backward, prove value parity, release CUDA gradient storage, and quantify VRAM reduction before the update. State explicitly that a standard CUDA optimizer cannot consume those CPU gradients without rehydration or a CPU-side update design.

Part 2 - System and Memory Mental Model

Gradients are produced asynchronously on the current stream. A D2H stream must wait for backward completion, retain source storage until copying finishes, and commit the CPU copy before setting `parameter.grad=None`. With gradient accumulation, offload is legal only after the last microbatch or into an accumulating CPU destination with correct reduction semantics.

Part 3 - Quantitative Prediction Before Execution

Predict gradient payload as the sum of trainable parameter elements times each gradient element size. During D2H, both copies coexist, so immediate peak does not fall. After commit and reference release, allocated VRAM should fall by approximately the gradient payload, subject to views and allocator rounding; reserved VRAM may remain unchanged.

Part 4 - Experimental Design and Instrumentation

Measure allocated, reserved, and peak VRAM before forward, after backward, while copies are pending, and after commit. Store per-parameter norms and checksums. Compare the

offloaded run with a baseline at identical seed, model, batch, and loss. Verify that no optimizer step occurs with missing gradients.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def offload_gradients(model):
    transfer = torch.cuda.Stream()
    producer_done = torch.cuda.Event()
    producer_done.record(torch.cuda.current_stream())
    records = []
    with torch.cuda.stream(transfer):
        transfer.wait_event(producer_done)
        for name, parameter in model.named_parameters():
            if parameter.grad is None:
                continue
            source = parameter.grad
            destination = torch.empty_like(
                source, device="cpu", pin_memory=True)
            destination.copy_(source, non_blocking=True)
            source.record_stream(transfer)
            records.append((name, parameter, source, destination))
    transfer.synchronize()
    cpu_gradients = {}
    for name, parameter, source, destination in records:
        cpu_gradients[name] = destination
        parameter.grad = None
    return cpu_gradients

def E06(model, loss):
    loss.backward()
    before = torch.cuda.memory_allocated()
    gradients = offload_gradients(model)
    after = torch.cuda.memory_allocated()
    return gradients, before, after
```

Part 6 - Measurement, Validation, and Failure Analysis

Keeping `source` records after commit would keep gradient storage alive; discard the returned transaction metadata when it is no longer required. `empty_cache()` is not the proof: allocated bytes and tensor inventory are. Hooks can observe partial gradients, and shared parameters must be canonicalized rather than copied twice.

Part 7 - Engineering Interpretation and Philosophy

Gradient offload is a lifecycle transformation. The meaningful question is not whether bytes reached CPU, but whether accumulation, update order, authority, and failure recovery remain identical while the GPU copy is absent.

19.16.8 Exercise 07

Exercise Title: Move AdamW Optimizer States to CPU and Quantify the VRAM Saved by Removing First- and Second-Moment Buffers from the GPU

Part 1 - Exercise Contract and Why It Matters

Materialize AdamW state, inventory it, move tensor state to CPU without losing parameter association, and measure VRAM savings. Distinguish storage offload from execution: ordinary fused or foreach CUDA AdamW cannot update with moments resident on CPU.

Part 2 - System and Memory Mental Model

AdamW state is lazy and normally appears after the first step. Each parameter typically has first and second moments plus a step counter whose representation depends on implementation. Optimizer dictionaries remain keyed by parameter objects even when state tensors move. A later GPU step requires promotion, or the update must run on CPU.

Part 3 - Quantitative Prediction Before Execution

For full-precision moments, expect roughly two parameter-sized tensor sets, plus small step state. Mixed-precision training may also maintain master weights. Predict the temporary peak during D2H because GPU and CPU copies coexist, then compare unique state bytes with the observed allocated-VRAM drop.

Part 4 - Experimental Design and Instrumentation

Run one real optimizer step, classify every state value by parameter, key, dtype, device, and bytes, then move state. Verify exact tensor equality and preserved scalar values. Measure allocated and reserved VRAM before and after, and prove that an unmodified CUDA optimizer step is not silently attempted.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def optimizer_tensor_bytes(optimizer, device_type):
    total = 0
    for state in optimizer.state.values():
        for value in state.values():
            if torch.is_tensor(value):
                if value.device.type == device_type:
                    total += value.numel() * value.element_size()
    return total

def move_optimizer_state(optimizer, device):
    moved = 0
    for state in optimizer.state.values():
        for key in list(state.keys()):
            value = state[key]
            if not torch.is_tensor(value):
                continue
            moved += value.numel() * value.element_size()
```

```
            state[key] = value.detach().to(device).clone()
    return moved

def E07(optimizer):
    before = optimizer_tensor_bytes(optimizer, "cuda")
    moved = move_optimizer_state(optimizer, "cpu")
    after = optimizer_tensor_bytes(optimizer, "cuda")
    return before, moved, after
```

Part 6 - Measurement, Validation, and Failure Analysis

An empty optimizer state before the first step is not evidence of low memory. Moving only `exp_avg` while leaving `exp_avg_sq` on CUDA gives an incomplete result. Cloning can temporarily duplicate host state. Optimizer variants may require step tensors on a particular device, so restore and test the complete state contract before resuming GPU updates.

Part 7 - Engineering Interpretation and Philosophy

Optimizer state is attractive to offload because it is large and used once per step, but its movement lies directly on the update path. Capacity savings are real only when the chosen update architecture absorbs the resulting latency.

19.16.9 Exercise 08

Exercise Title: Implement a CPU-Based Optimizer Step for GPU Model Parameters and Measure the Cost of Round-Trip Parameter Synchronization

Part 1 - Exercise Contract and Why It Matters

Maintain CPU master parameters and AdamW moments, consume CPU gradients, update the masters, and copy new values to GPU. Compare the numerical update with a reference and account for both gradient D2H and parameter H2D traffic.

Part 2 - System and Memory Mental Model

Before the step, GPU parameters are the forward authority and CPU gradients are complete. The CPU update makes CPU masters authoritative. H2D synchronization then publishes a new GPU version before the next forward. No computation may read mixed parameter versions, and tied parameters require one master update.

Part 3 - Quantitative Prediction Before Execution

Each step moves approximately gradient bytes to CPU and updated parameter bytes to GPU. CPU stores require masters plus two Adam moments. Estimate transfer time from measured bandwidth and CPU update time from memory bandwidth; predict step latency as the non-overlapped critical path, not merely kernel time.

Part 4 - Experimental Design and Instrumentation

Use a tiny deterministic model, compare one update against `AdamW` in float64, and report maximum parameter error. Then benchmark large tensors with reused pinned staging. Record D2H, CPU update, H2D, total step time, RAM, and peak VRAM separately.

Part 5 - PyTorch/CUDA Implementation

```
import math
import torch

class CpuAdamW:
    def __init__(self, named_parameters, lr=1e-3,
                 betas=(0.9, 0.999), eps=1e-8, decay=0.01):
        self.lr = lr
        self.b1, self.b2 = betas
        self.eps, self.decay, self.step_number = eps, decay, 0
        self.items = []
        for name, parameter in named_parameters:
            master = parameter.detach().cpu().float().clone()
            first = torch.zeros_like(master)
            second = torch.zeros_like(master)
            self.items.append((name, parameter, master, first, second))

    @torch.no_grad()
    def step(self, cpu_gradients):
        self.step_number += 1
        correction1 = 1.0 - self.b1 ** self.step_number
        correction2 = 1.0 - self.b2 ** self.step_number
        for name, parameter, master, first, second in self.items:
            gradient = cpu_gradients[name].float()
            first.mul_(self.b1).add_(gradient, alpha=1.0 - self.b1)
            second.mul_(self.b2).addcmul_(
                gradient, gradient, value=1.0 - self.b2)
            denominator = second.sqrt() / math.sqrt(correction2)
            denominator.add_(self.eps)
            master.mul_(1.0 - self.lr * self.decay)
            master.addcdiv_(first / correction1,
                            denominator, value=-self.lr)
            parameter.copy_(master.to(parameter.device))

def E08(model):
    return CpuAdamW(model.named_parameters())
```

Part 6 - Measurement, Validation, and Failure Analysis

The compact implementation is educational, not a drop-in substitute for all AdamW variants. It casts masters to float32, omits parameter groups and sparse gradients, and performs blocking H2D copies. Validate update order, decay semantics, bias correction, ties, and publication completion before extending it with asynchronous staging.

Part 7 - Engineering Interpretation and Philosophy

A CPU optimizer converts expensive resident VRAM into repeated communication and host work. Its value depends on whether capacity is the hard constraint and whether update traffic can be amortized, sharded, overlapped, or reduced.

19.16.10 Exercise 09

Exercise Title: Compare Parameter Offload, Gradient Offload, and Optimizer-State Offload Independently to Determine Which Saves the Most VRAM

Part 1 - Exercise Contract and Why It Matters

Run a controlled ablation with four modes: full GPU, parameter offload only, gradient offload only, and optimizer-state offload only. Determine which saves the most peak and steady VRAM without mixing incompatible update mechanisms.

Part 2 - System and Memory Mental Model

The categories have different sizes, lifetimes, and reuse. Parameters are read throughout forward and backward; gradients live from production until reset; optimizer moments are large but touched only at the step. The same byte saving can therefore impose very different transfer frequency and synchronization.

Part 3 - Quantitative Prediction Before Execution

Predict per-mode resident bytes, transient copy peaks, D2H and H2D traffic, and critical-path time. Compute useful savings as baseline peak minus mode peak, and efficiency as saved peak bytes divided by transferred bytes per step. Keep RAM cost visible beside VRAM savings.

Part 4 - Experimental Design and Instrumentation

Use identical initialization, data, batch size, precision, accumulation, and number of measured steps. Warm up each mode in a fresh process. Report median and tail latency, samples per second, peak allocated VRAM, RAM peak, transfer bytes, final loss, and maximum parameter difference from its valid reference.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class AblationResult:
    mode: str
    peak_vram: int
    peak_ram: int
    transfer_bytes: int
    step_ms: float
    final_loss: float

def summarize_ablation(results, baseline_peak):
    rows = []
    for result in results:
        saved = baseline_peak - result.peak_vram
        if result.transfer_bytes == 0:
            efficiency = float("inf") if saved > 0 else 0.0
        else:
            efficiency = saved / result.transfer_bytes
        rows.append({
```

```
            "mode": result.mode,
            "saved_vram": saved,
            "ram_peak": result.peak_ram,
            "transfer_efficiency": efficiency,
            "step_ms": result.step_ms,
            "loss": result.final_loss,
        })
    return rows

def E09(results, baseline_peak):
    return summarize_ablation(results, baseline_peak)
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not compare modes that accidentally change optimizer semantics or effective batch size. Peak memory can occur at a different boundary in each mode, so one snapshot after the step is insufficient. Reserved memory is not live saving, and transferring a category twice per layer is not equivalent to once per step.

Part 7 - Engineering Interpretation and Philosophy

The largest tensor category is not automatically the best offload target. A defensible decision combines bytes saved, lifetime removed, reuse frequency, communication cost, RAM pressure, and semantic complexity.

19.16.11 Exercise 10

Exercise Title: Combine Parameter, Gradient, and Optimizer-State Offloading Under a Single Fixed GPU Memory Budget

Part 1 - Exercise Contract and Why It Matters

Create one controller that admits and evicts parameters, gradients, and optimizer state under a hard VRAM budget while also enforcing a host-RAM budget. Every decision must preserve authority, dependencies, and enough transient headroom to complete the next transfer or operator.

Part 2 - System and Memory Mental Model

The controller manages heterogeneous resources. Active parameters and saved activations may be protected; completed gradients can move only when their accumulation contract permits; optimizer moments must be present where the update executes. Pending transfers reserve source, destination, and stream lifetime simultaneously. A low watermark prevents constant emergency eviction.

Part 3 - Quantitative Prediction Before Execution

At each boundary calculate live GPU bytes, pending GPU bytes, next-operation workspace, required promotion bytes, CPU resident bytes, and pinned staging. A plan is admissible only when predicted peaks fit both budgets. Rank legal victims by saved VRAM divided by transfer and future-reload cost.

Part 4 - Experimental Design and Instrumentation

Replay a deterministic training-state trace through the policy before running CUDA. Log every admission, protection, offload, promotion, rejection, and watermark transition. Compare predicted against measured peaks and fail the run before allocation when either budget would be exceeded.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class StateResource:
    key: str
    category: str
    gpu_bytes: int
    cpu_bytes: int
    reload_ms: float
    next_use: int
    protected: bool = False
    on_gpu: bool = True

class UnifiedBudget:
    def __init__(self, gpu_limit, cpu_limit, reserve):
        self.gpu_limit = gpu_limit
        self.cpu_limit = cpu_limit
        self.reserve = reserve

    def choose_victims(self, resources, required_gpu):
        gpu_used = 0
        cpu_used = 0
        candidates = []
        for item in resources:
            if item.on_gpu:
                gpu_used += item.gpu_bytes
                if not item.protected:
                    score = item.gpu_bytes / max(item.reload_ms, 1e-9)
                    candidates.append((score, item))
            else:
                cpu_used += item.cpu_bytes
        candidates.sort(key=score_value, reverse=True)
        victims = []
        for score, item in candidates:
            if gpu_used + required_gpu + self.reserve <= self.gpu_limit:
                break
            if cpu_used + item.cpu_bytes > self.cpu_limit:
                continue
            victims.append(item)
            gpu_used -= item.gpu_bytes
            cpu_used += item.cpu_bytes
        if gpu_used + required_gpu + self.reserve > self.gpu_limit:
```

```
            raise MemoryError("no legal dual-budget plan")
        return victims

def score_value(pair):
    return pair[0]

def E10(gpu_limit, cpu_limit, reserve):
    return UnifiedBudget(gpu_limit, cpu_limit, reserve)
```

Part 6 - Measurement, Validation, and Failure Analysis

The named sorting callback keeps the decision rule explicit. More importantly, byte accounting must canonicalize tied storage and include pending copies. A GPU-feasible plan that exceeds RAM is invalid, as is a plan that evicts state required by an in-flight kernel.

Part 7 - Engineering Interpretation and Philosophy

Unified offloading is constrained scheduling, not three independent copies. Correctness comes from hard lifecycle rules; performance comes only after the remaining legal choices are optimized against measured costs.

19.16.12 Exercise 11

Exercise Title: Build an Asynchronous CPU Offload Queue Using Pinned Host Buffers and a Dedicated CUDA Transfer Stream

Part 1 - Exercise Contract and Why It Matters

Build a bounded D2H queue with reusable pinned destinations, a dedicated CUDA stream, completion events, source-lifetime protection, and explicit commit. Submission must never be confused with CPU-visible completion.

Part 2 - System and Memory Mental Model

The producer stream records readiness. The transfer stream waits on that event, copies into pinned RAM, and records completion. Until completion, the source and destination are protected and both budgets are reserved. Calling `record_stream` prevents the allocator from recycling source storage while the side stream still consumes it.

Part 3 - Quantitative Prediction Before Execution

Queue depth q can add up to q source payloads of protected VRAM and q pinned destinations in RAM. Predict service rate from D2H bandwidth and compare it with production rate. If arrival exceeds service, queue latency and memory must grow until backpressure activates.

Part 4 - Experimental Design and Instrumentation

Submit known tensors at a controlled rate, poll without global synchronization, and verify each committed checksum. Log queue depth, reserved bytes, submission latency, completion latency, backpressure time, and copy bandwidth. Inject a too-small queue budget and confirm safe rejection rather than OOM.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass
import torch

@dataclass
class PendingD2H:
    source: object
    destination: object
    done: object
    size_bytes: int

class CpuOffloadQueue:
    def __init__(self, max_pinned_bytes):
        self.stream = torch.cuda.Stream()
        self.max_pinned_bytes = max_pinned_bytes
        self.pending_bytes = 0
        self.pending = []

    def submit(self, source):
        size = source.numel() * source.element_size()
        if self.pending_bytes + size > self.max_pinned_bytes:
            raise MemoryError("pinned queue budget exceeded")
        ready = torch.cuda.Event()
        ready.record(torch.cuda.current_stream())
        destination = torch.empty_like(
            source, device="cpu", pin_memory=True)
        done = torch.cuda.Event()
        with torch.cuda.stream(self.stream):
            self.stream.wait_event(ready)
            destination.copy_(source, non_blocking=True)
            source.record_stream(self.stream)
            done.record(self.stream)
        item = PendingD2H(source, destination, done, size)
        self.pending.append(item)
        self.pending_bytes += size

    def poll(self):
        completed = []
        remaining = []
        for item in self.pending:
            if item.done.query():
                completed.append(item.destination)
                self.pending_bytes -= item.size_bytes
            else:
                remaining.append(item)
        self.pending = remaining
        return completed

def E11(max_pinned_bytes):
```

```
    return CpuOffloadQueue(max_pinned_bytes)
```

Part 6 - Measurement, Validation, and Failure Analysis

Fresh pinned allocation inside submission may serialize or add jitter; replace it with a size-class pool after validating semantics. Event completion is the commit boundary. Dropping the event, destination, or source too early creates a lifetime bug, while device-wide synchronization destroys concurrency.

Part 7 - Engineering Interpretation and Philosophy

An asynchronous queue is a protocol with ownership and backpressure. Its value is not that calls return early, but that useful compute proceeds while bounded, verifiable transfers complete independently.

19.16.13 Exercise 12

Exercise Title: Overlap GPU Computation with CPU Offload Transfers and Quantify How Much Transfer Latency Is Hidden

Part 1 - Exercise Contract and Why It Matters

Overlap D2H offload of completed state with independent GPU work, prove overlap on a timeline, and quantify hidden rather than merely submitted transfer time. Preserve only the producer-consumer dependencies required for correctness.

Part 2 - System and Memory Mental Model

Compute and transfer require separate streams and hardware engines capable of concurrency. The transfer waits for its source producer; later CPU consumption waits for transfer completion. Independent compute need not wait. Pageable RAM, default-stream coupling, or a global synchronization can serialize the design.

Part 3 - Quantitative Prediction Before Execution

For isolated copy time T_c and independent compute time T_g, the ideal combined time is approximately $\max(T_c, T_g)$ rather than $T_c + T_g$. Hidden copy time is $(T_c + T_g) - T_{overlap}$, bounded by $\min(T_c, T_g)$. Report overlap efficiency relative to that bound.

Part 4 - Experimental Design and Instrumentation

Measure copy-only, compute-only, sequential, and overlapped cases after warmup. Use events on both streams plus an outer synchronized interval. Export a trace to verify simultaneous copy-engine and compute-kernel activity. Check copied values after the completion event.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def overlap_once(source, matrix):
    transfer = torch.cuda.Stream()
    ready = torch.cuda.Event()
    copied = torch.cuda.Event()
```

```
    destination = torch.empty_like(
        source, device="cpu", pin_memory=True)
    ready.record(torch.cuda.current_stream())
    with torch.cuda.stream(transfer):
        transfer.wait_event(ready)
        destination.copy_(source, non_blocking=True)
        source.record_stream(transfer)
        copied.record(transfer)
    result = matrix @ matrix
    copied.synchronize()
    return destination, result

def overlap_metrics(copy_ms, compute_ms, overlapped_ms):
    hidden = copy_ms + compute_ms - overlapped_ms
    maximum = min(copy_ms, compute_ms)
    efficiency = hidden / maximum if maximum > 0 else 0.0
    return hidden, efficiency

def E12(source, matrix):
    return overlap_once(source, matrix)
```

Part 6 - Measurement, Validation, and Failure Analysis

The matrix operation must be independent of the offloaded tensor. CPU wall time without final synchronization measures submission, not completion. Lack of overlap may reflect hardware engine limits, too-small work, hidden stream dependencies, pageable destinations, or bandwidth contention rather than an incorrect API call.

Part 7 - Engineering Interpretation and Philosophy

Overlap does not make communication free; it places communication beneath a larger piece of independent work. The design succeeds only while dependencies remain correct and enough compute exists to cover the measured transfer.

19.16.14 Exercise 13

Exercise Title: Reproduce a CPU Offload Bottleneck Where PCIe Traffic Dominates Runtime Despite Large VRAM Savings

Part 1 - Exercise Contract and Why It Matters

Construct a workload that saves substantial VRAM yet becomes slower because state crosses PCIe too frequently. Diagnose the bottleneck with byte traffic, link utilization, copy-engine time, GPU idle time, and end-to-end throughput.

Part 2 - System and Memory Mental Model

Offloading reduces residency but adds communication. Low-reuse state may move profitably; parameters used every layer or microbatch can create repeated round trips. When transfer demand approaches sustainable PCIe bandwidth, prefetch cannot create capacity and compute streams wait for data.

Part 3 - Quantitative Prediction Before Execution

Compute traffic per step as all D2H and H2D payloads, including repeated loads. The bandwidth lower bound is traffic divided by measured sustainable bandwidth. If it approaches or exceeds baseline step time, large slowdown is predicted. Also report bytes transferred per byte of peak VRAM saved.

Part 4 - Experimental Design and Instrumentation

Sweep offload granularity from whole blocks to individual layers while holding model work constant. Measure traffic, copy duration, waiting time, kernel time, VRAM peak, and throughput. Export traces and correlate GPU gaps with parameter arrival events. Include a no-offload and idealized resident baseline.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class OffloadTelemetry:
    transferred_bytes: int
    saved_vram_bytes: int
    measured_bandwidth_Bps: float
    step_seconds: float
    gpu_idle_seconds: float

    def transfer_lower_bound(self):
        return self.transferred_bytes / self.measured_bandwidth_Bps

    def traffic_amplification(self):
        return self.transferred_bytes / max(self.saved_vram_bytes, 1)

    def pcie_dominates(self, threshold=0.6):
        ratio = self.transfer_lower_bound() / self.step_seconds
        return ratio >= threshold

def E13(telemetry):
    return {
        "lower_bound_s": telemetry.transfer_lower_bound(),
        "traffic_per_saved_byte": telemetry.traffic_amplification(),
        "pcie_dominates": telemetry.pcie_dominates(),
        "gpu_idle_fraction": (
            telemetry.gpu_idle_seconds / telemetry.step_seconds),
    }
```

Part 6 - Measurement, Validation, and Failure Analysis

Advertised PCIe rate is not the workload's sustainable bandwidth. Count both directions and copies caused by policy thrashing. An asynchronous trace full of copy activity can still be transfer-bound. Large VRAM savings should not be reported as success when samples per second collapses.

Part 7 - Engineering Interpretation and Philosophy

Capacity and throughput are separate objectives. CPU offload can make an otherwise impossible model executable, but the engineering report must expose the price and identify whether coarser residency or fewer state movements buys a better point on the trade-off curve.

19.16.15 Exercise 14

Exercise Title: Detect Host RAM Pressure Caused by Aggressive Offloading and Prevent the Process from Trading GPU OOM for CPU OOM

Part 1 - Exercise Contract and Why It Matters

Monitor managed tensor bytes, pinned bytes, process RSS, and system-available RAM, then reject or slow offload before host exhaustion. The safety contract is to avoid solving CUDA OOM by destabilizing the entire machine.

Part 2 - System and Memory Mental Model

Tensor payload is only part of RSS: allocator fragmentation, Python objects, DataLoader workers, file caches, and libraries also consume RAM. Pinned pages are especially constrained. System available memory is more actionable than the raw free counter. High and low watermarks create hysteresis and leave an emergency reserve for transient copies.

Part 3 - Quantitative Prediction Before Execution

Before a D2H transaction, predict final CPU payload plus temporary duplication, staging, worker prefetch, and safety reserve. Admission requires both managed budget headroom and sufficient system-available RAM. Choose thresholds below physical capacity; never stress real hardware to the operating-system limit.

Part 4 - Experimental Design and Instrumentation

Use an artificial budget, gradually submit known buffers, and confirm that the controller enters pressure mode at the high watermark and exits only below the low watermark. Record RSS, available RAM, managed pageable bytes, managed pinned bytes, queue depth, rejections, and reclaim actions.

Part 5 - PyTorch/CUDA Implementation

```
import os
import psutil

class HostPressureGuard:
    def __init__(self, managed_limit, reserve, high=0.9, low=0.75):
        self.managed_limit = managed_limit
        self.reserve = reserve
        self.high = high
        self.low = low
        self.managed_bytes = 0
        self.pressured = False

    def admit(self, requested_bytes):
```

```
        available = psutil.virtual_memory().available
        projected = self.managed_bytes + requested_bytes
        if projected >= self.high * self.managed_limit:
            self.pressured = True
        if available < requested_bytes + self.reserve:
            self.pressured = True
        if self.pressured:
            return False
        self.managed_bytes = projected
        return True

    def release(self, released_bytes):
        self.managed_bytes = max(0, self.managed_bytes - released_bytes)
        if self.managed_bytes <= self.low * self.managed_limit:
            self.pressured = False

    def process_rss(self):
        return psutil.Process(os.getpid()).memory_info().rss

def E14(managed_limit, reserve):
    return HostPressureGuard(managed_limit, reserve)
```

Part 6 - Measurement, Validation, and Failure Analysis

The example requires `psutil`; deployment code must handle its absence or use operating-system counters directly. RSS can fall later than tensor release, and available RAM changes because of unrelated processes. Admission and actual allocation must be transactional so a failed allocation rolls back reservations.

Part 7 - Engineering Interpretation and Philosophy

RAM is not an infinite extension of VRAM. A safe offloader treats the host as a shared, budgeted tier and prefers a controlled rejection or reduced workload to machine-wide swapping, pinned-page exhaustion, or process termination.

19.16.16 Exercise 15

Exercise Title: Build a RAM Budget Controller That Limits Offloaded Tensor Residency According to Available Host Memory

Part 1 - Exercise Contract and Why It Matters

Build a transactional controller for pageable and pinned offload residency. It must reserve bytes before allocation, commit only after success, reclaim legal entries to a low watermark, and reject work when host capacity cannot be made available safely.

Part 2 - System and Memory Mental Model

Reservations represent promised memory; committed bytes represent existing buffers. Both count against the budget. An entry can be reclaimed only when it is clean and reproducible elsewhere, or after dirty data is safely written to an authoritative tier. Pinned and pageable pools need separate sublimits even when they share a global RAM ceiling.

Part 3 - Quantitative Prediction Before Execution

Admission checks committed plus reserved plus requested bytes against the high watermark and system reserve. Reclamation continues toward the low watermark so the next small request does not immediately retrigger pressure. Predict worst transient memory when a replacement destination coexists with its victim.

Part 4 - Experimental Design and Instrumentation

Replay fixed and bursty size traces. Inject allocation failure between reserve and commit and prove rollback. Verify that protected and dirty-only copies are never discarded. Log admissions, denials, reclaimed bytes, pinned bytes, RSS, available memory, and time spent applying backpressure.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class HostEntry:
    key: str
    size_bytes: int
    pinned: bool
    protected: bool
    clean_replica: bool
    last_use: int

class RamBudgetController:
    def __init__(self, high_bytes, low_bytes, pinned_limit):
        self.high_bytes = high_bytes
        self.low_bytes = low_bytes
        self.pinned_limit = pinned_limit
        self.committed = 0
        self.reserved = 0
        self.pinned = 0
        self.entries = {}

    def reserve(self, size_bytes, pinned):
        if self.committed + self.reserved + size_bytes > self.high_bytes:
            return False
        if pinned and self.pinned + size_bytes > self.pinned_limit:
            return False
        self.reserved += size_bytes
        return True

    def commit(self, entry):
        if entry.size_bytes > self.reserved:
            raise RuntimeError("missing RAM reservation")
        self.reserved -= entry.size_bytes
        self.committed += entry.size_bytes
```

```
        if entry.pinned:
            self.pinned += entry.size_bytes
        self.entries[entry.key] = entry

    def rollback(self, size_bytes):
        self.reserved = max(0, self.reserved - size_bytes)

    def reclaimable(self):
        output = []
        for entry in self.entries.values():
            if not entry.protected and entry.clean_replica:
                output.append(entry)
        output.sort(key=entry_last_use)
        return output

def entry_last_use(entry):
    return entry.last_use

def E15(high_bytes, low_bytes, pinned_limit):
    return RamBudgetController(high_bytes, low_bytes, pinned_limit)
```

Part 6 - Measurement, Validation, and Failure Analysis

The example exposes reclaimable entries but leaves physical deletion to the owner, because deleting metadata before tensor release would falsify accounting. A robust implementation performs release and accounting as one commit. Shared storage must be counted once, and dirty authoritative CPU data is not a cache victim merely because it is old.

Part 7 - Engineering Interpretation and Philosophy

Memory budgets are admission-control systems. Their central purpose is to make failure predictable and local: work waits or is rejected before the process enters swap storms, allocation failure, or untraceable partial state.

19.16.17 Exercise 16

Exercise Title: Design a Cost-Aware Policy That Decides Which Training States Should Stay on GPU and Which Should Be Offloaded to CPU

Part 1 - Exercise Contract and Why It Matters

Design a policy that first applies correctness constraints and then ranks legal placement choices using size, reuse, next use, measured transfer cost, CPU update cost, recomputation option, and host pressure. Produce explanations for every decision rather than an opaque score alone.

Part 2 - System and Memory Mental Model

Residency has future value: keeping state avoids reload and waiting. Offloading has capacity value: it frees scarce VRAM. Parameters, gradients, moments, and activations have different consumers and mutability. A protected or sole dirty copy is ineligible regardless of score. Hysteresis prevents near-equal choices from causing repeated migration.

Part 3 - Quantitative Prediction Before Execution

Estimate keep benefit from expected future accesses times reload latency plus critical-path penalty. Estimate offload benefit from VRAM bytes freed times current pressure. Subtract D2H cost, CPU-memory penalty, and expected H2D reload. Calibrate weights from measurements and impose hard budget feasibility after ranking.

Part 4 - Experimental Design and Instrumentation

Train the cost model on measured bandwidth and per-category traces, then test on unseen stable, bursty, and variable-shape workloads. Compare against size-only, LRU, and static placement. Report OOM count, peak memories, migration bytes, stall time, throughput, and decision-prediction error.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class PlacementCandidate:
    key: str
    category: str
    size_bytes: int
    reuse_count: int
    next_use_distance: int
    d2h_ms: float
    h2d_ms: float
    cpu_pressure_cost: float
    protected: bool
    sole_dirty_copy: bool

def offload_score(item, pressure_weight, stall_weight):
    if item.protected or item.sole_dirty_copy:
        return float("-inf")
    capacity_value = pressure_weight * item.size_bytes
    urgency = 1.0 / max(item.next_use_distance, 1)
    future_stall = item.reuse_count * item.h2d_ms * urgency
    movement = item.d2h_ms + future_stall
    return capacity_value - stall_weight * movement \
        - item.cpu_pressure_cost

def rank_candidates(items, pressure_weight, stall_weight):
    scored = []
    for item in items:
        score = offload_score(item, pressure_weight, stall_weight)
        scored.append((score, item))
    scored.sort(key=score_component, reverse=True)
    return scored

def score_component(pair):
```

```
    return pair[0]

def E16(items):
    return rank_candidates(items, 1e-9, 1.0)
```

Part 6 - Measurement, Validation, and Failure Analysis

Score units must be normalized; raw bytes can dwarf milliseconds and make the formula meaningless. Future reuse is uncertain, so log prediction error and fall back to safe rules. Do not learn through correctness constraints. A high score never authorizes eviction of in-flight, shared, or uniquely dirty state.

Part 7 - Engineering Interpretation and Philosophy

A placement policy is an economic model operating beneath a correctness law. Measurements should revise its cost estimates, while invariants remain fixed. Explainable decisions make regressions and unexpected migration debuggable.

19.16.18 Exercise 17

Exercise Title: Compare Full-GPU Training Against CPU-Offloaded Training at Equal Batch Size and Report VRAM, RAM, Throughput, and Step Latency

Part 1 - Exercise Contract and Why It Matters

Benchmark full-GPU and CPU-offloaded training under identical semantics and batch size. Report capacity and performance together: peak VRAM, peak RAM, throughput, median and tail step latency, transfer traffic, and final numerical agreement.

Part 2 - System and Memory Mental Model

CUDA execution is asynchronous, so each measured step needs a well-defined completion boundary. RSS samples need a background peak monitor because the final value can miss transient host copies. Warmup materializes optimizer state, allocator segments, and kernels. Separate steady state from first-step costs.

Part 3 - Quantitative Prediction Before Execution

Before measurement, predict resident category bytes and transfer bytes for both modes. Estimate the offloaded lower bound from transfer bandwidth and CPU update time. Report VRAM reduction percentage, RAM increase, throughput loss, and milliseconds paid per GiB of peak VRAM saved.

Part 4 - Experimental Design and Instrumentation

Launch each mode in a clean process with the same seeds, data batches, model weights, optimizer hyperparameters, precision, and synchronization boundaries. Warm up, reset CUDA peak statistics, sample RAM at short intervals, and collect enough steps for median and p95. Compare loss and parameter checksums.

Part 5 - PyTorch/CUDA Implementation

```
import statistics
import time
import torch

def timed_steps(step_function, steps, examples_per_step):
    times = []
    torch.cuda.reset_peak_memory_stats()
    torch.cuda.synchronize()
    total_start = time.perf_counter()
    for _ in range(steps):
        torch.cuda.synchronize()
        start = time.perf_counter()
        step_function()
        torch.cuda.synchronize()
        times.append((time.perf_counter() - start) * 1000.0)
    elapsed = time.perf_counter() - total_start
    ordered = sorted(times)
    p95_index = max(0, int(0.95 * len(ordered)) - 1)
    return {
        "median_step_ms": statistics.median(times),
        "p95_step_ms": ordered[p95_index],
        "examples_per_second": steps * examples_per_step / elapsed,
        "peak_vram_bytes": torch.cuda.max_memory_allocated(),
    }

def E17(full_gpu_step, offload_step, steps, batch_size):
    full = timed_steps(full_gpu_step, steps, batch_size)
    offloaded = timed_steps(offload_step, steps, batch_size)
    return full, offloaded
```

Part 6 - Measurement, Validation, and Failure Analysis

Running both modes sequentially in one process can contaminate allocator and kernel state; subprocess isolation is stronger. The sample function measures CUDA completion but omits the required RAM-monitor thread and transfer counters, which must be included in the full experiment. Equal batch size alone is not enough if accumulation, loss scaling, or update precision differs.

Part 7 - Engineering Interpretation and Philosophy

The comparison is a Pareto analysis, not a contest with one universal winner. Full GPU favors latency; offloading buys feasibility and batch capacity. The right result states exactly how much capacity was purchased and its recurring cost.

19.16.19 Exercise 18

Exercise Title: Stress-Test CPU Offloading on a Model That Cannot Fit All Training State in VRAM and Diagnose Transfer Stalls, RAM Growth, and Synchronization Errors

Part 1 - Exercise Contract and Why It Matters

Run an oversized training-state workload under artificial VRAM and RAM limits, vary shapes and reuse, inject synchronization mistakes, and require the runtime to diagnose stalls, unbounded host growth, stale reads, and illegal authority transitions before data corruption or physical OOM.

Part 2 - System and Memory Mental Model

Stress combines several feedback loops: eviction creates transfers, transfers occupy staging buffers, buffers increase RAM, slow service grows queues, and aggressive prefetch restores VRAM pressure. Correctness additionally depends on events, stream lifetimes, version publication, tied ownership, and autograd dependencies.

Part 3 - Quantitative Prediction Before Execution

Derive maximum queue bytes from arrival and service rates, and predict the working-set minimum needed to avoid thrashing. Budget transient duplication and next-operator workspace. Define alarms for sustained queue growth, transfer wait fraction, RAM slope, repeated migrations, version mismatch, and invariant failure.

Part 4 - Experimental Design and Instrumentation

Start with a simulator, then use scaled real CUDA tensors below safe physical limits. Sweep model depth, tensor size, queue depth, bandwidth delay, and memory budgets. Inject a missing wait, early reference release, and leaked CPU entry. Require each fault to trigger a named diagnostic and preserve a trace.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class StressSample:
    step: int
    gpu_bytes: int
    cpu_bytes: int
    queue_bytes: int
    transfer_wait_ms: float
    migrations: int
    version_errors: int

class StressDiagnostics:
    def __init__(self, gpu_limit, cpu_limit):
        self.gpu_limit = gpu_limit
        self.cpu_limit = cpu_limit
        self.previous = None

    def inspect(self, sample):
        alarms = []
        if sample.gpu_bytes > self.gpu_limit:
            alarms.append("VRAM budget violation")
        if sample.cpu_bytes > self.cpu_limit:
            alarms.append("RAM budget violation")
```

```
        if sample.version_errors > 0:
            alarms.append("synchronization or authority error")
        if self.previous is not None:
            ram_growth = sample.cpu_bytes - self.previous.cpu_bytes
            queue_growth = sample.queue_bytes - self.previous.queue_bytes
            if ram_growth > 0 and queue_growth > 0:
                alarms.append("offload backlog and host growth")
            if sample.migrations - self.previous.migrations > 20:
                alarms.append("placement thrashing")
        self.previous = sample
        return alarms

def E18(gpu_limit, cpu_limit):
    return StressDiagnostics(gpu_limit, cpu_limit)
```

Part 6 - Measurement, Validation, and Failure Analysis

One positive slope is not a leak; require persistence over a window and account for warmup. An empty alarm list does not prove numerical correctness, so compare losses, gradients, versions, and parameter checksums with a smaller resident reference. Never discover physical limits by exhausting a shared workstation.

Part 7 - Engineering Interpretation and Philosophy

Stress testing asks whether the controller fails intelligibly at its boundary. An oversized model is a useful systems test only when limits are artificial, invariants are observable, and every recovery action leaves authority coherent.

19.16.20 Exercise 19

Exercise Title: Build and Benchmark a PyTorch CPU-Offload Runtime That Dynamically Moves Parameters, Gradients, and Optimizer State Between RAM and VRAM Under Fixed Memory Budgets

Part 1 - Exercise Contract and Why It Matters

Integrate a registry, dual-budget admission, pinned-buffer pool, asynchronous transactions, cost-aware placement, versioned authority, backpressure, instrumentation, and recovery into one runtime. Support parameters, gradients, and optimizer state without claiming that one movement rule fits all three.

Part 2 - System and Memory Mental Model

The registry is the control plane; CPU and GPU tensors are data-plane replicas. Every resource has a category-specific lifecycle, canonical identity, version, authority, protection set, next consumer, and pending transaction. Promotions publish GPU versions before use; demotions publish CPU versions before release. Budget reservations make every transition atomic from the controller's view.

Part 3 - Quantitative Prediction Before Execution

Given a future layer schedule, simulate residency and calculate peak committed and reserved bytes on both tiers, staging depth, transfer volume, predicted stall, and workspace headroom.

Refuse execution if no legal schedule exists. Benchmark prediction error as well as performance because a poor cost model can turn a safe controller into a thrashing one.

Part 4 - Experimental Design and Instrumentation

Validate individual transitions, then one inference, one training step, several accumulation steps, checkpoint save, interruption, and restore. Benchmark fixed and variable shapes against full GPU and static offload. Report correctness, peak VRAM, peak RAM, transfer traffic, queue depth, median and p95 latency, throughput, overlap, policy decisions, retries, and rejected admissions.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass, field

@dataclass
class RuntimeRecord:
    key: str
    category: str
    size_bytes: int
    authority: str
    version: int
    cpu_tensor: object = None
    gpu_tensor: object = None
    pending: object = None
    protected: set = field(default_factory=set)

class CpuOffloadRuntime:
    def __init__(self, gpu_budget, ram_budget, queue):
        self.gpu_budget = gpu_budget
        self.ram_budget = ram_budget
        self.queue = queue
        self.records = {}
        self.gpu_committed = 0
        self.ram_committed = 0

    def register(self, record):
        if record.key in self.records:
            raise KeyError("duplicate canonical state key")
        self.records[record.key] = record
        if record.gpu_tensor is not None:
            self.gpu_committed += record.size_bytes
        if record.cpu_tensor is not None:
            self.ram_committed += record.size_bytes
        self.check_budgets()

    def check_budgets(self):
        if self.gpu_committed > self.gpu_budget:
            raise MemoryError("managed VRAM budget exceeded")
        if self.ram_committed > self.ram_budget:
```

```
            raise MemoryError("managed RAM budget exceeded")

    def request_offload(self, key):
        record = self.records[key]
        if record.protected:
            raise RuntimeError("resource is protected")
        if record.gpu_tensor is None or record.pending is not None:
            raise RuntimeError("illegal offload state")
        if self.ram_committed + record.size_bytes > self.ram_budget:
            raise MemoryError("no RAM headroom for D2H destination")
        self.queue.submit(record.gpu_tensor)
        record.pending = "d2h"

    def commit_cpu(self, key, cpu_tensor):
        record = self.records[key]
        if record.pending != "d2h":
            raise RuntimeError("D2H completion was not expected")
        record.cpu_tensor = cpu_tensor
        record.gpu_tensor = None
        record.pending = None
        record.authority = "cpu"
        self.ram_committed += record.size_bytes
        self.gpu_committed -= record.size_bytes
        self.check_budgets()

def E19(gpu_budget, ram_budget, queue):
    return CpuOffloadRuntime(gpu_budget, ram_budget, queue)
```

Part 6 - Measurement, Validation, and Failure Analysis

The compact runtime shows control-plane invariants, not a universal training framework. A complete implementation must add H2D promotion, pooled pinned buffers, transaction IDs, rollback, tied-storage canonicalization, optimizer integration, checkpoint serialization, and process-failure recovery. Never commit from queue order alone; match the exact completion to its record.

Part 7 - Engineering Interpretation and Philosophy

The final runtime is a memory operating system for training state. Its quality is determined first by coherent authority and bounded failure, then by how well it converts transfer opportunities into useful model capacity without hiding cost, weakening numerical semantics, or exhausting the host.

19.17 DMM15

Chapter Title: NVMe Offloading and Three-Tier Memory Hierarchies
Specialization: Dynamic Memory Management and CUDA Systems

19.17.1 Exercise 00

Exercise Title: Build a Precise Mental Model of VRAM, RAM, and NVMe as a Three-Tier Memory Hierarchy with Different Capacity, Bandwidth, and Latency

Part 1 - Exercise Contract and Why It Matters

Build a quantitative model of three distinct storage tiers: GPU VRAM for active compute, CPU RAM for staging and hot state, and NVMe for large cold state. The contract must track authority, clean replicas, dirty state, residency, pending I/O, protection reasons, next use, and budgets for all three tiers.

Part 2 - System and Memory Mental Model

VRAM offers device-local bandwidth and kernel access but limited capacity. RAM is larger and CPU-addressable, yet GPU use requires H2D transfer. NVMe is much larger but block-oriented and high-latency; it cannot feed CUDA kernels as an ordinary tensor. Data normally travels NVMe to page cache or a host buffer, then through pinned staging to VRAM. Memory mapping changes when pages are faulted, not the physical need to move bytes through the hierarchy.

Part 3 - Quantitative Prediction Before Execution

For a shard of M bytes, predict $T_{IO} = L_{IO} + M/B_{IO}$ and $T_{H2D} = L_{H2D} + M/B_{H2D}$. Without overlap, delivery is their sum. With a three-stage pipeline, steady-state throughput is bounded by the slowest stage. Add file metadata, page cache, pinned buffers, GPU replicas, next-operation workspace, and transient source-destination coexistence to the budgets.

Part 4 - Experimental Design and Instrumentation

Measure cold and warm NVMe reads separately, then pageable-to-CUDA and pinned-to-CUDA transfer. Record logical bytes, physical I/O when available, minor and major faults, page-cache state, RAM RSS, pinned bytes, allocated and peak VRAM, queue depth, and GPU idle time. Use an isolated benchmark file and artificial budgets rather than exhausting the workstation.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass, field

@dataclass
class TieredState:
    key: str
    size_bytes: int
    authority: str
    nvme_path: str
    in_ram: bool = False
    in_vram: bool = False
    dirty: bool = False
    pending_stage: str = "none"
    version: int = 0
    protected: set = field(default_factory=set)
```

```
    def begin(self, stage):
        if self.pending_stage != "none":
            raise RuntimeError("another movement is pending")
        self.pending_stage = stage

    def commit_gpu(self):
        if self.pending_stage != "ram_to_vram":
            raise RuntimeError("unexpected H2D commit")
        self.in_ram = True
        self.in_vram = True
        self.pending_stage = "none"

def E00(key, size_bytes, path):
    return TieredState(key, size_bytes, "nvme", path)
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not call mapped virtual bytes resident RAM, or page-cache hits NVMe speed. Do not infer disk I/O from Python elapsed time alone. A file write is not durable until the required flush and filesystem guarantees complete. Duplicate views, compressed files, filesystem cache, and asynchronous submissions can all make logical accounting differ from physical movement.

Part 7 - Engineering Interpretation and Philosophy

Three-tier offloading is a scheduling problem governed by locality. NVMe makes capacity possible only when future use is predictable enough to hide I/O and when authority remains coherent across slow, failure-prone transitions.

19.17.2 Exercise 01

Exercise Title: Benchmark Sequential NVMe Read and Write Throughput and Compare It Against Measured RAM-to-VRAM Transfer Bandwidth

Part 1 - Exercise Contract and Why It Matters

Measure sequential NVMe read and write service separately from RAM-to-VRAM transfer, then determine which stage limits parameter delivery. Report cold and warm storage behavior and never compare buffered file submission with completed CUDA transfer.

Part 2 - System and Memory Mental Model

Filesystem writes may land in the page cache before durable storage. Repeated reads may be served from RAM. H2D copies require completed CUDA timing and gain asynchronous capability from pinned sources. Sequential bandwidth reflects large contiguous requests; it does not predict small-shard random access.

Part 3 - Quantitative Prediction Before Execution

Convert every result to bytes per second using completed bytes divided by completed time. Predict pipeline rate as the minimum of NVMe read bandwidth and H2D bandwidth, then include per-request latency. Calculate the shard size needed to amortize fixed latency and the maximum theoretical overlap speedup.

Part 4 - Experimental Design and Instrumentation

Create a file several times larger than the test block, warm up devices, use multiple offsets, and state whether caches were warm. Time file open, read, write, flush, buffer pinning, H2D submission, and H2D completion separately. Repeat trials and report median, p95, and variance without issuing unsafe cache drop commands on a shared system.

Part 5 - PyTorch/CUDA Implementation

```
import os
import time
import torch

def sequential_read(path, block_bytes):
    transferred = 0
    start = time.perf_counter()
    with open(path, "rb", buffering=0) as handle:
        while True:
            data = handle.read(block_bytes)
            if not data:
                break
            transferred += len(data)
    elapsed = time.perf_counter() - start
    return transferred, elapsed, transferred / elapsed

def pinned_h2d_bandwidth(numel, repetitions):
    source = torch.empty(numel, dtype=torch.float32,
                         pin_memory=True)
    start = torch.cuda.Event(enable_timing=True)
    end = torch.cuda.Event(enable_timing=True)
    start.record()
    for _ in range(repetitions):
        destination = source.to("cuda", non_blocking=True)
    end.record()
    end.synchronize()
    seconds = start.elapsed_time(end) / 1000.0
    total = source.numel() * source.element_size() * repetitions
    return total / seconds, destination

def E01(path, block_bytes, numel, repetitions):
    storage = sequential_read(path, block_bytes)
    transfer = pinned_h2d_bandwidth(numel, repetitions)
    return storage, transfer
```

Part 6 - Measurement, Validation, and Failure Analysis

The example's file read may be cached and its H2D loop allocates destinations; the full benchmark must reuse destinations and disclose cache state. For write durability, add flush and platform-appropriate synchronization deliberately. Background I/O, thermal throttling, filesystem compression, and shared PCIe topology can change both stages.

Part 7 - Engineering Interpretation and Philosophy

Bandwidth numbers matter only when tied to the actual transfer path. The engine must be designed for the slowest measured stage and its latency distribution, not for a vendor's peak specification.

19.17.3 Exercise 02

Exercise Title: Serialize Large PyTorch Parameter Tensors into Independent NVMe Shards and Reconstruct the Original Model Exactly

Part 1 - Exercise Contract and Why It Matters

Write each canonical parameter into an independently loadable shard, store a manifest, and reconstruct the model with exact names, shapes, dtypes, values, and tied relationships. A partially written generation must never replace the last valid model.

Part 2 - System and Memory Mental Model

The shard payload and metadata form one logical checkpoint. Canonical storage prevents tied parameters from becoming independent copies. A generation is published only after all temporary files, hashes, and manifest are complete. Loading untrusted pickle-based artifacts is unsafe; tensor-only formats and trusted manifests reduce the attack surface.

Part 3 - Quantitative Prediction Before Execution

Predict payload as unique parameter elements times element size, plus metadata, filesystem block rounding, and hashes. Estimate total creation time from serialization CPU work and sequential write bandwidth. Reassembly RAM depends on whether shards are loaded one at a time or all together.

Part 4 - Experimental Design and Instrumentation

Use a model with shared weights, noncontiguous source views, several dtypes, and buffers. Save into a private temporary directory, interrupt before manifest publication, and confirm the previous generation remains usable. Verify names, metadata, hashes, exact equality, and shared-parameter identity after restore.

Part 5 - PyTorch/CUDA Implementation

```
import hashlib
import json
from pathlib import Path
import torch

def tensor_digest(tensor):
    raw = tensor.detach().cpu().contiguous().numpy().tobytes()
    return hashlib.sha256(raw).hexdigest()

def write_parameter_shards(model, directory):
    root = Path(directory)
    root.mkdir(parents=True, exist_ok=True)
    manifest = []
```

```
    index = 0
    for name, parameter in model.named_parameters():
        cpu_tensor = parameter.detach().cpu().contiguous()
        filename = "parameter_%06d.pt" % index
        torch.save(cpu_tensor, root / filename)
        manifest.append({
            "name": name,
            "file": filename,
            "shape": list(cpu_tensor.shape),
            "dtype": str(cpu_tensor.dtype),
            "sha256": tensor_digest(cpu_tensor),
        })
        index += 1
    temporary = root / "manifest.json.tmp"
    final = root / "manifest.json"
    temporary.write_text(json.dumps(manifest, indent=2))
    temporary.replace(final)
    return manifest

def E02(model, directory):
    return write_parameter_shards(model, directory)
```

Part 6 - Measurement, Validation, and Failure Analysis

`named_parameters()` normally removes duplicate parameter objects, so a separate alias map is required to reconstruct ties. NumPy conversion needs a supported CPU dtype. Atomic rename alone does not guarantee durable data after power failure; production code must define directory and file synchronization, permissions, versioning, and recovery semantics.

Part 7 - Engineering Interpretation and Philosophy

Sharding converts a monolithic checkpoint into schedulable units, but it also creates a consistency protocol. Correct reconstruction and atomic publication are more important than maximizing parallel file creation.

19.17.4 Exercise 03

Exercise Title: Compare Loading a Complete PyTorch Checkpoint into RAM Against torch.load(..., mmap=True) Lazy Storage Access

Part 1 - Exercise Contract and Why It Matters

Compare eager checkpoint loading with memory-mapped loading while separating open latency, virtual address mapping, page faults, resident RAM growth, and time to touch or transfer each tensor. Prove that a quick mapping is not a completed model load.

Part 2 - System and Memory Mental Model

Ordinary loading reads tensor storages into memory during deserialization. With memory mapping enabled, storages are mapped and pages are faulted on demand. The operating system may retain them in page cache and evict them under pressure. Mapped virtual size can be large while resident physical RAM is initially low.

Part 3 - Quantitative Prediction Before Execution

Predict eager startup from total checkpoint bytes and storage bandwidth. For mapped loading, predict small metadata startup followed by per-tensor fault cost. If every tensor is eventually touched once, total physical I/O remains similar; the advantage is bounded residency, deferral, and selective access rather than eliminating reads.

Part 4 - Experimental Design and Instrumentation

In fresh processes, measure eager and mapped open, RSS immediately afterward, first-touch time per tensor, full-touch time, major/minor faults, and transfer to CUDA. Repeat warm and cold-cache conditions and preserve identical `map_location` and `weights_only` settings.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

def load_and_touch(path, use_mmap):
    start_open = time.perf_counter()
    state = torch.load(
        path,
        map_location="cpu",
        mmap=use_mmap,
        weights_only=True)
    open_seconds = time.perf_counter() - start_open

    checksum = 0.0
    start_touch = time.perf_counter()
    for tensor in state.values():
        if torch.is_tensor(tensor):
            checksum += tensor.float().sum().item()
    touch_seconds = time.perf_counter() - start_touch
    return open_seconds, touch_seconds, checksum

def E03(path):
    eager = load_and_touch(path, False)
    mapped = load_and_touch(path, True)
    return eager, mapped
```

Part 6 - Measurement, Validation, and Failure Analysis

Running eager then mapped in one process warms the page cache and biases the second result; use isolated processes or clearly label warm-cache data. RSS does not equal mapped virtual size. Accessing a single element does not represent a full tensor scan, and transferring to CUDA forces the required pages to become available before the copy can complete.

Part 7 - Engineering Interpretation and Philosophy

Memory mapping changes the timing and granularity of payment. It is valuable when the engine can exploit selective access and bounded working sets; it does not transform NVMe into RAM or make first-touch latency disappear.

19.17.5 Exercise 04

Exercise Title: Create File-Backed CPU Tensor Storage with torch.from_file() and Verify That Tensor Data Can Remain Backed by NVMe

Part 1 - Exercise Contract and Why It Matters

Create a CPU tensor whose storage is memory-mapped from a file, verify shape and dtype interpretation, distinguish shared and private mappings, and measure how touching pages changes residency. Demonstrate that CUDA still requires an explicit staged copy.

Part 2 - System and Memory Mental Model

`torch.from_file` maps a flat CPU storage; the programmer supplies the element count and dtype, then reshapes the view. In shared mode, modifications can be visible in the file; private mode does not provide the same persistence contract. The file's byte length, offset convention, endianness, dtype, and layout must match the manifest exactly.

Part 3 - Quantitative Prediction Before Execution

Required file payload is element count times dtype size. Predict initial virtual mapping with little resident growth, then page-fault and page-cache growth as regions are touched. A contiguous CUDA copy may fault the entire requested range, after which H2D time remains bounded by host-device bandwidth.

Part 4 - Experimental Design and Instrumentation

Write a known tensor, map it, reshape it, compare every value, mutate one region under shared and private modes, reopen the file, and verify persistence rules. Measure open, first touch, repeated touch, RSS, faults, and H2D completion. Reject truncated or oversized metadata before mapping.

Part 5 - PyTorch/CUDA Implementation

```
from pathlib import Path
import torch

def create_file_backed_tensor(path, shape, dtype, shared):
    elements = 1
    for dimension in shape:
        elements *= dimension
    probe = torch.empty((), dtype=dtype)
    required_bytes = elements * probe.element_size()
    actual_bytes = Path(path).stat().st_size
    if actual_bytes < required_bytes:
        raise ValueError("backing file is smaller than tensor metadata")
    flat = torch.from_file(
        str(path), shared=shared, size=elements, dtype=dtype)
    return flat.reshape(shape)

def E04(path, rows, columns):
    mapped = create_file_backed_tensor(
        path, (rows, columns), torch.float32, True)
```

```
    checksum = mapped.sum().item()
    cuda_copy = mapped.to("cuda")
    return mapped, cuda_copy, checksum
```

Part 6 - Measurement, Validation, and Failure Analysis

A successful reshape proves only element count, not semantic metadata. Incorrect dtype can produce plausible but meaningless numbers. Mapped storage is CPU-only and is not automatically pinned. File truncation while mapped, concurrent writes, missing flush policy, or mutable shared mappings can corrupt state.

Part 7 - Engineering Interpretation and Philosophy

File-backed tensors make storage addressable through tensor operations, but they do not erase the hierarchy. They are useful when metadata, ownership, and access locality are explicit enough to make page faults predictable.

19.17.6 Exercise 05

Exercise Title: Build an NVMe Parameter Store That Maps Every Model Parameter to a File, Byte Offset, Shape, Dtype, and Residency State

Part 1 - Exercise Contract and Why It Matters

Build a canonical parameter store whose manifest identifies every tensor by file, byte offset, length, shape, stride policy, dtype, checksum, owners, version, and residency. The store must distinguish durable authority from RAM and VRAM cache replicas.

Part 2 - System and Memory Mental Model

A parameter is a logical object; its bytes may have replicas in three tiers. The manifest is the address translation table. Offsets need alignment and must not overlap. Shared or tied parameters map several names to one canonical record. Dirty GPU or RAM state cannot be evicted to an older NVMe generation.

Part 3 - Quantitative Prediction Before Execution

Calculate each payload, alignment padding, file size, metadata size, RAM cache, pinned staging, and VRAM replica. Predict the number of file opens and seeks for a layer trace. A packed file favors sequential reads; independent files simplify replacement but can increase metadata and open overhead.

Part 4 - Experimental Design and Instrumentation

Register a model with ordinary and tied parameters, validate non-overlapping ranges, reopen the store, read every record, and compare checksums. Exercise NVMe-only, RAM-hot, H2D-pending, VRAM-clean, VRAM-dirty, and writeback-pending transitions. Corrupt one manifest offset and require early detection.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass, field

@dataclass
class ParameterRecord:
    key: str
    owners: list
    path: str
    offset_bytes: int
    length_bytes: int
    shape: tuple
    dtype: str
    checksum: str
    authority: str = "nvme"
    residency: str = "nvme_only"
    version: int = 0
    protected: set = field(default_factory=set)

class NvmeParameterStore:
    def __init__(self):
        self.records = {}

    def add(self, record):
        if record.key in self.records:
            raise KeyError("duplicate canonical parameter")
        for existing in self.records.values():
            if existing.path != record.path:
                continue
            left = max(existing.offset_bytes, record.offset_bytes)
            right = min(
                existing.offset_bytes + existing.length_bytes,
                record.offset_bytes + record.length_bytes)
            if left < right:
                raise ValueError("overlapping parameter ranges")
        self.records[record.key] = record

def E05():
    return NvmeParameterStore()
```

Part 6 - Measurement, Validation, and Failure Analysis

String dtype names require a validated conversion table before reconstruction. Offsets must be interpreted in bytes, not elements. Checksums verify content but do not replace atomic generation publication. Views with shared storage need a canonical base record plus view metadata rather than duplicated payload.

Part 7 - Engineering Interpretation and Philosophy

The parameter store is a small virtual-memory system. Its manifest makes placement replaceable while preserving identity, and its authority rules make dynamic movement safe rather than merely convenient.

19.17.7 Exercise 06

Exercise Title: Load One Parameter Shard from NVMe to CPU Only When Its Owning Layer Is About to Execute

Part 1 - Exercise Contract and Why It Matters

Implement just-in-time shard loading at the layer boundary. Load exactly the canonical state needed by the upcoming layer, verify it, bind it without losing ties, execute, and release or cache it according to the next-use schedule.

Part 2 - System and Memory Mental Model

On-demand loading minimizes RAM residency but puts storage latency directly on the critical path. A pre-forward hook knows the next layer, yet loading inside that hook is synchronous unless a prior prefetch exists. Parameter replacement must preserve module registration and autograd expectations; inference is the simpler first target.

Part 3 - Quantitative Prediction Before Execution

Peak RAM includes the current shard, read buffer, decoded tensor, and possibly the previous shard awaiting release. Step latency gains the sum of unhidden read times. Predict when caching a reused shard costs fewer bytes than re-reading it, using next-use distance and measured I/O cost.

Part 4 - Experimental Design and Instrumentation

Begin with an inference-only sequential model. Record layer start, read start, read completion, validation, binding, execution, and release. Compare output with a resident reference. Then repeat with shared layers and nonsequential control flow to expose assumptions in the schedule.

Part 5 - PyTorch/CUDA Implementation

```
from pathlib import Path
import torch

class JustInTimeShardLoader:
    def __init__(self, manifest):
        self.manifest = manifest
        self.active = {}

    def load_cpu(self, key):
        record = self.manifest[key]
        path = Path(record["path"])
        tensor = torch.load(
            path, map_location="cpu", weights_only=True)
        expected_shape = tuple(record["shape"])
        if tuple(tensor.shape) != expected_shape:
            raise ValueError("parameter shape mismatch")
        self.active[key] = tensor
        return tensor
```

```
    def release(self, key):
        if key in self.active:
            del self.active[key]

def E06(manifest):
    return JustInTimeShardLoader(manifest)
```

Part 6 - Measurement, Validation, and Failure Analysis

Loading a tensor does not automatically install it into a module. Production binding must use a controlled parameter materialization mechanism and preserve dtype, device, requires-grad, aliases, and version. Calling `torch.load` per tiny parameter magnifies open and metadata costs; shard at a useful layer or block granularity.

Part 7 - Engineering Interpretation and Philosophy

Just-in-time loading trades memory certainty for latency certainty. It is a correct baseline that reveals the real I/O cost; prefetching should be added only after this baseline preserves exact model behavior.

19.17.8 Exercise 07

Exercise Title: Stage NVMe Parameters Through a Reusable Pinned CPU Buffer Before Performing a Non-Blocking Transfer to CUDA

Part 1 - Exercise Contract and Why It Matters

Read a shard into bounded host memory, copy it into a reusable pinned buffer, then submit non-blocking H2D and publish the CUDA tensor only after completion. Protect the pinned slot from reuse throughout both CPU filling and DMA.

Part 2 - System and Memory Mental Model

Ordinary file APIs fill pageable memory. Pinned staging adds a CPU copy but enables the DMA path needed for asynchronous H2D. A slot has states free, filling, CPU-ready, H2D-pending, and free-after-event. Reusing it before the CUDA event completes can silently transfer bytes belonging to a later shard.

Part 3 - Quantitative Prediction Before Execution

Delivery time contains NVMe read, pageable-to-pinned copy, and H2D transfer. For one shard they add; across shards they may pipeline. Budget pageable read storage, pinned slot, CUDA destination, and transient duplicates. Select buffer size from the largest admitted shard or use bounded size classes.

Part 4 - Experimental Design and Instrumentation

Compare fresh and reused pinned allocation, verify byte equality after each event, and log slot state, I/O time, host-copy time, H2D time, total time, RAM, pinned bytes, and VRAM. Inject early slot reuse and require the checksum or state machine to detect it.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class PinnedStage:
    def __init__(self, elements, dtype):
        self.buffer = torch.empty(
            elements, dtype=dtype, pin_memory=True)
        self.stream = torch.cuda.Stream()
        self.done = None
        self.busy = False

    def submit(self, pageable_tensor):
        if self.busy:
            raise RuntimeError("pinned slot is still busy")
        if pageable_tensor.numel() > self.buffer.numel():
            raise ValueError("shard exceeds pinned slot")
        view = self.buffer[:pageable_tensor.numel()]
        view.copy_(pageable_tensor.reshape(-1))
        destination = torch.empty_like(view, device="cuda")
        self.done = torch.cuda.Event()
        with torch.cuda.stream(self.stream):
            destination.copy_(view, non_blocking=True)
            self.done.record(self.stream)
        self.busy = True
        return destination

    def reclaim(self):
        if self.busy and self.done.query():
            self.busy = False
            self.done = None
            return True
        return False

def E07(elements, dtype):
    return PinnedStage(elements, dtype)
```

Part 6 - Measurement, Validation, and Failure Analysis

Returning the destination does not mean it is ready on an unrelated stream; the consumer must wait on the completion event. CPU filling is complete before submission in this example. A fully asynchronous disk reader needs a separate future and lifetime. Dtype conversion or reshape must not change payload bytes without updating metadata and checksums.

Part 7 - Engineering Interpretation and Philosophy

Pinned staging is an explicit bridge between two asynchronous systems. Its correct unit is not a tensor alone but a buffer lease whose lifetime ends only when the final consumer dependency has been satisfied.

19.17.9 Exercise 08

Exercise Title: Compare Direct Lazy File Access, Pageable RAM Staging, and Pinned RAM Staging for NVMe-to-GPU Parameter Delivery

Part 1 - Exercise Contract and Why It Matters

Compare three delivery paths under identical shards: mapped lazy access followed by CUDA copy, explicit pageable staging, and explicit pageable-to-pinned staging. Measure complete first-use and steady-state latency, not file-open or H2D submission alone.

Part 2 - System and Memory Mental Model

Mapped access delegates paging to the OS and can avoid explicit full reads. Pageable staging makes I/O explicit but may require internal CUDA staging. Pinned staging adds a host copy and locked RAM while enabling asynchronous H2D. Warm page cache can make every path look like RAM rather than NVMe.

Part 3 - Quantitative Prediction Before Execution

Model mapped delivery as page-fault cost plus H2D. Model pageable delivery as read plus H2D path. Model pinned delivery as read plus host copy plus H2D, with potential overlap across shards. Predict crossovers by shard size and reuse: small cold reads favor fewer copies; sustained pipelines can favor pinned slots.

Part 4 - Experimental Design and Instrumentation

Use fresh processes, randomized file regions, and declared cold or warm cache. Measure open, read or fault, pin-copy, H2D completion, GPU-ready time, RSS, pinned memory, CPU utilization, faults, and effective storage bytes. Verify identical CUDA outputs and repeat across shard sizes.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class DeliveryResult:
    path: str
    shard_bytes: int
    open_ms: float
    storage_ms: float
    host_copy_ms: float
    h2d_ms: float
    ready_ms: float
    checksum: float

def compare_delivery(results):
    rows = []
    for result in results:
        rows.append({
            "path": result.path,
            "bytes": result.shard_bytes,
```

```
            "gpu_ready_ms": result.ready_ms,
            "storage_fraction": (
                result.storage_ms / max(result.ready_ms, 1e-9)),
            "checksum": result.checksum,
        })
    return rows

def E08(results):
    return compare_delivery(results)
```

Part 6 - Measurement, Validation, and Failure Analysis

Mapped access is not GPU-direct storage in this experiment. A warm-cache mapped result cannot be compared with a cold explicit read. Pinned staging is useful only when its extra copy and locked-memory cost are outweighed by faster or overlapped H2D. Verify completion on the exact consumer stream.

Part 7 - Engineering Interpretation and Philosophy

There is no universally shortest path. The correct default emerges from shard size, cache temperature, reuse, concurrency, and memory pressure, all measured at GPU-ready completion rather than API return.

19.17.10 Exercise 09

Exercise Title: Sweep Parameter Shard Size and Determine How Small Random Reads and Large Sequential Reads Affect Effective NVMe Throughput

Part 1 - Exercise Contract and Why It Matters

Sweep shard size and access order to locate the transition from latency-bound small reads to bandwidth-bound large sequential reads. Use the result to choose parameter grouping rather than assuming one file per tensor is efficient.

Part 2 - System and Memory Mental Model

Each request pays software, filesystem, queue, and device latency. Sequential access benefits from readahead and contiguous media requests. Random access can defeat readahead and multiply queue work. Concurrency may hide latency until it saturates queues or competes with GPU and H2D traffic.

Part 3 - Quantitative Prediction Before Execution

For request size S, approximate throughput by $S/(L+S/B)$. Small S is latency limited; large S approaches bandwidth B. Include useful-byte ratio when a large shard contains parameters not needed by the current layer. Optimal sharding balances I/O efficiency against overfetch and cache pressure.

Part 4 - Experimental Design and Instrumentation

Generate an aligned benchmark file, sweep powers-of-two sizes, and run sequential and seeded random offsets with equal total useful bytes. Record IOPS, throughput, median and p95 latency, CPU time, page faults, readahead effects, queue depth, and overfetch. Separate warm and cold-cache trials.

Part 5 - PyTorch/CUDA Implementation

```
import random
import time

def read_offsets(path, offsets, shard_bytes):
    latencies = []
    total = 0
    with open(path, "rb", buffering=0) as handle:
        for offset in offsets:
            handle.seek(offset)
            start = time.perf_counter()
            data = handle.read(shard_bytes)
            latencies.append(time.perf_counter() - start)
            total += len(data)
    return total, latencies

def seeded_random_offsets(file_bytes, shard_bytes, count, seed):
    generator = random.Random(seed)
    maximum = (file_bytes - shard_bytes) // shard_bytes
    offsets = []
    for _ in range(count):
        block = generator.randint(0, maximum)
        offsets.append(block * shard_bytes)
    return offsets

def E09(path, file_bytes, shard_bytes, count, seed=7):
    offsets = seeded_random_offsets(
        file_bytes, shard_bytes, count, seed)
    return read_offsets(path, offsets, shard_bytes)
```

Part 6 - Measurement, Validation, and Failure Analysis

Python buffered or cached reads can conceal device behavior. Direct I/O has alignment constraints and is platform-specific, so document the chosen path. Random offsets may repeat; report unique useful bytes separately. One-thread results do not describe an asynchronous queue, while excessive parallelism may measure contention rather than the device limit.

Part 7 - Engineering Interpretation and Philosophy

Shard size is a systems parameter. Good sharding makes physical access efficient without reading so much unused state that the hierarchy loses its capacity and locality advantages.

19.17.11 Exercise 10

Exercise Title: Build an Asynchronous NVMe Read Queue That Loads Future Parameter Shards While PyTorch Executes the Current Layer

Part 1 - Exercise Contract and Why It Matters

Build a bounded read queue that submits future shards from an explicit execution schedule while the current CUDA layer runs. Every request needs identity, generation, destination ownership, deadline, cancellation state, completion, and byte reservations.

Part 2 - System and Memory Mental Model

Python I/O can run in worker threads while CUDA work proceeds asynchronously. The future proves host-read completion, not GPU readiness. Queue depth consumes RAM and can issue useless reads after a control-flow change. Backpressure must bound submitted and completed-but-unconsumed bytes.

Part 3 - Quantitative Prediction Before Execution

Prefetch distance must cover read latency plus variation before the layer's deadline. If production rate exceeds storage service rate, pending bytes grow without bound. Estimate queue capacity from the maximum admitted byte sum, not request count, because parameter shards vary greatly in size.

Part 4 - Experimental Design and Instrumentation

Replay a deterministic layer order with artificial compute duration. Sweep worker count, lookahead, and byte budget. Record submission, start, completion, consumption, lateness, cancellation, useful bytes, wasted bytes, queue depth, RAM, and GPU wait. Inject reordered and skipped layers.

Part 5 - PyTorch/CUDA Implementation

```
from concurrent.futures import ThreadPoolExecutor
from dataclasses import dataclass
from pathlib import Path

@dataclass
class ReadRequest:
    key: str
    size_bytes: int
    future: object

def read_exact(path, expected_bytes):
    data = Path(path).read_bytes()
    if len(data) != expected_bytes:
        raise IOError("short or oversized shard read")
    return data

class AsyncNvmeQueue:
    def __init__(self, workers, byte_budget):
        self.pool = ThreadPoolExecutor(max_workers=workers)
        self.byte_budget = byte_budget
        self.pending_bytes = 0
        self.requests = {}

    def submit(self, key, path, size_bytes):
        if self.pending_bytes + size_bytes > self.byte_budget:
```

```
            raise MemoryError("NVMe read queue RAM budget exceeded")
        future = self.pool.submit(read_exact, path, size_bytes)
        self.requests[key] = ReadRequest(key, size_bytes, future)
        self.pending_bytes += size_bytes

    def consume(self, key):
        request = self.requests.pop(key)
        data = request.future.result()
        self.pending_bytes -= request.size_bytes
        return data

def E10(workers, byte_budget):
    return AsyncNvmeQueue(workers, byte_budget)
```

Part 6 - Measurement, Validation, and Failure Analysis

`read_bytes()` allocates one complete bytes object and is deliberately simple; production code should read into reusable bounded buffers. Duplicate keys, worker exceptions, cancellation, shutdown, and reservation rollback need explicit handling. More threads can reduce latency hiding by creating storage contention and RAM bursts.

Part 7 - Engineering Interpretation and Philosophy

Prefetch is a prediction backed by a reservation. The queue is valuable only when useful reads arrive before their deadlines and incorrect predictions are bounded, cancellable, and visible in telemetry.

19.17.12 Exercise 11

Exercise Title: Implement Double-Buffered NVMe Staging with One Host Buffer Being Filled from Storage While Another Transfers to CUDA

Part 1 - Exercise Contract and Why It Matters

Create two pinned buffer leases so one slot can be filled from an asynchronously completed NVMe read while the other performs H2D. Enforce slot ownership and events so neither CPU writers nor later shards overwrite in-flight DMA data.

Part 2 - System and Memory Mental Model

Each slot cycles through free, reading, host-copying, CPU-ready, H2D-pending, GPU-ready, and free. Conventional file APIs often fill pageable memory first; the CPU then copies to the pinned slot. Double buffering overlaps different shards, not two operations that access the same slot concurrently.

Part 3 - Quantitative Prediction Before Execution

For storage time T_s, host-copy time T_h, and H2D time T_g, the pipelined steady-state period approaches the slowest constrained stage. Two slots consume twice the maximum pinned payload plus pageable read buffers and two CUDA destinations. Predict whether the extra buffer is affordable under both budgets.

Part 4 - Experimental Design and Instrumentation

Run alternating equal shards, then variable-size shards. Record slot state and timestamps, verify content after CUDA completion, and compare one versus two slots. Inject a missing event wait and early reuse. Report stage utilization, stall reason, pinned RAM, peak VRAM, and throughput.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass
import torch

@dataclass
class StageSlot:
    host: object
    state: str = "free"
    event: object = None
    key: str = ""

class DoubleBuffer:
    def __init__(self, elements, dtype):
        self.stream = torch.cuda.Stream()
        self.slots = []
        for _ in range(2):
            host = torch.empty(
                elements, dtype=dtype, pin_memory=True)
            self.slots.append(StageSlot(host))

    def load_host(self, index, key, pageable):
        slot = self.slots[index]
        if slot.state != "free":
            raise RuntimeError("slot is not available")
        slot.host[:pageable.numel()].copy_(pageable.reshape(-1))
        slot.key = key
        slot.state = "cpu_ready"

    def submit_h2d(self, index, elements):
        slot = self.slots[index]
        if slot.state != "cpu_ready":
            raise RuntimeError("slot has no completed host payload")
        view = slot.host[:elements]
        output = torch.empty_like(view, device="cuda")
        slot.event = torch.cuda.Event()
        with torch.cuda.stream(self.stream):
            output.copy_(view, non_blocking=True)
            slot.event.record(self.stream)
        slot.state = "h2d_pending"
        return output

def E11(elements, dtype):
    return DoubleBuffer(elements, dtype)
```

Part 6 - Measurement, Validation, and Failure Analysis

The example needs a poll method that frees a slot only after its event completes. It also assumes the pageable read has already finished. CUDA output consumption on another stream requires an event wait. If one stage is much slower than all others, two buffers may add memory without improving throughput.

Part 7 - Engineering Interpretation and Philosophy

Double buffering is a lifetime discipline before it is a speed technique. Its two slots turn serialization into overlap only when each transition has one owner, one completion signal, and bounded memory.

19.17.13 Exercise 12

Exercise Title: Build a Three-Stage Pipeline That Overlaps NVMe Read, RAM-to-VRAM Transfer, and GPU Computation

Part 1 - Exercise Contract and Why It Matters

Build a dependency-aware pipeline in which shard $N + 2$ reads from NVMe, shard $N + 1$ transfers from pinned RAM to VRAM, and layer N computes. Publish each resource only at its stage completion and bound all in-flight bytes.

Part 2 - System and Memory Mental Model

The storage worker, transfer stream, and compute stream are separate executors. Edges carry futures or CUDA events. Compute waits only for its parameter H2D event; H2D waits for the host payload; buffer reclamation waits for DMA; VRAM eviction waits for the final CUDA consumer. Global synchronization would erase the pipeline.

Part 3 - Quantitative Prediction Before Execution

Sequential time per shard is $T_s + T_h + T_c$. After fill, ideal pipeline period is approximately $\max(T_s, T_h, T_c)$. Total time also includes fill and drain. Predict RAM from read and pinned queues and VRAM from current plus prefetched weights, active outputs, activations, and workspaces.

Part 4 - Experimental Design and Instrumentation

First measure each stage independently. Then compare sequential, two-stage, and three-stage versions for the same layer trace. Export storage and CUDA timelines, record deadline misses and stage stalls, and verify every layer output against a resident model. Sweep computation intensity and shard size.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class PipelineItem:
    key: str
    stage: str
    size_bytes: int
```

```
    read_future: object = None
    h2d_event: object = None
    cuda_tensor: object = None

class ThreeStageController:
    legal = {
        "planned": "nvme_read",
        "nvme_read": "cpu_ready",
        "cpu_ready": "h2d",
        "h2d": "gpu_ready",
        "gpu_ready": "compute",
        "compute": "finished",
    }

    def advance(self, item, next_stage):
        expected = self.legal.get(item.stage)
        if expected != next_stage:
            raise RuntimeError("illegal pipeline transition")
        item.stage = next_stage

    def validate_gpu_use(self, item):
        if item.stage != "gpu_ready":
            raise RuntimeError("compute requested before GPU readiness")
        if item.cuda_tensor is None:
            raise RuntimeError("missing CUDA parameter")

def E12():
    return ThreeStageController()
```

Part 6 - Measurement, Validation, and Failure Analysis

A state machine validates order but not actual completion; the runtime must advance only after the future or event proves it. Storage and H2D may share PCIe resources and fail to overlap fully. Variable layer times create bubbles, while excessive lookahead can exceed RAM or VRAM and cause eviction of useful data.

Part 7 - Engineering Interpretation and Philosophy

The pipeline makes latency tolerable by converting future knowledge into concurrency. Its speed is limited by the slowest stage, while its correctness is limited by the weakest dependency or lifetime rule.

19.17.14 Exercise 13

Exercise Title: Add an RAM Hot Cache Above NVMe and Keep High-Reuse Parameter Shards in Memory to Avoid Repeated Storage Reads

Part 1 - Exercise Contract and Why It Matters

Add a byte-bounded RAM cache between NVMe authority and pinned H2D staging. Promote shards after verified reads, update access metadata, retain high-value entries, and evict only clean unprotected replicas.

Part 2 - System and Memory Mental Model

The RAM cache converts repeated storage reads into memory copies. Cache entries can be pageable canonical tensors, while a smaller pinned pool handles DMA. Frequency alone favors stale history; recency alone can fail on scans. Reuse distance, size, read cost, next use, and host pressure together determine value.

Part 3 - Quantitative Prediction Before Execution

Measure byte hit rate as avoided NVMe bytes divided by requested bytes, not only request hit count. Estimate saved time from avoided read latency minus cache management and host-copy cost. Include duplicate page-cache and explicit-cache residency when interpreting physical RAM.

Part 4 - Experimental Design and Instrumentation

Replay looping, sequential, and mixed layer traces under several RAM budgets. Report request and byte hit rates, avoided reads, cache insertions, evictions, residency bytes, next-use prediction error, storage traffic, GPU wait, and throughput. Verify values and versions after every hit.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class RamCacheEntry:
    key: str
    tensor: object
    size_bytes: int
    accesses: int
    last_use: int
    protected: bool = False

class RamHotCache:
    def __init__(self, byte_budget):
        self.byte_budget = byte_budget
        self.used_bytes = 0
        self.clock = 0
        self.entries = {}

    def get(self, key):
        self.clock += 1
        entry = self.entries.get(key)
        if entry is None:
            return None
        entry.accesses += 1
        entry.last_use = self.clock
        return entry.tensor

    def insert(self, key, tensor):
```

```
        size = tensor.numel() * tensor.element_size()
        if size > self.byte_budget:
            return False
        self.entries[key] = RamCacheEntry(
            key, tensor, size, 1, self.clock)
        self.used_bytes += size
        return True

def E13(byte_budget):
    return RamHotCache(byte_budget)
```

Part 6 - Measurement, Validation, and Failure Analysis

The compact cache must add replacement, duplicate-key accounting, tied storage, and transactional insertion before production use. Replacing an existing key without subtracting its old bytes falsifies the budget. A dirty sole copy is not a cache entry and cannot be discarded. OS page cache can make apparent misses faster without appearing in this controller.

Part 7 - Engineering Interpretation and Philosophy

RAM should hold the working set whose absence would cost the most future time. A hot cache is successful when it reduces physical I/O and GPU waiting under a hard host budget, not when it merely reports a high request hit count.

19.17.15 Exercise 14

Exercise Title: Implement RAM-to-NVMe Eviction When Host Memory Crosses a High Watermark and Reload Evicted Tensors Transparently on Demand

Part 1 - Exercise Contract and Why It Matters

Evict eligible RAM entries toward a low watermark, write dirty authoritative state safely before release, and reload it transparently on demand. Preserve versions and never expose a partially written shard as current.

Part 2 - System and Memory Mental Model

A clean RAM replica backed by the same NVMe version can be dropped immediately. A dirty RAM authority needs writeback to a temporary generation, validation, durability according to policy, and atomic publication before RAM release. High and low watermarks provide hysteresis; pending writeback still consumes RAM.

Part 3 - Quantitative Prediction Before Execution

Choose victims until projected committed plus reserved RAM falls below the low watermark, while budgeting write buffers and metadata. Estimate future reload cost and avoid evicting near-use shards. Disk capacity also needs reservation, including temporary generations during atomic replacement.

Part 4 - Experimental Design and Instrumentation

Create clean, dirty, protected, and in-flight entries. Cross the artificial high watermark and verify legal victim order, writeback completion, RAM reduction, and exact reload. Inject

write failure, checksum failure, and interruption before rename; the old generation must remain authoritative.

Part 5 - PyTorch/CUDA Implementation

```
from pathlib import Path
import torch

def atomic_tensor_write(tensor, final_path):
    final = Path(final_path)
    temporary = final.with_suffix(final.suffix + ".pending")
    torch.save(tensor.detach().cpu(), temporary)
    temporary.replace(final)

class RamEvictor:
    def __init__(self, high_bytes, low_bytes):
        self.high_bytes = high_bytes
        self.low_bytes = low_bytes

    def choose(self, entries, used_bytes):
        victims = []
        ordered = []
        for entry in entries:
            if not entry["protected"]:
                ordered.append(entry)
        ordered.sort(key=last_use_value)
        projected = used_bytes
        for entry in ordered:
            if projected <= self.low_bytes:
                break
            victims.append(entry)
            projected -= entry["size_bytes"]
        return victims

def last_use_value(entry):
    return entry["last_use"]

def E14(high_bytes, low_bytes):
    if low_bytes >= high_bytes:
        raise ValueError("low watermark must be below high watermark")
    return RamEvictor(high_bytes, low_bytes)
```

Part 6 - Measurement, Validation, and Failure Analysis

Atomic rename is only one publication step; power-loss durability requires the appropriate file and directory synchronization for the platform. The selector must distinguish clean drop from dirty writeback. Transparent reload must verify generation, checksum, dtype, and shape before returning the tensor.

Part 7 - Engineering Interpretation and Philosophy

Eviction is a transfer of responsibility, not deletion. The slower tier becomes authoritative only after a complete, verified commit; until then the RAM source is protected even when pressure is high.

19.17.16 Exercise 15

Exercise Title: Offload Adam Optimizer State to NVMe and Measure the Storage Capacity Saved in RAM Against the Additional Training-Step Latency

Part 1 - Exercise Contract and Why It Matters

Move Adam first- and second-moment state from RAM to versioned NVMe shards, reload exactly the states needed for an update, and compare RAM saved with added read, write, synchronization, and optimizer-step latency. Preserve parameter association and step counters.

Part 2 - System and Memory Mental Model

Adam state is large and touched once per update, making it capacity-attractive but I/O-intensive. A CPU update needs moments and gradients in RAM; after the update, dirty moments must be written back before release. Reading and writing two full moment tensors per parameter can make storage bandwidth the dominant training resource.

Part 3 - Quantitative Prediction Before Execution

Full-precision first and second moments require roughly twice parameter payload, plus step state and metadata. Per-step NVMe traffic can approach a read and a write for both moments. Predict the lower-bound latency from total bytes divided by measured read and write bandwidth, then add CPU update and durability cost.

Part 4 - Experimental Design and Instrumentation

Materialize Adam state with one step, inventory unique bytes, offload it, and compare RSS. Run identical updates with RAM-resident and NVMe-resident moments. Report read/write bytes, faults, CPU time, median and p95 step latency, throughput, final loss, and maximum state and parameter error.

Part 5 - PyTorch/CUDA Implementation

```
from pathlib import Path
import torch

class NvmeAdamStateStore:
    def __init__(self, directory):
        self.root = Path(directory)
        self.root.mkdir(parents=True, exist_ok=True)
        self.index = {}

    def save_tensor(self, parameter_key, state_key, tensor, step):
        filename = "%s_%s_%08d.pt" % (
            parameter_key, state_key, step)
        path = self.root / filename
```

```
        temporary = path.with_suffix(".pt.pending")
        torch.save(tensor.detach().cpu(), temporary)
        temporary.replace(path)
        self.index[(parameter_key, state_key)] = {
            "path": str(path),
            "step": step,
            "shape": tuple(tensor.shape),
            "dtype": str(tensor.dtype),
        }

    def load_tensor(self, parameter_key, state_key):
        record = self.index[(parameter_key, state_key)]
        return torch.load(
            record["path"], map_location="cpu", weights_only=True)

def E15(directory):
    return NvmeAdamStateStore(directory)
```

Part 6 - Measurement, Validation, and Failure Analysis

Filenames derived from arbitrary parameter names need escaping or stable numeric IDs. Per-tensor files may create prohibitive small-I/O overhead; group states into update blocks after measuring. A successful save call is not automatically durable. The optimizer must not advance the published step until all required moment generations and parameters form a consistent checkpoint.

Part 7 - Engineering Interpretation and Philosophy

NVMe optimizer offload buys enormous host capacity at a recurring update cost. It is defensible when training feasibility is primary and when grouping, overlap, or reduced update frequency keeps state traffic below the critical path.

19.17.17 Exercise 16

Exercise Title: Detect NVMe Thrashing Caused by Poor Prefetch and Eviction Decisions and Quantify Wasted I/O, GPU Idle Time, and Repeated Reads

Part 1 - Exercise Contract and Why It Matters

Detect when the engine repeatedly reads, promotes, evicts, and rereads shards without enough useful compute between movements. Quantify wasted I/O, repeated reads, migration amplification, cache churn, deadline misses, and GPU idle time.

Part 2 - System and Memory Mental Model

Thrashing occurs when the admitted working set is smaller than the active reuse window or when prefetch and eviction policies oppose each other. A prefetched shard can evict a near-use hot shard, then be discarded unused. High storage utilization is therefore compatible with very low useful throughput.

Part 3 - Quantitative Prediction Before Execution

Define useful I/O as bytes consumed by computation before eviction. Wasted I/O includes unused prefetch, duplicate same-version reads, and read bytes evicted before adequate reuse. Track traffic amplification as total physical read bytes divided by unique useful bytes and stall fraction as GPU wait divided by step time.

Part 4 - Experimental Design and Instrumentation

Construct a cyclic trace whose working set is just above the RAM cache budget, then vary lookahead and victim policy. Record every request, version, read, cache hit, use, eviction, and reread. Trigger alarms from windows rather than a single event and verify that reducing lookahead or protecting the reuse window recovers throughput.

Part 5 - PyTorch/CUDA Implementation

```
from collections import defaultdict

class ThrashingDetector:
    def __init__(self, amplification_limit, idle_limit):
        self.amplification_limit = amplification_limit
        self.idle_limit = idle_limit
        self.read_counts = defaultdict(int)
        self.physical_read_bytes = 0
        self.useful_unique = set()
        self.useful_bytes = 0

    def record_read(self, key, version, size_bytes):
        self.read_counts[(key, version)] += 1
        self.physical_read_bytes += size_bytes

    def record_first_use(self, key, version, size_bytes):
        identity = (key, version)
        if identity not in self.useful_unique:
            self.useful_unique.add(identity)
            self.useful_bytes += size_bytes

    def diagnose(self, gpu_idle_seconds, elapsed_seconds):
        amplification = self.physical_read_bytes / max(
            self.useful_bytes, 1)
        idle_fraction = gpu_idle_seconds / max(elapsed_seconds, 1e-9)
        return {
            "amplification": amplification,
            "idle_fraction": idle_fraction,
            "thrashing": (
                amplification >= self.amplification_limit and
                idle_fraction >= self.idle_limit),
        }

def E16(amplification_limit=2.0, idle_limit=0.25):
    return ThrashingDetector(amplification_limit, idle_limit)
```

Part 6 - Measurement, Validation, and Failure Analysis

A second read can be legitimate after a long interval; interpret counts with reuse distance and capacity. Page-cache hits may not become physical NVMe reads, so label logical and device-level traffic separately. High GPU idle can originate from compute dependencies, CPU preprocessing, or H2D rather than storage.

Part 7 - Engineering Interpretation and Philosophy

Thrashing is failed prediction expressed as motion. The remedy is usually a smaller admitted future, a larger protected working set, coarser shards, or a policy that values near reuse more than speculative completeness.

19.17.18 Exercise 17

Exercise Title: Build a Cost-Aware Placement Policy That Chooses VRAM, RAM, or NVMe from Tensor Size, Reuse Distance, Transfer Cost, and Available Capacity

Part 1 - Exercise Contract and Why It Matters

Choose among VRAM, RAM, and NVMe after enforcing authority and dependency rules. The policy must use measured transfer costs, tensor size, reuse distance, frequency, dirty state, stage deadlines, and remaining capacities, and provide an explanation for every placement.

Part 2 - System and Memory Mental Model

VRAM residency avoids H2D and possibly storage work. RAM residency avoids NVMe but still requires H2D. NVMe residency maximizes memory capacity but incurs the longest path. Dirty or in-flight state has restricted legal tiers. Hysteresis prevents small score changes from causing repeated three-tier migration.

Part 3 - Quantitative Prediction Before Execution

Estimate the future stall avoided by each faster tier and divide it by bytes consumed there. Add pressure prices that rise near high watermarks. Include writeback for dirty state, not only reload. Solve hard feasibility first, then rank legal marginal benefits until capacities are filled.

Part 4 - Experimental Design and Instrumentation

Calibrate read, write, host-copy, and H2D costs from real benchmarks. Replay known traces and compare against all-NVMe, static placement, size-only, and LRU. Report capacity violations, I/O, migrations, stall, throughput, prediction error, and how often a decision changes before reuse.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class PlacementInput:
    key: str
    size_bytes: int
    reuse_distance: int
    reuse_count: int
```

```
    nvme_read_ms: float
    h2d_ms: float
    writeback_ms: float
    dirty: bool
    protected: bool

def tier_costs(item, vram_pressure, ram_pressure):
    if item.protected:
        return {"vram": 0.0, "ram": float("inf"),
                "nvme": float("inf")}
    urgency = 1.0 / max(item.reuse_distance, 1)
    future_uses = max(item.reuse_count, 1)
    vram = vram_pressure * item.size_bytes
    ram = item.h2d_ms * urgency * future_uses
    ram += ram_pressure * item.size_bytes
    nvme = (item.nvme_read_ms + item.h2d_ms)
    nvme *= urgency * future_uses
    if item.dirty:
        nvme += item.writeback_ms
    return {"vram": vram, "ram": ram, "nvme": nvme}

def choose_lowest_cost(costs):
    best_tier = None
    best_cost = float("inf")
    for tier, cost in costs.items():
        if cost < best_cost:
            best_tier = tier
            best_cost = cost
    return best_tier, best_cost

def E17(item, vram_pressure, ram_pressure):
    costs = tier_costs(item, vram_pressure, ram_pressure)
    return choose_lowest_cost(costs)
```

Part 6 - Measurement, Validation, and Failure Analysis

Costs must share consistent units and be recalibrated when cache temperature or contention changes. The local minimum does not itself satisfy global capacity; a controller must admit a set of placements and reserve transient buffers. Predicted reuse cannot override protection, unique dirty authority, or imminent consumer dependencies.

Part 7 - Engineering Interpretation and Philosophy

Placement assigns scarce fast capacity to the tensors that avoid the most future cost. It is an explainable optimization beneath non-negotiable correctness and budget laws, not a learned excuse to violate them.

19.17.19 Exercise 18

Exercise Title: Stress-Test a Model Larger Than Both Available VRAM and the Assigned RAM Budget and Keep Execution Alive Through Dynamic NVMe Swapping

Part 1 - Exercise Contract and Why It Matters

Execute a model whose total state exceeds both managed VRAM and RAM limits by keeping only a scheduled working set above NVMe. Survive variable layers, limited buffers, slow reads, and injected failures without physical OOM, stale state, unbounded queues, or silent numerical change.

Part 2 - System and Memory Mental Model

At least the current compute set, next delivery buffers, activations, workspace, and runtime overhead must fit. If even this minimum cannot fit, no placement policy can make the schedule legal. Swapping pressure couples queues: slow NVMe grows RAM backlog, while aggressive H2D prefetch grows VRAM pressure and triggers counterproductive eviction.

Part 3 - Quantitative Prediction Before Execution

Simulate the full layer trace before allocation. At every boundary include committed and reserved bytes, temporary duplication, dirty writeback, and next operator workspace. Predict working-set misses, total traffic, lower-bound time, queue peaks, and whether the planned low watermarks leave sufficient emergency headroom.

Part 4 - Experimental Design and Instrumentation

Use artificial managed limits well below physical capacity. Increase model depth and shard variability, then inject latency, read failure, checksum mismatch, missing event wait, and host allocation rejection. Require controlled rollback or backpressure and compare every output with a smaller resident reference.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class TierSample:
    step: int
    vram_committed: int
    vram_reserved: int
    ram_committed: int
    ram_reserved: int
    nvme_queue_bytes: int
    version_errors: int
    checksum_errors: int

class ThreeTierStressGuard:
    def __init__(self, vram_limit, ram_limit, queue_limit):
        self.vram_limit = vram_limit
        self.ram_limit = ram_limit
        self.queue_limit = queue_limit

    def validate(self, sample):
        failures = []
        if sample.vram_committed + sample.vram_reserved > self.vram_limit:
```

```
            failures.append("managed VRAM limit exceeded")
        if sample.ram_committed + sample.ram_reserved > self.ram_limit:
            failures.append("managed RAM limit exceeded")
        if sample.nvme_queue_bytes > self.queue_limit:
            failures.append("NVMe queue budget exceeded")
        if sample.version_errors:
            failures.append("state version mismatch")
        if sample.checksum_errors:
            failures.append("storage integrity failure")
        return failures

def E18(vram_limit, ram_limit, queue_limit):
    return ThreeTierStressGuard(vram_limit, ram_limit, queue_limit)
```

Part 6 - Measurement, Validation, and Failure Analysis

Managed counters omit framework, filesystem, and driver overhead, so physical headroom remains necessary. Keeping execution alive does not prove semantic correctness; validate versions, hashes, outputs, gradients, optimizer state, and checkpoint recovery. Do not create a real OOM or fill the real NVMe device when an artificial budget tests the same policy boundary safely.

Part 7 - Engineering Interpretation and Philosophy

The stress test proves graceful operation at a designed boundary. The goal is not maximal motion, but a bounded working set, predictable backpressure, and recoverable state when no two fast tiers can hold the complete model.

19.17.20 Exercise 19

Exercise Title: Build and Benchmark a PyTorch Three-Tier Offload Engine That Dynamically Moves Model State Across NVMe, RAM, and VRAM Without Exceeding Either Memory Budget

Part 1 - Exercise Contract and Why It Matters

Integrate canonical manifests, atomic NVMe generations, RAM hot caching, pinned buffer leases, asynchronous reads, CUDA events, three-tier placement, watermarks, prefetch, eviction, telemetry, rollback, and checkpoint recovery into one engine with hard VRAM and RAM limits.

Part 2 - System and Memory Mental Model

The control plane owns resource identity, authority, version, legal transitions, budget reservations, and future schedule. The data plane owns file ranges, CPU buffers, CUDA tensors, futures, streams, and events. A movement becomes visible only at commit. Failed I/O or transfer rolls back reservations while leaving the previous authoritative generation usable.

Part 3 - Quantitative Prediction Before Execution

Before execution, simulate each layer and optimizer boundary to predict VRAM, RAM, pinned, NVMe capacity, transfer traffic, queue depth, and stalls. During execution compare predictions with observations and adapt lookahead without changing correctness. Reject any schedule whose minimum working set plus workspace and transient copies cannot fit.

Part 4 - Experimental Design and Instrumentation

Validate transitions individually, then run inference, backward, accumulation, optimizer update, checkpoint, interruption, and resume. Benchmark full resident, static offload, and dynamic three-tier modes. Report output parity, peak budgets, read/write traffic, cache hits, wasted prefetch, overlap, median and p95 latency, throughput, retries, stalls, and recovery outcome.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass, field

@dataclass
class TierRecord:
    key: str
    size_bytes: int
    authority: str
    version: int
    nvme_path: str
    ram_tensor: object = None
    vram_tensor: object = None
    pending: str = "none"
    protected: set = field(default_factory=set)

class ThreeTierEngine:
    def __init__(self, vram_budget, ram_budget):
        self.vram_budget = vram_budget
        self.ram_budget = ram_budget
        self.vram_committed = 0
        self.vram_reserved = 0
        self.ram_committed = 0
        self.ram_reserved = 0
        self.records = {}

    def register(self, record):
        if record.key in self.records:
            raise KeyError("duplicate canonical state key")
        self.records[record.key] = record
        if record.ram_tensor is not None:
            self.ram_committed += record.size_bytes
        if record.vram_tensor is not None:
            self.vram_committed += record.size_bytes
        self.validate_budgets()

    def reserve_ram(self, size_bytes):
        projected = self.ram_committed + self.ram_reserved
        if projected + size_bytes > self.ram_budget:
            raise MemoryError("RAM admission rejected")
        self.ram_reserved += size_bytes
```

```
    def reserve_vram(self, size_bytes):
        projected = self.vram_committed + self.vram_reserved
        if projected + size_bytes > self.vram_budget:
            raise MemoryError("VRAM admission rejected")
        self.vram_reserved += size_bytes

    def begin_nvme_to_ram(self, key):
        record = self.records[key]
        if record.pending != "none":
            raise RuntimeError("resource movement already pending")
        self.reserve_ram(record.size_bytes)
        record.pending = "nvme_to_ram"

    def commit_ram(self, key, tensor, version):
        record = self.records[key]
        if record.pending != "nvme_to_ram":
            raise RuntimeError("unexpected RAM commit")
        if version != record.version:
            raise RuntimeError("stale NVMe generation")
        record.ram_tensor = tensor
        record.pending = "none"
        self.ram_reserved -= record.size_bytes
        self.ram_committed += record.size_bytes
        self.validate_budgets()

    def validate_budgets(self):
        if self.ram_committed + self.ram_reserved > self.ram_budget:
            raise MemoryError("RAM invariant violated")
        if self.vram_committed + self.vram_reserved > self.vram_budget:
            raise MemoryError("VRAM invariant violated")

def E19(vram_budget, ram_budget):
    return ThreeTierEngine(vram_budget, ram_budget)
```

Part 6 - Measurement, Validation, and Failure Analysis

The compact engine demonstrates control-plane invariants, not a complete generic trainer. Production work must add transaction IDs, asynchronous completion, H2D and writeback commits, pinned pools, alias preservation, autograd and optimizer integration, cancellation, rollback, durable manifests, and recovery. Budget reservations must be rolled back on every exceptional path.

Part 7 - Engineering Interpretation and Philosophy

A three-tier engine is a memory operating system specialized for a known model schedule. Its first obligation is coherent, recoverable state under hard limits; its second is to transform locality and future knowledge into useful overlap instead of uncontrolled I/O.

19.18 DMM16

Chapter Title: Out-of-Core Model Execution
Specialization: Dynamic Memory Management and CUDA Systems

19.18.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Out-of-Core Neural Network Execution When Model State Exceeds Available GPU Memory

Part 1 - Exercise Contract and Why It Matters

Define out-of-core execution as correct model computation while only a bounded working set resides in VRAM. Track parameters, buffers, activations, residuals, temporary workspaces, gradients, and optimizer state independently; exceeding VRAM with weights is only one of several possible failure modes.

Part 2 - System and Memory Mental Model

The model graph supplies an execution order and future-use schedule. Each block moves through cold, CPU-ready, transfer-pending, GPU-ready, executing, and evictable states. Parameter authority normally remains in RAM or NVMe for inference, while VRAM holds clean replicas. Activations follow graph edges and may outlive the layer that created them, especially across residual paths.

Part 3 - Quantitative Prediction Before Execution

At every boundary, predict peak VRAM as resident block weights plus inputs, outputs, long-lived activations, CUDA transfer destinations, operator workspace, and allocator overhead. RAM includes cold weights, staging, queued reads, and pinned buffers. A legal schedule must fit transient peaks, not only its final resident set.

Part 4 - Experimental Design and Instrumentation

Start with deterministic inference on a sequential model, then introduce skips, shared weights, variable shapes, and NVMe. Log block state, tensor versions, allocated and reserved VRAM, RAM, transfer bytes, execution deadlines, and eviction decisions. Compare every output against a fully resident reference.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass, field

@dataclass
class BlockState:
    key: str
    parameter_bytes: int
    authority: str
    residency: str
    version: int = 0
    pending: str = "none"
    next_use: int = -1
    protected: set = field(default_factory=set)
```

```
    def begin_h2d(self):
        if self.residency not in ("cpu", "cpu_and_nvme"):
            raise RuntimeError("block has no CPU source")
        if self.pending != "none":
            raise RuntimeError("movement already pending")
        self.pending = "h2d"

    def commit_gpu(self):
        if self.pending != "h2d":
            raise RuntimeError("unexpected GPU commit")
        self.residency = "cpu_and_gpu"
        self.pending = "none"

def E00(key, parameter_bytes):
    return BlockState(key, parameter_bytes, "cpu", "cpu")
```

Part 6 - Measurement, Validation, and Failure Analysis

Deleting a block does not release activations that still reference its outputs, and reduced allocated memory may leave reserved allocator segments unchanged. Module movement can invalidate optimizer or hook assumptions. A schedule that works for inference may fail in training because backward reverses dependencies and preserves forward state.

Part 7 - Engineering Interpretation and Philosophy

Out-of-core execution virtualizes model state over a known computation graph. Its foundation is not movement speed but a proof that the smallest legal working set, including graph state and temporary peaks, fits at every instant.

19.18.2 Exercise 01

Exercise Title: Measure the Complete Memory Footprint of a PyTorch Model and Prove That Its Parameters Cannot Fit Inside the Assigned VRAM Budget

Part 1 - Exercise Contract and Why It Matters

Inventory unique parameters and buffers before CUDA placement, then measure runtime activations and workspaces. Prove mathematically that parameter payload alone exceeds an assigned budget and separately predict the minimum executable working set.

Part 2 - System and Memory Mental Model

Named parameter bytes are not the entire model footprint. Tied objects and shared storages must be counted once. Buffers may be persistent, while activations and operator workspaces depend on batch, shape, precision, and backend. CUDA context and non-PyTorch allocations are outside ordinary tensor inventory.

Part 3 - Quantitative Prediction Before Execution

For each unique storage, multiply storage elements by element size, then sum parameters and buffers. Compare parameter bytes with the artificial budget and compute an excess ratio. Estimate execution headroom separately for the largest block, inputs, outputs, residuals, workspaces, and transfer duplication.

Part 4 - Experimental Design and Instrumentation

Generate a per-name table with shape, dtype, logical bytes, storage identity, owners, and trainability. Validate totals against a state dictionary while avoiding double counting. Run one small reference forward to measure activation and peak overhead, then scale predictions by controlled dimensions.

Part 5 - PyTorch/CUDA Implementation

```
def unique_model_bytes(model):
    seen = set()
    parameter_bytes = 0
    buffer_bytes = 0
    rows = []
    for name, parameter in model.named_parameters():
        identity = id(parameter)
        logical = parameter.numel() * parameter.element_size()
        if identity not in seen:
            seen.add(identity)
            parameter_bytes += logical
        rows.append((name, "parameter", logical, identity))
    for name, buffer in model.named_buffers():
        identity = id(buffer)
        logical = buffer.numel() * buffer.element_size()
        if identity not in seen:
            seen.add(identity)
            buffer_bytes += logical
        rows.append((name, "buffer", logical, identity))
    return parameter_bytes, buffer_bytes, rows

def E01(model, assigned_vram_bytes):
    parameters, buffers, rows = unique_model_bytes(model)
    excess = parameters - assigned_vram_bytes
    cannot_fit = excess > 0
    return parameters, buffers, excess, cannot_fit, rows
```

Part 6 - Measurement, Validation, and Failure Analysis

Object identity catches ordinary tying but not all distinct views over shared storage; production accounting should canonicalize untyped storage identity and range. Proving that all parameters do not fit does not prove that a blockwise schedule fits. Conversely, a model whose parameters fit may still OOM from activations or workspace.

Part 7 - Engineering Interpretation and Philosophy

The footprint proof converts an anecdotal OOM into a capacity argument. It also defines what virtualization must accomplish: fit a legal execution window, not pretend that total model size has vanished.

19.18.3 Exercise 02

Exercise Title: Artificially Restrict the Available GPU Memory Budget and Establish a Reproducible Out-of-Memory Baseline

Part 1 - Exercise Contract and Why It Matters

Create a safe, reproducible OOM baseline under a deliberately reduced allocator limit in a fresh process. Capture the exact failing boundary and memory evidence, then use the same model, batch, precision, and limit for later out-of-core tests.

Part 2 - System and Memory Mental Model

PyTorch's per-process fraction limits caching-allocator allocations relative to visible device memory. It does not partition all physical VRAM or include every external CUDA allocation. The limit should be set before significant CUDA allocation. A separate software budget remains necessary for scheduling and headroom.

Part 3 - Quantitative Prediction Before Execution

Choose a fraction whose allowed bytes are below full-model requirements but above CUDA initialization and a minimal test allocation. Predict the first failing operation from cumulative live bytes plus temporary peak. Record both the configured allocator limit and the stricter runtime working-set budget.

Part 4 - Experimental Design and Instrumentation

Launch one subprocess per trial, set the fraction, reset peak statistics, and run the resident baseline. Immediately before each boundary log allocated, reserved, peak, free, and total memory. Catch only the expected CUDA OOM, save the diagnostic, release references, and do not continue with corrupted state.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def configure_fraction(fraction, device_index=0):
    if fraction <= 0.0 or fraction > 1.0:
        raise ValueError("fraction must be in (0, 1]")
    torch.cuda.set_device(device_index)
    torch.cuda.memory.set_per_process_memory_fraction(
        fraction, device=device_index)
    total = torch.cuda.get_device_properties(
        device_index).total_memory
    return int(total * fraction)

def memory_sample():
    free_bytes, total_bytes = torch.cuda.mem_get_info()
    return {
        "allocated": torch.cuda.memory_allocated(),
        "reserved": torch.cuda.memory_reserved(),
        "peak": torch.cuda.max_memory_allocated(),
        "free": free_bytes,
```

```
        "total": total_bytes,
    }

def E02(fraction, device_index=0):
    allowed = configure_fraction(fraction, device_index)
    torch.cuda.reset_peak_memory_stats(device_index)
    return allowed, memory_sample()
```

Part 6 - Measurement, Validation, and Failure Analysis

Run this configuration only in a fresh process; earlier allocations can make the experiment irreproducible. The allocator limit is not a guarantee of exactly that many usable tensor bytes. Fragmentation, reserved blocks, context memory, other processes, and non-PyTorch CUDA allocations affect the failure point.

Part 7 - Engineering Interpretation and Philosophy

An artificial budget turns scarce hardware into a controlled experimental variable. The OOM baseline is valuable because every later success must beat the same constraint without changing the numerical task.

19.18.4 Exercise 03

Exercise Title: Partition a Deep PyTorch Model into Execution Blocks According to Parameter Size and Layer Order

Part 1 - Exercise Contract and Why It Matters

Partition an ordered model into blocks whose parameter payload and predicted execution peak fit a target working-set budget. Preserve graph order, atomic modules, residual boundaries, shared weights, and stateful layer semantics.

Part 2 - System and Memory Mental Model

A block is a transfer and scheduling unit, not merely a list slice. Larger blocks amortize movement and improve locality but occupy more VRAM. Splitting inside a fused or stateful unit may change behavior. Shared parameters belong to one canonical resource even when several blocks consume them.

Part 3 - Quantitative Prediction Before Execution

For each candidate block, sum unique parameter and buffer bytes, then add input, output, persistent activation, workspace, and H2D duplication. Greedily closing a block at the parameter limit is insufficient if its largest operator exceeds remaining headroom. Track both payload and predicted execution peak.

Part 4 - Experimental Design and Instrumentation

Extract a layer sequence, measure per-layer state, annotate graph constraints, and partition under several budgets. Validate that each layer appears once, order is preserved, shared resources are canonical, and every predicted peak is legal. Compare block counts, overfetch, and transfer frequency.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class LayerCost:
    name: str
    module: object
    parameter_bytes: int
    extra_peak_bytes: int

def partition_layers(layers, block_budget):
    blocks = []
    current = []
    current_parameters = 0
    current_extra = 0
    for layer in layers:
        next_parameters = current_parameters + layer.parameter_bytes
        next_extra = max(current_extra, layer.extra_peak_bytes)
        predicted = next_parameters + next_extra
        if current and predicted > block_budget:
            blocks.append(current)
            current = []
            current_parameters = 0
            current_extra = 0
        if layer.parameter_bytes + layer.extra_peak_bytes > block_budget:
            raise MemoryError("one layer exceeds block budget")
        current.append(layer)
        current_parameters += layer.parameter_bytes
        current_extra = max(current_extra, layer.extra_peak_bytes)
    if current:
        blocks.append(current)
    return blocks

def E03(layers, block_budget):
    return partition_layers(layers, block_budget)
```

Part 6 - Measurement, Validation, and Failure Analysis

The greedy function assumes a simple sequence and precomputed unique bytes. Branching graphs require a dependency-aware partitioner. Parameter bytes shared with an earlier block must not be charged as a new canonical allocation if kept resident, but must be charged if another replica is actually created.

Part 7 - Engineering Interpretation and Philosophy

Partitioning translates a neural graph into memory-sized transactions. The best partition respects semantics first and then balances transfer amortization against the smallest safe working set.

19.18.5 Exercise 04

Exercise Title: Execute a Model Block-by-Block by Loading Each Block from CPU to CUDA Immediately Before Use

Part 1 - Exercise Contract and Why It Matters

Implement a correctness-first inference runtime that promotes one CPU-resident block to CUDA immediately before use, executes it, returns its output, and makes the block eligible for eviction. Match a fully resident reference exactly within declared floating-point tolerance.

Part 2 - System and Memory Mental Model

Moving an `nn.Module` changes its parameters and buffers; its input must share the device. CUDA operations are asynchronous, so CPU movement after a forward call must not race with kernels still reading the block. The current output may remain on CUDA to feed the next block even after weights are evicted.

Part 3 - Quantitative Prediction Before Execution

Peak VRAM for a one-block runtime includes current weights, input, output, temporary workspace, and transient allocator state. If eviction copies clean weights back unnecessarily, traffic doubles; retain authoritative CPU weights and discard clean CUDA replicas instead. Predict total H2D bytes as the sum of block loads.

Part 4 - Experimental Design and Instrumentation

Use evaluation mode and inference mode, identical inputs, and a deterministic sequential network. Record promotion, forward, completion, eviction, allocated and peak VRAM, output shape, and checksum per block. Compare final and selected intermediate outputs with the resident reference.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class BlockwiseExecutor:
    def __init__(self, blocks, device="cuda:0"):
        self.blocks = blocks
        self.device = torch.device(device)

    @torch.inference_mode()
    def run(self, input_tensor):
        value = input_tensor.to(self.device)
        memory_trace = []
        for block in self.blocks:
            block.to(self.device)
            value = block(value)
            torch.cuda.current_stream().synchronize()
            block.to("cpu")
            memory_trace.append(torch.cuda.memory_allocated())
        return value, memory_trace
```

```
def E04(blocks, input_tensor):
    executor = BlockwiseExecutor(blocks)
    return executor.run(input_tensor)
```

Part 6 - Measurement, Validation, and Failure Analysis

The device synchronization is intentionally conservative and expensive; later exercises replace it with events. Repeated `module.to()` may allocate and copy more than a production parameter store. BatchNorm and dropout require eval mode. Hidden shared parameters can be moved by one block while another still expects them.

Part 7 - Engineering Interpretation and Philosophy

The serial executor is the reference implementation for out-of-core semantics. It sacrifices overlap to expose ownership, ordering, and numerical equivalence before optimization makes failures harder to localize.

19.18.6 Exercise 05

Exercise Title: Evict Completed Model Blocks from VRAM and Verify That Peak Memory Remains Below a Fixed Budget Throughout Inference

Part 1 - Exercise Contract and Why It Matters

Evict each clean block after its final CUDA use and prove, at every inference boundary, that committed plus reserved working-set bytes remain below a fixed budget. Separate logical residency from allocator reserved memory.

Part 2 - System and Memory Mental Model

The earliest safe eviction point follows the final consumer event, not Python return. Clean CUDA weights can be discarded because CPU authority remains. Outputs and residual activations are separate resources and may survive the block. Removing module references reduces allocated bytes; cached reserved segments may remain reusable inside PyTorch.

Part 3 - Quantitative Prediction Before Execution

Before promotion reserve block payload and transient headroom. After final-use completion, subtract only actual evicted storage. Compare peak allocated memory with the fixed budget and use reserved memory as allocator evidence, not live tensor truth. Leave margin for untracked CUDA and operator allocations.

Part 4 - Experimental Design and Instrumentation

Reset peak statistics, run several blocks, and sample before promotion, after promotion, after forward, after completion, and after eviction. Assert the software budget before every transition. Verify output parity and that no GPU parameter remains associated with evicted blocks.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class VramBudgetGuard:
    def __init__(self, limit_bytes, safety_bytes):
        self.limit_bytes = limit_bytes
        self.safety_bytes = safety_bytes
        self.logical_resident = 0

    def admit(self, block_bytes, transient_bytes):
        projected = self.logical_resident
        projected += block_bytes + transient_bytes
        if projected + self.safety_bytes > self.limit_bytes:
            raise MemoryError("out-of-core VRAM admission rejected")
        self.logical_resident += block_bytes

    def evicted(self, block_bytes):
        self.logical_resident -= block_bytes
        if self.logical_resident < 0:
            raise RuntimeError("negative residency accounting")

    def sample(self):
        return {
            "logical": self.logical_resident,
            "allocated": torch.cuda.memory_allocated(),
            "reserved": torch.cuda.memory_reserved(),
            "peak": torch.cuda.max_memory_allocated(),
        }

def E05(limit_bytes, safety_bytes):
    torch.cuda.reset_peak_memory_stats()
    return VramBudgetGuard(limit_bytes, safety_bytes)
```

Part 6 - Measurement, Validation, and Failure Analysis

Logical accounting must canonicalize shared storages and include transfer destinations before they become active. Calling `empty_cache()` every block is not eviction proof and can damage performance. If a CUDA event is not complete, moving or deleting the block can race with an in-flight kernel.

Part 7 - Engineering Interpretation and Philosophy

Eviction is successful when it shortens a resource's VRAM lifetime without shortening its semantic lifetime. Budget proof comes from complete lifecycle accounting, not from observing a low number after the iteration ends.

19.18.7 Exercise 06

Exercise Title: Build a Residency Map That Tracks Every Model Block Across NVMe, CPU RAM, and GPU VRAM During Execution

Part 1 - Exercise Contract and Why It Matters

Build one canonical residency map for all blocks, replicas, versions, pending transfers, consumer events, and protection reasons. Every prefetch, execution, eviction, and failure must be representable as a validated state transition.

Part 2 - System and Memory Mental Model

Residency is not a single device label: a block may have NVMe authority, a clean RAM cache, and an H2D-pending CUDA destination simultaneously. Authority tells which version survives replica loss. Protection can arise from execution, residual dependencies, shared consumers, transfers, backward, or checkpointing.

Part 3 - Quantitative Prediction Before Execution

Sum unique committed and reserved bytes by tier from the map. During movement, charge both endpoints until commit and release. Compare predicted totals with PyTorch and process counters. The map must also predict next-use distance so eviction can distinguish cold replicas from near-future blocks.

Part 4 - Experimental Design and Instrumentation

Replay a known schedule and save a row after every transition. Assert legal authority, monotonic versions, nonnegative budgets, completed dependencies, and one canonical block identity. Inject duplicate registration, stale completion, and eviction while protected; each must fail before memory changes.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass, field

@dataclass
class ResidencyRecord:
    key: str
    size_bytes: int
    authority: str
    version: int
    nvme_present: bool
    ram_present: bool
    vram_present: bool
    pending: str = "none"
    protected: set = field(default_factory=set)

class ResidencyMap:
    def __init__(self):
        self.records = {}

    def register(self, record):
        if record.key in self.records:
            raise KeyError("duplicate block identity")
        self.records[record.key] = record
```

```
    def tier_bytes(self):
        totals = {"nvme": 0, "ram": 0, "vram": 0}
        for record in self.records.values():
            if record.nvme_present:
                totals["nvme"] += record.size_bytes
            if record.ram_present:
                totals["ram"] += record.size_bytes
            if record.vram_present:
                totals["vram"] += record.size_bytes
        return totals

    def protect(self, key, reason):
        self.records[key].protected.add(reason)

    def unprotect(self, key, reason):
        self.records[key].protected.remove(reason)

def E06():
    return ResidencyMap()
```

Part 6 - Measurement, Validation, and Failure Analysis

The map counts logical payload and needs separate reserved-byte transactions for pending buffers and allocator overhead. Boolean presence is insufficient for multiple replicas on one tier; production records need storage IDs. Removing a protection reason that was never held must remain an error.

Part 7 - Engineering Interpretation and Philosophy

The residency map is the runtime's source of truth. Performance policies may change, but every optimization must first express a legal transition in this map or it has no defensible ownership model.

19.18.8 Exercise 07

Exercise Title: Compare One-Layer-at-a-Time Execution Against Multi-Layer Working Sets and Measure the Memory-Latency Trade-Off

Part 1 - Exercise Contract and Why It Matters

Compare a minimum one-layer working set with several multi-layer resident windows under the same model, input, precision, and VRAM budget. Quantify how additional residency reduces transfers and synchronization while increasing peak memory.

Part 2 - System and Memory Mental Model

One-layer execution minimizes weight residency but maximizes movement frequency and exposes every load. A wider window retains consecutive blocks, enables prefetch slack, and amortizes eviction. Its benefit ends when retained blocks crowd out activations, transfer destinations, or more valuable shared weights.

Part 3 - Quantitative Prediction Before Execution

For window size w, predict resident weight bytes from the largest consecutive window, not average layer size. Count block loads and reloads, H2D traffic, evictions, and synchronization. Report milliseconds saved per additional MiB of peak VRAM and detect the first infeasible window.

Part 4 - Experimental Design and Instrumentation

Sweep window sizes in clean processes, warm up, and run identical inputs. Record peak allocated and reserved VRAM, RAM, transfer bytes, promotion count, median and p95 latency, throughput, and numerical difference. Include variable layer sizes so equal layer count is not confused with equal bytes.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class WindowResult:
    resident_layers: int
    peak_vram: int
    transfer_bytes: int
    promotions: int
    latency_ms: float
    throughput: float

def compare_windows(results):
    if not results:
        return []
    baseline = results[0]
    rows = []
    for result in results:
        extra_memory = result.peak_vram - baseline.peak_vram
        latency_saved = baseline.latency_ms - result.latency_ms
        if extra_memory > 0:
            value = latency_saved / extra_memory
        else:
            value = 0.0
        rows.append({
            "layers": result.resident_layers,
            "peak_vram": result.peak_vram,
            "transfer_bytes": result.transfer_bytes,
            "latency_ms": result.latency_ms,
            "ms_saved_per_byte": value,
        })
    return rows

def E07(results):
    return compare_windows(results)
```

Part 6 - Measurement, Validation, and Failure Analysis

Layer count is a poor proxy for bytes and workspace. A larger window can appear faster because it accidentally changes synchronization or cache temperature. Ensure identical output and completion boundaries. Peak memory must include incoming blocks, not only the stable window after eviction.

Part 7 - Engineering Interpretation and Philosophy

Working-set width is a capacity investment. The useful setting is the smallest byte window that captures most locality and overlap while leaving safe headroom for graph state and temporary execution peaks.

19.18.9 Exercise 08

Exercise Title: Prefetch the Next Model Block from CPU While the Current Block Executes on GPU

Part 1 - Exercise Contract and Why It Matters

Transfer block $N+1$ on a dedicated CUDA stream while block N computes, then make the compute stream wait only for the incoming block's completion event. Protect source storage, destination VRAM, and current execution simultaneously.

Part 2 - System and Memory Mental Model

True overlap requires pinned CPU sources, non-blocking H2D, separate streams, independent copy hardware, and enough compute. The prefetch destination occupies VRAM before becoming usable. The current block cannot be evicted until its final compute event, and the pinned source cannot be reused until H2D completes.

Part 3 - Quantitative Prediction Before Execution

If current compute time is T_c and next transfer time is T_t, visible stall is ideally $\max(0, T_t - T_c)$. Peak VRAM includes current and incoming weights, current activations, outputs, and workspaces. Reject prefetch if this transient window exceeds the budget even when each block fits alone.

Part 4 - Experimental Design and Instrumentation

Measure transfer-only, compute-only, serial, and overlapped timelines. Use CUDA events and a profiler trace to prove concurrency. Sweep block size and compute intensity. Verify that the compute stream waits on the exact next-block event and that outputs match the serial executor.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class BlockPrefetch:
    def __init__(self):
        self.stream = torch.cuda.Stream()
        self.events = {}

    def submit(self, key, copy_function):
        if key in self.events:
```

```
            raise RuntimeError("duplicate prefetch")
        done = torch.cuda.Event()
        with torch.cuda.stream(self.stream):
            cuda_block = copy_function()
            done.record(self.stream)
        self.events[key] = (done, cuda_block)

    def acquire(self, key):
        done, cuda_block = self.events.pop(key)
        torch.cuda.current_stream().wait_event(done)
        return cuda_block

def E08():
    return BlockPrefetch()
```

Part 6 - Measurement, Validation, and Failure Analysis

The callback must perform genuinely non-blocking copies from correctly pinned sources and retain their leases. A CUDA event establishes GPU ordering but does not manage Python ownership automatically. Default-stream dependencies, pageable sources, insufficient hardware engines, or memory contention can remove overlap.

Part 7 - Engineering Interpretation and Philosophy

Prefetch turns known future demand into concurrent movement. It succeeds only when the prediction is right, the incoming bytes fit, and the current compute is long enough to hide measured transfer time.

19.18.10 Exercise 09

Exercise Title: Implement Double-Buffered Out-of-Core Execution with Separate CUDA Buffers for Current and Incoming Model Blocks

Part 1 - Exercise Contract and Why It Matters

Alternate two CUDA block slots: one owns the currently executing block and the other receives the next block. Enforce generation, capacity, transfer event, compute event, and lease rules so a slot is never overwritten while in use.

Part 2 - System and Memory Mental Model

Slots are memory reservations, not necessarily reusable tensor shapes. A slot cycles through free, H2D-pending, GPU-ready, executing, compute-complete, and free. Variable block sizes require capacity checks or size classes. Shared resources may live outside both slots when their reuse spans multiple blocks.

Part 3 - Quantitative Prediction Before Execution

Budget both slot capacities rather than only their current payloads. Add activations and operator workspace separately. After startup, double buffering can approach the slower of transfer and compute time. Fill and drain remain visible, and unbalanced blocks still create bubbles.

Part 4 - Experimental Design and Instrumentation

Run a repeating block sequence, record every slot transition and event, and assert one owner per slot. Compare serial and double-buffered execution by peak VRAM, transfer overlap, stalls, and output parity. Inject early overwrite and an oversized block; both must fail before corrupting storage.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class CudaBlockSlot:
    index: int
    capacity_bytes: int
    state: str = "free"
    key: str = ""
    version: int = -1
    transfer_event: object = None
    compute_event: object = None
    payload: object = None

class DoubleBufferedBlocks:
    def __init__(self, capacity_bytes):
        self.slots = []
        for index in range(2):
            self.slots.append(CudaBlockSlot(index, capacity_bytes))

    def reserve(self, index, key, version, size_bytes):
        slot = self.slots[index]
        if slot.state != "free":
            raise RuntimeError("CUDA block slot is busy")
        if size_bytes > slot.capacity_bytes:
            raise MemoryError("block exceeds slot capacity")
        slot.key = key
        slot.version = version
        slot.state = "reserved"
        return slot

    def release(self, index):
        slot = self.slots[index]
        if slot.state != "compute_complete":
            raise RuntimeError("slot released before compute completion")
        slot.state = "free"
        slot.key = ""
        slot.payload = None

def E09(capacity_bytes):
    return DoubleBufferedBlocks(capacity_bytes)
```

Part 6 - Measurement, Validation, and Failure Analysis

The controller models slots but must connect transitions to real CUDA events. Preallocating one flat buffer does not automatically let arbitrary modules use that storage; a parameter materialization layer is required. Counting slot capacity twice is conservative and necessary when both can be full concurrently.

Part 7 - Engineering Interpretation and Philosophy

Double buffering converts an execution sequence into a two-slot ownership protocol. Its performance comes from overlap, but its reliability comes from refusing every premature reuse even under memory pressure.

19.18.11 Exercise 10

Exercise Title: Extend the Runtime to Load Cold Model Blocks from NVMe Through CPU RAM Before Promoting Them to VRAM

Part 1 - Exercise Contract and Why It Matters

Add NVMe as the authoritative cold tier. Resolve a block through manifest, reserve RAM, read and verify its generation, optionally cache it, reserve VRAM, stage through pinned memory, and publish the CUDA replica only after H2D completion.

Part 2 - System and Memory Mental Model

NVMe cannot be treated as a CUDA device. The normal path is file storage to pageable or mapped CPU memory, then pinned staging, then VRAM. Each stage has a different completion mechanism and lifetime. A RAM cache hit skips storage but does not skip H2D; a VRAM hit skips both.

Part 3 - Quantitative Prediction Before Execution

Cold delivery time includes read latency, checksum or decoding, host copy, and H2D. Budget NVMe file capacity, RAM read and cache buffers, pinned staging, CUDA destination, and current execution state. Estimate overfetch when a block file contains parameters not needed by the current graph path.

Part 4 - Experimental Design and Instrumentation

Run cold-cache and warm-cache trials in isolated processes. Log manifest lookup, I/O start and completion, bytes, checksum, RAM admission, pin copy, H2D event, GPU publication, and later eviction. Corrupt a generation and verify that the old valid authority remains usable.

Part 5 - PyTorch/CUDA Implementation

```
from pathlib import Path
import torch

class ColdBlockLoader:
    def __init__(self, manifest):
        self.manifest = manifest
        self.ram_cache = {}
```

```
    def load_ram(self, key):
        if key in self.ram_cache:
            return self.ram_cache[key]
        record = self.manifest[key]
        path = Path(record["path"])
        state = torch.load(
            path, map_location="cpu", weights_only=True)
        if int(record["version"]) < 0:
            raise ValueError("invalid block generation")
        self.ram_cache[key] = state
        return state

    def release_ram(self, key):
        if key in self.ram_cache:
            del self.ram_cache[key]

def E10(manifest):
    return ColdBlockLoader(manifest)
```

Part 6 - Measurement, Validation, and Failure Analysis

The compact loader omits byte budgets, checksums, pinned staging, and CUDA publication, which the runtime must add as separate transactions. Deserializing many tiny files can dominate useful I/O. A successful `torch.load` does not prove that the block matches the expected graph or canonical shared state.

Part 7 - Engineering Interpretation and Philosophy

NVMe extends model capacity only by adding another scheduled dependency. Its benefit depends on predictable access and verified generations, while RAM and VRAM remain bounded caches rather than accidental full-model copies.

19.18.12 Exercise 11

Exercise Title: Build a Three-Stage Pipeline That Overlaps NVMe Read, CPU-to-GPU Transfer, and GPU Compute During Model Execution

Part 1 - Exercise Contract and Why It Matters

Pipeline block $N + 2$ storage read, block $N + 1$ H2D, and block N compute with explicit futures, pinned leases, CUDA events, budget reservations, and backpressure. Prove actual timeline overlap and exact output parity.

Part 2 - System and Memory Mental Model

The storage worker produces a verified CPU block. The transfer stream consumes that block and produces a GPU-ready event. The compute stream consumes the event and produces a final-use event. Reclamation travels backward: weights wait for compute, pinned slots wait for H2D, and read buffers wait for pin-copy.

Part 3 - Quantitative Prediction Before Execution

If stage times are T_s, T_t, and T_c, ideal steady-state period is their maximum after fill. Total latency includes pipeline fill and drain. Compute the maximum in-flight RAM and VRAM bytes for the selected lookahead and include variable block sizes rather than multiplying an average.

Part 4 - Experimental Design and Instrumentation

Measure stages independently and together. Export storage timestamps and CUDA trace events, and calculate stage utilization, visible stall, overlap efficiency, queue depth, and memory peaks. Sweep read workers and lookahead. Inject slow I/O and verify bounded backpressure instead of growing queues.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class OutOfCoreItem:
    key: str
    stage: str = "planned"
    read_future: object = None
    pinned_lease: object = None
    h2d_event: object = None
    compute_event: object = None

class PipelineOrder:
    def __init__(self):
        self.next_stage = {
            "planned": "reading",
            "reading": "cpu_ready",
            "cpu_ready": "transferring",
            "transferring": "gpu_ready",
            "gpu_ready": "computing",
            "computing": "finished",
        }

    def advance(self, item, requested):
        expected = self.next_stage.get(item.stage)
        if expected != requested:
            raise RuntimeError("illegal out-of-core stage transition")
        item.stage = requested

    def require_finished(self, item):
        if item.stage != "finished":
            raise RuntimeError("block execution is incomplete")

def E11():
    return PipelineOrder()
```

Part 6 - Measurement, Validation, and Failure Analysis

State labels do not prove completion; only the associated future or CUDA event can authorize advancement. NVMe and GPU traffic may share PCIe links and reduce overlap. Excessive lookahead may improve stage utilization while increasing overfetch, RAM, and VRAM beyond the legal budget.

Part 7 - Engineering Interpretation and Philosophy

The pipeline is a dependency graph with resources attached. Speed comes from keeping independent stages busy, while correctness comes from never advancing a resource because time probably passed.

19.18.13 Exercise 12

Exercise Title: Preserve Residual Connections and Long-Lived Activations While Model Layers Are Continuously Loaded and Evicted

Part 1 - Exercise Contract and Why It Matters

Track graph-edge activations independently from block weights so residual inputs survive until their final merge consumer. Evicting a producing block must never discard its still-live activation or reuse its storage prematurely.

Part 2 - System and Memory Mental Model

Weight residency follows layer execution; activation residency follows graph liveness. A residual tensor may bypass many blocks, increasing the minimum VRAM working set. It may remain on GPU, offload to RAM, or be recomputed, each with different cost and autograd implications. In-place operations can destroy a value required by a later branch.

Part 3 - Quantitative Prediction Before Execution

Perform liveness analysis: allocate an activation at its producer and release it after its final consumer. Add the maximum sum of simultaneously live activation bytes to block weights and workspace. Compare keeping a residual in VRAM with D2H plus H2D or recomputation cost before choosing a policy.

Part 4 - Experimental Design and Instrumentation

Use a small residual network with hooks that record tensor ID, shape, bytes, producer, consumer count, and final use. Compare resident and out-of-core intermediate values. Inject early release and an in-place mutation; require liveness or checksum validation to identify the exact edge.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class ActivationLease:
    key: str
    tensor: object
    remaining_consumers: int
    size_bytes: int
    protected: bool = True
```

```
class ActivationTracker:
    def __init__(self):
        self.leases = {}

    def produce(self, key, tensor, consumers):
        if key in self.leases:
            raise KeyError("duplicate activation key")
        size = tensor.numel() * tensor.element_size()
        self.leases[key] = ActivationLease(
            key, tensor, consumers, size)

    def consume(self, key):
        lease = self.leases[key]
        if lease.remaining_consumers <= 0:
            raise RuntimeError("activation consumed too many times")
        lease.remaining_consumers -= 1
        tensor = lease.tensor
        if lease.remaining_consumers == 0:
            lease.protected = False
        return tensor

    def release(self, key):
        lease = self.leases[key]
        if lease.protected:
            raise RuntimeError("activation still has future consumers")
        del self.leases[key]

def E12():
    return ActivationTracker()
```

Part 6 - Measurement, Validation, and Failure Analysis

Consumer counts must come from the real graph and control-flow path. Tensor views can share storage, so releasing one logical edge may not release bytes. Training preserves additional tensors for backward; inference-mode liveness cannot be reused unchanged. In-place mutation requires alias-aware analysis.

Part 7 - Engineering Interpretation and Philosophy

Out-of-core execution virtualizes weights, not mathematical dependencies. Graph liveness defines a hard floor beneath memory use, and any runtime that ignores that floor exchanges OOM for silent wrong answers.

19.18.14 Exercise 13

Exercise Title: Support Shared and Tied Parameters Without Loading Duplicate Copies into the Out-of-Core Working Set

Part 1 - Exercise Contract and Why It Matters

Canonicalize shared and tied parameters across blocks, load one GPU replica per canonical version, count all owners, and retain it until the final scheduled consumer. Preserve alias identity and prevent independent updates or duplicate VRAM charges.

Part 2 - System and Memory Mental Model

Several module names can reference the same `Parameter`, and distinct views can share underlying storage. The resource belongs to a canonical store; blocks hold leases. Eviction becomes legal when no active or near-future lease requires the replica. During training, all aliases must accumulate into the same logical gradient and optimizer state.

Part 3 - Quantitative Prediction Before Execution

Compare logical named bytes with unique canonical storage bytes. Predict saved VRAM from avoided duplicate replicas and additional lifetime from the span between first and final consumer. A shared embedding used at both ends may be cheaper to retain than evict and reload despite its large size.

Part 4 - Experimental Design and Instrumentation

Create tied input-output embeddings and a module reused twice. Build owner lists, canonical IDs, and consumer schedules. Verify one CUDA storage, identical data pointers where required, correct outputs, and one update path. Inject duplicate materialization and detect it from storage identity and budget accounting.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass, field

@dataclass
class SharedResource:
    canonical_id: int
    owners: list = field(default_factory=list)
    remaining_uses: int = 0
    cuda_tensor: object = None

class SharedParameterRegistry:
    def __init__(self):
        self.resources = {}
        self.owner_to_id = {}

    def register(self, owner, parameter, scheduled_uses):
        canonical_id = id(parameter)
        self.owner_to_id[owner] = canonical_id
        if canonical_id not in self.resources:
            self.resources[canonical_id] = SharedResource(canonical_id)
        resource = self.resources[canonical_id]
        resource.owners.append(owner)
        resource.remaining_uses += scheduled_uses

    def consume(self, owner):
        canonical_id = self.owner_to_id[owner]
```

```
        resource = self.resources[canonical_id]
        if resource.remaining_uses <= 0:
            raise RuntimeError("shared parameter use underflow")
        resource.remaining_uses -= 1
        return resource.cuda_tensor

def E13():
    return SharedParameterRegistry()
```

Part 6 - Measurement, Validation, and Failure Analysis

Object identity does not canonicalize distinct tensor views over the same storage. A robust registry includes storage pointer, byte range, offset, shape, and stride. Consumer counts must reflect the executed branch. Restoring aliases as equal-valued copies is numerically similar at first but semantically wrong for later mutation and optimization.

Part 7 - Engineering Interpretation and Philosophy

Sharing is part of model architecture, not a serialization accident. A correct out-of-core engine virtualizes one resource with many consumers rather than turning intentional identity into costly and divergent copies.

19.18.15 Exercise 14

Exercise Title: Detect and Eliminate Execution Bubbles Caused by Late Parameter Loads and Insufficient Prefetch Distance

Part 1 - Exercise Contract and Why It Matters

Measure GPU idle intervals caused specifically by missing block readiness, trace them to storage or H2D deadlines, and increase prefetch distance only within RAM, VRAM, and overfetch budgets. Prove recovered throughput on the same workload.

Part 2 - System and Memory Mental Model

An execution bubble begins when the compute stream is ready for a block whose GPU-ready event is incomplete. Its root can be late submission, slow storage, queueing, pageable staging, H2D contention, or a prediction miss. Larger lookahead creates more slack but reserves more memory and may fetch unused work.

Part 3 - Quantitative Prediction Before Execution

For each block, compare delivery deadline with measured read plus H2D latency. Minimum lookahead covers this time using expected compute of intervening blocks. Track p95 rather than average latency for reliability. Cap lookahead by the largest legal sum of prefetched RAM and VRAM bytes.

Part 4 - Experimental Design and Instrumentation

Record planned-use time, submission, storage completion, H2D completion, compute request, and compute start. Classify each wait by late stage. Sweep prefetch distance and report bubble time, utilization, memory peaks, overfetch, and throughput. Confirm that removing bubbles does not create thrashing.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class DeadlineRecord:
    key: str
    compute_requested_ms: float
    gpu_ready_ms: float
    read_done_ms: float
    h2d_done_ms: float

def classify_bubble(record):
    wait = max(0.0, record.gpu_ready_ms
               - record.compute_requested_ms)
    if wait == 0.0:
        reason = "none"
    elif record.read_done_ms >= record.compute_requested_ms:
        reason = "storage_late"
    elif record.h2d_done_ms >= record.compute_requested_ms:
        reason = "h2d_late"
    else:
        reason = "scheduler_late"
    return wait, reason

def total_bubbles(records):
    total = 0.0
    counts = {}
    for record in records:
        wait, reason = classify_bubble(record)
        total += wait
        counts[reason] = counts.get(reason, 0) + 1
    return total, counts

def E14(records):
    return total_bubbles(records)
```

Part 6 - Measurement, Validation, and Failure Analysis

An empty CUDA timeline can have causes unrelated to parameters; correlate exact events. CPU timestamps and CUDA event times use different clocks unless aligned. Increasing lookahead can hide one bubble while increasing queue delay, memory, or useless I/O. Optimize total throughput under budgets, not bubble count alone.

Part 7 - Engineering Interpretation and Philosophy

A bubble is a missed deadline made visible on the GPU. Prefetch distance should be the smallest evidence-based amount of future state that meets deadlines reliably, not an unbounded attempt to load the model early.

19.18.16 Exercise 15

Exercise Title: Detect Out-of-Core Thrashing When the Active Working Set Exceeds the Effective VRAM and RAM Cache Capacity

Part 1 - Exercise Contract and Why It Matters

Detect repeated block promotion and eviction that performs more movement than useful computation. Distinguish true working-set overflow from one-time cold startup and identify whether VRAM, RAM cache, or both are too small for the reuse window.

Part 2 - System and Memory Mental Model

Thrashing appears when blocks needed again soon are evicted to admit speculative or lower-value blocks. VRAM thrashing repeats H2D; RAM thrashing repeats NVMe reads; coupled thrashing repeats the entire path. Variable shapes and residuals can reduce effective capacity below configured payload budgets.

Part 3 - Quantitative Prediction Before Execution

Track unique useful bytes, total H2D bytes, total NVMe bytes, migrations per block, residency duration, reuse distance, and GPU idle. Define amplification ratios for each tier. A sustained ratio well above one plus high stall indicates thrashing; a first compulsory load does not.

Part 4 - Experimental Design and Instrumentation

Replay cyclic traces with working sets below, near, and above each cache budget. Use rolling windows and log every promotion, use, eviction, and reread. Verify that increasing protected capacity, reducing lookahead, or changing victims removes the alarm and improves throughput without violating memory limits.

Part 5 - PyTorch/CUDA Implementation

```
from collections import defaultdict

class OutOfCoreThrashing:
    def __init__(self, h2d_limit, nvme_limit, idle_limit):
        self.h2d_limit = h2d_limit
        self.nvme_limit = nvme_limit
        self.idle_limit = idle_limit
        self.h2d_bytes = 0
        self.nvme_bytes = 0
        self.useful_bytes = 0
        self.loads = defaultdict(int)
        self.first_uses = set()

    def record_load(self, key, tier, size_bytes):
        self.loads[(key, tier)] += 1
        if tier == "h2d":
            self.h2d_bytes += size_bytes
        elif tier == "nvme":
            self.nvme_bytes += size_bytes
```

```
    def record_use(self, key, version, size_bytes):
        identity = (key, version)
        if identity not in self.first_uses:
            self.first_uses.add(identity)
            self.useful_bytes += size_bytes

    def diagnose(self, gpu_idle_fraction):
        useful = max(self.useful_bytes, 1)
        h2d_amp = self.h2d_bytes / useful
        nvme_amp = self.nvme_bytes / useful
        active = h2d_amp > self.h2d_limit
        active = active or nvme_amp > self.nvme_limit
        active = active and gpu_idle_fraction > self.idle_limit
        return h2d_amp, nvme_amp, active

def E15():
    return OutOfCoreThrashing(2.0, 1.5, 0.20)
```

Part 6 - Measurement, Validation, and Failure Analysis

Useful unique bytes ignore legitimate repeated uses over long intervals; combine amplification with reuse distance and a rolling window. Logical NVMe reads may be page-cache hits, so device traffic needs operating-system evidence. GPU idle can also come from activations, CPU work, or synchronization unrelated to weights.

Part 7 - Engineering Interpretation and Philosophy

Thrashing means the hierarchy is moving state instead of exploiting locality. The cure is not faster eviction; it is a working set, partition, and prediction policy consistent with actual reuse and effective capacity.

19.18.17 Exercise 16

Exercise Title: Build an Adaptive Working-Set Controller That Changes Resident Layer Count from VRAM Pressure, Transfer Time, and Compute Time

Part 1 - Exercise Contract and Why It Matters

Adapt the number and byte size of resident and prefetched blocks from measured VRAM headroom, transfer latency, compute coverage, bubble time, and thrashing. Use hysteresis and hard safety constraints so adaptation does not oscillate or violate transient memory budgets.

Part 2 - System and Memory Mental Model

More residency improves reuse and deadline slack; less residency restores headroom for activations and workspaces. The controller operates on a slower timescale than individual transfers. High and low pressure thresholds prevent one noisy sample from reversing direction. Byte-aware windows are safer than a fixed layer count for heterogeneous blocks.

Part 3 - Quantitative Prediction Before Execution

Estimate blocks needed to cover p95 delivery time by summing upcoming compute times. Admit them only if current live bytes plus incoming payloads, activation forecast, workspace, and safety margin fit. Shrink when pressure or thrashing crosses high thresholds; grow after sustained low pressure and visible bubbles.

Part 4 - Experimental Design and Instrumentation

Feed the controller stable, bursty, and variable-shape traces. Log inputs, decision, reason, previous and new byte windows, migrations, bubbles, memory, and throughput. Compare fixed-window baselines. Inject noisy measurements and verify minimum dwell time and hysteresis prevent oscillation.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class ControlSample:
    pressure: float
    transfer_ms: float
    covered_compute_ms: float
    bubble_ms: float
    thrashing: bool

class WorkingSetController:
    def __init__(self, minimum, maximum, high, low):
        self.minimum = minimum
        self.maximum = maximum
        self.high = high
        self.low = low
        self.target = minimum

    def update(self, sample):
        if sample.pressure >= self.high or sample.thrashing:
            self.target = max(self.minimum, self.target - 1)
            return self.target, "shrink_for_pressure"
        delivery_uncovered = sample.transfer_ms
        delivery_uncovered -= sample.covered_compute_ms
        if sample.pressure <= self.low:
            if sample.bubble_ms > 0.0 and delivery_uncovered > 0.0:
                self.target = min(self.maximum, self.target + 1)
                return self.target, "grow_for_latency_coverage"
        return self.target, "hold"

def E16(minimum, maximum):
    return WorkingSetController(minimum, maximum, 0.90, 0.70)
```

Part 6 - Measurement, Validation, and Failure Analysis

Layer count must be translated into predicted bytes before admission. This minimal controller needs rolling statistics, dwell time, and emergency fallback. Shrinking a target does not authorize eviction of protected blocks. Growing a window can worsen bubbles if extra prefetch creates storage contention.

Part 7 - Engineering Interpretation and Philosophy

Adaptation should follow measured scarcity and deadlines, not chase every sample. Hard invariants define what is legal; a slow, explainable controller chooses a stable point inside that legal region.

19.18.18 Exercise 17

Exercise Title: Run a Transformer-Style PyTorch Model Whose Parameter Size Is Multiple Times Larger Than the Assigned VRAM Budget and Verify Numerical Correctness

Part 1 - Exercise Contract and Why It Matters

Execute a transformer-style stack whose unique parameter payload is several times the software VRAM budget. Partition and stream its blocks while retaining embeddings, normalization, attention masks, residuals, and tied output weights correctly. Compare logits with a resident CPU or larger-device reference.

Part 2 - System and Memory Mental Model

Transformer blocks are sequential at layer granularity but contain residual paths and temporary attention and FFN activations. Embedding and language-model head weights may be tied across the full schedule. Sequence length changes activation and attention workspace dramatically even when parameter bytes stay fixed. Evaluation mode removes dropout randomness for parity.

Part 3 - Quantitative Prediction Before Execution

Compute total unique parameters, largest block, persistent shared weights, two block slots, live residuals, attention masks, KV-like temporaries, and operator workspace. Prove total weights exceed the assigned budget by the target multiple while the predicted transient working set remains below it.

Part 4 - Experimental Design and Instrumentation

Use fixed seeds, tokens, masks, dtype, and deterministic options where supported. Generate reference logits, then run one-block and pipelined schedules. Compare maximum absolute and relative error, top-token agreement, intermediate checksums, peak memory, transfer traffic, and latency across sequence lengths.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def numerical_report(reference, candidate, atol, rtol):
    difference = (reference - candidate).abs()
    maximum_absolute = difference.max().item()
    denominator = reference.abs().clamp_min(1e-12)
    maximum_relative = (difference / denominator).max().item()
```

```
    close = torch.allclose(reference, candidate, atol=atol, rtol=rtol)
    reference_tokens = reference.argmax(dim=-1)
    candidate_tokens = candidate.argmax(dim=-1)
    token_agreement = reference_tokens.eq(candidate_tokens)
    agreement_rate = token_agreement.float().mean().item()
    return {
        "allclose": close,
        "max_absolute": maximum_absolute,
        "max_relative": maximum_relative,
        "token_agreement": agreement_rate,
    }

def E17(reference_logits, out_of_core_logits):
    return numerical_report(
        reference_logits, out_of_core_logits, 1e-5, 1e-4)
```

Part 6 - Measurement, Validation, and Failure Analysis

CPU and CUDA kernels can differ slightly, so tolerances must reflect dtype and operation while still catching wrong versions and ordering. Matching argmax is not enough when logits differ materially. A model that fits for short sequences may OOM at longer contexts because activations, not weights, dominate.

Part 7 - Engineering Interpretation and Philosophy

The transformer experiment proves that capacity virtualization preserves a real architecture, not only a sequential toy. Numerical correctness remains the primary acceptance criterion; memory and speed improvements are secondary.

19.18.19 Exercise 18

Exercise Title: Benchmark Full Residency, CPU Offloading, NVMe Offloading, and Out-of-Core Pipelining by Peak VRAM, RAM Usage, Throughput, and Latency

Part 1 - Exercise Contract and Why It Matters

Benchmark four execution modes under identical model semantics and inputs: full GPU residency, serial CPU offload, serial NVMe offload, and pipelined out-of-core execution. Report capacity, traffic, latency, throughput, and correctness together.

Part 2 - System and Memory Mental Model

Full residency minimizes parameter movement but maximizes VRAM. CPU offload adds H2D. NVMe offload adds storage and host staging. Pipelining attempts to hide both behind compute but increases in-flight buffers. Warm page cache can make NVMe mode behave like RAM, so storage temperature is part of the result.

Part 3 - Quantitative Prediction Before Execution

Predict peak VRAM, RAM, pinned bytes, NVMe traffic, H2D traffic, and lower-bound step time for every mode. Report latency cost per GiB of VRAM saved and useful throughput per peak RAM byte. Identify infeasible modes rather than assigning them fabricated performance values.

Part 4 - Experimental Design and Instrumentation

Run modes in isolated processes with the same seeds, batches, precision, determinism, and completion boundaries. Warm kernels but declare cache state. Collect median and p95 latency, throughput, peaks, I/O, copies, bubbles, and output error. Repeat enough trials to expose variability.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class ModeResult:
    mode: str
    peak_vram: int
    peak_ram: int
    nvme_bytes: int
    h2d_bytes: int
    median_ms: float
    p95_ms: float
    throughput: float
    max_error: float
    feasible: bool

def benchmark_table(results, baseline_vram):
    rows = []
    for result in results:
        saved = baseline_vram - result.peak_vram
        if saved > 0:
            cost = result.median_ms / saved
        else:
            cost = 0.0
        rows.append({
            "mode": result.mode,
            "feasible": result.feasible,
            "peak_vram": result.peak_vram,
            "peak_ram": result.peak_ram,
            "median_ms": result.median_ms,
            "p95_ms": result.p95_ms,
            "throughput": result.throughput,
            "latency_per_saved_byte": cost,
            "max_error": result.max_error,
        })
    return rows

def E18(results, baseline_vram):
    return benchmark_table(results, baseline_vram)
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not run modes consecutively in one process and call allocator remnants equal. Do not label page-cache bytes NVMe reads. Synchronize CUDA at timing boundaries, monitor transient RSS, and verify outputs. The latency-per-saved-byte example uses total median rather than incremental baseline latency; the report should also include the incremental metric.

Part 7 - Engineering Interpretation and Philosophy

The benchmark describes a capacity-performance frontier. Out-of-core execution is often slower but uniquely feasible; pipelining is valuable when it moves that feasible point toward resident throughput without breaking budgets.

19.18.20 Exercise 19

Exercise Title: Build a Complete PyTorch Out-of-Core Execution Runtime That Runs an Oversized Model Across NVMe, RAM, and VRAM with Automatic Prefetch, Eviction, and Working-Set Control

Part 1 - Exercise Contract and Why It Matters

Integrate graph partitioning, canonical resources, three-tier residency, byte budgets, asynchronous reads, pinned leases, CUDA streams and events, activation liveness, shared weights, prefetch, eviction, adaptive working sets, telemetry, rollback, and numerical verification into one runtime.

Part 2 - System and Memory Mental Model

The scheduler knows future block and activation uses. The registry owns identity, authority, version, protection, and placement. Transaction managers own I/O and copy completion. Budget controllers reserve transient bytes before movement. Only a verified commit publishes a new replica; failures retain the last valid authority and release reservations.

Part 3 - Quantitative Prediction Before Execution

Simulate the graph before execution and reject schedules whose minimum legal weights, activations, workspaces, and buffers exceed a budget. Predict stage deadlines and traffic, then compare online observations to adjust lookahead and window width. Maintain emergency margins for allocator and shape uncertainty.

Part 4 - Experimental Design and Instrumentation

Test transitions, sequential inference, residuals, shared weights, variable shapes, NVMe failures, and resume before the full model. Benchmark against the four modes from E18. Report versions, parity, peak tiers, queue depths, cache hits, migrations, wasted prefetch, bubbles, throughput, and controlled failures.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass, field

@dataclass
class RuntimeBlock:
    key: str
    size_bytes: int
    authority: str
```

```
    version: int
    nvme_path: str
    ram_state: object = None
    cuda_state: object = None
    pending: str = "none"
    next_use: int = -1
    protected: set = field(default_factory=set)

class OutOfCoreRuntime:
    def __init__(self, vram_budget, ram_budget):
        self.vram_budget = vram_budget
        self.ram_budget = ram_budget
        self.vram_committed = 0
        self.vram_reserved = 0
        self.ram_committed = 0
        self.ram_reserved = 0
        self.blocks = {}

    def register(self, block):
        if block.key in self.blocks:
            raise KeyError("duplicate canonical block")
        self.blocks[block.key] = block

    def reserve(self, tier, size_bytes):
        if tier == "ram":
            projected = self.ram_committed + self.ram_reserved
            if projected + size_bytes > self.ram_budget:
                raise MemoryError("RAM reservation rejected")
            self.ram_reserved += size_bytes
        elif tier == "vram":
            projected = self.vram_committed + self.vram_reserved
            if projected + size_bytes > self.vram_budget:
                raise MemoryError("VRAM reservation rejected")
            self.vram_reserved += size_bytes
        else:
            raise ValueError("unknown memory tier")

    def begin_h2d(self, key):
        block = self.blocks[key]
        if block.ram_state is None or block.pending != "none":
            raise RuntimeError("block is not H2D-ready")
        self.reserve("vram", block.size_bytes)
        block.pending = "h2d"

    def commit_h2d(self, key, cuda_state, version):
        block = self.blocks[key]
        if block.pending != "h2d":
            raise RuntimeError("unexpected H2D completion")
```

```
        if version != block.version:
            raise RuntimeError("stale block version")
        block.cuda_state = cuda_state
        block.pending = "none"
        self.vram_reserved -= block.size_bytes
        self.vram_committed += block.size_bytes
        self.validate()

    def evict_vram(self, key):
        block = self.blocks[key]
        if block.protected or block.pending != "none":
            raise RuntimeError("block cannot be evicted")
        if block.cuda_state is not None:
            block.cuda_state = None
            self.vram_committed -= block.size_bytes

    def validate(self):
        if self.vram_committed + self.vram_reserved > self.vram_budget:
            raise MemoryError("VRAM invariant violated")
        if self.ram_committed + self.ram_reserved > self.ram_budget:
            raise MemoryError("RAM invariant violated")

def E19(vram_budget, ram_budget):
    return OutOfCoreRuntime(vram_budget, ram_budget)
```

Part 6 - Measurement, Validation, and Failure Analysis

The compact class demonstrates reservations and H2D lifecycle, not a complete generic executor. The final implementation must connect real modules, futures, events, pinned pools, RAM and NVMe commits, activation leases, alias restoration, transaction IDs, rollback, durable manifests, cancellation, and recovery. Every exceptional path must return its reserved bytes.

Part 7 - Engineering Interpretation and Philosophy

The complete runtime is a graph-aware memory operating system. It makes an oversized model executable by preserving a small correct working set and moving future state early enough, while treating numerical equivalence and bounded failure as stronger requirements than throughput.

19.19 DMM17

Chapter Title: Activation Checkpointing and Recomputation
Specialization: Dynamic Memory Management and CUDA Systems

19.19.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Activation Memory, Recomputation, and the Memory-Compute Trade-Off During Backpropagation

Part 1 - Exercise Contract and Why It Matters

Build a lifecycle model for forward inputs, outputs, saved tensors, residuals, temporary workspaces, gradients, and recomputed intermediates. Explain precisely which activations checkpointing avoids retaining and what must remain available to replay a region during backward.

Part 2 - System and Memory Mental Model

Ordinary autograd saves selected forward values until their backward consumers. Checkpointing retains boundary inputs and reconstructs internal values by running part of forward again. Memory falls because internal lifetimes shrink; compute rises because operations repeat. Parameters and gradients are not automatically reduced. Randomness, mutable state, device movement, and global side effects must behave consistently during replay.

Part 3 - Quantitative Prediction Before Execution

Without checkpointing, peak activation memory approximates the maximum sum of simultaneously live saved tensors. A checkpointed region replaces many internal saves with boundary tensors and recomputation temporaries. Estimate bytes saved, extra forward FLOPs, recomputation count, and step-time increase. Savings are not simply total forward outputs because autograd saves operation-specific data.

Part 4 - Experimental Design and Instrumentation

Instrument model creation, forward, loss, backward, and gradient reset. Use saved-tensor hooks or a profiler to classify autograd saves, and CUDA counters for stage peaks. Compare identical seeds and inputs with no checkpoint and a known region checkpointed. Count forward calls to prove recomputation occurred.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class ActivationRegion:
    key: str
    boundary_bytes: int
    internal_saved_bytes: int
    forward_flops: float
    checkpointed: bool = False

    def estimated_saved_bytes(self):
        if not self.checkpointed:
            return 0
        saved = self.internal_saved_bytes - self.boundary_bytes
        return max(saved, 0)

    def estimated_extra_flops(self):
        if self.checkpointed:
            return self.forward_flops
        return 0.0

def E00(key, boundary_bytes, internal_bytes, forward_flops):
```

```
    return ActivationRegion(
        key, boundary_bytes, internal_bytes, forward_flops)
```

Part 6 - Measurement, Validation, and Failure Analysis

Allocated-VRAM differences include allocator reuse and temporary peaks, so they are not a direct saved-tensor inventory. Checkpointing may shift the peak from forward to backward recomputation. Stateful modules or changed global variables can make replay differ from original forward. A region whose boundary is nearly as large as its internals may save little.

Part 7 - Engineering Interpretation and Philosophy

Activation checkpointing compresses a computation history by retaining selected boundaries and purchasing the missing history again with compute. It is useful when activation capacity is scarce and replay is cheaper than retaining every intermediate until backward.

19.19.2 Exercise 01

Exercise Title: Measure Activation Memory Separately from Parameters, Gradients, and Optimizer State in a PyTorch Training Step

Part 1 - Exercise Contract and Why It Matters

Separate persistent parameter, buffer, gradient, and optimizer-state bytes from dynamic activation and workspace memory. Produce a stage-by-stage accounting that estimates the activation contribution without calling every unexplained peak an activation.

Part 2 - System and Memory Mental Model

Parameters and optimizer state persist across steps. Gradients appear during backward and may remain until reset. Saved activations span forward to their backward consumers; operator workspaces and temporaries are shorter-lived. Allocator reserved memory is reusable capacity, not a tensor category. CUDA context and library allocations can lie outside PyTorch's live tensor counters.

Part 3 - Quantitative Prediction Before Execution

Inventory exact persistent tensor bytes. Measure allocated memory at an idle baseline, after forward, after loss, after backward, after optimizer step, and after gradient reset. The forward delta gives an upper bound containing saved activations, outputs, and workspaces. Saved-tensor hooks provide a more direct logical byte total, though shared storage still requires canonicalization.

Part 4 - Experimental Design and Instrumentation

Warm up the model and optimizer, then reset peaks. Record unique tensor storages and CUDA counters at each boundary. Repeat with inference mode to estimate forward temporaries without autograd saves. Hold batch, shape, dtype, and algorithm selection constant, and synchronize only at measurement boundaries.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def tensor_bytes(tensor):
    return tensor.numel() * tensor.element_size()

def model_state_bytes(model, optimizer):
    parameters = 0
    gradients = 0
    optimizer_state = 0
    for parameter in model.parameters():
        parameters += tensor_bytes(parameter)
        if parameter.grad is not None:
            gradients += tensor_bytes(parameter.grad)
    for state in optimizer.state.values():
        for value in state.values():
            if torch.is_tensor(value):
                optimizer_state += tensor_bytes(value)
    return parameters, gradients, optimizer_state

def cuda_sample(label):
    return {
        "label": label,
        "allocated": torch.cuda.memory_allocated(),
        "reserved": torch.cuda.memory_reserved(),
        "peak": torch.cuda.max_memory_allocated(),
    }

def E01(model, optimizer):
    return model_state_bytes(model, optimizer), cuda_sample("current")
```

Part 6 - Measurement, Validation, and Failure Analysis

The function double-counts tied parameters unless canonical storage is added. Optimizer state is lazy before the first step. Forward deltas can include outputs held by Python and workspace cache. Comparing a warmed training step with a cold inference step confounds activation saving with initialization.

Part 7 - Engineering Interpretation and Philosophy

Category separation prevents the wrong optimization. Activation checkpointing is a rational response only after evidence shows that long-lived forward state, not weights, gradients, optimizer state, or fragmentation, dominates the budget.

19.19.3 Exercise 02

Exercise Title: Build a Deep PyTorch Network That Exceeds a Fixed VRAM Budget Because of Saved Activations Rather Than Model Weights

Part 1 - Exercise Contract and Why It Matters

Construct a network whose parameters fit comfortably inside a fixed VRAM budget but whose training step exceeds it as depth, batch, or sequence grows because autograd retains activations. Prove the cause with accounting rather than assuming depth implies activation pressure.

Part 2 - System and Memory Mental Model

Repeated shape-preserving blocks can have modest parameter payload yet save one or more large tensors per layer. During forward their lifetimes accumulate. Inference mode removes autograd saves and should fit under the same shape when activations are the cause. Backward may introduce its own temporary peak.

Part 3 - Quantitative Prediction Before Execution

For depth D and per-block saved payload A, activation storage often grows approximately with DA before backward releases it. Parameters remain fixed per chosen width and depth formula. Select a software budget greater than parameters plus minimal workspace but less than the predicted training peak.

Part 4 - Experimental Design and Instrumentation

Sweep depth at fixed input and then input size at fixed depth. Record parameter bytes, inference peak, training forward peak, backward peak, and OOM boundary in fresh processes. Confirm the same model fits under inference mode and that checkpointing later moves the training boundary.

Part 5 - PyTorch/CUDA Implementation

```
import torch
from torch import nn

class ActivationHeavyBlock(nn.Module):
    def __init__(self, width):
        super().__init__()
        self.first = nn.Linear(width, width)
        self.second = nn.Linear(width, width)
        self.activation = nn.GELU()

    def forward(self, value):
        hidden = self.activation(self.first(value))
        return value + self.second(hidden)

def build_deep_model(depth, width):
    blocks = []
    for _ in range(depth):
        blocks.append(ActivationHeavyBlock(width))
    return nn.Sequential(*blocks)

def E02(depth=48, width=2048):
    model = build_deep_model(depth, width)
    parameter_bytes = 0
    for parameter in model.parameters():
```

```
        parameter_bytes += parameter.numel() * parameter.element_size()
    return model, parameter_bytes
```

Part 6 - Measurement, Validation, and Failure Analysis

This model's parameters also grow with depth, so choose width, batch, and budget from measured ratios rather than claiming activation dominance automatically. Allocator limits and OOM tests belong in isolated processes. Holding outputs or losses across iterations can create a Python reference leak that mimics saved activation growth.

Part 7 - Engineering Interpretation and Philosophy

A controlled activation OOM is a diagnostic instrument. It establishes that the model's mathematical state fits while its retained execution history does not, creating the exact condition recomputation is designed to solve.

19.19.4 Exercise 03

Exercise Title: Apply torch.utils.checkpoint.checkpoint() to a Single Block and Measure the Change in Peak CUDA Memory

Part 1 - Exercise Contract and Why It Matters

Checkpoint one measured block with explicit non-reentrant behavior, count its forward calls, and compare peak allocated VRAM against an identical ordinary step. Validate loss, gradients, and output before interpreting memory savings.

Part 2 - System and Memory Mental Model

The checkpoint wrapper runs the block in forward but avoids retaining selected internal intermediates. During backward PyTorch recomputes the needed portion and then differentiates it. Boundary inputs and required graph context remain. Non-reentrant checkpointing records an autograd graph and can stop replay when the needed intermediates have been rebuilt.

Part 3 - Quantitative Prediction Before Execution

Predict saved bytes as internal saved tensors minus retained boundary and replay temporaries. The global peak reduction can be smaller if another region defines the step peak. Extra compute is approximately one additional forward through the checkpointed block, though early-stop recomputation may reduce it.

Part 4 - Experimental Design and Instrumentation

Run checkpointed and ordinary modes in fresh or reset-equivalent trials with the same weights, input, seed, and optimizer. Warm up, reset peaks, synchronize at stage boundaries, and count block calls. Compare forward peak, backward peak, step peak, time, output, loss, and parameter gradients.

Part 5 - PyTorch/CUDA Implementation

```
import torch
from torch import nn
from torch.utils.checkpoint import checkpoint
```

```
class CountedBlock(nn.Module):
    def __init__(self, module):
        super().__init__()
        self.module = module
        self.calls = 0

    def forward(self, value):
        self.calls += 1
        return self.module(value)

def checkpointed_forward(block, value, enabled):
    if enabled:
        return checkpoint(block, value, use_reentrant=False)
    return block(value)

def E03(block, value, enabled=True):
    torch.cuda.reset_peak_memory_stats()
    output = checkpointed_forward(block, value, enabled)
    return output, torch.cuda.max_memory_allocated()
```

Part 6 - Measurement, Validation, and Failure Analysis

Peak memory read immediately after forward misses a larger backward peak; read again after backward. One block may not change the global peak if other live activations dominate. Forward-call counters must not themselves alter tensor state. Passing `use_reentrant` explicitly avoids version-dependent defaults.

Part 7 - Engineering Interpretation and Philosophy

Single-block checkpointing isolates the mechanism. A correct result shows that memory fell because internal history was reconstructed, not because gradients were disabled, outputs detached, or the numerical task changed.

19.19.5 Exercise 04

Exercise Title: Compare Training Time Before and After Checkpointing and Quantify the Extra Compute Required for Recomputation

Part 1 - Exercise Contract and Why It Matters

Measure end-to-end step time and operator work for identical checkpointed and ordinary training. Separate forward, backward, and optimizer time so additional recomputation is visible rather than attributed vaguely to checkpoint overhead.

Part 2 - System and Memory Mental Model

Checkpointed forward can be slightly cheaper in memory-management work, while backward becomes more expensive because it contains replay plus gradient computation. CUDA execution is asynchronous, so CPU wall time needs explicit completion. Kernel compilation, autotuning, and allocator warmup must be removed from steady-state comparison.

Part 3 - Quantitative Prediction Before Execution

If checkpointed blocks account for fraction f of forward work and replay them once, extra compute approaches f times one forward, not a full second training step. Compare measured block call counts and profiler FLOPs with this estimate. Report slowdown and memory saved together.

Part 4 - Experimental Design and Instrumentation

Warm up both modes, use identical pre-generated batches, reset optimizer state or clone it, and time many synchronized steps. Report median and p95 forward, backward, optimizer, and total time, throughput, peak VRAM, call counts, and final parameter agreement. Alternate mode order across repetitions.

Part 5 - PyTorch/CUDA Implementation

```
import statistics
import time
import torch

def time_completed_step(step_function, repetitions):
    samples = []
    for _ in range(repetitions):
        torch.cuda.synchronize()
        start = time.perf_counter()
        step_function()
        torch.cuda.synchronize()
        elapsed_ms = (time.perf_counter() - start) * 1000.0
        samples.append(elapsed_ms)
    ordered = sorted(samples)
    p95_index = max(0, int(0.95 * len(ordered)) - 1)
    return {
        "median_ms": statistics.median(samples),
        "p95_ms": ordered[p95_index],
        "all_samples_ms": samples,
    }

def E04(baseline_step, checkpoint_step, repetitions=30):
    baseline = time_completed_step(baseline_step, repetitions)
    checkpointed = time_completed_step(checkpoint_step, repetitions)
    return baseline, checkpointed
```

Part 6 - Measurement, Validation, and Failure Analysis

The wrapper measures total time only; use events or profiler regions for stage breakdown. Different dropout masks, data, loss scaling, or optimizer history invalidate timing and correctness comparisons. Profiler overhead must not be mixed into the headline steady-state benchmark.

Part 7 - Engineering Interpretation and Philosophy

Recomputation is the price paid for capacity. A useful report states how much peak memory was purchased per unit of extra completed step time, allowing the trade-off to be compared with offload, precision, or smaller batches.

19.19.6 Exercise 05

Exercise Title: Verify Numerical Equivalence Between Checkpointed and Non-Checkpointed Training Under Controlled Random Seeds

Part 1 - Exercise Contract and Why It Matters

Prove that checkpointing changes storage strategy rather than the training objective. Starting from identical model and optimizer state, compare outputs, losses, gradients, updated parameters, and optimizer tensors under controlled randomness and deterministic settings where available.

Part 2 - System and Memory Mental Model

Recomputation must evaluate the same function with the same inputs, parameters, mode, and random choices as original forward. PyTorch can preserve relevant RNG state for checkpointed regions. Floating-point kernels may still differ slightly across devices or algorithms, so equivalence uses justified tolerances plus discrete checks, not only bitwise identity.

Part 3 - Quantitative Prediction Before Execution

For deterministic elementwise and matrix operations in the same environment, expect close outputs and gradients. Define absolute and relative tolerances per dtype before running. Compare every named parameter and gradient; an aggregate loss alone can hide offsetting errors.

Part 4 - Experimental Design and Instrumentation

Clone one initial state into two models and optimizers, pre-generate the batch, set modes and seeds, and run exactly one step. Capture block call counts and RNG configuration. Repeat several seeds. Add dropout to verify preserved RNG and a stateful counter to demonstrate why side effects require special care.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def compare_named_gradients(model_a, model_b, atol, rtol):
    parameters_b = dict(model_b.named_parameters())
    report = {}
    for name, parameter_a in model_a.named_parameters():
        parameter_b = parameters_b[name]
        if parameter_a.grad is None or parameter_b.grad is None:
            report[name] = parameter_a.grad is parameter_b.grad
            continue
        report[name] = torch.allclose(
            parameter_a.grad, parameter_b.grad,
            atol=atol, rtol=rtol)
    return report

def maximum_tensor_error(first, second):
    return (first - second).abs().max().item()

def E05(model_a, model_b, output_a, output_b):
```

```
    gradients = compare_named_gradients(
        model_a, model_b, 1e-6, 1e-5)
    error = maximum_tensor_error(output_a, output_b)
    return gradients, error
```

Part 6 - Measurement, Validation, and Failure Analysis

Creating a dictionary of parameters is explicit conversion, not evidence of shared storage. Seeds must be set before stochastic operations in each isolated run. BatchNorm running statistics and custom counters can be updated twice by replay. Equal gradients before the optimizer do not prove equal optimizer state after a step.

Part 7 - Engineering Interpretation and Philosophy

Numerical equivalence is the boundary between an optimization and a new algorithm. Memory savings are acceptable only after recomputation preserves the same training transition within a declared and defensible tolerance.

19.19.7 Exercise 06

Exercise Title: Checkpoint Multiple Consecutive Blocks and Measure How Checkpoint Granularity Changes Peak VRAM and Runtime

Part 1 - Exercise Contract and Why It Matters

Checkpoint consecutive regions at several granularities and measure boundary storage, internal saved bytes, peak VRAM, recomputation calls, and runtime. Determine when finer boundaries retain too much state and when coarse regions recompute too much work.

Part 2 - System and Memory Mental Model

Every checkpoint boundary must remain available to replay its region. Many small regions create more boundaries but limit replay scope. One large region stores fewer boundaries but can repeat more operations. Non-reentrant early stopping may avoid executing the entire region when only earlier intermediates are needed.

Part 3 - Quantitative Prediction Before Execution

For each partition, sum boundary bytes and estimate internal bytes removed. Estimate extra FLOPs from the operations replayed by backward, not simply block count. Compare configurations by peak bytes saved per millisecond and identify the smallest configuration that satisfies the target budget.

Part 4 - Experimental Design and Instrumentation

Use identical block weights and batches. Sweep one block per region, several blocks per region, and one full region. Count calls per block to observe partial replay. Record forward, backward, total time, peak memory, output, gradient error, and the exact boundaries retained.

Part 5 - PyTorch/CUDA Implementation

```
from torch.utils.checkpoint import checkpoint

def run_region(blocks, start, end, value):
    result = value
    for index in range(start, end):
        result = blocks[index](result)
    return result

def checkpoint_blocks(blocks, value, group_size):
    result = value
    start = 0
    while start < len(blocks):
        end = min(start + group_size, len(blocks))

        def region(input_tensor, begin=start, finish=end):
            return run_region(blocks, begin, finish, input_tensor)

        result = checkpoint(region, result, use_reentrant=False)
        start = end
    return result

def E06(blocks, value, group_size):
    return checkpoint_blocks(blocks, value, group_size)
```

Part 6 - Measurement, Validation, and Failure Analysis

Default arguments freeze loop boundaries; capturing mutable loop variables without them can replay the wrong region. Nested function creation is explicit but compilation systems may prefer stable modules. Region outputs with complex structures need testing. Granularity results depend on actual saved tensors, not only source-level block boundaries.

Part 7 - Engineering Interpretation and Philosophy

Checkpoint granularity determines the resolution at which history is discarded and rebuilt. The best partition is a graph and cost decision, not a universal number of layers.

19.19.8 Exercise 07

Exercise Title: Use checkpoint_sequential() on a Deep Sequential Model and Sweep Segment Count to Find the Best Memory-Compute Trade-Off

Part 1 - Exercise Contract and Why It Matters

Apply `checkpoint_sequential` to a true sequential model, sweep segment count, and select the fastest configuration that remains under a target peak. Document which segment is not checkpointed and how uneven layer costs affect the nominal partition.

Part 2 - System and Memory Mental Model

The utility partitions a sequential function into segments and checkpoints the appropriate prefix segments while the final segment executes ordinarily. Equal layer counts do not imply

equal activation bytes or compute. Segment boundaries remain live, and reentrant behavior must be chosen explicitly.

Part 3 - Quantitative Prediction Before Execution

Sweep from no effective partition through many segments. Predict boundary count, largest internal region, replay work, and peak. The optimum under a budget is not necessarily the minimum memory result; it is the feasible configuration with the lowest validated step time.

Part 4 - Experimental Design and Instrumentation

Use a deep sequential model with intentionally unequal blocks. For each segment count run warmup and repeated synchronized steps from identical state. Record peak VRAM, median and p95 time, throughput, block calls, gradient parity, and segment composition. Reject counts outside the valid model length.

Part 5 - PyTorch/CUDA Implementation

```
from torch.utils.checkpoint import checkpoint_sequential

def run_checkpoint_sequential(model, value, segments):
    if segments < 1:
        raise ValueError("segments must be positive")
    if segments > len(model):
        raise ValueError("segments exceed sequential module count")
    return checkpoint_sequential(
        model,
        segments,
        value,
        use_reentrant=False)

def select_feasible(results, memory_budget):
    best = None
    for result in results:
        if result["peak_vram"] > memory_budget:
            continue
        if best is None or result["step_ms"] < best["step_ms"]:
            best = result
    if best is None:
        raise MemoryError("no segment count meets the budget")
    return best

def E07(model, value, segments):
    return run_checkpoint_sequential(model, value, segments)
```

Part 6 - Measurement, Validation, and Failure Analysis

This utility is for sequential execution; residual and branching graphs require manual regions. Segment count is not segment size in bytes. Changes across PyTorch versions should be checked against current documentation, especially explicit reentrant selection and final-segment behavior.

Part 7 - Engineering Interpretation and Philosophy

The sweep turns an API convenience into measured engineering. The selected segment count is justified by a specific model, graph, input shape, and hardware budget rather than by a folklore default.

19.19.9 Exercise 08

Exercise Title: Compare Reentrant and Non-Reentrant PyTorch Checkpointing Behavior and Identify Their Practical Differences

Part 1 - Exercise Contract and Why It Matters

Run the same eligible region with `use_reentrant=True` and `False`, then compare graph recording, recomputation extent, backward API compatibility, nested structures, detached values, peak memory, and time.

Part 2 - System and Memory Mental Model

Reentrant checkpointing runs the original forward under no-grad and re-enters the region during backward. It requires gradient participation at its boundary and has stricter limitations. Non-reentrant mode records the graph, supports more autograd patterns, and can stop recomputation once required intermediates exist. PyTorch recommends explicit selection, with non-reentrant generally the preferred modern path.

Part 3 - Quantitative Prediction Before Execution

For a simple region both modes may save similar memory, while non-reentrant early stop can reduce replay. Predict differences with call counters per operation. Define compatibility tests before timing because a faster mode that silently drops gradients is invalid.

Part 4 - Experimental Design and Instrumentation

Test plain tensor output, nested output, detached intermediate, backward with explicit inputs, and a region containing nested autograd. Record success, exception, output, gradients, calls, peak memory, and step time. Keep RNG and model state identical.

Part 5 - PyTorch/CUDA Implementation

```
from torch.utils.checkpoint import checkpoint

def run_checkpoint_mode(function, value, use_reentrant):
    return checkpoint(
        function,
        value,
        use_reentrant=use_reentrant,
        preserve_rng_state=True)

def compare_mode_result(name, output, peak_vram, elapsed_ms):
    return {
        "mode": name,
        "shape": tuple(output.shape),
        "peak_vram": peak_vram,
```

```
        "elapsed_ms": elapsed_ms,
        "requires_grad": output.requires_grad,
    }

def E08(function, value):
    non_reentrant = run_checkpoint_mode(function, value, False)
    reentrant = run_checkpoint_mode(function, value, True)
    return non_reentrant, reentrant
```

Part 6 - Measurement, Validation, and Failure Analysis

Executing both modes in one graph complicates memory and gradient comparisons; benchmark them in isolated trials. Reentrant mode can fail when no boundary input or output requires gradients and with detached tensors in the region. Do not infer semantic support from one simple linear block.

Part 7 - Engineering Interpretation and Philosophy

Reentrant selection is part of the program's autograd contract. Choose the mode that supports the graph correctly, then optimize it; backward compatibility is more important than a small benchmark difference.

19.19.10 Exercise 09

Exercise Title: Reproduce a Checkpointing Failure Caused by Detached Tensors or Incompatible Autograd Behavior and Diagnose the Graph Correctly

Part 1 - Exercise Contract and Why It Matters

Create a controlled region with a detached intermediate and demonstrate how an incompatible checkpoint mode fails or produces missing gradient paths. Diagnose the exact graph edge and repair the design without simply removing correctness assertions.

Part 2 - System and Memory Mental Model

`detach()` intentionally cuts autograd history. Reentrant checkpointing expects its recomputed outputs to participate under its boundary rules and can reject detached values inside the region. Non-reentrant mode supports more graph structures, but a deliberate detach still means no gradient should cross that edge. The repair depends on intended mathematics.

Part 3 - Quantitative Prediction Before Execution

Draw the graph and mark which tensors require gradients before and after the detach. Predict zero or absent gradients on the cut branch and ordinary gradients on the live branch. Compare predictions with hooks and finite differences where the mathematical function is differentiable.

Part 4 - Experimental Design and Instrumentation

Run the region without checkpointing, with non-reentrant checkpointing, and with reentrant checkpointing. Capture exceptions, requires-grad flags, grad functions, parameter gradients, and anomaly-detection traces. Then move the detach outside the region or remove it only if the model intended gradient flow.

Part 5 - PyTorch/CUDA Implementation

```
import torch
from torch.utils.checkpoint import checkpoint

def region_with_detach(value, weight):
    live = value @ weight
    stopped = live.detach()
    return live + stopped

def run_failure_case(value, weight, use_reentrant):
    def region(input_tensor):
        return region_with_detach(input_tensor, weight)

    return checkpoint(
        region,
        value,
        use_reentrant=use_reentrant)

def E09(value, weight):
    with torch.autograd.detect_anomaly():
        output = run_failure_case(value, weight, True)
        loss = output.square().mean()
        loss.backward()
    return output
```

Part 6 - Measurement, Validation, and Failure Analysis

The exercise intentionally may raise under reentrant behavior; run it as a test that captures the expected failure. Anomaly detection adds overhead and belongs in diagnosis, not benchmarks. Moving detach merely to silence an exception can change which parameters learn, so compare against the non-checkpointed intended graph.

Part 7 - Engineering Interpretation and Philosophy

Checkpointing exposes hidden assumptions about gradient topology. The correct fix preserves the intended derivative; an execution that finishes with the wrong graph is a more serious failure than an explicit exception.

19.19.11 Exercise 10

Exercise Title: Preserve Dropout Correctness During Recomputation and Measure the Cost of RNG State Preservation

Part 1 - Exercise Contract and Why It Matters

Checkpoint a region containing dropout with RNG preservation enabled, prove that original forward and recomputation use equivalent random choices, and measure the time spent saving and restoring random-generator state.

Part 2 - System and Memory Mental Model

Dropout consumes RNG and its mask affects outputs and gradients. Replaying the region without restoring RNG would normally draw a different mask. PyTorch checkpointing can

stash CPU and relevant device RNG state around the region so recomputation is deterministic relative to ordinary execution. Unexpected device movement inside the function can exceed that guarantee.

Part 3 - Quantitative Prediction Before Execution

RNG state payload is small, but capture and restore add fixed overhead per checkpoint invocation. Many tiny regions can therefore magnify the cost. Predict near-identical gradients with preservation and possible disagreement without it. Measure cost per region and per completed training step.

Part 4 - Experimental Design and Instrumentation

Clone identical models, set training mode and seed, and compare ordinary forward with checkpointed forward using preservation. Instrument dropout inputs and outputs or a custom mask probe. Sweep region count and tensor size. Report output and gradient parity, calls, total time, and preservation overhead.

Part 5 - PyTorch/CUDA Implementation

```
import torch
from torch import nn
from torch.utils.checkpoint import checkpoint

class DropoutRegion(nn.Module):
    def __init__(self, probability):
        super().__init__()
        self.dropout = nn.Dropout(probability)
        self.calls = 0

    def forward(self, value):
        self.calls += 1
        return self.dropout(value).square()

def run_preserved(region, value):
    return checkpoint(
        region,
        value,
        use_reentrant=False,
        preserve_rng_state=True)

def E10(value, probability=0.2):
    torch.manual_seed(41)
    torch.cuda.manual_seed_all(41)
    region = DropoutRegion(probability).to(value.device)
    output = run_preserved(region, value)
    return output, region.calls
```

Part 6 - Measurement, Validation, and Failure Analysis

The call count increases only after backward triggers replay. Seeds must be reset at comparable points in isolated runs. Comparing checkpointed forward alone does not test replay correctness. Custom modules that move tensors to a previously unseen device or mutate generator state can defeat assumptions about which RNG states are preserved.

Part 7 - Engineering Interpretation and Philosophy

Randomness is part of the function being differentiated. RNG preservation is a correctness service whose fixed cost becomes visible mainly when checkpoint regions are too small or unnecessarily numerous.

19.19.12 Exercise 11

Exercise Title: Disable RNG State Preservation Where Safe and Benchmark the Resulting Performance Improvement

Part 1 - Exercise Contract and Why It Matters

Disable checkpoint RNG preservation only for regions proven free of stochastic operations and compare completed-step performance. Build an automatic audit that rejects unsafe regions rather than treating a small speed gain as permission to change training randomness.

Part 2 - System and Memory Mental Model

Deterministic linear, normalization, and fixed activation operations do not consume RNG. A region may still call stochastic custom functions, stochastic depth, dropout, sampling, or random augmentation indirectly. Disabling preservation avoids state capture and restore, but any replay-time randomness can change gradients relative to ordinary execution.

Part 3 - Quantitative Prediction Before Execution

The benefit is mostly fixed overhead per invocation, so it is largest for many small deterministic regions. Predict negligible memory difference. Require output, gradient, and parameter-update parity across several seeds before accepting the faster configuration.

Part 4 - Experimental Design and Instrumentation

Inventory module types and custom functions in each candidate region. Benchmark preservation on and off with identical states, synchronized repetitions, and call counts. Add a hidden dropout module as a negative test and require the audit or numerical comparison to reject disabling preservation.

Part 5 - PyTorch/CUDA Implementation

```
from torch import nn
from torch.utils.checkpoint import checkpoint

STOCHASTIC_TYPES = (
    nn.Dropout,
    nn.Dropout1d,
    nn.Dropout2d,
    nn.Dropout3d,
)
```

```
def assert_no_known_dropout(module):
    for child in module.modules():
        if isinstance(child, STOCHASTIC_TYPES):
            raise ValueError("stochastic module found in checkpoint region")

def checkpoint_without_rng(module, value):
    assert_no_known_dropout(module)
    return checkpoint(
        module,
        value,
        use_reentrant=False,
        preserve_rng_state=False)

def E11(module, value):
    return checkpoint_without_rng(module, value)
```

Part 6 - Measurement, Validation, and Failure Analysis

Module-type inspection cannot see random functional calls or arbitrary custom code, so it is a conservative aid rather than proof. Training and evaluation modes may activate different stochastic paths. Performance gains smaller than run variance should not justify a more fragile configuration.

Part 7 - Engineering Interpretation and Philosophy

Disabling a safeguard is valid only after proving its precondition. The useful optimization is targeted removal of unnecessary RNG bookkeeping, not a global switch that makes replay a different stochastic program.

19.19.13 Exercise 12

Exercise Title: Checkpoint Transformer Blocks Selectively Based on Measured Activation Size Instead of Applying Checkpointing Uniformly

Part 1 - Exercise Contract and Why It Matters

Measure each transformer block's saved activation bytes and checkpoint only the largest or most memory-efficient candidates. Preserve residuals, attention masks, dropout RNG, shared embeddings, and numerical equivalence while avoiding replay of low-value blocks.

Part 2 - System and Memory Mental Model

Blocks with identical architecture can save different bytes because of sequence shape, attention implementation, masks, conditional paths, or layer-specific features. Checkpointing every block maximizes policy simplicity, not necessarily efficiency. Boundary tensors and long-lived residuals remain even for selected regions.

Part 3 - Quantitative Prediction Before Execution

For block i, estimate memory benefit S_i and recomputation cost C_i. Prioritize large S_i/C_i subject to the global peak timeline, not merely total saved bytes. A block outside the peak's live interval may not reduce peak VRAM even if it saves many bytes elsewhere.

Part 4 - Experimental Design and Instrumentation

Profile blocks on representative sequence lengths and batches. Rank candidates, apply incremental selections, and remeasure actual peak after every addition. Report per-block saves, replay time, call count, total peak, step time, output error, gradient error, and whether the selected block overlaps the peak.

Part 5 - PyTorch/CUDA Implementation

```
from torch import nn
from torch.utils.checkpoint import checkpoint

class SelectiveTransformerStack(nn.Module):
    def __init__(self, blocks, selected_indices):
        super().__init__()
        self.blocks = nn.ModuleList(blocks)
        self.selected_indices = set(selected_indices)

    def forward(self, hidden, attention_mask=None):
        value = hidden
        for index, block in enumerate(self.blocks):
            if index in self.selected_indices:
                def region(input_tensor, current=block):
                    return current(input_tensor, attention_mask)
                value = checkpoint(
                    region, value, use_reentrant=False)
            else:
                value = block(value, attention_mask)
        return value

def E12(blocks, selected_indices):
    return SelectiveTransformerStack(blocks, selected_indices)
```

Part 6 - Measurement, Validation, and Failure Analysis

Closing over the current block through a default argument avoids replaying the wrong loop iteration. Masks captured from outer scope must remain unchanged and alive. Real transformer blocks have varied signatures and structured outputs; the wrapper must preserve them. Selection from one sequence length may not generalize to another.

Part 7 - Engineering Interpretation and Philosophy

Selective checkpointing spends recomputation where it purchases the most peak memory. It treats checkpoint placement as measured resource allocation rather than a uniform architectural decoration.

19.19.14 Exercise 13

Exercise Title: Build an Activation Cost Profiler That Ranks Model Blocks by Saved Activation Bytes and Recomputation Cost

Part 1 - Exercise Contract and Why It Matters

Build a profiler that attributes saved-tensor bytes, lifetime, forward time, replay calls, and estimated recomputation cost to model blocks. Produce a ranked candidate report while separating logical saved bytes from global peak impact.

Part 2 - System and Memory Mental Model

Autograd saves tensors at operation boundaries, sometimes as views or shared storage. Saved-tensor hooks observe packing and unpacking but add overhead and need storage canonicalization. Block timing measures ordinary forward; replay cost can differ because non-reentrant checkpointing may stop early.

Part 3 - Quantitative Prediction Before Execution

For each block compute unique saved bytes, retained boundary bytes, lifetime overlap with global peak, measured forward milliseconds, and estimated replay milliseconds. Rank by effective peak bytes saved divided by extra time, while keeping absolute benefit and uncertainty visible.

Part 4 - Experimental Design and Instrumentation

Attach scoped block contexts and saved-tensor hooks, run representative inputs, and map each packed storage to the active block. Time with CUDA events and record shapes, dtypes, storage IDs, and unpack time. Validate totals against memory counters and repeat to estimate variance and profiler overhead.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class BlockActivationCost:
    key: str
    saved_bytes: int
    boundary_bytes: int
    recompute_ms: float
    overlaps_peak: bool

    def effective_bytes(self):
        if not self.overlaps_peak:
            return 0
        return max(self.saved_bytes - self.boundary_bytes, 0)

    def score(self):
        return self.effective_bytes() / max(self.recompute_ms, 1e-9)

def score_value(item):
    return item[0]

def rank_activation_costs(costs):
    scored = []
```

```
    for cost in costs:
        scored.append((cost.score(), cost))
    scored.sort(key=score_value, reverse=True)
    return scored

def E13(costs):
    return rank_activation_costs(costs)
```

Part 6 - Measurement, Validation, and Failure Analysis

Summing logical tensor sizes can double-count shared storage and does not reveal simultaneous liveness. Hook and profiler overhead can change timing and memory. An activation saved by one block may be consumed in another, so attribution requires a declared rule. Always remeasure actual peak after applying a choice.

Part 7 - Engineering Interpretation and Philosophy

The profiler converts invisible autograd retention into placement evidence. Its ranking is a hypothesis about useful checkpoint locations, validated only when the resulting complete training step meets both memory and time predictions.

19.19.15 Exercise 14

Exercise Title: Design a Greedy Checkpoint Placement Algorithm That Meets a Target VRAM Budget with Minimum Estimated Recomputation

Part 1 - Exercise Contract and Why It Matters

Select checkpoint regions until predicted peak VRAM falls below a target while minimizing estimated replay. Enforce graph compatibility, mandatory boundaries, RNG policy, and mutually exclusive or overlapping region constraints before ranking candidates.

Part 2 - System and Memory Mental Model

Checkpoint benefits are not always additive because regions' saved tensors and peak intervals overlap. A greedy selector is an approximation: after each choice it must recompute marginal peak benefit for remaining candidates. Infeasible targets below mandatory live state must be rejected explicitly.

Part 3 - Quantitative Prediction Before Execution

Let current predicted peak be P and target B. At each iteration choose the legal candidate with greatest marginal peak reduction per millisecond, update the liveness model, and continue until $P \leq B$. Report estimated replay, uncheckpointable floor, safety margin, and uncertainty.

Part 4 - Experimental Design and Instrumentation

Test the algorithm on synthetic cost tables with known optima, then real block profiles. Compare greedy selection against exhaustive search for small cases. Apply the plan and measure actual peak and runtime. If prediction misses the budget, update costs and add a safety margin rather than silently OOMing.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class Candidate:
    key: str
    estimated_peak_reduction: int
    recompute_ms: float
    compatible: bool = True
    selected: bool = False

def candidate_ratio(candidate):
    return candidate.estimated_peak_reduction / max(
        candidate.recompute_ms, 1e-9)

def greedy_placement(current_peak, target_peak, candidates):
    selected = []
    while current_peak > target_peak:
        best = None
        best_ratio = -1.0
        for candidate in candidates:
            if candidate.selected or not candidate.compatible:
                continue
            ratio = candidate_ratio(candidate)
            if ratio > best_ratio:
                best = candidate
                best_ratio = ratio
        if best is None or best.estimated_peak_reduction <= 0:
            raise MemoryError("target cannot be met by legal checkpoints")
        best.selected = True
        selected.append(best.key)
        current_peak -= best.estimated_peak_reduction
    return selected, current_peak

def E14(current_peak, target_peak, candidates):
    return greedy_placement(current_peak, target_peak, candidates)
```

Part 6 - Measurement, Validation, and Failure Analysis

The compact algorithm assumes fixed independent reductions; a production version recalculates marginal benefit after each selection. Mutating candidate state means the same list cannot be reused without reset. Estimates must include boundary tensors and backward replay peaks. Greedy is not guaranteed globally optimal.

Part 7 - Engineering Interpretation and Philosophy

Checkpoint placement is a constrained optimization over the autograd graph. Greedy reasoning is valuable because it is transparent and measurable, provided its approximations are exposed and every selected plan is verified empirically.

19.19.16 Exercise 15

Exercise Title: Compare Full Checkpointing, Selective Checkpointing, and No Checkpointing on the Same Model by VRAM, Step Time, and Throughput

Part 1 - Exercise Contract and Why It Matters

Benchmark three policies on identical training transitions: no checkpointing, all eligible blocks checkpointed, and a measured selective plan. Compare peak VRAM, stage time, throughput, recomputation, and numerical correctness.

Part 2 - System and Memory Mental Model

No checkpointing retains maximal forward history. Full checkpointing minimizes many internal saves but can repeat low-value compute and RNG bookkeeping. Selective checkpointing aims to cross the memory budget with less replay. Other memory categories remain common and set a floor beneath all three policies.

Part 3 - Quantitative Prediction Before Execution

Predict peak reduction and extra forward-equivalent work for each policy. Report absolute and percentage memory savings, slowdown, examples or tokens per second, and memory saved per millisecond. Mark an otherwise fast mode infeasible if it exceeds the assigned budget.

Part 4 - Experimental Design and Instrumentation

Run each mode in an isolated process with cloned model and optimizer state, identical batches, seeds, precision, and completion boundaries. Warm up and collect median and p95 forward, backward, optimizer, and step time. Record peak allocated and reserved memory, calls, gradients, loss, and final updates.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class CheckpointModeResult:
    mode: str
    peak_vram: int
    step_ms: float
    throughput: float
    forward_calls: int
    max_gradient_error: float
    feasible: bool

def summarize_modes(results, baseline_peak, baseline_ms):
    rows = []
    for result in results:
        saved = baseline_peak - result.peak_vram
        added_time = result.step_ms - baseline_ms
        rows.append({
            "mode": result.mode,
            "feasible": result.feasible,
```

```
            "saved_vram": saved,
            "added_step_ms": added_time,
            "throughput": result.throughput,
            "forward_calls": result.forward_calls,
            "gradient_error": result.max_gradient_error,
        })
    return rows

def E15(results, baseline_peak, baseline_ms):
    return summarize_modes(results, baseline_peak, baseline_ms)
```

Part 6 - Measurement, Validation, and Failure Analysis

Full checkpointing must mean all eligible intended regions, not the final loss or an outer wrapper that saves no useful memory. Process isolation avoids peak counter contamination. Throughput comparisons are invalid if effective batch, dropout sequence, optimizer history, or compilation state differs.

Part 7 - Engineering Interpretation and Philosophy

The comparison reveals a frontier rather than one winner. Selective checkpointing is justified when it obtains the required feasibility with substantially less replay than a uniform policy.

19.19.17 Exercise 16

Exercise Title: Increase Sequence Length Until Activation Memory Dominates Training and Determine How Checkpointing Changes the Maximum Trainable Context

Part 1 - Exercise Contract and Why It Matters

Sweep sequence length under a fixed model, batch, precision, and VRAM budget; identify where activation memory dominates and measure the longest context trainable with no, selective, and full checkpointing.

Part 2 - System and Memory Mental Model

Weights are independent of sequence length, while hidden activations grow at least linearly. Conventional attention may create quadratic score or workspace terms, though fused kernels can change what is materialized. Checkpointing reduces saved activations but cannot remove all live residuals, outputs, masks, or operator workspace.

Part 3 - Quantitative Prediction Before Execution

Fit measured peak to linear and quadratic terms rather than assuming one scaling law. Subtract stable model-state bytes to estimate the dynamic share. Predict the next safe length with a safety margin, then validate it. Report context extension factor and added time per additional token.

Part 4 - Experimental Design and Instrumentation

Run each length in a fresh process, beginning safely and increasing gradually. Measure forward and backward peaks, saved-tensor bytes, step time, throughput, and failure boundary. Use identical token statistics and masks. Stop before physical exhaustion through an allocator or software budget.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class ContextResult:
    sequence_length: int
    peak_vram: int
    step_ms: float
    succeeded: bool
    policy: str

def maximum_successful_context(results, policy, budget):
    maximum = 0
    for result in results:
        if result.policy != policy:
            continue
        if not result.succeeded:
            continue
        if result.peak_vram > budget:
            continue
        maximum = max(maximum, result.sequence_length)
    return maximum

def E16(results, budget):
    output = {}
    policies = ("none", "selective", "full")
    for policy in policies:
        output[policy] = maximum_successful_context(
            results, policy, budget)
    return output
```

Part 6 - Measurement, Validation, and Failure Analysis

OOM is a censored measurement, not an exact peak. Attention backend selection can change with shape and cause discontinuities. Tokens per second can fall even when examples per second appears similar. Checkpoint selection calibrated at a short length may be wrong at the longest context.

Part 7 - Engineering Interpretation and Philosophy

Maximum context is a systems result produced by model state, activation liveness, kernel workspace, and budget. Checkpointing extends it by shortening history, but the remaining mathematical working set defines a real limit.

19.19.18 Exercise 17

Exercise Title: Combine Activation Checkpointing with Mixed Precision and Measure Whether Their Memory Savings Are Additive or Overlapping

Part 1 - Exercise Contract and Why It Matters

Run a factorial experiment with full precision or mixed precision crossed with checkpointing off or on. Measure category bytes, peak VRAM, speed, stability, and numerical quality to determine whether savings add, overlap, or move the peak to another category.

Part 2 - System and Memory Mental Model

Mixed precision reduces bytes for many activations and kernels but may retain full-precision parameters, gradients, optimizer state, or master weights. Checkpointing removes selected saved activations regardless of their dtype. Their benefits overlap on the same activation bytes, so percentage savings are rarely simply additive. Loss scaling and sensitive operations affect stability.

Part 3 - Quantitative Prediction Before Execution

Let P_{00} be the baseline peak and measure the other three cells. Compute absolute savings and interaction: observed combined saving minus the sum of individual savings. A negative interaction indicates overlap or a shifted peak; a positive result can arise from allocator or workspace regime changes.

Part 4 - Experimental Design and Instrumentation

Use identical model states and batches for four isolated modes. Record parameter, gradient, optimizer, saved activation, workspace proxy, peak VRAM, scaler state, overflow skips, gradient norms, step time, throughput, and output or loss error. Repeat across sequence lengths.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class FactorialPeak:
    mixed_precision: bool
    checkpointing: bool
    peak_vram: int
    step_ms: float
    overflow_steps: int

def interaction_effect(full_none, mixed_none,
                       full_checkpoint, mixed_checkpoint):
    mixed_saving = full_none.peak_vram - mixed_none.peak_vram
    checkpoint_saving = (
        full_none.peak_vram - full_checkpoint.peak_vram)
    combined_saving = (
        full_none.peak_vram - mixed_checkpoint.peak_vram)
    interaction = combined_saving - mixed_saving - checkpoint_saving
    return {
        "mixed_saving": mixed_saving,
        "checkpoint_saving": checkpoint_saving,
        "combined_saving": combined_saving,
        "interaction": interaction,
```

```
    }

def E17(full_none, mixed_none, full_checkpoint, mixed_checkpoint):
    return interaction_effect(
        full_none, mixed_none, full_checkpoint, mixed_checkpoint)
```

Part 6 - Measurement, Validation, and Failure Analysis

Peaks must come from isolated, warmed trials. Mixed precision modes must use the same autocast dtype and scaling policy. A skipped optimizer step after overflow is not equivalent training. The interaction number describes measured peak, not a causal byte inventory unless category measurements support it.

Part 7 - Engineering Interpretation and Philosophy

Memory techniques compose through the tensors they affect. Factorial measurement prevents double-counting the same activation saving and reveals when one method changes which resource becomes the new bottleneck.

19.19.19 Exercise 18

Exercise Title: Build an Adaptive Checkpoint Controller That Enables or Disables Checkpointing per Block from Live VRAM Pressure and Estimated Recomputation Cost

Part 1 - Exercise Contract and Why It Matters

Choose a checkpoint plan before each stable execution interval using measured VRAM pressure, predicted activation shape, block benefit, and replay cost. Avoid changing policy inside a forward-backward pair, which would make the replay function inconsistent with the original execution.

Part 2 - System and Memory Mental Model

Checkpoint decisions are part of graph construction. A block checkpointed in forward must be replayed under compatible code and state during backward. The controller therefore selects a plan at step or bucket boundaries, freezes it through backward, and adapts slowly using high and low pressure thresholds.

Part 3 - Quantitative Prediction Before Execution

Under high pressure, add legal candidates in descending marginal saved bytes per millisecond until predicted peak plus safety margin fits. Under sustained low pressure, remove the least valuable candidate to recover throughput. Include sequence-dependent cost estimates and a minimum dwell period to prevent churn.

Part 4 - Experimental Design and Instrumentation

Feed stable, rising-context, and bursty sequences. Log pressure, candidate costs, frozen plan ID, changes, prediction error, actual peak, recomputation, OOM avoidance, and throughput. Inject a mid-step change request and require deferral until the next legal plan boundary.

Part 5 - PyTorch/CUDA Implementation

```
class AdaptiveCheckpointController:
    def __init__(self, high_pressure, low_pressure):
        self.high_pressure = high_pressure
        self.low_pressure = low_pressure
        self.selected = set()
        self.plan_frozen = False

    def begin_step(self):
        if self.plan_frozen:
            raise RuntimeError("previous step plan is still frozen")
        self.plan_frozen = True
        return frozenset(self.selected)

    def end_step(self):
        if not self.plan_frozen:
            raise RuntimeError("no frozen plan to release")
        self.plan_frozen = False

    def update_between_steps(self, pressure, ranked_keys):
        if self.plan_frozen:
            raise RuntimeError("checkpoint plan cannot change mid-step")
        if pressure >= self.high_pressure:
            for key in ranked_keys:
                if key not in self.selected:
                    self.selected.add(key)
                    return "enabled", key
        elif pressure <= self.low_pressure and self.selected:
            key = sorted(self.selected)[-1]
            self.selected.remove(key)
            return "disabled", key
        return "unchanged", None

def E18():
    return AdaptiveCheckpointController(0.90, 0.70)
```

Part 6 - Measurement, Validation, and Failure Analysis

Alphabetical removal is a placeholder; production removes the lowest marginal value using current costs. Pressure sampled after OOM is too late, so use prediction and safety margin. Dynamic control can cause graph recompilation and timing changes. Distributed ranks need compatible plans when their graph contracts require it.

Part 7 - Engineering Interpretation and Philosophy

Adaptive checkpointing is controlled graph selection, not emergency mutation. It should change slowly between complete training transactions and preserve a frozen, auditable recomputation contract within each one.

19.19.20 Exercise 19

Exercise Title: Train a Transformer-Style PyTorch Model That Previously OOMed by Designing and Benchmarking a Memory-Budgeted Selective Activation Checkpointing Strategy

Part 1 - Exercise Contract and Why It Matters

Take a transformer-style training step that reproducibly exceeds a fixed VRAM budget, profile its saved activations, select checkpoint regions, and achieve stable training under the same model, batch, context, precision, and optimizer semantics. Prove memory, numerical, and performance outcomes.

Part 2 - System and Memory Mental Model

Transformer activation memory includes residual streams, normalization inputs, attention and FFN intermediates, masks, outputs, and backend workspaces. Selective checkpointing removes only eligible saved history. Parameters, gradients, optimizer state, mandatory graph boundaries, and transient recomputation peaks remain and define the uncheckpointable floor.

Part 3 - Quantitative Prediction Before Execution

Measure baseline peak and target budget, rank regions by marginal peak benefit per replay time, and select until predicted peak plus margin fits. Predict total extra forward-equivalent work, maximum context, and throughput. Reject a target below the mandatory floor rather than overpromising a plan.

Part 4 - Experimental Design and Instrumentation

Establish the OOM in an isolated process, profile safely at a smaller shape or budget, build the plan, and rerun the original task. Compare no, full, and selective checkpoint modes. Record peak allocated and reserved VRAM, saved tensors, call counts, step stages, throughput, loss, gradients, updates, overflow, RNG policy, and multi-step convergence.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class StrategyResult:
    selected_blocks: tuple
    target_vram: int
    measured_peak_vram: int
    step_ms: float
    tokens_per_second: float
    maximum_gradient_error: float
    loss_difference: float
    completed_steps: int

    def validate(self, gradient_tolerance, loss_tolerance):
        if self.measured_peak_vram > self.target_vram:
            raise MemoryError("checkpoint strategy missed VRAM budget")
        if self.maximum_gradient_error > gradient_tolerance:
```

```
            raise RuntimeError("gradient equivalence failed")
        if self.loss_difference > loss_tolerance:
            raise RuntimeError("loss equivalence failed")
        if self.completed_steps <= 0:
            raise RuntimeError("training did not complete a step")
        return True

def E19(selected_blocks, target_vram):
    return StrategyResult(
        tuple(selected_blocks), target_vram, 0,
        0.0, 0.0, 0.0, 0.0, 0)
```

Part 6 - Measurement, Validation, and Failure Analysis

The returned object is an experiment record whose measured fields must be filled from completed runs, not a trainer by itself. A single successful step does not prove stability; run multiple updates and inspect scaler or loss behavior. Profiler settings, sequence length, attention backend, and compilation can move the peak and invalidate a stale plan.

Part 7 - Engineering Interpretation and Philosophy

The final strategy converts a previously impossible training configuration into a measured feasible one by spending compute only where it buys peak memory. Its success is defined by preserved learning semantics under budget, with the runtime cost stated openly rather than hidden behind an OOM-free result.

19.20 DMM18

Chapter Title: Activation Offloading to CPU Memory
Specialization: Dynamic Memory Management and CUDA Systems

19.20.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Activation Offloading and Compare Transfer Cost Against Recomputation Cost

Part 1 - Exercise Contract and Why It Matters

Build a lifecycle model for tensors saved by autograd: creation, packing, D2H completion, CPU residency, H2D prefetch or unpack, backward consumption, and release. Compare three legal choices for each candidate: keep on GPU, offload to CPU, or remove through checkpointed recomputation.

Part 2 - System and Memory Mental Model

Saved-tensor hooks replace the object retained by autograd with a packed object and reconstruct a tensor at unpack. CPU offload preserves the activation value but pays D2H, RAM, and H2D. Recomputation avoids those transfers but repeats forward work. Keeping avoids both costs but consumes VRAM across the activation's lifetime. Pinned asynchronous copies require events and storage leases.

Part 3 - Quantitative Prediction Before Execution

For activation bytes M, offload cost is D2H plus H2D latency and host-memory pressure, reduced by overlap. Recomputation cost is replay time and any RNG or state overhead. Keeping cost is M bytes times its live interval under VRAM pressure. Include transient source-destination coexistence during transfers.

Part 4 - Experimental Design and Instrumentation

Measure each candidate's size, producer, pack time, CPU-ready time, lifetime, unpack deadline, H2D completion, and backward use. Independently measure replay cost for its containing checkpoint region. Compare policies under identical seeds and verify outputs, losses, gradients, updates, RAM, VRAM, and step time.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class ActivationDecision:
    key: str
    size_bytes: int
    lifetime_ms: float
    d2h_ms: float
    h2d_ms: float
    recompute_ms: float
    policy: str = "keep"

    def offload_cost(self):
        return self.d2h_ms + self.h2d_ms

    def cheaper_recovery(self):
        if self.recompute_ms < self.offload_cost():
            return "recompute"
        return "offload"

def E00(key, size_bytes, lifetime_ms,
        d2h_ms, h2d_ms, recompute_ms):
    return ActivationDecision(
        key, size_bytes, lifetime_ms,
        d2h_ms, h2d_ms, recompute_ms)
```

Part 6 - Measurement, Validation, and Failure Analysis

Autograd may save views or shared storages, so logical bytes can double-count physical memory. Pack return does not imply asynchronous D2H completion. Unpack may be called more than once and must return correct content each time. Offload and recomputation decisions interact with the global peak, not only local size.

Part 7 - Engineering Interpretation and Philosophy

Activation management chooses which scarce resource pays for training history: VRAM capacity, PCIe bandwidth and RAM, or GPU compute. The best policy is tensor- and graph-specific, grounded in measured lifetime and recovery cost.

19.20.2 Exercise 01

Exercise Title: Measure the VRAM Occupied by Saved Tensors Required for Backward in a Deep PyTorch Training Step

Part 1 - Exercise Contract and Why It Matters

Measure tensors retained specifically for backward and relate their unique storage bytes to the forward-to-backward VRAM delta. Separate them from parameters, outputs held by user code, residual state, and temporary workspaces.

Part 2 - System and Memory Mental Model

Autograd nodes retain selected values needed by derivative formulas, not every forward output. A saved tensor may be packed as a different object while sharing storage. Its logical lifetime ends when the relevant backward node consumes it, unless retain-graph or Python references extend it.

Part 3 - Quantitative Prediction Before Execution

Sum unique storage ranges observed by a saved-tensor pack hook. Compare this logical total with allocated memory immediately before and after forward. Differences arise from outputs, temporaries, allocator reuse, aliases, and workspace. Measure the backward release curve rather than one final sample.

Part 4 - Experimental Design and Instrumentation

Warm up, reset peaks, and install non-transforming hooks only around forward. Record shape, dtype, device, storage identity, offset, and bytes for every save. Sample CUDA memory at block boundaries and backward hooks. Verify that hooks do not alter values and report their overhead.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class SavedByteCounter:
    def __init__(self):
        self.logical_bytes = 0
        self.records = []

    def pack(self, tensor):
        size = tensor.numel() * tensor.element_size()
        self.logical_bytes += size
        self.records.append({
            "shape": tuple(tensor.shape),
            "dtype": str(tensor.dtype),
            "device": str(tensor.device),
            "bytes": size,
        })
        return tensor

    def unpack(self, packed):
```

```
        return packed

def E01(model, input_tensor):
    counter = SavedByteCounter()
    with torch.autograd.graph.saved_tensors_hooks(
            counter.pack, counter.unpack):
        output = model(input_tensor)
    return output, counter
```

Part 6 - Measurement, Validation, and Failure Analysis

The counter sums logical sizes and can count aliases repeatedly; add storage range canonicalization for physical estimates. Returning the original tensor retains ordinary VRAM and is for measurement only. Hooks apply only to operations executed inside the context, and their Python work can perturb timing.

Part 7 - Engineering Interpretation and Philosophy

Saved-tensor measurement locates the history autograd truly requires. It turns activation offload from a broad model option into a targeted intervention on specific long-lived storage.

19.20.3 Exercise 02

Exercise Title: Inspect Which Intermediate Tensors Autograd Saves for Backward and Estimate Their Total Memory Footprint

Part 1 - Exercise Contract and Why It Matters

Inspect saved tensors by producer scope and derivative role, identify aliases, and estimate unique physical footprint and lifetime. Avoid relying on private `grad_fn` attribute names as the primary engineering interface.

Part 2 - System and Memory Mental Model

Derivative formulas save operands, results, masks, indices, or statistics according to operation. Packing may break Python object identity to avoid cycles while preserving storage or value. A tensor saved several times has several logical uses but may occupy one physical storage until all uses finish.

Part 3 - Quantitative Prediction Before Execution

Record logical bytes for each save, then canonicalize by storage identity and byte range. Calculate logical total, unique range estimate, reuse count, and largest candidates. Combine save time with unpack time to estimate the interval over which offloading could free VRAM.

Part 4 - Experimental Design and Instrumentation

Maintain an active producer-layer stack through forward hooks and associate each pack call with the current scope. During unpack, record backward order. Compare linear, normalization, activation, dropout, and residual blocks. Validate totals against memory counters and repeat with multiple input shapes.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class SavedTensorRecord:
    sequence: int
    producer: str
    shape: tuple
    dtype: str
    device: str
    size_bytes: int
    unpack_count: int = 0

class SavedTensorInventory:
    def __init__(self):
        self.records = []
        self.active_producer = "unknown"

    def pack(self, tensor):
        record = SavedTensorRecord(
            len(self.records), self.active_producer,
            tuple(tensor.shape), str(tensor.dtype),
            str(tensor.device),
            tensor.numel() * tensor.element_size())
        self.records.append(record)
        return record.sequence, tensor

    def unpack(self, packed):
        sequence, tensor = packed
        self.records[sequence].unpack_count += 1
        return tensor

def E02():
    return SavedTensorInventory()
```

Part 6 - Measurement, Validation, and Failure Analysis

This inventory returns the original tensor and therefore does not offload it. An active Python scope can be wrong for asynchronous or nested module behavior; use carefully nested hooks and operator profiling for stronger attribution. Private autograd internals are useful diagnostics but not stable contracts.

Part 7 - Engineering Interpretation and Philosophy

Inspection reveals that activation memory is created by derivative requirements, not by a vague property called depth. Good policies follow actual saved values, their identities, and their future consumers.

19.20.4 Exercise 03

Exercise Title: Implement Custom saved_tensors_hooks That Move Saved CUDA Activations to CPU and Restore Them During Backward

Part 1 - Exercise Contract and Why It Matters

Implement correctness-first pack and unpack hooks that copy CUDA saved tensors to CPU, preserve metadata, and restore them to the original device when backward requests them. Begin with blocking semantics before optimizing asynchronously.

Part 2 - System and Memory Mental Model

The pack hook's return value becomes the object stored in the graph. It can be a record containing a CPU tensor and original device. The unpack hook must produce a tensor equivalent to the original saved value. CPU authority persists between forward and backward; H2D occurs on demand and may become a backward stall.

Part 3 - Quantitative Prediction Before Execution

VRAM savings approach the size of eligible CUDA saves minus transfer overlap and boundary state. RAM increases by offloaded payload. Each saved byte generally crosses D2H and H2D once, though repeated unpack may add reloads unless cached. Predict total traffic and lower-bound time from measured bandwidth.

Part 4 - Experimental Design and Instrumentation

Use a deterministic model and record pack/unpack order, bytes, devices, timing, RAM, and VRAM. Compare output, loss, gradients, and updates with ordinary training. Exercise CPU saves, zero-sized tensors, noncontiguous views, several dtypes, and repeated unpack.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass
import torch

@dataclass
class CpuSavedTensor:
    tensor: object
    original_device: object

class BlockingActivationOffload:
    def __init__(self):
        self.d2h_bytes = 0
        self.h2d_bytes = 0

    def pack(self, tensor):
        if tensor.device.type != "cuda":
            return tensor
        cpu_tensor = tensor.detach().to("cpu")
        size = tensor.numel() * tensor.element_size()
        self.d2h_bytes += size
        return CpuSavedTensor(cpu_tensor, tensor.device)

    def unpack(self, packed):
        if not isinstance(packed, CpuSavedTensor):
            return packed
```

```
        size = packed.tensor.numel() * packed.tensor.element_size()
        self.h2d_bytes += size
        return packed.tensor.to(packed.original_device)

def E03(model, input_tensor):
    manager = BlockingActivationOffload()
    with torch.autograd.graph.saved_tensors_hooks(
            manager.pack, manager.unpack):
        output = model(input_tensor)
    return output, manager
```

Part 6 - Measurement, Validation, and Failure Analysis

`detach()` here removes the copied CPU tensor from a new autograd history; the original graph still owns the packed record and uses the unpacked value for its derivative. Blocking transfers are safe but slow. View strides and aliasing may not be preserved by every copy path, so validate operations requiring layout.

Part 7 - Engineering Interpretation and Philosophy

The custom hook makes the storage substitution explicit: autograd retains a recoverable CPU value instead of a CUDA value. Correctness-first blocking code is the baseline against which every asynchronous optimization must be judged.

19.20.5 Exercise 04

Exercise Title: Verify Gradient Equivalence Between Normal Training and CPU-Offloaded Saved Activations

Part 1 - Exercise Contract and Why It Matters

Compare ordinary and offloaded training from identical model, optimizer, batch, seed, and mode. Verify outputs, losses, every parameter gradient, updated parameters, and optimizer state before accepting memory savings.

Part 2 - System and Memory Mental Model

Offload should alter only saved-tensor residency. Pack and unpack must preserve value, dtype, shape, device semantics, and required layout. Stochastic layers, mutable buffers, mixed precision, and nondeterministic kernels can complicate equivalence even when storage movement is correct.

Part 3 - Quantitative Prediction Before Execution

Set absolute and relative tolerances appropriate to dtype and device. Expect exact or close losses and gradients for deterministic paths. Compare maximum error and allclose for each named tensor; one aggregate norm can conceal a localized corrupted activation.

Part 4 - Experimental Design and Instrumentation

Deep-copy initial state, pre-generate data, and run one completed step per mode in isolated trials. Repeat several seeds and include dropout with controlled RNG. Collect offloaded bytes, peak VRAM, RAM, time, output error, gradient error, parameter error, optimizer-state error, and finite-value checks.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def named_gradient_report(reference, candidate, atol, rtol):
    candidate_parameters = dict(candidate.named_parameters())
    report = {}
    for name, reference_parameter in reference.named_parameters():
        candidate_parameter = candidate_parameters[name]
        first = reference_parameter.grad
        second = candidate_parameter.grad
        if first is None or second is None:
            report[name] = first is second
            continue
        close = torch.allclose(first, second, atol=atol, rtol=rtol)
        maximum = (first - second).abs().max().item()
        report[name] = (close, maximum)
    return report

def E04(reference_model, offloaded_model):
    return named_gradient_report(
        reference_model, offloaded_model, 1e-6, 1e-5)
```

Part 6 - Measurement, Validation, and Failure Analysis

Matching gradients before optimizer step is necessary but incomplete; compare updates and optimizer state too. Reusing one model sequentially changes RNG and optimizer history. A synchronization bug may pass small tensors by chance, so repeat with large pinned transfers and deliberate stream pressure.

Part 7 - Engineering Interpretation and Philosophy

Gradient equivalence proves that the CPU copy is a faithful representation of training history. Without that proof, lower VRAM may simply mean backward used stale, incomplete, or different values.

19.20.6 Exercise 05

Exercise Title: Use torch.autograd.graph.save_on_cpu() to Offload Saved Forward Tensors Without Writing Custom Hooks

Part 1 - Exercise Contract and Why It Matters

Apply PyTorch's built-in `save_on_cpu` context around a measured model region, then verify peak VRAM reduction, RAM growth, step latency, and gradient equivalence without maintaining custom pack and unpack logic.

Part 2 - System and Memory Mental Model

Operations executed inside the context save their backward tensors on CPU and restore them to the original device when needed. Context scope matters: wrapping only one block affects that block's saves; wrapping an entire forward affects all eligible saves created inside.

The built-in mechanism reduces implementation risk but does not eliminate D2H and H2D cost.

Part 3 - Quantitative Prediction Before Execution

Predict RAM increase and transfer traffic from the saved tensors observed in the same region. Peak VRAM reduction can be smaller than logical bytes if another region defines the peak or transfer copies overlap in lifetime. Compare pageable and pinned behavior separately in the next exercise.

Part 4 - Experimental Design and Instrumentation

Run ordinary and built-in-offload steps from cloned state. Record forward, backward, total time, peak allocated and reserved VRAM, process RAM, output, loss, gradients, and update. Test nested scopes and prove only operations created inside the intended region are affected.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def forward_with_cpu_saved_tensors(model, value, pin_memory=False):
    with torch.autograd.graph.save_on_cpu(
            pin_memory=pin_memory,
            device_type="cuda"):
        output = model(value)
    return output

def E05(model, value):
    torch.cuda.reset_peak_memory_stats()
    output = forward_with_cpu_saved_tensors(
        model, value, pin_memory=False)
    return output, torch.cuda.max_memory_allocated()
```

Part 6 - Measurement, Validation, and Failure Analysis

The context applies during forward graph construction; backward can occur after the context exits. CPU RAM may remain high until backward releases the packed objects. Device movement inside the region and non-CUDA accelerators require explicit testing. A built-in API still needs numerical validation for the actual model and precision.

Part 7 - Engineering Interpretation and Philosophy

`save_on_cpu` provides a strong correctness baseline and a compact production option. Custom hooks are justified only when selective policy, pooling, asynchronous scheduling, or telemetry delivers measured additional value.

19.20.7 Exercise 06

Exercise Title: Compare save_on_cpu(pin_memory=False) and save_on_cpu(pin_memory=True) by Peak VRAM, Host RAM, and Step Latency

Part 1 - Exercise Contract and Why It Matters

Compare pageable and pinned CPU storage modes under identical saved tensors. Measure complete forward and backward behavior, including pinned allocation cost, host pressure, transfer capability, peak VRAM, and step latency.

Part 2 - System and Memory Mental Model

Pageable CPU memory is easier for the operating system to manage but does not provide the same asynchronous DMA path. Pinned pages can accelerate or overlap copies while consuming a scarce host resource. Pinning many short-lived saved tensors can add allocation overhead and fragment a workload into small transfers.

Part 3 - Quantitative Prediction Before Execution

Both modes offload similar logical payload, so ideal VRAM saving is similar. Pinned mode adds locked RAM equal to relevant packed storage and can reduce H2D stall. Predict a crossover by tensor size and lifetime; for small tensors, fixed pin and launch overhead may dominate any bandwidth benefit.

Part 4 - Experimental Design and Instrumentation

Run modes in isolated processes with the same warmup, data, state, and cache conditions. Record RSS, system available RAM, pinned payload if observable, pack and unpack counts, D2H and H2D completion, forward and backward time, peak VRAM, throughput, and gradients. Sweep tensor size and saved-tensor count.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass
import torch

@dataclass
class SaveOnCpuResult:
    pin_memory: bool
    peak_vram: int
    peak_ram: int
    step_ms: float
    gradient_error: float

def run_save_on_cpu_forward(model, value, pin_memory):
    with torch.autograd.graph.save_on_cpu(
            pin_memory=pin_memory,
            device_type="cuda"):
        return model(value)

def compare_save_modes(pageable, pinned):
    return {
        "vram_difference": pinned.peak_vram - pageable.peak_vram,
        "ram_difference": pinned.peak_ram - pageable.peak_ram,
        "step_difference_ms": pinned.step_ms - pageable.step_ms,
        "pageable_gradient_error": pageable.gradient_error,
        "pinned_gradient_error": pinned.gradient_error,
    }
```

```
def E06(pageable, pinned):
    return compare_save_modes(pageable, pinned)
```

Part 6 - Measurement, Validation, and Failure Analysis

Process RSS does not identify locked pages precisely, and system noise affects available RAM. A lower Python submission time is not completed transfer time. Pinned mode can harm the machine if used without a host budget. Compare repeated steady-state steps, not only first-time allocation.

Part 7 - Engineering Interpretation and Philosophy

Pinned memory is valuable when it shortens the backward critical path enough to justify locked host capacity. The best mode is measured per workload, not chosen from the word asynchronous alone.

19.20.8 Exercise 07

Exercise Title: Reproduce an Incorrect Result Caused by Accessing Asynchronously Offloaded Pinned CPU Data Before the Transfer Completes

Part 1 - Exercise Contract and Why It Matters

Deliberately omit the dependency between D2H completion and later H2D reload, then reproduce stale or corrupted activation recovery under stream pressure. Capture the failure as a test and never use the unsafe hook in real training.

Part 2 - System and Memory Mental Model

Non-blocking D2H returns after enqueue, while the pinned destination may still be changing. If unpack reads it on another stream without waiting, the reload can race the offload. Reusing or modifying the pinned buffer introduces a second race. Correctness can appear accidental for small copies or idle hardware.

Part 3 - Quantitative Prediction Before Execution

The race window grows with activation size, copy-engine contention, and immediate backward demand. Predict higher failure probability when a long operation delays D2H and another stream begins H2D. Compare checksums before submission, after true completion, and after unsafe reload.

Part 4 - Experimental Design and Instrumentation

Run only on disposable synthetic tensors. Use large patterned activations, separate streams, deliberate competing work, and repeated seeds. Record events without waiting, unsafe result error, and safe reference error. Mark a run inconclusive rather than correct when the race does not manifest.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass
import torch

@dataclass
class UnsafePacked:
    cpu_tensor: object
    original_device: object

class UnsafeActivationHooks:
    def __init__(self):
        self.offload_stream = torch.cuda.Stream()

    def pack(self, tensor):
        if tensor.device.type != "cuda":
            return tensor
        destination = torch.empty_like(
            tensor, device="cpu", pin_memory=True)
        with torch.cuda.stream(self.offload_stream):
            destination.copy_(tensor, non_blocking=True)
        return UnsafePacked(destination, tensor.device)

    def unpack(self, packed):
        if not isinstance(packed, UnsafePacked):
            return packed
        # Intentionally unsafe: no wait for D2H completion.
        return packed.cpu_tensor.to(
            packed.original_device, non_blocking=True)

def E07():
    return UnsafeActivationHooks()
```

Part 6 - Measurement, Validation, and Failure Analysis

This code is intentionally incorrect and belongs only in a controlled negative test. It also fails to protect the CUDA source from allocator reuse on the side stream. Lack of observed corruption does not establish safety; asynchronous ordering must be proven through events and lifetimes.

Part 7 - Engineering Interpretation and Philosophy

Asynchrony removes waiting from the caller, not dependencies from the program. A transfer without an explicit completion relationship is unfinished state, and using it is a correctness violation even when timing happens to hide the bug.

19.20.9 Exercise 08

Exercise Title: Build Correct Synchronization Around Pinned Activation Offloading Without Falling Back to Device-Wide Synchronization Everywhere

Part 1 - Exercise Contract and Why It Matters

Repair asynchronous activation offload with producer readiness, dedicated D2H stream, completion event, source lifetime protection, pinned ownership, and a consumer-stream wait. Avoid device-wide synchronization and verify gradient equivalence under stress.

Part 2 - System and Memory Mental Model

The offload stream waits for the forward producer event, then copies to pinned RAM and records D2H completion. The source records use by the side stream so the allocator cannot recycle it prematurely. Unpack makes the current backward stream wait for D2H before H2D. Stream ordering then delays consumers until the restored CUDA tensor is ready.

Part 3 - Quantitative Prediction Before Execution

Event operations add small fixed cost but allow unrelated work to continue. Peak memory includes source and pinned destination until D2H completes and the restored destination during H2D. Compare visible stall with device-wide synchronization and calculate overlap recovered.

Part 4 - Experimental Design and Instrumentation

Stress with large tensors, multiple saves, side-stream compute, and immediate backward. Verify checksums and gradients over many repetitions. Export a CUDA trace showing event waits rather than global barriers. Record D2H/H2D bytes, wait time, peak VRAM, pinned RAM, and completed step time.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass
import torch

@dataclass
class SafePacked:
    cpu_tensor: object
    original_device: object
    d2h_done: object

class SafeActivationHooks:
    def __init__(self):
        self.offload_stream = torch.cuda.Stream()

    def pack(self, tensor):
        if tensor.device.type != "cuda":
            return tensor
        destination = torch.empty_like(
            tensor, device="cpu", pin_memory=True)
        producer_ready = torch.cuda.Event()
        producer_ready.record(torch.cuda.current_stream())
        done = torch.cuda.Event()
        with torch.cuda.stream(self.offload_stream):
            self.offload_stream.wait_event(producer_ready)
            destination.copy_(tensor, non_blocking=True)
            tensor.record_stream(self.offload_stream)
            done.record(self.offload_stream)
```

```
        return SafePacked(destination, tensor.device, done)

    def unpack(self, packed):
        if not isinstance(packed, SafePacked):
            return packed
        current = torch.cuda.current_stream(packed.original_device)
        current.wait_event(packed.d2h_done)
        return packed.cpu_tensor.to(
            packed.original_device, non_blocking=True)

def E08():
    return SafeActivationHooks()
```

Part 6 - Measurement, Validation, and Failure Analysis

The packed object retains pinned storage and the event. The returned H2D tensor is ordered on the current stream; unusual multi-device graphs need device-aware stream handling. Fresh pinned allocation in every pack remains expensive and is replaced by a bounded pool later. Events must be created on compatible devices.

Part 7 - Engineering Interpretation and Philosophy

Correct synchronization names the exact dependency instead of stopping the entire device. Events preserve concurrency because they make only the consumer of a particular activation wait for the transfer that produces it.

19.20.10 Exercise 09

Exercise Title: Offload Activations from Only Selected Model Blocks and Determine Which Blocks Provide the Best VRAM Saving per Transfer Byte

Part 1 - Exercise Contract and Why It Matters

Apply saved-tensor hooks only around selected block forwards, measure actual peak reduction and transfer traffic, and rank blocks by useful peak bytes saved per byte moved and per millisecond of stall.

Part 2 - System and Memory Mental Model

Hook scope determines which autograd saves are packed. A block can save many bytes outside the global peak interval and therefore offer little peak benefit. Every offloaded byte generally incurs both D2H and H2D traffic. Small tensors may cost more in launches and bookkeeping than they save in VRAM.

Part 3 - Quantitative Prediction Before Execution

For block i, calculate measured marginal peak reduction divided by total D2H plus H2D bytes. Also report stall per saved MiB and RAM residency. Recompute marginal benefit after selecting another block because liveness and the peak location can change.

Part 4 - Experimental Design and Instrumentation

Profile each block alone, then incrementally combine high-ranked blocks. Use identical model state and inputs. Record packed tensor count and bytes, unique storage, pack/unpack

time, peak VRAM, RAM, total step time, output and gradient error, and whether the block overlaps the original peak.

Part 5 - PyTorch/CUDA Implementation

```
from torch import nn

class SelectiveActivationOffload(nn.Module):
    def __init__(self, blocks, selected, manager):
        super().__init__()
        self.blocks = nn.ModuleList(blocks)
        self.selected = set(selected)
        self.manager = manager

    def forward(self, value):
        result = value
        for index, block in enumerate(self.blocks):
            if index in self.selected:
                with torch.autograd.graph.saved_tensors_hooks(
                        self.manager.pack, self.manager.unpack):
                    result = block(result)
            else:
                result = block(result)
        return result

def E09(blocks, selected, manager):
    return SelectiveActivationOffload(blocks, selected, manager)
```

Part 6 - Measurement, Validation, and Failure Analysis

The code requires `torch` imported by the surrounding program; include it in a standalone script. Nested or overlapping hook contexts need explicit tests. One block's output may be saved by an operation executed in the next block, so producer attribution and context attribution are related but not identical.

Part 7 - Engineering Interpretation and Philosophy

Selective offload spends bandwidth only where tensor lifetime creates useful peak relief. Its quality is measured by avoided VRAM pressure, not by the gross number of bytes successfully copied to CPU.

19.20.11 Exercise 10

Exercise Title: Build an Activation Profiler That Records Tensor Size, Producer Layer, Lifetime, Device, and Backward Reuse Point

Part 1 - Exercise Contract and Why It Matters

Build an observation layer around autograd saved tensors. For every save, the profiler must record shape, dtype, device, logical producer, bytes, save order, unpack order, and elapsed lifetime. The result is not merely a list of large tensors: it is a liveness trace that explains

which values remain resident across the forward/backward boundary and when they become useful again.

Part 2 - System and Memory Mental Model

Autograd saves values rather than "layers." One operator can save several tensors, shared storage can appear through several views, and an activation created in one module may be saved by an operation in another module. A hook observes the save and restore boundary, while a module scope supplies a useful but approximate producer label. Lifetime is the interval from pack to first unpack; reuse order is often more actionable than wall-clock lifetime because the latter changes with hardware and scheduling.

Part 3 - Quantitative Prediction Before Execution

For tensor i, estimate $b_i = n_i s_i$ bytes, where n_i is the element count and s_i is element size. Predict peak saved bytes from the tensors live at each forward boundary, not by summing all saves blindly. Estimate transfer work as $2b_i$ when one D2H and one H2D copy are required. Predict that late backward consumers have longer reuse distance and therefore offer more time for prefetch, but also occupy host memory longer.

Part 4 - Experimental Design and Instrumentation

Run one warmup and one measured step with deterministic inputs. Attach module scope labels, reset CUDA peak statistics, and synchronize only at benchmark boundaries. Compare logical bytes, unique-storage bytes, peak VRAM, and hook overhead. Keep CPU timestamps separate from CUDA-event timing: they answer different questions and do not share a guaranteed clock domain.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass
import time
import torch

@dataclass
class ActivationRecord:
    record_id: int
    producer: str
    shape: tuple
    dtype: object
    device: object
    size_bytes: int
    save_order: int
    unpack_order: int
    saved_at: float
    unpacked_at: float

class ActivationProfiler:
    def __init__(self):
        self.records = []
        self.scope = "unscoped"
        self.unpack_count = 0
```

```
    def set_scope(self, name):
        self.scope = name

    def pack(self, tensor):
        size_bytes = tensor.numel() * tensor.element_size()
        record = ActivationRecord(
            len(self.records), self.scope, tuple(tensor.shape),
            tensor.dtype, tensor.device, size_bytes,
            len(self.records), -1, time.perf_counter(), -1.0)
        self.records.append(record)
        return (record.record_id, tensor)

    def unpack(self, packed):
        record_id, tensor = packed
        record = self.records[record_id]
        record.unpack_order = self.unpack_count
        record.unpacked_at = time.perf_counter()
        self.unpack_count += 1
        return tensor

    def report(self):
        rows = []
        for record in self.records:
            lifetime = record.unpacked_at - record.saved_at
            rows.append((record.record_id, record.producer,
                         record.size_bytes, lifetime,
                         record.unpack_order))
        return rows

def E10():
    return ActivationProfiler()
```

Part 6 - Measurement, Validation, and Failure Analysis

Identity packing intentionally leaves memory on CUDA; this exercise measures the baseline before changing residency. A tensor unpacked more than once needs all reuse points, not one overwritten field. Storage aliases can inflate the logical sum, while Python timing includes scheduling and hook overhead. Validate the inventory by comparing a no-hook step, an identity-hook step, and allocator peaks; reject any profiler that changes outputs or gradients.

Part 7 - Engineering Interpretation and Philosophy

Memory optimization begins with liveness evidence. Size says how much can be saved, lifetime says how long the capacity is recovered, and reuse order says whether the data can return before it is needed. None of those dimensions is a safe substitute for the others.

19.20.12 Exercise 11

Exercise Title: Classify Saved Activations as GPU-Resident or CPU-Offloaded from Their Size, Lifetime, Reuse Distance, and Transfer Cost

Part 1 - Exercise Contract and Why It Matters

Turn the profiler output into an explicit placement decision. The classifier must explain why each activation remains in VRAM or moves to RAM, enforce hard correctness and capacity constraints, and expose the cost assumptions behind its answer. A useful policy is auditable; it does not hide a bandwidth gamble inside a single unexplained threshold.

Part 2 - System and Memory Mental Model

Keeping consumes VRAM for the tensor's remaining lifetime but creates no copy. Offloading consumes host capacity and schedules D2H plus H2D traffic. Long reuse distance increases the opportunity to hide those copies, whereas high reuse count can make eviction expensive. The decision also depends on global pressure: a tensor that should normally stay may need to move when keeping it would cross the VRAM high watermark.

Part 3 - Quantitative Prediction Before Execution

Estimate transfer time as $t_{off} = b/B_{D2H} + b/B_{H2D} + t_{launch} + t_{queue}$. Compare it with the visible residency value $b\,t_{life}$ and predicted overlap window. Calculate both an optimistic stall, $\max(0, t_{H2D} - t_{prefetch})$, and a pessimistic stall with no overlap. A policy must report its measured bandwidth inputs and must not treat nominal PCIe bandwidth as achieved application bandwidth.

Part 4 - Experimental Design and Instrumentation

Calibrate D2H and H2D using representative pinned tensors, then replay recorded activation metadata under several VRAM/RAM budgets. Measure decision accuracy by actual peak relief, transfer bytes, host peak, and exposed backward stall. Include short-lived large values, long-lived small values, repeated consumers, and a budget that makes some offloading mandatory.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class ActivationCost:
    name: str
    size_bytes: int
    lifetime_ms: float
    reuse_distance_ms: float
    reuse_count: int
    d2h_ms: float
    h2d_ms: float

@dataclass
class PlacementDecision:
    name: str
    policy: str
    predicted_stall_ms: float
    reason: str

def classify_activation(item, vram_headroom, ram_headroom):
```

```
    if item.size_bytes > ram_headroom:
        return PlacementDecision(
            item.name, "keep", 0.0, "insufficient host budget")
    if item.size_bytes > vram_headroom:
        stall = max(0.0, item.h2d_ms - item.reuse_distance_ms)
        return PlacementDecision(
            item.name, "offload", stall, "required by VRAM budget")
    transfer_ms = item.d2h_ms + item.h2d_ms
    hidden_ms = min(item.reuse_distance_ms, transfer_ms)
    visible_ms = transfer_ms - hidden_ms
    residency_score = item.size_bytes * item.lifetime_ms
    transfer_score = item.size_bytes * (visible_ms + item.reuse_count)
    if residency_score > transfer_score:
        return PlacementDecision(
            item.name, "offload", visible_ms,
            "residency value exceeds transfer score")
    return PlacementDecision(
        item.name, "keep", 0.0, "transfer is not justified")

def E11(items, vram_headroom, ram_headroom):
    decisions = []
    for item in items:
        decision = classify_activation(
            item, vram_headroom, ram_headroom)
        decisions.append(decision)
    return decisions
```

Part 6 - Measurement, Validation, and Failure Analysis

The illustrative score mixes units deliberately only after normalization would be added in a real controller; raw byte-milliseconds and byte-weighted counts must not be tuned as though they were universal. Recalibrate after decisions, because concurrent transfers contend and the peak location moves. Fail closed to keeping when metadata, ownership, or synchronization safety is unknown.

Part 7 - Engineering Interpretation and Philosophy

Placement is a constrained scheduling problem, not a tensor-size rule. The right question is not "is this activation large?" but "what scarce resource will be spent if it stays, and what scarce resource will be spent if it moves?"

19.20.13 Exercise 12

Exercise Title: Implement an Activation Offload Threshold That Moves Only Saved Tensors Larger Than a Configurable Byte Limit

Part 1 - Exercise Contract and Why It Matters

Implement the simplest controllable policy: offload only CUDA tensors whose logical byte size reaches a chosen limit. Sweep the limit rather than choosing it by intuition, preserve all

values exactly, and demonstrate where fewer tiny copies improve latency without sacrificing the desired peak-VRAM reduction.

Part 2 - System and Memory Mental Model

Every saved tensor passes through the hook. Returning the original tensor keeps it alive on CUDA; returning a CPU representation permits the graph to release the hook's CUDA reference after packing. The threshold reduces launch, allocation, event, and metadata overhead, but it ignores lifetime and peak overlap. It is therefore a baseline policy, not an optimal one.

Part 3 - Quantitative Prediction Before Execution

From the E10 histogram, compute tensor count and bytes above each candidate threshold. Predict traffic as twice the selected bytes and pinned RAM as the maximum simultaneously live selected bytes. Predict diminishing returns when lowering the threshold adds many objects but little total capacity.

Part 4 - Experimental Design and Instrumentation

Test thresholds of zero, selected quantiles, one MiB, and infinity. Hold model, batch, seeds, and optimizer fixed. Record selected count, selected bytes, peak VRAM, host RAM, step time, D2H/H2D bandwidth, output error, and gradient error. Warm the allocator and use CUDA events around completed GPU work.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass
import torch

@dataclass
class ThresholdPacked:
    tensor: object
    original_device: object
    was_offloaded: bool

class ThresholdOffload:
    def __init__(self, threshold_bytes):
        self.threshold_bytes = threshold_bytes
        self.offloaded_bytes = 0
        self.offloaded_count = 0

    def pack(self, tensor):
        size_bytes = tensor.numel() * tensor.element_size()
        should_move = tensor.device.type == "cuda"
        should_move = should_move and size_bytes >= self.threshold_bytes
        if not should_move:
            return ThresholdPacked(tensor, tensor.device, False)
        cpu_tensor = tensor.to("cpu")
        self.offloaded_bytes += size_bytes
        self.offloaded_count += 1
        return ThresholdPacked(cpu_tensor, tensor.device, True)
```

```
    def unpack(self, packed):
        if not packed.was_offloaded:
            return packed.tensor
        return packed.tensor.to(packed.original_device)

def E12(threshold_bytes):
    return ThresholdOffload(threshold_bytes)
```

Part 6 - Measurement, Validation, and Failure Analysis

Blocking copies make this reference implementation easy to validate but slow; replace its transport with the event-safe mechanism from E08 after proving policy correctness. Views and shared storage make logical bytes differ from unique bytes. A lower peak with worse throughput may still be correct when it enables an otherwise impossible batch, so report capacity and speed separately.

Part 7 - Engineering Interpretation and Philosophy

A threshold is valuable because it is understandable, reproducible, and easy to falsify. Its limitation is equally important: size is only a proxy for the actual objective, which is useful peak relief per unit of transfer and stall.

19.20.14 Exercise 13

Exercise Title: Build a Pinned CPU Activation Buffer Pool and Reuse Host Storage Instead of Allocating New Offload Buffers Repeatedly

Part 1 - Exercise Contract and Why It Matters

Replace per-save pinned allocation with a bounded, reusable host buffer pool. The pool must lease storage by capacity and dtype, prevent reuse while a D2H or H2D operation still references it, respect a pinned-RAM budget, and expose hit, miss, wait, and fallback statistics.

Part 2 - System and Memory Mental Model

Pinned pages accelerate asynchronous DMA but are a constrained system resource. A buffer is not free merely because Python no longer needs its previous value; it is free only after every stream using that storage has completed. Pooling amortizes pinning cost and stabilizes host allocation, while careless pooling creates silent corruption by handing the same pages to two in-flight tensors.

Part 3 - Quantitative Prediction Before Execution

Choose size classes from the activation histogram. Predict pool capacity as the sum of class capacity times slot count, internal waste as capacity minus requested bytes, and required concurrency from peak in-flight offloads. A larger pool should reduce allocation misses until transfer concurrency is covered; beyond that point it consumes RAM without increasing throughput.

Part 4 - Experimental Design and Instrumentation

Compare fresh pinned allocation, exact-size pooling, and rounded size classes. Record pinned allocation count, pool hit rate, bytes reserved, internal waste, event-wait time, transfer

throughput, host RSS, peak VRAM, and gradients. Stress variable shapes and deliberately exhaust the budget to verify a safe fallback.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass
import torch

@dataclass
class PinnedLease:
    buffer: object
    capacity: int
    dtype: object
    ready_event: object
    in_use: bool

class PinnedActivationPool:
    def __init__(self, budget_bytes):
        self.budget_bytes = budget_bytes
        self.reserved_bytes = 0
        self.leases = []
        self.hits = 0
        self.misses = 0

    def _completed(self, lease):
        if lease.ready_event is None:
            return True
        return lease.ready_event.query()

    def acquire(self, numel, dtype):
        requested = numel * torch.empty((), dtype=dtype).element_size()
        for lease in self.leases:
            compatible = lease.dtype == dtype and lease.capacity >= numel
            if compatible and lease.in_use and self._completed(lease):
                lease.in_use = False
            if compatible and not lease.in_use:
                lease.in_use = True
                lease.ready_event = None
                self.hits += 1
                return lease
        if self.reserved_bytes + requested > self.budget_bytes:
            raise RuntimeError("pinned activation pool budget exceeded")
        buffer = torch.empty(numel, dtype=dtype, pin_memory=True)
        lease = PinnedLease(buffer, numel, dtype, None, True)
        self.leases.append(lease)
        self.reserved_bytes += requested
        self.misses += 1
        return lease
```

```
    def release_after(self, lease, event):
        lease.ready_event = event

def E13(budget_bytes):
    return PinnedActivationPool(budget_bytes)
```

Part 6 - Measurement, Validation, and Failure Analysis

The pool stores flat capacity; the manager must create a correctly shaped view without exceeding it. This minimal version scans linearly and marks a lease reusable only after its completion event queries true. Production code needs thread safety, per-device ownership, explicit shutdown, fragmentation policy, and either backpressure or pageable fallback when the budget is exhausted.

Part 7 - Engineering Interpretation and Philosophy

Pinned memory is not "free speed." It converts host flexibility into DMA predictability. A bounded pool makes that exchange explicit and turns buffer lifetime from an accidental Python property into a controlled systems contract.

19.20.15 Exercise 14

Exercise Title: Prefetch Offloaded Activations Back to CUDA Before Their Backward Operation Requires Them

Part 1 - Exercise Contract and Why It Matters

Use the measured backward consumption order to start H2D reload before an activation's unpack becomes blocking. The prefetcher must preserve exact values, honor the VRAM budget, coordinate streams with events, avoid duplicate reloads, and measure how much H2D latency becomes invisible.

Part 2 - System and Memory Mental Model

Offloading creates capacity during forward but introduces a future read miss. Prefetch moves that miss earlier. Too late exposes transfer latency; too early reoccupies VRAM and may destroy the original memory saving. The correct window lies between D2H completion and the backward consumer, subject to free VRAM and copy-engine contention.

Part 3 - Quantitative Prediction Before Execution

For activation i, calculate latest safe start as $t_{need,i} - t_{H2D,i}$. Its predicted visible stall is $\max(0, t_{H2D,i} - w_i)$ for available overlap window w_i. Sum prefetched bytes that are simultaneously resident to predict the new peak. If H2D shares bandwidth with D2H, include measured contention rather than adding isolated bandwidths.

Part 4 - Experimental Design and Instrumentation

Start with a known reverse-order sequential model, then test branched graphs. Compare demand reload, one-activation lookahead, and byte-budgeted lookahead. Record submission time, transfer completion, consumer wait, reload bytes, duplicate requests, prefetched residency, peak VRAM, and total backward time.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass
import torch

@dataclass
class PrefetchedActivation:
    cpu_tensor: object
    device: object
    d2h_done: object
    cuda_tensor: object
    h2d_done: object

class ActivationPrefetcher:
    def __init__(self, device="cuda"):
        self.device = torch.device(device)
        self.reload_stream = torch.cuda.Stream(device=self.device)

    def submit(self, item):
        if item.cuda_tensor is not None:
            return
        with torch.cuda.stream(self.reload_stream):
            if item.d2h_done is not None:
                self.reload_stream.wait_event(item.d2h_done)
            restored = item.cpu_tensor.to(
                item.device, non_blocking=True)
            done = torch.cuda.Event()
            done.record(self.reload_stream)
        item.cuda_tensor = restored
        item.h2d_done = done

    def acquire(self, item):
        if item.cuda_tensor is None:
            self.submit(item)
        current = torch.cuda.current_stream(item.device)
        current.wait_event(item.h2d_done)
        item.cuda_tensor.record_stream(current)
        return item.cuda_tensor

def E14(device="cuda"):
    return ActivationPrefetcher(device)
```

Part 6 - Measurement, Validation, and Failure Analysis

The scheduler, not autograd alone, must decide when to call `submit`; an unpack hook fires at demand time and is too late to create lookahead. Dynamic graphs can invalidate a previously learned order. Hold the pinned source until H2D completion and hold the CUDA destination through consumer use. Cancel or evict speculative reloads when the graph path is not taken.

Part 7 - Engineering Interpretation and Philosophy

Prefetch does not reduce work; it changes when the work is paid. Its success is the conversion of unavoidable transfer latency into overlapped background work without silently repurchasing all the VRAM that offloading was meant to save.

19.20.16 Exercise 15

Exercise Title: Overlap Activation Reload with Backward Computation Using Dedicated CUDA Streams and Dependency Events

Part 1 - Exercise Contract and Why It Matters

Construct an explicit backward pipeline in which the compute stream processes the current block while a reload stream restores the next block's activation. Use events for true producer-consumer dependencies, avoid device-wide barriers, and prove overlap from a GPU timeline rather than from optimistic wall time.

Part 2 - System and Memory Mental Model

CUDA streams supply potential concurrency, not a promise of it. The copy engine, memory fabric, and compute kernels can contend even when operations occupy different streams. D2H completion protects the host source; H2D completion protects the CUDA consumer; `record_stream` protects allocator lifetime. Only the block needing an activation should wait for its reload event.

Part 3 - Quantitative Prediction Before Execution

For consecutive backward blocks, the ideal stage time is approximately $\max(t_{compute}, t_{H2D})$ instead of $t_{compute} + t_{H2D}$. Predict speedup from measured isolated durations, then reduce it for launch gaps, bandwidth contention, insufficient prefetch distance, and the unhidden first/last pipeline stages. Predict the extra VRAM of the current and prefetched activations.

Part 4 - Experimental Design and Instrumentation

Benchmark demand reload and event-directed overlap against a globally synchronized baseline. Record CUDA events for each reload and compute interval, export a profiler trace, and verify temporal intersection. Sweep tensor size, compute intensity, and lookahead. Compare outputs and gradients bitwise where possible and with dtype-appropriate tolerances otherwise.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass
import torch

@dataclass
class ReloadTicket:
    value: object
    ready: object
    source: object

class BackwardReloadCoordinator:
    def __init__(self, device="cuda"):
        self.device = torch.device(device)
        self.reload_stream = torch.cuda.Stream(device=self.device)
```

```
    def reload(self, cpu_tensor, d2h_done=None):
        with torch.cuda.stream(self.reload_stream):
            if d2h_done is not None:
                self.reload_stream.wait_event(d2h_done)
            value = cpu_tensor.to(self.device, non_blocking=True)
            ready = torch.cuda.Event()
            ready.record(self.reload_stream)
        return ReloadTicket(value, ready, cpu_tensor)

    def consume(self, ticket):
        compute_stream = torch.cuda.current_stream(self.device)
        compute_stream.wait_event(ticket.ready)
        ticket.value.record_stream(compute_stream)
        return ticket.value

    def launch_next_while_computing(self, next_cpu, current_fn,
                                    current_value, d2h_done=None):
        ticket = self.reload(next_cpu, d2h_done)
        current_result = current_fn(current_value)
        return current_result, ticket

def E15(device="cuda"):
    return BackwardReloadCoordinator(device)
```

Part 6 - Measurement, Validation, and Failure Analysis

This coordinator demonstrates the transport dependency; integrating it with an arbitrary autograd graph requires a known execution schedule or engine-level callbacks. A short kernel may finish before H2D reaches steady bandwidth, and large kernels may saturate memory bandwidth so the nominal overlap slows both. Never infer overlap merely because Python returned before CUDA completed.

Part 7 - Engineering Interpretation and Philosophy

Concurrency is useful only when it shortens the critical path. Events express causality precisely; timelines reveal whether hardware found spare resources to execute independent work at the same time.

19.20.17 Exercise 16

Exercise Title: Compare Activation Checkpointing and Activation Offloading Under Equal VRAM Budgets and Identify Whether Compute or PCIe Bandwidth Is the Better Resource to Spend

Part 1 - Exercise Contract and Why It Matters

Compare two ways of buying activation capacity under the same enforceable VRAM budget. Checkpointing discards selected intermediates and recomputes them; offloading preserves them in RAM and transfers them twice. The experiment must match model, batch, numerical policy, and achieved peak memory before comparing step latency or throughput.

Part 2 - System and Memory Mental Model

Checkpointing spends GPU arithmetic and may repeat memory reads. Offloading spends PCIe/NVLink bandwidth, pinned RAM, copy engines, and scheduling state. Their savings overlap because both target saved forward tensors. A compute-rich but link-poor system often favors recomputation; an expensive block with spare transfer bandwidth can favor offload. Neither method removes parameters, gradients, or optimizer state.

Part 3 - Quantitative Prediction Before Execution

Estimate checkpoint cost from measured replay time t_r and offload cost from $b/B_{D2H} + b/B_{H2D}$ minus achievable overlap. Compare policies at the same peak, not the same number of blocks. Report arithmetic intensity, transferred bytes, replay FLOPs, exposed stall, host bytes, and energy when measurable.

Part 4 - Experimental Design and Instrumentation

Find configurations that meet a target peak: no optimization, full and selective checkpointing, full and selective offload. Warm each configuration, run repeated synchronized steps, and report median plus tail latency. Validate loss and parameter gradients from identical weights and random states. Repeat with compute-heavy and activation-heavy blocks.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class PolicyResult:
    name: str
    peak_vram: int
    host_bytes: int
    step_ms: float
    replay_ms: float
    transfer_ms: float
    transfer_bytes: int

def within_budget(results, budget_bytes):
    accepted = []
    for result in results:
        if result.peak_vram <= budget_bytes:
            accepted.append(result)
    return accepted

def choose_resource(results, budget_bytes):
    accepted = within_budget(results, budget_bytes)
    if len(accepted) == 0:
        raise RuntimeError("no policy satisfies the VRAM budget")
    best = accepted[0]
    for result in accepted[1:]:
        if result.step_ms < best.step_ms:
            best = result
```

```
    if best.replay_ms > best.transfer_ms:
        resource = "transfer bandwidth"
    else:
        resource = "recomputation"
    return best, resource

def E16(results, budget_bytes):
    return choose_resource(results, budget_bytes)
```

Part 6 - Measurement, Validation, and Failure Analysis

The returned label describes the winning measured configuration, not a universal hardware truth. Checkpointing with stochastic layers must preserve RNG behavior; offloading must preserve transfer ordering and dtype. Account for profiler overhead and CPU RAM. If two policies miss the target peak, their speed ranking is irrelevant to the stated capacity problem.

Part 7 - Engineering Interpretation and Philosophy

Memory has a shadow price. Checkpointing pays that price in computation; offloading pays it in movement. Sound engineering measures which resource has slack on the actual system instead of declaring either mechanism inherently superior.

19.20.18 Exercise 17

Exercise Title: Build a Hybrid Policy That Recomputes Cheap Activations, Offloads Expensive Activations, and Keeps High-Reuse Activations on GPU

Part 1 - Exercise Contract and Why It Matters

Combine three actions under one VRAM and RAM budget: keep values whose repeated reuse makes movement undesirable, checkpoint regions whose forward replay is cheap, and offload values whose recomputation is expensive but transfer is affordable. Every decision must map to an executable mechanism and retain a reason suitable for debugging.

Part 2 - System and Memory Mental Model

Saved-tensor hooks can keep or offload a tensor, but they cannot reconstruct an arbitrary discarded intermediate from nothing. Recompute therefore belongs at a checkpointable function or block boundary. Hybrid planning has two levels: choose checkpoint regions, then classify remaining saved tensors for keep or offload. Shared inputs, residuals, stochastic operations, and multiple consumers couple decisions that appear independent in a flat tensor list.

Part 3 - Quantitative Prediction Before Execution

For candidate i, compare keep cost from byte-lifetime pressure, offload cost from D2H/H2D plus exposed stall, and replay cost from measured block execution. Add host-budget and VRAM-budget constraints. Recalculate after every selection because checkpointing removes a set of saves and changes the remaining peak.

Part 4 - Experimental Design and Instrumentation

Profile regions and saved tensors separately. Evaluate all-keep, checkpoint- only, offload-only, and hybrid policies at equal peak. Record policy decisions, actual bytes removed from

the peak, replay time, transfer traffic, RAM, step time, and gradient error. Test stable and dynamic control flow.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass

@dataclass
class HybridCandidate:
    name: str
    size_bytes: int
    reuse_count: int
    keep_pressure: float
    offload_ms: float
    recompute_ms: float
    checkpointable: bool

@dataclass
class HybridDecision:
    name: str
    action: str
    estimated_ms: float
    reason: str

def decide_hybrid(item, ram_available):
    if item.reuse_count > 1 and item.keep_pressure < item.size_bytes * 10.0:
        return HybridDecision(
            item.name, "keep", 0.0, "high reuse favors residency")
    if item.checkpointable and item.recompute_ms <= item.offload_ms:
        return HybridDecision(
            item.name, "recompute", item.recompute_ms,
            "block replay is cheaper than transfer")
    if item.size_bytes <= ram_available:
        return HybridDecision(
            item.name, "offload", item.offload_ms,
            "transfer is cheaper than replay")
    return HybridDecision(
        item.name, "keep", 0.0, "host budget prevents offload")

def E17(items, ram_budget):
    decisions = []
    remaining = ram_budget
    for item in items:
        decision = decide_hybrid(item, remaining)
        decisions.append(decision)
        if decision.action == "offload":
            remaining -= item.size_bytes
    return decisions
```

Part 6 - Measurement, Validation, and Failure Analysis

This greedy rule is a transparent starting point, not a global optimizer. Its scores require calibration and its ordering affects the answer. Implement "recompute" with PyTorch activation checkpointing around the named region and "offload" with safe saved-tensor hooks; never pretend an unpack hook alone can regenerate a discarded value. Validate peak constraints after executing the combined plan.

Part 7 - Engineering Interpretation and Philosophy

Hybrid policy recognizes that activations have different economic character. Uniform treatment is simple but wasteful; disciplined heterogeneity spends compute, bandwidth, and capacity only where each resource produces the greatest reduction in the critical memory peak.

19.20.19 Exercise 18

Exercise Title: Stress-Test Activation Offloading with Long Transformer Sequences and Diagnose Host RAM Pressure, Transfer Bottlenecks, and Backward Stalls

Part 1 - Exercise Contract and Why It Matters

Increase sequence length until activation offloading meets its practical limits. The stress test must distinguish CUDA OOM avoidance from a healthy system: it must detect host growth, pinned-memory exhaustion, transfer-queue backlog, backward starvation, allocator fragmentation, and numerical failure before the operating system or CUDA runtime terminates the job.

Part 2 - System and Memory Mental Model

Transformer activation bytes grow with batch, sequence, width, layers, and the attention algorithm; conventional attention can also create quadratic score storage. Offloading moves part of this pressure to RAM and the interconnect. As context grows, a pipeline can remain below the VRAM limit while becoming slower every step because queued H2D reloads lie on the backward critical path.

Part 3 - Quantitative Prediction Before Execution

Before each run, predict saved bytes, host peak, traffic per step, and minimum copy time from measured bandwidth. Estimate sustainable traffic by comparing bytes per step times steps per second with achieved link bandwidth. Define stop margins below both VRAM and RAM capacity and predict the sequence length at which either margin is crossed.

Part 4 - Experimental Design and Instrumentation

Sweep context geometrically, then refine near failure. Warm every shape and record median and p95 step time, peak allocated/reserved VRAM, RAM RSS, pinned pool bytes, D2H/H2D bytes, queue depth, consumer wait, recomputation, and loss. Keep an unoptimized baseline at smaller lengths. Persist the last safe sample and stop proactively instead of relying on a destructive host OOM.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass
```

```
@dataclass
class StressSample:
    sequence_length: int
    peak_vram: int
    reserved_vram: int
    host_rss: int
    pinned_bytes: int
    transfer_bytes: int
    queue_depth: int
    backward_wait_ms: float
    step_ms: float
    finite: bool

class ActivationStressGuard:
    def __init__(self, vram_limit, ram_limit, queue_limit):
        self.vram_limit = vram_limit
        self.ram_limit = ram_limit
        self.queue_limit = queue_limit
        self.samples = []

    def add(self, sample):
        self.samples.append(sample)
        reasons = []
        if sample.peak_vram >= self.vram_limit:
            reasons.append("VRAM margin exhausted")
        if sample.host_rss + sample.pinned_bytes >= self.ram_limit:
            reasons.append("host RAM margin exhausted")
        if sample.queue_depth >= self.queue_limit:
            reasons.append("transfer queue is unstable")
        if not sample.finite:
            reasons.append("non-finite numerical result")
        return reasons

    def last_safe(self):
        safe = None
        for sample in self.samples:
            reasons = self.add_check(sample)
            if len(reasons) == 0:
                safe = sample
        return safe

    def add_check(self, sample):
        reasons = []
        if sample.peak_vram >= self.vram_limit:
            reasons.append("VRAM")
        if sample.host_rss + sample.pinned_bytes >= self.ram_limit:
            reasons.append("RAM")
        if sample.queue_depth >= self.queue_limit:
```

```
            reasons.append("queue")
        if not sample.finite:
            reasons.append("numeric")
        return reasons

def E18(vram_limit, ram_limit, queue_limit):
    return ActivationStressGuard(vram_limit, ram_limit, queue_limit)
```

Part 6 - Measurement, Validation, and Failure Analysis

RSS and pinned allocations can overlap, so replace the illustrative sum with non-overlapping counters on the target platform. Queue depth that rises across steady-state steps signals arrival rate above service rate. Distinguish a temporary allocator reserve from live tensors, and distinguish transfer stall from slow compute with CUDA events and timeline evidence.

Part 7 - Engineering Interpretation and Philosophy

Avoiding CUDA OOM is necessary but insufficient. A successful memory system keeps every tier bounded and keeps movement off the critical path. Stress tests reveal where a local optimization merely relocates failure to RAM, bandwidth, latency, or correctness.

19.20.20 Exercise 19

Exercise Title: Build and Benchmark a Memory-Aware PyTorch Activation Manager That Dynamically Chooses Keep, Offload, or Recompute for Saved Tensors Under a Fixed VRAM Budget

Part 1 - Exercise Contract and Why It Matters

Integrate profiling, policy, safe transport, bounded host buffers, prefetch, checkpoint regions, and verification into one activation manager. It must meet a fixed VRAM budget without unbounded RAM growth, explain each decision, preserve gradients, and report capacity gained together with latency paid.

Part 2 - System and Memory Mental Model

The manager has a control plane and a data plane. The control plane predicts liveness, chooses keep/offload/recompute, reserves budgets, and schedules prefetch. The data plane owns CUDA and pinned storage, streams, events, and state transitions. A packed activation moves through states such as GPU-live, D2H-in-flight, CPU-resident, H2D-in-flight, and consumed. Recompute remains a block-level checkpoint decision rather than a magical saved-tensor state.

Part 3 - Quantitative Prediction Before Execution

For every candidate, estimate peak reduction, host bytes, replay cost, transfer cost, and exposed stall. Solve for a working set below the VRAM high watermark and keep a low watermark to prevent oscillation. Reserve capacity before launching a move, update estimates from observed bandwidth and wait time, and reject a policy whose predicted RAM or transfer queue is unsustainable.

Part 4 - Experimental Design and Instrumentation

Benchmark keep-only, threshold offload, checkpoint-only, static hybrid, and the adaptive manager from identical initial states. Report peak allocated and reserved VRAM, host and

pinned peaks, traffic, pool hit rate, replay, transfer wait, step latency distribution, throughput, and gradient error. Test fixed and variable sequence lengths, forced pressure, dynamic paths, and recovery from budget denial.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass
import torch

@dataclass
class ManagedActivation:
    action: str
    original_device: object
    size_bytes: int
    gpu_tensor: object
    cpu_tensor: object
    d2h_done: object
    state: str

class ActivationManager:
    def __init__(self, vram_budget, ram_budget, threshold):
        self.vram_budget = vram_budget
        self.ram_budget = ram_budget
        self.threshold = threshold
        self.ram_reserved = 0
        self.offload_stream = torch.cuda.Stream()
        self.decisions = []

    def decide(self, tensor):
        size_bytes = tensor.numel() * tensor.element_size()
        can_offload = self.ram_reserved + size_bytes <= self.ram_budget
        if tensor.device.type != "cuda":
            return "keep"
        if size_bytes >= self.threshold and can_offload:
            return "offload"
        return "keep"

    def pack(self, tensor):
        size_bytes = tensor.numel() * tensor.element_size()
        action = self.decide(tensor)
        self.decisions.append((size_bytes, action))
        if action == "keep":
            return ManagedActivation(
                action, tensor.device, size_bytes, tensor,
                None, None, "gpu-live")
        cpu_tensor = torch.empty_like(
            tensor, device="cpu", pin_memory=True)
        producer_ready = torch.cuda.Event()
        producer_ready.record(torch.cuda.current_stream(tensor.device))
```

```
        done = torch.cuda.Event()
        with torch.cuda.stream(self.offload_stream):
            self.offload_stream.wait_event(producer_ready)
            cpu_tensor.copy_(tensor, non_blocking=True)
            tensor.record_stream(self.offload_stream)
            done.record(self.offload_stream)
        self.ram_reserved += size_bytes
        return ManagedActivation(
            action, tensor.device, size_bytes, None,
            cpu_tensor, done, "d2h-in-flight")

    def unpack(self, item):
        if item.action == "keep":
            return item.gpu_tensor
        current = torch.cuda.current_stream(item.original_device)
        current.wait_event(item.d2h_done)
        restored = item.cpu_tensor.to(
            item.original_device, non_blocking=True)
        restored.record_stream(current)
        item.state = "consumed"
        self.ram_reserved -= item.size_bytes
        return restored

def E19(vram_budget, ram_budget, threshold):
    return ActivationManager(vram_budget, ram_budget, threshold)
```

Part 6 - Measurement, Validation, and Failure Analysis

This compact implementation demonstrates safe keep/offload transitions. Its VRAM budget is not yet enforced by live-liveness accounting, and its fresh pinned allocations should be replaced by E13's pool. Real integration adds multiple-device streams, alias handling, repeated unpack, exception-safe reservation release, learned prefetch order, and checkpoint-region selection. Run exact output and gradient comparisons before trusting any speed result.

Part 7 - Engineering Interpretation and Philosophy

An activation manager is a resource scheduler embedded in autograd. Its purpose is not to maximize the amount moved or recomputed; it is to preserve semantics while keeping scarce memory bounded and minimizing the new critical-path work. The final artifact is therefore the evidence trail—predictions, decisions, events, budgets, measurements, and failures—as much as the code itself.

19.21 DMM19

Chapter Title: Mixed Precision Memory Management
Specialization: Dynamic Memory Management and CUDA Systems

19.21.1 Exercise 00

Exercise Title: Build a Precise Mental Model of How float32, float16, and bfloat16 Change Tensor Memory, Numerical Range, and GPU Execution

Part 1 - Exercise Contract and Why It Matters

Build a resource model that separates storage width, numerical range, precision, accumulation dtype, kernel choice, and conversion cost. Explain why two 16-bit formats occupy the same tensor bytes but can behave very differently, and why smaller tensors do not guarantee faster execution on every GPU operation.

Part 2 - System and Memory Mental Model

A tensor with N elements consumes N times the dtype element size before allocator rounding and workspace effects. float16 and bfloat16 use two bytes per element, but float16 spends more exponent bits on significand precision while bfloat16 preserves a float32-like exponent range. Autocast selects operation dtypes; it does not simply convert the whole program.

Part 3 - Quantitative Prediction Before Execution

For each candidate tensor, predict logical bytes, expected allocated-byte change, representable-range risk, rounding risk, Tensor Core eligibility, and whether accumulation remains in float32. Distinguish memory saved in parameters, gradients, optimizer state, activations, temporary buffers, and saved tensors rather than applying one global one-half multiplier.

Part 4 - Experimental Design and Instrumentation

Create identical CUDA workloads in float32, float16, and bfloat16. Record element size, logical tensor bytes, peak allocated and reserved VRAM, kernel time, output error, finite-value rate, and operation-level dtypes. Use synchronized warm-up and measured iterations, fixed seeds, identical shapes, and documented hardware capability.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass
import torch

@dataclass
class PrecisionProfile:
    name: str
    element_size: int
    logical_bytes: int
    maximum: float
    minimum_normal: float
    epsilon: float

def describe_dtype(dtype, number_of_elements):
    information = torch.finfo(dtype)
    probe = torch.empty((), dtype=dtype)
    return PrecisionProfile(
        str(dtype),
        probe.element_size(),
```

```
        number_of_elements * probe.element_size(),
        information.max,
        information.tiny,
        information.eps)

def E00(number_of_elements):
    dtypes = [torch.float32, torch.float16, torch.bfloat16]
    profiles = []
    for dtype in dtypes:
        profile = describe_dtype(dtype, number_of_elements)
        profiles.append(profile)
    return profiles
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not infer physical VRAM from logical bytes alone. CUDA allocator bins, alignment, cached blocks, workspaces, master weights, gradients, and optimizer state can dominate the result. Verify that every tested operation supports the requested dtype and report fallbacks, promotions, non-finite values, and unsupported kernels instead of silently excluding them.

Part 7 - Engineering Interpretation and Philosophy

Mixed precision is a placement policy over numerical formats. It spends representational accuracy and implementation complexity to reduce memory traffic and unlock specialized hardware, while preserving float32 where range or accumulation demands it. The correct unit of reasoning is the operation and tensor role, not the marketing label attached to a training run.

19.21.2 Exercise 01

Exercise Title: Calculate the Exact VRAM Required by Identical Tensor Shapes Stored in float64, float32, bfloat16, and float16

Part 1 - Exercise Contract and Why It Matters

Calculate exact logical storage for the same shapes in four floating-point dtypes, including scalar, vector, matrix, batched, and Transformer-shaped tensors. Then separate payload bytes from allocator-observed VRAM so that a correct arithmetic answer is not confused with a physical allocation measurement.

Part 2 - System and Memory Mental Model

Logical storage equals the product of all dimensions multiplied by element size: eight bytes for float64, four for float32, and two for bfloat16 or float16. Views can share storage, noncontiguous tensors can expose a different logical region, and CUDA allocations are rounded and cached, so allocated and reserved bytes answer different questions.

Part 3 - Quantitative Prediction Before Execution

Before allocating, compute elements and bytes for every shape and dtype. Predict exact ratios relative to float32, convert bytes to MiB and GiB using binary units, and estimate the total for multiple simultaneous copies. State whether gradients, outputs, and temporary tensors are included in each total.

Part 4 - Experimental Design and Instrumentation

Allocate one tensor at a time in a fresh process or after controlled cleanup. Compare formula bytes with tensor element_size, untyped storage bytes, memory_allocated change, and memory_reserved change. Repeat for contiguous tensors, views, clones, and deliberately noncontiguous layouts without claiming that cache cleanup represents normal training behavior.

Part 5 - PyTorch/CUDA Implementation

```
import math
import torch

def logical_tensor_bytes(shape, dtype):
    number_of_elements = math.prod(shape)
    element_size = torch.empty((), dtype=dtype).element_size()
    return number_of_elements * element_size

def E01(shape):
    dtypes = [
        torch.float64,
        torch.float32,
        torch.bfloat16,
        torch.float16,
    ]
    result = {}
    for dtype in dtypes:
        result[str(dtype)] = logical_tensor_bytes(shape, dtype)
    return result
```

Part 6 - Measurement, Validation, and Failure Analysis

Check integer overflow in manual products, zero-sized dimensions, aliasing, and storage offsets. Report bytes, MiB, and GiB with the conversion definition. A mismatch between formula bytes and allocator deltas is not automatically an error; attribute it to rounding, caching, pre-existing blocks, or additional runtime allocations using controlled evidence.

Part 7 - Engineering Interpretation and Philosophy

Exact byte arithmetic is the lower bound of memory engineering. It is simple enough to calculate before execution, but valuable only when its scope is explicit. The allocator measurement adds system truth; neither number should be substituted for the other.

19.21.3 Exercise 02

Exercise Title: Measure Real CUDA Memory Usage for the Same PyTorch Workload Across float32, float16, and bfloat16

Part 1 - Exercise Contract and Why It Matters

Measure the complete CUDA memory behavior of one workload across three dtypes. Capture current, peak, allocated, and reserved VRAM as well as execution time and numerical validity. Keep shapes and algorithmic work identical so dtype is the controlled variable.

Part 2 - System and Memory Mental Model

Changing dtype reduces tensor payload, but peak VRAM also contains library workspaces, allocator fragmentation, cached blocks, outputs retained by Python, and autograd state. Some kernels accumulate in float32 or create promoted temporaries. Reserved memory can remain high even after live tensors are released.

Part 3 - Quantitative Prediction Before Execution

Create a component estimate for input, parameters, activations, gradients, and expected workspaces. Predict which components halve and which remain unchanged. State expected peak ordering and an error tolerance for each reduced dtype before running the workload.

Part 4 - Experimental Design and Instrumentation

Run each dtype in an isolated process with identical warm-up, seed, model, batch, and synchronization. Reset peak statistics immediately before the measured region. Record allocated and reserved peaks, median step time, output error against float32, finite gradients, and any unsupported-operation failure.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

def measure_workload(workload, dtype, device="cuda"):
    torch.cuda.synchronize(device)
    torch.cuda.reset_peak_memory_stats(device)
    start = time.perf_counter()
    output = workload(dtype=dtype, device=device)
    torch.cuda.synchronize(device)
    elapsed = time.perf_counter() - start
    return {
        "output": output,
        "peak_allocated": torch.cuda.max_memory_allocated(device),
        "peak_reserved": torch.cuda.max_memory_reserved(device),
        "elapsed_seconds": elapsed,
    }

def E02(workload):
    results = {}
    for dtype in [torch.float32, torch.float16, torch.bfloat16]:
        results[str(dtype)] = measure_workload(workload, dtype)
    return results
```

Part 6 - Measurement, Validation, and Failure Analysis

Synchronize around timing and do not compare a first-run compilation or autotuning cost with warmed runs. Confirm identical model topology and tensor shapes. Preserve failures as results, including OOM, unsupported dtype, non-finite output, or accuracy outside tolerance. Explain allocated versus reserved memory explicitly.

Part 7 - Engineering Interpretation and Philosophy

Real memory reduction is an empirical property of the entire workload, not a direct synonym for smaller element size. A defensible result connects a byte model to allocator telemetry, kernel behavior, and numerical evidence under controlled execution.

19.21.4 Exercise 03

Exercise Title: Compare Full-Precision and Half-Precision Model Parameters and Verify the Expected Reduction in Parameter Memory

Part 1 - Exercise Contract and Why It Matters

Compare model parameter storage after conversion to float32, float16, and bfloat16. Count unique storages, preserve tied parameters, and distinguish parameter payload from gradients, buffers, master copies, and optimizer state.

Part 2 - System and Memory Mental Model

Casting a module usually creates parameters in the requested dtype, but tied weights and views can make a naive sum double-count physical storage. Training systems may retain float32 master weights even when the model-facing parameter is half precision. Integer and boolean buffers do not follow floating-point conversion rules.

Part 3 - Quantitative Prediction Before Execution

Predict parameter bytes from unique element counts and dtype width. Extend the estimate to gradients and a selected optimizer, declaring whether momentum or variance tensors remain float32. State the expected ratio for inference-only parameters and why full training state may not achieve it.

Part 4 - Experimental Design and Instrumentation

Clone identical model state into each dtype, enumerate named parameters and buffers, canonicalize storage identity, and record logical and unique bytes. Run forward and backward to observe gradient dtypes, then initialize the optimizer state with one update and recount every state tensor.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def unique_tensor_bytes(tensors):
    seen = set()
    total = 0
    for tensor in tensors:
        storage = tensor.untyped_storage()
        key = (tensor.device, storage.data_ptr(), storage.nbytes())
        if key not in seen:
            seen.add(key)
            total += storage.nbytes()
    return total
```

```
def parameter_report(model):
    parameters = list(model.parameters())
    logical_bytes = 0
    dtype_names = set()
    for parameter in parameters:
        parameter_bytes = parameter.numel() * parameter.element_size()
        logical_bytes += parameter_bytes
        dtype_names.add(str(parameter.dtype))
    return {
        "logical_bytes": logical_bytes,
        "unique_storage_bytes": unique_tensor_bytes(parameters),
        "dtypes": dtype_names,
    }

def E03(model):
    return parameter_report(model)
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify value conversion against a float32 reference and check shared-weight identity after cloning or casting. Do not include cached allocator blocks in the parameter byte count. Report mixed buffer dtypes, master copies, optimizer tensors, and any parameter that was intentionally kept in float32.

Part 7 - Engineering Interpretation and Philosophy

Parameter memory is the cleanest mixed-precision component, yet even here ownership and duplication matter. The engineering question is not only how wide a parameter is, but how many authoritative and auxiliary copies the training system must retain.

19.21.5 Exercise 04

Exercise Title: Compare Activation Memory Across float32, float16, and bfloat16 During the Same Forward Pass

Part 1 - Exercise Contract and Why It Matters

Measure activation payload and liveness during an identical forward pass in three precision modes. Record both module outputs and tensors saved for backward, then relate their unique bytes to the observed forward peak.

Part 2 - System and Memory Mental Model

Activation memory depends on output dtype, saved-tensor policy, aliasing, operation-specific autocast rules, and lifetime. A float16 input can produce float32 statistics or reductions, while views may share a larger backing storage. The forward peak is controlled by overlapping live tensors, not the sum of every output ever observed.

Part 3 - Quantitative Prediction Before Execution

Predict activation bytes layer by layer from shapes and expected dtypes. Mark operations likely to promote or retain float32. Estimate the peak live set and the saved-for-backward set separately, then specify expected differences between explicit model casting and autocast.

Part 4 - Experimental Design and Instrumentation

Use forward hooks and saved_tensors_hooks around one deterministic model. Record producer name, shape, dtype, logical bytes, storage identity, save order, and peak CUDA memory. Compare pure float32, float16 autocast, and bfloat16 autocast with identical inputs and graph structure.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass
import torch

@dataclass
class ActivationRecord:
    producer: str
    shape: tuple
    dtype: object
    bytes: int

def collect_forward_activations(model, inputs):
    records = []
    handles = []
    for name, module in model.named_modules():
        if name == "":
            continue
        def hook(current_module, current_inputs, output, name=name):
            if torch.is_tensor(output):
                records.append(ActivationRecord(
                    name, tuple(output.shape), output.dtype,
                    output.numel() * output.element_size()))
        handles.append(module.register_forward_hook(hook))
    model(*inputs)
    for handle in handles:
        handle.remove()
    return records

def E04(model, inputs):
    return collect_forward_activations(model, inputs)
```

Part 6 - Measurement, Validation, and Failure Analysis

Hooks can miss tensors created inside functional operations and can change retention if outputs are stored carelessly. Detach metadata rather than retaining full activations. Deduplicate storage, include tensors saved by autograd, and verify equal output shapes and control flow across modes before attributing a peak difference to precision.

Part 7 - Engineering Interpretation and Philosophy

Activation savings emerge from dtype decisions distributed across the graph. A useful analysis reconstructs liveness and operation policy instead of assuming every intermediate follows the input dtype or remains resident for the same duration.

19.21.6 Exercise 05

Exercise Title: Run a Complete Training Step in Pure float32 and Establish Baseline Peak VRAM, Throughput, and Numerical Behavior

Part 1 - Exercise Contract and Why It Matters

Establish a rigorous float32 baseline before introducing mixed precision. Measure forward, loss, backward, optimizer update, peak memory, latency distribution, throughput, loss trajectory, gradient norms, and finite-value status from a reproducible initial state.

Part 2 - System and Memory Mental Model

The baseline defines the semantic and performance reference. CUDA execution is asynchronous, allocator state persists, and the first iterations may include library initialization or algorithm selection. Model mode, stochastic layers, optimizer state initialization, and gradient clearing strategy all affect both memory and numerical traces.

Part 3 - Quantitative Prediction Before Execution

Estimate parameter, gradient, optimizer, activation, and workspace memory. Predict the first-step peak and steady-state peak separately. Define acceptable loss, output, gradient, and update differences that later precision modes must satisfy.

Part 4 - Experimental Design and Instrumentation

Use fixed seeds and cloned model, optimizer, and batch state. Warm up without recording, then run multiple synchronized float32 steps. Report median and tail latency, samples or tokens per second, peak allocated and reserved VRAM, per-parameter gradient norms, loss, and non-finite counts.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

def float32_training_step(model, optimizer, inputs, targets, loss_function):
    optimizer.zero_grad(set_to_none=True)
    outputs = model(inputs.float())
    loss = loss_function(outputs, targets)
    loss.backward()
    optimizer.step()
    return outputs.detach(), loss.detach()

def E05(model, optimizer, inputs, targets, loss_function):
    torch.cuda.reset_peak_memory_stats()
    torch.cuda.synchronize()
    start = time.perf_counter()
    outputs, loss = float32_training_step(
        model, optimizer, inputs, targets, loss_function)
    torch.cuda.synchronize()
    return {
        "outputs": outputs,
```

```
        "loss": loss,
        "seconds": time.perf_counter() - start,
        "peak_allocated": torch.cuda.max_memory_allocated(),
        "peak_reserved": torch.cuda.max_memory_reserved(),
    }
```

Part 6 - Measurement, Validation, and Failure Analysis

Confirm that all trainable parameters and expected activations use float32. Detect non-finite loss, outputs, gradients, and updated parameters. Store the initial state and inputs required for later replay. Do not compare a single cold float32 step with warmed mixed-precision steps.

Part 7 - Engineering Interpretation and Philosophy

Optimization claims need a stable control. The baseline is not administrative overhead; it is the evidence that later capacity or speed gains preserve the intended learning computation within declared tolerances.

19.21.7 Exercise 06

Exercise Title: Enable torch.autocast(device_type="cuda", dtype=torch.float16) and Identify Which PyTorch Operations Change Precision Automatically

Part 1 - Exercise Contract and Why It Matters

Enable CUDA float16 autocast around a controlled operation suite and identify the actual output dtype chosen for each operation. Separate eligible Tensor Core operations, float32-sensitive operations, and operations that preserve or promote input dtype.

Part 2 - System and Memory Mental Model

Autocast uses operation-specific dispatch rules rather than a universal cast. Inputs may be converted internally, outputs can use float16 or float32, and in-place operations or explicit dtype arguments may bypass autocast behavior. Parameters usually remain stored in float32 while eligible kernels consume casted views.

Part 3 - Quantitative Prediction Before Execution

Before execution, classify matrix multiplication, convolution, normalization, reduction, elementwise, softmax, and loss operations by expected autocast behavior. Predict output dtype and numerical risk, then mark any classification that depends on the installed PyTorch and CUDA versions.

Part 4 - Experimental Design and Instrumentation

Run the same operation suite outside and inside autocast with fixed inputs spanning normal and extreme values. Record input, parameter, output, and gradient dtypes plus profiler kernel names. Repeat with out-of-place and in-place variants where legal and document version-specific results.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def operation_dtype_report(x, weight):
    report = {}
    with torch.autocast(device_type="cuda", dtype=torch.float16):
        matrix = x @ weight
        report["matmul"] = matrix.dtype
        report["sum"] = matrix.sum().dtype
        report["softmax"] = torch.softmax(matrix, dim=-1).dtype
        report["relu"] = torch.relu(matrix).dtype
        report["norm"] = torch.linalg.vector_norm(matrix).dtype
    return report

def E06(x, weight):
    return operation_dtype_report(x, weight)
```

Part 6 - Measurement, Validation, and Failure Analysis

Treat observed dtype as the result for the tested environment, not a timeless rule. Check explicit dtype arguments, disabled subregions, nested contexts, CPU tensors, and unsupported hardware. Validate output against float32 and preserve operation failures or surprising promotions in the report.

Part 7 - Engineering Interpretation and Philosophy

Autocast is a curated numerical policy encoded in dispatch. Understanding its decisions turns mixed precision from a magic context manager into an inspectable contract that can be tested, overridden, and revised as software changes.

19.21.8 Exercise 07

Exercise Title: Inspect Intermediate Tensor Dtypes Inside an Autocast Region and Explain Why Not Every Operation Runs in float16

Part 1 - Exercise Contract and Why It Matters

Instrument a real model inside float16 autocast and build an operation-level dtype trace. Explain every float32 island, dtype-preserving path, and promoted output using numerical sensitivity, input policy, or explicit implementation behavior.

Part 2 - System and Memory Mental Model

An autocast region can contain mixed outputs because eligibility is per operation. Reductions, losses, normalization statistics, or custom functions may use float32 for stability, while binary operations can promote to the widest input. Stored parameters can remain float32 even when computation uses lower precision.

Part 3 - Quantitative Prediction Before Execution

Draw the expected dtype path through the selected model before running it. Mark boundaries where values enter and leave float16, estimate conversion bytes, and predict which saved tensors autograd will retain in each dtype.

Part 4 - Experimental Design and Instrumentation

Attach carefully scoped module hooks and saved-tensor hooks, and use the profiler for functional operations. Record operation name, input dtypes, output dtypes, saved-tensor dtypes, shapes, and bytes. Compare the trace against a no-autocast run without retaining tensor payloads in the logger.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class DtypeTracer:
    def __init__(self):
        self.records = []
        self.handles = []

    def attach(self, model):
        for name, module in model.named_modules():
            if name == "":
                continue
            def hook(current_module, inputs, output, name=name):
                input_dtypes = []
                for item in inputs:
                    if torch.is_tensor(item):
                        input_dtypes.append(item.dtype)
                output_dtype = output.dtype if torch.is_tensor(output) else None
                self.records.append((name, input_dtypes, output_dtype))
            self.handles.append(module.register_forward_hook(hook))

    def remove(self):
        for handle in self.handles:
            handle.remove()

def E07(model, inputs):
    tracer = DtypeTracer()
    tracer.attach(model)
    with torch.autocast(device_type="cuda", dtype=torch.float16):
        model(*inputs)
    tracer.remove()
    return tracer.records
```

Part 6 - Measurement, Validation, and Failure Analysis

A module hook does not expose every operator and composite modules may repeat information. Avoid storing outputs, which changes liveness and memory. Cross-check representative regions with profiler events or explicit probes, and verify the trace under the exact PyTorch version recorded in the experiment.

Part 7 - Engineering Interpretation and Philosophy

A mixed-precision graph is intentionally heterogeneous. Float32 islands are often safeguards rather than missed optimizations, and a dtype trace makes the boundary between deliberate stability and accidental promotion visible.

19.21.9 Exercise 08

Exercise Title: Compare float16 Autocast Against bfloat16 Autocast by Peak VRAM, Throughput, Output Error, and Numerical Stability

Part 1 - Exercise Contract and Why It Matters

Compare two autocast dtypes under identical training conditions. Measure capacity, speed, error, overflow, underflow, gradient stability, and convergence evidence rather than selecting a format from element size alone.

Part 2 - System and Memory Mental Model

Both formats occupy two bytes, but float16 provides finer significand precision and a narrower exponent range, while bfloat16 sacrifices precision for a float32-like range. Hardware support and kernel availability can make throughput differ even when memory payload is similar.

Part 3 - Quantitative Prediction Before Execution

Predict equal logical activation bytes but different finite-value behavior on extreme workloads. State expected throughput from the tested GPU architecture, define reference error metrics, and identify operations whose internal accumulation may reduce the visible difference.

Part 4 - Experimental Design and Instrumentation

Replay cloned model, optimizer, inputs, and RNG state for float16 and bfloat16 autocast. Run warm-up and repeated synchronized steps. Record peak allocated and reserved VRAM, throughput, output and gradient error against float32, non-finite counts, loss trajectory, and profiler evidence for executed kernels.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def autocast_step(model, inputs, loss_function, targets, dtype):
    model.zero_grad(set_to_none=True)
    with torch.autocast(device_type="cuda", dtype=dtype):
        outputs = model(inputs)
        loss = loss_function(outputs, targets)
    loss.backward()
    finite_gradients = True
    for parameter in model.parameters():
        if parameter.grad is not None:
            current_is_finite = torch.isfinite(parameter.grad).all()
            if not bool(current_is_finite):
                finite_gradients = False
    return outputs.detach(), loss.detach(), finite_gradients
```

```
def E08(model, inputs, targets, loss_function):
    results = {}
    for dtype in [torch.float16, torch.bfloat16]:
        results[str(dtype)] = autocast_step(
            model, inputs, loss_function, targets, dtype)
    return results
```

Part 6 - Measurement, Validation, and Failure Analysis

Restore identical state before each mode and use dtype-appropriate tolerances. Check whether the GPU provides native bfloat16 acceleration. Separate arithmetic error from stochastic divergence, and report unsupported or slower kernels rather than assuming every 16-bit format reaches Tensor Cores equally.

Part 7 - Engineering Interpretation and Philosophy

The preferred 16-bit format is workload- and hardware-specific. Equal storage width creates a capacity tie, but numerical range, precision, kernel support, and optimization dynamics determine the engineering winner.

19.21.10 Exercise 09

Exercise Title: Reproduce float16 Overflow and Underflow with Controlled Tensor Operations and Compare the Same Experiment in bfloat16 and float32

Part 1 - Exercise Contract and Why It Matters

Construct controlled operations that cross the maximum finite value and minimum representable scale of each dtype. Observe overflow, subnormal behavior, rounding to zero, and propagation through multiplication, reduction, exponential, and gradient computation.

Part 2 - System and Memory Mental Model

Overflow produces infinity when magnitude exceeds the finite range. Underflow can produce subnormal values or zero when magnitude becomes too small, with device behavior affected by flush-to-zero policies. bfloat16 has a wide exponent range but coarse precision; float16 has a much narrower range.

Part 3 - Quantitative Prediction Before Execution

Use torch.finfo to calculate candidate thresholds before execution. Predict the first scale at which each operation becomes non-finite or zero for each dtype. Include accumulation length in reduction predictions and distinguish forward underflow from gradient underflow.

Part 4 - Experimental Design and Instrumentation

Sweep logarithmic input scales and operation depths for all three dtypes. Record values, finite masks, zero counts, relative error, and gradients against float64 or analytically known references. Synchronize and document hardware because subnormal handling can vary.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def range_probe(dtype, device="cuda"):
    information = torch.finfo(dtype)
    large = torch.tensor([information.max], dtype=dtype, device=device)
    small = torch.tensor([information.tiny], dtype=dtype, device=device)
    return {
        "maximum": large,
        "overflow": large * torch.tensor(2.0, dtype=dtype, device=device),
        "minimum_normal": small,
        "underflow": small * torch.tensor(0.25, dtype=dtype, device=device),
    }

def E09(device="cuda"):
    results = {}
    dtypes = [torch.float16, torch.bfloat16, torch.float32]
    for dtype in dtypes:
        results[str(dtype)] = range_probe(dtype, device)
    return results
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not use only one hand-picked value. Verify thresholds with a sweep and inspect both finite and zero status. Avoid converting to Python floats before recording dtype behavior. Explain subnormal or flush-to-zero observations and validate gradients separately from forward results.

Part 7 - Engineering Interpretation and Philosophy

Numerical range failures are predictable resource limits, not mysterious training instability. A controlled reproduction provides the evidence needed to decide between a wider exponent, scaling, or a float32 computation island.

19.21.11 Exercise 10

Exercise Title: Identify Numerically Sensitive Operations and Force Selected Autocast Subregions to Execute in float32

Part 1 - Exercise Contract and Why It Matters

Identify unstable regions from measured error or non-finite values, then create minimal float32 islands inside a lower-precision autocast graph. Preserve the memory and throughput benefit of surrounding operations while repairing numerical behavior.

Part 2 - System and Memory Mental Model

Nested autocast contexts can disable automatic casting for a subregion, but incoming half-precision tensors may need explicit float conversion. Reductions, exponentials, normalization statistics, divisions by small values, and ill-conditioned linear algebra are common candidates, but evidence must select the actual boundary.

Part 3 - Quantitative Prediction Before Execution

Rank candidate operations by predicted range and cancellation risk. Estimate extra bytes, conversions, and latency for each proposed float32 island. Define the smallest acceptable island and the error or finite-value improvement it should produce.

Part 4 - Experimental Design and Instrumentation

Instrument a failing workload, then compare all-autocast, targeted float32 islands, and full float32 from identical state. Record operation error, gradients, peak VRAM, conversion traffic, kernel time, and end-to-end throughput. Expand the island only when the smaller boundary fails validation.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def sensitive_softmax_region(logits):
    with torch.autocast(device_type="cuda", enabled=False):
        float_logits = logits.float()
        probabilities = torch.softmax(float_logits, dim=-1)
    return probabilities

def mixed_region(x, weight):
    with torch.autocast(device_type="cuda", dtype=torch.float16):
        logits = x @ weight
        probabilities = sensitive_softmax_region(logits)
    return probabilities

def E10(x, weight):
    return mixed_region(x, weight)
```

Part 6 - Measurement, Validation, and Failure Analysis

Confirm that profiler events and tensor probes show the intended subregion in float32. Check output and gradient error, not only absence of NaN. Measure whether implicit conversions create unexpected temporaries or widen the island. Test boundary values and multiple seeds.

Part 7 - Engineering Interpretation and Philosophy

Selective precision is more powerful than an all-or-nothing dtype choice. The goal is to place accuracy where it protects semantics and reduced precision where it buys capacity and speed, using the smallest measured intervention.

19.21.12 Exercise 11

Exercise Title: Build a Mixed-Precision Policy That Assigns Different Dtypes to Parameters, Inputs, Activations, and Numerically Sensitive Computations

Part 1 - Exercise Contract and Why It Matters

Create an explicit, auditable policy for storage and compute dtypes across tensor roles. The policy must distinguish parameter storage, master parameters, inputs, activations, reductions, losses, gradients, optimizer state, and output reporting.

Part 2 - System and Memory Mental Model

Storage dtype and compute dtype are separate decisions. Autocast can feed lower-precision operands to eligible kernels while parameters remain float32. Gradients and optimizer state may use yet another policy. Conversions create memory traffic, temporary tensors, and possible loss of information at each boundary.

Part 3 - Quantitative Prediction Before Execution

For every tensor role, predict resident bytes, conversion count, arithmetic range, and expected error. Calculate a policy memory budget and identify hard float32 constraints. State how the policy changes under float16 versus bfloat16 hardware support.

Part 4 - Experimental Design and Instrumentation

Represent the policy as data rather than scattered casts. Apply it to a small model, log actual dtypes at boundaries, and compare against float32 and default autocast. Report peak memory, throughput, conversion operations, gradient error, optimizer-state bytes, and any policy violation.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass
import torch

@dataclass
class MixedPrecisionPolicy:
    parameter_storage: object = torch.float32
    input_compute: object = torch.float16
    activation_compute: object = torch.float16
    sensitive_compute: object = torch.float32
    optimizer_state: object = torch.float32

    def validate(self):
        allowed = {torch.float16, torch.bfloat16, torch.float32}
        values = [
            self.parameter_storage,
            self.input_compute,
            self.activation_compute,
            self.sensitive_compute,
            self.optimizer_state,
        ]
        return all(value in allowed for value in values)

def E11(activation_dtype=torch.float16):
    return MixedPrecisionPolicy(
        input_compute=activation_dtype,
        activation_compute=activation_dtype)
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify actual tensor and profiler dtypes against the declared policy. Detect redundant conversions, accidental permanent parameter casts, unsupported kernels, and optimizer state that silently uses a different dtype. A policy that saves memory but violates error constraints must be recorded as infeasible.

Part 7 - Engineering Interpretation and Philosophy

Mixed precision becomes manageable when represented as a resource policy rather than a sequence of ad hoc casts. Explicit roles make memory accounting, numerical review, and adaptation possible across models and hardware.

19.21.13 Exercise 12

Exercise Title: Compare Pure Half-Precision Training Against Autocast Mixed Precision and Explain Why Casting the Entire Model Is Not Equivalent

Part 1 - Exercise Contract and Why It Matters

Compare a model converted entirely to float16 or bfloat16 with a float32 model trained under autocast. Track stored parameters, operation dtypes, gradients, optimizer state, memory, speed, and numerical failures to explain the semantic difference.

Part 2 - System and Memory Mental Model

model.half changes persistent floating parameters and buffers, whereas autocast normally leaves storage in float32 and chooses compute dtype per eligible operation. Pure half can expose optimizer updates, reductions, normalization, and accumulations to limited range or precision. Autocast retains selected float32 paths.

Part 3 - Quantitative Prediction Before Execution

Predict which persistent components shrink only in pure half, which operations remain float32 only under autocast, and where pure half is expected to overflow or lose small updates. Estimate both parameter-state memory and activation memory before execution.

Part 4 - Experimental Design and Instrumentation

Clone identical initial weights into pure-half, autocast, and float32 branches. Use compatible inputs and optimizers, then run repeated training steps. Record peak VRAM, parameter and optimizer bytes, operation dtypes, throughput, loss, gradient and update error, and non-finite counts.

Part 5 - PyTorch/CUDA Implementation

```
import copy
import torch

def pure_half_step(model, inputs, targets, loss_function):
    half_model = copy.deepcopy(model).half()
    outputs = half_model(inputs.half())
    loss = loss_function(outputs.float(), targets)
    loss.backward()
    return half_model, outputs, loss
```

```
def autocast_step(model, inputs, targets, loss_function):
    with torch.autocast(device_type="cuda", dtype=torch.float16):
        outputs = model(inputs)
        loss = loss_function(outputs, targets)
    loss.backward()
    return model, outputs, loss

def E12(model, inputs, targets, loss_function):
    return {
        "pure_half": pure_half_step(model, inputs, targets, loss_function),
        "autocast": autocast_step(model, inputs, targets, loss_function),
    }
```

Part 6 - Measurement, Validation, and Failure Analysis

Restore state before each branch and check parameter, buffer, gradient, and optimizer dtypes explicitly. Ensure loss computation is comparable. Report operations unsupported in pure half and distinguish a lower persistent-memory footprint from a valid training result.

Part 7 - Engineering Interpretation and Philosophy

Pure half precision and autocast solve different problems. One rewrites storage broadly; the other selects computation locally. Capacity gains cannot justify treating them as equivalent when their numerical contracts and optimizer behavior differ.

19.21.14 Exercise 13

Exercise Title: Profile Mixed-Precision Tensor Core Operations and Determine Whether Reduced Memory Traffic Produces Real Throughput Gains

Part 1 - Exercise Contract and Why It Matters

Use profiler evidence to determine whether mixed precision executes Tensor Core-eligible kernels and whether smaller operands reduce exposed memory traffic or step time. Separate theoretical FLOP capability from achieved end-to-end throughput.

Part 2 - System and Memory Mental Model

Tensor Core use depends on GPU architecture, dtype, shape alignment, library selection, and operation. Reduced operand width can lower memory traffic, but conversions, launch overhead, noneligible operations, communication, or input pipelines may remain on the critical path. Kernel speedup can be hidden at step level.

Part 3 - Quantitative Prediction Before Execution

For selected GEMM and convolution shapes, predict arithmetic intensity, transferred bytes, alignment eligibility, and expected kernel speedup. Identify a compute-bound case, a memory-bound case, and a small case dominated by overhead.

Part 4 - Experimental Design and Instrumentation

Profile float32, float16 autocast, and bfloat16 autocast after warm-up. Capture CUDA kernel time, shapes, memory activity, Tensor Core indicators available in the profiler, and end-to-end synchronized throughput. Sweep aligned and misaligned dimensions while holding useful work constant.

Part 5 - PyTorch/CUDA Implementation

```
from contextlib import nullcontext
import torch
from torch.profiler import ProfilerActivity, profile

def profile_region(workload, dtype):
    precision_context = (
        nullcontext()
        if dtype is torch.float32
        else torch.autocast(device_type="cuda", dtype=dtype))
    with profile(
            activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
            record_shapes=True,
            profile_memory=True) as current_profile:
        with precision_context:
            workload()
        torch.cuda.synchronize()
    return current_profile.key_averages().table(
        sort_by="self_cuda_time_total", row_limit=20)

def E13(workload):
    results = {}
    dtypes = [torch.float32, torch.float16, torch.bfloat16]
    for dtype in dtypes:
        results[str(dtype)] = profile_region(workload, dtype)
    return results
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not claim Tensor Core use from dtype alone. Correlate kernel names, shapes, hardware capability, and timing. Exclude warm-up and profiler overhead from final throughput measurements. Verify identical outputs within tolerance and report cases where a faster kernel does not improve the full step.

Part 7 - Engineering Interpretation and Philosophy

Specialized arithmetic is valuable only when it shortens the workload's critical path. Profiling links precision policy to executed kernels and prevents theoretical throughput or payload reduction from being reported as an achieved application gain.

19.21.15 Exercise 14

Exercise Title: Increase Batch Size Until float32 Reaches OOM and Measure How Much Additional Batch Capacity Mixed Precision Unlocks

Part 1 - Exercise Contract and Why It Matters

Find the maximum stable training batch size under a fixed VRAM budget for float32, float16 autocast, and bfloat16 autocast. Measure capacity, throughput, numerical behavior, and fragmentation rather than stopping at the first avoided OOM.

Part 2 - System and Memory Mental Model

Batch growth increases inputs, activations, saved tensors, and some workspaces, while parameters and optimizer state remain mostly fixed. OOM thresholds can be nonmonotonic because algorithm selection, allocator bins, fragmentation, and temporary peaks change with shape. Mixed precision reduces only dtype-sensitive components.

Part 3 - Quantitative Prediction Before Execution

Build a fixed-plus-variable memory model and predict the maximum batch for each mode. Estimate the theoretical capacity gain from the variable component rather than assuming a twofold increase. Define a safety margin below the observed maximum.

Part 4 - Experimental Design and Instrumentation

Search batch sizes in isolated trials, beginning with exponential growth and finishing with a bounded search. Run warm-up and several full training steps at every candidate. Record OOM stage, peak allocated and reserved VRAM, throughput, loss and gradient validity, and the largest size that survives repeated execution.

Part 5 - PyTorch/CUDA Implementation

```
from contextlib import nullcontext
import torch

def trial(run_step, batch_size, dtype):
    try:
        torch.cuda.reset_peak_memory_stats()
        precision_context = (
            nullcontext()
            if dtype is torch.float32
            else torch.autocast(device_type="cuda", dtype=dtype))
        with precision_context:
            run_step(batch_size)
        torch.cuda.synchronize()
        return {
            "success": True,
            "peak": torch.cuda.max_memory_allocated(),
            "error": None,
        }
    except torch.cuda.OutOfMemoryError as error:
        return {"success": False, "peak": None, "error": str(error)}

def E14(run_step, batch_sizes):
    results = {}
    for dtype in [torch.float32, torch.float16, torch.bfloat16]:
        dtype_results = []
        for batch_size in batch_sizes:
            trial_result = trial(run_step, batch_size, dtype)
            dtype_results.append((batch_size, trial_result))
        results[str(dtype)] = dtype_results
    return results
```

Part 6 - Measurement, Validation, and Failure Analysis

An OOM trial can leave allocator and Python state unsuitable for the next candidate, so isolate trials or perform verified cleanup outside the measured path. Confirm full forward, backward, and update success. Repeat the selected maximum and report a safe operating batch rather than a one-time edge result.

Part 7 - Engineering Interpretation and Philosophy

The practical value of memory reduction is often additional useful work per step. Capacity testing translates byte savings into batch size while exposing fixed memory, nonlinear workspaces, and the difference between a fragile maximum and an operational budget.

19.21.16 Exercise 15

Exercise Title: Increase Transformer Sequence Length Until float32 Reaches OOM and Measure the Context-Length Gain from Mixed Precision

Part 1 - Exercise Contract and Why It Matters

Measure the maximum stable Transformer sequence length under a fixed batch, model, attention implementation, and VRAM budget. Compare precision modes while accounting for linear and quadratic activation components and validating attention numerics.

Part 2 - System and Memory Mental Model

Sequence length grows token representations approximately linearly, while materialized attention scores can grow quadratically. Kernel-fused attention may avoid storing the full score matrix and change the scaling law. Precision reduces floating payload but does not alter indices, masks, allocator overhead, or every workspace.

Part 3 - Quantitative Prediction Before Execution

Derive a memory model with fixed, linear, and quadratic terms for the selected implementation. Predict maximum length for each dtype and explain why halving bytes does not necessarily double context length when a quadratic term dominates.

Part 4 - Experimental Design and Instrumentation

Use fixed batch, hidden size, layers, attention backend, checkpointing state, and padding policy. Search increasing lengths in isolated full-step trials. Record peak VRAM, reserved memory, tokens per second, attention output error, gradient validity, OOM stage, and the last repeatedly stable length.

Part 5 - PyTorch/CUDA Implementation

```
from contextlib import nullcontext
import torch

def sequence_trial(run_step, sequence_length, dtype):
    try:
        torch.cuda.reset_peak_memory_stats()
        precision_context = (
            nullcontext()
```

```
            if dtype is torch.float32
            else torch.autocast(device_type="cuda", dtype=dtype))
        with precision_context:
            metrics = run_step(sequence_length)
        torch.cuda.synchronize()
        return True, torch.cuda.max_memory_allocated(), metrics
    except torch.cuda.OutOfMemoryError as error:
        return False, None, str(error)

def E15(run_step, sequence_lengths):
    report = {}
    for dtype in [torch.float32, torch.float16, torch.bfloat16]:
        report[str(dtype)] = []
        for length in sequence_lengths:
            report[str(dtype)].append(
                (length, sequence_trial(run_step, length, dtype)))
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Confirm that attention backend and kernel choice remain documented across modes and lengths. Test complete backward and update, not forward alone. Detect non-finite attention probabilities or gradients before OOM. Repeat boundary lengths and retain a VRAM safety margin.

Part 7 - Engineering Interpretation and Philosophy

Context length is a workload capability produced by a memory policy and an attention algorithm together. Mixed precision buys bytes, but the graph's scaling law determines how those bytes translate into additional tokens.

19.21.17 Exercise 16

Exercise Title: Combine Mixed Precision with Activation Checkpointing and Determine Which Memory Components Each Technique Actually Reduces

Part 1 - Exercise Contract and Why It Matters

Combine autocast mixed precision with activation checkpointing and attribute every memory change to reduced tensor width, discarded activations, recomputation, or unchanged training state. Compare combined behavior against each technique alone under identical semantics.

Part 2 - System and Memory Mental Model

Mixed precision shrinks eligible tensor payload and may change kernels. Checkpointing removes selected saved activations and replays forward computation during backward. Their savings overlap on checkpointed tensors, so percentages are not simply additive. Recomputed operations must reproduce the intended autocast context and RNG behavior.

Part 3 - Quantitative Prediction Before Execution

Partition memory into parameters, gradients, optimizer state, retained activations inside and outside checkpoint regions, and workspaces. Predict float32 baseline, mixed-only, checkpoint-only, and combined peaks. Estimate replay FLOPs and whether lower precision accelerates recomputation.

Part 4 - Experimental Design and Instrumentation

Benchmark the four modes from cloned state with identical checkpoint boundaries and non-reentrant or reentrant choice documented. Record peak VRAM, saved-tensor bytes, replay count, step latency, throughput, output and gradient error, and RNG-sensitive behavior.

Part 5 - PyTorch/CUDA Implementation

```
import torch
from torch.utils.checkpoint import checkpoint

def checkpointed_forward(block, hidden_states, use_checkpoint):
    if use_checkpoint:
        return checkpoint(block, hidden_states, use_reentrant=False)
    return block(hidden_states)

def run_mode(block, hidden_states, dtype, use_checkpoint):
    with torch.autocast(device_type="cuda", dtype=dtype):
        return checkpointed_forward(
            block, hidden_states, use_checkpoint)

def E16(block, hidden_states):
    return {
        "mixed_only": run_mode(
            block, hidden_states, torch.float16, False),
        "mixed_checkpoint": run_mode(
            block, hidden_states, torch.float16, True),
    }
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify outputs and gradients against float32, preserve RNG where required, and ensure recomputation runs under the expected autocast policy. Measure actual saved tensors rather than inferring checkpoint success. Explain overlapping savings and report profiler overhead separately.

Part 7 - Engineering Interpretation and Philosophy

Memory techniques compose through the tensors they affect, not through headline percentages. Component attribution reveals whether a combination removes new bytes, shrinks the same bytes twice in theory, or moves cost onto recomputation.

19.21.18 Exercise 17

Exercise Title: Combine Mixed Precision with Activation Offloading and Measure the Reduction in Both VRAM Usage and CPU-GPU Transfer Volume

Part 1 - Exercise Contract and Why It Matters

Apply activation offloading under float32, float16 autocast, and bfloat16 autocast. Measure how reduced saved-tensor width changes GPU residency, host residency, D2H and H2D traffic, transfer time, overlap, and backward stalls.

Part 2 - System and Memory Mental Model

Offloading moves tensors saved by autograd, whose dtypes follow operation-specific precision policy. A 16-bit saved tensor can halve payload and transfer bytes relative to float32, but float32 islands remain wide. Latency contains fixed per-transfer overhead, allocation cost, synchronization, and bandwidth-dependent payload time.

Part 3 - Quantitative Prediction Before Execution

Use the saved-tensor trace to predict bytes moved by dtype and producer. Estimate D2H and H2D time from measured bandwidth plus fixed overhead, and identify small tensors for which width reduction will not halve latency. Predict host RAM and peak VRAM for each mode.

Part 4 - Experimental Design and Instrumentation

Run keep-only and offload modes for all precision policies from identical state. Record unique saved bytes by dtype, selected transfers, host and pinned peaks, D2H and H2D traffic, event wait time, peak VRAM, step latency, and gradient error. Preserve the same offload threshold and also compare an equal-VRAM policy.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass
import torch

@dataclass
class OffloadedTensor:
    cpu_tensor: object
    original_device: object
    logical_bytes: int

class TransferCounter:
    def __init__(self):
        self.d2h_bytes = 0
        self.h2d_bytes = 0

    def pack(self, tensor):
        size = tensor.numel() * tensor.element_size()
        self.d2h_bytes += size
        return OffloadedTensor(tensor.to("cpu"), tensor.device, size)

    def unpack(self, item):
        self.h2d_bytes += item.logical_bytes
        return item.cpu_tensor.to(item.original_device)
```

```
def E17(model_region, dtype):
    counter = TransferCounter()
    with torch.autograd.graph.saved_tensors_hooks(
            counter.pack, counter.unpack):
        with torch.autocast(device_type="cuda", dtype=dtype):
            output = model_region()
    return output, counter
```

Part 6 - Measurement, Validation, and Failure Analysis

The compact hook is a blocking measurement baseline, not an optimized transport. Deduplicate aliases where physical traffic differs from logical saves, synchronize transfer timing, and verify outputs and gradients. Report wide tensors that remain after autocast and separate lower bytes from reduced exposed stall.

Part 7 - Engineering Interpretation and Philosophy

Precision and placement interact through the payload crossing the memory hierarchy. Smaller tensors can reduce both residency and traffic, but only measurement reveals whether bandwidth, fixed copy overhead, or backward dependency controls the final step time.

19.21.19 Exercise 18

Exercise Title: Build an Adaptive Precision Controller That Changes Selected Tensor Dtypes from VRAM Pressure and Numerical Error Measurements

Part 1 - Exercise Contract and Why It Matters

Build a feedback controller that selects float32, bfloat16, or float16 for eligible regions using VRAM pressure, measured error, non-finite events, and throughput. Decisions must be bounded, auditable, and stable rather than oscillating every step.

Part 2 - System and Memory Mental Model

Precision selection is a control problem with delayed and noisy observations. Lower precision can reduce memory and latency but increase error risk. High and low VRAM watermarks, error budgets, cooldown periods, and emergency promotion rules provide hysteresis and safety. Dtype changes can alter kernels and allocator state.

Part 3 - Quantitative Prediction Before Execution

For each region, estimate bytes saved and error sensitivity for every dtype. Define pressure and error thresholds, promotion and demotion order, observation windows, and a maximum change rate. Predict controller behavior for low memory, sudden numerical failure, and recovered pressure.

Part 4 - Experimental Design and Instrumentation

Replay workloads with variable batch or sequence length and injected numerical stress. Log every observation, decision, reason, predicted bytes, achieved peak, error, finite status, and latency. Compare against static float32, float16, and bfloat16 policies under the same accuracy and VRAM constraints.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass
import torch

@dataclass
class PrecisionObservation:
    peak_vram: int
    error: float
    finite: bool

class AdaptivePrecisionController:
    def __init__(self, high_watermark, low_watermark, error_limit):
        self.high_watermark = high_watermark
        self.low_watermark = low_watermark
        self.error_limit = error_limit
        self.dtype = torch.float32
        self.history = []

    def update(self, observation):
        if not observation.finite or observation.error > self.error_limit:
            self.dtype = torch.float32
            reason = "numerical protection"
        elif observation.peak_vram > self.high_watermark:
            self.dtype = torch.bfloat16
            reason = "memory pressure"
        elif observation.peak_vram < self.low_watermark:
            reason = "hold below low watermark"
        else:
            reason = "hold inside hysteresis band"
        self.history.append((observation, self.dtype, reason))
        return self.dtype

def E18(high_watermark, low_watermark, error_limit):
    return AdaptivePrecisionController(
        high_watermark, low_watermark, error_limit)
```

Part 6 - Measurement, Validation, and Failure Analysis

Test invalid watermark ordering, noisy measurements, repeated non-finite events, unsupported dtype regions, and workload changes. Require a float32 recovery path and verify that decisions actually change executed operation dtypes. Report oscillation, constraint violations, and adaptation overhead.

Part 7 - Engineering Interpretation and Philosophy

Adaptive precision treats numerical format as a schedulable resource. A trustworthy controller protects semantics first, uses hysteresis to remain stable, and leaves an evidence trail explaining why capacity was purchased with a particular precision choice.

19.21.20 Exercise 19

Exercise Title: Design and Benchmark a Memory-Budgeted Mixed-Precision PyTorch Training Strategy That Maximizes Model Size or Batch Size Without Violating Accuracy and VRAM Constraints

Part 1 - Exercise Contract and Why It Matters

Integrate dtype policy, autocast, selective float32 islands, capacity search, profiling, and validation into one memory-budgeted training strategy. Maximize a declared objective such as batch size, sequence length, or model width while satisfying hard VRAM and numerical constraints.

Part 2 - System and Memory Mental Model

The strategy has a planner and an executor. The planner predicts component bytes and ranks precision policies by capacity, error risk, and throughput. The executor applies autocast boundaries, measures the actual peak, validates outputs and gradients, and feeds evidence back to the planner. Feasibility precedes optimization.

Part 3 - Quantitative Prediction Before Execution

Enumerate candidate policies and workload sizes. For each, estimate parameter, optimizer, activation, saved-tensor, workspace, and allocator-safety bytes together with expected error and throughput. Reject candidates above budget or risk limits before ranking the feasible set.

Part 4 - Experimental Design and Instrumentation

Benchmark float32, default float16 and bfloat16 autocast, selective float32 islands, and the adaptive strategy from identical initial state. Search the workload objective, then report the largest repeatably feasible configuration, peak allocated and reserved VRAM, safety margin, throughput distribution, loss and gradient error, non-finite rate, and policy decisions.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass
import torch

@dataclass
class StrategyCandidate:
    name: str
    dtype: object
    workload_size: int
    predicted_vram: int
    predicted_error: float
    predicted_throughput: float

def feasible(candidate, vram_budget, error_budget, safety_margin):
    memory_limit = vram_budget - safety_margin
    return (
        candidate.predicted_vram <= memory_limit
        and candidate.predicted_error <= error_budget)
```

```
def choose_strategy(candidates, vram_budget, error_budget, safety_margin):
    accepted = []
    for candidate in candidates:
        if feasible(candidate, vram_budget, error_budget, safety_margin):
            accepted.append(candidate)
    accepted.sort(
        key=lambda item: (item.workload_size, item.predicted_throughput),
        reverse=True)
    return accepted[0] if accepted else None

def E19(candidates, vram_budget, error_budget, safety_margin):
    return choose_strategy(
        candidates, vram_budget, error_budget, safety_margin)
```

Part 6 - Measurement, Validation, and Failure Analysis

Predictions are admission filters, not proof. Execute the selected configuration repeatedly and reject it after any VRAM, finite-value, or error-budget violation. Include allocator fragmentation, warm-up, optimizer initialization, dynamic shapes, and tail latency. Compare against the strongest static feasible baseline.

Part 7 - Engineering Interpretation and Philosophy

A memory-budgeted precision strategy converts mixed precision from a local optimization into a constrained systems decision. Success is the largest useful workload that remains numerically valid, operationally stable, and explainable within a measured safety margin.

19.22 DMM20

Chapter Title: Gradient Scaling and Dynamic Loss Scaling
Specialization: Dynamic Memory Management and CUDA Systems

19.22.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Gradient Underflow, Overflow, Loss Scaling, and Their Relationship to Mixed-Precision Training

Part 1 - Exercise Contract and Why It Matters

Build a lifecycle model that follows a loss through scaling, backward, finite-value inspection, unscaling, clipping, optimizer update, and scale adjustment. Separate forward arithmetic precision from gradient representability and explain why loss scaling helps float16 gradients without increasing the information stored in the forward pass.

Part 2 - System and Memory Mental Model

Multiplying loss L by scale S multiplies every derivative by S through the chain rule. The larger gradient is more likely to remain representable in float16. Before the optimizer update, dividing by S restores the intended gradient. Overflow detection must happen before parameters are changed; a failed step is skipped and the scale is reduced.

Part 3 - Quantitative Prediction Before Execution

Predict the smallest gradient rescued by a chosen scale, the largest scaled gradient that remains finite, and the safe interval for S. Include accumulation dtype, reduction length, clipping threshold, optimizer state, and the distinction between loss overflow and gradient overflow.

Part 4 - Experimental Design and Instrumentation

Create controlled scalar and model experiments in float32, float16, and bfloat16. Record loss, scaled loss, raw gradients, unscaled gradients, finite masks, skipped updates, parameter deltas, and scale history. Fix seeds and compare every accepted update with a float32 reference.

Part 5 - PyTorch/CUDA Implementation

```
from dataclasses import dataclass
import torch

@dataclass
class ScaleState:
    scale: float
    growth_factor: float
    backoff_factor: float
    growth_interval: int
    successful_steps: int = 0

def scaled_gradient(loss, parameter, scale):
    scaled_loss = loss * scale
    gradient = torch.autograd.grad(scaled_loss, parameter)[0]
    return gradient, gradient / scale

def E00(initial_scale=65536.0):
    return ScaleState(initial_scale, 2.0, 0.5, 2000)
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not accept a finite loss as proof of finite gradients. Inspect every gradient before the update, verify that unscaled values match the reference within dtype-appropriate tolerances, and confirm that skipped steps leave parameters and optimizer state unchanged. Report underflow, infinity, NaN, and scale oscillation separately.

Part 7 - Engineering Interpretation and Philosophy

Loss scaling temporarily relocates gradients into a usable numerical range; it does not change the mathematical update when scaling and unscaling are correct. Dynamic scaling is therefore a numerical safety controller whose evidence is the sequence of accepted and rejected updates.

19.22.2 Exercise 01

Exercise Title: Construct Tiny float16 Gradients That Underflow to Zero and Compare Them Against float32 and bfloat16

Part 1 - Exercise Contract and Why It Matters

Construct analytically known gradients whose magnitudes cross the float16 normal and subnormal ranges. Compare stored gradient values in float16, bfloat16, and float32 and identify when nonzero mathematical derivatives become zero on the GPU.

Part 2 - System and Memory Mental Model

A gradient can vanish during an intermediate multiplication or when it is stored in a narrow dtype. float16 has a limited exponent range, while bfloat16 preserves a float32-like exponent range with fewer significand bits. Device flush-to-zero behavior can affect subnormal observations.

Part 3 - Quantitative Prediction Before Execution

Use torch.finfo to choose logarithmically spaced coefficients around each dtype threshold. Predict the exact derivative and classify it as normal, subnormal, rounded, or zero before execution.

Part 4 - Experimental Design and Instrumentation

Differentiate simple products and a short chain on CUDA for all three dtypes. Record analytical gradient, stored gradient, zero status, relative error, and the first coefficient that underflows. Repeat with and without a float32 parameter receiving a half-precision computation.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def tiny_gradient(dtype, coefficient, device="cuda"):
    parameter = torch.ones((), dtype=dtype, device=device, requires_grad=True)
    multiplier = torch.tensor(coefficient, dtype=dtype, device=device)
    loss = parameter * multiplier
    loss.backward()
    return parameter.grad.detach()

def E01(coefficients, device="cuda"):
    results = {}
    dtypes = [torch.float16, torch.bfloat16, torch.float32]
    for dtype in dtypes:
        gradients = []
        for coefficient in coefficients:
            gradients.append(tiny_gradient(dtype, coefficient, device))
        results[str(dtype)] = gradients
    return results
```

Part 6 - Measurement, Validation, and Failure Analysis

Avoid converting coefficients to the target dtype before preserving the analytical reference. Record subnormal behavior on the tested GPU and distinguish a zero gradient from a zero loss. Synchronize failures and repeat boundary values.

Part 7 - Engineering Interpretation and Philosophy

Underflow is a representational failure that can be exposed with a tiny controlled graph. Once its location is measurable, scaling or a wider-exponent dtype can be evaluated as a targeted remedy.

19.22.3 Exercise 02

Exercise Title: Construct Excessively Large float16 Gradients That Overflow to Infinity and Trace Their Effect Through an Optimizer Step

Part 1 - Exercise Contract and Why It Matters

Create finite forward values whose derivatives exceed float16 range, then trace infinity through momentum, parameter updates, and subsequent losses. Compare an unsafe update with a protected path that rejects the step.

Part 2 - System and Memory Mental Model

Overflow can occur in backward even when the forward loss is finite. If infinity reaches an optimizer, momentum or variance state can become permanently non-finite and corrupt later steps. Detection must precede any mutation of parameter or optimizer state.

Part 3 - Quantitative Prediction Before Execution

Derive the gradient magnitude from the chosen function and predict the first input or multiplier that exceeds torch.finfo(float16).max. Predict which optimizer state tensors will be contaminated after one unsafe update.

Part 4 - Experimental Design and Instrumentation

Clone identical parameter and optimizer state into unsafe and guarded branches. Sweep gradient magnitudes, record finite status before and after step, parameter deltas, optimizer-state values, and recovery on the next normal batch.

Part 5 - PyTorch/CUDA Implementation

```
import copy
import torch

def overflow_trial(parameter, optimizer, multiplier, allow_update):
    optimizer.zero_grad(set_to_none=True)
    loss = parameter * multiplier
    loss.backward()
    gradient_is_finite = bool(torch.isfinite(parameter.grad).all())
    before = parameter.detach().clone()
    if allow_update or gradient_is_finite:
        optimizer.step()
    after = parameter.detach().clone()
    return gradient_is_finite, before, after

def E02(parameter, optimizer, multiplier):
    return overflow_trial(parameter, optimizer, multiplier, True)
```

Part 6 - Measurement, Validation, and Failure Analysis

Inspect loss, gradients, parameters, and every tensor in optimizer state. A skipped update must cause no mutation. Restore state between trials and do not call zero_grad as a substitute for repairing contaminated optimizer state.

Part 7 - Engineering Interpretation and Philosophy

Overflow protection is transactional: either a numerically valid update and its state changes commit together, or the entire step is rejected.

19.22.4 Exercise 03

Exercise Title: Manually Multiply the Loss by a Fixed Scale Factor and Verify That Backpropagated Gradients Increase by the Same Factor

Part 1 - Exercise Contract and Why It Matters

Prove experimentally that multiplying a scalar loss by fixed S multiplies each parameter gradient by S. Use multiple layers, nonuniform gradient magnitudes, and several scales while keeping the forward graph identical.

Part 2 - System and Memory Mental Model

The chain rule gives the derivative of S times L with respect to a parameter as S times the derivative of L. The relationship holds before rounding or overflow; finite-precision arithmetic can break exact proportionality at extreme scales.

Part 3 - Quantitative Prediction Before Execution

Calculate reference gradients in float32 and predict scaled values for every parameter. Define absolute and relative tolerances and a scale range expected to remain finite.

Part 4 - Experimental Design and Instrumentation

Run reference and scaled backward passes from identical model state and inputs. Record per-parameter maximum error between scaled gradient and S times reference, zero counts, and non-finite counts.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def named_gradients(model):
    result = {}
    for name, parameter in model.named_parameters():
        if parameter.grad is not None:
            result[name] = parameter.grad.detach().clone()
    return result

def E03(model, loss, scale):
    model.zero_grad(set_to_none=True)
    scaled_loss = loss * scale
    scaled_loss.backward()
    return named_gradients(model)
```

Part 6 - Measurement, Validation, and Failure Analysis

Rebuild the graph for each backward pass and compare cloned initial state. Check that S is represented in a suitable dtype and report rounding, underflow rescue, overflow, and parameters with no gradients separately.

Part 7 - Engineering Interpretation and Philosophy

Fixed scaling is the algebraic core of dynamic loss scaling. Proving proportionality before automation makes every later skip and scale adjustment interpretable.

19.22.5 Exercise 04

Exercise Title: Manually Unscale Gradients Before the Optimizer Step and Prove That the Final Parameter Update Matches an Unscaled Reference

Part 1 - Exercise Contract and Why It Matters

Implement fixed loss scaling followed by explicit in-place gradient division before optimizer.step. Demonstrate equivalence of gradients, parameter updates, and optimizer state against an unscaled reference.

Part 2 - System and Memory Mental Model

The optimizer must consume gradients divided by the same scale used on the loss. Unscaling after optimizer.step is too late because parameter and optimizer state have already incorporated the enlarged gradient.

Part 3 - Quantitative Prediction Before Execution

Predict every reference update for SGD and one stateful optimizer. State tolerances for unscaled gradients, parameters, momentum or variance, and loss on the following step.

Part 4 - Experimental Design and Instrumentation

Clone model and optimizer state into reference, correct-unscale, and missing-unscale branches. Compare gradients immediately before step and all mutated state immediately after step.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def unscale_gradients(model, scale):
    inverse_scale = 1.0 / scale
    for parameter in model.parameters():
        if parameter.grad is not None:
            parameter.grad.mul_(inverse_scale)

def E04(model, optimizer, loss, scale):
    optimizer.zero_grad(set_to_none=True)
    (loss * scale).backward()
    unscale_gradients(model, scale)
    optimizer.step()
    return model
```

Part 6 - Measurement, Validation, and Failure Analysis

Check gradient finiteness before and after unscale. Use identical optimizer hyperparameters and initialized state. Verify that the incorrect branch diverges by the predicted scale rather than merely reporting different loss.

Part 7 - Engineering Interpretation and Philosophy

Scaling remains mathematically invisible only when the optimizer boundary receives restored gradients. That boundary is the central invariant of the training loop.

19.22.6 Exercise 05

Exercise Title: Train a Small CUDA Model in float16 Without Gradient Scaling and Record When Gradients Become Zero, Infinite, or NaN

Part 1 - Exercise Contract and Why It Matters

Establish a pure float16 no-scaling failure baseline. Track per-layer gradients, parameter updates, loss, and optimizer state until underflow, overflow, or NaN appears.

Part 2 - System and Memory Mental Model

Small gradients can round to zero while large reductions can overflow. Once a non-finite value enters stateful optimizer buffers, later normal batches may not recover. Pure half storage also exposes parameter updates to limited precision.

Part 3 - Quantitative Prediction Before Execution

Predict vulnerable layers from activation scale, depth, loss magnitude, and gradient norms. Define zero-rate, non-finite-rate, and update-to-weight ratio thresholds before training.

Part 4 - Experimental Design and Instrumentation

Train from fixed state on a reproducible batch sequence. Record every step, layer, gradient minimum nonzero magnitude, zero fraction, infinity and NaN counts, parameter delta, and first failure step.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def gradient_health(model):
    report = {}
    for name, parameter in model.named_parameters():
        gradient = parameter.grad
        if gradient is None:
            report[name] = None
            continue
        report[name] = {
            "zero": int((gradient == 0).sum()),
            "infinite": int(torch.isinf(gradient).sum()),
            "nan": int(torch.isnan(gradient).sum()),
        }
```

```
    return report

def E05(model, optimizer, loss):
    optimizer.zero_grad(set_to_none=True)
    loss.backward()
    report = gradient_health(model)
    optimizer.step()
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not treat an improving loss as proof of healthy gradients. Inspect every layer and optimizer tensor. Preserve the first failing batch and state so the scaled experiment can replay it exactly.

Part 7 - Engineering Interpretation and Philosophy

A failure baseline identifies the numerical debt that scaling must repay and prevents stability claims from relying on an easy workload.

19.22.7 Exercise 06

Exercise Title: Add torch.amp.GradScaler to the Same Training Loop and Measure the Change in Numerical Stability

Part 1 - Exercise Contract and Why It Matters

Add GradScaler to the identical autocast training loop and quantify rescued gradients, skipped updates, scale changes, loss behavior, and throughput relative to the no-scaling baseline.

Part 2 - System and Memory Mental Model

GradScaler scales loss, unscales and checks gradients during step, skips unsafe optimizer updates, then adjusts its scale. It is useful for float16; bfloat16 often does not need the same underflow protection.

Part 3 - Quantitative Prediction Before Execution

Predict the initial transient, expected overflow frequency, stable scale range, and number of gradients rescued from zero. Estimate the runtime overhead of checks and scale bookkeeping.

Part 4 - Experimental Design and Instrumentation

Replay identical batches and state with scaling disabled and enabled. Record loss, scale before and after update, update-skipped inference, gradient health after unscale, peak VRAM, and synchronized step time.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def E06(model, optimizer, inputs, targets, loss_function, scaler):
    optimizer.zero_grad(set_to_none=True)
```

```
    with torch.autocast(device_type="cuda", dtype=torch.float16):
        outputs = model(inputs)
        loss = loss_function(outputs, targets)
    scale_before = scaler.get_scale()
    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
    scale_after = scaler.get_scale()
    return loss.detach(), scale_before, scale_after
```

Part 6 - Measurement, Validation, and Failure Analysis

Use torch.amp.GradScaler with the API supported by the installed PyTorch version. Infer skipped steps carefully and verify parameters directly. Compare only after identical warm-up and state restoration.

Part 7 - Engineering Interpretation and Philosophy

Automation is valuable when its internal control signal remains observable. Scale history and accepted updates are part of the training evidence, not hidden implementation details.

19.22.8 Exercise 07

Exercise Title: Inspect the Gradient Scale Across Training Steps and Explain Why It Can Increase, Decrease, or Remain Constant

Part 1 - Exercise Contract and Why It Matters

Build a scale timeline and associate every transition with finite or non-finite gradients, growth interval progress, and optimizer-update status.

Part 2 - System and Memory Mental Model

The scale decreases after detected overflow according to the backoff factor. It grows after a configured number of consecutive successful updates and otherwise remains constant. A skipped update resets or interrupts growth progress.

Part 3 - Quantitative Prediction Before Execution

Simulate a known success-failure sequence and predict scale at every step from initial scale, growth factor, backoff factor, and growth interval.

Part 4 - Experimental Design and Instrumentation

Run a controlled training loop with injected overflow steps. Record scale before backward, after update, success counter, finite status, and parameter checksum. Compare observation with the predicted state machine.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def E07(run_step, scaler, number_of_steps):
    history = []
```

```
    for step_index in range(number_of_steps):
        scale_before = scaler.get_scale()
        run_step(scaler, step_index)
        scale_after = scaler.get_scale()
        history.append({
            "step": step_index,
            "before": scale_before,
            "after": scale_after,
        })
    return history
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not assume a decreased scale is the only skip indicator across all configurations. Verify parameter changes and gradient finiteness. Record scaler configuration and software version with the timeline.

Part 7 - Engineering Interpretation and Philosophy

Dynamic scaling is understandable as a small state machine. Making transitions explicit turns numerical adaptation into reproducible system behavior.

19.22.9 Exercise 08

Exercise Title: Force an Overflow and Verify That Gradient Scaling Skips the Unsafe Optimizer Update Instead of Corrupting Model Parameters

Part 1 - Exercise Contract and Why It Matters

Inject a deterministic overflow into one gradient and prove that GradScaler prevents both parameter and optimizer-state mutation for that step.

Part 2 - System and Memory Mental Model

After unscale, GradScaler checks for non-finite gradients associated with the optimizer. If any are found, optimizer.step is not executed and the scale is backed off. The next valid batch can proceed from intact state.

Part 3 - Quantitative Prediction Before Execution

Predict scale change, parameter checksum, optimizer-state checksum, and step count for overflow and recovery steps.

Part 4 - Experimental Design and Instrumentation

Snapshot model and optimizer state, inject infinity after scaled backward, call scaler.step and scaler.update, then compare all tensors with the snapshot. Follow with a finite batch and verify recovery.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def clone_parameters(model):
    copies = []
    for parameter in model.parameters():
        copies.append(parameter.detach().clone())
    return copies

def E08(model, optimizer, scaler, loss):
    optimizer.zero_grad(set_to_none=True)
    before = clone_parameters(model)
    scaler.scale(loss).backward()
    first_parameter = next(model.parameters())
    first_parameter.grad.fill_(float("inf"))
    scaler.step(optimizer)
    scaler.update()
    after = clone_parameters(model)
    return before, after, scaler.get_scale()
```

Part 6 - Measurement, Validation, and Failure Analysis

Compare optimizer state as well as parameters and ensure overflow is injected before scaler.step. Confirm a subsequent valid update succeeds. Test multiple optimizers because state mutation timing can differ.

Part 7 - Engineering Interpretation and Philosophy

Skipping is a correctness mechanism, not lost progress. One rejected update is cheaper than silently poisoning every future update.

19.22.10 Exercise 09

Exercise Title: Track Consecutive Successful Steps and Determine When Dynamic Loss Scaling Increases the Scale Factor

Part 1 - Exercise Contract and Why It Matters

Measure the exact success streak required for scale growth and verify that a single overflow changes the streak and scale as configured.

Part 2 - System and Memory Mental Model

Growth interval counts consecutive unskipped updates. When the interval is reached, scale is multiplied by the growth factor and the tracker restarts. Overflow applies backoff and resets stable progress.

Part 3 - Quantitative Prediction Before Execution

Write the expected scale sequence for several intervals, including an overflow immediately before and after a growth boundary.

Part 4 - Experimental Design and Instrumentation

Use deterministic finite batches and scheduled overflow injection. Record accepted updates, internal outcome inferred from state, scale transitions, and the first step at each new scale.

Part 5 - PyTorch/CUDA Implementation

```
def E09(run_step, scaler, schedule):
    records = []
    successful_steps = 0
    for step_index, force_overflow in enumerate(schedule):
        before = scaler.get_scale()
        update_happened = run_step(scaler, force_overflow)
        after = scaler.get_scale()
        if update_happened:
            successful_steps += 1
        else:
            successful_steps = 0
        records.append((step_index, successful_steps, before, after))
    return records
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify the run_step outcome from parameter or optimizer state rather than scale alone. Test off-by-one behavior at growth boundaries and record the exact scaler configuration.

Part 7 - Engineering Interpretation and Philosophy

Growth is earned confidence: repeated valid updates permit more aggressive scaling, while any failure returns the controller to caution.

19.22.11 Exercise 10

Exercise Title: Compare Several Initial Scale Values and Measure Their Effects on Early Training Stability, Overflow Frequency, and Convergence

Part 1 - Exercise Contract and Why It Matters

Sweep conservative, moderate, and aggressive initial scales while preserving identical training state and batch order. Measure warm-up failures, time to stable scale, rescued small gradients, and convergence.

Part 2 - System and Memory Mental Model

A scale that is too small may not prevent underflow; a scale that is too large causes early skipped updates and backoff. Dynamic adaptation can eventually converge, but the transient costs time and can interact with schedules.

Part 3 - Quantitative Prediction Before Execution

Predict a safe interval from measured gradient extrema and float16 range. For each initial scale, estimate required backoff steps and expected stable range.

Part 4 - Experimental Design and Instrumentation

Run isolated trials across multiple seeds. Record scale timeline, overflow count, skipped updates, gradient zero fraction, loss curve, final metric, peak VRAM, and total time to a fixed number of accepted updates.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def E10(initial_scales, training_trial):
    results = {}
    for initial_scale in initial_scales:
        scaler = torch.amp.GradScaler(
            "cuda", init_scale=initial_scale)
        results[initial_scale] = training_trial(scaler)
    return results
```

Part 6 - Measurement, Validation, and Failure Analysis

Compare by accepted updates as well as attempted steps. Restore model, optimizer, RNG, scheduler, and batches for every trial. Preserve failed initial scales as evidence.

Part 7 - Engineering Interpretation and Philosophy

Initialization is a prior belief about gradient magnitude. A good prior shortens adaptation, while dynamic control protects correctness when that belief is wrong.

19.22.12 Exercise 11

Exercise Title: Sweep Growth and Backoff Factors and Discover How Aggressive Dynamic Scaling Changes Recovery from Numerical Failures

Part 1 - Exercise Contract and Why It Matters

Sweep growth and backoff factors with fixed growth interval to measure scale oscillation, skipped-update rate, recovery time, and useful gradient range.

Part 2 - System and Memory Mental Model

Large growth factors explore higher scales quickly but can overshoot repeatedly. Strong backoff recovers from overflow rapidly but may return to underflow-prone scales. The best balance depends on gradient nonstationarity.

Part 3 - Quantitative Prediction Before Execution

Simulate scale trajectories for each factor pair under a fixed success-failure schedule before training. Predict oscillation amplitude and attempts lost after a spike.

Part 4 - Experimental Design and Instrumentation

Run factor pairs on identical dynamic workloads with sudden loss spikes. Record scale, finite status, accepted update, gradient zero rate, time to recovery, loss, and throughput.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def E11(factor_pairs, training_trial):
    results = {}
    for growth_factor, backoff_factor in factor_pairs:
        scaler = torch.amp.GradScaler(
            "cuda",
            growth_factor=growth_factor,
            backoff_factor=backoff_factor)
        key = (growth_factor, backoff_factor)
        results[key] = training_trial(scaler)
    return results
```

Part 6 - Measurement, Validation, and Failure Analysis

Reject invalid factor configurations and compare equal accepted updates. Report instability caused by the workload separately from oscillation created by the controller.

Part 7 - Engineering Interpretation and Philosophy

Growth and backoff determine how aggressively the controller searches numerical range. They are control parameters that must be evaluated against real gradient dynamics.

19.22.13 Exercise 12

Exercise Title: Unscale Gradients Before Gradient Clipping and Reproduce the Incorrect Result Produced by Clipping Scaled Gradients Directly

Part 1 - Exercise Contract and Why It Matters

Compare correct unscale-then-clip ordering with incorrect clipping of scaled gradients. Quantify the effective clipping threshold and resulting parameter update.

Part 2 - System and Memory Mental Model

Clipping scaled gradients at threshold C is equivalent to clipping true gradients near C divided by S. GradScaler.unscale_ must run before norm measurement and clipping, and only once per optimizer per step.

Part 3 - Quantitative Prediction Before Execution

Calculate true norm, scaled norm, correct clipping coefficient, and incorrect coefficient for chosen S and C. Predict parameter deltas in both branches.

Part 4 - Experimental Design and Instrumentation

Clone state into reference, correct, and incorrect branches. Record norms before and after unscale, returned clip norm, gradient tensors, parameter updates, and scaler outcome.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def E12(model, optimizer, scaler, loss, maximum_norm):
    optimizer.zero_grad(set_to_none=True)
    scaler.scale(loss).backward()
    scaler.unscale_(optimizer)
    total_norm = torch.nn.utils.clip_grad_norm_(
        model.parameters(), maximum_norm)
    scaler.step(optimizer)
    scaler.update()
    return total_norm
```

Part 6 - Measurement, Validation, and Failure Analysis

Rebuild the graph and restore state for each ordering. Ensure the incorrect branch clips before unscale, then prove its effective threshold mathematically. Detect non-finite norms before step.

Part 7 - Engineering Interpretation and Philosophy

Gradient transformations must operate in a declared numerical coordinate system. Unscaling restores the units expected by clipping and the optimizer.

19.22.14 Exercise 13

Exercise Title: Combine Gradient Scaling with Gradient Accumulation and Determine the Correct Points for Scaling, Unscaling, and Optimizer Updates

Part 1 - Exercise Contract and Why It Matters

Implement microbatch accumulation with one consistent scale across the effective batch. Determine when to divide loss, backward, unscale, clip, step, update the scaler, and clear gradients.

Part 2 - System and Memory Mental Model

Each microbatch loss is scaled before backward. The scale should remain unchanged during one accumulation window. Unscale, clipping, finite checks, optimizer step, and scaler update occur once after the final microbatch.

Part 3 - Quantitative Prediction Before Execution

Derive the accumulated gradient for sum and mean loss conventions. Predict how dividing each microbatch loss by accumulation steps affects the final update and clipping norm.

Part 4 - Experimental Design and Instrumentation

Compare a large-batch reference with accumulated microbatches from identical samples. Record scale per microbatch, accumulated gradient, finite status, clipping norm, update, and memory peak.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def E13(model, optimizer, scaler, microbatches,
        loss_function, accumulation_steps):
    optimizer.zero_grad(set_to_none=True)
    for inputs, targets in microbatches:
        with torch.autocast(device_type="cuda", dtype=torch.float16):
            outputs = model(inputs)
            loss = loss_function(outputs, targets)
            normalized_loss = loss / accumulation_steps
        scaler.scale(normalized_loss).backward()
    scaler.unscale_(optimizer)
    scaler.step(optimizer)
    scaler.update()
    return scaler.get_scale()
```

Part 6 - Measurement, Validation, and Failure Analysis

Require exactly the intended number of microbatches or adjust normalization explicitly. Do not update scale or optimizer inside the window. Verify equivalence within tolerances and test overflow in an early microbatch.

Part 7 - Engineering Interpretation and Philosophy

Accumulation creates one logical optimizer transaction from several backward passes. Precision control must respect that transaction boundary.

19.22.15 Exercise 14

Exercise Title: Detect NaN and Infinite Gradients Before the Optimizer Step and Build a Per-Layer Numerical Failure Report

Part 1 - Exercise Contract and Why It Matters

Build a diagnostic report after unscale that locates NaN, positive and negative infinity, zeros, extreme magnitudes, and affected layers before the update.

Part 2 - System and Memory Mental Model

A global overflow flag protects the update but does not identify the source. Per-parameter aggregation connected to module names reveals whether failure originates in one layer and then propagates.

Part 3 - Quantitative Prediction Before Execution

Define warning thresholds from dtype range and reference gradient statistics. Predict which layer fails first in an injected experiment.

Part 4 - Experimental Design and Instrumentation

Unscale gradients, scan without modifying them, and record shape, dtype, finite counts, zero fraction, minimum nonzero magnitude, maximum magnitude, and norm per named parameter. Correlate with activations and loss.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def gradient_failure_report(model):
    report = {}
    for name, parameter in model.named_parameters():
        gradient = parameter.grad
        if gradient is None:
            report[name] = {"missing": True}
            continue
        finite = torch.isfinite(gradient)
        report[name] = {
            "missing": False,
            "nan": int(torch.isnan(gradient).sum()),
            "positive_infinity": int(torch.isposinf(gradient).sum()),
            "negative_infinity": int(torch.isneginf(gradient).sum()),
            "zero": int((gradient == 0).sum()),
            "finite": int(finite.sum()),
        }
    return report

def E14(model):
    return gradient_failure_report(model)
```

Part 6 - Measurement, Validation, and Failure Analysis

Run the report after unscale and before any clipping that could hide magnitude. Avoid retaining graphs or full gradient copies unnecessarily. Verify counts on injected NaN and infinity patterns.

Part 7 - Engineering Interpretation and Philosophy

Protection answers whether a step is safe; diagnostics answer why it is unsafe. Both are required for engineering a robust training system.

19.22.16 Exercise 15

Exercise Title: Compare float16 with Dynamic Gradient Scaling Against bfloat16 Training Without Scaling Under the Same CUDA Workload

Part 1 - Exercise Contract and Why It Matters

Compare float16 autocast plus dynamic scaling with bfloat16 autocast without scaling under identical workload, memory budget, and acceptance criteria.

Part 2 - System and Memory Mental Model

float16 benefits from scaling because of narrow exponent range. bfloat16 often represents small gradients without scaling, but has coarser precision. Hardware kernel support and accumulation policy influence speed and error.

Part 3 - Quantitative Prediction Before Execution

Predict scale behavior, overflow rate, gradient zero rate, memory equality, throughput, and reference error for both modes. Identify workloads that favor range versus precision.

Part 4 - Experimental Design and Instrumentation

Replay cloned state and batch order for multiple seeds. Record accepted updates, scale history, non-finite and zero gradients, loss, gradient error, peak VRAM, runtime, and final metric.

Part 5 - PyTorch/CUDA Implementation

```
import torch

def E15(float16_trial, bfloat16_trial):
    float16_scaler = torch.amp.GradScaler("cuda")
    float16_result = float16_trial(float16_scaler)
    bfloat16_result = bfloat16_trial()
    return {
        "float16_with_scaling": float16_result,
        "bfloat16_without_scaling": bfloat16_result,
    }
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify native bfloat16 capability and identical operation coverage. Compare accepted rather than attempted updates and use float32 reference evidence. Report unsupported kernels and precision-sensitive divergence.

Part 7 - Engineering Interpretation and Philosophy

Scaling compensates for a format's range; choosing bfloat16 changes the format itself. The better system is determined by stability, accuracy, and achieved hardware performance together.

19.22.17 Exercise 16

Exercise Title: Measure the VRAM and Runtime Overhead of Gradient Scaling and Determine Whether Its Memory Cost Is Significant

Part 1 - Exercise Contract and Why It Matters

Isolate the memory and latency overhead of GradScaler from autocast and the model workload. Measure scaler state, finite checks, unscale, skipped-step handling, and synchronization effects.

Part 2 - System and Memory Mental Model

GradScaler stores little persistent state, but unscale and non-finite checks traverse gradients and launch kernels. The relative overhead depends on model size, number of gradient tensors, kernel launch cost, and whether the step is compute-bound.

Part 3 - Quantitative Prediction Before Execution

Estimate persistent bytes and bytes read or written during unscale. Predict overhead for many tiny parameters versus a few large tensors and for successful versus overflow steps.

Part 4 - Experimental Design and Instrumentation

Benchmark identical autocast steps with scaling disabled and enabled after warm-up. Record peak allocated and reserved VRAM, memory snapshots, kernel time, synchronized step latency distribution, throughput, and number of gradient tensors.

Part 5 - PyTorch/CUDA Implementation

```
import time
import torch

def timed_scaled_step(run_step, scaler):
    torch.cuda.reset_peak_memory_stats()
    torch.cuda.synchronize()
    start = time.perf_counter()
    run_step(scaler)
    torch.cuda.synchronize()
    return {
        "seconds": time.perf_counter() - start,
        "peak_allocated": torch.cuda.max_memory_allocated(),
        "peak_reserved": torch.cuda.max_memory_reserved(),
        "scale": scaler.get_scale(),
    }

def E16(run_step):
    scaler = torch.amp.GradScaler("cuda")
    return timed_scaled_step(run_step, scaler)
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not include first-use initialization only in the scaled branch. Run repeated synchronized trials and report median and tail latency. Separate allocator noise from persistent scaler state and profiler overhead from production timing.

Part 7 - Engineering Interpretation and Philosophy

A safety mechanism should be measured, but its value is not judged by zero cost. The relevant question is whether overhead is small relative to prevented numerical failure and unlocked float16 throughput.

19.22.18 Exercise 17

Exercise Title: Implement a Minimal Dynamic Loss Scaler from Scratch That Detects Overflow and Adjusts the Scale Without Using GradScaler

Part 1 - Exercise Contract and Why It Matters

Implement scale, unscale, finite detection, conditional optimizer step, growth tracking, and backoff without calling GradScaler. Keep state transitions explicit and auditable.

Part 2 - System and Memory Mental Model

The scaler multiplies loss before backward, divides gradients once, checks all gradients, commits only finite updates, grows after a success interval, and backs off immediately after failure.

Part 3 - Quantitative Prediction Before Execution

Write the state transition table and predict scale history for a fixed finite-overflow schedule. Define lower and upper scale bounds and behavior for missing gradients.

Part 4 - Experimental Design and Instrumentation

Compare the custom scaler with GradScaler on deterministic injected failures. Verify parameter and optimizer state, scale timeline, skipped updates, clipping compatibility, and multiple optimizers.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class MinimalDynamicLossScaler:
    def __init__(self, initial_scale=65536.0,
                 growth_factor=2.0, backoff_factor=0.5,
                 growth_interval=2000):
        self.scale = initial_scale
        self.growth_factor = growth_factor
        self.backoff_factor = backoff_factor
        self.growth_interval = growth_interval
        self.successful_steps = 0

    def scale_loss(self, loss):
        return loss * self.scale

    def unscale_and_check(self, model):
        all_finite = True
        inverse_scale = 1.0 / self.scale
        for parameter in model.parameters():
            if parameter.grad is None:
                continue
            parameter.grad.mul_(inverse_scale)
            if not bool(torch.isfinite(parameter.grad).all()):
                all_finite = False
        return all_finite

    def update(self, successful):
        if not successful:
            self.scale *= self.backoff_factor
            self.successful_steps = 0
            return
        self.successful_steps += 1
```

```
        if self.successful_steps >= self.growth_interval:
            self.scale *= self.growth_factor
            self.successful_steps = 0

def E17():
    return MinimalDynamicLossScaler()
```

Part 6 - Measurement, Validation, and Failure Analysis

Prevent repeated unscale, clamp invalid scales, and ensure a failed step mutates nothing. Test sparse or missing gradients, multiple devices, gradient accumulation, exceptions, and optimizer closures as declared limitations.

Part 7 - Engineering Interpretation and Philosophy

Reimplementation exposes the controller's invariants. The goal is not replacing a mature library utility, but understanding the transactional and numerical responsibilities it fulfills.

19.22.19 Exercise 18

Exercise Title: Stress-Test the Custom Dynamic Loss Scaler with Sudden Loss Spikes, Long Stable Periods, Tiny Gradients, and Repeated Overflows

Part 1 - Exercise Contract and Why It Matters

Drive the custom scaler through adversarial regimes that exercise underflow rescue, rapid overflow recovery, long-term growth, repeated rejection, and transitions between regimes.

Part 2 - System and Memory Mental Model

A robust controller must avoid unbounded scale growth, prolonged underflow, oscillation around a failure threshold, and state corruption during consecutive skips. Nonstationary gradients expose weaknesses hidden by steady workloads.

Part 3 - Quantitative Prediction Before Execution

Predict the scale trajectory for each scripted regime and define maximum recovery steps, allowed skip rate, scale bounds, and gradient-health targets.

Part 4 - Experimental Design and Instrumentation

Create a deterministic schedule of tiny, normal, spike, and repeated-overflow losses. Record scale, success counter, gradient extrema, zero and non-finite counts, accepted update, parameter checksum, and recovery latency every step.

Part 5 - PyTorch/CUDA Implementation

```
def E18(training_step, scaler, regimes):
    history = []
    for step_index, regime in enumerate(regimes):
        result = training_step(scaler, regime)
        history.append({
            "step": step_index,
            "regime": regime,
```

```
            "scale": scaler.scale,
            "successful_steps": scaler.successful_steps,
            "result": result,
        })
    return history
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify scripted magnitudes at the actual gradient, not only the loss. Test boundary conditions for scale floor and ceiling, consecutive failures, and recovery after optimizer state remains unchanged. Preserve any oscillatory trace.

Part 7 - Engineering Interpretation and Philosophy

Stress testing asks whether the controller remains safe when its assumptions stop being stationary. Robustness is demonstrated by bounded state, valid updates, and explainable recovery.

19.22.20 Exercise 19

Exercise Title: Build and Benchmark a Numerically Robust Mixed-Precision PyTorch Training Loop with Automatic Scaling, Overflow Recovery, Gradient Clipping, and Detailed Scale Diagnostics

Part 1 - Exercise Contract and Why It Matters

Integrate autocast, GradScaler, accumulation support, unscale-before-clipping, non-finite diagnostics, conditional updates, scale telemetry, and reproducible benchmarking into one training loop.

Part 2 - System and Memory Mental Model

The loop is a numerical transaction: forward and scaled backward create a candidate update; unscale, diagnostics, and clipping validate and transform it; scaler.step commits only when safe; scaler.update adjusts future range. Measurement must not alter ordering.

Part 3 - Quantitative Prediction Before Execution

Define memory, throughput, gradient norm, error, skip-rate, scale-stability, and convergence budgets. Predict steady scale and recovery behavior for normal and injected-stress phases.

Part 4 - Experimental Design and Instrumentation

Benchmark float32, float16 without scaling, and the robust loop from identical states. Report peak allocated and reserved VRAM, latency distribution, throughput, loss, reference error, clipping norms, non-finite locations, scale history, skipped updates, and final metric.

Part 5 - PyTorch/CUDA Implementation

```
import torch

class RobustMixedPrecisionTrainer:
    def __init__(self, model, optimizer, maximum_gradient_norm):
        self.model = model
```

```
        self.optimizer = optimizer
        self.maximum_gradient_norm = maximum_gradient_norm
        self.scaler = torch.amp.GradScaler("cuda")
        self.history = []

    def step(self, inputs, targets, loss_function):
        self.optimizer.zero_grad(set_to_none=True)
        with torch.autocast(device_type="cuda", dtype=torch.float16):
            outputs = self.model(inputs)
            loss = loss_function(outputs, targets)
        scale_before = self.scaler.get_scale()
        self.scaler.scale(loss).backward()
        self.scaler.unscale_(self.optimizer)
        gradient_norm = torch.nn.utils.clip_grad_norm_(
            self.model.parameters(), self.maximum_gradient_norm)
        self.scaler.step(self.optimizer)
        self.scaler.update()
        scale_after = self.scaler.get_scale()
        record = {
            "loss": float(loss.detach()),
            "gradient_norm": float(gradient_norm),
            "scale_before": scale_before,
            "scale_after": scale_after,
        }
        self.history.append(record)
        return record

def E19(model, optimizer, maximum_gradient_norm):
    return RobustMixedPrecisionTrainer(
        model, optimizer, maximum_gradient_norm)
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate parameter and optimizer state after every injected overflow, detect non-finite clipping norms, and confirm unscale occurs exactly once. Warm up before timing, compare accepted updates, and document API version, hardware, seeds, tolerances, and known limitations.

Part 7 - Engineering Interpretation and Philosophy

A robust training loop combines capacity and speed with evidence that every committed update is numerically admissible. Diagnostics, recovery, and reproducibility are part of the implementation, not optional monitoring.

19.23 DMM21

Chapter Title: Gradient Accumulation and Effective Batch Scaling
Specialization: Dynamic Memory Management and CUDA Systems

19.23.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Micro-Batches, Effective Batch Size, Gradient Accumulation, and Peak VRAM Reduction

Part 1 - Exercise Contract and Why It Matters

Construct a complete lifecycle for one logical optimizer update: divide the effective batch into micro-batches, execute forward and backward for each micro-batch, retain parameter gradients across the accumulation window, normalize the contribution correctly, perform one optimizer update, and then clear gradients. The final report must distinguish a micro-step from an optimizer step and a processed sample from an accepted update.

The learner must produce an executable experiment, a measurement table, and a short engineering conclusion. The experiment must preserve enough evidence to reconstruct which samples entered each optimizer update, which configuration was active, and whether the claimed memory or numerical behavior was actually observed. A result without controls, units, seeds, or a declared reference is incomplete.

Part 2 - System and Memory Mental Model

Gradient accumulation reduces the activation working set because only one micro-batch performs forward and backward at a time. Parameters, gradients, optimizer state, and many persistent workspaces do not shrink. Gradient tensors normally remain allocated across the window, while the autograd graph for a completed micro-batch should be released unless the program accidentally retains it. Consequently, accumulation changes the variable activation term but not the entire VRAM footprint.

Reason separately about the CUDA allocator, live tensor storage, autograd graph lifetime, gradient-buffer lifetime, and optimizer-state lifetime. Current allocated memory describes live allocations known to PyTorch; reserved memory also includes cached allocator blocks. Neither value alone identifies tensor ownership, so phase measurements and component accounting must accompany peak statistics.

Part 3 - Quantitative Prediction Before Execution

Let micro-batch size be b, accumulation steps be K, and distributed world size be W. The global effective batch size is

$$B_{\text{eff}} = bKW.$$

For a mean-reduced loss and equally sized micro-batches, the target gradient is

$$g = \frac{1}{K} \sum_{k=1}^{K} g_k.$$

A first-order memory model is

$$M_{\text{peak}}(b) \approx M_{\text{fixed}} + M_{\text{activation}}(b) + M_{\text{workspace}}(b),$$

where parameters, gradients, and optimizer state belong primarily to M_{fixed}.

Before execution, calculate the effective batch, expected optimizer-update count, activation payload, gradient payload, and a lower bound for peak VRAM. Predict which allocations survive between micro-steps and whether the final micro-step should have the same peak as the first. Include the expected throughput penalty caused by repeated launches and reduced kernel occupancy.

Write the prediction before running CUDA. Every predicted quantity must have a unit, a scope, and an assumption. At minimum, predict effective samples per optimizer update, optimizer-update count, peak allocated VRAM, peak reserved VRAM, synchronized update latency, throughput, and the gradient statistic that will decide correctness.

Part 4 - Experimental Design and Instrumentation

Use one CUDA model, one deterministic batch, and cloned initial model and optimizer state. Compare a full-batch update with several micro-batch decompositions that preserve the same effective batch. Record allocated and reserved VRAM at forward peak, after backward, before optimizer step, after optimizer step, and after zeroing gradients. Record step time, samples per second, gradient norms, parameter deltas, and graph-retention evidence.

Use fixed seeds, identical initial model and optimizer states, the same ordered samples, the same precision policy, and the same model mode for controlled comparisons. Warm up CUDA before timing, synchronize only at deliberate measurement boundaries, reset peak statistics immediately before the measured region, and report median together with a tail statistic across repeated trials. Keep profiler runs separate from final latency measurements.

Part 5 - PyTorch/CUDA Implementation

Complete the following structure. The missing operations are intentionally represented by extttEllipsis; implement them from the contract, predictions, and measurement requirements above. Do not replace the experiment with a library trainer that hides accumulation boundaries or memory observations.

```
import torch

def calculate_effective_batch_size(
        micro_batch_size,
        accumulation_steps,
        world_size):
    effective_batch_size = ...
    return effective_batch_size

def run_accumulation_window(
        model,
        optimizer,
        micro_batches,
        loss_function,
        accumulation_steps):
    optimizer.zero_grad(set_to_none=True)

    for micro_step_index in range(accumulation_steps):
        inputs, targets = micro_batches[micro_step_index]

        outputs = ...
        loss = ...
        normalized_loss = ...

        # Accumulate gradients without updating the parameters yet.
        ...
```

```
    # Perform exactly one optimizer update for the complete window.
    ...

    return ...

def E00(
        model,
        optimizer,
        micro_batches,
        loss_function,
        accumulation_steps,
        world_size):
    effective_batch_size = ...
    window_result = ...
    return effective_batch_size, window_result
```

Part 6 - Measurement, Validation, and Failure Analysis

Detect loss-normalization mistakes, premature zeroing, accidental optimizer steps inside the window, retained graphs, variable micro-batch sizes, and differences introduced by stochastic layers or batch-dependent normalization. A capacity result is invalid unless the final gradients and parameter update are compared with an appropriate reference.

Validate loss, every named parameter gradient, updated parameters, and optimizer state against the declared reference using dtype-appropriate absolute and relative tolerances. Record missing gradients, zero fractions, non-finite values, OOM stage, and unsuccessful configurations instead of deleting them. Separate theoretical prediction, measured value, absolute error, relative error, and interpretation in the final report.

Part 7 - Engineering Interpretation and Philosophy

Gradient accumulation exchanges time and launch efficiency for a smaller activation working set. It does not create free memory and it does not automatically reproduce large-batch semantics; those properties must be established by accounting, normalization, and measured equivalence.

The final recommendation must state which resource was saved, which cost increased, where the critical path moved, and under which workload and hardware the conclusion is valid. Preserve negative results and limitations: a memory technique that fits a larger workload but violates numerical equivalence, throughput requirements, or operational headroom has not solved the complete engineering problem.

19.23.2 Exercise 01

Exercise Title: Train with a Full Batch and Measure Peak VRAM, Step Time, and Gradient Statistics as a Baseline

Part 1 - Exercise Contract and Why It Matters

Establish a full-batch reference that later accumulation experiments must match. The baseline must include a complete training transaction—gradient clearing, forward, loss, backward,

gradient inspection, optimizer update, and synchronized measurement—rather than a forward-only memory number.

The learner must produce an executable experiment, a measurement table, and a short engineering conclusion. The experiment must preserve enough evidence to reconstruct which samples entered each optimizer update, which configuration was active, and whether the claimed memory or numerical behavior was actually observed. A result without controls, units, seeds, or a declared reference is incomplete.

Part 2 - System and Memory Mental Model

The full batch maximizes simultaneous activation residency and often gives efficient large kernels. Its peak may occur during forward, backward, or optimizer initialization depending on the model and optimizer. CUDA execution is asynchronous, allocator reservations persist, and the first optimizer update can allocate state that later steps reuse.

Reason separately about the CUDA allocator, live tensor storage, autograd graph lifetime, gradient-buffer lifetime, and optimizer-state lifetime. Current allocated memory describes live allocations known to PyTorch; reserved memory also includes cached allocator blocks. Neither value alone identifies tensor ownership, so phase measurements and component accounting must accompany peak statistics.

Part 3 - Quantitative Prediction Before Execution

For batch size B, define throughput and update gradient norm as

$$T = \frac{B}{t_{\text{step}}}, \qquad \|g\|_2 = \sqrt{\sum_i \|g_i\|_2^2}.$$

Record both peak allocated memory $M_{\text{alloc}}^{\max}$ and peak reserved memory $M_{\text{reserved}}^{\max}$ because they describe live tensors and allocator capacity differently.

Compute parameter, gradient, optimizer-state, input, and approximate activation bytes before execution. Predict the peak stage and the difference between the first measured update and steady-state updates. Define expected gradient range, finite-value requirements, and timing variability.

Write the prediction before running CUDA. Every predicted quantity must have a unit, a scope, and an assumption. At minimum, predict effective samples per optimizer update, optimizer-update count, peak allocated VRAM, peak reserved VRAM, synchronized update latency, throughput, and the gradient statistic that will decide correctness.

Part 4 - Experimental Design and Instrumentation

Run isolated warm-up steps, restore the documented measurement state, reset peak statistics, and execute repeated synchronized steps. Report median and tail latency, samples per second, loss, per-layer gradient norm and zero fraction, maximum gradient magnitude, parameter-update norm, and peak allocated and reserved VRAM.

Use fixed seeds, identical initial model and optimizer states, the same ordered samples, the same precision policy, and the same model mode for controlled comparisons. Warm up CUDA before timing, synchronize only at deliberate measurement boundaries, reset peak statistics immediately before the measured region, and report median together with a tail statistic across repeated trials. Keep profiler runs separate from final latency measurements.

Part 5 - PyTorch/CUDA Implementation

Complete the following structure. The missing operations are intentionally represented by extttEllipsis; implement them from the contract, predictions, and measurement requirements above. Do not replace the experiment with a library trainer that hides accumulation boundaries or memory observations.

```
import time
import torch

def collect_gradient_statistics(model):
    statistics = {}

    for parameter_name, parameter in model.named_parameters():
        if parameter.grad is None:
            continue

        gradient = parameter.grad
        statistics[parameter_name] = {
            "norm": ...,
            "maximum_absolute_value": ...,
            "zero_fraction": ...,
            "is_finite": ...,
        }

    return statistics

def E01(model, optimizer, inputs, targets, loss_function):
    optimizer.zero_grad(set_to_none=True)
    torch.cuda.reset_peak_memory_stats()

    torch.cuda.synchronize()
    start_time = time.perf_counter()

    outputs = ...
    loss = ...
    ...

    gradient_statistics = ...
    ...

    torch.cuda.synchronize()
    step_time = ...

    return {
        "loss": ...,
        "step_time": step_time,
        "peak_allocated": ...,
        "peak_reserved": ...,
        "gradient_statistics": gradient_statistics,
    }
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not compare a cold full-batch step with warmed accumulation. Verify that all samples and targets are identical and that the baseline fits without hidden retry or checkpointing. Preserve OOM as a result if the requested full batch cannot execute.

Validate loss, every named parameter gradient, updated parameters, and optimizer state against the declared reference using dtype-appropriate absolute and relative tolerances. Record missing gradients, zero fractions, non-finite values, OOM stage, and unsuccessful configurations instead of deleting them. Separate theoretical prediction, measured value, absolute error, relative error, and interpretation in the final report.

Part 7 - Engineering Interpretation and Philosophy

A baseline is the semantic and systems control for the chapter. Every claimed memory saving or equivalence result is meaningful only relative to a fully specified reference update.

The final recommendation must state which resource was saved, which cost increased, where the critical path moved, and under which workload and hardware the conclusion is valid. Preserve negative results and limitations: a memory technique that fits a larger workload but violates numerical equivalence, throughput requirements, or operational headroom has not solved the complete engineering problem.

19.23.3 Exercise 02

Exercise Title: Split the Same Batch into Micro-Batches and Accumulate Gradients Before One Optimizer Step

Part 1 - Exercise Contract and Why It Matters

Partition the exact full batch into an ordered sequence of micro-batches and accumulate their contributions into the same parameter gradient buffers. Execute one optimizer update only after every sample has contributed, and preserve a trace that maps sample ranges to micro-steps.

The learner must produce an executable experiment, a measurement table, and a short engineering conclusion. The experiment must preserve enough evidence to reconstruct which samples entered each optimizer update, which configuration was active, and whether the claimed memory or numerical behavior was actually observed. A result without controls, units, seeds, or a declared reference is incomplete.

Part 2 - System and Memory Mental Model

Calling backward repeatedly adds into existing parameter.grad tensors. Each micro-batch graph can be freed after its backward, while gradient buffers persist. The data partition, reduction convention, and loss normalization determine whether the accumulated gradient represents a sum or mean over the original full batch.

Reason separately about the CUDA allocator, live tensor storage, autograd graph lifetime, gradient-buffer lifetime, and optimizer-state lifetime. Current allocated memory describes live allocations known to PyTorch; reserved memory also includes cached allocator blocks. Neither value alone identifies tensor ownership, so phase measurements and component accounting must accompany peak statistics.

Part 3 - Quantitative Prediction Before Execution

If the full-batch objective is a sample mean and every micro-batch has size b, then

$$L_{\text{full}} = \frac{1}{K}\sum_{k=1}^{K} L_k, \qquad \nabla L_{\text{full}} = \sum_{k=1}^{K} \nabla\left(\frac{L_k}{K}\right).$$

For unequal micro-batch sizes b_k, weighting must follow b_k/B rather than blindly using $1/K$.

Predict the gradient norm after every micro-step and the final parameter delta. Estimate peak activation memory from the largest micro-batch, while keeping parameter, gradient, and optimizer memory fixed. State the number of forward and backward launches per optimizer update.

Write the prediction before running CUDA. Every predicted quantity must have a unit, a scope, and an assumption. At minimum, predict effective samples per optimizer update, optimizer-update count, peak allocated VRAM, peak reserved VRAM, synchronized update latency, throughput, and the gradient statistic that will decide correctness.

Part 4 - Experimental Design and Instrumentation

Slice a stored full batch without shuffling or augmentation changes. Log micro-batch indices, sizes, loss numerator, normalization weight, cumulative gradient norm, peak VRAM, and graph release after each backward. Compare the final state with the full-batch baseline.

Use fixed seeds, identical initial model and optimizer states, the same ordered samples, the same precision policy, and the same model mode for controlled comparisons. Warm up CUDA before timing, synchronize only at deliberate measurement boundaries, reset peak statistics immediately before the measured region, and report median together with a tail statistic across repeated trials. Keep profiler runs separate from final latency measurements.

Part 5 - PyTorch/CUDA Implementation

Complete the following structure. The missing operations are intentionally represented by extttEllipsis; implement them from the contract, predictions, and measurement requirements above. Do not replace the experiment with a library trainer that hides accumulation boundaries or memory observations.

```
import torch

def create_micro_batches(inputs, targets, micro_batch_size):
    micro_batches = []
    number_of_samples = ...

    start_index = 0
    while start_index < number_of_samples:
        end_index = ...
        micro_inputs = ...
        micro_targets = ...
        micro_batches.append((micro_inputs, micro_targets))
        start_index = end_index

    return micro_batches

def E02(
        model,
```

```
        optimizer,
        inputs,
        targets,
        loss_function,
        micro_batch_size):
    micro_batches = ...
    optimizer.zero_grad(set_to_none=True)

    for micro_inputs, micro_targets in micro_batches:
        outputs = ...
        micro_loss = ...
        normalized_loss = ...
        ...

    ...
    return ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Check that no optimizer step or zero_grad occurs inside the accumulation window. Detect dropped, repeated, or reordered samples and verify that model state such as running statistics does not invalidate equivalence.

Validate loss, every named parameter gradient, updated parameters, and optimizer state against the declared reference using dtype-appropriate absolute and relative tolerances. Record missing gradients, zero fractions, non-finite values, OOM stage, and unsuccessful configurations instead of deleting them. Separate theoretical prediction, measured value, absolute error, relative error, and interpretation in the final report.

Part 7 - Engineering Interpretation and Philosophy

Accumulation is repeated construction of one logical gradient. The correctness boundary is the optimizer update, not the individual micro-step.

The final recommendation must state which resource was saved, which cost increased, where the critical path moved, and under which workload and hardware the conclusion is valid. Preserve negative results and limitations: a memory technique that fits a larger workload but violates numerical equivalence, throughput requirements, or operational headroom has not solved the complete engineering problem.

19.23.4 Exercise 03

Exercise Title: Prove That Correctly Scaled Gradient Accumulation Matches the Gradients of an Equivalent Full Batch

Part 1 - Exercise Contract and Why It Matters

Produce a numerical equivalence proof between one full-batch backward pass and an accumulation window that contains the same samples. The proof must cover loss, every named gradient, global norm, parameter update, and optimizer state—not only a single output tensor.

The learner must produce an executable experiment, a measurement table, and a short engineering conclusion. The experiment must preserve enough evidence to reconstruct which samples entered each optimizer update, which configuration was active, and whether the claimed

memory or numerical behavior was actually observed. A result without controls, units, seeds, or a declared reference is incomplete.

Part 2 - System and Memory Mental Model

Linearity of differentiation gives equivalence when the objective decomposes over samples and normalization weights match. Floating-point reduction order can produce small differences. BatchNorm, dropout, random augmentation, token-count normalization, and other batch-coupled behavior can create legitimate semantic differences even with correct arithmetic scaling.

Reason separately about the CUDA allocator, live tensor storage, autograd graph lifetime, gradient-buffer lifetime, and optimizer-state lifetime. Current allocated memory describes live allocations known to PyTorch; reserved memory also includes cached allocator blocks. Neither value alone identifies tensor ownership, so phase measurements and component accounting must accompany peak statistics.

Part 3 - Quantitative Prediction Before Execution

For sample losses ℓ_j and total batch size B,

$$L = \frac{1}{B}\sum_{j=1}^{B}\ell_j, \qquad \nabla_\theta L = \frac{1}{B}\sum_{j=1}^{B}\nabla_\theta \ell_j.$$

For micro-batch k with b_k samples and mean loss L_k,

$$\nabla_\theta L = \sum_{k=1}^{K}\frac{b_k}{B}\nabla_\theta L_k.$$

Calculate the expected weight for every micro-batch and define absolute and relative tolerances by dtype. Predict which parameters can differ most from reduction order and which model features must be disabled or controlled for an equivalence test.

Write the prediction before running CUDA. Every predicted quantity must have a unit, a scope, and an assumption. At minimum, predict effective samples per optimizer update, optimizer-update count, peak allocated VRAM, peak reserved VRAM, synchronized update latency, throughput, and the gradient statistic that will decide correctness.

Part 4 - Experimental Design and Instrumentation

Clone initial model and optimizer state. Execute full-batch and accumulation branches with identical inputs, RNG state, model mode, and numerical policy. Generate a named comparison table containing maximum absolute error, maximum relative error, norm difference, allclose status, and finite status for every gradient and updated parameter.

Use fixed seeds, identical initial model and optimizer states, the same ordered samples, the same precision policy, and the same model mode for controlled comparisons. Warm up CUDA before timing, synchronize only at deliberate measurement boundaries, reset peak statistics immediately before the measured region, and report median together with a tail statistic across repeated trials. Keep profiler runs separate from final latency measurements.

Part 5 - PyTorch/CUDA Implementation

Complete the following structure. The missing operations are intentionally represented by extttEllipsis; implement them from the contract, predictions, and measurement requirements above. Do not replace the experiment with a library trainer that hides accumulation boundaries or memory observations.

```
import copy
import torch

def clone_named_gradients(model):
    gradients = {}
    for parameter_name, parameter in model.named_parameters():
        if parameter.grad is None:
            gradients[parameter_name] = None
        else:
            gradients[parameter_name] = ...
    return gradients

def compare_named_gradients(
        reference_gradients,
        accumulated_gradients,
        absolute_tolerance,
        relative_tolerance):
    comparison = {}

    for parameter_name in reference_gradients:
        reference = reference_gradients[parameter_name]
        candidate = accumulated_gradients[parameter_name]

        # Compare missing gradients, shapes, values, and norms.
        comparison[parameter_name] = ...

    return comparison

def E03(model, optimizer, full_batch, micro_batches, loss_function):
    reference_model = ...
    accumulated_model = ...

    reference_gradients = ...
    accumulated_gradients = ...

    return compare_named_gradients(
        reference_gradients,
        accumulated_gradients,
        absolute_tolerance=...,
        relative_tolerance=...)
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not hide missing gradients or compare references that have already been mutated. Explain every tolerance failure and repeat in float64 or a simpler deterministic model to distinguish algorithmic error from rounding order.

Validate loss, every named parameter gradient, updated parameters, and optimizer state against the declared reference using dtype-appropriate absolute and relative tolerances. Record missing gradients, zero fractions, non-finite values, OOM stage, and unsuccessful configurations

instead of deleting them. Separate theoretical prediction, measured value, absolute error, relative error, and interpretation in the final report.

Part 7 - Engineering Interpretation and Philosophy

Equivalence is a claim with explicit assumptions. Proving it teaches where gradient accumulation is mathematically exact, numerically approximate, or semantically different.

The final recommendation must state which resource was saved, which cost increased, where the critical path moved, and under which workload and hardware the conclusion is valid. Preserve negative results and limitations: a memory technique that fits a larger workload but violates numerical equivalence, throughput requirements, or operational headroom has not solved the complete engineering problem.

19.23.5 Exercise 04

Exercise Title: Reproduce Incorrect Gradient Magnitudes by Forgetting to Divide the Loss by the Accumulation Factor

Part 1 - Exercise Contract and Why It Matters

Deliberately omit loss normalization and demonstrate how the final gradient and parameter update are amplified. Preserve the incorrect branch as a regression test that catches this common accumulation error.

The learner must produce an executable experiment, a measurement table, and a short engineering conclusion. The experiment must preserve enough evidence to reconstruct which samples entered each optimizer update, which configuration was active, and whether the claimed memory or numerical behavior was actually observed. A result without controls, units, seeds, or a declared reference is incomplete.

Part 2 - System and Memory Mental Model

Backward adds each micro-batch gradient. If every micro-batch reports a mean loss and none is divided or weighted, the buffer contains the sum of K micro-batch means rather than the full-batch mean. Optimizer nonlinearities, clipping, momentum, and adaptive state can make the final damage more complex than a simple K-times parameter delta.

Reason separately about the CUDA allocator, live tensor storage, autograd graph lifetime, gradient-buffer lifetime, and optimizer-state lifetime. Current allocated memory describes live allocations known to PyTorch; reserved memory also includes cached allocator blocks. Neither value alone identifies tensor ownership, so phase measurements and component accounting must accompany peak statistics.

Part 3 - Quantitative Prediction Before Execution

With equal micro-batches, the incorrect gradient is

$$g_{\text{wrong}} = \sum_{k=1}^{K} g_k = K g_{\text{correct}}.$$

For plain SGD without clipping or momentum,

$$\Delta\theta_{\text{wrong}} = K \Delta\theta_{\text{correct}}.$$

This proportional update relationship need not hold for Adam, clipping, or weight decay interactions.

Predict gradient norm ratio, clipping behavior, and SGD parameter-delta ratio before running. For an adaptive optimizer, predict which state tensors change and avoid claiming a universal K-times update.

Write the prediction before running CUDA. Every predicted quantity must have a unit, a scope, and an assumption. At minimum, predict effective samples per optimizer update, optimizer-update count, peak allocated VRAM, peak reserved VRAM, synchronized update latency, throughput, and the gradient statistic that will decide correctness.

Part 4 - Experimental Design and Instrumentation

Run full-batch, correctly normalized accumulation, and unnormalized accumulation from identical state. Compare per-micro-step cumulative norm, final gradient ratio, clip coefficient, parameter delta, optimizer state, and next-step loss.

Use fixed seeds, identical initial model and optimizer states, the same ordered samples, the same precision policy, and the same model mode for controlled comparisons. Warm up CUDA before timing, synchronize only at deliberate measurement boundaries, reset peak statistics immediately before the measured region, and report median together with a tail statistic across repeated trials. Keep profiler runs separate from final latency measurements.

Part 5 - PyTorch/CUDA Implementation

Complete the following structure. The missing operations are intentionally represented by extttEllipsis; implement them from the contract, predictions, and measurement requirements above. Do not replace the experiment with a library trainer that hides accumulation boundaries or memory observations.

```
import torch

def run_accumulation_branch(
        model,
        optimizer,
        micro_batches,
        loss_function,
        accumulation_steps,
        normalize_loss):
    optimizer.zero_grad(set_to_none=True)

    for inputs, targets in micro_batches:
        outputs = ...
        loss = ...

        if normalize_loss:
            backward_loss = ...
        else:
            backward_loss = ...

        ...

    gradients = ...
    parameter_state_before = ...
    ...
```

```
    parameter_state_after = ...

    return gradients, parameter_state_before, parameter_state_after

def E04(model, optimizer, micro_batches, loss_function, accumulation_steps):
    correct_result = ...
    incorrect_result = ...
    return correct_result, incorrect_result
```

Part 6 - Measurement, Validation, and Failure Analysis

Ensure the wrong result is caused only by missing normalization. Do not simultaneously change learning rate, scheduler, batch order, or zeroing policy. Check zero and near-zero reference gradients before computing ratios.

Validate loss, every named parameter gradient, updated parameters, and optimizer state against the declared reference using dtype-appropriate absolute and relative tolerances. Record missing gradients, zero fractions, non-finite values, OOM stage, and unsuccessful configurations instead of deleting them. Separate theoretical prediction, measured value, absolute error, relative error, and interpretation in the final report.

Part 7 - Engineering Interpretation and Philosophy

A deliberately reproduced failure turns a silent scaling mistake into a measurable invariant that future training-loop changes can test automatically.

The final recommendation must state which resource was saved, which cost increased, where the critical path moved, and under which workload and hardware the conclusion is valid. Preserve negative results and limitations: a memory technique that fits a larger workload but violates numerical equivalence, throughput requirements, or operational headroom has not solved the complete engineering problem.

19.23.6 Exercise 05

Exercise Title: Compare Peak VRAM Across Multiple Micro-Batch Sizes While Keeping Effective Batch Size Constant

Part 1 - Exercise Contract and Why It Matters

Sweep micro-batch size and accumulation steps while holding the effective batch constant. Build a memory curve that separates fixed training state from micro-batch-dependent activations and workspaces.

The learner must produce an executable experiment, a measurement table, and a short engineering conclusion. The experiment must preserve enough evidence to reconstruct which samples entered each optimizer update, which configuration was active, and whether the claimed memory or numerical behavior was actually observed. A result without controls, units, seeds, or a declared reference is incomplete.

Part 2 - System and Memory Mental Model

Using fewer samples per micro-step usually reduces activation peaks. Allocator bins, workspaces, padding, and shape-dependent algorithm selection can still make the curve non-smooth. Gradient buffers remain full-size, and repeated micro-steps do not multiply peak activation memory when graphs are released correctly.

Reason separately about the CUDA allocator, live tensor storage, autograd graph lifetime, gradient-buffer lifetime, and optimizer-state lifetime. Current allocated memory describes live allocations known to PyTorch; reserved memory also includes cached allocator blocks. Neither value alone identifies tensor ownership, so phase measurements and component accounting must accompany peak statistics.

Part 3 - Quantitative Prediction Before Execution

For one device and fixed effective batch B, valid pairs satisfy

$$bK = B.$$

Measure the marginal memory slope

$$\alpha(b_1, b_2) = \frac{M_{\text{peak}}(b_2) - M_{\text{peak}}(b_1)}{b_2 - b_1}$$

rather than assuming perfect linearity.

List every valid factor pair, estimate activation bytes for each, and predict the fixed-memory intercept. State where throughput is expected to fall because micro-batches become too small.

Write the prediction before running CUDA. Every predicted quantity must have a unit, a scope, and an assumption. At minimum, predict effective samples per optimizer update, optimizer-update count, peak allocated VRAM, peak reserved VRAM, synchronized update latency, throughput, and the gradient statistic that will decide correctness.

Part 4 - Experimental Design and Instrumentation

Run every pair in an isolated, warmed trial with identical model, optimizer, samples, precision, and update count. Record allocated and reserved peaks by phase, throughput, total update latency, gradient equivalence, and allocator snapshot summary.

Use fixed seeds, identical initial model and optimizer states, the same ordered samples, the same precision policy, and the same model mode for controlled comparisons. Warm up CUDA before timing, synchronize only at deliberate measurement boundaries, reset peak statistics immediately before the measured region, and report median together with a tail statistic across repeated trials. Keep profiler runs separate from final latency measurements.

Part 5 - PyTorch/CUDA Implementation

Complete the following structure. The missing operations are intentionally represented by extttEllipsis; implement them from the contract, predictions, and measurement requirements above. Do not replace the experiment with a library trainer that hides accumulation boundaries or memory observations.

```
import torch

def build_valid_configurations(effective_batch_size):
    configurations = []

    for micro_batch_size in range(1, effective_batch_size + 1):
        if effective_batch_size % micro_batch_size != 0:
            continue

        accumulation_steps = ...
```

```
        configurations.append(
            (micro_batch_size, accumulation_steps))

    return configurations

def measure_configuration(configuration, training_trial):
    micro_batch_size, accumulation_steps = configuration
    torch.cuda.reset_peak_memory_stats()
    result = ...
    return {
        "micro_batch_size": micro_batch_size,
        "accumulation_steps": accumulation_steps,
        "peak_allocated": ...,
        "peak_reserved": ...,
        "training_result": result,
    }

def E05(effective_batch_size, training_trial):
    configurations = ...
    measurements = []

    for configuration in configurations:
        measurement = ...
        measurements.append(measurement)

    return measurements
```

Part 6 - Measurement, Validation, and Failure Analysis

Detect graph retention, changing sample counts, unequal final windows, and OOM recovery side effects. Repeat each configuration and report a safety margin rather than a single lowest peak.

Validate loss, every named parameter gradient, updated parameters, and optimizer state against the declared reference using dtype-appropriate absolute and relative tolerances. Record missing gradients, zero fractions, non-finite values, OOM stage, and unsuccessful configurations instead of deleting them. Separate theoretical prediction, measured value, absolute error, relative error, and interpretation in the final report.

Part 7 - Engineering Interpretation and Philosophy

The sweep exposes the real memory-time frontier. The best micro-batch is not automatically the smallest; it is the point that meets the budget with the least throughput cost.

The final recommendation must state which resource was saved, which cost increased, where the critical path moved, and under which workload and hardware the conclusion is valid. Preserve negative results and limitations: a memory technique that fits a larger workload but violates numerical equivalence, throughput requirements, or operational headroom has not solved the complete engineering problem.

19.23.7 Exercise 06

Exercise Title: Measure Throughput Loss as Micro-Batches Become Smaller and Kernel Launch Overhead Becomes More Significant

Part 1 - Exercise Contract and Why It Matters

Measure how reducing micro-batch size changes kernel efficiency, launch count, host overhead, GPU idle gaps, and samples per second for a fixed effective batch.

The learner must produce an executable experiment, a measurement table, and a short engineering conclusion. The experiment must preserve enough evidence to reconstruct which samples entered each optimizer update, which configuration was active, and whether the claimed memory or numerical behavior was actually observed. A result without controls, units, seeds, or a declared reference is incomplete.

Part 2 - System and Memory Mental Model

Small micro-batches reduce parallel work per kernel while increasing the number of forward and backward passes per optimizer update. Launch latency, Python dispatch, data transfer, and synchronization can occupy a larger fraction of time. Some kernels also lose Tensor Core alignment or choose different algorithms.

Reason separately about the CUDA allocator, live tensor storage, autograd graph lifetime, gradient-buffer lifetime, and optimizer-state lifetime. Current allocated memory describes live allocations known to PyTorch; reserved memory also includes cached allocator blocks. Neither value alone identifies tensor ownership, so phase measurements and component accounting must accompany peak statistics.

Part 3 - Quantitative Prediction Before Execution

For effective batch $B = bK$, define

$$T_{\text{update}}(b) = \sum_{k=1}^{K} (t_{f,k} + t_{b,k}) + t_{\text{opt}} + t_{\text{overhead}},$$

and

$$\text{throughput}(b) = \frac{B}{T_{\text{update}}(b)}.$$

Report GPU active time separately from synchronized wall time.

Predict launch-count growth and identify a micro-batch size below which occupancy or arithmetic intensity is likely to collapse. Estimate the memory saved per percentage point of throughput lost.

Write the prediction before running CUDA. Every predicted quantity must have a unit, a scope, and an assumption. At minimum, predict effective samples per optimizer update, optimizer-update count, peak allocated VRAM, peak reserved VRAM, synchronized update latency, throughput, and the gradient statistic that will decide correctness.

Part 4 - Experimental Design and Instrumentation

Profile selected configurations after warm-up. Record CPU and CUDA time, kernel count, median kernel duration, GPU idle gaps, data-loader time, optimizer time, synchronized update latency, and throughput. Use profiler runs for attribution and separate unprofiled runs for final performance.

Use fixed seeds, identical initial model and optimizer states, the same ordered samples, the same precision policy, and the same model mode for controlled comparisons. Warm up CUDA

before timing, synchronize only at deliberate measurement boundaries, reset peak statistics immediately before the measured region, and report median together with a tail statistic across repeated trials. Keep profiler runs separate from final latency measurements.

Part 5 - PyTorch/CUDA Implementation

Complete the following structure. The missing operations are intentionally represented by extttEllipsis; implement them from the contract, predictions, and measurement requirements above. Do not replace the experiment with a library trainer that hides accumulation boundaries or memory observations.

```
import time
import torch

def time_accumulation_configuration(
        training_update,
        micro_batch_size,
        accumulation_steps,
        effective_batch_size):
    torch.cuda.synchronize()
    start_time = time.perf_counter()

    update_result = ...

    torch.cuda.synchronize()
    update_time = ...
    throughput = ...

    return {
        "micro_batch_size": micro_batch_size,
        "accumulation_steps": accumulation_steps,
        "update_time": update_time,
        "throughput": throughput,
        "update_result": update_result,
    }

def E06(configurations, training_update, effective_batch_size):
    results = []
    for micro_batch_size, accumulation_steps in configurations:
        result = ...
        results.append(result)
    return results
```

Part 6 - Measurement, Validation, and Failure Analysis

Avoid timing asynchronous CUDA work without synchronization. Exclude compilation and autotuning transients, and ensure data loading does not become a different bottleneck across configurations.

Validate loss, every named parameter gradient, updated parameters, and optimizer state against the declared reference using dtype-appropriate absolute and relative tolerances. Record

missing gradients, zero fractions, non-finite values, OOM stage, and unsuccessful configurations instead of deleting them. Separate theoretical prediction, measured value, absolute error, relative error, and interpretation in the final report.

Part 7 - Engineering Interpretation and Philosophy

Accumulation spends execution efficiency to purchase memory capacity. Throughput attribution reveals exactly where that price is paid.

The final recommendation must state which resource was saved, which cost increased, where the critical path moved, and under which workload and hardware the conclusion is valid. Preserve negative results and limitations: a memory technique that fits a larger workload but violates numerical equivalence, throughput requirements, or operational headroom has not solved the complete engineering problem.

19.23.8 Exercise 07

Exercise Title: Determine the Largest Micro-Batch That Fits a Fixed VRAM Budget and Maximizes GPU Utilization

Part 1 - Exercise Contract and Why It Matters

Search for the largest repeatably safe micro-batch under a strict VRAM limit, then verify that it delivers better utilization or throughput than smaller feasible candidates.

The learner must produce an executable experiment, a measurement table, and a short engineering conclusion. The experiment must preserve enough evidence to reconstruct which samples entered each optimizer update, which configuration was active, and whether the claimed memory or numerical behavior was actually observed. A result without controls, units, seeds, or a declared reference is incomplete.

Part 2 - System and Memory Mental Model

The largest allocation that avoids one OOM is not necessarily operationally safe. Fragmentation, dynamic shapes, optimizer initialization, and asynchronous peaks require headroom. Feasibility and performance are separate filters.

Reason separately about the CUDA allocator, live tensor storage, autograd graph lifetime, gradient-buffer lifetime, and optimizer-state lifetime. Current allocated memory describes live allocations known to PyTorch; reserved memory also includes cached allocator blocks. Neither value alone identifies tensor ownership, so phase measurements and component accounting must accompany peak statistics.

Part 3 - Quantitative Prediction Before Execution

A candidate is admissible when

$$M_{\text{peak}}(b) + M_{\text{safety}} \leq M_{\text{budget}}.$$

For target effective batch B, its accumulation count is

$$K = \left\lceil \frac{B}{bW} \right\rceil,$$

with final-window weighting handled explicitly when equality is not exact.

Estimate the feasible upper bound from the memory model and choose a safety margin. Predict utilization and throughput ordering among nearby feasible sizes.

Write the prediction before running CUDA. Every predicted quantity must have a unit, a scope, and an assumption. At minimum, predict effective samples per optimizer update, optimizer-update count, peak allocated VRAM, peak reserved VRAM, synchronized update latency, throughput, and the gradient statistic that will decide correctness.

Part 4 - Experimental Design and Instrumentation

Use exponential growth to find a failure bracket, then a bounded search with isolated trials. Repeat the chosen size under representative dynamic inputs. Record peak VRAM, headroom, OOM stage, utilization proxy, throughput, and gradient validity.

Use fixed seeds, identical initial model and optimizer states, the same ordered samples, the same precision policy, and the same model mode for controlled comparisons. Warm up CUDA before timing, synchronize only at deliberate measurement boundaries, reset peak statistics immediately before the measured region, and report median together with a tail statistic across repeated trials. Keep profiler runs separate from final latency measurements.

Part 5 - PyTorch/CUDA Implementation

Complete the following structure. The missing operations are intentionally represented by extttEllipsis; implement them from the contract, predictions, and measurement requirements above. Do not replace the experiment with a library trainer that hides accumulation boundaries or memory observations.

```
import torch

def test_micro_batch_candidate(
        micro_batch_size,
        vram_budget,
        training_trial):
    try:
        torch.cuda.reset_peak_memory_stats()
        trial_result = ...
        peak_vram = ...
        fits_budget = ...
        return {
            "success": True,
            "fits_budget": fits_budget,
            "peak_vram": peak_vram,
            "result": trial_result,
        }
    except torch.cuda.OutOfMemoryError as error:
        return {
            "success": False,
            "fits_budget": False,
            "peak_vram": None,
            "error": str(error),
        }

def E07(candidate_sizes, vram_budget, training_trial):
    candidate_reports = []
    for micro_batch_size in candidate_sizes:
```

```
        report = ...
        candidate_reports.append(report)

    selected_candidate = ...
    return selected_candidate, candidate_reports
```

Part 6 - Measurement, Validation, and Failure Analysis

Recover from OOM in a clean trial rather than trusting a potentially damaged process state. Reject edge candidates that fail repeated execution or violate the safety margin.

Validate loss, every named parameter gradient, updated parameters, and optimizer state against the declared reference using dtype-appropriate absolute and relative tolerances. Record missing gradients, zero fractions, non-finite values, OOM stage, and unsuccessful configurations instead of deleting them. Separate theoretical prediction, measured value, absolute error, relative error, and interpretation in the final report.

Part 7 - Engineering Interpretation and Philosophy

The optimum is the largest safe working set that keeps the GPU productive, not the largest tensor allocation that survives once.

The final recommendation must state which resource was saved, which cost increased, where the critical path moved, and under which workload and hardware the conclusion is valid. Preserve negative results and limitations: a memory technique that fits a larger workload but violates numerical equivalence, throughput requirements, or operational headroom has not solved the complete engineering problem.

19.23.9 Exercise 08

Exercise Title: Build an Effective Batch Calculator from Micro-Batch Size, Accumulation Steps, and Distributed World Size

Part 1 - Exercise Contract and Why It Matters

Build an auditable calculator that distinguishes local micro-batch, local accumulated batch, global effective batch, samples per optimizer update, and optimizer updates per epoch.

The learner must produce an executable experiment, a measurement table, and a short engineering conclusion. The experiment must preserve enough evidence to reconstruct which samples entered each optimizer update, which configuration was active, and whether the claimed memory or numerical behavior was actually observed. A result without controls, units, seeds, or a declared reference is incomplete.

Part 2 - System and Memory Mental Model

In data parallel training, each rank processes its own micro-batch and gradients are synchronized across ranks. Accumulation changes how many local backward passes contribute before an optimizer step. Distributed sampler padding, dropped samples, and partial windows can make nominal and observed effective batch differ.

Reason separately about the CUDA allocator, live tensor storage, autograd graph lifetime, gradient-buffer lifetime, and optimizer-state lifetime. Current allocated memory describes live allocations known to PyTorch; reserved memory also includes cached allocator blocks. Neither value alone identifies tensor ownership, so phase measurements and component accounting must accompany peak statistics.

Part 3 - Quantitative Prediction Before Execution

The nominal global effective batch is

$$B_{\text{eff}} = bKW.$$

With N observed samples and full windows, optimizer updates per epoch are approximately

$$U = \left\lceil \frac{N}{B_{\text{eff}}} \right\rceil,$$

but the final update may have a smaller observed batch and must be reported separately.

Calculate nominal local and global batch values for several configurations, including world size one. Predict the final update size with drop_last enabled and disabled.

Write the prediction before running CUDA. Every predicted quantity must have a unit, a scope, and an assumption. At minimum, predict effective samples per optimizer update, optimizer-update count, peak allocated VRAM, peak reserved VRAM, synchronized update latency, throughput, and the gradient statistic that will decide correctness.

Part 4 - Experimental Design and Instrumentation

Compare calculator output with counters inside a simulated or real distributed loop. Record samples processed by every rank, micro-steps, synchronization events, optimizer updates, padding, and dropped samples.

Use fixed seeds, identical initial model and optimizer states, the same ordered samples, the same precision policy, and the same model mode for controlled comparisons. Warm up CUDA before timing, synchronize only at deliberate measurement boundaries, reset peak statistics immediately before the measured region, and report median together with a tail statistic across repeated trials. Keep profiler runs separate from final latency measurements.

Part 5 - PyTorch/CUDA Implementation

Complete the following structure. The missing operations are intentionally represented by extttEllipsis; implement them from the contract, predictions, and measurement requirements above. Do not replace the experiment with a library trainer that hides accumulation boundaries or memory observations.

```
from dataclasses import dataclass

@dataclass
class EffectiveBatchReport:
    micro_batch_size: int
    accumulation_steps: int
    world_size: int
    local_effective_batch: int
    global_effective_batch: int
    full_optimizer_updates: int
    final_update_samples: int

def validate_positive_integer(name, value):
    ...

def E08(
```

```
        dataset_size,
        micro_batch_size,
        accumulation_steps,
        world_size,
        drop_last):
    validate_positive_integer("dataset_size", dataset_size)
    validate_positive_integer("micro_batch_size", micro_batch_size)
    validate_positive_integer("accumulation_steps", accumulation_steps)
    validate_positive_integer("world_size", world_size)

    local_effective_batch = ...
    global_effective_batch = ...
    full_optimizer_updates = ...
    final_update_samples = ...

    return EffectiveBatchReport(...)
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate positive integer inputs, distinguish rank-local from global counts, and avoid assuming dataset length divides the distributed sampler or accumulation window.

Validate loss, every named parameter gradient, updated parameters, and optimizer state against the declared reference using dtype-appropriate absolute and relative tolerances. Record missing gradients, zero fractions, non-finite values, OOM stage, and unsuccessful configurations instead of deleting them. Separate theoretical prediction, measured value, absolute error, relative error, and interpretation in the final report.

Part 7 - Engineering Interpretation and Philosophy

Clear batch terminology prevents silent learning-rate, scheduler, and reproducibility errors when accumulation and distribution are combined.

The final recommendation must state which resource was saved, which cost increased, where the critical path moved, and under which workload and hardware the conclusion is valid. Preserve negative results and limitations: a memory technique that fits a larger workload but violates numerical equivalence, throughput requirements, or operational headroom has not solved the complete engineering problem.

19.23.10 Exercise 09

Exercise Title: Handle a Final Partial Accumulation Window Correctly When Dataset Length Is Not Divisible by Accumulation Steps

Part 1 - Exercise Contract and Why It Matters

Handle the final window when fewer than K micro-batches remain. Preserve the intended sample-mean gradient, perform exactly one final optimizer update when appropriate, and report its actual effective batch.

The learner must produce an executable experiment, a measurement table, and a short engineering conclusion. The experiment must preserve enough evidence to reconstruct which samples entered each optimizer update, which configuration was active, and whether the claimed

memory or numerical behavior was actually observed. A result without controls, units, seeds, or a declared reference is incomplete.

Part 2 - System and Memory Mental Model

Dividing every loss by the configured K underweights a partial window. Dividing by the number of remaining micro-batches works only for equal micro-batch sizes. The most general approach weights each mean loss by its sample count relative to total samples in the current window.

Reason separately about the CUDA allocator, live tensor storage, autograd graph lifetime, gradient-buffer lifetime, and optimizer-state lifetime. Current allocated memory describes live allocations known to PyTorch; reserved memory also includes cached allocator blocks. Neither value alone identifies tensor ownership, so phase measurements and component accounting must accompany peak statistics.

Part 3 - Quantitative Prediction Before Execution

For final-window micro-batch sizes $b_1, \ldots, b_R$ and $B_f = \sum_{k=1}^{R} b_k$,

$$L_f = \sum_{k=1}^{R} \frac{b_k}{B_f} L_k, \qquad g_f = \sum_{k=1}^{R} \frac{b_k}{B_f} g_k.$$

Using $1/K$ when $R < K$ scales the update incorrectly.

Calculate final-window sample count, weights, optimizer-update count, and expected gradient norm. Compare drop, underweighted, and correctly weighted policies.

Write the prediction before running CUDA. Every predicted quantity must have a unit, a scope, and an assumption. At minimum, predict effective samples per optimizer update, optimizer-update count, peak allocated VRAM, peak reserved VRAM, synchronized update latency, throughput, and the gradient statistic that will decide correctness.

Part 4 - Experimental Design and Instrumentation

Choose dataset lengths that produce full windows, a one-micro-batch remainder, and unequal last micro-batch size. Compare each final update with a direct batch reference containing exactly the same remaining samples.

Use fixed seeds, identical initial model and optimizer states, the same ordered samples, the same precision policy, and the same model mode for controlled comparisons. Warm up CUDA before timing, synchronize only at deliberate measurement boundaries, reset peak statistics immediately before the measured region, and report median together with a tail statistic across repeated trials. Keep profiler runs separate from final latency measurements.

Part 5 - PyTorch/CUDA Implementation

Complete the following structure. The missing operations are intentionally represented by extttEllipsis; implement them from the contract, predictions, and measurement requirements above. Do not replace the experiment with a library trainer that hides accumulation boundaries or memory observations.

```
import torch

def collect_accumulation_windows(data_loader, accumulation_steps):
    current_window = []
```

```
    for batch in data_loader:
        current_window.append(batch)

        if len(current_window) == accumulation_steps:
            yield current_window
            current_window = []

    if len(current_window) > 0:
        yield current_window

def process_window(model, optimizer, window, loss_function):
    total_samples_in_window = ...
    optimizer.zero_grad(set_to_none=True)

    for inputs, targets in window:
        current_micro_batch_size = ...
        loss_weight = ...
        outputs = ...
        loss = ...
        ...

    ...
    return ...

def E09(model, optimizer, data_loader, loss_function, accumulation_steps):
    update_reports = []
    for window in collect_accumulation_windows(
            data_loader, accumulation_steps):
        update_report = ...
        update_reports.append(update_report)
    return update_reports
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not carry residual gradients into the next epoch unless explicitly designed. Verify scheduler and logging step counts and handle an empty remainder without creating a false optimizer update.

Validate loss, every named parameter gradient, updated parameters, and optimizer state against the declared reference using dtype-appropriate absolute and relative tolerances. Record missing gradients, zero fractions, non-finite values, OOM stage, and unsuccessful configurations instead of deleting them. Separate theoretical prediction, measured value, absolute error, relative error, and interpretation in the final report.

Part 7 - Engineering Interpretation and Philosophy

Partial windows expose whether accumulation logic represents samples or merely loop iterations. Correct weighting preserves the actual objective.

The final recommendation must state which resource was saved, which cost increased, where the critical path moved, and under which workload and hardware the conclusion is valid. Preserve negative results and limitations: a memory technique that fits a larger workload

but violates numerical equivalence, throughput requirements, or operational headroom has not solved the complete engineering problem.

19.23.11 Exercise 10

Exercise Title: Compare Zeroing Gradients Before and After an Accumulation Window and Diagnose Accidental Gradient Carry-Over

Part 1 - Exercise Contract and Why It Matters

Compare valid gradient-clearing placements and reproduce accidental carry-over between optimizer updates. Track gradient-buffer identity, existence, value, and memory behavior for zero tensors versus set_to_none.

The learner must produce an executable experiment, a measurement table, and a short engineering conclusion. The experiment must preserve enough evidence to reconstruct which samples entered each optimizer update, which configuration was active, and whether the claimed memory or numerical behavior was actually observed. A result without controls, units, seeds, or a declared reference is incomplete.

Part 2 - System and Memory Mental Model

Backward accumulates into parameter.grad. Gradients must be cleared once per logical update, either before a new window or immediately after the previous step. Clearing inside the window destroys contributions; never clearing mixes distinct effective batches. set_to_none can change allocation and first-write behavior.

Reason separately about the CUDA allocator, live tensor storage, autograd graph lifetime, gradient-buffer lifetime, and optimizer-state lifetime. Current allocated memory describes live allocations known to PyTorch; reserved memory also includes cached allocator blocks. Neither value alone identifies tensor ownership, so phase measurements and component accounting must accompany peak statistics.

Part 3 - Quantitative Prediction Before Execution

If update windows are $u = 1, 2, \ldots$, correct state satisfies

$$g^{(u)} = \sum_{k \in u} w_k g_k.$$

Without clearing, the next window receives

$$\widetilde{g}^{(u+1)} = g^{(u)} + g^{(u+1)},$$

which couples two optimizer updates incorrectly.

Predict gradient norms before first backward, after each micro-step, after optimizer step, and after clearing for each policy. Predict memory differences between zeroing values and setting references to None.

Write the prediction before running CUDA. Every predicted quantity must have a unit, a scope, and an assumption. At minimum, predict effective samples per optimizer update, optimizer-update count, peak allocated VRAM, peak reserved VRAM, synchronized update latency, throughput, and the gradient statistic that will decide correctness.

Part 4 - Experimental Design and Instrumentation

Run clear-before, clear-after, clear-inside, and never-clear branches from identical state. Record gradient checksums and identities at every boundary, parameter updates, optimizer state, allocated VRAM, and next-window loss.

Use fixed seeds, identical initial model and optimizer states, the same ordered samples, the same precision policy, and the same model mode for controlled comparisons. Warm up CUDA before timing, synchronize only at deliberate measurement boundaries, reset peak statistics immediately before the measured region, and report median together with a tail statistic across repeated trials. Keep profiler runs separate from final latency measurements.

Part 5 - PyTorch/CUDA Implementation

Complete the following structure. The missing operations are intentionally represented by extttEllipsis; implement them from the contract, predictions, and measurement requirements above. Do not replace the experiment with a library trainer that hides accumulation boundaries or memory observations.

```
import torch

def snapshot_gradient_state(model):
    snapshot = {}
    for parameter_name, parameter in model.named_parameters():
        if parameter.grad is None:
            snapshot[parameter_name] = {
                "exists": False,
                "value": None,
            }
        else:
            snapshot[parameter_name] = {
                "exists": True,
                "value": ...,
            }
    return snapshot

def run_zeroing_policy(
        model,
        optimizer,
        windows,
        loss_function,
        zeroing_policy):
    timeline = []

    for window_index, window in enumerate(windows):
        # Apply only the selected clearing policy at this boundary.
        ...

        for inputs, targets in window:
            ...

        timeline.append(...)
        ...
```

```
    return timeline

def E10(model, optimizer, windows, loss_function):
    return ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Distinguish intentional accumulation from accidental carry-over. Optimizer.step does not clear gradients automatically. Ensure diagnostic logging does not itself retain gradient tensors.

Validate loss, every named parameter gradient, updated parameters, and optimizer state against the declared reference using dtype-appropriate absolute and relative tolerances. Record missing gradients, zero fractions, non-finite values, OOM stage, and unsuccessful configurations instead of deleting them. Separate theoretical prediction, measured value, absolute error, relative error, and interpretation in the final report.

Part 7 - Engineering Interpretation and Philosophy

Gradient lifetime is explicit program state. Correct clearing defines where one logical training update ends and the next begins.

The final recommendation must state which resource was saved, which cost increased, where the critical path moved, and under which workload and hardware the conclusion is valid. Preserve negative results and limitations: a memory technique that fits a larger workload but violates numerical equivalence, throughput requirements, or operational headroom has not solved the complete engineering problem.

19.23.12 Exercise 11

Exercise Title: Combine Gradient Accumulation with torch.amp.GradScaler and Place Scale, Unscale, and Optimizer Operations Correctly

Part 1 - Exercise Contract and Why It Matters

Combine float16 autocast, GradScaler, and accumulation while preserving one scale across the complete window. Place unscale, finite checking, scaler.step, and scaler.update exactly once at the optimizer boundary.

The learner must produce an executable experiment, a measurement table, and a short engineering conclusion. The experiment must preserve enough evidence to reconstruct which samples entered each optimizer update, which configuration was active, and whether the claimed memory or numerical behavior was actually observed. A result without controls, units, seeds, or a declared reference is incomplete.

Part 2 - System and Memory Mental Model

Every normalized micro-batch loss is scaled before backward, so parameter buffers accumulate scaled gradients. Changing the scale inside the window would mix incompatible units. unscale_ converts the complete accumulated gradient once; scaler.step can then reject a non-finite window, and update adjusts the scale for the next window.

Reason separately about the CUDA allocator, live tensor storage, autograd graph lifetime, gradient-buffer lifetime, and optimizer-state lifetime. Current allocated memory describes live allocations known to PyTorch; reserved memory also includes cached allocator blocks. Neither

value alone identifies tensor ownership, so phase measurements and component accounting must accompany peak statistics.

Part 3 - Quantitative Prediction Before Execution

With constant scale S during a window,

$$\widetilde{g} = \sum_{k=1}^{K} S w_k g_k = S \sum_{k=1}^{K} w_k g_k.$$

One unscale operation yields the intended gradient. If scales S_k differ within the window, no single division reconstructs the sum correctly.

Predict scale value at every micro-step, the final scaled norm, unscaled norm, overflow outcome, and whether the optimizer update commits. Include an overflow injected into an early micro-step.

Write the prediction before running CUDA. Every predicted quantity must have a unit, a scope, and an assumption. At minimum, predict effective samples per optimizer update, optimizer-update count, peak allocated VRAM, peak reserved VRAM, synchronized update latency, throughput, and the gradient statistic that will decide correctness.

Part 4 - Experimental Design and Instrumentation

Compare three branches: a full-batch float32 reference, accumulated autocast with GradScaler, and a deliberately incorrect branch that updates inside the window. Record scale timeline, finite status after unscale, skipped windows, parameter and optimizer checksums, peak VRAM, and throughput.

Use fixed seeds, identical initial model and optimizer states, the same ordered samples, the same precision policy, and the same model mode for controlled comparisons. Warm up CUDA before timing, synchronize only at deliberate measurement boundaries, reset peak statistics immediately before the measured region, and report median together with a tail statistic across repeated trials. Keep profiler runs separate from final latency measurements.

Part 5 - PyTorch/CUDA Implementation

Complete the following structure. The missing operations are intentionally represented by extttEllipsis; implement them from the contract, predictions, and measurement requirements above. Do not replace the experiment with a library trainer that hides accumulation boundaries or memory observations.

```
import torch

def run_scaled_accumulation_window(
        model,
        optimizer,
        scaler,
        micro_batches,
        loss_function,
        accumulation_steps):
    optimizer.zero_grad(set_to_none=True)
    scale_at_window_start = scaler.get_scale()

    for inputs, targets in micro_batches:
```

```
        with torch.autocast(
                device_type="cuda",
                dtype=torch.float16):
            outputs = ...
            loss = ...
            normalized_loss = ...

        scaled_loss = ...
        ...

        # The scale must not be updated inside this loop.
        ...

    # Unscale exactly once after the complete accumulation window.
    ...

    # Perform a protected optimizer step, then update the scale once.
    ...

    return {
        "scale_before": scale_at_window_start,
        "scale_after": ...,
        "window_result": ...,
    }

def E11(model, optimizer, scaler, micro_batches, loss_function):
    return ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not call scaler.update or unscale_ more than once per optimizer per window. If any micro-step overflows, the complete logical update must be rejected. Verify API behavior for the installed PyTorch version.

Validate loss, every named parameter gradient, updated parameters, and optimizer state against the declared reference using dtype-appropriate absolute and relative tolerances. Record missing gradients, zero fractions, non-finite values, OOM stage, and unsuccessful configurations instead of deleting them. Separate theoretical prediction, measured value, absolute error, relative error, and interpretation in the final report.

Part 7 - Engineering Interpretation and Philosophy

Accumulation and scaling share the same transaction boundary. Keeping numerical units stable across that boundary is the core correctness rule.

The final recommendation must state which resource was saved, which cost increased, where the critical path moved, and under which workload and hardware the conclusion is valid. Preserve negative results and limitations: a memory technique that fits a larger workload but violates numerical equivalence, throughput requirements, or operational headroom has not solved the complete engineering problem.

19.23.13 Exercise 12

Exercise Title: Combine Gradient Accumulation with Gradient Clipping Without Clipping Intermediate Scaled or Partial Gradients Incorrectly

Part 1 - Exercise Contract and Why It Matters

Apply gradient clipping only after the complete logical gradient exists and, when GradScaler is active, only after it has been unscaled. Reproduce and quantify clipping of partial or scaled gradients as failure cases.

The learner must produce an executable experiment, a measurement table, and a short engineering conclusion. The experiment must preserve enough evidence to reconstruct which samples entered each optimizer update, which configuration was active, and whether the claimed memory or numerical behavior was actually observed. A result without controls, units, seeds, or a declared reference is incomplete.

Part 2 - System and Memory Mental Model

Clipping each micro-step changes the direction and magnitude of the final sum. Clipping a scaled gradient uses the threshold in the wrong units. The correct sequence is accumulate, unscale once if required, measure complete norm, clip, then step.

Reason separately about the CUDA allocator, live tensor storage, autograd graph lifetime, gradient-buffer lifetime, and optimizer-state lifetime. Current allocated memory describes live allocations known to PyTorch; reserved memory also includes cached allocator blocks. Neither value alone identifies tensor ownership, so phase measurements and component accounting must accompany peak statistics.

Part 3 - Quantitative Prediction Before Execution

For clipping threshold C, the complete gradient transformation is

$$g_{\text{clip}} = g \min\left(1, \frac{C}{\|g\|_2}\right).$$

Generally,

$$\sum_k \text{clip}(g_k, C) \neq \text{clip}\left(\sum_k g_k, C\right).$$

Calculate individual, cumulative, scaled, unscaled, and final clipped norms for a controlled example with opposing micro-gradients. Predict parameter updates for correct and incorrect orderings.

Write the prediction before running CUDA. Every predicted quantity must have a unit, a scope, and an assumption. At minimum, predict effective samples per optimizer update, optimizer-update count, peak allocated VRAM, peak reserved VRAM, synchronized update latency, throughput, and the gradient statistic that will decide correctness.

Part 4 - Experimental Design and Instrumentation

Run clip-after-window, clip-each-micro-step, and clip-scaled-gradient branches from cloned state. Record gradient direction similarity, norm, clip coefficient, parameter delta, and next loss.

Use fixed seeds, identical initial model and optimizer states, the same ordered samples, the same precision policy, and the same model mode for controlled comparisons. Warm up CUDA before timing, synchronize only at deliberate measurement boundaries, reset peak statistics

immediately before the measured region, and report median together with a tail statistic across repeated trials. Keep profiler runs separate from final latency measurements.

Part 5 - PyTorch/CUDA Implementation

Complete the following structure. The missing operations are intentionally represented by extttEllipsis; implement them from the contract, predictions, and measurement requirements above. Do not replace the experiment with a library trainer that hides accumulation boundaries or memory observations.

```
import torch

def E12(
        model,
        optimizer,
        scaler,
        micro_batches,
        loss_function,
        maximum_gradient_norm):
    optimizer.zero_grad(set_to_none=True)

    for inputs, targets in micro_batches:
        with torch.autocast(
                device_type="cuda",
                dtype=torch.float16):
            outputs = ...
            loss = ...
            normalized_loss = ...

        ...

    # Restore the complete accumulated gradient to its true units.
    ...

    # Measure and clip only after accumulation and unscaling.
    gradient_norm_before_clipping = ...
    ...

    # Commit or skip the optimizer update through the scaler.
    ...

    return {
        "gradient_norm_before_clipping": gradient_norm_before_clipping,
        "gradient_norm_after_clipping": ...,
        "scale": ...,
    }
```

Part 6 - Measurement, Validation, and Failure Analysis

Use the same norm type and parameter set in every branch. Detect non-finite norms before clipping and verify that logging occurs without mutating gradients.

Validate loss, every named parameter gradient, updated parameters, and optimizer state against the declared reference using dtype-appropriate absolute and relative tolerances. Record missing gradients, zero fractions, non-finite values, OOM stage, and unsuccessful configurations instead of deleting them. Separate theoretical prediction, measured value, absolute error, relative error, and interpretation in the final report.

Part 7 - Engineering Interpretation and Philosophy

Gradient clipping is a transformation of one complete optimizer update. Applying it to incomplete or differently scaled state changes the optimization algorithm.

The final recommendation must state which resource was saved, which cost increased, where the critical path moved, and under which workload and hardware the conclusion is valid. Preserve negative results and limitations: a memory technique that fits a larger workload but violates numerical equivalence, throughput requirements, or operational headroom has not solved the complete engineering problem.

19.23.14 Exercise 13

Exercise Title: Measure Activation and Gradient Memory Across Accumulation Steps and Explain Which Memory Components Actually Shrink

Part 1 - Exercise Contract and Why It Matters

Create a phase-by-phase memory timeline across an accumulation window and attribute bytes to inputs, activations, saved tensors, gradients, parameters, optimizer state, workspaces, and allocator reservation.

The learner must produce an executable experiment, a measurement table, and a short engineering conclusion. The experiment must preserve enough evidence to reconstruct which samples entered each optimizer update, which configuration was active, and whether the claimed memory or numerical behavior was actually observed. A result without controls, units, seeds, or a declared reference is incomplete.

Part 2 - System and Memory Mental Model

Activation graphs should rise and fall during each micro-step. Parameter gradients appear during the first backward and then persist for accumulation. Parameters and initialized optimizer state remain fixed. If peak memory grows every micro-step, the program may be retaining graphs, losses, outputs, or references.

Reason separately about the CUDA allocator, live tensor storage, autograd graph lifetime, gradient-buffer lifetime, and optimizer-state lifetime. Current allocated memory describes live allocations known to PyTorch; reserved memory also includes cached allocator blocks. Neither value alone identifies tensor ownership, so phase measurements and component accounting must accompany peak statistics.

Part 3 - Quantitative Prediction Before Execution

A simplified peak after the first backward is

$$M_k \approx M_{\text{param}} + M_{\text{grad}} + M_{\text{opt}} + M_{\text{activation}}(b) + M_{\text{workspace}}.$$

Correct graph release implies M_k should not grow approximately linearly with k.

Predict the first allocation of gradient buffers, the recurring activation peak, and memory after each backward. Create an expected timeline before instrumentation.

Write the prediction before running CUDA. Every predicted quantity must have a unit, a scope, and an assumption. At minimum, predict effective samples per optimizer update, optimizer-update count, peak allocated VRAM, peak reserved VRAM, synchronized update latency, throughput, and the gradient statistic that will decide correctness.

Part 4 - Experimental Design and Instrumentation

Measure allocated and reserved VRAM at window start, before and after every forward and backward, before optimizer step, after step, and after zeroing. Combine allocator telemetry with saved-tensor bytes and memory snapshots for selected steps.

Use fixed seeds, identical initial model and optimizer states, the same ordered samples, the same precision policy, and the same model mode for controlled comparisons. Warm up CUDA before timing, synchronize only at deliberate measurement boundaries, reset peak statistics immediately before the measured region, and report median together with a tail statistic across repeated trials. Keep profiler runs separate from final latency measurements.

Part 5 - PyTorch/CUDA Implementation

Complete the following structure. The missing operations are intentionally represented by extttEllipsis; implement them from the contract, predictions, and measurement requirements above. Do not replace the experiment with a library trainer that hides accumulation boundaries or memory observations.

```
import torch

def capture_memory_point(label):
    return {
        "label": label,
        "allocated": ...,
        "reserved": ...,
        "maximum_allocated": ...,
        "maximum_reserved": ...,
    }

def E13(
        model,
        optimizer,
        micro_batches,
        loss_function):
    timeline = []
    optimizer.zero_grad(set_to_none=True)
    timeline.append(capture_memory_point("window_start"))

    for micro_step_index, (inputs, targets) in enumerate(micro_batches):
        timeline.append(capture_memory_point(
            f"micro_{micro_step_index}_before_forward"))

        outputs = ...
        loss = ...
```

```
        timeline.append(capture_memory_point(
            f"micro_{micro_step_index}_after_forward"))

        ...

        timeline.append(capture_memory_point(
            f"micro_{micro_step_index}_after_backward"))

    ...
    timeline.append(capture_memory_point("after_optimizer_step"))
    return timeline
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not infer component ownership from one scalar peak. Detect Python lists retaining losses or outputs, retain_graph usage, anomaly detection overhead, and profiler-induced retention.

Validate loss, every named parameter gradient, updated parameters, and optimizer state against the declared reference using dtype-appropriate absolute and relative tolerances. Record missing gradients, zero fractions, non-finite values, OOM stage, and unsuccessful configurations instead of deleting them. Separate theoretical prediction, measured value, absolute error, relative error, and interpretation in the final report.

Part 7 - Engineering Interpretation and Philosophy

Accumulation saves activation concurrency, not gradient storage. A memory timeline makes that distinction visible and exposes leaks that aggregate peaks hide.

The final recommendation must state which resource was saved, which cost increased, where the critical path moved, and under which workload and hardware the conclusion is valid. Preserve negative results and limitations: a memory technique that fits a larger workload but violates numerical equivalence, throughput requirements, or operational headroom has not solved the complete engineering problem.

19.23.15 Exercise 14

Exercise Title: Compare Gradient Accumulation Against Activation Checkpointing Under the Same Peak VRAM Budget

Part 1 - Exercise Contract and Why It Matters

Compare two different ways to meet the same measured VRAM budget: smaller micro-batches with more accumulation, and activation checkpointing with larger micro-batches and recomputation.

The learner must produce an executable experiment, a measurement table, and a short engineering conclusion. The experiment must preserve enough evidence to reconstruct which samples entered each optimizer update, which configuration was active, and whether the claimed memory or numerical behavior was actually observed. A result without controls, units, seeds, or a declared reference is incomplete.

Part 2 - System and Memory Mental Model

Accumulation reduces concurrent batch activations but repeats forward and backward launches. Checkpointing removes selected saved activations and replays forward regions during backward. They affect different axes and can produce different throughput, RNG, normalization, and communication behavior.

Reason separately about the CUDA allocator, live tensor storage, autograd graph lifetime, gradient-buffer lifetime, and optimizer-state lifetime. Current allocated memory describes live allocations known to PyTorch; reserved memory also includes cached allocator blocks. Neither value alone identifies tensor ownership, so phase measurements and component accounting must accompany peak statistics.

Part 3 - Quantitative Prediction Before Execution

Match achieved peak memory rather than nominal settings:

$$M_{\text{peak}}^{\text{accum}} \approx M_{\text{peak}}^{\text{checkpoint}} \leq M_{\text{budget}}.$$

Compare cost per accepted sample using

$$C = \frac{T_{\text{update}}}{B_{\text{eff}}}.$$

Estimate launch overhead for accumulation and replay FLOPs for checkpointing. Predict which model regions or batch sizes favor each technique.

Write the prediction before running CUDA. Every predicted quantity must have a unit, a scope, and an assumption. At minimum, predict effective samples per optimizer update, optimizer-update count, peak allocated VRAM, peak reserved VRAM, synchronized update latency, throughput, and the gradient statistic that will decide correctness.

Part 4 - Experimental Design and Instrumentation

Tune configurations until measured peaks are within a declared tolerance. Keep effective batch, precision, model, optimizer, and samples fixed. Record throughput, replay time, kernel count, gradient error, RNG behavior, and energy or GPU-time proxy.

Use fixed seeds, identical initial model and optimizer states, the same ordered samples, the same precision policy, and the same model mode for controlled comparisons. Warm up CUDA before timing, synchronize only at deliberate measurement boundaries, reset peak statistics immediately before the measured region, and report median together with a tail statistic across repeated trials. Keep profiler runs separate from final latency measurements.

Part 5 - PyTorch/CUDA Implementation

Complete the following structure. The missing operations are intentionally represented by extttEllipsis; implement them from the contract, predictions, and measurement requirements above. Do not replace the experiment with a library trainer that hides accumulation boundaries or memory observations.

```
import torch
from torch.utils.checkpoint import checkpoint

def run_accumulation_strategy(configuration, training_context):
    # Configure micro-batch size and accumulation steps.
    ...
    return ...
```

```
def run_checkpoint_strategy(configuration, training_context):
    # Configure checkpoint regions and the larger micro-batch.
    ...
    return ...

def compare_equal_budget_results(
        accumulation_result,
        checkpoint_result,
        vram_budget,
        peak_tolerance):
    # Reject comparisons that do not satisfy the same budget.
    ...
    return ...

def E14(
        accumulation_configuration,
        checkpoint_configuration,
        training_context,
        vram_budget):
    accumulation_result = ...
    checkpoint_result = ...
    comparison = ...
    return comparison
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not compare unmatched memory peaks or effective batches. Checkpointing must preserve required RNG state, and accumulation must control batch-dependent layers.

Validate loss, every named parameter gradient, updated parameters, and optimizer state against the declared reference using dtype-appropriate absolute and relative tolerances. Record missing gradients, zero fractions, non-finite values, OOM stage, and unsuccessful configurations instead of deleting them. Separate theoretical prediction, measured value, absolute error, relative error, and interpretation in the final report.

Part 7 - Engineering Interpretation and Philosophy

Equal-budget comparison reveals which resource is cheaper to spend: extra micro-step overhead or recomputation inside the graph.

The final recommendation must state which resource was saved, which cost increased, where the critical path moved, and under which workload and hardware the conclusion is valid. Preserve negative results and limitations: a memory technique that fits a larger workload but violates numerical equivalence, throughput requirements, or operational headroom has not solved the complete engineering problem.

19.23.16 Exercise 15

Exercise Title: Combine Gradient Accumulation with Mixed Precision and Activation Checkpointing to Maximize Effective Batch Size

Part 1 - Exercise Contract and Why It Matters

Compose three techniques under one VRAM budget and attribute their overlapping savings and costs. Search for the largest validated effective batch rather than multiplying headline savings.

The learner must produce an executable experiment, a measurement table, and a short engineering conclusion. The experiment must preserve enough evidence to reconstruct which samples entered each optimizer update, which configuration was active, and whether the claimed memory or numerical behavior was actually observed. A result without controls, units, seeds, or a declared reference is incomplete.

Part 2 - System and Memory Mental Model

Mixed precision shrinks eligible tensor payload and changes kernels. Checkpointing discards selected saved activations. Accumulation reduces simultaneous examples. Their benefits overlap on activation bytes, while parameters, gradients, optimizer state, replay, launches, and scaling logic remain.

Reason separately about the CUDA allocator, live tensor storage, autograd graph lifetime, gradient-buffer lifetime, and optimizer-state lifetime. Current allocated memory describes live allocations known to PyTorch; reserved memory also includes cached allocator blocks. Neither value alone identifies tensor ownership, so phase measurements and component accounting must accompany peak statistics.

Part 3 - Quantitative Prediction Before Execution

The search objective is

$$\max_{b,K,P,C} \; B_{\text{eff}} = bKW$$

subject to

$$M_{\text{peak}}(b, P, C) + M_{\text{safety}} \leq M_{\text{budget}}, \quad E(P, C) \leq E_{\max},$$

where P is precision policy and C is checkpoint policy.

Build a component table identifying which technique affects each memory category. Predict feasible configurations, recomputation, scale behavior, and throughput before running the search.

Write the prediction before running CUDA. Every predicted quantity must have a unit, a scope, and an assumption. At minimum, predict effective samples per optimizer update, optimizer-update count, peak allocated VRAM, peak reserved VRAM, synchronized update latency, throughput, and the gradient statistic that will decide correctness.

Part 4 - Experimental Design and Instrumentation

Benchmark single techniques, pairwise combinations, and the full combination from identical state. Record effective batch, peak VRAM, safety margin, saved tensors, replay, micro-step count, scale history, throughput, gradient error, and convergence evidence.

Use fixed seeds, identical initial model and optimizer states, the same ordered samples, the same precision policy, and the same model mode for controlled comparisons. Warm up CUDA before timing, synchronize only at deliberate measurement boundaries, reset peak statistics immediately before the measured region, and report median together with a tail statistic across repeated trials. Keep profiler runs separate from final latency measurements.

Part 5 - PyTorch/CUDA Implementation

Complete the following structure. The missing operations are intentionally represented by extttEllipsis; implement them from the contract, predictions, and measurement requirements above. Do not replace the experiment with a library trainer that hides accumulation boundaries or memory observations.

```
import torch
from torch.utils.checkpoint import checkpoint

def configure_training_strategy(
        micro_batch_size,
        accumulation_steps,
        autocast_dtype,
        checkpoint_regions):
    strategy = {
        "micro_batch_size": micro_batch_size,
        "accumulation_steps": accumulation_steps,
        "autocast_dtype": autocast_dtype,
        "checkpoint_regions": checkpoint_regions,
    }
    return strategy

def run_combined_strategy(
        model,
        optimizer,
        scaler,
        strategy,
        effective_batch,
        loss_function):
    # Split the effective batch using the selected micro-batch size.
    micro_batches = ...

    # Execute checkpointed regions inside the intended autocast policy.
    ...

    # Accumulate scaled gradients and update once at the boundary.
    ...

    return ...

def E15(candidate_strategies, training_context, vram_budget):
    feasible_results = []
    for strategy in candidate_strategies:
        result = ...
        if ...:
            feasible_results.append(result)

    selected_result = ...
    return selected_result, feasible_results
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate every combined configuration independently. Ensure recomputation executes under the intended autocast context and that GradScaler updates only at accumulation boundaries.

Validate loss, every named parameter gradient, updated parameters, and optimizer state against the declared reference using dtype-appropriate absolute and relative tolerances. Record missing gradients, zero fractions, non-finite values, OOM stage, and unsuccessful configurations instead of deleting them. Separate theoretical prediction, measured value, absolute error, relative error, and interpretation in the final report.

Part 7 - Engineering Interpretation and Philosophy

Memory techniques compose through a shared graph and allocator. A combined strategy succeeds only when its interactions are measured, not when individual savings are added on paper.

The final recommendation must state which resource was saved, which cost increased, where the critical path moved, and under which workload and hardware the conclusion is valid. Preserve negative results and limitations: a memory technique that fits a larger workload but violates numerical equivalence, throughput requirements, or operational headroom has not solved the complete engineering problem.

19.23.17 Exercise 16

Exercise Title: Build an Adaptive Accumulation Controller That Changes Micro-Batch Size and Accumulation Steps from Live VRAM Headroom

Part 1 - Exercise Contract and Why It Matters

Build a controller that observes live peak VRAM and adjusts micro-batch size while preserving a target effective batch through accumulation-step changes. Decisions must be bounded, stable, and applied only at safe update boundaries.

The learner must produce an executable experiment, a measurement table, and a short engineering conclusion. The experiment must preserve enough evidence to reconstruct which samples entered each optimizer update, which configuration was active, and whether the claimed memory or numerical behavior was actually observed. A result without controls, units, seeds, or a declared reference is incomplete.

Part 2 - System and Memory Mental Model

Memory headroom varies with sequence length, shape, allocator state, and temporary workspaces. Increasing micro-batch can improve utilization but risks OOM; decreasing it raises accumulation and overhead. High and low watermarks, cooldown, and conservative prediction prevent oscillation.

Reason separately about the CUDA allocator, live tensor storage, autograd graph lifetime, gradient-buffer lifetime, and optimizer-state lifetime. Current allocated memory describes live allocations known to PyTorch; reserved memory also includes cached allocator blocks. Neither value alone identifies tensor ownership, so phase measurements and component accounting must accompany peak statistics.

Part 3 - Quantitative Prediction Before Execution

Let measured headroom be

$$H = M_{\text{budget}} - M_{\text{peak}} - M_{\text{safety}}.$$

After choosing new b', preserve target batch B_t using

$$K' = \left\lceil \frac{B_t}{b'W} \right\rceil,$$

then report any final-window mismatch explicitly.

Define watermarks, candidate steps, cooldown, minimum and maximum micro-batch, and admission prediction. Simulate controller behavior for high headroom, low headroom, and noisy alternating inputs.

Write the prediction before running CUDA. Every predicted quantity must have a unit, a scope, and an assumption. At minimum, predict effective samples per optimizer update, optimizer-update count, peak allocated VRAM, peak reserved VRAM, synchronized update latency, throughput, and the gradient statistic that will decide correctness.

Part 4 - Experimental Design and Instrumentation

Run scripted memory regimes and variable sequence lengths. Log observation, prediction, chosen configuration, reason, achieved peak, throughput, update frequency, and constraint violations. Compare with the best static safe configuration.

Use fixed seeds, identical initial model and optimizer states, the same ordered samples, the same precision policy, and the same model mode for controlled comparisons. Warm up CUDA before timing, synchronize only at deliberate measurement boundaries, reset peak statistics immediately before the measured region, and report median together with a tail statistic across repeated trials. Keep profiler runs separate from final latency measurements.

Part 5 - PyTorch/CUDA Implementation

Complete the following structure. The missing operations are intentionally represented by extttEllipsis; implement them from the contract, predictions, and measurement requirements above. Do not replace the experiment with a library trainer that hides accumulation boundaries or memory observations.

```
from dataclasses import dataclass

@dataclass
class AccumulationConfiguration:
    micro_batch_size: int
    accumulation_steps: int
    effective_batch_size: int

@dataclass
class MemoryObservation:
    peak_vram: int
    vram_budget: int
    safety_margin: int
    throughput: float

class AdaptiveAccumulationController:
    def __init__(
            self,
            target_effective_batch,
```

```
            world_size,
            minimum_micro_batch,
            maximum_micro_batch,
            cooldown_updates):
        self.target_effective_batch = target_effective_batch
        self.world_size = world_size
        self.minimum_micro_batch = minimum_micro_batch
        self.maximum_micro_batch = maximum_micro_batch
        self.cooldown_updates = cooldown_updates
        self.history = []

    def propose(self, current_configuration, observation):
        headroom = ...
        proposed_micro_batch_size = ...
        proposed_accumulation_steps = ...
        reason = ...
        proposal = ...
        self.history.append((observation, proposal, reason))
        return proposal

def E16(controller, current_configuration, observation):
    return controller.propose(current_configuration, observation)
```

Part 6 - Measurement, Validation, and Failure Analysis

Never change accumulation count inside an active window. Reject candidates whose predicted headroom is negative, and include an emergency fallback after OOM. Test oscillation and stale peak observations.

Validate loss, every named parameter gradient, updated parameters, and optimizer state against the declared reference using dtype-appropriate absolute and relative tolerances. Record missing gradients, zero fractions, non-finite values, OOM stage, and unsuccessful configurations instead of deleting them. Separate theoretical prediction, measured value, absolute error, relative error, and interpretation in the final report.

Part 7 - Engineering Interpretation and Philosophy

Adaptive accumulation treats batch decomposition as a runtime scheduling decision. Stability and explainability matter as much as capacity.

The final recommendation must state which resource was saved, which cost increased, where the critical path moved, and under which workload and hardware the conclusion is valid. Preserve negative results and limitations: a memory technique that fits a larger workload but violates numerical equivalence, throughput requirements, or operational headroom has not solved the complete engineering problem.

19.23.18 Exercise 17

Exercise Title: Recover Automatically from a Micro-Batch OOM by Reducing Micro-Batch Size and Increasing Accumulation Steps While Preserving Effective Batch Size

Part 1 - Exercise Contract and Why It Matters

Design a bounded OOM recovery transaction that abandons the failed window, restores clean gradient state, selects a smaller micro-batch, recalculates accumulation, and replays the same logical samples without duplication or loss.

The learner must produce an executable experiment, a measurement table, and a short engineering conclusion. The experiment must preserve enough evidence to reconstruct which samples entered each optimizer update, which configuration was active, and whether the claimed memory or numerical behavior was actually observed. A result without controls, units, seeds, or a declared reference is incomplete.

Part 2 - System and Memory Mental Model

CUDA OOM can occur after partial forward or backward work. Partial gradients, graph references, RNG progress, and data-iterator position can make naive retry incorrect. Recovery requires a defined rollback boundary and a clean trial, not only empty_cache.

Reason separately about the CUDA allocator, live tensor storage, autograd graph lifetime, gradient-buffer lifetime, and optimizer-state lifetime. Current allocated memory describes live allocations known to PyTorch; reserved memory also includes cached allocator blocks. Neither value alone identifies tensor ownership, so phase measurements and component accounting must accompany peak statistics.

Part 3 - Quantitative Prediction Before Execution

For target batch B_t and reduced micro-batch b', choose

$$K' = \frac{B_t}{b'W}$$

when divisible. Otherwise construct an explicitly weighted partial final micro-batch so the replayed sample count remains B_t.

For each fallback size, predict accumulation count, retry cost, memory peak, and maximum number of allowed retries. Define what state must be restored after failure.

Write the prediction before running CUDA. Every predicted quantity must have a unit, a scope, and an assumption. At minimum, predict effective samples per optimizer update, optimizer-update count, peak allocated VRAM, peak reserved VRAM, synchronized update latency, throughput, and the gradient statistic that will decide correctness.

Part 4 - Experimental Design and Instrumentation

Force OOM at known sizes, snapshot sample identifiers and checksums, and verify that the recovered update consumes every intended sample exactly once. Record failed stage, cleanup, retry configuration, peak VRAM, final gradients, parameter update, and recovery latency.

Use fixed seeds, identical initial model and optimizer states, the same ordered samples, the same precision policy, and the same model mode for controlled comparisons. Warm up CUDA before timing, synchronize only at deliberate measurement boundaries, reset peak statistics immediately before the measured region, and report median together with a tail statistic across repeated trials. Keep profiler runs separate from final latency measurements.

Part 5 - PyTorch/CUDA Implementation

Complete the following structure. The missing operations are intentionally represented by extttEllipsis; implement them from the contract, predictions, and measurement requirements above. Do not replace the experiment with a library trainer that hides accumulation boundaries or memory observations.

```
import torch

def calculate_recovery_configuration(
        current_micro_batch_size,
        target_effective_batch_size,
        world_size):
    reduced_micro_batch_size = ...
    new_accumulation_steps = ...
    return reduced_micro_batch_size, new_accumulation_steps

def restore_clean_window_state(
        model,
        optimizer,
        window_checkpoint):
    # Restore gradients, RNG state, and any required mutable state.
    ...

def E17(
        training_window,
        model,
        optimizer,
        initial_configuration,
        target_effective_batch_size,
        world_size,
        maximum_retries):
    current_configuration = initial_configuration
    recovery_history = []

    for retry_index in range(maximum_retries + 1):
        window_checkpoint = ...

        try:
            result = ...
            return result, recovery_history
        except torch.cuda.OutOfMemoryError as error:
            ...
            current_configuration = ...
            recovery_history.append(...)

    raise RuntimeError("No safe micro-batch configuration was found")
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not continue from partial gradients or advance the data stream silently. Handle repeated OOM, minimum micro-batch failure, stochastic replay, distributed rank agreement, and optimizer state integrity.

Validate loss, every named parameter gradient, updated parameters, and optimizer state against the declared reference using dtype-appropriate absolute and relative tolerances. Record missing gradients, zero fractions, non-finite values, OOM stage, and unsuccessful configurations

instead of deleting them. Separate theoretical prediction, measured value, absolute error, relative error, and interpretation in the final report.

Part 7 - Engineering Interpretation and Philosophy

OOM recovery is a correctness-preserving rollback and reschedule operation. Merely surviving the exception is insufficient.

The final recommendation must state which resource was saved, which cost increased, where the critical path moved, and under which workload and hardware the conclusion is valid. Preserve negative results and limitations: a memory technique that fits a larger workload but violates numerical equivalence, throughput requirements, or operational headroom has not solved the complete engineering problem.

19.23.19 Exercise 18

Exercise Title: Stress-Test Adaptive Gradient Accumulation with Variable Sequence Lengths and Track VRAM Peaks, Throughput, and Optimizer Update Frequency

Part 1 - Exercise Contract and Why It Matters

Stress the adaptive controller with changing sequence lengths, padding ratios, and memory demand. Measure whether it preserves the target effective batch and stable optimizer-update semantics while reacting to live pressure.

The learner must produce an executable experiment, a measurement table, and a short engineering conclusion. The experiment must preserve enough evidence to reconstruct which samples entered each optimizer update, which configuration was active, and whether the claimed memory or numerical behavior was actually observed. A result without controls, units, seeds, or a declared reference is incomplete.

Part 2 - System and Memory Mental Model

Transformer activation memory can change sharply with sequence length and attention implementation. A configuration safe for one batch can fail on the next. Frequent adaptation changes micro-step count and optimizer-update wall-clock frequency, affecting schedules expressed in steps or tokens.

Reason separately about the CUDA allocator, live tensor storage, autograd graph lifetime, gradient-buffer lifetime, and optimizer-state lifetime. Current allocated memory describes live allocations known to PyTorch; reserved memory also includes cached allocator blocks. Neither value alone identifies tensor ownership, so phase measurements and component accounting must accompany peak statistics.

Part 3 - Quantitative Prediction Before Execution

For update u, record actual tokens

$$Q_u = \sum_{k=1}^{K_u} \sum_{j=1}^{b_u} \ell_{u,k,j},$$

where ℓ is non-padding sequence length. Samples per update may remain constant while tokens, memory, and compute vary substantially.

Construct short, medium, long, and adversarial length regimes. Predict micro-batch, accumulation count, peak VRAM, tokens per update, throughput, and adaptation delay for each.

Write the prediction before running CUDA. Every predicted quantity must have a unit, a scope, and an assumption. At minimum, predict effective samples per optimizer update, optimizer-update count, peak allocated VRAM, peak reserved VRAM, synchronized update latency, throughput, and the gradient statistic that will decide correctness.

Part 4 - Experimental Design and Instrumentation

Run deterministic and randomized length schedules. Log shape distribution, padding, configuration, peak VRAM, headroom, samples and tokens per update, optimizer updates per minute, throughput, OOM recovery, and gradient validity.

Use fixed seeds, identical initial model and optimizer states, the same ordered samples, the same precision policy, and the same model mode for controlled comparisons. Warm up CUDA before timing, synchronize only at deliberate measurement boundaries, reset peak statistics immediately before the measured region, and report median together with a tail statistic across repeated trials. Keep profiler runs separate from final latency measurements.

Part 5 - PyTorch/CUDA Implementation

Complete the following structure. The missing operations are intentionally represented by extttEllipsis; implement them from the contract, predictions, and measurement requirements above. Do not replace the experiment with a library trainer that hides accumulation boundaries or memory observations.

```
def summarize_sequence_batch(inputs, attention_mask):
    sequence_statistics = {
        "batch_size": ...,
        "padded_length": ...,
        "non_padding_tokens": ...,
        "padding_fraction": ...,
    }
    return sequence_statistics

def E18(
        adaptive_controller,
        training_engine,
        variable_length_batches):
    stress_test_history = []

    for batch_index, batch in enumerate(variable_length_batches):
        inputs, targets, attention_mask = batch
        sequence_statistics = ...
        current_configuration = ...

        update_result = ...
        memory_observation = ...
        next_configuration = ...

        stress_test_history.append({
            "batch_index": batch_index,
            "sequence_statistics": sequence_statistics,
            "current_configuration": current_configuration,
```

```
            "update_result": update_result,
            "memory_observation": memory_observation,
            "next_configuration": next_configuration,
        })

    return stress_test_history
```

Part 6 - Measurement, Validation, and Failure Analysis

Detect oscillation, repeated retries, schedule drift, partial windows, and comparisons based only on samples when tokens drive work. Include the cost of adaptation and failed attempts.

Validate loss, every named parameter gradient, updated parameters, and optimizer state against the declared reference using dtype-appropriate absolute and relative tolerances. Record missing gradients, zero fractions, non-finite values, OOM stage, and unsuccessful configurations instead of deleting them. Separate theoretical prediction, measured value, absolute error, relative error, and interpretation in the final report.

Part 7 - Engineering Interpretation and Philosophy

Dynamic workloads test whether accumulation policy is truly memory-aware or merely tuned to one static shape.

The final recommendation must state which resource was saved, which cost increased, where the critical path moved, and under which workload and hardware the conclusion is valid. Preserve negative results and limitations: a memory technique that fits a larger workload but violates numerical equivalence, throughput requirements, or operational headroom has not solved the complete engineering problem.

19.23.20 Exercise 19

Exercise Title: Build and Benchmark a PyTorch Memory-Aware Gradient Accumulation Engine That Maximizes Effective Batch Size Under a Strict VRAM Budget

Part 1 - Exercise Contract and Why It Matters

Integrate batch calculation, micro-batch partitioning, weighted accumulation, mixed precision, clipping, memory telemetry, adaptive configuration, OOM rollback, and diagnostics into one engine. Its objective is the largest validated effective batch within a strict budget and numerical contract.

The learner must produce an executable experiment, a measurement table, and a short engineering conclusion. The experiment must preserve enough evidence to reconstruct which samples entered each optimizer update, which configuration was active, and whether the claimed memory or numerical behavior was actually observed. A result without controls, units, seeds, or a declared reference is incomplete.

Part 2 - System and Memory Mental Model

The engine needs a control plane and a training data plane. The control plane selects and revises configuration from measured headroom and throughput. The data plane executes one transactional window, preserves sample identity, accumulates gradients, validates the complete update, and commits or rolls back. Every decision requires a reason and measured outcome.

Reason separately about the CUDA allocator, live tensor storage, autograd graph lifetime, gradient-buffer lifetime, and optimizer-state lifetime. Current allocated memory describes live

allocations known to PyTorch; reserved memory also includes cached allocator blocks. Neither value alone identifies tensor ownership, so phase measurements and component accounting must accompany peak statistics.

Part 3 - Quantitative Prediction Before Execution

Choose configuration $c = (b, K, P, C)$ to maximize

$$J(c) = B_{\text{eff}}(c)$$

subject to

$$M_{\text{peak}}(c) + M_{\text{safety}} \leq M_{\text{budget}}, \quad E(c) \leq E_{\max}, \quad R_{\text{skip}}(c) \leq R_{\max}.$$

Among equal effective batches, rank by validated throughput and stability.

Build a candidate table containing predicted fixed and variable memory, effective batch, micro-steps, throughput, numerical risk, and safety margin. Define admission, fallback, and stopping rules before benchmarking.

Write the prediction before running CUDA. Every predicted quantity must have a unit, a scope, and an assumption. At minimum, predict effective samples per optimizer update, optimizer-update count, peak allocated VRAM, peak reserved VRAM, synchronized update latency, throughput, and the gradient statistic that will decide correctness.

Part 4 - Experimental Design and Instrumentation

Compare full batch, static accumulation, best static combined strategy, and adaptive engine from identical states. Test steady and variable sequences, forced OOM, partial windows, and overflow. Report capacity, peak VRAM, headroom, latency distribution, throughput, accepted update frequency, gradient error, retries, scale history, and final metric.

Use fixed seeds, identical initial model and optimizer states, the same ordered samples, the same precision policy, and the same model mode for controlled comparisons. Warm up CUDA before timing, synchronize only at deliberate measurement boundaries, reset peak statistics immediately before the measured region, and report median together with a tail statistic across repeated trials. Keep profiler runs separate from final latency measurements.

Part 5 - PyTorch/CUDA Implementation

Complete the following structure. The missing operations are intentionally represented by extttEllipsis; implement them from the contract, predictions, and measurement requirements above. Do not replace the experiment with a library trainer that hides accumulation boundaries or memory observations.

```
from dataclasses import dataclass
import torch

@dataclass
class EngineConfiguration:
    micro_batch_size: int
    accumulation_steps: int
    effective_batch_size: int
    autocast_dtype: object
    maximum_gradient_norm: float

@dataclass
```

```
class EngineBudget:
    vram_budget: int
    safety_margin: int
    maximum_retries: int

class MemoryAwareAccumulationEngine:
    def __init__(
            self,
            model,
            optimizer,
            loss_function,
            configuration,
            budget):
        self.model = model
        self.optimizer = optimizer
        self.loss_function = loss_function
        self.configuration = configuration
        self.budget = budget
        self.scaler = ...
        self.history = []

    def split_effective_batch(self, inputs, targets):
        micro_batches = ...
        return micro_batches

    def execute_window(self, inputs, targets):
        window_checkpoint = ...
        micro_batches = ...

        self.optimizer.zero_grad(set_to_none=True)

        for micro_step_index, micro_batch in enumerate(micro_batches):
            # Run autocast forward and weighted backward.
            ...

        # Unscale, validate, clip, and commit exactly once.
        ...

        memory_report = ...
        numerical_report = ...
        return memory_report, numerical_report

    def recover_from_oom(self, failed_window):
        # Roll back and propose a smaller micro-batch configuration.
        ...

    def benchmark(self, batches):
        benchmark_report = ...
```

```
        return benchmark_report

def E19(
        model,
        optimizer,
        loss_function,
        initial_configuration,
        budget):
    engine = MemoryAwareAccumulationEngine(
        model,
        optimizer,
        loss_function,
        initial_configuration,
        budget)
    return engine
```

Part 6 - Measurement, Validation, and Failure Analysis

Require repeated feasibility, exact sample accounting, bounded retries, clean rollback, and gradient equivalence. The engine must fail explicitly when no configuration satisfies the budget instead of silently reducing the learning objective.

Validate loss, every named parameter gradient, updated parameters, and optimizer state against the declared reference using dtype-appropriate absolute and relative tolerances. Record missing gradients, zero fractions, non-finite values, OOM stage, and unsuccessful configurations instead of deleting them. Separate theoretical prediction, measured value, absolute error, relative error, and interpretation in the final report.

Part 7 - Engineering Interpretation and Philosophy

A memory-aware accumulation engine is a scheduler for training work. Its final product is not only a large effective batch, but a reproducible evidence trail showing that every committed update respected memory and numerical constraints.

The final recommendation must state which resource was saved, which cost increased, where the critical path moved, and under which workload and hardware the conclusion is valid. Preserve negative results and limitations: a memory technique that fits a larger workload but violates numerical equivalence, throughput requirements, or operational headroom has not solved the complete engineering problem.

19.24 DMM22

Chapter Title: Adaptive Batch Sizing Under Dynamic VRAM Budgets
Specialization: Dynamic Memory Management and CUDA Systems

19.24.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Batch Size, Sequence Length, Activation Memory, and Peak VRAM Scaling

Part 1 - Exercise Contract and Why It Matters

Build a memory ledger for one complete training update, from input transfer through gradient clearing. Identify who owns each allocation, when it becomes live, when it can be released, and whether its size depends on batch size b, padded sequence length L, hidden width d, or optimizer choice. Produce one lifecycle diagram, a numerical prediction sheet, and a phase measurement table. Explain why doubling the batch does not necessarily double peak VRAM.

Use a small MLP first and a Transformer-style block second. Prerequisites are autograd, tensor byte counts, mixed precision, and gradient accumulation from DMM19–DMM21. Use one CUDA device and eager execution initially. Training and inference need separate contracts: a successful forward pass cannot certify a training batch. Keep a logical optimizer batch distinct from the physical micro-batch that determines the immediate working set.

Part 2 - System and Memory Mental Model

Parameters persist across updates. Gradients appear during backward and may persist through accumulation. Optimizer state can be created lazily during the first update. Inputs, saved activations, attention intermediates, and operator workspaces have overlapping lifetimes. The peak is the maximum of their simultaneous totals; it is not the sum of separate maxima of each component. Views can share storage, so adding every tensor's logical byte count can double-count the same allocation.

Let A be live allocated bytes, R reserved allocator bytes, F device-global free bytes, and T device total bytes. Normally $A \le R$, while $T - F$ also includes other processes, CUDA context allocations, and memory outside the PyTorch allocator. Cached bytes $R - A$ are not all guaranteed to be reusable for an arbitrary next allocation. Fragment size, pending stream work, and allocator backend matter. Treat these as different observations, not interchangeable definitions of VRAM use.

For ordinary attention that materializes score matrices, a term proportional to bhL^2 can dominate. An efficient attention backend can avoid retaining that full matrix, so the same model dimensions do not imply the same memory curve. Record the actual backend and masks. A growing loss-history list that retains graphs is a lifetime bug, not an intrinsic batch-scaling law.

Part 3 - Quantitative Prediction Before Execution

Let P denote unique parameter elements and s bytes per stored element. Tensor payload is Ns bytes; $1\,\mathrm{MiB} = 2^{20}$ bytes and $1\,\mathrm{GiB} = 2^{30}$ bytes. A deliberately simplified FP32 Adam ledger is $4P$ parameter bytes, $4P$ gradient bytes, and $8P$ moment bytes, or $16P$ in total, excluding scalar state, workspaces, and temporary optimizer buffers. Do not apply this estimate blindly to every optimizer or precision policy.

$$M_{\mathrm{peak}}(b, L) = \max_{\tau} \{ M_{\mathrm{persistent}}(\tau) + M_{\mathrm{activation}}(b, L, \tau) + M_{\mathrm{workspace}}(b, L, \tau) \}.$$

At fixed shape and backend, a local approximation is

$$\widehat{M}(b, L) = M_0 + \alpha bLd + \beta bhL^2 + W(b, L).$$

Coefficients include layer count, dtype, and saved-tensor multiplicity. They are empirical, not universal constants.

For an illustrative fixed-length workload, let $M_0 = 2\,\mathrm{GiB}$ and the variable cost be $96\,\mathrm{MiB}$ per sample. Predict peaks for $b = 8, 16, 32$; then repeat with a $256\,\mathrm{MiB}$ workspace appearing at $b = 24$. Compute the ratios and show why a local linear model can miss a step change. Before CUDA execution, predict the peak phase and what changes when sequence length doubles.

Part 4 - Experimental Design and Instrumentation

1. Record GPU model, device identity, VRAM size, driver, CUDA runtime, PyTorch version, dtype, optimizer, model mode, attention backend, and allocator configuration. Fix CPU and CUDA seeds.

2. Save identical initial states on CPU. Measure the cold first update separately from warmed updates, including lazy optimizer initialization. Use a CPU master batch and transfer only the active physical batch.

3. Synchronize at measured-region boundaries and reset peak statistics there. Observe allocated and reserved bytes separately. Use the `mem_get_info` API in `torch.cuda` for a global snapshot. Global free memory is not a historical peak monitor.

4. Mark input transfer, forward, backward, optimizer update, and cleanup. End-of-phase counters can miss within-phase peaks; collect peak counters for the whole update and use separate diagnostic runs for finer attribution.

5. Sweep b at fixed L, then L at fixed b. Keep model width, layer count, loss, mode, and optimizer unchanged. Store scalar observations on CPU.

Use CUDA events for device timing and a synchronized wall clock for the end-to-end region. Report which includes transfer and allocation. Avoid per-operator synchronization in the throughput run.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region, and measure complete updates. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every `...` marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch
import torch.nn as nn

@dataclass
class MemoryObservation:
    phase: str
    allocated_bytes: int
    reserved_bytes: int
    peak_allocated_bytes: int
    peak_reserved_bytes: int
    global_free_bytes: int

def describe_unique_storage(model, optimizer):
```

```
    # Avoid counting aliased storage more than once.
    ledger = ...
    return ledger

def predict_peak(ledger, batch_size, sequence_length):
    # Separate persistent, activation, and workspace assumptions.
    prediction = ...
    return prediction

def observe_phase(device, phase):
    # Read counters and create a CPU-only observation.
    observation = ...
    return observation

def run_instrumented_update(model, optimizer, cpu_batch, policy):
    # Transfer only the active batch.
    ...
    # Measure forward, backward, update, and cleanup lifetimes.
    ...
    observations = ...
    return observations

def compare_cold_and_warm_runs(factory, cpu_batch, policy):
    # Recreate state and identify lazy allocations.
    report = ...
    return report

def E00(factory, cpu_batch, policy):
    # Produce the ledger, predictions, and measured phase table.
    result = ...
    return result
```

Part 6 - Measurement, Validation, and Failure Analysis

Your table must contain phase, b, L, predicted bytes, allocated bytes, reserved bytes, global free bytes, and a separate explanation of the residual. Check non-finite loss or gradients before accepting a run. For a small deterministic FP32 MLP, compare gradients and one parameter update to a reference with declared absolute and relative tolerances; initial tolerances such as 10^{-6} and 10^{-4} must be justified by the observed scale.

Fail the exercise if activation and workspace categories are silently merged, if first-update state is omitted, or if the report adds peaks from non-overlapping phases. Deliberately retain an output graph in a diagnostic run and distinguish that leak from a stable allocator cache. Restore the baseline before the final comparison.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

A useful mental model predicts both a value and the conditions under which that value stops being reliable. State whether the bottleneck comes from persistent model state, shape-dependent intermediates, a workspace transition, or the available device budget. These causes require different interventions.

End with two falsifiable claims: one about changing b at fixed L, and one about changing L at fixed b. Identify the next measurement that could disprove each. The remainder of this chapter turns this ledger into an admission and scheduling policy; it does not replace measurement with a single bytes-per-sample constant.

19.24.2 Exercise 01

Exercise Title: Measure Peak CUDA Memory Across Increasing Batch Sizes and Build an Empirical Batch-to-VRAM Scaling Curve

Part 1 - Exercise Contract and Why It Matters

Create a reproducible batch-capacity dataset for a fixed CUDA training workload. Measure complete updates for increasing physical batch sizes, then fit and inspect a local scaling curve. Deliver raw trial records, an allocated-memory plot, a reserved-memory plot, and a latency or throughput plot with identical batch-size axes.

The primary artifact is evidence, not a fitted line alone. Every point must identify precision, input shape, warm-up policy, initial state, outcome, and repetition index. Begin with small batches and stop at a predeclared bound or a recoverable OOM. Training data should reside on CPU until the active batch is needed so the input pool does not dominate your measured capacity.

Part 2 - System and Memory Mental Model

Measured peaks combine a fixed baseline with shape-dependent work. An increasing sweep may reuse allocator blocks from earlier candidates, whereas a fresh worker begins with a different cache history. Both are valid experiments but answer different questions. Use isolated workers for an approximately independent capacity curve and a separate in-process sweep to study the intended runtime's cache history.

Warm-up removes some startup timing effects but can also initialize optimizer state and trigger kernel or workspace selection. A candidate must survive its cold lifecycle as well as repeated steady execution. Resetting peak statistics does not free tensors, initialize missing optimizer state, or subtract the baseline. A peak counter is an absolute process measurement since reset.

Feasible memory capacity and efficient execution capacity differ. The largest batch can lose throughput through kernel choices, transfer cost, or extra padding. Keep failed trials as censored outcomes with their phase; an OOM is not an exact measurement of the unknown requested peak.

Part 3 - Quantitative Prediction Before Execution

For two feasible measurements at fixed shape,

$$\widehat{\alpha} = \frac{M(b_2) - M(b_1)}{b_2 - b_1}, \qquad \widehat{M}_0 = M(b_1) - \widehat{\alpha} b_1.$$

For a fit over several points, inspect residuals $e_i = M_i - \widehat{M}(b_i)$ and the largest positive residual, since underprediction is the dangerous direction. Estimate uncertainty using repeated runs or held-out candidates rather than treating an in-sample fit as a guarantee.

Suppose illustrative peaks are 2.5 GiB at $b = 8$ and 3.0 GiB at $b = 16$. Derive the slope, intercept, and prediction at $b = 24$. Predict how your conclusion changes if that held-out batch measures 4.2 GiB. For measured wall time t_i seconds, report $Q_i = b_i/t_i$ samples/s. Later, for variable-length data, also report useful tokens/s; samples/s alone will not remain comparable.

Part 4 - Experimental Design and Instrumentation

1. Fix architecture, optimizer, dtype, and shape. Use candidate batches such as $1, 2, 4, 8, 12, 16$, extending only after successful checks. Declare the maximum candidate before probing.

2. In each worker, create identical state, record a cold update, then use at least five warm-up updates and ten measured updates if practical. Restore comparable logical state for correctness comparisons; do not mix those comparisons with uncontrolled warm-up drift.

3. Record absolute peak allocated and reserved bytes plus allocated baseline. Measure complete-update wall and device time separately. Save OOM stage and error class without retaining GPU tensors or traceback objects.

4. Repeat selected candidates and randomize order in a control. Fit on one subset and predict held-out intermediate and larger batches.

5. Plot individual points and a labelled fit. Mark failures as failures, never as zero-memory samples. Use MiB or GiB consistently.

Store creation settings with results so the curve can be regenerated after a backend or driver change. Keep profiler runs separate from timing runs.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region, and measure complete updates. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import time
import torch

@dataclass
class TrialRecord:
    batch_size: int
```

```
    repetition: int
    status: str
    peak_allocated_bytes: object
    peak_reserved_bytes: object
    wall_seconds: object
    failure_phase: object

def run_candidate_worker(config, batch_size, repetition):
    # Create and warm the exact workload; measure complete updates.
    record = ...
    return record

def collect_scaling_trials(config, batch_sizes, repetitions):
    # Keep failures and successful observations separately.
    records = ...
    return records

def fit_memory_curve(training_records):
    # Do not use held-out trials to fit the model.
    fitted_model = ...
    return fitted_model

def evaluate_prediction(fitted_model, held_out_records):
    # Report signed residuals and unsafe underprediction.
    errors = ...
    return errors

def export_curves(records, fitted_model, output_path):
    # Export CPU-only tables and labelled plots.
    ...

def E01(config, batch_sizes, repetitions, output_path):
    records = ...
    fitted_model = ...
    validation = ...
    return records, fitted_model, validation
```

Part 6 - Measurement, Validation, and Failure Analysis

Require consistent units, a reproducible worker seed, and evidence that each successful point includes backward and optimizer update. Compare warm-up and measured peaks so a lower steady-state number cannot hide a cold-start failure. Report median and p95 latency; state the repetition count and the limited precision of tail statistics from a small sample.

Reject a curve if live memory grows across identical repetitions without an explained lifetime change, if timing ends before queued GPU work completes, or if persistent GPU copies of the dataset contaminate the baseline. Audit baseline recovery after each candidate. A failed fit is useful: preserve its residual pattern for E07 rather than deleting inconvenient points.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved

bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

An empirical curve is conditional on architecture, shape, precision, allocator, and execution history. State the interpolation range supported by data and the extrapolation range that remains speculative.

Recommend a candidate for subsequent search using both uncertainty and runtime cost. Explain why a curve that accurately predicts allocated bytes may still miss device-global pressure or fragmentation. Its engineering value lies in making uncertainty visible before a controller begins acting on the estimate.

19.24.3 Exercise 02

Exercise Title: Estimate the Largest Safe Batch Size from Free VRAM Before Launching the First Training Step

Part 1 - Exercise Contract and Why It Matters

Build a conservative preflight estimator that proposes an initial physical batch before the target job performs its first update. Inputs must include a device snapshot, a workload fingerprint, future persistent allocations, a memory model, and a safety policy. Output a proposed batch, its assumptions, and a reasoned refusal when even one sample is not admissible.

Distinguish zero-trial prediction from a calibrated estimate using earlier measurements. Reuse an E01 profile only when shape, optimizer, dtype, and backend match. An estimator with no profile must state its weaker evidence and use analytic bounds or a small conservative proposal. Predictions remain provisional until a complete update validates them.

Part 2 - System and Memory Mental Model

Free VRAM is an instantaneous device-global observation. It can fall between measurement and allocation. A snapshot before model creation differs from one after loading parameters: define the boundary explicitly. Count only future incremental demand against remaining free memory; otherwise the estimator may charge an already resident model twice.

Conversely, parameters exclude future gradient buffers, lazy optimizer moments, inputs, activations, and temporary update buffers. Omitting these creates an optimistic first-step estimate. Cached memory already owned by this process may help, but $R - A$ does not guarantee that an arbitrary allocation can use all those bytes. Use zero cache credit for a conservative base policy and study alternative credits separately.

The job budget and device availability are different constraints. A process can exceed its application budget while the GPU still has free memory. Admission must satisfy both. Isolation or resource reservation is needed for guarantees against arbitrary external demand; a preflight Python check cannot create that isolation.

Part 3 - Quantitative Prediction Before Execution

Let A_0 be allocated bytes already present, F_0 global free bytes, J the allocated-byte job cap, S_J job headroom, and S_D device headroom. Let ΔM_0 be future fixed demand and α incremental per-sample cost. A conservative model is

$$C_0 = \min(J - S_J - A_0,\ F_0 - S_D), \qquad \Delta M_0 + \alpha b \le C_0.$$

For positive α,

$$b_{\text{estimate}} = \left\lfloor \frac{C_0 - \Delta M_0}{\alpha} \right\rfloor .$$

If the numerator is negative, do not force a minimum batch of one. Report no feasible prediction. Round down to required batch granularity.

Assume $F_0 = 3\,\text{GiB}$, $S_D = 512\,\text{MiB}$, $\Delta M_0 = 768\,\text{MiB}$, and $\alpha = 80\,\text{MiB/sample}$, with a nonbinding job cap. Calculate the proposal and remaining headroom. Repeat when another process consumes 600 MiB. These are hypothetical inputs, not claims about your device.

Part 4 - Experimental Design and Instrumentation

1. Test the CPU-side arithmetic before CUDA. Include negative headroom, zero slope, missing profile, and a budget below resident state.

2. Snapshot after model loading but before the first target update. Identify gradient and optimizer allocations still absent. Record time, device identity, and profile provenance.

3. Generate proposals under several margins and later compare them to full-step capacity. Freeze predictions so measured results cannot retroactively change the original proposal.

4. Measure overshoot, undershoot, and refusal rates. Count unsafe overestimates separately from conservative proposals.

5. Change length, optimizer, or precision to create stale profiles. Require fingerprint mismatch detection before reusing calibration.

Hint: `torch.cuda.mem_get_info(device)` returns bytes. Convert display units at export time, not repeatedly inside admission logic.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region, and measure complete updates. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class PreflightSnapshot:
    allocated_bytes: int
    reserved_bytes: int
    free_bytes: int
```

```
    total_bytes: int
    timestamp_seconds: float

@dataclass
class CapacityProposal:
    batch_size: object
    predicted_increment_bytes: object
    admissible: bool
    reason: str

def capture_preflight_snapshot(device):
    snapshot = ...
    return snapshot

def validate_profile(profile, workload_fingerprint):
    # Refuse incompatible or insufficient calibration.
    verdict = ...
    return verdict

def estimate_future_fixed_bytes(model_spec, optimizer_spec):
    # Count only allocations absent at the snapshot boundary.
    future_fixed_bytes = ...
    return future_fixed_bytes

def propose_initial_batch(snapshot, profile, budgets, limits):
    # Respect both job and device headroom constraints.
    proposal = ...
    return proposal

def compare_prediction_with_capacity(proposal, measurements):
    comparison = ...
    return comparison

def E02(device, profile, workload_fingerprint, budgets, limits):
    snapshot = ...
    proposal = ...
    return proposal
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate hand-computed byte-level cases. Predicted increment plus device margin must not exceed snapshot availability under the chosen cache policy. Check that increasing the margin cannot increase the proposed batch when other inputs remain fixed.

Classify errors as stale measurement, wrong fixed-state accounting, incorrect slope, workspace transition, or external interference. Do not label every OOM an arithmetic failure. Report the largest measured candidate separately from the largest repeatedly safe candidate. Show an explicit no-fit case, since silently proposing one sample can create an endless retry loop.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved

bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Preflight estimation reduces expensive trial work; it is not proof of future availability. Explain the cost of conservatism and what evidence would justify reducing it. Preserve the distinction between a good first proposal and a validated operating point.

Conclude with a policy for missing or stale calibration: conservative launch, controlled probing, or explicit deferral. Tie that decision to failure cost and available headroom rather than inventing a universal safety factor.

19.24.4 Exercise 03

Exercise Title: Implement a Linear Search That Increases Batch Size Until the First CUDA Out-of-Memory Failure

Part 1 - Exercise Contract and Why It Matters

Implement an ascending search over integer batch sizes. Increase from a small candidate by a declared increment until the first genuine CUDA allocation OOM or configured ceiling. Return the last validated success, first failing candidate, complete history, and a status distinguishing a found boundary from a bounded search without failure.

Every trial must represent complete updates and repeated feasibility, not model construction alone. Search is disposable: trial updates must not advance the real job or consume its sample cursor. Use fresh isolated CUDA workers as the primary design, making cleanup and state equivalence explicit at the cost of process startup time.

Part 2 - System and Memory Mental Model

OOM may occur during transfer, forward, backward, lazy optimizer initialization, or an update workspace allocation. A backward failure can leave partial gradients; an optimizer failure may follow partial parameter or state mutation. Retrying that object without restoration changes the experiment.

Use the spawn process model and create CUDA state inside the child. Retain CPU-only configuration and results in the parent. Worker exit releases its allocations, but external pressure may still change between trials. In-process alternatives need reference cleanup and full-state restoration. Emptying cache cannot free live tensors or repair mutated optimizer state.

Catch the specific CUDA OOM condition. Shape errors, unsupported kernels, device assertions, and unrelated runtime exceptions are not capacity evidence. Fatal device errors may require discarding the CUDA context. A timeout or crashed worker must have a distinct status rather than being guessed to mean OOM.

Part 3 - Quantitative Prediction Before Execution

For $b_k = b_{\text{start}} + k\Delta b$, let $\phi(b_k)$ mean that every required trial update completed and passed numerical checks. A first failure brackets a boundary only at the tested resolution:

$$b_{\text{last success}} \leq b_{\max} < b_{\text{first failure}},$$

assuming a stable budget and monotone feasibility. If $\Delta b > 1$, untested integer batches remain inside the interval.

For a true threshold of 19, a start of 1, and increments of 1 versus 4, predict trial counts, last success, first failure, and bracket width. For a failure on the candidate grid, the trial count is $1+(b_{\text{fail}}-b_{\text{start}})/\Delta b$. Add startup, warm-up, and repeated-update cost to predict total search time; counting function calls alone can hide most of the cost.

Part 4 - Experimental Design and Instrumentation

1. Build a CPU fake trial function with configurable threshold, unrelated exception, timeout, and no-fit outcome. Validate bookkeeping without GPU allocation.

2. Recreate each candidate from identical CPU state and ordered data. Perform a cold update plus repeated warmed updates. Return only scalar statistics, error class, and failure phase.

3. Set maximum candidate, attempts, and timeout. Record pre-trial headroom; stop on changed environment rather than claiming a stationary capacity boundary.

4. Rerun the last success in a fresh worker after observing the first OOM. Explain inconsistency and lower confidence if necessary.

5. Keep real training untouched. Subsequently launch a new production instance at a separately chosen safe operating batch.

The deliberate OOM belongs in a bounded isolated experiment. Do not terminate unrelated CUDA processes as a cleanup strategy.
Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region, and measure complete updates. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import multiprocessing as mp
import torch

@dataclass
class SearchLimits:
    first_batch: int
    increment: int
    maximum_batch: int
    maximum_attempts: int
    timeout_seconds: float
```

```
def trial_worker(configuration, batch_size, result_channel):
    # Initialize CUDA in this child and run complete trials.
    # Distinguish allocation OOM from every other failure.
    ...

def launch_isolated_trial(configuration, batch_size, timeout):
    # Use spawn, collect scalar results, and join safely.
    trial_result = ...
    return trial_result

def linear_capacity_search(trial_function, limits):
    # Maintain last success, first OOM, and all attempts.
    # Distinguish no-fit, ceiling, timeout, and other error.
    search_report = ...
    return search_report

def revalidate_last_success(configuration, search_report):
    validation = ...
    return validation

def E03(configuration, limits):
    # Do not pass real training state into trial workers.
    report = ...
    return report

def main():
    # Build CPU-only configuration and invoke the exercise.
    ...

if __name__ == "__main__":
    main()
```

Part 6 - Measurement, Validation, and Failure Analysis

Check first-candidate OOM, success at ceiling, non-unit increments, and no feasible candidate. Verify that the returned success was actually tested. Log bytes, shape, precision, duration, repetition count, and failure stage. A first-OOM boundary is not a strictly safe production configuration.

Verify cleanup through the global snapshot after worker exit, allowing for unrelated activity and driver state. Compare identically initialized successful runs for finite losses and expected parameter changes. Reject searches that advance real data iterators, ignore worker failures, or retain GPU tensors through stored exception tracebacks.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Linear search offers a transparent baseline for faster searches. Its cost grows with the number of candidates, but dense evidence can reveal irregular behavior that sparse search misses.

Explain what it proved: feasibility at specific conditions, increment, and confidence. Separately explain what remains unknown about future input shapes, external processes, and sustained operation. That distinction matters when binary search and adaptive control reuse this trial interface.

19.24.5 Exercise 04

Exercise Title: Replace Linear Search with Binary Search to Find the Maximum Feasible Batch Size Using Fewer Trial Runs

Part 1 - Exercise Contract and Why It Matters

Replace E03's ascending search with bracket construction and binary refinement. Reuse exactly the same complete-update trial contract so the comparison measures search strategy rather than different definitions of feasibility. Return tested bounds, candidate history, probe count, wall time, and a termination reason. A ceiling reached without failure is a lower-bound result, not proof of an unconstrained maximum.

Support a known bracket and an unknown bracket. For the latter, use bounded expansion before refinement. Include no-fit at batch one, exact endpoint success, and a ceiling below the actual capacity. Preserve the real training state through the disposable-worker design.

Part 2 - System and Memory Mental Model

Binary search needs a stable predicate: success at b should imply success at smaller candidates under the same shape, precision, state, and budget. Kernel selection, workspace transitions, allocation history, and external pressure can violate practical monotonicity. Fewer probes therefore buy less direct evidence about irregular regions.

Maintain a feasible lower bound that was actually tested and an infeasible upper bound that was actually tested. A synthetic sentinel is not an OOM measurement. With stride constraints, search indices in the admissible candidate list rather than mixing integer batch values and index arithmetic. Repeated contradictory outcomes mean the predicate is uncertain; they are not permission to keep an arbitrary earlier answer.

Use fresh workers to stabilize allocator state and repeat near-boundary candidates. This reduces avoidable variation but cannot freeze another process's demand. If monotonicity is contradicted, report the contradiction and fall back to a bounded local sweep or a conservative tested configuration.

Part 3 - Quantitative Prediction Before Execution

Given a tested feasible l and infeasible u, an integer midpoint satisfies

$$m = \left\lfloor \frac{l+u}{2} \right\rfloor, \qquad l < m < u$$

while $u - l > 1$. The bracket width decreases roughly by half per probe. Refinement requires at most $\lceil \log_2(u-l) \rceil$ probes for a monotone integer predicate, excluding endpoint tests and repetition.

Starting from $l = 8$ and $u = 64$ with a hypothetical threshold of 37, write the complete predicted probe sequence and bound changes by hand. Compare with a unit-stride linear search.

For expansion, include its separate $O(\log b_{\max})$ probes and the ceiling checks. Compute total runtime as

$$t_{\text{search}} = \sum_i (t_{\text{startup},i} + t_{\text{warm},i} + t_{\text{trial},i}),$$

because faster probe counts do not guarantee lower time when workers differ in launch overhead or candidate size.

Part 4 - Experimental Design and Instrumentation

1. Implement a deterministic fake predicate for capacities 0, 1, 37, and a capacity beyond the ceiling. Verify termination and exact tested bounds.
2. Add a deliberately nonmonotone fake predicate and a time-varying one. Define how contradictory measurements invalidate the bracket.
3. Run linear and binary search against the same CUDA worker, seed, shape, optimizer state, and repetition requirements.
4. Use bounded expansion only if no upper failure is known. Log each expansion and never allocate beyond the declared candidate ceiling.
5. Recheck the proposed maximum and its immediate admissible neighbor. Measure total search time and the time of this validation separately.

Record an environmental fingerprint per trial, including global free memory. The binary-search controller should remain CPU-only and independently testable.
Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region, and measure complete updates. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class CapacityBracket:
    last_feasible: object
    first_infeasible: object
    maximum_allowed: int
    environment_fingerprint: str
```

```
def establish_bracket(trial_function, initial_batch, limits):
    # Perform bounded expansion and distinguish ceiling from OOM.
    bracket = ...
    history = ...
    return bracket, history

def refine_bracket(trial_function, bracket, history):
    # Implement termination and preserve tested-bound invariants.
    result = ...
    return result

def detect_contradictions(history):
    # Compare outcomes only within compatible environments.
    contradictions = ...
    return contradictions

def validate_boundary(trial_function, candidate, limits):
    # Repeat the success and test the neighboring candidate.
    evidence = ...
    return evidence

def E04(trial_function, initial_batch, limits):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Require strict bracket shrinkage, no duplicate midpoint loop, and no invented successful endpoint. Test non-unit candidate strides and a bracket of width one. Every returned batch must have completed its full feasibility trial. Failures other than allocation OOM must leave the search with a distinct error status.

Compare final capacities with E03 only under comparable stable conditions. Report both calls saved and seconds saved. If results disagree, rerun a small neighborhood and inspect shape, budget, optimizer-state, and workspace differences. Do not hide disagreement by presenting only the faster method.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Binary search is an efficiency improvement built on an assumption about feasibility. State where that assumption was tested and where it remains uncertain. A short search log can be less persuasive than a longer stable one when the device is shared.

Choose whether your production runtime should use binary search, a local sweep, or profile-guided admission. Base the choice on measured probe cost, repeatability, and the risk of an incorrect boundary.

19.24.6 Exercise 05

Exercise Title: Add a Configurable VRAM Safety Margin and Explain Why Running at the Exact OOM Boundary Is Unsafe

Part 1 - Exercise Contract and Why It Matters

Extend the capacity policy with absolute and relative headroom, explicit accounting scope, and uncertainty allowance. Compare three operating choices: largest observed feasible batch, batch with fixed-byte margin, and batch with a combined fixed/relative margin. Deliver a capacity-versus-risk table and a justified operating recommendation.

The contract must define whether the cap concerns allocated bytes, reserved bytes, or device-global usage. Use allocated-byte admission plus a separate global-headroom observation as the base experiment. Label cached reservations separately. A successful allocation is not sufficient if the application margin was violated.

Part 2 - System and Memory Mental Model

Exact-boundary operation leaves no room for uncommon shapes, delayed frees, workspace variation, prediction error, or another process. Repeated success on one batch establishes only observed feasibility. A safety reserve reduces exposure to bounded disturbances; it cannot defeat unbounded demand or a race after the final check.

Absolute margins protect a minimum number of bytes, while relative margins scale with the selected budget. Define their composition explicitly: maximum or sum gives different policies. Do not silently change the rule between estimation and runtime. Distinguish model-error allowance from operational reserve so the same uncertainty is not charged twice accidentally.

Allocated and reserved peaks may occur at different instants. A high reserved value can reflect a harmless cache or a costly footprint on a shared GPU. Your job policy may limit both, but subtracting reserved bytes from global free memory again would double-count already unavailable memory.

Part 3 - Quantitative Prediction Before Execution

For a declared total job budget J, define

$$S_J = \max(S_{\text{absolute}}, \rho J), \qquad \widehat{M}_{\text{upper}}(b) = \widehat{M}(b) + U(b).$$

Admit a candidate only if

$$\widehat{M}_{\text{upper}}(b) + S_J \leq J.$$

Use an additional incremental device-availability check from E02. $U(b)$ represents prediction uncertainty, not the safety margin itself. A percentile of positive residuals is one empirical choice; it is not a hard upper bound.

For $J = 8\,\text{GiB}$, a $512\,\text{MiB}$ absolute margin, and $\rho = 0.10$, calculate the usable cap in bytes. Suppose a candidate predicts $7.1\,\text{GiB}$ and uncertainty is $0.2\,\text{GiB}$. Decide admission before measuring. Repeat for $J = 6\,\text{GiB}$ and explain which margin component binds. Compare these results with the sum rule.

Part 4 - Experimental Design and Instrumentation

1. Choose a baseline workload and its measured curve. Keep dtype, optimizer, shape, and initial state identical across policies.

2. Sweep several margins within an application cap smaller than physical VRAM. Count policy violations even if CUDA does not raise OOM.

3. Inject a bounded temporary workspace in diagnostic trials and include a longer input already declared in the disturbance envelope.

4. Run repeated updates, collecting peak allocated bytes, reserved peak, snapshot headroom, successful throughput, and time spent refusing work.

5. Recompute decisions from the stored log. The offline and online admission results must match for identical observations.

Use a common wall-clock evaluation horizon. Repeatedly rejecting all batches is safe only in a trivial sense; report lost useful work and no-fit duration.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region, and measure complete updates. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class SafetyPolicy:
    job_budget_bytes: int
    absolute_margin_bytes: int
    relative_margin: float
    device_margin_bytes: int
    uncertainty_mode: str

def calculate_job_margin(policy):
    margin_bytes = ...
    return margin_bytes

def estimate_upper_peak(profile, candidate, uncertainty):
    upper_peak_bytes = ...
    return upper_peak_bytes

def admit_candidate(snapshot, prediction, policy):
    # Check job scope and incremental device availability.
    decision = ...
```

```
    return decision

def audit_realized_margin(trial_record, policy):
    # A completed update can still violate the policy.
    audit = ...
    return audit

def E05(trial_function, profile, policies, disturbances):
    comparison = ...
    return comparison
```

Part 6 - Measurement, Validation, and Failure Analysis

Test zero and negative budgets, relative margins outside the allowed range, and a margin larger than the budget. A no-fit result must be explicit. Check monotonicity of admission as the margin grows and verify all arithmetic uses bytes before formatting.

Report violation count, OOM count, minimum observed headroom, useful throughput, and refusals separately. A headroom snapshot does not prove the minimum global free memory between samples. State sampling resolution and observation limits. Describe any bounded stress that defeated the selected margin and which assumption failed.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Safety is a resource tradeoff with an operating envelope. A larger reserve may waste capacity or may improve useful throughput by avoiding costly retries. Judge it using accepted work over total elapsed time.

Recommend a margin together with its measurement scope, uncertainty estimate, tested disturbances, and fallback behavior. Avoid presenting a fixed percentage as universally correct across devices, shapes, or workloads.

19.24.7 Exercise 06

Exercise Title: Compare Estimated Batch Capacity Against Measured Capacity for CNN, MLP, and Transformer-Style PyTorch Workloads

Part 1 - Exercise Contract and Why It Matters

Compare prediction quality across MLP, CNN, and Transformer workloads. Build and measure a separate memory profile for each, predict a safe batch under a common application cap, and validate it using E03 or E04. Deliver an architecture specification, predicted and measured capacities, and an explanation of the error for each family.

The purpose is not to declare one architecture more efficient in general. Their tasks, input units, and computations differ. Compare prediction error and capacity within each fixed workload; treat cross-family throughput as separate sample definitions unless you establish a common task.

Part 2 - System and Memory Mental Model

An MLP often exposes dense hidden activations with approximately linear batch dependence. A CNN adds spatial feature maps, convolution workspace choices, layout effects, and possibly batch-dependent normalization. A Transformer adds padded length, attention backend, mask shape, and potentially quadratic intermediates. Similar parameter counts therefore do not imply similar peaks.

Keep complete-update accounting common: parameters, gradients, optimizer state, inputs, saved activations, and temporary buffers. Use FP32 and a fixed optimizer initially to avoid confounding architecture with precision. BatchNorm changes statistics with the physical batch, so use a batch-independent baseline or label that semantic difference explicitly.

Each profile needs a fingerprint containing layer sizes, input shape, precision, optimizer, model mode, and backend. Reusing an MLP slope for a CNN is intentionally a negative control, not a plausible generic estimator. If a workload fails at batch one, lower its dimensions and publish the change.

Part 3 - Quantitative Prediction Before Execution

Use local models such as

$$\begin{aligned}\widehat{M}_{\mathrm{MLP}}(b) &= M_0 + \alpha b d,\\ \widehat{M}_{\mathrm{CNN}}(b) &= M_0 + \alpha b \sum_{\ell} C_\ell H_\ell W_\ell + W_{\mathrm{conv}}(b),\\ \widehat{M}_{\mathrm{attn}}(b, L) &= M_0 + \alpha b L d + \beta b h L^2 + W(b, L).\end{aligned}$$

These equations are modelling hypotheses, not exact architecture formulas. For measured safe capacity b^* and predicted $\widehat{b}$, report

$$e_b = \widehat{b} - b^*, \qquad e_{\mathrm{relative}} = \frac{\widehat{b} - b^*}{b^*},$$

with no relative-error value when $b^* = 0$. Positive error is an unsafe overestimate; negative error represents conservatism.

Before running, rank expected sensitivity to doubling spatial resolution, hidden width, and sequence length. Predict which rank might change with an efficient attention backend. Include byte predictions at two small batches per architecture and reserve a third for validation.

Part 4 - Experimental Design and Instrumentation

1. Define small reproducible architectures with documented dimensions. Use synthetic CPU inputs and compatible regression or classification targets. Disable dropout for exact numerical controls.

2. Hold optimizer, precision, application budget, and safety policy fixed. Measure separate cold and warm baselines for each architecture.

3. Calibrate on small feasible batches, then freeze the prediction. Use a separate capacity search to obtain measured safe capacity.

4. Repeat after changing only one shape parameter: image resolution, feature width, or sequence length. Require profile invalidation or explicit shape-conditioned prediction.

5. Produce one plot per workload and a shared prediction-error table. Keep confidence intervals or repetition ranges next to measured capacities.

Use the same complete-update timing definition. Record backend selection and layout instead of inferring them from model class names.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region, and measure complete updates. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch
import torch.nn as nn

@dataclass
class WorkloadSpecification:
    family: str
    input_shape: tuple
    model_dimensions: dict
    precision_name: str
    optimizer_name: str

def build_mlp(specification):
    model = ...
    return model

def build_cnn(specification):
    model = ...
    return model

def build_transformer(specification):
    model = ...
    return model

def calibrate_workload(specification, candidate_batches):
    profile = ...
    return profile

def measure_safe_capacity(specification, budget, search_limits):
    capacity_report = ...
```

```
    return capacity_report

def compare_prediction_errors(predictions, measurements):
    comparison = ...
    return comparison

def E06(specifications, budget, search_limits):
    # Preserve separate units, shapes, and profiles per workload.
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Require a finite loss and finite gradients for every accepted architecture. Check a small deterministic update against that architecture's own reference, using dtype-appropriate tolerances. There is no meaningful equality test between unrelated model outputs.

Detect stale cross-family calibration, omitted convolution workspaces, inconsistent optimizer state, and incompatible target shapes. Keep the batch-one no-fit outcome in the table if it occurs. Explain prediction errors using measured phases or profiler evidence rather than stereotypes about model families.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

A reusable estimator needs a reusable interface, not one universal slope. Identify which metadata must select a profile and which measurements can be shared across families.

Conclude with a profile-selection policy and the conditions that trigger recalibration. Explain whether the limiting uncertainty came from persistent state, shape, workspace selection, or changing device availability.

19.24.8 Exercise 07

Exercise Title: Detect Nonlinear Batch Memory Growth Caused by Temporary Buffers and Operator Workspace Allocations

Part 1 - Exercise Contract and Why It Matters

Investigate a memory curve that departs from a linear batch model. Locate the transition, associate it with an execution phase, and separate a workspace change from graph retention, allocator history, or input-shape changes. Deliver a residual plot, a phase trace, and evidence supporting or rejecting the proposed cause.

Do not assume your GPU will naturally exhibit a convenient discontinuity. If no natural transition is observed, publish that negative result and use an explicitly labelled synthetic temporary buffer to validate your detector. The controlled demonstration must never be presented as a measured kernel implementation detail.

Part 2 - System and Memory Mental Model

Operator implementations may switch algorithms as shapes change. Workspace size can then jump even when tensor payload grows smoothly. The temporary buffer may be released before an end-of-phase observation, leaving only a peak counter or trace as evidence. Optimizer implementations can also allocate temporary tensor lists or buffers.

Reserved-memory steps can arise from allocator block acquisition without equivalent jumps in live tensor demand. Conversely, an allocation outside the PyTorch allocator may affect global usage without appearing in its tensor snapshots. Keep measurement scope attached to each causal claim.

Changing batch order may change allocator fragmentation and cache reuse. A steadily growing current allocation after cleanup points toward retained references; a reproducible isolated peak jump at one shape suggests a different mechanism. Neither observation alone identifies a specific kernel. Use a separate profiler run to connect the transition with an operator.

Part 3 - Quantitative Prediction Before Execution

For adjacent candidates, define a finite-difference slope

$$s_i = \frac{M(b_{i+1}) - M(b_i)}{b_{i+1} - b_i}.$$

Compare s_i with the baseline slope and variation across repetitions. A simple change model is

$$\widehat{M}(b) = M_0 + \alpha b + W_{\text{jump}}\, \mathbf{1}_{b \geq b_c}.$$

It describes one transition; it does not establish its physical cause. Predict the residual pattern if you fit one straight line across the jump.

For illustrative $M_0 = 1\,\text{GiB}$, $\alpha = 40\,\text{MiB/sample}$, $W_{\text{jump}} = 384\,\text{MiB}$, and $b_c = 20$, compute peaks at $b = 16, 19, 20, 21, 24$. Determine the slope immediately across the boundary and the conservative prediction error below and above it. Specify a detection threshold using repeated-run noise before examining real results.

Part 4 - Experimental Design and Instrumentation

1. Start from an E01 workload with frozen shapes, precision, backend settings, and optimizer. Collect a coarse sweep, then a dense local sweep around suspected transitions.

2. Compare isolated workers, ascending order, and descending order. Record allocated baseline, peak, reserved peak, and global snapshots.

3. Use `torch.profiler` with memory and shape recording in a diagnostic run. Treat its overhead separately from benchmark latency.

4. Trace forward, backward, and optimizer phases. If using allocator snapshots, record PyTorch version and allocator backend, and recognize memory outside the allocator as an observation limit.

5. Inject a temporary tensor of declared byte size in one chosen phase. Retain it only for that phase, then compare the detector's result with the known intervention.

If testing algorithm-related settings, change one setting per experiment and retain the default-setting result as a reference.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region, and measure complete updates. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch
from torch.profiler import profile, ProfilerActivity

@dataclass
class TransitionCandidate:
    left_batch: int
    right_batch: int
    observed_slope: float
    baseline_slope: float
    evidence_label: str

def calculate_local_slopes(records):
    slopes = ...
    return slopes

def detect_transitions(records, noise_model):
    candidates = ...
    return candidates

def run_diagnostic_profile(workload, batch_size, output_path):
    # Capture shapes, phases, and memory in a separate run.
    trace_summary = ...
    return trace_summary

def inject_temporary_workspace(phase, requested_bytes, device):
    # Leave the controlled allocation and lifetime to the learner.
    workspace = ...
    return workspace

def distinguish_peak_jump_from_retention(records, traces):
    diagnosis = ...
    return diagnosis
```

```
def E07(workload, candidate_batches, noise_model):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify that the detector rejects ordinary repeated-run noise and recognizes the labelled injected jump at the expected interval. Compare current post-cleanup allocation with peak allocation to distinguish retained storage from temporary demand. Check that diagnostic tensors are not kept in the measurement log.

Reject unsupported causal claims: a residual alone does not prove a convolution workspace, and a reserved-memory jump alone does not prove a larger activation tensor. If the transition disappears with isolated workers, report allocator history as a plausible explanation and test it directly.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

A capacity model should encode regimes when the data show regimes. Explain whether to use piecewise calibration, an uncertainty envelope, or more conservative admission around a transition.

State the evidence supporting each interpretation and what remains unobserved. Good engineering can conclude that a transition exists while remaining uncertain about its exact low-level allocation source.

19.24.9 Exercise 08

Exercise Title: Build a Batch Controller That Shrinks the Next Batch When Peak VRAM Crosses a High-Watermark Threshold

Part 1 - Exercise Contract and Why It Matters

Build a one-way controller that can hold or reduce the next physical batch. It observes a completed update's peak and a fresh budget snapshot, then selects an admissible batch for the next update. Deliver the decision trace, reasons for each shrink, a comparison against a static batch, and an explicit no-fit outcome when batch one cannot satisfy the policy.

This exercise does not yet attempt growth. Keep that separation so a shrink decision can be evaluated without a competing expansion rule. Preserve sample identity when smaller batches change the data iterator. For numerical equivalence tests, replay the same logical update through weighted accumulation; otherwise acknowledge that changing update batch changes training semantics.

Part 2 - System and Memory Mental Model

The controller has delayed information: a peak from update t describes past work, while the next input and budget can differ. Shrinking the next batch cannot undo a violation that

already occurred. Combine retrospective high-watermark feedback with prospective admission for the next known shape. Record both reasons when they disagree.

A naive proportional reduction based on total peak assumes all memory scales with batch. Persistent parameters and optimizer state invalidate that assumption. Predict the variable portion separately, and use a conservative tested fallback if calibration is unreliable. A smaller batch is not automatically enough when the budget falls below persistent state.

Operate at a clear boundary after an update is committed and transient graph references are released. Do not resize live inputs in the middle of backward. Keep the control function pure and CPU-only; GPU telemetry is an input, not a hidden side effect. Clear separation allows synthetic traces to exercise rare transitions without generating real OOMs.

Part 3 - Quantitative Prediction Before Execution

Let J_t be the current allocated-byte job cap and M_t the completed update peak. Define $u_t = M_t/J_t$ only for $J_t > 0$. A high watermark $0 < h < 1$ triggers review when $u_t \geq h$. If the desired next cap is hJ_{t+1} and a local model is $M_0 + \alpha b$, then

$$b_{\text{target}} \leq \left\lfloor \frac{hJ_{t+1} - M_0 - U}{\alpha} \right\rfloor,$$

where U is uncertainty. Apply integer, minimum, maximum, and device-headroom constraints explicitly. The threshold is not a substitute for E05's margin.

Assume $M_0 = 2\,\text{GiB}$, $\alpha = 128\,\text{MiB/sample}$, $b_t = 24$, and $J_{t+1} = 5\,\text{GiB}$ with $h = 0.85$. Predict a candidate with $U = 128\,\text{MiB}$. Compare it with blindly halving b_t and with proportional scaling of the total peak. Explain each policy's expected remaining headroom.

Part 4 - Experimental Design and Instrumentation

1. Create a deterministic budget trace with a steady segment followed by two reductions. Replay it through a pure controller before CUDA.

2. Choose a static baseline that is feasible initially. Feed the controller peak observations plus next-shape metadata and budget timestamps.

3. Apply the selected batch only at the next update boundary. Keep a CPU queue of unconsumed sample identifiers; smaller batches must not skip or duplicate samples.

4. Compare feedback-only shrink with feedback plus preflight admission. Count violations during the first update after each budget reduction.

5. Record previous batch, selected batch, utilization, cap, predicted peak, observed peak, reason, and accepted samples per wall-clock second.

Treat application-cap violations separately from CUDA OOM, since the device can have more physical memory than the test budget.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region, and measure complete updates. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class ShrinkObservation:
    completed_batch: int
    peak_bytes: int
    next_budget_bytes: int
    next_shape: tuple
    timestamp_seconds: float

@dataclass
class BatchDecision:
    selected_batch: object
    action: str
    reason: str
    predicted_peak_bytes: object

def evaluate_high_watermark(observation, policy):
    trigger = ...
    return trigger

def propose_smaller_batch(observation, profile, policy):
    # Separate fixed state from batch-dependent memory.
    decision = ...
    return decision

def admit_next_batch(decision, snapshot, next_shape, policy):
    validated_decision = ...
    return validated_decision

def consume_sample_prefix(cpu_queue, selected_batch):
    # Commit the cursor only when the training contract allows it.
    batch_record = ...
    return batch_record

def E08(workload, budget_trace, profile, policy):
    trace = ...
    return trace
```

Part 6 - Measurement, Validation, and Failure Analysis

Test equality at the threshold, budget zero, minimum batch, negative predicted headroom, and stale observations. The one-way controller must never increase batch size. A no-fit decision must lead to deferral or an explicit stop, not an infinite sequence of identical failing attempts.

Audit sample IDs and accepted update counts. For deterministic replay, compare the resulting logical gradient and parameters with the static reference. Record how many updates elapsed before headroom recovered after each disturbance. If only future updates recover, do not claim that the triggering violation was prevented.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Shrink control is a reaction to evidence and a proposal about future demand. Discuss its response delay, conservatism, and dependence on accurate shape metadata. Identify a disturbance it can handle and one it cannot.

Explain whether lowering the batch restored useful operation or merely postponed an unavoidable no-fit state. A controller needs an honest stopping condition as much as it needs a recovery action.

19.24.10 Exercise 09

Exercise Title: Build a Batch Controller That Gradually Increases Batch Size When Sustained VRAM Headroom Is Available

Part 1 - Exercise Contract and Why It Matters

Add cautious growth after sustained successful operation below a low watermark. Require a configurable observation window, a bounded growth step, and preflight validation of the proposed larger batch. Compare this policy with immediate growth and with a permanently conservative static batch. Report recovered capacity and the cost of failed or rejected probes.

Growth must use only observations available at decision time. A benchmark oracle may know future budgets, but the implemented controller must not. Do not increase the real learning batch accidentally: use a fixed logical batch when assessing equivalent training, and label variable-update-batch experiments separately.

Part 2 - System and Memory Mental Model

One low-memory update can be an unusually short input or a temporary budget increase. Sustained evidence reduces that noise but delays exploitation. Headroom alone does not establish that a larger batch improves throughput. The controller should require validated useful-work benefit or at least measure it after a bounded trial.

Define consecutive success using actual accepted optimizer updates, not attempted updates or micro-steps. An OOM, margin violation, overflow-skipped update, changed shape class, or stale budget may invalidate the observation window. Specify exactly which events reset it.

The counter and growth limit belong to the controller state. Warm-up and rejected trials must not masquerade as steady successful operation. When a proposed shape enters an unprofiled region, either run an isolated calibration trial or refuse speculative growth. Keep probing cost in the end-to-end throughput denominator.

Part 3 - Quantitative Prediction Before Execution

Let l be a low watermark and k the required consecutive accepted observations satisfying $u_t \leq l$. Define a counter c_t whose update you must specify for success, violation, skip, and profile change. A bounded proposal may be expressed as

$$b_{\text{proposal}} \leq \min(b_{\max},\, b_t + \Delta b,\, \lfloor (1+\gamma) b_t \rfloor).$$

Explain how integer rounding can prevent progress at small b_t and define an explicit admissible-grid rule rather than forcing unsafe growth.

For $b_t = 8$, $k = 4$, and an additive increment of two, manually replay the following utilizations with $l = 0.60$:

$$(0.55,\ 0.56,\ 0.82,\ 0.54,\ 0.53,\ 0.52,\ 0.51).$$

Predict the first growth opportunity. Then repeat with an overflow skip on the sixth observation. For a successful growth, compare additional useful samples with probe time:

$$Q_{\text{net}} = \frac{N_{\text{accepted}}}{t_{\text{training}} + t_{\text{probing}} + t_{\text{recovery}}}.$$

Part 4 - Experimental Design and Instrumentation

1. Use a synthetic trace containing sustained headroom, a single low outlier, a budget increase, and an abrupt budget decrease after growth.

2. Validate the counter and action sequence on CPU. Fix the rule for which events count as accepted updates.

3. Run the CUDA workload from a conservative batch. Admit each growth proposal against the current budget and next known shape.

4. Compare at least two patience values and two growth increments. Keep safety margin and shrink rule identical across conditions.

5. Measure time to recover a useful batch, rejected growth attempts, post-growth violations, and net throughput over the entire trace.

Use repeated trials with the same workload ordering. Report a no-benefit growth result if larger batches fit but do not improve useful throughput.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region, and measure complete updates. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class GrowthState:
    current_batch: int
    consecutive_accepted_low: int
    last_growth_update: object
    profile_fingerprint: str

def classify_growth_observation(observation, policy):
    classification = ...
    return classification

def update_patience_state(state, classification, policy):
    next_state = ...
    return next_state

def propose_growth(state, profile, next_shape, policy):
    # Apply integer limits and uncertainty-aware admission.
    proposal = ...
    return proposal

def validate_growth_benefit(before_records, after_records):
    benefit = ...
    return benefit

def E09(workload, trace, initial_state, policy):
    # Count probe and recovery time in the final comparison.
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Check off-by-one patience errors, repeated growth without fresh evidence, and a stale counter surviving an OOM. Test minimum and maximum batches, small multiplicative increments, and a low watermark never reached.

Your trace must explain every hold and growth action. Accepted work must exclude failed retries and overflow-skipped optimizer updates. State how small timing differences are treated as noise. Reject a policy that claims a gain solely by excluding its calibration or probe costs.
Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Growth is an investment in future throughput. Discuss when the observation window is too short to trust and when it is so long that available capacity goes unused.

Recommend a growth policy for one measured workload, with a stopping rule when larger batches provide no benefit. The goal is useful progress within constraints, not reaching the largest integer on the batch-size axis.

19.24.11 Exercise 10

Exercise Title: Add Hysteresis to Adaptive Batch Sizing and Prevent Batch Size from Oscillating Between Two Values

Part 1 - Exercise Contract and Why It Matters

Combine shrink and growth into a controller with distinct low and high watermarks, growth patience, and a cooldown. Demonstrate how a single threshold can alternate between two batch sizes under noisy observations, then measure whether the revised controller reduces unnecessary changes without delaying emergency shrink.

Deliver a state-transition table, a manually predicted decision trace, and a CUDA replay comparing the policies. Preserve each decision's raw observation, smoothed statistic if used, state, and reason. Smoothing must not conceal a hard admission failure.

Part 2 - System and Memory Mental Model

Changing batch size changes the measured utilization. A controller using the same threshold for both directions can create its own oscillation: the small batch looks underutilized, the larger batch looks overutilized, and each observation reverses the previous action.

A deadband $l < h$ separates growth and shrink triggers. Patience rejects short low-memory transients; cooldown limits repeated growth after a recent change. None guarantees stability for arbitrary noise, discrete batch jumps, or rapidly changing budgets. Quantization matters when moving one sample changes memory more than the deadband width.

Emergency admission failure must override growth cooldown. If smoothing is used for ordinary adaptation, retain raw peak and fresh budget checks for safety. State resets are part of the design: old patience should not survive a profile change, failed attempt, or a large drop in the available budget.

Part 3 - Quantitative Prediction Before Execution

Let u_t denote observed utilization and optionally use

$$\bar{u}_t = \lambda u_t + (1 - \lambda)\bar{u}_{t-1}, \qquad 0 < \lambda \leq 1.$$

Define growth below l, ordinary shrink above h, and hold in between. Calculate the memory change caused by one batch increment: $\Delta M \approx \alpha \Delta b$. Compare $\Delta M / J$ with the width $h - l$ to predict whether one discrete action can jump across the band.

For batches 12 and 16 producing utilizations near 0.72 and 0.86, predict a single-threshold controller at 0.80. Then try $l = 0.70$, $h = 0.90$, patience three, and cooldown two accepted updates. Construct a separate emergency trace where the budget suddenly drops. Measure switching count

$$N_{\text{switch}} = \sum_{t=2}^{n} \mathbf{1}_{b_t \neq b_{t-1}}$$

along with time to regain safe operation and accepted throughput.

Part 4 - Experimental Design and Instrumentation

1. Implement controller transitions as a pure function of state and observation. Hand-check boundary equalities at both watermarks.

2. Replay constant load with small measurement noise, alternating short and long inputs, a step budget decrease, and a temporary budget increase.

3. Compare single threshold, deadband only, and deadband with patience and cooldown. Keep growth increment and memory model fixed.

4. Execute one reproducible CUDA trace. Separate raw observations from any filtered utilization and log emergency overrides explicitly.

5. Measure switch count, rejected proposals, response delay, no-fit duration, margin violations, and useful throughput.

Do not tune thresholds on the same trace used for the final comparison. Hold out a second trace to expose overfitting to a particular pattern.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region, and measure complete updates. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class ControllerState:
    batch_size: int
    low_count: int
    cooldown_remaining: int
    filtered_utilization: object
    profile_fingerprint: str

@dataclass
class HysteresisPolicy:
    low_watermark: float
    high_watermark: float
    patience: int
    cooldown_updates: int

def transition_controller(state, observation, policy, profile):
```

```
    # Emergency safety has priority over ordinary cooldown.
    next_state = ...
    decision = ...
    return next_state, decision

def replay_controller(initial_state, observations, policy, profile):
    trace = ...
    return trace

def measure_switching_and_response(trace):
    metrics = ...
    return metrics

def E10(workload, training_trace, held_out_trace, policy):
    comparison = ...
    return comparison
```

Part 6 - Measurement, Validation, and Failure Analysis

Reject invalid policies with $l \geq h$, negative patience, or negative cooldown. Verify that no ordinary growth occurs during cooldown while emergency shrink remains possible. Test counter resets and exact threshold equalities.

Demonstrate at least one trace where oscillation decreases and one where discrete shapes or fast budget changes still force legitimate switching. Do not label all switching as failure. Distinguish unnecessary reversals from required safety actions, and report any stability gained by sacrificing throughput or response speed.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Hysteresis makes a controller less sensitive to ambiguous observations. It does not remove the need for a reliable admission model or fresh budget. Explain the feedback mechanism behind the observed oscillation and how the chosen band changes it.

State where your policy balances response and stability well, and where finer batch increments, shape buckets, or stronger isolation would be a better intervention than wider thresholds.

19.24.12 Exercise 11

Exercise Title: Adapt Batch Size to Variable Input Shapes and Demonstrate Why Sample Count Alone Is an Insufficient Memory Predictor

Part 1 - Exercise Contract and Why It Matters

Extend the controller so admission depends on the next input shape as well as sample count. Compare equal-size batches with different sequence lengths or image resolutions. Build

a shape-conditioned predictor, demonstrate a failure of a batch-only model, and document the response to unseen shapes.

Deliver a shape manifest, a memory-versus-shape dataset, prediction errors, and a decision trace that includes the next batch's actual padded shape. Shape metadata must be available before transferring the full batch to CUDA; discovering the shape only after an OOM defeats preflight admission.

Part 2 - System and Memory Mental Model

For padded sequences, one long item increases the padded length of the entire batch. For images, resizing or collating to a larger spatial grid increases every feature map downstream. Hidden workspace changes can compound these payload effects. Sample count is therefore only one feature.

Distinguish maximum padded length from the sum of useful lengths. Masks may exclude padding from the loss while dense kernels still process or allocate for padded positions. A model using packed or variable-length attention follows a different contract and needs a separate profile.

A shape fingerprint should include dimensions, dtype, layout, attention backend, and checkpoint policy. Interpolation inside a measured shape range has stronger evidence than extrapolation to an unseen context length. Set explicit maximum dimensions and a conservative fallback for unprofiled shapes. Keep persistent-state accounting shared where it truly remains fixed.

Part 3 - Quantitative Prediction Before Execution

For padded sequences with lengths L_i, define

$$L_{\max} = \max_i L_i, \quad N_{\text{padded}} = bL_{\max}, \quad N_{\text{useful}} = \sum_i L_i.$$

A simple shape model is

$$\widehat{M}(b, L_{\max}) = M_0 + \alpha bL_{\max} + \beta bL_{\max}^2 + W(b, L_{\max}).$$

If length doubles at fixed b, predict the change in the linear term and the quadratic term separately. Do not assume both terms dominate.

Compare length sets $(128, 128, 128, 128)$ and $(128, 128, 128, 1024)$. Calculate useful and padded token counts, padding fractions, and relative quadratic score-matrix sizes under ordinary dense attention. Repeat the reasoning for square images whose side length doubles. State which parts of each prediction require empirical confirmation.

Part 4 - Experimental Design and Instrumentation

1. Create a CPU dataset with stable sample IDs and several known length or resolution classes, including rare larger inputs.

2. Calibrate at a grid of batch sizes and shapes, holding architecture, optimizer, precision, and backend fixed. Reserve shapes for validation.

3. Compare a batch-only predictor with a shape-conditioned one on the same held-out inputs and budgets.

4. Introduce a single long outlier into an otherwise short batch. Record the collated shape before CUDA transfer and the controller decision.

5. Test a shape outside the calibration domain. Require explicit deferral, conservative probing, or another declared fallback.

For loss comparisons, normalize over the intended samples or valid tokens and keep the mask definition identical across batching arrangements.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region, and measure complete updates. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class ShapeDescriptor:
    sample_ids: tuple
    sample_count: int
    padded_shape: tuple
    valid_elements: int
    backend_name: str

def inspect_cpu_batch_shape(samples, collator_policy):
    descriptor = ...
    return descriptor

def fit_shape_conditioned_profile(calibration_records):
    profile = ...
    return profile

def check_shape_domain(descriptor, profile):
    domain_verdict = ...
    return domain_verdict

def select_batch_for_shape(cpu_queue, budget, profile, policy):
    # Inspect the actual collated shape before GPU allocation.
    proposal = ...
    return proposal

def evaluate_shape_prediction(profile, held_out_records):
    errors = ...
```

```
    return errors

def E11(dataset, shape_grid, budgets, policy):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Check a one-item batch, equal lengths, one extreme outlier, and an input outside the supported domain. Zero-length or empty inputs need an explicit data policy, not an accidental division by zero. Verify masks, target alignment, sample IDs, and loss denominators.

Report unsafe underprediction by shape class, not just an average that hides rare long inputs. Confirm that a smaller sample count can still consume more memory when its padded shape is larger. A controller passes only if its decisions use the actual prospective batch shape. **Evidence format.** Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Admission is about the shape of work, not a single count. Explain the smallest set of metadata that predicts your workload well enough to make safe choices. More features are useful only when they improve held-out decisions.

Identify the remaining uncertainty from operator workspaces, padding, backend selection, and unobserved shapes. This motivates token-aware scheduling and length bucketing in the next exercises.

19.24.13 Exercise 12

Exercise Title: Build a Token-Budget Batch Scheduler for Variable-Length Transformer Sequences Instead of Using a Fixed Number of Samples

Part 1 - Exercise Contract and Why It Matters

Build a scheduler that forms Transformer batches under a declared token budget and a separate memory admission check. Begin with dense padded execution. Compare fixed sample count, useful-token-only budgeting, and padded-token budgeting augmented by a shape-aware memory estimate. Deliver batch manifests, utilization metrics, and a correctness audit.

Keep every sample exactly once in the intended epoch unless an explicit data policy says otherwise. Do not silently truncate long sequences or discard a sample that exceeds the budget. Such a sample must be isolated, deferred with a reason, or rejected through a documented contract.

Part 2 - System and Memory Mental Model

A useful-token budget counts real content. A padded-token budget approximates dense linear work. Neither alone bounds a quadratic attention term when the maximum length changes. Equal token totals can correspond to very different sequence geometries and workspace choices.

The collator's actual execution representation matters. Padded tensors, packed sequences, and variable-length attention kernels need different predictors. This exercise uses padded tensors unless the learner builds and validates an explicitly separate packed backend.

Queue policy also affects learning and fairness. Greedily filling batches with short sequences may starve long ones or reorder the data substantially. Use bounded lookahead and preserve sample IDs. Declare maximum waiting time or a fair fallback for deferred long samples. Loss masking and normalization must remain aligned with the chosen per-token or per-sequence objective.

Part 3 - Quantitative Prediction Before Execution

For a proposed batch $\mathcal{B}$, define

$$U(\mathcal{B}) = \sum_{i \in \mathcal{B}} L_i, \qquad P(\mathcal{B}) = |\mathcal{B}| \max_{i \in \mathcal{B}} L_i.$$

With padded-token budget K, require $P(\mathcal{B}) \leq K$ and also

$$\widehat{M}_{\text{upper}}(\mathcal{B}) + S_J \leq J.$$

For materialized attention, test whether a separate $|\mathcal{B}|L_{\max}^2$ term is necessary.

Two hypothetical batches have eight sequences of length 128 and one sequence of length 1024. Both have 1024 useful tokens and no padding. Calculate their respective bL^2 values and explain why token count alone does not equate attention memory. For lengths $(120, 130, 140, 900)$, compare useful and padded tokens when batched together versus splitting off the long sequence. Compute padding efficiency $\eta = U/P$ for each arrangement.

Part 4 - Experimental Design and Instrumentation

1. Create a reproducible length distribution with short, medium, and rare long sequences. Keep an immutable CPU sample-ID ledger.

2. Define useful-token and padded-token units explicitly. Inspect the collated shape before admitting a proposal.

3. Compare the three schedulers under the same memory budget, precision, backend, and total useful-token workload.

4. Add a single sample exceeding the padded-token budget. Exercise the declared fallback without silently changing its content.

5. Measure useful tokens/s, padded tokens/s, padding efficiency, queue delay, batch-size distribution, peak memory, and refusal reasons.

For the numerical control, aggregate gradients over identical logical token sets with the same valid-token denominator. Disable stochastic and batch-dependent effects for the exact comparison.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region, and measure complete updates. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed

seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class SequenceRecord:
    sample_id: int
    length: int
    payload_reference: object
    arrival_index: int

@dataclass
class TokenBatch:
    sample_ids: tuple
    useful_tokens: int
    padded_tokens: int
    maximum_length: int
    predicted_peak_bytes: int

def summarize_proposed_batch(sequence_records):
    summary = ...
    return summary

def check_token_and_memory_limits(summary, budgets, profile):
    decision = ...
    return decision

def schedule_token_batch(queue, budgets, profile, fairness_policy):
    # Leave packing, lookahead, and fairness decisions incomplete.
    batch = ...
    next_queue_state = ...
    return batch, next_queue_state

def handle_oversized_sample(sample, budgets, policy):
    outcome = ...
    return outcome

def audit_epoch_coverage(input_ids, emitted_batches):
    audit = ...
    return audit
```

```
def E12(dataset, budgets, profile, fairness_policy):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify both admission constraints for every emitted batch, exact ID coverage, valid masks, and target alignment. Test empty input, one sample, equal lengths, budget equality, and an oversized sequence. Check that deferred samples eventually receive an explicit outcome.

Demonstrate the equal-token, unequal-memory counterexample. A scheduler that enforces only useful tokens has not proved a VRAM bound. Compare loss and logical gradients with the fixed reference; different per-sequence weighting must be reported as an objective change, not numerical noise.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Token budgeting is meaningful only when its token unit matches execution. Explain which memory terms your budget constrains and which still require shape-aware admission.

Discuss the tradeoff between packing efficiency, queue latency, and sample order. Recommend a policy whose useful throughput gain survives the cost of collation, lookahead, and fairness handling.

19.24.14 Exercise 13

Exercise Title: Sort and Bucket Variable-Length Sequences and Measure How Reduced Padding Changes Maximum Safe Batch Capacity

Part 1 - Exercise Contract and Why It Matters

Build a length-bucketing policy and compare it with unbucketed random batches and a global length-sort baseline. Measure how padding reduction changes physical batch capacity, useful-token throughput, and ordering. Deliver per-bucket profiles and a coverage audit for the complete dataset.

Global sorting is an efficiency reference, not automatically a training recommendation. It can correlate sample order with difficulty or labels. Use shuffled bucket order and shuffled contents, or bounded-window sorting, and state precisely which order changes the final method permits.

Part 2 - System and Memory Mental Model

Bucketing limits the longest sequence that determines dense batch padding. This can reduce saved activations and attention intermediates without changing model parameters. The gain depends on the length distribution, backend, loss masking, and how tightly each bucket is formed.

A single safe batch size across all buckets may waste capacity on short inputs. Separate per-bucket capacities can exploit that headroom, but switching shapes affects cache history,

kernel selection, and possibly compilation overhead. Keep eager execution initially and treat compiled execution as a separate extension with recompilation costs included.

Reducing padding does not reduce the mathematical number of real tokens. Useful-token throughput is therefore a better comparison than padded-token throughput. A faster run that skips long samples, changes the loss denominator, or over-represents short samples violates the experiment.

Part 3 - Quantitative Prediction Before Execution

For batch j, define padding waste

$$W_j = b_j L_{\max,j} - \sum_{i \in j} L_i, \qquad \eta_{\text{epoch}} = \frac{\sum_j \sum_{i \in j} L_i}{\sum_j b_j L_{\max,j}}.$$

If the variable memory term is proportional to $bL_{\max}$, reducing the bucket's maximum length can increase feasible b approximately inversely with length after subtracting fixed state. A quadratic term produces a different capacity ratio; derive both predictions.

For lengths $(64, 80, 96, 112, 512, 576, 640, 704)$, compare two mixed batches of four with two batches grouped by length. Specify the mixed grouping before calculating; do not select it after seeing the result. Calculate total padded tokens, waste, and efficiency. Predict capacity under a fixed byte cap for short and long buckets using an E11 profile.

Part 4 - Experimental Design and Instrumentation

1. Freeze a dataset with sample IDs, lengths, and labels or targets. Save one random-order baseline permutation.

2. Define bucket boundaries before tuning capacity. Compare fixed-width, quantile, or bounded-window buckets using a held-out length distribution.

3. Measure safe capacity per bucket with the same full-update contract, optimizer state, dtype, and margin.

4. Execute complete epochs with no dropped final partial batches. Shuffle bucket order and internal order under recorded seeds.

5. Report useful tokens/s, padding efficiency, per-bucket batch capacity, peak VRAM, time in collation, and waiting time for rare long samples.

For gradient equivalence, compare a common fixed logical sample set and objective. For longer training, separately evaluate whether reordered batches change learning behavior; memory equivalence alone cannot answer that question.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region, and measure complete updates. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class BucketSpecification:
    lower_length: int
    upper_length: int
    maximum_batch: object
    profile_fingerprint: str

def assign_length_buckets(sequence_records, boundaries):
    buckets = ...
    return buckets

def shuffle_bucket_schedule(buckets, seed, order_policy):
    schedule = ...
    return schedule

def calibrate_bucket_capacity(bucket, workload, budget):
    capacity = ...
    return capacity

def build_bucketed_batches(schedule, capacities, tail_policy):
    batches = ...
    return batches

def compare_padding_and_coverage(reference_ids, batches):
    audit = ...
    return audit

def E13(dataset, boundaries, workload, budget, order_policy):
    comparison = ...
    return comparison
```

Part 6 - Measurement, Validation, and Failure Analysis

Check boundary inclusivity so every length belongs to exactly one bucket. Test empty buckets, one oversized item, equal lengths, and final partial batches. Confirm exact ID coverage and unchanged valid-token count.

Measured capacity must include the optimizer step and switching between buckets, not isolated forward passes only. Report when isolated per-bucket profiles underestimate the runtime peak after shape transitions. Separate padding savings from changes caused by reordered samples or different loss normalization.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Bucketing changes the representation and order of work rather than the underlying model. Explain whether its gain came from lower peak demand, less useless arithmetic, better kernel shapes, or a combination.

Recommend boundaries and a shuffle policy with explicit limits on ordering bias and queue delay. If the length distribution is narrow, a negative result may justify avoiding the added scheduling complexity.

19.24.15 Exercise 14

Exercise Title: Dynamically Trade Micro-Batch Size Against Gradient Accumulation Steps While Preserving a Target Effective Batch Size

Part 1 - Exercise Contract and Why It Matters

Build a planner that changes physical micro-batch size under memory pressure while preserving a fixed logical optimizer batch. This exercise uses one GPU and a target of B samples per accepted update. The planner must handle nondivisible micro-batch sizes, replay after failure, and the final partial dataset window without silently changing weights.

Deliver a partition manifest for every logical update, gradient comparisons against a full-batch reference, sample-ID accounting, and memory/throughput measurements. Keep token-weighted objectives as a separate declared variant. Distributed synchronization is outside the base implementation; do not pretend that multiplying by world size solves unequal-rank weighting.

Part 2 - System and Memory Mental Model

The physical batch controls activation lifetime, while the logical batch controls the samples contributing to one optimizer update. When the micro-batch size changes, a fixed integer accumulation count may no longer produce the desired logical total. The final micro-batch may need fewer samples.

Mean-reduced losses over unequal micro-batches need weighting by sample count. For a token objective, use the number of valid loss-bearing tokens, not padded positions or sequence count. Backpropagating unweighted micro-batch means overweights smaller groups.

Hold parameters fixed until the logical window is complete, accumulate gradients, and update once. Keep data on CPU until each micro-batch is needed. Storing the entire effective batch on GPU can erase the memory benefit. For exact controls, use batch-independent layers and disable dropout; BatchNorm and random masks can break equivalence even with correct loss weights.

Part 3 - Quantitative Prediction Before Execution

For physical sizes $b_1, \ldots, b_K$,

$$\sum_{k=1}^{K} b_k = B, \qquad \mathcal{L} = \sum_{k=1}^{K} \frac{b_k}{B} \overline{\mathcal{L}}_k, \qquad \nabla\mathcal{L} = \sum_{k=1}^{K} \frac{b_k}{B} \nabla\overline{\mathcal{L}}_k.$$

For a constant upper micro-batch size b, $K = \lceil B/b \rceil$ but the last b_k may be smaller. For valid-token counts n_k, replace b_k/B with $n_k / \sum_j n_j$ for the token-average objective.

With $B = 100$ and upper micro-batches 32, 24, and 16, write the exact partitions and weights. Predict the total gradient error caused by dividing every micro-batch mean by K instead. For a tail window containing only 37 samples, state whether to process a smaller logical update, carry it forward, or drop it explicitly; implement the chosen policy without claiming that 37 samples equal the target of 100.

Part 4 - Experimental Design and Instrumentation

1. Create a small deterministic model whose full batch fits for reference. Save model, optimizer, and input state on CPU.

2. Compare several partitions of the same logical sample set, including nondivisible sizes and a mid-plan reduction before the next micro-batch.

3. Accumulate weighted contributions and perform exactly one optimizer update. Record gradient values before the update and parameter changes after.

4. Apply a dynamic budget trace while preserving the logical sample IDs. If an attempt fails, discard its partial window and replay from restored state rather than adding duplicate gradients.

5. Measure physical peak, micro-step count, accepted update frequency, and samples/s over total time. Include planning and retry costs.

Define the complete rollback boundary for parameters, buffers, optimizer, RNG, scaler if present, and the sample cursor. A gradient reset alone is insufficient after partial optimizer mutation.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region, and measure complete updates. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class LogicalWindow:
    sample_ids: tuple
    target_samples: int
```

```
    actual_samples: int
    normalization_count: int

def plan_micro_batches(window, maximum_micro_batch):
    # Preserve the exact total, including the final remainder.
    plan = ...
    return plan

def calculate_loss_weight(micro_batch, logical_window, objective):
    weight = ...
    return weight

def execute_weighted_micro_step(model, micro_batch, weight, policy):
    # Leave forward, loss construction, and backward incomplete.
    observation = ...
    return observation

def commit_logical_update(model, optimizer, window, policy):
    # Validate and update once, after all contributions.
    commit_record = ...
    return commit_record

def restore_failed_window(snapshot, runtime):
    # Restore all state covered by the transaction contract.
    ...

def E14(runtime, cpu_dataset, target_batch, budget_trace):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify $\sum b_k = B$ for full windows and exact coverage for tails. Compare all named gradients and updated parameters, not only loss or gradient norm. Use scale-aware absolute/relative tolerances and report maximum discrepancies.

Inject failure after an early micro-step and verify that replay includes each sample exactly once in the accepted update. Test batch one, $b > B$, nondivisible targets, and token masks with unequal valid counts. Document expected inequivalence from batch-dependent layers instead of disguising it as floating-point error.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Adaptive physical batching can preserve the learning objective only when the runtime explicitly preserves logical sample membership and weights. Explain which state belongs to that

logical transaction.

Discuss the price of smaller micro-batches in launch overhead and update latency, alongside the memory saved. The target effective batch is a training choice, not a quantity that should drift whenever VRAM changes.

19.24.16 Exercise 15

Exercise Title: Combine Adaptive Batch Sizing with Mixed Precision and Quantify How Precision Changes the Controller's Feasible Search Space

Part 1 - Exercise Contract and Why It Matters

Compare adaptive batching under FP32, FP16 autocast with gradient scaling, and BF16 autocast where supported. Build separate capacity profiles and measure how precision changes feasible physical batch size, numerical error, and accepted useful throughput.

Keep model architecture, optimizer, logical batch, data order, and safety policy fixed. Precision is selected at a declared logical update boundary. Do not reuse an FP32 profile unchanged for autocast or infer that reduced element size necessarily doubles batch capacity.

Part 2 - System and Memory Mental Model

Autocast selects operation dtypes; it is not equivalent to converting all parameters and optimizer state to half precision. A common autocast setup keeps parameters and optimizer state in FP32 while reducing selected activation and intermediate storage. Sensitive operations can remain FP32. The actual memory change depends on which terms dominate.

FP16 needs a numerical stability policy, commonly gradient scaling. Keep the scale fixed while accumulating one logical update; unscale once after all contributions, then validate or clip as required, and perform the scaler-managed step/update at the effective-update boundary. An overflow skip is not accepted training progress.

BF16 availability and performance must be checked on the actual device. Unsupported configurations receive an explicit status. Changing precision can also change kernel and workspace selection, so every precision profile needs measured shape coverage and a separate uncertainty estimate.

Part 3 - Quantitative Prediction Before Execution

For precision policy p,

$$\widehat{M}_p(b, L) = M_{\text{fixed},p} + M_{\text{variable},p}(b, L) + W_p(b, L).$$

If $M_{\text{fixed}} = 3\,\text{GiB}$ and FP32 variable cost is $100\,\text{MiB/sample}$, predict capacity under a usable $6\,\text{GiB}$ cap. Repeat assuming only the variable term halves. Compare the resulting capacity ratio with the simplistic factor of two.

Define numerical error against an FP32 reference, for example

$$E_g = \frac{\|g_p - g_{\text{ref}}\|_2}{\max(\|g_{\text{ref}}\|_2, \epsilon)}.$$

The selected configuration must satisfy memory and numerical constraints, not memory alone. Before execution, predict which policy improves accepted tokens/s and which has the highest overflow risk; state the assumptions behind both predictions.

Part 4 - Experimental Design and Instrumentation

1. Check CUDA and dtype support, then create identical FP32 master parameters and optimizer settings for each policy.

2. Calibrate each policy independently using full updates, including cold state creation, autocast behavior, and gradient-scaler overhead.

3. Use E14 to preserve the same logical batch while adapting physical sizes. Keep correct sample or valid-token weighting.

4. Compare gradient and parameter errors on a small deterministic case, then benchmark repeated realistic updates with declared tolerances.

5. Log dtype policy, scale, overflow skips, physical batch, accumulation count, memory, wall time, and accepted useful throughput.

Use `torch.autocast` and `torch.amp.GradScaler` according to the installed PyTorch API. Record precision-policy changes and invalidate incompatible calibration before the next admission decision.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region, and measure complete updates. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every `...` marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class PrecisionPolicy:
    name: str
    autocast_dtype: object
    scaling_enabled: bool
    absolute_tolerance: float
    relative_tolerance: float

def check_precision_support(device, policy):
    support = ...
    return support

def create_policy_scaler(policy):
    scaler = ...
    return scaler
```

```
def run_precision_window(runtime, window, micro_plan, policy):
    # Keep scale constant across the complete accumulation window.
    # Unscale, validate, and commit only at its boundary.
    result = ...
    return result

def calibrate_precision_profile(workload, policy, shapes):
    profile = ...
    return profile

def compare_numerical_and_memory_results(reference, candidates):
    comparison = ...
    return comparison

def E15(workload, policies, logical_batch, budget_trace):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Reject unsupported BF16 as unsupported, not OOM. Verify no parameter casting occurred accidentally when the intended policy was autocast. Check finite losses, gradient error, parameter changes, and optimizer-step acceptance. Do not infer skipped updates solely from an optimizer return value.

Test one injected overflow and verify it does not inflate accepted throughput or advance a scheduler that is defined per accepted update. Compare capacity using identical logical work and report memory, speed, and accuracy tradeoffs separately. A lower-memory unstable policy is not a winning configuration.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Precision changes the feasible set, the numerical behavior, and sometimes the kernel implementation. Explain which memory component actually shrank and whether additional capacity translated to useful progress.

Recommend a precision-aware controller policy with separate calibration, numerical acceptance thresholds, and profile invalidation rules. Avoid switching precision opportunistically without recording its effect on training semantics and error.

19.24.17 Exercise 16

Exercise Title: Combine Adaptive Batch Sizing with Activation Checkpointing and Decide When Increasing Recomputation Is Better Than Reducing Batch Size

Part 1 - Exercise Contract and Why It Matters

Compare two responses to a tighter budget: reduce physical batch size or checkpoint more activation regions. Search a small declared set of micro-batch/checkpoint configurations while preserving the logical batch. Select by accepted useful throughput subject to memory and numerical constraints, and explain when recomputation pays for itself.

Deliver a configuration table, phase timings, memory curves, gradient comparisons, and a decision rule. Include the possibility that neither intervention can help because persistent parameters and optimizer state already exceed the budget.

Part 2 - System and Memory Mental Model

Activation checkpointing retains fewer forward intermediates and reconstructs them during backward. It can lower activation storage while adding compute, but recomputation itself has a peak and temporary workspaces. Input tensors required to rerun a region and outputs needed elsewhere still have lifetimes. It does not generally remove optimizer state or parameter storage.

Use an explicit checkpoint API policy, such as `use_reentrant=False`, and record it. Stochastic operations and mutable module buffers need careful equivalence checks. RNG preservation helps reproduce random operations, but arbitrary stateful side effects are not made safe automatically. Start with stateless blocks and dropout disabled, then investigate stochastic cases separately.

Reducing batch can increase micro-step count and launch overhead. Checkpointing at a larger physical batch might maintain more efficient kernels but pay additional backward recomputation. Which is faster depends on the actual workload and hardware; memory savings alone cannot decide. Change checkpoint policy only at a logical-window boundary.

Part 3 - Quantitative Prediction Before Execution

For logical batch B, candidate physical size b, and checkpoint plan c, use

$$K = \lceil B/b \rceil , \qquad t_{\text{update}}(b,c) = \sum_{k=1}^{K} \Big(t_{\text{fwd},k} + t_{\text{bwd},k} + t_{\text{recompute},k} \Big) + t_{\text{optimizer}} + t_{\text{overhead}}.$$

The exact final micro-batch size still follows E14. Choose among candidates with predicted upper peak plus margin within the cap and acceptable gradient error.

For an illustrative $B = 64$, suppose $b = 8$ without checkpointing needs 10 ms per micro-step, while $b = 16$ with checkpointing needs 17 ms. Ignoring common optimizer cost initially, predict update time and throughput. Then add a 20 ms policy-switch/calibration cost amortized over one versus 100 updates. Explain why the preferred action can depend on how long the budget regime lasts.

Part 4 - Experimental Design and Instrumentation

1. Partition a deterministic model into checkpointable blocks. Document which tensors cross each boundary and which buffers can change.

2. Compare no checkpointing, selected blocks, and a more extensive plan. Calibrate each plan separately at several physical batch sizes.

3. Preserve identical logical sample sets, loss weights, optimizer, and precision. Compare gradients and parameters with an uncheckpointed reference.

4. Benchmark full updates under two budgets: one that permits all plans and one that forces a tradeoff. Include cold switching and warm steady cost.

5. Add a budget trace whose tight period is short, then long. Measure whether switching cost is repaid in accepted useful work.

Use separate diagnostic timing to attribute recomputation. A total backward time increase alone is not an exact measurement of the recomputed region.
Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region, and measure complete updates. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch
from torch.utils.checkpoint import checkpoint

@dataclass
class CheckpointPlan:
    plan_name: str
    selected_blocks: tuple
    preserve_rng_state: bool
    use_reentrant: bool

def execute_block_with_policy(block, inputs, checkpoint_plan):
    # Implement the explicit checkpoint API policy.
    outputs = ...
    return outputs

def profile_batch_checkpoint_pair(workload, batch_size, plan):
    result = ...
    return result

def estimate_logical_update_cost(profile, logical_batch):
    cost = ...
    return cost

def select_recompute_or_shrink(candidates, budget, quality_limits):
    decision = ...
    return decision
```

```
def validate_checkpoint_equivalence(reference, candidate):
    errors = ...
    return errors

def E16(workload, plans, logical_batch, budget_trace):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Check finite loss, all parameter gradients, parameter updates, and mutable buffers. A matching loss with mismatched buffers is insufficient. Test checkpointed blocks with and without random operations and report the assumptions required for the comparison.

Confirm full-update peaks include recomputation and optimizer state. Report candidates that use more memory or run slower than expected. Reject a recommendation based only on larger feasible batch if accepted throughput falls after recomputation and switching costs.
Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

The correct intervention depends on the bottleneck. Recomputation can be valuable when memory-bound activations prevent efficient batch sizes; it can be wasteful when compute is already limiting.

Explain the measured break-even regime and what invalidates it after a shape, precision, or hardware change. Recommend a bounded candidate search with profile reuse, rather than assuming maximal checkpointing is always the most memory-efficient end-to-end plan.

19.24.18 Exercise 17

Exercise Title: React to External VRAM Pressure from Another CUDA Process and Reduce Batch Size Without Restarting the Training Job

Part 1 - Exercise Contract and Why It Matters

Build a controlled two-process experiment: a training runtime and a separate pressure worker that allocates a bounded amount of VRAM on the same physical GPU. The trainer must observe reduced availability, shrink future physical batches, preserve logical updates, and remain alive through recoverable allocation OOM.

Deliver coordinated and uncoordinated pressure traces, accepted-update accounting, rollback evidence, and a no-fit/defer outcome. Do not terminate unrelated processes. The worker is created and controlled by this experiment, has a declared allocation ceiling, and releases its memory through an explicit command or cleanup path.

Part 2 - System and Memory Mental Model

PyTorch process counters do not include the worker's live tensors. A device-global free-memory snapshot can reveal pressure but cannot prevent new allocations immediately after the check. Use a coordinated mode first, where the pressure worker changes demand at known logical boundaries; then test an asynchronous mode to expose the race.

OOM recovery is a transaction problem as well as a memory problem. Backward may leave partial gradients, and optimizer operations may mutate some state before failure. A retry must restore model parameters, buffers, optimizer state, CPU/CUDA RNG, scaler state if used, and sample cursor according to the declared window boundary. Clear failed graph references before retry. Cache release is only optional cleanup of unused blocks.

Store independent CPU snapshots, not aliased views of live state. Snapshot creation, restoration, and host-memory use have costs and must be measured. Keep the checkpoint alive until the update commit is confirmed. A fatal CUDA error is outside recoverable allocation OOM; exit with a recoverable checkpoint rather than claiming all device errors can be repaired in place.

Part 3 - Quantitative Prediction Before Execution

Let worker demand change from E_t to E_{t+1}. A measured free-memory change is only an observation, since other allocations and reservations can change simultaneously. With trainer allocated baseline A_t and free bytes F_t, conservatively require

$$\widehat{\Delta M}_{\text{upper}}(b) \le \min(J - S_J - A_t,\ F_t - S_D)$$

when giving no credit for reusable cache.

Assume the trainer needs a 1.4 GiB incremental peak and requires 0.4 GiB device headroom. Predict admission with $F_t = 2.5\,\text{GiB}$, then after a worker adds 1 GiB. Predict the outcome if that addition occurs after admission instead. For recovery, report

$$Q_{\text{accepted}} = \frac{N_{\text{committed}}}{t_{\text{normal}} + t_{\text{failed}} + t_{\text{restore}} + t_{\text{deferred}}}.$$

Define maximum retries and a no-fit waiting policy before running.

Part 4 - Experimental Design and Instrumentation

1. Use spawn to create a dedicated pressure worker, initialize CUDA inside it, and verify both processes refer to the same physical GPU.

2. Exchange ready/allocated/released acknowledgements. Bound requested bytes relative to a measured safe envelope and handle worker OOM explicitly.

3. Run a coordinated pressure step between logical updates. Compare a static trainer with the adaptive trainer under identical work.

4. Run a bounded asynchronous pressure change during forward or backward. Catch allocation OOM, end the failed attempt, release references, restore the entire transaction, and replay with a smaller plan.

5. Test failure after a deliberately partial optimizer mutation using a controlled fault hook. Require full restoration before accepting another update; resetting gradients alone must fail this test.

6. Release worker allocations in cleanup even if training fails. Verify the trainer process remained alive for recoverable cases.

Track application-cap violations separately from OOM. Do not claim a strict global reserve between samples in the uncoordinated mode.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region, and measure complete updates. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import multiprocessing as mp
import torch

@dataclass
class PressureCommand:
    command_id: int
    action: str
    requested_bytes: int

@dataclass
class WindowSnapshot:
    model_state_cpu: object
    optimizer_state_cpu: object
    scaler_state: object
    rng_state: object
    data_cursor: object

def pressure_worker(command_channel, event_channel, device_spec):
    # Allocate only this worker's bounded test buffers.
    # Acknowledge completion and release them in cleanup.
    ...

def capture_window_snapshot(runtime):
    # Create independent CPU copies, including mutable buffers.
    snapshot = ...
    return snapshot

def execute_recoverable_attempt(runtime, window, micro_plan):
    # Keep failure scope separate so graph references can die.
```

```
    outcome = ...
    return outcome

def restore_transaction(runtime, snapshot):
    # Restore all state, even after partial optimizer mutation.
    ...

def react_to_pressure(runtime, window, snapshot, limits):
    # Bound retries; defer explicitly if one sample cannot fit.
    result = ...
    return result

def E17(runtime, pressure_schedule, limits):
    report = ...
    return report

def main():
    # Create CPU-only configuration and spawn the test worker.
    ...

if __name__ == "__main__":
    main()
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify accepted sample IDs, optimizer counters, scaler state, mutable buffers, and parameters against a clean replay from the same transaction snapshot. Test OOM before backward, after partial accumulation, and after partial optimizer mutation. The last case specifically invalidates gradient-reset-only recovery.

Bound attempts, snapshot age, and deferred time. Distinguish no-fit, worker failure, allocation OOM, and fatal device error. Record trainer PID continuity without claiming that context-fatal errors were survived. Snapshots and pressure buffers must not leak across windows. Report the minimum observed headroom with sampling limitations.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

A live controller can adapt to measured external demand but cannot reserve future VRAM merely by observing it. Explain the stronger guarantee in coordinated mode and the best-effort behavior in asynchronous mode.

Discuss the price of transactional recovery: CPU snapshot memory, copying, lost work, and latency. Recommend how the runtime should defer or stop when no plan fits, while preserving a usable checkpoint and honest progress log.

19.24.19 Exercise 18

Exercise Title: Stress-Test the Adaptive Batch Controller with Sudden Sequence-Length Spikes, Fragmentation, Temporary Memory Peaks, and Changing VRAM Availability

Part 1 - Exercise Contract and Why It Matters

Build a reproducible adversarial test suite for the controller developed so far. Cover shape spikes, changing device availability, temporary workspaces, allocator-history pressure, numerical skips, and failures at different transaction phases. Deliver a scenario manifest, expected invariants, event traces, and a failure report for every unsuccessful case.

Stress testing must separate two goals: deterministic controller logic testing with synthetic observations, and real CUDA integration testing. A simulated OOM validates state handling but does not prove allocator recovery. A GPU trace validates one runtime environment but does not exhaust all controller transitions.

Part 2 - System and Memory Mental Model

Sequence spikes change the demanded shape. External workers change availability. Temporary buffers change within-step peak. Retained live tensors and allocator fragmentation can reduce the ability to satisfy a large request despite seemingly sufficient totals. These are different mechanisms and need separate labels.

Allocator fragmentation is backend- and history-dependent. Create varied allocation sizes, retain selected live blocks, release others, and replay a large request; then compare with a fresh-worker baseline. If evidence supports only live-memory pressure, report it that way. Do not call every OOM with reserved memory present fragmentation.

The controller should preserve sample identity, logical weights, bounded retries, and complete rollback regardless of disturbance. Safety, progress, and observability are distinct invariants. A controller that never starts work can avoid OOM but fails useful progress; a fast controller that loses samples fails correctness.

Part 3 - Quantitative Prediction Before Execution

Define a scenario with phase-indexed events $\mathcal{S} = \{(t_i, \text{phase}_i, \text{event}_i, \text{size}_i)\}$. Use logical update or phase indices for reproducible replay rather than relying only on wall-clock sleep timing.

$$R_{\text{violation}} = \frac{N_{\text{margin violations}}}{N_{\text{attempts}}}, \qquad A_{\text{retry}} = \frac{N_{\text{attempts}}}{N_{\text{commits}}}.$$

When the denominator is zero, report undefined and the no-progress reason. Also report recovery latency, accepted useful throughput, and minimum observed headroom.

Before running, predict outcomes for: a length doubling, a temporary 256 MiB buffer at backward, a budget below persistent state, and an OOM after partial optimizer mutation. Declare which should shrink, retry, defer, or stop. For repeated tests with zero observed failures, avoid claiming zero true risk; under independent identical trials, the approximate rule-of-three bound is $3/n$, and correlated stress trials may not satisfy that assumption.

Part 4 - Experimental Design and Instrumentation

1. Write at least eight scenarios, each with event schedule, seed, maximum demand, timeout, expected invariant, and cleanup behavior.

2. Test pure logic with budget equality, stale snapshots, contradictory profiles, repeated failures at batch one, and growth during cooldown.

3. Run CUDA cases for a rare long input, temporary forward/backward workspace, external pressure, and changing allocation history.

4. Separate fault injection from real allocation failures in the log. Use fresh-worker controls to investigate suspected fragmentation.

5. Audit transaction state after each injected failure and compare a successful replay to a clean deterministic reference.

6. Repeat representative scenarios with different seeds and an unseen disturbance schedule. Include test harness overhead in reports where it affects measured response time.

Terminate only workers created by this experiment. Cleanup must execute after expected and unexpected errors, and original sample manifests remain immutable evidence.
Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region, and measure complete updates. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class StressEvent:
    update_index: int
    phase: str
    event_type: str
    magnitude: object

@dataclass
class StressScenario:
    name: str
    seed: int
    events: tuple
    maximum_attempts: int
    expected_outcome: str

def replay_synthetic_observations(controller, scenario):
    trace = ...
    return trace
```

```
def create_allocator_history(device, allocation_plan):
    # Keep retained live blocks distinct from released cache.
    history_handle = ...
    return history_handle

def execute_cuda_scenario(runtime, scenario, fault_hooks):
    result = ...
    return result

def audit_transaction_invariants(before, after, sample_ledger):
    audit = ...
    return audit

def classify_failure_evidence(result, fresh_worker_control):
    diagnosis = ...
    return diagnosis

def E18(runtime_factory, scenarios):
    # Export every outcome, including no-progress scenarios.
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Require explicit evidence for every invariant: no duplicated accepted sample, no missing sample, correct loss denominator, no update before a complete window, no partial state surviving rollback, and bounded retries. Check that cleanup returns the live-memory baseline within a declared tolerance, while allowing documented cache reservations.

Report all failures, their seeds, phase, classification, and minimal reproducer. Distinguish a valid no-fit refusal from a controller deadlock. Measure recovery time only for recoverable scenarios; do not hide permanent failures by omitting them from the denominator.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Stress tests describe an operating envelope, not universal immunity. Explain which failure classes the suite covers and which remain outside the runtime's assumptions, including fatal device errors and unbounded external allocation.

Prioritize improvements by lost accepted work, correctness impact, and frequency in the declared workload. Preserve negative results as design evidence instead of tuning the stress suite until every chart looks stable.

19.24.20 Exercise 19

Exercise Title: Build and Benchmark a PyTorch Adaptive Batch Runtime That Maximizes Throughput While Maintaining a Strict Dynamic VRAM Safety Margin

Part 1 - Exercise Contract and Why It Matters

Integrate profiling, shape-aware admission, bounded capacity search, hysteresis, token scheduling, exact logical accumulation, precision, checkpointing, telemetry, and transactional recovery into one runtime. Select the highest-throughput validated configuration from a finite declared candidate set while enforcing explicit job-budget admission constraints. Deliver the runtime skeleton completed by the learner, benchmark manifests, raw traces, and an engineering decision report.

Define the meaning of strict before claiming it. In the base experiment, budget changes occur at coordinated logical-update boundaries and workloads remain inside a declared shape/disturbance envelope. Require no observed margin violation there. Under arbitrary asynchronous external allocation, the runtime can provide conservative admission and recovery, not a hard physical guarantee. Report that mode separately.

Part 2 - System and Memory Mental Model

Separate four responsibilities: telemetry observes state; the planner proposes a configuration; admission accepts or refuses it; the executor commits a complete logical update or restores its snapshot. Logging must record the inputs and reason behind each decision. A profile learned from one shape, dtype, backend, or checkpoint plan is not automatically valid for another.

The configuration contains physical batch size, shape/token limits, precision, checkpoint plan, and an exact logical-window partition. The executor owns model and optimizer state, RNG, scaler, sample cursor, and any learning-rate schedule whose progress follows accepted updates. Failed attempts cannot advance accepted-work counters.

Memory fit is a constraint, not the optimization objective. The largest batch may be slower than a smaller one. Include calibration, shape collation, snapshot copying, retries, deferrals, and policy switching in net throughput. Reserve a held-out workload trace for evaluation; the controller must not inspect future budgets or lengths beyond its declared lookahead.

Strict enforcement has layers. A predicted upper envelope supports admission, post-step observations audit realized peaks, and resource isolation supports stronger device-level guarantees. A percentile model plus a reserve is an empirical policy, not proof against all possible workspaces and external processes.

Part 3 - Quantitative Prediction Before Execution

Let $\mathcal{C}_t$ be the finite validated candidate set and let c specify batch, precision, checkpoint plan, and shape policy. Optimize

$$c_t^* = \arg\max_{c \in \mathcal{C}_t} \widehat{Q}_{\text{accepted}}(c)$$

subject to

$$\begin{aligned} \widehat{M}_{\text{upper}}(c, x_t) + S_{J,t} &\le J_t, \\ \widehat{\Delta M}_{\text{upper}}(c, x_t) + S_{D,t} &\le F_t, \\ E_{\text{numerical}}(c) &\le \varepsilon_{\max}, \\ \sum_k b_{t,k} &= B_t. \end{aligned}$$

The device inequality gives no cache credit in the conservative base policy. Document any refinement and avoid double-counting resident state.

Compute the realized objective as accepted useful samples or tokens divided by total benchmark wall time. Report the unit and objective normalization. Build a pre-run table of at least

six candidate configurations with predicted peak, uncertainty, physical batch, accumulation plan, throughput, and error. Identify dominated candidates and explain any exclusion before seeing the held-out results.

Distinguish an empirical best candidate from a global optimum. An offline oracle with future trace knowledge may be an upper comparison bound but is not a deployable causal controller.

Part 4 - Experimental Design and Instrumentation

1. Implement independent telemetry, profile registry, planner, admission, executor, and event-log interfaces. Validate pure CPU decision logic first.

2. Create a deterministic numerical reference and verify exact sample membership, weights, gradients, optimizer updates, and rollback behavior.

3. Compare four baselines: conservative static batch, largest repeatedly safe static batch, simple reactive shrink/grow, and the integrated runtime. Keep logical work, precision eligibility, and candidate opportunities fair.

4. Run a steady trace, a coordinated dynamic-budget trace, a variable length trace, and the E18 stress suite. Evaluate asynchronous pressure separately from the strict coordinated contract.

5. Tune only on a training trace, freeze policy, then evaluate held-out traces with repeated seeds. Measure warm-up, calibration, cold first update, and steady execution as separate categories.

6. Export raw scalar event logs and aggregate results. Include accepted useful throughput, median/p95 update latency, margin violations, OOMs, retries, no-fit duration, prediction error, switches, and numerical error.

Define acceptance thresholds before benchmarking. At minimum require exact sample accounting, complete rollback, bounded recovery, finite accepted updates, and no observed margin violations in the coordinated envelope. Failure to beat a strong static baseline is a valid measured conclusion.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region, and measure complete updates. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class RuntimeConfiguration:
    micro_batch_size: int
    precision_name: str
    checkpoint_plan_name: str
    shape_profile_key: str
    logical_batch_size: int

@dataclass
class RuntimeLimits:
    job_budget_bytes: int
    job_margin_bytes: int
    device_margin_bytes: int
    maximum_retries: int
    maximum_deferred_seconds: float

@dataclass
class CommitRecord:
    update_id: int
    accepted_sample_ids: tuple
    accepted_useful_tokens: int
    attempt_count: int
    peak_allocated_bytes: int
    wall_seconds: float

class AdaptiveBatchRuntime:
    def __init__(self, workload, profiles, policy, limits):
        self.workload = workload
        self.profiles = profiles
        self.policy = policy
        self.limits = limits
        self.controller_state = ...
        self.sample_ledger = ...
        self.event_log = ...

    def observe_boundary(self):
        observation = ...
        return observation

    def build_candidates(self, window, observation):
        # Match shape, precision, backend, and checkpoint profiles.
        candidates = ...
        return candidates

    def choose_and_admit(self, candidates, observation):
```

```
        # Optimize accepted throughput among admissible candidates.
        decision = ...
        return decision

    def snapshot_transaction(self, window):
        snapshot = ...
        return snapshot

    def execute_attempt(self, window, configuration):
        # Implement weighted accumulation and a single commit.
        outcome = ...
        return outcome

    def rollback_attempt(self, snapshot, outcome):
        # Restore every mutable state covered by the contract.
        ...

    def run_logical_window(self, window):
        # Bound retries, support no-fit, and preserve sample IDs.
        commit_record = ...
        return commit_record

    def export_report(self, output_path):
        # Export observations, decisions, failures, and commits.
        ...

def benchmark_runtime(runtime_factory, baselines, held_out_traces):
    comparison = ...
    return comparison

def E19(workload, profiles, policy, limits, held_out_traces):
    runtime = ...
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Audit every commit against the sample manifest, normalization denominator, optimizer-step count, and numerical reference. Inject failure at transfer, backward, and partial optimizer update; compare the accepted replay against a clean transaction. Validate scheduler and scaler progression if present. Check that state restoration includes buffers and that snapshots are independent copies.

Check each realized peak against the cap and margin active during that attempt. A completed update with insufficient headroom is still a policy violation. Do not replace peak allocated bytes with a lower end-of-step counter. Report reserved memory and global observations separately and state what cannot be observed between samples.

Use common evaluation work or a declared common time horizon across baselines. Do not count failed attempts, duplicate replayed tokens, or overflow-skipped updates as accepted work. Report uncertainty across repetitions and preserve all no-fit cases. The runtime passes

the correctness contract even if its performance result is negative; a throughput claim requires evidence beyond merely fitting a larger batch.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

The product is a trustworthy scheduler for constrained training work. Its evidence should explain why each configuration was admitted, what work was actually committed, and how failures affected progress.

Conclude with the strongest claim your experiments support: the best validated configuration or controller under specified budgets, shapes, hardware, and numerical constraints. State the limits of the strict coordinated guarantee and the best-effort asynchronous mode. Recommend deployment only within that measured envelope, with profile invalidation, bounded fallback, and reproducible diagnostics.

19.25 DMM23

Chapter Title: CUDA Out-of-Memory Detection and Recovery
Specialization: Dynamic Memory Management and CUDA Systems

19.25.1 Exercise 00

Exercise Title: Build a Precise Mental Model of CUDA OOM Causes, Failure Boundaries, Allocator State, and Recoverable Versus Non-Recoverable Memory Pressure

Part 1 - Exercise Contract and Why It Matters

Build a lifecycle model for an allocation failure during CUDA training. Distinguish failure to obtain storage from a corrupted execution context, and distinguish a recoverable CUDA context from a recoverable training transaction. Deliver a memory ledger, a failure-boundary table, and a decision diagram connecting detection, diagnosis, cleanup, rollback, reconfiguration, and either retry or explicit termination.

Use one GPU, eager PyTorch, and a small model. DMM19–DMM22 provide the precision, accumulation, and admission prerequisites. Distributed collectives, CUDA graph capture, custom kernels, and external side effects require additional contracts and are outside the base experiment. A successful retry must preserve accepted training work, not merely avoid another exception.

Part 2 - System and Memory Mental Model

OOM means a requested allocation could not be satisfied under the current conditions. Demand includes resident parameters, gradients, optimizer state, inputs, saved activations, temporary workspaces, and library allocations. Pressure may come from other processes, retained Python references, non-reusable allocator fragments, or a brief overlap of otherwise manageable buffers. Several causes can coexist.

Allocated bytes describe live allocator-managed tensor storage; reserved bytes also include allocator-held blocks. Global free bytes describe the device at one instant. Their difference is not a complete map of reusable blocks. A failed request does not become a successful live allocation, so the recorded peak can be lower than the memory the operation attempted to require. Driver and library allocations may be outside PyTorch snapshots.

An allocation OOM often leaves the CUDA context usable after failed work is unwound. It does not undo mutations already performed. Forward can change buffers and RNG; backward can leave partial gradients; an optimizer can modify some parameters before a later allocation fails. Illegal-address and device-assert failures are different error classes and can make further CUDA execution unusable. Repeated cache clearing is not a remedy for them. For device-fatal conditions, preserve an existing CPU checkpoint and exit.

Part 3 - Quantitative Prediction Before Execution

Let A_t be current allocated bytes, R_t reserved bytes, F_t global free bytes, and q the requested allocation. In a simplified allocator, failure depends on suitable reusable blocks and possible segment acquisition, not merely on whether $q > F_t$. Avoid treating $R_t - A_t$ as one freely coalescible block.

$$M_{\text{live}}(\tau) = M_{\text{persistent}}(\tau) + M_{\text{transient}}(\tau), \qquad M_{\text{peak}} = \max_{\tau} M_{\text{live}}(\tau).$$

Construct a hypothetical trace with $A = 5\,\text{GiB}$, $R = 6\,\text{GiB}$, $F = 1\,\text{GiB}$, and a request for $1.5\,\text{GiB}$. Explain why the three scalar counters cannot alone prove whether reuse and retry can satisfy that request.

Represent logical training state as

$$S = (\theta, \mathcal{B}, \mathcal{G}, \mathcal{O}, \mathcal{R}, \mathcal{A}, \mathcal{D}, \mathcal{H}),$$

where the components are parameters, mutable buffers, gradients, optimizer, RNG, AMP scaler, data cursor, and scheduler state. For each failure phase, predict which components may have changed. Separate exact rollback from equivalence after changing micro-batch partition: dropout and BatchNorm can make the latter a different computation.

Part 4 - Experimental Design and Instrumentation

1. Describe allocation, forward, backward, optimizer, and commit boundaries. Specify the last known-good state before each attempt.

2. Use a bounded disposable allocation experiment to observe one genuine OOM. Do not intentionally corrupt the CUDA context.

3. Use labelled fault injection for forward, backward, and partial optimizer mutation. Injection tests state handling; it is not evidence of real allocator behavior.

4. Record the phase where the exception surfaces and the last completed phase. CUDA work is asynchronous, so these may not establish the exact origin without diagnostic synchronization.

5. Classify outcomes as success, retryable allocation OOM, no feasible configuration, unrelated error, or context-fatal error. State which require no further CUDA calls.

Produce a small state audit before and after each attempt. Keep CPU-only evidence and avoid retaining the exception object or its tensor-owning frames.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class FailureBoundary:
    phase: str
    last_completed_phase: str
    exception_type: str
    context_status: str
    state_may_be_mutated: bool

def describe_training_state(runtime):
    state_ledger = ...
    return state_ledger

def capture_memory_ledger(device, phase):
    memory_ledger = ...
    return memory_ledger

def classify_failure(exception_record, phase_evidence):
    # Do not infer recoverability from the message alone.
    classification = ...
    return classification

def decide_recovery_boundary(classification, state_ledger):
    decision = ...
    return decision

def audit_post_failure_state(before, after):
    state_difference = ...
    return state_difference
```

```
def E00(runtime, controlled_scenarios):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Require separate evidence for context usability, restored training state, and successful admitted work. A small allocation after OOM checks only a limited aspect of context usability. It does not prove gradients or optimizer state are correct.

Reject classifications that equate every RuntimeError with OOM, assume forward is side-effect free, or count a failed attempt as an accepted update. Check that all requested quantities include units and measurement time. If origin or allocator details are unobserved, retain an unknown classification rather than inventing a diagnosis.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Recovery has two obligations: regain a viable memory configuration and preserve the training transaction. Explain why satisfying only one is insufficient.

Conclude with a precise recoverability contract for this chapter. It should allow bounded in-process recovery from allocation pressure while refusing unsupported guarantees about context-fatal errors, arbitrary external allocations, and untracked side effects.

19.25.2 Exercise 01

Exercise Title: Trigger a Controlled CUDA OOM with a Single Oversized Tensor Allocation and Capture the Full PyTorch Exception Context

Part 1 - Exercise Contract and Why It Matters

Create a bounded isolated experiment that issues one oversized CUDA tensor allocation and records the resulting exception. Capture its complete message, formatted traceback, type, requested shape and dtype, device identity, timestamp, phase, and surrounding memory observations. Deliver a structured incident record and a successful small-allocation control.

Use a dedicated spawned worker, with CUDA initialized inside it. Choose a request just above physical device capacity under ordinary PyTorch device allocation, not an absurd dimension that first triggers integer overflow or a shape-validation error. The parent receives only CPU strings and scalar metadata. The experiment must not allocate a huge CPU source tensor.

Part 2 - System and Memory Mental Model

A request made through a CUDA allocation API can fail before any tensor storage is successfully created. There may therefore be no tensor object to delete. The allocator may already have attempted internal cache release or retries before presenting the error; the exception-time snapshot need not match the immediately preceding snapshot.

The exception message is evidence, not a stable structured schema. Some versions include requested allocation size and process memory detail; do not assume every message contains identical fields or units. The requested shape/dtype supplied by this experiment gives an independent payload estimate. Allocator rounding can make the actual request differ.

A traceback object holds stack frames and their local references. Serialize the traceback to text without capturing locals, then let the exception scope end. Storing the exception itself in a report can retain GPU objects from more complex attempts. Full context means complete relevant execution metadata, not arbitrary tensor contents or every local variable.

Part 3 - Quantitative Prediction Before Execution

For a tensor with dimensions $n_1, \ldots, n_k$ and element size s bytes,

$$q_{\text{payload}} = s \prod_{i=1}^{k} n_i.$$

Using a byte tensor makes the intended payload directly auditable. Let T be total physical bytes and choose a small declared increment δ, giving $q_{\text{payload}} = T + \delta$ in a fresh ordinary device-allocation worker. Bound δ and verify the dimension is representable before requesting storage.

For a hypothetical $T = 8\,\text{GiB}$ and $\delta = 64\,\text{MiB}$, calculate the element count for `uint8` and the corresponding count for `float32`. Predict current and peak allocated counters after the failed request: they should not be interpreted as if the oversized storage were created. Also predict the outcome of a separate 1 MiB control request, subject to the actual available environment.

Part 4 - Experimental Design and Instrumentation

1. Validate the request on CPU, establish upper bounds, and record device total capacity. Fail explicitly if no CUDA device is available.

2. In a fresh worker, initialize a small control allocation and release it. Synchronize at the declared boundary and capture the pre-request record.

3. Issue the single oversized request. Capture the specific CUDA OOM type when raised; other exceptions must remain separately classified.

4. Format the full traceback as text without local-variable capture. Attempt best-effort scalar telemetry only for a usable allocation-OOM context, preserving any secondary diagnostic failure separately.

5. Leave the exception scope. Perform a small recovery control in the same worker, report its result, then exit and join cleanly.

Record whether the environment imposed an allocator limit or another constraint. A limited-process OOM and a physical-capacity OOM need distinct labels even when they raise the same exception class.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer

update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import multiprocessing as mp
import traceback
import torch

@dataclass
class AllocationSpecification:
    shape: tuple
    dtype_name: str
    requested_payload_bytes: int
    device_identity: str

def choose_bounded_oversized_request(device_total, increment):
    specification = ...
    return specification

def serialize_exception_context(error, metadata):
    # Return text and scalars, never exception or frame objects.
    record = ...
    return record

def allocation_attempt(specification):
    # Leave the actual allocation and OOM detection incomplete.
    outcome = ...
    return outcome

def allocation_worker(configuration, result_channel):
    # Run controls and the failed request inside this worker.
    ...

def E01(configuration):
    # Spawn, receive CPU-only evidence, and join with a timeout.
    report = ...
    return report

def main():
    ...

if __name__ == "__main__":
```

```
main()
```

Part 6 - Measurement, Validation, and Failure Analysis

Confirm that the raised failure is CUDA allocation OOM, not a host allocation error, integer overflow, or invalid shape. Compare computed payload bytes with any requested-size field present in the raw message; preserve both and explain rounding or unit ambiguity.

Require a complete traceback string and enough metadata to reproduce the attempt. Verify that serialized output contains no tensor, exception, frame, or traceback objects. Distinguish a successful same-worker control from success in a newly created process. Worker timeout and diagnostic failure must not be silently converted into successful recovery.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

This exercise establishes the smallest reproducible OOM incident. Explain what its failure proves about the request and what it does not prove about training-state recovery.

Use the incident schema as the foundation for later handlers. Reliable diagnostics preserve raw evidence while keeping the failed allocation scope free to release its references.

19.25.3 Exercise 02

Exercise Title: Distinguish Capacity OOM, Fragmentation-Related OOM, Temporary-Peak OOM, and Memory-Leak OOM Through Reproducible Experiments

Part 1 - Exercise Contract and Why It Matters

Construct four controlled scenarios with different intended pressure mechanisms: excessive live demand, fragmented allocation history, a brief temporary peak, and unintended reference retention over iterations. Deliver the intervention for each, its measured signature, and a diagnosis with evidence strength rather than an unjustified categorical label.

The four names are causal hypotheses, not necessarily mutually exclusive runtime exception types. A leak eventually creates a capacity problem, and a temporary workspace can fail in a fragmented allocator. Keep ground-truth interventions separate from what a diagnostic tool could infer from counters.

Part 2 - System and Memory Mental Model

Capacity pressure arises when necessary simultaneous live demand cannot fit. Temporary-peak pressure appears when overlapping buffers exceed capacity briefly even though end-of-step usage looks small. A retention leak leaves objects live beyond their intended lifetime, creating baseline growth. Fragmentation concerns whether available allocator blocks can serve the particular request under backend-specific reuse and segment rules.

High reserved bytes alone do not prove fragmentation. Compare allocation history, inactive split statistics where meaningful, actual block layout, and a fresh-process control with similar live payload. Native allocator statistics do not transfer unchanged to `cudaMallocAsync`. If

fragmentation cannot be demonstrated reliably on the device, report that result instead of relabelling ordinary live-memory pressure.

For a leak intervention, deliberately retain graph-connected outputs in a bounded Python container. A control retaining detached GPU tensors still retains their storage, while a CPU scalar log does not retain those GPU outputs. This helps separate graph retention from intentional storage retention and from an allocator cache.

Part 3 - Quantitative Prediction Before Execution

Define end-of-iteration live baseline A_i and within-iteration peak P_i. For a bounded retention intervention of roughly r bytes per iteration,

$$A_i \approx A_0 + ir$$

until storage reuse or more complex graph lifetimes change the approximation. For a temporary workspace of w bytes, expect a peak increase near w only when its lifetime overlaps the existing maximum.

Create a hypothetical stable baseline of 2 GiB, ordinary peak of 3 GiB, retained growth of 64 MiB/iteration, and a transient buffer of 512 MiB. Predict baseline and peak traces for ten iterations under each intervention. Do not add maxima from non-overlapping phases.

For fragmentation, list the extra evidence needed beyond A, R, and F. Predict why two runs with similar totals can yield different outcomes for one large request, and why a fresh-worker success is suggestive but not sufficient unless other conditions match.

Part 4 - Experimental Design and Instrumentation

1. Establish a repeatable no-pressure reference, recording cold and warmed allocator behavior and the backend.

2. Increase simultaneous live payload for the capacity case, with a bounded maximum and an explicit cleanup list.

3. Insert one temporary buffer during a chosen forward, backward, or optimizer phase; release it before the next iteration.

4. Retain outputs in a bounded list, then compare graph-connected, detached-GPU, and CPU-scalar variants under identical work.

5. Create alternating allocation sizes and retained live blocks for the fragmentation candidate. Compare with reordered allocations and a fresh-worker control.

6. Repeat each case and classify mixed signatures as mixed or undetermined. Never infer a unique cause from message wording alone.

Keep diagnostic profiling separate from final timing measurements.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use

a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class PressureScenario:
    mechanism: str
    maximum_live_bytes: int
    maximum_iterations: int
    allocation_order: tuple
    seed: int

def run_capacity_intervention(workload, scenario):
    evidence = ...
    return evidence

def run_temporary_peak_intervention(workload, scenario):
    evidence = ...
    return evidence

def run_reference_retention_intervention(workload, scenario):
    evidence = ...
    return evidence

def run_fragmentation_candidate(workload, scenario):
    evidence = ...
    return evidence

def classify_pressure_evidence(evidence, control):
    # Preserve uncertainty and permit multiple contributing causes.
    diagnosis = ...
    return diagnosis

def E02(workload, scenarios):
    comparison = ...
    return comparison
```

Part 6 - Measurement, Validation, and Failure Analysis

Require known intervention sizes and lifetimes, a matched control, and repeated observations. Verify live memory returns after deliberate retention is removed. A stable reserved cache with stable allocated baseline must not be diagnosed as a growing leak.

For fragmentation, state which backend statistics are meaningful and which are absent or not applicable. A zero in an unsupported metric is not evidence of no fragmentation. Preserve inconclusive scenarios and identify a measurement that could resolve them.
Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Diagnosis is an argument built from interventions and observations. Explain why the proposed recovery action addresses the measured cause: smaller work, shorter overlap, reference cleanup, or a different allocation pattern.

Avoid a universal cache-clearing remedy. The same exception class can represent several memory mechanisms, and the cheapest effective response depends on distinguishing them.

19.25.4 Exercise 03

Exercise Title: Record memory_allocated, memory_reserved, Free VRAM, and Peak Memory Immediately Before a Deliberate OOM

Part 1 - Exercise Contract and Why It Matters

Build a low-overhead telemetry recorder around one deliberately failing allocation or training phase. Capture current allocated and reserved bytes, their peaks, global free and total bytes, timestamps, device identity, and the next requested operation. Deliver pre-attempt, exception-time, and post-cleanup records with explicit observation ordering.

The phrase immediately before means the last available observation before the declared request, not an atomic view of the whole GPU. Identify the gap between that observation and allocation. Do not reconstruct a missing pre-failure record from counters sampled after cleanup.

Part 2 - System and Memory Mental Model

Counter reads occur at different times and have different scopes. Allocated and reserved counters belong to the selected process/device allocator; global free bytes include effects of other processes. Peak counters measure successful allocations since their reset, not the size of the request that failed.

CUDA execution and cross-stream frees can be asynchronous. A synchronized diagnostic boundary makes the experiment easier to interpret but changes overlap and can move the peak. Keep a minimally instrumented run for performance and a synchronized run for attribution. State which produced each observation.

An OOM handler can itself fail while collecting information. Preserve the last-known pre-attempt record on CPU, and treat exception-time telemetry as best effort. Do not allocate large diagnostic tensors on the already constrained GPU. Peak resets must not erase the evidence of an earlier phase in the same attempt.

Part 3 - Quantitative Prediction Before Execution

Let A_0 be live bytes at region start and P_A the peak since reset. The incremental observed rise is $P_A - A_0$, not P_A itself. Let P_R be peak reserved bytes. In general,

$$\max_{\tau} R(\tau) - \max_{\tau} A(\tau) \neq \max_{\tau} \Big(R(\tau) - A(\tau) \Big).$$

Thus subtracting the two independently timed peaks does not yield maximum cached bytes.

For observations $A_0 = 2\,\text{GiB}$, $P_A = 3.5\,\text{GiB}$, $P_R = 4\,\text{GiB}$, and a failed $1\,\text{GiB}$ request, calculate the observed rise. Explain why $4.5\,\text{GiB}$ is not automatically the required true peak. The request may occur at a different live baseline or after internal allocator retries.

Predict how resetting peaks after the OOM would change the report. Define the timing uncertainty $\Delta t = t_{\text{request}} - t_{\text{last snapshot}}$ and record it whenever the request boundary can be instrumented.

Part 4 - Experimental Design and Instrumentation

1. Define a CPU-only record schema with values, units, timestamp, scope, and availability status for each metric.
2. Initialize the context and fixed state, synchronize the declared start, then reset peak counters before the measured attempt.
3. Capture the final pre-request snapshot and a phase marker. Run the failing request and retain the pre-request record unchanged.
4. Capture best-effort exception-time metrics before cleanup, then a separate post-cleanup snapshot. Record secondary errors.
5. Compare synchronized diagnostic and ordinary asynchronous runs. Report their different overhead and observability.

Use the CUDA memory counter APIs and `mem_get_info` from `torch.cuda`; verify installed-version names before implementing.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import time
import torch

@dataclass
class MemoryRecord:
    attempt_id: str
    phase: str
```

```
    monotonic_time: float
    allocated_bytes: object
    reserved_bytes: object
    peak_allocated_bytes: object
    peak_reserved_bytes: object
    free_bytes: object
    total_bytes: object
    diagnostic_errors: tuple

def begin_measured_region(device, synchronization_policy):
    baseline = ...
    return baseline

def read_memory_record(device, attempt_id, phase):
    # Preserve unavailable metrics and their failure reasons.
    record = ...
    return record

def capture_attempt_timeline(workload, request, policy):
    # Keep pre-request evidence immutable across cleanup.
    timeline = ...
    return timeline

def compare_observation_scopes(timeline):
    interpretation = ...
    return interpretation

def E03(workload, request, synchronization_policies):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Check nonnegative byte values, correct device selection, and monotonic record ordering. Treat $A \leq R$ as an allocator-accounting expectation to investigate, not a licence to discard conflicting measurements. Global and process counters are not simultaneous enough to establish arbitrary exact equalities.

Demonstrate that the failed request does not appear as successfully allocated storage. Verify the original pre-failure record survives both cleanup and a failing diagnostic read. Report data gaps explicitly instead of filling them with zero or a later value.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Telemetry is valuable when its time and scope are clear. Explain which conclusions can be supported by your trace and which require an allocation history or operator-level profile.

Choose a production logging frequency that balances evidence and overhead. A low-overhead last-known record plus bounded incident escalation may be more useful than continuously perturbing every allocation.

19.25.5 Exercise 04

Exercise Title: Build a Safe try/except CUDA OOM Handler That Separates Memory Failures from Unrelated Runtime Errors

Part 1 - Exercise Contract and Why It Matters

Design a handler that recognises CUDA allocation OOM, records the failure, ends the failed attempt's scope, and permits a bounded recovery decision. Unrelated exceptions must propagate with their original context. Deliver an exception-routing table, synthetic routing tests, and one genuine allocation-OOM integration test.

Keep the handler's responsibilities narrow. It classifies and captures evidence; later exercises decide how to change work and restore training. It must not silently clear arbitrary state, skip data, or treat every exception as permission to halve the batch.

Part 2 - System and Memory Mental Model

Use the specific CUDA OOM exception exposed by the supported PyTorch version. Broad RuntimeError handling plus substring matching is fragile: message wording changes and unrelated errors can contain similar phrases. If a legacy compatibility path is required, declare the supported version and test it explicitly instead of weakening the primary path.

Failed function frames can retain model outputs, losses, and autograd graphs through the traceback. Converting the error to a CPU-only record and leaving that scope lets references be released before another large attempt. Retrying deeply inside the same exception block can preserve the very memory that caused the problem.

Diagnostic errors must not replace the original training failure. Preserve a primary failure record and a secondary diagnostic-error list. For device-fatal or unclassified errors, stop the recovery path; do not assume that telemetry calls or new allocations are safe. A retry budget should also bound time, not only attempt count.

Part 3 - Quantitative Prediction Before Execution

Define mutually exclusive routing outcomes for one observed exception: allocation OOM, unrelated error, and explicitly identified fatal context error. The routing class is separate from the eventual action: an OOM can still lead to no-fit termination.

Let $r_{\max}$ be allowed retries after the initial attempt. Maximum attempts are $1 + r_{\max}$; make that convention explicit. For per-attempt time t_i, diagnostic time d_i, and cleanup time c_i,

$$t_{\text{incident}} = \sum_i (t_i + d_i + c_i).$$

Set a wall-time limit as well as $r_{\max}$.

Build a hand-classified matrix containing a true allocation OOM, a shape-mismatch RuntimeError, a ValueError from configuration, a timeout, and a failure in the report writer. Predict the primary outcome, allowed diagnostics, propagation behaviour, and whether another CUDA attempt is authorised by the runtime contract. Do not induce a device assert just to fill a test row.

Part 4 - Experimental Design and Instrumentation

1. Test exception routing with labelled synthetic exceptions and no GPU work. Preserve tracebacks when unrelated exceptions propagate.

2. Implement a small attempt function whose locals include temporary objects. Return only an outcome record after its scope has unwound.

3. Add CPU-only diagnostic serialization without tensor locals. Inject failure into the serializer and verify primary evidence survives.

4. Run E01's genuine CUDA OOM and compare routing with the synthetic case. Mark their evidence sources distinctly.

5. Demonstrate bounded retries and a time-limit outcome. Recovery must occur outside the failed exception scope with an explicit new proposal.

Record context health as known usable, known fatal, or unknown. A generic message alone should not be used to assert device health.
Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every `...` marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import traceback
import torch

@dataclass
class AttemptOutcome:
    status: str
    primary_error_type: object
    primary_error_text: object
    traceback_text: object
    diagnostic_errors: tuple
    retry_eligible: bool

def format_cpu_failure_record(error, phase, metadata):
    record = ...
    return record
```

```
def run_attempt_in_own_scope(workload, configuration):
    # Implement narrow exception routing and preserve other errors.
    # Return no tensor-owning exception or traceback objects.
    outcome = ...
    return outcome

def collect_best_effort_diagnostics(primary_record, telemetry):
    enriched_record = ...
    return enriched_record

def enforce_retry_limits(history, time_budget, retry_budget):
    decision = ...
    return decision

def E04(workload, injected_cases, limits):
    routing_report = ...
    return routing_report
```

Part 6 - Measurement, Validation, and Failure Analysis

Require exact routing for every test case. An unrelated failure must not be converted into a retry or a false successful result. Check that the original exception remains primary when diagnostics fail.

Use weak references or a controlled lifetime probe to show the failed scope can release temporary objects. Inspect serializable records for tensor, frame, traceback, and exception objects. Verify that retry count and time budgets terminate even when every attempt produces OOM.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

A safe handler preserves meaning. It distinguishes insufficient memory from a broken program and preserves enough evidence for the next decision without prolonging failed object lifetimes.

Explain why exception routing belongs below recovery policy. Keeping them separate makes it possible to change the recovery strategy without accidentally swallowing new classes of bugs.

19.25.6 Exercise 05

Exercise Title: Release Dead Tensor References After OOM and Prove Which Allocations Remain Alive Because Python Still Holds References

Part 1 - Exercise Contract and Why It Matters

Build a reference-lifetime experiment that explains why memory can remain allocated after a variable is deleted or a failed function returns. Compare direct aliases, views, detached tensors, graph-connected outputs, container references, closures, and exception tracebacks. Deliver an ownership diagram and a before/after allocation table.

Restrict cleanup to objects created by the experiment. Model parameters and optimizer state may remain live intentionally; deleting them to improve a memory counter does not demonstrate a valid recovery path.

Part 2 - System and Memory Mental Model

Python names refer to objects; tensor objects can share underlying storage. Deleting one name does not free storage still referenced by another tensor or graph. A detached tensor may share storage with its source, and a view can keep a large base allocation alive even when its own logical shape is small. An independent clone owns separate storage rather than freeing the original allocation.

Autograd may retain tensors needed by a future backward pass. Saving a loss or output tensor in a history list can therefore retain more than its visible payload. Converting a scalar observation to a CPU number avoids keeping that graph in the log, although extracting a CUDA scalar can introduce synchronization and should be timed separately.

Garbage collection addresses unreachable Python objects, including cycles; it cannot free objects still intentionally reachable. A weak reference becoming dead proves a particular Python object died, not that all aliases to its storage disappeared. Storage-lifetime evidence must combine reference controls and allocator observations.

Part 3 - Quantitative Prediction Before Execution

Let an experiment allocate one storage of N elements of size s, with k aliases. Its payload is Ns, not kNs. If a small view covers $m \ll N$ elements, the view's logical bytes ms need not equal the retained base allocation.

For a hypothetical 256 MiB base tensor, a 1 MiB view, one detached alias, and one independent clone, predict live storage after deleting each owner in a declared order. Explain which deletions change Python reachability without releasing any allocator bytes.

$$\Delta A = A_{\text{before}} - A_{\text{after}}$$

measures released live allocator storage at the observation boundaries, subject to asynchronous work and other allocations.

For a bounded history of graph-connected outputs, predict whether memory growth follows visible output payload alone. State which internal graph storage makes the analytic estimate only a lower bound.

Part 4 - Experimental Design and Instrumentation

1. Warm the allocator with a small control, then record a clean live baseline. Use allocation sizes large enough to distinguish signal from minor bookkeeping variation.

2. Construct each ownership case independently and record storage identity without using it as a long-term reference.

3. Remove one owning reference at a time. Synchronize at the declared boundary and observe live and reserved bytes separately.

4. Compare reachable containers with a deliberately unreachable cycle. Use garbage collection only where the object-lifetime experiment calls for it, and measure its cost.

5. Reproduce a failed attempt whose traceback is retained, then replace the retained exception with formatted text and let the failure scope end.

6. Verify the same model and optimizer remain usable after cleanup.

Keep lifetime probes weak where possible; a diagnostic tool must not become the last strong reference to the object it is studying.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every `...` marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import gc
import weakref
import torch

@dataclass
class OwnershipObservation:
    case_name: str
    removed_owner: str
    tensor_object_alive: bool
    allocated_bytes: int
    reserved_bytes: int

def create_alias_and_view_case(device, payload_bytes):
    ownership_handles = ...
    return ownership_handles

def create_graph_retention_case(model, inputs):
    ownership_handles = ...
    return ownership_handles

def observe_after_owner_release(handles, owner_name, device):
    # Remove only the designated experiment-owned reference.
    observation = ...
```

```
    return observation

def compare_exception_object_with_text_record(workload):
    evidence = ...
    return evidence

def prove_model_state_survived_cleanup(runtime, reference):
    audit = ...
    return audit

def E05(runtime, ownership_cases):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Require an explicit list of owners, their removal order, and predicted versus observed storage changes. Check that a view or detached alias can outlive the original Python tensor object. Do not treat a dead weak reference as sufficient proof of storage release.

Verify post-cleanup live allocation returns near the controlled baseline, while reserved cache may remain. Check model parameters, optimizer references, and a subsequent update for correctness. If logging changes the observed lifetime, identify and remove that instrumentation artifact.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Reference cleanup is an ownership operation, not a command that magically frees everything unused by the current line of code. Explain which layer owns each remaining allocation and whether that ownership is justified.

Recommend logging and attempt-scope patterns that prevent accidental retention without deleting training state. These design choices are often more effective than repeated allocator-cache operations.

19.25.7 Exercise 06

Exercise Title: Test torch.cuda.empty_cache() During Recovery and Prove Why It Cannot Free Memory Still Owned by Live PyTorch Tensors

Part 1 - Exercise Contract and Why It Matters

Measure cache release under three conditions: live tensors remain, tensor owners are released, and released blocks coexist with retained fragments or pending work. Compare live allocated bytes, reserved bytes, global free memory, and the cost of the next allocation. Deliver a controlled before/after table and a recovery recommendation supported by evidence.

The goal is to establish what cache release can and cannot do. It must not be described as deletion of model state, a universal fragmentation repair, or a substitute for reducing a genuinely oversized working set.

Part 2 - System and Memory Mental Model

PyTorch's allocator can retain unused blocks for reuse. Releasing unused cache can return releasable memory to the device allocator and make it available to other applications. Storage owned by live tensors remains allocated. Blocks tied to live parts of an allocation or pending stream work may not be releasable at the chosen instant.

Because cached blocks were already reusable by the process under allocator rules, clearing them need not increase that process's ability to run the same working set. It may change fragmentation behaviour in particular cases, and it can increase later allocation cost by removing reusable cache. Allocator backends differ; record the selected backend and do not assume an exact reservation drop.

An OOM may occur only after internal allocator retries and cache-release attempts. Therefore another manual cache clear is not guaranteed to change the outcome. First determine whether unnecessary references or future demand changed. Separate logical state restoration from allocator cleanup.

Part 3 - Quantitative Prediction Before Execution

Let A be live allocated bytes and R reserved bytes before cache release. The gap $R - A$ is an upper accounting quantity for unused reservations, not a guaranteed release amount. In a controlled single-process experiment, expect no removal of live tensor payload:

$$A_{\text{after}} \approx A_{\text{before}}$$

when owners remain and no other workload changes.

For illustrative $A = 2\,\text{GiB}$ and $R = 3\,\text{GiB}$, predict the range of plausible reservation changes if all, some, or none of the unused blocks are releasable. Then remove a 512 MiB tensor's last owner and predict the difference between immediate live-storage release and later reservation release.

Measure the tradeoff

$$t_{\text{recovery}} = t_{\text{reference cleanup}} + t_{\text{cache release}} + t_{\text{next allocation}}.$$

Predict why a per-step cache-clear policy can lose throughput even if global free memory looks larger between updates.

Part 4 - Experimental Design and Instrumentation

1. Use a dedicated worker and a fixed allocation pattern. Record baseline and backend, then retain a known live tensor.

2. Call cache release while that tensor remains reachable. Verify its content and observe allocated and reserved values.

3. Release its last owner, observe again, then release unused cache. Keep these as separate experimental stages.

4. Repeat with another live alias and with a controlled allocation history that may leave non-releasable fragments. Report inconclusive cases.

5. Compare no manual cache release, release only after OOM cleanup, and release before every update under the same work.

Time synchronization and cache operations explicitly. A noisy global free reading on a shared GPU is not an exact measure of this process's release.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class CacheExperimentRecord:
    stage: str
    allocated_bytes: int
    reserved_bytes: int
    global_free_bytes: int
    elapsed_seconds: float
    live_tensor_valid: object

def create_controlled_cache_state(device, allocation_plan):
    handles = ...
    return handles

def observe_cache_release_with_live_owners(handles, device):
    record = ...
    return record

def observe_release_after_owner_cleanup(handles, device):
    record = ...
    return record

def benchmark_cache_policy(workload, policy, repetitions):
    measurements = ...
    return measurements

def E06(workload, allocation_plan, policies):
```

```
comparison = ...
return comparison
```

Part 6 - Measurement, Validation, and Failure Analysis

Require the live tensor to remain readable and unchanged after cache release. Verify that deleting only one of several aliases does not release their shared storage. Separate reservation change from live allocation change in all charts.

Keep failed recovery attempts in the comparison. A successful retry after both deleting references and clearing cache does not prove cache release was the necessary cause; include the reference-cleanup-only control. Report next-allocation latency and accepted throughput, not only the temporary rise in global free memory.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Cache release changes allocator ownership of unused blocks. It does not reduce the live workload by itself. Explain when returning unused blocks is helpful for cooperation with other processes and when it adds cost without solving the incident.

Recommend the narrowest effective cleanup policy for the measured workload, with a clear distinction between reference lifetime, allocator reuse, and physical device availability.

19.25.8 Exercise 07

Exercise Title: Reset a Failed Training Iteration Without Corrupting Model Parameters, Gradients, Optimizer State, or Random Number State

Part 1 - Exercise Contract and Why It Matters

Implement transactional rollback for one logical optimizer update. Capture independent CPU state before the window, inject failures at different phases, restore the last committed state, and compare a replay with a clean reference. Deliver a state inventory, snapshot cost, restore cost, and a component-by-component correctness report.

Begin at a clean gradient boundary, recording whether each gradient is absent or a tensor. Use a deterministic model for exact tests. The base runtime owns all state it mutates; external database writes, arbitrary callbacks, and uncontrolled data-loader workers need additional handling and are not silently covered by a model state dictionary.

Part 2 - System and Memory Mental Model

Model parameters are only part of a training transaction. Buffers such as running statistics can change during forward. Optimizer moments and per-parameter step counters can change during an update. RNG includes Python, NumPy when used, CPU PyTorch, and each CUDA generator used by the attempt. Data order, prepared augmentation, scaler, and scheduler progress also belong to the contract.

A state dictionary can contain references to live storage. Make independent CPU copies before mutation; copying a dictionary alone is insufficient. Restore parameter values without

breaking the optimizer's association with parameter objects, or recreate and rebind both explicitly. Absent gradients and zero-valued gradients can have different optimizer semantics, including whether a parameter is updated.

AMP recovery must restore a coherent scaler lifecycle. A saved scaler state may not capture transient per-optimizer state from a failed unscale/step phase. Recreate the scaler from a clean committed checkpoint or otherwise establish a documented fresh lifecycle before replay. Likewise, restore RNG before recreating stochastic work. Restoring the seed does not guarantee identical results after changing kernel shapes or micro-batch partitions.

Part 3 - Quantitative Prediction Before Execution

Let S_0 be the committed snapshot and S_f the state after a failed partial update. Rollback should establish

$$\text{restore}(S_f, S_0) = S_0$$

for all state covered by the contract, before retry. For deterministic replay of the same plan, compare S_{retry} with S_{clean}. Use exact equality for counters, IDs, and RNG states, and declared absolute/relative tolerances for numerical tensors.

For a model with P FP32 parameters and Adam moments, estimate CPU snapshot payload of parameters and two moments as $12P$ bytes, then add buffers, gradients if included, counters, and serialization overhead. For $P = 10^7$, calculate MiB and predict copy/restore time from a measured transfer bandwidth rather than a marketing specification.

Construct a partial-optimizer fault after some parameters have changed. Predict why clearing gradients and restoring only model parameters cannot reproduce the next Adam update.

Part 4 - Experimental Design and Instrumentation

1. Materialize the exact CPU input window and sample IDs before the snapshot, so replay does not depend on uncontrolled prefetch or augmentation.

2. Capture independent model, optimizer, buffer, gradient-presence, RNG, scaler, scheduler, and cursor state at the committed boundary.

3. Inject faults after forward mutation, after partial backward, and after deliberately partial optimizer mutation. Label synthetic faults.

4. End the failed scope, clean temporary references, restore all components, then perform a clean replay of the same plan.

5. Compare with a no-failure reference from the same snapshot. Measure CPU memory, snapshot time, restore time, and accepted-update latency.

6. Mutate one live tensor after snapshot to prove snapshot independence.

For initial exact testing, use batch-independent layers and no dropout. Add stochastic cases only with explicit equality or distributional criteria.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use

a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import random
import torch

@dataclass
class TrainingCheckpoint:
    model_cpu: object
    optimizer_cpu: object
    gradient_state: object
    rng_states: object
    scaler_state: object
    scheduler_state: object
    data_state: object

def capture_independent_checkpoint(runtime, prepared_window):
    checkpoint = ...
    return checkpoint

def inject_phase_failure(runtime, failure_phase, mutation_plan):
    # Leave controlled partial mutation and fault injection incomplete.
    ...

def restore_committed_state(runtime, checkpoint):
    # Preserve parameter bindings and reset transient scaler lifecycle.
    ...

def compare_state_components(reference, candidate, tolerances):
    audit = ...
    return audit

def replay_from_checkpoint(runtime, checkpoint, prepared_window):
    outcome = ...
    return outcome

def E07(runtime, prepared_window, failure_cases, tolerances):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Require equality for sample cursor, counters, RNG, scheduler progress, and gradient-presence pattern. Compare parameters, buffers, and nested optimizer tensors individually. Matching loss alone does not certify rollback.

Test snapshots before and after lazy optimizer-state initialization. Check independence of nested tensor copies and correct optimizer bindings after restoration. Verify the retry applies exactly one accepted update. Include a deliberately incomplete rollback control and show the state component that reveals its error.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Rollback converts recovery from guesswork into a transaction with a testable boundary. Explain which costs the full snapshot incurs and which failures could permit a cheaper specialised rollback only after proving that no additional state mutated.

Define the supported replay semantics clearly: exact same-plan replay under deterministic conditions, numerical equivalence for compatible repartitioning, or a separately justified stochastic criterion.

19.25.9 Exercise 08

Exercise Title: Detect an OOM During Forward and Retry the Same Input with a Smaller Micro-Batch Without Restarting the Process

Part 1 - Exercise Contract and Why It Matters

Recover from a forward allocation OOM by replaying the same logical input window through smaller physical micro-batches. Preserve the process, sample membership, target alignment, and one-update learning objective. Deliver a phase trace, retry plan, same-process evidence, and a numerical comparison against a clean execution of the final successful plan.

Keep the master input window on CPU and transfer only the active micro-batch. Slicing a full GPU batch after failure may retain its oversized storage and defeat recovery. Use a bounded retry policy and return no-fit when the smallest admissible input still cannot run.

Part 2 - System and Memory Mental Model

Forward is not necessarily free of state changes. Dropout consumes random numbers, BatchNorm updates buffers, and hooks may mutate application state. A forward failure late in the model can therefore require rollback even when no backward call began. Use E07's transaction boundary unless the model has a proven narrower side-effect contract.

When a forward attempt fails, its frame can retain inputs and partial outputs through the exception traceback. End the failed scope and release temporary owners before retrying. A smaller new allocation cannot help if failed graphs and large inputs remain live elsewhere.

Splitting inputs does not universally preserve the same mathematical computation. Batch-dependent normalisation, contrastive objectives, and cross-sample operations can change. Use a sample-separable loss and batch-independent model for the base equivalence test. Compare stochastic runs with a declared criterion rather than demanding unexplained bitwise equality across changed shapes.

Part 3 - Quantitative Prediction Before Execution

For a logical window of B samples and micro-batch sizes b_k,

$$\sum_k b_k = B, \qquad \mathcal{L} = \sum_k \frac{b_k}{B} \overline{\mathcal{L}}_k.$$

Retain the intended denominator even when the last micro-batch is short. An approximate peak model is $M_{\text{fixed}} + \alpha b + W(b)$. Shrinking b affects variable demand, not all persistent memory.

For $B = 30$, predict partitions for upper micro-batches 16, 8, and 4. Suppose forward at 16 fails but 8 fits. Calculate the final partial micro-batch and its weight. Predict memory savings if $M_{\text{fixed}} = 2\,\text{GiB}$ and $\alpha = 64\,\text{MiB/sample}$, ignoring workspace changes initially.

Include wasted first-attempt time and restoration in throughput:

$$Q = \frac{B}{t_{\text{failed}} + t_{\text{restore}} + t_{\text{successful window}}}.$$

Do not compare only the final successful attempt with the static baseline.

Part 4 - Experimental Design and Instrumentation

1. Create a deterministic sample-separable model and CPU window with immutable IDs. Save a committed snapshot.

2. Establish a controlled forward pressure threshold in an isolated calibration, then trigger a genuine forward OOM or use a clearly labelled fault hook when hardware conditions do not permit a reliable trigger.

3. Record the surfacing phase and last completed marker. End the failed scope and verify temporary live allocations are released.

4. Restore the snapshot, reduce the physical plan, and replay the entire logical window with correct weights and one final update.

5. Compare with a clean reference that starts from the same snapshot and uses the eventual successful micro-batch plan.

6. Record process identity, attempt count, state audits, peaks, accepted IDs, and full incident time.

Probe at most a bounded set of smaller plans. If batch one fails, stop or defer explicitly rather than retrying an identical request forever.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import os
import torch

@dataclass
class ForwardRetryPlan:
    logical_sample_ids: tuple
    maximum_micro_batch: int
    retry_index: int
    normalization_count: int

def execute_forward_attempt(runtime, cpu_micro_batch, plan):
    # Report only CPU metadata after the failed scope unwinds.
    outcome = ...
    return outcome

def release_failed_forward_owners(runtime, outcome):
    # Keep committed model and optimizer state intact.
    ...

def propose_smaller_forward_plan(previous_plan, limits):
    next_plan = ...
    return next_plan

def replay_logical_window(runtime, checkpoint, cpu_window, plan):
    result = ...
    return result

def audit_forward_recovery(reference, recovered, sample_ids):
    audit = ...
    return audit

def E08(runtime, cpu_window, initial_plan, limits):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify each logical sample appears once in the accepted update, even when several failed attempts processed it speculatively. Check loss weights, final remainder, parameters, buffers, optimizer state, and gradient values against the compatible reference.

Test early and late forward failures, a short final micro-batch, and batch-one no-fit. Verify process continuity and ensure the CPU input master is not accidentally replaced by a full-window GPU tensor. Report any semantic limitation from stochastic or batch-dependent layers.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Forward recovery reduces the active working set while preserving a logical transaction. Explain how ownership cleanup, rollback, and repartitioning each contribute a different part of the solution.

Recommend the largest tested safe retry plan only within the observed shape and budget. A forward success still does not certify that the subsequent backward and optimizer phases fit.

19.25.10 Exercise 09

Exercise Title: Detect an OOM During Backward and Recover Without Accidentally Applying Partial or Stale Gradients

Part 1 - Exercise Contract and Why It Matters

Recover from a backward OOM without applying partially accumulated or stale gradients. Demonstrate failure after some parameter gradients exist, reset the complete failed logical window, and replay from the committed snapshot. Deliver per-parameter gradient evidence and a comparison against a clean successful update.

The base policy discards all gradient work from the failed logical window. Selective salvage is an advanced extension requiring a proof of which contributions are complete and nonduplicated; it must not be the implicit behaviour of the first implementation.

Part 2 - System and Memory Mental Model

Backward executes many gradient computations, not one atomic operation. Some parameters may have new gradients while others still have none or retain earlier accumulation contributions. An OOM does not mark those tensors with a reliable complete-window flag.

The partially consumed autograd graph is also not a reusable transaction. Some intermediates may already be freed, while others remain reachable through failure frames. Reusing it with graph-retention settings does not establish correct gradient coverage. Rebuild forward computation from restored state and replay the full logical window.

Clearing gradients is necessary for the base strategy but may not be sufficient: earlier forwards can mutate buffers and RNG, and hooks can change custom state. If failure occurs before the optimizer, parameter values may still match the snapshot, yet the complete state audit remains necessary. Gradient presence and values both matter.

Part 3 - Quantitative Prediction Before Execution

Let g_j be the intended total gradient of parameter j, and let $\widetilde{g}_j$ be an unknown partial value after failure. Naively replaying into existing buffers produces

$$g_j^{\text{wrong}} = \widetilde{g}_j + g_j$$

when replay contributes a full window. In general, the unknown partial term differs by parameter and cannot be corrected by one global factor.

For two equally weighted micro-batches, suppose the first contribution completed and the second backward failed midway. Write the possible gradient states for three parameters: complete, partial, and untouched by the failing pass. Predict why matching the overall gradient norm cannot prove correctness.

Measure per-parameter error

$$E_j = \frac{\|g_j^{\text{retry}} - g_j^{\text{ref}}\|_2}{\max(\|g_j^{\text{ref}}\|_2, \epsilon)}.$$

Before running, predict error under full rollback, gradient-reset-only with mutable buffers, and naive continuation.

Part 4 - Experimental Design and Instrumentation

1. Use a deterministic multi-layer model and a logical window that contains at least two micro-batches. Save the committed state.

2. Trigger or inject a backward failure after selected gradients have been written. Capture presence and summary values on CPU.

3. Run a labelled incorrect control that demonstrates stale gradient contamination without committing it to the main runtime.

4. End the failed scope, discard all window gradients, restore the transaction, and rebuild forward/backward using a smaller safe plan.

5. Compare every parameter gradient before the update, then compare parameters and optimizer state after the single accepted update.

6. Repeat with forward-mutated buffers and with AMP, treating numerical overflow separately from allocation OOM.

Do not step the optimizer from a failed backward result. Diagnostic gradient copying must be bounded so it does not become a new failure source.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class GradientObservation:
    parameter_name: str
    gradient_present: bool
    finite_status: object
    norm_value: object
    contribution_status: str

def capture_partial_gradient_evidence(model, size_limit):
    evidence = ...
    return evidence

def run_backward_attempt(runtime, window, micro_plan, fault_hook):
    outcome = ...
    return outcome

def invalidate_failed_gradient_window(runtime, checkpoint):
    # Discard the complete logical window and restore its state.
    ...

def rebuild_and_replay_backward(runtime, checkpoint, window, plan):
    recovered = ...
    return recovered

def compare_named_gradients(reference, recovered, tolerances):
    errors = ...
    return errors

def E09(runtime, window, failure_cases, limits):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Require no optimizer step before a complete validated backward window. Check gradient presence, finite status, per-parameter values, and accepted sample counts. A norm-only or loss-only comparison is insufficient.

Test failure in the first micro-batch, after earlier successful micro-batches, and at different layers. Verify no tensors from a partially used graph are reused on replay. If hooks or buffers mutate, confirm their state is restored or explicitly excluded from supported models.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Backward recovery must treat unknown partial gradients as uncommitted work. Explain why complete-window replay is a simple correctness baseline even when it wastes computation.

Discuss what evidence would be required for finer-grained salvage. Any optimisation must preserve exactly the same accepted gradient contributions, not just reduce the number of repeated kernels.

19.25.11 Exercise 10

Exercise Title: Implement Recursive Batch Splitting That Automatically Divides an OOMing Batch Until Every Sub-Batch Fits in VRAM

Part 1 - Exercise Contract and Why It Matters

Build a recursive partition planner that splits an OOMing batch into smaller ordered sub-batches. Include odd batch sizes, a minimum leaf size, bounded depth, and a singleton no-fit outcome. Deliver the split tree, attempt history, accepted leaf manifest, and exact sample-coverage proof.

For training, use a conservative transaction model: a failure refines the partition, then the complete logical window is rolled back and replayed. Do not preserve earlier successful leaf gradients accidentally while replaying those leaves again. Distinguish this training contract from independent inference calls where leaf outputs can be safely concatenated.

Part 2 - System and Memory Mental Model

A split tree represents a partition of the logical sample set. Every internal node must produce disjoint children whose union is the parent, preserving the declared order. A leaf becomes feasible only after the required forward, backward, and relevant complete-window phases have been validated.

Success at a small isolated leaf does not guarantee the final optimizer step fits, especially when lazy state is created there. Likewise, aggregating all leaf outputs on GPU can recreate the original memory problem. Stream CPU-only evidence and retain only state required by the training transaction.

Recursion cannot shrink a singleton's sequence length or persistent model state. If the smallest leaf fails, return no-fit for this action and allow a different recovery strategy later. Bounds on depth alone do not bound repeated whole-window replay; track total attempts and time as separate limits.

Part 3 - Quantitative Prediction Before Execution

For a node containing $n > 1$ samples, use a balanced integer partition:

$$n_{\text{left}} = \lfloor n/2 \rfloor, \qquad n_{\text{right}} = n - n_{\text{left}}.$$

If splitting all the way to singletons, the resulting binary tree contains at most $2B - 1$ nodes and has balanced depth $\lceil \log_2 B \rceil$. These are tree bounds, not bounds on GPU calls when failed windows are replayed repeatedly.

For $B = 13$ and a hypothetical feasible leaf limit of four, draw the tree, list leaf sizes, and verify their weights sum to one for a sample-average objective. Predict how much earlier leaf work must be replayed if a later leaf fails.

Define a progress measure based on unresolved splittable nodes or maximum failing leaf size. Prove that each refinement makes progress, while a singleton failure exits explicitly.

Part 4 - Experimental Design and Instrumentation

1. Validate the planner on CPU with a fake feasibility function. Test $B = 1$, odd sizes, exact thresholds, and no feasible leaf.

2. Maintain immutable sample ranges in tree nodes. Check disjointness, coverage, and order after every refinement.

3. Integrate with a CUDA attempt function and E07 rollback. On failure, refine the implicated leaf and restart the logical window from its committed snapshot.

4. Validate the complete optimizer update, not only individual leaves. Retain a bound on retries and elapsed time across the whole tree.

5. Compare with repeated global halving under the same input and budget. Measure total work replayed and accepted throughput.

If an inference-only variant is implemented, compare it separately and explain why its output-handling and transaction rules differ.
Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every `...` marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class BatchNode:
    node_id: str
    start_index: int
    stop_index: int
    depth: int
    status: str

def split_failed_node(node, minimum_leaf_size):
    # Produce disjoint ordered children or a no-fit outcome.
    children = ...
    return children
```

```
def audit_partition_tree(root_ids, leaf_nodes):
    coverage_audit = ...
    return coverage_audit

def refine_partition_after_failure(tree, failed_node, limits):
    refined_tree = ...
    return refined_tree

def execute_partition_transaction(runtime, window, tree):
    # Discard all window gradients if any required phase fails.
    outcome = ...
    return outcome

def replay_refined_tree(runtime, checkpoint, window, tree, limits):
    result = ...
    return result

def E10(runtime, window, initial_tree, limits):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Require leaf sizes to be positive, ordered, disjoint, and exhaustive. No recursive call may receive the same unsplittable input indefinitely. Check depth, total-attempt, and wall-time limits independently.

Verify every accepted sample contributes once with the correct weight despite repeated speculative processing. Compare gradients and the final optimizer state with a clean execution of the final leaf plan. Test optimizer-step OOM after all leaves individually succeed and return it as a separate recovery failure.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Recursive splitting localises shape pressure and provides an auditable partition tree. It is not a universal OOM solution because some demand does not scale with sample count.

Explain whether finer partitioning saved time compared with global halving once whole-window replay was counted. Prefer the simpler strategy when the extra tree machinery provides no measured benefit.

19.25.12 Exercise 11

Exercise Title: Preserve the Intended Effective Batch Size After OOM Recovery by Combining Micro-Batch Reduction with Gradient Accumulation

Part 1 - Exercise Contract and Why It Matters

Integrate OOM recovery with exact logical-batch accumulation. A retry may change physical partitioning but must preserve the same logical samples, loss denominator, accepted optimizer-step count, and scheduler progress. Deliver partition manifests, gradient comparisons, and a recovery-aware training trace.

Use a sample-separable objective first. Add a valid-token-average variant with masking and unequal sequence lengths. Batch-coupled losses need specialised algorithms and must not be claimed equivalent through ordinary accumulation alone.

Part 2 - System and Memory Mental Model

For equal-sized micro-batches, dividing each mean loss by a constant accumulation count can be correct. Recovery often produces unequal sizes, making that shortcut wrong. Weight each contribution according to the logical objective: sample counts for a sample mean, or valid loss-bearing token counts for a token mean.

The optimizer, gradient clipping, scaler progression, and accepted-update scheduler belong at the logical boundary. With AMP, keep the scale consistent through the accumulation window. OOM rollback must restore a coherent scaler lifecycle before replay; numerical overflow is a separate outcome that must not be treated as insufficient VRAM.

Changing physical batch size should not cause the data cursor to commit early. Keep immutable prepared inputs and advance accepted state once. Regularisation, auxiliary losses, and metrics need their own normalisation rules so a term is not multiplied by the number of retry micro-batches.

Part 3 - Quantitative Prediction Before Execution

For logical count N and micro-batch counts n_k,

$$\sum_k n_k = N, \qquad \mathcal{L} = \sum_k \frac{n_k}{N} \overline{\mathcal{L}}_k, \qquad g = \sum_k \frac{n_k}{N} \nabla \overline{\mathcal{L}}_k.$$

Use the intended logical denominator, not the size of a failed physical attempt. Define behaviour for zero valid tokens explicitly.

For a 100-sample window, compare partitions capped at 32, 24, and 16. Write all sizes and weights. For valid-token counts $(200, 500, 300)$, compute the weights and compare them with equal micro-batch weighting.

For a final dataset tail of 37 samples, state whether to accept a smaller logical update, carry it to the next epoch, or drop it explicitly. Do not label that window as 100 effective samples without adding real, accounted-for data. Predict the accepted update count under the chosen policy.

Part 4 - Experimental Design and Instrumentation

1. Create a reference logical window whose full-batch computation fits on a small deterministic model. Save committed state.

2. Trigger failure under a larger physical plan, then recover with an unequal partition and replay the same sample IDs.

3. Compare exact weighting with a labelled incorrect equal-weight control. Capture gradients before the single final optimizer update.

4. Add clipping after the complete gradient is available. For AMP, use the appropriate unscale/validate/clip/step boundary once per window.

5. Test overflow skip, zero-valid-token data, and a final partial dataset window under declared policies.

6. Record accepted samples or valid tokens, retries, physical plan, optimizer steps, scheduler steps, scaler state, and full wall time.

The base experiment is single-device. Multiple ranks with unequal work require collective agreement and correctly derived global weights.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every `...` marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class LogicalObjective:
    sample_ids: tuple
    denominator_kind: str
    normalization_count: int
    tail_policy: str

def build_recovery_partition(window, maximum_micro_batch):
    partition = ...
    return partition

def determine_contribution_weight(micro_batch, objective):
    weight = ...
    return weight

def accumulate_recovery_window(runtime, partition, objective):
    # Leave weighted loss, backward, and scaler placement incomplete.
    outcome = ...
    return outcome
```

```
def validate_and_commit_once(runtime, objective, outcome):
    commit_record = ...
    return commit_record

def audit_accepted_update_semantics(reference, recovered):
    audit = ...
    return audit

def E11(runtime, windows, initial_plan, recovery_limits):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Check count conservation, nonoverlapping sample IDs, correct token masking, and exact accepted step counts. Compare every named gradient, updated parameter, and optimizer state with the compatible reference. Report non-finite values separately from allocation failures.

Test an OOM late in accumulation and ensure no earlier contribution is duplicated after replay. A scaler skip must not be counted as an accepted update if the benchmark defines acceptance by optimizer commit. Document the treatment of auxiliary objectives and tail windows.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

The effective batch is a learning contract. Memory recovery is allowed to change execution only within the semantics that contract permits. Explain how normalisation and commit accounting preserve it.

Report the cost of smaller micro-batches and full-window retries alongside memory savings. A runtime that survives OOM by quietly changing sample weights has not recovered the intended training job.

19.25.13 Exercise 12

Exercise Title: Recover from a Sequence-Length OOM by Dynamically Reducing Token Budget Instead of Only Reducing Sample Count

Part 1 - Exercise Contract and Why It Matters

Recover from variable-length sequence pressure using a token-aware partitioner and shape-aware admission. Compare sample-count reduction, useful-token budgeting, and padded-token budgeting with an explicit attention-memory model. Deliver the original and recovered batch manifests, memory measurements, and exact valid-token accounting.

Preserve sequence content, targets, masks, and the intended logical objective. Lowering a token budget is a scheduling change; it is not permission to truncate text or change context

length silently. A singleton that exceeds memory requires another strategy or an explicit no-fit outcome.

Part 2 - System and Memory Mental Model

One long sequence can force an entire dense batch to use a large padded length. Useful-token count measures content, while padded-token count better reflects some dense activations. Neither alone bounds all attention implementations: materialised score matrices can scale with bL^2, and workspace choices can add discontinuities.

Recovery should inspect CPU length metadata before rebuilding CUDA inputs. Preserve a logical window of prepared samples and change its physical packing, while weighting the loss by the original valid-token denominator if that is the objective.

Bucketing can reduce padding without shortening a sequence, but it changes execution order and may change stochastic behaviour. Compare deterministic clean replay of the final schedule. Packed attention or sequence chunking requires a separate validated contract; arbitrary chunks do not preserve full-context attention dependencies.

Part 3 - Quantitative Prediction Before Execution

For a proposed dense batch $\mathcal{B}$,

$$U = \sum_{i \in \mathcal{B}} L_i, \qquad P = |\mathcal{B}| L_{\max}, \qquad C_{\text{attn}} = |\mathcal{B}| L_{\max}^2.$$

A token cap constrains a declared unit, for example $P \leq K$. Admission must also constrain predicted upper bytes under the selected backend and precision.

Compare eight sequences of length 128 with one of length 1024. Both contain 1024 useful tokens, but calculate their respective C_{attn} values. For lengths $(128, 160, 192, 1024)$, compute padding when batched together and when the long item is isolated.

Before execution, predict the outcome of reducing the sample count to one for a sequence whose singleton peak already exceeds the cap. State which recovery actions can preserve its full context and which would change the learning task.

Part 4 - Experimental Design and Instrumentation

1. Freeze a CPU dataset with IDs, actual lengths, masks, and target counts. Include rare long sequences and one declared oversized singleton.

2. Trigger or inject a sequence-related OOM and record the exact collated shape, not just sample count.

3. Restore the logical transaction and propose a smaller padded-token budget with shape-aware validation.

4. Repack without dropping or truncating tokens. Execute the complete logical update with the original normalisation denominator.

5. Compare with a clean final-plan reference and with sample-only halving. Measure useful tokens/s, padding, retries, memory, and no-fit time.

6. Route singleton no-fit to a later recovery action or explicit deferral; do not loop on an impossible token cap.

For every sample, preserve a content identity or checksum computed on CPU so accidental truncation is detectable.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class SequenceWindow:
    sample_ids: tuple
    sequence_lengths: tuple
    valid_token_counts: tuple
    content_identities: tuple

def inspect_failed_sequence_shape(failure_record, cpu_window):
    shape_evidence = ...
    return shape_evidence

def propose_reduced_token_budget(previous_budget, limits):
    proposal = ...
    return proposal

def repack_without_truncation(window, token_budget, memory_profile):
    plan = ...
    return plan

def classify_oversized_singletons(plan, memory_profile, limits):
    no_fit_items = ...
    return no_fit_items

def replay_token_weighted_window(runtime, checkpoint, window, plan):
    outcome = ...
    return outcome

def E12(runtime, sequence_windows, token_policy, limits):
    report = ...
```

```
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify identical sequence content, valid tokens, targets, and masks before and after recovery. Check exact sample coverage and the original loss denominator. Compare gradients and accepted parameters with the compatible clean reference.

Test a zero-valid-token item, one extreme outlier, equal-token batches with different shapes, and a singleton no-fit. Report whether the token budget measures useful or padded tokens. A lower token total is not itself proof of lower peak VRAM.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Token-aware recovery changes how unchanged sequence work is grouped. Explain which memory terms it reduces and why some full-context sequences still need a different intervention.

Recommend a policy that preserves content and makes no-fit visible. Silent truncation may keep the process alive while changing the task, which is outside this recovery contract.

19.25.14 Exercise 13

Exercise Title: Build a Recovery Ladder That Tries Smaller Batch Size, Activation Checkpointing, CPU Offload, and Lower Precision in a Defined Order

Part 1 - Exercise Contract and Why It Matters

Implement an ordered recovery ladder: reduce the physical batch, add activation checkpointing, offload saved activations to CPU, and use a permitted lower-precision policy. Define whether stages accumulate changes or replace the previous configuration; use cumulative stages as the base experiment. Deliver the ladder trace, eligibility decisions, quality checks, and a terminal no-fit result.

Each stage begins from a restored logical transaction and a fresh graph. Keep sequence content and logical batch fixed. Precision changes require an explicit numerical acceptance contract; they must not silently replace an exactness requirement. The ladder is a predefined policy, not yet a claim of optimal cost.

Part 2 - System and Memory Mental Model

The actions target different memory components. Smaller micro-batches reduce the active input/activation working set. Checkpointing trades saved activations for recomputation. Saved-activation CPU offload trades GPU storage for host storage and transfers. Autocast can reduce selected intermediate dtypes without halving all model and optimizer state. Combined savings are not generally additive because actions may target the same tensors.

Define CPU offload narrowly: offload tensors saved for backward, using a supported saved-tensor mechanism such as `save_on_cpu`. Do not move the entire model to CPU and assume optimizer bindings, device placement, and throughput remain unchanged. Pinned host memory, transfer ordering, source lifetime, and CPU RAM limits belong to the offload contract.

Changing a checkpoint or offload policy affects graph construction, so activate it before the replay forward. Changing precision requires compatible autocast/scaler handling and new calibration. Some transitions temporarily require both old and new allocations; admission must cover transition peak, not only steady-state memory.

Part 3 - Quantitative Prediction Before Execution

For cumulative stage k, define configuration c_k and measure

$$\Delta M_k = M_{\text{peak}}(c_{k-1}) - M_{\text{peak}}(c_k).$$

Do not estimate the total by adding isolated savings from incompatible baselines. A valid stage must satisfy GPU cap, host-memory cap, numerical tolerance, and replay invariants.

For a hypothetical $B = 64$, compare smaller micro-batches with a checkpointed/offloaded version at the same logical batch. Suppose offloaded payload is H bytes and effective transfer bandwidth is v bytes/s. The transfer-only estimate for one outward and one inward copy is $2H/v$, before overlap, allocation, and contention effects. Calculate this for $H = 512\,\text{MiB}$ and a measured $v = 10\,\text{GiB/s}$.

Predict why a lower precision stage can be rejected despite fitting memory, and why adding offload after checkpointing may save less than offload alone.

Part 4 - Experimental Design and Instrumentation

1. Define each ladder stage, its prerequisites, maximum attempts, expected targeted storage, and permitted numerical error.

2. Calibrate stages on small inputs. Check checkpoint API settings, offload host capacity, dtype support, and scaler lifecycle.

3. Begin with a controlled OOM. Restore the committed transaction, apply the next eligible stage, and replay the entire logical window.

4. Measure transition peak, steady peak, host-memory demand, transfer cost, recomputation, accepted throughput, and numerical error.

5. Test unavailable BF16, insufficient host RAM, singleton pressure, and stages that save no measurable GPU memory.

6. Verify final no-fit after all eligible stages fail. Preserve a usable CPU checkpoint and a complete action history.

Use simple stateless blocks for initial checkpoint/offload equivalence. Keep asynchronous-transfer optimisation as a separately validated extension.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch
from torch.utils.checkpoint import checkpoint
from torch.autograd.graph import save_on_cpu

@dataclass
class RecoveryStage:
    name: str
    maximum_micro_batch: object
    checkpoint_plan: object
    saved_tensor_offload: bool
    precision_policy: object
    numerical_tolerance: object

def validate_stage_prerequisites(stage, runtime, resource_snapshot):
    eligibility = ...
    return eligibility

def configure_replay_graph(runtime, stage):
    # Apply graph-construction policies before forward replay.
    configuration = ...
    return configuration

def estimate_transition_peak(current_stage, next_stage, profile):
    estimate = ...
    return estimate

def execute_ladder_stage(runtime, checkpoint_state, window, stage):
    result = ...
    return result

def run_ordered_recovery_ladder(runtime, window, stages, limits):
    # Restore the committed transaction between failed stages.
    outcome = ...
    return outcome

def E13(runtime, window, stages, limits):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Check stage order, cumulative configuration identity, bounded attempts, and a recorded reason for every skipped stage. Verify that failure at one stage cannot leave modified parameters or scaler state in the next.

Compare accepted gradients, parameters, and buffers under the declared numerical criteria. Check CPU memory and transfer lifetime when offload is active. Reject claims of additive savings unless the experiment actually demonstrates them. A precision stage that violates quality limits is ineligible even if memory fits.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

A fixed ladder makes recovery predictable and auditable. Its order encodes preferences about recomputation, transfer, and numerical change, but those preferences may not minimise time on every GPU.

Explain which stage solved each incident and why. Use these measurements to motivate cost-aware selection rather than making the ladder more complex without evidence.

19.25.15 Exercise 14

Exercise Title: Rank Recovery Actions by Estimated Memory Saving and Runtime Penalty and Choose the Cheapest Strategy That Can Satisfy the Missing VRAM

Part 1 - Exercise Contract and Why It Matters

Build a cost-aware selector over a finite set of validated recovery configurations. Rank actions or compatible combinations by expected time to accepted work while satisfying GPU, host-memory, and numerical constraints. Compare the selector with E13's fixed ladder.

Deliver per-action profiles, uncertainty estimates, predicted and observed choices, and the regret relative to the best measured candidate under the same incident. Treat the missing-VRAM estimate as uncertain; the failed request size is not automatically the amount an action must save.

Part 2 - System and Memory Mental Model

The exception may reveal one requested allocation, but that request occurred at a particular live baseline after possible internal retries. Global free memory and cached blocks have different roles. Additional future workspaces may still be needed. Therefore subtracting free bytes from the message's request size does not provide a universal deficit.

Use a calibrated upper-demand model or bounded candidate probes to estimate feasibility. An action's saving must be measured against the current configuration and shape. Checkpointing plus offload or lower precision may have overlapping effects, so profile combinations rather than blindly summing individual reductions.

Cost includes activation of a policy, state restoration, graph rebuilding, recomputation, transfer, and future update time. A costly transition can pay off over a long budget regime but lose for one short incident. Use only information available at decision time; future regime duration is an assumption or estimate, not privileged knowledge.

Part 3 - Quantitative Prediction Before Execution

For predicted upper peak $\widehat{M}_{\text{upper}}$ and usable cap $J - S$, define an estimated deficit

$$\widehat{D} = \max(0, \widehat{M}_{\text{upper}} + S - J).$$

This is a model-derived quantity, not a direct reading from OOM text. For candidate a over a horizon of H accepted updates, compare

$$C(a) = t_{\text{restore}}(a) + t_{\text{transition}}(a) + H\, t_{\text{update}}(a) + \widehat{C}_{\text{failure}}(a).$$

Require upper predicted peak and transition peak to fit, plus quality and host-resource constraints.

Create a hypothetical table where micro-batch reduction saves 400 MiB at 20 ms/update, checkpointing saves 900 MiB at 35 ms/update, and offload saves 700 MiB at 28 ms/update, with distinct transition costs. For a 600 MiB estimated deficit, rank eligible actions at horizons one and 100. Supply transition costs before ranking and explain how uncertainty can change eligibility.

Part 4 - Experimental Design and Instrumentation

1. Build separate profiles for each action and selected combination, including transition demand and full-update cost.
2. Split incident traces into calibration and held-out evaluation. Freeze the selector before seeing held-out outcomes.
3. For each incident, record the demand estimate, uncertainty, eligible candidates, chosen action, and predicted cost.
4. Replay from identical snapshots under the fixed ladder and the cost-aware selector. Include failed probes and restoration time.
5. Measure the best candidate offline on the same incident as a comparison reference; do not give its future result to the online selector.
6. Test an action with high mean saving but variable peak and an incident outside the profile's shape domain.

If no candidate is credibly feasible, defer or run a bounded conservative probe. Do not always select the cheapest action regardless of uncertainty.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class RecoveryActionProfile:
    action_id: str
    predicted_peak_bytes: int
    uncertainty_bytes: int
    transition_peak_bytes: int
    transition_seconds: float
    update_seconds: float
    host_bytes: int
    numerical_error: float

def estimate_incident_demand(incident, workload_profile):
    estimate = ...
    return estimate

def filter_feasible_actions(profiles, resources, quality_limits):
    candidates = ...
    return candidates

def estimate_action_cost(profile, horizon, failure_model):
    cost = ...
    return cost

def select_lowest_cost_recovery(candidates, horizon, policy):
    decision = ...
    return decision

def compare_with_measured_best(decisions, measured_candidates):
    regret_report = ...
    return regret_report

def E14(runtime, incidents, action_profiles, policy):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate unit consistency and compare predicted memory saving with measured complete-update peaks. Check that transition peak, host RAM, and numerical limits are included in eligibility.

Report unsafe underprediction, unnecessary conservatism, action ranking errors, and end-to-end time regret. Do not count a fast failed action as a low-cost success. Test no feasible candidate, stale profile, and nonadditive action combinations. State how small cost differences are treated when measurement noise exceeds the apparent benefit.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved

bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

The cheapest recovery is the one that restores accepted progress at lowest total cost within constraints. It may differ from both the action with greatest memory saving and the one with fastest isolated kernel time.

Explain when profile-based selection outperforms a simple ladder and when uncertainty makes the simpler policy preferable. Preserve the finite candidate-set limit of any optimality claim.

19.25.16 Exercise 15

Exercise Title: Detect Repeated OOM-Recovery Oscillation and Add Hysteresis So the Runtime Does Not Immediately Return to an Unsafe Configuration

Part 1 - Exercise Contract and Why It Matters

Prevent a runtime from repeatedly returning to a configuration that just failed. Add failure memory, separate grow/shrink thresholds, growth patience, cooldown, and explicit conditions for revalidating a previously failed configuration. Deliver a policy state table and traces comparing an eager-growth baseline with the revised controller.

Key failure memory by relevant workload and resource conditions: batch, padded shape, precision, checkpoint/offload policy, backend, and budget regime. A failure for one long shape should not prohibit all short inputs forever, but it must not be forgotten after a single successful small update.

Part 2 - System and Memory Mental Model

An OOM can cause batch reduction, which then creates headroom. A naive controller interprets that headroom as permission to restore the same unsafe batch, generating a repeating failure cycle. Switching policies can also incur repeated graph rebuilding, transfers, and checkpoint costs.

Use a deadband between ordinary growth and shrink triggers, require several accepted stable updates before growth, and retain the evidence of recent failure. Cooldown should delay speculative growth, not block emergency safety actions. Raw admission failures must override a smoothed utilisation estimate.

Failure memory needs an expiry or revalidation rule. A larger stable budget, a revised shape profile, or a validated new configuration can justify another probe. Time passage alone may be weak evidence. Count accepted logical updates, not failed attempts or successful micro-steps, toward growth patience.

Part 3 - Quantitative Prediction Before Execution

Let b_t be physical batch and u_t observed utilisation under a declared cap. Choose $l < h$ for growth and shrink triggers. Define switching count and repeated-incident rate:

$$N_{\text{switch}} = \sum_{t=2}^{n} \mathbf{1}_{c_t \neq c_{t-1}}, \qquad R_{\text{repeat}} = \frac{N_{\text{repeated equivalent OOM}}}{N_{\text{attempts}}}.$$

Configuration c_t includes more than batch size. Define equivalence of incidents by a documented fingerprint.

Manually replay a workload where batch eight fits at utilisation 0.55 and batch twelve OOMs at the same budget. Compare immediate regrowth with patience four and cooldown six accepted updates. Then introduce a real budget increase and predict when a bounded revalidation should be permitted. Explain why wider hysteresis alone cannot make an infeasible configuration fit.

Part 4 - Experimental Design and Instrumentation

1. Implement pure CPU transitions for failure memory, cooldown, patience, and configuration selection.
2. Replay a constant-budget failure cycle, alternating long/short shapes, a brief budget increase, and a sustained budget increase.
3. Add emergency pressure during cooldown and verify immediate shrink or deferral remains possible.
4. Compare controller variants with the same admission model, action set, and retry budget.
5. Run a reproducible CUDA trace and measure repeated OOMs, switches, lost work, time to regain useful capacity, and net throughput.

Tune on one trace and retain a separate evaluation trace. Log every decision that reopens a previously failed configuration, including its new evidence.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class FailureMemoryEntry:
    configuration_key: str
    shape_key: str
    budget_regime: str
    last_failure_update: int
    revalidation_required: bool
```

```
@dataclass
class RecoveryControllerState:
    current_configuration: object
    accepted_stable_updates: int
    cooldown_remaining: int
    failure_memory: object

def record_failed_configuration(state, incident):
    next_state = ...
    return next_state

def determine_revalidation_eligibility(entry, observation, policy):
    verdict = ...
    return verdict

def transition_recovery_controller(state, observation, policy):
    # Emergency safety must override speculative-growth cooldown.
    next_state = ...
    decision = ...
    return next_state, decision

def measure_oscillation_and_lost_work(trace):
    metrics = ...
    return metrics

def E15(runtime_factory, traces, policies):
    comparison = ...
    return comparison
```

Part 6 - Measurement, Validation, and Failure Analysis

Test threshold equality, cooldown off-by-one errors, stale patience surviving OOM, and growth counters advanced by failed attempts. Verify a failed configuration cannot be selected without its declared revalidation condition.

Demonstrate fewer avoidable reversals while reporting any delayed capacity recovery. Legitimate switches caused by changing shapes or budgets are not automatically oscillation bugs. Check that failure memory remains bounded and does not permanently exclude newly feasible configurations.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Recovery policies need memory of failure as well as measurements of current headroom. Explain the causal loop that produced repeated OOM and how the new state breaks it.

Recommend a stability policy with explicit revalidation rules. Avoid claiming universal stability: discrete configuration changes, unseen shapes, and rapidly changing external pressure can still require frequent legitimate adaptation.

19.25.17 Exercise 16

Exercise Title: Capture memory_stats() and a CUDA Memory Snapshot Automatically When Recovery Attempts Repeatedly Fail

Part 1 - Exercise Contract and Why It Matters

Build a bounded diagnostic escalation policy for repeated allocation OOM. Capture allocator statistics, configuration, and a memory snapshot when the failure threshold is reached. Deliver the trigger trace, snapshot metadata, capture overhead, and a fallback record for diagnostic failure.

Use lightweight scalar records continuously and an explicitly bounded history policy when pre-failure allocation history is needed. Enabling history only after an incident cannot reconstruct earlier allocation events. State whether a snapshot represents current allocator state, recorded history, or both.

Part 2 - System and Memory Mental Model

Allocator statistics include current and peak quantities and accumulated event counters. Examples can include allocation retries, OOM count, and inactive split bytes, but availability and interpretation depend on the installed version and allocator backend. Native allocator metrics are not all meaningful under `cudaMallocAsync`.

Snapshots describe memory visible to the PyTorch allocator. Other CUDA libraries, communication buffers, and external processes may consume memory absent from that snapshot. Compare scope with global observations without claiming an exact attribution from their difference.

History and snapshot interfaces can include private APIs such as `_record_memory_history` or a snapshot-dump helper. Check the installed interface and version, isolate it behind an adapter, and record unsupported status rather than crashing the recovery engine. History storage consumes host memory; bound event count and escalation frequency. A ring buffer may overwrite old evidence.

Capture failures must remain secondary. An allocation-constrained or context-fatal runtime may not support further diagnostics. Preserve existing CPU evidence, cap output volume, and avoid recursive escalation when the diagnostic path itself fails.

Part 3 - Quantitative Prediction Before Execution

Let o_t be an accumulated allocator OOM counter. Compute the delta $o_t - o_{\text{baseline}}$ only when counter scope and reset history match. Peak-stat resets and accumulated-stat resets have different meanings; never assume resetting one resets the other.

If each stored history event costs an observed average of e host bytes and the limit is N, predict

$$H_{\text{history}} \approx Ne$$

plus indexing, stack, and serialization overhead. For a hypothetical $N = 20{,}000$ and $e = 1\,\text{KiB}$, calculate the baseline estimate and propose a host-memory cap with measured uncertainty.

Design a trigger such as three OOMs within one logical window, with at most one expensive snapshot per incident. Predict trigger behaviour for failure sequences separated by success, configuration changes, or a new incident ID. Define whether these reset the threshold.

Part 4 - Experimental Design and Instrumentation

1. Inspect supported statistics and snapshot interfaces before the experiment. Record PyTorch version, allocator backend, and adapter status.

2. Enable bounded history before a controlled workload when earlier events are required; otherwise label the snapshot as current-state only.

3. Generate repeated failures with a configured threshold and capture statistics before cleanup when feasible, then a separate post-cleanup record. Preserve their ordering.

4. Serialize the snapshot to a local diagnostic artifact with metadata, size, checksum, and incident ID. Keep recovery reports CPU-only.

5. Inject missing API, file-write failure, and history-limit cases. Verify no diagnostic error replaces the original OOM or triggers recursion.

6. Compare runtime and host-memory overhead with diagnostics disabled, lightweight-only, and escalated modes.

Match any recorded OOM event to the incident using device, phase, and time evidence; do not assume the last event necessarily belongs to it.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every `...` marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class DiagnosticPolicy:
    repeated_oom_threshold: int
    maximum_history_entries: int
    maximum_snapshot_bytes: int
    maximum_captures_per_incident: int

def inspect_diagnostic_capabilities(torch_module, device):
    capabilities = ...
    return capabilities
```

```
def configure_bounded_memory_history(capabilities, policy):
    # Adapt to the installed version; no assumed private signature.
    history_handle = ...
    return history_handle

def should_escalate_diagnostics(incident_history, policy):
    decision = ...
    return decision

def capture_allocator_evidence(device, capabilities, phase):
    evidence = ...
    return evidence

def persist_snapshot_best_effort(evidence, incident_id, limits):
    artifact_record = ...
    return artifact_record

def E16(runtime, failure_schedule, diagnostic_policy):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify trigger count, one-capture limits, counter scope, and reset behaviour. Distinguish genuine allocator OOM counts from synthetic exceptions that may not increment allocator counters.

Check snapshot artifact integrity, version metadata, event truncation, and the timestamp relative to cleanup. Mark absent or inapplicable metrics explicitly. Demonstrate that a failure before history was enabled lacks retrospective trace evidence.

Require the original incident to remain reportable if snapshot capture or writing fails. Report measured diagnostic time and host-memory cost; do not hide them in the recovery benchmark.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Diagnostic escalation spends resources to reduce uncertainty. Explain which questions a snapshot answered that scalar counters could not, and which remained outside allocator visibility.

Recommend a bounded history and capture policy based on incident frequency, memory cost, and usefulness. More recorded data is not automatically more reliable evidence if its scope and timing are unclear.

19.25.18 Exercise 17

Exercise Title: Build an OOM Incident Report That Identifies the Failed Allocation Size, Peak VRAM, Active Memory, Reserved Memory, and Current Runtime Policy

Part 1 - Exercise Contract and Why It Matters

Create a machine-readable incident schema and a human-readable report for each OOM episode. Include the failed request size when known, observed memory peaks, active/live memory fields, reserved memory, runtime configuration, attempts, recovery actions, and final outcome. Deliver a sample report generated from an actual controlled incident and tests for incomplete evidence.

Use explicit provenance for each value: application-known request, message-derived value, snapshot event, allocator counter, or unavailable. The report must not invent a failed allocation size when the source does not expose one.

Part 2 - System and Memory Mental Model

One incident can contain several failed attempts followed by success or no-fit termination. Keep incident IDs, attempt IDs, logical update IDs, and action IDs distinct. The first failure describes the initiating problem; later failures describe how proposed fixes performed.

Active/live memory is ambiguous unless defined. Report allocated tensor bytes and, when supported, the allocator's active-byte statistic as separate fields with their API scope. Do not label both as an interchangeable single number. Peak allocated bytes describe successful allocations during the declared interval, not the attempted missing storage.

Message parsing is optional enrichment. Preserve the raw message and version, and tolerate wording, units, and missing fields. A snapshot OOM event may provide a more direct request size, but must be correlated to the correct attempt. Use null with a reason rather than zero for unknown quantities.

Include physical device identity, process-local index, allocator, precision, physical and logical batch, padded shape, checkpoint/offload policy, budget, margin, and profile version. These determine whether two incidents represent the same configuration.

Part 3 - Quantitative Prediction Before Execution

Store all byte values canonically while preserving original displayed units. Use $1\,\mathrm{MiB} = 2^{20}$ and $1\,\mathrm{GiB} = 2^{30}$; do not silently interpret an ambiguous unit string as exact bytes.

For a request reported as 768 MiB, a known application payload of 750 MiB, and an allocator snapshot showing a rounded request, retain each observation with its provenance. Explain why allocator overhead or rounding may differ from payload.

$$t_{\text{incident}} = t_{\text{final outcome}} - t_{\text{first failure}}, \qquad N_{\text{wasted}} = N_{\text{speculatively processed}} - N_{\text{accepted}}.$$

Define speculative work carefully when the same sample is replayed. Report accepted unique samples or valid tokens separately.

Before implementation, fill an example schema by hand with one unknown request size and one missing exception-time memory reading.

Part 4 - Experimental Design and Instrumentation

1. Define typed fields, units, required metadata, optional evidence, and schema version. Separate observations from causal interpretation.

2. Build the report from immutable CPU event records. Preserve the original message and formatted traceback without frame objects.

3. Add bounded message parsing with tests for known supported formats, missing fields, ambiguous units, and changed wording.

4. Correlate a snapshot event when available and retain disagreements rather than overwriting earlier evidence silently.

5. Render a compact summary with initial failure, attempted actions, accepted outcome, lost time, and unresolved uncertainty.

6. Test report generation after both successful recovery and terminal no-fit, including unavailable diagnostic artifacts.

Do not embed training tensor contents in an incident report. Configuration, shape, state identifiers, and scalar evidence should be enough to link to a controlled reproducer.
Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import json
import torch

@dataclass
class EvidenceValue:
    value: object
    unit: str
    source: str
    timestamp: object
    unavailable_reason: object

@dataclass
class IncidentIdentity:
    incident_id: str
    logical_update_id: str
    device_identity: str
    schema_version: str
```

```
def extract_optional_request_size(raw_message, version):
    # Preserve unknown or ambiguous values with a reason.
    request_evidence = ...
    return request_evidence

def correlate_snapshot_event(incident, snapshot_metadata):
    correlation = ...
    return correlation

def assemble_incident_report(identity, events, artifacts):
    report = ...
    return report

def validate_report_schema(report):
    validation = ...
    return validation

def render_human_incident_summary(report):
    summary_text = ...
    return summary_text

def E17(incident_events, diagnostic_artifacts):
    report = ...
    summary = ...
    return report, summary
```

Part 6 - Measurement, Validation, and Failure Analysis

Require schema validation, correct units, immutable attempt ordering, and explicit unavailable values. Check that the report never substitutes a post-cleanup reading for a missing pre-failure reading.

Compare derived counts and durations with the raw event log. Test multiple attempts, conflicting request-size evidence, missing snapshots, and a final no-fit outcome. A diagnosis must cite its observations and permit uncertainty; message parsing alone cannot establish a unique cause such as fragmentation.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

An incident report is an evidence trail for a future engineer. Explain whether that engineer can reproduce the configuration, understand why each action was chosen, and identify what remains unknown without replaying the original conversation.

Prefer a small trustworthy schema over a large report full of inferred numbers presented as measurements. The report should support comparison across versions without hiding differences in measurement scope.

19.25.19 Exercise 18

Exercise Title: Stress-Test the Recovery Engine with Random Batch Sizes, Variable Sequence Lengths, Artificial VRAM Pressure, and Deliberate Memory Leaks

Part 1 - Exercise Contract and Why It Matters

Build a seeded stress suite for the integrated recovery engine. Vary batch sizes and sequence lengths, apply a controlled external pressure worker, create bounded intentional reference leaks, and inject failures at distinct transaction phases. Deliver scenario manifests, expected invariants, traces, and a minimal reproducer for each failure.

Use deterministic CPU control tests alongside real CUDA integration tests. Synthetic exceptions validate control flow but do not prove allocator behaviour. Real OOM tests validate one hardware/runtime environment but cannot exhaust every state transition.

Part 2 - System and Memory Mental Model

Randomness is useful only when failures remain reproducible. Record the generator seed and realised event schedule. Use logical update and phase indices for coordinated tests; wall-clock timing can introduce uncontrolled variation. Keep asynchronous pressure as a separately labelled stress mode.

An intentional leak is experiment-owned retention with a strict size and iteration limit. The engine should diagnose growing live baseline and identify unresolved ownership, not delete arbitrary tensors from unknown owners. If the owner cannot release memory, correct behaviour may be deferral or termination with preserved state.

Artificial pressure comes from a worker created by the test on the same physical GPU. Use acknowledgements, byte limits, and cleanup. Never terminate unrelated processes. Deliberate context corruption is unnecessary for this suite; test fatal-error routing synthetically. Verify accepted-work invariants independently of whether recovery eventually succeeds.

Part 3 - Quantitative Prediction Before Execution

Let a realised scenario be

$$\mathcal{S} = \{(u_i, p_i, e_i, m_i)\}_{i=1}^{n},$$

where u_i is logical update, p_i phase, e_i event type, and m_i magnitude or shape. Persist this schedule for replay. Define

$$R_{\text{recovery}} = \frac{N_{\text{recovered recoverable incidents}}}{N_{\text{declared recoverable incidents}}}, \qquad A_{\text{attempt}} = \frac{N_{\text{attempts}}}{N_{\text{accepted updates}}}.$$

Report undefined ratios when their denominator is zero and explain the outcome. Do not remove terminal failures from the dataset.

Before running, predict response to an oversized singleton, a 256 MiB temporary pressure pulse, a leak accumulating 32 MiB/update, and partial optimizer mutation. Specify which should retry, change strategy, diagnose, defer, or terminate. Derive the maximum retained leak under the chosen bound.

Part 4 - Experimental Design and Instrumentation

1. Write at least ten scenarios including no-pressure control, forward OOM, backward OOM, partial optimizer failure, long singleton, temporary pressure, persistent pressure, bounded leak, repeated oscillation, and diagnostic-write failure.

2. Generate bounded random shape schedules with saved seeds and retain a held-out distribution for evaluation.

3. Run synthetic controller tests first, then representative genuine CUDA scenarios. Keep evidence labels distinct.

4. Audit sample IDs, loss weights, optimizer/scaler/scheduler progress, rollback state, retry bounds, and process continuity.

5. Release test-owned pressure and leak containers in cleanup. Record live and reserved memory after recovery, separately.

6. Reduce any failing schedule to the smallest retained event set that reproduces it, without deleting the original trace.

Measure host-memory and diagnostic overhead as well as GPU memory. A recovery engine can fail through unbounded CPU snapshots even when its CUDA allocations are controlled.
Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import random
import multiprocessing as mp
import torch

@dataclass
class StressEvent:
    logical_update: int
    phase: str
    event_kind: str
    magnitude: object
    evidence_mode: str

@dataclass
```

```
class StressLimits:
    maximum_live_pressure_bytes: int
    maximum_leak_bytes: int
    maximum_attempts: int
    maximum_wall_seconds: float

def generate_seeded_schedule(seed, shape_limits, event_limits):
    schedule = ...
    return schedule

def execute_controlled_pressure_worker(channels, device, limits):
    ...

def apply_bounded_reference_leak(owner, event, limits):
    observation = ...
    return observation

def run_stress_scenario(runtime_factory, schedule, limits):
    result = ...
    return result

def audit_stress_invariants(reference_manifest, result):
    audit = ...
    return audit

def minimise_failure_schedule(schedule, replay_function):
    minimal_schedule = ...
    return minimal_schedule

def E18(runtime_factory, seeds, scenario_limits):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Require exact accepted sample accounting, finite accepted gradients, correct logical weights, complete rollback, bounded attempts, and cleanup of experiment-owned resources. Verify that a recovered parameter update matches the compatible clean reference.

Separate recoverable allocation incidents, valid no-fit refusals, unrelated exceptions, and diagnostic failures. Test that a growing leak is not mistaken for harmless reserved cache. Report PID continuity for successful in-process recovery and preserve every terminal outcome. Repeated success in a bounded suite is evidence of that tested envelope, not universal immunity to OOM.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Stress testing asks whether recovery remains correct under combinations of disturbances, not merely whether an exception can be caught. Explain which invariant failed first in each unsuccessful scenario.

Prioritise improvements by lost accepted work and correctness impact. Preserve difficult cases and negative results instead of narrowing the test distribution until the engine appears reliable.

19.25.20 Exercise 19

Exercise Title: Build and Benchmark a Self-Recovering PyTorch Training Runtime That Survives CUDA OOM Events and Dynamically Reconfigures Memory Strategy Without Restarting

Part 1 - Exercise Contract and Why It Matters

Integrate narrow OOM detection, bounded diagnostics, ownership cleanup, transactional rollback, shape-aware splitting, exact accumulation, recovery-action selection, hysteresis, and incident reporting into one single-GPU training runtime. It must survive declared recoverable allocation OOMs in the same process and preserve accepted training semantics while changing execution policy.

Deliver the learner-completed runtime, configuration schema, incident reports, raw traces, and a benchmark against simpler baselines. Define terminal outcomes for singleton no-fit, exhausted resources, unrelated errors, and context-fatal conditions. The chapter title's without-restarting claim applies to the recoverable allocation envelope, not every possible CUDA error.

Part 2 - System and Memory Mental Model

Use explicit boundaries between observation, policy selection, admission, attempt execution, rollback, and commit. The executor owns all mutable state covered by E07. The policy sees CPU-only observations and proposes a configuration. The incident recorder captures evidence without retaining failed graphs.

The runtime begins a logical window from a committed checkpoint, materialises its CPU data, validates an execution plan, and attempts the update. On allocation OOM, it records bounded evidence, unwinds the failed scope, restores state, and chooses a permitted alternative. On success, it validates and commits exactly once. On no-fit or exhaustion, it preserves a usable checkpoint and reports the reason.

Restoration must include transient execution lifecycles where persistent state dictionaries are insufficient, such as a scaler already unscaled within a failed attempt. Model buffers, parameter bindings, prepared inputs, RNG, scheduler counters, and custom supported hooks also matter. Do not retain both failed GPU graphs and a newly constructed replay.

Arbitrary asynchronous external allocation cannot be prevented by a preflight free-memory check. Admit conservatively and measure recovery under such pressure separately from coordinated tests. Distributed runtimes require cross-rank agreement and collective-failure handling; they are outside this single-device implementation.

Part 3 - Quantitative Prediction Before Execution

For accepted useful work N_{acc}, measure

$$Q_{\text{net}} = \frac{N_{\text{acc}}}{t_{\text{normal}} + t_{\text{failed}} + t_{\text{snapshot}} + t_{\text{restore}} + t_{\text{diagnostic}} + t_{\text{deferred}}}.$$

Define nonoverlapping time categories so elapsed time is not double-counted. Also report the direct full wall-clock denominator. Choose a finite candidate configuration c subject to

$$\begin{aligned} \widehat{M}_{\text{upper}}(c,x) + S_J &\le J, \\ \widehat{\Delta M}_{\text{upper}}(c,x) + S_D &\le F, \\ H_{\text{CPU}}(c) &\le H_{\max}, \\ E_{\text{numerical}}(c) &\le \varepsilon_{\max}. \end{aligned}$$

Use an explicitly conservative cache-credit policy for incremental device admission. Enforce exact logical membership and weighting as correctness conditions, not optional performance metrics.

Before benchmarking, predict incident count, recovery latency, snapshot memory, accepted throughput, and the winning strategy for three workloads: stable shape, variable length, and external pressure. Estimate how often recovery must occur before a conservative static batch becomes faster than a more aggressive adaptive policy.

Part 4 - Experimental Design and Instrumentation

1. Implement independent interfaces for telemetry, snapshots, attempt execution, admission, recovery policy, and incident reporting. Validate pure CPU state transitions before CUDA integration.

2. Establish a small deterministic numerical reference. Check full-state restoration after forward, backward, and partial optimizer failure, then verify a clean accepted replay.

3. Compare conservative static batching, fail-fast execution, simple batch-halving recovery, and the integrated runtime. Keep logical work, eligible precision policies, and data order comparable.

4. Run steady, variable-length, coordinated pressure, and asynchronous pressure traces. Apply E18's held-out stress cases.

5. Include cold initialization, lazy optimizer state, calibration, snapshotting, failed attempts, action transitions, diagnostics, and deferral in the end-to-end benchmark.

6. Export memory peaks, accepted samples or valid tokens, accepted updates, median/p95 incident latency, retry amplification, action switches, numerical errors, host-memory use, and terminal outcomes.

7. Require same-process continuity for successful recoverable incidents. Report unsupported fatal-context outcomes separately.

Do not hide fail-fast baseline termination by comparing only its short pre-failure runtime with a completed recovery run. Use a common work target or time horizon and report completion status alongside speed.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer

update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class ExecutionPolicy:
    micro_batch_limit: int
    padded_token_limit: object
    checkpoint_plan: object
    saved_tensor_offload: bool
    precision_policy: object
    profile_key: str

@dataclass
class RuntimeLimits:
    gpu_budget_bytes: int
    gpu_margin_bytes: int
    host_budget_bytes: int
    maximum_retries: int
    maximum_incident_seconds: float

@dataclass
class LogicalCommit:
    update_id: str
    sample_ids: tuple
    normalization_count: int
    attempt_count: int
    accepted: bool

class SelfRecoveringTrainingRuntime:
    def __init__(self, workload, profiles, recovery_policy, limits):
        self.workload = workload
        self.profiles = profiles
        self.recovery_policy = recovery_policy
        self.limits = limits
        self.controller_state = ...
        self.commit_ledger = ...
        self.incident_recorder = ...

    def prepare_logical_window(self, cpu_data_source):
```

```
        window = ...
        return window

    def snapshot_committed_state(self, window):
        checkpoint_state = ...
        return checkpoint_state

    def select_and_admit_policy(self, window, incident_history):
        decision = ...
        return decision

    def execute_attempt_in_scope(self, window, execution_policy):
        # Implement weighted accumulation and narrow error routing.
        outcome = ...
        return outcome

    def rollback_failed_attempt(self, checkpoint_state, outcome):
        # Restore state and execution lifecycles before replay.
        ...

    def handle_incident(self, window, checkpoint_state, outcome):
        # Bound diagnosis, retries, and action escalation.
        recovery_decision = ...
        return recovery_decision

    def validate_and_commit(self, window, outcome):
        # Advance accepted state once, only after validation.
        commit = ...
        return commit

    def run_logical_window(self, cpu_data_source):
        # Implement success, recoverable failure, and terminal paths.
        result = ...
        return result

    def export_evidence(self, destination):
        ...

def benchmark_recovery_runtimes(factories, traces, work_target):
    comparison = ...
    return comparison

def E19(workload, profiles, recovery_policy, limits, traces):
    runtime = ...
    benchmark_report = ...
    return benchmark_report
```

Part 6 - Measurement, Validation, and Failure Analysis

Acceptance requires exact logical sample accounting, correct loss normalisation, finite accepted updates, coherent scaler and scheduler progress, bounded retries, and complete state restoration. Compare accepted gradients and parameter updates with a compatible clean reference using declared tolerances. Audit mutable buffers and nested optimizer state, not only model outputs.

Inject failures at transfer, forward, backward, and after partial optimizer mutation. Verify no partial update survives rollback and that cleanup does not delete required model state. Validate terminal no-fit, exhausted host memory, unsupported precision, diagnostic failure, and unrelated-error propagation.

Report all incidents and all overhead. Failed or replayed samples are speculative work, not additional accepted progress. Separate application-margin violations from physical CUDA OOM, and distinguish snapshot scope from total device use. If the integrated runtime loses to a strong static baseline, retain that result and explain why.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

A self-recovering runtime is a transaction engine under resource constraints. Its success is accepted training progress with preserved semantics and a clear evidence trail, not the absence of uncaught exceptions alone.

Conclude with the operating envelope actually demonstrated: device, allocator, shapes, objectives, precision policies, pressure bounds, and recovery limits. Explain which errors are handled in process, which result in a checkpointed stop, and which require a fresh CUDA process. Recommend the simplest measured policy that satisfies those requirements.

19.26 DMM24

Chapter Title: Memory-Aware Scheduling
Specialization: Dynamic Memory Management and CUDA Systems

19.26.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Memory-Aware Scheduling Across Compute, Transfer, Placement, Prefetch, Eviction, and Recovery Decisions

Part 1 - Exercise Contract and Why It Matters

Construct a scheduler model that coordinates the lifetime and movement of data as well as the order of computation. Identify tasks for forward, backward, input transfer, parameter prefetch, activation offload, eviction, and recovery. Deliver a dependency graph, a storage-ownership ledger, and a timeline showing when each resource is required and released.

Use one GPU with eager PyTorch and explicit coarse tasks. The scheduler chooses when supported calls are submitted; it does not preempt arbitrary CUDA kernels or replace autograd's internal scheduling. Begin with immutable inference weights, then add a training case with resident parameters and a clearly defined logical optimizer-update boundary.

Part 2 - System and Memory Mental Model

A task can be dependency-ready but memory-inadmissible, or fit memory while its inputs are still in transfer. Submission, device execution, completion, and storage release are distinct events. A completed forward may leave outputs and saved activations alive for backward. Eviction can require a host allocation and copy before any GPU storage is releasable.

Placement records must identify storage, location, validity, version, owners, pending readers, and transfer completion. A CPU copy is a useful backing store only if it contains the version required by future work. Deleting a Python name does not release storage still owned by a view, autograd graph, optimizer, or another stream.

Separate allocator reservations from application reservations. Allocator-reserved bytes are real memory held by PyTorch; a scheduler reservation is a promise for admitted future work. They cannot be subtracted or added indiscriminately. Likewise, stream ordering protects data dependencies while allocator lifetime tracking protects reuse; neither alone supplies the other.

Recovery is a control transition that stops dependent admission, settles or safely abandons managed work, and restores the supported transaction. It cannot make a fatal CUDA context healthy or guarantee availability against an arbitrary external allocator.

Part 3 - Quantitative Prediction Before Execution

For a set of live storage objects $\mathcal{L}(t)$, count unique storage:

$$M_{\text{live}}(t) = \sum_{s \in \mathcal{L}(t)} \text{bytes}(s).$$

Shared aliases must not appear as independent payloads. Let A_t be materialised live bytes and U_t outstanding commitments for additional bytes not yet materialised. A conservative job policy is

$$A_t + U_t + S_J \leq J_t.$$

As promised storage materialises, reconcile its commitment into actual ownership without counting it twice. A pending eviction contributes no immediate negative demand.

For a toy $J = 8\,\text{GiB}$ and $S_J = 0.5\,\text{GiB}$, suppose $A = 4\,\text{GiB}$ and $U = 1\,\text{GiB}$. Predict admission of a task needing an additional $1.75\,\text{GiB}$. Then suppose a planned eviction promises $1\,\text{GiB}$ but its copy is incomplete: explain why that promise does not change the current answer. Draw the ownership transfer after the copy, last reader, and actual release complete.

Part 4 - Experimental Design and Instrumentation

1. Describe a small workload as a directed acyclic task graph. Include compute that depends on prefetched data and backward that retains forward-produced activations.

2. Annotate each task with required versions, incremental peak, persistent outputs, temporary storage, host demand, and stream.

3. Hand-simulate ready, reserved, submitted, completed, and retired states. Show why completion need not mean all bytes are freed.

4. Implement a serial CUDA reference and compare a proposed schedule against its outputs and, where relevant, gradients.

5. Add one delayed transfer and one bounded memory-budget reduction. Record whether the scheduler waits, changes placement, or defers work.

Use a CPU simulation to explore event order before introducing concurrent CUDA streams. Do not intentionally create a use-after-free on the GPU.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class TaskDescriptor:
    task_id: str
    task_kind: str
    dependency_ids: tuple
    required_storage_versions: tuple
    incremental_peak_bytes: int
    retained_output_bytes: int

@dataclass
class StorageOwnership:
    storage_id: str
    version: int
    location: str
    owner_ids: tuple
    last_use_events: tuple

def build_workload_graph(workload_specification):
    graph = ...
    return graph

def describe_storage_lifetimes(graph):
    ledger = ...
    return ledger

def simulate_admission_and_completion(graph, budgets, event_order):
    timeline = ...
    return timeline
```

```
def compare_with_serial_reference(reference, scheduled_result):
    audit = ...
    return audit

def E00(workload_specification, budgets, event_order):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify graph dependencies, unique-storage accounting, version validity, and release after the last required use. Show a task that fits bytes but must wait for data, and a ready task that must wait for capacity.

Compare serial and scheduled results under declared numerical tolerances. For training, preserve logical sample membership and optimizer update count. Reject timelines that free saved activations after forward completion or credit unfinished eviction as available memory. Label unknown external demand and unsupported execution modes.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each task or attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Memory-aware scheduling coordinates promises, data readiness, and lifetimes. Explain how this differs from selecting the next operation solely by duration or free-byte snapshots.

Conclude with the scheduler's ownership boundary: which calls, allocations, transfers, and state transitions it controls. Guarantees must be limited to that boundary and the validity of its resource estimates.

19.26.2 Exercise 01

Exercise Title: Define a Runtime Memory State Vector from Free VRAM, Reserved Memory, Active Memory, RAM Usage, Transfer Load, and Pending Work

Part 1 - Exercise Contract and Why It Matters

Design a typed runtime state vector for scheduler decisions. Include GPU live allocation, allocator reservations, global free memory, host-memory availability and usage, scheduler-owned pinned storage, transfer backlog, pending work, logical commitments, and observation age. Deliver a schema, collection adapter, and replayable state trace.

For every field, declare its unit, source, scope, update frequency, and unavailable-state representation. A compact vector is useful only if its entries retain their meaning. Do not combine unrelated measurements into a single unqualified memory-pressure percentage.

Part 2 - System and Memory Mental Model

Allocated tensor bytes and an allocator's active-byte statistic are not interchangeable names. Record them separately when available. Some allocator metrics are backend-specific, so absence

or inapplicability is different from zero. Global free memory reflects the whole device at one instant, while process counters describe a narrower ownership scope.

Process resident RAM and system available RAM answer different questions. Scheduler-owned pinned bytes are usually a subset of host allocations, not an independent amount to add to process RSS. Track managed buffers by ownership and report system observations separately. Avoid claiming exact machine-wide pinned usage from a process-local registry.

Transfer load includes queued and in-flight bytes, direction, measured bandwidth, and completion events. A queue length is not bandwidth, and a copy submission is not completed transfer. Pending work includes dependencies and resource commitments, not just the number of tasks.

The vector is a time-labelled observation, not an atomic GPU snapshot. Use an observation epoch and a ledger version; stale data must be recognised before admission.

Part 3 - Quantitative Prediction Before Execution

One possible state is

$$s_t = (A_t, R_t, F_t, X_t, H_t, P_t, Q_t^{\text{in}}, Q_t^{\text{out}}, U_t, N_t^{\text{ready}}, a_t),$$

where X is a supported active-byte metric, H is host available memory, P managed pinned bytes, Q transfer backlog, U unmaterialised commitments, and a observation age. Keep device identity and validity metadata alongside the vector.

Define job headroom $h_J = J - S_J - A - U$. A conservative device check gives no cache credit and considers $F - S_D - U_{\text{device}}$ for future allocations. Do not subtract allocator-reserved bytes from F again.

For $A = 3\,\text{GiB}$, $R = 4\,\text{GiB}$, $F = 2\,\text{GiB}$, and $U = 0.75\,\text{GiB}$, calculate job and device headroom under explicitly chosen caps and margins. Then mark the transfer-bandwidth estimate unavailable and propose a decision rule that does not interpret it as zero-cost transfer.

Part 4 - Experimental Design and Instrumentation

1. Define the schema and units before implementing probes. Use integer bytes for accounting and seconds for timestamps.

2. Collect CPU-only telemetry from a fixed workload. Record counter sources and supported allocator backend.

3. Maintain pinned-buffer and commitment ledgers explicitly. Compare them with broad host/GPU observations without forcing equalities that their scopes do not justify.

4. Inject missing metrics, stale timestamps, a changed budget epoch, and transfer completion arriving between reads.

5. Replay stored vectors through a simple admission policy and verify deterministic decisions for identical inputs.

Measure collection overhead at several sampling frequencies. Choose a lightweight boundary sampler before adding expensive continuous profiling.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate

measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import time
import torch

@dataclass
class RuntimeState:
    device_identity: str
    observation_epoch: int
    timestamp_seconds: float
    allocated_bytes: object
    reserved_bytes: object
    active_bytes: object
    global_free_bytes: object
    host_available_bytes: object
    managed_pinned_bytes: int
    pending_transfer_bytes: dict
    unmaterialised_commitment_bytes: int
    ready_task_ids: tuple
    unavailable_fields: tuple

def collect_gpu_observations(device, capabilities):
    observations = ...
    return observations

def collect_host_observations(host_probe, ownership_registry):
    observations = ...
    return observations

def reconcile_pending_work(task_registry, transfer_registry):
    pending_state = ...
    return pending_state

def validate_state_freshness(state, budget_epoch, policy):
    verdict = ...
    return verdict

def E01(runtime, sampling_policy):
```

```
state_trace = ...
return state_trace
```

Part 6 - Measurement, Validation, and Failure Analysis

Check units, device identity, timestamps, ledger version, and explicit missing fields. Test stale state and backend-specific unsupported statistics. A negative derived headroom must remain visible rather than being silently clipped and treated as valid capacity.

Compare repeated collection on an idle controlled workload and explain variation. Ensure collection creates no persistent GPU tensors and does not count pinned RAM twice. Verify pending transfers leave the backlog only under the declared completion condition.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each task or attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

The state vector is an interface between uncertain measurements and decisions. Explain which fields are authoritative scheduler records and which are external observations.

Recommend a sampling and freshness policy that matches task duration and disturbance speed. More frequent collection is useful only if it improves decisions enough to justify its overhead.

19.26.3 Exercise 02

Exercise Title: Build a PyTorch Scheduler That Selects the Next Executable Operation Only When Its Estimated Memory Requirement Fits the Current Budget

Part 1 - Exercise Contract and Why It Matters

Implement a serial admission scheduler that selects dependency-ready tasks only when their estimated additional peak fits the current GPU and host budgets. Use one compute stream initially. Deliver admission decisions, refusal reasons, completion records, and a comparison with an unrestricted serial reference.

Distinguish a task that must wait for an existing completion from one that can never fit under the current policy. Provide bounded waiting, no-fit detection, and an explicit outcome for an empty ready queue caused by a dependency error.

Part 2 - System and Memory Mental Model

Task memory must describe additional demand above the currently materialised state. A profiled absolute peak often includes parameters already resident; adding it to the live baseline double-counts them. Conversely, profiling forward alone misses later backward or optimizer requirements if the task contract includes those phases.

Admission is a compound predicate: prerequisites are valid and ready, the resource estimate is compatible with the shape and policy, budgets permit the reservation, and execution is supported. Memory fit cannot justify reading incomplete data or a stale parameter version.

The serial baseline has at most one active task but may retain outputs from earlier tasks. Release temporary storage only after completion; keep persistent outputs in the ownership ledger. A task's completion does not reset live memory to the starting baseline automatically.

Part 3 - Quantitative Prediction Before Execution

For candidate i, let d_i be an upper estimate of additional peak above the current state and u_i an uncertainty allowance if not already included. Require

$$A_t + U_t + d_i + u_i + S_J \le J_t, \qquad d_i + u_i + S_D \le F_t - U_{\text{device},t}.$$

The second inequality is a conservative no-cache-credit policy. Apply a corresponding host-memory check when tasks allocate RAM. Choose one uncertainty convention and avoid charging it twice.

Let the usable additional job budget be 1.5 GiB. Three ready tasks require 0.6, 1.2, and 1.8 GiB including uncertainty. Predict admissible tasks. Then suppose the first task retains a 0.4 GiB output: recompute the next admission rather than restoring the full initial headroom.

Write a manual schedule with one memory wait, one dependency wait, and one irreducible no-fit.

Part 4 - Experimental Design and Instrumentation

1. Define simple tensor tasks with known inputs, outputs, and temporary storage. Assign a fixed application cap below physical VRAM.
2. Build a CPU admission simulator and hand-check the proposed schedule before CUDA execution.
3. Execute one task at a time, recording baseline, region peak, completion, and retained outputs. Reconcile the ledger at each boundary.
4. Compare predicted and observed increments. Refuse incompatible profiles rather than silently using an old shape estimate.
5. Introduce a budget reduction before admission, then a controlled change during execution to expose the limit of preflight checks.

Use DMM23's narrow OOM routing if estimates fail. Recovery is a separate path, not evidence that admission was correct.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class AdmissionDecision:
    task_id: str
    admitted: bool
    reason: str
    incremental_upper_bytes: int
    observation_epoch: int

def identify_ready_tasks(graph, completed_tasks, storage_registry):
    ready_tasks = ...
    return ready_tasks

def estimate_additional_demand(task, runtime_state, profiles):
    estimate = ...
    return estimate

def evaluate_task_admission(task, state, estimate, budgets):
    decision = ...
    return decision

def execute_serial_task(task, runtime):
    completion_record = ...
    return completion_record

def reconcile_retained_outputs(completion_record, ledger):
    updated_ledger = ...
    return updated_ledger

def E02(runtime, task_graph, budgets):
    schedule_report = ...
    return schedule_report
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify each admitted task was ready, used the intended data version, and respected the predicted budget. Audit observed peaks separately: a completed task can violate the application cap without CUDA OOM.

Test a task exactly at the margin, a task below physical capacity but above the application cap, retained outputs, no-fit, and a dependency cycle. Require numerical agreement with the serial reference and explicit failure reasons rather than an indefinite idle loop.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each task or attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Admission converts an estimate into a controlled commitment. Explain the distinction between a prediction-respecting schedule and a proven physical-memory bound.

State where inaccurate demand, stale availability, or allocations outside the scheduler can invalidate the decision. This serial baseline provides the correctness reference for later concurrent policies.

19.26.4 Exercise 03

Exercise Title: Assign Memory Costs to Forward, Backward, Transfer, Prefetch, and Eviction Tasks and Compare Estimated Costs Against Real Measurements

Part 1 - Exercise Contract and Why It Matters

Build task-level resource profiles for forward, backward, H2D/D2H transfer, prefetch, and eviction. Describe resource lifetimes rather than assigning only one unsigned memory number to each task. Deliver calibrated profiles, held-out prediction errors, and an explanation of how profiles compose in a schedule.

Use fixed shapes and policies first, then validate at unseen compatible sizes. Keep GPU persistent state, host staging, retained outputs, temporary workspace, transfer bytes, and release conditions separate.

Part 2 - System and Memory Mental Model

Forward can produce activations retained until backward. Backward consumes those activations, creates or accumulates gradients, and uses temporary workspaces. Transfer often requires source and destination to coexist. Prefetch creates future residency before consumption. Eviction is a protocol that may allocate host storage and copy data before a GPU release becomes possible.

A transfer's logical direction does not imply the immediate sign of its GPU memory effect. H2D creates a destination; D2H leaves the source live until copy completion and all readers finish. Shared storage and aliases can prevent an anticipated release. Represent release as a completion-conditioned event, not negative admission cost.

Isolated task peaks do not automatically compose. Absolute profiles can repeat the same resident baseline, while concurrent tasks can overlap temporaries or share buffers. Calibrate incremental profiles and preserve storage identity. Profile diagnostic runs separately from timing runs because instrumentation may change execution.

Part 3 - Quantitative Prediction Before Execution

Represent task cost by a structured tuple

$$c_i = (d_i^{\mathrm{GPU}}, d_i^{\mathrm{host}}, p_i^{\mathrm{retained}}, r_i^{\mathrm{conditional}}, v_i^{\mathrm{H2D}}, v_i^{\mathrm{D2H}}, t_i).$$

Here d denotes peak additional demand, p retained output bytes, r a release eligible only after its guards, v transfer payload, and t measured duration. Do not add retained bytes again if they are already inside peak demand during the task.

For a 256 MiB activation offload, predict peak host demand, source GPU lifetime, transfer volume, and earliest GPU release. If a live alias remains, predict the realised release. For a forward that peaks at 3.5 GiB from a 2 GiB baseline and ends at 2.6 GiB, compute incremental peak and retained growth.

Use signed residual $e_i = M_i^{\text{observed}} - M_i^{\text{predicted}}$ and analyse positive residuals explicitly because they threaten admission safety.

Part 4 - Experimental Design and Instrumentation

1. Specify each task's region, storage inputs, created outputs, owners, and guards for completion and release.
2. Measure fixed-shape tasks in controlled runs with identical initial model and optimizer state where applicable.
3. Collect GPU peak, post-task live bytes, host demand, transferred bytes, and duration. Record cold and warmed behaviour separately.
4. Fit profiles on a calibration subset and test held-out sizes or lengths. Retain uncertainty and unsupported-domain information.
5. Compose two tasks serially, then a permitted overlapping pair. Compare union-of-lifetimes prediction against observed peak.
6. Add a retained-alias control for eviction and a workspace-spike control for compute.

Use supported stream events and host readiness checks for transfer timing; a nonblocking submission time is not transfer duration.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class TaskCostProfile:
    task_kind: str
    shape_key: str
    additional_gpu_peak_bytes: int
    additional_host_peak_bytes: int
    retained_output_bytes: int
    conditional_release_bytes: int
```

```
    h2d_bytes: int
    d2h_bytes: int
    duration_seconds: float
    release_guards: tuple

def measure_task_region(task, controlled_state, instrumentation):
    measurements = ...
    return measurements

def build_incremental_profile(task, measurements):
    profile = ...
    return profile

def predict_composed_lifetimes(tasks, profiles, storage_registry):
    prediction = ...
    return prediction

def evaluate_held_out_costs(profiles, held_out_measurements):
    errors = ...
    return errors

def E03(runtime, task_specifications, shape_grid):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify no baseline is counted twice and no release is credited before all its guards. Check that transferred bytes agree with the intended unique payload, while allocator rounding and metadata are reported separately.

Require finite outputs and gradient agreement for compute tasks. For transfers, verify content after proper completion; for eviction, verify the backing version and actual GPU release. Preserve failed predictions and explain whether shape, aliasing, workspace, or overlap caused the residual.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each task or attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

A resource profile is a description of demand over time. Explain which fields the scheduler needs for admission, which for placement, and which for evaluating future cost.

Choose the smallest profile family that predicts held-out schedules well enough. A compact but honest model is preferable to a single precise-looking peak number that hides lifetimes and retained outputs.

19.26.5 Exercise 04

Exercise Title: Implement a First-Fit Memory Scheduler That Admits Operations According to Available VRAM and Observe Its Utilization Inefficiencies

Part 1 - Exercise Contract and Why It Matters

Implement a queue-first feasible policy: scan dependency-ready tasks in a fixed order and select the first one whose resource contract fits. Name this application scheduling rule explicitly so it is not confused with the CUDA allocator's block-placement algorithm. Deliver a deterministic trace and counterexamples showing unused capacity, critical-path delay, or starvation.

Use a CPU discrete-event model to examine overlapping admitted work, then validate a serial CUDA implementation with the same task costs and dependencies. Real concurrent GPU execution is added in E12; simulation results must not be labelled measured CUDA utilisation.

Part 2 - System and Memory Mental Model

Queue order is a policy choice. Scanning beyond an oversized head avoids one form of head-of-line blocking, but small later tasks can delay a large task repeatedly. Selecting the earliest fitting item also ignores duration, downstream dependencies, retained outputs, and future releases.

Empty capacity is not necessarily inefficiency. A task may be dependency-blocked, resources may be reserved for admitted work, or the remaining capacity may be too small for any safe candidate. Compare against a feasible alternative schedule, not an impossible ideal that fills every byte.

Tasks need not relinquish all memory at completion. Retained outputs can reduce the next task's admissible space. Reserve demand before submission in the simulation and reconcile actual ownership at completion in CUDA. Unused reservations and live output storage are different reasons for apparent headroom loss.

Part 3 - Quantitative Prediction Before Execution

With ready queue $\mathcal{Q}_t$ and incremental upper demand d_i, select the earliest i satisfying the complete admission predicate. Memory fit alone can be expressed as $d_i \leq h_t$ only after headroom h_t has included margins and outstanding commitments.

Consider 10 abstract memory units and ready tasks with (d_i, t_i) equal to $(6, 8)$, $(5, 1)$, and $(4, 1)$, with units and seconds respectively. Compare fixed queue orders in a nonpreemptive discrete-event simulation. Declare which tasks may overlap and release all storage at completion for this toy case. Then add a downstream task dependent on the second item and predict how queue order changes makespan.

Measure memory slack area

$$I_M = \int_0^T \Big(J - S - M_{\text{committed}}(t) \Big)\, dt$$

in byte-seconds, together with makespan and useful completed work. Low slack is not itself evidence of high compute efficiency.

Part 4 - Experimental Design and Instrumentation

1. Implement a pure ready-queue selector with stable ordering and an explicit tie rule. Hand-check admission on a small task set.

2. Simulate arrivals, dependencies, completion events, and retained outputs. Compare at least three queue orders.

3. Construct one large-task starvation case and one case where first-fit performs as well as a more expensive policy.

4. Run serial CUDA tasks from controlled initial states and compare measured costs with the simulation inputs.

5. Record queue wait, admission refusals, memory slack, critical-path delay, and accepted throughput.

Bound queue waiting and report impossible-to-fit tasks. Do not classify every wait as a scheduler bug when no legal alternative exists.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class QueueEntry:
    task_id: str
    arrival_order: int
    ready_since: object
    additional_peak_bytes: int
    retained_bytes: int

def select_first_feasible(ready_queue, state, admission_policy):
    selection = ...
    return selection

def advance_simulated_completions(simulation_state, event_queue):
    next_state = ...
    return next_state

def simulate_first_fit(task_graph, queue_order, resource_limits):
    trace = ...
```

```
    return trace

def execute_serial_first_fit(runtime, graph, queue_order, limits):
    measurements = ...
    return measurements

def compare_queue_orders(traces, reference_work):
    comparison = ...
    return comparison

def E04(runtime, task_graph, queue_orders, limits):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify that only ready, fitting tasks are selected and that the earliest feasible task is chosen under the declared ordering. Test an empty queue, all tasks blocked, a too-large task, and retained outputs that change later feasibility.

Require identical completed task sets and numerical outputs across policy comparisons. Report starvation separately from valid no-fit. Distinguish measured CUDA times from simulated overlap, and compare memory slack alongside makespan rather than treating fuller memory as an automatic performance improvement.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each task or attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

First-fit is a transparent baseline with low decision overhead. Explain which workload properties make its simplicity valuable and which expose its lack of duration or dependency awareness.

Use the counterexample to motivate the next policy while preserving the cases where first-fit remains competitive. More scheduling logic must earn its cost through measured improvement.

19.26.6 Exercise 05

Exercise Title: Implement a Best-Fit Scheduler That Chooses Workloads Based on Remaining VRAM After Admission and Compare Fragmentation Behavior

Part 1 - Exercise Contract and Why It Matters

Implement an application best-fit policy that chooses the ready admissible task leaving the least predicted headroom after admission. Compare it with E04 using the same dependencies, arrivals, costs, and budgets. Deliver schedules, queue delays, memory slack, and an allocator-behaviour study with appropriate controls.

Define best-fit as task selection, not physical allocation placement. The scheduler does not control exact CUDA block addresses. A smaller residual application budget is not proof of reduced allocator fragmentation.

Part 2 - System and Memory Mental Model

Best-fit can pack current capacity tightly but ignore future releases or critical dependencies. A large low-priority task may occupy memory for a long time and delay several short useful tasks. Retained outputs can make a locally efficient choice globally expensive.

Task ordering changes allocation history and may influence allocator reuse, but backend behaviour, size classes, stream activity, and live aliases also matter. High reserved memory alone does not establish fragmentation. Compare current/peak live bytes, supported inactive split metrics, and block-history evidence where available.

Use fresh-process controls to separate policy effects from history carried over between experiments. Keep allocator configuration unchanged. If both task policies produce indistinguishable allocator behaviour, report that instead of attributing a hypothetical benefit to the word best-fit.

Part 3 - Quantitative Prediction Before Execution

For feasible ready set $\mathcal{F}_t$, define

$$i^* = \arg\min_{i \in \mathcal{F}_t} (h_t - d_i).$$

Declare tie-breaking by arrival order or another fixed rule. Host capacity and dependencies remain constraints even if GPU residual is the ranking criterion.

With $h_t = 10$ units, candidate demands $3, 7, 9$ and durations $1, 2, 20$ seconds, predict the selected item. Construct a dependency where choosing the nine-unit task delays the critical path. Compare makespan with first-fit under a specified order.

Define allocator reservation overhead $R - A$ as a descriptive quantity, not a fragmentation score. For native-backend inactive split bytes I, explain the limited meaning of I/R and why it cannot be compared blindly across backends. Predict which measurements would support an actual reuse advantage.

Part 4 - Experimental Design and Instrumentation

1. Test the selector on CPU for ties, no feasible task, and host-inadmissible candidates that otherwise fit GPU memory.

2. Replay identical synthetic task graphs under first-fit and best-fit. Include short and long tasks with retained outputs.

3. Execute fixed-shape CUDA allocation/compute workloads in fresh workers under each ordering, with repeated seeds.

4. Record live and reserved baselines, peaks, supported allocator statistics, and reuse behaviour after completion.

5. Add a controlled mixed-size allocation history and compare fresh-worker versus continued-process outcomes.

Treat allocator diagnostics as separate runs when their overhead would distort timing. Preserve explicit backend metadata.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every `...` marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class CandidateResidual:
    task_id: str
    remaining_gpu_bytes: int
    arrival_order: int
    admissible: bool

def calculate_candidate_residuals(ready_tasks, state, policy):
    residuals = ...
    return residuals

def select_best_fit(residuals, tie_policy):
    task_id = ...
    return task_id

def measure_allocator_history(runtime, ordered_tasks, diagnostics):
    evidence = ...
    return evidence

def compare_slack_and_fragmentation(first_fit, best_fit, backend):
    # Keep application slack distinct from allocator evidence.
    comparison = ...
    return comparison

def E05(runtime_factory, task_graphs, budgets):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify minimum residual among all actually feasible candidates, stable tie-breaking, and preserved dependency order. Check complete work and numerical equivalence before comparing performance.

Do not claim reduced fragmentation from lower slack or lower reserved memory alone. Require matched histories and backend-valid evidence. Report queue starvation, critical-path delays, and decision overhead even if best-fit fills memory more densely.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each task or attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Best-fit optimises a local packing criterion. Explain whether that criterion aligned with useful progress in the measured workload.

Recommend it only where its capacity use outweighs duration and dependency costs. State any allocator conclusions separately and limit them to the observed backend and allocation history.

19.26.7 Exercise 06

Exercise Title: Build a Priority Scheduler That Favors Compute Tasks over Prefetch and Eviction When GPU Execution Would Otherwise Stall

Part 1 - Exercise Contract and Why It Matters

Build a priority policy that keeps useful compute supplied without starving the transfers and evictions needed to make that compute ready. Compare it with first-fit and strict compute-first ordering. Deliver priority reasons, stall attribution, fairness metrics, and accepted useful throughput.

Define eligible compute as dependency-ready and memory-admissible. A task with missing parameters is not ready compute merely because its task type says forward. A prerequisite transfer or capacity- releasing eviction may need temporary priority promotion.

Part 2 - System and Memory Mental Model

Task type alone is insufficient for priority. A speculative prefetch for distant work can wait, while a transfer that unblocks the next critical compute is urgent. Similarly, unnecessary eviction can waste bandwidth, but an eligible eviction may be required before any compute can fit.

Use critical-path information or estimated time-to-use to distinguish urgent prerequisites from optional lookahead. Propagate urgency to their dependency chain. Add aging or bounded waiting so lower-ranked tasks cannot be postponed indefinitely by a steady stream of compute.

CUDA kernels already running are not made preemptible by a Python priority queue. The scheduler controls future submissions and stream dependencies. Stream priority is a separate device feature, not a guarantee that a submitted high-priority task immediately interrupts all other activity.

Part 3 - Quantitative Prediction Before Execution

Define an illustrative ranking tuple with hard eligibility first, then critical-path class, deadline slack, and waiting age. For a task with expected time-to-use D_i and prerequisite service time T_i, slack is

$$s_i = D_i - T_i.$$

Negative slack predicts exposure to a stall unless another task can cover it. Units must be seconds, not a mixture of bytes and arbitrary priorities.

Construct a trace with ready compute lasting 4 ms, a required parameter transfer lasting 3 ms, and a speculative transfer lasting 8 ms. Predict schedules under strict compute-first and urgency-aware priority. Then add an eviction required for the compute destination. Explain why its priority must change despite its task type.

Measure dependency-induced idle time and memory-induced idle time separately. A lower idle interval is valuable only if accepted work and correctness remain unchanged.

Part 4 - Experimental Design and Instrumentation

1. Annotate a small task graph with critical successors and estimated durations. Hand-label urgent versus speculative movement.

2. Replay first-fit, compute-first, and dependency-aware priority in a discrete-event simulator with the same resource constraints.

3. Add continuous compute arrivals to test transfer starvation, then add aging or a maximum waiting rule.

4. Validate one CUDA schedule against a serial reference. Measure complete wall time and task intervals rather than enqueue times alone.

5. Attribute stalls to missing data, capacity, dependency, or host scheduling overhead using recorded evidence.

Do not use a sampled GPU-busy percentage as the sole objective. Report completed useful tasks, samples, or tokens per second.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch
```

```
@dataclass
class TaskPriorityEvidence:
    task_id: str
    critical_successor: object
    predicted_slack_seconds: object
    waiting_seconds: float
    urgency_class: str

def identify_compute_blockers(graph, ready_state, memory_state):
    blockers = ...
    return blockers

def propagate_dependency_urgency(graph, blockers, duration_profiles):
    urgency = ...
    return urgency

def rank_eligible_tasks(tasks, urgency, fairness_policy):
    ranking = ...
    return ranking

def attribute_compute_stalls(event_trace):
    stall_report = ...
    return stall_report

def E06(runtime, task_graph, priority_policies):
    comparison = ...
    return comparison
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify that a prerequisite can outrank speculative work and that priority never bypasses dependency or memory checks. Test a graph where only eviction can create progress.

Require bounded waiting or an explicit no-fit/dependency error. Compare outputs and accepted task counts across policies. Report whether reduced compute stalls increased transfer traffic, decision overhead, or long-task waiting time.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each task or attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Favouring compute means protecting its supply chain, not always selecting compute-labelled tasks first. Explain how dependency urgency changed the effective priorities.

Recommend the least complex priority scheme that reduces measured critical stalls while preserving fairness and bounded resource use.

19.26.8 Exercise 07

Exercise Title: Add Dependency Tracking So Memory-Safe Operations Cannot Execute Before Their Required Parameters, Inputs, or Activations Are Ready

Part 1 - Exercise Contract and Why It Matters

Implement dependency and data-version tracking alongside memory admission. A task must not execute until required inputs, parameters, activations, and transfers are valid for its version. Deliver a task DAG, storage-readiness registry, event trace, and tests for missing, cyclic, stale, and failed dependencies.

Distinguish host submission readiness from actual device start readiness. A consumer can sometimes be enqueued with an explicit stream wait before its producer completes, but it must not execute its read before the producer's completion event.

Part 2 - System and Memory Mental Model

Logical dependencies state what data or prior update is required. CUDA events and stream waits implement execution ordering. Tensor ownership and allocator lifetime tracking ensure the storage remains valid through the consumer's use. These are three separate responsibilities.

Parameter and activation identities require versions. A cached parameter copy from an earlier optimizer update may have the right shape and enough allocated bytes but still be semantically wrong. A failed producer must not mark its outputs valid or release dependent tasks for execution.

Completion of a transfer makes its destination ready only after the relevant event. For asynchronous D2H, CPU consumers also need completion before reading the destination. Enqueuing a wait on a GPU stream does not by itself make data ready for immediate host access.

Part 3 - Quantitative Prediction Before Execution

For task i with predecessor set $\mathrm{pred}(i)$, logical readiness requires valid outputs of all predecessors. In a duration model,

$$t_i^{\mathrm{start}} \geq \max_{j \in \mathrm{pred}(i)} t_j^{\mathrm{complete}}.$$

Resource limits may delay it further. Submission can precede this start when a correct stream dependency is installed.

For parameter prefetch taking 3 ms, input transfer taking 2 ms, and forward taking 5 ms, predict the earliest forward start if both transfers can begin at time zero on permitted engines. Then impose a shared serial copy resource and recompute the bound. Do not assume two named streams imply two independent copy engines.

Represent storage readiness as $(\mathrm{id}, \mathrm{version}, \mathrm{location}, \mathrm{event})$. Give a counterexample where all byte checks pass but the version or event does not satisfy the consumer.

Part 4 - Experimental Design and Instrumentation

1. Validate a DAG on CPU, including cycle detection, unknown predecessors, and stable task identities.

2. Track produced versions and consumer requirements separately from physical buffer addresses.

3. Simulate delayed and failed producers. Verify their consumers remain blocked or receive a propagated failure outcome.

4. Implement CUDA events for supported producer-consumer edges. Compare a fully synchronised reference with explicit stream waits.

5. Add an asynchronous D2H result and require host readiness before content validation.

Test unsafe-ordering logic in simulation; do not rely on repeated racy GPU executions to establish a safe dependency implementation.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class DataRequirement:
    storage_id: str
    required_version: int
    required_location: str

@dataclass
class ProducerRecord:
    task_id: str
    output_versions: tuple
    completion_event: object
    outcome: str

def validate_dependency_graph(task_graph):
    validation = ...
    return validation

def check_required_versions(task, storage_registry):
    verdict = ...
    return verdict

def install_consumer_waits(task, producer_records, target_stream):
    # Establish execution ordering before consumer reads.
    wait_record = ...
```

```
    return wait_record

def publish_successful_outputs(task, completion, storage_registry):
    updated_registry = ...
    return updated_registry

def propagate_producer_failure(task_graph, failed_task):
    affected_tasks = ...
    return affected_tasks

def E07(runtime, task_graph, delayed_producers):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Check every consumer's required version and actual start ordering. Verify no output is published from a failed producer. Test missing dependencies, cycles, duplicate completion notifications, and a parameter update invalidating a prefetched old version.

Compare serial and scheduled numerical outputs. Distinguish event completion evidence from host enqueue timestamps. Verify storage remains alive through all consumers, including side-stream users. Memory fit without these properties is not correct scheduling.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each task or attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Dependency tracking turns a collection of individually safe tasks into a correct execution graph. Explain where host readiness, device ordering, and storage lifetime differ.

State how the registry handles version changes and failures so future placement and prefetch decisions cannot accidentally resurrect stale data.

19.26.9 Exercise 08

Exercise Title: Build a Scheduler That Chooses Between Waiting for Free VRAM and Evicting Existing Tensors Based on Estimated Stall Cost

Part 1 - Exercise Contract and Why It Matters

Implement a decision between waiting for a known release and evicting eligible resident data to admit blocked work. Compare estimated time to useful compute under both choices. Deliver a release forecast, victim eligibility evidence, selected action, and measured counterfactual cost in controlled replay.

Waiting must be bounded and tied to an event that can actually progress. Eviction must preserve a valid backing version or a supported recomputation path. Do not treat deleting arbitrary autograd tensors as a valid eviction operation.

Part 2 - System and Memory Mental Model

A pending task may release storage soon, making waiting cheaper than a copy out followed by a copy back. Conversely, waiting for an uncertain external process can stall indefinitely while an eligible local victim could enable progress.

Estimate release time from known completion events and lifetimes, not just task duration. A task may complete but leave an output needed later. An eviction may need pinned host staging, copy bandwidth, a valid destination, and completion of all GPU readers before releasing storage. Account for its transient peak.

Resource deadlock is possible even with an acyclic data graph: blocked tasks hold memory needed by the operations that would release it. Identify circular resource waits and no-fit states. Reserve enough control resources for a permitted recovery or offload action when the design requires them.

Part 3 - Quantitative Prediction Before Execution

Let T_w be time until sufficient confirmed release for the blocked task. Let T_e include victim readiness, required copying, and actual release. Add expected future reload cost T_r and contention cost T_c when comparing eviction:

$$C_{\text{wait}} = \widehat{T}_w, \qquad C_{\text{evict}} = \widehat{T}_e + p_{\text{reuse}}\widehat{T}_r + \widehat{T}_c.$$

These are modelled incremental costs; overlapping components must not be summed twice. State the horizon and uncertainty.

For a known release in 4 ms versus eviction requiring 2 ms outbound and 3 ms expected reload with reuse probability 0.8, compare the simple costs before adding contention. Recompute when the waiting event cannot progress until the blocked task executes. Explain why this is a deadlock issue rather than a larger finite wait estimate.

Predict the decision when host staging cannot be allocated.

Part 4 - Experimental Design and Instrumentation

1. Build a controlled graph with a blocked compute task, one independently progressing release, and one reloadable immutable victim.

2. Measure release and transfer costs separately, then freeze the prediction before choosing an action.

3. Replay the incident with waiting and eviction from identical states to estimate the counterfactual cost.

4. Vary time-to-release, victim size, reuse distance, and transfer contention one factor at a time.

5. Add a resource-wait cycle and an unknown external release. Require cycle detection or bounded deferral with an explicit reason.

Verify content and version after reload. For training, only use eviction mechanisms already proven compatible with the logical update contract.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate

measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class ReleaseForecast:
    storage_id: str
    eligible_release_bytes: int
    completion_event: object
    estimated_seconds: object
    independent_progress: bool

def forecast_confirmable_releases(runtime, blocked_task):
    forecasts = ...
    return forecasts

def estimate_eviction_path(victim, resources, transfer_profile):
    estimate = ...
    return estimate

def detect_resource_wait_cycle(task_graph, ownership_registry):
    cycle_report = ...
    return cycle_report

def choose_wait_or_evict(wait_forecasts, eviction_options, policy):
    decision = ...
    return decision

def compare_counterfactual_replays(runtime_factory, incident):
    comparison = ...
    return comparison

def E08(runtime, blocked_tasks, decision_policy):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Check that predicted releases are not credited before completion and last use. Verify a waited-for event can progress independently of the blocked task. Test unknown completion time, insufficient host staging, no eligible victim, and resource deadlock.

Require numerical correctness after any reload and exact victim version identity. Compare predicted and observed stalls including future reload. A fast immediate eviction can still lose over the declared horizon.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each task or attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Waiting and eviction are alternative investments in future progress. Explain which uncertainty dominated the decision and how a shorter horizon can change the preferred action.

Recommend a bounded policy with a clear fallback when neither choice can make progress. An honest no-fit result is preferable to an unbounded wait disguised as scheduling.

19.26.10 Exercise 09

Exercise Title: Schedule Parameter Prefetch Early Enough to Hide Transfer Latency Without Consuming VRAM Too Far Ahead of Use

Part 1 - Exercise Contract and Why It Matters

Build just-in-time parameter prefetch for a sequential inference workload whose immutable layer weights have CPU backing. Choose launch lead time from measured transfer duration and expected compute time-to-use. Deliver a timeline, exposed transfer stalls, prefetched residency, and output agreement with a fully resident reference.

Use fixed-version inference parameters first so placement can change without altering autograd or optimizer bindings. A training extension must separately preserve parameter identity, version, gradient flow, and backward requirements. Replacing an nn.Parameter with a detached CPU/GPU copy is not a valid training offload scheme.

Part 2 - System and Memory Mental Model

Prefetch hides latency only when transfer can overlap independent compute and complete before the consumer. Pinned source buffers, appropriate streams, available copy resources, and real workload independence matter. A nonblocking API return does not demonstrate overlap.

The destination consumes GPU storage before first use and may remain necessary through later consumers. CPU pinned sources must stay valid and unmodified until transfer completion. Consumer execution needs a dependency wait, while destination storage must remain protected until all its stream uses finish.

Forecast time-to-use can change with shape, batch, compute contention, and scheduler decisions. Prefetching too far ahead increases residency and can crowd out active computation. Use bounded lookahead and preserve the required parameter version. Do not refill a buffer while a previous consumer still reads it.

Part 3 - Quantitative Prediction Before Execution

For payload q_i, measured effective bandwidth v_i, and setup/queue delay a_i,

$$\widehat{T}_i^{\text{copy}} = a_i + q_i/v_i.$$

If predicted use is at t_i^{use}, target launch near $t_i^{\text{use}} - \widehat{T}_i^{\text{copy}} - \delta_i$, subject to resource admission and nonnegative available lead time. δ_i is an uncertainty allowance, not free capacity.

For a 64 MiB layer and measured 12 GiB/s transfer rate, estimate copy duration before setup overhead. Compare lead opportunities from one 3 ms compute block versus two such blocks.

Measure early residency cost as

$$C_{\text{residency}} = \sum_i q_i \max(0, t_i^{\text{first use}} - t_i^{\text{destination allocation}}).$$

Use actual allocation time, since memory may be consumed before the copy finishes. Predict how deeper lookahead changes this cost.

Part 4 - Experimental Design and Instrumentation

1. Create CPU-backed immutable layer weights and a fully resident reference model. Assign explicit version IDs.
2. Measure copy and compute durations across representative sizes. Separate pinned-buffer preparation from transfer time.
3. Compare demand loading, one-layer lookahead, and time-to-use-based prefetch under the same GPU budget.
4. Track destination allocation, copy start/completion, consumer start, and final storage release.
5. Vary compute duration and bandwidth contention, then measure late arrivals and unnecessarily early residency.
6. Check a bounded GPU buffer pool whose reuse waits for all required consumers.

Use event traces to establish overlap and retain the serial reference for numerical validation.
Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class ParameterPrefetch:
    layer_id: str
    parameter_version: int
    payload_bytes: int
    predicted_use_seconds: float
    copy_duration_seconds: float
    destination_buffer_id: object

def estimate_parameter_use_times(layer_graph, compute_profiles):
    use_times = ...
    return use_times

def select_prefetch_launches(candidates, state, lookahead_policy):
    launches = ...
    return launches

def submit_versioned_parameter_copy(candidate, transfer_stream):
    transfer_record = ...
    return transfer_record

def bind_ready_inference_weights(layer, transfer_record, stream):
    # Preserve immutable-version and consumer-ordering contracts.
    binding = ...
    return binding

def audit_prefetch_lifetimes(event_trace, buffer_registry):
    audit = ...
    return audit

def E09(inference_workload, transfer_profiles, budgets):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Require reference-equivalent outputs, correct layer/version selection, and complete transfers before consumer reads. Verify source pin-buffer lifetime and destination protection. Do not infer correctness from the absence of CUDA errors.

Measure exposed stalls, copy overlap, peak GPU and pinned host bytes, residency byte-seconds, and end-to-end latency. Check delayed copies, variable compute duration, and a full destination pool. A deeper prefetch queue is not automatically a performance improvement.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each task or attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an

explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Good prefetch arrives early enough to be useful and late enough to avoid unnecessary residency. Explain the measured lead-time window and its sensitivity to changing workload speed.

State the inference-only parameter-placement boundary and what additional machinery a training implementation would require. This prevents a scheduling optimisation from silently breaking gradient or optimizer semantics.

19.26.11 Exercise 10

Exercise Title: Detect Over-Prefetching and Dynamically Delay Future Transfers When Prefetched Data Begins Crowding Out Active Compute Memory

Part 1 - Exercise Contract and Why It Matters

Detect when prefetched residency and pending destination commitments consume capacity needed by near-term compute. Add a bounded prefetch controller that delays future submissions while preserving already required transfers. Compare fixed lookahead with dynamic throttling under variable compute speed and budgets.

Deliver a prefetch-residency trace, reasons for delayed transfers, compute-admission stalls, and net throughput. Keep submitted copies distinct from unsent proposals: ordinary queued CUDA work cannot be assumed safely cancellable.

Part 2 - System and Memory Mental Model

Over-prefetching is a relationship between future residency and current demand, not merely a large queue length. A few large unused destinations can be more harmful than many small transfers already near consumption. Combine bytes, time-to-use, and compute's projected peak.

Track materialised unused destinations separately from unmaterialised copy commitments so their costs are not counted twice. A transfer submitted but not completed still needs source and destination lifetime protection. Marking it obsolete does not make its memory instantly releasable.

Throttle work that has not yet been submitted. If already submitted data becomes unnecessary, wait for safe completion and last-use conditions before retiring it. Keep urgency for the next compute prerequisite so throttling does not create the very stall it is intended to avoid.

Part 3 - Quantitative Prediction Before Execution

Let P_t be live prefetched bytes not yet consumed, U_t^P their unmaterialised commitments, and C_t the conservative additional requirement of near-term compute. A policy may require

$$A_t + U_t + C_t + S_J \leq J_t$$

before accepting discretionary prefetch, with each term reconciled to avoid overlap. Also impose a configurable prefetch-residency limit.

For a usable cap of 6 GiB, live baseline 3 GiB including 1 GiB of unused prefetch, outstanding commitments of 0.5 GiB, and imminent compute demand of 2 GiB, decide whether an extra 0.75 GiB prefetch can be admitted. Do not add P_t again when it is already inside A_t.

Measure wasted transfer fraction as bytes evicted or invalidated before any valid use divided by completed prefetched bytes, with an explicit zero-denominator rule.

Part 4 - Experimental Design and Instrumentation

1. Use E09's versioned workload and a fixed lookahead baseline. Create a trace where compute slows and prefetched residency grows.
2. Record live unused bytes, future commitments, time-to-use, compute demand, and transfer urgency at each decision.
3. Add throttle/release thresholds and bounded pending work. Delay only unsent discretionary transfers.
4. Introduce a budget drop after some copies are submitted. Track them to safe completion instead of assuming cancellation.
5. Compare stalls, memory peaks, unused residency, wasted transfer volume, and accepted useful throughput.

Include one workload where generous prefetch remains beneficial so the controller is not tuned to suppress every transfer.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every `...` marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class PrefetchPressure:
    live_unused_bytes: int
    unmaterialised_bytes: int
    imminent_compute_bytes: int
    earliest_use_seconds: object
    submitted_transfer_ids: tuple

def classify_prefetch_pressure(registry, compute_forecast, state):
    pressure = ...
```

```
    return pressure

def select_unsent_transfers_to_delay(queue, pressure, policy):
    delayed_ids = ...
    return delayed_ids

def retire_obsolete_completed_prefetch(registry, completion_events):
    # Release only after all lifetime and version guards.
    retired_records = ...
    return retired_records

def update_prefetch_throttle(controller_state, observation, policy):
    next_state = ...
    return next_state

def E10(runtime, workload_trace, prefetch_policies):
    comparison = ...
    return comparison
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify no double-counting of materialised destinations and commitments. Check that urgent prerequisites still progress while discretionary prefetch is delayed.

Test a budget drop during a copy, invalidated versions, empty queues, and destination reuse after completion. Compare numerical outputs and verify no host source mutation occurs before copy completion. A claimed cancellation must not release storage still used by submitted CUDA work.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each task or attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Prefetch is useful only when it reduces exposed latency more than the residency it imposes. Explain which signals detected crowding before a compute OOM.

Recommend a policy balancing near-term compute headroom and transfer readiness. State how quickly it responds and when already submitted work limits immediate recovery.

19.26.12 Exercise 11

Exercise Title: Integrate Eviction Decisions into the Scheduler and Select Victims from Reuse Distance, Size, Reload Cost, and Current Memory Pressure

Part 1 - Exercise Contract and Why It Matters

Build an eviction selector over explicitly managed, reloadable storage. Rank eligible victims using reclaimable size, predicted reuse, reload/transfer cost, and memory pressure. Deliver

eligibility evidence, victim choices, realised releases, and future reload cost compared with simple size- or recency-based policies.

Start with immutable prefetched inference data and clean CPU backing. Training activations may be evicted only through a supported offload/recomputation mechanism that preserves autograd's contract. Arbitrary deletion of saved tensors is not a valid policy implementation.

Part 2 - System and Memory Mental Model

Eligibility precedes ranking. A tensor with an in-flight reader, an invalid backing version, an unresolved writer, or necessary untracked aliases cannot be assumed immediately reclaimable. Logical size and reclaimable storage can differ because several objects share a base allocation.

Reuse distance may be measured in future task count or estimated time; keep that unit explicit. A small soon-needed tensor may be expensive to evict despite a low byte size, while a large distant immutable copy may be a good victim. Predictions can be wrong, so record actual next use and policy error.

Offloading dirty data needs host capacity and transfer work before release. Dropping a clean cache copy with valid backing can be cheaper. Retiring storage still requires correct stream and ownership guards. Pressure changes how much must be reclaimed, not whether those guards can be bypassed.

Part 3 - Quantitative Prediction Before Execution

Let D be required additional headroom and r_j the conservatively reclaimable bytes of eligible victim j. Choose a set $\mathcal{V}$ satisfying

$$\sum_{j \in \mathcal{V}} r_j \geq D$$

while minimising a declared predicted eviction-plus-reload cost. Shared storage invalidates a naive sum unless victim groups are deduplicated and jointly releasable.

For three clean victims of 128, 256, and 512 MiB, with expected next-use times 2, 20, and 100 ms, assign measured reload costs and select a set to reclaim 300 MiB. Compare largest-first, farthest-reuse, and cost-aware choices.

If using a score, normalise byte, time, and reuse features before combining them. Explain why adding raw bytes to milliseconds produces an arbitrary ranking.

Part 4 - Experimental Design and Instrumentation

1. Build a storage registry with version, backing state, owner count, last-use events, logical size, and reclaimable storage group.

2. Construct a controlled task graph with known future uses for an offline comparison and limited lookahead for the online policy.

3. Evaluate victim eligibility before ranking. Include aliases, in-flight readers, and dirty data as negative cases.

4. Execute valid evictions, measure actual release, and track later reload latency and transferred bytes.

5. Compare simple and cost-aware policies on held-out reuse patterns and changing pressure.

Do not give the online policy future information beyond its declared lookahead; an omniscient victim order is only a comparison bound.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class EvictionCandidate:
    storage_group_id: str
    version: int
    reclaimable_bytes: int
    backing_valid: bool
    pending_reader_events: tuple
    predicted_reuse_seconds: object
    estimated_reload_seconds: float

def identify_eligible_victims(storage_registry, event_state):
    candidates = ...
    return candidates

def rank_victim_groups(candidates, pressure, cost_policy):
    ranking = ...
    return ranking

def select_reclaim_set(ranking, required_headroom):
    selection = ...
    return selection

def retire_storage_after_guards(runtime, selected_groups):
    realised_releases = ...
    return realised_releases

def compare_predicted_and_actual_reuse(eviction_log, use_trace):
    errors = ...
    return errors
```

```
def E11(runtime, task_trace, eviction_policies):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify valid backing versions, no in-flight readers at release, and no double-counted storage groups. Compare predicted reclaimable bytes with observed live-allocation reduction. An alias-held allocation must not be reported as freed.

Reload and compare content with the reference before later use. For supported training offload, compare gradients and accepted updates. Test no eligible victim and insufficient host capacity; both require explicit wait/defer outcomes.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each task or attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Eviction is a prediction about future value under present pressure. Explain where your reuse model was accurate and where wrong predictions created unnecessary traffic or stalls.

Recommend a victim policy together with its eligibility rules, not a ranking formula alone. Correct lifetime management is a prerequisite for every performance comparison.

19.26.13 Exercise 12

Exercise Title: Coordinate Multiple CUDA Streams Through a Central Scheduler and Prevent Unsafe Concurrent Memory Use

Part 1 - Exercise Contract and Why It Matters

Coordinate compute and transfer streams through a central task scheduler. Enforce data dependencies, storage lifetime, and aggregate resource admission before submission. Deliver a multi-stream event trace, comparison with a serial reference, and a memory audit covering overlapping operations.

Begin with one compute stream and one transfer stream; add a second compute or transfer stream only after the first design passes. Extra streams are execution queues, not a guarantee of simultaneous kernels, independent bandwidth, or faster work.

Part 2 - System and Memory Mental Model

Operations on one stream are ordered, but cross-stream consumers need explicit dependencies. An event recorded after a producer can make a consumer stream wait without blocking the host. For CPU consumption of D2H data, the host must observe actual copy completion before reading it.

Data ordering and allocation lifetime are separate. A tensor used on a side stream must remain protected through that use. Use an explicit ownership protocol, appropriate allocator stream tracking such as `record_stream`, or documented manual lifetime management. Allocator tracking does not insert the producer-consumer data dependency by itself.

Concurrent tasks overlap temporaries, destinations, and retained outputs. Each individually fitting task can still cause aggregate OOM. A central admission decision must account for all active commitments. Host function return means work was submitted, not that resources are ready to retire.

If a task fails, later submitted work may still exist. Stop new dependent admissions, preserve in-flight ownership, and settle managed events safely before rollback. Do not overwrite buffers that a still-running stream may use.

Part 3 - Quantitative Prediction Before Execution

For disjoint incremental task lifetimes $m_i(t)$ above resident baseline M_0,

$$M_{\text{concurrent}}(t) = M_0 + \sum_i m_i(t).$$

Shared allocations must be deduplicated. A conservative upper envelope sums active task bounds, but adding absolute isolated peaks would count M_0 repeatedly.

With a 2 GiB baseline, compute demand 1.5 GiB, and transfer-destination demand 1 GiB, compare serial and overlapping peaks under explicit lifetimes. Decide whether overlap fits a usable 4 GiB cap.

For compute time 8 ms and copy time 5 ms, the ideal fully overlapped lower comparison is $\max(8, 5)$ ms, while serial time is 13 ms. Add dependency, launch, and bandwidth effects before treating either as a measured result.

Part 4 - Experimental Design and Instrumentation

1. Build a fully serial correct baseline with fixed data. Record outputs and gradients where the workload trains.

2. Declare stream ownership, producer events, consumer waits, and final-use events for every shared buffer.

3. Add central admission and a provisional commitment ledger before enabling overlap. Test bookkeeping with a CPU simulator.

4. Execute the supported multi-stream graph and capture actual compute/copy intervals. Use a joined end event or declared synchronised boundary for full-schedule timing.

5. Compare serial, dependency-correct overlap, and a conservative nonoverlap fallback under the same cap.

6. Inject delayed completion and allocation failure through controlled hooks. Verify no resource is retired early.

Avoid global synchronisation after every task in the performance run, since it would destroy the overlap being evaluated.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use

a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class StreamTaskHandle:
    task_id: str
    stream_id: str
    completion_event: object
    protected_storage_ids: tuple
    reservation_id: str

def construct_scheduler_streams(device, stream_policy):
    streams = ...
    return streams

def install_task_dependencies(task, producers, target_stream):
    dependency_record = ...
    return dependency_record

def protect_cross_stream_storage(task, ownership_registry):
    protection_record = ...
    return protection_record

def submit_admitted_task(task, stream, reservation):
    handle = ...
    return handle

def retire_completed_handles(handles, ownership_registry):
    retirements = ...
    return retirements

def E12(runtime, task_graph, stream_policy, budgets):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Require numerical equivalence with the serial reference and event evidence for all cross-stream reads. Check that source pinned buffers remain valid during H2D and host reads wait for D2H completion.

Audit aggregate peak, commitment lifetime, duplicate completion, and allocation failure with other tasks still in flight. Verify stream tracking is not mistaken for data synchronisation. Report actual overlap and useful throughput; two stream names alone do not prove concurrency or speedup.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each task or attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Concurrency is safe only when execution order, ownership, and aggregate resources agree. Explain which constraint limited overlap in the measured workload.

Recommend the smallest stream set that provides a benefit under the budget. More concurrency can raise memory and contention enough to reduce accepted throughput.

19.26.14 Exercise 13

Exercise Title: Build a Memory Reservation System That Prevents Concurrent Tasks from Independently Assuming the Same Free VRAM Is Available

Part 1 - Exercise Contract and Why It Matters

Implement atomic application reservations for future task memory. Selection, feasibility checking, and reservation creation must share a consistent ledger epoch so two submissions cannot both claim the same headroom. Deliver reservation state transitions, concurrency tests, and a CUDA audit of materialisation and release.

Application reservations are logical promises, not physical CUDA allocations. They prevent oversubscription only among cooperating scheduler-managed work and only within the validity of demand estimates. External processes and hidden unprofiled allocations remain outside that guarantee.

Part 2 - System and Memory Mental Model

Two host submitters can read identical free-memory snapshots and independently admit tasks whose combined demand exceeds the cap. The GIL is not a substitute for an explicit compound check-and-reserve operation. Use one event-loop owner or a lock-protected transaction, with generation IDs for stale events.

As promised storage materialises, it moves from unmaterialised commitment into actual ownership. Do not keep the full promise and add all materialised bytes again, or refund the entire reservation at first allocation while later workspaces remain. Completion converts persistent outputs into resident ownership; only genuinely retired temporary demand is released.

Exact materialisation accounting can be difficult for opaque PyTorch calls. A conservative alternative uses a reconciled resident baseline plus full additional task envelopes until completion, without adding observed in-flight allocations again. Refresh the baseline only at a safe reconciliation point or through trusted ownership events.

Reservation expiry cannot free bytes used by already submitted CUDA work. Cancellation before submission and retirement after submission have different rules. Duplicate and late callbacks must be idempotent and version-checked.

Part 3 - Quantitative Prediction Before Execution

With materialised live bytes A and additional unmaterialised commitments U,

$$A + U + S_J \leq J.$$

A trusted materialisation of x bytes changes $A \leftarrow A + x$ and $U \leftarrow U - x$, preserving the accounted total. Further task demand remains reserved. In the envelope alternative, account

$$M_{\text{accounted}} = M_{\text{resident}} + \sum_{i \in \text{active}} d_i,$$

and do not separately add the same active allocations.

For available headroom 1.5 GiB and two submitters requesting 1 GiB each, manually interleave a broken check-then-reserve sequence and the atomic sequence. Show the required rejection or wait.

For a task reserving 1 GiB that completes with a 256 MiB retained output, calculate the maximum releasable commitment under the declared reconciliation model. Explain why submit-time return releases nothing by itself.

Part 4 - Experimental Design and Instrumentation

1. Define states for planned, reserved, submitted, completed-with-outputs, cancelled-before-submit, and retired. Specify legal transitions and their accounting effects.

2. Use deterministic CPU interleaving tests for two submitters, then multithreaded tests if the implementation permits threads.

3. Test partial materialisation, retained outputs, early cancellation, duplicate completion, and stale generation IDs.

4. Integrate with E12 and compare the ledger with measured allocator observations at safe reconciliation boundaries.

5. Introduce a budget reduction while reservations exist. Stop new admission and resolve outstanding work; do not retroactively claim the old promises never existed.

Keep GPU, host, and pinned-buffer commitments separately scoped, while documenting which host quantities are subsets.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import threading
import torch

@dataclass
class Reservation:
    reservation_id: str
    task_id: str
    generation: int
    gpu_upper_bytes: int
    host_upper_bytes: int
    state: str

class ReservationLedger:
    def __init__(self, budgets, accounting_mode):
        self.budgets = budgets
        self.accounting_mode = accounting_mode
        self.entries = ...
        self.epoch = ...
        self.lock = ...

    def try_reserve(self, task, demand, observation):
        # Perform one atomic check-and-reserve transaction.
        decision = ...
        return decision

    def mark_submitted(self, reservation_id, generation, handle):
        ...

    def reconcile_materialisation(self, ownership_event):
        ...

    def complete_task(self, handle, retained_outputs):
        ...

    def cancel_unsubmitted(self, reservation_id, generation):
        ...

    def audit_invariants(self):
        audit = ...
        return audit

def E13(runtime, concurrent_submission_cases, budgets):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify no two cooperating submissions overspend one ledger epoch. Check invariants after

every transition, including exceptions during submission. Test that a submitted task cannot be released by timeout alone.

Require exact handling of retained outputs and no double counting of active allocations. Compare at reconciled boundaries rather than asserting all telemetry reads are atomic. Record external-pressure OOM separately from a reservation-bookkeeping failure.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each task or attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Reservations coordinate intent before the allocator acts. Explain the protection they provide among cooperating tasks and the assumptions that limit it.

Choose between detailed materialisation tracking and conservative envelopes based on observability and cost. A slightly conservative correct ledger is preferable to an optimistic one that refunds memory before work is finished.

19.26.15 Exercise 14

Exercise Title: Add High- and Low-Watermark Memory Pressure States and Change Scheduling Behavior Before the Runtime Reaches OOM

Part 1 - Exercise Contract and Why It Matters

Add normal, pressured, critical, and no-fit scheduling states with distinct high and low watermarks. Change discretionary prefetch, admission concurrency, and permitted recovery behaviour before physical OOM. Deliver a transition table, manually predicted traces, and observed response delay.

Define the monitored quantity precisely: use application- accounted demand relative to a job budget for the primary state and retain device/host checks as separate constraints. High allocator reservations caused by idle cache must not automatically be interpreted as high live compute demand.

Part 2 - System and Memory Mental Model

Hysteresis prevents noise near one threshold from repeatedly switching policy. A high watermark enters pressure; a lower watermark plus sustained evidence exits it. Critical admission failure must override ordinary patience or smoothing.

Pressure changes policy for future work. It cannot instantly cancel already running kernels or free submitted copy buffers. A sudden cap below existing commitments requires stopped admission, orderly completion or supported recovery, and an honest report of the temporary violation.

Avoid a feedback loop where pressure pauses the very eviction or prerequisite transfer needed for progress. Emergency eligibility and dependency urgency remain active. No-fit is a distinct state when the required resident working set cannot satisfy current limits.

Part 3 - Quantitative Prediction Before Execution

Let $M_t^{\text{accounted}}$ be the ledger's consistent demand measure and define $u_t = M_t^{\text{accounted}}/J_t$ for $J_t > 0$. Choose thresholds $0 < l < h < c < 1$ with declared entry/exit rules; absolute admission constraints still apply.

For $l = 0.65$, $h = 0.80$, and $c = 0.92$, manually replay

$$(0.60,\ 0.82,\ 0.78,\ 0.81,\ 0.64,\ 0.63,\ 0.95).$$

Use two accepted stable observations for pressure exit. Predict prefetch and concurrency decisions at each point. Then halve the budget without changing live demand and explain the immediate state change.

Measure response latency from pressure observation to changed admission, and separately to realised memory reduction. Those times differ when existing work is still in flight.

Part 4 - Experimental Design and Instrumentation

1. Implement state transitions as a pure CPU function and check exact threshold equalities and invalid budgets.
2. Replay steady noise, slow pressure growth, a cap step down, and a prolonged no-fit state.
3. Map each state to explicit changes in optional prefetch, concurrency, victim search, and admission conservatism.
4. Run a CUDA workload under a cap below physical VRAM. Count cap/margin violations even when no OOM occurs.
5. Compare single-threshold and hysteretic policies on a held-out trace, including their throughput cost.

Log raw observations, any filtered values, state transitions, and the reason for emergency overrides.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class PressureState:
```

```
    state_name: str
    stable_low_count: int
    last_transition_time: float
    budget_epoch: int

@dataclass
class PressureThresholds:
    low: float
    high: float
    critical: float
    exit_patience: int

def classify_resource_pressure(runtime_state, ledger, budgets):
    observation = ...
    return observation

def transition_pressure_state(previous, observation, thresholds):
    next_state = ...
    return next_state

def select_state_dependent_actions(state, ready_graph, policy):
    actions = ...
    return actions

def audit_pressure_response(event_trace):
    metrics = ...
    return metrics

def E14(runtime, budget_trace, thresholds):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify valid threshold ordering, explicit zero-budget handling, and pressure-exit patience. Test that raw admission failure overrides a smoothed low utilisation reading.

Check no new discretionary work is admitted after critical pressure unless its contract allows it, while necessary release work can still progress. Distinguish cap reduction below existing commitments from a new-admission bookkeeping error. Report both violations and delayed progress.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each task or attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Pressure states provide early behavioural changes, not retroactive control of past allocations. Explain which signals gave useful warning and which changed too quickly for preventive action.

Recommend thresholds and patience for the measured workload with an explicit no-fit policy. Avoid treating one utilisation percentage as universally safe across shapes and allocators.

19.26.16 Exercise 15

Exercise Title: Dynamically Choose Between Larger Batches, Activation Checkpointing, CPU Offloading, and Reduced Precision from Current Memory Pressure

Part 1 - Exercise Contract and Why It Matters

Choose a configuration at logical update boundaries from physical batch sizes, checkpoint plans, saved-activation CPU offload, and permitted precision policies. Increase batch only when headroom and useful-throughput evidence justify it; use memory-saving alternatives when pressure rises. Deliver a candidate table, selection trace, and numerical validation.

Preserve the target logical batch through weighted accumulation. Apply precision and graph-construction changes only between committed logical windows. Placement changes also need a safe boundary. Only enable training offload mechanisms already proven compatible with autograd and optimizer state.

Part 2 - System and Memory Mental Model

The candidate actions affect different resources. Larger physical batches can improve kernel efficiency but raise activation demand. Checkpointing trades saved tensors for recomputation. Offload adds host residency and transfer pressure. Autocast can change selected intermediate storage and numerical error without halving all persistent state.

Combined gains are not additive because actions may target the same tensors. Each candidate requires compatible shape, backend, precision, and checkpoint/offload calibration. Transition costs and temporary duplicate buffers may exceed the steady-state peak of either configuration.

Apply changes after a committed update and safe retirement of transfers involving affected storage versions. Keep scaler state coherent and the scale fixed across an accumulation window. Numerical overflow and allocation pressure remain distinct outcomes.

Part 3 - Quantitative Prediction Before Execution

For candidate $c = (b, p, k, o)$, where b is physical batch, p precision, k checkpoint plan, and o offload policy, select useful throughput subject to

$$\begin{aligned} \widehat{M}_{\text{upper}}(c) + S_J &\le J, \\ \widehat{H}_{\text{upper}}(c) &\le H_{\max}, \\ E_{\text{numerical}}(c) &\le \varepsilon_{\max}. \end{aligned}$$

Check transition peak separately. For logical sample count B, partitions must satisfy $\sum_j b_j = B$, with weights b_j/B for sample-average losses. Valid-token objectives use their original valid-token denominator.

For $B = 100$, compare upper micro-batches 32 and 16 with the exact final remainder. Create at least four candidate profiles with measured or explicitly hypothetical GPU, host, transfer, and latency costs. Predict selections in normal and pressured states before running.

Part 4 - Experimental Design and Instrumentation

1. Establish a deterministic FP32 reference with a small model, batch-independent layers, and fixed sample IDs.

2. Calibrate a finite candidate set, including combinations, with complete training updates and lazy optimizer state.

3. Validate gradients, parameters, and buffers under each candidate's declared numerical tolerance.

4. Apply a changing budget trace. Select policies only at permitted update boundaries with safe transfer retirement.

5. Compare fixed configurations with the adaptive policy, counting calibration, transitions, retries, and host pressure.

6. Test unsupported dtype, insufficient host RAM, and no feasible candidate.

Use explicit checkpoint and AMP APIs appropriate to the installed version; do not infer capability from a policy name.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every `...` marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch
from torch.utils.checkpoint import checkpoint

@dataclass
class TrainingCandidate:
    micro_batch_size: int
    precision_name: str
    checkpoint_plan: str
    offload_policy: str
    profile_key: str
    quality_limit: float

def enumerate_supported_candidates(capabilities, policy_limits):
    candidates = ...
    return candidates
```

```
def validate_candidate_semantics(candidate, logical_objective):
    validation = ...
    return validation

def estimate_candidate_and_transition_costs(current, candidates):
    estimates = ...
    return estimates

def choose_boundary_configuration(estimates, resources, pressure):
    decision = ...
    return decision

def apply_configuration_at_safe_boundary(runtime, decision):
    transition_record = ...
    return transition_record

def E15(runtime, logical_windows, budget_trace, candidate_set):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Require exact logical sample coverage and accepted update counts, plus numerical agreement under declared criteria. Check that unsupported precision is rejected explicitly and that offload respects host capacity.

Verify no policy changes within a live graph or partial accumulation window. Test a transition whose temporary peak exceeds both steady profiles. Report cases where a larger batch fits but lowers accepted throughput.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each task or attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Memory pressure narrows the feasible configuration set; it does not identify the fastest correct policy by itself. Explain the measured compute, transfer, and numerical tradeoffs behind each selected configuration.

Recommend bounded, profile-aware adaptation with a conservative fallback. Retain the distinction between changing execution and changing the learning objective.

19.26.17 Exercise 16

Exercise Title: Build a Cost Function That Balances GPU Utilization, Peak VRAM, PCIe Traffic, Recomputation, and Total Step Latency

Part 1 - Exercise Contract and Why It Matters

Design and evaluate a dimensionally consistent scheduling cost function that compares accepted work, step latency, GPU memory, transfer traffic, and recomputation. Keep memory, host-resource, dependency, and numerical constraints as hard eligibility conditions. Deliver a candidate metric table, normalisation choices, sensitivity analysis, and selection on held-out workloads.

Do not allow a favourable weighted score to excuse an OOM, wrong output, or invalid dependency. Compare schedules that perform the same logical work under the same observation scope before ranking their resource tradeoffs.

Part 2 - System and Memory Mental Model

Raw bytes, seconds, and utilisation percentages cannot be added meaningfully without normalisation or explicit conversion coefficients. Reference scales should be fixed from calibration or a declared baseline, not recomputed per candidate so the objective changes while ranking.

GPU busy time is not necessarily useful compute. Copies, recomputation, and failed work can increase activity while accepted throughput falls. A union of useful-compute event intervals gives a duty fraction, not hardware occupancy or SM efficiency. State the exact metric and measurement tool.

Step wall time already includes effects of overlap, contention, and recomputation. Additional cost terms may encode resource preferences, but are not extra independent seconds to add to an already measured critical path. Discuss correlation and avoid claiming that summing all task durations equals concurrent makespan.

Weights reflect operating priorities. Use a Pareto view alongside a weighted score to show when conclusions depend strongly on arbitrary coefficients.

Part 3 - Quantitative Prediction Before Execution

For feasible candidate c, an illustrative score is

$$C(c) = w_T\, T(c)/T_{\text{ref}} + w_M\, M_{\text{peak}}(c)/J \\ + w_P\, V_{\text{PCIe}}(c)/V_{\text{ref}} + w_R\, T_{\text{recompute}}(c)/T_{\text{ref}} - w_U\, U_{\text{useful}}(c).$$

All terms are dimensionless after the declared normalisation. Choose nonnegative weights and explain the subtraction of a bounded useful-duty metric. Do not double-count transfer directions when V_{PCIe} already includes both.

Construct three hypothetical candidates: one with low latency/high VRAM, one with lower VRAM/more PCIe traffic, and one with lower traffic/more recomputation. Supply their values and reference scales before choosing weights. Compute rankings under latency-focused and transfer-limited preferences, then identify dominated candidates.

For useful-compute intervals I_i, define $U_{\text{useful}} = |\bigcup_i I_i|/T$; summing overlapping interval lengths can incorrectly produce a value above one.

Part 4 - Experimental Design and Instrumentation

1. Define metrics, units, scope, and hard constraints. Freeze reference scales using a calibration workload.

2. Measure complete schedules including submission overhead, policy transitions, failed work, and recovery.

3. Attribute recomputation and transfer volume in separate diagnostic runs when needed. State attribution uncertainty.

4. Rank a finite feasible candidate set under several declared weight choices and plot the tradeoffs.

5. Select weights on calibration traces and evaluate the chosen policy on held-out shapes and pressure regimes.

6. Compare with minimising full-step wall time alone. Explain whether extra terms improve the intended operating goal.

Keep accepted samples or useful tokens as an independent throughput measure so a high activity score cannot hide less useful work.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every `...` marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class ScheduleMetrics:
    wall_seconds: float
    peak_gpu_bytes: int
    pcie_bytes: int
    recomputation_seconds: float
    useful_compute_duty: float
    accepted_work: int
    constraints_satisfied: bool

def validate_metric_scopes(metrics, measurement_contract):
    validation = ...
    return validation

def normalise_metrics(metrics, fixed_reference_scales):
    normalised = ...
    return normalised

def evaluate_schedule_cost(normalised_metrics, weights):
    cost = ...
```

```
    return cost

def identify_pareto_candidates(candidate_metrics):
    frontier = ...
    return frontier

def test_weight_sensitivity(candidates, weight_sets, held_out):
    analysis = ...
    return analysis

def E16(runtime, candidate_policies, calibration_and_test_traces):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Reject invalid reference scales, mixed units, and scores assigned to infeasible candidates as if they were eligible. Check useful-duty values remain within their defined range.

Verify rankings against hand-computed cases and report sensitivity to weights and measurement noise. Compare full-wall-time throughput across equal logical work. Do not claim a global optimum outside the tested candidate set or a hardware-occupancy improvement from a duty metric.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each task or attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

A cost function makes preferences explicit; it does not remove judgement. Explain which resource tradeoff the chosen weights express and which constraints remain non-negotiable.

Recommend a policy only when its advantage survives reasonable uncertainty or explain the tradeoff when no candidate is uniformly best.

19.26.18 Exercise 17

Exercise Title: Reproduce Scheduler Thrashing Caused by Rapidly Alternating Prefetch, Eviction, and Batch-Size Decisions and Stabilize It with Hysteresis

Part 1 - Exercise Contract and Why It Matters

Create a reproducible feedback loop in which speculative prefetch consumes headroom, eviction removes soon-needed data, reload consumes bandwidth, and batch adaptation reverses earlier decisions. Stabilise it with coordinated hysteresis, minimum residency where safe, action cooldown, and failure memory. Deliver causal traces and net throughput comparisons.

Differentiate avoidable policy churn from legitimate responses to changing budgets or versions. The controller must still react promptly to critical pressure and cannot protect a minimum residency interval at the expense of correctness or required memory constraints.

Part 2 - System and Memory Mental Model

Independent local controllers can work against each other. A prefetcher may regard free bytes after eviction as spare capacity and immediately refill them; a batch controller may then increase compute demand and trigger another eviction. Each decision seems locally plausible while the combined schedule repeatedly transfers the same data.

Use one shared state and coherent action priorities. Record why data was evicted, which future consumer still needs it, and what evidence justifies refetching it. Cooldown can suppress speculative reversals, while a dependency-critical reload must remain eligible.

Minimum residency and reuse penalties are soft policy rules constrained by safety. Emergency pressure may override them for eligible victims, but lifetime guards cannot be overridden. A cancelled unsent prefetch and retirement of an in-flight copy are still different actions.

Part 3 - Quantitative Prediction Before Execution

For storage version v, count prefetch/eviction cycles before accepted reuse. Define excess traffic relative to a declared necessary-transfer baseline:

$$V_{\text{excess}} = V_{\text{observed}} - V_{\text{necessary}}, \qquad R_{\text{churn}} = \frac{N_{\text{avoidable reversals}}}{N_{\text{accepted updates}}}.$$

Explain how the baseline and avoidability label are derived; do not mark every repeat transfer as unnecessary.

Construct a trace with two 512 MiB prefetched objects and a compute working-set increase that repeatedly displaces one. Calculate H2D/D2H volume for three cycles under a stated clean-copy or dirty-offload policy.

Predict the effect of separate low/high thresholds, four-update growth patience, and a residency period longer than one ordinary compute interval. Then introduce a sudden critical budget drop and identify valid overrides.

Part 4 - Experimental Design and Instrumentation

1. Build a CPU scenario exposing the three-controller feedback loop and record the decision reason at every step.

2. Compare independent controllers, shared hysteresis, and shared hysteresis with bounded residency/cooldown.

3. Freeze policies before a held-out alternating-shape trace so thresholds are not fitted to one cycle.

4. Run a CUDA workload with versioned managed data and measure transfers, compute stalls, memory, and accepted throughput over the whole trace.

5. Add critical pressure and a newly urgent dependency. Verify stabilisation does not block required progress.

Record action duration and graph/placement transition costs; counting switches alone misses their severity.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate

measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class StorageActionHistory:
    storage_id: str
    version: int
    last_prefetch_time: object
    last_eviction_time: object
    next_required_use: object
    prior_action_reason: str

@dataclass
class UnifiedControlState:
    pressure_state: str
    growth_patience: int
    action_cooldowns: object
    storage_history: object
    current_batch: int

def identify_feedback_reversals(action_trace, dependency_trace):
    reversals = ...
    return reversals

def coordinate_controller_actions(state, proposals, resources):
    decision = ...
    return decision

def apply_hysteresis_and_residency_rules(state, decision, policy):
    filtered_decision = ...
    return filtered_decision

def quantify_excess_traffic(trace, necessary_transfer_baseline):
    metrics = ...
    return metrics

def E17(runtime, thrashing_scenarios, control_policies):
```

```
report = ...
return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify shared state prevents contradictory simultaneous decisions and records any safety override. Check that cooldown cannot permit stale-version use or premature storage release.

Measure excess traffic, lost compute time, batch switches, memory violations, and recovery delay. Include a trace where frequent switching is legitimately required. Report throughput gains only after including all transfer and transition work.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each task or attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Thrashing is often a coordination failure among reasonable local policies. Explain the feedback path that created the observed repeated work and how the shared policy changed it.

Recommend stabilisation rules with explicit urgent exceptions. Wider hysteresis is not useful if it merely delays necessary work or hides an incorrect admission model.

19.26.19 Exercise 18

Exercise Title: Stress-Test the Scheduler with Variable Sequence Lengths, Random Memory Spikes, Concurrent Transfers, and Artificially Changing VRAM Budgets

Part 1 - Exercise Contract and Why It Matters

Build a seeded stress suite that exercises memory admission, dependencies, reservations, transfers, eviction, pressure states, and recovery together. Include variable lengths, temporary workspace spikes, concurrent copy activity, budget changes, stale observations, and delayed completion. Deliver scenario manifests, invariant audits, complete traces, and minimal reproducers for failures.

Use deterministic CPU event simulation for rare interleavings and representative CUDA integration tests for actual allocation and transfer behaviour. Clearly distinguish synthetic events from measured device failures.

Part 2 - System and Memory Mental Model

Dynamic shape changes demand; external pressure changes availability; concurrent transfers extend source and destination lifetimes. A budget drop can arrive while valid old commitments are still active. These mechanisms need different diagnoses and cannot all be labelled a reservation bug.

Stress completion ordering as well as resource size. Duplicate callbacks, stale generation IDs, failed producers, and delayed D2H readiness can corrupt a scheduler without generating OOM. Test such logic in simulation rather than deliberately racing GPU buffer reuse.

Recovery must not restore mutable state while submitted tasks still write to it. Stop new dependent admission, resolve managed in-flight work according to the supported context state,

and only then restore a logical transaction. Known context-fatal failures require a stopped process with preserved CPU evidence, not speculative new CUDA work.

Part 3 - Quantitative Prediction Before Execution

Represent a scenario by

$$\mathcal{S} = \{(u_i, p_i, e_i, m_i, g_i)\},$$

where u is logical update or scheduler epoch, p is phase, e event type, m magnitude/shape, and g the affected generation. Preserve the realised schedule, not only its random seed.

Define invariant violations by category: dependency, version, lifetime, reservation, budget, and accepted-work accounting. For each report count and first failing event. Measure retry amplification

$$A_{\text{retry}} = \frac{N_{\text{attempts}}}{N_{\text{accepted windows}}},$$

with explicit no-progress handling when the denominator is zero.

Predict behaviour for a length doubling, an extra 256 MiB workspace, a delayed copy completion, and a budget below current resident state. Distinguish preventable admission failure from an unavoidable mid-task cap reduction under the declared contract.

Part 4 - Experimental Design and Instrumentation

1. Define at least ten bounded scenarios covering normal work, shape spikes, workspace spikes, transfer contention, budget drops, stale observations, duplicate completion, failed producers, no-fit, and recovery.

2. Use a seeded CPU simulator to explore event order, including two simultaneous reservation requests.

3. Run representative CUDA cases with a controlled pressure worker on the same physical GPU, bounded by an explicit byte ceiling and cleanup protocol.

4. Audit every state transition and storage version. Compare accepted outputs/updates with a compatible serial reference.

5. Record host memory, diagnostic overhead, transfer volume, peak demand, waits, refusals, and completion status for all scenarios.

6. Minimise a failing schedule while preserving the original trace and environment metadata.

Terminate only experiment-created workers. Never clear unrelated application tensors to force a stress case to succeed.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import random
import torch

@dataclass
class SchedulerStressEvent:
    epoch: int
    phase: str
    event_kind: str
    magnitude: object
    target_generation: object
    evidence_mode: str

def generate_bounded_scheduler_scenario(seed, workload, limits):
    scenario = ...
    return scenario

def replay_event_interleavings(simulator, scenario):
    traces = ...
    return traces

def execute_cuda_stress(runtime_factory, scenario, limits):
    result = ...
    return result

def audit_scheduler_invariants(trace, reference_manifest):
    violations = ...
    return violations

def reduce_failing_scenario(scenario, reproducible_replay):
    reduced = ...
    return reduced

def E18(runtime_factory, scenario_seeds, stress_limits):
    report = ...
    return report
```

Part 6 - Measurement, Validation, and Failure Analysis

Require correct dependencies, data versions, lifetime guards, atomic reservation accounting, and exact accepted sample/task coverage. Verify event duplication cannot refund memory twice.

Test stalled in-flight work, singleton no-fit, insufficient host staging, and exceptions during recovery. Preserve terminal outcomes and distinguish bounded deferral from deadlock. Check

cleanup returns managed live storage to the expected baseline, allowing documented allocator cache.

Report the tested envelope and sampling limits. Passing random scenarios does not prove safety against arbitrary external allocation or hidden operator workspaces.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each task or attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

An integrated scheduler must remain correct across interacting disturbances, not just perform well in steady state. Explain the first broken invariant in each failing scenario.

Prioritise fixes by correctness and lost accepted work. Keep negative cases as evidence of the operating boundary rather than removing them from the benchmark.

19.26.20 Exercise 19

Exercise Title: Build and Benchmark a Unified PyTorch Memory-Aware Scheduler That Coordinates Compute, Prefetch, Placement, Eviction, Offloading, and OOM Prevention Under Dynamic Memory Budgets

Part 1 - Exercise Contract and Why It Matters

Integrate the chapter into a unified scheduler with task dependencies, versioned placement, resource profiles, atomic reservations, compute/transfer priorities, prefetch control, eligible eviction, pressure states, configuration selection, and bounded OOM recovery. Deliver the learner-completed runtime, configuration schema, raw event logs, invariant audits, and benchmark comparison against simpler policies.

Use two explicit workload modes. In inference, manage immutable CPU-backed parameter copies and input/output placement. In training, retain optimizer-bound parameters and manage supported input, saved-activation offload, checkpoint, precision, and micro-batch policies. Parameter migration during training is a separate extension requiring proven gradient and version semantics.

Part 2 - System and Memory Mental Model

The task graph specifies required data versions and legal execution order. The placement registry records where valid copies live and which readers own them. The reservation ledger accounts for future additional demand. The policy ranks eligible work, and the executor submits calls with explicit stream dependencies and lifetime protection.

Every task progresses through planned, dependency-ready, reserved, submitted, completed, and retired states, with explicit failure/cancellation paths. A completed task may retain outputs; a retired reservation does not imply every related tensor is gone. Reconcile ownership and commitments using one consistent accounting model.

For training, a logical window is the unit of commit. Keep its sample IDs and loss denominator fixed, and perform the optimizer update only after all required gradient work completes. Avoid overlapping independent mutable optimizer updates in the base design. Restore a failed transaction only after safely resolving work that could still access or modify its storage.

Prevention means refusing predicted-inadmissible work and preserving margins inside a declared envelope. It cannot guarantee zero OOM against arbitrary external allocation or unbounded prediction error. Evaluate coordinated budget changes separately from asynchronous external pressure, and retain DMM23's terminal handling for context-fatal errors.

Part 3 - Quantitative Prediction Before Execution

For candidate schedule policy π, optimise accepted useful throughput over a finite validated policy set:

$$\pi^* = \arg\max_{\pi \in \Pi} \frac{N_{\text{accepted}}(\pi)}{T_{\text{full wall}}(\pi)}.$$

Use E16's cost function only when additional resource preferences are required and keep hard constraints:

$$\begin{aligned} M_{\text{accounted}}(t) + S_J(t) &\le J(t), \\ H_{\text{accounted}}(t) &\le H_{\max}(t), \\ \text{dependencies valid}(i) &= \text{true}, \\ E_{\text{numerical}}(\pi) &\le \varepsilon_{\max}. \end{aligned}$$

Also apply a conservative fresh device-availability check for additional allocations. Distinguish predicted admission constraints from post-execution observed peak audits.

Prepare a pre-run table for serial demand loading, first-fit, priority with fixed prefetch, and the unified policy. Predict peak GPU/host bytes, transfer volume, exposed stalls, accepted throughput, and recovery overhead. Include a case where scheduling overhead may make the simple baseline fastest.

Part 4 - Experimental Design and Instrumentation

1. Implement independent interfaces for the task DAG, storage registry, telemetry, cost profiles, reservation ledger, policy, stream executor, and incident recorder.
2. Validate CPU event transitions and ownership invariants before enabling CUDA overlap.
3. Establish serial references for inference and training separately. Check outputs, logical samples, gradients, parameters, buffers, and optimizer state where applicable.
4. Compare the four baselines under identical work, resource caps, candidate policies, and observation scope. Give each policy the same legal lookahead.
5. Evaluate steady shapes, variable lengths, coordinated budget changes, transfer contention, and asynchronous pressure. Use held-out traces after policy calibration.
6. Include initialization, calibration, pinned buffer preparation, policy switching, failed attempts, diagnostics, and recovery in total elapsed time.
7. Export accepted work, latency distribution, memory peaks, commitments, PCIe bytes, recomputation, queue wait, starvation, eviction/reload cycles, OOMs, margin violations, and terminal outcomes.

Record simulation and CUDA measurements separately. An offline future-aware schedule is a comparison bound, not evidence of the online scheduler's achievable policy.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier.

Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable training implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch

@dataclass
class SchedulerLimits:
    gpu_budget_bytes: int
    gpu_margin_bytes: int
    host_budget_bytes: int
    pinned_pool_limit_bytes: int
    maximum_inflight_tasks: int
    maximum_retries: int

@dataclass
class SchedulerOutcome:
    logical_window_id: str
    accepted_work: int
    completed_task_ids: tuple
    failed_task_ids: tuple
    terminal_reason: object

class UnifiedMemoryAwareScheduler:
    def __init__(self, workload, profiles, policy, limits):
        self.workload = workload
        self.profiles = profiles
        self.policy = policy
        self.limits = limits
        self.task_graph = ...
        self.storage_registry = ...
        self.reservation_ledger = ...
        self.streams = ...
        self.event_log = ...

    def observe_and_reconcile(self):
        runtime_state = ...
        return runtime_state
```

```
    def identify_eligible_tasks(self, runtime_state):
        candidates = ...
        return candidates

    def choose_compute_or_movement(self, candidates, state):
        decision = ...
        return decision

    def reserve_and_submit(self, task, state):
        # Atomically reserve, then establish waits and lifetimes.
        handle = ...
        return handle

    def process_completions(self):
        # Publish valid outputs and retain required storage.
        completions = ...
        return completions

    def handle_pressure_or_failure(self, incident):
        # Stop unsafe admissions and preserve in-flight ownership.
        decision = ...
        return decision

    def commit_logical_window(self, window):
        # Validate all required work and advance accepted state once.
        outcome = ...
        return outcome

    def run(self, cpu_work_source):
        report = ...
        return report

    def export_evidence(self, destination):
        ...

def compare_scheduler_policies(factories, workload_traces):
    comparison = ...
    return comparison

def E19(workload, profiles, policy, limits, held_out_traces):
    runtime = ...
    benchmark_report = ...
    return benchmark_report
```

Part 6 - Measurement, Validation, and Failure Analysis

Acceptance requires valid dependencies and versions, protected source/destination lifetimes, no double-spent reservations, correct retained-output accounting, and bounded no-fit/recovery behaviour. Audit all of these at each transition, not only after the final output.

For inference, compare outputs with the resident reference. For training, verify logical sample coverage, loss weights, gradients, buffers, parameters, optimizer, scaler, and scheduler progress under declared tolerances. Test a delayed producer, stale version, duplicate completion, budget drop, unavailable host staging, and failure with other tasks in flight.

Distinguish application-margin violations from physical OOM and external pressure from accounting bugs. Report unsupported cases and all failed trials. Count accepted useful work once, excluding retries, redundant prefetch, and discarded speculative work. Performance improvement is a measured outcome, not a prerequisite for reporting a correct negative result.
Evidence format. Keep prediction, observation, and interpretation in separate columns. Each task or attempt needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

The unified scheduler's value is coordinated progress: the right data version is available at the right time, memory promises remain consistent, and completed work preserves the intended computation.

Conclude with the strongest supported claim for the tested devices, allocators, workload modes, shapes, budgets, and policy set. Explain where simple serial scheduling remains preferable and where coordination earns its overhead. State the limits of OOM prevention and the conditions requiring deferral, rollback, or a fresh process.

19.27 DMM25

Chapter Title: Memory Pooling and Buffer Reuse
Specialization: Dynamic Memory Management and CUDA Systems

19.27.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Memory Pools, Reusable Buffers, Allocation Amortization, and Why Reuse Reduces Runtime Overhead

Part 1 - Exercise Contract and Why It Matters

Build a vocabulary and an accounting model before writing a pool. Your deliverables are a storage-lifetime diagram, an amortized-cost prediction, and a controlled comparison of repeated requests with retained-buffer reuse. Distinguish a tensor object, its underlying storage, an application-owned buffer, and a block managed by the PyTorch CUDA allocator. A successful exercise explains both a possible speedup and a workload where a custom pool loses. Repeated allocation requests are not evidence of repeated driver allocations.

Limit the initial experiment to contiguous tensors with one dtype on one CUDA device, outside autograd. Specify the requested element count, initialization rule, and maximum simultaneous borrowers. You must account for data initialization in both paths, preserve equal numerical work, and return a buffer only after its declared use ends. Do not begin by trying to replace internal cuBLAS workspaces or PyTorch's allocator.

Part 2 - System and Memory Mental Model

An application pool keeps tensor storage alive and lends it repeatedly. The caching allocator operates below this layer and can already recycle storage when ordinary tensors die. Thus a pool may reduce Python object creation, allocator bookkeeping, or host pinning calls, while adding lookup, ownership, and synchronization costs of its own. The name `torch.cuda.MemPool` refers to a separate allocator API; it is not the Python lease manager constructed in this chapter.

Use three disjoint capacity states: leased, pending completion, and free for application reuse. Their sum is retained pool capacity. Free pool buffers are still live tensors, so they contribute to allocated memory; allocator-reserved memory is a different level of accounting. A logical view can request fewer bytes than its backing buffer owns. That difference is internal waste, whereas an entirely idle buffer is retained idle capacity. Never count several views of one storage as separate backing allocations.

Uninitialized storage and reused storage both require a defined write before a read. Python scope ending does not prove that GPU work finished, and a returned tensor view can keep storage alive. The first implementation therefore uses a strict borrower contract and synchronized release. Later exercises relax synchronization through completion events, not through assumptions about the speed of the GPU.

Part 3 - Quantitative Prediction Before Execution

For request count N, new-storage count K, acquisition cost t_q, storage-creation cost t_a, release cost t_r, and useful work t_w, use the following explicitly simplified serial model:

$$T_{\text{pool}} \approx Kt_a + N(t_q + t_r + t_w), \qquad \bar{t}_{\text{creation}} = \frac{Kt_a}{N}.$$

Compare it with a measured baseline of warmed PyTorch allocations. Raw `cudaMalloc` latency is not that baseline. The model omits overlap, contention, variable allocation sizes, and occasional trimming.

Let c_j be unique buffer capacity and q_j the requested bytes of an active lease. Predict

$$C_{\text{pool}} = \sum_j c_j, \qquad W_{\text{live}} = \sum_{j \in \mathcal{L}} (c_j - q_j).$$

For a hypothetical request of 2^{20} float32 elements and three simultaneous borrowers, calculate the minimum backing capacity in MiB. Then predict what changes if one borrower releases while its GPU work is pending. State separately whether the pool can lend that capacity and whether the process still owns it. Use these predictions as hypotheses, not as measured GPU results.

Part 4 - Experimental Design and Instrumentation

1. Draw ownership timelines for ordinary allocation, synchronous reuse, and an attempted early release. Mark host return and device completion as separate instants.

2. Run a fixed-size request trace with one outstanding borrower, then repeat with three. Use the same initialization and checksum operation for the ordinary and pooled paths.

3. Sample allocated and reserved bytes before construction, after first acquisition, after safe release, and after dropping every pool reference. Record driver-visible free memory separately.

4. Measure a cold run and several warmed runs. Include pool construction in startup cost and exclude it only in a clearly labeled steady-state view.

5. Repeat with tiny and moderately sized buffers. Explain whether bookkeeping, initialization bandwidth, or useful computation dominates.

Keep a storage registry containing scalar IDs and sizes; evidence logging must not hold tensors. A data pointer may help explain a trace but cannot serve as a permanent identity across deallocation and reallocation.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import time
import torch


@dataclass
class BufferRecord:
    buffer_id: int
    capacity_bytes: int
    state: str
    generation: int


def describe_storage(tensor):
    # Report logical bytes and unique backing-storage metadata.
    ...


def predict_costs(request_count, storage_count, cost_terms):
    # Declare units and the assumptions of the serial model.
    ...


def trace_ordinary_requests(spec, repetitions):
    # Allocate, initialize, consume, and release explicitly.
    ...
```

```
def trace_retained_reuse(spec, repetitions):
    # Keep storage alive and enforce one borrower per buffer.
    ...


def E00(config):
    # Record predictions before running either experiment.
    ...
    # Compare cold and warm costs using identical useful work.
    ...
    # Export scalar evidence and an ownership timeline.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Your report must reconcile requested bytes, backing capacity, allocated bytes, and reserved bytes without treating them as interchangeable. Show a row in which all leases have ended but retained capacity remains nonzero. Demonstrate that the contents of a previously used buffer are not a valid initialization guarantee. Validate checksums only after the corresponding CUDA work completes.

Acceptance requires one explanation of overhead amortization and one counterexample to unconditional speedup. A result dominated by initialization is valid if initialization was equivalent. Reject a comparison where the ordinary path allocates and fills but the pooled path only returns a tensor. Inspect lingering aliases when allocated memory fails to fall after pool destruction. Label allocator caching as a possible reason reserved memory does not fall with allocated memory.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Pooling exchanges recurring management work for retained resources and stronger lifetime obligations. A fast pointer lookup is only a small part of that exchange. The useful engineering question is whether explicit reuse improves the complete workload under a declared capacity limit.

Connect this model to DMM24: a scheduler needs to know when memory becomes reusable, not merely when a host function returns. Explain which state transitions your initial pool can prove and which require the later event protocol. Finish with a falsifiable statement describing when you expect pooling to help, including the request size distribution and concurrency assumptions.

19.27.2 Exercise 01

Exercise Title: Measure the Cost of Repeated CUDA Tensor Allocation and Deallocation in a Tight PyTorch Loop

Part 1 - Exercise Contract and Why It Matters

Establish an honest allocation baseline before claiming that reuse is faster. Produce request-latency distributions and complete-loop timings for repeated fixed-size CUDA tensors. Separate Python request overhead, initialization kernels, allocator cache reuse, and device allocation activity. The deliverable is a measurement protocol that another learner can replay without accidentally timing only queued GPU work.

Use several tensor sizes and two lifetime patterns: immediate release and a bounded set of simultaneously live tensors. Keep the largest experiment comfortably within an explicit VRAM allowance. This exercise studies allocation behavior, not deliberate OOM. A cold-start observation and a warmed steady-state observation must be reported separately.

Part 2 - System and Memory Mental Model

`torch.empty` creates an uninitialized tensor; its call need not launch a data-filling kernel or obtain a fresh device allocation. Releasing the last Python reference normally returns ownership to the allocator, whose cache can retain device memory. The cost of repeated empty tensors therefore differs from the cost of repeated filled tensors and from a loop that purges the allocator cache every iteration.

Host timing around a complete synchronized loop captures Python and allocator work as well as GPU completion. CUDA event timing describes positions on a stream timeline; it is not a direct measurement of Python allocation overhead and can include device idle time induced by host submission. Per-iteration synchronization changes the workload and must be a separate diagnostic variant. An empty-loop control estimates timer and iteration overhead but should not be blindly subtracted when it is comparable to the signal.

Allocator counters have distinct meanings. The cumulative allocation counter counts allocator requests, while segment counters describe allocator-managed segments. Available fields and interpretation depend on the backend. Record the backend and supported keys; do not rename a request counter as a driver allocation counter. Native-backend inactive split statistics are not a portable fragmentation metric for every backend.

Part 3 - Quantitative Prediction Before Execution

Let T_N be elapsed wall time for N requests and T_0 the matched empty-loop time. Report both

$$\bar{t}_{\text{raw}} = \frac{T_N}{N}, \qquad \bar{t}_{\text{adjusted}} = \frac{T_N - T_0}{N},$$

with uncertainty from repeated complete runs. If the adjusted estimate is unstable or negative, report insufficient resolution rather than claiming negative allocation cost.

For L simultaneous tensors of n elements and element size s, predict logical live bytes Lns. For a hypothetical 256 KiB request, calculate the expected logical increase when the live set changes from one tensor to 32. Predict why reserved bytes can exceed this amount. Before execution, state which cases should create new allocator segments, which may reuse cached capacity, and why those are hypotheses rather than guarantees about a specific allocator implementation.

Part 4 - Experimental Design and Instrumentation

1. Initialize CUDA outside the steady-state interval. Capture an optional fresh-process startup trace separately, with initialization costs identified.

2. Sweep small, medium, and large byte sizes using a fixed dtype. Generate shapes on CPU before timing and verify their byte counts.

3. Measure empty allocation and release, then allocation plus a full initialization write, using the same repetition counts and live-set limit.

4. Synchronize before and after each complete timed region. Collect per-request host timings only in a separate instrumented run to quantify the effect of timers and logging.

5. Record allocator statistics before and after each region. Warm each size intentionally; randomize comparison order across repeats.

6. Add a cache-purge diagnostic outside the main benchmark. If you also purge every iteration, label that a different allocator workload.

Hold CPU threads, background GPU load, dtype, and device constant. Report median and tail latency from sufficient samples, along with run-to-run variation. Do not present the minimum observation as the typical cost.

Useful measurement interfaces include:

- `time.perf_counter`: host wall-clock timing.
- `torch.cuda.synchronize`: a deliberate completion boundary.
- `torch.cuda.memory_stats`: allocator counter inspection.

Inspect the cumulative request key `allocation.all.allocated`. Inspect segment activity through `segment.all.allocated`. Check backend support before interpreting either counter. Use tensor `numel()` and `element_size()` to verify logical bytes, and storage-level size inspection when backing capacity differs.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every `...` marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
import time
import torch


def make_allocation_specs(byte_sizes, dtype):
    # Convert exact byte requests to supported tensor shapes.
    ...
```

```
def capture_allocator_counters(device):
    # Return scalar counters and identify unsupported fields.
    ...


def run_allocation_loop(spec, repeats, live_limit, initialize):
    live_tensors = []
    # Bound the live set and avoid tensor retention in logs.
    ...


def measure_region(operation, device):
    # Separate synchronized wall time from stream event time.
    ...


def E01(config):
    # Establish cold, warm, and empty-loop measurements.
    ...
    # Sweep sizes without changing unrelated conditions.
    ...
    # Export distributions and correctly named counter deltas.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report request size in bytes, simultaneous live count, repetitions, initialization mode, raw wall time, adjusted time, device elapsed time when meaningful, peak allocated bytes, and peak reserved bytes. Counter differences must use a matching measurement interval. Include missing-field status instead of silently replacing a missing counter with zero.

Check that tensor references do not accumulate in a list intended only for logging. Compare requested live bytes with observed allocated growth and explain allocator granularity. Investigate unexpectedly high variance with a fresh repeat and a smaller instrumentation footprint. A pointer reappearing proves address reuse in that trace, not a universal allocator policy. A near-zero CUDA event interval is not proof that allocation has no CPU cost.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

The baseline determines the meaning of every later speedup. If ordinary PyTorch allocation already reuses storage efficiently, a Python pool must earn its complexity by reducing other costs or controlling lifetimes and capacity more effectively.

Explain how the benchmark changes when the workload is dominated by large memory writes, by tiny requests, or by many live buffers. Identify one measurement that would distinguish host bookkeeping from additional device allocation activity. Carry the baseline protocol

forward unchanged so that later pooling improvements are attributable to a specific design choice.

19.27.3 Exercise 02

Exercise Title: Build a Fixed-Size CUDA Buffer Pool That Reuses Previously Allocated Tensors Instead of Allocating New Storage

Part 1 - Exercise Contract and Why It Matters

Construct the smallest pool with a defensible ownership contract: a fixed number of identical contiguous CUDA buffers and at most one active lease per buffer. Produce acquisition and release interfaces, a registry of buffer states, and an experiment demonstrating reuse without reallocation after construction. The learner must implement the ownership checks and all CUDA operations.

Choose a fixed capacity and an exhaustion policy before running: bounded waiting or an explicit rejection. Silent growth is outside this exercise. Use a synchronized, single-stream consumer so that release can prove device completion. Require the caller to initialize the requested region and forbid use of any returned tensor or view after lease release.

Part 2 - System and Memory Mental Model

A lease is a temporary right to use storage, not a new storage allocation. Store a buffer ID and a monotonically advancing generation with each lease. A release must match the current owner and generation. These fields detect double release and stale handles, but cannot revoke a raw tensor alias that a caller deliberately retains. State that limitation as an API contract; Python objects do not become invalid merely because a registry changes their status.

Preallocated free buffers remain live CUDA tensors. They consume the full pool capacity even when no borrower exists. Acquiring a buffer should change ownership metadata, not storage size, dtype, or device. Avoid operations such as growing `resize_` that can allocate new storage behind a supposedly fixed-capacity record.

Correctness requires two separate conditions: no other lease owns the buffer, and all prior uses have finished. In this initial design, the caller performs an explicit completion step before release. Later exercises introduce pending states and asynchronous reclamation. Autograd-managed activations are excluded because their lifetime can extend through backward even after a forward function has returned.

Part 3 - Quantitative Prediction Before Execution

For m buffers of n elements and item size s, predict

$$C = mns, \qquad 0 \le L(t) \le m, \qquad F(t) + L(t) = m$$

for the synchronized two-state pool. F denotes free buffers and L leased buffers; an invalid release must not alter either count.

Use a hypothetical pool of four buffers, each with 2^{18} float32 elements. Calculate total capacity, then sketch the state sequence for four acquisitions, one rejected fifth request, two releases, and two new acquisitions. Predict the number of backing allocations and the number of successful leases separately. Explain why using one buffer 100 times should increase lease count without increasing pool capacity. Write the expected effect of an old-generation release before testing it.

Part 4 - Experimental Design and Instrumentation

1. Define a typed configuration and validate positive dimensions, device, dtype, and buffer count before construction.

2. Allocate all backing tensors outside the warmed acquisition interval. Record scalar storage identities and allocated memory growth.

3. Acquire every buffer, write a distinct pattern to each, run a deterministic consumer, and validate results after synchronization.

4. Exercise exhaustion without allocating an extra buffer. Release selected leases and confirm only those buffers become eligible.

5. Attempt a double release and a stale-generation release using metadata-only misuse tests. They must leave the registry unchanged.

6. Replay a long sequence with no more than the configured concurrency. Measure acquisition latency and prove no backing storage is added.

Do not deliberately race stale aliases with GPU execution. Demonstrate the alias limitation through a safe sequential example and document the borrower's responsibility.
Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch


@dataclass(frozen=True)
class Lease:
    buffer_id: int
    generation: int
    tensor: torch.Tensor


class FixedBufferPool:
    def __init__(self, shape, dtype, device, buffer_count):
        # Validate configuration and establish fixed storage.
        ...
```

```
    def acquire(self):
        # Grant an exclusive lease or report exhaustion.
        ...

    def release(self, lease, completion_proof):
        # Validate ownership and completed use before release.
        ...

    def audit(self):
        # Check unique ownership and constant capacity.
        ...


def E02(config):
    # Build the pool and preserve scalar storage identities.
    ...
    # Exercise reuse, exhaustion, and invalid lease release.
    ...
    # Compare numerical results with independent tensors.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Acceptance requires constant backing capacity after construction, unique ownership for every live buffer, correct output patterns, and explicit exhaustion behavior. Show registry snapshots before and after each misuse test. Invalid operations must not increase the free count or allow two borrowers to receive the same storage.

Check both logical shape and backing storage size. A correct-shaped tensor with replaced storage violates the intended fixed-pool experiment. Delete all experimental leases before studying destruction; their tensor fields are strong references. If capacity appears to remain after deleting the pool, inspect these references rather than assuming allocator leakage. Separate the pool's retained capacity from allocator-reserved bytes.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

The fixed pool turns allocation into admission control. Exhaustion becomes an explicit condition the caller can wait for or handle, rather than unplanned growth. Its strongest advantage may be predictability rather than raw speed.

Explain the costs of preallocating for worst-case concurrency when typical concurrency is low. Identify what must change to support variable sizes and what must change to support asynchronous consumers. These are different extensions: size classes solve placement compatibility, while events solve completion. A reliable design keeps those obligations separate.

19.27.4 Exercise 03

Exercise Title: Compare Fresh Allocation Against Buffer Reuse by Latency, Peak VRAM, Reserved Memory, and Allocation Count

Part 1 - Exercise Contract and Why It Matters

Benchmark the fixed pool against ordinary per-request PyTorch allocation using identical useful work and request lifetimes. Deliver a comparison that distinguishes faster execution, lower memory consumption, and fewer management operations. None of these outcomes implies the others. The phrase fresh allocation means a new tensor request in this experiment, not necessarily a fresh driver allocation.

Use the E01 timing protocol and the E02 ownership contract. Include a startup-inclusive view and a warmed view, because preallocation moves cost rather than erasing it. Require numerical agreement before reporting a speedup. A pool that retains more memory or runs slower remains a valid experimental result.

Part 2 - System and Memory Mental Model

There are at least three relevant counters: application requests, new backing tensors created by the custom pool, and allocator activity. A high request count can coexist with few new allocator segments in the ordinary path. In the pooled path, a stable backing allocation count can coexist with substantial Python lease bookkeeping.

Peak allocated memory measures live tensor storage through PyTorch; peak reserved memory includes the allocator's retained segments. An application pool generally retains its free tensors, so its allocated baseline can be higher even if peak allocation churn is lower. Compare absolute peaks and increases above the mode's own baseline. Reporting only the increase can conceal a large preallocated pool.

The useful workload must initialize and consume the same bytes in each mode. If the consumer returns outputs, account for those separately and use identical output lifetimes. Pool capacity should match actual concurrency rather than the total number of requests. Cache purging, garbage collection, and synchronization inserted into only one path destroy the main comparison.

Part 3 - Quantitative Prediction Before Execution

For ordinary and pooled elapsed times T_o, T_p, define

$$S = \frac{T_o}{T_p}, \qquad \Delta M_{\text{peak}} = M_{p,\text{peak}} - M_{o,\text{peak}}.$$

Report uncertainty and both signs of the memory difference. If pool construction costs T_c and each completed request saves an assumed $\delta t > 0$, the simplified break-even request count is

$$N_{\text{break}} \approx \frac{T_c}{\delta t}.$$

This estimate is inapplicable when measured savings are nonpositive.

For a hypothetical construction cost of 2 ms and a hypothesized saving of 0.5 microseconds per request, predict the break-even scale. Then explain how a capacity of eight buffers can increase startup allocated memory relative to a trace with one simultaneous request. Record the prediction without presenting either hypothetical timing as a GPU measurement.

Part 4 - Experimental Design and Instrumentation

1. Build one request trace and replay it for both modes. Match byte sizes, simultaneous leases, initialization, consumer kernels, and outputs.

2. Use fresh processes for cold-start comparisons where practical. For warm comparisons, warm both modes and randomize their execution order.

3. Reset peak statistics at the same logical boundary. Sample mode baseline, complete-region peaks, and final allocated and reserved bytes.

4. Count requests explicitly. Count custom backing tensor creation inside the pool. Collect available allocator counter differences without claiming they all represent driver calls.

5. Repeat at multiple request sizes and concurrency levels. Compare medians, tail latencies, and total completed requests per second.

6. Run a minimally instrumented throughput pass and a detailed diagnostic pass to estimate logging overhead.

Keep pool construction and final cleanup visible in the startup-inclusive account. Use the same maximum in-flight work so overlap does not become an uncontrolled advantage for one mode.

Use these interfaces from `torch.cuda`:

- `memory_allocated` and `memory_reserved`: current allocator observations.
- `max_memory_allocated`: peak allocated bytes.
- `max_memory_reserved`: peak reserved bytes.
- `reset_peak_memory_stats`: reset at the declared boundary.
- `mem_get_info`: the separate device-free observation.

These interfaces provide different accounting views; their values must not be added together as disjoint memory components.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch
```

```
@dataclass
class TrialResult:
    mode: str
    request_count: int
    backing_creations: int
    elapsed_seconds: float
    peak_allocated_bytes: int
    peak_reserved_bytes: int


def replay_trace(trace, allocation_mode, consumer):
    # Match initialization, concurrency, and useful work.
    ...


def measure_trial(trace, allocation_mode, config):
    # Capture baseline, peaks, completion time, and counters.
    ...


def compare_trials(ordinary_rows, pooled_rows):
    # Calculate uncertainty and distinguish counter meanings.
    ...


def E03(config):
    # Validate numerical equivalence before timing.
    ...
    # Run cold and warm comparisons in balanced order.
    ...
    # Explain memory and latency results independently.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Provide columns for ordinary request count, custom backing creations, supported allocator counters, startup time, warmed time, throughput, absolute allocated peak, allocated increase, and reserved peak. Use not-applicable for a custom-pool counter in the ordinary mode instead of disguising it as zero storage activity.

Check output equivalence with dtype-appropriate tolerances declared before measurements; simple integer-pattern copy checks should be exact. Investigate suspiciously large speedups by confirming initialization and synchronization. Reject any run with differing completed work counts. Show at least one memory-versus-latency tradeoff, or report that none was observed within the measured range. Do not infer reduced allocator fragmentation from lower application request count alone.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use

an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Performance claims need a clearly defined counterfactual. Ordinary PyTorch allocation already benefits from caching, so the comparison measures the incremental value of an application pool over that existing machinery.

Use the results to decide whether a custom pool is justified for the tested workload. A small latency gain can be undesirable when it reserves capacity needed by a model, while a bounded pool can be valuable even without a speedup. State which constraint dominates your decision and what additional workload would most likely overturn it.

19.27.5 Exercise 04

Exercise Title: Implement a Pool Free List That Returns Released CUDA Buffers to Reusable Size Classes

Part 1 - Exercise Contract and Why It Matters

Extend the fixed pool into a registry of compatible size classes, each with a free list. Produce interfaces that distinguish requested shape, backing capacity, and lease identity. Demonstrate reuse across requests in one class without mixing incompatible dtypes or devices. The core deliverable is a set of invariants that remains true across acquisition, release, exhaustion, and invalid requests.

Keep the first implementation single-stream with synchronized release. Use typed contiguous one-dimensional backing tensors and shape views whose logical element count does not exceed capacity. Support a documented subset of layouts rather than claiming arbitrary-stride compatibility. Do not combine separate storages into one larger contiguous buffer.

Part 2 - System and Memory Mental Model

A free list is an index of eligible buffers, not the owner of a second copy of their storage. Maintain a canonical record per buffer; free-list entries refer to its ID. A record must appear in exactly one eligible list or be leased, never both. A release reestablishes eligibility only after ownership and completion checks pass.

The class key needs device identity, dtype, and a declared capacity unit. If the initial pool supports only contiguous tensors with no special alignment promise, say so. A float16 capacity in elements is not equivalent to the same float32 element count in bytes. Shape can be a view-level property when the storage and contiguous layout are compatible.

Returning a tensor view does not transfer the underlying allocation away from the pool. Every alias remains subject to the lease contract. An old handle with a reused buffer ID is rejected using its generation. When no eligible buffer exists, growing the pool creates a new record only after successful allocation; failed growth must not leave a ghost free-list entry. Pending asynchronous buffers will need a separate index in E11 and must never be counted as ready here.

Part 3 - Quantitative Prediction Before Execution

For class k, let n_k be its buffer count and c_k its capacity in bytes. Predict

$$C = \sum_k n_k c_k, \qquad n_k = f_k + \ell_k$$

for this synchronized design. Define a hit as finding an eligible compatible buffer at the first lookup; a later successful growth is a miss followed by creation.

Construct a hypothetical trace with class capacities 1 MiB and 2 MiB. Acquire two buffers from the first class, release one, acquire from the second class, then request the first class again. Predict list lengths, leased counts, total capacity, and hit status at each operation. Repeat the reasoning with the same element count in a different dtype. Explain which property prevents an incorrect cross-class reuse.

Part 4 - Experimental Design and Instrumentation

1. Write the compatibility key specification before coding. Reject unsupported layouts and nonintegral byte-to-element conversions.

2. Implement canonical records and free-list membership checks. Record each state transition using IDs and integer counts.

3. Replay a short hand-computable trace, validating the registry after every operation. Compare the trace against your Part 3 ledger.

4. Repeat with multiple shapes that share capacity and with dtypes that must remain in separate classes. Fully initialize each logical view.

5. Exercise double release, stale generation, wrong pool identity, and an allocation failure simulated before registry insertion.

6. Run a longer bounded trace and inspect acquisition latency as the number of classes grows.

Keep failure injection at a defined interface so metadata rollback can be tested without destabilizing the CUDA context. Use ordinary reference tensors to validate each supported view's result.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every `...` marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch
```

```
@dataclass(frozen=True)
class ClassKey:
    device: str
    dtype: torch.dtype
    capacity_elements: int


class ClassedPool:
    def __init__(self, class_policy, device):
        # Establish canonical records and per-class indexes.
        ...

    def classify(self, shape, dtype):
        # Validate layout and derive a compatible key.
        ...

    def acquire(self, shape, dtype):
        # Remove one eligible ID or perform controlled growth.
        ...

    def release(self, lease, completion_proof):
        # Validate generation and restore unique membership.
        ...

    def audit_membership(self):
        # Reconcile every free-list entry with its record.
        ...


def E04(config):
    # Replay a deterministic trace and failure cases.
    ...
    # Validate view contents, membership, and byte accounting.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Acceptance requires one canonical record for every retained backing storage, no duplicate free-list entries, no leased record in a free list, and matching class capacity and dtype. Show the ledger before and after every rejected operation. Rejection must not consume a valid lease or increase available capacity.

Measure class count, successful acquisitions, initial hits, growth attempts, successful creations, and current capacity. Explain why a miss counter can exceed successful creations after an allocation failure. Check that reshaping a request does not unexpectedly clone storage. Verify byte accounting from actual element size and backing capacity, not from the logical shape of the smallest view.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use

an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

The free list makes availability explicit, but correctness comes from the registry and ownership rules behind it. A very fast list can still be wrong if it admits incompatible storage or duplicates a release.

Explain why class design and search policy are separate decisions: classification defines compatibility and capacity, while selection chooses among eligible records. This separation allows later experiments to change rounding or fit policy without changing the lifetime contract. Identify the invariant most likely to fail when asynchronous release is introduced and state how a pending index would preserve it.

19.27.6 Exercise 05

Exercise Title: Build Exact-Size and Rounded-Size Allocation Policies and Compare Their Memory Waste and Reuse Rate

Part 1 - Exercise Contract and Why It Matters

Compare exact-size classes with a configurable rounding quantum using the same request and release trace. Quantify the tradeoff between fewer classes, more potential reuse, and unused capacity inside active buffers. Produce a policy comparison that includes total retained capacity and does not mistake internal waste for all memory overhead.

Use identical dtype, device, concurrency, capacity limits, and completion rules. Define rounding in elements or in aligned bytes, then use that definition consistently. The code must leave class calculation and selection to the learner. Include a held-out trace so a policy chosen on one distribution is evaluated on a different one.

Part 2 - System and Memory Mental Model

Exact-size policies minimize class slack for each request, but a sequence of nearby sizes may populate many rarely reused classes. Rounding can allow those requests to share capacity across time. It does not permit two overlapping leases to share one writable buffer and does not reduce the logical amount of data the computation needs.

Internal waste is capacity minus requested bytes for leased buffers. Idle retained buffers and pending buffers belong in separate categories. A rounded policy may have more slack per live lease yet less total retained capacity because it reuses a smaller collection of buffers. Alternatively, rounding can increase both waste and capacity.

Use an element-compatible quantum. For byte-based classes, the resulting capacity must be divisible by the element size, and any stronger alignment requirement must be specified. Do not reinterpret an arbitrary uint8 buffer as several dtypes without a validated alignment and layout design. The initial policy instead keeps typed storage classes separate. Define explicit behavior for zero-element tensors and requests above the configured maximum before measuring them.

Part 3 - Quantitative Prediction Before Execution

For positive requested bytes q and a compatible byte quantum a, an ideal rounded class has

$$c(q) = a \left\lceil \frac{q}{a} \right\rceil, \qquad w(q) = c(q) - q, \qquad 0 \le w(q) < a.$$

Exact-size application classes have $c(q) = q$, although the underlying allocator can still apply its own rounding.

For a hypothetical quantum of 256 KiB, predict class capacity and slack for requests of 513, 700, and 768 KiB. Then compare a sequential trace with a trace in which all three requests overlap. Calculate the minimum number of buffers in each case under exact-class matching. Predict how a repeated 513/700 KiB alternation differs from a stream of never-repeated sizes. State which conclusions depend on the trace's release order.

Part 4 - Experimental Design and Instrumentation

1. Generate three traces: one stable size, nearby jittered sizes, and a broad distribution. Preserve explicit acquire and release events.

2. Replay each trace with exact classes and several rounding quanta. Fix the cache-retention and growth rules across policies.

3. At every transition, record requested live bytes, live slack, idle capacity, pending capacity if supported, and unique retained storage.

4. Count class cardinality and initial lookup hits. Separate cold-start misses from steady-state misses.

5. Measure a CUDA workload using a subset of the trace, initializing only the complete logical region required by the consumer.

6. Choose one quantum from calibration traces and evaluate it on the held-out trace without retuning.

Do not read uninitialized class slack or include it in the output shape. If an implementation zeroes whole backing buffers, include those additional writes in its measured overhead and label the initialization policy.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch
```

```
@dataclass
class SizeDecision:
    requested_bytes: int
    capacity_bytes: int
    class_key: object


def exact_size_policy(request):
    # Validate the request and choose its exact class.
    ...


def rounded_size_policy(request, quantum_bytes):
    # Respect dtype element size and declared alignment.
    ...


def replay_policy(trace, policy, pool_config):
    # Keep lifetime, retention, and growth rules identical.
    ...


def summarize_waste(rows):
    # Separate live slack, idle retention, and total capacity.
    ...


def E05(config):
    # Predict classes for a hand-computable request sequence.
    ...
    # Calibrate on one trace and evaluate another unchanged.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Check every class decision against the declared rounding rule. For positive supported requests, capacity must cover the logical bytes and obey element-size divisibility. Exact-size application slack should be zero; allocator rounding must appear in a different measurement.

Report hit rate, class count, time-weighted live slack, peak pool capacity, and complete workload time. Validate the time weighting: a large idle interval should not be treated as equal to one short transition merely because both produce one log row. Explain at least one case where better hit rate does not imply lower memory use. Reject comparisons that changed the request trace, live concurrency, or trimming policy while changing the rounding quantum.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Size classes are a prediction about which future requests can share storage. Rounding deliberately spends capacity to increase that sharing opportunity. The quality of the prediction depends on both sizes and lifetimes.

Discuss whether a globally fixed quantum is appropriate for the measured range or whether separate small and large regimes deserve investigation. Do not introduce a new policy solely because it improves the calibration trace. State the evidence that would justify the extra complexity and the memory cost of a mistaken prediction.

19.27.7 Exercise 06

Exercise Title: Design Power-of-Two Size Classes and Measure How Internal Fragmentation Changes Compared with Exact-Size Pools

Part 1 - Exercise Contract and Why It Matters

Implement and evaluate power-of-two size classes with a documented minimum class and maximum supported request. Derive their ideal slack bounds, then identify where minimum-class and alignment rules change those bounds. Compare against the exact-size pool on traces concentrated near class boundaries as well as ordinary variable workloads.

Produce a boundary table, an application-capacity ledger, and an empirical comparison. Use the term internal fragmentation specifically for unused space inside an assigned buffer. Do not equate it with the allocator's inability to satisfy a contiguous request or with idle cached capacity.

Part 2 - System and Memory Mental Model

Geometric classes reduce the number of class sizes needed across a wide range. Requests just above a power-of-two boundary can receive almost twice their requested capacity, while requests exactly on the boundary have no application-level slack. A single class can be reused by many different logical shapes when dtype and layout compatibility hold.

The mathematical rule is easy to state but still needs a robust implementation. Define whether rounding uses elements or bytes, handle zero explicitly, and reject negative or unsupported requests. Avoid floating-point logarithms for integer boundary decisions if their precision could select an undersized class; the learner must choose and justify an exact integer method. Verify multiplication into bytes before requesting storage.

A minimum class breaks the usual relative-waste bound for tiny requests. Likewise a maximum pool budget can reject a rounded class even when the unrounded request would have fit. Underlying allocator rounding introduces another layer that must be measured independently. The power-of-two policy has no authority over physical segment placement.

Part 3 - Quantitative Prediction Before Execution

For positive q with no extra minimum or alignment constraint, the ideal byte capacity is

$$c(q) = 2^{\lceil \log_2 q \rceil}, \qquad q \le c(q) < 2q.$$

Consequently, $w = c - q < q$ and $w/c < 1/2$. These strict bounds are for the ideal rule, not for all practical class configurations.

For requests $2^k - 1$, 2^k, and $2^k + 1$, predict capacity and both waste fractions w/q and w/c. Use byte requests representable by the selected dtype when constructing actual tensors. Then introduce a 4 KiB minimum class and a hypothetical 16-byte request: determine which relative

bound no longer holds. Predict the number of distinct classes for a chosen finite request interval and the effect of excluding zero from the logarithmic formula.

Part 4 - Experimental Design and Instrumentation

1. Build an integer-only boundary test table before touching CUDA. Include exact powers, adjacent representable requests, zero, and maxima.
2. Generate a sequential trace concentrated just above boundaries. Compare exact-size, fixed-quantum, and power-of-two class assignments.
3. Add overlapping lifetimes so the experiment cannot mistake temporal reuse for simultaneous sharing.
4. Replay bounded CUDA traces with identical initialization and consumers. Record live slack, idle capacity, pool peaks, hits, and acquisition time.
5. Test requests whose rounded capacity exceeds the limit. Report rejection without silently allocating outside the budget.
6. Repeat with a workload whose sizes naturally cluster near powers of two and compare the change in the tradeoff.

Keep the ordinary allocator backend fixed. If its reserved-memory behavior differs between policies, describe that as an observation requiring allocator evidence rather than as a direct consequence of the application slack formula.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
import torch


def choose_power_class(requested_elements, itemsize, limits):
    # Implement exact integer rounding and boundary handling.
    ...


def predict_boundary_rows(exponents, dtype, minimum_bytes):
```

```
    # Derive capacity and both definitions of waste fraction.
    ...


def build_boundary_trace(config):
    # Create valid requests with explicit overlap intervals.
    ...


def compare_class_policies(trace, policies, config):
    # Preserve identical lease and retention semantics.
    ...


def E06(config):
    # Validate class arithmetic before CUDA allocation.
    ...
    # Measure adversarial and naturally clustered traces.
    ...
    # Explain every exception to the ideal slack bound.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require exact boundary classification and no undersized buffers. Show the ideal bound separately from minimum-class cases. Include requested bytes, chosen class, slack bytes, slack divided by request, and slack divided by capacity in the table; the two percentages answer different questions.

Check that a rejected rounded request leaves accounting unchanged. Validate views and outputs for all accepted requests. Investigate a very high hit rate alongside high capacity: coarse classes may be reusable but expensive. Confirm that differences in reserved memory are not caused by running one policy immediately after another without controlling allocator history. Document cold and warm results instead of discarding an unfavorable startup cost.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

A bounded number of class sizes simplifies metadata but can impose substantial slack near boundaries. The ideal bound is useful precisely because its assumptions are explicit and testable.

Use your results to explain whether power-of-two classes are appropriate for the chapter's target workload. A hybrid class scheme is a possible future experiment, not an automatic improvement. Describe the measurement that would justify finer classes and the extra bookkeeping or reduced reuse that finer granularity could introduce.

19.27.8 Exercise 07

Exercise Title: Implement Best-Fit Buffer Selection and Compare It Against First-Fit Reuse Under Variable Tensor Sizes

Part 1 - Exercise Contract and Why It Matters

Compare first-fit and best-fit selection among compatible, safely reusable buffers. Keep class construction, pool growth, request order, and release order fixed. Deliver a hand-worked trace and a measured comparison showing selection overhead, live slack, misses, and retained capacity. The implementation must distinguish choosing a candidate from deciding whether growing the pool is allowed.

Allow a request to use an oversized compatible buffer when the policy permits it. Require the learner to define deterministic iteration and tie-breaking rules. An unavailable or pending buffer must never become a candidate solely because its size is attractive.

Part 2 - System and Memory Mental Model

First-fit chooses the first eligible buffer large enough in a specified order; best-fit chooses the smallest eligible capacity that covers the request. Best-fit minimizes immediate slack within the candidate set. It does not generally minimize future allocations, latency, or allocator fragmentation over the whole trace.

A best-fit search over an unsorted list can inspect every candidate, while an indexed structure changes both lookup and update costs. Measure those costs instead of attributing all differences to memory. First-fit results depend on list order, so insertion order, size order, and recency order are different policies and must not be mixed.

Application buffers are separate storages. Unused tails are not reusable suballocations unless a separate arena allocator with offsets and alias rules is implemented. That extension is outside this exercise. Two free 4 MiB tensors do not form one contiguous 8 MiB tensor. Describe fragmentation at the application inventory level before attempting to infer anything about allocator segments.

Part 3 - Quantitative Prediction Before Execution

For eligible set $\mathcal{F}$ and request q, best-fit targets

$$c^* = \min\{c_j : j \in \mathcal{F},\ c_j \geq q,\ j \text{ compatible}\}.$$

The immediate slack is $c^* - q$. If the set is empty, the selection result is no fit; growth or waiting is a separate decision.

Consider free buffers in order 16, 8, and 12 MiB, followed by overlapping requests of 7 and 15 MiB. Predict each policy's choices and whether a second allocation is needed. Then devise a different trace where both policies use the same capacity but best-fit performs more comparisons. For a linear scan of F free records, state a worst-case comparison bound and distinguish it from measured average search work.

Part 4 - Experimental Design and Instrumentation

1. Implement one candidate-compatibility predicate shared by both policies. Include device, dtype, capacity, layout, and readiness.

2. Verify the hand-worked trace with scalar simulated records. Log candidate order and selected buffer ID for every request.

3. Replay identical seeded size-and-lifetime traces in separate pools. Preserve the same growth limit and eviction policy.

4. Measure lookup time and comparison count separately from total workload time, where CUDA completion is included.

5. Include traces with many small requests followed by a large request, and traces with a narrow range of sizes.

6. Compare application slack and retained capacity with allocator statistics, keeping the accounting layers distinct.

Do not reorder first-fit's free list between runs without labeling the policy change. When there is no fit, log why each plausible buffer failed eligibility rather than calling every failure a size shortage.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every `...` marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch


@dataclass
class Candidate:
    buffer_id: int
    capacity_bytes: int
    compatible: bool
    ready: bool


def eligible_candidates(records, request):
    # Apply one shared compatibility and readiness contract.
    ...


def select_first_fit(candidates, requested_bytes):
    # Preserve the declared deterministic iteration order.
    ...
```

```
def select_best_fit(candidates, requested_bytes):
    # Choose by capacity and an explicit tie-break rule.
    ...


def replay_selection(trace, selection_policy, pool_config):
    # Separate no-fit selection from admission and growth.
    ...


def E07(config):
    # Verify the hand trace, then compare seeded workloads.
    ...
    # Export slack, comparisons, misses, and complete latency.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Check that selected candidates are ready, compatible, and large enough. For best-fit, audit the complete candidate set in a diagnostic pass to prove no smaller eligible capacity was skipped. For first-fit, audit that no earlier eligible record was skipped. Keep these expensive audits out of the primary throughput interval and report their overhead.

Use a results table containing policy order, live slack, pool peak, growth count, mean comparisons, lookup tail latency, and complete throughput. Include a no-fit event despite sufficient total free bytes spread across separate buffers. Do not label that evidence as physical allocator fragmentation without a separate allocator-level investigation. Numerical outputs and completed request counts must match across policies.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Selection is an online decision with incomplete knowledge of future requests. Minimizing immediate slack can preserve large buffers, but the benefit depends on the future trace and may cost additional search work.

State whether the observed gain comes from better inventory use, lower search overhead, or neither. Explain how stronger workload predictability might favor static preallocation instead. The experiment should leave the reader with a conditional policy choice grounded in evidence, not a universal ranking of fit algorithms.

19.27.9 Exercise 08

Exercise Title: Track Pool Hit Rate, Miss Rate, Reuse Count, Wasted Bytes, and Peak Pool Capacity During a Real PyTorch Workload

Part 1 - Exercise Contract and Why It Matters

Instrument the pool with metrics that remain interpretable under growth, waiting, rejection, and repeated reuse. Integrate it into a real PyTorch inference or input-staging workload with supported buffers, rather than claiming control over every allocation made inside a model. Deliver a scalar event log, a reconciled summary, and a measured instrumentation overhead comparison.

Define every denominator before collecting results. Track application requests separately from retry attempts and count unique storage rather than tensor views. Include a period with no active leases and a period with several in-flight leases. The report must explain why a high hit rate does not by itself imply an efficient memory system.

Part 2 - System and Memory Mental Model

A request can miss at its first lookup, wait, and eventually receive an existing buffer. It can also miss and grow the pool, or be rejected. Choose a mutually exclusive first-lookup classification, then record the terminal outcome separately. This avoids counting one request as both a hit and a miss when it is retried. Repeated internal polls are attempts, not new application requests.

Reuse count describes repeated successful loans of a backing buffer after its first successful loan. It is not identical to hit count when the pool was preallocated before any request. Report that convention. Live slack, free capacity, pending capacity, and total retained capacity have distinct meanings and must be tracked at transitions that can change them.

Metrics themselves can change performance. GPU scalar extraction can synchronize, and retaining tensors in a log changes their lifetime. Prefer host metadata for ownership events and periodic scalar CUDA memory observations at deliberate boundaries. Pool misses do not describe allocations inside opaque operators, so measure whole-process peaks alongside the managed subset.

Part 3 - Quantitative Prediction Before Execution

Let R be distinct application requests, H first-lookup hits, and M first-lookup misses:

$$R = H + M, \qquad h = H/R \quad (R > 0).$$

If buffer j has u_j successful loans, define total reuse as

$$U = \sum_j \max(u_j - 1, 0).$$

For time-varying slack $W(t)$, compare its peak with its time average

$$\overline{W} = \frac{1}{T} \int_0^T W(t)\, dt.$$

State how event timestamps approximate that integral.

For a hypothetical ten-request trace containing three initial misses, two waits, one rejection, and seven first-lookup hits, determine which facts are insufficient to calculate reuse. Explain why terminal outcomes must be linked to request IDs. Predict the difference between a large slack spike lasting one millisecond and the same slack lasting one second.

Part 4 - Experimental Design and Instrumentation

1. Write a metric schema with units, counting scope, reset behavior, and zero-denominator conventions. Validate it on a small simulated trace.

2. Integrate the pool into repeated inference input staging or explicit temporary tensors whose lifetimes you control.

3. Run stable shapes, variable shapes, and a short burst of increased concurrency. Keep model parameters and numerical settings fixed.

4. Log request and buffer IDs, state transitions, requested and capacity bytes, first-lookup result, terminal outcome, and completion time.

5. Reconstruct summary metrics from the raw log and compare them with live counters.

6. Repeat with detailed logging disabled to quantify overhead.

Store tensor-free records. If whole-process allocation rises while pool capacity is stable, inspect unmanaged model outputs or activations instead of blaming the pool's class policy.
Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch


@dataclass
class PoolEvent:
    timestamp_seconds: float
    request_id: int
    buffer_id: int
    event_kind: str
    requested_bytes: int
    capacity_bytes: int


class PoolMetrics:
    def __init__(self):
        # Establish explicit counters and empty-log semantics.
```

```
        ...

    def observe(self, event):
        # Count requests, attempts, and outcomes separately.
        ...

    def reconcile(self, registry):
        # Compare live counters with unique storage records.
        ...

    def summarize(self):
        # Derive rates, peaks, and time-weighted quantities.
        ...


def E08(model, request_trace, config):
    # Run a supported workload and verify output equivalence.
    ...
    # Reconstruct metrics independently from scalar events.
    ...
    # Measure the cost of instrumentation.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Acceptance requires $R = H + M$, one terminal outcome per completed request, and agreement between reconstructed and live capacity metrics. Pending requests at shutdown must be labeled unfinished until drained or cancelled under a defined protocol. Zero-request intervals should have undefined rates, not misleading perfect hit rates.

Report median and tail acquisition latency, wait duration, hit rate, successful creations, reuse count, live slack, idle capacity, pool peak, and process memory peaks. Verify that the metrics logger contains no tensor objects or autograd graphs. If instrumentation changes throughput, present both measurements and use the minimally instrumented run for performance claims. Show a high-hit-rate case with substantial retention or explain why your measured trace did not produce one.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Observability is part of the pool design because incorrect metrics can reward unsafe or wasteful behavior. A request that waits for an in-flight buffer may be memory-efficient yet have poor tail latency; one number cannot capture both effects.

Use the evidence to identify the current limiting factor: insufficient capacity, poor size matching, overly long ownership, or instrumentation. Propose the smallest experiment that

distinguishes your leading explanation from an alternative. Carry the same event schema into later asynchronous and budgeted pools so comparisons remain meaningful.

19.27.10 Exercise 09

Exercise Title: Build Separate CPU Pinned-Memory and GPU VRAM Buffer Pools for Repeated Host-to-Device Transfers

Part 1 - Exercise Contract and Why It Matters

Build separate pools for pinned CPU staging buffers and CUDA destination buffers. Demonstrate repeated host-to-device transfers with explicit ownership on both sides and independent RAM and VRAM accounting. Compare against an ordinary allocation-and-transfer path using equivalent CPU preparation and GPU consumption.

The initial implementation may use conservative completion barriers. Its purpose is to establish correct paired lifetimes and budgets before E10 introduces overlap. Do not assume pinned memory is free or that pinning every iteration is cheaper than retaining a bounded pinned pool. Measure construction and warmed reuse separately.

Part 2 - System and Memory Mental Model

A pinned source buffer occupies host memory and supports transfer behavior different from ordinary pageable storage. The GPU destination occupies device memory. These allocations are not one interchangeable pool, even when they represent the same logical batch. Their dtypes, capacities, and initialization regions must match the transfer contract.

For an H2D transfer, the CPU source must remain unchanged until the copy finishes. The destination cannot be consumed before the copy completes and cannot be overwritten until its last GPU consumer finishes. Those are separate completion points. In the conservative baseline, synchronize at defined boundaries and release only when the corresponding condition is established.

Pinned memory is a subset of host memory, so avoid summing a process RAM metric and pinned bytes as though they were disjoint. The process may also have dataset pages, Python objects, and pageable staging allocations. Likewise the GPU process contains model storage outside the destination pool. A pool cap controls managed capacity, not all memory on the machine. Host and device allocator caching can persist after application records are dropped, so retain both application and process-level observations.

Part 3 - Quantitative Prediction Before Execution

For n_h host buffers and n_d device buffers of capacity c, predict

$$C_{\text{pinned}} = n_h c, \qquad C_{\text{device}} = n_d c.$$

These are separate resource requirements. A simplified serial transfer model is

$$T_{\text{batch}} = t_{\text{fill}} + t_{\text{H2D}} + t_{\text{compute}}.$$

Pooling changes preparation and allocation costs; it does not make the transfer payload disappear.

For a hypothetical 8 MiB payload, three pinned buffers, and two device buffers, calculate each pool's capacity and the total logical bytes copied over 100 batches. Predict why the two pools need not have equal depth. Explain what extra measurement is needed before using a bandwidth estimate to predict transfer time for the chosen payload size.

Part 4 - Experimental Design and Instrumentation

1. Specify independent pinned-RAM and VRAM caps and leave room for the dataset, model, and runtime outside the pools.
2. Allocate typed pinned CPU buffers and typed CUDA buffers with matching transfer layouts. Verify pinning and device properties.
3. Fill a host lease with a request-specific pattern, copy it to a device lease, and validate a deterministic GPU consumer.
4. Release the source only after copy completion and the destination only after consumer completion.
5. Compare ordinary pageable transfer, repeated pinning if supported, and reused pinned staging as separate labeled modes.
6. Measure CPU fill time, transfer time, complete batch time, host retention, device retention, and startup cost.

Include the cost of copying dataset data into the pinned staging buffer. Do not compare a prepared pooled source with an unprepared baseline and attribute the difference solely to allocation reuse.

Investigate `torch.empty` with CPU placement and `pin_memory=True`, tensor `is_pinned()`, and a destination tensor's `copy_` interface. A conversion such as `to()` can create a new destination; verify that your chosen operation actually uses the acquired destination storage.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch


@dataclass
class TransferLeases:
    request_id: int
    host_lease: object
    device_lease: object
```

```
def acquire_transfer_pair(host_pool, device_pool, request):
    # Acquire compatible leases with rollback on failure.
    ...


def prepare_host_payload(host_lease, batch):
    # Initialize the complete logical transfer region.
    ...


def copy_and_consume(pair, consumer, device):
    # Establish distinct copy and compute completion points.
    ...


def release_transfer_pair(pair, completion):
    # Respect host-source and device-destination lifetimes.
    ...


def E09(config):
    # Compare startup and warmed transfer modes fairly.
    ...
    # Validate payload identity and both resource ledgers.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require correct device contents for every transferred request, no premature source mutation, and no destination reuse before consumption. Verify that failure to acquire the second lease returns the first safely without leaking capacity. A metadata rollback cannot cancel an already submitted CUDA copy; submitted work needs its completion protocol.

Report host and device capacities in separate columns and distinguish pinned bytes from total process RAM. Show a shutdown procedure that drains work before discarding buffers. If a transfer is slower than the pageable baseline, inspect CPU fill cost, small-payload overhead, and measurement boundaries. Treat the result as evidence rather than assuming pinned reuse must always win.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Transfer pooling is a two-resource scheduling problem. The host buffer and device buffer can become reusable at different times, so coupling their lifetimes unnecessarily can retain memory

and reduce throughput.

Explain which pool is the bottleneck in your experiment and whether adding capacity to the other pool would help. Relate the answer to the measured fill, copy, and compute durations. This reasoning provides the basis for a pipelined design whose depth is justified by service times and memory budgets.

19.27.11 Exercise 10

Exercise Title: Reuse Staging Buffers Across Non-Blocking Transfers Without Overwriting Data Still Used by an In-Flight CUDA Operation

Part 1 - Exercise Contract and Why It Matters

Turn the transfer baseline into a bounded pipeline with non-blocking copies and request-specific completion tracking. The objective is to overlap independent work while preserving data identity. Produce a timeline showing host fill, H2D copy, GPU consumption, and the two different release points.

Use a configurable ring depth and compare depth one with deeper pipelines. A correct implementation may wait when the next slot is still busy. Never make an occupied slot appear free merely because the Python copy call has returned.

Part 2 - System and Memory Mental Model

Non-blocking submission means the host can continue before the transfer finishes; it does not authorize mutation of the pinned source. A GPU wait can order device work, but it does not stop the CPU from overwriting a host buffer. Host reuse needs proof that the copy has completed, such as an appropriate event query or synchronization.

The GPU destination has a longer possible lifetime. Copy completion permits its consumer to begin, while the final consumer's completion permits another producer to overwrite it. If several consumers read the destination, all relevant uses must be covered. For D2H extensions, the host destination must not be read before transfer completion or reused before the host consumer releases it.

Separate streams can permit overlap when hardware and dependencies allow it, but do not guarantee it. Pinned source storage, sufficient independent work, copy-engine capability, and scheduling all matter. A ring index is only a location selector; its generation and completion state determine whether that slot is eligible.

Part 3 - Quantitative Prediction Before Execution

Let τ be the production interval and L_h, L_d the host and device lease lifetimes. Under a regular ideal pipeline, useful depth estimates are

$$n_h \gtrsim \lceil L_h/\tau \rceil , \qquad n_d \gtrsim \lceil L_d/\tau \rceil .$$

These are planning estimates, not guarantees under jitter or contention. An ideal steady-state lower bound on batch interval is

$$\tau \geq \max(t_{\text{fill}}, t_{\text{copy}}, t_{\text{compute}})$$

only when the stages can overlap as modeled.

For hypothetical fill, copy, and compute durations of 1, 2, and 5 ms, sketch a depth-one serial timeline and a feasible deeper timeline. Identify when each host and device slot can

be reused. Predict how extra depth changes memory capacity even after throughput stops improving.

Part 4 - Experimental Design and Instrumentation

1. Create a safe depth-one reference and verify its outputs exactly for copied integer patterns.
2. Add a copy stream and compute stream with explicit producer-to-consumer dependencies. Associate events with request and slot generations.
3. Refill a host slot only after its copy event is complete. Reassign a device slot only after its last consumer completes.
4. Sweep pipeline depth within fixed RAM and VRAM caps. Measure complete throughput, wait time, and retained capacity.
5. Vary CPU preparation and GPU consumer duration so either side can become the bottleneck.
6. Add a safe simulator that rejects an attempted early refill. Use it as the negative control instead of intentionally racing live DMA.

Do not synchronize every iteration in the overlapped benchmark. Drain the pipeline before the final wall-clock timestamp and before validating results or destroying its buffers.
Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch


@dataclass
class StagingSlot:
    generation: int
    host_lease: object
    device_lease: object
    copy_done: object
    consumers_done: object
```

```
def slot_host_ready(slot):
    # Establish CPU-safe source reuse for this generation.
    ...


def slot_device_ready(slot):
    # Cover every previous GPU consumer.
    ...


def submit_pipeline_stage(slot, payload, streams):
    # Fill, copy, order consumption, and record completions.
    ...


def drain_pipeline(slots, device):
    # Finish submitted work before validation or shutdown.
    ...


def E10(config):
    # Compare ring depths with identical complete workloads.
    ...
    # Export ownership intervals and producer wait times.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Every consumed payload must match its request ID and expected contents. Log the generation occupying each slot so a correct value from a previous batch cannot pass unnoticed. Check that slot reuse timestamps follow the relevant completion evidence; host function return is not sufficient.

Report pipeline depth, completed batches, total time, host wait time, device wait time, transfer bytes, and both pool capacities. Inspect the timeline if throughput fails to improve: the workload may lack independent work or may already saturate one stage. A depth increase that only grows memory is an important result. Require bounded waiting and an explicit timeout outcome for injected missing-completion cases.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Safe overlap is a lifetime transformation, not just a non-blocking flag. The pipeline gains freedom by keeping multiple generations alive, and that freedom consumes capacity.

Explain the smallest depth that achieves the observed throughput within measurement uncertainty. Compare the cost of waiting with the cost of holding another buffer. Carry forward the distinction between host-safe and device-safe reuse: later multi-stream pooling must preserve both, even when the physical buffer selection becomes more dynamic.

19.27.12 Exercise 11

Exercise Title: Add CUDA Event Tracking So Buffers Return to the Pool Only After Their Last Asynchronous Consumer Has Completed

Part 1 - Exercise Contract and Why It Matters

Introduce an explicit pending-completion state between lease release and reusable availability. Build an event-aware reclamation interface that returns buffers to free lists only after every declared asynchronous consumer has finished. Deliver a state-transition audit and a comparison against globally synchronized release.

Specify who registers consumers, who closes the lease to new consumers, and who polls completion. A buffer cannot be safely reclaimed if new users may still be registered after the final event is chosen. The learner must implement this protocol and its validation.

Part 2 - System and Memory Mental Model

Use the state sequence free, leased, pending, free. Releasing a lease closes its ownership generation but may only move the record to pending. Capacity remains retained and unavailable until the completion proof passes. A successful event query establishes completion of work covered by that event; it does not cover commands submitted later.

For multiple consumers, retain a set of completion events or construct a join on a stream that waits for all consumers before recording one final event. An event recorded after only one consumer is insufficient. Avoid re-recording an event for a new generation while old pool records still depend on its meaning.

`record_stream` informs the allocator about uses when tensor storage is eventually deallocated. It does not prevent an application pool from overwriting the same still-live tensor. This pool therefore needs its own lease and completion protocol even if allocator lifetime tracking is also used. Completion polling should be bounded and must not turn a CUDA error into a false claim of readiness.

Part 3 - Quantitative Prediction Before Execution

For a closed lease with consumer event set $\mathcal{E}_j$, the eligibility condition is conceptually

$$\text{ready}(j) = \text{closed}(j) \ \wedge \bigwedge_{e \in \mathcal{E}_j} \text{complete}(e).$$

Predict the capacity conservation relation

$$C = C_{\text{free}} + C_{\text{leased}} + C_{\text{pending}}.$$

No capacity is removed merely by moving from leased to pending.

For a hypothetical 16 MiB buffer with consumers completing at 3 and 7 ms, identify the earliest globally safe return time. If polling occurs at 4 and 8 ms, distinguish actual completion from observed reclamation. Predict how a longer polling interval can increase required capacity without changing useful GPU work.

Part 4 - Experimental Design and Instrumentation

1. Implement a metadata-only completion simulator and prove all state transitions before connecting real CUDA events.

2. Run one-consumer and two-consumer CUDA cases with request-specific patterns and explicit data dependencies.

3. Close the lease, record the relevant completion evidence, and place the record in pending without putting it in a free list.

4. Poll at configurable intervals; measure pending bytes, polling cost, and delay between completion and observed reclamation.

5. Compare with a synchronized-release baseline under equal maximum capacity and useful work.

6. Inject stale-generation callbacks, duplicate completion processing, and missing consumer registration in the simulator.

Ensure diagnostics do not reuse event objects prematurely. At shutdown, drain pending work or report an explicit incomplete state; do not label undrained buffers as free.

Inspect `torch.cuda.Event`, its `record` and `query` methods, and stream `wait_event`. Distinguish a host completion query from inserting a dependency into another stream. Events used only for completion do not require timing measurements to be enabled.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every `...` marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch


@dataclass
class PendingLease:
    buffer_id: int
    generation: int
    completion_events: tuple
    closed: bool
```

```
class EventReclaimer:
    def __init__(self, pool):
        # Track pending records independently of free lists.
        ...

    def close_lease(self, lease, consumer_events):
        # Reject future consumers and capture this generation.
        ...

    def poll(self, limit):
        # Reclaim only records whose complete proof is ready.
        ...

    def audit(self):
        # Prevent duplicate refunds and stale-event reuse.
        ...


def E11(config):
    # Compare one consumer, multiple consumers, and a join.
    ...
    # Measure polling overhead and delayed reclamation.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require that pending records never appear in free lists and that every reclaimed record corresponds to the current closed generation. Repeated polls after completion must not refund capacity twice. Validate the two-consumer case against a synchronized reference and show that the earlier event alone cannot authorize reuse.

Report pending peak bytes, completion-observation delay, poll count, acquisition stalls, and complete throughput. Keep true GPU completion and host observation timestamps conceptually separate. A timeout or CUDA failure must produce a diagnosable terminal status, not silently release the buffer. Explain how an incomplete consumer-registration contract could invalidate otherwise correct event code.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Asynchronous reclamation separates logical release from physical reuse. That distinction lets the host continue without globally synchronizing, but it requires a proof that covers all users of each generation.

Discuss how polling frequency trades CPU overhead against retained pending capacity. Identify whether your workload benefits more from frequent polling, larger capacity, or shorter

consumer lifetimes. The resulting policy should be justified by measured delay and throughput rather than by assuming events are free.

19.27.13 Exercise 12

Exercise Title: Build a Multi-Stream Buffer Pool and Prevent Two CUDA Streams from Reusing the Same Storage Unsafely

Part 1 - Exercise Contract and Why It Matters

Extend the event-aware pool to several CUDA streams and, optionally, several host submitters. Preserve exclusive write ownership and explicit producer-to-consumer ordering. Deliver a stream-use registry, a contention trace, and correctness evidence under deliberately varied completion orders.

Begin with the conservative rule that a free buffer is globally reusable only after all previous consumers finish. A more aggressive stream-ordered reuse policy is an optional extension requiring its own proof. Keep host-buffer reuse rules separate because GPU waits cannot prevent a CPU producer from overwriting pinned storage.

Part 2 - System and Memory Mental Model

Two kinds of ordering are required. Data dependencies ensure that a consumer reads results after their producer writes them. Lifetime dependencies ensure that a future borrower does not overwrite storage while an earlier consumer still uses it. Satisfying one does not automatically satisfy the other.

The host registry must also allocate ownership atomically. Two threads cannot both observe the same free record and then independently lease it. Use an explicit synchronization design rather than relying on the Python GIL as the ownership protocol. Metadata locks should not be held while performing an unbounded CUDA wait; define how to wait and then revalidate the state.

Allocator stream tracking concerns deallocation and later allocator reuse. An application pool retains the same tensor, so its manual overwrite must be governed by its own events and generations. If storage is evicted while side-stream work is still outstanding, the design is already violating the conservative pool contract. For permitted deallocation paths, also honor the allocator's stream lifetime requirements. An event on the default stream does not automatically describe work on every other stream.

Part 3 - Quantitative Prediction Before Execution

For buffer j, define owner count o_j and outstanding closed-generation consumer count u_j. Require

$$o_j \in \{0, 1\}, \qquad j \in \mathcal{F} \Rightarrow o_j = 0 \ \wedge \ u_j = 0.$$

These are metadata invariants; correct data ordering requires additional producer dependencies.

With three streams and two buffers, predict the outcome when each stream requests a writable lease before any release. Then consider one producer whose output is read by two streams: distinguish one shared read phase under a closed ownership protocol from two independent writable leases. Draw the event dependencies and identify which event or join permits the next writer. Predict how conservative global readiness can reduce overlap relative to a proven stream-ordered design.

Part 4 - Experimental Design and Instrumentation

1. Implement the ownership transition logic in a host-only simulator. Force two submitters to contend for the same free record.

2. Run a single-stream reference, then the same request trace across multiple streams with explicit producer dependencies.

3. Register all consumers before closing each lease. Reclaim through the E11 completion protocol.

4. Vary consumer duration and submission order while keeping the numerical workload fixed. Record request, stream, buffer, and generation.

5. Validate unique ownership and output patterns after draining. Measure waits caused by both capacity shortage and incomplete consumers.

6. If adding stream-ordered reuse, state its stronger assumptions and compare it separately with globally ready reuse.

Use bounded metadata misuse tests rather than intentionally submitting conflicting writes. A safety checker should reject an invalid transition before it becomes a real CUDA data race.
Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every `...` marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import threading
import torch


@dataclass
class StreamUse:
    stream_id: int
    generation: int
    completion_event: object


class MultiStreamPool:
    def __init__(self, config):
        # Establish records, locks, and completion tracking.
```

```
        ...

    def acquire(self, request, producer_stream):
        # Atomically reserve one compatible globally ready record.
        ...

    def register_consumer(self, lease, stream, ready_event):
        # Enforce data ordering and record the consumer lifetime.
        ...

    def close(self, lease):
        # Seal registration before forming completion evidence.
        ...

    def reclaim(self):
        # Revalidate generations without unbounded locked waits.
        ...


def E12(config):
    # Exercise contention and out-of-order stream completion.
    ...
    # Audit ownership separately from numerical correctness.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require no duplicate writable ownership, no reuse of pending storage, and correct output for every request. A test that only checks final outputs can miss a race that happened not to corrupt data, so also audit the event and ownership graph. A graph with correct ordering but a forgotten consumer must fail the lifetime audit.

Report per-stream completed requests, acquisition stalls, pending capacity, metadata contention time, and complete throughput. Check shutdown while streams have unequal outstanding work. Validate that one submitter timing out cannot release another submitter's lease. Document any unsupported cross-device or external-stream use rather than assuming the single-device proof extends automatically.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Concurrency changes the pool from a container into a coordination system. The central requirement is a consistent ownership decision across host threads and device streams.

Explain where the conservative protocol spends performance to gain simpler reasoning. Consider relaxing it only when measurements identify that restriction as a significant bottleneck.

An optimization that depends on undocumented stream behavior is weaker than a slower design with explicit dependencies and reproducible evidence.

19.27.14 Exercise 13

Exercise Title: Add Maximum Pool Capacity and Evict Cached Buffers When the Pool Exceeds a Fixed VRAM Budget

Part 1 - Exercise Contract and Why It Matters

Add a hard limit on application-managed GPU buffer capacity. Before growing the pool, account for existing storage and outstanding growth reservations; evict only eligible free buffers when policy allows it. Demonstrate bounded behavior under concurrent growth attempts, allocation failure, and a runtime reduction of the configured cap.

Define the limit in bytes and include leased, pending, and free buffers. The cap applies to this pool, while model storage and other allocations remain outside it. A pool cap is not a guarantee that the entire process or GPU cannot run out of memory.

Part 2 - System and Memory Mental Model

Admission must occur before allocation. If two requests each check the same remaining capacity, both can overcommit unless the decision and growth reservation are atomic. When storage is materialized, replace its reservation with actual capacity rather than counting both. On allocation failure, remove the reservation exactly once.

Eviction means removing the pool's ownership of a free buffer. It cannot reclaim a leased or pending buffer without violating the lifetime contract. Dropping the last application tensor reference can reduce allocated bytes, but the CUDA allocator may retain the freed block. `empty_cache()` affects unused allocator cache, not live tensors that still belong to the pool.

If the configured cap shrinks below unavoidable live and pending capacity, the pool is temporarily over the new limit. Stop growth and new unsafe admissions, drain work, and report the condition. Do not retroactively claim the smaller cap was honored. Requests larger than the cap need an explicit rejection or a separately specified fallback that is still charged to the relevant budget.

Part 3 - Quantitative Prediction Before Execution

Let C be materialized unique capacity, G outstanding growth reservations, c a proposed buffer, and B the cap. Admission needs

$$C + G + c \leq B$$

after eligible eviction and before materialization. During successful creation, the increase in C must match the decrease in G.

For a hypothetical 64 MiB cap, 40 MiB leased, 8 MiB pending, and 16 MiB free, analyze a new 24 MiB request. Determine whether evicting all free buffers is sufficient. Then analyze a cap reduction to 32 MiB and state what can and cannot be reclaimed immediately. Distinguish admitted capacity from driver-visible free VRAM in both examples.

Part 4 - Experimental Design and Instrumentation

1. Implement an atomic capacity ledger and validate it with concurrent host-only growth simulations.

2. Use a deterministic victim policy over free records, recording evicted capacity and expected future reuse cost.

3. Replay a CUDA trace with alternating small and large requests. Compare unlimited retention with several bounded capacities.

4. Inject allocation failure after reservation but before record publication. Confirm exact rollback and unchanged existing leases.

5. Reduce the cap while work is pending. Record oversubscription, blocked growth, completion, and eventual compliance.

6. Observe allocated, reserved, and driver-free memory before and after dropping eligible buffers; treat cache release as a separate action.

Keep a reserve for unmanaged GPU allocations. A favorable free-memory sample does not lock out another process, so report allocation failures even when the pool's internal ledger was valid. **Reproducibility record.** Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch


@dataclass
class CapacityLedger:
    limit_bytes: int
    materialized_bytes: int
    reserved_growth_bytes: int


class BudgetedPool:
    def __init__(self, config):
        # Establish atomic accounting and free-only eviction.
        ...

    def reserve_growth(self, request):
        # Select eligible victims and reserve before allocating.
```

```
        ...

    def materialize(self, reservation):
        # Publish storage or roll back a failed reservation.
        ...

    def set_limit(self, new_limit_bytes):
        # Handle limits below current non-evictable capacity.
        ...

    def audit_capacity(self):
        # Reconcile actual storage and unmaterialized promises.
        ...


def E13(config):
    # Exercise concurrent growth, failure, and cap reduction.
    ...
    # Compare application capacity with allocator observations.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require capacity conservation and no uncharged allocation path. At a fixed cap, every admitted growth must satisfy the ledger inequality. Under cap reduction, explicitly identify the interval in which previously admitted storage exceeds the new limit. Free-only eviction must preserve all active and pending generations.

Report rejection count, waiting time, eviction count, growth retries, pool peak, and allocated/reserved peaks. Verify that stale aliases are not silently counted as freed storage; the borrower contract must be honored before physical-release claims. Test a request exactly at the cap and one whose rounded capacity exceeds it. Explain any difference between reduced application capacity and unchanged driver-visible usage.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

A capacity limit turns cache growth into an admission decision. Its meaning is strongest when the accounting boundary is explicit: this pool can control its own buffers, while the surrounding runtime must manage the rest of the process.

Discuss the throughput cost of a smaller cap and whether evictions mostly remove cold buffers or force repeated recreation of hot ones. Identify when waiting for pending work is cheaper than eviction and growth. This prepares the pressure-aware trimming policy without weakening the hard accounting invariant.

19.27.15 Exercise 14

Exercise Title: Implement Pool Trimming That Releases Long-Unused Buffers When Runtime Memory Pressure Crosses a High Watermark

Part 1 - Exercise Contract and Why It Matters

Implement pressure-triggered trimming of safely free buffers using idle age and a declared target. Add separate high and low watermarks so small fluctuations do not trigger repeated trimming and regrowth. Deliver a trace connecting pressure samples, victim selection, capacity reduction, and later misses.

Keep the hard cap from E13 independent of this policy. Trimming is a proactive retention decision; it cannot reclaim leased or pending storage. Use an explicit simulated-budget trace before introducing bounded real memory pressure.

Part 2 - System and Memory Mental Model

Choose the pressure signal deliberately. Pool occupancy, process allocated memory, allocator-reserved memory, and device free memory answer different questions. A reserved-memory ratio may remain high after pool trimming because the allocator caches freed blocks. A device-free signal also reflects other processes and can change between measurement and action.

Idle age begins when a buffer becomes safely free, not when its lease was acquired or when a host release moved it to pending. Track last use and free-since timestamps separately. A recently completed long operation is not necessarily a long-idle buffer.

Use hysteresis for policy state and a cooldown or minimum retention rule only when compatible with the hard cap. Under severe pressure, keeping a hot free buffer may be unaffordable, so document which constraints take precedence. Dropping references and releasing allocator cache are distinct interventions and should be timed separately. If no free victims exist, the correct response is waiting, admission restriction, or explicit failure, not eviction of active storage.

Part 3 - Quantitative Prediction Before Execution

For a declared byte-pressure measure $M(t)$ and budget $B(t) > 0$, define $p(t) = M(t)/B(t)$. Choose

$$0 < p_{\text{low}} < p_{\text{high}} < 1.$$

Enter trimming mode at the high watermark and leave it only after the low-watermark condition and any declared stability interval hold. Do not mix a numerator from one accounting layer with a budget from another.

For an idle-age threshold a, eligible victims satisfy

$$t - t_{j,\text{free}} \geq a, \qquad j \in \mathcal{F}.$$

Given hypothetical free buffers of 8, 16, and 32 MiB with idle ages 10, 2, and 20 seconds, predict which are eligible at a 5-second threshold. Select a declared policy to release at least 24 MiB and calculate its overshoot. Explain why eligibility alone does not determine victim order.

Part 4 - Experimental Design and Instrumentation

1. Define pressure units, watermarks, age threshold, polling interval, and trim target before implementation.

2. Replay a synthetic pressure trace crossing the high threshold once, oscillating near it, and later staying below the low threshold.

3. Populate the pool with hot and cold size classes, including pending buffers that must remain ineligible.

4. Measure trimming time, removed capacity, later misses, and changes in the pressure state.

5. Run a bounded CUDA pressure experiment using a declared managed allocation or controlled budget change. Record the mechanism explicitly.

6. Compare age-based trimming with an equally budgeted no-age policy and with no proactive trimming.

Use a monotonic host clock for age. Sample pressure at controlled points and log its age, since delayed observations can explain late reactions.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import time
import torch


@dataclass
class TrimConfig:
    high_ratio: float
    low_ratio: float
    minimum_idle_seconds: float
    cooldown_seconds: float


def pressure_observation(device, pool, budget):
    # Name the accounting layer and timestamp the observation.
    ...


def eligible_trim_victims(records, now, config):
    # Include only safely free records with sufficient age.
    ...
```

```
def choose_trim_target(observation, config):
    # Apply hysteresis without overriding the hard cap.
    ...


def apply_trim(pool, victim_ids):
    # Retire eligible records and reconcile released ownership.
    ...


def E14(config):
    # Replay threshold crossings and measure regrowth cost.
    ...
    # Explain allocated, reserved, and free-memory responses.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require that no leased or pending buffer is selected, that free-since age is measured correctly, and that repeated noisy pressure samples do not cause uncontrolled state changes. Show one trace in which trimming removes application capacity without immediately reducing reserved bytes.

Report bytes requested for trimming, bytes actually retired, overshoot, trim latency, state transitions, later growth count, and complete workload throughput. Inspect a no-victim case explicitly. A policy that reports success despite freeing nothing must distinguish target unmet from successful pressure relief. Verify that hard-cap enforcement still works during cooldown and that logging retains no evicted tensors.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Trimming is a prediction that idle capacity will be less valuable than the headroom it releases. Idle age is evidence for that prediction, not a proof about future demand.

Use the measured regrowth cost to judge the threshold. If trimming repeatedly removes hot capacity, a lower retention target may be too aggressive or the pressure signal may be mismatched to the goal. Explain how hysteresis stabilizes decisions while preserving a prompt response to a real shortage.

19.27.16 Exercise 15

Exercise Title: Detect Pool Bloat Where Cached Buffers Consume Far More VRAM Than the Active Working Set Requires

Part 1 - Exercise Contract and Why It Matters

Build diagnostics that distinguish legitimate warm capacity from sustained pool bloat. Analyze live demand, retained idle classes, pending lifetimes, and storage that remains alive through escaped references. Deliver a time-series report and a targeted mitigation supported by a before-and-after comparison.

Use a workload with a temporary large-shape phase followed by a smaller steady phase. Include a short quiet interval so ratio definitions must handle zero active bytes explicitly. Do not declare every cache above the instantaneous working set wasteful.

Part 2 - System and Memory Mental Model

Retaining capacity can prevent future allocation cost, but a pool may continue holding many obsolete classes after demand changes. A high hit rate during the smaller phase can coexist with a large idle inventory left by the earlier phase. Hit rate alone cannot detect this.

Separate four causes: active class slack, free cold buffers, long-pending buffers, and storage outside the pool's current registry that remains alive through references. The last case is an ownership or observability problem, not ordinary cache retention. Pending capacity may reflect legitimate slow consumers or a missing completion transition.

Instantaneous demand is too narrow a denominator for every diagnosis. Compare retained capacity with a recent demand window and with a known replay of future requests when evaluating offline. During zero demand, report retained bytes and quiet duration rather than dividing by zero or hiding the interval with an arbitrary denominator. Pool bloat also differs from allocator cache retention after tensors are released; keep application and allocator measurements separate.

Part 3 - Quantitative Prediction Before Execution

For positive requested live bytes $Q(t)$, define the diagnostic ratio

$$\beta(t) = \frac{C_{\text{pool}}(t)}{Q(t)}.$$

It is undefined at $Q(t) = 0$. A complementary time-window measure is

$$I_{\text{idle}} = \int_{t_0}^{t_1} C_{\text{free}}(t)\, dt,$$

with units byte-seconds. Compare it with latency saved by future hits, not with an unrelated count.

For a hypothetical pool retaining 512 MiB while live demand is 32 MiB, calculate the ratio. Then distinguish a one-millisecond dip in demand from a ten-minute low-demand phase. Construct a ledger separating 64 MiB pending, 384 MiB free, and the remaining materialized capacity. Explain what additional information is needed to calculate active slack and to classify the pending bytes as suspicious.

Part 4 - Experimental Design and Instrumentation

1. Run a three-phase trace: small steady requests, a large-shape burst, then the original small requests for a sustained interval.

2. Collect per-class capacity, free-since age, loan count, recent hits, pending duration, and requested live bytes.

3. Add a safe reference-retention diagnostic: retain one known tensor alias, retire its pool record after completed use, and observe storage lifetime until that alias is removed.

4. Compare unlimited retention, hard-cap-only retention, and age-based trimming under the same request trace.

5. Evaluate future miss cost and throughput after mitigation, so capacity reduction is not treated as the only objective.

6. Replay the trace with a later return to large shapes and measure whether the chosen mitigation was too aggressive.

Keep the deliberate alias confined to this diagnostic and document that it violates the normal no-use-after-release contract.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch


@dataclass
class ClassHealth:
    class_key: object
    requested_live_bytes: int
    free_bytes: int
    pending_bytes: int
    retained_bytes: int


def collect_class_health(pool, now):
    # Reconcile unique storage and ownership durations.
    ...


def diagnose_bloat(history, quiet_interval):
    # Separate ratios, idle retention, and pending anomalies.
    ...
```

```
def recommend_retention_change(diagnostics, config):
    # State expected capacity gain and future miss penalty.
    ...


def E15(config):
    # Replay demand shifts and a bounded alias diagnostic.
    ...
    # Compare memory relief with later recreation cost.
    ...
    # Explain zero-demand intervals without invalid ratios.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

The report must identify which category dominates retention and show evidence for the proposed cause. High capacity by itself is insufficient to claim a leak. A missing pending-to-free transition needs a completion and generation trace; an escaped reference needs an ownership trace.

Report retained bytes during quiet intervals, time-weighted idle capacity, recent demand, pending-age tails, hit rate, and later growth latency. Verify that mitigation preserves outputs and does not evict live storage. After releasing the diagnostic alias, compare allocated and reserved memory separately. Confirm that a lower bloat ratio is not merely the result of increasing active batch size and consuming more memory.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Useful caching is a bet on reuse, while bloat is retained capacity whose expected value no longer justifies its cost. The distinction depends on workload phases and the resource pressure faced by the surrounding runtime.

Explain whether a fixed age threshold, a capacity cap, or a demand window best addresses the measured cause. Avoid adding a complex retention policy to conceal a broken ownership transition. First prove the accounting and lifetimes, then optimize the cache's economic tradeoff between capacity and recreation cost.

19.27.17 Exercise 16

Exercise Title: Build a Shape-Aware Temporary Tensor Pool for Repeated Attention or Matrix-Multiplication Workspaces

Part 1 - Exercise Contract and Why It Matters

Adapt the pool to explicitly managed temporary tensors for repeated matrix multiplication or a transparent attention implementation. Define supported shapes, dtypes, layouts, and lifetimes, then compare pooled scratch with ordinary temporary allocation under identical mathematics. Deliver numerical validation and a breakdown of which allocations the pool actually controls.

Start in inference or a no-gradient region using operations whose documented output-buffer behavior is suitable. This pool cannot supply arbitrary internal workspaces to opaque cuBLAS, cuDNN, or fused attention operators through ordinary tensor reuse. A training extension must separately prove autograd compatibility and saved-tensor lifetime.

Part 2 - System and Memory Mental Model

A shape-aware key can include dtype, device, capacity, contiguous layout, and the operation's output contract. Shape equality is sufficient for some reuse but not necessary when a larger typed contiguous buffer can provide an appropriate view. Conversely, equal element count is insufficient if an operator requires a different stride or layout.

Operations with an `out=` argument must be checked individually for supported shape, dtype, aliasing, and autograd behavior. Many such operations do not support automatic differentiation when participating inputs require gradients. Pre-size supported outputs so implicit resizing does not invalidate the pool's capacity accounting. Never use `.data` or bypass version checks to force unsafe reuse.

Autograd may save an input or intermediate for backward even when the forward computation is complete. Overwriting it can trigger a version-counter error or invalidate differentiation if checks are bypassed. A scratch buffer is reusable only after its last semantic consumer, not simply after the kernel that wrote it.

Explicit attention materializes tensors that a fused backend may avoid. Comparing those two implementations confounds pooling with algorithm and backend selection. The primary pooling comparison must use the same operator structure and materialization policy.

Part 3 - Quantitative Prediction Before Execution

For a matrix product with output $M \times N$ and element size s, predict output bytes

$$q_{\text{out}} = MNs.$$

For explicitly materialized attention scores with batch B, heads H, and sequence length S, predict

$$q_{\text{scores}} = BHS^2 s.$$

This describes that explicit score tensor, not the peak of every attention backend. Peak scratch depends on overlapping lifetimes of scores, probabilities, outputs, and any other temporary storage.

For hypothetical $B = 2$, $H = 8$, $S = 512$, and float32 scores, calculate score bytes and the increase when S doubles. Draw a lifetime chart to determine whether two equally sized intermediates can share one buffer under the chosen operations. Explain what changes if backward still requires the earlier intermediate.

Part 4 - Experimental Design and Instrumentation

1. Choose an explicit matrix-product or attention computation and document which tensors are pooled and which remain operator-owned.

2. Build a reference with ordinary allocation and a pooled variant using the same operations, precision settings, and initialization.

3. Validate supported output-buffer behavior and reject unsupported gradient-enabled or layout cases before execution.

4. Sweep repeated shapes and variable shapes, recording logical scratch bytes, backing capacity, process peaks, and complete latency.

5. Keep all scratch leases until their last consumer completes. Check aliases before using an output as another operation's input.

6. As an optional training study, use a separate supported design and compare gradients with a reference; do not assume inference scratch rules extend to backward.

If comparing fused attention separately, label it an algorithm/backend comparison and report its different materialization behavior.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch


@dataclass(frozen=True)
class WorkspaceSpec:
    operation: str
    shape: tuple
    dtype: torch.dtype
    device: str
    layout: str


def validate_workspace_contract(spec, inputs, grad_enabled):
    # Reject unsupported output, alias, layout, or grad cases.
    ...
```

```
def plan_workspace_lifetimes(operation_graph):
    # Identify last consumers and overlapping storage needs.
    ...


def run_reference(inputs, config):
    # Use ordinary allocation with the chosen operator graph.
    ...


def run_with_workspaces(inputs, pool, config):
    # Acquire compatible views and preserve their lifetimes.
    ...


def E16(config):
    # Validate outputs before comparing shapes and latency.
    ...
    # Distinguish managed scratch from hidden operator memory.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require numerical agreement with predeclared absolute and relative tolerances appropriate to dtype and operation. Report maximum error and non-finite values, not just a pass flag. The primary comparison must use identical backend and precision settings.

Audit actual backing storage before and after output operations to detect implicit reallocation. Reject unsupported layouts instead of silently cloning them and omitting the extra allocation from accounting. For any training extension, validate gradients and parameter updates while preserving saved tensors until backward completes.

Report requested scratch, retained capacity, unmanaged peak contribution where measurable, and complete workload latency. State that a reduction in application scratch requests does not prove elimination of internal library workspaces. Include the largest tested shape and all rejected or unsupported cases.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Shape-aware pooling is most effective where the application owns both the temporary storage and its lifetime. Opaque operators may already use specialized memory strategies that a Python-level pool cannot replace.

Explain whether the measured opportunity comes from repeated output storage, staging, or explicit intermediates. If a fused implementation uses less memory, that does not invalidate the pooling experiment; it changes the best engineering choice for the application. Keep algorithm selection, precision, and storage reuse as separately measured decisions.

19.27.18 Exercise 17

Exercise Title: Compare Static Preallocation, Dynamic Pooling, and Ordinary PyTorch Allocation Under Stable and Variable Workloads

Part 1 - Exercise Contract and Why It Matters

Benchmark three complete allocation strategies: fixed static buffers, dynamic pooling, and ordinary PyTorch tensor allocation. Compare stable shapes, variable shapes, and phase changes under matched resource limits and useful work. Deliver a decision table describing where each strategy is feasible and where it is competitive.

Specify static capacity before running. If a later request exceeds that capacity, record rejection or a declared fallback and charge its cost. Dynamic pooling must include growth, lookup, event management, and trimming overhead. Ordinary allocation retains the normal caching allocator behavior unless a separate experiment explicitly changes it.

Part 2 - System and Memory Mental Model

Static preallocation offers predictable application capacity and low management work when shapes and concurrency are known. It can waste memory when provisioned for rare maxima or fail when an unseen shape exceeds the contract. Dynamic pooling adapts its inventory but can accumulate classes, introduce search overhead, and require complex ownership tracking.

Ordinary allocation delegates reuse to PyTorch's allocator and keeps the application simpler. It may already perform well enough that a custom pool has no practical benefit. The three strategies operate above the same CUDA hardware but expose different application-level lifetimes and metadata costs.

Fairness requires matching peak in-flight work, initialization, output retention, and stream synchronization. A static buffer reused by one serialized consumer is not directly comparable to a dynamic pool supporting four overlapping consumers. Likewise, a strategy that rejects difficult requests cannot claim higher throughput by counting only the requests it happened to accept.

Part 3 - Quantitative Prediction Before Execution

For strategy s, decompose startup-inclusive time as

$$T_s = T_{s,\text{setup}} + T_{s,\text{useful}} + T_{s,\text{management}} + T_{s,\text{waits}} + T_{s,\text{cleanup}}.$$

This is an accounting decomposition of non-overlapping wall-time categories; do not add overlapping stage timings as though they were serial costs.

For a static design with m slots of maximum capacity $c_{\max}$, predict $C_{\text{static}} = mc_{\max}$. Compare it with the dynamic inventory required by a specific trace, including retained old classes. Use a hypothetical workload with 95 requests of 1 MiB and five requests of 32 MiB. Explain why frequencies alone are insufficient to determine peak capacity: request overlap and release order are also needed. Predict which strategy is favored under a 16 MiB cap if every request must be completed unchanged.

Part 4 - Experimental Design and Instrumentation

1. Define one supported consumer and one request-trace format for all strategies. Include explicit concurrency and completion events.

2. Create stable, jittered, broad-size, and phase-shift traces. Reserve one trace for evaluation after policy calibration.

3. Set equal application resource limits and disclose unmanaged process memory. Define feasibility and rejection rules consistently.

4. Measure startup-inclusive and warmed complete execution, plus management-only diagnostics in separate runs.

5. Validate outputs and request coverage before comparing throughput. Report incomplete or infeasible runs with their failure point.

6. Compare latency distributions, pool or static capacity, allocated and reserved peaks, and implementation complexity.

Use fresh processes or a documented balanced warmup strategy to control allocator history. Do not clear the cache inside only the ordinary path.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch


@dataclass
class StrategyOutcome:
    name: str
    accepted_requests: int
    completed_requests: int
    rejected_requests: int
    elapsed_seconds: float
    capacity_bytes: int


def build_strategy(name, specification):
    # Establish static, dynamic, or ordinary allocation rules.
    ...


def replay_common_workload(strategy, trace, consumer):
    # Preserve concurrency, initialization, and request coverage.
```

```
    ...


def evaluate_feasibility(outcome, trace):
    # Keep rejection and incomplete work visible.
    ...


def E17(config):
    # Calibrate on one trace and evaluate held-out workloads.
    ...
    # Compare startup-inclusive and steady-state outcomes.
    ...
    # Produce a conditional strategy decision table.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Acceptance requires matching numerical results for completed requests and full accounting for every rejected or unfinished request. Report throughput using a clearly defined workload boundary; never compare successful-only subsets with complete-trace execution without labeling the difference.

Include strategy, trace, budget, startup time, complete elapsed time, median and tail request latency, completed coverage, capacity peak, allocated peak, and reserved peak. Show at least one capacity or shape-contract limitation of static preallocation. Investigate dynamic pool results that improve hits but worsen latency. A finding that ordinary allocation is best is acceptable when the comparison is fair and repeatable.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

The best storage strategy depends on predictability, concurrency, capacity pressure, and maintenance cost. Static allocation moves decisions to configuration time; dynamic pooling makes them online; ordinary allocation delegates more of them to the framework.

Choose a strategy for each measured workload regime and state the constraint that drives the choice. Include a condition that would change your recommendation, such as a tighter cap or an altered shape distribution. Engineering judgment here is the ability to select an appropriately simple mechanism from evidence.

19.27.19 Exercise 18

Exercise Title: Stress-Test the Pool with Thousands of Random Tensor Sizes and Diagnose Excessive Misses, Internal Waste, Stale Buffers, and Synchronization Hazards

Part 1 - Exercise Contract and Why It Matters

Build a reproducible stress harness that generates thousands of requests with explicit sizes, dtypes, lifetimes, and consumer patterns. Exercise the pool's invariants under bounded capacity rather than allocating thousands of simultaneously live tensors indiscriminately. Deliver replayable seeds, minimized failure traces, and a diagnosis for every observed violation.

Separate host-only adversarial metadata tests from CUDA correctness tests. The simulator should reject unsafe transitions before they become real races. Test both ordinary randomized traces and targeted adversarial sequences; randomness alone is unlikely to cover all generation and completion bugs.

Part 2 - System and Memory Mental Model

Stress dimensions interact. A broad size distribution increases class diversity; overlapping lifetimes increase required concurrency; delayed completion increases pending capacity; and bursty demand exposes growth-reservation races. Stale handles and double releases attack ownership invariants even when sizes are constant.

Generate request IDs and buffer generations independently. Data patterns should identify the request that wrote the logical region so stale contents cannot pass a shape-only check. Every read region must be initialized in the valid implementation. An intentionally omitted initialization can be demonstrated sequentially as a safe negative control, not by racing a real writer against a reader.

Use bounded event deadlines and distinguish a slow workload from a missing completion transition. After each phase, drain and audit capacity, then compare against the pool's documented retention policy. Reserved memory remaining after drain is not by itself evidence of stale live tensors. A useful failure report identifies whether the problem is class selection, ownership, completion, capacity accounting, or an external allocation failure.

Part 3 - Quantitative Prediction Before Execution

For N requests and state transitions indexed by t, define

$$V = \sum_{t} \mathbf{1}\{\text{an invariant fails at } t\}.$$

The correctness target is $V = 0$, not merely a high successful-request percentage. Budget rejection is a valid outcome when policy requires it.

Predict peak live demand from an explicit lifetime trace:

$$Q_{\text{peak}} = \max_{t} \sum_{i \in \mathcal{L}(t)} q_i.$$

This is a lower bound on required application capacity before class slack, pending storage, and retained free buffers.

Design a hypothetical 10,000-request campaign with a bounded live set, a rare large-size burst, and delayed consumers. State expected failure signals for a deliberately broken double-release checker, a missing consumer event, and a too-coarse class policy. Only the first two necessarily violate correctness; excessive waste is a policy outcome.

Part 4 - Experimental Design and Instrumentation

1. Create a seeded CPU trace generator with explicit acquire, use, close, completion, and invalid-operation events.

2. Run a reference state model and the pool's metadata logic against the same trace. Stop on the first invariant divergence.

3. Add targeted cases: boundary sizes, zero requests, oversize requests, stale generations, duplicate callbacks, growth failure, and cap shrink.

4. Run supported CUDA traces with bounded simultaneous bytes and request-specific content checks after completion.

5. Vary stream count, polling interval, size distribution, and pressure independently before combining them in a final campaign.

6. Reduce any failure to a smaller replay trace while preserving the original seed, configuration, and first failing event.

Keep diagnostic logging bounded and tensor-free. Capture allocator statistics or a memory snapshot only in a designated diagnostic run, with installed-version API support recorded.
Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import random
import torch


@dataclass
class StressAction:
    action_id: int
    request_id: int
    kind: str
    requested_bytes: int
    generation: int


def generate_trace(seed, request_count, constraints):
    # Bound live demand and include targeted corner cases.
    ...
```

```
def reference_transition(state, action):
    # Implement the independent ownership and budget model.
    ...


def run_metadata_campaign(pool, trace):
    # Reject unsafe actions before any CUDA submission.
    ...


def run_cuda_campaign(pool, trace, config):
    # Validate initialized payloads and completion ordering.
    ...


def minimize_failure(trace, failure_predicate):
    # Preserve the first reproducible invariant violation.
    ...


def E18(config):
    # Execute seeded campaigns and export replay artifacts.
    ...
    # Classify correctness failures separately from poor policy.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report seeds, request count, accepted and rejected counts, invariant violations, output mismatches, peak capacity, pending-age tails, hit rate, waste, and total runtime. A passing run must also drain cleanly and reconcile the final registry. Zero observed failures means only that the tested campaign passed, not that all possible schedules are proven safe.

For each injected metadata bug, demonstrate that the harness detects it without relying on random GPU corruption. For poor-policy cases, show measurable excess misses or waste rather than labeling them memory safety failures. Preserve the smallest failing trace and the last valid ledger. Explicitly distinguish pool-cap rejection, allocator OOM, unsupported requests, and synchronization errors.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Stress testing is strongest when randomness explores a space defined by precise invariants. A large request count is less valuable than a smaller campaign that can reproduce and explain its failures.

Use the results to prioritize fixes by correctness, capacity compliance, and performance. Repair ownership and completion defects before tuning hit rate. Identify a remaining untested dimension, such as external libraries or cross-device transfers, and keep it outside the manager's claimed support contract until it receives its own evidence.

19.27.20 Exercise 19

Exercise Title: Build and Benchmark a PyTorch Memory Pool Manager That Reuses CPU and CUDA Buffers Safely Across Dynamic Workloads Under Fixed RAM and VRAM Budgets

Part 1 - Exercise Contract and Why It Matters

Integrate the chapter's components into a single manager with typed requests, exclusive leases, compatible size classes, event-aware release, independent host and device budgets, bounded growth, trimming, and replayable diagnostics. The deliverable is a student-completed runtime and a benchmark dossier; the scaffold below intentionally leaves all policy and execution logic unfinished.

Define the support contract before implementation: devices, dtypes, layouts, inference or training-safe buffer roles, host pinning, stream registration, maximum request size, timeout behavior, and shutdown. Require correct outputs and capacity accounting before optimizing throughput. The runtime must explicitly reject unsupported requests instead of silently allocating untracked fallback storage.

Part 2 - System and Memory Mental Model

Separate the manager into request validation, class selection, admission, storage ownership, completion tracking, retention policy, and observation. These modules share a canonical ledger but answer different questions. A suitable-size buffer is not necessarily ready; a ready buffer may belong to the wrong device; a new compatible buffer may exceed the cap.

Track pageable host capacity and pinned host capacity as components of the manager's total host budget, with an optional additional pinned sub-limit. Track CUDA capacity separately for each supported device. Do not double-count pinned capacity in total host use. Process RSS and device free-memory observations remain external signals because datasets, model state, operator temporaries, and other processes are outside the manager's ownership boundary.

Acquisition must publish exactly one valid lease or a well-defined failure. Growth reservations are atomic and become materialized capacity only after successful creation. Release closes consumer registration, then waits through a pending state until the completion proof is valid. Eviction can retire only safely free records. Generation IDs protect metadata against stale releases but do not revoke escaped tensor aliases.

For training, manage only buffers whose autograd lifetime is explicitly supported; do not overwrite saved tensors or assume output-buffer operators are differentiable. OOM handling must preserve submitted work and ledgers. Some CUDA errors require terminating the experiment; the manager must not turn every runtime error into a retry. Dynamic external pressure cannot be eliminated by an internal pool cap.

Part 3 - Quantitative Prediction Before Execution

For resource r, require the fixed-cap admission invariant

$$C_{r,\text{leased}} + C_{r,\text{pending}} + C_{r,\text{free}} + G_r \leq B_r.$$

Here G_r counts only unmaterialized growth promises. Define host capacity without double counting:

$$C_{\text{host}} = C_{\text{pageable}} + C_{\text{pinned}}.$$

For a declared useful unit such as completed batches or valid tokens, measure

$$\text{throughput} = \frac{\text{completed useful units}}{\text{complete elapsed seconds}}.$$

Report request latency and byte traffic separately.

Before execution, construct a capacity plan for a hypothetical pipeline with 16 MiB payloads, three pinned source slots, two CUDA destination slots, and two additional 8 MiB scratch buffers. Allocate scratch to the appropriate resource and state all outside-manager memory. Predict feasibility under several independent host and device caps. Add delayed consumers and explain why lower nominal request size need not reduce pending capacity immediately.

Part 4 - Experimental Design and Instrumentation

1. Write a support matrix and invariants, then connect the components through typed requests and generation-bearing leases.

2. Validate capacity and ownership in the host simulator before CUDA integration. Inject reservation failure and duplicate callbacks.

3. Integrate a pinned-to-CUDA staging pipeline and a supported explicit scratch workload. Preserve complete consumer registration.

4. Benchmark ordinary allocation, static preallocation, and the manager on stable, variable, phase-shift, and bursty traces.

5. Sweep host and device caps independently. Record infeasible configurations and exact refusal reasons.

6. Add delayed completion, bounded pressure, and repeated growth/trim cycles. Measure tail latency and useful throughput after warmup as well as startup-inclusive cost.

7. Run ablations for rounding, fit policy, event polling, and trimming. Change one mechanism at a time on the same trace.

8. Perform graceful shutdown: stop admission, close or reject remaining registrations, drain submitted work, reconcile ledgers, and retire eligible storage.

Calibrate policies on one trace and evaluate them on held-out traces. Keep numerical precision and operator backend fixed during pooling comparisons. Publish the completed-request count and failure ledger with every timing result.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import threading
import torch


@dataclass(frozen=True)
class BufferRequest:
    request_id: int
    shape: tuple
    dtype: torch.dtype
    device: str
    pinned: bool
    role: str


@dataclass
class ResourceBudget:
    host_bytes: int
    pinned_limit_bytes: int
    device_limits: dict


class MemoryPoolManager:
    def __init__(self, budgets, policy, support_contract):
        # Establish registries, counters, locks, and resource caps.
        ...

    def validate_request(self, request):
        # Check supported layout, dtype, role, and capacity.
        ...

    def acquire(self, request, producer_stream=None):
        # Select ready storage or reserve budgeted growth.
        ...

    def register_consumer(self, lease, consumer, ready_event):
        # Register host or device use under the lease contract.
        ...

    def close_lease(self, lease):
        # Seal registration and establish pending completion.
        ...
```

```
    def poll_completions(self, work_limit):
        # Reclaim completed generations without duplicate refunds.
        ...

    def trim(self, pressure_observation):
        # Retire only eligible free storage under declared policy.
        ...

    def handle_creation_failure(self, reservation, error):
        # Roll back accounting and classify the failure.
        ...

    def audit(self):
        # Verify ledgers and declared ownership/completion records.
        ...

    def shutdown(self, deadline):
        # Stop admission, drain work, and report incomplete cases.
        ...


def benchmark_manager(manager_factory, traces, references):
    # Preserve useful work and compare all allocation strategies.
    ...


def E19(config):
    # Validate support contracts and simulator invariants first.
    ...
    # Run CUDA correctness, benchmark, ablation, and stress phases.
    ...
    # Export evidence, limitations, and a justified policy choice.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Acceptance has four independent gates. First, every accepted request must produce correct data under its supported operation contract. Second, ownership must remain exclusive and reuse must follow all declared completion evidence. Third, each fixed resource cap must hold for materialized capacity plus growth reservations. Fourth, the benchmark must account for all requested work, refusals, waits, and failures. A faster result cannot compensate for failure of another gate.

Provide a dossier containing environment metadata, support matrix, policy configuration, request traces, seed list, per-resource capacity timeline, latency distributions, useful throughput, transfer bytes, hit/miss definitions, live slack, pending ages, trimming cost, and allocator observations. Include numerical error summaries for floating point workloads and exact request-identity checks for staging.

Inspect shutdown with active work, missing completion evidence, stale leases, and growth failure. Report any remaining references that keep storage alive after manager retirement.

Distinguish application release, allocator cache retention, and device-visible memory relief. Preserve negative results and identify the smallest configuration in which the manager fails to improve on ordinary allocation.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use an explicit not-measured value instead of inventing hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

The completed manager should make a precise promise: it controls the storage it owns under a documented lifetime and budget contract. That promise is more useful than an unsupported claim to eliminate all allocation overhead or all CUDA OOM events.

Use the benchmark to decide which components justify their cost for the target workload. Some applications need only static staging, while others benefit from event-aware dynamic reuse and proactive trimming. Explain the tradeoff among retained capacity, tail latency, CPU bookkeeping, and implementation complexity. End with an evidence-based deployment recommendation and the next unsupported condition that would need validation before broadening the manager's contract.

19.28 DMM26

Chapter Title: Memory Fragmentation Management
Specialization: Dynamic Memory Management and CUDA Systems

19.28.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Internal Fragmentation, External Fragmentation, Contiguous Allocation Requirements, and CUDA Memory Waste

Part 1 - Exercise Contract and Why It Matters

Build an accounting and lifetime model that distinguishes internal fragmentation, external fragmentation, reusable cache, and genuinely live demand. Deliver a labeled segment diagram, a quantitative example, and a small PyTorch observation that tests the model. Your explanation must identify the layer at which each claim is made: application buffer, PyTorch allocator block, allocator segment, or CUDA virtual memory.

Use one CUDA device and a documented allocator backend. Begin outside autograd with bounded contiguous tensors. Explain a case with substantial reserved memory that is harmless and a case where scattered inactive space can obstruct reuse. Do not define all unused memory as fragmentation.

Part 2 - System and Memory Mental Model

Internal fragmentation is unused capacity inside an assigned allocation: an application may request q useful bytes while a rounded buffer or allocator block occupies $c > q$. External fragmentation concerns free regions whose placement or eligibility prevents satisfying a particular request, despite an apparently adequate aggregate. Both definitions need an explicit allocation layer and request.

The native CUDA caching allocator can split segments into blocks and reuse inactive blocks. Live neighbors can prevent separate holes from coalescing into a larger reusable block. Entire inactive ordinary segments may be releasable, whereas a partially occupied segment can retain inactive space. Stream dependencies can also delay reuse after Python releases a tensor; delayed completion is not automatically structural fragmentation.

A contiguous tensor has a layout and backing storage contract. This does not require physically adjacent VRAM chips or a single run of physical pages. CUDA addresses are virtual, and allocation backends can obtain new segments or use virtual-memory mechanisms. Therefore the largest cached inactive block is not a universal upper bound on the next successful allocation.

Reserved minus allocated bytes can include reusable cache, split holes, and pending activity. A pool from DMM25 keeps idle tensors alive, so its free buffers may still count as allocated at the PyTorch layer. Do not add overlapping accounting categories or identify a leak from a single large reserved-memory observation.

Part 3 - Quantitative Prediction Before Execution

For allocated blocks in a declared set, define internal slack

$$W_{\text{int}} = \sum_i (c_i - q_i), \qquad \eta = \frac{\sum_i q_i}{\sum_i c_i}.$$

For a simplified fixed arena with eligible free holes h_j, define

$$F = \sum_j h_j, \qquad L = \max_j h_j.$$

A request with $L < q \leq F$ cannot fit in one existing hole in that arena. The conclusion assumes no growth, remapping, or live relocation. It is not a complete model of PyTorch's allocation path.

Draw a hypothetical 64 MiB arena with live blocks separating free holes of 8, 12, and 4 MiB. Test a 16 MiB request and explain the difference between aggregate capacity and immediate placement. Then allow a new segment and reconsider the prediction. For an independent 10 MiB logical request placed in a 16 MiB application buffer, calculate internal slack and state why it should not be added to the arena's free holes.

Part 4 - Experimental Design and Instrumentation

1. Construct a paper ledger with requested, assigned, live, pending, inactive, and reserved categories. Mark which categories overlap.

2. Inspect bounded PyTorch allocations before creation, while live, after completed release, and after a separate cache-release intervention.

3. Collect current allocated and reserved bytes plus device-free bytes. Compare these with a native allocator snapshot when supported.

4. Classify the largest inactive block as an observation with device, stream, pool, and timestamp scope, not as guaranteed allocation capacity.

5. Explain the difference between a noncontiguous tensor view and fragmented allocator free space using two independent examples.

6. Record what evidence would distinguish capacity shortage, long-lived references, and split-block trapping.

Preserve the exact backend and version. If snapshot semantics differ from the simplified arena, revise the mapping rather than forcing the observation to match the diagram.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch


@dataclass
class MemoryView:
    allocated_bytes: int
    reserved_bytes: int
    driver_free_bytes: int
    backend: str
    observation_time: float


def explain_arena(blocks, requested_bytes):
    # Compute layer-specific capacity and placement quantities.
    ...


def capture_memory_view(device):
    # Keep allocator and driver observations separate.
    ...


def classify_snapshot_blocks(snapshot, scope):
    # Identify live, pending, and inactive records.
    ...


def E00(config):
```

```
# Write predictions before observing PyTorch behavior.
...
# Compare the fixed-arena model with real allocator growth.
...
# Export a diagram and a reconciled accounting ledger.
...
```

Part 6 - Measurement, Validation, and Failure Analysis

Acceptance requires a correct distinction between virtual contiguity, tensor layout, and physical placement. Show why reserved bytes are not equivalent to currently free bytes. Include a counterexample to the claim that a large reserved-minus-allocated gap always signals damage.

Validate the arithmetic of the hypothetical arena independently of CUDA. For real observations, record whether uses have completed and whether references remain. Explain any unknown category instead of forcing a false conservation identity. No conclusion about unseen physical pages or allocations made directly by another library can be established from a PyTorch snapshot alone.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use explicit not-measured and unsupported values. A fragmentation pattern or OOM that does not reproduce is an experimental outcome, not permission to invent hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Fragmentation is a relationship between a request and the organization of available memory. A scalar utilization percentage cannot describe that relationship completely.

Use the model to state a testable diagnosis rather than a slogan: which request, which existing blocks, which live neighbors, and which growth constraint matter? This discipline connects DMM25's explicit buffer ownership to the allocator behavior below it and prevents later mitigation policies from optimizing a misleading metric.

19.28.2 Exercise 01

Exercise Title: Create a Controlled CUDA Allocation Pattern That Leaves Free Memory Split Across Noncontiguous Blocks

Part 1 - Exercise Contract and Why It Matters

Design an allocation-and-release trace intended to leave inactive blocks separated by live blocks within native allocator segments. Verify the result using allocator evidence rather than assuming that alternating Python list entries correspond to neighboring storage. Deliver the request trace, segment maps before and after selected releases, and a statement of whether the intended pattern actually formed.

Run the experiment in a fresh worker process with a bounded memory allowance. Keep initial experiments on one stream with completed uses. Use a host-only fixed-arena simulator as a deterministic conceptual control, labeled separately from real CUDA allocator behavior.

Part 2 - System and Memory Mental Model

Allocation order is not a promise about address order or segment membership. PyTorch may reuse older blocks, choose different pools, split a larger cached block, or obtain a new segment. If each tensor receives its own segment, releasing alternating tensors can simply produce independently releasable segments rather than trapped holes.

One candidate construction is to seed reusable capacity, then issue smaller requests and retain selected neighbors. Whether the allocator actually subdivides that capacity must be confirmed from the snapshot. Use sizes calibrated to the installed backend and a fresh allocator history. Do not hard-code an implementation threshold as an eternal property of CUDA.

Dropping a list entry removes only one reference. Views, temporary variables, closures, or exception tracebacks may keep storage live. After all relevant uses complete, inactive blocks adjacent inside the same compatible segment can coalesce; holes separated by live blocks cannot be assumed to merge. Separate segments do not become one tensor merely because their sizes add up.

Part 3 - Quantitative Prediction Before Execution

For a known segment of size S, construct a predicted map with alternating live capacities a_i and holes h_j:

$$S = \sum_i a_i + \sum_j h_j$$

for a fully represented fixed-arena model. Predict total inactive bytes, largest hole, and releasability before touching CUDA.

Use a hypothetical 48 MiB segment divided into six equal blocks. Release the second, fourth, and sixth blocks while retaining the others. Calculate inactive total and largest hole, then predict the effect of releasing the third block. State the adjacency and coalescing assumptions behind the result. In the real experiment, compare segment IDs and block addresses before using the same arithmetic.

Predict a contrasting outcome when the six requests occupy six separate segments. This control demonstrates why identical Python lifetime patterns can lead to different allocator states.

Part 4 - Experimental Design and Instrumentation

1. Implement the trace in a fixed-arena simulator and verify the expected holes and coalescing transitions.

2. Launch a fresh native-backend CUDA worker. Warm initialization outside the trace and record the initial allocator state.

3. Try a bounded seed-and-subdivide pattern. Capture a snapshot after allocation and check actual segment membership.

4. Release selected tensors after completed use, clearing every intended reference. Capture the resulting block map.

5. Release one separating live neighbor and inspect whether inactive regions merge as predicted.

6. Repeat with a few calibrated request scales and record both successful and unsuccessful constructions.

Keep addresses as diagnostic labels, not permanent storage identities. Record allocation generations because the same address can be reused. Stop the bounded search if the intended pattern does not reproduce.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch


@dataclass
class AllocationAction:
    action_id: int
    operation: str
    tensor_id: int
    requested_bytes: int


def build_candidate_trace(scale_bytes, live_pattern):
    # Specify allocations and complete reference releases.
    ...


def simulate_fixed_arena(trace, arena_bytes):
    # Model adjacency and coalescing with explicit assumptions.
    ...


def replay_cuda_trace(trace, config):
    # Capture actual segment membership at chosen boundaries.
    ...


def verify_split_pattern(snapshot, expected_roles):
    # Prove separation by live neighbors in observed segments.
    ...
```

```
def E01(config):
    # Calibrate within a bounded allowance and attempt count.
    ...
    # Report reproduced, not reproduced, or unsupported.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require a segment map showing at least two inactive regions separated by live storage before claiming success. A list of tensor addresses without segment membership is insufficient. Distinguish a block still awaiting stream completion from a truly inactive hole.

Show requested sizes, actual block capacities, segment size, retained neighbors, inactive total, and largest observed inactive block at each boundary. If the allocator obtained separate segments or immediately coalesced the holes, report that outcome and explain it. Do not label the simulator's deterministic result a measured PyTorch fragmentation result. Confirm that final cleanup releases all intended tensor references.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use explicit not-measured and unsupported values. A fragmentation pattern or OOM that does not reproduce is an experimental outcome, not permission to invent hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Controlled experiments require control over the variable being claimed. Python allocation order alone gives less placement control than a fixed-arena model suggests.

Explain which parts of the trace were deterministic and which depended on allocator policy. The verified pattern becomes a useful fixture for subsequent diagnostics. An unsuccessful construction is also valuable evidence about the allocator's reuse and coalescing behavior on the tested configuration.

19.28.3 Exercise 02

Exercise Title: Trigger an Allocation Failure Even When Total Free or Reserved Memory Appears Sufficient and Explain the Fragmentation Mechanism

Part 1 - Exercise Contract and Why It Matters

Attempt a bounded fragmentation-related allocation failure and build an evidence chain distinguishing it from ordinary capacity exhaustion. Capture the requested size, pre-attempt block map, live references, growth conditions, allocator retries, and final outcome. A successful large allocation is a valid outcome if the allocator can grow or reclaim capacity; the exercise must explain that result rather than hide it.

Use the verified pattern from E01, a fresh worker, and an explicit attempt limit. Include a capacity-only control. Any imposed process allocation limit or pressure allocation must be recorded as part of the cause, not presented as physical VRAM fragmentation.

Part 2 - System and Memory Mental Model

An apparent contradiction often comes from comparing unlike quantities. Reserved bytes include live tensors; driver-free bytes exclude allocator cache; and total inactive bytes may be split across unsuitable blocks. No one of these scalars guarantees that the next request can succeed.

If no cached block fits, the allocator may request another segment, release eligible cached segments, and retry. Thus a request larger than every cached hole can still succeed. Conversely, live neighbors can trap free regions while growth is constrained by actual capacity, an explicit allocator limit, or external pressure. A post-failure snapshot may already reflect reclamation and differ from the state that motivated the attempt.

Catch the specific CUDA OOM exception supported by the installed PyTorch version. Do not convert unrelated runtime failures into OOM. Preserve scalar diagnostics without retaining traceback frames and their tensor references. An OOM is not proof that the CUDA context is unusable, but illegal memory accesses and other device errors need different handling. This isolated experiment performs no optimizer update, so it avoids partial training-state recovery.

Part 3 - Quantitative Prediction Before Execution

For the simplified non-growing arena, choose a request satisfying

$$L_{\text{cached}} < q \leq F_{\text{cached}}.$$

This establishes lack of a single fitting cached hole only within the declared eligibility model. Real failure additionally needs an explanation for unsuccessful growth and reclamation.

For hypothetical cached holes of 24 and 24 MiB, separated by live neighbors, analyze a 40 MiB request. Consider three independent cases: sufficient driver headroom for a new segment, insufficient headroom, and a lower configured allocator cap. Predict the likely outcome under each set of assumptions and list the observations needed to test it. Calculate why quoting the entire reserved total would overstate reusable capacity.

Part 4 - Experimental Design and Instrumentation

1. Reproduce and verify the split-block fixture before attempting the larger request. Record all live holders and inactive candidates.

2. Sample allocator counters and device-free bytes immediately before the attempt. Preserve an event-history window if supported.

3. Attempt one declared allocation. Record success or typed OOM and the complete exception text as evidence.

4. Capture post-attempt counters and state. Identify changes caused by allocator reclamation or retry.

5. Repeat in a fresh worker with the same live demand but a less fragmented construction, then with an intentionally impossible capacity-only request.

6. Compare outcomes without increasing memory pressure indefinitely just to force an OOM.

If using an allocator memory-fraction limit, document its accounting scope and supported backend. A refusal under that limit is not equivalent to exhausting every free byte on the device.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch


@dataclass
class AttemptEvidence:
    attempt_id: int
    requested_bytes: int
    outcome: str
    exception_text: str
    pre_state: object
    post_state: object


def establish_verified_pattern(config):
    # Reuse E01 evidence rather than assuming placement.
    ...


def attempt_large_request(request, diagnostics):
    # Catch only the supported CUDA OOM exception.
    ...


def compare_failure_mechanisms(evidence, controls):
    # Separate placement limits, capacity, and imposed caps.
    ...


def cleanup_attempt(handles):
    # Release dead references without discarding evidence.
    ...
```

```
def E02(config):
    # Execute bounded attempts in fresh isolated workers.
    ...
    # Explain successful allocations as well as failures.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Claim fragmentation-related failure only when the evidence links unsuitable cached regions, live separators, and a failed path to additional usable capacity. A large reserved-minus-allocated gap plus an exception is insufficient by itself. Use supported, inconclusive, and contradicted classifications for the diagnosis.

Report pre- and post-attempt states separately, with requested bytes, driver-free observation age, live bytes, inactive split bytes, candidate hole sizes, retry deltas, and exception category. The capacity-only control should fail for an independently explainable reason. If no real OOM reproduces, retain the fixed-arena demonstration and explain how the actual allocator avoided the predicted obstruction.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use explicit not-measured and unsupported values. A fragmentation pattern or OOM that does not reproduce is an experimental outcome, not permission to invent hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Failure analysis is strongest when it explains why an apparent solution was unavailable. In this case the missing explanation is often the allocator's ability to grow, reclaim, or remap memory.

Use the controls to state the narrowest supported claim. An allocation limit can create a useful controlled experiment, but it becomes part of the mechanism. The goal is a reproducible diagnosis that guides mitigation, not an impressive OOM message without a causal account.

19.28.4 Exercise 03

Exercise Title: Use torch.cuda.memory_stats() to Track Inactive Split Bytes, Segment Count, Allocation Retries, and Other Fragmentation Signals

Part 1 - Exercise Contract and Why It Matters

Build a backend-aware statistics collector and interpret its fields without turning every counter into a fragmentation score. Produce a time series aligned with allocation and release phases, a dictionary of metric meanings, and a reconciled comparison with one snapshot. The collector must preserve missing or unsupported fields explicitly.

Use native-backend traces first, then inspect what changes when a different backend is selected. Separate current values, peaks, and cumulative counters. Resetting peak memory statistics is not a substitute for recording the beginning value of every cumulative counter.

Part 2 - System and Memory Mental Model

`memory_stats()` provides allocator-specific counters. Inactive split bytes describe inactive, non-releasable split blocks in the documented native accounting; they are not all free device memory and are not universally comparable across backends. Active bytes can include blocks awaiting stream completion, while allocated bytes track a different ownership boundary.

Segment count describes allocator-managed segments, not tensor count. Allocation retry counters indicate failed device-allocation paths that lead to cache release and retry under the documented implementation. They can rise because of capacity pressure, not only fragmentation. OOM count records errors already encountered; it is a lagging signal rather than an early prediction.

Requested bytes and assigned allocation bytes can help study allocator rounding when sampled over matching scopes. Application pool slack can remain invisible to that comparison because the allocator sees the larger backing request. Some statistics are reported as zero or have no meaningful equivalent with `cudaMallocAsync`. Preserve the backend and field-support status alongside every value.

Part 3 - Quantitative Prediction Before Execution

For a cumulative counter C, report interval change

$$\Delta C = C(t_1) - C(t_0).$$

For current native inactive split bytes I_s and reserved bytes $R > 0$, define a descriptive ratio

$$f_s = I_s/R.$$

This is a trapped-split share under the chosen accounting, not an OOM probability or a universal fragmentation percentage.

For a hypothetical interval with retries changing from 3 to 5, reserved bytes of 256 MiB, and inactive split bytes of 64 MiB, calculate the interval retry count and ratio. Explain why the same ratio with 16 GiB of driver headroom has a different practical meaning. Predict which counters should remain monotonic unless explicitly reset, and which current quantities can decrease.

Part 4 - Experimental Design and Instrumentation

1. Enumerate supported keys on the installed backend and record the allocator configuration before collecting measurements.

2. Sample before allocation, at peak live demand, after selected reference releases, after completion, and after optional cache release.

3. Track these native keys when supported:

   - `inactive_split_bytes.all.current`.
   - `segment.all.current`.
   - `active_bytes.all.current`.
   - `reserved_bytes.all.current`.
   - `num_alloc_retries` and `num_ooms`.

4. Add requested and allocated byte counters with clearly matched current or cumulative semantics.

5. Compare an ordinary reusable-cache trace, a verified split pattern, and a capacity-pressure control.

6. Reconstruct interval deltas from the raw samples and validate them against the exported summary.

Keep observation overhead outside the primary timing run when possible. Use one timestamp convention and record whether a sample synchronized.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import time
import torch


@dataclass
class MetricValue:
    name: str
    value: object
    supported: bool
    scope: str


def discover_metric_support(device, backend):
    # Preserve missing and backend-inapplicable distinctions.
    ...


def capture_stats(device, phase, support):
    # Read scalar counters with a timestamp and phase ID.
    ...


def calculate_interval_deltas(before, after):
    # Handle cumulative counters and explicit reset boundaries.
    ...
```

```
def reconcile_with_snapshot(stats, snapshot):
    # Match device, time, block states, and accounting scope.
    ...


def E03(config):
    # Compare cache, split-block, and capacity-pressure traces.
    ...
    # Export raw samples and a metric interpretation table.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require nonnegative interval deltas only where no reset occurred. Flag mismatched reset epochs rather than silently clipping a negative result. Confirm that the collector never treats an unsupported zero as proof of no fragmentation. Do not require two asynchronously sampled accounting sources to match exactly without a controlled observation boundary.

Report units and semantics for each field, plus a plot of current bytes and a separate plot of interval retry counts. Investigate counter changes against the request trace and snapshot. A retry without increasing inactive split bytes is an important counterexample to a single-cause interpretation. Preserve it in the report.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use explicit not-measured and unsupported values. A fragmentation pattern or OOM that does not reproduce is an experimental outcome, not permission to invent hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Counters are evidence about an allocator's internal state transitions, not self-explanatory diagnoses. Their value increases when they are aligned with the exact workload phase and backend.

Choose a small set of signals sufficient to distinguish your controls. Explain why adding more counters can create confusion if their scopes differ. This collector becomes the observation layer for later predictors and controllers, so its missing-data and reset semantics must be as precise as its arithmetic.

19.28.5 Exercise 04

Exercise Title: Capture a CUDA Memory Snapshot and Visually Identify Partially Used Segments and Trapped Free Blocks

Part 1 - Exercise Contract and Why It Matters

Capture allocator snapshots and use their visual representation to identify live blocks, pending frees, inactive holes, and completely inactive segments. Deliver annotated views at selected

workload events and a table connecting each visible region to scalar accounting. The exercise is about explaining an observed allocator state, not merely producing a colorful timeline.

Use a bounded history window and a native-backend trace with known allocation roles. Record the exact PyTorch version and snapshot API. Treat unsupported schemas or backends as explicit limitations. Do not invent block information when the chosen backend does not expose it.

Part 2 - System and Memory Mental Model

A snapshot describes memory visible to the PyTorch allocator. It does not enumerate every GPU allocation made directly by CUDA libraries, other processes, or communication runtimes. Segment addresses are virtual-address labels and reveal no physical-page layout.

The documented native schema distinguishes these block states:

- `active_allocated`: owned by a live allocation.
- `active_awaiting_free`: awaiting stream completion.
- `inactive`: cached without an active allocation.

The second state is particularly important: a host reference can be released while a stream still has outstanding uses. A plot that colors both inactive and awaiting-free regions as available exaggerates immediate reuse.

A current-state snapshot and an allocation-history snapshot answer different questions. History can associate allocation and free events with stacks and earlier allocator states. If a history buffer wraps, older causal events may be absent. Capturing only after OOM recovery can miss the original arrangement because retries may have changed the cache.

Ordinary partially occupied segments can contain inactive blocks that are not individually releasable to the driver. Expandable segments and other allocator modes require their own schema and release interpretation. Do not extend the ordinary-segment model without checking.

Part 3 - Quantitative Prediction Before Execution

For an ordinary fully represented segment s, reconcile its block capacities as

$$S_s = A_s + P_s + I_s,$$

where A_s is allocated-block capacity, P_s awaiting-free capacity, and I_s inactive capacity. Verify this identity against the actual schema; report any unrepresented range explicitly.

For a hypothetical 32 MiB segment containing 12 MiB allocated, 4 MiB awaiting completion, and 16 MiB inactive, calculate state shares. Explain why 20 MiB must not automatically be called immediately reusable. If the inactive portion consists of holes of 6 and 10 MiB, predict the largest observed hole and distinguish that from the size of a potential new allocation.

Sketch the expected visual change after the pending work completes and after a separating allocation is released.

Part 4 - Experimental Design and Instrumentation

1. Inspect installed signatures for the snapshot interfaces. The public state snapshot and the history-oriented private helpers have different output and stability contracts.
2. When available, use bounded history recording through the `torch.cuda.memory` helper named `_record_memory_history`. Save evidence with the supported `_dump_snapshot` interface.

3. Capture initial, partially released, post-completion, and post-cleanup states. Use identical event labels in the trace.

4. Inspect the files with the official PyTorch memory visualizer or parse the documented schema into your own segment plot.

5. Annotate axes in bytes or MiB, segment IDs, block states, event times, and the live allocations that separate inactive holes.

6. Compare the visual totals with statistics collected at a matching controlled boundary.

Use snapshots you generated or otherwise trust; serialized diagnostic files are not arbitrary data to execute. Measure history overhead separately from the primary throughput experiment.
Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch


@dataclass
class SnapshotPlan:
    phases: tuple
    maximum_history_entries: int
    output_directory: str


def verify_snapshot_support():
    # Record version, supported signatures, and schema.
    ...


def capture_phase_snapshot(phase, plan):
    # Bound history and associate the artifact with an event.
    ...


def summarize_segments(snapshot, device):
```

```
    # Keep allocated, pending, and inactive states distinct.
    ...


def annotate_segment_view(summary, request_trace):
    # Label units, live separators, and missing history.
    ...


def E04(config):
    # Capture and inspect controlled allocator transitions.
    ...
    # Reconcile visual block totals with scalar evidence.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Provide at least one annotated partially occupied segment, or explain why the intended pattern did not occur. Every claimed trapped hole must be linked to its containing segment and live or pending neighbors. Show a completely inactive ordinary segment as a contrasting case.

Check state totals, byte units, device identity, and snapshot timestamps. Do not compare a pre-synchronization snapshot with post-synchronization statistics as though they were simultaneous. Record missing stack history and unknown allocations rather than guessing their owners. Preserve the original snapshot alongside the annotated interpretation so another learner can challenge the diagnosis.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use explicit not-measured and unsupported values. A fragmentation pattern or OOM that does not reproduce is an experimental outcome, not permission to invent hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Visualization exposes structure that aggregate counters hide: which objects prevent coalescing and when those objects were created. Its usefulness depends on correct state semantics and a clear event boundary.

Explain one inference supported by the picture and one claim the picture cannot establish. A rigorous visual diagnosis connects the allocation pattern to a specific request and then tests a counterfactual change, rather than assuming that visible empty regions are necessarily harmful.

19.28.6 Exercise 05

Exercise Title: Build a Fragmentation Metric from Reserved Bytes, Active Bytes, Inactive Split Bytes, and Largest Reusable Block

Part 1 - Exercise Contract and Why It Matters

Build a small family of fragmentation indicators with explicit scopes, instead of presenting one ratio as a complete allocator diagnosis. Combine retained capacity, active occupancy, native split-block signals, and request-specific block-fit evidence. Deliver definitions, synthetic counterexamples, and a comparison with observed allocation outcomes.

The largest reusable block must be defined carefully. In a controlled application arena it can be exact. In a PyTorch snapshot it is usually the largest observed inactive candidate under known filters, with incomplete knowledge of actual allocator eligibility and growth.

Part 2 - System and Memory Mental Model

Reserved and active bytes describe occupancy at an allocator boundary. Native inactive split bytes identify one kind of retained inactive space. A large inactive block describes local fit opportunity. These quantities overlap and must not be summed into an invented total waste.

Let the eligible-set definition include device, known stream constraints, pool identity, capture mode if relevant, and completion state. If the snapshot does not expose enough information to verify eligibility, call the result a candidate metric and record the unknowns. Do not assume there is a public API returning the maximum CUDA allocation that would succeed now.

A request can succeed by growing the allocator despite a small largest cached candidate. A request can fail despite a large apparent candidate because of restrictions omitted from the filter. Snapshot age also matters: another allocation or completion can change the set immediately. Use the metric to guide investigation and prediction, not as a reservation or a mathematical guarantee about the real allocator.

Part 3 - Quantitative Prediction Before Execution

For $R > 0$, active capacity A_{act}, and native inactive split bytes I_s, define

$$f_{\text{idle}} = \frac{R - A_{\text{act}}}{R}, \qquad f_s = \frac{I_s}{R}.$$

The first is a scoped inactive share, not pure fragmentation. For a declared candidate set with total $F > 0$ and largest L, define dispersion and request coverage:

$$d = 1 - \frac{L}{F}, \qquad k(q) = \frac{L}{q} \quad (q > 0).$$

Undefined denominators must remain undefined.

Compare hypothetical free sets $\{32, 32\}$ MiB and $\{64\}$ MiB. Compute d and coverage for a 48 MiB request. Then consider a 1 MiB request and explain why a high dispersion score may be irrelevant to it. Add driver headroom as separate context rather than inserting it into the candidate-block formula.

Part 4 - Experimental Design and Instrumentation

1. Define each metric's accounting layer, units, eligibility filters, missing-data status, and timestamp.

2. Validate formulas on synthetic block lists with one hole, many holes, no holes, and pending-only space.

3. Collect matching statistics and native snapshots for the E01 trace and a harmless cache-retention control.

4. Compare metric values with later request outcomes from separate replays. Probing changes allocator state, so do not reuse a mutated state as the original observation.

5. Test a case with a small cached candidate but ample growth headroom and a case with insufficient total live-capacity allowance.

6. Report which metric distinguishes the cases and which produces a misleading ranking.

Do not merge inactive blocks across separate segments simply because their addresses appear numerically adjacent. Coalescing requires the allocator's actual ownership and compatibility conditions.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every `...` marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch


@dataclass
class FragmentationIndicators:
    inactive_share: object
    split_share: object
    candidate_dispersion: object
    request_coverage: object
    observation_age_seconds: float
    limitations: tuple


def select_inactive_candidates(snapshot, request_scope):
    # Preserve unknown eligibility rather than fabricating fit.
    ...


def calculate_indicators(stats, candidates, request):
    # Apply scoped formulas and explicit undefined cases.
    ...
```

```
def compare_with_outcomes(indicator_rows, replay_results):
    # Evaluate candidate fit separately from growth success.
    ...


def E05(config):
    # Validate synthetic counterexamples first.
    ...
    # Assess which indicators explain the controlled traces.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require a counterexample to each proposed universal interpretation. Examples include a high inactive share with healthy reuse, a small cached block followed by successful growth, and a high dispersion score with requests small enough to fit existing holes.

Report raw $R, A_{\text{act}}, I_s, F, L, q$ beside derived values so the result can be audited. Flag observations where sampling time or unsupported fields invalidate a formula. A predictor must not silently substitute zero for unknown candidate capacity. Separate diagnostic usefulness from predictive accuracy, and do not call a correlation a causal fragmentation proof.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use explicit not-measured and unsupported values. A fragmentation pattern or OOM that does not reproduce is an experimental outcome, not permission to invent hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

A useful metric compresses information while preserving the distinction needed for a decision. Fragmentation requires more than a global percentage because request size and allocator capabilities matter.

Choose a minimal indicator set for the later warning detector. Explain which remaining uncertainty belongs to the allocator and which can be reduced through better workload instrumentation. The strongest outcome is an honest, request-aware signal with known failure cases.

19.28.7 Exercise 06

Exercise Title: Compare Fixed-Shape and Variable-Shape Tensor Workloads and Quantify Their Impact on Fragmentation Over Time

Part 1 - Exercise Contract and Why It Matters

Compare fixed and changing tensor shapes while controlling useful work, live-demand envelopes, and lifetime structure as far as the experiment allows. Deliver a longitudinal allocator trace and an explanation of whether shape variability changed reuse, internal slack, or trapped inactive space.

Use both an allocation-only experiment and a supported PyTorch workload. The allocation-only trace isolates allocator behavior; the workload reveals operator workspace and activation effects. Do not assume variable shapes must cause fragmentation or that rising reserved memory proves degradation.

Part 2 - System and Memory Mental Model

Stable repeated shapes can reuse a warmed inventory. Changing shapes can introduce nearby or dissimilar allocation requests that fit existing blocks differently. Yet the effect depends on lifetime overlap, rounding policy, segment growth, and operator behavior. A variable trace may still reuse memory efficiently.

Training introduces confounders: optimizer state may be allocated lazily, gradient buffers can appear on the first backward pass, and kernels may select different workspaces for different shapes. Warm these states or report their startup contribution separately. Keep precision, checkpointing, optimizer, and backend fixed during the primary comparison.

For padded sequence workloads, fixed physical shape can perform more work on padding even when useful token count is unchanged. Measure both physical and useful work. Allocator-only controls can match total requested bytes and maximum live bytes more closely than full model workloads, but they lack realistic operator temporaries. Use both experiments to narrow the interpretation.

Part 3 - Quantitative Prediction Before Execution

Let Q_t be requested live bytes at step t, and let R_t and $I_{s,t}$ be reserved and native inactive split bytes at a matched phase. Predict time series rather than only final values:

$$Q_{\text{peak}} = \max_t Q_t, \qquad \Delta R = R_{\text{end}} - R_{\text{warm}}.$$

Growth in R is retained capacity, whose cause still needs inspection.

For a hypothetical trace alternating 8 and 12 MiB requests, compare it with a 10 MiB fixed-request trace over equal request counts. Calculate total requested bytes, then explain why equal totals do not imply equal peak live demand or equal allocator placement. Specify lifetimes that make the comparison informative. Predict how a fixed maximum-size padded workload changes useful-byte efficiency relative to those two traces.

Part 4 - Experimental Design and Instrumentation

1. Prepare fixed, alternating-nearby, and broad-variable shape sequences with explicit allocation lifetimes.

2. Match request count, dtype, device, and release rules. Report any unavoidable difference in peak demand.

3. Run each trace in a fresh worker, with a declared warmup. Sample statistics at the same lifecycle phase.

4. Repeat using a small model or explicit operator workload. Record shapes, useful units, padding, and workspace-sensitive choices.

5. Capture snapshots only at selected early and late windows to inspect whether retained capacity forms reusable blocks or partially occupied segments.

6. Repeat the comparison with several seeds and reversed phase order to expose history dependence.

Keep primary timing separate from detailed history recording. End each run with a complete drain so pending activity can be distinguished from persistent retained capacity.
Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch


@dataclass
class WorkloadShape:
    step_id: int
    shape: tuple
    useful_units: int
    lifetime_group: int


def build_matched_shape_traces(config):
    # Document matched quantities and remaining confounders.
    ...


def run_allocation_control(trace, config):
    # Replay explicit live intervals without model changes.
    ...


def run_operator_workload(trace, model, config):
    # Preserve precision, backend, and numerical semantics.
    ...


def compare_longitudinal_rows(rows):
    # Separate startup growth, demand, and split-block changes.
```

```
    ...


def E06(config):
    # Compare controlled allocation traces and real workloads.
    ...
    # Export useful-work-normalized results and snapshots.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report request count, useful work, physical work, live-demand peak, allocated and reserved peaks, split-byte trajectory, retry count, and complete throughput. Verify that apparent fragmentation growth is not simply the larger trace requiring more simultaneous memory.

Show early and late segment evidence when claiming worsening block organization. If reserved capacity plateaus and reuse is healthy, report stabilization rather than degradation. Validate model outputs under shape changes with the intended masks and semantics. Do not compare different useful workloads using raw steps per second as the only performance metric. **Evidence format.** Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use explicit not-measured and unsupported values. A fragmentation pattern or OOM that does not reproduce is an experimental outcome, not permission to invent hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Shape diversity is one contributor to allocation diversity, not a standalone diagnosis. The value of this experiment lies in separating size changes from increased live demand and backend workspace changes.

Explain whether the measured problem is best addressed by bucketing, reuse, lifetime management, or no intervention. State which control supports that decision and what additional trace would most strongly challenge it.

19.28.8 Exercise 07

Exercise Title: Generate Random Tensor Sizes and Measure How Allocation Size Variability Changes Reuse Efficiency and Fragmentation

Part 1 - Exercise Contract and Why It Matters

Generate reproducible random size traces and measure how their distribution interacts with reuse and fragmentation. Vary size variability while holding request count, live concurrency, and total or mean demand under a declared matching scheme. Deliver seeds, trace files, distributions, and allocator observations.

Include bounded uniform, clustered, and heavy-tail-like distributions. Truncate every distribution at a documented request limit and enforce a simultaneous-memory allowance. Do not create an uncontrolled OOM campaign by drawing arbitrary large sizes.

Part 2 - System and Memory Mental Model

Variance describes size dispersion but not temporal locality. Two traces with identical size histograms can have different allocator behavior if one clusters similar requests and the other alternates them. Lifetimes matter as well: sequential requests can reuse the same storage while overlapping requests require separate capacity.

A large rare request can establish a reusable cache that helps later work, or it can introduce partially occupied segments depending on subsequent allocation patterns. Neither high variance nor a heavy tail guarantees fragmentation. Keep the request distribution and temporal order as separate experimental factors.

Random trials need replayability. Generate the trace on CPU before timing and store exact integer byte or element requests, not just the seed. Library versions can change random sampling behavior. The evidence should also preserve release events, stream assignment, and any rounding applied before the allocator sees the request.

Part 3 - Quantitative Prediction Before Execution

For request sizes q_i, define

$$\mu = \frac{1}{N}\sum_i q_i, \qquad \sigma^2 = \frac{1}{N}\sum_i (q_i - \mu)^2, \qquad \mathrm{CV} = \sigma/\mu \quad (\mu > 0).$$

These characterize the generated finite trace. They are not fragmentation metrics.

Compare hypothetical sequences containing equal counts of 4 and 12 MiB requests: one groups equal sizes, another alternates them. Calculate their shared mean and variance, then predict why their reuse may differ under an overlapping lifetime pattern. Define the overlap explicitly. Add one rare 64 MiB request and predict how it changes both the statistical summary and the required bounded-memory plan.

Part 4 - Experimental Design and Instrumentation

1. Create a trace generator with seed, dtype-compatible size limits, request count, and maximum live demand.

2. Match finite-sample means or total bytes across chosen distributions where feasible; report residual mismatch exactly.

3. Compare original, shuffled, and size-clustered orders of the same request multiset.

4. Replay each trace in a fresh process under identical allocator settings and completion rules.

5. Collect request-size histograms, size transitions, native split bytes, segment growth, retries, and timing.

6. Inspect a few snapshots at equal completed-request counts to connect statistical differences with actual block organization.

If estimating allocator reuse from traces, distinguish reuse without new segments from exact reuse of one backing address. Pointer equality alone is not a complete allocation history.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need

an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import random
import torch


@dataclass
class DistributionSpec:
    name: str
    seed: int
    request_count: int
    minimum_bytes: int
    maximum_bytes: int


def generate_size_trace(spec, lifetime_rules):
    # Enforce dtype alignment and bounded live demand.
    ...


def describe_trace(trace):
    # Compute finite-trace size and temporal statistics.
    ...


def reorder_same_multiset(trace, order_policy):
    # Preserve declared workload and lifetime constraints.
    ...


def replay_random_trace(trace, allocator_config):
    # Capture allocator changes without sampling in the loop.
    ...


def E07(config):
    # Compare variability and ordering as separate factors.
    ...
    # Preserve exact traces and analyze repeated seeds.
```

. . .

Part 6 - Measurement, Validation, and Failure Analysis

Require matching trace counts and documented demand differences before interpreting results. Check that all requests satisfy the configured bounds and that the live set never exceeds the intended test allowance through a generator bug.

Report distribution statistics alongside temporal-locality measures, allocator metrics, and complete runtime. Show at least one pair with identical size statistics but different ordering. If allocator behavior is indistinguishable, report that finding within measurement uncertainty. Do not attribute a single unlucky random run to a universal property of the distribution.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use explicit not-measured and unsupported values. A fragmentation pattern or OOM that does not reproduce is an experimental outcome, not permission to invent hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Randomness is a tool for exploring workload diversity, not a replacement for an explanatory model. Size histograms need lifetime and ordering information before they can predict reuse.

Use the results to identify which workload property deserves management. A policy aimed at variance may be ineffective when the real problem is overlap or a long-lived separator. Preserve the exact traces so subsequent mitigation experiments can test that diagnosis fairly.

19.28.9 Exercise 08

Exercise Title: Bucket Variable Tensor Shapes into Size Classes and Measure Whether Reduced Shape Diversity Lowers Fragmentation

Part 1 - Exercise Contract and Why It Matters

Bucket variable shapes into a smaller set of physical allocation shapes and test whether that improves reuse under the same useful workload. Deliver a bucketing policy, a useful-versus-padded work ledger, and allocator evidence before and after bucketing. The learner must implement classification, masks, trace replay, and measurements.

Start with sequences or matrices whose padding semantics can be validated against a reference. Distinguish shape bucketing from allocator rounding: bucketing changes the physical tensor shape seen by operators, whereas allocator rounding can preserve the original logical shape.

Part 2 - System and Memory Mental Model

Fewer physical shapes can stabilize allocation requests, but padding increases tensor size and may increase arithmetic. A policy that reduces split blocks while raising peak live memory can still be undesirable. Different physical shapes can also select different kernels or attention backends, so record those changes.

Reordering samples into buckets changes temporal locality and possibly training order. To isolate physical shape diversity, first preserve request order while padding to class boundaries.

Then evaluate reordering as a separate experiment with the same sample multiset and clearly stated update semantics.

Masking must preserve the intended computation. Attention masks, loss masks, and output cropping are distinct operations. Padding tokens must not alter the loss denominator or leak into meaningful attention positions. For a numerical comparison, disable uncontrolled stochastic differences or explain their effect. Reduced shape cardinality does not itself prove reduced allocator fragmentation.

Part 3 - Quantitative Prediction Before Execution

For sequence lengths l_i and bucketed physical lengths $b_i \geq l_i$, define token padding fraction

$$p = 1 - \frac{\sum_i l_i}{\sum_i b_i}.$$

For explicitly materialized attention, a rough score-storage ratio is

$$\rho_{\text{scores}} = \frac{\sum_i b_i^2}{\sum_i l_i^2}$$

under otherwise equal independent sequence computations. This is not a universal fused-attention cost model.

Assign hypothetical lengths 129, 190, and 257 to bucket ceilings 128, 256, and 512, with an explicit oversize rule. Predict the physical lengths, padding fraction, and shape count. Explain why the 257-token request can be costly under this policy. Compare with a finer ceiling and predict the additional class count.

Part 4 - Experimental Design and Instrumentation

1. Choose bucket boundaries from a calibration length distribution and freeze them before evaluation.

2. Replay original-order requests with exact shapes and with padded bucket shapes under the same backend and precision.

3. Validate masks, loss normalization, and cropped outputs against the exact-shape reference.

4. Measure physical bytes, useful tokens, padding, complete latency, split-byte trajectory, and allocator growth.

5. Add a separate sorted-bucket experiment and disclose its ordering change.

6. Evaluate held-out lengths near bucket boundaries and requests above the largest configured class.

Do not silently truncate oversize sequences. Reject, use a declared larger class, or apply a separately authorized semantic transformation. Keep allocator settings fixed so the experiment tests shape bucketing.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need

an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch


@dataclass
class BucketedRequest:
    request_id: int
    logical_shape: tuple
    physical_shape: tuple
    useful_units: int


def choose_bucket(logical_shape, boundaries):
    # Cover the request and handle oversize inputs explicitly.
    ...


def prepare_padded_input(request, source):
    # Construct data and masks without changing useful content.
    ...


def validate_bucket_semantics(reference, candidate, masks):
    # Compare outputs and the intended loss denominator.
    ...


def compare_bucket_traces(trace, policies, config):
    # Separate physical-shape changes from order changes.
    ...


def E08(config):
    # Calibrate boundaries, then freeze them for evaluation.
    ...
    # Compare allocator behavior and useful-work throughput.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Acceptance requires preserved useful inputs, correct masks, and no hidden truncation. Report the sample multiset and order for every mode. Validate a boundary case where padding almost doubles physical sequence length.

Include physical-shape count, padding fraction, useful tokens per second, allocated and reserved peaks, native split bytes, retries, and snapshot evidence. If the allocator metric improves but useful throughput or feasibility worsens, retain the tradeoff. Do not call the policy successful solely because fewer shapes reached the allocator. Separate changed kernel selection from improved storage reuse where the evidence permits.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use explicit not-measured and unsupported values. A fragmentation pattern or OOM that does not reproduce is an experimental outcome, not permission to invent hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Bucketing spends extra physical work to make demand more predictable. Its value depends on whether the resulting reuse and kernel behavior outweigh padding costs.

Choose boundaries from measured workloads and state where they fail. A useful recommendation may favor bucketing only a subset of the length range. Explain why fewer classes is not an objective by itself and how useful-work throughput keeps the decision tied to the application.

19.28.10 Exercise 09

Exercise Title: Compare Exact-Size Allocation Against Rounded Allocation Classes and Quantify the Trade-Off Between Internal and External Fragmentation

Part 1 - Exercise Contract and Why It Matters

Compare exact requested sizes with explicit rounded backing capacities while preserving logical tensor shapes and useful computation. Quantify internal slack and allocator-level placement evidence separately. Deliver a capacity ledger showing whether the policy trades one form of waste for another.

Use one rounding layer at a time. The primary experiment can use application-managed typed buffers; allocator configuration changes belong in a separate controlled variant. Ordinary exact-size tensor requests can still be rounded by the underlying allocator, so the label exact refers to the application request.

Part 2 - System and Memory Mental Model

Application rounding asks PyTorch for a larger backing tensor and exposes a logical view. The allocator sees the backing request, not the smaller useful region. Therefore its requested-versus-allocated statistics do not include all application slack.

Track three byte quantities: useful logical demand q, application backing request c, and allocator-assigned block capacity a. Internal slack can occur at both boundaries. External placement signals concern separate inactive regions and cannot be obtained by subtracting only these three numbers.

Rounded classes may improve future fit and reuse, but they can consume the very headroom needed for a large request. The result depends on size distribution and overlapping lifetimes. Buffer views must remain within capacity, preserve dtype and layout, and avoid exposing uninitialized slack. A hidden clone or resize would invalidate the claimed accounting.

Part 3 - Quantitative Prediction Before Execution

For matching live allocations, decompose internal slack as

$$W_{\text{app}} = \sum_i (c_i - q_i), \qquad W_{\text{alloc}} = \sum_i (a_i - c_i).$$

Their sum equals $\sum_i (a_i - q_i)$ only when the same allocations, states, and time boundary are used. Idle retained buffers are accounted separately.

For a compatible byte quantum u, predict

$$c(q) = u \left\lceil q/u \right\rceil .$$

Use hypothetical requests 5, 6, and 7 MiB with an 8 MiB class. Calculate application slack for sequential and simultaneously live requests, making the buffer-retention policy explicit. Predict why sequential reuse can require one backing buffer while overlapping requests cannot. State what snapshot evidence would be needed to claim reduced external fragmentation.

Part 4 - Experimental Design and Instrumentation

1. Fix dtype, device, logical shapes, lifetime trace, and allocator configuration.
2. Implement exact backing requests and several rounded application policies with the same retention rules.
3. Log useful bytes, backing requests, actual storage capacity, and observed allocator block capacity where available.
4. Compare size-jittered and boundary-heavy traces under matched useful work and maximum concurrency.
5. Capture selected native snapshots and compute only scoped candidate-fit or split-block indicators.
6. Evaluate total process peaks and request success under several budgets, including cases where rounding causes rejection.

Avoid merging allocator-level rounding and application-level rounding into one unlabeled policy. Initialize the entire logical read region and record any extra writes made to slack.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch


@dataclass
class CapacityLayers:
    useful_bytes: int
    backing_request_bytes: int
    assigned_block_bytes: object
    allocation_id: int


def choose_backing_capacity(request, rounding_policy):
    # Preserve dtype compatibility and the logical shape.
    ...


def match_allocation_layers(request_log, snapshot):
    # Match identities, generations, and observation states.
    ...


def calculate_internal_slack(layer_rows):
    # Keep application and allocator contributions distinct.
    ...


def compare_placement_evidence(snapshot_rows, next_requests):
    # Assess request-specific fit without claiming global capacity.
    ...


def E09(config):
    # Replay exact and rounded backing policies fairly.
    ...
    # Report internal costs and external-placement evidence.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require correct logical outputs and complete byte accounting for every live backing allocation. Do not add idle capacity to active slack twice. If allocator block matching is unavailable, mark that layer unknown instead of substituting the tensor's logical byte size.

Report internal slack by layer, retained idle capacity, native split bytes, candidate-fit evidence, peak VRAM, and complete throughput. Include a case where rounding improves reuse and one where it imposes a significant cost, or state which case was not observed. A lower split-byte ratio caused only by a larger reserved denominator is not sufficient evidence of reduced absolute fragmentation.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use explicit not-measured and unsupported values. A fragmentation pattern or OOM that does not reproduce is an experimental outcome, not permission to invent hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Rounding is an explicit exchange: capacity is spent inside allocations in the hope of improving reuse between allocations. Only the complete memory and runtime ledger reveals whether that exchange was favorable.

Choose a policy for the tested distribution and explain how a change in overlap or budget could reverse the decision. The goal is a measured tradeoff rather than a promise that coarser classes always improve memory efficiency.

19.28.11 Exercise 10

Exercise Title: Use Buffer Reuse to Replace Repeated Allocate-Free Cycles and Measure the Reduction in Fragmentation Growth

Part 1 - Exercise Contract and Why It Matters

Apply a bounded DMM25-style buffer pool to an allocation-heavy workload and measure whether it stabilizes allocator behavior. Compare ordinary tensor requests with retained backing reuse under identical logical work, shapes, and concurrency. Deliver a longitudinal comparison of allocator activity, retained capacity, and useful throughput.

Restrict pooled storage to lifetimes you can control, such as input staging or explicit inference scratch. Do not overwrite autograd-saved tensors or claim that a Python pool replaces opaque operator workspaces. A result showing no fragmentation reduction remains valid.

Part 2 - System and Memory Mental Model

Ordinary PyTorch allocation already uses a caching allocator. Replacing Python tensor creation with application reuse may reduce allocator requests, but it does not automatically reduce device allocation calls or fragmentation. The custom pool can also retain obsolete classes and increase live allocated memory.

Free application buffers remain live backing tensors. As a result, allocator inactive split bytes may decrease simply because more capacity is held as active allocations. That metric change can be a reclassification rather than actual memory relief. Account for useful demand, pool slack, idle pool capacity, and whole-process memory.

Reuse requires completed consumers and exclusive ownership. `record_stream` protects allocator lifetime when storage is deallocated; it does not authorize early manual overwrite of the same live pooled tensor. Use the event and generation protocol from DMM25 when asynchronous consumers are involved. Reference logs must not keep supposedly released tensors alive.

Part 3 - Quantitative Prediction Before Execution

Let N be logical requests and K new application backing allocations. Define a creation-avoidance measure

$$r_{\text{avoid}} = 1 - K/N \quad (N > 0),$$

with startup allocations and successful requests counted by a stated convention. This is not allocator fragmentation.

Decompose retained pool capacity as

$$C_{\text{pool}} = C_{\text{leased}} + C_{\text{pending}} + C_{\text{free}}.$$

For a hypothetical repeating trace requiring at most three simultaneous 8 MiB buffers, predict the minimum fixed inventory and backing-creation count after warmup. Then add a one-time 32 MiB request and predict how unrestricted retention can increase the pool baseline. Explain how inactive split bytes could fall while total live allocated bytes rise.

Part 4 - Experimental Design and Instrumentation

1. Select one supported workload and implement equivalent ordinary-allocation and pooled paths.
2. Match initialization, output lifetimes, concurrent work, precision, and completion boundaries.
3. Compare startup-inclusive and warmed execution, preserving the same exact request trace.
4. Record pool capacity by state together with allocator statistics and selected snapshots.
5. Add a phase shift from large requests to smaller requests and compare bounded trimming with unrestricted retention.
6. Measure useful throughput and request tail latency with minimal logging, then use a separate detailed diagnostic pass.

Count application requests, backing creations, and supported device-allocation counters separately. Do not call every `torch.empty` request a fresh device allocation.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch


@dataclass
class ReuseObservation:
    request_id: int
    useful_bytes: int
    pool_capacity_bytes: int
    idle_pool_bytes: int
    new_backing_created: bool


def run_ordinary_path(trace, consumer, config):
    # Preserve normal allocator caching and equal useful work.
    ...


def run_pooled_path(trace, pool, consumer, config):
    # Enforce exclusive leases and complete consumer lifetimes.
    ...


def reconcile_pool_and_allocator(pool_rows, allocator_rows):
    # Detect reclassification rather than real capacity relief.
    ...


def E10(config):
    # Validate numerical equivalence before timing.
    ...
    # Compare stable, phase-shifted, and trimmed runs.
    ...
    # Explain both memory and runtime tradeoffs.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Acceptance requires equal completed work, correct outputs, no premature reuse, and a reconciled pool ledger. Show absolute allocated and reserved peaks, not only increases above a larger pooled baseline.

Report backing creations, allocator activity, inactive split trajectory, free pool capacity, useful throughput, and trimming cost. Investigate lower split bytes accompanied by higher active occupancy before claiming less waste. If ordinary allocation already stabilizes, report the small or absent incremental benefit of the custom pool. Include the memory cost of retaining rare large buffers.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use

explicit not-measured and unsupported values. A fragmentation pattern or OOM that does not reproduce is an experimental outcome, not permission to invent hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Buffer reuse moves lifetime management upward into the application. That can improve predictability while making more memory appear permanently live to the allocator.

Explain whether the measured benefit comes from lower bookkeeping, fewer new segments, or better shape predictability. Choose reuse only where its ownership complexity and retained capacity are justified by the complete workload results.

19.28.12 Exercise 11

Exercise Title: Compare Long-Lived and Short-Lived CUDA Tensors and Show How Interleaving Their Lifetimes Can Trap Free Memory Inside Segments

Part 1 - Exercise Contract and Why It Matters

Study how persistent tensor lifetimes interact with transient allocations. Construct grouped and interleaved lifetime traces, verify actual segment placement, and identify live allocations that prevent inactive regions from becoming fully reclaimable. Deliver a lifetime diagram and a segment-level explanation.

Use explicit synthetic tensors before a model-based example. Keep persistent byte count and transient demand comparable. Do not mistake a tensor retained accidentally by a logger for an intended persistent object.

Part 2 - System and Memory Mental Model

A long-lived tensor can keep an ordinary segment partially occupied after neighboring transient tensors are released. Inactive pieces may remain reusable for suitably sized requests but unavailable as one large hole or as a fully releasable segment. The harm is request-dependent.

Interleaving lifetime classes in Python does not guarantee that they share segments. Allocation size, pool selection, history, and stream use can separate them. Verify the placement before claiming a trapped-region mechanism. A small view can retain an entire backing storage, so logical view size is not necessarily the persistent footprint.

GPU completion creates another lifetime boundary. A tensor released on the host may still await work on a stream. Compare completed-release and pending-release observations separately. Releasing a live separator is only valid when the program no longer needs its data and all consumers have finished; moving or destroying live model state is not a general defragmentation operation.

Part 3 - Quantitative Prediction Before Execution

Define lifetime duration $d_i = t_{i,\text{release}} - t_{i,\text{acquire}}$ using a declared release meaning. For ordinary segment s, define a snapshot retention indicator

$$z_s = \mathbf{1}\{A_s + P_s > 0\},$$

where A_s and P_s represent allocated and pending block capacities. Inactive bytes inside such segments are candidates for trapped retention, subject to backend semantics.

For a hypothetical 32 MiB segment with one 4 MiB persistent block and 28 MiB of transient blocks, predict the state after all transient uses finish. Determine what additional information is required to know the largest inactive hole. Compare a persistent block at one end with one in the middle, then predict the effect of its legitimate final release.

Part 4 - Experimental Design and Instrumentation

1. Build two traces with the same persistent and transient sizes: one groups lifetime classes, another interleaves them.
2. Replay in fresh native-backend workers using bounded memory and completed single-stream uses.
3. Capture snapshots during the transient peak and after transient release. Confirm segment membership and adjacency.
4. Retain a small view of a larger backing tensor in a separate control to demonstrate storage-level lifetime.
5. Release persistent objects only at their declared last use, then observe coalescing and possible segment release.
6. Repeat with a pending stream consumer to distinguish asynchronous retention from structural trapping.

Keep all evidence tensor-free. Use allocation IDs and generation-aware address labels when matching lifetime records to snapshots.
Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch


@dataclass
class LifetimeSpec:
    tensor_id: int
    requested_bytes: int
```

```
    acquire_event: int
    last_use_event: int
    release_event: int
    lifetime_class: str


def build_lifetime_variants(config):
    # Preserve demand while changing lifetime interleaving.
    ...


def replay_lifetime_trace(trace, config):
    # Release only after semantic use and CUDA completion.
    ...


def identify_live_separators(snapshot, lifetime_log):
    # Match actual segment placement to persistent holders.
    ...


def E11(config):
    # Compare grouped, interleaved, view, and pending controls.
    ...
    # Explain segment retention through specific allocations.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Claim trapping only when the snapshot shows inactive space retained within a segment containing the identified live or pending allocation. Distinguish a large reusable inactive block from several small holes. If persistent and transient objects occupy separate segments, report that counterexample.

Report persistent logical bytes, persistent backing bytes, transient peak, retained inactive bytes, largest candidates, and complete runtime. Validate that the small-view control retains backing storage without attributing that effect to allocator fragmentation alone. Show how legitimate final release changes the map and whether reserved memory remains cached afterward.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use explicit not-measured and unsupported values. A fragmentation pattern or OOM that does not reproduce is an experimental outcome, not permission to invent hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Lifetime structure can matter as much as allocation size. The allocator cannot freely repack arbitrary live tensors while preserving all pointers, aliases, and asynchronous users.

Use the evidence to identify a scheduling opportunity: which objects could be allocated together or released earlier without changing semantics? Do not recommend moving live model state merely to improve a diagram. The next exercise tests legal lifetime reordering as a measured intervention.

19.28.13 Exercise 12

Exercise Title: Reorder Tensor Allocation Lifetimes to Group Similar-Lived Buffers and Measure the Effect on Allocator Reuse

Part 1 - Exercise Contract and Why It Matters

Test whether legal changes to allocation timing and release timing improve allocator reuse. Group persistent buffers and transient buffers where program dependencies permit it, then compare with an interleaved baseline. Deliver a dependency-preserving schedule, a live-demand ledger, and allocator evidence for the resulting memory layout.

The experiment changes future allocation behavior; it does not magically compact existing live tensors. Any explicit copy or relocation must be a separate variant with its own transient memory cost and alias correctness proof.

Part 2 - System and Memory Mental Model

Allocation can sometimes be moved earlier without changing values, while release can move earlier only after the last semantic consumer and required CUDA completion. Moving allocation earlier may increase overlap and peak live demand. A schedule that reduces fragmentation but increases true memory demand can be a poor tradeoff.

Group persistent model-independent buffers before transient work when their construction permits it. Do not assume optimizer state, lazy module state, and backend workspaces all exist before the first training step. Include their initialization phase in the model or warm it separately.

The allocator may still place lifetime classes in unexpected segments, so verify actual maps. Preserving the dependency graph is necessary but not always sufficient for numerical equivalence: changing stochastic execution order can change random draws, and in-place aliases can impose additional ordering constraints. Keep the initial comparison deterministic and treat concurrent scheduling as a later extension.

Part 3 - Quantitative Prediction Before Execution

For allocation i with capacity c_i, define the live interval $[a_i, r_i)$ under one schedule:

$$M(t) = \sum_i c_i \mathbf{1}\{a_i \le t < r_i\}, \qquad M_{\text{peak}} = \max_t M(t).$$

This counts declared capacity before hidden operator temporaries. For every dependency $u \to v$, require the chosen execution order to preserve the required producer completion.

Create a hypothetical trace containing two persistent 8 MiB buffers and four transient 4 MiB buffers used in pairs. Calculate peak live capacity before and after moving all transient allocations to startup. Explain why that apparently simple grouping can be counterproductive. Propose a different legal ordering that keeps transient overlap bounded and predict its segment-reuse opportunity.

Part 4 - Experimental Design and Instrumentation

1. Write a dependency graph and identify each tensor's first and last semantic use.
2. Construct an interleaved schedule, a grouped persistent schedule, and an intentionally over-eager preallocation control.
3. Verify live-demand peaks in a host simulator before running CUDA.
4. Replay each schedule in a fresh worker with identical sizes, values, backend, and complete useful work.
5. Compare native snapshots at matched logical milestones, not merely equal wall-clock times.
6. Measure allocated and reserved peaks, split bytes, growth events, stalls, and complete latency.

For training-related variants, validate gradients and parameter updates under a declared deterministic reference. Do not release an activation merely because its forward consumer has completed if backward still needs it.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch


@dataclass
class ScheduledLifetime:
    tensor_id: int
    allocate_event: int
    last_consumer_event: int
    release_event: int
    capacity_bytes: int


def derive_lifetime_constraints(operation_graph):
    # Include aliases, backward uses, and stream completion.
    ...
```

```
def propose_grouped_schedule(constraints, policy):
    # Reorder only operations allowed by the support contract.
    ...


def predict_live_peak(schedule):
    # Account for overlap introduced by earlier allocation.
    ...


def validate_schedule(reference, candidate, config):
    # Compare dependencies, outputs, and supported gradients.
    ...


def E12(config):
    # Replay schedules and compare allocator placement.
    ...
    # Separate reduced live demand from improved reuse.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Acceptance requires preserved dependencies and correct numerical results, with no premature release. Report the predicted and observed live-demand peaks for every schedule. A reduction in reserved memory caused by lower true live demand must be distinguished from reduced fragmentation.

Show actual segment membership when claiming improved lifetime grouping. Include the preallocation control to demonstrate that grouping can increase overlap. If a copy-based relocation variant is attempted, account for both source and destination being live during the copy, all aliases, and transfer completion. Do not present it as a free allocator compaction operation. **Evidence format.** Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use explicit not-measured and unsupported values. A fragmentation pattern or OOM that does not reproduce is an experimental outcome, not permission to invent hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Good lifetime scheduling reduces unnecessary overlap and may improve future allocator placement. Those benefits come from understanding program dependencies, not from rearranging Python statements blindly.

Use the results to identify a legal, maintainable scheduling change. Explain whether its benefit is robust across traces or relies on a specific allocator history. Prefer a simple schedule with clear ownership over a fragile placement trick that fails under a nearby shape change.

19.28.14 Exercise 13

Exercise Title: Experiment with PyTorch CUDA Allocator Configuration and Measure How Split-Size or Backend Choices Affect a Fragmentation-Prone Workload

Part 1 - Exercise Contract and Why It Matters

Run a controlled allocator-configuration study using a verified fragmentation-prone trace. Compare the default native behavior with selected split or rounding settings, and optionally a supported alternative backend. Deliver a configuration matrix with measured outcomes and explicit unsupported cases.

Each configuration must run in a fresh process with its environment set before CUDA allocator initialization. Do not attempt to switch allocator backends halfway through a training process. The scaffold leaves configuration validation, worker launch, and interpretation to the learner.

Part 2 - System and Memory Mental Model

Allocator settings change reuse and growth policy; they do not reduce the bytes genuinely needed by live tensors. A split-size restriction can preserve larger blocks but reduce flexibility and increase runtime or internal waste. It is appropriate to investigate when native split-block evidence supports the diagnosis, not as an automatic first response to every OOM.

Current documentation names `PYTORCH_ALLOC_CONF`; `PYTORCH_CUDA_ALLOC_CONF` is the compatibility name documented in older releases such as PyTorch 2.9. Check the installed version and set one supported variable consistently. Do not define conflicting values under both names. Preserve the exact configuration string and verify the effective backend in the worker.

Native-only options include split-size and rounding policies whose effects do not carry over to `cudaMallocAsync`. Some are ignored on that backend. Experimental expandable segments are a separate feature with platform and version constraints, not a guaranteed improvement. Keep unsupported or ignored settings out of the efficacy comparison.

Part 3 - Quantitative Prediction Before Execution

For configuration j, compare a vector rather than one score:

$$\mathbf{v}_j = (T_j, M_{\text{alloc},j}, M_{\text{reserve},j}, I_{s,j}, N_{\text{retry},j}, N_{\text{fail},j}).$$

Fields without meaningful backend support remain unavailable. Define speed ratio T_{base}/T_j only for equivalent completed work.

For a hypothetical native cache containing a 128 MiB inactive block and recurring 48 MiB requests, predict how preventing certain large-block splits might change reuse. List possible outcomes rather than assuming it must help. Explain why the raw setting's documented unit and observed byte capacities should both be recorded. Predict which measurements would distinguish preserved large-block fit from increased internal waste.

Part 4 - Experimental Design and Instrumentation

1. Confirm the baseline pattern and select a small, justified configuration matrix.
2. Inspect the installed documentation for native split-size configuration. Test one change at a time.

3. Evaluate power-of-two division rounding as a separate native configuration.
4. Launch sequential fresh workers with structured argument lists and an explicit environment. Record startup failures and effective backend.
5. Replay the exact same trace and useful work for every supported configuration.
6. Collect comparable timing and peak metrics plus native diagnostics where meaningful.
7. Evaluate the best calibration candidates on held-out traces without retuning.

Backend selection is process-wide in the documented allocator configuration. Avoid simultaneous benchmark workers competing for the same GPU. Treat optional expandable segments as a separately labeled experiment.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import os
import subprocess
import sys


@dataclass
class AllocatorTrial:
    trial_id: str
    environment_name: str
    configuration: str
    trace_path: str


def validate_trial_support(trial, version_information):
    # Reject unsupported options and record ignored settings.
    ...


def build_worker_environment(trial, base_environment):
    # Set one supported allocator variable before child startup.
```

```
    ...


def launch_fresh_worker(trial, worker_path):
    # Use an argument list and capture exit status and evidence.
    ...


def compare_configuration_trials(results):
    # Compare only equivalent completed work and supported metrics.
    ...


def E13(config):
    # Run a small calibration matrix and held-out evaluation.
    ...
    # Export effective settings, failures, and tradeoffs.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require proof that every worker used the intended backend and configuration. Mark invalid startup, unsupported feature, ignored setting, successful execution, and OOM as different outcomes. A worker that failed to start is not a fast run.

Report startup-inclusive and warmed time, completed work, peak memory, native split bytes where supported, retries, and output correctness. Explain a setting that helps one trace and harms another, or state that no such reversal was observed. Do not claim that a native-only option improved an asynchronous-backend run when the option was ignored.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use explicit not-measured and unsupported values. A fragmentation pattern or OOM that does not reproduce is an experimental outcome, not permission to invent hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Allocator tuning is workload-specific policy selection. Its credibility comes from reproducing the original problem, changing a defined mechanism, and evaluating the result outside the calibration trace.

Recommend a setting only with its version, backend, workload, and measured cost. A complicated configuration that marginally improves one trace may be less valuable than a simple workload-level change. Preserve the default baseline so future PyTorch upgrades can be assessed fairly.

19.28.15 Exercise 14

Exercise Title: Compare Native CUDA Caching Allocation Behavior Against cudaMallocAsync Backend Behavior Under Variable-Size Workloads

Part 1 - Exercise Contract and Why It Matters

Compare native and CUDA asynchronous allocator backends on identical variable-size workloads in isolated processes. Measure feasibility, useful throughput, latency, and comparable memory observations. Deliver a support matrix showing which diagnostics can and cannot be compared across backends.

Verify runtime, driver, device, and PyTorch support before running. PyTorch documentation specifies CUDA 11.4 or newer for the asynchronous backend, but that version requirement alone does not establish every feature's availability. Do not replace unavailable diagnostics with zeros.

Part 2 - System and Memory Mental Model

The native backend manages its own cached block and segment organization. The asynchronous backend uses CUDA's stream-ordered allocation mechanisms and driver-managed pools. Their reuse, release, and diagnostic behavior can differ even when the application issues identical tensor requests.

CUDA pool mechanisms can use virtual-memory management to improve reuse under some fragmentation patterns. This is not a guarantee that every large allocation succeeds or that all forms of fragmentation disappear. Live demand, pool growth requirements, external pressure, and supported implementation behavior still constrain allocations.

Native inactive-split statistics and some retry counters may be meaningless or always zero with the asynchronous backend. A zero therefore cannot prove superior fragmentation. Snapshot support and granularity must also be verified. Compare common application outcomes first, then interpret backend-specific diagnostics within their own scope.

Synchronization can affect both timing and memory reclamation. Use matched boundaries and distinguish a serial diagnostic run from a pipelined workload. Do not assume the CUDA driver's default pool release configuration is identical to the configuration used internally by a given PyTorch release.

Part 3 - Quantitative Prediction Before Execution

Define useful throughput for each backend as

$$U_b = \frac{W_{\text{completed}}}{T_b}.$$

For a common supported memory quantity M_b, compare absolute peak and final retention separately. Do not subtract unlike native and driver pool counters.

Using a hypothetical trace with a 64 MiB request followed by repeated 40 and 48 MiB requests, predict which questions must be measured: additional pool growth, reuse latency, retained capacity, and later large-request success. Explain why no native split-byte formula alone predicts the asynchronous backend's answer. Plan a matched single-stream control and a multi-stream variant with correct producer and consumer ordering.

Part 4 - Experimental Design and Instrumentation

1. Launch fresh workers with explicit native and asynchronous backend selection before allocator initialization.

2. Verify effective backend, supported APIs, and device identity. Record unavailable features as unsupported.

3. Replay stable, nearby-variable, broad-variable, and mixed-lifetime traces with identical useful work.

4. Measure complete wall time, request latency, application live bytes, and documented comparable memory observations.

5. Add native-specific statistics only to the native diagnostic view. Use supported driver or backend diagnostics for the other view without renaming them as equivalent.

6. Repeat with a correctly ordered multi-stream workload and with selected synchronization frequencies.

Catch and classify typed OOM outcomes. Do not keep retrying arbitrary device errors or infer a universal backend ranking from one trace.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch


@dataclass
class BackendCapabilities:
    backend: str
    supported_metrics: tuple
    snapshot_support: str
    environment_description: str


def inspect_backend_capabilities(device):
    # Verify support rather than assuming native diagnostics.
    ...


def run_common_trace(trace, capabilities, config):
    # Preserve useful work, lifetimes, and stream ordering.
    ...
```

```
def collect_backend_evidence(capabilities, phase):
    # Separate comparable observations from backend-specific data.
    ...


def compare_backends(native_result, async_result):
    # Keep unsupported fields out of numerical rankings.
    ...


def E14(config):
    # Coordinate isolated workers and matched trace replays.
    ...
    # Explain feasibility, retention, and latency differences.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require equivalent completed work and correct outputs. Backend selection must be confirmed from inside each worker. Show the diagnostic support matrix next to the results so missing values cannot be mistaken for perfect allocator behavior.

Report whole-trace success, useful throughput, latency tails, application peak demand, supported memory peaks, and final retention. Investigate performance changes caused by synchronization or different release behavior. If one backend fails a trace the other completes, preserve the request and environment evidence without claiming that the successful backend can never fragment.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use explicit not-measured and unsupported values. A fragmentation pattern or OOM that does not reproduce is an experimental outcome, not permission to invent hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Backend choice changes mechanisms and observability. A fair comparison begins with what the application needs to accomplish and uses internal statistics to explain those outcomes.

Recommend a backend for the tested workload and document its support constraints. Explain when simpler native diagnostics might be useful even if another backend performs better, and why a future runtime upgrade requires re-evaluation rather than blind reuse of an old conclusion.

19.28.16 Exercise 15

Exercise Title: Build an Early-Warning Detector That Predicts Fragmentation Risk Before the First Allocation Retry or OOM

Part 1 - Exercise Contract and Why It Matters

Build a detector that uses only information available before a defined allocation event to warn of fragmentation risk. Evaluate it against healthy cache retention, capacity pressure, and verified split-block patterns. Deliver warning traces, lead-time measurements, false-alarm rates, and clearly stated diagnostic uncertainty.

Define the target event and prediction horizon before calibration. The first retry or OOM is measured relative to a declared warmed baseline and counter epoch. An event already visible in a post-failure counter must not be used as an early predictor of itself.

Part 2 - System and Memory Mental Model

A useful warning can combine increasing native split bytes, deteriorating candidate coverage for a known queued request, low driver headroom, and persistent live separators. Each signal has alternative explanations: healthy cache can raise inactive bytes, pending work can temporarily reduce availability, and new shapes can legitimately increase live demand.

Use known queued request sizes only when the scheduler actually has them at decision time. Otherwise forecast them from past data and record uncertainty. A snapshot is an observation, not a reservation; its age and scope limit the warning's confidence.

Retries and OOM events are outcomes, while fragmentation attribution requires additional evidence. Maintain separate labels for event prediction and mechanism diagnosis. Some backends lack meaningful retry metrics, so do not transfer a native label definition silently. No positive events in an evaluation set means detection recall cannot be estimated, not that recall is perfect.

Part 3 - Quantitative Prediction Before Execution

For warning time t_w and later event time t_e, define positive lead time

$$\ell = t_e - t_w.$$

For predeclared event windows, calculate precision and recall:

$$\text{precision} = \frac{TP}{TP + FP}, \qquad \text{recall} = \frac{TP}{TP + FN}.$$

Undefined denominators remain undefined. Also report false alarms per unit runtime and time spent in warning state.

Use a hypothetical sequence with warnings at steps 10 and 40, a retry at 14, and no later event through step 60. Under a five-step horizon, classify the warnings and calculate lead time for the matched event. Explain how changing the horizon changes the evaluation. Predict a healthy-cache false positive for a detector using only reserved-minus-allocated bytes.

Part 4 - Experimental Design and Instrumentation

1. Define causal features, observation cadence, target event, warning horizon, and matching rules.

2. Split traces chronologically or by independent workload family into calibration and evaluation sets.

3. Build a simple threshold-based detector before considering a learned model.

4. Replay observations in order and prohibit access to future snapshots, outcome counters, or future shapes not present in the actual queue.

5. Measure event prediction and fragmentation attribution separately against controlled traces.

6. Add noisy headroom, missing metrics, stale snapshots, and long stable periods.

Bound monitoring overhead and compare monitored with unmonitored throughput. Mark incomplete event windows near the end of a run as censored or apply a declared evaluation rule.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch


@dataclass
class WarningDecision:
    step_id: int
    risk_level: str
    reason_codes: tuple
    observation_age_seconds: float
    predicted_request_bytes: object


def build_causal_features(history, known_queue, capabilities):
    # Exclude future outcomes and unsupported diagnostics.
    ...


def decide_warning(features, calibrated_policy):
    # Apply a declared horizon and uncertainty handling.
    ...


def match_warnings_to_events(warnings, events, horizon):
    # Handle repeated warnings and censored final windows.
    ...
```

```
def E15(config):
    # Calibrate on separate traces and freeze the detector.
    ...
    # Report event prediction, attribution, and overhead.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require positive lead time for a true early warning. Audit feature timestamps to detect leakage from post-retry counters or future allocation history. Include healthy-cache and capacity-only controls in the evaluation.

Report precision, recall where defined, false alarms, missed events, lead-time distribution, warning duration, and monitoring cost. Present a mechanism-confidence label separately from the event alarm. If no retry or OOM occurs, report the detector's warning behavior and leave event-recall claims unsupported. Explain how missing native metrics change behavior instead of treating their absence as low risk.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use explicit not-measured and unsupported values. A fragmentation pattern or OOM that does not reproduce is an experimental outcome, not permission to invent hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Prediction is useful only if it gives time for a valid intervention and avoids excessive false alarms. A detector that recognizes an OOM after it happens is a reporter, not an early-warning system.

Use the evaluation to choose an operating point appropriate to the measured cost of waiting or reshaping. Preserve uncertainty in the diagnosis so the later scheduler can choose a bounded action without pretending the allocator's future is known.

19.28.17 Exercise 16

Exercise Title: Trigger Fragmentation During a Long Transformer Training Run and Diagnose the Exact Allocation Pattern Responsible for Memory Degradation

Part 1 - Exercise Contract and Why It Matters

Run a bounded long-duration Transformer training experiment with variable sequence lengths and investigate any memory degradation. Attempt to create a reproducible fragmentation-prone pattern, then identify the responsible allocation sizes, lifetimes, and call sites through history and controlled replay. Deliver a training timeline, incident dossier, and minimized allocation trace. If fragmentation does not appear, report that outcome and explain the observed stabilization.

Keep model dimensions, optimizer, precision, checkpointing, and attention backend fixed in the primary study. Distinguish fragmentation from legitimate activation growth, optimizer initialization, retained autograd graphs, and temporary operator workspaces. The title's proposed failure is a hypothesis to test, not a required fabricated result.

Part 2 - System and Memory Mental Model

Sequence-length changes alter multiple allocation families: embeddings, projections, saved activations, masks, attention temporaries, and outputs. Some sizes scale linearly with length; explicitly materialized attention scores scale quadratically. Fused attention may avoid those score tensors, so record the actual backend rather than applying one universal formula.

Optimizer state and gradient buffers can create startup growth unrelated to fragmentation. A logger that stores loss tensors with their graphs can produce increasing live demand, while a logger of detached CPU scalars does not preserve those graphs. Include a bounded, separately labeled retention control to distinguish this mechanism from allocator trapping.

A credible diagnosis links a size-and-lifetime sequence to partially occupied segments and subsequent allocation pressure. Stack traces identify candidate call sites, but causal attribution needs a counterfactual replay. Change one suspected allocation pattern while preserving useful work as closely as possible, and disclose changes to arithmetic, shape, or peak live demand.

Do not restore training state by simply clearing gradients after an arbitrary failed update. Place diagnostic replays in separate workers with known model and optimizer state. Recovery during training requires the transactional discipline developed in DMM23.

Part 3 - Quantitative Prediction Before Execution

For batch B, length S, hidden width D, and element size s, a single activation family may require

$$q_{\text{linear}} = BSDs.$$

For explicitly materialized attention scores with H heads, the corresponding family requires

$$q_{\text{score}} = BHS^2s.$$

Neither equation is total training memory.

For hypothetical $B = 2, D = 512, H = 8$, compare lengths 256 and 384 at a declared dtype. Predict the relative growth of these two families and explain why the measured peak can differ. Add a persistent 64 MiB allocation family to the lifetime model and predict how it could separate transient blocks if they share segments.

Before running, sketch expected startup, steady-state, shape-shift, and possible degradation phases. State what stable active bytes with rising split bytes would suggest, and what rising active bytes would require you to investigate first.

Part 4 - Experimental Design and Instrumentation

1. Build a small Transformer training workload with bounded lengths and reproducible useful-token traces. Record attention implementation and optimizer settings.

2. Establish a fixed-length baseline including complete forward, backward, and optimizer steps.

3. Run a nearby-length oscillation trace and a phase-shift trace after a declared warmup.

4. Log pre-forward, post-forward, post-backward, post-update, and completed-cleanup memory at selected steps. Use scalar-only training logs.

5. Enable bounded history in targeted windows or a separate diagnostic replay. Capture before suspected degradation and near a retry or OOM if one occurs.

6. Match problematic blocks to allocation stacks, sizes, generations, and persistent neighbors.

7. Minimize the suspected trace and test one counterfactual mitigation while preserving workload semantics or explicitly documenting differences.

Compare losses and gradients for short deterministic reference windows. Keep lengthy numerical validation and CPU state snapshots outside the primary timing run.
Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch


@dataclass
class TrainingObservation:
    step_id: int
    phase: str
    sequence_length: int
    useful_tokens: int
    loss_scalar: float
    allocator_state: object


def build_training_reference(config):
    # Establish model, optimizer, precision, and backend.
    ...


def run_training_phase(model, optimizer, batch, observer):
    # Expose full lifecycle phases without retaining graphs.
    ...


def capture_diagnostic_window(window, config):
    # Bound history and preserve pre-incident evidence.
```

```
    ...


def trace_block_to_operation(snapshot, training_log):
    # Match call sites, sizes, generations, and live neighbors.
    ...


def replay_suspected_pattern(pattern, counterfactual, config):
    # Isolate one mechanism in a fresh known-state worker.
    ...


def E16(config):
    # Compare stable, variable, and bounded retention controls.
    ...
    # Diagnose only mechanisms supported by replay evidence.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require complete-step measurement and a clear startup boundary. Report useful tokens, physical tokens, loss, allocated and reserved peaks, native split trajectory, retries, and history-window coverage. Label missing or overwritten history explicitly.

An exact-pattern claim needs a reproducible minimized trace and a counterfactual that changes the suspected mechanism. If several mechanisms remain plausible, report a candidate diagnosis. A growing list of live loss graphs is a retention defect even if fragmentation also exists. A larger sequence requiring more activations is increased demand, not evidence of allocator degradation.

Preserve every OOM or unsupported-shape outcome and verify that successful runs complete the intended updates. Do not call a shorter or truncated workload a recovery of the same training experiment.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use explicit not-measured and unsupported values. A fragmentation pattern or OOM that does not reproduce is an experimental outcome, not permission to invent hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

Long training runs make allocator history important, but they also create many confounding state changes. The strongest investigation reduces a complex incident to a small, explainable allocation and lifetime pattern.

Explain which part of the diagnosis is directly observed, which is inferred, and which is tested by intervention. Recommend a mitigation only for the supported cause. If the allocator remains stable, describe the conditions that enabled that stability rather than forcing a fragmentation narrative.

19.28.18 Exercise 17

Exercise Title: Build a Fragmentation-Aware Scheduler That Delays, Reuses, or Reshapes Allocations When the Largest Contiguous Free Block Becomes Too Small

Part 1 - Exercise Contract and Why It Matters

Build a scheduler that uses request-aware fragmentation evidence to choose among waiting, reusing owned buffers, or applying a semantics-preserving shape policy. Deliver decision traces, ownership checks, and a comparison with a scheduler using only aggregate headroom.

Interpret largest contiguous free block at the correct layer. An application-owned pool can expose exact eligible buffers; a native allocator snapshot provides observed inactive candidates and cannot reserve them for the scheduler. The implementation must preserve this distinction.

Part 2 - System and Memory Mental Model

A pending request can be delayed until known consumers finish, provided the schedule still makes progress. Waiting is ineffective when no scheduled release can improve the state. Add deadlines or bounded retries and identify infeasible requests explicitly.

Explicit reuse requires a compatible, owned buffer and a valid completed lease. A snapshot address is not a tensor handle and does not grant permission to select or overwrite an allocator block. For ordinary PyTorch allocations, the allocator still chooses the actual block.

Reshaping by a view does not reduce backing bytes. Bucketing or padding can improve future reuse while increasing immediate demand; token or micro-batch splitting can change execution and requires correct normalization and effective-batch semantics. Do not truncate content to make the scheduler appear feasible.

Cached candidate size below the request is a warning, not a universal refusal condition, because new segments may be obtained. Conversely, an apparently large candidate is not an allocation guarantee. Combine evidence with live-demand estimates, known release events, observation age, and explicit uncertainty.

Part 3 - Quantitative Prediction Before Execution

For action a, define an estimated completion cost

$$\widehat{J}(a) = \widehat{T}_{\text{wait}}(a) + \widehat{T}_{\text{execute}}(a) + \widehat{T}_{\text{transform}}(a).$$

Minimize over actions that preserve semantics and satisfy the scheduler's declared admission model. Estimated capacity is not a physical reservation.

For a hypothetical 40 MiB request, a 24 MiB largest cached candidate, and a 48 MiB owned buffer becoming free after 3 ms, compare waiting with a fresh allocation when driver headroom is uncertain. State the information needed to choose responsibly. Then analyze a proposed padded class of 64 MiB: explain why it may aid future reuse yet fail the current budget. Identify a starvation case for always favoring small requests.

Part 4 - Experimental Design and Instrumentation

1. Define typed request metadata, owned-buffer eligibility, dependencies, deadlines, and known future release events.

2. Build a host simulator for wait, reuse, bounded allocation attempt, and supported shape transformations.

3. Compare an aggregate-headroom scheduler with a request-aware policy on identical traces.
4. Integrate CUDA only after ownership and progress invariants pass.
5. Measure wait time, tail latency, completed work, rejected requests, process peaks, and allocator-specific diagnostics.
6. Add stale snapshots, delayed completions, and requests that cannot fit any supported strategy.

Apply hysteresis or cooldown to avoid immediate reversal of costly shape decisions. Keep backend switching out of the live scheduler; that belongs to process startup experiments.
Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch


@dataclass
class ScheduledRequest:
    request_id: int
    required_bytes: int
    logical_shape: tuple
    deadline_seconds: float
    dependencies: tuple


def observe_request_context(request, allocator, owned_pool):
    # Separate exact owned capacity from snapshot candidates.
    ...


def enumerate_legal_actions(request, context, policy):
    # Preserve data, normalization, ownership, and progress.
    ...
```

```
def estimate_action_cost(action, calibrated_costs):
    # Include wait and transformation costs with uncertainty.
    ...


def execute_scheduled_action(action, runtime):
    # Enforce real dependencies and bounded failure handling.
    ...


def E17(config):
    # Compare schedulers on the same request and release trace.
    ...
    # Audit starvation, stale evidence, and useful work.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Acceptance requires no early reuse, no hidden data truncation, and bounded behavior for infeasible requests. Show that the scheduler cannot obtain ownership merely by reading a snapshot address. Validate every shape transformation against a numerical reference.

Report action counts, wait duration, transformation cost, completed useful work, starvation indicators, OOM outcomes, and memory peaks. Include a case where new allocation succeeds despite a small cached candidate and a case where waiting fails to improve feasibility. These prevent the policy from treating one diagnostic as an infallible oracle.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use explicit not-measured and unsupported values. A fragmentation pattern or OOM that does not reproduce is an experimental outcome, not permission to invent hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

A fragmentation-aware scheduler should use incomplete information to make better bounded decisions. It cannot turn allocator observations into ownership or make live memory disappear.

Explain which action produced the measured benefit and what correctness assumptions it required. Prefer a policy that admits uncertainty, preserves progress, and handles failure cleanly over one that reports perfect prevention by rejecting all difficult work.

19.28.19 Exercise 18

Exercise Title: Stress-Test Fragmentation Management with Thousands of Mixed-Lifetime, Mixed-Size CUDA Allocations and Compare Multiple Mitigation Strategies

Part 1 - Exercise Contract and Why It Matters

Build a reproducible stress suite combining size variability, long-lived separators, delayed completion, and changing demand. Compare multiple mitigations on the same exact traces and resource limits. Deliver a benchmark matrix, replay artifacts, and minimized failures.

Generate thousands of requests while bounding simultaneous live bytes and total experiment duration. Separate simulator failures, application budget refusals, allocator OOM, and unrelated CUDA errors. The suite should expose weak diagnoses as well as weak policies.

Part 2 - System and Memory Mental Model

Mixed-size and mixed-lifetime traces can create more challenging histories than either factor alone. Yet some traces will stabilize, and some failures will be ordinary capacity limits. Keep healthy and capacity-only controls in the suite so a detector cannot succeed by labeling every pressure event fragmentation.

Compare default allocation, bucketing, rounded buffers, bounded reuse, legal lifetime grouping, and justified allocator settings. Backend variants need separate workers and capability-aware diagnostics. Changing a mitigation must not silently change the amount of useful work.

Stress correctness as well as memory efficiency. Delayed consumer events, stale leases, and missing references can produce hazards unrelated to fragmentation. Test invalid transitions in a host simulator before they reach CUDA. Use request-specific patterns and controlled completion to validate accepted work.

Keep diagnostic overhead bounded. A huge history buffer or tensor-retaining log can create the very memory growth being investigated. Preserve raw traces and short targeted snapshots instead of recording every detail indefinitely.

Part 3 - Quantitative Prediction Before Execution

For each exact trace, calculate a logical live-demand lower bound

$$Q_{\text{peak}} = \max_{t} \sum_{i \in \mathcal{L}(t)} q_i.$$

Compare measured peak capacity with that bound while accounting for rounding, pending work, and operator temporaries.

For mitigation m, report a vector of completed coverage, useful throughput, peak VRAM, retry outcomes, and diagnostic evidence. A normalized throughput comparison is valid only when the numerator represents equivalent useful work.

Design a hypothetical 10,000-request suite with a fixed maximum live set and three lifetime classes. Predict which scenarios challenge each mitigation: boundary sizes, rare large requests, persistent separators, or delayed completion. State why a lower fragmentation ratio caused by rejecting large requests is not a fair improvement.

Part 4 - Experimental Design and Instrumentation

1. Generate exact integer request sizes, allocation roles, release events, stream dependencies, and seed metadata.
2. Validate a reference state model and the trace's live-demand bound.
3. Run each mitigation in a fresh worker with matched environment and useful work.
4. Vary one stress dimension at a time, then run selected combinations.
5. Collect minimal performance evidence for all runs and detailed diagnostics for representative failures or disagreements.
6. Minimize a failing trace while preserving the mechanism, initial allocator setup, and relevant lifetime dependencies.

7. Evaluate policies on held-out seeds and workload families without retuning.

Use timeouts and explicit outcomes. Do not kill unrelated GPU processes or fill all available VRAM indefinitely to manufacture a dramatic failure.

Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import random
import torch


@dataclass
class StressScenario:
    scenario_id: str
    seed: int
    request_count: int
    live_limit_bytes: int
    lifetime_policy: str
    size_policy: str


def generate_mixed_trace(scenario):
    # Preserve exact sizes, roles, and completion dependencies.
    ...


def verify_trace_demand(trace, limits):
    # Validate logical live demand before allocating on CUDA.
    ...


def run_strategy_worker(trace, strategy, config):
    # Use isolated startup configuration and bounded execution.
    ...
```

```
def classify_stress_outcome(result, diagnostic_evidence):
    # Separate capacity, placement, ownership, and support failures.
    ...


def minimize_reproducer(trace, initial_state, predicate):
    # Preserve the causal pattern and replay configuration.
    ...


def E18(config):
    # Compare strategies and retain all failed or censored runs.
    ...
    # Export a fair benchmark matrix and replay artifacts.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require exact trace identity across strategy comparisons or a clearly documented semantics-preserving transformation. Record all rejected and unfinished requests. Successful-only latency statistics must not conceal reduced workload coverage.

Report seed, request count, live-demand bound, coverage, useful throughput, memory peaks, native split metrics where supported, retries, OOM, and correctness failures. Include run-to-run variation rather than only the best seed. A zero observed-failure count establishes the tested scope, not universal allocator correctness.

Every claimed fragmentation failure should have block or history evidence, or remain a candidate diagnosis. Preserve the smallest reproducer and its last valid accounting state.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use explicit not-measured and unsupported values. A fragmentation pattern or OOM that does not reproduce is an experimental outcome, not permission to invent hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

A good stress suite challenges both the mechanism and the story told about it. It should reveal when an apparent improvement comes from extra padding, lower coverage, or a changed synchronization pattern.

Use the results to rank mitigations by feasibility, correctness, and useful performance under the actual budget. State the remaining untested conditions and avoid extending a single-device native-backend conclusion to every CUDA environment.

19.28.20 Exercise 19

Exercise Title: Build and Benchmark a PyTorch Fragmentation Management System That Detects, Explains, and Reduces CUDA Memory Waste Under Highly Dynamic Workloads

Part 1 - Exercise Contract and Why It Matters

Integrate observation, request-aware risk detection, causal diagnosis, and legal mitigation into a coherent fragmentation-management runtime. Deliver the student-completed implementation, a benchmark dossier, and a support contract that precisely states what the system can observe and control. The scaffold intentionally leaves all calculations, decisions, CUDA operations, and validation unfinished.

The system should detect concerning allocation patterns, explain them with evidence, and select a bounded intervention while preserving useful work. It must also recognize healthy cache retention, genuine capacity exhaustion, unsupported metrics, and inconclusive evidence. Correctness and diagnostic honesty are independent requirements from performance.

Part 2 - System and Memory Mental Model

Separate five responsibilities. The observer collects backend-aware statistics and bounded history. The request model describes logical and backing demand, dependencies, and known releases. The detector estimates risk from causal observations. The diagnoser tests explanations against snapshot and replay evidence. The controller chooses only supported actions.

The controller may delay work, reuse owned buffers, bucket supported shapes, or change future allocation lifetimes. It cannot choose an arbitrary cached block by address, compact live tensors without handling aliases, or switch allocator backend inside a running process. Configuration and backend selection belong to offline calibration and worker startup.

Cache release is an optional measured intervention, not a universal fix. It does not free live tensors or generally compact partially occupied ordinary segments. Repeated cache release can increase allocation cost. Similarly, lower inactive split bytes can merely reflect a larger active application pool.

Preserve the distinction between a logical resource budget, allocator reservation, and physical device availability. External processes and unmanaged library allocations can change available memory despite a correct internal policy. Failure handling must classify typed OOM separately from unrelated runtime errors and preserve training-state integrity when training is supported.

Part 3 - Quantitative Prediction Before Execution

Define the optimization goal with useful work W, complete elapsed time T, and declared memory and correctness constraints:

$$\max_{\pi} \frac{W_{\text{completed}}(\pi)}{T(\pi)} \quad \text{subject to valid semantics and declared budgets.}$$

Do not optimize a fragmentation ratio alone. Report a diagnostic vector containing active demand, reserved capacity, internal slack, supported split-block signals, request coverage, and observation uncertainty.

For action a, estimate benefit and cost on comparable boundaries:

$$\widehat{\Delta M}(a) = \widehat{M}_{\text{before}} - \widehat{M}_{\text{after}}, \qquad \widehat{\Delta T}(a) = \widehat{T}_{\text{action}} + \widehat{T}_{\text{later penalty}}.$$

The memory term must name its accounting layer. Reducing application capacity is different from reducing driver-visible usage.

Construct a hypothetical case with stable live demand, increasing native split bytes, a known large queued request, and limited growth headroom. Predict the evidence needed to choose waiting, owned-buffer reuse, or a legal shape policy. Then construct a capacity-only case and explain why the same diagnosis would be wrong.

Part 4 - Experimental Design and Instrumentation

1. Write a support matrix for backends, snapshot schemas, buffer roles, shapes, precision, and training or inference semantics.

2. Integrate scalar observations with typed request and lifetime metadata. Preserve unsupported and stale-data states.

3. Calibrate risk thresholds and action costs on separate traces. Freeze them before evaluation.

4. Validate ownership, dependency, budget, and progress invariants in a host simulator.

5. Run CUDA correctness checks, then compare default allocation, individual mitigations, and the integrated controller.

6. Evaluate stable, variable, mixed-lifetime, Transformer, and capacity-only traces. Include healthy-cache negative controls.

7. Trigger bounded diagnostic capture on warnings or incidents. Minimize representative failures and verify a counterfactual cause.

8. Run ablations for monitoring, candidate-fit information, hysteresis, reuse, and shape policies, changing one component at a time.

Measure startup-inclusive and warmed useful throughput, tail latency, memory peaks, workload coverage, and monitoring overhead. Preserve all failed and unsupported configurations.
Reproducibility record. Preserve GPU/device identity, PyTorch and CUDA versions, allocator backend, seed, model dimensions, optimizer, precision, shape, and initial-state identifier. Record cold-start behavior separately from warmed execution. Synchronize only at deliberate measurement boundaries, reset peaks for the declared region. Allocation-only experiments need an explicit allocation region; training experiments include forward, backward, and optimizer update. Export CPU-only scalars: never retain live autograd graphs in the evidence log. Use a fixed seed for the primary comparison and additional seeds to check repeatability. State whether timing includes transfer, collation, calibration, and recovery.

Part 5 - PyTorch/CUDA Implementation

Complete the following scaffold. Every ... marks work for the learner: calculations, control decisions, CUDA operations, validation, and evidence export remain intentionally unfinished. The scaffold is not a runnable implementation until those steps are completed. Keep the implementation explicit; do not hide the exercise inside a high-level trainer.

```
from dataclasses import dataclass
import torch


@dataclass
class RuntimeContract:
    backend: str
    supported_roles: tuple
    supported_actions: tuple
    memory_budget_bytes: int
    maximum_observation_age: float
```

```
@dataclass
class Diagnosis:
    incident_id: int
    classification: str
    evidence_ids: tuple
    confidence: str
    unresolved_causes: tuple


class FragmentationManager:
    def __init__(self, contract, observer, calibrated_policy):
        # Establish ledgers, support checks, and bounded history.
        ...

    def observe(self, request_context):
        # Preserve metric scope, support, age, and reset epoch.
        ...

    def predict_risk(self, observation, known_queue):
        # Use causal evidence and request-specific uncertainty.
        ...

    def diagnose(self, incident, snapshots):
        # Separate capacity, retention, pending work, and trapping.
        ...

    def enumerate_actions(self, request, diagnosis):
        # Admit only ownership-safe and semantic-preserving actions.
        ...

    def choose_action(self, actions, estimated_costs):
        # Balance useful performance, budget, and bounded progress.
        ...

    def execute_action(self, action):
        # Enforce dependencies and classify execution failures.
        ...

    def capture_incident(self, incident):
        # Save bounded evidence without retaining live graphs.
        ...

    def audit(self):
        # Reconcile owned capacity and completed useful work.
        ...
```

```
    def shutdown(self, deadline):
        # Stop admission, drain supported work, and report leftovers.
        ...


def benchmark_system(trace_suite, strategies, config):
    # Compare equivalent workloads and preserve failed outcomes.
    ...


def E19(config):
    # Validate contracts, causal features, and runtime invariants.
    ...
    # Run correctness, performance, ablation, and stress phases.
    ...
    # Export a diagnosis dossier and conditional recommendation.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Acceptance requires four independent gates. First, accepted work preserves the intended numerical and data semantics. Second, ownership and completion rules prevent unsafe reuse. Third, every diagnosis separates direct evidence from inference and unknowns. Fourth, performance comparisons account for complete useful work and all resource costs.

The dossier must include environment and configuration, support matrix, exact traces, predictions, observations, snapshot references, numerical errors, memory accounting by layer, warning precision and lead time, action counts, latency distributions, coverage, and useful throughput. Report internal slack and retained idle capacity separately from native split-block indicators.

Demonstrate safe behavior with missing metrics, stale snapshots, no feasible action, delayed completion, and an unrelated runtime error. A scheduler that rejects every difficult request does not demonstrate successful fragmentation management. If no benefit over default allocation is observed, state that clearly and identify the tested scope.

Evidence format. Keep prediction, observation, and interpretation in separate columns. Each request or experiment needs an ID, configuration, input shape, budget in bytes, peak allocated/reserved bytes, elapsed seconds, outcome, and failure phase when applicable. Use explicit not-measured and unsupported values. A fragmentation pattern or OOM that does not reproduce is an experimental outcome, not permission to invent hardware results. Include the smallest counterexample that invalidates a proposed rule, or state that no counterexample was observed in the tested range.

Part 7 - Engineering Interpretation and Philosophy

The completed system should make a modest but precise promise: it observes supported memory behavior, explains incidents with evidence, and applies legal workload changes when their expected benefit justifies their cost. It does not eliminate every form of memory waste or guarantee that external pressure cannot cause OOM.

Use the results to recommend the smallest effective set of mechanisms. Explain when default allocation is sufficient, when workload regularization helps, and when backend-specific tuning

deserves a separate evaluation. Finish with a supported deployment scope and one concrete experiment needed before expanding that scope.

19.29 DMM27

Chapter Title: Tensor Reuse and Buffer Reuse
Specialization: Dynamic Memory Management and CUDA Systems

19.29.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Tensor Storage Reuse, Temporary Buffers, Workspace Lifetimes, and Allocation Elimination

Part 1 - Exercise Contract and Why It Matters

Build a lifecycle model for tensor objects, logical values, backing storage, temporary outputs, and operator workspaces. Identify which allocations can be removed by retaining an owned destination and which values must remain distinct. The result must explain both storage savings and the conditions that make reuse legal.

Part 2 - System and Memory Mental Model

A tensor describes values through metadata and backing storage. Different views can share storage; assigning a fresh result to an old variable does not imply storage reuse. Application reuse retains ownership across iterations, whereas the caching allocator can recycle a released allocation underneath the program. A value remains protected until its final consumer, including aliases and asynchronous device readers, has finished or is correctly ordered before the next write. Start with single-stream inference, not saved training activations.

Part 3 - Quantitative Prediction Before Execution

For owned slot capacities c_j, calculate $B_{\text{pool}} = \sum_j c_j$. For distinct live values, calculate $B_{\text{live}}(t) = \sum_{i \in L(t)} b_i$, without charging aliases as independent physical storage. Consider three 4 MiB values: a remains live while b is consumed; c begins after b is no longer needed. Predict which values may share a slot. Repeat after a view of b escapes to a later consumer. These are hypothetical predictions, not GPU measurements.

Part 4 - Experimental Design and Instrumentation

Draw separate logical-value and physical-storage tables. Mark each producer, reader, alias, and release boundary. Compare fresh outputs, a persistent output, and a view of that output. Use deterministic values and inspect each generation before overwrite. State whether returned outputs are borrowed until the next call or independently retainable. Record setup capacity separately from warmed allocation requests and reserved memory.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the code is deliberately incomplete and is not a finished implementation.

```
from dataclasses import dataclass
import torch
```

```
@dataclass
class TensorLifetime:
    key: str
    payload_bytes: int
    birth: int
    final_consumer: int
    alias_group: str
    owner: str


def storage_record(tensor, owner, generation):
    # Record base, offset, capacity, shape, and ownership evidence.
    ...


def legal_reuse(previous, following, lifetime_table):
    # Check value lifetimes and all aliases before permitting a write.
    ...


def E00(config):
    # Construct the three cases and verify the predicted sharing.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Check intermediate values, not only the final result. A repeated address after release does not establish continuous ownership, and a different view pointer does not establish different backing storage. Include an escaped-output case that makes an otherwise valid overwrite illegal. Workspace demand not observed by the instrument must remain unknown.

Part 7 - Engineering Interpretation and Philosophy

Reuse shortens physical storage ownership requirements without shortening the semantic lifetime of a required value. The optimization starts with an explicit consumer contract; it does not start with a pointer or a lower memory counter.

19.29.2 Exercise 01

Exercise Title: Measure the Number and Total Bytes of Temporary CUDA Tensors Created During a Repeated PyTorch Workload

Part 1 - Exercise Contract and Why It Matters

Measure temporary allocation activity during a repeated CUDA workload. Separate logical tensor-producing operations, requested payload bytes, allocator events, and driver allocation calls when observable. A final live-memory difference is not the cumulative amount of temporary storage requested.

Part 2 - System and Memory Mental Model

A materialized result can request storage even when the allocator satisfies it from its cache. A view creates tensor metadata without allocating another full payload. Profiler memory activity, operator call counts, and backend counters have different scopes. Nested operator aggregates can double count activity; net memory change can cancel allocations against frees. Choose one supported event layer and document its attribution rules.

Part 3 - Quantitative Prediction Before Execution

For N iterations that deliberately materialize two independent n-element outputs of element width s, predict logical request traffic $Q = 2Nns$. Exclude inputs, views, and unobserved workspaces from this formula. Estimate peak live bytes from the lifetime timeline rather than from Q. Add one view and predict the effect on object count and backing payload separately.

Part 4 - Experimental Design and Instrumentation

Use fixed shapes, deterministic inputs, eager execution, and an explicitly bounded repeated region. Warm up before the main census. Collect a short memory trace, preserve positive allocations and frees separately, and reconcile one tiny trace by hand. Compare supported allocator-counter deltas over the same region. Run a view-only control and repeat timing without profiling. Record GPU, PyTorch/CUDA versions, backend, seed, shape, dtype, and reset boundaries.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the code is deliberately incomplete and is not a finished implementation.

```
from dataclasses import dataclass
import torch

@dataclass
class AllocationCensus:
    logical_requests: object
    payload_bytes: object
    allocator_events: object
    driver_events: object
    unsupported_fields: tuple


def repeated_workload(inputs, repetitions):
    # Materialize the declared outputs; consume and release each safely.
    ...


def collect_census(workload, config):
    # Choose supported profiling/counter APIs and one accounting layer.
    ...


def reconcile_events(trace, logical_operations):
    # Deduplicate nested attribution and retain unknown allocations.
    ...
```

```
def E01(config):
    # Validate the short trace before measuring a longer replay.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not sum inclusive memory columns across nested operators as unique traffic. A zero final delta does not mean zero allocation. Unsupported profiler schemas or backend fields must not become zero counts. Compare profiler-on and profiler-off behavior and report observer overhead. Direct library allocations outside PyTorch's allocator may be invisible to its memory diagnostics.

Part 7 - Engineering Interpretation and Philosophy

An allocation count is meaningful only when the counted entity and interval are explicit. Preserve traffic, peak demand, and final residency as separate measurements so later reuse claims can be audited at the same layer.

19.29.3 Exercise 02

Exercise Title: Replace Repeated Tensor Creation with a Preallocated CUDA Output Buffer and Measure Allocation Reduction

Part 1 - Exercise Contract and Why It Matters

Replace repeated fixed-shape result creation with one preallocated CUDA output. Compare numerical values and output-allocation requests with a fresh-result baseline. Define the result as borrowed until the next write, or charge the extra storage needed for independently retained outputs.

Part 2 - System and Memory Mental Model

A destination allocated before the loop moves allocation cost into setup and retains capacity between calls. It does not remove setup or every internal workspace. Use a supported destination-writing operation with disjoint inputs and output. Start with matching shape, dtype, device, and contiguous layout in inference mode. Do not disable gradients in a training workload merely to admit an output-buffer API.

Part 3 - Quantitative Prediction Before Execution

For N calls producing b payload bytes, predict N baseline logical output requests versus one setup request when direct reuse is supported. Nominal removed request traffic is $(N-1)b$, not necessarily peak-VRAM savings. Predict why a promptly released baseline can already have a small peak. Then retain K outputs and calculate the additional ownership required.

Part 4 - Experimental Design and Instrumentation

Pre-generate inputs and establish an independent reference. Allocate the reuse destination once, validate every generation before overwrite, and inspect storage metadata. Measure setup-inclusive and warmed runs separately. Count requests at E01's layer; reset peaks at declared boundaries but retain absolute owned capacity. Include a test where the caller keeps an old output and require an explicit lease rejection, extra slot, or charged copy.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the code is deliberately incomplete and is not a finished implementation.

```
from dataclasses import dataclass
import torch

@dataclass
class OutputLease:
    generation: int
    shape: tuple
    borrowed_until_next_write: bool


def fresh_step(inputs):
    # Produce the reference result using ordinary allocation.
    ...


def buffered_step(inputs, destination, lease):
    # Validate compatibility and write directly where supported.
    ...


def E02(config):
    # Allocate the persistent destination and account for setup.
    ...
    # Check outputs, lease behavior, and output request reduction.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Check reference independence: it must not alias the destination being tested. Report absolute allocated/reserved peaks and end-state capacity; a peak reset after pool creation must not conceal the pool. Unexpected storage changes need shape, dtype, layout, and API investigation. A correct final result does not validate an old retained output that was silently overwritten.

Part 7 - Engineering Interpretation and Philosophy

Preallocation trades flexible result ownership for predictable persistent capacity. Recommend it only when the caller accepts the output lifetime and the measured request benefit justifies retained memory and compatibility checks.

19.29.4 Exercise 03

Exercise Title: Reuse a Persistent Tensor with copy_() Instead of Allocating a New Tensor Every Iteration and Compare Runtime Behavior

Part 1 - Exercise Contract and Why It Matters

Compare a fresh independent copy with copying successive already available source values into one persistent destination. Measure destination requests, copy traffic, values, and complete runtime. Separately examine compute-then-copy so source-expression allocations are not hidden.

Part 2 - System and Memory Mental Model

The destination can remain owned while its values change. Copying still moves bytes, and a newly computed source can still allocate every iteration. Begin with identical shape, dtype, and device; broadcasting, conversion, and transfers are separate cases. A view of the destination sees later writes. Keep the initial experiment outside gradient recording and define when consumers release each generation.

Part 3 - Quantitative Prediction Before Execution

For a b-byte payload copied N times, predict Nb logical copied bytes. A simple same-device bandwidth model reads and writes about $2Nb$ bytes, excluding cache and implementation effects. Compare $T_{\text{fresh}} \approx T_{\text{allocation}} + T_{\text{copy}}$ with $T_{\text{reuse}} \approx T_{\text{copy}} + T_{\text{checks}}$. Predict what remains unchanged when the source itself is freshly allocated.

Part 4 - Experimental Design and Instrumentation

Prepare deterministic sources with a declared lifetime. Compare a fresh clone and persistent-destination copy with identical consumption. Inspect each result before the next overwrite and correlate destination metadata with allocation evidence. Run a compute-then-copy control and charge its source creation. Time completion at comparable boundaries, excluding validation from both timing paths or including it in both. Test retained views separately.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the code is deliberately incomplete and is not a finished implementation.

```
import torch


def fresh_copy(source):
    # Create an independent destination for the reference contract.
    ...


def copy_into_existing(source, destination, contract):
    # Check shape/dtype/device, lease, and gradient restrictions.
    ...
    # Perform the copy and establish any required dependencies.
    ...


def account_source_and_destination(events):
    # Separate source allocations, destination requests, and traffic.
    ...
```

```
def E03(config):
    # Compare existing-source and compute-then-copy cases explicitly.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Check each source generation against an independent reference. A persistent address does not prove source allocation elimination. A result requiring broadcasting or conversion is a changed test contract and must be labeled. Report total bytes copied and numerical error. Retaining historical outputs requires copies or additional slots whose costs belong in the comparison.

Part 7 - Engineering Interpretation and Philosophy

Persistent copying is useful when the destination must remain stable. Direct production into a destination solves a different problem: it can avoid the source-result materialization that compute-then-copy still pays for.

19.29.5 Exercise 04

Exercise Title: Use PyTorch out= Variants Where Supported and Verify That Existing Storage Is Reused Without Changing Numerical Results

Part 1 - Exercise Contract and Why It Matters

Use supported PyTorch output variants to write directly into existing storage. Build a support table covering a pointwise operation, matrix multiplication, and deliberately rejected configurations. Verify values and destination storage without claiming that all operator workspace allocation disappears.

Part 2 - System and Memory Mental Model

An output argument is operation-specific. Cases needing gradients are not supported by the documented out semantics. Use correctly shaped, compatible, disjoint destinations in the initial inference experiment. Do not depend on resizing a nonempty incorrectly shaped output. Layout, dtype promotion, and input/output overlap must satisfy the selected operator's contract.

Part 3 - Quantitative Prediction Before Execution

For output shape (m, n) and element width s, predict mns payload bytes. For matrix multiplication, separately account for both inputs and measured workspace. Predict acceptance for a matching inference destination, rejection or unsupported status for a gradient-required case, and the diagnostic outcome for a wrong-shape destination. Write expected and observed statuses separately.

Part 4 - Experimental Design and Instrumentation

Read the actual installed-version operator signatures. Create independent fresh references and preallocated outputs. Compare values with declared precision and tolerances, then record shape, strides, storage base, and observed requests. Change dtype, layout, shape, overlap, and gradient requirements one at a time. Preserve rejection reasons and warnings instead of silently casting, resizing, or disabling differentiation.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the code is deliberately incomplete and is not a finished implementation.

```
from dataclasses import dataclass
import torch

@dataclass
class OutCase:
    operator: str
    shape: tuple
    dtype: object
    requires_grad: bool
    expected_status: str


def admit_out_case(case, inputs, destination):
    # Check the selected operator's actual destination contract.
    ...


def execute_out_case(case, inputs, destination):
    # Complete only supported direct-output operations.
    ...


def E04(config):
    # Build the support table; validate values and storage evidence.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Acceptance requires correct results, the promised output-storage behavior, and an unchanged gradient contract. Input/output aliasing is not universally legal just because a destination argument exists. A supported noncontiguous output needs its own test. Internal allocation observed during a valid out call must be attributed rather than misreported as a fresh output allocation.

Part 7 - Engineering Interpretation and Philosophy

A measured allowlist is stronger than assuming every operation can write into arbitrary scratch. Keep unsupported behavior explicit so the eventual planner can reject it or use a charged fallback without changing program semantics.

19.29.6 Exercise 05

Exercise Title: Compare Fresh Allocation, out= Reuse, and copy_() Reuse by Allocation Count, Peak VRAM, Reserved Memory, and Latency

Part 1 - Exercise Contract and Why It Matters

Benchmark fresh result allocation, direct output reuse, and compute-then-copy reuse under equivalent useful work. Report allocation count, peak allocated VRAM, reserved memory, and latency separately. Preserve source-result costs in the copy variant and validate correctness before ranking performance.

Part 2 - System and Memory Mental Model

Warm caching can satisfy many requests without driver allocation. Persistent buffers retain capacity even when idle, and compute-then-copy can keep a fresh result and destination live together. These effects can move different metrics in opposite directions. Match output leases, arithmetic, input residency, and completion boundaries across all three strategies.

Part 3 - Quantitative Prediction Before Execution

At one named counting layer, define request reduction $\rho_A = 1 - A_r/A_b$ for $A_b > 0$ and speedup $S = T_b/T_r$ for $T_r > 0$. Keep undefined ratios undefined. For a b-byte temporary copied into a persistent destination, predict an additional b live bytes while both coexist. Construct a prediction where request count improves but latency or retained capacity does not.

Part 4 - Experimental Design and Instrumentation

Use small and larger fixed shapes with deterministic inputs. Measure cold setup and warmed execution separately. Capture short allocation traces in separate diagnostic replays, then run repeated unprofiled timing trials in randomized strategy order. Synchronize at complete trial boundaries, record absolute peaks and end-state capacity, and keep raw latency samples. If outputs must be retained, charge all strategies for the same retention guarantee.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the code is deliberately incomplete and is not a finished implementation.

```
from dataclasses import dataclass
import torch

@dataclass
class ReuseResult:
    strategy: str
    request_layer: str
    request_count: object
    peak_allocated_bytes: object
    peak_reserved_bytes: object
    elapsed_seconds: object
    numerical_error: object


def run_strategy(strategy, inputs, contract, phase):
    # Implement equivalent fresh, direct-output, and copy variants.
    ...


def compare_strategies(results, raw_samples):
    # Check equivalence before comparing counts, bytes, and timings.
```

```
    ...


def E05(config):
    # Separate profiling from timing and retain adverse results.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

A mismatched lease, omitted source allocation, or incomplete CUDA timing invalidates the ranking. Distinct pointer counts are not allocation counts. Report median and a declared tail statistic with trial counts, plus setup cost and owned capacity. Explain unchanged reserved memory without treating it as proof that allocation reduction failed.

Part 7 - Engineering Interpretation and Philosophy

The winning strategy depends on the measured objective and caller contract. Choose among latency, request reduction, and capacity predictability explicitly; a single favorable counter does not establish a universally better execution.

19.29.7 Exercise 06

Exercise Title: Build a Shape-Compatible Scratch Buffer and Reuse It Across Repeated Matrix Operations

Part 1 - Exercise Contract and Why It Matters

Build reusable scratch for a repeated matrix chain while preserving every matrix input until its final read. Deliver a dimension-checked slot layout, correct intermediate values, and a comparison with fresh intermediate results. Determine the required number of simultaneous destinations before allocating.

Part 2 - System and Memory Mental Model

For $Y = AB$ and $Z = YC$, producing Z reads Y. Do not overwrite Y with Z unless the selected operation explicitly permits that overlap. A dedicated scratch slot plus a separate final output is a safe initial design. Capacity alone does not establish compatibility: shape, dtype, device, layout, aliasing, and completion also matter. Begin with one-stream inference.

Part 3 - Quantitative Prediction Before Execution

For $A : (m, k)$, $B : (k, n)$, $C : (n, p)$ and element width s, calculate $b_Y = mns$ and $b_Z = mps$. Under a disjoint-output contract, account for both during the second multiplication. Extend the chain and compare exact-sized slots with two maximum-capacity alternating slots. Predict retained slack as well as slot count, keeping library workspace separate.

Part 4 - Experimental Design and Instrumentation

Draw read/write sets and dimensions for each operation. Allocate deterministic inputs, build a fresh reference, and complete a supported direct-destination chain. Validate each stage in a diagnostic replay, then benchmark without those extra captures. Test incompatible dimensions, a retained intermediate, and a noncontiguous case with explicit support or rejection. Record matrix precision settings and use justified numerical tolerances.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the code is deliberately incomplete and is not a finished implementation.

```
from dataclasses import dataclass
import torch

@dataclass
class MatrixScratch:
    slot_id: int
    shape: tuple
    capacity_bytes: int
    dtype: object
    device: object


def validate_matrix_destination(left, right, output, spec):
    # Check dimensions, capacity, layout, and unsupported overlap.
    ...


def matrix_chain(inputs, scratch_slots, final_output):
    # Assign destinations without overwriting still-readable inputs.
    ...


def E06(config):
    # Compare stage values, allocation requests, and complete runtime.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

A small correct final output does not excuse an invalid overlap schedule. Verify intermediate values and reject the retained-input conflict. Report visible scratch requests separately from operator workspace. Do not hide validation copies in only one strategy's timing or present inference output reuse as a gradient-preserving training implementation.

Part 7 - Engineering Interpretation and Philosophy

Scratch compatibility combines geometry with time. The smallest useful pool preserves the actual matrix dependencies; a larger generic pool is justified only when its flexibility compensates for extra retained capacity.

19.29.8 Exercise 07

Exercise Title: Resize a Reusable Tensor Across Variable Input Shapes and Determine When PyTorch Can Preserve Existing Storage

Part 1 - Exercise Contract and Why It Matters

Investigate when a dedicated reusable tensor can change logical shape without growing its backing storage. Record shape, strides, capacity, and pointer observations across shrink, within-capacity growth, and beyond-capacity growth. Treat this as scratch management rather than semantic reshaping of old values.

Part 2 - System and Memory Mental Model

The low-level resize operation retains backing storage when fewer elements are required and grows storage when capacity is insufficient. It can reinterpret strides as contiguous. It is not a general replacement for view or reshape. Start with owned, offset-zero contiguous scratch, no escaped aliases, and no gradient recording. Initialize every region before reading it; newly exposed bytes are not valid old tensor contents.

Part 3 - Quantitative Prediction Before Execution

For shape h_t and element width s, calculate logical demand $b_t = s \prod_d h_{t,d}$ and compare it with observed storage bytes. Predict transitions for element counts $1024, 256, 768, 2048, 512$. Identify which requests fit current measured capacity without assuming allocator rounding. Add equal numel shapes with different dimensions and explain why storage persistence does not preserve an intended matrix arrangement.

Part 4 - Experimental Design and Instrumentation

Capture old shape, strides, storage base, tensor pointer, storage offset, and capacity before each transition. Admit resize only after old borrowers are released. Resize, initialize the active region, and validate a deterministic operation against a fresh reference. Record growth and initialization time separately. Include an outstanding-view case that the admission policy rejects before mutation.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the code is deliberately incomplete and is not a finished implementation.

```
from dataclasses import dataclass
import torch

@dataclass
class ResizeRecord:
    before_shape: tuple
    after_shape: tuple
    before_capacity: int
    after_capacity: int
    before_base: int
    after_base: int


def resize_owned_scratch(buffer, shape, ownership):
    # Reject aliases/borrowers and unsupported resize conditions.
    ...
    # Resize, record evidence, and initialize the entire active view.
    ...
```

```
def E07(config):
    # Predict transitions, then compare measured capacity behavior.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Never validate newly exposed elements by reading uninitialized memory. Empty tensors need separate handling because pointer values are not useful allocation witnesses. A stable address alone does not establish continuous identity after a growth transition. Report stale-view rejection, capacity changes, and unsupported storage contracts explicitly.

Part 7 - Engineering Interpretation and Philosophy

Logical size and retained capacity are different controls. Use the experiment to define when shape changes are cheap and when a new storage generation is required, then make those distinctions visible in the buffer interface.

19.29.9 Exercise 08

Exercise Title: Build a Capacity-Based Reusable Buffer That Grows Only When Required and Never Shrinks During Stable Workloads

Part 1 - Exercise Contract and Why It Matters

Build a typed capacity buffer that serves contiguous shaped prefix views, grows only when required, and never shrinks during the stable phase. Implement bounded admission, generation-aware borrowing, and a clear policy for growth while old users remain active.

Part 2 - System and Memory Mental Model

The active view can be smaller than its owned base without returning unused capacity. Growing must not silently retarget escaped views to new storage. Either reject or defer growth until release, or retain old generations until their readers finish. Charge both generations when they coexist. Use one dtype and device per pool and exclude unsupported strides or raw byte reinterpretation.

Part 3 - Quantitative Prediction Before Execution

For request r_t elements, capacity C_t, and growth factor $g > 1$, consider

$$C_{t+1} = \begin{cases} C_t, & r_t \le C_t, \\ \max(r_t, \lceil gC_t \rceil), & r_t > C_t. \end{cases}$$

Compare exact growth and $g = 2$ on $3, 5, 4, 9, 8$ elements. Calculate growth count, retained slack, and old-plus-new transition bytes. Handle zero initial capacity, empty requests, oversized shapes, and budget limits explicitly.

Part 4 - Experimental Design and Instrumentation

Simulate the capacity policy before allocating CUDA storage. Complete acquire and release with generation checks, then compare against fresh allocation on a fixed shape trace. Validate every active view against independent values. Test an outlier request, an outstanding borrower,

and a controlled growth refusal. Ensure failure leaves the previous usable state coherent and retries bounded.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the code is deliberately incomplete and is not a finished implementation.

```
from dataclasses import dataclass
import torch

@dataclass
class CapacityLease:
    generation: int
    shape: tuple
    release_token: object

class CapacityBuffer:
    def __init__(self, dtype, device, budget_bytes, growth_factor):
        self.dtype = dtype
        self.device = device
        self.budget_bytes = budget_bytes
        self.growth_factor = growth_factor
        # Initialize capacity, owner, generation, and borrower state.
        ...

    def acquire(self, shape):
        # Admit shape, grow safely if needed, and return a leased view.
        ...

    def release(self, lease):
        # Validate generation and completion before allowing overwrite.
        ...


def E08(config):
    # Compare growth events, transition peaks, slack, and correctness.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Assert monotone stable-phase capacity and sufficient capacity for every accepted view. Reject stale releases and unsafe growth with live borrowers. Distinguish budget refusal from allocator OOM. A final pool that fits the budget can still require excessive transition memory; report that peak rather than only end-state slack.

Part 7 - Engineering Interpretation and Philosophy

Growth policy trades allocation frequency for retained capacity. Choose it from measured shape demand and available headroom, and require an explicit phase boundary before any later shrink or pool reset.

19.29.10 Exercise 09

Exercise Title: Track Tensor data_ptr() Values Across Iterations and Prove When Storage Is Actually Reused Versus Reallocated

Part 1 - Exercise Contract and Why It Matters

Track tensor addresses together with storage bases, offsets, capacities, owner generations, and allocation events. Distinguish persistent owned reuse, views, recycled addresses after release, and replacement after growth. State what the combined evidence can actually establish.

Part 2 - System and Memory Mental Model

A tensor pointer identifies its first element; an offset view can have another pointer while sharing the same base storage. Conversely, a later fresh tensor can receive an old numerical address after the previous owner is released. Pointer equality is not a proof of continuous ownership. Keep a live owner for the persistent case and use the planner's explicit generation ledger.

Part 3 - Quantitative Prediction Before Execution

For supported ordinary dense tensors, test $p_{\text{tensor}} = p_{\text{storage}} + o\,s$, where o is offset in elements and s is element width. Predict four cases: persistent owned storage, an offset view, a released allocation followed by a new one, and growth. Identify which claims need allocation history and which follow from a continuously retained owner. Unique address count is descriptive, not an exact allocation census.

Part 4 - Experimental Design and Instrumentation

Record scalar metadata without retaining tensors in the evidence log. Run each identity case separately and correlate observations with E01's supported event layer. Validate generations before overwriting. Include an address-only classifier and a counterexample to its inference. Mark missing event history, empty tensors, and unsupported storage abstractions as inconclusive.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the code is deliberately incomplete and is not a finished implementation.

```
from dataclasses import dataclass
import torch

@dataclass
class StorageObservation:
    owner: str
    generation: int
    tensor_pointer: int
    storage_pointer: int
    storage_offset: int
    element_bytes: int
    capacity_bytes: int
```

```
def observe_storage(tensor, owner, generation):
    # Capture public metadata without extending the tensor's lifetime.
    ...


def classify_reuse(before, after, ownership, allocation_events):
    # Separate sharing, persistence, address recycling, and unknowns.
    ...


def E09(config):
    # Test the pointer-only counterexamples and stronger evidence.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require correct classification of nonzero-offset views and refusal to call a recycled address continuous ownership. Evidence timestamps must cover the claimed interval. Private object IDs are not portable allocation IDs. If an instrument cannot exclude an intervening release/reallocation, preserve that uncertainty instead of labeling the observation proven reuse.

Part 7 - Engineering Interpretation and Philosophy

A pointer is evidence about location, not a complete identity history. The planner can make stronger claims about its continuously owned slots than about arbitrary tensors observed only through intermittent snapshots.

19.29.11 Exercise 10

Exercise Title: Reuse Intermediate Buffers Across Multiple Layers Without Accidentally Overwriting Values Still Required by Later Operations

Part 1 - Exercise Contract and Why It Matters

Reuse intermediate buffers across a multilayer inference graph containing a skip connection. Build read/write sets and reject any assignment that overwrites a value still required by another operation or an external consumer.

Part 2 - System and Memory Mental Model

Layer order does not imply simple last use. A residual path can retain an early activation through several later layers. Views extend the lifetime of the same storage. Start with one stream, a fixed graph, and no gradient recording; backward adds consumers that an inference-only lifetime table does not describe. Unsupported in-place overlap is forbidden even if one test happens to work.

Part 3 - Quantitative Prediction Before Execution

For $h_1 = f(x)$, $h_2 = g(h_1)$, and $y = h_2 + h_1$, predict why writing h_2 over h_1 destroys the residual input. Under disjoint destinations, compute the live bytes during each operation. Add another branch retaining a view of h_1 and update its final-use boundary. Distinguish payload demand from retained slot capacity.

Part 4 - Experimental Design and Instrumentation

Create a fixed residual graph with deterministic inputs. Enumerate producers, all readers, aliases, and external outputs. Validate a legal slot plan and an intentionally invalid plan in host metadata; execute only the admitted CUDA plan. Compare final and selected intermediate values against independent fresh execution. Repeat with multiple sentinel patterns so zero inputs do not hide premature overwrites.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the code is deliberately incomplete and is not a finished implementation.

```
from dataclasses import dataclass
import torch

@dataclass
class LayerAccess:
    operation_id: int
    reads: tuple
    writes: tuple
    allowed_aliases: tuple


def validate_layer_plan(operations, assignment, alias_groups):
    # Reject future-reader overwrites and unsupported same-op overlap.
    ...


def execute_layers(inputs, operations, slots, plan):
    # Use only admitted destinations and preserve output leases.
    ...


def E10(config):
    # Compare the safe plan; reject the invalid residual assignment.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Validation must reject the bad plan independently of its sampled numerical result. Check every required intermediate before its slot is reassigned. A retained alias must extend protection. Do not treat deleting a local variable as release proof or silently switch off autograd to accept a training graph.

Part 7 - Engineering Interpretation and Philosophy

The graph's consumers determine legal reuse. An extra buffer is justified when it preserves a required residual or output; fewer buffers are not an improvement if they change the computation's observable values.

19.29.12 Exercise 11

Exercise Title: Analyze Tensor Lifetimes and Reuse the Same Storage for Non-Overlapping Intermediate Activations

Part 1 - Exercise Contract and Why It Matters

Derive activation lifetimes from a fixed execution schedule and assign the same storage to compatible values whose lifetimes do not overlap. Deliver a producer, consumer, alias, and release table with an explicit operation-boundary convention.

Part 2 - System and Memory Mental Model

A lifetime begins when storage is required for a value and ends only after its last required read. Treat an operation's inputs and disjoint output as live together during that operation. Aliases protect their shared region until the last relevant use; escaped outputs remain live until caller release. A linear schedule initially models one stream, not arbitrary asynchronous execution.

Part 3 - Quantitative Prediction Before Execution

Use half-open intervals $[b_i, e_i)$ with e_i defined after the last consuming operation. Temporal overlap occurs when $\max(b_i, b_j) < \min(e_i, e_j)$. Calculate a legal slot assignment for intervals $[0, 3), [1, 2), [3, 5), [2, 4)$ with sizes $8, 4, 8, 4$ MiB. Extend the first interval to 5 for an escaped alias and recalculate. Equal endpoint numbers must not excuse unsupported input/output overlap within one indivisible operation.

Part 4 - Experimental Design and Instrumentation

Select a small DAG and freeze its topological execution order. Derive lifetimes from actual read/write sets, union alias protection, and construct a live-byte timeline. Simulate sharing before materializing slots. Compare admitted CUDA execution with a fresh reference. Include fork/join dependencies, a retained output, and one uncertain lifetime that is conservatively extended.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the code is deliberately incomplete and is not a finished implementation.

```
from dataclasses import dataclass
import torch

@dataclass
class ValueInterval:
    key: str
    birth: int
    release_after: int
    payload_bytes: int
    alias_group: str


def derive_intervals(graph, schedule, external_releases):
    # Account for all consumers, aliases, and operation boundaries.
```

```
    ...


def assign_nonoverlapping_values(intervals, compatibility):
    # Respect both lifetimes and slot capacities.
    ...


def E11(config):
    # Check intervals in a host model before executing shared storage.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Audit all pairs sharing a slot, not only adjacent entries. Compare predicted live bytes and owned slot capacity without assuming equality. Reject a plan made unsafe by alias lifetime extension. A last host-use index is not a multi-stream completion proof, and an inference lifetime table does not account for tensors saved for backward.

Part 7 - Engineering Interpretation and Philosophy

A lifetime table is a claim about when information stops being needed. Its usefulness depends on honest boundaries and complete consumers, not on how aggressively it permits storage sharing.

19.29.13 Exercise 12

Exercise Title: Build a Simple Lifetime-Based Buffer Planner That Assigns Multiple Temporary Tensors to Shared Storage Slots

Part 1 - Exercise Contract and Why It Matters

Build a small planner assigning temporary values to owned storage slots using the validated lifetime table. Support a fixed, single-stream inference graph and an explicit operation allowlist. Materialize the plan only after checking its capacity, alias, and output-ownership invariants.

Part 2 - System and Memory Mental Model

Plan whole typed slots: at most one live value occupies a slot, with contiguous views for compatible shapes. Separate dtype and device pools. The application selects its own slots; it cannot select arbitrary cached allocator blocks by address. Unsupported destination-writing operations require a measured fallback or rejection. Returned views need leases or independently owned copies.

Part 3 - Quantitative Prediction Before Execution

For assignment $s(i)$ and request bytes b_i, require $C_{s(i)} \geq b_i$. For an exact static whole-slot plan, compute $C_j = \max_{i:s(i)=j} b_i$ and $B_{\text{owned}} = \sum_j C_j$. Compare this capacity with no-sharing bytes and the maximum live-payload lower bound. Add a retained output and predict which assignment becomes illegal.

Part 4 - Experimental Design and Instrumentation

Implement a host-only request and slot model first. Validate complete coverage, non-overlap, compatibility, and capacities independently of the assignment routine. Allocate slot bases once and execute a small supported graph. Compare fresh and shared execution with matching output contracts. Test an unsupported operator, oversized request, and stale output lease. Charge setup, fallbacks, and retained capacity in the benchmark.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the code is deliberately incomplete and is not a finished implementation.

```
from dataclasses import dataclass
import torch

@dataclass
class TemporaryRequest:
    key: str
    shape: tuple
    dtype: object
    device: object
    birth: int
    release_after: int

class BufferPlanner:
    def __init__(self, budget_bytes, supported_operators):
        self.budget_bytes = budget_bytes
        self.supported_operators = tuple(supported_operators)
        # Initialize slot specifications and ownership records.
        ...

    def plan(self, requests, aliases):
        # Assign compatible values to non-interfering whole slots.
        ...

    def validate(self, plan):
        # Independently check coverage, capacity, and lifetime rules.
        ...

    def execute(self, graph, inputs, plan):
        # Materialize owned slots and preserve supported semantics.
        ...


def E12(config):
    # Compare the legal plan against equivalent fresh execution.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require every accepted request to have one valid assignment. Count shared views once in owned capacity and reject unsupported dtype reinterpretation or strides. Reconcile pool bytes with the ownership ledger, then report allocator peaks separately. Reduced pool capacity does not guarantee equal driver-visible savings, and a fallback allocation must not disappear from the census.

Part 7 - Engineering Interpretation and Philosophy

A constrained planner is useful when its support and accounting are precise. Its promise is safe sharing of application-owned storage, not control over every allocation performed by PyTorch or its libraries.

19.29.14 Exercise 13

Exercise Title: Construct an Interference Graph from Tensor Lifetimes and Prevent Buffers with Overlapping Lifetimes from Sharing Storage

Part 1 - Exercise Contract and Why It Matters

Construct an interference graph from alias-aware tensor lifetimes. Retain an explanation for every temporal conflict and prevent conflicting values from sharing a whole slot. Distinguish lifetime conflicts from dtype, device, or layout incompatibility.

Part 2 - System and Memory Mental Model

A temporal edge means two values require protected storage during overlapping intervals. Aliases need union protection or explicit shared-region modeling; they are not independent values that can be freely colored. A fixed one-stream schedule gives interval order. Concurrent execution later requires a happens-before relation, not merely host submission indices.

Part 3 - Quantitative Prediction Before Execution

Define $(i, j) \in E$ when $i \neq j$ and $I_i \cap I_j \neq \varnothing$ under E11's endpoint convention. Build the graph for $[0, 4), [1, 2), [2, 3), [4, 6)$ and identify touching endpoints. For interchangeable equal-capacity slots, maximum concurrent live count is a lower bound. Appropriate interval-coloring algorithms can attain it for interval graphs; arbitrary greedy orders and weighted capacity objectives need separate analysis.

Part 4 - Experimental Design and Instrumentation

Create hand-checkable disjoint, chain, clique, fork/join, and escaped-alias examples. Generate edges with overlap witnesses and record compatibility partitions separately. Compare graph construction with an independent pairwise host oracle. Validate proposed slot assignments against every edge and capacity constraint before CUDA execution. Verify actual read/write boundaries agree with the interval model.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the code is deliberately incomplete and is not a finished implementation.

```
from dataclasses import dataclass
import torch

@dataclass
```

```
class Interference:
    first: str
    second: str
    reason: str
    overlap_witness: tuple


def build_interference(intervals, alias_groups, compatibility):
    # Generate symmetric conflicts with explicit supporting facts.
    ...


def validate_assignment(nodes, conflicts, assignment, capacities):
    # Reject shared slots across edges and incompatible contracts.
    ...


def E13(config):
    # Check hand examples and the independent graph oracle first.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Check symmetry, missing overlapping pairs, unintended self-edges, and endpoint treatment. A graph-valid assignment can still fail capacity or operator overlap rules. Extend an escaped alias and require rejection of the now-invalid plan. Do not call an inference graph a complete model of backward or stream completion.

Part 7 - Engineering Interpretation and Philosophy

An interference edge should explain the exact value or contract preventing sharing. Preserve that explanation independently of the heuristic so a rejected assignment remains understandable when allocation policy changes.

19.29.15 Exercise 14

Exercise Title: Implement a Greedy Buffer Assignment Algorithm That Minimizes the Number of CUDA Storage Blocks Required by a Fixed Computation Graph

Part 1 - Exercise Contract and Why It Matters

Implement a deterministic greedy assignment aiming to reduce application-owned storage slots for a fixed graph. Report retained bytes as a separate objective. Compare legal plans with a bounded exact oracle on small instances; the title states an optimization goal, not a universal optimality guarantee.

Part 2 - System and Memory Mental Model

A candidate slot must have compatible dtype, device, layout, capacity, and non-interfering lifetimes. Request order and tie-breaking affect a heuristic's result. Keep the execution schedule fixed during the primary comparison. Application slots do not necessarily correspond one-to-one with allocator blocks or CUDA driver allocation calls.

Part 3 - Quantitative Prediction Before Execution

Let K be slot count and $B = \sum_j C_j$ retained capacity. Compare a slots-first objective (K, B) with a bytes-first objective. Construct variable-size requests where these preferences differ. For tiny cases with exact optimum K^*, report $K_{\text{greedy}} - K^*$ and capacity gap under the same model. An oracle timeout is not an optimality certificate.

Part 4 - Experimental Design and Instrumentation

Implement first-fit and one alternative with explicit deterministic ordering. Validate both using a checker independent of their selection logic. Enumerate legal tiny assignments with a declared size/time limit. Test multiple orders, equal-size interval cases, and adversarial variable-size cases. Materialize only valid plans and measure setup cost, requests, retained capacity, and runtime. Keep hidden workspaces outside slot count but inside complete memory evidence.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the code is deliberately incomplete and is not a finished implementation.

```
from dataclasses import dataclass
import torch

@dataclass
class PlanQuality:
    heuristic: str
    ordering: str
    slot_count: int
    retained_bytes: int
    oracle_status: str


def greedy_assignment(requests, conflicts, ordering, tie_break):
    # Select only compatible slots with no protected live conflict.
    ...


def exact_tiny_assignment(requests, conflicts, limits):
    # Search bounded cases and distinguish optimum from timeout.
    ...


def E14(config):
    # Validate plans before comparing objective values and hardware cost.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require legality before quality comparisons. Preserve worst observed gaps and adverse orderings, not only favorable examples. Matching an oracle on tested instances is not proof for every graph. Equal-size interval-coloring guarantees apply only when their ordering, compatibility, and objective assumptions hold.

Part 7 - Engineering Interpretation and Philosophy

A heuristic is trustworthy when its choices are reproducible, its legality is independently checked, and its quality limits are stated. Use measured workload needs to decide whether fewer slots or fewer retained bytes matters more.

19.29.16 Exercise 15

Exercise Title: Reuse Pinned CPU Staging Buffers Across Repeated Host-to-Device and Device-to-Host Transfers

Part 1 - Exercise Contract and Why It Matters

Reuse a bounded pinned-CPU staging pool for repeated H2D and D2H transfers. Define which event releases each host and device buffer, verify every payload, and compare complete transfer cycles with fresh staging allocation.

Part 2 - System and Memory Mental Model

Pinned sources must not be overwritten during in-flight H2D transfers, and D2H destinations must not be read or recycled before completion. Persistent pinning avoids repeated setup but retains page-locked host capacity. Nonblocking submission is not completion, and useful overlap depends on hardware, streams, dependencies, and workload. Host and device buffers can have different release boundaries.

Part 3 - Quantitative Prediction Before Execution

For K slots with separate host input and output sizes b_{in} and b_{out}, predict $B_{\text{pinned}} = K(b_{\text{in}} + b_{\text{out}})$. Count device staging separately. Compare a serialized fill/H2D/compute/D2H/consume time with a pipelined model bounded by its slowest effective stage and shared resource contention. Predict a two-slot ring's capacity cost and the evidence needed to establish overlap.

Part 4 - Experimental Design and Instrumentation

Build a serialized reference first. Complete persistent staging acquisition, asynchronous transfers, event waits, and host consumption, then add a two-slot ring. Tag each batch with a generation-specific sentinel. Apply backpressure when no slot is safely free. Measure setup-inclusive and warmed throughput, pinned bytes, device peaks, and transfer waits. Drain all outstanding work at shutdown.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the code is deliberately incomplete and is not a finished implementation.

```
from dataclasses import dataclass
import torch

@dataclass
class StagingLease:
    slot_id: int
    generation: int
    state: str
    h2d_done: object
```

```
    d2h_done: object

class PinnedStagingPool:
    def __init__(self, slots, host_budget, device_budget):
        # Create bounded owned buffers and per-direction states.
        ...

    def acquire_for_fill(self, shape):
        # Return only a host source safe for CPU writes.
        ...

    def transfer_and_compute(self, lease, streams):
        # Establish H2D, compute, and D2H dependencies.
        ...

    def consume(self, lease):
        # Confirm completion before CPU reads or recycling.
        ...


def E15(config):
    # Validate batches and compare serialized/pipelined reuse.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Use a host state-machine test to reject premature reads/writes without relying on a hardware race. Validate payloads only after readiness. Distinguish pinned allocation failure from CUDA OOM. Report observed overlap rather than inferring it from nonblocking flags, and never release an in-flight generation after an unrelated batch failure.

Part 7 - Engineering Interpretation and Philosophy

Pinned staging is a scheduling resource with two budgets: locked host memory and device capacity. Use the smallest measured ring that sustains throughput and explain each resource's actual release condition.

19.29.17 Exercise 16

Exercise Title: Make Reusable Buffers Safe Across Multiple CUDA Streams Using Events, Stream Dependencies, and Explicit Lifetime Tracking

Part 1 - Exercise Contract and Why It Matters

Make reusable slots safe across producer and consumer CUDA streams. Record write readiness, all reader completions, and next-write dependencies. Compare a validated concurrent design with equivalent serialized execution.

Part 2 - System and Memory Mental Model

A producer finishing does not release storage still read by another stream. Execution dependencies and allocator lifetime protection are separate duties. Recording stream use helps

prevent allocator recycling after release; it does not order an application's overwrite of a still-live persistent buffer. Retain owners or use appropriate allocator lifetime recording, and independently establish every required read/write dependency.

Part 3 - Quantitative Prediction Before Execution

For generation g, require $W_g \prec R_{g,1}$ and $W_g \prec R_{g,2}$, then $R_{g,1}, R_{g,2} \prec W_{g+1}$. Here $\prec$ denotes enforced happens-before. Draw a fast and slow reader and predict the earliest legal next write. Compare one-slot serialization with two-slot overlap, including capacity and event cost. A stream wait can order future work without blocking the host; state whether slot release means physical completion or safely ordered future reuse.

Part 4 - Experimental Design and Instrumentation

Model dependencies on the host first and reject missing producer readiness, missing final-reader joins, and stale generation tokens. Complete a bounded persistent CUDA pool using generation-specific events. Validate deterministic payloads against a serialized reference with multiple seeds. Measure overlap without placing a global synchronization inside every iteration. Test delayed readers and safe shutdown without executing deliberately invalid CUDA plans.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the code is deliberately incomplete and is not a finished implementation.

```
from dataclasses import dataclass
import torch

@dataclass
class StreamLease:
    slot_id: int
    generation: int
    ready_event: object
    reader_events: tuple

class StreamSafeBuffers:
    def acquire_for_write(self, shape, writer_stream):
        # Order this writer after every previous reader of the slot.
        ...

    def publish(self, lease, producer_stream):
        # Record readiness for this exact generation.
        ...

    def begin_read(self, lease, reader_stream):
        # Wait for readiness and maintain storage lifetime protection.
        ...

    def end_read(self, lease, reader_stream):
        # Register completion so future writers cannot race.
        ...
```

```
def E16(config):
    # Check dependencies, then validate concurrent values and timing.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Numerical success alone does not establish race freedom: a missing dependency may not fail on the sampled run. Require a ledger for each readiness and read-next-write edge. Stale events must not authorize another generation. Report host/event overhead, serialization, and allocator requests independently.

Part 7 - Engineering Interpretation and Philosophy

With multiple streams, lifetime becomes a dependency relation. A sound design separates who owns storage, when values become readable, when they may be overwritten, and when the allocator may reclaim their allocation.

19.29.18 Exercise 17

Exercise Title: Compare Buffer Reuse Against Activation Checkpointing for a Temporary-Memory-Heavy Workload and Explain Which Problem Each Technique Solves

Part 1 - Exercise Contract and Why It Matters

Compare legal buffer reuse with activation checkpointing for identical training work. Identify temporary scratch and saved activations separately, validate outputs and gradients, and measure complete forward/backward/update iterations. Compare combined behavior only when both mechanisms preserve the same semantics.

Part 2 - System and Memory Mental Model

Reuse shares storage after a value is no longer required; checkpointing avoids retaining selected forward intermediates and reconstructs them during backward. A saved activation cannot simply be overwritten because forward has passed it. Do not use inference-only out variants on differentiable paths. For an initial legal comparison, pair an ordinary differentiable model with nondifferentiable auxiliary scratch executed equally in every variant; label that limited reuse scope. Keep checkpoint recomputation independent of mutable scratch state.

Part 3 - Quantitative Prediction Before Execution

Draw the complete training lifetime timeline including parameters, gradients, optimizer state, saved activations, scratch, and workspace. For strategy m, compare $\Delta P_m = P_{\text{base}} - P_m$ and $\Delta T_m = T_m - T_{\text{base}}$. Do not add category peaks from different times. Predict an activation-dominated case and a scratch-heavy case. Explain why combined peak savings need not equal the sum of separate savings.

Part 4 - Experimental Design and Instrumentation

Clone model and optimizer state and pre-generate matching inputs. Compare baseline, legal scratch reuse, checkpointing, and their supported combination. Specify checkpoint implementation explicitly and preserve RNG/recomputation semantics. Validate loss, outputs,

gradients, updates, and multiple iterations before timing. Include auxiliary work, recomputation, and optimizer update in all matching regions. Vary saved-activation and scratch demand separately.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the code is deliberately incomplete and is not a finished implementation.

```
from dataclasses import dataclass
import torch
from torch.utils.checkpoint import checkpoint

@dataclass
class TrainingMode:
    checkpoint_enabled: bool
    scratch_reuse_enabled: bool
    reuse_scope: str


def differentiable_block(inputs, parameters):
    # Implement a replay-consistent function without stale scratch.
    ...


def auxiliary_scratch_work(inputs, pool, reuse_enabled):
    # Keep identical nondifferentiable work in every strategy.
    ...


def complete_training_step(model, optimizer, batch, mode, pool):
    # Preserve gradients and charge forward, backward, and update.
    ...


def E17(config):
    # Restore equal state and compare validated supported strategies.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Fail changed gradient participation, omitted updates, or state-dependent recomputation. Record dtype, tolerances, precision, RNG policy, and optimizer initialization boundaries. Auxiliary reuse is not evidence of safe shared saved activations. If broader differentiable reuse is unsupported, state that rather than disabling gradients or dropping required work.

Part 7 - Engineering Interpretation and Philosophy

Checkpointing spends compute to recover retained information; reuse controls storage for compatible lifetimes. Choose by the measured bottleneck and preserve the training transition before crediting any memory or throughput benefit.

19.29.19 Exercise 18

Exercise Title: Stress-Test a Reuse Planner with Variable Shapes, Dynamic Execution Paths, Shared Buffers, and Asynchronous CUDA Operations

Part 1 - Exercise Contract and Why It Matters

Stress-test the reuse planner with variable shapes, branches, aliases, retained outputs, and asynchronous readers. Deliver replayable traces, complete outcomes, and minimized failures. Test useful progress as well as memory safety.

Part 2 - System and Memory Mental Model

A plan valid for one path is not automatically valid for another. Use validated path-specific plans, conservative lifetimes, or a charged fallback. Growth must respect old generations and transition budgets; release must respect every reader. Generate invalid transitions as host metadata and reject them before CUDA execution rather than deliberately racing storage.

Part 3 - Quantitative Prediction Before Execution

For admitted requests N_a and correct completions N_c, calculate $\eta = N_c/N_a$ when $N_a > 0$. Also report total attempted coverage and useful throughput $W_{\text{correct}}/T_{\text{complete}}$. Predict growth, wait, fallback, or rejection for alternating small/large shapes, a long-lived branch output, a delayed reader, and an over-budget request. Declare outstanding-lease limits and a shutdown deadline.

Part 4 - Experimental Design and Instrumentation

Use fixed replay traces plus seed-controlled randomized host traces. Check capacity, generations, aliases, and dependencies independently. Execute only admitted CUDA cases and compare each generation with a fresh reference. Test unseen shapes, empty tensors, branch switches, retained views, controlled allocation refusal, and safe drain. Minimize failures while preserving the same path and release sequence; record versions, device, backend, seed, and trace ID.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the code is deliberately incomplete and is not a finished implementation.

```
from dataclasses import dataclass
import torch

@dataclass
class StressRequest:
    request_id: int
    shape: tuple
    branch: str
    retain_output: bool
    readers: tuple


def generate_traces(seed, support, limits):
    # Include valid adversarial cases and host-only invalid cases.
```

```
    ...


def validate_and_execute(trace, planner, reference):
    # Admit safe transitions and classify every request outcome.
    ...


def minimize_failure(trace, replay_predicate, limits):
    # Reduce a reproducible failure without changing its cause.
    ...


def E18(config):
    # Export coverage, progress, resources, and replayable evidence.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report correct completions, justified rejections, fallbacks, numerical failures, typed OOM, timeouts, and unrelated errors separately. A planner that rejects all hard requests has not demonstrated useful optimization. Passing stress tests supports the tested scope, not every possible schedule. Preserve an unreproduced failure honestly and never present simulated metadata results as GPU outcomes.

Part 7 - Engineering Interpretation and Philosophy

A production claim needs both accepted-work correctness and explainable refusals. Use measured coverage, bounded resource use, and small counterexamples to define the planner's supported envelope before expanding it.

19.29.20 Exercise 19

Exercise Title: Build and Benchmark a PyTorch Tensor-Reuse Planner That Minimizes Temporary CUDA Allocations Through Lifetime-Aware Storage Sharing

Part 1 - Exercise Contract and Why It Matters

Integrate lifetime analysis, storage assignment, capacity control, safe buffer execution, and measurement into a PyTorch tensor-reuse planner. Reduce temporary allocation requests while preserving useful computation and output ownership. Deliver a supported-operation contract and benchmark evidence, not an unsupported claim of zero allocations or globally optimal memory.

Part 2 - System and Memory Mental Model

Separate logical values and aliases, lifetime analysis, slot assignment, execution dependencies, and observation. Start with typed contiguous whole slots in fixed single-stream inference. Add dynamic paths and multiple streams only through their validated contracts. Gradient-required graphs need saved-value lifetimes and derivative validation; otherwise reject or use an equivalent charged fallback. The planner controls its owned buffers, not arbitrary cached allocator blocks or every operator workspace.

Part 3 - Quantitative Prediction Before Execution

Minimize $A_{\text{temp}}(\pi)$ at a declared allocation layer subject to correctness, budget, coverage, and progress. Derive no-sharing and legal-plan request/capacity predictions. Report slot count, owned bytes, peak allocated and reserved bytes, end-state capacity, and latency separately. Compare tiny cases with an exact oracle where feasible. Predict when retained capacity or planning overhead makes reuse worse despite fewer requests.

Part 4 - Experimental Design and Instrumentation

Freeze the support matrix and tune on separate calibration traces. Validate host plans independently before allocating slots. Compare fresh allocation, dedicated preallocation, simple scratch, and the integrated planner on equivalent matrix chains, residual graphs, variable shapes, dynamic paths, retained outputs, and multi-stream readers. Separate correctness, diagnostic profiling, and timing replays. Include setup, fallback, growth transitions, and shutdown obligations. Run ablations without changing useful work and preserve adverse outcomes.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the code is deliberately incomplete and is not a finished implementation.

```
from dataclasses import dataclass
import torch

@dataclass
class ReuseContract:
    operators: tuple
    dtypes: tuple
    layouts: tuple
    execution_modes: tuple
    output_lease: str
    budget_bytes: int

class TensorReusePlanner:
    def __init__(self, contract):
        self.contract = contract
        # Initialize bounded slots, generations, and evidence state.
        ...

    def analyze(self, graph, shapes, aliases):
        # Derive complete lifetimes and operator restrictions.
        ...

    def plan(self, analysis):
        # Assign compatible slots using the declared objective.
        ...

    def validate(self, plan):
        # Check coverage, interference, capacity, and ownership.
        ...
```

```
    def execute(self, graph, inputs, plan):
        # Enforce leases/dependencies and account for fallbacks.
        ...

    def release(self, lease, completion_tokens):
        # Prevent reuse until every required consumer is protected.
        ...

    def shutdown(self, deadline):
        # Stop admission, drain work, and report unresolved leases.
        ...


def E19(config):
    # Validate small plans, benchmark equivalent work, and stress-test.
    ...
    # Export measurements and a supported engineering recommendation.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Acceptance requires correct values, legal storage sharing, honest allocation accounting, and complete useful-work timing. Record environment, seeds, graph and trace IDs, shapes, precision, leases, raw request counts, owned capacity, allocated/reserved peaks, numerical errors, latency samples, coverage, fallbacks, and failures. Export CPU-only records without retaining live graphs.

Test an unsupported operator, late reader, outlier shape, retained output, budget refusal, and safe drain. Do not invent hardware results. Greedy success is not universal optimality, and unchanged reserved memory does not refute a measured reduction in allocation requests.

Part 7 - Engineering Interpretation and Philosophy

The planner succeeds when it safely shares owned storage and explains the resulting costs at named accounting layers. Recommend the smallest effective mechanism for the measured workload, and state the evidence needed before expanding operator, shape, stream, or training support.

19.30 DMM28

Chapter Title: In-Place Operations and Storage Aliasing
Specialization: Dynamic Memory Management and CUDA Systems

19.30.1 Exercise 00

Exercise Title: Build a Precise Mental Model of In-Place Operations, Tensor Storage, Views, Aliasing, and Autograd Version Counters

Part 1 - Exercise Contract and Why It Matters

Build a precise lifecycle model connecting tensor metadata, logical values, backing storage, views, mutation, and backward consumers. Explain which value an in-place operation destroys and why retaining the same storage does not necessarily preserve the same computation. Deliver a storage/consumer diagram and predictions for ordinary inference and gradient-recorded execution.

Part 2 - System and Memory Mental Model

An in-place operation writes through an existing tensor's storage. Other aliases can observe that write even when their Python objects differ. A tensor needed by backward can remain semantically live after its last visible forward use. Native autograd mutation bookkeeping can detect a changed saved value, but it does not define the application's intended output contract or establish cross-stream ordering. Treat physical sharing, gradient relationships, and remaining value consumers as separate facts in the initial model.

Part 3 - Quantitative Prediction Before Execution

For a dense nonempty tensor with n elements of width s, predict payload $b = ns$. Compare a fresh materialized result with an admitted in-place write: identify the potential b-byte result request avoided without claiming that peak memory must fall by exactly b. Draw live intervals for the original input, a view, an independent clone, and a backward-saved value. Predict which observers change after mutation and which require the old value.

Part 4 - Experimental Design and Instrumentation

Create deterministic small tensors before large CUDA experiments. Record shape, strides, storage offset, base storage address, tensor address, gradient requirements, and ownership. Compare fresh arithmetic, mutation through a view, and mutation of an independent copy. Keep ordinary inference and training cases separate. Use an independent reference snapshot only in a diagnostic replay; charge any copy required by the optimized algorithm in performance runs. Record device, versions, seed, precision, and execution mode.

Label each returned object as borrowed, owned, or externally retained. The same allocation can contain several logical generations over time. Record generation changes even when its address remains stable, so an old observer cannot be accidentally compared against a newer value.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class MutationContract:
    value_id: str
    alias_group: str
    forward_readers: tuple
    backward_requirement: str
    external_owners: tuple
```

```
def describe_tensor(tensor, owner, generation):
    # Capture metadata and supported storage evidence only.
    ...


def predict_mutation_effects(contract, operation):
    # Identify changed values, protected readers, and unknown facts.
    ...


def E00(config):
    # Compare predicted and observed inference/autograd cases.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Acceptance requires distinguishing storage sharing from graph connectivity. A detached alias is not an independent snapshot, and a clone can preserve a gradient path while owning different storage. Include a case whose forward output changes without any autograd error. Version diagnostics must not be used as permission to overwrite application-visible values. Mark unsupported storage abstractions or unobserved consumers as unknown rather than safe.

Part 7 - Engineering Interpretation and Philosophy

In-place optimization spends the right to destroy an old value in exchange for avoiding another result allocation. The right must come from complete consumer and ownership knowledge. A smaller allocation trace is valuable only after the intended computation and observable histories remain intact.

19.30.2 Exercise 01

Exercise Title: Compare Out-of-Place x + y Against In-Place x.add_(y) and Measure Temporary CUDA Allocation Differences

Part 1 - Exercise Contract and Why It Matters

Compare ordinary addition with an in-place addition on equivalent inputs. Measure the visible output request, numerical result, and input side effects. Specify whether the caller permits input destruction before claiming an optimization. The experiment should establish a scoped allocation difference, not imply that all CUDA allocation or memory traffic disappears.

Part 2 - System and Memory Mental Model

Ordinary addition produces a separate result; in-place addition overwrites the left operand. Begin with same-shape, same-dtype, disjoint contiguous tensors outside gradient recording. Broadcasting and type promotion can make the in-place contract differ, so introduce them only as separate compatibility cases. Restoring a mutated input between trials is real work whose inclusion must be explicit. Allocator cache reuse can hide driver calls while logical result requests still differ.

Part 3 - Quantitative Prediction Before Execution

For n elements of width s, predict a $b = ns$ result payload in the out-of-place case. Construct a live-set diagram with both operands and the result, then remove only the result allocation allowed by the destructive contract. Predict allocation traffic separately from allocated and reserved peaks. For a caller that needs the original left operand afterward, estimate the extra preservation copy and reconsider the expected saving.

Part 4 - Experimental Design and Instrumentation

Pre-generate deterministic inputs and establish one reference addition. Use independent starting state for each strategy. Complete a short allocation census at a named layer, then time unprofiled replays with matching CUDA completion boundaries. Inspect original inputs, result values, and storage metadata. Run both a destructive-ownership case and a preserve-input control. Capture setup separately and avoid clearing the allocator cache inside every iteration just to exaggerate fresh-allocation overhead.

Add a read-only caller test that checks its left input after the function returns. If the contract requires that value unchanged, mutation must be refused or preservation charged. This catches a semantic mismatch that ordinary result comparison alone would overlook.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def fresh_add(left, right):
    # Implement the ordinary result-producing reference.
    ...


def destructive_add(left, right, ownership):
    # Admit compatible mutation only when the old left value is dead.
    ...
    # Perform the in-place addition and preserve required evidence.
    ...


def E01(config):
    # Match initial values, account for resets, and measure requests.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate output equality and the explicitly expected input mutation. Reject comparisons where one strategy includes cloning or initialization while the other does not, unless that difference is required by the caller contract and charged openly. A repeated pointer is not an allocation census. Include a retained alias that makes destructive addition semantically invalid, even though the arithmetic operation itself may execute successfully.

Part 7 - Engineering Interpretation and Philosophy

Addition becomes a memory optimization only when destructive ownership is available. State separately what the operator supports, what the caller allows, and what the measurement shows. If preservation costs cancel the apparent benefit, that is the correct result for that contract.

19.30.3 Exercise 02

Exercise Title: Measure Peak VRAM and Runtime Across Repeated Out-of-Place and In-Place Elementwise Operations

Part 1 - Exercise Contract and Why It Matters

Measure repeated out-of-place and in-place elementwise execution under the same recurrence, initial state, precision, and useful work. Report latency, allocation traffic, and absolute allocated/reserved peaks. Distinguish reducing request frequency from reducing live capacity or improving end-to-end runtime.

Part 2 - System and Memory Mental Model

A recurrence that replaces its current state can often reuse the old state's storage when no observer needs prior generations. A history-producing workload cannot discard those generations without changing its outputs. A warm allocator may already recycle buffers efficiently, while kernel and memory-bandwidth costs remain. In-place execution does not inherently fuse multiple operations or remove their reads and writes. Keep inference benchmarking separate from gradient-recorded recurrent graphs.

Part 3 - Quantitative Prediction Before Execution

For N updates of a b-byte state, estimate cumulative result-request traffic for the materializing recurrence and the persistent-state alternative. Predict peak live bytes from actual evaluation order, not from Nb. Model total time as setup, kernel execution, allocation/dispatch, synchronization, and any required restoration. Construct small and large state predictions where allocation savings may have different significance relative to bandwidth.

Part 4 - Experimental Design and Instrumentation

Fix a numerically bounded recurrence and generate identical initial states. Warm both strategies, validate several intermediate generations in diagnostic runs, and measure repeated unprofiled trials in randomized order. Reset peak counters at stated boundaries without hiding preallocated state. Use CUDA events for the intended device region and completed wall time for total work. State whether setup, input restoration, and output retention are included. Preserve raw timing samples rather than a single fastest trial.

For a recurrence that retains only its final state, ensure neither benchmark appends every intermediate tensor to a Python list. Such logging silently changes the live set and would turn a constant-history workload into a history-retaining experiment.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
```

```
@dataclass
class RepeatedTrial:
    strategy: str
    updates: int
    request_count: object
    peak_allocated_bytes: object
    peak_reserved_bytes: object
    wall_seconds: object
    device_seconds: object


def repeated_update(state, operand, count, strategy, contract):
    # Implement the same recurrence and explicit history policy.
    ...


def measure_replay(strategy, initial_state, config):
    # Separate setup, profiling, validation, and completion timing.
    ...


def E02(config):
    # Compare distributions, memory peaks, and generation values.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Check that both variants execute the same update count and recurrence. Overflow, NaNs, or divergent initial state must not masquerade as equivalence. Report numerical tolerances and finite-value checks. Reserved memory may remain high after request reduction because cached capacity is retained. A loss in latency or no measurable improvement must remain in the evidence rather than being excluded from the workload selection.

Part 7 - Engineering Interpretation and Philosophy

Repeated execution exposes the difference between allocation overhead and useful arithmetic. Recommend mutation from complete workload evidence, including history requirements. Do not translate fewer temporary requests into an unmeasured throughput claim or assume every tensor size has the same bottleneck.

19.30.4 Exercise 03

Exercise Title: Track data_ptr() Before and After In-Place Operations to Prove That Existing Tensor Storage Is Reused

Part 1 - Exercise Contract and Why It Matters

Track storage evidence around admitted native in-place operations and compare it with fresh results, views, and allocator address recycling. The title's proof requires a continuous ownership model, not only pointer equality. Deliver an observation ledger that states precisely which identity claim each measurement supports.

Part 2 - System and Memory Mental Model

The tensor data pointer addresses its first element; the storage base can precede that address when the tensor has a nonzero offset. Keeping the same live owned allocation across a supported in-place arithmetic call provides stronger evidence than observing the same address after release. Views and fresh allocations can defeat address-only classifiers. Restrict the main experiment to ordinary nonempty dense tensors and exclude opaque or copy-on-write storage behavior from the initial support contract.

Part 3 - Quantitative Prediction Before Execution

For supported dense tensors, predict $p_{\text{tensor}} = p_{\text{storage}} + s\,o$, with element width s and storage offset o. Write expected before/after fields for an in-place update, a fresh update, an offset view, and a released-then-reallocated tensor. Explain which cases could have equal numerical addresses without the same ownership history. Predict capacity and shape stability separately from value changes.

Part 4 - Experimental Design and Instrumentation

Retain the owner during each persistence test and assign a generation on allocation or replacement. Capture shape, strides, offset, base address, tensor address, storage bytes, and matching trace IDs. Use deterministic inputs and verify values before and after the operation. Run fresh/released cases in separate replays and preserve event-history uncertainty. Never keep tensors in the logging structure merely to collect scalar metadata, because that would change their observed lifetimes.

Record whether pointer inspection itself is supported without materialization for the chosen tensor abstraction. The primary experiment excludes lazy or opaque storage. Do not turn an observer-induced copy into evidence about the original operation's allocation behavior.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class PointerEvidence:
    owner_id: str
    generation: int
    tensor_pointer: int
    storage_base: int
    offset_elements: int
    storage_bytes: int


def pointer_evidence(tensor, owner_id, generation):
    # Capture public metadata without extending logged lifetimes.
    ...
```

```
def classify_storage_continuity(before, after, ledger, events):
    # Separate persistent ownership, shared views, and recycling.
    ...


def E03(config):
    # Reject pointer-only conclusions using controlled counterexamples.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Acceptance requires recognizing storage sharing despite differing view pointers and refusing continuous-ownership claims after an unobserved release. A stable pointer establishes neither unchanged values nor gradient safety. Treat empty tensors separately and do not use a private storage identifier as a portable allocation identity. Reconcile the claimed interval with both the owner ledger and supported allocation observations.

Part 7 - Engineering Interpretation and Philosophy

Identity has a temporal component. Strong claims about an owned allocation can be justified within a controlled lifetime; intermittent pointer snapshots of arbitrary tensors require weaker language. Use that distinction in the later mutation analyzer rather than promoting a convenient address proxy to a universal storage proof.

19.30.5 Exercise 04

Exercise Title: Compare add, mul, relu, and Their In-Place Variants and Identify Which Temporary Allocations Disappear

Part 1 - Exercise Contract and Why It Matters

Compare addition, multiplication, and ReLU with their supported in-place variants. Identify which visible result requests disappear, which operands are overwritten, and which configurations are rejected. Deliver an operator-by-case support table rather than generalizing one successful mutation to every operation, dtype, layout, or training use.

Part 2 - System and Memory Mental Model

The underscore convention suggests mutation but does not erase operator-specific shape, casting, overlap, and autograd restrictions. Addition and multiplication read different value sets from ReLU, and backward may depend on inputs or outputs differently. A visible output request can disappear while internal workspaces, launch overhead, and memory traffic remain. Use non-overlapping same-shaped operands first and treat expanded or internally overlapping views as unsupported mutation destinations unless specifically justified.

Part 3 - Quantitative Prediction Before Execution

For each case, calculate output payload ns and predict request differences under an admitted destructive contract. Write separate predictions for same-dtype inference, dtype promotion, broadcasting that would change the left-hand shape, a leaf requiring gradients, and an appropriate nonleaf. Estimate the live-set peak from the schedule and list which facts remain unknown until operator and backward behavior are inspected.

Part 4 - Experimental Design and Instrumentation

Build independent references and starting inputs for every operator. Validate values and permitted side effects, then collect short traces at one accounting layer. Probe one restriction at a time and record actual acceptance, warning, or exception category. Run inference and training tests separately. For accepted training cases compare input gradients, not only outputs. Preserve installed version, backend, shape, dtype, strides, seed, and precision with each row.

Keep input restoration outside both timed paths for a destructive-state benchmark, or inside both when resets are part of useful work. Record that choice in the support table because repeated mutation otherwise changes the values each operator receives.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class OperatorMutationCase:
    operation: str
    shape: tuple
    dtype: object
    layout: str
    execution_mode: str
    expected_status: str


def admit_operator_case(case, inputs, alias_contract):
    # Check the actual operator, ownership, and autograd restrictions.
    ...


def execute_operator_case(case, inputs):
    # Implement the selected fresh or in-place operation explicitly.
    ...


def E04(config):
    # Export numerical, side-effect, request, and rejection evidence.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not silently cast, broadcast to another shape, detach, or disable gradient recording to turn a rejected case into a different task. A successful forward does not establish safe backward or safe application-visible aliases. Include numerically awkward values and empty inputs where supported. Classify a lower request count without lower peak memory as a valid distinct outcome, not a measurement contradiction.

Part 7 - Engineering Interpretation and Philosophy

An operator allowlist should encode complete contracts, not just names ending with underscores. The useful artifact is a table that explains when mutation preserves the intended computation and what measured cost changes under those conditions.

19.30.6 Exercise 05

Exercise Title: Build a Chain of Elementwise Operations and Rewrite It Selectively In-Place to Minimize Peak CUDA Memory

Part 1 - Exercise Contract and Why It Matters

Rewrite an elementwise chain selectively in-place while preserving the declared input and intermediate-value contracts. Compare a fully materializing baseline with a validated partial rewrite. Identify the old values that can be destroyed and the branch or observer that forces a separate result.

Part 2 - System and Memory Mental Model

A chain of operations need not require every intermediate to have distinct storage, but the first input may still belong to the caller. A residual branch can require an earlier value long after the next arithmetic call. Starting with one owned copy and mutating that copy may be legal yet remove less traffic than mutating caller input. Native in-place operations still launch their kernels; this exercise does not claim automatic fusion or a new arithmetic association.

Part 3 - Quantitative Prediction Before Execution

Choose a chain such as multiply, add, ReLU, and a final scale with fixed operation order. Draw live intervals for each materialized result and calculate payload peaks for destructive-input and preserved-input contracts. Add a branch that reads a pre-ReLU value and recompute the legal overwrite points. Predict which request savings survive after charging a necessary initial copy or preserved branch value.

Part 4 - Experimental Design and Instrumentation

Create a host read/write schedule and freeze operator order, dtype, precision, and output retention. Mark every candidate mutation as admitted, rejected, or unknown with a reason. Complete the selected rewrite and compare intermediate values at declared observation points in separate diagnostic runs. Profile short matching traces, then time equivalent unprofiled replays. Introduce one retained alias and verify the policy changes before executing the revised plan.

Do not introduce algebraic reassociation while changing destination ownership. A multiply-add rewrite into another fused primitive is a different arithmetic experiment. Preserve the sequence of operations first, then study fusion separately if it becomes a later objective.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
```

```
@dataclass
class ChainDecision:
    operation_id: int
    candidate_input: str
    remaining_readers: tuple
    decision: str
    reason: str


def plan_chain_mutations(operations, owners, aliases):
    # Include branches, outputs, and operation-specific restrictions.
    ...


def execute_chain(inputs, decisions, preserve_input):
    # Complete only admitted in-place rewrites in the original order.
    ...


def E05(config):
    # Compare peaks and requests after charging preservation copies.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Fail any rewrite that alters a promised input, residual, or historical output. An independent reference must not share mutable scratch with the candidate. Report required copies and absolute owned capacity, not just removed result requests. Keep training out of the initial inference comparison; an eventual training extension needs saved-value and gradient validation. Retain a case where copy cost or workload size removes the expected benefit.

Part 7 - Engineering Interpretation and Philosophy

Selective mutation is a value-lifetime decision rather than a mechanical replacement of every operator. The best rewrite preserves the original arithmetic schedule and destroys only information that no supported consumer still needs.

19.30.7 Exercise 06

Exercise Title: Reproduce an Autograd Failure by Modifying a Tensor In-Place After It Has Been Saved for Backward

Part 1 - Exercise Contract and Why It Matters

Reproduce a saved-for-backward version mismatch using a small controlled graph. Distinguish this deferred error from the immediate restriction on mutating a leaf requiring gradients. Explain which original value backward needed, why it became invalid, and which legal repair preserves the intended objective.

Part 2 - System and Memory Mental Model

A backward formula can retain a forward tensor together with mutation metadata. If native mutation changes that saved value before backward reads it, autograd can reject the inconsistent history. Use a nonleaf intermediate so the experiment reaches this check instead of stopping at a leaf mutation. Executing the mutation under no-grad changes recording behavior but does not make a previously saved value unnecessary. Detaching a sharing alias also does not create an independent preserved value.

Part 3 - Quantitative Prediction Before Execution

Construct a nonleaf $h = 2x$ and a scalar loss based on h^2. Before execution, derive the baseline gradient and identify the h value its backward needs. Predict outcomes for no mutation, mutation after loss construction, and mutation of a separate clone only. Compare storage and preservation cost for repairs. Do not redefine the baseline loss after mutation and claim equivalence to the original forward graph.

Part 4 - Experimental Design and Instrumentation

Use tiny deterministic tensors and build a fresh graph for every case. Capture forward success, mutation success, backward success or failure, and actual exception phase. Use anomaly detection only in a diagnostic replay to help locate the saved-value producer. Validate the unmodified reference and each legal repair with independent gradients. Record PyTorch version and execution mode; do not suppress errors through data aliases, manual counter manipulation, or a blanket backward exception handler.

When using anomaly diagnostics, include forward graph construction in the diagnostic scope so the report can identify the saved-value producer. Keep that diagnostic replay separate from timing and preserve the actual failing backward phase in the record.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def build_saved_value_case(input_value):
    # Construct a nonleaf and a loss whose backward needs its old value.
    ...


def run_mutation_case(initial_value, mutation_policy):
    # Build a fresh graph and preserve expected failure phases.
    ...
    # Perform backward or record the specific expected rejection.
    ...


def validate_repair(reference, repaired, tolerances):
    # Check the same forward objective and corresponding gradients.
    ...
```

```
def E06(config):
    # Separate leaf rejection from saved-for-backward invalidation.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require the intended saved-value failure rather than accepting any RuntimeError as success. Validate the baseline analytically on the tiny graph. A repair that merely detaches the loss or drops a gradient path is invalid. If the chosen operation does not save the value expected in the installed version, revise the case transparently and explain the actual dependency before drawing a conclusion.

Part 7 - Engineering Interpretation and Philosophy

The error protects consistency between a forward computation and its backward history. Repair the value lifecycle or eliminate an unnecessary mutation; do not treat an exception as an obstacle to bypass or confuse an error-free new objective with preserved training semantics.

19.30.8 Exercise 07

Exercise Title: Inspect Tensor Version Counters and Explain How PyTorch Detects Unsafe In-Place Mutations Required by Backward

Part 1 - Exercise Contract and Why It Matters

Observe mutation bookkeeping around ordinary tensors, native views, detached aliases, and independent clones. Explain how saved-value checks use version information and distinguish that mechanism from a general proof of mutation safety. Deliver a diagnostic timeline and explicitly label private or unsupported inspection paths.

Part 2 - System and Memory Mental Model

Ordinary native in-place operations update autograd's mutation bookkeeping. Saved-value validation compares the relevant state when backward accesses the saved tensor. Direct counter inspection through `_version` is a version-sensitive laboratory diagnostic, not a stable public analyzer contract. Inference tensors can lack such counters. A counter change can indicate a write but does not identify every external reader, prove numerical change, or supply CUDA stream dependencies.

Part 3 - Quantitative Prediction Before Execution

Predict a timeline for a no-op-valued native write, an arithmetic write, view creation without writing, clone creation, and backward after a protected value changes. Compare expected counter relationships for ordinary native aliases without generalizing to arbitrary externally constructed storage wrappers. Predict the inspection status for a tensor created in inference mode. Separate unknown counter state from a measured zero and from an unchanged observed counter.

Part 4 - Experimental Design and Instrumentation

Feature-check the diagnostic attribute and retain unsupported outcomes. Use small ordinary tensors for the main timeline, recording observations before saving, after saving, after mutation, and at backward. Keep a live owner and log scalar metadata only. Compare original, native view, detached alias, and clone cases with fresh graphs. Add an inference-created tensor as a

documented exception test. Do not manually increment counters to make a failing graph pass; incrementing cannot recover an overwritten value.

Compare a zero-valued write with a read-only metadata observation. Both may leave tensor values unchanged, but they need not have the same bookkeeping behavior. This distinguishes a numerical equality test from evidence about recorded mutation history.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class VersionObservation:
    tensor_label: str
    phase: str
    observed_version: object
    inspection_status: str
    execution_mode: str


def inspect_version_if_supported(tensor, label, phase, mode):
    # Treat private diagnostic access and unavailable counters explicitly.
    ...


def record_version_timeline(case, config):
    # Observe native writes and the actual backward outcome.
    ...


def E07(config):
    # Compare ordinary alias, clone, and inference-created cases.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not hard-code a universal exact increment count across composite operations or versions. A write adding zero may still invalidate a saved value despite unchanged numerical contents. Counter absence is not permission to mutate. Distinguish immediate leaf errors, backward mismatch, unsupported inspection, and successful supported cases. Production admission should rely on declared semantics and dependencies, not only a private diagnostic number.

Part 7 - Engineering Interpretation and Philosophy

Version bookkeeping is a consistency alarm with a defined scope. Its value is explaining why a saved history became invalid, while the analyzer must still reason about ownership, application consumers, operator contracts, and stream ordering independently.

19.30.9 Exercise 08

Exercise Title: Create Multiple Views of the Same Tensor and Prove That an In-Place Mutation Through One View Changes the Shared Underlying Storage

Part 1 - Exercise Contract and Why It Matters

Create several ordinary views of one owned tensor and demonstrate exactly which logical elements change when a permitted in-place write goes through a view. Distinguish sharing the same backing allocation from overlapping the same elements. Deliver a coordinate map, alias diagram, and observed value changes.

Part 2 - System and Memory Mental Model

A view can change shape, strides, or starting offset while referring to the same backing storage. Two disjoint slices can share an allocation without sharing elements; a transpose can describe the same elements in another coordinate order. Begin outside gradient recording and use ordinary dense positive-stride views. Internally overlapping expanded views are not ordinary writable destinations and should be rejected in the initial mutation contract.

Part 3 - Quantitative Prediction Before Execution

For a contiguous base with shape $(4, 6)$ and element width s, predict element addresses from offset and strides. Define two overlapping row slices, a disjoint slice, and a transpose. Mark which coordinates a selected view write will change in every observer. Calculate the base payload once rather than charging every view a full allocation, and predict the difference between shared-base and overlapping-region reports.

Part 4 - Experimental Design and Instrumentation

Fill the base with unique deterministic values and preserve an independent expected snapshot. Record shape, stride, offset, tensor pointer, storage base, and capacity for each view. Mutate one admitted view and compare every observer with the coordinate prediction. Repeat with another slice offset and a transposed view. Run a clone control and an expanded-view rejection case. Keep instrumentation copies outside the measured allocation region unless they are part of the declared workload.

Enumerate element offsets only for tiny cases with a strict size limit. Large coordinate sets can overwhelm host memory and are unnecessary for the CUDA benchmark. Retain the tiny oracle as independent validation of the conservative geometry calculation.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class ViewDescription:
    label: str
    shape: tuple
    strides: tuple
    offset_elements: int
```

```
    storage_owner: str


def map_view_elements(description, limits):
    # Enumerate tiny supported cases for an independent overlap oracle.
    ...


def mutate_selected_view(views, selected, mutation_contract):
    # Admit the write, then compare all mapped observers.
    ...


def E08(config):
    # Distinguish same-storage sharing from actual element overlap.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate untouched coordinates as carefully as changed ones. Matching base addresses alone cannot identify region overlap, and differing first-element pointers cannot exclude it. A bounding-range overlap can be conservative for strided views; exact tiny coordinate enumeration helps expose false positives. Reject unsupported internal overlap before launching the write rather than relying on one accidental runtime outcome.

Part 7 - Engineering Interpretation and Philosophy

Aliasing has geometry. Knowing that two tensors share storage is a starting point; deciding whether a mutation affects another value requires element regions and consumers. Preserve that distinction when building a conservative tracker later in the chapter.

19.30.10 Exercise 09

Exercise Title: Reproduce a Hidden Aliasing Bug Where Modifying One Tensor Unexpectedly Changes Another Tensor View

Part 1 - Exercise Contract and Why It Matters

Reproduce an application-level aliasing bug in which a helper mutates a view and unexpectedly changes values another component still needs. Demonstrate that this error can occur during inference without an autograd exception. Repair the ownership boundary and compare the costs of the legal alternatives.

Part 2 - System and Memory Mental Model

A Python variable, a slice result, or a container entry is not automatically an independent value snapshot. A helper receiving a view can modify a region of its caller's tensor. Detaching can remove gradient connectivity while preserving this sharing. The bug depends on the intended contract: mutation may be legal for an explicitly borrowed mutable argument but illegal for read-only input or historical data. Name those contracts before repairing anything.

Part 3 - Quantitative Prediction Before Execution

Use a base with unique values, create a slice representing a stored historical feature, and apply a normalization-like destructive update through another view. Predict every affected coordinate and a later aggregate that should have used the old values. Calculate the bytes required for cloning the slice versus cloning the entire base. Compare these costs with a read-only helper that returns a fresh result only when needed.

Part 4 - Experimental Design and Instrumentation

Construct a tiny deterministic reproduction with a clearly stated expected historical value. Capture evidence before helper entry, after mutation, and at the later consumer. Use separate fresh setups for the buggy helper, a read-only rewrite, and an ownership-copy repair. Validate complete observations rather than just the helper's returned result. Include a detached-alias non-repair case and a legitimate mutable-borrow case so the rule does not become an unconditional ban on all mutation.

Include a container-based historical record holding a view instead of a copy. Verify the record before and after helper execution. This makes the contract failure visible even when the mutating helper returns a numerically correct result for its immediate caller.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class ArgumentContract:
    name: str
    access: str
    historical_readers: tuple
    alias_group: str


def destructive_helper(value, contract):
    # Enforce mutable ownership before completing the operation.
    ...


def reproduce_hidden_alias(config):
    # Build overlapping observers and identify the violated expectation.
    ...


def compare_ownership_repairs(reference, candidate_cases):
    # Check history preservation and charge required copying.
    ...


def E09(config):
    # Compare bug, legal repairs, and the detached-alias non-repair.
```

. . .

Part 6 - Measurement, Validation, and Failure Analysis

Require an explicit failed assertion for the later consumer, not merely a pointer match. Validate that the repair restores the original observable history. A full-base copy may be unnecessarily expensive, but a narrow slice copy must still cover all required values. Preserve allocation and latency costs of each repair. Absence of backward or a version error does not make the original helper correct.

Part 7 - Engineering Interpretation and Philosophy

The hidden bug is often an undocumented ownership transfer. Make read-only, borrowed-mutable, and independently retained values explicit. A repair should restore that contract with the smallest justified copy or change in helper behavior.

19.30.11 Exercise 10

Exercise Title: Distinguish Views, Copies, Clones, and Detached Tensors by Storage Pointer, Gradient History, and Mutation Behavior

Part 1 - Exercise Contract and Why It Matters

Distinguish views, actual copies, clones, detached aliases, and reshape results using storage observations, mutation behavior, and gradient relationships. Build a behavior matrix that avoids equating a new Python object with a new allocation or equating independent storage with a disconnected computation graph.

Part 2 - System and Memory Mental Model

Ordinary views share backing storage. Clone creates a separate value copy and can remain differentiable with respect to its source. Detach returns a tensor without that gradient relationship while sharing storage. Reshape and contiguous operations can return sharing results or new storage depending on layout and requirements. Classify observed behavior for the actual case rather than assuming every method name implies one permanent allocation rule. Start with ordinary dense tensors and record unsupported abstractions explicitly.

Part 3 - Quantitative Prediction Before Execution

Predict matrix columns for operation, object identity, backing-storage sharing, region overlap, gradient path, mutation effect, and payload request. Include a contiguous reshape, a noncontiguous case requiring materialization, a clone, a detached alias, and an independent detached clone. Calculate backing bytes once per live owner. Predict how a mutation propagates to existing observers and which scalar losses can legitimately produce source gradients.

Part 4 - Experimental Design and Instrumentation

Create deterministic source tensors with controlled layout and gradient state. Build one fresh graph per gradient test and separate mutation demonstrations from already-saved backward values. Record shape, strides, offset, storage base, requires-grad status, leaf status, and graph-function description where available. Validate values and gradients with independent references. Use tiny coordinate enumeration for region overlap and a short allocation trace for copy claims. Check same-device no-op conversions separately from forced copies.

Test ordinary slicing separately from advanced indexing. Do not assign sharing behavior to all indexing expressions by appearance. Record the exact indexing operation, actual storage evidence, and observed mutation propagation in each row of the behavior matrix.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class TensorBehavior:
    operation: str
    shares_storage: object
    overlaps_source: object
    source_gradient_path: object
    source_changes_on_write: object
    payload_request: object


def construct_behavior_case(source, operation):
    # Apply the selected view/copy/detach transformation.
    ...


def verify_behavior(source, candidate, expected, config):
    # Test storage, values, and gradients in independent replays.
    ...


def E10(config):
    # Export the behavior matrix with unknown/unsupported states.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not use a differing tensor pointer as proof of different storage or a missing graph function as proof of independent memory. Nonleaf gradients are not populated by default; test source gradients through the intended loss rather than calling every absent gradient a failure. Preserve layout changes introduced by copying. If an operation can choose view or copy behavior, record the observed case and its conditions rather than making a universal method-name claim.

Part 7 - Engineering Interpretation and Philosophy

Storage independence and differentiation are separate axes. A reliable mental model names both explicitly, then asks what each observer can read after a write. That model is more useful than memorizing a list of methods as simply safe or unsafe.

19.30.12 Exercise 11

Exercise Title: Compare clone(), detach(), and detach().clone() by Memory Allocation, Storage Sharing, and Autograd Semantics

Part 1 - Exercise Contract and Why It Matters

Compare clone, detach, and detach-followed-by-clone for payload allocation, storage sharing, gradient connectivity, and mutation effects. Select each operation according to a concrete ownership and differentiation requirement, not merely because it suppresses a warning or changes a graph display.

Part 2 - System and Memory Mental Model

Clone gives independent storage while preserving a gradient relationship when the source is tracked. Detach disconnects the result from the source's gradient history without supplying an independent snapshot. Detach-followed-by-clone combines disconnection with independent storage. These descriptions concern the constructed result; later explicitly enabling gradients on an independent result creates a new local differentiation task rather than restoring the original connection. Copy cost remains real even when no backward graph is recorded for it.

Part 3 - Quantitative Prediction Before Execution

For an n-element dense source of width s, predict additional copied payload for the three choices and compare with allocator request measurements. Define a scalar function of each result and predict whether the original source has a gradient path. Include one result that cannot support the proposed backward call because it has no tracked inputs. Predict observer changes under a permitted write and explain why gradient detachment does not isolate storage.

Part 4 - Experimental Design and Instrumentation

Use separate source instances for numerical mutation tests and gradient-path tests. Keep source values deterministic and preserve independent expected snapshots outside the measured region. Collect storage and allocation evidence, then time repeated transformations without profiling overhead. Test a regular contiguous source and a supported noncontiguous view, recording resulting layout. Compare the practical requirements of a read-only logged snapshot, a differentiable branch, and an independent trainable copy.

The source may remain physically resident because another alias still owns its storage, even when a detached result has no gradient connection. Track residency and connectivity independently so a graph simplification is not misreported as released memory.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class CopyChoice:
    name: str
    independent_storage_required: bool
    source_gradient_required: bool
```

```
    expected_payload_bytes: object


def construct_copy_choice(source, choice):
    # Implement the selected transformation without changing the contract.
    ...


def test_gradient_connection(source, result, expectation):
    # Distinguish no path, expected backward refusal, and a valid path.
    ...


def E11(config):
    # Compare storage isolation, traffic, layout, and gradient evidence.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

A detached alias is not a safe historical snapshot of a tensor that will be mutated. A clone is not automatically detached. Do not invoke backward on an untracked scalar and classify the resulting error as a corrupted autograd engine. Record expected path absence explicitly. Charge required snapshots and any retained source ownership, including capacity that remains live because an alias still refers to its backing allocation.

Part 7 - Engineering Interpretation and Philosophy

Choose the cheapest transformation that satisfies both ownership and gradient requirements. A smaller allocation count is irrelevant if an intended snapshot keeps changing, and removing a desired gradient path is a semantic change rather than a memory optimization.

19.30.13 Exercise 12

Exercise Title: Use copy_() to Refill Preallocated CUDA Buffers and Compare It Against Recreating Destination Tensors Every Iteration

Part 1 - Exercise Contract and Why It Matters

Refill preallocated CUDA destinations with successive source values and compare against independently recreating destination tensors each iteration. Measure request elimination, copy traffic, destination identity, and observable history. Do not conflate stable destination storage with eliminating source creation or making its contents safe for every consumer.

Part 2 - System and Memory Mental Model

Copy is an in-place write to the destination. A persistent destination can avoid repeated output allocation but still needs exclusive permission to overwrite its current generation. Broadcasting, conversion, and host/device movement expand the copy contract; exclude them from the initial same-device, same-shape test. A destination saved for backward is not ordinary free staging space, and previous returned views see subsequent refills unless they have an independent ownership guarantee.

Part 3 - Quantitative Prediction Before Execution

For N copies of b bytes, calculate Nb logical copied payload and estimate simple same-device read-plus-write traffic separately. Predict fresh destination request count versus one setup allocation. Draw a case where newly computed sources still allocate every iteration, and another where a retained prior output forces another slot or copy. Predict absolute live bytes during source and destination coexistence instead of treating cumulative traffic as peak memory.

Part 4 - Experimental Design and Instrumentation

Prepare deterministic source generations and declare the destination lease. Complete a fresh-copy baseline and persistent-refill path with identical consumption. Inspect every generation before overwrite, record storage metadata, and collect a short allocation trace. Time warmed unprofiled runs with completed CUDA boundaries and report setup separately. Add a retained-view rejection case and a destination-needed-by-backward case. Keep nonblocking cross-device transfer extensions separate and require their completion rules.

A supported training extension must consider destinations whose previous values were saved in an earlier graph. Reusing a destination across iterations requires completed backward obligations or an explicit alternative value lifecycle, not just an iteration counter advancing.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class RefillLease:
    destination_id: str
    generation: int
    consumers: tuple
    completion_tokens: tuple


def admit_refill(destination, source, lease, contract):
    # Check ownership, compatibility, autograd use, and readiness.
    ...


def refill_destination(destination, source, lease):
    # Complete the copy only after legal admission.
    ...


def E12(config):
    # Compare fresh copies and persistent generations with equal work.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Check the reference does not alias the refilled buffer. A saved value must not be overwritten merely because its Python forward consumer has returned. Report source requests and destination requests separately. Do not hide required copies in setup for only one strategy. Preserve unsupported, rejected, and delayed-consumer outcomes; a stable address alone says nothing about whether an old generation remains valid to its observer.

Part 7 - Engineering Interpretation and Philosophy

Refill is controlled destruction of one generation and publication of another. Its usefulness depends on lease duration and copy cost, not only persistent addresses. A preallocated buffer is safe when the ownership protocol makes every read and overwrite boundary explicit.

19.30.14 Exercise 13

Exercise Title: Identify When In-Place ReLU Saves Memory and When Autograd Requirements Eliminate or Reduce the Expected Benefit

Part 1 - Exercise Contract and Why It Matters

Determine when in-place ReLU removes a useful temporary and when ownership or autograd needs reduce that benefit. Compare inference, an admitted nonleaf training chain, and a branched graph requiring the original preactivation. Validate complete outputs and gradients before measuring savings.

Part 2 - System and Memory Mental Model

ReLU replaces negative entries with zero, destroying information about the original negative magnitudes. A later branch may still need that original preactivation. A supported native in-place ReLU can work in some training graphs, but a leaf requiring gradients and later mutation of backward-needed results create different restrictions. The retained values needed by backward and other live regions determine whether removing one visible result actually changes the total training peak.

Part 3 - Quantitative Prediction Before Execution

For a b-byte preactivation, predict the visible result request potentially avoided. Compare a sequential graph with a residual branch using preactivation values before ReLU. Add a preservation copy and calculate whether its bytes cancel the local saving. Sketch forward and backward peaks separately, including parameters, gradients, and operator workspaces at their actual times. Do not subtract b from a global peak unless the removed allocation was live there.

Part 4 - Experimental Design and Instrumentation

Build independent models and inputs with identical state and precision. Use a nonleaf activation for the admitted training case and a distinct test for leaf rejection. Compare fresh and in-place ReLU under matching arithmetic order, then add the preactivation branch. Check outputs, input and parameter gradients, and multi-step updates before timing. Use tiny diagnostic saved-value observations separately from low-overhead peak measurements so the observer does not retain activations and move the peak.

Run a second mutation after the admitted ReLU in a separate diagnostic graph to test whether the result remains protected for backward. Classify that follow-up independently; acceptance of the ReLU does not grant unrestricted permission to overwrite its output.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class ReluCase:
    mode: str
    preactivation_readers: tuple
    expected_acceptance: str
    preservation_required: bool


def execute_relu_case(model, inputs, case, inplace):
    # Preserve the full graph and admit only legal mutation sites.
    ...


def compare_relu_training(reference, candidate, tolerances):
    # Check outputs, all required gradients, and updates.
    ...


def E13(config):
    # Compare local requests and actual complete-step memory peaks.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not interpret a valid nonleaf example as permission to mutate every ReLU input. A branch can be wrong without triggering the intended autograd error. Include values on both sides of zero and state the framework convention at zero. If preservation is required, charge its allocation and lifetime. Report small or absent global savings honestly even when the local result request has disappeared.

Part 7 - Engineering Interpretation and Philosophy

An activation's local implementation cannot determine its global memory benefit alone. The useful question is whether the preactivation can be destroyed and whether its removed result overlaps the true peak without requiring an equally large preserved substitute.

19.30.15 Exercise 14

Exercise Title: Build an Alias Tracker That Detects Tensors Sharing the Same Underlying Storage Before Allowing an In-Place Mutation

Part 1 - Exercise Contract and Why It Matters

Build a conservative tracker for registered tensors sharing owned dense storage. Before admitting mutation, identify potential observers and the region affected. Deliver explicit same-storage, disjoint-region, overlapping-region, and unknown classifications with reasons. Do not claim universal discovery of every Python or external-library alias.

Part 2 - System and Memory Mental Model

Storage grouping and exact element overlap are different tasks. A base address, device, capacity, and continuous owner generation can identify ordinary owned backing storage within a restricted runtime. Address reuse after owner release requires another generation. Positive-stride region bounds can conservatively flag overlap but may overapproximate strided element sets. Escaped or externally constructed aliases and unsupported tensor layouts must force conservative handling or refusal, not an invented disjoint result.

Part 3 - Quantitative Prediction Before Execution

For a positive-stride dense view, calculate a conservative addressed span from offset, shape, strides, and element width. Explain why overlapping spans do not always mean overlapping elements, while disjoint valid spans can establish non-overlap in the supported case. Enumerate tiny stepped slices to compare exact element sets with bounds. Define how empty tensors and different devices are classified without grouping unrelated zero pointers.

Part 4 - Experimental Design and Instrumentation

Register a base, overlapping slices, disjoint slices, transpose, clone, and detached alias. Keep live-owner metadata explicit without allowing diagnostic logs to retain tensors indefinitely. Compare conservative reports against a tiny coordinate-enumeration oracle. Introduce release/reallocation and verify generation separation. Test an unregistered escape and unsupported internal overlap; admission should remain unknown or rejected. Check the tracker before mutation, then validate all registered observers afterward.

Use allocation generations in the registry key and invalidate stale records after owner replacement. A coincidentally repeated address must not transfer an old alias relationship to unrelated storage or erase an obligation belonging to an older still-live generation.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class AliasRecord:
    tensor_key: str
    owner_generation: tuple
    shape: tuple
    strides: tuple
    offset_elements: int
    element_bytes: int
    support_status: str

class AliasTracker:
```

```
    def register(self, tensor, ownership):
        # Record supported geometry and continuously owned storage.
        ...

    def potential_observers(self, tensor_key, write_region):
        # Separate proven disjointness from conservative conflicts.
        ...

    def admit_mutation(self, tensor_key, operation, consumers):
        # Refuse unknown escapes and protected overlapping readers.
        ...


def E14(config):
    # Compare tracker reports with independent tiny overlap cases.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require no false-safe classifications in the supported test suite; report conservative false positives separately. Storage aliases must not be assumed to share every internal autograd bookkeeping detail under arbitrary external construction. A reference-count or container scan is not a proof that no alias escaped. Preserve the registration contract, owner generations, and unsupported geometry in each mutation decision.

Part 7 - Engineering Interpretation and Philosophy

A conservative tracker earns trust by distinguishing what it can establish from what it cannot observe. Its purpose is safe admission within an explicit ownership boundary, with explanations for rejected opportunities rather than an unsupported global claim about all tensors.

19.30.16 Exercise 15

Exercise Title: Create a Memory Planner That Marks Intermediate Tensors as Safe or Unsafe for In-Place Reuse from Their Remaining Consumers

Part 1 - Exercise Contract and Why It Matters

Create a planner that classifies candidate intermediate mutations as safe, unsafe, or unknown from remaining consumers and explicit operator contracts. Account for forward readers, backward-saved requirements, aliases, external outputs, and device completion. Deliver reasoned decisions and an independently validated schedule, not a rewrite based only on the next Python use.

Part 2 - System and Memory Mental Model

A value can be dead for one forward branch yet still needed by another branch or backward. The planner must model the specific value an operator overwrites, not merely the lifetime of its tensor object. Native autograd support also depends on leaf/view restrictions and the selected operation. Begin with a fixed graph and one stream; unknown operators or saved-value behavior require an unchanged out-of-place path. Remaining-consumer count alone is therefore necessary context, not a complete safety certificate.

Part 3 - Quantitative Prediction Before Execution

For value v at candidate site j, list remaining forward readers $F(v, j)$, backward requirements $B(v, j)$, and external observers $O(v, j)$. Construct a sufficient admission rule combining discharged obligations, compatible aliases, supported operator semantics, and ordered final readers. Compare a chain, a residual branch, and a backward-saved intermediate. Predict which decisions stay unknown when backward requirements cannot be established without relying on private graph internals.

Part 4 - Experimental Design and Instrumentation

Represent explicit read/write sets, alias groups, and ownership contracts in host metadata. Derive candidate decisions before executing the optimized graph. Validate them using an independent checker and small hand-labeled examples. Execute only admitted rewrites, keeping unknown sites out-of-place. Compare values and gradients with the original graph. Change a consumer or expose an output alias and require invalidation of the old plan. Log the exact reason and evidence for every acceptance and rejection.

Plan validity belongs to a graph and contract version. Adding a branch, changing an output lease, or altering an operator can invalidate previous last-use facts. Require reanalysis or a checked unchanged path instead of reusing stale decisions.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class MutationDecision:
    operation_id: int
    value_id: str
    forward_obligations: tuple
    backward_obligations: tuple
    alias_obligations: tuple
    status: str
    reason: str


def classify_candidate(site, graph_contract, alias_tracker):
    # Combine consumers with operator, ownership, and backward rules.
    ...


def validate_mutation_plan(plan, independent_model):
    # Reject stale graph assumptions and unsupported safety claims.
    ...


def E15(config):
```

```
    # Compare predicted decisions with preserved graph behavior.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Test at least one site with no remaining ordinary forward reader that is still unsafe because backward or an external alias requires the old value. Require unknown decisions to remain unmodified. A successful backward run supports the executed case but does not discover every possible caller contract. Preserve false-safe findings as correctness failures and conservative rejections as missed opportunities, with distinct metrics.

Part 7 - Engineering Interpretation and Philosophy

Planning mutation means discharging all obligations to the old value. A useful planner can explain why a site is refused and what extra information would change that decision. It should optimize within known semantics rather than guess those semantics from an empty forward-use list.

19.30.17 Exercise 16

Exercise Title: Rewrite a Memory-Heavy PyTorch Forward Pass with Selective In-Place Operations While Preserving Exact Outputs and Gradients

Part 1 - Exercise Contract and Why It Matters

Rewrite a memory-heavy forward graph using selected supported in-place operations while preserving the same outputs and gradients. Treat exact preservation as the target: keep arithmetic order and derivative participation unchanged, and distinguish bitwise evidence from tolerance-based numerical agreement. Never present an approximate rewrite as exact.

Part 2 - System and Memory Mental Model

A legal storage rewrite changes where values reside, not which function is computed. Protect saved activations, residual inputs, caller-owned tensors, and returned histories. Replacing operations can still interact with floating point behavior, layout, or kernel selection; do not assume identical bits without checking the actual environment. A clone used to establish ownership must be included in memory and time. Parameter updates under no-grad do not justify arbitrary mutation during forward.

Part 3 - Quantitative Prediction Before Execution

Build an operation-by-operation live-byte prediction for the original and candidate graphs. Calculate which result requests disappear and where required preservation copies reintroduce bytes. Predict the true peak by time, not by summing removed payloads. Define output and gradient acceptance thresholds before testing, including a separate bitwise comparison where supported. Choose finite deterministic inputs for the strict primary comparison and additional boundary-value cases for robustness.

Part 4 - Experimental Design and Instrumentation

Clone identical model and optimizer initial state and pre-generate matching batches. Freeze dtype, precision settings, RNG policy, layout, operation order, and training mode. Validate every named parameter gradient, input gradient, output, loss, and update across several steps. Diagnose mismatches before profiling performance. Use independent fresh graphs for each

strategy and exclude anomaly tracing from timed runs. Add a residual branch and an aliased input to verify the admission policy refuses invalid opportunities.

Check caller-visible input tensors and model buffers as well as trainable parameters. A rewrite that preserves loss and parameter gradients can still modify a promised input or running statistic, changing later behavior outside the immediate comparison.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class EquivalenceReport:
    outputs_bitwise_equal: object
    gradients_bitwise_equal: object
    maximum_output_error: object
    maximum_gradient_error: object
    missing_gradients: tuple
    update_status: str


def original_forward(model, inputs):
    # Build the reference graph with unchanged intended arithmetic.
    ...


def selective_forward(model, inputs, validated_plan):
    # Apply only justified storage rewrites and charge required copies.
    ...


def verify_complete_equivalence(reference, candidate, contract):
    # Compare outputs, inputs, named gradients, losses, and updates.
    ...


def E16(config):
    # Restore identical state and validate multi-step semantics first.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

A matching scalar loss can hide incorrect individual gradients. Missing or unexpected gradients require explicit comparison, and a detached branch is a failure even if output values match. If only tolerance agreement is achieved, state that limitation rather than calling the result bitwise exact. Keep nonfinite handling explicit and preserve the exact failing configuration. No performance recommendation is accepted before the declared equivalence gate passes.

Part 7 - Engineering Interpretation and Philosophy

Storage optimization is justified by a preserved computation, not by a smaller memory graph alone. Separate the intended exact semantics from the empirical numerical evidence available on a particular device, and keep all required preservation costs in the final engineering result.

19.30.18 Exercise 17

Exercise Title: Profile the Optimized and Original Forward Pass and Determine Whether Reduced Allocations Improve Real Performance or Only VRAM Usage

Part 1 - Exercise Contract and Why It Matters

Profile original and validated selective-in-place execution to determine whether fewer allocations improve latency, peak VRAM, both, or neither. Keep the successful equivalence contract from E16 fixed. Deliver diagnostic traces, unprofiled timing distributions, and a conclusion tied to the actual workload rather than the number of rewritten sites.

Part 2 - System and Memory Mental Model

Allocation overhead may be small relative to kernels, memory bandwidth, and host dispatch. In-place operations do not automatically fuse a chain or reduce its arithmetic count. A warm allocator can already recycle temporary storage, while preserved aliases or gradients can keep capacity live. Profiling can retain metadata or change overhead, so use separate diagnostic and timing replays. Treat eager and compiled execution as different explicit experiments if compilation is introduced at all.

Part 3 - Quantitative Prediction Before Execution

Define relative request reduction $1 - A_m/A_b$ for $A_b > 0$, memory reduction $P_b - P_m$ at a named peak layer, and speedup T_b/T_m for $T_m > 0$. Keep undefined ratios undefined. Predict outcomes for tiny tensors dominated by dispatch, large bandwidth-bound tensors, and a graph whose peak lies outside the optimized region. Estimate added checks and preservation-copy costs before measurement and explain why fewer requests need not lower reserved memory.

Part 4 - Experimental Design and Instrumentation

Use the same graph, model state, inputs, gradient mode, precision, output contract, and useful iteration count. Collect a bounded profiler trace with memory activity and an operation timeline. Reconcile visible request savings, then perform repeated unprofiled warmed trials in randomized order. Record complete wall time and an appropriately bounded device-time region. Include forward/backward/update for training comparisons and label a forward-only benchmark explicitly. Report cold setup separately and keep raw samples.

If checking whether results match forces a device synchronization, keep that check out of both timed paths or explicitly include it in both. Avoid accidentally giving one strategy a completed kernel region and the other only a host-submission interval.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
```

```
@dataclass
class PerformanceEvidence:
    strategy: str
    allocation_layer: str
    request_count: object
    peak_allocated_bytes: object
    peak_reserved_bytes: object
    wall_samples: tuple
    device_samples: tuple


def diagnostic_profile(workload, config):
    # Collect bounded allocation and kernel evidence in its own replay.
    ...


def unprofiled_trials(workload, trial_order, config):
    # Match state, completion boundaries, and complete useful work.
    ...


def E17(config):
    # Preserve the equivalence gate and compare measured distributions.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not compare a profiled original with an unprofiled optimized run. Attribute extra copies, synchronization, and plan-check overhead. Report median and a declared tail statistic with sample counts and variability. A memory-only benefit can be useful without a speedup, while a slower optimized graph is an adverse result that must remain visible. Never invent benchmark values when a CUDA environment or measurement field is unavailable.

Part 7 - Engineering Interpretation and Philosophy

Performance is the completed workload's behavior, not the aesthetic quality of an allocation trace. Recommend the rewrite only for the measured objective and operating range, explaining when saved capacity matters more than latency and when the original execution is already sufficient.

19.30.19 Exercise 18

Exercise Title: Stress-Test In-Place Optimization with Views, Residual Connections, Shared Storage, Autograd, and Multiple CUDA Streams

Part 1 - Exercise Contract and Why It Matters

Stress-test mutation decisions with views, residuals, shared storage, saved backward values, and multiple CUDA streams. Validate both accepted rewrites and explainable refusals. Preserve replayable traces and minimized counterexamples so a planner's claimed support can be audited beyond one successful graph.

Part 2 - System and Memory Mental Model

Mutation safety depends on which reader sees which generation. Stream order must place a write after all old-value readers and before all new-value readers. Allocator lifetime protection does not itself order application overwrites. Autograd can detect some invalid saved histories but is not a complete detector for stream races or wrong forward contracts. Apply host dependency validation before executing concurrent CUDA tests, and use supported runtime diagnostics only as additional evidence.

Part 3 - Quantitative Prediction Before Execution

Construct a schedule with two streams reading an old generation and a later writer publishing a new one. Predict the required happens-before edges, then remove one edge in host metadata and require rejection. Add an escaped view, a residual consumer, and a backward-saved value to the same ownership model. Report accepted workload coverage, correct completions, rejected opportunities, and bounded outstanding generations rather than an error rate alone.

Part 4 - Experimental Design and Instrumentation

Generate small deterministic cases plus seed-controlled randomized graph and alias metadata. Use an independent oracle for tiny overlap and dependency cases. Execute only validated plans with matching fresh references; compare outputs and gradients where supported. Test delayed readers, variable shapes, retained outputs, repeated backward requirements, unsupported layouts, and shutdown with outstanding work. Rebuild state for every replay. Minimize failures while preserving the same dependency and alias cause; never run a known-invalid concurrent write merely to seek visible corruption.

Protect host diagnostic state from unbounded growth during long stress runs. Use bounded scalar records and replay IDs; retain detailed traces only for selected incidents. Instrumentation must not keep every graph or tensor alive and thereby change the memory problem.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class MutationStressCase:
    case_id: str
    alias_description: tuple
    residual_readers: tuple
    backward_requirements: tuple
    stream_dependencies: tuple
    expected_decision: str


def validate_stress_schedule(case, independent_model):
    # Reject missing old-reader/new-writer ordering before CUDA use.
    ...
```

```
def execute_accepted_case(case, analyzer, reference, config):
    # Compare values, gradients, and generation-specific observations.
    ...


def minimize_mutation_failure(case, replay_predicate, limits):
    # Preserve the causal conflict in a bounded small reproduction.
    ...


def E18(config):
    # Export coverage, admitted correctness, refusals, and replay data.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Passing numeric tests alone cannot prove a race-free dependency graph. Require explicit edge and ownership invariants. Classify expected leaf or saved-value errors separately from numerical bugs, timeouts, OOM, and unrelated failures. A planner rejecting all difficult work has low useful coverage even if it never corrupts memory. Preserve unreproduced hypotheses honestly and distinguish host simulation evidence from actual CUDA observations.

Part 7 - Engineering Interpretation and Philosophy

A trustworthy optimizer knows its boundary. Use adversarial traces to refine that boundary, with zero tolerated false-safe decisions in the tested scope and transparent conservative refusals. Expand support only when the missing alias, backward, or completion information can be validated.

19.30.20 Exercise 19

Exercise Title: Build and Benchmark a PyTorch In-Place Optimization Analyzer That Identifies Safe Mutation Opportunities and Quantifies Memory Savings

Part 1 - Exercise Contract and Why It Matters

Build an analyzer that identifies supported safe mutation opportunities, explains refused or unknown sites, and benchmarks selected rewrites. Integrate ownership, alias geometry, remaining consumers, backward requirements, and stream order. Quantify measured memory and request savings while preserving the exact intended computation and useful workload coverage.

Part 2 - System and Memory Mental Model

Separate observation, semantic analysis, admission, execution, and measurement. Start from an explicit fixed graph and a small operator allowlist rather than claiming automatic correctness for arbitrary Python programs. Registered owned storage defines the alias boundary. Unknown consumers, opaque operators, unsupported layouts, or unavailable backward requirements keep a site out-of-place. Diagnostic version observations can explain an incident but are not the sole production admission rule. Accepted sites still need numerical and dependency validation.

Part 3 - Quantitative Prediction Before Execution

For candidate plan π, define an objective such as reducing temporary request bytes under correctness, coverage, memory-budget, and progress constraints. Predict removed result requests, preservation-copy demand, peak live bytes, and analyzer overhead at each site. Aggregate peaks on a timeline rather than summing local savings. Compare an unchanged baseline, one-site rewrites, and the integrated plan. State that safety and measured improvement do not imply global minimal memory or a universally optimal selection.

Part 4 - Experimental Design and Instrumentation

Freeze the support matrix and calibrate decisions on separate traces. Validate plans with an independent host checker, then execute equivalent inference and supported training graphs from identical states. Include elementwise chains, residuals, views, detached aliases, saved activations, and stream-dependent consumers. Compare output values, input and parameter gradients, updates, request traffic, absolute allocated/reserved peaks, and unprofiled timing. Run ablations for alias analysis and consumer information without weakening the safety gate. Keep rejected and unknown opportunities in the report.

Report opportunity coverage separately from execution coverage. An unchanged out-of-place fallback can preserve useful work while admitting no mutations in a difficult region. That is different from refusing the workload entirely and must have a different reported outcome.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class AnalyzerContract:
    supported_operations: tuple
    supported_layouts: tuple
    execution_modes: tuple
    alias_boundary: str
    backward_policy: str
    stream_policy: str

class InPlaceOptimizationAnalyzer:
    def __init__(self, contract):
        self.contract = contract
        # Initialize bounded metadata, evidence IDs, and support checks.
        ...

    def analyze(self, graph, ownership, aliases, dependencies):
        # Derive old-value obligations and candidate mutation sites.
        ...

    def classify(self, site, analysis):
        # Return safe, unsafe, or unknown with explicit supporting facts.
```

```
        ...

    def select(self, decisions, memory_model):
        # Balance expected savings with preservation and checking cost.
        ...

    def validate(self, plan, independent_checker):
        # Verify operator, alias, backward, output, and stream contracts.
        ...

    def execute(self, graph, inputs, validated_plan):
        # Apply only admitted rewrites without changing arithmetic order.
        ...

    def explain(self, site_id):
        # Export reasons without retaining live computation graphs.
        ...


def E19(config):
    # Validate semantics, profile separately, and benchmark useful work.
    ...
    # Export a supported recommendation and unresolved limitations.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Acceptance requires four gates: preserved outputs and gradient participation; valid ownership, alias, and completion obligations; honest layer-specific allocation accounting; and equivalent complete-workload timing. Record device, versions, backend, seeds, precision, graph/trace IDs, support status, decision reasons, preservation copies, request counts/bytes, absolute peaks, error metrics, timing samples, and workload coverage. Export CPU-only metadata.

Demonstrate a useful accepted site, protected residual, detached-alias hazard, saved-value rejection, unknown operator, and delayed reader. No site may be called safe merely because its sampled backward succeeds. Preserve missing hardware measurements as not measured and retain adverse performance results.

Part 7 - Engineering Interpretation and Philosophy

The analyzer is useful when it can justify a small safe rewrite and quantify its actual consequence. Its promise is conditional optimization within known contracts, not universal mutation safety. Recommend the smallest effective configuration and name the next specific experiment required before extending operator, alias, training, or stream support.

19.31 DMM29

Chapter Title: Meta Device and Zero-Allocation Model Construction
Specialization: Dynamic Memory Management and CUDA Systems

19.31.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Meta Tensors, Tensor Metadata, Deferred Storage Allocation, and Zero-Data Model Construction

Part 1 - Exercise Contract and Why It Matters

Build a lifecycle model separating a model's structural description, allocated parameter storage, valid initialized values, checkpoint-backed values, and execution readiness. Explain what meta construction postpones and which costs remain. Deliver a state diagram and an accounting table rather than equating a large logical parameter count with already resident physical weights.

Part 2 - System and Memory Mental Model

Meta tensors represent metadata without real element values. Shapes, strides, dtypes, and gradient requirements can describe future computation, but they cannot supply unknown weights. Python modules, tensor metadata, registries, and the interpreter still consume host memory. Zero-data construction therefore means avoiding parameter payload allocation, not zero process allocation. Materialization supplies storage; initialization or checkpoint loading supplies valid contents. Execution also needs supported operators, inputs, buffers, activation capacity, and any required device transfers.

Part 3 - Quantitative Prediction Before Execution

For logical tensors with element counts n_i and target element widths s_i, predict nominal payload $B = \sum_i n_i s_i$ under an explicitly untied dense model. Separate metadata overhead, buffers, gradients, optimizer state, and workspaces. Draw lifecycle peaks for ordinary construction, meta followed by initialization, and meta followed by checkpoint assignment. Mark which values and resources exist at each stage and identify the assumptions behind every predicted avoided allocation.

Part 4 - Experimental Design and Instrumentation

Use a small ordinary model as a numerical reference and a configurable meta skeleton for structural comparisons. Record architecture, versions, dtype, parameter/buffer counts, process baseline, and CUDA initialization state. Inspect metadata without reading nonexistent values. Create a readiness record that distinguishes meta-only, allocated-uninitialized, checkpoint-loaded, and verified states. Keep model construction, storage allocation, value production, and first execution as separate measured phases. Log only bounded scalar records so instrumentation does not become a second model registry.

Keep logical size, storage readiness, and value authority in separate fields. A later consumer must not infer readiness from the presence of a Parameter object or from a non-meta device label alone; both can precede valid contents.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
from torch import nn
```

```
@dataclass
class ModelLifecycle:
    architecture_id: str
    logical_parameter_elements: int
    nominal_payload_bytes: int
    storage_state: str
    value_state: str
    ready_for_execution: bool


def classify_model_state(model, value_ledger):
    # Distinguish metadata, allocated storage, valid values, and readiness.
    ...


def predict_construction_cost(architecture, target_policy):
    # Separate payload, metadata, buffers, and execution headroom.
    ...


def E00(config):
    # Validate lifecycle predictions with small supported cases.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not treat a reported logical byte size as physically resident data on meta. A successful metadata-only forward is not numerical validation of real weights. A model can contain a mixture of meta and real tensors, so inspect all relevant parameters and buffers rather than one representative weight. Preserve unknown or unsupported state explicitly. A materialized model whose contents have never been initialized or loaded must remain unavailable for numerical use.

Part 7 - Engineering Interpretation and Philosophy

Meta construction separates knowing what a model requires from paying for its values. That separation is useful for planning, but it does not create missing data or promise cheap execution. A trustworthy loader makes every transition from description to valid resident state explicit.

19.31.2 Exercise 01

Exercise Title: Create PyTorch Tensors on device="meta" and Inspect Shape, Stride, Dtype, and Gradient Metadata Without Allocating Real Storage

Part 1 - Exercise Contract and Why It Matters

Create ordinary dense meta tensors and inspect their metadata without materializing element data. Compare shape, strides, dtype, device, element count, and gradient flags across factory and view operations. State which observations describe logical structure and which would require actual values.

Part 2 - System and Memory Mental Model

A meta factory can describe a tensor with a very large logical shape without allocating its element payload. A stride change or view can remain metadata-only when a supported meta implementation exists. Data-dependent results such as a scalar value extraction cannot be inferred from shape alone. Gradient metadata and even a structural autograd graph are not numerical gradients. Keep these facts separate from process overhead and unsupported meta operators.

Part 3 - Quantitative Prediction Before Execution

For a contiguous shape (a, b, c), derive expected element count and strides before execution. Predict metadata after a transpose and a compatible reshape. Calculate nominal bytes for two dtypes but label that quantity future logical payload. List expected outcomes for metadata inspection, a supported shape-only operation, scalar extraction, and an operation whose result shape depends on values. Do not predict zero host RSS merely because the payload is absent.

Part 4 - Experimental Design and Instrumentation

Choose small hand-checkable shapes plus a bounded larger shape. Complete meta factories and collect public metadata in CPU-only records. Compare with tiny real tensors having the same structure and layout. Feature-test operator support rather than assuming every PyTorch operation has a meta kernel. Add intentional value-dependent probes in isolated cases and record their actual failure categories. Measure the process baseline separately from newly created tensor metadata, and do not inspect pointers as though meta addresses proved real backing allocations.

Compare metadata inspection with a tiny real tensor only after matching the layout contract. CPU and CUDA metadata behavior can differ for some operators. Preserve the intended target semantics and treat a structural disagreement as a case requiring investigation.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class TensorMetadata:
    shape: tuple
    strides: tuple
    dtype: object
    device: object
    requires_grad: bool
    is_meta: bool
    logical_elements: int


def construct_meta_case(shape, dtype, requires_grad):
    # Create the requested metadata-only tensor explicitly.
    ...
```

```
def inspect_metadata(tensor):
    # Export structural fields without scalar value extraction.
    ...


def E01(config):
    # Compare structural predictions and supported/unsupported probes.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require exact agreement on the declared metadata in supported cases. Record unsupported kernels independently from value-dependent impossibility. Empty shapes and noncontiguous views need their own predictions. A storage object's logical capacity, if exposed, is not evidence of allocated real data. Do not silently replace a failed meta operation with a real CPU allocation and then report the result as a metadata-only experiment.

Part 7 - Engineering Interpretation and Philosophy

Metadata is useful information with precise limits. The exercise succeeds when the learner can state what a meta tensor establishes, what remains unknown until values exist, and what a missing operator implementation prevents within the tested software version.

19.31.3 Exercise 02

Exercise Title: Compare CPU, CUDA, and Meta Tensor Construction and Measure the RAM and VRAM Avoided by Meta Allocation

Part 1 - Exercise Contract and Why It Matters

Compare CPU, CUDA, and meta construction at matching shapes and dtypes. Measure host process changes, CUDA allocated/reserved peaks, and complete phase times. Separate uninitialized allocation from initialization so deferred physical page commitment or skipped value production does not distort the comparison.

Part 2 - System and Memory Mental Model

CPU allocation can reserve virtual space before every page is physically touched. CUDA context and allocator setup can dominate a first tensor. Meta avoids element payload but still creates host metadata. An empty real tensor and a filled real tensor therefore test different commitment boundaries. Reported tensor bytes, RSS, virtual size, allocator bytes, and driver-visible usage are different measurements and must not be merged into one memory number.

Part 3 - Quantitative Prediction Before Execution

Predict nominal payload $b = ns$ for each real tensor and zero real element payload for its meta counterpart. Add expected framework and metadata overhead as separate unknown or measured terms. Predict CPU RSS before and after touching all elements and CUDA peaks with a warmed context. Construct a timeline showing why a quick empty allocation cannot establish the full resident cost of an initialized model or a future execution.

Part 4 - Experimental Design and Instrumentation

Run independent bounded processes for each device and phase. Record a baseline, factory return, explicit real-data initialization, and release observations. Use supported process-memory instruments and named CUDA counters, with explicit synchronization for completed GPU phases. Choose sizes admitted by a safe preflight estimate; skip unsupported CUDA rather than inventing measurements. Repeat small and larger cases, preserving cold-context and warmed-context results separately. State whether page touching, validation, garbage collection, and allocator cache retention are included in every reported interval.

For CPU initialized trials, record whether data pages were actually touched. For CUDA trials, record when the context was first created. These controls prevent a virtual reservation or one-time framework startup from being confused with parameter-payload savings.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class ConstructionMeasurement:
    device_kind: str
    phase: str
    rss_bytes: object
    virtual_bytes: object
    cuda_allocated_bytes: object
    cuda_reserved_bytes: object
    elapsed_seconds: object


def construct_and_measure(shape, dtype, device, phase, config):
    # Keep allocation and value initialization as distinct regions.
    ...


def compare_device_trials(trials, measurement_contract):
    # Compare matching commitment and completion boundaries only.
    ...


def E02(config):
    # Preflight sizes and retain unavailable/unsupported outcomes.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require the same logical shape and dtype and distinguish factory latency from initialization latency. A sampled RSS value is not a guaranteed process high watermark. A meta timing need not beat a tiny CPU operation. Report process, allocator, and physical-memory uncertainty

honestly. Avoid a deliberately unsafe real allocation merely to demonstrate OOM; predicted oversized real cases can remain analytically classified and unexecuted.

Part 7 - Engineering Interpretation and Philosophy

Avoided payload is a structural benefit; measured resident savings depend on commitment, caching, and observation boundaries. The meaningful comparison names those boundaries and includes the future cost when data is eventually needed.

19.31.4 Exercise 03

Exercise Title: Instantiate an Entire nn.Module Inside with torch.device("meta") and Verify That Its Parameters Contain Metadata but No Real Values

Part 1 - Exercise Contract and Why It Matters

Construct an entire supported module under a meta device context and audit all registered parameters and buffers. Verify that its architecture and metadata match a tiny real reference while identifying any constructor work that escapes the metadata-only contract.

Part 2 - System and Memory Mental Model

A device context supplies a default to eligible factory calls; an explicit CPU or CUDA argument can override it. Constructors may also allocate ordinary Python data, load files, perform value-dependent decisions, or retain unregistered tensors. Meta support must therefore be checked for the concrete module rather than inferred from one context manager. Registered persistent and nonpersistent buffers both matter for later execution, even though their checkpoint participation differs.

Part 3 - Quantitative Prediction Before Execution

Predict parameter shapes and element counts for a short multilayer module including a registered buffer. Estimate future parameter and buffer payloads separately. Add a deliberately explicit CPU factory inside one test constructor and predict how a complete audit detects the escape. Compare module count, parameter-object count, and logical element count without treating tied names as independent physical values.

Part 4 - Experimental Design and Instrumentation

Define a transparent small module with known submodules and buffer registration. Complete construction in the requested device context and traverse all registered state with duplicate handling documented. Compare shapes, dtypes, requires-grad flags, and expected buffer persistence against a real reference. Audit declared constructor-owned nonregistered resources separately. Include one explicit-device escape and one value-dependent constructor as negative controls. Do not run a real numerical forward through a model that still contains unmaterialized required state.

Inspect buffers registered with persistence disabled even though they do not appear in an ordinary state dictionary. A checkpoint-key audit alone cannot verify all execution state; maintain an architecture inventory covering both checkpointed and separately initialized resources.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
from torch import nn

@dataclass
class StateAudit:
    name: str
    role: str
    shape: tuple
    dtype: object
    device: object
    is_meta: bool
    checkpoint_expected: bool


def build_module_on_meta(factory, architecture):
    # Apply the context and construct only the supported architecture.
    ...


def audit_module_state(model, declared_extra_resources):
    # Include parameters, all registered buffers, and explicit escapes.
    ...


def E03(config):
    # Compare the structural reference and constructor escape controls.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Acceptance requires that every required payload is meta in the supported case, not merely that the first parameter is meta. Record legitimate exceptions or fail the zero-data construction contract. A missing persistent checkpoint entry and a nonpersistent buffer are different conditions. Exclude unsupported custom constructors from broad claims and keep structural agreement separate from numerical model equivalence.

Part 7 - Engineering Interpretation and Philosophy

A construction context is a useful default, not an enforcement sandbox. Reliable deferred construction starts with an audited architecture and a complete inventory of what its constructor creates, retains, and requires for later readiness.

19.31.5 Exercise 04

Exercise Title: Construct a Model Too Large to Fit in Available RAM Using the Meta Device Without Triggering an Allocation Failure

Part 1 - Exercise Contract and Why It Matters

Describe a dense model whose hypothetical real parameter payload exceeds available host RAM, then construct its metadata-only skeleton within a bounded host budget. Establish feasibility of the description without executing a known-oversized real allocation or promising that every constructor can avoid failure.

Part 2 - System and Memory Mental Model

Logical payload and metadata complexity scale differently. A few very large matrix shapes can represent enormous parameter counts with relatively few objects. Millions of separate tensors can instead exhaust host metadata despite using meta. Shape integer limits, unsupported operations, explicit device allocations, and value-dependent initialization can also fail. The title's allocation-avoidance goal concerns element payload, not immunity from every host or framework resource limit.

Part 3 - Quantitative Prediction Before Execution

For L independent dense square weights of width d and target element width s, calculate Ld^2s nominal bytes plus declared biases and buffers. Choose bounded (L, d) cases whose logical payload crosses a documented RAM reference. Predict host metadata growth for fixed L with increasing d, then for increasing L. Explain why neither experiment establishes that real weights, activations, or training state will fit.

Part 4 - Experimental Design and Instrumentation

Use a dedicated child process with a deadline and a host-memory ceiling for controlled construction tests. Preflight shape arithmetic and prohibit real materialization in the oversized path. Start with a tiny real structural reference, then scale the meta architecture in steps. Record object counts, logical bytes, process high-water evidence, elapsed time, and outcomes. Keep explicit CPU factory escapes out of the supported model and test their detection only at safe sizes. Preserve unsupported or interrupted cases without turning them into successful constructions.

Do not generate a real checkpoint of the oversized artificial model as part of this exercise. A tiny representative checkpoint is sufficient for format validation elsewhere; oversized meta construction needs no real payload file and must not silently create one.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
from torch import nn

@dataclass
class OversizedModelCase:
    layer_count: int
    width: int
    target_dtype: object
    logical_payload_bytes: int
    host_metadata_limit: int
```

```
def preflight_oversized_case(case, limits):
    # Check logical arithmetic and bounded metadata complexity.
    ...


def construct_bounded_meta_model(case):
    # Never create real parameter payload in the oversized path.
    ...


def E04(config):
    # Scale meta cases and report measured overhead without real OOM tests.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require an audit that all required oversized parameter payloads remain meta. A constructor interrupted by its resource limit is a failure or incomplete case, not proof of small memory. Report host overhead relative to logical bytes without calling it literally zero. Avoid checksums, cloning to real devices, or value-dependent probes that accidentally materialize or invalidate the experiment.

Part 7 - Engineering Interpretation and Philosophy

A large model can be describable before it is affordable to instantiate with values. Use that capability to reject impossible placement plans early, not to imply that a logical billion-parameter object is already an executable or trainable resident model.

19.31.6 Exercise 05

Exercise Title: Calculate the Future Parameter Memory Footprint of a Meta Model Before Materializing Any Parameter Storage

Part 1 - Exercise Contract and Why It Matters

Calculate future parameter payload from a meta model before creating real parameter storage. Deliver named logical totals, canonical unique-parameter totals, buffer totals, and declared layout assumptions. Distinguish an estimate for dense weights from a complete training-memory budget.

Part 2 - System and Memory Mental Model

Shape and target dtype determine nominal dense payload, but names can refer to one tied parameter. Distinct parameter objects can also share regions under an explicitly declared architecture. Meta addresses cannot serve as real storage identity evidence. Use architectural ties and canonical ownership records rather than grouping every meta tensor by a nonexistent data address. Target conversion, padding, quantization metadata, sparse layouts, and future optimizer state require separate models.

Part 3 - Quantitative Prediction Before Execution

For canonical dense parameters p with shape dimensions $h_{p,k}$, compute $B_{\text{parameters}} = \sum_p s_p \prod_k h_{p,k}$. Calculate a named sum as a separate diagnostic and explain the gap from tied names. Add

registered buffers without merging them into parameter count. Compare target float32 and bfloat16 plans, then identify exclusions such as allocator rounding, activations, gradients, master weights, and optimizer state.

Part 4 - Experimental Design and Instrumentation

Create a small architecture containing an ordinary parameter, a tied parameter with two names, and a buffer. Enumerate with duplicate handling explicit and build a canonical alias manifest. Check the formulas against a tiny real model with the same declared ties. Test integer, floating, and frozen parameter metadata without assuming all buffers use the default parameter dtype. Export bytes and MiB/GiB with conversion units stated. Reject unsupported sparse or quantized accounting rather than silently applying dense element-width formulas.

For one Parameter object registered under several names, preserve every name while counting its canonical payload once. For distinct shared views, require an explicit storage/region manifest rather than assuming object-identity deduplication completely captures physical sharing.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class ParameterEstimate:
    canonical_key: str
    names: tuple
    shape: tuple
    source_dtype: object
    target_dtype: object
    nominal_bytes: int


def canonicalize_parameter_names(model, architecture_aliases):
    # Preserve true ties without treating meta pointers as identities.
    ...


def estimate_dense_payload(records, target_policy):
    # Compute target bytes and retain excluded/unsupported categories.
    ...


def E05(config):
    # Validate tied and untied estimates against a tiny real model.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require exact arithmetic for the supported dense case and a traceable mapping from each name to canonical ownership. A nominal total is not a physical peak or driver allocation prediction. Detect incompatible target-device or dtype requests for a single tied group. Report whether frozen weights still occupy storage and why the absence of gradients changes training additions rather than removing the weight payload itself.

Part 7 - Engineering Interpretation and Philosophy

Planning improves when totals preserve ownership and scope. The useful number is not the largest impressive parameter count, but an auditable payload estimate that states what is counted once, what remains separate, and which execution costs have not yet been budgeted.

19.31.7 Exercise 06

Exercise Title: Traverse a Meta Model and Build a Layer-by-Layer Memory Plan from Parameter Shapes, Dtypes, and Target Devices

Part 1 - Exercise Contract and Why It Matters

Traverse a meta model and produce a layer-level placement and memory plan before materialization. Map every required parameter and buffer to a target device, dtype policy, canonical owner, and loading group. Include transfer boundaries and headroom rather than optimizing weight bytes in isolation.

Part 2 - System and Memory Mental Model

Module hierarchy is not always a unique ownership tree: modules and parameters can be shared across paths. Layer-level device choices must therefore agree with canonical tied groups. Buffers and inputs must be compatible with the operations that use them. A mixed CPU/CUDA assignment does not automatically insert activation transfers or provide a dynamic execution engine. Keep the placement description separate from the mechanism that will execute it.

Part 3 - Quantitative Prediction Before Execution

For each target device d, calculate canonical planned payload $B_d = \sum_{g:\text{place}(g)=d} B_g$. Add known buffers and named margins for staging, activations, and libraries. Draw a sequential model split between CPU and CUDA and count activation transfer boundaries. Introduce a tied group spanning two layers and predict why conflicting placements require rejection or an explicitly changed replication contract.

Part 4 - Experimental Design and Instrumentation

Freeze a small architecture and build a complete name-to-owner manifest. Traverse parameters and buffers without dropping shared paths, then assign canonical groups. Validate device/dtype compatibility, budgets, and expected execution transfers in a host simulator. Compare predicted payloads with a tiny materialized plan and verify output parity using explicit transfers. Include missing-map entries, double-counted shared weights, and unsupported custom modules as negative controls. Record plan and architecture version IDs so later loading cannot silently use stale shapes or ownership.

Record whether placement groups are execution layers or larger canonical tie groups. They need not coincide. A cross-layer shared weight can force several layer names to follow one ownership decision even when their other parameters are placed independently.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class PlacementGroup:
    group_id: str
    state_names: tuple
    canonical_owners: tuple
    target_device: object
    target_dtype: object
    predicted_payload_bytes: int


def build_layer_inventory(model, alias_manifest):
    # Include buffers, shared paths, and execution boundary requirements.
    ...


def validate_placement(groups, budgets, execution_contract):
    # Check coverage, ties, dtypes, and target headroom before allocation.
    ...


def E06(config):
    # Compare host predictions with a bounded materialized reference.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Every required state entry needs a target or an explicit deferred status. Reject conflicting tied-owner placements and distinguish unsupported execution from a valid storage map. A lower GPU weight total is not proof the next forward fits. Preserve modeled and unmodeled headroom separately; do not interpret a software weight budget as a guarantee about external device memory pressure.

Part 7 - Engineering Interpretation and Philosophy

A placement map is a resource contract for a concrete execution plan. It becomes useful when ownership, required values, transfer boundaries, and headroom are all visible before the loader commits real storage.

19.31.8 Exercise 07

Exercise Title: Reproduce the Failure of Trying to Copy Data Directly Out of a Meta Tensor and Explain Why Meta Tensors Have No Underlying Values

Part 1 - Exercise Contract and Why It Matters

Reproduce failure when attempting to copy values directly from a meta tensor to CPU or CUDA. Explain the difference between moving known contents and creating new storage with newly supplied contents. Provide legal alternatives without pretending that meta had retained original random or checkpoint data.

Part 2 - System and Memory Mental Model

A meta tensor describes possible data but contains no element values to copy. A factory can allocate a compatible real tensor, and module materialization can create uninitialized real state, but neither operation reconstructs missing weights. Loading a checkpoint with meta mapping can inspect structure while also discarding access to real loaded values in that object. Keep the original checkpoint source available when later materialization needs authoritative contents.

Part 3 - Quantitative Prediction Before Execution

Predict the outcomes of metadata inspection, direct conversion to CPU, conversion to CUDA, a compatible real factory, and initialization from a separately defined rule. For each, state whether it preserves existing values, creates unspecified values, or supplies valid new values. Calculate target payload from shape and dtype without labeling it transferred data. Define the expected failure phase and distinguish it from CUDA unavailability or OOM.

Part 4 - Experimental Design and Instrumentation

Use tiny ordinary meta tensors and one small real reference. Isolate each conversion probe with a specific expected outcome and preserve the actual exception category/message. Complete a legal real-allocation path and then initialize or load it from an explicit authority before numerical comparison. Test scalar extraction separately from device conversion. Avoid catching every exception as the intended no-data failure. If CUDA is unavailable, retain the CPU reproduction and mark that CUDA case untested rather than fabricating it.

If the source was itself loaded with a meta map location, retain the checkpoint path as the authority for a later real load. The meta object is a structural representation, not a lazy real-valued source that ordinary copying can recover.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class ConversionOutcome:
    operation: str
    expected_status: str
    observed_status: str
    value_authority: str
    failure_phase: object
```

```
def probe_meta_conversion(meta_tensor, target, support):
    # Capture the specific no-data outcome separately from other errors.
    ...


def allocate_and_supply_values(metadata, target, authority):
    # Distinguish storage creation from initialization/checkpoint data.
    ...


def E07(config):
    # Compare forbidden copies and valid authority-backed alternatives.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Acceptance requires the intended no-values failure in the supported direct-copy case and correct values only after a declared authority supplies them. A real empty tensor is not equivalent to a previously initialized model. Do not replace the source with arbitrary zeros and claim successful copying. Preserve shape and dtype agreement as structural evidence, not proof that unknown weights were recovered.

Part 7 - Engineering Interpretation and Philosophy

A failed copy reveals a missing information source rather than merely a missing memory block. The repair must answer where valid contents come from, then allocate storage and publish those contents under an explicit readiness contract.

19.31.9 Exercise 08

Exercise Title: Materialize a Meta Module with Module.to_empty() and Initialize Its Real Parameters Safely on CPU or CUDA

Part 1 - Exercise Contract and Why It Matters

Materialize a small supported meta module with the module empty-materialization API, then supply valid parameters and buffers before execution. Demonstrate that allocated storage and initialized values are different states. Preserve shared parameter relationships and construct optimizers only after final parameter identity and placement are established.

Part 2 - System and Memory Mental Model

Empty materialization creates device storage without copying nonexistent meta values. The resulting contents must be initialized or loaded, including required buffers that are not trainable parameters. A generic recursive initialization pass can initialize shared objects repeatedly or miss module-specific state. Use a declared per-module initialization policy and canonical ownership ledger. Avoid applying whole-model empty materialization after some weights have already been loaded, because it can replace valid state with uninitialized storage.

Part 3 - Quantitative Prediction Before Execution

Predict target parameter and buffer bytes, initial host metadata, and allocation peaks. Define readiness transitions from meta to allocated-uninitialized to initialized-verified. For a

tied parameter, calculate capacity once and predict what verification is needed after conversion. Specify a deterministic initialization rule for a tiny model so expected values and outputs can be computed without relying on an assumed unchanged random-number consumption order.

Part 4 - Experimental Design and Instrumentation

Audit the meta skeleton, materialize once on a chosen CPU or CUDA target, and mark every required entry unready. Complete an initialization pass covering parameters, persistent buffers, and nonpersistent execution buffers. Verify values, dtypes, devices, and ties before a tiny forward. Compare with an independent model initialized by the same declared rule. Test a deliberately omitted buffer and require readiness rejection before execution. Measure allocation and initialization as separate phases.

Define initialization callbacks for the supported architecture explicitly. Not every container implements a reset method, and blindly applying callbacks at both parent and child levels can repeat initialization. Verify canonical values and required buffer contents after the pass.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
from torch import nn

@dataclass
class StateReadiness:
    canonical_key: str
    storage_allocated: bool
    values_supplied: bool
    source: str
    verified: bool


def materialize_empty(model, target, ownership_manifest):
    # Allocate supported state once and invalidate value readiness.
    ...


def initialize_required_state(model, policy, readiness):
    # Initialize each canonical parameter and every required buffer.
    ...


def E08(config):
    # Verify contents/ties before forwarding or creating an optimizer.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Never validate uninitialized memory by checking whether it happens to look finite. Require positive evidence that every entry received its intended values. Re-audit tied identity and shared storage after conversion rather than assuming preservation. An initialization routine unsupported by the architecture must fail explicitly. A successful allocation is not a successful numerical model construction until readiness and reference checks pass.

Part 7 - Engineering Interpretation and Philosophy

Materialization is a controlled resource transition, not value recovery. An explicit readiness ledger prevents accidental execution on garbage contents and makes it clear whether weights came from initialization, a checkpoint, or another declared authority.

19.31.10 Exercise 09

Exercise Title: Compare Ordinary Model Initialization Against Meta Construction Followed by to_empty() by Peak RAM, Peak VRAM, and Initialization Time

Part 1 - Exercise Contract and Why It Matters

Compare ordinary real model construction and initialization with meta construction followed by empty materialization and the same valid initialization. Measure complete readiness time, peak host memory, peak VRAM, and final state. Do not confuse skipping useful initialization with making initialization cheaper.

Part 2 - System and Memory Mental Model

Meta can avoid a real default initialization that would otherwise be replaced, but a newly initialized model still needs valid values eventually. Constructors may perform architecture work independent of payload. Allocation, page touching, CUDA setup, and initialization each influence measured peaks and latency. Equal random seeds alone do not guarantee equal initialized values if the two paths consume random streams differently; define the equivalence policy explicitly.

Part 3 - Quantitative Prediction Before Execution

Draw phase timelines for direct construction and deferred construction. Calculate final dense payload and identify any avoidable temporary or duplicate initialization. Predict a case with little improvement because both routes must initialize the same final weights, and a case where an ordinary temporary CPU model precedes CUDA placement. Mark the latter as a separate baseline rather than silently comparing it with direct CUDA construction.

Part 4 - Experimental Design and Instrumentation

Use a supported transparent architecture and a deterministic initialization rule for the primary comparison. Run isolated trials from matching process and CUDA context states. Record construction return, allocation completion, initialization completion, verification, and first-ready-forward boundaries. Touch real data through legitimate initialization, not arbitrary benchmark-only reads. Compare final values, buffers, and ties before reporting performance. Repeat cold and warmed trials, stating whether checkpoints, page cache, constructor defaults, and validation are part of the experiment.

Use a separate numerical equivalence run when verifying every tensor would change the timing region. If verification is part of readiness, include that cost consistently in both strategies. The benchmark must state which boundary it actually measures.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class InitializationTrial:
    strategy: str
    architecture_id: str
    initialization_policy: str
    peak_rss_bytes: object
    peak_cuda_allocated: object
    complete_ready_seconds: object
    equivalent: bool


def run_initialization_strategy(factory, strategy, policy, config):
    # Measure all phases through verified real values and readiness.
    ...


def compare_ready_models(reference, candidate, contract):
    # Check values, buffers, ownership, and supported outputs.
    ...


def E09(config):
    # Compare equivalent complete initialization, including adverse cases.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

A timer ending before value initialization is not comparable with a fully initialized baseline. Report default initialization skipped and final initialization performed separately. Preserve CPU high-water uncertainty and CUDA context overhead. If numerical initialization differs, classify the comparison accordingly instead of presenting a memory benefit as evidence of exact model equivalence. Retain neutral or slower deferred-construction results.

Part 7 - Engineering Interpretation and Philosophy

Deferred construction is most valuable when it removes work that the final state does not need. The honest benchmark follows both paths to the same verified readiness boundary and explains whether it saved payload overlap, initialization work, or neither.

19.31.11 Exercise 10

Exercise Title: Load a Checkpoint with torch.load(..., mmap=True, weights_only=True) While Avoiding an Eager Full-RAM Copy of Tensor Storage

Part 1 - Exercise Contract and Why It Matters

Load a locally generated, supported checkpoint using memory mapping and the restricted weights-only path. Compare mapping, first value access, complete value access, and ordinary eager loading. Demonstrate avoidance of an eager full payload copy without equating mapping with zero RAM use or zero disk I/O.

Part 2 - System and Memory Mental Model

Mapping makes supported serialized storage accessible through virtual memory; physical pages become resident as touched and managed by the operating system. Tensor dictionaries and metadata still deserialize. Use an explicit CPU map location so saved CUDA tags do not trigger unintended GPU placement. The weights-only flag restricts deserialization; it does not mean only trainable parameters are returned or that all metadata and buffers disappear. A standard path-based compatible checkpoint format is part of the initial contract.

Part 3 - Quantitative Prediction Before Execution

Let B be checkpoint storage payload and B_t the subset accessed so far. Predict mapped virtual space, metadata cost, file-backed residency, anonymous copy demand, and page-cache effects separately. Compare mapping return with a full-read checksum phase. Predict how dtype conversion or a GPU transfer can create new allocations and force source pages to be read. Do not call reduced return latency a completed full-model I/O speedup.

Part 4 - Experimental Design and Instrumentation

Generate a small trusted plain tensor state dictionary outside the measured region and save it with a supported current serialization format. Feature-check path-based mapping support and record platform/filesystem details. Complete ordinary and mapped loads with explicit CPU placement. Measure baseline, load return, bounded subset access, complete data access, and release phases. Keep file-cache state described as observed or uncontrolled rather than claiming a cold run without evidence. Test unsupported file-object or format cases through documented rejection, not guessed compatibility.

Keep the backing checkpoint immutable for the initial mapping experiment. A separate controlled mutation test should record mapping policy and any private page growth; it must not overwrite the evidence source or assume changes are written back automatically.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class MappingPhase:
    phase: str
    virtual_bytes: object
    rss_bytes: object
    anonymous_bytes: object
    file_backed_bytes: object
    elapsed_seconds: object
```

```
def load_checkpoint_mapping(path, support_contract):
    # Use explicit mmap, weights_only, and CPU map-location choices.
    ...


def measure_checkpoint_access(state, access_schedule, config):
    # Separate mapping return from actual payload touch and conversion.
    ...


def E10(config):
    # Compare phase-matched eager and mapped runs with valid files.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate tensor keys, shapes, dtypes, and actual values on the bounded case. An unsupported mapping option must not silently fall back to eager loading inside the mapped benchmark. Deleting a local variable does not guarantee mapping release while tensor aliases survive. Record private-copy behavior if mutation is tested under the supported platform's default mapping policy; do not assume mapped training updates are free or persisted to the file.

Part 7 - Engineering Interpretation and Philosophy

Memory mapping changes when storage is read and copied, not whether useful values eventually require physical resources. A good result distinguishes lazy access, reduced anonymous duplication, page residency, and complete readiness cost.

19.31.12 Exercise 11

Exercise Title: Load Memory-Mapped Checkpoint Tensors into a Meta-Initialized Model with load_state_dict(assign=True)

Part 1 - Exercise Contract and Why It Matters

Load a mapped real checkpoint into a meta skeleton using assignment semantics. Verify values, device/dtype properties, remaining meta state, parameter identity, and tied relationships. Establish when the model becomes ready and create the optimizer only after its final parameter set and placement are settled.

Part 2 - System and Memory Mental Model

Ordinary state loading generally copies into existing destinations; copying into meta does not supply real resident values. Assignment can install the state dictionary's real tensor properties instead, while parameter gradient requirements follow the module contract. CPU-mapped weights do not become CUDA weights merely because the skeleton's intended execution target was CUDA. Shared references can keep the mapped storage alive after the source dictionary is removed. Architecture ties need explicit verification.

Part 3 - Quantitative Prediction Before Execution

Predict the state of a CPU-mapped parameter before and after assignment into meta. Compare it with copying into a preallocated real destination and with ordinary loading into meta. Calculate payload overlap under each ownership model, avoiding double counting a model and dictionary sharing the same backing storage. Predict which registered nonpersistent buffers remain meta and why checkpoint assignment alone cannot make those buffers ready.

Part 4 - Experimental Design and Instrumentation

Create matching tiny source and meta architectures with persistent and nonpersistent state plus a declared tie. Load the supported checkpoint on CPU with mapping, audit keys and shapes, and complete assignment. Inspect missing and unexpected keys rather than using non-strict loading as blanket acceptance. Supply any required noncheckpoint state, verify ties and real values, finalize placement, and only then construct an optimizer if training is tested. Remove source references and verify model values remain valid without claiming their storage was freed.

Record model-state properties after assignment rather than only before it. The skeleton's requested dtype is not a conversion instruction for assigned checkpoint tensors. Any required target cast or transfer needs its own explicit operation, memory estimate, and validation.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class AssignmentAudit:
    missing_keys: tuple
    unexpected_keys: tuple
    remaining_meta_names: tuple
    incorrect_devices: tuple
    broken_ties: tuple
    values_verified: bool


def assign_mapped_state(model, mapped_state, contract):
    # Validate schema, apply assign semantics, and audit final state.
    ...


def finish_noncheckpoint_state(model, initialization_policy):
    # Supply required nonpersistent buffers and other declared values.
    ...


def E11(config):
    # Verify readiness/ownership before optional optimizer creation.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

A successful key-match message is not sufficient readiness evidence. Check all execution-required state, final device/dtype, and parameter references. Do not call empty materialization after assignment as a way to move valid checkpoint weights; it does not preserve their contents. If tying must be reestablished, use a declared canonical binding and validate values and gradient behavior before constructing the optimizer.

Part 7 - Engineering Interpretation and Philosophy

Assignment is an ownership transition, not a promise of a new independent copy or automatic target-device placement. Its benefit comes from avoiding unnecessary duplicate destinations while making the final model's actual values and references explicit.

19.31.13 Exercise 12

Exercise Title: Compare Conventional Checkpoint Loading Against mmap=True + meta + assign=True and Quantify Peak Host-Memory Reduction

Part 1 - Exercise Contract and Why It Matters

Compare conventional checkpoint loading with the combined mapped-checkpoint, meta-skeleton, and assignment path. Quantify host memory, VRAM, and complete ready-to-use latency at matching workload boundaries. Report reduced duplication separately from differences in page residency or lazy return time.

Part 2 - System and Memory Mental Model

A conventional path can hold initialized model payload and checkpoint payload at once before copying. The combined path can avoid that duplicate destination when it adopts compatible real checkpoint storage. File-backed pages still consume resources as accessed, and GPU placement creates device storage and reads source pages. Model, state dictionary, and mapped file may share owners; count unique backing rather than adding every named tensor's bytes.

Part 3 - Quantitative Prediction Before Execution

Draw phase peaks for construct/load/copy and map/meta/assign. Estimate a nominal two-payload overlap only under explicitly independent dense model and checkpoint storage assumptions. Add target conversion, buffers, and GPU transfer staging where used. Predict CPU-resident and GPU-ready final variants separately. Calculate memory reduction at a specified boundary and explain why mapping-only RSS and first-complete-forward RSS can differ substantially.

Part 4 - Experimental Design and Instrumentation

Use identical architecture, checkpoint, final dtype/device, execution mode, and required buffers. Run isolated process trials with recorded baseline and cache context. Verify keys, values, ties, and a real forward before accepting readiness. Measure mapping return, assignment/copy completion, target placement, and first useful execution separately. Capture supported host high-water metrics and CUDA peaks, with absolute values rather than hidden postallocation deltas. Repeat trial order and retain neutral or adverse results.

Compare peak measurements in isolated processes when earlier allocations or mappings would otherwise contaminate the next strategy. Resetting a CUDA peak counter does not reset process RSS history or the operating system file cache, so preserve those distinctions.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class LoadingTrial:
    strategy: str
    final_device_policy: str
    checkpoint_id: str
    peak_host_bytes: object
    peak_cuda_bytes: object
    ready_seconds: object
    first_forward_seconds: object
    equivalent: bool


def conventional_load(factory, checkpoint_path, target, config):
    # Include the declared real constructor and loading overlap.
    ...


def mapped_meta_assign_load(factory, checkpoint_path, target, config):
    # Include all phases through the same verified readiness boundary.
    ...


def E12(config):
    # Compare equivalent resident models, not only load-return times.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Fail comparisons where one model remains meta, uninitialized, on another device, or missing buffers. A lower RSS can reflect untouched file pages rather than permanently removed resource demand; explain the access schedule. If host instrumentation lacks a true high-water metric, label sampled estimates. Do not claim complete-model reduction from a tiny inspected subset or omit final conversion cost for only one strategy.

Part 7 - Engineering Interpretation and Philosophy

The strongest comparison follows equivalent data to equivalent readiness. Savings should be attributable to removed duplicate storage or work, with laziness and cache effects described separately so the conclusion survives real model use.

19.31.14 Exercise 13

Exercise Title: Reproduce an Optimizer Reference Bug Caused by Creating the Optimizer Before load_state_dict(assign=True) and Repair the Initialization Order

Part 1 - Exercise Contract and Why It Matters

Reproduce a stale optimizer-reference bug by creating the optimizer before assignment-based loading under a configuration that replaces parameter objects. Repair the order and verify that the optimizer updates the model's current parameters. Distinguish version/configuration-dependent reference preservation from a failed reproduction.

Part 2 - System and Memory Mental Model

An optimizer holds parameter references and may also hold state indexed by those parameters. Assignment or later conversion can invalidate an optimizer created too early. Supported swap-based behavior can preserve references in some configurations, so record the actual future-flag and software version. The robust initial protocol is load, finalize dtype/device and ties, verify state, then construct the optimizer. Identity mismatch and absent model updates provide stronger evidence than a step call returning normally.

Part 3 - Quantitative Prediction Before Execution

List parameter identities held by the model and optimizer before loading, after assignment, and after final placement. Predict gradients and updates for a stale optimizer versus a correctly constructed one. Define an equality check on the current parameter set and a tiny loss whose nonzero gradients should produce a detectable update. Include optimizer state bytes separately from weight payload and distinguish state initialization from model loading.

Part 4 - Experimental Design and Instrumentation

Use a tiny controlled checkpoint and fresh model per trial. Explicitly select and record a supported replacement configuration for the main reproduction, restoring global configuration afterward. Construct the early optimizer, assign the state, run forward/backward, and inspect which objects receive gradients and which objects the optimizer owns. Repeat with final-order construction and compare updates. Add a reference-preserving configuration as a distinct control when available; if the bug does not reproduce, retain that observed outcome rather than inventing stale references.

For the repaired case, inspect optimizer-owned references after final placement and retie operations, not just immediately after loading. A later parameter replacement can recreate the same class of bug even when the initial loading order was correct.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class OptimizerReferenceAudit:
    model_parameter_ids: tuple
    optimizer_parameter_ids: tuple
    missing_current_parameters: tuple
    stale_parameters: tuple
    current_update_verified: bool
```

```
def audit_optimizer_references(model, optimizer):
    # Compare live parameter identity sets and declared tied ownership.
    ...


def run_optimizer_order_case(factory, state, order, swap_policy):
    # Build fresh state, load/place, and verify actual gradients/updates.
    ...


def E13(config):
    # Compare early creation, repaired order, and supported controls.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not treat a silent optimizer step as proof that model weights changed. Check named current parameters and expected updates. Rebinding optimizer parameter groups alone may leave old momentum or other state inconsistent; use fresh construction for the primary repair. Record whether a later device conversion changes identity. The reference audit must occur after all loading, retie, and placement operations that can alter parameter ownership.

Part 7 - Engineering Interpretation and Philosophy

Initialization order is part of correctness whenever components retain object references. The repair is to publish a stable final parameter set before building dependent optimizer state, with explicit tests for the actual runtime configuration.

19.31.15 Exercise 14

Exercise Title: Materialize Different Meta Model Layers Directly onto CPU and CUDA According to a Precomputed Memory Placement Map

Part 1 - Exercise Contract and Why It Matters

Materialize distinct meta model groups directly on planned CPU or CUDA targets without first creating the complete real model on one device. Preserve valid contents, buffers, canonical ties, and a complete name-to-location audit. Keep storage placement separate from the transfers needed for execution.

Part 2 - System and Memory Mental Model

A whole-model recursive materialization can overwrite or relocate already loaded state, so selective work must respect owning modules and recurse policy. For supported independent layer groups, allocate only that group's required state, then initialize or copy authoritative values. Cross-group ties need one canonical storage contract and compatible placement. Assigning a CPU checkpoint tensor to a meta parameter installs CPU state unless a target conversion was explicitly performed; a placement map alone changes no tensor. Choose target dtypes while state is still meta when the supported architecture permits it, so a later real-to-real cast does not introduce an avoidable full group copy.

Part 3 - Quantitative Prediction Before Execution

Calculate target payload per canonical group and device. Include the largest overlap of live source values, conversion workspace, and target destinations. Draw a three-layer CPU/CUDA split with explicit activation transfers. Compare direct group placement with an unnecessary full CPU model followed by transfer. Add a tie spanning proposed devices and predict refusal unless the model's semantics explicitly permit replication and its consistency obligations are separately implemented.

Part 4 - Experimental Design and Instrumentation

Validate the complete map before allocating. Complete one supported group's allocation and value supply at a time, avoiding recursive actions on already ready descendants. Audit parameters, persistent buffers, and nonpersistent execution state after each group. Verify final ties and values against a tiny ordinary reference. Execute with explicit activation transfer boundaries and record their cost. Test missing entries, conflicting ties, and a midload failure that must not publish a partially ready model.

Before materializing a group, check that its ownership boundary does not recurse into an already loaded child. Use the actual module API's supported recurse behavior and an explicit group manifest rather than relying on incidental attribute traversal order.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class MaterializationGroup:
    group_id: str
    owner_names: tuple
    target_device: object
    target_dtype: object
    source_keys: tuple
    expected_bytes: int


def materialize_group(model, group, authority, ownership):
    # Allocate only intended owners and supply valid target values.
    ...


def verify_group_placement(model, group, readiness):
    # Check buffers, ties, dtype/device, and authoritative contents.
    ...


def E14(config):
    # Validate the map, load groups, and execute a tiny mixed-device case.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require every ready group to remain valid after later groups are processed. Detect accidentally recursive materialization and changed tied bindings. A mixed-device forward failure is not necessarily a loading error; inspect input and activation placement. Record unavailable CUDA as unsupported or untested, not successful placement. A final CPU weight sharing mapped source storage remains an owner even after loader-local references are dropped.

Part 7 - Engineering Interpretation and Philosophy

Direct placement removes unnecessary intermediate ownership only when the loader follows the canonical map precisely. Its result should be a verified resident state with explicit execution boundaries, not merely a collection of tensors distributed across devices.

19.31.16 Exercise 15

Exercise Title: Load and Materialize Model Parameters One Layer at a Time Instead of Creating the Entire Real Model in Memory Simultaneously

Part 1 - Exercise Contract and Why It Matters

Load real state into a meta skeleton one supported layer or ownership group at a time. Bound temporary source/conversion demand and preserve final model values. Distinguish construction-time incremental loading from execution-time offloading: final resident weights still accumulate when every layer remains loaded.

Part 2 - System and Memory Mental Model

Incremental materialization can avoid a duplicate full real destination model, but it does not make retained final payload disappear. A standard mapped checkpoint exposes a full metadata dictionary while allowing selected data access; it is not automatically a per-key file-stream API. Sharded checkpoints provide another explicit access model. Tied groups may cross layer boundaries and require canonical treatment larger than one syntactic layer. Source references can be released only when no retained destination or pending copy still depends on them.

Part 3 - Quantitative Prediction Before Execution

Let F_i be final payload retained after group i and T_i active temporary source/conversion capacity. Predict peak as a timeline involving $F_i + T_i$, not merely the largest layer size. Add staging and library margins separately. Compare a mapped source adopted by a CPU destination with a new converted GPU destination. Explain why deleting a source dictionary entry can remove a reference without reducing final owned CPU payload.

Part 4 - Experimental Design and Instrumentation

Create a small checkpoint with several groups and a declared tie. Freeze the source format, ordering, target map, and readiness protocol. Validate names, shapes, dtypes, and aliases before publishing values. Complete each group's materialization, verification, dependency completion, and source-reference release. Compare final values with an ordinary loader and track phase peaks. Inject a missing key and a controlled budget refusal, requiring incomplete state to remain unpublished. Record group order and failure phase for replay.

Include a source dictionary reference audit in the small case. Removing one entry can leave another tied name or local alias owning the same storage. Reference release is not equivalent to immediate physical page eviction or reduction of final model bytes.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class GroupLoadState:
    group_id: str
    canonical_keys: tuple
    target_device: object
    source_status: str
    destination_status: str
    ready: bool


def load_one_group(model, source, group, placement, ledger):
    # Bound active conversion and preserve canonical ownership.
    ...


def release_group_sources(group_state, completion, retained_owners):
    # Release references only after all actual obligations finish.
    ...


def E15(config):
    # Compare final state and accumulated payload/temporary peaks.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not claim a whole resident model fits within the largest single layer's bytes. Verify all loaded parameters and required buffers, including cross-group ties. A mapped tensor assigned to the model keeps source storage alive; report that ownership instead of declaring it freed. Partial non-strict loads must reconcile intentional deferred names with accidental missing names before final readiness is granted.

Part 7 - Engineering Interpretation and Philosophy

Layerwise loading controls transient overlap, while final residency follows the placement policy. The distinction prevents a useful construction technique from being misrepresented as a complete out-of-core execution system.

19.31.17 Exercise 16

Exercise Title: Combine Meta Construction with Dynamic Parameter Placement to Build a Model Whose Initial Residency Is Determined by a Fixed VRAM Budget

Part 1 - Exercise Contract and Why It Matters

Combine a meta skeleton with budget-aware initial parameter placement. Determine which canonical groups become GPU-resident, CPU-resident, or explicitly deferred, then load only admitted storage. Define readiness for the supported execution mode and distinguish an initial VRAM plan from a future runtime migration policy.

Part 2 - System and Memory Mental Model

GPU weight capacity is only part of useful execution demand. Inputs, activations, operator workspace, buffers, transfer staging, and external pressure also need headroom. CPU fallback consumes real host resources; deferred meta weights are not executable values. A runtime that later promotes groups needs an explicit authority, completion protocol, and execution schedule. This exercise establishes initial residency and those interfaces without pretending the placement map alone implements dynamic inference or training.

Part 3 - Quantitative Prediction Before Execution

For GPU budget V, choose group placement satisfying $B_{\text{GPU weights}} + B_{\text{GPU buffers}} + H \leq V$, with named headroom H. Compute CPU and deferred totals separately. Predict transient demand when one group is converted while earlier groups remain resident. Compare static group order and a workload-informed priority under the same budget. Reject a target below mandatory state rather than reporting a feasible plan that omits required work.

Part 4 - Experimental Design and Instrumentation

Preflight architecture, canonical ties, source availability, and both host/GPU budgets. Simulate admission before allocating and retain a decision ledger. Load accepted groups directly to their target locations, verifying actual payload and readiness after each step. Test budgets above and below a group boundary, unavailable CUDA, a tied-placement conflict, and a controlled allocation failure. Run a small supported execution schedule only after its needed groups are ready and explicit transfers are established. Record predicted and observed peaks without calling external memory availability guaranteed.

Attach an authority and readiness status to every deferred group. A group that remains meta cannot participate in an ordinary forward. The initial plan must either provide a supported promotion schedule or report that execution mode as unavailable.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class InitialResidency:
    group_id: str
    state: str
    device: object
    payload_bytes: int
    authority: str
    ready: bool
```

```
def plan_initial_residency(groups, budgets, execution_headroom):
    # Admit canonical groups without hiding CPU or deferred demand.
    ...


def execute_residency_plan(model, source, plan, ledger):
    # Materialize validated placements and preserve failure consistency.
    ...


def E16(config):
    # Test budget boundaries and verify the supported readiness contract.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

A plan meeting the weight budget can still fail the first forward; preserve that distinction. CPU-resident groups must have valid data, and deferred meta groups must remain blocked from ordinary execution. Account for partial-load cleanup after failure without damaging previously authoritative sources. Report admission refusal separately from CUDA OOM and unrelated runtime errors. Training state requires a further budget beyond the initial weight placement.

Part 7 - Engineering Interpretation and Philosophy

A fixed budget is useful when it governs real commitments and states what remains outside them. Initial placement should leave a coherent authority and readiness model so later dynamic movement can be added without inventing missing values or capacity.

19.31.18 Exercise 17

Exercise Title: Build a Streaming Checkpoint Loader That Reads, Materializes, Places, and Releases Parameter Groups Without Holding Duplicate Full-Model Copies

Part 1 - Exercise Contract and Why It Matters

Build a bounded checkpoint loader that processes validated parameter groups, converts or copies them to planned targets, and releases transient references when safe. Avoid duplicate full-model payload while preserving canonical ties, complete value verification, and explainable failure outcomes.

Part 2 - System and Memory Mental Model

Streaming can mean lazy access through one mapped checkpoint or bounded access to explicit shard files. State which source interface is implemented; repeatedly calling ordinary full-file loading for each layer is not a bounded per-group reader. Final CPU tensors may intentionally retain mapped source storage. Converted destinations and asynchronous transfers need separate lifetime rules. A bounded queue must charge active source owners, pinned staging, conversion buffers, and destination reservations rather than only counting group objects.

Part 3 - Quantitative Prediction Before Execution

For final retained payload $F(t)$, in-flight group set $Q(t)$, and transient demand $T_g(t)$, predict a host/device timeline from $F(t)$ and the relevant $\sum_{g \in Q(t)} T_g(t)$. Avoid double counting a mapped source shared with its CPU destination. Compare queue depths one and two and calculate transition headroom. Include file metadata, storage aliases, and whole-shard granularity when one group access keeps more source storage alive than its logical slice.

Part 4 - Experimental Design and Instrumentation

Generate trusted small checkpoints in the declared format and a complete key-to-group manifest. Validate schemas and aliases before target publication. Implement bounded source access, admission, conversion/transfer, completion, verification, and release. Compare with a normal loader on final values and first useful execution. Test missing keys, shape mismatch, controlled read failure, duplicate group requests, and cancellation. Capture exact source IDs, group ordering, in-flight capacity, final ownership, and failure phase for replay. Do not remove backing files while live consumers still rely on their mapped storage.

Keep group publication atomic at the logical readiness boundary: readers see either a verified completed group or an unavailable one. For asynchronous copies, release source references only after the relevant completion obligation is satisfied, even if host submission returned.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class GroupTicket:
    group_id: str
    source_owner_ids: tuple
    reserved_host_bytes: int
    reserved_device_bytes: int
    completion_token: object
    status: str

class StreamingCheckpointLoader:
    def open_source(self, path, format_contract):
        # Establish mapped or sharded access with bounded metadata rules.
        ...

    def admit(self, group, budgets):
        # Reserve source, staging, conversion, and destination capacity.
        ...

    def materialize_and_verify(self, ticket, model, placement):
        # Supply values and publish only verified completed groups.
        ...

    def release_transients(self, ticket, retained_owners):
        # Keep references required by final CPU state or pending copies.
```

```
        ...

    def shutdown(self, deadline):
        # Stop admission, finish supported work, and report leftovers.
        ...


def E17(config):
    # Compare queue bounds, correctness, failures, and complete loading cost.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Acceptance requires complete key coverage, correct final contents, valid ties, and no false release of storage still used by a destination. A small queue does not prove a small peak if each shard or conversion is large. Report mapped and anonymous residency separately where instruments support it. Preserve partial/cancelled loads as unavailable rather than publishing a model with mixed valid and uninitialized state.

Part 7 - Engineering Interpretation and Philosophy

Streaming is a lifetime and admission discipline over a concrete source format. Its success is removing unnecessary overlap while retaining the final state the application needs, with bounded in-flight commitments and honest ownership after each release.

19.31.19 Exercise 18

Exercise Title: Stress-Test Zero-Allocation Construction with an Artificial Model Containing Billions of Logical Parameters and Verify That Construction Memory Remains Small

Part 1 - Exercise Contract and Why It Matters

Stress-test meta construction with billions of logical parameters while bounding actual host metadata and prohibiting real payload materialization. Measure scaling with shape and with object count separately. Demonstrate that construction can remain small relative to hypothetical weight bytes without claiming constant or zero process memory.

Part 2 - System and Memory Mental Model

A huge dimension on a few dense weights can represent billions of elements with modest object overhead. Increasing module count or tensor count grows Python and framework metadata. Constructor escapes, unsupported meta kernels, integer limits, and host-resource exhaustion still matter. The stress test checks describability and metadata cost, not checkpoint availability, numerical correctness, training feasibility, or the ability to execute a full forward on the target hardware.

Part 3 - Quantitative Prediction Before Execution

Define two sweeps: fixed layer count with growing width, and fixed width with growing layer count. Calculate logical parameter bytes for each case and predict different metadata scaling. Estimate baseline-adjusted host overhead per module from measured supported cases, not a universal constant. Choose cases beyond the declared real-memory reference while retaining

explicit object-count, time, and host-memory ceilings. Predict refusal or interruption states rather than demanding every arbitrary scale succeed.

Part 4 - Experimental Design and Instrumentation

Use isolated bounded child processes and progressive sizes. Audit every required state entry for meta status and forbid conversion, value extraction, real initialization, or checkpoint generation at oversized scales. Validate shape/count formulas against tiny real models first. Record logical elements, module/tensor counts, supported host high-water evidence, elapsed time, software version, architecture ID, and completion status. Include a safe-size explicit CPU allocation escape as a detection control. Preserve resource-limit stops and unsupported constructors separately from completed meta-only cases.

Bound the observer by module and tensor count rather than creating one record per logical element. A billion-element tensor requires a small metadata record; an accidentally expanded diagnostic list would test the observer instead of meta construction.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class MetaStressResult:
    case_id: str
    logical_parameter_elements: int
    nominal_real_payload_bytes: int
    module_count: int
    tensor_count: int
    peak_host_bytes: object
    completed: bool
    payload_audit_status: str


def generate_meta_scale_cases(shape_limits, object_limits):
    # Separate dimension growth from metadata-object growth.
    ...


def run_bounded_construction(case, process_limits):
    # Audit meta-only state without creating real oversized payload.
    ...


def E18(config):
    # Export scaling evidence, safe controls, and interrupted outcomes.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require all audited payload to remain meta and every successful case to finish within its declared limits. A small current RSS after a process exits is not its construction peak. Report logical billions precisely and avoid expanding arrays of parameter names or per-element records in the observer. A shape-only forward may itself be unsupported or costly; it is optional and must not be substituted for the construction-memory measurement.

Part 7 - Engineering Interpretation and Philosophy

Large logical scale is useful when it enables reliable planning before real commitment. The test should expose where metadata complexity or architecture behavior becomes the limiting resource rather than celebrate an unexecuted parameter count as a working model.

19.31.20 Exercise 19

Exercise Title: Build and Benchmark a PyTorch Memory-Aware Model Loader That Constructs on Meta, Memory-Maps the Checkpoint, and Materializes Parameters Directly into Their Planned CPU or GPU Locations

Part 1 - Exercise Contract and Why It Matters

Integrate meta construction, mapped checkpoint access, canonical placement, bounded materialization, readiness validation, and measurement into a model loader. Produce verified CPU/GPU state without unnecessary duplicate full-model payload. Preserve complete useful-work timing and explicit limits on source format, architecture, aliasing, and execution support.

Part 2 - System and Memory Mental Model

Separate architecture metadata, authoritative checkpoint storage, planned locations, active loading resources, and published ready values. The loader must not confuse assignment with conversion or empty allocation with initialized contents. Canonical tied groups constrain placement and parameter identity. Mapped source owners can survive as final CPU model storage. Optimizers depend on final references; deferred meta state remains blocked from unsupported execution. Unknown custom modules and tensor layouts require explicit refusal or a measured alternative path.

Part 3 - Quantitative Prediction Before Execution

For each plan, predict final CPU/GPU payload, buffer demand, maximum active source/conversion overlap, staging capacity, and readiness time components. Compare conventional loading, mapped/meta/assignment, and budgeted group placement under the same final model contract. Define the objective as reducing unnecessary overlap subject to correct contents, complete coverage, budgets, and progress. Report nominal bytes, resident memory evidence, allocator peaks, and latency independently; a valid plan is not a proof of globally minimal memory or future execution feasibility.

Part 4 - Experimental Design and Instrumentation

Freeze an architecture/source support matrix and validate keys, shapes, dtypes, ties, and noncheckpoint buffers. Build and independently check the placement plan before real allocation. Complete group admission, value supply, completion, verification, source release, and final publication. Compare tiny reference outputs and supported training gradients/updates before timing. Evaluate CPU, mixed-device, tight-budget, shared-weight, missing-key, conversion, and controlled-failure cases. Separate mapping return, ready-model completion, first useful

execution, and later warmed execution. Preserve cache context, version/configuration, and raw phase samples.

Separate source-read completion from final-resident ownership in the dossier. A loader can finish reading and still retain mapped storage as valid CPU weights. Report that useful retained state rather than forcing unnecessary clones solely to make a source-release counter look better.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class LoaderContract:
    architecture_id: str
    checkpoint_format: str
    supported_layouts: tuple
    target_policy: tuple
    host_budget_bytes: int
    device_budget_bytes: int
    readiness_policy: str

class MemoryAwareModelLoader:
    def __init__(self, contract):
        self.contract = contract
        # Initialize canonical ownership, bounded ledgers, and support checks.
        ...

    def construct_meta(self, factory):
        # Audit required state and constructor escapes.
        ...

    def open_checkpoint(self, path):
        # Establish explicit mapped CPU source and restricted loading policy.
        ...

    def plan(self, model_metadata, checkpoint_metadata):
        # Validate schemas, aliases, dtypes, placement, and headroom.
        ...

    def materialize(self, plan):
        # Process admitted groups without hidden full-model duplication.
        ...

    def verify_readiness(self, model, reference_contract):
        # Check authoritative values, buffers, ties, and final locations.
        ...
```

```
    def finalize_dependents(self, model, training_contract):
        # Create optimizers only after final parameter identities settle.
        ...

    def shutdown(self, deadline):
        # Resolve active copies and preserve retained source ownership.
        ...


def E19(config):
    # Compare equivalent loaders and retain all unsupported/failure outcomes.
    ...
    # Export complete memory, readiness, numerical, and timing evidence.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Acceptance requires correct authoritative contents, complete required state, valid alias/identity contracts, coherent failure handling, and comparable useful work. Record architecture/checkpoint IDs, device and software versions, source format, target dtype/device, ties, buffer policies, phase peaks, mapping/resident metrics, allocation counters, readiness latency, output/gradient errors, and request outcomes. Export scalar metadata without retaining duplicate graphs.

Demonstrate one mapped CPU owner retained by the model, one GPU conversion, a tied-placement conflict, a missing buffer, a stale-optimizer prevention check, and an interrupted group. Missing hardware measurements remain not measured. Do not publish partial or uninitialized state as a loaded model.

Part 7 - Engineering Interpretation and Philosophy

The loader succeeds when a model moves from description to verified resident values with unnecessary overlap removed and necessary ownership preserved. Recommend the smallest supported loading path that meets measured budgets, and identify the next concrete experiment required before expanding architecture, checkpoint, alias, training, or multi-device support.

19.32 DMM30

Chapter Title: Lazy Parameter Materialization
Specialization: Dynamic Memory Management and CUDA Systems

19.32.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Lazy Parameter Materialization, Deferred Storage Allocation, and On-Demand Weight Creation

Part 1 - Exercise Contract and Why It Matters

Build a lifecycle model for weights whose shape, storage, or values become available at different times. Distinguish first-input shape inference from on-demand checkpoint residency. Deliver a state diagram covering request, admission, creation, valid-value publication, consumption, release, and later recreation, with an explicit authority for every required weight value.

Part 2 - System and Memory Mental Model

An unknown-shape uninitialized parameter differs from a known-shape meta parameter. Allocating storage is also different from supplying correct values. Built-in lazy layers primarily defer shape-dependent initialization; they do not automatically provide repeated eviction, checkpoint paging, or budget scheduling. The later runtime is an explicitly designed inference system with canonical ownership and completion rules. Host metadata remains allocated, and a model whose weights are described but not ready is not yet an executable real-valued model.

Part 3 - Quantitative Prediction Before Execution

For group g, predict target payload $B_g = \sum_i n_i s_i$ under a dense canonical ownership model. Draw eager and deferred timelines for a two-branch model where one branch is unused. Estimate startup payload avoided, first-use allocation, and final residency separately. Add a second access to the same group and predict reuse versus recreation. List which costs return when the weight is actually needed and which can be permanently avoided for an unused branch.

Part 4 - Experimental Design and Instrumentation

Use a tiny eager reference and a transparent lazy architecture. Record shape status, storage state, value source/version, target device, initialization state, and readiness generation. Exercise unknown shape, known-shape meta, allocated-uninitialized, and valid-resident cases separately. Begin with single-process eager inference and no optimizer. Capture constructor time, first useful execution, and warmed execution as distinct phases. Record versions, seed, architecture, dtype, device, and the exact path used; do not compare an unexecuted lazy skeleton with a completed eager workload.

For newly initialized rather than checkpoint-backed values, define how a canonical value becomes reproducible across later demand. Reusing a global random stream at another time can produce different weights; recreation must not silently change the function being executed.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
from torch import nn

@dataclass
class WeightLifecycle:
    canonical_key: str
    shape_status: str
    storage_status: str
    value_authority: str
    value_version: int
    resident_generation: int
    readiness: str


def classify_lazy_state(parameter, specification, value_ledger):
    # Separate unknown shape, meta metadata, storage, and valid values.
```

```
    ...


def predict_first_use(request, lifecycle, execution_contract):
    # Include admission, creation, value supply, and completion.
    ...


def E00(config):
    # Compare lifecycle predictions with tiny supported examples.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require explicit readiness before computation and a retained authority before any later dematerialization. An empty allocation or finite-looking garbage cannot establish initialized contents. Shape-only execution does not establish numerical correctness. Shared names must refer to canonical values rather than independent accidental allocations. Keep unsupported training, custom storage, and dynamic compiler behavior outside the initial runtime contract instead of silently assuming they follow the same lifecycle.

Part 7 - Engineering Interpretation and Philosophy

Laziness changes when an obligation is paid; it does not remove the obligation to provide correct values. The useful design makes shape knowledge, resource commitment, and value authority independently visible so on-demand work can be admitted, delayed, or refused honestly.

19.32.2 Exercise 01

Exercise Title: Create an nn.Module Whose Parameters Exist as Metadata Before Real Tensor Storage Is Allocated

Part 1 - Exercise Contract and Why It Matters

Create a module whose weights are represented before real payload allocation. Support a known-shape metadata case and an unknown-input-shape case, while making their different inspection capabilities explicit. Preserve proper module registration so later enumeration, loading, and readiness checks can find the weights.

Part 2 - System and Memory Mental Model

A known-shape meta parameter can carry dimensions and dtype without values. An uninitialized parameter can defer the shape itself, so querying its shape or element count may fail. Neither is equivalent to an arbitrary Python tensor attribute that the module registry ignores. Device and dtype preferences can be declared before real materialization. Constructor-owned buffers and explicit CPU allocations remain part of the audit even when the main weight is lazy.

Part 3 - Quantitative Prediction Before Execution

Choose an output width o and distinguish a known input width d from an unknown width discovered later. For the known case predict weight and bias metadata and future payload $(od + o)s$ when bias is enabled. For the unknown case leave payload unresolved rather than

inventing zero. Predict which inspection fields are available in each state and which require runtime input or an external architecture specification.

Part 4 - Experimental Design and Instrumentation

Define a small registered module and a separate serializable specification record. Complete both lazy representations without real weight initialization. Audit parameters, buffers, and declared extra resources. Inspect the known metadata safely and feature-test uninitialized-state detection before requesting shape fields. Compare a tiny materialized reference after valid values exist. Add an unregistered tensor attribute and an explicit CPU factory as safe-sized negative controls. Report their detection rather than claiming a device context prohibits every constructor escape.

Check serialization behavior without assuming all deferred parameters already have ordinary tensor metadata. A state dictionary may contain uninitialized entries. Preserve the separate specification needed to distinguish an unknown shape from a known shape whose values are deferred.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
from torch import nn
from torch.nn.parameter import UninitializedParameter

@dataclass
class DeferredWeightSpec:
    output_features: int
    input_features: object
    dtype: object
    preferred_device: object
    value_source: str

class DeferredModule(nn.Module):
    def __init__(self, specification):
        super().__init__()
        self.specification = specification
        # Register the appropriate known- or unknown-shape state.
        ...

    def forward(self, value):
        # Require admitted, value-ready state before computation.
        ...


def E01(config):
    # Audit registration and distinguish available metadata fields.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not call element-count or shape inspection on an unknown-shape parameter without first classifying it. A zero placeholder is not an estimated payload. Require all execution-relevant registered and declared extra state to appear in the audit. A successful constructor alone does not prove future loading or numerical execution is supported. Preserve unsupported constructors and buffer initialization obligations explicitly.

Part 7 - Engineering Interpretation and Philosophy

A lazy module needs a discoverable contract before it needs real weights. The specification provides that contract without pretending unknown dimensions are already known or confusing registered deferred parameters with ordinary untracked Python attributes.

19.32.3 Exercise 02

Exercise Title: Compare Eager Parameter Initialization Against Lazy Materialization by Peak RAM, Peak VRAM, and Startup Time

Part 1 - Exercise Contract and Why It Matters

Compare eager construction and initialization with lazy first-use materialization under the same architecture, value policy, and useful workload. Measure startup, first complete result, warmed latency, host peaks, and VRAM peaks separately. Include both exercised and unexercised branches in the interpretation.

Part 2 - System and Memory Mental Model

Eager construction pays for weights before use; lazy construction can defer that work and avoid unused branch payload. A workload that eventually uses every group may simply move cost into first execution. Random initialization order can differ between eager and lazy modules even with the same global seed. CUDA context setup, CPU page commitment, allocator caching, and checkpoint page access also affect the observed boundary. Define equivalent values through a controlled initializer or common checkpoint.

Part 3 - Quantitative Prediction Before Execution

For branch groups with payloads B_1 and B_2, predict resident weight demand when only branch one executes and when both eventually execute. Model first result as construction plus admitted materialization plus completed computation. Calculate potential startup deferral without claiming it is an end-to-end speedup. Predict a tiny workload where hook or policy overhead exceeds the allocation saving and a larger unused branch where retained capacity differs substantially.

Part 4 - Experimental Design and Instrumentation

Run independent bounded trials with matching initial state, input path, dtype, execution mode, and output contract. Capture process baseline, constructor return, first ready result, and repeated warmed execution. Use supported host high-water instruments and absolute CUDA allocator peaks, synchronizing only at declared completion boundaries. Verify actual weights and outputs with a small eager reference. Randomize strategy order, describe file-cache/context conditions, and retain raw timing samples. Charge initialization, checkpoint reads, placement, and required validation consistently across strategies.

For a conditional workload, report which branch outputs were requested and completed. Avoid timing an eager model on both branches while the lazy model executes one, unless the comparison explicitly measures unused eager initialization rather than different inference work.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class LazyStartupTrial:
    strategy: str
    executed_path: tuple
    construction_seconds: object
    first_result_seconds: object
    warm_samples: tuple
    peak_host_bytes: object
    peak_cuda_bytes: object
    verified: bool


def measure_startup_strategy(factory, strategy, authority, inputs, config):
    # Follow equivalent weights through first complete useful execution.
    ...


def E02(config):
    # Compare startup deferral, final residency, and warmed cost separately.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

A timer stopping before real values are ready is not comparable with an eager first-forward result. Do not infer peak physical memory from nominal payload or a single RSS sample. Report branch coverage so skipped work does not look like a faster equivalent computation. Preserve cases with no memory benefit, slower first-use latency, or unavailable CUDA instead of selecting only favorable trials.

Part 7 - Engineering Interpretation and Philosophy

The benefit of laziness depends on what the workload actually needs and when. A fair result states whether it avoided unused work, reduced overlap, shifted cost into first use, or merely changed the measurement boundary.

19.32.4 Exercise 03

Exercise Title: Build a Lazy Linear Layer That Allocates Its Weight Storage Only When the First Real Input Arrives

Part 1 - Exercise Contract and Why It Matters

Build a linear layer whose real weight allocation and initialization occur only after a valid first input supplies the missing input width. Compare it with a built-in lazy linear reference and verify that later compatible calls reuse the initialized parameter set rather than resetting it.

Part 2 - System and Memory Mental Model

For a linear transformation, the last input dimension determines input features; leading dimensions describe batches or other independent positions. A built-in lazy linear layer performs one-time shape initialization, not automatic reshaping for every later feature width. Device and dtype preferences must already agree with the supported input contract. A custom implementation needs an explicit state transition so invalid input, partial initialization, or a second call cannot silently replace correct existing weights.

Part 3 - Quantitative Prediction Before Execution

For input shape (b, t, d) and output width o, predict weight shape (o, d), optional bias shape (o), output shape (b, t, o), and dense payload. Compare first-call creation with a later call that changes only b or t. Predict rejection for a changed feature width after initialization. Define deterministic values for a tiny case so numerical expectations do not depend on different random-number consumption order.

Part 4 - Experimental Design and Instrumentation

Create a transparent custom module and a built-in reference at the intended dtype/device before their initialization runs. Test registration before use, first valid input, repeated compatible input, disabled bias, and invalid rank or feature width. Complete value initialization and verify parameters and outputs against a conventional linear reference with matching weights. Count materialization generations and real requests in a diagnostic replay. Create an optimizer only after the selected custom implementation's final parameter identity is stable if training is separately tested.

Publish weight and bias readiness as one supported layer transition. A failure after allocating only one component must not expose a half-ready linear layer. Keep the previous deferred state or a specific failed ticket that later callers can diagnose.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
from torch import nn
from torch.nn.parameter import UninitializedParameter

class StudentLazyLinear(nn.Module):
    def __init__(self, output_features, bias=True, device=None, dtype=None):
        super().__init__()
        self.output_features = output_features
        self.use_bias = bias
        # Register uninitialized weight/bias and explicit lifecycle state.
        ...
```

```
    def initialize_from_input(self, value):
        # Validate shape/device/dtype, allocate once, and supply values.
        ...

    def forward(self, value):
        # Initialize only when needed; otherwise check compatibility.
        ...


def E03(config):
    # Compare first use and repeated calls with matching reference weights.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require one valid materialization for repeated compatible calls. An incompatible later input must not silently destroy learned or checkpoint-loaded weights. Check both bias policies and empty leading batch dimensions where supported. A first-input error must leave a coherent deferred or failed state, not a half-published parameter set. Distinguish built-in lazy-layer behavior from additional features implemented by the student runtime.

Part 7 - Engineering Interpretation and Philosophy

The first real input can complete a structural contract, but later inputs must obey the contract it established. One-time shape discovery is useful because it removes guesswork, not because it permits arbitrary parameter replacement during ordinary execution.

19.32.5 Exercise 04

Exercise Title: Initialize Parameter Shape from Runtime Input Dimensions and Verify Correct Materialization on the Target Device

Part 1 - Exercise Contract and Why It Matters

Infer parameter shape from runtime input under an explicit rank, feature, dtype, and device contract. Verify direct placement and initialized values. Separate shape inference from resource admission so an unexpectedly large input cannot force an uncontrolled weight allocation.

Part 2 - System and Memory Mental Model

Input metadata can reveal dimensions without revealing valid parameter values. A target device preference is not a guarantee that every input tensor and operator is compatible with it. Leading dimensions can change while the learned feature contract remains fixed. Dtype promotion or CPU/GPU movement must be intentional, not an incidental attempt to make a mismatched first call succeed. Shape arithmetic also needs bounds before requesting physical storage.

Part 3 - Quantitative Prediction Before Execution

For a weight derived from runtime feature width d and output width o, calculate ods bytes and optional bias demand. Predict output shapes for a vector, a matrix, and a higher-rank supported input. Declare how zero feature width, empty batches, oversized dimensions, and a

changed later feature width are handled. Estimate allocation and execution headroom before deciding that the target can admit the request.

Part 4 - Experimental Design and Instrumentation

Use a table of deterministic input metadata and expected decisions before running real allocations. Complete shape normalization, dtype/device checks, budget preflight, and value initialization as separate steps. Test CPU and CUDA only where available and supported, with independent reference weights and outputs. Record actual parameter device/dtype after materialization rather than inferring it from the input alone. Include a controlled oversized request that is rejected before allocation and a compatible later batch-shape change.

If a checkpoint authority already fixes the weight shape, validate the runtime feature width against that shape before allocation. Input-driven inference must not override an authoritative checkpoint dimension merely to admit an incompatible request.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class InferredParameterRequest:
    weight_shape: tuple
    bias_shape: object
    target_device: object
    target_dtype: object
    payload_bytes: int
    admitted: bool


def infer_request(input_metadata, layer_specification):
    # Bound dimensions and preserve the established feature contract.
    ...


def materialize_inferred_request(request, initializer, budget):
    # Admit before allocation and verify actual device/dtype/values.
    ...


def E04(config):
    # Test supported ranks, batch changes, and preflight rejections.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require exact inferred shapes and explicit refusal of unsupported cases. Do not query nonexistent parameter shape as a substitute for input-driven inference. A successful empty-batch

forward does not establish initialized weights were consumed meaningfully; verify readiness independently. Preserve shape errors, policy refusals, unavailable devices, and genuine allocation failures as distinct outcomes. No silent cast or migration may change the comparison contract.

Part 7 - Engineering Interpretation and Philosophy

Runtime shape information is a planning input, not an allocation command. A good layer converts that information into a bounded request, then creates and publishes valid state only after compatibility and admission are established.

19.32.6 Exercise 05

Exercise Title: Detect Uninitialized Parameters Before Forward Execution and Produce a Clear Failure Instead of Silent Incorrect Behavior

Part 1 - Exercise Contract and Why It Matters

Detect deferred or invalid parameter state before numerical module execution. Produce a clear supported transition or a specific failure instead of using missing, meta, partially initialized, or stale values. Include required buffers and checkpoint readiness rather than testing only one parameter class.

Part 2 - System and Memory Mental Model

Unknown-shape parameters, known-shape meta tensors, allocated-but-uninitialized storage, in-progress copies, and valid resident weights are different states. Some can be classified through public tensor/module information; semantic value readiness needs an explicit runtime ledger. Shape inspection itself can fail on unknown-shape state. A preflight guard can legitimately allow first-use initialization when a valid input and policy are available, but must prevent execution until that transition completes.

Part 3 - Quantitative Prediction Before Execution

Design a state/decision table for unresolved shape, meta-only, allocating, loading, ready, failed, and stale-authority cases. Predict which can initialize, which must wait, and which must fail. Include one nonpersistent buffer absent from the checkpoint. Define error evidence containing module path, canonical key, lifecycle state, requested device, missing obligation, and retry policy without pretending a zero payload means a resolved unknown shape.

Part 4 - Experimental Design and Instrumentation

Build tiny modules representing each state and complete a guard that classifies before reading unsupported properties. Exercise explicit initialization, automatic first-use admission, direct guarded execution, and a failed initializer. Verify errors occur before arithmetic and preserve their phase. Test a partly loaded module with one required buffer missing. Compare ready cases with an eager reference, and ensure logs contain scalar metadata rather than live weights. Keep exceptions narrow so unrelated computation errors are not misclassified as deferred-parameter failures.

Feature-check built-in lazy inspection methods because a layer may change its class after initialization. The guard must still recognize the resulting ready ordinary module instead of assuming the same lazy-only method exists throughout its lifecycle.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class ReadinessFailure:
    module_path: str
    canonical_key: str
    state: str
    missing_obligation: str
    target_device: object
    retry_allowed: bool


def inspect_deferred_state(module, lifecycle_ledger):
    # Classify unknown shapes safely before accessing their properties.
    ...


def require_ready_or_materialize(module, request, policy):
    # Resolve valid transitions or raise a precise pre-compute failure.
    ...


def E05(config):
    # Test every lifecycle state, including missing buffers and load errors.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

A non-meta device label or finite-looking storage is insufficient evidence of initialized values. A guard that allows a pending CUDA copy to execute without the necessary dependency is incorrect. Preserve failed and unknown states distinctly and do not publish readiness after a swallowed exception. Repeated checks on an already ready generation must not reinitialize or replace its weights.

Part 7 - Engineering Interpretation and Philosophy

Clear readiness checks turn laziness into an explicit protocol rather than a source of accidental late errors. The guard should explain what remains unresolved and whether it can be supplied, while protecting already valid state from unnecessary replacement.

19.32.7 Exercise 06

Exercise Title: Materialize Lazy Parameters Directly on CUDA Without Creating an Intermediate Full CPU Copy

Part 1 - Exercise Contract and Why It Matters

Materialize supported lazy weights directly on CUDA after input-shape and budget validation, avoiding an unnecessary full CPU weight allocation. Separate direct random initialization from checkpoint loading, whose source bytes still need an authority and transfer path.

Part 2 - System and Memory Mental Model

A built-in unknown-shape lazy layer can receive its intended device and dtype before its first initialization run. A custom known-shape runtime can allocate real target storage directly. Neither path guarantees zero host memory: metadata, source mappings, and optional staging remain. A CPU checkpoint cannot magically become GPU values without reading and transferring its data. Define which full CPU destination copy is actually eliminated and which source resources are still present.

Part 3 - Quantitative Prediction Before Execution

For weight and bias payload B, predict direct target allocation and compare with a baseline creating a full independent CPU model before CUDA placement. Add input/output and operator headroom separately. For checkpoint-backed initialization, predict source residency, transfer traffic, and transient staging. Distinguish the avoided redundant CPU model from the authoritative checkpoint pages that still provide values.

Part 4 - Experimental Design and Instrumentation

Use small same-dtype layers with a deterministic initializer or common checkpoint. Preflight target capacity, configure the lazy module before first use, and complete direct CUDA materialization. Verify all parameters, buffers, and outputs on the intended device. Measure construction, allocation, value supply, and first result separately. Compare with an explicitly labeled CPU-first baseline in isolated trials. Inspect allocation evidence rather than merely checking the final device. Preserve unavailable CUDA and policy refusal as nonexecuted outcomes.

A meta-only shape pass is not a real CUDA materialization. If such a pass resolves dimensions, keep storage and value readiness unresolved until actual target allocation and initialization or loading complete. Report those phases separately.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
from torch import nn


def prepare_cuda_lazy_module(layer_specification, dtype, device):
    # Set target preferences before any supported shape-initializing run.
    ...


def create_cuda_values(request, authority, budget_ledger):
    # Allocate directly, supply values, and enforce completion/readiness.
    ...


def compare_cpu_first_and_direct(trials, measurement_contract):
    # Isolate redundant CPU payload from required source/staging demand.
    ...
```

```
def E06(config):
    # Verify values/device placement and phase-matched memory evidence.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not count checkpoint source pages as an unexpected full model destination, or claim they do not exist. A constructor that silently allocates real CPU weights violates the direct-creation contract. Target initialization must produce valid contents, not only allocate empty storage. Record allocator peaks and host peaks independently; lower nominal payload does not guarantee that external GPU pressure cannot cause allocation failure.

Part 7 - Engineering Interpretation and Philosophy

Direct creation is useful when it removes a redundant ownership stage. The precise claim names that stage and retains the unavoidable costs of metadata, authoritative values, transfer, and the final device allocation.

19.32.8 Exercise 07

Exercise Title: Materialize Lazy Parameters Directly on CPU When the Current VRAM Budget Cannot Admit Them

Part 1 - Exercise Contract and Why It Matters

Materialize weights on CPU when a declared GPU admission budget cannot accept them. Verify valid CPU values and a supported execution route rather than allowing a GPU input to meet CPU weights accidentally. Account for host capacity and any transfer cost introduced by fallback.

Part 2 - System and Memory Mental Model

CPU fallback is placement, not allocation cancellation. It consumes host RAM and can require moving inputs or intermediate activations. Some dtype/operator combinations differ in support or performance across devices. Keep the initial case to an explicitly supported CPU linear computation. A policy may choose CPU before attempting CUDA rather than repeatedly provoking OOM. If neither device can admit the required resources, it must return a bounded refusal instead of looping indefinitely.

Part 3 - Quantitative Prediction Before Execution

Calculate weight/bias payload and execution headroom on both devices. Given a hypothetical GPU allowance below the request and a sufficient CPU allowance, predict the selected location and transfer boundaries. Add input and output bytes to a fallback latency model. Compare CPU-resident computation with a future promotion policy without assuming the latter exists already. Construct a second case where both budgets reject the request.

Part 4 - Experimental Design and Instrumentation

Use a software admission limit to force a safe fallback at small sizes. Complete CPU allocation and initialization or loading without an unnecessary failed CUDA attempt. Validate device/dtype, inputs, buffers, outputs, and the caller-visible output-device contract. Compare

numerical results with matching weights in a supported eager reference. Record fallback reason, capacity reservations, transfer bytes, and completed latency. Test CPU budget denial and unsupported CPU dtype/operator cases as explicit refusals.

The fallback contract should specify whether results remain on CPU or return to the caller's original device. Either is possible under an explicit API, but silently changing it can break later layers even when the fallback calculation itself is correct.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class FallbackDecision:
    selected_device: object
    reason: str
    weight_bytes: int
    execution_headroom_bytes: int
    input_transfer_required: bool
    output_contract: str


def choose_cpu_fallback(request, budgets, support_contract):
    # Reject unsupported execution or insufficient host capacity.
    ...


def execute_cpu_materialized(module, inputs, decision, authority):
    # Supply valid CPU weights and perform explicit input/output routing.
    ...


def E07(config):
    # Force safe small fallback and verify both placement and useful work.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not report success after moving only the weights while computation still fails on a device mismatch. Preserve output placement semantics or charge explicit restoration. Host RSS, mapped source ownership, and any pinned staging remain separate costs. Distinguish planned GPU refusal from genuine OOM and unrelated exceptions. A supported fallback must complete the same intended operation, not skip the difficult layer.

Part 7 - Engineering Interpretation and Philosophy

Fallback is successful when it preserves useful work under an honest resource and transfer contract. A placement decision should explain the new bottleneck and its cost rather than presenting CPU residency as free memory relief.

19.32.9 Exercise 08

Exercise Title: Build a Device-Aware Materialization Policy That Chooses CPU or GPU from Parameter Size and Live Memory Headroom

Part 1 - Exercise Contract and Why It Matters

Build a materialization policy choosing CPU, CUDA, waiting, or refusal from canonical payload, reserved commitments, execution headroom, and supported operations. Separate the runtime's logical budgets from sampled physical availability and retain uncertainty in every decision.

Part 2 - System and Memory Mental Model

A device can appear to have free memory while several admitted requests have not allocated yet. Reservations must therefore include in-flight commitments without double counting already realized storage. Allocator reserved capacity, active tensor bytes, and driver availability describe different layers. External processes and unmanaged library allocations can change conditions after sampling. CPU placement also needs host budget and execution support; an optimistic device choice is not a guarantee against OOM.

Part 3 - Quantitative Prediction Before Execution

Define projected owned commitment as resident canonical bytes plus admitted unrealized reservations plus request transition demand and named headroom. Predict decisions for a resident cache hit, a new GPU group, a group larger than remaining GPU allowance but acceptable on CPU, and no feasible target. Record observation age and units. Explain why subtracting allocator reserved bytes from driver free bytes can double count memory already reflected in the physical observation.

Part 4 - Experimental Design and Instrumentation

Implement the policy first in a host simulator with fixed observations and auditable reservation transitions. Feed simultaneous hypothetical requests to verify admission does not overcommit. Test stale or unsupported metrics, changing external headroom, resident hits, and CPU support denial. Execute small admitted CUDA/CPU cases and compare predicted with observed payload, headroom, and outcomes. Preserve actual allocation failure without claiming the software budget guaranteed physical success. Release reservations only through explicit completion, failure, or cancellation transitions.

Use one coherent observation snapshot per admission decision and preserve its timestamp. Mixing an old driver observation with a new reservation ledger can hide overcommitment; the report should expose that uncertainty rather than manufacturing an exact free-capacity result.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class CapacityObservation:
    device: object
```

```
    driver_free_bytes: object
    owned_resident_bytes: int
    unrealized_reservations: int
    sampled_at: float
    unsupported_fields: tuple


def decide_materialization(request, observations, budgets, support):
    # Separate accounting layers and include all admitted commitments.
    ...


def reconcile_reservation(ticket, outcome, ledger):
    # Convert or release reservations exactly once after valid transitions.
    ...


def E08(config):
    # Validate host admission invariants before bounded device trials.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require nonnegative ledgers and exactly-once reservation settlement. Unknown or stale observations must not become fictitious zero demand or unlimited capacity. A resident alias hit must not reserve duplicate canonical storage. Report action, reason, uncertainty, and subsequent outcome. Include a case where the decision was reasonable but external pressure still caused OOM; classify it rather than rewriting the original observation.

Part 7 - Engineering Interpretation and Philosophy

A placement policy is a disciplined estimate under uncertainty. Its credibility comes from coherent commitments, bounded fallback, and honest outcomes, not from pretending a sampled memory number predicts every future allocation.

19.32.10 Exercise 09

Exercise Title: Load Parameter Values from a Memory-Mapped Checkpoint Only When the Owning Module Is First Needed

Part 1 - Exercise Contract and Why It Matters

Use a memory-mapped checkpoint as the value authority and materialize a module's weights only when it is first demanded. Compare logical source access, physical page activity where observable, target creation, and cache hits without claiming that opening the checkpoint costs no metadata or I/O.

Part 2 - System and Memory Mental Model

Mapping can expose tensor metadata before all data pages are touched, but standard loading is not a per-key disk-stream API. Keep CPU map location explicit to avoid saved GPU tags causing eager placement. CPU assignment can adopt mapped storage and retain its owner;

GPU placement reads and transfers values into new device storage. First demand must verify keys, shapes, dtype, canonical ties, and required buffers before publishing readiness.

Part 3 - Quantitative Prediction Before Execution

For a checkpoint with groups A and B, predict source metadata at open, touched payload when only A is demanded, target bytes for A, and no materialization of unused B. Then request A again and predict cache-hit behavior. Add a dtype conversion and account for its temporary overlap. Separate demand trace bytes from actual page-cache/read-ahead behavior, which may exceed the logical requested group.

Part 4 - Experimental Design and Instrumentation

Generate a trusted small compatible checkpoint and a complete name-to-group manifest. Open it under the declared mapping policy, then execute a branch that demands only one module. Complete value supply, readiness verification, and publication at that demand. Record source-access IDs, allocation events, phase timings, and actual touched-page evidence only where supported. Add repeat access, a missing key, and a changed checkpoint-version test. Compare with eager loading through the same first useful output boundary.

Associate the checkpoint with a stable value-version contract, not only a filename. A changed file or schema invalidates an old materialization assumption. Keep the initial backing source immutable while resident mapped aliases or pending transfers depend on it.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class CheckpointDemand:
    group_id: str
    source_version: str
    target_device: object
    target_dtype: object
    expected_keys: tuple


def open_lazy_checkpoint(path, format_contract):
    # Use explicit mapped CPU loading and preserve authoritative ownership.
    ...


def satisfy_checkpoint_demand(model, demand, source, registry):
    # Validate, create or adopt values, and publish one ready generation.
    ...


def E09(config):
    # Compare first demand, cache hit, unused groups, and schema failures.
```

. . .

Part 6 - Measurement, Validation, and Failure Analysis

A low mapping-return RSS is not proof that subsequent computation needs little RAM. Deleting a dictionary entry does not free source storage retained by a CPU parameter. Do not replace checkpoint values with random initialization on a load error. Preserve incomplete readiness and specific failure phase. If physical page-access instrumentation is unavailable, report logical demand without inventing exact disk traffic.

Part 7 - Engineering Interpretation and Philosophy

Demand loading is useful when it avoids unnecessary value access or target ownership. The correct evidence distinguishes those benefits from mapping laziness, operating-system cache effects, and storage that remains intentionally owned by the loaded module.

19.32.11 Exercise 10

Exercise Title: Materialize One Transformer Block at a Time from Checkpoint Storage and Release Initialization Buffers Immediately After Loading

Part 1 - Exercise Contract and Why It Matters

Materialize transformer-style blocks from checkpoint storage one ownership group at a time and release only their transient initialization resources. Verify attention, feed-forward, normalization, and buffer contents against a tiny eager reference. Distinguish bounded loading overlap from a complete weight-eviction runtime.

Part 2 - System and Memory Mental Model

A block's required state can include several projections, biases, normalization weights, buffers, and cross-block ties. Loading one block does not imply that previously loaded blocks have been released. Initialization scratch, mapped source owners, pinned staging, final weights, activations, and optional key/value caches have separate lifetimes. A source used by an asynchronous transfer must remain protected until completion; host copy submission is not an immediate release boundary.

Part 3 - Quantitative Prediction Before Execution

Calculate canonical block payload and predict a timeline of accumulated final weights plus the active group's source, conversion, and staging demand. Compare one-group loading with full duplicate model construction. If previous blocks remain resident, show why total final weights still accumulate. Add an optional cache and a tied projection group and calculate their distinct residency obligations. Do not count persistent execution caches as discarded initialization buffers.

Part 4 - Experimental Design and Instrumentation

Use a small deterministic transformer-style architecture and compatible checkpoint, with dropout disabled or controlled for numerical comparison. Build group manifests and validate cross-block ties before allocation. Complete one group's materialization, verification, completion, and scratch release before admitting the next under a one-group policy. Compare block

outputs with the eager reference using matching attention backend and masks. Track temporary reference ownership and actual allocator peaks. Introduce a controlled load failure and ensure partial block state is not published as ready.

If one tied projection spans blocks, treat it as a canonical ownership group rather than releasing it with the first block's temporary buffers. The loading schedule can be block-oriented without pretending every resource is owned exclusively by one block.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class BlockMaterialization:
    block_id: str
    canonical_groups: tuple
    final_weight_bytes: int
    temporary_bytes: int
    completion_event: object
    ready: bool


def materialize_transformer_block(block, source, plan, ledger):
    # Include every projection, normalization state, buffer, and tie.
    ...


def release_block_initialization_resources(ticket, retained_owners):
    # Wait or order completion; never release live final weights/caches.
    ...


def E10(config):
    # Verify outputs and separate accumulated residency from scratch overlap.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require all expected state and masks to match the reference. A checksum or comparison that touches every mapped tensor can warm the whole checkpoint; keep verification scope and timing explicit. Report source storage adopted by CPU weights as retained ownership. Unchanged reserved memory after scratch release does not prove a leak, while dropping a Python variable does not prove allocator or driver capacity was returned.

Part 7 - Engineering Interpretation and Philosophy

Blockwise creation controls the temporary working set. Its claim becomes stronger when every retained final weight, cache, and transient source is named, so an apparent release cannot accidentally discard useful state or conceal accumulating residency.

19.32.12 Exercise 11

Exercise Title: Build Forward Pre-Hooks That Trigger Parameter Materialization Automatically Before Module Execution

Part 1 - Exercise Contract and Why It Matters

Build forward pre-hooks that invoke the validated materialization protocol before a supported module computes. Preserve positional and keyword argument contracts, avoid reentrant initialization, and ensure an already ready generation is not recreated on every call.

Part 2 - System and Memory Mental Model

A pre-hook runs through the module call mechanism, not as a universal interceptor for every direct invocation of a forward method. Hooks can observe arguments and, under the configured signature, return modified arguments. Materializing weights on another device does not itself move inputs. Hook order, keyword handling, handles, exceptions, and nested module calls must be explicit. One-time built-in lazy initialization and a persistent demand-runtime hook serve different lifecycles.

Part 3 - Quantitative Prediction Before Execution

Predict hook counts and materialization counts for first use, repeated ready use, a nested module call, and a direct forward-method bypass test. Design transitions for cold, in-progress, ready, and failed states. State when the hook returns no argument modification and when an explicit placement policy requires transformed inputs. Include the cost of waiting for readiness rather than counting only hook dispatch.

Part 4 - Experimental Design and Instrumentation

Implement a small custom module with a clear call path and register the hook using the appropriate keyword-argument policy. Route demand to a registry rather than recursively calling the same module to initialize it. Verify first-use values, repeated-call stability, failure propagation, and hook removal at shutdown. Test an input passed by keyword and a CPU fallback needing explicit argument routing. Record global/local hook ordering if other hooks are present. Keep the main benchmark in eager mode and mark tracing or compilation support as a separate unvalidated extension.

Keep lease release and device completion explicit outside the pre-hook. Hook entry ensures readiness before a call; it does not by itself establish when all readers finish or automatically provide exception-safe lifecycle cleanup for every later operation.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def materialization_pre_hook(module, args, kwargs):
    # Resolve readiness without recursively invoking the same module.
    ...
    # Return the documented input form only if routing changes arguments.
    ...
```

```
def install_materialization_hooks(model, registry, support_contract):
    # Configure signatures/order and retain removable handles.
    ...


def remove_materialization_hooks(handles, runtime_state):
    # Stop new demand and remove hooks only through supported lifecycle.
    ...


def E11(config):
    # Test first call, ready hit, keyword input, bypass, and failure paths.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require numerical equivalence and exactly the expected materialization count. Do not claim a pre-hook protects direct forward bypasses; enforce the supported entrypoint or provide an explicit guard. A post-hook returning on the host would not prove CUDA consumers finished. Failed initialization must not leave a ready marker or leaked admission ticket. Preserve unexpected exceptions rather than treating them as an ordinary cache miss.

Part 7 - Engineering Interpretation and Philosophy

A hook is an integration point, not the ownership protocol itself. Keeping materialization in a validated registry makes automatic first use understandable, testable, and removable without hiding resource or execution obligations.

19.32.13 Exercise 12

Exercise Title: Prevent Duplicate Materialization When Shared or Tied Parameters Are Referenced by Multiple Modules

Part 1 - Exercise Contract and Why It Matters

Prevent multiple module paths from independently materializing one shared or tied weight. Build a canonical registry that preserves the intended parameter identity or declared shared-region relationship and charges resident capacity once per actual owner.

Part 2 - System and Memory Mental Model

A shared parameter can have several module names but one value authority. Meta addresses do not provide useful real storage identity, and two different parameter objects with equal values are not necessarily tied. Start with explicit architecture-declared ties of ordinary dense parameters. Cross-device requests for one tied value can conflict with the single-resident contract; replication requires a separately specified consistency policy rather than silently breaking the tie.

Part 3 - Quantitative Prediction Before Execution

Compare a named payload sum with a canonical sum for two modules sharing one weight and owning independent biases. Predict one first materialization, multiple bindings, and repeated cache hits. Add incompatible shape, dtype, source version, and target-device requests for the same canonical key. Specify which are compatible, which need a new validated generation, and which must be refused while current readers remain active.

Part 4 - Experimental Design and Instrumentation

Construct a tiny tied model with a matching eager reference and explicit name-to-owner manifest. Complete canonical demand resolution and binding after one value-ready materialization. Verify identity or the declared storage-region tie, outputs through both module paths, and canonical byte accounting. Test reversed access order and an unused alias. Add a conflicting target request and require a coherent decision. Keep training out of dynamic replacement unless optimizer references and saved values have been explicitly handled.

Equal checkpoint values under two names do not prove the architecture ties those parameters. Preserve declared ties and independent owners separately so accidental value equality does not merge parameters that should be able to evolve independently.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class CanonicalWeight:
    key: str
    bound_names: tuple
    shape_contract: tuple
    authority_version: str
    resident_generation: int
    readiness: str

class SharedMaterializationRegistry:
    def resolve(self, module_path, parameter_name):
        # Use explicit architecture ownership, never meta pointer equality.
        ...

    def acquire(self, canonical_key, request):
        # Reuse one compatible resident/in-progress generation.
        ...

    def bind_aliases(self, canonical_key, model):
        # Preserve all declared names and parameter identity semantics.
        ...


def E12(config):
    # Verify one canonical creation and correct behavior through both paths.
```

. . .

Part 6 - Measurement, Validation, and Failure Analysis

Require no duplicate charged storage for an ordinary tied owner and no silent split into independent weights. An identity test alone is insufficient for more general shared views; those require a supported region manifest or explicit refusal. Preserve authority version and shape contract in cache keys. A stale module binding must not keep an older generation available after the registry claims it was replaced or dematerialized.

Part 7 - Engineering Interpretation and Philosophy

Canonical ownership converts many logical names into one coherent materialization obligation. It is essential for both memory accounting and model semantics: saving allocation bytes is invalid if the resulting model no longer shares the values its architecture requires.

19.32.14 Exercise 13

Exercise Title: Reproduce a Race Condition Where Two Execution Paths Attempt to Materialize the Same Parameter Concurrently and Fix It Safely

Part 1 - Exercise Contract and Why It Matters

Reproduce a duplicate-materialization race in a small controlled host-side registry, then implement a protocol that lets competing requesters share one result or one failure. Protect publication and reservation settlement without relying on timing luck or unsafe concurrent GPU writes.

Part 2 - System and Memory Mental Model

A check-then-create sequence can let two paths observe a cold key before either publishes its allocation. The Python interpreter lock is not a complete application protocol across waits or operations that release it. Canonical states need an atomic owner transition and a generation-specific completion object. Host publication and CUDA value readiness are different events. A lock protecting state does not automatically make a pending device copy complete or permit arbitrary concurrent mutation of module bindings.

Part 3 - Quantitative Prediction Before Execution

Draw two request timelines passing a cold-state check, reserving bytes, and attempting publication. Predict duplicate allocation and possible last-writer replacement under the broken host model. Design a single-creator transition with waiting requesters and bounded failure propagation. Calculate reservation settlement for success, load failure, and requester cancellation. Explain why cancelling one waiter must not free storage still required by another.

Part 4 - Experimental Design and Instrumentation

Use barriers or an explicit deterministic host interleaving to expose the broken check-then-create logic on tiny safe mock or CPU resources. Record creator count, ticket count, published generation, waiter outcomes, and cleanup. Complete the repaired protocol with supported host synchronization and a separate device-ready token where needed. Test two simultaneous demands, creator failure, cancellation, timeout, and a stale late completion. Serialize actual module binding under the supported contract; do not deliberately launch a known-invalid CUDA data race to demonstrate the bug.

Bound every barrier and waiter in the deterministic race test. A deadlocked test is an outcome to diagnose, not a successful repaired protocol. Record which requester became creator and whether every remaining waiter received the same published generation or failure.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class CreationTicket:
    canonical_key: str
    generation: int
    creator_id: str
    waiter_ids: tuple
    completion: object
    outcome: str

class SingleCreatorRegistry:
    def request(self, key, demand):
        # Atomically become creator or join a compatible existing ticket.
        ...

    def publish(self, ticket, values, readiness_token):
        # Verify generation and publish exactly once after valid creation.
        ...

    def fail(self, ticket, error):
        # Notify all waiters and settle reservations exactly once.
        ...


def E13(config):
    # Reproduce the host race, then test repaired controlled interleavings.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require one published compatible generation, bounded waiters, and no leaked or double-released reservations. A passing unsynchronized run does not prove safety; the deterministic broken interleaving must actually expose the intended logical conflict. Preserve creator errors for every waiter. Ignore or safely reject stale completion tokens rather than allowing an old generation to overwrite newer authoritative state.

Part 7 - Engineering Interpretation and Philosophy

Concurrent laziness needs a publication protocol, not merely a cache dictionary. The useful repair establishes who creates, who waits, when values are readable, and who settles each resource commitment under every supported outcome.

19.32.15 Exercise 14

Exercise Title: Add Reference Counting to Track Which Materialized Parameters Are Still Required by Active or Future Computation

Part 1 - Exercise Contract and Why It Matters

Add explicit lease and reference accounting for resident weights used by active or declared future computation. Separate ordinary Python references, logical consumers, device completion, and speculative future hints. Use the accounting to decide when release is safe without confusing a scheduling prediction with a guaranteed future use.

Part 2 - System and Memory Mental Model

An active request can finish its Python call while GPU kernels still read the weights. A resident group can also be retained by aliases, external outputs, backward, or optimizer state under broader execution contracts. A simple module reference count does not capture those obligations. Future conditional branches may be unknown; they need a bounded prefetch reservation or recreation authority, not an indefinitely increasing fictional consumer count. Start with explicit inference leases and declared completion tokens.

Part 3 - Quantitative Prediction Before Execution

For canonical generation g, track active leases, completed-but-not-yet-reconciled readers, and bounded prefetch reservations separately. Define release eligibility from discharged hard obligations plus retained valid authority. Compare a known sequential next use, an untaken speculative branch, and an external consumer holding a lease. Predict why zero host leases alone cannot establish safe overwrite or free while a device reader remains outstanding.

Part 4 - Experimental Design and Instrumentation

Build a host lifecycle simulator before connecting real CUDA completion. Complete generation-aware acquire, attach-reader, release, cancellation, and reconciliation transitions. Test shared-module consumers, delayed GPU completion, repeated release, stale leases, and speculative branch cancellation. Validate bounded resident ownership against an independent event trace. Use explicit runtime tokens rather than interpreter reference-count functions as the primary oracle. If training is unsupported, reject it clearly; do not release weights needed by saved activations or dependent optimizer references.

Check count reconciliation against a separate event list on tiny traces. Looking only at the final zero can hide an earlier premature decrement. Audit each transition, including the interval where the host lease is gone but a CUDA read remains protected.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class WeightLease:
    canonical_key: str
    generation: int
```

```
    consumer_id: str
    completion_token: object
    purpose: str

class WeightLeaseLedger:
    def acquire(self, key, generation, consumer, purpose):
        # Register a hard reader or separately bounded speculative claim.
        ...

    def release(self, lease):
        # Validate generation and completion; prevent duplicate settlement.
        ...

    def eligible_for_release(self, key, authority_state):
        # Include all readers, aliases, and retained-value obligations.
        ...


def E14(config):
    # Test host/device lifetime differences and conditional cancellation.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require nonnegative counts and exactly-once release for every supported lease. A stale generation must never decrement the current generation's active count. Report undeclared or escaped consumers as unknown and retain storage or refuse eviction. Preserve delayed completion without pretending it is a leak. A future hint may be cancelled, but a hard reader cannot be dropped merely because keeping it resident exceeds the preferred cache target.

Part 7 - Engineering Interpretation and Philosophy

Reference accounting is useful when it counts real obligations rather than convenient objects. Safety comes from complete active lifetimes; efficiency comes from deciding which future hints are worth retaining or recreating within that safe boundary.

19.32.16 Exercise 15

Exercise Title: Dematerialize Cold Parameters Back to Metadata Plus External Storage and Measure the Cost of Recreating Them Later

Part 1 - Exercise Contract and Why It Matters

Dematerialize cold resident weights into metadata plus a retained external value authority, then recreate exactly the required version on later demand. Measure release effects and recreation latency. Begin with immutable checkpoint-backed inference weights and refuse unsupported dirty or training-dependent state.

Part 2 - System and Memory Mental Model

Discarding storage is legal only after readers finish and another authority can reproduce the same values. Moving a tensor description to meta loses its resident contents; it is not a

persistence mechanism. A changed trained weight cannot be recovered from an older checkpoint unless validated writeback updates the authority. Tied module bindings, external aliases, registry references, and asynchronous readers can all keep storage live or make replacement unsafe. Allocator reserved capacity can remain after real tensor ownership ends.

Part 3 - Quantitative Prediction Before Execution

For group payload B, predict released logical residency, remaining authority storage, and recreation demand. Model reload cost as source access, conversion, transfer, validation, and synchronization. Compare retaining a hot group with recreating a cold one after an idle interval, without inventing a universal break-even time. Add a dirty version newer than its checkpoint and predict refusal or required durable writeback before dematerialization.

Part 4 - Experimental Design and Instrumentation

Create a small checkpoint-backed inference group with known values and explicit leases. Verify authority version and completed readers, capture required metadata, and complete a supported unbinding/rebinding path for every tied name. Remove actual resident owners without private data hacks. Record allocated/reserved memory and retained references separately. Demand the group again, validate exact values and outputs, and charge complete recreation. Test outstanding readers, escaped aliases, dirty versions, missing source files, and stale module bindings as rejection cases.

A failed recreation must leave the external authority intact and the group unavailable, not publish partially loaded bytes. Retain metadata and the failure reason so another supported attempt can proceed without losing the only correct copy of the model values.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class DematerializationPlan:
    canonical_key: str
    resident_generation: int
    authority_version: str
    metadata_specification: object
    bound_names: tuple
    eligible: bool
    reason: str


def plan_dematerialization(key, registry, leases, authority):
    # Require completed obligations and reproducible current values.
    ...


def dematerialize_and_rebind(plan, model, registry):
    # Replace supported bindings with metadata and remove true owners.
```

```
    ...


def recreate_and_verify(key, authority, target, registry):
    # Restore the same value version before publishing a new generation.
    ...


def E15(config):
    # Compare retention, release, recreation, and refused dirty-state cases.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require unchanged authoritative values after recreation and coherent ties. A low count in the registry is not proof old tensors were released elsewhere. An unchanged reserved-memory reading is not by itself failure. Do not discard weights still referenced by autograd or an optimizer under an unsupported training contract. If writeback is not implemented, dirty groups remain ineligible; random reinitialization is never a repair for missing authoritative contents.

Part 7 - Engineering Interpretation and Philosophy

Cold storage eviction is a reversible residency decision only when the value history survives elsewhere. The useful runtime trades measured recreation cost against retained capacity while preserving authority, readers, and the exact model version.

19.32.17 Exercise 16

Exercise Title: Combine Lazy Materialization with Prefetching So the Next Layer Is Created Before the Current Layer Finishes Computing

Part 1 - Exercise Contract and Why It Matters

Prefetch the next supported weight group while the current group computes. Preserve bounded source/staging ownership and order consumer execution after value readiness. Compare demand-only loading with the same workload under prefetch rather than assuming asynchronous submission guarantees overlap.

Part 2 - System and Memory Mental Model

Source-page access, CPU conversion, pinned staging, host-to-device transfer, and GPU compute can overlap only when their resources and dependencies permit. A mapped pageable source does not automatically provide a pinned asynchronous pipeline. Prefetched groups consume capacity before use and may never be used on a conditional path. The canonical registry must deduplicate prefetch and demand for the same generation. Cancellation cannot release a source or destination still used by in-flight work.

Part 3 - Quantitative Prediction Before Execution

For current compute time T_c and next-group ready time T_l, predict an idealized exposed stall $\max(0, T_l - T_c)$ when independent resources permit full overlap. Add contention, host preparation, and synchronization as explicit limitations. Calculate peak weights for current

plus prefetched groups and staging. Compare lookahead zero and one, then predict wasted bytes/time for a mispredicted branch. These are models to test, not promised timings.

Part 4 - Experimental Design and Instrumentation

Use a fixed two- or three-group inference trace with known checkpoint values. Implement separate supported preparation/transfer work and generation-specific ready events. Make the compute consumer wait on the correct token, and protect source/staging buffers until copies complete. Compare outputs with demand-only execution. Record a timeline or equivalent supported overlap evidence, completed wall time, exposed waits, peak capacity, and unused prefetches. Test a demand racing with its prefetch and a cancelled speculative branch. Keep profiling separate from low-overhead timing.

Record the ready event after all required conversion and transfer work, not merely after source access begins. If preparation runs on the host, connect its completion to the device work explicitly before claiming the consumer can safely proceed.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class PrefetchTicket:
    canonical_key: str
    generation: int
    source_owners: tuple
    reserved_bytes: int
    ready_event: object
    status: str


def schedule_prefetch(next_request, registry, budgets, streams):
    # Join existing creation or admit one bounded speculative ticket.
    ...


def wait_for_prefetched_group(ticket, consumer_stream):
    # Order actual use after generation-specific value readiness.
    ...


def cancel_prefetch(ticket, completion, ledger):
    # Release only after in-flight obligations allow safe cleanup.
    ...


def E16(config):
    # Compare demand-only and lookahead execution with equal useful work.
```

. . .

Part 6 - Measurement, Validation, and Failure Analysis

Require correct values and no duplicate creation on a demand/prefetch collision. Record pinned capacity and conversion buffers rather than only GPU weights. A host timer ending at submission cannot establish overlap or readiness. Preserve slow or unused prefetch outcomes; useful throughput must not count speculatively loaded but unexecuted work as completed computation. Supported cancellation must not erase a hard demand already waiting for the same group.

Part 7 - Engineering Interpretation and Philosophy

Prefetch spends capacity now to reduce a later critical-path wait. It is justified when measured overlap and path predictability compensate for earlier ownership, staging demand, and wasted speculation within the same safety protocol as ordinary demand.

19.32.18 Exercise 17

Exercise Title: Build a Materialization Scheduler That Limits Concurrent Weight Creation According to RAM, VRAM, and Transfer Budgets

Part 1 - Exercise Contract and Why It Matters

Build a scheduler that bounds simultaneous weight creation using host bytes, device bytes, staging demand, and transfer concurrency. Preserve progress for hard demands while limiting speculative work. Deliver admission decisions, resource ledgers, queue behavior, and failure/cancellation accounting.

Part 2 - System and Memory Mental Model

A count of active groups is not a capacity budget when groups differ in size. Resources also have different units: queued transfer bytes, concurrency slots, measured bandwidth, and time deadlines cannot be added as one number. Reserve complete transition requirements before creation, converting reservations to resident ownership when realized. Avoid deadlock from partially holding one resource while waiting indefinitely for another. A hard request larger than the supported execution floor needs an explicit alternative or refusal.

Part 3 - Quantitative Prediction Before Execution

Represent admission demand as a vector of host bytes, device bytes, pinned bytes, and transfer slots. For each resource r, require existing commitment plus the new request's increment not to exceed its declared allowance. Compute a timeline for two unequal groups and a next-layer prefetch. Predict when queueing, cancellation of speculative work, CPU fallback, or refusal is legal. Separate a per-transfer concurrency cap from a workload-level throughput estimate and define a bounded progress rule.

Part 4 - Experimental Design and Instrumentation

Implement the scheduler in a deterministic host simulator with atomic or ordered multiresource admission. Test mixed group sizes, already-resident hits, shared in-progress tickets, deadline expiry, creator failure, and waiter cancellation. Validate nonnegative ledgers and

exactly-once settlement independently. Then run bounded small CPU/CUDA traces with explicit events and reference outputs. Measure queue wait, transfer wait, recreation count, speculative waste, peak owned resources, and useful throughput. Freeze policy parameters before evaluating held-out traces so tuning does not hide adverse cases.

Do not hold a registry lock while waiting indefinitely for a creator whose publication needs the same lock. The host simulator should include this lock/dependency pattern and verify bounded progress or clear timeout handling under the declared protocol.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class ResourceDemand:
    host_bytes: int
    device_bytes: int
    pinned_bytes: int
    transfer_slots: int
    speculative: bool

class MaterializationScheduler:
    def admit(self, request, resource_demand, observations):
        # Reserve compatible resources without partial-hold deadlock.
        ...

    def dispatch(self, ticket, runtime):
        # Launch only supported creation work and retain ownership.
        ...

    def settle(self, ticket, outcome):
        # Reconcile realized residency, failures, and cancellations once.
        ...

    def shutdown(self, deadline):
        # Stop admission, drain supported work, and report unresolved state.
        ...


def E17(config):
    # Verify host invariants before measuring bounded real execution.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require bounded outstanding commitments and explain any refusal that reduces workload coverage. A scheduler that rejects all large requests does not prove successful memory-aware

execution. External pressure can still cause a real allocation failure; preserve it separately from admission denial. Do not release reservations merely because the host function returned or a speculative waiter disappeared while other consumers still depend on the ticket.

Part 7 - Engineering Interpretation and Philosophy

Scheduling is a joint capacity and progress contract. The best policy is the smallest one that keeps hard work moving within declared resource bounds, with enough evidence to explain waits, fallbacks, refused work, and the cost of speculation.

19.32.19 Exercise 18

Exercise Title: Stress-Test Lazy Materialization on a Deep Model with Conditional Execution Paths, Shared Weights, and a VRAM Budget Far Below Total Model Size

Part 1 - Exercise Contract and Why It Matters

Stress-test lazy residency on a deep conditional inference model whose total weight payload exceeds its declared GPU allowance. Exercise shared weights, recreation, prefetch, and branch changes while preserving outputs and bounded resource commitments. Distinguish software-budget testing from deliberately exhausting physical VRAM.

Part 2 - System and Memory Mental Model

A budget far below total model size can be feasible only if the active supported working set, transition storage, and execution headroom fit somewhere. At least one required group or operation can still exceed the feasible floor. Conditional paths alter future demand and can invalidate speculative priorities. Shared weights require canonical ownership even when different branches access them. A full authoritative checkpoint and host resources remain necessary; a small GPU pool does not imply the entire system has negligible memory demand.

Part 3 - Quantitative Prediction Before Execution

Calculate total canonical payload and the largest mandatory active set for selected traces. Include current weights, next-group prefetch, activations, residuals, staging, and library margins where applicable. Predict feasible and infeasible budget levels before execution. For branch traces, estimate cache hits, recreations, unused prefetches, and useful path coverage. Explain why a claim of arbitrarily tiny GPU execution would be false without additional operator splitting or an explicit CPU route.

Part 4 - Experimental Design and Instrumentation

Use trusted small checkpoints and a configurable deep model with an eager reference that fits the validation environment. Force low residency through software admission caps. Generate deterministic path traces plus seeded conditional cases, including shared-group revisits and delayed completion. Validate every accepted output, generation version, lease transition, and budget ledger. Test missing authority, conflicting ties, no feasible target, midcreation failure, and shutdown. Minimize reproducible failures and retain the source IDs, path, seed, policy, and completion ordering needed to replay them.

Keep the eager validation reference small enough for the test environment. The low-budget claim can be exercised through a smaller software cap without constructing an uncontrolled oversized real model or fabricating a physical OOM to make the demonstration convincing.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class LazyStressTrace:
    trace_id: str
    path: tuple
    shared_groups: tuple
    software_vram_limit: int
    prefetch_policy: str
    completion_schedule: tuple


def predict_active_floor(trace, architecture, execution_contract):
    # Include all mandatory live and transition resources.
    ...


def execute_stress_trace(trace, runtime, eager_reference):
    # Compare accepted outputs and audit generation/resource invariants.
    ...


def minimize_lazy_failure(trace, replay_predicate, limits):
    # Preserve the causal branch, ownership, and completion conflict.
    ...


def E18(config):
    # Report coverage, correctness, recreation cost, and infeasible cases.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require useful completed work at the claimed budget and no false release or stale-generation use in admitted traces. Distinguish expected infeasibility, policy refusal, numerical error, OOM, timeout, and unrelated exceptions. A nonreproducing failure remains unresolved evidence. Do not claim a successful path skipped necessary layers or replace a missing checkpoint group with random values to keep a benchmark running.

Part 7 - Engineering Interpretation and Philosophy

The stress test defines a deployment envelope rather than a universal memory miracle. A strong result identifies the smallest feasible working set and shows correct progress across changing paths, with every remaining authority, host cost, and unsupported case accounted for.

19.32.20 Exercise 19

Exercise Title: Build and Benchmark a PyTorch Lazy Weight Runtime That Materializes, Places, Prefetches, and Dematerializes Parameters On Demand Under Dynamic Memory Budgets

Part 1 - Exercise Contract and Why It Matters

Integrate shape readiness, canonical ownership, demand creation, placement, prefetch, leases, dematerialization, and budget scheduling into a lazy weight runtime. Begin with immutable checkpoint-backed eager inference and publish explicit support limits. Benchmark equivalent useful work rather than counting only fast constructor returns or successful cache lookups.

Part 2 - System and Memory Mental Model

Separate architecture specifications, authoritative value versions, resident generations, pending creation tickets, consumers, and scheduler commitments. Built-in one-time lazy initialization is an optional front-end mechanism, not the complete runtime. A canonical registry deduplicates shared and concurrent demand. Ready values require correct contents and ordered completion; eviction requires discharged readers and reproducible authority. Dynamic parameter replacement during training needs additional optimizer, gradient, and saved-value protocols and is unsupported unless those mechanisms are explicitly implemented and validated.

Part 3 - Quantitative Prediction Before Execution

Define the objective as completed correct useful work per complete elapsed time under host, device, staging, and progress constraints. Predict demand payload, cache-hit opportunities, recreation cost, prefetch overlap, authority residency, and transition peaks for each trace. Compare eager full residency, demand-only lazy loading, lazy eviction, and the integrated prefetched policy. Report startup, first output, warmed throughput, tail latency, and absolute memory peaks separately. Safety and good measured behavior do not establish globally optimal placement or zero allocation.

Part 4 - Experimental Design and Instrumentation

Freeze an architecture/checkpoint/operator support matrix and calibrate on separate traces. Validate host admission, lease, generation, and publication invariants before CUDA execution. Complete demand through creation, value verification, binding, consumption, and safe release. Compare small outputs with an eager reference under matching weights, masks, precision, and path. Evaluate sequential, conditional, shared-weight, concurrent-request, tight-budget, and delayed-completion traces. Run ablations without weakening correctness. Separate diagnostic profiling from unprofiled timing and retain CPU fallback, unsupported, failed, and cancelled outcomes in coverage reports.

Document the final runtime as inference-only unless training has its own validated value-update and identity protocol. Supporting first-use allocation in a small training layer does not establish that arbitrary dynamic eviction and parameter replacement preserve optimizer or backward semantics.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class LazyRuntimeContract:
    architecture_id: str
    checkpoint_format: str
    execution_mode: str
    supported_layouts: tuple
    host_budget_bytes: int
    device_budget_bytes: int
    pinned_budget_bytes: int
    maximum_creation_tickets: int

class LazyWeightRuntime:
    def __init__(self, contract):
        self.contract = contract
        # Initialize canonical authority, generations, leases, and ledgers.
        ...

    def request(self, canonical_key, input_contract, consumer):
        # Resolve shape, join existing creation, or admit one new ticket.
        ...

    def materialize(self, ticket):
        # Supply authoritative values at the selected target safely.
        ...

    def publish(self, ticket, readiness):
        # Bind one valid generation and notify compatible waiters.
        ...

    def prefetch(self, future_request):
        # Admit bounded speculation without duplicating hard demand.
        ...

    def release(self, lease, completion):
        # Reconcile all actual readers before release eligibility.
        ...

    def dematerialize(self, canonical_key):
        # Preserve current authority and invalidate every stale binding.
        ...

    def audit(self):
        # Reconcile committed resources, values, generations, and outcomes.
        ...
```

```
    def shutdown(self, deadline):
        # Stop admission, resolve work, and report unresolved obligations.
        ...


def E19(config):
    # Validate tiny references and adversarial lifecycle cases first.
    ...
    # Benchmark equivalent work and export a supported recommendation.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Acceptance requires correct authoritative values and outputs, valid ownership and completion rules, coherent resource admission, and honest useful-work timing. Record versions, device identity, backend, source/version IDs, architecture, path/seed, dtype, masks, initial state, requests, creation counts, hit/recreation rates, authority residency, host/VRAM/pinned peaks, queue waits, latency samples, coverage, and all failure phases. Export bounded scalar records rather than retaining graphs or full weights in telemetry.

Demonstrate deduplicated tied demand, CPU fallback, a failed creator with waiters, a cancelled prefetch, a protected late reader, cold recreation, and an infeasible budget. Missing hardware results remain not measured; no benchmark values may be invented.

Part 7 - Engineering Interpretation and Philosophy

A useful lazy runtime preserves the model's values while deciding when their physical residency is worth paying for. Recommend the smallest measured policy that meets the workload contract, and name the evidence required before expanding shape, alias, concurrency, source-format, training, or compiler support.

19.33 DMM31

Chapter Title: Memory-Mapped Data and Large Dataset Access
Specialization: Dynamic Memory Management and CUDA Systems

19.33.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Memory Mapping, Virtual Address Space, Page Faults, Lazy Loading, and File-Backed Tensor Storage

Part 1 - Exercise Contract and Why It Matters

Build a lifecycle model connecting file bytes, virtual mappings, page access, resident file-backed pages, ordinary tensor copies, pinned staging, and CUDA storage. Explain precisely what memory mapping defers and which resources still exist. Deliver an accounting diagram and predictions that distinguish mapping a large file from reading or copying all its contents.

Part 2 - System and Memory Mental Model

A mapping exposes a virtual address range backed by a file; it does not promise that every page is already resident or that every future read is free. Page faults can establish resident

mappings without disk I/O or require storage access, depending on cache and operating-system state. Readahead can touch more data than the requested logical slice. File-backed tensor views, private modifications, collation, transforms, pinned copies, and CUDA transfers introduce different ownership and allocation boundaries. Zero eager payload copy does not mean zero host memory.

Part 3 - Quantitative Prediction Before Execution

For file size F, page size P, touched page set S, and independent batch copies totaling B, distinguish mapped virtual size, observed resident file pages, and anonymous/pinned payload. Estimate a page-cover lower bound for a small requested byte range, but do not claim it is exact physical I/O. Draw mapping-only, sparse access, full scan, and batch-to-GPU timelines. Explain why summing RSS over processes can count shared pages repeatedly.

Part 4 - Experimental Design and Instrumentation

Use a tiny deterministic raw file and a larger safely bounded fixture. Record file format, logical and allocated size, dtype, byte order, platform, filesystem, process baseline, mapping policy, and access schedule. Collect supported RSS, PSS, anonymous/file-backed, page-fault, and I/O observations separately. Label unavailable instruments rather than substituting zeros. Verify values on selected regions before any performance claims. Keep mapping creation, first useful access, complete scan, collation, and device transfer as distinct phases, with controlled or explicitly uncontrolled cache conditions.

Add a tiny page-address worksheet using the platform's observed granularity. For each phase, mark which layer owns the bytes and which event first needs their values. Include a batch transform that returns independent storage and a view that retains the mapping. Explain how those choices change release eligibility even when their numerical contents match. Keep the worksheet separate from empirical counters so its predictions can be disproved by the actual access path.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class MappingObservation:
    phase: str
    mapped_virtual_bytes: object
    resident_bytes: object
    proportional_resident_bytes: object
    minor_faults: object
    major_faults: object
    logical_bytes_requested: int
    observed_io_bytes: object


def classify_pipeline_storage(file_spec, views, staging, device_state):
    # Distinguish file-backed pages from independent batch allocations.
```

```
    ...


def E00(config):
    # Compare predicted and observed phases without inventing OS metrics.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

A minor fault is not a disk read, and a major-fault count is not an exact byte counter. A full scan can grow residency even when no full anonymous dataset tensor is created. Do not identify unused logical regions with pages proven never touched by the OS. Shared source pages and batch copies require separate accounting. A numerically correct slice does not establish that a benchmark avoided accidental full-file validation or copying elsewhere.

Attach an accounting-layer label to every reported byte field. A process resident total, a system cache observation, and a logical tensor payload must not be added as disjoint quantities without checking overlap. When the instrument cannot resolve sharing, preserve a bounded estimate or unknown rather than inventing an exact physical total.

Part 7 - Engineering Interpretation and Philosophy

Memory mapping offers controlled access to file-backed values while the OS manages page residency. The strongest mental model preserves the difference between requested data, cached data, copied data, and completed useful work instead of treating the mapping itself as an infinite-memory abstraction.

19.33.2 Exercise 01

Exercise Title: Create a Large Binary Tensor File and Compare Ordinary File Loading Against Memory-Mapped Access Without Loading the Entire Dataset into RAM

Part 1 - Exercise Contract and Why It Matters

Create a deterministic raw binary tensor file in bounded chunks and compare eager loading with mapped selective access. Specify the binary format and measure equivalent requested values. Keep file generation separate from the read benchmark, including its effect on storage cache and file allocation.

Part 2 - System and Memory Mental Model

A headerless raw file needs an external schema: dtype, byte order, element count, shape, and data offset. It is not interchangeable with a serialized PyTorch checkpoint. Generating a full source tensor before writing would already defeat the bounded-construction goal. Truncating a file can create sparse regions whose zero pages do not represent ordinary disk-backed data throughput. Eager reading and mapping return also have different readiness boundaries until the requested values are actually consumed.

Part 3 - Quantitative Prediction Before Execution

For N elements of width s, predict $F = Ns$ raw payload and a bounded generation workspace C independent of F. Compare eager independent payload ownership with a mapping plus selected-region copies. Calculate bytes requested by the same sample trace for both

readers and distinguish them from bytes read by the device. Predict why a freshly generated file may already be warm and why mapping-return timing alone would give a misleading result.

Part 4 - Experimental Design and Instrumentation

Preflight free disk space, chosen fixture size, and generation workspace. Write deterministic chunks with sample identifiers and an integrity manifest; close publication before readers start. Validate the complete tiny fixture, then use bounded spot checks for large timing runs with validation scope recorded. Complete eager and mapped readers and consume the same selected regions to a scalar result. Record generation time separately, cache context, logical/allocated file size, mapping time, access time, memory observations, and actual I/O only where supported.

Embed sample identifiers and a bounded verification pattern in each generated chunk so selective reads can be checked without scanning all bytes. Publish the manifest only after the writer has completed the declared file protocol. Record whether the filesystem uses compression, sparse allocation, or another feature affecting physical storage demand. A large filename size alone is insufficient to establish that the benchmark contains an equally large ordinary payload on the underlying device.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class RawTensorFileSpec:
    path: str
    dtype_name: str
    byte_order: str
    element_count: int
    shape: tuple
    payload_bytes: int


def create_chunked_fixture(specification, chunk_bytes, limits):
    # Write real deterministic payload without building the entire tensor.
    ...


def compare_eager_and_mapped(specification, region_trace, config):
    # Consume equivalent requested values after explicit readiness points.
    ...


def E01(config):
    # Verify format, bounded generation, and phase-matched reading evidence.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Reject truncated files, schema mismatch, unsupported endianness, and unsafe sizes before mapping. A sparse all-zero fixture must be labeled and must not stand in for realistic storage throughput. Do not compare a full eager scan with a tiny mapped subset as the same workload; identify that as a selective-access comparison. Preserve file-generation cache effects and avoid claiming cold storage without an independent justified procedure.

Run an intentionally wrong dtype interpretation on a tiny disposable fixture and require schema validation to reject it. Equal file length does not make two dtype/byte-order contracts equivalent. Keep accidental reinterpretation errors separate from numerical transform differences, and retain the exact manifest used by each reader for reproducibility.

Part 7 - Engineering Interpretation and Philosophy

The binary file is a data contract, not just a convenient extension. Bounded generation and fair access traces make it possible to attribute savings to avoided eager copying rather than to missing data, sparse allocation, or unequal useful work.

19.33.3 Exercise 02

Exercise Title: Create a File-Backed CPU Tensor with torch.from_file() and Verify That Its Storage Is Backed Directly by a Memory-Mapped File

Part 1 - Exercise Contract and Why It Matters

Create an ordinary dense CPU tensor from a compatible raw file and verify file-backed behavior using independent value and mapping-policy tests. Separate private copy-on-write access from shared write-through behavior without modifying the benchmark's authoritative dataset.

Part 2 - System and Memory Mental Model

The raw file tensor API takes an element count, not a byte count, and uses an explicit dtype contract. Its ordinary mapped storage is CPU storage and cannot be created directly as pinned storage under the documented restriction. Private mapping prevents writes from changing the file but is not a read-only tensor guarantee; writes can create private pages. Shared mapping allows file changes and interprocess visibility but does not supply synchronization or durable-publication guarantees by itself.

Part 3 - Quantitative Prediction Before Execution

Compute required bytes as Ns and predict failure when an existing private-map file is too short. For a tiny disposable file, predict observed values after one private mutation and after one shared mutation using separate mappings. Calculate the single element's byte location from dtype width. Distinguish file-backed residency from private dirty-page growth and explain why a stable pointer alone would not establish the backing relationship.

Part 4 - Experimental Design and Instrumentation

Prepare an immutable benchmark file and separate tiny scratch copies for mutation tests. Complete explicit CPU/dtype/size mapping and compare selected values with an independent byte reader. On scratch files, test private and shared changes, coordination, reopening, and the supported persistence boundary. Record policy, file identity, mapping lifetime, and verification scope. Keep all training-dataset access logically read-only. Test empty requests, length mismatch, and unsupported pinned/device choices with precise status instead of silently allocating another kind of tensor.

Use one scratch-file element and one untouched neighbor to verify the exact mutation extent. Reopen independently after the declared observation boundary and compare both locations. If a persistence experiment is included, specify its actual file/mapping synchronization mechanism rather than assuming a PyTorch tensor offers a generic flush method. The main dataset benchmark must continue to use an unchanged source and a nonmutating transform contract.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class FileMappingContract:
    path: str
    element_count: int
    dtype: object
    shared: bool
    application_writable: bool


def open_file_tensor(contract):
    # Validate file bounds and explicit CPU mapping properties.
    ...


def verify_backing_behavior(contract, disposable_copy, probes):
    # Compare private/shared behavior using independent file observations.
    ...


def E02(config):
    # Keep mutation controls separate from immutable dataset evidence.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require independent file observations, not only tensor aliases reflecting the same memory. Shared visibility is not proof of completed durable writeback; state the supported flush/close contract if durability is measured. Never mutate an OS read-only mapping through a tensor wrapper to test protection. File truncation or replacement during access is outside the immutable-source contract and must not be used as a routine benchmark case.

Check that the mapping remains alive while a returned view is consumed and that shutdown does not close or replace its backing prematurely. A tensor's logical byte count can exceed its currently resident pages; report these separately. The read/write policy test should never depend on deliberately causing an invalid access to a truncated mapping.

Part 7 - Engineering Interpretation and Philosophy

File-backed storage has an explicit policy and lifetime. Correct use depends on knowing whether a write changes private memory or authoritative file data, and on preserving that distinction when the tensor enters the larger input pipeline.

19.33.4 Exercise 03

Exercise Title: Read Selected Tensor Regions from a Memory-Mapped File and Prove That Unused Dataset Regions Do Not Need to Be Materialized Up Front

Part 1 - Exercise Contract and Why It Matters

Read selected regions from a much larger logical mapping and establish that an independent full dataset payload need not be created before useful access. State the observation model behind the title's proof and distinguish application materialization from operating-system residency and readahead.

Part 2 - System and Memory Mental Model

Creating a slice can create only a view; it may not read element data until a computation consumes the values. Converting, cloning, stacking, or computing an aggregate can touch pages and allocate outputs. The OS can prefetch neighboring pages even when the application requests a smaller region. A file can therefore be partly resident without a full anonymous tensor copy, but exact untouched-page claims need page-level evidence and cannot follow from source code or RSS alone.

Part 3 - Quantitative Prediction Before Execution

For a requested byte interval $[a, a + b)$ and observed page size P, calculate its intersecting page set and logical payload. Compare two distant intervals, adjacent intervals, and repeated access to one interval. Predict mapping-only, view-only, consumed-view, and copied-view outcomes. Explain how a checksum over the entire mapping would invalidate a claim that the experiment accessed only a few selected regions, even if the checksum is called validation.

Part 4 - Experimental Design and Instrumentation

Generate deterministic tiny truth data and a safely sized larger file. Define an exact region trace before opening. Complete selective views and consume only those regions through bounded operations. Capture process/file-backed observations and supported page/I/O evidence at each phase. Run a deliberately full-scan control in another replay, not before the selective test. Compare values against the manifest or an independent bounded reader. Preserve cache state, sample trace, dtype, offset units, mapping owner, and any observer activity that itself touches data.

Include a view-only phase that records metadata without calculating a sum, then a consumed-view phase that actually needs the selected values. This control reveals whether the measurement labels an unexecuted slice as a completed read. Add a bounded copied-view phase and account for both its source access and independent destination. Compare those phases with a separate full traversal using identical lightweight consumption per requested byte.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class RegionRequest:
    request_id: str
    byte_offset: int
    byte_length: int
    consumer: str
    copy_required: bool


def validate_region(request, file_specification):
    # Check bounds, dtype alignment, and declared address units.
    ...


def read_selected_regions(mapping, requests, ownership_contract):
    # Distinguish view creation from actual value consumption and copying.
    ...


def E03(config):
    # Compare selective access with an independent full-scan control.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require correct selected values and no unintended full materialization by transforms, logging, or validation. Low RSS cannot prove every unrequested page remained untouched; label that claim unsupported unless directly observed. A repeated region can be cached and must not be counted as new disk traffic by assumption. Keep logical bytes, fault counts, resident pages, and independent copy bytes in distinct fields.

A profiler or logging helper can itself inspect the entire tensor. Audit such observers before trusting sparse-access evidence. If the OS reads neighboring pages, preserve that observation without calling the application eager. The claim is absence of a required upfront independent full payload, not control over every page the kernel may cache.

Part 7 - Engineering Interpretation and Philosophy

Selective mapping is valuable because useful work can begin from a bounded requested subset. A precise claim describes which application copies were avoided and which access was observed, while leaving OS speculation and unmeasured page residency as explicit uncertainty.

19.33.5 Exercise 04

Exercise Title: Benchmark Sequential and Random Access Patterns on the Same Memory-Mapped Tensor and Explain the Difference in Effective Throughput

Part 1 - Exercise Contract and Why It Matters

Benchmark sequential and random region access on the same mapped tensor. Explain effective throughput using locality, caching, page faults, access size, and storage behavior rather than assuming one pattern always wins. Keep requested work, coverage, and consumption identical or explicitly label the differences.

Part 2 - System and Memory Mental Model

Sequential access can benefit from locality and readahead; random access can increase per-request overhead and reduce cache reuse. A fully cached file may primarily measure CPU and memory behavior rather than disk. A random permutation without replacement differs from independent random indices with repeats. Fault counters and storage-byte counters answer different questions. The chosen reduction or transform can also dominate timing, hiding the data-access cost the experiment aims to study.

Part 3 - Quantitative Prediction Before Execution

For K requests of b bytes, calculate logical throughput Kb/T only when all requests complete and the numerator matches consumed data. Predict unique page demand for tiny hand-checkable traces and contrast it with repeated requests. Model a coarse request cost as access latency plus bytes divided by effective bandwidth, stating its limits. Predict cold-like and warm-cache rankings separately without inventing cache control.

Part 4 - Experimental Design and Instrumentation

Pre-generate sequential and seeded permutation traces outside the timed region. Use matching blocks, total coverage, scalar consumption, and trial boundaries. Record file/system context and run several randomized trial orders. Measure mapping setup separately, then capture complete access time, faults, supported I/O counters, and memory. Include a repeated-small-working-set control and a larger traversal that stresses locality. Preserve all samples and classify cache state as measured or uncontrolled. Compare storage characteristics without assuming HDD and NVMe must exhibit the same magnitude of effect.

For the random policy, record whether accesses are a permutation or sampled with replacement, and keep both when studying reuse. Compute a tiny trace's exact unique-byte and unique-page sets as an independent reference. Generate index order once before timing so permutation generation does not inflate only the random path. Include a consumer-only control operating on already resident bounded data to estimate the transform/dispatch floor.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class AccessTrial:
    pattern: str
    request_count: int
    logical_bytes: int
    unique_pages_estimate: object
    elapsed_seconds: object
    io_bytes: object
```

```
    fault_counts: object
    checksum: object


def generate_access_trace(pattern, bounds, seed):
    # Make coverage/replacement policy explicit before timing.
    ...


def run_access_trace(mapping, trace, config):
    # Consume equivalent values and measure completed access.
    ...


def E04(config):
    # Compare sequential, permutation, and cached-working-set controls.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate the scalar result or sample IDs so skipped reads cannot appear fast. Report repeated logical bytes separately from unique payload and observed I/O. A warmed second trial is not independent evidence of storage speed. Do not convert major faults directly into bytes or label every slowdown random-I/O thrashing. Preserve neutral outcomes, timing variability, and instrument limitations alongside any locality advantage.

Check whether the consumer forces completion and whether results remain finite and meaningful. A reduction over unread or uninitialized output is not evidence of successful file access. Report useful requested throughput, unique-payload coverage, and physical I/O only at their actual observed scopes, with no assumed conversion between them.

Part 7 - Engineering Interpretation and Philosophy

Access order is part of the resource demand. The useful conclusion identifies when locality helps this workload and whether the bottleneck was storage, resident-memory access, or CPU consumption, with enough trace detail to test the explanation.

19.33.6 Exercise 05

Exercise Title: Sweep Access Block Size and Determine How Small Reads, Large Reads, and OS Page Behavior Affect Dataset Throughput

Part 1 - Exercise Contract and Why It Matters

Sweep logical access block size while holding the total requested payload and consumption work comparable. Identify when request overhead, page coverage, readahead, CPU processing, and storage bandwidth explain the result. Separate application block size from operating-system page size and mapping granularity.

Part 2 - System and Memory Mental Model

A request smaller than a page can still require a whole page to become accessible, while adjacent requests may reuse that page. Larger blocks can amortize Python and dispatch overhead but increase working set or read unneeded data. Mapping offset alignment rules and page

sizes differ by platform; use observed supported values instead of assuming one universal size. A vectorized transform over a large block may change the compute implementation, so access and processing costs need separate controls.

Part 3 - Quantitative Prediction Before Execution

For each block size b, compute request count for a fixed logical payload and predict page-cover counts on a tiny trace. Include aligned and deliberately misaligned requests that remain valid for the selected API. Estimate useful bytes divided by completed time and separate physical read amplification where measurable. Predict a plateau or reversal when larger blocks no longer remove per-request overhead and instead raise memory or cache pressure.

Part 4 - Experimental Design and Instrumentation

Choose a bounded size sweep spanning below-page, near-page, and larger requests. Precompute traces and control coverage, replacement, and boundary handling. Measure a lightweight consumer and a fixed representative transform separately. Capture request time, completed throughput, faults, supported I/O bytes, resident memory, and variability across trials. Record cache context and actual page granularity. Include the last partial block consistently rather than dropping it from only one configuration. Keep fixture generation, trace generation, and full-data verification outside timed regions unless explicitly included.

Add two alignment cases with the same useful payload but different starting offsets inside a page. For a tiny fixture, calculate page intersections by hand and compare with the index arithmetic. Keep mapping creation alignment rules distinct from logical slice alignment: a supported full-file mapping can expose slices that do not themselves begin at a page boundary. Record which layer of alignment the test changes.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class BlockSweepCase:
    block_bytes: int
    request_count: int
    alignment_policy: str
    logical_payload_bytes: int
    consumer_kind: str


def build_block_sweep(file_specification, sizes, coverage_contract):
    # Preserve useful payload and define partial-boundary handling.
    ...


def measure_block_case(mapping, case, trace, instruments):
    # Separate access cost, consumer cost, page evidence, and I/O.
    ...
```

```
def E05(config):
    # Explain the observed curve using controls rather than one counter.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not report speedup from reading fewer useful bytes or changing transform semantics. A page-cover estimate is not a measured disk-byte count. Preserve cached and uncontrolled runs as such. If the smallest block is dominated by Python-loop overhead, demonstrate that with a compute/access control rather than calling the device slow. Report peak staging or copied block capacity when it changes with block size.

If a larger block reads extra padding, exclude that padding from useful-byte throughput and report it as additional access. Preserve the last-block handling and total sample coverage for every size. A benchmark that drops awkward boundaries is not measuring the same dataset and cannot establish a fair block-size optimum.

Part 7 - Engineering Interpretation and Philosophy

Block size is a tradeoff between overhead amortization and working-set demand. Select a range justified by the measured data path, not a single page-sized rule assumed to apply to every dataset, storage device, transform, or platform.

19.33.7 Exercise 06

Exercise Title: Compare Resident RAM Growth for Eager Dataset Loading and Memory-Mapped Access While Processing an Identical Dataset

Part 1 - Exercise Contract and Why It Matters

Compare resident-memory evolution for eager dataset ownership and mapped access while processing identical samples. Measure the complete timeline, including batches and transforms, and distinguish removed anonymous duplication from file-backed residency that grows as data is touched.

Part 2 - System and Memory Mental Model

Eager reading can create an independent anonymous dataset copy while file pages also remain cached. Mapping can avoid that full copy yet still accumulate resident file-backed pages during a scan. RSS, PSS, anonymous memory, shared file pages, and system page cache are related but noninterchangeable. Summing process RSS can overstate physical use when pages are shared. Queues, collated batches, pinned buffers, and retained outputs can dominate memory even with a perfectly lazy source.

Part 3 - Quantitative Prediction Before Execution

For dataset payload F, active batch copies B, index metadata I, and mapped resident pages $R_f(t)$, sketch a scoped model separating independent payload from file-backed residency. Predict mapping-only, first batch, full epoch, and idle-after-epoch states. Compare selected-subset and complete-scan workloads separately. Define the measured peak and explain why a final RSS delta can miss a transient eager-copy peak or retained queue.

Part 4 - Experimental Design and Instrumentation

Run eager and mapped strategies in isolated processes with the same access order, transform, batching, and reduction. Capture baseline and phase memory using supported high-water and sampled instruments, labeling their scopes. On Linux, distinguish process file/anonymous and proportional observations when available; other platforms must report unavailable fields honestly. Record worker count as zero initially and add workers only in a separate extension. Verify all requested sample IDs and values. Avoid storing batch tensors in the evidence log, and measure release without assuming the OS immediately evicts cached pages.

Track the owner count of batches held by the application in addition to source mappings. Include one controlled diagnostic run retaining several batches, then remove that retention in the equivalent production comparison. This demonstrates why a mapped source alone does not bound total pipeline memory. Keep the retained-batch control small and label its changed contract rather than treating it as the eager strategy's default behavior.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class ResidentMemoryPoint:
    phase: str
    rss_bytes: object
    pss_bytes: object
    anonymous_bytes: object
    file_backed_bytes: object
    independent_batch_bytes: int
    measurement_scope: str


def process_identical_trace(reader, trace, consumer, config):
    # Keep sample coverage and bounded batch ownership equivalent.
    ...


def compare_memory_timelines(eager, mapped, metric_contract):
    # Separate high-water evidence from sampled and final observations.
    ...


def E06(config):
    # Audit complete work and retain all memory-accounting limitations.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

A mapped full scan reaching a large RSS is not proof the source was eagerly copied into anonymous memory. Conversely, low source residency does not excuse unbounded output retention. Compare each reported layer with an independent owner inventory. Do not add process and system cache bytes without explaining shared accounting. Preserve failed or unsupported measurements rather than turning them into a perfect reduction percentage.

Record when each memory observation was sampled relative to consumer completion and reference release. Sampling after cleanup can miss the only overlap that matters. A claimed peak based on periodic samples needs its interval and limitations stated; a platform high-water value needs its process lifetime scope rather than being treated as a resettable phase counter.

Part 7 - Engineering Interpretation and Philosophy

Resident-memory savings should be attributed to specific ownership removed, not to a convenient final counter. The useful result follows the same data through the same work and explains which pages were mapped, copied, retained, and merely eligible for reclamation.

19.33.8 Exercise 07

Exercise Title: Build a Memory-Mapped Dataset Class That Converts File Offsets into PyTorch Tensor Samples Without Loading the Whole File

Part 1 - Exercise Contract and Why It Matters

Build a map-style dataset translating validated sample IDs into bounded tensor regions of a raw mapped file. Preserve schema, source ownership, and explicit copy semantics. Demonstrate correct sample access without creating a full independent dataset payload at construction.

Part 2 - System and Memory Mental Model

A file path and a dtype are insufficient without sample layout, shape, byte order, offsets, and bounds. For a fixed-size dense format, a process-local mapping can expose sample views, but transforms must not mutate authoritative source bytes or accidentally private-copy the dataset. Batching may intentionally stack samples into new bounded storage. Keep mapping owners alive while views are used and avoid relying on unspecified serialization of a live huge mapped view across processes.

Part 3 - Quantitative Prediction Before Execution

For fixed sample shape h, element width s, and sample index i, calculate sample bytes $b = s \prod_k h_k$ and byte offset ib under a headerless contiguous format. Convert byte offsets to element indices only after checking divisibility. Predict constructor payload, one view, a transformed sample copy, and a collated batch separately. Include index bounds and a truncated-file case before any real read.

Part 4 - Experimental Design and Instrumentation

Create a tiny deterministic sample file and manifest. Complete lazy opening in the consuming process, sample range validation, view construction, and read-only source usage. Compare every tiny sample with an independent in-memory reference, then test sparse selected access on a larger bounded file. Audit constructor behavior and returned view ownership. Add an in-place transform that must receive an independent bounded copy. Document the collation contract so future worker processes transmit bounded batches rather than an unintended full mapped backing storage.

Give each sample an explicit ownership policy: borrowed immutable view or independent bounded tensor. The collator must know which form it receives and create any necessary writable batch storage before transforms that mutate. Test a sample retained across several later fetches to ensure the implementation does not reuse a temporary destination behind its caller's back. Keep labels and sample IDs bound to their exact data region.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
from torch.utils.data import Dataset

@dataclass
class FixedSampleSchema:
    path: str
    sample_count: int
    sample_shape: tuple
    dtype: object
    byte_order: str
    payload_bytes: int

class MappedTensorDataset(Dataset):
    def __init__(self, schema):
        self.schema = schema
        # Store lightweight metadata and unopened process-local state.
        ...

    def __len__(self):
        # Return the validated logical sample count.
        ...

    def __getitem__(self, index):
        # Open locally, validate range, and return the declared sample form.
        ...


def E07(config):
    # Compare samples and audit bounded source/copy ownership.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require exact sample IDs, shapes, dtypes, and values for the supported format. Negative indexing, empty datasets, and out-of-range IDs need an explicit policy. Do not interpret a raw byte offset as an element offset. A returned view is not an independently writable sample. Detect unexpected constructor scanning, full-source cloning, or hidden materialization by a transform. Keep unsupported endianness or layouts outside the initial format contract.

Verify file size and source version at opening, then rely on the immutable source contract during normal access. Reopening a replaced path is not necessarily the same data authority. A production extension needing live updates would require versioned publication and reader lifetimes; it must not be assumed by this fixed-file dataset.

Part 7 - Engineering Interpretation and Philosophy

A mapped dataset is an address translation interface with an ownership contract. Its value comes from making a requested sample cheap and auditable, while keeping later transforms, collation, and process transfer explicit rather than claiming the entire pipeline is copy-free.

19.33.9 Exercise 08

Exercise Title: Store Variable-Length Samples in a Shared Memory-Mapped Data File with a Separate Offset and Length Index

Part 1 - Exercise Contract and Why It Matters

Store variable-length samples in one immutable raw data file with a separate validated offset/length index. Recover each sample lazily and define batching, padding, or packed output semantics. Account for index memory as well as data payload.

Part 2 - System and Memory Mental Model

Variable-length records need unambiguous units, dtype, byte order, shape interpretation, and source version. An offset can refer to bytes while a length can refer to elements; mixing them creates silent addressing errors. Overlapping ranges may represent intentional sharing or corrupt metadata and must follow the declared format. A large index can itself exceed memory, so mapping or compact fixed-width index representation belongs to the design. Padding and collation materialize bounded output even when source access is view-based.

Part 3 - Quantitative Prediction Before Execution

For records (o_i, ℓ_i), validate each byte range using element width and file payload bounds. Calculate index bytes for two fixed-width fields plus any shape/label metadata, without assuming Python object tuples have that compact representation. For one batch, compare packed payload with padded payload using the longest record and include masks. Predict a long outlier's effect on batch capacity and define an admission rule.

Part 4 - Experimental Design and Instrumentation

Write deterministic short, empty, and longer records in bounded chunks and publish the data/index pair under one versioned manifest. Complete validated index access, bounded sample views, and the selected collator. Compare every tiny reconstructed sample with a reference. Test misaligned offsets, overflow arithmetic, negative lengths, truncated data, stale index versions, and an explicitly allowed overlap. Keep malformed-case validation in host metadata rather than intentionally dereferencing invalid mapped pages. Measure index, source residency, and padded batch bytes separately.

Compare an index stored as compact fixed-width records with a deliberately small Python-object index. Measure metadata overhead separately from data pages, and do not extrapolate a Python tuple's cost from its serialized field width. For batching, retain original lengths so a padding mask can be checked against every reconstructed record. Include one record spanning a locality window and verify it is not split or duplicated accidentally.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class VariableRecord:
    sample_id: int
    byte_offset: int
    element_count: int
    shape_metadata: tuple
    data_version: str


def validate_index_record(record, file_specification, limits):
    # Check units, alignment, overflow, version, and complete byte bounds.
    ...


def fetch_variable_record(mapping, record, ownership_contract):
    # Construct only the supported bounded sample representation.
    ...


def collate_variable_samples(samples, batch_policy):
    # Bound packing/padding and preserve lengths, masks, and labels.
    ...


def E08(config):
    # Validate data/index agreement and outlier-batch admission.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require matching sample content, lengths, masks, and labels, not only total file size. Empty records must not create invalid reads or misleading pointer identity. A compact on-disk index does not prove compact resident Python metadata. Reject stale or corrupt manifests before source access. Preserve outlier refusal and unsupported shapes explicitly instead of truncating records silently to make a batch fit.

Bounds checks must cover the full addressed record, not only its first element. Validate arithmetic before converting to a narrower integer field. If overlapping ranges are supported, record whether the overlap is intentional shared content or unrelated logical samples; the consumer's mutability contract must preserve the authoritative bytes for both.

Part 7 - Engineering Interpretation and Philosophy

Variable-length access makes metadata part of the data path. The store is reliable when every record can be translated and bounded independently and when batching costs remain visible rather than being hidden behind a lazy source interface.

19.33.10 Exercise 09

Exercise Title: Build a DataLoader Around a Memory-Mapped Dataset and Verify Safe Access Across Multiple Worker Processes

Part 1 - Exercise Contract and Why It Matters

Use a mapped dataset with multiple DataLoader worker processes and verify correct sample coverage, process-local source opening, reproducible transforms, and bounded batch transfer. Explicitly select and test supported process-start behavior instead of assuming one platform default.

Part 2 - System and Memory Mental Model

Workers are separate processes with their own dataset state and runtime resources. Start methods can inherit or serialize objects differently and can vary with platform and Python version. Keep datasets and initialization/collation functions picklable where needed and use a guarded main entrypoint. Open or reopen mappings in the consuming process using lightweight source metadata. Return CPU samples and collated batches; CUDA transfer belongs in the main training process under the initial contract.

Part 3 - Quantitative Prediction Before Execution

For W workers and configured prefetch depth p, estimate queued batch payload from Wpb as a starting component, then add active workers, main-process batches, IPC, metadata, and optional pinning separately. Predict shared-source page accounting versus duplicated Python state. Specify expected IDs for a map-style epoch and distinguish it from worker-sharded iterable behavior. Predict which failures arise when live mapping objects are blindly serialized.

Part 4 - Experimental Design and Instrumentation

Begin with one process and validate all tiny samples. Complete a top-level worker-safe dataset, process-local mapping cache, pickling state, collator, and explicit start-method configuration. Test zero, one, and several workers, with bounded batches materialized before IPC. Verify sample IDs, labels, values, and transform seeds across epochs. Record worker PIDs, open/reopen events, resource observations, persistent-worker policy, and shutdown. Test worker failure on a controlled bad manifest and preserve its useful diagnostic without launching unsafe reads or CUDA work in workers.

Exercise at least one explicitly configured spawn-compatible run where the platform supports it, using top-level definitions and a guarded entrypoint. Audit whether opening occurs in the parent, worker, or both, with PIDs recorded. A lightweight path/schema object should reconstruct local access without serializing an entire live mapping. Include a second epoch with persistent workers to verify source identity and per-epoch randomness do not silently become stale.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
from torch.utils.data import Dataset, DataLoader

class WorkerMappedDataset(Dataset):
    def __init__(self, schema):
        self.schema = schema
        # Keep only serializable metadata and process-local lazy state.
        ...

    def __getstate__(self):
        # Exclude live process-owned mappings from serialized state.
        ...

    def __len__(self):
        ...

    def __getitem__(self, index):
        # Reopen locally when required and validate bounded access.
        ...


def worker_initialize(worker_id):
    # Set required independent RNG state and process-local resources.
    ...


def E09(config):
    # Use a guarded script entrypoint and verify coverage/cleanup.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not assume multiple workers multiply unique physical source pages by W, or that all Python metadata is shared. Default collation can create a batch copy; account for it. With disabled batching or a passthrough collator, returning views of huge source storage needs an explicit IPC/lifetime design. Preserve dropped-last and duplicate-ID policies so apparent throughput cannot hide missing or repeated data.

Compare complete epoch ID multisets with the sampler's declared expectation. Process scheduling can change completion order without justifying lost or duplicated samples. Keep worker-local file resources alive through batch construction and resolve them at shutdown; do not rely on a destructor's unobserved timing as the only correctness argument.

Part 7 - Engineering Interpretation and Philosophy

Safe multiprocessing starts from explicit process ownership and sample semantics. A mapped source can be shared physically while each process still needs correct metadata, lifecycle, randomness, and bounded independent outputs.

19.33.11 Exercise 10

Exercise Title: Compare num_workers Values on a Memory-Mapped Dataset and Identify When Storage Bandwidth, Page Faults, or CPU Processing Become the Bottleneck

Part 1 - Exercise Contract and Why It Matters

Sweep DataLoader worker count and identify the measured limiting stage for a mapped dataset. Compare storage demand, page activity, transforms, collation, IPC, and consumer waits. Preserve complete sample coverage and queue capacity so a larger worker count does not silently change the workload.

Part 2 - System and Memory Mental Model

Workers can overlap useful preparation but can also compete for storage, CPU cores, memory bandwidth, and cache. More workers increase queue and metadata resources and can worsen random access. A low observed GPU utilization is not by itself proof of storage starvation; compute size, transfer dependencies, and host synchronization can produce similar symptoms. Worker count and prefetch depth are separate policy dimensions, and total in-flight payload must remain part of the comparison.

Part 3 - Quantitative Prediction Before Execution

Construct a stage model for source access, transform, collation/IPC, transfer, and GPU compute. Predict throughput bounded by the slowest effective stage, including contention rather than summing nominal capacities. For each worker count estimate queued/active batch bytes and useful samples per second. Predict a storage-heavy trace and a CPU-transform-heavy trace, then state the observations that would distinguish bandwidth saturation from CPU saturation or excess page churn.

Part 4 - Experimental Design and Instrumentation

Freeze sample order, batch size, transforms, useful consumer work, and seeds. Sweep a safe small set of worker counts with explicit valid zero-worker configuration. Run source-only and transform-heavy controls, then the full pipeline. Record worker CPU time, batch-ready latency, supported storage/page metrics, main-process waits, pinned capacity, and completed useful throughput. Measure worker startup separately from steady epochs and document persistent workers. Use equal useful data and preserve queue changes as measured policy costs. Repeat trial order and cache context without selectively discarding slow runs.

Add one trial holding approximate in-flight bytes constant while changing worker count, and another preserving the normal prefetch policy so total queue cost can vary. These are different policy questions and should have separate tables. Observe main-thread batch waits and worker preparation times together; an aggregate samples-per-second number cannot reveal whether extra workers hide I/O or merely move contention into another queue.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
```

```
class WorkerSweepResult:
    workers: int
    prefetch_depth: object
    completed_samples: int
    batch_wait_samples: tuple
    host_peak_bytes: object
    storage_evidence: object
    cpu_evidence: object
    complete_seconds: object


def configure_worker_trial(dataset, case, support_contract):
    # Handle zero-worker options and bounded in-flight resources explicitly.
    ...


def diagnose_pipeline_limit(stage_evidence, controlled_replays):
    # Separate directly measured facts from bottleneck hypotheses.
    ...


def E10(config):
    # Compare worker counts on equivalent source/transform/full workloads.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

A higher sample rate is invalid if IDs are duplicated, dropped unexpectedly, or transformed differently. Compare trial duration through completed device work, not only loader iteration submission. Fault counts alone do not establish disk contention, and CPU occupancy alone does not establish useful transforms. Preserve the smallest worker count attaining stable measured performance and report the uncertainty when instruments cannot separate competing explanations.

Report CPU transform threads as well as worker processes where relevant. Nested thread pools can oversubscribe cores even at a modest worker count. Do not infer a storage bottleneck from low throughput without a controlled source-only or resident-data comparison. Keep the alternative explanation visible until supporting evidence resolves it.

Part 7 - Engineering Interpretation and Philosophy

Concurrency is beneficial only while it hides a real bottleneck without creating a worse one. Choose worker count from complete stage evidence and resource cost, not from available core count or the assumption that more parallel readers always feed the GPU faster.

19.33.12 Exercise 11

Exercise Title: Combine Memory-Mapped Dataset Access with Pinned Host Buffers and Non-Blocking CUDA Transfers

Part 1 - Exercise Contract and Why It Matters

Stage mapped samples into bounded pinned host buffers and transfer them to CUDA with explicit completion ordering. Demonstrate correct values, safe buffer reuse, and complete latency. Identify every copy instead of labeling the path zero-copy because the original source is mapped.

Part 2 - System and Memory Mental Model

File-backed tensor storage is not directly pinned by the documented raw-file mapping API. Collation and pinning can create independent batches before host-to-device transfer. A pinned source must not be overwritten while its transfer is in flight, and a CUDA destination must not be reused while readers remain. Nonblocking submission does not prove completed transfer or actual overlap. Automatic DataLoader pinning and a custom reusable pinned pool are different strategies whose costs should not be silently combined or omitted.

Part 3 - Quantitative Prediction Before Execution

For batch payload b and K staging slots, predict pinned capacity Kb plus any collated source batch and device slots. Count source-to-batch, batch-to-pinned, and pinned-to-device traffic under the selected route. Draw the timeline for one serialized slot and a two-slot ring. Estimate copy/setup overhead and potential overlap limits, keeping the source mapping's resident pages separate from locked host storage and VRAM.

Part 4 - Experimental Design and Instrumentation

Start with deterministic small CPU batches and a serialized reference. Complete one explicit staging strategy with generation tags and readiness/completion events, then compare it with a supported automatic-pinning baseline. Verify checksums only after completion and before overwrite. Measure setup, fill, transfer, GPU consumption, waits, and complete time consistently. Apply bounded backpressure when slots are unavailable and test delayed transfer completion. Keep CUDA work in the main process, and record whether custom batch types support the selected DataLoader pinning behavior.

Use a host-side lease simulator to validate one-slot and two-slot transfer schedules before CUDA execution. A new batch generation cannot inherit a stale completion token from an older use of the same slot. Check a delayed consumer and early-stop sequence, including all retained source owners. If a custom pool replaces automatic pinning, ensure automatic pinning is not still adding an unintended independent pinned copy.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class BatchTransferLease:
    batch_id: int
    slot_id: int
    generation: int
    source_owner: object
    copy_done: object
    consumer_done: object
```

```
def fill_pinned_slot(mapped_batch, lease, pool_contract):
    # Admit a safe host destination and account for the staging copy.
    ...


def transfer_and_consume(lease, streams, device_slots):
    # Order H2D and compute; protect source and destination lifetimes.
    ...


def E11(config):
    # Verify values and compare serialized versus bounded staged execution.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require no host overwrite before H2D completion and no device overwrite before consumer release. Host state-machine tests should reject illegal transitions without intentionally racing memory. Report pinned bytes and any double staging introduced by configuration. If CUDA or pinning support is unavailable, keep the case untested or unsupported. A faster submission timer must not be presented as higher completed transfer bandwidth.

Measure copied payload and retained capacity independently. The same pinned buffer can carry many generations, so cumulative transfer bytes can be much larger than pool size. Conversely, allocating a new pinned tensor for every queued batch can grow retained capacity even when each transfer is correct; report that ownership cost explicitly.

Part 7 - Engineering Interpretation and Philosophy

Pinning creates a controlled transfer resource with its own cost and lease. The useful pipeline makes ownership visible from file view through batch copies to the final device consumer, then measures whether that extra staging actually reduces critical-path waits.

19.33.13 Exercise 12

Exercise Title: Build a Dataset Prefetcher That Reads Future Memory-Mapped Regions While the GPU Computes the Current Batch

Part 1 - Exercise Contract and Why It Matters

Prefetch future mapped regions or prepared batches while the GPU consumes the current batch. Implement bounded lookahead, explicit readiness, and cancellation. Separate application prefetch from OS readahead and DataLoader's own worker prefetch to avoid unbounded overlapping queues.

Part 2 - System and Memory Mental Model

A prefetch request can prepare source pages, decode/transform samples, create collated storage, fill pinned buffers, or submit a transfer. These stages have different readiness and memory costs. The OS need not keep prefetched pages resident until use, and asynchronous hints do not guarantee useful overlap. A CUDA consumer must wait for actual batch readiness.

Abandoned future work still needs safe source/staging cleanup, especially if transfer or compute has already begun.

Part 3 - Quantitative Prediction Before Execution

For current compute time T_c and next-batch preparation time T_p, model an ideal exposed wait as $\max(0, T_p - T_c)$ under independent resources. Add queue contention and transfer dependencies explicitly. Calculate payload for current, prefetched, collated, pinned, and device batches at the chosen lookahead. Predict behavior for a slow storage burst and an early epoch stop. Compare logical prefetched bytes with bytes ultimately used by the workload.

Part 4 - Experimental Design and Instrumentation

Use a fixed sample trace and demand-only reference first. Complete one bounded prefetch layer while recording existing worker and OS policies. Publish a batch only after all required preparation and transfer dependencies are valid. Compare one- and two-batch lookahead where budgets allow, using supported timeline evidence and completed throughput. Test early termination, a failed source read, delayed GPU consumption, and shutdown. Preserve sample order, labels, and no-duplication guarantees. Measure unused preparation and queue residency rather than counting speculative bytes as useful data.

Label readiness by stage: source accessed, transformed, collated, pinned, H2D submitted, or device-ready. A single generic ready flag can authorize a later consumer too early. Record the stage that the prefetcher actually owns and define how it hands off to existing loader and transfer components. Use a fixed queue capacity in both successful and cancelled cases rather than allowing exception paths to retain unlimited prepared batches.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class DatasetPrefetchTicket:
    batch_id: int
    sample_ids: tuple
    stage: str
    reserved_host_bytes: int
    ready_token: object
    cancelled: bool

class BoundedDatasetPrefetcher:
    def request(self, sample_ids):
        # Join existing demand or admit bounded future preparation.
        ...

    def publish(self, ticket, batch, readiness):
        # Preserve order and make completed stage semantics explicit.
        ...
```

```
    def consume(self, ticket, consumer_stream):
        # Wait on the actual generation/stage dependencies.
        ...

    def shutdown(self, deadline):
        # Drain or cancel without freeing in-flight resources early.
        ...


def E12(config):
    # Compare useful work, exposed waits, residency, and unused prefetch.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require equivalent completed sample IDs and correct values. An application prefetch queue must not secretly multiply the loader's already queued batches beyond budget. A page hint alone cannot authorize CUDA use. Preserve failures and cancellation as explicit outcomes, settling each resource exactly once. If no measurable overlap occurs, report the tested stages and explain the evidence rather than assuming the tool failed to observe a promised speedup.

A completed batch may still wait for its ordered turn; retain that state separately from a failed preparation. If a later batch finishes first, preserve the declared ordering or explicitly label a changed ordering contract. Useful prefetch analysis must include head-of-line waits and unused work, not only the latency of the fastest future request.

Part 7 - Engineering Interpretation and Philosophy

Prefetch spends resources early to reduce a later wait. It is useful when measured locality and preparation overlap compensate for extra residency and unused work under a bounded lifecycle, not simply when more data is queued.

19.33.14 Exercise 13

Exercise Title: Compare Sequential Sampling, Fully Random Sampling, and Locality-Aware Shuffling and Measure Their Effects on Storage Throughput and GPU Idle Time

Part 1 - Exercise Contract and Why It Matters

Compare sequential order, full random permutations, and locality-aware shuffling for storage behavior, batch readiness, and GPU idle intervals. Preserve epoch coverage and distinguish systems efficiency from the statistical consequences of changing training order.

Part 2 - System and Memory Mental Model

Sequential order can exploit physical locality but may follow labels or time if the file was sorted. Fully random order can disrupt storage locality while improving some mixing properties. Locality-aware order trades between those objectives and is not automatically equivalent to a uniform permutation. Different SGD orders can produce different updates even with identical sample coverage. GPU inactivity needs stage evidence before being attributed to data starvation.

Part 3 - Quantitative Prediction Before Execution

For tiny indexed samples, compute page/locality groups and the number of region transitions under each order. Predict useful bytes, repeated page coverage, batch-wait intervals, and transfer/compute overlap. Define equal coverage and drop-last policy, then specify training-quality measurements separately from exact per-sample correctness. Predict a label-sorted dataset where excellent sequential throughput could hide a harmful ordering effect.

Part 4 - Experimental Design and Instrumentation

Generate all sample orders before timing with recorded seeds. Use identical batches sizes, transforms, worker settings, and useful compute. First compare source and full-pipeline stage evidence, then run a bounded training-quality comparison under stated epochs or examples and validation protocol. Record class/length composition, lagged order correlations, sample coverage, storage observations, batch-ready latency, and supported GPU idle evidence. Preserve cache context and raw trial variability. Do not infer model-quality equivalence from a histogram alone or call unmatched training trajectories numerical loading failures.

Include a tiny dataset sorted by class or time as a diagnostic ordering control, then a representative realistic ordering for the actual recommendation. Inspect per-batch composition and local adjacency without assuming either metric establishes statistical independence. Report comparisons under both fixed examples processed and, if useful, fixed elapsed budget as separate experiments. They answer different questions about efficiency and attainable training progress.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class SamplingComparison:
    policy: str
    seed: int
    sample_order: tuple
    locality_evidence: object
    batch_waits: tuple
    gpu_idle_evidence: object
    quality_metrics: object


def build_sampling_orders(index, policies, epoch_contract):
    # Preserve coverage while making each distribution explicit.
    ...


def compare_sampling_pipelines(orders, workload, config):
    # Separate system throughput from statistical/optimization outcomes.
    ...


def E13(config):
```

```
    # Compare sequential, random, and locality-aware policies honestly.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Reject missing/duplicate IDs unless the sampler contract explicitly permits them. GPU utilization percentages alone cannot quantify data-wait time. Preserve length-dependent padding and class imbalance as possible confounders. A locality policy that changes sample coverage or useful training work cannot claim pure I/O acceleration. Retain adverse validation outcomes even if throughput improves, and report uncertainty where the tested training budget cannot distinguish quality effects.

Preserve the exact seed schedule and order trace for reproducibility, while acknowledging that changing order intentionally changes the training trajectory. The numerical data-reader check should compare the same sample values; the learning comparison should evaluate the resulting model under a separate validation protocol rather than demand identical SGD gradients.

Part 7 - Engineering Interpretation and Philosophy

Sampling order is both a systems policy and a learning choice. Recommend a tradeoff only after measuring locality, useful pipeline progress, and the relevant training behavior instead of assuming one convenient shuffle offers all benefits without consequences.

19.33.15 Exercise 14

Exercise Title: Build a Page-Locality-Aware Batch Sampler That Reduces Random File Access While Preserving Useful Training Randomness

Part 1 - Exercise Contract and Why It Matters

Build a page-locality-aware batch sampler with explicit randomization and exact epoch coverage. Group nearby sample regions while varying window order and sample order reproducibly. Measure locality and useful training behavior rather than claiming page grouping preserves every property of full random sampling.

Part 2 - System and Memory Mental Model

A sample can span several pages, and variable-length records can overlap multiple locality windows. Page grouping therefore needs a deterministic primary assignment or another explicit policy to avoid duplicates. A page-size estimate is a locality heuristic, not an exclusive OS-residency contract. Randomizing windows and within-window order can leave correlations related to file layout, labels, lengths, or time. The sampler must make those limitations inspectable and keep batch formation compatible with the loader.

Part 3 - Quantitative Prediction Before Execution

Using observed page size P, derive each sample's start page and page span from byte metadata. Build a tiny hand-checkable assignment with samples crossing boundaries and calculate exact sample coverage. Predict batch payload, padding, page-span spread, and window transitions. Define the random variables used each epoch and their seed schedule. Compare these properties with a full permutation without asserting equal distributions or guaranteed convergence.

Part 4 - Experimental Design and Instrumentation

Implement the sampler independently of data reads first. Validate a bijection between expected IDs and emitted IDs under the declared last-batch policy. Test overlapping records, long outliers, empty windows, partial batches, epoch restart, and fixed-seed reproducibility. Integrate through a supported batch sampler configuration rather than simultaneously supplying conflicting shuffle options. Compare page-locality evidence and full pipeline throughput. On a small representative training task, inspect class/length mixing and validation quality under equal useful work. Store compact metadata instead of a per-byte or per-page Python object for every source element.

Test epoch restart and checkpoint-resume semantics on the sampler itself. Record enough seed, epoch, and position metadata to reproduce the intended remaining batch order without silently repeating already consumed examples. If resume is unsupported, state that limit. Compare compact window metadata with a full explicit permutation, charging both construction time and host bytes rather than assuming locality bookkeeping is free at very large scale.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
from torch.utils.data import Sampler

@dataclass
class LocalityWindow:
    window_id: int
    sample_ids: tuple
    first_page: int
    last_page: int

class PageLocalBatchSampler(Sampler):
    def __init__(self, index, batch_policy, window_policy, seed):
        # Build compact deterministic ownership and epoch configuration.
        ...

    def set_epoch(self, epoch):
        # Select reproducible epoch-specific randomization.
        ...

    def __iter__(self):
        # Emit batches with exact declared coverage and bounded payload.
        ...

    def __len__(self):
        # Match the actual batch/drop policy rather than a rough guess.
        ...
```

```
def E14(config):
    # Validate sampler invariants before evaluating system/quality tradeoffs.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require no false coverage from assigning a spanning sample to several windows. A reproducible seed alone does not prove adequate training mixing. Report window, batch, and length policies with every result. Reject oversized batches or define an explicit outlier route; silent truncation changes the dataset. If locality gains vanish on a cached workload, retain that result rather than forcing an artificial general conclusion.

Check class and length correlations within windows and across their boundaries. A window permutation with within-window shuffle is not a full uniform permutation, even when each ID appears once. Report its actual distributional contract and require task-appropriate evidence before calling the available mixing useful for training.

Part 7 - Engineering Interpretation and Philosophy

A locality sampler should offer a controlled and measurable ordering tradeoff. Its credibility rests on exact sample semantics and transparent randomization, with learning-quality evidence appropriate to the actual task rather than an unsupported promise of unchanged randomness.

19.33.16 Exercise 15

Exercise Title: Load a Large PyTorch Checkpoint with torch.load(..., mmap=True) and Process Individual Tensors Without Eagerly Loading Every Storage into RAM

Part 1 - Exercise Contract and Why It Matters

Load a supported PyTorch checkpoint with mapped storage and process selected tensors without eagerly creating an independent copy of every storage. Distinguish this archive-aware loading path from a raw dataset mapping and measure mapping, access, conversion, and retained ownership separately.

Part 2 - System and Memory Mental Model

A PyTorch checkpoint contains metadata and serialized storages under a defined format; it is not a headerless raw tensor file. Mapping can defer storage page access while metadata still loads. Explicit CPU map location avoids unintended restoration to saved CUDA devices. Shared storage and views can make several state names refer to one payload. Processing a tensor by cloning, converting dtype, or moving it to CUDA creates additional resources and reads source values, so lazy loading is not free materialization.

Part 3 - Quantitative Prediction Before Execution

For named tensors sharing canonical storage, calculate logical named bytes and unique payload separately. Predict resources at mapping return, selected tensor processing, full scan, and release. Include conversion output, source pages, and transfer staging where used. Explain why deleting one state-dictionary entry may not release storage held by another alias or final model parameter. A lower immediate load time is not a completed full-checkpoint processing speedup.

Part 4 - Experimental Design and Instrumentation

Generate a trusted tiny plain tensor checkpoint with a view and tied storage, plus a larger bounded fixture in the supported format. Complete mapped CPU loading with an explicit deserialization policy, validate schema, and process one selected group at a time. Compare outputs with ordinary eager loading on equivalent selected work. Track source owners, aliases, memory phases, and observed page/I/O activity where available. Test unsupported mapping inputs without silent eager fallback. Preserve immutable source files until all live mapped owners and pending copies finish.

Use a checkpoint containing two views with distinct element offsets into one storage. Process them independently and inspect whether source references still share a canonical owner. A conversion can break that sharing and create additional payload; record whether preserving the original alias relationship is required. Keep the checkpoint immutable during the primary comparison so private dirty pages or shared writes do not alter the access experiment.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class CheckpointTensorRequest:
    name: str
    expected_shape: tuple
    expected_dtype: object
    target_device: object
    copy_policy: str


def open_mapped_checkpoint(path, format_contract):
    # Use explicit CPU placement and restricted supported loading options.
    ...


def process_checkpoint_group(state, requests, ownership_ledger):
    # Account for aliases, conversions, targets, and source lifetimes.
    ...


def E15(config):
    # Compare selected and full processing at matching completion boundaries.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require correct selected values and preserve state aliases where the contract needs them. A private mapped update can cause copy-on-write memory and must not be assumed persistent or free. Do not apply a raw file offset formula to the checkpoint archive without validated

format support. If every tensor is scanned during validation, record that full access instead of claiming only a small group was touched.

A state dictionary is not necessarily composed only of independent floating weights. Validate buffers, scalar tensors, and the supported layout/dtype set. Weights-only loading describes a restricted deserialization policy, not removal of buffers or a guarantee that every arbitrary checkpoint schema can be served by the same bounded reader.

Part 7 - Engineering Interpretation and Philosophy

Archive-aware mapping lets useful tensor processing begin before an eager full independent payload is needed. Its value depends on the access schedule and final ownership, with metadata, file pages, conversions, and aliases all remaining part of the account.

19.33.17 Exercise 16

Exercise Title: Inspect a Large Checkpoint with FakeTensorMode and Extract Tensor Shapes, Dtypes, Strides, and Storage Offsets Without Allocating the Real Tensor Data

Part 1 - Exercise Contract and Why It Matters

Inspect a compatible checkpoint under FakeTensorMode and extract structural tensor metadata without allocating its real payload. Separate element offsets within storage from optional byte offsets within the checkpoint archive. Treat private inspection fields as feature-gated diagnostics, not universal portable reconstruction APIs.

Part 2 - System and Memory Mental Model

Fake tensors describe metadata and logical device behavior without supplying real element values. They are not ordinary resident CPU weights, and their visible device need not be a simple meta label. The documented inspection recipe uses an underscore-prefixed import path, while checkpoint byte-offset annotations are also version-sensitive. A tensor storage offset is measured in elements relative to its storage; a checkpoint storage offset addresses archive bytes. Shared views require both layers to interpret locations.

Part 3 - Quantitative Prediction Before Execution

For a tiny dense tensor view, calculate shape, strides, element width, and its logical storage displacement. If a supported archive storage-byte offset is available, derive the first addressed payload location under the declared uncompressed dense-format assumptions. State why that does not alone validate arbitrary strided span, sparse layout, compression, endianness, or file integrity. Predict metadata memory scaling with tensor count rather than logical element count and distinguish no payload allocation from zero process overhead.

Part 4 - Experimental Design and Instrumentation

Create a trusted small checkpoint with contiguous tensors, an offset view, and shared storage. Feature-check FakeTensorMode and optional offset annotations. Complete inspection and export only CPU scalar metadata, marking unavailable fields explicitly. Compare with ordinary loading on the tiny reference for shapes, strides, dtypes, and storage relationships. Do not extract numerical values from fake tensors. For large inspection, measure metadata cost without performing an accidental real load. Preserve format/version and source identity, and test unsupported cases through safe classification rather than blind byte reads.

Build a metadata unit test with a base storage, an offset slice, and a transpose. Check the element-displacement formula independently on the tiny real reference before considering any

archive location. Export the presence or absence of optional annotations explicitly, preserving the software and format version that produced them. For a large report, stream metadata rows or cap retained details instead of constructing unbounded Python diagnostic objects.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class CheckpointMetadata:
    name: str
    shape: tuple
    strides: tuple
    dtype: object
    storage_offset_elements: int
    checkpoint_storage_offset_bytes: object
    inspection_support: str


def inspect_fake_checkpoint(path, feature_contract):
    # Feature-check the documented diagnostic mode before opening.
    ...


def extract_fake_metadata(state, source_manifest):
    # Keep element offsets and optional archive byte offsets distinct.
    ...


def E16(config):
    # Compare tiny real metadata without using fake tensors as values.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Unknown checkpoint offsets must not become zero. Do not infer real payload residency or validity from a fake tensor pointer or device string. An offset annotation is not permission to mutate arbitrary archive bytes. If a private field changes across versions, retain a partial metadata report rather than inventing compatibility. Numerical equality requires a separate legitimate real load; metadata inspection alone cannot verify checkpoint contents.

The documented fake inspection path may change because it uses internal names. Feature checks should distinguish missing import, unsupported archive, and absent optional storage annotation. Do not catch all three and emit a fabricated valid offset. A partial shape/dtype report can remain useful without claiming complete raw payload reconstruction support.

Part 7 - Engineering Interpretation and Philosophy

Structural inspection is valuable before committing real storage, provided its limits remain visible. The strongest output is a versioned metadata report with exact units and honest unsupported fields, not a claim that fake values or private offsets replace a validated data reader.

19.33.18 Exercise 17

Exercise Title: Build a Unified File-Backed Tensor Store That Serves Dataset Samples and Model Tensors Through Byte Offsets, Metadata, and Lazy Materialization

Part 1 - Exercise Contract and Why It Matters

Build a unified metadata and ownership interface for dataset records and checkpoint tensors, while preserving separate format-specific readers. Serve bounded views or lazy materialized results using validated byte metadata, source versions, and explicit leases. Do not flatten every file format into one unchecked raw-address operation.

Part 2 - System and Memory Mental Model

Raw dense sample files, variable-length indexes, and serialized checkpoints have different layout contracts. A common record can describe source identity, format, dtype, shape, strides, logical offset, and ownership, but an archive adapter must still respect its supported schema. Endianness conversion, noncontiguous views, aliases, pinning, and device placement can require copies. Mapped owners and outstanding transfers must survive while returned values still depend on them. Read-only source usage is an application contract.

Part 3 - Quantitative Prediction Before Execution

Define a resource model for mapped source residency, compact metadata, independent output bytes, pinned staging, and target device bytes. Calculate span bounds for a supported dense record and reject malformed ranges before access. For two aliased checkpoint names, distinguish logical record count from unique source owner count. Predict a request that returns a view, one requiring conversion, and one requiring CUDA placement, including each copy and completion obligation.

Part 4 - Experimental Design and Instrumentation

Implement raw-dataset and checkpoint adapters behind one validated request schema. Use tiny authoritative fixtures for exact values and alias checks. Validate source/version, format support, range arithmetic, dtype, shape, and layout before creating outputs. Complete lease acquisition, lazy access, conversion, transfer, and release with bounded caches. Test stale manifests, unsupported endianness/layout, shared owners, cancellation, and shutdown. Benchmark equivalent consumer requests while reporting adapter-specific costs. Keep source files immutable and preserve mapped owners until their final leases and device dependencies are discharged.

Give each adapter a clear decision between borrowed view and independently owned output. The unified store must preserve that decision in its lease so a consumer cannot assume all results are writable. Track source-version and canonical-owner keys independently of record names, and test two requests sharing storage alongside two equal-valued but independent records. Equality of contents is not an ownership identity test.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class FileBackedRecord:
    record_id: str
    source_id: str
    format_id: str
    dtype: object
    shape: tuple
    strides: tuple
    offset_contract: object
    canonical_owner: str

class FileBackedTensorStore:
    def register_source(self, manifest, adapter):
        # Validate immutable source identity and format support.
        ...

    def acquire(self, record_id, target_contract):
        # Return a bounded leased view or charged materialized result.
        ...

    def release(self, lease, completion):
        # Protect source owners, staging, and target readers correctly.
        ...

    def shutdown(self, deadline):
        # Stop admission and resolve live mappings and transfers safely.
        ...


def E17(config):
    # Verify both adapters and reconcile all ownership/copy boundaries.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require the same values as each format's legitimate independent reader. A common interface cannot bypass bounds or archive compatibility checks. Do not identify raw byte offsets, tensor storage offsets, and checkpoint archive offsets as interchangeable fields. Report unknown ownership or unsupported geometry as refusal. A source cache hit must not double-count mapped payload, while a real conversion must not disappear from allocation and latency totals.

Avoid interpreting a supported diagnostic archive offset as blanket support for direct raw mapping of every checkpoint. Ordinary archive-aware loading can remain the adapter's correct

path. Report any copying or decoding that path requires instead of discarding format checks to meet an artificial zero-copy target.

Part 7 - Engineering Interpretation and Philosophy

Unification belongs in the request, metadata, and ownership contracts, not in pretending all files have one layout. A reliable store keeps format knowledge local while exposing every materialization and lifetime cost to the consumer.

19.33.19 Exercise 18

Exercise Title: Stress-Test Memory-Mapped Training on a Dataset Larger Than Available RAM and Diagnose Page Thrashing, Random-I/O Bottlenecks, Worker Contention, and GPU Starvation

Part 1 - Exercise Contract and Why It Matters

Stress-test training access when dataset payload exceeds an explicitly stated available-memory reference. Diagnose page churn, storage limits, worker contention, and GPU waits using controlled replays. Preserve learning work and process stability instead of deliberately exhausting the machine to obtain a dramatic failure.

Part 2 - System and Memory Mental Model

A file larger than RAM can be processed with bounded batches, but access locality and active working set determine repeated I/O. Random revisits can cause refaults, while worker queues and pinned buffers consume additional nonreclaimable or anonymous resources. Major faults, swap activity, and device utilization each have limited diagnostic scope. A GPU can be idle for reasons other than storage. Sparse files or highly compressible fixtures can also misrepresent the actual storage path.

Part 3 - Quantitative Prediction Before Execution

Calculate logical dataset size and compare it with a documented physical or isolated-workload allowance, specifying which is used. Estimate batch/index/queue/ pinned/device demand separately from source pages. Predict sequential one-pass, random-revisit, oversubscribed-worker, and CPU-heavy-transform traces. State which measurements would support thrashing rather than simply a slow first scan. Define a minimum supported active set and classify budgets below it as infeasible rather than claiming arbitrary low-memory training.

Part 4 - Experimental Design and Instrumentation

Preflight real disk capacity and choose an isolated bounded test environment. Generate or use an actual payload with allocation/compression properties recorded. Start with a smaller exact reference, then increase dataset or access working set safely. Freeze useful sample count, transform, model, batch, and training state for causal comparisons. Capture stage timelines, faults, supported I/O/reclaim observations, host and pinned peaks, worker resources, GPU waits, and completed updates. Vary one suspected cause at a time and minimize reproducing failures. Preserve abort limits, unavailable metrics, and unexecuted oversized cases honestly.

If an isolated memory limit is used, record its scope and whether file cache, anonymous batches, and pinned pages are charged by the environment. Do not assume it is equivalent to reducing physical installed RAM. Keep a minimum active-set estimate and stop before an unsupported configuration endangers the test host. Source generation and full verification may warm the file, so describe their relation to each stress replay explicitly.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class OutOfRamStressCase:
    trace_id: str
    dataset_payload_bytes: int
    memory_reference_bytes: int
    access_pattern: str
    workers: int
    queue_policy: tuple
    stop_limits: tuple


def preflight_stress_case(case, storage_manifest, limits):
    # Bound disk, metadata, batch, staging, and runtime commitments.
    ...


def run_stress_training(case, pipeline, model, instruments):
    # Preserve useful updates and record causal stage evidence.
    ...


def diagnose_memory_io_incident(evidence, controlled_replays):
    # Separate cache misses, thrashing, CPU limits, and transfer waits.
    ...


def E18(config):
    # Report completed scope, aborts, failures, and minimized explanations.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not call a software queue cap proof that the dataset exceeded physical RAM. State the actual memory reference and test environment. Preserve samples, labels, losses, and update counts so skipped work cannot explain a speedup. If no thrashing occurs, report that instead of inventing counters. An unavailable GPU leaves GPU-starvation claims unmeasured, while CPU-only source behavior can still be analyzed within its scope.

Evidence for thrashing should involve repeated page/refault or reclaim pressure consistent with the trace and useful throughput loss, where supported counters exist. A slow initial scan alone is insufficient. If the experiment cannot observe refault behavior, present a qualified hypothesis and the controlled comparison that would test it.

Part 7 - Engineering Interpretation and Philosophy

Out-of-RAM feasibility comes from a bounded active pipeline and a tolerable access pattern. The stress test should establish where that design works, where repeated data movement dominates, and which measured intervention changes the cause without changing the intended training work.

19.33.20 Exercise 19

Exercise Title: Build and Benchmark a PyTorch Out-of-RAM Data Pipeline That Streams a Dataset Larger Than System Memory Through Memory Mapping, Prefetching, Pinned Buffers, and CUDA Transfers

Part 1 - Exercise Contract and Why It Matters

Integrate validated mapped sources, indexes, workers, locality sampling, prefetch, pinned staging, CUDA transfer, and measurement into a bounded out-of-RAM training pipeline. Preserve samples and useful updates while quantifying memory, I/O, waits, and throughput. State the tested dataset, platform, and resource envelope rather than promising universal streaming performance.

Part 2 - System and Memory Mental Model

Separate immutable source/version authority, process-local mappings, sample addressing, batch ownership, queue commitments, pinned leases, and device consumers. Mapping removes an eager full independent dataset copy, not every batch allocation or page-cache cost. Source reads, transforms, collation, IPC, pinning, transfer, and compute each have readiness boundaries. Sampling policy can alter optimization behavior; worker and prefetch policy can multiply resident copies. Every accepted request needs a safe release path, including early termination and failed workers.

Part 3 - Quantitative Prediction Before Execution

Define useful throughput as completed correct examples or updates divided by complete elapsed time. Predict source/metadata demand, independent queued batches, pinned capacity, device slots, page-locality behavior, and stage waits under the same workload contract. Compare eager feasible controls, mapped demand loading, worker tuning, and integrated prefetch/staging. Report startup, first batch, steady throughput, tail waits, and absolute resource peaks separately. If eager full loading is infeasible at the target size, retain an honest analytical or smaller control rather than pretending it ran.

Part 4 - Experimental Design and Instrumentation

Freeze format, transforms, seeds, order policy, batch/drop rules, model, precision, optimizer, and update count. Validate tiny fixtures and every component's ownership invariants before scaling. Run equivalent-data comparisons with independent output checks, then evaluate genuine larger-than-reference payload access under bounded limits. Use diagnostic traces separately from unprofiled timing. Include sequential/random/locality policies, variable lengths, multiple workers, delayed GPU consumers, early stop, malformed metadata, and controlled worker failure. Perform ablations without weakening sample correctness or hiding added copies. Publish all unsupported, refused, aborted, and failed cases alongside successful coverage.

Keep an explicit accounting inventory through source, sample, collated batch, IPC, pinned staging, device input, and output consumption. A resource can have several logical names but

one owner, or one logical batch can have several simultaneous physical copies. Audit those cases on a tiny trace before scaling. Check that shutdown stops new work and discharges existing leases without invalidating mapped views or outstanding DMA sources.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class OutOfRamPipelineContract:
    source_manifest_id: str
    format_id: str
    sampling_policy: str
    host_budget_bytes: int
    pinned_budget_bytes: int
    device_budget_bytes: int
    maximum_inflight_batches: int
    worker_policy: tuple

class OutOfRamDataPipeline:
    def __init__(self, contract):
        self.contract = contract
        # Initialize validated metadata, bounded queues, and ownership ledgers.
        ...

    def open_sources(self):
        # Establish process-local immutable mappings and format adapters.
        ...

    def plan_epoch(self, epoch):
        # Preserve declared coverage, seeds, locality, and batch semantics.
        ...

    def prepare(self, batch_request):
        # Bound source access, transforms, collation, IPC, and staging.
        ...

    def transfer(self, prepared_batch, consumer_stream):
        # Publish only correctly ordered generation-specific device values.
        ...

    def release(self, batch_lease, completion):
        # Reconcile source, host, pinned, and device obligations once.
        ...

    def audit(self):
```

```
        # Distinguish requested bytes, copied bytes, residency, and useful work.
        ...

    def shutdown(self, deadline):
        # Stop workers/admission and resolve live mappings and transfers.
        ...


def E19(config):
    # Validate correctness and bounded lifecycle before performance trials.
    ...
    # Export complete evidence and a scope-limited recommendation.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Acceptance requires correct samples/labels, complete useful updates, valid batch lifetimes, bounded commitments, and layer-specific memory/I/O accounting. Record source identity/size/allocation, schema, OS/filesystem, Python/PyTorch/CUDA versions, workers/start method, seeds, transforms, sampler, batch policy, queue depth, pinning, source/cache context, faults, supported physical I/O, RSS/PSS scope, pinned/device peaks, timing distributions, validation errors, coverage, and failure phases. Export bounded scalar evidence, not retained batch tensors or graphs.

Demonstrate a private-copy hazard prevented, a worker-safe reopen, a long-record outlier, a cancelled prefetch, a delayed transfer, and infeasible resource admission. Missing OS/GPU measurements remain unmeasured. No fabricated throughput, OOM, or thrashing result is acceptable.

Performance tables must identify whether their denominator includes worker startup, cache preparation, validation, transfer waits, and final GPU drain. Preserve raw phase samples so an apparent speedup can be traced to completed work. If the larger-than-memory target was not executed, distinguish the validated small-scale pipeline from the unmeasured target-scale claim.

Part 7 - Engineering Interpretation and Philosophy

A useful out-of-RAM pipeline makes large data accessible through a small controlled active set while preserving the computation that consumes it. Recommend the smallest measured configuration meeting the workload's memory and progress goals, and identify the next experiment needed before expanding format, worker, sampling, device, or scale support.

19.34 DMM32

Chapter Title: DataLoader Memory Optimization
Specialization: Dynamic Memory Management and CUDA Systems

19.34.1 Exercise 00

Exercise Title: Build a Precise Mental Model of DataLoader Memory Flow Across Dataset Storage, Worker Processes, Shared Memory, Pinned RAM, and GPU VRAM

Part 1 - Exercise Contract and Why It Matters

Build a lifecycle diagram for data moving from dataset authority through worker-local objects, batch collation, interprocess transport, optional pinning, CUDA transfer, and GPU consumption. Identify owners, possible copies, retention, and completion boundaries. Deliver a scoped memory ledger rather than one formula that treats every batch name as independent physical storage.

Part 2 - System and Memory Mental Model

DataLoader coordinates sampling and preparation but is not a universal memory budget enforcer. Workers can own independent metadata and caches while source pages or tensor transport storage are shared. Collated, pinned, and CUDA batches can coexist. Prefetch bounds outstanding work in batches under the configured policy, not a strict universal byte ceiling. Persistent workers extend process and dataset lifetimes. Host function return does not establish transfer or GPU consumer completion, and allocator caching differs from live tensor ownership.

Part 3 - Quantitative Prediction Before Execution

For workers W, prefetch factor p, and payload b, use Wpb only as an initial queued-work estimate when multiprocessing batching is active. Add worker working sets, active consumer batches, transport, pinning, and device slots with overlap checked. Draw one batch generation across the pipeline and predict which resources remain live when another batch is prefetched. Keep RSS, PSS, shared-memory occupancy, pinned payload, and VRAM as distinct measures.

Part 4 - Experimental Design and Instrumentation

Use a tiny deterministic dataset carrying IDs and labels, plus a bounded representative workload. Record process start method, framework versions, worker policy, batch schema, source authority, transforms, and device support. Capture constructor, iterator startup, first batch, warmed steps, epoch end, and final drain phases. Build an owner table and a completed-use timeline before collecting performance numbers. Export scalar evidence without retaining batch tensors or autograd graphs. Classify unsupported instruments explicitly, and distinguish predicted commitments from actual observed memory at each layer.

Draw the lifecycle for one identified batch twice: once with a single process and once with two workers and pinning enabled. Annotate the moment that each owner may release its reference and the event that permits reuse. Include a sample view that retains a larger backing storage, because logical payload size alone can hide retained capacity. Keep file cache observations outside process-owned allocation totals.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class BatchMemoryFlow:
    batch_id: int
    source_owner: str
```

```
    worker_id: object
    transport_state: str
    pinned_generation: object
    device_generation: object
    consumer_completion: object


def describe_batch_owners(batch_schema, loader_policy, execution_contract):
    # Identify shared ownership, independent copies, and pending readers.
    ...


def E00(config):
    # Reconcile the predicted lifecycle with bounded observed evidence.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not sum process RSS as an exact unique physical total or count pinned payload as an additional disjoint layer without checking where it is already included. A lower live counter after an epoch does not reveal its transient peak. Validate sample IDs and required updates so skipped work cannot explain apparent memory savings. Public queue configuration is not an exact observation of internal queue occupancy.

Reconcile the ledger at iterator creation, first delivery, steady consumption, and epoch completion. Explain why a worker's mapped pages, an IPC tensor, and a main-process view may describe overlapping physical resources. If unique ownership cannot be established, present separate scoped observations and an explicit uncertainty rather than subtracting an assumed shared fraction.

Part 7 - Engineering Interpretation and Philosophy

Memory optimization starts by naming what owns each useful value and when that ownership ends. A reliable pipeline model separates admission, preparation, transport, and completed consumption instead of assuming DataLoader settings translate directly into one predictable memory total.

19.34.2 Exercise 01

Exercise Title: Build a Single-Process DataLoader Baseline and Measure CPU RAM, Batch Preparation Time, GPU Idle Time, and End-to-End Throughput

Part 1 - Exercise Contract and Why It Matters

Build a zero-worker baseline that measures complete batch preparation, source/transform/collation costs where observable, host memory, transfer, and useful GPU work. Establish correctness and measurement boundaries before adding multiprocessing, pinning, or automatic tuning.

Part 2 - System and Memory Mental Model

With zero workers, sample fetching and collation run in the main process. The configuration must omit or disable options that require multiprocessing. A call waiting for the next batch can include source work, transforms, and collation; it is not automatically pure queue delay.

CUDA launches are asynchronous, so a short host call can coexist with unfinished device work. GPU idle time needs timeline or dependency evidence rather than inference from one utilization percentage.

Part 3 - Quantitative Prediction Before Execution

For batch payload b and N completed batches, calculate useful examples per complete elapsed time. Sketch preparation, H2D, forward/backward/update, and drain boundaries. Predict the initial serialized schedule and distinguish constructor time from first result and warmed throughput. Estimate host source, active batch, and device input capacity without treating cumulative bytes processed as peak memory. Declare whether validation and model/optimizer initialization belong to setup or useful work.

Part 4 - Experimental Design and Instrumentation

Use a supported deterministic dataset and fixed model state, dtype, batch policy, and seed. Complete a valid zero-worker loader with no multiprocessing prefetch or persistent workers. Validate sample/label order and a small reference update. Run diagnostic phase timing separately from low-overhead completed throughput trials. Synchronize only at declared measurement boundaries and preserve raw timings. Record CPU-only outcomes when CUDA is unavailable rather than inventing idle metrics. Sample or measure high-water host memory with its scope explicit and keep outputs bounded.

Use a deterministic sample generator with bounded dimensions before adding real storage. Save expected sample IDs and a checksum derived from consumed values so a faster loop cannot win by skipping preparation or GPU work. Decide whether first-use library initialization belongs to startup or warm execution, and apply that decision consistently to every later comparison.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
from torch.utils.data import DataLoader

@dataclass
class SingleProcessBaseline:
    completed_samples: int
    preparation_samples: tuple
    transfer_samples: tuple
    compute_samples: tuple
    complete_seconds: object
    peak_host_bytes: object
    idle_evidence: object


def build_single_process_loader(dataset, batch_contract):
    # Configure only supported zero-worker options.
    ...
```

```
def measure_baseline(loader, model, optimizer, config):
    # Separate diagnostic phase timing from complete useful-work trials.
    ...


def E01(config):
    # Verify data/update semantics and export the baseline evidence.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require the same useful step count and include final device completion in the wall interval. A profiler-enabled baseline must not be directly compared with an unprofiled candidate. Per-step synchronization changes overlap and must be labeled as diagnostic. A sampled host peak is an estimate, not a resettable process high-water counter. Preserve unavailable stage attribution and instrument overhead as limitations.

Measure an empty timing harness and a compute-only replay with already resident inputs. These controls describe instrumentation overhead and an input supply ceiling; neither is a replacement for complete pipeline time. Explain whether the baseline is constrained by preparation, transfer, or computation, and retain unsupported GPU idle estimates as unavailable observations.

Part 7 - Engineering Interpretation and Philosophy

The baseline is the reference contract for every later optimization. Its purpose is to make useful work and costs reproducible, so worker or transfer changes can be assessed against a complete computation rather than a faster but unfinished host loop.

19.34.3 Exercise 02

Exercise Title: Sweep num_workers from Zero to High Concurrency and Find the Point Where Additional Workers Stop Improving Throughput

Part 1 - Exercise Contract and Why It Matters

Sweep worker count from zero through a safely bounded high-concurrency range. Find the tested throughput plateau and the least costly configuration near it. Preserve sample coverage, transformations, batching, and complete model work while measuring concurrency overhead.

Part 2 - System and Memory Mental Model

Additional workers can hide source or CPU preparation latency but can also increase storage contention, process overhead, shared-memory demand, and queue capacity. Worker processes may themselves use multithreaded libraries, causing CPU oversubscription. Zero workers is a distinct valid configuration, not an ordinary point with every multiprocessing option unchanged. Persistent-worker, prefetch, and pinning policies must be controlled or reported because they change the resource question being answered.

Part 3 - Quantitative Prediction Before Execution

For each W, predict preparation parallelism, approximate outstanding batch payload, worker metadata, and useful throughput limits from the stage model. Compare a slow-source workload

with an expensive-transform workload. Define a plateau rule using repeated completed trials and variability, not the single fastest observation. Predict a case where more workers reduce GPU waits but increase host memory, and another where contention lowers throughput.

Part 4 - Experimental Design and Instrumentation

Preflight a bounded worker range and host/shared-memory headroom. Freeze sample trace, batch size, useful updates, transform semantics, and per-sample random choices for the primary comparison. Use valid options per worker count and record the process context. Capture startup and warmed epochs separately, with randomized trial order and raw samples. Collect main batch waits, worker CPU evidence, memory, transport pressure, and complete useful throughput. Preserve refused and failed high-concurrency configurations without repeatedly exhausting the host or changing the data to make them appear feasible.

Record worker process IDs and the number of runnable CPU threads observed within the trial. Include dataset transforms that invoke threaded native libraries, because process concurrency can multiply internal parallelism. Keep CPU affinity and thread settings documented and stable during the main sweep. Change them only in a separately labeled intervention with matching workload and sample coverage.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class WorkerCountCase:
    workers: int
    prefetch_factor: object
    persistent_workers: bool
    cpu_thread_policy: tuple
    expected_host_bytes: object


def configure_worker_count(dataset, case, support_contract):
    # Handle zero-worker restrictions and explicit process context.
    ...


def choose_plateau_configuration(results, tolerance_policy):
    # Use repeated completed trials and report uncertainty/cost.
    ...


def E02(config):
    # Compare causal workload controls and preserve all trial outcomes.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Reject a faster configuration that duplicates, drops, or differently transforms samples. Keep worker startup outside steady-state timing only when it is excluded for every candidate. GPU utilization alone does not locate the plateau's cause. Report nested thread counts and queue policy. The best tested worker count is conditional on this workload, platform, source, batch shape, and measured range; it is not a universal setting.

A noisy winner at the largest worker count does not establish a plateau. Repeat adjacent candidates and show the spread of complete-run throughput, startup latency, and memory peaks. Choose an operational point with explicit uncertainty and budget margin. Report early termination or a resource limit as a failed candidate rather than assigning an invented throughput value.

Part 7 - Engineering Interpretation and Philosophy

Concurrency should be increased until measured hidden latency no longer justifies its memory and scheduling cost. A useful recommendation names the smallest stable near-plateau worker count and the evidence explaining why further workers stop helping.

19.34.4 Exercise 03

Exercise Title: Measure Host RAM Growth as num_workers Increases and Identify Dataset Objects Replicated or Retained Across Worker Processes

Part 1 - Exercise Contract and Why It Matters

Measure host-memory growth across worker counts and identify which dataset objects, caches, mappings, and batch resources are replicated or retained. Use controlled representations to distinguish metadata costs from source payload and transport queues.

Part 2 - System and Memory Mental Model

Process-start behavior affects inheritance and serialization, while later Python access or mutation can change physical sharing. Large Python object graphs can cost much more than compact numeric metadata. File-backed pages can be shared even when each worker has its own mapping object. Parent and worker RSS are therefore not disjoint physical totals. Worker-local caches, transforms, prefetched batches, IPC buffers, and pinning must not be blamed on dataset replication without separate evidence.

Part 3 - Quantitative Prediction Before Execution

For W workers, compare a dataset backed by a Python list of records with a compact index and process-local file access. Predict object-count growth, possible sharing, and independent batch demand. Define measured fields for RSS, proportional/private memory where available, source residency, and queued payload. Distinguish constructor duplication from growth during first access and from retained state after several epochs.

Part 4 - Experimental Design and Instrumentation

Use identical tiny sample semantics and two explicit metadata representations. Test the chosen start methods where supported, recording Python version and PIDs. Measure parent-only construction, worker startup, partial access, complete epoch, and idle retained workers. Keep prefetch and transform policies fixed or subtract only directly measured components. Audit cache entry counts and source-opening behavior with bounded scalar logs. Compare tiny exact owner inventories against supported OS memory evidence. Avoid retaining full batches in telemetry or relying on raw interpreter reference counts as a physical memory oracle.

Construct two equivalent dataset indexes: a collection of individual Python objects and a compact representation with the same identifiers. Verify that both produce identical samples. Observe memory before workers start and after they touch most index entries, since process creation alone can miss later copying or retention. State whether the platform starts workers through fork, spawn, or another supported context.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class ProcessMemoryAudit:
    process_role: str
    process_id: int
    phase: str
    rss_bytes: object
    pss_bytes: object
    private_bytes: object
    dataset_cache_entries: object


def inspect_dataset_representation(dataset, representation_contract):
    # Inventory metadata, mappings, caches, and declared live objects.
    ...


def compare_worker_memory(cases, phase_samples, owner_inventory):
    # Separate replication from sharing and active pipeline resources.
    ...


def E03(config):
    # Attribute growth through controlled representation/process replays.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

A shared page appearing in multiple RSS values must not be counted repeatedly as proven duplication. Conversely, a compact on-disk index does not guarantee compact worker Python state. Preserve unsupported private-memory metrics and uncertain attribution. A parent-only dataset change may not propagate to persistent worker replicas. Any claimed representation improvement must preserve sample content, labels, indexing, and transform behavior.

Do not interpret a larger sum of resident sets as an equal increase in unique physical memory. Where proportional accounting is available, report it beside private memory and shared mappings. Elsewhere preserve the measurement limit. Check that the compact representation did not merely move the same metadata into another unmeasured process or external cache.

Part 7 - Engineering Interpretation and Philosophy

Memory attribution requires both process evidence and knowledge of the objects being retained. Replace expensive representations only after demonstrating which ownership is duplicated or extended, rather than assuming every worker inherits the dataset cheaply or copies it completely.

19.34.5 Exercise 04

Exercise Title: Sweep prefetch_factor and Quantify the Trade-Off Between Ready Batches, Host Memory Consumption, and GPU Starvation

Part 1 - Exercise Contract and Why It Matters

Sweep multiprocessing prefetch depth and quantify readiness, queueing, host/shared-memory pressure, and useful GPU progress. Find when additional prepared work reduces waits and when it merely increases retention or speculation. Treat zero-worker loading as a separate unsupported-prefetch case.

Part 2 - System and Memory Mental Model

The prefetch factor controls the configured number of batches scheduled in advance per worker under multiprocessing. It is not an exact count of fully ready resident batches or a universal byte bound. Active workers, out-of-order completion, the main consumer, transport, and optional pinned copies can add other live resources. Variable-size batches make the same factor correspond to very different capacity. More prefetch cannot overcome a persistently slower source stage in steady state.

Part 3 - Quantitative Prediction Before Execution

For fixed payload b, use Wpb as a first estimate of scheduled batch payload, then list additional overlap terms. Predict exposed waits for bursty preparation and for a sustained producer bottleneck. For variable shapes, compare a mean payload estimate with a conservative supported upper bound. Define useful completed examples per second and unused prefetched work at early termination without assuming all scheduled batches were consumed.

Part 4 - Experimental Design and Instrumentation

Fix worker count, sample trace, batch semantics, model work, pinning, and process context. Sweep valid positive prefetch values by constructing separate loaders at controlled boundaries. Measure first batch, warmed throughput, batch-wait distributions, host/shared-memory evidence, and retained capacity. Use a slow-burst source control and a sustained-slow-source control to test the explanation. Include early stop and oversized variable batches. Preserve exact coverage and cleanup obligations and report uncertain internal queue occupancy rather than reading private fields as a stable public instrument.

Include a workload with a deliberately bounded occasional slow sample and another with stable service time. Match their total sample work across each prefetch setting. Collect delivery intervals and outstanding task estimates without claiming access to an exact public ready-queue length. Use variable batch sizes to show why an identical task count can retain different byte volumes across trials.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class PrefetchDepthCase:
    workers: int
    prefetch_factor: int
    batch_payload_bound: object
    expected_scheduled_bytes: object
    host_limit_bytes: int


def plan_prefetch_depth(case, overlap_model):
    # Include active, transport, main, and pinned generations separately.
    ...


def measure_prefetch_depth(loader, workload, instruments):
    # Compare readiness and complete useful work, including early stop.
    ...


def E04(config):
    # Use fresh valid loaders and retain memory/latency tradeoffs.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not present scheduled tasks as measured ready batches. A configuration that exceeds budget or loses samples is not a faster successful trial. Preserve startup versus steady-state differences and source-cache conditions. Increasing prefetch while silently shrinking useful sample content changes the experiment. If GPU waits do not improve, retain the result and identify whether preparation, transfer, or compute limits remain unresolved.

Inspect the tail of each run as well as steady state: prefetched batches may remain when a short benchmark stops. Drain the declared finite workload or record cancelled tasks explicitly. Compare both typical and largest observed host payloads. Explain whether extra prefetch absorbed variance, enlarged unused retention, or exposed a different downstream bottleneck.

Part 7 - Engineering Interpretation and Philosophy

Prefetch is an early ownership decision intended to absorb variation. The best depth balances measured readiness against capacity and wasted work, with no expectation that an arbitrarily deep queue can repair every upstream bottleneck.

19.34.6 Exercise 05

Exercise Title: Combine num_workers and prefetch_factor in a Grid Search and Find the Highest Throughput Configuration Under a Fixed RAM Budget

Part 1 - Exercise Contract and Why It Matters

Search a bounded grid of worker count and prefetch depth for the highest measured useful throughput meeting an explicit host-memory contract. Separate calibration from evaluation and report infeasible, failed, and statistically indistinguishable configurations rather than publishing only a winning row.

Part 2 - System and Memory Mental Model

DataLoader has no generic automatic RAM admission cap for arbitrary dataset code and variable batches. A grid search therefore needs a declared resource model, preflight limits, bounded trials, and observed budget checks. Worker and prefetch choices interact with transport, pinning, caches, and active batch ownership. Changing configuration means constructing a valid loader; assigning new values to attributes is not a supported universal way to resize a live worker pool.

Part 3 - Quantitative Prediction Before Execution

Define feasible cases using named measured host scope and declared safety margin, keeping shared-memory and pinned constraints separate when relevant. Predict scheduled payload, worker overhead, and maximum supported batch overlap for each grid point. Define an objective from completed useful examples per second and a tie-break favoring lower resource cost within measured uncertainty. Explain why the optimum over a tested grid is not a global optimum for every workload or later epoch.

Part 4 - Experimental Design and Instrumentation

Choose safe candidate ranges from baseline evidence. Include zero-worker loading with valid options as its own case. Run isolated or cleanly scoped trials with identical data/order, transformations, model work, and initial state. Verify data correctness before performance ranking. Measure repeated complete runs, host peaks, shared-memory pressure, batch waits, and startup separately. Freeze the selection rule before evaluating held-out paths or shapes. Abort boundedly on a predicted or observed limit rather than repeatedly crashing workers. Preserve queue teardown and memory settling between candidates.

Separate candidate evaluation from final selection. Use the same finite calibration trace for the grid, then validate promising configurations on an unseen trace that includes a large batch and a slow sample. Preserve the exact RAM budget definition, sampling interval, and margin. Distinguish a measured process-group limit from a system-wide available-memory heuristic.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class LoaderGridCandidate:
    workers: int
    prefetch_factor: object
    predicted_host_bytes: object
    feasibility: str
    reason: str
```

```
def preflight_loader_grid(candidates, workload_bounds, budgets):
    # Reject invalid combinations and unbounded batch assumptions early.
    ...


def evaluate_loader_candidate(candidate, trial_contract):
    # Build a fresh loader, verify useful work, and enforce stop limits.
    ...


def select_budgeted_candidate(results, decision_rule):
    # Rank only comparable correct feasible trials and retain uncertainty.
    ...


def E05(config):
    # Calibrate, freeze selection, and evaluate on held-out workload cases.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

A row missing peak evidence cannot automatically be called budget-compliant. Keep sampled-peak uncertainty and process scope explicit. A faster trial with fewer completed updates is invalid. Report failures and censored trials as such, not slow finite observations. If several candidates overlap in timing variation, justify a conservative choice rather than claiming a precisely identified throughput maximum.

Keep a rejected-candidate table containing the configuration, failure phase, resource observation, and reason for exclusion. A candidate that produces incorrect sample coverage cannot be feasible even if it is fast and small. Show whether the selected point remains feasible during worker startup and iterator replacement, when transient coexistence can exceed steady memory.

Part 7 - Engineering Interpretation and Philosophy

Configuration search is a constrained measurement problem, not a parameter lottery. Its useful result is a reproducible near-best choice that stays within a stated resource envelope and explains where the evidence may stop applying.

19.34.7 Exercise 06

Exercise Title: Enable pin_memory=True and Measure Pinned Host Memory, H2D Transfer Latency, and GPU Utilization Against an Unpinned Baseline

Part 1 - Exercise Contract and Why It Matters

Compare pinned and unpinned loader output for known live host payload, H2D completion latency, GPU progress, and end-to-end throughput. Verify actual pinning support instead of treating a configuration flag as proof all batch tensors are page-locked.

Part 2 - System and Memory Mental Model

Automatic pinning can create additional host storage after preparation and transport. Allocation and host-copy work can offset transfer benefits, especially for small batches. Pinned tensor payload is not necessarily the allocator's total retained pinned capacity, and generic process locked-memory counters need not measure every accelerator-managed pinned allocation. CPU-only or unsupported configurations may not perform pinning as expected. Use actual leaf inspection and supported allocator observations where available, while retaining unavailable totals as unknown.

Part 3 - Quantitative Prediction Before Execution

For unique batch tensor payload b and K simultaneously retained pinned batches, predict known live pinned payload and source-copy overlap. Estimate H2D time as a latency-plus-bandwidth model with explicit scope. Compare a first pinned allocation with reuse/cache-warmed behavior. Predict cases where transfer time improves but complete workload time does not, and distinguish that outcome from absent actual pinning.

Part 4 - Experimental Design and Instrumentation

Use identical sample/order, worker/prefetch policy, model work, and initial state. Complete a batch tensor-leaf inventory with alias-aware accounting and verify pin status at main-process delivery. Measure unpinned and pinned paths separately through H2D completion and useful GPU consumption. Include pinning work in full-pipeline wall time. Run repeated trials without diagnostic synchronization in the throughput path, and preserve raw samples. Record supported host allocation metrics, known payload bounds, retained generations, and the limitation of every inferred total.

Compare pageable and pinned input paths using the same source values, tensor layout, batch dimensions, and destination operations. Record whether the loader actually returns pinned CPU leaves and whether custom or unsupported members remain unchanged. Avoid repeatedly pinning already pinned data inside the timed consumer unless that redundant operation is part of the declared workload.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class PinningEvidence:
    batch_id: int
    tensor_leaf_count: int
    supported_pinned_leaf_count: int
    known_live_pinned_payload: int
    allocator_pinned_total: object
    h2d_seconds: object
    complete_seconds: object


def inspect_batch_pinning(batch, schema):
    # Check supported CPU leaves and deduplicate declared shared storage.
```

```
    ...


def compare_pinned_loading(strategies, workload, instruments):
    # Include preparation/pinning cost and completed transfer/use.
    ...


def E06(config):
    # Separate actual pinning, known payload, and measured performance.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not equate the sum of leaf sizes with exact OS-locked or allocator-reserved memory. Report empty or unsupported leaves explicitly. A CUDA event duration alone excludes host pinning cost; preserve completed wall time too. Pinning must not change sample values, metadata, labels, or the declared batch order. Keep unavailable accelerator experiments unmeasured rather than claiming a universal speedup.

Separate pin preparation cost, host transfer submission time, and completed transfer duration. A short submission interval cannot establish high transfer bandwidth. Also report end-to-end time and bounded host residency after several batches. If the operating system or build lacks a trustworthy total pinned-byte counter, report known live pinned tensor payload and state that narrower scope.

Part 7 - Engineering Interpretation and Philosophy

Pinned output is a transfer resource with preparation and retention costs. It is justified by measured complete progress under the chosen pipeline, with actual pin status and accounting scope established independently of the flag that requested it.

19.34.8 Exercise 07

Exercise Title: Move Pinned DataLoader Batches to CUDA with non_blocking=True and Verify Whether Host-to-Device Transfer Is Hidden Behind GPU Computation

Part 1 - Exercise Contract and Why It Matters

Transfer pinned loader batches with nonblocking submission and test whether H2D work overlaps useful computation. Preserve source/destination lifetimes, stream dependencies, and completed output correctness. Separate host asynchrony from device overlap and from end-to-end performance.

Part 2 - System and Memory Mental Model

A nonblocking copy can return before transfer completion. Operations on one stream remain ordered, so copy followed by compute on that same stream does not automatically overlap those two operations. A separate transfer stream and additional batch/device capacity can enable a pipeline when hardware and workload allow it. The consumer must wait on the correct ready event, while pinned source and device input storage remain protected through their final readers. Allocator lifetime protection is not a replacement for execution ordering.

Part 3 - Quantitative Prediction Before Execution

Draw serialized and pipelined timelines for two batches. Predict an ideal steady bottleneck from transfer and compute stage times, plus startup/drain cost. Calculate active pinned and device slots for the chosen overlap schedule. State evidence required to claim hidden transfer: actual overlap and reduced exposed wait at comparable completed work. Predict a compute-short or bandwidth-contended case where nonblocking submission does not hide transfer.

Part 4 - Experimental Design and Instrumentation

Build a serialized pinned baseline and a generation-tagged transfer/compute pipeline using supported CUDA streams/events. Verify batch IDs, values, and outputs after appropriate completion. Keep diagnostic timeline capture separate from unprofiled throughput. Measure submission, transfer completion, consumer wait, complete elapsed time, and memory peaks distinctly. Test delayed compute, a reused host source, and early termination in a host lifecycle model before running admitted CUDA schedules. Preserve fallback behavior if overlap support or memory headroom is insufficient.

Use two reusable batch slots only after defining their individual ownership states. A source slot becomes refillable after its transfer completes; a CUDA destination becomes overwriteable after all consuming kernels complete. Associate events with slot generations so an old completion cannot authorize reuse of a newer batch. Keep correctness checks outside the critical timing path but validate every slot during a small diagnostic replay.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class AsyncBatchLease:
    batch_id: int
    generation: int
    pinned_source: object
    ready_event: object
    compute_done: object


def enqueue_batch_transfer(batch, lease, transfer_stream, device_slot):
    # Protect source ownership and record correct generation readiness.
    ...


def consume_transferred_batch(lease, compute_stream, workload):
    # Establish readiness before reads and final-reader completion.
    ...


def E07(config):
```

```
    # Compare serialized and validated overlapped schedules with equal work.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

A host timer ending at enqueue cannot establish bandwidth or overlap. Require a dependency ledger and independent value checks; a race-free-looking sampled run is insufficient by itself. Do not release pinned storage before H2D completion or reuse device storage before compute finishes. Report extra slot capacity and synchronization overhead. Retain no-overlap or slower outcomes without silently changing the batch or compute workload.

Compare timeline evidence from a short trace with an unprofiled throughput run. Profiling overhead can change the overlap being investigated. Require visible copy and compute concurrency before claiming overlap, and quantify whether this concurrency reduces completed-work time. Include a negative same-stream control whose correct ordering serializes the dependent operations.

Part 7 - Engineering Interpretation and Philosophy

Asynchrony is an opportunity to schedule work, not a performance guarantee. The useful design demonstrates that correct generation lifetimes and real resource overlap reduce exposed waits enough to justify the extra capacity and coordination.

19.34.9 Exercise 08

Exercise Title: Build a Custom Batch Type with a pin_memory() Method and Verify That Nested Tensors Are Correctly Page-Locked by the DataLoader

Part 1 - Exercise Contract and Why It Matters

Implement a custom batch type whose pinning method handles every declared nested tensor leaf and preserves non-tensor metadata. Verify DataLoader invokes the supported custom protocol and returns a structurally correct batch whose eligible CPU tensors are actually pinned.

Part 2 - System and Memory Mental Model

A custom container is not guaranteed to receive the same automatic recursive handling as built-in structures. Its pinning method must return the supported batch object form and define whether it mutates the container or creates a new one. Pinning a tensor can create a new storage owner, so external aliases and repeated references need an explicit policy. Non-tensor IDs, masks encoded as metadata, and nested ordering must remain associated with the correct samples. Unsupported tensor devices/layouts require clear handling.

Part 3 - Quantitative Prediction Before Execution

Specify a batch containing features, targets, nested auxiliary tensors, sample IDs, and an optional field. Predict eligible leaf count, unique payload, and any new pinned storage. Include repeated tensor references and explain whether the implementation preserves alias identity or deliberately supplies independent pinned copies. Define expected container shape and values before and after pinning without treating every Python object as tensor data.

Part 4 - Experimental Design and Instrumentation

Create a tiny deterministic dataset and top-level collator returning the custom batch. Complete recursive schema-aware pinning and a separate verifier. Compare direct method testing

with actual loader delivery under supported accelerator configuration. Check nested values, labels, optional fields, metadata, and leaf pin status. Exercise repeated references, empty supported tensors, already pinned leaves, and an unsupported device case. Track known payload and copy count separately from unknown allocator totals, and keep the implementation usable by the later main-process CUDA transfer path.

Define a batch schema containing inputs, labels, optional masks, nested containers, and a non-tensor sample identifier. Decide which leaves must be CPU tensors and which metadata should remain ordinary Python objects. Preserve the public return convention of the custom pinning method and verify that DataLoader dispatch reaches it in the installed version and accelerator setup.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class StudentBatch:
    features: object
    targets: object
    auxiliary: object
    sample_ids: tuple

    def pin_memory(self):
        # Handle declared nested CPU tensor leaves and return the batch form.
        ...


def collate_student_batch(samples):
    # Preserve schema, labels, and IDs while constructing bounded output.
    ...


def verify_custom_pinning(before, after, schema_contract):
    # Check structure, values, alias policy, and supported leaf pin status.
    ...


def E08(config):
    # Verify the actual DataLoader custom-batch protocol, not only a flag.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Reject a custom method that pins only the top-level feature tensor and leaves required nested targets unhandled. Do not silently replace metadata or change sample order. A leaf's pin status does not establish the whole batch's transfer readiness. If alias semantics change,

document and charge that change under the contract. Preserve unsupported pinning cases instead of forcing CUDA tensors through a CPU pin operation.

Check both tensor content and structural identity constraints after pinning. If two fields intentionally refer to the same object or view, state whether that relationship must survive the transformation. A recursive implementation that pins each leaf independently can alter sharing and allocation volume. Test absent optional fields and an empty batch according to the declared schema.

Part 7 - Engineering Interpretation and Philosophy

A custom batch protocol is an explicit schema transformation. It should make eligible storage ready for transfer while preserving the meaning, relationships, and ownership rules of the data the model will consume.

19.34.10 Exercise 09

Exercise Title: Compare persistent_workers=False and persistent_workers=True Across Multiple Epochs and Measure Worker Startup Cost and Retained RAM

Part 1 - Exercise Contract and Why It Matters

Compare persistent and nonpersistent workers across multiple epochs, measuring startup, epoch transition latency, warmed throughput, and retained host state. Preserve per-epoch sample/transform semantics instead of treating surviving workers as a purely mechanical speed setting.

Part 2 - System and Memory Mental Model

Persistence keeps worker processes and their dataset instances alive after an epoch. This can avoid repeated startup but retain caches, mappings, file handles, RNG state, and dataset-specific objects. Worker initialization is not an automatic per-epoch reset under persistence. Mutating the main process's dataset attributes does not generally mutate worker replicas. Sampler epoch control and worker-local epoch/augmentation state are separate channels with explicit propagation requirements.

Part 3 - Quantitative Prediction Before Execution

For E epochs, estimate repeated startup cost without persistence versus one startup plus retained-state overhead with it. Predict memory at startup, after each epoch, and between epochs. Define expected IDs and epoch-dependent random choices for both strategies. Include a dataset whose worker-local cache reaches a bounded plateau and another controlled case whose cache grows, without assuming persistence itself always causes a leak.

Part 4 - Experimental Design and Instrumentation

Use a replayable dataset with explicit epoch and randomness contracts. Complete both valid loader configurations and run the same epoch count, sample coverage, and useful model work. Record worker PIDs, initialization counts, epoch transition time, caches/resources, and supported private/proportional memory. Test the supported mechanism for worker epoch updates or reconstruct the loader when necessary. Verify that an old worker dataset version is not silently reused after source changes. Keep startup-inclusive and steady-state results separately and observe final lifecycle cleanup.

Compare several complete epochs rather than repeatedly timing only the first batch. Record iterator construction, first delivery, final delivery, and the between-epoch idle interval. Establish

how workers learn the epoch number before using it for randomized transforms. A main-process dataset assignment is not sufficient evidence that a persistent worker replica received a change.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class EpochWorkerObservation:
    epoch: int
    persistent: bool
    worker_ids: tuple
    initialization_count: int
    transition_seconds: object
    retained_host_bytes: object
    dataset_epoch_version: object


def run_epoch_worker_policy(dataset_factory, policy, epoch_contract):
    # Propagate required epoch state and preserve sample semantics.
    ...


def compare_worker_persistence(results, resource_inventory):
    # Separate avoided startup from retained caches and live resources.
    ...


def E09(config):
    # Verify multiple epochs and final cleanup with both strategies.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

A first-epoch speed difference does not establish a long-job benefit. Retained memory can be intentional caching or unbounded growth; distinguish them by ownership and progression. Do not claim main-dataset reset methods ran inside workers without evidence. Persistent workers require multiprocessing, so a zero-worker case must be configured separately. Preserve exact restart and seed behavior rather than ignoring changed transformations in the comparison.

Show per-worker cache size, open-resource observations, and memory after each epoch when supported. Persistent state may be useful retained capacity rather than a leak, so distinguish bounded reuse from unbounded growth. Validate that both persistence policies produce the required epoch coverage and augmentation policy before comparing startup savings and retained memory.

Part 7 - Engineering Interpretation and Philosophy

Persistence trades recurring process creation for a longer-lived resource state. It is useful when that state remains bounded and correct across epochs, with the saved startup time measured together with its retention and lifecycle costs.

19.34.11 Exercise 10

Exercise Title: Reproduce Excessive Host Memory Growth from Persistent Workers and Design a Cleanup Strategy for Long-Running Training Jobs

Part 1 - Exercise Contract and Why It Matters

Reproduce excessive retained host state in a bounded persistent-worker example, identify its actual owner, and design cleanup that remains correct across epochs and long runs. Separate a genuine growing cache from allocator retention, file-page residency, or intentionally live batches.

Part 2 - System and Memory Mental Model

Persistence allows worker-local objects to survive, but it does not automatically make every dataset leak. Common causes include unbounded decoded-sample caches, append-only history, unreleased handles, and external references. Main-process cleanup does not directly clear worker-local containers. Garbage collection cannot reclaim objects that remain owned, and released objects do not guarantee immediate RSS reduction. Cleanup must also preserve mappings, leases, and samples still in use by queued or device work.

Part 3 - Quantitative Prediction Before Execution

Create a tiny worker-local cache with a known per-entry payload and an explicit capacity. Predict linear growth for a deliberately unbounded control and a plateau for bounded eviction. Include Python metadata and allocator uncertainty separately from cached tensor bytes. Define thresholds for unacceptable growth and a safe intervention boundary. Predict which counters should change after logical cleanup even if process RSS remains above its initial value.

Part 4 - Experimental Design and Instrumentation

Run isolated short jobs with strict sample/epoch and memory stop limits. Complete the controlled retained-state example and record cache entries, owned payload, worker PIDs, and supported memory metrics. Compare bounded cache, epoch reset through a verified worker communication mechanism, and orderly loader/process recreation. Validate all samples and any required epoch-dependent state after cleanup. Test early exit and a failed reset without pretending the worker became clean. Keep returned batches and logs bounded so the observer does not create the growth being diagnosed.

Use a deliberately bounded worker-local cache with a visible entry counter and a small controlled resource limit. Keep the experiment independent of the production dataset and stop before system pressure threatens unrelated work. Compare no cache, bounded eviction, and orderly worker replacement. Each policy must use equivalent sample computation and report any repeated initialization or reopened storage cost.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
```

```
@dataclass
class WorkerRetentionReport:
    worker_id: int
    epoch: int
    cache_entries: int
    known_cache_payload_bytes: int
    open_resources: object
    observed_host_bytes: object
    cleanup_status: str


def inspect_worker_retention(worker_state, schema):
    # Attribute declared live owners rather than guessing from RSS alone.
    ...


def apply_cleanup_policy(worker_state, policy, lifecycle_boundary):
    # Bound caches or reset/recreate only after current obligations finish.
    ...


def E10(config):
    # Compare controlled growth, bounded cleanup, and preserved correctness.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require the known retained owner to be reduced or bounded before crediting cleanup. A temporary RSS decline does not prove leak removal, and unchanged RSS does not prove live objects remain. Do not invoke undocumented worker internals as a portable cleanup API. Verify worker exit or reset completion through supported application lifecycle controls. Preserve failures and resource leftovers instead of claiming a clean long-running state.

A fall in Python object count without a fall in resident memory can reflect allocator retention rather than a failed logical cleanup. Report both outcomes without forcing equality. Conversely, stable resident memory does not prove that file handles or cache entries are bounded. Require acknowledgement or process exit evidence before declaring worker-local cleanup complete.

Part 7 - Engineering Interpretation and Philosophy

Long-running reliability comes from bounded ownership and verified lifecycle boundaries. A useful cleanup policy identifies what it releases, who executes that release, and which useful state remains valid afterward.

19.34.12 Exercise 11

Exercise Title: Replace Per-Sample ___getitem___() Work with Batched ___getitems___() Fetching and Measure the Reduction in Python and Worker Overhead

Part 1 - Exercise Contract and Why It Matters

Implement batched sample fetching through the supported map-style dataset protocol and compare it with per-sample access. Preserve one result per requested index, ordering, duplicates, labels, and transform semantics while reducing avoidable Python or source-operation overhead.

Part 2 - System and Memory Mental Model

Batched fetching can amortize source queries, vectorize parsing, or exploit locality. It does not automatically eliminate collation or return a ready-made batch under every loader contract. The standard supported protocol returns the requested sequence of samples for subsequent collation. Returning one already stacked tensor without matching the collator can double-batch or change shape. Batch-aware transformations can also consume randomness differently, so correctness needs a defined per-sample policy.

Part 3 - Quantitative Prediction Before Execution

For a batch of n requested IDs, compare n individual source/dispatch costs with one bounded grouped operation plus per-sample work. Predict output order for indices containing duplicates and a nonmonotonic sequence. Calculate intermediate memory for a vectorized read and the later collated result. Identify cases where the source already batches internally or the grouped implementation merely loops over individual access, giving little expected benefit.

Part 4 - Experimental Design and Instrumentation

Create a deterministic dataset supporting both paths and record which method the actual installed DataLoader invokes under batching. Complete a true bounded grouped fetch with a single-index fallback and compatible collator. Validate IDs, values, labels, lengths, duplicates, and augmentations against independent per-sample references. Compare source-call counts, host object/copy evidence, worker CPU time, batch-ready time, and completed workload throughput. Test partial last batches, empty requested groups where relevant, invalid IDs, and supported wrapper/subset behavior without bypassing overridden dataset logic.

Include repeated and nonconsecutive indices in one requested batch so a backend fetch optimization cannot silently sort away sampler order or remove duplicates. Compare the ordinary sample path with batched fetching under the same collation contract. Count dispatch calls and underlying storage requests separately; reducing Python calls does not necessarily reduce physical I/O or tensor copying.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
from torch.utils.data import Dataset

class BatchedFetchDataset(Dataset):
    def __init__(self, source, schema, transform_policy):
        self.source = source
        self.schema = schema
        self.transform_policy = transform_policy
        ...

    def __len__(self):
```

```
        ...

    def __getitem__(self, index):
        # Preserve the authoritative single-sample semantics.
        ...

    def __getitems__(self, indices):
        # Return requested samples in matching order for the collator.
        ...


def E11(config):
    # Verify real dispatch, sample equivalence, and grouped-operation cost.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not claim batched-fetch acceleration when the new method silently changes sample count or pre-stacks into an incompatible collation contract. Preserve duplicate indices and source side effects where required. A vectorized transform with different random choices is a separate semantic experiment unless controlled by the contract. Feature-check the protocol and wrapper behavior rather than assuming every dataset adapter forwards it identically.

Verify the values, order, labels, and randomized transform policy for every requested position. If bulk decoding changes numerical results or augmentation randomness, classify it as a separate algorithmic variant. Test wrappers and subsets explicitly because the effective dataset object controls dispatch. Retain a fallback path when the installed framework cannot support the intended batched interface.

Part 7 - Engineering Interpretation and Philosophy

Batch-aware access is useful when it removes repeated overhead while retaining the meaning of each requested sample. Its proof is the dispatch and source work actually reduced under the same sample contract, not simply the presence of a plural method name.

19.34.13 Exercise 12

Exercise Title: Optimize collate_fn to Minimize Temporary Tensor Copies, Intermediate Python Objects, and Unnecessary Host Allocations

Part 1 - Exercise Contract and Why It Matters

Optimize collation to reduce unnecessary host copies, intermediate Python objects, and dtype/layout conversions while preserving the exact batch schema. Measure bounded output ownership and complete pipeline effects rather than assuming the fewest visible tensor calls guarantee the least memory.

Part 2 - System and Memory Mental Model

Collation turns per-sample values into a model-facing batch. Stacking commonly requires independent output storage; padding variable lengths adds meaningful capacity and masks. Avoiding that output is possible only if the consumer accepts another representation. Default collation may already use an efficient multiprocessing transport path, so a custom routine can

accidentally add copies. Reusing one output buffer can corrupt previously returned batches while they are queued, pinned, or consumed asynchronously.

Part 3 - Quantitative Prediction Before Execution

For fixed sample shape and n samples, calculate final tensor payload and list unavoidable versus optional copies for the declared schema. Compare list-to-tensor conversion, repeated concatenation, and one bounded final allocation as candidate designs without assuming their observed costs. For variable lengths, calculate padded data and mask bytes and preserve true lengths. Predict overlap between sample owners, collated output, transport, and pinned generations.

Part 4 - Experimental Design and Instrumentation

Start with a tiny default-collation reference and a fixed schema. Complete a custom routine with explicit dtype, layout, labels, IDs, optional fields, and ownership. Validate fixed and variable-size cases, partial batches, noncontiguous inputs, and retained prior batches. Collect allocation/copy and object-count evidence in separate diagnostics, then time equal useful work without that overhead. Compare in both single-process and multiprocessing contexts where supported. Charge conversion and transport work introduced by the custom path rather than reporting only its local function timer.

Build a copy ledger for decoding output, padding, stacking, dtype conversion, and contiguous layout conversion. Mark which steps are semantically required and which are accidental intermediates. A custom collator that returns views must preserve source lifetime until consumers finish. Test a short final batch and variable-length samples so a fixed-size optimization cannot hide invalid padding or stale values.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class CollationContract:
    schema_id: str
    output_dtype: object
    output_layout: str
    length_policy: str
    ownership_policy: str
    maximum_batch_bytes: int


def optimized_collate(samples, contract):
    # Construct the required output once where legal; preserve metadata.
    ...


def verify_collation(reference, candidate, input_owners, contract):
    # Check values, masks, labels, layout, and retained-batch independence.
```

```
    ...


def E12(config):
    # Compare default and custom collation through the complete data path.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

A local zero-copy view may retain a huge source storage or violate the model's contiguous-batch contract. Preserve or explicitly change representation rather than silently omitting required padding. Any reusable output needs a supported lease/pool lifecycle across worker transport and device use; a single global scratch tensor is not automatically safe. A custom routine that loses the default transport optimization can be slower despite fewer local calls.

Measure collation in isolation only as a diagnostic, then repeat the complete worker and transfer pipeline. IPC handling can make a locally faster collator slower overall. Compare actual output payload, temporary peaks, object counts when available, and complete throughput. Do not reuse one global writable output for multiple queued batches without a proven ownership and completion protocol.

Part 7 - Engineering Interpretation and Philosophy

Good collation allocates the batch the model actually needs with no unnecessary intermediate forms. Optimize the complete ownership path and preserve schema semantics rather than treating every required output copy as a mistake.

19.34.14 Exercise 13

Exercise Title: Reproduce /dev/shm Exhaustion with Aggressive Multiprocessing and Diagnose Shared-Memory Pressure from Batch Size, Workers, and Prefetch Depth

Part 1 - Exercise Contract and Why It Matters

Reproduce shared-memory pressure in a disposable isolated multiprocessing test with a deliberately small explicit allowance. Diagnose the contribution of batch payload, worker count, prefetch, transport owners, and consumer retention. Keep the host's general shared-memory resources and unrelated jobs outside the experiment.

Part 2 - System and Memory Mental Model

Linux shared-memory transport can consume tmpfs-backed resources such as those exposed through the conventional shared-memory filesystem. Exhaustion can cause worker or IPC failures, but similar messages can arise from RAM, file-descriptor, address-space, or other process failures. Mount size, available capacity, sharing strategy, and open/unlinked objects need observation. A formula based only on scheduled batches is an estimate, not a universal exact usage law. Other platforms may expose different facilities.

Part 3 - Quantitative Prediction Before Execution

Predict transport pressure from variable/fixed batch payload and concurrent owners, distinguishing active consumer references from prefetched work. Compare small controlled worker/prefetch settings against an isolated allowance. Define a failure threshold and stop plan before the test.

List evidence needed to classify shared-memory exhaustion separately from CUDA OOM, host allocation failure, file-descriptor exhaustion, or an unrelated worker exception.

Part 4 - Experimental Design and Instrumentation

Use a disposable child/container test with bounded time, tiny known data, and a controlled shared-memory quota when supported. If isolation is unavailable, run only a host model or pressure preflight and mark actual exhaustion untested. Complete increasing-pressure trials and record mount/resource observations, batch owner counts, worker exits, and the original error phase. Compare lower prefetch, fewer workers, and released consumer references as controlled replays. Preserve useful IDs and failure evidence. Remove only resources owned by the test through its lifecycle; never delete arbitrary shared-memory entries as a cleanup shortcut.

Specify the disposable environment's shared-memory quota before launching workers. Select small synthetic batches and a bounded concurrency ramp that can reach that quota without exhausting host RAM. Capture mount capacity, process logs, worker exit status, and open-file limits when accessible. A simulated resource model is acceptable when isolation is unavailable, but must remain clearly separate from an observed operating-system failure.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class SharedMemoryPressureCase:
    workers: int
    prefetch_factor: int
    maximum_batch_bytes: int
    isolated_shm_limit_bytes: object
    retained_consumer_batches: int
    stop_deadline_seconds: float


def preflight_shm_case(case, platform_observations):
    # Require supported isolation and distinguish estimates from quotas.
    ...


def run_isolated_pressure_case(case, workload, evidence_contract):
    # Preserve bounded failures without exhausting the general host.
    ...


def E13(config):
    # Compare pressure controls and retain the original failure classification.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

A worker bus error alone is not a complete diagnosis. Correlate resource pressure with the failing phase and counterfactual replay. Retain platform limitations and unknown usage rather than fabricating a quota measurement. Unlinked shared objects can remain live while referenced, and changing a sharing strategy is not a general memory-capacity solution. Any completed reproduction must show the controlled environment and its actual limits.

Correlate the first failed phase with capacity observations and an isolated lower-pressure replay. Shared-memory usage may outlive an individual tensor reference because other processes still own it. Let the test environment perform its own cleanup after workers terminate. Never delete unidentified shared objects to make a retry succeed or infer a diagnosis from one generic worker exception alone.

Part 7 - Engineering Interpretation and Philosophy

A useful failure experiment makes the cause small, isolated, and explainable. The engineering result is evidence linking ownership pressure to a bounded resource limit, with interventions that reduce the same cause rather than mask an unrelated error.

19.34.15 Exercise 14

Exercise Title: Recover from Shared-Memory Exhaustion by Dynamically Reducing Worker Count, Prefetch Depth, or Batch Memory Footprint

Part 1 - Exercise Contract and Why It Matters

Recover from a classified shared-memory failure by selecting a smaller valid loader configuration and resuming from an explicit committed-work boundary. Preserve sample/update semantics, teardown old ownership, and include failure, rebuild, and replay costs in the result.

Part 2 - System and Memory Mental Model

A live loader does not provide a universal supported hot-resize API for worker count and prefetch settings. Recovery therefore chooses new settings and recreates the loader or isolated input job at a defined boundary. Dispatched or prefetched IDs are not the same as successfully consumed IDs or committed optimizer updates. Stochastic transforms and nonseekable streams complicate exact replay. Reducing batch size can also change model behavior and must not be presented as an equivalent memory-only setting change.

Part 3 - Quantitative Prediction Before Execution

Define state recording committed update ID, consumed sample IDs, sampler position/replay authority, transform seed contract, and pending work. Predict which batches must be replayed or discarded after failure without losing or doubling committed updates. Compare first reducing prefetch or workers while keeping batch semantics fixed. If microbatching is considered, state its limitations for batch-dependent modules, randomness, and optimizer equivalence instead of assuming accumulation is always identical.

Part 4 - Experimental Design and Instrumentation

Use a tiny deterministic replayable dataset and isolated controlled failure from E13. Save training state at declared safe commits and record the input trace. Stop or drain old work through supported application/process lifetime controls, verify owned resources are released, then construct the reduced loader. Resume according to an explicit rollback/replay policy and compare with uninterrupted reference work. Bound retries and retain failure diagnostics.

Test one unrecoverable nonreplayable source case as refusal, not silent sample loss. Measure downtime, repeated work, final coverage, and complete training outcome.

Prepare a finite replayable source and an explicit committed-update record before injecting the controlled failure. Save sampler progress and random state at a supported boundary, then distinguish consumed batches from completed optimizer updates. The replacement loader must begin from a declared recovery point. Account for replayed preparation and computation as recovery cost, not as additional successfully trained examples.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class RecoveryBoundary:
    committed_update_id: int
    model_state_id: str
    optimizer_state_id: str
    sampler_state: object
    transform_seed_contract: object
    consumed_sample_ids: tuple


def choose_recovery_configuration(failure, current, workload, budgets):
    # Prefer valid smaller queues/workers without changing batch semantics.
    ...


def rebuild_and_resume(boundary, configuration, source_contract):
    # Stop old ownership and replay only under a verified recovery policy.
    ...


def E14(config):
    # Compare recovered useful updates with an uninterrupted reference.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not mutate loader attributes and claim existing processes reconfigured. A counter saying the next batch index advanced does not prove an update committed. Preserve IDs, weights, optimizer state, and required randomness. If exact state restoration is not implemented, label the recovery weaker or unsupported. Do not delete shared-memory objects belonging to other jobs; resource cleanup must follow the failed job's actual ownership.

Validate the recovered sample and update history against a fault-free control. If exact replay cannot be guaranteed for the source or stochastic operations, report the weaker recovery contract explicitly. Confirm that old workers have exited before measuring the replacement's

stable footprint. Stop after the bounded retry policy instead of progressively changing semantics until some configuration happens to finish.

Part 7 - Engineering Interpretation and Philosophy

Recovery is a transactional training/data lifecycle problem, not merely a smaller worker number. A good implementation restores a coherent computation with bounded retries and complete cost accounting, or explains precisely why its source cannot support that guarantee.

19.34.16 Exercise 15

Exercise Title: Build an IterableDataset with Multiple Workers and Eliminate Duplicate Samples Through Explicit Worker-Aware Data Sharding

Part 1 - Exercise Contract and Why It Matters

Build a multiworker iterable dataset with explicit source partitioning and verify no unintended duplicate or missing samples. Define epoch, restart, ordering, and last-batch behavior for the actual source. Keep worker sharding separate from map-style sampler dispatch.

Part 2 - System and Memory Mental Model

Each iterable worker receives a dataset replica and can otherwise traverse the same source. A worker-aware partition must assign authoritative records once under the chosen finite-epoch contract. Filtering a complete source in every worker can remove duplicate output while still duplicating expensive I/O, so compare physical source partitions when available. Last incomplete batches and loader length estimates require care because iterable worker replicas can finish unevenly. Persistent workers need explicit epoch-state handling rather than assumed reinitialization.

Part 3 - Quantitative Prediction Before Execution

For N known records and W workers, derive a deterministic partition and its expected ID union/intersection before execution. Calculate per-worker counts, partial final batches, and the declared drop policy. Compare contiguous shards and strided IDs for source locality without giving either an automatic quality guarantee. Predict behavior for fewer records than workers, an empty source, and a checkpoint-resume request requiring source-position authority.

Part 4 - Experimental Design and Instrumentation

Create a tiny finite authoritative source with IDs and labels. Complete worker-aware iteration using supported worker information and a zero-worker path. Verify exact ID coverage, labels, values, and duplicate policy for several worker counts. Compare cheap output filtering with real independent shard access if the source supports it. Test uneven shards, persistent epochs, controlled source failure, and an explicitly unsupported nonseekable resume. Record actual source reads as well as emitted samples. Keep distributed-rank sharding outside the initial scope or implement its separate composition explicitly rather than counting local worker uniqueness as global uniqueness.

Give every source record a stable identifier and define worker shards before applying worker-local transforms. Test source lengths that are not divisible by the worker count or batch size. Include one fewer sample than workers and a normal longer stream. Explain whether sharding avoids duplicate decoding and storage reads or only filters duplicates after expensive work has already been performed.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
from torch.utils.data import IterableDataset, get_worker_info

class ShardedIterableDataset(IterableDataset):
    def __init__(self, source_manifest, epoch_contract):
        self.source_manifest = source_manifest
        self.epoch_contract = epoch_contract
        ...

    def __iter__(self):
        # Derive process-local partition, open it, and emit each assigned ID.
        ...


def verify_iterable_coverage(observed_ids, partition_contract):
    # Check duplicates, omissions, partial batches, and source-read work.
    ...


def E15(config):
    # Compare zero-worker, multiple shards, and epoch/restart cases.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not use the loader's length heuristic as proof of exact iterable coverage. Record which samples a per-worker last-batch policy drops. Output deduplication alone does not establish nonduplicated source I/O or exactly-once training updates after failure. Preserve source authority and the declared replay model. An epoch-dependent transform must follow explicit worker-state propagation, with retained replica state audited under persistence.

Audit the multiset of delivered identifiers rather than relying on the loader's length estimate. With per-worker partial batches, dropping incomplete batches may discard more records than a global batching intuition predicts. Record exact omissions and duplicates by worker and epoch. For a non-replayable stream, restrict the recovery claim rather than inventing a deterministic restart position.

Part 7 - Engineering Interpretation and Philosophy

Iterable correctness starts with an ownership rule for records, not merely parallel iteration. A useful dataset makes its finite coverage, read work, epoch boundary, and restart limitations testable before concurrency is credited with improved throughput.

19.34.17 Exercise 16

Exercise Title: Compare Ordered and Out-of-Order Multi-Worker Batch Delivery and Measure the Throughput and Buffering Effects of in_order

Part 1 - Exercise Contract and Why It Matters

Compare ordered and supported out-of-order multiworker batch delivery under controlled service-time variation. Measure useful throughput, head-of-line waits, buffering, and delivered-data order. Treat training-distribution and reproducibility changes as part of the policy contract.

Part 2 - System and Memory Mental Model

Ordered delivery can wait for an earlier slow task even when later batches are prepared. Out-of-order delivery can reduce that wait but changes the sequence seen by the consumer. It need not eliminate every buffer or guarantee faster training. Fast and slow samples can correlate with labels or lengths, causing time-limited prefixes to become skewed. Feature-check the installed loader option and keep zero-worker behavior separate; lack of support must not silently substitute the ordered strategy under the same label.

Part 3 - Quantitative Prediction Before Execution

Construct a tiny dispatch trace containing one delayed batch and several quick batches. Predict ordered and out-of-order delivery IDs and waits under a clear model. Calculate expected full-epoch ID multisets and compare with time-limited prefixes. Predict buffering and capacity conservatively rather than assigning an exact internal queue count from the flag. Define learning-quality and reproducibility checks distinct from numerical per-sample data correctness.

Part 4 - Experimental Design and Instrumentation

Use deterministic sample values and controlled bounded preparation delays in an isolated diagnostic workload. Complete both supported configurations with the same dispatched work and worker/prefetch/pinning settings. Record batch IDs, task durations, arrival/delivery evidence where available, main waits, resource peaks, and completed throughput. Compare full epochs and fixed-time prefixes separately. Evaluate a label/length-correlated delay case to expose bias. Preserve training order as part of the result, and do not promise identical gradients across intentionally different SGD sequences.

Create a controlled service-time imbalance correlated with a visible sample class, then compare ordered and out-of-order delivery using identical worker assignments where practical. Inspect both a fixed-duration prefix and complete finite coverage. The prefix can reveal class skew even when every sample is eventually delivered exactly once. Preserve sample identities in the measurement trace without retaining full tensors.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class DeliveryObservation:
    task_id: int
    sample_ids: tuple
    preparation_interval: object
    delivered_position: int
```

```
    main_wait_seconds: object
    prefix_metadata: object


def configure_delivery_policy(dataset, in_order, support_contract):
    # Feature-check multiworker delivery semantics explicitly.
    ...


def compare_delivery_traces(ordered, unordered, workload_contract):
    # Separate coverage, prefix skew, head-of-line evidence, and throughput.
    ...


def E16(config):
    # Test controlled stragglers and realistic supported workload variation.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

A faster prefix containing easier or shorter data is not necessarily more useful training progress. Require exact final coverage under the declared epoch policy and report prefix composition. Do not infer exact buffer occupancy from delivery order alone. Preserve unsupported options, timing variability, and cases where out-of-order delivery does not improve complete work. Keep semantic permission for reordered training explicit.

Treat order changes as a semantic permission for the workload, not simply a speed switch. Training updates can change when batch arrival order changes, including effects on optimization and reproducibility. Report delivery gaps, completed throughput, and memory observations independently. If the installed DataLoader lacks the option, mark the experiment unsupported and retain the ordered baseline rather than monkey-patching internal queues.

Part 7 - Engineering Interpretation and Philosophy

Delivery order is a tradeoff among waiting, buffering, reproducibility, and learning behavior. Choose it only with measured service-time evidence and a consumer contract that permits the resulting sequence, rather than treating out-of-order execution as a free scheduling improvement.

19.34.18 Exercise 17

Exercise Title: Build a Runtime Monitor That Separates Storage Read Time, Worker Processing Time, Queue Wait Time, Pinning Time, H2D Transfer Time, and GPU Compute Time

Part 1 - Exercise Contract and Why It Matters

Build a bounded monitor for source access, worker preparation, collation, transport/queueing, pinning, H2D, and GPU compute. Define exact observable intervals, estimated bounds, and unavailable stages. Correlate task IDs across processes without pretending every internal loader transition is publicly instrumented.

Part 2 - System and Memory Mental Model

Main-process time waiting for the next batch can include unfinished worker work, transport, reordering, pinning, and scheduler delay. It is not pure queue residence. Wrapping dataset and collator functions reveals only their own intervals. Exact internal queue timing needs a version-pinned audited adapter or an explicitly owned transport stage; otherwise report combined delay or bounds. CPU clocks and CUDA event intervals have distinct semantics, and overlapping worker durations cannot be summed as serial wall time.

Part 3 - Quantitative Prediction Before Execution

Draw a correlated task timeline and define start/end events for each observable stage. Predict which intervals overlap, which include waiting, and which can only be bounded. Calculate useful complete time and a critical-path hypothesis without subtracting unrelated clock domains. Define instrumentation overhead and a maximum retained event count. A missing stage must remain unavailable, not zero milliseconds in a misleading stacked total.

Part 4 - Experimental Design and Instrumentation

Implement wrappers for owned source, transform, collation, and custom pinning boundaries where available. Attach bounded scalar timing metadata and stable task/batch IDs without retaining tensor graphs. Validate CPU timestamp domains before aligning processes and use CUDA events with deliberate completion boundaries. Test the monitor on a controlled synthetic pipeline with known stage placement, then compare instrumented and uninstrumented runs. Record transport intervals as exact only when the chosen supported instrumentation actually observes their boundaries. Preserve uncertainty and sampling mode in exported records.

Define a timestamp schema with process identity, batch identity, clock domain, and measurement boundary. Wrap only stages whose entry and exit are actually observable. A worker's completion timestamp and the consumer's return timestamp bound several transport and scheduling stages together unless additional instrumentation separates them. Keep instrumentation queues bounded so the monitor cannot become the dominant retained-memory source.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class StageInterval:
    task_id: str
    process_id: int
    stage: str
    clock_domain: str
    start: object
    end: object
    observation_kind: str
    limitations: tuple

class InputPipelineMonitor:
```

```
    def begin_stage(self, task_id, stage, clock_domain):
        # Record only owned/observed boundaries with bounded storage.
        ...

    def end_stage(self, token):
        # Preserve local interval meaning and incomplete/failure status.
        ...

    def correlate(self, event_records, support_contract):
        # Distinguish exact, estimated, combined, and unavailable stages.
        ...

    def export(self):
        # Emit CPU scalar metadata and observer-overhead evidence.
        ...


def E17(config):
    # Validate attribution limits before diagnosing real workloads.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Do not label elapsed next-batch wait as isolated queue time without evidence. Do not report worker clock values as CUDA timing or force concurrent intervals into an additive total. A private queue-size query is not a portable exact latency instrument. Preserve dropped-event counts and incomplete spans. A monitor changing overlap through frequent synchronization must be labeled diagnostic and compared with an unprofiled useful-work run.

Verify clock assumptions before subtracting timestamps from different processes or devices. CUDA events measure appropriate device intervals but do not directly share a timestamp origin with a Python wall clock. Show an overhead control and explain missing spans. Stage durations can overlap, so their sum is neither an automatic critical-path estimate nor a valid replacement for measured wall time.

Part 7 - Engineering Interpretation and Philosophy

A useful monitor says what it observed and what remains inferred. Honest stage boundaries reveal a causal bottleneck more reliably than a complete-looking chart filled with unsupported zeros or serial sums of concurrent activity.

19.34.19 Exercise 18

Exercise Title: Stress-Test the Input Pipeline with Slow Storage, Expensive CPU Transforms, Variable-Size Samples, Limited RAM, and a Fast GPU to Identify the True Bottleneck

Part 1 - Exercise Contract and Why It Matters

Stress-test an input pipeline against slow sources, expensive transforms, variable batch sizes, constrained host resources, and fast consumption. Identify the true limiting stage using counterfactual replays, while preserving useful training work and resource bounds.

Part 2 - System and Memory Mental Model

Several bottlenecks can coexist or move as settings change. Faster workers can saturate storage; deeper queues can increase RAM and shared-memory pressure; pinning can improve transfer while adding host copies. Variable lengths can inflate padding and alter compute cost. A fast GPU exposes preparation latency only if measured execution and dependencies justify that interpretation. Synthetic delays are controlled models, not measurements of actual disk or production transform performance.

Part 3 - Quantitative Prediction Before Execution

Build separate source-limited, transform-limited, transfer-limited, memory-limited, and compute-limited hypotheses. Predict stage intervals, queue behavior, resource peaks, and useful throughput under each. Calculate a conservative maximum active batch footprint for variable shapes. Define stop limits and infeasible cases before execution. Include an intervention expected to change one cause, such as resident-data replay or bounded transform substitution, without claiming that altered workload is the production performance result.

Part 4 - Experimental Design and Instrumentation

Use tiny exact references and safely bounded larger traces. Freeze sample IDs, labels, transformation semantics, model state, useful step count, and randomness where equivalence is required. Vary one stressor first, then test selected interactions. Capture monitor evidence with its attribution limits, worker/host/shared/pinned/device resources, and completed updates. Verify data and masks for every supported variable-length case. Include controlled worker failure, early stop, persistent-state growth, and unavailable accelerator outcomes. Minimize reproducing incidents with their source, trace, seed, configuration, and lifecycle boundary recorded.

Change one injected bottleneck at a time before combining stressors. Use a bounded transform cost, a documented storage-delay model, and a variable-shape sequence with a reproducible maximum payload. Keep sample content and required updates fixed. Compare the pipeline against compute-only and preparation-only controls so a fast GPU's apparent starvation has a measurable reference rather than a visual utilization guess.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class PipelineStressCase:
    case_id: str
    source_policy: str
    transform_policy: str
    sample_shape_trace: tuple
    host_budget_bytes: int
    shared_memory_limit: object
    consumer_policy: str
    stop_limits: tuple
```

```
def predict_stress_case(case, resource_model, support_contract):
    # State causal expectations and infeasible floors before execution.
    ...


def run_stress_case(case, pipeline, model, monitor):
    # Preserve useful work and classify every failure/abort.
    ...


def diagnose_stressor(evidence, controlled_replays):
    # Separate directly observed stages from unresolved hypotheses.
    ...


def E18(config):
    # Report validated scope, resource limits, and causal counterexamples.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

A high GPU idle fraction is insufficient to identify storage as the cause. A speedup obtained by dropping long samples or changing transforms is not an equivalent pipeline optimization. Preserve padding and sample coverage costs. Report enforced versus sampled memory limits and keep actual failures distinct from preflight refusal. If a suspected bottleneck does not reproduce, retain that result and the next discriminating experiment rather than inventing supporting metrics.

Record the resource and phase at the first sustained degradation, including whether a queue is empty, downstream work is blocked, or workers are throttled. An artificial sleep models service delay but does not reproduce real storage cache and I/O behavior. Label that distinction and test at least one real source when available. Preserve all untested hardware conditions as explicit gaps.

Part 7 - Engineering Interpretation and Philosophy

Stress testing should expose the pipeline's operating envelope and the reason it fails or slows. Its strongest outcome is a small causal explanation linked to preserved useful work, with interventions and limits that remain visible when conditions change.

19.34.20 Exercise 19

Exercise Title: Build and Benchmark a Self-Tuning PyTorch DataLoader That Dynamically Chooses Worker Count, Prefetch Depth, Pinning, and Batch Delivery Strategy Under RAM and Throughput Constraints

Part 1 - Exercise Contract and Why It Matters

Build a controller that selects worker, prefetch, pinning, persistence, and supported delivery policies under explicit host/shared-memory and useful-throughput constraints. Reconfigure

at validated lifecycle boundaries, preserve training semantics, and include tuning, teardown, startup, replay, and failure costs.

Part 2 - System and Memory Mental Model

Self-tuning requires observation, candidate admission, evaluation, semantic validation, and controlled replacement. It is not arbitrary live mutation of loader attributes. Replayable source/sampler state, stable sample IDs, transform randomness, and committed model/optimizer boundaries determine which transitions are supportable. Out-of-order delivery and changed batch footprints require separate semantic permission. Unsupported streaming recovery or uncertain resource evidence should restrict candidates rather than produce silent data loss or invented budget compliance.

Part 3 - Quantitative Prediction Before Execution

Define an objective from completed correct useful work over full elapsed time, subject to named resource scopes, coverage, and progress. Estimate worker state, scheduled/active batches, transport, pinning, device overlap, and transition cost for each candidate. Add hysteresis and a minimum evidence window to avoid noisy oscillation. Predict when a small throughput gain cannot repay rebuild overhead. Compare fixed baseline, offline best-tested settings, and the controller under identical permitted workload semantics.

Part 4 - Experimental Design and Instrumentation

Freeze the support matrix and calibrate on separate traces. Implement valid candidate generation, bounded trials, independent correctness/resource gates, and a decision ledger. Apply replacements at an epoch boundary or another explicit committed replay boundary, verifying old ownership ends before new queues increase pressure. Evaluate stable and changing source/transform/shape traces with controlled failure. Run ablations for monitoring, decision cost, hysteresis, and allowed policy dimensions without weakening sample correctness. Report held-out useful throughput, resource evidence, rebuild count, downtime, replayed work, coverage, and failures.

Use a small candidate set with a conservative known-valid fallback. Require minimum observation windows and a benefit threshold before replacement so noise cannot trigger continual worker churn. Reserve enough memory for a safe transition, or terminate the previous loader before constructing its successor. Treat ordering permission, batch semantics, and source replayability as hard constraints that the throughput objective cannot override.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class LoaderTuningContract:
    source_replay_policy: str
    sample_semantics_id: str
    allowed_delivery_policies: tuple
    host_budget_bytes: int
    shared_memory_budget_bytes: object
```

```
    pinned_budget_bytes: object
    reconfiguration_boundary: str
    maximum_retries: int

class SelfTuningDataLoaderController:
    def __init__(self, contract):
        self.contract = contract
        # Initialize bounded observations, candidates, and decision state.
        ...

    def observe(self, stage_evidence, resource_evidence):
        # Preserve support, age, scope, and uncertainty of every metric.
        ...

    def candidates(self, workload_bounds):
        # Generate valid settings without unsupported live mutation.
        ...

    def evaluate(self, candidate, trial_boundary):
        # Check useful-work semantics, resources, and complete cost.
        ...

    def choose(self, results, current_configuration):
        # Account for uncertainty, hysteresis, and replacement overhead.
        ...

    def reconfigure(self, selected, committed_state):
        # Stop old ownership and rebuild at the supported replay boundary.
        ...

    def recover(self, failure, committed_state):
        # Classify cause, bound retries, and preserve exact supported replay.
        ...

    def shutdown(self, deadline):
        # Drain or stop owned processes/resources and report leftovers.
        ...


def E19(config):
    # Validate lifecycle/correctness, then compare held-out useful work.
    ...
    # Export decisions, resource evidence, limitations, and recommendation.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Acceptance requires preserved samples/labels, allowed ordering, coherent training commits, valid resource accounting, bounded lifecycle, and honest time denominators. Record

versions, process context, source/trace IDs, seeds, transforms, schema, batch policy, worker count, prefetch depth, persistence, pinning, and delivery settings, RSS/PSS scope, shared-memory evidence, known pinned payload, device peaks, stage intervals with support labels, update counts, latency samples, reconfiguration cost, coverage, and failure phases.

Demonstrate a rejected invalid zero-worker setting, a retained-worker cleanup, a batched-fetch equivalence check, custom pinning, a controlled shared-memory recovery, and a forbidden semantic reordering. Missing CPU/OS/GPU metrics remain unmeasured; no fabricated speedup or failure is acceptable.

Evaluate the controller on a held-out phase change and a stable workload. Compare total completed work against the best fixed feasible candidate, including calibration, replacement, and recovery costs. Report oscillations, rejected actions, and fallback use. The controller succeeds only when its resource contract and sample correctness hold throughout the run, not merely when its final selected configuration looks reasonable.

Part 7 - Engineering Interpretation and Philosophy

A self-tuning loader is useful when it makes a few justified configuration changes while preserving the computation it serves. Recommend the smallest controller that improves measured complete work within its tested envelope, and state the next evidence required before expanding source, ordering, recovery, device, or scale support.

19.35 DMM33

Chapter Title: Paged Memory and Virtualized Tensor Storage
Specialization: Dynamic Memory Management and CUDA Systems

19.35.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Virtual Addresses, Physical Memory, Pages, Frames, Page Tables, and Memory Translation

Part 1 - Exercise Contract and Why It Matters

Define the vocabulary and supported abstraction for the entire exercise sequence. Deliver a diagram separating a logical tensor address, a software page identifier, a CUDA tensor storage address, and hardware address translation. Use ordinary PyTorch tensors as application-managed page payloads. The resulting pager is a restricted software object with explicit access methods, not an ordinary tensor whose arbitrary operators transparently fault or translate.

Part 2 - System and Memory Mental Model

A virtual address names a location within an address space; a physical frame supplies backing storage at a lower layer. A software page table in this project maps logical page numbers to owned pool slots and generations. It does not inspect hardware page tables, choose GPU physical frames, or change CUDA MMU mappings. CUDA driver VMM is a separate lower-level interface with device support and granularity requirements. Ordinary tensor pointers are addresses, not evidence of physical adjacency. Keep these meanings distinct whenever using the phrase physical page in later exercises.

At the hardware layer, translation conceptually combines a virtual page number with an unchanged within-page byte offset. Page-table entries describe backing and access permissions; an invalid translation or forbidden access is not the same as a software pool miss. Translation

caches can retain recent mappings, so hardware access cost is not one literal full table walk for every element. Page sizes and translation mechanisms depend on the platform. A CPU model is useful for vocabulary, but must not be presented as a specification of every GPU's translation hierarchy, fault service, or migration behavior.

Part 3 - Quantitative Prediction Before Execution

For a flat logical element index i, positive page capacity P in elements, and element width s in bytes, predict page number and within-page offset:

$$p(i) = \lfloor i/P \rfloor, \qquad o(i) = i \bmod P.$$

A length n needs $\lceil n/P \rceil$ payload slots when every page is resident. Separate logical payload, slot capacity, table metadata, temporary results, and allocator-reserved memory. Compute a small boundary example before any CUDA operation, including an empty tensor and the final partially used page.

Part 4 - Experimental Design and Instrumentation

Start with a single device, one dtype, contiguous flattened values, explicit read and write methods, and no autograd support. Define half-open valid index ranges and integer bounds. Classify CPU backing, GPU residency, asynchronous ownership, and shared mappings as later extensions. Record the PyTorch build, CUDA runtime, device, allocator backend, page units, random seed, and supported operations. Build a CPU reference model whose values and ownership can be checked after every state transition.

Draw one address example using element offsets and another using byte offsets, then identify the conversion explicitly. A software page capacity must be positive and its byte width must be representable by the chosen integer type. Do not reuse a hardware term as a variable name without recording its layer. Include object generation and device identity in the translation contract so an otherwise valid offset cannot select a different object's data.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class LogicalAddress:
    object_id: int
    element_index: int

@dataclass
class PageHandle:
    slot_id: int
    generation: int


def translate(address, object_metadata, page_table):
    # Validate bounds and resolve a software slot, not a hardware frame.
```

```
    ...


def E00(config):
    # Draw layers, predict accounting, and validate CPU boundary cases.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require a glossary with a falsifiable claim for each layer. Reject claims that a tensor's contiguous layout proves physical contiguity, or that a list of tensors is automatically usable as one dense CUDA tensor. Preserve prediction, observation, and interpretation separately. Export CPU scalar records without keeping live tensor references in the evidence log. No real hardware page fault, speedup, or allocation outcome may be inferred solely from the software model.

Ask a second reader to classify each diagram arrow as software lookup, payload access, or lower-level translation. Any arrow that implies direct control over hardware frames needs a separately implemented backend and support evidence. Record that backend as absent in the primary exercise; ordinary storage pointers cannot fill this evidence gap.

Part 7 - Engineering Interpretation and Philosophy

Explain which decisions the pager controls and which remain with PyTorch, CUDA, the driver, and the operating system. A precise boundary is essential: the project can demonstrate logical placement, sharing, and explicit migration without claiming a general virtual memory implementation or transparent support for every tensor operator.

19.35.2 Exercise 01

Exercise Title: Compare Contiguous Tensor Allocation Against Fixed-Size Paged Allocation and Identify Where Each Strategy Wastes Memory

Part 1 - Exercise Contract and Why It Matters

Compare two application allocation policies under an identical finite capacity: a contiguous interval allocator and a fixed-size slot allocator. Deliver traces that separate rounding waste, free-space dispersion, retained capacity, and metadata overhead. Add a real PyTorch payload implementation, but keep the controlled arena's conclusions scoped to its declared rules.

Part 2 - System and Memory Mental Model

Contiguous here means consecutive positions in the controlled arena, not adjacent hardware frames. Paging permits an object's logical pages to occupy separate available slots. Both policies still require actual backing capacity. A contiguous dense tensor can already be backed through lower-level virtual memory, so an application-level contiguous failure does not prove that a normal CUDA tensor needs contiguous physical VRAM. Pool slack and allocator cache retention are different accounting quantities.

Part 3 - Quantitative Prediction Before Execution

For requested lengths n_j, calculate useful payload $U = s\sum_j n_j$ and paged assigned capacity $C = sP\sum_j \lceil n_j/P \rceil$. Tail waste is $C - U$; it excludes free pool slots, table bytes, and cached backend allocations. Predict the largest contiguous arena interval and total free capacity for a

fragmented trace. Give one request each policy can satisfy and one neither can satisfy. Keep zero-length objects explicit instead of charging an unnecessary payload page.

Part 4 - Experimental Design and Instrumentation

Replay the same ordered allocation and release events through both policies. Use equal arena bytes, alignment, lifetime boundaries, and failure semantics. Include equal-size objects, a mixed-size distribution, and lengths just above a page boundary. Measure host bookkeeping time separately from completed CUDA payload initialization. Repeat with the same seed and additional seeds, and report startup separately from warmed reuse. Do not allow one policy to grow its arena during a nominally fixed-capacity comparison.

Include a contiguous policy with exact-length intervals and one with the same alignment as the paged policy. This separates placement effects from an unfair rounding advantage. Use a trace where all objects die together as a control, then another with staggered lifetimes. Label each denied request with the resource that was insufficient rather than using one undifferentiated failure count.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class AllocationEvent:
    operation: str
    object_id: int
    length: int


def replay_arena(events, policy, capacity_bytes):
    # Enforce equal capacity, ownership, and failure boundaries.
    ...


def compare_waste(contiguous_rows, paged_rows):
    # Keep tail slack, free holes, metadata, and backend metrics separate.
    ...


def E01(config):
    # Predict adversarial traces, then replay both allocation policies.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report raw requested lengths, assigned slots, contiguous free intervals, failed request size, and metadata scope. Validate that every accepted object contains the expected values and that no released object remains readable. A lower internal-waste ratio does not imply lower total

memory or faster execution. Mark a real allocator observation as a separate experiment; never rename an arena admission rejection a measured CUDA out-of-memory exception.

Recompute occupancy directly from the event trace instead of trusting the allocator's counters. Check that no live intervals overlap and no slot is assigned to two private objects. Compare the time at which memory becomes logically free with the time the backend is allowed to reuse it, especially if initialization or consumption is asynchronous.

Part 7 - Engineering Interpretation and Philosophy

Identify the workload assumptions under which each policy is attractive. Paging trades placement flexibility for translation, metadata, and often operator integration costs. Recommend a policy from measured useful work and complete capacity accounting rather than treating every retained byte as fragmentation.

19.35.3 Exercise 02

Exercise Title: Split a Large PyTorch Tensor into Fixed-Size Logical Pages and Reconstruct the Original Tensor Exactly

Part 1 - Exercise Contract and Why It Matters

Represent a dense tensor as fixed-capacity logical pages and reconstruct its original values and shape exactly. Implement both an aliasing view control and an owning-page variant so the evidence distinguishes slicing from copying. The owning representation becomes the basis for later independently managed payload slots.

Part 2 - System and Memory Mental Model

A slice can be a view of the original storage and retain the entire source allocation. Such a list is a logical partition, not independent backing blocks. Owning pages require explicit storage allocation and copying. The last page has a valid-length field; unused capacity is not part of the tensor. Rejoining with concatenation or another dense output materializes storage and must be counted. Reconstruction does not create a zero-copy dense view across arbitrary independent CUDA allocations.

Part 3 - Quantitative Prediction Before Execution

Predict page count, valid tail length, copied elements, and reconstruction output bytes for lengths 0, 1, $P - 1$, P, and $P + 1$. For each representation, list which storages remain live when the original tensor reference is dropped. Estimate the peak while source, independent pages, and reconstructed output coexist. State whether page construction or reconstruction dominates that peak for the chosen measurement boundary.

Part 4 - Experimental Design and Instrumentation

Use a bounded contiguous tensor first, then test noncontiguous input under an explicit normalization policy whose copy cost is measured. Preserve shape, dtype, and logical element order in metadata. Populate values that reveal page permutation and tail leakage. Compare reconstruction on CPU and CUDA when available, with deliberate completion boundaries. Retain only scalar checksums and mismatch summaries after validation, not every reconstructed tensor from every repetition.

Construct the owning variant with a source pattern that identifies both page number and element offset. Keep source and result lifetimes explicit when measuring peak memory; deleting

a Python name does not remove other views. For an empty source, define shape restoration without manufacturing an unnecessary payload page. Record any contiguous normalization performed before flattening a strided input.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class PagedImage:
    shape: tuple
    dtype: object
    page_elements: int
    valid_lengths: tuple
    pages: list


def split_pages(tensor, page_elements, ownership_policy):
    # Make aliasing or owning behavior explicit and preserve valid lengths.
    ...


def reconstruct(image):
    # Materialize only valid elements in their original logical order.
    ...


def E02(config):
    # Validate empty, boundary, layout, and ownership cases.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require elementwise equality for the supported finite test data. If claiming bitwise preservation, use an appropriate byte-level comparison and include signed zero and selected nonfinite encodings; ordinary equality treats NaNs differently. Explain any unsupported dtype or layout. Verify source mutation propagation for the view control and independence for owning pages. Report allocated and reserved CUDA peaks separately, including temporary reconstruction.

Inspect storage ownership separately from numerical equality. Two outputs can have identical values while one retains the source allocation and the other owns independent storage. Compare storage offsets as well as tensor pointers when diagnosing views. If validating exact raw encodings, avoid a comparison that normalizes or converts the values before checking them.

Part 7 - Engineering Interpretation and Philosophy

Explain why a correct round trip establishes representation correctness but not an allocation reduction. Identify when views suffice, when independent page ownership is necessary, and when reconstructing a dense result defeats the memory goal of a paged consumer.

19.35.4 Exercise 03

Exercise Title: Build a Physical Page Pool in PyTorch That Allocates and Releases Fixed-Size CUDA Blocks on Demand

Part 1 - Exercise Contract and Why It Matters

Build a bounded application page pool with acquire, release, and validation operations. Allocate fixed-capacity CUDA payload blocks when required and reuse released slots safely. Deliver a lifecycle ledger that distinguishes returning a slot to the application pool from returning memory to PyTorch or the driver.

Part 2 - System and Memory Mental Model

The pool's physical page is an owned tensor payload, not a chosen GPU frame. Choose independent blocks for the primary implementation. An optional slab backend must disclose that all slot views retain the slab and cannot release its individual backing allocations. A free-list entry is usable only when no logical owner or in-flight operation still depends on the slot. Generation numbers distinguish a reused slot from an old handle with the same index.

Part 3 - Quantitative Prediction Before Execution

For K fixed slots, predict payload capacity KPs, currently assigned pages, free pages, and pending-release pages. Require disjoint states and a count invariant covering every allocated slot. Predict backend allocation calls for a cold burst followed by equal-size reuse. Do not predict that reserved CUDA memory falls when the application merely appends a released slot to its free list. Include page metadata and temporary initialization bytes separately.

Part 4 - Experimental Design and Instrumentation

Start with one host thread and one CUDA stream, bounded requests, and explicit quiescent release. Test pool exhaustion through admission control before attempting unsafe real-device exhaustion. Assign recognizable payload patterns, release slots in a shuffled order, and verify later reacquisition. Add stale handle, double release, foreign handle, and wrong-device tests. Compare cold creation, warmed reuse, and final pool destruction with matching measurement boundaries and peak resets.

Add a bounded allocation-failure injector that rejects the third requested slot before any CUDA allocation is attempted. Verify that provisional state is cleaned without relying on a device-wide failure. Exercise both retained free slots and a policy that destroys independent free blocks after quiescence; these policies have different backend footprints even when their logical free counts match.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass(frozen=True)
class PageHandle:
    slot_id: int
    generation: int
```

```
class CudaPagePool:
    def __init__(self, page_elements, max_slots, device, dtype):
        # Establish bounded free, owned, and pending slot state.
        ...

    def acquire(self):
        # Validate admission and return a generation-checked handle.
        ...

    def release(self, handle, completion_token):
        # Reuse only after ownership and execution dependencies end.
        ...

    def audit(self):
        # Reconcile every slot and backend allocation.
        ...


def E03(config):
    # Exercise cold allocation, reuse, exhaustion, and invalid handles.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Audit state after every operation and compare values before and after reacquisition according to the initialization contract. Export live owned payload, free retained payload, pending payload, allocation-call counts, and backend allocated and reserved peaks. Report unsupported allocator counters rather than substituting zeros. If a slot is released with unfinished work, the implementation must defer reuse or reject the operation; reference deletion alone is not proof of completion.

Record generation changes across multiple acquire and release cycles, not just the first reuse. A raw pointer can repeat because the allocator reuses an address; that does not make a stale handle valid. Confirm that evidence logging stores scalar identifiers rather than tensor objects whose references would accidentally extend page lifetime.

Part 7 - Engineering Interpretation and Philosophy

Choose whether the pool optimizes predictable capacity, reduced allocator calls, or both. Explain the retained-memory cost of keeping free pages and why safe reuse depends on lifetime evidence beyond a Python free list.

19.35.5 Exercise 04

Exercise Title: Implement a Page Table That Maps Logical Tensor Pages to Noncontiguous Physical CUDA Blocks

Part 1 - Exercise Contract and Why It Matters

Implement logical page tables whose entries reference nonconsecutive pool slots. Deliver lookup, installation, replacement, and object destruction operations with checked bounds and

generations. Preserve a stable logical object identity even when its payload placement changes. This exercise manages software indirection without creating a contiguous CUDA virtual address range.

Part 2 - System and Memory Mental Model

An entry must describe a valid mapping, not merely contain an integer slot index. Record object length, page capacity, valid tail length, placement, generation, and access policy. Distinguish absent, resident, and invalid handles. The initial CPU-owned table may drive Python dispatch; it is not automatically available inside a CUDA kernel. A device table extension requires explicit representation, upload, and publication dependencies, all measured separately.

Part 3 - Quantitative Prediction Before Execution

Translate several indices around page boundaries through a deliberately permuted mapping. Calculate required entries and metadata bytes under a declared packed representation. Python object overhead cannot be inferred from the packed field widths alone. Predict the result of a stale mapping after slot reuse and identify the generation check that must reject it. Explain why two objects may use the same logical page number without sharing any backing.

Part 4 - Experimental Design and Instrumentation

Use independently initialized pool slots and install a randomized bijection for one object. Compare every translation against a CPU oracle. Test missing entries, negative indices under a reject policy, oversized indices, invalid object IDs, stale generations, and foreign pool handles. Define failure atomicity: a failed replacement leaves the old table and ownership intact. Keep host table updates serialized in the primary implementation before adding asynchronous table consumers.

Give one object a table whose logical pages map to slot IDs in descending order, and another a different permutation of separate slots. This makes an accidental identity mapping fail immediately. Define whether table replacement invalidates cached translations and how the consumer learns the new version. A debug lookup cache must not silently outlive the table generation that made its entry valid.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class MappingEntry:
    handle: object
    state: str
    valid_elements: int

class SoftwarePageTable:
    def lookup(self, object_id, logical_page):
        # Resolve a checked entry without inventing missing residency.
        ...

    def install(self, object_id, logical_page, entry):
```

```
        # Validate the mapping before publishing ownership changes.
        ...

    def replace(self, object_id, logical_page, expected, replacement):
        # Preserve the old mapping if preparation or validation fails.
        ...


def E04(config):
    # Validate shuffled placement, bounds, and stale generation rejection.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require the mapping ledger to reconcile with pool ownership at every stable boundary. Verify translations independently from payload reads so a coincidental pattern cannot hide wrong placement. Report table construction and lookup time separately from CUDA work. If exposing a table snapshot to a consumer, specify how that snapshot remains valid until the consumer finishes. Never reuse an entry's slot while an older table version can still authorize access.

Test a replacement whose candidate belongs to the wrong pool despite having a numerically valid slot index. Require rejection before ownership changes. Then audit a successful replacement by recomputing table references and old slot release eligibility. If metadata lives on the device in an extension, include its publication event and allocated bytes in the evidence.

Part 7 - Engineering Interpretation and Philosophy

Explain how indirection decouples logical position from placement and why this benefit creates a publication and lifetime obligation. State which metadata would be needed for a future kernel implementation without claiming that the current Python table already supplies hardware translation.

19.35.6 Exercise 05

Exercise Title: Read a Logical Tensor Through Page-Table Translation Without Requiring Its Physical Blocks to Be Contiguous

Part 1 - Exercise Contract and Why It Matters

Read selected logical elements and ranges through the software page table. Support shuffled payload placement without first reconstructing the entire object. Deliver a correct explicit access path and quantify its translation, dispatch, and output-materialization costs. Restrict the primary path to read-only, non-autograd data with the established device and dtype contract.

Part 2 - System and Memory Mental Model

Each logical access resolves an object-relative page number and an offset, then reads the corresponding resident payload. A range crossing a boundary requires multiple page fragments. Gathered output is a new dense result unless a specific single-page view contract is chosen. Returning internal writable views would expose a mutation bypass, so define ownership and access rights before exposing them. Software translation adds work and does not make arbitrary PyTorch operators consume the pager transparently.

Part 3 - Quantitative Prediction Before Execution

For a range $[a, b)$, predict touched pages and valid fragment lengths, including an empty range and an unaligned final element. For an index list, compare access count with distinct page count. Repeated indices need repeated output values but need not require repeated page lookup. Estimate output bytes, translation metadata, and dispatch calls for scalar, per-page grouped, and full-reconstruction strategies before measuring any of them.

Part 4 - Experimental Design and Instrumentation

Construct a random physical placement with monotonically identifiable logical values. Test contiguous ranges, shuffled indices, duplicates, empty requests, and one request crossing every boundary. Compare results against a dense CPU oracle in the original requested order. Benchmark hot reads separately from fault handling, which has not been implemented yet. Use completed CUDA regions and include dense output allocation in the declared read cost. Keep translation instrumentation bounded and export only scalar summaries.

Use a request order that revisits pages after visiting distant ones, so a grouped implementation must restore original output order rather than return page-sorted values. Include a request whose last element is the final valid logical element. Define the lifetime of any single-page read view and whether it is a copied result or a managed lease; do not mix these meanings in one unlabeled benchmark.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

class PagedReader:
    def read_range(self, object_id, start, stop):
        # Validate, split at boundaries, translate, and assemble valid data.
        ...

    def gather(self, object_id, logical_indices):
        # Preserve request order and duplicate reads under grouped dispatch.
        ...


def compare_read_paths(pager, dense_reference, requests):
    # Separate translation, payload work, output bytes, and full latency.
    ...


def E05(config):
    # Predict touched pages, then validate sparse and range access paths.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require exact values for the supported finite test data and explicitly compare output order. Record requested elements, unique touched pages, table lookups, output bytes, and completed

latency. A Python loop with many small CUDA launches may be slower than dense reconstruction; report that outcome. Reject nonresident or stale entries according to the current contract rather than reading arbitrary storage. State whether host index conversion or device index validation introduces synchronization.

Compare measured data movement with the required output payload, then explain extra full-page copies or temporary index buffers. An access that touches one element on every page can have low useful bytes and high dispatch overhead. Report that unfavorable case even if a large sequential read makes the same implementation appear efficient.

Part 7 - Engineering Interpretation and Philosophy

Decide when sparse logical access avoids enough materialization to justify indirection. Explain why correctness on small reads does not establish a competitive general tensor backend, and identify the workload that would justify a grouped kernel extension.

19.35.7 Exercise 06

Exercise Title: Write Through Logical Page Addresses and Verify That Updates Reach the Correct Physical CUDA Blocks

Part 1 - Exercise Contract and Why It Matters

Implement explicit writes through logical addresses while preserving all untouched values. Support ranges and a declared indexed-write policy. Verify that updates reach the mapped slots rather than assuming logical and physical page numbers coincide. Keep sharing and copy-on-write disabled until their later ownership contracts are implemented.

Part 2 - System and Memory Mental Model

A logical mutation can touch multiple independent payloads and may require several CUDA operations. Bounds checking and value-shape validation must precede mutation if the operation promises all-or-nothing admission. Repeated destination indices require a defined overwrite or reduction rule; unordered duplicate writes cannot be treated as deterministic last-writer behavior without an implementation that enforces that order. Autograd versioning and arbitrary external tensor aliases are outside the primary software pager contract.

Part 3 - Quantitative Prediction Before Execution

Predict the page fragments changed by a write starting at $P-1$ with length three. Identify all unchanged neighboring elements. Calculate useful write bytes versus page capacity touched, and distinguish initialization of a new page from rewriting a resident page. For a duplicate-index example, write the expected result under a deterministic sequential policy and contrast it with a commutative reduction policy whose floating-point ordering may still matter.

Part 4 - Experimental Design and Instrumentation

Initialize every page with distinct values and place sentinels around the valid logical region. Validate source values, destination bounds, dtype, and layout before dispatch. Test a single-page write, boundary crossing, empty write, full-object write, invalid tail access, and repeated indices. Compare the whole object with a CPU reference after each operation. Start with one stream and one writer, then add explicit completion evidence before exposing the object to a different stream.

Choose an initial write policy that rejects duplicate indices, then add a separately tested deterministic implementation if duplicate writes are needed. Make value broadcasting either

explicitly supported with reference checks or explicitly rejected. A permissive tensor copy primitive must not accidentally expand the pager's public contract. Record whether each write marks its page dirty even when the new value equals the previous value.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

class PagedWriter:
    def write_range(self, object_id, start, values):
        # Validate the entire request before committing supported writes.
        ...

    def write_indices(self, object_id, indices, values, duplicate_policy):
        # Enforce the declared conflict rule rather than assuming ordering.
        ...

    def completion(self, operation_id):
        # Return evidence that downstream readers may safely consume data.
        ...


def E06(config):
    # Compare mapped updates and unchanged regions against a CPU oracle.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report logical bytes modified, pages touched, backend operations, temporary bytes, and completed latency. Check untouched regions and padding, not only the requested output. Inject an invalid index at the end of a request to verify the declared rejection boundary. Distinguish validation failures before publication from execution failures after partial work; do not promise general rollback of arbitrary asynchronous device errors.

Use a source tensor that would alias the destination under a proposed view extension and reject it until overlap semantics are defined. Partial self-copy can require a temporary snapshot. Check that a completed write is visible to a later reader on the supported stream dependency, rather than inferring visibility from a quickly returned Python method.

Part 7 - Engineering Interpretation and Philosophy

Explain why writable software memory needs a stronger contract than read-only translation. Choose the smallest deterministic mutation interface suitable for later copy-on-write and migration, and document every unsupported alias or concurrency pattern that could bypass it.

19.35.8 Exercise 07

Exercise Title: Allocate Variable-Length Logical Tensors from a Shared Fixed-Size Page Pool and Measure Internal Memory Waste

Part 1 - Exercise Contract and Why It Matters

Allocate many variable-length logical objects from one fixed-size page pool. Measure per-object tail waste, aggregate assigned capacity, retained free slots, and metadata separately. Produce a replayable lifetime trace rather than estimating total memory from average sequence length alone.

Part 2 - System and Memory Mental Model

Each nonempty object owns enough page mappings for its current length. Unused elements in its final page are internal slack under the chosen policy. An unassigned retained slot is pool capacity, not another object's tail waste. Small independent objects cannot share a tail slot unless a suballocation scheme is explicitly added; such a scheme changes translation and ownership and is outside this exercise. Shared page mappings are also excluded here so payload accounting remains directly auditable.

Part 3 - Quantitative Prediction Before Execution

For lengths 1, $P-1$, P, and $P+1$, derive assigned pages and slack. Calculate aggregate waste both as bytes and as a fraction of assigned payload capacity, leaving an empty denominator undefined. Compare two length distributions with the same mean but different boundary positions. Predict admission when free slots equal the required count and when they are one short. Include object-table metadata and peak simultaneous ownership.

Part 4 - Experimental Design and Instrumentation

Generate bounded short, long, and mixed object traces with fixed creation and destruction events. Preserve the same useful values and lifetime overlap across policy comparisons. Audit allocation and release after every event. Repeat with a shape distribution concentrated near page boundaries and a separate held-out distribution. Measure CPU bookkeeping, CUDA initialization, and steady reads independently before reporting complete useful-work time. Predeclare whether the pool grows on demand up to a maximum or starts fully allocated.

Keep object initialization policy constant when comparing size distributions. Zero-filled payloads and uninitialized allocations have different costs and correctness guarantees. Record each object's birth and death event so peak simultaneous assignment can be reconstructed independently. Add a distribution with many empty objects to expose metadata overhead that a payload-only metric would miss completely.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class LogicalObject:
    object_id: int
    length: int
    page_handles: list
```

```
def allocate_object(pool, length, initialization_policy):
    # Admit all required pages or leave pool ownership unchanged.
    ...


def waste_ledger(objects, pool):
    # Separate logical payload, tail slack, free slots, and metadata.
    ...


def E07(config):
    # Replay variable lengths and compare predictions with audited states.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report object count, length histogram, useful elements, assigned slots, free retained slots, pending slots, and rejected allocations. Reconcile all bytes using the same dtype and units. A reported small waste percentage is misleading if free capacity or table overhead is silently omitted from total residency. Validate that a failed multi-page allocation returns any provisional slots without corrupting earlier objects or exposing partially initialized data.

Audit the sum of each final page's valid length and slack against assigned capacity. An object with a length exactly divisible by page capacity has no tail waste. Report both per-object statistics and the byte-weighted aggregate; a simple average of waste percentages can rank workloads differently from aggregate wasted bytes.

Part 7 - Engineering Interpretation and Philosophy

Recommend a page size only for the tested length and lifetime distribution. Explain why low tail waste, high admission probability, and low total device residency are related but distinct objectives. Preserve the workload trace so later allocator changes can be compared fairly.

19.35.9 Exercise 08

Exercise Title: Sweep Page Size and Measure the Trade-Off Between Internal Fragmentation, Page-Table Size, Allocation Overhead, and Reuse Flexibility

Part 1 - Exercise Contract and Why It Matters

Sweep application page capacity and quantify the competing costs of tail slack, table entries, allocation activity, dispatch overhead, and reusable capacity. Select a tested operating point under explicit memory and latency constraints. Do not identify the software page size with an operating-system page size or CUDA VMM mapping granularity.

Part 2 - System and Memory Mental Model

Smaller software pages can reduce tail slack and improve placement flexibility while increasing mappings and per-page work. Larger pages can improve transfer and dispatch efficiency while wasting more capacity for short objects. Backend allocation rounding may make measured reserved memory differ from application payload calculations. A fixed byte budget implies a different maximum slot count for each page size, which must be reflected in admission comparisons.

Part 3 - Quantitative Prediction Before Execution

For each candidate P, calculate assigned capacity and mapping count from the same length trace. State a packed metadata model and separately measure actual host bookkeeping when possible. Predict page touches for the same range and random-access requests. Include a transfer-amplification estimate when a small update eventually requires moving a whole page. Identify candidates that violate the payload budget before running timed experiments.

Part 4 - Experimental Design and Instrumentation

Use a bounded geometric page-size sweep compatible with the supported dtype. Keep useful object lengths, access order, mutation pattern, and byte budget constant. Rebuild the pool for each candidate and separate cold creation from warmed reuse. Include short-object, long-object, and mixed traces. Repeat near the apparent optimum on a held-out trace rather than selecting from one noisy measurement. Report both per-operation latency and full completed useful work, including dense outputs or staging buffers that the consumer requires.

Keep software page capacity in elements and byte capacity beside it for every trial. Changing dtype while holding element capacity would change the byte budget, so treat dtype as fixed in the primary sweep. Use the same maximum logical object sizes and retain failed candidates in the report. A candidate cannot win by omitting requests that no longer fit its rounded capacity.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class PageSizeTrial:
    page_elements: int
    payload_budget_bytes: int
    trace_id: str


def predict_page_costs(trial, lengths, accesses, dtype_bytes):
    # Compute separate capacity, table, touch, and transfer estimates.
    ...


def run_page_size_trial(trial, workload):
    # Rebuild equal-budget state and measure complete correct work.
    ...


def E08(config):
    # Evaluate feasible candidates and validate the selected compromise.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Keep raw lengths, bytes, mapping counts, allocator observations, failures, and timing distributions beside derived scores. Test a counterexample to the rule that smaller pages always save memory and another to the rule that larger pages are always faster. Mark unavailable metadata measurements explicitly. If the sweep changes backend allocation sizes and caching behavior, describe that additional mechanism instead of attributing every difference to tail fragmentation.

Inspect whether a smaller-page candidate reduces assigned payload but increases backend reserved memory or metadata. Explain that outcome using the recorded layers rather than treating it as contradictory. Include confidence intervals or repeated-run spreads around close latency rankings, and prefer a robust feasible point when the apparent optimum is noisy.

Part 7 - Engineering Interpretation and Philosophy

Choose a minimal justified policy for this project, possibly a single page size. Explain what evidence would justify multiple size classes and the extra ownership complexity they introduce. A benchmark optimum is conditional on its workload, budget, backend, and measurement region.

19.35.10 Exercise 09

Exercise Title: Reproduce External Fragmentation with Contiguous Allocation and Demonstrate How Noncontiguous Paging Avoids the Same Failure

Part 1 - Exercise Contract and Why It Matters

Construct a deterministic external-fragmentation failure in a bounded application arena, then replay the same lifetime trace with nonconsecutive page assignment. Demonstrate a request that paging admits despite lacking a sufficiently large contiguous free interval. Preserve equal total backing capacity and label the failure at the application admission layer.

Part 2 - System and Memory Mental Model

Use six equal arena slots as a minimal example. Allocate one object per slot, then release alternating objects. Three free slots exist but none are adjacent. A contiguous request for two slots fails under the arena's interval policy; a paged request can use two separate free slots. This says nothing about hardware frame adjacency or the current CUDA allocator's ability to grow, coalesce, or remap its own storage. The arena is deliberately fixed and its rules are part of the experiment.

Part 3 - Quantitative Prediction Before Execution

List free intervals and total free slots after every trace event. Predict both policy decisions for two-slot and four-slot requests. Explain why the four-slot request fails under either policy. Add a request requiring $P + 1$ elements so rounding is visible. Compare useful payload and admitted capacity without merging tail slack with separated free intervals. State the exact smallest trace that distinguishes the two admission rules.

Part 4 - Experimental Design and Instrumentation

Implement the arena model on CPU first and verify the expected decisions. Attach real CUDA payloads only to accepted slots in a separate execution phase. Prevent either policy from increasing arena capacity or silently compacting live objects. Keep initialization and logical value checks identical. If exploring real allocator fragmentation later, use an independent

bounded experiment with backend/version evidence; do not treat the controlled trace as a promised CUDA out-of-memory reproducer.

Visualize the arena with a table of slot index, current owner, and free-run identity after every event. The same slot table must drive both admission predictions and independent value checks. Add a compacting contiguous variant only as a separate comparison that accounts for moved payload and updated references; compaction is not free and changes the original failure model.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class ArenaDecision:
    request_slots: int
    accepted: bool
    free_slots: int
    largest_free_run: int


def fragmented_trace(slot_count):
    # Construct and label the minimal alternating-release example.
    ...


def replay_fixed_arena(events, placement_policy):
    # Enforce fixed capacity and preserve all live object values.
    ...


def E09(config):
    # Compare predicted admission with CPU model and optional CUDA payloads.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require a complete event ledger and a value audit of every surviving object. Report policy rejection separately from actual backend allocation failure. Include an all-free control, a no-free control, and a fragmented case whose total capacity is still insufficient. A successful paged admission does not prove less total residency: mapping metadata and tail waste remain present. Do not fabricate a hardware outcome when CUDA is unavailable.

Demonstrate that numerical adjacency of CUDA pointers is irrelevant to the arena's declared consecutive-slot rule. Likewise, separated pointers do not reveal hardware frame placement. If an actual allocation unexpectedly succeeds or fails, retain that observation with its backend context rather than editing the controlled trace to manufacture agreement.

Part 7 - Engineering Interpretation and Philosophy

Explain the precise failure paging avoids and the capacity failures it cannot avoid. Use the controlled counterexample to challenge broad fragmentation claims, then identify the additional evidence needed before transferring the conclusion to a production allocator.

19.35.11 Exercise 10

Exercise Title: Grow a Logical Tensor Incrementally by Attaching New Physical Pages Without Reallocating or Copying Existing Pages

Part 1 - Exercise Contract and Why It Matters

Grow a logical object by appending mappings while retaining all existing page payloads. Preserve old logical values and handle identities, initialize newly exposed elements according to a declared policy, and support failure without publishing a partially grown object. This is logical growth through indirection rather than an in-place resize of one dense tensor storage.

Part 2 - System and Memory Mental Model

Growth within a partial tail can expose previously invalid elements without acquiring another slot. Crossing a page boundary requires additional pages. The logical length and table update must become visible only after required allocation and initialization dependencies are established. Growing a Python list or device page table may copy metadata even when no old payload page moves. Distinguish payload stability, table stability, and a reconstructed dense output's unrelated pointer.

Part 3 - Quantitative Prediction Before Execution

For an initial length $P - 2$, predict new slots needed after growth by one, three, and $P + 3$ elements in separate replays. Calculate payload initialization bytes and table changes. List which old slots and generations must remain unchanged. Predict peak provisional ownership while preparing a multi-page growth. If the pool cannot supply the complete request, specify the unchanged length, table, and values that rejection must preserve.

Part 4 - Experimental Design and Instrumentation

Use a deterministic append trace crossing multiple boundaries. Record old handles and checksums before each growth, then verify the entire prefix and newly exposed region afterward. Test zero growth, invalid negative growth, integer overflow, tail-only growth, and capacity rejection. Keep a separate reference implementation that reconstructs a dense object on every append, and compare complete cost while acknowledging that the two representations provide different operator interfaces. Do not expose uninitialized tail bytes as valid user data.

Choose newly exposed values from an append counter so tail-only growth cannot accidentally pass using old padding contents. Capture a scalar snapshot of the old mapping handles before preparation. If table growth allocates new metadata storage, report its transient coexistence with the old version. Keep previously issued consumer leases valid or reject concurrent growth under the initial serialized contract.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
```

```
class GrowablePagedObject:
    def plan_growth(self, new_length):
        # Count tail exposure, new pages, metadata, and provisional capacity.
        ...

    def grow(self, new_length, initialization_policy):
        # Prepare dependencies and publish length/table coherently.
        ...

    def audit_prefix(self, previous_state):
        # Confirm original payload handles and values remain valid.
        ...


def E10(config):
    # Test boundary growth, initialization, and failure atomicity.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report old payload copies separately from new-element initialization and metadata movement. A claim of no copying applies only to preserved payloads, not to all work in the operation. Verify rejection leaves no leaked provisional pages. Benchmark after deliberate CUDA completion and reset peaks for the specified region. If growth shares a tail with another object in later extensions, require copy-on-write before modifying that shared page.

Verify prefix preservation after several growth and shrink cycles, not only a single append. A stable tensor pointer by itself does not prove the prefix was not overwritten; compare both identity and content. For an admission failure, inspect the provisional free list and ensure the next unrelated allocation can still use every legitimately free slot.

Part 7 - Engineering Interpretation and Philosophy

Explain when stable page payloads help incremental workloads and when indirection or a dense materialization remains expensive. State the exact stability guarantee that downstream consumers may rely on, including the conditions that invalidate previously obtained views or table snapshots.

19.35.12 Exercise 11

Exercise Title: Shrink a Logical Tensor and Return Only Its Unused Physical Pages to the Shared Free-Page Pool

Part 1 - Exercise Contract and Why It Matters

Shrink a logical object and return only wholly unused pages to the pool. Preserve retained values and apply explicit lifetime checks before reuse. Define truncation semantics so later regrowth cannot accidentally reveal data that the contract considers discarded or uninitialized.

Part 2 - System and Memory Mental Model

Shrinking changes valid length before it necessarily changes physical backing. The page containing the new final element remains owned even when most of it becomes slack. Removed

mappings release logical ownership, but pending readers, writes, transfers, or shared owners may still prevent slot reuse. Returning a slot to a retained pool does not guarantee a fall in allocator-reserved memory. A slab-backed pool cannot free one slot's backend allocation independently.

Part 3 - Quantitative Prediction Before Execution

For length $3P + 2$, compute retained pages after shrinking to $3P$, $P + 1$, P, and zero in separate trials. Distinguish removable mappings from actually reusable slots. Predict how much assigned slack remains after each shrink and which bytes a later regrowth must initialize. Include a case where a page has no logical mapping but still has an in-flight reader, so immediate reuse would violate correctness.

Part 4 - Experimental Design and Instrumentation

Fill objects with boundary-sensitive values, shrink them, and verify surviving prefixes against a CPU oracle. Test unchanged length, complete destruction, invalid larger target length, and repeated shrink requests. Start with quiescent single-owner objects, then use a controlled completion token to defer one release. Reacquire released slots for a different object and ensure the old object cannot access them through a stale table. Keep physical release policy separate from logical truncation.

Test shrinking by one element when the old final page remains necessary, and shrinking by one element when that change removes an entire page. These nearby lengths expose off-by-one reclamation errors. If readers hold leases to removed mappings, record retired state separately from the current object's table. Later completion must release that retired ownership without restoring the old logical length.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class ShrinkResult:
    retained_length: int
    removed_mappings: int
    reusable_slots: int
    deferred_slots: int


def shrink_object(pager, object_id, new_length):
    # Remove only unused mappings and preserve retained tail ownership.
    ...


def reclaim_completed(pool, completion_evidence):
    # Move deferred slots to free state only after all users finish.
    ...
```

```
def E11(config):
    # Validate prefix survival, stale access rejection, and later regrowth.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Audit free, mapped, and deferred counts after every shrink and completion. Report application-assigned capacity, pool-retained capacity, and backend metrics independently. Check that a truncated region is inaccessible and that regrowth obeys the initialization policy. If data sanitization is part of the chosen contract, measure it explicitly rather than equating release with erasure. Do not infer a leak merely because cached reserved memory stays high.

Destroy an object whose length is already zero and define whether the second destruction is a clear invalid-handle error. Avoid silent counter decrements. Check that a delayed completion for a retired page cannot free its new generation after reuse. This audit requires matching the completion token to the exact slot generation it protects.

Part 7 - Engineering Interpretation and Philosophy

Explain why logical deletion, ownership release, reusable capacity, and backend deallocation are distinct events. Choose a truncation and regrowth policy suitable for the later multi-object manager and document the consumer lifetime conditions required for safe reclamation.

19.35.13 Exercise 12

Exercise Title: Implement Page Reference Counting So Multiple Logical Tensors Can Safely Share Physical Pages

Part 1 - Exercise Contract and Why It Matters

Allow multiple logical objects to share payload pages using explicit mapping reference counts. Deliver checked share, unshare, destruction, and deferred reclamation operations. Keep mutation disabled for shared pages until the copy-on-write exercise defines the required behavior.

Part 2 - System and Memory Mental Model

A mapping reference count measures owning table entries, not arbitrary Python references and not the number of CUDA kernels still reading a page. Maintain in-flight use separately. A page becomes reusable only after its mapping count and outstanding use obligations both reach zero. Internal tensor views exposed outside the pager would create untracked aliases, so disallow them or define an explicit lease. Generation checks remain necessary after a slot is recycled.

Part 3 - Quantitative Prediction Before Execution

Start with two objects sharing three full pages and one object with a private tail. Calculate unique payload slots, total mappings, and counts per slot. Destroy the objects in different orders and predict which slots become reusable at each step. Add a retained read lease to one page and show why count zero alone cannot authorize reuse. Compare logical bytes across objects with unique backing bytes without double-counting shared payload.

Part 4 - Experimental Design and Instrumentation

Create shared objects through a single controlled API rather than copying table dictionaries directly. Verify each count against a recomputation from all live table entries. Test duplicate mappings within one object, partial sharing, complete sharing, destruction order, stale handles, and double unshare. Serialize host ownership changes initially. Record pending leases and their completion boundaries so the reference model can independently predict release. Keep random traces bounded to allow an audit after every event.

Create one object with two logical entries pointing to the same payload and another with one entry to it. Count three mapping references under this model, not two object references. State whether this alias pattern is supported for reads only until copy-on-write behavior is defined. Store mapping identities so repeated attach or detach calls cannot increment or decrement a counter twice without detection.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class SharedPageState:
    mapping_references: int
    inflight_leases: int
    generation: int

class PageOwnership:
    def attach(self, handle, mapping_id):
        # Validate and increment exactly once for a new owning mapping.
        ...

    def detach(self, handle, mapping_id):
        # Remove the mapping and defer reuse while leases remain.
        ...

    def audit(self, all_tables, active_leases):
        # Recompute ownership independently from the maintained counters.
        ...


def E12(config):
    # Exercise sharing and destruction orders against the ownership oracle.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require nonnegative counts, one ownership contribution per live mapping, and no free slot reachable from a valid mapping or lease. Demonstrate a counterexample where counting logical objects rather than mapping entries is wrong. Measure metadata overhead and avoided payload duplication separately. A reduced unique payload count does not establish total memory reduction if new tables or retained leases dominate the workload.

Use an independently generated multiset of mapping handles to recompute counts. Do not let the audit read the same maintained counters it is supposed to verify. Test a retained lease after all tables disappear, then complete that lease and check reclamation. This distinguishes logical reachability from outstanding execution lifetime.

Part 7 - Engineering Interpretation and Philosophy

Explain how sharing changes memory accounting and reclamation without making writes safe automatically. Define the ownership evidence that copy-on-write will need, and state why ordinary external aliases cannot be inferred completely from the pager's own mapping counters.

19.35.14 Exercise 13

Exercise Title: Implement Copy-on-Write for Shared PyTorch Pages and Allocate New Physical Storage Only When a Shared Page Is Modified

Part 1 - Exercise Contract and Why It Matters

Implement software copy-on-write for shared pages behind the pager's controlled write API. Preserve snapshot values for other logical owners while allowing a writer to modify its private version. Allocate replacement payload only for pages that require separation, and include allocation, copy, and publication costs in the evidence.

Part 2 - System and Memory Mental Model

PyTorch tensor writes do not automatically implement this project's page copy-on-write semantics. The manager must intercept every supported mutation. For a shared page, prepare private backing, copy the values that must survive, apply the update, and publish a coherent replacement mapping with correct ownership. A partial update usually needs the untouched old values. A complete valid-page overwrite may avoid that copy under a declared policy. Keep source pages and old table versions alive until all readers and copies finish.

Part 3 - Quantitative Prediction Before Execution

For two objects sharing four pages, predict new slots and copied bytes for a one-element write, a cross-boundary write, and a complete-page overwrite. Distinguish unique pages touched from number of elements written. Calculate peak ownership while old shared payloads and provisional private pages coexist. Show a pool-full case where copy-on-write must reject or defer the mutation; being able to read shared data does not imply enough space to privatize it.

Part 4 - Experimental Design and Instrumentation

Create two snapshots with identical logical values. Mutate one through only the controlled API and compare both complete objects against independent CPU references. Test repeated writes to an already private page, duplicate logical mappings, partial tail updates, and allocation failure. Serialize writers in the primary version; a concurrent extension requires locking or another defined publication protocol. Reject external writable view access that bypasses the manager. Start with no autograd rather than promising gradient preservation.

Define whether repeated aliases within one logical object belong to a shared mutation group or represent independent snapshot mappings. The selected copy-on-write behavior must preserve that declared relationship. For the primary snapshot model, page replacement affects only the targeted mapping unless the contract says otherwise. Do not infer snapshot semantics from a reference count without defining which mappings a write should update.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

class CopyOnWritePager:
    def plan_write(self, object_id, region):
        # Determine touched sharing groups and reserve transition capacity.
        ...

    def prepare_private_pages(self, plan):
        # Copy only required surviving values and retain old ownership.
        ...

    def commit_write(self, plan, values):
        # Apply supported mutation and publish checked mapping replacements.
        ...


def E13(config):
    # Validate snapshot isolation, copy accounting, and failure boundaries.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require the unmodified snapshot to remain exactly unchanged and the writer to match its reference. Audit counts and generations after success and rejection. Report copied payload, newly initialized payload, provisional capacity, and steady unique backing. Distinguish admission atomicity from recovery after an unexpected asynchronous execution failure. No general transactional rollback of device faults is required or implied by this exercise.

Inject a failure after one of several replacement pages is prepared but before publication. Require provisional resources to be returned while old snapshots stay readable. On successful publication, preserve the old source until every copy and reader finishes. Record whole-page copy amplification for sparse writes so sharing savings are assessed alongside later mutation cost.

Part 7 - Engineering Interpretation and Philosophy

Explain when snapshot sharing defers rather than eliminates allocation. Recommend a mutation policy from the measured write density and available transition headroom, and identify why a transparent general tensor copy-on-write system would need stronger integration than this restricted manager.

19.35.15 Exercise 14

Exercise Title: Add CPU-Resident Pages and Implement Explicit GPU Page-In and Page-Out Operations Under a Fixed VRAM Page Budget

Part 1 - Exercise Contract and Why It Matters

Extend the pager with CPU backing and explicit GPU page-in and page-out under a fixed application CUDA slot budget. Preserve values across migration and make residency transitions observable. Distinguish the manager's payload budget from total device usage, which also includes outputs, workspaces, metadata, other allocations, and the caching allocator.

Part 2 - System and Memory Mental Model

A nonresident logical page retains an authoritative CPU version or another explicit reconstructible source. Track clean and dirty versions, transfer state, and backing ownership. Do not reuse a GPU slot until its latest values are safely backed and all device users finish. CPU backing consumes RAM; pinned staging consumes additional bounded capacity. Host-to-device submission is not transfer completion, and pinned sources must remain unchanged while transfers still depend on them.

Part 3 - Quantitative Prediction Before Execution

For K GPU payload slots and a larger logical page set, predict residency, CPU backing, dirty writeback bytes, and staging requirements for a short access trace. Count incoming reserved slots and outgoing slots still holding live data against the budget. A full pool may require completing an eviction before admitting a new page. Calculate both logical useful bytes and whole-page transfer amplification for the chosen access pattern.

Part 4 - Experimental Design and Instrumentation

Begin with synchronous migration at deliberate completion boundaries. Test clean eviction, dirty eviction, repeated page-in, a partial final page, and an object destroyed during a deferred transition under a defined cancellation policy. Compare all values with a CPU oracle. Add asynchronous transfers only after defining events, source lifetime, destination readiness, and slot reuse. Bound both host backing and staging. Measure cold population separately from steady migration, including writeback and synchronization cost.

Specify the authority rule for clean CPU-backed pages and dirty GPU pages. An old CPU copy cannot satisfy a fault for a newer version. If CPU backing is pageable, use a synchronous primary path or include a bounded pinned staging step with its own lifetime. Choose whether valid tail elements or full page capacity are transferred, and make the byte ledger follow that choice.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class ResidencyRecord:
    backing_version: int
    device_version: object
    state: str
    slot_handle: object
    transfer_token: object

class ExplicitPager:
```

```
    def page_in(self, page_id, consumer_stream):
        # Admit a slot, copy authoritative values, and publish readiness.
        ...

    def page_out(self, page_id):
        # Preserve dirty data and defer slot reuse until all work completes.
        ...


def E14(config):
    # Validate migration, versions, and every budget transition.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export maximum simultaneous slot ownership, CPU backing bytes, pinned staging scope, transfer bytes by direction, and completed latency. Report backend device peaks separately from the application slot budget. Validate a round trip after mutation and demonstrate that a clean backed page can avoid unnecessary writeback. A slot returned to the pool may remain allocated or reserved by PyTorch; this does not invalidate a correctly scoped payload-budget claim.

Test a dirty page with multiple logical mappings under the chosen sharing policy, then verify the authoritative version after eviction and reload. Check both source and destination lifetime during asynchronous copies. A completed host method is insufficient evidence of completed device work; require the correct event or deliberate synchronization before reclaiming staging or payload slots.

Part 7 - Engineering Interpretation and Philosophy

Explain which memory pressure was relieved and which costs moved to CPU RAM, transfer bandwidth, or synchronization. State why explicit paging is a workload management technique rather than transparent CUDA Unified Memory or a hardware page-fault handler.

19.35.16 Exercise 15

Exercise Title: Trigger a Simulated Page Fault When a Required Logical Page Is Not Resident in VRAM and Restore It from CPU Memory

Part 1 - Exercise Contract and Why It Matters

Trigger an application-level simulated page fault when an explicit read or write requests a nonresident logical page. Restore the authoritative values from CPU backing, then resume the supported operation. Deliver fault records, correct retry behavior, and a clear distinction between software miss handling and hardware or operating-system page faults.

Part 2 - System and Memory Mental Model

The access API checks residency before dispatching payload work. A miss enters a controlled state machine rather than dereferencing an invalid CUDA pointer. Possible states include absent, loading, resident, evicting, and failed. A second request for a loading page must wait for the same operation or follow another explicit deduplication policy. Publishing an incomplete destination as resident can expose stale values even when its slot has already been allocated.

Part 3 - Quantitative Prediction Before Execution

For a trace of logical page requests, predict first-touch misses, later hits, repeated misses after eviction, and bytes transferred. Count fault requests separately from unique page-in operations. A range spanning two absent pages may generate two restorations. Derive the latency components of a blocking fault and identify which can overlap only after the later scheduler is added. Include a pool-full miss with no currently evictable victim.

Part 4 - Experimental Design and Instrumentation

Use a tiny bounded GPU slot count and a larger CPU-backed object. Force selected pages nonresident through the manager, then issue boundary reads and writes. Validate fault identity, restored version, resumed output, and unchanged neighbors. Simulate a page-in preparation failure before publication and require a coherent recoverable state. Never provoke an illegal device memory access to represent a software miss. Keep the primary host dispatcher serialized and define a bounded wait or admission failure when no victim can be reclaimed.

Issue two simulated requests for the same absent version before completing the first load in the transition simulator. Expect one page-in reservation and two dependent requests. Then cancel one requester and verify that the remaining requester can still complete. Define how the fault record refers to an object generation so an old request cannot resume against a new object with a recycled numeric identifier.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class FaultRecord:
    request_id: int
    page_id: int
    prior_state: str
    required_version: int
    outcome: str

class FaultDispatcher:
    def ensure_resident(self, page_id, access_mode):
        # Deduplicate transitions and return readiness or an explicit failure.
        ...

    def resume(self, logical_operation, readiness_tokens):
        # Execute only after all required pages are valid and available.
        ...


def E15(config):
    # Predict misses, force software faults, and validate resumed values.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report request misses, unique restorations, wait time, transfer bytes, retries, and failure phase without conflating them with OS fault counters. Check that a failed restoration does not install a valid mapping or leak a slot. Verify repeated requests do not allocate duplicate destinations for one version. Preserve unsupported concurrency cases as explicit limitations instead of assuming a Python dictionary alone implements a safe state machine.

Classify waiting for capacity separately from transferring data and waiting for an already scheduled restoration. These causes have different remedies. A timeout should report its blocked dependencies and preserve coherent state, not install a fabricated zero page. Test explicit failure reporting when the CPU backing version required by the request is unavailable.

Part 7 - Engineering Interpretation and Philosophy

Explain the benefit and overhead of moving residency checks into a controlled API. Identify which future operations can be resumed safely and why arbitrary PyTorch kernels cannot be intercepted by this software fault mechanism without additional integration.

19.35.17 Exercise 16

Exercise Title: Combine Page Prefetching and Page Eviction to Maintain a Dynamic GPU Working Set Without Exceeding the Physical Page Budget

Part 1 - Exercise Contract and Why It Matters

Add bounded prefetch and eviction scheduling while maintaining the declared GPU payload slot budget at every transition. Compare demand-only paging with lookahead policies using the same correct logical workload. Account for useful prefetch, wasted transfers, dirty writeback, waits, and scheduling overhead.

Part 2 - System and Memory Mental Model

Prefetch reserves capacity before use and may evict something needed sooner. An eviction candidate must satisfy ownership and completion rules, not simply have an old timestamp. Protect active leases and incomplete transfers. Maintain an explicit residency state machine and versioned table publication. Logical access order is a policy input; a host timestamp or kernel submission order alone does not establish when a device reader has finished with its payload.

Part 3 - Quantitative Prediction Before Execution

For a small slot budget, hand-simulate demand-only and one-page-lookahead execution on a sequential trace and an alternating working-set trace. Count misses, useful prefetched pages, evictions, writebacks, and peak reserved slots. Show a case where prefetch harms demand admission. Define a useful-prefetch ratio with an explicit lifetime window and leave an empty denominator undefined. Include the transition where an outgoing dirty page still occupies its slot.

Part 4 - Experimental Design and Instrumentation

Implement a conservative eviction baseline and bounded lookahead depth. Keep payload, staging, and outstanding-transfer limits separate. Start with synchronous scheduling, then add a transfer stream and event-based readiness. Require consumers to wait for the correct page generation and writers to respect migration state. Use the same finite trace for correctness and

separate repeated runs for timing. Compare unprofiled complete latency with a short diagnostic timeline; include prefetch setup and drain costs in end-to-end results.

Give demand and speculative work separate admission paths. A prefetched page that has not been consumed may be evicted under a declared policy, but an in-flight destination cannot be overwritten to satisfy demand. For dirty victims, reserve host backing or staging before starting writeback. Bound lookahead metadata and pending requests as well as payload slots so the controller cannot shift unbounded growth into a host queue.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class PagerBudgets:
    gpu_slots: int
    cpu_backing_bytes: int
    pinned_staging_bytes: int
    max_transfers: int

class WorkingSetScheduler:
    def plan(self, demand, lookahead, states):
        # Admit demand before speculative work within transition limits.
        ...

    def evictable(self, page_id, completion_evidence):
        # Exclude active, stale, dirty-unbacked, or in-flight dependencies.
        ...

    def advance(self, completed_tokens):
        # Publish finished transitions and reclaim only safe slots.
        ...


def E16(config):
    # Compare policies on sequential and adversarial working sets.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require no interval where slot ownership exceeds the bound, including incoming reservations and outgoing live pages. Log a scalar transition ledger and independently audit it. Report wall time, demand stall intervals, useful work, transfer volume, writebacks, and wasted prefetch. Concurrent intervals must not be added as though they were serial. A low fault count can coexist with poor throughput if speculative transfers occupy bandwidth or block demand.

Compare a perfectly predicted sequential trace with a deliberately misleading lookahead trace. Record how often speculation delays demand and whether the fallback policy restores progress. Include a working set one page larger than capacity to expose thrashing. Check that

reported overlap actually shortens completed-work time rather than merely placing transfers on another stream.

Part 7 - Engineering Interpretation and Philosophy

Choose the smallest tested prefetch policy that improves useful work without weakening resource or correctness contracts. Explain the difference between hiding latency, reducing transfer volume, and reducing capacity demand; one policy need not improve all three.

19.35.18 Exercise 17

Exercise Title: Build a Multi-Sequence Page Allocator That Serves Many Dynamically Growing Tensor Objects from One Shared CUDA Page Pool

Part 1 - Exercise Contract and Why It Matters

Serve many growing and shrinking sequence objects from one page pool. Add per-object tables, generation-checked identities, lifecycle operations, and fair admission under shared capacity. Support selected shared prefixes and private suffixes only through the ownership and copy-on-write contracts already validated, without assuming a specialized attention kernel.

Part 2 - System and Memory Mental Model

Sequence identity, logical length, page mappings, shared groups, residency, and active leases belong to separate records. A destroyed object's identifier may be reused only with a new generation or another stale-reference defense. One sequence's partial tail remains its own capacity unless explicit sharing permits otherwise. Admission must consider transition headroom as well as steady pages, especially when a shared prefix is about to be modified.

Part 3 - Quantitative Prediction Before Execution

Predict peak unique backing for several sequences with a common full-page prefix and different private suffix lengths. Compare that with the sum of logical lengths and the fully duplicated baseline. Calculate required pages for simultaneous growth and identify an admission order that can starve a small request. Define the chosen fairness or priority rule and state whether it optimizes completed sequences, latency, or another useful-work objective.

Part 4 - Experimental Design and Instrumentation

Replay a bounded multi-sequence trace containing creation, append, read, mutation, shrink, share, and destruction. Preserve stable sequence IDs and expected values in a CPU reference. Compare equal-work isolated pools with the shared pool under equal aggregate payload capacity. Include simultaneous pending growth, cancellation before publication, a shared-tail restriction, and object-ID reuse. Separate allocation service from any model computation; if adding a model consumer, use only the manager's explicitly supported access interface and account for materialized dense inputs.

Use sequences with identical shared prefixes but different final partial pages, and initially permit prefix sharing only at complete-page boundaries. This keeps tail ownership explicit. Later expand sharing only with a reference model for valid lengths and mutation behavior. Label request priorities and record arrival order so a fairness claim can be checked against events rather than inferred from aggregate throughput.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass(frozen=True)
class SequenceHandle:
    sequence_id: int
    generation: int

class MultiSequencePager:
    def create(self, length, shared_prefix=None):
        # Validate sharing boundaries and atomically admit initial mappings.
        ...

    def append(self, sequence, values):
        # Respect fairness, tail ownership, COW, and transition capacity.
        ...

    def destroy(self, sequence):
        # Retire the identity and safely release mappings and pending work.
        ...


def E17(config):
    # Audit the shared pool through mixed sequence lifecycle traces.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report accepted and rejected requests, wait times by sequence, unique payload, mapping count, tail slack, shared capacity saved, and transition peaks. Validate all surviving sequences after every event and ensure a cancelled operation cannot publish into a newly reused identity. A high aggregate throughput can hide starvation; report the worst waiting sequence and the tested fairness contract. Do not claim model-level acceleration from allocator-only timing.

Cancel a waiting sequence, reuse its numeric identifier with a new generation, and then complete the old sequence's outstanding transfer. The old completion must not modify the new sequence. Audit unique backing and reference counts through this interleaving. Check that a large sequence cannot consume all transition reservations indefinitely if the declared policy promises bounded service to smaller requests.

Part 7 - Engineering Interpretation and Philosophy

Explain where shared capacity improves admission and where contention or metadata erodes the benefit. Distinguish a general software sequence allocator from a complete paged attention implementation, and identify the required consumer integration before applying the design to a real model.

19.35.19 Exercise 18

Exercise Title: Stress-Test the Pager with Random Growth, Shrinkage, Sharing, Page Faults, Evictions, and Severe VRAM Pressure

Part 1 - Exercise Contract and Why It Matters

Stress-test the pager with reproducible randomized lifecycle and residency traces, including severe pressure within a small declared slot budget. Find minimal counterexamples to ownership, value, transition, and budget invariants. Separate deterministic fault injection from actual resource exhaustion and keep all real-device allocations bounded.

Part 2 - System and Memory Mental Model

The difficult failures combine individually valid operations: a shared page is being copied while one object shrinks, a dirty victim has pending readers, or a cancelled fault completes after object-ID reuse. A state machine must preserve version and generation identity across these transitions. Resource pressure can mean no admissible slot rather than a CUDA allocation failure. Do not force device-wide exhaustion when a small fixed application budget can exercise the same admission logic safely.

Part 3 - Quantitative Prediction Before Execution

Before execution, define invariants for every state and predict one valid and one invalid short trace. Calculate maximum transition capacity for growth, copy-on-write, page-in, and dirty eviction combinations. Define bounded waiting and progress expectations under a trace that eventually completes outstanding leases. A permanently pinned working set may make admission impossible; the correct result can be an explicit rejection rather than guaranteed progress.

Part 4 - Experimental Design and Instrumentation

Generate traces from a fixed seed with explicit operation probabilities, maximum lengths, page count, host backing limit, and transfer limit. Compare every stable state with a dense CPU oracle and recomputed ownership ledger. Add deterministic scheduler interleavings or completion-token simulation before running actual asynchronous streams. Inject preparation failures at named boundaries. Reduce any failing trace by replaying smaller subsequences while preserving dependencies, then save the smallest reproducible counterexample. Run additional seeds only after the primary trace is auditable.

Separate invalid-operation tests from valid randomized histories. An intentional stale handle should produce the expected rejection without being counted as a pager correctness failure. Save the exact scheduler decisions and completion order for asynchronous reproductions. A random seed alone may not reproduce real timing, so deterministic interleaving simulation is the primary tool for shrinking state-machine failures.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class StressLimits:
    max_objects: int
    max_length: int
    gpu_slots: int
    cpu_bytes: int
```

```
    steps: int


def generate_trace(seed, limits):
    # Produce bounded valid operations and labeled intentional violations.
    ...


def audit_state(pager, oracle, pending_operations):
    # Check values, counts, generations, versions, and all resource states.
    ...


def minimize_failure(trace, replay):
    # Preserve dependencies while finding a smaller reproducible failure.
    ...


def E18(config):
    # Test deterministic interleavings, then supported CUDA executions.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report seeds, trace IDs, exact failure phase, last valid state, and whether reproduction succeeds. Include stale-handle rejection, double release, leaked provisional pages, invalid tail access, failed copy-on-write, duplicate page-in, and forbidden eviction controls. Distinguish a simulator pass from a CUDA execution pass. Export bounded CPU-only records and never invent hardware results when the execution environment lacks the required device or libraries.

When minimizing a trace, retain object creation, backing availability, and required sharing dependencies. A shortened trace that fails because its setup was removed is a different failure. Report the original and reduced histories with their invariant violations. Track pending work at shutdown so a test cannot appear to pass simply by exiting before delayed corruption becomes observable.

Part 7 - Engineering Interpretation and Philosophy

Explain which invariants survived the tested envelope and which remain unsupported. A randomized run without a failure is evidence about that run, not a proof of correctness under every interleaving. Recommend the next smallest experiment that addresses the most consequential unresolved risk.

19.35.20 Exercise 19

Exercise Title: Build and Benchmark a PyTorch Virtual Tensor Memory Manager with Logical Addressing, Page Tables, Physical Page Pools, Sharing, Prefetching, and Eviction

Part 1 - Exercise Contract and Why It Matters

Integrate the restricted pager into one benchmarkable virtual tensor memory manager with logical addressing, checked tables, bounded pools, sharing, copy-on-write, CPU backing, explicit faults, prefetch, and eviction. Preserve the distinction between its software interface and

hardware virtual memory. Deliver a support matrix, reproducible traces, correctness evidence, and a workload-specific engineering recommendation.

Part 2 - System and Memory Mental Model

The manager coordinates object identity, mapping ownership, versions, residency, transfer completion, and admission. Use one source of authority for each record and explicit commit boundaries between prepared and published state. Ordinary PyTorch operators do not gain transparent paged access simply because a Python object exposes a familiar method name. The primary version supports a narrow no-autograd flat-value interface; shape metadata can be preserved without promising arbitrary views, gradients, layouts, or kernels.

Part 3 - Quantitative Prediction Before Execution

Predict complete payload residency, CPU backing, pinned staging, table bytes, tail slack, transition headroom, output materialization, and useful-work time for each benchmark trace. Compare dense materialization, a resident-only paged baseline, demand paging, and prefetch with equal useful values and supported semantics. Identify when a model larger than the GPU slot budget can be served and when its active working set or consumer outputs still make admission impossible. No paging policy creates additional bandwidth or physical capacity.

Part 4 - Experimental Design and Instrumentation

Freeze the support matrix and validate each subsystem before integration. Run short deterministic traces, randomized stress, and held-out sequential, sparse, shared-prefix, write-heavy, and thrashing workloads. Keep the same requests, outputs, and completion boundaries across baselines. Measure cold construction, warmed access, transition costs, and final drain separately. Ablate sharing, copy-on-write, prefetch, and eviction choices to identify mechanisms. Record device and software versions, dtype, page units, budgets, seeds, trace IDs, allocator backend, and initial-state identifiers.

Publish a supported-operation matrix before performance results: creation, range reads, indexed access, mutation, sharing, resizing, migration, and cancellation each need precise semantics. Mark unsupported layouts, external aliases, autograd, graph capture, and concurrent writers explicitly. Preserve one immutable benchmark trace per comparison and use scalar evidence IDs to link outputs, transitions, and resource observations without retaining their payloads.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class VirtualTensorContract:
    device: str
    dtype_name: str
    page_elements: int
    gpu_slots: int
    cpu_backing_bytes: int
    staging_bytes: int
```

```
    max_transfers: int
    supported_operations: tuple

class VirtualTensorMemoryManager:
    def __init__(self, contract):
        # Initialize authority, ownership, state machines, and bounded pools.
        ...

    def access(self, object_handle, request):
        # Validate, restore, translate, and dispatch supported operations.
        ...

    def mutate(self, object_handle, request, values):
        # Respect COW, versions, leases, and coherent publication.
        ...

    def schedule(self, demand, lookahead):
        # Enforce demand priority and every transition resource bound.
        ...

    def audit(self):
        # Recompute mappings, ownership, residency, versions, and budgets.
        ...

    def shutdown(self):
        # Finish or safely cancel owned work and report unreclaimed state.
        ...


def E19(config):
    # Validate the integrated contract before complete-work benchmarking.
    ...
    # Export evidence, counterexamples, limitations, and recommendation.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Acceptance requires exact supported values, coherent shared snapshots, checked stale identities, no unsafe reuse, truthful budget accounting, and bounded failure handling. Report useful completed operations, latency spread, slot peaks, host backing, backend allocated and reserved peaks, transfer bytes, faults, evictions, copies, metadata scope, and failure phases. Keep predictions, observations, and interpretations separate. A benchmark with incorrect results or omitted materialization cost cannot qualify as a performance improvement.

Include a workload where dense storage wins and another where the active set is too large for any permitted policy. Report rejection or poor performance honestly. Validate shutdown after normal completion and controlled cancellation, including old mappings and pending transfers. Separate measured delivered features from proposed kernel or VMM extensions so the final engineering recommendation cannot imply capabilities the scaffold leaves to the learner.

Part 7 - Engineering Interpretation and Philosophy

Recommend the smallest manager justified by the evidence, including a dense baseline when it wins. Explain which benefits came from software placement, sharing, reduced copying, or latency overlap. State what remains necessary for production integration, including supported kernels and stronger concurrency or autograd contracts, without presenting unimplemented extensions as delivered capabilities.

19.36 DMM34

Chapter Title: Paged KV Cache for LLM Inference
Specialization: Dynamic Memory Management and CUDA Systems

19.36.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Transformer KV Cache Growth, Token Residency, Block Allocation, and Why Long-Context Inference Becomes Memory-Bound

Part 1 - Exercise Contract and Why It Matters

Define the lifecycle of attention keys and values during prefill and incremental decoding. Deliver a diagram linking request identity, processed token positions, per-layer KV values, block ownership, and attention reads. Explain capacity pressure and bandwidth pressure separately. This chapter builds a restricted inference cache in PyTorch; its reference attention path may gather blocks into dense tensors and must disclose that materialization cost.

Part 2 - System and Memory Mental Model

For causal self-attention, previously processed tokens supply reusable keys and values at every cached layer. Queries are newly computed for the current positions; caching K and V does not remove attention over their history. A sampled next-token ID normally has no cache entry until its forward execution processes that token. Logical token order is independent of pool slot order. The term physical KV block means application-owned CUDA payload, not a chosen hardware frame or an implicit CUDA virtual-memory mapping.

Part 3 - Quantitative Prediction Before Execution

For homogeneous full-attention layers, predict useful KV payload from layers, KV heads, head dimension, dtype width, and processed token counts. Distinguish query heads from KV heads under grouped-query or multi-query attention. Estimate how much history each decode step reads and why a workload may become bandwidth limited without proving that every long-context run is memory-bound. Separate cache payload from weights, attention outputs, workspaces, metadata, and allocator-reserved memory.

Part 4 - Experimental Design and Instrumentation

Start with one device, deterministic evaluation, no gradients, fixed model weights, unquantized equal K/V dimensions, and full causal attention. Use a tiny explicit decoder as a transparent reference rather than a high-level serving wrapper. State tensor layouts, token-position convention, positional encoding, masks, batch membership, and completion boundaries. Record model configuration, software versions, CUDA device, allocator backend, seed, and initial state. Exclude sliding-window, cross-attention, latent-attention, offloading, and quantization unless separately implemented and validated.

Draw a prefill of several tokens followed by two decode iterations, marking when each token receives its layer-specific cache entries. Include the final sampled token that is never processed because the request stops. Use separate colors or labels for persistent history and one-step temporary tensors. The same diagram must distinguish token positions from pool offsets so a later block permutation cannot change the meaning of attention.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class KVContract:
    layers: int
    query_heads: int
    kv_heads: int
    head_dim: int
    dtype_name: str
    block_tokens: int
    position_policy: str


def predict_kv_lifecycle(contract, request_trace):
    # Separate processed tokens, sampled tokens, capacity, and read traffic.
    ...


def E00(config):
    # Define the supported decoder and audit its token/cache timeline.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require every cached vector to have a request, layer, head, and processed position identity. Keep prediction, observation, and interpretation in separate fields. A lower cache capacity does not prove reduced attention work, and allocator-only timing does not prove model throughput. Export bounded CPU-only evidence without retaining cache tensors in logs. Unsupported GPU measurements remain unmeasured rather than being replaced by synthetic hardware results.

Check the mental model against a small trace before introducing performance claims. A capacity-limited request and a bandwidth-limited decode can have similar latency symptoms but require different evidence. Preserve the distinction between useful KV reads, cache allocation, and model weight traffic. Do not infer the limiting resource solely from context length or an external utilization percentage.

Part 7 - Engineering Interpretation and Philosophy

Explain which benefits require only better allocation and which require an attention implementation that consumes paged storage directly. Paging can improve capacity utilization

without eliminating context reads or guaranteeing lower latency. Use that distinction throughout the chapter's engineering comparisons.

19.36.2 Exercise 01

Exercise Title: Calculate the Exact KV Cache Memory per Token from Layer Count, KV Head Count, Head Dimension, Dtype, and Batch Size

Part 1 - Exercise Contract and Why It Matters

Derive the exact unquantized KV tensor payload for the declared architecture and token population. Produce a calculator that distinguishes one token in one sequence from one decode step across a batch. Validate formulas against small tensor storage allocations and state every excluded source of memory.

Part 2 - System and Memory Mental Model

Ordinary full-attention caching stores a key and a value for each processed position at each layer. Query head count does not determine stored head count when KV heads are shared. Batch size multiplies only when each sequence has the same cached length. Unequal lengths require their sum. Quantization scales, block padding, packed layouts, backend rounding, model weights, and temporary head expansion are outside the simple useful-payload formula.

Part 3 - Quantitative Prediction Before Execution

For L identical layers, H_{kv} KV heads, head dimension D, and s bytes per stored element, derive

$$b_{\text{token}} = 2LH_{kv}Ds, \qquad B_{\text{useful}} = b_{\text{token}} \sum_{j=1}^{B} T_j.$$

For heterogeneous layers, sum their separate K and V widths and element sizes. Calculate an example with 24 layers, eight KV heads, dimension 128, and two-byte elements, then compare query-head counts 8 and 32. Predict bytes added by one processed token per active sequence and distinguish bytes, KiB, and MiB.

Part 4 - Experimental Design and Instrumentation

Implement validation for positive architecture dimensions, supported dtypes, nonnegative lengths, and arithmetic bounds. Use tiny CPU tensors for broad formula checks and bounded CUDA tensors when available. Compare requested payload with actual unique backing for independent K and V arrays; do not sum alias views as independent storage. Test equal lengths, ragged lengths, empty requests, GQA, MQA, and an intentionally incompatible architecture that the simple calculator must reject or route to a generalized formula.

Use independent K and V element-size fields in the generalized calculator, even when the primary model stores them identically. A layer with different value width must not be forced into the homogeneous expression. Include a batch with one empty history and another with a long history. State whether one-token cost refers to adding one position globally or one position to each active request.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class LayerKVSpec:
    kv_heads: int
    key_dim: int
    value_dim: int
    key_element_bytes: int
    value_element_bytes: int


def useful_kv_bytes(layer_specs, processed_lengths):
    # Validate the architecture and calculate exact logical payload.
    ...


def compare_with_storage(prediction, key_tensors, value_tensors):
    # Count unique backing separately from views and backend statistics.
    ...


def E01(config):
    # Validate hand calculations and report excluded capacity components.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report raw layer specifications, lengths, useful elements, requested bytes, unique storage scope, and backend allocated/reserved observations separately. Demonstrate a counterexample to using query heads in the formula and another to multiplying maximum length by batch size for a ragged useful-payload count. An exact tensor formula is not an exact prediction of total process VRAM. Explicitly label padding and alignment when the measured representation adds them.

For a storage check, count each unique backing allocation once and record logical view payload separately. Compare the calculator with a deliberately padded block representation to show why exact useful bytes can be smaller than allocated payload. Keep model-family exclusions explicit: latent or hybrid cache structures require their own architectural accounting rather than a guessed KV-head multiplier.

Part 7 - Engineering Interpretation and Philosophy

Explain how an auditable payload calculation becomes an admission estimate only after adding retained capacity and transient headroom. State which model families invalidate the homogeneous formula and why counting architectural objects must precede optimization.

19.36.3 Exercise 02

Exercise Title: Implement a Conventional Contiguous KV Cache in PyTorch and Measure VRAM Growth During Autoregressive Decoding

Part 1 - Exercise Contract and Why It Matters

Build a conventional contiguous cache and measure its growth during a bounded autoregressive trace. Compare a fixed-capacity in-place baseline with a separately labeled concatenation-based growing baseline. Preserve identical processed positions and numerical outputs while exposing their different capacity and copying behavior.

Part 2 - System and Memory Mental Model

A static cache allocates capacity before valid tokens fill it; allocated payload may stay flat while logical occupancy grows. A concatenation cache creates larger backing and can temporarily retain old and new arrays, copying history repeatedly. Both can be contiguous layouts, so contiguous does not mean one unique growth policy. Cache length is a validity boundary, not the allocated sequence dimension. Record whether stored keys are already positionally transformed and never apply that transformation twice.

Part 3 - Quantitative Prediction Before Execution

Predict static capacity bytes from maximum sequence lengths and useful bytes from current processed lengths. For repeated concatenation, derive the sum of historical elements copied over a chosen decode trace and estimate temporary coexistence. Identify the first step at which a capacity limit is reached. Predict separate curves for useful payload, allocated cache capacity, and backend reserved memory; they need not have the same slope.

Part 4 - Experimental Design and Instrumentation

Use a tiny fixed-weight decoder in evaluation with attention dropout disabled. Compare cached decoding with full-prefix recomputation on the same teacher-forced token trace. Reset cache state between trials. Measure constructor, prefill, warmed decode, and final drain separately. Synchronize at deliberate region boundaries and use device events where suitable. Do not retain every step's logits or KV arrays in Python lists. Treat real generation as a later additional check because token divergence can change the work being compared.

Use a fixed maximum context and a trace that reaches the limit exactly, then attempts one additional append. The failure must leave previously valid cache contents unchanged. Check per-layer valid-write bookkeeping so a failed forward cannot publish a token whose K exists in some layers but whose V or later-layer data is missing. A scalar global length alone cannot describe that preparation state safely.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

class ContiguousKVCache:
    def __init__(self, contract, capacities, growth_policy):
        # Define static or concatenating backing and per-request validity.
        ...

    def append_layer(self, request, layer, positions, keys, values):
        # Validate shape, position, capacity, and layer update ownership.
        ...
```

```
    def commit_tokens(self, request, completed_layers):
        # Publish valid lengths only when the required KV writes are ready.
        ...


def E02(config):
    # Compare full recomputation, static cache, and growing cache correctly.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require logits and attention outputs within declared dtype-appropriate tolerances before interpreting memory or speed. Check that unwritten capacity is masked out. Record useful tokens, capacity, copy bytes, peak allocated and reserved memory, elapsed time, and failure phase. A flat static-cache allocation curve is expected behavior, not evidence that additional tokens require no KV storage. Unsupported device execution remains explicitly untested.

Compare cached execution with full-prefix recomputation at several positions, not only the final token. A stale prefix can accidentally produce plausible logits while diverging earlier. Report host submission time separately from completed decode time, and record whether model construction, tokenization, transfer, warm-up, and output validation are inside the timed region.

Part 7 - Engineering Interpretation and Philosophy

Explain which cost paging will target in each baseline: reserved unused capacity, repeated growth copies, or placement inflexibility. Keep the baseline policy visible so a later improvement is not attributed to paging when it actually comes from avoiding a particularly inefficient concatenation strategy.

19.36.4 Exercise 03

Exercise Title: Generate Multiple Variable-Length Sequences and Quantify Wasted VRAM Caused by Preallocated Contiguous KV Cache Capacity

Part 1 - Exercise Contract and Why It Matters

Quantify unused KV capacity for variable-length requests under conventional preallocation. Compare per-request capacity and a padded batch layout while holding actual processed token histories fixed. Deliver occupancy timelines and a decomposition of unused capacity rather than one unexplained waste percentage.

Part 2 - System and Memory Mental Model

A request can reserve future decode capacity beyond its current prefix. A padded batch can additionally allocate positions up to the longest capacity for every row. These mechanisms overlap unless the accounting decomposition is carefully defined. Finished requests can leave idle rows in a static batch. None of these unused positions are useful history. The GPU allocator's cached free blocks are a separate layer and must not be added again as cache slack.

Part 3 - Quantitative Prediction Before Execution

For request capacities C_j and current lengths T_j, predict per-request unused payload from the differences. For a batch allocated to a common maximum, separate padding beyond individual capacities from unused positions inside those capacities. Define occupancy with an explicit denominator and leave an empty allocation undefined. Construct two length traces with the same total useful tokens but different arrival, completion, and reservation patterns.

Part 4 - Experimental Design and Instrumentation

Replay bounded short, medium, and long requests with explicit prefill and decode lengths. Compare static per-request arrays with padded batch backing using the same model, dtype, and useful token trace. Preserve masks, row-to-request identity, and completion events. Measure useful payload and capacity after each step, then report temporal peak and average occupancy separately. Add a control whose final lengths fill their reservations to expose how workload knowledge affects the baseline. Avoid device exhaustion by enforcing a small byte budget.

Include a trace where requests reserve generous future output lengths but finish early, and another where they nearly fill capacity. Keep prompt tokens and generated-token accounting separate so the same useful workload is compared. For a padded batch, record row reassignment and active masks explicitly. Do not count a finished row as active merely because its backing tensor still exists.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class RequestCapacity:
    request_id: int
    processed_tokens: int
    reserved_tokens: int
    finished: bool


def capacity_decomposition(requests, layout_policy, bytes_per_token):
    # Separate useful history, future reservation, and batch padding.
    ...


def replay_contiguous_requests(trace, cache_contract):
    # Preserve request rows and masks across arrivals and completions.
    ...


def E03(config):
    # Predict and measure occupancy for identical useful histories.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report lengths, reservations, active rows, useful bytes, capacity bytes, allocator observations, and elapsed useful work. Demonstrate why averaging per-request occupancy percentages can disagree with a byte-weighted aggregate. Validate no padding enters attention and no completed row is accidentally reused before pending work ends. A high unused-capacity fraction does not establish slow execution; capacity and throughput require separate evidence.

Recompute useful and unused capacity from scalar request histories independent of cache counters. Compare peak unused bytes with time-averaged unused bytes; the two may rank policies differently. Validate that a static-capacity baseline cannot improve apparent throughput by ignoring long requests, dropping padding checks, or changing output limits relative to the paged workload.

Part 7 - Engineering Interpretation and Philosophy

Explain how reservation policy and batching contribute to apparent waste. Choose a realistic contiguous reference for later comparisons and identify the workload information it legitimately knows, avoiding a comparison against an unnecessarily pessimistic maximum-length allocation.

19.36.5 Exercise 04

Exercise Title: Split Key and Value Storage into Fixed-Token CUDA Blocks and Build a Shared Physical KV Block Pool

Part 1 - Exercise Contract and Why It Matters

Create a shared pool of fixed-token KV block bundles for the supported model. Each bundle holds K and V storage for the same logical token interval across all cached layers. Deliver bounded acquisition, release, initialization, and audit operations with generation-checked handles.

Part 2 - System and Memory Mental Model

Use a documented layout such as $[L, H_{kv}, P, D]$ for each of the K and V arrays, where P is token capacity per block. A bundle may consist of two independent tensors; it is one application allocation unit, not necessarily one backend allocation or hardware page. Retained free blocks remain backed. A slab variant must disclose that slot views retain the slab. Pending CUDA users prevent recycling even when logical mappings have been removed.

Part 3 - Quantitative Prediction Before Execution

Derive bundle payload bytes as Pb_{token} under the homogeneous contract. For K pool slots, predict total backed payload, currently mapped slots, free retained slots, and pending slots. Calculate backend allocation calls for the chosen K/V representation rather than assuming one call per bundle. Include metadata, initialization temporaries, and consumer headroom outside the pool payload budget.

Part 4 - Experimental Design and Instrumentation

Start with one host dispatcher, one CUDA stream, no shared mappings, and a small maximum slot count. Initialize block contents with identifiable layer, head, token, and component values. Test acquire/release cycles, bounded pool exhaustion, double release, stale generations, and wrong-pool handles. Compare cold creation and warmed reuse with separate peak resets. If blocks are allocated lazily, reject requests beyond the declared maximum before attempting unbounded backend allocations. Record both useful and retained capacity.

Choose whether blocks are created lazily or all at pool startup and keep that choice fixed within a comparison. Distinguish a bundle acquisition from a backend tensor allocation, especially when K and V are separate arrays. Include page initialization cost only where the contract requires it, while ensuring invalid bytes are never exposed to attention. Test both zero-capacity admission and an exactly full bounded pool.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass(frozen=True)
class KVBlockHandle:
    slot_id: int
    generation: int

class KVBlockPool:
    def __init__(self, contract, maximum_blocks):
        # Establish bounded K/V bundles and free, mapped, pending states.
        ...

    def acquire(self):
        # Return a valid initialized or explicitly uninitialized bundle.
        ...

    def release(self, handle, completion_token):
        # Recycle only after mappings and device use obligations end.
        ...

    def audit(self):
        # Reconcile unique payload and every slot state.
        ...


def E04(config):
    # Validate layout, cold allocation, reuse, and invalid handles.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Check each K and V coordinate against the initialization pattern. Report pool-backed bytes, mapped capacity, free retained capacity, pending capacity, allocation calls, and backend peaks without double-counting shared storage. A returned Python method does not prove that device initialization completed. Evidence logs must store scalar handles and counters rather than references that keep blocks alive. Distinguish pool admission failure from measured CUDA allocation failure.

Audit that K and V bundles share a coherent handle generation even when their tensor pointers differ. A pointer may repeat after backend reuse and cannot identify an old valid request. When a slab variant is explored, count its unique backing once and explain why dropping individual slot views does not return their capacity to the driver.

Part 7 - Engineering Interpretation and Philosophy

Explain the capacity and allocator-traffic objectives of the pool. State why retaining free payload improves reuse while potentially keeping VRAM residency high, and identify the lifetime evidence required before a completed request's blocks become safe inputs for another request.

19.36.6 Exercise 05

Exercise Title: Build a Per-Sequence Block Table That Maps Logical Token Blocks to Noncontiguous Physical KV Cache Blocks

Part 1 - Exercise Contract and Why It Matters

Build a per-request block table mapping logical token blocks to arbitrary pool bundle handles. Preserve request identity, layer correspondence, valid lengths, and generations. Deliver checked lookup, installation, replacement, and destruction operations before using tables in an attention consumer.

Part 2 - System and Memory Mental Model

One logical block covers the same token interval across all layers in the bundle representation. Its table entry must not confuse K from one request with V from another. Committed token length and any layer-write readiness are distinct metadata. The primary table is host-owned Python state; a device kernel does not automatically see it. A future device table requires measured upload, versioning, and publication dependencies. Logical block adjacency does not imply pointer or hardware adjacency.

Part 3 - Quantitative Prediction Before Execution

For positions around $P - 1$, P, and $2P$, predict logical block numbers, within-block offsets, and the handles selected by a deliberately shuffled table. Calculate entry count for a ragged batch. Estimate packed table bytes using a declared entry format, and distinguish that estimate from Python object footprint. Predict stale-handle rejection after a slot is reused with a new generation and a different request owns the same numeric slot ID.

Part 4 - Experimental Design and Instrumentation

Assign distinct values to every layer, head, token, K/V component, and request. Install non-identity permutations in separate request tables. Validate all lookups against an independent CPU oracle. Test missing entries, invalid positions, foreign pool handles, request-ID reuse, and partially written layers. Define admission atomicity for replacements so a failed preparation cannot leave the table pointing to invalid payload. Serialize host updates initially and retain old table versions while supported readers still depend on them.

Give two requests identical logical lengths but different shuffled mappings, and verify that their tables never cross. Add a nonzero absolute position base without changing zero-based table indexing. For each mapping publication, record expected request generation, table version, and slot generation. This makes a stale cached translation detectable without relying on the payload's numeric pattern to reveal the error.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class RequestKVState:
    request_generation: int
    committed_length: int
    block_handles: list
    position_base: int

class KVBlockTables:
    def lookup(self, request, logical_block):
        # Validate request generation, bounds, slot generation, and readiness.
        ...

    def install(self, request, logical_block, handle):
        # Publish only a coherent mapping with established ownership.
        ...

    def retire(self, request):
        # Remove mappings without invalidating outstanding device readers.
        ...


def E05(config):
    # Validate shuffled mappings and reject stale request/block identities.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Recompute mapping ownership from all tables and compare it with pool state. Verify exact K/V pair identity independently from lookup success. Report table construction, lookup overhead, metadata scope, and backend payload separately. A coincidentally correct token value cannot establish correct request or layer mapping. Mark unsupported concurrent table access explicitly instead of relying on an unqualified assumption that a Python container makes publication safe.

Test a valid slot index from a different pool and reject it before updating ownership. Then replace a mapping successfully and verify that old readers retain the version they were authorized to consume. If an optional device table is used, include upload bytes and readiness events; a host dictionary update is not evidence that a running kernel sees coherent new entries.

Part 7 - Engineering Interpretation and Philosophy

Explain how table indirection enables independent request growth and block placement. Identify the extra readiness and lifetime guarantees needed before an optimized attention kernel can safely consume the same mapping structure.

19.36.7 Exercise 06

Exercise Title: Append New Tokens by Allocating Additional KV Blocks Without Reallocating or Copying Existing Cache Storage

Part 1 - Exercise Contract and Why It Matters

Append processed-token K and V vectors by filling an existing private tail or acquiring additional blocks. Preserve all earlier cache values and payload handles without copying their history. Commit a new valid length only after every required layer's writes are ready for supported attention consumers.

Part 2 - System and Memory Mental Model

A decode step produces one new K/V pair per layer for each processed token. Unused tail capacity may avoid a new block; crossing a boundary requires another bundle. Table growth can move metadata even when old payload does not move. New token IDs, positions, and layer writes must correspond to the same request generation. Shared-tail mutation is excluded until copy-on-write is implemented. A sampled output token is not committed merely because its ID exists in the scheduler.

Inside a forward operation, layer attention may consume that layer's prepared new KV through the operation plan after its writes are ordered correctly. This internal readiness is distinct from the globally committed length exposed to other operations. Otherwise, waiting for every layer before letting the first layer attend would create an impossible execution order. Do not publish partial per-layer preparation as a reusable complete-prefix cache entry.

Part 3 - Quantitative Prediction Before Execution

Starting with length $P - 1$, predict additional bundles for appends of one, two, and $P + 2$ processed tokens in separate trials. Calculate new K/V write bytes and distinguish them from historical copies and table movement. Predict provisional capacity for a multi-block append and the state preserved when that full request cannot be admitted. Explain the layer at which a partially completed forward must remain uncommitted.

Part 4 - Experimental Design and Instrumentation

Drive the cache with a deterministic teacher-forced trace and explicit per-layer K/V outputs. Check every preserved prefix and newly appended position against the contiguous reference. Test zero-length append, boundary crossing, invalid shapes, inconsistent positions, missing layer writes, and bounded pool exhaustion. Reject the complete invalid request before mutation when possible. Measure allocation, new-value writes, metadata work, and completed append latency separately. Do not describe the whole operation as free of copying when newly produced values still require storage writes.

Make newly appended keys position-sensitive and values token-sensitive so swapped positions or omitted layers cannot pass by coincidence. Preserve the positionally transformed key convention used by the decoder. For a multi-token prefill append, validate all target positions before any publication and state which partial execution states are internal only. Keep growth metadata and payload lifetime records separate.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

class KVAppender:
    def plan(self, request, token_positions):
        # Validate positions and reserve complete append capacity.
        ...

    def write_layer(self, plan, layer, keys, values):
        # Fill only the supported tail/new slots and record readiness.
        ...

    def commit(self, plan, completion_evidence):
        # Publish coherent token validity after all required layers.
        ...


def E06(config):
    # Check prefix identity, exact new values, and rejected append cleanup.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report committed lengths, provisional blocks, old payload copies, new K/V writes, table bytes moved, and latency. Verify that a rejected append leaks no slots and exposes no partly valid token. Unexpected asynchronous execution failures are a separate failure phase; admission atomicity does not imply transactional rollback of arbitrary device errors. Preserve a stable prefix handle ledger and value checks rather than using pointer equality alone.

Check several appends after a rejected growth to ensure provisional cleanup did not corrupt the free list. Stable old pointers are only one part of the no-history-copy claim; compare old values and count actual copied payload. If a new table representation is allocated, report that metadata movement rather than describing the entire growth operation as zero allocation.

Part 7 - Engineering Interpretation and Philosophy

Explain the precise no-history-copy guarantee and its measurement boundary. Identify when block growth improves a workload and when attention reads, projection computation, or table dispatch still dominate total decode time.

19.36.8 Exercise 07

Exercise Title: Implement Logical Token-to-Physical Block Translation and Retrieve the Correct Key and Value Vectors for Attention

Part 1 - Exercise Contract and Why It Matters

Implement token-to-block translation and retrieve the correct K and V vectors for selected layers, heads, and positions. Preserve requested order and repeated reads across nonconsecutive payload slots. Deliver a validated retrieval path whose materialization cost is visible before building paged attention.

Part 2 - System and Memory Mental Model

For a zero-based logical token index t, compute block index and token offset using the software token capacity. Absolute model positions may include a separate base and are not interchangeable with table indices. Retrieval must respect valid length and the chosen positional-key representation. A dense gather result usually allocates new storage. Grouping accesses by block can reduce dispatch, but the final result must return to logical request order.

Part 3 - Quantitative Prediction Before Execution

Translate a shuffled request list containing positions P, 0, $P-1$, and a repeated P. Predict touched logical blocks, unique payload bundles, read elements, and output bytes. Repeat for two layers and different requests to expose index-order mistakes. Estimate the difference between retrieving one token and reconstructing its entire block or full history. Account for host index conversion or synchronization if the implementation performs it.

Part 4 - Experimental Design and Instrumentation

Use deterministic coordinate-coded K and V arrays and random nonidentity block placement. Test single positions, boundary ranges, repeated indices, empty reads, final valid positions, invalid tails, and different request bases. Compare with contiguous reference retrieval before invoking any attention. Start with a no-alias copied-output contract so callers cannot bypass cache mutation rules. Benchmark scalar dispatch, grouped retrieval, and full-history materialization separately under the same returned-value contract.

Include a retrieval request that changes both layer and request after every few indices. This exposes implementations that cache the wrong bundle base. Check head order independently from token order, particularly when query heads outnumber KV heads. The retrieval API should return stored KV heads; any query-head expansion belongs to the attention consumer and its own temporary-memory ledger.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class TokenLocation:
    handle: object
    token_offset: int
    absolute_position: int


def translate_token(request_state, logical_token, pool):
    # Check identities, validity, and the logical/absolute position mapping.
    ...


def retrieve_kv(cache, request, layer, token_indices):
    # Gather matching K/V vectors and preserve the requested order.
    ...
```

```
def E07(config):
    # Validate translation independently from attention numerics.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require exact retrieval values for the supported test dtype and initialization. Record requested tokens, unique blocks, output bytes, temporary bytes, dispatch count, and completed time. Validate both K and V and every selected head; checking only final attention output can hide compensating errors. Demonstrate that reading padding or another request's slot is rejected. State which output storage owns returned values and when it can be released.

Use one invalid index late in a read request and define whether validation rejects the whole request before output allocation. Report this boundary. Compare a copied-output path with any leased-view extension under clearly different contracts. A returned internal view can keep a block live and must not silently permit mutation or become stale after recycling.

Part 7 - Engineering Interpretation and Philosophy

Explain when sparse or grouped retrieval is useful and when reconstructing history erases the capacity advantage of paging. Identify the evidence required before replacing this transparent reference with a device kernel that translates block tables directly.

19.36.9 Exercise 08

Exercise Title: Reconstruct Attention over a Paged KV Cache and Verify Numerical Equivalence with a Conventional Contiguous Cache

Part 1 - Exercise Contract and Why It Matters

Compute attention using paged KV and verify numerical equivalence against contiguous caching and full-prefix recomputation. Implement a transparent gather-then-attend reference first. A direct blockwise attention extension is optional and must preserve global normalization rather than independently normalizing each block and averaging its outputs.

Part 2 - System and Memory Mental Model

Attention depends on values, head mapping, scale, valid lengths, positions, and masks. For query at absolute position q, allow only valid keys with positions no greater than q under the full causal contract. A rectangular causal helper may align queries differently from incremental decoding; inspect the API rather than assuming a one-row query sees all past keys. Set attention dropout explicitly to zero for evaluation. GQA expansion or specialized backend support also has numerical and memory consequences.

Part 3 - Quantitative Prediction Before Execution

Write a hand-checkable one-head attention example spanning two blocks and a partial tail. Predict allowed key positions for a single decode query and a multi-token suffix. Compare global softmax with the incorrect average of block-local attention outputs. Estimate dense gather output, attention workspace, and any repeated-head temporary storage. State numerical tolerances by dtype and accumulation/backend policy before comparing results.

Part 4 - Experimental Design and Instrumentation

Use fixed Q/K/V values first, then the tiny decoder with cached positional keys and a fixed teacher-forced token trace. Test unequal request lengths, partial blocks, nonzero position bases, GQA, and multiple new queries. Construct an explicit position-aware mask and validate its boolean or additive semantics. Record attention backend and compare like-for-like paths where possible. Include gather, mask construction, and output storage in end-to-end timing and peak memory. Profile a short diagnostic run separately from throughput runs.

For a single decode query, explicitly list all valid historical keys that should participate, then compare that list with the generated mask. Repeat with a two-query suffix and different request lengths. Test boolean-mask polarity rather than copying a padding mask from another attention API. Where head expansion is necessary, record whether it allocates repeated K/V storage or uses a supported backend path.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
import torch.nn.functional as F


def allowed_keys(query_positions, key_positions, valid_lengths):
    # Construct and validate the declared full-causal visibility relation.
    ...


def paged_attention_reference(query, cache, request, layer, mask_policy):
    # Gather valid KV and apply correctly mapped, globally normalized attention.
    ...


def compare_attention_paths(cases, tolerances):
    # Check contiguous, paged, and recomputed outputs before performance.
    ...


def E08(config):
    # Test rectangular masks, GQA mapping, partial tails, and decoder logits.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report maximum and aggregate numerical errors, mask mismatches, backend, dtype, temporary peaks, and complete elapsed time. Use a negative mask control that hides legitimate history and require validation to catch it. Different floating-point reduction orders can prevent bitwise equality while satisfying an explicit tolerance. Greedy token agreement is useful additional evidence, but does not replace direct attention and logit checks.

Use a small high-precision mathematical attention reference to diagnose errors before testing lower precision. Validate normalization over the union of blocks, including a very large score in a later block. If implementing online blockwise softmax, compare its running maxima,

denominators, and weighted sums with the dense result. Exclude fully masked queries or define and test their behavior explicitly.

Part 7 - Engineering Interpretation and Philosophy

Separate representation correctness from attention-kernel efficiency. Explain why a gather-based prototype can validate paged storage yet perform poorly, and what global-softmax, masking, and lifetime guarantees a direct paged kernel must retain.

19.36.10 Exercise 09

Exercise Title: Sweep KV Block Size and Measure Internal Fragmentation, Block-Table Overhead, Allocation Frequency, and Decode Latency

Part 1 - Exercise Contract and Why It Matters

Sweep token capacity per KV block and quantify internal slack, table size, allocation frequency, translation work, and completed decode latency. Choose a tested compromise under explicit payload and total-headroom constraints. Keep the same request trace and attention implementation across candidates.

Part 2 - System and Memory Mental Model

Smaller token blocks can reduce final-block slack and provide finer sharing and recycling granularity. They also increase table entries and dispatch or kernel metadata work. Larger blocks may reduce per-block overhead but retain more unused tail capacity. The software token capacity is not a CUDA hardware page size. Under equal payload bytes, changing capacity changes the number of available bundles and must change the admission calculation accordingly.

Part 3 - Quantitative Prediction Before Execution

For each candidate P, calculate assigned capacity from current request lengths, useful payload, tail slack, and mapping count. Predict new-block acquisitions over a decode trace, distinguishing pool reuse from backend allocation. Include a prefix-sharing trace whose matching prefix ends inside a block. Estimate copy-on-write amplification for later sparse tail writes, and distinguish packed table bytes from measured Python metadata.

Part 4 - Experimental Design and Instrumentation

Run a bounded geometric sweep with fixed dtype, layers, heads, request arrivals, prompt lengths, output lengths, and token values. Rebuild equal-budget state for each trial and separate cold pool creation from warmed reuse. Use deterministic short, long, and mixed request traces, then validate promising settings on held-out traces. Measure useful completed tokens, peak slot ownership, table work, dense gather temporaries, and full latency. Preserve rejected candidates and their failure phases rather than silently dropping them.

Hold the total pool payload budget in bytes constant and derive the maximum bundle count for every candidate block size. Record any unusable budget remainder. Use matching prompts whose common prefix ends just before and just after a block boundary so sharing granularity is observable. Keep attention backend and gather policy stable; otherwise a kernel-selection change may dominate the block-size comparison.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class KVBlockTrial:
    block_tokens: int
    payload_budget_bytes: int
    trace_id: str


def predict_block_trial(trial, request_trace, kv_contract):
    # Estimate capacity, mappings, allocation events, and sharing boundaries.
    ...


def benchmark_block_trial(trial, validated_runtime):
    # Compare identical useful work with cold and warm costs separated.
    ...


def E09(config):
    # Select a robust feasible block size from auditable measurements.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Keep raw lengths, capacities, block counts, allocator peaks, error checks, and timing distributions beside summary scores. Require a counterexample to smaller-blocks-always-win and another to larger-blocks-always-run-faster. A lower slack percentage can coexist with higher total peak memory because metadata or gather outputs grow. Report sampling limitations and uncertainty when latency differences are smaller than run-to-run variability.

Show whether the selected block size remains feasible during copy-on-write and request replacement, not only in steady state. Compare calculated slack with direct assigned-capacity accounting at each event. A lower number of bundle acquisitions does not imply fewer backend allocations after warm-up; report these counters separately and preserve uncertainty in unavailable allocator fields.

Part 7 - Engineering Interpretation and Philosophy

Recommend a block size conditional on the tested workload and consumer. Explain whether the limiting mechanism was capacity, metadata, launch overhead, or attention work. Avoid translating a prototype's optimum into a universal recommendation for optimized serving kernels.

19.36.11 Exercise 10

Exercise Title: Free KV Blocks Immediately When a Sequence Finishes and Reuse Them for Newly Arriving Requests

Part 1 - Exercise Contract and Why It Matters

Release a finished request's KV mappings promptly and recycle eligible blocks for later arrivals. Interpret immediately as prompt logical retirement followed by reuse at the earliest safe completion boundary. Deliver lifecycle evidence proving that a new request never receives storage still used by an older request's attention or append operations.

Part 2 - System and Memory Mental Model

Request completion may follow sampling, an EOS decision, a length limit, or cancellation. Previously submitted CUDA work can remain active. Retiring the request prevents new operations but does not make unfinished readers vanish. Keep pending blocks separate from reusable free blocks. Initially there is no prefix-cache retention; later reusable cached blocks require a separate ownership policy. Backend allocated or reserved bytes may remain high while application slots are successfully recycled.

Part 3 - Quantitative Prediction Before Execution

For staggered arrivals and completions, predict mapped, pending, and reusable counts after every event. Include a request that finishes while its last attention read has not completed. Determine which next arrival can be admitted before and after that event. Calculate new backend allocations versus pool reuses. Predict why a pool with unchanged retained payload can serve more total requests without reducing its measured backend residency.

Part 4 - Experimental Design and Instrumentation

Use bounded requests with deterministic output lengths and a separate controlled completion-token model before introducing actual CUDA events. Test EOS, maximum length, cancellation before append publication, and repeated completion notification. Reuse numeric request IDs only with new generations. Fill recycled payload with a new recognizable pattern and verify request isolation. Measure finish-to-logical-retirement and finish-to-safe-reuse delays separately. Include shutdown drain in complete-run throughput and do not abandon outstanding work to make recycling appear instant.

Create a completion event whose request has no remaining scheduler work but still has an outstanding reader lease. Submit a new request before that lease completes. It must use another eligible slot or wait under the declared policy. After completion, verify the old generation is retired and the slot can be reused. Treat cancellation as a lifecycle transition rather than a magic synchronization operation.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class RetiredRequest:
    request_id: int
    generation: int
    pending_handles: tuple
    completion_tokens: tuple


def finish_request(runtime, request, reason):
```

```
    # Retire identity and detach mappings without premature storage reuse.
    ...


def recycle_completed(pool, completion_evidence):
    # Reconcile exact generations before moving pending slots to free.
    ...


def E10(config):
    # Replay arrivals, completion, cancellation, and delayed reader events.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require every free block to be unreachable from a live mapping or outstanding lease. Audit request tables and pending work independently from the free list. Report admitted requests, reuse count, pending peaks, delays, backend peaks, and rejected stale operations. A completion callback for an old generation must not release a newly reused slot. Report cached reservation separately from a genuine ownership leak.

Record a delayed completion for a block that has already been safely recycled through another path and require generation validation to prevent double free. Check finish notifications for idempotence or a clear duplicate-operation error. Measure application slot reuse independently of backend memory release, which may not occur while the pool intentionally retains free tensors.

Part 7 - Engineering Interpretation and Philosophy

Explain how safe recycling improves sustained capacity without necessarily lowering retained VRAM. Choose a lifecycle policy with explicit cancellation and shutdown behavior, then state the extra ownership rules needed when completed prefixes remain available for reuse.

19.36.12 Exercise 11

Exercise Title: Build Reference Counting for Shared KV Blocks and Prevent Shared Prefix Memory from Being Released While Another Sequence Still Uses It

Part 1 - Exercise Contract and Why It Matters

Track explicit references when requests share complete prefix blocks. Prevent reclamation while any supported owner or in-flight reader remains. Deliver independent reference-count audits and a decomposition of logical mapped capacity versus unique payload backing.

Part 2 - System and Memory Mental Model

Count owning mapping entries rather than only request objects. Separate active request references, optional reusable-prefix retention, and device leases. A zero active-request count can coexist with a valid cached prefix; a zero logical count can coexist with an unfinished reader. Read-only sharing is the initial contract. Arbitrary external writable tensor aliases are not tracked automatically and must be disallowed or represented by explicit leases.

Part 3 - Quantitative Prediction Before Execution

For three requests sharing two full blocks and owning separate tails, calculate total mappings, unique bundles, and counts per bundle. Destroy requests in multiple orders and predict reclaimable capacity. Add a retained prefix-cache owner and a delayed attention lease in separate trials. Compare avoided duplicate payload with added table and ownership metadata. Define whether duplicate entries in one request are supported and how they contribute to the count.

Part 4 - Experimental Design and Instrumentation

Create sharing through one controlled attach operation and detach through one checked lifecycle path. Recompute counts from a multiset of live mapping handles and separately recorded cache owners. Test shared-prefix destruction, request cancellation, duplicate attach, double detach, stale handles, and completion after the last request disappears. Keep host ownership changes serialized initially. Validate the complete K/V identity and immutable content for every sharing request after each event, not just reference-count totals.

Create three mappings to one block, including a controlled duplicate mapping case if supported, and remove them in different orders. Track active mappings, retained prefix ownership, and device leases as separate contributions to release eligibility. Define whether an index entry itself owns payload or only references an independently owned cache record. Never leave this choice implicit in the reference-count implementation.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class KVReferences:
    active_mappings: int
    retained_cache_owners: int
    inflight_leases: int
    generation: int

class KVReferenceTracker:
    def attach(self, handle, mapping_identity):
        # Add exactly one supported ownership contribution.
        ...

    def detach(self, handle, mapping_identity):
        # Remove ownership and defer reuse while other obligations remain.
        ...

    def audit(self, request_tables, prefix_index, active_leases):
        # Recompute counters without trusting maintained reference totals.
        ...


def E11(config):
    # Validate shared prefix lifetime under all request completion orders.
```

. . .

Part 6 - Measurement, Validation, and Failure Analysis

Require nonnegative counts and no reusable block with a valid owner or lease. Record unique payload, mapped capacity, retained reusable prefixes, metadata, and pending reclamation independently. Demonstrate a counterexample to freeing blocks when the original producing request finishes. Also demonstrate why counting references alone is insufficient for safe mutation. Missing external alias tracking must remain an explicit interface restriction.

Use an independently enumerated mapping multiset for the audit rather than reading maintained totals back as their own proof. Verify a page with zero active requests but a retained immutable cache owner remains available for a later hit. Then evict that cache record, invalidate lookup reachability, and reclaim only when the remaining lease obligations have ended.

Part 7 - Engineering Interpretation and Philosophy

Explain when sharing reduces unique storage and when retained cache owners reduce capacity available for active work. State the reference and immutability conditions that copy-on-write must preserve to let requests diverge safely.

19.36.13 Exercise 12

Exercise Title: Implement Copy-on-Write When Two Sequences Share a Prefix Block and One Sequence Extends or Modifies Its Logical Cache

Part 1 - Exercise Contract and Why It Matters

Implement copy-on-write for a shared partial prefix block before a request extends or changes that cache region. Preserve the other request's KV values, valid length, and future outputs. Allocate and copy only the necessary private replacement while accounting for transition headroom and failure behavior.

Part 2 - System and Memory Mental Model

Sharing a complete immutable prefix block does not require copying it when a new token starts a new logical block. Extending a shared partial block requires a private tail under this exercise's immutable-sharing policy. Mutation of an earlier token's KV is not generally an equivalent model edit: subsequent layers or positions may depend on the original history. Restrict the primary workload to append-only branches from identical processed prefixes; label direct KV modifications as storage-isolation tests, not semantic text edits.

Part 3 - Quantitative Prediction Before Execution

For two requests sharing $P-1$ valid tokens in one block, predict private bundles and copied valid K/V bytes when one appends. Contrast with a shared full block followed by a new block. Calculate peak unique payload while old and replacement bundles coexist, including all layers. Predict the pool-full case and the unchanged state required if private capacity cannot be admitted. Separate copied prefix values from newly computed token writes.

Part 4 - Experimental Design and Instrumentation

Create identical deterministic decoder prefixes, branch their teacher-forced suffix tokens, and compare each branch with an independent contiguous reference. Test partial and full

boundaries, repeated private appends, cancellation during preparation, and insufficient transition capacity. Serialize writers initially. Define publication only after the private copy and new writes have the required dependencies. Keep old shared blocks live for other requests and pending readers. Reject writable view access that bypasses copy-on-write interception.

Choose an immutable-sharing rule even for currently unused bytes in a partial block. This conservative policy makes private-tail publication easier to audit. Contrast it with a possible more complex append-sharing policy without claiming that the latter is implemented. For storage-isolation mutation tests, do not present manually edited KV as equivalent to changing the original prompt tokens and rerunning the model.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

class KVCopyOnWrite:
    def plan_append(self, request, new_positions):
        # Identify immutable shared tails and reserve private capacity.
        ...

    def prepare_tail(self, plan):
        # Preserve valid old KV and establish copy completion dependencies.
        ...

    def publish(self, plan, layer_write_evidence):
        # Replace only the writer's mapping and update reference ownership.
        ...


def E12(config):
    # Compare branching logits and audit full/partial boundary behavior.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require both branches' attention outputs and logits to match their references within declared tolerances. Verify the untouched branch's stored prefix exactly and audit counts after success and admission failure. Report copied bytes, new writes, provisional blocks, steady sharing, and completed latency. Distinguish pre-publication cleanup from arbitrary device-fault recovery; do not claim transactional rollback after a failed asynchronous kernel.

Inject failure after preparing one replacement but before committing a multi-block operation. Old snapshots must remain valid and provisional blocks must become recoverable. Verify that a full shared prefix followed by a private new block incurs no unnecessary copy of completed history. Report copy-on-write latency and peak headroom beside steady shared-memory savings.

Part 7 - Engineering Interpretation and Philosophy

Explain how sharing shifts some allocation to the moment of divergence. Recommend a tail-sharing policy from measured branch frequency and headroom, and preserve the distinction between storage mutation safety and the mathematical validity of editing a Transformer history.

19.36.14 Exercise 13

Exercise Title: Build Prefix KV Cache Sharing for Requests with Identical Prompt Prefixes and Measure the Reduction in KV Memory and Prefill Work

Part 1 - Exercise Contract and Why It Matters

Reuse validated prompt-prefix KV across requests and measure both unique storage savings and avoided prefill computation. Preserve exact model context and output semantics. Distinguish reuse of already computed KV from merely sharing a copied tensor after independently recomputing every request's prefix.

Part 2 - System and Memory Mental Model

A prefix match concerns the complete preceding token context under the same model weights, adapter, positional convention, masks, and other relevant inputs. Matching a middle token block alone is insufficient because its hidden states depend on earlier tokens. Initially share complete immutable blocks and compute partial suffixes privately. Reused KV does not automatically include the final prompt logits or hidden state needed to choose the next token. Define whether a small boundary recomputation or separately retained output state supplies it.

Part 3 - Quantitative Prediction Before Execution

For requests sharing m complete blocks and different suffixes, predict unique backing and token positions whose prefill projections and earlier hidden computations can be skipped. Account for suffix queries still attending to shared history. Calculate matching tokens lost to an incomplete block. Compare first-request cold cost, later hit cost, and retained-prefix capacity. Include lookup and any boundary recomputation in the expected latency savings.

Part 4 - Experimental Design and Instrumentation

Use a fixed tiny decoder and explicit token-ID sequences with identical prefixes but different suffixes. Compare independent prefill, post-compute storage sharing, and actual prefix-skip execution. Validate per-layer KV, attention outputs, and next-token logits. Include nearly matching prompts, different position bases, and an incompatible model-context identifier that must miss. Separate cold population from warmed hits and bound retained prefix ownership so memory savings are not claimed while cached blocks grow without limit.

Keep model weights, adapter, input embeddings, causal policy, and positional configuration fixed during a hit comparison. Context-dependent positional scaling can invalidate an apparently identical prefix, so unsupported changes must force a miss. Establish how a fully reused prompt obtains the final hidden state or logits; recomputing a boundary token requires the corresponding cache validity and overwrite policy to be defined.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class PrefixReusePlan:
    matched_full_blocks: int
    resumed_position: int
    boundary_output_policy: str
    context_identity: tuple


def plan_prefix_reuse(request, cache_index, model_context):
    # Validate complete causal prefix identity and output-state availability.
    ...


def resume_prefill(plan, suffix_tokens, decoder):
    # Skip valid reused work and obtain correct boundary/suffix logits.
    ...


def E13(config):
    # Separate storage sharing from actual prefill work avoided.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report matched tokens, skipped computed positions, remaining suffix work, unique payload, retained cache bytes, lookup time, and complete time-to-first-token. Verify a full-prompt hit under the declared boundary-output policy; KV alone must not be mistaken for ready logits. Check numerical parity before reporting speedup. A higher cache hit rate need not improve end-to-end latency if lookup, materialization, or retained-capacity pressure dominates.

Measure skipped prefill positions directly rather than inferring them from shared block counts. Post-compute sharing can save storage without avoiding any projections. Include a long shared prefix with a short suffix and another with a long suffix, since suffix attention still reads the prefix. Report retained cache capacity and additional output-state storage alongside the observed time-to-first-token change.

Part 7 - Engineering Interpretation and Philosophy

Explain the semantic identity required for valid prefix reuse and the workload conditions that amortize retained-prefix storage. State which input families are unsupported instead of generalizing a text-only fixed-context prototype to every model or prompt representation.

19.36.15 Exercise 14

Exercise Title: Hash Complete Prefix Blocks and Build a Global KV Block Lookup Table for Automatic Prefix Cache Reuse

Part 1 - Exercise Contract and Why It Matters

Build a global lookup index for complete immutable prefix blocks using context-aware chained hashes and verified candidate matches. Connect index ownership to block lifetime so cached hits cannot resolve to recycled payload. Deliver collision, invalidation, and stale-generation tests before automatic reuse is enabled.

Part 2 - System and Memory Mental Model

A block key must depend on its preceding prefix, current token IDs, and semantic model context. Identical current tokens after different histories can produce different KV. A hash selects candidates; it is not mathematical proof of equality. Store sufficient identity evidence to verify matches under the chosen threat and correctness model. Do not hash uninitialized partial blocks or mutable payload as stable prefixes. The index's retained ownership and eviction policy are separate from active-request references.

Part 3 - Quantitative Prediction Before Execution

Construct two prompts whose second blocks contain identical tokens after different first blocks and predict a required miss between their second-block identities. Calculate index entry count, retained payload, and host token metadata for a finite prompt set. Predict duplicate-content candidates and lookup behavior after block recycling. Define the expected longest reusable complete-prefix boundary when only some earlier blocks remain cached.

Part 4 - Experimental Design and Instrumentation

Use deterministic canonical key encoding and a documented digest choice. Include model revision, adapter identity, position policy, dtype/layout, and cache namespace as required by the support contract. Force collisions with a deliberately tiny test digest and require full identity verification to reject false hits. Invalidate entries before recycling their blocks and validate generations on every hit. Test duplicate insertion, partial-block rejection, context changes, and a missing earlier block in a prefix chain.

Verify ancestry as well as current-block tokens when resolving a collision. A parent digest can collide too, so a verified chained identity needs sufficient prefix evidence under the selected policy. Canonical encoding must preserve token boundaries and context fields unambiguously. Make the cache namespace part of the allowed reuse contract and keep unsupported multimodal or custom embedding inputs outside the text-only index.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import hashlib

@dataclass
class PrefixCandidate:
    digest: bytes
    parent_identity: object
    block_tokens: tuple
    model_context: tuple
    handle: object

class PrefixBlockIndex:
```

```
    def lookup(self, parent, tokens, context):
        # Hash canonically, then verify semantic identity and generation.
        ...

    def insert(self, candidate, completion_evidence):
        # Publish only immutable complete KV with valid retained ownership.
        ...

    def invalidate(self, handle):
        # Remove reachability before payload recycling.
        ...


def E14(config):
    # Test forced collisions, stale entries, and context-dependent misses.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report hit candidates, verified hits, collisions rejected, stale entries, lookup cost, index bytes, and retained unique payload. A single dictionary entry per digest must not overwrite distinct colliding prefixes without a safe resolution policy. Verify attention and logits after automatic hits. A successful hash test does not establish cross-model compatibility or correct reuse when omitted context changes the cached values.

Populate the index, recycle a candidate's slot, and attempt lookup again. The stale entry must be invalidated or rejected by generation checking. Force two distinct histories into the same digest bucket and verify both remain resolvable without false hits. Record lookup metadata growth and bound it; a compact GPU pool does not justify an unbounded host index.

Part 7 - Engineering Interpretation and Philosophy

Explain why automatic reuse combines content identity, lifetime management, and admission policy. Choose a small auditable index with bounded retention before adding complex eviction heuristics, and identify what evidence makes a reported hit safe for the supported decoder.

19.36.16 Exercise 15

Exercise Title: Serve Continuously Arriving Variable-Length Requests from a Fixed KV Block Pool and Detect Allocation Pressure Before Exhaustion

Part 1 - Exercise Contract and Why It Matters

Serve a replayable stream of variable-length requests from a fixed KV pool. Detect pressure before admitting appends that cannot finish safely. Deliver explicit admit, queue, pause, reject, and completion decisions under a declared policy, preserving useful token histories and bounded host scheduling state.

Part 2 - System and Memory Mental Model

A free-block count is only one admission input. The next step may need new tail blocks, copy-on-write replacements, temporary attention outputs, and pending-reader headroom. Cached

inactive prefixes occupy retained slots and may be reclaimable under their ownership policy; active history is not arbitrarily disposable in this full-context chapter. Offloading and lossy context eviction belong to later chapters. Never silently truncate context to make a request fit.

Part 3 - Quantitative Prediction Before Execution

For each scheduler step, predict active mapped blocks, retained prefix blocks, pending slots, reusable slots, and provisional reservations. Calculate new bundles for the next permitted token quantum. Define a pressure threshold in a named scope and show a case where equal free counts imply different next-step demand. Estimate maximum admitted concurrency for one trace without treating average request length as a hard capacity guarantee.

Part 4 - Experimental Design and Instrumentation

Replay fixed arrival times, prompt lengths, output lengths, and token traces. Start with one host scheduler and explicit microbatch boundaries. Reserve capacity before processing a step and commit only completed supported work. Bound waiting requests and prefix retention. Test a burst of arrivals, many partial tails, a long request, and a temporarily unreclaimable pending block. Compare simple FIFO and a documented fairness policy while reporting rejected and unfinished requests. Include prefill and decode separately in the useful-work ledger rather than mixing all tokens into one ambiguous rate.

Reserve upcoming boundary-crossing and private-tail requirements before launching the next token quantum. Protect active full-context history from reclamation. If retained inactive prefixes are evicted, invalidate their lookup entries and release cache ownership before reuse, while respecting any device leases. Track the temporary headroom required by the selected attention consumer instead of admitting solely from nominal KV payload.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class KVAdmissionState:
    free_blocks: int
    pending_blocks: int
    retained_prefix_blocks: int
    provisional_blocks: int
    workspace_headroom_bytes: int

class ContinuousKVScheduler:
    def pressure(self, state, next_step_demands):
        # Predict complete supported step demand before exhaustion.
        ...

    def admit(self, arrivals, active_requests, state):
        # Enforce capacity, bounded queues, and the declared fairness policy.
        ...

    def commit_step(self, completed_work):
```

```
        # Advance only coherent processed-token and completion state.
        ...


def E15(config):
    # Replay steady traffic and bursts with exact request accounting.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Audit every admission and transition against the slot and temporary-memory contracts. Report completed requests, generated tokens, queued/rejected work, time-to-first-token, inter-token gaps, tail latency, and peak state. A scheduler that maximizes token rate by starving long requests has not met an unspecified fairness objective. Do not report success only for admitted survivors while hiding rejected arrivals. Mark unsupported total-VRAM guarantees explicitly.

Replay a pressure burst followed by a recovery period and verify queued requests retain their original identities and histories. Compare admission predictions with observed slot changes at every step. Report waiting time and rejected work alongside throughput; excluding slow or rejected requests from the denominator can make an overloaded policy appear unfairly successful.

Part 7 - Engineering Interpretation and Philosophy

Explain the chosen pressure signal and the response it authorizes. Distinguish predicting block exhaustion from controlling all GPU memory, and identify the workload envelope in which the admission policy preserves full-context semantics and useful progress.

19.36.17 Exercise 16

Exercise Title: Compare Contiguous and Paged KV Cache Designs by Peak VRAM, Memory Waste, Maximum Concurrent Sequences, and Decode Throughput

Part 1 - Exercise Contract and Why It Matters

Compare contiguous and paged cache designs under equal semantics and auditable resource budgets. Report peak memory, unused capacity, maximum tested concurrency, and completed decode throughput. Preserve the cache growth and attention consumer policies so differences can be attributed to actual mechanisms rather than labels.

Part 2 - System and Memory Mental Model

A static contiguous cache, a growing concatenation cache, and a paged cache make different capacity and copying choices. A paged reference that reconstructs dense KV still pays temporary memory and data movement. An optimized paged kernel is a different consumer and must be labeled as such if implemented. Equal pool payload does not imply equal total peak VRAM. Weights, outputs, workspaces, retained prefixes, and table storage need consistent accounting.

Part 3 - Quantitative Prediction Before Execution

Predict memory curves for a fixed batch and for continuously arriving ragged requests. Estimate a concurrency bound from capacity, then add slot rounding, sharing, pending work, and temporary headroom. State whether maximum concurrency means a tested safe admission

count, a measured completed workload, or an analytical estimate. Predict a case where paging admits more sequences but has worse per-step latency because its reference gather dominates.

Part 4 - Experimental Design and Instrumentation

Benchmark equal teacher-forced traces first and separately test deterministic greedy generation. Keep weights, precision, masks, positions, attention backend, arrival trace, and output limits fixed. Report cache-only microbenchmarks separately from complete decoder execution. Reset initial state and peaks for each region, repeat trials, and separate cold prefill from warmed decode. For concurrency sweeps, use bounded admission rather than allocating to uncontrolled device failure. Evaluate sharing disabled and enabled as separate ablations so allocation and prefix benefits are not conflated.

Define prefill-token rate, generated-token rate, and request throughput with separate numerators and complete elapsed-time denominators. A processed prompt token is not a generated output token. For maximum-concurrency tests, include the long-context peak and request replacement transitions. Record whether the baseline knows final output lengths or only receives the same conservative limits available to the paged runtime.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class KVBenchmarkCase:
    cache_policy: str
    attention_consumer: str
    sharing_enabled: bool
    payload_budget_bytes: int
    request_trace_id: str


def compare_cache_designs(cases, decoder, reference_outputs):
    # Validate equal work and include all consumer/materialization costs.
    ...


def concurrency_sweep(case, tested_limits):
    # Record admission, completed work, and resource evidence per candidate.
    ...


def E16(config):
    # Explain measured capacity and performance differences by mechanism.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report allocated and reserved peaks, useful KV bytes, assigned capacity, tail slack, retained free payload, metadata, temporary peaks, and elapsed completed work. Include error tolerances, failures, admission counts, unfinished requests, and latency spread. Do not infer model throughput from an allocator loop or borrow published optimized-kernel speedups as measurements of this prototype. A negative or inconclusive performance result is valid evidence.

Repeat close comparisons and report uncertainty rather than ranking from one run. Keep failed and unfinished cases in the result table. If actual generation diverges because of floating-point differences, return to equal teacher-forced traces for performance attribution and report generation divergence separately. A change in sampled token histories changes the workload and cannot be ignored.

Part 7 - Engineering Interpretation and Philosophy

Recommend a design for the tested workload and support matrix. Explain when contiguous storage remains preferable and when paging's capacity flexibility outweighs indirection. Preserve uncertainty when close rankings are dominated by measurement noise or a consumer implementation difference.

19.36.18 Exercise 17

Exercise Title: Implement a Dynamic KV Block Allocator That Grows, Shrinks, Shares, and Recycles Physical Blocks Across Concurrent Sequences

Part 1 - Exercise Contract and Why It Matters

Integrate grow, shrink, sharing, copy-on-write, and recycling into a dynamic KV allocator serving multiple live requests. Preserve coherent per-layer validity and checked identities across every operation. Keep allocator service and model execution evidence separate while connecting them through explicit completion and ownership tokens.

Part 2 - System and Memory Mental Model

Growing appends processed tokens. Shrinking may truncate an uncommitted speculative suffix or destroy a completed request, but removing required past history changes the full-context task. Define supported truncation to a retained prefix under the fixed causal positional policy. Later tokens must be recomputed if the continuation changes. A retained partial block can contain invalid old tail bytes; validity and future writes must prevent those bytes from becoming visible accidentally.

Part 3 - Quantitative Prediction Before Execution

Hand-simulate a trace containing two shared prefixes, divergent appends, partial truncation, cancellation, and request-ID reuse. Predict unique blocks, mappings, private replacements, pending blocks, and reusable capacity after each event. Calculate the maximum provisional ownership, not just final live capacity. Show when truncation removes a full block and when it merely increases slack in a retained partial block.

Part 4 - Experimental Design and Instrumentation

Build all operations on one checked ownership authority instead of separate free lists maintained by different subsystems. Compare every stable state against independently reconstructed request histories and reference counts. Test failure before publication, delayed completions,

stale block handles, shared-tail truncation, and duplicate lifecycle notifications. Use one host scheduler initially; concurrency support requires an explicit serialization or synchronization protocol. Bound metadata and pending operation records. Validate the decoder after regrowing a truncated prefix with a different suffix.

Define truncation as retaining a known causal prefix and invalidating only later positions. If the next continuation changes, recompute its K/V through the model rather than reviving old tail bytes. A shared block that remains partially valid must preserve other owners' values and follow the declared copy-on-write rule before new writes. Keep table changes coherent with per-layer valid-write state.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class KVMutationPlan:
    request_generation: int
    expected_length: int
    target_length: int
    provisional_handles: tuple
    dependencies: tuple

class DynamicKVAllocator:
    def plan_change(self, request, operation):
        # Validate semantics, ownership, and complete transition capacity.
        ...

    def prepare(self, plan):
        # Reserve/copy/initialize while old committed mappings stay valid.
        ...

    def commit(self, plan, readiness):
        # Publish checked length/table changes and retire old obligations.
        ...

    def reclaim(self, completed_tokens):
        # Reuse only exact generations whose obligations have ended.
        ...


def E17(config):
    # Audit mixed lifecycle operations and resumed decoder correctness.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require exact supported KV values, correct logits within tolerance, count reconciliation, no stale publication, and no capacity violation. Report failed plans and provisional cleanup as well as successful operations. A successful shrink must invalidate removed positions without erasing another request's shared prefix. Distinguish logical allocator rejection from model execution failure and avoid promising rollback after arbitrary CUDA errors.

Exercise a delayed completion after truncation and after request generation reuse. It must retire only the ownership it originally protected. Compare mixed-operation histories with a dense CPU lifecycle oracle and actual decoder checkpoints. State whether failures are rejected before commit or leave the request in an explicit failed state; never silently advance length after incomplete layer execution.

Part 7 - Engineering Interpretation and Philosophy

Explain which semantic operations the allocator genuinely supports and why storage reclamation cannot authorize arbitrary context deletion. Choose a small coherent lifecycle API that later stress tests can exercise completely rather than exposing many loosely synchronized mutation entry points.

19.36.19 Exercise 18

Exercise Title: Stress-Test the Paged KV Runtime with Long Contexts, Shared Prefixes, Random Sequence Completion, Block Exhaustion, and Heavy Request Churn

Part 1 - Exercise Contract and Why It Matters

Stress-test the paged KV runtime with long bounded contexts, shared prefixes, random completion, block exhaustion, and heavy request churn. Search for minimal reproducible failures in values, attention semantics, ownership, identity, and resource bounds. Keep controlled slot pressure separate from uncontrolled hardware exhaustion.

Part 2 - System and Memory Mental Model

Critical failures combine valid operations: a shared tail is copied while a request is cancelled, a prefix index points to a recycled block, or an old attention completion arrives after request-ID reuse. Numerical parity alone cannot detect a leaked slot, and a correct free count cannot detect incorrect masking. Maintain independent semantic, ownership, and resource oracles. Long context is bounded relative to the tested environment, not a requirement to consume all available device memory.

Part 3 - Quantitative Prediction Before Execution

Define expected invariants and calculate worst-case transition demand for an adversarial short trace before random testing. Predict exhaustion where active history consumes every slot and no retained prefix is safely reclaimable. Specify the correct queue or rejection response. Construct one trace that crosses every block boundary and another that repeatedly creates partial shared tails. Define progress only for histories whose dependencies eventually finish and whose resource demand becomes admissible.

Part 4 - Experimental Design and Instrumentation

Use fixed seeds, finite request counts, bounded lengths, an explicit arrival trace, and controlled completion order. First run a deterministic transition simulator, then supported actual CUDA execution. Add named preparation failures and forced hash collisions. Compare

stored KV and teacher-forced logits with a contiguous oracle at selected checkpoints and audit ownership after every event. Save failing traces and reduce them while preserving setup and dependencies. Repeat additional seeds after the primary trace is reproducible.

Separate expected invalid-operation rejections from correctness failures in valid traces. Save completion ordering and scheduler decisions as well as the random seed, because actual asynchronous timing may not reproduce from seed alone. Include a deterministic interleaving where a request ends while a shared-tail copy is pending. Keep all payload and metadata bounds small enough for repeated diagnostic replay.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class KVStressBounds:
    max_requests: int
    max_context_tokens: int
    pool_blocks: int
    steps: int
    max_pending_operations: int


def generate_kv_trace(seed, bounds):
    # Produce supported histories and labeled invalid-operation controls.
    ...


def audit_runtime(runtime, semantic_oracle, lifecycle_oracle):
    # Check KV identity, masks, counts, generations, and capacity separately.
    ...


def minimize_kv_failure(trace, replay):
    # Preserve causal/setup dependencies while reducing a failing history.
    ...


def E18(config):
    # Validate adversarial traces before supported asynchronous stress.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report trace IDs, seeds, exact failure phase, reproducibility, and smallest counterexamples. Include double release, delayed readers, stale prefix hits, partial-block leakage, duplicate append, pool exhaustion, and shutdown with pending work. Mark simulator-only and CUDA-validated outcomes separately. A run without observed failure is not a proof over all

interleavings. Never invent hardware results, memory peaks, or throughput when the required execution was not performed.

When reducing a failure, retain the prefix population, sharing, and request creation needed to reproduce it. A shorter trace with missing setup is a different error. Audit after final drain so corruption cannot hide in pending work. Record when no counterexample is found in the tested range and avoid turning that absence into a universal concurrency proof.

Part 7 - Engineering Interpretation and Philosophy

Identify the remaining highest-impact uncertainty and design its next bounded test. State which contracts survived the tested envelope and which operations remain unsupported. Treat an honest rejected allocation or unreproduced failure as an experimental outcome rather than pressure to fabricate a success story.

19.36.20 Exercise 19

Exercise Title: Build and Benchmark a PyTorch Paged KV Cache Runtime That Serves Concurrent Autoregressive Sequences Through Dynamic Block Allocation, Prefix Sharing, Copy-on-Write, and Block Recycling

Part 1 - Exercise Contract and Why It Matters

Build and benchmark the integrated PyTorch paged KV runtime with dynamic allocation, validated prefix reuse, copy-on-write, continuous request service, and safe recycling. Deliver a precise support matrix and reproducible evidence for correctness, capacity utilization, and complete useful-work performance. All substantive implementation remains learner work in the scaffold.

Part 2 - System and Memory Mental Model

Coordinate model context, processed positions, per-layer writes, block tables, reference ownership, immutable prefix identity, pending consumers, and scheduler admission. One authority must decide each mapping and lifecycle transition. The reference attention consumer may gather KV and must account for it. No general optimized PagedAttention kernel, transparent arbitrary-model support, offloading, quantization, or distributed execution is implied by this chapter's software interface. Preserve the fixed full-context inference contract.

Part 3 - Quantitative Prediction Before Execution

Predict useful KV, assigned capacity, unique shared backing, retained prefix payload, free pool capacity, metadata, provisional copies, and attention headroom for each benchmark trace. Compare contiguous, paged without sharing, paged with manual sharing, and automatic prefix reuse under equal supported work. Estimate time-to-first-token and decode-rate effects separately. Identify a workload where sharing helps and another where churn or materialization cost overwhelms its benefit.

Part 4 - Experimental Design and Instrumentation

Freeze the supported model and operation matrix. Validate subsystem oracles, then run deterministic and randomized integration traces. Evaluate held-out short, long, shared-prefix, divergent-tail, and high-churn traffic. Record device, PyTorch/CUDA versions, allocator backend, model revision, positional policy, dtype, head counts, page units, budgets, seeds, and

initial-state identifiers. Separate cold construction, prefill, warmed decode, index lookup, lifecycle transitions, and drain. Run ablations for sharing, copy-on-write, and indexing; keep correctness and admission gates unchanged.

Publish supported operations and model assumptions before the benchmark: creation, prefill, append, lookup, sharing, truncation, cancellation, and shutdown need precise semantics. Use scalar IDs to link request traces, output checks, resource observations, and lifecycle events. Bound logs and clear diagnostic tensor references between trials. Keep the final pool budget and consumer headroom identical across integrated ablations.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class PagedKVRuntimeContract:
    model_context: tuple
    block_tokens: int
    maximum_blocks: int
    workspace_headroom_bytes: int
    maximum_waiting_requests: int
    attention_consumer: str
    prefix_boundary_output_policy: str
    supported_operations: tuple

class PagedKVRuntime:
    def __init__(self, contract):
        # Establish decoder, tables, ownership, prefix index, and scheduler.
        ...

    def submit(self, request):
        # Validate identity/context and queue or admit within all limits.
        ...

    def step(self):
        # Reserve capacity, compute supported work, and commit coherent KV.
        ...

    def finish(self, request, reason):
        # Retire active ownership while preserving supported prefix retention.
        ...

    def audit(self):
        # Independently reconcile values, masks, references, and budgets.
        ...

    def shutdown(self):
```

```
        # Drain or cancel owned work and report all remaining obligations.
        ...


def E19(config):
    # Validate semantic/lifecycle contracts before complete-work benchmarks.
    ...
    # Export evidence, counterexamples, limitations, and recommendation.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Acceptance requires validated KV and logits, correct request boundaries, causal masks, coherent positions, safe sharing, no premature reuse, and truthful resource accounting. Report completed requests, prefill positions computed, decode tokens, latency distributions, rejection/waiting counts, capacity peaks, tail slack, copy bytes, index behavior, and backend allocated/reserved peaks. Keep prediction, observation, and interpretation distinct. Report actual supported execution only; an incomplete scaffold supplies neither benchmark results nor an implemented serving engine.

Include a case where contiguous caching wins, a full-prompt reuse boundary, a forced hash collision, an exhausted pool, and cancellation with delayed readers. Verify complete shutdown after both success and controlled failure. Separate proposed optimized kernels from delivered reference attention paths. The strongest final report explains correct useful work and the resource trade-off without inventing speedups or suppressing unfavorable workloads.

Part 7 - Engineering Interpretation and Philosophy

Recommend the smallest design justified by the evidence, including contiguous caching when it wins. Explain which benefit came from flexible allocation, shared storage, avoided prefill, or consumer implementation. Identify the next kernel, concurrency, or model integration needed before broader use, and keep proposed extensions separate from measured capabilities.

19.37 DMM35

Chapter Title: KV Cache Offloading Between VRAM and RAM
Specialization: Dynamic Memory Management and CUDA Systems

19.37.1 Exercise 00

Exercise Title: Build a Precise Mental Model of KV Cache Residency, CPU Offloading, Reload Latency, and the VRAM-Bandwidth Trade-Off During LLM Decoding

Part 1 - Exercise Contract and Why It Matters

Model lossless KV movement between CPU and GPU during full-context Transformer inference. Deliver a timeline of per-layer cache use, append, writeback, reload, and reclamation, with explicit value authority and completion boundaries. Separate the capacity benefit from transfer cost and GPU stall cost. Offloading changes residency, not which historical tokens the model is allowed to attend to.

Part 2 - System and Memory Mental Model

Previously processed tokens retain keys and values at each attention layer. CPU backing can preserve that history while GPU slots serve the active layer or blocks. Every valid copy needs a version and an ownership lifetime. A dirty GPU update cannot be replaced by an older CPU copy. A submitted transfer is not necessarily finished, and a free Python reference is not proof that its storage is reusable. Full-context active history is generally revisited each decode step, so cold does not automatically mean irrelevant.

Part 3 - Quantitative Prediction Before Execution

For homogeneous unquantized layers, useful payload is

$$B_{\mathrm{KV}} = 2LH_{kv}Ds\sum_{j} T_j.$$

Predict GPU residency, CPU backing, pinned staging, transition duplicates, and attention temporaries separately. Estimate reload time from measured bytes and effective link bandwidth, then add latency and scheduling overhead. Compare full-layer round trips with a policy retaining an authoritative CPU prefix and writing back only newly appended values. Do not promise the same transfer volume for these different protocols.

Part 4 - Experimental Design and Instrumentation

Use a tiny explicit decoder with fixed weights, evaluation, zero attention dropout, and no autograd. Start with unquantized full causal attention, one device, and a teacher-forced token trace. Preserve GQA head mapping, positions, mask visibility, request identity, and layer order. Keep model weights resident so cache offloading is the controlled variable. Record versions, device, link, dtype, dimensions, seed, trace ID, allocator backend, and initial state. Offloading model weights, lossy KV eviction, and quantization are separate work.

Draw separate CPU and GPU version records for a cache before append, after append, after writeback submission, and after completion. Mark which operations may read each copy in every state. Include the last sampled token that is never processed because a request ends. Its token ID does not create an extra KV position. Keep layer-local prepared values separate from globally committed request progress.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class KVOffloadContract:
    model_context: tuple
    cache_dtype: str
    gpu_payload_budget_bytes: int
    cpu_backing_budget_bytes: int
    pinned_budget_bytes: int
    authority_policy: str
```

```
def predict_offload_timeline(contract, decode_trace):
    # Track valid versions, residency, transfers, and completion dependencies.
    ...


def E00(config):
    # Define lossless semantics and audit a small layer-by-layer timeline.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require separate columns for prediction, observation, and interpretation. Report cache useful bytes, retained capacity, transfer direction, completed latency, and unsupported metrics honestly. A capacity saving is not proof of a throughput gain. Export bounded CPU scalar logs without retaining live KV tensors or graphs. Hardware results must come from actual execution; a simulator can validate protocol transitions but cannot establish PCIe bandwidth.

Compare a model where all active history is revisited every step with a trace containing inactive requests. Explain which apparent cold blocks are merely waiting for their next mandatory use. The residency model should expose both cases without changing attention masks or token histories. A convenient software label cannot establish semantic dispensability or guarantee future reuse timing.

Part 7 - Engineering Interpretation and Philosophy

Explain which limitation moved from VRAM to host capacity or transfer bandwidth. Identify when lossless offloading makes an otherwise inadmissible workload possible and when fully resident execution remains preferable. The objective is correct useful inference within a declared resource envelope.

19.37.2 Exercise 01

Exercise Title: Measure Per-Layer KV Cache Growth During Autoregressive Generation and Determine When KV Memory Begins to Compete with Model Weights for VRAM

Part 1 - Exercise Contract and Why It Matters

Measure cache growth separately for every attention layer and determine when its residency creates pressure alongside fixed model weights. Produce an auditable token-by-layer memory ledger rather than subtracting arbitrary process totals and calling the difference KV memory.

Part 2 - System and Memory Mental Model

Useful per-layer KV payload follows processed lengths, KV heads, dimensions, and dtype. Static capacity can remain flat while valid occupancy grows; concatenation-based growth can temporarily retain old and new arrays. Parameters may share storage and backend memory includes context and workspace costs. There is no universal context length where KV becomes dominant: model layout, concurrency, available headroom, and temporary peaks change the crossover.

Part 3 - Quantitative Prediction Before Execution

Derive each layer's useful bytes and aggregate them across ragged requests. For equal K/V width, calculate $2H_{kv}Ds\sum_j T_j$ per layer. Predict the length where useful KV equals unique

parameter payload under a named model, then explain why actual admission can fail earlier. Estimate static reservation, growth-copy peaks, and the next decode quantum's additional values separately. Keep query head count out of stored-head accounting under GQA.

Part 4 - Experimental Design and Instrumentation

Use bounded contexts and a fixed-weight decoder. Capture scalar observations after prefill and selected completed decode steps, with layer identities and valid lengths preserved. Compare direct cache allocation accounting with CUDA allocated/reserved peaks and driver observations under their separate scopes. Reset peaks around declared regions. Do not synchronize after every layer in the primary throughput run merely to simplify logging; use a separate diagnostic replay for detailed attribution. Validate logits against the same baseline before interpreting resource pressure.

Count parameter storage independently from parameter names because tied weights may expose multiple names for the same backing. Record useful cache values and allocated capacity per layer before aggregation. Include an empty request and a short request beside a longer one to expose incorrect maximum-length multiplication. For heterogeneous layers, preserve their own dimensions and valid lengths rather than forcing a homogeneous shortcut.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class LayerKVObservation:
    layer_id: int
    processed_lengths: tuple
    useful_bytes: int
    capacity_bytes: int
    observation_phase: str


def layer_kv_ledger(cache, architecture, phase):
    # Count unique KV storage and preserve each measurement scope.
    ...


def predict_weight_kv_crossover(architecture, parameter_storage):
    # Separate payload equality from actual headroom/admission constraints.
    ...


def E01(config):
    # Compare predictions with bounded per-layer growth observations.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report raw dimensions, dtype widths, lengths, weight payload, useful KV, assigned capacity, temporary peaks, and measurement timestamps. Flag aliases and unsupported storage counters. Demonstrate a static-cache case where allocated bytes do not grow with every token, and a ragged case where maximum length times batch size overestimates useful payload. Do not manufacture an OOM to establish that cache and weights share finite capacity.

Plot or tabulate several checkpoints including prefill, early decode, and near the bounded context limit. A single final measurement cannot reveal a transient growth-copy peak. If measuring detailed layer boundaries changes synchronization and timing, retain those results as a diagnostic replay and use an uninstrumented or minimally instrumented complete run for the performance baseline.

Part 7 - Engineering Interpretation and Philosophy

Explain whether pressure comes from useful history, overreservation, transient copies, or non-cache allocations. Use that diagnosis to define a small offload target and leave a margin for future growth rather than choosing a policy solely from one memory percentage.

19.37.3 Exercise 02

Exercise Title: Build a PyTorch KV Cache That Stores Every Layer on CUDA and Establish Peak VRAM, Decode Latency, and Throughput Baselines

Part 1 - Exercise Contract and Why It Matters

Establish a fully GPU-resident KV baseline with correct prefill and decode. Measure cache capacity, complete peak VRAM, per-token latency, and useful throughput before introducing migration. Preserve enough metadata to reproduce the exact inference workload and compare every later policy fairly.

Part 2 - System and Memory Mental Model

Choose and name the baseline growth policy: bounded static capacity or a separately measured dynamic policy. Committed lengths exclude unwritten tail capacity. Layer attention can consume its own prepared new KV after ordered writes, while global token commit waits for required layer completion. Positionally transformed keys must retain their convention. Attention masks for incremental queries must expose the correct historical positions, including ragged requests.

Part 3 - Quantitative Prediction Before Execution

Predict useful cache growth, backing capacity, headroom for one decode quantum, and relevant attention temporaries. Derive separate prefill and decode token rates with clear numerators. For a static cache, predict a flat capacity curve and increasing occupancy. For a dynamic cache, include historical copy volume if concatenation is used. Define whether constructor, transfers, sampling, validation, and final drain belong to the end-to-end region.

Part 4 - Experimental Design and Instrumentation

Use evaluation with explicit zero dropout and an inference-safe no-gradient boundary. Compare cached outputs with full-prefix recomputation on a fixed teacher-forced trace. Record the attention backend, model revision, positions, GQA mapping, masks, batch shape, and dtype. Warm relevant kernels deliberately and report cold start separately. Use wall-clock completed regions and CUDA events for suitable device intervals. Keep evidence scalar and release intermediate logits between validation points instead of retaining the entire trace on GPU.

Include a rectangular incremental-attention mask test whose query sees every legitimate earlier position. Verify mask polarity and position offsets instead of assuming an API's generic causal flag matches decoding alignment. Preserve the same backend and output-validation policy in all later comparisons. If using GQA expansion in the reference consumer, count any repeated-head temporary storage in the baseline's peak.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

class ResidentKVBaseline:
    def initialize(self, architecture, capacities):
        # Establish explicit backing layout and per-layer validity metadata.
        ...

    def prefill(self, requests):
        # Preserve causal positions, masks, and complete cache readiness.
        ...

    def decode_step(self, teacher_forced_tokens):
        # Process tokens, append each layer, and commit coherent state.
        ...


def benchmark_resident_case(runtime, trace, measurement_contract):
    # Check outputs before reporting complete useful-work metrics.
    ...


def E02(config):
    # Produce the auditable all-CUDA correctness and performance baseline.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require attention outputs and logits within declared tolerances. Report useful KV, capacity, allocated and reserved peaks, output tokens, processed prompt tokens, latency spread, and failures separately. A short unsynchronized host submission interval is not completed decode latency. If actual generation diverges between variants, return to equal teacher-forced traces for performance attribution and report generation behavior as a separate validation outcome.

Record processed prompt tokens, generated tokens, and completed requests as separate quantities. A prompt-token rate cannot be substituted for decode throughput. Include a final completion boundary so trailing kernels are charged to the run. Release validation tensors between trials and confirm diagnostic logging does not retain whole cache histories whose lifetimes distort later memory measurements.

Part 7 - Engineering Interpretation and Philosophy

Identify the baseline bottleneck and the performance ceiling offloading cannot exceed without another mechanism. Preserve the resident reference as a correctness oracle even when larger future workloads no longer fit it; use smaller matching cases rather than inventing oversized baseline results.

19.37.4 Exercise 03

Exercise Title: Move Selected Layer KV Tensors from GPU to CPU After Attention and Restore Them Before Their Next Decode-Step Use

Part 1 - Exercise Contract and Why It Matters

Move selected layers' KV to CPU after their last supported attention use and restore it before the next use. Begin with a synchronous lossless protocol, then quantify memory relief and round-trip cost. Preserve every historical key and value; migration must not silently truncate context.

Part 2 - System and Memory Mental Model

Each layer requires an authority rule: CPU current, GPU current, or a coherent replicated version. An append makes new GPU values dirty relative to older backing. Writeback must finish before CPU data is authoritative and before the GPU source can be reused. Reload must finish before attention reads the destination. Dropping a CUDA tensor while a view or evidence record retains it does not reduce live cache residency. Backend reservation may remain even after logical offload succeeds.

Part 3 - Quantitative Prediction Before Execution

For a selected layer set, predict GPU bytes removed, CPU bytes added, and H2D/D2H traffic over a finite decode trace. Compare a simple full-history writeback with an append-aware protocol that keeps an unchanged CPU prefix and transfers only dirty new ranges. Include transient old/new coexistence, new-token growth, and any dense reconstruction. Estimate a serialized latency bound using measured effective bandwidth rather than a nominal bus label.

Part 4 - Experimental Design and Instrumentation

Choose one layer first and use deliberate completion boundaries around copies. Validate the entire cache and logits after reload. Expand to several layers only after the basic version protocol is auditable. Test empty history, partial capacity, different lengths, and failure before publishing a new CPU version. Keep weights, model operations, dtype, masks, and token trace fixed. Measure migration, capacity change, and complete decode time separately. Bound CPU backing and reclaim it when its request no longer needs that version.

Define the dirty-range protocol only for append-only fixed causal KV, where older stored values remain unchanged. If the supported operation overwrites earlier KV, expand dirty tracking accordingly or reject it. Compare full-history writeback and suffix-only writeback with identical restored values, not merely the same reported cache length. Record whether CPU backing is persistent or newly allocated on every offload.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class LayerResidency:
    cpu_version: object
    gpu_version: object
    valid_tokens: int
    dirty_ranges: tuple
    state: str


def offload_layer(layer_state, cpu_destination):
    # Preserve the latest values and publish authority only after completion.
    ...


def restore_layer(layer_state, gpu_destination):
    # Restore authoritative valid KV before attention can consume it.
    ...


def E03(config):
    # Compare full writeback and supported dirty-range transfer protocols.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report useful values, stored capacity, transferred bytes by direction, version transitions, peak duplicate residency, and completed latency. Check that the next step does not read a stale CPU prefix or omit newly appended values. A failed migration must remain an explicit state, not a fabricated zero-filled cache. Do not infer total-VRAM reduction from the number of tensors whose device field changed.

Use a failure injector before authority publication to test that an incomplete CPU destination cannot become the next reload source. A cancelled request may make its values unnecessary, but its transfer endpoints still need safe lifetime handling. Report partial-copy state explicitly and keep retry behavior bounded; do not claim recovery from arbitrary device errors solely because preparation failure was handled.

Part 7 - Engineering Interpretation and Philosophy

Explain the capacity gained per transferred byte and why frequent layer reuse can make naive round trips costly. State which data becomes authoritative at each boundary so later asynchronous extensions have a precise protocol to preserve.

19.37.5 Exercise 04

Exercise Title: Verify Exact Numerical Equivalence Between Fully GPU-Resident KV Cache and Lossless CPU-Offloaded KV Cache

Part 1 - Exercise Contract and Why It Matters

Verify that lossless CPU migration preserves stored KV exactly and preserves the intended numerical inference result. Separate byte-preserving transport from attention arithmetic equivalence. Deliver staged comparisons that locate whether an error originates in copying, position mapping, masks, head layout, versioning, or a changed execution backend.

Part 2 - System and Memory Mental Model

A same-dtype copy can preserve cache values without quantization, but changing layout or attention backend can alter floating-point reduction order. Exact stored-value equality and bitwise output equality are therefore different claims. The exercise title's exactness requirement applies to preserved KV representations under the declared copy contract; inference comparisons need explicit numerical tolerances unless identical deterministic arithmetic is actually established. NaNs also require an appropriate comparison convention.

Part 3 - Quantitative Prediction Before Execution

Define expected differences for same-dtype finite-value round trips, selected raw encodings, and intentionally converted values. Predict zero cache corruption under the supported lossless path. State tolerances for attention and logits by dtype and accumulation policy. Construct negative controls for a stale version, swapped K/V, wrong layer, incorrect causal visibility, and an omitted last token. Explain which validation stage should detect each error.

Part 4 - Experimental Design and Instrumentation

First copy coordinate-coded KV through CPU and back without attention. Then compare contiguous resident and offloaded caches at every selected layer and decode position. Use a fixed teacher-forced trace with zero dropout and stable backend selection. Include partial tails, ragged sequences, GQA, nonzero positions, and boundary growth. For raw bit-pattern claims, compare bytes without converting values first. Run greedy generation only as additional evidence after direct cache, attention, and logit validation succeeds.

Create transport tests with finite coordinate-coded values, signed zero, and selected nonfinite encodings under an explicit byte-comparison policy. Keep dtype and layout fixed for the lossless baseline. A separate conversion control should fail the exact-transport criterion when information changes. For model comparisons, record accumulation precision and deterministic settings rather than assuming a fixed random seed controls every backend detail.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def compare_kv_transport(before, after, encoding_policy):
    # Test exact supported values or raw encodings without dtype conversion.
    ...


def compare_inference_paths(resident, offloaded, cases, tolerances):
    # Locate cache, attention, and logit differences separately.
```

```
    ...


def inject_validation_control(case, fault_kind):
    # Produce a labeled semantic or copy error that validation must catch.
    ...


def E04(config):
    # Establish transport exactness and appropriately scoped output parity.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report maximum and aggregate errors, mismatch coordinates, comparison policy, dtype, backend, and token position. A matching sampled token is insufficient: large logit errors can preserve the same argmax. Conversely, a tiny numerical change can alter a near-tied sampled decision without proving a lossy copy. Keep exact-copy tests separate from tolerance-based arithmetic tests and do not weaken tolerances after inspecting failures without documenting why.

Validate at more than one layer and token position so errors cannot compensate in a final output. Report the first mismatch coordinate and whether the copy already differed before attention ran. An intentionally stale version should be rejected by metadata as well as detected numerically. This distinguishes prevention by protocol from accidental detection by favorable test values.

Part 7 - Engineering Interpretation and Philosophy

Explain which equivalence was established and under what assumptions. Lossless offloading preserves the required information; it does not license unqualified claims of universal bitwise-identical outputs across devices, kernels, layouts, or software versions.

19.37.6 Exercise 05

Exercise Title: Compare Pageable and Pinned CPU Memory as KV Offload Destinations by Transfer Latency, RAM Cost, and Decode Throughput

Part 1 - Exercise Contract and Why It Matters

Compare pageable and pinned CPU destinations for KV offload using identical valid values and transfer volume. Measure preparation cost, completed copy latency, host capacity, and end-to-end decode throughput. Do not treat a quickly returned nonblocking call as proof that transfer completed or overlapped.

Part 2 - System and Memory Mental Model

Pinned memory changes host transfer capabilities and consumes a bounded system resource. Pinning a pageable tensor can allocate and copy rather than magically changing all existing references. For asynchronous D2H, the CPU must not consume incomplete destination values. For H2D from pinned memory, the source must remain alive and unchanged until transfer finishes. Reusable host buffers require ownership tracking; allocator caching and logical live pinned payload are different measurement scopes.

Part 3 - Quantitative Prediction Before Execution

Predict valid transfer bytes and destination capacity for both paths, then estimate bandwidth from completed regions. Include initial pinning and any pageable-to-pinned staging copy. Compare steady reuse with pinning on every iteration as separately labeled experiments. Derive host capacity when CPU backing and pinned staging coexist and identify the maximum outstanding slots allowed by the declared host budgets.

Part 4 - Experimental Design and Instrumentation

Preallocate matched CPU destinations and CUDA sources with the same dtype, layout, valid lengths, and values. Verify actual pin status on relevant tensors. Test both directions with explicit completion before CPU validation or source reuse. Warm deliberately, repeat trials, and separate cold allocation from reused buffers. Integrate each path into the same decoder trace after isolated copy checks. Hold stream policy fixed before studying overlap. Record whether pageable copies introduce hidden staging or synchronization only when observed, not by assuming a universal implementation detail.

Use reusable pinned destinations and a separately labeled pin-every-iteration control. Ensure the pageable comparison does equivalent copying and has the same valid capacity. A view retaining a larger host buffer can make logical payload smaller than retained RAM; record unique backing where available. Check that asynchronous D2H output is not inspected by a checksum or logging operation before its completion boundary.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class HostKVBuffer:
    storage: object
    pinned: bool
    capacity_bytes: int
    ownership_state: str
    completion_token: object


def allocate_host_kv(spec, pinned_policy, host_budget):
    # Preallocate within the correct pageable/pinned accounting scopes.
    ...


def transfer_trial(source, destination, completion_policy):
    # Measure completed copies and preserve both endpoint lifetimes.
    ...


def E05(config):
```

```
# Compare isolated transfer and complete decode behavior fairly.
...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report submission time, completed duration, transfer bytes, effective bandwidth, known live pinned payload, CPU backing, and complete throughput. If total OS or allocator pinned-memory usage is unavailable, say so instead of equating it with one tensor's payload. Require exact valid KV recovery. A pinned path that loses after including preparation is a legitimate result, not a reason to omit its allocation and copying cost.

Measure both isolated directional copies and their behavior inside decode. Host preparation or simultaneous link traffic can change effective throughput. Do not reuse a bandwidth number from another transfer size or topology without labeling it as an estimate. Known live pinned payload is an auditable lower-level quantity, not a substitute for an unavailable total OS page-lock measurement.

Part 7 - Engineering Interpretation and Philosophy

Choose host storage and staging from complete measured work and bounded capacity. Explain when pinning enables the intended pipeline and when its preparation cost or retained capacity outweighs the benefit for the tested cache sizes and reuse pattern.

19.37.7 Exercise 06

Exercise Title: Build a Layer-Wise KV Offload Pipeline That Keeps Only the Current Attention Layer Cache Resident in VRAM

Part 1 - Exercise Contract and Why It Matters

Build a serialized layer-wise pipeline that retains only the current layer's active KV payload on GPU. Keep other layer histories in CPU backing and restore them just in time. Establish this conservative capacity reference before introducing prefetch that necessarily adds simultaneous resident payload.

Part 2 - System and Memory Mental Model

Only current-layer cache residency is an application KV scope, not a promise that total VRAM equals one cache. Weights, activations, outputs, masks, workspaces, and allocator retention remain. A previous layer cannot release its source until all attention reads and required writeback finish. The current layer needs prepared new KV before its own attention, while global token commit follows required layer completion. Use reusable bounded storage rather than retaining separate CUDA arrays for every supposedly offloaded layer.

Part 3 - Quantitative Prediction Before Execution

Predict maximum layer-cache capacity for the supported dimensions and request lengths. Add new-token append space and any copy temporaries. Derive serialized H2D, attention, and D2H time over one decode step using separate measured components. Compare full-history writeback with a validated append-only dirty suffix protocol. State whether varying layer sizes require a maximum-sized slot and how much unused slot capacity remains.

Part 4 - Experimental Design and Instrumentation

Use one transfer/compute sequence with explicit completion at handoff boundaries. Process layers in decoder order and validate per-layer outputs against the resident baseline. Test prefill-to-decode transition, multiple decode steps, ragged requests, and maximum bounded length. Measure time waiting for reload and writeback separately from computation. Audit every live GPU KV reference, including views and diagnostic variables. Keep the no-prefetch policy intact in this experiment so its residency bound remains interpretable.

Start with a maximum-sized reusable current-layer slot and charge its full capacity even when a short layer uses fewer valid bytes. Later compare exact-size allocation as a separate policy with its growth and allocator costs. Test the first layer of a new decode step while the previous step's final layer has only just retired, because step boundaries can hide retained references.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

class SerialLayerKVOffloader:
    def acquire_layer(self, layer, request_state):
        # Restore authoritative valid history into the bounded active slot.
        ...

    def run_attention(self, layer, prepared_tokens):
        # Order new KV writes and consume the correct full history.
        ...

    def retire_layer(self, layer, update_record):
        # Finish required writeback and readers before releasing the slot.
        ...


def E06(config):
    # Validate the one-active-layer protocol and account for total peaks.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require correct KV, attention, and logits at selected checkpoints. Report maximum live active cache bytes, actual slot capacity, temporary KV duplicates, host backing, backend peaks, and complete decode latency. If the implementation requires a second full CUDA cache while growing or copying, disclose it and revise the claimed bound. A retained allocator reservation is distinct from a second live active cache.

Audit tensor views and local variables as well as cache dictionaries when checking the one-active-layer claim. A deleted table entry does not prove the payload lost all owners. If appending requires a temporary larger dense cache, count that transient capacity. Preserve a clear distinction between a small live working set and unchanged backend reserved memory after safe reuse.

Part 7 - Engineering Interpretation and Philosophy

Explain the best capacity guarantee achieved by serialization and the stalls it exposes. Use this reference to evaluate later overlap as an explicit exchange of extra simultaneous capacity for reduced wait time.

19.37.8 Exercise 07

Exercise Title: Asynchronously Prefetch the Next Layer KV Cache from Pinned RAM While the Current Layer Performs Attention Computation

Part 1 - Exercise Contract and Why It Matters

Prefetch the next layer's KV from pinned CPU backing while the current layer computes. Preserve correctness with explicit readiness and source-lifetime evidence. Compare complete decode time and peak simultaneous KV residency against the serialized pipeline rather than promising overlap from API flags.

Part 2 - System and Memory Mental Model

Current-layer payload and next-layer destination coexist during useful prefetch. Any outgoing source still in use adds further residency. A transfer stream must wait for the correct CPU-backed version and available destination; the compute stream must wait for its reload-ready event before attention. Pinned source data cannot be rewritten while H2D still reads it. Prefetch crossing the final layer into the next decode step must respect token and version readiness, not merely cyclic layer indices.

Part 3 - Quantitative Prediction Before Execution

For measured next-layer transfer time and current-layer compute time, predict a best-case overlap window and the residual exposed stall. This is a hypothesis, not guaranteed concurrency. Calculate live current and incoming payload, staging, and any retained outgoing source. Construct a small transfer case likely to be hidden and a large case whose copy exceeds the available compute window. Keep host submission and device completion intervals separate.

Part 4 - Experimental Design and Instrumentation

Add one-layer lookahead to the validated serial engine using a dedicated transfer stream and per-generation events. Start with one request and static capacity, then test growth and ragged histories. Validate restored KV before attention in diagnostic replays. Compare blocking restoration, nonblocking submission without overlap, and actual prefetch under identical token traces. Use a short timeline to establish concurrency, then unprofiled repeated runs for throughput. Include pipeline startup and drain in end-to-end time and retain only bounded event records.

Use ticket identities containing layer, step, version, and destination generation. Reusing one event object without a protocol can make old readiness appear to authorize a new payload. Initially allocate a bounded event/ticket set and recycle it only after consumers finish. Include the last-layer-to-first-layer wraparound explicitly in the dependency diagram rather than treating the model as an unversioned ring.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class KVPrefetchTicket:
    layer_id: int
    decode_step: int
    required_version: int
    destination_generation: int
    ready_event: object


def schedule_kv_prefetch(engine, next_layer, dependency_state):
    # Protect pinned source and reserve a nonconflicting destination.
    ...


def consume_prefetched_layer(ticket, compute_stream):
    # Wait for exact-version readiness before using KV in attention.
    ...


def E07(config):
    # Verify timeline overlap, numerical parity, and extra residency costs.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report transfer and compute intervals, residual stall, completed decode time, active/incoming bytes, and backend peaks. A separate stream can still serialize through dependencies or resource contention. Do not infer overlap from a shorter host call. Include a source-mutation negative control in a safe transition simulator and require the lifetime protocol to reject it, rather than deliberately racing real device memory.

Compare the expected overlap window with an observed short timeline and then with complete unprofiled time. Prefetch can overlap attention yet compete for memory bandwidth and slow that attention. Report both effects. Do not add compute time and copy time to calculate wall time when they overlap, and include cold pipeline fill and final drain in end-to-end comparisons.

Part 7 - Engineering Interpretation and Philosophy

Explain whether prefetch reduced the critical path or merely shifted work. Choose a bounded lookahead depth justified by measured benefit and headroom, and state which versions can be prepared before the next token is actually computed.

19.37.9 Exercise 08

Exercise Title: Asynchronously Evict the Previous Layer KV Cache to CPU Without Blocking the Current Attention Layer

Part 1 - Exercise Contract and Why It Matters

Write back the previous layer's dirty KV while the current layer performs attention, without blocking unrelated compute unnecessarily. Preserve source lifetime, CPU destination ownership, and authoritative-version publication. Measure whether asynchronous eviction reduces stalls and what capacity remains occupied until completion.

Part 2 - System and Memory Mental Model

An outgoing GPU cache is still live while D2H reads it. It becomes reusable only after its last device consumer and transfer complete. The CPU destination is not valid backing until the correct event finishes, and cannot simultaneously serve an incompatible reader or writer. With prefetch enabled, outgoing, current, and incoming payloads may coexist. Two staging slots cannot bypass these dependencies; they may force waiting before a slot can be overwritten.

Part 3 - Quantitative Prediction Before Execution

Predict transferred bytes for full-cache and dirty-range writeback protocols. Draw a timeline containing previous-layer D2H, current-layer computation, and next-layer H2D. Count simultaneous source and destination ownership at each boundary. Estimate a best-case hidden interval and exposed writeback tail. Do not assume opposite-direction copies run at independent full bandwidth: copy engines, host memory, and the link can contend.

Part 4 - Experimental Design and Instrumentation

Begin with the verified prefetch engine and introduce a separate eviction stream only where dependencies permit. Order eviction after append completion and the last attention read. Publish CPU backing after transfer completion, then release GPU ownership. Test a layer needed again before its previous writeback finishes, a growing tail, and request cancellation. Compare a serialized writeback control, asynchronous submission, and evidenced overlap. Measure full decode time with final drain and record bounded transition state.

Distinguish writing a dirty suffix into persistent CPU backing from replacing the entire backing tensor. For suffix writeback, preserve the unchanged prefix and publish the combined version only after new values arrive. An H2D reader of an overlapping CPU range can conflict with D2H writes; serialize overlapping ranges or use separate bounded backing versions under an explicit protocol.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class KVEvictionTicket:
    source_generation: int
    layer_id: int
    version: int
    dirty_ranges: tuple
    complete_event: object
```

```
def schedule_kv_eviction(engine, layer_state, last_reader_event):
    # Protect the source and destination through ordered D2H completion.
    ...


def publish_cpu_backing(ticket, residency_state):
    # Advance authority and release exact-generation GPU ownership safely.
    ...


def E08(config):
    # Validate overlapped writeback and any additional residency or stalls.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require exact backed values and correct next-use logits. Report D2H bytes, writeback delay, pending-source capacity, H2D contention, and completed useful work. Verify that a delayed old event cannot publish over a newer CPU version. A visible overlap interval is not sufficient if total decode becomes slower. Record capacity-induced waits explicitly instead of hiding them as generic attention time.

Inject delayed writeback completion in a simulator and issue the next demand for that layer. The engine must wait, use a valid newer source, or reject according to its contract. It must not restore stale values merely because the older CPU tensor exists. Verify source retirement matches the exact GPU slot generation that the D2H transfer actually read.

Part 7 - Engineering Interpretation and Philosophy

Explain the conditions under which eviction is off the critical path and when it competes with more urgent reloads. Choose a transfer ordering policy that prioritizes correct demand readiness while retaining bounded outgoing work and truthful peak-capacity accounting.

19.37.10 Exercise 09

Exercise Title: Coordinate KV Prefetch, Attention Compute, and KV Eviction with Dedicated CUDA Streams and Events Without Device-Wide Synchronization

Part 1 - Exercise Contract and Why It Matters

Coordinate reload, attention, append, and eviction using explicit CUDA stream dependencies and events. Remove device-wide synchronization from the steady pipeline while preserving meaningful measurement and CPU-consumption boundaries. Deliver a dependency graph and a generation-aware transition audit.

Part 2 - System and Memory Mental Model

Compute must wait for the required KV version; eviction waits for the last reader and new writes; slot reuse waits for all previous users. The host may submit work ahead without making it complete. Events establish ordering, not ownership by themselves. PyTorch allocator

lifetime mechanisms can protect backend deallocation, but do not authorize overwriting a still-live application pool slot. A CPU consumer of D2H output needs actual completion before reading.

Part 3 - Quantitative Prediction Before Execution

Draw the dependency graph for two decode steps and three layers with bounded staging. Predict which waits are necessary and identify a dependency cycle that would deadlock a naive ring schedule. Count pending events and live slot generations. Estimate critical-path time from dependencies and observed intervals; do not add overlapping stream durations as if all operations were serial. State where outer benchmark synchronization remains necessary.

Part 4 - Experimental Design and Instrumentation

Use named compute, reload, and eviction streams with operation IDs tied to request, layer, step, version, and slot generation. Start with deterministic simulated completion orders, then run supported CUDA traces. Audit every wait before optimizing it away. Compare against a fully synchronized correctness control and validate all outputs. Test event reuse, next-step wraparound, cancellation, and shutdown. Keep timing instrumentation separate from dependency events and include its overhead in a control run.

Name every edge in the dependency graph by the value or storage lifetime it protects. Event readiness, allocator deallocation safety, and application overwrite permission are separate obligations. Keep timing events separate from dependency-only events when configuring measurements. A CPU wait for a specific D2H completion can be valid even when the steady compute schedule contains no device-wide barrier.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class KVDependencyIdentity:
    request_generation: int
    layer_id: int
    decode_step: int
    version: int
    slot_generation: int

class KVStreamCoordinator:
    def reload_dependencies(self, identity):
        # Wait only on required source and destination obligations.
        ...

    def compute_dependencies(self, identity):
        # Order attention after the matching reload and new KV writes.
        ...

    def eviction_dependencies(self, identity):
```

```
        # Order writeback and reclamation after all exact-generation users.
        ...


def E09(config):
    # Validate the dependency graph before measuring asynchronous execution.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require identical valid KV and appropriate numerical output parity with the synchronized control. Report event/wait counts, stage intervals, host submission, completed wall time, and peak pending state. Mark inferred intervals separately from directly observed ones. Absence of device-wide synchronization does not prove absence of stalls. A missing dependency detected by simulation must be repaired before running real asynchronous accesses.

Test an event recorded for an old step against a new step's ticket and require identity validation to reject the mismatch. Check for cycles in the simulated wait graph before launching real CUDA work. At shutdown, account for work submitted on every owned stream so a successful return cannot conceal delayed writes or live sources that outlast their logical request.

Part 7 - Engineering Interpretation and Philosophy

Explain which synchronization was removed and which dependencies remain essential. Recommend the smallest explicit graph that preserves correctness; additional streams are worthwhile only when measured concurrency improves useful work within resource limits.

19.37.11 Exercise 10

Exercise Title: Build Double-Buffered KV Staging Memory So One Layer Cache Transfers While Another Layer Uses GPU-Resident KV Data

Part 1 - Exercise Contract and Why It Matters

Build double-buffered KV staging with explicit per-slot states and generations. Let one layer transfer while another uses its cache where dependencies and capacity permit. Demonstrate when two slots suffice and when pending writeback forces a stall instead of silently allocating an uncounted third buffer.

Part 2 - System and Memory Mental Model

A staging slot has capacity, assigned identity, valid version, readiness, and outstanding users. Alternating index modulo two is not a safe lifecycle protocol. A slot holding previous-layer dirty data cannot receive next-layer H2D until eviction and all readers finish. Host backing and pinned staging have independent lifetimes. Different layer lengths or dimensions can require maximum-sized slots or explicit size classes, whose retained capacity must be included in the budget.

Part 3 - Quantitative Prediction Before Execution

Hand-simulate three layers over two decode steps with H2D, compute, and D2H durations. Predict slot states and the first required reuse wait. Calculate two-slot capacity using the supported maximum layer size and separate valid payload from unused slot capacity. Add a

case where long D2H prevents ideal overlap. Estimate host staging occupancy and event state at startup, steady execution, and final drain.

Part 4 - Experimental Design and Instrumentation

Implement a checked slot state machine before introducing CUDA transfers. Require acquisition to prove all prior-generation obligations ended. Use the same layer trace as the event-coordinated reference. Test odd layer counts, step wraparound, unequal cache sizes, growth, and cancellation. Measure warm steady-state overlap separately from complete latency. Validate contents after every handoff in diagnostic replays and compare model logits. A three-slot variant is an optional separately budgeted control, not an invisible fallback.

Use a schedule where eviction is slower than computation and another where reload dominates. Determine which phase prevents the next slot reuse in each case. For unequal layer sizes, record valid bytes and maximum slot capacity separately. A two-buffer implementation must not quietly allocate a temporary third full-layer tensor when a slot is unavailable; waiting is a valid bounded response.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class KVStagingSlot:
    slot_id: int
    generation: int
    capacity_bytes: int
    assigned_identity: object
    state: str
    pending_tokens: tuple

class DoubleBufferedKV:
    def acquire_slot(self, identity, required_bytes):
        # Reject or wait until the prior generation is safely reusable.
        ...

    def transition(self, slot, event, new_state):
        # Validate the state graph and exact completion identity.
        ...

    def audit(self):
        # Count all live payload, capacity, and pending obligations.
        ...


def E10(config):
    # Validate two-slot schedules including capacity-induced stalls.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report actual allocated slot capacity, live valid bytes, slot waits, transfer intervals, temporary buffers, and complete throughput. Verify no hidden allocation exceeds the two-slot contract during growth or reconstruction. A stall can be the correct outcome of a safe schedule. Distinguish waiting for buffer availability from waiting for link completion and from attention computation; they suggest different remedies.

Validate slot identity after several odd-length layer cycles and decode-step wraparounds. A simple alternating counter may appear correct in a two-layer toy model while failing later. Include cancellation between transfer submission and consumption, then confirm the cancelled generation's completion cannot release storage assigned to another request. Audit pinned host slots using the same lifetime discipline.

Part 7 - Engineering Interpretation and Philosophy

Explain whether double buffering buys enough overlap for its retained capacity. State the workload and transfer assumptions under which two slots are sufficient and why buffer count must follow lifetime analysis rather than a fixed rule that two always hides all movement.

19.37.12 Exercise 11

Exercise Title: Compare Full KV Offloading Against Selective Layer Offloading and Identify Which Layers Are Worth Keeping Permanently Resident in VRAM

Part 1 - Exercise Contract and Why It Matters

Compare full offloading with selective permanent residency for chosen layers. Determine which retained caches save the most exposed decode cost under a fixed GPU KV budget. Keep the same processed-token trace and distinguish transfer volume saved from actual critical-path latency saved.

Part 2 - System and Memory Mental Model

In a homogeneous full-context decoder, every active layer's history is normally used once per decode step. Early or late layer position alone does not make one intrinsically hotter. Residency value can differ through cache size, transfer time, available overlap windows, and contention. Optional heterogeneous architectures need their own accounting. Retaining one layer uses capacity that could otherwise provide staging or admit another request.

Part 3 - Quantitative Prediction Before Execution

Estimate bytes and exposed stall avoided by retaining each candidate layer. Construct two layers with equal payload but different overlap opportunities, and predict why transfer volume alone gives an incomplete ranking. Calculate resident-layer capacity plus staging and transition headroom for each subset. Compare full offload, no offload, a simple fixed subset, and a measured selective policy under equal useful work.

Part 4 - Experimental Design and Instrumentation

Calibrate copy and compute intervals on separate traces. Evaluate small layer subsets first so exhaustive comparisons remain auditable, then test a simple heuristic. Preserve model weights, dtype, request lengths, mask semantics, and attention backend. Include stable contexts and growth phases. Measure cold population, warmed decode, and request completion separately.

Validate all outputs and report whether staging capacity changed when a layer became permanently resident. Test selected policies on held-out traces rather than choosing and evaluating on the same favorable sequence.

Evaluate whether retaining a layer reduces critical-path reload, dirty writeback, or both under the chosen authority protocol. Saving full writeback bytes in one protocol does not imply the same gain in a suffix-only protocol. Keep CPU-backed copies and initialization costs in the comparison. A permanent resident layer still grows and needs budget admission for later tokens.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class LayerResidencyCost:
    layer_id: int
    resident_bytes: int
    transfer_bytes_avoided: int
    exposed_stall_estimate: float
    uncertainty: object


def rank_resident_layers(measurements, staging_contract, budget):
    # Compare saved exposed cost per constrained capacity, not layer order.
    ...


def evaluate_layer_subset(subset, decoder_trace, resource_contract):
    # Include all retained, staging, and transition memory in the comparison.
    ...


def E11(config):
    # Compare full and selective offload with held-out correctness checks.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report retained layers, payload bytes, staging capacity, transfers by direction, demand stalls, complete latency, and backend peaks. Include a counterexample where the largest cache is not the best retention choice under the measured objective. Do not claim a universal best layer subset from one model or trace. A policy that improves one request's latency but lowers useful concurrency must report that trade-off.

Test the chosen subset on a different context-length and concurrency trace. The best overlap opportunity can change when attention compute grows or copy contention increases. Report a case where the simple fixed subset performs as well as the more complex policy. Prefer the simpler choice when the measured gain is smaller than uncertainty or monitoring overhead.

Part 7 - Engineering Interpretation and Philosophy

Choose a minimal selective policy supported by measurements and explain its workload sensitivity. Distinguish preserving capacity for admission from using that capacity to reduce transfer stalls, and state which objective the experiment actually optimized.

19.37.13 Exercise 12

Exercise Title: Build a VRAM Watermark Policy That Begins KV Offloading Before Cache Growth Reaches the CUDA OOM Boundary

Part 1 - Exercise Contract and Why It Matters

Begin KV demotion before predicted growth and transient work exhaust the declared GPU budget. Implement a watermark policy with explicit metric scope, hysteresis, and next-step admission checks. Avoid treating a sampled free-byte value as an atomic reservation or a guarantee against every CUDA OOM.

Part 2 - System and Memory Mental Model

Application live KV, pool-backed capacity, PyTorch allocated memory, allocator-reserved memory, and driver headroom describe different layers. Offloading can release logical capacity while cached reservations remain. A watermark must name its accounting scope and include staging, provisional copies, output workspace, and pending sources. Other device users and allocator constraints can change real headroom between observation and allocation. The engine can strictly enforce only the resources it actually owns and admits.

Part 3 - Quantitative Prediction Before Execution

Predict the next token quantum's growth and maximum simultaneous transition capacity. Define high and low thresholds in bytes, with a margin and a bounded observation age. Hand-simulate a growth trace that triggers early offload, then stabilizes without repeated promotion/demotion. Construct a transient copy peak that a steady-state-only watermark misses. Leave unsupported or stale headroom evidence unknown rather than silently treating it as zero.

Part 4 - Experimental Design and Instrumentation

Use a small application budget to trigger pressure deterministically before real device exhaustion. Separate a known-valid engine budget from advisory driver observations. Test growth spikes, delayed writeback, retained free pool capacity, and fluctuating synthetic external pressure in a clearly labeled model. Apply actions at coherent operation boundaries and reserve complete transition demand before dispatch. Compare fixed residency, reactive high-water response, and hysteresis with matching useful traces. Record rejected actions and the dependencies that prevent immediate reclamation.

Keep a watermark ledger with measurement layer, timestamp, support status, and proposed capacity relief. Do not subtract planned evictions from current usage until their sources are actually reclaimable. Include retained free pool payload in a backed-capacity scope and distinguish it from driver-visible free bytes. Use an explicit unknown state when a counter is unsupported or too old for the action.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class KVWatermarkEvidence:
    scope: str
    observed_bytes: object
    sample_age_seconds: float
    high_bytes: int
    low_bytes: int
    next_step_demand_bytes: int


def choose_watermark_action(evidence, ownership_state, budgets):
    # Validate scope/age and account for growth and transition headroom.
    ...


def admit_offload_transition(action, pending_operations):
    # Ensure the proposed relief does not exceed capacity while in flight.
    ...


def E12(config):
    # Replay growth and delayed reclamation without uncontrolled OOM probing.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report thresholds, observed scope, sample age, predicted demand, actual peaks, action count, stalls, and admission failures. A policy must not claim recovered bytes until ownership truly becomes reusable. Record backend OOM separately if encountered; a watermark test can succeed without producing one. Explain any remaining gap between an enforced payload budget and total device usage rather than calling the controller universally OOM-proof.

Validate a sawtooth growth trace and a stable near-threshold trace. Excessive promotion/demotion indicates weak hysteresis or inaccurate demand prediction. A reserved-memory value can stay high after logical pressure falls, so a controller using it needs a justified policy rather than endlessly repeating offload. Report no universal protection against unrelated processes changing the device's available headroom.

Part 7 - Engineering Interpretation and Philosophy

Explain which warning is actionable and which uncertainty remains outside the engine's control. Choose margins and hysteresis from tested growth and transition behavior, while preserving honest limits on prediction under other processes, changing workspaces, or unsupported allocator fields.

19.37.14 Exercise 13

Exercise Title: Implement Hot and Cold KV Block Classification from Access Frequency, Layer Reuse Timing, Sequence Activity, and Reload Cost

Part 1 - Exercise Contract and Why It Matters

Classify KV blocks using declared access, reuse, activity, and reload-cost signals. Deliver a score with evidence age and uncertainty, then test whether it predicts useful residency decisions. Do not confuse cold placement priority with permission to delete required full-context history.

Part 2 - System and Memory Mental Model

For an active full-attention request, historical blocks are usually consumed at each relevant layer on every decode step. Old tokens are not automatically cold merely because they were created long ago. Inactive requests, reusable prefixes, different next-use timing, and scheduling gaps can create residency opportunities. A host method call count may not equal actual attention reads. Access instrumentation must describe which events it observes and how they relate to the consumer.

Part 3 - Quantitative Prediction Before Execution

Define score inputs with units: expected next-use delay, bytes, reload time, observed uses, active-request status, and uncertainty. Predict classifications for an active long history, an inactive retained prefix, and a soon-resuming request. Construct a trace where LRU ranks required old blocks poorly. Calculate the consequences of a false-hot and false-cold prediction without altering attention semantics.

Part 4 - Experimental Design and Instrumentation

Use fixed traces with known future accesses for diagnostic evaluation, but keep the online policy limited to information actually available at decision time. Compare recency, activity-only, and a simple cost-aware score. Record observed reads separately from predicted reuse. Include a homogeneous active decode control where block frequencies are nearly uniform. Measure instrumentation overhead and bound logs. Keep model outputs fixed across classifications; the policy chooses placement only and every required block remains retrievable.

Define observed-use events at the consumer boundary rather than counting only translation calls. One grouped attention operation can read many blocks, while repeated lookup requests may share one pending restoration. Record sequence activity and scheduled next use independently. A hypothetical future trace may evaluate prediction quality, but its oracle timing must remain unavailable to the online classifier.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class KVHeatEvidence:
    block_identity: tuple
    active_owner_count: int
    observed_uses: int
    predicted_next_use_seconds: object
    reload_seconds: object
    sample_age_seconds: float
```

```
def classify_kv_blocks(evidence_rows, policy, support_contract):
    # Preserve unknowns and rank placement without dropping required history.
    ...


def evaluate_heat_predictions(predictions, later_accesses):
    # Separate predictive quality from complete residency-policy usefulness.
    ...


def E13(config):
    # Test controlled hot/cold examples and full-context counterexamples.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report score inputs, classifications, later uses, false predictions, transfer consequences, and complete useful work. A correlation between recency and reuse is not a universal cache law. Mark unavailable timestamps or access counts explicitly. Demonstrate that a semantic attention-importance score is a different mechanism from lossless residency ranking and is outside this chapter unless separately implemented as a changed task.

Use two old blocks with different request activity and two active blocks with equal reuse frequency. Explain why age alone misses the first distinction and provides little information for the second. Evaluate classifications through actual residency outcomes and full-context correctness. A false-cold page must be restored, not omitted from attention to make the classifier's performance appear better.

Part 7 - Engineering Interpretation and Philosophy

Choose the smallest classifier justified by the tested scheduling pattern. Explain whether genuine reuse differences exist in the workload and when uniform full-context accesses leave little opportunity for block-level hot/cold optimization beyond scheduling and transfer overlap.

19.37.15 Exercise 14

Exercise Title: Keep Hot KV Blocks in VRAM and Demote Cold Blocks to CPU While Maintaining a Strict GPU KV Cache Budget

Part 1 - Exercise Contract and Why It Matters

Keep preferred blocks on GPU and demote lower-priority blocks to CPU while enforcing a strict named GPU KV payload budget. Preserve lossless full-context attention through explicit restore or a validated streaming consumer. Deliver budget audits covering transitions, staging, and pending users.

Part 2 - System and Memory Mental Model

A block can be absent from VRAM yet still required by the next attention. The consumer must restore it or process it through a correct blockwise method. If attention gathers the complete history into a dense CUDA tensor, that tensor counts toward live KV payload and may exceed the budget. Splitting attention requires global softmax normalization across blocks,

not averaging independent block outputs. A budget below the consumer's minimum feasible working set requires explicit rejection or a different validated consumer.

Part 3 - Quantitative Prediction Before Execution

Predict resident hot bytes, demanded cold bytes, staging capacity, outgoing sources, and provisional copies for a short trace. Calculate peak concurrent ownership rather than subtracting planned evictions immediately. Include a case where the full dense history cannot fit even after all cold blocks are demoted. State whether the strict limit covers valid bytes or allocated payload capacity; the primary contract must charge actual live slot capacity.

Part 4 - Experimental Design and Instrumentation

Start with the validated layer consumer and budgets large enough for its working set. Add block classification and controlled demotion without changing values or masks. Test delayed eviction, demand arrival during reload, unavailable victims, and a too-small budget that must be rejected. An optional streamed attention extension needs independent numerical checks for global normalization, positions, and GQA. Track CPU backing and pinned staging separately. Keep a scalar transition ledger that can be audited independently of policy counters.

Charge all allocated GPU KV slot backing to the strict payload-capacity budget, including free retained pool slots, staging, and independent gather outputs. Views of the same backing contribute once. Moving a mapping to CPU does not reduce backed capacity while its GPU tensor remains retained. Either reuse that fixed capacity within the bound or explicitly destroy eligible independent backing under a separate release policy.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class KVCapacityState:
    free_retained_capacity_bytes: int
    resident_capacity_bytes: int
    staging_capacity_bytes: int
    pending_source_bytes: int
    provisional_bytes: int
    budget_bytes: int


def admit_kv_residency(plan, state, consumer_requirements):
    # Count unique live backing and reject impossible working sets.
    ...


def demote_cold_blocks(engine, candidates, completion_evidence):
    # Preserve authority and defer capacity relief until safe reclamation.
    ...
```

```
def E14(config):
    # Validate strict capacity and full-context outputs through all transitions.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require no stable or transitional state above the declared live-capacity bound. Report unique backing so a staging view is not double-counted, while independent duplicates are not omitted. Record payload budget, total backend peaks, transfer volume, stalls, and rejected operations separately. A correct budget rejection is preferable to claiming that demotion alone permits an attention consumer whose dense inputs cannot fit.

Test a demand whose pages individually fit but whose full dense materialization would exceed the limit. Require early rejection or the separately validated streaming consumer. For blockwise attention, global normalization and mask positions must match the dense reference. Report total backend reserved memory beside the enforced payload bound so the reader can audit the boundary of the guarantee.

Part 7 - Engineering Interpretation and Philosophy

Explain the budget actually guaranteed and the consumer assumptions behind it. Distinguish lossless movement from context pruning, and state when a streamed attention implementation is required to make a smaller working-set contract achievable.

19.37.16 Exercise 15

Exercise Title: Combine Paged KV Cache with CPU Offloading So Individual Physical KV Blocks Can Be Page-In or Page-Out Independently

Part 1 - Exercise Contract and Why It Matters

Combine paged KV addressing with independent CPU/GPU migration of individual layer blocks. Preserve logical token order, shared-prefix ownership, copy-on-write, and valid versions through page-in and page-out. Make the migration unit explicit when adapting the all-layer bundle representation from the earlier paged-cache chapter.

Part 2 - System and Memory Mental Model

For layer-wise transfer, a page can own K and V for one layer and a fixed token interval. Its identity includes request-sharing group, layer, logical range, and version. An earlier table entry covering all layers must be refined or linked to per-layer residency records; it cannot silently become a one-layer page. Physical means application-owned payload, not a hardware frame. Shared mappings reference one authoritative page version, while residency and active transfer ownership are tracked separately.

Part 3 - Quantitative Prediction Before Execution

Derive per-layer page capacity from token capacity, KV heads, head dimension, and dtype. Calculate touched pages for a range spanning a partial tail, and predict migration bytes versus useful requested elements. Include shared references, clean CPU backing, dirty tail writeback, and provisional private copies. Count duplicate page-in requests separately from actual unique transfers. Predict a full-pool miss with no safely reclaimable victim.

Part 4 - Experimental Design and Instrumentation

Adapt checked logical tables and generation-aware pools rather than bypassing them with raw tensor pointers. Start with synchronous migration and a dense consumer whose working set fits, then use the established stream protocol. Test partial tails, cross-block reads, same-page demand from two requests, shared-prefix destruction, copy-on-write, and stale transfer completion after slot reuse. Compare every restored layer history and decoder output with a resident reference. Bound CPU backing, staging, outstanding operations, and GPU payload separately.

Record the precise page unit in bytes and tokens for each layer. If the previous chapter used all-layer bundles, preserve a parent logical identity while adding checked per-layer backing records. Test two requests sharing one layer page and ensure only one restoration of the required version is scheduled. A page-in operation must not inflate reference ownership merely because multiple requesters await it.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class LayerKVPage:
    layer_id: int
    logical_block: int
    valid_tokens: int
    version: int
    gpu_handle: object
    cpu_backing: object
    residency_state: str

class PagedKVOffloader:
    def ensure_resident(self, page_identity, consumer):
        # Deduplicate restoration and return exact-version readiness.
        ...

    def page_out(self, page_identity):
        # Preserve authoritative values and active/shared ownership.
        ...

    def publish_mapping(self, operation, completion_evidence):
        # Validate page and slot generations before publishing residency.
        ...


def E15(config):
    # Validate paged translation and independent layer-page migration.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Audit mappings, references, versions, residency, transfer state, and capacity independently. Report page-in/out bytes, duplicated backing, pending ownership, cache misses, stalls, and full decode time. A simulated page fault is a software residency miss, not an illegal device pointer access. Rejected demand must remain explicit when the working set is infeasible. Do not claim a transparent general tensor or optimized paged-attention backend from this restricted migration interface.

Cancel one requester while another still needs the shared restoration. The remaining requester must receive correct data and the cancelled one must not publish a stale mapping later. Invalidate any GPU handle before recycling its slot, preserve valid CPU backing, and check partial-tail boundaries after reload. Track per-layer migration bytes independently from global logical token capacity.

Part 7 - Engineering Interpretation and Philosophy

Explain which granularity improves placement flexibility and which overheads it adds. State whether the actual consumer benefits from page-level residency or still reconstructs full layer histories, and use that distinction to frame the measured capacity and performance results.

19.37.17 Exercise 16

Exercise Title: Detect KV Cache Thrashing When Frequently Reused Blocks Are Repeatedly Offloaded and Reloaded Across PCIe

Part 1 - Exercise Contract and Why It Matters

Detect repeatedly moved KV working sets and distinguish unavoidable offload traffic from avoidable policy churn. Deliver a diagnostic that relates repeated migration to exposed stalls and complete useful-work degradation. A large transfer count alone is not sufficient to diagnose an incorrect residency policy.

Part 2 - System and Memory Mental Model

Full-context decoding can require every offloaded layer history again at the next step. Repeated reload is then inherent to the chosen capacity and consumer contract. Additional promotion/demotion before any useful use, misleading prefetch, or preventable eviction of soon-needed pages can be avoidable churn. A detector needs a named comparison policy and a time window. Fault count, bytes moved, stall duration, and missed throughput are related but nonidentical signals.

Part 3 - Quantitative Prediction Before Execution

Construct traces for mandatory layer cycling, a fitting working set with avoidable eviction, and a working set just larger than capacity. Predict reloads per useful access, transfers that produce no use, dirty writeback, and exposed wait intervals. Define ratios with explicit denominators and leave empty cases undefined. Calculate whether a larger retained set or a smaller admitted batch could remove the repeated capacity conflict without changing required attention history.

Part 4 - Experimental Design and Instrumentation

Replay deterministic access traces through demand-only, simple recency, and bounded lookahead policies. Keep model/token semantics and total resource scopes fixed. Record page version and generation so repeated requests for one pending transfer are not counted as multiple completed migrations. Use a counterfactual feasible policy or controlled capacity change to

test attribution. Measure diagnostics overhead separately and retain bounded scalar windows. Validate numerical outputs before comparing transfer and latency patterns.

Label a movement cycle by whether the page was usefully consumed between reload and eviction. This helps separate wasted speculation from mandatory full-context traffic. Compare the same trace with enough capacity to hold its working set and with one fewer slot. Keep the model's required history fixed in both cases, even when the constrained run becomes substantially slower.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class KVMovementWindow:
    useful_accesses: int
    reload_bytes: int
    writeback_bytes: int
    unused_prefetch_bytes: int
    exposed_wait_seconds: float
    comparison_policy: str


def detect_kv_thrash(window, trace_features, support_contract):
    # Separate mandatory traffic, avoidable churn, and unknown attribution.
    ...


def validate_thrash_diagnosis(trace, controlled_replays):
    # Test a specific mechanism without silently changing the computation.
    ...


def E16(config):
    # Compare migration patterns with useful work and latency consequences.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report raw movement events and ratios beside the diagnostic label. Include a mandatory round-trip trace that should not be called an avoidable-policy failure and a deliberately poor policy that should be detected. A correlation between reloads and latency does not establish which decision caused the stalls. Preserve unknown attribution when no controlled comparison supports a stronger claim.

Use a controlled policy change to test whether the detector's proposed remedy reduces exposed waits. A larger hit rate can still lose if the policy retains many small cheap pages and repeatedly reloads one large expensive page. Preserve raw bytes, event counts, and latency evidence rather than collapsing every pattern into one unqualified thrashing percentage.

Part 7 - Engineering Interpretation and Philosophy

Recommend an actionable response only when its mechanism is supported: retain a tested working set, reduce speculative movement, alter scheduling, or admit less simultaneous work. Explain when the observed cost is the unavoidable price of the chosen capacity target and consumer.

19.37.18 Exercise 17

Exercise Title: Build a Cost-Aware KV Residency Policy That Balances Available VRAM, Block Size, Expected Reuse, PCIe Bandwidth, and GPU Stall Time

Part 1 - Exercise Contract and Why It Matters

Build a cost-aware residency policy combining constrained GPU capacity, page size, predicted reuse, measured transfer cost, and exposed GPU wait. Compare it with simple baselines under equal lossless inference semantics. Deliver a decision ledger and held-out evaluation rather than claiming a globally optimal policy from one trace.

Part 2 - System and Memory Mental Model

A retention decision saves future reload only if the page is actually reused while resident. Dirty eviction can add writeback and transition headroom. Available overlap changes exposed cost, while shared pages serve multiple owners. Capacity is discrete and current leases can make an attractive victim temporarily unavailable. Future use predictions have uncertainty; offline knowledge may provide a diagnostic bound but must not leak into the online policy's inputs.

Part 3 - Quantitative Prediction Before Execution

Estimate transfer time as measured fixed overhead plus bytes divided by effective bandwidth, then estimate its exposed fraction under the supported schedule. Define retention value per constrained byte using expected reuse and avoided cost, with explicit uncertainty. Account for dirty writeback, shared benefit, and transition feasibility. Construct an example where minimizing bytes moved differs from minimizing latency and another where maximizing hit rate chooses the wrong resident set.

Part 4 - Experimental Design and Instrumentation

Calibrate copy and compute behavior on separate bounded cases, including concurrent H2D/D2H if the engine uses them. Use a small auditable candidate set and feasibility gates before scoring. Compare static layer residency, activity-based placement, recency, and the cost-aware policy on held-out traces. Add hysteresis or minimum residence only when justified by observed churn. Preserve request fairness and output semantics. Include calibration, monitoring, scheduling, and transition costs in complete-run results.

Use measured transfer-size curves instead of a single bandwidth constant when fixed overhead dominates small pages. Include the effect of simultaneous H2D/D2H and pinned staging limits where the schedule permits concurrency. Score only actions whose ownership and transition requirements are currently feasible. Reserve a known-valid fallback so inaccurate predictions cannot force an unsupported mutation or loss of history.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class KVResidencyDecision:
    page_identity: tuple
    action: str
    predicted_saved_stall_seconds: object
    required_capacity_bytes: int
    uncertainty: object
    reason: str


def score_residency_candidates(evidence, cost_model, budgets):
    # Reject infeasible actions before comparing uncertain expected value.
    ...


def choose_residency_actions(candidates, ownership_state):
    # Preserve demand readiness, fairness, and bounded transition ownership.
    ...


def E17(config):
    # Evaluate held-out useful work and explain every selected action.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report predictions, actual later reuse, transferred bytes, stalls, resource peaks, action count, and complete latency. Show sensitivity to inaccurate bandwidth or next-use predictions. A policy that improves an isolated copy metric but worsens decode or starvation must report that outcome. Keep model fit quality distinct from causal proof that a particular decision saved time. Unsupported measurements remain unknown.

Evaluate a stable trace, a phase change, and misleading reuse predictions. Report policy overhead, oscillation, demand waits, and request fairness. A future-aware oracle may establish a diagnostic comparison, but the online policy must use only available evidence. Separate improvements in a fitted cost estimate from improvements in measured completed decode work.

Part 7 - Engineering Interpretation and Philosophy

Choose the simplest policy that improves complete useful work within tested budgets. Explain whether complexity is justified by real heterogeneity and reuse differences, and identify the next measurement needed before extending the policy to another architecture, link, or concurrency pattern.

19.37.19 Exercise 18

Exercise Title: Stress-Test KV Offloading with Long Contexts, Concurrent Sequences, Variable Generation Lengths, Shared Prefixes, and Artificially Restricted VRAM

Part 1 - Exercise Contract and Why It Matters

Stress-test lossless KV offloading under bounded long contexts, concurrent requests, variable completion, shared prefixes, and a deliberately small engine VRAM budget. Find minimal reproducible failures in semantic outputs, version authority, asynchronous lifetime, or resource accounting. Avoid uncontrolled machine-wide exhaustion.

Part 2 - System and Memory Mental Model

Difficult failures combine valid transitions: a shared page is copied while its request ends, a CPU buffer is reused before H2D finishes, or an old D2H completion overwrites a newer backing version. Correct logits alone cannot prove capacity compliance, and correct byte totals cannot prove masking or request isolation. Maintain independent semantic, ownership, residency, and budget oracles. Restrict pressure through admission limits rather than consuming all physical VRAM.

Part 3 - Quantitative Prediction Before Execution

Define invariants and worst-case transition demand before random execution. Predict a trace where no victim is currently safe and the correct response is waiting or rejection. Include long contexts relative to the bounded test model, many partial pages, and shared-tail divergence. State progress assumptions: pending work eventually completes, and the admitted working set is feasible. No policy must promise progress when the consumer's minimum working set exceeds its declared capacity.

Part 4 - Experimental Design and Instrumentation

Use fixed seeds, trace IDs, finite arrivals, explicit output lengths, maximum host backing, pinned capacity, GPU payload, and pending-operation limits. Begin with deterministic completion simulation, then run supported CUDA traces. Inject failures before named publication points, not illegal pointer races. Compare selected cache and logit checkpoints with a resident reference on matching smaller cases. Audit every transition and final drain. Save failing histories and reduce them while preserving sharing, allocation, and request creation dependencies.

Save exact scheduler decisions and completion ordering along with random seeds. Real asynchronous timing may not reproduce from a seed alone, so use deterministic transition simulation to shrink state-machine failures. Keep expected rejection controls separate from valid traces that expose bugs. An intentionally too-small working set budget should produce a clear unsupported or inadmissible outcome, not a false correctness failure.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class KVOffloadStressBounds:
    max_requests: int
    max_context_tokens: int
    gpu_payload_bytes: int
    cpu_backing_bytes: int
    pinned_bytes: int
```

```
    max_pending_operations: int
    steps: int


def generate_offload_trace(seed, bounds):
    # Produce bounded valid operations and labeled rejection controls.
    ...


def audit_offload_state(engine, semantic_oracle, lifecycle_oracle):
    # Check values, masks, versions, ownership, and every resource scope.
    ...


def minimize_offload_failure(trace, replay):
    # Preserve setup and completion dependencies while reducing the failure.
    ...


def E18(config):
    # Validate deterministic interleavings before supported CUDA stress.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report seeds, traces, failure phases, last valid states, reproducibility, and smallest counterexamples. Include stale CPU versions, duplicate reload, delayed writeback, cancellation, slot-generation reuse, pinned-buffer reuse, and infeasible working sets. Separate simulator-only and actual CUDA outcomes. A failure that does not reproduce remains an outcome; it is not permission to invent evidence or silently claim the bug is fixed.

Audit after final drain and after controlled cancellation, including CPU backing versions, pinned buffers, retained shared pages, and pending events. A process that exits early can hide delayed corruption. When minimizing a failure, retain the prefix-population and allocation dependencies needed to reproduce it. State when no counterexample was observed rather than claiming a proof over every interleaving.

Part 7 - Engineering Interpretation and Philosophy

State which contracts survived the tested envelope and which concurrency or model patterns remain unsupported. Select the next smallest experiment that addresses the most consequential unresolved risk. A clean random run supports a bounded claim, not correctness under every interleaving.

19.37.20 Exercise 19

Exercise Title: Build and Benchmark a PyTorch Memory-Aware KV Cache Offload Engine That Dynamically Pages Hot and Cold Cache Blocks Between VRAM and RAM While Minimizing Decode Stalls

Part 1 - Exercise Contract and Why It Matters

Integrate a memory-aware KV offload engine with explicit CPU backing, versioned residency, page pools, bounded staging, stream/event coordination, pressure monitoring, and cost-aware placement. Benchmark correct concurrent autoregressive work while minimizing measured exposed stalls under declared budgets. All substantive implementation remains learner work.

Part 2 - System and Memory Mental Model

One authority must coordinate request identities, layer/token validity, shared page versions, CPU backing, GPU slots, copy-on-write, pending readers, and transfer completion. Hot/cold classification changes placement, not the required full-context computation. The selected attention consumer determines the minimum feasible working set and must include gather or streaming costs. Weights remain fixed for the primary comparison. A software engine does not provide transparent hardware paging or universal OOM prevention.

Part 3 - Quantitative Prediction Before Execution

Predict useful KV, GPU slot capacity, CPU backing, pinned staging, provisional duplicates, attention temporaries, and transfer bytes for each trace. Compare resident, serialized layer offload, overlapped offload, selective residency, and paged cost-aware policies on equal supported workloads. Estimate prefill, time-to-first-token, warmed decode, and request completion separately. Identify a capacity-limited case offloading can admit and a bandwidth-limited case where fully resident execution should win.

Part 4 - Experimental Design and Instrumentation

Freeze the model and operation support matrix. Validate copy exactness, attention/logit parity, lifecycle oracles, and budget checks before performance. Evaluate held-out sequential, mixed-length, shared-prefix, bursty, and high-churn traffic. Record hardware, link topology, software versions, model context, dtype, masks, positions, budgets, page units, seeds, and initial state. Include cold setup, calibration, monitoring, transitions, and drain in complete cost. Run ablations for dirty-range writeback, pinning, overlap, classification, and cost-aware placement without weakening correctness or fairness gates.

Publish a support matrix for prefill, append, page migration, sharing, copy-on-write, truncation, cancellation, and shutdown before benchmarking. State the minimum consumer working set and every resource scope. Use scalar evidence IDs to connect traces, copy validation, policy actions, and outputs. Bound logs and release diagnostic payloads so monitoring cannot become the engine's largest unreported retained allocation.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class KVOffloadEngineContract:
    model_context: tuple
    gpu_payload_bytes: int
    cpu_backing_bytes: int
    pinned_staging_bytes: int
    max_pending_transfers: int
```

```
    attention_consumer: str
    supported_operations: tuple

class MemoryAwareKVOffloadEngine:
    def __init__(self, contract):
        # Establish authority, pools, backing, streams, and bounded policy state.
        ...

    def submit(self, request):
        # Validate semantics and queue or admit within complete resource limits.
        ...

    def step(self):
        # Plan residency, order copies/compute, and commit coherent token state.
        ...

    def finish(self, request, reason):
        # Retire mappings while protecting shared and in-flight obligations.
        ...

    def audit(self):
        # Recompute exact-version authority and unique resource ownership.
        ...

    def shutdown(self):
        # Drain or safely cancel work and report unreclaimed obligations.
        ...


def E19(config):
    # Validate the integrated contract before held-out useful-work benchmarks.
    ...
    # Export evidence, counterexamples, limits, and engineering recommendation.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Acceptance requires preserved valid KV, numerical output parity under the stated policy, correct masks/positions, no unsafe reuse, bounded resources, and honest useful-work accounting. Report completed requests, processed prompt positions, generated tokens, latency distributions, rejected/waiting work, transfer bytes, exposed stalls, residency peaks, host capacity, and backend allocated/reserved peaks. Keep predictions, observations, and interpretations separate. An unfinished scaffold supplies no hardware benchmark results.

Include an unfavorable offload case, a strict-budget rejection, a delayed writeback, a shared-page cancellation, and a long growth phase. Report unfinished and rejected requests as well as successful survivors. Distinguish actual CUDA execution from simulation and proposed extensions. The final recommendation must connect correct useful inference to measured capacity relief and its complete transfer, host-memory, and latency cost.

Part 7 - Engineering Interpretation and Philosophy

Recommend the smallest engine justified by complete measured work, including no offload when the resident baseline wins. Explain the capacity gained and the bandwidth, host memory, or latency paid. Separate delivered supported behavior from proposed kernel, concurrency, quantization, or distributed extensions.

19.38 DMM36

Chapter Title: KV Cache Quantization
Specialization: Dynamic Memory Management and CUDA Systems

19.38.1 Exercise 00

Exercise Title: Build a Precise Mental Model of KV Cache Precision, Quantization Error, Compression Ratio, and Decode-Time Memory Trade-Offs

Part 1 - Exercise Contract and Why It Matters

Define a precise contract for compressing stored attention keys and values while retaining the same token history. Deliver a representation diagram, a byte ledger, and a numerical evaluation plan. Separate lossless integer packing from lossy floating-to-integer quantization, and distinguish compressed storage capacity from the working memory required to consume that storage.

Part 2 - System and Memory Mental Model

The cache contains processed-token K and V at each attention layer, not queries or merely sampled token IDs. Quantization stores codes plus parameters needed to reconstruct approximate values. Keys affect attention scores and values affect weighted outputs, so reconstruction error alone does not establish model quality. A reference consumer can unpack and dequantize before floating attention, creating temporary full-width arrays. This prototype does not imply a fused low-bit attention kernel or universal speedup.

Part 3 - Quantitative Prediction Before Execution

For homogeneous layers, start from useful unquantized KV bytes

$$B_{\mathrm{fp}} = 2LH_{kv}Ds\sum_{j} T_j.$$

Predict code bytes, scales, zero points, padding, recent high-precision values, tables, and dequantization workspace separately. Define compression ratio as named baseline storage divided by complete compared storage. Leave empty denominators undefined. Calculate how an ideal bit-width ratio can overstate usable capacity when metadata and transient float reconstruction remain live.

Part 4 - Experimental Design and Instrumentation

Use a tiny explicit full-causal decoder, fixed weights, zero dropout, no gradients, and teacher-forced token traces. Keep model arithmetic fixed while varying stored KV precision. Declare layout, KV head mapping, scale axes, rounding, clipping, dtype, positional-key convention, masks, and valid lengths. Record software, device, allocator backend, seeds, model context, and

initial state. Use bounded synthetic tensors before integrating model execution. Quantization is intentionally lossy; context pruning and model-weight compression are separate variables.

Draw the lifetime of original KV, codes, metadata, and reconstructed inputs for one attention call. Mark which allocations coexist and which can actually be released. Define the stored representation independently from the arithmetic dtype used by projections and softmax. A storage experiment that also changes weights or attention arithmetic is a different comparison. Include one complete format identity covering grouping, code mapping, parameter dtype, layout, valid length, and positional convention.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class KVQuantizationContract:
    code_bits: int
    scheme: str
    reduction_axes: tuple
    group_shape: tuple
    scale_dtype: str
    position_policy: str
    quality_thresholds: tuple


def predict_quantized_kv(contract, architecture, trace):
    # Separate codes, metadata, residuals, padding, and consumer workspace.
    ...


def E00(config):
    # Define representation and quality claims before implementing compression.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require separate representation-correctness, arithmetic-parity, model-quality, and resource checks. Preserve prediction, observation, and interpretation as distinct fields. A matching greedy token does not prove matching logits or acceptable long-context quality. Export bounded CPU scalar evidence without retaining full GPU references or original caches that invalidate memory claims. Unperformed hardware experiments remain explicitly unmeasured.

Define quality at three levels: local reconstructed values, fixed-input attention behavior, and integrated decoder behavior on held-out data. State which level a reported metric supports. A tiny synthetic decoder can expose implementation bugs but cannot establish quality for a production model. Research methods can motivate experiments, but their published numerical gains are not results of this learner implementation. Preserve every untested model or hardware condition as an explicit gap.

Part 7 - Engineering Interpretation and Philosophy

Explain the resource purchased by compression and the numerical risk it introduces. The useful outcome is a measured capacity-quality-latency trade-off within a declared model and workload envelope, not the assumption that fewer stored bits automatically improve complete inference.

19.38.2 Exercise 01

Exercise Title: Measure KV Cache Memory in float32, float16, and bfloat16 for the Same Transformer Configuration and Context Length

Part 1 - Exercise Contract and Why It Matters

Measure cache storage for float32, float16, and bfloat16 under identical architecture and logical token lengths. Distinguish element-width savings from numerical range and precision differences. Produce an exact useful-payload calculation and separately measured backing and backend memory observations.

Part 2 - System and Memory Mental Model

float16 and bfloat16 both use two bytes per element but represent numbers differently. float32 uses four bytes. Identical shape and element width do not imply equal numerical behavior or backend support. Keep model weights and attention arithmetic fixed in the storage-only comparison. A dtype conversion can temporarily retain source and destination. Views can share storage, and static reservation can exceed valid token payload. None of these extra costs is represented by element count alone.

Part 3 - Quantitative Prediction Before Execution

Calculate per-token and total useful K/V bytes using KV heads rather than query heads under GQA. Predict ratios for the three storage dtypes and the peak during conversion when both copies coexist. Include ragged lengths, empty history, and static capacity larger than valid lengths. State whether measurements cover only cache payload, unique backing, or total process/device residency, and do not substitute one scope for another.

Part 4 - Experimental Design and Instrumentation

Create bounded coordinate-coded cache tensors with identical valid shapes. Compare storage allocation first, then a separately labeled numerical experiment that stores and reconstructs values in each dtype. Verify hardware and attention backend support before CUDA arithmetic. Reset state and peaks between cases, separate construction from warmed execution, and record conversion temporaries. Test representable finite values, small magnitudes, and large finite ranges without claiming every dtype can hold them equally. Keep alias ownership and padding explicit in the byte ledger.

For storage comparisons, allocate each dtype in an independent replay so source arrays from another case do not inflate the measured peak. If conversion itself is the experiment, retain and charge both sides deliberately. Include views with nonzero storage offsets and count their backing once. Report the valid tensor payload beside maximum reserved cache capacity. Identical useful shapes do not imply identical retained storage when one layout uses padding or a larger base allocation.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class KVDtypeCase:
    storage_dtype: object
    logical_lengths: tuple
    capacity_tokens: tuple
    architecture_identity: tuple


def predict_dtype_payload(case, layer_specs):
    # Validate element widths and distinguish useful values from capacity.
    ...


def measure_dtype_storage(case, measurement_contract):
    # Count unique backing and conversion peaks under explicit boundaries.
    ...


def E01(config):
    # Compare storage costs and separately report numerical/support differences.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report shapes, element sizes, valid lengths, backing capacity, conversion peaks, and allocated/reserved observations. Require the predicted useful bytes to match the supported tensor representation, while explaining backend rounding or retention separately. If bfloat16 arithmetic is unsupported in the tested path, retain its storage calculation but mark execution unsupported. Do not report a numerical result from an unrun fallback.

Use range-sensitive values only in the numerical comparison, not as evidence that byte counts changed. Compare actual element widths rather than inferring them from dtype names. Where arithmetic support differs, distinguish a storage-only pass from an executed attention pass. A bfloat16 storage estimate can remain valid even when the chosen device/backend cannot run the desired consumer; mark that consumer result unsupported rather than substituting a different precision without labeling it.

Part 7 - Engineering Interpretation and Philosophy

Explain why equal storage cost is not interchangeable numerical precision. Choose a justified cache baseline for the remaining exercises and preserve the fixed arithmetic policy so later changes can be attributed to storage quantization.

19.38.3 Exercise 02

Exercise Title: Implement a Baseline float16 KV Cache in PyTorch and Record Peak VRAM, Decode Latency, and Output Logits

Part 1 - Exercise Contract and Why It Matters

Build an explicit float16 KV reference and record complete peak VRAM, decode latency, and selected output logits. Establish semantic correctness before using this cache as the comparison for lossy compression. Preserve a repeatable teacher-forced trace and a documented attention consumer.

Part 2 - System and Memory Mental Model

Choose a bounded static or clearly labeled dynamic growth policy. Valid token length differs from allocated capacity, and per-layer prepared new KV differs from globally committed request progress. Attention must see the correct causal history with consistent GQA heads, positions, and positional-key convention. float16 cache storage does not require changing all model arithmetic to float16. Store reference outputs for evaluation without retaining every live cache version in the measured GPU run.

Part 3 - Quantitative Prediction Before Execution

Predict useful float16 payload, assigned capacity, growth temporaries, masks, and attention workspace. Define prompt processing, decode tokens, and request throughput with separate numerators. State whether constructor, warm-up, sampling, validation, transfer, and final drain are included. Calculate a bounded context limit from the declared budget with headroom rather than finding it by uncontrolled allocation failure.

Part 4 - Experimental Design and Instrumentation

Use fixed weights, evaluation, explicit zero attention dropout, and a no-gradient inference boundary. Compare cached decoding with full-prefix recomputation on small cases. Include ragged requests, partial capacities, multiple layers, and query/KV head differences. Record attention backend and accumulation policy. Use deliberate completion boundaries for wall time and appropriate CUDA events for device intervals. Run detailed validation separately from primary throughput when frequent checks would synchronize every step.

Define how current-layer attention sees prepared new KV before the request's global token length is committed. A partial forward must not become a reusable complete cache snapshot. Verify a one-query decode mask against explicitly listed allowed key positions, then repeat with a multi-query suffix. The reference must establish this before compression, otherwise later numerical differences may be caused by a baseline masking error rather than quantization.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

class Float16KVReference:
    def initialize(self, architecture, capacities):
        # Establish explicit cache layout and valid-position metadata.
        ...
```

```
    def prefill(self, token_trace):
        # Preserve positions, causal masks, head mapping, and layer readiness.
        ...

    def decode_step(self, tokens):
        # Process actual tokens and commit coherent new KV across layers.
        ...


def record_reference(runtime, cases, measurement_contract):
    # Export selected outputs without contaminating measured GPU lifetimes.
    ...


def E02(config):
    # Validate and benchmark the reproducible float16 cache baseline.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require attention and logit checks within declared reference tolerances. Report processed lengths, useful and capacity bytes, allocated/reserved peaks, completed time, latency variability, and unsupported conditions. An unsynchronized host timer measures submission rather than completed decode. If greedy generation diverges later under quantization, use equal teacher-forced traces to attribute performance and separately evaluate generation quality.

Separate reference capture from measured inference. Saving all logits or all original KV on GPU can erase apparent compression savings in later tests. Use bounded CPU exports with explicit synchronization and record their cost outside or inside the declared region consistently. Count prompt-token processing, newly processed decode tokens, and final sampled outputs separately. A sampled token that is never forwarded contributes no additional cached K/V position.

Part 7 - Engineering Interpretation and Philosophy

Explain which properties of this reference are semantic requirements and which are implementation choices. A poor concatenation baseline must not make compression appear to save work that actually came from changing cache growth or attention backend.

19.38.4 Exercise 03

Exercise Title: Quantize Key and Value Tensors to int8 with a Simple Per-Tensor Scale and Measure the Resulting Memory Reduction

Part 1 - Exercise Contract and Why It Matters

Implement a simple symmetric int8 quantizer with separate per-tensor scales for K and V under a precisely defined tensor scope. Measure actual code and metadata storage after releasing measured-path source copies. Validate quantizer invariants before integrating the compressed representation into decoding.

Part 2 - System and Memory Mental Model

For finite values, choose a declared signed code range such as $[-127, 127]$, leaving the remaining int8 code unused. Scale selection, rounding, clipping, and all-zero handling are part of the format. K and V need not share their scale. A scale spanning a growing history can change when a new outlier arrives; old codes cannot simply be interpreted under a new scale. Quantization parameters must remain associated with the exact values they encode.

Part 3 - Quantitative Prediction Before Execution

With scale $a > 0$, define

$$q = \mathrm{clip}(\mathrm{round}(x/a), q_{\min}, q_{\max}), \qquad \hat{x} = aq.$$

Derive an absmax scale for the declared range and a valid all-zero convention. Predict code bytes, scale bytes, and a round-trip error bound only for unclipped finite inputs under the chosen rounding. Compare nominal float16-to-int8 payload reduction with metadata-inclusive storage and conversion peak.

Part 4 - Experimental Design and Instrumentation

Begin with fixed bounded K and V arrays, including zeros, sign changes, midpoint ties, extreme finite values, and outliers. Specify reduction axes and whether one tensor means one layer and request over a fixed token interval. Use a documented rounding policy. Reject or explicitly handle nonfinite inputs. For append integration, choose frozen calibrated scales with clipping evidence or a correctly accounted rescaling protocol; do not silently change old scales. Measure old/new coexistence and retain originals only outside the production memory path or in a separately charged reference store.

Perform scale estimation in a documented working dtype and validate that stored scales stay finite and positive where required. Small scales can underflow in a narrower metadata dtype; large intermediate values can overflow before clamping if conversion order is wrong. Apply the all-zero convention consistently on encode and decode. Reject nonfinite calibration values under the primary contract instead of allowing an invalid scale to propagate through the cache.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class Int8KVPayload:
    key_codes: object
    value_codes: object
    key_scale: object
    value_scale: object
    valid_tokens: int
    format_identity: tuple
```

```
def quantize_per_tensor(keys, values, format_contract):
    # Select valid scales and apply the declared rounding/clipping policy.
    ...


def int8_storage_ledger(payload, source_lifetimes):
    # Count codes, parameters, padding, and retained originals separately.
    ...


def E03(config):
    # Validate edge cases and truthful steady/transition memory reduction.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report code range, scale values, clipping counts, error distribution, source and destination bytes, and transition peaks. Test a new outlier appended after initial encoding and require the scale policy to preserve format validity. A float tensor containing rounded integers is not compressed int8 storage. A low code-payload total does not establish a lower device peak while full-width sources or dequantized results remain live.

Test midpoint rounding and clipping before casting to int8. An integer cast by itself does not implement the declared round-to-nearest policy and can produce a different quantizer. Compare a frozen-scale append with a full rescale only when their history handling is explicit. If rescaling uses reconstructed rather than original values, label the composed approximation and evaluate its error separately. A metadata change cannot reinterpret existing codes silently.

Part 7 - Engineering Interpretation and Philosophy

Explain what the simple quantizer loses and which uncertainty remains before model use. Keep its format auditable rather than introducing finer groups before per-tensor scale identity, clipping, and byte accounting are correct.

19.38.5 Exercise 04

Exercise Title: Dequantize int8 KV Tensors Back to Floating Point During Attention and Verify End-to-End Numerical Correctness

Part 1 - Exercise Contract and Why It Matters

Dequantize stored int8 KV for attention and validate the complete inference path. Separate correct decoding of the format from acceptable lossy model behavior. Deliver per-layer and token-position evidence for reconstructed KV, attention outputs, logits, and a declared quality metric.

Part 2 - System and Memory Mental Model

Dequantization reconstructs approximate values using the matching scale and format metadata. It does not recover the original float16 values generally. A reference path may build full floating histories before attention, adding workspace and memory traffic. Positions, valid lengths, head mapping, and masks must remain unchanged. Local replay of frozen reference KV

isolates quantizer effects; integrated decoding can produce different later KV because earlier quantized attention changed hidden states.

Part 3 - Quantitative Prediction Before Execution

Predict reconstructed shapes and dtype, code/metadata reads, floating output bytes, and temporary coexistence with compressed storage. Define allowed KV, attention, and logit errors before running comparisons. Construct negative controls for swapped K/V scales, wrong group broadcasting, stale metadata, and invalid tail inclusion. Explain why agreement of one sampled token cannot establish that the format or model-quality contract holds.

Part 4 - Experimental Design and Instrumentation

Validate dequantization on small synthetic arrays first. Then use fixed queries and reference K/V to isolate local arithmetic, followed by integrated teacher-forced decoding with quantized caches. Keep model weights, attention backend, dropout, positions, and masks fixed. Include partial tails, GQA, ragged lengths, and multi-query suffixes with explicitly verified causal visibility. Measure unpack/dequantize, attention, and complete decode separately, but include every temporary in the complete peak. Use independent held-out cases for quality after calibration choices are fixed.

Validate reconstructed K and V independently before invoking attention, including a case where their scales differ strongly. Keep reference queries fixed for the local experiment. In integrated decoding, newly computed queries and later KV can diverge because earlier approximate attention changed hidden states; this is an additional effect to evaluate, not a scale-broadcast bug by definition. Label the two experiment modes in every result row.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
import torch.nn.functional as F


def dequantize_int8_kv(payload, compute_dtype):
    # Validate metadata and reconstruct only the supported valid values.
    ...


def quantized_attention_reference(query, payload, attention_contract):
    # Preserve head mapping, masks, positions, and the floating consumer policy.
    ...


def evaluate_quantized_decode(reference, compressed, cases, quality_contract):
    # Distinguish format errors, local distortion, and integrated model effects.
    ...


def E04(config):
    # Validate correctness of the approximation, not lossless equivalence.
```

. . .

Part 6 - Measurement, Validation, and Failure Analysis

Report reconstruction errors, attention/logit deviations, clipping, quality metric, backend, dtype, elapsed time, and complete memory peak. Treat threshold failures as failures even if generation looks plausible. Compare local frozen-KV replay with integrated rollout explicitly. If a higher-precision oracle is unavailable at runtime, do not claim the engine can verify every future token's original-output error automatically.

Include a high-precision mathematical attention control for small cases, then compare the actual reference backend at its declared dtype. Validate boolean mask polarity and rectangular query alignment. If model quality is evaluated with negative log-likelihood, define target-token alignment and evaluation corpus; if using perplexity, derive it from the same token-weighted loss. Do not equate logit similarity on one prompt with a general task-quality guarantee.

Part 7 - Engineering Interpretation and Philosophy

Explain which numerical contract was satisfied and the scope of the evidence. Quantized decoding can be a correct implementation of an approximation while failing an application's accuracy requirement; those outcomes need separate labels and separate engineering decisions.

19.38.6 Exercise 05

Exercise Title: Compare Symmetric and Asymmetric KV Quantization and Measure Error Distribution in Keys, Values, and Attention Outputs

Part 1 - Exercise Contract and Why It Matters

Compare symmetric and affine asymmetric KV quantization at the same stored bit width. Measure error distributions in keys, values, attention, and logits, including metadata and calibration cost. Keep grouping, model arithmetic, inputs, and quality thresholds fixed so the comparison isolates the scheme.

Part 2 - System and Memory Mental Model

A symmetric format uses a zero-centered code interpretation and scale. An affine format adds a zero point and may use an unsigned code range. The reconstruction is $\hat{x} = a(q - z)$ for the declared scale and zero point. Different integer ranges have different endpoint conventions. Range estimation, rounding, clipping, inclusion of zero, and constant-tensor handling must be specified together. An offset distribution can favor affine placement, but that is a hypothesis to test rather than a universal property of KV.

Part 3 - Quantitative Prediction Before Execution

Derive scale and zero-point choices for an affine min/max range that includes zero, with an explicit all-zero fallback. Predict behavior for positive-only, negative-only, centered, constant, and outlier-heavy inputs. Calculate code, scale, and zero-point bytes under actual metadata dtypes. Compare expected unclipped rounding error with clipping error and stored-scale rounding; a simple half-step bound does not cover all three.

Part 4 - Experimental Design and Instrumentation

Use identical synthetic distributions and captured reference KV. Separate calibration from held-out evaluation. Compare local frozen-KV attention before integrated teacher-forced decoding. Preserve scale scope, layout, positional representation, head mapping, and backend. Report both overall and per-head error summaries so a few severe errors are not hidden by a small average. Include a new range-shifting append under the declared update policy. Keep signed int8 and unsigned affine-code interpretations explicit at every boundary.

For affine min/max calibration, explicitly include zero in the represented range and define the constant-input branch. Test zero, positive constants, negative constants, and a narrow offset range. Record actual unsigned versus signed code storage and do not subtract zero points in an unsigned dtype that can wrap. Use a sufficiently wide intermediate before converting to the requested floating reconstruction dtype.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class AffineKVParameters:
    scale: object
    zero_point: object
    code_min: int
    code_max: int
    reduction_axes: tuple


def estimate_affine_parameters(tensor, format_contract):
    # Include zero/range conventions and handle constant finite inputs.
    ...


def compare_quantization_schemes(cases, schemes, attention_contract):
    # Preserve equal bits and grouping while counting metadata and errors.
    ...


def E05(config):
    # Test distribution-dependent trade-offs and update behavior.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require correct code bounds, finite valid scales, valid zero points, and metadata broadcasting. Report clipping, bias, error quantiles, worst coordinates, attention/logit errors, memory, and time. An asymmetric scheme is not guaranteed to beat symmetric under every distribution or quality metric. Keep parameter estimation arithmetic and stored-parameter precision visible when interpreting small differences.

Compare schemes with identical calibration data and grouping, then evaluate a shifted held-out range to reveal clipping sensitivity. Report error bias and tail behavior as well as mean squared error. A stored zero point is part of the interpretation metadata and must follow the same generation as its codes and scale. A deliberate stale-zero-point control should fail even when its tensor shape still broadcasts successfully.

Part 7 - Engineering Interpretation and Philosophy

Choose a scheme from measured data and the application's quality contract. Explain whether the extra parameters buy enough error reduction to justify storage and kernel complexity, and state the conditions under which the simpler symmetric reference remains preferable.

19.38.7 Exercise 06

Exercise Title: Implement Per-Head KV Quantization and Compare It Against Per-Tensor Scaling by Reconstruction Error and Metadata Cost

Part 1 - Exercise Contract and Why It Matters

Implement per-head KV scales and compare them with the per-tensor reference. Measure reconstruction, attention distortion, metadata, and complete decode cost. Define head identity using stored KV heads rather than expanded query heads so GQA does not duplicate the quantizer's ownership.

Part 2 - System and Memory Mental Model

For a documented layout such as $[B, H_{kv}, T, D]$, a per-request per-head scale reduces over the selected token and channel dimensions without mixing requests. K and V retain independent parameters. A per-head scale over an entire growing history still has an update problem when its range changes. Finer grouping can reduce the influence of another head's outlier but cannot remove outliers inside the same group or guarantee lower downstream error.

Part 3 - Quantitative Prediction Before Execution

Predict parameter counts for one scale per tensor versus one per stored head, separately for K and V and each layer/request. Calculate metadata bytes using the actual stored scale dtype and optional zero-point format. Construct two heads with sharply different ranges and another example with equal ranges. Estimate how parameter overhead changes the effective compression ratio for short and long histories.

Part 4 - Experimental Design and Instrumentation

Implement parameter shape and broadcasting assertions before CUDA timing. Test shuffled head values, GQA with multiple query heads per KV head, empty valid histories, partial token intervals, and a new head-local outlier. Use fixed code width, rounding, clipping, and update policy across variants. Compare synthetic reconstruction, frozen-KV attention, and integrated decode on separate held-out traces. Count metadata and any expanded floating heads in the consumer peak. Measure scale estimation and update work rather than assuming per-head metadata is free.

Give different heads easily distinguished amplitudes and reverse their order in one negative control. Use a batch where requests have very different ranges so accidental reduction across the batch dimension is observable. Specify whether empty heads have no parameters or a defined neutral metadata record. A valid broadcast shape is necessary but insufficient: each parameter must describe the correct request, stored KV head, and token interval.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class HeadQuantizationLayout:
    tensor_axes: tuple
    head_axis: int
    reduced_axes: tuple
    request_scope: str
    parameter_shape: tuple


def quantize_per_head(keys, values, layout, format_contract):
    # Validate KV-head identity and retain matching independent parameters.
    ...


def compare_head_granularity(reference_cases, quantizers):
    # Count metadata and trace downstream errors, not only average MSE.
    ...


def E06(config):
    # Validate head broadcasting and measure complete granularity trade-offs.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report group count, parameter shape, code bytes, metadata, error distribution, clipping, attention/logit quality, and timing. Use a wrong-head permutation negative control and require detection. A lower KV mean-squared error can coexist with worse logits, so preserve both observations. Reject scale shapes that broadcast numerically but associate parameters with the wrong request or head.

Count scales and zero points separately for K and V at every cached layer. If metadata is stored in a padded tensor, report its capacity rather than only the number of meaningful entries. Compare short and long histories to show amortization. Keep query-head expansion out of stored-parameter counts, while including any expanded floating tensors in the attention consumer's peak. This prevents a GQA layout change from being mistaken for a quantization storage effect.

Part 7 - Engineering Interpretation and Philosophy

Explain where head-level independence helps and where its extra parameters or update work dominate. Recommend a granularity only for the tested model, length distribution, and consumer, preserving the distinction between stored heads and arithmetic head expansion.

19.38.8 Exercise 07

Exercise Title: Implement Per-Channel KV Quantization and Determine Whether Finer-Grained Scaling Improves Attention Accuracy Enough to Justify Extra Metadata

Part 1 - Exercise Contract and Why It Matters

Implement a precisely defined per-channel KV format and determine whether its downstream accuracy benefit justifies added parameters and computation. Compare with per-head and per-tensor formats under equal code width and complete resource accounting.

Part 2 - System and Memory Mental Model

The term channel must name an axis. In the primary layout, define one parameter per KV head and head-dimension coordinate over a bounded token interval, reducing over tokens and preserving channel identity. This differs from a per-token format that reduces over channels. Keys and values can have different distributions, and positional transformations can change those distributions. Keep pre- or post-positional key representation fixed rather than silently adding another optimization.

Part 3 - Quantitative Prediction Before Execution

Derive parameter shapes and counts for the selected layout and interval. Calculate code bytes, scales, zero points, and group padding. Construct a channel-local outlier case and a case where short groups make metadata dominate. Predict differences between reducing over tokens and reducing over channels. Include scale-update costs when a growing interval changes existing channel ranges; old encoded values must remain associated with their original parameters.

Part 4 - Experimental Design and Instrumentation

Use axis-labeled synthetic tensors that expose transposed parameters and head/channel swaps. Compare the same captured K/V under per-tensor, per-head, and per-channel formats, optionally adding a separately labeled per-token control. Keep calibration/evaluation separation and fixed model arithmetic. Test partial groups, empty intervals, large channel ranges, and held-out lengths. Validate reconstruction and attention before integrated decode. Record actual metadata dtypes and any layout-conversion copies required by the implementation.

Write the reduction expression in axis names and tensor indices before coding. In the primary per-channel scheme, one channel coordinate is observed across its chosen token interval within a KV head; it is not a reduction over every channel of one token. Test token/channel transposes with unequal dimensions so an incorrect implementation cannot pass because two axes happen to have the same length.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class ChannelQuantizationLayout:
    channel_axis: int
    token_axis: int
```

```
    head_axis: int
    token_group_length: int
    parameter_shape: tuple


def quantize_per_channel(tensor, layout, format_contract):
    # Reduce only declared axes and preserve parameter-to-channel identity.
    ...


def evaluate_channel_format(cases, decoder, quality_contract):
    # Compare metadata-inclusive capacity and downstream numerical effects.
    ...


def E07(config):
    # Validate axis semantics before selecting finer-grained storage.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report reduction axes, groups, parameter counts, metadata bytes, clipping, error quantiles, attention/logit metrics, complete peaks, and latency. Reject an ambiguous axis label as missing format information. Do not transfer published results for a particular model and quantization method to this simpler prototype. A measured benefit must identify the representation, grouping, and positional convention that produced it.

Keep the key positional representation fixed when comparing granularity. Pre-positional and post-positional quantization are different formats requiring different consumer work; published methods using one do not validate the other. Measure scale-update work or sealed-group conversion explicitly. Include one case where parameter overhead dominates a tiny group and another where a channel outlier motivates finer scaling, then assess downstream quality rather than assuming the synthetic advantage transfers unchanged to the model.

Part 7 - Engineering Interpretation and Philosophy

Explain whether finer scale independence improved the application's quality metric enough to pay for its metadata and execution cost. Keep K and V choices independent and resist assuming that one grouping is universally optimal for both components or every positional representation.

19.38.9 Exercise 08

Exercise Title: Sweep KV Quantization Bit Widths and Compare Estimated 16-Bit, 8-Bit, and 4-Bit Cache Capacity Under the Same VRAM Budget

Part 1 - Exercise Contract and Why It Matters

Estimate and compare 16-bit, 8-bit, and packed 4-bit cache capacity under one named GPU budget. Deliver a byte-level capacity model with metadata, tail rounding, recent full-precision storage, and consumer headroom. Distinguish analytical estimates from implemented and measured formats.

Part 2 - System and Memory Mental Model

A bit-width label describes code precision, not necessarily actual tensor storage. Four-bit values stored one per uint8 still consume eight payload bits until packing is implemented. Scales, zero points, per-group alignment, page capacity, and metadata may differ across formats. Dequantization output can dominate peak memory even when persistent codes shrink. Equal nominal bits do not establish equal model quality or maximum safe concurrency.

Part 3 - Quantitative Prediction Before Execution

For N codes at b bits, begin with ideal bytes $\lceil Nb/8 \rceil$, then apply the actual packing boundary for each independently stored group or block. Add parameter and residual bytes and reserve the selected consumer's workspace. Compute a capacity bound using identical model dimensions and ragged request traces. Show a small-block case where metadata invalidates the nominal fourfold float16-to-4-bit gain. Leave measured maximum context undefined until tested.

Part 4 - Experimental Design and Instrumentation

Use a fixed set of lengths and grouping contracts across estimates. Implement real 8-bit storage and label 4-bit as estimated until the next exercise's codec is validated. Keep allocator capacity and total device usage separate from useful code payload. Test empty objects, odd code counts, partial groups, and multiple request tails. Use a bounded admission sweep when validating estimates; never allocate until the workstation becomes unstable. Require each candidate to satisfy the same declared quality gate before ranking usable capacity.

Use per-block packing calculations when blocks are independently allocated. Rounding one global code count can undercount bytes when many odd groups each require their own padding nibble. Add residual-window capacity and metadata alignment for the actual implementation. If pool slots retain the float16 maximum size regardless of code width, record that no physical pool capacity has yet been saved. Distinguish useful code bytes from owned backing bytes.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class KVBitWidthEstimate:
    code_bits: int
    packing_scope: str
    scale_bytes: int
    zero_point_bytes: int
    group_elements: int
    residual_tokens: int
    implementation_status: str


def estimate_quantized_capacity(spec, architecture, lengths, budgets):
    # Count actual packing/metadata rules and complete consumer headroom.
    ...
```

```
def validate_capacity_bound(estimate, bounded_trials, quality_contract):
    # Separate analytical capacity from supported measured execution.
    ...


def E08(config):
    # Compare feasible quality-qualified capacities rather than bit labels.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report every storage component, assumed alignment, budget scope, headroom, quality support, and estimate status. Include counterexamples to both ideal ratio equals total savings and smallest bit width gives highest usable capacity. A format that fails quality or cannot be consumed within the budget is not a valid winner even if its codes alone are smallest.

Build a table showing estimate assumptions, implemented status, tested quality, and measured feasibility for every bit width. A hypothetical packed format may be listed as an estimate but cannot win a measured benchmark. Include consumer reconstruction headroom before converting a byte bound into maximum context or concurrency. A policy that fits persistent codes but cannot perform attention within budget must be rejected or paired with a separately validated streaming consumer.

Part 7 - Engineering Interpretation and Philosophy

Explain how storage accounting becomes a tested capacity decision. Choose which low-bit representation deserves implementation next and identify the largest uncertainty in metadata, working-set feasibility, quality, or execution cost.

19.38.10 Exercise 09

Exercise Title: Build Packed 4-Bit KV Storage in PyTorch and Verify Correct Packing and Unpacking of Quantized Values

Part 1 - Exercise Contract and Why It Matters

Build an exact nibble packing and unpacking codec for four-bit integer codes. Validate it independently from floating quantization, then connect it to a declared KV format. Demonstrate real two-codes-per-byte storage with explicit odd-length handling and group boundaries.

Part 2 - System and Memory Mental Model

The codec maps unsigned codes 0 through 15 to byte nibbles using a specified low/high order. A symmetric signed quantizer needs a separate declared code mapping, such as an offset convention; unpacking alone does not define floating values. Preserve original code count, shape, grouping, and valid tail information. Padding nibbles are not logical cache values. Avoid sign-extension, overflow, and implicit dtype behavior by validating the integer representation used for shifts and masks.

Part 3 - Quantitative Prediction Before Execution

Predict packed byte counts for zero, one, two, three, and several independently stored odd groups. Derive the byte and nibble selected by each logical code index. Specify padding

nibble contents and how they are excluded on decode. Separate the exact invariant unpack(pack(codes)) equals codes from the lossy invariant that reconstructed floats approximate their originals. Calculate complete packed KV storage including parameters and alignment.

Part 4 - Experimental Design and Instrumentation

Exhaustively test all sixteen codes and all two-code pairs on CPU before supported CUDA tests. Add random arrays, odd lengths, empty groups, noncontiguous inputs under an explicit normalization policy, and invalid codes. Test group boundaries with different scales so adjacent packed bytes cannot be decoded using another group's metadata. Validate K and V shape restoration separately. Measure packing/unpacking temporaries and actual uint8 backing bytes rather than counting a logical code list.

Specify the signed-to-unsigned code interpretation separately from nibble packing. For example, an offset-coded symmetric range may leave an unused nibble value; the codec can still test all sixteen bit patterns independently. Define byte order, low/high nibble order, group offsets, and original length in versioned metadata. Keep padding bytes deterministic in diagnostic tests so accidental tail consumption produces a recognizable error.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class PackedKV4:
    bytes_payload: object
    logical_code_count: int
    logical_shape: tuple
    group_offsets: tuple
    nibble_order: str
    quantizer_parameters: object


def pack_4bit_codes(codes, packing_contract):
    # Validate range/order and define padding for every independent group.
    ...


def unpack_4bit_codes(payload):
    # Restore exactly the valid integer codes before floating dequantization.
    ...


def E09(config):
    # Exhaustively validate the codec and audit true packed storage.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require exact integer equality and record the smallest failing code pair or boundary. Verify invalid values are rejected before silently truncating high bits. Report packed bytes, padding, metadata, temporary peaks, and latency. A correct integer codec does not prove acceptable model quality; run the separate quantization and attention validation before using packed KV in decode.

Test every adjacent two-code pair and independent odd groups, then verify that unpacking one group cannot borrow another group's padding nibble. Reject negative or oversized unsigned input codes before masking, since masking can hide invalid values. For noncontiguous inputs, count normalization copies. When integrated with KV, verify exact codes first and approximate floating values second, then assess attention quality under its separate numerical contract.

Part 7 - Engineering Interpretation and Philosophy

Explain which part of compression is lossless representation and which part is lossy approximation. Keep the packing format stable and versioned so later paging, sharing, and offloading can move bytes without guessing their numerical interpretation.

19.38.11 Exercise 10

Exercise Title: Measure Quantize and Dequantize Overhead Separately from Attention Compute and Determine When Compression Becomes Slower Than Full-Precision KV Access

Part 1 - Exercise Contract and Why It Matters

Measure quantization, packing, unpacking, dequantization, and attention costs separately, then compare complete decode against float16 storage. Identify workloads where lower persistent capacity is useful but the reference compressed consumer is slower. Do not infer a fused-kernel speedup from storage reduction.

Part 2 - System and Memory Mental Model

A PyTorch reference often materializes integer intermediates and full floating KV before attention. Quantizing history again on every append can add substantial work beyond encoding newly sealed data. Separate one-time cache conversion, new-token updates, parameter estimation, and repeated consumption. Attention backend selection or layout changes can alter arithmetic cost independently of quantization. Host submission time and completed CUDA time answer different questions.

Part 3 - Quantitative Prediction Before Execution

Predict bytes read and written by every stage and the maximum live intermediate capacity. Compare short and long histories and small and larger batches. Estimate when conversion overhead exceeds any saved memory traffic under the actual consumer, not an imagined direct low-bit kernel. Define an amortization window for one-time compression and account for final drain. Keep concurrent intervals from being summed as if they were serial.

Part 4 - Experimental Design and Instrumentation

Use identical validated code formats, queries, token traces, and model arithmetic across cases. Benchmark isolated stages with appropriate completion, then unprofiled complete decode regions. Record warm-up, repetitions, shapes, backend, and timer semantics. Include per-tensor and grouped formats only after correctness checks pass. Add an ablation that encodes once and reads repeatedly, and another following the real append/update policy. Bound logging and avoid synchronizing every operation in the primary throughput run solely for profiling.

Benchmark update protocols separately: encoding a sealed block once, quantizing only new eligible values, and re-encoding a growing interval. Keep the same valid history and state the source used for every re-encoding. A fast read path does not compensate automatically for expensive updates. Include parameter estimation, packing, layout normalization, and host dispatch in the appropriate stage ledger instead of timing only the final quantized attention call.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class KVStageMeasurement:
    stage: str
    input_bytes: int
    output_bytes: int
    temporary_bytes: object
    completed_seconds: float
    measurement_scope: str


def benchmark_quantization_stages(cases, completion_contract):
    # Measure real representation work and preserve timing boundaries.
    ...


def compare_complete_decode(reference, compressed, trace):
    # Include updates, materialization, attention, and final drain.
    ...


def E10(config):
    # Explain both favorable capacity cases and slower compressed cases.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report stage intervals, complete wall time, temporary peaks, useful tokens, and quality gates. Explain differences between isolated sums and measured pipeline time. A reduced cache allocation cannot be presented as reduced decode latency without execution evidence. Preserve negative or inconclusive results, including contexts where metadata and conversion cost dominate.

Use a short profiler trace to attribute work and repeated unprofiled regions for throughput. Profiling or validation synchronization can change the measured schedule. Compare completed wall time with device intervals under named boundaries. Include a negative performance case where compression saves persistent bytes but increases total latency. Explain whether the cause

is temporary bandwidth, small kernels, parameter work, or a changed attention backend rather than assigning every slowdown to low-bit arithmetic.

Part 7 - Engineering Interpretation and Philosophy

Choose the simplest consumer justified by complete measurements. State the additional implementation and validation needed for direct packed attention, and keep that proposal separate from the delivered reference path's measured behavior.

19.38.12 Exercise 11

Exercise Title: Compare Quantizing Keys Only, Values Only, and Both Keys and Values to Determine Which Component Is More Sensitive

Part 1 - Exercise Contract and Why It Matters

Compare keys-only, values-only, and joint KV quantization to diagnose component sensitivity. Hold code width, grouping, calibration, and model arithmetic fixed for the primary comparison. Report error and storage together rather than assuming equal-sized components have equal numerical importance.

Part 2 - System and Memory Mental Model

Key error changes attention scores before softmax; value error changes the vectors combined by those weights. Their effects depend on queries, score margins, outliers, and later layers. The same KV reconstruction MSE can lead to different attention or logit changes. Quantizing only one component retains the other at its actual baseline precision and needs a mixed-format consumer. No universal K-versus-V sensitivity ranking follows from the equations alone.

Part 3 - Quantitative Prediction Before Execution

Construct a small example where key perturbation changes attention weights, and another where similar key errors leave them nearly unchanged. Construct a value perturbation case with fixed attention weights. Calculate mixed-format bytes for keys-only, values-only, and both, including their separate parameters. Define how quality is compared at equal code bits and, in a secondary experiment, at approximately equal complete storage.

Part 4 - Experimental Design and Instrumentation

Use fixed Q/K/V replay to isolate local effects, then integrated teacher-forced decoding on held-out traces. Measure attention weights where the transparent reference exposes them, attention outputs, logits, and the chosen model-quality metric. Preserve positional-key convention and masks. Include different layers, heads, and lengths without selecting only a favorable example. Record separate K and V clipping and reconstruction errors. Keep any attention-weight diagnostic allocation outside the primary throughput measurement.

Preserve the same frozen queries when isolating key and value sensitivity. If queries change in integrated decoding, label the result as a compounded model effect. Compare errors per layer/head and include score distributions with close and widely separated maxima. Softmax can attenuate or amplify the consequence of score perturbation depending on the case. This makes a universal sensitivity ordering an empirical question rather than a property of tensor size.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class KVComponentPolicy:
    key_format: str
    value_format: str
    key_grouping: tuple
    value_grouping: tuple
    calibration_identity: str


def evaluate_component_policy(policy, frozen_cases, decoder_cases):
    # Separate score/weight effects from value aggregation and rollout drift.
    ...


def compare_component_tradeoffs(results, quality_contract):
    # Compare complete bytes and quality without assuming equal sensitivity.
    ...


def E11(config):
    # Test K-only, V-only, and joint formats under explicit matching rules.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report K/V errors, clipping, weight changes, attention/logit metrics, complete bytes, workspace, and latency. Preserve cases whose component ranking reverses. Equal MSE is not equal model quality and a single prompt is not a model-wide sensitivity study. Distinguish frozen local evidence from integrated cache trajectories that already differ because of earlier quantization.

Compute actual bytes for the untouched component and its compressed counterpart, including separate parameter shapes. At approximately equal storage, document which grouping or bit-width changes were necessary; that is a secondary comparison, not the same controlled experiment as equal bits. Report cases where lower reconstruction MSE fails to improve logits. Keep quality thresholds fixed before observing which component appears more favorable.

Part 7 - Engineering Interpretation and Philosophy

Choose component-specific formats only when the evidence supports the extra complexity. Explain the tested sensitivity mechanism and uncertainty, without turning observations from one model or head into a universal compression rule.

19.38.13 Exercise 12

Exercise Title: Preserve a Recent Full-Precision KV Window While Quantizing Older Tokens and Measure the Memory-Accuracy Trade-Off

Part 1 - Exercise Contract and Why It Matters

Keep a recent high-precision KV window and quantize older processed tokens. Measure persistent bytes, transition peaks, update cost, and quality as window length changes. Preserve exact logical token order and complete causal history across the boundary.

Part 2 - System and Memory Mental Model

Recent refers to processed positions within each request, not wall-clock age or a global batch counter. A token aging out of the window changes format but remains part of attention. Group boundaries can require delaying conversion until a complete block exists or handling a partial group explicitly. The high-precision window is high precision relative to stored codes; it need not be float32. Once original values are discarded, later promotion from low-bit reconstruction does not recover them.

Part 3 - Quantitative Prediction Before Execution

For request length T and nominal window W, predict floating and quantized token counts under the exact boundary policy, including $T < W$ and an empty history. Add scales, packing tails, and possible block rounding. Calculate maximum coexistence while sealing old values into codes. Predict conversion frequency and compare a zero-window policy, several bounded windows, and an all-float reference with the same consumer.

Part 4 - Experimental Design and Instrumentation

Implement explicit region metadata and transition ownership. Test crossing the window boundary by one token, partial quantization groups, multiple requests with different lengths, and shrink/regrowth under a supported prefix policy. Use a fixed teacher-forced trace and independent held-out quality cases. Validate that the mixed consumer neither duplicates nor omits positions and uses matching positional keys. Charge temporary originals, code destinations, and reconstructed attention inputs. Measure update latency as well as steady read latency.

Define the exact processed-position boundary for each request and assert that quantized and floating regions neither overlap logically nor leave a gap. Test several requests aging at different rates in the same batch. If conversion waits for a complete block, report the additional high-precision residual capacity beyond the nominal window. Old floating capacity retained in a reusable buffer still counts even after its logical region becomes quantized.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class KVPrecisionRegions:
    quantized_stop: int
    floating_start: int
    committed_length: int
    boundary_policy: str
    region_formats: tuple


def advance_recent_window(cache, request, new_length, policy):
```

```
    # Convert eligible old values and publish nonoverlapping valid regions.
    ...


def read_mixed_precision_kv(cache, request, attention_contract):
    # Restore correct logical order without dropping boundary positions.
    ...


def E12(config):
    # Sweep windows with truthful persistent and transition accounting.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report actual floating token counts, quantized groups, parameters, residual capacity, workspace, and quality. A nominal window can differ from actual retained floating capacity because of grouping and allocator policy; report both. Include a case where a larger window costs more without meaningful quality gain and one where aggressive conversion violates the quality gate. Do not assume recent tokens are always the most important.

Test a token crossing the boundary, a group crossing a page boundary, and a request truncated then regrown under a supported causal-prefix policy. Do not resurrect stale tail values. When a shared prefix ages into a different format, preserve every owner's interpretation through immutable replacement or a coordinated supported policy. Charge any retained originals used for quality comparison outside the measured production path and report their actual storage scope separately.

Part 7 - Engineering Interpretation and Philosophy

Choose a window policy conditional on measured quality and update cost. Explain what information its recency heuristic uses and why retained high precision is a capacity-quality choice rather than a guarantee about every long-range dependency.

19.38.14 Exercise 13

Exercise Title: Build a Block-Wise Quantized KV Cache Where Every Physical KV Block Stores Independent Scale Metadata

Part 1 - Exercise Contract and Why It Matters

Build independently decodable quantized KV blocks with per-block format and scale metadata. Support sealed immutable blocks and a declared writable-tail policy. Deliver checked block encoding, decoding, publication, and byte accounting as the foundation for paged compression.

Part 2 - System and Memory Mental Model

A physical block is application-owned payload for a documented layer and token interval, not a hardware page. Its codes, scale/zero-point arrays, valid length, group shape, format version, and positional convention form one interpretation unit. Independent block parameters avoid changing every historical scale when a new block arrives, but reduce the quantizer's observation scope. A mutable partial block still needs a defined update or residual policy.

Part 3 - Quantitative Prediction Before Execution

For a fixed-token block, derive code bytes and parameter counts under each supported grouping. Include odd packing groups and valid tails. Predict how many complete blocks can be sealed during prefill and one decode step. Calculate transition capacity while source floating data and encoded destination coexist. Compare independent-block storage with whole-history per-tensor scaling at the same code width without assuming equal numerical error.

Part 4 - Experimental Design and Instrumentation

Use coordinate-coded block identities and different value ranges per block. Test shuffled physical placement, partial tails, zero groups, large outliers, and format mismatches. Seal blocks only after required layer writes are ready and validated. Keep a residual floating tail or an explicit frozen-scale policy; do not reassign parameters to old codes. Compare each block's reconstructed values and integrated logits with references. Retain only bounded scalar diagnostics in the measured memory path.

Associate code payload, parameters, shape, and valid length with one block generation. Reject a decoder receiving matching shapes but mismatched format identity. Test blocks with very different scales adjacent in logical order. A sealed block can be immutable while its placement changes; its numerical interpretation must stay attached during migration. State whether one block covers one layer or multiple layers and derive parameter counts accordingly.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class QuantizedKVBlock:
    layer_id: int
    valid_tokens: int
    payload: object
    parameters: object
    format_identity: tuple
    generation: int
    sealed: bool


def seal_kv_block(floating_block, quantizer_contract):
    # Encode matching codes/parameters and preserve publication dependencies.
    ...


def decode_kv_block(block, consumer_contract):
    # Validate complete format identity before reconstructing valid KV.
    ...
```

```
def E13(config):
    # Test independent scales, writable tails, and metadata-inclusive costs.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require exact packed-code behavior, valid parameters, correct tail boundaries, and quality thresholds. Report block payload, parameter arrays, alignment, residual storage, temporary coexistence, and completed time. A stale scale from another generation must be rejected, not silently broadcast. Verify failed encoding leaves no published block or leaked provisional allocation.

Compare actual allocated backing with useful code count, especially for partial blocks and independently padded groups. A per-block quantizer changes its calibration scope relative to whole-history scaling, so evaluate quality rather than assuming numerical equivalence. Inject failure after destination allocation but before publication and audit provisional cleanup. Old consumers must retain their original codes and parameters until their leases finish, even after a new block generation is ready.

Part 7 - Engineering Interpretation and Philosophy

Explain how independent interpretation units simplify paging and sharing, and what statistical or metadata cost they introduce. Choose a block/tail contract that remains auditable before adding mixed precision and dynamic re-encoding.

19.38.15 Exercise 14

Exercise Title: Combine Paged KV Cache with Quantized Physical Blocks and Recalculate Maximum Concurrent Sequence Capacity

Part 1 - Exercise Contract and Why It Matters

Combine logical paging with quantized physical blocks and recalculate quality-qualified concurrent-request capacity. Preserve checked tables, generation identity, shared-prefix ownership, and correct attention positions while accounting for representation-dependent allocation sizes.

Part 2 - System and Memory Mental Model

Logical token capacity and physical byte capacity are different. Two blocks covering the same number of tokens can require different bytes because of bit width, grouping, metadata, and residual policy. A pool may use separate size classes or fixed maximum-sized slots; unused slot capacity still counts. Sharing requires compatible format and model context. A float reconstruction of shared codes can create separate temporary payload even when persistent codes are unique.

Compressed prefix reuse also depends on the numerical trajectory that produced the cached values. Identical tokens and weights can yield different later KV after earlier approximate attention differs. Include the relevant precision-policy and canonical prefix-construction identity, or validate a deliberately weaker equivalence contract. Do not assume that matching final code format alone makes prefixes computed under different adaptive histories interchangeable.

Part 3 - Quantitative Prediction Before Execution

Calculate unique encoded backing, logical mappings, table bytes, free retained pool capacity, and consumer workspace for ragged requests. Derive a tested concurrency bound for homogeneous precision, then a mixed-format case. Include partial tails, shared prefixes, and a copy-on-write conversion. Compare nominal payload admission with complete working-set feasibility; packed storage alone does not prove the dense attention reconstruction will fit.

Part 4 - Experimental Design and Instrumentation

Adapt checked block tables with explicit format identifiers and per-layer layout. Start with one precision class, then add another using a documented pool policy. Replay the same bounded request arrivals, lengths, token traces, and completion events as the float reference. Validate table translations and codec correctness before model comparisons. Test stale handles, format-class reuse, shared-prefix destruction, and a budget rejection. Measure complete capacity peaks through request replacement and conversion, not only stable resident codes.

Use separate byte-accounted size classes or explicitly charged maximum-size slots. A free retained slot is still owned GPU backing and must remain in the strict capacity ledger. Test an admission where code bytes fit but metadata, residuals, and the dense consumer do not. Keep logical token mapping independent from physical slot order, and verify GQA head identity after mixed-format block decoding.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class QuantizedBlockMapping:
    logical_block: int
    handle: object
    format_identity: tuple
    valid_tokens: int
    model_context: tuple


def admit_quantized_requests(trace, pools, consumer_requirements, budgets):
    # Count real slot classes, metadata, residuals, and working-set headroom.
    ...


def audit_quantized_tables(tables, pool_state, shared_owners):
    # Verify generation, format, positions, and unique backing ownership.
    ...


def E14(config):
    # Validate paged numerical behavior before reporting concurrent capacity.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report maximum tested admission and completed concurrency separately from analytical bounds. Include useful tokens, unique codes, metadata, padding, retained slots, residuals, workspace, quality, latency, and rejected work. A format stored in oversized fixed slots may save no pool capacity despite smaller logical codes. Do not borrow optimized paged-kernel throughput results as measurements of this reference consumer.

Share only compatible immutable representations under the declared model context. Two requests can share original semantic prefixes but require different precision policies; define whether they share a common supported encoding or receive private replacements. Count unique backing once and every logical mapping separately. Report tested completed concurrency, queued/rejected work, and quality beside analytical capacity so a high admission estimate cannot hide unfinished or invalid inference.

Part 7 - Engineering Interpretation and Philosophy

Explain whether compression improved real admission, only persistent payload, or complete decode. State the pool and consumer assumptions behind the capacity claim, and preserve the separate contributions of paging, sharing, and lossy precision changes.

19.38.16 Exercise 15

Exercise Title: Combine KV Quantization with CPU Offloading and Compare Transfer Volume Against Lossless float16 KV Offloading

Part 1 - Exercise Contract and Why It Matters

Combine compressed KV storage with CPU offloading and compare actual transfer volume, host residency, GPU peaks, quality, and complete decode against lossless float16 offload. Preserve the stream/event and authority protocols established for migration while adding exact format metadata to every transferred block.

Part 2 - System and Memory Mental Model

Transfer savings require packed codes and parameters to cross the link. Dequantizing on CPU and transferring float16 does not provide low-bit link volume merely because CPU storage is compressed. Quantization adds approximation; moving the resulting bytes can still be lossless. Codes and scales must describe the same version, and their sources remain live and unchanged until the transfer completes. CPU backing, pinned staging, decoded GPU values, and retained references all consume separate capacity.

Part 3 - Quantitative Prediction Before Execution

Calculate H2D and D2H bytes for float16 offload versus the implemented compressed protocol, including parameter arrays, padding, and dirty updates. Estimate transfer time from measured size-dependent bandwidth and fixed cost. Add packing/dequantization and attention workspace. Predict a small-page case where metadata and launch overhead erase link savings, and a larger case where bandwidth reduction may matter. Compare model-quality changes separately from transport correctness.

Part 4 - Experimental Design and Instrumentation

Begin with synchronous compressed round trips and exact code/metadata checks. Then reuse bounded pinned buffers and generation-aware CUDA events. Keep weights, positions, masks, token trace, and consumer backend fixed. Test partial blocks, shared pages, cancellation,

and delayed metadata transfer. Validate that a block cannot be published ready until both payload and parameters are ready. Measure complete cold and warmed decode, not only DMA submission. Charge any retained float originals to their real host or GPU scope.

Capture actual transferred payload and metadata bytes by direction. A compressed CPU store followed by float16 H2D should be reported as float16 link traffic. For packed transfers, preserve valid code counts and grouping at the destination. A metadata copy can complete later than its code copy; publish readiness only when both are valid. Use bounded pinned staging and generation-aware identities for every outstanding transfer.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class QuantizedKVTransfer:
    block_generation: int
    format_identity: tuple
    payload_version: int
    parameter_version: int
    ready_events: tuple
    destination_state: str


def transfer_quantized_block(block, destination, transfer_contract):
    # Move matching packed payload/parameters and preserve endpoint lifetimes.
    ...


def publish_transferred_block(ticket, residency_state):
    # Require exact-generation readiness for every interpretation component.
    ...


def E15(config):
    # Compare link bytes, complete cost, and quality against float16 offload.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report actual direction-specific bytes, payload/metadata sizes, pinned capacity, host backing, GPU workspace, transport mismatches, numerical quality, and complete latency. A bit-width ratio is not a measured PCIe compression ratio. Keep the lossless baseline's semantics visible and do not call a lossy quantized model result numerically identical merely because transferred bytes were preserved exactly.

Test a delayed parameter transfer and reject a consumer that attempts to use newly arrived codes with old scales. Compare codec-preserving round trips independently from model-quality

changes. Include host backing, staging, original-value retention, reconstructed GPU history, and pending sources in the memory ledger. Do not infer link savings from bit width or use nominal PCIe bandwidth as a measured transfer result.

Part 7 - Engineering Interpretation and Philosophy

Explain whether compression relieved link bandwidth, persistent capacity, or neither after conversion costs. Recommend offload only within measured quality and resource limits, preserving the distinction between approximate KV representation and correct migration of that representation.

19.38.17 Exercise 16

Exercise Title: Build a Dynamic Precision Policy That Keeps Important or Recent KV Blocks at Higher Precision and Compresses Older Blocks More Aggressively

Part 1 - Exercise Contract and Why It Matters

Choose higher precision for selected recent or importance-ranked blocks and compress other blocks more aggressively. Deliver an explicit policy, quality evaluation, and byte ledger. Keep importance as a tested predictor, not permission to omit required attention history or claim guaranteed future accuracy.

Part 2 - System and Memory Mental Model

Mixed precision changes representation, not token visibility. Importance may use available recency or measured sensitivity proxies, but a future-aware oracle belongs only to diagnostic evaluation. Attention weights from one step do not universally measure future importance. Promoting a low-bit reconstruction to a wider format does not restore discarded information. Genuine restoration requires retained originals or recomputation under a supported model protocol, whose cost and lifetime must be counted.

Changing historical cache precision does not retroactively recompute later KV or hidden states that were already produced from an earlier approximation. The controller manages that evolving approximate state; its evaluation must follow the actual integrated trajectory rather than substituting freshly quantized reference KV and calling the two histories identical.

Part 3 - Quantitative Prediction Before Execution

For a fixed byte budget, predict code and metadata capacity for a mixed precision assignment, including residuals and size-class slack. Compare a recency-only policy with a simple calibrated sensitivity policy and a uniform format. Construct one trace where recent blocks matter and another requiring older history. Estimate transition capacity and conversion error for proposed format changes rather than treating precision switches as cost-free labels.

Part 4 - Experimental Design and Instrumentation

Calibrate proxies on separate traces and freeze their interpretation before held-out evaluation. Start with one-time assignments to sealed blocks, then add bounded changes with generation-aware publication. Preserve query/head mapping, positional convention, masks, and model weights. Measure task-level quality or a clearly labeled proxy, logits, byte peaks, conversion frequency, and complete latency. If a higher-precision fallback needs originals, store them in an explicitly charged scope. Otherwise allow queuing or rejection instead of pretending lost precision is recoverable.

Specify whether importance is a calibrated sensitivity estimate, a current attention diagnostic, or recency. Do not mix these labels. A future-aware assignment may be a useful offline upper comparison, but it must remain separate from the online controller. Use one stable workload and another with an older critical dependency to expose proxy failure. Preserve full causal visibility for every precision choice.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class KVPrecisionEvidence:
    block_identity: tuple
    age_tokens: int
    sensitivity_proxy: object
    current_format: tuple
    original_values_available: bool
    uncertainty: object


def choose_block_precisions(evidence, budgets, calibrated_quality_policy):
    # Use available information and reject unsupported restoration promises.
    ...


def apply_precision_assignment(plan, engine):
    # Preserve ownership, interpretation metadata, and transition headroom.
    ...


def E16(config):
    # Compare mixed formats with uniform and recency-only policies honestly.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report score inputs, assigned formats, quality gates, predicted and actual bytes, transitions, and latency. Include a proxy misranking and an attempted promotion without originals. Label calibration-based quality estimates as such; a live engine without an oracle cannot certify every unseen token's deviation from original inference. Do not use held-out reference outputs as online policy inputs while claiming an ordinary deployable controller.

Attempt promotion after discarding originals and require the engine to report that reconstruction width is increasing without recovering information. If originals are retained, count their bytes and transfer/recompute cost before claiming a reversible policy. Report every quality failure and policy fallback. An unseen request outside the calibrated envelope may need conservative handling or rejection; it cannot receive an automatic accuracy guarantee from a recency score.

Part 7 - Engineering Interpretation and Philosophy

Choose the smallest policy whose quality-capacity benefit exceeds conversion and monitoring cost. Explain uncertainty and reversibility explicitly, especially when the preferred precision would require information already discarded.

19.38.18 Exercise 17

Exercise Title: Select KV Precision from Live VRAM Pressure and Automatically Re-Quantize Existing Cache Blocks When Memory Crosses a High Watermark

Part 1 - Exercise Contract and Why It Matters

React to a named memory watermark by re-quantizing existing blocks when admissible and supported by the quality policy. Preserve valid formats, ownership, and transition capacity. Demonstrate that a pressure response cannot assume old storage is freed before replacement codes and metadata are ready.

Part 2 - System and Memory Mental Model

Re-quantization from reconstructed low-bit data composes approximation error; it is not equivalent to quantizing original values once. A retained original source changes host or GPU memory accounting. Shared blocks may require copy-on-write or a coordinated policy for every owner. Watermarks from live payload, allocator reservation, and driver headroom have different meanings. The controller can enforce its owned-byte admission contract, not guarantee universal freedom from CUDA OOM or unseen quality degradation.

Part 3 - Quantitative Prediction Before Execution

Predict old format, decoded temporary, replacement codes/parameters, and pending-reader coexistence for one conversion. Calculate eventual savings and immediate peak separately. Hand-simulate high/low thresholds with hysteresis and a context-growth spike. Include an action whose final representation fits but whose conversion peak does not. Compare one-shot and repeated conversion errors against the same original reference in an offline diagnostic.

Part 4 - Experimental Design and Instrumentation

Use a small application budget and deterministic pressure traces rather than uncontrolled device exhaustion. Admit complete transition demand before dispatch and publish checked format/version replacements only after readiness. Test missing originals, active readers, shared-prefix ownership, stale completion, and an infeasible conversion. Choose queuing, rejection, or another supported fallback when quality evidence or headroom is insufficient. Keep reference-based quality evaluation outside the online decision unless its availability and cost are explicitly part of the contract. Record sample age and unsupported counters.

Use high and low watermarks with a named byte scope, timestamp, and next-step growth estimate. Before conversion, reserve capacity for decoded intermediates, new codes, new metadata, and old-reader retention. Include a case where the engine has too little headroom to perform its preferred compression. The supported response must be explicit rather than freeing old live storage prematurely or allocating outside the declared budget.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class KVRequantizationPlan:
    block_identity: tuple
    source_format: tuple
    target_format: tuple
    source_is_original: bool
    transition_capacity_bytes: int
    expected_final_bytes: int
    quality_evidence: object


def plan_pressure_requantization(engine, watermark_evidence):
    # Admit complete peaks and reject unsupported quality/reversibility claims.
    ...


def commit_requantization(plan, readiness, ownership_state):
    # Publish matching codes/parameters and retire old exact-generation users.
    ...


def E17(config):
    # Test pressure, hysteresis, cumulative error, and failed-action cleanup.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report metric scope, thresholds, age, proposed savings, transition peaks, actual reclaimed capacity, quality, and failures. A fixed oversized pool slot may remain the same capacity after re-encoding, so code-size savings need not relieve pressure. Keep old backing counted until all users finish. A rejected action is correct when conversion or quality prerequisites cannot be satisfied.

Compare repeated conversions with one-shot quantization from the same original cache in an offline diagnostic, while preserving the online source policy. A later wider code format cannot erase earlier error. Test shared-owner disagreement and stale conversion completion after a slot is recycled. Validate hysteresis on a stable near-threshold trace so a precision policy does not oscillate without useful capacity or quality benefit.

Part 7 - Engineering Interpretation and Philosophy

Explain whether reactive compression provides real usable headroom and what approximation risk it purchases. Recommend thresholds and fallbacks from tested behavior while preserving uncertainty about external allocations, future quality, and irreversible repeated compression.

19.38.19 Exercise 18

Exercise Title: Stress-Test Quantized KV Caching with Long Contexts, Variable Sequence Lengths, Shared Prefixes, Paged Blocks, and Aggressive Memory Pressure

Part 1 - Exercise Contract and Why It Matters

Stress-test quantized KV with long bounded contexts, variable lengths, shared prefixes, paged formats, and deliberately tight resource limits. Search for minimal failures in codecs, numerical quality, format identity, lifecycle, and byte accounting. Separate intentional quality rejection from implementation corruption.

Part 2 - System and Memory Mental Model

Critical cases combine valid transitions: a shared block changes precision while a reader uses its old format, an odd packed tail crosses a page boundary, or a stale scale is paired with recycled payload. A byte-perfect codec can still produce unacceptable model error; acceptable logits do not prove safe ownership. Maintain separate format, semantic, resource, and quality oracles. Memory pressure should be bounded by engine admission limits, not machine-wide exhaustion.

Part 3 - Quantitative Prediction Before Execution

Predict complete peaks for residual-window sealing, copy-on-write compression, offload, and repeated re-quantization combinations. Construct short traces that expose zero scales, clipping, odd nibble padding, stale metadata, and shared-format disagreement. Define which quality failures should reject a configuration and which invalid operations should produce explicit errors. State progress assumptions when conversion requires temporary headroom or original values that may not be available.

Part 4 - Experimental Design and Instrumentation

Use fixed seeds, trace IDs, finite requests, bounded contexts, host/GPU budgets, and pending-operation limits. Begin with CPU codec and deterministic transition checks, then supported CUDA execution. Evaluate held-out model quality and teacher-forced logits at selected checkpoints. Inject preparation failures before publication and forced stale-handle controls. Audit after every transition and final drain. Save and reduce failing histories while preserving calibration identity, prefix setup, format metadata, and completion order.

Keep exact codec tests, valid approximate inference, intentional quality failure, and unsupported operations as different outcome classes. Save format and calibration identities with seeds and completion order. A seed alone may not reproduce asynchronous behavior. Start from deterministic short traces that expose each invariant before running randomized combinations, and bound pending conversions and diagnostic logs as well as payload storage.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class QuantizedKVStressBounds:
    max_requests: int
    max_context_tokens: int
    gpu_bytes: int
    host_bytes: int
    max_pending_conversions: int
```

```
    steps: int


def generate_quantized_trace(seed, bounds, supported_formats):
    # Mix valid histories with labeled codec/metadata rejection controls.
    ...


def audit_compressed_runtime(engine, format_oracle, semantic_oracle):
    # Separate exact code invariants, approximation quality, and resource state.
    ...


def minimize_compression_failure(trace, replay):
    # Preserve source/calibration and asynchronous setup dependencies.
    ...


def E18(config):
    # Test deterministic failures before bounded supported CUDA stress.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report seeds, traces, exact failure phase, quality thresholds, reproducibility, and smallest counterexamples. Distinguish unsupported execution, expected admission rejection, quality failure, and corruption. A run with no observed counterexample is not proof over all inputs or interleavings. Do not fabricate hardware peaks, throughput, or model-quality scores when those experiments were not performed.

Check final drain and cancellation so delayed metadata publication cannot hide after the test appears finished. Reduce a failure only while preserving its source values, grouping, shared-prefix setup, and required lifecycle events. Record both original and reduced traces with their invariant violation. If quality fails without codec corruption, preserve that distinction and reject the tested format/policy rather than silently widening the acceptance threshold after seeing the result.

Part 7 - Engineering Interpretation and Philosophy

Identify the most consequential unresolved numerical or lifecycle risk and its next bounded test. State the supported envelope honestly, keeping exact representation correctness separate from empirical model-quality evidence.

19.38.20 Exercise 19

Exercise Title: Build and Benchmark a PyTorch Adaptive KV Cache Compression Engine That Dynamically Chooses Precision per Block to Maximize Context Capacity Under Accuracy and VRAM Constraints

Part 1 - Exercise Contract and Why It Matters

Integrate an adaptive per-block KV compression engine with explicit formats, quality policy, byte admission, residual windows, paging, optional offload, and safe re-encoding. Benchmark quality-qualified context and concurrency capacity under named VRAM limits. All substantive implementation remains learner work in the scaffold.

Part 2 - System and Memory Mental Model

One authority coordinates request positions, layer validity, codes, parameters, format generations, shared owners, pending readers, and residency. The attention consumer sets working-set requirements and includes unpack/dequantization or streaming costs. Dynamic precision is an approximation policy, not an online proof of model accuracy. Original-value retention and restoration are supported only when their storage and computation are explicitly budgeted. Keep model weights and required full-context semantics fixed for the primary comparison.

Part 3 - Quantitative Prediction Before Execution

Predict persistent codes, parameters, residuals, page slack, retained free slots, conversion duplicates, decoded workspace, and offload staging. Compare float16, uniform int8, implemented packed 4-bit, fixed mixed precision, and adaptive policies under equal supported token traces. Define quality thresholds and capacity objectives before evaluation. Estimate both steady savings and maximum transition demand. A candidate outside quality or consumer-feasibility constraints is not an admissible capacity improvement.

Part 4 - Experimental Design and Instrumentation

Freeze support for layouts, model context, grouping, sharing, append, truncation, conversion, cancellation, and migration. Validate codecs and metadata, then numerical and lifecycle oracles before benchmarking. Use separate calibration, policy selection, and held-out evaluation traces. Record device, software versions, allocator backend, model revision, dtype, masks, positions, formats, budgets, seeds, and initial state. Include setup, conversion, calibration, monitoring, attention, and drain in complete cost. Run ablations without weakening quality or resource gates.

Publish supported schemes, code mappings, scale axes, parameter dtypes, packing, residuals, sharing, and consumer semantics before the integrated benchmark. Keep model arithmetic and weights fixed for the main comparison. Evaluate a memory-constrained case and a short-context case where compression may lose. Separate estimated maximum context from actually completed inference, and count rejected or unfinished requests in the outcome record.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class AdaptiveKVCompressionContract:
    model_context: tuple
    supported_formats: tuple
    gpu_capacity_bytes: int
    host_capacity_bytes: int
    attention_consumer: str
```

```
    quality_policy_identity: str
    original_value_policy: str
    maximum_pending_conversions: int

class AdaptiveKVCompressionEngine:
    def __init__(self, contract):
        # Establish formats, tables, ownership, budgets, and calibrated policy.
        ...

    def append(self, request, layer, keys, values):
        # Preserve valid positions and the declared tail/group update policy.
        ...

    def attention(self, query, request, layer):
        # Decode exact format identity and preserve the causal consumer contract.
        ...

    def adapt_precision(self, pressure_evidence):
        # Gate actions by capacity, quality evidence, and reversibility support.
        ...

    def audit(self):
        # Recompute codes/metadata ownership, quality support, and complete bytes.
        ...

    def shutdown(self):
        # Drain or cancel owned work and report outstanding obligations.
        ...


def E19(config):
    # Validate integrated semantics before held-out capacity/quality benchmarks.
    ...
    # Export evidence, counterexamples, limitations, and recommendation.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Acceptance requires exact codec behavior, valid parameter association, correct masks/positions, quality within declared tested limits, safe ownership, and truthful complete capacity. Report context/concurrency estimates and tested completed cases separately. Include quality metrics, clipping, code/metadata bytes, temporary peaks, conversion count, transfer volume, latency distributions, and rejected work. Keep prediction, observation, and interpretation separate. An incomplete scaffold supplies no hardware or model-quality results.

Report complete quality-qualified capacity with codes, parameters, residuals, retained free pools, decoded workspace, transitions, and optional host originals. A strict memory gate can be enforced from owned allocations, while an empirical accuracy envelope needs held-out evidence and explicit uncertainty. Do not present a calibration proxy as a guaranteed online oracle.

Conclude with the smallest supported format/policy and the next experiment needed to justify more aggressive adaptation.

Part 7 - Engineering Interpretation and Philosophy

Recommend the smallest quality-qualified design supported by complete measurements, including float16 when compression does not help. Explain the capacity gained, numerical approximation accepted, and conversion or transfer cost paid. Keep proposed fused kernels, wider model support, and untested accuracy guarantees separate from delivered evidence.

19.39 DMM37

Chapter Title: KV Cache Eviction and Context Retention Policies
Specialization: Dynamic Memory Management and CUDA Systems

19.39.1 Exercise 00

Exercise Title: Build a Precise Mental Model of KV Cache Eviction, Context Retention, Attention Importance, and the Memory-Accuracy Trade-Off

Part 1 - Exercise Contract and Why It Matters

Define the distinct meanings of KV eviction before implementing retention policies. Deliver a lifecycle and semantic diagram separating active-context pruning, CPU demotion, compression, and reclamation of unused reusable prefixes. The first changes the attention inputs; the others can have different numerical, residency, and future-prefill consequences. Keep their outcomes distinguishable throughout the chapter.

Part 2 - System and Memory Mental Model

Removing an active token's KV generally changes attention and renormalizes weights over retained keys. It is not a cache miss that automatically reloads an unchanged computation. Removing an unreferenced prefix-cache entry can leave current inference unchanged and require future recomputation. Physical storage reclamation still waits for every owner and device reader. Retained KV values may already encode information from earlier context; pruning storage is not equivalent to recomputing a model on only the retained text.

Part 3 - Quantitative Prediction Before Execution

Predict useful token entries, unique backing, metadata, score arrays, workspace, and transition copies for a bounded policy. Separate logical token removal from physically reusable bytes. Define accuracy and resource contracts before measurement: retained-position correctness, perplexity protocol, retrieval accuracy, and complete decode cost. Construct a case where memory falls but quality fails and another where prefix hit rate falls without changing a completed request's outputs.

Part 4 - Experimental Design and Instrumentation

Use a transparent tiny decoder for implementation invariants and a fixed supported trained model for meaningful quality evaluation when available. Preserve weights, tokenization, model revision, attention backend, positions, GQA mapping, masks, and precision in the primary comparison. Start with one global retained-position set per request across layers; per-head variants are separate extensions. Keep original absolute positions in the reference policy and

stay within supported positional limits. Record hardware, software, seed, trace ID, budgets, and initial state.

Draw four actions applied to the same logical token: keep on GPU, move an unchanged representation to CPU, store an approximate compressed representation, and remove its direct attention entry. Record what later retrieval requires in each case. Include inactive reusable prefixes as a separate ownership class. A permanent-delete label means no restoration path in the declared runtime; if original input is retained for recomputation, record that recovery capability and its host storage explicitly.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class KVRetentionContract:
    eviction_semantics: str
    position_policy: str
    retention_scope: str
    maximum_tokens: int
    physical_budget_bytes: int
    quality_policy_identity: str


def classify_eviction_action(action, ownership, backing_state):
    # Distinguish semantic pruning, movement, compression, and cache reclaim.
    ...


def E00(config):
    # Define the supported task and audit semantic/resource transitions.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require separate prediction, observation, and interpretation fields. Report logical retained tokens, real backing capacity, protected owners, quality, and completed time. Do not claim lossless equivalence after active context deletion or universal semantic importance from attention magnitude. Bound scalar evidence and avoid retaining the full cache in diagnostic tensors. Unperformed model or CUDA experiments remain explicitly unmeasured.

Classify every result by semantic correctness, approximation quality, physical reclamation, and complete performance. A policy can pass one and fail another. Do not interpret a good sentence as a resource audit or a low byte count as quality validation. State that deleted direct KV is not a guarantee of erasing all influence of earlier text, since surviving model state can already depend on it. This exercise studies inference approximation, not machine unlearning.

Part 7 - Engineering Interpretation and Philosophy

Explain the resource saved and the computation or future work changed by each action. A useful retention policy has an honest approximation contract, a checked lifecycle, and quality evidence rather than a single memory-saving percentage.

19.39.2 Exercise 01

Exercise Title: Establish a Full-KV Baseline and Measure VRAM Growth, Decode Latency, Perplexity, and Retrieval Accuracy as Context Length Increases

Part 1 - Exercise Contract and Why It Matters

Establish full-context KV behavior over increasing bounded context lengths. Measure useful cache growth, complete VRAM peaks, decode latency, token-weighted perplexity, and controlled retrieval accuracy. Preserve the model's supported context range and separate implementation validation from meaningful language quality measurement.

Part 2 - System and Memory Mental Model

Full KV retains every processed position required by the declared attention architecture. Cached token count differs from output tokens sampled but not forwarded. Static reservation and dynamic-growth copies produce different memory curves. Perplexity depends on tokenization, evaluation targets, and available preceding context. Retrieval accuracy depends on whether the model can perform the baseline task at all; a random-weight toy decoder cannot establish useful long-range language quality.

Part 3 - Quantitative Prediction Before Execution

Predict unquantized useful bytes from layer specifications, KV heads, dimensions, dtype, and ragged processed lengths. Define

$$\mathrm{PPL} = \exp\left(\frac{\sum_i \ell_i}{N_{\mathrm{scored}}}\right),$$

where each loss term scores a declared target once under its allowed context. Specify retrieval answer normalization and success criteria. Predict capacity and latency separately and identify limits where a full-KV reference is not feasible rather than inventing results beyond them.

Part 4 - Experimental Design and Instrumentation

Use fixed model/tokenizer revisions, zero dropout, no gradients, stable precision, and explicit causal masks. Score the same target positions for all future policies; document window/stride and padding exclusions. Build retrieval cases with known answers at controlled distances and distractor counts. Separate prefill from decode and teacher-forced evaluation from free generation. Warm deliberately, synchronize at completed region boundaries, and repeat timing. Store selected CPU reference outputs outside the measured GPU lifetime.

Freeze tokenizer and scoring conventions before comparing context lengths. Use an identical set of target tokens where policies can be fairly compared, and state how context available to each target differs. For fixed-length model evaluation, record stride and overlap, mask unscored targets, and avoid scoring one target repeatedly merely because it appears in several windows. Report any prefix whose first token has no valid preceding prediction context under the chosen evaluation protocol.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class FullKVQualityCase:
    trace_id: str
    context_tokens: int
    scored_target_positions: tuple
    retrieval_answer: object
    evaluation_protocol: str


def evaluate_full_kv(model, cases, measurement_contract):
    # Preserve full valid history and count each scored target exactly once.
    ...


def measure_retrieval(prediction, expected_answer, normalization_policy):
    # Apply the predefined task criterion without post-hoc reinterpretation.
    ...


def E01(config):
    # Establish capacity, timing, perplexity, and retrieval reference evidence.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report useful and capacity bytes, backend allocated/reserved peaks, processed lengths, scored-token counts, NLL, perplexity, retrieval outcomes, and latency spread. Do not average perplexities from unequal token counts as though that were corpus perplexity. Separate a baseline task failure from an eviction-induced failure. Mark unsupported lengths, model availability, and device measurements explicitly.

Keep local-language and known-answer retrieval datasets separately identified. A model unable to retrieve the full-cache answer provides a baseline failure, not proof of eviction damage. Preserve those cases in aggregate reporting and provide a competent-baseline diagnostic stratum without hiding the original population. Record maximum supported positions and refuse unvalidated extrapolation. A short toy model can validate the measurement pipeline while meaningful quality claims require an actually executed trained model and suitable data.

Part 7 - Engineering Interpretation and Philosophy

Explain which quality and resource baselines later policies must be compared against. Preserve a smaller feasible reference for correctness when a longer full-cache workload does not fit, without claiming measured parity at the unexecuted larger size.

19.39.3 Exercise 02

Exercise Title: Implement a Fixed Sliding-Window KV Cache That Evicts the Oldest Tokens Once a Maximum Token Budget Is Reached

Part 1 - Exercise Contract and Why It Matters

Implement a fixed-budget sliding cache that retains the newest processed positions and removes older entries. Preserve ordered token identities, correct per-layer K/V association, and the declared positional convention. Measure actual backing rather than assuming logical slicing releases memory.

Part 2 - System and Memory Mental Model

Cache slot indices and model positions are distinct. In the primary policy, retained keys preserve original absolute positions and query position advances with total processed tokens, not retained length. A ring buffer can bound capacity but needs logical ordering and masks. A slice view can retain the whole original storage. Evicting old KV does not recompute surviving values under a truncated text context and does not automatically match a model trained with native sliding-window attention.

Part 3 - Quantitative Prediction Before Execution

For token budget W, hand-simulate append and retention at lengths $W-1$, W, and $W+1$. Define whether the budget includes the current query's new K/V and reserve before append accordingly. Predict valid entries, backing capacity, compaction copies, and any provisional slot. List expected absolute key/query positions and visibility after the first wraparound.

Part 4 - Experimental Design and Instrumentation

Start with coordinate-coded KV and a CPU retained-position oracle. Implement a bounded ring or explicit compaction policy and document its temporary peak. Use one retention set across layers. Test zero/invalid budgets, multi-token appends larger than available capacity, ragged requests, and repeated wraparound. Then compare model outputs against the same explicitly pruned reference, not an assumed full-context equivalence. Keep positional limits valid and measure prefill pruning separately from decode admission.

Use a tiny retained-position oracle with absolute positions deliberately far from ring indices. Fill every layer's K and V with coordinate-coded values before wraparound tests. Compare current-token admission before and after the capacity boundary, including a multi-token suffix. If compaction is used, protect source values from overlapping writes and count its temporary destination. If a ring is used, validate consumer order independently from physical layout.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class SlidingKVState:
    total_processed_tokens: int
    retained_absolute_positions: object
    valid_slot_count: int
    ring_generation: int
```

```
def append_sliding_kv(state, layer_values, positions, budget):
    # Admit current values and evict oldest positions under the exact contract.
    ...


def ordered_sliding_view(state, consumer_contract):
    # Preserve key order and absolute positions without retaining hidden backing.
    ...


def E02(config):
    # Validate wraparound, masks, capacity, and the changed attention computation.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require exact retained IDs and K/V values for synthetic cases, then report model-quality differences from full KV. Audit allocated slot capacity, metadata, compaction temporaries, and backend peaks. A view with fewer elements is not evidence of reclaimed backing. Changing only position metadata cannot correct already positionally transformed keys if another reindexing policy is attempted.

Test that a sparse retained set preserves correct causal visibility rather than letting every retained key participate in every query. An earlier query in a multi-query suffix must not see later keys. If experimenting with compact positions, treat it as a separate positional model and transform keys/queries correctly; changing integer labels on already rotated keys is insufficient. Measure backing capacity directly so a short slice of a large tensor cannot masquerade as bounded physical storage.

Part 7 - Engineering Interpretation and Philosophy

Explain the bounded-memory guarantee and the contextual information no longer directly available. State which positional model was actually implemented and avoid describing a generic newest-token policy as a reproduction of every streaming-attention method.

19.39.4 Exercise 03

Exercise Title: Compare Full Cache and Sliding-Window Cache on Local Continuation and Long-Range Retrieval Tasks

Part 1 - Exercise Contract and Why It Matters

Compare full and sliding KV on local continuation and long-range retrieval using matched inputs and evaluation rules. Identify quality effects by distance, retained evidence, and task type rather than expecting one global quality number to explain every retention trade-off.

Part 2 - System and Memory Mental Model

A recent window may preserve local transitions while removing the only explicit retrieval evidence. Remaining hidden states can still carry indirect information, so loss of direct KV does not guarantee failure. Full KV can also fail difficult tasks. Retention changes model state over time; feeding identical teacher-forced tokens controls input history but does not make the internal histories identical. Free generation introduces further divergence.

Part 3 - Quantitative Prediction Before Execution

Construct evidence positions inside and outside several tested windows. Predict which source positions survive at answer time and what each policy can directly attend to. Define local quality through token-weighted NLL and a separate generation rubric if used. Define retrieval success and stratify by distance, distractors, and context length. Estimate memory and latency without assuming fewer keys always yield faster complete decoding.

Part 4 - Experimental Design and Instrumentation

Fix model/tokenizer, prompts, target tokens, precision, positions, masks, and attention backend. Use held-out local text and controlled fact-retrieval cases with known answers. Compare teacher-forced losses at identical targets, then separately evaluate generated answers. Randomize irrelevant fact placement and wording with fixed seeds. Retain baseline-failure cases in the report and distinguish them from eviction damage. Keep compaction and gather costs inside complete performance regions while reporting diagnostic stages separately.

Construct matched prompt families with a fact just inside and just outside the tested recent windows. Vary distractors independently from answer distance. For local continuation, score identical targets instead of comparing different freely generated text as if it were the same workload. For generation, freeze sampling parameters and report the resulting history divergence. Define answer normalization before evaluation so a desired policy does not receive more lenient retrieval grading.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class RetentionTaskCase:
    case_id: str
    task_kind: str
    evidence_positions: tuple
    answer_positions: tuple
    expected_answer: object


def compare_context_policies(model, cases, policies, evaluation_contract):
    # Match targets and report evidence retention with actual task outcomes.
    ...


def summarize_distance_effects(result_rows):
    # Preserve baseline failures, uncertainty, and task-specific denominators.
    ...


def E03(config):
    # Evaluate local and long-range behavior without conflating their metrics.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report retained evidence IDs, local NLL/PPL, retrieval accuracy, generated answer outcomes, bytes, and complete latency. Include uncertainty across the tested cases and seeds. A coherent-looking continuation is not a retrieval success, and one lost fact is not proof every long-range task fails. Preserve counterexamples to simple recency claims rather than selecting only expected wins or losses.

Report byte and timing observations for the same execution region and separate quality-only diagnostic overhead. A smaller attention set can still lose end-to-end because of score collection or compaction. Preserve full-cache failures, eviction failures, and occasional wins distinctly. Include an uncertainty estimate appropriate to the finite task set rather than treating one example as a corpus-wide conclusion. State which dependency distances the tested model actually handles without eviction.

Part 7 - Engineering Interpretation and Philosophy

Choose a window only in relation to a task's tested dependency pattern. Explain which quality loss the capacity saving purchases and where the evidence does not justify deployment to another corpus, model, or context distribution.

19.39.5 Exercise 04

Exercise Title: Implement FIFO Token Eviction and Measure How Aggressive Memory Reduction Changes Generation Quality

Part 1 - Exercise Contract and Why It Matters

Implement FIFO token eviction and compare its quality-capacity behavior with the fixed sliding reference. Define insertion order, admission quantum, and protected exceptions explicitly. Use this exercise to expose policy identity and implementation costs rather than manufacture a distinct algorithm where the semantics coincide.

Part 2 - System and Memory Mental Model

For one append-only request with no reinsertion or protected tokens, FIFO removing the oldest inserted token is equivalent to retaining the most recent fixed-size window. Differences can arise from batched admission, global versus per-request queues, or protection rules, all of which must be named. A global FIFO can sacrifice one request's history to another's arrivals. Neither policy is lossless active-context cache management.

Part 3 - Quantitative Prediction Before Execution

Hand-simulate a token trace under FIFO and sliding retention and identify conditions producing identical retained sets. Add a multi-request global FIFO case with different arrivals and predict which request loses evidence. Calculate budget usage before and after an eviction quantum and include queue metadata and physical compaction. Define quality thresholds before sweeping aggressive budgets.

Part 4 - Experimental Design and Instrumentation

Build a checked insertion queue using stable request and absolute token identities. Compare per-request FIFO with the existing sliding implementation through a retained-set oracle. Introduce global FIFO only as a separate fairness experiment. Use the same local and retrieval tasks, targets, model, precision, positional policy, and masks. Test stale entries after request

completion, duplicate notifications, large appends, and shared backing restrictions. Measure complete useful work and real reclaimed bytes.

Make the insertion clock independent of token numeric ID, since repeated token IDs at different positions are different cache entries. Test one-request FIFO equivalence first, then a global queue where arrivals interleave. Include stale entries from a completed request generation and require rejection before victim selection. Define batch-append eviction quantum explicitly; removing several entries at once can create different transient costs even when final retained sets coincide.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass(frozen=True)
class FIFOEntry:
    request_id: int
    request_generation: int
    absolute_position: int
    insertion_sequence: int


def choose_fifo_victims(queue, admission, protection, scope):
    # Preserve declared global/per-request semantics and hard eligibility.
    ...


def apply_fifo_eviction(cache, victims, completion_evidence):
    # Update logical visibility and safely reclaim only owned storage.
    ...


def E04(config):
    # Prove the simple equivalence and evaluate any explicit policy differences.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require identical retained sets for equivalent FIFO/sliding cases. Report any difference as an implementation error until a semantic distinction explains it. Keep per-request quality, starvation, bytes, and latency visible under global capacity. Do not report two versions of the same retention rule as independent evidence for algorithmic superiority.

Compare queue metadata and physical movement for implementations with equal retention semantics. An apparent quality difference between equivalent FIFO and sliding sets may indicate a position, validity, or masking bug. For global FIFO, report per-request retained evidence and completion metrics so one request's capacity gain cannot hide another's starvation. Keep protected-owner rules as hard constraints rather than permitting an old shared block to be freed solely because its insertion time is small.

Part 7 - Engineering Interpretation and Philosophy

Explain why precise policy definitions prevent misleading comparisons. Use FIFO as an auditable baseline and state exactly which extra constraint, protection, or scheduling rule changes the result.

19.39.6 Exercise 05

Exercise Title: Protect the Most Recent Tokens from Eviction and Determine the Minimum Recent Window Required to Preserve Local Coherence

Part 1 - Exercise Contract and Why It Matters

Protect a recent-token quota while allowing other eligible entries to be pruned. Determine the smallest tested recent window meeting a predefined local quality criterion within a named workload. Do not present this empirical result as a universal minimum context needed for coherence.

Part 2 - System and Memory Mental Model

Recency uses absolute processed positions within each request, not current physical slot order. The protected quota consumes the same total token budget as every other retained entry. Short histories can overlap other protection sets and must be counted once. Local coherence is not a single intrinsic cache property: token-weighted likelihood, grammatical continuity, and task consistency are different measurements. Long-range facts can fail even when local continuation remains fluent.

Part 3 - Quantitative Prediction Before Execution

For total budget B and protected recent quota R, derive remaining eligible capacity and reject $R > B$. Predict retained sets for short histories, window crossing, and multiple-token appends. Define a tested window grid and quality threshold before evaluation. Do not assume quality is monotonic in window size or use an unjustified binary search to claim the global minimum. Estimate both stable storage and boundary-copy peaks.

Part 4 - Experimental Design and Instrumentation

Use a fixed policy for the unprotected portion so only recent quota changes. Compare against pure recency and full KV on held-out local continuation and separate retrieval tasks. Record model/tokenizer, positions, masks, precision, seeds, and score protocol. If using human coherence ratings, define the rubric and blind policy labels; do not replace objective measurements with a few selected anecdotes. Measure all tested windows with repetitions and retain failures. Charge protection metadata and physical layout costs.

Specify the exact recent-position set from total processed length, including the current query's committed or prepared status. Validate empty histories, short histories, and protected-union overlap. Use a small explicit grid rather than assuming a monotone quality curve. If an initial quota is also protected, subtract only distinct positions from the total budget. Reject an infeasible protection contract before launching a model step or silently increasing physical capacity.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class RecentProtectionPolicy:
    total_token_budget: int
    protected_recent_tokens: int
    remainder_policy: str
    quality_threshold_identity: str


def protected_recent_positions(processed_length, policy):
    # Define the absolute-position set and deduplicate overlapping protection.
    ...


def evaluate_recent_windows(cases, policies, quality_contract):
    # Report the smallest tested passing setting with its uncertainty.
    ...


def E05(config):
    # Sweep recent quotas without changing total capacity or task semantics.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report every tested quota, actual protected set, total retained tokens, quality metrics, byte peaks, and latency. Mark no passing configuration when appropriate. A passing mean can hide severe cases, so preserve distribution and retrieval outcomes. State calibration versus held-out use and avoid choosing thresholds after seeing the preferred policy's result.

Keep calibration and held-out window selection separate. Report the smallest tested setting that passes, its neighboring failures, and whether the metric is mean, worst-case, or percentile quality. Subjective coherence ratings need a stated rubric and sample set; they cannot establish retained factual accuracy. Track boundary-copy or ring-update costs and protection metadata. A tested window suitable for local continuation may still fail an older explicit fact, which must remain visible in the recommendation.

Part 7 - Engineering Interpretation and Philosophy

Recommend a recent quota conditional on model, corpus, task, and total budget. Explain the uncertainty in the smallest tested passing value and the dependency types that remain unsupported despite acceptable local continuation.

19.39.7 Exercise 06

Exercise Title: Protect Initial Attention-Sink Tokens While Evicting Middle-Context KV Entries and Measure the Effect on Long-Sequence Stability

Part 1 - Exercise Contract and Why It Matters

Protect a bounded set of initial attention-sink candidates alongside recent positions while pruning middle KV. Measure stability, quality, and storage under the same total budget. Treat the sink mechanism as an empirical hypothesis, not proof that initial tokens preserve all removed information.

Part 2 - System and Memory Mental Model

Some models assign substantial attention mass to initial positions even when those tokens carry little task content. Retaining such entries can alter normalization and stability, but does not restore middle-context facts. Initial tokens may also contain meaningful instructions or data. Keep semantic content and structural behavior separate. The primary implementation retains original absolute positions; a compact or shifted positional scheme is a separate model-specific experiment requiring correctly transformed keys and queries.

Part 3 - Quantitative Prediction Before Execution

For initial quota S, recent quota R, and total budget B, calculate their union and remaining capacity without double-counting short histories. Predict source positions surviving a middle deletion. Construct a retrieval fact in the discarded middle and a local continuation case. Estimate protected block rounding if token-level choices are later mapped to physical pages. State stability metrics and expected failure evidence before execution.

Part 4 - Experimental Design and Instrumentation

Compare full KV, recent-only, initial-plus-recent, and a matched-size control protecting different positions. Keep total budget, precision, model, masks, positions, and task traces fixed. Measure selected attention distributions, losses, retrieval, and generation under a declared rubric. Stay within supported positional limits. If exploring a published streaming position scheme, implement and validate its model-specific transformation separately; do not relabel this simple ablation as a complete reproduction. Count diagnostic attention workspace outside the primary throughput path or explicitly within its budget.

Create an initial-protection control with the same number of noninitial positions so capacity alone cannot explain a benefit. Record whether initial tokens contain model delimiters, meaningful task content, or ordinary filler. Count overlap with recent protection once. Use finite streams inside the primary model's positional support, and keep any long-stream positional extension separately identified. Sink-related literature motivates the test, not a borrowed result for the chosen model.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class SinkRetentionPolicy:
    initial_tokens: int
    recent_tokens: int
    total_tokens: int
    positional_convention: str
```

```
def select_sink_recent_union(history, policy):
    # Preserve absolute IDs and resolve overlap within the fixed quota.
    ...


def evaluate_sink_ablation(model, cases, policies):
    # Separate normalization/stability evidence from retained factual content.
    ...


def E06(config):
    # Test initial protection and middle-loss counterexamples honestly.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report protected identities, union size, quality, retrieval, stability, attention diagnostics, and complete cost. Include a case where the proposed sink benefit is not observed or record that no such counterexample appeared in the tested range. Do not equate stable fluent generation with successful retention of distant facts. A position-label change alone cannot rephase already encoded rotary keys.

Measure local loss and retrieval separately from apparent generation stability. A stable attention denominator or fluent output does not restore a removed middle fact. When comparing alternate positional conventions, account for any key transformation, extra storage, and recomputation. Do not call the simple initial-plus-recent reference an exact implementation of a published streaming system. Preserve negative controls where initial protection provides no benefit or displaces genuinely useful evidence under the fixed total budget.

Part 7 - Engineering Interpretation and Philosophy

Explain which evidence supports a sink-related effect and which uncertainty remains. Choose initial protection only when its measured benefit justifies capacity displaced from other context under the tested model and task.

19.39.8 Exercise 07

Exercise Title: Record Token-Level Attention Scores Across Decode Steps and Rank Cached Tokens by Accumulated Attention Importance

Part 1 - Exercise Contract and Why It Matters

Record bounded token-level attention evidence and rank retained positions by accumulated attention mass. Define score units, scope, aggregation, and update timing before using them for decisions. Measure instrumentation cost and preserve the distinction between an observed attention weight and causal task importance.

Part 2 - System and Memory Mental Model

Post-softmax attention weights describe one query over the currently visible keys. Raw logits have different scale and cannot be substituted silently. Aggregate across layers and query heads under a declared rule, including GQA head mapping. Eviction changes the denominator and future attention distribution. Older tokens may accumulate more observations simply because

they existed longer. An optimized attention API may not expose weights, so a transparent diagnostic consumer or separate measured instrumentation is required.

Part 3 - Quantitative Prediction Before Execution

Derive score shape for the retained absolute-position set and predict its metadata bytes. Compare cumulative sum, age-normalized average, and bounded history windows on a short synthetic attention trace. Specify newborn-token initialization, tie handling, and removal of retired score entries. Estimate diagnostic attention-matrix memory and show why retaining all steps' matrices can exceed the cache savings being studied.

Part 4 - Experimental Design and Instrumentation

Validate score arithmetic on hand-defined weights before model execution. Use zero dropout and explicit valid-key masks. Map every score back to request, absolute token, layer/head scope, and observation count. Compare instrumented attention outputs with the ordinary reference and report backend differences. Keep only bounded online aggregates in production measurements. A full-history oracle can diagnose missed evidence offline, but cannot supply scores for already removed tokens to an online policy without disclosing extra storage and computation.

Write the aggregation rule in explicit layer/head/query terms. Use query-head weights consistently under GQA and do not accidentally count duplicated storage as extra semantic evidence. State how invalid padding and fully masked queries are handled or excluded. Maintain observation counts separately from accumulated mass. A newborn-token rule and a bounded-history rule need independent tests because they change ranking even with identical raw attention weights.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class TokenAttentionEvidence:
    request_generation: int
    absolute_position: int
    accumulated_mass: float
    observation_count: int
    aggregation_identity: str


def update_attention_evidence(state, weights, token_ids, scope):
    # Validate normalization, head mapping, age handling, and bounded storage.
    ...


def rank_attention_tokens(evidence, ranking_contract):
    # Preserve stable identities, ties, and explicit unsupported observations.
    ...
```

```
def E07(config):
    # Validate scores independently from eviction and quantify monitoring cost.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report score definitions, counts, ranks, missing observations, temporary matrix peaks, and instrumentation overhead. A large accumulated weight does not prove semantic relevance or future use. Include an old low-use token and a newborn important token to expose age bias. Reject score updates attached to stale request generations or physical slot indices instead of token identity.

Compare an ordinary consumer with the diagnostic weight-producing reference for numerical parity and backend cost. Keep only bounded score vectors for currently tracked identities rather than every attention matrix from every step. Full-history offline scores may explain a lost fact, but supplying them to the online selector would add inaccessible information or uncounted work. Report unavailable weights explicitly. Attention evidence should remain associated with absolute token and request generation through every physical relocation.

Part 7 - Engineering Interpretation and Philosophy

Choose the smallest auditable attention signal suitable for later experiments. Explain its observational limitations and why a policy using it needs direct quality tests rather than an assumption that high mass always means valuable context.

19.39.9 Exercise 08

Exercise Title: Implement Heavy-Hitter KV Eviction That Retains High-Attention Tokens While Removing Low-Importance Cache Entries

Part 1 - Exercise Contract and Why It Matters

Retain high-ranked attention tokens and remove lower-ranked eligible entries under a fixed budget. Build an attention-based baseline inspired by heavy-hitter methods, with exact selection and update rules. Measure quality and real storage reclamation without claiming a reproduction of a published algorithm unless its complete protocol is implemented.

Part 2 - System and Memory Mental Model

Selection can use only evidence available before the decision. If new attention weights are needed to choose victims, the current step must already have admissible storage; it cannot exceed a strict budget invisibly. Retained positions can be sparse while preserving absolute model positions and causal visibility. Stable sorting, score initialization, protection, and prefill pruning affect behavior. Surviving KV still reflects its original computation history, not recomputation on the selected text.

Part 3 - Quantitative Prediction Before Execution

Hand-simulate a fixed budget with previous-step scores, a new token, ties, and a late important token. Predict victims and attention visibility before and after pruning. Count selection metadata and gather/compaction temporaries. Specify prefill treatment separately:

full-prompt prefill followed by pruning has a different peak from budget-aware incremental ingestion. Define quality metrics against full KV and simple recency before running the model.

Part 4 - Experimental Design and Instrumentation

Validate selector output using a CPU oracle and coordinate-coded KV. Use one retained-position set across layers initially. Reserve append capacity before dispatch, update online scores only at declared observation points, and publish coherent mapping changes. Compare on local and retrieval tasks with fixed weights, precision, positions, masks, and targets. Measure scoring, selection, relocation, attention, and complete decode. Test stale scores, unknown observations, repeated token IDs at different positions, and no eligible victim under protection.

Specify decision order for append, attention, score update, victim selection, and publication. A strict budget may require selecting from prior observations before a new token enters; a post-attention selector needs an explicitly reserved temporary slot. Prefill followed by pruning must report its larger initial peak. Use deterministic tie handling and newborn initialization so replays can identify whether a quality change comes from selection or incidental implementation order.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class HeavyHitterPolicy:
    token_budget: int
    score_contract: str
    append_admission_policy: str
    tie_policy: str
    prefill_policy: str


def choose_heavy_hitter_victims(history, evidence, policy):
    # Use available observations and preserve hard ownership/protection rules.
    ...


def apply_token_retention(cache, keep_positions, readiness):
    # Preserve K/V identity, positions, and all required lifetime boundaries.
    ...


def E08(config):
    # Validate selection then compare complete quality-capacity behavior.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report retained IDs, score ages, actual bytes, scoring overhead, full peaks, quality, and failures. Use a distractor case to test attention importance rather than assume it. Logical selection that only masks entries in a fully retained allocation must not be presented as reclaimed VRAM. Keep simulated selector success separate from actual model and CUDA validation.

Test a case with high-scoring distractors and a late low-observation fact. Preserve all candidate scores available at decision time, then compare actual retention with the CPU oracle. If physical compaction is absent, label a mask-only result as logical sparsity rather than reclaimed storage. Include selection and weight-observation cost in complete performance. A sound imitation of the broad heavy-hitter idea is not automatically a reproduction of every published method's per-head budgets and update schedule.

Part 7 - Engineering Interpretation and Philosophy

Explain where the measured attention signal helps and where recency or simple protection performs as well. State the exact policy scope and avoid transferring published quality or speedups to this transparent reference.

19.39.10 Exercise 09

Exercise Title: Combine Heavy-Hitter Retention with a Protected Recent Window and Compare It Against Pure Recency and Pure Attention Policies

Part 1 - Exercise Contract and Why It Matters

Combine a protected recent quota with attention-based retention under one fixed token budget. Compare pure recency, pure attention, and the hybrid with matched scoring and measurement rules. Test whether the hybrid addresses real failure modes rather than receiving extra hidden capacity.

Part 2 - System and Memory Mental Model

Recent protection and heavy-hitter sets can overlap. Count their union once, fill remaining capacity deterministically, and specify what happens when the history is shorter than the budget. The current token's admission belongs to the same capacity contract. Adding an initial sink quota is a separately labeled extension whose slots must come from the existing budget. No quota guarantees recovery of already deleted history.

Part 3 - Quantitative Prediction Before Execution

For budget B and recent quota R, predict the eligible historical quota and select from synthetic scores with overlaps and ties. Calculate actual union size, metadata, and append peak. Construct a local-dependency case, a long-range fact case, and an abrupt topic change. Predict how pure attention's age bias and pure recency's missing distant evidence could produce different outcomes, then leave model results to measurement.

Part 4 - Experimental Design and Instrumentation

Use identical prompts, targets, token budget, positions, precision, model, and attention consumer. Fix score aggregation and update timing. Sweep a small set of quota splits on calibration traces and validate held-out cases. Include no-protection and all-recent endpoints. Report monitoring overhead fairly; if pure recency does not require weights, its production path should not be charged unnecessary attention instrumentation except in a separately labeled matched-observation diagnostic. Test growth, short histories, and infeasible protected sets.

Use synthetic histories where every recent token is also a heavy hitter, where the two sets are disjoint, and where history is shorter than the quota. Define deterministic fill order so the retained set reaches capacity only when enough candidates exist. Keep any sink quota as an explicit ablation. The same bytes and semantic support must be available to every comparator; additional protected entries cannot be hidden outside the reported total.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class HybridRetentionPolicy:
    total_tokens: int
    recent_tokens: int
    attention_ranking: str
    overlap_fill_policy: str


def select_hybrid_positions(history, evidence, policy):
    # Form a deduplicated protected union and fill only remaining capacity.
    ...


def compare_retention_quotas(cases, policies, evaluation_contract):
    # Preserve equal budget and separate required instrumentation costs.
    ...


def E09(config):
    # Validate quota arithmetic and held-out hybrid quality trade-offs.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report actual quota use, retained positions, quality distributions, storage, selection cost, and complete latency. A hybrid that wins only by retaining more entries has not met the comparison contract. Preserve cases where the hybrid loses or the ranking is uncertain. A passing calibration result is not evidence that the chosen split is universally robust.

Compare both fixed equal-token budgets and actual block-rounded capacity when the implementation becomes paged. A recent quota can protect an entire partial physical block, changing usable remaining bytes. Report required instrumentation separately for policies that do not need attention weights. Use held-out abrupt topic changes to expose stale accumulation. Preserve per-request quality distributions and rejected protected-set configurations rather than presenting a favorable average as universal robustness.

Part 7 - Engineering Interpretation and Philosophy

Choose a quota split conditional on tested tasks and available signals. Explain which failure it mitigates and which context dependencies remain vulnerable under a finite approximation budget.

19.39.11 Exercise 10

Exercise Title: Build a Block-Level Eviction Policy for Paged KV Cache and Evict Physical Blocks Instead of Individual Token Entries

Part 1 - Exercise Contract and Why It Matters

Implement eviction at the physical KV block level and compare it with logical token pruning. Measure real reclaimable capacity, retained token overshoot, and quality under an equal byte budget. Keep block ownership and pending device readers as hard reclamation constraints.

Part 2 - System and Memory Mental Model

Selecting fewer tokens does not free a block that still contains live selected entries. Masking positions in a retained slab can save no backing. Block eviction removes every active mapping in its logical scope only under a declared approximation policy; otherwise it can reclaim only unused payload. Aggregate token scores to blocks using a named rule. Block size, partial tails, shared owners, and protected sink/recent entries change eligibility and reclaimed bytes.

Part 3 - Quantitative Prediction Before Execution

Construct a four-block example where one retained token remains in every block. Predict token sparsity and zero whole-block reclamation without compaction. Compare sum, mean, and maximum token-score aggregation on unequal valid lengths. Calculate block-rounded protected capacity and temporary copies for optional compaction. Predict which block can actually return to a reusable pool after mapping removal and reader completion.

Part 4 - Experimental Design and Instrumentation

Use checked block tables with request, layer, token-position, and generation identity. Start with one owner and synchronous retirement, then add controlled pending leases. Compare equal-budget token selection, whole-block selection, and a separately charged compaction variant. Preserve positions and causal masks after sparse retention. Evaluate quality on matched tasks and report score/selection/gather overhead. Recompute actual unique backing from the pool rather than inferring VRAM relief from a count of deleted table entries.

Give each physical block at least one selected token in a diagnostic trace and verify that whole-block reclamation remains zero without relocation. Then run a separately charged compaction control that preserves every selected ID and position. Protect retained source values from overlapping movement. Report score aggregation over valid tokens only; padding must not lower a block's average importance or create an apparently empty victim while valid entries remain.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
```

```
class KVEvictionBlock:
    handle: object
    absolute_positions: tuple
    valid_tokens: int
    capacity_bytes: int
    protected_positions: tuple
    ownership_state: object


def aggregate_block_scores(blocks, token_evidence, rule):
    # Define valid-length handling and keep hard eligibility separate.
    ...


def evict_physical_blocks(engine, victims, completion_evidence):
    # Apply supported logical changes and reclaim only safe whole backing.
    ...


def E10(config):
    # Audit the gap between sparse token retention and physical memory relief.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report selected tokens, retained blocks, protected overshoot, reclaimed capacity, free retained pool slots, compaction copies, quality, and latency. A reusable free slot still occupies backed pool capacity until destroyed or reused. Distinguish application admission relief from backend memory release. Do not claim successful block eviction when a shared owner or pending kernel still requires the payload.

Audit unique backing, mapped capacity, pending readers, and reusable slots after every deletion and completion. A returned free-list slot may improve application admission while keeping backend allocated memory constant. A slab cannot release an individual slot's backing independently. Compare quality at actual physical-byte budgets as well as logical token budgets. Include whole-block collateral loss when only one low-ranked token motivated a victim, so reclaimed bytes are assessed alongside the information removed.

Part 7 - Engineering Interpretation and Philosophy

Explain the quality and placement granularity purchased by whole-block policy. Choose a block strategy only after accounting for protection rounding, reclamation feasibility, and the consumer's actual working set.

19.39.12 Exercise 11

Exercise Title: Use KV Block Reference Counts to Prevent Eviction of Prefix Blocks Still Shared by Active Sequences

Part 1 - Exercise Contract and Why It Matters

Use checked reference ownership to prevent global reclamation of prefix blocks still needed by active requests. Separate one request detaching its logical mapping from deleting shared backing. Validate prefix lifetime under completion, cancellation, pruning, and delayed attention reads.

Part 2 - System and Memory Mental Model

Count active mapping entries, retained reusable-prefix ownership, and in-flight leases separately. A request may prune its own mapping under its approximation contract while another request keeps the shared prefix intact. That detach does not free physical storage. A zero active-request count may still leave a reusable cached prefix, and zero logical ownership may still leave pending readers. Protection by ownership is a hard gate, not a score that an aggressive policy can override.

Part 3 - Quantitative Prediction Before Execution

For three requests sharing two prefix blocks and owning private tails, predict counts and reusable capacity through multiple completion orders. Add one logical prune and one delayed read lease. Calculate unique backing versus total mapped capacity. Predict an attempted global eviction while one active owner remains and the correct rejection. Define whether duplicate mappings within one request are supported and how they contribute.

Part 4 - Experimental Design and Instrumentation

Route all attach/detach operations through one authority and independently recompute counts from live tables and cache records. Test duplicate detach, stale block generation, request-ID reuse, cancellation, and old completion after retirement. Validate each surviving request's KV and outputs against its declared retention reference. Keep host updates serialized initially. If mutating a shared tail, require copy-on-write before publication rather than assuming reference counts alone make writes safe.

Create three active mappings to one prefix block and detach them in different orders. Track inactive cache-retention ownership separately from request ownership, and retain one delayed attention lease after all mappings disappear. The block becomes globally reclaimable only when the supported cache owner is removed and all device obligations end. A request-local prune can be legal without producing any physical relief while another request still maps the same backing.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class KVBlockOwners:
    active_mappings: int
    reusable_prefix_owners: int
    inflight_leases: int
    block_generation: int


def can_reclaim_prefix(handle, owners, completion_evidence):
```

```
    # Enforce hard lifetime gates independently of any eviction score.
    ...


def detach_request_prefix(request, mapping, ownership_authority):
    # Remove only that request's ownership without invalidating other readers.
    ...


def E11(config):
    # Validate shared lifetime and delayed-reader reclamation cases.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require nonnegative independently audited counts and no reusable block reachable by an authorized mapping or lease. Report logical removals separately from physical relief. A prevented eviction is a correct outcome under active sharing. Verify that a delayed event references the exact generation it protected and cannot free newly assigned storage with the same numeric slot.

Recompute ownership from a mapping multiset rather than trusting the counters under test. Reject duplicate detach and stale generations before decrementing. A delayed callback for retired storage must not act on a new generation with the same slot ID. Test that pruning one owner never changes another's retained values or positions. Prefix mutation requires a separate copy-on-write protocol; reference counts alone authorize neither writes nor unsafe global eviction.

Part 7 - Engineering Interpretation and Philosophy

Explain why sharing changes which memory can be reclaimed without changing another request's computation. Preserve the distinction between a quality choice for one logical context and a global ownership obligation for backing used by several contexts.

19.39.13 Exercise 12

Exercise Title: Implement LRU Eviction for Unreferenced Prefix KV Blocks and Compare Cache Hit Rate Against FIFO Block Eviction

Part 1 - Exercise Contract and Why It Matters

Implement LRU and FIFO replacement for inactive reusable prefix blocks. Compare verified hit rate, avoided prefill work, time-to-first-token, and capacity under equal ownership constraints. This exercise evicts reusable cached results, not required active history.

Part 2 - System and Memory Mental Model

Eligible blocks have no active mapping or unfinished reader and can release their cache-retention ownership safely. LRU needs a declared last-use event; FIFO needs insertion order. Reusing a cached prefix must verify complete causal prefix identity, model context, format, positions, and generations. A miss recomputes from original input under the reference protocol, so correct replacement affects cost rather than authorizing context omission. Cache retention itself consumes host metadata and backed GPU capacity.

Part 3 - Quantitative Prediction Before Execution

Hand-simulate a finite prompt-arrival trace under equal byte or block capacity. Predict eligible victims, verified prefix hits, and skipped prefill positions. Include a scan pattern and a repeated-prefix pattern where policy rankings can differ. Distinguish block hit rate, token-weighted hit rate, and useful compute avoided. A full-prompt KV hit still requires a defined boundary-output policy for next-token logits.

Part 4 - Experimental Design and Instrumentation

Use canonical token prefixes and fixed model/format context. Record active reference transitions independently from replacement metadata. Test duplicate insertion, hit/touch updates, stale lookup after recycling, missing ancestor blocks, and forced identity mismatch. Compare LRU/FIFO on a fixed calibration trace and held-out arrivals. Preserve identical full inference outputs by recomputing misses. Measure index lookup, recomputation, retained capacity, and complete request latency. Invalidate lookup reachability before freeing backing.

Define LRU touch as a verified supported reuse event, not an unsuccessful lookup or a speculative probe, unless the policy explicitly chooses otherwise. FIFO insertion order needs a rule for duplicate content and reinsertion. Keep the same active-reference gates and byte capacity for both policies. A parent-prefix miss may prevent using a later cached block even when that block's token IDs match, so count verified usable prefixes rather than raw dictionary hits.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class InactivePrefixRecord:
    verified_identity: tuple
    handle: object
    insertion_order: int
    last_verified_use: int
    eligible: bool


def choose_prefix_replacement(records, required_capacity, policy):
    # Select only inactive safe candidates under the same byte budget.
    ...


def reuse_or_recompute_prefix(request, index, decoder_contract):
    # Verify hits and preserve complete inference semantics on misses.
    ...


def E12(config):
    # Compare cache replacement without conflating it with active pruning.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report eligible capacity, hit definitions, reused tokens, computed prefill, latency, metadata, and output parity. A higher block hit rate can save less work if hits cover smaller or cheap prefixes. No policy may reclaim an active block to improve its score. Retain miss costs and failed admissions in the result table rather than reporting only successful hits.

Verify complete outputs on misses by recomputing required history, including the final prompt boundary needed for logits. KV alone is not necessarily ready next-token output state. Report lookup, prefill skipped, prefill recomputed, retained capacity, and end-to-end request latency. Include a streaming scan that defeats recency and a reuse pattern favorable to it. Replacement quality here concerns saved work and latency, not permission to omit active semantic context.

Part 7 - Engineering Interpretation and Philosophy

Recommend replacement from observed request reuse and complete cost. Explain why this lossless reuse-cache experiment has a different correctness contract from approximate attention-history pruning, even though both use the word eviction.

19.39.14 Exercise 13

Exercise Title: Build a Composite Eviction Score from Recency, Attention Mass, Block Size, Prefix Sharing, and Estimated Future Reuse

Part 1 - Exercise Contract and Why It Matters

Build a composite candidate score using recency, attention evidence, size, sharing, and predicted reuse. Keep incompatible semantics in separate candidate classes and enforce ownership/resource feasibility before ranking. Compare against simple baselines with held-out quality and complete cost.

Part 2 - System and Memory Mental Model

Active-context pruning trades quality for capacity; inactive-prefix eviction trades possible future recomputation for capacity. Their raw scores are not interchangeable. Normalize feature units and state the objective within each class. Active owners and pending readers are hard eligibility gates, not soft penalties. A shared block's physical relief can be zero when only one mapping is detached. Future-reuse estimates cannot use unseen answers or future traces in an online policy.

Part 3 - Quantitative Prediction Before Execution

Construct candidates with equal attention mass but different byte sizes, and candidates with equal recency but different sharing. Predict actual reclaimable bytes and compare a weighted score with a cost-per-byte formulation. Include uncertainty in estimated reuse and quality damage. Calculate metadata and selection overhead. Define what the controller does when every high-scoring candidate is ineligible or no action meets quality/resource gates.

Part 4 - Experimental Design and Instrumentation

Start with a small feature set and documented normalization. Calibrate weights on separate traces, then freeze them for evaluation. Compare recency, attention-only, activity-aware, and composite policies using matched budgets and tasks. Add feature ablations to identify mechanisms. Preserve per-request quality and fairness, not only aggregate token throughput. Record

every candidate's semantic class, evidence age, eligibility reason, predicted relief, chosen action, and later outcome in bounded scalar logs.

Attach units and support status to every feature. Recency in steps, bytes, attention mass, owner count, and expected reload time cannot be summed as uninterpreted raw numbers. Enforce zero-ownership and completion rules before ranking candidates. A large sharing feature must never make active backing eligible. Use actual reclaimable bytes as the resource effect, not the size of a tensor whose storage another mapping continues to retain.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class KVEvictionCandidate:
    action_semantics: str
    reclaimable_bytes: int
    recency_feature: object
    attention_feature: object
    sharing_state: object
    predicted_reuse: object
    eligible: bool
    uncertainty: object


def score_eviction_candidates(candidates, objective_contract):
    # Normalize valid features after hard lifetime and semantic gates.
    ...


def evaluate_composite_policy(cases, baselines, held_out_contract):
    # Separate prediction quality from actual quality/capacity/latency outcomes.
    ...


def E13(config):
    # Validate scoring with counterexamples and complete held-out comparisons.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report raw features, normalization, selected actions, real reclaimed bytes, quality, hit/recompute cost, monitoring overhead, and failures. Preserve unknown features as unknown instead of substituting zeros. A fitted score is not a proof of causal importance. Include a case where the composite loses to a simple baseline and state whether that outcome changes the recommendation.

Keep separate objective and outcome tables for approximate active pruning and inactive-prefix replacement. A coefficient fitted to retrieval quality cannot be assumed meaningful

for future prefill reuse. Evaluate feature ablations and uncertainty on held-out traces. If all features are stale or unsupported, use a declared simple fallback or reject the action. A score that predicts its own labels well may still fail to improve complete quality, capacity, or latency.

Part 7 - Engineering Interpretation and Philosophy

Choose the smallest policy supported by measured benefit and uncertainty. Explain which features predict a useful decision in the tested semantic class and which add complexity without improving complete work.

19.39.15 Exercise 14

Exercise Title: Construct an Adversarial Structured Prompt Where Attention-Based Eviction Retains Irrelevant Structural Tokens and Diagnose the Failure

Part 1 - Exercise Contract and Why It Matters

Construct a controlled structured-prompt test for the hypothesis that attention-based retention can overprotect structural tokens while losing answer-bearing evidence. Diagnose an observed failure or report that the hypothesis was not reproduced. Do not invent a model outcome to satisfy the exercise title.

Part 2 - System and Memory Mental Model

Delimiter, role, punctuation, and repeated-template tokens can affect attention and may also carry genuine task structure. High attention mass is not sufficient evidence that they are irrelevant, nor is a human structural label sufficient evidence that they are safe to remove. The experiment must link retained identities, lost evidence, and an actual task outcome. Sink behavior and semantic content are competing explanations to test.

Part 3 - Quantitative Prediction Before Execution

Design prompts containing a known answer-bearing fact, repeated structural scaffolding, distractors, and a controlled query. Predict which positions are likely candidates under the declared score, but leave observed ranks unknown. Specify equal-budget controls with structure reduced, evidence relocated, and a retention oracle used only offline. Define retrieval scoring and model baseline competence before inspecting eviction results.

Part 4 - Experimental Design and Instrumentation

Use fixed model/tokenizer and seeded variations of layout, delimiter frequency, fact distance, wording, and query location. Compare full KV, pure attention, recency, and the hybrid. Record token-level labels, attention ranks, retained positions, and generated/teacher-forced outcomes. Hold task content constant where a control changes only formatting and record token-count differences. Reduce any failure to a smaller prompt preserving the mechanism. Report instrumentation workspace and selection overhead separately from production performance.

Use multiple structured templates with the same answer-bearing content, and controls where delimiters or role labels actually carry the answer. Preserve tokenizer revision and annotate exact token positions rather than human-visible character spans alone. Record baseline model competence, evidence distance, structural token count, and retained ranks. If formatting changes token count, match or report that difference instead of assuming only one variable changed.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class StructuredRetentionCase:
    case_id: str
    token_ids: tuple
    structural_positions: tuple
    evidence_positions: tuple
    expected_answer: str
    control_family: str


def build_structured_cases(spec, tokenizer, seed):
    # Create matched, auditable cases without assuming the hypothesized failure.
    ...


def diagnose_structural_bias(result_rows, retained_position_logs):
    # Test rank, evidence loss, and outcome links with controlled alternatives.
    ...


def E14(config):
    # Reproduce and minimize a failure, or explicitly record no reproduction.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report full-cache competence, ranks, structural/evidence retention, quality, bytes, and every tested control. A correlation between structural mass and failure is not enough to establish the mechanism; controlled replays should change the suspected choice while preserving other conditions. Do not discard unfavorable or non-reproducing cases. Human labels are diagnostic annotations, not ground-truth attention importance.

Require the diagnosis to connect a specific victim choice with lost evidence and a changed outcome through controlled replay. An offline oracle can force retention to test the mechanism, but must not be presented as an online policy. Reduce an observed failure without removing the critical structure or fact. If the trained model, attention signal, or hardware is unavailable, provide only the designed protocol and mark results unmeasured. No hypothesized failure may become a fabricated observation.

Part 7 - Engineering Interpretation and Philosophy

Explain the strongest supported failure mechanism and its limits. Use the counterexample to motivate a testable filter, while preserving the possibility that structural tokens are useful and aggressive removal can worsen quality.

19.39.16 Exercise 15

Exercise Title: Build a Role-Aware or Content-Aware Eviction Filter That Reduces Structural-Token Bias Without Increasing the KV Memory Budget

Part 1 - Exercise Contract and Why It Matters

Add a role- or content-aware retention filter to test the diagnosed structural bias without increasing the KV budget. Preserve protected lifetime rules and compare against an unchanged attention selector. Treat labels as imperfect features whose usefulness must be measured on held-out prompts.

Part 2 - System and Memory Mental Model

Role boundaries, delimiters, punctuation, and template tokens may carry important instructions, syntax, or evidence. A blanket structural-token ban can damage the task. Labels must map tokenizer spans to exact absolute token positions and model/chat-template revision. The filter changes ranking or eligibility under a declared approximation contract, not the text presented to the original model. Sink protection and semantic role rules can overlap and must share the same total capacity.

Part 3 - Quantitative Prediction Before Execution

Define a small label schema and a deterministic filter or score adjustment. Predict retained sets on the minimized failure and controls where structure is answer-bearing. Calculate quota overlap, byte rounding, label metadata, and selection cost. Compare equal token and equal physical-byte budgets explicitly. State what happens when protected labels exceed capacity rather than silently expanding the cache.

Part 4 - Experimental Design and Instrumentation

Calibrate on separate structured cases and freeze labels/weights before held-out evaluation. Use exact tokenizer offsets or a validated alignment protocol. Compare no filter, role-only, content-only, and a minimal combined variant. Include nested structure, code, repeated delimiters, meaningful role headers, and facts split across subword tokens. Preserve model, positions, masks, precision, target protocol, and total budget. Test ordinary text and retrieval to detect regressions beyond the motivating failure.

Validate alignment from role/content spans to subword token IDs, including facts split across multiple tokens and nested structured fields. Record labels as features with uncertainty rather than immutable truth. Keep a control where punctuation, code syntax, or a role boundary is necessary. If sink/recent protection overlaps a content label, deduplicate it. All additional labels and scores consume metadata capacity even when the number of retained KV entries stays fixed.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class TokenRetentionLabel:
    absolute_position: int
```

```
    role: str
    content_class: str
    alignment_identity: str
    uncertainty: object


def label_retention_tokens(prompt, tokenizer, template_contract):
    # Align imperfect labels to exact model token positions.
    ...


def filter_eviction_candidates(candidates, labels, policy, budget):
    # Adjust only supported choices without adding hidden capacity.
    ...


def E15(config):
    # Test the targeted fix and held-out structural/content counterexamples.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report alignment errors, label uncertainty, retained evidence, quality, physical capacity, metadata, overhead, and regressions. A fix that wins only on the constructed prompt is not general validation. Include a case where structural tokens are necessary and the filter should retain them or fail honestly. If no robust gain appears, recommend the simpler baseline.

Test whether the filter fixes the minimized failure while preserving ordinary retrieval and local-continuation quality. Compare the same byte budget after block rounding; token counts alone can hide extra backing. Use held-out templates and task domains to reveal overfitting. Report no improvement when warranted. A role-aware filter is a heuristic retention rule, not a general proof that one category of prompt tokens is dispensable or that attention magnitude has been causally corrected.

Part 7 - Engineering Interpretation and Philosophy

Explain what the filter actually learned from available annotations and where those annotations are unreliable. Prefer a small tested correction to a universal claim that attention or a structural label reveals semantic value.

19.39.17 Exercise 16

Exercise Title: Dynamically Increase the KV Budget When Model Confidence Falls and Prune More Aggressively When Next-Token Confidence Is High

Part 1 - Exercise Contract and Why It Matters

Adapt the retained-token budget using an explicit next-token confidence signal, bounded capacity, and calibrated uncertainty. Increase available retention when the signal warrants it and prune more only under the tested policy. Evaluate high-confidence errors and the irreversibility of previous context deletion.

Part 2 - System and Memory Mental Model

Maximum next-token probability, entropy, and logit margin are different signals and depend on temperature and evaluation conventions. None is a direct proof that older context is unnecessary. A confident wrong answer can follow missing evidence. Increasing the budget prevents some future deletions but does not restore already discarded KV. Restoration requires explicit CPU backing or recomputation from retained input under a supported protocol, including its capacity, time, and state-transition cost.

Part 3 - Quantitative Prediction Before Execution

Define one confidence statistic from unsampled logits under a fixed convention. Hand-simulate low/high thresholds, hysteresis, minimum and maximum budgets, and shared global headroom. Include confident-wrong and uncertain-correct controls. Predict what a budget increase can recover with backing, without backing, and with a charged recomputation path. Estimate transition demand rather than treating a larger integer limit as available memory.

Part 4 - Experimental Design and Instrumentation

Calibrate confidence against task outcomes on separate traces, then freeze the policy for held-out tests. Keep full-reference outputs unavailable to the online decision unless their cost is explicitly included. Bound budget changes and preserve request fairness. Test topic changes, delayed evidence needs, and exhausted physical headroom. Compare fixed-budget, recency-adaptive, and confidence-adaptive controls under matching resource envelopes. Record the action, available evidence, recovery support, and later quality without claiming a universal online accuracy guarantee.

Choose one statistic and freeze its temperature, vocabulary, masking, and sampling convention. Compute confidence from the appropriate current logits, not from future reference answers. Entropy and maximum probability may rank cases differently, so evaluate them separately before selecting one. Include high-confidence wrong retrieval and low-confidence correct continuation cases. Budget adaptation must respect per-request minima, a global maximum, and fairness for other active requests.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class ConfidenceRetentionEvidence:
    statistic_name: str
    value: float
    temperature_convention: str
    calibrated_domain: str
    current_budget: int
    recoverable_history: bool
    available_capacity_bytes: int


def choose_confidence_budget(evidence, policy, ownership_state):
```

```
    # Respect hard headroom, calibration limits, and restoration availability.
    ...


def apply_budget_change(plan, engine):
    # Distinguish future protection from explicit history reconstruction.
    ...


def E16(config):
    # Evaluate confidence failures and bounded adaptive retention honestly.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report confidence/quality calibration, chosen budgets, actual bytes, reconstruction work, latency, and rejected actions. A high confidence score must not override ownership or capacity limits. Do not label a larger budget as restored context when missing entries remain absent. Preserve cases where the adaptive policy loses despite a plausible confidence signal.

Test a budget increase after irreversible deletion and require an honest record of which positions remain missing. If recovery is implemented, charge original-token storage, CPU backing, transfers, and model recomputation. Rebuilt history cannot repair outputs already emitted. Report calibration quality on held-out tasks and domain changes; a good calibration curve does not guarantee every individual retention action. Use hysteresis and a measured fallback rather than treating confidence as an unconditional accuracy certificate.

Part 7 - Engineering Interpretation and Philosophy

Explain whether confidence predicts a useful retention choice in the tested domain. Choose conservative limits or a fixed policy when uncertainty, irreversible loss, or overhead makes adaptation unjustified.

19.39.18 Exercise 17

Exercise Title: Combine Eviction, Quantization, and CPU Offloading into a Three-Level KV Policy That Chooses Delete, Compress, Offload, or Keep

Part 1 - Exercise Contract and Why It Matters

Combine keep, compress, offload, and permanent logical deletion in one explicit KV decision policy. Use three named retained-representation levels and a separate deleted state. Compare capacity, quality, transfer, reconstruction, and complete inference cost without treating these actions as interchangeable.

Part 2 - System and Memory Mental Model

Keeping full-width GPU KV preserves current representation; quantizing it adds approximation; CPU offloading preserves the chosen representation but requires restoration; deleting active history changes future attention. Deleting an inactive reusable prefix has different semantics. Track authority, format, shared ownership, and completion. Promoting compressed data does not recover lost precision, and increasing residency cannot resurrect deleted values without

backing or recomputation. Subsequent retained KV may already reflect earlier approximation history.

Part 3 - Quantitative Prediction Before Execution

For each candidate block, predict final and transitional GPU bytes, host backing, code metadata, decoded workspace, transfer cost, and quality evidence. Compare a GPU-float, GPU-compressed, CPU-backed hierarchy with a deleted state, recording CPU format explicitly. Construct actions whose final state fits but whose conversion peak fails. Define hard eligibility and calibrated quality gates before using a composite cost objective.

Part 4 - Experimental Design and Instrumentation

Integrate the validated codecs, transfer tickets, tables, reference counts, and retention rules through one authority. Start with deterministic transitions before adding concurrency. Test shared blocks with conflicting desired actions, dirty versions, partial tails, compression before offload, and active-reader retirement. Compare fixed keep-only, lossless offload, uniform compression, pruning-only, and combined policies on matched traces. Preserve explicit masks, positions, model context, and calibration identities. Count original-token logs, host originals, and recomputation if they provide recovery.

For each action, record visibility change, representation change, authority, recoverability, and actual capacity relief. CPU-backed quantized data should retain its format/version metadata just as GPU data does. Shared owners may disagree about approximation, requiring a compatible common policy or private replacement. Count old readers, new encoded payload, decoded workspace, and pinned staging during transitions. A final low-byte state is insufficient if the path to it exceeds the admitted budget.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class KVHierarchyAction:
    block_identity: tuple
    semantic_scope: str
    action: str
    target_format: tuple
    final_gpu_bytes: int
    transition_gpu_bytes: int
    host_bytes: int
    quality_evidence: object


def plan_kv_hierarchy_actions(candidates, budgets, quality_contract):
    # Gate lifetime, recovery, and semantic consequences before ranking.
    ...


def commit_kv_hierarchy_action(plan, readiness, ownership_authority):
```

```
    # Publish exact-version representation or visibility changes safely.
    ...


def E17(config):
    # Validate each action and compare the complete integrated trade-off.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report each action class, actual relief, host growth, copies/transfers, retained positions, quality, and complete latency. Do not claim a lossless combined policy when it deletes active context or quantizes values. Prevent cached lookup entries from exposing recycled or incompatible approximate prefixes. A rejected transition is correct when no action satisfies resource and declared quality prerequisites.

Compare one-shot and repeated approximation histories explicitly. Wider codes cannot recover values lost earlier, and restoring a page to GPU does not restore deleted positions. Prefix reuse identity must account for compatible retention/precision trajectories or a canonical reconstruction protocol, not only matching text tokens. Track host originals and token logs if they enable recovery. Keep inactive-prefix eviction separate so a lossless miss/recompute path is not confused with permanent active-context pruning.

Part 7 - Engineering Interpretation and Philosophy

Explain which hierarchy level solves which bottleneck and what information can actually be recovered. Recommend the smallest action set supported by measurements, keeping empirical accuracy limits distinct from hard owned-byte and lifetime guarantees.

19.39.19 Exercise 18

Exercise Title: Stress-Test Adaptive KV Eviction with Long Retrieval Tasks, Shared Prefixes, Multiple Concurrent Requests, Abrupt Topic Changes, and Severe VRAM Pressure

Part 1 - Exercise Contract and Why It Matters

Stress-test retention and hierarchy policies with long retrieval tasks, shared prefixes, concurrent requests, topic changes, and small bounded KV budgets. Search for minimal quality and lifecycle counterexamples. Separate expected approximation failures from corrupted positions, stale metadata, and unsafe storage reuse.

Part 2 - System and Memory Mental Model

Abrupt topic changes can make accumulated attention stale, while shared prefixes constrain physical reclamation. Concurrent admission introduces fairness and transition peaks. A low-memory run can be operationally correct but fail its retrieval threshold. A high-quality sample can conceal leaked or still-live blocks. Maintain independent semantic-position, ownership, resource, and task-quality checks, with explicit recovery assumptions for deleted or compressed history.

Part 3 - Quantitative Prediction Before Execution

Predict a short trace involving shared protection, middle pruning, a late retrieval query, and request cancellation. Calculate maximum live backing, provisional copies, score metadata, and

offload staging. Define quality failures and expected admission rejections in advance. Include impossible working-set and protection-union cases whose correct result is refusal rather than unlimited progress. State the model's positional and context-range support.

Part 4 - Experimental Design and Instrumentation

Use fixed seeds, finite arrival traces, known-answer tasks, bounded output lengths, host/GPU budgets, and pending-operation limits. Begin with CPU selector and deterministic completion simulation before actual supported CUDA runs. Keep policy calibration separate from held-out stress. Check retained IDs and code/format integrity after each event and model outcomes at selected checkpoints. Inject preparation failures before publication, not illegal device races. Save failing traces and minimize them without removing required prefix population, calibration, or sharing setup.

Separate deliberate rejection controls from failures in valid histories. Save task, model, and calibration identifiers, retained sets, and completion order along with seeds. Actual asynchronous timing may not reproduce from a seed alone. Start with deterministic examples of protected-union overflow, old-score domination after a topic shift, and a shared block with no physical relief. Bound score histories and diagnostic payload so stress monitoring cannot become an unreported leak.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class AdaptiveRetentionStressBounds:
    maximum_requests: int
    maximum_context_tokens: int
    gpu_bytes: int
    host_bytes: int
    maximum_pending_actions: int
    steps: int


def generate_retention_trace(seed, bounds, task_contract):
    # Mix supported histories, topic shifts, and labeled invalid controls.
    ...


def audit_adaptive_retention(engine, position_oracle, ownership_oracle):
    # Separate exact invariants from empirical quality and resource evidence.
    ...


def minimize_retention_failure(trace, replay):
    # Preserve model/task/calibration and asynchronous setup dependencies.
    ...
```

```
def E18(config):
    # Test adversarial small traces before bounded integrated stress.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report seeds, exact histories, failure phases, smallest counterexamples, reproducibility, quality thresholds, and actual execution support. Include stale scores, protected-set overflow, shared-block reclamation rejection, confidence miscalibration, lost retrieval facts, and shutdown with pending readers. An unobserved or unreproduced failure remains a stated limitation, not permission to invent hardware or quality evidence.

Audit after final drain and controlled cancellation, including old table versions, shared owners, transfer endpoints, and pending score updates. Minimize a failure only while preserving the setup that causes it; a missing prefix initialization is a different error. Report quality failures, operational corruption, infeasible admission, and unsupported execution separately. A finite clean trace supports a tested-envelope claim, not proof that every future context, concurrency pattern, or confidence transition is safe.

Part 7 - Engineering Interpretation and Philosophy

Identify the highest-impact remaining uncertainty and its next bounded experiment. State the tested envelope without claiming universal quality, progress, or concurrency correctness from a finite stress run.

19.39.20 Exercise 19

Exercise Title: Build and Benchmark a PyTorch Adaptive KV Cache Manager That Dynamically Chooses Which Context to Keep, Compress, Offload, or Permanently Evict Under Accuracy and VRAM Constraints

Part 1 - Exercise Contract and Why It Matters

Integrate an adaptive manager choosing keep, compress, offload, or delete under explicit quality and resource contracts. Benchmark completed inference, context retention, prefix reuse, and recovery cost. Deliver an auditable support matrix and held-out evidence; the scaffold leaves substantive implementation to the learner.

Part 2 - System and Memory Mental Model

The manager coordinates active visibility, absolute positions, per-layer validity, scores, representation, residency, shared owners, and pending users. Separate active pruning from inactive-prefix replacement throughout the API. Quality is an empirical approximation envelope unless a specified online oracle is genuinely available and charged. The attention consumer defines working-set needs and monitoring overhead. One authority must prevent a score decision from bypassing ownership, generation, or complete transition admission.

Part 3 - Quantitative Prediction Before Execution

Predict useful context, physical capacity, score/label/index metadata, compressed parameters, residuals, host backing, reconstruction workspace, and pending old/new copies. Compare full KV, recency, attention-only, hybrid, role-aware, and integrated hierarchy policies on equal supported traces. Define quality-qualified capacity and include latency/fairness constraints.

Estimate both steady savings and transient peaks. Distinguish analytical capacity from tested completed requests.

Part 4 - Experimental Design and Instrumentation

Freeze support for positional handling, model context, grouping, sharing, append, pruning, compression, migration, restoration, cancellation, and shutdown. Validate synthetic selectors, codecs, ownership, and masks before model quality. Use separate calibration, selection, and held-out evaluation. Record software, hardware, model/tokenizer revisions, precision, attention backend, seeds, budgets, task identities, and initial state. Include prefill, instrumentation, selection, compaction, transfers, reconstruction, attention, and drain in complete cost. Run ablations without loosening correctness or resource gates.

Publish a support matrix before performance: active pruning, inactive-prefix replacement, sink/recent protection, scoring, label alignment, quantization, offload, restoration, cancellation, and shutdown each need precise semantics. Keep attention instrumentation and consumer changes visible. Use a trained model for meaningful quality results and a tiny reference for exact state invariants. Separate analytical context capacity from actual quality-qualified completed inference and retain the fixed model's positional limits.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class AdaptiveKVManagerContract:
    model_context: tuple
    position_policy: str
    gpu_bytes: int
    host_bytes: int
    attention_consumer: str
    quality_policy_identity: str
    recoverable_history_policy: str
    supported_actions: tuple

class AdaptiveKVCacheManager:
    def __init__(self, contract):
        # Establish tables, authority, scores, formats, residency, and budgets.
        ...

    def observe(self, request, model_evidence):
        # Retain only supported bounded signals with age and uncertainty.
        ...

    def plan(self, demands):
        # Gate semantics, lifetime, quality evidence, fairness, and full peaks.
        ...
```

```
    def step(self):
        # Order admitted work and commit coherent context/representation state.
        ...

    def audit(self):
        # Recompute token visibility, ownership, formats, and unique backing.
        ...

    def shutdown(self):
        # Drain or safely cancel owned work and report unresolved obligations.
        ...


def E19(config):
    # Validate integrated contracts before held-out quality/capacity benchmarks.
    ...
    # Export evidence, counterexamples, limitations, and recommendation.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Acceptance requires correct retained IDs, positions, masks, format decoding, shared lifetime, bounded resources, and declared tested quality. Report perplexity with scored counts, retrieval outcomes, retained evidence, prefix hits/recompute work, confidence behavior, action counts, byte peaks, latency, and rejected/unfinished requests. Keep predictions, observations, and interpretations separate. An unfinished scaffold supplies no measured hardware or model-quality result.

Report an unfavorable policy case, a confident wrong prediction, a structural counterexample or honest non-reproduction, a shared-block rejection, and a transition with insufficient headroom. Include rejected and unfinished requests alongside successful survivors. Hard byte/lifetime invariants and empirical accuracy evidence have different strengths. Recommend the simplest supported policy with its measured trade-off and next required experiment, without inventing speedups, retrieval scores, or universal accuracy guarantees.

Part 7 - Engineering Interpretation and Philosophy

Recommend the smallest policy supported by complete evidence, including full retention when approximation is unjustified. Explain what was kept, changed, moved, or permanently lost and the capacity-quality-latency exchange. Separate proposed optimized kernels or broader model support from tested capabilities and remaining uncertainty.

19.40 DMM38

Chapter Title: Fully Sharded Data Parallel (FSDP) Memory Management
Specialization: Dynamic Memory Management and CUDA Systems

19.40.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Parameter, Gradient, and Optimizer-State Sharding Across Multiple GPUs

Part 1 - Exercise Contract and Why It Matters

Model parameter, gradient, and optimizer ownership across ranks and time. Deliver a diagram separating persistent shards, temporary gathered units, activations, collective buffers, and optimizer updates. Use one CUDA device per process on one multi-GPU host; two physical computers are not required. Capacity claims must include initialization and complete training, not only idle post-wrap state.

Part 2 - System and Memory Mental Model

The primary API track is FSDP1, the FullyShardedDataParallel wrapper, because these exercises study its sharding strategies, auto-wrap policies, and rate limiter. Pin the installed version and do not mix its constructor controls with FSDP2 fully_shard, DTensor state, or FSDP2 precision/offload policies. FSDP2 is a separate supported-design comparison. Under full sharding, local persistent state shrinks while full parameters for a computation unit are materialized when needed. Activations and ignored or replicated objects do not automatically divide by world size.

Part 3 - Quantitative Prediction Before Execution

For a named all-trainable model, derive an ideal persistent-state estimate from parameter, gradient, and optimizer bytes divided by sharding-group size. Add replicated state, padding, gathered units, prefetch overlap, and workspace separately. Explain why

$$M_{\text{peak}} \neq M_{\text{single}}/W$$

in general. Distinguish global world size from the size of a particular sharding group and calculate a one-unit versus nested-unit example before execution.

Part 4 - Experimental Design and Instrumentation

Use a small deterministic model first, fixed precision and optimizer, identical initial weights, and one forward/backward/update per step. Bind LOCAL_RANK to its CUDA device, initialize the declared process group, and verify rank-consistent topology and configuration. Construct the optimizer after FSDP wrapping. Record model/wrap manifest, software and NCCL versions, device/link topology, seeds, batches, allocator backend, and initial-state ID. Start without accumulation, CPU offload, or mixed precision so later changes are controlled.

Write the rank map and collective groups before launching. Distinguish local CUDA index, global rank, and group-local rank even when a single-host example makes their numbers coincide. Freeze the process count and model/wrap manifest for a trial. Include a rank-consistency check before the first model collective and a finite failure protocol. Configuration disagreement should stop the trial rather than enter incompatible collective order and wait indefinitely.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
import torch.distributed as dist
from torch.distributed.fsdp import FullyShardedDataParallel as FSDP

@dataclass
```

```
class FSDPExperimentContract:
    api_track: str
    world_size: int
    sharding_group_size: int
    per_rank_batch: int
    optimizer_identity: str
    wrap_manifest_identity: str


def predict_rank_memory(contract, model_inventory, execution_plan):
    # Separate persistent ownership and temporary computation/collective state.
    ...


def E00(config):
    # Validate rank/device setup and the version-pinned FSDP1 experiment model.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require per-rank phase records and a declared aggregation: maximum rank peak, rank distribution, and useful global throughput. Keep predictions, observations, and interpretation separate. Do not fabricate multi-GPU results when only a CPU model or fewer devices are available. All ranks must execute compatible collectives; a local failure cannot safely be hidden while peers continue the same training loop.

Define the training boundary as forward, loss, backward, optimizer update, and the declared gradient-clearing point. State whether data transfer, collation, checkpointing, and recovery are timed. Require fixed-seed numerical checks and additional seeds for repeatability, without claiming all kernels are bitwise deterministic. The evidence ledger should distinguish initialization, first update, and warmed execution on every rank, then report the maximum operational peak rather than only rank zero's convenient observation.

Part 7 - Engineering Interpretation and Philosophy

Explain the memory-communication exchange and the minimum unit that still must fit on one GPU. A sharded runtime increases feasible capacity only when its transient computation, initialization, and communication also fit the available hardware.

19.40.2 Exercise 01

Exercise Title: Measure Per-GPU Parameter, Gradient, and Optimizer-State Memory in Standard DistributedDataParallel as a Baseline

Part 1 - Exercise Contract and Why It Matters

Measure standard DistributedDataParallel model, gradient, optimizer-state, and communication residency per GPU. Establish a complete-step baseline with identical initial values and useful global data accounting. Separate startup, first optimizer update, and warmed training.

Part 2 - System and Memory Mental Model

Ordinary DDP replicates model parameters on every rank and synchronizes gradients. An ordinary per-rank optimizer also maintains replicated state; this baseline does not use a sharded optimizer. Optimizer tensors may appear lazily after the first update. Gradient buckets can alias gradient storage under certain settings, so logical gradient bytes and unique backing are different. Buffers, activations, input batches, allocator reservation, and communication-library allocations need separate accounting.

Part 3 - Quantitative Prediction Before Execution

For an all-trainable float32 parameter count P, calculate parameter and gradient payload, then derive state for the actual optimizer and observed state dtypes. An Adam-like two-moment estimate is not a universal optimizer formula. Predict the first-step increase from lazy state and possible foreach workspace. Define global batch from world size, local batch, and accumulation count, with accumulation fixed to one in the initial baseline.

Part 4 - Experimental Design and Instrumentation

Initialize identical model values, shard the dataset explicitly, and keep rank-local work shapes matched. Fix DDP bucket and gradient-view settings and record them. Measure before forward, after forward, after backward, after update, and after gradient clearing at deliberate diagnostic boundaries. Use a separate minimally instrumented run for complete step time. Capture CPU-only scalar inventories without retaining graphs or duplicate GPU tensors. Compare losses and a small reference update under matching loss normalization.

Enumerate actual optimizer tensor fields after the first update instead of assuming an empty state dictionary means the optimizer has no memory cost. Record step counters and non-tensor state separately from large tensor moments. Choose a fixed gradient-bucket view policy and inspect unique backing after warm-up. Test gradient clearing with the declared set-to-none behavior, because it changes whether gradient payload remains live at the next observation.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
from torch.nn.parallel import DistributedDataParallel as DDP

@dataclass
class RankStateInventory:
    rank: int
    phase: str
    parameter_bytes: int
    gradient_bytes: int
    optimizer_bytes: int
    communication_scope: str


def inventory_ddp_state(model, optimizer, phase, measurement_contract):
    # Deduplicate backing and record lazy state, views, and unsupported bytes.
    ...
```

```
def run_ddp_reference_step(model, optimizer, local_batch):
    # Preserve distributed data/loss semantics through a complete update.
    ...


def E01(config):
    # Establish per-rank memory and useful global-step reference evidence.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report actual state tensors, dtypes, phases, unique backing, allocated and reserved peaks, step duration, and global sample count. Do not attribute every residual byte to DDP or sum aliased buckets twice. Rank timing must include completed CUDA work. A first-step measurement cannot stand in for warmed steady state when optimizer initialization or bucket rebuilding changes memory.

Use deterministic distinct sample IDs on each rank and compare their global multiset with the intended batch. DDP does not by itself prove the dataset was sharded correctly. Equal local mean losses need compatible normalization for a matching global update. Avoid collecting every rank's activations on rank zero for validation. Export reduced scalar checks or a small deliberate parameter-inspection replay outside the primary memory and timing region.

Part 7 - Engineering Interpretation and Philosophy

Explain which replicated components FSDP can target and which remain driven by workload or kernel execution. Preserve the DDP baseline as both a numerical reference and a clearly scoped resource measurement.

19.40.3 Exercise 02

Exercise Title: Wrap a PyTorch Model with FSDP and Verify That Model State Is Sharded Across the Distributed World Size

Part 1 - Exercise Contract and Why It Matters

Wrap a small model with version-pinned FSDP1 full sharding and verify persistent ownership of parameters, gradients, and optimizer state. Use the same data-parallel training semantics as DDP. Demonstrate sharding with an inventory rather than inferring it from a wrapper name.

Part 2 - System and Memory Mental Model

Managed parameters can be flattened and replaced by views during execution. Use a declared use_orig_params setting consistently; the primary simple track uses the flattened-parameter mode. Outside computation, persistent managed storage is sharded; forward and backward may temporarily expose gathered values. Buffers and ignored modules may remain replicated. The optimizer must reference wrapped parameters, and its state must be measured after a real update rather than assumed present at construction.

Part 3 - Quantitative Prediction Before Execution

Predict local managed payload from each unit's global elements and sharding group size, then include padding and unsharded objects separately. Calculate expected gradient and optimizer ownership after synchronization/update under fixed precision. Predict the larger forward materialization. Explain why inspecting parameters inside a full-parameter gathering context would change the state being measured.

Part 4 - Experimental Design and Instrumentation

Bind rank/device, initialize the group, create the same model and wrap manifest on every rank, then build the optimizer. Use supported device and initial-state synchronization rules. Run one deterministic training step and compare outputs or reconstructed small-model values with the DDP reference at a deliberate validation boundary. Inventory persistent state after each phase and note parameter-view identity changes. Avoid calling an unwrapped internal submodule directly when its FSDP hooks would be bypassed.

Record which parameter objects the optimizer receives after wrapping and reject obsolete references from the unwrapped model. Do not keep saved parameter views across forward calls when FSDP changes their backing interpretation. Start with all parameters trainable and a simple supported ownership layout; frozen/tied mixtures are separate support tests. Verify the chosen device and initial-state synchronization path on every rank before training.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
import torch.distributed as dist
from torch.distributed.fsdp import FullyShardedDataParallel as FSDP
from torch.distributed.fsdp import ShardingStrategy


def construct_fsdp1_model(model_spec, distributed_context, wrap_contract):
    # Build matching units and validate the chosen full-shard API settings.
    ...


def construct_wrapped_optimizer(sharded_model, optimizer_spec):
    # Bind optimizer ownership after wrapping, not to obsolete parameters.
    ...


def verify_persistent_shards(model, optimizer, inventory_contract):
    # Measure supported post-phase local backing without forcing full gathers.
    ...


def E02(config):
    # Validate small-model sharding and one complete distributed update.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report rank, group, wrap units, logical global sizes, local bytes, padding status, buffers, and optimizer state. Require numerical agreement within a predeclared tolerance after the matching update. Do not claim every state object is sharded or every rank has identical byte counts. Unsupported private layout details must remain unknown instead of being inferred from names.

Observe a quiescent sharded phase and a deliberate small-model gathered validation phase separately. The latter changes memory and cannot supply the former's measurement. Inspect optimizer state after a completed update and gradients before their declared clearing point. Validate globally reconstructed small-model values only through a supported collective path with all required participants. If public metadata cannot explain a layout detail, record the unknown rather than accessing unstable internals as a general solution.

Part 7 - Engineering Interpretation and Philosophy

Explain the ownership evidence that establishes actual full sharding and its phase dependence. State the initialization and wrapping assumptions needed before scaling the model or adding more complex training options.

19.40.4 Exercise 03

Exercise Title: Compare Per-GPU Peak VRAM Between DDP and FSDP Under the Same Model, Batch Size, Precision, and Optimizer

Part 1 - Exercise Contract and Why It Matters

Compare complete per-GPU peaks for DDP and FSDP with the same model, world size, local/global batch, precision, optimizer, and useful updates. Separate feasibility, memory savings, and throughput. Keep wrapping policy and activation workload visible in the comparison.

Part 2 - System and Memory Mental Model

FSDP reduces persistent replicated state but adds gather and reduce-scatter work and temporary full parameters. DDP has its own gradient communication and bucket behavior. Neither method guarantees lower total peak for every small model or workload. Data distribution and loss normalization determine whether the updates are genuinely comparable. The maximum rank peak and slowest completed rank matter for operational capacity and step time.

Part 3 - Quantitative Prediction Before Execution

Predict parameter, gradient, optimizer, activation, and temporary components for both cases. Define expected persistent savings and a separate range for materialization peaks. Calculate global sample count per update and an explicit throughput denominator. Include first-step initialization and warmed-step measurements separately. Identify a case dominated by activations where state sharding alone is unlikely to yield proportional peak reduction.

Part 4 - Experimental Design and Instrumentation

Use independent clean trials with identical initial checkpoint, sampler trace, model dimensions, precision, optimizer flags, and zeroing policy. Reset allocator peaks around a complete forward/backward/update region after declared warm-up. Synchronize deliberately and aggregate completed durations without adding hidden per-layer barriers. Validate a short loss/update trajectory before performance. Keep profiling and full-state numerical inspection outside the primary timed region. Record initialization peaks as a separate admission constraint rather than omitting them.

Use matching logical batches, loss scaling, optimizer flags, zeroing policy, and successful update counts. A lower memory result obtained by skipping an update is invalid. Use clean independent trials so allocator history or retained profiling tensors from the first method do not contaminate the second. Declare whether model construction is outside step timing while still measuring its admission peak. Repeat close timing comparisons before ranking configurations.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class ParallelTrainingCase:
    method: str
    world_size: int
    local_batch: int
    precision_identity: str
    optimizer_identity: str
    initial_state_identity: str


def compare_ddp_fsdp(cases, model_spec, data_trace, measurement_contract):
    # Enforce matching updates and include complete per-rank training peaks.
    ...


def aggregate_rank_results(rank_rows):
    # Preserve maximum peaks, timing spread, and useful global samples.
    ...


def E03(config):
    # Validate semantics before interpreting memory or throughput differences.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report per-rank allocated/reserved peaks, host scope, step latency spread, throughput, loss parity, and failure phase. A DDP-infeasible large case is a capacity result, not a measured finite DDP step time. Use a smaller matched case for numerical comparison. Avoid changing batch or precision only for one method and calling the result a controlled sharding experiment.

For timed distributed regions, ensure required CUDA work completes and then aggregate rank durations under the declared start/end protocol. Summing rank times is not step latency. Report global useful samples divided by the complete operational duration and retain straggler spread. A comparison at another world size must state how local and global batch changed; otherwise communication and activation differences can be mistaken for a pure sharding effect.

Part 7 - Engineering Interpretation and Philosophy

Explain which memory components changed and what communication cost was purchased. Recommend a method for the tested capacity/performance objective, preserving cases where DDP remains preferable.

19.40.5 Exercise 04

Exercise Title: Calculate Expected Per-Rank Parameter Shard Size and Compare It Against Real CUDA Memory Measurements

Part 1 - Exercise Contract and Why It Matters

Calculate expected local parameter shards unit by unit and compare them with supported CUDA storage measurements. Separate ideal partitioning, implementation padding, replicated objects, and temporary gathers. Produce an auditable reconciliation rather than expecting exact total VRAM from parameter count alone.

Part 2 - System and Memory Mental Model

A FSDP1 managed unit can flatten multiple logical parameters into local storage. Shared/tied parameters must be counted once in the logical inventory. Padding, alignment, representation settings, and group size affect actual shards. A local parameter view may not expose the same shape as the original logical parameter. FULL_SHARD post-phase state differs from a gathered computation state or another sharding strategy.

Part 3 - Quantitative Prediction Before Execution

For a unit with n_i managed elements and group size W_i, use $\lceil n_i/W_i \rceil$ as a clearly labeled padded-partition model when its assumptions match the representation. Sum units using their actual element widths and add ignored/replicated payload separately. Predict a nondivisible unit and a tiny unit. Keep unknown extra alignment distinct from arithmetic padding rather than silently forcing the prediction to match measurements.

Part 4 - Experimental Design and Instrumentation

Capture a logical CPU metadata inventory before wrapping without retaining full GPU copies. Freeze wrap manifest, use_orig_params, strategy, precision, and process groups. Measure at a quiescent full-shard boundary after a complete step. Deduplicate unique backing and exclude validation gathers from the measurement region. Test uneven unit sizes and tied weights supported by the chosen layout. Compare multiple available world sizes only with explicitly updated batch/scaling rules.

Capture logical parameter identity, shape, dtype, ties, ignored ownership, and target unit before flattening. Count a tied storage once even when it has multiple names. Test a small unit with fewer elements than ranks and a unit whose size is not divisible by its group. Record the declared original-parameter representation so local view sizes are not confused with the logical inventory.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
```

```
class ParameterShardPrediction:
    unit_identity: str
    logical_elements: int
    sharding_group_size: int
    element_bytes: int
    predicted_padding_elements: object
    replicated_bytes: int


def predict_unit_shards(logical_inventory, wrap_manifest, representation):
    # State padding assumptions and preserve unsupported alignment as unknown.
    ...


def reconcile_shard_storage(predictions, measured_local_backing):
    # Separate persistent shards from gathers, aliases, and allocator overhead.
    ...


def E04(config):
    # Audit divisible, nondivisible, tied, and ignored-state examples.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report raw logical counts, unique ties, unit membership, group sizes, dtypes, predicted padding, measured backing, and unexplained residuals. Do not use the global world size when the actual sharding group is smaller. A zero-sized original-parameter view is not proof the model lost that parameter. Do not label total allocator reservation minus shard bytes as fragmentation.

Compare modeled shard padding with observed local storage only at matching phases. Additional implementation alignment must be identified or left unknown, not silently inserted into the formula after measurement. Add buffers and ignored parameters separately. Check that no source reference in the accounting code keeps a full pre-wrap GPU model alive. The total allocator reservation includes retained capacity and other allocations, so reconciliation should explain scope rather than force an exact equality to model payload.

Part 7 - Engineering Interpretation and Philosophy

Explain how far the prediction accounts for the observed persistent state and what remains implementation- or allocator-specific. Use the reconciled model for planning without promoting it to a guarantee about full training peak memory.

19.40.6 Exercise 05

Exercise Title: Trace Parameter All-Gather Before Forward Computation and Reduce-Scatter During Backward with PyTorch Profiler

Part 1 - Exercise Contract and Why It Matters

Trace parameter gathering before computation and gradient reduction/sharding during backward. Associate collective events with FSDP units and memory phases. Distinguish application payload estimates, profiler event durations, and actual link traffic; none is automatically a substitute for the others.

Part 2 - System and Memory Mental Model

Full sharding restores unit parameters for computation and returns gradients to shard ownership through the supported reduction protocol. Backward can require parameter gathers again after forward resharding. Prefetch overlaps communication with another unit's work, and repeated/shared execution can change collective counts. Profiler names and attribution vary by build. Collective ordering must be compatible across ranks; a local trace cannot establish complete distributed correctness by itself.

Part 3 - Quantitative Prediction Before Execution

Predict a simple unit execution sequence and expected gather/reduce-scatter payloads using actual padded sizes and communication dtypes. State any collective algorithm assumption when estimating per-rank traffic. Do not equate full-tensor payload with each rank's wire bytes. Draw expected overlapping and serialized intervals and identify where parameter and gradient buffers can coexist. Leave unsupported traffic measurements unknown.

Part 4 - Experimental Design and Instrumentation

Warm the process group and model before a short bounded profiler run with CPU/CUDA activities and declared memory recording. Add public user scopes around model phases and unit calls where supported. Export a trace per rank with rank/device/group identity and compare collective sequence metadata. Avoid full state-dict collection or global synchronization inside each unit. Run a separate unprofiled timing control. Validate a complete optimizer update before interpreting communication overlap or reducing scheduling constraints.

Label events by rank, group, unit, step, and phase, then align collectives using their supported sequence information. Do not compare raw CUDA event origins across devices as a universal common clock. A collective can have CPU launch, enqueue, and device execution intervals; preserve which one the profiler exposed. Include warm-up that initializes communication so startup is not attributed to every training step.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
from torch.profiler import profile, ProfilerActivity, record_function

@dataclass
class CollectiveEvidence:
    rank: int
    group_identity: str
    unit_identity: str
    collective_kind: str
    payload_bytes: object
    attribution_support: str
```

```
def profile_fsdp_step(runtime, profiler_contract):
    # Record bounded traces without changing collective order across ranks.
    ...


def map_collectives_to_units(trace_rows, wrap_manifest):
    # Preserve uncertain attribution and distinguish payload from wire traffic.
    ...


def E05(config):
    # Compare predicted ordering, measured traces, and unprofiled complete time.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report observed event families, unit mapping support, payload model, durations, memory intervals, and profiler overhead. Do not sum overlapping communication durations as total step cost. Cross-device clocks do not provide a shared timestamp origin automatically. A missing event label can be a tracing limitation rather than proof that sharding communicated nothing.

Derive traffic estimates from actual padded unit payloads and dtypes under a stated algorithm model. A gather's logical full payload is not each rank's measured bus traffic, and a sum of overlapping event durations is not wall time. Report profiler overhead with an unprofiled control. Missing allocation attribution or kernel labels must remain unsupported, while numerical update validation confirms the trace came from a correct training execution.

Part 7 - Engineering Interpretation and Philosophy

Explain which collectives are exposed on the critical path and which overlap useful work. Use evidence to choose the next memory/scheduling experiment, without treating communication presence alone as a performance diagnosis.

19.40.7 Exercise 06

Exercise Title: Measure Temporary Full-Parameter Materialization Peaks During FSDP Forward Execution

Part 1 - Exercise Contract and Why It Matters

Measure temporary full-parameter residency during forward and relate peaks to wrapping granularity and prefetch. Separate transient gathered capacity from persistent shards and activations. Demonstrate why an apparently small idle model can still have a large computation peak.

Part 2 - System and Memory Mental Model

A full computation unit must become available on each participating rank under this data-parallel FSDP design. Its full payload can coexist with local shards, activations, prefetched

units, and collective workspace, depending on implementation lifetimes. Root-managed parameters and ignored objects matter. Memory counters alone do not label the owner of a peak. A diagnostic that forces a full-model gather changes the phenomenon being measured.

Part 3 - Quantitative Prediction Before Execution

Calculate the largest managed unit, its gather dtype, and a bounded set of simultaneously live units under the configured schedule. Add persistent state and forward activations without double-counting alias backing. Compare a single root unit with nested blocks under the same useful forward. Predict when the largest unit or its temporary overlap exceeds the local budget even though ideal persistent shards fit.

Part 4 - Experimental Design and Instrumentation

Use bounded models and the already validated wrap manifest. Capture phase counters, a short profiler trace, and supported memory history/snapshots in separate diagnostic runs. Compare root-only wrapping, smaller units, and prefetch settings without changing model arithmetic or inputs. Reset peaks for the declared forward region but also report complete-step and initialization peaks. Avoid retaining parameter views in hooks or evidence records. Measure unprofiled throughput separately from heavily synchronized attribution.

Use a model with a known large root-managed object and smaller repeated blocks so the dominant gather can be predicted independently of module count. Record possible current/next overlap under each scheduling option. Add a control with the same parameters but a different input shape to isolate activation contribution. If hooks record tensors rather than scalar metadata, they can extend full-parameter lifetime and create the very peak being investigated.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class MaterializationPrediction:
    unit_identity: str
    full_payload_bytes: int
    persistent_shard_bytes: int
    possible_overlap_units: tuple
    attribution_limitations: tuple


def predict_forward_materialization(manifest, precision, schedule):
    # Model live backing and leave unknown alias/workspace relationships explicit.
    ...


def measure_forward_peaks(runtime, observation_contract):
    # Observe without forcing a full state dict or retaining parameter views.
    ...
```

```
def E06(config):
    # Link temporary peaks to auditable units and controlled schedule changes.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report raw peak/current allocated and reserved values, unit sizes, activation context, schedule, and attribution evidence. An external device monitor includes memory not captured by PyTorch tensor accounting. Do not call every unexplained byte a gathered parameter. Include a counterexample to peak equals persistent shard bytes and preserve cases where smaller units increase other overhead.

Keep three resource results distinct: persistent post-step state, forward region peak, and complete training peak. Resetting a counter around forward does not prove backward or optimizer fit. Compare allocator counters with a short supported history trace and external device scope only where available. Do not treat a numerical residual between these layers as a named tensor owner. A bounded model can demonstrate the materialization mechanism without requiring a real out-of-memory event.

Part 7 - Engineering Interpretation and Philosophy

Explain the minimum materialization constraint and the capacity/performance trade-off of splitting units. Recommend a next wrap or schedule change only when the observed peak mechanism supports it.

19.40.8 Exercise 07

Exercise Title: Compare Different FSDP Sharding Strategies and Quantify Their Memory-Communication Trade-Offs

Part 1 - Exercise Contract and Why It Matters

Compare FSDP1 sharding strategies with explicit ownership and reshard timing. Measure per-rank memory, collective payload, and complete step time under matched useful training. Keep topology-dependent hybrid strategies optional when the available process-group layout supports a meaningful comparison.

Part 2 - System and Memory Mental Model

FULL_SHARD, SHARD_GRAD_OP, and NO_SHARD differ in which state is sharded and when full parameters remain available. NO_SHARD is not a promise of FSDP memory savings; ordinary DDP is the practical replication baseline. Hybrid sharding uses a shard group plus replication, so dividing by global world size can be wrong. FSDP2 expresses comparable behavior through a different API and must not receive FSDP1 strategy enums as if they were interchangeable.

Part 3 - Quantitative Prediction Before Execution

Predict persistent parameters, post-forward full residency, synchronized gradients, optimizer state, and backward gathers for each configured strategy. Use the actual shard and replication group sizes. Estimate collective payload with stated dtype and padding assumptions. Include

the gradient-accumulation configuration in the prediction even though the primary comparison uses one forward/backward/update. Define a topology where hybrid dimensions are both nontrivial and mark unavailable layouts untested.

Part 4 - Experimental Design and Instrumentation

Rebuild independent trials from identical initial weights, batches, optimizer, precision, and wrap units. Verify group construction and configuration on all ranks before collective work. Measure cold and warmed complete steps, post-forward/post-backward diagnostics, and per-rank peak distributions. Compare a short numerical update trajectory. Record unsupported version/strategy combinations rather than silently falling back. A single-host multi-GPU machine is sufficient for the primary comparisons; multi-node setup is not required.

Build process-group membership deterministically and verify every rank's view before strategy comparison. For hybrid experiments, record shard and replica dimensions explicitly. A single-host group can support the conceptual test if enough devices exist, but two GPUs do not create every nondegenerate hybrid layout. Keep unsupported cases unexecuted instead of choosing an undocumented fallback that changes the policy being compared.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
from torch.distributed.fsdp import ShardingStrategy

@dataclass
class ShardingStrategyCase:
    strategy_identity: str
    sharding_group_ranks: tuple
    replication_group_ranks: tuple
    wrap_manifest_identity: str
    precision_identity: str


def validate_strategy_topology(case, distributed_context):
    # Check supported groups and rank-consistent collective membership.
    ...


def compare_sharding_strategies(cases, training_trace):
    # Preserve matching updates and record phase-specific state ownership.
    ...


def E07(config):
    # Evaluate feasible strategy/topology combinations without invented scaling.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report group sizes, state phases, actual bytes, collective counts/payload scope, step times, and numerical checks. Do not compare a hybrid configuration with different total useful work without labeling the changed experiment. A degenerate replication dimension is not evidence of a full hybrid topology. Retain failures and untested configurations explicitly.

Compare state at the phase named by the strategy contract. Post-forward parameter residency can differ even when post-update ownership looks similar. Record gradient synchronization and accumulation settings; a no_sync extension can retain larger gradients and is a separate workload. Keep optimizer state and data normalization fixed. Explain each observed change through group sizes, reshard timing, or communication evidence rather than a universal stage label that hides implementation-specific lifetime behavior.

Part 7 - Engineering Interpretation and Philosophy

Choose the smallest strategy satisfying the tested capacity objective with acceptable communication cost. Explain when retaining parameters trades memory for fewer gathers and why the result depends on topology and unit lifetimes.

19.40.9 Exercise 08

Exercise Title: Build an Auto-Wrap Policy That Shards Transformer Blocks at a Chosen Module Granularity

Part 1 - Exercise Contract and Why It Matters

Build a FSDP1 auto-wrap policy selecting Transformer block boundaries at a declared granularity. Produce a deterministic ownership manifest covering all managed parameters, including root leftovers and ties. Validate construction and one training update before tuning performance.

Part 2 - System and Memory Mental Model

Wrapping defines gather units, not merely visual module labels. A root wrapper still owns parameters not assigned to child units. Embeddings, heads, and normalization can dominate that root payload. Shared/tied parameters require supported consistent ownership rather than arbitrary independent wrapping of both aliases. All ranks must create the same model and wrap plan. Optimizer references are established after wrapping.

Part 3 - Quantitative Prediction Before Execution

Inventory each candidate block's unique parameter elements and estimate its full gather bytes and local shards. Predict unit count and root-managed residual. Compare grouping several consecutive blocks with one wrapper per block. Calculate the largest materialized unit and possible prefetch overlap. Identify ties crossing proposed boundaries and reject unsupported ownership before timing.

Part 4 - Experimental Design and Instrumentation

Use explicit block classes or a validated size/type policy with the public FSDP1 wrapping interface. Test a small model containing embedding, repeated blocks, final normalization, and an output head. Include a supported tied-weight case as a separate validation. Export only CPU metadata for unit ownership, not full GPU weights. Check no unmanaged gap or duplicate ownership. Keep use_orig_params, strategy, initial values, and optimizer configuration fixed. Record the installed API signature and reject FSDP2-only controls.

Audit the root remainder after selecting Transformer blocks. Embedding and output-head aliases can cross apparent module boundaries, so choose a supported common ownership plan or reject the proposed partition. A type predicate and a size predicate may produce different units on the same model; record the actual manifest, not merely the policy name. All ranks must agree on that manifest before optimizer construction and collective execution.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
from torch.distributed.fsdp.wrap import transformer_auto_wrap_policy

@dataclass
class FSDPWrapPlan:
    block_class_names: tuple
    blocks_per_unit: int
    root_parameter_names: tuple
    shared_parameter_groups: tuple
    expected_unit_manifest: tuple


def build_fsdp1_wrap_policy(model, plan):
    # Choose supported boundaries and preserve shared ownership constraints.
    ...


def audit_wrap_ownership(model, logical_inventory, plan):
    # Reconcile every unique parameter and root-managed remainder.
    ...


def E08(config):
    # Validate deterministic wrapping before memory/communication experiments.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report unit boundaries, unique sizes, root leftovers, ties, ignored state, construction peak, and update parity. Module count is not parameter ownership. A policy that wraps every block but leaves one huge root embedding still has a huge materialization constraint. Do not correct rank-dependent plans by allowing mismatched collective sequences to run.

Test the policy on a tiny model where expected unique parameters and units are hand-auditable, including a large embedding and a supported tie. Verify numerical output and one optimizer update. Do not introduce rank-dependent wrapping to balance uneven shapes. If extending to frozen parameters or original-parameter mode, document those constraints as a new case. Keep actual initialization peak in the result so an apparently ideal execution plan is not accepted when construction already exceeds capacity.

Part 7 - Engineering Interpretation and Philosophy

Explain how wrapping granularity creates the memory schedule later experiments will tune. Prefer an auditable type-based plan over a complicated heuristic until root ownership and shared-parameter behavior are verified.

19.40.10 Exercise 09

Exercise Title: Sweep FSDP Wrapping Granularity and Measure Peak VRAM, Communication Volume, Kernel Utilization, and Step Time

Part 1 - Exercise Contract and Why It Matters

Sweep wrapping granularity and measure complete peaks, communication, compute gaps, and step time. Hold model and useful training fixed while varying only documented unit grouping. Select a tested compromise under explicit per-rank capacity and throughput constraints.

Part 2 - System and Memory Mental Model

Fine units can reduce full-parameter peaks but increase collective count, launch overhead, and scheduling work. Coarse units can expose larger gathers with fewer calls. Root state, mixed precision, activations, and prefetch can change the result. Profiler utilization is a scoped observation, not a universal measure of device efficiency. Communication payload estimates and measured bus traffic must remain distinct.

Part 3 - Quantitative Prediction Before Execution

For each grouping, predict unit sizes, root residual, padded shard capacity, collective count, and overlap envelope. State assumptions for byte-volume estimates and identify a candidate whose largest unit is infeasible. Define complete global samples per second and rank-tail step time. Construct a case where the smallest parameter unit is not the fastest configuration.

Part 4 - Experimental Design and Instrumentation

Use a bounded set of groupings, independent initial-state replays, and the same per-rank samples, precision, optimizer, strategy, and scheduling flags. Separate initialization, first update, warmed timing, and a short profiler run. Reset declared peaks and aggregate maximum rank usage. Validate numerical updates before admitting a candidate as feasible. Test the chosen grouping on a held-out shape/length trace without changing collective execution order. Report failures rather than silently reducing the batch only for difficult candidates.

Use a fixed global model and a small explicit grouping grid, including a root-only control and several nested choices. Keep the same successful update count and data trace. Predict the root-managed residual and largest gather for every candidate before launching. If a case is analytically unsafe, record that admission result rather than deliberately allocating to failure. Retain initialization and first-optimizer-state peaks as separate feasibility gates.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
```

```
class FSDPGranularityTrial:
    blocks_per_unit: int
    wrap_manifest_identity: str
    per_rank_budget_bytes: int
    training_trace_identity: str


def predict_granularity_trial(trial, model_inventory, schedule):
    # Count root, units, padding, and complete materialization requirements.
    ...


def benchmark_granularity_trial(trial, runtime_contract):
    # Validate equal work and keep profiler overhead out of primary timing.
    ...


def E09(config):
    # Select a robust feasible grouping with explicit uncertainty.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report raw manifests, rank peaks, step distributions, payload scope, collective counts, separate compute and communication intervals, and validation outcomes. A small average rank memory cannot hide one infeasible rank. A profile with more busy time does not prove higher useful throughput. Preserve uncertainty when close timing differences are smaller than run-to-run variation.

Report rank-tail latency and maximum per-rank peak, not only mean values. Profile a short diagnostic trace separately from the primary timing loops. A higher utilization percentage can reflect more overhead work rather than more useful training. Preserve uncertainty in communication-volume estimates and close performance rankings. Validate the selected point on held-out shapes with compatible module order and keep all rejected, unsupported, or failed candidates in the comparison ledger.

Part 7 - Engineering Interpretation and Philosophy

Explain which mechanism determines the selected grouping and its tested shape/topology dependence. Keep a simple stable configuration when tuning complexity offers no demonstrated benefit.

19.40.11 Exercise 10

Exercise Title: Combine FSDP with Mixed Precision and Measure Parameter, Gradient, and Communication Memory Changes

Part 1 - Exercise Contract and Why It Matters

Combine the validated FSDP1 runtime with an explicit mixed-precision policy. Measure persistent parameters, gathered computation values, gradients, reduction communication, buffers,

optimizer state, and complete peaks separately. Keep a full-precision reference for numerical and update validation.

Part 2 - System and Memory Mental Model

FSDP1 MixedPrecision distinguishes parameter computation, reduction, and buffer dtypes. Persistent parameter shards and optimizer state need not use the gathered parameter dtype. Gradient storage after reduction depends on settings such as keeping low-precision gradients and optimizer compatibility. Autocast is another arithmetic control, not the entire sharding policy. Hardware support, overflow handling, and norm clipping must be validated rather than inferred from a dtype label.

Part 3 - Quantitative Prediction Before Execution

Predict bytes for each state/communication component using its actual dtype. Compare reduced gather precision with unchanged full-width persistent shards. Estimate temporary cast buffers and reduction payload. State whether loss scaling is needed for the chosen path; a supported bfloat16 case and an fp16 case require different numerical considerations. Predict memory savings without assuming every optimizer tensor or gradient automatically halves.

Part 4 - Experimental Design and Instrumentation

Start with a supported low-precision policy on the small deterministic model and unchanged batch, optimizer, wrapping, and data trace. Record every FSDP1 precision field and autocast scope. Verify optimizer state dtypes after a real step. If using fp16, use a version-supported distributed/FSDP-aware scaling and overflow protocol so ranks do not silently disagree about skipped updates. Use the supported FSDP norm-clipping path when clipping is enabled. Compare loss, gradients through a supported inspection boundary, and a short parameter trajectory within predeclared tolerances.

Inventory parameter computation, persistent shards, buffers, reduction communication, stored gradients, optimizer moments, and cast workspace using their actual dtypes. Keep input casting and autocast scopes explicit, especially when nested units use different policies. A low-precision communication flag does not establish the dtype of persistent optimizer state. Test supported hardware before selecting a format and record unsupported combinations rather than substituting another dtype without labeling the changed case.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
from torch.distributed.fsdp import MixedPrecision

@dataclass
class FSDPPrecisionContract:
    parameter_compute_dtype: str
    reduction_dtype: str
    buffer_dtype: str
    keep_low_precision_gradients: bool
    autocast_policy: str
    distributed_overflow_policy: str
```

```
def build_fsdp1_precision_policy(contract, hardware_support):
    # Validate dtype/scaler/optimizer compatibility for the pinned API.
    ...


def inventory_precision_states(model, optimizer, phase):
    # Observe actual persistent, gathered, gradient, and optimizer dtypes.
    ...


def E10(config):
    # Validate coherent updates before complete memory/communication comparison.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report actual dtypes, payloads, cast temporaries, overflows/skipped updates, communication scope, peaks, and complete time. Equal sample counts with different numbers of successful optimizer updates are not identical training work. Do not loosen numerical tolerances after seeing a preferred result without justifying the changed contract. Mark unsupported hardware/precision cases explicitly.

Verify successful update counts and any overflow decisions agree across ranks. Compare a short loss and parameter trajectory with the full-precision reference under predeclared tolerances. If clipping is used, use a supported global/sharded norm protocol rather than clipping each shard independently and assuming equivalence. Record state after backward and after update separately. A lower peak obtained through skipped work or incompatible optimizer gradients is not a valid mixed-precision training improvement.

Part 7 - Engineering Interpretation and Philosophy

Explain which representation changed and what numerical/communication trade-off was measured. Recommend only a supported policy whose complete-step benefit and numerical behavior meet the stated experiment contract.

19.40.12 Exercise 11

Exercise Title: Combine FSDP with Activation Checkpointing and Determine Whether Their VRAM Savings Target Independent Memory Components

Part 1 - Exercise Contract and Why It Matters

Combine FSDP state sharding with activation checkpointing and evaluate a factorial comparison: neither, checkpointing alone, FSDP alone, and both. Determine which memory components each technique targets and whether their combined execution introduces extra communication or recomputation.

Part 2 - System and Memory Mental Model

FSDP primarily changes model-state ownership and materialization; checkpointing changes saved activation lifetimes by recomputing operations during backward. These targets differ

but their execution costs interact. Checkpoint boundaries can trigger FSDP hooks or repeated gathers depending on wrapping and reshard policy. Savings are not automatically additive. RNG behavior, dropout, mutation, and checkpoint implementation affect numerical correctness.

Part 3 - Quantitative Prediction Before Execution

Predict persistent state, saved activations, gathered units, recomputation, and collective sequence for each case. Use a bounded non-reentrant checkpoint reference with explicit boundaries. Estimate the complete peak rather than subtracting two independently measured savings from one baseline. Construct a model where activations dominate and another where model state dominates, keeping every comparison's useful update semantics documented.

Part 4 - Experimental Design and Instrumentation

Use public checkpointing with an explicit use_reentrant choice, initially non-reentrant, and a deterministic block model. Freeze sharding, precision, optimizer, batch, and data trace. Validate loss and parameter updates before resource timing. Add RNG-preservation tests separately when dropout is introduced. Record checkpoint and FSDP unit manifests and observe the actual collective sequence during recomputation. Separate diagnostic profiling from complete training timing. Include backward and optimizer update; a forward-only measurement cannot establish checkpointing's training peak.

Draw checkpoint recomputation boundaries alongside FSDP gather units and predict where hooks will execute again. Keep use_reentrant explicit and start with a deterministic no-dropout reference. If adding stochastic operations, validate RNG preservation and resumed update semantics independently. A checkpoint function must not change its computation between original forward and recomputation. Do not retain activation tensors in diagnostics while measuring the memory savings intended from dropping them.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
from torch.utils.checkpoint import checkpoint

@dataclass
class FSDPCheckpointingCase:
    fsdp_enabled: bool
    activation_checkpointing_enabled: bool
    checkpoint_boundaries: tuple
    use_reentrant: bool
    rng_policy: str


def apply_activation_checkpoint_plan(model, case, wrap_contract):
    # Preserve supported hook order and deterministic recomputation behavior.
    ...


def compare_memory_components(cases, training_trace):
```

```
    # Validate updates and measure recomputation/collectives with full peaks.
    ...


def E11(config):
    # Test independent targets and interacting execution costs explicitly.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report saved-activation scope, state shards, materialization peaks, recomputed work, collective counts, complete latency, and numerical outcomes. A lower activation estimate does not prove a lower total peak if extra gathers overlap differently. Do not claim additive savings or identical communication without trace evidence. Preserve unsupported checkpoint/wrapping combinations as explicit limitations.

Measure the four factorial cases under the same model, batch, optimizer, precision, and successful updates. Report full-step peaks and recomputation communication instead of combining separately observed savings arithmetically. A forward-only trace misses the primary recomputation phase. Include a case where extra work is too costly for the achieved capacity and preserve it in the final recommendation. Verify that the selected wrapper/checkpoint order uses only the supported version-specific integration.

Part 7 - Engineering Interpretation and Philosophy

Explain when the two techniques complement each other and when their extra work is not justified. Choose boundaries from measured component lifetimes rather than a blanket rule to checkpoint every wrapped module.

19.40.13 Exercise 12

Exercise Title: Enable FSDP CPU Offloading and Quantify the Trade-Off Between GPU Memory Savings and Host-Device Transfer Cost

Part 1 - Exercise Contract and Why It Matters

Enable supported FSDP1 parameter/gradient CPU offload and measure GPU relief, host capacity, transfers, optimizer placement, and complete step time. Preserve the validated training semantics and make restrictions part of the experiment support matrix.

Part 2 - System and Memory Mental Model

FSDP1 CPUOffload moves supported parameter and gradient state according to its protocol; optimizer placement and state must be observed after update. CPU memory is finite and shared by all rank processes on the host. Pinned staging, local shards, temporary gathers, and initialization can coexist. The documented FSDP1 accumulation restrictions matter: the primary test uses one forward/backward/update and does not assume arbitrary accumulation outside no_sync is supported with CPU offload.

Part 3 - Quantitative Prediction Before Execution

Predict GPU persistent payload removed, host shards/state added, H2D/D2H traffic, and computation-unit materialization that remains necessary. Estimate transfer time from measured effective bandwidth and actual dtype/bytes. Include CPU optimizer time and pinned-memory scope. Compare GPU-only and offloaded configurations under equal useful updates, and identify a case where host capacity or transfer becomes the limiting resource.

Part 4 - Experimental Design and Instrumentation

Use the pinned FSDP1 CPUOffload interface on the small model first. Build the optimizer after wrapping and inspect actual state devices after a real step. Keep world size, batches, precision, wrap units, and data fixed. Validate numerical updates, then measure complete steps including CPU work and final transfer completion. Record host memory per rank and a host-wide scope without blindly summing overlapping RSS. Bound allocations and test only supported accumulation/offload combinations as separately labeled extensions.

Keep the initial offload experiment to one synchronized backward and optimizer update per step. Do not enable a generic accumulation loop by copying a DDP pattern; validate the documented FSDP1/no_sync restrictions first. Record where optimizer moments and scalar counters actually reside. Measure host capacity across all workers sharing the machine, including construction replicas and pinned staging, while keeping per-process RSS distinct from unique physical memory when sharing cannot be resolved.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
from torch.distributed.fsdp import CPUOffload

@dataclass
class FSDPCPUOffloadContract:
    enabled: bool
    optimizer_device_policy: str
    host_budget_bytes: int
    pinned_scope: str
    accumulation_steps: int
    supported_accumulation_protocol: str


def configure_fsdp1_cpu_offload(contract, version_support):
    # Reject unsupported accumulation/optimizer combinations before launch.
    ...


def measure_cpu_offload_step(runtime, measurement_contract):
    # Include CPU optimizer, migration, materialization, and complete updates.
    ...


def E12(config):
```

```
    # Compare honest host/GPU/transfer trade-offs with the resident reference.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report state devices/dtypes, host backing, known pinned payload, device peaks, transfer evidence, CPU update time, complete throughput, and parity. Do not infer host-optimizer state solely from an offload flag. A GPU memory saving that exceeds the host budget is infeasible. Preserve unsupported total pinned-memory counters as unknown instead of equating them with one tensor's payload.

Compare complete work rather than only device attention or backward kernels. CPU optimizer and transfers can dominate an apparently small GPU trace. Record actual copied dtype and payload, and use completed transfer intervals for bandwidth. If total pinned-memory usage is unavailable, report supported known-live payload only. A host-memory infeasible configuration cannot qualify as a solution merely because its GPU allocation decreases. Keep an explicit support matrix for optimizer, precision, and accumulation extensions.

Part 7 - Engineering Interpretation and Philosophy

Explain which capacity moved to the CPU and which GPU minimum remains. Recommend offload only when complete measured work and host limits justify its transfer and CPU-computation cost.

19.40.14 Exercise 13

Exercise Title: Reproduce an FSDP Memory Spike Caused by Excessive Concurrent All-Gathers and Diagnose It with CUDA Memory Statistics

Part 1 - Exercise Contract and Why It Matters

Construct a bounded experiment testing whether aggressive gather issuance or prefetch produces a larger live materialization peak. Compare the FSDP1 rate limiter enabled and disabled under a compatible static execution trace. Reproduce an observed spike or record that it was not reproduced; actual OOM is not required.

Part 2 - System and Memory Mental Model

Outstanding collective calls and simultaneously live full-parameter buffers are different counts. CPU issuance can run ahead of GPU execution, while allocator reuse and dependencies affect residency. FSDP1 limit_all_gathers constrains its supported gather schedule; FSDP2 has a different memory-management model and no interchangeable flag. CUDA counters alone cannot attribute a peak to concurrent gathers without a trace or controlled comparison.

Part 3 - Quantitative Prediction Before Execution

Predict unit sizes and a conservative overlap envelope for rate-limited and aggressive schedules. Add activations, gradients, root state, and workspace. Select dimensions whose tested bound stays below a declared safe budget. Identify evidence that would support the spike hypothesis: more live gathered capacity at a matched phase and a reproducible peak change under the control. Do not predict a guaranteed hardware failure.

Part 4 - Experimental Design and Instrumentation

Use identical weights, units, batches, optimizer, precision, and static module execution. Change only supported limiter/prefetch settings in clean trials. Record per-rank counters, peaks, and a short memory/collective trace. Warm the process group and separate profiling from timing. Stop bounded trials when admission estimates become unsafe. If a rank fails, terminate/restart the distributed trial through a coordinated protocol; do not catch an exception on one rank and let peers continue mismatched collectives.

Predict a safe upper envelope before disabling the limiter in any trial. Use small units and finite steps so the test explores relative peaks without requiring actual OOM. Match execution order and all useful computation between controls. Distinguish a count of queued collective launches from allocations that remain concurrently live. A trace without a spike is an honest non-reproduction, not a reason to change unrelated workload dimensions until a desired claim appears true.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class AllGatherPressureCase:
    limit_all_gathers: bool
    forward_prefetch: bool
    backward_prefetch_identity: str
    wrap_manifest_identity: str
    per_rank_safe_budget_bytes: int


def predict_gather_pressure(case, unit_inventory, phase_model):
    # Separate issued operations, live buffers, and unsupported attribution.
    ...


def run_bounded_gather_pressure(case, distributed_trial):
    # Preserve matched work and coordinated lifecycle with diagnostic evidence.
    ...


def E13(config):
    # Reproduce the peak mechanism or report an honest non-reproduction.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report rank peaks, live-buffer attribution support, collective sequence, settings, time, and reproducibility. A CPU launch gap can be intentional rate limiting and is not proof of GPU idle time. A peak difference caused by changed activations or workload does not establish excessive all-gather concurrency. Preserve unavailable fields and failed/non-reproducing cases.

Collect rank-local phase records and supported allocation history in a separate diagnostic replay. Do not infer owner identity from the difference between reserved and allocated counters alone. Repeat an observed peak change and test whether restoring the limiter reverses the mechanism. If a worker fails, record the phase and stop/restart the trial coherently; independent local retries can create mismatched collectives. Preserve failed and unavailable hardware cases explicitly in the evidence ledger.

Part 7 - Engineering Interpretation and Philosophy

Explain the strongest supported mechanism and the next controlled scheduling change. Use a reproducible capacity signal rather than deliberately exhausting the machine or inventing an OOM result.

19.40.15 Exercise 14

Exercise Title: Limit or Schedule Parameter All-Gathers to Reduce Peak VRAM Without Introducing Excessive GPU Idle Time

Part 1 - Exercise Contract and Why It Matters

Limit or schedule FSDP1 gather work to reduce complete per-rank peaks with acceptable exposed communication and idle intervals. Evaluate documented limiter and prefetch controls under a validated execution pattern rather than patching private allocator or stream internals.

Part 2 - System and Memory Mental Model

Less lookahead can reduce simultaneous full parameters while exposing communication before computation. More prefetch can hide latency at the cost of extra live state. FSDP1 backward-prefetch modes and forward prefetch have specific version/execution assumptions; forward prefetch must not be blindly applied to an incompatible dynamic order. A CPU rate-limiter wait and a GPU critical-path wait are different observations.

Part 3 - Quantitative Prediction Before Execution

Draw candidate dependency schedules and predict materialized capacity, gradient overlap, and exposed gather waits. Compare limiter enabled with bounded prefetch choices and a conservative serialized diagnostic control. Define a maximum acceptable peak and complete-step regression before evaluation. Identify when memory saved is too small to justify lost throughput, and when a slower feasible schedule is the only admissible choice.

Part 4 - Experimental Design and Instrumentation

Rebuild clean matched trials with rank-consistent settings and a static small model first. Validate losses and updates, then use short profiles for attribution and unprofiled repeats for complete time. Keep all unit sizes, precision, batch, optimizer, and data fixed. Measure maximum rank peak and slowest completed duration, not just rank zero. Test selected settings on held-out shapes with compatible collective execution. Record unsupported scheduling combinations and coordinated failure/restart behavior.

Use a small schedule grid with documented FSDP1 controls and no private stream surgery. Validate static-order assumptions before enabling forward prefetch. Keep current/next unit sizes, gradient state, and activation demand in the peak model. A limiter can intentionally pause CPU issuance while GPU work continues; only a device timeline and complete timing establish exposed idle cost. Do not interpret host gaps as automatic throughput loss.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
from torch.distributed.fsdp import BackwardPrefetch

@dataclass
class GatherScheduleContract:
    limit_all_gathers: bool
    backward_prefetch_identity: str
    forward_prefetch: bool
    static_order_required: bool
    maximum_rank_peak_bytes: int
    maximum_step_regression: float


def validate_gather_schedule(contract, execution_manifest):
    # Check public API support and identical collective order on every rank.
    ...


def compare_gather_schedules(cases, training_trace):
    # Measure full peaks, exposed waits, and complete useful work.
    ...


def E14(config):
    # Select a feasible evidence-based memory/communication compromise.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report configuration, capacity, observed CPU/GPU waits, collective overlap, complete throughput, numerical checks, and uncertainty. No scheduling option guarantees the GPU never waits. Do not remove essential dependencies to improve a utilization metric. A documented limiter's effect is scoped to its buffers, not a universal bound on activations or all process memory.

Select a schedule on calibration traces, then verify held-out compatible workloads. Report feasibility and timing uncertainty independently. A slightly slower point can be necessary to meet capacity, while a lower peak that saves no useful admission may not justify extra stalls. Keep every rank's setting identical and rebuild at a validated boundary rather than mutating the active collective schedule mid-step. Essential dependencies must remain even when a utilization metric would look better without them.

Part 7 - Engineering Interpretation and Philosophy

Choose the smallest tested scheduling change that achieves the required capacity while preserving useful work. Explain when a deliberate wait is a correct resource-control cost rather than an implementation failure.

19.40.16 Exercise 15

Exercise Title: Build Full and Sharded Model Checkpoints and Compare Save-Time RAM, Storage Size, and Reload Behavior

Part 1 - Exercise Contract and Why It Matters

Save full and sharded model/optimizer checkpoints and compare per-rank GPU peaks, host RAM, file payload, save time, and reload correctness. Use a committed training boundary and distinguish logical state size from distribution of that state across files or ranks.

Part 2 - System and Memory Mental Model

A full checkpoint may gather model and optimizer state and concentrate host capacity on a designated rank. Sharded checkpoints distribute state and need metadata plus compatible loading logic. Aggregate useful state does not become world-size times smaller merely because each rank writes a shard. Full-state collection can require participation from every rank even when only rank zero writes the final file. Checkpoint extraction is itself a memory workload.

Part 3 - Quantitative Prediction Before Execution

Predict parameter, optimizer, scalar, and metadata bytes in both formats. Estimate rank-zero full-state RAM and simultaneous model/optimizer extraction capacity. Predict sharded per-rank payload and aggregate file size with padding and metadata separated. Define save completion and consistent restart state: optimizer update, scheduler, per-rank RNG, sampler progress, and logical step. Do not confuse dispatched prefetched batches with committed training progress.

Part 4 - Experimental Design and Instrumentation

Use a small trained model first and freeze the FSDP1 checkpoint adapter. For full state, use supported full-state configuration with CPU offload and rank-zero-only result materialization while all required ranks participate. For sharded state, use version-supported distributed state-dict and DCP save APIs with all ranks. Include optimizer conversion/layout handling through supported APIs. Save at a completed update boundary with declared gradient clearing. Compare synchronous save first; asynchronous staging is a separately budgeted extension. Validate reload and the next matching update.

Capture model, optimizer, scheduler, sampler position, per-rank random state, and successful update count at a coherent boundary. Define gradients as cleared or explicitly included. A checkpoint from different logical steps on different ranks is not a valid restart. For full extraction, distinguish participation from result ownership: all required ranks execute collection even if only rank zero stores the full CPU result. Charge simultaneous extracted model and optimizer state to save-time RAM.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
import torch.distributed.checkpoint as dcp
from torch.distributed.checkpoint.state_dict import get_state_dict
```

```
@dataclass
class FSDPCheckpointContract:
    format_identity: str
    api_track: str
    optimizer_included: bool
    cpu_offload_result: bool
    full_result_rank_zero_only: bool
    committed_step: int


def save_full_fsdp1_checkpoint(runtime, destination, contract):
    # Participate collectively and materialize/write only supported full state.
    ...


def save_sharded_checkpoint(runtime, destination, contract):
    # Preserve distributed model/optimizer layout and completion metadata.
    ...


def E15(config):
    # Compare save-time resources, aggregate bytes, and resumed-update parity.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report per-rank host/device peaks, aggregate logical/file bytes, metadata, completed save duration, and next-update parity. Mark partial checkpoints uncommitted until the declared completion protocol succeeds. Do not call rank-zero-only ordinary saving a valid sharded checkpoint. Record software, format, architecture, wrap, and optimizer compatibility rather than promising arbitrary cross-version reload.

Count filesystem bytes once on a shared destination and separately report logical tensor payload, padding, and metadata. Sharding changes distribution, not necessarily aggregate storage. Use a declared completion marker/protocol and filesystem assumptions before labeling a checkpoint committed. Inject an incomplete-save control only in an isolated test destination and require reload to reject it. Compare the next resumed update, not merely successful key parsing. Asynchronous save is a separate experiment with staging and completion costs explicitly charged.

Part 7 - Engineering Interpretation and Philosophy

Explain which format meets training-restart and deployment needs with available host capacity. Preserve the difference between storage distribution, full-state interchange, and a complete reproducible training checkpoint.

19.40.17 Exercise 16

Exercise Title: Load a Sharded Checkpoint Directly into an FSDP Model Without Creating an Unnecessary Full-Model Copy on Every Rank

Part 1 - Exercise Contract and Why It Matters

Load distributed checkpoint state into a pre-established FSDP layout without creating a full-model copy on every rank. Measure construction, target allocation, read/staging, application, and first resumed-update peaks. Avoid moving an unnecessary full model through CPU or GPU before sharding.

Part 2 - System and Memory Mental Model

The target model and optimizer need supported sharding/layout metadata before DCP can place local state. A checkpoint directory is not simply an ordinary file loaded independently in full by every worker. Meta/deferred construction can avoid eager data allocation, but FSDP1 parameter initialization and unit materialization still have real peaks. A param_init_fn must follow the documented own-module initialization contract instead of recursively reinitializing children or silently breaking tied ownership.

Part 3 - Quantitative Prediction Before Execution

Predict target persistent shards, largest initialization unit, loader staging, optimizer state, and temporary duplicates. Compare an unnecessary full-per-rank load with the supported sharded path on a small feasible control. Calculate host-wide and worst-rank peaks. State whether world-size resharding is supported by the saved metadata and loader; test it only on sizes that fit the destination configuration.

Part 4 - Experimental Design and Instrumentation

Build matching model structure through a validated bounded/meta initialization path, establish FSDP1 ownership, and create the optimizer afterward. Use public distributed state-dict targets and DCP loading, then apply model/optimizer state through the supported adapter. Do not run an uninitialized forward or make an untracked optimizer update just to prepare loading. Restore committed step, RNG, scheduler, and sampler state. Validate missing/incompatible keys explicitly. Compare small-model tensors and the next update with an uninterrupted reference, then measure a larger bounded case.

Audit construction before loading. A full CPU model on every rank can defeat the host-memory objective even when GPU loading is sharded. Use a supported meta/bounded materialization adapter and initialize only the documented own-module state, preserving ties and child ownership. FSDP1 initialization happens before its flatten/shard operation for the relevant unit, so largest-unit peaks still need admission. Do not call a global model-to-CUDA move and later claim that only local shards were ever created.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
import torch.distributed.checkpoint as dcp
from torch.distributed.checkpoint.state_dict import get_state_dict, set_state_dict

@dataclass
class ShardedLoadPlan:
    checkpoint_identity: str
```

```
    target_world_size: int
    target_wrap_manifest: tuple
    initialization_policy: str
    optimizer_layout_policy: str
    version_support_identity: str


def build_sharded_load_target(plan, model_spec, optimizer_spec):
    # Establish valid local storage/layout without eager full-model replicas.
    ...


def load_and_apply_sharded_state(runtime, plan):
    # Load compatible distributed targets and restore committed training state.
    ...


def E16(config):
    # Validate restart parity and audit every initialization/loading peak.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report bytes read/staged, target state, host/device peaks, construction phases, supported resharding, and resumed-update errors. A successful local load on one rank is not proof of a consistent distributed restart. Do not promise zero allocation or zero full-unit materialization. If the chosen initialization path still constructs full data on every rank, disclose and correct that path before claiming the exercise objective.

Prepare model and optimizer layout targets through compatible public state-dict APIs and apply state at a supported update boundary. Check canonical parameter identity and optimizer mapping, not only tensor counts. Do not perform an unrecorded training step to initialize lazy state. Test world-size changes on a smaller feasible model and keep global data/update semantics documented. A larger destination-infeasible reload must be rejected or marked untested, not presented as automatic unlimited resharding.

Part 7 - Engineering Interpretation and Philosophy

Explain which unnecessary copies were removed and which layout/initialization costs remain unavoidable. Recommend the smallest compatible restart protocol whose correctness and resource behavior were actually validated.

19.40.18 Exercise 17

Exercise Title: Diagnose Rank-Imbalanced Memory Usage Caused by Uneven Module Wrapping, Shared Parameters, or Rank-Specific Workloads

Part 1 - Exercise Contract and Why It Matters

Diagnose unequal per-rank memory using synchronized phase evidence, wrap ownership, shared parameters, and workload shape. Distinguish legitimate layout/padding differences, rank-specific work, measurement skew, and invalid rank-dependent collective structure.

Part 2 - System and Memory Mental Model

All ranks normally require a compatible model and wrap plan. Uneven unit sizes common to every rank do not by themselves prove large rank imbalance. Rank-dependent wrapping can be an invalid distributed program, not a useful balancing option. Shared parameters need consistent supported ownership. Different sequence lengths, extra validation, rank-zero checkpoint materialization, retained graphs, or different phase timing can explain asymmetric peaks without unequal global model ownership.

Part 3 - Quantitative Prediction Before Execution

Predict shard padding and replicated state per rank, then add rank-local input/activation demand and any rank-zero-only work. Define imbalance from named per-rank scopes and leave empty denominators undefined. Construct a legitimate variable-shape case with identical collective order and a static manifest mismatch that must be rejected before training. Predict which phase each proposed mechanism should affect.

Part 4 - Experimental Design and Instrumentation

Collect rank/device IDs, wrap-manifest hashes, unit sizes, tied groups, input shapes, optimizer state, and observation phase. Compare matched shape controls before variable shapes. Keep conditional execution compatible with collectives; do not make one rank skip a wrapped module for an imbalance test. Record rank-zero diagnostic/checkpoint work separately. Use controlled interventions that remove one suspected cause while preserving updates. Gather scalar evidence at explicit boundaries without retaining all ranks' tensors on rank zero.

Hash and compare rank manifests before profiling a valid workload. Unequal module sizes shared by every rank are not equivalent to rank-dependent wrapping. Use variable input lengths only when every rank still follows compatible module/collective order and step count. Normalize losses to the intended global sample or valid-token objective. Record padding and sampler behavior so a short final batch cannot quietly change both memory and optimization semantics.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class RankImbalanceEvidence:
    rank: int
    phase: str
    wrap_manifest_identity: str
    input_shape: tuple
    local_state_bytes: int
    allocated_peak_bytes: int
    extra_work_identity: str


def validate_rank_manifests(rank_manifests):
    # Reject incompatible ownership/collective structure before execution.
```

```
    ...


def diagnose_rank_memory(rows, controlled_replays):
    # Separate layout, workload, instrumentation, and unsupported attribution.
    ...


def E17(config):
    # Reproduce a valid imbalance and test its cause with matched controls.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report rank distributions and worst rank, not only averages. A higher reserved value may reflect allocator history rather than more live shards. Shared parameter names are not independent bytes. Preserve uncertainty when observations were taken at different execution phases. A manifest mismatch requires repair before meaningful performance conclusions, not an attempt to tolerate mismatched collectives.

Control rank-zero validation, checkpoint extraction, tensor logging, and extra buffers separately. A retained autograd graph can create rank-specific growth even with identical shards. Compare matched phases and allocator history before attributing differences to persistent ownership. Shared/tied names must be deduplicated and their supported wrapper ownership verified. An invalid manifest should fail preflight; it must not be used as a live imbalance experiment that risks stranding other ranks in collectives.

Part 7 - Engineering Interpretation and Philosophy

Explain the strongest supported cause and a coordinated corrective action. Use attribution evidence rather than assuming all per-rank memory must match exactly or all differences come from wrapping.

19.40.19 Exercise 18

Exercise Title: Stress-Test FSDP with a Model That Cannot Fit Fully on Any Single GPU and Measure Scaling Across Increasing GPU Counts

Part 1 - Exercise Contract and Why It Matters

Test a bounded model whose declared fully resident training configuration cannot fit on one available GPU, then evaluate feasible FSDP configurations across available GPU counts. Separate actual distributed execution, analytical infeasibility, and unsupported hardware. One multi-GPU host is sufficient.

Part 2 - System and Memory Mental Model

Specify what cannot fit: parameters alone, complete persistent training state, or complete unsharded training peak. These are different claims. FSDP still needs each computation unit and workspace locally. A model with parameters too large for one GPU may also exceed sharded optimizer budgets at small world size; sharding alone is not guaranteed to make it feasible. A huge meta-only structure demonstrates construction, not an executed large-model training result.

Part 3 - Quantitative Prediction Before Execution

Calculate persistent shards, largest gather unit, activations, optimizer state, initialization, and checkpoint peaks for each candidate rank count. Use actual shard-group size and precision. Define strong scaling with fixed global work separately from weak scaling with fixed local work. Predict infeasible candidates before launch. Include a model whose largest indivisible unit cannot fit even when average shards would, and explain why more ranks alone do not solve that constraint.

Part 4 - Experimental Design and Instrumentation

Validate small-model numerical behavior first. Select dimensions and local batches from a safe capacity model, then run complete updates with warm-up and repeated bounded trials. Use only available device counts; mark others as estimated or untested. Avoid a full-model GPU construction before nested sharding. Measure host initialization and optional checkpoint/reload capacity. If offload or precision changes are needed for feasibility, label them as separate configurations rather than attributing all gains to world size. Coordinate failure/restart of the worker group.

State precisely whether the nonfitting claim concerns parameters alone, all persistent training state, or the complete unsharded execution. Build a phase-specific feasibility table before launch. More ranks do not reduce the full computation unit that one GPU must gather under this design. Use only available device counts and mark other points analytical. A one-rank FSDP configuration may not represent the intended multi-rank sharding behavior, so label its actual supported strategy.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class FSDPCapacityTrial:
    model_identity: str
    nonfitting_claim_scope: str
    world_size: int
    scaling_mode: str
    global_batch: int
    precision_identity: str
    offload_identity: str


def admit_capacity_trial(trial, predicted_phases, device_budgets):
    # Reject known-infeasible construction, units, or complete training state.
    ...


def run_capacity_scaling(trials, available_devices, training_contract):
    # Record actual complete work separately from estimates and unsupported N.
    ...
```

```
def E18(config):
    # Test feasible larger models without uncontrolled single-GPU OOM probing.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report claim scope, dimensions, rank counts, per-rank/host peaks, useful samples or tokens, complete time, update validity, and failure phase. A two-GPU run does not establish four- or eight-GPU scaling. A skipped infeasible single-GPU run has no measured throughput. Record the largest tested feasible case as a bound within the explored configuration set, not a proof of globally maximum model capacity.

For strong scaling, hold global useful work fixed and document local-batch changes; for weak scaling, hold local work fixed and acknowledge global batch growth. Preserve optimizer semantics and successful updates within each comparison. A logically huge meta model is a construction result until real training executes. Include initialization, first optimizer state, and checkpoint feasibility. Report the largest tested admissible model/configuration as a measured lower bound on explored capacity, not an exhaustive maximum over every possible precision, offload, or wrap policy.

Part 7 - Engineering Interpretation and Philosophy

Explain which constraints sharding relieved and which remain at unit, activation, host, or link level. Recommend the next bounded capacity experiment supported by the available hardware instead of extrapolating unmeasured scaling.

19.40.20 Exercise 19

Exercise Title: Build and Benchmark a PyTorch FSDP Training Runtime That Maximizes Model Capacity While Balancing Per-GPU VRAM, Communication, CPU Memory, and Throughput

Part 1 - Exercise Contract and Why It Matters

Integrate a version-pinned FSDP1 training runtime with deterministic wrapping, precision/offload support, gather scheduling, per-rank monitoring, and distributed checkpoint/restart. Benchmark quality-preserving useful updates while maximizing tested feasible model capacity under GPU, host, and throughput constraints. Substantive implementation remains learner work.

Part 2 - System and Memory Mental Model

One rank-consistent configuration governs model ownership, collectives, optimizer references, precision, checkpoint layout, and data/loss semantics. Persistent sharding does not remove full-unit materialization or activation peaks. Public FSDP1 controls must not be mixed with FSDP2 semantics. Changing wrap/strategy or optimizer layout during live training requires a separately validated coordinated rebuild/restart protocol, not arbitrary local attribute mutation. Distributed errors can strand peers if lifecycle handling is not coordinated.

Part 3 - Quantitative Prediction Before Execution

Predict construction, persistent state, gathered units, gradients, activations, optimizer workspace, CPU offload, communication, and checkpoint peaks for candidate configurations. Define quality/update parity, per-rank budgets, host budget, complete throughput, and failure criteria before selection. Compare DDP, conservative FSDP, and a few tested improvements under equal supported work. Keep strong/weak scaling and changed precision/offload configurations separately identified.

Part 4 - Experimental Design and Instrumentation

Freeze the support matrix and validate small deterministic updates before capacity tuning. Use separate calibration and held-out shape traces. Record hardware topology, PyTorch/CUDA/NCCL versions, API track, groups, wrap manifest, use_orig_params, precision fields, optimizer flags, data/sampler state, seeds, and committed-step IDs. Measure full training and checkpoint lifecycle, including first-update and initialization costs. Run ablations for wrapping, precision, checkpointing, offload, and scheduling without weakening correctness. Select configuration before launch or through a validated coordinated restart.

Publish supported API/version, optimizer, precision, accumulation, checkpoint, wrap, and topology combinations before benchmarking. Start from small correct updates and admission estimates. Use rank-consistent configuration selection before launch, with any change requiring a validated coordinated restart. Keep complete-phase budgets and worst-rank peaks as hard feasibility gates. A throughput objective must not hide dropped samples, skipped updates, incomplete checkpoints, or rank failures.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
import torch.distributed as dist

@dataclass
class FSDPTrainingRuntimeContract:
    api_track: str
    model_identity: str
    wrap_manifest_identity: str
    per_rank_budget_bytes: int
    host_budget_bytes: int
    precision_identity: str
    optimizer_identity: str
    checkpoint_protocol: str
    supported_features: tuple

class FSDPTrainingRuntime:
    def __init__(self, contract):
        # Establish rank-consistent topology, model, optimizer, and monitoring.
        ...

    def train_step(self, local_batch):
        # Preserve matching collectives and complete useful update semantics.
```

```
        ...

    def checkpoint(self, destination, committed_state):
        # Save a compatible complete distributed restart boundary.
        ...

    def restore(self, checkpoint_identity):
        # Rebuild valid target layout and verify coherent resumed progress.
        ...

    def audit(self):
        # Reconcile phase-specific state, peak scopes, and rank invariants.
        ...

    def shutdown(self, outcome):
        # Coordinate owned work/process-group lifecycle and report failures.
        ...


def E19(config):
    # Validate runtime contracts before held-out capacity/throughput evaluation.
    ...
    # Export evidence, limitations, counterexamples, and recommendation.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Acceptance requires correct updates, compatible collective order, supported state/precision handling, bounded resource phases, and validated checkpoint restart. Report all rank peaks, host scope, communication evidence, useful samples/tokens, completed/failed updates, timing distributions, checkpoint cost, and tested capacity. Keep prediction, observation, and interpretation distinct. An incomplete scaffold supplies no measured multi-GPU result.

Evaluate an unfavorable FSDP case, a gather-limited case, host-limited offload, a checkpoint/reload boundary, and an unavailable GPU-count point. Report useful samples/tokens, complete wall time, rank spread, communication evidence, actual resource scopes, and numerical outcomes. An operational failure needs a whole-group lifecycle response, not a local catch-and-continue loop. Recommend the smallest validated configuration and name the next bounded experiment needed before adding FSDP2 migration, runtime rewrapping, or untested scaling claims.

Part 7 - Engineering Interpretation and Philosophy

Recommend the smallest validated configuration meeting the real constraint, including DDP when it wins. Explain the capacity gained and communication, recomputation, host memory, or latency paid. Separate measured features from proposed FSDP2 migration, additional GPU counts, or unsupported runtime adaptation.

19.41 DMM39

Chapter Title: ZeRO-Style Memory Partitioning

Specialization: Dynamic Memory Management and CUDA Systems

19.41.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Redundant Model States in Data Parallel Training and How ZeRO Eliminates Replication

Part 1 - Exercise Contract and Why It Matters

Model redundant training state and the successive ownership changes called ZeRO stages. Deliver a phase diagram for parameters, reduced gradients, Adam state, communication, and temporary computation buffers. Build an explicit educational PyTorch engine rather than claim a complete DeepSpeed reimplementation or silently delegate the exercise to a high-level trainer.

Part 2 - System and Memory Mental Model

Stage 0 replicates model state. Stage 1 partitions optimizer ownership; Stage 2 also partitions reduced gradients; Stage 3 partitions persistent parameters and materializes computation units when needed. These labels do not specify every peak, bucket, or synchronization detail. The primary track uses float32, a static functional dense model, all trainable parameters used each step, one process per CUDA device, and one multi-GPU host. Arbitrary modules, mutation, higher-order gradients, ties, and conditional execution need separately validated support.

Part 3 - Quantitative Prediction Before Execution

Let P, G, and O denote full parameter, reduced-gradient, and optimizer payload bytes under declared dtypes, and let N be partition-group size. Predict ideal persistent payloads:

$$M_0 = P + G + O, \quad M_1 = P + G + O/N, \quad M_2 = P + (G + O)/N, \quad M_3 = (P + G + O)/N.$$

Add padding, replicas, activations, buckets, gathered units, and workspaces. These are ownership estimates, not exact peak-VRAM formulas.

Part 4 - Experimental Design and Instrumentation

Define one canonical logical parameter/offset map on every rank. Preserve parameter IDs, valid spans, padding, dtype, and optimizer step semantics. Validate small single-rank Adam behavior, then a distributed replicated reference before partitioning. Record local/global rank, device binding, group membership, model/layout identity, versions, topology, seeds, data/loss normalization, and initial state. Use one update per step initially. Keep custom collectives separate from DDP synchronization so gradients are never reduced twice.

Create a state-ownership table with rows for logical parameters, reduced gradients, first moments, second moments, step counters, gathered copies, and communication workspaces. Give each row an owner, replication factor, creation phase, last reader, and release condition. In a two-rank paper trace, follow one scalar through local gradient production, averaging, owner update, and publication to both ranks. Show the corresponding change for each stage. Use this trace to distinguish eliminating redundant storage from discarding necessary information. Include non-model allocations in a separate context row so the model-state diagram does not masquerade as a complete device inventory. Identify one invariant that remains identical at every stage: the intended global optimizer update.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
import torch.distributed as dist

@dataclass
class ZeROStyleContract:
    stage: int
    partition_group_size: int
    model_layout_identity: str
    parameter_dtype: str
    gradient_dtype: str
    optimizer_identity: str
    supported_graph_identity: str


def predict_zero_ownership(contract, state_inventory, phase_plan):
    # Separate ideal persistent state and complete live execution capacity.
    ...


def E00(config):
    # Freeze the restricted engine semantics and validate canonical rank layout.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Keep prediction, observation, and interpretation distinct. Require correct useful updates, compatible collective order, complete phase accounting, and honest execution support. A smaller post-step state is not proof of a smaller backward peak. Export bounded CPU scalar evidence without retaining full GPU state. Unrun multi-GPU results remain unmeasured rather than being filled with published benchmark numbers.

Part 7 - Engineering Interpretation and Philosophy

Explain how partitioning removes redundancy without eliminating each rank's need for correct computation inputs and synchronized updates. The strongest outcome is an auditable restricted engine whose limits are clear before adding persistence, prefetch, or offload.

19.41.2 Exercise 01

Exercise Title: Calculate Per-GPU Memory for Parameters, Gradients, and Adam Optimizer States Before Applying Any ZeRO Partitioning

Part 1 - Exercise Contract and Why It Matters

Calculate replicated per-rank parameter, gradient, and Adam-state memory before partitioning. Separate actual float32 storage from a conventional mixed-precision model with master parameters. Validate the calculation after optimizer state exists and record excluded workspace and metadata.

Part 2 - System and Memory Mental Model

Adam tracks first and second moments plus step-related state. Its dtypes, lazy allocation, epsilon, bias correction, weight decay, and implementation flags matter. The primary all-float32 model does not need a second float32 master copy. A mixed-precision implementation may retain one, which belongs in the optimizer/authority ledger exactly once. Gradient buckets, parameter views, tied storage, and foreach/fused temporaries must not be double-counted.

Part 3 - Quantitative Prediction Before Execution

For n all-trainable elements, derive payload from each component's element width. Compare float32 parameters/gradients/two float32 moments with a separately declared low-precision parameter/gradient plus float32 master/moment model. Explain why two configurations can have the same nominal bytes for different reasons. Add step counters and padding separately. Calculate memory before and after the first optimizer update.

Part 4 - Experimental Design and Instrumentation

Use small tensors with a standard PyTorch Adam reference and fixed supported flags; begin without weight decay or AMSGrad. Inspect actual moment and counter state after a complete step. Deduplicate storage and count trainable, frozen, or unused entries only under declared semantics. Include zero versus absent gradient behavior as a separate support test. Record parameter shapes, dtypes, loss reduction, optimizer settings, gradient-clearing policy, and measurement boundaries. Compare payload accounting with allocated/reserved CUDA observations under their separate scopes.

Use two explicit optimizer inventories rather than one customary bytes-per- parameter constant. The primary inventory has FP32 parameters and gradients with FP32 first and second moments. The optional mixed-precision inventory must declare whether it actually stores an FP32 master copy, its gradient dtype, and which process owns each representation. List integer or floating step counters according to observed implementation, including per-parameter rather than assumed per-element costs. Compare numerical element counts with unique backing storage to avoid double-counting views. If an optimizer implementation retains additional tensor lists or update workspaces, report them as measured implementation overhead. Never subtract an imaginary master copy from a run that never allocated one.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class AdamStateSpec:
    parameter_element_bytes: int
    gradient_element_bytes: int
    moment_element_bytes: int
    master_element_bytes: int
    master_copy_present: bool
    optimizer_flags: tuple


def predict_replicated_adam_bytes(parameter_inventory, spec):
```

```
    # Count each representation once and leave workspace outside payload.
    ...


def inspect_initialized_adam(model, optimizer, phase):
    # Observe lazy state, actual dtypes, counters, and unique backing.
    ...


def E01(config):
    # Reconcile declared formulas with initialized optimizer storage.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report element counts, actual tensor fields, dtype widths, master-copy policy, scalar state, useful payload, workspace, and backend peaks. An empty optimizer state before its first update is not evidence of zero steady-state Adam cost. Do not apply a familiar bytes-per-parameter rule without its dtype and master-copy assumptions. Preserve unknown overhead rather than labeling it fragmentation.

Part 7 - Engineering Interpretation and Philosophy

Explain which bytes partitioning can distribute and which belong to data, activations, communication, or implementation overhead. Use the inventory as the shared basis for later stage comparisons.

19.41.3 Exercise 02

Exercise Title: Build a Standard PyTorch Data-Parallel Baseline and Measure Replicated Model-State Memory Across All GPU Ranks

Part 1 - Exercise Contract and Why It Matters

Build a standard PyTorch distributed data-parallel training baseline and measure replicated state across all ranks. Establish numerical update, global data, and complete-step timing references before introducing custom ownership or synchronization.

Part 2 - System and Memory Mental Model

DDP replicates parameters and synchronizes gradients; an ordinary local Adam keeps its state on every rank. DDP does not distribute input samples automatically. Local loss normalization and rank reduction together define the global update. Optimizer state can appear lazily, gradient buckets can alias gradients, and first-step initialization can change memory. The custom engine later replaces the gradient communication path rather than stacking another reduction on top of DDP.

Part 3 - Quantitative Prediction Before Execution

Predict replicated parameter, gradient, moment, buffer, and bucket payload for each rank. Define global batch from local batch, world size, and accumulation, with accumulation initially one. Predict initialization and first-update peaks separately from warmed execution. Define

global samples or valid tokens per completed elapsed second and use a named worst-rank timing/peak aggregation.

Part 4 - Experimental Design and Instrumentation

Bind devices using local rank, initialize the supported process group, construct identical model values, and use explicit distributed sampling. Run fixed seeded batches with matching loss reduction and optimizer flags. Compare a small update with a single-process global-batch reference. Capture state before/after forward, backward, update, and clearing in a diagnostic replay. Use separate minimally instrumented timing and scalar logs, avoiding rank-zero collection of live tensors from every worker.

Keep the primary dataset small enough to make global sample identities auditable. Record the rank-local sample IDs and verify that their concatenation matches the reference global batch at each update. Avoid batch-normalization state and stochastic layers in this first oracle. A reported mean of local loss scalars is comparable only when the weighting and sample counts agree. Document whether a final partial batch is dropped or weighted. Measure DDP communication buffers separately when observable and include their capacity in device peaks. Confirm that all parameters have received gradients before sampling initialized Adam state. Use an additional seed to check repeatability without changing the primary comparison's initial-state identifier.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP

@dataclass
class ReplicatedRankEvidence:
    rank: int
    phase: str
    state_bytes: object
    allocated_peak_bytes: int
    reserved_peak_bytes: int
    completed_samples: int


def build_replicated_reference(model_spec, distributed_context):
    # Establish correct data, optimizer, and DDP ownership/synchronization.
    ...


def measure_replicated_steps(runtime, trace, measurement_contract):
    # Validate complete updates and preserve rank-specific resource scopes.
    ...
```

```
def E02(config):
    # Establish the replicated numerical and resource baseline.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report rank distributions, initialized state, unique backing, complete peaks, successful update count, sample coverage, and latency variability. Do not sum aliased gradient buckets twice or infer total device usage solely from tensor inventories. An unsynchronized host interval is not completed training time. Missing devices or unsupported collectives must remain explicit execution limits.

Part 7 - Engineering Interpretation and Philosophy

Identify the replicated state targeted by Stage 1 and the baseline costs that will remain. Preserve this reference for small-model equivalence even when later larger models become infeasible without partitioning.

19.41.4 Exercise 03

Exercise Title: Implement a Simplified ZeRO Stage 1 Optimizer-State Partitioning Scheme Across Multiple PyTorch Processes

Part 1 - Exercise Contract and Why It Matters

Implement a restricted Stage 1 scheme that partitions Adam state while keeping model parameters and reduced gradients replicated. Use a canonical flattened layout and explicit owner-span updates, then synchronize updated parameters so every rank starts the next step identically.

Part 2 - System and Memory Mental Model

Each valid element has one optimizer owner with local moments and correct logical-parameter step state. Owner updates operate on its span of the replicated parameter backing; they need not create an uncounted master replica in the all-float32 primary model. The gradient is averaged exactly once before owner updates. Updated spans must reach every rank afterward. A simple owner-broadcast protocol can use more communication than optimized production ZeRO and must be measured as implemented.

Part 3 - Quantitative Prediction Before Execution

Predict valid/padded elements and moment bytes per owner, replicated P/G, step metadata, and synchronization payload. Hand-simulate two ranks updating different spans over several Adam steps with identical hyperparameters. Include a logical parameter crossing a partition boundary and define its step-counter consistency. Estimate full-gradient reduction plus owner-parameter broadcast cost without borrowing an optimized algorithm's traffic formula.

Part 4 - Experimental Design and Instrumentation

First validate partitioned scalar Adam arithmetic against torch.optim.Adam on a small deterministic model. Freeze rounding/epsilon/decay conventions and start with all logical parameters used each update. Implement custom communication without DDP's reducer: aggregate gradients under the declared averaging rule, update only valid owned spans, then broadcast

canonical updated spans in the same order on all ranks. Exclude padding from optimization. Use one host process per available CUDA device and verify layout hashes before collective work.

For a logical parameter cut by an ownership boundary, attach both owned spans to the same logical optimizer-step record. Their elementwise moment updates must use the same hyperparameters and bias-correction step. A rank with no valid elements still participates in required communication but need not create an invalid empty optimizer instance. Pad only the communication representation; exclude padding from state initialization, norms, and updates. Build a tiny reference with two differently sized parameters so a counter accidentally advanced per span becomes visible after several steps. Run parameter publication in a fixed owner order on every rank, including owners whose valid ranges are empty, according to the chosen common protocol.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
import torch.distributed as dist

@dataclass
class OptimizerOwnerSpan:
    owner_rank: int
    flat_start: int
    valid_elements: int
    padded_elements: int
    logical_parameter_intersections: tuple

class PartitionedAdam:
    def initialize_owned_state(self, layout, optimizer_contract):
        # Allocate only owned moments and required step/master state.
        ...

    def update_owned_spans(self, replicated_parameters, averaged_gradients):
        # Match Adam semantics and leave nonowned/padded elements untouched.
        ...


def synchronize_updated_parameters(layout, parameters, group):
    # Publish every owner update through a canonical matching collective order.
    ...


def E03(config):
    # Validate several updates against the replicated Adam reference.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require matching global updated parameters, moments on their owners, and loss trajectory within a predeclared tolerance. Record communication, source lifetimes, padding, local state bytes, and complete peaks. A rank updating only its own span without synchronizing replicas has not implemented correct Stage 1 training. Reject unsupported absent-gradient or shared-parameter semantics rather than silently approximating them.

Part 7 - Engineering Interpretation and Philosophy

Explain what was partitioned and the extra synchronization chosen for this transparent implementation. Keep production ZeRO efficiency claims separate from the educational engine's actual protocol and supported graph.

19.41.5 Exercise 04

Exercise Title: Verify That Every Rank Stores Only Its Assigned Adam State Partition While Model Parameters and Gradients Remain Replicated

Part 1 - Exercise Contract and Why It Matters

Verify optimizer-state ownership independently of the Stage 1 counters. Confirm that parameters and reduced gradients remain replicated at their declared phase, while moment/master state exists only for each rank's assigned valid span. Validate ownership and numerical correctness together.

Part 2 - System and Memory Mental Model

Logical parameter names and physical span ownership are different views. A parameter can intersect multiple owner spans; its step semantics must still match the reference. Padding may occupy storage but is not a trainable element. Debug references, conversion copies, or full optimizer reconstruction can accidentally retain replicated state. A memory drop alone cannot prove unique ownership, and matching loss alone cannot prove memory was partitioned.

Part 3 - Quantitative Prediction Before Execution

From the canonical map, calculate each rank's valid intersections, moment tensors, optional masters, and counters. Predict which components must be equal across replicas after synchronization. Construct a deliberately duplicated moment allocation and a missing owner interval as negative controls in a bounded metadata/state test. Define a unique-coverage invariant over all valid logical elements.

Part 4 - Experimental Design and Instrumentation

Enumerate actual state tensors after multiple updates and compare them with a recomputed interval-cover oracle. Inspect full parameter/gradient parity only at a small deliberate validation boundary, then release copies before memory timing. Test empty owner spans, nondivisible sizes, boundary-crossing parameters, stale layout IDs, and padding. Verify all ranks use the same map and successful update count. Keep the primary all-used-gradient contract explicit so zero and absent gradients are not accidentally conflated.

Audit absence of hidden replicas by traversing actual state tensor storages, not by counting dictionary keys. For each owned interval, verify coverage, nonoverlap, dtype, device, and logical parameter identity. Keep a compact CPU manifest for cross-rank comparison rather than collecting all moments on every GPU. As a controlled failure, intentionally retain a full moment copy in a separate diagnostic replay and show that the audit catches its bytes even

if the optimizer never uses it. Check parameter replicas after publication, because owner-only moments with unsynchronized parameter updates would pass a narrow state audit while implementing incorrect data parallel training. Preserve both checks as independent acceptance conditions.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass

@dataclass
class Stage1OwnershipEvidence:
    rank: int
    layout_identity: str
    valid_owned_intervals: tuple
    moment_storage_bytes: int
    master_storage_bytes: int
    logical_step_state: tuple


def audit_stage1_ownership(layout, actual_state_rows):
    # Recompute exact coverage and reject gaps, duplicates, and invalid padding.
    ...


def verify_replicated_values(parameters, gradients, validation_contract):
    # Check supported post-sync parity without retaining diagnostic full copies.
    ...


def E04(config):
    # Validate ownership, replica agreement, and negative controls independently.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report actual state allocations, interval coverage, dtype, padding, replica differences, and retained debug storage. A checksum can be a useful screen but needs an exact small-tensor comparison to establish the oracle. Do not infer private ownership solely from an optimizer class name or rank-local state dictionary size. Preserve unsupported introspection fields as unknown.

Part 7 - Engineering Interpretation and Philosophy

Explain the evidence proving that redundancy was removed without losing required updates. Use the same ownership oracle when gradients and parameters become partitioned in later stages.

19.41.6 Exercise 05

Exercise Title: Measure ZeRO Stage 1 Per-Rank VRAM Savings and Compare Them Against the Theoretical Optimizer-State Partitioning Formula

Part 1 - Exercise Contract and Why It Matters

Measure the memory saved by optimizer-state partitioning and reconcile it with the ownership audit. Deliver both a steady-state comparison and the maximum allocated memory of a complete training step. A smaller optimizer state dictionary alone is insufficient evidence of a lower training peak.

Part 2 - System and Memory Mental Model

Use the same parameter and gradient dtypes in both runs. Stage 1 retains replicated parameters and gradients while splitting the selected optimizer state. Temporary update buffers, packed communication payloads, allocator rounding, and the distribution of valid elements can change the observed saving. The simple formula predicts a persistent component, not the entire process. Adam initializes state lazily, so compare after every supported parameter has received an update. Retaining a full reference optimizer in the measured process would erase the intended saving; use a separate correctness replay instead.

Part 3 - Quantitative Prediction Before Execution

With byte quantities from E01, predict

$$M_0 = P + G + O, \qquad M_1 = P + G + O/N, \qquad \Delta M = O(1 - 1/N).$$

Evaluate this for one million FP32 elements and two ranks, counting the two FP32 moments and listing counters separately. Then replace equal division by each rank's actual assigned valid state. Predict why the most heavily loaded rank can determine feasibility even when the mean agrees with the ideal.

Part 4 - Experimental Design and Instrumentation

Warm both implementations from identical initial values and identical data. Record allocation after initialization, after backward, after update, and after parameter synchronization. Reset peaks only at the declared full-step boundary. Use repeated warmed steps and independent process launches. Synchronize at measurement boundaries, not after every layer. Record model dimensions, seed, Adam options, device identity, backend, versions, topology, world size, and whether timing includes input transfer. Gather CPU scalar summaries after the region. Preserve every rank's peak rather than averaging away the worst rank.

Add a phase in which gradients are cleared using the same policy in both runs, and state whether clearing releases storage or merely zeroes retained buffers. Otherwise the sampled boundary can manufacture a difference unrelated to optimizer partitioning. Use a CPU-side category ledger and deduplicate shared storage before summing. Compare the sum with allocator observations without assuming equality: the allocator also serves temporary and rounded requests. Record the discrepancy's sign and phase before proposing a cause. Include world size one as a sanity check where ownership partitioning alone predicts no redundancy saving. For larger world sizes, use the actual rank assignment rather than scaling one observed rank mechanically.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def predict_stage1_bytes(inventory, owner_map, world_size):
    ...


def sample_training_phases(engine, batch, measurement):
    ...


def E05(config):
    # Separate correctness replay from the warmed memory comparison.
    ...
    # Reconcile per-rank persistent state and complete-step peaks.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report predicted persistent bytes, observed live storage by category, peak allocated bytes, reserved bytes, and discrepancy for each rank. Distinguish rounding and temporary allocations from unexplained residuals. A reserved-memory plateau can persist after tensors are released; do not call it a replicated optimizer without inspecting storage ownership. Include an uninitialized-Adam negative control to expose a misleading comparison. If Stage 1 has a larger peak because of the educational synchronization protocol, report that outcome and its phase. Do not manufacture an expected improvement.

Part 7 - Engineering Interpretation and Philosophy

Explain which portion of the measured saving follows from ownership and which portion depends on implementation details. Choose a diagnostic that could invalidate the formula's assumptions before using it for capacity planning.

19.41.7 Exercise 06

Exercise Title: Extend the Runtime to ZeRO Stage 2 by Partitioning Reduced Gradients Alongside Optimizer States

Part 1 - Exercise Contract and Why It Matters

Extend the educational runtime so each rank retains only its owned reduced gradient shard along with its optimizer state. Keep parameters replicated. Deliver a correct Stage 2 update and a precise statement of whether the implementation reduces only persistent gradients or also their production peak.

Part 2 - System and Memory Mental Model

A gradient is first produced by local differentiation and then combined across ranks. Its final owner does not imply that only that owner produced it. An initial reference implementation may form a full gradient vector on every rank before reduce-scatter. This can demonstrate sharded retention, but its backward peak still includes the full vector. A bounded production path needs an explicit bucket lifecycle or validated custom differentiation boundary. Returning

None from an ordinary tensor gradient hook does not suppress normal gradient accumulation. Do not label an unverified hook as a memory-saving reducer.

Part 3 - Quantitative Prediction Before Execution

For equal ownership, predict persistent model state

$$M_2 = P + (G + O)/N, \qquad M_1 - M_2 = G(1 - 1/N).$$

Draw separate timelines for full-gradient production followed by partitioning and for bounded bucket production. Add the largest live production bucket, its communication input, its output shard, and overlapping buffers to the appropriate phase. Explain why a persistent saving need not reduce the maximum allocated bytes of the first timeline.

Part 4 - Experimental Design and Instrumentation

Start with one tiny functional network, FP32, fixed execution order, and all parameters used. Obtain local gradients, pack them in the canonical order, combine them exactly once, update owner spans, and synchronize changed parameter spans before the next forward. Compare against the same global-batch Adam reference for several updates. Only after correctness passes, implement a bounded producer with a documented ownership transfer at each boundary. Prevent rank-local readiness from changing collective order. Record whether full leaf gradients, packing copies, or stale aliases survive in each implementation.

Treat the gradient producer as a separately named component with its own correctness and memory contract. The simple full-gradient producer is allowed as a reference but must retain that label in tables and plots. A bounded producer must demonstrate that no full gradient replica survives outside its declared window, including leaf gradients, packing aliases, and temporary outputs. Validate the producer on a two-unit graph before applying it to the whole network. If the implementation cannot meet this condition, preserve the correct Stage 2 retention result and state the missing peak-memory property. Do not blur a future optimization plan with a capability already demonstrated by the completed learner implementation.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
import torch.distributed as dist


def produce_gradient_buckets(graph, batch, layout, schedule):
    # Declare full-gradient reference or validated bounded producer.
    ...


def stage2_step(engine, batch, evidence):
    # Reduce each canonical bucket once and retain only owned results.
    ...


def E06(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Compare reconstructed gradients and parameters with the reference outside the production timing region. Audit both post-reduction bytes and backward peak bytes. Reject double averaging, omitted parameter spans, and simultaneous DDP reduction plus custom reduction. Use explicit unsupported status for a producer that cannot safely bound its full gradients. Inspect gradients after a second step to catch accidental accumulation from retained full replicas. An unchanged peak is a useful result when activations or full-gradient production dominate.

Part 7 - Engineering Interpretation and Philosophy

Explain the difference between choosing the destination of a result and controlling the memory needed to produce it. State the additional engineering work required before the runtime may claim bounded gradient memory.

19.41.8 Exercise 07

Exercise Title: Implement Reduce-Scatter for Gradient Partitioning and Verify That Each Rank Retains Only the Gradient Shard It Owns

Part 1 - Exercise Contract and Why It Matters

Implement and validate reduce-scatter for the canonical gradient layout. Each rank must receive the correct reduced slice, with padding excluded from logical updates. Deliver an independently checked collective protocol before using the operation as an optimization.

Part 2 - System and Memory Mental Model

For the equal-size tensor interface, pack rank-ordered output chunks into one contiguous input. Every participating rank supplies a matching layout and compatible dtype, count, device, group, and collective sequence. The output contains one reduced chunk, not a full reduced vector. Uneven logical ownership can be represented with explicit padding and a valid-element mask. Do not assume that changing a rank's local length changes the collective contract. Keep input and output storage alive until the selected completion protocol permits reuse; launch completion and downstream CUDA consumption require correct stream dependencies.

Part 3 - Quantitative Prediction Before Execution

Let a padded gradient vector contain NQ elements. Predict an output of Q elements per rank and account separately for the full input buffer. For rank r, the average-gradient oracle is

$$y_r = \frac{1}{N} \sum_{j=0}^{N-1} x_j[rQ : (r+1)Q].$$

Use rank-distinct integer-like FP32 inputs to calculate every output by hand. Then identify how unequal local sample counts would change the loss weighting; world-size division alone is insufficient for arbitrary uneven batches.

Part 4 - Experimental Design and Instrumentation

Use equal local batch sizes in the primary experiment. Compare the result with an all-reduce-and-slice oracle in a separate replay. Cover one rank, two available ranks, a nondivisible logical length, a zero-valid owner, and a logical parameter crossing a chunk boundary. Preserve identical collective order on all ranks. Test synchronous execution first, then an asynchronous

variant with explicit completion and consumer dependencies. Capture launch, completion, and first use as distinct trace events. Never intentionally hang a real run to test inconsistent counts; validate contracts before launch.

Include a hand-computed case with different contributions from every rank so a scatter without reduction cannot accidentally pass. Use another case with identical contributions to expose double division by world size. Check a logical parameter that spans two output chunks and verify each slice against its expected global index range. Padding values should begin from a known neutral contribution for SUM, but no padded output may reach Adam. Inspect storage extents for asynchronous input and output reuse. The collective validator should compare metadata before launch using a compatible control path and terminate the group coherently on rejection. Do not place an extra debug collective into only one rank's execution path.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
import torch.distributed as dist


def pack_padded_gradient(local_gradient, layout):
    ...


def reduce_scatter_owned(packed, layout, group, completion):
    # SUM, normalization, output ownership, and waits remain learner work.
    ...


def E07(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report maximum and relative error on valid elements, padding behavior, normalization convention, and the exact output interval for every rank. Check that input aliases are not overwritten before completion. Distinguish algorithmic tensor payload from measured link traffic: backend algorithms can move data differently. Preserve the oracle's extra storage outside the memory comparison. Include a deliberate omitted-normalization case and show the resulting update discrepancy without presenting it as a valid implementation.

Part 7 - Engineering Interpretation and Philosophy

Explain why a collective is also an agreement about shape, ordering, ownership, and time. Identify the smallest metadata mismatch your validator catches before it could become a distributed failure.

19.41.9 Exercise 08

Exercise Title: Compare ZeRO Stage 1 and Stage 2 by Peak VRAM, Communication Volume, Gradient Memory, and Training Throughput

Part 1 - Exercise Contract and Why It Matters

Compare validated Stage 1 and Stage 2 implementations with identical training semantics. Measure persistent gradients, complete-step memory, communication, and throughput. Keep the full-gradient Stage 2 reference and any validated bounded producer as separate named implementations.

Part 2 - System and Memory Mental Model

Stage 1 can use full-gradient all-reduce followed by owner updates and parameter broadcasts. The educational Stage 2 substitutes gradient reduce-scatter while retaining the same explicit parameter synchronization. Its communication need not match the optimized algorithms reported for a production ZeRO implementation. Packing, launch count, overlap, update scheduling, and activation memory can dominate the practical comparison. The two stages must still implement the same averaged global objective and the same Adam recurrence; a faster incorrect update is not a valid point.

Part 3 - Quantitative Prediction Before Execution

Predict retained gradient bytes for each rank from the actual ownership map. Build a communication ledger listing operation, payload dtype, tensor count, valid elements, padding, and calls per update. Keep logical payload separate from a backend-dependent estimate of transmitted bytes. Predict a case where Stage 2 saves persistent storage but does not improve throughput, and another where a larger usable model offsets additional communication.

Part 4 - Experimental Design and Instrumentation

Use fresh jobs and the same seed, logical parameter initialization, global batch, local batch, accumulation schedule, Adam configuration, and number of updates. Validate several parameter updates before the timed region. Warm kernels and optimizer states, then measure a complete forward, backward, update, and replica synchronization. Collect rank-local times and report the slowest synchronized step as the distributed critical path. Run profiling separately from low-overhead timing. Include both a communication-light network and a larger network within the tested device budget; do not change model size inside the primary equal-model comparison.

State how throughput is computed from completed global examples rather than multiplying an arbitrary rank's local counter. If accumulation is fixed, include all microbatches belonging to an optimizer update in the timing region. Keep CPU evidence gathering after the region and bound its frequency so logging does not dominate the experiment. For communication, show the educational parameter broadcasts even if the gradient collective becomes smaller. A table that includes only gradient traffic would hide an unchanged or dominant cost. Use repeated runs to report variability and preserve a slow-rank trace when the average appears attractive. Compare the same successful logical updates, not merely the same number of kernel launches.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def build_communication_ledger(engine, measured_steps):
```

```
    ...


def compare_stage1_stage2(replay_specs, evidence):
    ...


def E08(config):
    # Correctness, isolated replays, and comparison remain incomplete.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export per-rank persistent gradient bytes, peak allocated and reserved bytes, phase durations, examples per second, collective counts, and numerical errors. Label link-level traffic unsupported if only tensor payload is known. Explain any rank imbalance before summarizing across ranks. Include a case where full-gradient production hides the retained-gradient saving at the peak. Do not interpret a lower peak after a changed batch size as a stage-only effect. If no bounded producer was validated, limit the conclusion accordingly.

Part 7 - Engineering Interpretation and Philosophy

Choose the stage on evidence about the limiting resource. Explain why model-state theory, communication scheduling, and observed throughput answer different questions, and identify the next experiment that would resolve an ambiguous result.

19.41.10 Exercise 09

Exercise Title: Add Contiguous Gradient Buffers and Measure Their Effect on Allocation Count, Fragmentation, and Communication Efficiency

Part 1 - Exercise Contract and Why It Matters

Add contiguous gradient buffers with a declared capacity and reuse policy. Compare allocation behavior and communication packing overhead while preserving the verified reduction and optimizer semantics. Deliver a storage alias map and a lifetime trace, not merely a flat tensor allocation.

Part 2 - System and Memory Mental Model

A contiguous buffer may replace many gradient storages, or it may become an additional copy beside them. Those cases have different memory costs. Preallocation can reduce repeated allocation requests while retaining capacity between steps. Neither fewer allocations nor one flat address proves lower allocator fragmentation. Distinguish tensor creation, allocator block requests, reserved capacity, and driver allocation activity. Contiguous gradients do not make the full model state contiguous or remove activation workspaces.

Part 3 - Quantitative Prediction Before Execution

For bucket capacity C elements, predict bytes for producer storage, communication input, owned output, padding, and any overlap between successive buckets. Count distinct storage once when views share a backing allocation. Estimate collective calls from the canonical bucket

boundaries, including a partial final bucket. Compare a reuse implementation with a copy-then-reduce implementation and predict which can have the larger live peak despite identical payload sizes.

Part 4 - Experimental Design and Instrumentation

Freeze parameter ordering and bucket boundaries across ranks. Build a storage inventory before and after backward. Determine whether the producer writes directly into communication storage or copies into it. If using custom autograd or hooks, document when ordinary gradient storage is created and released; do not assume a hook eliminates it. Sweep a small set of bucket capacities using independent replays. Gate launch order by canonical sequence rather than local readiness. Test a second update, a partial bucket, and an accumulation boundary. Keep addresses only as diagnostic metadata and record storage identity plus offset and extent for aliases.

Test buffer capacity at one element below, exactly at, and just above a logical bucket boundary. Verify that reshaping a view does not create an unrecorded independent storage and that a stale view cannot mutate a bucket already assigned to the next step. If gradients accumulate in place, define when zeroing occurs relative to communication completion. Compare stable shape reuse with a separately controlled capacity-growth event. Include the allocation and initialization cost of growth in cold behavior instead of removing it from all reports. A permanent oversized gradient arena is a legitimate design choice only when its retained capacity is visible in the memory budget and comparison.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


class GradientBufferPlan:
    def allocate(self, layout, capacity, device):
        ...

    def accept_produced_gradient(self, parameter_id, gradient):
        ...

    def release_after_completion(self, bucket_id, completion):
        ...


def E09(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report payload, allocated capacity, internal padding, copied bytes, allocation events supported by the selected instrument, and complete-step peaks. Use native allocator statistics only with their backend-specific scope; unknown counters remain unsupported. Include a synthetic fragmented free-set example to show why allocation count cannot diagnose fit for a later

request. Check buffer versions across steps and fail on premature overwrite. A lower packing time with a higher reserved plateau is a legitimate trade-off.

Part 7 - Engineering Interpretation and Philosophy

Decide whether the buffer improves this workload through reuse, reduced copying, fewer launches, or another measured mechanism. Avoid turning a useful layout technique into a universal claim about fragmentation.

19.41.11 Exercise 10

Exercise Title: Extend the Runtime to ZeRO Stage 3 by Partitioning Model Parameters in Addition to Gradients and Optimizer States

Part 1 - Exercise Contract and Why It Matters

Partition persistent parameters in addition to gradients and optimizer state. Build a restricted Stage 3 runtime whose gathered computation units are temporary and whose local shards are the authoritative trainable state. Deliver correct multi-step training before claiming the ideal memory reduction.

Part 2 - System and Memory Mental Model

Ordinary autograd may retain a full weight for an input-gradient calculation. Replacing its data with an empty tensor after forward does not establish a valid release protocol. Use a restricted custom autograd unit: gather parameters for forward, save inputs and shard/version metadata rather than full weights, regather the same version in backward, and recompute the functional unit under gradient recording. Reduce-scatter its parameter gradients exactly once and return local shard gradients to the sharded leaf parameters. Do not additionally wrap this path in a DDP reducer.

Start with deterministic dense units, no dropout, no mutable running buffers, no shared parameters, and first-order gradients only. These restrictions are part of the contract, not general properties of Stage 3. Save tensors through the appropriate autograd mechanism and reject unsupported higher-order use.

Part 3 - Quantitative Prediction Before Execution

Predict ideal persistent bytes as

$$M_3 = (P + G + O)/N.$$

Add the largest gathered unit, recomputation intermediates, live activations, full unit-gradient production, reduction buffers, and allocator capacity on a phase timeline. Explain why one giant computation unit can destroy much of the apparent saving. Count local shards and gathered copies separately.

Part 4 - Experimental Design and Instrumentation

Initialize local shards directly from a canonical small-model state and verify reconstruction outside the timed region. Implement one custom linear unit, check its input and parameter gradients, then compose several units. Compare several Adam updates with the Stage 0 oracle. Keep optimizer updates after all backward readers of the old version have completed. Rebuild each stage from the same logical initialization; changing a stage flag cannot convert optimizer

ownership and storage layout safely. Record the gather, recompute, reduce-scatter, and release boundaries explicitly.

For the custom differentiation boundary, list every tensor input and the matching backward return entry. Non-tensor runtime arguments require the appropriate no-gradient result. The sharded leaf must receive the owned parameter derivative once; the unit input must receive its ordinary local input derivative so earlier units can differentiate correctly. Test the first unit with ordinary inputs that do not require gradients as well as an interior unit whose inputs do. Recompute from the exact recorded parameter version. For the first implementation, reject stochastic recomputation and mutation of saved inputs. These restrictions keep a memory demonstration from silently changing the function whose gradients the optimizer receives.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
import torch.distributed as dist


class ShardedFunctionalUnit(torch.autograd.Function):
    @staticmethod
    def forward(ctx, inputs, local_shard, runtime, unit_id):
        # Gather, execute, save required inputs/version, and release.
        ...

    @staticmethod
    def backward(ctx, output_gradient):
        # Regather old version, recompute, reduce once, return shard grad.
        ...


def E10(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Audit saved tensors to detect unintended full-parameter retention. Compare outputs, input gradients, owned gradient shards, moments, and updated parameters. Reconstruct state only in a separate correctness replay. Reject an implementation that matches forward outputs but corrupts backward. Distinguish the persistent formula from measured peaks, including recomputation cost. Mark arbitrary modules, tied weights, and higher-order differentiation unsupported until separately implemented and validated.

Part 7 - Engineering Interpretation and Philosophy

Explain which invariants permit full weights to disappear between uses. State why the educational recomputation design is one valid restricted approach rather than a description of every production ZeRO implementation.

19.41.12 Exercise 11

Exercise Title: Gather Parameter Shards Immediately Before Layer Execution and Repartition Them After Their Required Computation Completes

Part 1 - Exercise Contract and Why It Matters

Build the gather and release lifecycle for a computation unit. Materialize only the required parameter version, wait before consumption, and release only after every relevant reader completes. Demonstrate both forward and backward lifetimes with the validated custom unit from E10.

Part 2 - System and Memory Mental Model

A Python reference, an autograd dependency, and an outstanding CUDA read are separate reasons storage may remain live. The custom unit avoids saving full parameters across forward and backward, but it still needs a full unit during execution and recomputation. A host-side return from a launch does not prove that GPU readers finished. Gather completion must precede computation; release or reuse must follow the last reader. The authoritative local shard must retain the required version until backward no longer needs it.

Part 3 - Quantitative Prediction Before Execution

For each unit, draw a state sequence from sharded to gathering, ready, in use, and releasable. Assign a byte count and completion condition to every transition. Predict the maximum number of simultaneously live gathered units for a serialized schedule. Then add one prefetched unit and explain which storage intervals overlap. Specify whether a release returns capacity to an application pool or actually drops a PyTorch allocation.

Part 4 - Experimental Design and Instrumentation

Implement the serialized path first. Gather one unit in a rank-consistent order, execute it, and use explicit completion records before reusing its buffer. In backward, regather the recorded old version, recompute, reduce parameter gradients, and complete input-gradient production before release. Validate each boundary with a small known network. Add delayed CUDA work as a lifetime test without depending on a particular race reproducing. Keep all ranks on the same collective sequence. Avoid device-wide synchronization inside the final overlap path, while retaining deliberate boundary synchronization for isolated measurement.

Keep the lease ledger separate from the allocator cache: logical release can make an application buffer reusable while its storage remains allocated. Record both events if the application pool retains capacity. For backward, identify the last input-gradient computation that reads the gathered weight; reducing weight gradients alone does not establish that lifetime has ended. Use generation numbers to distinguish a new lease from a recycled address. Validate no consumer holds an old-generation view when the buffer is reused. On failure, retain a short CPU event trace with unit, rank, generation, and version so the explanation survives without keeping the problematic GPU tensors alive through the evidence system.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
```

```
@dataclass
class GatherLease:
    unit_id: int
    parameter_version: int
    storage: object
    completion: object
    readers: int


def acquire_unit(runtime, unit_id, version):
    ...


def release_unit(runtime, lease, final_reader_event):
    ...


def E11(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export lease generation, unit version, gather completion, first read, last read, release request, and reuse eligibility. Ensure logs contain CPU metadata rather than tensors that retain full storage. Compare against a safe serialized oracle. Treat an early reuse detected by the state machine as a failure even if outputs happen to match in one run. A peak that remains large after release may reflect pool retention or another live alias; inspect both before assigning a cause.

Part 7 - Engineering Interpretation and Philosophy

Describe the release proof in terms of ownership and completed readers. Explain why an address that gets reused is insufficient evidence that its previous contents were no longer needed.

19.41.13 Exercise 12

Exercise Title: Build a Parameter Persistence Policy That Keeps Small or Frequently Reused Parameters Resident Instead of Repeatedly Gathering Them

Part 1 - Exercise Contract and Why It Matters

Add a bounded persistence policy for gathered parameters that are small or repeatedly used. Compare communication avoided with additional residency. Keep local shards authoritative and require every persistent full copy to carry an explicit parameter version.

Part 2 - System and Memory Mental Model

Persistence is a cache of a gathered representation, not a second independent optimizer owner. After an owner update, cached replicas become stale and must be invalidated or refreshed before use. Parameter selection must agree across ranks because bypassing a gather

changes collective order. A per-parameter size threshold alone does not bound cumulative residency. Production DeepSpeed configuration is a reference for policy concepts; its parameter-count thresholds are measured in elements, whereas this exercise's application budget is explicitly in bytes.

Part 3 - Quantitative Prediction Before Execution

Predict saved gather calls and bytes for a static repeated-unit trace. Calculate the sum of retained full copies plus local shards, gradients, optimizer state, and required execution workspace. Construct many individually small parameters whose cumulative persistence exceeds the budget. Give a second case where a frequently used parameter changes version every update, limiting cross-step reuse despite high apparent access frequency.

Part 4 - Experimental Design and Instrumentation

Use a rank-consistent manifest of candidates and a deterministic budgeted selection rule. Start with persistence within one update, where versions are fixed until the optimizer boundary. Then add invalidation across updates and verify that no stale full copy reaches forward or backward. Compare no persistence, a size threshold, and a bounded reuse-aware selection. Replay the same training inputs and collective schedule. Record admitted bytes, rejected candidates, cache hits by version, refresh work, and peak gathered capacity. Keep diagnostic reference copies outside the measured process.

Separate a reused unit's logical access count from the number of distinct parameter versions it accesses. A repeated unit within one update can have valuable same-version reuse, whereas the same unit across many optimizer steps needs refresh. In the restricted primary graph, use a static repeated functional unit only after its accumulated parameter gradient is validated; otherwise use an offline access trace for policy prediction and label it as such. Make admission decisions from a common manifest rather than each rank's local memory query. Local free-memory readings can differ and cannot safely determine whether one rank alone skips a collective. Record the manifest fingerprint with every persistence comparison.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch


@dataclass
class PersistentEntry:
    unit_id: int
    version: int
    bytes: int
    lease: object


def choose_persistent_units(manifest, byte_budget, schedule):
    ...
```

```
def invalidate_after_update(cache, changed_versions):
    ...


def E12(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Check equality with the no-persistence oracle across several optimizer updates, not just within one forward. Include a negative control that reuses an old version and detect its numerical error. Report cumulative retained bytes and observed complete-step peaks. Separate savings in calls from savings in actual transmitted traffic. An unfavorable memory-throughput trade-off is an admissible outcome; do not assume persistence helps every small unit.

Part 7 - Engineering Interpretation and Philosophy

Choose a persistence rule whose budget and invalidation conditions can be audited. Explain how a local cache optimization becomes a distributed protocol decision when it changes which collectives execute.

19.41.14 Exercise 13

Exercise Title: Prefetch Future Parameter Shards and Tune Prefetch Distance to Hide Collective Communication Without Causing VRAM Spikes

Part 1 - Exercise Contract and Why It Matters

Prefetch future gathered units while current computation proceeds. Tune lookahead under a byte budget and prove that readiness, versioning, and collective order remain correct. Compare the pipelined schedule with the serialized E11 oracle.

Part 2 - System and Memory Mental Model

Prefetch overlaps communication with available computation but creates additional live full-parameter storage. A collective's asynchronous work handle, CUDA stream ordering, and the lifetime of its buffers all need explicit handling. Use documented backend completion semantics; a host launch time is not a device-ready timestamp. Keep the rank-consistent schedule static at first. Forward and backward have different use orders, so a forward-only lookahead predictor is insufficient for the whole step.

Part 3 - Quantitative Prediction Before Execution

Estimate a next-unit gather duration and the current-unit compute interval, then predict a lower-bound uncovered wait as the positive part of their difference. State the assumptions: independent resources, no contention, and available buffer capacity. Calculate the maximum live gathered bytes for lookahead zero, one, and two using actual unit sizes. Explain why longer lookahead can increase latency through contention even when it hides an individual gather wait.

Part 4 - Experimental Design and Instrumentation

Start with one outstanding prefetch and a dedicated communication stream where supported. Establish producer-to-consumer and final-reader dependencies, retain input/output

buffers until completion, and reject mismatched versions. Advance collectives in the same order on every rank. Admit a prefetch only when its full capacity fits the measured application budget and required workspace reserve. Sweep a short list of lookahead settings using fresh identical replays. Capture profiling traces separately from timing and include both forward and backward. Record stalled admission, ready-before-use time, exposed waits, and actual peak residency.

Maintain a monotonically increasing collective sequence number and attach unit and version metadata to each issued request. Allow local completion times to differ without allowing issue order to diverge. Compare the planned lookahead with actually admitted lookahead because the byte budget may reject larger units. Plotting only the requested distance would conceal that behavior. A prefetched unit that becomes unusable must still finish its outstanding work before its buffers are reclaimed. In the static primary schedule, unneeded prefetch should be rare; test cancellation bookkeeping with a controlled offline schedule first. Keep communication stream dependencies explicit when handing a gathered tensor to the default compute stream.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
import torch.distributed as dist


class ParameterPrefetcher:
    def admit(self, unit_id, version, required_bytes):
        ...

    def launch_next(self, canonical_schedule):
        ...

    def acquire_ready(self, unit_id, consumer_stream):
        ...

    def retire(self, lease, final_reader_event):
        ...


def E13(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require numerical agreement with the serialized runtime and no version or lease violations. Report gather wait, total step time, outstanding work, live gathered bytes, and rejected admissions per rank. Identify overlap from an execution timeline rather than from non_blocking or async_op flags alone. If hardware cannot overlap the selected collectives and kernels effectively, report the measured limitation. Do not reuse a buffer merely because the Python work object was dropped.

Part 7 - Engineering Interpretation and Philosophy

Choose lookahead by the joint latency and residency evidence. Explain which part of the prediction fails when communication contention or a slow rank changes the available overlap window.

19.41.15 Exercise 14

Exercise Title: Reproduce a Stage 3 Memory Spike Caused by Excessive Live Gathered Parameters and Control It with a Maximum-Live-Parameter Budget

Part 1 - Exercise Contract and Why It Matters

Create a controlled Stage 3 trace in which excessive gathered residency raises the peak. Add an admission policy for maximum live parameter capacity and show whether it controls that peak while preserving training correctness. A reproduced spike is sufficient; a physical CUDA OOM is not required.

Part 2 - System and Memory Mental Model

Multiple gathered units can remain live because of prefetch, persistence, active readers, or accidental aliases. A live-capacity policy must include all of those categories, pending gathers, and allocation rounding within its scope. It does not reserve the entire GPU against activations, backend workspaces, other processes, or allocator growth. DeepSpeed's max_live_parameters concept uses parameter elements; the educational controller must label its byte units and convert from dtype explicitly instead of copying a number blindly.

Part 3 - Quantitative Prediction Before Execution

Construct a static sequence of unequal units. Compute the live gathered set at every event for an aggressive schedule and for a bounded schedule. Predict the earliest rejected admission and the additional wait it introduces. Include the unavoidable largest-unit requirement: a budget below the required unit plus working reserve needs a clear unsupported or infeasible result, not a retry loop that can never make progress.

Part 4 - Experimental Design and Instrumentation

Use a bounded workload that fits the declared test environment. Increase lookahead or persistence gradually and record the observed peak without assuming an OOM. Repeat with the byte controller enabled and the same logical training work. Count unique gathered storage once while separately counting local shards and other categories. Release completed readers before admitting queued gathers. Make the rank-consistent decision before collective launch. If an allocation failure occurs, stop and recreate the whole worker group from a committed checkpoint; do not let one rank continue alone.

Use a ledger invariant that reserved pending capacity plus admitted ready capacity never exceeds the application gather budget. Do not charge the same lease twice when it changes from pending to ready. Conversely, do not stop charging it at a Python release request before readers complete. A bounded schedule can deadlock if it waits for space held by an object whose consumer cannot execute; validate progress on the static event graph. Provide a clear infeasible result for the oversized-unit case. Compare observed budget compliance with total CUDA allocation separately so a correct ledger is not misrepresented as control over activations, external allocations, or another process using the device.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


class LiveParameterBudget:
    def request(self, unit_id, version, capacity_bytes):
        ...

    def record_pending(self, lease):
        ...

    def complete_release(self, lease, completion):
        ...


def E14(config):
    # Compare predicted live sets with bounded and aggressive replays.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report predicted and observed live capacity, total allocated and reserved peaks, admission stalls, and correctness status. Use phase evidence to attribute a spike to gathered parameters; an allocated-minus-reserved subtraction cannot prove that cause. Include accidental retention as a separate diagnostic case. If the spike does not reproduce, retain that outcome and state the tested range. Verify progress for a unit exactly at the admissible capacity boundary.

Part 7 - Engineering Interpretation and Philosophy

Explain the scope of the controller's guarantee and the remaining GPU headroom uncertainty. Choose a conservative reserve based on measured working requirements rather than claiming a byte budget makes real allocation infallible.

19.41.16 Exercise 15

Exercise Title: Compare ZeRO Stages 0, 1, 2, and 3 Under the Same Model and Report Per-GPU Memory, Communication, Throughput, and Maximum Trainable Model Size

Part 1 - Exercise Contract and Why It Matters

Compare stages zero through three using one common model and training contract. Report memory, communication, throughput, and an independently measured capacity sweep. Keep the equal-model experiment separate from the largest-model experiment so their conclusions remain interpretable.

Part 2 - System and Memory Mental Model

Stage labels identify progressively partitioned state, but the educational implementations also differ in collective protocol and Stage 3 recomputation. Those costs must remain visible. The largest trainable model depends on activations, unit granularity, batch size, optimizer

temporaries, and available headroom, not only persistent state. One machine with multiple CUDA devices is sufficient for the distributed comparison; do not require two computers. Only claim world sizes actually tested. Single-rank runs are useful controls but cannot demonstrate removal of inter-rank replication.

Part 3 - Quantitative Prediction Before Execution

Use the four ideal persistent-state expressions from E00 and the actual padding map to predict per-rank bytes. Add measured or separately estimated peak working components without treating all category maxima as simultaneous. Specify a model-width or depth sweep and a fixed global batch. Predict the largest feasible candidate as a hypothesis, then define how successful full updates will confirm only a tested lower bound on capacity.

Part 4 - Experimental Design and Instrumentation

Launch each stage in a fresh worker group with the same logical initial parameters, data order, precision, loss normalization, and Adam settings. Validate several updates against the reference at a small shared size. Measure the fixed model first. For capacity, increase one documented dimension with a bounded search, requiring complete forward, backward, optimizer update, and parameter synchronization or publication. Record initialization failure separately from training failure. Keep activation checkpointing and offload disabled unless they are a separately named experimental factor. Include warm state allocation and several steady updates before declaring a candidate usable.

For the capacity sweep, preserve identical optimizer and precision choices across stages even when one stage could support a more favorable setting. Any later mixed-precision optimization belongs to a separately labeled sweep. Require the same input shape family and training objective at each model-size candidate. A candidate that initializes successfully but cannot complete the first initialized-Adam update is not trainable under the declared contract. Record whether validation and checkpoint export are included in capacity acceptance or measured separately. Avoid gathering a full model onto every rank just to print a checksum during a capacity run. Use the small-model correctness suite and compact integrity metadata instead.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def build_stage_replay(stage, logical_initial_state, config):
    ...


def compare_fixed_model(replays, evidence):
    ...


def search_tested_capacity(stage, bounded_candidates, config):
    ...
```

```
def E15(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export one row per rank and stage with persistent categories, complete-step peak, reserved peak, communication ledger, step latency, throughput, and correctness status. Report the maximum rank as well as the distribution. For capacity, preserve successful and failed candidate IDs and failure phases. Do not import theoretical production communication constants into the naive broadcast implementation. Attribute Stage 3 recomputation time explicitly. If a test ends at the largest permitted candidate without failure, report a lower bound rather than an exact maximum.

Part 7 - Engineering Interpretation and Philosophy

Choose a stage for a stated resource budget and training objective. Explain why the most partitioned stage can be essential for capacity yet unnecessary for a smaller model whose communication dominates.

19.41.17 Exercise 16

Exercise Title: Add CPU Optimizer-State Offloading to the ZeRO Runtime and Measure the Trade-Off Between VRAM Savings, RAM Usage, and PCIe Traffic

Part 1 - Exercise Contract and Why It Matters

Move owner-local optimizer state to CPU while preserving the distributed update contract. Measure GPU savings, host memory, staging capacity, PCIe traffic, and CPU update time. Implement the educational offload path explicitly rather than assume the performance of a specialized production CPU optimizer.

Part 2 - System and Memory Mental Model

The owning rank remains responsible for its update even when moments live in host memory. Transfer its reduced gradient shard to a valid CPU staging area, perform the same Adam recurrence, and publish the resulting parameter version back to CUDA. Stage 1 and Stage 2 then synchronize replicated parameter spans; Stage 3 publishes updated owner shards before later gathers. An FP32 CPU parameter authority, if retained for update, is additional state that must be counted. It is not free simply because the moments were already offloaded. Pinned staging enables particular transfer paths but consumes a separate bounded host resource.

Part 3 - Quantitative Prediction Before Execution

Predict device bytes removed for owner moments and device bytes added for staging. List host moments, optional parameter authority, gradient staging, update staging, and pending copies separately. For one optimizer update, estimate transfer payload from the actual chosen protocol, not the whole model by default. Establish an optimistic bandwidth lower bound and identify CPU compute, launch overhead, and dependency waits it omits.

Part 4 - Experimental Design and Instrumentation

Start with serialized D2H, CPU update, H2D, and parameter synchronization. Compare moments and updated parameters with the GPU-only reference over several steps. Then add bounded pinned buffers and asynchronous transfers with explicit completion before CPU reads

and before GPU consumes updates. Do not overwrite CPU or CUDA staging while a copy still uses it. Keep accumulation semantics unchanged and update once at the declared optimizer boundary. Measure process host memory, pinned capacity when observable, CPU update time, exposed transfer waits, and distributed step time in separate instrumented replays.

Account for CPU update arithmetic and host thread configuration in the reproducibility record. A changed thread count can alter offload latency without changing transfer volume. State whether host RSS includes pinned staging and whether system metrics count shared pages more than once. If pinned-memory usage cannot be measured directly, report allocated pinned capacity under application control and preserve the broader quantity as unknown. Compare a serialized correct path with the overlapped path before claiming overlap benefits. Include an update whose shard size exceeds one staging chunk to prove chunk boundaries preserve logical Adam counters and publication waits. Never publish a mixture of old and new owner chunks.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


class OwnerOptimizerOffload:
    def allocate_host_state(self, ownership, config):
        ...

    def receive_gradient(self, owned_gradient, completion):
        ...

    def update_on_cpu(self, logical_step):
        ...

    def publish_cuda_version(self, runtime, completion):
        ...


def E16(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Audit the latest authoritative version across CPU and CUDA at each transition. Require matching moments, counters, parameters, and losses within declared numerical tolerances. Report actual bytes copied and distinguish those from measured bus traffic if unavailable. A low GPU peak with excessive RAM or a much slower update is a trade-off, not a failed measurement. Do not reuse CPU buffers after a nonblocking D2H launch until completion is established. Include first-update host-state allocation outside warmed timing but report its cold-start cost separately.

Part 7 - Engineering Interpretation and Philosophy

Explain whether the workload is limited by GPU capacity, host update compute, or transfer dependencies after offload. State which resource the technique releases and which resources it consumes.

19.41.18 Exercise 17

Exercise Title: Extend Stage 3 with Parameter Offloading to CPU or NVMe and Build a Three-Tier Residency Plan for Partitioned Model State

Part 1 - Exercise Contract and Why It Matters

Extend owner-shard residency across CUDA, CPU, and optional NVMe files. Build a bounded three-tier plan with explicit authoritative versions and materialization prerequisites. Treat disk-backed residency as an educational extension, not a claim to reproduce ZeRO-Infinity's optimized implementation.

Part 2 - System and Memory Mental Model

A parameter shard must be available on CUDA before its collective gather can consume it. NVMe file reading, CPU materialization, H2D completion, collective completion, and computation readiness are distinct events. Ordinary file access is not direct GPU storage access. CPU page cache, mapped pages, pinned staging, and application buffers contribute different host-memory views. After an optimizer update, a stale on-disk shard cannot replace the new version. Dirty writeback must complete before discarding the only current copy. State checkpoints must include optimizer state and logical progress, not just a collection of parameter files.

Part 3 - Quantitative Prediction Before Execution

For each owner shard, record its valid bytes, tier, version, file extent, and required future use. Predict CUDA and host staging peaks for one incoming and one outgoing shard. Add storage service time and transfer time to a serialized critical-path estimate. Then specify which intervals could overlap under a bounded pipeline and which dependencies remain unavoidable.

Part 4 - Experimental Design and Instrumentation

Implement CPU residency first, then add optional rank-local file extents with dtype, shape, logical offsets, ownership, version, and integrity metadata. Write to a temporary generation and publish a committed manifest only after all required rank state is durable under the declared filesystem assumptions. Validate file bounds and metadata before allocating or reading. Stage owner shards through bounded host buffers and issue gathers in the canonical order. Limit outstanding disk and CUDA work. Test a small round trip, a modified shard, interrupted publication in an isolated directory, and restart from the last committed update. Never overwrite an unrelated user file.

Use separate file namespaces for rank, checkpoint generation, and logical shard identity. Validate that two ranks cannot unintentionally write the same extent. Record whether file storage is local NVMe, another block device, or an unavailable optional tier; device names alone do not establish measured bandwidth. A successful operating-system write call is not automatically the durability guarantee required by a crash-recovery claim. Define the chosen commit assumptions and test only those assumptions. During restoration, verify parameter and moment generations together before any forward executes. Keep duplicate generations visible in disk-space accounting even when they are no longer resident in the explicit application buffers.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch


@dataclass
class ShardLocation:
    parameter_id: int
    owner_rank: int
    version: int
    tier: str
    byte_extent: tuple
    integrity: object


def materialize_owner_shard(record, staging, runtime):
    ...


def commit_partitioned_state(engine, checkpoint_directory):
    ...


def E17(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report per-tier resident capacity, queued bytes, file bytes read and written, exposed disk waits, transfer waits, and end-to-end update time. Distinguish cold and warm filesystem-cache behavior without claiming a particular cache state that was not controlled. Verify resumed updates against an uninterrupted run and reject mixed checkpoint generations. Mark NVMe unsupported if unavailable rather than substituting fabricated disk results. Do not claim total process RAM is bounded solely because explicit staging is.

Part 7 - Engineering Interpretation and Philosophy

Explain how tiering changes the capacity bottleneck into a scheduling problem. Identify the minimum metadata and commit protocol required to avoid restoring numerically valid but outdated parameter shards.

19.41.19 Exercise 18

Exercise Title: Stress-Test ZeRO-Style Partitioning with Uneven Parameter Sizes, Gradient Accumulation, Communication Contention, CPU Offload, and Severe Per-GPU VRAM Limits

Part 1 - Exercise Contract and Why It Matters

Stress the partitioned runtime with uneven layouts, accumulation, communication contention, offload, and constrained per-rank capacity. Require correct updates and bounded, explicit failure behavior. Produce small counterexamples that isolate violated assumptions before attempting large runs.

Part 2 - System and Memory Mental Model

Partition boundaries may split a logical parameter or leave an owner with no valid elements. Padding must never enter the optimizer recurrence. Gradient accumulation changes when reduction and update occur and can retain additional buffers. A zero gradient differs from an absent gradient for Adam state and step semantics. Dynamic or unused parameters are unsupported in the primary fixed graph; a stress extension must implement a rank-consistent presence protocol or reject them clearly. Rank-local branches must not silently change collective order.

Part 3 - Quantitative Prediction Before Execution

Construct cases with prime-sized tensors, a parameter larger than one shard, and more ranks than valid elements where hardware permits. Predict valid and padded ownership and logical step counters. For equal microbatches, derive the loss weighting that produces the same global mean as the reference. Compare reducing each microbatch with deferring reduction, including their different live gradient requirements. Predict a contention case that increases wait time without changing logical communication payload.

Part 4 - Experimental Design and Instrumentation

Run a deterministic small-model matrix before capacity stress. Cover one and multiple accumulation steps, uneven parameter sizes, a final partial bucket, no-valid owner behavior, CPU staging pressure, and delayed communication. Use a bounded application memory limit first; severe physical allocation pressure is optional. Keep all test seeds and schedule traces. Inject logical errors into an offline validator instead of launching mismatched collectives. If a worker fails, terminate and recreate the entire group from a committed checkpoint. Record expected rejection separately from incorrect training or an unexpected backend failure.

Construct a zero-versus-absent-gradient diagnostic in a separate extension with a tiny conditional graph. If the engine has no validated presence protocol, require rejection before launching a divergent sequence. If it does, compare Adam counters and moments against the reference rather than accepting only matching parameter values after one step. For accumulation, use an uneven final microbatch only with correctly derived sample weighting. Otherwise reject that configuration or restrict the test to equal sizes. Keep resource-pressure changes independent from semantic changes so a failed update has a useful explanation. Minimize failures first by removing offload, then prefetch, then partition complexity while preserving the mismatch.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def make_partition_stress_cases(config):
    ...
```

```
def validate_accumulated_update(engine, reference, case):
    ...


def recover_worker_group(committed_checkpoint, launcher):
    ...


def E18(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

For each case, export layout, seed, global and local batch sizes, accumulation count, reduction timing, resource budgets, peak memory, latency, outcome, and failure phase. Compare parameters, moments, and counters after several updates. Include the smallest mismatch reproducer and distinguish normal tolerance drift from a missing or double update. If an optional shared parameter or conditional path is unsupported, preserve a clear rejection instead of treating it as a passing stress test. Absence of an OOM or race in the tested range is not proof of universal safety.

Part 7 - Engineering Interpretation and Philosophy

Choose the most informative counterexample and explain the invariant it breaks. Describe how deterministic rejection and checkpointed group recovery make a restricted runtime more trustworthy than silent partial progress.

19.41.20 Exercise 19

Exercise Title: Build and Benchmark a PyTorch ZeRO-Style Distributed Training Engine That Progressively Partitions Optimizer States, Gradients, and Parameters While Coordinating Gather, Reduce-Scatter, Prefetch, and Offload

Part 1 - Exercise Contract and Why It Matters

Integrate the validated components into a PyTorch educational distributed engine exposing stages zero through three, optional owner-state offload, bounded gathers, and explicit prefetch. Deliver an auditable benchmark and a capability matrix. Keep the implementation visible and intentionally complete it as learner work rather than hiding it in a high-level trainer.

Part 2 - System and Memory Mental Model

The engine combines a logical parameter registry, owner map, gradient protocol, optimizer recurrence, versioned gather leases, and tiered residency. All components must agree on the same update boundary. Stage selection is a construction-time choice in the primary implementation. Converting an active engine to a different stage requires an explicit state migration protocol; a flag assignment is insufficient. The restricted functional graph and first-order differentiation contract remain in force. Production DeepSpeed supports substantially more cases and optimizations than this exercise.

Part 3 - Quantitative Prediction Before Execution

Before coding the integration, predict persistent bytes for every stage, actual owner imbalance, live gather capacity, reduction-buffer capacity, offload staging, and complete-step critical path. State assumptions beside each prediction. Define acceptance criteria for update correctness, memory budget compliance within the application's scope, progress, and checkpoint restart. Choose a primary two-rank run only if two CUDA devices are available; otherwise label the distributed benchmark not measured.

Part 4 - Experimental Design and Instrumentation

Build the engine from one canonical initialization and deterministic schedule. Validate the smallest supported model against ordinary Adam and Stage 0 before benchmarking. Run equal-model stage comparisons, then bounded capacity sweeps and the stress suite. Keep correctness reconstruction, profiling, and ordinary timing in separate replays. Preserve a committed partitioned checkpoint and compare resumed training with uninterrupted training. Record hardware, topology, versions, allocator backend, precision, optimizer options, model dimensions, seed, batch semantics, stage, bucket capacity, prefetch distance, persistence budget, and tier budgets. Include forward, backward, update, and required synchronization in training timing.

Publish a capability matrix in the exercise evidence with separate entries for replicated baseline, owner-only moments, sharded reduced gradients, bounded gradient production, autograd-safe gathered units, persistence, prefetch, CPU optimizer offload, parameter tiering, and restart. Each entry needs supported, rejected, or not-tested status and a validating run ID. Do not treat a scaffold signature as implemented support. Include the smallest counterexample that invalidates any proposed universal speed or memory claim, or state that none was observed in the tested range. Keep external API versions in the record because backend behavior and production configuration interfaces can change independently of the educational engine's explicitly stated mathematical contract.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
import torch.distributed as dist


class EducationalZeROEngine:
    def construct(self, logical_model, stage, config):
        ...

    def forward_backward(self, batch, accumulation_boundary):
        ...

    def update_and_publish(self):
        ...

    def checkpoint_committed_state(self, destination):
        ...

    def export_cpu_evidence(self):
        ...
```

```
def E19(config):
    # Assemble validated components and run independent benchmark replays.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export prediction, observation, and interpretation in separate columns. Every run needs an ID, configuration, input shape, world size, resource budgets, per-rank memory peaks, elapsed time, throughput, numerical errors, outcome, and failure phase. Use explicit unsupported and not-measured values. Do not retain live autograd graphs in evidence logs. Reject conclusions based only on forward equality, uninitialized optimizer state, or a theoretical memory formula. Report any capacity or throughput regression with its trace evidence. A failure to reproduce a predicted spike remains an experimental outcome.

Part 7 - Engineering Interpretation and Philosophy

Select an operating point for a concrete budget and explain the remaining uncertainty. The strongest result is a runtime whose update semantics, ownership, temporary residency, and unsupported cases can all be checked, with measured trade-offs rather than promised universal speedups.

19.42 DMM40

Chapter Title: Tensor Parallel Memory Placement
Specialization: Dynamic Memory Management and CUDA Systems

19.42.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Tensor Parallelism, Sharded Matrix Operations, Local Tensor Storage, and Cross-GPU Communication

Part 1 - Exercise Contract and Why It Matters

Define tensor parallelism as partitioned computation within a logical layer. Deliver a layout ledger relating global tensors, local storage, mathematical operations, and collective dependencies. Distinguish this design from data parallel replicas and from temporary full-parameter materialization in FSDP or ZeRO Stage 3.

Part 2 - System and Memory Mental Model

Use the convention $X \in \mathbb{R}^{M \times K}$ and PyTorch Linear weight $W \in \mathbb{R}^{N \times K}$, so $Y = XW^T + b$. Column-parallel means partitioning output features: stored weight dimension zero. Row-parallel means partitioning input features: stored weight dimension one. These names refer to the conventional untransposed matrix formulation; never infer the stored axis from the word column alone.

All ranks in one TP group cooperate on the same logical batch. They are not independent data replicas. Start with FP32, deterministic dense operations, equal feature partitions, and no dropout. Training requires correct distributed derivatives and one logical loss, not merely matching forward outputs.

Part 3 - Quantitative Prediction Before Execution

Let TP degree be T and element width be s bytes. Derive local weight bytes for each partition and list which inputs or outputs remain replicated. Draw a two-layer path from replicated input through sharded hidden features to a reduced output. Predict where communication is required in forward and backward. Add gradients, optimizer states, temporary buffers, and activations before proposing a complete training-memory estimate.

Part 4 - Experimental Design and Instrumentation

Freeze a global parameter registry, axis convention, group membership, input layout, output layout, and loss semantics. Establish small serial forward, gradient, and update oracles. Bind one process to each CUDA device and validate rank-consistent configuration before collective work. Record PyTorch and CUDA versions, backend, topology, seed, dtype, model dimensions, batch and sequence shapes, allocator backend, and initial-state identity. Treat API availability as a checked capability: reference documentation does not establish what the installed environment supports.

Build a paper trace for two ranks with a four-feature input and six-feature output. Label every local tensor as complete replica, disjoint shard, or unfinished contribution. Carry those labels through matrix multiplication, bias addition, activation, and the next layer. Add backward arrows with the required derivative ownership. This trace must explain why TP participants consume the same logical samples, whereas data replicas consume different samples. Define a single logical loss at the outer boundary and document how its replicated representation is differentiated. Avoid adding a data-parallel average to tensor-parallel sums. Extend the trace to one optimizer update so parameter shards, replicated small parameters, and Adam moments all have explicit owners. Use that completed trace as the contract for later APIs, rather than inferring semantics from their class names.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
import torch.distributed as dist


@dataclass
class TensorParallelContract:
    global_weight_shape: tuple
    stored_shard_axis: int
    input_layout: str
    output_layout: str
    tensor_parallel_degree: int
    logical_loss_identity: str


def predict_layout_storage(contract, phase_plan):
    ...
```

```
def E00(config):
    # Validate layout, derivative, and measurement contracts first.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Keep prediction, observation, and interpretation separate. Require exact coverage of logical features and numerical agreement within declared floating-point tolerances. Missing devices, unsupported operators, and unrun training remain explicit statuses. Do not infer physical memory from a global DTensor shape or claim reduced communication from an attractive diagram alone.

Part 7 - Engineering Interpretation and Philosophy

Explain how a tensor layout constrains both local work and required communication. The objective is a checkable distributed computation whose memory benefits and synchronization costs follow from explicit ownership.

19.42.2 Exercise 01

Exercise Title: Split a Large PyTorch Weight Matrix Across Two GPUs and Verify That Each Rank Stores Only Its Local Tensor Shard

Part 1 - Exercise Contract and Why It Matters

Place disjoint shards of one logical weight on two CUDA ranks and verify actual local storage ownership. Deliver reconstruction evidence on a small case and a capacity audit on a larger bounded case without retaining an unnecessary full CUDA weight on every rank.

Part 2 - System and Memory Mental Model

A slice can be a view retaining its entire parent storage. Copying a slice to an independent local allocation and dropping the parent has different storage behavior. The full logical matrix may exist on CPU for a small oracle, but startup CUDA peaks still count if a construction path first creates the whole weight on each GPU. Parameter initialization must produce the same logical values as the reference; using independent random draws with the same seed does not automatically reproduce global slices.

Part 3 - Quantitative Prediction Before Execution

For divisible output width, predict NKs/T useful local weight bytes and an N/T by K local shape. Compare this with the backing size of a view into a full allocation. Calculate offsets and valid extents for a small matrix whose entries encode row and column indices. Separate useful bytes, padding, retained parent capacity, and construction-time temporary bytes.

Part 4 - Experimental Design and Instrumentation

Use a tiny CPU reference to validate shard boundaries and reconstruction. Then construct local CUDA storage from bounded CPU slices or a validated coordinate-based initializer. Preserve the global initialization contract, including fan-in, rather than applying a locally changed initialization rule silently. Check device binding, dtype, contiguity, storage extent, and absence of parent references. Collect only CPU metadata for the large case. Reconstruct full weights in a separate small replay so the audit does not create the replication it is supposed to measure.

Test both stored axes even though the primary construction uses output features. A row slice and a column slice can have different stride and contiguity behavior, and a contiguous conversion can allocate another buffer. Record the conversion before dropping its source. For initialization, compare against a tiny global reference by coordinate, including bias and global fan-in. A local row-parallel layer initialized with its smaller local fan-in can use a different distribution from the original layer, invalidating a controlled comparison before any communication occurs. Keep global reference construction on CPU bounded and use shard-wise loading for larger cases. Count file-loading or host staging separately if introduced. The large-case claim must concern actual independent local storage, not only an attractive printed shape or the expected arithmetic byte count.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch


@dataclass
class WeightShardRecord:
    rank: int
    global_shape: tuple
    local_shape: tuple
    global_interval: tuple
    useful_bytes: int
    backing_bytes: int


def construct_independent_local_weight(spec, rank, device):
    ...


def audit_local_weight_storage(weight, spec):
    ...


def E01(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report useful and backing bytes, global coverage, local shape, initialization peak, and reconstruction error. Include a negative control retaining a full parent allocation behind a small view. Pointer values from different processes cannot establish shared storage or disjoint ownership by themselves; combine logical intervals with rank-local backing inventories. Require no gaps or overlap in valid logical weight coverage.

Part 7 - Engineering Interpretation and Philosophy

Explain why a small local tensor shape is insufficient proof of a small allocation. Choose a construction route whose startup memory is consistent with the intended steady-state placement.

19.42.3 Exercise 02

Exercise Title: Implement Column-Wise Parallel Linear Algebra from Scratch and Reconstruct the Full Output Across Multiple GPU Ranks

Part 1 - Exercise Contract and Why It Matters

Implement output-feature partitioned linear algebra explicitly and reconstruct the full output for validation. Preserve the same logical input on all TP ranks. Deliver forward equivalence plus a separate derivative check before using the scratch implementation in training.

Part 2 - System and Memory Mental Model

Partition stored weight rows and the matching bias entries. Each rank computes distinct output features from the full input. Concatenating outputs in feature order reconstructs the result; summing them is incorrect. An all-gather used only for diagnostics must be kept outside production timing. Raw communication calls are not automatically a complete autograd-aware TP operator. Define the derivative contract explicitly or use validated DTensor training later in the chapter.

Part 3 - Quantitative Prediction Before Execution

For partition r, derive

$$Y_r = XW_r^T + b_r, \qquad Y = \operatorname{concat}_r Y_r.$$

Given a global upstream derivative D, derive local weight and bias gradients from its matching feature slice. Input-gradient contributions must sum across feature owners. This is a sum over one logical layer, not an average over independent data replicas. Predict local output bytes and the extra replicated capacity required by a diagnostic gather.

Part 4 - Experimental Design and Instrumentation

Start with rank-distinct known weights, common input values, and divisible widths. Compare reconstructed outputs with ordinary Linear. Supply one known global upstream derivative and validate local weight gradients and the summed input gradient analytically. If building a custom autograd boundary, check its return arity and replicated-input gradient semantics. Do not attach a naive gather to a local loss and assume training equivalence. Run the forward benchmark only after the small correctness replay, then release reconstructed outputs before memory sampling.

Use an upstream derivative with different values in every output feature so incorrect concatenation and feature-to-gradient matching cannot cancel. Compute each rank's input-gradient contribution and show that omitting one changes at least one input element. Check a noncontiguous input separately if the scratch implementation accepts it; otherwise reject it clearly. The diagnostic gather may need packed contiguous payloads and temporary receive storage, which belong to its own accounting region. If later adding a differentiable gather, define whether outputs represent one replicated logical value or independently consumed rank-local values before choosing its adjoint. Validate the exact intended contract with the serial oracle. Do not turn a

working inference gather into a training primitive solely by leaving requires_grad enabled on its input.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
import torch.distributed as dist


def column_parallel_forward(inputs, local_weight, local_bias):
    ...


def reconstruct_output_for_validation(local_output, feature_map, group):
    ...


def verify_column_derivatives(reference, shards, upstream_gradient):
    ...


def E02(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report local and global output shapes, reconstruction order, maximum error, local parameter bytes, and diagnostic gather capacity. Detect reversed rank concatenation and an omitted input-gradient sum with small counterexamples. Use nonzero bias to ensure its partition is tested. A forward-only result must remain labeled forward-only until derivatives and several updates pass. Do not divide TP feature contributions by world size.

Part 7 - Engineering Interpretation and Philosophy

Explain why output sharding permits independent local matrix products but leaves a backward dependency for shared inputs. State when reconstructing the full output is needed for a consumer and when it is merely a debugging cost.

19.42.4 Exercise 03

Exercise Title: Implement Row-Wise Parallel Linear Algebra from Scratch and Use Collective Reduction to Produce the Correct Final Output

Part 1 - Exercise Contract and Why It Matters

Implement input-feature partitioned linear algebra and combine partial outputs with a collective sum. Verify the final result, bias placement, and derivative semantics. Deliver a valid row-parallel counterpart to E02.

Part 2 - System and Memory Mental Model

Partition stored weight columns and matching input features. Every rank produces a contribution to every output feature, so equal local output shapes do not imply replicated complete values. Bias belongs after the contribution sum, or requires an explicitly equivalent distributed treatment. Adding the full bias on every rank before SUM multiplies it by TP degree. A Partial layout denotes these unfinished contributions rather than ordinary replicas.

Part 3 - Quantitative Prediction Before Execution

Derive

$$Z_r = X_r W_r^T, \qquad Y = \sum_r Z_r + b.$$

Compute a small example by hand, including nonzero bias. Predict local input and weight bytes alongside the full-sized partial output. Derive weight and input-slice gradients for one global upstream derivative. Explain why repeated evaluation of the same replicated loss must not introduce another factor of T through an inappropriate backward reduction.

Part 4 - Experimental Design and Instrumentation

Use matching feature intervals for the input and weight. Implement a forward SUM, then add bias once in logical semantics. Validate the scratch backward analytically with the same upstream derivative on the complete output. A custom replicated-output boundary must preserve the single-logical-output interpretation; the generic adjoint of independent rank-local losses is a different problem. Compare several updates only after that contract is tested. Use the DTensor path for integrated training if a scratch differentiable collective has not been validated. Record all temporary reduction buffers.

Choose a hand example in which both ranks make nonzero contributions to every output and the bias is also nonzero. Verify the wrong result produced by adding bias before reduction and derive its dependence on TP degree. For derivative validation, use the same complete upstream derivative on the logical output and inspect each weight/input interval independently. A replicated bias needs the reference update once logically; it must not accumulate duplicate loss copies through an unexamined generic collective backward. Keep the scratch analytical check separate from the high-level DTensor training path. Report whether input partitioning begins with local slices of an already replicated input or with genuinely sharded upstream storage, because those construction choices have different memory lifetimes despite identical local matrix products.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
import torch.distributed as dist


def row_parallel_partial(local_input, local_weight):
    ...


def combine_row_partials(partial_output, bias, group):
```

```
    # SUM contributions and implement bias exactly once logically.
    ...


def verify_row_derivatives(reference, shards, upstream_gradient):
    ...


def E03(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require complete-output agreement, correct bias gradients, local weight gradients, and reconstructed input gradients. Include the deliberately incorrect pre-reduction full-bias case and expose its error. A local partial output that coincidentally equals the answer for zero-valued other shards is not a sufficient test. Preserve forward-only status when training derivatives are unfinished. Report logical payload separately from backend link traffic.

Part 7 - Engineering Interpretation and Philosophy

Explain why row sharding trades smaller local inputs for full-sized partial outputs and reduction work. Relate the bias rule to the difference between ownership of a contribution and ownership of a complete value.

19.42.5 Exercise 04

Exercise Title: Compare Column-Wise and Row-Wise Tensor Parallelism by Per-GPU Parameter Memory, Activation Memory, Communication Volume, and Latency

Part 1 - Exercise Contract and Why It Matters

Compare column and row layouts for the same logical Linear operation. Measure parameter storage, activation boundaries, communication, and completed latency. Include the cost of satisfying a common external input/output contract so the comparison does not favor a layout by omitting conversion.

Part 2 - System and Memory Mental Model

Column partitioning naturally consumes replicated inputs and returns feature shards. Row partitioning naturally consumes feature shards and reduces to complete outputs. Their useful local weight counts can be equal while activation residency and collective work differ. A caller that already owns sharded inputs has a different starting point from one with full replicas. Backward and optimizer costs must be measured separately from a forward-only algebra comparison.

Part 3 - Quantitative Prediction Before Execution

For fixed M, K, N, T, s, calculate weight payload, natural input capacity, natural output capacity, and conversion buffers for each layout. Use one comparison with natural boundaries and another with replicated input and replicated output on both paths. Predict which gather, slice, or reduction is needed. Estimate collective payload and state an explicit algorithmic assumption before translating it into estimated transmitted bytes.

Part 4 - Experimental Design and Instrumentation

Run isolated replays from the same logical values and input data. Validate outputs first. Then benchmark natural boundaries and common boundaries as separate experiments. Measure warmed execution with boundary synchronization, CUDA timing where appropriate, and end-to-end host timing for completed work. Reset peaks for the named region and report cold allocation separately. If testing training, include forward, backward, optimizer update, and necessary layout communication. Profile in a separate replay so trace overhead does not become the reported baseline latency.

Include a workload with narrow output and wide input, then reverse the aspect ratio while keeping the accounting rules fixed. This exposes which activation boundary dominates rather than assuming square matrices represent all layers. Count local slices that retain full input backing by their backing capacity in the common-boundary comparison. Record whether reduction is in-place, whether the unreduced partial is retained, and whether a full output gather overlaps the source shard. Those details can change the peak without changing the final answer. Use repeated process launches to estimate variability and preserve the slowest rank's evidence. Compare reported logical payload with supported trace events, but leave physical traffic unknown when the instrument only knows tensor sizes. Explain any performance claim within its measured shape and topology range.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch


@dataclass
class LinearBoundaryContract:
    input_layout: str
    output_layout: str
    includes_backward: bool
    includes_optimizer: bool


def compare_linear_layouts(spec, boundary_contract, measurements):
    ...


def E04(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export each boundary contract, per-rank useful/backing bytes, allocated and reserved peaks, communication calls and payload, elapsed time, and numerical error. Do not count a diagnostic output gather only on one side. Report cases where the lower-memory layout is slower. Identify whether a result comes from matrix shape, conversion, reduction, or another traced dependency before attributing it to a universal advantage of column or row sharding.

Part 7 - Engineering Interpretation and Philosophy

Choose a layout based on its neighbors and the limiting resource. A layer's isolated local parameter size cannot determine the best distributed layout for an entire model.

19.42.6 Exercise 05

Exercise Title: Build a DeviceMesh with PyTorch and Map Logical Tensor-Parallel Dimensions onto Physical CUDA Devices

Part 1 - Exercise Contract and Why It Matters

Create a DeviceMesh that maps a logical TP dimension to actual CUDA ranks. Verify membership, coordinate order, process groups, and device binding before running tensor operations. Deliver an explicit mapping rather than assuming rank adjacency implies a particular physical interconnect.

Part 2 - System and Memory Mental Model

A mesh describes logical rank organization and communication groups. It does not manufacture faster links or move an already allocated tensor merely by assigning a dimension name. Global rank, local rank, CUDA ordinal, and mesh coordinate are related but distinct identifiers. Every participating process must construct a consistent mesh definition. Conflicting definitions can lead to incorrect groups or hangs, so validate a shared configuration before issuing data collectives.

Part 3 - Quantitative Prediction Before Execution

For two ranks, list the ordered membership of the one-dimensional TP mesh and the feature interval assigned to each coordinate. For a hypothetical four-rank grid with named data and tensor dimensions, enumerate the two kinds of subgroup. Explain why reshaping the same rank list can change which devices communicate. Predict mapping metadata only for unavailable topologies; do not report hypothetical link performance as a measurement.

Part 4 - Experimental Design and Instrumentation

Bind CUDA devices from launcher-provided local rank, establish the process group, and create a named mesh from the validated configuration. Export a CPU record of rank, coordinate, device, and subgroup membership. Run a small rank-distinct collective to check ordering. Compare the mapping with available physical topology evidence and record unknown links. Keep one-process-per-GPU execution as the primary track. Validate world-size products, unique rank membership, and device availability before starting the distributed program. Use offline negative tests for inconsistent meshes instead of creating hangs.

Name the mesh dimension and persist the rank order with the experiment. A rank permutation changes the association between a feature interval and its producer even if all local tensor shapes remain equal. Test that the output reconstruction uses mesh coordinates rather than an unrelated CUDA ordinal. In a multi-dimensional plan, pass the one-dimensional TP submesh to parallelize_module instead of the whole grid. Validate that subgroup dimensions have the intended sizes and that no rank appears twice in the mesh. Separate launcher environment validation from collective execution so a missing device is diagnosed before the computation begins. If topology evidence is unavailable, record only the logical mapping and leave link assumptions unverified. A successful numerical test does not measure bandwidth or prove a placement is optimal for the physical machine.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
import torch.distributed as dist
from torch.distributed.device_mesh import init_device_mesh


@dataclass
class MeshCoordinateRecord:
    global_rank: int
    local_rank: int
    cuda_device: int
    mesh_coordinate: tuple
    tensor_group_ranks: tuple


def build_and_validate_tp_mesh(launch_context, mesh_spec):
    ...


def E05(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require complete and consistent mapping records across ranks. A successful mesh constructor alone does not validate data ordering or topology assumptions. Report group creation cost separately from warmed execution. Record backend and software versions because mesh and TP interfaces evolve. Mark unavailable multi-dimensional experiments not measured while retaining their offline membership checks.

Part 7 - Engineering Interpretation and Philosophy

Explain how logical organization affects placement and synchronization without promising a physical bandwidth guarantee. Choose a mapping that can be justified by the devices and links actually available.

19.42.7 Exercise 06

Exercise Title: Create a DTensor and Experiment with Shard, Replicate, and Placement Changes Across a Multi-GPU Device Mesh

Part 1 - Exercise Contract and Why It Matters

Construct DTensors with Shard and Replicate placements and inspect their logical shapes alongside local storage. Introduce Partial as pending contributions requiring reduction. Deliver a placement ledger and numerical checks for each representation.

Part 2 - System and Memory Mental Model

DTensor metadata describes one logical tensor distributed over a mesh. Its global numel is not the number of elements allocated on each rank. Creating a DTensor from local tensors asserts that those pieces already obey the supplied placement; it is not a request to magically distribute unrelated local values. Replicate requires consistent complete values. Partial requires contributions to a declared reduction. Keep global shape and stride metadata correct, especially when local shard shapes are uneven.

Part 3 - Quantitative Prediction Before Execution

Choose a small two-dimensional logical tensor. Predict its local shapes and useful bytes for sharding each axis and for replication. Give a valid sum-partial example whose local contributions differ but whose sum equals the reference. Explain why the same local buffers cannot simply be relabeled from Partial to Replicate without changing values. Distinguish metadata construction from an actual redistribution operation.

Part 4 - Experimental Design and Instrumentation

Use public DTensor creation APIs under the installed version's contract. Validate local contents before construction; enable supported checks in a diagnostic replay and account for their communication separately. Compare local shape, placement, mesh, global shape, dtype, stride, and backing bytes. Reconstruct small cases outside timing. For uneven local shards, supply required global metadata explicitly and test reconstruction rather than relying on equal-shard inference. Start with inference-only creation, then validate the documented autograd path using a small differentiable computation.

Use coordinate-coded tensor entries to distinguish sharding axis zero from sharding axis one even when a square tensor makes local shapes ambiguous. Build a replica test with intentionally different original values, verify the precondition failure, and distinguish that from an optional check mode that broadcasts one source's values. Record the values before such a check if the audit needs to establish original consistency. For Partial, construct actual summands of a known logical result and verify the declared reduction. Do not create arbitrary placements by calling an internal tensor subclass constructor. When crossing to_local, preserve an explicit description of the local value and future gradient layout. The returned Python tensor does not carry the whole placement contract for an unaware downstream consumer. Validate gradients on this boundary before including it in training.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
from torch.distributed.tensor import (
    DTensor, Shard, Replicate, Partial, distribute_tensor,
)


def create_checked_dtensor(local_tensor, mesh, layout_spec):
    # Establish valid local values and global metadata before construction.
    ...
```

```
def inspect_global_and_local_storage(distributed_tensor):
    ...


def E06(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report global and local shapes, actual local bytes, placement, creation communication, and reconstruction error. Reject rank-distinct values falsely claimed to be replicas. Treat checks that broadcast replica data as observable behavior, not proof that arbitrary original values were already consistent. Do not use private DTensor construction as a substitute for a validated public autograd route. Preserve unsupported creation modes explicitly.

Part 7 - Engineering Interpretation and Philosophy

Explain how a distributed layout is a semantic claim about values as well as a storage description. Identify which information is lost when a DTensor is converted to a plain local tensor without a boundary contract.

19.42.8 Exercise 07

Exercise Title: Redistribute a DTensor Between Sharded and Replicated Layouts and Measure the Collective Communication Triggered by Each Layout Change

Part 1 - Exercise Contract and Why It Matters

Redistribute a valid DTensor among supported placements on the same mesh and measure the resulting work. Build a transition ledger containing expected collectives, observed events, temporary storage, and correct values.

Part 2 - System and Memory Mental Model

A shard-to-replica transition needs missing pieces, whereas replica-to- shard can select local pieces. Sum-partial to replica completes a reduction; sum-partial to shard combines reduction with partitioned output. Changing the sharded tensor axis can require data exchange. The public implementation and backend may introduce padding, packing, or alternative event names. Limit the primary experiment to supported same-mesh transitions instead of assuming arbitrary cross-mesh redistribution.

Part 3 - Quantitative Prediction Before Execution

Predict all-gather for Shard to Replicate, local selection for Replicate to Shard, all-reduce for sum-Partial to Replicate, and reduce-scatter for sum-Partial to Shard. Include a supported shard-axis change as an additional exchange experiment. For each, calculate logical input/output bytes and a live-buffer timeline. State that local selection can retain a larger parent backing even when its useful output size decreases.

Part 4 - Experimental Design and Instrumentation

Construct valid source values for each transition, warm the path, and measure forward redistribution independently from any backward communication. Use a second replay to profile

actual operations and a third small replay to check values and derivatives where supported. Keep creation checks outside the transition region. For asynchronous execution, wait through the documented completion path before reading values or stopping a completed-work timer. Retain buffers until communication and consumers finish. Record dtype casts only if explicitly requested as part of a separate experiment.

Use a fresh valid source for every transition so a previous redistribution cannot alter the intended baseline. For a no-op same-placement request, record whether the implementation returns an alias and whether any local allocation occurs. For shard-axis exchange, count possible packing or padding rather than asserting that an all-to-all event must be the only visible work. Test asynchronous completion with a dependent consumer and verify its output, not merely a successful launch. Keep source and destination references only as long as the declared region requires; retaining every sweep result would inflate later peaks. Do not use undocumented conversion to Partial as a portable shortcut. The primary four transitions can be constructed from valid replicated, sharded, and sum-partial sources without assuming arbitrary placement relabeling is permitted by the installed version.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
from torch.distributed.tensor import Shard, Replicate, Partial


def predict_redistribution(source_spec, target_placements):
    ...


def measure_redistribution(source, target_placements, measurement):
    ...


def E07(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Match predicted transitions with observed traces and explain differences without inventing backend traffic. Count forwards and backwards separately. Report useful local output bytes and retained backing capacity. A transition with no network collective can still allocate or copy locally; do not label it cost-free from its name. Preserve unknown link traffic and unsupported transitions. Validate that asynchronous timing did not measure launch only.

Part 7 - Engineering Interpretation and Philosophy

Explain when a layout change pays for missing information and when it only discards redundant local information. Choose one transition whose temporary memory matters more than its final output size.

19.42.9 Exercise 08

Exercise Title: Apply PyTorch ColwiseParallel to a Linear Layer and Verify Local Parameter Shapes and Distributed Output Layouts

Part 1 - Exercise Contract and Why It Matters

Apply ColwiseParallel to a supported Linear layer and verify parameter shards and output layouts. Compare its behavior with E02 while preserving DTensor metadata long enough to audit the boundary and gradients.

Part 2 - System and Memory Mental Model

The API's style name does not change PyTorch's stored weight orientation. Inspect the local parameter rows and the output feature partition explicitly. A plain local output can represent only one feature shard even though its Python type looks ordinary. Use explicit input/output placements and the local-output setting rather than relying on defaults. The input annotation must describe what the caller really supplies; falsely annotating different inputs as replicas changes the logical computation.

Part 3 - Quantitative Prediction Before Execution

Predict local weight and bias shapes, global output shape, and local output extent for a divisible test layer. Predict the difference between retaining a feature-sharded output and requesting a replicated output. Calculate the additional reconstruction capacity and expected communication. For backward, identify the reduction needed for the replicated input derivative and the ownership of weight-gradient elements.

Part 4 - Experimental Design and Instrumentation

Check installed API signatures, build the mesh, and parallelize a small layer using a declared plan. Initialize from the same logical reference and inspect parameters before the optimizer is created. Keep distributed outputs for layout inspection, then cross a plain-tensor boundary only with known local semantics. Compare outputs, input derivatives, weight-gradient shards, and several Adam updates with the serial reference. Measure larger bounded layers in isolated runs without retaining the full reference model on CUDA. Record conversion and initialization peaks separately.

Inspect the logical parameter registry after parallelization and before optimizer construction. Verify that weight, bias, and their gradients use the expected ownership and that old full parameters are not retained by a reference optimizer or debugging closure. Compare the local bias slice as well as the matrix because zero-bias tests can conceal a wrong partition. Record the installed default settings, but make the experiment's choices explicit so a later default change does not silently modify its contract. If output is a plain local tensor, check the next consumer's expected feature extent before execution. If output remains a DTensor, verify global reshaping and placement propagation using supported operators. Both paths require numerical checks; the abstraction alone does not establish that an arbitrary module between the Linear layers respects its distributed layout.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
from torch.distributed.tensor import Shard, Replicate
from torch.distributed.tensor.parallel import (
    parallelize_module, ColwiseParallel,
)


def build_colwise_plan(boundary_contract):
    ...


def audit_colwise_layer(module, inputs, reference, mesh):
    ...


def E08(config):
    # Apply the plan, validate updates, and measure local storage.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require matching logical values, valid local storage, correct parameter coverage, and update equivalence. Record whether output is DTensor or plain local Tensor and whether it is sharded or replicated. A global shape printed from a DTensor must not be reported as local allocation. Detect a consumer that accidentally treats local output width as full width. Mark API or dtype combinations unsupported when they fail the capability check.

Part 7 - Engineering Interpretation and Philosophy

Explain which correctness obligations the distributed abstraction handles and which remain at plain-tensor boundaries. Decide whether retaining output metadata makes the next layer's contract easier to audit.

19.42.10 Exercise 09

Exercise Title: Apply RowwiseParallel to the Following Linear Layer and Build a Communication-Efficient Two-Layer Tensor-Parallel MLP

Part 1 - Exercise Contract and Why It Matters

Pair the column-parallel first Linear with a row-parallel second Linear to form a two-layer MLP. Preserve sharded hidden features through the activation and avoid an unnecessary full hidden reconstruction. Validate complete training updates as well as the expected communication path.

Part 2 - System and Memory Mental Model

An elementwise activation can act independently on feature shards. The following row-parallel layer consumes those same feature intervals and combines its partial outputs. Gathering hidden features and then splitting them again introduces work without changing the logical MLP. More complex operations between the layers can change this reasoning: normalization

over hidden features or a global feature permutation needs additional treatment. Bias and replicated-output loss semantics remain part of the contract.

Part 3 - Quantitative Prediction Before Execution

For hidden width F, derive the local shapes of stored weights $W_1 \in \mathbb{R}^{F\times H}$ and $W_2 \in \mathbb{R}^{H\times F}$. Predict hidden activation bytes $BSFs/T$ for a feature-sharded buffer. Compare this with a gathered hidden buffer and include simultaneous source/destination capacity. Predict forward output reduction and backward input-contribution combination for the basic replicated-boundary MLP.

Part 4 - Experimental Design and Instrumentation

Use deterministic GELU or another fixed elementwise activation, identical logical parameters, and common input values across TP ranks. Set the two styles' boundary placements explicitly. Validate output, input gradients, local parameter gradients, and Adam updates for several steps. Compare the compatible pair with a deliberately redundant gather-then-split plan in fresh replays. Include forward, backward, and update in training timing. Keep final output reconstruction consistent across variants and record its cost if required by the common loss boundary.

Use a hidden activation whose feature values differ across every interval. Compare the compatible pair with a diagnostic gather that reconstructs all features in the correct order, so the inefficient reference remains numerically valid. At the second layer, verify that its weight columns correspond exactly to the first layer's output rows. Changing rank order between those assignments can preserve all local dimensions while computing a permuted MLP. Validate the intermediate activation separately in a tiny replay, then remove its reconstruction from production timing. Inspect backward communication too: eliminating a forward gather does not imply all backward exchanges disappear. Keep replicated input and output boundaries consistent when comparing complete training steps. Report whether the optimizer stores local moments for shards and full moments for any replicated bias parameters.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
from torch.distributed.tensor import Shard, Replicate
from torch.distributed.tensor.parallel import (
    parallelize_module, ColwiseParallel, RowwiseParallel,
)


def make_paired_mlp_plan(boundary_contract):
    ...


def validate_mlp_updates(model, reference, batches, mesh):
    ...


def E09(config):
```

. . .

Part 6 - Measurement, Validation, and Failure Analysis

Report hidden useful and backing bytes, collective events by phase, complete-step peaks, latency, and update errors. Confirm that no hidden gather survives in the intended compatible pair. A missing collective is not a success if a local shard was misinterpreted as a full tensor. Include nonzero bias and a second optimizer update to reveal stale replicas or incorrect reduction semantics.

Part 7 - Engineering Interpretation and Philosophy

Explain why choosing adjacent layouts together can outperform optimizing each layer in isolation. State the algebraic property that permits the activation to remain local and a counterexample where that property fails.

19.42.11 Exercise 10

Exercise Title: Tensor-Parallelize Transformer Attention Projection Matrices and Verify Correct Head and Hidden-Dimension Partitioning Across GPUs

Part 1 - Exercise Contract and Why It Matters

Partition Transformer query, key, value, and output projections with correct head ownership. Start with ordinary multi-head attention whose query and KV head counts are equal and divisible by TP degree. Deliver attention output and gradient equivalence before optional grouped-query extensions.

Part 2 - System and Memory Mental Model

Column-parallel Q, K, and V projections produce local complete heads. Each local head still attends over all required sequence positions. The output projection is row-parallel over concatenated head features. Local tensors must reshape with local head counts; DTensor-aware reshaping must preserve the intended global layout. Splitting a head's feature dimension accidentally is not equivalent to splitting whole independent heads. Grouped-query or multi-query attention needs a declared query-to-KV mapping and may require KV replication when KV heads cannot be evenly partitioned.

Part 3 - Quantitative Prediction Before Execution

With hidden width $H = hd$, batch B, and length S, predict local Q/K/V shapes and projection storage for h/T heads. Derive local attention-score capacity only for an implementation that actually materializes the score matrix. Contrast that with a memory-efficient attention backend without assuming identical workspace. Predict which operation combines the output projection's partial results. Check the causal mask's logical positions.

Part 4 - Experimental Design and Instrumentation

Use a tiny deterministic attention block with dropout disabled, fixed position handling, and one named attention implementation. Validate projection slices and head identities with coordinate-coded values. Compare final outputs, input gradients, projection gradients, and several optimizer updates with a serial oracle. Inspect transpose, reshape, and concatenation boundaries. Maintain identical masks and positions across TP ranks. Test a nontrivial causal sequence rather than a single token only. Reject unsupported head counts before collective

launch. Keep GQA as a separately validated extension, not an automatic consequence of splitting the hidden dimension.

Test each projection separately before the attention composition. For fused QKV storage, define whether the global ordering groups all queries then keys then values or interleaves them by head; a naive contiguous slice can split the wrong logical groups. The primary exercise may use separate Q, K, and V layers to keep this mapping auditable. Preserve positional transforms on the correct head coordinates and absolute token positions. If testing GQA, require that every local query head can access its corresponding KV head and count any replicated KV storage explicitly. A query-head count divisible by degree does not establish KV compatibility. Keep attention scaling, causal masking, precision settings, and backend fixed across the reference and TP runs, allowing declared floating-point tolerance rather than promising bitwise identity after a changed reduction order.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
from torch.distributed.tensor.parallel import (
    parallelize_module, ColwiseParallel, RowwiseParallel,
)


def plan_attention_heads(attention_spec, tp_degree):
    ...


def verify_local_head_mapping(module, layout, reference):
    ...


def E10(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export global and local head counts, head dimension, weight placements, local activation bytes, selected attention backend, mask identity, errors, and collective events. Check parameter and input gradients, not only logits. Label KV replication explicitly when tested. Avoid a universal quadratic memory formula for a backend that does not retain full scores. An unavailable operator sharding rule is unsupported rather than permission to silently gather every projection and call the result fully tensor parallel.

Part 7 - Engineering Interpretation and Philosophy

Explain why head independence permits local attention while sequence coverage must remain complete. Identify how a plausible reshape can preserve shape compatibility yet change which logical heads the model computes.

19.42.12 Exercise 11

Exercise Title: Tensor-Parallelize Transformer Feed-Forward Layers and Compare Their Memory and Communication Behavior with Attention Layers

Part 1 - Exercise Contract and Why It Matters

Tensor-parallelize Transformer feed-forward layers and compare their memory and communication with the validated attention block. Include an ordinary two-matrix MLP and, optionally, a separately validated gated MLP.

Part 2 - System and Memory Mental Model

The feed-forward expansion and contraction naturally form a column/row pair. In a gated MLP, both expansion branches must assign the same hidden feature intervals before the elementwise product. A matching shape alone cannot prove that gate and value features align. Attention additionally reshapes heads and may have sequence-dependent workspace, whereas the MLP's large expansion activation scales with its intermediate width. Compare complete blocks under the same boundary layout and training semantics.

Part 3 - Quantitative Prediction Before Execution

For expansion width F, predict the two-matrix weight elements $2HF$ and feature-sharded hidden activation elements BSF/T. For a three-matrix gated design, derive the changed weight count and the lifetimes of both branches and their product. Do not sum all buffer maxima as if they necessarily coexist. Compare those timelines with the selected attention implementation at the same B, S, H, T and precision.

Part 4 - Experimental Design and Instrumentation

Build the ordinary paired MLP first, then align gate/value shard manifests if implementing the gated variant. Validate output and gradient equivalence with a deterministic serial block. Measure attention and feed-forward regions in diagnostic replays, then measure the whole Transformer block to capture cross-region lifetimes. Keep norm and residual boundary layouts explicit. Sweep sequence length and expansion width independently. Record saved activations, temporary elementwise outputs, communication buffers, and the initial-state identifier. Fix optimizer and attention backend across the primary comparison.

Measure saved activations and transient forward outputs separately. A gated product can retain both branches for backward, so its cost cannot be inferred from the product's local size alone. Do not use logging hooks that keep GPU tensors alive beyond their actual computation lifetimes. Export metadata or detached CPU scalars at deliberate boundaries. In the attention comparison, state whether the backend materializes probabilities and whether its backward recomputes intermediates; the MLP comparison must use the same training-region definition. Add a controlled expansion-width sweep at fixed sequence length and a sequence-length sweep at fixed expansion width. These experiments distinguish linear activation growth from the selected attention implementation's behavior. Report the observations even if parameter sharding saves less peak memory than the expanded intermediate or another replicated boundary consumes.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
from torch.distributed.tensor.parallel import (
    parallelize_module, ColwiseParallel, RowwiseParallel,
)


def plan_feed_forward(spec, feature_partition):
    ...


def verify_gate_feature_alignment(branch_layouts):
    ...


def compare_attention_and_mlp(replay_specs, evidence):
    ...


def E11(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report useful local parameters, retained activations, peak allocated and reserved bytes, collective payload, completed latency, and numerical errors for each region and the full block. Identify the lifetime that controls the whole-block peak. Include a permuted gate-shard negative control. Do not infer that equal parameter counts imply equal activation memory or that isolated region peaks can simply be added to predict the training peak.

Part 7 - Engineering Interpretation and Philosophy

Choose an optimization target from measured memory components. Explain why the MLP may dominate one configuration while attention dominates another, and how a compatible boundary layout affects both.

19.42.13 Exercise 12

Exercise Title: Introduce Sequence Parallelism for LayerNorm or RMSNorm Activations and Measure the Reduction in Replicated Activation Memory

Part 1 - Exercise Contract and Why It Matters

Shard sequence-position activations around a supported LayerNorm or RMSNorm and integrate that boundary with TP attention and MLP. Measure which replicated activations disappear and which all-gather or reduce-scatter operations replace the previous boundary communication.

Part 2 - System and Memory Mental Model

For batch-sequence-hidden layout, sequence sharding assigns complete hidden vectors for different positions to different ranks. Normalization over hidden features can remain local for each token; hidden-feature sharding would need different normalization logic. Norm parameters

remain replicated and their gradient contributions from different token subsets must be combined correctly. Sequence parallelism around norms is not context-parallel attention: ordinary head-parallel attention still needs its full relevant token sequence.

Part 3 - Quantitative Prediction Before Execution

Predict selected boundary activation bytes changing from $BSHs$ to $BSHs/T$ per rank for divisible length. Add full-sequence inputs materialized for attention or MLP, retained saved activations, and redistribution buffers. Draw the path from sequence shards through gathered inputs to row-parallel outputs reduced into sequence shards. Explain why the whole-model peak need not fall by a factor of T.

Part 4 - Experimental Design and Instrumentation

Use an explicitly supported normalization module and check installed SequenceParallel behavior. Initialize replicated norm weights consistently, including custom initialization. Set sequence dimension from the actual tensor layout. Preserve DTensor metadata through residual additions and prepare the attention and MLP input layouts explicitly. Validate outputs, norm parameter gradients, other parameter gradients, and updates against the no-SP reference. Start without dropout. Compare isolated norm storage and complete-step peaks in separate replays. Test multiple blocks so an unintended gather at every boundary becomes visible.

Validate normalization with nontrivial scale and bias where supported, not only default unit weights. Their replicas must agree before computation and after every optimizer update. Token-sharded norm gradients contain contributions from different positions; verify that the distributed path combines them according to the full logical objective. Otherwise matching forward values can conceal divergent replicated weights after the first step. Check residual additions with both operands on the same sequence intervals. Test two consecutive blocks and the final loss boundary so a local sequence shard is never silently treated as a complete sequence. Use a supported LayerNorm as the primary norm if RMSNorm implementation support is uncertain. A full-context attention gather can remain necessary even when norm and residual activations are successfully sharded; include that temporary capacity in the end-to-end measurement.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
from torch.distributed.tensor import Shard, Replicate
from torch.distributed.tensor.parallel import (
    SequenceParallel, PrepareModuleInput, parallelize_module,
)


def plan_sequence_boundaries(model_spec, tensor_order):
    ...


def verify_replicated_norm_updates(model, reference, batches):
    ...
```

```
def E12(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report boundary placements, local token intervals, norm parameter parity, collectives by phase, useful activation bytes, and complete-step peaks. Detect an incorrect norm that operates on only a hidden-feature slice. If a particular RMSNorm implementation lacks support, use a verified supported implementation or mark it unsupported. Do not infer reduced attention context or altered causal semantics from the phrase sequence parallelism.

Part 7 - Engineering Interpretation and Philosophy

Explain which independent token computations permit sequence sharding and which consumers require more information. Distinguish a scoped activation saving from a guarantee about the model's total peak memory.

19.42.14 Exercise 13

Exercise Title: Trace All-Gather, All-Reduce, Reduce-Scatter, and Redistribution Operations Produced by Tensor-Parallel Execution

Part 1 - Exercise Contract and Why It Matters

Trace communication generated by the validated TP model and connect each operation to a layout requirement. Cover all-gather, all-reduce, reduce-scatter, and redistribution work without assuming a fixed event name across versions and backends.

Part 2 - System and Memory Mental Model

One high-level redistribution may create packing, padding, communication, and local-copy events. Forward and backward can need different collectives. Initialization, diagnostic reconstruction, data-parallel synchronization, and TP execution must remain distinguishable. Trace event counts are not a direct measurement of physical link bytes. A communication launch nested inside a Python scope may complete after that scope exits, so dependency-aware timelines are needed to interpret overlap.

Part 3 - Quantitative Prediction Before Execution

Construct an expected ledger for the paired MLP and the SP-enabled block. For every operation, specify source layout, target layout, logical tensor shape, dtype, group, phase, and payload bytes. Mark input-gradient combination separately from forward output reduction. Predict which collectives should vanish in a no-op same-layout transition and which remain required by the logical computation.

Part 4 - Experimental Design and Instrumentation

Annotate model regions and collective boundaries in an instrumented replay. Collect traces on all participating ranks with common step IDs. Warm the model before the traced steady region and record initialization separately. Inspect both forward and backward, then run ordinary timing without profiler overhead. Map supported backend events to the layout ledger and preserve ambiguous matches as unknown. Use completion dependencies to distinguish useful

overlap from queued launches. Keep full_tensor debug reconstruction outside the production region. Export compact CPU summaries and trace artifact references.

Maintain a common logical step and region identifier in every rank's trace. Where clock alignment is unavailable, compare rank-local durations and causal dependencies rather than treating unaligned absolute timestamps as simultaneous. Attribute collective groups explicitly so an FSDP gather is not counted as a TP activation gather in the optional composition. Include a same-layout transition control and a deliberately redundant gather-then-split boundary as known trace cases. Record profiler configuration and whether memory tracing was enabled. Bound instrumentation overhead through a separate ordinary run. A communication kernel with a long duration can include waiting for a slow rank; inspect launch and dependency evidence before interpreting it as poor link bandwidth. Preserve unmatched or fused events as uncertain instead of forcing a one-to-one mapping that the trace does not support.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
from torch.profiler import profile, record_function


def expected_tp_collective_ledger(layout_plan, training_contract):
    ...


def correlate_trace_events(trace, expected_ledger, rank_metadata):
    ...


def E13(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report predicted and observed calls, unmatched events, group membership, payload estimates, exposed wait, overlap, and trace overhead. Do not sum concurrent event durations and call the result step latency. A collective absent from one rank's trace can reflect incomplete capture; check other ranks and the recording window. State when physical traffic cannot be measured. Retain at least one explained mismatch between a naive expectation and the actual trace if one is observed.

Part 7 - Engineering Interpretation and Philosophy

Explain communication as an information dependency rather than an unqualified overhead category. Choose one operation that can be removed by layout compatibility and another required by the current mathematical contract.

19.42.15 Exercise 14

Exercise Title: Detect an Incorrect Tensor Placement That Causes an Unnecessary Collective Operation and Rewrite the Layout to Eliminate It

Part 1 - Exercise Contract and Why It Matters

Introduce an inefficient layout boundary that causes unnecessary communication, detect it with E13, and rewrite the plan. Keep inefficient but correct layouts separate from false annotations that compute wrong values.

Part 2 - System and Memory Mental Model

A gathered MLP intermediate immediately split for the next row-parallel layer is a candidate redundancy. Removing the gather is valid only if the consumer receives the correct feature interval and every intervening operation supports the sharded layout. Relabeling a partial result as complete can remove a collective while corrupting the function. A successful optimization must preserve outputs, gradients, optimizer updates, and any external boundary contract that the original program promised.

Part 3 - Quantitative Prediction Before Execution

Predict the gather payload and live hidden capacity in a deliberately replicated intermediate plan. Derive the feature-compatible alternative and identify which events should disappear. Include any changed packing or local copy cost. Construct a counterexample with normalization over the full hidden axis to show why an arbitrary gather cannot be removed without redesigning the intervening operation.

Part 4 - Experimental Design and Instrumentation

Use the paired MLP as the primary reproducer, with exactly one changed boundary. Verify both versions against the same reference before profiling. Inspect the collective ledger and retained storage, then implement the compatible sharded edge. Repeat correctness over several updates and an additional seed. Measure completed forward and training latency in separate regions, with equivalent output reconstruction and logging. Audit a plain local output boundary so it does not silently discard placement knowledge. Do not introduce fusion, dtype changes, or a different activation into the same controlled comparison.

Represent the rewrite as a change to one producer-consumer edge with its before/after placements, local feature map, and required operator rules. Verify those preconditions independently of the profiler. The original and rewritten versions must start from identical parameter values and receive the same upstream loss meaning. Compare local weight-gradient slices and replicated parameters after several updates, because a shape-compatible wrong annotation can escape a single forward check. Include an intervening feature permutation as another boundary test: it may be local only if its mapping respects the shard intervals or is redesigned explicitly. A lower collective count is evidence about work, not sufficient evidence about correctness or speed. Keep the smallest correct redundant example and the smallest invalid rewrite as distinct artifacts in the learner's evidence record.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def find_redundant_layout_edges(graph, layout_plan, trace_ledger):
    ...
```

```
def propose_compatible_edge(edge, operator_capabilities):
    ...


def verify_layout_rewrite(original, rewritten, reference, batches):
    ...


def E14(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require a trace showing the expected removed work and numerical evidence for the rewritten model. Report old/new local capacity, allocated and reserved peaks, payload, latency, and any new work. If latency does not improve, examine remaining bottlenecks and variability rather than inventing a speedup. Include a false-placement negative control detected before benchmarking. Keep a minimal reproducer of the redundant boundary in the evidence.

Part 7 - Engineering Interpretation and Philosophy

Explain the proof that the removed communication was redundant. State the operator or layout assumption whose violation would invalidate that proof.

19.42.16 Exercise 15

Exercise Title: Build a Memory Cost Model That Predicts Parameter, Activation, and Communication Buffer Memory for Different Tensor-Parallel Degrees

Part 1 - Exercise Contract and Why It Matters

Build a placement-aware memory cost model for candidate TP degrees. Predict persistent parameters, gradients, optimizer state, activations, and communication buffers per rank and phase. Validate the model against measured storage rather than equating global numel divided by TP degree with peak VRAM.

Part 2 - System and Memory Mental Model

Large projection weights can be sharded while normalization parameters, selected biases, boundary activations, and workspaces remain replicated. Optimizer state follows actual parameter ownership and optimizer dtype policy. Gathered and partial buffers have lifetimes that may overlap with saved activations. Backend allocations and allocator retention extend beyond the logical tensor inventory. A useful model preserves those scopes and exposes unknowns instead of forcing every residual into fragmentation.

Part 3 - Quantitative Prediction Before Execution

Let P_s and P_r be sharded and replicated parameter bytes. Start with

$$P_{\text{local}} = P_s/T + P_r.$$

Replace equal division with actual local extents where necessary. Add gradient and initialized optimizer bytes under declared dtypes. Define peak prediction as a maximum over phase-specific live unique storage, including padding, communication, and workspace allowances. Do not add unrelated category maxima without a common timeline. Produce optimistic and conservative estimates when workspace information is incomplete.

Part 4 - Experimental Design and Instrumentation

Inventory each supported tensor by global identity, local shape, placement, dtype, backing storage, creation, and last use. Calibrate a small number of known workspace or rounding terms on one configuration and validate on held-out shapes and degrees. Keep uninitialized-optimizer and warmed measurements separate. Use actual collective traces to refine buffer lifetimes. Report prediction residuals by phase and rank. Preserve the cost model's calibration inputs and assumptions so later selection does not silently learn from the same test results it claims to predict.

Include construction as a separate phase in the cost model. A plan with small warmed shards can still fail if initialization creates a full model on every CUDA device. Count model loading, parallelization conversion, and optimizer creation under their own lifetimes. For optimizer estimates, inspect actual initialized moments and any master copy instead of applying a familiar bytes-per-parameter constant without assumptions. Represent aliases with a common backing identity so views do not multiply payload counts. Workspace bounds should name the attention and matrix-multiplication configurations to which they apply. Validate on a held-out sequence length where activation memory matters, not only another weight size. Report a predicted feasible plan that failed if encountered, and use its phase evidence to revise the model without rewriting the original prediction.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch


@dataclass
class TensorLifetimeCost:
    logical_identity: str
    placement: tuple
    local_capacity_bytes: int
    first_phase: str
    last_phase: str
    storage_identity: str


def predict_rank_peak(layout_plan, lifetimes, workspace_bounds):
    ...


def validate_cost_model(predictions, held_out_measurements):
    ...
```

```
def E15(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export raw component bytes, assumptions, predicted phase peaks, measured peaks, residuals, and uncertainty status. Check storage aliases before summing. An underestimated attention workspace or optimizer temporary needs its own explanation. Include a case with substantial replicated activation memory to invalidate an all-memory-divides-by-T rule. Do not use reserved memory as if it were simultaneously live tensor payload. A prediction interval is not an allocator reservation or a guarantee of successful allocation.

Part 7 - Engineering Interpretation and Philosophy

Choose the smallest model that preserves the distinctions needed for placement decisions. Explain which uncertainty can be reduced by profiling and which depends on backend behavior or concurrent device use.

19.42.17 Exercise 16

Exercise Title: Sweep Tensor-Parallel Degree Across Available GPUs and Identify When Lower Per-GPU VRAM Is Offset by Higher Communication Cost

Part 1 - Exercise Contract and Why It Matters

Sweep TP degree across available devices and identify the trade-off between lower per-rank memory and communication cost. Separate equal-model latency from the ability to run a larger model, and keep global useful work constant within each comparison.

Part 2 - System and Memory Mental Model

TP ranks cooperate on one batch, so adding TP ranks does not multiply the number of independent examples processed. A smaller local matrix can be less efficient, and launch and communication overhead can dominate at small sizes. Physical topology and contention affect performance. Degree one is a useful baseline, while degrees requiring unavailable GPUs remain predictions or not measured. One multi-GPU machine is sufficient for the available-degree experiments.

Part 3 - Quantitative Prediction Before Execution

Use E15 to predict peak bytes at each permitted degree and calculate local matrix dimensions, heads, and divisibility constraints. Build an optimistic latency model from local compute plus exposed communication, stating the assumed overlap and bandwidth. Predict a small-workload case where increasing degree is slower and a capacity-limited case where sharding permits useful work otherwise infeasible. Do not claim linear speedup.

Part 4 - Experimental Design and Instrumentation

Launch fresh jobs for each supported degree with the same logical initialization, batch, sequence, precision, optimizer, attention backend, checkpointing policy, and number of updates. Validate small-model updates before timing. Warm kernels and Adam state, reset peaks for full training steps, and synchronize only at declared boundaries. Record rank-local timings and the

distributed critical path. Profile representative cases separately. Run the equal-model sweep first, then a bounded capacity sweep whose changed model sizes are clearly labeled. Include cold startup and parallelization cost outside warmed timing but preserve them in the report.

Use global useful-token counts that exclude padding where the objective excludes it. TP participants processing the same tokens contribute only one logical count, while independent data groups contribute distinct counts. Keep degree sweeps on a fixed device subset specification and record rank mapping, because a different link arrangement can change results independently of degree. If only two CUDA devices are available, the empirical sweep is limited to degrees one and two; larger degrees can remain cost-model scenarios. Do not replace unavailable points with benchmark numbers from another machine. For capacity search, preserve candidate dimensions, successful complete update count, first failing phase, and whether validation or checkpointing belongs to the acceptance region. A model that passes forward but fails initialized Adam update does not satisfy the training capacity contract.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def enumerate_available_tp_degrees(device_inventory, model_spec):
    ...


def run_isolated_degree_replay(degree, logical_state, config):
    ...


def compare_degree_tradeoffs(rows, resource_budget):
    ...


def E16(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report per-rank peak allocated and reserved bytes, persistent components, step latency distribution, global examples or valid tokens per second, communication payload, correctness, and failure phase. Count each logical example once across a TP group. Preserve failed candidate configurations and not-measured degrees. A maximum trainable size is only a tested lower bound when the search ends without a failure boundary. Do not compare a smaller batch on one degree as if the difference came from degree alone.

Part 7 - Engineering Interpretation and Philosophy

Select a degree for a stated memory and throughput goal. Explain why the optimum can change with hidden width, sequence length, batch size, or interconnect even when the local parameter formula is unchanged.

19.42.18 Exercise 17

Exercise Title: Combine Tensor Parallelism with FSDP on Separate DeviceMesh Dimensions and Verify That Tensor and Data Sharding Do Not Conflict

Part 1 - Exercise Contract and Why It Matters

Compose tensor parallelism and FSDP on separate named mesh dimensions. Verify that TP partitions computation while FSDP partitions state among data replicas of each TP coordinate. Require correct data ownership, gradients, and optimizer updates before reporting combined memory savings.

Part 2 - System and Memory Mental Model

Use a mesh shaped data-degree by TP-degree. TP groups hold cooperating ranks at a fixed data coordinate; FSDP groups connect corresponding TP coordinates across data replicas. Inputs match within a TP group and differ appropriately across data groups. Apply a supported DTensor-based TP plan and the compatible fully_shard route in a version-checked order. Avoid mixing configuration and parameter semantics from incompatible FSDP generations. A full parameter inside an FSDP group is still that group's TP-local slice, not necessarily the full global matrix.

Part 3 - Quantitative Prediction Before Execution

For data degree D and TP degree T, require DT ranks. Enumerate a 2 by 2 mesh and its four two-rank groups. Predict large parameter shard bytes near $P_s/(DT)$ only at the persistent boundary, then add transient FSDP gathers of TP-local parameters and replicated components. Explain why two devices cannot demonstrate both dimensions larger than one. A degenerate mesh is a control, not proof of nontrivial two-dimensional composition.

Part 4 - Experimental Design and Instrumentation

Validate mesh membership offline, then use at least four available CUDA ranks for the primary nontrivial run. If unavailable, complete metadata checks and label the distributed experiment not measured. Compare a small combined update against the same global-batch serial reference and against TP-only training where feasible. Construct the optimizer after the supported parallelization transformations. Check parameter placements at rest and during execution, gradient ownership, optimizer state, and sample IDs. Keep FSDP communication distinguishable from TP events in traces. All devices may be on one host; multiple computers are not required.

Check data sampling by TP and data coordinates. All ranks in a TP group must operate on matching sample identities, masks, and target values, while data groups cover the intended distinct portions of the global batch. Compute the global loss normalization accordingly; do not average again over the TP dimension. Inspect optimizer state after actual updates and compare reconstructed logical parameters only in a separate small-model replay. Keep the one-dimensional TP submesh and the data-sharding submesh explicit when applying the compatible APIs. Test a degenerate grid as a software control without labeling it a full composition benchmark. A nontrivial two-by- two grid can run on four GPUs in one host; the mathematical group requirement is about ranks and devices, not about requiring multiple networked computers.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
from torch.distributed.device_mesh import init_device_mesh
from torch.distributed.tensor.parallel import parallelize_module
from torch.distributed.fsdp import fully_shard


def validate_two_dimensional_groups(mesh_spec, available_devices):
    ...


def build_tp_fsdp_model(model, named_mesh, plans):
    ...


def verify_global_batch_update(engine, reference, batch_manifest):
    ...


def E17(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report group maps, at-rest and transient local shapes, initialized state, collective classification, complete peaks, errors, and supported status. Do not reuse the same ranks as both data and tensor groups without a valid orthogonal mesh. Detect identical data across all data replicas and divergent inputs within a TP group. Preserve unavailable hardware and version support as explicit limits rather than simulating GPU benchmark results.

Part 7 - Engineering Interpretation and Philosophy

Explain which redundancy each dimension removes and which transient representations remain necessary. Describe why group orthogonality and data semantics are as important as local tensor sizes.

19.42.19 Exercise 18

Exercise Title: Stress-Test Tensor Parallelism with Large Hidden Dimensions, Long Sequences, Uneven Tensor Shapes, and Tight Per-GPU VRAM Budgets

Part 1 - Exercise Contract and Why It Matters

Stress the TP runtime with large widths, long sequences, uneven shapes, and constrained application budgets. Require numerical correctness, rank- consistent rejection, and bounded recovery behavior. Distinguish an unsupported layout from an incorrect execution or genuine resource exhaustion.

Part 2 - System and Memory Mental Model

DTensor support for uneven shards does not imply every high-level TP style, attention reshape, collective path, or kernel supports them. Primary high-level plans may require evenly partitioned feature dimensions. Padding can enable a separate implementation only when

padded features are masked out of all relevant algebra, losses, and updates. Large sequence lengths can shift the bottleneck from weights to activations or attention workspace. Rank-local resource decisions must not change collective order independently.

Part 3 - Quantitative Prediction Before Execution

Construct bounded cases with widths just below and above divisibility boundaries, head counts incompatible with degree, and sequence lengths not divisible by the SP degree. Predict accept, explicit reject, or a separately validated uneven/padded path. Calculate valid versus padded storage and choose a capacity limit that can be tested without depending on a physical OOM. Predict the rank with the largest local extent.

Part 4 - Experimental Design and Instrumentation

Run small deterministic correctness cases first, then increase one shape or resource-pressure variable at a time. Include zero-length or empty local shards only where the installed operator path supports them. Compare actual outputs and updates against a serial reference before scaling. Validate configuration on every rank before launching data collectives. Save committed training state at safe boundaries. If a worker fails, recreate the entire group from a committed state rather than allowing one rank to continue. Use additional seeds and keep the smallest mismatch reproducer.

Build an offline support matrix before launching the stress jobs. Include feature divisibility, head mapping, sequence partition, dtype, norm module, attention backend, and optimizer path. Uneven DTensor construction can pass while the following high-level layer rejects its shape, so validate the whole path. If padding is implemented, test that padded channels cannot influence unpadded outputs through bias, normalization, gating, or loss. Keep padded moment state and capacity visible even when logical gradients are masked. Exercise shapes just around bucket or allocation boundaries without asserting a particular allocator failure will occur. On unsupported input, prefer coherent validation failure before a collective rather than rank-specific exceptions after divergent launches. Preserve the minimized shape and seed for every numerical mismatch that survives the support checks.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def build_tp_stress_cases(model_spec, available_degrees):
    ...


def classify_layout_support(case, installed_capabilities):
    ...


def run_bounded_stress_replay(case, reference, evidence):
    ...
```

```
def E18(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export shape, valid and padded extents, degree, sequence layout, backend, resource budget, per-rank peaks, elapsed time, numerical errors, outcome, and failure phase. A correct rejection counts as rejection, not as successful training. Record OOM or performance instability that does not reproduce as such. Check padding gradients and optimizer state if padding is implemented. Do not attribute every failure to communication without a phase trace and support check.

Part 7 - Engineering Interpretation and Philosophy

Choose a counterexample that breaks a naive placement rule and explain the missing assumption. A restricted runtime with explicit rejection is more useful than a broad claim that silently changes the model or loses gradients.

19.42.20 Exercise 19

Exercise Title: Build and Benchmark a PyTorch Tensor-Parallel Transformer Runtime That Chooses Sharding Layouts to Minimize Per-GPU Memory While Controlling Collective Communication

Part 1 - Exercise Contract and Why It Matters

Integrate the verified layouts into an explicit tensor-parallel Transformer runtime and benchmark a bounded placement selector. Minimize per-rank memory among tested correct candidates while controlling collective cost. Keep model computation, plan construction, validation, and evidence visible rather than hiding the exercise in a high-level trainer.

Part 2 - System and Memory Mental Model

The runtime combines a logical parameter registry, mesh, operator capability map, boundary placements, autograd semantics, and memory/communication costs. A legal plan must preserve attention heads, residual layouts, normalization, loss meaning, and optimizer ownership. Choosing a plan is a constrained search, not proof of a global optimum. Start with a fixed deterministic Transformer, ordinary multi-head attention, FP32, and no dropout; enable other features only with named validation. FSDP composition remains optional and hardware-gated.

Part 3 - Quantitative Prediction Before Execution

Enumerate candidate degree and layout combinations, including paired projection layouts and optional sequence boundaries. Predict persistent bytes, phase peaks, collective payload, and exposed waits before benchmarking. Define accuracy tolerances, application memory limits, required training progress, and a latency objective. Reject unsupported candidates before costly runs. Record why each remaining candidate is expected to meet the budget, including workspace uncertainty and construction-time requirements.

Part 4 - Experimental Design and Instrumentation

Build one canonical logical initialization and apply each candidate in a fresh replay. Validate small-model outputs, gradients, and several optimizer updates against the serial reference. Run separate profiling, ordinary timing, and capacity experiments. Include forward, backward, optimizer update, and required collectives in training timing. Export only CPU scalar evidence

and bounded metadata. Preserve device identity, topology, versions, allocator, seed, model dimensions, precision, optimizer settings, batch/sequence shape, mesh, layouts, attention backend, and initial-state identifier. Validate a sharded save/reload round trip before treating the runtime as restartable; avoid a full CUDA reconstruction merely for checkpoint export.

Define a finite candidate set and deterministic selection rule before benchmarking. Use the cost model to prune obvious infeasible plans, then validate and measure the survivors. Report any feasible plan rejected by an overly conservative model if later tested, rather than claiming the selector found a universal optimum. Keep the capability matrix separate from the scaffold's method names: a checkpoint signature does not prove restart correctness. Require a save/reload comparison of subsequent updates for supported restart. Export versioned local shard metadata with parameter IDs, mesh coordinates, global extents, dtype, and logical optimizer progress. Make final recommendations conditional on the measured shapes, devices, and backend. Preserve unsupported features such as arbitrary GQA layouts, dropout RNG handling, or unavailable FSDP composition instead of treating them as implicitly supported by the integrated runtime.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


class TensorParallelTransformerRuntime:
    def validate_candidate(self, logical_model, mesh_spec, layout_plan):
        ...

    def construct_local_state(self, logical_initialization):
        ...

    def forward_backward_update(self, batch):
        ...

    def export_cpu_evidence(self):
        ...

    def checkpoint_local_state(self, destination):
        ...


def E19(config):
    # Assemble validated components and benchmark isolated candidates.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Keep prediction, observation, and interpretation in separate columns. Every run needs an ID, configuration, input shape, budgets in bytes, per-rank peak allocated and reserved bytes, elapsed seconds, useful throughput, numerical errors, outcome, and failure phase. Use explicit

unsupported and not-measured values. Include the smallest counterexample to a proposed universal rule or state that none was observed in the tested range. Do not invent multi-GPU results or substitute published performance for the local experiment. Report the best tested feasible plan with its limitations.

Part 7 - Engineering Interpretation and Philosophy

Justify the selected plan using verified values, gradients, ownership, and resource measurements. Identify untested configurations that could change the decision.

19.43 DMM41

Chapter Title: Pipeline Parallelism and Pipeline Memory Scheduling
Specialization: Dynamic Memory Management and CUDA Systems

19.43.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Pipeline Stages, Micro-Batches, Activation Transfers, Pipeline Bubbles, and Per-Stage Memory Residency

Part 1 - Exercise Contract and Why It Matters

Define the pipeline as a dependency graph of stage executions and boundary messages. Deliver a model of parameter ownership, microbatch state, activation lifetime, and idle capacity. Use synchronous flushed training as the primary contract: every microbatch in one optimizer update uses the same parameter version.

Part 2 - System and Memory Mental Model

Pipeline ranks own different model stages, not independent replicas of the full model. Microbatches partition one logical minibatch. Forward activations travel downstream; their derivatives travel upstream. In a manual multi-process implementation, raw send/receive does not preserve an autograd graph across processes. Each receiver needs an explicit local differentiation boundary and the runtime must route its input derivative back to the sender. GPipe-style fill/drain and synchronous 1F1B reorder work without authorizing an optimizer update after each microbatch.

Part 3 - Quantitative Prediction Before Execution

For p balanced stages and m microbatches, derive the ideal fill/drain step time $(m + p - 1)(t_f + t_b)$ under uniform per-stage forward/backward costs, negligible communication, and no recomputation. Derive the corresponding idle fraction $(p - 1)/(m + p - 1)$. State why this idealization cannot predict arbitrary imbalance, overlap, or GPU utilization. Sketch live activation sets separately from the execution timeline.

Part 4 - Experimental Design and Instrumentation

Start with a deterministic sequential dense model, FP32, no dropout, no batch-dependent normalization, no cross-stage tied weights, and one process per CUDA device on one host. Define stage IDs, rank placement, boundary shapes, message identity, global batch, loss denominator, and update boundary. Validate small serial outputs, gradients, and Adam updates. Record hardware, versions, backend, topology, seeds, precision, model dimensions, and initial-state ID. Check installed pipeline API capabilities before adopting a documented feature.

Make a paper trace with two stages, four microbatches, and one shared optimizer-update generation. Label each event by stage, microbatch, direction, and parameter version. Show where the final-stage loss seeds backward and where the earliest stage receives its boundary derivative. Mark all graph and transport readers before assigning a release event. State whether the reported bubble is idle time per stage, aggregate idle device-time, or a fraction of makespan; these denominators are not interchangeable. The primary model deliberately excludes batch-coupled operations so a serial full-batch oracle is meaningful. Document this exclusion before evaluating schedules, not after a mismatch appears. Treat stochastic-layer equivalence, tied weights, and arbitrary branching as separate capabilities rather than consequences of implementing a sequential stage chain correctly.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
import torch.distributed as dist


@dataclass
class PipelineContract:
    stage_to_rank: tuple
    microbatch_count: int
    logical_batch_size: int
    boundary_specs: tuple
    loss_denominator: object
    optimizer_boundary: str


def validate_pipeline_contract(contract, model_spec):
    ...


def E00(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require correct logical sample coverage, message matching, derivatives, and parameter versions. Export CPU metadata without retaining live graphs. A smaller bubble prediction is not measured throughput, and a count of live microbatches is not a byte measurement. Unsupported configurations and unrun multi-GPU results remain explicit rather than borrowing published outcomes.

Part 7 - Engineering Interpretation and Philosophy

Explain how scheduling changes memory lifetimes while preserving the intended update. Identify the assumptions that must remain visible when comparing two schedules.

19.43.2 Exercise 01

Exercise Title: Split a Deep PyTorch Model Manually Across Two GPUs and Verify Correct Forward Execution Through Explicit Stage Boundaries

Part 1 - Exercise Contract and Why It Matters

Split a small deep model across two CUDA devices and verify manual forward execution through a named stage boundary. Deliver a layer ownership manifest, boundary specification, and reference comparison before introducing concurrent microbatches or distributed backward.

Part 2 - System and Memory Mental Model

A stage boundary cuts computation and may require carrying multiple values, masks, or metadata. The primary model uses a single sequential tensor edge so the ownership is explicit. A single-process cross-device tensor copy can participate in ordinary autograd, whereas a multi-process send/receive needs the explicit derivative protocol developed later. Keep these execution modes distinct. Construct only the assigned stage on each device instead of retaining a full CUDA model behind references to selected layers.

Part 3 - Quantitative Prediction Before Execution

Choose a cut after a named layer and derive each stage's parameter bytes and boundary tensor shape. Calculate forward transfer payload from actual shape and dtype. Predict the serialized critical path as stage computation plus exposed transfer and receiving work. Include initialization copies and input/output staging as separate memory phases. Explain which part of the model stays on each GPU.

Part 4 - Experimental Design and Instrumentation

Build a tiny serial CPU reference from the same logical initialization. Assign consecutive layers to the two stages, establish device/rank identity, and implement a serialized forward path. Validate boundary values and final outputs using coordinate-coded and seeded random inputs. Run multiple batches to expose stale receive buffers. Keep the inference-only baseline explicitly without backward; a training extension must validate parameter and input gradients separately. Preserve sample IDs and target ordering. Use a bounded larger case only after the small reference passes.

Verify the complete forward expression, including reshapes, activations, and positional or mask arguments that are not standalone parameterized layers. A partition made by slicing a module list can lose work performed between list entries. Compare the boundary tensor against the serial intermediate before checking the final answer. Use a tiny input whose values differ by sample and feature so a stale buffer or sample permutation cannot pass by symmetry. If using the single-process cross-device baseline, record it as such and do not mix its automatic copy-gradient behavior with the multi-process P2P runtime. Include both devices' initialization peaks and confirm that a reference to the original full CUDA model is not retained by an optimizer, closure, or debug object. Keep a full logical reference only in the separate small correctness replay.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch


@dataclass
class StageBoundarySpec:
    source_stage: int
    target_stage: int
    shape: tuple
    dtype: str
    tensor_order: tuple


def construct_manual_stage(model_spec, stage_id, device):
    ...


def run_serialized_forward(stages, inputs, boundary_transport):
    ...


def E01(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report stage ownership, local parameter backing bytes, boundary values, output errors, and transfer payload. Detect a missing non-module operation such as an activation between two layer lists. A correct layer manifest alone does not prove the split forward matches the original graph. Record full-model reference storage outside the measured CUDA process, and flag ties or skip edges unsupported until their boundary protocol is implemented.

Part 7 - Engineering Interpretation and Philosophy

Explain which computation edge was cut and why all information required by the next stage still reaches it. Use this explicit partition as the fixed reference for later scheduling experiments.

19.43.3 Exercise 02

Exercise Title: Measure Parameter, Activation, and Temporary Buffer Memory Separately on Every Pipeline Stage

Part 1 - Exercise Contract and Why It Matters

Measure parameter, activation, and temporary-buffer storage separately on every stage. Include gradient and initialized optimizer state in the training inventory. Deliver a phase-aware byte ledger that can be reconciled with per-device allocator peaks.

Part 2 - System and Memory Mental Model

Parameters persist across microbatches; saved tensors persist until their backward readers finish; transport and kernel temporaries have different lifetimes. Views may share backing storage, and a small retained view can keep a larger allocation alive. Category maxima may occur

at different times. Allocator reserved capacity is not the sum of live tensor payloads. Stage-local Adam state can be absent until the first update, making cold measurements misleading for steady training.

Part 3 - Quantitative Prediction Before Execution

For stage i, derive persistent parameter, gradient, and optimizer bytes under the declared FP32 policy. Define activation bytes as a function of microbatch shape and execution phase, including internal saved tensors rather than boundary outputs alone. Add communication buffers and workspace bounds on a timeline. Predict a stage whose parameters are smaller but whose saved activations make its training peak larger.

Part 4 - Experimental Design and Instrumentation

Inventory unique local backing storage at initialization, forward, backward, update, and clearing boundaries. Use supported saved-tensor metadata instrumentation in a diagnostic replay without storing extra GPU references. Reset allocator peaks for a complete declared region and synchronize only at deliberate boundaries. Warm all used optimizer state before steady comparisons. Record allocated and reserved peaks on every device separately. Profile in a different replay from ordinary timing. Preserve unknown kernel workspace or non-PyTorch allocation scope rather than calling the residual activation memory automatically.

Keep parameter bytes, gradient clearing policy, and optimizer initialization state explicit at every sampled boundary. Setting a gradient to None and zeroing an existing gradient have different storage retention behavior; use the same policy in comparisons. Saved-tensor instrumentation should record shape, dtype, capacity, storage identity, and lifetime metadata without copying or retaining the graph. If a saved view shares parameter backing, identify its category overlap instead of adding its full size to an invented independent activation allocation. Compare a warmed full-update peak with the corresponding phase ledger, not an unrelated forward-only measurement. Include receive buffers allocated before the first forward and output buffers retained for merging. These capacities can remain present even when no microbatch graph is logically live.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch


@dataclass
class StageMemoryRow:
    stage_id: int
    phase: str
    parameter_bytes: object
    gradient_bytes: object
    optimizer_bytes: object
    saved_activation_bytes: object
    communication_bytes: object
    allocated_peak_bytes: int
```

```
def inventory_stage_storage(stage, phase, lifetime_ledger):
    ...


def E02(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export raw storage counts, dtypes, aliases, category lifetimes, and allocator observations. Include an intentionally retained activation in a separate diagnostic replay and show the ledger catches it. Do not obtain activation memory by subtracting independently measured peaks. A gap between inventory and allocator totals needs a scoped explanation or an unknown label. Report first-update allocation separately from warmed execution.

Part 7 - Engineering Interpretation and Philosophy

Choose the memory component that actually limits each stage. Explain why balancing only parameter bytes can leave the pipeline infeasible.

19.43.4 Exercise 03

Exercise Title: Build Point-to-Point Activation Transfer Between Pipeline Stages and Measure Communication Latency and Transfer Volume

Part 1 - Exercise Contract and Why It Matters

Implement explicit point-to-point activation transport and measure its payload and completed latency. Extend the protocol to return boundary derivatives for training. Deliver matching-message and buffer-lifetime checks before attempting overlapping sends and computation.

Part 2 - System and Memory Mental Model

A message needs update generation, microbatch ID, stage edge, direction, shape, dtype, and ordering. On receipt for training, create a local leaf requiring gradients; retain its upstream correspondence and the sender's local output graph. After local backward, send the received leaf's input derivative upstream and apply it to the matching saved output there. Raw P2P does not supply that routing automatically. Keep send and receive buffers alive until the documented completion and CUDA dependency conditions permit reuse.

Part 3 - Quantitative Prediction Before Execution

Calculate forward bytes from boundary activations and backward bytes from their actual derivative dtype. Separate useful payload from metadata, packing, and backend traffic. Predict round-trip dependency time for a small boundary. Define one-way timing only with a valid clock/acknowledgment method; a host launch interval cannot establish completed transfer latency.

Part 4 - Experimental Design and Instrumentation

Begin with a serialized matched send/receive sequence on two ranks. Validate rank-distinct known payloads, then a two-stage differentiable network. Check received inputs, returned

derivatives, and several optimizer updates against a serial reference. Define a rank-consistent communication order and prevalidate counts so negative tests do not intentionally hang the backend. Then introduce nonblocking work with explicit waits and downstream stream dependencies. Measure multiple sizes after warmup and capture packing costs in a named boundary contract. Record intra-host topology without assuming peer-to-peer bandwidth from device numbering.

Separate the application message key from any backend tag support. Do not assume a backend can distinguish arbitrarily reordered messages by tag; use a validated matching order where necessary. A source activation can be noncontiguous, in which case packing may allocate a new buffer with its own completion and release events. Validate the reverse derivative shape and microbatch identity before applying local backward. If an input derivative is not needed at the first stage, define the protocol's termination rather than sending an unmatched message. Use a ping-pong or acknowledged sequence for transfer timing when cross-device clocks are not aligned, and state whether the reported value is round-trip or an inferred one-way estimate. Do not divide by two without naming the symmetry assumptions.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
import torch.distributed as dist


@dataclass
class PipelineMessageKey:
    update_id: int
    microbatch_id: int
    stage_edge: tuple
    direction: str
    buffer_generation: int


def transfer_boundary(payload, message_key, transport, completion):
    ...


def route_boundary_backward(stage_state, received_gradient, key):
    ...


def E03(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report payload, completed latency, packing, waits, buffer generations, and numerical errors. Check gradients at the earliest stage; correct final outputs can hide a detached training graph.

Detect stale-generation delivery and premature reuse through protocol validation even if a race does not reproduce. If physical link traffic cannot be measured, preserve it as unknown instead of equating it with logical tensor bytes.

Part 7 - Engineering Interpretation and Philosophy

Explain how transport correctness includes differentiation and lifetime, not just delivery of values. Identify the smallest protocol violation that would corrupt a later microbatch without changing tensor shapes.

19.43.5 Exercise 04

Exercise Title: Split a Mini-Batch into Micro-Batches and Execute Them Sequentially Across Pipeline Stages as a Baseline

Part 1 - Exercise Contract and Why It Matters

Split one logical minibatch into microbatches and run them sequentially through the complete pipeline. Complete backward for each microbatch before starting the next, accumulate gradients, and update once after the full batch. Use this low-concurrency path as the numerical and lifetime baseline.

Part 2 - System and Memory Mental Model

Microbatching changes execution grouping, not the intended objective. For separable losses, contributions must be weighted by their valid sample or token counts. Averaging microbatch means equally is correct only when those denominators match. Primary experiments use deterministic layers without batch-dependent state. Clear parameter gradients once per optimizer update, not once per microbatch. Do not average across pipeline stages: they own different parts of the same computation.

Part 3 - Quantitative Prediction Before Execution

For microbatch loss sums S_j and a global valid count D, define

$$L = \frac{\sum_j S_j}{D}.$$

Derive accumulated gradients by backpropagating each S_j/D. Calculate a case with unequal microbatch sizes and show why an unweighted mean of means fails. Predict the sequential schedule's maximum activation residency and its lack of stage concurrency.

Part 4 - Experimental Design and Instrumentation

Start with divisible batch sizes, then add a separately supported unequal final chunk. Preserve sample and target identities. On the final stage compute the globally normalized loss contribution, route boundary derivatives upstream, and finish each microbatch before reuse. Accumulate all stage-local parameter gradients, validate against a serial full-batch reference, and perform one Adam update per stage only after the pipeline is drained. Repeat multiple updates to check moment state and clearing. Keep the same reference values and objective for later GPipe and 1F1B comparisons.

Build a concrete three-sample example split into chunks of two and one. Choose losses with different means and derive the global objective and each chunk's coefficient. For token losses, replace sample counts with the valid unmasked-token denominator; equal sample counts

can still have unequal valid tokens. Handle a zero-valid-token chunk through a declared safe convention or reject it in the primary path. Ensure targets are chunked along the same logical sample axis as inputs and metadata. Validate gradients before Adam because an adaptive optimizer can conceal a uniform gradient-scale mistake in a short parameter-only comparison. Compare moments and counters after several updates as well as parameters. Keep the logical batch unchanged when later scheduling moves these chunks into overlapping execution.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def split_logical_batch(batch, microbatch_spec):
    ...


def run_sequential_microbatches(runtime, microbatches, loss_contract):
    # Normalize once, route backward, accumulate, and update after drain.
    ...


def E04(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report sample coverage, valid counts, scalar loss, all stage gradients, Adam moments, updated parameters, elapsed time, and memory peaks. Include an incorrect equal-weight mean-of-means control for unequal chunks. A matched loss scalar alone is insufficient; inspect gradients before update. Reject unsupported batch-coupled layers rather than claiming microbatch equivalence for arbitrary models. Keep unequal-shape support separate from the primary static communication-buffer path.

Part 7 - Engineering Interpretation and Philosophy

Explain why this deliberately serialized schedule is a strong oracle. Identify the semantics that concurrent schedules must preserve even when execution and release order change.

19.43.6 Exercise 05

Exercise Title: Build a GPipe-Style Schedule and Measure Pipeline Fill, Drain, Bubble Time, Activation Memory, and GPU Utilization

Part 1 - Exercise Contract and Why It Matters

Build a GPipe-style synchronous fill/drain schedule with all forward microbatches before the backward wave. Measure fill, drain, dependency idle time, activation residency, and completed training throughput against E04. Keep activation checkpointing disabled in the primary schedule-only comparison.

Part 2 - System and Memory Mental Model

Fill/drain permits different stages to execute different microbatches concurrently. Forward graphs accumulate until their backwards release them. The last stage starts backward only after the forward phase required by the chosen schedule. Parameters remain at one version throughout the logical update. This exercise studies the fill/drain ordering; it does not assume all recomputation choices or optimizations of the original GPipe system. GPU activity, communication wait, and dependency bubbles need separate definitions.

Part 3 - Quantitative Prediction Before Execution

Derive the balanced no-communication estimate from E00 and draw a complete two-stage, four-microbatch event graph. Mark fill and drain explicitly and count saved microbatches per stage. Estimate saved bytes from the actual microbatch activation function. Predict how a slow stage or a large boundary transfer invalidates the uniform-slot model. State which idle intervals are schedule dependencies rather than measured kernel inefficiency.

Part 4 - Experimental Design and Instrumentation

Generate one global event plan and validate every receive dependency, forward-before-backward condition, and message order offline. Execute the small deterministic model using retained per-microbatch stage graphs. Route derivatives with E03 and accumulate gradients with E04's loss normalization. Update only after all backwards and outstanding readers complete. Compare several updates with the sequential oracle before measuring larger cases. Capture stage compute and communication timelines separately from ordinary timing. Record warmup, fill, forward steady region where present, backward region, and final drain under clearly named intervals.

Distinguish all-forward-then-backward ordering from an implementation that launches every forward without waiting for required upstream data. Every event still needs its predecessors complete. Keep the exact backward microbatch order in the plan because different valid drain orders can produce different transient lifetimes. Use the same parameter version for every local graph and verify it before applying its backward derivative. A global drained boundary must also account for outstanding sends, not merely completion of Python backward calls. Report fill and drain as portions of a complete update, including cases with too few microbatches to have a substantial steady region. Do not compare only a favorable central interval against E04's full elapsed step. Record diagnostic GPU utilization separately from a theoretically computed dependency-idle fraction.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def build_fill_drain_plan(stage_count, microbatch_count):
    ...


def validate_pipeline_event_dependencies(plan, boundary_specs):
    ...
```

```
def execute_gpipe_style(runtime, plan, microbatches, evidence):
    ...


def E05(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report predicted and observed makespan, per-stage busy/idle intervals, peak live microbatches, saved bytes, allocated/reserved peaks, and update errors. Do not call the fraction of time without a compute kernel a pure pipeline bubble without checking transfer and host work. A fill/drain schedule can improve concurrency while increasing retained activations relative to E04. Report that trade-off without promising a speedup on every interconnect.

Part 7 - Engineering Interpretation and Philosophy

Explain why concurrency requires retained state and why bubbles are properties of the schedule and stage costs together. Preserve this measured fill/drain path as the GPipe-style reference for 1F1B.

19.43.7 Exercise 06

Exercise Title: Increase Micro-Batch Count and Quantify the Trade-Off Between Smaller Pipeline Bubbles and Larger Activation Residency

Part 1 - Exercise Contract and Why It Matters

Sweep microbatch count and distinguish two different experiments: fixed global batch with smaller chunks, and fixed chunk size with a larger global batch. Quantify bubbles, activation bytes, buffer capacity, and step time without assuming their trends must be monotonic.

Part 2 - System and Memory Mental Model

More chunks can reduce an ideal bubble fraction but increase launches, messages, metadata, and allocator overhead. At fixed global batch, per-chunk activation size often falls as chunk count rises; fill/drain retained bytes may stay roughly constant in a simple linear model. At fixed chunk size, retaining more forward graphs can increase bytes. Kernel efficiency, padding, workspace selection, and nonlinear activation costs can change both cases. A microbatch-count headline must always name which batch quantity stayed fixed.

Part 3 - Quantitative Prediction Before Execution

Let logical batch $B = mb$ in the divisible case and let $a_i(b, S)$ denote saved bytes per microbatch on stage i. Compare $ma_i(B/m, S)$ with $ma_i(b, S)$ under their different controls. Add per-microbatch fixed overhead. Predict the ideal bubble trend and a counterexample where many tiny chunks increase total latency. Do not extend a linear memory approximation to every kernel or variable-length workload.

Part 4 - Experimental Design and Instrumentation

Use a short valid count sweep with equal chunks for the primary run. Rebuild shape-specific buffers and stage wrappers outside warmed timing where required, recording reconstruction cost separately. Preserve initial parameters, logical examples, loss objective, schedule, optimizer, dtype, and sequence length in the fixed-global-batch comparison. Run the fixed-chunk-size experiment as a separate series with changed logical batches explicitly labeled. Validate gradients and several updates for every configuration. Collect per-stage residency traces and completed-step timing after warmup, with profiling in separate replays.

Add a synthetic activation-cost function with a per-chunk fixed term and a batch-proportional term. At fixed logical batch, show that the proportional part can stay constant while the fixed part grows with chunk count. Then add padding to a shape bucket and derive the resulting discontinuity. Use this calculation to formulate predictions before measuring a real model. Keep parameter gradients and optimizer state unchanged in the fixed-batch sweep so the activation effect remains interpretable. If small chunks use views into a full input batch, include the full backing retained by those views. If output merging retains every microbatch result, account for that capacity or use the same supported no-merge policy across runs. A change in message count can alter latency even when total useful payload remains equal.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def make_microbatch_sweep(logical_batch, count_candidates, policy):
    ...


def predict_microbatch_residency(stage_costs, sweep_case, schedule):
    ...


def E06(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export global batch, chunk sizes, valid counts, live microbatch counts, actual retained bytes, padding, message counts, bubble definitions, latency, and update errors. Reject a claim that more microbatches always consumes more activation bytes. Show both useful bytes and retained backing capacity when small views keep a larger parent alive. Explain any throughput deterioration through trace evidence rather than treating the ideal bubble formula as a measurement.

Part 7 - Engineering Interpretation and Philosophy

Choose a microbatch count for the declared batch policy and resource limit. Explain why the same count can be favorable for one batch policy and unfavorable for the other.

19.43.8 Exercise 07

Exercise Title: Build a 1F1B Forward-Backward Schedule and Compare Peak Activation Memory Against GPipe Under the Same Global Batch

Part 1 - Exercise Contract and Why It Matters

Build a synchronous flushed 1F1B schedule with warmup, alternating steady work, and cooldown. Compare activation memory with the GPipe-style schedule under the same logical batch, chunk sizes, partition, and optimizer semantics.

Part 2 - System and Memory Mental Model

1F1B can retire old microbatch graphs earlier by interleaving forward and backward. The resident set depends on stage position, pipeline depth, microbatch count, and exact event order. It is not one microbatch on every stage. This primary schedule performs no asynchronous per-microbatch updates and needs no claim about PipeDream weight stashing. Reducing activation residency does not automatically reduce the ideal synchronous pipeline bubble or improve measured utilization.

Part 3 - Quantitative Prediction Before Execution

Construct two-stage and three-stage event graphs offline. Count warmup forwards and the largest outstanding forward set on each stage, including transient forward-before-backward ordering within a slot. Compare those sets with fill/drain for identical microbatch activation costs. Predict the same parameter and optimizer residency but different activation lifetimes. Derive makespan from the event graph rather than asserting a universal 1F1B speedup.

Part 4 - Experimental Design and Instrumentation

Validate the global plan before running communication. Use a consistent message order or supported batched P2P pattern that avoids both peers blocking on unmatched sends. Route matching derivatives, retain each local graph until its backward completes, and accumulate globally weighted losses exactly as in E04. Drain the whole update before stepping any optimizer. Compare outputs, gradients, moments, and several updates with GPipe and the serial oracle. Measure peak live graphs and actual storage independently. Add stage imbalance only after the balanced correctness case passes.

Enumerate forward and backward dependencies before deciding which local event alternates next. Warmup differs by stage position, and a short update with few microbatches may never enter the steady alternating region assumed by a textbook diagram. Validate those small counts instead of applying a large-count approximation blindly. Keep the final-stage loss and earliest-stage input derivative cases explicit. If transport uses batched send/receive calls, verify operation ordering and buffer lifetimes against the installed backend. A rank cannot reclaim a saved output merely because another stage began backward for that microbatch. Compare gradients accumulated at the drained boundary and inspect Adam moments to detect missed or duplicated microbatch contributions. Present the memory advantage, if measured, independently from any claim about a smaller bubble or faster step.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def build_flushed_1f1b_plan(stage_count, microbatch_count):
    ...


def prove_message_progress(plan, transport_contract):
    ...


def execute_flushed_1f1b(runtime, plan, microbatches):
    ...


def E07(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report each stage's maximum live microbatch count and activation bytes. Include complete-step peaks, communication, makespan, and numerical errors. Include the warmup and cooldown in timing. An implementation that updates weights while old microbatches still require them violates this exercise's contract even if it is called 1F1B. If memory improves but throughput does not, preserve that result and identify the remaining critical path.

Part 7 - Engineering Interpretation and Philosophy

Explain which release opportunities 1F1B creates and which schedule dependencies remain. State why a name shared by synchronous and asynchronous pipeline systems cannot establish identical training semantics.

19.43.9 Exercise 08

Exercise Title: Trace the Exact Lifetime of Every Micro-Batch Activation from Forward Production Until Its Backward Consumer Completes

Part 1 - Exercise Contract and Why It Matters

Trace each microbatch activation from production through its final backward and transport readers. Deliver an exact logical event ledger for the controlled runtime and distinguish logical release, completed device use, and retained allocator capacity.

Part 2 - System and Memory Mental Model

A boundary output can be sent downstream while its local graph remains necessary for backward. A received input leaf can retain backing needed by local differentiation. Send completion alone does not prove the activation is no longer needed. Conversely, backward completion can leave an outstanding gradient send using related storage. Python references, autograd saved tensors, and CUDA operations impose separate lifetimes. A recorded host launch is not a completed GPU reader event.

Part 3 - Quantitative Prediction Before Execution

Define allocation, first write, send completion, backward start/end, last-reader completion, release eligibility, and reuse events for one stage and microbatch. Predict different lifetimes under sequential, fill/drain, and 1F1B plans. Track unique storage plus view intervals so two names for one allocation do not create two physical byte contributions. Explain where the logical schedule gives exact order but instrumentation only bounds completion.

Part 4 - Experimental Design and Instrumentation

Assign update, microbatch, stage, storage, and buffer-generation IDs. Instrument a tiny run with explicit dependency events, keeping CPU metadata rather than live GPU tensors. Use supported saved-tensor inspection without changing saved values or extending their lifetime. Trace forward/backward buffers and both transport directions. Validate against a hand event graph, then compare with a low-overhead run for instrumentation effects. Test a controlled early-release request in an offline state validator; do not require a nondeterministic CUDA race to prove the invariant is necessary.

Use one cache key containing update generation, virtual stage, microbatch, and tensor role. A key containing only microbatch ID becomes ambiguous across updates and when multiple local stages exist. Record storage generations as well as addresses because a new allocation may reuse an old pointer. Keep all last-reader events, including delayed sends and local backward kernels, attached to the activation record until release is justified. Avoid calling item or global synchronization for every event in the final timing path; use a separate diagnostic replay with explicit overhead reporting. If the instrument cannot observe an internal saved tensor's last consumer precisely, report a conservative interval rather than an invented exact timestamp. Validate that this conservative record never underestimates the hand-checked small runtime's logical residency.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch


@dataclass
class ActivationLifetimeEvent:
    update_id: int
    microbatch_id: int
    stage_id: int
    storage_identity: str
    generation: int
    event_kind: str
    completion_reference: object


def record_lifetime_event(ledger, event):
    ...
```

```
def validate_release_eligibility(ledger, activation_key):
    ...


def E08(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export expected and observed event order, timing scope, outstanding readers, and actual reuse eligibility. A repeated data_ptr does not prove that a previous graph was safely finished. Preserve missing completion evidence as unknown instead of calling a host timestamp exact. Verify that logging itself does not retain activation storage. Check that backward for one microbatch cannot consume another microbatch's cached output.

Part 7 - Engineering Interpretation and Philosophy

Explain release as a proof about completed consumers rather than an instruction to delete a variable. Use the ledger to distinguish application reuse from physical memory returned by the allocator.

19.43.10 Exercise 09

Exercise Title: Build an Activation Residency Timeline and Calculate the Maximum Number of Simultaneously Live Micro-Batches per Stage

Part 1 - Exercise Contract and Why It Matters

Convert validated lifetime events into per-stage and per-device residency timelines. Calculate maximum simultaneous live microbatches and peak unique activation bytes. Compare schedule predictions with observed storage without conflating the two quantities.

Part 2 - System and Memory Mental Model

A microbatch can retain several buffers of different sizes. Counts are useful for equal-cost synthetic cases, but heterogeneous sequence lengths or checkpoint policies require weighted byte intervals. Distinct virtual stages on one device contribute to the same physical memory budget. Release and allocation events at the same timestamp need a defined ordering; otherwise a sweep can undercount a transient overlap. Retained pools remain separate from logically live activation bytes.

Part 3 - Quantitative Prediction Before Execution

For intervals $[a_k, r_k)$ with unique storage capacities c_k, define

$$A_i(t) = \sum_{k \in \mathcal{L}_i(t)} c_k, \qquad A_i^{\max} = \max_t A_i(t).$$

Deduplicate backing identities and apply byte changes only when their storage becomes live or fully releasable. Construct equal-count states with unequal bytes and a transient overlap example. Calculate counts and bytes by hand before processing real traces.

Part 4 - Experimental Design and Instrumentation

Validate the sweep on synthetic no-overlap, nested, aliasing, pending-send, and simultaneous-event cases. Build schedule-predicted intervals separately from instrumented intervals and compare by update and microbatch ID. Aggregate all local virtual stages on each device before taking a device maximum. Include confidence or unknown status for incomplete completion events. Compare with allocated and reserved peaks under their different scopes. Export CPU rows suitable for an auditable timeline; do not retain activations to draw it.

Define how the sweep orders allocation and release events with equal host timestamps. Prefer causal completion references or a conservative overlap when ordering is unknown. Count a backing allocation once while any of its retained views or pending consumers keeps it live. Track useful subranges separately if the application reuses an arena, but do not report those subrange bytes as released physical capacity while the arena remains allocated. Build a two-microbatch example where one large activation dominates bytes and another example where several small activations dominate count. Export the peak-contributing set so a learner can audit the maximum without replaying the whole trace. When aggregating virtual stages, take a common device timeline rather than adding maxima sampled at unrelated points in time.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass


@dataclass
class ResidencyInterval:
    stage_id: int
    device_id: int
    microbatch_id: int
    storage_identity: str
    capacity_bytes: int
    start_event: object
    release_event: object


def sweep_unique_storage_residency(intervals, event_order):
    ...


def E09(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report maximum live microbatch count, live bytes, peak time/event, and the storage IDs contributing to it. Explain differences between stage and device maxima. Flag incomplete intervals instead of assuming release at the end of the log. Provide a minimal counterexample showing why multiplying one average activation size by a peak count can fail. Reconcile the timeline with E08's release proof and allocator observations.

Part 7 - Engineering Interpretation and Philosophy

Choose whether counts, byte intervals, or conservative bounds are needed for the next scheduling decision. Explain why a precise count can still be a poor memory predictor.

19.43.11 Exercise 10

Exercise Title: Partition Transformer Layers Across Pipeline Stages by Parameter Count and Demonstrate Why Equal Layer Count Can Still Produce Memory Imbalance

Part 1 - Exercise Contract and Why It Matters

Partition Transformer blocks across pipeline stages and compare equal layer count with parameter-aware cuts. Demonstrate why neither layer count nor parameter count alone guarantees balanced training peaks or stage time.

Part 2 - System and Memory Mental Model

Embedding and output projections can concentrate parameters near the ends, while attention, feed-forward width, sequence length, and selected kernels shape activation and workspace costs. Early pipeline stages can retain more microbatches under a given 1F1B schedule. Residual connections inside complete blocks simplify boundaries; splitting inside a block may require multiple messages. Primary experiments use untied embeddings and output weights. Cross-stage tied parameters need a separate ownership and gradient protocol.

Part 3 - Quantitative Prediction Before Execution

Inventory parameter bytes per ordered block plus embeddings and output head. Predict equal-count and equal-parameter stage totals. Add stage-specific activation residency from the chosen schedule and fixed input shape. Construct a small heterogeneous example where the equal-parameter cut produces a larger activation peak on one stage. Predict the bottleneck using both memory and forward/backward service time.

Part 4 - Experimental Design and Instrumentation

Use a small deterministic Transformer with masks and positional inputs explicitly routed to the correct stages. Preserve complete blocks in the primary partition. Build equal-count and parameter-aware contiguous cuts from the same logical model. Validate serial outputs, gradients, and updates before measuring. Profile each stage with the actual microbatch shape and schedule rather than one isolated block only. Record embedding/head ownership and input/target placement. Keep attention backend, checkpointing, precision, and optimizer fixed while comparing cuts. Test a deliberately heterogeneous block width or head placement as a controlled imbalance case.

Start with untied embedding and output weights so the primary partition has unambiguous owners. If a tied-weight extension crosses stages, it must specify synchronization and gradient combination before it can be accepted. Keep attention masks and positional metadata with the samples they describe, including any non-tensor arguments that must be replicated or reconstructed at a later stage. Profile the actual stage module with its microbatch shape, not only a standalone layer whose surrounding saved activations are absent. Use a named attention backend because long-sequence workspace can differ substantially across implementations. Preserve the same complete Transformer when comparing cuts. Moving a memory-heavy head while silently changing its vocabulary width would not demonstrate a partitioning benefit. Report which stage controls memory feasibility and which controls compute throughput.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def inventory_transformer_units(model_spec, dtype_policy):
    ...


def make_parameter_aware_contiguous_cuts(units, stage_count):
    ...


def compare_stage_balance(partitions, schedule, measurements):
    ...


def E10(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report parameter and initialized-state bytes, live activation peaks, workspace observations, boundary payload, service time, and complete device peaks for every stage. Demonstrate imbalance with measured evidence if it occurs; otherwise preserve the tested outcome. Equal layer counts should not be called equal work without cost evidence. Verify targets and masks after repartitioning and keep cross-stage ties unsupported unless validated.

Part 7 - Engineering Interpretation and Philosophy

Explain whether the observed imbalance comes from persistent state, activation lifetimes, compute, or transport. Identify a partition change that targets the actual bottleneck rather than a convenient layer-count statistic.

19.43.12 Exercise 11

Exercise Title: Build a Memory-Aware Stage Partitioning Algorithm from Parameter Bytes, Activation Bytes, and Estimated Compute Cost

Part 1 - Exercise Contract and Why It Matters

Build a constrained partitioner for contiguous model units using parameter bytes, activation residency, boundary payload, and estimated compute costs. Choose the best feasible plan within a finite candidate set and expose its assumptions rather than claiming a globally optimal arbitrary graph partition.

Part 2 - System and Memory Mental Model

Stage cost depends on the cut and the schedule. Per-layer isolated activation peaks do not add directly to a stage peak, and moving a cut changes both boundary traffic and the number

of retained stage inputs. A memory budget must include gradients, initialized optimizer state, transport buffers, and workspace reserve. A plan that minimizes maximum compute time can violate memory constraints, while a plan with fewer boundary bytes can worsen compute balance.

Part 3 - Quantitative Prediction Before Execution

Define candidate contiguous cuts and a per-stage predicted memory bound under a specified schedule and microbatch shape. Minimize a stated objective such as predicted makespan subject to each device's byte limit. Include infeasible-unit detection and uncertainty margins. Construct a small ordered cost list and enumerate all legal cuts by hand to validate the algorithm's choice and tie-breaking.

Part 4 - Experimental Design and Instrumentation

Calibrate unit and boundary costs on small representative shapes in separate replays. Use exhaustive search or dynamic programming only when its cost assumptions match the modeled structure; otherwise label a greedy or bounded search honestly. Evaluate candidates with the event simulator to capture schedule residency and communication dependencies. Keep calibration and held-out validation configurations separate. Validate numerical equivalence of the chosen partition and measure all stage peaks and complete-step time. Preserve rejected candidates, rejection reasons, and predicted versus observed costs. Do not migrate live optimizer state implicitly when changing cuts.

Validate search behavior on an ordered list small enough for exhaustive manual enumeration. Include a single unit that exceeds every available stage budget so infeasibility is detected without futile search. Include another case with two equally scored plans and define deterministic tie-breaking. If the objective uses maximum service time as a proxy for makespan, label that approximation and validate it against the event simulator. Add boundary transfer estimates and schedule-specific saved-input lifetimes before declaring a plan feasible. Keep the parameter-only baseline even when it performs poorly; its failure explains which added cost information matters. Stage migration at runtime is outside the primary search: construct a fresh model from the same logical state for each measured partition. An offline chosen cut does not itself implement optimizer-state movement.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass


@dataclass
class PartitionCandidate:
    contiguous_cuts: tuple
    predicted_stage_peaks: tuple
    predicted_stage_times: tuple
    boundary_payload_bytes: tuple
    feasibility: str


def search_memory_feasible_partition(units, schedule, budgets, costs):
```

```
    ...


def validate_selected_partition(candidate, held_out_measurements):
    ...


def E11(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report candidate count, objective, constraints, uncertainty, selected cuts, measured peaks, makespan, and numerical errors. Preserve a prediction that underestimated memory rather than overwriting it after the run. Compare against equal-count and parameter-only baselines. A plan rejected because its uncertainty bound is too large is different from a measured infeasible plan. Use actual local optimizer and activation costs rather than global model bytes divided by stage count.

Part 7 - Engineering Interpretation and Philosophy

Explain why the selected plan is the best tested feasible candidate under the declared objective. State which model or cost changes require repartitioning and which can be handled by schedule tuning alone.

19.43.13 Exercise 12

Exercise Title: Use torch.distributed.pipelining.PipelineStage to Implement the Same Model Partition and Compare It Against the Manual Runtime

Part 1 - Exercise Contract and Why It Matters

Implement the same fixed partition with PipelineStage and a supported schedule. Compare the framework path with the validated manual runtime under identical model values, logical batch, loss normalization, and update boundary.

Part 2 - System and Memory Mental Model

PipelineStage manages stage communication and backward support, but its shape, dtype, chunking, and module contracts still need validation. Use an explicit static microbatch specification for the primary experiment. Some versions expose first-microbatch metadata inference; that does not establish unrestricted variable shapes within a schedule. Check the installed version rather than relying on contradictory general descriptions or main-branch features. Framework communication-buffer allocation can differ from the manual runtime even with the same logical partition.

Part 3 - Quantitative Prediction Before Execution

Predict stage input/output shapes from microbatch dimensions, not the full logical batch. Count the framework's expected buffers separately from local model state. Derive the gradient scaling for the chosen loss. For a loss that already returns each microbatch sum divided by the global denominator, ensure the schedule does not divide again. For equal-size microbatch means, verify the supported schedule scaling instead of applying both conventions.

Part 4 - Experimental Design and Instrumentation

Check public imports and signatures, construct only the assigned submodule, and provide representative boundary metadata under the supported API. Configure loss reduction and schedule gradient scaling explicitly. Compare outputs, all stage gradients, Adam moments, and several updates against E04 and the serial reference. Test one schedule first. Keep module modes, optimizer flags, precision, sample order, and output collection equal across implementations. Measure buffer construction outside warmed timing but record its cost and peak. Use diagnostic framework logs in separate correctness replays.

Verify the installed schedule's loss-gradient scaling with a tiny known gradient before using the full model. One supported convention is a globally normalized sum loss with schedule division disabled; another uses equal-sized microbatch means with exactly one count-based division. Do not combine the two. Scalar loss lists may contain unscaled microbatch values, so compute the reported global objective according to the declared convention rather than assuming the list's mean is always correct. Use representative microbatch examples for shapes and dtypes, including tuple outputs if supported. Treat newer first-microbatch inference as a version-specific feature with tested limits, not as permission for arbitrary shape changes. Keep manual and framework output-merging behavior equal because retaining merged outputs can change the peak even when both schedules compute identical gradients.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
from torch.distributed.pipelining import PipelineStage, ScheduleGPipe


def build_checked_pipeline_stage(local_module, stage_spec, group):
    ...


def configure_schedule_loss_scaling(stage, batch_contract, api_caps):
    ...


def compare_framework_manual_updates(replay_specs, reference):
    ...


def E12(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report installed version, supported metadata mode, chunking contract, loss/scaling settings, buffer capacity, peaks, latency, and update errors. Detect double division by microbatch count with a known-gradient test. Do not interpret an API constructor accepting examples as proof that every later shape is supported. A framework failure on an unsupported contract must be recorded clearly rather than bypassed through undocumented internal state mutation.

Part 7 - Engineering Interpretation and Philosophy

Explain which bookkeeping the framework replaces and which mathematical contracts remain the learner's responsibility. Choose the runtime for later experiments using measured support and correctness evidence.

19.43.14 Exercise 13

Exercise Title: Compare ScheduleGPipe, Schedule1F1B, and Interleaved Scheduling by Peak VRAM, Bubble Fraction, Communication, and Step Time

Part 1 - Exercise Contract and Why It Matters

Compare ScheduleGPipe, Schedule1F1B, and a supported interleaved schedule using complete training updates. Separate schedule-order effects from changes to the logical model's virtual-stage partition and physical placement.

Part 2 - System and Memory Mental Model

Single-stage-per-rank schedules and multi-stage schedules have different interfaces and constraints. Interleaving can create extra communication edges and several live local stage states. It is not a promise of lower memory or fewer bubbles for every model. Use one flushed optimizer update per logical batch throughout. Check installed admissibility rules for microbatch counts and rank/stage mapping; do not assume a divisibility restriction from another release is universal.

Part 3 - Quantitative Prediction Before Execution

Generate explicit event plans for the tested schedules. Predict live storage, critical path, message counts, and the definition of dependency idle fraction. For GPipe versus 1F1B, keep the same cuts. For interleaving, report the additional virtual cuts and compare against a suitable control with those cuts where supported. Explain why a single ideal bubble formula cannot decide the interleaved result.

Part 4 - Experimental Design and Instrumentation

Validate every plan with the same logical initialization, samples, objective, precision, and optimizer. Run small serial update oracles before larger benchmarking. Measure fill, warmup, steady intervals, cooldown, output collection, and required completion under the same timing boundary. Capture all ranks' traces separately from ordinary timing. Keep activation checkpointing disabled in the primary schedule comparison. Report the virtual-to-physical stage map and supported constraints alongside each run. Treat unsupported schedule/count combinations as rejected candidates before launching.

Use a candidate table that records physical rank count, virtual stage count, cut locations, microbatch count, and installed admissibility status. This makes a changed interleaved partition visible instead of hiding it behind a schedule name. For the direct GPipe-versus-1F1B comparison, use exactly the same stage modules and loss convention. For the interleaved case, compare against a reference with the same complete logical model and explain any additional virtual boundaries. Measure per-device union busy time rather than summing overlapping compute and transfer durations. If profiling clocks cannot be aligned across ranks, use rank-local durations and dependency relations rather than a fabricated shared timestamp axis. Preserve a case where reduced activation bytes fails to improve throughput, or state that none occurred in the tested range.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
from torch.distributed.pipelining import (
    ScheduleGPipe, Schedule1F1B, ScheduleInterleaved1F1B,
)


def validate_schedule_candidate(candidate, installed_caps):
    ...


def compare_pipeline_schedules(replay_specs, reference, evidence):
    ...


def E13(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export per-device peak VRAM, virtual-stage live sets, bubble definition, collective/P2P payload, complete-step time, useful throughput, and numerical errors. Attribute additional messages to virtual partitioning when appropriate. Do not sum overlapping event durations as makespan or report launch time as completed training time. A schedule with lower activation bytes can still be slower; preserve the actual trade-off and rank imbalance.

Part 7 - Engineering Interpretation and Philosophy

Select a schedule within the tested topology and workload. Explain which part of the comparison isolates scheduling and which part includes the cost of a different virtual-stage layout.

19.43.15 Exercise 14

Exercise Title: Place Multiple Pipeline Stages on Each Rank and Measure How Interleaving Changes Stage Memory and Pipeline Utilization

Part 1 - Exercise Contract and Why It Matters

Place multiple ordered virtual stages on each physical rank and execute a validated interleaved schedule. Measure device-wide memory and utilization while preserving logical layer order and the single-update training contract.

Part 2 - System and Memory Mental Model

A device can own nonadjacent logical stages, so stage ID is not rank ID. The communication graph may revisit a rank and contain both local edges and cross-rank edges. Local edges still have autograd and buffer-lifetime rules. All local stage parameters, gradients, optimizer states, and simultaneous activation sets share one device budget. Adding each stage's independent

peak can overestimate or underestimate the real joint timeline depending on what was included; aggregate actual live storage first.

Part 3 - Quantitative Prediction Before Execution

Map four virtual stages onto two ranks in a declared looped order and list every logical edge and physical transfer. Predict local persistent bytes, message count, and live activation intervals under the chosen schedule. Contrast with two contiguous physical stages implementing the same logical model. Identify whether smaller chunks reduce exposed bubbles enough to offset extra transfers and local scheduling overhead.

Part 4 - Experimental Design and Instrumentation

Validate the global virtual-stage order and physical placement offline. Construct only assigned submodules, with a common logical parameter registry. Use a supported multi-stage scheduler and version-checked microbatch constraints. Compare outputs, gradients, and several updates with the noninterleaved reference. Capture per-virtual-stage service times and per-device union busy time. Audit local edge semantics and outstanding transport work before reuse. Measure ordinary completed steps in separate replays. Keep the same complete model and global batch; report changed cut locations as part of the configuration.

Test two local virtual stages receiving the same microbatch ID during one update. Their caches, gradients, and message identities must remain distinct. A local edge may use a normal differentiable tensor connection or an explicit local boundary protocol; select one validated approach and avoid accidentally applying its derivative twice. Count parameters and moments for every assigned local module in the device inventory. Include concurrent saved graphs from nonadjacent virtual stages in the residency sweep. Compare a looped mapping with a contiguous mapping only under a supported scheduler and document the changed physical edge traffic. More virtual stages can increase messages and bookkeeping even while shortening compute chunks. Do not infer that independent logical stages can execute compute simultaneously on a physical GPU without measured stream and resource evidence.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch


@dataclass
class VirtualStagePlacement:
    virtual_stage_id: int
    owner_rank: int
    logical_layer_interval: tuple
    predecessor: object
    successor: object


def validate_virtual_stage_mapping(mapping, model_registry):
    ...
```

```
def measure_joint_device_residency(local_stages, lifetime_events):
    ...


def E14(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report virtual and physical maps, unique device-wide capacity, peak contributing stages, message payload, busy-time union, step time, and numerical errors. Detect a rank-based cache key that confuses two local stages for the same microbatch. Do not claim physical concurrency between two compute regions merely because they belong to different virtual stages on one GPU. Preserve cases where interleaving adds traffic without improving makespan.

Part 7 - Engineering Interpretation and Philosophy

Explain how finer logical partitioning changes scheduling opportunities while leaving physical capacity finite. Identify when the added placement complexity is justified by the measured workload.

19.43.16 Exercise 15

Exercise Title: Reproduce a Pipeline OOM Caused by Excessive In-Flight Micro-Batches and Build a Memory Admission Limit for the Schedule

Part 1 - Exercise Contract and Why It Matters

Reproduce an excessive-activation admission failure in a bounded pipeline experiment and build a byte-aware schedule limit. Use a deterministic application quota as the primary reproducer; label a real CUDA OOM separately and run it only as an optional isolated test.

Part 2 - System and Memory Mental Model

Live activations, pending sends, receive capacity, and saved graphs all consume resources. Limiting forward launches changes the event graph and can prevent progress. A fill/drain schedule that requires all forwards before backward cannot simply pause below that requirement while waiting for backward to free space. Either reject the configuration or select an explicitly changed valid schedule. A controller must leave capacity for backward and communication, not fill the device with forward state and hope release becomes possible.

Part 3 - Quantitative Prediction Before Execution

Construct a synthetic stage with known activation costs and a quota that rejects a specified admission. Predict the failure event and peak live set. Determine the minimum warmup/progress requirements of the chosen 1F1B plan. Add a backward workspace reserve and pending-transfer capacity. Explain why a count-only cap can admit a large variable-shape microbatch that violates the byte limit.

Part 4 - Experimental Design and Instrumentation

Validate admission and progress on the offline event graph first. Execute a small fitting run, then a quota-rejected excessive-residency run, followed by a feasible revised plan with identical

logical training semantics. Make schedule decisions rank-consistent before issuing messages. Do not insert rank-local waits that strand peers on unmatched communication. Preserve the initial prediction and compare observed live sets. If a real worker allocation failure is tested, terminate and recreate the whole group from a committed state. Never label a simulated quota failure as a measured CUDA OOM.

Build a minimal quota example where the next forward would exceed the budget but a ready backward can release an older activation. Verify that the selected schedule can choose that backward without violating message order. Contrast it with strict fill/drain, where the same cap can make the plan impossible unless it is rejected or explicitly changed. A windowed fill/drain extension changes the schedule and must retain one optimizer update for the full logical batch; it cannot silently step after every window. Reserve output-gradient and backward workspace capacity before admitting the last allowable forward. Validate progress in the offline dependency graph, then measure the small quota reproducer. If real CUDA exhaustion is not run, state that no hardware OOM evidence was collected even when the simulated failure matches the predicted event.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


class PipelineAdmissionPolicy:
    def validate_progress(self, event_plan, resource_bounds):
        ...

    def reserve_for_forward(self, microbatch, stage_id, cost_bound):
        ...

    def release_after_last_reader(self, activation_key, completion):
        ...


def E15(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report quota versus physical failure type, rejected event, reserved pending bytes, live capacity, workspace allowance, revised plan, and update correctness. A lower peak is invalid if some microbatches were silently skipped. Verify all logical samples still contribute once and the pipeline drains. An application ledger cannot guarantee arbitrary real allocations against external device use or unknown workspaces; state its precise scope.

Part 7 - Engineering Interpretation and Philosophy

Explain how memory admission and dependency progress constrain each other. Choose a limit that is both byte-feasible and executable under the validated schedule.

19.43.17 Exercise 16

Exercise Title: Combine Pipeline Parallelism with Activation Checkpointing and Determine Which Stages Benefit Most from Recomputation

Part 1 - Exercise Contract and Why It Matters

Add activation checkpointing to selected pipeline stages and determine which choices improve feasible memory or throughput. Compare identical schedules and logical batches before re-tuning the schedule with recomputation enabled.

Part 2 - System and Memory Mental Model

Checkpointing retains a smaller set of inputs and recomputes selected intermediates during backward. Boundary inputs, communication buffers, and parameter/optimizer state still consume memory. Recomputation lengthens stage service time and can change the bottleneck and bubble pattern. Keep distributed send/receive operations outside checkpointed local functions in the primary design so backward recomputation does not issue duplicate protocol messages. Preserve parameter versions and deterministic local computation across original forward and recomputation.

Part 3 - Quantitative Prediction Before Execution

For each candidate stage, estimate bytes removed from saved intermediates, bytes retained for checkpoint inputs, and added backward compute. Apply those lifetimes to the current schedule and predict the new device peak and critical path. Construct a case where checkpointing a noncritical memory-heavy stage helps and another where checkpointing the slowest stage worsens makespan. Mark estimates that depend on unsupported saved-tensor instrumentation.

Part 4 - Experimental Design and Instrumentation

Use a supported checkpoint route with use_reentrant explicitly selected; start with the documented non-reentrant path where compatible. Keep dropout disabled initially. Validate outputs, stage gradients, moments, and several updates against the uncheckpointed oracle. Test no stages, one selected stage, and all eligible stages in separate replays. Measure recomputation and communication separately. Keep schedule and microbatch count fixed in the first comparison, then run a separately labeled re-tuning experiment. Count all retained boundary inputs and framework buffers rather than equating checkpointing with zero activation residency.

Place checkpoint boundaries entirely inside local stage computation in the first implementation. Keep P2P sends, receives, evidence logging, and mutable schedule state outside the recomputed function. Validate that the recomputed values use the original parameter version and unchanged inputs. A checkpointed stage still needs its received boundary input and any state required by the selected checkpoint API. If stochastic operations are later introduced, test RNG restoration under the documented device assumptions; moving tensors to an unexpected device inside recomputation can change those assumptions. Compare several optimizer updates because stateful side effects can escape a one-step output check. Record whether saved-tensor memory and recompute time are measured directly or estimated, and keep checkpoint-induced changes in the slowest stage's service time visible in the makespan model.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
from torch.utils.checkpoint import checkpoint


def select_local_checkpoint_regions(stage_spec, cost_model):
    ...


def execute_checkpointed_local_region(module, inputs, policy):
    # Keep distributed communication outside the recomputed function.
    ...


def E16(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export checkpoint regions, retained input bytes, saved-byte changes, recompute time, stage service times, complete peaks, makespan, and numerical errors. If stochastic layers are added, validate RNG and device assumptions explicitly. Detect state mutation or parameter updates between forward and recomputation. Do not claim the original schedule's critical path remains unchanged after adding compute. Preserve unfavorable stage selections as counterexamples to blanket checkpointing rules.

Part 7 - Engineering Interpretation and Philosophy

Choose checkpointing by the joint memory and scheduling evidence. Explain why the most activation-heavy stage is not automatically the best place to spend additional compute.

19.43.18 Exercise 17

Exercise Title: Build an Adaptive Pipeline Scheduler That Changes Micro-Batch Count from Per-Stage VRAM Headroom, Bubble Time, and Communication Cost

Part 1 - Exercise Contract and Why It Matters

Build an adaptive microbatch selector using per-stage memory, dependency idle time, and communication costs. Change configuration only between fully drained optimizer updates in the primary runtime. Preserve one fixed logical batch and loss objective while choosing a new valid chunking plan.

Part 2 - System and Memory Mental Model

Changing microbatch count changes tensor shapes, kernel efficiency, message sequences, and possibly framework communication buffers. Updating a schedule field in place cannot establish compatibility. All ranks must agree on the new generation and supported shape contract before execution. Free VRAM is contextual evidence rather than a reservation. Use measured bounds, reserves, and hysteresis to avoid oscillation. Stage repartitioning remains a separate operation requiring explicit state migration or a fresh replay.

Part 3 - Quantitative Prediction Before Execution

Define a finite count set dividing the primary logical batch, plus supported schedule constraints. Predict candidate peaks and critical paths from E09 and E11. State an objective and admission margin. Construct a trace where a small noisy headroom change should not trigger reconfiguration and a sustained pressure change should. Include buffer reconstruction and lost warmup time in the amortized cost.

Part 4 - Experimental Design and Instrumentation

Calibrate on named training shapes, then test on held-out updates and controlled pressure changes. Drain the old plan, finish pending work, agree on a rank-consistent choice, rebuild required shape-specific wrappers/buffers, and preserve optimizer state and logical progress. Verify that no microbatch is duplicated or omitted across the transition. Start with conservative reject-and-keep behavior when evidence is insufficient. Compare against fixed counts and an offline best-tested configuration. Include monitoring and reconfiguration overhead in end-to-end timing. Use fresh jobs for stage-cut changes instead of silently moving live parameters.

Keep a monotonic configuration generation in every message and activation cache. Reconfiguration is allowed only after the old generation is fully drained and its communication buffers are no longer in use. All ranks must agree before any new message sequence begins. At fixed logical batch, prefer count candidates that create supported equal chunks; uneven chunks require the separately validated weighting and shape path. Preserve optimizer moments, counters, parameter values, and logical data position while rebuilding the schedule wrappers. Compare the next update against a reference that uses the same samples with the new chunk count. Include the monitoring interval, hysteresis thresholds, rejected proposals, reconstruction peak, and transition latency in evidence. Do not treat a favorable pre-change memory observation as guaranteed headroom for the next shape or external allocation.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


class AdaptiveMicrobatchSelector:
    def propose(self, stage_evidence, supported_candidates):
        ...

    def agree_at_drained_boundary(self, proposal, runtime):
        ...

    def rebuild_shape_contract(self, runtime, accepted_plan):
        ...


def E17(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report every proposal, accepted generation, rejection reason, predicted and observed peaks, logical batch, chunk sizes, transition cost, throughput, and update errors. Verify equivalent parameter updates across a changed microbatch count using the deterministic model. Label the selector best among tested candidates rather than globally optimal. A policy that reacts quickly but repeatedly pays reconstruction costs may lose to a fixed schedule.

Part 7 - Engineering Interpretation and Philosophy

Explain which evidence justifies adaptation and which uncertainty favors keeping the current plan. Choose a policy that preserves semantics and progress before pursuing a smaller reported bubble fraction.

19.43.19 Exercise 18

Exercise Title: Stress-Test Pipeline Parallel Training with Uneven Stage Sizes, Variable Sequence Lengths, Slow Interconnects, and Tight Per-GPU Memory Budgets

Part 1 - Exercise Contract and Why It Matters

Stress pipeline training with uneven stage costs, variable sequence lengths, constrained transport, and tight application budgets. Require correct logical updates, coherent rejection, and recoverable worker-group behavior. Produce small counterexamples before attempting larger pressure experiments.

Part 2 - System and Memory Mental Model

Shape changes affect boundaries, targets, loss denominators, saved tensors, and workspace requirements. A static framework stage may need padding, shape buckets, or reconstruction at a drained boundary; first-microbatch inference does not automatically permit arbitrary later changes. Artificial transport delays measure scheduler sensitivity, not the physical bandwidth of a different machine. Tight memory can expose progress errors as well as allocation limits. All ranks must retain matching protocol generations and microbatch identities.

Part 3 - Quantitative Prediction Before Execution

Define cases with one slow stage, a large embedding stage, long sequences, unequal valid-token counts, and a pressure limit near predicted residency. Predict outcomes and failure phases. For padded shape buckets, calculate valid loss counts and retained padded capacity separately. Model a delayed boundary as added dependency time and predict how it changes live sets and the critical path rather than simply adding its duration to every stage.

Part 4 - Experimental Design and Instrumentation

Run deterministic small-model correctness cases first. Change one factor at a time, then selected combinations with preserved seeds. Use equal static chunks for the primary path and label variable-shape support extensions. Validate metadata before communication and reject unsupported contracts coherently. Introduce bounded transport delays in a named simulation layer. Keep committed parameter, optimizer, and logical-data progress at safe boundaries. After worker failure, rebuild the group and compare resumed updates with an uninterrupted run. Record all failed and rejected cases, including those that do not reproduce consistently.

Add a variable-length case with equal sample counts but unequal valid token counts across microbatches. Derive the full-batch denominator before backward and verify masked padding contributes neither loss nor gradient. If static buckets are used, include their padded storage in

the budget. For a slow-interconnect sensitivity test, specify whether the injected delay blocks a communication resource, a dependency, or a host launch; these models can produce different overlap behavior. Do not describe an artificial delay as measured PCIe bandwidth. Include a stage imbalance case where the smallest parameter stage is the throughput bottleneck. Minimize numerical failures by returning first to serialized execution, then removing checkpointing, while preserving the same logical model and samples. Store the minimized event trace with the failure classification and reproducibility record.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def make_pipeline_stress_cases(model_spec, protocol_caps):
    ...


def validate_shape_bucket_and_loss(case, global_batch_contract):
    ...


def run_stress_case(case, runtime, reference, evidence):
    ...


def E18(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export stage map, schedule, shapes, valid/padded counts, delays, budgets, peak residency, latency, numerical errors, outcome, and failure phase. Separate simulated pressure or delay from hardware measurements. Check earliest stage gradients, not only final outputs. Preserve the smallest mismatch and its event trace. Unsupported dynamic shapes are rejected configurations, not successful stress runs. An absent OOM or deadlock in a bounded test is not proof of universal safety.

Part 7 - Engineering Interpretation and Philosophy

Explain which stress case most directly tests the runtime's core invariant. Use its result to refine supported scope, resource bounds, or recovery behavior without concealing unsuccessful experiments.

19.43.20 Exercise 19

Exercise Title: Build and Benchmark a PyTorch Memory-Aware Pipeline Runtime That Jointly Optimizes Stage Placement, Micro-Batch Scheduling, Activation Residency, and Pipeline Utilization

Part 1 - Exercise Contract and Why It Matters

Integrate the validated components into an explicit memory-aware pipeline runtime. Jointly search a bounded set of stage placements, schedules, microbatch counts, and local checkpoint choices. Benchmark complete correct training under per-device budgets and preserve a capability matrix.

Part 2 - System and Memory Mental Model

The runtime combines logical parameter ownership, boundary protocols, a dependency scheduler, autograd caches, loss weighting, and a storage ledger. A legal candidate must preserve one logical update and use compatible message and shape contracts. Primary placement search uses fresh replays; online adaptation changes only supported microbatch plans at drained boundaries. Arbitrary graph cuts, cross-stage ties, stochastic behavior, and live stage migration remain unsupported until separately validated. This exercise builds an educational engine, not a claim to replace every production pipeline system.

Part 3 - Quantitative Prediction Before Execution

Before integration, predict candidate persistent bytes, activation live sets, transport buffers, workspace reserves, stage service times, and makespan. Define correctness tolerances, budget scope, progress conditions, and the selection objective. Enumerate enough candidates to compare equal-count, parameter-aware, and memory-aware partitions, plus sequential, fill/drain, and flushed 1F1B schedules where supported. Treat interleaving as an optional explicit virtual-stage configuration.

Part 4 - Experimental Design and Instrumentation

Validate a tiny model from canonical initialization against several serial full-batch Adam updates. Run correctness checks, profiling, timing, and capacity searches in separate replays. Time complete forward, backward, update, and required completion. Preserve E00's reproducibility record plus valid counts, chunk shapes, schedule, placement, checkpoint policy, and application budgets. Export CPU evidence. Verify committed stage-state restart against uninterrupted updates. Use available devices on one host; report unavailable configurations as not measured.

Publish a capability matrix with validating run IDs for each implemented component; scaffold signatures do not establish support. Preserve predicted candidate costs and rejection reasons. Count samples once across the pipeline. Commit matching stage versions, optimizer progress, data position, and configuration identity. Limit conclusions to tested shapes, device maps, schedule constraints, and measured uncertainty.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


class MemoryAwarePipelineRuntime:
    def validate_candidate(self, model_spec, placement, schedule):
        ...
```

```
    def construct_local_stages(self, logical_initialization):
        ...

    def run_drained_training_update(self, logical_batch):
        ...

    def export_cpu_evidence(self):
        ...

    def checkpoint_committed_state(self, destination):
        ...


def E19(config):
    # Integrate validated components and benchmark isolated candidates.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Keep prediction, observation, and interpretation in separate columns. Each run needs an ID, configuration, input shapes, budgets in bytes, stage and device peaks, reserved peaks, elapsed seconds, useful throughput, numerical errors, outcome, and failure phase. Use explicit unsupported and not-measured values. Report the best tested feasible plan, its search limits, and the smallest counterexample to any universal rule, or state none was observed. Do not invent GPU results or convert a simulated quota failure into hardware evidence. Report regressions and adaptation costs alongside improvements.

Part 7 - Engineering Interpretation and Philosophy

Justify the selected plan through checked gradients, ownership, progress, and resource measurements. Identify untested workloads or hardware conditions that could change the decision.

19.44 DMM42

Chapter Title: Multi-GPU Memory Balancing
Specialization: Dynamic Memory Management and CUDA Systems

19.44.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Memory Imbalance, Compute Imbalance, Communication Cost, and Device Capacity Across Multiple GPUs

Part 1 - Exercise Contract and Why It Matters

Model balancing as a constrained placement problem with separate memory, compute, communication, and correctness objectives. Deliver explicit device budgets, ownership maps, phase lifetimes, and supported execution modes before choosing a balance metric or moving state.

Part 2 - System and Memory Mental Model

Equal byte counts do not imply equal capacity pressure on heterogeneous GPUs. Equal pressure does not imply equal stage time or low communication. Process-local allocated and reserved memory differ from driver-wide used memory, and peak values need a named measurement region. Spare capacity on another device cannot automatically satisfy a request on the constrained one. The primary placement and migration track is a deterministic single-process multi-GPU sequential model; distributed sharded compositions are separate version-checked extensions, not freely movable ordinary modules.

Part 3 - Quantitative Prediction Before Execution

Define capacity C_i, application budget B_i, phase-specific predicted live bytes $M_i(t)$, and a required workspace reserve. Calculate feasibility before minimizing imbalance. Compare a placement with equal bytes on unequal capacities against one with unequal bytes but equal pressure. Add a communication edge cost and explain why memory feasibility and throughput are distinct constraints rather than interchangeable scores.

Part 4 - Experimental Design and Instrumentation

Establish a small serial reference for outputs, gradients, and several Adam updates. Start with FP32, deterministic layers, no cross-device tied weights, no stochastic modules, and explicit copies between devices. Freeze logical parameter IDs and input/loss semantics. Record device identity, capacity, topology, process mode, software versions, allocator backend, seed, model, precision, optimizer, batch/sequence shape, and initial-state ID. Validate installed capabilities before using DeviceMesh, TP, FSDP, or pipeline APIs.

Write an ownership table for the primary single-process model and another for each distributed extension. Include logical parameters, actual representations, optimizer owners, sample identities, and communication participants. A rank is a process identity; a device is a physical execution resource; a stage or shard is a logical role. Keep those meanings distinct even when the primary mapping is one-to-one. Use a two-device paper example to show a placement that balances parameters but overloads one device during backward. Add another example where a low-memory device is the compute bottleneck. These examples should determine which measurements are required, not become fabricated local benchmark results. Define whether the task is initialization, inference, or training capacity and use complete training updates for the primary conclusions. State the loss objective and global sample coverage before evaluating any balancing policy so a smaller peak cannot be purchased by silently omitting work or changing the model.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch


@dataclass
class BalancingContract:
    process_mode: str
    device_budgets_bytes: tuple
    logical_parameter_registry: object
    supported_migration_units: tuple
```

```
    loss_identity: str
    measurement_region: str


def validate_balancing_contract(contract, model_spec):
    ...


def E00(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Keep prediction, observation, and interpretation separate. Require correct useful updates and compatible communication before accepting a lower balance score. Unknown overhead remains unknown. Simulated budgets, offline topology examples, and unrun distributed experiments must not be reported as hardware measurements. Export CPU evidence without retaining live training graphs.

Part 7 - Engineering Interpretation and Philosophy

Explain which constraint makes a device limiting and which trade-offs remain after feasibility. A useful balancer preserves training semantics while making explicit, measured resource decisions.

19.44.2 Exercise 01

Exercise Title: Measure Allocated, Reserved, Peak, and Free VRAM Independently on Every CUDA Device in a Multi-GPU PyTorch Process

Part 1 - Exercise Contract and Why It Matters

Measure current allocated, reserved, peak allocated, peak reserved, and driver free/total bytes independently on every visible CUDA device used by the declared process. Deliver device-indexed measurements with explicit scope, phase, and timestamp.

Part 2 - System and Memory Mental Model

Allocator counters describe the calling process's allocator on a device. Driver free/total information has broader device scope and can reflect other processes and non-allocator allocations. Reserved capacity includes cached storage; it is not additional to allocated payload. Reading every device in one process is not equivalent to summing every distributed rank's counters. Queries can initialize contexts or perturb timing, so cold initialization and warmed observations must be distinguished.

Part 3 - Quantitative Prediction Before Execution

Predict the response of each counter to allocating, retaining, releasing, and reusing a bounded tensor on one chosen GPU. Specify whether peaks were reset and what region they cover. Calculate driver used bytes as total minus free under that API's scope, but do not equate it with this process's live tensor bytes. Predict what a second process's allocation would change.

Part 4 - Experimental Design and Instrumentation

Use explicit device arguments rather than relying on current-device state. Warm declared contexts deliberately, then reset peaks at synchronized region boundaries on each participating device. Run an allocation-only trace first and a full forward/backward/update trace separately. Record sampling order and age because readings across devices are not atomic. In one-rank-per-GPU extensions, each rank samples its owned device and exchanges CPU metadata; do not have every rank create contexts on all GPUs merely to query them. Preserve backend-specific unsupported counters.

Use stable device identity alongside the visible CUDA ordinal, since visibility settings can renumber devices. Record the process mode and whether a sample caused first use of a context. Reset peaks on every participating device before the named region; resetting a peak counter neither releases memory nor reserves future capacity. Sample a bounded retained tensor, a released tensor, and a reused allocation to distinguish current and historical values. Keep any optional empty_cache experiment separate from ordinary timing because it changes allocator reuse behavior. In a distributed run, aggregate process-local counters only with complete rank ownership metadata; several ranks sharing one device can have distinct allocator totals but repeat the same global free/total observation. Timestamp each row and reject an allegedly simultaneous comparison when its sampling skew exceeds the declared tolerance. Unknown external activity should remain context, not be subtracted from a process ledger through an unsupported assumption.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch


@dataclass
class DeviceMemorySample:
    process_id: int
    device_identity: str
    phase: str
    timestamp_seconds: float
    allocated_bytes: object
    reserved_bytes: object
    peak_allocated_bytes: object
    peak_reserved_bytes: object
    driver_free_bytes: object
    driver_total_bytes: object


def sample_explicit_device(device, measurement_contract):
    ...


def E01(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export raw bytes, device mapping, reset boundary, timestamp, and scope. Check that a deliberate allocation appears on the intended device. Do not sum driver-wide used readings duplicated across ranks. Explain cached reserved memory after tensor release without calling it a leak automatically. An asynchronous phase sample needs a clear completion convention; a host function return is not a synchronized training boundary.

Part 7 - Engineering Interpretation and Philosophy

Choose which counter answers a live-payload, allocator-capacity, or device-headroom question. Explain why no single used-memory number covers all three scopes.

19.44.3 Exercise 02

Exercise Title: Build a Per-GPU Memory Dashboard That Tracks Parameters, Gradients, Activations, Temporary Buffers, and Free Capacity During Training

Part 1 - Exercise Contract and Why It Matters

Build a CPU-side per-GPU dashboard from validated samples and a storage ledger. Display parameters, gradients, initialized optimizer state, saved activations, communication/temporary buffers, and driver headroom with separate scopes. The learner implements the monitor and evidence export explicitly.

Part 2 - System and Memory Mental Model

Allocator APIs do not automatically classify bytes by model role. Category accounting needs a registry and lifetime instrumentation, with aliases counted once. Some backend workspace and external allocations remain unattributed. A dashboard must not subtract unrelated category peaks and label the residual activation memory. Free capacity is sampled context, not a promise that the next allocation or migration will succeed.

Part 3 - Quantitative Prediction Before Execution

Predict persistent bytes for a small two-device model after Adam is initialized. Add saved activations and transport buffers on a phase timeline. Design the display so current live categories, historical peaks, and driver-wide headroom cannot be mistaken for simultaneous additive quantities. Define unknown, unsupported, stale, and not-measured states instead of filling missing fields with zero.

Part 4 - Experimental Design and Instrumentation

Collect metadata at deliberate training boundaries and lower-frequency runtime samples. Use supported saved-tensor inspection only in a diagnostic replay, exporting storage IDs and byte counts without retaining GPU tensors. Deduplicate shared backing across categories and preserve overlap annotations. Attach process/rank and device identity to every row. Separate rendering or logging overhead from ordinary timing. Bound log retention and avoid per-layer global synchronization in the final monitor. Validate the dashboard against a tiny hand-checked allocation and training trace.

Design each dashboard row with a field for scope, observation age, and provenance. Separate current categories from maxima and show whether an entry comes from direct storage inspection, a supported instrument, or a model estimate. Parameter counts by name can duplicate tied storage; use backing identity and retain the logical alias relationship. Saved activations

can include views of parameters or larger input buffers, so record category overlap before summing. Include optimizer scalar state, not just moments, and state whether it resides on CPU or CUDA. A dashboard can present historical phase peaks without implying those phases coexist. Validate the displayed free-capacity field against its original driver sample rather than deriving it from physical capacity minus process allocations. Bound CPU history size and sampling frequency, then measure monitor overhead. Do not let the evidence log itself become the source of the apparent memory growth it is meant to diagnose.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch


@dataclass
class DeviceCategoryRow:
    device_identity: str
    phase: str
    category_bytes: dict
    unattributed_status: str
    sample_age_seconds: float


def build_device_category_rows(storage_ledger, raw_samples):
    ...


def export_dashboard_cpu_rows(rows, destination):
    ...


def E02(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require raw values alongside derived display fields. Include a retained view and a delayed transport buffer to test category lifetimes. Compare process-local allocated bytes with the category inventory under declared scope, preserving unexplained residuals. Verify that logs do not keep live autograd graphs or GPU references. Report the dashboard's sampling overhead and age rather than presenting stale rows as exact simultaneous state.

Part 7 - Engineering Interpretation and Philosophy

Explain which decision each field supports and which uncertainties remain. Prefer an auditable incomplete inventory to an apparently complete breakdown assembled from incompatible counters.

19.44.4 Exercise 03

Exercise Title: Create an Intentionally Imbalanced Model Placement and Quantify the Difference Between Maximum and Minimum Per-GPU VRAM Usage

Part 1 - Exercise Contract and Why It Matters

Create a deliberately imbalanced placement of the same logical model and quantify per-device memory differences. Compare it with a controlled reference placement while preserving inputs, model values, optimizer, and complete training semantics.

Part 2 - System and Memory Mental Model

A device can be overloaded by persistent parameters, long-lived activations, a large output head, or transfer staging. The primary single-process model uses explicit device transitions with autograd-aware copies. A placement change can alter both memory and communication, so classify the source of imbalance. Small local tensor shapes do not prove independent backing, and initial construction can create larger peaks than warmed execution.

Part 3 - Quantitative Prediction Before Execution

Choose heterogeneous layer widths and assign most large layers to one GPU. Predict local parameter and initialized-state bytes, boundary payload, and activation lifetimes. Calculate maximum-minus-minimum current allocated bytes at one aligned boundary and the analogous spread of region peaks. Explain why peak spread need not describe an instantaneous state when peaks occur at different times.

Part 4 - Experimental Design and Instrumentation

Use a bounded deterministic network with no ties or batch-coupled state. Construct only the planned local modules from a canonical initialization. Validate outputs, gradients, moments, and several updates against a serial oracle. Measure both intentionally imbalanced and reference placements in fresh replays. Sample each device at common completed phases and preserve sampling order. Include cold construction and warmed full-update peaks as separate regions. Record extra cross-device copies and output collection.

Use a tiny model whose layer dimensions make the intended imbalance obvious by hand. Include at least one nonzero activation and nonzero loss contribution from every sample so skipped work cannot improve the memory score unnoticed. Measure the same completed boundary on both GPUs and store the sampling timestamps with the spread. For peak comparisons, preserve the event or phase where each device reached its own maximum. An average of those maxima is an operational summary, not a reconstructed simultaneous state. Compare initialization separately because creating a whole model on one GPU and redistributing it later can dominate the apparent imbalance. Check that no old full model survives behind parameter groups or Python closures. If both placements fit, compare useful throughput; if one fails, report the phase and successful work count rather than assigning it a fictitious step time. The experiment should reveal the limiting device without assuming that every unequal placement is bad.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def make_intentionally_imbalanced_map(layer_inventory, devices):
    ...


def run_checked_placement_replay(placement, logical_state, config):
    ...


def quantify_device_spread(device_rows, region_contract):
    ...


def E03(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report per-device raw bytes, category inventory, current spread, peak spread, timestamps, throughput, and numerical errors. Do not call the least allocated GPU idle without compute evidence. If activation costs reverse the predicted ranking, retain that outcome and identify the responsible phase. A lower spread produced by wasting memory on the emptier GPU is not an acceptable optimization.

Part 7 - Engineering Interpretation and Philosophy

Explain the concrete source of imbalance and which device limits feasibility. Use this reproducer to evaluate later metrics rather than assuming all unequal byte totals require migration.

19.44.5 Exercise 04

Exercise Title: Define Memory Balance Metrics from Per-GPU Used Memory, Peak Memory, Capacity, and Utilization Ratios

Part 1 - Exercise Contract and Why It Matters

Define a family of balance metrics with named accounting layers and units. Compare absolute spread, normalized pressure, peak pressure, and predicted slack. Deliver counterexamples to treating any one metric as a universal placement objective.

Part 2 - System and Memory Mental Model

Capacity and usable application budget can differ. Device-wide used memory and process-local allocated bytes answer different questions and must not be mixed inside one vector. Historical per-device peaks may occur at different times. A memory pressure ratio is not GPU compute utilization. Balance is subordinate to feasibility and useful work: making every device fuller can lower relative imbalance while reducing safety margin.

Part 3 - Quantitative Prediction Before Execution

For a declared same-scope vector U_i, define

$$\Delta_U = \max_i U_i - \min_i U_i, \qquad \rho_i = U_i / B_i \quad (B_i > 0).$$

Define predicted slack from application budget minus a phase peak and reserve, counting the reserve exactly once. If using coefficient of variation, keep zero-mean cases undefined or explicitly handled. Compare equal bytes on unequal budgets and unequal bytes on equal pressure. Add a request-size fit counterexample.

Part 4 - Experimental Design and Instrumentation

Validate formulas on synthetic two- and three-device cases before measuring hardware. Include all-zero use, missing capacity, one unavailable sample, heterogeneous budgets, and a stale driver reading. Define aggregation boundaries and timestamp tolerance. Evaluate metric rankings against observed full-update feasibility and latency in separate replays. Keep original predictions and raw vectors beside derived values. Do not infer allocator fragmentation or causal compute imbalance from a memory spread alone.

Choose one accounting layer for each metric vector and put that choice in its name or metadata. Validate budget-normalized pressure with positive budgets only; a missing or zero budget cannot silently produce zero pressure. For peak vectors, preserve each component's region and time. Add a case in which all devices have equal pressure above their application budgets, showing that perfect balance can still be infeasible. Add a case where increasing unused retained cache makes an absolute-spread statistic look better without creating any useful capacity. Request size and contiguous-fit restrictions can defeat a simplistic free-byte ranking, so keep a separate next-phase allocation estimate and unknown eligibility status. If using coefficient of variation, calculate population or sample convention explicitly and test zero mean. Do not interpret a memory-utilization ratio as SM activity or kernel occupancy. Counterexamples must accompany the formulas before those formulas are allowed to drive an automatic move.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass


@dataclass
class BalanceMetricResult:
    accounting_scope: str
    raw_usage_bytes: tuple
    budget_bytes: tuple
    absolute_spread_bytes: object
    normalized_pressure: object
    predicted_slack_bytes: object
    limitations: tuple


def calculate_balance_metrics(samples, budgets, prediction_region):
    ...
```

```
def E04(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report undefined denominators and unknown inputs explicitly. Require a minimal counterexample for every proposed universal ranking. Distinguish current balance from worst-phase capacity risk. Show a case where a modest imbalance is preferable because it removes expensive transfers. If a predicted safe placement fails, preserve the failure phase rather than silently redefining the metric after observing the result.

Part 7 - Engineering Interpretation and Philosophy

Choose a minimal metric set for feasibility screening and investigation. Explain which decision requires byte slack, which requires normalized pressure, and which requires actual compute or communication measurements.

19.44.6 Exercise 05

Exercise Title: Partition Model Layers Across GPUs by Equal Layer Count and Demonstrate Why Equal Counts Do Not Guarantee Equal Memory Consumption

Part 1 - Exercise Contract and Why It Matters

Partition ordered layers by equal count and test whether this balances memory. Compare useful persistent bytes and complete training peaks, including activations and transfers, on the same logical model.

Part 2 - System and Memory Mental Model

Layers can differ greatly in matrix dimensions, persistent buffers, optimizer state, and saved activations. Embedding and output layers can dominate parameters while another stage dominates activation residency. A layer list is not necessarily the complete forward graph; residuals and non-module operations must remain intact. Equal-count partitioning is a baseline with explicit limits rather than a memory guarantee.

Part 3 - Quantitative Prediction Before Execution

Construct a short ordered model with equal numbers of layers per GPU but unequal widths. Calculate parameter and buffer bytes from shapes and dtypes. Add initialized gradients and Adam moments under the declared flags. Predict a boundary activation and local saved-state peak. Give a second example with similar parameter totals but different activation costs.

Part 4 - Experimental Design and Instrumentation

Choose contiguous cuts so the primary comparison has a simple device transition graph. Preserve every activation, reshape, mask, and residual operation. Use untied deterministic modules and the same initial values. Validate the partition against serial outputs, all gradients, and multiple updates. Measure cold construction and warmed full steps separately. Record layer inventory, cut positions, local capacities, transfers, and phase-specific peaks. Keep batch size, sequence length, precision, optimizer, and timing boundaries fixed while changing only the partition rule.

Keep complete logical computation units rather than assuming every module boundary is a safe device cut. A residual edge may carry an earlier activation past several counted layers, changing both transport and residency. Use contiguous complete blocks for the primary baseline and record any unavoidable extra boundary values. Count registered buffers as well as parameters; some layers contain non-parameter state whose placement matters. Compare the same number of completed optimizer updates and the same loss weighting. If a batch-dependent layer is present, either implement its intended semantics across the chosen execution grouping or exclude it from the deterministic oracle. Include a synthetic example with a large embedding or output layer and several tiny normalization layers. Its equal-count partition can be calculated without GPU execution and should predict a concrete persistent imbalance. Then measure whether activations or workspace reinforce or reverse that prediction in the supported training model.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def equal_count_contiguous_partition(ordered_units, devices):
    ...


def audit_partition_forward_graph(model_spec, placement):
    ...


def compare_equal_count_memory(placement, reference, measurements):
    ...


def E05(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report layer counts beside actual bytes, peak phase, compute time, communication, and correctness. Demonstrate unequal memory when observed; otherwise state the tested model did not expose the proposed effect. Include a hand-calculated heterogeneous counterexample regardless of hardware outcome. Do not fix a poor baseline by changing model width or dropping an operation. Explain any mismatch between persistent-byte predictions and training peaks.

Part 7 - Engineering Interpretation and Philosophy

Identify what equal-count partitioning ignores. State which additional cost information is necessary before using it for a capacity decision.

19.44.7 Exercise 06

Exercise Title: Build a Greedy Layer Placement Algorithm That Assigns Modules According to Their Parameter and Buffer Memory Footprints

Part 1 - Exercise Contract and Why It Matters

Build a deterministic greedy placement algorithm using parameter and persistent-buffer footprints. Deliver valid execution maps, explicit feasibility checks, and comparison with equal-count placement. Label this as a persistent- state heuristic rather than a complete training-memory planner.

Part 2 - System and Memory Mental Model

Greedy assignment can reduce predicted persistent imbalance while adding many device transitions or ignoring activation peaks. Module ownership must respect shared storage, execution dependencies, and the runtime's supported cuts. In the primary sequential model, noncontiguous device assignments are allowed only with explicit autograd-aware transfers at every changed edge. Optimizer state and gradients are additional persistent training costs whose omission must remain visible.

Part 3 - Quantitative Prediction Before Execution

Sort eligible units by a declared footprint and assign them to the feasible device with the smallest resulting chosen load score. Define tie- breaking and treatment of indivisible units. Hand-simulate unequal unit sizes and a unit that fits nowhere. Predict parameter/buffer totals and count the cross-device execution edges introduced by the resulting map.

Part 4 - Experimental Design and Instrumentation

Inventory unique parameter and buffer storage from the logical model, including dtype and alias relationships. Group tied or shared units as indivisible constraints or reject them in the primary track. Validate the algorithm on tiny cases and compare its score with exhaustive enumeration where feasible. Construct the selected map from canonical values in a fresh replay. Check complete forward and gradient semantics before benchmarking. Report the omitted activation and temporary costs explicitly. Keep the heuristic's objective and execution-order constraints fixed during comparison.

Define the placement unit independently of the order in which a framework traverses modules. Preserve a stable logical ID and an eligibility set. If two units share storage, either merge them into a constrained unit or reject the unsupported configuration; assigning their shared names independently would create an invalid byte ledger. State whether the greedy order is descending size and whether the load score is bytes or normalized budget pressure. Use deterministic tie-breaking so the same inventory produces the same map. Compare a tiny case with all legal assignments to expose the heuristic's limits. The execution engine must follow the original logical layer order, even if placement order was size-sorted. Add explicit transfers at every cross-device edge and verify their gradients. Do not execute layers in greedy assignment order. Keep the omitted activation and communication costs visible in every result row until later exercises add them to the model.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass


@dataclass
class PlacementUnit:
    logical_id: str
    parameter_bytes: int
    persistent_buffer_bytes: int
    eligible_devices: tuple
    alias_group: object


def greedy_persistent_placement(units, budgets, tie_break_policy):
    ...


def E06(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export assignment order, candidate device scores, rejection reasons, final persistent loads, execution edges, measured peaks, and update errors. A heuristic that balances its input cost has not proved training feasibility. Check oversized units and zero eligible devices without entering a futile retry loop. Retain a case where the greedy plan is worse than the tiny exhaustive oracle, or state none was found within the enumerated range.

Part 7 - Engineering Interpretation and Philosophy

Explain what the heuristic optimizes and why its simplicity can be useful. Identify the missing cost that most plausibly invalidates its placement on the chosen workload.

19.44.8 Exercise 07

Exercise Title: Extend the Placement Cost Model to Include Activation Memory and Discover Cases Where Parameter-Only Balancing Produces GPU OOM

Part 1 - Exercise Contract and Why It Matters

Extend the placement model with schedule- and shape-dependent activation memory. Construct a bounded case where parameter-only screening accepts a plan that exceeds a declared training budget. Distinguish a simulated quota failure from an optionally observed physical CUDA OOM.

Part 2 - System and Memory Mental Model

Saved activations, copies across device boundaries, output merging, backward temporaries, and workspace can dominate weight bytes. Their lifetimes matter: adding isolated layer maxima is not an exact stage peak. In a serial forward/backward graph, many saved values can coexist across devices until backward consumes them. An application budget failure is deterministic evidence about the declared model, not proof of a hardware allocator failure.

Part 3 - Quantitative Prediction Before Execution

Define a per-unit activation function of batch size, sequence length, dtype, and checkpoint policy. Combine its intervals with persistent state, transport buffers, and workspace reserve. Construct equal parameter totals with a long-sequence or wide-intermediate activation spike on one device. Predict the first budget violation and a revised placement that reduces it. Keep the original prediction before executing the trace.

Part 4 - Experimental Design and Instrumentation

Validate the cost model on synthetic lifetime lists and a tiny training oracle. Use quota admission for the primary failure reproducer. Compare parameter-only and activation-aware plans on the same logical batch in fresh replays. Warm optimizer state, measure complete updates, and inspect the peak phase. If a physical OOM test is chosen, isolate it and restart cleanly after failure rather than continuing from partial updates. Do not retain diagnostic GPU tensors while measuring. Record unsupported workspace fields as unknown with a conservative planning allowance.

Construct one activation-heavy unit with small parameter payload and a large saved intermediate. Predict its peak using a lifetime interval rather than only its forward output shape. Include backward workspace and initialized optimizer state in the application quota. A parameter-only map should pass its own declared screening, then fail the more complete quota test at a named event. Preserve that distinction rather than calling the first heuristic incorrect for an input cost it never modeled. If real CUDA exhaustion is not observed, report the deterministic quota counterexample and the tested hardware range honestly. For failed training, no stage may be considered committed until the whole logical update is complete. Reconstruct from the last committed state before a new comparison. A revised placement must run the same model and samples, and its new transfer edges must be included in the peak and timing ledger rather than hidden outside measurement.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def estimate_activation_lifetimes(model_spec, shape, placement, schedule):
    ...


def predict_training_peak(persistent_state, lifetimes, workspace_bounds):
    ...


def compare_parameter_only_and_activation_aware(cases, evidence):
    ...


def E07(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report useful and backing activation bytes, predicted and measured peaks, quota/hardware failure type, failure phase, and update correctness. A failed attempt must not silently advance only some local optimizers. Preserve nonreproducing hardware outcomes honestly. Show which activation intervals invalidate the parameter-only decision and whether the revised plan introduces additional transfer costs.

Part 7 - Engineering Interpretation and Philosophy

Explain why parameter balance and training feasibility are different claims. Choose the minimum activation information needed for the next placement decision without pretending to know every backend allocation exactly.

19.44.9 Exercise 08

Exercise Title: Incorporate GPU Capacity into Placement Decisions and Balance a Model Across Devices with Different Available VRAM Budgets

Part 1 - Exercise Contract and Why It Matters

Incorporate heterogeneous available budgets into placement. Compare equal byte loading with normalized-pressure and slack-aware decisions while keeping logical training work constant. Support equal physical GPUs with explicitly simulated unequal application budgets as a separate controlled experiment.

Part 2 - System and Memory Mental Model

Physical capacity, current driver free bytes, allocator cached capacity, and application allowance are different quantities. External use can change headroom between sampling and allocation. Adding driver free bytes to all reserved bytes would double-count live allocations and still would not prove request fit. A device with more total VRAM can have less usable headroom. The placement must also fit its initialization and migration transient peaks.

Part 3 - Quantitative Prediction Before Execution

Choose two unequal budgets and an indivisible unit list. Derive feasible assignments and compare maximum normalized pressure with minimum byte slack. Include a target that has enough steady-state space but insufficient temporary space for construction. State how workspace reserve is included once. Predict a rejection when no device can admit the largest required unit.

Part 4 - Experimental Design and Instrumentation

Represent budgets as explicit bytes with source and timestamp: physical capacity, configured application quota, or measured contextual headroom. Validate the policy on synthetic heterogeneous cases. Use real differing capacities only if available, otherwise label the quota experiment accurately. Construct selected placements in fresh replays and measure initialized full updates. Keep the same model, batch, precision, optimizer, and initial state. Record per-device pressure, predicted slack, actual peaks, topology, and uncertainty margins. Do not manufacture a different GPU by silently changing the reported hardware identity.

Record physical capacity, configured application limit, sampled driver headroom, and reserve as separate fields. State exactly which quantity bounds the cost model. An artificial smaller application budget on one of two identical GPUs is useful for testing capacity-aware

logic but does not reproduce every performance property of a different GPU. Do not alter clock, architecture, or link claims to match the simulated capacity. Include initialization and transaction phases because a destination can fit the eventual model yet fail while old and new copies coexist. Test an indivisible unit larger than all budgets and require a clear infeasible result. Compare equal-byte, equal- pressure, and maximum-slack assignments on the same tiny inventory. The selected metric should match the desired next-phase feasibility, not merely aesthetic symmetry. Preserve uncertainty due to external allocations and avoid treating one driver sample as a durable promise of available capacity.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass


@dataclass
class DeviceBudget:
    device_identity: str
    physical_capacity_bytes: int
    application_limit_bytes: int
    workspace_reserve_bytes: int
    evidence_source: str
    sample_timestamp: object


def place_under_heterogeneous_budgets(units, budgets, cost_model):
    ...


def E08(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report physical and application capacities separately, every rejected candidate, initial and warmed peaks, and numerical errors. A normalized score below one is only meaningful within its accounted scope and reserve assumptions. Include a case where equal byte use overloads the smaller budget. Preserve unknown external activity rather than attributing a surprise allocation failure to the placement algorithm without evidence.

Part 7 - Engineering Interpretation and Philosophy

Choose a capacity-aware criterion for a named operating constraint. Explain why the least-used device in absolute bytes is not necessarily the best destination.

19.44.10 Exercise 09

Exercise Title: Measure Cross-GPU Activation Transfer Volume and Add Communication Cost to the Memory Placement Objective

Part 1 - Exercise Contract and Why It Matters

Measure cross-device activation and derivative traffic and add its cost to placement selection. Build an execution-edge ledger rather than counting only parameter bytes moved during construction.

Part 2 - System and Memory Mental Model

Every changed device edge may require activation copies in forward and derivative copies in backward. Noncontiguous tensors can add packing buffers, and skip edges can transmit more than one value. A topology link may permit peer access without delivering a guaranteed bandwidth. Logical payload is not measured bus traffic. Kernel launches, waits, staging, and overlap determine exposed communication time, which need not equal the sum of transfer durations.

Part 3 - Quantitative Prediction Before Execution

For each directed edge, calculate forward/backward payload from actual shape and dtype. Estimate transfer time under an explicitly measured or hypothetical bandwidth plus launch overhead. Compare a balanced alternating device map with a contiguous map. Predict added live copies and a case where avoiding one large activation boundary is more valuable than equalizing a small parameter imbalance.

Part 4 - Experimental Design and Instrumentation

Validate edge values and gradients with the serial oracle. Instrument cross-device copies in the single-process runtime, or P2P messages only in a separately validated distributed mode. Warm representative transfer sizes and measure completed dependent work. Record topology, peer-access capability, packing, stream dependencies, and payload. Use separate profiling and ordinary timing replays. Add communication as a latency estimate or constraint with compatible units; do not subtract milliseconds directly from byte relief. Preserve shared-link contention and unmeasured traffic as explicit uncertainty.

Build the edge ledger from the actual forward graph, including residual values and metadata that cross devices. A forward activation may need a reverse derivative even when its destination has no trainable parameters, so infer backward traffic from differentiation requirements rather than parameter ownership alone. Count packing buffers and source/destination copies under their actual lifetimes. Measure a representative tensor size and dtype instead of applying one tiny-transfer latency to all boundaries. Record whether peer access is supported, but keep that capability separate from measured bandwidth and end-to-end exposed time. A transfer can overlap compute yet remain on the critical path at a later consumer. Validate the consumer result before stopping a dependent-work timer. If host staging is used, include its RAM, pinning, and extra copy cost. Keep logical payload, estimated transport work, and actual physical traffic as three independently labeled quantities.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
```

```
@dataclass
class PlacementEdgeCost:
    producer: str
    consumer: str
    source_device: str
    target_device: str
    forward_payload_bytes: int
    backward_payload_bytes: int
    exposed_seconds_estimate: object


def build_placement_edge_ledger(graph, placement, shape_contract):
    ...


def E09(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export payload, packing capacity, completed transfer/dependent latency, overlap evidence, and total step time. Do not infer overlap solely from non_blocking flags. Check that the backward path is included in training traffic. If physical traffic is unavailable, label estimates by their algorithmic assumptions. A memory-only improvement with higher end-to-end latency is a valid trade-off that the objective must expose.

Part 7 - Engineering Interpretation and Philosophy

Explain communication as a consequence of graph cuts and ownership. Choose the edge whose placement most affects the measured critical path.

19.44.11 Exercise 10

Exercise Title: Compare Memory-Only Placement Against
Memory-and-Communication-Aware Placement on the Same Multi-GPU Model

Part 1 - Exercise Contract and Why It Matters

Compare a memory-only placement with a memory-and-communication-aware placement on the same model. Require both plans to satisfy the same resource budgets and numerical contract before judging latency or throughput.

Part 2 - System and Memory Mental Model

The most evenly loaded assignment can scatter adjacent layers and create large repeated transfers. A somewhat uneven assignment can be preferable if every device remains feasible and the critical path shrinks. Communication and memory costs interact through duplicate live boundary buffers. Compute imbalance is another independent contributor. Keep the objective explicit: minimizing normalized pressure and minimizing completed-step time are different goals, even when one is used as a constraint on the other.

Part 3 - Quantitative Prediction Before Execution

Enumerate a tiny model's feasible maps and calculate persistent load, activation peak, edge payload, and a stated latency estimate. Find a pair where a small pressure difference trades against a large boundary transfer. Define a throughput floor or an objective in seconds, with memory as a hard constraint. Predict the preferred candidate before timing.

Part 4 - Experimental Design and Instrumentation

Build both placements from identical logical state in fresh replays. Keep global batch, sequence length, dtype, optimizer, activation policy, and output collection fixed. Validate outputs, gradients, moments, and several updates. Profile representative copies and compute regions separately from ordinary timing. Warm initialization and optimizer state, then measure complete forward/backward/update regions on all devices. Report candidate search limits and any uncertainty in estimated edge bandwidth. Include a simple contiguous baseline so an alternating greedy map is not the only comparison.

Fix the finite candidate set and objective before observing its timing results. Use memory as a hard constraint with a named reserve and use seconds for a latency objective, or present a measured Pareto comparison without inventing a unitless weighted score. The communication-aware plan can remain slightly imbalanced if every phase fits. Include compute costs so moving a large layer to a slower device is not assumed free. Keep initialization and output collection comparable across plans. Validate gradients before Adam and compare moments after updates, since an optimizer can conceal some gradient- scale errors in a short parameter-only check. Report variability across fresh runs and preserve the device mapping. If the apparent speed difference is within measurement variability, state that the tested evidence does not resolve the ranking. A lower predicted communication estimate alone is not a measured throughput improvement.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def score_feasible_placement(placement, memory_model, edge_model, objective):
    ...


def compare_memory_and_joint_plans(replays, reference, evidence):
    ...


def E10(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export predicted and observed pressure, allocated/reserved peaks, communication payload, service times, complete latency, throughput, and numerical errors. Do not change batch size on only one plan and attribute the result to placement. A feasible joint plan that is slower

than expected needs an explained or unknown residual, not a fabricated improvement. Preserve both successful and rejected candidate IDs.

Part 7 - Engineering Interpretation and Philosophy

Choose a placement for the stated budget and throughput target. Explain why a balance score is a diagnostic rather than the entire engineering objective.

19.44.12 Exercise 11

Exercise Title: Build a DeviceMesh and Represent Independent Data, Tensor, and Pipeline Parallel Dimensions Without Manually Managing Every Process Group

Part 1 - Exercise Contract and Why It Matters

Represent independent data, tensor, and pipeline dimensions using a named DeviceMesh. Verify rank coordinates and subgroup membership before using those submeshes with supported parallelism APIs. Deliver the mapping, not an assertion that creating a mesh automatically parallelizes a model.

Part 2 - System and Memory Mental Model

A mesh organizes ranks and communication groups. Data groups connect replicas of equivalent logical model pieces; tensor groups cooperate within a layer; pipeline coordinates own different stage positions. The APIs still need explicit plans and compatible submeshes. Logical rank order does not guarantee physical link speed. Every rank must agree on the mesh definition and group creation sequence.

Part 3 - Quantitative Prediction Before Execution

For degrees D, T, P, require DTP ranks. Enumerate a hypothetical 2 by 2 by 2 grid and the groups obtained by varying one coordinate at a time. Explain why eight ranks are needed when all three degrees exceed one. With two devices, demonstrate only valid degenerate dimensions and retain the full-grid example as an offline mapping test.

Part 4 - Experimental Design and Instrumentation

Bind devices from local rank and validate unique membership, dimension names, product, and rank order before communication. Construct named submeshes through supported DeviceMesh interfaces and pass only the relevant dimensional view to each API. Export CPU mapping records and test a tiny rank-distinct group operation. Compare logical membership with available topology evidence. Record versions and supported composition modes. Do not create inconsistent meshes as a live negative test; validate malformed specifications offline instead. One multi-GPU host can provide the required ranks if enough devices are available.

Enumerate subgroup membership by fixing every coordinate except the one being varied. Verify that a data group connects equivalent logical pieces, a tensor group owns complementary feature pieces, and a pipeline coordinate represents ordered stage position. Keep sample semantics with the map: corresponding TP ranks cooperate on matching samples, while data replicas cover their intended distinct samples. Pipeline neighbors pass the same microbatch identities downstream. A mesh constructor does not establish any of these input contracts automatically. The full hypothetical three-dimensional example needs eight ranks when every degree is two; a two-device run must retain degenerate dimensions or test only one nontrivial dimension. Check that each parallelism API receives its supported submesh rather than the whole grid blindly.

Record rank order, visibility mapping, and versions so a logical coordinate cannot be confused with a physical CUDA ordinal in later memory rows or checkpoint metadata.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
from torch.distributed.device_mesh import init_device_mesh


@dataclass
class ParallelCoordinate:
    global_rank: int
    local_device: int
    data_coordinate: int
    tensor_coordinate: int
    pipeline_coordinate: int
    dimension_group_members: dict


def build_named_parallel_mesh(spec, launch_context):
    ...


def E11(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report complete membership, device bindings, subgroup sizes, capability status, and tested group results. A successful constructor does not prove correct input sampling or parameter ownership. Label unavailable nontrivial grids not measured rather than fabricating benchmarks. Verify no dimension is silently reused as an unrelated synchronization group.

Part 7 - Engineering Interpretation and Philosophy

Explain how a named mesh reduces bookkeeping while leaving parallelism semantics explicit. Identify which coordinate determines samples, tensor ownership, and stage position in the intended composition.

19.44.13 Exercise 12

Exercise Title: Combine Tensor Parallelism and FSDP on Separate Device-Mesh Dimensions and Measure the Resulting Per-GPU Memory Distribution

Part 1 - Exercise Contract and Why It Matters

Compose TP and FSDP on orthogonal named dimensions and measure per-rank memory distribution. Verify actual parameter placements at rest and during execution instead of assuming every component divides by the product of degrees.

Part 2 - System and Memory Mental Model

TP partitions layer computation; FSDP distributes state among data replicas of corresponding TP slices and temporarily gathers those slices for execution. Small replicated parameters, activations, communication buffers, and full TP-local gathers remain. Apply a version-compatible TP plan and fully_shard route in the supported order. These wrapped parameters are not eligible for the ordinary module migration path introduced later.

Part 3 - Quantitative Prediction Before Execution

Require at least four ranks for a nontrivial $D = 2, T = 2$ experiment. Enumerate both group families. Predict persistent large-parameter payload near its valid sharded extents and transient FSDP gathers of TP-local pieces. Add gradients, actual Adam state, activations, and padding by phase. Explain why two devices can provide controls but cannot demonstrate both dimensions larger than one.

Part 4 - Experimental Design and Instrumentation

Validate a tiny model and mesh before scaling. Match input samples within each TP group and distribute distinct intended samples across data groups. Derive the loss normalization and data reduction exactly once. Construct the optimizer after supported parallelization transformations. Compare outputs, gradient shards, moments, and several updates with a serial global-batch oracle. Record at-rest and transient local shapes, FSDP reshard policy, TP boundary placements, and initialization peaks. Profile communication in a separate replay and sample each rank's owned GPU only.

Audit the local physical representation at three boundaries: initialized persistent state, a parameter-gather window, and completed backward/update. A full parameter gathered inside an FSDP data group can still be one TP-local slice of the global matrix. Count it separately from the persistent shard when they coexist. Keep small replicated norm or bias parameters under their actual policy instead of dividing every byte by both degrees. Validate head/feature divisibility and TP output layouts before composition. Check data normalization with a tiny known-gradient global batch to detect double averaging. Only supported accumulation and reshard policies may be used in the benchmark. Include communication buffers and initialization copies in the cost comparison. If four devices are unavailable, the two-by-two mapping and analytical memory prediction remain useful offline checks but are not an empirical demonstration of orthogonal tensor and data sharding.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
from torch.distributed.tensor.parallel import parallelize_module
from torch.distributed.fsdp import fully_shard


def build_checked_tp_fsdp(model, named_mesh, tp_plan, fsdp_policy):
    ...


def audit_tp_fsdp_rank_storage(runtime, phase, logical_registry):
```

```
    ...


def E12(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export group maps, local storage by representation, peak phase, normalized pressure, collective ledger, latency, and numerical errors. Do not sum the same driver-wide device reading from several ranks. Preserve unavailable hardware or unsupported API combinations explicitly. A lower persistent shard total is not proof that gathered-state peaks are balanced.

Part 7 - Engineering Interpretation and Philosophy

Explain which redundancy each dimension removes and which transient asymmetry remains. Use measured rank distributions to identify the limiting phase rather than relying on a product-of-degrees memory formula.

19.44.14 Exercise 13

Exercise Title: Combine Pipeline Parallelism with FSDP and Diagnose Stage-Level Memory Imbalance from Uneven Parameters and Live Activations

Part 1 - Exercise Contract and Why It Matters

Combine pipeline stages with FSDP data groups and diagnose stage-level memory imbalance. Keep pipeline ownership and data-shard ownership distinct, and compare persistent versus live-activation bottlenecks.

Part 2 - System and Memory Mental Model

Pipeline coordinates own different submodels. Each stage's FSDP group connects equivalent submodels across data replicas, not different stages of one pipeline. Parameter gathers and gradient reductions follow compatible microbatch order within each data group. Saved activations depend on the pipeline schedule and stage position. Multiple local chunks add joint device residency. Do not add an independent DDP reducer to a path already using FSDP without a separately validated composition.

Part 3 - Quantitative Prediction Before Execution

For $D = 2, P = 2$, enumerate the four-rank groups and sample flow. Predict stage-specific persistent shards, full gathered stage units, communication buffers, and GPipe or flushed-1F1B activation sets. Compare two stages with similar persistent state but different live microbatch counts. Explain which parameter or activation component limits the worst rank.

Part 4 - Experimental Design and Instrumentation

Use an installed, supported FSDP/pipeline composition and static microbatch shapes initially. Validate the module/chunk boundary and optimizer construction order. Keep identical stage structures within each data group. Preserve sample/target identities along a pipeline and correct sampling across data replicas. Derive microbatch weighting and data averaging once, then validate several global updates against a serial oracle. Record schedule, reshard and accumulation policies, peak phase, outstanding gathers, and activation lifetimes. If four devices or

compatible support is unavailable, complete offline group and cost checks and label execution not measured.

Create a rank table with data coordinate and stage coordinate, then attach module identity and microbatch schedule. Equivalent FSDP replicas must own the same stage structure and execute compatible collective order even when their sample values differ. Different pipeline stages must not be placed in one data-sharding group as though their parameters were interchangeable. Use equal local batches and supported static shapes first. Derive the joint microbatch loss scaling and FSDP reduction convention before execution; pipeline stage count is not another averaging dimension. Trace full gathered units alongside saved microbatch graphs and their last readers. Earlier stages can have different outstanding activations under 1F1B, so persistent shards alone cannot predict balance. Keep stage/chunk wrapper construction and optimizer references under the installed API's contract. If a composition cannot be validated, reject it clearly and continue the primary ordinary-module balancing experiments without claiming distributed execution support.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
from torch.distributed.pipelining import PipelineStage, Schedule1F1B
from torch.distributed.fsdp import fully_shard


def build_checked_pipeline_fsdp(local_chunk, mesh, contracts):
    ...


def trace_stage_and_shard_residency(runtime, measurement):
    ...


def E13(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report rank-to-stage maps, shard and gather capacities, peak activation sets, memory pressure, update errors, and communication by group. Distinguish one stage's compute imbalance from its memory imbalance. A wrapper constructor success is not sufficient evidence of correct accumulated gradients. Unsupported reshard or accumulation paths must remain rejected rather than patched through private internal state.

Part 7 - Engineering Interpretation and Philosophy

Explain why sharding persistent state may leave pipeline activation imbalance unchanged. Choose a supported cut, schedule, or gather-policy change that targets the observed limiting phase.

19.44.15 Exercise 14

Exercise Title: Build a Unified Per-Rank Memory Cost Model for Parameters, Shards, Activations, Communication Buffers, and Temporary All-Gathered State

Part 1 - Exercise Contract and Why It Matters

Build a unified per-rank cost model covering logical parameters, persistent shards, gradients, optimizer state, saved activations, communication buffers, and temporary gathered representations. Predict complete phase peaks with explicit uncertainty.

Part 2 - System and Memory Mental Model

Different representations of one logical parameter can coexist physically: a local shard and a gathered TP-local computation tensor are separate allocations. Views into one backing allocation are not. Representation identity, ownership, dtype, and lifetime are necessary to reconcile both cases. Global tensor size does not describe local storage. Reserved allocator capacity and driver-wide use remain observations outside the exact tensor-payload sum.

Part 3 - Quantitative Prediction Before Execution

Define the predicted peak as the maximum over phase-specific unique live backing storage plus declared workspace bounds. Apply actual local extents instead of ideal divisions for uneven shards. Include initialization, optimizer update, and migration as separate phases. Construct a case where the persistent inventory fits but simultaneous gathers violate the budget, and another where aliases make a naive sum overestimate memory.

Part 4 - Experimental Design and Instrumentation

Inventory each representation by logical ID, storage ID, device/rank, placement, dtype, capacity, first use, and final-reader completion. Calibrate only named workspace terms on a small configuration and validate on held-out shapes and parallel modes. Feed actual schedule dependencies into the lifetime sweep. Keep unexplained or backend-specific terms as bounds or unknowns. Export predictions before measurement and preserve residuals by phase and rank. Avoid gathering a full global model for diagnostics in the capacity run; use a separate small correctness replay.

Give each physical representation a kind such as persistent shard, gathered computation copy, replicated small state, saved activation, or transport buffer. Preserve a common logical parameter ID while assigning different storage IDs to actual copies. Views share backing identity and need a last- reader rule before that capacity becomes releasable. Use a common event timeline for each device, including several local virtual stages where applicable. Calibrate workspace bounds on one shape and validate on another sequence length where activation memory changes. Keep original predictions immutable in the evidence so an underestimated phase can be audited. Model current live bytes and historical peak separately from reserved allocator capacity. For migration, include source copies retained for rollback and destination initialization temporaries. An attractive post-move inventory cannot establish that the transaction itself fits. Report uncertainty explicitly before the cost model is used as an admission decision.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass


@dataclass
class RankRepresentationCost:
    logical_identity: str
    representation_kind: str
    storage_identity: str
    rank: int
    capacity_bytes: int
    first_event: object
    last_reader_event: object


def predict_rank_phase_peaks(representations, workspace_bounds, schedule):
    ...


def E14(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report raw component bytes, predicted peak-contributing sets, measured peaks, residuals, calibration inputs, and uncertainty. Do not add all category maxima without a common timeline. Include per-rank rather than mean-only validation. A model that predicts a safe plan incorrectly must retain the failed prediction and failure phase. Distinguish application budget compliance from a guarantee against external allocations or unsupported workspaces.

Part 7 - Engineering Interpretation and Philosophy

Choose the smallest representation model that supports the next balancing decision. Explain which uncertainty instrumentation can reduce and which requires conservative reserves or explicit rejection.

19.44.16 Exercise 15

Exercise Title: Detect the Most Memory-Constrained GPU at Runtime and Migrate an Eligible Layer or Tensor Working Set to a Less-Loaded Device

Part 1 - Exercise Contract and Why It Matters

Detect the limiting device and migrate one eligible ordinary module or stateless working set at a completed update boundary. Preserve model values, optimizer state, references, and subsequent updates. Implement a transactional primary path in the single-process multi-GPU runtime.

Part 2 - System and Memory Mental Model

The limiting device is identified by predicted next-phase slack and required allocation size, not simply the largest current byte count. Migration can temporarily duplicate state. A trainable module needs parameters, buffers, Adam moments/counters, hyperparameters, and

execution-map references preserved. Calling module.to after optimizer construction is not a complete migration protocol. Wrapped DDP/FSDP/TP parameters, tied units, active graphs, pending readers, captured graphs, and hooks with untracked references are ineligible in the primary implementation.

Part 3 - Quantitative Prediction Before Execution

Predict source relief after old storage is released, target construction and migration peak, and changed future activation traffic. Include current source state retained for rollback until commit. Calculate a destination that fits steady state but cannot fit the transactional peak. Explain why source reserved bytes or driver free bytes need not improve immediately even when live allocated payload falls.

Part 4 - Experimental Design and Instrumentation

Use independent per-module Adam optimizers with identical declared hyperparameters and one shared logical update boundary; validate equivalence to the serial reference first. Drain forward/backward and transfers, complete updates, and clear gradients under the fixed policy. Reconstruct the eligible module on the target, copy values, and build its new optimizer referencing the new parameters. Map all initialized state by stable logical IDs, preserving counter dtype/device policy and hyperparameters. Validate copied state before atomically switching the application map. Retain the unchanged source until commit, then release old references after completion. Compare several subsequent updates with a no-migration reference.

Use an ordinary dense unit with no aliases, shared parameters, custom hooks, stateful randomness, scheduler dependencies, or captured execution in the first migration test. Its independent Adam optimizer uses fixed supported flags, initially without foreach or fused temporaries. Per-module optimizers must first match the intended global Adam update for this uncoupled case. Do not assume optimizer state_dict integer identifiers are stable logical parameter names; map state through the registry and validate shape, dtype, counter, and group options. Copy scalar state according to the optimizer's required device policy instead of moving every counter blindly to CUDA. Construct the destination unit without mutating the source, validate state, and switch the application registry only at commit. Failed preparation must leave the source optimizer and map usable. Subsequent training is the decisive check: compare gradients, moments, counters, and parameters after several updates, not merely the copied weight values immediately after migration.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def identify_limiting_device(samples, next_phase_prediction, budgets):
    ...


def validate_migration_eligibility(unit, runtime_state):
    ...
```

```
def prepare_migration_transaction(unit, destination, optimizer_registry):
    ...


def commit_or_rollback_migration(transaction, placement_registry):
    ...


def E15(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report logical state equality, optimizer parameter references, moments, counters, transaction peak, source relief, migration time, future traffic, and next-update errors. Test a failed preparation that leaves the source unchanged. Do not discard or reinitialize moments to make migration appear cheap. If a stateless scratch unit is chosen, prove no aliases or outstanding consumers require its old storage. Preserve ineligible status explicitly.

Part 7 - Engineering Interpretation and Philosophy

Explain the difference between moving bytes and transferring ownership of a trainable state. Choose migration only when correctness and transient capacity are both established.

19.44.17 Exercise 16

Exercise Title: Implement High- and Low-Watermark Rebalancing That Prevents Repeated Tensor Migration Between GPUs

Part 1 - Exercise Contract and Why It Matters

Add high and low watermarks, persistence requirements, and cooldown to reduce repeated migration. Validate behavior on synthetic pressure traces and actual bounded training without claiming hysteresis can prevent every possible move under arbitrary workloads.

Part 2 - System and Memory Mental Model

A high threshold can trigger investigation while a lower target provides separation after a move. Noisy samples, alternating shapes, stale driver headroom, and migration-induced peaks can otherwise cause oscillation. Watermarks must use the same named pressure scope and account for next-phase workspace. A destination must pass both migration-peak and future-load checks. Cooldown cannot override imminent infeasibility; the policy needs a documented reject, pause, or replan outcome.

Part 3 - Quantitative Prediction Before Execution

Define $0 \leq \ell < h < 1$ for a chosen budget-normalized signal, a minimum number of sustained observations, and a cooldown measured in completed updates. Construct traces near h, sustained above h, and alternating large shapes. Predict trigger, rejection, commit, and cooldown events. Include a case where no feasible move exists and ensure the policy does not repeatedly attempt the same impossible transaction.

Part 4 - Experimental Design and Instrumentation

Validate the state machine offline with missing and stale samples. Run the E15 eligible runtime using one unchanged workload first, then a controlled shape/pressure sequence. Make decisions only at drained update boundaries. Record pending migration, last commit generation, post-move target, and rollback state. Compare a threshold-only controller with the hysteretic controller using the same input trace and initial state. Include monitoring and migration overhead in end-to-end timing. Preserve numerical equivalence through every accepted move.

Use a pressure trace with observations alternating just above and below the high threshold, then a sustained increase, then a destination-pressure increase after a move. Predict the complete state-machine path before running training. Keep sample-age limits and minimum dwell time explicit. A move's own transient allocation should not be mistaken for a new unrelated pressure signal that immediately reverses it. Record pending and committed placement generations and reject stale recommendations. Test the no-feasible-destination case without repeated attempts on every update. If cooldown blocks a move but the next update is predicted infeasible, the runtime must choose a named pause/reject/replan path rather than proceeding optimistically. Compare total moved bytes and end-to-end useful throughput, not only move count. Fewer moves can still be worse if the controller remains stuck on a slow or infeasible placement.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch


@dataclass
class RebalanceWatermarks:
    high_pressure: float
    low_pressure: float
    sustained_sample_count: int
    cooldown_updates: int
    maximum_sample_age_seconds: float


def update_rebalance_state(policy, evidence, completed_update_id):
    ...


def E16(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export raw pressure, sample age, thresholds, state transitions, attempted and committed moves, rejected reasons, bytes moved, latency, and update errors. Verify suppression of the specific oscillation trace rather than asserting universal stability. Show a sustained infeasible

condition that produces an explicit blocked outcome. A cooldown that silently allows an invalid next update has not met the memory contract.

Part 7 - Engineering Interpretation and Philosophy

Choose hysteresis parameters from measured noise and migration cost. Explain the responsiveness-stability trade-off and the conditions that require replanning instead of another local move.

19.44.18 Exercise 17

Exercise Title: Build a Cost-Aware Rebalancer That Moves State Only When Expected VRAM Relief Exceeds Migration and Future Communication Cost

Part 1 - Exercise Contract and Why It Matters

Build a cost-aware rebalancer that first requires useful memory relief and feasibility, then evaluates migration time and future communication. Keep byte constraints and time objectives dimensionally separate.

Part 2 - System and Memory Mental Model

A move can release source live bytes yet slow every subsequent update through new device edges. It can also be essential because the old plan cannot execute the next shape. Compare time costs only over an explicit reuse horizon and account for uncertainty. A best-fit destination by current free bytes may be poor after incoming optimizer state and activations are included. Migration eligibility and rollback conditions remain those of E15.

Part 3 - Quantitative Prediction Before Execution

Require a minimum source byte relief and feasible source/target phase peaks. For a predicted horizon of H updates, compare migration time t_m with $H(t_{\text{old}} - t_{\text{new}})$ only when both plans execute the same useful work. Include future edge traffic in the new step estimate. If the old plan is infeasible, use an explicit capacity-first objective and throughput floor instead of pretending to measure its nonexistent completed latency.

Part 4 - Experimental Design and Instrumentation

Enumerate a bounded set of eligible module/destination candidates at a drained boundary. Use E14 for transaction and future peaks, E09 for changed edges, and measured migration costs from separate calibration. Reject candidates with insufficient evidence or excessive uncertainty. Compare always-move, watermark-only, and cost-aware policies on identical deterministic traces. Preserve predicted horizon and cost before each decision, then measure realized relief and latency. Include monitoring, failed preparation, rollback, and cooldown overhead. Validate subsequent parameter and optimizer updates for every accepted candidate.

Separate a capacity-restoring move from a performance-improving move. For the former, require a feasible next update and a declared throughput floor; the old infeasible plan has no measured completed latency for the same work. For the latter, choose a finite expected residence horizon and include migration preparation, optimizer-state copies, destination validation, new edge traffic, and monitoring overhead. Retain uncertainty bounds when the horizon or bandwidth is unknown. Reject a small theoretical gain that is within those bounds instead of pretending the score is exact. Compare predicted source relief with actual live-byte reduction only after old references are released. Reserved allocator capacity may remain cached, so do not declare the transaction ineffective solely from driver free memory. Conversely, do not

claim useful capacity from a lower balance score caused by allocating extra destination buffers. Preserve failed amortization examples as policy learning evidence.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def evaluate_move_candidate(candidate, memory_model, edge_model, horizon):
    ...


def choose_feasible_profitable_move(candidates, objective, uncertainty):
    ...


def compare_rebalancing_policies(replay_specs, reference, evidence):
    ...


def E17(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report predicted/observed source relief, transaction peaks, bytes copied, migration seconds, new step time, horizon, break-even status, and numerical errors. Do not compare gigabytes directly with milliseconds in an invented unscaled score. Preserve moves that failed to amortize their cost and explain whether horizon error, bandwidth uncertainty, or changed shapes caused the miss. A no-move decision is valid when evidence does not justify a transition.

Part 7 - Engineering Interpretation and Philosophy

Select a policy for a stated workload horizon and capacity requirement. Explain when preserving the current placement is better than improving a balance statistic through expensive migration.

19.44.19 Exercise 18

Exercise Title: Stress-Test Multi-GPU Balancing with Unequal GPU Capacities, Variable Sequence Lengths, Memory Spikes, Heavy Collective Communication, and Artificial VRAM Pressure

Part 1 - Exercise Contract and Why It Matters

Stress the balancer with unequal budgets, variable sequences, transient memory pressure, and communication contention. Preserve correctness, scoped failure classifications, and recovery evidence across both the eligible migration track and static distributed extensions.

Part 2 - System and Memory Mental Model

Artificial quotas, held allocation buffers, and injected communication delays model different phenomena. A held buffer consumes real device capacity; a quota changes application admission; a delay is not a measurement of another interconnect. Variable shapes can alter activation and kernel workspace costs without changing parameter bytes. Distributed collective order and ownership remain fixed unless a supported coordinated reconstruction is performed. Ordinary migration must not alter sharded wrappers behind their runtime.

Part 3 - Quantitative Prediction Before Execution

Define cases with equal physical GPUs and unequal application budgets, a larger next sequence, a bounded pressure buffer, and contention in a separately validated distributed mode. Predict accept, reject, migrate, or restart outcomes and failure phases. Calculate transaction peaks and next-update reserves. Include a pressure spike shorter than the expected migration amortization horizon and a case with no feasible destination.

Part 4 - Experimental Design and Instrumentation

Run deterministic small-model correctness first, then change one resource factor at a time. Keep seeds and logical sample IDs. Apply actual pressure buffers only within a predeclared bound. Use fresh groups for distributed configuration changes and coordinated shutdown after worker failure. Commit checkpoints at completed boundaries and compare resumed training with an uninterrupted reference. Test missing/stale samples and failed migration preparation offline before hardware. Record whether heterogeneity is physical or simulated. Preserve optional multi-dimensional experiments as not measured when devices or version support are unavailable.

Keep a table identifying whether each condition is real hardware heterogeneity, an application quota, a retained pressure allocation, an injected delay, or an offline scenario. Record the bytes and lifetime of any real pressure buffer and release it under the same controlled schedule across policy comparisons. Variable sequence lengths should retain the same valid-token loss semantics; padding capacity remains part of memory even when padding is excluded from loss. A heavy-collective extension needs compatible rank order and a verified parallel runtime before contention is introduced. Do not migrate an FSDP parameter through the ordinary path to escape a stress failure. Use committed checkpoint reconstruction for changes to distributed ownership. Minimize numerical failures by disabling adaptation, then returning to a static serial placement while preserving model values and samples. Save the minimal case, logical parameter registry, configuration generation, and failure phase so the result can be reproduced without retaining the original GPU graph.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def make_balancer_stress_cases(model_spec, runtime_capabilities):
    ...
```

```
def apply_bounded_pressure(case, device_inventory, budget_contract):
    ...


def run_balancer_stress_case(case, runtime, reference, evidence):
    ...


def E18(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export case configuration, physical/application capacities, shape, pressure mechanism, parallel mode, raw samples, predicted and observed peaks, moves, latency, numerical errors, outcome, and failure phase. Distinguish coherent rejection from successful execution. Preserve nonreproducing OOMs or spikes as experimental outcomes. Do not call every imbalance a leak or every failure fragmentation. Include the smallest numerical or ownership counterexample.

Part 7 - Engineering Interpretation and Philosophy

Explain which invariant each stress case tests and how its result changes supported scope or planning reserves. Prefer explicit refusal to an unsupported state migration that silently corrupts training.

19.44.20 Exercise 19

Exercise Title: Build and Benchmark a PyTorch Multi-GPU Memory Balancer That Dynamically Optimizes Placement Across Heterogeneous VRAM Budgets While Preserving Throughput

Part 1 - Exercise Contract and Why It Matters

Integrate validated monitoring, placement, cost prediction, and eligible rebalancing into an explicit PyTorch runtime. Benchmark correct full updates under heterogeneous byte budgets while controlling throughput regressions. Choose the best tested feasible policy without promising a universal optimum.

Part 2 - System and Memory Mental Model

Maintain separate execution modes: the primary mutable single-process model with eligible ordinary modules, and optional statically planned distributed TP/FSDP/pipeline compositions. They share cost/evidence schemas but do not share an unrestricted migration mechanism. The runtime needs logical parameter identity, optimizer ownership, placement generation, phase lifetimes, and explicit capability checks. A lower normalized spread is useful only when training remains correct and the selected memory/latency constraints hold.

Part 3 - Quantitative Prediction Before Execution

Predict a finite candidate set's per-device phase peaks, initialization and transaction peaks, edge traffic, compute costs, and adaptation overhead. Define hard budgets, uncertainty reserves,

minimum useful relief, and a throughput floor. Include equal-count, persistent-only, activation-aware, and communication-aware static baselines, then eligible adaptive policies. State which distributed degrees are actually available before planning tests.

Part 4 - Experimental Design and Instrumentation

Validate several serial-equivalent Adam updates from canonical state. Run correctness, profiling, timing, and capacity checks separately. Measure complete training plus adaptive monitoring and migration. Preserve E00's record and all budgets, sample scopes, thresholds, cost-model versions, eligibility decisions, and placement generations. Export CPU evidence. Validate committed restart against uninterrupted updates. Report unavailable or unsupported modes explicitly.

Publish a capability matrix distinguishing measurement scope, static placement, eligible module/scratch migration, hysteresis, cost selection, distributed compositions, and restart. Every supported entry needs a validating run ID. Preserve candidate predictions and rejections. Count useful samples once across model-parallel ranks under the actual data contract. Report construction, migration, and warmed peaks separately. Check committed parameter identities, optimizer progress, and placement generation before validating resumed updates. State the selected policy's throughput cost, search limits, and unsupported cases.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


class MultiGPUMemoryBalancer:
    def validate_runtime_mode(self, model_spec, capabilities):
        ...

    def construct_checked_placement(self, logical_state, plan):
        ...

    def run_training_update(self, batch):
        ...

    def rebalance_at_completed_boundary(self, evidence):
        ...

    def export_cpu_evidence(self):
        ...


def E19(config):
    # Integrate checked components and benchmark isolated candidates.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Keep prediction, observation, and interpretation in separate columns. Each run needs ID, configuration, shape, budgets in bytes, per-device allocated and reserved peaks, driver-headroom scope, elapsed seconds, useful throughput, numerical errors, outcome, and failure phase. Include adaptation costs, rejections, and failed predictions. Provide the smallest counterexample to any universal rule or state none was observed in the tested range. Distinguish simulated budget failures from real CUDA outcomes. Require validating run IDs for every claimed capability.

Part 7 - Engineering Interpretation and Philosophy

Justify the selected policy through ownership, correct updates, measured resource relief, and end-to-end cost. Identify untested shapes or device conditions that could change the decision.

19.45 DMM43

Chapter Title: CUDA Events and Precise GPU Timing
Specialization: Dynamic Memory Management and CUDA Systems

19.45.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Asynchronous CUDA Execution, Host Timing, Device Timing, Streams, and Event-Based Measurement

Part 1 - Exercise Contract and Why It Matters

Define a timing contract separating host submission, host-observed completion, device event intervals, and dependency stalls. Deliver a stream and event graph before interpreting any duration as operation latency.

Part 2 - System and Memory Mental Model

CUDA work is commonly queued asynchronously relative to Python. Stream order gives local sequencing; different streams need explicit dependencies when values are shared. An event marks progress at its recorded stream position, not the instant Python called record. Event intervals can include resource contention, stream waits, and host submission gaps between markers; they are not automatically exclusive kernel execution time. Host and device clocks have different meanings and cannot be subtracted as if they were one timestamp domain.

Part 3 - Quantitative Prediction Before Execution

Draw a host enqueue timeline and a device execution timeline for one matrix multiplication. Predict what an unfenced Python interval, a completed wall-clock interval, and a same-stream event pair include. Add unrelated stream work and a producer-consumer wait, then identify which completion boundary covers each dependency. State that event-pair elapsed time requires compatible same-device timing events and completed recordings.

Part 4 - Experimental Design and Instrumentation

Use a single CUDA device and one stream initially, fixed FP32 shapes, preallocated inputs, and checked numerical results. Record GPU identity, PyTorch/CUDA versions, driver, dtype, precision policy, stream IDs, shapes, seed, allocator backend, warmup, clock/load context when observable, and timing boundaries. Keep event creation and lazy initialization outside warmed

regions. Use monotonic host timing. Mark CUDA Graph capture and distributed clock alignment unsupported in the primary implementation until separately validated.

Create a synthetic timeline with a short host submission interval, a longer queued GPU operation, and a later host consumer. Add a second stream whose work is unrelated to that consumer. Label every number as a hypothetical example and derive which portions each timer observes. Then insert a host preparation gap after the start marker can execute but before the operation is enqueued. Explain why the resulting event interval can include that gap. This prevents the simplistic rule that all event times are pure GPU arithmetic and all wall times are the same arithmetic plus a fixed overhead. Define completion as a dependency property, not as the host having reached the next line of Python. Give each region an explicit unit, device, stream, generation, and validity state. Keep the primary experiment on one device with ordinary eager execution; any extension involving graphs, processes, or multiple devices must establish its own event-domain and completion contract before numbers are accepted.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch


@dataclass
class TimingContract:
    region_id: str
    device_identity: str
    stream_identity: str
    host_boundary: str
    device_boundary: str
    completion_rule: str
    warmup_policy: str


def validate_timing_contract(contract, dependency_graph):
    ...


def E00(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Preserve prediction, observation, and interpretation separately. Verify completion and values before accepting a sample. Unknown clocks, incomplete events, and unrun hardware measurements remain explicit. A small number is not evidence of fast GPU execution when it only measures host submission.

Part 7 - Engineering Interpretation and Philosophy

Explain which decision each timing scope supports. The goal is an honest measurement system whose dependency and completion boundaries can be audited.

19.45.2 Exercise 01

Exercise Title: Measure a CUDA Tensor Operation with Python Wall-Clock Time and Demonstrate Why Asynchronous Execution Produces Misleading Results

Part 1 - Exercise Contract and Why It Matters

Measure a CUDA operation with an unfenced Python wall clock and demonstrate what that number actually represents. Compare submission latency with independently established completion rather than assuming a small interval is execution time.

Part 2 - System and Memory Mental Model

A Python call can return while kernels remain queued. An unfenced interval can include dispatch, allocation, host-side preparation, and accidental blocking, so it is not guaranteed to be pure launch overhead either. First use may initialize CUDA or select kernels. Reading a CUDA scalar or copying output to CPU inside the interval can change the experiment by adding synchronization.

Part 3 - Quantitative Prediction Before Execution

Predict the host interval around a preallocated matrix operation with no completion wait. Draw the queued work that can remain after the timer stops. Give a counterexample where an implicit synchronization makes the interval longer than expected. Specify the numerical output check and completion wait that will occur only after the deliberately unfenced measurement.

Part 4 - Experimental Design and Instrumentation

Use a monotonic high-resolution Python clock, fixed operands, and a bounded workload. Run cold and warmed cases separately. Capture only the call interval for the unfenced case, then complete the work before validating output or recycling buffers. Avoid printing CUDA tensors or calling item inside the measured submission path. Compare against a completed reference in a separate replay with identical values and shapes. Record whether output allocation is included, and do not switch to an out variant on only one side. Drain outstanding work between isolated trials.

Compare a single isolated operation with a bounded loop that queues several operations without intermediate waits. The latter can eventually encounter host-side backpressure or allocation behavior that changes submission time. It is not a collection of completed request latencies. Keep the output policy explicit: replacing one result reference can still leave queued CUDA consumers that require storage safety. Retain the required buffers and complete them before cleanup. Record whether input creation, random generation, output allocation, and CPU validation occur inside or outside the host bracket. Use a known small result for correctness so checking it does not dominate the measured workload. If an unfenced call blocks unexpectedly, inspect the trace and surrounding operations before claiming CUDA stopped being asynchronous. A meaningful result can be a classification of what the host interval included, even when it does not display the anticipated large timing gap on the tested hardware. Preserve that outcome instead of increasing workload size without a predeclared bound.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import time
import torch


def measure_unfenced_host_interval(operation, evidence):
    # Preserve output lifetime; validate completion after the host interval.
    ...


def validate_after_completion(result, reference, device):
    ...


def E01(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report the host interval as submission-path time with its exact inclusions. Record completed latency separately, plus cold/warm status and result error. An unfenced measurement that happens to match completed latency is a result to investigate, not proof the operation is synchronous in every case. Do not allow work from one trial to inflate the next unnoticed. Keep failed or incomplete samples out of latency summaries without replacing them with zero.

Part 7 - Engineering Interpretation and Philosophy

Explain why the timer is not wrong; its boundary answers a different question. Identify the smallest change that would turn this measurement into completed operation latency.

19.45.3 Exercise 02

Exercise Title: Add torch.cuda.synchronize() Around Wall-Clock Timing and Compare the Result Against the Unsynchronized Measurement

Part 1 - Exercise Contract and Why It Matters

Measure completed CUDA work with device synchronization before and after a monotonic host interval. Compare with E01 while keeping operation, allocation policy, shapes, precision, and warmup identical.

Part 2 - System and Memory Mental Model

The pre-measurement synchronization drains earlier work on the selected device; the post-measurement synchronization ensures the bracketed work is complete. The measured wall interval includes host submission and the completion wait. A device-wide wait can also cover unrelated work in other streams, so its scope is broader than one operation. It does not synchronize every GPU in the machine merely because no stream was named.

Part 3 - Quantitative Prediction Before Execution

Write the required ordering: drain, start host clock, submit operation, wait for the selected device, stop host clock. Predict the bias if the first drain is omitted. Add an unrelated stream

and predict when the final wait would extend the interval. State whether the intended metric is isolated operation latency or completed workload latency on a busy device.

Part 4 - Experimental Design and Instrumentation

Start with one stream and no unrelated work. Use an explicit device argument for synchronization and a monotonic host clock. Measure bounded repeated trials after warmup, retaining outputs until complete. Validate values outside timing. Compare unfenced and fenced paths in independent replays or with complete draining between trials. Then introduce controlled unrelated stream work as a separate scope experiment. Record any intentional changes to normal overlap caused by boundary synchronization. Do not put a global wait after every layer when measuring a production overlapped pipeline.

Use an explicit selected device for both drains and verify that the operation actually executes there. A synchronization on another GPU does not complete this device's work. Keep the initial drain outside the timed interval, but report its existence and the isolated state it creates. If testing a busy-device scenario, launch the same finite unrelated workload in both comparison replays and record its dependency relationship. The final device wait can cover work that starts after the target operation was submitted, so name the resulting metric honestly. Do not insert a scalar correctness read before the intended final fence; that would add another completion path. Use a no-operation region as a diagnostic for clock and synchronization cost without assuming its value can be exactly subtracted from a real sample. When timing many updates, distinguish average completed throughput from single-update latency and keep input transfer and CPU batch preparation inclusions consistent across the runs.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import time
import torch


def measure_completed_wall_time(operation, device, contract):
    # Drain, time submission plus completion, and validate outside the region.
    ...


def compare_fenced_unfenced(replay_specs, evidence):
    ...


def E02(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report explicit device, pre-drain policy, measured inclusions, host time, and numerical status. Check for earlier queued work and other-stream interference before interpreting discrepancies. Synchronized timing deliberately alters scheduling; distinguish an isolated benchmark

from normal application throughput. An empty-region control estimates overhead but should not be blindly subtracted from every sample as an exact correction.

Part 7 - Engineering Interpretation and Philosophy

Choose when completed wall time is the appropriate metric. Explain why its broader scope can be useful even after event timing becomes available.

19.45.4 Exercise 03

Exercise Title: Create torch.cuda.Event(enable_timing=True) Start and End Events and Measure the Execution Time of a CUDA Matrix Multiplication

Part 1 - Exercise Contract and Why It Matters

Use timing-enabled CUDA start/end events around a matrix multiplication on a declared stream. Wait for the end recording before reading elapsed milliseconds and validate the operation separately.

Part 2 - System and Memory Mental Model

Events are recorded into a stream's ordered work. Creating an event object does not record a timestamp. The pair must have timing enabled, compatible device ownership, and the intended record order. Event resources may initialize lazily. elapsed_time computes a device interval; it is not a completion-polling substitute. A same-stream pair brackets intervening stream progress, including waits or contention that occur there.

Part 3 - Quantitative Prediction Before Execution

Predict the ordering of start marker, multiplication, end marker, end completion, and elapsed-time read. Convert reported milliseconds to seconds explicitly. For very short work, predict why one sample can be dominated by instrumentation or resolution. Derive a repeated-operation average as a separate throughput-like measurement, not a distribution of single-call latency.

Part 4 - Experimental Design and Instrumentation

Preallocate fixed operands and initialize event resources outside the warmed measurement region. Select the device and stream explicitly. Record start, enqueue the operation, record end, and establish end completion before reading the pair. Keep operands/output alive through their consumers. Use fixed dtype and precision settings, record layout/contiguity, and check output against a trusted small reference after timing. Compare a single multiplication with a bounded repeated region, documenting whether output allocation and buffer reuse differ. Do not add CPU validation inside the event-bracketed launch sequence.

Record both markers with an explicit stream argument or a verified stream context and store that stream identity in the sample. Warm their underlying resources with a completed diagnostic recording before reusing the pair for measurement. Keep event generation changes visible in the registry. Use a matrix shape whose operation is large enough to interpret, but do not claim a universal minimum measurable size. If deriving achieved arithmetic throughput, state the matrix multiplication FLOP convention and divide only by a valid completed interval. Preserve dtype, layout, TF32 or other precision settings, and output-allocation choice with the result. A repeated region can amortize marker cost, but its time divided by iteration count is an average over that region rather than independent single-call samples. Compare both designs

only under their declared meaning. Result validation occurs after completion and must not leave an extra live reference model or output that changes later memory-pressure experiments.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def create_timing_pair(device, stream):
    ...


def measure_matmul_with_events(inputs, outputs, timing_pair, stream):
    # Recording, completion, elapsed read, and unit conversion remain work.
    ...


def E03(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export event generation, device/stream, record order, completion status, elapsed milliseconds, converted seconds, shape, dtype, and result error. Reject non-timing, unrecorded, mismatched-device, or unfinished pairs explicitly. A zero-valued tiny interval is not automatically a failed event; assess resolution and region size without fabricating positive time. Preserve raw samples before aggregation.

Part 7 - Engineering Interpretation and Philosophy

Explain the precise device interval the pair observes. State which host or application costs remain outside that measurement.

19.45.5 Exercise 04

Exercise Title: Compare CUDA Event Timing Against Synchronized Python Wall-Clock Timing and Explain the Difference Between GPU and End-to-End Latency

Part 1 - Exercise Contract and Why It Matters

Compare event timing with completed Python wall-clock timing for the same CUDA region. Explain their different boundaries without requiring either number to equal the other or treating their difference as an exact overhead decomposition.

Part 2 - System and Memory Mental Model

A host interval can include dispatch, waiting, and Python work outside the device markers. An event interval can include host submission gaps after the start marker executes, stream dependencies, and resource contention. Neither automatically reports exclusive kernel duration.

Different replays can experience different clocks, load, or allocator behavior. A fair comparison needs matched region semantics and an explicit completion rule.

Part 3 - Quantitative Prediction Before Execution

Draw host and stream timelines with both clock pairs. Mark event creation, recording, CPU preparation, GPU work, and final synchronization. Predict a case with extra host preparation and another with a stream wait inside the event pair. Explain why subtracting event time from host time cannot uniquely identify Python overhead or PCIe time.

Part 4 - Experimental Design and Instrumentation

Use identical fixed operands, output-allocation policy, precision, warmup, and number of operations. Collect a nested measurement only after naming exactly what the host bracket includes; also use independent replay comparisons to assess instrumentation effects. Wait on the end event or selected completion boundary before the host timer stops. Validate results outside the region. Record external load when observable and repeat enough trials to show variability. Keep profiler-based exclusive kernel analysis in a separate instrumented replay rather than equating it with event-pair elapsed time.

Build one measurement in which the host bracket starts before both event records and stops after end-event completion. List the event-record API and synchronization costs that the host interval includes. Then build another contract that starts host timing after CPU data preparation, showing why an end-to-end claim depends on where the application boundary is placed. Keep these comparisons descriptive rather than forcing a fixed wall-minus-device identity. Concurrent GPU load can change the event interval even though the same kernel code is launched. Host scheduling can also delay enqueueing the end marker after the start has executed. Use separate instrumented traces when attributing a discrepancy to either cause. Report both distributions with identical shape and precision metadata. If two scopes are measured in different replays, do not pair their sample-by-sample differences as if they shared the same execution history. Preserve uncertainty where the available evidence cannot isolate a component.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import time
import torch


def measure_host_and_device_scopes(operation, stream, timing_contract):
    ...


def compare_timing_scopes(sample_rows):
    ...


def E04(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report both raw durations, boundary definitions, completion method, variability, and correctness. Flag negative inferred overhead or inconsistent scope as a diagnostic rather than clamping it silently. A mismatch may be expected from the boundaries; explain it with the timeline and preserve remaining uncertainty. Do not claim events exclude all host effects or all unrelated GPU interference.

Part 7 - Engineering Interpretation and Philosophy

Choose a metric for a kernel-region question and one for user-visible completed latency. Explain how both can be valid while answering different engineering questions.

19.45.6 Exercise 05

Exercise Title: Demonstrate Why Reading elapsed_time() Before the End Event Completes Can Fail or Produce Invalid Timing Assumptions

Part 1 - Exercise Contract and Why It Matters

Demonstrate the completion precondition for elapsed_time without assuming that an early read must always fail. Build explicit recorded/pending/complete states and accept timing only after the correct end recording completes.

Part 2 - System and Memory Mental Model

Short GPU work can finish before Python attempts the read, so an immediate call may succeed. Pending work can instead produce a not-ready error or another reported API failure. Neither outcome authorizes treating elapsed_time as a poll. Unrecorded events, disabled timing, and stale generations are separate invalid conditions. Query or synchronization establishes completion of a recorded event, while application state establishes that the intended record actually occurred.

Part 3 - Quantitative Prediction Before Execution

Predict the possible outcomes of immediate reads for short and longer bounded regions. Draw a case where a reused event object refers to a newer recording than the sample expects. Define which states are valid for reading and which must be rejected or deferred. Explain why a zero duration is not an acceptable placeholder for pending work.

Part 4 - Experimental Design and Instrumentation

Use fresh timing pairs or a generation-checked pool. Submit a controlled bounded workload, record the end, and attempt an early read only in the isolated diagnostic test. Capture the observed status rather than requiring a particular error. Then establish end completion and read again through the valid path. Test unrecorded and timing-disabled misuse in a metadata/API validation case. Avoid intentionally launching invalid kernels. Keep operand and event lifetimes intact, and record the installed versions and exact error classification without manufacturing a reproducible race.

Track event creation, first record, latest record generation, completion, and result consumption separately. A newly created event has not measured a region even if a completion query reports no pending work. An end event that belongs to a different generation must not satisfy an older sample's read precondition. Test those metadata cases deterministically before the hardware diagnostic. For an actual early read, classify success, not-ready, invalid configuration,

or another reported error without requiring a particular race outcome. Some asynchronous failures can surface at a later API call; retain the launch context and do not automatically blame the timing read for causing the original failure. After a valid completion, compare repeated reads before any re-record to check stable sample identity. Do not invent negative or random timing values to illustrate premature access. The educational failure is an invalid assumption about readiness, whether the particular attempt happened to finish early enough to succeed or not.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch


@dataclass
class TimingPairState:
    generation: int
    start_recorded: bool
    end_recorded: bool
    completion_status: str
    timing_enabled: bool


def read_completed_elapsed(pair, expected_generation):
    ...


def E05(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report attempted state, query result where meaningful, early-read outcome, completed elapsed time, and generation. A successful early read means the precondition happened to hold; it does not prove implicit waiting. Reject stale, unrecorded, or incomplete samples rather than converting failures into latency values. State if a pending-read failure was not observed within the bounded workload range.

Part 7 - Engineering Interpretation and Philosophy

Explain why a timing API and a completion API have different responsibilities. Choose an invariant that makes the instrument correct regardless of host/GPU relative speed.

19.45.7 Exercise 06

Exercise Title: Synchronize Only the End Event Instead of the Entire CUDA Device and Measure the Reduction in Unnecessary Global Blocking

Part 1 - Exercise Contract and Why It Matters

Compare waiting for a recorded end event with synchronizing the whole selected device. Measure host blocking scope while preserving correctness of the task and all buffers still used by unrelated work.

Part 2 - System and Memory Mental Model

An event wait covers work causally preceding that recording, including explicit dependencies it inherits. It need not wait for independent later or other-stream work. A device-wide synchronization has broader scope. Event synchronization blocks host progress until that event completes; it is different from enqueueing a stream dependency. Narrower scope does not guarantee lower measured time when resource contention or workload ordering already couples the streams.

Part 3 - Quantitative Prediction Before Execution

Construct a target task and a longer bounded independent background task on separate streams. Predict which work is required before target output can be consumed and which is required before all buffers can be released. Draw event-only and device-wide host wait intervals. Include a case where an explicit target dependency makes both waits cover the same work.

Part 4 - Experimental Design and Instrumentation

Preallocate disjoint buffers, establish their initial readiness, and warm event resources. Launch the same task set in separate replays for the two completion methods. Stop the target-latency host timer at its declared boundary, then drain background work outside that region before cleanup. Validate both outputs. Use profiling separately to inspect contention and unintended dependencies. Keep measured device, stream mapping, task sizes, and precision fixed. Do not place a hidden device-wide synchronization in the event-only path before stopping its timer.

Specify that background work is independent both mathematically and in storage: no shared mutable outputs, no reused scratch buffer, and no hidden readiness edge from the target to the background completion. Warm and prepare both streams before the trial. Keep the same finite task set across event-only and device-wide replays, and measure the target host wait at the same point in submission order. After stopping the target interval, retain background buffers until a separate cleanup fence proves them safe. Add a dependency control where the target deliberately waits for the background and predict why event synchronization now inherits that work. Actual resource scheduling can serialize independent kernels, so a narrower logical scope may show little or no latency improvement. Report that observation. Do not shorten the background workload only for the device-wide comparison or let its unfinished work leak into a later trial. The comparison should demonstrate scope and any measured benefit, not guarantee an improvement by definition.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import time
import torch
```

```
def compare_event_and_device_completion(task_set, device, evidence):
    ...


def drain_remaining_work_for_cleanup(task_set):
    ...


def E06(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report target completion latency, background completion, host blocked time, dependency graph, and errors. Distinguish a scope reduction from an actual observed speedup. If the background finishes first or serializes with the target, report no reduction rather than altering the result. Verify no background storage is reused while still in flight after the target event has completed.

Part 7 - Engineering Interpretation and Philosophy

Choose the narrowest completion boundary that proves the intended consumer safe. Explain why finishing one task does not imply the whole device is idle.

19.45.8 Exercise 07

Exercise Title: Use Event.query() to Poll Completion Without Blocking the CPU Thread and Build a Non-Blocking GPU Task Monitor

Part 1 - Exercise Contract and Why It Matters

Build a bounded nonblocking completion monitor using Event.query and application-owned event state. Allow the CPU to perform other work between polls and export completed timing records without waiting in every monitoring call.

Part 2 - System and Memory Mental Model

Query reports captured-work completion without waiting for it. It does not prove an intended task was submitted; an unrecorded event must be rejected by the monitor's own state. Polling consumes CPU/API time and detects completion only at sampling instants. A task can finish between polls, so host detection latency differs from device elapsed time. Re-recording pooled events before consuming their samples can corrupt task identity.

Part 3 - Quantitative Prediction Before Execution

Define pending, completed, consumed, failed, and retired states with generation IDs. Predict the detection delay bound for a fixed polling cadence under stated host-scheduling assumptions. Compare frequent polling with a coarser cadence in CPU overhead and readiness delay. State which quantity comes from device events and which from host poll timestamps.

Part 4 - Experimental Design and Instrumentation

Submit bounded tasks with unique task IDs and event generations. Poll only recorded ends, return promptly for pending tasks, and perform a small CPU-side work unit between

polling passes. Read elapsed time only after the matching pair is complete. Retain task buffers through final consumers and retire event leases only after record consumption. Implement bounded pending capacity and explicit backpressure. Compare against a blocking baseline in separate replays. Include timeout as a monitor outcome, not permission to free storage still used by CUDA.

Give the monitor a maximum number of pending records and a policy for capacity exhaustion, such as rejecting new instrumentation or applying explicit backpressure at a named boundary. Never overwrite a pending pair to maintain a prettier queue length. Bound each polling pass by task count or CPU time and return control to useful host work. Detection latency includes both polling cadence and host scheduling delay; it is not the device's execution duration. Record host submission, first observed completion, and collection timestamps alongside the event duration without subtracting across clock domains. A timed-out task remains an owner of its buffer lease until completion or safe process recovery. Test this invariant offline. Compare monitor CPU overhead, pending high-water mark, and completed useful throughput against the blocking baseline and an uninstrumented run. If the host has nothing useful to do, frequent polling may simply create busy waiting; state that behavior rather than calling it free asynchronous progress.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch


@dataclass
class PendingTimedTask:
    task_id: str
    generation: int
    start_event: object
    end_event: object
    recorded: bool
    buffer_lease: object


def poll_completed_tasks(pending, host_timestamp, budget):
    ...


def E07(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report polling cadence, CPU overhead, device duration, host detection delay, pending high-water mark, dropped/rejected submissions, and correctness. Do not replace pending with zero elapsed time. Test stale generations and unrecorded tasks offline. A timeout must preserve safe

cleanup or explicit process recovery. Measure monitor overhead against an uninstrumented workload rather than calling query cost-free.

Part 7 - Engineering Interpretation and Philosophy

Explain how nonblocking monitoring trades detection latency against CPU work. Choose a bounded policy suitable for feeding scheduling decisions without turning the monitor into a busy-wait benchmark.

19.45.9 Exercise 08

Exercise Title: Record Events on a Custom CUDA Stream and Verify That Timing Measures Work Executed in the Intended Stream

Part 1 - Exercise Contract and Why It Matters

Record timing events on a custom CUDA stream and verify that the measured region contains the intended work. Preserve input readiness, output consumption, and allocator lifetime across the default/custom-stream boundary.

Part 2 - System and Memory Mental Model

A stream context changes where eligible operations are enqueued, but an event recorded on another stream brackets different progress. Data created on the default stream may need an explicit dependency before custom-stream use. A later default-stream consumer also needs the correct completion edge. Ordering readiness and preventing premature allocator reuse are separate obligations; record_stream is not a substitute for a producer-consumer wait.

Part 3 - Quantitative Prediction Before Execution

Draw default-stream input production, custom-stream wait, start event, operation, end event, and consuming-stream wait. Predict which waits belong inside the timing interval under two declared contracts. Construct a wrong- stream timer example that produces a plausible duration without enclosing the target operation. Identify the validation that rejects it.

Part 4 - Experimental Design and Instrumentation

Select device and stream explicitly and preinitialize events. Use a correct readiness dependency before computation, then record both markers on the intended stream. Preserve buffer lifetime through explicit leases until completion or a validated allocator stream-recording policy. Transfer output readiness to its consumer stream before numerical checks. Run a metadata negative control with mismatched recorded-stream identity instead of depending on a race to reveal the bug. Compare with a serialized correct reference and inspect a separate profiler trace.

Make input production and first custom-stream use visible in the dependency graph. If a tensor was allocated or filled on another stream, establish the required readiness before recording the measured operation. Decide whether that initial wait belongs inside the requested metric. On output handoff, the consumer needs its own readiness edge even when the Python function has already returned. Keep explicit strong references until all consumers complete in the primary implementation; use record_stream only under a separately validated lifetime policy. Recording allocator use does not order producer and consumer values. Compare a correctly timed worker region with an offline wrong-stream annotation that the validator rejects. A tiny wrong-stream interval might look plausible, so numerical correctness of the operation alone

cannot validate the timer's placement. Export current-stream metadata at recording points and restore application stream context after instrumentation. Avoid an instrumentation helper that silently moves the operation onto the default stream and thereby changes the workload it claims to measure.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def prepare_custom_stream_inputs(inputs, producer_stream, worker_stream):
    ...


def time_on_declared_stream(operation, worker_stream, timing_pair):
    ...


def handoff_completed_output(output, producer_event, consumer_stream):
    ...


def E08(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export event/operation stream IDs, dependencies, timing scope, completion, result errors, and buffer release conditions. Reject a pair that never encloses the claimed work. A correct value in one run does not prove missing dependencies safe. Record whether wait time is included or excluded by marker placement. Do not infer a cross-stream dependency from Python statement order alone.

Part 7 - Engineering Interpretation and Philosophy

Explain how stream placement defines measurement meaning. Separate value readiness from storage-lifetime safety in the final interpretation.

19.45.10 Exercise 09

Exercise Title: Measure Independent Operations on Two CUDA Streams and Determine Whether Their Execution Actually Overlaps

Part 1 - Exercise Contract and Why It Matters

Measure two independent stream workloads and test whether they actually overlap. Compare serialized and concurrent execution with a common same-device fork/join timing boundary and separate branch measurements.

Part 2 - System and Memory Mental Model

Two streams permit concurrency but hardware resources, kernel occupancy, copy engines, shared memory bandwidth, and launch behavior can prevent it. Adding branch durations does not give concurrent makespan. Establish a common start dependency and join both recorded branch completions before the common end marker. Cross-stream event endpoints on one device are meaningful only under an explicitly defined interval; events from different GPUs cannot be used as one elapsed-time pair.

Part 3 - Quantitative Prediction Before Execution

Let isolated branch durations be a and b. Compare ideal concurrent makespan $\max(a, b)$ with serialized $a + b$ under no contention and negligible overhead. State why actual branch durations may change when co-running. Predict fork/join dependencies and an example where contention makes the concurrent schedule no faster. Distinguish temporal overlap from throughput improvement.

Part 4 - Experimental Design and Instrumentation

Use disjoint preallocated inputs and outputs with valid initial readiness. Record a common start on a coordinator stream, make workers wait for it, record branch regions, and join their end events before the coordinator end. Wait for the common end before reading durations. Compare with an equivalent serialized replay and inspect a separate execution trace for actual concurrency. Keep precision and operation counts fixed. Validate both outputs and preserve buffer leases. Use multiple workload sizes rather than asserting every pair of matrix multiplications can overlap effectively.

Use a coordinator start marker as the common same-device anchor. Worker streams wait for that recording, then produce their own start/end records; the coordinator waits for both ends before recording the makespan end. Check the graph has no cycle and that all required work lies between the anchor and join. This arrangement supplies causal interval boundaries without assuming Python launch order serializes the workers. Calculate relative branch start/end offsets only from compatible same-device completed event recordings. If a profiler provides a timeline, keep its clock interpretation separate from host timestamps. Compare serialized and concurrent results for both small and resource-heavy operations. Concurrent branch durations can lengthen through contention, so do not compute speedup from isolated durations alone. Preserve the fixed useful operation count and buffer capacity across trials. A claim of overlap should cite actual interval/trace evidence, and a claim of speedup should use completed makespan or useful throughput; neither claim follows automatically from the other.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def build_same_device_fork_join(tasks, worker_streams, coordinator):
    ...


def compare_serialized_concurrent(replay_specs, measurements):
```

```
    ...


def E09(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report isolated and concurrent branch durations, common makespan, completion graph, trace evidence, result errors, and variability. Do not infer overlap from a nonblocking launch or two stream objects. A shorter concurrent makespan is useful evidence but still needs boundary validation. Preserve no-speedup or slowdown outcomes and identify observable contention without inventing hardware causes.

Part 7 - Engineering Interpretation and Philosophy

Explain which resource or dependency limits observed concurrency. Choose whether overlap is worth retaining for the tested workload and measurement objective.

19.45.11 Exercise 10

Exercise Title: Build Cross-Stream Dependencies with CUDA Events and Make One Stream Wait for Another Without Calling Device-Wide Synchronization

Part 1 - Exercise Contract and Why It Matters

Build producer-consumer stream dependencies with events while allowing the host to continue. Verify correct values and timing scope without inserting a device-wide synchronization into the dependency path.

Part 2 - System and Memory Mental Model

A consumer stream waiting for an event delays its future queued work; it does not wait for that work to finish on the CPU. The producer must have recorded the intended event generation before the wait is established. Dependencies do not automatically keep every buffer alive. A timing start before the consumer wait includes dependency delay; a start after that wait measures a different interval. Neither alone isolates all resource-related stall causes.

Part 3 - Quantitative Prediction Before Execution

Draw producer computation, event record, consumer wait, and consumer computation. Predict host submission order and device execution order. Place timing markers before and after the wait to define two distinct metrics. Add independent host work and state when the host must eventually fence before reading results or reusing storage.

Part 4 - Experimental Design and Instrumentation

Use a producer that fills a known tensor and a consumer that transforms it. Establish the event wait explicitly, retain storage through the last consumer, and verify against a serialized reference. Test stale or unrecorded event generations in the validator. Compare host submission and completed latency in separate regions. Drain for cleanup only after the measured asynchronous path. Capture a profiler trace to distinguish waiting, computation, and interference. Keep all event timing pairs on compatible same-device recordings even if an optional cross-device wait is supported.

Record the producer event before enqueueing the consumer wait for that generation. Re-recording the event later must not be assumed to update an already established dependency to a new task. Store generation and task ID with both ends of the edge. Keep a validation case where the consumer waits for an old completed recording and reject it before numerical benchmarking. Measure a consumer interval beginning before the wait and another beginning after it, with the difference interpreted only under the controlled experiment's assumptions. Resource contention or delayed host submission can contribute to those intervals too. The CPU can proceed after scheduling a wait, but any later CPU output read still requires the appropriate completion fence. A safe output handoff can therefore be asynchronous without being unconditionally ready. Preserve buffer ownership through the consumer's last use and keep cleanup waits outside the target host-submission region. Do not remove the wait merely because a race failed to reproduce in a short run.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def enqueue_producer_consumer(producer, consumer, buffers, event_lease):
    ...


def measure_consumer_wait_inclusions(plan, timing_contract):
    ...


def E10(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report dependency generations, host return time, consumer inclusive and post-wait intervals, completion boundary, and result error. Verify no device-wide wait is hidden in the measured path. A difference between intervals is only a scoped wait estimate with stated interference assumptions; do not label it a complete causal stall decomposition. Preserve safe buffer cleanup after the host has returned.

Part 7 - Engineering Interpretation and Philosophy

Explain the distinction between scheduling a wait and blocking the host. Choose marker positions that answer the intended readiness or compute-region question.

19.45.12 Exercise 11

Exercise Title: Measure Host-to-Device Transfer Latency with CUDA Events and Compare Pageable and Pinned Memory Under Identical Tensor Sizes

Part 1 - Exercise Contract and Why It Matters

Measure H2D copies from pageable and pinned host storage using matched sizes, contents, destination reuse, and stream boundaries. Report both host submission/completion and device event intervals, with pinning cost separately identified.

Part 2 - System and Memory Mental Model

Pinned storage can support asynchronous transfer paths, but performance and overlap depend on hardware, driver, size, and execution conditions. Pageable sources can involve staging and host blocking. An event interval around a copy can reflect host submission gaps or staging-related delay and must not be called pure DMA time automatically. Pre-pinning outside a region answers a different question from pinning inside an end-to-end transfer path.

Part 3 - Quantitative Prediction Before Execution

Calculate useful bytes from element count and dtype. Predict which costs belong to preallocated-copy, pin-and-copy, and completed end-to-end contracts. Draw start/copy/end enqueue order for a host-blocking staging case and explain why GPU markers need not remove host effects. Predict buffer lifetime requirements until the copy completion event.

Part 4 - Experimental Design and Instrumentation

Create equal-valued pageable and pinned sources, preallocate CUDA destinations, and warm event resources. Measure copy_ into existing storage with identical stream and nonblocking policy where supported. Keep a separate pin-and-copy replay that includes host allocation/pinning. Never mutate or release the pinned source while a copy can still read it. Wait for the end before validating copied values. Measure isolated transfer latency first, then separately test overlap with computation. Record host submission duration, completed wall time, device interval, and pinned capacity.

Separate three cases: already-pinned source to preallocated CUDA destination, pageable source to the same kind of destination, and explicit pin-then-copy including preparation. Record the actual source pin status rather than relying on its variable name. Match tensor size, dtype, contiguity, values, stream, and destination reuse. Keep source data unchanged until the copy completion condition holds; a nonblocking request is not permission for immediate reuse of pinned host storage. Pageable staging can delay host submission and influence the time between GPU markers, so describe the device interval as the observed bracketed copy path rather than asserting a clean DMA-only measurement. Validate transfer correctness after the fence with a bounded CPU comparison. When testing overlap later, use independent compute and explicit ready events instead of inferring overlap from the faster isolated transfer. Include pinned capacity and pinning cost in the runtime's resource accounting even if those costs are excluded from the reusable-copy latency contract.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import time
import torch


def prepare_matched_host_sources(shape, dtype, seed):
    ...
```

```
def measure_h2d_copy(source, destination, stream, contract):
    ...


def E11(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export source pin status, bytes, destination reuse, allocation/pinning inclusions, stream, nonblocking request, all timing scopes, and result error. A flag does not prove overlap; use dependency and trace evidence. Do not compare an allocating to call with a preallocated copy on the other path. Preserve cases where pinned memory is slower for a small transfer. State when staging or physical bus traffic cannot be directly observed.

Part 7 - Engineering Interpretation and Philosophy

Explain which transfer contract matters to the memory-aware runtime. Choose whether pinning is reusable preparation or part of its repeated critical path, and report that choice clearly.

19.45.13 Exercise 12

Exercise Title: Measure Device-to-Host and Device-to-Device Copy Latency Separately and Convert Transfer Time into Effective Bandwidth

Part 1 - Exercise Contract and Why It Matters

Measure D2H and same-device D2D copies separately and convert completed copy intervals into useful-payload bandwidth. Treat cross-GPU peer copies as an optional separately scoped extension.

Part 2 - System and Memory Mental Model

A pinned D2H destination may remain unreadable until its transfer completes. Pageable behavior does not remove the need for an explicit correctness boundary. Same-device D2D uses device memory resources rather than PCIe by default. Useful payload per second differs from a read-plus-write traffic convention. For cross-GPU work, never form an elapsed-time pair by recording one endpoint on each device; use validated local stream intervals or completed host timing across the participating devices.

Part 3 - Quantitative Prediction Before Execution

For payload B bytes and duration $t_{\mathrm{ms}} > 0$, derive

$$\mathrm{GB/s} = \frac{B}{10^6 t_{\mathrm{ms}}}, \qquad \mathrm{GiB/s} = \frac{1000B}{2^{30} t_{\mathrm{ms}}}.$$

Keep zero or unresolved tiny durations undefined for bandwidth. State whether one payload or twice the payload defines the reported convention. Predict why small-copy effective bandwidth can be dominated by overhead.

Part 4 - Experimental Design and Instrumentation

Preallocate matched source/destination buffers and initialize sources with known values. Measure D2H into pinned storage and D2D on the selected device with explicit stream events. Wait for completion before CPU reads, validation, or buffer reuse. Keep output allocation, pinning, and data-type conversion outside the preallocated-copy region; measure those separately if relevant. Sweep a bounded size range and repeat trials. For optional peer copies, record topology and peer capability and validate the actual timing stream/device contract before making bandwidth claims.

Check the arithmetic with a synthetic payload and a synthetic positive duration before processing hardware samples. Label that calculation as a unit-conversion test, not a bandwidth measurement. Preserve raw bytes and milliseconds beside decimal GB/s and binary GiB/s. For same-device copies, one useful payload divided by time is a valid convention; counting read plus write traffic is another convention and must not be substituted silently. For D2H, wait for the recorded copy completion before inspecting a pinned destination on the CPU. For optional peer copies, identify the actual stream that owns the operation and keep both event endpoints on a compatible device. If that contract cannot be verified, use completed host timing with all required device completion and report its broader scope. Do not compare an allocating cast/copy path against a pure preallocated copy as though their only difference were direction. Zero or unresolved event durations leave bandwidth undefined rather than infinitely fast.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def measure_copy_direction(source, destination, stream, direction):
    ...


def calculate_effective_bandwidth(payload_bytes, elapsed_ms, convention):
    ...


def E12(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report direction, devices, pin status, payload convention, milliseconds, GB/s, GiB/s, completion, and numerical error. Reject an unfinished D2H buffer read and cross-device elapsed-time pair in validation. Do not call same-device memory-copy bandwidth PCIe bandwidth. Preserve unknown staging and backend traffic rather than inferring physical link utilization from useful bytes.

Part 7 - Engineering Interpretation and Philosophy

Explain what effective bandwidth includes and what it omits. Choose a copy metric appropriate for transfer admission without confusing transport resources across directions.

19.45.14 Exercise 13

Exercise Title: Measure Compute and Transfer Regions Separately in an Overlapped Pipeline and Calculate the Percentage of Transfer Latency Hidden by Computation

Part 1 - Exercise Contract and Why It Matters

Instrument compute and transfer regions in a bounded overlapped pipeline. Measure component intervals and completed makespan, then estimate hidden transfer time under explicit assumptions. Distinguish temporal overlap from latency removed from the critical path.

Part 2 - System and Memory Mental Model

Concurrent region durations can overlap and can change under contention. Their sum is not the pipeline's elapsed time. A timeline intersection shows simultaneous activity, not automatically useful hiding. A serialized baseline must perform the same copies and compute with matching buffers and boundaries. Fill, drain, host staging, and launch overhead remain part of the declared end-to-end region.

Part 3 - Quantitative Prediction Before Execution

Let matched serialized and overlapped makespans be T_s, T_o, and isolated transfer time be $T_x > 0$. Evaluate $(T_s - T_o)/T_x$ only as a scoped attribution estimate under unchanged compute and overhead assumptions. Separately calculate transfer-interval overlap with the union of compute intervals. Construct a case where overlap exists but contention eliminates speedup. Keep negative or above-one raw estimates visible as assumption diagnostics.

Part 4 - Experimental Design and Instrumentation

Use bounded double buffering with pinned host sources, a transfer stream, and a compute stream. Establish readiness and final-reader dependencies before slot reuse. Record per-copy and compute intervals plus a same-device fork/join makespan; collect completed wall time as a separate metric. Run serialized, transfer-only, compute-only, and overlapped replays with identical useful work. Validate outputs and inspect a separate profiler trace. Include fill and drain in primary pipeline timing. Do not synchronize after every component in the path whose overlap is being measured.

Keep pipeline depth, useful chunks, allocation policy, and validation work fixed across serialized and overlapped replays. Record fill and drain, not only a favorable steady interval. The isolated transfer baseline can change under concurrent compute, so the difference-of-makespans fraction is an estimate with assumptions, not an exact physical decomposition. Compute temporal overlap from interval unions to avoid counting two overlapping compute regions twice. A high temporal fraction can coexist with a slower overall pipeline if kernels or copies contend. Preserve a negative raw benefit estimate and investigate the workload rather than clamping it to zero. An estimate above one can expose unmatched baseline overhead or changed compute costs; report the inconsistency explicitly. Use validated slot generations and last-reader events before overwriting double buffers. Extra staging capacity is part of the memory trade-off, so a faster overlapped path must still fit the declared application budget.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def instrument_overlapped_pipeline(plan, buffers, timing_pool):
    ...


def estimate_hidden_transfer(serialized, overlapped, isolated, timeline):
    ...


def E13(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export raw component intervals, makespans, completed wall time, temporal overlap fraction, attribution estimate, assumptions, and result errors. Do not silently clamp implausible percentages. Explain scope changes or contention with traces where possible. A smaller exposed transfer wait is not proof of lower total step time if compute slowed. Missing timeline alignment remains unknown rather than invented overlap.

Part 7 - Engineering Interpretation and Philosophy

Choose the metric that helps the runtime decide whether prefetch is worth its extra residency. Explain why simultaneous activity and critical-path benefit require different evidence.

19.45.15 Exercise 14

Exercise Title: Add Warmup Iterations and Demonstrate How CUDA Initialization, Kernel Selection, and Allocator Warmup Distort First-Iteration Timing

Part 1 - Exercise Contract and Why It Matters

Measure cold and warmed behavior separately and identify how initialization, algorithm selection, allocation, event initialization, and execution state affect the first iterations. Define a reproducible warmup policy without deleting unfavorable data after seeing it.

Part 2 - System and Memory Mental Model

The first CUDA use can include host/runtime setup. New shapes or kernels can trigger distinct preparation, and allocator reuse can change subsequent behavior. Event pairs may exclude some setup while still include host submission gaps after their start. Warmup also changes device temperature, clocks, cache state, and workload history. A warmed fixed-shape result is not a universal first-request latency.

Part 3 - Quantitative Prediction Before Execution

Draw cold-start host and device regions and predict which setup terms each timing scope can observe. Define fixed warmup counts or a predeclared stability criterion with a maximum budget. Add a new-shape case after warmup and predict why earlier calibration might not transfer. Keep initialization and warmed execution as separate reported outcomes.

Part 4 - Experimental Design and Instrumentation

Use fresh processes for true process-cold comparisons and explicitly state what environmental coldness is not controlled. Warm operands, kernels, event resources, and the intended allocation policy before the steady region. Keep seed, shape, dtype, precision, thread settings, and operation count fixed. Measure shape-switch behavior in a named separate series. Record every warmup observation if used by the stability policy, but exclude it from warmed statistics only by the predeclared rule. Validate outputs outside timing and bound workload size so calibration cannot grow indefinitely.

Distinguish process-cold, new-shape, allocator-cold, and warmed fixed-shape conditions. A fresh Python process does not prove the entire machine, driver, filesystem, or GPU cache is cold. State what is controlled and what remains unknown. Warm timing-event resources as well as kernels, consuming the diagnostic record before reusing its generation. If calibration uses a stability criterion, define its window, tolerance, and maximum iterations beforehand. Do not continue warmup until a preferred number appears. Record shape changes, precision settings, allocator policy, and whether outputs are preallocated. A new shape can invalidate a prior warmed cost estimate even if the high-level operation name is unchanged. Preserve first-use costs in a separate result series and include initialization in user-visible startup timing where relevant. Compare fresh runs to distinguish within-run warmup from run-to-run variability. Avoid explaining every slow first sample as algorithm selection unless a supported trace or configuration check substantiates that particular cause.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def define_warmup_contract(workload_spec, stability_policy):
    ...


def collect_cold_and_warm_replays(config, evidence):
    ...


def E14(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report process state, event initialization, warmup count/rule, shape history, allocation policy, device context, and both timing distributions. Do not call a repeated loop process-cold or infer which internal algorithm changed without supported evidence. Preserve slow warmed samples rather than reclassifying them as warmup afterward. State if stability was not reached within the bounded calibration range.

Part 7 - Engineering Interpretation and Philosophy

Explain which production scenario each distribution represents. Choose cold or warmed estimates according to the runtime decision rather than whichever looks faster.

19.45.16 Exercise 15

Exercise Title: Run Hundreds of Event-Timed Iterations and Report Median, Mean, Minimum, Maximum, and Tail Latency Instead of Relying on a Single Measurement

Part 1 - Exercise Contract and Why It Matters

Collect hundreds of valid event samples and report median, mean, minimum, maximum, and selected tail quantiles. Preserve raw data, sample identity, measurement mode, and failure counts instead of relying on one attractive iteration.

Part 2 - System and Memory Mental Model

An isolated call with a wait after each iteration has different queueing and scheduling behavior from many queued operations. Repeated-region averages are not individual request samples. Instrumentation, clocks, thermal state, concurrent work, and allocator policy can shape the distribution. The minimum is the smallest observed sample, not a guarantee or hardware lower bound. Tail quantiles need a stated finite-sample convention and adequate support.

Part 3 - Quantitative Prediction Before Execution

Define a sample count in the hundreds, warmup exclusion rule, percentile method, and outlier policy before collection. Compare isolated-latency and queued-region designs with their different boundaries. Calculate statistics by hand on a small list containing repeated values and a large outlier. Explain why p99 estimated from only a modest number of samples has limited resolution and uncertainty.

Part 4 - Experimental Design and Instrumentation

Use a bounded event pool and fixed operand/output policy. Assign each iteration an independent pair generation and do not re-record it before consuming its result. In queued mode, collect completion at declared batch boundaries rather than synchronizing every operation; in isolated mode, wait per sample deliberately. Keep modes separate. Retain CPU scalar samples and configuration IDs, not live GPU outputs. Record acquisition order and any system context available. Repeat fresh runs to expose run-to-run variability. Validate correctness outside timing and account for failed or unfinished iterations explicitly.

Use separate labels for isolated latency, queued operation intervals, and repeated-region per-operation averages. Event pairs queued back to back can observe a different scheduling environment from a request whose host waits before the next submission. Do not pool those populations. Define percentile interpolation or nearest-rank convention and preserve acquisition order so trends and bursts remain inspectable. Report finite sample count with tail statistics; a single high-order observation cannot establish a stable service level for all future requests. Keep failures and incomplete records in a parallel outcome table instead of treating them as fast samples. A predeclared invalid-record rule may exclude missing markers, but a long valid duration is not invalid simply because it is an outlier. Use bounded batches of event leases and collect them before re-recording. Track CPU collection overhead and event-pool capacity so hundreds of observations do not introduce unreported memory pressure or synchronization into the workload.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def collect_event_sample_batch(workload, pool, sampling_contract):
    ...


def summarize_valid_samples(rows, quantile_method, failure_policy):
    ...


def E15(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export raw milliseconds, sample count, min/max/mean/median, chosen quantiles, percentile convention, and rejected/error counts. Never convert failures to zero or discard slow samples because they weaken the result. Report legitimate predeclared exclusions separately. Do not combine different shapes, timing modes, or cold/warm populations into one unexplained histogram. Preserve variation instead of reporting unsupported precision.

Part 7 - Engineering Interpretation and Philosophy

Choose summary statistics for throughput planning and latency-sensitive admission. Explain the practical limits of tail estimates from the tested sample size.

19.45.17 Exercise 16

Exercise Title: Build a Hierarchical Event Timer That Measures Prefetch, Transfer, Forward, Backward, Offload, and Eviction Regions Independently

Part 1 - Exercise Contract and Why It Matters

Build a hierarchical instrumentation system for prefetch, transfer, forward, backward, offload, and eviction. Assign host/device scope to each region and support multiple streams without falsely treating overlapping children as additive durations.

Part 2 - System and Memory Mental Model

Prefetch can contain CPU file access, pinning, allocation, and CUDA copy; CUDA events measure only an explicitly bracketed device interval. Eviction can mean a D2H copy, logical release, allocator-cache retention, or a policy decision. These need different evidence. A same-device parent makespan must join all child completion dependencies it claims to cover. Parent minus summed children is not valid exclusive time when children overlap or have different clock domains.

Part 3 - Quantitative Prediction Before Execution

Define region IDs, parent relationships, device/stream, generation, logical request, timing scope, and completion. Construct a two-stream example whose child durations sum above the parent makespan. Define exclusive time only for a validated compatible nonoverlapping/nested

timeline. Predict which prefetch and eviction components require monotonic host clocks or memory counters rather than event pairs.

Part 4 - Experimental Design and Instrumentation

Start with one-stream nested compute regions, then add transfer-stream children with explicit fork/join parents. Use distinct event leases per outstanding region and retain buffers through last consumers. Record host preparation separately. For training, include forward, backward, and optimizer update in a named completed-step boundary even if component regions are reported independently. Resolve completed event records asynchronously through a bounded collector. Export CPU rows and inspect a separate profiler trace. Avoid forcing a device synchronization at every region exit.

Define each region's clock domain before assigning a parent. CPU file read or pinning can be a host child of prefetch, while H2D is a device child with its own stream and completion. A parent completed-wall interval may cover both without allowing their timestamps to be subtracted directly. For a device parent spanning streams, establish a start anchor and join all included child completions. Exclusive time is meaningful only under verified compatible interval relationships; subtracting overlapping children can produce a meaningless negative result. Treat an eviction policy decision, a D2H copy, logical buffer retirement, and allocator release as separate events or counters. A no-op CUDA event pair around Python deletion does not measure memory reclamation. Keep optimizer update and recovery inclusions in the complete training boundary. Export region identity, dependency edges, and completion status so the memory-aware scheduler can distinguish a ready copy from an unfinished prefetch request with the same region name.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch


@dataclass
class HierarchicalTimingRegion:
    region_id: str
    parent_id: object
    request_id: str
    device_identity: object
    stream_identity: object
    scope: str
    generation: int
    completion_status: str


def begin_instrumented_region(spec, timing_pool, dependency_graph):
    ...


def finalize_completed_regions(registry, cpu_evidence):
```

```
    ...


def E16(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report inclusive durations, parent makespans, overlap relationships, host preparation, release/capacity observations, and uncertainty. Reject cross-device elapsed pairs and negative invented exclusive time. A logical eviction timestamp is not proof the driver recovered physical memory. Verify that instrumentation neither retains graphs nor serializes the pipeline it claims to observe.

Part 7 - Engineering Interpretation and Philosophy

Choose region semantics that correspond to runtime decisions. Explain why a hierarchy of names is insufficient without a hierarchy of dependencies and compatible clocks.

19.45.18 Exercise 17

Exercise Title: Detect an Unintended Synchronization Point by Comparing Expected Asynchronous Timing Against Recorded CUDA Event Dependencies

Part 1 - Exercise Contract and Why It Matters

Detect and localize an unintended host synchronization through matched host timing, event dependencies, and profiler evidence. Distinguish genuine synchronization from resource contention or deliberate stream waiting.

Part 2 - System and Memory Mental Model

A CPU scalar read, certain data-dependent operations, blocking copies, or explicit waits can make an expected asynchronous path block. Event intervals alone do not identify the cause: markers can include delayed submission, and host waiting can occur outside their scope. A GPU-side stream dependency is not the same as a blocked Python thread. Confirm the suspected boundary with an independent trace or controlled rewrite before making a causal claim.

Part 3 - Quantitative Prediction Before Execution

Draw a pipeline with expected host submission, compute, transfer, and consumer dependencies. Insert a known scalar extraction as a diagnostic synchronization case. Predict which host interval grows and how the event interval may or may not change depending on marker placement. Add a no-host- sync resource-contention counterexample with a similar total latency.

Part 4 - Experimental Design and Instrumentation

Run a small correct asynchronous reference and one version with an intentional synchronization candidate. Keep device computation and output validation consistent, moving diagnostic CPU reads outside timing in the reference. Record host call intervals and event completion graph, then profile separately for synchronization events. Remove or relocate the suspected boundary and repeat. Use supported synchronization-debug facilities only where available and

label their coverage limits. Preserve all required value and storage dependencies when rewriting the path; deleting a necessary wait is not a valid optimization.

Use a controlled scalar extraction as a known host-wait example, then move it outside the submission-critical path while preserving required output validation. Capture host call durations and the GPU timeline in separate instrumented replays. Compare a resource-contention case that contains no new host fence to show why a timing gap is not sufficient causal evidence. A GPU stream wait can delay its consumer without blocking Python, whereas a host event synchronization deliberately blocks the calling thread. Keep both categories visible. Supported debug facilities can help localize some synchronization operations, but their absence of warnings is not proof that no blocking occurred. Do not remove a necessary data dependency to improve a benchmark. Validate outputs, buffer generations, and completed work after the rewrite. If a suspected operation is outside the event markers, the event interval may remain almost unchanged while wall latency changes; use that outcome to refine the boundary explanation rather than forcing the timer readings to agree.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import time
import torch


def compare_expected_and_observed_dependencies(plan, timing_rows, trace):
    ...


def localize_sync_candidate(reference_replay, modified_replay, evidence):
    ...


def E17(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export suspected operation, host blocking evidence, marker placement, trace match, rewritten result, numerical error, and remaining uncertainty. Do not claim every host/device timing gap is synchronization. A faster unsafe rewrite fails validation. Include a contention case or state if it was not observed. Report instrumentation/debug overhead separately from ordinary latency.

Part 7 - Engineering Interpretation and Philosophy

Explain the evidence linking the operation to lost asynchrony. Choose a rewrite that preserves completion requirements while moving unnecessary blocking off the critical path.

19.45.19 Exercise 18

Exercise Title: Stress-Test the Timing System with Multiple Streams, Variable Tensor Sizes, Concurrent Transfers, Memory Pressure, and Dynamic Scheduling Decisions

Part 1 - Exercise Contract and Why It Matters

Stress the timing system with multiple streams, variable shapes, concurrent copies, bounded memory pressure, and dynamic task decisions. Require correct task identity, completion, buffer safety, and honest timing scope under load.

Part 2 - System and Memory Mental Model

Dynamic shapes can change kernel choice and warmup status. Contention can lengthen event intervals without adding a new explicit dependency. Pressure can trigger allocation work or failures outside the timed device region. A task can time out at the monitor while still running on CUDA. Multiple GPUs have separate event domains; one pair cannot time a cross-device interval. A pooled event generation must survive delayed collection.

Part 3 - Quantitative Prediction Before Execution

Define bounded cases with two streams, alternating sizes, H2D/D2H competition, a retained pressure buffer, and pending-record capacity exhaustion. Predict completed, pending, rejected, and failed states. Calculate memory for instrumentation and task buffers as well as workloads. Include an offline cross-device-pair rejection and a stale-generation case. State which outcomes are synthetic policy tests versus real device observations.

Part 4 - Experimental Design and Instrumentation

Validate deterministic small tasks before applying pressure. Change one factor at a time, then selected combinations with fixed seeds. Use a bounded pool, explicit backpressure, and safe delayed collection. Drain or recover the process before freeing buffers after a timeout or asynchronous failure. Preserve a failed task's diagnostic context without inserting it as a zero-latency sample. Check numerical outputs and stream dependencies. Run profiling separately from ordinary timing. Label unavailable multi-GPU or transfer paths not measured rather than fabricating clock alignment or hardware results.

Create a record-integrity matrix for unrecorded events, timing-disabled pairs, cross-device endpoints, stale generations, delayed collectors, and capacity exhaustion. Validate deterministic rejection offline before adding CUDA load. Keep dynamic shape and cold/warm state in every record so a scheduler does not reuse a small warm cost for a larger first-use task. Memory pressure tests must be bounded and identify whether pressure is a real held allocation or an application quota. A timeout does not cancel a CUDA kernel or make its buffers safe to free. Preserve leases until a verified drain or process-recovery path resolves outstanding work. If an asynchronous error appears while querying an event, retain the originating task context and failure phase without claiming the query caused the kernel failure. For multiple GPUs, summarize local event domains and completed host intervals separately; never invent a common event clock or subtract raw timestamps from unrelated processes.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def make_timing_stress_cases(capabilities, resource_budget):
    ...
```

```
def validate_timing_record_integrity(records, task_registry):
    ...


def run_bounded_timing_stress(case, instrumentation, evidence):
    ...


def E18(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export shapes, devices, streams, pressure mechanism, event generations, pending high-water mark, overhead, raw intervals, completion status, errors, and failure phase. Separate timeout detection from actual CUDA completion. Keep nonreproducing races or slowdowns as outcomes. Do not infer allocator fragmentation from timing variability alone. Preserve the smallest record- identity or dependency counterexample found.

Part 7 - Engineering Interpretation and Philosophy

Explain which stress case invalidates the strongest naive timing claim. Use the result to refine supported scope, pool capacity, or completion policy without hiding failed samples.

19.45.20 Exercise 19

Exercise Title: Build and Benchmark a Reusable PyTorch CUDA Event Instrumentation System That Feeds Accurate Compute, Transfer, and Stall Measurements into a Memory-Aware Runtime

Part 1 - Exercise Contract and Why It Matters

Integrate a reusable CUDA event instrumentation system that supplies validated compute, transfer, completion, and scoped stall evidence to a memory-aware runtime. Keep collection bounded and preserve normal overlap while benchmarking its overhead.

Part 2 - System and Memory Mental Model

The system combines region contracts, event leases, stream dependencies, completion monitoring, buffer lifetimes, and CPU evidence. Timing values must carry shape, dtype, device, load context, and age before a scheduler uses them as cost estimates. Event intervals do not provide a complete causal stall decomposition or guarantee future latency. Host-only prefetch, logical eviction, and cross-device completion need their own clocks and evidence. Primary support excludes CUDA Graph capture and distributed timestamp alignment until separately validated.

Part 3 - Quantitative Prediction Before Execution

Predict event count, pending capacity, storage overhead, collection delay, and instrumentation time for the chosen workload. Define accepted error/tolerance checks, completion invariants, and a maximum monitoring overhead objective. Choose a finite calibration set and

held-out shapes. State how stale or missing measurements cause conservative runtime behavior rather than zero-cost predictions.

Part 4 - Experimental Design and Instrumentation

Validate tiny known dependency graphs, then instrument a deterministic transfer/compute workload and a complete training step where available. Use separate correctness, profiling, and ordinary timing replays. Compare uninstrumented, coarse-region, and fine-region instrumentation on identical useful work. Preserve E00's reproducibility record plus region schemas, pool sizes, generations, sampling modes, and completion rules. Export CPU scalars without live graph retention. Test event-pool reuse and safe cleanup. Record unavailable CUDA execution or unsupported paths explicitly instead of supplying invented timing results.

Publish a capability matrix for completed same-stream timing, custom-stream handoff, fork/join makespan, polling, copy directions, hierarchical regions, safe pool reuse, and scoped stall diagnosis. Each supported entry needs a validating run ID. Compare coarse and fine instrumentation against the same uninstrumented useful work, including memory retained by event pools and evidence queues. Feed the runtime distributions with shape, dtype, device, load context, sample count, age, and validity rather than a single universal latency constant. Missing or stale costs should trigger conservative fallback or explicit rejection, not a zero estimate. Keep CPU preparation and logical memory-release evidence outside CUDA-only claims. Record unsupported Graph capture or cross-process alignment honestly. Preserve prediction errors on held-out shapes and any overhead regression. The final result should show which measurements improve a concrete scheduling decision while retaining correct outputs and safe lifetimes, with bounded instrumentation cost and clear limits on causal interpretation.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


class CUDAEventInstrumentation:
    def validate_region(self, contract, dependency_graph):
        ...

    def acquire_event_lease(self, task_id, device, stream):
        ...

    def record_region_end(self, lease, completion_dependencies):
        ...

    def collect_ready_cpu_records(self):
        ...

    def retire_completed_lease(self, lease):
        ...


def E19(config):
```

```
# Assemble checked components and compare instrumentation overhead.
...
```

Part 6 - Measurement, Validation, and Failure Analysis

Keep prediction, observation, and interpretation separate. Every run needs ID, configuration, input shape, byte budgets, peak allocated/reserved memory, host and device timing scopes, raw durations, completion status, correctness, outcome, and failure phase. Report distributions, overhead, rejected records, and the smallest counterexample to a universal timing rule, or state none was observed. Preserve unsupported/not-measured values. A method signature is not capability evidence; each supported feature needs a validating run ID.

Part 7 - Engineering Interpretation and Philosophy

Justify the chosen instrumentation granularity through correct dependencies, safe lifetimes, measured overhead, and useful runtime decisions. Identify untested conditions that could invalidate its cost predictions.

19.46 DMM44

Chapter Title: Memory and Transfer Bandwidth Profiling
Specialization: Dynamic Memory Management and CUDA Systems

19.46.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Theoretical Bandwidth, Effective Bandwidth, Latency, Throughput, and Memory-Bound Execution

Part 1 - Exercise Contract and Why It Matters

Define a bandwidth measurement contract containing path, useful work, byte convention, timing scope, completion, cache state, and concurrency. Distinguish a theoretical hardware ceiling from measured effective throughput and from application latency.

Part 2 - System and Memory Mental Model

Useful-payload bandwidth for a transfer counts delivered bytes, while a device-memory copy benchmark often counts both requested reads and writes. Neither automatically equals physical DRAM or link traffic. Caches, transaction size, staging, conversion, and protocol overhead can change actual traffic. Small requests can be latency- or launch-limited even when large requests approach a sustained plateau. A high bandwidth number is meaningless without its numerator, denominator, and memory path.

Part 3 - Quantitative Prediction Before Execution

For bytes B and completed seconds $t > 0$, derive useful B/t and a copy's requested read/write rate $2B/t$ under their different conventions. Convert decimal GB/s and binary GiB/s consistently. Build the latency model $t(B) = \alpha + B/\beta$ as a scoped approximation and identify which data range could estimate its overhead and sustained rate. Give a counterexample to interpreting every slow operation as bandwidth-bound.

Part 4 - Experimental Design and Instrumentation

Start with eager inference-only copies, preallocated independent buffers, matching dtypes, fixed seeds, and verified contents. Preserve GPU identity, software/driver versions, allocator, topology, host-memory pinning, NUMA context when known, shapes, byte counts, precision, warmup, concurrency, and timing boundaries. Use completed same-device event pairs for compatible CUDA regions and monotonic host clocks for CPU/file/end-to-end work. Treat cross-device clock pairs, unsupported profiling counters, and unavailable paths explicitly.

Build a paper path diagram for a host buffer, a CUDA destination, a same-device consumer, and an optional second GPU. Label requested payload, requested reads/writes, and physical traffic as distinct accounting layers. A successful host-to-device copy can deliver one payload while touching host and device memory through several internal stages; do not add those stages to a link numerator without a defined convention. Create synthetic small- and large-message examples with the same sustained-rate assumption but different fixed overhead fractions. These are unit/model tests, not observed hardware points. Define latency as completed time for one declared request, and throughput as useful work over a declared completed region. A repeated batch can improve throughput without improving the first request's latency. State what cache state, concurrency, and resource sharing are held fixed before fitting a model. If citing theoretical bandwidth, use authoritative specifications for the exact hardware and compatible units; otherwise leave the ceiling unknown. Keep memory capacity and bandwidth separate because a fast transfer path can still require staging buffers that exceed the application budget.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch


@dataclass
class BandwidthContract:
    path_identity: str
    byte_convention: str
    timing_scope: str
    completion_rule: str
    cache_policy: str
    concurrency_identity: str


def validate_bandwidth_contract(contract, buffer_specs):
    ...


def E00(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Keep prediction, observation, and interpretation distinct. Export raw bytes and durations beside derived rates. Unknown traffic remains unknown; zero or unresolved durations do not imply infinite bandwidth. Do not invent hardware results or use published device figures as local measurements.

Part 7 - Engineering Interpretation and Philosophy

Explain what resource each rate describes and which runtime decision it can support. An honest path-specific cost model is more useful than one universal bandwidth constant.

19.46.2 Exercise 01

Exercise Title: Calculate Effective Bandwidth from Bytes Transferred and CUDA Event Timing for a Controlled PyTorch Memory Copy

Part 1 - Exercise Contract and Why It Matters

Calculate effective bandwidth for a controlled PyTorch copy using completed CUDA event timing. Validate the byte numerator and timing domain before summarizing throughput.

Part 2 - System and Memory Mental Model

A copy can allocate, convert dtype, broadcast, or pack strided storage unless those behaviors are excluded. Use equal-shaped, equal-dtype independent buffers for the primary pure-copy region. Event intervals include stream progress and possible interference rather than guaranteed exclusive engine time. Source and destination must remain valid through completion, and the timing pair must belong to one compatible device domain.

Part 3 - Quantitative Prediction Before Execution

For payload B bytes and elapsed $t_{\mathrm{ms}} > 0$, derive

$$\mathrm{GB/s} = B/(10^6 t_{\mathrm{ms}}), \qquad \mathrm{GiB/s} = 1000B/(2^{30} t_{\mathrm{ms}}).$$

State whether this exercise reports one delivered payload or requested reads plus writes. Calculate synthetic unit-conversion examples and preserve their synthetic status. Predict why tiny intervals need larger regions or an explicit unresolved result.

Part 4 - Experimental Design and Instrumentation

Preallocate and initialize buffers, warm event resources and the selected copy path, and record start/copy/end on the declared stream. Establish end completion before reading elapsed time. Validate copied values outside timing. Collect bounded repeated samples with generation-safe event reuse. Record allocation and dtype-conversion exclusions explicitly. Compare a single-copy latency series with repeated-region throughput only under separate labels. Retain CPU scalar evidence rather than every GPU output.

Check the byte numerator using actual tensor metadata rather than a shape label alone. Source and destination must have the same logical shape and dtype in the pure-copy case, with no overlap, broadcasting, or implicit conversion. Use known coordinate-coded contents to detect a no-op alias or wrong region. Record whether the time covers one copy or several repetitions and multiply bytes by that repetition count only when every copy actually executes. A final output equal to the source does not by itself prove all repeated requests happened; preserve the operation ledger. Initialize and consume diagnostic event records before reusing their generation

for warmed timing. Read elapsed time only after the matching end has completed. A zero-valued tiny interval can reflect resolution and must leave bandwidth undefined until a valid larger-region contract is used. Report both payload and read/write conventions if helpful, but do not silently switch them between rows. Validate decimal/binary unit conversions on a hand-calculated synthetic case before applying them to real distributions.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def count_copy_payload(source, destination, byte_convention):
    ...


def time_checked_copy(source, destination, stream, timing_pair):
    ...


def calculate_completed_copy_rate(payload_bytes, elapsed_ms, convention):
    ...


def E01(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report shape, dtype, useful bytes, requested-byte convention, event generation, device/stream, duration, completion, and copy error. Reject aliasing, unfinished pairs, unrecorded events, or a changing numerator. Do not replace failed samples with zero time. A bandwidth above a presumed hardware ceiling needs a scope/cache/convention investigation before any performance claim.

Part 7 - Engineering Interpretation and Philosophy

Explain the exact rate calculated and what it does not measure. Choose whether the runtime needs per-request latency, sustained useful payload, or requested memory traffic.

19.46.3 Exercise 02

Exercise Title: Benchmark CPU-to-GPU Transfer Bandwidth Across Increasing Tensor Sizes and Identify the Region Where PCIe Throughput Saturates

Part 1 - Exercise Contract and Why It Matters

Sweep H2D payload size and identify an observed large-transfer throughput plateau within the tested range. Confirm the actual host-GPU path before calling the result PCIe throughput.

Part 2 - System and Memory Mental Model

Small transfers include launch and staging overhead; larger transfers may approach a sustained rate limited by host memory, interconnect, or device conditions. A plateau is empirical evidence, not proof of the nominal bus ceiling. Pageable and pinned sources are different contracts. Preallocation, NUMA locality, contention, and warmup can change the curve.

Part 3 - Quantitative Prediction Before Execution

Choose a bounded geometric size sweep and predict latency with $\alpha + B/\beta$. Define a plateau criterion using neighboring large-size rates and variability before observing results. Explain why an unsaturated curve cannot establish the link maximum. Predict the difference between copy-only and completed end-to-end regions.

Derive a crossover payload from the fitted fixed-overhead and sustained-rate terms. For a declared acceptable overhead fraction, solve for a candidate minimum payload and test that prediction on held-out sizes. Keep a failed prediction when the simple model misses staging thresholds or implementation changes. This is a model-specific batching hint, not a hardware saturation law. Report the chosen fraction, fit interval, residual uncertainty, and whether the proposed payload remains within pinned and CUDA buffer budgets.

Part 4 - Experimental Design and Instrumentation

Use pinned sources and preallocated CUDA destinations as the primary reusable-copy path. Match contents and dtype across sizes, with no conversion. Warm each shape and time completed regions with event and host scopes recorded separately. Keep source storage unchanged until completion. Repeat sizes in an order that exposes drift, and record distributions rather than one sample. Preserve topology evidence and hardware identity; if the physical host link is unverified, label the result H2D rather than asserting PCIe. Measure pinning or allocation in separate end-to-end experiments.

Choose size limits from explicit host pinned-memory and CUDA budgets. A geometric sweep should include several points in any candidate plateau region rather than declaring saturation from the largest single sample. Repeat or interleave size order to distinguish systematic size trends from thermal/load drift. Keep source placement and host-memory locality recorded where observable. An H2D plateau can arise from host memory, staging, submission, link behavior, or another resource, so topology confirmation and traces matter before assigning the bottleneck to PCIe. Preserve raw completed times and fit residuals. If the smallest sizes are dominated by overhead, do not force a line through all sizes or interpret a negative fitted intercept as a physical latency. Restrict the model or report the fit failure. Keep an unverified host-link type in the capability record rather than guessing from the GPU model. Compare copy-only calibration with a preparation-inclusive application query so later runtime decisions do not omit a cost that their actual request must pay.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def make_bounded_h2d_size_sweep(resource_budget):
    ...
```

```
def fit_scoped_latency_model(size_rows, fit_range, uncertainty):
    ...


def detect_measured_plateau(rows, declared_criterion):
    ...


def E02(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export sizes, pin status, raw latency distributions, effective rates, fit residuals, plateau criterion, topology evidence, and correctness. Report no plateau observed when appropriate. Do not extrapolate a fitted rate beyond its calibrated range or attribute every limit to PCIe without evidence. Preserve host staging and CPU submission contributions under their named scopes.

Part 7 - Engineering Interpretation and Philosophy

Choose a transfer-size regime for the runtime's cost model. Explain when a fixed overhead matters more than large-message bandwidth.

19.46.4 Exercise 03

Exercise Title: Benchmark GPU-to-CPU Transfer Bandwidth and Compare It Against Host-to-GPU Throughput Under Identical Buffer Sizes

Part 1 - Exercise Contract and Why It Matters

Benchmark D2H throughput using the same payload-size set as H2D and compare directions without assuming symmetric performance. Preserve matched host-memory and buffer-reuse contracts.

Part 2 - System and Memory Mental Model

A D2H copy into pinned host memory can still be unfinished when Python returns. Reading its destination requires an appropriate completion fence. H2D and D2H can differ through engine behavior, host memory, topology, and staging. Equal byte counts do not guarantee equal end-to-end costs. Device events and host timers must retain the same meanings across directions.

Part 3 - Quantitative Prediction Before Execution

Predict useful bytes and latency/rate curves for both directions under an explicit model. Derive direction ratios only for matched size and timing scope. Identify what evidence would justify an asymmetry conclusion and what variation would leave it unresolved. Include a case where host allocation would dominate a comparison if it were included on only one path.

Part 4 - Experimental Design and Instrumentation

Preallocate GPU sources and pinned CPU destinations for D2H. Match the H2D dataset's dtype, byte sizes, stream policy, warmup, repetitions, and allocation exclusions. Initialize source values before timing. Record completed CUDA regions and completed host intervals separately. Wait before checking CPU contents or reusing buffers. Alternate direction order across controlled replays to reduce drift bias. Keep validation and profiling outside ordinary timing, and record any unsupported path explicitly.

Use the same payload sequence and warmup rule for both directions, but initialize each source independently before its timed copy. A CPU destination that will receive D2H must not be read or overwritten until its actual completion. If the primary comparison uses pinned memory, ensure both the H2D source and D2H destination satisfy that policy. Keep CPU allocation outside both reusable-copy regions, and include it on both sides in any allocating comparison. Distinguish the host call's return interval from completed wall time and the event-bracketed interval. Record direction order and run-to-run variation, because sequentially benchmarking all H2D first and all D2H much later can confound direction with changing load. Compare distributions or paired configurations rather than arbitrary sample indices from separate replays. If a direction is consistently slower, state the measured difference before proposing a hardware explanation. Preserve unsupported physical traffic counters and unresolved staging effects as unknown. A later offload policy must use the appropriate direction model instead of averaging away an important asymmetry.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def benchmark_matched_directions(size_specs, host_buffers, gpu_buffers):
    ...


def compare_direction_distributions(h2d_rows, d2h_rows):
    ...


def E03(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report per-direction raw bytes, distributions, pinning, completion, effective bandwidth, comparison uncertainty, and result error. Do not inspect an unfinished D2H buffer to test correctness. A direction difference is an observation to explain with supported evidence, not permission to assert a particular hardware limitation. Preserve unresolved or nonreproducing rankings.

Part 7 - Engineering Interpretation and Philosophy

Choose separate cost estimates when direction matters. Explain why CPU offloading cannot assume the reload path and eviction path have the same latency.

19.46.5 Exercise 04

Exercise Title: Compare Pageable and Pinned Host Memory by Effective H2D and D2H Bandwidth Across Small, Medium, and Large Transfers

Part 1 - Exercise Contract and Why It Matters

Compare pageable and pinned host storage for H2D and D2H across small, medium, and large payloads. Separate reusable transfer cost from allocation, pinning, and staging preparation.

Part 2 - System and Memory Mental Model

Pinning can enable useful asynchronous transfer paths but consumes a bounded host resource and can be expensive to perform repeatedly. Pageable copies can involve staging or host blocking. An event-bracketed region does not automatically isolate pure DMA because host submission delays can affect its interval. Different buffer contracts answer different application questions.

Part 3 - Quantitative Prediction Before Execution

Predict four reusable-copy curves by direction and pin status. Add separate pin-and-copy and allocating-copy contracts. Estimate how a fixed preparation cost changes small-message effective bandwidth. Construct a case where pre-pinned reuse helps while repeated pinning removes that advantage. Keep those as predictions until measured.

Part 4 - Experimental Design and Instrumentation

Use equal-valued, equal-dtype host buffers and independent preallocated GPU storage. Verify actual pin status, contiguity, and byte count. Measure reusable copies and preparation-inclusive paths in distinct replays with matched timing boundaries. Keep pinned H2D sources unchanged and D2H destinations unread until completion. Bound pinned capacity and outstanding copies. Collect host submission, completed wall time, and compatible event intervals. Inspect staging only through supported traces; preserve unknown internal behavior rather than inventing it.

Allocate equal-valued pageable and pinned buffers with matched shape, dtype, and layout. Verify pin status from the object used by the copy, not from an earlier buffer that was subsequently replaced. Distinguish already- pinned reusable storage, explicit pinning of an existing pageable tensor, and a path that allocates and fills new host storage for every request. The end-to-end application comparison must include whichever preparation its request really requires. Keep source contents stable for nonblocking H2D and preserve D2H destination ownership through the fence. Record total pinned capacity and outstanding requests so the fastest mode does not hide unbounded host residency. Small-message rankings can differ from large-message rankings; retain those observations rather than imposing one winning mode. If event and wall intervals differ, inspect their boundaries and host staging before subtracting them as an exact overhead. Include a repeated-reuse trace to measure amortization explicitly. The conclusion should name the workload and resource constraint under which pinning helps, not assert a universal speedup from the presence of a flag.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def prepare_host_memory_comparison(spec, pinned_capacity_budget):
    ...


def benchmark_host_memory_modes(cases, timing_contract):
    ...


def E04(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report direction, size class, pin status, preparation inclusions, timing scopes, throughput distributions, host capacity, and copy errors. Do not compare an allocating path with a preallocated path without labeling the difference. Preserve slower pinned results where observed. A nonblocking flag is not evidence that compute-transfer overlap occurred.

Part 7 - Engineering Interpretation and Philosophy

Choose a host-buffer strategy for repeated offload under both RAM and latency constraints. Explain which costs are amortized and which remain on the critical path.

19.46.6 Exercise 05

Exercise Title: Compare Hundreds of Small Transfers Against One Batched Transfer with the Same Total Byte Count and Quantify Per-Transfer Overhead

Part 1 - Exercise Contract and Why It Matters

Compare hundreds of small transfers with one larger transfer carrying the same useful bytes. Quantify submission overhead and preparation costs without hiding packing or output reconstruction.

Part 2 - System and Memory Mental Model

Many requests can pay repeated dispatch, event, and staging costs. Batching can reduce those costs but may need CPU packing, GPU unpacking, additional storage, or longer waiting before a batch is ready. Equal payload bytes alone do not make the paths operationally equivalent. Per-copy synchronization would introduce another experimental factor and must not appear on only one side of the primary comparison.

Part 3 - Quantitative Prediction Before Execution

For n chunks of size b, compare $n(\alpha + b/\beta)$ with $\alpha + nb/\beta$ under the simple serialized model. Add packing/unpacking and batch-formation delay as separate terms. Define one copy-only comparison and one end-to-end application comparison. Predict when the added preparation could exceed the saved request overhead.

Part 4 - Experimental Design and Instrumentation

Preallocate a fixed total payload and equivalent destinations. For the small path, submit the declared chunk count under one completed batch boundary; for the large path, use one contiguous payload. Validate sample ordering and contents after completion. Add a separate fragmented-source case that requires real packing and include its cost. Keep pinning, dtype, total bytes, warmup, and stream policy matched. Measure CPU submission, completed time, event makespan, and peak staging capacity. Repeat with bounded chunk counts and preserve the full operation ledger.

Define whether the small chunks already occupy one contiguous parent or begin as independent fragmented application objects. In the first case, a larger transfer can be a fair copy-only alternative without real packing. In the second, CPU packing and any GPU unpacking belong to the application cost. Preserve the same logical order of samples and validate every destination interval. Use hundreds of requests only within the declared buffer budget, and retain completion before slot reuse. Do not synchronize after each small copy unless that is a separately named latency contract; otherwise the test would combine request granularity with a changed synchronization policy. Measure one completed batch makespan, host submission duration, and useful throughput. If estimating per-request overhead from a fitted model, retain its size range and uncertainty rather than extrapolating it to arbitrary concurrency. Add batch-formation delay for an online workload that waits for requests to accumulate. A bandwidth gain can coexist with worse response time or higher staging memory, so the final runtime decision needs both constraints.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def build_equal_payload_transfer_cases(total_bytes, chunk_counts):
    ...


def measure_transfer_batch(case, buffers, timing_contract):
    ...


def estimate_request_overhead(rows, preparation_contract):
    ...


def E05(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report total bytes, chunks, per-chunk size, preparation, completion, latency, throughput, and memory overhead. A fitted per-request cost is scoped to its range and concurrency. Do not exclude real packing from an end-to-end batching claim. A batched path that delays readiness

or increases peak memory can be worse for a latency-sensitive runtime despite higher copy-only bandwidth.

Part 7 - Engineering Interpretation and Philosophy

Choose batching based on useful completed work, preparation, residency, and response-time constraints. Explain why equal bytes is necessary but insufficient for a fair transfer comparison.

19.46.7 Exercise 06

Exercise Title: Sweep float16, bfloat16, float32, and float64 Transfers and Determine Whether Achieved Bandwidth Depends on Element Count or Total Bytes

Part 1 - Exercise Contract and Why It Matters

Sweep float16, bfloat16, float32, and float64 copies under both fixed- element and fixed-byte controls. Determine which observed differences follow from payload and which require another explanation.

Part 2 - System and Memory Mental Model

A raw matching-dtype copy moves storage bytes without arithmetic conversion. Changing element width at fixed element count changes payload. At fixed bytes, shapes or element counts differ and may select different implementation paths. A device's support for fast bfloat16 arithmetic is not identical to support for copying bfloat16 storage. Casting during the benchmark is a different workload and must not masquerade as pure transfer.

Part 3 - Quantitative Prediction Before Execution

Calculate payload for each dtype and predict fixed-element latency from the fitted size model. Construct equal-byte cases with valid integral element counts. Predict that byte-matched large contiguous copies may be similar under a simple transport model, but leave dtype/path effects as tested hypotheses. Define numerical or exact-storage correctness checks appropriate to initialized values.

Part 4 - Experimental Design and Instrumentation

Create source/destination pairs with identical dtype within each copy. Verify element_size and total bytes from the actual tensors. Exclude random generation, conversion, and allocation from reusable-copy timing. Warm each supported case and measure the same direction, stream, pinning, and completion contract. Run fixed-element and fixed-byte sweeps separately. Record unsupported storage or transfer combinations rather than silently substituting another dtype. Keep values bounded and deterministic so copy validation does not become a test of different arithmetic precision.

Create a table of actual element widths and derive every case's byte count from numel times element_size. Fixed-element and fixed-byte sweeps must be separate result families. For the latter, choose byte targets divisible by each tested width and preserve the same logical layout class. Use matching source/destination dtypes so the path does not cast while copying. Explicitly record unsupported storage or transport behavior without assuming that missing fast arithmetic support forbids byte movement. Initialization can use simple finite values exactly representable in all selected dtypes; its CPU/GPU compute cost stays outside the reusable-copy region. Validate byte-preserving or value-preserving behavior according to the declared check. If equal-byte rates differ, investigate size, kernel path, alignment, layout, staging, and variability

before attributing the result to dtype. Do not force a byte-only conclusion merely because the simple model predicts it. Conversely, a larger fixed-element payload explains additional time without invoking slower floating-point arithmetic. Keep conversion-inclusive application requests as a different path model.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def make_dtype_transfer_cases(element_counts, byte_targets, capabilities):
    ...


def compare_dtype_byte_controls(case_rows, latency_model):
    ...


def E06(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report actual dtype, element width, numel, bytes, control policy, conversion status, timing distributions, and errors. Do not attribute a fixed-element float64 slowdown to arithmetic when it moved more bytes. Conversely, do not force equal rates if matched-byte measurements differ. Inspect path, layout, overhead, and variability before proposing a cause.

Part 7 - Engineering Interpretation and Philosophy

Choose bytes as the primary transfer-cost input while preserving dtype and layout as capability/context fields. Explain when element count alone would mislead the runtime.

19.46.8 Exercise 07

Exercise Title: Measure CUDA Device-to-Device Copy Bandwidth and Compare It Against Host-Device Transfer Bandwidth

Part 1 - Exercise Contract and Why It Matters

Measure same-device CUDA copy performance and compare it with host-device transfers using clearly labeled numerator and resource conventions.

Part 2 - System and Memory Mental Model

A same-GPU D2D copy primarily accesses device memory resources; H2D/D2H uses a host-device path and host memory. Comparing device read-plus-write bytes with host useful-payload bytes without labels creates a factor-of-two ambiguity. Repeated device copies can be served partly by caches, so the observed effective rate is not automatically sustained device DRAM bandwidth.

Part 3 - Quantitative Prediction Before Execution

Calculate one-payload and requested read/write rates for the same D2D sample, then identify the valid comparison with H2D useful throughput. Predict small-size overhead and large-size behavior without borrowing a hardware peak figure. Explain why rates from different resources need not share a common ceiling.

Part 4 - Experimental Design and Instrumentation

Use independent contiguous GPU buffers with equal shapes/dtypes and preallocated copies. Warm the path and record completed same-device intervals. Sweep sizes within a bounded memory budget and use a documented reuse/cache policy. Compare with matched host-transfer rows under both timing and byte labels. Validate contents after completion and prevent source/destination aliasing. Record whether cache-sensitive repetition or rotating buffers were used. Profile separately if attempting to identify kernel versus copy-engine behavior, and keep unsupported hardware counters unknown.

Specify same-device D2D explicitly so it cannot be confused with peer movement between GPUs. Use independent backing storage and reject a source view that aliases the destination in an overlapping way. Record source and destination strides and verify no conversion or broadcasting occurs. For comparison with host paths, present useful-payload rates under one convention and requested device read/write rates under another clearly labeled convention. The two numbers may differ by a factor of two without any measurement conflict. Run repeated-buffer and larger/rotating-working-set experiments separately, with a finite memory bound and the same completion rule. A large allocation can reduce some cache reuse but does not prove every byte reached DRAM. If supported counters are unavailable, call the result effective copy bandwidth for that working set. Keep ordinary timing free of profiler replay overhead. An apparent rate higher than a nominal device-memory figure should trigger a check of numerator, cache regime, interval validity, and specification scope, not an immediate claim that the hardware exceeded its physical limit.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def benchmark_same_device_copy(spec, buffers, timing_pool):
    ...


def compare_device_and_host_paths(rows, numerator_conventions):
    ...


def E07(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export path, useful/requested bytes, cache policy, completed duration, rate, memory capacity, and correctness. Do not label same-device D2D as PCIe bandwidth or claim a repeated small-buffer result measures DRAM saturation. Preserve high apparent rates with a scoped interpretation rather than deleting them or claiming impossible hardware performance.

Part 7 - Engineering Interpretation and Philosophy

Explain which resource makes the paths different. Choose a calibration that matches the memory-aware runtime's actual access pattern.

19.46.9 Exercise 08

Exercise Title: Benchmark Peer-to-Peer Tensor Transfers Between Two GPUs and Compare Direct P2P Movement Against CPU-Staged Copies

Part 1 - Exercise Contract and Why It Matters

Benchmark GPU-to-GPU movement and an explicit CPU-staged alternative. Verify peer capability and available path evidence before labeling the GPU copy direct P2P.

Part 2 - System and Memory Mental Model

Peer access capability is not proof of the exact path selected by every high-level copy. Topology, driver, platform restrictions, and implementation can cause fallback or different transport. The explicit staged path contains D2H completion followed by H2D from valid host storage, with staging capacity and possible overlap defined. Cross-device event endpoints cannot form one valid elapsed-time pair.

Part 3 - Quantitative Prediction Before Execution

Predict one-payload delivery rate for each complete path. For serialized staging, model D2H plus H2D plus preparation rather than dividing the payload by only one leg. Include any rotating staging buffers in the memory bound. Predict a topology where the direct candidate may not be available and define an honest unsupported or unverified label.

Part 4 - Experimental Design and Instrumentation

Use two available CUDA devices with explicit identity and peer capability checks in each direction. Preallocate source, destination, and pinned staging. Time completed end-to-end movement with a monotonic host clock and all required device completions. Use local event intervals only with verified owning-stream semantics. Validate values after every path. Seek supported profiler/topology or diagnostic evidence of direct movement; otherwise call it the high-level GPU-copy candidate. Keep CPU staging explicit without changing global system peer settings. Record direction, topology, sizes, warmup, and buffer reuse.

Test capability and complete correctness for each GPU direction because a pair's observed behavior can be asymmetric. Preserve stable device identities, visible ordinals, topology evidence, and process mode. A high-level copy candidate may succeed through a path the profiler cannot distinguish; label that uncertainty instead of promoting capability to proof. The CPU-staged reference must complete D2H before H2D consumes that host buffer, and its host capacity belongs to the accounting. If double-buffering the staged path, validate slot generations and report its additional residency and changed overlap. Use complete host timing with both relevant device completions as the portable primary end-to-end metric; never subtract events recorded on different GPUs. Local leg timing is optional only with verified stream semantics.

Do not change machine-wide peer settings just to obtain a preferred comparison. An optional supported diagnostic can add path evidence, but its performance is a separate instrumented result. Preserve a not-measured outcome when the second device or reliable path evidence is unavailable.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def inspect_peer_path_capabilities(device_pair, evidence_sources):
    ...


def benchmark_gpu_copy_and_cpu_staging(spec, buffers, completion_contract):
    ...


def E08(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report capability versus observed-path evidence separately, complete latency, staging leg times, useful bytes, host capacity, and errors. Do not invent a direct path from a fast rate or a successful capability query. If only one GPU is available, retain offline protocol checks and mark execution not measured. Preserve asymmetric direction results and unsupported diagnostics.

Part 7 - Engineering Interpretation and Philosophy

Choose a path based on measured complete cost and verified support. Explain why the runtime should retain path identity and uncertainty instead of assuming all GPU pairs communicate equally.

19.46.10 Exercise 09

Exercise Title: Build a PyTorch GPU Memory-Copy Benchmark and Estimate Effective Device-Memory Bandwidth from Total Bytes Read and Written

Part 1 - Exercise Contract and Why It Matters

Build a GPU copy benchmark that reports requested device-memory bytes read and written. Estimate effective memory bandwidth while preserving the distinction from measured physical DRAM traffic.

Part 2 - System and Memory Mental Model

For an independent equal-dtype copy of N elements of width s, requested traffic is $2Ns$ bytes. Caches, write behavior, transaction granularity, and implementation choices can make physical traffic differ. Reusing a small working set can measure a cache-influenced regime.

Unsupported performance counters do not justify relabeling requested bytes as actual DRAM bytes.

Part 3 - Quantitative Prediction Before Execution

Derive the requested numerator for one copy and a repeated region. Calculate how counting one payload instead changes the reported number. Choose a bounded range and rotation policy intended to explore cache and larger-working-set behavior. State that a larger buffer does not alone prove caches had no effect. Predict the evidence needed for a physical-traffic claim.

Part 4 - Experimental Design and Instrumentation

Preallocate independent buffers and use matching shape/dtype, no broadcasting, no conversion, and no autograd graph in the primary memory benchmark. Warm copies and timing resources. Measure completed event regions and raw distributions. Compare repeated-buffer and rotating-buffer cases as separate contracts, recording total resident capacity. Validate all touched outputs. If supported, collect memory-traffic counters in a separate Nsight Compute replay and record metric names, device, version, and collection limitations. Keep those instrumented results separate from ordinary timing.

Count requested traffic for the exact operation sequence, not only the final output's size. A copy of independent storage requests a read and a write, whereas aliasing can turn an intended benchmark into a no-op or another unsupported case. Include repeated-region count and buffer-rotation policy in the numerator record. Cache-influenced effective rates are useful observations when labeled, but they cannot establish sustained external-memory bandwidth. If hardware counters are collected, record the specific memory level and whether the tool replayed kernels or adjusted cache behavior. Counter availability and permissions can vary; unsupported metrics remain unsupported rather than being reconstructed from elapsed time. Compare requested and physical bytes only under compatible workload and memory-level assumptions. Avoid summing load/store counters that refer to different scopes or double-counting transactions already included in an aggregate metric. Use held-out working sets to test the calibration's portability. Preserve the raw scalar timings, copy result checks, resident capacity, and peak allocation alongside the bandwidth estimate so the runtime can reject an otherwise fast calibration that would exceed its memory budget.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def requested_copy_traffic_bytes(source_spec, repeat_count):
    ...


def benchmark_device_memory_copy(case, buffers, timing_contract):
    ...


def reconcile_requested_and_profiled_traffic(rows, hardware_metrics):
    ...
```

```
def E09(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report requested reads/writes, useful payload, elapsed time, cache policy, working-set capacity, effective rate, actual-counter status, and correctness. An apparent rate above a nominal DRAM figure may reflect caches or conventions; investigate before claiming hardware violation. Do not copy theoretical specifications into the measured column or confuse kernel replay counters with uninstrumented application timing.

Part 7 - Engineering Interpretation and Philosophy

Explain when requested-byte bandwidth is useful despite incomplete physical-traffic knowledge. Identify the uncertainty that prevents a stronger DRAM utilization claim.

19.46.11 Exercise 10

Exercise Title: Compare Contiguous and Noncontiguous Tensor Copies and Measure the Bandwidth Lost to Strided Memory Access and Extra Materialization

Part 1 - Exercise Contract and Why It Matters

Compare contiguous and strided copy paths while separating useful payload, backing capacity, index work, and explicit materialization. Verify that both paths deliver the intended logical values.

Part 2 - System and Memory Mental Model

A noncontiguous view can retain a larger parent allocation, and a backend may choose a specialized kernel or temporary. Calling contiguous before timing changes the measured contract; calling it inside adds materialization work. A strided tensor alone does not establish how warp lanes access memory. Do not attribute every difference to coalescing without lower-level evidence.

Part 3 - Quantitative Prediction Before Execution

Define matched logical shapes with distinct strides and derive useful copied bytes, parent backing capacity, and requested extra materialization bytes where known. Compare direct strided copy with materialize-then-copy including and excluding preparation under separate labels. Predict a case where preparation amortizes across repeated consumers and another where it does not.

Part 4 - Experimental Design and Instrumentation

Construct coordinate-coded inputs and independent destinations. Record shape, stride, storage offset, contiguity, dtype, and backing capacity. Avoid source/destination overlap and broadcasting. Warm supported paths and measure completed intervals with matched allocation policy. Profile separately for extra kernels or copies. Validate logical ordering after completion, including sliced and transposed cases. Preserve unsupported layouts as rejected rather than silently converting them before the supposedly strided benchmark.

Use coordinate-coded tensors to verify logical order after transposition or slicing. Record shape, stride, storage offset, source backing capacity, destination layout, and any temporary representation. A view with few logical elements can retain a much larger parent storage. Do not include that backing span in useful copied bytes unless the path actually moves it, but do include it in residency. Compare direct strided copy, explicit materialization, and repeated reuse of a materialized representation as three named contracts. If materialization is outside the copy-only interval, retain its measured cost for end-to-end analysis. Profile extra kernels and copies without assuming the high-level API uses one implementation on every shape. A noncontiguous source and noncontiguous destination can produce different work from a strided source copied into contiguous output, so do not combine them into one unexplained category. Keep storage overlap excluded. The resulting performance difference is a layout-path effect unless lane mapping and transaction evidence support a narrower claim about coalescing.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def build_matched_layout_copy_cases(spec):
    ...


def measure_layout_and_materialization(case, timing_contract):
    ...


def E10(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report useful bytes, backing capacity, preparation inclusions, stride, extra materialization, duration, effective rate, and copy error. Do not infer physical traffic from view size alone. A faster prepared path must include its one-time cost in an end-to-end reuse analysis. Explain remaining kernel or cache uncertainty rather than calling all lost bandwidth strided access.

Part 7 - Engineering Interpretation and Philosophy

Choose a layout strategy according to consumer count, capacity, and measured complete cost. State which evidence would be needed to attribute the result specifically to warp-level access efficiency.

19.46.12 Exercise 11

Exercise Title: Design Coalesced and Poorly Localized GPU Memory-Access Workloads and Measure Their Effective Bandwidth Difference

Part 1 - Exercise Contract and Why It Matters

Design controlled GPU access patterns that differ in locality, and establish when the evidence supports a coalescing claim. Keep a PyTorch-level locality comparison separate from an optional explicit-lane kernel experiment.

Part 2 - System and Memory Mental Model

Coalescing concerns addresses requested by lanes of a warp, not merely the printed stride of a Python tensor. High-level gathers can include index reads, bounds work, allocation, and implementation-dependent thread mapping. A custom CUDA or supported kernel-language extension can control lane mapping, but it adds a separate capability and validation requirement. Timing alone cannot prove the number of physical memory transactions.

Part 3 - Quantitative Prediction Before Execution

Specify adjacent versus dispersed lane addresses for a hypothetical kernel and derive requested data bytes separately from index bytes and backing span. Construct a high-level gather experiment with the same number of outputs but different index locality. Explain why its rate includes indexing overhead. Predict which profiler transaction or traffic evidence could support the coalescing interpretation.

Part 4 - Experimental Design and Instrumentation

Start with deterministic PyTorch access workloads and label them locality tests. Preallocate outputs where supported and exclude index construction from the access region while recording its cost separately. If implementing the optional explicit-lane kernel, validate bounds, output equality, lane mapping, and supported launch configuration before benchmarking. Keep useful values and operation counts matched. Collect supported traffic metrics in a separate replay and preserve unknowns. Use bounded working sets and distinguish cache reuse from access-transaction effects.

Define the access pattern independently of the Python operator name. For the optional controlled-lane experiment, specify which lane requests which element, alignment assumptions, bounds, and whether reads, writes, or both are dispersed. Keep the same useful output count and validate every value. For the high-level PyTorch experiment, include index-array reads and other known requested bytes separately from useful data bytes. Advanced indexing may allocate or use a different kernel, so its slowdown cannot be assigned solely to memory transactions. Exclude index generation from the access-only region and retain it in an end-to-end path if the application regenerates indices. Test cache reuse and working-set size under bounded capacity. Use supported profiler metrics to compare requested work with physical traffic where possible, preserving version and device scope. Do not invent a custom-kernel implementation hidden inside the scaffold. If that capability is unavailable, complete the locality experiment honestly and mark direct warp-level coalescing verification unsupported.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def build_access_pattern_cases(spec, index_policy):
    ...
```

```
def launch_validated_access_case(case, output, kernel_capabilities):
    # Optional controlled-lane kernel remains learner implementation work.
    ...


def evaluate_locality_evidence(rows, traffic_metrics, lane_mapping):
    ...


def E11(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report pattern, useful/index bytes, working set, output validation, latency, effective convention, lane-mapping status, and hardware evidence. Do not call a gather slowdown a pure coalescing penalty without sufficient control. Mark the custom-kernel path unsupported if unavailable and retain the valid high-level locality result. Preserve cases where caching changes the expected ranking.

Part 7 - Engineering Interpretation and Philosophy

Explain the strongest conclusion justified by the experiment. Distinguish a useful locality signal from a proven microarchitectural cause.

19.46.13 Exercise 12

Exercise Title: Measure Simultaneous H2D Transfer and GPU Compute and Calculate How Much PCIe Traffic Is Hidden Behind Computation

Part 1 - Exercise Contract and Why It Matters

Measure simultaneous H2D transfer and GPU compute with a validated bounded pipeline. Report temporal overlap and completed latency benefit as separate quantities.

Part 2 - System and Memory Mental Model

Independent streams and pinned sources permit useful overlap but do not guarantee it. Compute and copies can contend for device memory or other resources. Total bytes delivered do not decrease merely because transfer latency is hidden. A serialized baseline must perform the same useful work and include the same fill, drain, preparation, and output requirements.

Part 3 - Quantitative Prediction Before Execution

Let matched serialized and overlapped times be T_s, T_o, and isolated transfer time be $T_x > 0$. Treat $(T_s - T_o)/T_x$ as an attribution estimate only under unchanged-compute/overhead assumptions. Separately calculate overlap of transfer intervals with the union of compute intervals. Predict a case with temporal overlap but no end-to-end speedup. Preserve negative or above-one raw estimates as diagnostics.

Part 4 - Experimental Design and Instrumentation

Use pinned sources, preallocated double buffers, one transfer stream, and one compute stream. Establish data-ready and final-reader events before slot reuse. Record compatible component intervals plus a fork/join makespan, and completed host time separately. Run transfer-only, compute-only, serialized, and overlapped replays with identical shapes and useful chunks. Validate results after completion. Profile in a separate replay and do not synchronize after each component in the overlapped timing path. Record actual path evidence before describing the host link as PCIe.

Keep the serialized and overlapped pipelines identical in useful copies, compute, output collection, and preparation policy. Double buffering adds capacity and slot-lifetime constraints even if it shortens the critical path. Use a common same-device anchor and joined completion for makespan, plus separate component intervals and completed wall time. Compute temporal overlap from interval unions so simultaneous compute regions are not counted twice. The difference-of-makespans estimate assumes compute and overhead do not change; contention can violate that assumption. Preserve negative benefit or an estimate above one and investigate unmatched baselines rather than clamping the report. Include fill and drain, especially when only a few chunks are processed. Useful transfer bytes remain delivered even when their latency overlaps compute; do not describe the percentage as bytes eliminated from the link. Record the confirmed host-link type or retain a generic H2D label. A faster overlap policy must still satisfy the pinned and CUDA staging budget and output correctness requirements.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def build_safe_transfer_compute_pipeline(spec, buffers, stream_plan):
    ...


def compare_overlap_and_latency_benefit(replays, timeline):
    ...


def E12(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export component durations, makespan, useful payload, overlap fraction, benefit estimate, assumptions, memory overhead, and correctness. Do not claim PCIe traffic vanished or silently clamp an implausible hidden percentage. If compute slowed under contention, include that cost. A faster pipeline still needs to satisfy its extra staging-capacity budget.

Part 7 - Engineering Interpretation and Philosophy

Choose whether overlap is worthwhile for the tested workload. Explain why transfer activity, exposed wait, and total throughput answer different runtime questions.

19.46.14 Exercise 13

Exercise Title: Run Concurrent H2D and D2H Transfers on Separate CUDA Streams and Determine Whether Bidirectional Transfer Engines Can Operate Simultaneously

Part 1 - Exercise Contract and Why It Matters

Run simultaneous H2D and D2H on separate CUDA streams and test bidirectional progress under the available hardware. Compare aggregate useful throughput with per-direction latency and a serialized reference.

Part 2 - System and Memory Mental Model

Two stream objects do not prove independent copy engines or full-duplex performance. Host memory, topology, staging, and device resources can limit concurrency. Aggregate throughput sums delivered bytes over a common completed makespan; summing two independently calculated rates can misrepresent the same interval. D2H destinations and H2D sources require separate safety rules.

Part 3 - Quantitative Prediction Before Execution

For payloads B_h, B_d and common duration T, define aggregate useful rate $(B_h + B_d)/T$ and retain each direction's completion latency. Compare ideal independent and serialized bounds under explicit assumptions. Predict a case where aggregate rate rises but one direction's tail latency worsens. State what trace evidence would establish simultaneous activity.

Part 4 - Experimental Design and Instrumentation

Preallocate disjoint pinned host and CUDA buffers with known contents. Use a common start anchor and join both stream completions before the end marker on the same device. Measure equivalent serialized and concurrent replays after warmup. Wait before CPU validation or buffer reuse. Capture a separate timeline to inspect overlap, not just launch order. Sweep matched sizes and preserve per-direction distributions. Do not infer exact copy-engine count solely from timing; use supported device evidence where available.

Use disjoint H2D sources and D2H destinations so the two directions do not create accidental data dependencies through one host buffer. Pin and initialize storage before the reusable-copy region. A common fork/join boundary defines aggregate useful rate, while each branch retains its own start/completion interval. Validate CPU results only after the D2H branch has completed. Record whether direction workloads are equal or deliberately asymmetric, and keep total work matched in the serialized control. An aggregate rate can exceed a single-direction rate because it includes both directions; it is not evidence that one direction exceeded the link's unilateral ceiling. Conversely, separate streams may still serialize or contend. Use traces and supported capability data before claiming simultaneous engine execution. Do not infer exact engine count from one timing curve. Preserve per-direction tail latency because an offload scheduler can maximize aggregate traffic while delaying the next required reload. Include host-memory bandwidth and staging as possible shared constraints without assigning causality unsupported by the measurements.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def build_bidirectional_fork_join(case, buffers, streams):
    ...


def calculate_direction_and_aggregate_rates(case, completed_rows):
    ...


def E13(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report direction bytes, individual completion, common makespan, aggregate rate, temporal overlap, host pinning, and errors. Keep unsupported hardware capability fields unknown. A throughput improvement without simultaneous trace evidence needs a cautious interpretation. Preserve no-overlap or slowdown outcomes and explain what the observed evidence can and cannot attribute.

Part 7 - Engineering Interpretation and Philosophy

Choose direction-specific and aggregate costs for concurrent offload. Explain why maximizing aggregate rate alone may violate a latency-sensitive reload requirement.

19.46.15 Exercise 14

Exercise Title: Saturate the Host-GPU Link with Multiple Concurrent Transfers and Determine the Point Where Additional Streams Stop Increasing Aggregate Bandwidth

Part 1 - Exercise Contract and Why It Matters

Sweep concurrent transfer streams and identify the tested point where additional streams cease improving aggregate completed throughput. Include memory capacity and per-request latency, not only a peak aggregate rate.

Part 2 - System and Memory Mental Model

More streams can expose concurrency or merely divide a fixed resource among more requests. Shared host memory and link bandwidth can saturate, while staging and queueing increase latency. Fixed total payload and fixed per-stream payload are different experiments. A plateau within a tested range does not establish a universal optimal stream count.

Part 3 - Quantitative Prediction Before Execution

Define a fixed-total-byte sweep first and a separate fixed-per-stream series. Calculate aggregate bytes over one fork/join makespan. Predict how per-request latency and outstanding capacity can grow even if aggregate rate stops increasing. Set a plateau criterion and maximum concurrency before collecting results.

Part 4 - Experimental Design and Instrumentation

Use bounded pinned and GPU buffer pools with distinct task/slot generations. Launch matched copies across candidate stream counts, establish a common start and joined completion, and validate every output. Keep direction, dtype, size distribution, warmup, preparation, and topology fixed within each series. Record host submission overhead, common completed time, per-request latency, and peak staging capacity. Profile separately for contention. Apply backpressure when pool capacity is reached rather than silently overwriting an in-flight buffer.

Define fixed total bytes before constructing the concurrency sweep. If each extra stream instead receives another full payload, report that as a separate growing-work experiment. Use a bounded stream/event/buffer pool and preserve a generation for each outstanding slot. Aggregate bytes are divided by one completed common makespan, not by summed branch times. Record CPU submission duration, queueing, and per-request completion so a throughput plateau does not hide rising latency. Predeclare the criterion for insignificant rate improvement, the maximum tested streams, and the latency/memory constraints. Repeat nearby candidate counts to assess variability. Host staging, link resources, and device memory can be shared even across different transfer streams. Preserve topology and pinning with the curve, because a calibrated concurrency point is contextual rather than universal. If the largest tested count still improves rate, report that the tested range did not locate a plateau. A stream count rejected for insufficient buffer capacity is not a measured bandwidth failure and should retain its own outcome.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def make_stream_concurrency_sweep(total_payload, resource_limits):
    ...


def benchmark_transfer_concurrency(case, buffers, timing_registry):
    ...


def select_tested_concurrency(rows, latency_and_memory_constraints):
    ...


def E14(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export stream count, total/per-stream bytes, common makespan, aggregate rate, latency tails, host overhead, pool capacity, and correctness. Do not sum overlapping durations to compute aggregate time. Report unresolved plateaus or variability honestly. If a higher stream count improves rate but exceeds the memory or latency bound, classify it infeasible for the declared objective.

Part 7 - Engineering Interpretation and Philosophy

Choose a concurrency limit from the joint rate, latency, and residency measurements. Explain why a modest limit can be preferable to the largest available stream count.

19.46.16 Exercise 15

Exercise Title: Build a Roofline-Style Experiment That Distinguishes Memory-Bound and Compute-Bound PyTorch Operations from Arithmetic Intensity and Measured Bandwidth

Part 1 - Exercise Contract and Why It Matters

Build a Roofline-style experiment with explicit arithmetic and byte models. Distinguish evidence consistent with memory limits from compute, launch, latency, and implementation bottlenecks.

Part 2 - System and Memory Mental Model

Arithmetic intensity is operations per byte at a named memory level. Requested device-memory bytes, cache traffic, and host-transfer bytes are not interchangeable denominators. A matrix multiplication can reuse data far more than a pointwise operation. Precision, Tensor Core policy, fusion, and actual kernel selection affect achievable compute. A point below a roof is not automatically bandwidth-bound.

Part 3 - Quantitative Prediction Before Execution

For F FLOPs and bytes Q, define $I = F/Q$ and the scoped roof

$$P \leq \min(P_{\text{compute}}, I\,\beta_{\text{memory}}).$$

Derive a pointwise add's requested reads/writes and a matrix product's ideal input/output byte model under stated reuse assumptions. Count fused multiply-add consistently. Compare empirical sustained ceilings with theoretical ones and preserve their different provenance.

Part 4 - Experimental Design and Instrumentation

Choose a bounded pointwise workload and matrix-multiplication sweep with known shapes, dtype, precision, and allocation policy. Validate numerical outputs. Measure warmed completed intervals and useful FLOPs; collect supported memory counters in a separate profiling replay if available. Calibrate an empirical bandwidth and compute reference under matching precision/resource conditions. Keep eager and compiled/fused paths separate. Vary working set, shape, and reuse to test the predicted limiting region. Do not use H2D bandwidth as a device-DRAM roof or count ideal minimum bytes as measured traffic.

Choose the memory level for each Roofline point before calculating intensity. Requested minimum bytes from a mathematical operation can differ from actual DRAM bytes because of tiling, cache reuse, temporary tensors, and fusion. Label analytical intensity accordingly. Keep compute ceilings matched to dtype and precision policy, including any Tensor Core or TF32 behavior, and separate vendor theoretical values from empirical sustained references. For an eager pointwise chain, account for intermediate passes rather than assuming it was fused. For a compiled version, establish the actual kernel/traffic evidence separately. Small operations can be dominated by launch or latency and remain far below both simple roofs. Use a shape or reuse sweep and a controlled intervention before assigning a cause. A copy bandwidth calibration is an empirical reference for its resource and pattern, not a universal ceiling for every kernel. If performance counters are unavailable, retain a model-based classification with uncertainty and

avoid claiming an exact physical-memory utilization percentage. Preserve numerical checks after precision or implementation changes.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def model_workload_flops_and_bytes(spec, memory_level, fusion_policy):
    ...


def construct_scoped_roofline(reference_measurements, workload_rows):
    ...


def classify_supported_bottleneck_evidence(point, uncertainty, trace):
    ...


def E15(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report FLOP convention, byte model, memory level, precision, ceilings, measured performance, counters, residuals, and correctness. Include a small launch-limited case that neither simple roof explains well. Use counterfactual shape/reuse changes to test a proposed bottleneck rather than assigning it from one chart point. Mark unknown actual traffic and unsupported metrics.

Check calibration compatibility before comparing a point with its roof. A matrix shape that exposes little parallelism can have a lower empirical compute reference than a larger shape of the same dtype. A cache-sensitive copy reference can also misrepresent a DRAM-limited kernel. Preserve these scope differences and report a classification as unresolved when the ceilings are not comparable. Do not strengthen a model-based classification into a causal claim merely because a plotted point lies near a line.

Part 7 - Engineering Interpretation and Philosophy

Explain the strongest bottleneck conclusion supported by the experiment. Choose whether reducing traffic, increasing reuse, changing granularity, or another measured intervention is the next useful test.

19.46.17 Exercise 16

Exercise Title: Measure the Transfer Cost of CPU Offloading and Calculate the Minimum Parameter Reuse Required to Make Keeping a Tensor in VRAM More Efficient

Part 1 - Exercise Contract and Why It Matters

Measure CPU offload/reload cost and derive a workload-specific residency break-even rule. Include authoritative-value policy, dirty writeback, reuse horizon, and the opportunity cost of occupying VRAM.

Part 2 - System and Memory Mental Model

An immutable parameter can retain a valid CPU copy after a reload; it does not require a fresh D2H writeback on every use. Modified state needs a version-aware dirty policy. With free VRAM and no adverse effect on other work, avoiding any unnecessary reload already saves latency; there is no universal positive reuse threshold. A meaningful threshold needs explicit admission cost, future misses, competing residency, or a throughput objective.

Part 3 - Quantitative Prediction Before Execution

Let resident incremental cost be $A + O(k)$ for admission and opportunity cost over k uses. Let the offloaded alternative cost $E + q(k)R$ for required eviction/writeback and reload misses. Compare these time quantities after canceling common compute. Derive a break-even only under a declared model for O and q. Keep memory feasibility as a separate byte constraint, not seconds subtracted from gigabytes.

If a threshold is derived, specify integer rounding and strict versus non-strict break-even. Evaluate the neighboring reuse counts explicitly on a synthetic trace before hardware testing. Include a trace with several uses per reload so reuse count and miss count differ. A policy that reloads once for an entire burst has a different threshold from one that evicts after every use, even when both bursts contain the same number of parameter accesses.

Part 4 - Experimental Design and Instrumentation

Use a small deterministic inference workload with read-only weights first. Measure complete offload/reload paths with reusable pinned staging and validated contents. Replay fixed access traces with different reuse counts and capacity pressure. Record which host version remains authoritative and which reloads are actually needed. Add a separately validated dirty-state case if implemented. Compare retained versus offloaded policies on identical useful work, including admission, writeback, staging, and future communication. Preserve next-use predictions before observing the trace outcome.

State the authoritative value at every transition. Immutable weights can keep a valid CPU backup and need only reloads after eviction; modified weights or KV-like blocks need a dirty/version policy. Count actual dirty writeback bytes rather than forcing a full D2H on every access. Define whether the resident and offloaded alternatives start from the same initial location and include their distinct admission costs. If both are feasible with free VRAM and no opportunity cost, retaining a tensor avoids any unnecessary reload, so a universal minimum reuse count is not implied. A finite break-even can arise when initial placement, displaced competing state, or different future misses create explicit extra costs. Keep those costs in seconds and capacity in bytes. Replay a known access trace and preserve the predicted reuse horizon before observing it. Compare complete useful execution, staging, and induced future communication. If the assumed horizon is wrong, report the policy miss rather than revising the prediction retrospectively. Safe version and buffer lifetimes remain mandatory even when the transfer-cost model recommends eviction.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def measure_offload_policy_trace(trace, buffers, authority_contract):
    ...


def derive_residency_break_even(costs, miss_model, opportunity_model):
    ...


def E16(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export uses, misses, versions, dirty bytes, copied bytes, measured path times, staging capacity, opportunity assumptions, and outcome correctness. Report no finite threshold or no useful threshold when the model implies it. Do not invent a universal reuse count. A resident policy that displaces more valuable state must include that induced cost rather than claiming all avoided transfer time is free benefit.

Part 7 - Engineering Interpretation and Philosophy

Choose a residency rule for a specified trace and budget. Explain which assumption would change the break-even and which uncertainty requires a conservative no-move decision.

19.46.18 Exercise 17

Exercise Title: Build a Runtime Bandwidth Model for NVMe-to-RAM, RAM-to-VRAM, GPU Memory, and GPU-to-GPU Paths and Predict the Bottleneck of a Three-Tier Offload Pipeline

Part 1 - Exercise Contract and Why It Matters

Build a path-specific cost model for file/NVMe-to-RAM, RAM-to-VRAM, device memory, and GPU-to-GPU movement. Predict a bounded three-tier pipeline's bottleneck and validate on held-out transfer sizes and schedules.

Part 2 - System and Memory Mental Model

Disk reads, page-cache hits, host staging, H2D, and device consumption have different resources and clocks. Ordinary PyTorch copies do not establish direct NVMe-to-GPU access. A file-to-RAM result may be dominated by OS cache rather than physical NVMe. GPU peer paths need the evidence from E08. Shared host memory, PCIe roots, or storage resources can invalidate an independent- stage overlap model.

Part 3 - Quantitative Prediction Before Execution

Fit scoped latency functions $t_j(B) = \alpha_j + B/\beta_j$ where data support them. Compare serial completion as a sum of required stages with an ideal steady pipeline bound based on the maximum resource service time per chunk. Add fill, drain, finite buffers, and shared-resource contention. Predict a case where the bottleneck shifts when chunk size or concurrency changes.

Part 4 - Experimental Design and Instrumentation

Use a bounded benchmark-owned file and verify storage identity before calling the path NVMe. Measure completed file reads with a monotonic host clock, recording cache-control limits. Do not flush global system caches or modify unrelated files. Calibrate CPU staging and CUDA paths separately, then validate the complete pipeline with checksums and explicit buffer leases. Use bounded pinned/CUDA pools and held-out sizes. Record actual edges, service times, resource sharing, bytes, versions, and cold/warm status. Unavailable paths remain unsupported/not measured.

Use benchmark-owned bounded files and validate offsets, lengths, and integrity before reading. Record whether storage identity is verified as NVMe and whether cache state is controlled, merely observed, or unknown. A warm page-cache read is a RAM-backed path even if the file resides on NVMe; do not label its measured rate physical storage throughput. Avoid global cache-flush actions and unrelated file mutation. File I/O and CPU packing use monotonic host timing, while CUDA legs use compatible completed events or broader host completion contracts. Keep one path model per direction, size range, layout, pinning, and concurrency. A serial sum and a steady-state pipeline maximum answer different questions and require fill/drain corrections for finite work. Include shared-resource constraints: two nominal stages can compete for one host-memory subsystem or interconnect root. Validate on held-out chunk sizes and bounded buffers, retaining checksum results and prediction residuals. Missing path support should produce a capability gap or conservative fallback, not a fabricated universal NVMe-to-GPU bandwidth.

Separate path latency from a pipeline's first-result and steady-throughput metrics. Record read submission, completed host-buffer readiness, CUDA-buffer readiness, and final consumer completion as distinct events. Validate partial final chunks with their actual byte counts; padding belongs to copied payload only when the path really transfers it. If a stage model assumes a full chunk while the measured last chunk is smaller, preserve that discrepancy rather than adjusting the denominator silently. Include finite-buffer stalls in the pipeline model and identify whether they arise from capacity or a shared resource.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch


@dataclass
class MemoryPathModel:
    source_tier: str
    destination_tier: str
    size_range_bytes: tuple
    context_identity: str
    overhead_seconds: object
    sustained_bytes_per_second: object
    uncertainty: object
```

```
def predict_three_tier_pipeline(path_models, schedule, resource_graph):
    ...


def E17(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report raw path samples, fit ranges/residuals, file-cache evidence, complete pipeline makespan, memory bounds, correctness, and bottleneck prediction errors. Do not equate page-cache throughput with NVMe bandwidth or sum isolated bandwidths to obtain a pipeline rate. Preserve shared-resource uncertainty and distinguish initial latency from steady useful throughput.

Part 7 - Engineering Interpretation and Philosophy

Choose the chunking and prefetch decision supported by the calibrated resource graph. Explain why the slowest isolated path is not always the observed pipeline bottleneck.

19.46.19 Exercise 18

Exercise Title: Stress-Test the Bandwidth Profiler with Concurrent Prefetching, Eviction, KV Transfers, Multi-GPU Communication, and GPU Compute and Identify Link Contention

Part 1 - Exercise Contract and Why It Matters

Stress the profiler with prefetch, eviction, KV-like copies, optional multi-GPU communication, and compute. Identify contention through matched replays and supported trace evidence while preserving data validity.

Part 2 - System and Memory Mental Model

Concurrent tasks can share host memory bandwidth, interconnect roots, copy resources, device DRAM, or CPU submission capacity. Slowdown alone does not uniquely identify the shared link. KV or dirty-state transfers need stable versions and buffer lifetimes; moving mutable data without an ownership protocol can corrupt results. Heavy collectives require a validated rank/group contract and must not be introduced by mismatched launch order.

Part 3 - Quantitative Prediction Before Execution

Define isolated, pairwise, and combined bounded cases with fixed useful bytes and compute. Predict which resource may be shared and what measurement would distinguish it from another cause. Include a case with rising aggregate rate but worse reload-tail latency. Calculate staging and event-pool capacity before launching contention tests.

Part 4 - Experimental Design and Instrumentation

Validate each component independently, then change one concurrent load at a time. Use disjoint versioned buffers, explicit ready/last-reader events, and fixed seeds. Keep optional collective groups and operation order consistent. Measure completed per-path and common makespan distributions, with profiling in separate replays. Record topology, directions, sizes, pinning, shape, dtype, concurrency, and pressure mechanism. Use bounded application quotas or retained buffers with honest labels. Recover safely after failure before another comparison, preserving failed samples and configurations.

Create isolated calibrations, then matched pairwise and combined replays with fixed useful work. Keep tensor versions stable while a transfer reads them; mutable KV data requires a snapshot or an explicit producer-consumer handoff before offload. Use separate slots for incoming prefetch and outgoing eviction, with generations and final-reader events before reuse. Record whether a slowdown affects one direction, all transfers, compute, or host submission. That pattern can guide investigation but does not alone identify a physical shared link. Optional collectives require consistent rank groups and launch order before adding contention; unavailable multi-GPU execution remains not measured. Distinguish actual retained pressure allocations from application quotas or injected delays. Preserve latency tails and aggregate rate because a high total throughput can coexist with unacceptable reload stall. After asynchronous failure or timeout, do not free in-flight storage until a safe completion or recovery path resolves it. Minimize counterexamples by removing one concurrent load at a time while keeping shapes, values, and the remaining dependency graph unchanged.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def build_bandwidth_contention_cases(capabilities, resource_limits):
    ...


def validate_versioned_transfer_lifetimes(case, task_registry):
    ...


def compare_isolated_and_contended_paths(replays, evidence):
    ...


def E18(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export useful bytes, path/context, isolated and contended distributions, aggregate rate, tails, pressure, peaks, errors, and failure phase. Do not call every slowdown PCIe saturation or fragmentation. Include a minimal counterexample to an independence assumption, or state none was observed. Unavailable devices and unsupported counters remain explicit rather than borrowed hardware results.

Part 7 - Engineering Interpretation and Philosophy

Explain the strongest supported contention attribution and its remaining uncertainty. Choose a scheduling or residency change that targets the observed resource conflict without violating data lifetimes.

19.46.20 Exercise 19

Exercise Title: Build and Benchmark a PyTorch Bandwidth-Aware Runtime Profiler That Measures Every Memory Path and Feeds Transfer-Cost Estimates into Dynamic Placement and Offloading Decisions

Part 1 - Exercise Contract and Why It Matters

Integrate a reusable path-aware bandwidth profiler that supplies latency and transfer-cost estimates to placement/offload decisions. Benchmark complete correct useful work and instrumentation overhead under explicit resource limits.

Part 2 - System and Memory Mental Model

A runtime model needs direction, size, dtype, layout, pinning, topology, cache policy, concurrency, timing scope, and age. One scalar bandwidth cannot represent every memory path. Supported paths require validating runs; unavailable NVMe, peer copies, custom kernels, counters, or collectives remain explicit capability gaps. A prediction is a scoped estimate rather than a reservation of bandwidth or future completion time.

Part 3 - Quantitative Prediction Before Execution

Predict pool capacity, pending tasks, event overhead, and calibration cost before integration. Define held-out sizes, acceptable error, timing validity, and a concrete decision such as batch size, prefetch distance, or residency choice. Compare fixed-rate and path-aware models on identical traces. Keep feasibility in bytes and latency/throughput objectives in compatible units.

Part 4 - Experimental Design and Instrumentation

Validate unit conversions, copy correctness, event domains, and lifetimes before collecting calibration data. Use separate correctness, profiling, ordinary timing, and contention replays. Preserve E00's record plus path schemas, fit ranges, generations, capacity limits, and authority policies. Export bounded CPU evidence. If testing training, time complete forward, backward, optimizer update, and required completion rather than only the favorable transfer region. Include calibration/monitoring overhead in adaptive throughput. Reject stale or unsupported cost queries conservatively.

Publish a capability matrix for H2D/D2H, same-device D2D, verified or unverified peer candidates, CPU-staged copies, file/RAM paths, layout tests, optional controlled-lane kernels, traffic counters, concurrency, and Roofline analysis. Each supported entry needs a validating run ID. A method signature or available import is not proof that every path was measured. Store model fit range, timing scope, byte convention, sample count, context, uncertainty, and age with each cost estimate. Test held-out queries and preserve residuals rather than silently expanding a calibration's validity. Compare the concrete runtime decision against a fixed-rate baseline on the same trace, including monitoring and calibration overhead. Keep memory feasibility separate from latency objectives and reject unsupported or stale queries conservatively. A requested-byte rate, temporal overlap percentage, and physical traffic counter answer different questions and should remain separately labeled. The final conclusion should identify the best tested decision, its complete cost, and its capability limits without promising that every memory path shares one bandwidth model or that theoretical ceilings predict application throughput exactly.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


class BandwidthAwareRuntimeProfiler:
    def validate_path(self, path_spec, capabilities):
        ...

    def collect_completed_samples(self, workload, contract):
        ...

    def fit_scoped_path_model(self, sample_rows):
        ...

    def predict_transfer_cost(self, request, execution_context):
        ...

    def export_cpu_evidence(self):
        ...


def E19(config):
    # Integrate validated profiling and test a concrete runtime decision.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Keep prediction, observation, and interpretation separate. Each run needs ID, configuration, shapes, bytes/convention, path, budgets, peak allocated and reserved memory, raw timing scopes, distributions, correctness, outcome, and failure phase. Report rejected records, model residuals, overhead, and the smallest counterexample to a universal bandwidth rule, or state none was observed. Do not invent hardware outcomes. Every supported feature needs a validating run ID, not merely a scaffold method.

Part 7 - Engineering Interpretation and Philosophy

Justify the selected runtime decision through correct values, completed measurements, scoped cost models, and end-to-end benefit. Identify untested conditions that could change its prediction.

19.47 DMM45

Chapter Title: CUDA Memory Leak Detection and Root-Cause Analysis
Specialization: Dynamic Memory Management and CUDA Systems

19.47.1 Exercise 00

Exercise Title: Build a Precise Mental Model of True Memory Leaks, Tensor Retention, Allocator Caching, Autograd Graph Retention, and Non-PyTorch CUDA Allocations

Part 1 - Exercise Contract and Why It Matters

Define a leak investigation contract that separates unintended retention, allocator caching, graph structure, saved tensors, pending CUDA work, and allocations outside the PyTorch allocator. Deliver hypotheses and falsifiable tests rather than diagnosing every rising VRAM graph as a true leak.

Part 2 - System and Memory Mental Model

A reachable Python container retaining tensors is unintended application retention when the workload no longer needs them; it is not necessarily an unreachable native allocation leak. Reserved cache can remain after live storage is released. Backward normally releases saved tensors when their retention is not requested, even if a Python tensor still references graph structure. External libraries and other processes can affect device-wide readings outside PyTorch's allocator visibility.

Part 3 - Quantitative Prediction Before Execution

Draw reference ownership, storage aliases, autograd nodes, saved tensors, and stream readers for one iteration. Predict current live bytes, reserved capacity, and historical peak under steady training, growing history, and one-time optimizer initialization. Define a matching completed measurement phase and a bounded test range. Explain why a peak counter alone cannot establish a persistent growth trend.

Part 4 - Experimental Design and Instrumentation

Use a small deterministic FP32 model and fixed input shape. Choose supported Adam settings and include forward, backward, and optimizer update. Freeze gradient-clearing and loss semantics. Record GPU identity, versions, allocator backend, seed, shape, optimizer, warmup, phase, stream completion, process mode, and initial state. Keep diagnostic CPU histories bounded and avoid GPU objects in evidence logs. Pin history/snapshot APIs to installed capabilities, including documented underscore-prefixed diagnostic interfaces whose schemas can change.

Create a classification table for reachable tensor history, a reference cycle, an intentionally bounded cache, pending stream release, first-update optimizer state, and an external allocator. For each, state what remains owned, which counter can see it, and which intervention would test the hypothesis. Keep the word leak qualified until ownership and intended lifetime are established. A Python object can disappear while another view keeps the backing allocation alive, and a native allocation can remain without a visible Python tensor. Draw those cases separately. Choose one fixed completed training phase for trend detection and preserve unsynchronized samples only under an explicit pending-work contract. The investigator must not change loss weighting, batch size, or completed update count to obtain a flatter curve. Keep memory limits and stop conditions independent of whether the expected bug has appeared. The strongest conclusion is a scoped causal explanation supported by a minimal repair and matched useful work, with unknown visibility left unresolved rather than assigned to a convenient allocator or operator name.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch


@dataclass
class LeakInvestigationContract:
    measurement_phase: str
    warmup_policy: str
    resource_limit_bytes: int
    iteration_limit: int
    allocator_backend: str
    expected_persistent_state: object


def classify_memory_growth_evidence(samples, contract, ownership):
    ...


def E00(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Keep prediction, observation, and interpretation separate. Require same-phase evidence, owner or lifetime clues, and a controlled intervention before claiming root cause. Unknown fields and unrun GPU cases remain explicit. A diagnostic system must not retain the graphs it is trying to investigate.

Part 7 - Engineering Interpretation and Philosophy

Explain the distinction between allocation origin and the object that keeps it alive. Choose the smallest evidence set that can distinguish the competing explanations without overstating certainty.

19.47.2 Exercise 01

Exercise Title: Build a Stable PyTorch Training Loop and Establish the Expected Allocated, Reserved, and Peak VRAM Pattern Across Hundreds of Iterations

Part 1 - Exercise Contract and Why It Matters

Establish a stable fixed-shape training baseline over a bounded run of hundreds of iterations. Measure expected allocated, reserved, and peak patterns at named phases before injecting retention bugs.

Part 2 - System and Memory Mental Model

Adam state can appear lazily, gradients can be allocated or cleared, and kernels or allocators can warm up. A stable baseline is phase-dependent, not a requirement that every reading remain

identical from process start. Historical peaks are nondecreasing unless reset. Reserved cache can plateau above live payload. Variable shapes, compilation, distributed initialization, and legitimate caches need separate baselines.

Part 3 - Quantitative Prediction Before Execution

Inventory initialized parameters, gradients, moments, buffers, and expected transient activations. Predict first-update changes and a warmed post-update/post-clear range. Define the interval used for each peak and the evidence needed for a plateau. Predict a legitimate staircase or one-time jump that a naive positive-slope detector could misclassify.

Part 4 - Experimental Design and Instrumentation

Use fixed seeded batches, model dimensions, precision, optimizer flags, and gradient-clearing policy. Run a small numerical reference first. Sample current counters at a common completed phase and record per-iteration region peaks separately. Reset peaks only for the declared interval. Warm all used optimizer state before the primary steady window. Export CPU scalar rows, not tensors. Measure monitoring overhead against an uninstrumented replay. Repeat fresh processes and a secondary seed while preserving the primary initial-state identifier.

Use a small model whose parameters and initialized Adam moments can be counted from shapes and actual dtypes. Include a first iteration where those moments do not yet exist and a warmed window where they do. State whether gradients are set to None or zeroed in place, and keep that choice fixed. Sample before forward, after backward, after update, and after clearing in a diagnostic replay, then select one common phase for ordinary monitoring. The peak for an iteration must have a declared reset boundary; the process- lifetime peak cannot fall and should not be fitted as live use. Bound CPU sample history even when hundreds of iterations are requested. A fresh process helps isolate retained Python state but does not guarantee identical external GPU load. Record that context. Validate gradients and updates before introducing diagnostic changes, because a lower memory footprint obtained by skipping backward is not a stable training control. Report any legitimate state initialization that falls outside the chosen warmup rather than erasing it from the raw trace.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def run_stable_training_baseline(config, measurement_contract):
    ...


def sample_completed_iteration(runtime, iteration_id, phase):
    ...


def E01(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report raw current/peak allocated and reserved bytes, initialized-state inventory, reset boundaries, sample age, update count, and numerical checks. Do not fit startup and warmed samples as one unexplained trend. If no stable window is reached, retain that result and investigate shape/state assumptions before labeling a leak. Include the expected allocator noise range and unsupported backend counters.

Part 7 - Engineering Interpretation and Philosophy

Explain what stable means for this declared workload. Preserve the baseline as the control for every later leaking and repaired replay.

19.47.3 Exercise 02

Exercise Title: Introduce a Deliberate CUDA Tensor Leak by Appending GPU Tensors to a Python List and Detect the Linear Memory Growth

Part 1 - Exercise Contract and Why It Matters

Introduce bounded unintended retention by appending independent CUDA tensors to a Python list. Detect same-phase live-memory growth and verify the container's ownership with a controlled release.

Part 2 - System and Memory Mental Model

A list can keep tensor storage reachable after the iteration's computation ends. Appending views or the same object repeatedly does not necessarily add independent storage, so the expected slope depends on actual backing ownership. Allocator rounding and segment growth affect counters differently. This is a deliberately bounded reproducer, not a request to exhaust the GPU.

Part 3 - Quantitative Prediction Before Execution

For n independent retained tensors with payload b bytes, predict useful retained bytes nb before rounding and metadata. Compare with repeated references to one tensor and small views of a larger parent. Define a stop limit below the declared resource budget. Predict current allocated and reserved trajectories and what changes after clearing the only owner.

Part 4 - Experimental Design and Instrumentation

Start from E01's stable training. Inject independent allocation retention at a named diagnostic point with fixed size and a hard iteration/byte cap. Record list length, unique backing identities, useful capacity, and same-phase memory. Validate numerical training is unchanged. In a separate intervention, remove the container ownership and complete required stream work, then observe live and cached counters. Keep ownership metadata on CPU without adding another strong GPU reference. Compare baseline, leaking, and release replays from the same logical initial state.

Choose a retained tensor size that produces a detectable payload trend without approaching physical exhaustion. Derive a hard maximum list length from the application budget before the run. Each appended object should have independently allocated storage in the primary reproducer; record that fact rather than assuming a new Python wrapper means new bytes. Add controls that append the same tensor repeatedly and that append small views of one large parent. Their list lengths can grow while their unique backing behavior differs. Release the

list under a named intervention and remove other diagnostic strong references before sampling. A remaining loop variable can retain the last item legitimately, so account for it rather than requiring the counter to return to an impossible exact baseline. Complete pending readers before interpreting reclamation. Keep training values and optimizer progress matched between leak and release variants. The result should identify owner-driven live storage growth even if reserved capacity changes only in coarse steps or remains flat during the bounded test.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def inject_bounded_tensor_history(history, tensor_spec, budget):
    ...


def audit_history_backing_ownership(history, observation_contract):
    ...


def E02(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report predicted payload slope, observed live slope, reserved steps, unique storage count, ownership intervention, and correctness. Do not require reserved bytes to drop immediately after release. A remaining alias can invalidate the expected release and needs investigation. Preserve cases where rounding masks a short-window slope or the tested range does not reach an allocator growth boundary.

Part 7 - Engineering Interpretation and Philosophy

Explain why reachable retention is sufficient to cause operational memory failure without proving a native unreachable leak. Identify the owner whose lifecycle the repair must change.

19.47.4 Exercise 03

Exercise Title: Fix the Leaking Tensor History with detach(), CPU Transfer, or Scalar Extraction and Prove That VRAM Usage Stabilizes

Part 1 - Exercise Contract and Why It Matters

Repair GPU history according to the diagnostic requirement and prove same-phase live memory stabilizes within the tested range. Compare detachment, CPU export, scalar extraction, and bounded retention under explicit contracts.

Part 2 - System and Memory Mental Model

detach severs gradient history but shares the original storage. Appending large detached CUDA tensors indefinitely still retains GPU bytes. A detached view can also keep a large backing allocation. For CPU diagnostics, use a graph-free export such as detach followed by CPU transfer, or a scalar extraction when only one value is needed. CPU history itself needs a retention budget; relocating an unbounded history changes the resource problem rather than making it disappear.

Part 3 - Quantitative Prediction Before Execution

Predict graph and storage ownership for original outputs, detached CUDA outputs, detached CPU copies, and CPU scalars. Define an expected plateau for a bounded GPU ring and growing host storage for an unbounded CPU archive. Identify which repairs satisfy the chosen diagnostic semantics rather than merely lowering one memory counter.

Part 4 - Experimental Design and Instrumentation

Replay the same workload and history insertion site with each policy. Keep detach-only as a negative control for independent large CUDA payloads. Use bounded CPU summaries or a bounded detached GPU ring for valid repairs. Wait appropriately before consuming asynchronous host copies and preserve source lifetime. Measure both CUDA and relevant host capacity, logging latency, and result equivalence. Complete hundreds of bounded warmed iterations where feasible. Record same-phase current bytes and distinguish retained cache from live payload after old GPU references are released.

State the diagnostic requirement before selecting a repair. If only a scalar metric is needed, storing a whole output tensor is unnecessary; if images or embeddings are needed, a bounded archive or streaming export has a different resource cost. Keep detach-only on a list of independent large CUDA values as a negative control. A clone of a detached CUDA tensor can create additional independent storage and does not solve unbounded history. For CPU export, sever unwanted gradient history and verify destination lifetime and completion before consuming it. CPU memory must also be bounded or intentionally archived; a flat CUDA curve with unbounded RAM is not a complete resource repair. Scalar extraction may block the host, so measure its logging overhead separately from ordinary GPU work. Confirm metric values and model updates still match the reference. Report the plateau only for the tested fixed-shape range and name the retention policy that makes it plausible, instead of claiming the absence of every possible leak from one flat graph.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def export_graph_free_diagnostic(value, policy, history_budget):
    ...


def verify_repaired_history_plateau(replay_rows, baseline, contract):
    ...
```

```
def E03(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report storage/graph policy, history bounds, CUDA slope, host retention, logging cost, and numerical checks. Do not claim detach alone frees storage or that CPU transfer alone necessarily removes gradient references. Scalar extraction can synchronize the host; include its performance cost. State plateau evidence for the tested range rather than proving universal absence of leaks.

Part 7 - Engineering Interpretation and Philosophy

Choose the smallest diagnostic representation that preserves needed information. Explain whether the repair removes ownership, removes graph history, bounds retention, or moves data to another resource.

19.47.5 Exercise 04

Exercise Title: Create an Accidental Autograd Graph Leak by Accumulating Loss Tensors and Explain Why Every Iteration Retains Previous Computation Graphs

Part 1 - Exercise Contract and Why It Matters

Reproduce loss-accumulation graph retention with explicit backward timing. Distinguish retained graph structure from retained saved activations and identify which lifecycle produces the measured CUDA growth.

Part 2 - System and Memory Mental Model

Accumulating differentiable losses links iterations through autograd. If backward is deferred over a growing sum, saved tensors can remain live until that backward. If ordinary backward already ran each iteration without retention, saved buffers are normally released; keeping loss graph references does not imply every previous activation remains resident. A later backward through those freed graphs can fail. The title's growth hypothesis therefore needs a declared execution variant, not a universal claim.

Part 3 - Quantitative Prediction Before Execution

Draw graphs for deferred aggregate backward and for per-iteration backward followed by logging-style accumulation. Predict nodes, scalar storage, and saved tensors separately. Define a bounded window for the deferred case and compare one aggregate gradient with correctly normalized sequential accumulation. Explain why resetting only the Python variable can leave other references alive.

Part 4 - Experimental Design and Instrumentation

Use a tiny deterministic model and fixed data. Run separate bounded variants: deferred loss-sum backward, ordinary per-step backward with retained loss references, and graph-free logging. Keep optimizer updates only at the correct objective boundary for each valid training comparison. Measure same-phase CUDA counters plus graph/owner metadata and supported saved-tensor evidence. Validate gradients and updates against the matching serial objective. Do not execute a second backward through freed graphs as if it were a valid repair; classify its expected failure separately.

Define two different valid objectives rather than mixing them in one training comparison. Deferred aggregate loss over a bounded batch/window needs a corresponding normalized full-window reference and an optimizer step after its required backward. Ordinary per-iteration training computes and backpropagates each loss before the next update; retaining its output for logging is a separate ownership decision. In the latter case, graph nodes can remain referenced while default backward has released saved intermediates. A later backward on the accumulated logging total can therefore fail instead of providing a valid aggregate gradient. Do not use that failure as a reason to add retain_graph everywhere. Measure graph metadata and saved CUDA storage under their different scopes. Reset the aggregate and release all owners at the bounded-window end, checking aliases and pending work. Preserve cases where host metadata grows more visibly than CUDA bytes. The exercise should show which lifecycle retains which objects, not repeat the false universal claim that every surviving loss keeps every previous activation alive.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def build_bounded_loss_accumulation_case(config, backward_policy):
    ...


def inspect_loss_graph_and_saved_state(case, observation_contract):
    ...


def E04(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report backward order, retain policy, objective, graph references, current live bytes, host growth, and gradient correctness. Avoid attributing all CUDA growth to graph nodes without storage evidence. Preserve a per-step case with little CUDA growth if that is observed. Show the intervention that releases the relevant owner and compare with graph-free metrics.

Part 7 - Engineering Interpretation and Philosophy

Explain precisely which objects survive and why. A useful diagnosis must distinguish an expanding differentiation history from full saved-activation retention.

19.47.6 Exercise 05

Exercise Title: Compare running_loss += loss, running_loss += loss.item(), and running_loss += loss.detach() by Graph Retention and CUDA Memory Growth

Part 1 - Exercise Contract and Why It Matters

Compare the three running-loss expressions under one explicit logging contract. Measure graph references, live CUDA bytes, host state, and synchronization cost without assuming every differentiable accumulation retains all activations.

Part 2 - System and Memory Mental Model

A Python numeric accumulator updated with loss.item stays graph-free and does not retain a CUDA loss tensor, but extraction can synchronize. An in-place scalar accumulator updated with detached losses can remain a constant-sized CUDA tensor if no history stores each detached value. Updating with differentiable losses can extend graph structure. After ordinary per-step backward, saved buffers may already be freed even though the accumulation still links nodes. These behaviors differ from a list of large outputs.

Part 3 - Quantitative Prediction Before Execution

Define accumulator initialization, dtype, device, in-place behavior, and reset boundary. Predict graph connectivity and persistent scalar storage for each expression. Separate a fixed scalar accumulator from an append-only history. Include a graph-retaining deferred-backward variant only as a separately named experiment. Predict host synchronization differences without inventing measured speedups.

Part 4 - Experimental Design and Instrumentation

Use the same model, data, per-step backward, optimizer, and clearing policy across the primary logging variants. Never backpropagate the running logging sum as an extra training loss. Inspect requires_grad/grad_fn and supported owner metadata without retaining new GPU objects. Measure warmed same-phase bytes and logging latency separately. Compare gradients and several updates to confirm logging has not changed training. Reset accumulators at a named epoch or window boundary and preserve valid scalar totals.

Initialize each running accumulator explicitly so the comparison does not depend on an accidental Python-to-tensor promotion. Record its device, dtype, requires_grad state, and update form. A constant-sized detached CUDA scalar updated in place differs from a Python list that stores every detached loss. Likewise, a Python float total has no CUDA payload but an external history of Python objects still has a host-memory lifecycle. Use per-step loss backward in the primary logging comparison and never backpropagate the running diagnostic total afterward. Verify accumulated scalar values under a declared tolerance, since reduction order and scalar dtype can differ. Inspect model gradients before Adam and moments after several updates to ensure the logging variants did not change the objective. Keep item-related host synchronization visible in end-to-end timing. If differentiable accumulation extends graph structure but saved activations were already released, report that exact result rather than forcing the expected large CUDA slope. The repair choice should follow the needed diagnostic representation and performance budget.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def define_running_loss_case(expression_policy, reset_contract):
    ...
```

```
def compare_running_loss_variants(cases, reference, measurements):
    ...


def E05(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report initialization, expression, graph state, scalar capacity, history policy, CUDA/host trend, logging time, and numerical totals. Do not call detached scalar accumulation an unbounded CUDA leak without evidence. Do not claim item is free. If graph metadata grows mainly on the host, record that scope rather than inventing large retained activation bytes.

Part 7 - Engineering Interpretation and Philosophy

Choose the logging policy according to required diagnostics and acceptable synchronization. Explain why graph-free and storage-free are different properties.

19.47.7 Exercise 06

Exercise Title: Leak Model Outputs Through a Metrics or Debug Dictionary and Trace the Retained CUDA Storage Back to the Owning Python Container

Part 1 - Exercise Contract and Why It Matters

Retain model outputs through a metrics/debug dictionary and identify the owning application path. Repair the lifetime while preserving required metrics and numerical training.

Part 2 - System and Memory Mental Model

A fixed dictionary key overwritten each iteration differs from a key per iteration or a nested append-only list. Values can share backing with large tensors, and aliases elsewhere can survive dictionary cleanup. Allocation traces locate creation but do not enumerate Python owners automatically. Introspection can itself create references, so ownership evidence needs a bounded, nonretaining design.

Part 3 - Quantitative Prediction Before Execution

Construct fixed-key, growing-key, and nested-history cases. Predict unique backing bytes rather than summing every tensor name. Add a small-view case that retains a large output backing. Define the exact application owner path and a repair that bounds or exports the needed diagnostic. Predict why clearing one container might not release storage if another alias exists.

Part 4 - Experimental Design and Instrumentation

Instrument insertion with logical output IDs, owner path strings, iteration, and storage metadata exported to CPU. Use bounded weak-reference or application-registry checks where appropriate, recognizing wrapper lifetime is not identical to backing lifetime. Compare baseline, leak, and repaired replays with the same training objective. Release diagnostic traversal locals

before sampling. Use controlled container removal and an alias negative control. Avoid printing whole CUDA values or retaining gc/referrer results inside the investigation report.

Instrument the actual ownership path: a top-level debug dictionary can contain a nested list, a callback closure, or a per-request object that outlives training. Record path strings and logical output generations at insertion rather than storing another tensor in the registry. A fixed-key overwrite can be bounded even though the tensor object changes every iteration, while an iteration-keyed dictionary can grow. Count unique backing capacity and preserve alias relationships. In a small controlled case, remove one owner while keeping another alias to show why one cleanup is insufficient. Weak references can help observe a Python wrapper's lifetime, but their expiry does not prove a shared storage has no remaining owners. Referrer tools can expose their own temporary frames and containers; release traversal results before measuring and avoid retaining them in reports. Keep value formatting on CUDA outside ordinary timing. The causal test is a bounded/exported ownership policy that removes the cohort's growth while preserving the same metrics and training updates.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def record_container_insertion(owner_path, value, iteration, registry):
    ...


def inspect_declared_container_ownership(container_spec, registry):
    ...


def repair_metrics_retention(policy, diagnostic_contract):
    ...


def E06(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report owner paths, key/history growth, unique backing, graph policy, release intervention, same-phase trend, and update errors. Do not infer all owners from a snapshot address. If introspection is incomplete, mark the unresolved path. Verify the investigation's registry contains metadata rather than the retained GPU values themselves.

Part 7 - Engineering Interpretation and Philosophy

Explain how a correct metric implementation can become an ownership bug. Choose a diagnostic representation whose retention is intentional and bounded.

19.47.8 Exercise 07

Exercise Title: Create a Forward Hook That Accidentally Retains Activations Across Iterations and Repair the Hook Without Losing Required Diagnostics

Part 1 - Exercise Contract and Why It Matters

Create a forward hook that retains activation storage across iterations, then repair its diagnostics and hook lifecycle. Demonstrate numerical equivalence and bounded retention after the change.

Part 2 - System and Memory Mental Model

A hook can capture outputs in a closure, append them to external history, or be registered repeatedly. Removing one handle does not clear already retained values or other registered hooks. Detaching a captured CUDA activation removes graph history but still retains its backing. Hooks should not accidentally return a replacement output or modify training values while attempting to observe them.

Part 3 - Quantitative Prediction Before Execution

Predict history and registration growth for one persistent hook versus one new hook per iteration. Calculate retained payload for independent activations and shared-view cases. Define a bounded CPU summary or bounded activation policy that preserves the intended diagnostic. Specify registration, handle removal, history cleanup, and last-reader completion as separate lifecycle events.

Part 4 - Experimental Design and Instrumentation

Register the controlled hook once and record owner metadata at insertion. Use a fixed deterministic model and hard run/memory limits. Validate the leak pattern, then repair capture/export and guarantee handle cleanup under the runtime's lifecycle. Test repeated setup/teardown to catch duplicate registration. Compare outputs, gradients, and several updates with the no-hook reference. Measure hook overhead separately from ordinary training. Ensure asynchronous CPU exports use safe completion before consumption and bounded staging capacity.

Separate hook registration count from captured-value count. A single correctly registered hook can leak through an unbounded closure history, and repeatedly registering even a bounded hook can multiply work and owners. Keep handles in an explicit lifecycle registry and test cleanup on normal and exceptional exit paths. Removing a handle prevents future callbacks; it does not automatically clear old captured values. The primary hook observes output and returns no replacement, avoiding accidental modification of forward semantics. Record a tiny view case that keeps a larger activation backing alive. Compare bounded graph-free CPU summaries with detached CUDA history under their real storage costs. If asynchronous export is supported, retain the source until copy completion and bound staging as well as history. Run repeated setup/teardown to expose duplicate hooks and verify callback counts. Report diagnostic overhead and numerical equivalence. Do not accept a flat curve obtained by removing a hook that was required for actual model computation; the primary diagnostic hook must be observational.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def install_bounded_diagnostic_hook(module, diagnostic_policy, registry):
    ...


def remove_hook_and_owned_history(handle, owner_registry, completion):
    ...


def E07(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export handle identities, registration count, owner history, storage policy, CUDA trend, overhead, and numerical checks. Removing a hook without clearing its retained history is not a complete repair. A detached CUDA-history variant is a negative control, not proof of released storage. Preserve any untracked closure ownership as unresolved.

Part 7 - Engineering Interpretation and Philosophy

Explain the hook's ownership chain and cleanup proof. Choose diagnostics that observe the model without extending activation lifetimes unnecessarily.

19.47.9 Exercise 08

Exercise Title: Reproduce Memory Growth from Repeated retain_graph=True Usage and Determine When Retaining the Graph Is Actually Necessary

Part 1 - Exercise Contract and Why It Matters

Reproduce bounded graph retention using retain_graph=True together with an explicit surviving owner. Determine when repeated differentiation genuinely requires retention and when normal graph release or recomputation is preferable.

Part 2 - System and Memory Mental Model

retain_graph preserves saved state for another traversal of a graph; it does not by itself prove an indefinitely growing leak if no references survive and the graph becomes unreachable. Fresh graphs stored each iteration with retained saved tensors can accumulate substantial memory. create_graph is a different higher-order differentiation choice with its own reference risks. Parameters must not be updated while a later backward still requires their old graph version.

Part 3 - Quantitative Prediction Before Execution

Compare one graph traversed twice with bounded lifetime, fresh retained graphs held in a growing container, and fresh graphs released normally. Predict saved-tensor and graph-structure lifetimes. Define a legitimate multi-loss derivative calculation and its final releasing backward. Contrast with the accidental use of retention to suppress an unrelated autograd error.

Part 4 - Experimental Design and Instrumentation

Use a tiny deterministic model with a hard graph-count budget. Validate required gradients against a matching reference before timing memory. Keep the optimizer step after all necessary backwards for that parameter version. For the accumulating case, preserve explicit owners and measure same-phase state; for the release control, remove owners and complete pending work. Do not use in-place mutation or a second backward through freed state as a valid training path. Treat higher-order cases as optional separately validated support.

Use one legitimate graph with two required derivative traversals as a bounded control. Preserve its parameter version until both traversals are finished, then use the appropriate release and remove its Python owners. Compare with a case that stores a fresh graph each iteration after requesting saved-state retention. Both the flag and the surviving ownership policy belong in the report. If no references remain, do not assume retain_graph alone creates an infinite collection of graphs. create_graph introduces higher-order differentiation semantics and possible cycles that need separate validation; it is not merely another spelling of retain_graph. Inspect saved state through supported diagnostics rather than private grad_fn fields treated as a universal schema. Never update a weight between backwards that still need the old value, and do not mask a version-counter error by disabling checks. Preserve the exact gradient objective and compare multiple updates. A successful repair removes unnecessary retention or recomputes deliberately, while retaining any derivative the algorithm genuinely requires.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def make_retained_graph_case(config, ownership_policy):
    ...


def validate_required_backward_lifecycle(case, reference):
    ...


def E08(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report graph count, surviving owners, retain/create settings, saved state, parameter version, live bytes, and derivative correctness. Include a control where retain_graph does not yield unbounded growth. Do not recommend retention as a generic fix for an autograd error. Verify final release of the legitimate multi-traversal case separately from the leaked-container case.

Part 7 - Engineering Interpretation and Philosophy

Explain which differentiation requirement justifies retention and which application owner makes it persist. A useful repair changes that lifecycle without discarding a required derivative.

19.47.10 Exercise 09

Exercise Title: Build an Iteration-Level Leak Detector That Fits a Memory-Growth Trend and Flags Persistent VRAM Growth Above Expected Allocator Noise

Part 1 - Exercise Contract and Why It Matters

Build an iteration-level detector for persistent same-phase memory growth. Fit scoped trends, flag candidate anomalies, and evaluate false alarms against legitimate initialization, cache growth, and variable shapes.

Part 2 - System and Memory Mental Model

A positive slope can reflect intended state, pending work, changing shapes, or instrumentation. Reserved-memory steps and historical peaks do not equal live leaks. Short noisy windows can hide small retention or overreact to one jump. The detector identifies suspicious growth, not root cause or proof of future OOM.

Part 3 - Quantitative Prediction Before Execution

For warmed current bytes A_k at a fixed phase, define a trend model $A_k \approx \alpha + \beta k$. Keep bytes per iteration distinct from bytes per second. Compare a persistent linear trace, a one-time step then plateau, a sawtooth, and a noisy stationary trace. Declare window length, minimum effect, persistence rule, and uncertainty policy before evaluation.

Part 4 - Experimental Design and Instrumentation

Validate on synthetic traces with known labels, then on E01, E02, and repaired controls in fresh replays. Preserve sample phase, shape, update count, backend, and completion convention. Use multiple windows or a stated robust method rather than blindly accepting one fitted line. Record warmup exclusions and missing samples. Calibrate thresholds on named traces and test held-out growth rates and patterns. Bound history and alert frequency so the detector does not become another retention source.

Build synthetic traces with known byte units and iteration IDs before fitting hardware data. Include a slow linear trend hidden by noise, a one-time jump, a bounded growing cache, and a missing-sample interval. Keep current allocated bytes separate from reserved capacity and nonreset peaks. Define minimum sustained effect and sample-window requirements so a short startup step does not automatically trigger a leak alert. If the shape changes, stratify by shape or establish a new expected-state contract rather than fitting incompatible populations. Preserve fit residuals and an indeterminate status when evidence is weak. Evaluate false positives and false negatives on held-out patterns instead of choosing thresholds after seeing each test. A time-based slope requires actual elapsed timestamps and is not interchangeable with bytes per update. Conditional OOM extrapolation needs capacity, continued growth, unchanged workload, and allocator assumptions; report it as a hypothesis rather than a deadline. Bound retained sample history and alert artifacts so detection overhead remains measured and finite.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
```

```
@dataclass
class GrowthTrendDecision:
    window_identity: str
    phase: str
    slope_bytes_per_iteration: object
    uncertainty: object
    classification: str
    limitations: tuple


def fit_and_classify_growth(cpu_samples, detector_contract):
    ...


def E09(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report raw samples, slope, fit range, residuals/uncertainty, threshold, label, detection delay, and false positives/negatives. Do not insert zero for missing readings or fit nonreset peaks as current use. A predicted OOM iteration is at most a conditional extrapolation with explicit assumptions. Preserve indeterminate classifications where evidence is insufficient.

Part 7 - Engineering Interpretation and Philosophy

Choose a detector for investigation triage rather than automatic blame. Explain which extra ownership or lifetime evidence the alert should trigger.

19.47.11 Exercise 10

Exercise Title: Compare memory_allocated(), memory_reserved(), and torch.cuda.device_memory_used() to Distinguish Live PyTorch Tensors from Cached and External CUDA Memory

Part 1 - Exercise Contract and Why It Matters

Compare process-local allocated/reserved counters with device-level used memory. Distinguish live PyTorch storage, cached allocator capacity, external-library use, and unrelated process activity without an exact residual claim unsupported by the measurement scopes.

Part 2 - System and Memory Mental Model

memory_allocated and memory_reserved describe the calling process's allocator on the selected device. device_memory_used reports global device use through supported management interfaces, not this process's tensor total. The gap can include contexts, libraries, other processes, rounding, sampling skew, and retained cache. Reserved bytes already encompass allocated capacity; they must not be added as independent categories.

Part 3 - Quantitative Prediction Before Execution

Predict readings for a live tensor, its release into cache, an optional bounded external allocation, and another process using the same GPU. Define sampling order and timestamp

tolerance. Compare device use against allocator reserved capacity only as contextual evidence, not a precise external-allocation inventory. Explain why device use minus allocated bytes also includes PyTorch cache and cannot identify a culprit.

Part 4 - Experimental Design and Instrumentation

Warm contexts deliberately and use explicit device identity. Sample all three counters at matching completed phases, recording backend and management- API availability. Run isolated allocator cases first and controlled external or second-process cases only if available. In distributed mode, collect owned- device CPU rows and avoid duplicating a global device reading when summarizing. Keep unknown external activity explicit. Measure the effect of cache release only in a separate diagnostic intervention, not in every baseline iteration.

Use explicit device identity and record whether the first query initialized another context or management interface. Current allocator counters belong to the calling process and selected device, while global device usage can include unrelated processes. Capture sampling timestamps because subtracting values taken during different phases can create a false residual. Include a cache-retention control where live allocated bytes fall after release but reserved bytes remain. A separate optional cache-release intervention can show unused capacity returning, yet it cannot release live references or identify an external library. Do not add allocated and reserved bytes as independent use. If several ranks share a physical GPU, their process-local allocator inventories may differ while their global readings repeat the same device state; avoid summing that global number several times. Preserve backend and API availability, including management-library failures. A growing visibility gap should trigger more specific evidence, not an immediate native-leak label. Explain which part of the observation is exact API scope and which part remains a hypothesis about ownership.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def sample_allocator_and_device_scopes(device, phase, capabilities):
    ...


def compare_memory_scopes(rows, process_inventory, sampling_contract):
    ...


def E10(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export raw counters, timestamps, device identity, process scope, API support, and residual interpretation. Do not treat a positive difference as proof of a native leak. If management data is unavailable, report unsupported rather than substituting reserved or zero. A cache intervention cannot free live objects or prove external ownership.

Part 7 - Engineering Interpretation and Philosophy

Choose the next investigation step from the visibility gap. Explain which claims require per-process library evidence beyond these three counters.

19.47.12 Exercise 11

Exercise Title: Record CUDA Allocation History and Capture Baseline and Leaking memory_snapshot() States at Matching Training Iterations

Part 1 - Exercise Contract and Why It Matters

Enable bounded allocation history and capture baseline/leaking snapshots at matching completed training phases. Preserve versioned schema and trace coverage so later diffs can distinguish observation from missing history.

Part 2 - System and Memory Mental Model

Allocator snapshots show managed segments/blocks and, when recorded, allocation origins and event history. They do not automatically identify Python retaining owners or all native CUDA allocations. Public memory_snapshot and documented underscore-prefixed history/snapshot helpers can expose different structures depending on version. History ring buffers can overwrite older events; missing allocation/free records are not evidence those events never occurred.

Part 3 - Quantitative Prediction Before Execution

Choose warmup, recording start, capture iteration IDs, phase, maximum history entries, and artifact bounds. Predict which E02 retained blocks should survive between two captures. Define requested versus rounded block size and pending-free states separately. Estimate diagnostic host/storage overhead and identify the history coverage needed for lifetime reconstruction.

Part 4 - Experimental Design and Instrumentation

Check installed API signatures and normalize the actual returned schema through a versioned adapter. Enable bounded recording before the allocation window of interest. Capture baseline and leaking fresh replays at equivalent post-update/post-clear boundaries, with initialization and data held fixed. Record stream-completion policy and history truncation. Export snapshot artifacts from the learner runtime with run/phase metadata; disable recording at the declared end. Keep ordinary memory/timing runs separate from heavy diagnostics and avoid retaining all snapshots in an unbounded Python list.

Enable history early enough to include the cohort of interest and record its start boundary. A bounded ring can overwrite old allocation events, so the adapter must expose coverage and truncation instead of pretending every lifetime is complete. Capture the same iteration and completed phase in fresh baseline, leak, and repair replays, with the same model/data state. Preserve requested size, rounded capacity, block state, device, frames, and any supported allocation-generation evidence. Public snapshot output and richer diagnostic snapshot dictionaries should not be parsed through one assumed structure. Validate the installed schema and fail clearly on unsupported fields. Keep recording limits, capture frequency, and artifact retention bounded. Heavy diagnostic runs may change timing and host memory, so compare them with a minimally instrumented replay. Stop recording at the declared boundary and export CPU summaries. The learner's system should retain references to capture artifacts and metadata, not an indefinitely growing list of raw snapshots inside the training loop. Missing stacks remain missing rather than invented source lines.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def configure_bounded_allocator_history(capabilities, capture_contract):
    ...


def capture_matching_snapshot(runtime, run_id, iteration, phase):
    ...


def normalize_snapshot_schema(raw_snapshot, version_identity):
    ...


def E11(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report API/version, schema, capture phase, trace range, truncation, segment/block states, bytes, and instrumentation overhead. Preserve unsupported fields and incomplete coverage. A snapshot file's existence is not proof that stack recording was enabled early enough. Do not label allocations outside visibility as absent from the real device.

Part 7 - Engineering Interpretation and Philosophy

Explain which questions the capture can answer and which need ownership or external-library evidence. Choose the smallest diagnostic window that preserves the required allocation history.

19.47.13 Exercise 12

Exercise Title: Diff Two Memory Snapshots and Identify Allocations That Persist or Grow Only in the Leaking Workload

Part 1 - Exercise Contract and Why It Matters

Diff normalized snapshots and identify suspicious persistent or growing allocation cohorts. Distinguish within-run allocation identity from cross-run semantic comparison.

Part 2 - System and Memory Mental Model

An address can be freed and reused for a different allocation. Exact within-run persistence needs generation/event evidence where available. Two states alone can only show endpoint presence under their limits. Fresh baseline and leaking runs need not share addresses; compare cohorts by supported stack, size, role, phase, and workload identity rather than pointer subtraction. Persistent parameters and moments are expected survivors.

Part 3 - Quantitative Prediction Before Execution

Define new, freed, surviving, reused/ambiguous, and pending-free classifications. Hand-diff synthetic records containing address reuse, rounding, and missing history. Calculate useful requested bytes and actual block capacity separately. Predict the E02 cohort's growth after removing expected persistent state, while preserving uncertainty when allocation generations are unavailable.

Part 4 - Experimental Design and Instrumentation

Use E11's matching captures and a bounded history window. Normalize backend/schema fields before comparison. Match within-run generations only with adequate trace evidence and classify truncation honestly. Build cross-run cohort summaries under the same model, shape, seed, and phase. Compare baseline, leak, and repair. Export compact CPU suspect rows and stack references rather than copying every tensor. Validate the diff on synthetic cases before interpreting real captures. Keep inactive cache and active-awaiting-free blocks distinct from live allocated blocks.

Test the diff on an address allocated, freed, and reused between captures. The address can appear in both endpoint states without continuous lifetime. When event/generation evidence is missing, classify identity as ambiguous. For separate runs, compare semantic cohorts using supported creation stacks, requested sizes, logical roles, and phase; different raw addresses are expected. Do not remove all repeated stacks as baseline state, because an operator can create both expected parameters and unintended retained outputs. Preserve cohort counts and byte deltas alongside expected persistent ownership. Separate inactive cache, active allocation, and awaiting-free states before ranking. A pending free that completes after a deliberate reader boundary has a different explanation from a list retaining tensors. Include trace coverage and schema version in every diff result. Validate a proposed suspect against the repaired replay and owner evidence. The output should distinguish what is observed at endpoints, what is reconstructed from event history, and what is inferred from semantic matching across runs.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass


@dataclass
class AllocationDiffRow:
    cohort_identity: str
    identity_evidence: str
    classification: str
    requested_byte_delta: object
    block_capacity_delta: object
    lifetime_status: str


def diff_normalized_snapshots(before, after, trace_coverage, baseline):
    ...
```

```
def E12(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report match rules, coverage, expected survivors, suspect cohorts, byte changes, ambiguity, and intervention outcome. Do not infer continuous lifetime merely because an address appears twice. A large survivor is not a leak without an unnecessary-retention hypothesis. Preserve a small counterexample where naive pointer diff misclassifies reused storage.

Part 7 - Engineering Interpretation and Philosophy

Explain what makes a cohort suspicious and what further evidence would confirm its retaining owner. Prefer honest ambiguity to a false exact lifetime.

19.47.14 Exercise 13

Exercise Title: Trace Persistent CUDA Allocations to Their Python Stack Frames and Locate the Exact Source Line Responsible for Retention

Part 1 - Exercise Contract and Why It Matters

Trace suspicious allocations to their creation stack and locate the application retention site with separate ownership evidence. Produce a minimal validated repair rather than blaming an allocating operator automatically.

Part 2 - System and Memory Mental Model

Allocation frames identify where storage was created. The line appending an existing tensor to a list can be elsewhere and may not allocate any CUDA bytes itself. Native/library frames can obscure the application origin. Debuggers and referrer inspection can add references or expose only part of the ownership chain. Exact retention-line attribution requires a supported application trace and a controlled intervention.

Part 3 - Quantitative Prediction Before Execution

Construct a tensor created in one helper and stored by another helper. Predict the allocation stack versus insertion line. Define an evidence chain from allocation cohort to logical output ID, owner path, insertion iteration, and repair. Add an alias case where fixing only one owner leaves the storage alive. Distinguish exact, candidate, and unresolved attribution statuses.

Part 4 - Experimental Design and Instrumentation

Capture allocation stacks with E11 and instrument candidate container/hook insertions using bounded CPU metadata. Correlate logical storage generations within one run; do not match pointers across fresh runs. Rank application frames but preserve native frames and missing stacks. Remove or bound the suspected ownership in a fresh replay while keeping useful training unchanged. Verify the cohort and slope disappear within the tested range and gradients/ updates still match. Release introspection locals before measuring and avoid storing live referrer objects in the report.

Use two helpers in the minimal example: one creates an output and another adds it to a diagnostic owner. The creation stack should point to the former; the retention instrumentation should identify the latter. Preserve both in the evidence chain. A container insertion can

allocate no CUDA bytes itself, so a memory snapshot may never list that retaining line as an allocation origin. Rank suspects by unexplained surviving bytes, cohort growth, lifetime, and owner evidence rather than by stack depth alone. Keep a confidence or coverage status with each source location. If compiled or native frames obscure the application line, retain that limitation and seek a smaller eager reproducer. Controlled removal of the suspected owner must leave useful forward/backward/update work unchanged. Test an extra alias so the investigator cannot declare the first removed reference the only owner without checking. Do not retain referrer objects, frames, or tensors in the final report. Exact attribution is earned by correlated ownership and a matched repair, not by a plausible stack string.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def correlate_allocation_origins_and_owner_events(suspects, owner_registry):
    ...


def rank_retention_site_candidates(evidence_chain, coverage):
    ...


def verify_retention_site_intervention(replay_spec, reference):
    ...


def E13(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report creation frame, retaining line/path evidence, confidence/coverage, remaining aliases, intervention, byte trend, and numerical checks. Do not claim snapshots alone expose the exact Python owner. If only an allocation site is known, retain that limited result. A causal repair needs matched training and a restored ownership lifecycle, not simply a lower allocation count after deleting computation.

Part 7 - Engineering Interpretation and Philosophy

Explain why origin and retention are complementary clues. Choose the smallest repair that removes unnecessary ownership without changing the computation being debugged.

19.47.15 Exercise 14

Exercise Title: Use PyTorch Profiler Memory Records to Find an Operator or Abandoned Debug Path Responsible for Unexpected Long-Lived Allocations

Part 1 - Exercise Contract and Why It Matters

Use profiler memory records to localize unexpected allocation activity and correlate it with longer-lived suspect state. Keep allocation cost and retention cause distinct.

Part 2 - System and Memory Mental Model

Operator memory tables attribute allocation/deallocation activity within a capture, not every object's final Python owner. Large temporary operators can be healthy; a tiny debug insertion can retain their outputs without allocating. Shape/stack tracing adds overhead and can hold tensor references, changing behavior. Profiler evidence therefore needs comparison with a minimally instrumented baseline and snapshots/owner events.

Part 3 - Quantitative Prediction Before Execution

Predict a large temporary allocation that is released each iteration and a smaller operator whose output is retained. Define which profiler events and lifetime evidence distinguish them. Specify capture range and enabled features, and estimate whether their diagnostic overhead could affect the observed trend. Keep net memory, peak memory, and cumulative allocated traffic as different measures.

Part 4 - Experimental Design and Instrumentation

Profile a bounded warmed interval with named training and debug regions. Enable supported memory/stack/shape options deliberately and record their settings. Compare baseline, leaking, and repaired fresh replays under the same workload. Correlate operator frames with E12 cohorts and E13 owner metadata. Run ordinary timing/memory measurement separately without profiler reference retention. Validate outputs and several updates. Export tables, trace references, and unknown mappings to CPU rather than retaining captured GPU objects in the investigation system.

Record the profiler configuration and its capture phase with every memory table. Shape recording can temporarily hold references and add copies, and stack tracing adds overhead; use a separate minimally instrumented run to check whether the apparent trend survives. A large positive allocation event may be a healthy temporary and a negative event may occur outside the chosen capture. Net operator memory is therefore not a complete lifetime map. Correlate named debug regions with snapshot cohorts and insertion metadata. Include one large temporary that releases normally and one small retained output to test whether ranking follows evidence rather than raw size. Preserve unsupported event-to-cohort mappings. If a repair changes the operator graph, explain the changed expected memory baseline instead of claiming a pure ownership fix. Compare numerical gradients and several optimizer updates. Report profiling overhead and capture artifacts separately from ordinary training throughput. The profiler should narrow the origin/activity question, while owner events and interventions resolve why a particular allocation survived.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
from torch.profiler import profile, record_function
```

```
def profile_bounded_memory_regions(workload, capture_contract):
    ...


def correlate_operator_activity_with_survivors(profile_rows, cohorts, owners):
    ...


def E14(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report operator activity, capture options, relevant phase, persistent cohort evidence, owner hypothesis, overhead, and repair validation. Do not call the largest allocating operator the leak automatically. Preserve mismatches introduced by instrumentation and mark unmatched stack/owner relationships unknown. A profiler's net memory column alone cannot prove continuous allocation lifetime.

Part 7 - Engineering Interpretation and Philosophy

Choose whether operator traces, snapshots, or owner instrumentation should guide the next step. Explain how each narrows a different part of the causal chain.

19.47.16 Exercise 15

Exercise Title: Build a Binary-Search Leak Isolation Tool That Enables and Disables Training-Loop Sections to Locate the Smallest Memory-Growing Code Region

Part 1 - Exercise Contract and Why It Matters

Build a controlled section-isolation tool for memory-growing behavior. Use binary search only when its independence and monotonicity assumptions hold; otherwise use a bounded delta-debugging search and report the smallest tested reproducer.

Part 2 - System and Memory Mental Model

Training sections depend on each other. Disabling backward or optimizer changes legitimate memory behavior and can invalidate the original baseline. Two individually harmless logging paths can leak through an interaction. A binary split is not guaranteed to find a globally minimal region when effects are nonmonotonic. Start with optional metrics, debugging, and hook sections whose removal preserves the primary training update.

Part 3 - Quantitative Prediction Before Execution

Define a growth predicate from E09 and a capability/dependency graph for toggled sections. Construct one independent-retention case suitable for binary elimination and one interacting case that violates it. Count the replays required under the declared search strategy and hard budget. Distinguish smallest tested subset from a proof of global minimality.

Part 4 - Experimental Design and Instrumentation

Create fresh matched replays for each valid configuration from the same logical initialization, data, and shape. Toggle optional side effects first and verify outputs, gradients, and updates.

If core computation is replaced, construct a new explicit expected-memory baseline rather than interpreting lower memory as leak removal. Preserve configuration IDs, predicate uncertainty, search decisions, and rejected invalid combinations. Bound run count and instrumentation storage. Confirm the final suspect through an ownership intervention and repair replay rather than relying on search ranking alone.

Define which sections can be toggled without changing the mathematical training update. Logging, optional visual diagnostics, and observational hooks are suitable initial candidates; removing loss computation is not. Validate the dependency graph before launching a replay. For an independent single retention site, binary elimination can be efficient. For interacting sections, include a case where one creates a container and another inserts values so neither half alone reproduces the original behavior. Use a bounded delta- debugging strategy or report the unresolved interaction rather than asserting binary-search monotonicity. Every replay needs fresh state, matching samples, phase, and a growth-predicate uncertainty. Preserve invalid configurations as rejected, not negative leak tests. Confirm the smallest tested reproducer through ownership evidence and a repair. Do not claim global minimality without an exhaustive or otherwise justified search. If core training work is replaced by a stub, create a new explicit expected-memory contract and keep it separate from the final causal repair comparison.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass


@dataclass
class IsolationConfiguration:
    enabled_sections: tuple
    dependency_validity: str
    expected_memory_contract: str
    replay_identity: str


def isolate_growth_region(section_graph, predicate, replay_budget):
    ...


def E15(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export valid/rejected configurations, search order, growth decisions, uncertainty, numerical checks, and final reproducer. Do not claim exact source minimality from a heuristic search. Preserve an interacting-section counterexample and indeterminate predicate cases. A deleted forward pass is not a valid repair of an output-history bug.

Part 7 - Engineering Interpretation and Philosophy

Explain which assumptions make the isolation result trustworthy. Choose a final experiment that separates a causal retention repair from a workload reduction.

19.47.17 Exercise 16

Exercise Title: Detect a Memory Leak Hidden Behind Stable Reserved Memory by Tracking Active Allocation Count, Live Bytes, and Allocation Lifetimes Instead of VRAM Usage Alone

Part 1 - Exercise Contract and Why It Matters

Detect growing live storage while reserved capacity remains flat. Track supported current allocation counts, live/requested bytes, and allocation lifetimes under a declared allocator backend.

Part 2 - System and Memory Mental Model

Existing cached segments can absorb new retained allocations without increasing reserved memory initially. Cumulative allocation totals naturally grow in healthy training and are not current live counts. Active allocator state can include blocks awaiting completion on another stream; that is not automatically reachable Python retention. Counter availability and meaning can differ by backend.

Part 3 - Quantitative Prediction Before Execution

Construct a bounded trace that first establishes reusable reserved capacity and then retains new independent payload within it. Predict a flat reserved interval with rising current live bytes. Compare current allocation count with cumulative allocations and a pending-free control. Estimate the maximum test range before another segment could be needed without assuming a particular allocator rounding pattern.

Part 4 - Experimental Design and Instrumentation

Use synthetic traces to validate metric selection, then a bounded native-allocator replay where supported. Prewarm the declared capacity in a separate phase and record it. Sample same-phase current counts/bytes and use E11 history to inspect lifetimes. Keep pending stream work explicit and repeat after deliberate completion to distinguish pending frees from retention. Do not force a reserved plateau by altering the leaking workload's useful computation. Validate the repaired owner policy against baseline training. Preserve unsupported statistics as unknown rather than substituting cumulative counters.

Choose supported current fields by backend and keep their full semantic names in the evidence. A cumulative count of allocations grows whenever a healthy training loop repeatedly creates temporaries; it cannot substitute for the current number of live blocks. Requested bytes and rounded allocator capacity answer different questions. Prewarm a bounded arena or reusable capacity before the synthetic/controlled retention phase, recording that setup separately. New live tensors can consume inactive blocks within already reserved segments, exposing growth without a reserved increase. Do not require this exact allocator pattern to occur on every backend or size range. Add a separate cross-stream pending-free case and sample again after its readers complete. That backlog can temporarily increase active state without an unnecessary Python owner. Combine current bytes, supported counts, and lifetime history rather than declaring a leak from one field. Validate the owner repair against the fixed baseline. Preserve a not-observed hardware plateau with the synthetic counterexample instead of manipulating the test until the expected picture appears.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def select_supported_current_allocation_metrics(backend, capabilities):
    ...


def detect_growth_under_reserved_plateau(rows, lifetime_evidence):
    ...


def E16(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report current and cumulative metric names, live bytes, reserved bytes, backend, lifetime states, completion boundaries, and intervention result. A flat device graph is not proof of no retention. Conversely, pending-free blocks that retire after readers complete are not sufficient evidence of a leak. If the plateau does not appear within the tested range, retain that outcome and the synthetic counterexample.

Part 7 - Engineering Interpretation and Philosophy

Explain why reserved capacity can hide a real live-storage trend. Choose the additional counter or lifetime observation that exposes the issue without creating false alarms from normal allocation traffic.

19.47.18 Exercise 17

Exercise Title: Diagnose CUDA Memory Growth Invisible to the PyTorch Allocator by Comparing PyTorch Statistics Against Device-Level Memory Consumption

Part 1 - Exercise Contract and Why It Matters

Investigate device-memory growth not explained by PyTorch allocator statistics. Use scoped comparisons and optional bounded external-allocation controls without claiming visibility into every library or process.

Part 2 - System and Memory Mental Model

CUDA contexts, communication libraries, third-party allocators, and other processes can consume device memory outside PyTorch-managed segments. Another allocator may cache released tensors, producing its own plateau. A global device counter cannot attribute all growth to the current process. PyTorch snapshots cannot reconstruct stacks for allocations they never observed.

Part 3 - Quantitative Prediction Before Execution

Predict PyTorch counters and device-level use for a healthy external cache, a deliberately retained external allocation, and unrelated process activity. Define evidence needed to distinguish them. Keep device use minus reserved capacity as a visibility-gap indicator with timing/context

limits, not exact external leak bytes. Predict cleanup behavior under the external allocator's actual ownership/cache policy.

Part 4 - Experimental Design and Instrumentation

Run the stable PyTorch control first. If a supported third-party or native allocation interface is available, use a small explicitly bounded allocation lifecycle with documented cleanup and completion. Record its library, version, process identity, owner, cache policy, and sample timestamps. Do not install or assume an external backend merely to fill a result table. Compare fresh isolated replays and a release intervention. For optional distributed cases, initialize communication separately and distinguish one-time buffers from repeated growth. Preserve unsupported per-process visibility.

Identify the optional external allocator before running its control. Its tensor deletion, pool release, and actual driver release can be different operations, just as PyTorch has caching. Use a documented bounded allocation and cleanup lifecycle with explicit stream completion; do not leave an unowned raw allocation to simulate a permanent failure. Record whether the library allocates through PyTorch's allocator, its own pool, or a direct CUDA interface, because visibility follows that choice. A driver-level rise can also come from another process, so preserve process context and timestamp skew. Compare initialization-only growth with repeated growth after warmup. If no external interface is installed, retain the analytical scope test and mark the hardware control not measured. Do not install dependencies or infer NCCL behavior simply to fill every row. An unresolved global-minus-allocator gap is a valid investigation result until library-specific owner, cache, or process evidence narrows it. Absence of allocation frames in PyTorch's snapshot cannot prove a native allocation was never made.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def collect_external_visibility_evidence(device, process_context):
    ...


def run_bounded_external_allocator_control(adapter, resource_contract):
    ...


def classify_unexplained_device_growth(rows, external_evidence):
    ...


def E17(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report raw scopes, external adapter status, owner/cache evidence, cleanup, initialization, device trend, and remaining uncertainty. Do not label an unexplained gap a proven native leak. If no external interface is available, report the control not measured while retaining the scope analysis. Never treat absence from snapshots as absence from the device.

Part 7 - Engineering Interpretation and Philosophy

Choose the next library/process-specific diagnostic based on actual visibility. Explain where automated PyTorch-only attribution must stop.

19.47.19 Exercise 18

Exercise Title: Stress-Test the Leak Detector with Graph Retention, Python Containers, Hooks, External CUDA Allocations, Fragmentation, and Legitimate One-Time Memory Growth

Part 1 - Exercise Contract and Why It Matters

Stress the detector and investigation pipeline with retention, hooks, external visibility gaps, cache behavior, fragmentation-like patterns, and legitimate one-time growth. Evaluate false positives as carefully as true retention detection.

Part 2 - System and Memory Mental Model

Different causes can share a rising reserved-memory curve. Variable shapes can change caching and fit, while one-time optimizer or communicator state can create a step. Fragmentation concerns reuse/fit and cannot be proved from a reserved-minus-allocated gap alone. Graph structure, saved activations, pending frees, and external allocations need separate labels. Diagnostic overhead can create its own growth.

Part 3 - Quantitative Prediction Before Execution

Build a labeled synthetic matrix with linear retention, one-time jump, noisy plateau, sawtooth collection, pending-free backlog, stable-reserved live growth, and external-only growth. Add bounded hardware cases when supported. Predict detector label, required follow-up, and ambiguity for every case. Define detection delay, false-alarm rate, evidence coverage, and resource limits before testing.

Part 4 - Experimental Design and Instrumentation

Validate individual cases before combining them. Preserve seed, shape schedule, phase, allocator backend, and expected persistent state. Use bounded list/hook/graph controls and explicit external capability checks. Change one cause at a time, then selected combinations that test interacting ownership. Record instrumentation host memory and timing. After failure, restart from a clean state before comparing policies. Keep numerical training checks for valid workload variants. Preserve nonreproducing OOM or fit outcomes without inventing fragmentation evidence or a universal allocator diagnosis.

Create separate expected labels for reachable payload retention, graph- structure retention, saved-activation retention, pending frees, cache growth, external visibility, and legitimate persistent initialization. Some cases can be indeterminate with the available counters; do not force one exact cause. Use synthetic histories to test address reuse and trace truncation without requiring an unreliable hardware race. Keep bounded real list, hook, and retained-graph reproducers with explicit owners. For variable shapes or fragmentation-like workloads, record

the request sequence and actual outcomes; a gap between reserved and allocated bytes is not causal fragmentation proof. Include diagnostic self-retention as a test: an evidence logger that stores GPU tensors must be detected or rejected. Track host overhead from history and profiler records as well as CUDA memory. Evaluate detection, localization, and successful repair separately, since an alert can be correct while its top-ranked suspect is wrong. Preserve all rejected and unsupported cases with their reasons, and validate that repaired training still performs the intended updates.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


def make_leak_investigation_stress_matrix(capabilities, budget):
    ...


def evaluate_detector_and_attribution(case, evidence, expected_labels):
    ...


def E18(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Export raw traces, labels, predicted/observed diagnoses, false positives/ negatives, capture overhead, bytes, outcome, and failure phase. Include the smallest counterexample to a naive rule or state none was observed. A valid unsupported/indeterminate result must not be counted as a correct exact diagnosis. Verify clean repairs preserve outputs and optimizer updates.

Part 7 - Engineering Interpretation and Philosophy

Explain which stress case most limits the system's claims. Refine supported scope, phase contracts, or follow-up policy without hiding unsuccessful cases.

19.47.20 Exercise 19

Exercise Title: Build and Benchmark an Automated PyTorch Memory Leak Investigation System That Detects Growth, Captures Snapshots, Diffs Allocations, Ranks Suspect Stack Traces, and Produces a Root-Cause Report

Part 1 - Exercise Contract and Why It Matters

Integrate growth detection, bounded capture, schema normalization, allocation diffs, owner evidence, and candidate ranking into an automated investigation system. Produce a root-cause report whose confidence follows from validated evidence and controlled repairs.

Part 2 - System and Memory Mental Model

The system must distinguish alerts from diagnoses and creation stacks from retaining owners. It needs run/phase identity, allocator visibility, trace coverage, expected state, and a bounded evidence lifecycle. Automatic ranking can propose suspects but cannot guarantee exact attribution for external allocations or missing histories. Preserve unresolved status rather than forcing every investigation into a confident leak label.

Part 3 - Quantitative Prediction Before Execution

Predict monitoring, history, snapshot, and report storage overhead. Define trigger thresholds, capture limits, maximum replay count, phase alignment, and numerical acceptance. Choose a finite set of known retention and healthy controls plus held-out cases. Specify report fields that separate facts, hypotheses, interventions, results, and unsupported visibility.

Part 4 - Experimental Design and Instrumentation

Build from E01's deterministic baseline and validate controlled leaks and repairs. Use separate ordinary measurement, profiler, and capture replays. Preserve E00's reproducibility record plus detector version, API/schema, history coverage, owner registry, and configuration IDs. Time complete training and investigation overhead separately. Export bounded CPU summaries and learner-generated capture references without live graph retention. Confirm candidate repairs through matched gradients and several updates. Unavailable GPU or external paths remain not measured.

Define the root-cause report schema before integration. It should include workload/phase contract, raw counter scopes, growth decision, capture coverage, suspect cohorts, creation frames, retaining-owner evidence, intervention, numerical validation, and unresolved alternatives. Keep facts separate from ranked hypotheses. A system can successfully detect suspicious growth yet remain unable to prove an exact retaining line, especially with external allocations or missing event history. Bound automatic capture count, artifact retention, replay budget, and diagnostic host memory. Include collection and analysis overhead in the benchmark rather than reporting only the stable training loop. Use known cases for development and held-out cases for evaluation. Every supported feature needs a validating run ID; imports and method names do not establish capability. Preserve minimal counterexamples to universal rules and failures to reproduce. The final report should recommend the next smallest discriminating experiment or a validated repair, with an honest confidence/coverage statement rather than automatic blame assigned to the largest allocation stack.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch


class MemoryLeakInvestigationSystem:
    def monitor_completed_iteration(self, runtime, phase):
        ...

    def capture_bounded_diagnostic_state(self, trigger):
        ...

    def diff_and_rank_candidates(self, captures, ownership_evidence):
```

```
        ...

    def validate_candidate_intervention(self, replay_spec):
        ...

    def export_root_cause_report(self):
        ...


def E19(config):
    # Assemble checked diagnostics and evaluate known/held-out cases.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Keep prediction, observation, and interpretation separate. Each run needs ID, configuration, shape, byte budgets, current/peak allocated and reserved bytes, device-level scope, elapsed time, correctness, outcome, and failure phase. Report raw growth, suspect cohorts, origin/owner distinction, coverage, intervention results, overhead, and remaining uncertainty. Include minimal counterexamples and unsupported/not-measured values. A scaffold method is not capability evidence; every supported feature needs a validating run ID. Never invent GPU results or declare a suspected line proven without ownership and repair evidence.

Part 7 - Engineering Interpretation and Philosophy

Justify the final diagnosis or unresolved classification through scoped counters, lifetimes, ownership, and matched interventions. Identify the next smallest experiment that would resolve remaining uncertainty.

19.48 DMM46

Chapter Title: Autograd Graph Memory Analysis
Specialization: Dynamic Memory Management and CUDA Systems

19.48.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Autograd Nodes, Saved Tensors, Gradient Edges, Tensor Lifetimes, and Backward-Time Memory Release

Part 1 - Exercise Contract and Why It Matters

Construct a vocabulary and an accounting contract for the entire exercise sequence. Distinguish a backward node, a gradient edge, a saved tensor slot, a Python Tensor wrapper, and an underlying storage allocation. Deliver a small annotated computation, predicted lifetime transitions, and counterexamples to simplistic memory rules. The objective is to explain what must remain available for the requested derivatives before proposing any memory optimization.

Part 2 - System and Memory Mental Model

Autograd records differentiable operations during forward execution and uses the resulting dependencies during backward. Nodes may save inputs, outputs, or other values needed by

their derivative formulas. Several saved slots can reference one storage, while a tiny view can keep a large backing allocation alive. Ordinary backward usually releases saved intermediates as they become unnecessary; graph structure can remain reachable through an output afterward. Allocator caching and asynchronous pending frees are separate phenomena.

Part 3 - Quantitative Prediction Before Execution

For distinct observed storages with capacities c_j and conservative live intervals $[a_j, b_j)$, predict a storage occupancy envelope:

$$M_{\text{observed}}(t) = \sum_j c_j \, \mathbf{1}[a_j \le t < b_j].$$

Compare this with the sum of logical saved-slot bytes. Construct one example with repeated saves and another with a small slice of a large tensor. Explain why neither node count nor saved-slot sum alone equals the total CUDA peak.

Part 4 - Experimental Design and Instrumentation

Begin with a deterministic eager model using smooth operations, fixed shapes, and one scalar loss. Draw forward consumers separately from backward dependency edges. Mark parameters, gradient buffers, optimizer states, temporary workspace, and cached capacity as different accounting categories. Follow the same state through forward, backward, gradient clearing, and owner deletion. Record the installed PyTorch version and restrict claims to the inspected operations; unsupported fields must remain explicitly unknown.

Create a reproducibility record containing hardware identity, allocator backend, PyTorch and CUDA versions, seeds, dimensions, precision, optimizer, and initial state identifier. Keep cold-start and warmed behavior separate. For each experiment preserve an ID, configuration, input shape, byte budget, timing scope, outcome, and failure phase when applicable. Do not imply that logical zero-data metadata means zero host memory. Use explicit unsupported and not-measured values throughout the sequence. A CPU-only analytical control can validate accounting and graph invariants when CUDA is unavailable, but it cannot supply a measured GPU peak or establish real asynchronous release behavior.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class StorageRecord:
    generation: object
    capacity_bytes: object
    owners: tuple
    completion_status: str

def classify_autograd_objects(loss, scope):
    ...
```

```
def E00(config):
    # Draw dependencies and predict release boundaries first.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require a counterexample to the claims that every node saves a tensor, every saved slot owns distinct memory, and deleting one wrapper frees its storage. Validate derivative values against an analytical toy case. Keep predictions, observations, and interpretations in separate columns. Do not retain CUDA values inside the evidence logger. A CPU metadata record can describe an allocation without proving that the corresponding GPU kernel has completed.

Part 7 - Engineering Interpretation and Philosophy

Memory analysis is a study of obligations: which future computation still needs a value, which owner preserves it, and which execution dependency permits reuse. A useful model admits uncertainty about opaque workspaces and backend behavior. State which decisions can already be justified from the graph and which require a controlled measurement. Carry this distinction into every later recommendation rather than treating a diagram as a complete allocator or execution trace.

19.48.2 Exercise 01

Exercise Title: Inspect grad_fn and next_functions to Trace the Autograd Graph of a Multi-Layer PyTorch Computation

Part 1 - Exercise Contract and Why It Matters

Build an iterative traversal of the backward graph rooted at a declared loss. Export node records and dependency edges without recursively duplicating shared subgraphs. Deliver a readable adjacency representation, an explicit coverage statement, and a demonstration involving a branch that rejoins. The traversal should support investigation without quietly becoming a new long-lived owner of the objects it investigates.

Part 2 - System and Memory Mental Model

A tensor's grad_fn identifies its producing backward node when applicable. The next_functions interface exposes backward dependencies and edge metadata; it does not enumerate every forward tensor or reconstruct the complete source program. Some entries have no node. Accumulator nodes and shared dependencies need distinct treatment. Identities are meaningful within the diagnostic traversal and must not be compared as permanent identifiers across replays.

Part 3 - Quantitative Prediction Before Execution

Predict the unique node count and edge count for a chain and for a shared branch before inspecting them. If a shared dependency is encountered twice, count the node once while preserving both incoming relationships. Explain why a walk that sums descendants independently overestimates the DAG size. Keep node counts separate from byte estimates: a small number of nodes can retain very large tensors.

Part 4 - Experimental Design and Instrumentation

Use a short multi-layer model with named application regions and an explicit shared intermediate. Traverse all requested roots with a visited set and a bounded work queue. Preserve

edge positions and null entries rather than inventing missing computation. Run traversal on a diagnostic replay, export CPU-only labels, then discard node references before the ordinary memory measurement. Test whether enabling the analyzer changes the post-run memory baseline.

Choose traversal depth and event limits before inspection. Test a deep chain that would overflow a naive recursive walker, a diamond with a shared node, and multiple roots sharing an ancestor. Distinguish repeated discovery from a graph cycle: the backward representation of a normal eager computation is not made cyclic merely by a shared edge. Keep application labels stable while allowing internal node names to vary by version. If the implementation keeps strong references to nodes for identity stability during traversal, release that collection after export and before judging cleanup. Include traversal overhead and the completeness of the visited roots in the final record.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def walk_backward_roots(roots, traversal_id):
    # Deduplicate nodes within this traversal; preserve edge slots.
    ...

def export_adjacency(nodes, edges, output_path):
    ...

def E01(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate that the branch is represented as shared structure rather than two independent copies. Include a constant input with no gradient requirement and a disconnected auxiliary computation outside the chosen root. Mark those coverage boundaries explicitly. Avoid assigning bytes to a node simply because its name resembles a known operator. If holding traversal objects changes release behavior, report that observer effect and use a separate measurement replay.

Part 7 - Engineering Interpretation and Philosophy

A backward graph is a dependency view for a particular differentiation task. It is valuable because it narrows which paths can explain a retained value, but it cannot independently prove ownership or execution timing. Explain what the adjacency export establishes, what the application labels add, and what must wait for saved-tensor or allocator evidence. Prefer an incomplete graph with honest boundaries over a complete-looking reconstruction assembled from unverified assumptions.

19.48.3 Exercise 02

Exercise Title: Compare Leaf and Non-Leaf Tensors and Determine Which Objects Store Gradients and Which Only Participate in the Graph

Part 1 - Exercise Contract and Why It Matters

Classify leaf and non-leaf tensors, gradient requirements, and gradient storage as separate properties. Produce a table before and after backward, then repeat with retain_grad enabled on one intermediate. Include direct CUDA creation, a differentiable device or dtype conversion, a detached tensor, and a tensor that never requires gradients. Explain each outcome from the construction history rather than from whether the .grad field happens to be populated.

Part 2 - System and Memory Mental Model

Leaf tensors requiring gradients normally receive accumulated gradients from backward. Non-leaf tensors participate in differentiation without retaining their own .grad by default; retain_grad opts into that storage. A tensor that does not require gradients can also be a leaf. The autograd.grad interface returns derivatives instead of following the same accumulation contract. The words leaf, trainable, and currently holding a gradient are therefore not interchangeable classifications.

Part 3 - Quantitative Prediction Before Execution

Predict which .grad fields are None, which have the tensor's logical shape, and which add retained CUDA bytes. For a retained intermediate with n elements and gradient element size s, use ns as a logical payload estimate, not a guarantee about rounded allocator capacity. Predict how the next iteration changes when gradients are set to None rather than filled with zeros.

Part 4 - Experimental Design and Instrumentation

Use a small differentiable expression whose derivative is analytically known. Record is_leaf, requires_grad, grad_fn status, device, dtype, and the chosen gradient policy as CPU scalars or strings. Compare ordinary backward with a fresh replay using autograd.grad. Clear gradients at matching phase boundaries. Avoid probing non-leaf .grad repeatedly in a way that confuses expected warnings with execution failures. Never infer a missing derivative from None without checking the API contract.

Use an explicit construction table rather than mutating one tensor through all role tests. A Parameter created directly on its target device is a useful leaf reference; a real differentiable conversion demonstrates a different history. State whether a detached value is made into a new differentiated leaf and which earlier derivatives are intentionally absent in that case. Compare retained intermediate gradients at matching points after backward, not against a baseline already cleared for the next step. For sparse or nonstandard layouts, narrow the supported scope instead of applying a dense payload formula without checking its meaning. Preserve the distinction between no gradient requested and a computed zero.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def classify_tensor_role(tensor, construction_label):
    ...

def compare_gradient_retention(config):
    # Separate retain_grad from retain_graph.
```

```
    ...

def E02(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Check derivative values and accumulation over two backward calls on freshly constructed graphs. A conversion test must actually change device or dtype; a no-op conversion may return the original tensor. Report gradient-buffer memory separately from saved activations. Include a leaf that is unused by the loss and document the chosen unused-input behavior. Distinguish a valid None result from a numerical zero gradient that was computed and stored.

Part 7 - Engineering Interpretation and Philosophy

The distinction matters when evaluating optimizations: removing unnecessary retained intermediate gradients can reduce memory without altering the model, whereas detaching an intermediate changes the differentiation path. Explain which experiment changes observation policy and which changes mathematical work. Keep the gradient storage contract visible in every later comparison, so apparent savings are not merely the consequence of comparing different sets of requested or accumulated derivatives.

19.48.4 Exercise 03

Exercise Title: Measure CUDA Memory Before Forward, After Forward, During Backward, and After Graph Release to Reconstruct the Autograd Memory Lifecycle

Part 1 - Exercise Contract and Why It Matters

Measure the autograd memory lifecycle at matching boundaries and reconstruct where the high-water mark occurs. Include the first optimizer update so lazy optimizer state creation is visible. Deliver a phase table and a separate backward-region peak measurement. Endpoint measurements alone cannot locate a temporary peak inside backward, so the design must distinguish boundary occupancy from a high-water mark observed during an interval.

Part 2 - System and Memory Mental Model

Forward can retain values for backward; backward can allocate gradient buffers and workspace while releasing saved state. The maximum may occur before either phase endpoint. memory_allocated tracks allocator-visible live tensor memory, while memory_reserved includes cached capacity. Device-level use has a broader scope. Deleting the loss after ordinary backward may remove remaining graph structure without causing a second release of already-freed saved tensors.

Part 3 - Quantitative Prediction Before Execution

Build an accounting prediction with persistent parameters, gradients, optimizer state, saved activations, and temporary workspace. Do not add independently measured category peaks as if they occurred simultaneously. Predict a range for the backward peak using overlap assumptions, then state which assumption the experiment can test. Compare a cold first step with warmed repeated steps and explain why their optimizer and allocator baselines differ.

Part 4 - Experimental Design and Instrumentation

Define deliberate synchronization points, reset peak statistics for each measurement region, and record both allocated and reserved bytes. Use a separate diagnostic replay with bounded history to inspect events during backward without synchronizing after every operator. Keep input dimensions, precision, optimizer, gradient clearing, and initial state identical across replays. Record transfer and setup timing separately when excluded from the measured region, and state every inclusion explicitly.

Give every sample an initial-state identifier and a precise phase name. An allocation-only control needs its own declared region; a training measurement includes forward, backward, and optimizer update. Resetting a peak counter does not free memory or restore the initial graph. Use independent replays when a probe changes ownership or execution order. If recording CPU memory, name the metric and process scope rather than adding unrelated RSS and CUDA values. Keep transfer, collation, calibration, and recovery time explicit in the timing contract. Repeat fixed-seed comparisons first, then use additional seeds to check whether the observed pattern depends on particular values.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def sample_boundary(device, phase, run_id):
    ...

def measure_backward_region(loss, device, evidence):
    # Reset the declared peak region and wait at its boundary.
    ...

def E03(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require a stable warmed baseline before interpreting growth. Compare sampled endpoints with the interval peak and explain any gap using supported evidence. Do not claim exact tensor ownership from a counter alone. Export numeric scalars without preserving live loss tensors or gradients in the log. A missing CUDA device produces a not-measured result rather than an invented lifecycle trace. Report synchronization and instrumentation costs alongside the minimally instrumented runtime.

Part 7 - Engineering Interpretation and Philosophy

The lifecycle is a set of overlapping allocations and obligations, not four universal monotonic phases. Identify which phase dominates the tested workload and whether a proposed intervention can affect that phase. A forward saving that shifts the peak into backward can still be useful, but it must be judged using the full step's high-water mark and validated updates rather than a single favorable endpoint.

19.48.5 Exercise 04

Exercise Title: Inspect Saved Tensors Required by Backward and Estimate Their Contribution to Peak VRAM

Part 1 - Exercise Contract and Why It Matters

Inspect saved values in a few controlled derivative formulas and estimate their memory contribution without equating logical tensor size with exclusive VRAM. Deliver a saved-slot inventory, a storage-sharing inventory, and a prediction of the minimum backing capacity those observations can keep alive. Restrict internal attribute inspection to small version-checked examples; the exercise must not depend on a universal parser for private node fields.

Part 2 - System and Memory Mental Model

An operation saves only what its backward implementation needs for the chosen gradient requirements. Addition may need no input values, while multiplication or nonlinear derivatives can need inputs or outputs. Packed saved tensors can be unpacked into a different Python object sharing the same storage. Parameters may already belong to the persistent baseline, so saving their references does not necessarily allocate another parameter-sized CUDA block.

Part 3 - Quantitative Prediction Before Execution

Calculate logical saved bytes as a sum over slots, and separately calculate unique observed backing capacities. Consider two overlapping views of one large tensor and a parameter saved by several operations. Predict the amount of double counting and identify which storage was already resident. Bound the incremental saved-state footprint without pretending that unobserved workspace or allocator rounding has been measured.

Part 4 - Experimental Design and Instrumentation

Compare a small set of scalar and matrix operations in isolated eager replays. Vary which inputs require gradients while holding shapes constant. Record supported saved attributes or hook observations, strides, storage offsets, and storage capacities without storing inspected CUDA tensors in the report. Release temporary inspection results promptly. Recheck the ordinary run without inspection to detect whether accessing saved objects or retaining wrappers changed their lifetimes.

Build a synthetic storage catalog before relying on runtime pointers. Include one allocation saved twice, two views with different offsets, a zero-element view, and a small slice that pins a large backing capacity. Compare these with independent clones having identical values. Preserve device and generation in every identity record. A tensor's data pointer can include its offset; it is not a universal backing-storage identifier. Conversely, identical numeric addresses in separate lifetimes do not establish a shared allocation. Explain how empty tensors and unsupported layouts are represented without inventing capacity or collapsing distinct records. These controls validate bookkeeping without claiming any particular allocator rounding policy.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def inspect_supported_saved_fields(node, version_scope):
```

```
    # Private illustrative fields require explicit support checks.
    ...

def summarize_slots_and_storages(slot_rows, persistent_catalog):
    ...

def E04(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Include a small view whose backing allocation is much larger than its logical payload and an operation with no observed saved tensor. Unknown inspection coverage must remain unknown rather than zero. Validate outputs and gradients against an uninstrumented replay. A storage pointer match is useful within a controlled live interval but is insufficient across allocation generations. Report logical bytes, unique capacity, and persistent overlap in adjacent columns so the reader can audit the estimate.

Part 7 - Engineering Interpretation and Philosophy

Saved-state analysis explains one reason a storage remains necessary. It does not establish exclusive ownership by one operation or guarantee immediate release at backward. Explain how the inventory narrows optimization candidates and where aliases or pre-existing persistent state limit possible savings. The most useful estimate distinguishes removable capacity from bytes that would remain resident even if the saved reference disappeared.

19.48.6 Exercise 05

Exercise Title: Use torch.autograd.graph.saved_tensors_hooks to Log the Shape, Dtype, Device, and Size of Every Tensor Saved for Backward

Part 1 - Exercise Contract and Why It Matters

Instrument supported saved-tensor events with saved_tensors_hooks and export shape, stride, dtype, device, logical size, and scoped storage metadata. Every means every covered save callback inside the declared context, not every possible tensor retained by Python, a custom extension, or an opaque backend. The logging mechanism must preserve the values needed by backward and must not become an unbounded GPU history.

Part 2 - System and Memory Mental Model

The pack hook receives a tensor and returns a payload that the unpack hook can reconstruct into the required value. A metadata dictionary alone cannot satisfy that contract. The packed payload must not hold the original input Tensor object, which can create a reference cycle. Use a validated detached representation for a first-order logging experiment while keeping report rows CPU-only. This representation choice is not permission to detach the forward computation.

Part 3 - Quantitative Prediction Before Execution

Predict pack count, possible repeated unpack calls, and logical bytes for a small known expression. Do not assume a one-to-one relationship between pack and unpack counts. Estimate host log size under a fixed event limit, and distinguish payloads required for backward from

optional diagnostic rows. Predict what a nested hook context records, then verify the installed version's innermost-hook behavior.

Part 4 - Experimental Design and Instrumentation

Start with first-order gradients and compare an instrumented replay against an uninstrumented oracle. Preserve values, shape, dtype, and original device through pack and unpack; do not mutate hook arguments or payloads. Include a repeated backward case to test idempotent reconstruction while the payload is needed. Keep counters and copied metadata separate from payload ownership. Test higher-order differentiation separately before claiming that the same representation supports it.

Set an event budget with an explicit truncation marker. A partial log cannot support an assertion that every save in the entire run was recorded. Avoid formatting full tensor values, converting whole tensors to lists, or reading CUDA scalars inside each callback; such actions can add transfers and synchronization. Test nested contexts using distinguishable metadata tags and restore the previous context even when the workload raises. If a payload is shared across retrievals, keep its ownership valid until all uses end. Treat unpack failures as correctness failures, not as missing observations that can be ignored. Report whether instrumentation changes supported higher-order behavior.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class PackedValue:
    representation: object
    event_id: object

def pack_saved_value(tensor, scope, recorder):
    ...

def unpack_saved_value(payload):
    ...

def E05(config):
    # Install hooks only around the declared forward scope.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify numerical outputs and requested gradients, including a case that unpacks the same save more than once. Report context, nesting, thread assumptions, unsupported retention paths, and logging overhead. The callback does not provide the owning autograd node, so do not fabricate node attribution from its timestamp. Validate that disabling optional logging removes report data without changing required payload ownership or the final memory baseline.

Part 7 - Engineering Interpretation and Philosophy

An observation interface is useful only when its correctness contract is preserved. Explain why returning metadata instead of a value is not a memory optimization, and why retaining the original tensor inside a report is a self-inflicted retention path. Carry callback coverage and attribution uncertainty forward into rankings; callback availability alone does not make the instrumented view complete.

19.48.7 Exercise 06

Exercise Title: Rank Autograd Operations by Total Saved-Tensor Bytes and Identify the Most Memory-Expensive Backward Dependencies

Part 1 - Exercise Contract and Why It Matters

Rank application regions or supported operations by saved-state burden while separating logical save traffic from unique retained storage. Deliver multiple rankings and a counterexample where the operation with the largest saved-slot sum is not the best memory-saving target. Establish the attribution method before labeling rows as operator costs; ambiguity must be represented rather than resolved through guesses about callback order.

Part 2 - System and Memory Mental Model

A pack callback receives a tensor but no universally reliable owning-node identifier. Controlled module scopes, explicit application regions, or tagged custom functions can provide partial attribution. Shared values can be saved under multiple regions, so region totals are not additive exclusive memory. Call count and tensor size describe save activity; lifetime overlap and alternative owners determine whether eliminating one save releases capacity.

Part 3 - Quantitative Prediction Before Execution

For each region predict slot bytes, unique referenced capacity, persistent capacity overlap, and a removable-capacity bound. Construct two regions that save the same parameter and one that saves a short-lived independent tensor. Predict how rankings change when using byte-time rather than byte totals. State whether a storage shared by several regions receives shared attribution or an explicitly non-additive reference in each row.

Part 4 - Experimental Design and Instrumentation

Instrument named regions in separate diagnostic replays and retain the same workload order. Include nested modules, shared parameters, and a repeated operation. Export unresolved attribution with its candidate scopes. Use a small known custom function as an attribution control if supported, while keeping its mathematical derivative checked. Compare the strongest ranked candidate with a correctness-preserving intervention in an independent replay; ranking alone is not causal evidence.

Provide a ranking table with columns for attribution scope, observed save count, logical bytes, unique referenced capacity, persistent overlap, lifetime coverage, and ambiguity. Record the aggregation rule next to each column. Include a deliberately misleading module-level example where nested scope totals double count the same save events. Compare flat exclusive application regions with hierarchical inclusive reporting, labeling their semantics. If a region is called several times, preserve invocation IDs before aggregating. A candidate's rank should not become more confident merely because several ambiguous callbacks were assigned the same guessed label. Validate the claimed saving through one controlled intervention and record a null result honestly.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def attribute_save_events(events, application_scopes):
    ...

def rank_saved_state(attributions, storage_catalog, lifetime_bounds):
    # Export non-additive shared capacity and ambiguous ownership.
    ...

def E06(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Audit whether totals count aliases, repeated saves, or persistent parameters multiple times. A zero-byte row must mean proven absence under the stated coverage; otherwise use unknown. Compare proposed savings with measured whole-step peak changes and explain differences in concurrency or workspace. Report rank stability across repeated runs without claiming statistical certainty from a small sample. Include the smallest counterexample that invalidates the largest-save-means-largest-saving rule.

Part 7 - Engineering Interpretation and Philosophy

Ranking is a prioritization aid, not an automatic deletion policy. A region can have many saved bytes yet be cheap to keep because its values overlap with unavoidable state. Another can retain modest bytes across the actual peak and become a better target. Explain how the selected ranking supports the next experiment and where incomplete attribution limits its interpretation.

19.48.8 Exercise 07

Exercise Title: Reproduce Graph Retention by Storing a Tensor with grad_fn Across Iterations and Measure the Resulting CUDA Memory Growth

Part 1 - Exercise Contract and Why It Matters

Construct bounded examples of graph retention through stored outputs and measure which memory actually persists. Compare storage before backward, ordinary completed backward, and backward with retained state. The title's hypothesis must be tested: storing a tensor with grad_fn does not universally preserve all saved activations after ordinary backward has released them. Deliver distinct outcomes for output storage, graph structure, and saved state.

Part 2 - System and Memory Mental Model

A Python container can keep an output and its reachable graph structure alive. Before backward, or when saved state is explicitly retained, that reachability can also preserve large

backward dependencies. After ordinary backward, many saved buffers have already been released even though grad_fn remains visible. A detached output still occupies its own CUDA storage; CPU graph metadata and GPU saved activation storage are different resources.

Part 3 - Quantitative Prediction Before Execution

Predict growth over a bounded number of iterations for three cases: storing outputs without backward, storing outputs after ordinary backward, and storing outputs after retained backward. Separate output bytes from expected saved capacity and avoid counting shared parameters per iteration. State which case should show materially steeper growth and which operator-specific saved-state assumptions make that prediction conditional.

Part 4 - Experimental Design and Instrumentation

Use independent replays with identical shapes, loss, seeds, and container limits. Stop before a configured allocation budget rather than seeking a physical OOM. Sample at matching completed boundaries and delete all stored roots at the end. Compare diagnostic graph records with allocator-visible bytes and host metadata growth. Keep evidence rows free of tensor objects, and verify that the reporter itself does not retain each iteration's loss.

Keep the stored values identical across cases, including shape and dtype, so output payload is a fair control. Compare list storage with a bounded ring buffer to distinguish unbounded ownership from a fixed diagnostic window. Before-backward retention should use deferred work intentionally, while the after-backward cases must complete the selected backward policy before sampling. Do not accumulate multiple losses inadvertently while constructing the test. Release local loop variables and diagnostic roots as well as the container when testing cleanup. If host graph metadata grows after ordinary backward while CUDA saved-state capacity does not, report that split outcome explicitly rather than calling the entire retained graph absent.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def run_retention_case(config, backward_policy, owner_policy):
    ...

def compare_retained_categories(case_rows):
    ...

def E07(config):
    # Bound iterations and release diagnostic graph references.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

A non-reproducing growth pattern is an outcome to explain, not permission to invent a trace. Verify that container length, stored output sizes, and selected backward policy match the design. Compare cleanup after container clearing with a fresh baseline while allowing cache retention. Do not interpret stable reserved memory as proof that no new live outputs were retained, or a remaining grad_fn as proof that all saved tensors survived.

Part 7 - Engineering Interpretation and Philosophy

The important question is which obligation remains, not whether a single attribute exists. Explain why the stored-output experiment distinguishes legitimate diagnostics, unnecessary CUDA payload retention, and preserved backward work. A good repair removes an unwanted owner while preserving the intended derivative computation; it need not force the allocator to return cached segments to the driver.

19.48.9 Exercise 08

Exercise Title: Break an Unwanted Autograd History with detach() and Prove Which Graph Nodes and Saved Tensors Become Eligible for Release

Part 1 - Exercise Contract and Why It Matters

Remove an unwanted differentiation history from a diagnostics branch and identify which release conditions actually change. Compare detach, CPU export, and removal of the original owner. Deliver a proof of preserved training gradients for the diagnostic repair, plus a deliberately invalid intervention that detaches a required activation. Keep that invalid case as a counterexample, not as an optimization with equivalent training semantics.

Part 2 - System and Memory Mental Model

detach creates a tensor without the original differentiation history while sharing underlying storage. Keeping the original output, another graph root, or an alias can preserve the same backing allocation. Release eligibility requires all relevant owners and future consumers to disappear, and actual reuse must respect asynchronous completion. Detaching a forward activation needed by the loss changes which derivatives are computed; it cannot be justified merely by observing a lower peak.

Part 3 - Quantitative Prediction Before Execution

Predict graph reachability and storage ownership for an original output, a detached alias, a detached CPU copy, and a scalar diagnostic. Include a small view backed by a larger allocation. Estimate which CUDA capacity remains in each case and when removing the original root can matter. State why the prediction differs before backward and after ordinary backward has already released saved dependencies.

Part 4 - Experimental Design and Instrumentation

Use the bounded retention cases from E07 and move only the optional metrics path across the differentiation boundary. Record every remaining application owner, then clear them in a controlled order. Compare the required parameter gradients and optimizer updates with the original loop. Perform the invalid required-activation detach in a separate labeled replay and demonstrate its derivative mismatch. Do not hide that semantic change behind an apparently successful forward-value comparison.

For the valid diagnostic repair, compare at least several complete parameter updates from the same initial state, samples, and optimizer configuration. A matching loss in the first forward pass cannot prove unchanged gradients. Keep CPU export completion explicit before inspecting its values, and note that scalar extraction can introduce synchronization. Compare full diagnostic retention with a bounded summary rather than assuming every analysis needs all historical values. If the only way to meet the memory budget is truncating required history, classify that proposal as a different algorithm with its own validation criteria. Do not present it as an exact implementation repair of the original differentiation task.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def export_diagnostic(value, export_policy):
    ...

def test_release_after_owner_removal(config, owner_sequence):
    ...

def E08(config):
    # Validate the training derivative contract before accepting detach.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

A disappearing Python wrapper is not proof of released CUDA storage. Combine owner removal, scoped graph reachability, and matching allocator samples while preserving unknown native owners. Compare shared-storage identities only within controlled generations. Export derivative errors and not just output errors. If cleanup leaves a detached CUDA payload in a list, classify the remaining footprint as payload retention rather than claiming that detach failed to break the gradient edge.

Part 7 - Engineering Interpretation and Philosophy

A differentiation boundary is a mathematical decision. It is appropriate for metrics and explicitly nondifferentiated branches, but it must be reviewed against required gradient paths. Explain which evidence establishes safe history removal and which only establishes shared storage. Later analyzers may suggest detach as a candidate for such boundaries, but they must never automatically truncate required training derivatives to satisfy a budget.

19.48.10 Exercise 09

Exercise Title: Compare backward() with backward(retain_graph=True) and Measure the Memory Cost of Preserving the Graph for Reuse

Part 1 - Exercise Contract and Why It Matters

Compare ordinary backward with backward retaining saved state under identical forward computation and gradient requirements. Measure the post-backward footprint, full-step peak, and cleanup after roots are released. The experiment must distinguish preserving reusable saved values from creating a derivative graph. A single boolean should not be treated as a universal leak switch without considering which graph owners remain reachable.

Part 2 - System and Memory Mental Model

retain_graph controls whether the graph's saved state is freed for the relevant backward traversal. It does not by itself construct the differentiable graph of the derivative; create_graph addresses that separate requirement. Retaining one graph for a legitimate subsequent traversal

differs from retaining a new graph every iteration in a container. Gradient accumulation also remains a separate source of live buffers and changing numerical results.

Part 3 - Quantitative Prediction Before Execution

Predict retained saved capacity using the unique storage inventory, excluding unavoidable parameter overlap. Compare a one-graph retained replay with a bounded multi-graph history. Explain why the retained-state difference may appear after backward yet not change a forward-dominated overall peak. Predict the effect of a final non-retaining backward and of dropping every root without another traversal.

Part 4 - Experimental Design and Instrumentation

Generate fresh inputs and model state for each policy. Match gradient clearing and include an identical completed sampling point after backward. Preserve only the roots required by the chosen experiment; ensure the diagnostic traversal is not another hidden root. Use operators with saved values and a trivial no-save control. Record selected options explicitly rather than relying on implicit defaults that may depend on create_graph.

Record the number of retained graphs and the intended future use of each. A single graph intentionally reused twice is a different workload from many abandoned retained graphs. Include a control that deletes all roots without a second backward, and distinguish logical release eligibility from actual completion-dependent allocator reuse. If measuring retained state after backward, keep gradient and optimizer state identical between cases. Do not use a fresh optimizer in one case and a warmed optimizer in the other. Report whether the output itself is large enough to dominate the remaining memory, so a modest saved-state difference is not mistaken for a failed experiment.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def compare_backward_retention(config, retain_policy):
    ...

def measure_after_final_use(state, sampling_scope):
    ...

def E09(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Check gradients against the same reference before interpreting memory savings. Repeated backward accumulates gradients unless cleared; account for that numerical contract. A no-save graph can show little retained CUDA difference, which is valid. Report output storage and residual graph metadata separately from saved activations. Treat allocator reservation as retained cache unless other evidence establishes live ownership; empty_cache is not a substitute for releasing a reachable graph.

Part 7 - Engineering Interpretation and Philosophy

Retention is justified by future derivative work, not by habit. Explain which subsequent computation requires the preserved state and when that obligation ends. Compare retaining, recomputing the forward, and combining losses in a later exercise. The preferred policy minimizes unnecessary residency while preserving all requested derivatives, and its benefit must be evaluated at the actual whole-step peak.

19.48.11 Exercise 10

Exercise Title: Perform Multiple Backward Passes Through the Same Graph and Determine Exactly When retain_graph=True Is Required

Part 1 - Exercise Contract and Why It Matters

Perform multiple vector-Jacobian products through a shared forward graph and establish when earlier traversals must preserve saved dependencies. Compare retention, recomputation, and a combined-loss traversal. Deliver gradients, memory, and runtime for each valid strategy. The word exactly applies to the chosen graph and requested derivative task; it must not become a universal claim that every repeated backward always fails without retention.

Part 2 - System and Memory Mental Model

A later traversal needs retained state if it revisits backward operations whose required saved values were already freed. Some trivial derivative formulas require no saved values and may permit repetition without an error. Such behavior is not a general programming guarantee. Independent forward graphs do not require shared retention. For scalar losses, combining objectives can compute the sum of gradients without separately traversing a shared subgraph twice.

Part 3 - Quantitative Prediction Before Execution

Predict the parameter gradient for two losses sharing an intermediate, including the weighting and accumulation convention. Compare two retained traversals with one combined traversal and two independently recomputed forwards. Estimate how long saved capacity persists under each schedule. Include a nonlinear saved-value case and a simple no-save control; predict different failure behavior without using the control as a universal rule.

Part 4 - Experimental Design and Instrumentation

Keep parameters unchanged until all derivative work using their forward values has completed. Avoid optimizer steps or in-place mutations between traversals. Use explicit retain settings for the nonfinal and final uses, then clear accumulated gradients consistently. If computing separate gradients with autograd.grad, compare each requested result rather than only their sum. Track forward count and timing so recomputation cost is not excluded from an allegedly equivalent benchmark.

Write the derivative requests in advance: individual loss gradients, a weighted sum, or multiple vector-Jacobian products. These are related but not identical outputs. Document any normalization by batch size and the effect of gradient accumulation. Recomputing requires matching input values and relevant random or mutable state; start with a pure deterministic function before introducing those complications. Include a test where two losses share only parameters but come from independent forwards, so the learner does not infer graph sharing from parameter identity alone. Treat an attempted optimizer update between shared traversals as a rejected schedule and preserve the reason in the evidence record.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def execute_derivative_schedule(config, schedule):
    ...

def compare_requested_gradients(results, oracle):
    ...

def E10(config):
    # Reject parameter mutation before all old-value consumers finish.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Record whether a repeated traversal fails, the failure phase, and the installed operator behavior. Do not depend on exact exception wording. Confirm that the final use permits saved-state release and that logging has not retained roots. Check the combined-loss gradient against the weighted sum while recognizing that it does not return individual gradients by itself. Validate memory and runtime using the same total requested work for each comparison.

Part 7 - Engineering Interpretation and Philosophy

Scheduling derivative work is a memory decision. Retaining the graph may be appropriate for separate derivative queries; a combined objective or fresh forward can remove that need in other tasks. Explain the smallest valid schedule for the application and the evidence that its release boundary is correct. An optimization is meaningful only after the required outputs of differentiation have been specified.

19.48.12 Exercise 11

Exercise Title: Reproduce a Memory Explosion from Repeated create_graph=True Usage and Explain How Higher-Order Gradient Graphs Expand Memory Requirements

Part 1 - Exercise Contract and Why It Matters

Demonstrate bounded growth caused by retained higher-order derivative work without asserting that create_graph always causes an explosion. Compare fresh independent graphs that are released with a differentiable unroll or stored derivative history that remains reachable. Deliver the derivative dependency map, memory trend, and explicit stopping policy. A large measured increase is an outcome; no physical allocation failure is required.

Part 2 - System and Memory Mental Model

create_graph records operations used to compute derivatives so that later derivatives can flow through them. The resulting derivative graph has its own saved values and owners. Repeated use can remain bounded when old work is released. Using backward with create_graph can create parameter-gradient reference cycles; autograd.grad is the preferred starting point for this exercise, with explicit ownership and cleanup of returned derivative tensors.

Part 3 - Quantitative Prediction Before Execution

Choose a smooth nonlinear objective with a nonzero second derivative. Predict which derivative outputs still require gradients and how an unroll of depth k extends dependencies. Do not assume linear or exponential memory growth without deriving the chosen construction's retained work. Include a constant derivative example to explain why create_graph does not force every returned gradient to carry further differentiable structure.

Part 4 - Experimental Design and Instrumentation

Sweep a modest unroll depth under both host and CUDA budgets, starting from fresh state per depth. Separate retained derivative outputs from CPU-only trend records. Use an analytic scalar or small-vector oracle before scaling. If including backward-created gradient cycles as a secondary control, specify how .grad references are removed during cleanup. Stop on configured admission limits and record budget rejection separately from CUDA OOM.

Choose both a released independent-iteration control and a deliberately connected unroll. Track whether each new state depends differentiably on the previous state and whether returned gradients are appended to a history. These are separate mechanisms and should be changed one at a time. In the optional backward-based cycle case, document clearing .grad references to None after use, rather than assuming numeric zeroing breaks object relationships. Restrict the highest depth using predicted storage and a safety margin. If a budget stops the sweep, report the largest admitted depth and measured trend without extrapolating a hardware failure that was never observed.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def build_bounded_derivative_unroll(config, depth):
    ...

def release_derivative_owners(state, gradient_policy):
    ...

def E11(config):
    # Compare retained unrolls with independent released graphs.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate first and second derivatives before trusting the memory experiment. Report graph ownership, retained depth, logical saved bytes, unique capacity, and measured peak as different quantities. A failure because a derivative is constant is not evidence of corrupted autograd. Compare cleanup with a fresh baseline and inspect lingering gradient references. Reject any conclusion that attributes unbounded growth to create_graph alone when the reproducer also stores an ever-growing history.

Part 7 - Engineering Interpretation and Philosophy

Higher-order differentiation requests more mathematical work, so its memory cannot be judged against a first-order task as if both obligations were the same. The useful question is whether retained history is necessary for the chosen derivative. Explain where a truncation would change the algorithm, where release is safe, and which growth pattern the bounded experiment actually supports.

19.48.13 Exercise 12

Exercise Title: Compare First-Order and Second-Order Gradient Computation by Node Count, Saved Tensor Memory, Peak VRAM, and Runtime

Part 1 - Exercise Contract and Why It Matters

Compare a first-order gradient task with a precisely defined second-order task, using graph structure, saved-state estimates, peak VRAM, and runtime. Use a Hessian-vector product as the primary higher-order result so that the experiment does not accidentally allocate a dense Hessian. Each task needs its own numerical oracle; the second-order run performs additional required work and is not simply an inefficient version of the first-order run.

Part 2 - System and Memory Mental Model

A first gradient computed with create_graph can support differentiation of its dot product with a fixed direction. That procedure produces a Hessian- vector product for a smooth scalar objective. Dense Hessian construction has a different output-size and memory contract. Node counts reflect observed backward dependencies and host graph metadata; they are not direct byte measurements and can change with implementation details.

Part 3 - Quantitative Prediction Before Execution

For n differentiated scalar inputs, compare the output payload sizes of a gradient, one Hessian-vector product, and a dense Hessian. Predict which saved values must survive until the second derivative is evaluated. Estimate a range for peak overlap rather than summing independently observed peaks. Use a quadratic or another smooth analytic objective to predict exact reference derivatives before any CUDA measurements.

Part 4 - Experimental Design and Instrumentation

Match model state, input values, precision, device, and warmup policy. Measure full task latency including the first gradient required by the second-order run. Keep the direction fixed and nondifferentiated unless an additional mixed derivative is explicitly requested. Count graph nodes only in separate diagnostic replays. Export derivative errors as CPU scalars and release all derivative roots at the stated final-use boundary.

Use a fixed normalized direction for the Hessian-vector product and preserve its seed. Validate the gradient of the scalar objective and the directional second derivative independently. Count nodes with the same traversal coverage for each task and avoid retaining the counting result's live node objects. If adding a dense Hessian experiment, bound its dimension and include output storage in the cost rather than comparing only internal graphs. Distinguish higher-order support failures of a chosen operator from a general autograd failure. Report whether timing includes graph construction, derivative execution, and result transfer, since excluding different components can reverse an apparently favorable runtime comparison.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def request_first_order(objective, inputs, policy):
    ...

def request_hessian_vector_product(objective, inputs, direction):
    ...

def E12(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Compare both requested derivatives with their own oracles and check finite differences on a small well-conditioned case where appropriate. Report dtype and error tolerances rather than demanding bitwise equality across different kernels. A zero Hessian can be mathematically valid. Include current allocation, peak allocation, reserved capacity, and cleanup results. Explain any graph count difference without claiming that twice as many nodes implies twice the GPU memory.

Part 7 - Engineering Interpretation and Philosophy

The comparison should reveal the incremental residency and execution cost of a declared derivative task. It should also identify unnecessary ownership that can be removed without changing that task. Explain why switching from a dense Hessian to a Hessian-vector product is a change in requested output, while releasing finished diagnostic records is an implementation improvement within the same mathematical contract.

19.48.14 Exercise 13

Exercise Title: Analyze a Residual Network and Determine Why Skip Connections Extend Activation Lifetimes Beyond Their Producing Layer

Part 1 - Exercise Contract and Why It Matters

Analyze activation lifetimes in a residual network using both forward consumers and backward requirements. Deliver a branch-aware timeline and a counterexample to the claim that every skip addition saves its entire input for backward. Distinguish architectural comparisons from correctness-preserving rewrites: a plain chain and a residual network generally compute different functions, so their outputs are not interchangeable equivalence references.

Part 2 - System and Memory Mental Model

A skip value must survive until its forward merge consumer has read it. The backward formula for an addition often does not need the input values, while operations within the branch may save substantial state. Aliases, repeated uses, normalization, and gradient requirements can extend lifetimes further. The producing layer's end is therefore not a universal release point, but the skip edge alone does not prove backward-time retention of that value.

Part 3 - Quantitative Prediction Before Execution

Draw a residual block with a long branch and identify the last forward reader of the skip tensor. Then add an operation whose derivative saves a related value and predict the new lifetime. Calculate unique live storage at several cuts through the schedule. Explain how branch overlap can move the memory peak even when the total logical number of produced activation elements is unchanged.

Part 4 - Experimental Design and Instrumentation

Use a deterministic residual model with named branch boundaries. Compare forward-use metadata with saved-tensor observations and completed memory samples. Include an addition-only control and a nonlinear branch that saves values. Validate any proposed early release or offload within the same residual function. Track whether a view or another consumer still owns the backing storage before declaring the skip tensor reclaimable.

Mark branch creation, merge submission, merge completion, and saved backward reads as separate observations. Forward submission order does not establish that the skip storage is safe to overwrite. Include a case where an intermediate is reused twice within the branch and another where the skip is a view. For an offload candidate, retain a valid restoration route and account for source lifetime through the copy. Compare the proposed interval against actual last readers and aliases. The residual's existence may explain a longer forward lifetime, but only observed derivative requirements justify extending the same value through backward. Preserve both kinds of evidence in the annotation.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def trace_residual_obligations(config, region_labels):
    ...

def predict_branch_overlap(storage_records, consumer_events):
    ...

def E13(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Check outputs and parameter gradients against the original residual model for all interventions. Compare observed forward lifetime extension with saved backward lifetime extension separately. Report cases where the skip shares storage with another activation or where a compiler changes the operation representation. Keep eager execution as the primary controlled scope. A higher memory peak than the chain does not by itself identify the exact residual value responsible.

Part 7 - Engineering Interpretation and Philosophy

Residual connections illustrate why a local layer view can miss future users. The correct release boundary follows every required consumer, including those outside the producing branch.

Explain which lifetime extension belongs to the architecture and which belongs to an implementation or an accidental owner. That distinction determines whether optimization requires recomputation, transport, ownership cleanup, or a change in model semantics.

19.48.15 Exercise 14

Exercise Title: Build a Tensor Lifetime Timeline from Forward Creation Through Last Backward Use and Identify Opportunities for Early Release or Offloading

Part 1 - Exercise Contract and Why It Matters

Build a timeline from value production through the last required consumer and identify conservative release or offload opportunities. Represent uncertainty between host observations and actual device completion. Deliver both a host event log and a dependency-aware GPU interpretation. A hook callback timestamp must not be presented as an exact CUDA execution timestamp or a proof that storage is immediately safe to reuse.

Part 2 - System and Memory Mental Model

A saved tensor can be unpacked before the consuming kernel finishes reading it. Several consumers, aliases, or streams can share the backing allocation. Python last-use observations therefore provide only part of the lifetime argument. GPU events and explicit stream dependencies can bound completion; multiple streams form a partial order rather than one universally comparable sequence of wall-clock events. Pending frees also separate owner disappearance from allocator reuse.

Part 3 - Quantitative Prediction Before Execution

For each observed storage, predict production, forward last use, backward retrievals, and final completion bounds. Calculate a conservative byte-time measure over the chosen observation clock, documenting its limits. Include a cross-stream reader that outlives the last Python unpack event. Predict why reclaiming at that callback would be unsafe and how an explicit completion dependency changes the admissible reuse boundary.

Part 4 - Experimental Design and Instrumentation

Use bounded CPU metadata records with event IDs and causal links. Place CUDA events on the relevant streams in a separate timing replay and join only where the measurement contract requires completion. Preserve shared save payloads until all required consumers finish. Avoid device-wide synchronization after every callback because it would destroy the overlap being studied. Record timestamp domains and distinguish measured intervals from inferred or conservative bounds.

Define interval endpoints and inclusion conventions in the exported schema. If using half-open intervals for completed ownership, explain how simultaneous events and unresolved completion are represented. Never subtract timestamps from different devices or clock domains without a supported relationship. On multiple streams, use causal dependencies to justify ordering and retain incomparable events as such. Include a payload unpacked twice and verify that the first retrieval does not close its lifetime. If callback events are missing, keep the upper bound open. Validate the byte-time calculation on synthetic intervals, including overlapping aliases that must contribute only one backing capacity to the unique live envelope.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def record_lifetime_observation(kind, storage_id, causal_links):
    ...

def derive_completion_bounds(host_records, stream_events):
    ...

def E14(config):
    # A final unpack callback is not final device-read completion.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate the timeline against a small serial case with deliberately completed boundaries, then add shared and asynchronous cases. Reject negative durations caused by mixing clocks instead of silently clipping them. If the final reader is unknown, retain an open lifetime bound. Compare peak estimates with allocator measurements while accounting for missing workspace. Report the timeline's collection overhead and any scheduling changes introduced by timing events or hooks.

Part 7 - Engineering Interpretation and Philosophy

A conservative timeline is useful even when exact physical release is unknown. It can rule out unsafe reuse and identify intervals long enough to consider offloading. Explain which opportunity remains valid under the uncertainty bounds and what additional observation would narrow them. An early-release recommendation must satisfy both ownership and execution dependencies, rather than relying on a visually convenient gap in a host trace.

19.48.16 Exercise 15

Exercise Title: Build an Autograd Memory Graph That Annotates Every Node with Saved-Tensor Bytes, Lifetime, and Downstream Consumers

Part 1 - Exercise Contract and Why It Matters

Integrate observed backward nodes, saved slots, and underlying storages into an auditable memory graph. Annotate every observed node with coverage status, attributed saves where supported, and lifetime bounds. Preserve shared and ambiguous relationships instead of assigning each allocation to one apparent owner. Deliver a schema and a small validated export that can support later policy recommendations without claiming complete visibility into all autograd or allocator internals.

Part 2 - System and Memory Mental Model

A node-to-slot-to-storage model separates computational dependency from memory ownership. Several nodes can save references to one backing allocation, and one node can need several saves. Reverse dependency edges describe consumers within the observed graph, not

every possible application consumer. Some node attributes are unsupported or unmapped; zero is appropriate only when absence is established under the declared inspection coverage.

Part 3 - Quantitative Prediction Before Execution

Predict graph records for a shared parameter, overlapping views, and a saved activation used by two branches. Compute logical slot totals and unique live storage envelopes separately. State why summing per-node reference totals does not reconstruct the peak. Add persistent state and unobserved workspace as separate categories, and predict how an unresolved attribution changes a confidence statement without altering the raw observation.

Part 4 - Experimental Design and Instrumentation

Define run IDs, traversal IDs, storage generations, scope labels, and nullable fields before integration. Link hook observations only through supported application attribution or controlled examples; callback order alone is not a node mapping. Build reverse edges with coverage annotations. Export scalar metadata and bounded adjacency records, then release diagnostic objects. Compare an analyzer-enabled replay with a clean replay to detect observer retention and instrumentation costs.

Version the export schema and attach field provenance. Distinguish observed capacity, derived logical size, inferred association, and unsupported data. Use stable application labels to compare replays, while keeping pointer and node identifiers local to their run. Provide a synthetic graph with an unmapped save so downstream aggregation must propagate uncertainty. Check that removing a node reference from the export does not erase a still-shared storage record. Keep a separate persistent-state catalog for parameters and gradients. Reports should expose both per-node references and storage-level occupancy without presenting either as a complete measurement of context, communication, or opaque backend workspace.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class MemoryGraph:
    nodes: tuple
    saved_slots: tuple
    storages: tuple
    relations: tuple
    coverage: object

def build_memory_graph(traversal, save_events, lifetime_bounds):
    ...

def E15(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate schema invariants on synthetic graphs before collecting CUDA data. Every relation must reference an existing record, shared storage must remain shared, and unknown capacity must never become zero through aggregation. Test an allocation address reused in another generation. Compare a known small computation with its exported graph and explain remaining mismatches. Record whether a lifetime is measured, inferred, bounded, or unresolved in the exported field itself.

Part 7 - Engineering Interpretation and Philosophy

The integrated graph should preserve distinctions needed for decisions rather than conceal them behind a single attractive number. Explain how a recommendation can trace back to observations, assumptions, and missing evidence. Completeness is scoped to the inspected roots and supported save paths. An explicit unknown is more valuable than a fabricated node attribution that could lead to an unsafe detach or premature storage reuse.

19.48.17 Exercise 16

Exercise Title: Compare Normal Backpropagation, Activation Checkpointing, and Saved-Tensor CPU Offloading Using the Same Autograd Memory Graph

Part 1 - Exercise Contract and Why It Matters

Compare normal backpropagation, activation checkpointing, and saved-tensor CPU offloading for the same model and derivative task. Deliver a memory-time-host RAM comparison and explain the different obligations each technique changes. Use the memory graph as a common application-level reference, while allowing the observed backward representation and number of recomputation events to differ. Numerical validation is a requirement before any memory-saving claim.

Part 2 - System and Memory Mental Model

Checkpointing retains enough inputs and state to recompute selected forward work instead of keeping all intermediate saved values. CPU offloading preserves values in another memory tier and restores them for backward. Neither makes all activation memory disappear. Transfers need host capacity and completion dependencies; recomputation consumes time and can move the peak. Hook contexts can override rather than automatically compose with one another.

Part 3 - Quantitative Prediction Before Execution

Predict saved CUDA capacity avoided, checkpoint input residency, CPU payload, transfer volume, and recomputation work for one selected region. Include the possibility of temporary coexistence during transport. Estimate latency from measured transfer and compute costs without assuming perfect overlap. Predict which region is the best candidate under a fixed host budget, then state what observation could reverse that prediction.

Part 4 - Experimental Design and Instrumentation

Use version-checked non-reentrant checkpointing with use_reentrant=False as the primary checkpoint policy. Start with deterministic pure region functions and explicit inputs. Use supported save_on_cpu or a validated pack/unpack implementation for offloading, preserving values and restoration device. Collect diagnostic save graphs in separate passes when hook composition is not validated. Measure ordinary execution without heavy tracing and include backward plus optimizer update in the full-step region.

For CPU offloading, cap pageable and pinned host use and test restoration before scaling. With asynchronous pinned transfers, do not inspect CPU payload contents or recycle source and destination buffers before the relevant copy completes. Use completion dependencies appropriate to the actual streams. For checkpointing, include recomputed work in runtime and distinguish original and recomputation event generations. Compare application roles rather than requiring identical internal node graphs. If a logging hook overrides an offload hook, the run is not a valid offload measurement. Verify the active policy or use separate collection passes, and record the installed version's composition behavior in the reproducibility evidence.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
from torch.utils.checkpoint import checkpoint

def execute_memory_policy(model, batch, policy, host_budget):
    ...

def compare_policy_correctness(reference, candidates, tolerances):
    ...

def E16(config):
    # Validate hook/checkpoint composition for the installed version.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Compare losses, required parameter gradients, and several optimizer updates from identical initial state. Report host peak, CUDA peak, reserved bytes, latency, and transfer volume. If adding randomness, preserve and test the relevant RNG behavior; if adding mutable state, prove recomputation safety. Higher-order support requires its own validation. A missing feature or unsupported hook combination is a reported limitation, not an equivalent fallback silently inserted into the result.

Part 7 - Engineering Interpretation and Philosophy

Keep, offload, and recompute move different costs across time and memory tiers. Explain which policy addresses the measured peak and which merely lowers a noncritical endpoint. Compare at the same derivative contract and host budget. The best policy is workload-specific: storage lifetime, transfer readiness, and recomputation cost matter more than assuming that a particular technique always reduces whole-step memory or improves throughput.

19.48.18 Exercise 17

Exercise Title: Detect an Unexpected Long-Lived Activation by Combining Autograd Graph Traversal, Saved-Tensor Hooks, and CUDA Memory Snapshots

Part 1 - Exercise Contract and Why It Matters

Investigate one unexpectedly long-lived activation using graph dependencies, saved-tensor observations, and CUDA allocator snapshots. Deliver a causal hypothesis, a minimal discriminating intervention, and a report separating allocation origin from retaining ownership. The three observation sources have different scopes and timestamps; matching them must preserve uncertainty rather than forcing a complete one-to-one identity from an address alone.

Part 2 - System and Memory Mental Model

A snapshot describes allocator-visible blocks and states, while a graph traversal describes differentiation dependencies and hooks describe covered save activity. A creation stack can locate where storage originated without explaining why it survives. Pending stream readers and cached inactive blocks can resemble retention in coarse samples. Diagnostic tools themselves can keep tensors or node objects alive, creating the behavior they are meant to explain.

Part 3 - Quantitative Prediction Before Execution

Predict the expected lifetime of a chosen activation and the boundary after which its continued residency would be surprising. Construct alternative hypotheses: legitimate saved state, a Python diagnostics owner, an alias, pending completion, or analyzer self-retention. State which measurement would distinguish each pair. Estimate the unique capacity recoverable by removing one owner, including a case where another owner makes that estimate zero.

Part 4 - Experimental Design and Instrumentation

Use bounded, version-checked allocation history in a diagnostic replay and capture matching completed phase boundaries. Preserve allocation generations when correlating addresses. Record container insertions and removals with CPU metadata, and avoid storing suspect tensors in the evidence report. Compare an intervention removing only the optional owner against the original workload. Keep model math, gradients, and input schedule unchanged throughout the causal comparison.

Make the suspected owner optional so its removal does not alter the loss or gradient path. A debug list or hook-owned history is a useful controlled starting point. Preserve allocation-history bounds and note whether earlier events were overwritten before the snapshot. Include an address-reuse synthetic case so correlation cannot silently equate two generations. When a snapshot shows a block awaiting completion, compare a later deliberate completion boundary before classifying it as unnecessary retention. If creation frames are absent, keep the allocation origin unresolved. The final report must state which owner intervention was tested and whether its predicted unique-capacity relief was actually observed.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def correlate_retention_evidence(graph, saves, snapshots, owner_events):
    ...

def test_owner_intervention(config, intervention):
    ...

def E17(config):
```

```
# Preserve ambiguity and snapshot capture coverage.
...
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify that the suspected storage persists across the declared boundary and that its generation is not a reused address. Compare numerical results after the intervention and require repeated evidence of the ownership effect. If the suspected allocation remains, check aliases and pending work before rejecting the hypothesis. Missing frames or truncated history limit the report's coverage. Do not turn correlation between a large save and a peak into a causal proof.

Part 7 - Engineering Interpretation and Philosophy

A useful investigation narrows alternatives and identifies the smallest next experiment. It may conclude that a value is legitimately live or that the current evidence cannot identify an owner. Explain which finding is established, which remains a ranked hypothesis, and what change was validated. Attribution confidence should follow the evidence chain, not the apparent precision of a stack trace or storage pointer.

19.48.19 Exercise 18

Exercise Title: Stress-Test Autograd Memory with Long Sequences, Residual Connections, Higher-Order Gradients, Retained Graphs, and Selective Checkpointing

Part 1 - Exercise Contract and Why It Matters

Stress-test the analyzer across long sequences, residual branches, higher-order derivatives, retained graphs, and selective checkpointing. Include healthy controls and instrumentation-induced retention. Deliver a coverage matrix, correctness outcomes, overhead measurements, and minimal counterexamples to unsupported universal rules. Stress means exploring difficult interactions under explicit budgets, not deliberately exhausting a shared machine until it becomes unusable.

Part 2 - System and Memory Mental Model

Interactions can invalidate assumptions that held in isolated exercises. Checkpoint recomputation changes save-event sequences; higher-order graphs introduce new derivative obligations; shared storage couples release decisions; and asynchronous readers extend completion bounds. Dynamic shapes change capacity without implying a leak. A single memory trend or node count cannot distinguish these causes without the workload's ownership and derivative contract.

Part 3 - Quantitative Prediction Before Execution

Predict expected categories for each test: bounded healthy residency, intentionally retained state, ambiguous attribution, unsupported feature, or invalid semantic rewrite. Estimate admission limits from conservative live storage and host logging budgets. Design a pair of cases with similar peak VRAM but different ownership causes, and another pair with similar saved-slot totals but different unique backing capacity. State what the analyzer should and should not infer.

Part 4 - Experimental Design and Instrumentation

Build a deterministic small test matrix before scaling sequence length or unroll depth. Add dimensions one at a time, then selected interacting pairs. Use fresh initial state and separate diagnostic and ordinary measurement replays. Put deadlines, event limits, and memory admission bounds around every case. Validate checkpoint and hook interactions against the installed version. Record explicit not-supported and not-measured statuses instead of silently omitting difficult combinations.

Maintain separate expected labels for saved-state retention, output payload retention, graph metadata retention, asynchronous pending use, and legitimate persistent state. A case can have multiple causes. Add a bounded dynamic-shape sequence where memory stabilizes after reaching its maximum shape, and a fixed-shape case with an expanding diagnostics container. Keep an intentionally incorrect hook payload as a rejected correctness control. Evaluate the system on cases not used to choose ranking thresholds. Report collection slowdown, host log size, and unsupported feature interactions alongside successful detections. A refusal to recommend under missing evidence can be correct behavior and should not be scored as a false negative automatically.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def build_stress_matrix(config, feature_support):
    ...

def run_admitted_case(case, budgets, evidence_sink):
    ...

def E18(config):
    # Separate healthy, invalid, unsupported, and retained cases.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Score graph coverage, storage deduplication, lifetime bounds, and recommendation correctness separately. An accurate alert with a wrong owner is not successful root-cause analysis. Validate all claimed equivalent outputs and gradients; invalid detach rewrites must be rejected even when memory improves. Report false positives on warmup, caches, and legitimate retained derivative work. Keep failures to reproduce and budget rejections in the results, with their actual reasons.

Part 7 - Engineering Interpretation and Philosophy

The stress suite should expose the analyzer's boundary of competence. Explain which assumptions survived interaction and which require narrower support claims. A counterexample improves the system when it becomes an explicit eligibility rule or an uncertainty field. The goal is not to produce a recommendation for every case, but to make valid recommendations traceable and prevent unsupported ones from being mistaken for safe optimizations.

19.48.20 Exercise 19

Exercise Title: Build and Benchmark a PyTorch Autograd Memory Analyzer That Maps Saved Tensors, Graph Dependencies, Lifetimes, and Peak VRAM to Recommend Keep, Offload, Recompute, or Detach Decisions

Part 1 - Exercise Contract and Why It Matters

Integrate a bounded PyTorch autograd memory analyzer that connects observed saved values, graph dependencies, ownership, lifetime bounds, and measured whole-step peaks. Produce candidate keep, offload, recompute, or detach decisions with explicit eligibility checks and validating replays. Deliver an auditable report and benchmarks against simpler policies. The runtime must be allowed to return insufficient evidence instead of automatically modifying an uncertain gradient path.

Part 2 - System and Memory Mental Model

Correctness constraints precede cost ranking. Keep preserves the value in place; offload changes residency with restoration obligations; recompute requires a reproducible region and its inputs; detach changes differentiation and is eligible only for declared nondifferentiated branches or an explicitly different algorithm. Shared storage, future consumers, transport buffers, and incomplete attribution constrain the savings available from each action. A graph annotation is evidence for a decision, not execution authorization by itself.

Part 3 - Quantitative Prediction Before Execution

Predict a candidate's reduction at the actual peak rather than its isolated logical size. Compare expected time costs for transport, recomputation, and waiting, with host and CUDA capacity constraints. Include shared capacity that would remain after the action. Rank only eligible candidates and preserve uncertainty intervals. Define success as validated required derivatives within budget, accompanied by measured latency and analyzer overhead.

Part 4 - Experimental Design and Instrumentation

Integrate the previous schemas, supported collectors, scope-aware deduplication, and conservative lifetime analysis. Separate collection, recommendation, and validation stages. Use known small cases for development and held-out workloads for evaluation. Benchmark normal execution, analysis overhead, and the selected policy independently. Bound capture frequency, host log size, replay count, and optional intervention attempts. Keep all automatic actions inside explicitly validated capability and correctness limits.

Define the final report before running integration. Each candidate needs a run ID, application target, derivative contract, evidence coverage, unique-capacity estimate, timing assumptions, host requirement, eligibility decision, and validating replay. Keep predicted effects separate from measurements and interpretation. Preserve rejected actions with reasons, particularly required-gradient detach, unsafe asynchronous reuse, and unsupported hook composition. Measure end-to-end analysis and validation cost rather than only the resulting optimized step. Do not claim global optimality from a greedy candidate search. Report the best validated policy in the tested range, remaining uncertainty, and the smallest next experiment that could improve the decision.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class Recommendation:
    action: str
    target: object
    prerequisites: tuple
    predicted_effect: object
    uncertainty: tuple

def propose_actions(memory_graph, derivative_contract, budgets):
    ...

def validate_candidate(config, recommendation, reference):
    ...

def E19(config):
    # Refuse unsafe detach and unsupported release assumptions.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require an evidence chain from each recommendation to observations, assumptions, eligibility rules, and an independent validation result. Report losses, gradients, updates, peak bytes, host footprint, runtime, and unsupported cases. Compare against always-keep and one simple eligible policy rather than only favorable handpicked examples. Measure prediction error separately from diagnostic value. A recommendation that saves memory while changing required derivatives is a correctness failure, regardless of its apparent capacity benefit.

Part 7 - Engineering Interpretation and Philosophy

The finished analyzer should help an engineer decide what to investigate or change with confidence proportional to the evidence. Explain remaining uncertainty about opaque state, execution timing, and hidden owners. Prefer a small supported policy set with clear counterexamples over broad automatic claims. The strongest result combines validated savings, preserved mathematical obligations, bounded instrumentation, and an honest statement of cases where keeping the existing computation is the only justified action.

19.49 DMM47

Chapter Title: Detach, No-Grad, and Inference Mode Memory Control
Specialization: Dynamic Memory Management and CUDA Systems

19.49.1 Exercise 00

Exercise Title: Build a Precise Mental Model of Gradient Tracking, Graph Construction, Tensor History, and Memory Saved When Autograd Is Disabled

Part 1 - Exercise Contract and Why It Matters

Define gradient tracking as a requirement of the workload rather than a switch chosen from memory pressure alone. Distinguish default grad execution, no-grad execution, inference execution, parameter freezing, output detachment, and module evaluation behavior. Deliver a decision table with required parameter and input derivatives, future output consumers, ownership, and mode eligibility. Every later exercise must preserve this declared derivative contract before interpreting any reduction in memory.

Part 2 - System and Memory Mental Model

Graph construction depends on execution mode, differentiable operations, and input gradient requirements. Disabling recording avoids corresponding backward state but still allocates outputs, temporary workspace, parameters, and caches. model.eval changes behavior of relevant modules without disabling autograd. A detached result shares storage with its source. Inference execution removes additional tracking overhead but imposes stronger restrictions on tensors it creates, especially when later autograd work would need those values.

Part 3 - Quantitative Prediction Before Execution

Predict memory as persistent state plus concurrently live payload, saved backward state, and temporary capacity. Avoid summing peaks observed at different times. For a saved view, compare logical bytes with unique backing capacity. Construct a workload with no trainable inputs or parameters and predict why default execution may already construct no backward graph. Explain why disabling autograd does not imply zero CUDA allocation or a fixed percentage saving.

Part 4 - Experimental Design and Instrumentation

Begin with a deterministic eager model and explicit derivative requirements. Create separate training and evaluation contracts, recording model behavior mode independently from grad mode. Name the future consumers of each exported tensor. Include one evaluation result used only for metrics and one feature used by a later trainable layer. Mark unsupported layouts or differentiation features explicitly. Use synthetic accounting examples before attempting real CUDA measurements.

Preserve GPU identity, installed software versions, allocator backend, seed, model dimensions, optimizer, precision, and initial state identifier. Separate cold setup from warmed measurements and use additional seeds only after a fixed-seed comparison is stable. State whether timing includes transfers, collation, calibration, metric finalization, and recovery. Use deliberate completion boundaries and reset peaks for the declared region. A training comparison includes forward, backward, and optimizer update; allocation-only or forward-only experiments need their own explicit regions. Record host and CUDA budgets separately, and never retain live graphs in the evidence log merely to simplify later reporting.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class DerivativeContract:
    parameter_targets: tuple
```

```
    input_targets: tuple
    output_consumers: tuple
    forward_ad_required: bool

def classify_mode_eligibility(contract, tensor_provenance):
    ...

def E00(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require counterexamples to the claims that evaluation disables graph recording, that detachment creates independent storage, and that inference allocates no payload. Validate the requested derivatives on a small analytic case. Keep predictions, observations, and interpretations separate. Every measured row needs a run ID, configuration, shape, byte budget, timing scope, outcome, and failure phase when applicable. Unknown capacity or unsupported behavior must not silently become zero. Report a missing CUDA measurement honestly rather than supplying representative hardware numbers.

Part 7 - Engineering Interpretation and Philosophy

The purpose of mode control is to remove obligations the application never requested while preserving those it did. Explain which distinction changes a real decision: graph history, storage ownership, module behavior, or later consumer compatibility. Carry that explanation into the controller design. A lower peak obtained by silently removing a required derivative is a correctness failure, even when forward values still look plausible.

19.49.2 Exercise 01

Exercise Title: Compare a Normal Forward Pass, torch.no_grad(), and torch.inference_mode() by Peak VRAM, Allocations, and Runtime

Part 1 - Exercise Contract and Why It Matters

Compare the same forward workload under default grad, no-grad, and inference execution. Measure whole-region peak allocation, supported allocation activity, reserved capacity, and latency. Deliver an output equivalence check and a scoped interpretation of memory differences. Forward-only comparisons should not include backward in only one candidate; a separate training reference can explain additional costs without becoming a misleading benchmark baseline.

Part 2 - System and Memory Mental Model

Default execution can save values for backward when differentiable inputs or parameters require gradients. No-grad suppresses reverse graph recording for the scoped computations. Inference execution additionally avoids some tracking work. These changes do not necessarily remove parameter storage, output payload, workspace, or allocator caches. Similar reserved memory between modes can coexist with different live allocations, while allocation counts depend on the instrument's accounting layer and backend support.

Part 3 - Quantitative Prediction Before Execution

Predict which tensors remain live at the forward endpoint and which disappear when its output is released. Estimate saved-state savings from a diagnostic inventory, deduplicating shared storage. Define latency and memory improvement using the same reference scope and units. Predict a case with little additional inference benefit and another with many small eager operations where tracking overhead could matter; these are hypotheses rather than guaranteed outcomes.

Part 4 - Experimental Design and Instrumentation

Hold model.eval status, inputs, weights, shapes, precision, and output lifetime constant across modes. Warm each candidate independently, separate cold setup, and reset peaks around the declared forward region. Use completed CUDA event intervals and a synchronized end-to-end region with explicit inclusion rules. Collect allocation diagnostics in separate replays. Do not call empty_cache inside one measured candidate or retain every output merely to make another candidate appear memory intensive.

Choose an ordinary dense eager workload before testing compilation or alternative backends. Preserve the same output-retention boundary, since deleting one candidate's output before sampling would invalidate the comparison. Define allocation activity using supported fields and take cumulative-counter deltas only over matching regions. An increasing lifetime allocation counter is normal for repeated temporary creation and is not a live allocation count. Report GPU event milliseconds and wall-clock seconds with explicit units. If the device timer and wall time disagree, investigate launch overhead, waiting, and scope differences before declaring one measurement incorrect.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def run_forward_mode(model, batch, mode, measurement_scope):
    ...

def compare_forward_evidence(rows, reference_outputs):
    ...

def E01(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Check output values at declared tolerances before comparing performance. Report current and peak allocated bytes separately from reserved bytes. State whether an allocation metric counts tensor creations, allocator requests, or driver segment growth; these are not interchangeable. Repeat trials and report latency distributions with instrumentation overhead. A null or negative measured benefit is valid. Preserve unsupported counters rather than approximating them from an unrelated byte difference.

Part 7 - Engineering Interpretation and Philosophy

Choose a mode based on both eligibility and measured benefit. A large graph- recording reduction can coexist with negligible runtime change in a compute- dominated forward. Explain whether the experiment addresses latency, capacity, or both, and which later consumers limit inference eligibility. The result should support a workload-specific decision rather than establish a universal ranking of the three execution modes.

19.49.3 Exercise 02

Exercise Title: Inspect requires_grad, grad_fn, and Version-Counter Behavior Under Normal, No-Grad, and Inference Execution

Part 1 - Exercise Contract and Why It Matters

Inspect tensor metadata and mode state without collapsing them into one boolean. Record requires_grad, grad_fn presence, leaf status, inference status, and supported version-counter behavior. Include tensors created before the context, tensors created inside it, and views. Deliver a provenance-aware table and explicit expected exceptions, demonstrating why the current mode alone does not describe every tensor visible in that scope.

Part 2 - System and Memory Mental Model

Entering no-grad does not retroactively clear requires_grad on existing tensors. Factory functions accepting a requires_grad argument are an important exception to the simple no-grad result rule. Inference tensors do not track version counters; inspecting _version on one can raise. Existing normal tensors and views require separate classification rather than an assumption that entering inference converts everything. The _version field is a diagnostic detail, not a stable public memory-management interface.

Part 3 - Quantitative Prediction Before Execution

Predict metadata for a differentiable input, a computed result, an explicit factory-created leaf, and a view in each context. Mark version access as supported, unsupported, or expected failure instead of substituting zero. Compare an actual mutation of a normal tensor with an inference tensor created in scope. State which predictions concern reverse AD and which require separate forward-mode AD testing.

Part 4 - Experimental Design and Instrumentation

Use fresh small tensors per case so one mutation cannot invalidate another test. Record the queried grad and inference context states alongside tensor provenance. Use the supported inference-status query to classify tensors independently. Catch only the expected diagnostic failure and preserve its phase. Test context restoration after exit and exception. Avoid reading full CUDA values during metadata collection, and do not use pointer equality as a proxy for gradient history or tensor kind.

Include an ordinary tensor allocated before inference, a fresh allocation inside inference, and a view of each. Verify their actual tensor kind; normal inputs are not converted retroactively. For supported diagnostic mutations, normal tensors retain normal versioning obligations even when used inside an inference scope. A view can also preserve relationships not captured by a simplistic all-results-are-false rule. Add a no-grad factory leaf with explicit requires_grad and distinguish that exception from recorded differentiable operations. Catch expected version-inspection errors without suppressing unrelated exceptions, and record whether each field is public or diagnostic.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def inspect_tensor_state(tensor, provenance, version_policy):
    ...

def probe_mode_metadata(config, cases):
    ...

def E02(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Verify each prediction against the installed version and keep actual exceptions in the evidence. Do not infer absence of a graph merely because one output is nondifferentiable. Include a pre-existing trainable parameter whose flag remains true inside no-grad while its newly computed ordinary result has no recorded reverse history. Distinguish factory behavior from differentiable operation recording. A successful metadata probe does not prove that later training consumption of an inference tensor is safe.

Part 7 - Engineering Interpretation and Philosophy

Tensor provenance and dynamic execution state answer different questions. The audit must preserve both if it is to explain unexpected graphs or compatibility errors. Explain which fields describe current context, intrinsic tensor properties, or version-sensitive implementation details. Use those boundaries to prevent the final controller from classifying tensors solely from a region label such as validation or preprocessing.

19.49.4 Exercise 03

Exercise Title: Use detach() on an Intermediate Tensor and Trace Exactly Where the Autograd Graph Is Cut

Part 1 - Exercise Contract and Why It Matters

Detach one selected intermediate and trace the derivative paths removed by that boundary. Compare a required activation path with an optional diagnostics branch. Deliver forward values, input and parameter derivatives, storage-sharing evidence, and a graph sketch. The experiment must make clear that matching forward results does not establish equivalent training after a graph cut.

Part 2 - System and Memory Mental Model

detach creates a new wrapper without the source differentiation history while sharing backing storage. Upstream graph state can remain reachable through other roots and consumers. Later operations on a detached value can create a new graph for their own trainable inputs, but that does not reconnect the original history. Calling requires_grad on a suitable detached value creates a new differentiation boundary rather than recovering the removed connection.

Part 3 - Quantitative Prediction Before Execution

Use a smooth two-stage scalar expression and predict derivatives for the uncut computation, a cut between stages, and a detached diagnostics-only branch. Identify exactly which requested derivatives disappear or change. Predict storage ownership before and after removing the original root. Include a shared branch so the learner can explain why one detach does not necessarily release the upstream graph or its saved values.

Part 4 - Experimental Design and Instrumentation

Run each variant from fresh identical input and parameter values. Keep the required derivative contract fixed for the equivalence tests, labeling deliberate contract changes separately. Traverse only diagnostic replays and export CPU metadata so graph inspection does not retain the measured roots. Check storage sharing within a live generation. Preserve all aliases until their planned release points and test whether cleanup changes the observed allocation baseline.

Keep a small reference expression whose chain rule can be checked by hand. Use separate roots for the main objective and diagnostics, then show how retaining the main root limits cleanup after detaching the diagnostic alias. When constructing a new detached leaf, distinguish its derivative from the derivative with respect to the original input. Include a constant derivative case so absent history is not confused with numerical zero. A candidate that passes forward checks but removes one requested gradient must remain explicitly rejected, even when the downstream trainable layer continues to receive plausible gradients.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def compare_cut_locations(config, derivative_contract):
    ...

def export_cut_dependencies(roots, traversal_scope):
    ...

def E03(config):
    # A new detached leaf does not reconnect the original history.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate analytical derivatives, not just loss values. Reject the required-path cut as an equivalent optimization even if it lowers VRAM. Verify that a metrics-only detach preserves every required parameter update. Report surviving aliases and graph roots before claiming release. Include a case where the source output remains stored and therefore preserves history despite another detached alias. Distinguish lost gradient connectivity from a computed gradient whose numerical value is zero.

Part 7 - Engineering Interpretation and Philosophy

A detach boundary should express an intended mathematical separation. It is useful for observation and explicit truncated algorithms, but automatic placement requires knowledge of

the workload's derivative targets. Explain why a memory controller cannot infer that intention solely from tensor size, lifetime, or a low observed gradient magnitude. The safest useful recommendation names the contract that authorizes the cut.

19.49.5 Exercise 04

Exercise Title: Compare detach(), detach_(), and clone().detach() by Storage Sharing, Allocation Cost, and Gradient Semantics

Part 1 - Exercise Contract and Why It Matters

Compare detach, detach_ and clone().detach using wrapper identity, storage sharing, allocation cost, and derivative semantics. Deliver a mutation-based alias test and expected failures for unsupported in-place detachment. Treat clone().detach as the requested comparison while also considering detach().clone as an independent-copy alternative. No case should imply that removing history alone produces an immutable snapshot of the source values.

Part 2 - System and Memory Mental Model

detach shares backing storage through a new tensor wrapper. detach_ changes the tensor's autograd relationship in place and cannot detach views in place. Other Python names referencing the same tensor object observe that object change; distinct aliases need separate analysis. clone creates independent storage and is differentiable in ordinary grad execution, so cloning before detaching can perform transient tracking work that is absent when detachment precedes the independent copy.

Part 3 - Quantitative Prediction Before Execution

Predict wrapper and backing identities, logical payload bytes, and surviving gradient paths for each operation. Include a contiguous tensor, a view with a nonzero offset, and a small view pinning a large allocation. Estimate copy payload from element count and dtype while distinguishing allocator rounding and nonstandard layout. Predict which source mutations affect the detached result and which leave an independent clone unchanged.

Part 4 - Experimental Design and Instrumentation

Use fresh owned tensors for each case and isolate mutation tests from derivative validation. Do not mutate a source needed by an unfinished backward. Record shape, stride, storage offset, supported backing capacity, and scoped identities. Attempt detach_ on a view only as a labeled failure control. Measure allocation and runtime in separate clean replays, with clone setup and output lifetime included consistently. Do not retain inspected tensor objects in metadata exports.

Add a second Python name referencing the exact source object and a distinct view sharing its storage. Compare how detach_ affects object metadata versus how later value mutations propagate through aliases. Do not attempt unsupported in-place detachment as a production fallback. Restrict storage accounting to supported dense layouts and scope identities to one allocation generation. For a small view, compare a detached alias with an independent compact copy, including temporary overlap while both exist. Explain when copying fewer logical elements can reduce long-term backing residency after the original owners are removed.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def compare_detachment_variants(config, cases):
    ...

def test_alias_mutation(source, candidate, mutation_contract):
    ...

def E04(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Check values before mutation, changes after mutation, and required derivatives where applicable. A successful pointer match does not prove immutable data, and a different tensor data pointer does not by itself prove different backing storage for offset views. Report expected view-detachment failures distinctly from allocation failures. Compare cleanup with all source and alias owners accounted for. Any measured transient clone history should be reported with its scope rather than generalized to a fixed overhead.

Part 7 - Engineering Interpretation and Philosophy

Choose sharing when the ownership and mutation contract permit it, and copying when an independent value snapshot is required. The choice affects both memory and semantics. Explain why a cheap detach can retain a large backing allocation, while a small independent copy may release it once all original owners vanish. The best option follows required values and lifetimes rather than a blanket preference for in-place operations.

19.49.6 Exercise 05

Exercise Title: Reproduce a Hidden Memory Leak by Storing Validation Outputs with Active Autograd History Across Many Batches

Part 1 - Exercise Contract and Why It Matters

Reproduce validation memory growth caused by storing outputs from a graph- tracked forward across batches. Bound the retained history and compare it with controls that store only payload or nothing. Deliver an ownership explanation, a memory trend, and cleanup evidence. Call the behavior unwanted retention when Python still owns the values; do not imply a defect in the allocator or that every validation loop necessarily leaks.

Part 2 - System and Memory Mental Model

Calling model.eval does not stop graph construction when differentiable parameters or inputs still require gradients. Storing an output before any backward can keep its graph and saved state reachable. Even without a graph, retaining CUDA predictions consumes memory. If ordinary backward has already run, much saved state may be released despite grad_fn remaining visible, so that case has a different retention contract from the primary validation reproducer.

Part 3 - Quantitative Prediction Before Execution

Predict growth for stored tracked predictions, stored detached predictions, and outputs released each batch. Separate output payload from unique additional saved state and shared persistent parameters. Derive a bounded trend expectation using fixed batch shape, then explain why allocator reservation can plateau while live bytes increase. Predict what happens after clearing the history and all remaining roots at a completed measurement boundary.

Part 4 - Experimental Design and Instrumentation

Use a deterministic model in eval behavior mode, with grad mode deliberately left enabled for the faulty primary case. Perform no optimizer updates during validation. Record history length, batch shape, output size, and supported allocated and reserved counters. Use fresh replays for controls, enforce a memory admission limit, and stop before exhaustion. Make the evidence logger CPU-only so it is not a second hidden owner of every output.

Track history insertion and removal as application events with CPU metadata. Use a fixed number of batches and identical output sizes in the primary case, then a bounded ring-buffer control to demonstrate intentional limited retention. Sample before and after deleting container contents, local output variables, and diagnostic traversal references. Compare after deliberate stream completion to avoid misclassifying pending use. If the model has no relevant gradient requirements, treat the failure to create history as a configuration finding. Do not force growth by changing the workload until it matches the expected picture.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def run_validation_retention_case(config, output_policy, mode):
    ...

def release_validation_owners(state, completion_policy):
    ...

def E05(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Confirm that saved outputs actually have recorded history in the faulty case. Compare post-batch growth and post-cleanup state with matching controls. A failure to reproduce the predicted slope is a legitimate result to investigate. Do not add empty_cache as a substitute for removing reachable owners. Report host graph metadata separately from CUDA payload and saved state. Verify that an increasing reserved counter alone is insufficient to identify the retained Python container.

Part 7 - Engineering Interpretation and Philosophy

The root cause is the mismatch between intended evaluation work and retained ownership. Explain which part of the growth comes from unnecessary history and which part is the deliberately stored prediction dataset. The repair must address both if bounded validation residency is the requirement. This distinction prevents a successful no-grad change from being overstated as eliminating all epoch-length output memory.

19.49.7 Exercise 06

Exercise Title: Fix Validation Memory Growth with torch.no_grad() and Prove That the Computation Graph Is No Longer Retained

Part 1 - Exercise Contract and Why It Matters

Repair the unwanted validation graph construction with no-grad and prove the scope of the repair. Deliver output equivalence, covered saved-state observations, and an ownership-bounded memory comparison. The exercise must preserve the counterexample where graph-free CUDA outputs still accumulate in a list. Absence of a backward graph does not establish constant total validation memory unless output retention is also bounded.

Part 2 - System and Memory Mental Model

No-grad suppresses reverse-mode recording for operations in its scope while leaving ordinary tensor payload and existing external graphs intact. The context does not delete previously stored tracked outputs, and it does not switch Dropout or BatchNorm behavior to evaluation. Factory-created leaves can retain explicit requires_grad settings. Proving that a selected forward creates no covered saved state is narrower than proving that no graph exists anywhere in the process.

Part 3 - Quantitative Prediction Before Execution

Predict the saved-state reduction for fresh validation batches after entering no-grad. Compare an empty history, a previously populated tracked history, and a new list of graph-free predictions. Estimate the irreducible output payload for the latter. State which observations establish suppressed recording and which establish released ownership. Predict why switching modes halfway through an epoch does not reclaim graphs retained from earlier batches.

Part 4 - Experimental Design and Instrumentation

Compare clean replays of the faulty loop, no-grad with unchanged output storage, and no-grad with bounded CPU summaries. Keep module behavior, weights, input order, and metrics identical. Use diagnostic saved-tensor hooks only in separate supported first-order replays, counting coverage without retaining values. Inspect selected output metadata and completed memory samples. Explicitly clear the old history when testing recovery rather than crediting the context manager with that cleanup.

Use an eligibility control where evaluation intentionally requests an input gradient, such as a small saliency calculation. That region must keep required tracking even though it uses eval behavior. The repaired ordinary-validation case should clear only phase-owned diagnostics, preserving persistent training state and any explicitly retained gradients. Compare no-grad output storage with a bounded metric accumulator and record the different information contracts. If hooks are unavailable or interact with another feature, retain output metadata and coverage limitations rather than inventing a complete graph-free proof from an unsupported zero save count.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
```

```
def validate_without_reverse_history(config, output_budget):
    ...

def compare_graph_suppression(reference, repaired, coverage):
    ...

def E06(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Check validation metrics and outputs against the original computation. Confirm no grad_fn on covered freshly computed ordinary outputs and no covered backward saves, while documenting nondifferentiable outputs and factory exceptions. A zero hook count is scoped evidence, not universal proof about opaque extensions or previously existing roots. Report continuing payload growth separately. Verify that grad mode is restored for the next training batch and that its required gradients still match the training reference.

Part 7 - Engineering Interpretation and Philosophy

A good repair removes the unwanted obligation and verifies the next legitimate one. Explain why graph suppression and output-budget enforcement are two related but separate controls. This separation makes the eventual runtime policy reviewable: each rule names the state it changes, the evidence supporting it, and the training derivatives or validation metrics that remain required.

19.49.8 Exercise 07

Exercise Title: Replace torch.no_grad() with torch.inference_mode() and Measure the Additional Memory and Runtime Differences

Part 1 - Exercise Contract and Why It Matters

Benchmark inference execution as a replacement for an already-correct no-grad evaluation region. Establish that its outputs do not need incompatible later autograd consumption. Deliver performance and memory distributions, compatibility checks, and a case with no meaningful additional benefit. The exercise tests the additional effect beyond no-grad; it must not attribute all graph suppression savings to inference mode alone.

Part 2 - System and Memory Mental Model

Both contexts suppress ordinary reverse graph recording. Inference mode additionally avoids some view and version tracking for inference tensors and disables forward-mode AD. It is thread-local and does not imply model.eval. Gains depend on the workload and may be predominantly host execution overhead rather than CUDA payload savings. Existing normal tensors are not all retroactively converted merely because the dynamic context changes.

Part 3 - Quantitative Prediction Before Execution

Predict which memory categories should remain similar between no-grad and inference execution and which overhead could differ. Compare a large matrix workload with a sequence of small eager operations. State that GPU peak reductions may be negligible while end-to-end

timing differs. Include any output conversion required by later consumers in the cost estimate rather than assuming an inference tensor can always cross the boundary unchanged.

Part 4 - Experimental Design and Instrumentation

Use identical eval behavior, initial state, inputs, precision, output policy, and measurement boundaries. Alternate or randomize benchmark order across independent warmed trials to reduce systematic drift. Record host wall time and completed device timing with clear scopes. Validate downstream consumers before declaring the region inference eligible. Keep allocation profiling separate from latency trials and record the installed version and active mode inside the actual execution thread.

Use paired trials with a consistent warmup criterion and report sample count, median, and tail behavior. Do not present tiny differences below measurement resolution as decisive improvements. Keep model behavior identical, particularly when testing Dropout or BatchNorm, and distinguish model evaluation from context selection. If forward-mode derivatives are required, inference is ineligible; no-grad alone does not disable forward AD. Document this separately from reverse-mode results so a controller trained on first-order backward examples does not generalize its policy to every differentiation mechanism without additional support checks.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def benchmark_inference_replacement(config, consumer_contract):
    ...

def summarize_incremental_effect(no_grad_rows, inference_rows):
    ...

def E07(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate output values and metrics while separately testing boundary compatibility. Report absolute bytes and time along with relative differences, using undefined ratios where denominators are invalid. If confidence intervals overlap or trials are too noisy, report an inconclusive advantage. Do not promise a positive saving. Include context-entry, conversion, and cleanup costs when part of production use. A fast isolated forward is insufficient if its output needs an expensive independent copy before the next stage.

Part 7 - Engineering Interpretation and Philosophy

Inference mode is a measured specialization for regions with stricter lifetime and consumer assumptions. Explain whether the tested gain survives the complete application boundary, and whether that gain justifies the additional compatibility rules. A correct no-grad region can remain the better choice when interoperability or forward AD matters. The controller should preserve that option instead of enforcing inference execution wherever backward is absent locally.

19.49.9 Exercise 08

Exercise Title: Reproduce an Error Caused by Reusing an Inference-Mode Tensor in a Later Gradient-Tracked Computation and Explain the Restriction

Part 1 - Exercise Contract and Why It Matters

Reproduce a controlled failure when a newly created inference tensor is later needed as saved data by a gradient-tracked operation. Repair the boundary with a normal independent copy created outside inference mode, or choose no-grad for the producer. Deliver a compatibility matrix and numerical gradient checks. Do not claim every conceivable arithmetic use outside inference must raise; use an operation whose derivative actually requires saving the inference value.

Part 2 - System and Memory Mental Model

Inference tensors lack normal version-counter guarantees required by autograd saved-value checks. A constant feature multiplied by a trainable input is a useful failure case because the input gradient needs that feature. Addition can be a misleading control because its derivative may not need the constant. A detach alias does not turn an inference tensor into an ordinary independent tensor. Enabling grad alone inside active inference mode does not remove the inference context.

Part 3 - Quantitative Prediction Before Execution

Predict the output and trainable-input gradient for a constant feature used in multiplication. Mark the inference-to-tracked boundary invalid when the value must be saved. Compare no-grad-produced features and an outside-context clone of inference features. Estimate the extra copy payload and transient overlap. Include a no-save arithmetic control as a version-checked observation, not a general guarantee that inference output can safely enter training.

Part 4 - Experimental Design and Instrumentation

Create fresh values inside each producer context and record their inference status. Exit inference mode before allocating the normal repair copy; verify its status and expected storage independence. Preserve the original producer's lack of differentiation history, since copying cannot reconstruct it. Exercise the later tracked operation and backward in separate labeled cases. Catch the expected compatibility error without depending on its exact wording, and retain the failure phase.

Make the failing value a newly allocated inference tensor rather than an ordinary tensor merely passed through the context. Test a multiplying consumer with a trainable ordinary input and compare it with a no-save control. The repair copy must be created with inference disabled; a context that only enables reverse grad is insufficient while inference remains active. Verify normal provenance after copying and budget for both source and copy during the transition. If a future consumer needs the original producer's gradient, reject both graph-free producer policies because no conversion can reconstruct the missing computation history.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
```

```
def test_inference_training_boundary(config, producer_policy):
    ...

def prepare_normal_consumer_value(value, boundary_contract):
    # Allocate an independent normal copy outside inference mode.
    ...

def E08(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Check the repaired trainable-input gradient against an analytical oracle and confirm that upstream producer gradients remain intentionally absent. Include a clone mistakenly made inside inference as a rejected repair. Do not assume .to or contiguous creates a new normal tensor when it can return the same object. Report conversion time and peak overlap. Distinguish expected boundary failure from a general claim that inference mode corrupts tensor values or that all later operations are unsupported.

Part 7 - Engineering Interpretation and Philosophy

The boundary is governed by future derivative requirements, not only the producer's local intent. Explain when no-grad avoids unnecessary conversion and when inference plus an explicit normal copy is justified. The repair must preserve value semantics, required downstream gradients, and the intended absence of upstream gradients. This exercise supplies a concrete eligibility rule for mixed pipelines and the final execution controller.

19.49.10 Exercise 09

Exercise Title: Build a Training Loop with Periodic Evaluation and Ensure Evaluation Allocations Do Not Accumulate into the Training Memory Budget

Part 1 - Exercise Contract and Why It Matters

Integrate periodic evaluation into a training loop without allowing evaluation owners to accumulate across phases. Preserve training gradients, optimizer state, and module behavior on return. Deliver per-phase memory baselines, validation metrics, and several matched training updates. The memory budget must include legitimate persistent training state during evaluation rather than pretending the model starts from an empty device at each phase.

Part 2 - System and Memory Mental Model

Evaluation changes module behavior independently of gradient recording. Training state such as gradient buffers and optimizer moments can remain resident through evaluation. New caches, predictions, and diagnostic owners can increase that residency. Returning to training must restore the actual prior module flags and context, including heterogeneous submodule states when relevant. Allocator reservation can remain high after validation even when its live payload has been released.

Part 3 - Quantitative Prediction Before Execution

Predict the pre-evaluation live baseline, temporary evaluation peak, post- evaluation baseline, and next training peak. Separate intentional retained training state from evaluation-only

owners. State how gradient accumulation changes the baseline and why evaluating between micro-batches must not clear partial training gradients. Predict a bounded plateau for repeated evaluation phases and a growth pattern when a phase-owned history is accidentally kept.

Part 4 - Experimental Design and Instrumentation

Begin with evaluation only at completed optimizer-step boundaries, then add a controlled accumulation case. Snapshot behavior flags and enter the selected gradient context explicitly. Bound validation output retention and release phase-owned references after required metrics finish. Restore state in exception- safe cleanup. Use fresh matched training replays with and without periodic evaluation, controlling random state and any mutable buffers that could change the next update.

Preserve per-submodule behavior flags if the model intentionally mixes training and evaluation behavior. Blindly calling train on the entire model afterward can change previously frozen evaluation submodules. Test restoration after an expected evaluation error without skipping cleanup. If evaluation consumes random numbers, control or record that state when comparing subsequent stochastic training updates. Include retained optimizer moments in the baseline even if validation never touches them. Report a one-time high-water mark separately from a continuing increase across repeated evaluation phases, using matching completed boundaries and the same gradient-clearing policy.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def evaluate_phase(model, validation_batches, phase_contract):
    ...

def restore_phase_state(saved_state, exception_status):
    ...

def E09(config):
    # Preserve partial accumulated gradients when the contract requires them.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Compare losses, required gradients, and parameter updates after returning to training. Verify behavior flags, gradient mode, and selected buffer values. Check completed post-evaluation live bytes rather than insisting reserved capacity returns to its old level. Include an evaluation exception to test restoration and cleanup. Report changed RNG consumption or stochastic trajectories as confounders until explicitly controlled. A harmless one-time cache warmup must not be mislabeled as repeated evaluation retention.

Part 7 - Engineering Interpretation and Philosophy

A phase boundary is a state-management contract as well as a memory boundary. Explain which objects must survive evaluation, which must be released, and which contexts must be

restored. The strongest integration reduces unnecessary residency while leaving the next training update correct. A lower isolated validation peak cannot compensate for silently discarding accumulated gradients or leaving parts of the model in the wrong behavior mode.

19.49.11 Exercise 10

Exercise Title: Compare Keeping GPU Predictions, Detached GPU Predictions, CPU Predictions, and Scalar Metrics Across an Entire Validation Epoch

Part 1 - Exercise Contract and Why It Matters

Compare four epoch-level result policies: tracked GPU predictions, detached GPU predictions, CPU predictions without retained history, and streaming scalar metrics. Deliver CUDA and host memory growth, transfer cost, and metric correctness. Specify which outputs the consumer actually requires. A scalar summary is not a drop-in replacement when later analysis genuinely needs every prediction, and moving values to CPU without detaching can still preserve graph history.

Part 2 - System and Memory Mental Model

Detached CUDA predictions remove their own history edge but retain payload and possibly a larger shared backing allocation. A CPU copy can carry a differentiable transfer edge when made in grad mode; export must intentionally remove unwanted history. Keeping all CPU predictions shifts epoch storage to host RAM rather than eliminating it. Streaming sufficient statistics can bound storage for suitable metrics, while nonlinear global metrics may require different algorithms or retained data.

Part 3 - Quantitative Prediction Before Execution

For fixed prediction shape and batch count, estimate raw output payload on each memory tier. Compare that with the unique backing capacity of stored views. Derive a streaming sum and count for an additive metric and identify why averaging batch means is incorrect for unequal batch sizes. Predict transfer volume, scalar synchronization cost, and the asymptotic storage of each policy under the same declared consumer requirement.

Part 4 - Experimental Design and Instrumentation

Run independent epoch replays from identical model state and batch order. Keep tracking mode explicit so tracked and detached policies differ intentionally. Export CPU predictions from detached values or graph-free execution and verify that no source graph owner is hidden in the export object. Use bounded device aggregation for scalar metrics and transfer summaries at deliberate boundaries. Record all conversion, synchronization, and metric-finalization work in the chosen timing scope.

Define which metrics admit exact streaming sufficient statistics. Counts, sums, and denominators must preserve masking and valid-item semantics. For global ranking metrics or later per-example analysis, keep the necessary information or use a separately validated algorithm; do not claim that a scalar average replaces every prediction dataset. If buffering CPU outputs, set a host budget and report its growth. Asynchronous copies require completion before CPU consumption or buffer reuse. Include a small slice that pins large CUDA storage to show why logical prediction size alone can underestimate the retained GPU footprint.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def export_predictions(predictions, retention_policy, consumer_contract):
    ...

def update_streaming_metric(state, predictions, targets, valid_count):
    ...

def E10(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Compare final metrics with an exact small retained-data reference and test an unequal final batch. Report host memory as well as CUDA memory and identify whether the whole prediction dataset remains stored. Verify source ownership cleanup and copy completion before CPU inspection. Do not equate a detached alias with an independent value snapshot. Mark unsupported streaming metrics as such rather than quietly replacing them with a biased batch-average approximation.

Part 7 - Engineering Interpretation and Philosophy

Output policy often matters as much as execution mode. Explain which representation is sufficient for the actual downstream question and which information each reduction discards. The controller can choose a bounded scalar path only when the metric contract permits it. Memory savings are meaningful when their information and synchronization costs are visible alongside the resulting capacity and runtime measurements.

19.49.12 Exercise 11

Exercise Title: Selectively Disable Gradient Tracking for a Frozen Submodule While Preserving Gradients Through the Trainable Remainder of the Model

Part 1 - Exercise Contract and Why It Matters

Disable unnecessary recording around a frozen submodule only when required derivatives do not need to pass through it. Compare a frozen feature extractor before a trainable head with a frozen middle block after a trainable upstream module. Deliver gradient checks for all required parameters and inputs, a mode-eligibility table, and memory measurements for valid policies. The middle- block counterexample must prevent a misleading frozen-means-no-grad rule.

Part 2 - System and Memory Mental Model

Freezing weights removes those parameter derivative targets; it does not remove the mathematical dependence of outputs on inputs. A frozen middle block may still need autograd recording to propagate gradients to earlier trainable layers or a differentiated input. A frozen prefix fed nondifferentiated data can often run graph-free and supply normal features to a

trainable head. Inference-produced features require additional compatibility analysis if the head needs to save them.

Part 3 - Quantitative Prediction Before Execution

Write the chain rule for a trainable upstream mapping, a frozen block, and a trainable downstream loss. Identify the derivative factor removed by wrapping the frozen block in no-grad. Predict which parameter gradients remain and which incorrectly vanish. Compare with the prefix case where upstream input gradients are explicitly not requested. Estimate saved-state reduction only for policies that satisfy their respective derivative contracts.

Part 4 - Experimental Design and Instrumentation

Construct both topologies with deterministic smooth operations and a scalar loss. Freeze selected parameters explicitly, clear stale gradients, and keep optimizer participation consistent with the intended update policy. Use no-grad only for eligible prefixes. Test required input gradients separately, since saliency or adversarial objectives can make the same prefix ineligible. Compare against an ordinary-grad frozen reference rather than against a fully trainable model with different derivative work.

Add a trainable adapter before a frozen linear block and a trainable head after it. Compare required adapter gradients with and without an invalid no-grad middle region. Then move the frozen block to an eligible prefix fed nondifferentiated data. Keep these as distinct topologies with separate reference gradients. Include an input-gradient requirement as another reason a frozen prefix may need recording. A module's lack of parameter updates does not establish that its input Jacobian is irrelevant. Record this counterexample in the policy table before optimizing any larger transfer-learning model.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def classify_frozen_region(region, upstream_targets, consumers):
    ...

def compare_frozen_topologies(config, derivative_contracts):
    ...

def E11(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate gradients for upstream and downstream trainable modules and any requested input derivative. A matching downstream gradient alone is insufficient. Reject the middle-block no-grad rewrite under the original contract. Confirm that frozen weights do not update and that stale optimizer state is handled explicitly. Report saved-state and peak differences without claiming that freezing removes all activation memory. Retain the counterexample as an eligibility test for later automation.

Part 7 - Engineering Interpretation and Philosophy

Frozen is a statement about parameter updates, while graph-free is a statement about computation recording. Their overlap depends on the complete derivative path. Explain how a policy can exploit the eligible prefix without breaking the middle block. This distinction is essential for transfer learning, adapters, and input-gradient applications, where local parameter flags do not describe all the derivatives the workload requires.

19.49.13 Exercise 12

Exercise Title: Freeze Parameters with requires_grad_(False) and Compare Its Memory Behavior Against Wrapping the Same Module in torch.no_grad()

Part 1 - Exercise Contract and Why It Matters

Compare parameter freezing with no-grad around the same module under explicitly different input-gradient requirements. Deliver a matrix showing when the two policies perform equivalent requested work and when they do not. Include parameter gradients, input gradients, saved state, optimizer ownership, and peak VRAM. Do not compare policies with silently different derivative targets and then call the lower-memory result an equivalent optimization.

Part 2 - System and Memory Mental Model

requires_grad_(False) on parameters excludes their derivative accumulation, but operations can still be recorded when another input requires gradients. No-grad suppresses recording for the scoped operations, including paths needed for input derivatives. Freezing does not automatically delete stale .grad buffers or optimizer moments. An optimizer holding stale gradients can still update a nominally frozen parameter under its own update rules, so update eligibility must be managed separately from graph recording.

Part 3 - Quantitative Prediction Before Execution

Predict cases with frozen parameters and ordinary data inputs, frozen parameters and differentiated inputs, and unfrozen parameters inside no-grad. Specify which derivative targets each case can satisfy. Estimate parameter-gradient storage avoided and separate it from retained optimizer state and activation saves. Predict why a frozen linear layer may still need its weight values to compute an input gradient even though no weight gradient is requested.

Part 4 - Experimental Design and Instrumentation

Use a small module with analytically or independently checked input and weight gradients. Freeze only leaf parameters, clear stale gradients to None where appropriate, and configure optimizer participation explicitly. Match behavior mode and output lifetimes. Sample before freezing, after gradient cleanup, after forward, and after the declared derivative task. Track optimizer state without assuming a flag change releases its tensors. Reject input-gradient queries after an ineligible no-grad rewrite.

Create stale-state controls deliberately and safely: run one ordinary update, then freeze the selected weights while recording existing gradient buffers and optimizer state. Configure the intended frozen update policy before resuming. Do not assume zero gradients are equivalent to None for momentum or weight-decay behavior. Releasing optimizer moments is a separate decision that may affect later unfreezing and must be labeled. Compare actual parameter values across several steps, not merely their requires_grad flags. Separate memory released by explicit state removal from memory never allocated because those derivatives were not requested.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def configure_frozen_parameters(module, optimizer, freeze_contract):
    ...

def compare_freeze_and_no_grad(config, derivative_targets):
    ...

def E12(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate all required derivatives and parameter updates. Test a parameter with existing gradient and optimizer state before freezing so cleanup behavior is observable. Record zero gradients and None gradients distinctly because optimizer handling can differ. Compare equivalent subsets separately from intentionally different tasks. A remaining optimizer allocation is not evidence that freezing failed, and a missing input gradient after no-grad is not a harmless performance trade-off under the original contract.

Part 7 - Engineering Interpretation and Philosophy

A parameter flag controls participation in differentiation, not the complete lifecycle of parameter-related state. Explain the savings attributable to reduced derivative work, explicit gradient cleanup, and optimizer-state policy separately. That accounting allows a controller to preserve frozen-layer input derivatives while avoiding unwanted parameter updates. A single module-level label cannot replace the required input, parameter, and optimizer contracts.

19.49.14 Exercise 13

Exercise Title: Build a Mixed Training-Inference Pipeline Where Only Selected Branches Construct Autograd Graphs

Part 1 - Exercise Contract and Why It Matters

Build a mixed pipeline with a trainable student branch, a nondifferentiated teacher or auxiliary branch, and shared inputs or features. Select execution modes per region while preserving the exact student objective. Deliver a boundary map, target and input derivative checks, memory evidence, and a case where an inference-produced teacher value needs conversion before the student loss can safely save it.

Part 2 - System and Memory Mental Model

Teacher parameters can be frozen while student gradients depend numerically on teacher outputs. Those outputs need values but usually not teacher history for ordinary distillation. A no-grad teacher result remains an ordinary tensor that downstream backward may save. An inference tensor can violate that saved-value requirement unless a normal independent copy

is prepared outside inference. Shared upstream features require careful contracts when both branches should contribute gradients.

Part 3 - Quantitative Prediction Before Execution

Write a small distillation objective and predict which teacher and student parameters receive gradients. Compare teacher targets created under no-grad with inference targets crossing an explicit normal-copy boundary. Estimate copy payload and lifetime overlap against saved teacher-state savings. Add a joint-training variant requiring teacher gradients and predict why the same graph-free policy becomes invalid despite identical forward target values.

Part 4 - Experimental Design and Instrumentation

Use deterministic teacher and student modules with fixed initial state and simple dense outputs. Set behavior flags separately for each branch. Freeze teacher parameters and control optimizer membership only for the declared nondifferentiated case. Keep the student and loss in required grad execution. Verify the tensor kind at each crossing and account for copy completion. Use fresh replays when comparing target policies, with identical loss scaling and batch ordering.

Use a teacher loss whose backward needs target values, so an inference-target boundary is exercised rather than accidentally bypassed. Check teacher target ownership after the student step, including caches and logging containers. A normal copy resolves tensor-kind compatibility but does not guarantee bounded retention. For shared features, state whether teacher, student, or both should contribute to upstream gradients and validate each requested component. If the teacher changes through an exponential moving average, keep that update outside gradient recording under its own contract and compare the same update schedule across memory policies.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def execute_branch_pipeline(batch, branches, contracts, boundary_policy):
    ...

def validate_branch_gradients(results, required_targets, reference):
    ...

def E13(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Compare student losses, required input gradients, and several parameter updates against a valid no-grad-teacher reference. Confirm teacher gradients are absent only when authorized. Record incompatible inference-target failures distinctly from numerical errors. Include shared-feature cases where detaching one branch changes upstream gradients. Report end-to-end memory and runtime including conversions, teacher work, and loss evaluation rather than only the fastest isolated branch.

Part 7 - Engineering Interpretation and Philosophy

A region can be locally inference-like yet produce values that participate in later differentiation. Explain how boundary contracts capture that distinction and how sharing complicates automatic selection. The policy should minimize unneeded history without changing the role of teacher targets or shared features. Every boundary needs both a derivative decision and a value-lifetime decision; neither can be inferred solely from the module's name.

19.49.15 Exercise 14

Exercise Title: Detect Unnecessary Gradient Tracking in Data Preprocessing, Teacher Models, Metrics, and Auxiliary Inference Modules

Part 1 - Exercise Contract and Why It Matters

Audit preprocessing, teacher models, metrics, and auxiliary modules for graph tracking that the declared workload does not require. Deliver a suspect list, justified eligibility rules, and validating interventions. Include legitimate exceptions such as differentiable augmentation, input optimization, and jointly trained auxiliary objectives. The word unnecessary requires an application contract; a module category alone cannot establish that its gradients are irrelevant.

Part 2 - System and Memory Mental Model

Graph recording can arise from trainable parameters, differentiated floating inputs, or captured intermediate values even when a region looks observational. Integer data paths and nondifferentiable operations have different behavior. A metric may intentionally contribute a regularizer, and a teacher may be jointly trained. Detaching only the final logged value can remove retention without avoiding upstream recording, whereas graph-free execution is valid only when no required path crosses the region.

Part 3 - Quantitative Prediction Before Execution

Predict required gradients for four explicit use cases and list the runtime observations that would suggest a mismatch. Separate the benefit of preventing a graph from the benefit of releasing a stored output. Estimate candidate saved-state savings and include a case where the audit finds tracking but correctly recommends keeping it. Define false positives as recommendations to remove required derivative work, not merely regions with large graphs.

Part 4 - Experimental Design and Instrumentation

Add named application scopes and collect mode state, input and output metadata, and covered save events in a diagnostic replay. Export no live tensors. Compare findings with declared derivative targets and consumer paths before ranking candidates. Apply one eligible intervention at a time in fresh matched runs. Validate gradients beyond the candidate region, including upstream inputs when requested. Report unsupported native or compiled scopes rather than asserting complete tracking coverage.

Separate three findings: recording happened, a graph-owning output escaped, and the recording was unnecessary under the declared objective. Any one can occur without the others. For example, detaching a final metric can stop history retention even when recording occurred earlier, while a differentiable regularizer legitimately needs the graph. Preserve application scope and invocation IDs, and include multiple calls to the same helper under different contracts. A repair recommendation needs a validating run ID; recognizing a suspicious function name or observing a requires_grad flag is not sufficient evidence of removable work.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def find_tracking_mismatches(region_rows, derivative_contracts):
    ...

def validate_audit_intervention(config, candidate, reference):
    ...

def E14(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Include an ordinary diagnostics region with unwanted history and a differentiable preprocessing region that must retain it. Score finding tracking separately from deciding it is unnecessary. Compare output values, metrics, and required gradients after each accepted change. Report unchanged saved-state or memory when an apparently promising candidate has persistent aliases. Check that an auditing hook does not create its own retained activation history or introduce a synchronization-heavy per-tensor reporting path.

Part 7 - Engineering Interpretation and Philosophy

An audit combines runtime facts with human-specified intent. Explain which part of each finding is observed and which depends on the derivative contract. The report should recommend a small testable change or ask for a missing contract in its output, rather than treating uncertainty as authorization to detach. A good negative finding preserves a legitimate gradient path and is as valuable as discovering removable tracking.

19.49.16 Exercise 15

Exercise Title: Build an Autograd-Tracking Audit That Reports Which Runtime Regions Create Graphs and Which Execute Without Gradient State

Part 1 - Exercise Contract and Why It Matters

Build a bounded runtime audit that distinguishes active mode state, observed graph creation, tensor provenance, and saved-state coverage by region. Deliver structured CPU metadata and a report with explicit unknowns. The audit must not claim that grad-enabled means every operation creates a graph, or that one graph-free output proves an entire region executed without autograd activity. Include nested contexts and execution in another thread.

Part 2 - System and Memory Mental Model

Grad and inference contexts are thread-local. Default grad mode permits recording but still needs differentiable operations with relevant requirements. A region can return an integer index while creating differentiable intermediates internally. A no-grad region can receive an already

tracked tensor without erasing its history. Hooks expose selected events and module boundaries, not a complete universal record of every native allocation, derivative transformation, or compiled operation.

Part 3 - Quantitative Prediction Before Execution

Predict audit rows for nested no-grad and enable_grad scopes, factory-created leaves, non-differentiable outputs, and pre-existing tracked inputs. Treat inference state separately: enable_grad alone is not an escape from active inference restrictions. Define which fields can be observed directly and which require event coverage. Estimate report memory from event count and metadata size, including an explicit truncation policy before scaling the workload.

Part 4 - Experimental Design and Instrumentation

Instrument named regions with exception-safe context handling and bounded records. Query actual mode state in the thread executing each region. Collect supported saved-tensor diagnostics in independent replays where composition is validated. Tag input provenance without preserving tensors in the report. Record nesting and invocation IDs so a module called under different policies is not collapsed into one label. Compare minimally instrumented execution with the audited replay to quantify overhead.

Keep region entry and exit records even when computation fails. A nested context may restore grad permission while inference state remains active; record both fields rather than deriving one from the other. Thread-local contexts do not automatically configure separate workers, so test mode state inside their execution functions. Register diagnostic hooks with bounded lifetimes and remove them after collection. If a hook returns or stores an activation, it can change behavior or ownership; require observational CPU metadata only. Treat compiled or custom opaque regions as separate support scopes instead of extending eager coverage claims automatically.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class RegionAudit:
    region_id: object
    mode_state: object
    tensor_metadata: tuple
    graph_observations: tuple
    coverage: object

def audit_region(region, inputs, recorder, scope):
    ...

def E15(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate the report against synthetic known scopes, including a graph created internally whose returned value has no grad_fn. Confirm mode restoration after an exception and independence of a deliberately configured worker thread. Use distinct statuses for observed no recording, no covered save, and unknown. Test truncation and missing hooks without replacing unknown values with zeros. Verify that disabling the audit removes its records and does not change the next iteration's live CUDA baseline.

Part 7 - Engineering Interpretation and Philosophy

The audit should describe evidence rather than pronounce every region safe for conversion. Explain the difference between permission to record, actual recorded paths, and the obligation to preserve them. Those distinctions make its output useful to the policy engine while protecting against simplistic rules based on output flags or module labels. Coverage and observer cost belong in the report, not in a hidden implementation assumption.

19.49.17 Exercise 16

Exercise Title: Compare Long-Sequence LLM Generation Under Normal Grad Mode, No-Grad Mode, and Inference Mode by KV Memory, Temporary Allocations, and Decode Throughput

Part 1 - Exercise Contract and Why It Matters

Compare controlled autoregressive decoding under default grad, no-grad, and inference execution while separating required KV payload from unnecessary history. Deliver per-token KV accounting, temporary-allocation evidence, decode throughput, and output checks. Use an explicit low-level decoding loop whose mode can actually change; a high-level generation entry point may already be decorated to disable tracking and would not establish a genuine default-grad baseline.

Part 2 - System and Memory Mental Model

KV storage remains necessary for the selected cached attention algorithm regardless of autograd mode. Default grad can additionally retain differentiable history through cache values when the implementation permits it. Integer sampling does not itself carry a differentiable token-selection path. Static, detached, or inference-only cache implementations can change which comparisons are supported. Attention workspace, logits histories, and model weights must not be mislabeled as KV payload or eliminated merely by changing execution mode.

Part 3 - Quantitative Prediction Before Execution

For uniform dense KV geometry, predict payload from active token counts:

$$M_{\mathrm{KV}} = 2\,L\,H_{\mathrm{KV}}\,D\,s\sum_i T_i.$$

State assumptions about dtype, layers, KV heads, head dimension, and sequence lengths. Add reserved capacity, padding, metadata, and temporary copies separately. Predict graph-related growth conditionally on connected cache history rather than assuming every grad-enabled cache has the same behavior. Explain why no-grad cannot remove the required KV term.

Part 4 - Experimental Design and Instrumentation

Use a tiny deterministic causal model first and a cache representation compatible with the intended comparison. Force identical token sequences for the primary replay so divergent sampling does not change shapes or work. Keep attention backend, context lengths, precision, batch policy, and output retention fixed. Measure prefill and decode separately and include per-token or end-to-end scopes explicitly. Inspect actual mode state inside the model rather than assuming an outer context overrides a decorated generation helper.

Avoid an in-place cache implementation that forbids autograd and then calling its failure evidence that default grad is slower. Use a compatible functional cache for the primary controlled comparison or mark the mode unsupported. Account for old and new cache coexistence when concatenation grows storage. Disable optional histories of logits or hidden states unless they are part of every compared contract. In grouped-query attention use KV head count, not query head count, in the payload formula. Record active sequences and actual token lengths, and avoid claiming sampled integer decisions preserve a differentiable generation path.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def decode_explicitly(model, token_trace, cache_policy, grad_policy):
    ...

def account_kv_payload(cache_metadata, model_geometry):
    ...

def E16(config):
    # Verify the generation entry point does not mask the intended mode.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Compare logits at declared tolerances and generated tokens in a separate sampling replay. Report KV logical bytes, unique backing capacity, saved history, temporary peak, and reserved memory independently. Mark unsupported cache-mode combinations rather than silently switching cache algorithms. Count throughput using a stated token numerator and completed elapsed time, including or excluding prefill explicitly. Do not attribute a result caused by retaining all logits to the cache alone.

Part 7 - Engineering Interpretation and Philosophy

Autograd control removes derivative work, while KV management preserves the state required by attention. Explain which dominates the measured long-context peak and whether mode changes improve capacity or mostly overhead. The lesson should connect to earlier paged and offloaded KV designs without claiming that graph suppression replaces them. A valid comparison preserves both the decoding algorithm and the actual execution-mode distinction.

19.49.18 Exercise 17

Exercise Title: Build a Memory-Aware Execution Policy That Automatically Chooses Gradient-Tracked, No-Grad, or Inference Execution for Different Runtime Phases

Part 1 - Exercise Contract and Why It Matters

Build an execution policy that selects grad-tracked, no-grad, or inference regions from declared derivative and consumer contracts. Use memory estimates only after establishing eligibility. Deliver policy decisions, rejection reasons, mode restoration tests, and independent validation of selected actions. The controller must be allowed to report an infeasible memory budget instead of silently disabling a gradient that the workload requires.

Part 2 - System and Memory Mental Model

A phase name is insufficient: validation can require input gradients, preprocessing can be differentiable, and teacher outputs may be saved by student backward. Mode selection must consider future consumers, tensor kind, forward AD, shared owners, and nested context. Inference is not a stronger universally safe substitute for no-grad. New mode contexts do not erase pre-existing graphs or control output-retention policies automatically.

Part 3 - Quantitative Prediction Before Execution

Define a decision table with hard derivative constraints, tensor-boundary compatibility, and measured costs. Predict outcomes for training, ordinary evaluation, saliency evaluation, teacher targets, and terminal reporting. Estimate complete boundary costs including conversions and retained payload. Add a case where no eligible policy meets the byte budget. Preserve that infeasibility rather than ranking an invalid graph-free choice as the best memory option.

Part 4 - Experimental Design and Instrumentation

Start with an explicit phase registry and small supported dense eager regions. Validate contracts against the audit evidence from E15. Implement exception- safe context entry and restoration, recording actual mode state in the executing thread. Keep model.train or eval behavior changes as separately declared state transitions. Cache policy decisions only while their shapes, consumer requirements, and derivative targets remain valid. Use fresh replays to validate any newly selected policy.

Validate nested-policy precedence and invalidate decisions after changes to shapes, derivative targets, or consumers. Record actual mode and tensor kind. Restore state after exceptions. Report infeasibility with raw estimates when no eligible mode fits; silently changing batch size, precision, or history length changes the workload contract.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class ModeDecision:
    mode: object
    eligible: bool
    prerequisites: tuple
    rejection_reasons: tuple
    predicted_cost: object
```

```
def choose_execution_mode(phase_contract, tensor_provenance, costs):
    ...

def E17(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Compare decisions with a manually reviewed eligibility matrix and validate all required derivatives after execution. Include nested contexts and an exception path; check restoration and later training updates. Reject inference tensor crossings lacking a valid conversion or consumer proof. Report prediction error separately from correctness. A successful low-memory result with a missing required input gradient is a failed policy, even if its loss values match the reference.

Part 7 - Engineering Interpretation and Philosophy

The controller should automate choices whose semantic prerequisites are known, not infer the intended mathematics from current VRAM pressure. Explain which decisions are proved eligible, which depend on measured cost, and which remain unsupported. When no valid mode satisfies the budget, the report can identify other memory techniques for a future experiment while preserving the original derivative contract and declaring the current request infeasible.

19.49.19 Exercise 18

Exercise Title: Stress-Test Gradient-Control Policies with Training, Validation, Frozen Modules, Teacher-Student Inference, Cached Outputs, and Shared Tensor Storage

Part 1 - Exercise Contract and Why It Matters

Stress-test mode control across training, evaluation, frozen prefixes and middle blocks, teacher-student losses, cached outputs, and shared storage. Deliver a coverage matrix with numerical checks, memory results, expected rejections, and overhead. Include adversarial cases that expose policies based only on module names or output flags. A stress test should explore interactions under bounded resources rather than require an uncontrolled physical OOM.

Part 2 - System and Memory Mental Model

Interactions can invalidate isolated assumptions. A graph-free producer can feed a later backward that needs its values; a detached cache can still pin large storage; a frozen middle block can be necessary for upstream derivatives. Context state is local to a thread, while module behavior flags and Python owners are different shared state. Restoration, tensor provenance, and derivative requirements must therefore be checked together.

Part 3 - Quantitative Prediction Before Execution

Predict expected classifications before execution: valid graph suppression, required tracking, incompatible inference boundary, bounded payload retention, or unsupported observation. Construct pairs with similar memory but different derivative obligations and pairs with matching forward values but different gradients. Estimate admission bounds for each case from peak overlap and host output storage. Define scoring so correct rejection of an unsafe optimization counts as success rather than lack of aggressiveness.

Part 4 - Experimental Design and Instrumentation

Build deterministic small cases first, then vary shape, sequence length, output-history length, and selected context nesting. Add a no-grad factory exception, a view that cannot detach in place, and a reused inference value requiring saved data. Include a separate worker-thread case. Match initial state and batch traces across replays, bound event logging and runtime, and preserve unsupported feature combinations explicitly. Do not silently repair an invalid candidate before recording its rejection.

Include changed input-gradient requirements, teacher values saved by backward, and detached aliases modified through another owner. Test exception restoration and worker contexts separately before combining cases. Keep unsupported AD features explicit. Growing graph-free prediction histories must still trigger output-budget diagnostics, while correctly rejected unsafe conversions count as successful policy decisions.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def build_mode_stress_cases(config, support_matrix):
    ...

def evaluate_policy_case(case, controller, oracle, budgets):
    ...

def E18(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Score derivative correctness, boundary compatibility, mode restoration, memory prediction, and overhead separately. Check parameter and requested input gradients, not only scalar loss. Keep legitimate cache warmup and graph-free payload growth as distinct controls. Report false safe recommendations prominently. A missing CUDA device or unsupported differentiation feature produces an explicit status rather than a fabricated pass. Preserve minimal counterexamples and cases where predicted savings do not appear in the observed whole-step peak.

Part 7 - Engineering Interpretation and Philosophy

The suite establishes the supported boundary of automation. Explain which rules survived interaction and which need a narrower contract or additional measurement. A trustworthy controller can refuse a mode change while still explaining the memory bottleneck. The goal is reliable removal of unnecessary autograd state, not the largest percentage reduction obtained by changing the mathematical workload or discarding required results.

19.49.20 Exercise 19

Exercise Title: Build and Benchmark a PyTorch Gradient-Tracking Memory Controller That Minimizes Autograd State While Preserving Exactly the Gradients Required by the Workload

Part 1 - Exercise Contract and Why It Matters

Integrate a PyTorch gradient-tracking memory controller that selects supported execution modes and output policies while preserving the workload's exact derivative targets. Deliver an auditable report, eligibility decisions, validating replays, and benchmark comparisons. Every saving must be attributed to graph suppression, history removal, output ownership, or another explicit state change. Do not bundle several interventions into an unexplained before-and-after memory number.

Part 2 - System and Memory Mental Model

The controller combines region contracts, tensor provenance, observed graph activity, memory costs, and consumer compatibility. Grad mode is required where derivatives must cross a region; no-grad can produce normal values for later tracked use; inference is eligible only under validated restrictions. Detached aliases still share payload, and frozen parameters may leave input derivatives necessary. Module behavior, optimizer updates, and output-retention budgets remain independent parts of the execution contract.

Part 3 - Quantitative Prediction Before Execution

Predict complete phase peaks and latency using unique backing capacity and measured boundary costs. Include persistent training state, validation outputs, conversion overlap, and required caches. Rank only eligible policies and preserve unknown estimates. Define success as correct required gradients and metrics within the admitted resource budget, with measured overhead. Include an infeasible case to demonstrate that the controller never satisfies a memory target by silently narrowing derivative requirements.

Part 4 - Experimental Design and Instrumentation

Integrate the audit and policy schemas with bounded collection and exception- safe phase handling. Use known cases to develop rules and held-out cases to evaluate them. Compare always-grad, manually selected valid modes, and the controller, keeping output contracts explicit. Measure collection, decision, conversion, and validation costs as well as steady execution. Limit retries, capture frequency, and host report size. Export CPU metadata and release diagnostic owners after their declared use.

Report phase IDs, derivative targets, behavior flags, actual modes, tensor provenance, output contracts, memory counters, timing boundaries, and validation results. Separate predictions from observations and interpretation. Include analysis and conversion overhead, rejected actions, unsupported features, and infeasible budgets. Claim only the best validated policy within the tested range, preserving unresolved consumer requirements.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class ControllerReport:
    decisions: tuple
    derivative_checks: tuple
```

```
    memory_measurements: tuple
    timing_measurements: tuple
    limitations: tuple

def execute_controlled_phase(phase, contract, controller, evidence):
    ...

def validate_controller(config, policy, reference):
    ...

def E19(config):
    # Refuse mode changes that remove required derivatives.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require a traceable chain from each accepted action to its contract, observed state, prerequisites, prediction, and independent validation. Compare losses, all required gradients, parameter updates, metrics, memory, and runtime. Preserve unsupported, rejected, and not-measured outcomes. Evaluate false safe choices separately from missed savings. Report whether measured gains survive complete consumer boundaries and subsequent training, rather than only an isolated forward whose outputs were never used.

Part 7 - Engineering Interpretation and Philosophy

The final controller should make intended differentiation explicit and remove only work that intention does not require. Explain remaining uncertainty about opaque scopes, hidden owners, dynamic consumers, and unsupported AD features. The strongest result is a small validated policy set with known counterexamples, bounded overhead, and honest infeasibility reporting. Preserving the required mathematics is the condition for calling a lower-memory execution an optimization at all.

19.50 DMM48

Chapter Title: Custom Memory Manager in PyTorch
Specialization: Dynamic Memory Management and CUDA Systems

19.50.1 Exercise 00

Exercise Title: Design the Architecture of a Custom PyTorch Memory Manager with Budgets, Residency States, Policies, Events, and Runtime Decisions

Part 1 - Exercise Contract and Why It Matters

Design an application-level memory manager with explicit managed handles, budgets, ownership, residency, transfer events, and policy decisions. Deliver component contracts, invariants, a supported-workload boundary, and a synthetic execution trace. The manager coordinates registered values through controlled access; it is not automatically a replacement for the PyTorch CUDA allocator and cannot transparently migrate every arbitrary tensor, operator workspace, or external CUDA allocation in the process.

Part 2 - System and Memory Mental Model

Separate logical tensors from physical storage replicas and allocation generations. Several views can share one backing allocation, while one logical value can have CPU and GPU replicas simultaneously. Residency, validity, transfer progress, and in-use leases are related but distinct state dimensions. A raw tensor reference held outside the manager can defeat release or observe stale data; controlled access and mutation contracts are therefore architectural requirements, not optional bookkeeping refinements.

Part 3 - Quantitative Prediction Before Execution

Predict resource occupancy for a value promoted from CPU to GPU, used, demoted, and released. Include simultaneous source and destination capacity, staging, and pending reservations. Define which budgets bound managed backing storage and which include an external safety allowance. Construct an example where the registry's logical byte sum understates retained storage and another where adding logical aliases overstates it. Explain why neither sum equals driver-level device use.

Part 4 - Experimental Design and Instrumentation

Start with deterministic dense inference values and immutable registered storage, then introduce controlled mutations and training-aware adapters later. Specify registry, accounting, admission, transition engine, transfer scheduler, policy plugins, and telemetry responsibilities. Define failure outcomes before integration. Use an event-driven simulator with deterministic completion order as the primary invariant oracle, followed by bounded CUDA replays when available. Every decision should name its prerequisites and the evidence that permits commit or rejection.

Define the supported operation boundary before choosing policies. Initial experiments should use manager-owned immutable dense values accessed through leases, with no hidden writes or raw references escaping the scope. Training, compiled graphs, capture, distributed collectives, and external allocators need separate adapters and support declarations. A Python registry cannot rewrite every existing reference when a tensor migrates. State whether access returns a short-lived tensor borrow or a managed operation, and reject consumers that cannot honor that contract. Preserve an explicit derivative and precision contract so later residency choices cannot silently become graph cuts or approximate arithmetic.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class ManagerContract:
    managed_scope: object
    derivative_requirements: object
    budgets: object
    external_allowance: object

def design_manager_components(contract):
```

```
    ...

def E00(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require counterexamples to the claims that each tensor owns one allocation, that zero manager references establish safe reuse, and that admitted requests are guaranteed to succeed in CUDA. Record hardware, versions, allocator backend, seeds, shapes, precision, and initial-state identifiers. Keep predictions, observations, and interpretations separate. An unavailable CUDA device produces not-measured hardware fields, not fabricated benchmark results. Reject unsupported raw aliases or mutation paths explicitly rather than promising transparent management that the access interface cannot enforce.

Part 7 - Engineering Interpretation and Philosophy

The design succeeds when every byte and transition has a clear scope and every unsafe decision can be refused. Explain which uncertainty belongs to unmanaged work, allocator behavior, or future execution. The manager should preserve required values and derivatives while optimizing eligible residency. Reducing its internal counters is not a success if a caller still holds the old CUDA storage or receives a value from an obsolete generation.

19.50.2 Exercise 01

Exercise Title: Build a Global Tensor Registry That Tracks Tensor Identity, Shape, Dtype, Bytes, Device, Owner, Lifetime, and Current Residency

Part 1 - Exercise Contract and Why It Matters

Build a registry for all values within the declared managed domain. Track logical identity, tensor metadata, backing identity, owner leases, generation, and replica state. Deliver registration, lookup, controlled mutation, and retirement contracts. Global means global to this manager instance or explicitly coordinated process scope, not automatic discovery of every tensor in Python, other processes, native libraries, or opaque autograd state.

Part 2 - System and Memory Mental Model

Python object identity, tensor data pointer, backing storage identity, and logical handle identity answer different questions. Views can have different data pointers because of offsets while sharing storage. Numeric addresses can be reused across lifetimes. Strong registry references deliberately keep storage alive; weak wrapper references alone cannot prove device-read completion. A manager-owned handle should resolve the current valid replica rather than allow callers to cache an untracked raw tensor indefinitely.

Part 3 - Quantitative Prediction Before Execution

Predict records for a base tensor, two overlapping views, an independent clone, and a retired handle whose address is later reused. Count logical objects and distinct backing allocations separately. Estimate host registry size as records grow and specify a retirement policy. Include a mutation that invalidates a CPU replica while retaining GPU authority, identifying which generation and validity fields must change before another consumer can read.

Part 4 - Experimental Design and Instrumentation

Begin with supported contiguous dense tensors and explicitly register view relationships when extending scope. Preserve shape, stride, dtype, offset, device, capacity status, and provenance as CPU metadata. Define whether callers borrow raw tensors only inside leases. Add duplicate registration and stale- handle controls. Keep registry retirement separate from optional historical telemetry so diagnostic records do not retain live tensors or resurrect retired ownership. Mark unobservable owners as unsupported or unknown.

Include base storage capacity even when a registered view spans only a small region. Do not merge unrelated allocations because their addresses look adjacent. Use allocation generations and logical handle versions to distinguish retirement, reuse, and mutation. Keep descriptive owner metadata separate from strong owning references, labeling both when the registry intentionally owns values. A weak reference becoming dead describes a wrapper, not the final GPU reader. Test idempotent registration and retirement, and make stale handle lookup fail clearly. If a raw alias escapes, record that loss of coverage and prevent unsupported release guarantees.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class TensorHandle:
    logical_id: object
    generation: int

@dataclass
class RegistryEntry:
    metadata: object
    storage_links: tuple
    replica_records: tuple
    active_leases: tuple

def register_managed_value(registry, tensor, ownership_contract):
    ...

def E01(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate that aliases share a backing record without collapsing distinct logical metadata. Reject stale generations and handle reuse. Test that an external borrowed tensor cannot silently outlive its lease under the supported contract; otherwise narrow the release guarantee. Compare registry-enabled and clean runs for self-retention. Do not use sys.getrefcount as a complete native ownership oracle. Export only CPU descriptions of owners, not containers holding the tensors being investigated.

Part 7 - Engineering Interpretation and Philosophy

Identity design determines whether later accounting and safety claims are meaningful. Explain why a stable logical handle can survive migration while a physical pointer cannot serve as permanent identity. A registry is useful when its coverage and ownership rules are enforceable. Honest rejection of untracked aliases is preferable to an apparently comprehensive registry whose release decisions ignore real consumers.

19.50.3 Exercise 02

Exercise Title: Implement Exact Tensor Memory Accounting and Compare the Manager's Internal Bookkeeping Against PyTorch CUDA Memory Measurements

Part 1 - Exercise Contract and Why It Matters

Implement exact accounting within a supported registered-storage domain, then reconcile it with broader PyTorch CUDA counters. Deliver logical payload, unique backing capacity, per-tier replica charges, pending commitments, and a coverage-aware residual. Exact must describe an explicit accounting boundary; it cannot mean perfectly predicting total process VRAM, allocator rounding, workspace, context allocations, or memory owned by other libraries from registered tensor shapes alone.

Part 2 - System and Memory Mental Model

Logical dense payload is element count times element size, but views, offsets, capacity, and nonstandard layouts complicate backing occupancy. Multiple logical values can reference one storage, while replicas on separate tiers consume separate capacity. memory_allocated and memory_reserved describe different allocator scopes. An allocator snapshot has its own visibility limits. Registry capacity cannot simply be added to memory_allocated when those bytes are already represented in that counter.

Part 3 - Quantitative Prediction Before Execution

For supported dense tensors calculate logical payload and unique backing capacity separately:

$$B_{\text{logical}} = \sum_i n_i s_i, \qquad B_{\text{backing}} = \sum_{j \in \mathcal{U}} c_j.$$

Define the unique set by tier and allocation generation. Add a large base with small views and a CPU-GPU replica pair. Predict differences from allocator live bytes and reservation, identifying which residual terms are expected rather than labeling all disagreement an accounting defect.

Part 4 - Experimental Design and Instrumentation

Validate arithmetic using synthetic catalogs before runtime measurements. Include overlapping views, independent clones, zero-element tensors, and capacity larger than current logical shape. Restrict unsupported sparse, quantized, or external layouts instead of forcing a dense formula. Sample at deliberate completed boundaries with matched timestamps. Record persistent unmanaged state and backend-specific unsupported fields. Compare deltas in a controlled arena rather than demanding absolute equality with all device memory.

Keep pinned host bytes as a subset of total managed host occupancy when that is the chosen RAM definition, rather than adding overlapping categories into an invented total. File size, virtual mappings, resident pages, and page cache also need separate labels. For a tensor serialized as a small view, verify the actual stored backing extent instead of assuming logical

payload determines file size. Record timestamp and completion state for runtime comparisons. A zero-element tensor may have nontrivial backing through an alias; validate its supported storage relationship before assigning zero capacity. Include unsupported layouts in the coverage table.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def account_registered_storage(registry, accounting_scope):
    ...

def reconcile_with_cuda(ledger, cuda_sample, coverage):
    ...

def E02(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require exact synthetic totals and explain every supported-domain mismatch. Use explicit unknown capacity instead of zero. Keep requested payload, backing capacity, allocator block size, and reserved segment capacity in separate columns. A positive residual can reflect unmanaged tensors or opaque allocations; a negative residual can reflect timestamp skew or double counting. Investigate before assigning a cause. Report host metadata and file-backed capacity independently from resident host pages.

Part 7 - Engineering Interpretation and Philosophy

Accounting is reliable when it states what it measures and why quantities differ across layers. Explain which ledger is suitable for admission and which counters provide external context. A scoped exact result can coexist with uncertain whole-process use. The manager should reconcile these boundaries without inventing a universal conversion from tensor payload to guaranteed available CUDA capacity.

19.50.4 Exercise 03

Exercise Title: Define VRAM, RAM, and Optional NVMe Memory Budgets and Reject Unsafe Allocations Before They Reach the CUDA OOM Boundary

Part 1 - Exercise Contract and Why It Matters

Define independent VRAM, pageable RAM, pinned RAM, and optional NVMe capacity budgets with admission rules. Reject requests that exceed the manager's known safe commitments before issuing their allocations. Deliver a budget ledger, safety margins, and accepted, rejected, or uncertain decisions. The title's preventive goal is scoped: external allocations and allocator eligibility can still cause CUDA OOM after a request passes application-level checks.

Part 2 - System and Memory Mental Model

A migration can require destination capacity, retained source, temporary conversion workspace, and staging simultaneously. Driver free memory is context, not an atomic reservation for this process. Cached allocator space and application logical free capacity are different resources. NVMe capacity is storage space rather than a device accepted by Tensor.to; file serialization, metadata, integrity checks, and host staging form an explicit optional path.

Part 3 - Quantitative Prediction Before Execution

Define a conservative per-tier admission condition:

$$C_r + P_r + \Delta_r + H_r \leq B_r.$$

Name committed capacity, not-yet-materialized reservations, incremental request cost, and headroom separately. Ensure already materialized bytes are not counted in both committed and pending terms. Predict a request that fits its destination payload but fails because staging and overlap exceed another tier's budget. Keep undefined estimates explicitly unresolved.

Part 4 - Experimental Design and Instrumentation

Use a controlled inference arena and synthetic near-limit requests before bounded hardware tests. Estimate worst supported overlap, reserve all affected tiers atomically, and release reservations on rejected preparation. Validate file capacity and host staging before optional NVMe operations. Keep a separate allowance for unmanaged workspace and measurement uncertainty. Do not claim a public API reports the largest CUDA allocation that would certainly succeed at the next instant.

Give every budget a resource name, unit, version, and enforcement scope. If total RAM includes pinned staging, enforce the total and pinned subset constraints without double counting their occupancy. Distinguish known managed bytes from a conservative unmanaged allowance. If broader measured use already includes managed bytes, do not add the same registry capacity again. Budget rejection should return required incremental resources and the limiting tier. For optional NVMe, reserve file and metadata space, validate path ownership, and clean incomplete artifacts after failure. Do not interpret a successful filesystem allocation as proof that future I/O latency is acceptable.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class BudgetDecision:
    admitted: bool
    tier_charges: object
    uncertainty: tuple
    reasons: tuple

def admit_operation(operation, ledger, budgets, allowances):
    ...

def E03(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Test exact-fit, over-budget, unknown-size, and concurrent-request cases. Report manager admission rejection separately from an actual CUDA allocation failure. Check that a rejected request leaves no leaked reservation or partial destination. A successful allocation does not validate all future workload headroom. Record budget values in bytes, current samples, timestamps, and safety assumptions. If optional NVMe support is absent, mark it unsupported rather than replacing it with an unbounded host copy.

Part 7 - Engineering Interpretation and Philosophy

Budgets are enforceable promises about controlled resources, not guarantees about an entire shared machine. Explain which safety assumptions make a request admissible and which external risks remain. The admission system should fail clearly when information is insufficient. Its value is preventing known overcommit and making residual failures diagnosable, while preserving an honest distinction between policy approval and allocator success.

19.50.5 Exercise 04

Exercise Title: Build Explicit Tensor Residency States for GPU-Resident, CPU-Resident, NVMe-Resident, Prefetching, Evicting, and In-Use Memory

Part 1 - Exercise Contract and Why It Matters

Define explicit residency records covering GPU, CPU, optional NVMe, prefetching, evicting, and in-use values without forcing incompatible facts into one enum. Deliver a state model, legal combinations, replica authority rules, and transition examples. In-use should normally be an orthogonal lease or activity state: a GPU-resident tensor can be in use, while CPU and GPU replicas can both exist during a transfer.

Part 2 - System and Memory Mental Model

Location, validity, authority, transfer progress, and active consumers are separate dimensions. A destination allocation may exist before its contents are ready. A completed CPU replica can become stale after a GPU write. Eviction can mean dropping a redundant clean GPU copy or copying dirty authoritative values first. Optional file residency needs a committed versioned record, not merely a pathname or an incomplete write.

Part 3 - Quantitative Prediction Before Execution

Predict the full record after promotion starts, after copy completion, after a GPU mutation, and after demotion commits. Count every occupied tier during overlap. Construct a stale CPU replica case and identify why its existence does not permit immediate GPU release. Predict which fields remain valid if a transfer fails before completion and which source copy must preserve the latest value.

Part 4 - Experimental Design and Instrumentation

Write a transition table with preconditions, acquired reservations, submitted events, completion rules, and failure cleanup. Begin with immutable inference values, then add a versioned writer lease. Use a simulator to exercise delayed completion and simultaneous replicas. Restrict unsupported external mutation. If implementing NVMe, keep temporary files distinct from committed records and require validated shape, dtype, length, and value integrity before making a file version authoritative.

Model replica versions independently from allocation generations: an unchanged allocation can hold newly written values, while migration can create another allocation holding the same value version. A reader must request the expected version and wait for its readiness. Test a clean redundant replica, a stale replica, and the sole authoritative copy. For optional files, define whether commit promises process-local recoverability or durable persistence and implement the corresponding protocol; do not imply durability from a host write call alone. Unknown transfer completion keeps a replica pending, even when its memory has already been charged to the ledger.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class ReplicaState:
    tier: str
    value_version: int
    ready_event: object
    valid: bool
    capacity_bytes: object

@dataclass
class ValueState:
    replicas: tuple
    authoritative_version: int
    reader_leases: tuple
    writer_lease: object
    pending_transitions: tuple

def E04(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Reject contradictory states such as no valid recoverable value with a live handle, two conflicting authoritative write versions, or a ready flag set before completion. Preserve transfer errors explicitly. Check that a clean replica drop and a dirty writeback have different costs and safety conditions. Never read an incomplete CPU destination simply because the host copy call returned. Validate the state record against actual values in small completed replays.

Part 7 - Engineering Interpretation and Philosophy

A state model is useful when it makes unsafe assumptions difficult to express. Explain why orthogonal activity and residency fields represent asynchronous work better than a single GPU-or-CPU label. The model should support a conservative answer when transfer progress or mutation provenance is unknown. Future policy plugins may choose actions, but only the transition engine can authorize state changes that preserve the latest required value.

19.50.6 Exercise 05

Exercise Title: Implement Safe Residency State Transitions and Detect Illegal Transitions Such as Evicting a Tensor Still Used by CUDA Work

Part 1 - Exercise Contract and Why It Matters

Implement a transactional transition engine that validates state, schedules safe work, and commits only after required completion. Detect requests that would discard a value still needed by CUDA work or publish an incomplete replica. Deliver illegal-transition cases, failure cleanup, and an auditable state log. A policy suggestion must not directly mutate residency records or bypass the engine's ownership and completion checks.

Part 2 - System and Memory Mental Model

An asynchronous copy submission is not a completed state transition. The source must remain valid through its last reader, and the destination must remain reserved and unavailable to incompatible users until ready. Existing read leases, writer leases, aliases, and stream events constrain release. A clean read-only copy may overlap other reads when dependencies permit it; an unsafe source release or conflicting write remains forbidden.

Part 3 - Quantitative Prediction Before Execution

For a promotion and a dirty demotion, predict prepare, submit, complete, commit, and abort stages. Identify the point at which destination bytes are materialized and when reservations change category. Construct a late-completion case where an old operation attempts to commit after the logical value has advanced to another generation. Predict rejection without corrupting the new record or releasing its valid storage.

Part 4 - Experimental Design and Instrumentation

Use operation IDs and expected generations for every transition. Reserve resources before submission, preserve the source, and make commit idempotent. Inject failures before allocation, after destination creation, and before publication. Simulate event completion deterministically, then validate supported CUDA dependencies in bounded replays. Keep state locks short and avoid waiting for device completion while holding a lock needed by another completion handler.

Write a failure table for each stage with retained source, destination charge, reservation ownership, published metadata, and retry eligibility. Submission can leave real work in flight even after a caller cancels. Keep an operation record until cleanup is safe, and make late completion unable to mutate a newer generation. A synchronous state lock does not serialize CUDA execution; explicit device dependencies remain necessary. Test two writers competing for a value and a reader arriving during eviction. Reject or safely wait according to the declared access contract rather than returning an unready destination or silently dropping a required version.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def prepare_transition(registry, request, ledger):
    ...
```

```
def commit_transition(registry, operation, completion_evidence):
    ...

def abort_transition(registry, operation, failure):
    ...

def E05(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Check that every failed operation preserves a valid source or reports an unrecoverable value explicitly. Test double completion, stale operation IDs, and cancellation after submission. Releasing a reservation cannot free a buffer still used by an in-flight copy. Compare values and generations after all admitted transitions. Export CPU-only event descriptions and distinguish submission time from completion time. No illegal transition should silently become a forced device-wide synchronization fallback.

Part 7 - Engineering Interpretation and Philosophy

Safety belongs in one enforceable mechanism. Explain how the engine separates a policy's preference from the proof that a transition is legal. Transactional metadata cannot undo arbitrary external writes, so the supported access contract must remain explicit. The strongest implementation refuses stale or unsafe operations, preserves authoritative data through failures, and makes both successful and rejected transitions reviewable.

19.50.7 Exercise 06

Exercise Title: Add Reference Counting and Last-Use Tracking So the Manager Knows When Tensor Storage Can Be Released, Reused, or Evicted

Part 1 - Exercise Contract and Why It Matters

Track manager leases and completed last uses so storage can be released, reused, or evicted only when all required obligations permit it. Deliver reader and writer accounting, generation-safe retirement, and shared-storage controls. Reference counting applies to the manager's controlled ownership model; it is not a complete count of all Python wrappers, autograd saves, native holders, or outstanding GPU readers in an arbitrary PyTorch program.

Part 2 - System and Memory Mental Model

A zero manager-reader count can coexist with pending CUDA work submitted under an earlier lease. Several logical handles can share one allocation, so backing release requires resolving all relevant owners. Python garbage collection and device execution proceed on different timelines. record_stream helps the allocator handle cross-stream lifetime, but it does not decide application data dependencies or authorize overwriting a value still needed by another consumer.

Part 3 - Quantitative Prediction Before Execution

Predict lease and pending-event counts for two handles sharing storage and two streams reading at different times. Identify the earliest legal wrapper retirement, backing release, and

overwrite boundaries separately. Include a planned future reuse and explain why it can favor keeping data without necessarily making a zero-current-reader value impossible to evict. Distinguish a policy prediction about future use from an active correctness obligation.

Part 4 - Experimental Design and Instrumentation

Use scoped acquisition and release tokens rather than raw integer increments scattered throughout application code. Associate each submitted use with its completion evidence and generation. Test double release, forgotten release, and an external alias that violates the supported borrowing contract. For training extensions, integrate supported saved-tensor or checkpoint adapters; do not assume ordinary autograd references are automatically represented by manager leases.

Separate a borrowing lease from a future scheduling hint. A current reader can prevent overwrite or release, while a future reader can often tolerate eviction followed by reload. For shared backing, track obligations at both logical-value and storage levels. Include an autograd-saved value only through a supported adapter that preserves its semantics; never infer its native ownership from Python reference counts. On shutdown, stop new borrows, settle or safely retain pending operations, then retire storage. A garbage-collection pass is not a substitute for completing the relevant stream dependencies and resolving managed leases.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def acquire_value(registry, handle, access_mode, execution_scope):
    ...

def release_lease(registry, token, completion_event):
    ...

def retire_backing_if_safe(registry, storage_id, evidence):
    ...

def E06(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate shared backing is never released after only one alias retires. Include a CPU wrapper disappearing while a CUDA reader remains pending, and a completed-reader case whose caching allocator retains reservation. Report unknown external ownership rather than treating it as zero. Detect lease imbalance with bounded diagnostics that do not retain raw tensors. Verify values after reuse to expose premature overwrites that might otherwise appear only as intermittent numerical corruption.

Part 7 - Engineering Interpretation and Philosophy

Correct release is the intersection of ownership and execution safety. Explain which counts are exact within the manager and which obligations require external adapters or conservative

refusal. A future-use estimate helps optimize residency, while a current reader determines legality. Keeping those concepts separate allows policy flexibility without turning a mistaken reuse prediction into a correctness failure.

19.50.8 Exercise 07

Exercise Title: Build High- and Low-Watermark Memory Pressure States That Change Runtime Behavior Before VRAM Becomes Critically Full

Part 1 - Exercise Contract and Why It Matters

Build pressure states using distinct high and low watermarks, pending commitments, and bounded reaction rules. Deliver transition traces showing when admission, prefetch, and eviction behavior changes. The policy should respond before known commitments exceed its capacity, while avoiding repeated promotion and demotion around a single noisy threshold. A smaller configured budget must not authorize unsafe release of values currently required by computation.

Part 2 - System and Memory Mental Model

Allocated, reserved, managed backing, and driver free bytes describe different pressure views. The manager's admission pressure should use its consistent committed-plus-pending ledger with allowances, while external counters provide context. Hysteresis reduces oscillation by requiring a lower threshold before returning to relaxed behavior. A budget decrease can leave existing obligations above the new target; that is a draining or infeasible state, not proof that immediate eviction is legal.

Part 3 - Quantitative Prediction Before Execution

Choose low and high thresholds with units and a clear denominator. Predict state changes for a trace hovering near the high threshold, a transfer burst, and a sudden budget reduction. Include pending destinations so pressure is visible before allocation completion. Estimate how much eligible capacity must be released to return below the low threshold, and preserve the possibility that pinned or in-use state prevents that target.

Part 4 - Experimental Design and Instrumentation

Validate the policy on deterministic byte traces before CUDA integration. Define cooldown, sampling cadence, and separate emergency admission rejection. Change one pressure input at a time to understand behavior. When entering pressure, reduce optional prefetch and select only eligible eviction candidates. On a budget decrease, stop incompatible new commitments and drain safely. Record state transition reasons, effective budget versions, and actions rejected because the required storage cannot yet be released.

Use a budget version in every pressure decision so a delayed action cannot apply thresholds from an obsolete capacity target. Test a falling budget below unavoidable persistent residency and report the deficit without deleting live state. Optional prefetch may be canceled before submission, but in-flight copies retain their cleanup obligations. Distinguish pressure relief promised by an eviction proposal from capacity actually released after completion. Settle the ledger only at the corresponding lifecycle boundary. Report the number of state flips and migration cost when comparing hysteresis settings, not just whether the final counter is below a threshold.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def update_pressure_state(previous, ledger, budgets, thresholds):
    ...

def plan_pressure_response(state, eligible_values, pending_operations):
    ...

def E07(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Check threshold equality conventions and ensure hysteresis does not mask an actual overcommit. Test zero or changing budgets and invalid threshold ordering. Measure oscillation count, time under pressure, rejected admissions, and stalled work. Do not force an apparent recovery by dropping required values. Compare managed pressure with broader CUDA samples without treating a high reserved cache as identical to live data demand. Report a failure to reach the low target honestly.

Part 7 - Engineering Interpretation and Philosophy

Pressure states translate accounting into predictable behavior. Explain which actions are optional performance changes and which are mandatory admission constraints. Hysteresis is useful only when its scope and response costs are visible. The manager should preserve correctness while acknowledging that some workloads cannot fit a newly reduced budget without changing the workload or waiting for existing obligations to end.

19.50.9 Exercise 08

Exercise Title: Implement Pluggable FIFO, LRU, Size-Aware, and Cost-Aware Eviction Policies Behind a Common Memory-Manager Interface

Part 1 - Exercise Contract and Why It Matters

Implement FIFO, LRU, size-aware, and cost-aware candidate ranking behind one policy interface. Deliver deterministic decisions, tie-breaking rules, and comparisons on identical access traces. The interface must rank only actions already permitted by ownership, replica validity, and completion rules. A policy cannot make an in-use authoritative value evictable merely by assigning it a favorable score.

Part 2 - System and Memory Mental Model

FIFO uses an explicitly defined admission age; LRU uses a defined application access event. Size-aware ranking can favor large recoverable capacity but must account for aliases that prevent actual release. Cost-aware ranking balances expected reuse, reload, writeback, and stalls. These are imperfect future-use estimates, not correctness facts. Full-attention KV blocks can all be reused each step; low attention weight alone does not make their lossless deletion valid.

Part 3 - Quantitative Prediction Before Execution

Predict each policy's choice for a shared backing allocation, a dirty large value, and a small value needed soon. Compute expected recoverable unique GPU capacity rather than logical tensor size. State the units of the cost score and how ties are broken. Construct a trace where LRU is worse than FIFO and another where largest-first causes expensive immediate reload, showing that no ranking is universally best.

Part 4 - Experimental Design and Instrumentation

Use an immutable candidate snapshot from the transition engine and return ranked proposals with reasons, never direct mutations. Revalidate eligibility at commit because state can change after ranking. Replay the same deterministic trace under each policy with matching budgets and initial state. Measure writebacks, reloads, hit rate, stalls, and end-to-end runtime. Include a no- eviction baseline when capacity permits and preserve cases where no eligible victim exists.

Define LRU's access event as an application use or completed demand, rather than a telemetry inspection that accidentally makes every value recent. Choose whether FIFO age resets on reload and document it. Size-aware scores must use unique capacity recoverable after all aliases and obligations are considered. For dirty values, include host destination admission and writeback cost before ranking them as victims. Compare policy decisions on a frozen candidate snapshot, then revalidate the selected version. Preserve shared-prefix or shared-parameter cases where removing one handle cannot release the backing allocation, even when that handle has the largest logical size.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class EvictionProposal:
    handle: object
    expected_generation: int
    recoverable_bytes: object
    score: object
    reasons: tuple

def rank_eviction_candidates(candidates, policy, cost_model):
    ...

def E08(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Check that all policies preserve identical required values and derivatives in their supported workload. Test stale proposals, shared replicas, and dirty writeback costs. A hit-rate improvement without reduced stall or runtime is a valid mixed result. Distinguish policy miss

from incorrect eligibility. Report counterexamples and unsupported cost estimates rather than claiming global optimality. A deterministic tie rule should make identical synthetic traces reproducible.

Part 7 - Engineering Interpretation and Philosophy

The common interface makes policy differences reviewable without weakening safety. Explain which ranking suits the tested access pattern and which assumptions made it effective. A robust manager can change heuristics while keeping the same transition invariants. The best tested policy is a measured choice within a workload range, not a replacement for ownership and completion proofs.

19.50.10 Exercise 09

Exercise Title: Implement a Prefetch Planner That Selects Future Tensors from Execution Order, Reuse Distance, Transfer Cost, and Available VRAM

Part 1 - Exercise Contract and Why It Matters

Build a prefetch planner that predicts future needs, admits destination capacity, and schedules transfers before demand. Deliver deadlines, confidence, reservations, and observed useful or wasted prefetch outcomes. The planner must work within the same budget as foreground allocations rather than treating prefetched bytes as free. A future execution estimate can be wrong, so cancellation and fallback demand loading need explicit contracts.

Part 2 - System and Memory Mental Model

Prefetch advances a necessary transfer in time and increases early residency. Its benefit depends on available overlap, source readiness, link contention, and consumer dependencies. A request selected from predicted execution order is not guaranteed to execute on a conditional path. A ready destination still needs generation and validity checks at use. Prefetch cannot conceal transfer time that exceeds the available independent work, nor create extra physical capacity.

Part 3 - Quantitative Prediction Before Execution

Estimate a transfer deadline from expected consumer time, measured copy cost, and a safety margin. Predict when limited capacity forces a shorter prefetch horizon even if distant data is certain to be used. Compare useful bytes ready before demand, late bytes, and unused speculative bytes. Construct a case where prefetch improves transfer overlap but evicts a more valuable resident value and increases total runtime.

Part 4 - Experimental Design and Instrumentation

Begin with a fixed execution trace and known sizes, then add conditional branches with recorded predictions. Reserve destination and staging capacity before scheduling. Track operation IDs, value versions, predicted consumer, and completion evidence. Limit outstanding requests and prioritize mandatory demand over optional speculation under pressure. Replay with identical access traces and compare against demand-only loading, including planning and cancellation costs in the benchmark.

Record whether each future need is known from a fixed trace or inferred from a conditional path. Use measured size-dependent transfer costs rather than one constant bandwidth for every request. Demand loading should not wait behind unlimited speculative work; bound

queue depth and define priorities. If a prefetched version becomes stale after a write, invalidate it before counting a hit. Cancellation can prevent publication without undoing an already submitted copy. Measure speculative traffic and occupancy as well as demand stalls, and include a case where disabling prefetch is the best validated choice under the tested budget.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def plan_prefetch(trace_hint, registry, ledger, bandwidth_model):
    ...

def reconcile_prefetch_demand(operation, consumer, actual_state):
    ...

def E09(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate that a prefetch hit refers to the correct value version ready before its actual demand. Separate late completion, unused speculation, and canceled work already in flight. Do not release buffers merely because the consumer prediction disappeared. Report host and CUDA occupancy peaks, useful overlap, link traffic, and stalls. A prediction failure should degrade performance through a safe fallback, not expose incomplete or stale tensor data.

Part 7 - Engineering Interpretation and Philosophy

Prefetch is a trade between earlier residency and less waiting. Explain which horizon is justified by measured costs and which uncertainty limits it. The planner should remain optional: refusing speculative work under pressure can improve stability. Its strongest result is reduced demand stalls without budget violations or a hidden increase in migration traffic that overwhelms the benefit.

19.50.11 Exercise 10

Exercise Title: Add Pinned CPU Staging Buffers and Non-Blocking CUDA Transfers for Asynchronous Tensor Promotion and Demotion

Part 1 - Exercise Contract and Why It Matters

Add reusable pinned CPU staging buffers and non-blocking transfers for promotion and demotion. Deliver separate host and CUDA budgets, buffer leases, readiness events, and transfer correctness tests. Pinned memory is a constrained host resource, not free bandwidth. Non-blocking submission does not mean the destination is immediately readable or that the source can be overwritten before the copy completes.

Part 2 - System and Memory Mental Model

Pageable and pinned host paths have different staging and synchronization behavior. A GPU-to-CPU destination must not be consumed before completion; a pinned CPU source must remain unchanged through an asynchronous host-to- device read. Reusing a staging slot requires its transfer and consumers to finish. Calling pin_memory can itself allocate and copy, so repeatedly pinning inside the critical path is different from using a bounded reusable pool.

Part 3 - Quantitative Prediction Before Execution

Predict simultaneous occupancy for a two-slot transfer pipeline, including pinned slots, source values, GPU destinations, and temporary conversion if present. Estimate transfer cost from measured directional bandwidth and launch overhead. Compare one large transfer with many small transfers of the same payload. Identify when additional slots cannot improve overlap because the link or compute path is already saturated.

Part 4 - Experimental Design and Instrumentation

Start with same-dtype dense contiguous copies and fixed slot capacities. Reserve each slot and destination before submission, retain source leases, and record completion on the actual transfer stream. Verify CPU values only after the required event boundary. Add variable-size requests and reject or safely grow insufficient slots through the admission system. Keep transfer-only measurements separate from the complete producer-transfer-consumer pipeline and report both.

Treat a pinned buffer's capacity, valid region, transfer direction, and lease generation as explicit fields. Prevent a late completion from freeing a slot already assigned to another generation. Keep source and destination alive until their actual readers finish, even when library allocator mechanisms also help track lifetime. Repeatedly allocating pinned tensors can change host behavior and must not be hidden in setup for only one candidate. Compare pageable and reusable pinned paths at the same payload and timing scope. Report any host-side pinning, packing, dtype conversion, or noncontiguous materialization included in the complete transfer path.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def acquire_staging_slot(pool, request_bytes, reservation):
    ...

def submit_residency_copy(source, destination, slot, stream, operation):
    ...

def release_staging_after_completion(pool, slot, completion):
    ...

def E10(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate round-trip values, slot reuse, source lifetime, and destination readiness. Use deterministic delayed-completion simulation for unsafe-reuse rejection rather than relying on a nondeterministic corruption race. Measure pinned host footprint, CUDA peak, transfer volume, and latency. Do not infer complete pipeline overlap from a short host submission time. Preserve unsupported async paths and quantify pinning or conversion costs that the chosen timing scope includes.

Part 7 - Engineering Interpretation and Philosophy

Staging is a resource with a lifetime, not a transfer flag. Explain how pool capacity and leases make overlap safe and predictable. The best pipeline balances host residency, destination admission, and actual useful concurrency. A lower host call duration is not sufficient evidence of higher throughput if the consumer still waits for the same copy or a prematurely reused slot corrupts its data.

19.50.12 Exercise 11

Exercise Title: Coordinate Prefetch, Compute, and Eviction Through CUDA Streams and Events Without Using Global Device Synchronization

Part 1 - Exercise Contract and Why It Matters

Coordinate prefetch, compute, and eviction through explicit stream dependencies and events, avoiding device-wide synchronization in the steady-state pipeline. Deliver a causal execution graph, safe lifetimes, and measured overlap. Bounded validation can use deliberate completed boundaries, but routine operation should not serialize all streams after every request. Removing global waits must preserve producer readiness, last-reader completion, and allocator safety.

Part 2 - System and Memory Mental Model

CUDA streams order their own work; cross-stream consumers need explicit relationships. Event waits establish device-side readiness, while record_stream or an equivalent correctly implemented lifetime protocol informs safe reuse. These solve different problems: allocator lifetime tracking alone does not make a consumer wait for the producer. Host callback order is not proof of GPU completion. A cycle in dependency waits can deadlock an otherwise valid allocation plan.

Part 3 - Quantitative Prediction Before Execution

Draw dependencies for one prefetched value, one compute consumer, and one writeback. Predict which operations may overlap and which must wait. Calculate a lower bound on pipeline latency from the critical path rather than summing all event durations. Add a second reader stream and identify the extra completion obligation before release. Predict how a misplaced early wait eliminates potential overlap without changing value correctness.

Part 4 - Experimental Design and Instrumentation

Use dedicated streams with recorded operation IDs and value generations. Record readiness after production or copy, then make each consuming stream wait before use. Preserve source and destination ownership through all readers. Use supported allocator lifetime tracking or a fully explicit creation-stream handoff protocol. Validate dependency acyclicity in simulation and instrument actual streams in separate timing replays. Keep steady-state global synchronization out of the implementation contract.

Choose fresh or safely reusable event generations for each dependency. Recording a new event state while another operation still interprets the old one can invalidate the logical dependency model. Include a two-reader case where one stream finishes late and a canceled consumer whose already submitted work still requires completion handling. Distinguish CPU waiting on an event from a device stream waiting on an event. Use the least broad wait that satisfies the contract, but validate correctness before optimizing overlap. A record_stream call informs allocator reuse and cannot replace explicit readiness waits between producer and consumer streams.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def connect_stream_dependencies(operation, producer, consumers, events):
    ...

def finalize_cross_stream_lifetime(storage, reader_events, owner_stream):
    ...

def E11(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Compare numerical results with a deliberately serial reference. Test multiple readers, delayed copies, event reuse, and a rejected cyclic dependency graph. Report overlap from completed event intervals with clock and device scope, not host launch proximity. Distinguish readiness waits from allocator lifetime handling. Check that telemetry does not insert hidden synchronization into every operation. Unsupported cross-device timing must remain explicitly unresolved rather than mixed into one absolute timeline.

Part 7 - Engineering Interpretation and Philosophy

Concurrency is safe when dependencies are explicit enough to justify every read and release. Explain which waits are essential and which only reduce parallelism. The manager should expose its causal graph so performance changes can be audited without weakening lifetimes. A fast submission loop is meaningful only when the completed pipeline remains correct and its true critical path has improved.

19.50.13 Exercise 12

Exercise Title: Build a Memory Reservation System That Prevents Multiple Pending Operations from Overcommitting the Same Future VRAM

Part 1 - Exercise Contract and Why It Matters

Build atomic memory reservations so concurrent pending operations cannot each spend the same apparent free capacity. Deliver reservation tokens, materialization and commit rules, cancellation, and a race-controlled simulator test. The reservation system protects the manager's

logical budget; it does not reserve physical VRAM against other processes or guarantee that the CUDA allocator will satisfy a later request.

Part 2 - System and Memory Mental Model

A free-memory sample followed by an allocation is not an atomic admission protocol. Pending destinations, staging, and workspace allowances must be charged before submission. Once destination memory is materialized, its charge must move consistently from unmaterialized pending capacity to committed capacity. Counting it in both categories overstates pressure; releasing it from both during a transition creates an overcommit window. Source capacity cannot be reclaimed before its own safe release.

Part 3 - Quantitative Prediction Before Execution

Predict the result when two requests individually fit but jointly exceed a budget. Model interleavings of check, reserve, allocate, complete, and release. Compute a conservation invariant for each tier and test an allocation failure between reservation and commit. Include a cancellation after CUDA submission: the reservation's ownership must follow the actual pending work rather than vanish because a caller no longer wants its result.

Part 4 - Experimental Design and Instrumentation

Use one coordinated ledger or a clearly defined transactional lock discipline for all affected tiers. Assign unique tokens, operation generations, requested amounts, and lifecycle states. Avoid blocking transfers while holding admission locks. Test deterministic concurrent interleavings and idempotent cleanup before real asynchronous execution. Reconcile actual materialized capacity with the estimate, rejecting unsupported underestimated growth or acquiring additional capacity before proceeding where possible.

Define one conservation equation for every reservation lifecycle transition. A materialized destination moves from pending to committed without a gap or double charge. Aborted preparation returns only unmaterialized commitments; live partial allocations remain charged until safe release. Reserve multiple tiers in one transaction or use ordered locking with complete rollback. Never hold a host lock while waiting for a completion path that needs that lock. Test two requests using the same initial free-capacity observation, and require only the admissible combination to receive tokens. Record actual capacity adjustments if allocator-visible backing differs from the initial supported estimate.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class ReservationToken:
    operation_id: object
    generation: int
    tier_amounts: object
    lifecycle: str

def reserve_atomically(ledger, request, budgets):
```

```
    ...

def settle_reservation(ledger, token, materialized, completion):
    ...

def E12(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Assert that accepted commitments never exceed the declared ledger limit under tested interleavings. Test double settlement, expired tokens, stale generations, and partial multi-tier preparation. Report admission rejection separately from an allocator OOM after valid reservation. Verify cancellation does not release live transfer buffers. Use exact synthetic invariants and bounded hardware measurements; neither a successful single-thread run nor a momentarily high free-memory sample proves concurrency safety.

Part 7 - Engineering Interpretation and Philosophy

Reservations turn future occupancy into explicit present commitments. Explain which race they prevent and which external allocations remain beyond their control. The ledger should expose why capacity is unavailable even before all destinations appear in CUDA counters. This makes prefetch, eviction, and recovery share one budget instead of competing through independent optimistic estimates.

19.50.14 Exercise 13

Exercise Title: Detect and Prevent Thrashing by Monitoring Tensor Migration Frequency, PCIe Traffic, Reuse Distance, and GPU Stall Time

Part 1 - Exercise Contract and Why It Matters

Detect repeated migrations that consume bandwidth and stall work without providing useful residency. Deliver migration cohorts, a thrashing predicate, and validated mitigation policies. Distinguish necessary streaming of a working set larger than capacity from avoidable oscillation caused by poor prefetch or eviction decisions. High transfer volume alone is not a proof that the manager is malfunctioning.

Part 2 - System and Memory Mental Model

Thrashing depends on reuse, cost, and available alternatives. A value evicted just before reuse can require immediate reload; speculative prefetch can cause that eviction. Cooldowns, minimum residency, reduced prefetch horizons, and admission throttling can help, but also constrain useful eviction. A full-attention working set may require every KV block repeatedly, so a hot/cold classification cannot invent lossless unused data or bypass a fundamentally insufficient capacity budget.

Part 3 - Quantitative Prediction Before Execution

Define windowed migration bytes, repeated round trips per generation, demand stall fraction, and useful computation between migrations. Compare two traces with the same transfer

volume but different stall and reuse behavior. Predict a case where a cooldown improves runtime and another where it blocks a necessary eviction. State thresholds, observation windows, and the evidence needed before calling a pattern avoidable thrashing.

Part 4 - Experimental Design and Instrumentation

Replay alternating working sets, predictable layer streaming, and speculative wrong-path prefetch under identical budgets. Collect per-value migrations and causal waits as CPU metadata. Apply one mitigation at a time and compare completed runtime, useful prefetch, rejected admissions, and memory peaks. Use measured transfer cost under contention rather than assuming an isolated bandwidth result remains constant. Keep correctness eligibility unchanged when adjusting the heuristic.

Use rolling windows tied to workload progress and preserve tensor generations in round-trip counts. A repeatedly reused logical ID may refer to different values, so migration frequency alone can misidentify a healthy replacement stream. Compare useful computation completed per transferred byte and direct consumer wait time. Include contention from simultaneous promotions and demotions when calibrating costs. Mitigation may reduce speculation, delay admission, or change victim ranking, but cannot delete required data or bypass leases. If the working set fundamentally exceeds budget, report necessary streaming and its measured cost instead of claiming a heuristic eliminated the capacity constraint.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def identify_thrashing(telemetry_window, workload_contract, thresholds):
    ...

def propose_thrashing_mitigation(findings, eligible_controls):
    ...

def E13(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate that alerts distinguish predictable necessary streaming from avoidable back-and-forth motion on controlled traces. Report ambiguous cases and false positives. Check that mitigation does not violate deadlines, budgets, or value requirements. Compare the number of migrations with transferred bytes and stalls; fewer copies can still mean more waiting. Preserve a case with no effective mitigation under the current budget rather than claiming every working set can be made resident through better policy.

Part 7 - Engineering Interpretation and Philosophy

Thrashing diagnostics should explain a cost pattern and test an alternative, not blame transfers in isolation. Explain whether the root limitation is capacity, link throughput, speculative scheduling, or victim choice. A useful manager can recognize when policy tuning is exhausted and report that the workload needs another capacity or computation strategy, while maintaining correct values throughout.

19.50.15 Exercise 14

Exercise Title: Integrate Activation Policies So the Manager Can Choose Keep, CPU Offload, or Recomputation Under Changing Memory Pressure

Part 1 - Exercise Contract and Why It Matters

Integrate activation keep, CPU offload, and recomputation as eligible alternatives under pressure. Deliver derivative-aware region contracts, cost estimates, and matched training benchmarks. Use supported autograd integration rather than pretending the registry can replace arbitrary tensors already saved inside backward nodes. The manager must preserve the requested model function, gradients, and update sequence for policies labeled equivalent.

Part 2 - System and Memory Mental Model

Keeping preserves residency; offloading preserves values in another tier; recomputation preserves a recipe and its required inputs or state. A valid recipe needs reproducible execution and cannot ignore mutable buffers, randomness, or parameter versions. Saved-tensor hooks and supported checkpointing provide specific integration points with their own contracts. Hook nesting and selective checkpoint behavior are version-sensitive, so logging and offload do not automatically compose into complete coverage.

Part 3 - Quantitative Prediction Before Execution

Predict unique activation capacity avoided at the actual peak, retained checkpoint inputs, host offload bytes, restoration overlap, and recomputation time. Include a shared backing allocation that remains resident through another owner and therefore offers no immediate capacity relief. Compare costs under a fixed host and CUDA budget. State when no eligible intervention can reduce the peak enough without altering required derivatives.

Part 4 - Experimental Design and Instrumentation

Begin with deterministic pure regions and first-order gradients. Use explicit non-reentrant checkpoint configuration and supported save_on_cpu or validated saved-tensor adapters. Do not mutate saved inputs or replace parameter storage through .data. Collect diagnostic graphs separately when hook composition is not proven. Compare fresh identical initial states, data, precision, optimizer, and full-step timing. Validate higher-order or stochastic extensions separately before claiming support.

Preserve checkpoint inputs, parameter versions, and any RNG or mutable state needed to replay a region correctly. A recipe that reads changed weights during backward is not equivalent to the original forward. Keep transport buffers within host and device reservations, including restoration overlap. Validate hook composition in the installed version or collect diagnostics in separate passes. If compression is added to saved values, move that case to a declared approximate derivative contract rather than treating it as lossless offload. Report both prediction error and measured full-step savings, including cases where recomputation creates a different temporary peak.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
from torch.utils.checkpoint import checkpoint
```

```
def classify_activation_options(region, derivative_contract, state):
    ...

def schedule_activation_policy(region, choice, manager, reservations):
    ...

def E14(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Check losses, required gradients, and several optimizer updates against normal backpropagation. Measure host peak, CUDA peak, runtime, transfer volume, and instrumentation overhead. Confirm a requested offload actually occurred under the active hook contexts. A shifted backward peak is part of the result, not an excuse to report only forward savings. Mark unsupported composition or stateful recomputation explicitly and reject any hidden detach that removes required history.

Part 7 - Engineering Interpretation and Philosophy

Activation policies exchange residency for transport or repeated computation. Explain which obligation each choice preserves and which prerequisites permit it. The manager should use measured cost after correctness eligibility, with budgets that include transient overlap. A policy that only reduces an internal saved-byte estimate without reducing the measured whole-step peak still needs an honest, limited interpretation.

19.50.16 Exercise 15

Exercise Title: Integrate Precision Policies So Selected Tensors Can Be Stored at Reduced Precision When Capacity Is More Important Than Accuracy

Part 1 - Exercise Contract and Why It Matters

Add optional reduced-precision storage policies for explicitly eligible tensors. Deliver precision contracts, representation metadata, error budgets, conversion costs, and end-to-end quality checks. Unlike lossless residency changes, lossy storage generally changes values and potentially gradients. It must be opt-in under a declared accuracy contract, not silently presented as exact equivalence when memory pressure rises.

Part 2 - System and Memory Mental Model

Storage dtype, compute dtype, accumulation dtype, and quantization metadata are different choices. A cast can allocate a destination while the original remains alive. Quantization requires scales, grouping, packing, and restoration rules; nominal bit width alone does not specify resident capacity. Autocast selects operation execution dtypes and does not automatically compress every existing stored tensor. Shared aliases and parameter or optimizer ownership can make arbitrary storage replacement unsafe.

Part 3 - Quantitative Prediction Before Execution

Predict compressed payload plus metadata, padding, staging, and conversion overlap. For a fixed tensor compare floating casts and a declared quantized representation, preserving its

scale overhead. Estimate reconstruction and downstream output errors on a bounded reference set. Include an outlier case that violates the accuracy budget and must be rejected even when compression would satisfy the byte target.

Part 4 - Experimental Design and Instrumentation

Start with immutable inference values and independent versioned representations. Require an explicit eligible-tensor list and accuracy tolerance. Validate restoration and consumers before retiring the original version. Add training saved-state compression only as a separately validated approximate policy with gradient and update checks. Do not alter live parameter or alias storage in place. Reserve both representations and workspace during conversion, and account for rollback or rejected calibration results.

Define whether a precision change is reversible from a retained master or only approximately reconstructible. Retaining an original master consumes capacity and must appear in every claimed saving. Include scale metadata, packing tails, alignment, and host copies when measuring effective compression. Reject mutable shared aliases unless all consumers observe an authorized versioned representation. Never use .data replacement as a shortcut around autograd or optimizer ownership. Test extreme values and a task-sensitive tensor that exceeds its tolerance. A representation accepted for inference must not automatically become eligible for parameter updates or higher-order derivatives.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class PrecisionContract:
    allowed_representations: tuple
    value_error_budget: object
    task_quality_budget: object
    derivative_tolerance: object

def plan_precision_change(handle, contract, calibration, ledger):
    ...

def E15(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report effective resident bytes, metadata, conversion latency, value error, task metrics, and required derivative error where applicable. A valid numerical tolerance must be chosen before selecting favorable examples. Preserve precision policy rejections and degradation failures. Check NaNs, infinities, extreme magnitudes, and repeated conversion effects. A lower measured footprint is not sufficient if consumers read a stale version or quality exceeds the declared loss budget.

Part 7 - Engineering Interpretation and Philosophy

Precision is a resource and accuracy trade, while lossless migration changes location. Explain that distinction in reports and policy decisions. The manager should preserve an exact mode and refuse approximate fallback unless authorized by the tensor's contract. A useful compression option states its representation, measured capacity, quality limits, and conversion costs rather than offering a universal promise based on nominal bits.

19.50.17 Exercise 16

Exercise Title: Build Automatic OOM Recovery That Rolls Back an Unsafe Operation, Changes Memory Policy, and Retries Without Restarting the Process

Part 1 - Exercise Contract and Why It Matters

Implement bounded recovery for explicitly retryable allocation failures within a declared transactional region. Deliver prepare, abort, cleanup, policy change, and retry records, plus cases that must fail without retry. The title does not authorize universal rollback of arbitrary PyTorch execution. A manager can safely recover only when required values, mutable state, randomness, and external side effects are preserved or restored under a supported contract.

Part 2 - System and Memory Mental Model

An OOM can occur after temporary allocations or state changes. Python exception tracebacks can retain failed-frame tensors, so cleanup and retry must not keep those frames alive. empty_cache releases unused allocator cache rather than live values or partially modified application state. Illegal memory access, device assertions, or corrupted execution context are not ordinary recoverable allocation failures. Partial optimizer updates and distributed collectives require stronger coordination than local retry.

Part 3 - Quantitative Prediction Before Execution

Predict state before a pure retryable operation, after an injected allocation failure, and after an admitted lower-memory retry. Account for retry workspace and retained rollback state. Compare an unchanged exact-policy retry with an optional precision change that has a separate accuracy contract. Set maximum attempts and define a decreasing resource requirement or other progress criterion so repeated failure cannot become an unbounded loop.

Part 4 - Experimental Design and Instrumentation

Begin with deterministic inference regions that have no external side effects. Catch the version-supported CUDA OOM type specifically, record scalar failure metadata, exit the exception scope, and release failed temporaries before retry. Use deterministic fault injection as the primary rollback oracle, then a bounded real allocation-failure test if appropriate. Snapshot only state required by the declared region and include its storage cost. Reject unsafe mutation or optimizer-step boundaries.

Use fault injection at known stages to validate cleanup without requiring an unpredictable real OOM. Keep injected failure labels distinct from hardware allocation failures. Catch only the supported allocation exception and preserve other failures for normal error handling. Move retry outside the exception scope and avoid storing exception tracebacks in telemetry, since they can retain failed-frame tensors. Include a pure region that retries successfully and a stateful or partially updated operation that is rejected. Optional batch-size or precision changes require an explicitly different workload contract; do not hide them as an exact retry of unchanged work.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def classify_retry_safety(operation, side_effect_contract, failure):
    ...

def abort_failed_attempt(transaction, manager, failure_metadata):
    ...

def retry_with_admitted_policy(operation, transaction, retry_budget):
    ...

def E16(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Compare the successful result with a failure-free oracle, including RNG, mutable buffers, required gradients, and update count when supported. Verify all failed reservations and live temporaries are settled without prematurely reusing pending buffers. Report injected failure, admission rejection, actual OOM, and fatal CUDA error distinctly. In distributed extensions, refuse uncoordinated rank-local retries around collectives. Preserve nonrecoverable outcomes honestly; the same-process goal applies only to supported recoverable cases.

Part 7 - Engineering Interpretation and Philosophy

Recovery is a transaction contract, not an exception handler that blindly repeats work. Explain what was restored, why retry is safe, and which state lies outside the supported boundary. A good manager avoids known overcommit, recovers bounded eligible failures, and stops clearly when correctness or progress cannot be established. It should never trade an apparently completed request for duplicated updates or silently changed values.

19.50.18 Exercise 17

Exercise Title: Add Runtime Telemetry for Allocated Memory, Reserved Memory, Residency, Transfers, Evictions, Prefetch Hits, Stalls, and Policy Decisions

Part 1 - Exercise Contract and Why It Matters

Add bounded telemetry for manager commitments, allocator use, residency, transfers, policy proposals, prefetch outcomes, stalls, and failures. Deliver a schema with units, clocks, coverage, and operation identifiers. The observer must not retain live tensors or synchronize every event, changing the memory and concurrency behavior it reports. Collection and analysis overhead are part of the benchmark, not invisible implementation costs.

Part 2 - System and Memory Mental Model

Manager backing capacity, pending reservations, allocator allocated bytes, allocator reserved bytes, and driver-level usage have different scopes. Submission and completion timestamps

differ, and multiple streams form a causal partial order. Transfer bytes can include repeated migrations of the same logical data. A prefetch hit must specify whether the correct version was ready before demand, while a policy proposal is distinct from a committed state transition.

Part 3 - Quantitative Prediction Before Execution

Predict telemetry for one useful prefetch, one late prefetch, one canceled in-flight copy, and one failed eviction proposal. Define transfer bandwidth using stated directional payload and completed time. Calculate stall fractions with a declared denominator and avoid adding overlapping intervals twice. Estimate host log size and dropped-record behavior under an event budget. State how incomplete telemetry limits conclusions without changing raw measurements into invented zeros.

Part 4 - Experimental Design and Instrumentation

Use CPU scalars, IDs, generations, state versions, and event references with bounded retention. Sample broad CUDA counters at deliberate boundaries and record timestamp skew. Instrument detailed event timing in diagnostic replays, then compare minimally instrumented execution. Preserve workload IDs, seeds, hardware, software, allocator backend, precision, shapes, budgets, and initial state. Separate cold setup, warmed operation, calibration, and recovery timing with explicit inclusion rules.

Give every measurement a unit, scope, timestamp domain, and sampling coverage. Distinguish payload bytes from estimated physical link traffic and avoid labeling an effective copy rate as raw hardware bandwidth. For overlapping work, define whether utilization and stall fractions use a union of intervals or per-consumer totals. Keep dropped-record counts and truncation markers. Provide raw resource counters beside derived pressure values so aggregation can be audited. A report row can reference an event ID without retaining the CUDA tensor. Confirm that optional detailed tracing can be disabled without changing the manager's correctness decisions or persistent ownership.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class TelemetryRow:
    operation_id: object
    generation: object
    event_kind: str
    timestamp_domain: str
    measurements: object
    coverage: object

def record_manager_event(recorder, event, scalar_metadata):
    ...

def E17(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate schemas on deterministic event traces and audit every aggregate's scope. Reject negative durations caused by mixed clocks rather than silently clipping them. Test dropped records, reused addresses, and repeated operation completion. Compare output values and memory baselines with telemetry disabled. Report overhead, capture coverage, and unsupported fields. A discrepancy between registry and device use is a diagnostic observation, not automatic evidence of a leak, fragmentation, or accounting corruption.

Part 7 - Engineering Interpretation and Philosophy

Telemetry should make decisions explainable without creating a false sense of completeness. Explain which counters describe controlled resources and which provide broader context. The report should connect a proposal, its eligibility, actual transition, and observed effect while preserving failed predictions. This gives an engineer enough evidence to improve policy rather than merely display a smooth memory chart.

19.50.19 Exercise 18

Exercise Title: Stress-Test the Memory Manager with Variable Tensor Sizes, Long Transformer Sequences, Concurrent CUDA Streams, Changing VRAM Budgets, and Deliberate Memory Pressure

Part 1 - Exercise Contract and Why It Matters

Stress-test the manager across variable sizes, long-sequence workloads, concurrent streams, changing budgets, and deliberate bounded pressure. Deliver an invariant matrix, correctness outcomes, policy behavior, and resource and runtime measurements. Include simulated faults and unsupported cases. Pressure should be created through admitted budget limits and controlled workloads, not by relying on a dangerous race or exhausting the entire host to obtain a dramatic failure.

Part 2 - System and Memory Mental Model

Interactions can expose bugs hidden by individual features: prefetch reservations compete with demand, demotion needs pinned capacity, writes invalidate replicas, and delayed readers postpone release. Changing shapes alter capacities without implying leaks. Long Transformer sequences can exceed both saved-activation and KV budgets under different workload contracts. A smaller runtime budget may require safe draining or rejection rather than immediate success through more aggressive eviction.

Part 3 - Quantitative Prediction Before Execution

Predict outcomes before execution: admitted completion, safe waiting, admission rejection, retryable failure, unsupported operation, or irreducible budget infeasibility. Construct traces with equal logical bytes but different alias capacity and traces with equal transfer volume but different stalls. Estimate worst supported overlap and event-log size. Define failure as an invariant or correctness violation, keeping legitimate rejection separate from manager malfunction.

Part 4 - Experimental Design and Instrumentation

Build a deterministic simulator suite first, including late completion, stale generations, cancellation, dirty replicas, partial allocation failure, and duplicate callbacks. Add small real CUDA cases one feature at a time, then selected interactions. Use fixed traces and initial state,

deadlines, resource admission limits, and bounded capture. Test exact and approximate precision contracts separately. Preserve every case's supported feature matrix and actual stopping reason.

Include budget reduction below active commitments, no eligible victim, wrong-path prefetch, exhausted pinned staging, stale replica reads, and duplicated completion notifications. Add variable shapes with a healthy eventual plateau and an intentional registry-owner leak as distinct cases. Optional NVMe tests need short writes, corrupted records, and capacity rejection in a controlled private area. Stop before unbounded host growth and retain actual stopping reasons. Evaluate policies on traces not used to tune thresholds. Score invariants, numerical correctness, prediction quality, recovery, and overhead separately so an aggressive but unsafe policy cannot win through throughput alone.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def build_manager_stress_cases(config, supported_features):
    ...

def run_bounded_manager_case(case, manager, oracle, budgets):
    ...

def E18(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Check authoritative-value preservation, no premature reuse, budget conservation, reservation cleanup, bounded telemetry, and required numerical results. For exact training extensions compare gradients and updates; for approximate storage compare predeclared quality tolerances. Report peak host and CUDA use, throughput, stalls, retries, and rejected work. Include a case with no eligible victim and one with unavoidable streaming. Do not score an unsafe forced completion as better than a correct infeasibility report.

Part 7 - Engineering Interpretation and Philosophy

The stress suite defines where the manager's guarantees hold. Explain which invariants survived interaction and which features require narrower support. Keep minimal counterexamples to universal policy claims and convert confirmed bugs into focused invariant checks. A trustworthy runtime can remain useful when it refuses unsupported migration, preserves required derivatives, and reports that the workload exceeds its controlled resources.

19.50.20 Exercise 19

Exercise Title: Build and Benchmark a Complete PyTorch Dynamic Memory Manager That Controls Tensor Residency, Prefetch, Eviction, Precision, Recomputation, and Recovery Across VRAM and RAM

Part 1 - Exercise Contract and Why It Matters

Integrate a complete manager within an explicitly supported application domain, combining registry, accounting, admission, residency, transfer scheduling, eviction, prefetch, optional precision, recomputation, recovery, and telemetry. Deliver a reviewable architecture and benchmark report. Complete means the supported components obey shared contracts; it does not mean transparent control of every tensor or a finished production allocator hidden inside a high-level trainer.

Part 2 - System and Memory Mental Model

The transition engine enforces correctness, the ledger enforces controlled commitments, and policy plugins optimize eligible choices. Managed handles preserve logical identity across physical replicas, while derivatives, precision, mutation, and side effects restrict permissible actions. External allocations and backend workspace remain scoped uncertainty. Optional NVMe persistence requires its own file protocol. No component may silently bypass reservations, release in-use storage, or change required mathematical work to satisfy a capacity target.

Part 3 - Quantitative Prediction Before Execution

Predict full workload peaks, transferred bytes, demand stalls, and runtime for an unmanaged baseline, a simple valid policy, and the integrated manager. Include initialization, calibration, planning, telemetry, and recovery costs in clearly declared scopes. Define success using correct values and derivatives, resource admission, and measured useful throughput. Avoid claiming optimality from one heuristic or treating a successful small workload as proof of universal OOM prevention.

Part 4 - Experimental Design and Instrumentation

Integrate one feature at a time behind common schemas and invariant checks. Use known cases for development and held-out traces for evaluation. Benchmark matched inputs, precision contracts, derivatives, seeds, budgets, and output retention. Keep clean and diagnostic replays separate. Bound speculative work, retry attempts, report size, and optional file storage. Export CPU-only evidence with run IDs and coverage. Verify shutdown drains or safely retains all pending work before releasing owned resources.

The final report needs workload and derivative contracts, resource scopes, registry coverage, budget versions, raw counters, policy decisions, transition outcomes, numerical checks, timing boundaries, and unresolved assumptions. Show exact lossless results separately from precision-degraded outcomes. Compare matched initial states and output-retention policies, including cold and warmed behavior. Report rejected actions and infeasible requests instead of removing them from throughput denominators without explanation. State whether setup, transfers, calibration, recovery, and telemetry are included. Claim only the best validated configuration within the tested range, and identify the smallest remaining experiment needed to resolve a material uncertainty.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch
```

```
@dataclass
class ManagerReport:
    contracts: object
    decisions: tuple
    invariant_checks: tuple
    numerical_checks: tuple
    resource_measurements: tuple
    limitations: tuple

def execute_managed_workload(workload, manager, contracts):
    ...

def benchmark_manager(config, policies, reference):
    ...

def E19(config):
    # Integrate policies only through admission and safe transitions.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require an evidence chain from every claimed saving to a policy decision, eligibility proof, actual transition, and validated result. Report exact and approximate outcomes separately. Compare current and peak allocated bytes, reserved capacity, managed commitments, host footprint, transfers, latency, throughput, and recovery outcomes. Keep not-supported, not-measured, rejected, and infeasible cases visible. A low ledger total is not successful memory management when live aliases, lost values, or incorrect gradients remain.

Part 7 - Engineering Interpretation and Philosophy

The final manager should make memory behavior explicit enough to reason about and flexible enough to optimize within known limits. Explain remaining uncertainty about unmanaged state, allocator eligibility, future reuse, and unsupported side effects. The strongest result is a validated runtime with bounded overhead, conserved commitments, safe asynchronous lifetimes, and honest failure modes. Correct refusal is part of its functionality, not a defect to conceal behind an aggressive policy.

DMM49 ==========

19.51 DMM49

Chapter Title: Custom Offload Engine and Final Runtime
Specialization: Dynamic Memory Management and CUDA Systems

19.51.1 Exercise 00

Exercise Title: Design the Architecture of a Complete PyTorch Offload Engine Across VRAM, RAM, and NVMe with Explicit Runtime States and Memory Budgets

Part 1 - Exercise Contract and Why It Matters

Design the final runtime as an application-level execution engine spanning CUDA storage, host memory, and optional NVMe-backed records. Deliver component contracts, state transitions, resource ledgers, supported workloads, and failure boundaries. Complete means coherent integration within that declared domain; it does not mean automatic interception of every PyTorch allocation or transparent migration of arbitrary raw tensors held by external code.

Part 2 - System and Memory Mental Model

Separate logical values, physical replicas, value versions, allocation generations, and reader or writer leases. Parameters, activations, KV blocks, and optimizer state have different mutation and derivative obligations. CPU I/O completion, CUDA copy completion, and compute completion are distinct events. A valid replica can exist on several tiers simultaneously, but a pending destination must not be published as ready or counted as released source capacity.

Part 3 - Quantitative Prediction Before Execution

Predict occupancy across one layer's read, staging, promotion, compute, and retirement. Include source-destination overlap, pinned buffers, activations, KV growth, workspace allowances, and pending reservations. Define budgets per tier and a headroom assumption for unmanaged state. Construct a trace where parameter payload fits but the full layer execution cannot be admitted. Explain why storage capacity alone does not predict feasible throughput when sustained transfer demand exceeds link service capacity.

Part 4 - Experimental Design and Instrumentation

Begin with immutable inference weights and deterministic stage execution. Specify registry, tier stores, planner, admission engine, transfer pipeline, policy modules, telemetry, and recovery interfaces. Add training only through explicit derivative-aware adapters. Use a deterministic event simulator for invariants, then bounded hardware replays. Give every action prepare, submit, complete, commit, and abort semantics. Keep exact lossless execution separate from optional approximate precision or context-retention policies.

Define the initial support matrix explicitly: immutable dense inference state, managed access leases, known stage boundaries, and private versioned storage. Record extensions for training, dynamic graphs, compiled execution, capture, distributed collectives, and direct storage as separate capabilities. A Python registry cannot rewrite every raw alias when a value moves. Choose whether consumers borrow tensors only within leases or invoke managed operations. State who owns teardown and how pending I/O and CUDA work are drained. Preserve an exact baseline contract so optional approximation cannot become an invisible fallback when resource pressure increases.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class RuntimeContract:
    workload_scope: object
    derivative_targets: tuple
    precision_contract: object
```

```
    tier_budgets: object
    side_effect_rules: object

def design_offload_runtime(contract):
    ...

def E00(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require counterexamples to immediate copy completion, automatic parameter reference repair, and guaranteed success after admission. Preserve hardware, versions, allocator backend, shapes, seeds, precision, optimizer, and initial state identifiers. Keep prediction, observation, and interpretation separate. Record byte budgets, timing scopes, outcomes, and failure phases. A missing GPU or NVMe path produces explicit not-measured or unsupported fields, never invented end-to-end benchmark numbers.

Part 7 - Engineering Interpretation and Philosophy

The runtime should make every residency decision traceable to a correctness contract and a resource commitment. Explain which limits are enforced and which uncertainty belongs to unmanaged allocations, future reuse, or backend behavior. Autonomous scheduling is valuable only when it preserves required values and derivatives, refuses unsupported operations, and reports infeasibility instead of concealing it through changed mathematics.

19.51.2 Exercise 01

Exercise Title: Build a Unified Model-State Registry for Parameters, Buffers, Activations, KV Cache Blocks, Gradients, and Optimizer State

Part 1 - Exercise Contract and Why It Matters

Build one registry schema for parameters, buffers, activations, KV blocks, gradients, and optimizer state while preserving their different semantics. Deliver role-specific ownership, mutation, version, lifetime, and access rules. Unified means a shared identity and resource framework, not one interchangeable policy for every state category. The registry must preserve tied parameters and shared storage without accidentally duplicating logical ownership or losing required update relationships.

Part 2 - System and Memory Mental Model

A module state_dict is not a complete runtime inventory: activations, some buffers, caches, gradients, and live optimizer internals require explicit adapters. Optimizers hold parameter references, and tied module names can refer to the same parameter. Graph saves or raw aliases can retain backing storage beyond a registry's apparent owner count. State role determines whether a replica is immutable, dirty, recomputable, shared, or required by a future derivative.

Part 3 - Quantitative Prediction Before Execution

Predict registry records for tied embeddings, a mutable running buffer, a saved activation view, a shared prefix block, a gradient, and Adam moments. Count unique backing capacity separately from named entries. Include a parameter update that invalidates older replicas and

a KV copy-on-write transition that separates shared ownership. Explain which state can be recreated from a recipe and which requires preserved current values.

Part 4 - Experimental Design and Instrumentation

Register supported roles through explicit adapters with logical IDs, generations, value versions, derivative contracts, and lease rules. Keep strong owning references separate from descriptive CPU metadata. Test duplicate names, shared storage, stale handles, and optimizer reference consistency. Preserve nonpersistent buffers even when absent from a saved state dictionary if they are managed at runtime. Reject unsupported escaped aliases or mutation paths rather than claiming comprehensive process-wide discovery.

Map tied names to one intended parameter identity and preserve optimizer state relationships separately from serialized key names. Include a nonpersistent buffer, a view sharing a parameter's backing, and an optimizer moment created only after its first update. Record logical role and physical storage links independently. A detached activation can still share storage with a saved value, so absence of history is not proof of independent ownership. If the registry holds strong references, identify that deliberate ownership and retire it explicitly. Export descriptive metadata without keeping another live copy of every managed value in telemetry.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class ModelStateRecord:
    logical_id: object
    role: str
    aliases: tuple
    value_version: int
    replicas: tuple
    ownership_rules: object
    derivative_rules: object

def register_model_state(model, optimizer, runtime_adapters):
    ...

def E01(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate shared parameters retain one intended update relationship and that optimizer targets still match the active model. Test live gradients and moments after a supported update, checking replica invalidation. Do not infer all owners from sys.getrefcount or assume a weak wrapper's death means CUDA reads completed. Compare registry-enabled and clean runs for diagnostic self-retention. Report unsupported state categories and missing coverage rather than assigning them zero capacity.

Part 7 - Engineering Interpretation and Philosophy

Role-aware identity allows one scheduler to coordinate many memory consumers without erasing their obligations. Explain why the same transfer mechanism can serve several roles while eviction eligibility differs. The registry should preserve both sharing and semantic boundaries so later policy choices cannot discard a gradient, stale a parameter replica, or free a prefix still referenced by another sequence.

19.51.3 Exercise 02

Exercise Title: Calculate the Complete Memory Footprint of Every Managed Object and Construct a Global VRAM-RAM-NVMe Residency Map

Part 1 - Exercise Contract and Why It Matters

Construct a complete residency map for every object inside the registered supported domain. Deliver logical payload, unique backing capacity, representation metadata, per-tier replicas, pending commitments, and host staging charges. Complete is scoped to observed managed objects; it does not imply that shapes alone determine all CUDA use, resident host pages, filesystem allocation, allocator cache, or opaque workspace.

Part 2 - System and Memory Mental Model

Several logical objects can share backing storage, while one value can occupy multiple tiers and precision representations. Pinned memory may be a subset of total host RAM, so overlapping categories must not be added twice. File bytes, memory mapping size, resident pages, and page cache differ. PyTorch allocated and reserved CUDA bytes have broader or different boundaries than registry totals and require timestamped reconciliation rather than forced equality.

Part 3 - Quantitative Prediction Before Execution

Predict unique per-tier capacity for tied weights, overlapping views, a CPU-GPU pair, and a compressed replica with scales and padding. Add temporary coexistence during conversion. Build a peak envelope from simultaneously live backing allocations instead of summing independent object peaks. Identify the unmanaged allowance and unsupported fields. Explain why total checkpoint size need not equal currently resident RAM or the parameter-only CUDA payload.

Part 4 - Experimental Design and Instrumentation

Validate a synthetic residency catalog before collecting live measurements. Scope identity by tier and allocation generation, preserving value versions separately. Record shape, stride, dtype, offset, supported backing capacity, file representation, and confidence. Compare manager deltas with completed CUDA samples and named host metrics. Keep pending reservations distinct from already materialized allocations. Narrow unsupported sparse, quantized, or external layouts instead of applying a dense formula blindly.

Use one definition of total managed host occupancy and treat pinned staging as a subset where appropriate. Avoid counting the same destination in both pending reservations and materialized capacity after allocation. File-backed mappings require separate virtual, resident, and storage-size fields. A small view can retain or serialize a large backing extent depending on its representation; measure the actual supported format. Preserve zero-element and unknown-capacity cases in synthetic tests. Compare current and peak counters at matched boundaries,

and state which unmanaged allowance covers workspace, context, or external library use without claiming exact whole-process visibility.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def build_residency_map(registry, pending_operations, accounting_scope):
    ...

def reconcile_resource_views(residency_map, allocator_samples, coverage):
    ...

def E02(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require exact totals for supported synthetic cases, including zero-element views with retained backing and repeated references to one parameter. Propagate unknown capacity rather than substituting zero. Reconcile differences as hypotheses about scope, rounding, timestamps, or unmanaged state before assigning a cause. Export raw counters beside derived totals. Report file and host accounting separately so an apparently small mmap process footprint cannot be misrepresented as zero system RAM use.

Part 7 - Engineering Interpretation and Philosophy

The map should support decisions rather than collapse every memory layer into one number. Explain which totals enforce admission and which provide context about the complete process. Scoped exact bookkeeping can coexist with uncertain external usage. Its strength is exposing overlap, sharing, and missing coverage so the scheduler avoids double counting or spending capacity that has already been promised.

19.51.4 Exercise 03

Exercise Title: Implement a Three-Tier Storage Interface That Moves PyTorch Tensors Transparently Between CUDA, CPU Memory, and NVMe

Part 1 - Exercise Contract and Why It Matters

Implement one logical storage interface for CUDA replicas, CPU replicas, and versioned NVMe records. Deliver controlled access, transfer, serialization, readiness, and failure contracts. Transparency applies at managed-handle boundaries: a caller requests a value and receives a valid supported replica. It does not mean Tensor.to accepts an NVMe device or that arbitrary existing tensor references automatically change location when a registry entry changes.

Part 2 - System and Memory Mental Model

The ordinary NVMe path is explicit file I/O through host storage and staging. File-read futures and CUDA events have different completion semantics. A readable pathname is insufficient evidence of a complete valid value; metadata, length, encoding, version, and integrity must

agree. Dirty authoritative state must remain recoverable until a new replica commits. Lossless file representations should preserve required values and sharing according to their declared format.

Part 3 - Quantitative Prediction Before Execution

Predict resource and event stages for NVMe-to-RAM-to-CUDA and its reverse. Include file buffers, pageable-to-pinned copies, GPU destinations, and source retention. Estimate actual stored bytes rather than assuming a tensor slice serializes only its logical elements. Construct short-read, stale-version, and incomplete-write cases, then identify the last valid source and which reservations remain charged during cleanup.

Part 4 - Experimental Design and Instrumentation

Begin with a documented dense format or supported checkpoint-backed source, restricting inputs to trusted generated artifacts. Define manifests and temporary versus committed records. Use explicit I/O workers or futures without labeling blocking reads intrinsically asynchronous. Publish readiness only after validated I/O and required CUDA completion. Restrict direct-storage extensions to separately verified capabilities. Preserve caller access through leases and reject unsupported raw alias mutation.

Document byte order, dtype, layout, shape, offsets, and representation version for raw records, or use the actual supported checkpoint interface without assuming its archive layout matches a plain tensor file. Validate read length and identity before publishing a replica. Keep temporary writes separate from committed manifests and retain the previous valid source until commit. If durable recovery is promised, implement the necessary persistence protocol and test interruption; otherwise state process-local guarantees. A canceled I/O future can leave work or buffers in use, so cleanup follows actual completion rather than the caller's interest in the result.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class TierAccess:
    handle: object
    expected_version: int
    requested_tier: str
    completion_token: object

def request_tier_access(store, request, reservations):
    ...

def commit_file_replica(store, operation, integrity_evidence):
    ...

def E03(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate byte-preserving round trips and supported tensor metadata. Test missing files, truncated data, corrupted metadata, stale records, and cancellation while I/O is pending. No failed write should invalidate the last authoritative source. Report whether file commit provides process-local recovery or durable persistence, implementing the corresponding protocol. Measure host footprint and complete path latency rather than reporting only the CUDA copy after unbounded file loading.

Part 7 - Engineering Interpretation and Philosophy

A common interface is valuable when it hides mechanical differences while preserving readiness and failure obligations. Explain which guarantees callers receive and which representation or storage limitations remain visible. The interface should make unsupported paths explicit and keep state versioning intact across tiers. Convenience cannot replace a correct file lifecycle or authorize stale data publication.

19.51.5 Exercise 04

Exercise Title: Build a Layer Execution Planner That Determines Parameter Load, Activation Residency, Compute, and Eviction Order Before Each Forward Pass

Part 1 - Exercise Contract and Why It Matters

Build a layer execution plan that orders weight readiness, activation production, compute, and retirement under resource budgets. Deliver an executable dependency schedule for a supported fixed graph and guarded plans for dynamic paths. Planning before forward does not mean all future lifetimes are known exactly; unknown shapes or branches require conservative admission, runtime guards, or replanning without violating already submitted work.

Part 2 - System and Memory Mental Model

Execution order differs from module registration order, especially with residuals, shared modules, and tied weights. Forward last use does not imply backward last use. A training plan needs the selected derivative schedule and parameter versions. Parameters, activations, KV state, and transfer buffers compete at each schedule cut. A valid plan specifies completion dependencies and peak overlap, not merely a sorted list of layer names.

Part 3 - Quantitative Prediction Before Execution

Draw a two-branch model and predict required weight versions, activation consumers, and per-tier occupancy across its schedule. Compare equal layer counts with unequal layer payloads. Identify a stage that cannot fit together with its required input, output, and workspace allowance even after all eligible eviction. Predict whether changing prefetch distance can solve the problem or whether the computation must be subdivided under another explicit contract.

Part 4 - Experimental Design and Instrumentation

Use a deterministic staged model with known shapes and named boundaries. Build a dependency DAG from application contracts or validated tracing, preserving unsupported dynamic behavior. Reserve materialization and workspace allowances before stage submission. Add residual and shared-layer controls. Validate plans in simulation with delayed completions, then compare actual execution against the same serial function. Replan only at supported boundaries with safe state and budget versions.

Separate graph dependencies from resource scheduling. A topologically valid layer order can still exceed memory because prefetch or retained activations overlap. Include a shared layer called twice and a residual value read after several later layers. For training, include forward production and backward last use in the planned lifetime rather than retiring state at forward return. If tracing misses dynamic control flow, mark that coverage and guard execution. An invalid plan must be rejected before it overwrites shared buffers or spends unreserved resources. Compare planned and actual resource envelopes with uncertainty about opaque workspace made explicit.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def plan_layer_execution(graph_contract, state_registry, cost_model):
    ...

def admit_stage(plan, stage_id, runtime_state, budgets):
    ...

def E04(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Check every consumer waits for the correct ready version and every release follows all required readers. Compare values and requested derivatives with a serial reference. Report plan prediction error separately from invalid execution. Test a changing input shape and reject an obsolete capacity estimate. A planner that cannot admit a stage must report the limiting resource; it must not force an allocation or silently move part of the mathematical work outside measurement.

Part 7 - Engineering Interpretation and Philosophy

Planning converts future execution into explicit resource commitments and dependencies. Explain what is statically known, what is guarded, and what must remain a runtime decision. The strongest plan exposes infeasible stages early and preserves dynamic correctness when predictions fail. Better ordering can reduce overlap, but it cannot make every indivisible operation fit an arbitrarily small device budget.

19.51.6 Exercise 05

Exercise Title: Implement On-Demand Parameter Loading That Materializes Only the Weights Required by the Current Execution Stage

Part 1 - Exercise Contract and Why It Matters

Load only the weight groups required by the admitted stage and preserve module semantics, ties, value versions, and ownership. Deliver lazy metadata, materialization, execution, and retirement contracts. Establish a correct inference path before training extensions. Do not

equate successful tensor copying with safe replacement of live Parameters whose references may already belong to an optimizer or a backward graph.

Part 2 - System and Memory Mental Model

Meta construction stores metadata without initialized values. to_empty allocates uninitialized storage, so checkpoint loading or initialization must complete before use. assign-style loading can replace parameter references and needs version-checked optimizer handling. A controlled stage can use explicit functional parameter mappings or stable supported execution buffers, but tied weights, mutable buffers, and gradient accumulation require deliberate treatment. Training cannot overwrite a weight slot still needed by backward.

Part 3 - Quantitative Prediction Before Execution

Predict memory for metadata, one current weight group, optional lookahead, input/output activations, and workspace. Include temporary checkpoint and staging storage rather than assuming direct materialization has no overlap. Compare tied weight groups with duplicated copies. Identify when a stage cannot fit and why on-demand loading does not solve an oversized indivisible matrix operation by itself. State which training consumers extend each weight version's lifetime.

Part 4 - Experimental Design and Instrumentation

Use a tiny staged module with known checkpoint values and a tied-parameter control. Choose one supported execution adapter and document its reference semantics. Validate keys, shapes, dtypes, and value versions before publication. Avoid .data replacement and untracked parameter reassignment. If using assign, create the optimizer after final assignment unless a verified reference-preserving mechanism is explicitly enabled. Keep training support separate until its backward ownership protocol is validated.

For functional execution, validate mapping keys, tied-weight behavior, parametrizations, and mutable buffers against the installed interface. For stable execution slots, enforce leases so copies cannot overwrite values required by unfinished backward or another stream. Do not present repeated load_state_dict assignment as a universal streaming training mechanism. Parameter replacement can leave old optimizer targets alive, and a slot reused for another layer can invalidate saved values. Include an inference-only implementation scope first and label training support unsupported until its reference and gradient ownership tests pass. Count any retained CPU master and mapped checkpoint source in resource reports.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def materialize_stage_weights(stage, source, adapter, reservation):
    ...

def execute_with_weight_version(stage, inputs, weight_lease):
    ...

def retire_stage_weight_version(stage, completion_evidence):
    ...
```

```
def E05(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Compare stage outputs and tied relationships with the original model. Check no meta or uninitialized value reaches computation. Test missing keys, wrong shapes, stale sources, and optimizer-reference mismatches. For supported training, compare gradients and updates across several steps. A copied buffer must not be reused merely because forward submitted; its final required reader must complete. Report actual peak, including temporary loading overhead, and preserve unsupported adapters honestly.

Part 7 - Engineering Interpretation and Philosophy

On-demand weights reduce residency by shortening supported storage lifetimes, not by erasing model state obligations. Explain how the adapter preserves the model's identity and required values while physical copies change. A small correct inference implementation is a sound foundation; adding training requires proving backward and optimizer relationships rather than assuming forward equivalence covers them.

19.51.7 Exercise 06

Exercise Title: Build a Lookahead Prefetch Planner That Loads Future Layers from NVMe to RAM and RAM to VRAM Before Their Execution Deadline

Part 1 - Exercise Contract and Why It Matters

Build a two-hop lookahead planner from NVMe to RAM and RAM to VRAM with separate deadlines, reservations, and completion evidence. Deliver demand fallback, speculative cancellation, and useful-prefetch measurements. The planner should reduce waiting when resources permit, while respecting host staging and device budgets. It must not spend the same future capacity for multiple pending layers or assume every predicted branch will execute.

Part 2 - System and Memory Mental Model

An I/O future completing only makes a host representation ready under its format contract. A later CUDA copy still needs destination admission, source lifetime, and an event. Earlier prefetch increases residency and can evict useful state. NVMe service, page cache, CPU copying, pinning, and PCIe may be bottlenecks at different stages. More lookahead cannot hide a sustained rate deficit or remove mandatory dependency waits.

Part 3 - Quantitative Prediction Before Execution

Predict latest start times for both hops from consumer deadlines and measured service costs with uncertainty margins. Include queueing and competing transfers. Construct a case where weights arrive in RAM early but miss the GPU deadline, and another where speculative reading consumes the host budget needed by demand. Estimate useful ready bytes, late bytes, unused speculation, and peak commitments for each lookahead horizon.

Part 4 - Experimental Design and Instrumentation

Begin with a fixed trace and measured size-dependent costs. Reserve host, pinned, and CUDA capacity through one ledger before submission. Tag operations with expected value

versions and consumer stages. Bound queue depth and prioritize mandatory demand over speculation. Add a conditional path and compare predictions with actual demand. Collect cold and warm file-access results separately; a warm page-cache read is not equivalent to a physical NVMe read.

Use independent queues and timestamps for file preparation and CUDA promotion. Host data readiness must include any required decoding, packing, or pinning before an H2D event can represent completion. Measure size-specific costs, including small-read overhead and cache state, rather than applying one peak bandwidth to all layers. Prevent speculative reads from consuming every host slot needed by mandatory demand or writeback. If predictions become stale, stop future submission and settle already started work safely. Count useful, late, stale, and unused prefetch separately; only the correct ready version before demand is a successful hit.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def plan_two_hop_prefetch(stage_trace, costs, registry, budgets):
    ...

def dispatch_prefetch_plan(plan, io_workers, cuda_streams, ledger):
    ...

def E06(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Count a hit only when the correct version is ready at the required tier before demand. Distinguish late prefetch, unused work, canceled submission, and in-flight work that still needs cleanup. Compare end-to-end stalls, traffic, host peak, CUDA peak, and runtime with demand-only loading. Preserve cases where shorter lookahead wins. Do not infer a successful pipeline from low host submission latency while the eventual consumer waits for unfinished I/O.

Part 7 - Engineering Interpretation and Philosophy

Lookahead trades capacity and speculation for less demand waiting. Explain which hop limits the tested workload and which horizon remains useful after contention and budgets are included. A robust planner can reduce or disable prefetch when pressure rises, preserving safe demand loading. Its benefit must survive the complete storage-to-consumer path rather than only one isolated transfer benchmark.

19.51.8 Exercise 07

Exercise Title: Implement Asynchronous Eviction That Demotes Completed Layers and Tensors Without Blocking Active GPU Computation

Part 1 - Exercise Contract and Why It Matters

Demote eligible completed values without blocking unrelated active computation. Deliver an asynchronous eviction state machine with read completion, dirty writeback, publication, and cleanup rules. Distinguish dropping a redundant clean device replica from copying authoritative modified state to another tier. The target is safe overlap, not a promise that every eviction has zero host cost or no effect on bandwidth available to compute.

Part 2 - System and Memory Mental Model

A layer's forward completion does not necessarily end its parameter or activation lifetime in training. A last-use lease must reflect all required backward and asynchronous readers. Clean immutable weights already valid on CPU or NVMe can often drop their GPU replica without D2H writeback. Dirty state requires preserving the latest version before release. Pending copies retain both source and destination obligations even if a caller cancels the requested eviction.

Part 3 - Quantitative Prediction Before Execution

Predict occupancy and dependencies for clean weight retirement, dirty optimizer state demotion, and an activation with a late reader. Estimate actual recoverable unique GPU capacity after aliases and pending uses. Include host admission and writeback cost. Construct a case where an apparently completed layer cannot yet evict its weights, and another where needless D2H of immutable weights wastes bandwidth without improving recoverability.

Part 4 - Experimental Design and Instrumentation

Use explicit eviction proposals revalidated by the transition engine. Check expected version, active leases, valid alternate replicas, and all required events. Reserve destination and staging before copying dirty data. Submit on supported side streams, then commit only when completion makes the target valid. Keep locks independent of device waits and test cancellation, stale proposals, and late completion using deterministic simulation before CUDA replays.

Classify each candidate as redundant clean copy, dirty recoverable state, recomputable state, or ineligible required state. Evicting immutable weights with a valid source should not automatically trigger D2H. Dirty buffers or optimizer state require versioned writeback and future-use protection. Include a shared allocation whose logical handle retires while another handle still uses it. Track final reader completion across every relevant stream, using allocator lifetime handling separately from readiness dependencies. A successful copy does not authorize source release if another consumer still needs the same generation. Report promised relief and actually released capacity in different fields.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def classify_eviction_path(handle, replica_state, required_consumers):
    ...

def submit_safe_eviction(proposal, ledger, streams, io_workers):
    ...

def commit_eviction(operation, completion_evidence):
    ...
```

```
def E07(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Compare values and required derivatives with serial execution. Test two reader streams, shared backing, dirty invalid replicas, and failure before publication. Confirm canceled in-flight work retains safe cleanup ownership. Measure recoverable bytes, writeback volume, demand stalls, and overlap. No premature source release is acceptable even if ordinary tests happen to produce correct values. A policy with no eligible victim must wait or reject admission rather than force eviction.

Part 7 - Engineering Interpretation and Philosophy

Eviction is a correctness-preserving replica lifecycle, not merely deletion of a Python name. Explain which state can be dropped, which must be written, and which remains needed. Good scheduling avoids needless writeback and overlaps legal transfers while preserving all future consumers. Honest eligibility limits are essential when a device budget is smaller than the unavoidable live working set.

19.51.9 Exercise 08

Exercise Title: Build a Multi-Stage Pipeline That Overlaps NVMe Reads, CPU Staging, H2D Transfers, GPU Compute, D2H Evictions, and Buffer Reuse

Part 1 - Exercise Contract and Why It Matters

Integrate file reads, host preparation, H2D, compute, D2H, and buffer reuse into one bounded multi-stage pipeline. Deliver a causal schedule, resource leases, backpressure, and completed throughput measurements. Concurrency must be visible in actual completion evidence rather than inferred from multiple threads or streams. Blocking I/O can run in a worker, but that does not make it a CUDA operation or guarantee independent storage service.

Part 2 - System and Memory Mental Model

Each stage has a queue, service capacity, readiness condition, and storage lifetime. File-read completion, pinning, copy completion, compute completion, and writeback commit are different boundaries. Shared PCIe, host memory, or storage bandwidth can couple nominally independent stages. Backpressure prevents upstream production from filling all RAM while a slower stage drains. Ready data must retain its correct version and slot generation until the consumer's final read completes.

Part 3 - Quantitative Prediction Before Execution

Draw the critical path for a small repeated-layer trace. Estimate steady throughput from stage service demands and shared-resource constraints rather than adding all isolated rates. Predict queue growth when file production outpaces GPU consumption and when evictions contend with promotions. Include buffer capacities and transition overlap. State which stages can overlap under the actual hardware and which dependencies necessarily serialize.

Part 4 - Experimental Design and Instrumentation

Implement bounded queues and completion tokens in a simulator first. Map CPU I/O to workers or futures and GPU copies to explicit streams and events. Reserve downstream capacity before launching work that would otherwise become unbounded. Use consistent lock ordering and avoid completion handlers waiting on locks held by blocked submitters. Compare the complete pipeline with a serial reference using identical traces and output-retention policy.

Specify maximum queue depth and buffer capacity for each stage, not only a thread count. A worker can be blocked on storage or on downstream capacity, and these waits have different causes. Keep host I/O completion tokens distinct from CUDA events. Use backpressure that does not deadlock by holding a lock needed by the draining stage. Test pipeline fill, steady state, final drain, and cancellation during all three. Include source and destination overlap in the ledger. If direct GPU storage is unavailable, retain the explicit host-staged path and avoid implying ordinary file reads bypass CPU memory.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def build_pipeline_dependencies(plan, resource_pools):
    ...

def advance_pipeline(runtime, io_completions, cuda_completions):
    ...

def E08(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate values at every handoff and after repeated buffer reuse. Test delayed I/O, delayed compute, empty queues, full queues, and shutdown with pending work. Record queue waits, service times, resource peaks, and end-to-end throughput. A short H2D submission call is not evidence that its latency was hidden. Check telemetry does not synchronize every stage. Preserve cases where concurrency increases contention and slows the whole pipeline.

Part 7 - Engineering Interpretation and Philosophy

A pipeline is effective when it moves waiting off the critical path without turning capacity into unbounded queues. Explain the actual bottleneck and how backpressure preserves progress. Overlap is a property of completed execution under specific dependencies and hardware, not a feature implied by an asynchronous API name. The manager should expose both useful concurrency and its resource cost.

19.51.10 Exercise 09

Exercise Title: Add Double and Triple Buffering and Determine the Minimum Buffer Depth Required to Hide Storage and PCIe Latency

Part 1 - Exercise Contract and Why It Matters

Compare single, double, and triple buffering and determine the smallest tested depth meeting a declared stall target. Deliver capacity, latency, throughput, and slot-lifetime evidence for the same pipeline. The minimum is conditional on workload, service variation, resources, and target; no fixed depth can guarantee hiding every storage or PCIe delay, and buffering cannot fix sustained insufficient bandwidth.

Part 2 - System and Memory Mental Model

Additional slots let one stage prepare future data while another consumes current data, but increase early residency and pending commitments. Variable sizes, tail I/O latency, and shared links can make average service time a poor predictor. Slots may belong to pageable, pinned, or GPU pools with different constraints. Reuse is legal only after transfer and final consumer completion, not after the producer enqueues its next request.

Part 3 - Quantitative Prediction Before Execution

For a regular schedule, estimate the required lookahead from preparation latency and consumer spacing, stating overlap assumptions. Then add variability and a slower sustained upstream stage to show where that estimate fails. Predict memory for each depth using actual capacities, including larger variable-shape slots. Define a quantitative stall target before sweeping and allow the result that no admitted tested depth meets it.

Part 4 - Experimental Design and Instrumentation

Hold model, trace, precision, budgets, and consumer work constant while sweeping buffer depth. Preallocate or grow slots through admission with versioned leases. Separate cold fill and final drain from steady-state timing, while also reporting the complete run. Use repeatable synthetic delay traces and bounded real I/O. Test depths beyond three only when admitted and needed to establish the tested range, rather than claiming triple buffering is universally sufficient.

Give each slot a pool identity, generation, capacity, valid extent, and current lease set. Triple buffering at one tier does not require identical depth at every other tier; measure the limiting stages and shared budgets. Use a regular deterministic schedule to validate the initial latency estimate, then add recorded tail delays. A sustained slower storage stage cannot be hidden indefinitely by finite buffers. Define minimum as the smallest admitted tested depth meeting the stated target on the tested trace. Include fill and drain costs in complete runtime even when steady-state throughput is reported separately.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def evaluate_buffer_depth(depth, pipeline_contract, budgets, trace):
    ...

def select_tested_depth(results, stall_target, resource_limits):
    ...

def E09(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate no slot is overwritten before all readers finish. Report peak host and CUDA commitments, useful overlap, tail stalls, and throughput. Compare average and worst observed delays without presenting a finite run as a hard latency guarantee. State whether a depth was rejected by capacity or failed the performance target. Include a case where extra buffering lowers throughput by provoking eviction or link contention.

Part 7 - Engineering Interpretation and Philosophy

Buffering spends memory to tolerate latency and variation. Explain the smallest validated depth for the tested target and why further slots stop helping. A robust runtime should shrink speculative depth under pressure without invalidating active slots. The useful conclusion is a bounded policy supported by measured resource and timing behavior, not an unconditional rule that more buffers mean more overlap.

19.51.11 Exercise 10

Exercise Title: Integrate Activation Checkpointing and CPU Activation Offloading and Dynamically Choose Recompute, Keep, or Offload per Activation

Part 1 - Exercise Contract and Why It Matters

Integrate keep, checkpoint recompute, and CPU activation offload into the final scheduler. Deliver derivative-aware eligibility, versioned region inputs, budgeted restoration, and matched training comparisons. These actions preserve different representations of the same required information. They must be connected to supported autograd mechanisms rather than implemented by replacing arbitrary saved tensors or detaching required history behind the registry.

Part 2 - System and Memory Mental Model

Checkpointing saves a recipe and required inputs; offloading preserves values elsewhere; keeping retains device state. Recompute must use the correct parameter versions, RNG behavior, and mutable state. Offload needs source lifetime, host capacity, and restoration dependencies. Supported saved-tensor hooks and non-reentrant checkpointing have version-sensitive composition rules. A scalar metadata record is not a valid replacement for a tensor that backward needs to read.

Part 3 - Quantitative Prediction Before Execution

Predict unique peak reduction, checkpoint-input residency, host payload, transfer volume, and recomputation cost for each eligible region. Account for shared backing and another owner that prevents actual release. Compare whole-step overlap rather than summing independent savings. Include a region whose recomputation causes a backward workspace peak and a region whose offload cannot fit the available host budget.

Part 4 - Experimental Design and Instrumentation

Use deterministic pure regions and explicit use_reentrant=False for the primary checkpoint path. Integrate validated save_on_cpu or pack/unpack adapters with required value, dtype, and device restoration. Collect diagnostics in separate replays when hook composition is not proven. Hold model state, precision, optimizer, gradient clearing, and data fixed. Add stochastic, stateful, and higher-order extensions only through separate tests and support contracts.

Store the parameter and state version required by every recompute recipe. If an optimizer update occurs before the recipe's final backward use, the plan must preserve the old version

or reject that schedule. Do not assume RNG preservation covers all custom randomness or mutable side effects. Offloaded saved values need valid restoration for each retrieval, including repeated backward where supported. Keep hook inputs immutable and avoid payload reference cycles. Logging hooks and checkpoint policies may override or bypass one another depending on version and feature; verify active behavior or separate diagnostic collection from the actual policy benchmark.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
from torch.utils.checkpoint import checkpoint

def classify_activation_actions(region, derivative_contract, versions):
    ...

def dispatch_activation_action(region, action, runtime, reservations):
    ...

def E10(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Compare outputs, required gradients, and several updates with ordinary backpropagation. Check active hooks actually perform the intended offload. Report host peak, CUDA peak, latency, restored bytes, and observer overhead. A lower forward endpoint does not establish a lower training peak. Reject stale parameter recipes, unsupported mutable effects, or hidden detach. Preserve cases where only keeping is eligible or no eligible policy meets the workload budget.

Part 7 - Engineering Interpretation and Philosophy

Activation strategy should follow correctness eligibility before cost optimization. Explain which information survives, where it lives, and when it must be available again. Dynamic pressure can change the preferred strategy only at supported boundaries. The runtime should expose the trade between recomputation, transport, and residency while preserving the original derivative task and its actual whole-step measurement scope.

19.51.12 Exercise 11

Exercise Title: Integrate Paged KV Cache Management with Keep, Quantize, CPU-Offload, and Evict Decisions for Long-Context Inference

Part 1 - Exercise Contract and Why It Matters

Integrate paged KV storage with keep, quantize, CPU offload, and eviction choices for long-context inference. Deliver block versions, reference counts, copy-on-write rules, precision contracts, and attention validation. Distinguish lossless removal of a redundant device replica from permanently deleting logical context. The latter changes full-attention semantics unless

a separately declared retention algorithm permits it; quantization likewise needs an explicit approximation contract.

Part 2 - System and Memory Mental Model

Logical token blocks map to physical key/value storage, possibly shared across sequences. Active references prevent releasing shared backing prematurely. A CPU-offloaded block remains part of the logical cache and must be restored for its required attention use. Unreferenced prefix-cache entries can be reclaimed under their cache contract. Prefix identity depends on more than raw token text, including model version and relevant positional or adapter configuration.

Part 3 - Quantitative Prediction Before Execution

Predict bytes per physical block from declared layer scope, KV heads, head dimension, token capacity, dtype, and metadata. Count shared blocks once and unused tail capacity separately. Compare lossless offload with quantization and logical deletion, specifying value and task-error requirements. Include a partially filled shared tail that requires copy-on-write when one sequence extends, and predict transient capacity before publication.

Part 4 - Experimental Design and Instrumentation

Use a small explicit attention implementation and a contiguous full-cache reference. Preserve original positions, masks, token ordering, and model configuration through paged retrieval. Start with lossless keep/offload, then evaluate approximate actions separately. Reserve copy-on-write destination and transfer capacity before mutation. Test shared-prefix release, stale block versions, active-reader protection, and whole-context restoration under an admitted budget.

Record a prefix-sharing identity that includes the model and relevant execution configuration, original positions, token prefix, and adapter context. Hashing a token block without its prefix context is insufficient because its KV values can depend on earlier tokens. Shared-tail writes require copy-on-write and transient destination admission. A block's physical release must respect all references and readers. Distinguish deleting an unreferenced cached prefix from dropping active logical context. If quantized blocks are shared, include representation metadata in compatibility checks. Report exact lossless and approximate attention outcomes separately, with positions and masks preserved for the retained context algorithm.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def classify_kv_actions(block, sequence_contracts, references, budgets):
    ...

def schedule_kv_action(proposal, block_table, runtime):
    ...

def E11(config):
    # Logical context deletion is distinct from lossless replica eviction.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Check attention outputs and logits against the lossless reference at declared tolerances, and evaluate approximate policies against separate quality targets. Report physical bytes, tail waste, metadata, transfers, and decode stalls. A lower attention score alone does not prove a block is unused by full attention. Include refcount and copy-on-write invariants. Preserve failures to fit an indivisible working set rather than claiming paging or eviction guarantees arbitrary context capacity.

Part 7 - Engineering Interpretation and Philosophy

The KV policy manages both residency and, when authorized, representation or retained information. Explain which decisions preserve the same context and which alter it. A unified runtime must keep those semantics visible so a capacity gain cannot be mistaken for exact inference. The strongest policy preserves shared-block safety and reports measured quality and performance for every optional approximation.

19.51.13 Exercise 12

Exercise Title: Build a Cost Model That Estimates Compute Time, H2D Time, D2H Time, NVMe I/O, Recomputation Cost, and Expected Reuse Before Every Memory Decision

Part 1 - Exercise Contract and Why It Matters

Build a measured cost model for compute, directional copies, file I/O, recomputation, reuse, and queueing before policy decisions. Deliver calibration, uncertainty, held-out prediction error, and safe fallback. The model ranks eligible actions; it cannot authorize an unsafe transition because an estimate looks cheap. Before every decision means using valid cached or updated estimates, not necessarily running an expensive benchmark inside every scheduling step.

Part 2 - System and Memory Mental Model

Costs vary with size, dtype, layout, kernel choice, cache state, device, queue depth, and concurrent traffic. Isolated NVMe, PCIe, and compute timings are not independent under contention. Reuse distance is a forecast, not a promise. Host launch latency differs from completed device service. A file read served from page cache should not be treated as a measurement of sustained physical storage throughput.

Part 3 - Quantitative Prediction Before Execution

Fit a simple transfer estimate with fixed overhead and size-dependent service, stating the range where it is useful:

$$\widehat{t}(x) = \widehat{\alpha} + x/\widehat{\beta}.$$

Compare the estimate with piecewise measurements and contention corrections. Predict keep, offload, and recompute costs including future reloads and capacity overlap. Construct a case where uncertain reuse reverses the preferred action and require the scheduler to preserve that uncertainty.

Part 4 - Experimental Design and Instrumentation

Calibrate bounded representative sizes with warmup, repeated trials, completed events, and explicit timing scopes. Record cold and warm file access separately. Keep some traces out of calibration and evaluate prediction error on them. Invalidate cached estimates after relevant backend, shape, or contention changes. Bound calibration time and metadata. Use conservative

resource admission independently of performance estimates so prediction error causes slower execution rather than incorrect memory commitments.

Attach calibration provenance to every estimate: device, software, backend, shape range, dtype, layout, cache state, and contention scenario. Maintain separate directional transfer models and include fixed per-request overhead. A nominal storage rate does not capture page-cache service or decoding work. Reuse forecasts need confidence and an invalidation rule when execution branches change. For overlapping actions, predict critical-path effects instead of subtracting their isolated times twice. Evaluate held-out traces and bound calibration cost. If estimates are stale or unsupported, choose an eligible conservative fallback rather than treating absent data as zero cost.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class CostEstimate:
    seconds: object
    uncertainty: object
    resource_overlap: object
    calibration_scope: object

def estimate_action_cost(action, workload_state, calibrated_models):
    ...

def E12(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report predicted and measured times, bias, tail error, sample count, and coverage. Do not claim transfer bandwidth from host enqueue duration. Compare serial and overlapped complete paths to reveal contention or double-counted savings. Preserve unsupported estimates and extrapolation warnings in the report. Validate that an inaccurate cost forecast cannot bypass generations, leases, reservations, or derivative requirements. Separate diagnostic usefulness from predictive accuracy.

Part 7 - Engineering Interpretation and Philosophy

A cost model should make trade-offs explicit and falsifiable. Explain where its assumptions hold and how uncertainty affects decisions. Simple measured models can be useful when they are cheap, scoped, and checked against actual outcomes. The strongest scheduler can revise a poor estimate while preserving correctness, rather than pretending every action's future cost is known precisely.

19.51.14 Exercise 13

Exercise Title: Implement a Unified Runtime Scheduler That Jointly Chooses Placement, Prefetch Distance, Eviction Victims, Precision, and Activation Strategy

Part 1 - Exercise Contract and Why It Matters

Implement a scheduler that jointly considers placement, prefetch distance, eviction, precision, and activation strategy under common resource and semantic constraints. Deliver eligible action sets, joint reservations, decisions, and independent validation. Autonomous means executing supported choices within declared contracts; it does not authorize silent loss of gradients, quality, context, or update consistency when a cheaper schedule is unavailable.

Part 2 - System and Memory Mental Model

Policies interact: longer prefetch consumes capacity needed for activations, compression adds conversion workspace, offload consumes host staging, and recompute needs preserved inputs and parameter versions. Local greedy choices can conflict or oscillate. The transition engine and ledger enforce legality while cost models rank alternatives. Some budgets admit no feasible schedule, especially when one indivisible stage exceeds unavoidable working capacity.

Part 3 - Quantitative Prediction Before Execution

Predict a joint decision for a small graph under three budgets and two accuracy contracts. Include competing host and CUDA demands and a candidate that appears cheap in isolation but blocks the next required stage. Define an objective such as completed latency under hard correctness and capacity constraints. State the heuristic's search scope and avoid implying a globally optimal solution from a limited candidate set.

Part 4 - Experimental Design and Instrumentation

Use one state snapshot with expected generations and budget versions. Generate only eligible actions, estimate joint overlap, reserve resources atomically, and dispatch through safe transition interfaces. Revalidate stale plans before commit. Begin with a deterministic fixed trace, then conditional demands. Separate policy proposal, admission, submission, and observed completion in telemetry. Keep exact mode as the baseline and require opt-in contracts for approximation or altered context retention.

Define precedence among semantic eligibility, hard admission, performance objectives, and heuristic tie-breaking. Precision or context changes require explicit opt-in quality budgets before they enter the action set. Use one coordinated ledger so two individually attractive proposals cannot spend the same future capacity. Plans carry expected state and budget versions; stale ones must be revalidated. Test a case where local largest-first eviction conflicts with near-term reuse and another where long lookahead blocks a required activation. Record candidate rejection reasons and search bounds, without claiming a finite heuristic discovers a global optimum.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def generate_eligible_runtime_actions(state, contracts):
```

```
    ...

def choose_joint_plan(actions, costs, ledger, objective):
    ...

def dispatch_joint_plan(plan, runtime, expected_versions):
    ...

def E13(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate numerical results and all invariant checks after each supported plan. Compare against simple valid policies and a manually reviewed small oracle, not just a favorable unmanaged baseline. Report decision overhead, search budget, rejected plans, and prediction error. Test stale state and concurrent admissions. A lower planned cost without a completed valid execution is not a successful optimization. Preserve infeasible workloads as explicit outcomes.

Part 7 - Engineering Interpretation and Philosophy

Joint scheduling is useful because memory techniques compete for the same resources. Explain which interactions the heuristic captures and which remain outside support. Correctness and admission should remain stable as policy ranking evolves. The runtime should provide a reasoned decision or refusal with enough evidence to review it, rather than hide assumptions behind an automatic mode label.

19.51.15 Exercise 14

Exercise Title: Detect Thrashing, Pipeline Bubbles, Bandwidth Saturation, Fragmentation, and Over-Prefetching and Automatically Reconfigure the Runtime

Part 1 - Exercise Contract and Why It Matters

Diagnose thrashing, pipeline bubbles, link saturation, candidate fragmentation, and over-prefetching, then test bounded reconfiguration. Deliver separate signals, counterexamples, and intervention outcomes. Do not collapse all performance or allocation failures into one memory-pressure score. Fragmentation requires request-aware allocator evidence; a large gap between reserved and allocated bytes alone is not a causal diagnosis.

Part 2 - System and Memory Mental Model

Similar symptoms can have different causes: late demand can reflect slow I/O, wrong-path speculation, queueing, or insufficient lead time. Repeated migration can be necessary for a working set larger than capacity. Native inactive split metrics describe one allocator phenomenon and may be unsupported under another backend. A small cached candidate does not guarantee failure because the allocator can grow, while hidden eligibility restrictions can invalidate apparent fit.

Part 3 - Quantitative Prediction Before Execution

Predict diagnostic signals for paired traces with equal traffic but different stalls and equal reserved-minus-allocated gaps but different request outcomes. Define windows, thresholds, and

supported allocator scope. Forecast which reconfiguration should improve each trace and what observation would falsify that interpretation. Preserve uncertainty when several causes fit the same data, rather than automatically changing every policy at once.

Part 4 - Experimental Design and Instrumentation

Use controlled traces for demand-only streaming, speculative churn, compute imbalance, and bounded allocator patterns. Collect detailed diagnostics in separate replays because probing changes state. Apply one reversible eligible control at a time, such as shorter lookahead or queue depth. Use hysteresis and cooldown with hard admission limits unchanged. Recheck actual outcomes under the original workload contract and keep all unsupported backend fields explicit.

Include a high inactive-share cache with healthy reuse and a small cached block followed by successful allocator growth. These counterexamples prevent simple reserved gaps or block scores from becoming universal failure predictors. Scope native split-block statistics to supported allocator backends. Distinguish observed request outcomes from candidate eligibility guesses. Reconfiguration experiments require fresh or carefully matched states because probing itself changes allocation and queue history. Preserve a stable working policy as a control. If no causal distinction is available, report competing explanations and the next discriminating measurement rather than automatically calling the symptom fragmentation.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def diagnose_runtime_bottlenecks(telemetry, allocator_evidence, coverage):
    ...

def propose_bounded_reconfiguration(findings, contracts, cooldown_state):
    ...

def E14(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Evaluate alert accuracy, causal localization, and successful intervention separately. Include high reserved cache with healthy reuse, unavoidable streaming, and a prefetch policy that already performs well. Report false positives, missed cases, runtime, and migration changes. A positive correlation between a counter and stalls is not proof of fragmentation or policy causality. Do not promise that tuning can make every budget or interconnect configuration feasible.

Part 7 - Engineering Interpretation and Philosophy

Reconfiguration should follow a discriminating experiment, not a convenient label. Explain which evidence supports each diagnosis and which alternatives remain. A conservative runtime can preserve a stable policy when evidence is weak, or report that the bottleneck is structural. The useful result is a validated improvement with its causal limits and resource costs visible.

19.51.16 Exercise 15

Exercise Title: Add OOM Recovery That Rolls Back an Unsafe Execution Step, Tightens Memory Policy, and Retries Without Restarting the Model

Part 1 - Exercise Contract and Why It Matters

Add bounded same-process recovery for retryable execution regions, with rollback, policy tightening, and exact-state validation. Deliver failure injection, cleanup, retry limits, and explicit nonrecoverable cases. The engine cannot generally undo arbitrary optimizer updates, external side effects, or a poisoned CUDA context. Recovery must be scoped to a transaction whose state can actually be preserved or restored.

Part 2 - System and Memory Mental Model

Allocation can fail after partial materialization, RNG consumption, or buffer mutation. Failed-frame tracebacks can retain tensors, so retry outside the exception scope and avoid keeping traceback objects in logs. Source replicas and pending transfer buffers remain charged until safe cleanup. empty_cache releases unused cache rather than restoring model state. Distributed collectives and partially applied updates require coordinated protocols beyond a local exception handler.

Part 3 - Quantitative Prediction Before Execution

Predict transaction state before failure, after abort, and before retry, including reservations and retained rollback data. Set a finite retry budget and a measurable progress condition, such as reduced optional lookahead. Compare an exact policy retry with an explicitly authorized approximate fallback. Construct a stateful operation that must be refused and a pure inference stage that can be retried safely without duplicating externally visible work.

Part 4 - Experimental Design and Instrumentation

Start with deterministic stage-local inference and injected allocation failures at known preparation points. Catch the supported CUDA OOM type specifically, record scalar metadata, exit the exception scope, clean failed temporaries, and re-admit the modified plan. Include RNG and mutable buffers in the transaction when relevant, charging snapshot cost. Do not retry fatal CUDA errors or uncoordinated rank-local collective failures. Test a bounded real OOM separately when hardware allows.

Write a rollback table for every supported stage: original state, possible mutations, live partial allocations, pending operations, and restoration action. Charge any snapshot storage to the correct budget. Do not retry while failed exception frames or diagnostic containers still retain temporaries. Inject failure before allocation, after preparation, and before publication to validate cleanup. Include a partially updated optimizer or external side-effect case that is rejected. On multiple ranks, do not perform unilateral retry around collective operations. A policy change that alters batch semantics, precision, or context must use its own declared workload contract instead of masquerading as an exact unchanged retry.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch
```

```
def prepare_retry_transaction(stage, runtime, side_effect_contract):
    ...

def abort_stage_attempt(transaction, failure_metadata, runtime):
    ...

def choose_retry_plan(transaction, policies, attempts_remaining):
    ...

def E15(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Compare successful results with a failure-free oracle, including update count, RNG, buffers, values, and required gradients for supported training. Verify ledger conservation and no premature reuse of in-flight storage. Distinguish injected failure, admission rejection, actual OOM, and fatal errors. Report all retries and recovery time in the chosen end-to-end scope. Preserve failed recovery rather than declaring that every OOM is solvable without restarting.

Part 7 - Engineering Interpretation and Philosophy

Recovery is a supported transaction protocol with a clear stopping rule. Explain why the selected retry preserves the workload and what policy actually changed. A trustworthy engine can recover eligible failures and stop when state integrity or progress is uncertain. Avoiding a process restart is useful only when the completed result has the same declared mathematical and side-effect contract.

19.51.17 Exercise 16

Exercise Title: Extend the Offload Engine to Multiple GPUs and Dynamically Balance Resident Model State Across Unequal VRAM Capacities

Part 1 - Exercise Contract and Why It Matters

Extend supported residency and stage placement to multiple GPUs with unequal available budgets. Deliver device-local ledgers, topology-aware transfer plans, safe migration boundaries, and per-device measurements. More devices add capacity but also movement and synchronization costs. The extension must not assume arbitrary live parameters can move while their optimizer references, backward consumers, process ownership, or distributed layout remain unchanged.

Part 2 - System and Memory Mental Model

Each device has its own allocator, streams, events, workspace, and competing allocations. Peer capability and actual topology constrain direct copies; CPU-staged paths require additional host capacity. Event synchronization and timing have device-specific rules, so do not combine cross-device elapsed measurements as one clock without support. Migration can temporarily occupy both devices and must preserve logical value versions, aliases, and required consumers.

Part 3 - Quantitative Prediction Before Execution

Predict normalized pressure using each GPU's effective budget and compare it with raw used bytes. Include source-destination overlap, activation boundary traffic, and unequal compute speeds. Construct a migration that relieves the busiest device but increases critical-path transfer time. State which safe-boundary moves are eligible and which require another supported parallelism strategy rather than a placement change.

Part 4 - Experimental Design and Instrumentation

Start with single-process stage inference across two devices and explicit activation transfers. Measure topology and supported peer paths rather than assuming direct movement. Reserve destination and any host staging before migration, then retire source replicas only after final readers. Compare static and adaptive placements with identical workload traces. Add training or distributed execution only with validated optimizer, gradient, and collective ownership adapters.

Record process and device ownership, peer capability, actual transfer route, and transient reservations. Single-process tensor movement does not reconfigure FSDP or DTensor layouts. Test safe migration and an unsupported gradient-dependent move. Normalize pressure by effective budgets and include later communication costs when judging capacity relief.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def plan_multi_gpu_placement(stages, device_ledgers, topology, costs):
    ...

def migrate_at_safe_boundary(proposal, runtime, reservations):
    ...

def E16(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Validate outputs and supported derivatives against a matched serial reference. Record current and peak allocated/reserved bytes per device, normalized pressure, migration volume, and complete latency. Test destination admission failure and stale migration proposals. Include a topology where host staging is required or mark it untested. Do not claim correctness from balanced memory alone, and do not treat extra GPUs as a guarantee that every indivisible stage or communication buffer fits.

Part 7 - Engineering Interpretation and Philosophy

Balancing should optimize useful execution under heterogeneous constraints, not merely make counters equal. Explain when migration's capacity relief justifies its transient overlap and communication cost. The runtime should retain a stable placement when moving data would worsen the objective, and report unsupported distributed semantics explicitly rather than presenting single-process movement as a universal parallel training engine.

19.51.18 Exercise 17

Exercise Title: Run a Transformer-Style Model Whose Parameter Footprint Is Several Times Larger Than Available VRAM and Verify Exact Output Equivalence Against a Fully Resident Reference

Part 1 - Exercise Contract and Why It Matters

Execute a Transformer-style model whose parameter payload exceeds the engine's admitted GPU budget by a declared multiple, and validate lossless output behavior against a legitimate fully resident reference. Separate artificial budget and physical-capacity experiments. The exact-equivalence title is a validation target under controlled arithmetic, not permission to invent a resident reference that cannot fit or to label cross-device tolerance agreement as guaranteed bitwise identity.

Part 2 - System and Memory Mental Model

Lossless transfers preserve stored values, but compute results can differ with backend, precision, operation order, shape, or nondeterministic algorithms. A same-device resident reference can be run separately when the model fits physical VRAM but exceeds a smaller engine quota. A model exceeding physical VRAM needs a larger reference device or another explicitly identified oracle. CPU execution can support tolerance-based numerical checks, but does not automatically establish exact CUDA equality.

Part 3 - Quantitative Prediction Before Execution

Predict unique parameter payload, the ratio to the declared managed budget, largest stage working set, activations, cache, workspace, and staging. Choose a quota permitting at least one valid stage and a payload several times that quota. State actual physical capacity separately. Predict full resident and streamed peaks, and verify the reference can fit its own declared hardware before planning either measurement.

Part 4 - Experimental Design and Instrumentation

Use a deterministic small Transformer-style model first, disabling approximation and logical context deletion. Preserve weight values, tied relationships, masks, positions, dtype, shapes, arithmetic settings, and output retention. Run resident and streamed same-device replays separately for the artificial- quota primary test. Compare exact transferred representations, intermediate values, and final outputs. Scale physical oversize only when a feasible reference path and adequate host or file capacity are available.

Record reference hardware, arithmetic settings, checkpoint identity, and feasibility. Label the artificial-quota ratio separately from physical capacity. A physically oversized model needs a feasible larger-device resident reference for that claim; otherwise mark it unavailable. CPU and layerwise checks retain their actual numerical criteria. Preserve exact mismatches and localize them with intermediate comparisons.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def plan_reference_feasibility(model_metadata, hardware, engine_quota):
```

```
    ...

def compare_resident_and_streamed(config, reference_plan, contracts):
    ...

def E17(config):
    # Label artificial quota, physical capacity, and equality criteria.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Report bitwise equality and tolerance-based error as separate outcomes using predeclared criteria. Preserve any exact mismatch and investigate arithmetic settings before making an equivalence claim. A CPU oracle or unavailable larger GPU must be labeled honestly. Export the full configuration, payload- to-budget ratio, peak bytes, throughput, and reference provenance. A layerwise oracle is useful but is not itself a fully resident reference; do not rename it to satisfy the title.

Part 7 - Engineering Interpretation and Philosophy

The strongest demonstration proves the actual claim being made. Explain whether the result establishes lossless transport, same-device exact output, or broader numerical agreement. Distinguishing quota oversize from physical oversize makes the experiment reproducible and auditable. The runtime's capacity gain remains valuable when its hardware and numerical limits are reported accurately, without overstating what one available machine can verify.

19.51.19 Exercise 18

Exercise Title: Stress-Test the Complete Runtime with Long Contexts, Variable Batch Sizes, Artificial VRAM Limits, RAM Pressure, Slow NVMe, Concurrent Streams, and Injected OOM Failures

Part 1 - Exercise Contract and Why It Matters

Stress-test the integrated engine with changing contexts and batches, constrained VRAM and RAM, delayed I/O, concurrent streams, and injected failures. Deliver correctness, invariant, resource, performance, and recovery matrices. Use bounded artificial budgets and fault injection before optional physical pressure. A stress test should reveal unsupported interactions and infeasible workloads, not force a misleading successful result or exhaust the host to obtain a dramatic trace.

Part 2 - System and Memory Mental Model

Interactions couple resources and semantics: larger batches increase activation and transfer footprints, long contexts increase KV state, slow I/O expands queues, and retries retain rollback state. A pinned pool can become the limiting resource even with spare CUDA bytes. Context-retention or precision changes are not lossless fallback unless their separate contracts permit them. Mutable state and distributed operations constrain where recovery or replanning is safe.

Part 3 - Quantitative Prediction Before Execution

Predict expected outcomes for each case: admitted completion, bounded wait, policy rejection, unsupported feature, retryable failure, or irreducible capacity deficit. Include equal

logical payload with different backing sharing, and equal migration traffic with different consumer stalls. Estimate worst supported transient overlap, file space, and log size. Define success using correct values and valid outcomes rather than completion rate alone.

Part 4 - Experimental Design and Instrumentation

Build deterministic event traces for stale generations, short reads, delayed copies, duplicate callbacks, cancellation, dirty state, and admission races. Add small real CUDA and private-file tests one feature at a time, then selected interactions. Keep fresh initial state, fixed traces, deadlines, retry limits, and bounded telemetry. Separate injected I/O delay from measured hardware I/O behavior. Evaluate held-out cases not used for tuning policy thresholds.

Test short reads, stale manifests, delayed events, reservation races, and duplicate completions before hardware-dependent faults. Injected I/O delay is not measured NVMe behavior. Include independent host and GPU admission limits and invalid approximate fallback rejection. Score correctness, safe refusal, recovery, and performance separately, preserving minimal invariant reproducers.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each `...` marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
import torch

def build_integrated_stress_cases(config, feature_support):
    ...

def execute_bounded_stress_case(case, runtime, oracle, resource_limits):
    ...

def E18(config):
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Check authoritative-value preservation, no premature reuse, reservation conservation, reference correctness, bounded queues, safe cleanup, and numerical contracts. Report exact and approximate cases separately. Include false diagnostic alerts, failed savings, recovery attempts, and actual stopping reasons. A correct refusal under insufficient capacity is not a runtime bug, while a fast result with missing gradients or deleted context under an exact contract is a failure. Preserve unavailable hardware tests as not measured.

Part 7 - Engineering Interpretation and Philosophy

The suite defines the evidence-supported operating envelope. Explain which features remain valid together and which need narrower contracts. Minimal counterexamples should improve eligibility or invariants instead of being removed from the report. The goal is a runtime that remains understandable under pressure, preserves required work, and fails safely when its supported resources or semantics are exceeded.

19.51.20 Exercise 19

Exercise Title: Build and Benchmark a Production-Style PyTorch Dynamic Memory Runtime That Executes Models Larger Than VRAM Through Autonomous VRAM-RAM-NVMe Placement, Prefetching, Eviction, Compression, Recomputation, Scheduling, and Recovery

Part 1 - Exercise Contract and Why It Matters

Integrate a production-style educational runtime spanning registry, tier storage, planning, scheduling, admission, prefetch, eviction, optional compression, activation strategy, recovery, and telemetry. Deliver an auditable implementation plan and benchmark protocol for models beyond an admitted GPU budget. The scaffold remains learner work. Production-style describes disciplined interfaces, invariants, observability, and failure handling; it is not a claim of production certification or universal support for arbitrary PyTorch applications.

Part 2 - System and Memory Mental Model

The engine controls supported handles and execution boundaries, not all external allocations. State identity, validity, derivatives, precision, and side effects constrain policy choices. Resource ledgers include pending commitments and transient replicas. CPU and file workers coordinate with CUDA through explicit readiness and lifetime dependencies. Approximate compression or logical KV deletion must stay visibly separate from exact residency and recomputation policies. Multi-GPU and training support require their own validated adapters.

Part 3 - Quantitative Prediction Before Execution

Predict total throughput, latency, resource peaks, transferred bytes, stalls, and policy overhead for resident feasible, serial offload, simple valid, and adaptive configurations. State reference feasibility and separate artificial quota from physical-capacity claims. Include calibration, planning, conversion, recovery, and telemetry in declared timing scopes. Define success as preserved workload semantics within admitted resource constraints, with honest rejection or infeasibility where no supported plan exists.

Part 4 - Experimental Design and Instrumentation

Integrate components through one transition engine and reservation ledger. Develop on known cases and evaluate held-out traces. Match weights, shapes, precision contracts, derivative targets, output retention, seeds, and initial state. Separate clean performance replays from diagnostic collection. Bound queue depth, retries, speculative traffic, file capacity, and host report size. Validate startup, normal execution, cancellation, and shutdown with pending work, preserving valid authoritative sources throughout cleanup.

Report contracts, adapters, reference provenance, budgets, raw measurements, decisions, transitions, and numerical checks. Separate prediction, observation, and interpretation. State cold and warmed timing inclusions, quality contracts, and output retention. Preserve rejected work in throughput denominators and claim only the best validated configuration within the tested range.

Part 5 - PyTorch/CUDA Implementation

Complete the scaffold below. Each ... marks required learner work; the implementation is intentionally incomplete and is not a finished solution.

```
from dataclasses import dataclass
import torch

@dataclass
class RuntimeReport:
    supported_contracts: object
    reference_provenance: object
    policy_decisions: tuple
    invariant_results: tuple
    numerical_results: tuple
    measurements: tuple
    limitations: tuple

def run_offload_engine(workload, engine, contracts, evidence_sink):
    ...

def benchmark_final_runtime(config, variants, reference_plan):
    ...

def E19(config):
    # Every autonomous action must pass semantic and resource admission.
    ...
```

Part 6 - Measurement, Validation, and Failure Analysis

Require a traceable chain from each claimed improvement to eligibility, a decision, a committed transition, and a validated outcome. Report lossless and approximate results separately, including quality, gradients, updates, and context semantics when applicable. Export raw memory scopes beside derived pressure, and completed timing beside throughput. Keep rejected, unsupported, not-measured, and failed cases visible. A small internal ledger is not success if external aliases retain CUDA storage or required values were lost.

Part 7 - Engineering Interpretation and Philosophy

The final runtime should make complex memory behavior explicit enough to review and flexible enough to optimize within demonstrated limits. Explain remaining uncertainty about future reuse, opaque allocations, backend eligibility, and unsupported side effects. The strongest result combines correct execution, conserved resource commitments, safe asynchronous lifetimes, measured useful overlap, and bounded recovery. Autonomous operation is justified by those contracts and evidence, not by an unqualified promise to run every model on any amount of VRAM.

20

Conclusion of the Volume of Books

20.1 What next

The real goal is to delve deeper into artificial intelligence, machine learning, deep learning, and perhaps even self-programming in the future. Explore the philosophy of building mathematical models and move away from the notion that artificial intelligence will become uncontrollable. Research seeks solutions and development, not media attention. Focus on your research in artificial intelligence and create value in this world. What you've learned in these books is just a drop in the ocean. Artificial intelligence is like a black hole; once you get close to it, you'll be fascinated. And the strange thing is, you won't know if it has an end. Build your future with your own hands. Place yourself among the researchers. You are responsible for developing artificial intelligence. Be a role model for others.

20.2 Does this volume of books have an end

Yes, it has an end, and that's when the author dies. What I do is compile what I've learned throughout my life. Emirati society is my responsibility; I try to build a bright future for it, placing it among the ranks of the developed world. I believe we must expand our knowledge and compete with the rest of the world. The future must be bright, and bright, for the modern United Arab Emirates.

20.3 There are those who are smarter than you in artificial intelligence.

There are people who are smarter or more skilled than you, not because they learned more or started earlier, but because they learned something in a specific angle that you have not explored yet. There are also those who have studied artificial intelligence for many more years than you, and there is no shame in being late; the real problem begins when you say, "I know everything." The moment you feel that you have learned everything in AI, that is dangerous, because there is no such thing as learning everything in artificial intelligence. It is a vast world where learning never stops. Even when you specialize in one area, you will not say, "I know it all," but instead you will say, "There is a problem with no solution yet, so I must find one,"

and that is when you begin writing scientific papers and research. That is science: solving problems.